cadence®

Innovus Text Command Reference

Product Version 23.10
January 2024

© 2024 Cadence Design Systems, Inc.
Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Open SystemC, Open SystemC Initiative, OSCI, SystemC, and SystemC Initiative are trademarks or registered trademarks of Open SystemC Initiative, Inc. in the United States and other countries and are used with permission.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 800.862.4522.

All other trademarks are the property of their respective holders.

Restricted Permission: This publication is protected by copyright law and international treaties and contains trade secrets and proprietary information owned by Cadence. Unauthorized reproduction or distribution of this publication, or any portion of it, may result in civil and criminal penalties. Except as specified in this permission statement, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. Unless otherwise agreed to by Cadence in writing, this statement grants Cadence customers permission to print one (1) hard copy of this publication subject to the following conditions:

1. The publication may be used only in accordance with a written agreement between Cadence and its customer.
2. The publication may not be modified in any way.
3. Any authorized copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement.
4. The information contained in this document cannot be used in the development of like products or software, whether for internal or external use, and shall not be used for the benefit of any other party, whether or not for consideration.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information. Cadence is committed to using respectful language in our code and communications. We are also active in the removal and replacement of inappropriate language from existing content. This product documentation may however contain material that is no longer considered appropriate but still reflects long-standing industry terminology. Such content will be addressed at a time when the related software can be updated without end-user impact.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

Contents

About This Manual 66
Audience 66
Conventions Used in This Manual 66
Related Documents 68
Innovus Product Documentation 68
Innovus Stylus Common UI Product Documentation 69
1 70
Advanced Timing Tcl Scripting Commands 70
Overview 71
all_* Commands 71
Examples 71
get_* Commands 71
Examples 72
property Commands 72
Examples 72
collection Commands 73
Examples 73
add_to_collection 73
all_clocks 75
all_connected 76
all_fanin 77
all_fanout 78
all_inputs 80
all_instances 81
all_outputs 82
all_registers 83
append_to_collection 85
compare_collections 87
copy_collection 88
define_property 89
filter_collection 91
foreach_in_collection 93

get_activity 95
get_arcs 98
get_cells 99
get_clocks 100
get_designs 102
get_generated_clocks 103
get_lib_arcs 104
get_lib_cells 105
get_lib_pg_pins 107
get_lib_pins 108
get_libs 110
get_nets 111
get_object_name 112
get_path_groups 113
get_pg_nets 114
get_pg_pins 115
get_pins 116
get_ports 118
get_power 119
get_property 121
185
index_collection 188
list_property 189
query_objects 190
range_collection 191
remove_from_collection 193
report_property 194
sizeof_collection 195
sort_collection 196
2 198
ART Based Flow Commands 198
clearActiveLogicView 198
createActiveLogicView 200
getActiveLogicViewMode 202
getHierMode 205
getModuleView 207

setActiveLogicViewMode 209
setHierMode 212
setModuleView 215
3 217
Basic Database Access Tcl Commands and Global Variables 217
Basic Database Access Tcl Commands 217
add_cell_obs 218
add_line_end_track 221
add_via 222
add_via_definition 223
applyGlobalNets 227
clearGlobalNets 228
compress_special 229
create_virtual_shape_on_pg_wires 230
db_pt 231
db_rect 232
dbEdge 234
dbGet 235
dbQuery 243
dbSchema 246
dbSet 250
dbShape 251
dbTransform 254
dbu2uu 255
decompress_special 256
delete_cell_obs 257
deleteDanglingNet 258
free_memory_of_compressed_objects 259
get_via_pillars 259
getDbGetMode 260
globalNetConnect 262
report_metal_stack 266
report_net_wires 269
Node Information 269
Edge Information 269
report_preserves 271

report_via 274
reportDanglingNet 275
reportRC 277
reportWirePath 280
set_via_pillars 281
setDbGetMode 283
uniquify 286
uu2dbu 288
Basic Database Access Tcl Global Variables 289
restore_db_version 289
layerNameNoAbbreviation 289
mustjoinallports_is_one_pin 289
4 290
Bus Plan Commands 290
createBusGuide 290
deleteBusGuide 294
deselectBusGuide 297
resetBusGuideMultiColors 300
selectBusGuide 301
selectBusGuideSegment 304
setBusGuideMultiColors 306
update_bus_guide 308
5 312
Clock Tree Synthesis Commands 312
add_clock_tree_source_group_roots 314
assign_clock_tree_source_groups 318
calculate_ccopt_cannot_clone_reason 319
ccopt_add_exclusion_drivers 320
ccopt_design 322
ccopt_pro 325
check_ccopt_clock_tree_convergence 328
clock_opt_design 329
close_ctd_win 331
commit_ccopt_clock_tree_route_attributes 332
connect_clock_tree_mesh_drivers 333
convert_lib_clock_tree_latencies 334

create_ccopt_clock_spine 339
create_ccopt_clock_tree 342
create_ccopt_clock_tree_source_group 344
create_ccopt_clock_tree_spec 345
create_ccopt_flexible_htree 347
create_ccopt_generated_clock_tree 356
create_ccopt_macro_model_spec 359
create_ccopt_preferred_cell_stripe 360
create_ccopt_skew_group 361
create_route_type 365
cts_refine_clock_tree_placement 368
ctd_save_histogram 371
ctd_save_view 372
ctd_show_skew_group 373
ctd_trace 375
ctd_win 379
delete_ccopt_clock_spines 383
delete_ccopt_clock_tree_source_group 384
delete_ccopt_clock_tree_spec 385
delete_ccopt_clock_trees 386
delete_ccopt_flexible_htrees 387
delete_ccopt_preferred_cell_stripe 388
delete_ccopt_skew_groups 389
delete_clock_tree_repeaters 390
delete_route_type 391
get_ccopt_clock_spines 392
get_ccopt_clock_tree_capacitance 393
get_ccopt_clock_tree_cells 395
get_ccopt_clock_tree_nets 398
get_ccopt_clock_tree_sinks 400
get_ccopt_clock_tree_slew 402
get_ccopt_clock_tree_source_groups 404
get_ccopt_clock_trees 405
get_ccopt_dag_traversal 407
get_ccopt_delay_corner 409
get_ccopt_effective_max_capacitance 410
get_ccopt_flexible_htrees 413

get_ccopt_preferred_cell_stripe 414
get_ccopt_property 415
get_ccopt_skew_group_delay 419
get_ccopt_skew_group_path 424
get_ccopt_skew_groups 427
get_ctd_win_id 429
get_ctd_win_title 431
get_lib_clock_tree_path_delay 433
gui_zoom_ctd 436
merge_clock_cells 439
modify_ccopt_skew_group 442
preserve_ccopt_port 444
report_ccopt_cell_filtering_reasons 446
report_ccopt_cell_halo_violations 450
report_ccopt_clock_tree_convergence 453
report_ccopt_clock_tree_drv 454
report_ccopt_clock_tree_structure 456
report_ccopt_clock_trees 460
report_ccopt_pin_insertion_delays 465
report_ccopt_preserved_clock_tree_ports 468
report_ccopt_skew_groups 469
report_ccopt_worst_chain 474
reset_all_ccopt_preserved_clock_tree_ports 476
reset_ccopt_config 477
reset_ccopt_preserved_clock_tree_port 478
reset_ccopt_routing_state 479
restore_ccopt_config 480
route_ccopt_clock_tree_nets 481
route_ccopt_flexible_htrees 482
save_ccopt_config 484
set_ccopt_preserved_clock_tree_port 485
set_ccopt_property 486
set_ctd_win_title 489
show_ccopt_cell_name_info 491
synthesize_ccopt_flexible_htrees 493
unset_ccopt_property 497
update_ccopt_clock_tree 500

update_clock_latencies 501
update_clock_tree_source_latency 502
update_clock_tree_spec_annotations 504
6 508
Conformal Commands 508
runCLP 508
7 511
Delay Calculation Commands and Global Variables 511
Delay Calculation Commands 511
addDeCap 512
addDeCapCellCandidates 516
cleanupExcludeNet 517
clearDeCapCellCandidates 518
deleteDeCap 519
getDelayCalMode 520
reportDeCap 522
reportDeCapCellCandidates 523
reportDelayCalculation 525
saveExcludeNet 532
saveSignalStormConstraint 533
setDelayCalMode 534
translateSNDCSetupFile 547
writeSetLoad 548
Delay Calculation Global Variables 550
delaycal_default_net_delay 550
550
delaycal_default_net_load 552
delaycal_input_transition_delay 553
delaycal_rd_rnet_fraction_threshold 555
delaycal_support_min_max_pin_cap 556
delaycal_support_rise_fall_pin_cap 557
delaycal_support_wire_load_model 558
delaycal_use_default_delay_limit 559
8 562
Flip Chip Commands and Global Variables 562

Flip Chip Commands 562
addBumpConnectTargetConstraint 563
assignBump 566
assignPGBumps 571
assignSigToBump 577
changeBumpMaster 579
checkBondPadSpacing 581
checkBump 582
ciopLoadBumpColorMapFile 599
copy_bump 600
create_bump 602
createSignalPin 625
deleteAIoFiller 628
deleteBumpConnectTargetConstraint 629
deleteBumps 631
deselect_bump 633
edit_bump_name 636
editBumpConnectTargetConstraint 638
fcroute 639
findPinPortNumber 647
fixBondPad 652
get_physical_info 653
getBondPad 655
getFlipChipMode 656
ioInstOverlapCheck 658
placeBondPad 659
readFlipChipProperty 660
readIoUpdate 661
readPackage 662
reportProbePins 663
reportSpecialRoute 664
routePointToPoint 673
select_bump 676
setBumpFixed 680
setBumpPlacementStatus 681
setFlipChipMode 682
setProbePin 696

setSpecialRouteType 697
spaceBondPad 699
staggerBondPad 701
swapSignal 705
unassignBump 706
unfixBondPad 707
unfixBump 708
unsetProbePin 709
verifyFlipChipRoutingConstraints 710
viewBumpConnection 711
writeFlipChipProperty 715
Flip Chip Global Variables 716
flipchip_allow_routed_bump_edit 716
9 717
Floorplan Commands 717
add_ndr 721
add_shape 724
add_text 727
addAloFiller 728
addAIORow 730
addDummyBoundaryWires 733
addHaloToBlock 736
addInstToInstGroup 739
addIoFiller 740
addIoInstance 742
addIoRowFiller 745
addModuleToFPlan 748
addObjFPlanCutBox 749
addRoutingHalo 750
addSizeBlockage 753
adjustFPlanChannel 754
alignObject 756
changeFloorplan 759
changeIoConstraints 760
check_macro_place_constraint 762
check_ufc 763

checkFPlan 766
checkFPlanSpace 767
checkMacroLLOnTrack 768
convertFenceToLef 769
copyObject 770
create_bump_grid 771
create_pg_model_for_macro_place 772
create_relative_floorplan 773
createAbuttedFPlan 778
createDensityArea 784
createExclusiveGroups 786
createFence 788
createGuide 790
createInstGroup 792
createIoRow 794
createLogicHierarchy 797
createPGPin 799
createPhysicalPin 802
createPlaceBlockage 804
createRegion 810
createRouteBlk 812
createRouteBlkAlongTrack 818
createRow 820
createSoftGuide 827
createSpecialDrcRegion 829
createSpecialMarkLayerShape 831
createStairwayBoundary 833
cutRectilinearInst 837
cutRow 840
delete_bump_grid 842
delete_relative_floorplan 843
deleteAllDensityAreas 844
deleteAllFPObjects 845
deleteAllInstGroups 846
deleteAllPowerPreroutes 847
deleteAllSignalPreroutes 848
deleteExclusiveGroups 849

deleteFPObject 850
deleteHaloFromBlock 851
deleteInstFromInstGroup 852
deleteInstGroup 853
deleteIoFiller 854
deleteIoInstance 855
deleteIoRowFiller 856
deleteNetWeight 857
deletePGPin 858
deletePlaceBlockage 859
deleteRouteBlk 860
deleteRoutingHalo 863
deleteRow 864
deleteSelectedFromFPlan 865
deleteSizeBlockage 866
deselectAll 867
deselectGroup 868
deselectInst 869
deselectInstByCellName 870
deselectInstOnNet 871
deselectIOPin 872
deselectNet 873
display_obj_connectivity 874
exportNdr 876
finishFloorplan 877
fix_boundary_overlaps 888
fix_floorplan 889
fixAllIos 891
flipModule 892
flipOrRotateObject 893
floorPlan 896
generate_fence 906
get_macro_place_constraint 909
get_snap_grid_info 912
get_trace_obj_connectivity_mode 913
getDrawView 914
getFinishFPlanMode 915

getFPlanMode 916
getIoFlowFlag 918
getNetWeight 919
getObjFPlanBoxList 920
getObjFPlanPolygon 921
getRailPrototypeMode 922
getResizeFPlanMode 923
initCoreRow 924
insert_physical_cell 925
legalizeFPlan 928
loadFPlan 929
loadIoFile 931
modify_ndr 932
move_obj 935
moveGroupPins 937
moveMacroInsideModule 939
pasteObject 940
placeAIO 941
placePadIO 943
placePIO 944
predict_floorplan 947
queryFPlanObject 948
refine_macro_place 949
refineMacro 950
report_narrow_channel 953
report_obj_connectivity 956
reportNetGroup 958
reportSeedConnection 959
reportSelect 960
reportUnsnapBlocks 961
reset_macro_place_constraint 962
resizeFloorplan 964
save_global 966
saveFPlan 967
saveIoFile 969
select_row 972
selectGroup 973

selectInst 974
selectInstByCellName 975
selectInstOnNet 976
selectIOPin 977
selectNet 978
selectPGPin 979
selectRouteBlk 980
set_macro_place_constraint 982
set_trace_obj_connectivity_mode 989
setBottomIoPadOrient 991
setDrawView 992
setFinishFPlanMode 993
setFixedBlockSize 996
setFlipping 997
setFPlanMode 998
setFPlanRowSpacingAndType 1009
setInstancePlacementStatus 1010
setInstGroupPhyHier 1011
setIoFlowFlag 1012
setIoRowMargin 1013
setObjFPlanBox 1014
setObjFPlanBoxList 1016
setObjFPlanPolygon 1018
setRailPrototypeMode 1019
setResizeFPlanMode 1020
setResizeLine 1022
setRouteBlkDefaultLayer 1024
setSelectedDensityArea 1026
setSelectedObstruct 1028
setSelectedRouteBlk 1029
setSelectedStripBoxShape 1031
setSelectedStripBoxState 1032
shiftOrigin 1033
snapFPlan 1037
snapFPlanIO 1039
spaceIoInst 1040
spaceObject 1041

specifyNetWeight 1043
stretchRows 1044
swapPins 1045
trace_obj_connectivity 1046
unfixAllIos 1048
unplaceAllBlocks 1049
unplaceAllGuides 1050
unplaceAllInsts 1051
unplaceGuide 1052
unplaceGuideConstraints 1053
unsetFixedBlockSize 1054
update_floorplan_for_macro_place 1055
write_macro_place_constraint 1056
writeFPlanScript 1057
add_interposer_route_block 1061
create_interposer_route_block 1062
10 1064
General Commands and Global Variables 1064
General Commands 1064
alias 1067
bindKey 1070
check_ccr 1073
check_syntax 1074
checkNetlist 1076
compare_release 1079
decrypt 1082
define_proc_arguments 1083
deleteDangling1b1Or0s 1087
deleteDanglingPort 1089
deselect_obj 1090
encMessage 1091
encrypt 1093
eval_common_ui 1094
find_global 1097
generateFFSetupFile 1101
get_message 1102

get_metric 1104
getAllLayers 1106
getBuildArch 1108
getCheckMode 1109
getCompressLevel 1112
getLayerPreference 1113
getPreference 1116
getVersion 1117
gui_get_legend 1118
gui_set_legend 1120
help 1123
is_common_ui_mode 1127
lminus 1128
man 1129
parse_proc_arguments 1130
placeCursor 1133
print_message 1134
Puts 1136
read_taf 1137
redirect 1138
rename_obj 1141
report_area 1146
report_command_mode 1152
report_disk 1154
report_hidden_usage 1155
report_message 1157
reportDanglingPort 1160
reportGateCount 1161
reportNetStat 1163
resume 1164
savePreference 1165
select_obj 1166
selectObjByProp 1167
set_global_always_keep 1169
set_message 1170
set_proc_verbose 1174
setCheckMode 1176

setCompressLevel 1181
setLayerPreference 1182
setLicenseCheck 1185
setLimitedAccessFeature 1188
setMessageLimit 1189
setPreference 1191
setTopCell 1208
setWindowPreference 1209
source 1211
summaryReport 1212
suppressMessage 1219
suspend 1222
time_info 1223
uiGetRecordObjByInfo 1225
unsetMessageLimit 1226
unsuppressMessage 1227
update_names 1229
viewLog 1238
vPuts 1239
win 1240
writeFlowTemplate 1241
General Global Variables 1242
auto_file_dir 1244
auto_file_prefix 1245
enable_legacy_metric 1246
enc_partial_cmd_argument_matching 1247
enc_print_full_message_summary 1248
enc_source_continue_on_error 1249
enc_source_echo_filename 1251
enc_source_verbose 1252
enc_source_verbose_cmd_print_length_limit 1253
enc_source_verbose_output 1254
enc_tcl_error_info 1256
enc_tcl_return_display_limit 1257
encEnableMetric 1259
gui_verbose 1260
highFreqInterposerFlow 1261

load_netlist_ignore_undefined_cell 1262
metric_advanced_url_endpoint 1263
metric_capture_3d_hotspots 1264
metric_capture_depth 1265
metric_capture_design_image 1266
metric_capture_design_image_blockages 1267
metric_capture_design_image_blockages_threshold 1268
metric_capture_design_image_power_intent 1269
metric_capture_design_image_route_drc 1270
metric_capture_max_drc_markers 1271
metric_capture_min_count 1272
metric_capture_overwrite 1273
metric_capture_pba_tns_histogram 1274
metric_capture_per_view 1275
metric_capture_reg2reg_metrics 1276
metric_capture_timing_analysis_mode 1277
metric_capture_timing_path_groups 1277
metric_capture_timing_paths 1277
metric_capture_tns_histogram 1278
metric_capture_tns_histogram_buckets 1279
metric_capture_tns_histogram_max_slack 1280
metric_capture_tns_histogram_paths 1281
metric_capture_vth_per_power_domain 1282
metric_category_default 1283
metric_current_run_id 1284
metric_enable 1285
metric_summary_metrics 1286
restore_db_directory 1286
restore_db_file_check 1286
restore_db_stop_at_design_in_memory 1286
restore_db_tool 1287
script_search_path 1288
soft_stack_size_limit 1288

11 1289

GUI Commands 1289

add_gui_marker 1293

add_gui_shape 1295
add_gui_text 1299
clearAllRulers 1303
createRuler 1304
createSnapshot 1306
createUserBundleNet 1308
db_browser 1311
dehighlight 1313
delete_gui_object 1315
deleteUserBundleNet 1316
deleteWorkspace 1318
deselectModule 1320
deselectPin 1321
display_temperature_result 1322
displayUserBundleNet 1322
dumpPictures 1326
dumpToGIF 1330
editCopy 1332
fit 1338
get_visible_bumps 1339
get_visible_netGroups 1340
get_visible_nets 1341
gui_attach_to_cursor 1342
gui_clear_trace_flightline 1344
gui_close_cell_view 1345
gui_deselect 1346
gui_dim_foreground 1348
gui_dump_picture 1349
gui_group_hinst 1353
gui_open_cell_view 1355
gui_report_trace_flightline 1356
gui_save_flightline 1359
gui_schematic_load_view 1361
gui_schematic_save_view 1362
gui_select 1363
gui_show_edge_number 1365
gui_trace_flightline 1367

gui_ungroup_hinst 1371
highlight 1373
highlight_pin 1377
highlight_pin_connection 1380
lineSelect 1384
list_gui_marker 1386
loadWorkspace 1387
open_schematic 1389
pan 1395
panCenter 1397
panPage 1398
read_temperature_map_file 1400
redo 1400
redraw 1403
register_gui_edit_callback 1404
remove_gui_marker 1406
report_gui_edit_callback 1407
resetMultiColorsHier 1410
saveColorPreference 1412
saveHInstColor 1413
saveWorkspace 1415
select_highlighted 1417
selectModule 1418
selectPin 1419
set_object_color 1420
set_power_rail_display 1423
set_power_rail_layers_nets 1435
set_visible_bumps 1441
set_visible_netGroups 1443
set_visible_nets 1444
setDefaultWorkspace 1445
setHInstColorId 1447
setMultiColorsHier 1449
setSelectedPtnCut 1451
setSelectedPtnFeedthrough 1452
setSelectedPtnPinBlk 1453
setSelectedPtnPinGuide 1454

setSelectedWireState 1456
setSelHInstColor 1457
ui_view_box 1459
uiAdd 1460
uiDelete 1465
uiFind 1467
uiGet 1469
uiSet 1475
uiSetTool 1481
undo 1484
unregister_gui_edit_callback 1487
viewLast 1489
viewNext 1490
viewSnapshot 1491
windowDeselect 1493
windowSelect 1494
windowToggleSelect 1496
zoomBox 1497
zoomIn 1498
zoomOut 1499
zoomSelected 1500
zoomTo 1502
12 1504
Hierarchical Design Commands 1504
saveModel 1505
13 1509
ILM and FlexILM Commands 1509
commit_module_model 1510
createInterfaceLogic 1513
flattenIlm 1517
getIlmMode 1518
getIlmType 1521
import_ilm_data 1523
read_ilm_eco_db 1526
reportIlmStatus 1528
setIlmMode 1529

setIlmType 1533
specifyIlm 1536
unflattenIlm 1538
unspecifyIlm 1540
update_partition 1541
write_ilm_eco_db 1543
14 1546
Import and Export Commands and Global Variables 1546
Import and Export Commands 1546
check_design 1547
checkDesign 1551
checkUnique 1556
createTrack 1557
defComp 1558
defIn 1562
defOut 1566
defOutBySection 1571
deleteModule 1574
deleteTrack 1575
disconnectDanglingPort 1576
dumpOutVias 1577
generateLef 1579
generateVias 1581
genPinText 1583
get_abstract_mode 1585
getCmdLogFileName 1587
getDesignMode 1588
getExportMode 1590
getGenerateViaMode 1592
getImportMode 1593
getLogFileName 1595
getStreamOutMode 1596
init_design 1598
loadDefFile 1601
loadLefFile 1602
read_name_mapping 1603

read_stream 1605
relink_db_files 1608
replaceLefMacro 1609
report_gates 1610
report_sequential 1612
restoreDesign 1613
run_abstract 1616
saveDesign 1617
saveNetlist 1623
set_abstract_mode 1628
set_cell_power_domain 1632
setDesignMode 1633
setExportMode 1638
setGenerateViaMode 1640
setImportMode 1642
setLibraryUnit 1645
setNet 1646
setStreamOutMode 1647
specify_lib 1653
streamOut 1654
trimDesign 1659
write_do_lec 1662
write_lef_abstract 1663
write_name_mapping 1669
Import and Export Global Variables 1671
cds_lib 1672
check_design_across_hierarchy 1673
check_design_report_async_pins 1674
check_design_report_scan_pins 1675
check_design_threshold_fanout 1676
cvd 1677
defHierChar 1678
defInCheckMaskShifts 1679
defInShapeBasedDefFile 1680
defOutCompressVia 1681
defOutLefNDR 1682
defOutLefVia 1683

defOutPolygonDieArea 1684
defOverlapWireReportFileName 1685
defStreamOutCheckUncolored 1686
enc_save_binary_timing_constraints 1687
enc_save_timing_constraints_always 1688
fp_vertical_row 1689
ignore_max_via_stack_rule 1690
init_abstract_view 1691
init_check_output_pin_constant 1692
init_cpf_file 1693
init_design_netlisttype 1694
init_design_settop 1695
init_design_uniquify 1696
init_gnd_net 1697
init_ignore_pgpin_polarity_check 1698
init_import_mode 1699
init_io_file 1700
init_layout_view 1701
init_lef_check_antenna 1702
init_lef_check_mask_shifts 1703
init_lef_file 1704
init_mmmc_file 1705
init_mmmc_version 1706
init_oa_default_rule 1707
init_oa_design_cell 1708
init_oa_design_lib 1709
init_oa_design_view 1710
init_oa_foundry_rule 1711
init_oa_ref_lib 1712
init_oa_search_lib 1713
init_oa_special_rule 1714
init_original_verilog_files 1715
init_pwr_net 1716
init_top_cell 1717
init_verilog 1718
init_verilog_tolerate_port_mismatch 1719
inn_save_lef_ignore_abstracts 1720

lefdefInputCheckColoredShape 1721
lefDefOutVersion 1722
lefExtend2DCellShapes 1723
lefExtendCellObsShapeUnderTrim 1724
lefIgnoreLayerNames 1725
merge_multi_port_to_single_port 1726
readWriteLefCheckUncoloredShapes 1727
save_cmd_file_limit 1728
write_lec_dft_constraints 1728
write_lec_directory_naming_style 1728
write_lec_files 1728
writeLefAbstractCustomerHeader 1729
15 1730
Integrated Hierarchical Database Commands 1730
check_module_model 1731
create_module_model 1733
define_add_on_module_model 1736
import_module_model 1738
pack_module_model_libs 1741
report_module_model 1742
restore_module_model 1744
set_module_model 1747
update_module_model 1751
write_module_model_context 1754
16 1758
Interactive ECO Commands and Global Variables 1758
Interactive ECO Commands 1758
addHierInst 1759
addInst 1761
addModulePort 1764
addNet 1765
attachDiode 1767
attachIOBuffer 1768
attachModulePort 1774
attachTerm 1775
bitblast_ports 1777

delete_feedthru_buffer 1779
deleteBNet 1780
deleteEmptyModule 1781
deleteInst 1782
deleteModulePort 1783
deleteNet 1784
deleteNotchFill 1785
deleteUninstantiatedModules 1786
detachModulePort 1787
detachTerm 1788
displayBufTree 1789
ecoAddRepeater 1790
ecoChangeCell 1796
ecoCompareNetlist 1798
ecoDefIn 1800
ecoDeleteRepeater 1803
ecoDesign 1805
ecoOaDesign 1808
ecoPlace 1810
ecoRemoveTiedInputs 1813
ecoSwapSpareCell 1814
endECO 1815
fillNotch 1816
fix_multi_drivers 1817
getEcoMode 1818
group 1820
initECO 1822
loadECO 1823
remove_assigns 1825
setEcoMode 1829
ungroup 1832
ECO Global Variables 1833
bit_blasted_port_style 1833
ecoDisableNetRenamingForFlatNetlist 1834
group_instance_suffix 1835
init_no_new_assigns 1836

17 1838
Low Power Commands and Global Variables 1838
Low Power Commands 1839
addPowerSwitch 1840
Column Type Insertion Parameters 1843
Power Switch Prototyping Parameters 1844
Ring Type Insertion Parameters 1844
Syntax Conventions 1877
Syntax Examples 1877
check_power_intent 1879
cloneMSVGate 1880
commit_power_intent 1881
compare_power_intent 1883
cutBoxListFromPowerDomain 1884
cutPowerDomainByOverlaps 1885
debugPtnBoundaryPorts 1886
deletePowerSwitch 1887
exportPowerSwitch 1889
PSO File Generation and Usage 1889
free_power_intent 1891
getCPFUserAttributes 1892
getInstPowerDomain 1893
getMsvMode 1894
getPGPinUseSignalRoute 1895
getUPFPortAttr 1896
getValidPowerSwitchLocation 1897
importPowerSwitch 1898
PSO File Generation and Usage 1898
mark_physical_power_domains 1900
modifyPowerDomainAttr 1901
movePowerSwitch 1904
read_power_intent 1906
rechainPowerSwitch 1907
replacePowerSwitch 1912
reportIsolation 1914
reportPowerDomain 1915

reportPowerSwitch 1918
reportShifter 1919
reportVoltage 1920
restorePowerSwitch 1921
routePGPinUseSignalRoute 1921
savePowerSwitch 1923
setMsvMode 1923
setPGPinUseSignalRoute 1927
swapRegularCellWithAlwaysOnCell 1928
verifyPowerDomain 1929
verifyPowerSwitch 1932
write_power_intent 1935
Low Power Global Variables 1938
init_power_intent_file 1938
18 1940
Machine Learning Commands 1940
init_ml 1940
write_ml_design 1943
write_ml_library 1946
write_ml_timing_graph 1948
write_ml_timing_path 1950
19 1954
Metal and Via Fill Commands and Global Variables 1954
Metal and Via Fill Commands 1954
add_metal_fill_signoff 1955
add_pg_fill 1965
Pegasus PG Fill 1965
add_track_fill 1968
addViaFill 1970
deleteMetalFill 1973
fixOpenFill 1974
get_metal_fill_signoff_mode 1975
get_pg_fill_config 1977
get_track_fill_config 1979
getMetalFillSpacingTable 1981
report_metal_fill 1983

run_pvs_drc_rules 1985
set_metal_fill_signoff_mode 1988
set_pg_fill_config 1994
set_track_fill_config 1996
setMetalFill 1998
setMetalFillSpacingTable 2003
setViaFill 2005
trimMetalFill 2006
trimMetalFillNearNet 2008
Metal and Via Fill Global Variables 2011
fill_setting_save 2011
20 2013
Mixed Signal Commands 2013
deleteIntegRouteConstraint 2013
getIntegRouteConstraint 2016
pull_block_constraint 2021
run_vsr 2024
setIntegRouteConstraint 2028
writeIntegRouteConstraint 2042
21 2047
Multiple-CPU Processing Commands 2047
checkMultiCpuUsage 2047
getDistributeHost 2050
getMultiCpuUsage 2052
monitor_hosts 2054
report_resource 2058
setDistributeHost 2062
setMultiCpuUsage 2069
22 2075
OpenAccess Commands 2075
add_oa_library_property 2076
compare_cellview 2077
copyOaRestoreFiles 2079
createLib 2081
dd_get 2083

get_oa_default_rule_lib 2085
getOaxMode 2087
oaIn 2089
oaOut 2092
registerTrigger 2094
report_oa_lib 2095
save_abstract 2097
saveOaBlackboxes 2099
set_cell_binding 2100
setOaxMode 2101
unlockOaDesign 2112
update_oa_lib 2113
write_lef_library 2115
write_oa_techfile 2117
23 2119
Partition Commands 2119
add_connection_to_neighbor_partition 2121
addNetToNetGroup 2123
addPinToPinGroup 2124
alignPtnClone 2125
assembleDesign 2129
assignIoPins 2133
assignPtnPin 2137
changeBBoxMasterFromR0 2143
changeBBoxMasterToR0 2144
checkHierRoute 2145
checkPinAssignment 2147
clonePtnObj 2152
commit_pushdown_eco 2154
comparePinAssignStatistics 2155
connectMacroFeedthrough 2156
convertBlackBoxToFence 2160
convertFenceToBlackBox 2161
create_partition_wrapper 2162
create_pushdown_eco 2164
createNetGroup 2166

createPinBlkg 2168
createPinGroup 2170
createPinGuide 2172
createPtnCut 2174
createPtnFeedthrough 2175
definePartition 2176
deleteAllPtnCuts 2181
deleteAllPtnFeedthroughs 2182
deleteNetFromNetGroup 2183
deleteNetGroup 2184
deletePartition 2185
deletePinBlkg 2188
deletePinFromPinGroup 2189
deletePinGroup 2190
deletePinGuide 2191
deletePtnCut 2193
editPin 2194
fix_feedthru_assign 2202
flattenCoverCell 2203
flattenPartition 2204
get_ptn_fplan_mode 2206
getBlackBoxArea 2207
getClonePtnOrient 2208
getPinAssignMode 2209
getPinConstraint 2211
getPtnPinStatus 2213
handlePtnArealo 2214
hier_clock_route 2215
hiliteFeedthroughNets 2217
insertPtnFeedthrough 2218
legalizePin 2222
loadBlackBoxNetlist 2225
loadPtnPin 2226
merge_hierarchical_def 2227
partition 2230
partitionPushSpecialWires 2236
pinAlignment 2238

pinAnalysis 2241
place_cells_at_center_for_feedthrus 2242
place_partition_clone 2243
push_ptn_network 2245
recreatePtnCellBlockage 2248
report_partitions 2249
reportPinAssignStatistics 2251
reportTwoPinChain 2253
reportUnalignedNets 2255
resizeBlackBox 2260
saveBlackBox 2261
savePartition 2262
savePtnPin 2266
selectPtnPinGuide 2267
set_ptn_fplan_mode 2268
setClonePtnOrient 2269
setPinAssignMode 2270
setPinConstraint 2275
setPromotedMacroPin 2279
setPtnPinStatus 2281
setPtnPinUSE 2283
showPtnWireX 2284
specifyBlackBox 2287
specifyPartition 2291
undo_uniquify_partition 2292
uniquify_partition 2294
unloadPtnPin 2295
unsetPinConstraint 2296
unspecifyBlackBox 2299
write_generalized_feedthru_paths 2300
changeCurrentDesign 2302
highlightSyncObj 2303
createBusNetGroup 2304
check_bus_guides 2306
write_net_groups_topo 2308
24 2310

Placement Commands and Global Variables 2310
Placement Commands 2310
add_decomp_filler 2313
add_gate_array_filler 2316
add_tap_walls 2318
addEndCap 2319
addFiller 2321
addFillerGap 2326
addMimCap 2327
addSpareInstance 2328
addWellTap 2330
change_lB_cell_site_name 2335
check_base_layer_density 2336
check_library 2338
checkFiller 2339
checkPlace 2341
clearScanDisplay 2344
clearSpareCellDisplay 2345
clonePlace 2346
congRepair 2347
create_inst_space_group 2348
createGAFillerGroup 2349
createPipelineBusGuide 2350
createPipelineNetGroup 2351
createSpareModule 2352
delete_cell_stack_area 2354
delete_cell_stack_group 2355
delete_cell_virtual_align 2356
delete_inst_space_group 2357
delete_pg_keepout 2358
deleteAllCellPad 2359
deleteAllScanCells 2360
deleteCellEdgeSpacing 2361
deleteCellEdgeType 2362
deleteCellPad 2363
deleteFiller 2364

deleteGAFillerGroup 2366
deleteInstPad 2367
deleteMimCap 2369
deletePipelineNetGroup 2370
deleteScanCell 2371
deleteScanChain 2372
deleteScanChainPartition 2373
deleteSpareModule 2374
displayScanChain 2375
displaySpareCell 2376
generate_vertical_cell_edge_constraint 2377
get_well_tap_mode 2378
getDensityMapMode 2379
getEndCapMode 2380
getFillerMode 2382
getPinDensityMapMode 2384
getPlaceMode 2385
getScanReorderMode 2388
modifyPipelineNetGroup 2390
pack_align_macros 2391
place_connected 2393
place_design 2396
place_opt_design 2399
placeInstance 2401
placeJtag 2403
placePad 2406
placePipeline 2407
placeSpareModule 2408
power_integrity_place 2411
queryDensityInBox 2412
queryPinDensity 2413
refinePlace 2414
report_cell_edge_spacing 2416
report_cell_edge_type 2417
report_cell_stack_area 2418
report_cell_stack_group 2419
report_cell_virtual_align 2420

report_inst_space_group 2421
report_pg_keepout 2422
report_scan_chain 2423
reportCellPad 2424
reportCongestion 2426
reportDensityMap 2429
reportInstPad 2431
reportJtagInst 2432
reportPinDensityMap 2433
reportPipeline 2435
reportScanCell 2436
reportScanChainPartition 2437
reset_spare_insts 2438
scanReorder 2440
scanTrace 2443
set_well_tap_mode 2445
setDensityMapMode 2449
setEndCapMode 2451
setFillerMode 2459
setPinDensityMapMode 2463
setPlaceMode 2464
setScanReorderMode 2478
specify_cell_stack_area 2481
specify_cell_stack_group 2482
specify_cell_virtual_align 2483
specify_pg_keepout 2484
specifyCellEdgeSpacing 2486
specifyCellEdgeType 2488
specifyCellPad 2491
specifyInstPad 2492
specifyJtag 2494
specifyLockupElement 2496
specifyScanCell 2497
specifyScanChain 2498
specifyScanChainPartition 2499
specifySelectiveBlkgGate 2500
specifySpareGate 2501

swap_well_taps 2502
traceJtag 2503
unplaceJtag 2505
unspecifyJtag 2506
unspecifySelectiveBlkgGate 2507
Placement Global Variables 2508
spgM3StripePushDown 2508
spgM3StripeShrink 2509
25 2510
Power Analysis Commands 2510
getPGNetResis 2510
getSpecialNetResis 2513
26 2517
Power Calculation Commands 2517
analyze_thermal 2518
create_thermal_model 2519
dump_unannotated_nets 2523
extract_metal_density 2525
encrypt_thermal_file 2526
get_default_switching_activity 2528
get_dynamic_power_simulation 2529
get_power_analysis_mode 2530
map_activity_file 2532
merge_vtm 2536
propagate_activity 2538
query_power_data 2539
read_activity_file 2542
report_instance_power 2553
report_power 2554
reset_power_activity 2576
restore_power_database 2577
set_default_switching_activity 2579
set_dynamic_power_simulation 2585
set_inst_temperature_file 2588
set_metal_density_options 2589
set_power 2593

set_power_analysis_mode 2604
set_power_calc_temperature 2663
set_power_include_file 2664
set_power_output_dir 2665
set_switching_activity 2666
set_thermal_analysis_mode 2671
set_twf_attribute 2673
set_virtual_clock_network_parameters 2677
write_power_constraints 2679
write_tcf 2680
27 2683
Power-Grid Library Commands 2683
check_pg_library 2683
generate_pg_library 2689
generate_tech_pg_lib 2690
generate_xpgv 2692
merge_pg_library 2694
rename_pg_library 2696
set_advanced_pg_library_mode 2697
set_pg_library_mode 2710
trim_pg_library 2722
validate_pg_library 2724
28 2729
Power Planning Commands 2729
add_target_pg 2729
addRing 2735
addSpecialRoute 2741
addSplitPowerVia 2742
addStripe 2746
colorizePowerMesh 2759
editPowerVia 2760
get_add_target_pg_mode 2766
get_reinforce_pg_mode 2767
getAddRingMode 2768
getAddStripeMode 2770
getViaGenMode 2772

loadSpecialRoute 2774
reinforce_pg 2775
route_pg 2785
saveSpecialRoute 2787
set_add_target_pg_mode 2788
set_reinforce_pg_mode 2791
setAddRingMode 2794
setAddStripeMode 2798
setViaGenMode 2818
trim_pg 2839
29 2842
Power Route Commands 2842
fixVia 2842
getSrouteMode 2844
reportPowerRoute 2846
setSrouteMode 2849
sroute 2862
30 2879
Prototyping Commands 2879
FlexModel Commands 2879
assemble_proto_model 2881
create_buffer_psPM_table 2884
create_flexfiller_route_blockage 2887
create_proto_model 2889
create_ps_per_micron_model 2892
get_proto_design_mode 2894
get_proto_mode 2896
get_proto_model 2898
hilite_proto_model 2902
identify_proto_model 2906
proto_design 2908
replace_proto_model 2910
report_proto_model 2913
set_proto_design_mode 2918
set_proto_mode 2920
set_proto_model 2929

set_proto_model_physical_constraint 2933
set_proto_timing_settings 2935
SoC Architecture Info (SAI) Commands 2937
add_sai_boundary_flops 2938
end_sai 2941
read_sai 2942
report_sai_constraint 2944
31 2947
Rail Analysis Commands 2947
analyze_esd_network 2948
analyze_esd_voltage 2950
analyze_joule_heat 2952
analyze_package 2954
analyze_rail 2956
analyze_resistance 2957
analyze_self_heat 2963
analyze_signal_resistance 2965
calculate_didt 2966
calculate_differential_voltage 2967
calculate_noise_margin 2968
convert_gds_to_def 2970
create_current_region 2976
create_die_model 2979
create_hier_view 2987
create_power_pads 2991
create_what_if_shape 2994
debug_irdrop 2997
extract_package 3001
generate_esd_rlrp_report 3003
generate_power_up_report 3004
get_ir_insight 3006
map_dies 3011
map_die_package 3014
read_power_rail_results 3015
report_power_rail_results 3019
scale_what_if_capacitance 3023

scale_what_if_current 3025
scale_what_if_resistance 3027
set_advanced_package_options 3029
set_advanced_rail_options 3030
set_die_model 3031
set_dynamic_rail_simulation 3033
set_multi_die_analysis_mode 3035
set_net_group 3037
set_offchip_package_trace 3038
set_package 3040
set_pg_nets 3042
set_power_data 3044
set_power_pads 3047
set_rail_analysis_domain 3050
set_rail_analysis_mode 3051
set_self_heat_analysis_mode 3088
set_voltage_regulator_module 3090
view_dynamic_movie 3092
view_dynamic_waveform 3093
view_package_results 3095
32 3096
RC Extraction Commands and Global Variable 3096
RC Extraction Commands 3096
create_parasitic_coordinate_transform 3097
extractRC 3100
generateCapTbl 3101
generateRCFactor 3104
getExtractRCMode 3108
rcOut 3111
read_parasitics 3113
report_net_parasitics 3116
report_rcdb 3118
report_unit_parasitics 3120
reset_parasitics 3123
restoreRC 3124
saveRC 3125

setExtractRCMode 3126
setShrinkFactor 3146
spefIn 3147
wireload 3152
write_extraction_spec 3155
RC Extraction Global Variable 3157
extract_shrink_factor 3157
33 3159
Route Commands and Global Variables 3159
Route Commands 3159
add_tracks 3161
addChannelDensityControl 3166
check_tracks 3167
checkTQuantusModelFile 3169
colorize_cell_library 3170
createShield 3171
createTQuantusModelFile 3172
deleteShield 3173
detailRoute 3174
dumpCongestArea 3175
dumpNanoCongestArea 3176
dumpNetsInCongestedArea 3177
earlyGlobalRoute 3178
ecoRoute 3180
fix_drc_with_patch 3183
getAttribute 3184
getNanoRouteMode 3185
getRouteMode 3188
globalDetailRoute 3190
globalRoute 3191
report_route 3192
report_tracks 3198
reportCongestArea 3199
reportRoute 3202
reportShield 3205
reportWire 3207

route_fix_signoff_drc 3210
routeDesign 3211
saveRouteGuide 3216
setAttribute 3217
setNanoRouteMode 3224
setRouteMode 3251
writeHif 3256
route_fix_ir 3257
Route Global Variables 3259
trPrintIgnoredPadNets 3259
34 3261
Signal ElectroMigration Commands 3261
get_signal_em_analysis_mode 3261
set_signal_em_analysis_mode 3263
35 3275
Signal Integrity Commands 3275
check_noise 3276
create_spice_deck 3278
define_bundled_bus 3287
fixACLimitViolation 3288
get_glitch_threshold 3291
getSIMode 3292
readTransitionFile 3294
report_bundled_bus 3295
report_double_clocking 3297
report_instance_cdb 3299
report_noise 3300
report_noise_lib_pin 3307
report_noise_propagation 3308
report_voltage_scaling 3309
report_voltage_swing 3310
reset_glitch_derate 3311
reset_noise_lib_pin 3312
set_annotated_glitch 3313
set_cdb_binding 3315
set_glitch_derate 3316

set_glitch_threshold 3317
set_model_priority 3319
set_mutex_condition 3320
set_noise_lib_pin 3321
set_quiet_attacker 3322
setOutboundReport 3323
setSIMode 3324
update_glitch 3331
analyze_ir_paths 3331
define_glitch_holding_resistance 3333
36 3335
Structured Data Path Commands 3335
addSdpGroupMember 3336
addSdpObject 3339
checkSdpGroup 3341
createSdpGroup 3345
deleteSdpObject 3350
detachSdpGroup 3353
flipSdpObject 3354
getSdpGroupAttribute 3356
getSdpMode 3358
moveSdpObject 3360
placeSdpGroup 3366
readSdpFile 3369
setSdpGroupAttribute 3372
setSdpMode 3375
setSdpObjectStatus 3380
setSdpTopGroup 3382
writeSdpFile 3385
37 3388
Timing Analysis Commands 3388
Path Exception Priorities 3392
Examples of Path Exception Priorities 3394
Bidirectional Pin Defaults 3394
Defaults for Pin (or Port) Constraints 3394
Defaults for Port Constraints 3395

all_analysis_views 3396
all_constraint_modes 3398
all_delay_corners 3400
all_hold_analysis_views 3402
all_library_sets 3403
all_op_conds 3404
all_rc_corners 3405
all_setup_analysis_views 3407
calNegSlack 3408
check_instance_library_in_views 3409
check_ldb_version 3410
check_timing 3411
checkTimingLibrary 3429
create_analysis_view 3432
create_constraint_mode 3434
create_delay_corner 3436
create_library_set 3444
create_op_cond 3448
create_property_alias 3450
create_rc_corner 3451
createUserDisableForCombLoopBreak 3456
display_timing_map 3458
get_analysis_view 3459
get_capacitance_unit 3461
get_clock_network_objects 3462
get_constant_for_timing 3463
get_constraint_mode 3465
get_delay_corner 3467
get_equivalent_cells 3470
get_interactive_constraint_modes 3472
get_lib_cell_leakage_power 3473
get_library_set 3475
get_op_cond 3477
get_pba_mode 3479
get_propagated_clock 3480
get_rc_corner 3481
get_resistance_unit 3483

get_socv_rc_variation_factor 3484
get_socv_reporting_nsigma_multiplier 3485
get_time_unit 3486
getAnalysisMode 3487
getTimeLibFile 3490
read_path_descriptions 3491
read_sdf 3493
read_spdf 3499
read_twf 3500
report_analysis_coverage 3502
report_analysis_summary 3511
report_analysis_views 3515
report_annotated_assertions 3518
report_annotated_check 3520
report_annotated_delay 3526
report_annotated_parasitics 3529
report_annotations 3533
report_aocv_derate 3537
report_case_analysis 3542
report_cell_instance_timing 3547
report_clock_gating_check 3553
report_clock_propagation 3556
report_clock_timing 3559
report_clocks 3576
report_constraint 3586
report_cppr 3601
report_design 3606
report_fanin 3610
report_fanout 3613
report_inactive_arcs 3616
report_instance_library 3621
report_lib_arcs 3624
report_min_pulse_width 3630
report_mode 3635
report_net 3637
report_path_exceptions 3641
report_path_groups 3645

report_ports 3647
report_socv_interconnect_variation 3653
report_socv_library 3654
report_statistical_timing_derate_factors 3656
report_timing 3659
report_timing_derate 3714
report_wire_load 3720
reportTimingLib 3724
reset_instance_library 3726
reset_path_adjust_group 3727
reset_sdf_assertions 3728
reset_timing_derate 3729
reset_wire_load_mode 3731
reset_wire_load_model 3732
reset_wire_load_selection_group 3734
set_analysis_view 3735
set_aocv_thresholds 3739
set_default_view 3740
set_instance_library 3742
set_interactive_constraint_modes 3743
set_io_thresholds 3745
set_normalized_driver_waveform_lib 3748
set_path_adjust 3749
set_path_adjust_group 3751
set_pba_mode 3753
set_socv_constraint_config 3754
set_socv_rc_variation_factor 3757
set_socv_reporting_nsigma_multiplier 3758
set_table_style 3760
set_timing_derate 3769
set_wire_load_mode 3778
set_wire_load_model 3781
set_wire_load_selection_group 3784
setAnalysisMode 3786
signoffTimeDesign 3799
timeDesign 3801
update_analysis_view 3809

update_constraint_mode 3811
update_delay_corner 3814
update_io_latency 3822
update_library_set 3823
update_rc_corner 3826
write_global_slack_report 3830
write_ldb 3835
write_path_descriptions 3837
write_sdf 3838
write_timing_windows 3856
writeAnnotatedTransition 3860
writeDesignTiming 3863
writeTimingCon 3864
38 3868
Timing Budgeting Commands 3868
checkPartitionSdc 3868
deriveTimingBudget 3871
getBudgetingMode 3874
justifyBudget 3876
justifyException 3878
modifyBudget 3879
resetModifiedBudget 3880
saveTimingBudget 3881
setBudgetingMode 3883
setCycleBudgetRatio 3896
setFixedBudget 3900
setPtnUserCnsFile 3901
39 3902
Timing Constraint Commands 3902
create_clock 3905
create_generated_clock 3908
current_design 3914
current_instance 3915
get_sdc_mode 3916
group_path 3917
reset_annotated_check 3919

reset_annotated_delay 3922
reset_annotated_transition 3923
reset_aocv_stage_weight 3925
reset_case_analysis 3926
reset_clock 3927
reset_clock_gating_check 3929
reset_clock_groups 3930
reset_clock_latency 3932
reset_clock_sense 3935
reset_clock_transition 3936
reset_clock_tree_latency 3939
reset_clock_uncertainty 3940
reset_data_check 3943
reset_disable_clock_gating_check 3945
reset_disable_timing 3946
reset_drive 3949
reset_driving_cell 3950
reset_generated_clock 3951
reset_ideal_latency 3953
reset_ideal_network 3954
reset_ideal_transition 3955
reset_input_delay 3956
reset_load 3958
reset_max_capacitance 3959
reset_max_fanout 3960
reset_max_time_borrow 3961
reset_max_transition 3962
reset_min_capacitance 3963
reset_min_fanout 3964
reset_min_pulse_width 3965
reset_min_transition 3966
reset_mode 3967
reset_output_delay 3969
reset_path_exception 3973
reset_path_group 3978
reset_pll_timing 3979
reset_propagated_clock 3980

reset_property 3983
reset_pulse_clock_max_transition 3984
reset_pulse_clock_max_width 3985
reset_pulse_clock_min_transition 3986
reset_pulse_clock_min_width 3987
reset_resistance 3988
set_active_clocks 3989
set_annotated_check 3990
set_annotated_delay 3992
set_annotated_transition 3996
set_aocv_interface_path_offset 3998
set_aocv_stage_weight 4000
set_case_analysis 4001
set_clock_exclusivity 4003
set_clock_gating_check 4004
set_clock_groups 4007
set_clock_latency 4010
set_clock_sense 4013
set_clock_transition 4016
set_clock_uncertainty 4018
set_data_check 4021
set_disable_clock_gating_check 4024
set_disable_timing 4025
set_dont_touch 4027
set_dont_touch_network 4028
set_dont_use 4029
set_drive 4030
set_driving_cell 4032
set_false_path 4035
set_fanout_load 4040
set_ideal_latency 4041
set_ideal_network 4043
set_ideal_transition 4045
set_input_delay 4048
set_input_transition 4051
set_load 4053
set_logic_one 4056

set_logic_zero 4057
set_max_capacitance 4058
set_max_delay 4060
set_max_fanout 4065
set_max_time_borrow 4067
set_max_transition 4069
set_min_capacitance 4072
set_min_delay 4075
set_min_fanout 4082
set_min_pulse_width 4084
set_min_transition 4086
set_mode 4089
set_multicycle_path 4091
set_output_delay 4097
set_pll_timing 4100
set_propagated_clock 4104
set_property 4106
set_pulse_clock_max_transition 4107
set_pulse_clock_max_width 4109
set_pulse_clock_min_transition 4111
set_pulse_clock_min_width 4113
set_resistance 4115
set_sense 4117
reset_sense 4119

40 4121

Timing Debug Commands 4121

analyze_paths_by_basic_path_group 4122
analyze_paths_by_bottleneck 4124
analyze_paths_by_clock_domain 4127
analyze_paths_by_critical_false_path 4129
analyze_paths_by_drv 4131
analyze_paths_by_hier_port 4134
analyze_paths_by_hierarchy 4136
analyze_paths_by_view 4141
create_histogram_png 4143
create_path_category 4144

delete_path_category 4154
dump_histogram_view 4155
highlight_timing_report 4156
highlight_timing_report 4156
load_path_categories 4157
load_timing_debug_report 4159
open_gtd 4162
save_path_categories 4163
write_category_summary 4164
write_path_list_summary 4167
write_text_timing_report 4168
41 4172
Timing Global Commands 4172
get_global 4172
report_globals 4173
set_global 4174
42 4176
Timing Global Variables 4176
aocv_chip_size 4186
aocv_core_size 4188
distributed_mmmc_disable_reports_auto_redirection 4189
lib_build_asynch_de_assert_arc 4190
report_precision 4191
report_precision_capacitance 4192
report_precision_derate 4193
report_precision_power 4194
report_precision_sensitivity 4195
report_timing_format 4196
timing_all_registers_include_icg_cells 4197
timing_allow_input_delay_on_clock_source 4198
timing_analysis_precision_ps 4199
timing_aocv_analysis_mode 4200
timing_aocv_derate_mode 4201
timing_aocv_slack_threshold 4202
timing_aocv_stage_count_recalculate_on_timing_reset 4203
timing_apply_check_derate_to_external_output_delay 4204

timing_apply_default_primary_input_assertion 4205
timing_apply_exceptions_to_data_check_related_pin 4206
timing_apply_setup_hold_exceptions_to_data_check_related_pin 4208
timing_cap_unit 4209
timing_case_analysis_for_icg_propagation 4210
timing_case_analysis_for_sequential_propagation 4211
timing_case_analysis_propagation 4212
timing_check_timing_report_all_checks 4213
timing_check_timing_signal_level_high_to_low_threshold 4214
timing_check_timing_signal_level_low_to_high_threshold 4215
timing_clock_phase_propagation 4216
timing_clock_source_use_driving_cell 4217
timing_clock_uncertainty_from_to_precedence 4218
timing_collection_all_fanin_fanout_traversal_mode 4219
timing_collection_result_display_limit 4220
timing_collection_variable_assignment_compatibility 4221
timing_constraint_disable_min_max_input_delay_worst_casing 4222
timing_constraint_enable_detailed_report_invalid_begin_end_points 4223
timing_constraint_enable_drv_limit_override 4224
timing_constraint_enable_logging 4225
timing_constraint_enable_report_invalid_begin_end_points 4226
timing_constraint_enable_search_path 4227
timing_constraint_enable_separate_multicycle_data_checks 4228
timing_constraint_load_minimal_set_for_automated_eco 4229
timing_constraint_path_delay_exclude_check_delay_from_ignore_clock_latency 4230
timing_constraint_path_delay_exclude_io_delay_from_ignore_clock_latency 4231
timing_constraint_path_delay_exclude_unconstrained_endpoints 4232
timing_constraint_path_delay_include_clock_pin_endpoints 4233
timing_constraint_update_io_latency_averaging_mode 4234
timing_constraint_warn_for_timing_derate_exceeding_max_limit 4235
timing_context_apply_default_primary_input_assertion 4236
timing_context_clock_mapping_check_waveform 4237
timing_context_clock_mapping_percentage_tolerance 4238
timing_context_port_based_clock_mapping 4239
timing_continue_on_error 4241
timing_cppr_enable_mismatch_transition_mode 4242
timing_cppr_opposite_edge_mean_scale_factor 4244

timing_cppr_opposite_edge_sigma_scale_factor 4245
timing_cppr_opposite_edge_sigma_scale_factor_cell 4246
timing_cppr_opposite_edge_sigma_scale_factor_net 4247
timing_cppr_propagate_thru_latches 4248
timing_cppr_remove_clock_to_data_crp 4249
timing_cppr_self_loop_mode 4250
timing_cppr_skip_clock_reconvergence 4251
timing_cppr_skip_clock_reconvergence_for_unmatched_clocks 4252
timing_cppr_threshold_ps 4253
timing_cppr_transition_sense 4254
timing_create_clock_default_propagated 4255
timing_create_clock_use_ideal_slew 4256
timing_default_opcond_per_lib 4257
timing_defer_mmmc_object_updates 4258
timing_derate_aocv_dynamic_delays 4259
timing_derate_aocv_reference_point 4260
timing_derate_dynamic_compatibility 4261
timing_derate_incremental_adjust_additive_mode 4262
timing_derate_incremental_multiply_accumulative_mode 4263
timing_derate_negative_delay_backward_compatibility 4264
timing_derate_ocv_reference_point 4265
timing_derate_spatial_distance_unit 4266
timing_derate_voltage_scaling_mode 4267
timing_disable_bidi_output_timing_checks 4268
timing_disable_black_box_endpoints 4269
timing_disable_bus_contention_check 4270
timing_disable_clock_gating_checks 4271
timing_disable_clockperiod_checks 4272
timing_disable_constant_propagation_for_sequential_cells 4273
timing_disable_drv_reports_on_constant_nets 4274
timing_disable_floating_bus_check 4275
timing_disable_genclk_combinational_blocking 4276
timing_disable_inferred_clock_gating_checks 4277
timing_disable_internal_inout_cell_paths 4278
timing_disable_internal_inout_net_arcs 4279
timing_disable_lib_pulsewidth_checks 4280
timing_disable_library_data_to_data_checks 4281

timing_disable_library_tiehi_tielo 4282
timing_disable_netlist_constants 4283
timing_disable_nochange_checks 4284
timing_disable_non_sequential_checks 4285
timing_disable_output_as_clock_port 4286
timing_disable_parallel_arcs 4287
timing_disable_pulsewidth_same_edge_si_cppr_mode 4288
timing_disable_recovery_removal_checks 4289
timing_disable_report_header_info 4290
timing_disable_retime_clock_path_slew_propagation 4291
timing_disable_sdf_retain_arc_merging 4292
timing_disable_skew_checks 4293
timing_disable_test_signal_arc 4294
timing_disable_timing_model_latch_inferencing 4295
timing_disable_tristate_disable_arcs 4296
timing_disable_user_data_to_data_checks 4297
timing_driving_cell_override_library 4298
timing_dynamic_loop_breaking 4299
timing_enable_all_fanin_fanout_levels_compatibility 4300
timing_enable_aocv_slack_based 4301
timing_enable_case_analysis_conflict_warning 4302
timing_enable_clock2clock_clockgating_check 4303
timing_enable_clock_phase_based_rise_fall_derating 4304
timing_enable_data_through_clock_gating 4305
timing_enable_derating_for_pulsewidth_checks 4306
timing_enable_early_late_data_slews_for_setuphold_mode_checks 4307
timing_enable_genclk_divide_by_inherit_parent_duty_cycle 4308
timing_enable_genclk_edge_based_source_latency 4309
timing_enable_genclk_source_path_register_limit 4310
timing_enable_get_object_escaped_name_backward_compatible 4311
timing_enable_get_objects_regexp_compatibility 4312
timing_enable_hierarchical_get_nets_support 4313
timing_enable_latch_thru_mode 4314
timing_enable_latency_through_clock_gating 4315
timing_enable_mmmc_loop_handling 4316
timing_enable_multi_drive_net_reduction_with_assertions 4317
timing_enable_multifrequency_latch_analysis 4318

timing_enable_path_delay_to_unconstrained_endpoints_compatibility 4319
timing_enable_pessimistic_cppr_for_reconvergent_clock_paths 4320
timing_enable_power_ground_constants 4321
timing_enable_preset_clear_arcs 4322
timing_enable_pulsed_latch 4323
timing_enable_sdc_compatible_data_check_mcp 4324
timing_enable_si_cppr 4325
timing_enable_simultaneous_setup_hold_mode 4326
timing_enable_spatial_derate_mode 4327
timing_enable_tristate_clock_gating 4328
timing_enable_uncertainty_for_clock_checks 4329
timing_enable_uncertainty_for_pulsewidth_checks 4330
timing_extract_model_aocv_mode 4331
timing_extract_model_case_analysis_in_library 4332
timing_extract_model_check_arcs_as_lvf 4333
timing_extract_model_consider_design_level_drv 4334
timing_extract_model_disable_cycle_adjustment 4335
timing_extract_model_exhaustive_validation_dir 4336
timing_extract_model_exhaustive_validation_mode 4337
timing_extract_model_gating_as_nochange_arc 4338
timing_extract_model_ideal_clock_latency_arc 4339
timing_extract_model_include_applied_load_in_characterization_range 4340
timing_extract_model_include_applied_slew_in_characterization_range 4341
timing_extract_model_max_feedthrough_characterization_load 4342
timing_extract_model_non_borrowing_latch_path_as_setup 4343
timing_extract_model_slew_propagation_mode 4344
timing_extract_model_write_clock_checks_as_arc 4345
timing_extract_model_write_clock_checks_as_scalar_tables 4346
timing_extract_model_write_lvf 4347
timing_extract_model_write_min_max_clock_tree_path 4348
timing_generate_normalized_driver_waveform 4349
timing_generated_clocks_allow_nested_assertions 4350
timing_generated_clocks_inherit_ideal_latency 4351
timing_get_of_objects_hier_compatibility 4352
timing_hier_object_name_compatibility 4353
timing_hierarchical_context_continue_on_top_block_clock_mismatch 4354
timing_ignore_clock_based_path_delay_on_data_checks 4355

timing_ignore_lumped_rc_assertions 4356
timing_io_use_clock_network_latency 4357
timing_library_convert_async_setuphold_to_recrem 4358
timing_library_genclk_use_group_name 4359
timing_library_hold_constraint_corner_sigma_multiplier 4360
timing_library_hold_sigma_multiplier 4361
timing_library_infer_async_pins_from_timing_arcs 4362
timing_library_infer_cap_range_from_ccs_receiver_model 4363
timing_library_infer_cap_range_from_ecsm_receiver_model 4364
timing_library_infer_socv_from_aocv 4365
timing_library_read_ccs_noise_data 4366
timing_library_read_without_power 4367
timing_library_scale_aocv_to_socv_to_n_sigma 4368
timing_library_setup_constraint_corner_sigma_multiplier 4369
timing_library_setup_sigma_multiplier 4370
timing_library_use_trilib_drv_values 4371
timing_library_use_two_piece_receiver_cap 4372
timing_library_zero_negative_timing_check_arcs 4373
timing_max_transition_use_si_transition 4374
timing_multifrequency_clock_rounding_factor 4375
timing_normalized_driver_waveform_clip_linear_part 4376
timing_normalized_driver_waveform_weight_factor 4377
timing_null_collection_return_compatibility 4378
timing_path_based_enable_bounding_box_for_io_paths 4379
timing_path_based_enable_closest_common_pin_driver_for_bounding_box 4380
timing_path_based_enable_exhaustive_depth_bounded_by_gba 4381
timing_path_based_enable_report_launch_clock_path 4382
timing_path_based_enable_verbose_mode 4383
timing_path_based_exhaustive_enable_design_coverage 4385
timing_path_based_exhaustive_max_paths_limit 4386
timing_path_based_report_analysis_summary_max_paths_limit 4387
timing_pba_exhaustive_path_nworst_limit 4388
timing_pll_clock_use_driving_cell 4389
timing_prefix_module_name_with_library_genclk 4390
timing_propagate_latch_data_uncertainty 4391
timing_property_arrival_clocks_consider_clock_source_data_phase 4392
timing_property_arrival_clocks_consider_data_phase 4393

timing_property_arrival_clocks_consider_latency_phase 4394
timing_property_arrival_window_enable_tcl_dict_format 4395
timing_property_clock_used_as_data_unconstrained_clock_source_paths 4396
timing_property_return_null_collection_with_quiet 4397
timing_rail_swing_checks_high_voltage_threshold 4398
timing_rail_swing_checks_low_voltage_threshold 4399
timing_read_library_without_ecsm 4400
timing_read_library_without_sensitivity 4401
timing_recompute_sdf_in_setuphold_mode 4402
timing_reduce_multi_drive_net_arcs 4403
timing_reduce_multi_drive_net_arcs_threshold 4404
timing_remove_clock_reconvergence_pessimism 4405
timing_report_analysis_summary_csv_extended_new_format 4406
timing_report_begin_end_pair_max_path_limit 4407
timing_report_check_timing_unconstrained_endpoints_due_to_constants 4408
timing_report_clock_pin_as_begin_point 4409
timing_report_constraint_enable_extended_drv_format 4410
timing_report_constraint_format 4411
timing_report_constraint_rise_fall_clock_period_check 4412
timing_report_constraint_use_infinity_slack_for_unconstrained 4413
timing_report_disable_max_paths_per_group 4414
timing_report_drv_enable_clock_source_as_clock 4415
timing_report_drv_enable_frequency_per_view 4416
timing_report_drv_enable_ghz_notation_for_drv_fields 4417
timing_report_drv_enable_slew_threshold_scaling 4418
timing_report_drv_use_worst_timing_slack 4419
timing_report_enable_clock_unrolling 4420
timing_report_enable_cppr_point 4421
timing_report_enable_lead_trail_to_rise_fall_map 4422
timing_report_enable_markers 4423
timing_report_enable_max_capacitance_drv_for_constants 4424
timing_report_enable_max_path_limit_crossed 4425
timing_report_enable_report_clock_timing_across_clock_pin 4426
timing_report_enable_si_debug 4427
timing_report_enable_slack_summary_at_bottom 4428
timing_report_enable_unique_pins_multiple_capture_clock_paths 4429
timing_report_enable_use_valid_start_end_points 4430

timing_report_generated_clock_info 4431
timing_report_group_based_mode 4432
timing_report_max_transition_check_using_nsigma_slew 4434
timing_report_property_fastest_clock_consider_data_phase 4435
timing_report_pulse_width_matching_launch_capture_paths 4436
timing_report_redirect_message_types 4437
timing_report_retime_formatting_mode 4438
timing_report_skip_constraint_loop_check 4439
timing_report_socv_summary_mean_sigma 4440
timing_report_timing_header_detail_info 4441
timing_report_unconstrained_path_early_late_header 4443
timing_report_unconstrained_paths 4444
timing_report_use_receiver_model_capacitance 4445
timing_report_use_worst_parallel_cell_arc 4446
timing_resolve_driver_conflicts 4447
timing_sdf_adjust_negative_setuphold 4448
timing_sdf_enable_setuphold_scond_ccond 4449
timing_self_loop_paths_no_skew 4450
timing_self_loop_paths_no_skew_max_depth 4451
timing_self_loop_paths_no_skew_max_slack 4452
timing_set_clock_source_to_output_as_data 4453
timing_set_nsigma_multiplier 4454
timing_set_scaling_for_negative_checks 4455
timing_set_scaling_for_negative_delays 4456
timing_socv_preserve_variation_with_annotations 4457
timing_socv_rc_variation_mode 4458
timing_socv_statistical_min_max_mode 4459
timing_socv_view_based_nsigma_multiplier_mode 4460
timing_spatial_derate_chip_size 4461
timing_spatial_derate_distance_mode 4462
timing_suppress_escape_characters 4463
timing_suppress_ilm_constraint_mismatches 4464
timing_time_unit 4465
timing_timing_window_pessimism_removal_include_si_delay 4466
timing_use_clock_pin_attribute_for_clock_net_marking 4467
timing_use_incremental_si_transition 4468
timing_use_latch_early_launch_edge 4469

timing_use_latch_time_borrow 4470
timing_use_verilog_for_model_netlist 4471
timing_waveform_aware_pulse_width_checks_high_voltage_level 4472
timing_waveform_aware_pulse_width_checks_low_voltage_level 4473
timing_write_sdf_no_escape_backslash_control 4474
write_global_slack_worst_trigger_path_on_clocks 4475
43 4477
Timing Modeling Commands 4477
compare_model_timing 4477
do_extract_model 4487
merge_model_timing 4494
write_model_timing 4496
44 4503
Timing Optimization Commands 4503
addRepeaterByRule 4504
addTieHiLo 4509
checkFootPrint 4512
createBasicPathGroups 4513
createTBOptFile 4515
deleteBufferTree 4518
deleteTieHiLo 4520
dumpMultiBitFlopMappingFile 4521
ecoClone 4524
ecoCloneFlop 4527
ecoMergeCombinationalCell 4529
ecoSplitCombinationalCell 4531
ecoSplitComplexFlop 4534
ecoSplitFlop 4536
findLefEquivalentCells 4539
getLatencyFile 4540
getOptMode 4541
getSchedulingFile 4544
getSignoffOptMode 4545
getTieHiLoMode 4548
getUsefulSkewMode 4550
loadFootPrint 4552

optDesign 4554
optPower 4563
optVirtual 4566
reclaimArea 4567
reportAlwaysOnBuffer 4568
reportCapViolation 4569
reportCritInstance 4571
reportCritNet 4573
reportCritTerm 4574
reportFanoutViolation 4575
reportFootPrint 4577
reportIgnoredNets 4579
reportLengthViolation 4581
reportMultibit 4582
reportPathGroupOptions 4586
reportRouteTypeConstraints 4587
reportTranViolation 4593
reportVtInstCount 4595
resetPathGroupOptions 4598
route_opt_design 4599
setBufFootPrint 4602
setDelayFootPrint 4603
setDontUse 4604
setInvFootPrint 4607
setLatencyFile 4608
setMaxCapPerFreq 4609
setMaxCapPerFreqTran 4612
setMaxTranPerFreq 4614
setOptMode 4617
setPathGroupOptions 4651
setSchedulingFile 4656
setSignoffOptMode 4657
setTieHiLoMode 4676
setUsefulSkewMode 4679
signoffOptDesign 4682
skewClock 4686

45 4690
TSV Design Commands 4690
add_obj_on_bump 4691
addTSV 4694
assignTSV 4701
create_stress_rule 4704
createTSVNoLoadSPEF 4705
deleteTSV 4706
read_codesign_die_abstract 4707
read_design_stack_config 4708
readBumpLocation 4710
readDieAbstract 4715
readTSVConfig 4716
unassignTSV 4719
verify_stacked_die 4720
write_codesign_die_abstract 4721
write_design_stack_config 4726
writeBumpLocation 4727
writeDieAbstract 4730
writeMicroBumpMappingFile 4731
writeVSMappingFile 4734
46 4735
Unified Safety Format Commands 4735
USF File Commands 4735
apply_safety_mechanism 4736
create_esubpart 4738
create_failure_mode 4741
create_safety_mechanism 4744
set_safety_mechanism_rules_dcls 4747
set_safety_mechanism_rules_tmr 4751
USF Support Commands 4754
check_safety_mechanism 4754
read_safety_tmr 4757
read_usf 4760
reset_usf 4761
usf_version 4762

write_usf 4763
47 4765
Unix Commands 4765
innovus 4765
lef2oa 4777
nagelfar 4785
oa2lef 4788
48 4791
Verify Commands 4791
clearDrc 4792
createMarker 4793
get_signoff_verify_design_config 4796
get_verify_drc_mode 4798
loadDrc 4800
loadViolationReport 4802
optimizePattern 4806
reportFreqViolation 4812
run_pegasus_drc 4816
saveDrc 4819
set_signoff_verify_design_config 4821
set_verify_drc_mode 4827
setIsolatedCutRule 4833
setSnapGrid 4835
signoff_verify_design 4837
verify_antenna 4844
verify_drc 4846
verifyACLimit 4851
verifyCMP 4852
verifyConnectivity 4855
verifyCutDensity 4861
verifyEndCap 4863
verifyIO2BumpConnectivity 4866
verifyIsolatedCut 4867
verifyLitho 4869
verifyMetalDensity 4873
verifyPowerVia 4877

verifyTieCell 4886
verifyWellAntenna 4888
verifyWellTap 4890
verifyWireGap 4894
violationBrowser 4897
violationBrowserDeleteByArea 4901
violationBrowserReport 4902
49 4905
What-If Timing Commands 4905
checkWhatIfTiming 4906
createWhatIfInternalGeneratedClock 4907
deleteWhatIfTimingAssertions 4910
getWhatIfTimingAssertions 4913
getWhatIfTimingMode 4914
saveWhatIfConstraints 4915
saveWhatIfTimingAssertions 4917
saveWhatIfTimingModel 4918
setWhatIfClockLatency 4919
setWhatIfClockPort 4921
setWhatIfDriveType 4922
setWhatIfLoadType 4925
setWhatIfPortParameters 4927
setWhatIfPortPriority 4930
setWhatIfTimingMode 4931
50 4936
Wire Edit Commands 4936
deleteSecondaryPGNet 4937
deselectSecondaryPGNet 4939
editAddFillet 4940
editAddPoly 4941
editAddRoute 4942
editAddTrimMetal 4944
editAddVia 4946
editAdjust 4947
editChangeLayer 4952
editChangeMask 4953

editChangeNet 4956
editChangeRule 4957
editChangeStatus 4959
editChangeVia 4962
editCommitPoly 4965
editCommitRoute 4966
editCutWire 4967
editDelete 4971
editDeleteFillet 4976
editDeselect 4977
editDuplicate 4980
editFixWideWires 4982
editMerge 4983
editMove 4984
Moving Wires 4984
Moving Physical Pins 4984
editResize 4988
editRotate 4993
editSelect 4996
editSplit 5002
editTrim 5003
end_parallel_edit 5007
getEditMode 5008
prepareForEcoRoute 5012
read_parallel_edit_files 5014
reportLegalWireWidthForBump 5016
selectSecondaryPGNet 5017
setEditMode 5019
start_parallel_edit 5054
Appendix A: Database Object Information 5057
Appendix B: Tcl Globals 5058
List of Globals 5058
Index 5072

About This Manual

The Cadence® Innovus™ Implementation System family of products provides an integrated solution for an RTL-to-GDSII design flow. This manual describes the syntax for the Innovus™ Implementation System (Innovus) text commands.

The chapters are presented alphabetically by general command functionality. Within each chapter, the text commands appear alphabetically.

Audience

This manual is written for experienced designers of digital integrated circuits. Such designers must be familiar with design planning, placement and routing, block implementation, chip assembly, and design verification. Designers must also have a solid understanding of UNIX and Tcl/Tk programming.

Conventions Used in This Manual

This section describes the typographic and syntax conventions used in this manual.

`text`	Indicates text that you must type exactly as shown. For example: `set_message -severity info`
`text`	Indicates information for which you must substitute a name or value. In the following example, you must substitute the name of a specific file for *`depth_of_hierarchy`*: `report_area -depth` *`depth_of_hierarchy`*
text	Indicates the following: • Text found in the graphical user interface (GUI), including form names, button labels, and field names • Terms that are new to the manual, are the subject of discussion, or need special emphasis • Titles of manuals

`[ ]`	Indicates optional arguments. In the following example, you can specify none, one, or both of the bracketed arguments: `command [-arg1] [arg2 value]`
`[ | ]`	Indicates an optional choice from a mutually exclusive list. In the following example, you can specify any of the arguments or none of the arguments, but you cannot specify more than one: `command [arg1 | arg2 | arg3 | arg4]`
`{ | }`	Indicates a required choice from a mutually exclusive list. In the following example, you must specify one, and only one, of the arguments: `command {arg1 | arg2 | arg3}`
`{[ ] [ ]}`	Indicates a required choice of one or more items in a list. In the following example, you must choose one argument from the list, but you can choose more than one: `command {[arg1] [arg2] [arg3]}`
`{ }`	Indicates curly braces that must be entered with the command syntax. In the following example, you must type the curly braces: `command arg1 {x y}`
. . .	Indicates that you can repeat the previous argument.
. . .	Indicates an omission in an example of computer output or input.
Command - Subcommand	Indicates a command sequence, which shows the order in which you choose commands and subcommands from the GUI menu. In the following example, you choose *Power* from the menu, then *Power Planning* from the submenu, and then *Add Ring* from the displayed list: *Power - Power Planning - Add Ring* This sequence opens the *Add Rings* form.

Related Documents

For more information about the Innovus family of products, see the following documents. You can access these and other Cadence documents with the Doc Assistant documentation system.

Innovus Product Documentation

- What's New in Innovus
  Provides information about new and changed features in this release of the Innovus family of products.
- Innovus User Guide
  Describes how to install and configure the Innovus software, and provides strategies for implementing digital integrated circuits.
- Innovus Menu Reference
  Provides information specific to the forms and commands available from the Innovus graphical user interface.
- Innovus Database Access Command Reference
  Lists all the Innovus database access commands and provides a brief description of syntax and usage.
- Innovus Foundation Flows User Guide
  Describes how to use the scripts that represent the recommended implementation flows for digital timing closure with the Innovus software.
- Mixed Signal Interoperability Guide
  Describes the digital mixed-signal flow.
- README file
  Contains installation, compatibility, and other prerequisite information. You can read this file online at downloads.cadence.com.

Innovus Stylus Common UI Product Documentation

- Innovus Stylus Common UI Migration Guide
  Provides information on migrating from legacy to the Stylus Common UI version of the Innovus software.
- What's New in Innovus Stylus Common UI
  Provides information about new and changed features in this release of the Innovus family of products.
- Innovus Stylus Common UI User Guide
  Describes how to install and configure the Innovus Stylus Common UI software, and provides strategies for implementing digital integrated circuits.
- Innovus Stylus Common UI Text Reference Manual
  Describes the Innovus Stylus Common UI text commands, including syntax and examples.
- Innovus Stylus Common UI Menu Reference
  Provides information specific to the forms and commands available from the Innovus Stylus Common UI graphical user interface.
- Stylus Common UI Database Object Information
  Provides information about Stylus Common UI database objects.
- Innovus Stylus Common UI Mixed Signal (MS) Interoperability Guide
  Describes the digital mixed-signal flow using Innovus Stylus Common UI.

For a complete list of documents provided with this release, see the Doc Assistant online documentation system.

1

Advanced Timing Tcl Scripting Commands

- add_to_collection
- all_clocks
- all_connected
- all_fanin
- all_fanout
- all_inputs
- all_instances
- all_outputs
- all_registers
- append_to_collection
- compare_collections
- copy_collection
- define_property
- filter_collection
- foreach_in_collection
- get_activity
- get_arcs
- get_cells
- get_clocks
- get_designs
- get_generated_clocks
- get_lib_arcs
- get_lib_cells
- get_lib_pg_pins
- get_lib_pins
- get_libs
- get_nets
- get_object_name
- get_path_groups
- get_pg_nets
- get_pg_pins
- get_pins
- get_ports
- get_power
- get_property
- index_collection
- list_property
- query_objects

- range_collection
- remove_from_collection
- report_property
- sizeof_collection
- sort_collection

Overview

The following lists the Advanced Timing Tcl Scripting commands. For each category of similar commands, we explain what they are used for followed by some simple examples.

all_* Commands

The `all_*` commands return a collection of all the items of a specific type. Several of the commands provide options to filter the collection based on certain properties:

- `all_clocks`
- `all_connected`
- `all_fanin`
- `all_fanout`
- `all_inputs`
- `all_instances`
- `all_outputs`
- `all_registers`

Examples

- To set all the clocks to propagated mode, use the following command:
  ```
  set_propagated_clock [all_clocks]
  ```
- To report the worst timing path that begins at a register and ends at an output port, use the following command:
  ```
  report_timing -from [all_registers] -to [all_outputs]
  ```
- To store a collection of pins that fanout from a specific port, use the following command:
  ```
  set clk1Fanout [all_fanout -from clk1 -pin_levels 1]
  ```

get_* Commands

The `get_*` commands return a collection of items filtered by name pattern or object type. Additionally, you can use the `-filter` option for most of the `get_*` commands to filter the collection by property value. See the get_property command for a list of all the properties.

The `filter_expression` for the `-filter` option can be created by combining several object properties using the following relational and logical operators: `>`, `<`, `==`, `!=`, `<=`, `>=`, `&&`, `||`, `=~`, `!~`, `AND`, and `OR`. You also can use parentheses to create expressions. The `get_*` commands include the following:

- `get_arcs`
- `get_cells`
- `get_clocks`
- `get_designs`
- `get_generated_clocks`
- `get_lib_arcs`
- `get_lib_cells`
- `get_lib_pins`
- `get_libs`

- `get_nets`
- `get_object_name`
- `get_path_groups`
- `get_pins`
- `get_ports`

Examples

- To return a collection of all input ports (same as running `[all_inputs]`), use the following command:
  ```
  get_ports -filter "direction==in"
  ```
- To return a collection of pins that match `*/D`, use the following command:
  ```
  get_pins -hier -filter "hierarchical_name =~ */D"
  ```
- To return a collection of the libraries a cell belongs to, use the following command:
  ```
  get_lib_cells INVXL
  ```

property Commands

The list_property command lists the properties for the different object types. This command, without arguments, will list all the properties for all object types. Use the `-type` option to list properties for a specific object type. Legal option types are: `cell`, `clock`, `design`, `lib`, `lib_cell`, `lib_pin`, `lib_timing_arc`, `net`, `path_group`, `pin`, `port`, `timing_arc`, `timing_path`, or `timing_point`. For example:

```
list_property -type lib_timing_arc
```

The above command generates the following output:

```
object_type : lib_timing_arc
================================================
property                  | return_type
------------------------------------------------
from_lib_pin              | collection
has_ccs_noise             | boolean
has_socv                  | boolean
is_disabled               | boolean
mode                      | string
object_type               | string
output_signal_level_high  | float
output_signal_level_low   | float
sdf_cond                  | string
sense                     | string
timing_type               | string
to_lib_pin                | collection
when                      | string
when_end                  | string
when_start                | string
```

Examples

- The get_property command returns the value of the specified property. To report the clock period of clk1, use the following command:
  ```
  get_property [get_clocks clk1] period
  ```
- The report_property command returns the properties and their values for a specified object. To report the properties for clock clk1, use the following command:
  ```
  report_property [get_clocks clk1]
  ```
- For timing related properties such as properties for timing arcs, paths, and points run `report_timing -collection`. This will return the results in a collection of timing paths.
  - To report timing and store the top 10 paths as a collection in the variable `$paths`, use the following command:
    ```
    set paths [report_timing -collection -max_paths 10]
    ```
  - To filter `$paths` to include only those paths with slack between -0.806ns and 0ns, use the following command:
    ```
    set newPaths [filter_collection $paths {(slack > -0.806 && slack < 0.0)}]
    ```

collection Commands

Once you have the collection defined you can manipulate, filter, cycle through, and query it using the collection related commands:

- `add_to_collection`
- `append_to_collection`
- `compare_collections`
- `copy_collection`
- `filter_collection`
- `foreach_in_collection`
- `index_collection`
- `query_objects`
- `remove_from_collection`
- `sizeof_collection`
- `sort_collection`

Examples

- To list the contents of a collection, use the following command:
  ```
  query_objects $collectionName
  ```
- To cycle through the items of a collection, use the following command:
  ```
  foreach_in_collection iCell [get_cells *] {
  Puts "Cell = [get_property $iCell hierarchical_name]"
  }
  ```

add_to_collection

```
add_to_collection
[-help]
base_collection
second_collection_or_list
[-unique]
```

Creates a new collection by adding objects from one collection to another collection. The following rules apply:

- If all of the objects in the base collection are of the same type, only the objects in the second collection that are the same type as those in the base collection are added to the new collection. If you specify an object list instead, all of the objects in the list must be the same type as those in the base collection.
- If the base collection contains different types of objects, all objects in the second collection are added. In this situation, you cannot specify an object list.

If the software finds no objects to add, an empty collection is returned.

Parameters

`-help`	Prints out the command usage.
`base_collection`	Specifies the base collection of objects. Objects from the second collection or object list are added to this collection to create the new collection.
`second_collection_or_list`	Specifies the second object collection, or a list of objects.
`-unique`	Removes any duplicate objects from the new collection.

Examples

- The following command returns a collection which contains both the output ports as well as the input ports:
  ```
  add_to_collection [all_outputs] [all_inputs]
  ```

  Output:
  ```
  CLK4 CLK3 CLK2 CLK1 IN4 IN3 IN2 IN1 OUT4 OUT3 OUT2 OUT1 OUT5
  ```

  Here, the input and output ports are as follows:
  ```
  all_inputs
  CLK4 CLK3 CLK2 CLK1 IN4 IN3 IN2 IN1

  all_outputs
  OUT4 OUT3 OUT2 OUT1 OUT5
  ```

Related Information

- compare_collections
- copy_collection
- filter_collection

all_clocks

```
all_clocks
[-help]
```

Creates a collection of all the clocks in the current design. If no clocks were defined, an empty string `""` is returned.

To filter clocks, use the `get_clocks` command to create a collection of clocks matching a specific pattern. Assign these clocks to a variable or pass them using another command.

Parameters

None

Example

- The following example applies the `set_propagated_clock` command to all the clocks in the design:
  ```
  set_propagated_clock [all_clocks]
  ```

Related Information

- `create_clock`
- `get_clocks`
- `set_propagated_clock`
- `all_inputs`

all_connected

```
all_connected
[-help]
[-leaf]
single_object_collection_or_object
```

Returns a collection of all of the objects connected to the specified net, pin, or port.

Parameters

`-help`	Prints out the command usage.
`-leaf`	Returns only the flattened connection points (leaf pins) for the specified net. *Default:* Returns all hierarchical ports connected to the specified net
`single_object_collection_or_object`	Specifies the net, pin, or port for which to return the collection of connected objects. You can specify an object name, or a single-object collection.

Examples

- The following command returns a collection of all of the objects connected to the pin `b1_Sin1`:
  ```
  all_connected Sin1
  ```
  The software returns the following information:
  ```
  Sin1
  ```
- The following command returns a collection of all of the objects connected to a pin contained in the single collection object i_block1/b1_Sin1:
  ```
  all_connected [get_pins {i_block1/b1_Sin1}]
  ```
  The software returns the following information:
  ```
  Sin1
  ```
- The following command returns a collection of all of the hierarchical ports connected to the net `net_1`:
  all_connected [get_nets net_1]

  The software returns the following information:

  TL2/I2_in TL1/I1_out
- The following command returns a collection that contains only the flattened connection points for the net `net_1`:
  ```
  all_connected -leaf [get_nets net_1]
  ```
  The software returns the following information:
  ```
  TL2/L1/RX/D TL1/RX/Q
  ```

all_fanin

```
all_fanin
[-help]
[-hpin]
[-only_cells]
[-startpoints_only]
-to {collection | object_list}
[-trace_through {case_disable | user_disable | all | clocks | loop_snipped}]
[-view view_name]
[-levels value | -pin_levels value]
[ > | >> ]
```

Returns a collection of pins, ports, or cells that exist in the fanin cone of the specified objects.

Parameters

`-help`	Prints out the command usage.
`> | >>`	Stores the generated collection in the specified file. You can add the `.gz` extension to the file name to generate a compressed report. **Note:** The file name must be the last parameter specified.
`-hpin`	Returns hierarchical pins along with the flat pins.
`-levels value`	Specifies the number of gate levels to go back from a specific pin object to create the collection.
`-only_cells`	Returns a collection only of the cells in the fanin zone.
`-pin_levels value`	Specifies the number delay arcs to go back from a specific pin object to create the collection.
`-startpoints_only`	Returns a collection only of valid start points in the fanin zone.
`-trace_through {case_disable |user_disable | all | clocks | loop_snipped}`	You can specify one of the following options: • `case_disable`: Allows traversal of arcs disabled using `set_case_analysis` command. Arcs disabled using `set_disable_timing` are not traversed. • `user_disable`: Allows traversal of arcs disabled using `set_disable_timing` command, but arcs disabled using `set_case_analysis` command are not traversed. • `all`: Allows traversal through all the arcs • `clocks`: Allows traversal through pins for which clock constraints (such as, `create_clock`, `create_generated_clock`) are applied. The pins that are disabled using the `set_case_analysis` and `set_disable_timing` commands are not traversed. • `loop_snipped`: Allows trace through the loop snipped arcs.
`-to {collection | object_list}`	Specifies the objects for which to return the collection. You can specify a list of objects, or a collection of objects.
`-view view_name`	Specifies the view name for which fan cone traversal is to be performed.

Examples

- The following command returns the collection of cells in the fanin zone of the output port OUT1:
  ```
  all_fanin -to OUT1 -only_cells
  ```
- The following command returns all the valid startpoints (sequential elements and/or input ports) in the fanin zone of OUT1, allowing the traversal through arcs disabled by `set_case_analysis` and `set_disable_timing`:
  ```
  all_fanin -to OUT1 -startpoints_only -trace_through all
  ```

Related Information

- `all_fanout`

all_fanout

```
all_fanout
[-help]
[-endpoints_only]
[-hpin]
[-only_cells]
[-trace_through {case_disable | user_disable | all | clocks | loop_snipped}]
[-view view_name]
[-levels value | -pin_levels value]
{-from {collection | object_list} }
[ > | >> ]
```

Returns a collection of pins, ports, or cells that exist in the fanout cone of the specified objects.

Parameters

`-help`	Prints out the command usage.
`-endpoints_only`	Returns a collection only of valid end points in the fanout zone.
`-hpin`	Returns hierarchical pins along with the flat pins.
`> | >>`	Stores the generated collection in the specified file. You can add the `.gz` extension to the file name to generate a compressed report. **Note:** The file name must be the last parameter specified.
`-from {collection | object_list}`	Specifies the objects for which to return the collection. You can specify a list of objects, or a collection of objects.
`-levels value`	Specifies the number of gate levels to go forward from a specific pin object to create the collection.
`-only_cells`	Returns a collection only of the cells in the fanout zone.
`-pin_levels value`	Specifies the number delay arcs to go forward from a specific pin object to create the collection.
`-trace_through {case_disable |user_disable | all | clocks | loop_snipped}`	Allows traversal through disabled arcs. You can specify one of the following options: • `case_disable`: Arcs which are disabled using the `set_case_analysis` command and through physical constant are traversed. The arcs disabled using `set_disable_timing` are not traversed. • `user_disable`: Arcs disabled using `set_disable_timing` command are traversed but those disabled using `set_case_analysis` command are not traversed. • `all`: Enables trace through all arcs • `clocks`: Allows traversal through pins for which clock constraints (such as, `create_clock`, `create_generated_clock`) are applied. The pins that are disabled using the `set_case_analysis` and `set_disable_timing` commands are not traversed. • `loop_snipped`: Allows trace through the loop snipped arcs.
`-view view_name`	Specifies the view name for which fan cone traversal is to be performed.

Examples

- The following command returns the collection of cells in the fanout zone of the input port IN1:
  ```
  all_fanout -from IN1 -only_cells
  ```
- The following command returns all the valid endpoints (sequential elements and/oroutput ports) in the fanout zone of IN1, allowing the traversal through arcs disabled by `set_case_analysis` and `set_disable_timing`:
  ```
  all_fanout -from IN1 -endpoints_only -trace_through all
  ```

Related Information

- `all_fanin`

all_inputs

```
all_inputs
[help]
[-clock list_of_clocks]
[-no_clocks]
[-edge_triggered]
[-level_sensitive]
```

Creates a collection of all the input ports in the current design. To filter inputs, use the `get_ports` command to create a collection of ports matching certain criteria. Assign these ports to a variable or pass them using another command. You can limit the contents of the collection to ports with input delay relative to a specific clock.

Parameters

`-help`	Prints out the command usage.
`-clock list_of_clocks`	Considers only input ports with input delay relative to the specified list of clocks.
`-edge_triggered`	Returns a collection of input ports with the `set_input_delay -clock` constraint.
`-level_sensitive`	Returns a collection of input ports with the `set_input_delay -level_sensitive` constraint.
`-no_clocks`	Returns primary input ports that do not have `create_clock` or `create_generated_clock` constraints.

Example

- The following command returns the names all of the input ports with input delay relative to an `ideal_clk`:

  ```
  all_inputs -clock ideal_clk
  ```

Related Information

- `get_ports`
- `report_ports`
- `set_input_delay`

all_instances

```
all_instances
[-help]
object
[-hierarchical]
```

Returns a collection of all instances available in a design, or all instances of a specified library cell existing in the current design.

You cannot use the `all_instances` command on the top level of the design (`all_instances [get_design top]`).

However, you can use the command for hierarchical modules inside the top level. For example, if `SUB` is a hierarchical module in the top level of the design, you can specify the following command to return all instances of that module: `all_instances [get_design SUB]`.

Parameters

`-help`	Prints out the command usage.
`-hierarchical`	Specifies that the patterns should be matched hierarchically. If the hierarchical name of an instance at a particular hierarchical level matches the patterns, it is included in the result.
object	Specifies the name of a design or a library cell in the current design. You can specify a pattern for the design or library cell name, or a single collection of a design or library cell.

Examples

- The following command returns a collection of all the instances of the library cell BUF:
  ```
  all_instances BUF
  ```

all_outputs

```
all_outputs
[-help]
[-clock list_of_clocks]
[-edge_triggered]
[-level_sensitive]
```

Creates a collection of all output ports in the current design. To filter outputs, use the `get_ports` command to create a collection of ports matching certain criteria. Assign these ports to a variable or pass them using another command. You can limit the contents of the collection to ports with output delay relative to a specific clock.

Parameters

`-help`	Prints the command usage.
`-clock list_of_clocks`	Considers only output ports with an output delay relative to a specific list of clocks.
`-edge_triggered`	Returns a collection of output ports with the `set_output_delay -clock` constraint.
`-level_sensitive`	Returns a collection of output ports with the `set_output_delay -level_sensitive` constraint.

Example

- The following command returns the names all of the output ports with an output delay relative to an ideal_clk:
  ```
  > all_outputs -clock ideal_clk
  ```

Related Information

- `get_ports`
- `report_ports`
- `set_output_delay`

all_registers

```
all_registers
[help]
[-clock {clock_list}]
[-cells]
[-rise_clock {clock_list}]
[-fall_clock {clock_list}]
[-flops | -edge_triggered]
[-no_hierarchy]
[-latches | -level_sensitive]
[-macros]
[-master_slave]
[-data_pins]
[-clock_pins]
[-output_pins]
[-async_pins]
[-slave_clock_pins]
```

Returns a collection of instances and pins of flip-flops, latches, or sequential macros in the design. A cell is considered sequential if the library cell defines trigger or timing check arcs.

Note: The following options can be used only after building the timing graph:

- `-clock`
- `-rise_clock`
- `-fall_clock`

Parameters

`-help`	Prints out the command usage.
`-async_pins`	Returns a collection of async preset or clear pins.
`-cells`	Creates a collection of cells. This is the default option.
`-clock` {*`clock_list`*}	Specifies the name of a clock or a list of clocks. The `all_registers` command creates a collection of registers clocked by the specified *`clock_list`*.
`-clock_pins`	Returns a collection of register clock pins. You can use other parameters to further filter this collection.
`-data_pins`	Returns a collection of register data input pins. You can use other parameters to further filter this collection.
`-fall_clock` {*`clock_list`*}	Specifies the name of a clock or a list of clocks. The `all_registers` command creates a collection of registers opened by the falling edge of the clocks specified in the *`clock_list`*.
`-flops | -edge_triggered`	Returns a collection of edge-triggered sequential instances. Both the `-flops` and `-edge_triggered` options return the same collection. The `-flops` option is provided for backward compatibility.
`-latches | -level_sensitive`	Returns a collection of level-sensitive sequential instances. Both `-latches` and `-level_sensitive` options return the same collection. The `-latches` option is provided for backward compatibility.
`-macros`	Returns sequential instances that are neither flip-flops nor latches.
`-master_slave`	Returns a collection of master-slave register cells.
`-no_hierarchy`	Searches the current instance only; does not descend the hierarchy.
`-output_pins`	Returns a collection of register output pins.
`-rise_clock` {*`clock_list`*}	Specifies the name of a clock or a list of clocks. The `all_registers` command creates a collection of registers opened by the rising edge of the clocks specified in the *`clock_list`*.
`-slave_clock_pins`	Returns a collection of slave clock pins of the master-slave registers. You can specify the slave clock pins as `clocked_on_also` in the library.

Example

- The following command reports the10 worst paths going to registers in the design:
  ```
  report_timing -to [all_registers] -max_paths 10
  ```
- the following script using the `all_registers` command along with other Innovus commands can be used to get number of flops in the design:
  ```
  set flopName [all_registers -flops]

  set flopCount [sizeof_collection $flopName]

  puts "Number of flops in the design is $flopCount"
  ```

Related Information

- get_cells
- get_pins

append_to_collection

```
append_to_collection
[help]
var_name
second_collection_or_list
[-unique]
```

Appends objects from one collection to another collection. The following rules apply:

- If all of the objects in the base collection are of the same type, only the objects in the second collection that are the same type as those in the base collection are appended to the base collection.
- If the base collection contains different types of objects, all objects in the second collection are appended to it. In this situation, you cannot specify an object list.

If the software finds no objects to add, an empty collection is returned.

Parameters

`-help`	Prints out the command usage.
var_name	Specifies a pointer to the base collection of objects. The objects in the second collection or object list are appended to this collection. If you specify a pointer that does not refer to an existing base collection, the software creates a new collection that uses the pointer as a reference.
second_collection_or_list	Specifies the second collection of objects, or an object list.
`-unique`	Retains any duplicate objects that already exist in the base collection, but does not append duplicate objects from the second collection to the base collection.

Examples

- The following command appends the nets in the specified collection to the collection `newVar`:

  ```
  append_to_collection newVar [get_nets {CLK1}]
  ```

- The following command appends the specified nets to the collection `newVar`:

  ```
  append_to_collection newVar {Sin1 CLK1 Sout1}
  ```

- You can use the following commands to create another collection:

  ```
  tempus 6> set base_clock [get_clocks sbw_clk]
  0xe50
  tempus 7> set top_clock [get_clocks aclk]
  0xe51
  tempus 8> set base_top_clock ""
  tempus 13> append_to_collection base_top_clock $base_clock
  0xe54
  tempus 14> append_to_collection base_top_clock $top_clock
  0xe54
  tempus 15> get_clocks $base_top_clock
  sbw_clk aclk
  0xe55
  tempus 16> get_clocks $base_clock
  sbw_clk
  0xe56
  tempus 17> get_clocks $top_clock
  aclk
  0xe57
  ```

  Note: If a new collection variable is defined with an existing collection, then both point to the same object set. Any modifications made into either collection will be visible in both cases - because they reference each other.

Related Information

- add_to_collection
- compare_collections
- copy_collection
- filter_collection
- foreach_in_collection

compare_collections

```
compare_collections
[-help]
collection1
collection2
[-order_dependent]
```

Compares two collections, and returns a value of `0` if all of the objects contained in both collections are the same. If the objects contained in both collections are different, the software returns a value of `1`.

Parameters

`-help`	Prints out the command usage.
collection1	Specifies the first collection.
collection2	Specifies the second collection.
`-order_dependent`	Specifies that the order of the objects in both collections must be the same for the collections to be considered the same.

Examples

- The following example shows 3 collections as defined below:

```
set coll1 [all_registers]
set coll2 [get_cells -hierarchical * -filter "is_sequential==true"]
set coll3 [get_cells -hierarchical * -filter "is_combinational==true"]
```

- The collection coll1 and coll2 are identical, while coll1 and coll3 are not, and this can be checked by the following commands:

```
compare_collections $coll1 $coll2
0
compare_collections $coll1 $coll3
1
```

Related Information

- add_to_collection
- copy_collection
- filter_collection
- foreach_in_collection

copy_collection

```
copy_collection
[-help]
base_collection
```

Returns a collection that is an exact copy of the specified base collection.

Parameters

`-help`	Prints out the command usage.
`base_collection`	Specifies the base collection of objects.

Related Information

- `add_to_collection`
- `compare_collections`
- `filter_collection`
- `foreach_in_collection`

define_property

```
define_property
[-help]
-object_type {object_type_list}
-type data_type
[-lower_bound val]
[-upper_bound val]
[-string_values {values}]
property_name
```

Defines a new property.

Design objects can be associated with user-defined commands that can be used for analysis and debugging purposes.

You can use the `define_property` command if a define statement is not used for the specified property value(s). To read property values from the library, the command must be issued before the timing library is read.

Parameters

`-help`	Prints out the command usage.
`-lower_bound` *val*	Specifies a lower range - integer, float, or double.
`-object_type` *object_type_list*	Specifies the type of object on which the property is to be defined.
property_name	Specifies the name of the property.
`-string_values` *values*	Specifies the values that can be defined for the property.
`-type` *data_type*	Specifies the type of property.
`-upper_bound` *val*	Specifies a higher range - integer, float, or double.

Examples

- The following example defines property having object type `pin` and type of property `int`:
  ```
  define_property -object_type pin -type int slk
  ```
- The following example defines a property on a library arc:
  ```
  define_property -object_type lib_arc -type int libArcProperty
  ```
- The following example defines `aocv_weight` property on a library arc:
  ```
  define_property -type float -object_type lib_arc aocv_weight
  ```
- The following example defines certain nets as critical, using the `critical_net` property and then setting it to `true` for those nets:
  ```
  define_property critical_net -object_type net -type boolean
  set_property [get_nets n2] critical_net true
  ```

  You can view the report and query on critical nets, as shown below:

  ```
  innovus> report_property [get_nets n2]

  ...

  property                          | value
  ---------------------------------------------------
  capacitance_max                   | 0.002
  capacitance_min                   | 0.002
  driver_pins                       | {...}
  escaped_name                      | n2
  hierarchical_name                 | n2
  has_detailed_parasitics           | false
  is_dont_touch                     | false
  is_hierarchical                   | false
  load_pins                         | {...}
  name                              | n2
  object_type                       | net
  pin_capacitance_max               | 0.002
  ```

```
pin_capacitance_max_rise           | 0.002
pin_capacitance_max_fall           | 0.002
pin_capacitance_min                | 0.002
pin_capacitance_min_rise           | 0.002
pin_capacitance_min_fall           | 0.002
total_capacitance_max_fall         | 0.002
total_capacitance_max_rise         | 0.002
total_capacitance_min_rise         | 0.002
total_capacitance_min_fall         | 0.002
wire_capacitance_max               | 0.000
wire_capacitance_min               | 0.000
wire_capacitance_max_rise          | 0.000
wire_capacitance_max_fall          | 0.000
wire_capacitance_min_rise          | 0.000
wire_capacitance_min_fall          | 0.000
critical_net                       | true
```

```
innovus> get_property [get_nets n2] critical_net
true
```

```
innovus> get_nets -filter "critical_net==true"
n2
0x1c
```

Related Commands

- get_property
- report_property
- set_property
- reset_property

filter_collection

```
filter_collection
[-help]
base_collection
{filter_expression}
[-nocase]
[-quiet]
[-regexp]
```

Returns a collection of objects that were filtered from the specified collection of objects based on user-specified criteria (*filter_expression*). The following rules apply:

- The *filter_expression* can be created by combining several object properties using the following relational and logical operators: >, <, ==, !=, <=, >=, &&, ||, =~, !~, AND, OR. You also can use parentheses to create expressions.
- You can use the defined and undefined existence operators to determine whether an attribute is defined for an object.
- All design object properties supported through the get_property command can be used for filtering the base collection.

Note: You also can perform filtering operations by specifying the -filter parameter with the get_* commands.

Parameters

-help	Prints out the command usage.
base_collection	Specifies the collection of objects from which to filter objects for the new collection.
filter_expression	Specifies the criteria with which to filter objects. The software adds any objects in the base collection that match *filter_expression* to the new collection. See "Examples" on how to create filter expressions.
-nocase	Specifies that pattern matching is not case sensitive. **Note:** You must specify -regexp in order to use this parameter.
-quiet	Suppresses all the error and warning messages that are generated when the filter_collection command is run.
-regexp	Treats the specified patterns as regular expression patterns. *Default:* Treats the specified pattern as a wildcard.

Examples

- The following command returns a single-object collection containing the pin with the hierarchical name lat_3/Q:
  ```
  filter_collection [get_pins] {hierarchical_name == lat_3/Q}
  ```
- The following command returns a collection of pins that have a maximum slew that is greater than 0.1:
  ```
  filter_collection [get_pins] {slew_max_rise > 0.1}
  ```
- The following commands create a new collection called newPaths that contains all of the paths from the collection paths that have a slack value that is greater than -0.806 and less than 0.0:
  ```
  set paths [report_timing -collection -max_paths 10]
  set newPaths [filter_collection $paths {(slack > -0.806 && slack < 0.0)}]
  ```
- The following example creates a pin type user property to annotate useful_skew values which may be used in custom functions for analysis and implementation:
  ```
  define_property useful_skew -type float -object_type pin
  set_interactive_constraint_modes [all_constraint_modes -active]
  ```
  You can now apply some values to a few register clock pins:
  ```
  set_property [get_pin hier1/reg1/clock] useful_skew 1.5
  set_property [get_pin hier1/hier2/reg2/clock] useful_skew 2.5
  ```

In the script, you can look for pin objects that have (or does not have) this attributed applied:

```
find clock pins without useful skew
set pins_without_useful [filter_collection [all_registers -clock_pins] \
"undefined(useful_skew)"]
count the number of registers with useful skew applied
set pins_with_useful [filter_collection [all_registers -clock_pins] \
"defined(useful_skew)"]
```

- The following command returns a collection of object names that do not match with “*clk*”:

```
filter_collection $abc {full_name !~ *clk*}
out in1 in2 in3 select TopBus[2] TopBus[1] TopBus[0] bidi
0x53
```

- The following command returns a collection of object names that are not “*clk*” only:

```
filter_collection $abc {full_name != *clk*}
out clk1 clk2 in1 in2 in3 select TopBus[2] TopBus[1] TopBus[0] bidi
0x59
```

- The following command returns a collection of object names that match with “*clk*”:

```
filter_collection $abc {full_name =~ *clk*}
clk1 clk2
0x5f
```

Related Information

- add_to_collection
- compare_collections
- copy_collection
- foreach_in_collection

foreach_in_collection

```
foreach_in_collection
[-help]
var_name
collection
body
```

Iterates through all objects in the specified collection and executes the commands in the specified script.

Parameters

`-help`	Prints out the command usage.
body	Specifies the script of commands to execute during each iteration.
collection	Specifies the collection of objects through which to iterate.
var_name	Specifies the iterator variable. During each iteration, *var_name* is set to a collection of exactly one object. Any command that accepts collections can accept *var_name* as a value.

Examples

- A script with the following commands would print out the hierarchical names of all of the cells in the design:

```
foreach_in_collection iCell [get_cells *] {
puts "Cell = [get_property $iCell hierarchical_name]"

    }
    Cell = ck_0
    Cell = ck_1
    Cell = ck_2
    Cell = ck_4
```

- The following example shows a more complex use of the `foreach_in_collection` command.

```
set cellCollection [get_cells temp1_reg]
set libCellCollection [get_lib_cells -of_objects $cellCollection]
set libArc [get_lib_arcs -of_objects $libCellCollection]

foreach_in_collection arc $libArc {
set fpin [get_property $arc from_lib_pin]

set tpin [get_property $arc to_lib_pin]
set sense [get_property $arc timing_type]

set from_pin_name [get_property $fpin name]
set to_pin_name [get_property $tpin name]

puts [format "%25s -> %25s %-20s" $from_pin_name $to_pin_name $sense]
}
```

```
   CK -> D           setup_rising
   CK -> D           hold_rising
   CK -> CK          minimum_period
   CK -> CK          minimum_period
   CK -> CK          min_pulse_width

   foreach_in_collection.txt
   CK -> CK          min_pulse_width
   RN -> RN          min_pulse_width
   CK -> RN          recovery_rising
   CK -> RN          removal_rising
   CK -> Q           falling_edge
   RN -> Q           clear
```

Related Information

- add_to_collection
- compare_collections
- copy_collection
- filter_collection

get_activity

```
get_activity
[-help]
[-instance <string>]
[-list_of_nets_based_on_driver <string>]
[-list_of_nets_based_on_source_of_activity_info <string>]
[-net <string>]
[-outfile <string>]
[-pin <string>]
[-port <string>]
[-report_average_switching_activity]
[-report_cell_group_activity_summary_report]
[-summary]
[-tcl_list]
```

Determines the source of activity for the specified nets/pins/ports. The power analysis engine has various sources from where activities can be annotated from, such as switching activity specification (input/sequential/global/clock gate), clock definitions/constants through SDC/TWF, activity files (VCD/TCF/SAF), user defined activity and/or propagated values.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `get_activity` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_activity`
`-instance string`	
	Specifies the top/hierarchy instance or the leaf level instance name for which average activity is to be reported.
`-list_of_nets_based_on_driver {primary_inputs | sequential | combinational | memory | ICG | black_box}`	
	Specifies to output the list of nets based on the specified net driver. You can specify one or more net drivers at a time, as shown below: `get_activity -list_of_nets_based_on_driver primary_inputs|combinational| sequential | ICG | memory | black_box` To specify multiple drivers, you can use **/** (means 'or') or **&** (means 'and') between two drivers, without a space.
`-list_of_nets_based_on_source_of_activity_info {activity_file|set_switching_activity|propagated|default}`	
	Specifies to output the list of nets based on the specified activity source information.
`-net string`	Specifies the name of the net. **Note:** You can use the `get_nets` command to determine the activity source for a collection of nets.
`-outfile string`	Specifies the output file into which the activity sources are to be written.
`-pin string`	Specifies the name of the pin. **Note:** You can use the `get_pins` command to determine the activity source for a collection of pins.
`-port string`	Specifies the name of the port. **Note:** You can use the `get_ports` command to determine the activity source for a collection of ports.
`-report_average_switching_activity`	
	Specifies to report the average activity of the given instance/hierarchy/top-level.
`-report_cell_group_activity_summary_report`	
	Specifies to give a summary of cell group activity.

`-summary`	Generates a detailed activity annotation report of the specified nets, pins, and ports. This report contains the following information: • Primary Inputs • Sequential Outputs • Memory/Macro Outputs • Tristate Outputs **Note:** Use the `get_activity -summary` command to report activity annotation summary without printing information for nets/pins/ports.
`-tcl_list`	Produces the report in the Tcl list format instead of a tabular format. The `-tcl_list` and `-outfile` parameters are mutually exclusive; you cannot specify them together.

The activity report for pins is displayed in the following format:

```
<Pin Name> <Source of the Activity Value> <Duty Cycle> <Toggles/Sec>
```

The report fields are described below:

- `Pin Name`: Name of the specified pins.
- `Source of the Activity Value`: Source of activity for the specified nets, pins, or ports.
- `Duty Cycle`: Probability of signal at 1.
- `Toggles/Sec`: Number of toggles divided by the total simulation time.

Examples

- The following example returns the activity sources of all nets that match the specified pattern and writes the output information to a file named `nets.rpt`:

  ```
  get_activity -net [get_nets n*] -outfile nets.rpt
  ```

  The following is the example output of the nets.rpt file:

  ```
  int3 activity_file 0.5 5e+07
  int2 sequential_activity 0.5 2.5e+07
  int1 user_defined_activity 0.5 1e+07
  nt3 global_activity 0.5 5e+07
  nt3 input_activity 0.5 5e+07
  int5 clock_gate_activity 0.5 2.5e+07
  int1 user_defined_activity 0.5 1e+07out
  sequential_activity 0.5 2.5e+07
  clk sdc/twf 0.5 5e+08
  c_int2 propagation 0.5 5e+08
  ```

- The following example returns the activity source of the pin `i_4/A1` and writes the output information to a power report file named `pin.rpt`:

  ```
  get_activity -pin i_4/A1 -outfile pin.rpt
  ```

- The following command report the average activity of all nets:

  ```
  get_activity -report_average_switching_activity
  ```

  The following is the example output:

  ```
  Avg Switching activity of all nets 4.727e+17
  ```

- The following command gives a summary of cell group activity:

  ```
  get_activity -report_cell_group_activity_summary_report
  ```

  The following is the example output:

  ```
  Group Activity Summary Report:
  Source ->                   activity file           set_switching_activity   propagated              default                 Total
  All nets                    2                       1                        2                       5
  10
  Break-up based on net driver:
  Primary Input               2(40%)                  0(0%)                    0(0%)                   3(60%)
  5
  Sequential                  0(0%)                   1(33.33%)                0(0%)                   2(66.67%)
  3
  Combinational               0(0%)                   0(0%)                    2(100%)                 0(0%)
  2
  ```

```
Memory                    0(0%)                0(0%)                0(0%)                0(0%)
0
ICG                       0(0%)                0(0%)                0(0%)                0(0%)
0
Black Box                 0(0%)                0(0%)                0(0%)                0(0%)              0
```

- The following command gives the list of nets based on primary inputs:
  `get_activity -list_of_nets_based_on_driver primary_inputs`
  The following is the example output:
  `D2 D1 CP2 CP1 dummy0`
- The following command gives the list of nets based on primary inputs or sequential driver:
  `get_activity -list_of_nets_based_on_driver primary_inputs|sequential`
  The following is the example output:
  `Q2 Q1 D2 D1 CP2 CP1 n1 dummy0`
- The following command gives the list of nets based on primary inputs or sequential driver for the instance `ff1b`:
  `get_activity -list_of_nets_based_on_driver primary_inputs|sequential -instance ff1b`
- The following command gives the list of nets based on the propagated source for the instance `i2`:
  `get_activity -list_of_nets_based_on_source_of_activity_info propagated -instance i2`
- The following command gives a summary of cell group activity for the instance `i2`:
  `get_activity -instance i2 -report_cell_group_activity_summary_report`
- The following command gives the list of nets based on the default activity:
  `get_activity -list_of_nets_based_on_source_of_activity default`

  The following is the example output:
  `rst shutdown isolate coef_ld start bypass test_mode se sdi Xn_in[3] Xn_in[2] Xn_in[1] Xn_in[0]`
  `Yn_in[15] Yn_in[14] Yn_in[13] Yn_in[12] Yn_in[11] Yn_in[10] Yn_in[9] Yn_in[8] Yn_in[7] Yn_in[6]`
- The following command gives the list of nets based on the activity file:
  `get_activity -list_of_nets_based_on_source_of_activity activity_file`

  The following is the example output:
  `ref_clk test_clk pll_clk`
- The following command gives the list of nets based on the switching activity:
  `get_activity -list_of_nets_based_on_source_of_activity set_switching_activity`

  The following is the example output:
  `rst ref_clk shutdown isolate coef_ld start bypass test_clk test_mode se sdi sdo Xn_in[3] Xn_in[2]`
  `Xn_in[1] Xn_in[0] Yn_in[15] Yn_in[14] Yn_in[13]`

Related Information

- `get_power`
- `set_power_analysis_mode`
- `report_power`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*

get_arcs

```
get_arcs
[-help]
{ [-to to_list] [-from from_list]
 | -of_objects object_list}
[-filter expr]
[-quiet]
```

Creates a collection of timing arcs. Assign these arcs to a variable, or pass them to another command. If no objects match the criteria, an empty collection is returned.

Note: The timing graph must be built before you can use this command. A warning message is issued indicating that the timing graph is not generated.

Parameters

`-help`	Prints out the command usage.
`-filter expr`	Filters a collection based on the specified expression. For any arcs that match the pattern, the software evaluates the expression based on the attribute of the arc. For details on how to create filter expressions, refer to the `filter_collection` command.
`-from from_list`	Adds all timing arcs that fan in directly to the specified pins or ports to the collection. You can specify a list of pins or a list of ports, but you cannot specify a list that contains both pins and ports. You can specify a pattern for the pin or port name, a collection of pins or ports, or a pattern and a collection. **Note:** You cannot specify this parameter with the `-of_objects` parameter.
`-of_objects object_list`	Adds all timing arcs that are associated with the specified cells or nets to the collection. You can specify a list of cells or a list of nets, but you cannot specify a list that contains both cells and nets. You can specify a pattern for the cell or net name, a collection of cells or nets, or a pattern and a collection. **Note:** You cannot specify this parameter with the `-to` or `-from` parameters.
`-quiet`	Suppresses all error and warning messages generated when the `get_arcs` command is run.
`-to to_list`	Adds all timing arcs that fan out directly from the specified list of pins or ports to the collection. You can specify a list of pins or a list of ports, but you cannot specify a list that contains both pins and ports. You can specify a pattern for the pin or port name, a collection of pins or ports, or a pattern and a collection. **Note:** You cannot specify this parameter with the `-of_objects` parameter.

Examples

- The following command returns a collection of the timing arcs fan in directly to the pin `dff1/CP`:
  ```
  get_arcs -from dff1/CP
  ```
- The following command creates a collection of the timing arcs that are associated with the nets in the specified collection:
  ```
  get_arcs -of_objects [get_nets {n1}]
  ```

Related Information

- `get_property`

get_cells

```
get_cells
[-help]
[-hierarchical]
[-hsc char]
[-filter expr]
[-leaf]
[-regexp]
[-nocase]
[-quiet]
[<patterns> | -of_objects <object_list>]
```

Creates a collection of instances in the current design whose name matches the supplied pattern list. Assign this collection to a variable or pass it as an argument in another command.

Parameters

`-help`	Prints out the command usage.
`-filter expr`	Filters the collection with the specified expression. For any cells that match the pattern, the software evaluates the expression based on the attribute of the cell. You can filter cells using any of the cell properties supported through the `get_property` command. For details on how to create filter expressions, refer to the `filter_collection` command.

`-hierarchical`	Specifies that the patterns should be matched hierarchically. If the flat name of an instance at a particular hierarchical level matches the patterns, it is included in the result. For example. if there is top level hierarchical instance `Top` and it contains another sub-instance called `FF1`, then the following command can be used to get the instance `Top/FF1` (since the given pattern will be matched with instance flat name `FF1`):
	`get_cells -hier FF*`
	However, the following command will display an error:
	`get_cells -hier Top/FF*`

`-hsc char`	Specifies the hierarchical delimiter for patterns. For example, if the hierarchical delimiter is `@`, the `sub/I1@I2` pattern matches instance `I2` within the hierarchy limited at `sub/I1`.
`-leaf`	Returns leaf cells only. When the `-leaf` parameter is not specified, the `get_cells` command returns the hierarchical instances present on a net along with the flat instances at the same hierarchical level as the specified object. **Note:** The `-leaf` parameter can only be specified with the `-of_objects` parameter.
`-nocase`	Specifies that pattern matching is not case sensitive. **Note:** You must specify `-regexp` in order to use this parameter.

`-of_objects object_list`	Creates a collection of all cells associated with the specified pins, nets, library cells, or design cells.
	You can specify a list of pins or a list of nets, but you cannot specify a list that contains both pins and nets. You can specify patterns for the pin or net names, a collection of pins or nets, or a combination of patterns and a collection.
	The `-of_objects` parameter supports `get_designs` and `get_lib_cells` as valid collection objects.
	The use model is as follows:
	`get_cells -of_objects [get_designs object_list]`
	`get_cells -of_objects [get_lib_cells object_list]`

`patterns`	Specifies patterns for instance names. Patterns include the `*` and `?` wildcard characters and collections of instances. The `get_cells` command returns a collection of instances that match the patterns. If no such instance is found, an empty collection is returned.
`-quiet`	Suppresses all error and warning messages generated when the `get_cells` command is run.

`-regexp`	Treats the specified patterns as a regular expression patterns.
	When this parameter is specified, the given patterns `start` and `end` are assumed to be matching with the start and end part of the complete object name.
	For example, the following can be used to retrieve the hierarchical instance `Top/block1/block2/FF1`:
	`get_cells Top/*/*/FF1`
	`get_cells -regexp Top/.*/FF1`
	However, the following command will display an error :
	`get_cells Top/*/FF1`
	If this parameter is specified with `-hier` parameter, the given pattern will be matched with the complete hierarchical name of objects instead of only the flat name and all the successful matches are included in the result.
	For example, if there is a top level hierarchical instance `Top` and it contains another sub-instance called `FF1`, then either of these can be used to retrieve `Top/FF1` instance:
	`get_cells -hier FF1`
	`get_cells -hier -regexp Top/FF1`
	However, the following commands will display an error :
	`get_cells -hier Top/FF1`
	`get_cells -hier -regexp FF1`

Example

- The following command searches for pins within the hierarchy limited at `B`:
  `get_cells -hsc @ B@*`
- The following command searches for cells with set pattern and filter:
  `get_cells "o*" -filter "@ref_lib_cell_name != FD2 && @is_black_box==false"`

Related Information

- `get_libs`
- `get_lib_cells`
- `get_pins`

get_clocks

```
get_clocks
[-help]
[-filter expr ]
[-regexp]
[-nocase]
[-quiet]
[ patterns ]
```

Returns a collection of clocks whose names match the supplied patterns, filtered by the filter expression.

With no filter expression, the `get_clocks` command reports all clocks that match *patterns*, including inactive clocks.

Parameters

`-help`	Prints out the command usage.
`-filter expr`	Filters the collection with the specified expression. For any clocks that match the pattern, the software evaluates the expression based on the attribute of the clock. You can filter clocks using any of the clock properties supported through the `get_property` command. For details on how to create filter expressions, refer to the `filter_collection` command.
`-nocase`	Specifies that pattern matching is not case sensitive. **Note:** You must specify `-regexp` in order to use this parameter.
patterns	Specifies patterns for clock names. Patterns include the `*` and `?` wildcard characters, and collections of instances and clocks. This can either be a single *pattern* or a list of *patterns*
`-quiet`	Suppresses all the error and warning messages generated when the `get_clocks` command is run.
`-regexp`	Treats the specified patterns as regular expression patterns. *Default:* Treats the specified pattern as a wild card

Example

- The following example returns all clocks beginning with the name `CK`:
  ```
  get_clocks CK*
  ```
- The following example returns a collection of all clocks beginning with the name CK and Clk:
  ```
  get_clocks [ list CK* Clk* ]
  ```
- The following example returns a collection of all active clocks beginning with the name CK:
  ```
  get_clocks CK* -filter {is_active == true}
  ```

Related Commands

- `all_clocks`
- `create_clock`
- `get_pins`
- `get_ports`
- `report_clocks`

get_designs

```
get_designs
[-help]
[-quiet]
patterns
```

Creates a collection of modules and assigns this collection to a variable or pass it as an argument to an another command. The software uses this information during timing analysis.

Parameters

`-help`	Prints out the command usage.
patterns	Specifies patterns for module names. Patterns include the `*` and `?` wildcard characters and collections of modules. The `get_designs` command returns a collection of modules that match the patterns. If no such module is found, an empty collection is returned.
`-quiet`	Suppresses all error and warning messages generated when the `get_design` command is run.

Example

- The following command returns a complete list of modules:
  ```
  get_designs *
  ```

get_generated_clocks

```
get_generated_clocks
[-help]
[-filter expr]
[-regexp | -exact]
[-nocase]
[patterns]
```

Creates a collection of generated clocks whose name matches the supplied pattern list. Assign this collection to a variable, or pass it as an argument to another command.

Parameters

`-help`	Prints out the command usage.
`-exact`	Performs exact pattern matching against the object name, instead of wild card or regular expression matching. Use this parameter when the object name includes special characters such as `*`, `?`, and `+`. **Note:** If you specify this parameter, you cannot specify `-regexp`.
`-filter expr`	Filters the collection with the specified expression. For any clocks that match the pattern, the software evaluates the expression based on the attribute of the clock. For details on how to create filter expressions, refer to the `filter_collection` command.
`-nocase`	Specifies that pattern matching is not case sensitive.
patterns	Specifies patterns for generated clock names. Patterns include the `*` and `?` wildcard characters, and collections of generated clocks.
`-regexp`	Treats the specified pattern as a regular expression. *Default:* Treats the specified pattern as a wild card **Note:** If you specify this parameter, you cannot specify `-exact`.

Examples

- The following command returns the collection of all the generated clocks present in the design whose name starts with "CK":

```
get_generated_clocks CK*

CKGEN1 CKGEN2 CKGEN3
```

get_lib_arcs

```
get_lib_arcs
[-help]
{ [-to to_lib_pins] [-from from_lib_pins]
  | -of_objects {lib_cell_list | timing_arcs} }
[-filter expr]
[-quiet]
```

Creates a collection of library timing arcs. Assign these library timing arcs to a variable, or pass them to another command. If no objects match the criteria, an empty collection is returned.

Parameters

`-help`	Prints out the command usage.
`-filter expr`	Filters the collection with the specified expression. For any library arcs that match the pattern, the software evaluates the expression based on the attribute of the library arc. For details on how to create filter expressions, refer to the `filter_collection` command.
`-from from_lib_pins`	Creates a collection of all library arcs that originate from the specified library pins. You can specify a pattern for the pin name, a collection of pins, or a pattern and a collection. If you specify this parameter with the `-to` parameter, the software creates a collection of all library arcs that exist between the "to" and "from" library pins. **Note:** You cannot specify this parameter with the `-of_objects` parameter.
`-of_objects {lib_cell_list | timing_arcs}`	Creates a collection of all library arcs that are associated with the specified library cells or timing arcs. You can specify a list of library cells or a list of timing arcs, but you cannot specify a list that contains both library cells and timing arcs. You can specify a pattern for the library cell or timing arc name, a collection of library cells or timing arcs, or a pattern and a collection. **Note:** You cannot specify this parameter with the `-to` or `-from` parameters.
`-quiet`	Suppresses all error and warning messages generated when the `get_lib_arcs` command is run.
`-to to_lib_pins`	Creates a collection of all library arcs that terminate at the specified library pins. You can specify a pattern for the pin name, a collection of pins, or a pattern and a collection. If you specify this parameter with the `-from` parameter, the software creates a collection of all library arcs that exist between the "to" and "from" library pins. **Note:** You cannot specify this parameter with the `-of_objects` parameter.

Examples

- The following command reports the timing type off all the timing arcs of the flop 'DFF':

  ```
  get_property [get_lib_arcs -of_objects DFF] timing_type
  ```

  Sample output:

  ```
  minimum_period minimum_period min_pulse_width min_pulse_width setup_rising hold_rising rising_edge rising_edge
  ```

Related Information

- `report_lib_arcs`
- `get_lib_cells`
- `get_lib_pg_pins`
- `get_lib_pins`
- `get_libs`

get_lib_cells

```
get_lib_cells
[-help]
[-filter expr]
[-regexp]
[-nocase]
[-quiet]
{pattern_list | -of_objects object_list}
```

Creates a collection of library cells from the loaded libraries whose name matches the supplied pattern list. Assign these library cells to a variable or pass them to another command. If no objects match the criteria, an empty collection is returned.

Parameters

`-help`	Prints out the command usage.
`-filter expr`	Filters the collection with the specified expression. For any library cells that match the pattern, the software evaluates the expression based on the attribute of the library cell. You can filter library cells using any of the library cell properties supported through the `get_property` command. For details on how to create filter expressions, refer to the `filter_collection` command.
`-nocase`	Specifies that pattern matching is not case sensitive. **Note:** You must specify `-regexp` in order to use this parameter.
`-of_objects object_list`	Creates a collection of all library cells associated with the specified library pins or cells or libs. You can specify a list of library pins or a list of cells or a list of libs, but you cannot specify a list that contains mixed of library pins and cells and libs. You can specify patterns for the library pin or cell or lib names, a collection of library pins or cells or libs, or a combination of patterns and a collection.
`pattern_list`	Returns only those libraries in the collection whose name matches one or more of the patterns specified in the `pattern_list` argument. The characters `?` and `*` are wild cards. The `?` character matches any one character while the `*` character matches zero or more characters. If a library matches multiple patterns, it appears multiple times in the collection. A pattern can have only one hierarchical separator character. The left part of the pattern is the library name pattern and the right part is the cell name pattern. If there is no separator character, the pattern is compared to cell names in all the libraries. A pattern can have only one hierarchical separator character, dividing the pattern into one or two parts. If the hierarchical separator is `/`, the alternatives are: • `cell_name_pattern` Selects all cells whose name matches the `cell_name_pattern` from all libraries. • `lib_name_pattern/cell_name_pattern` Selects all cells whose name matches the `cell_name_pattern` from all libraries whose name matches the `lib_name_pattern`.
`-quiet`	Suppresses all error and warning messages generated when the `get_lib_cells` command is run.
`-regexp`	Treats the specified patterns as regular expression patterns. *Default:* Treats the specified pattern as wildcard

Examples

- The following command prints all cells for all libraries:
  ```
  get_lib_cells *
  ```
- The following command prints a collection with all cells in the `lca500kv` lib library:
  ```
  get_lib_cells lca500kv/*
  ```
- The following command prints a collection with all cells whose names starts with `AN` in all libraries starting with `lca`:
  ```
  get_lib_cells lca*/AN*
  ```
- The following command disables delay arcs on the `EF11L050032A` library cell. The `set_disable_cell_timing` command effects all instantiations of the

specified cell, and lets you disable delay arcs from and to internal pins.

```
set_disable_timing -from WACLK -to RBO1 [get_lib_cells {EF11L050032A}]
```

Related Information

- get_cells
- get_libs
- get_lib_pins

get_lib_pg_pins

```
get_lib_pg_pins
[-help]
[-filter expr]
{-of_objects object_list}
```

Returns a collection of library-level PG pins from the loaded libraries. An empty collection is returned if no PG pin definitions are found in the loaded library.

Parameters

`-help`	Prints out the command usage.
`-filter expr`	Filters the collection with the specified expression. For any library pg pins that match the specified expression, the software evaluates the expression based on the attribute of the library pg pin. You can filter library pg pins along with the `-of_objects` parameter - using the library pg pin properties of `lib_pg_pin` object type (supported through the `get_property` command). For details on how to create filter expressions, refer to the `filter_collection` command.

`-of_objects object_list`	Creates a collection of all library PG pins associated with the specified library cells.
	The `-of_objects` parameter supports `get_lib_cells` as valid objects.
	For example,
	`get_lib_pg_pins -of_objects [get_lib_cells sc9_cmos32lp/DFFQ_X1M] VDD VSS`
	`get_lib_pg_pins *`
	`**ERROR: (TCLCMD-1420): Currently only -of_objects option is supported`

Examples

- The following command gets the collection of all the library-level pg pins of the specified library cell:
  `get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/AO22X]`
- The following command filters the collection of all library-level pg pins of the library cell to return only the library pg pins containing the string name defined with the lib_pg_pin name attribute:
  `get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/AO22X] -filter "name =~ *VDD*"`

Related Information

- `get_lib_arcs`
- `get_lib_cells`
- `get_lib_pins`
- `get_libs`

get_lib_pins

```
get_lib_pins
[-help]
[-filter expr]
[-regexp]
[-nocase]
[-quiet]
{pattern_list | -of_objects object_list}
```

Creates a collection of library cell pins from the loaded libraries whose name matches the supplied pattern list. Assign these library cell pins to a variable or pass them to another command. If no objects match the criteria, an empty collection is returned.

Parameters

`-help`	Prints out the command usage.
`-filter expr`	Filters the collection with the specified expression. For any library cell pins that match the pattern, the software evaluates the expression based on the attribute of the library cell pin. You can filter library pins using any of the library pin properties supported through the `get_property` command. For details on how to create filter expressions, refer to the `filter_collection` command.
`-nocase`	Specifies that pattern matching is not case sensitive. **Note:** You must specify `-regexp` in order to use this parameter.
`-of_objects object_list`	Creates a collection of all library pins associated with the specified library cells or pins. You can specify a list of cells or a list of pins, but you cannot specify a list that contains both cells and pins. You can specify patterns for the cell or pin names, a collection of cells or pins, or a combination of patterns and a collection.
pattern_list	Returns only those libraries in the collection whose name matches one or more of the patterns specified in the *pattern_list* argument. The characters `?` and `*` are wild cards; The `?` character matches any one character while the `*` character matches zero or more characters. If a library cell matches multiple patterns, it appears multiple times in the collection. A pattern can have only two hierarchical separator characters, dividing the pattern into one, two, or three parts. If the hierarchical separator is `/`, the alternatives are: • *pin_name_pattern* Selects all pins whose name matches the *pin_name_pattern* from all cells in the libraries. • *cell_name_pattern/pin_name_pattern* Selects all pins whose name matches the *pin_name_pattern* from all cells whose name matches the *cell_name_pattern* in all libraries. • *lib_name_pattern/cell_name_pattern/pin_name_pattern* Select all pins whose name matches the *pin_name_pattern* from all cells whose name matches the *cell_name_pattern* from all libraries whose name matches the *lib_name_pattern*.
`-quiet`	Suppresses all error and warning messages generated when the `get_lib_pins` command is run.
`-regexp`	Treats the specified patterns as a regular expression patterns. *Default:* Treats the specified pattern as a wild card

Examples

- The following command returns all pins in all cells for all libraries:
  `get_lib_pins *`
- The following command returns a collection with all pins for cell `AO1` in any library:
  `get_lib_pins AO1/*`
- The following command returns a collection with all pins whose name starts with `Z` or `Q` for the `AO1` cell in any library: `get_lib_pins {AO1/Z* AO1/Q*}`
- The following command returns a collection with all pins for the `AO1` cell in the `lca500kv` lib library:

```
get_lib_pins lca500kv/AO1/*
```

- The following command returns a collection with all pins starting with `Z` in all cells whose names starts with `AN` in all libraries starting with `lca`:

```
get_lib_pins lca*/AN*/Z*
```

Related Information

- get_libs
- get_lib_cells

get_libs

```
get_libs
[-help]
[-filter expr]
[-quiet]
[-regexp]
[-nocase]
{-of_objects object_list | pattern_list}
```

Creates a collection of loaded libraries whose name matches those specified in the pattern list. Assign these libraries to a variable or pass them to another command. If no objects match the criteria, an empty collection is returned.

Parameters

`-help`	Prints out the command usage.
`-filter expr`	Filters the collection with the specified expression. For any libraries that match the pattern, the software evaluates the expression based on the attribute of the library. You can filter libraries using any of the library properties supported through the `get_property` command. For details on how to create filter expressions, refer to the `filter_collection` command.
`-nocase`	Specifies that pattern matching is not case sensitive. **Note:** You must specify `-regexp` in order to use this parameter.
`-of_objects object_list`	Creates a collection of libraries that belong to the library cells in a given object list.
pattern_list	Returns only those libraries in the collection whose name matches one or more of the patterns specified in the *pattern_list* argument. The characters `?` and `*` are wild cards; The `?` character matches any one character while the `*` character matches zero or more characters. If a library matches multiple patterns, it appears multiple times in the collection.
`-quiet`	Suppresses all error and warning messages generated when the `get_lib_cells` command is run.
`-regexp`	Treats the specified patterns as a regular expression patterns. *Default:* Treats the specified pattern as a wild card

Examples

- The following command displays a list of loaded libraries starting with a or l:
  ```
  get_libs {a* l*}
  ```
- The following command queries all libraries beginning with the letter `l`:
  ```
  get_libs l*
  ```
- The following command creates a collection of libraries that are associated with the library cells in the specified object list:
  ```
  get_libs -of_objects [get_lib_cells *]
  ```

Related Information

- `get_lib_cells`
- `get_lib_pins`

get_nets

```
get_nets
[-help]
[-hierarchical]
[-hsc char]
[-filter expr]
[-regexp]
[-nocase]
[-quiet]
[patterns | -of_objects object_list]
```

Creates a collection of nets in the current design whose name matches the supplied pattern list. Assign this collection to a variable or pass it as an argument to an another command. If no such net is found, an empty collection is returned.

When you use the `get_nets` command to specify hierarchical nets, the software matches the specified pattern and returns the top level net connected to it.

Parameters

`-help`	Prints out the command usage.
`-filter expr`	Filters the collection with the specified expression. For any nets that match the pattern, the software evaluates the expression based on the attribute of the net. You can filter nets using any of the net properties supported through the `get_property` command. For details on how to create filter expressions, refer to the `filter_collection` command.
`-hierarchical`	Specifies that the patterns should match hierarchically. If the hierarchical name of a net at any hierarchical level matches the patterns, it is included in the result.
`-hsc char`	Specifies the hierarchical delimiter for patterns.
`-nocase`	Specifies that pattern matching is not case sensitive. **Note:** You must specify `-regexp` in order to use this parameter.
`-of_objects object_list`	Creates a collection of all nets associated with the specified cells, pins, or ports. You can specify a list of cells, a list of pins, or a list of ports, but you cannot specify a list that contains cells, pins, and ports together. You can specify patterns for the cell, pin, or port names, a collection of cells, pins, or ports, or a combination of patterns and a collection.
`patterns`	Specifies patterns for net names. Patterns include the `*` and `?` wildcard characters and collections of nets.
`-quiet`	Suppresses all error and warning messages generated when the `get_nets` command is run.
`-regexp`	Treats the specified patterns as regular expression patterns. *Default:* Treats the specified pattern as wildcard

Examples

- The following command returns all nets within the hierarchy limited at `A`:
  ```
  get_nets -hsc @ A@*
  ```
- The following command returns all patterns that match hierarchy and identifies nets to be excluded from optimization.
  ```
  get_nets * -hierarchical -filter "@dont_touch== true"
  ```

Related Information

- `get_pins`

get_object_name

```
get_object_name
[-help]
collection
```

Returns the name of the object(s) contained in the specified collections.

By default, the command hides escape characters in the output. To enable printing of escape characters in the output, you can set `timing_enable_get_object_escaped_name_backward_compatible` global variable to `true`.

Parameters

`-help`	Prints out the command usage.
`collection`	Specifies multiple object collection that contain the objects.

Examples

- The following command returns a tcl list of names which can match the collection inputs:
  ```
  get_object_name [get_cells and*]
  ```
- The following command defines a variable called `endpoint_list` to store endpoints with the ten worst slacks:
  ```
  set endpoint_list [get_object_name [get_property \

        [report_timing -collection -max_points 10] capturing_point]]
  ```

Related Commands

- `query_objects`

get_path_groups

```
get_path_groups
[-help]
[-regexp]
[-nocase]
patterns
```

Creates a collection of path groups whose name matches those specified in the pattern list. Assign these path groups to a variable, or pass them to another command. If no objects match the criteria, an empty collection is returned.

Parameters

`-help`	Prints out the command usage.
`-nocase`	Specifies that pattern matching is not case sensitive.
patterns	Creates a collection of path groups whose name matches one or more of the specified patterns. Patterns include the `*` and `?` wildcard characters and collections of path groups. If a path group matches multiple patterns, it appears multiple times in the collection.
`-regexp`	Treats the specified patterns as regular expression patterns. *Default:* Treats the specified pattern as wildcard

Examples

- The following command returns a collection of all the path groups present in the design:
  ```
  get_path_groups *
  ```

  Sample output:
  ```
  reg2reg reg2gate
  ```

Related Information

- `group_path`
- `reset_path_group`

get_pg_nets

```
get_pg_nets
[-help]
[-filter expr]
[-of_objects object_list]
```

Creates a collection of netlist-level instance PG nets of leaf cells/pins in the current design. An empty collection is returned if no PG net definitions/connections are found in the design netlist.

Parameters

`-help`	Prints out the command usage.
`-filter` *expr*	Specifies the filters for the collection with the specified expression. For any pg nets that match the pattern, the software evaluates the expression based on the attribute of the net. You can filter pg nets along with the `-of_objects` parameter - using any of the pg net properties of object type `pg_net` (supported through the `get_property` command). For details on how to create filter expressions, refer to the `filter_collection` command.

`-of_objects` *object_list*	Creates a collection of all the PG nets, or PG nets associated with the specified cells or PG pins.
	You can specify a list of cells or a list of pg pins, but you cannot specify a list that contains cells and pg pins together.
	The `-of_objects` parameter supports `get_cells` and `get_pg_pins` as valid objects.
	For example,
	`get_pg_nets -of_objects [get_cells iA/out_reg] VSS VDD2`
	`get_pg_nets -of_objects [get_pg_pins -of_objects [get_cells iA/out_reg]] VSS VDD2`
	`get_pg_nets *`

Examples

- The following command filters the collection of all pg nets of the cell instance object to return only the pg nets containing the string name defined with the pg_net hierarchical_name attribute:
  ```
  get_pg_nets -of_objects [get_cells iA/out_reg] -filter "hierarchical_name =~ *VDD*"
  ```
- The following command gets the collection of all pg nets of the cell instance object:
  ```
  get_pg_nets -of_objects [get_cells iA/out_reg]
  ```
- The following command gets the collection of all pg nets of the pg_pins of the cell instance object:
  ```
  get_pg_nets -of_objects [get_pg_pins -of_objects [get_cells iA/out_reg]]
  ```

Related Information

- `get_pg_pins`
- `get_pins`

get_pg_pins

```
get_pg_pins
[-help]
[-filter expr]
[-of_objects object_list]
```

Returns a collection of netlist-level instance PG pins in the current design. An empty collection is returned if no PG pin definitions are found in the design netlist.

Parameters

`-help`	Prints out the command usage.
`-filter expr`	Specifies the filters for the collection with the given expression. For any pg pins that match the pattern, the software evaluates the expression based on the attribute of the pin. You can filter pg pins along with the `-of_objects` parameter - using the pg pin properties of object type `pg_pin` (supported through the `get_property` command). For details on how to create filter expressions, refer to the `filter_collection` command.

`-of_objects object_list`	Specifies the collection of all the pins associated with the specified cells or nets.
	The `-of_objects` parameter supports `get_cells` as valid objects.
	For example:
	`get_pg_pins -of_objects [get_cells iA/out_reg] iA/out_reg/VSS iA/out_reg/VDD`
	`get_pg_pins iA/out_reg/VDD`
	`**ERROR: (TCLCMD-1420): Currently only -of_objects option is supported.`

Examples

- The following command filters the collection of all the pg_pins of the cell instance object to return only the pg pins containing the string name defined with the pg_pin hierarchical_name attribute:
  ```
  get_pg_pins -of_objects [get_cells iA/out_reg] -filter "hierarchical_name =~ *VDD*"
  ```
- The following command gets the collection of all pg pins of the cell instance object:
  ```
  get_pg_pins -of_objects [get_cells iA/out_reg]
  ```

Related Information

- `get_pg_nets`
- `get_pins`

get_pins

```
get_pins
[-help]
[-hierarchical]
[-hsc char]
[-filter expr]
[-leaf]
[-regexp]
[-nocase]
[-quiet]
{patterns | -of_objects object_list}
```

Creates a collection of instance pins whose name matches the supplied pattern list. Assign this collection to a variable or pass it as an argument to another command. If no such pin is found, an empty collection is returned.

Parameters

`-help`	Prints out the command usage.
`-filter expr`	Filters the collection with the specified expression. For any pins that match the pattern, the software evaluates the expression based on the attribute of the pin. If the expression is set to true, the pin is included in the result. You can filter pins using any of the pin properties supported through the `get_property` command. For details on how to create filter expressions, refer to the `filter_collection` command.
`-hierarchical`	Specifies that patterns should match hierarchically. If the hierarchical name of a pin at any hierarchical level matches the patterns, it is included in the result.
`-hsc char`	Specifies the hierarchical delimiter for patterns.
`-leaf`	Returns leaf pins only. When the `-leaf` parameter is not used, the `get_pins` command returns the hierarchical pins present on a net along with the flat pins at the same hierarchical level as the specified object.
`-nocase`	Specifies that pattern matching is not case sensitive. **Note:** You must specify `-regexp` in order to use this parameter.
`-of_objects object_list`	Creates a collection of all pins associated with the specified cells or nets. You can specify a list of cells or a list of nets, but you cannot specify a list that contains both cells and nets. You can specify patterns for the cell or net names, a collection of cells or nets, or a combination of patterns and a collection.
patterns	Specifies patterns for pin names. Patterns include the `*` and `?` wildcard characters and collections of pins.
`-quiet`	Suppresses all error and warning messages generated when the `get_pins` command is run.
`-regexp`	Treats the specified patterns as regular expression patterns. *Default:* Treats the specified pattern as wildcard

Examples

- The following command returns all instance pins within the hierarchy limited at `A`:

  ```
  get_pins -hsc @ A@*
  ```

- The following command filters the collection for `pin_direction` pin attribute.

  ```
  get_pins "in*" -filter "@pin_direction == in"
  ```

Related Information

- create_clock
- get_cells
- get_ports
- get_pg_nets
- get_pg_pins
- get_property

get_ports

```
get_ports
[-help]
[-filter expr]
[-regexp]
[-nocase]
[-quiet]
[patterns | -of_objects object_list]
```

Creates a collection of ports whose name matches the supplied pattern list. Assign this collection to a variable or pass it as an argument to another command.

Note: The `get_ports` command always creates a collection of top level ports irrespective of the current instance set using the `current_instance` command.

Parameters

`-help`	Prints out the command usage.
`-filter expr`	Filters the collection with the specified expression. For any ports that match the pattern, the software evaluates the expression based on the attribute of the port. If the expression is set to true, the port is included in the result. You can filter ports using any of the port properties supported through the `get_property` command. For details on how to create filter expressions, refer to the `filter_collection` command.
`-nocase`	Specifies that pattern matching is not case sensitive. **Note:** You must specify `-regexp` in order to use this parameter.
`-of_objects object_list`	Creates a collection of all ports associated with the specified nets. You can specify patterns for the net names, a collection of nets, or a combination of patterns and a collection.
patterns	Specifies patterns for port names. Patterns include the `*` and `?` wildcard characters and collections of ports.The command returns a collection of ports that match the patterns. If no such port is found, an empty collection is returned.
`-quiet`	Suppresses all error and warning messages generated when the `get_ports` command is run.
`-regexp`	Treats the specified patterns as regular expression patterns. *Default:* Treats the specified pattern as wildcard.

Examples

- The following example returns all ports beginning with the name `inb`:
  ```
  get_ports inb*
  ```
- The following command filters the collection for `port_direction` port attribute:
  ```
  get_ports "mode*" -filter {port_direction == in}
  ```

Related Commands

- `get_clocks`
- `get_pins`
- `create_clock`
- `get_cells`
- `get_property`

get_power

```
get_power
[-help]
[-outfile filename]
[-nets nets_list | -pins pins_list | -instances instances_list]
[-attribute attributes_list]
[-include_unit]
[-tcl_list ]
[-pg_net {pg_net_name_list | all}]
```

Queries various power related properties of design objects like nets, pins, and instances. Using this command, you can retrieve power attributes, such as internal power, switching power, leakage power, total power, and toggle rate. This command is very useful for writing custom scripts.

Note: This command outputs toggle rates as N.A. if the clock domain information for a given net/pin/instance is not found.

Use this command after running the `report_power` or `restore_power_database` command.

Note: The `-nets`, `-pins`, and `-instances` parameters are mutually exclusive. You must specify only one of the parameters.

Parameters

`-help`	Prints out the command usage.
`-attribute attributes_list`	Specifies a list of power attributes. If an attribute is not available for the specified design object, the software returns its value as NA. See List of Power Attributes for a mapping of the list of available attributes with the design objects. This is a required parameter.
`-include_unit`	Specifies to include the units for the power attributes.
`-instances instances_list`	Specifies a list of instance names. The `-nets`, `-pins`, and `-instances` parameters are mutually exclusive. You must specify one, and only one, of the parameters. **Note:** You can use the `get_cells` command to determine the power attributes for a collection of instances.
`-nets nets_list`	Specifies a list of net names. The `-nets`, `-pins`, and `-instances` parameters are mutually exclusive. You must specify one, and only one, of the parameters. **Note:** You can use the `get_nets` command to determine the power attributes for a collection of nets.
`-outfile filename`	Specifies the output file into which the power related properties are to be written. The `-tcl_list` and `-outfile` parameters are mutually exclusive; you cannot specify them together.
`-pg_net {pg_net_name_list | all}`	
	Specifies to query various power-related properties of a specified power net for an instance. This parameter supports the argument all to query properties of all power nets connected to the instance. You must use the `-instances` parameter in conjunction with the `-pg_net` parameter. The `-pg_net` parameter supports the following attributes: `switching_power`, `internal_power`, `leakage_power`, `total_power`, and `dynamic_power`.
`-pins pins_list`	Specifies a list of pin names. The `-nets`, `-pins`, and `-instances` parameters are mutually exclusive. You must specify one, and only one, of the parameters. **Note:** You can use the `get_pins` command to determine the power attributes for a collection of pins.
`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. The `-tcl_list` and `-outfile` parameters are mutually exclusive; you cannot specify them together.

The following power attributes can be queried for the design objects:

List of Power Attributes with Attribute Description

- `switching_power (net, inst)`: switching power of a net or instance
- `internal_power (inst)`: internal power of an instance
- `dynamic_power (inst)`: switching and internal power of an instance
- `leakage_power (inst)`: leakage power of an instance
- `total_power (inst)`: total power of an instance
- `toggle_rate (net, pin, inst)`: Toggle rate - Number of toggles in the specified VCD/TCF in the duration
- `duty_cycle (net, pin)`: Static probability of the signal to stay high
- `ref_clock (net, pin, inst)`: Reference clock name (Fastest clock for multi clock domains)
- `ref_clock_period (net, pin, inst)`: Reference clock period
- `transition_density (net, pin)`: toggle_rate/ref_clock_period or toggle_rate*ref_clock_frequency
- `loading_cap (net, pin, inst)`: Loading capacitance
- `voltage (net, pin, inst)`: Operating voltage for a net or pin. For MSMV instance, this attribute can be used to get breakdown of domain specific power.
- `rise_slew (net, pin)`: Rise slew
- `fall_slew (net, pin)`: Fall slew
- `glitch_rate (net, pin)`: Glitch rate - Number of glitches per duration. Applies only to the vector-driven flow.
- `glitch_power (net, pin, inst)`: Glitch power. Applies only to the vector-driven flow.
- `activity_source (net, pin)`: Source of activity annotation. The options are: propagated, user_annotated, and constant.
- `pin_direction (pin)`: [input | output | inout]
- `cell_type (inst)`:Lists cell type as memory, standard cell, IO, sequential, and so on
- `all_attributes (net, pin, inst)`: Lists all related power attributes for the specified design object.

Examples

- The following command returns the specified power attributes for the net clock1:

  ```
  get_power -nets clock1 -attribute {switching_power internal_power toggle_rate}
  ```

  The following is the sample output:

  ```
  clock1 0.04 NA 2
  ```

- The following command returns the specified power attributes for all nets that match the specified pattern `master*`:

  ```
  get_power -nets [get_nets master*] -attribute {switching_power toggle_rate activity_source duty_cycle} -include_unit
  ```

  The following is the sample output:

  ```
  masterin[1] 0.001mW 0.2 user_annotated 0.5
  masterin[2] 0.002mW 0.2 user_annotated 0.5
  masterin[3] 0.003mW 0.1 propagated 0.5
  masterin[4] 0.005mW 1 constant 1
  ```

- The following command returns power-related properties of all power nets connected to the instance `clk_buf_1`:

  ```
  get_power -pg_net all -instances clk_buf_1 -attribute {switching_power internal_power leakage_power}
  ```

  The following is the sample output:

  ```
  clk_buf_1 VDD 0.002 0.001 0.000001
  clk_buf_1 VDDL 0.0001 0.0001 0.0000001
  ```

Related Information

- `get_activity`
- `set_power_analysis_mode`
- `report_power`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*

get_property

```
get_property
[-help]
var_name
property
[-clock clock_name]
[-view view_name]
[-quiet]
```

Retrieves the attribute value for the specified object(s) property.

You can query the following:

- Cell Properties
- Clock Properties
- Design Properties
- Library Properties
- Library Cell Properties
- Library Pin Properties
- Library Pin Properties for Power/Ground Objects
- Library Timing Arc Properties
- Net Properties
- Net Properties for Power/Ground Objects
- Path Group Properties
- Pin Properties
- Pin Properties for Power/Ground Objects
- Port Properties
- Timing Arc Properties
- Timing Path Properties
- Timing Point Properties
- si_victim Properties
- si_attacker Properties
- User-Defined Properties

The `get_property` command supports the `report_precision` global variable settings.

The `get_property` command supports DRV constraints (`set_min_fanout`, `set_max_fanout`, `set_min_transition`, `set_max_transition`, `set_min_capacitance`, and `set_max_capacitance`) applied on library pins.

The `get_property` command does not work with partial property names. You are now required to specify the full name of the property.

Parameters

`-help`	Prints out the command usage.
`-clock` *clock_name*	Specifies the clock for which to query certain object properties.
property	Specifies the property for which to retrieve the attribute value.
`-quiet`	Suppresses all error and warning messages generated when the `get_property` command is run.
var_name	Specifies a pointer to a collection containing the object. **Note:** If the collection contains more than one object, the `get_property` command returns the property values for each object in the specified collection.
`-view` *view_name*	Specifies the analysis view in which to query the object attribute. You can query properties only for the following objects in an analysis view: library cells, library pins, ports, cells, pins, timing arcs, and nets. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode. **Note**: The properties (described in the tables below), which are related to slack, delay, arrival, slew, transition, capacitance, uncertainty and latency, depend on the specified view. When the `-view` parameter is not specified, the maximum (or minimum) value for all the views is returned.

Cell Properties

`area`	Returns the area of the cell. If the cell is hierarchical, also includes the net area. *Return type:* float
`cell_footprint`	Returns the clock gating integrated cells present in a library. When specified, the software returns a value that is used to determine the integrated cell functionality to be used by clock gating tools. For example, `get_property [get_lib_cells 1.08_CTS_HT.db/CTB_F18_HT] cell_footprint` *Return Type*: String
`clock_gating_integrated_cell`	Returns the clock gating integrated cells present in a library cell. For example, `get_property [get_lib_cells CTS_HT.db/ICG_F4_HVT] clock_gating_integrated_cell` *Return Type*: String
`clock_gating_integrated_cell`	Returns the string specified for the clock_gating_integrated_cell attribute in the library for the corresponding cell. *Return Type*: Float For example, `get_property [get_cells <cell name>] clock_gating_integrated_cell`
`early_cell_check_derate_factor`	Returns the derating factor for early paths specified through the `set_timing_derate` command with the `-early` parameter. *Return type:* float
`early_cell_check_mean_derate_factor`	Returns the derating factor for early paths on the instance – when the `set_timing_derate -mean` parameter is specified. *Return type:* float
`early_clock_check_incremental_derate_factor`	Returns the early clock path instances' incremental derating factor specified through the `set_timing_derate -incremental_adjust` parameter. *Return type:* float

`early_data_check_incremental_derate_factor`	Returns the early data path instance's incremental derating factor specified through the `set_timing_derate -incremental_adjust` parameter. *Return type:* float
`early_cell_check_sigma_derate_factor`	Returns the derating factor for early paths on the instance - when the `set_timing_derate -sigma` parameter is specified. *Return type:* float
`early_clk_cell_derate_factor`	Returns the derating factor for early clock paths specified through the `set_timing_derate` command with the `-early` parameter. *Return type:* float
`early_clk_cell_mean_derate_factor`	Returns the derating factor for early clock paths – when the `set_timing_derate -mean` parameter is specified. *Return type:* float
`early_clk_cell_sigma_derate_factor`	Returns the derating factor for early clock paths – when the `set_timing_derate -sigma` parameter is specified. *Return type:* float
`early_data_cell_derate_factor`	Returns the derating factor for early paths specified through the `set_timing_derate` command with the `-early` parameter. *Return type:* float
`early_data_cell_mean_derate_factor`	Returns the derating factor for early paths – when the `set_timing_derate -mean` parameter is specified. *Return type:* float
`early_data_cell_sigma_derate_factor`	Returns the derating factor for early paths – when the `set_timing_derate -sigma` parameter is specified. *Return type:* float
`early_fall_cell_check_derate_factor`	Returns the early cell check derating factor specified through the `set_timing_derate` command with the `-fall` parameter. *Return type:* float
`early_fall_cell_check_mean_derate_factor`	Returns the early cell check derating factor – when the `set_timing_derate -cell_check` and -mean parameters are specified. *Return type:* float
`early_fall_cell_check_sigma_derate_factor`	Returns the early cell check derating factor – when the `set_timing_derate -cell_check` and `-sigma` parameters are specified. *Return type:* float
`early_fall_clk_cell_derate_factor`	Returns the early clock path derating factor specified through the `set_timing_derate` command with the `-fall` parameter. *Return type:* float
`early_fall_clk_cell_mean_derate_factor`	Returns the early clock path derating factor – when the `set_timing_derate -fall` and `-mean` parameters are specified. *Return type:* float
`early_fall_clk_cell_sigma_derate_factor`	Returns the early clock path derating factor – when the `set_timing_derate -fall` and `-sigma` parameters are specified. *Return type:* float
`early_fall_clock_incremental_derate_factor`	Returns the early clock path instances' incremental derating factor specified through the `set_timing_derate -incremental_adjust` parameter - when the `-fall` parameter is specified. *Return type*: float

`early_fall_data_cell_derate_factor`	Returns the early data cell check derating factor – when the `set_timing_derate -fall` parameter is specified. *Return type:* float
`early_fall_data_cell_mean_derate_factor`	Returns the early data cell check derating factor – when the `set_timing_derate -fall` and `-mean` parameters are specified. *Return type:* float
`early_fall_data_cell_sigma_derate_factor`	Returns the early data cell check derating factor – when the `set_timing_derate -fall` and `-sigma` parameters are specified. *Return type:* float
`early_fall_data_incremental_derate_factor`	Returns the early data path instances' incremental derating factor specified through the `set_timing_derate -incremental_adjust` parameter - when the `-fall` parameter is specified. *Return type*: float
`early_rise_cell_check_derate_factor`	Returns the early cell check derating factor specified through the `set_timing_derate` command with the `-rise` and `-cell_check` parameter. *Return type:* float
`early_rise_cell_check_mean_derate_factor`	Returns the early cell check derating factor – when the `set_timing_derate -rise` and `-mean` parameters are specified. *Return type:* float
`early_rise_cell_check_sigma_derate_factor`	Returns the early cell check derating factor – when the `set_timing_derate -rise` and `-sigma` parameters are specified. *Return type:* float
`early_rise_clk_cell_derate_factor`	Returns the early clock cell check derating factor specified through the `set_timing_derate` command with the `-rise` parameter. *Return type:* float
`early_rise_clk_cell_mean_derate_factor`	Returns the early clock cell check derating factor – when the `set_timing_derate -rise` and `-mean` parameters are specified. *Return type:* float
`early_rise_clk_cell_sigma_derate_factor`	Returns the early clock cell check derating factor – when the `set_timing_derate -rise` and `-sigma` parameters are specified. *Return type:* float
`early_rise_clock_incremental_derate_factor`	Returns the early clock path instances' incremental derating factor specified through the `set_timing_derate -incremental_adjust` parameter - when the `-rise` parameter is specified. *Return type*: float
`early_rise_data_cell_derate_factor`	Returns the early data cell check derating factor specified through the `set_timing_derate` command with the `-rise` parameter. *Return type:* float
`early_rise_data_cell_mean_derate_factor`	Returns the early data cell check derating factor – when the `set_timing_derate -rise` and `-mean` parameters are specified. *Return type:* float
`early_rise_data_cell_sigma_derate_factor`	Returns the early data cell check derating factor – when the `set_timing_derate -rise` and `-sigma` parameters are specified. *Return type:* float
`early_rise_data_incremental_derate_factor`	Returns the early data path instances' incremental derating factor specified through the `set_timing_derate -incremental_adjust` parameter - when the `-rise` parameter is specified. *Return type*: float

`has_sdc_assertions`	Returns a value of `true`, if the specified cell has SDC assertions applied on it. For example, `get_property [get_cells <cell name>] has_sdc_assertions` *Return Type*: Boolean
`hierarchical_level`	Returns the hierarchical level of the specified cell. For example, `get_property [get_cell -hierarchical *] hierarchical_level` *Return Type*: Integer
`hierarchical_name`	Returns the complete hierarchical name of the cell. *Return type:* string
`instance_model`	Reports the model type of an instance. The Instance model type can be ecsmn, ccsn, cdb, udn, or nldm. In case a noise model does not exist, the software returns 'NA'. For example, `get_property [get_cells inst1/buf22 ] instance_model` *Return type:* string
`is_black_box`	Returns a value of `true` if the cell is a black box cell. A cell is considered as a black box if the "function" definition of the cell is missing from the library. *Return type:* boolean
`is_buffer`	Returns a value of `true` if the associated library cell is recognized as a buffer cell. For example, `get_property [get_cells mmu/IB1] is_buffer` *Return type:* boolean
`is_clock_gating_check`	Returns a value of true if the cell is gated. *Return type:* boolean
`is_combinational`	Returns a value of `true` if the cell is a combinational cell (not a sequential cell). *Return type:* boolean
`is_disable_timing`	Returns a value of `true` if the cell's timing has been disabled using the `set_disable_timing` command. *Return type:* boolean
`is_dont_touch`	Returns a value of `true` if the cell definition includes `dont_touch:true`. *Return type:* boolean
`is_fall_edge_triggered`	Returns a value of `true` if the cell is triggered by the falling edge of the clock. *Return type:* boolean
`is_hierarchical`	Returns a value of `false` if the cell is a leaf or library cell. *Return type:* boolean
`is_integrated_clock_gating_cell`	Returns a value of `true` if the cell is an integrated clock gating cell. *Return type:* boolean
`is_interface_timing`	Returns a value of true if a library cell has interface timing specified for that cell. *Return Type*: boolean

`is_unmapped`	Returns a value of true if the specified cell name does not have a defined library cell. For example, `get_property [get_cells <cell name>] is_unmapped` *Return Type*: Boolean
`is_memory_cell`	Returns a value of `true` if the cell is a memory cell. *Return type:* boolean
`is_negative_level_sensitive`	Returns a value of `true` if the cell is used as a negative level-sensitive sequential element, such as a negative level-sensitive latch. *Return type:* boolean
`is_pad_cell`	Returns a value of `true` if the Liberty library `is_pad_cell` attribute is set to `true` for this pad cell. *Return type:* boolean
`is_positive_level_sensitive`	Returns a value of `true` if the cell is used as a positive level-sensitive sequential element, such as a positive level-sensitive latch. *Return type:* boolean
`is_rise_edge_triggered`	Returns a value of `true` if the cell is triggered by the rising edge of the clock. *Return type:* boolean
`is_sequential`	Returns a value of `true` if the cell is a latch or flip-flop, or if the cell has sequential timing checks. *Return type:* boolean
`is_tristate`	Returns a value of `true` if the cell definition includes `three_state:true`. *Return type:* boolean
`late_cell_check_derate_factor`	Returns the derating factor for late paths specified through the `set_timing_derate` command with the `-late` parameter. *Return type:* float
`late_cell_check_mean_derate_factor`	Returns the derating factor for late paths – when the `set_timing_derate -late` and `-mean` parameters are specified. *Return type:* float
`late_cell_check_sigma_derate_factor`	Returns the derating factor for late paths – when the `set_timing_derate -late` and `-sigma` parameters are specified. *Return type:* float
`late_clk_cell_derate_factor`	Returns the derating factor for late clock paths specified through the `set_timing_derate` command with the `-late` parameter. *Return type:* float
`late_clk_cell_mean_derate_factor`	Returns the derating factor for late clock paths – when the `set_timing_derate -late` and `-mean` parameters are specified. *Return type:* float
`late_clk_cell_sigma_derate_factor`	Returns the derating factor for late clock paths – when the `set_timing_derate -late` and `-sigma` parameters are specified. *Return type:* float
`late_clock_check_incremental_derate_factor`	Returns the late clock path instance's incremental derating factor specified through the `set_timing_derate -incremental_adjust` parameter. *Return type*: float
`late_data_cell_derate_factor`	Returns the derating factor for late data paths specified through the `set_timing_derate` command with the `-late` parameter. *Return type:* float

late_data_cell_mean_derate_factor	Returns the derating factor for late data paths – when the set_timing_derate -late and -mean parameters are specified. For example, get_property [get_cells <cell name>] late_data_cell_mean_derate_factor *Return Type*: Float
late_data_cell_sigma_derate_factor	Returns the derating factor for late data paths – when the set_timing_derate -late and -sigma parameters are specified. *Return type:* float
late_data_check_incremental_derate_factor	Returns the late data path instances' incremental derating factor specified through the set_timing_derate -incremental_adjust parameter. *Return type*: float
late_fall_cell_check_derate_factor	Returns the late cell check derating factor specified through the set_timing_derate command with the -fall parameter. *Return type:* float
late_fall_cell_check_mean_derate_factor	Returns the late cell check derating factor – when the set_timing_derate -fall and -mean parameters are specified. *Return type:* float
late_fall_cell_check_sigma_derate_factor	Returns the late cell check derating factor – when the set_timing_derate -fall and -sigma parameters are specified. *Return type:* float
late_fall_clk_cell_derate_factor	Returns the late clock cell check derating factor specified through the set_timing_derate command with the -fall parameter. *Return type:* float
late_fall_clk_cell_mean_derate_factor	Returns the late clock cell check derating factor – when the set_timing_derate -fall and -mean parameters are specified. *Return type:* float
late_fall_clk_cell_sigma_derate_factor	Returns the late clock cell check derating factor – when the set_timing_derate -fall and -sigma parameters are specified. *Return type:* float
late_fall_clock_incremental_derate_factor	Returns the late clock path instances' incremental derating factor specified through the set_timing_derate -incremental_adjust parameter - when the -fall parameter is specified. *Return type*: float
late_fall_data_cell_derate_factor	Returns the late data cell check derating factor specified through the set_timing_derate command with the -fall parameter. *Return type:* float
late_fall_data_cell_mean_derate_factor	Returns the late data cell check derating factor – when the set_timing_derate -fall and -mean parameters are specified. *Return type:* float
late_fall_data_cell_sigma_derate_factor	Returns the late data cell check derating factor – when the set_timing_derate -fall and -sigma parameters are specified. *Return type:* float
late_fall_data_incremental_derate_factor	Returns the late data path instances' incremental derating factor specified through the set_timing_derate -incremental_adjust parameter - when the -fall parameter is specified. Return type: float

`late_rise_cell_check_derate_factor`	Returns the late cell check derating factor specified through the `set_timing_derate` command with the `-rise` parameter. *Return type:* float
`late_rise_cell_check_mean_derate_factor`	Returns the late cell check derating factor – when the `set_timing_derate -rise` and `-mean` parameters are specified. *Return type:* float
`late_rise_cell_check_sigma_derate_factor`	Returns the late cell check derating factor – when the `set_timing_derate -fall` and `-sigma` parameters are specified. *Return type:* float
`late_rise_clk_cell_derate_factor`	Returns the late clock cell check derating factor specified through the `set_timing_derate` command with the `-rise` parameter. *Return type:* float
`late_rise_clk_cell_mean_derate_factor`	Returns the late clock cell check derating factor – when the `set_timing_derate -rise` and `-mean` parameters are specified. *Return type:* float
`late_rise_clk_cell_sigma_derate_factor`	Returns the late clock cell check derating factor – when the `set_timing_derate -fall` and `-sigma` parameters are specified. *Return type:* float
`late_rise_clock_incremental_derate_factor`	Returns the late clock path instances' incremental derating factor specified through the `set_timing_derate -incremental_adjust` parameter - when the `-rise` parameter is specified. *Return type*: float
`late_rise_data_cell_derate_factor`	Returns the late data cell check derating factor specified through the `set_timing_derate` command with the `-rise` parameter. *Return type:* float
`late_rise_data_cell_mean_derate_factor`	Returns the late data cell check derating factor – when the `set_timing_derate -rise` and `-mean` parameters are specified. *Return type:* float
`late_rise_data_cell_sigma_derate_factor`	Returns the late data cell check derating factor – when the `set_timing_derate -rise` and `-sigma` parameters are specified. *Return type:* float
`late_rise_data_incremental_derate_factor`	Returns the late data path instances' incremental derating factor specified through the `set_timing_derate -incremental_adjust` parameter - when the `-rise` parameter is specified. *Return type*: float
`lib_cell`	Returns a collection of all the library cell objects associated with the specified library pins/cells. For example, `get_property [get_cells AAA] lib_cell -view VIEW1` *Return type*: collection
`name`	Returns the leaf-level name of the cell. *Return type:* string
`object_type`	Returns the object type `cell`. *Return type:* string
`pin_count`	Returns the total number of signal pins. *Return type:* integer

`ref_lib_cell_name`	Returns the name of reference module of the given cell. The reference module of the cell can be a Verilog module or a leaf library cell. *Return type:* string
`threshold_voltage_group`	Returns name of a group/category – that is specified based on the cell's threshold voltage (high or low) characteristics. For example, `get_property [get_lib_cells AND2_X1] threshold_voltage_group` *Return type:* string
`timing_model_type`	Returns the model type for a given cell or instance. The supported values are `abstracted`, `extracted`, and `qtm`. *Return type:* string
`x_coordinate_max`	Returns the maximum x coordinate of the cell. *Return type:* float
`x_coordinate_min`	Returns the minimum x coordinate of the cell. *Return type:* float
`y_coordinate_max`	Returns the maximum y coordinate of the cell. *Return type:* float
`y_coordinate_min`	Returns the minimum y coordinate of the cell. *Return type:* float

Cell Properties for Power/Ground Objects	
`is_always_on`	Returns a value of `true` if a cell instance is an always-on cell. *Return type*: boolean For example, `get_property [get_cells mmu/U1] is_always_on`
`is_inverter`	Returns a value of `true` if a cell instance is an inverter cell. *Return type*: boolean For example, `get_property [get_cells mmu/INV1] is_inverter`
`is_isolation_cell`	Returns a value of `true` if a cell instance is an isolation cell. *Return type*: boolean For example, `get_property [get_cells mmu/U1] is_isolation_cell`
`is_level_shifter`	Returns a value of `true` if a cell instance is a level-shifter cell. *Return type*: boolean For example, `get_property [get_cells mmu/U1] is_level_shifter`
`is_power_switch`	Returns a value of `true` if a cell instance is a power switch cell. Return type: boolean For example, `get_property [get_cells mmu/U1] is_power_switch`
`is_retention`	Returns a value of `true` if a cell instance is a retention cell. *Return type*: boolean For example, `get_property [get_cells mmu/U1] is_retention`
`level_shifter_type`	Returns the type of level-shifter (e.g., LH, HL, or HL_LH) that a cell instance is. *Return type*: string For example, `get_property [get_cells mmu/U1] level_shifter_type`

`pg_pins`	Returns a collection of the PG pins that are present in a cell instance. *Return type*: collection For example, `get_property [get_cells mmu/U1] pg_pins`
`power_switch_type`	Returns the type of power switch (e.g., coarse-grain, fine-grain) that a cell instance is. *Return type*: string For example, `get_property [get_cells mmu/U1] power_switch_type`
`std_cell_main_rail_name`	Returns the name of the main power rail driving the power pins of a cell instance that is the `pg_pin`, which has the `std_cell_main_rail` library attribute set as `true` for the instance cell. *Return type*: string For example, `get_property [get_cells mmu/U1] std_cell_main_rail_name`

Clock Properties

`child_clocks`	Returns the first level of generated clocks. This property also works for nested generated clocks. *Return type*: collection
`comment`	Returns the comment set for the specified clock. *Return Type*: String For example, `get_property [get_clocks CK] comment`
`clock_network_pins`	Returns a collection of all pins on the clock network for the clock. *Return type:* collection
`delay_max_fall`	Returns the maximum falling delay value for the clock. *Return type:* float
`delay_max_rise`	Returns the maximum rising delay value for the clock. *Return type:* float
`delay_min_fall`	Returns the minimum falling delay value for the clock. *Return type:* float
`delay_min_rise`	Returns the minimum rising delay value for the clock. *Return type:* float
`divide_by`	Returns dfactor set for the specified clock. For example, `get_property [get_clocks ck] divide_by` *Return Type*: Integer
`driving_network_pins`	Returns a collection of the network pins of the driving clock. For example, `get_property [get_clocks clk] driving_network_pins` *Return Type*: Collection
`duty_cycle`	Returns the duty cycle for the specified clock. For example, `get_property [get_clocks CLK] duty_cycle` *Return Type*: Float

`edges`	Return the edges that are set for a specified clock. For example, `get_property [get_clocks ck] edges` *Return Type*: String
`edge_shift`	Return the edge shift that is set for a specified clock. For example, `get_property [get_clocks ck] edge_shift` Return Type: String
`generated_clocks_extended`	Returns generated clocks derived from the regular clock. *Return type:* collection
`hierarchical_name`	Returns the complete hierarchical name of the clock. *Return type:* string
`hold_uncertainty`	Returns the most conservative uncertainty of all the possible uncertainty assertions associated with a pin, when the `set_clock_uncertainty -hold` parameter is specified. For example, `get_property [get_clocks <clock name>] hold_uncertainty` *Return Type*: Float
`is_context_mapped`	Returns a value of `true` if a clock is mapped to the top in a context. Additionally, this is also reported with the `report_clocks` command. For example, `get_property [get_clocks <clock name>] is_context_mapped` *Return Type*: Boolean
`ideal_transition_max_fall`	Returns the ideal clock transition asserted on clock waveforms specified with the `set_clock_transition` command using the `-fall` and `-max` parameters. *Return type:* float
`ideal_transition_max_rise`	Returns the ideal clock transition asserted on clock waveforms specified with the `set_clock_transition` command using the `-rise` and `-max` parameters. *Return type:* float
`ideal_transition_min_fall`	Returns the ideal clock transition asserted on clock waveforms specified with `set_clock_transition` command using the `-fall` and `-min` parameters. *Return type:* float
`ideal_transition_min_rise`	Returns the ideal clock transition asserted on clock waveforms specified with `set_clock_transition` command using the `-rise` and `-min` parameters. *Return type:* float
`is_active`	Returns a value of `true` if the clock specified is active in the current view. *Return type:* boolean
`is_generated`	Returns a value of `true` if the clock is a generated clock. *Return type:* boolean
`is_inverted`	Returns a value of `true`, if a specified clock is inverted. For example, `get_property [get_clocks ck] is_inverted` Return Type: Boolean

`is_source_inverted`	Returns a value of `true` if a specified clock source is inverted. For example, `get_property [get_clocks clk] is_source_inverted` *Return Type*: Boolean
`is_propagated_clock`	Returns a value of `true` if the clock is in propagated mode. *Return type:* boolean
`is_virtual`	Returns a value of `true` if the specified clock is virtual. For example, `get_property [get_clocks v_clk] is_virtual` `Return Type`: Boolean
`latency_fall_max`	Returns the maximum fall latency for a clock, when the `set_clock_latency -max -fall` parameters are specified. *Return type:* float
`latency_fall_min`	Returns the minimum fall latency for a clock, when the `set_clock_latency -min -fall` parameters are specified. *Return type:* float
`latency_rise_max`	Returns the maximum rise latency for a clock, when the `set_clock_latency -max -rise` parameters are specified. For example, `get_property [get_clocks <clock name>] latency_rise_max` *Return Type*: Float
`latency_rise_min`	Returns the minimum rise latency for a clock, when the `set_clock_latency -min -rise` parameters are specified. For example, `get_property [get_clocks <clock name>] latency_rise_min` *Return Type*: Float
`lead_edge_type`	Returns an edge type (rise or fall) corresponding to the leading edge for a specified clock. For example, `get_property [get_clocks <clock name>] lead_edge_type` *Return Type*: String
`master_clock`	Returns a collection of the master clocks for a specified clock. For example, `get_property [get_clocks clk] master_clock` *Return Type*: Collection
`master_source`	Returns a collection of the master sources for a specified clock. For example, `get_property [get_clocks ck] master_source` *Return Type*: Collection
`max_capacitance_clock_path_fall`	Returns fall max_capacitance value on clock paths. *Return type:* float
`max_capacitance_clock_path_rise`	Returns rise max_capacitance value on clock paths. *Return type:* float
`max_capacitance_data_path_fall`	Returns fall max_capacitance value on data paths. *Return type:* float
`max_capacitance_data_path_rise`	Returns rise max_capacitance value on data paths. *Return type:* float
`max_transition_clock_path_fall`	Returns fall max_transition value on clock paths. *Return type:* float

`max_transition_clock_path_rise`	Returns rise max_transition value on clock paths. *Return type:* float
`max_transition_data_path_fall`	Returns fall max_transition value on data paths. *Return type:* float
`max_transition_data_path_rise`	Returns rise max_transition value on data paths. *Return type:* float
`min_pulse_width_high`	Returns the minimum pulse width for high signals for the specified clock, when the `set_min_pulse_width -high` parameter is specified. For example, `get_property [get_clocks <clock name>] min_pulse_width_high` *Return Type*: Float
`min_pulse_width_low`	Returns the minimum pulse width for low signals for the specified clock, when the `set_min_pulse_width -low` parameter is specified. For example, `get_property [get_clocks <clock name>] min_pulse_width_low` *Return Type*: Float
`multiply_by`	Returns mfactor set for the specified clock. For example, `get_property [get_clocks clk] multiply_by` *Return Type*: Integer
`name`	Returns the name of the clock. *Return type:* string
`object_type`	Returns the object type `clock`. *Return type:* string
`period`	Returns the clock period. *Return type:* float
`setup_uncertainty`	Returns the most conservative uncertainty of all the possible uncertainty assertions associated with pins, when the `set_clock_uncertainty -setup` parameter is specified for a clock. For example, `get_property [get_clocks <clock name>]setup_uncertainty` *Return Type*: Float
`source_latency_early_fall_max`	Returns the early source latency for the falling transition at the maximum corner, when the `set_clock_latency -source -early -fall -max` parameters are specified for the clock. For example, `get_property [get_clocks <clock name>] source_latency_early_fall_max` *Return Type*: float
`source_latency_early_fall_min`	Returns the early source latency for the falling transition at the min corner, when the `set_clock_latency -source -early -fall -min` parameters are specified for the clock. For example, `get_property [get_clocks <clock name>] source_latency_early_fall_min` *Return type:* float
`source_latency_early_rise_max`	Returns the early source latency for the rising transition at the max corner, when the `set_clock_latency -source -early -rise -max` parameters are specified for the clock. For example, `get_property [get_clocks <clock name>] source_latency_early_rise_max` *Return Type*: float

`source_latency_early_rise_min`	Returns the early source latency for the falling transition at the min corner, when the `set_clock_latency -source -early -rise -min` parameters are specified for the clock. For example, `get_property [get_clocks <clock name>] source_latency_early_rise_min` *Return type:* float
`source_latency_late_fall_max`	Returns the late source latency for the falling transition at the max corner, when the `set_clock_latency -source -late -fall -max` parameters are specified for the clock. For example, `get_property [get_clocks <clock name>] source_latency_late_fall_max` *Return type:* float
`source_latency_late_fall_min`	Returns the late source latency for the falling transition at the min corner, when the `set_clock_latency -source -late -fall -min` parameters are specified for the clock. For example, `get_property [get_clocks <clock name>] source_latency_late_fall_min` *Return type:* float
`source_latency_late_rise_max`	Returns the late source latency for the rising transition at the max corner, when the `set_clock_latency -source -late -rise -max` parameters are specified for the clock. For example, `get_property [get_clocks <clock name>] source_latency_late_rise_max` *Return type:* float
`source_latency_late_rise_min`	Returns the late source latency for the rising transition at the min corner, when the `set_clock_latency -source -late -rise -min` parameters are specified for the clock. For example, `get_property [get_clocks <clock name>] source_latency_late_rise_min` *Return type:* float
`sources`	Returns a collection of the clock's source pins or ports. *Return type:* collection
`view_name`	In multi-mode multi-corner mode, returns the analysis view name associated with a given clock. *Return type*: string
`waveform`	Returns the rising and falling edge times of the waveform. *Return type:* string

Design Properties

`analysis_type`	Returns the analysis mode: `single`, `bc_wc`, or `on_chip_variation`. *Return type:* string
`capacitance_unit_in_farad`	Returns the capacitance unit specified in the library. *Return type:* float
`chip_size`	Returns the chip size. The chip size can be specified with the `aocv_chip_size` global variable. If this variable is not set, then the software will calculate the chip size from the design node locations. The global variable setting takes precedence over the software setting. *Return type:* float

`current_unit_in_amp`	Returns the current unit specified in the library. *Return type:* float
`hierarchical_name`	Returns the complete hierarchical name of the design. *Return type:* string
`is_dont_touch`	Returns `true` if the design is specified through the `set_dont_touch` command. *Return type:* boolean
`is_max_transition_asserted`	Returns a value of `true` if a maximum transition value (set using the `set_max_transition` command) is defined in the design. *Return type:* boolean
`is_min_transition_asserted`	Returns a value of `true` if a minimum transition value (set using the `set_min_transition` command) is defined in the design. *Return type:* boolean
`max_capacitance`	Returns the maximum capacitance value for the design. *Return type:* float
`max_fanout`	Returns the maximum fanout value for the design. *Return type:* float
`max_transition`	Returns the maximum transition design rule limit for the design. *Return type:* float
`min_capacitance`	Returns the minimum capacitance value for the design. *Return type:* float
`object_type`	Returns the object type `design`. *Return type: string*
`operating_condition_max`	Returns the name of the maximum operating condition specified for the design. *Return type:* string
`operating_condition_min`	Returns the name of the minimum operating condition specified for the design. *Return type:* string
`process_max`	Returns the process value of the maximum operating condition specified for the design. *Return type:* float
`process_min`	Returns the process value of the minimum operating condition specified for the design. *Return type:* float
`resistance_unit_in_ohm`	Returns the resistance value from the library. *Return type:* float
`slew_threshold_percent_fall_high`	Returns the upper slew trip point for falling waveforms, specified as a percent. *Return type:* float
`slew_threshold_percent_fall_low`	Returns the lower slew trip point for falling waveforms, specified as a percent. *Return type:* float
`slew_threshold_percent_rise_high`	Returns the upper slew trip point for rising waveforms, specified as a percent. *Return type:* float
`slew_threshold_percent_rise_low`	Returns the lower slew trip point for rising waveforms, specified as a percent. *Return type:* float
`temperature_max`	Returns the temperature value of the maximum operating condition for the design. *Return type:* float

`temperature_min`	Returns the temperature value of the minimum operating condition for the design. *Return type:* float
`time_unit_in_second`	Returns the time units of the design, in seconds. that is, time unit of the design is 1.000000ns, the property returns 1.000000e-09. *Return type:* float
`tree_type_max`	Returns the value of the `tree_type` attribute for the specified maximum operating condition. *Return type:* string
`tree_type_min`	Returns the value of the `tree_type` attribute for the specified minimum operating condition. *Return type:* string
`voltage_max`	Returns the voltage value of the maximum operating condition for the design. *Return type:* float
`voltage_min`	Returns the voltage value of the minimum operating condition for the design. *Return type:* float
`voltage_unit_in_volt`	Returns the voltage units specified in the library for the design. *Return type:* float

Library Properties

`default_max_transition`	Returns the default maximum transition time limit specified for a given library. *Return type:* float
`default_operating_conditions`	Returns the default operating conditions specified for a given library. For example, `get_property [get_libs FreePDK45_lib_v1.0] default_operating_conditions` *Return type*: string
`hierarchical_name`	Returns the hierarchical path name for the library. *Return type:* string
`input_threshold_pct_fall`	Returns the value of threshold point on an input pin signal falling from 1 to 0, which is used in modelling the delay of a input pin to an output pin. *Return type:* integer
`input_threshold_pct_rise`	Returns the value of threshold point on an input pin signal rising from 0 to 1, which is used in modelling the delay of a input pin to an output pin. *Return type:* integer
`lib_file_path`	Reports full on-disk path of a library. In case of a library database (LDB) file, the property will return the full on-disk path which LDB was compiled. *Return type:* string
`lib_version`	Returns the software version in which the library file (LDB) was generated. *Return type:* integer
`nominal_process`	Returns a value of the process either modeled as nominal process or as the process in default operating condition in th For example, `get_property [get_libs sc_cmoslp] nominal_process` *Return Type:* float

`nominal_temperature`	Returns a value of the temperature either modeled as nominal temperature or as the temperature in default operating c For example, `get_property [get_libs sc_cmoslp] nominal_temperature` *Return Type:* float
`nominal_voltage`	Returns a value of the voltage either modeled as nominal voltage or as the voltage in default operating condition in the For example, `get_property [get_libs sc_cmoslp] nominal_voltage` *Return Type:* float
`object_type`	Returns the object type `library`. *Return type:* string
`output_threshold_pct_fall`	Returns the value of threshold point on an output signal falling from 1 to 0, which is used in modelling the delay of a sig pin to an output pin. *Return type:* integer
`output_threshold_pct_rise`	Returns the value of threshold point on an output signal rising from 0 to 1, which is used in the modelling of a signal tra an output pin. *Return type:* integer
`slew_derate_from_library`	Specifies how the transition times found in the library need to be derated to match the transition times between the cha *Return type:* float
`slew_lower_threshold_pct_fall`	Returns the value of lower threshold point used in modeling the delay of a pin falling from 1 to 0. *Return type:* float
`slew_lower_threshold_pct_rise`	Returns the value of lower threshold point used to model the delay of a pin rising from 0 to 1. *Return type:* float
`slew_upper_threshold_pct_fall`	Returns the value of upper threshold point used to model the delay of a pin falling from 1 to 0. *Return type:* float
`slew_upper_threshold_pct_rise`	Returns the value of upper threshold point used to model the delay of a pin rising from 0 to 1. *Return type:* float
`source_file_name`	Returns the full path of all library files that contain the specified library name. The use model is given below: `get_property [get_libs libname] source_file_name [-view viewName]` If the `-view` parameter is specified, the property returns the path of the library files in the given analysis view. If this parameter is not specified, the command returns the path of all library files in all the analysis views. *Return Type:* string
Library Properties for Power/Ground Objects	

`power_rails`	Returns the name of the power rails along with their corresponding voltages as defined in the `voltage_map` attribute in
	Return Type: string
	For example,
	`get_property [get_libs sc_cmoslp] power_rails`
	`Output: Is a list of lists, each list showing the power rail name along with its associated voltage.`
	`{ {VDDG 1.045} {VDD 1.045} {VSS 0} }`

Library Cell Properties

`area`	Returns the area of the library cell. *Return Type:* float
`early_clock_check_incremental_derate_factor`	Returns the early clock path library cell check incremental derating factor specified through the `set_ti` *Return Type:* float
`early_data_check_incremental_derate_factor`	Returns the early data path library cell check incremental derating factor specified through the `set_tir` *Return Type*: float
`early_fall_clock_incremental_derate_factor`	Returns the early clock path library cell check incremental derating factor specified through the `set_ti` parameter is specified. *Return Type*: float
`early_fall_data_incremental_derate_factor`	Returns the early data path library cell check incremental derating factor specified through the `set_tir` parameter is specified. *Return Type*: float
`early_rise_clock_incremental_derate_factor`	Returns the early clock path library cell check incremental derating factor specified through the `set_ti` parameter is specified. *Return Type*: float
`early_rise_data_incremental_derate_factor`	Returns the early data path library cell check incremental derating factor specified through the `set_tir` parameter is specified. *Return Type*: float
`early_slew_derate_factor`	Returns the derate factor applied on a library cell when the `set_timing_derate -transition` paramete For example, `get_property [get_lib_cells CTS_HT.db/ICG_F4_HT] early_slew_derate_factor` *Return Type*: float
`fall_input_switching_derate_factor`	Returns the derate factor applied on a library cell when the `set_timing_derate -input_switching` par For example, `get_property [get_lib_cells PDK45_v1.0/IV_X32] fall_input_switching_derate_factor` *Return Type*: float
`has_ccs_noise`	Returns a value of `true` if a library cell has a CCS noise look-up table. For example, `get_property [get_lib_cells tst/tst_nm6] has_ccs_noise` *Return type:* boolean
`has_socv`	Returns a value of `true` if a library cell has a spatial derate table (SOCV). For example, `get_property [get_lib_cells PK4_lib_v1.0/A4_X2] has_socv` *Return type:* boolean
`has_spatial`	Returns a value of `true` if a spatial derate exists for a library cell. *Return type:* boolean
`hierarchical_name`	Returns the complete hierarchical name of the library cell (the library name followed by the library cell For example, `get_property [get_lib_cells -of [get_cell AB_IT/E_0146_tl]] hierarchical_name` *Return Type:* string
`is_black_box`	Returns a value of `true` if the cell is a black box cell. A cell is considered as a black box if the "function" definition of the cell is missing from the library. *Return type:* boolean

`is_buffer`	Returns a value of `true` if the associated library cell is recognized as a buffer cell. For example, `get_property [get_lib_cells -of [get_cell AB_IT/E_0146_tl]] is_buffer` *Return type:* boolean
`is_combinational`	Returns a value of `true` if the library cell is a combinational cell (not a sequential cell). *Return type:* boolean
`is_dont_touch`	Returns a value of `true` if the library cell definition includes `dont_touch:true`. *Return Type:* boolean
`is_dont_use`	Returns a value of `true` if the library cell description includes `dont_use:true`. *Return Type:* boolean
`is_fall_edge_triggered`	Returns a value of `true` if the library cell is triggered by the falling edge of the clock. *Return Type:* boolean
`is_instantiated`	Returns a value of `true` if library cells are instantiated in a design. For example, `get_property [get_lib_cells P4_lib_v1.0/N4_X2] is_instantiated` *Return type*: boolean
`is_interface_timing`	Returns a value of `true` if a library cell has interface timing specified for that cell. *Return type*: boolean
`is_level_shifter`	Returns a value of `true` if a cell instance is a level-shifter cell. *Return type*: boolean
`is_integrated_clock_gating_cell`	Returns a value of `true` if a cell is an integrated clock gating cell. For example, `get_property [get_lib_cells CTS_HT.db/ICG_F7_HT] is_integrated_clock_gating_cell` *Return type*: boolean
`is_inverter`	Returns a value of `true` if a cell instance is an inverter cell. For example, `get_property [get_lib_cells -of [get_cells inv]] is_inverter` *Return type*: boolean
`is_level_shifter`	Returns a value of `true` if a cell instance is a level-shifter cell. For example, `get_property [get_lib_cells -of [get_libs mk_1pv_125c]] is_level_shifter` *Return type*: boolean
`is_macro_cell`	Returns a value of `true` if a macro cell exists in a library. For example, `get_property [get_lib_cells L_mro2/mro2y] is_macro_cell` *Return type*: boolean

`is_memory_cell`	Returns a value of `true` if the cell is a memory cell.
	Return type: boolean
	Note: The library cells that have a memory group defined in the dotlib are considered as memory cells
	`cell ( ) {`
	`:`
	`memory() {`
	`type : ram ;`
	`address_width : 10 ;`
	`word_width : 7 ;`
	`}`
	`}`

`is_msv`	Returns a value of `true` if a cell is an MSV cell. For example, `get_property [get_lib_cells mk_1pv_125c/L8MTR] is_msv` *Return type*: boolean
`is_negative_level_sensitive`	Returns a value of `true` if the library cell is used as a negative level-sensitive latch. *Return type:* boolean
`is_pad_cell`	Returns a value of `true` if the Liberty library `is_pad_cell` attribute is set to `true` for this pad cell. *Return type:* boolean
`is_pll_cell`	Returns a value of `true` if the Liberty library `is_pll_cell` attribute is set to `true` for this library cell. *Return type:* boolean
`is_positive_level_sensitive`	Returns a value of `true` if the library cell is used as a positive level-sensitive latch. *Return type:* boolean
`is_rise_edge_triggered`	Returns a value of `true` if the library cell is triggered by the rising edge of the clock. *Return Type:* boolean
`is_sequential`	Returns a value of `true` if the library cell is a latch or flip-flop, or if the cell has sequential timing check *Return type:* boolean
`is_tristate`	Returns a value of `true` if the library cell definition includes `three_state:true`. *Return type:* boolean
`late_clock_check_incremental_derate_factor`	Returns the late clock path library cell check incremental derating factor specified through the `set_tim` *Return Type*: float
`late_data_check_incremental_derate_factor`	Returns the late data path library cell check incremental derating factor specified through the `set_tim` *Return Type*: float
`late_fall_clock_incremental_derate_factor`	Returns the late clock path library cell check incremental derating factor specified through the `set_tim` parameter is specified. *Return Type*: float
`late_fall_data_incremental_derate_factor`	Returns the late data path library cell check incremental derating factor specified through the `set_tim` parameter is specified. *Return Type*: float
`late_fall_clock_period_derate_factor`	Returns the derate factor for the clock period when the `set_timing_derate -late` and `-fall` paramete *Return type:* float

`late_fall_clock_period_mean_derate_factor`	Returns the late/fall derate factor for the clock period when the `set_timing_derate -mean` parameter is *Return type:* float
`late_fall_clock_period_sigma_derate_factor`	Returns the late/fall derate factor for the clock period when the `set_timing_derate -sigma` parameter i *Return type:* float
`late_fall_pulse_width_derate_factor`	Returns the late/fall derate factor for the pulse width when the `set_timing_derate` -late and -fall param *Return type:* float
`late_fall_pulse_width_mean_derate_factor`	Returns the late/fall derate factor for the pulse width when the `set_timing_derate` -mean parameter is *Return type:* float
`late_fall_pulse_width_sigma_derate_factor`	Returns the late/fall derate factor for the pulse width when the `set_timing_derate` -sigma parameter is *Return type:* float
`late_rise_clock_incremental_derate_factor`	Returns the late clock path library cell check incremental derating factor specified through the `set_tim` parameter is specified. *Return Type*: float
`late_rise_data_incremental_derate_factor`	Returns the late data path library cell check incremental derating factor specified through the `set_tim` parameter is specified. *Return Type*: float
`late_rise_clock_period_derate_factor`	Returns the derate factor for the clock period when the `set_timing_derate` -late and -rise parameters *Return type:* float
`late_rise_clock_period_mean_derate_factor`	Returns the late/rise derate factor for the clock period when the `set_timing_derate -mean` parameter is *Return type:* float
`late_rise_clock_period_sigma_derate_factor`	Returns the late/rise derate factor for the clock period when the `set_timing_derate -sigma` parameter *Return type:* float
`late_rise_pulse_width_derate_factor`	Returns the derate factor for the pulse width check when the `set_timing_derate -late` and `-rise` para *Return type:* float
`late_rise_pulse_width_mean_derate_factor`	Returns the late/rise derate factor for the pulse width when the `set_timing_derate -mean` parameter is *Return type:* float
`late_rise_pulse_width_sigma_derate_factor`	Returns the late/rise derate factor for the pulse width when the `set_timing_derate -sigma` parameter i *Return type:* float
`late_slew_derate_factor`	Returns the derate factor applied on a library cell, when the `set_timing_derate -transition` paramet For example, `get_property [get_lib_cells PK4_v1.0/IV_32] late_slew_derate_factor` *Return type:* float
`name`	Returns the leaf-level name of the library cell. *Return Type:* string
`object_type`	Returns the object type `lib_cell`. *Return Type:* string
`physical_variant_cells`	Returns the physical variant cells that are available in a library cell. For example, `get_property [get_lib_cells lca_md/SR1_md] physical_variant_cells` *Return Type:* string

`pin_count`	Returns the total number of signal pins. For example, `get_property [get_lib_cells PK45_v1.0/IN_X32] pin_count` *Return Type:* integer
`receiver_capacitance_type`	Returns the value of input capacitance group being used for delay calculations – C1C2 (2-piece mode *Return Type:* string
`retention_cell`	Returns the value of the retention_cell attribute specified in the liberty. *Return type*: string
`rise_input_switching_derate_factor`	Returns the derate factor applied on a library cell, when the `set_timing_derate -input_switching` par For example, `get_property [get_lib_cells PK45_v1.0/DL_X2] rise_input_switching_derate_factor` *Return type:* float
`signal_pins`	Returns the signal pins of a cell, along with the library name, library cell name, and pin name. For example, `get_property [get_lib_cells PK45_v1.0/IN_X32] signal_pins` *Return Type:* string
`threshold_voltage_group`	Returns the threshold voltage group assigned to a cell in a library. For example, `get_property [get_lib_cells L40_HT.db/CG_F8_HT] threshold_voltage_group` *Return Type:* string
`output_signal_level_high`	Returns the maximum voltage value of an output pin after a rising transition. For example, `get_property [get_lib_arcs -of [get_lib_cells px_ipid_3/px_ipid_3]] output_signal_level_h` *Return type:* float
`output_signal_level_low`	Returns the minimum voltage value of an output pin after a falling transition. For example, `get_property [get_lib_arcs -of [get_lib_cells px_ipid_3/px_ipid_3]] output_signal_level_l` *Return type:* float
`timing_model_type`	Returns the model type for a given cell or instance. The supported values are `abstracted`, `extracted`, *Return type*: string
Library Cell Properties for Power/Ground Objects	
`ground_pins`	Returns a collection of the ground pins of a library cell. *Return type*: collection For example, `get_property [get_lib_cells sc_cmoslp/AO22X] ground_pins`
`is_always_on`	Returns a value of `true` if the `always_on` attribute of a library cell is set to `true` indicating that the library *Return type*: boolean For example, `get_property [get_lib_cells sc_cmoslp/AON22_X4M] is_always_on`
`is_isolation_cell`	Returns a value of `true` if the library cell is an isolation cell. *Return type*: boolean For example, `get_property [get_lib_cells sc_cmoslp/ISO22_X4M] is_isolation_cell`
`is_power_switch`	Returns a value of `true` when the `switch_cell_type` attribute is defined for a library cell indicating tha *Return type*: boolean For example, `get_property [get_lib_cells sc_cmoslp/AO22X] is_power_switch`

`is_retention`	Returns a value of `true` when the `retention_cell` attribute is defined for a library cell indicating that th *Return type*: boolean For example, `get_property [get_lib_cells sc_cmoslp/RET22_X4M] is_retention`
`level_shifter_type`	Returns the type of level-shifter (e.g., LH, HL, or HL_LH) as defined by the `level_shifter_type` attribu *Return type*: string For example, `get_property [get_lib_cells sc_cmoslp/A2LVLD_X4M] level_shifter_type`
`lib_pg_pins`	Returns a collection of all PG pins (both power and ground) defined in the library of a library cell. *Return type*: collection For example, `get_property [get_lib_cells sc_cmoslp/AO22X] lib_pg_pins`
`power_pins`	Returns a collection of the power pins of a library cell. *Return type*: collection For example, `get_property [get_lib_cells sc_cmoslp/AO22X] power_pins`
`power_switch_type`	Returns the type of power-switch (e.g., coarse-grain, fine-grain) as defined by the `switch_cell_type` a *Return type*: string For example, `get_property [get_lib_cells sc_cmoslp/AO22X] power_switch_type`
`std_cell_main_rail_name`	Returns the name of a power rail that has the `std_cell_main_rail` library attribute defined as `true` for *Return type*: string For example, `get_property [get_lib_cells sc_cmoslp/AO22X] std_cell_main_rail_name`

Library Pin Properties

`capacitance`	Returns the capacitance for the library pin. *Return type:* float
`capacitance_max_fall`	Returns the maximum value of the falling capacitance range. *Return type:* float
`capacitance_max_rise`	Returns the maximum value of the rising capacitance range. *Return type:* float
`capacitance_min_fall`	Returns the minimum value of the falling capacitance range. *Return type:* float
`capacitance_min_rise`	Returns the minimum value of the rising capacitance range. *Return type:* float
`cell_name`	Returns the library cell name for the corresponding library pins. For example, `get_property [get_lib_pins PK4_lib_v1.0/AOI211_X2/B] cell_name` *Return type*: string
`direction`	Returns the direction of the library pin: `in`, `out`, or `inout`. *Return type:* string
`fanout_load`	Returns the fanout load value of the library pin. *Return type:* float

`function`	Returns the function for the corresponding library pins. For example, `get_property [get_lib_pins PK4_lib_v1.0/AND2_X1/ZN] function` *Return type*: string
`hierarchical_name`	Returns the hierarchical name of the library pin. *Return type:* string
`input_signal_level`	Returns the power rail name/signal that is defined for a library cell input pin. For example, `get_property [get_lib_pins PK4_lib_v1.0/BUF_X32/A] input_signal_level` *Return type*: string
`input_signal_level_high`	Returns the upper limit of partial voltage swing value that is defined for a library cell input pin. For example, `get_property [get_lib_pins rvt_ss_0p9v/LVLL/A] input_signal_level_high` *Return type*: float
`input_signal_level_low`	Returns the lower limit of partial voltage swing value that is defined for a library cell input pin. For example, `get_property [get_lib_pins rvt_ss_0p9v/LVLL/A] input_signal_level_low` *Return type*: float
`is_analog`	Returns a value of `true` if the library pin is an analog pin. For example, `get_property [get_lib_pins PK4_lib_v1.0/AOI222_X2/ZN] is_analog` *Return type*: boolean
`is_async`	Returns a value of `true` if the library pin is an asynchronous preset pin, or an asynchronous clear pin. *Return type:* boolean
`is_clear`	Returns a value of `true` if the pin is an asynchronous clear pin. *Return type:* boolean
`is_clock`	Returns a value of `true` if the library pin definition includes `clock:true`. *Return type:* boolean
`is_clock_gating_clock`	Returns a value of `true` if the library pin is defined as a clock pin of a clock gating cell. For example, `get_property [get_lib_pins P40_CTS_HVT.db/ICG_HVT/CLKIN] is_clock_gating_clock` *Return type*: boolean
`is_clock_gating_enable`	Returns a value of `true` if the library pin corresponds to the enable pin of a clock gating cell. For example, `get_property [get_lib_pins P40_CTS_HVT.db/ICG_HVT/CLKIN] is_clock_gating_enable` *Return type*: boolean
`is_data`	Returns a value of `true` if the library pin with defined sequential arcs is the data pin of a flip-flop. *Return type:* boolean
`is_fall_edge_triggered_clock`	Returns a value of `true` if the library pin is the clock pin of a falling edge triggered device. *Return type:* boolean
`is_fall_edge_triggered_data`	Returns a value of `true` if the library pin is the data pin of a falling edge triggered device. *Return type:* boolean

`is_negative_level_sensitive_clock`	Returns a value of `true` if the library pin is an enable pin of an active low level-sensitive device. *Return type:* boolean
`is_negative_level_sensitive_data`	Returns a value of `true` if the pin is a data pin of an active low level-sensitive device. *Return type:* boolean
`is_pad`	Returns a value of `true` if the library pin is a pad pin. For example, `get_property [get_lib_pins *] is_pad` *Return type*: boolean
`is_pll_feedback_pin`	Returns a value of `true` if the Liberty library `is_pll_feedback_pin` attribute is set to `true` for this library pin. *Return type:* boolean
`is_pll_output_pin`	Returns a value of `true` if the Liberty library `is_pll_output_pin` attribute is set to `true` for this library pin. *Return type:* boolean
`is_pll_reference_pin`	Returns a value of `true` if the Liberty library `is_pll_reference_pin` attribute is set to `true` for this library pin. *Return type:* boolean
`is_positive_level_sensitive_clock`	Returns a value of `true` if the library pin is an enable pin of an active high level-sensitive device. *Return type:* boolean
`is_positive_level_sensitive_data`	Returns a value of `true` if the pin is a data pin of an active high level-sensitive device. *Return type:* boolean
`is_preset`	Returns a value of `true` if the pin is a preset pin. *Return type:* boolean
`is_rise_edge_triggered_clock`	Returns a value of `true` if the library pin is the clock pin of a rising edge triggered device. *Return type:* boolean
`is_rise_edge_triggered_data`	Returns a value of `true` if the library pin is the data pin of a rising edge triggered device. *Return type:* boolean
`is_tristate`	Returns a value of `true` if the library pin definition includes `three_state:true`. *Return type:* boolean
`max_capacitance`	Returns the maximum capacitance limit for the library pin. *Return type:* float
`max_fanout`	Returns the maximum fanout value for the library pin. *Return type:* float
`max_transition`	Returns the maximum transition time limit specified for a given library pin. If the limit is not specified for a library pi *Return type:* float
`min_transition`	Returns the minimum transition time limit specified for a given library pin. If the limit is not specified for a library pir *Return type:* float
`min_capacitance`	Returns the minimum capacitance limit for the library pin. *Return type:* float
`min_fanout`	Returns the minimum fanout value for the library pin. *Return type:* float
`name`	Returns the leaf-level name of the library cell pin. *Return type:* string

`object_type`	Returns the object type; in this case, `lib_pin`. *Return type:* string
`related_ground_pin_name`	Returns related ground pin names for the corresponding library pins. *Return type:* string For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] related_ground_pin_name`
`related_power_pin_name`	Returns related power pin names for the corresponding library pins. *Return type:* string
`slew_threshold_percent_fall_high`	Returns the upper slew trip point for falling waveforms, specified as a percent. *Return type:* float
`slew_threshold_percent_fall_low`	Returns the lower slew trip point for falling waveforms, specified as a percent. *Return type:* float
`slew_threshold_percent_rise_high`	Returns the upper slew trip point for rising waveforms, specified as a percent. *Return type:* float
`slew_threshold_percent_rise_low`	Returns the lower slew trip point for rising waveforms, specified as a percent. *Return type:* float
Library Signal Pin Properties for Power/Ground Objects	
`input_signal_level_voltage`	Returns the voltage value corresponding to the power rail name/signal which is defined for the library-cell input pi with the relative power rail name through the `voltage_map` attribute defined in the library. *Return type:* float For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] input_signal_level_voltage`
`is_always_on`	Returns a value of `true` if the `always_on` attribute of a library signal pin is set to `true` indicating that the library cell *Return type:* boolean For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] is_always_on`
`is_isolated`	Returns a value of `true` if the `is_isolated` attribute of a library signal pin is set to `true` indicating that the library ce *Return type:* boolean For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] is_isolated`
`is_isolation_cell_data`	Returns a value of `true` if the `isolation_cell_data_pin` attribute of a library signal pin is set to `true` indicating tha *Return type:* boolean For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] is_isolation_cell_data`
`is_isolation_cell_enable`	Returns a value of `true` if the `isolation_cell_enable_pin` attribute of a library signal pin is set to `true` indicating tl library cell. *Return type:* boolean For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] is_isolation_cell_enable`
`is_level_shifter_data`	Returns a value of `true` if the `level_shifter_data_pin` attribute of a library signal pin is set to `true`, indicating that *Return type:* boolean For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] is_level_shifter_data`

`is_level_shifter_enable`	Returns a value of `true` if the `level_shifter_enable_pin` attribute of a library signal pin is set to be `true` indicating cell. *Return type:* boolean For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] is_isolation_cell_enable`
`is_power_switch_enable`	Returns a value of `true` if the `switch_cell_type` attribute of the corresponding library cell is defined and the `switc` *Return type:* boolean For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] is_power_switch_enable`
`is_retention_cell_enable`	Returns a value of `true` if the `retention_pin` attribute of a signal pin is defined. *Return type:* boolean For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] is_retention_cell_enable`
`output_signal_level`	Returns the power rail name/signal that is defined for a library cell output and inout pins. *Return type:* string For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] output_signal_level`
`output_signal_level_high`	Returns the upper/higher limit of partial voltage swing value that is defined for a library cell output and inout pins. *Return type:* float For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] output_signal_level_high`
`output_signal_level_low`	Returns the lower limit of partial voltage swing value that is defined for a library cell output and inout pins. *Return type:* float For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] output_signal_level_low`
`related_ground_pin`	Returns the related ground pins as a collection for the corresponding library cell pins. *Return type:* collection For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] related_ground_pin`
`related_ground_pin_name`	Returns the related ground pin names for the corresponding library cell pins. *Return type:* string For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] related_ground_pin_name`
`related_ground_pin_rail_voltage`	Returns the voltage value corresponding to the ground rail name defined for the `related_ground_pin` attribute of th associated with the relative ground rail name through the `voltage_map` attribute defined in the library. *Return type:* float For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] related_ground_pin_rail_voltage`
`related_power_pin`	Returns the related power pins as a collection for the corresponding library cell pins. *Return type:* collection For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] related_power_pin`
`related_power_pin_name`	Returns the related power pin names for the corresponding library cell pins. *Return type:* string For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] related_power_pin_name`

related_power_pin_rail_voltage	Returns the voltage value corresponding to the power rail name defined for the related_power_pin attribute of a li associated with the relative power rail name through the voltage_map attribute defined in the library. *Return type:* float For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] related_power_pin_rail_voltage`

Library Pin Properties for Power/Ground Objects

is_std_cell_main_rail	Returns a value of true if the library power pg_pin is the main rail of the library cell. It is determined by the std_cell_main_rail library attribute of libray pg_pin. *Return type*: boolean For example, `get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/AO22X]] is_std_cell_main_rail`
name	Returns the name of the library pg pin as defined in the library and associated with the pg_pin library attribute. *Return type*: string For example, `get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/AO22X]] name`
object_type	Returns lib_pg_pin as the object type for a library pg pin. *Return type*: string For example, `get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/AO22X]] object_type`

pg_type	Returns the type of the library pg pin as associated with the pg_type attribute of the corresponding library pg pin.
	Return type: string
	For example,
	`get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/AO22X]] pg_type`
	`get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/AO22X] -filter " name =~ *VDD*" ] pg_type`

rail_name	Returns the name of the rail to which the pg-pin is connected and is associated with the voltage_name attribute of the library pg-pin. *Return type*: string For example, `get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/AO22X]] rail_name`
rail_voltage	Returns the voltage from the voltage_map association of the rail to which the library pg-pin is connected. *Return type*: float For example, `get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/AO22X]] rail_voltage`
tied_to	Returns the libray pg pin which is connected to substrate-bias pg pin. *Return type*: string For example, `get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/AO22X]] tied_to`

Library Timing Arc Properties

`from_lib_pin`	Returns a collection of the source library pins of the library timing arc. *Return type:* collection
`is_disabled`	Return a value of `true` if a `set_disable_timing` constraint exists for a an arc. *Return type:* boolean
`object_type`	Returns the object type `lib_timing_arc`. *Return type:* string
`sdf_cond`	Returns the `sdf_cond` expression specified for the timing library arc in the library. *Return type:* string
`sense`	Returns the `timing_sense` specified for the timing library arc in the library. *Return type:* string
`to_lib_pin`	Returns a collection of the sink library pins of the library timing arc. *Return type:* collection
`timing_type`	Returns the type specified for the timing library arc in the library. *Return type:* string
`when`	Returns the `when` condition specified for the library timing arc in the library. *Return type:* string

Net Properties

`attackers`	Reports the parasitic data on a net - capacitance, resistance, or cross-coupling. *Return type:* collection
`capacitance_max`	Returns the total capacitance of the net for maximum conditions. *Return type:* float
`coupling_capacitance_max`	Returns the maximum value of coupling capacitance of a net over all the views. *Return type:* float
`capacitance_min`	Returns the total capacitance of the net for minimum conditions. *Return type:* float
`coupling_capacitance_min`	Returns the minimum value of coupling capacitance of a net over all the views. *Return type:* float
`driver_pins`	Returns a collection of the driver pins of the net. *Return type:* collection
`early_fall_clk_net_derate_factor`	Returns the early fall clock net derating factor specified through `set_timing_derate` command. *Return type:* float
`early_fall_clk_net_delta_derate_factor`	Returns the early fall clock delta delay net derating factor specified through the `set_timing_derate` command. *Return type:* float
`early_fall_data_net_delta_derate_factor`	Returns the early fall data delta net derating factor specified through the `set_timing_derate` command. *Return type:* float

early_fall_data_net_derate_factor	Returns the early fall data net derating factor specified through set_timing_derate command. *Return type:* float
early_rise_clk_net_delta_derate_factor	Returns the early rise clock delta net derating factor specified through the set_timing_derate command. *Return type:* float
early_rise_clk_net_derate_factor	Returns the early rise clock net derating factor specified through the set_timing_derate command. *Return type:* float
early_rise_data_net_delta_derate_factor	Returns the early rise data delta net derating factor specified through the set_timing_derate command. *Return type:* float
early_rise_data_net_derate_factor	Returns the early rise data net derating factor specified through the set_timing_derate command. *Return type:* float
has_detailed_parasitics	Returns a value of true if the net, or one of its parts, has detailed parasitics associated with it from either RC extraction or SPEF annotation. *Return type:* boolean
hierarchical_name	Returns the complete hierarchical name of the net. *Return type:* string
is_ideal	Returns a value of true if set_ideal_latency, set_ideal_transition, or set_ideal_network constraints exist for the net. *Return type*: Boolean
late_fall_clk_net_delta_derate_factor	Returns the late fall clock delta net derating factor specified through the set_timing_derate command. *Return type:* float
late_fall_clk_net_derate_factor	Returns the late fall clock net derating factor specified through the set_timing_derate command. *Return type:* float
late_fall_data_net_delta_derate_factor	Returns the late fall data delta net derating factor specified through the set_timing_derate command. *Return type:* float
late_fall_data_net_derate_factor	Returns the late fall data net derating factor specified through the set_timing_derate command. *Return type:* float
late_rise_clk_net_delta_derate_factor	Returns the late rise clock delta net derating factor specified through the set_timing_derate command. *Return type:* float
late_rise_clk_net_derate_factor	Returns the late rise clock net derating factor specified through the set_timing_derate command. *Return type:* float
late_rise_data_net_delta_derate_factor	Returns the late rise data delta net derating factor specified through the set_timing_derate command. *Return type:* float
late_rise_data_net_derate_factor	Returns the late rise data net derating factor specified through the set_timing_derate command. *Return type:* float
is_dont_touch	Returns a value of true if a set_dont_touch constraint exists for the net. *Return type:* boolean
load_pins	Returns a collection of the load pins of the net. *Return type:* collection
lumped_capacitance_max	Returns the maximum value of capacitance of a net based on the input library and parasitic model. *Return type:* float

`lumped_capacitance_min`	Returns the minimum value of capacitance of a net based on the input library and parasitic model. *Return type:* float
`name`	Returns the name of the net. *Return type:* string
`num_driver_pins`	Returns the number of driver pins on a net. *Return type*: integer
`num_load_pins`	Returns the number of load pins on the net. *Return type*: collection
`object_type`	Returns the object type `net`. *Return type:* string
`pin_capacitance_max`	Returns the sum of all pin capacitances of the net for maximum conditions. *Return type:* float
`pin_capacitance_max_fall`	Returns the sum of all pin capacitances of the net for the maximum value of the rising capacitance range. *Return type:* float
`pin_capacitance_max_rise`	Returns the sum of all pin capacitances of the net for the maximum value of the falling capacitance range. *Return type:* float
`pin_capacitance_min`	Returns the sum of all pin capacitances of the net for minimum conditions. *Return type:* float
	Note: For all the `pin_capacitance_*` properties (described above), the pin capacitance matches with the capacitance displayed under the `pin_load` column of the `report_timing` output.
`resistance_max`	Returns the maximum value of resistance of a net. *Return type*: float
`resistance_min`	Returns the minimum value of resistance of a net. *Return type*: float
`total_lumped_capacitance_max_fall`	Returns the maximum value of falling capacitance of a net based on the input library and parasitic model. *Return type:* float
`total_lumped_capacitance_max_rise`	Returns the maximum value of rising capacitance of a net based on the input library and parasitic model. *Return type:* float
`total_lumped_capacitance_min_fall`	Returns the minimum value of falling capacitance of a net based on the input library and parasitic model. *Return type:* float
`total_lumped_capacitance_min_rise`	Returns the minimum value of rising capacitance of a net based on the input library and parasitic model. *Return type:* float
`wire_capacitance_max`	Returns the wire capacitance of the net for maximum conditions. *Return type:* float
`wire_capacitance_min`	Returns the wire capacitance of the net for minimum conditions. *Return type:* float

Net Properties for Power/Ground Objects

`hierarchical_name`	Returns the name of the PG net connected to the corresponding PG pin.
	Return type: string
	
	For example,
	`get_property [get_pg_nets -of_objects [get_cells iA/out_reg] ] hierarchical_name`
	`get_property [get_pg_nets -of_objects [get_cells iA/out_reg] -filter "is_power == true" ] hierarchical_name`

`is_ground`	Returns a value of `true` if the PG net is a ground net.
	Return type: boolean
	
	For example,
	`get_property [get_pg_nets -of_objects [get_cells iA/out_reg] ] is_ground`
	`get_property [get_pg_nets -of_objects [get_pg_pins -of_objects [get_cells iA/out_reg] ]] is_ground`

`is_power`	Returns a value of `true` if the PG net is a power net.
	Return type: boolean
	
	For example,
	`get_property [get_pg_nets -of_objects [get_cells iA/out_reg] ] is_power`
	`get_property [get_pg_nets -of_objects [get_cells iA/out_reg] -filter "name =~ *VDD*" ] is_power`

`voltage_max`	Returns the maximum voltage of a given PG net. This could be voltage associated with a driver term of a net, power intent or CPF, `related_pg_pins` attribute in Liberty, or the operating voltage of a power domain or system default voltage. *Return type*: float For example, `get_property [get_pg_nets -of_objects [get_cells iA/out_reg] ] voltage_max`
`voltage_min`	Returns the minimum voltage of a given PG net. This could be voltage associated with a driver term of a net, power intent or CPF, `related_pg_pins` attribute in Liberty or the operating voltage of a power domain or system default voltage. *Return type*: float For example, `get_property [get_pg_nets -of_objects [get_cells iA/out_reg] ] voltage_min`
`object_type`	Returns `pg_net` as the object type for a library PG pin. *Return type*: string For example, `get_property [get_pg_nets -of_objects [get_cells iA/out_reg]] object_type`

`parent_pg_net`	Returns a collection of a parent power net that is the net powering the power switch that drives the PG net. In case of designs that do not have any power switches powering the instances, this property will always return the net directly powering those instances. It therefore be noted that the usage of this property is more relevant for designs in which the leaf cell instances are powered through power switch cells.
	Return type: Collection
	
	For example,
	`get_property [get_pg_nets mmu/U1/vdd] parent_pg_net`
	`get_property [get_pg_nets -of_objects [get_cells iA/pwr_swtch] ] parent_pg_net`

`top_pg_net`	Returns a collection of the top most un-switched power net of the current PG net. Even in the case of a cascading of power switches present in a design, the software traces through all power switches back to the un-switched top-level primary power net. The top-most un-switched power net will be returned - switched output power net will not be returned.
	Return type: Collection
	
	For example,
	`get_property [get_pg_nets -of_objects [get_cells iA/pwr_swtch] ] top_pg_net`
	`get_property [get_pg_nets -of_objects [get_pg_pins -of_objects [get_cells iA/pwr_swtch] ]] top_pg_net`

Path Group Properties

`name`	Returns the name of the path group. *Return type:* string
`hierarchical_name`	Returns the hierarchical name of the path group. *Return type:* string
`group_type`	Returns the type of group, when `group_path` and `-name`, `-from`, `-to` parameters are specified for the path. For example, `get_property [get_path_groups] group_type` *Return Type*: string
`object_type`	Returns the object type `path_group`. *Return type:* string
`view_name`	Returns the view name for the specified internal clock group. For example, `get_property [get_path_groups m_clk -include_internal_groups] view_name` *Return type:* string

Pin Properties

`actual_latency_early_fall_max`	When specified with the `set_clock_latency -clock` parameter, returns the worst actual early fall insertion delay in setup mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the `-clock` parameter, returns the worst actual early fall insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the `-view` parameter, returns the worst latency for the given view. *Return type:* float
`actual_latency_early_fall_min`	When specified with the `set_clock_latency -clock` parameter, returns the worst actual early fall insertion delay in hold mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the `-clock` parameter, returns the worst actual early fall insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the `-view` parameter, returns the worst latency for the given view. *Return type:* float
`actual_latency_early_rise_max`	When specified with the `set_clock_latency -clock` parameter, returns the worst actual early rise source insertion delay in setup mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the `-clock` parameter, returns the worst actual early rise source insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the `-view` parameter, returns the worst latency for the given view. *Return type:* float

`actual_latency_early_rise_min`	When specified with the `set_clock_latency -clock` parameter, returns the worst actual early rise source insertion delay in hold mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the `-clock` parameter, returns the worst actual early rise source insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the `-view` parameter, returns the worst latency for the given view. *Return type:* float
`actual_latency_late_fall_max`	When specified with the `set_clock_latency -clock` parameter, returns the worst actual late fall source insertion delay in setup mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the `-clock` parameter, returns the worst actual late fall source insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the `-view` parameter, returns the worst latency for the given view. *Return type:* float
`actual_latency_late_fall_min`	When specified with the `set_clock_latency -clock` parameter, returns the worst actual late fall source insertion delay in hold mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the `-clock` parameter, returns the worst actual late fall source insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the `-view` parameter, returns the worst latency for the given view. *Return type:* float
`actual_latency_late_rise_max`	When specified with the `set_clock_latency -clock` parameter, returns the worst actual late rise source insertion delay in setup mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the `-clock` parameter, returns the worst actual late rise source insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the `-view` parameter, returns the worst latency for the given view. *Return type:* float
`actual_latency_late_rise_min`	When specified with the `set_clock_latency -clock` parameter, returns the worst actual late rise source insertion delay in hold mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the `-clock` parameter, returns the worst actual late rise source insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the `-view` parameter, returns the worst latency for the given view. *Return type:* float
`arrival_clocks`	Returns the data phases, clock source data phases, and latency phases. *Return type:* collection The following global variables support these phases: • `timing_property_arrival_clocks_consider_clock_source_data_phase`: Considers the clock source data phases. • `timing_property_arrival_clocks_consider_data_phase`: Considers the pure data phase. • `timing_property_arrival_clocks_consider_latency_phase`: Considers the latency phase. By default, these global variables are set to `true`.
`arrival_max`	Returns the value of the arrival max at specified pin. You can verify through `report_timing`. For example, `get_property [get_pins <pin name>] arrival_max` *Return Type:* float

`arrival_max_fall`	Returns the arrival time for the falling transition at the max corner. *Return type:* float
`arrival_max_rise`	Returns the arrival time for the rising transition at the max corner. *Return type:* float
`arrival_mean_max_fall`	Returns the mean component of the arrival time for the falling transition at the max corner. *Return type:* float
`arrival_mean_max_rise`	Returns the mean component of the arrival time for the rising transition at the max corner. *Return type:* float
`arrival_mean_min_fall`	Returns the mean component of the arrival time for the falling transition at the min corner. *Return type:* float
`arrival_mean_min_rise`	Returns the mean component of the arrival time for the rising transition at the min corner. *Return type:* float
`arrival_min`	Returns the value of the arrival min at specified pin. You can verify through `report_timing`. For example, `get_property [get_pins <pin name>] arrival_min` *Return Type:* Float
`arrival_min_fall`	Returns the arrival time for the falling transition at the min corner. *Return type:* float
`arrival_min_rise`	Returns the arrival time for the rising transition at the min corner. *Return type:* float
`arrival_sigma_max_fall`	Returns the sigma component of the arrival time for the falling transition at the max corner. *Return type:* float
`arrival_sigma_max_rise`	Returns the mean component of the arrival time for the rising transition at the max corner. *Return type:* float
`arrival_sigma_min_fall`	Returns the sigma component of the arrival time for the falling transition at the min corner. *Return type:* float
`arrival_sigma_min_rise`	Returns the sigma component of the arrival time for the falling transition at the min corner. *Return type:* float
`arrival_window`	Returns the arrival times for the earliest and latest rising and falling transitions in timing window format. *Return type:* string
`attacker_fall_slew`	Returns the minimum falling slew of an attacker driver pin. For example: `get_property [get_pins inst1/y] attacker_fall_slew` *Return type:* float
`attacker_rise_slew`	Returns the minimum rising slew of an attacker driver pin. For example: `get_property [get_pins inst1/y] attacker_rise_slew` *Return type:* float
`capacitance_max`	Returns the value of the maximum pin capacitance specified for the capacitance attribute in the library for the corresponding pin. For example, `get_property [get_pins <pin_name>] capacitance_max` *Return Type:* float

`capacitance_max_fall`	Returns the maximum value of the falling capacitance range of the corresponding library pin. *Return type:* float
`capacitance_max_rise`	Returns the maximum value of the rising capacitance range of the corresponding library pin. *Return type:* float
`capacitance_min`	Returns the value of the minimum pin capacitance specified for the capacitance attribute in the library for the corresponding pin. For example, `get_property [get_pins <pin_name>] capacitance_min` *Return Type:* float
`capacitance_min_fall`	Returns the minimum value of the falling capacitance range of the corresponding library pin. *Return type:* float
`capacitance_min_rise`	Returns the minimum value of the rising capacitance range of the corresponding library pin. **Note**: For the `capacitance_max_*` and `capacitance_min_*` properties (described above), the capacitance value does not match with the `pin_load` (`report_timing -format`) in the timing report because the pin load is a sum of all pin capacitances of a net. *Return type:* float
`cell_name`	Returns the cell name for the corresponding pins. For example, `get_property [get_pins addrs_4_/A] cell_name` *Return type*: string

`clock_pulse_width_high`	Returns the actual high pulse width for the specified pin.
	`create_clock -name CK -period 30 -waveform {0 10} [get_ports {CK}]`
	You can verify through `report_min_pulse_width -clock <clock name>`
	For example,
	`get_property [get_pins <pin_name>] clock_pulse_width_high`
	Return Type: float

`clock_pulse_width_low`	Returns the actual low pulse width for the specified pin.
	`create_clock -name CK -period 30 -waveform {0 10} [get_ports {CK}]`
	You can verify through `report_min_pulse_width -clock <clock name>`
	For example,
	`get_property [get_pins <pin_name>] clock_pulse_width_low`
	Return Type: float

`clock_sources`	Returns a collection of the source pins of all the clocks arriving at the pin. *Return type:* collection
`clocks`	Returns a collection of all of the clocks arriving at the pin. *Return type:* collection
`constant_value`	Returns a constant value of `0` or `1`. *Return type:* integer
`delay_max_fall`	Returns the maximum falling delay - specified using the `set_max_delay -fall` command. *Return type:* float
`delay_max_rise`	Returns the maximum rising delay - specified using the `set_max_delay -rise` command. *Return type:* float

`delay_min_fall`	Returns the minimum falling delay - specified using the `set_min_delay -fall` command. *Return type:* float
`delay_min_rise`	Returns the minimum rising delay - specified using the `set_min_delay -rise` command. *Return type:* float
`direction`	Returns the direction of the pin: `in`, `out`, or `inout`. *Return type:* string
`driver_cell`	Returns the driver cell name for the corresponding pins. For example, `get_property [get_pins ON28_n_17/A] driver_cell` *Return type:* string
`driver_cell_name`	Returns the driving cell instance for the specified pin. For example, `get_property [get_pins <pin name>]driver_cell_name` *Return Type:* string
`driver_ref_lib_cell_name`	Returns the library defined driving cell for the specified pin. For example, `get_property [get_pins <pin name>] driver_ref_lib_cell_name` *Return Type:* string
`fanin`	Returns the number of sources connected with net of the specified pin. *Return type:* integer
`fanout`	Returns the number of sinks connected with the net of the specified pin. *Return type:* integer
`fanout_load`	Returns the fanout load for the pin. *Return type:* float

`fastest_clock`	Returns the collection of the clock name defined for the specified pin.
	`create_generated_clock -name ck -source CK -divide_by 2 <pin name>`
	For example,
	`get_property [get_pins <pin_name>] fastest_clock`
	Return Type: collection

`fastest_clock_name`	Returns the clock name defined for the specified pin.
	`create_generated_clock -name clk -source CLK -multiply_by 2 <pin name>`
	For example,
	`get_property [get_pins <pin name>] fastest_clock_name`
	Return Type: string

`fastest_clock_period`	Returns the clock period of the clock defined for the specified pin.
	`create_clock -name CLK -period 10 -waveform {0 5} [get_ports {CK2}]`
	`create_generated_clock -name clk -source CLK`
	`-multiply_by 3 <pin name>`
	You can verify through `report_clocks`.
	For example,
	`get_property [get_pins <pin_name>] fastest_clock_period`
	Return Type: float

`hierarchical_level`	Returns the hierarchical level of the pin. For example, `get_property [get_pins <pin name>] hierarchical_level` *Return Type:* integer
`hierarchical_name`	Returns the complete hierarchical name of the pin. *Return type:* string
`hold_uncertainty`	Returns the most conservative uncertainty of all possible uncertainty assertions associated with the pin. *Return type:* float

`holding_resistance_high`	Returns the highest holding resistance of a pin.
	For example,
	`foreach pin $pinList {`
	`set vicArray [get_property [get_pins $pin] noise_victim]`
	`foreach_in_collection vic $vicArray {`
	`get_property $vic holding_resistance_high`
	`}`
	Return type: float

`holding_resistance_low`	Returns the lowest holding resistance of a pin. *Return type:* float
`interp_info_max`	Returns information for late analysis. When specified, the report output will show the data for interpolation, extrapolation, and/or matched information for late analysis, along with the library details. *Return type:* string
`interp_info_min`	Returns information for early analysis. When specified, the report output will show the data for interpolation, extrapolation, and/or matched information for early analysis, along with the library details. *Return type:* string
`is_async`	Returns a value of `true` if the pin is an asynchronous clear or preset pin. *Return type:* boolean
`is_clear`	Returns a value of `true` if the pin is an asynchronous clear pin. *Return type:* boolean
`is_clock`	Returns a value of `true` if the pin has the Liberty pin attribute: `clock`. *Return type:* boolean
`is_clock_gating`	Returns a value of `true` if the pin is defined as a pin of a clock gating cell. *Return type:* boolean
`is_clock_gating_clock`	Returns a value of `true` if the pin is defined as a clock pin of a clock gating cell. *Return type:* boolean
`is_clock_gating_enable`	Returns a value of `true` if the pin corresponds to the enable pin of a clock gating cell. *Return type:* boolean

`is_clock_root`	Returns `true` for a pin or port if real clock or generated clock is defined on the pin/port.
	`create_generated_clock -name clk -source CLK -multiply_by 2 <pin name>`
	For example,
	`get_property [get_pins <pin_name>] is_clock_root`
	Return Type: boolean

`is_clock_used_as_clock`	Returns a value of `true` if the pin lies in the clock network and at least one of the clocks arriving at the pin is used as a clock in the downstream network of the pin. When the `-clock` parameter is specified, this property returns `true` if the above condition is satisfied for the specified clock. *Return type*: boolean
`is_clock_used_as_data`	Returns a value of `true` if the pin lies in the clock source path and at least one of the clocks arriving on the pin is used as data in the downstream network of the pin. When the `-clock` parameter is specified, this property returns `true` if the above condition is satisfied for the specified clock. *Return type*: boolean
`is_data`	Returns a value of `true` if the pin is a data pin of a flip-flop (that is, is not a clock pin). *Return type:* boolean
`is_disable_timing`	Returns a value of `true` if the pin's timing has been disabled. Return type: boolean **Note**: This property checks if the associated arc for the pin is active for propagation and analysis.
`is_fall_edge_triggered_clock`	Returns a value of `true` if the pin is a clock pin of a flop, and is triggered by the falling edge of a clock. *Return type:* boolean
`is_fall_edge_triggered_data`	Returns a value of `true` if the pin corresponds to the data pin of a fall edge triggered device. *Return value:* boolean
`is_hierarchical`	Returns a value of `true` if the pin is a pin of a hierarchical module. *Return type:* boolean
`is_isolated`	Returns a value of `true` when a pin is connected to an isolated instance. The `is_isolation` attribute indicates that the isolation cell is inside a module. So if the module is used as macro or black-box, the software will interpret the pin to be already isolated and no additional isolation will be required. *Return type:* boolean
`is_isolation_cell_data`	Returns `true` if the instance signal pin corresponding to the `isolation_cell_data_pin` attribute of a library signal pin is set to `true`, indicating that the library cell signal pin is a data pin of an isolation library cell. **Note** : The `is_isolation_cell` attribute must be set to `true` in the library. For example, `get_property [get_pins <pin_name>] is_isolation_cell_data` *Return Type:* boolean
`is_isolation_cell_enable`	Returns `true` if the instance signal pin corresponding to the `isolation_cell_enable_pin` attribute of a library signal pin is set to `true`, indicating that the library cell signal pin is an enable or control pin of an isolation library cell. **Note** : The `is_isolation_cell` attribute must be set to `true` in the library. For example, `get_property [get_pins <pin_name>] is_isolation_cell_enable` *Return Type:* boolean

`is_latency_network_pin`	Returns `true` if the specified pin is having generated or created clock.
	`create_generated_clock -name ck -source CK -divide_by 2 <pin name>`
	For example,
	`get_property [get_pins <pin name>] is_latency_network_pin`
	Return Type: boolean

is_level_shifter_enable	Returns true if the instance signal pin corresponding to the level_shifter_enable_pin attribute of a library signal pin is set to true, indicating that the library cell signal pin is a enable pin of an isolation library cell. **Note:** The is_level_shifter attribute must be set to true in the library. For example, get_property [get_pins <pin_name>] is_isolation_cell_enable *Return Type:* boolean
is_level_shifter_data	Returns true if the instance signal pin corresponding to the level_shifter_data_pin attribute of a library signal pin is set to true, indicating that the library cell signal pin is a data pin of an isolation library cell. **Note:** The is_level_shifter attribute must be set to true in the library. For example, get_property [get_pins <pin_name>] is_level_shifter_data" *Return Type:* boolean
is_multiple_clock_fanin_point	Returns a value of true if multiple clock phases converge at the pin. *Return type:* boolean
is_mux_select_pin	Returns true if the specified pin is the mux select pin. For example, get_property [get_pins m1/S] is_mux_select_pin *Return Type:* boolean
is_negative_level_sensitive_clock	Returns a value of true if the library pin is an enable pin of an active low level-sensitive device. *Return type:* boolean
is_negative_level_sensitive_data	Returns a value of true if the pin is a data pin of an active low level-sensitive device. *Return type:* boolean
is_pin	Returns a value of true. The is_pin and is_port queries are provided for both pin and port objects to provide a quick way of determining the object type of an identifier. is_port always returns false for pins. is_pins always returns false for ports. *Return type:* boolean
is_port	Returns a value of false. The is_pin and is_port queries are provided for both pin and port objects to provide a quick way of determining the object type of an identifier. is_port always returns false for pins. is_pins always returns false for ports. *Return type:* boolean
is_positive_level_sensitive_clock	Returns a value of true if the library pin is an enable pin of an active high level-sensitive device. *Return type:* boolean
is_positive_level_sensitive_data	Returns a value of true if the pin is a data pin of an active high level-sensitive device. *Return type:* boolean
is_preset	Returns a value of true if the pin is a preset pin. *Return type:* boolean
is_propagated_clock	Returns a value of true if there is an explicit set_propagated_clock assertion at the pin. *Return type:* boolean
is_rise_edge_triggered_clock	Returns a value of true if the pin is a clock pin of a flop, and is triggered by the rising edge of a clock. *Return type:* boolean
is_rise_edge_triggered_data	Returns a value of true if the pin corresponds to the data pin of a rise edge-triggered device. *Return type:* boolean

`is_tristate`	Returns a value of `true` if the pin has the `three_state` attribute in the Liberty timing library. *Return type:* boolean
`is_tristate_enable`	Returns a value of `true` if the pin is the source pin of timing arcs with either the `three_state_enable` or `three_state_disable` attribute in the Liberty timing library. *Return type:* boolean
`is_tristate_output`	Returns a value of `true` if the pin corresponds to a three-state output pin. *Return type:* boolean
`latency_fall_max`	Returns the maximum fall insertion delay specified by an explicit `set_clock_latency` at the pin. *Return type:* float
`latency_fall_min`	Returns the minimum fall insertion delay specified by an explicit `set_clock_latency` at the pin. *Return type:* float
`latency_rise_max`	Returns the maximum rise insertion delay specified by an explicit `set_clock_latency` at the pin. *Return type:* float
`latency_rise_min`	Returns the minimum rise insertion delay specified by an explicit `set_clock_latency` at the pin. *Return type:* float
`lib_pins`	Returns the collection of the library pin for the specified pin. For example, `get_property [get_pins <pin name>] lib_pins` *Return Type:* collection
`max_capacitance`	Returns the maximum capacitance limit for the pin. *Return type:* float
`max_fanout`	Returns the maximum fanout load that the pin can drive. This value is set using `set_max_fanout` or the `default_max_fanout` library attribute. *Return type:* float
`max_time_borrow`	Returns the value of the maximum time borrowed for the specified pin when the `set_max_time_borrow` parameter is specified. For example, `get_property  [get_pins <pin name> ] max_time_borrow` *Return Type:* float
`max_transition`	Returns the maximum transition time limit specified for the pin. *Return type:* float
`min_capacitance`	Returns the value of the minimum capacitance for the specified pin when the `set_min_capacitance` parameter is specified. For example, `get_property [get_pins <pin name>] min_capacitance` *Return Type:* float
`min_fanout`	Returns the minimum fanout design rule limit of the corresponding library pin. *Return type:* float
`min_transition`	Returns the value of the minimum transition for the specified pin. For example, `get_property [get_pins <pin name>] min_transition` *Return Type:* float

`name`	Returns the leaf-level name of the pin. For example, `get_property [get_pins <pin name>] name` *Return Type:* string
`net_name`	Returns the name of the net connected to the specified pin. *Return type:* string
`noise_victim`	Returns a collection of `si_victim` objects that can be used to query glitch attributes. *Return type:* collection
`object_type`	Returns the object type `pin`. *Return type:* string
`power_rail_voltage_bidir_input_max`	Reports power rail voltage for input part of bidi pins/ports for maximum operating condition. *Return type:* float
`power_rail_voltage_bidir_input_min`	Reports power rail voltage for input part of bidi pins/ports for minimum operating condition. *Return type:* float
`power_rail_voltage_max`	Reports power rail voltage for maximum operating condition for bidirectional, input and output pins. For bidirectional pins, the power rail voltage for output signal is reported. *Return type:* float
`power_rail_voltage_min`	Reports power rail voltage for minimum operating condition for bidirectional, input and output pins. For bidirectional pins, the power rail voltage for output signal is reported. *Return type:* float
`receiver_ccc_type`	Returns the type of receiver cell model. The return values can be udn, single_ccc, dual_ccc, multi_ccc, or black_box. This attribute applies to receiver pins only. For example, `get_property [get_pins inst1/a] receiver_ccc_type` *Return type:* string
`ref_lib_pin_name`	Returns the name of the library pin associated with the instance pin. *Return type:* string
`setup_uncertainty`	Returns the most conservative uncertainty of all possible uncertainty assertions associated with the pin. *Return type:* float
`si_slew_max`	Returns the maximum slew from all the incoming signals after performing SI analysis. This value is a maximum of `si_slew_max_rise` and `si_slew_max_fall`. *Return type:* float
`si_slew_mean_max_fall`	Returns the mean component of maximum falling slew from the incoming signals after SI analysis is performed. *Return type:* float
`si_slew_mean_max_rise`	Returns the mean component of maximum rising slew from the incoming signals after SI analysis is performed. *Return type:* float
`si_slew_mean_min_fall`	Returns the mean component of minimum falling slew from the incoming signals after SI analysis is performed. *Return type:* float
`si_slew_mean_min_rise`	Returns the mean component of minimum rising slew from the incoming signals after SI analysis is performed. *Return type:* float
`si_slew_min`	Returns the minimum slew from all the incoming signals after performing SI analysis. This value is a minimum of `si_slew_min_rise` and `si_slew_min_fall`. *Return type:* float

`si_victim`	Returns a collection of the `si_victim` objects that can be used to query graph-based SI delay attributes. *Return type:* collection
`slack_max`	Returns the slack time for a timing path at the max corner. *Return type:* float
`slack_max_edge`	Returns the edge (rise or fall) of the worst slack-causing path at the specified pin in late mode. *Return type:* string
`slack_max_fall`	Returns the slack time for the falling transition at the max corner. *Return type:* float
`slack_max_rise`	Returns the slack time for the rising transition at the max corner. When the `-view` parameter is not specified, the worst slack for all the views is returned. *Return type:* float
`slack_mean_max`	Returns the maximum mean component of the slack time for a timing path. *Return type:* float
`slack_mean_min`	Returns the minimum mean component of the slack time for a timing path. *Return type:* float
`slack_mean_max_fall`	Returns the mean component of the slack time for the falling transition at the max corner. *Return type:* float
`slack_mean_max_rise`	Returns the mean component of the slack time for the rising transition at the max corner. *Return type:* float
`slack_mean_min_fall`	Returns the mean component of the slack time for the falling transition at the min corner. *Return type:* float
`slack_mean_min_rise`	Returns the mean component of the slack time for the rising transition at the min corner. Return type: float
`eslack_min`	Returns the slack time for a timing path at the min corner. *Return type:* float
`slack_min_edge`	Returns the edge (rise or fall) of the worst slack-causing path at the specified pin in early mode. *Return type:* string
`slack_min_fall`	Returns the slack time for the falling transition at the min corner. *Return type:* float
`slack_min_rise`	Returns the slack time for the rising transition at the min corner. *Return type:* float
`slack_sigma_max_fall`	Returns the sigma component of the slack time for the falling transition at the max corner. *Return type:* float
`slack_sigma_max_rise`	Returns the sigma component of the slack time for the rising transition at the max corner. *Return type:* float
`slack_sigma_min_fall`	Returns the sigma component of the slack time for the falling transition at the min corner. *Return type:* float
`slack_sigma_min_rise`	Returns the sigma component of the slack time for the rising transition at the min corner. *Return type:* float

`slew_max`	Returns the value of the maximum slew for the specified pin. You can verify through `report_timing`. For example, `get_property [get_pins <pin name>] slew_max` *Return Type:* float
`slew_max_fall`	Returns the slew time for the falling transition at the max corner. *Return type:* float
`slew_max_rise`	Returns the slew time for the rising transition at the max corner. *Return type:* float
`slew_mean_max_fall`	Returns the mean component of the slew time for the falling transition at the max corner. *Return type:* float
`slew_mean_max_rise`	Returns the mean component of the slew time for the rising transition at the max corner. *Return type:* float
`slew_mean_min_fall`	Returns the mean component of the slew time for the falling transition at the min corner. *Return type:* float
`slew_mean_min_rise`	Returns the mean component of the slew time for the rising transition at the min corner. *Return type:* float
`slew_min`	Returns the value of the minimum slew for the specified pin. You can verify through `report_timing`. For example, get_property [get_pins <pin name>] slew_min *Return Type:* float
`slew_min_fall`	Returns the slew time for the falling transition at the min corner. *Return type:* float
`slew_min_rise`	Returns the slew time for the rising transition at the min corner. *Return type:* float
`slew_sigma_max_fall`	Returns the sigma component of the slew time for the falling transition at the max corner. *Return type:* float
`slew_sigma_max_rise`	Returns the sigma component of the slew time for the rising transition at the max corner. *Return type:* float
`slew_sigma_min_fall`	Returns the sigma component of the slew time for the falling transition at the min corner. *Return type:* float
`slew_sigma_min_rise`	Returns the sigma component of the slew time for the falling transition at the min corner. *Return type:* float
`source_latency_early_fall_max`	Returns the maximum early fall source insertion delay specified by an explicit `set_clock_latency` at the pin. *Return type:* float
`source_latency_early_fall_min`	Returns the minimum early fall source insertion delay specified by an explicit `set_clock_latency` at the pin. *Return type:* float
`source_latency_early_rise_max`	Returns the maximum early rise source insertion delay specified by an explicit `set_clock_latency` at the pin. *Return type:* float
`source_latency_early_rise_min`	Returns the minimum early rise source insertion delay specified by an explicit `set_clock_latency` at the pin. *Return type:* float

`source_latency_late_fall_max`	Returns the maximum late fall source insertion delay specified by an explicit `set_clock_latency` at the pin. *Return type:* float
`source_latency_late_fall_min`	Returns the minimum late fall source insertion delay specified by an explicit `set_clock_latency` at the pin. *Return type:* float
`source_latency_late_rise_max`	Returns the maximum late rise source insertion delay specified by an explicit `set_clock_latency` at the pin. *Return type:* float
`source_latency_late_rise_min`	Returns the minimum late rise source insertion delay specified by an explicit `set_clock_latency` at the pin. *Return type:* float
`stolen_slack`	Returns the slack stolen at the specified pin. This property can be queried only for pins in late mode. **Note:** The specified pin must be the endpoint of a transparent latch flush arc. *Return type:* float
`user_constant_value`	Returns constant values from netlist or constraints. *Return type*: integer
`x_coordinate`	Returns the x coordinates of the placed pin. *Return type:* float
`y_coordinate`	Returns the y coordinates of the placed pin. *Return type:* float

Pin Properties for Power/Ground Objects	
`input_signal_level`	Returns the rail name of the `related_power_pin` of an input signal pin. *Return type:* string For example, `get_property [get_pins mmu/U1/I] input_signal_level`
`input_signal_level_voltage`	Returns the rail voltage of a rail of the `related_power_pin` attribute of the input signal pin. *Return type:* float For example, `get_property [get_pins mmu/U1/I] input_signal_level_voltage`
`is_always_on`	Returns a value of `true` if an instance pin is an always-on pin. *Return type:* boolean For example, `get_property [get_pins mmu/U1/AON] is_always_on`
`is_isolation_cell_data`	Returns a value of `true` if the instance signal pin is the data pin of an isolation cell instance. *Return type:* boolean For example, `get_property [get_pins mmu/U2/ISO] is_isolation_cell_data`
`is_isolation_cell_enable`	Returns a value of `true` if the instance signal pin is the enable pin of an isolation cell instance. *Return type:* boolean For example, `get_property [get_pins mmu/U2/ISO_EN] is_isolation_cell_enable`
`is_level_shifter_data`	Returns a value of `true` if the instance signal pin is the data pin of a level-shifter cell instance. *Return type:* boolean For example, `get_property [get_pins mmu/LVL/IN] is_level_shifter_data`

`is_level_shifter_enable`	Returns a value of `true` if the instance signal pin is the enable pin of a level-shifter cell instance. *Return type:* boolean For example, `get_property [get_pins mmu/LVL/EN] is_level_shifter_enable`
`output_signal_level`	Returns the rail name of the `related_power_pin` attribute of the output/inout signal pin. *Return type:* string For example, `get_property [get_pins mmu/U1/Y] output_signal_level`
`related_ground_pin`	Returns the related ground pins as a collection for the corresponding instance pins. *Return type:* collection For example, `get_property [get_pins mmu/U1/I] related_ground_pin`
`related_power_pin`	Returns the related power pins as a collection for the corresponding instance pins. *Return type:* collection For example, `get_property [get_pins mmu/U1/I] related_power_pin`

Pin Properties for Power/Ground Objects

`hierarchical_name`	Returns the complete hierarchical name of the PG pin.
	Return type: string
	
	For example,
	`get_property [get_pg_pins -of_objects [get_cells iA/out_reg] ] hierarchical_name`
	`get_property [get_pg_pins -of_objects [get_cells iA/out_reg] -filter "pg_type == primary_power" ] hierarchical_name`

`voltage_max`	Returns the maximum voltage of a given PG pin. This could be the voltage associated with a pin, power intent or CPF, `related_pg_pins` attribute in Liberty, or the operating voltage of a power domain or system default voltage. *Return type:* float For example, `get_property [get_pg_pins -of_objects [get_cells iA/out_reg]] voltage_max`
`voltage_min`	Returns the minimum voltage of a given PG pin. This voltage may be associated with a pin, power intent or CPF, `related_pg_pins` attribute in Liberty, or the operating voltage of a power domain or system default voltage. *Return type*: float For example, `get_property [get_pg_pins -of_objects [get_cells iA/out_reg]] voltage_min`
`net`	Returns the immediate PG net(s) connected to the corresponding PG pin objects, as a collection. *Return type*: Collection For example, `get_property [get_pg_pins -of_objects [get_cells iA/out_reg]] net`

`pg_type`	Returns the type of PG pin.
	Return type: string
	
	For example,
	`get_property [get_pg_pins -of_objects [get_cells iA/out_reg]] pg_type`
	`get_property [get_pg_pins -of_objects [get_cells iA/out_reg] -filter "hierarchical_name =~ *VDD*" ] pg_type`

`ref_lib_pin_name`	Returns the name of the library PG pin associated with an instance PG pin. *Return type*: string For example, `get_property [get_pg_pins -of_objects [get_cells iA/out_reg]] ref_lib_pin_name`

Port Properties

`arrival_max_fall`	Returns the worst arrival on the port for a falling signal. *Return type:* float
`arrival_max_rise`	Returns the worst arrival on the port for a rising signal. *Return type:* float
`arrival_min_fall`	Returns the best arrival on the port for a falling signal. *Return type:* float
`arrival_min_rise`	Returns the best arrival on the port for a rising signal. *Return type:* float
`arrival_window`	Returns the arrival times for the earliest and latest rising and falling transitions in timing window format. *Return type:* string
`clock_sources`	Returns a collection of the source pins of all the clocks arriving at the port. *Return type:* collection
`clocks`	Returns a collection of all of the clocks arriving at the port. *Return type:* collection
`constant_value`	Returns a constant value of `0` or `1`. *Return type:* integer
`capacitance_max`	Returns the maximum pin capacitance for the port. *Return type:* float
`capacitance_min`	Returns the minimum pin capacitance for the port. *Return type:* float
`delay_max_fall`	Returns the maximum falling delay value for the port. *Return type:* float
`delay_max_rise`	Returns the maximum rising delay value for the port. *Return type:* float
`delay_min_fall`	Returns the minimum falling delay value for the port. *Return type:* float
`delay_min_rise`	Returns the minimum rising delay value for the port. *Return type:* float
`direction`	Returns the direction of the port: `in`, `out`, or `inout`. *Return type:* string
`driving_cell_no_design_rule`	Returns a value of `true` if there is no design rule transfer from the driving cell set on a port. For example, if a driving cell is defined with the following command, then this property will return `true`: `set_driving_cell -lib_cell <lib_cell_name> -no_design_rule <port>`
`drive_resistance_fall_max`	Returns the falling linear drive resistance at the port for the max condition. *Return type:* float
`drive_resistance_fall_min`	Returns the falling linear drive resistance at the port for the min condition. *Return type:* float

`drive_resistance_rise_max`	Returns the rising linear drive resistance at the port for the max condition. *Return type:* float
`drive_resistance_rise_min`	Returns the rising linear drive resistance at the port for the min condition. *Return type:* float
`driving_cell_fall_max`	Returns the name of the library cell used to compute max falling delays and transitions at the port. *Return type:* string
`driving_cell_fall_min`	Returns the name of the library cell used to compute min falling delays and transitions at the port. *Return type:* string
`driving_cell_from_pin_fall_max`	Returns the input pin of the driving cell used in the max fall delay calculation at the port. *Return type:* string
`driving_cell_from_pin_fall_min`	Returns the input pin of the driving cell used in the min fall delay calculation at the port. *Return type:* string
`driving_cell_from_pin_rise_max`	Returns the input pin of the driving cell used in max rise delay calculation at the port. *Return type:* string
`driving_cell_from_pin_rise_min`	Returns the input pin of the driving cell used in min rise delay calculation at the port. *Return type:* string
`driving_cell_library_fall_max`	Returns the name of the library used to compute max falling delays and transitions at the port. *Return type:* string
`driving_cell_library_fall_min`	Returns the name of the library used to compute min falling delays and transitions at the port. *Return type:* string
`driving_cell_library_rise_max`	Returns the name of the library used to compute max rising delays and transitions at the port. *Return type:* string
`driving_cell_library_rise_min`	Returns the name of the library used to compute min rising delays and transitions at the port. *Return type:* string
`driving_cell_max_fall_itrans_fall`	Returns the falling input transition of the `from_pin` of the driving cell, which is used for the max falling delay calculation at the port. *Return type:* float
`driving_cell_max_fall_itrans_rise`	Returns the falling input transition of the `from_pin` of the driving cell, which is used for the max rising delay calculation at the port. *Return type:* float
`driving_cell_max_rise_itrans_fall`	Returns the rising input transition of the `from_pin` of the driving cell, which is used for the max falling delay calculation at the port. *Return type:* float
`driving_cell_max_rise_itrans_rise`	Returns the rising input transition of the `from_pin` of the driving cell, which is used for the max rising delay calculation at the port. *Return type:* float
`driving_cell_min_fall_itrans_fall`	Returns the falling input transition of the `from_pin` of the driving cell, which is used for the min falling delay calculation at the port. *Return type:* float
`driving_cell_min_fall_itrans_rise`	Returns the falling input transition of the `from_pin` of the driving cell, which is used for the min rising delay calculation at the port. *Return type:* float

`driving_cell_min_rise_itrans_fall`	Returns the rising input transition of the `from_pin` of the driving cell, which is used for the min falling delay calculation at the port. *Return type:* float
`driving_cell_min_rise_itrans_rise`	Returns the rising input transition of the `from_pin` of the driving cell, which is used for the min rising delay calculation at the port. *Return type:* float
`driving_cell_pin_fall_max`	Returns the name of the output pin used for max fall delay calculation at the port. *Return type:* string
`driving_cell_pin_fall_min`	Returns the name of the output pin used for min fall delay calculation at the port. *Return type:* string
`driving_cell_pin_rise_max`	Returns the name of the output pin used for max rise delay calculation at the port. *Return type:* string
`driving_cell_pin_rise_min`	Returns the name of the output pin used for min rise delay calculation at the port. *Return type:* string
`driving_cell_rise_max`	Returns the name of the library cell used to compute max rising delays and transitions at the port. *Return type:* string
`driving_cell_rise_min`	Returns the name of the library cell used to compute min rising delays and transitions at the port. *Return type:* string
`fanin`	Returns the number of sources connected with net of the specified port. *Return type:* integer
`fanout`	Returns the number of sinks connected with the net of the specified port. *Return type:* integer
`fanout_load`	Returns the fanout load for ports. (For input ports, the value is the sum of all fanout loads of all connected pins. For output ports, the value is set using `set_fanout_load`.) *Return type:* float
`hierarchical_name`	Returns the complete hierarchical name of the port. *Return type:* string
`hold_uncertainty`	Returns the most conservative uncertainty of all possible uncertainty assertions associated with the port. *Return type:* float
`input_slew_max_fall`	Returns the max falling slew for the input or inout ports. (This value is set with the `set_input_transition` command.) *Return type:* float
`input_slew_max_rise`	Returns the max rising slew for the input or inout ports. (This value is set with the `set_input_transition` command.) *Return type:* float
`input_slew_min_fall`	Returns the min falling slew for the input or inout ports. (This value is set with the `set_input_transition` command.) *Return type:* float
`input_slew_min_rise`	Returns the min rising slew for the input or inout ports. (This value is set with the `set_input_transition` command.) *Return type:* float
`is_disable_timing`	Returns a value of `true` if the timing of the port has been disabled using the `set_disable_timing` command. *Return type:* boolean
`is_hierarchical`	Returns a value of `true` if the port is a port of a hierarchical module. *Return type:* boolean

is_pin	Returns a value of false. The is_pin and is_port queries are provided for both pin and port objects to provide a quick way of determining the object type of an identifier. is_port always returns false for pins. is_pins always returns false for ports. *Return type:* boolean
is_port	Returns a value of true. The is_pin and is_port queries are provided for both pin and port objects to provide a quick way of determining the object type of an identifier. is_port always returns false for pins. is_pins always returns false for ports. *Return type:* boolean
is_propagated_clock	Returns a value of true if there is an explicit set_propagated_clock assertion at the port. *Return type:* boolean
latency_fall_max	Returns the maximum fall insertion delay specified by an explicit set_clock_latency at the port. *Return type:* float
latency_fall_min	Returns the minimum fall insertion delay specified by an explicit set_clock_latency at the port. *Return type:* float
latency_rise_max	Returns the maximum rise insertion delay specified by an explicit set_clock_latency at the port. *Return type:* float
latency_rise_min	Returns the minimum rise insertion delay specified by an explicit set_clock_latency at the port. *Return type:* float
max_fanout	Returns the maximum fanout load that the port can drive. (This value is set using set_max_fanout, or the default_max_fanout library attribute.) *Return type:* float
max_transition	Returns the maximum transition time limit specified for the port. *Return type:* float
net_name	Returns the name of the net connected to the specified port. *Return type:* string
noise_victim	Returns a collection of si_victim objects that can be used to query glitch attributes. *Return type:* collection
object_type	Returns the object type port. *Return type:* string
setup_uncertainty	Returns the most conservative uncertainty of all possible uncertainty assertions associated with the port. *Return type:* float
si_victim	Returns a collection of the si_victim objects that can be used to query graph-based SI delay attributes. *Return type:* collection
slack_max_edge	Returns the edge (rise or fall) of the worst slack-causing path at the specified port in late mode. *Return type:* string
slack_max_fall	Returns the worst setup slack for a falling signal at the port endpoint. *Return type:* float
slack_max_rise	Returns the worst setup slack for a rising signal at the port endpoint. When the -view parameter is not specified, the worst slack for all the views is returned. *Return type:* float

`slack_min_edge`	Returns the edge (rise or fall) of the worst slack-causing path at the specified port in early mode. *Return type:* string
`slack_min_fall`	Returns the worst hold slack for a falling signal at the port endpoint. *Return type:* float
`slack_min_rise`	Returns the worst hold slack for a rising signal at the port endpoint. *Return type:* float
`slew_max_fall`	Returns the maximum falling slew from all of the incoming signals on the port. *Return type:* float
`slew_max_rise`	Returns the maximum rising slew from all of the incoming signals on the port. *Return type:* float
`slew_min_fall`	Returns the minimum falling slew from all of the incoming signals on the port. *Return type:* float
`slew_min_rise`	Returns the minimum rising slew from all of the incoming signals on the port. *Return type:* float
`source_latency_early_fall_max`	Returns the maximum early fall source insertion delay specified by an explicit `set_clock_latency` at the port. *Return type:* float
`source_latency_early_fall_min`	Returns the minimum early fall source insertion delay specified by an explicit `set_clock_latency` at the port. *Return type:* float
`source_latency_early_rise_max`	Returns the maximum early rise source insertion delay specified by an explicit `set_clock_latency` at the port. *Return type:* float
`source_latency_early_rise_min`	Returns the minimum early rise source insertion delay specified by an explicit `set_clock_latency` at the port. *Return type:* float
`source_latency_late_fall_max`	Returns the maximum late fall source insertion delay specified by an explicit `set_clock_latency` at the port. *Return type:* float
`source_latency_late_fall_min`	Returns the minimum late fall source insertion delay specified by an explicit `set_clock_latency` at the port. *Return type:* float
`source_latency_late_rise_max`	Returns the maximum late rise source insertion delay specified by an explicit `set_clock_latency` at the port. *Return type:* float
`source_latency_late_rise_min`	Returns the minimum late fall source insertion delay specified by an explicit `set_clock_latency` at the port. *Return type:* float
`user_constant_value`	Returns constant values from netlist or constraints. *Return type*: integer
`x_coordinate`	Returns the x coordinates of the placed port. *Return type:* float
`y_coordinate`	Returns the y coordinates of the placed port. *Return type:* float

Timing Arc Properties

`aocv_derate_capture_clock_early_fall`	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of early capture clock paths with fall sink pin transitions. *Return type:* float
`aocv_derate_capture_clock_early_rise`	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of early capture clock paths with rise sink pin transitions. *Return type:* float
`aocv_derate_capture_clock_late_fall`	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of late capture clock paths with fall sink pin transitions. *Return type:* float
`aocv_derate_capture_clock_late_rise`	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of late capture clock paths with rise sink pin transitions. *Return type:* float
`aocv_derate_data_early_fall`	Returns AOCV derate values for a timing arc on an early data path with fall transitions. *Return type:* float
`aocv_derate_data_early_rise`	Returns AOCV derate values for a timing arc on an early data path with rise transitions. *Return type:* float
`aocv_derate_data_late_fall`	Returns AOCV derate values for a timing arc on a late data path with fall transitions. *Return type:* float
`aocv_derate_data_late_rise`	Returns AOCV derate values for a timing arc on a late data path with rise transitions. *Return type:* float
`aocv_derate_launch_clock_early_fall`	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of early launch clock paths with fall sink pin transitions. *Return type:* float
`aocv_derate_launch_clock_early_rise`	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of early launch clock paths with rise sink pin transitions. *Return type:* float
`aocv_derate_launch_clock_late_fall`	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of late launch clock paths with fall sink pin transitions. *Return type:* float
`aocv_derate_launch_clock_late_rise`	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of late launch clock paths with rise sink pin transitions. *Return type:* float
`aocv_stage_count_capture_clock_early`	Returns graph based AOCV stage count values in AOCV mode for a given timing arc that is a part of early capture clock paths. *Return type:* float
`aocv_stage_count_capture_clock_late`	Returns graph based AOCV stage count values in AOCV mode for a given timing arc that is a part of late capture clock paths. *Return type:* float
`aocv_stage_count_data_early`	Returns AOCV stage count values for a timing arc on an early data path. *Return type:* float
`aocv_stage_count_data_late`	Returns AOCV stage count values for a timing arc on a late data path. *Return type:* float
`aocv_stage_count_launch_clock_early`	Returns AOCV stage count values in AOCV mode for a given timing arc that is a part of early launch clock paths. *Return type:* float

`aocv_stage_count_launch_clock_late`	Returns AOCV stage count values in AOCV mode for a given timing arc that is a part of late launch clock paths. *Return type:* float
`arc_type`	Returns the Liberty `timing_type` attribute of the arc. *Return type:* string
`delay_max_fall`	Returns the fall delay for the max corner. *Return type:* float
`delay_max_rise`	Returns the rise delay for the max corner. *Return type:* float
`delay_mean_max_fall`	Returns the mean component of the maximum falling delay value for the timing arc. *Return type*: float
`delay_mean_max_rise`	Returns the mean component of the maximum rising delay value for the timing arc. *Return type*: float
`delay_mean_min_fall`	Returns the mean component of the minimum falling delay value for the timing arc. *Return type*: float
`delay_mean_min_rise`	Returns the mean component of the minimum rising delay value for the timing arc. *Return type*: float
`delay_min_fall`	Returns the fall delay for the min corner. *Return type:* float
`delay_min_rise`	Returns the rise delay for the min corner. *Return type:* float
`delay_sigma_max_fall`	Returns the 1-sigma variation component of the maximum falling delay for the specified timing arc. *Return type*: float
`delay_sigma_max_rise`	Returns the 1-sigma variation component of the maximum rising delay for the specified timing arc. *Return type*: float
`delay_sigma_min_fall`	Returns the 1-sigma variation component of the minimum falling delay for the specified timing arc. *Return type*: float
`delay_sigma_min_rise`	Returns the 1-sigma variation component of the minimum rising delay for the specified timing arc. Return type: float
`delta_delay_max_fall`	Returns the delta fall delay for the max corner. *Return type:* float
`delta_delay_max_rise`	Returns the delta rise delay for the max corner. *Return type:* float
`delta_delay_min_fall`	Returns the delta fall delay for the min corner. *Return type:* float
`delta_delay_min_rise`	Returns the delta rise delay for the min corner. *Return type:* float
`is_cellarc`	Returns a value of `true` if the arc belongs to a cell. *Return type:* boolean
`is_disabled`	Returns a value of true if `set_disable_timing` constraint exists for an arc. *Return type:* boolean

`mode`	Returns the mode specified for the timing arc in the library. *Return type:* string
`object_type`	Returns the object type `timing_arc`. *Return type:* string
`sdf_cond`	Returns the `sdf_cond` expression specified for the timing arc in the library. *Return type:* string
`sdf_cond_end`	Returns the `sdf_cond_end` expression specified for the timing arc in the library. *Return type:* string
`sdf_cond_start`	Returns the `sdf_cond_start` expression specified for the timing arc in the library. *Return type:* string
`sense`	Returns the sense of the timing arc. *Return type:* string
`sink_pin`	Returns the sink pin of the delay arc, or the reference pin of the check arc. *Return type:* collection
`source_pin`	Returns the source pin of the delay arc, or the signal pin of the check arc. *Return type:* collection
`when`	Returns the `when` condition specified for the timing arc in the library. *Return type:* string

Timing Path Properties

`applied_exception_type`	Returns the type of applied exceptions for a timing path. This property can be used to query dominant timing exceptions affecting a timing path. You can use this property to query on constrained/unconstrained timing paths. The possible return values are - multi_cycle, path_delay and false_path - indicating multicycle path, minimum/maximum delay path, and timing false path, respectively. The software will return one of these values for pin-based timing exceptions. However, the software returns “NA” when false paths are defined for clock objects, or if the paths do not have any related timing exception. *Return Type*: String
`arrival`	Returns the arrival time at the timing path. *Return type*: float
`arrival_mean`	Returns the mean component of the arrival time for the timing path. *Return type*: float
`arrival_sigma`	Returns the sigma component of the arrival time for the timing path. *Return type*: float
`capture_clock_path`	Returns the capturing clock path of the timing path. This property can be used only for path_type full_clock. *Return type:* collection
`capturing_clock`	Returns the capturing clock of the timing path. *Return type:* collection

`capturing_clock_close_edge_time`	Returns the closing edge time of the capturing clock. *Return type:* float
`capturing_clock_close_edge_type`	Returns the capturing clock edge direction: `rise` or `fall`. *Return type:* string
`capturing_clock_is_inverted`	Returns a value of true if the phase of the clock at the end point of the capture clock path is opposite to the phase of the capturing clock at the source. *Return type:* boolean
`capturing_clock_is_propagated`	Returns a value of `true` if the capturing clock is in the propagated mode, at the clock end point. *Return type:* boolean
`capturing_clock_latency`	Returns the ideal or propagated latency of the capturing clock. *Return type:* float
`capturing_clock_open_edge_type`	Returns the opening edge time of the capturing clock. *Return type:* string
`capturing_clock_pin`	Returns the name of the capturing clock pin. *Return type:* collection
`capturing_clock_source_arrival_time`	Returns the capturing clock arrival time at the clock root pin/port of the capturing clock path. This value is a sum of the clock edge time and the source latency constraints specified at the clock root. **Note:** You must specify `report_timing -path_type full_clock` in order to return capturing clock arrival time information. *Return type:* float
`capturing_clock_source_arrival_time_mean`	Returns the mean component of the capturing clock arrival time at the clock root pin/port of the capturing clock path. *Return type:* float
`capturing_clock_source_arrival_time_sigma`	Returns the sigma component of the capturing clock arrival time at the clock root pin/port of the capturing clock path. *Return type:* float
`capturing_point`	Returns a pointer to the end point of the timing path. *Return type:* collection
`capturing_point_is_level_sensitive`	Returns a value of `true` if the end point of the data path is level sensitive. *Return type:* boolean
`check_delay`	Returns the check delay value for the timing path. *Return type:* float
`check_delay_mean`	Returns the mean component of the check delay value for the timing path. *Return type:* float
`check_delay_sigma`	Returns the sigma component of the check delay value for the timing path. *Return type:* float
`clock_path_end_point`	Returns the last point in the capture clock path of the timing path. *Return type:* collection
`clock_uncertainty`	Returns the clock uncertainty of the timing path. *Return type:* float
`cppr_adjustment`	Returns the CPPR adjustment of the timing path. *Return type:* float

`cppr_adjustment_mean`	Returns the mean component of the CPPR adjustment of a timing path. *Return type:* float
`cppr_adjustment_sigma`	Returns the sigma component of the CPPR adjustment of a timing path. *Return type:* float
`cppr_branch_point`	Returns the CPPR branch point of the given timing path. *Return Type*: collection
`drive_adjustment`	Returns the drive adjustment value for the timing path. Return type: float
`drive_adjustment_mean`	Returns the mean component of the drive adjustment value for the timing path. Return type: float
`drive_adjustment_sigma`	Returns the sigma component of the drive adjustment value for the timing path. Return type: float
`external_delay`	Returns the value of the `set_output_delay` constraint at the end point of the path. *Return type:* float
`gba_slack`	Returns the original slack time for the timing path – before PBA retiming is performed. *Return type:* float
`gba_slack_mean`	Returns the mean component of the slack time for the timing path – before PBA retiming is performed. *Return type:* float
`gba_slack_sigma`	Returns the sigma component of the slack time for the timing path – before PBA retiming is performed. *Return type:* float
`hold`	Returns the hold time at the end point of the path. *Return type:* float
`is_clock_gating_hold`	Returns a value of `true` if the path is a clock gating hold check. *Return type*: boolean
`is_clock_gating_setup`	Returns a value of `true` if the path is a clock gating setup check. *Return type*: boolean
`is_transparent_latch`	Returns a value of `true` if the path is a transparent latch path. *Return type:* boolean
`launch_clock_path`	Returns the launching clock path of the timing path. This property can be used only for path_type full_clock. *Return type:* collection
`launching_clock`	Returns the launching clock of the timing path. *Return type:* collection
`launching_clock_is_inverted`	Returns a value of true if the phase of the clock at the start point of the launch clock path is opposite to the phase of the launch clock at the source. *Return type:* boolean
`launching_clock_is_propagated`	Returns a value of `true` if the launching clock is in the propagated mode, at the clock end point. *Return type:* boolean
`launching_clock_latency`	Returns the ideal or propagated latency of the launching clock. *Return type:* float

`launching_clock_latency_mean`	Returns mean component of latency of the launching clock. *Return type:* float
`launching_clock_latency_sigma`	Returns sigma component of latency of the launching clock. *Return type:* float
`launching_clock_open_edge_time`	Returns the opening edge time of the launching clock. *Return type:* float
`launching_clock_open_edge_type`	Returns the launching clock's open edge type at the launching point of the timing path. In case of input delay path with respect to a clock it will return the clock edge type, which is triggering the path. *Return type:* string
`launching_clock_source_arrival_time`	Returns the launching clock arrival time at the clock route pin/port of the launching clock path. This value is the sum of the clock edge time plus any source latency constraint specified at the clock root. **Note:** You must specify `report_timing -path_type full_clock` in order to return launching clock arrival time information. *Return type:* float
`launching_clock_source_arrival_time_mean`	Returns the mean component of the launching clock arrival time at the clock route pin/port of the launching clock path.
`launching_clock_source_arrival_time_sigma`	Returns the sigma component of the launching clock arrival time at the clock route pin/port of the launching clock path. *Return type:* float
`launching_input_delay`	Returns the input delay if the timing path starts from an input or output port. This value is set using `set_input_delay`. *Return type:* float
`launching_point`	Returns a pointer to the start point of the timing path. *Return type:* collection
`launching_point_is_level_sensitive`	Returns a value of `true` if the begin point of the data path is level sensitive. *Return type:* boolean
`nets`	Returns the nets for the specified timing paths. For example, `get_property [get_timing_paths] nets` *Return type*: string

`num_cell_arcs`	Returns the number of cell arcs in the given timing path.
	For example,
	`set path [report_timing -from <start_point> -to <end_point> -collection]`
	`get_property $path num_cell_arcs`
	Return Type: float

`num_net_arcs`	Returns the number of net arcs in the given timing path.
	For example,
	`set path [report_timing -from <start_point> -to <end_point> -collection]`
	`get_property $path num_net_arcs`
	Return Type: float

`object_type`	Returns the object type `timing_path`. *Return type:* string
`other_end_arrival`	Returns the capture clock path's begin time, In case of SOCV analysis, mean+N*sigma quantile value is reported. *Return type:* float

`other_end_arrival_mean`	Returns the mean component of capture clock path's begin time. *Return type:* float
`other_end_arrival_sigma`	Returns the sigma component of capture clock path's begin time. *Return type:* float
`path_delay`	Returns the delay of data path component of a timing path. In case of SOCV analysis, mean+N*sigma quantile delay is reported. *Return type:* float
`path_delay_mean`	Returns the mean delay component of data path of a timing path. *Return type:* float
`path_delay_sigma`	Returns the sigma delay component of data path of a timing path. *Return type:* float
`path_group`	Returns a collection of path groups associated with the timing path. *Return type:* collection
`path_group_name`	Returns the name of the path group for the timing path. *Return type:* string
`path_type`	Returns a value of `max` if the path is for setup, or `min` if the path is for hold. *Return type:* string
`period`	Returns the time duration between the launching clock edge and capturing clock edge of the path. The launching clock edge indicates the launching clock time when the signal at the start point of the path is launched, and the capturing clock edge indicates the capturing clock time when the signal at the path end point is captured. For both of these edges, their ideal clock cycle times are considered for calculating the path period. For those paths, where either the launching or capturing clock is absent or asynchronous, the property prints NA. The property works for both setup (late) paths, and hold (early) paths. Usually the path period is decided by a simple subtraction of effective launching edge from the effective capturing edge of the path. Under certain scenario, these edges are moved back or ahead in time, and accordingly the path period is also affected. The following factors impact period: a) Multicycle paths: The launch or capture edge will move by the cycle factor depending on the type of multi cycle exception, and this cycle factor is taken into account while computing the period. b) Max delay paths: Any max_delay specification overrides the relationship between capturing and launching edges and sets it to the max_delay value. The period of the path is thus returned as equal to the specified max delay value for sequential max delay paths. Note that a combinational max delay path has at least one of the launch or capture clocks either absent or asynchronous, and hence the effective period is not calculated for them. c) Clock uncertainty: The uncertainties applicable to the interaction of launch and capture clocks for this particular path also affect the effective edges, but this factor is not taken into consideration while computing the period. d) Unconstrained paths: The unconstrained paths have no capture clock and hence period can not be calculated for them. The property will return NA. *Return value*: float For more information, see "Examples".
`recovery`	Returns the recovery time at the end point of the path. *Return type:* float

`removal`	Returns the removal time at the end point of the path. *Return type:* float
`required_time`	Returns the required time for the timing path. *Return type:* float
`required_time_mean`	Returns the mean component of the required time for the timing path. *Return type:* float
`required_time_sigma`	Returns the sigma component of the required time for the timing path. *Return type:* float
`setup`	Returns the setup time at the end point of the path. *Return type:* float
`skew`	Returns the latency difference between the capturing clock latency and launching clock latency of the path (after considering the effect of CPPR factor). *Return type:* float
`slack`	Returns the slack time for the timing path. *Return type:* float
`slack_mean`	Returns the mean component of the slack time for the timing path. *Return type:* float
`slack_sigma`	Returns the sigma component of the slack time for the timing path. *Return type:* float
`slack_stddev`	Returns standard deviation of the slack. *Return type:* float
`spatial_distance`	Reports the bounding box distance for spatial derating applied on a timing path. *Return type:* float
`time_borrowed`	Returns the amount of time borrowed from the timing end point. *Return type:* float
`time_lent`	Returns the amount of time lent to the timing start point. *Return type:* float
`timing_points`	Returns a collection of the timing points that make up the path. *Return type:* collection
`underated_slack`	Returns the underated slack time for the timing path. The underated slack value is available with timing path object only if it is generated using the `-derate` parameter. *Return type:* float
`view_name`	Returns the analysis view name associated with the timing path. *Return type*: string

Timing Point Properties

`arrival`	Returns the arrival time at the timing point. *Return type*: float
`arrival_mean`	Returns the mean component of the arrival time at the timing point. *Return type*: float
`arrival_sigma`	Returns the sigma component of arrival time at the timing point. *Return type*: float
`delay`	Returns the delay value for the timing point. *Return type:* float
`delay_mean`	Returns the mean component of the delay value for the timing point. *Return type:* float
`delay_sigma`	Returns the sigma component of the delay value for the timing point. *Return type:* float
`delta_delay`	Returns the incremental delay on a timing point after SI analysis is performed. *Return type:* float
`fanin_arc`	Returns the delay arc for the corresponding timing point. *Return type*: collection
`object_type`	Returns the `timing_point` object type. *Return type:* string
`pin`	Returns a collection of pins or ports of the timing point. *Return type:* collection
`si_object`	Returns the `si_victim` object type that can be used to query SI delay attributes. **Note**: To enable SI objects, you can set the `timing_report_enable_si_debug` global variable to `true`.
`slew`	Returns the path-specific slew of the timing point. *Return type:* float
`slew_mean`	Returns the mean component of the path-specific slew of the timing point. *Return type:* float
`slew_sigma`	Returns the sigma component of the path-specific slew of the timing point. *Return type:* float
`transition_type`	Returns the transition type of the timing point - rise or fall. *Return type:* string

Signal Integrity Attributes

The signal integrity attributes can be queried using the following object types:

- `si_victim`
- `si_attacker`
- `noise_victim`: Returns a collection of `si_victim` objects that can be used to query glitch attributes.
  Return type: collection

These are explained below.

si_victim Properties

The following information is saved under the `si_victim` object type:

		Valid for Properties		
Property Name	**Description**	**si_victim**	**si_object**	**noise_victim**
`annotated_glitch`	Reports the annotated glitch. *Return type:* float	N	N	Y
`attackers`	Returns a collection of `si_attacker` objects that can be used to query attacker attributes. *Return type:* collection	Y	Y	Y
`capacitance`	Returns the total capacitance of victim net. *Return type:* float	Y	Y	Y
`coupling_capacitance`	Returns the total coupling capacitance of a victim net. *Return type:* float	Y	Y	Y
`glitch_area`	Returns the glitch area. *Return type:* float	N	N	Y
`glitch_width`	Returns the glitch width. *Return type:* float	N	N	Y
`hierarchical_name`	Returns the hierarchical name of a victim net. *Return type:* string	Y	Y	Y
`input_peak`	Returns the Receiver Input Peak (RIP) value. *Return type:* float	N	N	Y
`input_failure_threshold`	Returns the failure threshold at RIP. This will always be reported irrespective of whether failure happens at RIP or ROP. *Return type:* float	N	N	Y
`level`	Returns the noise level - VL or VH. *Return type:* string	N	N	Y
`num_active_attackers`	Returns the number of active attackers. *Return type:* float	Y	Y	Y
`num_attackers`	Returns the number of total attackers (distinct + accumulated small attacker - as 1). The attackers that are dropped from the accumulated small attackers will not be included with the total attackers. *Return type:* float	Y	Y	Y

`object_type`	Returns the object type `si_victim`. *Return type:* string	Y	Y	Y
`prop_glitch`	Reports the propagated glitch seen at the victim driver output. *Return type:* float	N	N	Y
`receiver_peak`	Returns the Receiver Output Peak (ROP) value. *Return type:* float	N	N	Y
`receiver_peak_threshold`	Returns the failure threshold at ROP. This will always be reported irrespective of whether failure happens at RIP or ROP. *Return type:* float	N	N	Y
`resistance`	Returns the total resistance of a victim net. *Return type:* string	Y	Y	Y
`transition`	Returns the victim transition direction (rise or fall). *Return type:* string	Y	Y	N
`view_type`	Returns the type of analysis (early or late). *Return type:* string	Y	Y	N

si_attacker Properties

The following Information is saved under the `si_attacker` object type for each attacker of the victim net in the path:

Property Name (for each attacker retrieved from a victim)	Description
`coupling_capacitance`	Returns the coupling of a victim net. *Return type:* float
`hierarchical_name`	Returns the hierarchical name of an attacker. *Return type:* string
`is_active`	Returns whether an attacker is active or not. *Return type:* string
`noise_peak`	Returns the Xtalk delay bump induced by an attacker. *Return type:* float
`object_type`	Returns the `si_attacker` object type. *Return type:* string
`slew`	Returns the slew of attackers. *Return type:* float

`state`	Returns the status of attackers. The status descriptions are given below: • SY: Synchronous • INF: Attacker with infinite timing window • PE: Filtered due to physical exclusion • UF: Filtered by user • CN: Filtered because it is a constant net • SB: Filtered due to small bump • TA: Filtered because of timing window does not overlap • LC: Filtered due to logical correlation • NA: Attacker status is not known *Return type:* string

Examples

The following `get_property` example shows the usage of SI attributes to query noise information:

```
source pinlist.tcl
set chan [open attribute.log w]
foreach pin $pinList {
   set vicArray [get_property [get_pins $pin] noise_victim]
   foreach_in_collection vic $vicArray {
       set input_peak [ get_property $vic input_peak]
       #if { $input_peak > 250 } {}

       set vic_name [ get_property $vic hierarchical_name ]
       set x [ get_property [get_net -of_objects $vic_name ] name ]
       set level [ get_property $vic level]
       set ccap [ get_property $vic capacitance]
       set vdd [get_property [get_pins $vic_name] power_rail_voltage_max]
       set libs [get_libs -of_objects [get_lib_cells -of_objects [get_cells -of_objects [get_pins $vic_name]]]]
       foreach_in_collection libName $libs {set library [get_property $libName hierarchical_name]}
       set prop_glitch [ get_property $vic prop_glitch]
       set cap [format %.2f $ccap]
       set annotated_glitch [ get_property $vic annotated_glitch]
       set receiver_peak [ get_property $vic receiver_peak]
       set input_peak_threshold [ get_property $vic input_failure_threshold ]
       set receiver_peak_threshold [ get_property $vic receiver_peak_threshold]
       puts $chan "--------------------------------------------------------------------------------"
         puts $chan "Peak(mV) Level TotalCap(fF) Vdd(V) Library VictimNet"
         puts $chan "$input_peak $level $cap $vdd $library $vic_name {$x}"

         if { $receiver_peak > 0 } {
           puts $chan "Receiver output peak:"
           puts $chan "Value ReceiverNet"
           puts $chan "$receiver_peak ( $receiver_peak_threshold) $vic_name (CELL)"
         }

         if { $input_peak > 0 } {
           puts $chan "Receiver Input peak:"
           puts $chan "Value ReceiverNet"
```

```
      puts $chan "$input_peak ( $input_peak_threshold) $vic_name (CELL)"
    }

    puts $chan "\nConstituents:"
    puts $chan "Source Peak(mV) Offset(ps) Slew(ps) Xcap(fF) Vdd(V) Edge Status Net"
    set aggs [ get_property $vic attackers ]
    foreach_in_collection current_agg $aggs {

      set hierarchical_name [ get_property $current_agg hierarchical_name ]
      set noise_peak [ get_property $current_agg noise_peak]
      set state [ get_property $current_agg state]
      set slew [ get_property $current_agg slew]
      set edge [ get_property $current_agg transition]
      if {[string match ~* $hierarchical_name]} {
      set vdd [get_property [get_pins $vic_name] power_rail_voltage_max]
      } else {
      set vdd [get_property [get_pins -of_objects [get_nets $hierarchical_name]] power_rail_voltage_max]
      }
      set coupling_capacitance [ get_property $current_agg coupling_capacitance]

      puts $chan "Cpl: $noise_peak $slew $edge $vdd $coupling_capacitance $state $hierarchical_name"
      }
    if { $prop_glitch > 0 } {
      puts $chan "Prp: $prop_glitch"
     }


}
close $chan
```

User-Defined Properties

The user-defined properties allow you to define certain properties which can be interpreted by the software in a predefined way. When these properties are not specified, the software will revert to the default behavior, as applicable.

Property Name	Type	Description
`capacitive_load_unit`	lib	Provides the load unit mentioned in the library.
`time_unit`	lib	Provides the time unit mentioned in the library.
`aocv_weight`	lib_cell	Defines AOCV weight for a given cell.
`k_input_switching_window_fall*`	lib	Defines the switching window threshold for fall signal during the application of SSI (simultaneous switching input) derating feature. *Default*: 1
`k_input_switching_window_rise*`	lib	Defines the switching window threshold for rise signal during the application of SSI (simultaneous switching input) derating feature. *Default*: 1

* When the above properties are defined in SSI derating environment, the delay of the respective cell multiplied by the above factor will be compared with the difference in (rise/fall) signal arrivals at two inputs to determine if switching derate needs to be applied to cell delay.

Examples

- The following command returns the hierarchical name for the pin contained in the collection referenced by `$pin`:
  ```
  get_property $pin hierarchical_name
  ```
- The following example shows how you can query a user-defined property from `stage_weighting` library at the cell level:
  ```
  set_property [get_lib_arcs -of fast/BUFX2] stage_weighting 3 // sets the value
  get_property [get_lib_arcs -of fast/BUFX2] stage_weighting
  ```
- The following example shows power and ground pins for the corresponding library pins:
  ```
  get_property [get_lib_pins CELLA/A] related_power_pin
  get_property [get_lib_pins CELLA/A] related_ground_pin
  ```

- The following example shows active clocks:

```
get_property [get_clocks Iclk1] is_active true
```

- The following commands define the switching window threshold for rise signal:

```
define_property -type float -object_type lib k_input_switching_window_rise
set_property [get_libs slow] k_input_switching_window_rise 1.1
```

- The following example shows that the phase shift is 2.0 ns, as returned by the "period" property:

```
report_timing -clock_to CLK_W_3 -clock_from CLK_W_3

  Path 1: MET Setup Check with Pin seg3/u14/CK
  Endpoint: seg3/u14/D (^) checked with leading edge of 'CLK_W_3'
  Beginpoint: seg3/u9/Q (^) triggered by leading edge of 'CLK_W_3'
  Path Groups: {CLK_W_3}
  Other End Arrival Time 0.621
  - Setup 0.186
  + Phase Shift 2.000
  = Required Time 2.434
  - Arrival Time 2.062
  = Slack Time 0.372
  Clock Rise Edge 0.000
  + Clock Network Latency (Prop) 0.682
  = Beginpoint Arrival Time 0.682
  ……..
  innovus > set path [rt -clock_to CLK_W_3 -clock_from CLK_W_3 -collection]
  0x14
  innovus > get_property $path period
  2.000
```

- In this following example, there is a clock reference for launch clock (Null phase- @), so Innovus does not show any clock period for this path:

```
report_timing -clock_to CLK_W_3 -early

  Path 1: VIOLATED Hold Check with Pin seg3/u3/CK
  Endpoint: seg3/u3/D (v) checked with leading edge of 'CLK_W_3'
  Beginpoint: IN3 (v) triggered by leading edge of '@'
  Path Groups: {CLK_W_3}
  Other End Arrival Time 0.682
  + Hold -0.034
  + Phase Shift 0.000
  = Required Time 0.648
  Arrival Time 0.117
  Slack Time -0.531
  Clock Rise Edge 0.000
  + Input Delay 0.000
  + Drive Adjustment -0.014
  = Beginpoint Arrival Time -0.014
  ---------------------------------------------------------------------------------------
  Load  Slew   Delay   Incr    Arc        Fanout  Cell  Arc           User   Arrival Instance
                      Delay   Annotation                             Derate Time
  ---------------------------------------------------------------------------------------
  0.013 0.029                 WLM        1       IN3     v                  -0.014
  0.013 0.057  0.054   0.000  WLM        1       CLKBUF  A v -> Y v  1.000  0.040   seg3/u1
  0.003 0.050  0.077   0.000  WLM        1       CLKBUF  A v -> Y v  1.000  0.117   seg3/u2
  0.003 0.050  0.000   0.000  WLM                DFF     D v         1.000  0.117   seg3/u3
  ---------------------------------------------------------------------------------------

  innovus > set path [report_timing -clock_to CLK_W_3 -collection -early]
  0x16
  innovus > get_property $path period
  NA
```

- The following example shows if a real or generated clock is defined on the pins or ports:

```
tempus > create_clock [get_ports {CLK_t}] -name clk -period 100 -waveform {0 50}
tempus > create_generated_clock -divide_by 2 -source CLK -name d_2 CLK_b
tempus > get_property [get_ports CLK_t] is_clock_root
true

tempus > get_property [get_pins CLK_b] is_clock_root
```

```
true
```

Related Commands

- define_property
- report_property
- set_property
- list_property
- get_clocks
- get_pins
- get_cells
- report_timing

index_collection

```
index_collection
[-help]
base_collection
index
```

Returns a collection containing the object that exists at the specified index of the specified object collection.

Parameters

`-help`	Prints out the command usage.
`base_collection`	Specifies the base collection.
`index`	Specifies the index number. You can specify a number from `0` to *`collection_size-1`*.

Examples

- The following command returns the object that exists at the index number `2` in the object collection referenced by `$nets`:

  ```
  index_collection $nets 2
  ```

Related Information

- `append_to_collection`
- `compare_collections`
- `copy_collection`
- `filter_collection`

list_property

```
list_property
[-help]
[-type {cell clock design lib lib_cell lib_pin lib_timing_arc net path_group pin port timing_arc timing_path timing_point pg_net pg_pin
lib_pg_pin si_victim si_attacker}]
```

Lists all of the properties associated with the specified object type.

If you do not specify any object type, this command lists all the supported object types, and all the properties associated with those object types.

Parameters

`-help`	Prints out the command usage.
`-type` *object_type*	Lists the properties associated with the specified object type. The following object types are supported: cell, clock, design, lib, lib_cell, lib_pg_pin, lib_pin, lib_timing_arc, net, path_group, pg_net, pg_pin, pin, port, si_attacker, si_victim, timing_arc, timing_path, or timing_point Additional SI properties can be included with the `timing_point` object. To enable SI properties, you can set `timing_report_enable_si_debug` global variable to `true`.

Related Commands

- `define_property`
- `get_property`
- `report_property`
- `reset_property`
- `set_property`

query_objects

```
query_objects
[-help]
collection
[-limit value]
```

Displays the names of the objects contained inside the specified collection.

⚠ `query_objects` displays object names only. To return a tcl list of object names, the `get_object_name` command may be used instead.

Parameters

`-help`	Prints out the command usage.
`collection`	Specifies the object collection for which to print names.
`-limit` *`value`*	Specifies the maximum number of object names to print. Specify a value of `0` to query the names of all of the objects in the collection. *Default*: Prints the first 100 object names in the collection.

Examples

- The following commands create collections of pins and nets, add them together to create a new collection, and prints the names of the objects in the collection:

  ```
  set pins [get_pins {i_block1/b1_Sin1 i_block1/b1_CLK1 i_block1/b1_Sout1}]
  set nets [get_nets {Sin1 CLK1 Sout1}]
  add_to_collection newVar $pins
  add_to_collection newVar $cells
  query_objects $newVar
  ```

  The software returns the following information:

  ```
  i_block1/b1_Sin1 i_block1/b1_CLK1 i_block1/b1_Sout1 Sin1 CLK1 Sout1
  ```

Related Information

- `get_object_name`

range_collection

```
range_collection
[-help]
base_collection
from
to
```

Returns a sub-collection of the base collection. The sub-collection range lies between the ‘from’ and ‘to’ (including ‘from’ and ‘to’) index numbers specification of this command.

Parameters

`-help`	Prints out the command usage.
base_collection	Specifies the base collection.
from	Specifies the ‘from’ index number.
to	Specifies the ‘to’ index number.

The index number can be an integer or an expression, such as, “end OR end-integer” (with no spaces within the expression).

If the specified to/from value is negative (less than 0), then the software will change this value to 0. If the value is greater than the “collection_size – 1”, then it will be changed to “collection_size – 1”.

Examples

- The following example sets `$points` as the base collection:

```
> set points [get_cells *]
0x13
> sizeof_collection $points
22
```

- The following example returns the sub-collection of the base collection `$points` between the index number range from 2 to 4:

```
> range_collection $points 2 4
clk_buf2 clk_buf3 clk_buf4
0x16
> range_collection $points 4 2
clk_buf2 clk_buf3 clk_buf4
0x17
```

- The following example returns a sub-collection starting from index number 0 till the end of the base collection:

```
> range_collection $points 0 end
clk_buf1 pad1 clk_buf2 clk_buf3 clk_buf4 buf1 buf2 buf3 buf4 mem dff1 dff2 clkand2 pu_cell_1 pu_lat_1 u1 cg4 genLoop comboLoop pad2
dff3 latch1
0x18
```

- The following example returns a sub-collection starting from end-integers 4 to 2:

```
> range_collection $points end-4 end-2
genLoop comboLoop pad2
0x19
```

- The following example shows a range starting from a negative value to a value greater than the collection size. The following warning messages are displayed:

```
> range_collection $points -5 100
**WARN: (TCLCMD-1419): 'from' index with value -5 is less than start index, so it is changed to 0 .
**WARN: (TCLCMD-1419): 'to' index with value 100 is greater than end index, so it is changed to 21 .
clk_buf1 pad1 clk_buf2 clk_buf3 clk_buf4 buf1 buf2 buf3 buf4 mem dff1 dff2 clkand2 pu_cell_1 pu_lat_1 u1 cg4 genLoop comboLoop pad2
dff3 latch1
0x1a
```

- The following example shows the warning message displayed for a negative start index number:

```
> range_collection $points end-25 end-20
**WARN: (TCLCMD-1419): 'from' index with value -4 is less than start index, so it is changed to 0 .
clk_buf1 pad1
0x1b
```

- The following command shows an error if the specified index number is not an integer:

```
> range_collection $points s end
**ERROR: (TCLCMD-1418): Bad index 's': must be integer? or end?[-]integer?
```

Related Commands

- index_collection

remove_from_collection

```
remove_from_collection
[-help]
base_collection
[-intersect]
object_collection_or_list
```

Creates a new collection by removing the objects specified in one collection from another collection.

Parameters

`-help`	Prints out the command usage.
`base_collection`	Specifies the base collection of objects. Objects from the second collection or object list are removed from this collection to create the new collection.
`-intersect`	Provides intersection of base collection elements with second collection of objects or lists.
`object_collection_or_list`	Specifies the collection or list of objects that should be removed from the base collection to create the new collection.

Examples

- The following commands create a collection of pins that does not include clock pins:

  ```
  set allP [get_pins *]
  set all_pin_no_clock [remove_from_collection $allP [get_pins -filter "is_clock == true"]]
  ```

  You can use the `sizeof_collection` command to see that the `all_pin_no_clock` collection contains fewer objects (that is, no clock pins) than the original `allP` collection:

  ```
  sizeof_collection $allP
  50

  sizeof_collection $all_pin_no_clock
  47
  ```

Related Information

- `add_to_collection`
- `append_to_collection`
- `compare_collections`
- `copy_collection`
- `filter_collection`

report_property

```
report_property
[-help]
{collection | list_of_collections}
[-property_list list of properties]
[{> | >>} filename]
```

Reports the properties associated with each object in the specified collection.

For properties that return a populated collection, the command output displays `{...}`. For any empty collections,`{}` will be displayed.

Parameters

`-help`	Prints out the command usage.
*collection	list_of_collections*	Specifies a collection, or a list of collections to report. The collections can contain any of the following object types: cell, clock, design, lib, lib_cell, lib_pin, lib_pg_pin, pg_net, pg_pin, lib_timing_arc, net, path_group, pin, port, timing_arc, timing_path, or timing_point
`-property_list` *list of properties*	Specifies a list of valid property names to report. **Note**: If the `-property_list` parameter is not specified, then all the properties associated with a collection of objects will be reported.
{> | >>} *filename*	When specified with >, the software writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, the software writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. You can add the `.gz` extension to the file name to generate a compressed report. **Note:** The file name must be the last argument in the list.

Examples

- The following command shows the values of the specified properties:

```
report_property [get_pins TU11/A] -property_list {name direction clocks}

property | value
name | TU11/A
direction | in
clocks | {...}
```

Related Commands

- get_property
- define_property
- list_property
- set_property
- reset_property

sizeof_collection

```
sizeof_collection
[-help]
collection
```

Returns the total number of objects contained in the specified collection. If the collection contains different types of objects, `sizeof_collection` returns the sum of all the objects of different types in the collection.

Parameters

`-help`	Prints out the command usage.
`collection`	Specifies the collection.

Related Information

- `add_to_collection`
- `append_to_collection`
- `compare_collections`
- `copy_collection`
- `filter_collection`

sort_collection

```
sort_collection
[-help]
collection
[-dictionary]
{property_or_list_of_properties}
[-descending]
```

Returns a sorted collection of objects based on the property.

Parameters

`-help`	Prints out the command usage.
collection	Specifies the collection to sort. The collection must be a homogenous collection of any one of the following object types: `cell`, `clock`, `design`, `lib`, `lib_cell`, `lib_pin`, `lib_timing_arc`, `net`, `path_group`, `pin`, `port`, `timing_arc`, `timing_path`, or `timing_point`.
`-descending`	Sorts the objects in the collection by descending order. *Default:* Sorts the collection in ascending order.
`-dictionary`	Sorts the objects in a collection by dictionary order.
property | *list_of_properties*	Sorts the collection by the specified property. You can specify any property associated with the object type. You can specify one or more properties. If you specify more than one property, the software sorts the collection in the order by which you specified the properties.

Examples

- The following command returns a collection of all the input ports in the design sorted on the basis of their hierarchical names:

```
sort_collection [all_inputs] hierarchical_name


CLK1 CLK2 CLK3 CLK4 IN1 IN2 IN3 IN4
```

Related Information

- `add_to_collection`
- `append_to_collection`
- `compare_collections`
- `copy_collection`
- `filter_collection`

2

ART Based Flow Commands

- clearActiveLogicView
- createActiveLogicView
- getActiveLogicViewMode
- getHierMode
- getModuleView
- setActiveLogicViewMode
- setHierMode
- setModuleView

clearActiveLogicView

```
clearActiveLogicView
[-help]
```

Resets the reduced timing graph created by `createActiveLogicView` back to the original timing graph.

Parameters

`-help`	Outputs a brief description of the `clearActiveLogicView` command. For a detailed description of the command, use the `man` command: `man clearActiveLogicView`

Related Topics

- `createActiveLogicView`
- Timing Analysis chapter of the *Innovus User Guide*
- Top-level Timing Closure Methodologies chapter of the *Innovus User Guide*

createActiveLogicView

```
createActiveLogicView
[help]
[-type {flatTop timingBudget topIO}[-setDontTouch | -unsetDontTouch ]]
```

Trims the timing graph to ignore logic inside the first level registers in partitions to improve runtime for timing-related commands. This capability should only be used for timing analysis purpose.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `createActiveLogicView` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man createActiveLogicView`.
`-setDontTouch`	Sets the dontTouch value as `true`. This freezes I/O interfaces (Interface logics and interface nets) and prevents the `optDesign -postCTS -hold` from optimizing the block interface logics. The ILM ECO data can be loaded back.
`-type {flatTop timingBudget topIO}`	Specifies the Active Logic View type. • `flatTop`: Used to mark the top-level timing graph to mask all logic inside the interface logic of each partition. • `timingBudget`: Instructs the command to use tuned settings for `deriveTimingBudget` when marking the top-level timing graph. This option should be specified if you plan to run `timeDesign` or `optVirtual` with ART based and later on run `deriveTimingBudget` based on the resulted timing graph. • `topIO`: Marks interface nets/instances to don't touch, so optimizer will preserve the I/O paths during optimization. Value of `-topIO` is `-setDontTouch` or `-unsetDontTouch`.
`-unsetDontTouch`	Sets dontTouch value as `false`. This resets the dontTouch on I/O logics before reading the ECO data.

Related Information

- `clearActiveLogicView`
- `timeDesign`
- `optVirtual`
- `deriveTimingBudget`
- Timing Analysis chapter of the *Innovus User Guide*
- Top-level Timing Closure Methodologies chapter of the *Innovus User Guide*

getActiveLogicViewMode

```
getActiveLogicViewMode
[-help]
[-keepAsync {false | true}]
[-keepHighFanoutPorts {true| false}]
[-keepInstInSdc {true|false}]
[-keepLoopBack {false | true}]
[-quiet]
[-nonDefault]
```

Controls certain behaviors of active logic view commands.

Use the `getActiveLogicViewMode` command to display the current settings for the `setActiveLogicViewMode` command.

Parameters

`-keepAsync {false | true}`	Discards or retains the portion of the netlist relating to asynchronous control and scan controls terminals. By default, the tool discards this portion of the netlist. This option potentially conflicts the setting of `setAnalysisMode -async true` when the analysis for an asynchronous circuit is desired. When this happens, a warning message is issued. *Default:* `false`
`-keepHighFanoutPorts {true | false}`	Retains or discards the portion of the netlist relating to high-fanout ports. By default, the tool retains this portion of the netlist. *Default:* `true`
`-keepInstInSdc {true|false}`	Keeps instances which have constraints on it. *Default:* `false`
`-keepLoopBack {false | true}`	Discards or retains the portion of the netlist relating to input and output loopback paths. By default, the tool discards this portion of the netlist. *Default:* `false`
`-help`	Outputs a brief description that includes type and default information for each `getActiveLogicViewMode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man getActiveLogicViewMode`
`-nonDefault`	Displays the current settings which are not default settings.
`-quiet`	Displays the current settings for the specified parameters in the Tcl list format. If you specify `-quiet` without any parameters, the software displays the current settings of all the `getActiveLogicViewMode` parameters in the Tcl format.

Example

- The following command displays the current setting for the `highFanoutPort` parameter in the Tcl list format:

  ```
  getActiveLogicViewMode -keepHighFanoutPorts -quiet
  ```

  The software displays the following information:

  ```
  true
  ```

- The following command displays the current setting for the `getActiveLogicViewMode` parameter:

  ```
  getActiveLogicViewMode
  ```

  The software displays the following information:

  ```
  -keepAsync false                    #bool, default=false
  -keepHighFanoutPorts true           #bool, default=true
  -keepLoopBack false                #bool, default=false

  {keepAsync false}  {keepHighFanoutPorts true}   {keepLoopBack false}
  ```

- The following command displays the current settings for all the `getActiveLogicViewMode` commands in the Tcl list format only:

  ```
  getActiveLogicViewMode -quiet
  ```

  The software displays the following information:

  ```
  {keepAsync false} {keepHighFanoutPorts true} {keepLoopBack false}
  ```

Related Topics

- `setActiveLogicViewMode`

getHierMode

```
getHierMode
[-help]
[-addAntennaCell]
[-quiet]
[-trialRouteHonorReadOnly]
[-optStage {preCTS | postCTS | unset}]
```

Returns information about the specified `setHierMode` parameter in the log file and in the console. The following information is returned:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the Innovus software returns values for all of the `setHierMode` parameters.

Parameters

parameter_names	Returns information for the specified parameter. You can specify one or more parameters. See `setHierMode` for descriptions of the parameters you can specify.
`-quiet`	Returns the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software returns the current settings of all `setHierMode` parameters in Tcl list format.

Examples

- The following command returns the current settings for all `setHierMode` parameters:

  ```
  getHierMode
  ```

  The software returns the following information:

  ```
  -addAntennaCell                         # bool, default=false


  {addAntennaCell false}
  ```

- The following command returns the current settings for all `setHierMode` parameters in Tcl list format only:

  ```
  getHierMode -quiet
  ```

  The software returns the following information:

  ```
  {addAntennaCell false}
  ```

Related Topics

- `setHierMode`
- Top-level Timing Closure Methodologies chapter of the *Innovus User Guide*

getModuleView

```
getModuleView
[-help]
[-topReadOnly]
[-hinst list_of_partition_hinsts]
[-partition list_of_paritions]
[-quiet]
```

Verifies the FlexView mode.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `getModuleView` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man getModuleView`
`-hinst` *`list_of_partition_hinsts`*	Displays the list of partition hinsts.
`-partition` *`list_of_paritions`*	Displays the list of partitions.
`-quiet`	Displays the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software displays the current settings of all `setModuleView` parameters in Tcl list format.
`-topReadOnly`	Displays if the top level is read only.

Examples

- The following command verifies the FlexView mode:

```
getModuleView


{TOP all}
{coreinst/ks_core1/amba_dsp1/ram2p_78kx32 readOnly}
{coreinst/ks_core1/leon1 interface}
```

Related Topic

- setModuleView

setActiveLogicViewMode

```
setActiveLogicViewMode
[-help]
[-reset]
[-keepAsync {false | true}]
[-keepHighFanoutPorts {true| false}]
[-keepInstInSdc {true|false}]
[-keepLoopBack {false | true}]
```

Controls certain behaviors of active logic view commands.

Use the `setActiveLogicViewMode` command to display the current settings for the `createActiveLogicView` command.

Parameters

`-keepAsync {false | true}`	
	Discards or retains the portion of the netlist relating to asynchronous control and scan controls terminals. By default, the tool discards this portion of the netlist. This option potentially conflicts the setting of `setAnalysisMode -asyncChecks async` when the analysis for an asynchronous circuit is desired. When this happens, a warning message is issued. *Default:* `false`
`-keepHighFanoutPorts {true | false}`	
	Retains or discards the portion of the netlist relating to high-fanout ports. By default, the tool retains this portion of the netlist. *Default:* `true`
`-keepInstInSdc {true|false}`	Keeps instances which have constraints on it. *Default:* `false`
`-help`	Outputs a brief description that includes type and default information for each `setActiveLogicViewMode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setActiveLogicViewMode`
`-keepLoopBack {false | true}`	
	Discards or retains the portion of the netlist relating to input and output loopback paths. By default, the tool discards this portion of the netlist. *Default:* `false`
`-reset`	Resets the active logic view mode command parameters to their default values. Any values that do not already match the default value and are reset will be reported.

Example

The following command resets all the `setActiveLogicViewMode` parameters to their default values:

```
setActiveLogicViewMode -reset
```

Related Topics

- `createActiveLogicView`
- `getActiveLogicViewMode`

setHierMode

```
setHierMode
[-help]
[-reset]
[-addAntennaCell {true | false}]
[-optStage {preCTS | postCTS | unset}]
```

Sets global parameters for hierarchy aware optimization. Parameters that are specified with `setHierMode` are used automatically when optimization is invoked by `optDesign` command.

To see the current settings for the `setHierMode` command, see `getHierMode`.

Parameters

`-addAntennaCell {true | false}`	Adds antenna cells to the interface nets crossing partition boundaries. The antenna violations could not be created due to the partition boundary nets. These violations are seen only when the full design is assembled. Therefore, when this parameter is set to `true`, it will add antenna cells to fix these violations. *Default*: `false`
`-help`	Outputs a brief description that includes type and default information for each `setHierMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setHierMode`.
`-reset`	Resets parameters to their default values. The `-reset` parameter *must* be the first parameter specified. If you specify `-reset` by itself, the software resets all `setHierMode` parameters to their default values. Any values that do not already match the default value and are reset will be reported.
`-optStage {preCTS | postCTS | unset}`	Ensures proper settings are turned on for corresponding `optDesign` stages. `-optStage` should be set before `optDesign`, and in cases where module view types are defined, `-optStage` should be defined after module view types for all partitions (including FlexILMs) are defined. For example, `setHierMode -optStage preCTS` ensures that `optDesign -preCTS` honors fence, keep partition boundary port, etc.

Examples

- The following example adds antenna cells to the top/partition boundary nets incase antenna violations are created:

  ```
  setHierMode -addAntennaCell true
  ```

Related Topics

- `optDesign`
- `getHierMode`
- Partitioning the Design chapter in the *Innovus User Guide*
- Top-level Timing Closure Methodologies chapter of the *Innovus User Guide*

setModuleView

```
setModuleView
[-help]
[-topReadOnly {true | false}]
[-hinst {list_of_partition_hinsts}]
[-partition {list_of_paritions}]
[-type {readOnly|interface|all}]
```

Specifies the FlexView (`readOnly|interface|all`) for the top level and each of the partitions. This way optimization can be limited to the selected sections of the partitions and the top-level logic.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setModuleView` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setModuleView`.
`-topReadOnly {true | false}`	Sets the top level to the read-only mode. By default, optimization can change the logic at the top level to improve timing. By setting this option to `true`, no change will be done to the entire top-level logic. *Default*: `false`
`-hinst` *`list_of_partition_hinsts`*	Applies the selected FlexView mode to the instance name or list of instance names of partitions.
`-partition` *`list_of_partitions`*	Applies the selected FlexView mode to a partition or list of partitions. You have a choice to either specify the instance name of the partition or just the partition itself.
`-type {readOnly|interface|all}`	Specifies the FlexView mode for the specified partition(s). • `readOnly` : Specifies that the optimization will not be performed inside the partition. • `interface`: Specifies that optimization will be restricted only to the interface logic of the partition. • `all` : Specifies that optimization will be done for the entire partition.

Related Topics

- `getModuleView`

3

Basic Database Access Tcl Commands and Global Variables

- Basic Database Access Tcl Commands
- Basic Database Access Tcl Global Variables

Basic Database Access Tcl Commands

- add_cell_obs
- add_line_end_track
- add_via
- add_via_definition
- applyGlobalNets
- clearGlobalNets
- compress_special
- create_virtual_shape_on_pg_wires
- db_pt
- db_rect
- dbEdge
- dbGet
- dbQuery
- dbSchema
- dbSet
- dbShape
- dbTransform
- dbu2uu
- decompress_special
- delete_cell_obs
- deleteDanglingNet
- free_memory_of_compressed_objects
- get_via_pillars
- getDbGetMode
- globalNetConnect
- report_metal_stack
- report_net_wires
- report_preserves
- report_via
- reportDanglingNet
- reportRC
- reportWirePath
- set_via_pillars
- setDbGetMode

- uniquify
- uu2dbu

add_cell_obs

```
add_cell_obs
[-help]
-cell cell_name_or_pointer
[-exceptPGNet]
-layer layer_name_or_pointer
[-mask int_value]
{-rects {{llx1 lly1 urx1 ury1} {llx2 lly2 urx2 ury2}...} | -polygon {x1 y1 x2 y2 x3 y3 ...}}
[-spacing value | -design_rule_width value]
```

Adds obstruction to a cell. The added obs is saved in the database by its own. You do not need to resource it after restoring the database. It adds an obstruction geometry to a cell on the specified layer and is helpful when you want to add a temporary cell obstruction without changing the cell's LEF and reloading Innovus.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `add_cell_obs` parameter. For a detailed description of the command and all its parameters, use the man command: `man add_cell_obs`
`-cell cell_name_or_pointer`	Specifies the cell name or pointer.
`-design_rule_width value`	Specifies the design rule width.
`-exceptPGNet`	Ignores the Power/Ground routing while checking for DRC violations (including shorts) involving the added obs. This paramete
`-layer layer_name_or_pointer`	Specifies the layer name or pointer.
`-mask int_value`	Specifies the integer value of the mask (within the mask range) for the created OBS. *Default:* `0`, which means that the mask is uncolored.
`-polygon {x1 y1 x2 y2 x3 y3 ...}`	Specifies the polygon shape.
`-rects {{llx1 lly1 urx1 ury1} {llx2 lly2 urx2 ury2}...}`	Specifies the Obs rect list for adding obstruction geometries.
`-spacing value`	Specifies the spacing value.

Related Topics

`delete_cell_obs`

add_line_end_track

```
add_line_end_track
[-help]
[-eol_width float]
[-except_eol_spacing float [-within float]]
-layer string
-offset float
[-track_distance float]
[-track_pitch float]
```

Creates a line-end track.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `add_line_end_track` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man add_line_end_track`
`-eol_width float`	Specifies the width of the line-end track.
`-except_eol_spacing float`	Specifies the spacing of the line-end track.
`-layer string`	Specifies the layer on which the track is to be created.
`-offset float`	Specifies the offset to the core boundary.
`-track_distance float`	Specifies the distance between two sets of tracks.
`-track_pitch float`	Specifies the track pitch.
`-within float`	Specifies the extension in the orthogonal direction to find a neighbor wire.

Related Information

Floorplanning the Design chapter in *Innovus User Guide*.

add_via

```
add_via
[-help]
[-net net]
-pt {x y}
[-shape {blockring |
blockagewire | blockwire | corewire | drcfill | fillopc | fillwire | followpin | iowire | none | padring | ring | stripe}]
[-shield_net net]
[-status {routed | fixed | cover | shield}]
[-user_class value]
-via via_def
```

Adds a new special-route via instance to the design. A DB pointer is returned to the via, if successful. You can use the command at any point in the design flow. To add regular route via, see `editAddVia`.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `add_via` parameter. For a detailed description of the command and all of parameters, use the man command: `man add_via`
`-net net`	Specifies the name or pointer of the net to associate with the via instance. If not specified, the via is assigned as a floating fillwire via, unless `-shape` is specified.
`-pt {x y}`	Specify the placement location (in microns) of the via instance.
`-shape {blockring | blockagewire | blockwire | corewire | drcfill | fillopc | fillwire | followpin | iowire | none | padring | ring | stripe}`	
	Specifies the special route shape (`DEF SPECIALNETS wiring+ SHAPE`). If `-net` is specified, the default shape is `none`. If `-net` is not specified, the default shape is `fillwire`.
`-shield_net net`	Specifies the name or pointer of the net to be shielded by the via instance. Only valid when `-status` is set to shield.
`-status {routed | fixed | cover | shield}`	
	Specifies the special route (`DEF SPECIALNETS`) wiring status. *Default*: `routed`
`-user_class value`	Specifies the user-defined string to be added to the via instance that is created.
`-via via_def`	Specify the via-master name or pointer of the via-master to place.

Examples

- The following command adds a via with via-master name of `my_via1`:
  ```
  add_via -via my_via1 -pt {100.1 200.5} -net VSS -shape followpin
  ```
- The following command adds a VSS via (`via cell VIA2X`) that shields reset net:
  ```
  add_via -via VIA2X -pt {13.0 145.5} -net VSS -shield_net reset -status shield
  ```
- To add a parameterized via you can look up the via-master with `add_via_definition` (in this case a 1x2 cut via with all the other parameters the default value).
  It will either return an existing via-master with the same parameters, or create it. Then use it to create an instance of the via.
  ```
  set myvia [add_via_definition -via_rule via1_rule -row_col {1 2}]
  add_via -via $myvia -pt {10.5 10.5} -net VS
  ```

add_via_definition

```
add_via_definition
[-help]
[-bottom_layer_masks int+]
[-cut_masks int+]
[-name viaName]
{{-via_rule via_def_rule
    [[-cut_size {x y}][-cut_spacing {x y}][-top_enclosure {x y}]
     [-bottom_enclosure {x y}][-top_offset {x y}][-bottom_offset {x y}][-row_col {x y}]
     [-origin {x y}] [-pattern pattern]]} |
 {{-cut_layer layer -cut_rects {{x1 y1 x2 y2} {x3 y3 x4 y4} ...}
  {[-top_layer layer] {-top_rects {{x1 y1 x2 y2} {x3 y3 x4 y4} ...} |
   -top_polygon {{x1 y1}{x2 y2}...}}}
  {[-bottom_layer layer] {-bottom_rects {{x1 y1 x2 y2} {x3 y3 x4 y4} ...} |
   -bottom_polygon {{x1 y1} {x2 y2}...}}}}}
[-top_layer_masks int+]
```

Creates a via definition (master) from the specified parameters or list of shapes. The via masters created are design (DEF) vias and are for use in DEF SPECIALNETS wiring, unless the modify_ndr command is used to add them the list of valid vias for a NONDEFAULTRULE.

There are two types of via-master definitions:

- Parameterized vias that require a `-via_rule name`, and a set of row, column, spacing and enclosure parameters. These are equivalent to LEF/DEF VIAs defined with VIARULE GENERATE or OpenAccess standardViaDef. Applications normally check the via parameters for comparing and finding via-masters. But for LEF/DEF and reporting purposes, the via-master name is automatically generated using the `-via_rule name` as a prefix with a "_1", "_2", etc. suffix added to make it unique so it will not collide with any existing via-master names. The auto-generated name is only valid during the current session, and may be different after save/restore and opening it in another session. If a via-master already exists with the same parameters, the existing via-master is returned instead of creating a redundant via-master. Hence, this command can also be used to "lookup" any equivalent via-master previously created.
- Fixed vias that require a `-name` and a list of rectangles and polygons. These are equivalent to LEF/DEF VIAs using the RECT and POLYGON statements, or OpenAccess customViaDefs. The via name is the only way to lookup the via-master, and must be unique. The name cannot collide with an existing via-master. These types of via-masters are normally not recommended for design specific or DEF vias, because other blocks may create via-master names that collide. These types of vias are normally only in the library (LEF or OA techfile) so they are shared with all block of the same chip.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `add_via_definition` parameter. For a detailed description of the command and all its parameters, use the man command: `man add_via_definition`
`-bottom_enclosure {x y}`	Specifies the bottom layer enclosure around the cut array. The enclosure measures the distance from the cut-array-edge to the metal-edge that encloses the cut-array. Values are specified in microns. Requires `-via_rule` option for a parameterized via. *Default:* Derived from `-via_rule`.
`-bottom_layer layer`	If not specified, the first layer below the `-cut_layer` is assumed. Requires `-name` option for a fixed via.
`-bottom_layer_masks int+`	Specifies the masks for the `rects` on the bottom layer for a `via_def`. The mask values and `rects` must be in the same order and there must be the same number of mask values as `rects`.
`-bottom_offset {x y}`	Specifies the `x,y` offset for bottom layer in the via. After the "non-shifted via" is computed, the bottom layer rectangle is offset by adding the x,y to the `-origin` offset. Values are specified in microns. Requires `-via_rule` option for a parameterized via. *Default*: {0 0}.
`-bottom_polygon {{x1 y1} {x2 y2}...}`	
	Specifies the polygon to create on the bottom layer. Values are specified in microns. Requires `-name` option for a fixed via.
`-bottom_rects {{x1 y1 x2 y2} {x3 y3 x4 y4} ...}`	Specifies the list of rectangles to create on the bottom layer. Values are specified in microns. Requires `-name` option for a fixed via.

`-cut_layer layer`	Specifies the name or pointer of the cut layer. The top/bottom layers of the via are derived from the routing layers above/below the cut layer. Requires `-name` option for a fixed via.
`-cut_masks int+`	Specifies the masks for the `rects` on the cut layer for a `via_def`. The mask values and `rects` must be in the same order and there must be the same number of mask values as `rects`.
`-cut_rects {{x1 y1 x2 y2} {x3 y3 x4 y4} ...}`	Specifies the list of rectangles to create on the cut layer. Values are specified in microns. Requires `-name` option for a fixed via.
`-cut_size {x y}`	Specifies the size (x=width, y=height) of the cut rectangles. Values are specified in microns and must be greater than 0. Requires `-via_rule` option for a parameterized via. *Default*: Derived from `-via_rule`.
`-cut_spacing {x y}`	Specifies the spacing between the cuts. The spacing is measured from cut-edge to the next cut-edge. Values are specified in microns and must be greater than 0. Requires `-via_rule` option. *Default*: Derived from `-via_rule`.
`-name viaName`	Specifies the name of via to create. For parameterized via definition creation, a new via name is derived from the `-via_rule` name unless `-name` is specified. The name cannot collide with any existing via-master names.
`-origin {x y}`	Specifies the x,y offset for all shapes in the via. By default the 0,0 origin of the via is the center of the cut-array and the center of the enclosing metal rectangles. After the “non-shifted via” is computed, all the cut and metal rectangles are offset by adding these values. size (x=width, y=height) of the cut rectangles. Values are specified in microns. Requires `-via_rule` option for a parameterized via. *Default*: {0 0}.
`-pattern pattern`	Specifies the cut pattern encoded as an ASCII string. This parameter is only required when some of the cuts are missing from the array of cuts. Requires `-via_rule` option for a parameterized via.
`-row_col {x y}`	Specifies the number of cut rows and columns to create. Values must be greater than 0. Requires `-via_rule` option for a parameterized via. *Default*: {1 1}.
`-top_enclosure {x y}`	Specifies the top layer enclosure around the cut array. The enclosure measures the distance from the cut-array-edge to the metal-edge that encloses the cut-array. Values are specified in microns. Requires `-via_rule` option for a parameterized via. *Default*: Derived from `-via_rule`.
`-top_layer layer`	If not specified the first layer above the `-cut_layer` is assumed. Requires `-name` option for a fixed via.
`-top_layer_masks int+`	Specifies the masks for the `rects` on the top layer for a `via_def`. The mask values and `rects` must be in the same order and there must be the same number of mask values as `rects`.
`-top_offset {x y}`	The x,y offset for top layer in the via. After the “non-shifted via” is computed, the top layer rectangle is offset by adding the x,y to the `-origin` offset. Values are specified in microns. Requires `-via_rule` option for a parameterized via. *Default*: {0 0}.
`-top_polygon {{x1 y1}{x2 y2}...}`	Specifies the polygon to create on the top layer. Values are specified in microns. Requires `-name` option for a fixed via.
`-top_rects {{x1 y1 x2 y2} {x3 y3 x4 y4} ...}`	Specifies the list of rectangles to create on the top layer. Values are specified in microns. Requires `-name` option for a fixed via.
`-via_rule via_def_rule`	Specifies the name or pointer of the via-rule (from LEF VIARULE GENERATE or OpenAccess standardViaDef) to use for the via master being created. Layers and default values are derived from the specified `via_rule`. When using `-via_rule`, a parameterized via master is created.

Examples

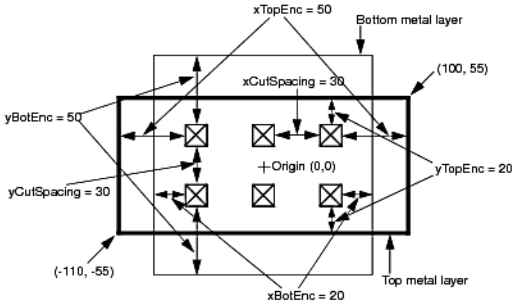
Example 1

Create a 1x2 cut via with default parameters.

```
set myvia [add_via_definition -via_rule via1_rule -row_col {1 2}]
dbGet $myvia.name #returns the via-master name
```

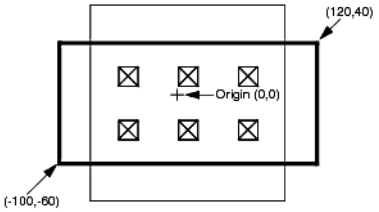
The following parameterized via-master describes a non-shifted via with two rows and three columns of via cuts.
In this example, all the size, spacing, and enclosure rules are given explicitly. In typical usage, they would not be specified and would use the default values in the via-rule.

```
add_via_definition -via_rule via1_rule -row_col {2 3} -cut_size {20 20} \
-cut_spacing {30 30} -bottom_enclosure {20 50} -top_enclosure {50 20}
```



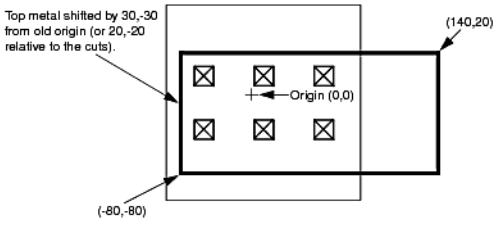
The same via rule with the –origin shifts all of the metal and cut rectangles by 10 in the x direction, and by -10 in the y direction.

```
add_via_definition -via_rule via1_rule -row_col {2 3} -cut_size {20 20} \
-cut_spacing {30 30} -bottom_enclosure {20 50} -top_enclosure {50 20} \
-origin (10 -10)
```



If the same via contains the following –origin and –top_offset parameters, all of the rectangles shift by 10, -10.
In addition, the top layer metal rectangle shifts by 20, -20, which means that the top metal shifts by a total of 30, -30.

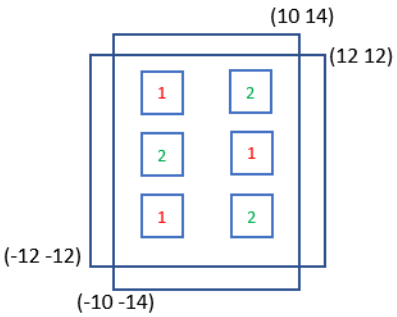
```
add_via_definition -via_rule via1_rule -row_col {2 3} -cut_size {20 20} \
-cut_spacing {30 30} -bottom_enclosure {20 50} -top_enclosure {50 20} \
-origin (10 -10) -top_offset {20 -20}
```



Example 2

The following command adds a `3x2` fixed via to the cut layer mask pattern `{1 2 1 2 1 2}`:

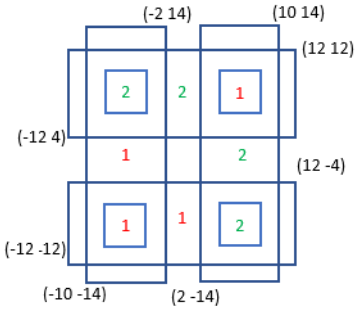
```
add_via_definition  -name my_fixed_via -cut_layer VIA23 \
-bottom_rects {{-10 -14 10 14}} \
-cut_rects {{-6 -10 -2 -6} {-6 -2 -2 2} {-6 6 -2 10} {2 -10 6 -6} {2 -2 6 2} {2 6 6 10}} \
-top_rects {{-12 -12 12 12}} \
-cut_masks {1 2 1 2 1 2}
```



Example 3

The following command adds a `2x2` fixed via to the bottom and top layer mask pattern `{1 2}` and cut layer mask pattern `{1 2 1 2}`:

```
add_via_definition -name my_fixed_via2 -cut_layer VIA23 \
-bottom_rects {{-10 -14 -2 14} {2 -14 10 14}} \
-cut_rects{{-8 -10 -4 -6} {-8 6 -4 10} {4 -10 8 -6} {4 6 8 10}} \
-top_rects {{-12 -12 12 -4} {-12 4 12 12}} \
-bottom_layer_masks {1 2} \
-top_layer_masks {1 2} \
-cut_masks {1 2 1 2}
```



applyGlobalNets

```
applyGlobalNets
[-help]
```

Applies or restores the global net connectivity rules to the design and creates the necessary connections between instances and these global nets.

Parameters

`-help`	Prints a brief description of the `applyGlobalNets` command usage. For a detailed description of the command, use the `man` command: `man applyGlobalNets`

Related Topics

Power Planning and Routing chapter in the *Innovus User Guide.*

clearGlobalNets

```
clearGlobalNets [-help]
```

Resets the logical power net connection. It also clears all the tie-high and tie-low nets logical power net connection.

Parameters

`-help`	Prints a brief description of the `clearGlobalNets` command usage. For a detailed description of the command, use the `man` command: `man clearGlobalNets`

Examples

If the logical connectivity of a `VDD` pin is set with `VDD!` power nets, `VSS` pin with `VSS!` and so on using the globalNetConnect command, the clearGlobalNets command will clear or undo all this logical power nets connectivity. Refer to the commands below:

```
globalNetConnect VDD! -pin VDD -override
globalNetConnect VSS! -pin VSS -override
clearGlobalNets
```

Related Topics

Power Planning and Routing chapter in the *Innovus User Guide*.

compress_special

```
compress_special
[-help]
[-area {x1 y1 x2 y2}]
[-incremental]
[-layers layer+]
[-nets net+]
```

Compresses Power-Grid (PG) based on the specified area, layer, or net.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `compress_special` parameter. For a detailed description of the command and all its parameters, use the man command: `man compress_special`
`-area {x1 y1 x2 y2}`	Specifies the coordinates for the area from which PG needs to be compressed. The `compress_special` command compresses only those special wires and special vias, which exist within the area specified by this parameter. *Default*: All PG structures are compressed.
`-incremental`	Recompresses the PG after editing the PG wires or vias. This parameter can only be specified when the current design is in compressed mode.
`-layers layer+`	Specifies the layers on which PG compression is needed. This parameter requires either LEF layer names or a routing layer index value (for example, `1` for the first routing layer, `2` for the second routing layer, and so on).
`-nets net+`	Specifies the nets from which PG compression is needed. If the specified net does not have any special route on it, it is ignored.

Example

The following command compresses `%s special_wires` and `%s special_vias`:

```
compress_special -area {100 100 120 120} -net VDD -incremental
```

Related Topics

`decompress_special`

create_virtual_shape_on_pg_wires

```
create_virtual_shape_on_pg_wires
[-help]
-layers {{virtual_layer, power_layer [mask_num]} ...}
[-name string]
[-pg_width_list {list_of_supported_width_values} | -pg_width_range {min_width max_width}]
```

Generates additional virtual shapes and creates markers on PG wires, when the existing trim shapes overlap with PG wires. The generated virtual shapes are routing blockages on the specified layer. These virtual shapes can be based on the output of the `defOut` and `streamOut` commands, and edited by the wire editing commands.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `create_virtual_shape_on_pg_wires` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man create_virtual_shape_on_pg_wires`
`-layers {{virtual_layer, power_layer [mask_num]} ...}`	Specifies the pair of layers, which need to create the virtual shapes. You can specify the optional argument `mask_num` with this parameter to select the PG wires.
`-name string`	Specifies the name of the marker shape to be generated. *Default*: `defLayerBlkName`
`-pg_width_list {list_of_supported_width_values}`	Creates virtual shapes on PG pin only for the specified width values. **Note:** This parameter cannot work with the `-pg_width_range` parameter. It means that you can define either a range of width, or a list of width. You can use the `-pg_width_list` parameter with the `-layer` and `-name` parameters.
`-pg_width_range {min_width max_width}`	Specifies the PG width range in which the virtual shapes are needed. *Default*: All the PG wires.

db_pt

```
db_pt
{-x | -y}
pt
[-help]
```

Returns the X or Y coordinate of any point. When you need to find the coordinates of a shape in order to either move the shape or change a coordinate of a shape, the command helps you find the coordinates.

Parameters

`{-x | -y}`	Returns the coordinates of the points. • `-x` returns `x` • `-y` returns `y`
pt	Specifies the point to access the value from. This is a mandatory parameter.
`-help`	Outputs a brief description that includes the type and default information for each `db_pt` parameter. For a detailed description of the command and all its parameters, use the man command: `man db_pt`

Example

- `db_pt -x {10.1 11.1}` returns `10.1`
- `db_pt -y {10.1 11.1}` returns `11.1`
- `db_pt 10.1 11.2 -y returns 11.2`

Related Commands

`db_rect`

db_rect

```
db_rect
[-help]
<rect> {-ll | -ur | -llx | -lly | -urx | -ury | -size | -sizex | -sizey | -width | -length | -center | -area}
```

Returns information about a rectangle, like coordinates, size, and area. When you need to find information about a rectangle, this command helps to find it.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `db_rect` parameter. For a detailed description of the command and all its parameters, use the man command: `man db_rect`
`<rect> {-ll | -ur | -llx | -lly | -urx | -ury | -size | -sizex | -sizey | -width | -length | -center | -area}`	
	Specified rectangle to access values from. This is a mandatory parameter. • `-area`: Returns the area of the rectangle. • `-center`: Returns the center of the rectangle. • `-length`: Returns the larger of the `-sizex` and `-sizey` values. • `-ll`: Returns the lower left point of the rectangle. • `-llx`: Returns the lower left x coordinate of the rectangle. • `-lly`: Returns the lower left y coordinate of the rectangle. • `-size`: Returns the size of the rectangle from lower left to upper right. • `-sizex`: Returns the x difference between the larger and smaller x values of the rectangle. • `-sizey`: Returns the y difference between the larger and smaller y values of the rectangle. • `-ur`: Returns the upper right point of the rectangle. • `-urx`: Returns the upper right x coordinate of the rectangle. • `-ury`: Returns the upper right y coordinate of the rectangle. • `-width`: Returns the smaller of the `-sizex` and `-sizey` values.

Examples

- `db_rect -area {8.1 9.2 10.3 11.4}` returns `4.84`
- `db_rect -center {8.1 9.2 10.3 11.4}` returns `9.2 10.3`
- `db_rect -length {8.1 9.2 10.3 11.4}` returns: `2.2`
- `db_rect -ll {8.1 9.2 10.3 11.4}` returns: `8.1 9.2`
- `db_rect -llx {8.1 9.2 10.3 11.4}` returns: `8.1`
- `db_rect -lly {8.1 9.2 10.3 11.4}` returns: `9.2`
- `db_rect -size {8.1 9.2 10.3 11.4}` returns: `2.2 2.2`
- `db_rect -sizex {8.1 9.2 10.3 11.9}` returns: `2.2`
- `db_rect -sizey {8.1 9.2 10.3 11.9}` returns: `2.7`
- `db_rect -ur {8.1 9.2 10.3 11.4}` returns: `10.3 11.4`
- `db_rect -urx {8.1 9.2 10.3 11.4}` returns: `10.3`
- `db_rect -ury {8.1 9.2 10.3 11.4}` returns: `11.4`
- `db_rect -width {8.1 9.2 10.3 11.4}` returns: `2.2`

Related Commands

db_pt

dbEdge

```
dbEdge
[-clockwise | -counterclockwise]
[{shapelist} [-direction direction] [-d]]
[-index index_number]
```

Returns the edges of a polygon of a shape along with the direction list (N, S, E, W and NE, NW, SE, SW). This helps you identify not only all the edges but also the direction in which the edges are pointing. At a time you can observe one polygon with 'n' number of points.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `dbedge` parameter. For a detailed description of the command and all its parameters, use the man command: `man dbEdge`
`-clockwise`	Sorts the returned list of edges clockwise.
`-counterclockwise`	Sorts the returned list of edges counterclockwise.
`-direction {N S E W NE NW SE SW}`	Returns the edges of a polygon of a shape along with the direction list (N, S, E, W and NE, NW, SE, SW). It can also be a list. For example: `dbEdge -direction {N S NW NE} {10 20 30 40}` If the `-direction` parameter is not used, all the directions in the shape will be returned.
`-d`	Returns user-specified values in database units. Default: All values are in user units, in microns.
`-index index_number`	Specifies the index number of the edge to be returned. The index number starts with `0` as the first edge specified in the shape list, and increases gradually along with the shape list. This number may range from `0` to '*number_of_edges* - 1'.

`{shapelist}`	Can be a rectangle like `{llx lly urx ury}` or a polygon like `{{x1 y1}{x2 y2}{x3 y3}.....}`. Only one rectangle or polygon is accepted. This is a mandatory parameter.
	Examples
	Using a simple L-shape polygon
	`set shape_list {{0 0} {2 0} {2 1} {1 1} {1 2} {0 2}}`
	• All edges
	`dbEdge $shape_list`
	`{{S{0 0}{2 0}} {E{2 0}{2 1}} {N{2 1}{1 1}} {E{1 1}{1 2}} {N{1 2}{0 2}} {W{0 2}{0 0}}}`
	• Only E facing edges
	`dbEdge -direction E $shape_list`
	`{{E{2 0} {2 1}} {E{1 1} {1 2}}}`
	The format of the output is
	`{{<direction> <edge_vertex_1> <edge_vertex_2 }....}`
	Note: The polygon must be rectilinear. Otherwise, the following error occurs:
	`set bad_shape_list {{0 0} {2 0} {2 1} {1 1} {1 2}}`
	`dbEdge $bad_shape_list`
	`**ERROR: (IMPDBTCL-621): The slope of the edge { {1 2} {0 0} } does not match any of the standard directions {N | S | E | W | NE | NW | SE | SW}.`

Related Commands

- db_rect
- dbShape

dbGet

```
dbGet
[-help]
[. objType ]…[. attrName | .? | .?? | .?h]
[-d]
[-e]
[expression]
[-i num]
[-l]
{obj | objList | head | top | selected}
[-p num]
[pattern]
[-regexp]
[-u]
[-v]
```

Returns object and attribute information for the specified database object in the design. The `dbGet` command takes an object or object list as a starting point, then uses a period to traverse to other related objects, or to access attributes. Additional periods can be used to continue traversing the database schema.

For example, the following command returns the master cell names for all of the instances in the design:

```
dbGet top.insts.cell.name
```

- `top` is a pointer to the top cell.
- `insts` is a list of pointers to the instances in the top cell.
- `cell` is a list of pointers to the master cells for each instance.
- `name` is the master cell name.

Copying and pasting pointers in an active session may cause the tool to crash. To prevent this situation, do not:

- Copy pointers from the output of the `dbGet` command and paste to another command as input.
- Copy a pointer from one session to another.

The `dbGet` command is case insensitive. An empty field returns a value of `0x0`.

For a list of the object attributes that can be queried for information, or changed, see Innovus Database Object Information.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `dbGet` parameter. For a detailed description of the command and all its parameters, use the man command: `man dbGet`
`-e`	To avoid NULL (0x0) pointers and results from being printed. This is an optional option of type boolean and would effect all kinds of outputs. In cases of nested `dbGet` statements: • The option should not be used in the inner `dbGet` statement. Correct usage example: `dbGet -e [dbGet -p2 [dbGet -p top.insts.pStatus fixed].cell.name INV1]` • If none of the instances have 'fixed' placement status, the outer dbGet will not return a 0x0 but an empty list which TCL would not consider as a list element. For example, `dbGet [dbGet -p2 [dbGet -p -e top.insts.pStatus fixed].cell.name INV1]`
`-i` *num*	For selecting a specific element from the list being returned as output. The option `-i` *num* would select the the *num*th element starting from 0. The option would also reduce a starting '{' and ending '}' for box and pt type attributes Example : `dbGet top.nets.name sum* -p -i 0` The option would return first pointer in the list of the nets whose name matches the pattern sum*
`-l`	Returns the result with auto-newline.
`-p` *num*	Specifies the level to traverse back through the specified objects for the query. For example, if you specify `-p` or `-p1`, the software traverses one level back through the specified objects. If you specify `-p2`, the software traverses two levels back through the specified objects.The `-p` parameter is used with pattern name matching (*pattern*) to create a subset of names by which to query.
`-u`	Removes duplicate objects from the query results so that the results list only contains unique entries. For information such as cell names, the results list can have the same entry multiple times. You can use the `-u` parameter to filter the results, so that the software only returns a unique list of different cell names.
`-regexp`	Uses regular expression pattern matching (Tcl `regexp` command). *Default:* Uses simple wild card pattern matching
`-d`	Returns values in database integers. *Default:* Returns values in floating point user units
`{`*objList* `| head | top | selected}`	
	Specifies the starting point for the query. • `objList`: Specifies one or more object pointers as the starting point. You can specify a homogeneous or heterogeneous list. • `head`: Specifies the pointer for the root, or head of the design as the starting point (also known as `dbgHead`). • `top`: Specifies the pointer to the top cell in the design as the starting point (also known as `dbgTopCell`). • `selected`: Specifies that the selected objects are the starting point.
`.`*objType*`*`	Specifies the object type to query.
`{`*attrNname* `| .? | .?? | .?h}`	

	Specifies the type of information to return. • *.attrName:* Returns the value of the specified object attribute. • .?: Returns a list of the available objects and attributes for the last object in the query. The software returns the object and attribute names only, not their values. For a list of objects, the field information is returned once for each type of object in the list. Most lists are of the same type; therefore, returning the same fields many times does not make sense. • .??: Returns a list of the objects and attributes, and their values, for the last object in the query. Use the setdbGetMode command to set controls for displaying `.??` query information. • .?h: Returns a list of the available objects and attributes for the last object in the query, and also includes the following information: ◦ A short description for each object and attribute. ◦ The type for each object. ◦ The legal enum values for each attribute. ◦ Whether the attribute value can be set (whether it is editable).

pattern	Specifies a string expression to use to match object or attribute names, or attribute values.
	When you use *pattern* to match object or attribute names, the *pattern* is case insensitive. For example the following commands all return the instance placement status (`inst: pStatus`):
	`dbGet top.insts.? ps*`
	`dbGet top.insts.? PS*`
	`dbGet top.insts.? pS*`
	`dbGet top.insts.? pstatus`
	`dbGet top.insts.? PSTATUS`
	`dbGet top.insts.? pStatus`
	When you use *pattern* to match attribute values (strings or enums), the *pattern* is case sensitive. For example, the following command returns the list of pointers for the instances that have `placed` status:
	`dbGet -p top.insts.pStatus placed`
	However, the following command returns `0x0`, because `PLACED` does not match the legal `placed` enum value:
	`dbGet -p top.insts.pStatus PLACED`
	Default: The software uses simple wildcard matching (`*`, `?`).

expression	Specifies a Tcl expression where values of attributes and also of objects can be accessed and utilized in Tcl "expr" style comparisons.
	Note: Only child attributes and objects of the obect to be tested can be accessed, no additional traversal is allowed.
	`dbGet top.insts {.cell.name == "myBlock"}`
	is not allowed, the correct usage would be:
	`set myBlock [dbGet -p head.allCells.name myBlock]`
	`dbGet top.insts {.cell == $myBlock}`
	For example,
	• Instead of a limited search like:
	`dbGet -p head.allCells.numRefs 5`
	Which is now equivalent to:
	`dbGet head.allCells {.numRefs == 5}`
	The search can also include a number of attributes and objects inside a Tcl expression, such as:
	`dbGet head.allCells {.numRefs > 5 && [string match AND* .name]}`
	• For objects, note that the search will be limited to the children of the current object.
	By a normal `dbGet` *`pattern`* method you use:
	`dbGet -p2 top.nets.sWires.layer.extName metal2`
	When you use the the *`expression`* method:
	`set L2 [dbGet -p head.layers.extName metal2]`
	`dbGet top.nets.sWires {$L2 == .layer}`
	• Logical matching:
	`dbGet top.nets {.name ne "clk"}`
	Returns a list of pointers to nets whose names do not match "clk".

`-v`	Returns all attributes that do not match the `pattern/ expression` or Reverses the result of `pattern/expression` matching. This is similar to the `grep -v` option in UNIX. **Example:** `dbGet -v top.nets.name clk*` Returns all net names that do not match the `clk*` pattern
• Expanded numeric matching: `dbGet top.nets {.numInputTerms > 5}`	Returns a list of pointers to nets that have more than five input terminals. • Multiple attribute matching: accessing multiple attributes of the same object. `dbGet top.nets {.name ne "clk" && .numInputTerms > 5 }` Returns list of pointers to nets with at least one input, but less than or equal to five input. **Note:** The `dbGet` command can not be executed inside a prior instance of `dbGet`. ⓘ Tokens in the expressions which begin a `.' and that are not intened to reference an attribute should be passed inside a Tcl variable, otherwise the expression will try and reference an attribute that does not exist (example: `> dbGet top.insts {[string match .xyzzy .name]}` should be changed to `> set test_name .xyzzy` `> dbGet top.insts {[string match $test_name .name]}`

Note 1: The system can make suggestions or auto-complete entries using predictive text in case you enter a wrong `dbGet` command. The system no longer throws up errors, instead it lists all the probable commands starting with the letters entered so that you can choose which the one which you want to use. This behavior is implemented with the use of the *tab*, like when you press the *Esc* key in UNIX.

For example,

```
innovus 1> dbGet top.n<tab>
name numInputs numPGTerms numPhysTerms
```

```
nets numInsts numPhysInsts numTerms
numBidirs numNets numPhysNets

innovus 1> dbGet top.<tab>
bumps numPhysNets statusPlaced
fPlan numPhysTerms statusPowerAnalyzed
hInst numTerms statusRCExtracted
insts objType statusRouted
markers pgTerms statusScanOpted
name physInsts symmetryR90
nets physNets symmetryX
numBidirs physTerms symmetryY
numInputs pinToCornerDist terms
numInsts props texts
numNets statusClockSynthesized
numPGTerms statusGRouted
numPhysInsts statusIoPlaced

innovus 1> dbGet top.insts.<tab>
box isJtagElem pHaloBot pgCellTerms rHaloTopLayer
boxes isPhysOnly pHaloBox pgTermNets
cell isSpareGate pHaloLeft props
instTerms name pHaloRight pt
isDontTouch objType pHaloTop rHaloBotLayer
isHaloBlock orient pStatus rHaloSideSize

innovus 1> dbGet top.insts.instTerms.name
```

Note 2: The "return" value from `dbGet` is always a list or a string, so when the attribute that is being queried is a list (like `pt` and `rect` attributes) the return result is a list-of-lists structure, even if there is only one internal list being returned. For example:

`$instPtrList` is a pointer to a list of instances
`$instPtr` is a pointer to a single instance

```
> dbGet $instPtrList.pt
{65.34 85.68} {81.84 65.52} {83.16 85.68} ...
```

What is listed above is what is printed to the screen. But since the value is a string and the quotes (") are not displayed.
The actual return result from a scripting point of view is:
`"{65.34 85.68} {81.84 65.52} {83.16 85.68} ..."`

Similarly, in case of a single instance:

```
> dbGet $instPtr.pt
{65.34 85.68}
```

What is listed above is what is printed to the screen. But since the value is a string and the quotes (") are not displayed, the actual return result from a scripting point of view is:

```
"{65.34 85.68}"
```

So, to access the X value for the instance,

```
set x [lindex [lindex [dbGet $instPtrList.pt] 0] 0]
```

or

```
set x [lindex [join [dbGet $instPtrList.pt]] 0]
```

Usage in foreach, as foreach breaks down the list-of-lists into each pt.

```
foreach pt [dbGet $instPtrList.pt] {puts "X= [lindex $pt 0]"}
```

As the behavior of `dbGet` always returns the list-of-lists form, the foreach can be used even if there is only a single internal list.

Frequently used queries

Get the attributes of the selected object	`dbGet selected.??`
Find all nets with name including “clk”	`dbGet top.nets.name *clk*`
All instance pointers of a particular cell type	`dbGet -p2 top.insts.cell.name INV1`
Get all of the inverter lib cells	`dbGet [dbGet -p head.libCells.isInverter 1].name`
All the cell names, which are used in the design	`dbGet -u top.insts.cell.name`
The num of insts in the design	`dbGet top.numInsts`
All the physical only insts	`dbGet [dbGet -p top.insts.isPhysOnly 1].name`
Get all power and ground name in the design	`dbGet [dbGet -p top.nets.isPwrOrGnd 1].name`
Find the cell name of an instance	`dbGet [dbGet -p top.insts.name pm_arbiter/mc_pm_arbiter_regs/U290].cell.name`
Pointers to all registers in the design	`dbGet -p2 top.insts.cell.isSequential 1`
All blocks/macros	`dbGet -p2 top.insts.cell.subClass block`
All top level ports	`dbGet top.terms.name`
Pointers to all CTS clock nets	`dbGet -p1 top.nets.isCTSClock 1`
All of the nets skipped during routing	`dbGet [dbGet top.nets.skipRouting 1 -p].name`
Find the net connected to a pin	`dbGet [dbGet -p top.insts.instTerms.name <full pin name>].net.name`
Pointers to all fixed nets	`dbGet -p2 top.nets.wires.status fixed`
Pointers to all donttouch insts	`dbGet top.insts.isDontTouch 1 -p`
Find all the power domains	`dbGet top.fplan.groups.pd.name`
Find nets with no DEF SPECIALNETS wiring	`dbGet [dbGet top.nets {.sWires == 0x0 && .sVias == 0x0}].name`

Find the metal2 routing pitch (X)	`dbGet [dbGet -p [dbGet -p1 head.layers.type routing].name *2].pitchX`
	or
	`dbGet [dbGet head.layers.name metal2 -p].pitchX`

Find all instance connected to a port	`dbGet [dbGet -p top.terms.name <port_name>].net.instTerms.inst.name`
Pointers to AND gates that used more than 5 times in the design	`dbGet head.allCells {.numRefs > 5 && [string match AND* .name]}`
Pointers to cells that are used more than 10 times in the design	`dbGet [dbGet -p head.allCells.objType libCell] {.numRefs > 10}`
Pointers to cells that have 4 or less inputs and more than 2 PG terms	`dbGet [dbGet -p head.allCells.objType libCell] {.numInputs <= 4 && .numPGTerms > 2}`
Pointers to non-clk nets that have more than five input terminals	`dbGet top.nets {.name ne "clk" && .numInputTerms > 5 }`

Examples

- The following command returns the instance names for the instances with names that match the `*reg*` pattern:
  `dbGet top.insts.name *reg*`
- The following command returns the names of all nets that start with the letter `I`. Specifying this command returns the same information as specifying the `dbFindNetsByName I*` command.
  `dbGet top.nets.name I*`
- The following command returns the names of all terminals that start with the letter `I`. Specifying this command returns the same information as specifying the `dbFindTermsByName I*` command.
  `dbGet top.terms.name I*`
- The following command returns a list of pointers to the instances that have a placement status of `Fixed`:
  `dbGet -p top.insts.pStatus fixed`

- The following command returns a list of names for instances that have a placement status of `Fixed`:
  ```
  dbGet [dbGet -p top.insts.pStatus fixed].name
  ```
- The following commands also return a list of instance names for instances that have a placement status of Fixed:
  ```
  set fixedInst [dbGet -p top.insts.pStatus fixed]
  dbGet $fixedInsts.name
  ```
- The following command returns the status of the top cell in the design:
  ```
  dbGet top.?? status*
  ```

  The software displays the following information:
  ```
  statusClockSynthesized: 0
  statusGRouted: 0
  statusIoPlaced: 1
  statusPlaced: 1
  statusPowerAnalyzed: 0
  statusRCExtracted: 0
  statusRouted: 0
  statusScanOpted: 0
  ```
- The following command returns a list of the available objects and attributes for instances of the top cell:
  ```
  dbGet top.insts.?
  ```

  The software displays the following information:
  ```
  inst: box cell instTerms isDontTouch isHaloBlock isJtagElem isPhysOnly isSpareGate name objType orient pStatus pgCellTerms pgTermNets
  pt
  ```
- The following command displays the available objects and attributes, and their values, for the instances in the top cell. By default, the software displays information for the first 10 instances only. Use the `setdbGetMode` command to change the display limit.
  ```
  dbGet top.insts.??
  ```

  The software displays the following information:
  ```
  box: {314.16 302.4 316.8 307.44}
  cell: 0x12186fa8
  instTerms: 0x1330fa38 0x1330fa6c
  isDontTouch: 0
  isHaloBlock: 0
  isJtagElem: 0
  isPhysOnly: 0
  isSpareGate: 0
  name: i_10181
  objType : inst
  orient : MX
  pStatus : placed
  pgCellTerms: 0x13811088 0x138110cc
  pgTermNets: 0x0
  pt: {314.16 302.4}
  box: {258.72 282.24 261.36 287.28}
  cell: 0x12186c38
  instTerms: 0x1330faa0 0x1330fad4
  isDontTouch: 0
  isHaloBlock: 0
  isJtagElem: 0
  isPhysOnly: 0
  isSpareGate: 0
  name: i_10177
  objType : inst
  orient : MX
  ```

```
pStatus : placed
pgCellTerms: 0x13810ef0 0x13810f34
pgTermNets: 0x0
pt: {258.72 282.24}
...
```

- The following command returns a list of the available objects and attributes for instances of the top cell. The software also displays a short description for each object and attribute, the type of each object, the legal enum values for each attribute, and whether the attribute value can be set by the user.

  ```
  dbGet top.insts.?h
  ```

  The software displays the following information:

  ```
  ========================================================================
  inst: Instance - canonical (flat), equivalent to DEF COMPONENT. Points to a libCell or ptnCell
  ========================================================================
  box: rect, Bounding box of instance
  cell: obj(libCell ptnCell), Pointer to child cell master or ptnCell
  instTerms: objList(instTerm), List of pointers to instance terminal
  isDontTouch: bool, Indicates that the instance is marked Don't Touch
  isHaloBlock: bool, Indicates that the instance is a halo block
  isJtagElem: bool, Indicates that the instance is a Jtag element
  isPhysOnly: bool, Indicates that the instance is physicl only
  isSpareGate: bool, Indicates that the instance is a spare gate
  name: string, Fully qualified (path) name of the instance
  objType: enum(bndry densityShape fPlan group hInst hInstTerm hNet hTerm head inst instTerm layer libCell net netGroup pBlkg pd pinGroup
  prop ptnCell rBlkg sNet sWire shape stdRow term topCell vCell via wire), Object type: Instance
  orient(settable): enum(MX MX90 MY MY90 R0 R180 R270 R90 Unknown), Instance placement orientation
  pStatus(settable): enum(cover fixed placed unplaced), Instance placement status
  pgCellTerms: objList(term), List of pointers to the instance's power terminals
  pgTermNets: objList(net), List of pointers to nets attached to P/G terminals
  pt(settable): pt, Location of the instance
  b
  ```

- In order to query all nets that have no wiring (regular or special) more efficently, use:

  ```
  set netsWithNoRouting [dbGet top.nets {.wires == 0x0 && .vias == 0x0 && .sWires== 0x0 && .sVias == 0x0} ]
  foreach netWithNoRouting $netsWithNoRouting { Puts "** Net [dbGet $netWithNoRouting.name] has no wiring."}
  ```

- In order to display the Power/Ground connections for a specified instance, use:

  ```
  set inst [dbGet -p top.insts.name my_inst_name]
  foreach pgCellTermName [dbGet $inst.pgCellTerms.name] pgTermNetName [dbGet $inst.pgTermNets.name]
  {Puts "Instance \"[dbGet $inst.name]\" terminal \"$pgCellTermName\" is connected to net \"$pgTermNetName\"."}
  ```

- **Finding nets with no DEF SPECIALNETS wiring**

  ```
  set netNames [dbGet [dbGet top.nets {.sWires == 0x0 && .sVias == 0x0}].name]
  foreach netName $netNames {
     Puts "$netName has no special wiring"
  }
  vs
  foreach net [dbGet top.nets] {
     if { [dbGet $net.sWires] == 0x0 && [dbGet $net.sVias] == 0x0 } {
   Puts "[dbGet $net.name] has no special wiring"
     }
  }
  ```

The first method calls `dbGet` once to access the DEF SPECIALNETS data, the second makes 2xN calls (where N is the number of nets in the design) to access the DEF SPECIALNETS data. The calls to access the DEF SPECIALNETS data do an internal initialization that can be expensive on large designs, so minimizing the number of times that happens can greatly improve the run time of a proc/script.

dbQuery

```
dbQuery
[-help]
[-areas {{llx1 lly1 urx1 ury1} {llx2 lly2 urx2 ury2} ...}]
[-bbox_overlap]
[-d]
[-layers {layer1 layer2 ...} [-strict_via_inst_layer_check]]
[-objType {bump busGuide inst instAllShapes instTerm pgInstTerm term marker net pBlkg pinShape pWire rBlkg regular resizeBlkg row special sViaInst sWire viaInst wire whatIfWire whatIfVia}]
[-polygons {x11 y11 x12 y12 x13 y13 ...}]
[-abut_only] | [-enclosed_only] | [-overlap_only]
```

Returns a list of objects that overlap or abut specific areas by default. It uses the polygon outline of the objects to check for overlap, as described in the description of the `-objType` parameter. This command supports all types of routing blockages such as partial, fill, and slot.

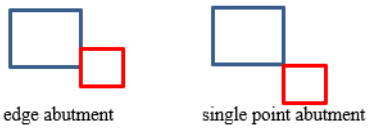
Parameters

`-help`	Prints a brief description that includes the type and default information for each `dbQuery` parameter. For a detailed description of the command and all its parameters, use the man command: `man dbQuery`
`-abut_only`	Returns only those objects that abut the search area. No part of the object polygon overlaps the search area, but at least one point or line of intersection occurs between the object polygon and the search area. A single point of abutment (such as a corner or crossing edges) is also considered as abutted. When multiple areas are specified, the object is returned as long as it abuts one of the search areas.
`-areas {{llx1 lly1 urx1 ury1} {llx2 lly2 urx2 ury2} ...}`	
	Specifies the search areas, where `ll` = lower left, `ur` = upper right. By default, all the objects overlapping or abutting these areas are returned, unless `-abut_only`, `-enclosed_only`, or `-overlap_only` is specified. The polygon shape of the object is used for the overlap check. The units are in microns (unless `-d` is specified). The input can be in either a single `rect {llx1 lly1 urx1 ury1}`, or a list of `rects {{llx1 lly1 urx1 ury1} {llx2 lly2 urx2 ury2}}`. A `rect` using "two points" format like `{{{llx1 lly1} {urx1 ury1}} {{llx2 lly2} {urx2 ury2}}}` is also allowed. But input format like `{llx1 lly1 urx1 ury1 llx2 lly2 urx2 ury2}` or `{{llx1 lly1} {urx1 ury1} {llx2 lly2} {urx2 ury2}}` is illegal and not allowed. The resulting area to check for overlap can be a rectangle, line, or a point. When one object intersects multiple search areas, it will only be returned once.
`-bbox_overlap`	Uses bounding-box of the object rather than the polygon shape of the object for overlap, abut, or enclosure check. For example, an `inst` that is of L-shape, will not be returned if the `-areas` parameter overlaps only the upper-right of the bounding-box without touching any part of the L-shaped, unless the `-bbox_overlap` parameter is included.
`-d`	Specifies the `-areas` values that are in database units rather than microns. *Default*: `um`
`-enclosed_only`	Returns only those objects, which are completely enclosed inside the search area. The object polygon is fully enclosed by the search area. No part of the object polygon sticks outside the search area, although it may abut the search area edges. When multiple areas are specified, the object will be returned as long as it is enclosed in one of the search areas.
`-layers {layer1 layer2 …}`	
	Returns only those objects, which are on the specified layers. This parameter applies only to object types: `pinShape`, `pWire`, `rBlkg`, `regular`, `special`, `sViaInst`, `sWire`, `viaInst`, `wire`, `whatIfWire`, and `whatIfVia`. A `viaInst`, `sViaInst`, or `whatIfVia` object will be returned if any of the via's bottom layer, cut layer, or top layer matches with one of the specified layers. Other object types ignore this parameter and will always be returned.
`-objType objTypeList`	

	Specifies a list of object types to search for overlaps in the area. Multiple object types can be searched at the same time. The objects currently supported include: `bump busGuide inst instAllShapes instTerm pgInstTerm term marker net pBlkg pinShape pWire rBlkg regular resizeBlkg row special sViaInst sWire viaInst wire whatIfWire whatIfVia` Here: • `regular` is the union of `wire`, `pWire`, and `viaInst`. • `special` is the union of `sWire` and `sViaInst`. Objects with multiple shapes or non-rectangular shapes use a polygon to check for overlap, as described below. • `bump` has no placement overlap area, so it uses the polygon of all the OBS and PIN shapes. • `inst` uses the polygon of its placement overlap checks (for example the LEF SIZE or OVERLAP shapes), including any placement halo (if there is any) but not any routing halo. OBS shapes or PIN shapes (like standard-cell power-pins) that stick outside the placement overlap area are not included. • `instAllShapes` is the same as `inst`, except that it includes all OBS shapes and PIN shapes even if they stick outside the placement overlap area. • `instTerm` uses the polygon of all the pin-shapes of the `instTerm`. • `pgInstTerm` uses the polygon of all the pin-shapes of the `pgInstTerm`. • `pinShape` uses the polygon of all the shapes of the top-level `pinShapes`. • `net` uses the bounding-box of all the regular routing shapes on the net. Pin-shapes or special-route shapes attached to the net are not included. If you want polygon-level checking, you must ask for `wires` and `viaInsts` instead. • `term` uses the polygon of all the pin-shapes of the top-level term. • `viaInst` uses the polygon of all the shapes of the via. • `sViaInst` uses the polygon of all the shapes of the via. *Default*: `inst`
`-overlap_only`	Returns only those objects, which are partially overlapped with the search area. Some part of the object polygon overlaps the search area, whereas some of its part is outside the search area. A single point of contact is not considered as an overlap. When multiple areas are specified, the object is returned as long as it overlaps with one of the search areas.
`-polygons {x11 y11 x12 y12 x13 y13 ...}`	
	Specifies a polygon search area. By default, all the objects overlapping or abutting this area are returned on using the `-polygons` parameter, unless the `-abut_only`, `-enclosed_only`, or `-overlap_only` parameter is specified. The units are in microns (unless `-d` is specified, which indicates the database units). The format can be in either like `{x11 y11 x12 y12 x13 y13 ...}` or `{{x11 y11} {x12 y12} {x13 y13} ...}`. The polygon must be well-formed, not have zero area, and not have crossing edges.
`-strict_via_inst_layer_check`	
	Returns only those objects on the specified layer, which match with one of the via's bottom, cut, or top layers. This parameter does not return the objects of the via on the unmatched layers. It works only for `sViaInst` and `viaInst`. ⚠ This parameter works only when the `-layers` parameter is used in the background.

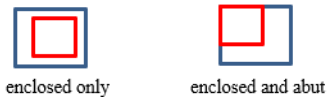
Examples

- The following command returns all the instances and routing-blockages on the `metal6` and `metal7` layers that are partially overlapped, abutted, or enclosed in the given triangle area:
  ```
  dbQuery -objType {inst rBlkg} -polygons {100 100 110 110 100 110} -layers {Metal6 Metal7}
  ```
- The following command returns all the instances and routing-blockages on the `metal6` and `metal7` layers that are partially overlapped, abutted, or enclosed in the given triangle area using the bounding-box of the objects:
  ```
  dbQuery -objType {inst rBlkg} -polygons {100 100 110 110 100 110} -layers {Metal6 Metal7}} -bbox_overlap
  ```
- The following command returns all the instances and regular wires on the `metal6` and `metal7`layers that are abutted in the give search areas:
  ```
  dbQuery -objType {inst wire} -areas {{0 0 100 100} {150 100 200 200}} -layers {Metal6 Metal7}} -abut_only
  ```
- The following command returns all the instances and regular wires on the `metal6` and `metal7`layers that are abutted in the given search areas using the bounding-box of the objects:
  ```
  dbQuery -objType {inst wire} -areas {{0 0 100 100} {150 100 200 200}} -layers {Metal6 Metal7}} -abut_only -bbox_overlap
  ```
- Following are the figures to illustrate each position relation. The instances presented by the red rectangles are returned with the specified parameter when the search area presented in the blue rectangle are specified.
  - **`-abut_only`**

  - **`-overlap_only`**

  - **`-enclosed_only`**

Note:

1. If none of the above three parameters is specified, all the instances presented by the red rectangles will be returned, by default.
2. For querying object with multi-shape (such as `instTerm` or `pinShape`) in multiple search areas, if one of its shapes is enclosed/abutted/overlapped with any one of the specified areas, the object will be returned.

dbSchema

```
dbSchema
[-help]
[objNamePattern [objAttrNamePattern]]
[-parent objNamePatternForParent [-list]]
```

Returns all of the available objects and attributes for the specified database object. It also includes the following information.

- A short description for each object and attribute.
- The type for each object.
- The legal enum values for each attribute.
- Whether the attribute value can be set (whether it is editable).

You can specify an object name, or a simple search pattern to match object names. If you specify `dbSchema`, the software returns a list of the database object names. If you specify `dbSchema *`, the software returns information for all database objects. All query information is output in the log file.

The `dbSchema` command also obeys the setDbGetMode `-displayFormat` setting to determine whether information is returned in simple or table format.

You can use the `dbSchema` command to return information on any database object listed in the Innovus Database Object Information.

Note: The information returned by the `dbSchema` command is equivalent to that returned by the dbGet command with the `.?h` operator; however, the `dbSchema` command does not require the object you query to actually exist in the database. For example, if you want to query related objects and attributes for a placement blockage, but do not have any placement blockages in the design, you can use the `dbSchema` command, but not the `dbGet` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `dbSchema` parameter. For a detailed description of the command and all its parameters, use the man command: `man dbSchema`.
`-list`	Return the list of parent objects. This option is of type boolean and is optional.
`objNamePattern`	Specifies the object for which to return the information. You can specify an object name (such as `inst`, or `term`), or a string expression to use to match object names. The software uses simple wildcard matching (`*`, `?`).
`objAttrNamePattern`	Specifies a secondary pattern for filtering object, or obejct attributes for the specified objects. This is similar to the `.?h [pattern]` filtering in the `dbGet` command.
`-parent` *`objNamePatternForParent`*	Displays the Object/Attribute for which parents need to be found from the specified object type(s). This option is of type string and is optional.

Examples

- The following command returns the available net attributes that begin with `is`:

```
dbSchema net is*

================================================================================
net: Canonical (flat) net (equivalent to connectivity in DEF NETS and
SPECIALNETS
================================================================================
isClock: bool, Indicates that net is a clock net
isExternal: bool, Indicates that net is connected to a terminal
isFixedBump: bool, Indicates that the net is connected to a bump
isGnd: bool, Indicates that net is ground
isPatternTrunk: bool, Indicates that the net is routed with a trunk pattern
isPhysOnly: bool, Indicates that the net is physical only
isPwrOrGnd: bool, Indicates that net is power or ground
isScanNet: bool, Indicates that net is a scan net
```

- The following command returns all objects that contain the attribute `pStatus`:

```
dbschema * pStatus
```

```
================================================================================
bump: Bump
----------
pStatus(settable): enum(cover fixed placed softFixed unplaced), Bump placement status
================================================================================================
inst: Instance - canonical (flat), equivalent to DEF COMPONENT. Points to a libCell or ptnCell.
================================================================================
pStatus(settable): enum(cover fixed placed softFixed unplaced), This attribute is the placement status of an instance during placement
and optimization. The placer will look at both pStatus and pStatusCTS and use the more restrictive value.
================================================
term: Terminal for libCell, ptnCell, or topCell
================================================
pStatus(settable): enum(cover fixed placed softFixed unplaced), Placement Status of terminal (not for libCell terms)
```

- The following command returns the available objects and attributes for an instance:

```
dbSchema inst
================================================================================
inst: Instance - canonical (flat), equivalent to DEF COMPONENT. Points to a libCell or ptnCell.
================================================================================
box: rect, Bounding box of the instance
cell: obj(libCell ptnCell), Pointer to child cell master or ptnCell
instTerms: objList(instTerm), List of pointers to instance terminals
isDontTouch: bool, Indicates that the instance is marked Don't Touch
isHaloBlock: bool, Indicates that the instance is a halo block
isJtagElem: bool, Indicates that the instance is a Jtag element
isPhysOnly: bool, Indicates that the instance is physical only
isSpareGate: bool, Indicates that the instance is a spare gate
name: string, Fully qualified (path) name of the instance
objType: enum(bndry densityShape fPlan group hInst hInstTerm hNet hTerm head inst instTerm layer libCell net netGroup pBlkg pd prop
ptnCell rBlkg row sWire shape term topCell vCell via wire), Object type: Instance
orient(settable): enum(MX MX90 MY MY90 R0 R180 R270 R90 Unknown), Instance placement orientation
pStatus(settable): enum(cover fixed placed unplaced), Instance placement status
pgCellTerms: objList(term), List of pointers to the instance's power terminals
pgTermNets: objList(net), List of pointers to nets attached to P/G terminals
pt(settable): pt, Location of the instance
```

- The following command returns the available objects and attributes for database objects that start with the letter p:

```
dbSchema p*
================================================
pBlkg: Placement blockage (hard, soft, partial)
================================================
attr(settable): enum(inst pushdown undefined), Indicates whether the blockage has been pushed down or is assoicated with an instance.
name(settable): string, Name of partial blockage
objType: enum(bndry densityShape fPlan group hInst hInstTerm hNet hTerm head inst instTerm layer libCell net netGroup pBlkg pd prop
ptnCell rBlkg row sWire shape term topCell vCell via wire), Object type: Placement blockage
shapes: objList(densityShape shape), List of pointers to shapes (hard/soft) or densityShapes (partial)
type(settable): enum(hard partial soft), Blockage type (hard, soft, partial)
================
pd: Power domain
================
core2Bot(settable): coord, Distance between the power domain edge and it's core box
core2Left(settable): coord, Distance between the power domain edge and it's core box
core2Right(settable): coord, Distance between the power domain edge and it's core box
core2Top(settable): coord, Distance between the power domain edge and it's core box
extBot(settable): coord, Maximum search distance for power connections
```

```
extLeft(settable): coord, Maximum search distance for power connections
extRight(settable): coord, Maximum search distance for power connections
extTop(settable): coord, Maximum search distance for power connections
gapBot(settable): coord, Minimum spacing to other power domains or rows
gapLeft(settable): coord, Minimum spacing to other power domains or rows
gapRight(settable): coord, Minimum spacing to other power domains or rows
gapTop(settable): coord, Minimum spacing to other power domains or rows
group: obj(group)
isAlwaysOn(settable): bool, Indicates that the power domain is always on
isDefault: bool, Indicates that the power domain is the default power domain
name: string, Name of the power domain
objType: enum(bndry densityShape fPlan group hInst hInstTerm hNet hTerm head inst instTerm layer libCell net netGroup pBlkg pd prop
ptnCell rBlkg row sWire shape term topCell vCell via wire), Object type: Power domain
==============
prop: Property
==============
name: string, Property name
objType: enum(bndry densityShape fPlan group hInst hInstTerm hNet hTerm head inst instTerm layer libCell net netGroup pBlkg pd prop
ptnCell rBlkg row sWire shape term topCell vCell via wire), Object type: Property
parent: obj(parent), Pointer to the parent object referencing the property
value: value, Property value (depends on valueType)
valuetype: enum(Double bits box integer loc pointer string), Type of value stored in the property (int, float, etc.)
=====================================================
ptnCell: Partition cell, created by partition command
=====================================================
name: string, Name of cell
numBidirs: int, Number of bidir terminals in the cell
numInputs: int, Number of input terminals in the cell
numPGTerms: int, Number of power/ground terminals in the cell
numRefs: int, Number of times cell is used in design
numTerms: int, Number of terminals in the cell
objType: enum(bndry densityShape fPlan group hInst hInstTerm hNet hTerm head inst instTerm layer libCell net netGroup pBlkg pd prop
ptnCell rBlkg row sWire shape term topCell vCell via wire), Object type: Partition cell
pgTerms: objList(term), List of power/ground terminals in the cell
props: objList(prop), List of pointers to properties
terms: objList(term), List of pointers to terminals in the cell
```

- The following command returns the available object and attributes for all types of blockages:

```
dbSchema *Blkg
================================================
pBlkg: Placement blockage (hard, soft, partial)
================================================
attr(settable): enum(inst pushdown undefined), Indicates whether the blockage has been pushed down or is assoicated with an instance.
name(settable): string, Name of partial blockage
objType: enum(bndry densityShape fPlan group hInst hInstTerm hNet hTerm head inst instTerm layer libCell net netGroup pBlkg pd prop
ptnCell rBlkg row sWire shape term topCell vCell via wire), Object type: Placement blockage
shapes: objList(densityShape shape), List of pointers to shapes (hard/soft) or densityShapes (partial)
type(settable): enum(hard partial soft), Blockage type (hard, soft, partial)
=======================
rBlkg: Routing blockage
=======================
attr(settable): enum(default exceptPGNet fills inst pushdown slots), Routing blockage qualifier (inst, pushdown, etc.)
layer: obj(layer), Pointer to layer of blockage
name(settable): string, Name of routing blockage
objType: enum(bndry densityShape fPlan group hInst hInstTerm hNet hTerm head inst instTerm layer libCell net netGroup pBlkg pd prop
```

```
ptnCell rBlkg row sWire shape term topCell vCell via wire), Object type: Routing blockage
shapes: objList(shape), List of pointers to shapes that define the blockage area
```

- The following commands return all available objects and attributes for a via in table format:

```
setDbGetMode -displayFormat table

dbSchema via
================================================
via: Via cell (equivalent to LEF VIA or DEF VIA)
================================================
botLayer                        | obj(layer), Pointer to bottom routing layer
botRects                        | rect, List of rectangles (typically only one) on bottom layer
cutLayer | obj(layer), Pointer to cut layer
cutRects | rect, List of rectangles on cut layer
isDefault | bool, Indicates that the via is a default via (LEF VIA DEFAULT)
isNonDefault                    | bool, Indicates that the via is declared in a LEF NONDEFAULTRULE
name | string, Via name
objType | enum(bndry densityShape fPlan group hInst hInstTerm hNet hTerm head inst instTerm layer libCell net netGroup pBlkg pd prop
ptnCell rBlkg row sWire shape term topCell vCell via wire), Object type: Via cell
topLayer | obj(layer), Pointer to top routing layer
topRects | rect, List of rectangles (typically only one) on top routing layer
```

- The following command returns clarifies that the attribute name is varied on querying it as an object or attribute:

```
dbschema fPlan pBlkg
**ERROR: (IMPDBTCL-325): No pattern of attribute 'pBlkg' found in the supported patterns. For help use 'dbSchema fPlan' or 'dbSchema
fPlan *' to get list of all supported attributes.
```

  Normally, the attribute name should be plural,as shown below:

```
dbschema fPlan pBlkgs

fPlan: Floorplan header with pointers to floorplan objects
pBlkgs: objList(pBlkg), List of pointers to placement blockages dbschema pBlkg
pBlkg: Placement blockage (hard, soft, partial, macroOnly)
area: area, Area of the placement blockage as defined by the LEF MACRO SIZE or OVERLAP information
attr(settable): enum(pushdown undefined), Indicates whether the blockage has been pushed down
boxes(settable): list(rect), List of rectangles that define the shape of placement blockage
density(settable): float, The max placement density percent allowed inside this pBlkg. It must be in the range of 5 to 100, in steps of
5. It is only valid if the type = partial or soft. For example, a partial placement percentage of 75 percent means that up to 75
percent of placement density is allowed in the area. If the type is not partial or soft, a value of 0 is returned.
inst: obj(inst), Pointer to the instance that the placement blockage is associated with (equivalent to DEF BLOCKAGES + COMPONENT)
isNoFlop(settable): bool, Indicate a partial placement blockage to prevent flops to be placed underneath.
isPushdown(settable): bool, Indicates that placement blockage has been pushed down from a higher level in the design hierarchy.
name(settable): string, Name of placement blockage
objType: enum(antennaData antennaModel bndry bump bus busGuide busSinkGroup cellDensity densityShape fPlan foreign gCellGridDef group
guiLine guiPoly guiRect guiText hInst hInstTerm hNet hTerm head inst instTerm io layer layerRule layerShape libCell marker net netGroup
pBlkg pWire pd pgInstTerm pin pinGroup pinGuide pkgComponent pkgObject prop ptn ptnCell ptnPinBlkg rBlkg resistor resizeBlkg routeType
row rule sViaInst sWire sdp shape shapeVia site stackViaRule term text topCell trackDef vCell vWire via viaInst viaRuleGenerate wire),
Object type: Placement blockage
shapes: objList(densityShape shape), List of pointers to shapes
type(settable): enum(hard macroOnly partial soft), Blockage type (hard, macroOnly, partial, soft)
```

dbSet

```
dbSet
[-d]
{objList | head | top | selected}
[.objType]*.attrName attrValue
[-help]
```

Changes the specified attribute value for a database object. The `dbSet` command takes an object or object list as a starting point, then uses a period to traverse to other related objects, or access attributes. Additional periods can be used to continue traversing the database schema.

The `dbGet` command also helps in finding out the settable objects. For example, after selecting an instance and querying its `pt*` attributes with `dbGet selected.?h pt*`, the software gives displays:

- pt(settable): pt, location of the instance
- pt_x(settable): x coordinate of location of the instance
- pt_y(settable): y coordinate of location of the instance.

The `pt`, `pt_x`, and `pt_y` attributes of an instance can also be changed using `dbSet`. In the following example, the name attribute of the selected instance cannot be changed with `dbSet`:

```
dbget selected.?h name
```

Following is the output of the above command:

```
name: string, Fully qualified (path) name of the instance
```

The `dbSet` command does not create or delete objects. You may also change the mask of the multi-color pillar via through the `dbSet` command, as shown below:

```
dbSet $via.cutMasks $mask
```

Use the `dbGet` command with the `.?h` operator, or see Innovus Database Object Information, to find out which object attributes can be set.

Parameters

`-d`	Specifies values in database units, specified in integers. *Default:* Specifies values in user units, in microns.
`-help`	Outputs a brief description that includes the type and default information for each `dbSet` parameter. For a detailed description of the command and all its parameters, use the man command: `man dbSet`.
`objList`	Specifies one or more object pointers. The list must be homogeneous.
`head`	Specifies the pointer for the root, or head of the design as the starting point (also known as `dbgHead`).
`top`	Specifies the pointer to the top cell in the design as the starting point (also known as `dbgTopCell`).
`selected`	Specifies that the selected object (or objects) is the starting point.
`.objType`	Specifies the object type.
`.attrName`	Specifies the object attribute to change.
`attrValue`	Specifies the new attribute value.

Examples

- The following command changes the placement status of the top cell instances to fixed:
  ```
  dbSet top.insts.pStatus fixed
  ```
- The following command sets all the pad instances to the placement status `fixed`:
  ```
  dbSet [dbGet top.insts.cell.baseClass pad -p2].pstatus fixed
  ```

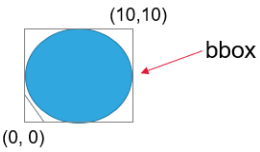
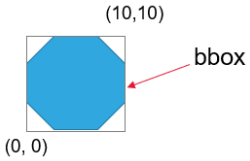
dbShape

```
dbShape
[-help]
[-d]
[-step step]
[-output polygon|rect|hrect|area]
shapeList [AND shapeList | ANDNOT shapeList | OR shapeList | XOR shapeList
| INSIDE shapeList | OUTSIDE shapeList | ENCLOSE shapeList | STRADDLE shapeList
| BBOX | HOLES | NOHOLES | BBOX_CIRCLE | BBOX_OCTAGON | MOVE {dx dy | SIZE value | SIZEX value | SIZEY value] ...
```

Performs geometric (such as SIZE and BBOX) and logical (such as AND and ANDNOT) operations on shapes (a polygon, a rectangle, or a list of rectangles and polygons).

Parameters

`-help`	Prints a brief description that includes the type and default information of each `dbShape` parameter. For a detailed description of the command and all its parameters, use the man command: `man dbShape`
`-d`	Specifies values in database units, specified in integers. All `shapeList` values as well as values for MOVE, SIZE, and step should u The output units are the same with the input units. *Default:* Specifies values in user-defined units, in microns.
`-step`	Specifies the step size if output format is rectangle, required to convert non-orthogonal shapes into a series of rectangles. *Default*: The `minwidth` value of the first routing layer. *Type:* coord, optional
`-output` *polygon|rect|hrect|area*	
	Specifies the output format. • `polygon`: Polygon shape • `rect`: Vertical rectangles **Note:** This value works only for the orthogonal shapes. • `hrect`: Horizontal rectangles **Note:** This value works only for the orthogonal shapes. • `area`: Total area of final shape *Default:* `rect` (string)
`shapeList`	Specifies a polygon or a rectangle, or a list of polygons and rectangles. The shape of the polygon or rectangle in one `shapeList` will • `polygon: {{x y} {x y} {x y} ... };` • `rect: {x1 y1 x2 y2}` *Type:* `list`
`AND` *shapeList*	Is a binary operator that is the intersection of `shapeLists`.
`ANDNOT` *shapeList*	Is a binary operator defining the initial `shapeList` minus the final `shapeList`.
`OR` *shapelist*	Is a binary operator defining the union of shapelists.
`XOR` *shapeList*	Is a binary operator of the `shapeList`: OR minus AND. It is a string and is optional.
`INSIDE` *shapeList*	Is a binary operator for the initial shapes that are inside (including touching) one of the shapes from the second list.
`OUTSIDE` *shapeList*	Is a binary operator for the initial shapes that are outside (including touching) of one of the shapes from the second list.
`ENCLOSE` *shapeList*	Is a binary operator for the initial shapes that completely enclose at least one shape from the second list.
`STRADDLE` *shapeList*	Is a binary operator for the initial shapes that are partially inside and partially outside at least one shape from the second list.
`MOVE {`*dx dy*`}`	Is a unary operator that moves the shape.

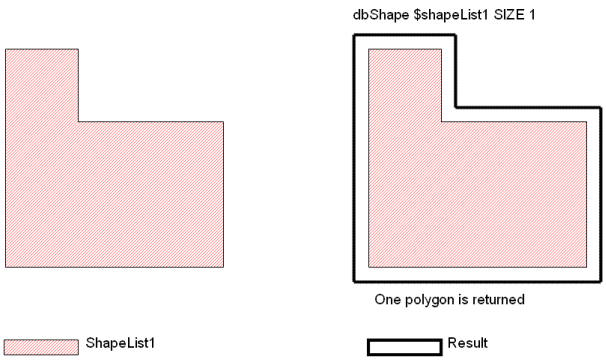
`BBOX`	Is a unary operator that computes the bounding box of the `shapeList`. This is a string and optional.
`BBOX_CIRCLE`	Is a unary operator that creates a circle shape. For example, the following command creates a circle within the specified coordinate `dbShape {0 0 10 10} BBOX_CIRCLE -output {polygon}` Following is the output of the above command: 
`BBOX_OCTAGON`	Is a unary operator that creates an octagon shape. For example, the following command creates an octagon within the specified coo `dbShape {0 0 10 10} BBOX_OCTAGON -output {polygon}` Following is the output of the above command: 
`HOLES`	Is a unary operator that gives the list of the new shapes that fill holes in the original `shapeList`.
`NOHOLES`	Is a unary operator that gives the list of the new shapes that include the filled holes in the original `shapeList`. This is same as `dbShape` *`shapeList`* `HOLES` OR *`shapeList`*
`SIZE` *`value`*	Is a unary operator that increases the size of the `shapeList` by `"value"`.
`SIZEX` *`value`*	Is a unary operator that increases the size of the shapes in the list in the X-direction by *`value`*. This parameter works only for the orth `-step` parameter to convert the shape into series of rectangles first.
`SIZEY` *`value`*	Is a unary operator that increases the size of the shapes in the list in the Y-direction by *`value`*. This parameter works only for the orth `-step` parameter to convert the shape into series of rectangles first.

Note: All the shapes processed by `dbShape` must have non-zero area. If a *shapeList* contains a list of rectangles and one of them is zero-area, it will be dropped. Also, during negative sizing or other operations, if a zero-area shape is created, it will be dropped from the output (or intermediate results).

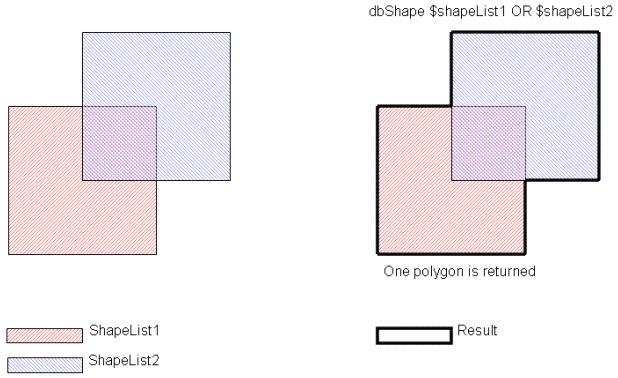
For example, a rectangle that is 4x4 or smaller will disappear when using `dbShape $rect SIZE -2 SIZE 2`, while shapes larger than 4x4 will be returned to their original state.
The same is true for protrusions with width less than 4.

Examples

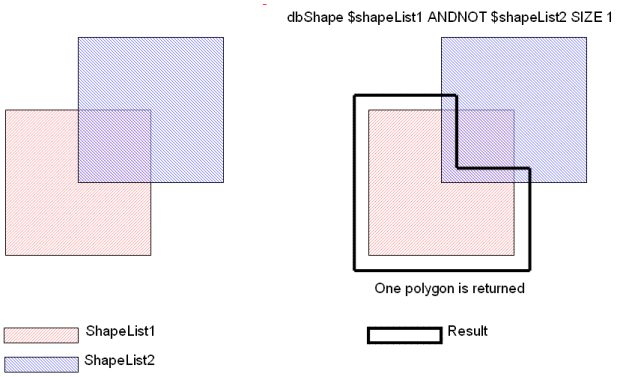
- **Unary**:



- **Binary**:



- **Combination**:



- The following command returns the intersection (as a list of rectangles) between the shapes in the `$shapes1` and the `$shapes2` lists:

  ```
  dbShape $shapes1 AND $shapes2
  ```

- The following command returns the area of the intersection between the shapes in the `$shapes1` and the `$shapes2` lists:

  ```
  dbShape -output area $shapes1 AND $shapes2
  ```

- The following commands return the merged polygon of `shapes1`. The rectangle shapes in `shapes1` will merge firstly before performing the `STRADDLE` operation, if they intersect with each other.

  ```
  set shapes1 [list {0.1 0.1 0.4 0.4} {0.2 0.2 0.6 0.6}]
  set shapes2 {0.1 0.1 0.5 0.5}
  dbShape -output polygon $shapes1 STRADDLE $shapes2
  {{0.6 0.6} {0.2 0.6} {0.2 0.4} {0.1 0.4} {0.1 0.1} {0.4 0.1} {0.4 0.2} {0.6 0.2}}
  ```

dbTransform

```
dbTransform
[-help]
[-d]{-inst instPtr | -hInst instPtr | {-cell cellPtr -orient orientEnum -pt {x y}}}
{-localPt list | -globalPt list}
```

Takes coordinates local to a cell and returns them in context with the global design space. You can specify a specific instance of a cell in the design, or you can specify a library cell and its location in the design, and instruct the software to perform a "what-if" translation of the coordinates within that cell.

Parameters

-help	Prints a brief description that includes the type and default information for each `dbTransform` parameter. For a detailed description of the command and all its parameters, use the man command: `man dbTransform`
`-cell` *cellPtr*	Specifies a pointer to a library cell. You must specify the `-pt` and `-orient` parameters with `-cell`.
`-d`	Specifies the values in database units, specified in integers. This is an optional parameter. If you use the `-d` option, the Boolean value is 1, so the value will be displayed in database units. If you do not specify the `-d` option, the Boolean value is 0, the default. *Default*: Specifies values in user units, in microns
`-globalPt` *globalPts*	Specifies the global coordinates to convert into local coordinates.
`-hInst`	Specifies the hierarchical instance of a database object. This option is mandatory.
`-inst` *instPtr*	Specifies a pointer to a specific instance/bump of a cell in the design.The software derives the coordinates and the orientation for the cell instance from the design.
`-localPt` *list*	Specifies the local coordinates to convert into global coordinates
`-orient` *orientEnum*	Specifies the orientation of the cell specified with `-cell`. You must specify the `-cell` and `-pt` parameters with `-orient`. Valid orientation values are: `R0`, `R180`, `R90`, `R270`, `MY`, `MX`, `MX90`, `MY90`.
`-pt` *x y*	Specifies the coordinates for the cell specified with `-cell`. You must specify the `-cell` and `-orient` parameters with `-pt`.

Examples

- Assume that a shape exists on `1` within a NAND2 cell, from (`1 1`) to (`2 2`). If an instance of the NAND2 cell is placed at (`10 10`), with an `R0` (N) orientation, the following command converts the local points (`1 1 2 2`) of the NAND2 cell into their global coordinates:
  ```
  dbTransform -cell [dbGet -p head.allCells.name NAND2] -pt {10 10} -orient R0 -localPt {1 1 2 2}
  ```
  The software returns the following information:
  ```
  {11 11 12 12}
  ```
- Following is an example with the same scenario as above, except that the orientation is MX instead of R0:
  ```
  dbTransform -cell [dbGet -p head.allCells.name NAND2] -pt {10 10} -orient MX -localPt {1 1 2 2}
  ```
  The software takes into account the orientation of the cell and returns:
  ```
  {11 13.04 12 14.04}
  ```
- Assume that the NAND2 cell is placed at (`10 10`), with an `R0` (N) orientation named `i1`. The following command converts the local points (`1 1 2 2`) of `i1` into their equivalent global coordinates:
  ```
  dbTransform -inst [dbGet -p top.insts.name i1] -localPt {1 1 2 2}
  ```
  The software returns the following information:
  ```
  {11 11 12 12}
  ```

dbu2uu

```
dbu2uu
[help]
value
[-unit {100 | 200 | 400 | 800 | 1000 | 2000 | 4000 | 8000 | 10000 | 20000}]
```

Takes an arbitrary list of values and returns it in the same form, with each database integer value converted to its user unit floating point equivalent.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `dbu2uu` parameter. For a detailed description of the command and all its parameters, use the man command: `man dbu2uu`
`-unit` *number*	Specifies the convert factor by which to divide the database integer value. This is an optional parameter. *Value:* One of `100`, `200`, `1000`, `2000`, `10000`, or `20000` *Default:* Uses the LEF `UNITS DATABASE MICRONS` value from the Innovus database technology file, or the OpenAccess equivalent.
{*value* | {*value_list*}}	
	Specifies the value or value list to convert. You can specify a single value, or an arbitrary list of values, including points, rectangles and coordinates. This is a mandatory parameter. *Type:* Integer

Examples

- The following command converts the database value `100` into its equivalent user unit value, based on a convert factor of `2000`:
  ```
  dbu2uu -unit 2000 100
  ```
  The software returns the following information:
  ```
  0.050
  ```
- The following command converts the database values for the list of numbers {1000 1000 2000 2000} into their equivalent user unit values, based on a convert factor of `1000`:
  ```
  dbu2uu -unit 1000 {1000 1000 2000 2000}
  ```
  The software returns the following information for the list of numbers. The list is implied, but not shown.
  ```
  1.000 1.000 2.000 2.000
  ```
- The following command converts the database value of the equivalent user unit value using the database units of the loaded database (`2000` in this example):
  ```
  dbu2uu 2
  ```
  The output of this command is:
  ```
  0.0010
  ```
  Post running the above command, the following command returns the database units of the loaded database as `2000`:
  ```
  dbget head.dbUnits
  ```

Related Commands

`uu2dbu`

decompress_special

```
decompress_special
[-help]
[-area {x1 y1 x2 y2}]
[-layers layer+]
[-nets net+]
```

Decompresses Power-Grid (PG) objects within the specified area, layer, or net. This command decompresses only the overlapped part in case of partial overlapping. The rest of the part is regrouped.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `decompress_special` parameter. For a detailed description of the command and all its parameters, use the man command: `man decompress_special`
`-area {x1 y1 x2 y2}`	Specifies the coordinates for the area from which PG needs to be decompressed. The `decompress_special` command decompresses only those special wires and special vias, which exist within the area specified by this parameter. *Default*: All PG structures are decompressed.
`-layers layer+`	Specifies the layers on which PG decompression is needed. This parameter requires either LEF layer names or a routing layer index value (for example, `1` for the first routing layer, `2` for the second routing layer, and so on).
`-nets net+`	Specifies the nets from which PG decompression is needed.

Example

The following command decompresses `%s special_wires` and `%s special_vias`:

```
decompress_special -area {100 100 120 120} -layer Via56
```

Related Topics

`compress_special`

delete_cell_obs

```
delete_cell_obs
[-help]
-cells {base_cell | design | module}+
{-obs_shapes shape+ | [{-rects {{llx1 lly1 urx1 ury1} {llx2 lly2 urx2 ury2}...} | -polygon {x1 y1 x2 y2 x3 y3 ...}] -layers layer+}
```

Helps in deleting an obstruction geometry or library obstruction for a cell on the specified layer. This command deletes the obstruction geometry added by the `add_cell_obs` command or library obstruction (`SPACING 0 OBS`).

Parameters

`-help`	Prints a brief description that includes the type and default information for each `delete_cell_obs` parameter. For a detailed description of the command and all its parameters, use the man command: `man delete_cell_obs`
`-cells {base_cell | design | module}+`	Specifies the cell name or pointer.
`-layers layer+`	Specifies the layer name or pointer.
`-obs_shapes shape+`	Specifies the cell obstruction shape pointers. **Note:** This parameter must be used with the `-cells` parameter, and cannot be specified with any other parameter except than `-help`.
`-polygon {x1 y1 x2 y2 x3 y3 ...}`	Specifies the polygon shape.
`-rects {{llx1 lly1 urx1 ury1} {llx2 lly2 urx2 ury2}...}`	Specifies the Obs rect list for deleting obstruction geometries.

Examples

Following is an example of using the `-obs_shapes` parameter for specifying the cell obstruction shape pointer:

- The `BUFX1` cell has multiple obs shapes on different layers:
  ```
  dbget [dbget top.insts.cell.name BUFX1 -p].allObstructions.shapes
  0x7fd9ee9941e0 0x7fd9ee9941c0 0x7fd9ee9941a0 0x7fd9ee994180 0x7fd9ee994160
  ```
- The following command deletes the first two shapes in the list:
  ```
  delete_cell_obs -obs_shapes {0x7fd9ee9941e0 0x7fd9ee9941c0} -cells BUFX1
  ```

Related Topics

`add_cell_obs`

deleteDanglingNet

```
 deleteDangingNet
[-help]
[-hinst string]
```

The `deleteDanglingNet` command deletes all the nets with zero terms except for assign nets and p/g nets. It supports zero-term net deletion in multiple instantiated cells.
This command takes no parameters and can be used whenever the design is read in.

After the command is executed, it reports the following:

- Total number of dangling nets `nrDnet`
- Total number of deleted dangling nets `nrDdnet`
- Total number of nets `ntNet`
- Ratio of `nrDdnet` over `nrNet`
- Ratio of `nrDnet` over `nrNet`
- `nrDdnet` over `nrDnet`

Parameters

`-help`	Prints a brief description that includes the type and default information for each `deleteDanglingNet` parameter. For a detailed description of the command and all its parameters, use the man command: `man deleteDanglingNet`
`-hinst string`	Specifies the name of hierarchical instance for allowing floating/dangling net being deleted within a specific module.

free_memory_of_compressed_objects

```
free_memory_of_compressed_objects
[-help]
```

Releases all the excess memory used by `dbGet`. When the `dbGet` command is used to query `sWire` or `sVia` under the compressed PG mode, it causes increased memory usage. This is because fake objects are created rapidly, but those cannot be released automatically. The `free_memory_of_compressed_objects` command is used to release such fake objects and to free such occupied memory for the user. Hence, the command reduces memory consumption caused by `dbGet` in the compressed PG mode.

Note: The object pointer released through this command becomes invalid and cannot be queried.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `free_memory_of_compressed_objects` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man free_memory_of_compressed_objects`

Related Information

`dbGet`

get_via_pillars

```
get_via_pillars
[-help]
{[-term term [-required]] | -instterm instterm}
```

The command returns a via pillar list on a cell pin.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `get_via_pillars` parameter. For a detailed description of the command and all its parameters, use the man command: `man get_via_pillars`
`-instterm instterm`	Specifies the `InstTerm` name or pointer.
`-required`	Returns the required flag.
`-term term`	Specifies the `Term` name (cell name/term base_name) or pointer.

Related Information

`set_via_pillars`

getDbGetMode

```
  getDbGetMode
[-help]
[-displayFormat]
[-displayLimit]
[-escapeBusChar]
[-nonDefault]
[-quiet]
```

Displays the following information about a `setDbGetMode` parameter in the Innovus log file and in the Innovus console.

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the software displays information for all of the `setDbGetMode` parameters.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `getDbGetMode` parameter. For a detailed description of the command and all its parameters, use the man command: `getDbGetMode`
`-displayFormat {simple | table}`	Determines whether to output the results of a `.??` query as a simple list, or in a table. *Default:* `simple`
`-displayLimit value`	Specifies the maximum number of objects, or object attributes to display in the output for a `.??` query. *Default:* `10`
`-escapeBusChar`	Specifies if bus characters must be escaped or not.
`-nonDefault`	Displays the current non-default settings.
`-quiet`	Displays the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software displays the current settings of all `setDbGetMode` parameters in Tcl list format.

Examples

- The following command displays the current setting of the `-displayFormat` parameter:
  ```
  getDbGetMode -displayFormat
  ```
  The software displays the following information:
  ```
  -displayFormat simple # enums={simple table}, default=simple
  simple
  ```
- The following command displays the current settings for all `setDbGetMode` parameters:
  ```
  getDbGetMode
  ```
  The software displays the following information:
  ```
  -displayFormat simple # enums={simple table}, default=simple
  -displayLimit 10 # int, default=10
  {displayFormat simple} {displayLimit 10}
  ```
- The following command displays the current setting of the `-displayFormat` parameter in Tcl list only:
  ```
  getDbGetMode -displayFormat -quiet
  ```
  The software displays the following information:
  ```
  simple
  ```
- The following command displays the current settings for all `setDbGetMode` parameters in Tcl list format only:

```
getDbGetMode -quiet
```

The software displays the following information:

```
{displayFormat simple} {displayLimit 10}
```

globalNetConnect

```
globalNetConnect
[-help]
globalNetName
{{-type pgpin -pin pinNamePattern |
-type tiehi [-pin pinNamePattern ] |
-type tielo [-pin pinNamePattern ]}
{{-singleInstance | -singleInst | -sinst} instName |
[{-instanceBasename | -instBasename} instBasenamePattern ]
[{-hierarchicalInstance | -hierInst | -module} hierInstName |
-region llx lly urx ury |
-powerDomain powerDomainName | -all]} |
-type net -net netBasenamePattern
[{-hierarchicalInstance | -hierInst | -module} hierInstName |
-powerDomain powerDomainName | -all]}
[-override]
[-verbose]
[-autoTie]
[-disconnect]
[-netlistOverride]
[-nonHierarchical]
```

The `globalNetConnect` command connects PG pins or 1'b0/1'b1 pins to the specified global net, which is either a power or ground net. You can use the command in the following three scenarios:

- Connecting pins in a single instance.
- Connecting pins in multiple instances.
- Moving connections on a net to the specified global net, which is typically higher in the design hierarchy.

Use the following command to report global nets of the complete design:

```
writeFplanScript -sections {globalNetConnect}
```

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `globalNetConnect` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man globalNetConnect`
`-all`	Applies the global net connection to all instances in the design.
`-autoTie`	Support supply/ground sensitivity. In a command for PG-pin connections, the option can only be specified together with `-type pgpin`. In addition to connecting power pin VDD to global net VDD for each instance, all the instance tie-high pins whose related power pin (as specified in CPF, .lib, or LEF) is VDD, will automatically be tied to global net VDD. For example, `globalNetConnect VDD -type pgpin -pin VDD -autoTie`
`-disconnect`	Disconnect p/g net connection. The option works only with `-sinst`, `-type pgpin` and `-pin` options. **Note:** This option is mutually exclusive with `globalNetName` argument.
`globalNetName`	Specifies the name of the global net to which the specified pins are connected.
`-hierarchicalInstance` *`hierInstName`*	
	Applies the global net connection to instances under *`hierInstName`*. Only the pins of the instances or the local nets under the specified module or hierarchical instance are connected. This parameter is mutually exclusive with the `-region` parameter. **Note:** You can also use the following spelling variations for this argument: • `-hierInst` • `-module`
`-instanceBasename` *`instBasenamePattern`*	
	Specifies the names of leaf instances for which pins are to be connected to the global net. You can use the wildcard (`*`) character to specify a pattern of instance basenames. An instance basename cannot contain the "`/`" character. This parameter is used in conjunction with the `-type pgpin` parameter, or with the `-type tiehi` or `-type tielo` parameters. **Note:** You can also use the `-instBasename` spelling variations for this argument.
`-net` *`netBaseName`*	Specifies the basenames of nets to connect to the global net. Use the wildcard (`*`) character to specify a pattern of net basenames. A net basename cannot contain the "`/`" character. This parameter is used in conjunction with the `-type net` parameter. **Note:** When this parameter is used without the `-hierarchicalInstance` parameter, the `globalNetConnect` command uses the top hinst (top Verilog module), by default.
`-netlistOverride`	Overrides previous global net connections. The option is similar to the option `-override`. However, in addition to overriding previous global net connections, `-netlistOverride` also overrides netlist-specified P/G-pin connections if applicable, which the option `-override` can't do.
`-nonHierarchical`	Connections are made only for instances in the specified hierarchical instance, and not those in its descendant hierarchical instances. The option is of type boolean and is optional.
`-override`	Specifies that the values that used with the `globalNetConnect` command override global net connection values that have been set previously.
`-pin` *`pinNamePattern`*	Specifies the pins to connect to the global net. You can use the wildcard (`*`) character to specify a pattern of pin names. This parameter is used in conjunction with the `-type pgpin` parameter, or optionally with the `-type tiehi` or `-type tielo` parameters.
`-powerDomain` *`powerDomainName`*	
	Applies the global net connection to instances within the specified power domain.
`-region` *`llx lly urx ury`*	

	Applies the global net connection to instances inside the specified region. The region is specified by the x and y coordinates of the lower left and upper right boundary. This parameter is mutually exclusive with the `-module` parameter.
`-singleInstance` *instName*	
	Connects pins in a single instance to a global net. `-singleInstance` is the instance's complete hierarchical name, without any wildcard characters. Use this parameter at the Floorplanning stage, while saving the FPlan file (or saveDesign). ⓘ The `-singleInstance` global net connect rule information is not saved in the *.fp file, but in the *.pg file. **Note:** You can also use the following spelling variations for this argument: • `-sinst` • `-singleInst`
`-type {net | pgpin | tielo | tiehi}`	
	Specifies the type of connection to be made to the global net. Use one of the following values: • `net`: Specifies that the pins connected to nets listed with the -net parameter are to be moved to the specified global net. • `pgpin`: Specifies that the power and ground pins listed with the `-pin` parameter are to be connected. • `tielo`: Specifies that tie low (`1'b0`) pins be connected. Before `tielo` net is assigned, no actual net is connected to the tie low pin. • `tiehi`: Specifies that tie high (`1'b1`) pins be connected. Before `tiehi` net is assigned, no actual net is connected to the tie high pin.
`-verbose`	Specifies that connection statistics and warning messages are displayed in the console. This parameter can be useful during troubleshooting.

Examples

- The following command connects the `vss` pin of all instances to `vss`:
  ```
  globalNetConnect VSS -type pgpin -pin VSS
  ```
- The following command moves all `vdd!` connections under `u1/u2` to `VDD`:
  ```
  globalNetConnect VDD -type net -hierInst u1/u2 -net vdd!
  ```
  Note: You can also ignore polarity checking during `globalNetConnect` and CPF/IEEE1801 operations through the global variable `init_ignore_pgpin_polarity_check` that allows USE POWER pin to be connected to a USE GROUND net (or vice versa).
- The following example shows how the power and ground nets are defined in the .globals file, and while importing the design data, the same .globals file is loaded:
  ```
  set init_pwr_net {VDD PWRD PWRA PWRIO}
  set init_gnd_net {VSS GNDD GNDA GNDIO}
  ```
- The following example presents how to remove all the power and ground net connections from the Innovus database:
  ```
  clearGlobalNets
  # For instances or standard cells where LEF pin name is vdd! for power and gnd! for ground, the net names VDD and GND should have been already defined as power and ground net.
  globalNetConnect VDD -type pgpin -pin vdd! -all
  globalNetConnect GND -type pgpin -pin gnd! -all
  # For tie pin connections to power and ground, the net names VDD and VSS should have been already defined as power and ground nets.In below case, tiehi/tielo pins are directly assigned to the VDD/VSS nets.
  globalNetConnect VDD -type tiehi
  globalNetConnect VSS -type tielo
  Note: The addTieHiLo command overrides the above direct connection.
  # For macro single instance U1/rom1 as example making power and ground pin connections to power and ground net, the net names PWRD GNDD PWRA and GNDA should have been already defined as power and ground nets.
  globalNetConnect PWRD -type pgpin -pin DIGPWR -sinst U1/rom1 -override
  globalNetConnect GNDD -type pgpin -pin DIGGRND -sinst U1/rom1 -override
  globalNetConnect GNDA -type pgpin -pin AGND -sinst U1/rom1 -override
  globalNetConnect PWRA -type pgpin -pin AVDD -sinst U1/rom1 -override
  globalNetConnect GNDA -type pgpin -pin AVSS -sinst U1/rom1 -override
  # For IO PAD single instance iopad_ring/RT5 making power and ground pin connections to power and ground net, the net names PWRD GNDD PWRIO and GNDIO should have been already defined as power and ground nets.
  globalNetConnect PWRD -type pgpin -pin VDD -sinst iopad_ring/RT5 -override
  globalNetConnect GNDD -type pgpin -pin VSS -sinst iopad_ring/RT5 -override
  globalNetConnect PWRIO -type pgpin -pin VDDIO -sinst iopad_ring/RT5 -override
  globalNetConnect GNDIO -type pgpin -pin VSSIO -sinst iopad_ring/RT5 -override
  ```
- The following example presents how to disconnect and reconnect the existing glbalNetConnection for specific instances (such as `eng/ext_mgnt/dram1_sram/s1ram/sram_100` and `eng/ext_mgmt/vpp_sram/sram/sram_070`):
  ```
  globalNetConnect -disconnect -pin VDD -type pgpin -singleInstance eng/ext_mgnt/dram1_sram/s1ram/sram_100
  globalNetConnect VDD_BLOCK -pin VDD -type pgpin -singleInstance engine/ext_mgnt/dram1_sram/s1ram/sram_100
  globalNetConnect -disconnect -pin VDD -type pgpin -singleInstance eng/ext_mgmt/vpp_sram/sram/sram_070
  globalNetConnect VDD_BLOCK -pin VDD -type pgpin -singleInstance eng/ext_mgmt/vpp_sram/sram/sram_070
  ```
- The following commands connect VDD pins of instances that match `Module_LVT/p*` to VDDL, VDD pins of instances that match `Module_HVT/r*` to VDDH, and VDD pins of instances that match `Module_RVT/*` to VDDR. (note that it is needed to specify hierarchical instance name to `-module`, and not module name):
  ```
  globalNetConnect VDDL -type pgpin -pin VDD -instBasename p* -module Module_LVT -verbose
  globalNetConnect VDDH -type pgpin -pin VDD -instinstBasename r* -module Module_HVT -verbose
  globalNetConnect VDDR -type pgpin -pin VDD -module Module_RVT -verbose
  ```

Related Topics

- `writeFplanScript`
- Power Planning and Routing chapter in the *Innovus User Guide.*

report_metal_stack

```
report_metal_stack
[-help]
[-non_preferred_direction]
```

Reports the metal stack configuration and routing track information. For each routing layer, this includes the layer name, preferred routing direction, routing offset (offset of routing track from 0), routing pitch (space between each routing track), min-width DRC rule, min-spacing DRC rule, and the routing default-width (wire width of the default routing rule). Note that for a horizontal direction track, the offset and pitch are in the Y direction (for example, the Y offset from 0, and the Y pitch of horizontal tracks). This command only supports uniform track-patterns with a single pitch value, and will not report non-uniform pitch routing track patterns that can be created with add_tracks or create_tracks.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `report_metal_stack` parameters. For a detailed description of the command and all its parameters, use: `man report_metal_stack`
`-non_preferred_direction`	By default, the `report_metal_stack` command displays the metal stack and routing information only related to the preferred routing direction. When the `-non_preferred_direction` parameter is used, the information related to the non-preferred routing direction is displayed.

Examples

- The following command displays the metal stack and routing information:
  `report_metal_stack`

  The following is the output of the above command:

```
================================================================================

Layer   Direction   Offset    Pitch      Min_Spacing  Min_Width   Default_Width

================================================================================

M1      horizontal  0.000000  0.040000   0.020000     0.020000     0.020000

M2      vertical    0.000000  0.057000   0.020000     0.037000     0.037000

M3      horizontal  0.000000  0.040000   0.020000     0.020000     0.020000

M4      vertical    0.000000  0.048000   0.020000     0.028000     0.028000

M5      horizontal  0.000000  0.080000   0.040000     0.040000     0.040000

M6      vertical    0.000000  0.076000   0.038000     0.038000     0.038000

M7      horizontal  0.000000  0.080000   0.040000     0.040000     0.040000

M8      vertical    0.000000  0.076000   0.038000     0.038000     0.038000

M9      horizontal  0.000000  0.080000   0.040000     0.040000     0.040000

M10     vertical    0.000000  0.076000   0.038000     0.038000     0.038000

M11     horizontal  0.000000  0.126000   0.064000     0.062000     0.062000

M12     vertical    0.000000  0.126000   0.064000     0.062000     0.062000

M13     horizontal  0.000000  0.252000   0.126000     0.126000     0.126000

M14     vertical    0.000000  0.252000   0.126000     0.126000     0.126000

M15     horizontal  0.000000  0.900000   0.450000     0.450000     0.450000
```

```
M16     vertical    0.000000  0.900000   0.450000     0.450000     0.450000

M17     horizontal  0.000000  3.600000   1.800000     1.800000     1.800000
```

- The following command displays the metal stack and routing information related to the non-preferred routing direction:
  `report_metal_stack -unpreferred_direction`

  The following is the output of the above command:

```
=================================================================================================

Layer  Direction     Preferred Offset    Pitch      Min_Spacing  Min_Width  Default_Width

=================================================================================================

M1      horizontal   yes       0.000000  0.040000   0.020000     0.020000     0.020000

M1      vertical     no        0.000000  0.040000   0.000000     0.000000     0.060000

M2      vertical     yes       0.000000  0.057000   0.020000     0.037000     0.037000

M2      horizontal   no        0.000000  0.057000   0.000000     0.000000     0.060000

M3      horizontal   yes       0.000000  0.040000   0.020000     0.020000     0.020000

M3      vertical     no        0.000000  0.040000   0.000000     0.000000     0.060000

M4      vertical     yes       0.000000  0.048000   0.020000     0.028000     0.028000

M4      horizontal   no        0.000000  0.048000   0.000000     0.000000     0.060000

M5      horizontal   yes       0.000000  0.080000   0.040000     0.040000     0.040000

M5      vertical     no        0.000000  0.080000   0.000000     0.080000     0.080000

M6      vertical     yes       0.000000  0.076000   0.038000     0.038000     0.038000

M6      horizontal   no        0.000000  0.076000   0.000000     0.076000     0.076000

M7      horizontal   yes       0.000000  0.080000   0.040000     0.040000     0.040000

M7      vertical     no        0.000000  0.080000   0.000000     0.080000     0.080000

M8      vertical     yes       0.000000  0.076000   0.038000     0.038000     0.038000

M8      horizontal   no        0.000000  0.076000   0.000000     0.076000     0.076000

M9      horizontal   yes       0.000000  0.080000   0.040000     0.040000     0.040000

M9      vertical     no        0.000000  0.080000   0.000000     0.080000     0.080000

M10     vertical     yes       0.000000  0.076000   0.038000     0.038000     0.038000

M10     horizontal   no        0.000000  0.076000   0.000000     0.076000     0.076000

M11     horizontal   yes       0.000000  0.126000   0.064000     0.062000     0.062000

M11     vertical     no        0.000000  0.126000   0.000000     0.000000     0.000000

M12     vertical     yes       0.000000  0.126000   0.064000     0.062000     0.062000

M12     horizontal   no        0.000000  0.126000   0.000000     0.000000     0.000000

M13     horizontal   yes       0.000000  0.252000   0.126000     0.126000     0.126000

M13     vertical     no        0.000000  0.252000   0.000000     0.000000     0.000000

M14     vertical     yes       0.000000  0.252000   0.126000     0.126000     0.126000
```

```
M14     horizontal   no        0.000000  0.252000   0.000000     0.000000     0.000000
M15     horizontal   yes       0.000000  0.900000   0.450000     0.450000     0.450000
M15     vertical     no        0.000000  0.900000   0.000000     0.000000     0.000000
M16     vertical     yes       0.000000  0.900000   0.450000     0.450000     0.450000
M16     horizontal   no        0.000000  0.900000   0.000000     0.000000     0.000000
M17     horizontal   yes       0.000000  3.600000   1.800000     1.800000     1.800000
M17     vertical     no        0.000000  3.600000   0.000000     0.000000     0.000000
```

report_net_wires

```
report_net_wires
[-help]
[-file fileName]
[-net netName]
```

Displays information about the regular net wires. If the `-net` parameter of this command is not specified, the wires of all the nets are displayed. If the `-file` parameter of this command is not specified,, the wire information on the screen (console) and log file are displayed. The information about the net regular wires is categorized as:

- Node Information
- Edge Information

Node Information

It includes XY location (like `Loc=(2.3, 52.11)`), metal layer (like `Layer=M1(2)`), number of edges on the current node (like `Edges=2`), tie special node name (like `Tie Special`), and global route node name (like `Global Route`). If the node has a terminal attached, the node information also includes the term information, like `Term = a/b/c`, `IO Pin = name`, or `PG Term = x/y/z`.

Edge Information

It includes node number (like `Node3 -> Node5`), edge type (like `Type=HV Wire`), and edge status (like `Status=Routed`). The edge status list includes `None`, `Routed`, `Fixed`, `Cover`, `Shield`, `No Shield`, `Global`, `Assigned`. For different edge types, the edge information reports different edge attributes. Following is the list of various edge types and their attributes:

- `HV Wire:`
  `NDR=name` (non-default rule name)
  `Width=3.3` (wire width)
  `Begin Extension=3.3` (wire beginning extension)
  `End Extension=3.3` (wire ending extension)
  `Mask=1` (color/mask value)
- `Via:`
  `Via Cell=name` (via cell name)
  `NDR=name` (non-default rule name)
  `Top Mask=1` (top metal layer's color/mask value)
  `Cut Mask=1` (cut layer's color/mask value)
  `Bottom Mask=1` (bottom metal layer's color/mask value)
- `Patch:`
  `Box=(100.0,200.0,223.6,250.4)` (patch box coordinates)
  `Mask=1` (patch color/mask value)
  `Attr=304` (patch attributes value)
  If it is a non-routing patch, it will also report trim metal information as shown below:
  `Trim Metal Box=(10.0, 10.0, 11.9, 11.9)` (trim metal box coordinates)
  `Mask=1` (trim metal color/mask value)
  `Layer=name` (trim metal layer name)
- `45 Degree:`
  `NDR=name` (non-default rule name)
  `Width=3.3` (wire width)
  `Mask=1` (color/mask value)
- `Zero Length`
- `Term Connect`
- `Virtual:`
  `Zero Length`, `Term Connect`, and `Virtual` have no extra attribute information. Following is an example of a single net:
  Wires for the `net1` net:
  `Node0: Loc=(44.102, 55.203), Layer=PO(2), Edges=1, Global Route`
  `Term = inst1/b`

```
Node1: Loc=(44.102, 66.304), Layer=Po(2), Edges=1
IO Pin = net1
Edge0: (Node0->Node1), Type=HV Wire, Status=Routed, NDR=ndr1, Width=0.056, Begin Extension=0.056, End Extension=0.056, Mask=1
```

Parameters

`-help`	Prints a brief description that includes the type and default information for each `report_net_wires` parameter. For a detailed description of the command and all its parameters, use the man command: `man report_net_wires`
`-file` *`fileName`*	Specifies the name of the file.
`-net` *`netName`*	Specifies the name of the net.

report_preserves

```
report_preserves
[-help]
[-computed]
[-obj_type {design | hinst | module | inst | net | hnet | pin | hpin | base_cell}]
[-dont_touch | -dont_use]
```

By default, this command displays all the preserves that affect optimization set in the library. This includes all the `.dont_touch*`, `.read_only*`, `.is_ilm`, `.dont_use*`, and `.use_cells` attributes that have a non-default value. The command does not show the computed attributes that include inheritance from their parents such as `.dont_touch_effective`, `.dont_use_cells_effective`, `dont_touch_sources`, or `.read_only_effective` to make it easy to see exactly what objects were set in the library (see `-computed` to include them).

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `report_preserves` parameter. For a detailed description of the command and all its parameters, use the man command: `man report_preserves`
`-computed`	Includes computed inheritance attributes such as `.dont_touch_effective`, `.dont_touch_sources`, `.read_only_effective`, and `.dont_use_cells_effective` attributes in the report (unless filtered by `-dont_touch` or `-dont_use`). This causes all objects to inherit a non-default value from a parent or the library to appear in the report.
`-obj_type {design | hinst | module | inst | net | hnet | pin | hpin | base_cell}`	Displays the report only on specific object types.
`-dont_touch`	Displays the report only related to `dont_touch`: `.dont_touch*`, `.read_only*`, and `is_ilm`.
`-dont_use`	Displays the report only related to `dont_use`: `.dont_use_cells*` and `.use_cells*`.

Examples

Example 1

```
dbSet [dbGet -p top.hInst.allInsts.name TDSP_CORE_INST].dontTouch sizeOk
```

Queries all hinsts

```
report_preserve -obj_type hinst -dont_touch
```

```
################################################################
# Generated by: Cadence Innovus 16.10-d085_1
# OS: Linux x86_64(Host ID vlsj-daven)
# Generated on: Thu Feb 18 15:22:28 2016
# Design: top
# Command: report_preserves
# -obj_type hinst
# -dont_touch
################################################################

hinsts
--------------------------------------------------------------------------------

hinst:TDSP_CORE_INST
dont_touch size_ok

Total of 1 hinst constrained

Query all objects

report_preserves

report_preserve -obj_type hinst -dont_touch

################################################################
# Generated by: Cadence Innovus 16.10-d085_1
# OS: Linux x86_64(Host ID vlsj-daven)
```

```
# Generated on: Thu Feb 18 15:22:28 2016
# Design: top
# Command: report_preserves
############################################################

base_cells
------------------------------------------------------------------------------

base_cell:rom_512x16A
dont_use true
base_cell:ram_256x16A
dont_use true
base_cell:pllclk
dont_use true
base_cell:DCAP4BW
dont_touch true
dont_use true

Total of 4 base_cells constrained

hinsts
------------------------------------------------------------------------------

hinst:TDSP_CORE_INST
dont_touch size_ok

Total of 1 hinst constrained

modules
------------------------------------------------------------------------------

module:tdsp_core
dont_touch size_ok

Total of 1 module constrained
```

Note: Object types with no objects constrained should not be reported and unconstrained objects should not be reported.

Example 2

```
setModuleView -partition tdsp_core -type read_only
dbSet [dbGet -p top.hInst.allInsts.allInsts.name TDSP_CORE_INST/MPY_32_INST].dontTouch sizeOk
```

Queries all hinsts and modules

```
report_preserve -obj_type hinst -dont_touch

############################################################
# Generated by: Cadence Innovus 16.10-d085_1
# OS: Linux x86_64(Host ID vlsj-daven)
# Generated on: Thu Feb 18 15:22:28 2016
# Design: top
# Command: report_preserves
# -obj_type {hinst module}
# -dont_touch
############################################################

hinsts
------------------------------------------------------------------------------
hinst:TDSP_CORE_INST
read_only true

hinst:TDSP_CORE_INST/MPY_32_INST
dont_touch size_ok

Total of 2 hinst constrained

modules
------------------------------------------------------------------------------
module:tdsp_core
read_only true

Total of 1 module constrained

report_preserve -obj_type hinst -dont_touch -computed

############################################################
# Generated by: Cadence Innovus 16.10-d085_1
# OS: Linux x86_64(Host ID vlsj-daven)
```

```
# Generated on: Thu Feb 18 15:22:28 2016
# Design: top
# Command: report_preserves -obj_type hinst -dont_touch -computed
############################################################

hinsts
------------------------------------------------------------
hinst:TDSP_CORE_INST
dont_touch_effective true
read_only_effective true
read_only true

hinst:TDSP_CORE_INST/MPY_32_INST
dont_touch_effective size_ok
dont_touch_sources {user size_ok} {parent none} {read_only_effective none}
read_only_effective true

hinst:TDSP_CORE_INST/MPY_32_INST/addinc_add_62_54_8
dont_touch_effective true
dont_touch_sources {user false} {parent size_ok} {read_only_effective true}
dont_touch size_ok
read_only_effective true
```

3 hinsts constrained.

report_via

```
report_via
[-help]
[-detail]
[-regular]
[-special]
```

Reports the single and multi-cut via summary for special and/or regular vias. Such a summary could be used for generating statistics regarding the usage of single and multi-cut used vias.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `report_via` parameter. For a detailed description of the command and all its parameters, use the man command: `man report_via`
`-detail`	Prints the detailed report related to every via.
`-regular`	Prints the report related to regular routing vias.
`-special`	Prints the report related to special routing vias.

Examples

Use the following command to display a detailed report related to every via:

```
report_via -detail
# Via-def                                  Layer(cut)     Single-cut     Total
#-------------------------------------------------------------------------------
# VIA1_20_7_20_7_VxaRECT_V                      VIA1       0            29733
# VIA1_20_7_20_7_VxaRECT_N                      VIA1       0            21322
# VIA1_20_7_20_7_VxaRECT_S                      VIA1       0            19468
# VIA1_20_7_7_20_VxaRECT_V                      VIA1       0             1266
# VIA1_0_25_0_25_VxaRECT_N49                    VIA1       0           152109
# VIA1_0_25_0_25_VxaRECT_S49                    VIA1       0           188660
...
#-------------------------------------------------------------------------------
# Total                                                                10149027
```

Use the following command to display a report about regular routing vias, including single and multi-cut vias number (ratio):

```
report_via -regular
# Layer(cut)              Single-cut               Multi-cut          Total
#-------------------------------------------------------------------------------
# VIA1              1872843 ( 74.2%)         649834 ( 25.8%)        2522677
# VIA2              1514819 ( 48.1%)        1632913 ( 51.9%)        3147732
# VIA3               914028 ( 50.4%)         900507 ( 49.6%)        1814535
# VIA4               514637 ( 47.1%)         578552 ( 52.9%)        1093189
# VIA5               203532 ( 40.2%)         302433 ( 59.8%)         505965
# VIA6                69496 ( 28.5%)         174355 ( 71.5%)         243851
# VIA7                11181 ( 13.9%)          69227 ( 86.1%)          80408
# VIA8                 4634 ( 10.1%)          41441 ( 89.9%)          46075
#-------------------------------------------------------------------------------
# Total             5105170 ( 54.0%)        4349262 ( 46.0%)        9454432
```

reportDanglingNet

```
 reportDanglingNet
[-help]
[-outfile fileName]
```

Reports the number of nets with zero terms, including the PG nets. This command can also be used to report the number of dangling nets, which could or could not be removed by the `deleteDanglingNet` command. The command reports the following information, when used:

- Total number of nets
- Number of dangling nets
- Number of removable dangling nets
- Number of non-removable dangling nets

The removable and non-removable nets are reported based on the usage of the `deleteDanglingNet` command. Those dangling nets, which could not be removed by the `deleteDanglingNet` command, are reported in the output file due to multiple possible reasons, such as the dangling net:

- The dangling net is a physical net.
- The dangling net is a PG net.
- The dangling net is a bus bit net.
- The dangling net is an assign net.
- The dangling net has multiple module ports and at least one port is associated with some constraints.
- The dangling net itself has some constraints, like an ILM module or some timing constraints.

Note: If a dangling net cannot be deleted due to any reason, only the first reason listed above will be reported in the output file.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `reportDanglingNet` parameter. For a detailed description of the command and all its parameters, use the man command: `man reportDanglingNet`
`-outfile fileName`	Reports the list of dangling nets that cannot be deleted by the `delete_floating_nets` command, along with its reasons.

Examples

- The following command reports the information about the nets listed in the `danglingnet.rpt` file:

```
reportDanglingNet -outfile danglingnet.rpt
```

- Following is the output of the above command:

```
#############################################################

#  Generated by:      Cadence Innovus 19.10-p002_1

#  OS:                Linux x86_64(Host ID sjfsb473)

#  Generated on:      Sat Aug 18 11:14:18 2018

#  Design:            designName

#  Command:           reportDanglingNet -outfile danglingNet.rpt

#############################################################
Total number of nets: 123456

The number of dangling nets: 26

Removable dangling nets: 22

Non-removable dangling nets: 4

   Is physical net: 2

   Is P/G net: 0

   Is bus net: 1

   Is assign net: 0

   Has module port:0

   Has constraints: 1
Reasons for the net not being removed:

Net <name> is physical net

Net <name> is physical net

Net <name> is bus net

Net <name> has constraints
```

reportRC

```
reportRC
[-help]
fileName
[-inst string]
[-layer string]
[-max_second_rc float]
```

Calculates the first and second RC values. The first RC value is the longest continuous vertical `rx` cut for a cell, where:

- Rx cut is always in the left or right boundaries of the cell, and at times in the internal cell.
- It is the longest vertical bar (number of rows) in which the cell's `rx` cut participate in.
- Multiple-height cells contribute the rows of height.

The second RC value is the largest distance (in microns) from the left or right boundary of the cell to the second `rx` cut in the left and right directions.

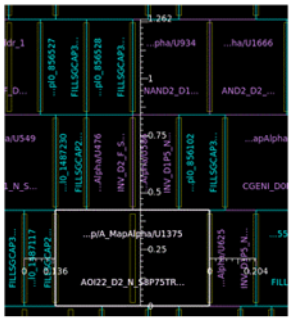
Parameters

`-help`	Prints a brief description that includes the type and default information for each `reportRC` parameter. For a detailed description of the command and all its parameters, use the man command: `man reportRC`
`fileName`	Specifies the report filename. **Note:** The report filename must be specified. Else, the software will display error message `IMPTCM-46`.
`-inst` *`string`*	Specifies the distance from inst to report.
`-layer` *`string`*	Specifies the layer of obs to calculate.
`-max_second_rc` *`float`*	Specifies the maximum distance to calculate the second RC. *Default*: `2`

Examples

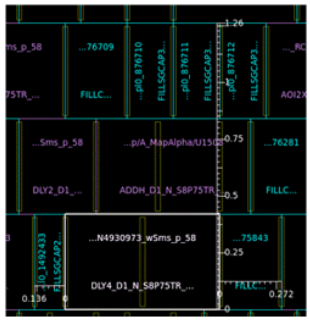
Refer to the following examples:

Example 1: STD Cell 1

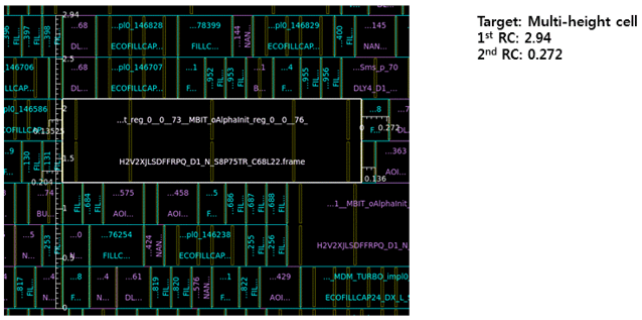


STD cell: AOI22
1st RC: 1.262
2nd RC: 0.204

Example 2: STD Cell 2



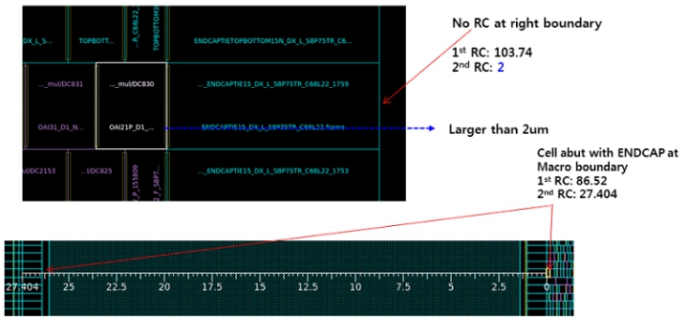
Example 3: Special Cells (Multi-height)



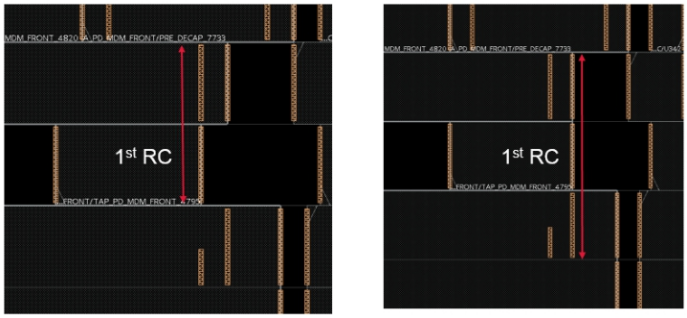
Example 4: Special Cells (ENDCAP)

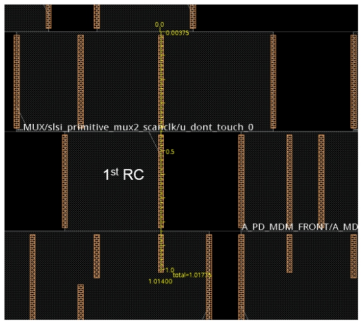
Note that:

- An ENDCAP cell has a single RC.
- The cell with ENDCAP must have a long second RC.
- The second RC effect is saturated at 2um. It means that the second RC value for the cell is 2 um.

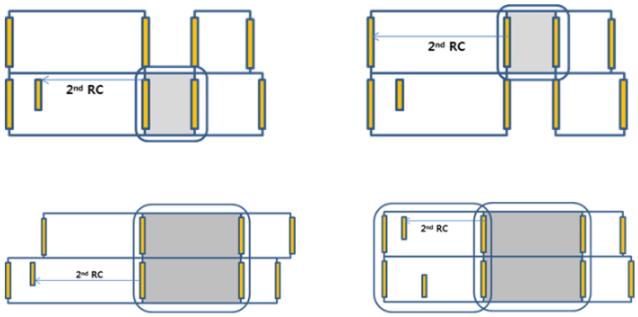


Example 5: First RC Values in Various Scenarios





Example 6: Second RC Values in Various Scenarios



Example 7: Input/Output Format of First and Second RC Generation

```
derive_rxcuts {
    -2nd_rc_saturation_length 2nd RC saturation length
    -output OUTPUT_FILE

}

innovus_shell> derive_rxcuts -2nd_rc_saturation_length  2.0 –output  ./rxcut.info.txt

dervie_rxcuts output format: (full_name  ref_name location  orientation  unit_cell_height  1st_RC 2nd_RC)

u_ee_toplevel/u_msg_u_rx_path/u_rx_conv_u_load_convert_top_LOAD_CONVERTER_LANES_1__u_load_convert_lane/DC2553
ln10lpp_sc_s8p75t_flk_rvt_c68l20_25c_sadhm_lvf_d:NAND2_D1_F_S8P75TR_C68L20:frame 65.09 392.28 MX 0.42 86.52 27.404
                                                                                  1st RC    2nd RC
```

reportWirePath

```
reportWirePath
[-help]
[-detail]
-end string
[-include_pin]
-start string
[-tcl [-no_output]]
```

Extracts the routing topology report in a point-to-point manner (for example between routing layer, wire length, via, resistor, and capacitance). The report contains the information such as timing path for each via, wire length, and width of path.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `reportWirePath` parameter. For a detailed description of the command and all its parameters, use the man command: `man reportWirePath`
`-detail`	Prints the detailed information related to the path.
`-end string`	Specifies the end terminal name, which can be an `instTerm`, `term`, `hinstTerm`, and `hTerm`.
`-include_pin`	Specifies whether to include pin in the report.
`-no_output`	Specifies whether to print the path when `-tcl` is used.
`-start string`	Specifies the start terminal name, which can be an `instTerm`, `term`, `hinstTerm`, and `hTerm`.
`-tcl`	Specifies whether to return the Tcl dict of the wire path.

Examples

Use the following command to get the routing topology report from `inst1/Z` to `inst2/A`:

```
reportWirePath -start inst1/Z -end inst2/A Start pt: 15.075, 6.72 Via NR_VIA1_PH Wire M2, Width: 0.032, Length: 0.01 Via NR_VIA2_PV Wire M3, Width: 0.032, Length: 0.832 Via NR_VIA2_PV Wire M2, Width: 0.032, Length: 0.35 Via NR_VIA1_PH End pt: 15.435, 5.888 Total wire length: 1.192 Manhattan length: 1.192
```

set_via_pillars

```
set_via_pillars
[-help]
{-term term | -instTerm instTerm}
stack_via_rule+
[-required {0 | 1 | 2 | 3 | 4}]
```

This command sets a via pillar list for a cell term or a single via pillar for an `instTerm`.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `set_via_pillars` parameter. For a detailed description of the command and all its parameters, use the man command: `man set_via_pillars`
`-instTerm instTerm`	Specifies the `instTerm` name or pointer. Only one via pillar is allowed for `instTerm`.

`stack_via_rule+`	Provides a list of via pillars for a cell term, or a single via pillar for a `instTerm`. This list further provides the candidates of `instTerm` via pillar setting. On running:
	`set_via_pillars -term BUFX1/Z -required 1 {vp_rule1 vp_rule2}`
	`set_via_pillars` automatically sets the minimal cost via pillar rule for the corresponding `instTerm`, which is `vp_rule1`. You can also set the other via pillar rule to the `instTerm` by running:
	`set_via_pillars -instTerm i1/i2/Y vp_rule2`
	An empty *`stack_via_rule`* will remove the via pillar from `instTerm`:
	`set_via_pillars -instTerm i1/i2/Y -required 1 ""`

`-required {0 | 1 | 2 | 3 | 4}`	Sets the `.stackViaRuleRequied` attribute for the *`term`*/*`instTerm`* values specified by the `-term` or `-instTerm` parameters. Setting the `.stackViaRuleRequied` attribute determines whether the stack vias are required for the specified term. • When set to `0` for a term, this parameter sets the `.stackViaRuleRequied` attribute to `0`, which ignores this `term` for `stackViaRule` checking. If the parameter is set to `0` only for an `instTerm`, other `instTerm` values instantiated from the same `term` will not be affected. • When set to `1` for a `term`, this parameter sets the `.stackViaRuleRequied` attribute to `1` on the `term` values. When set to `1` for an `instTerm`, it enforces the via pillar only on the specified `instTerm`. However, if the via pillar list is empty for an `instTerm`, no via pillar is used even when the required bit is `1`. If set to `1`, the software performs min via pillar insertion with dirty mode, in which the via pillars are added even if there is DRC. • When set to `2` for a `term`, this parameter sets the `.stackViaRuleRequied` attribute to `2` on the `term` values. If set to `2`, the software performs min via pillar insertion with clean mode, in which the via pillars are not added if there is DRC. • When set to `3` for a `term`, this parameter sets the `.stackViaRuleRequied` attribute to `3` on the `term` values. If set to `3`, the software performs multiple tries from large to min via pillar insertion with dirty mode, in which the via pillars are added even if there is DRC. • When set to `4` for a `term`, this parameter sets the `.stackViaRuleRequied` attribute to `4` on the `term` values. If set to `4`, the software performs multiple tries from large to min via pillar insertion with clean mode, in which the via pillars are not added if there is DRC. ⚠ The `2`, `3`, and `4` values of this parameter are applicable only for a `term`.
`-term term`	Specifies the `Term` name (cell name/term base_name) or pointer. A via pillar list is allowed for `term`.

Examples

- Where the cell of `inst i1/i2` and `i1/i3` are both `BUFX1`, the following commands indicate that `instTerm i1/i2/Z` requires a via pillar `M2M4_1X3`:
```
set_via_pillars -term BUFX1/Z -required 1 {M2M4_1X2 M2M4_1X3}
set_via_pillars -instTerm i1/i2/Z -required 1 M2M4_1X3   #The via pillar for instTerm must be on the cell term's via pillar list. Otherwise, a proper ERROR should be returned.
```
- The following command removes the via pillar `M2M4_1X3` from `instTerm i1/i2/Z`:
```
set_via_pillars -instTerm i1/i2/Z "" -required 1
```
- The following command sets a via pillar list on a cell pin:
```
set_via_pillars -term BUFX1/Z -required 1 {M2M4_1X2 M2M4_1X3}
```
- The following command sets a via pillar with via pillar list of the corresponding cell pin on a `instTerm`; the cell of inst `i1/i2` is `BUFX1`:
```
set_via_pillars -instTerm i1/i2/Z M2M4_1X3
```
- The following command sets via pillar to cell pin `AN2_NOM_D16*/Z`:
```
foreach cell [dbGet head.libCells.name -e AN2_NOM_D16*] {set_via_pillars -term $cell/Z -required 0 {VPPERF_M2_M4_1_2_1 VPPERF_M2_M5_1_2_2_1 VPPERF_M2_M9_1_2_2_2_2_2_2_1} }
```

Related Information

`get_via_pillars`

setDbGetMode

```
setDbGetMode
[-help]
[-reset]
[-displayFormat {simple | table}]
[-displayLimit value]
[-escapeBusChar {true | false}]
```

Controls certain aspects of how the dbGet command displays object information.

Use the getDbGetMode command to display the current settings for the `setDbGetMode` command.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `setDbGetMode` parameter. For a detailed description of the command and all its parameters, use the man command: `setDbGetMode`
`-displayFormat {simple | table}`	
	Determines whether to output the results of a `.??` query as a simple list, or in a table. *Default:* `simple`
`-displayLimit value`	Specifies the maximum number of objects, or object attributes to display in the output for a `.??` query. *Default:* `10`
`-escapeBusChar {true | false}`	
	Indicates whether bus characters will be escaped using {} or not. Use the option to avoid braces when command displays object information. *Default*: `true` (will escape names containing bus characters)
`-reset`	Resets parameters to their default values. The `-reset` parameter *must* be the first parameter specified. If you specify `-reset` by itself, the software resets all `setDbGetMode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset, will be reported.

Examples

- The following command instructs the `dbGet` command to output the results of a `.??` query as a table.

  ```
  setDbGetMode -displayformat table
  ```

  Query results are displayed in the following format:

  ```
  attribute/object | value
  --------------------------------------------------------------------------------
  box | {0 0 1.32 5.04}
  cell | 0x121ede48
  instTerms | 0x13450290 0x134502c4
  isDontTouch | 0
  isHaloBlock | 0
  isJtagElem | 0
  isPhysOnly | 0
  isSpareGate | 0
  name | IOPADS_INST/PKS_Opt_2126
  objType | inst
  orient | R0
  pStatus | unplaced
  pgCellTerms | 0x13a3db98 0x13a3dbdc
  pgTermNets | 0x131aa5c8 0x131aa688
  pt | {0 0}
  ```

- The following command resets the `-displayformat` parameter to its default value:

  ```
  setDbGetMode -reset -displayFormat
  ```

  Query results are displayed in a simple list format:

  ```
  box: {0 0 1.32 5.04}
  cell: 0x121ede48
  instTerms: 0x13450290 0x134502c4
  isDontTouch: 0
  isHaloBlock: 0
  isJtagElem: 0
  isPhysOnly: 0
  isSpareGate: 0
  name: IOPADS_INST/PKS_Opt_2126
  objType : inst
  orient : R0
  pStatus : unplaced
  pgCellTerms: 0x13a3db98 0x13a3dbdc
  pgTermNets: 0x131aa5c8 0x131aa688
  pt: {0 0}
  box: {0 0 1.32 5.04}
  cell: 0x121ede48
  instTerms: 0x134502f8 0x1345032c
  isDontTouch: 0
  isHaloBlock: 0
  isJtagElem: 0
  isPhysOnly: 0
  isSpareGate: 0
  name: IOPADS_INST/PKS_Opt_2125
  objType : inst
  orient : R0
  pStatus : unplaced
  ```

```
pgCellTerms: 0x13a3db98 0x13a3dbdc
pgTermNets: 0x131aa5c8 0x131aa688
pt: {0 0}
```

- The following command resets all `setDbGetMode` parameters to their default values:
  ```
  setDbGetMode -reset
  ```
- The `-escapeBusChar` option can be used as follows:
  ```
  Behaviour with -escapeBusChar option = true (Default):
  dbGet top.nets.name
  net1 {busnet[10]} ...
  Behaviour with -escapeBusChar option = false :
  dbGet top.nets.name
  net1 busnet[10] ...
  ```
- The `-displayLimit` can be used as follows:
  Default behavior (displayLimit is set to `10`) returns maximum of 10 pointers:
  ```
  dbget top.?? nets nets
  0x2ad51a5f4090 0x2ad51a5f40d8 0x2ad51a5f4120 0x2ad51a5f4168 0x2ad51a5f41b0 0x2ad51a5f41f8 0x2ad51a5f4240 0x2ad51a5f4288 0x2ad51a5f42d0
  0x2ad51a5f4318 ... (total length 6467)
  setDbGetMode -displayLimit 2
  ```
  The above command returns maximum of 2 pointers.
  ```
  dbget top.?? nets
  nets: 0x2ad51a5f4090 0x2ad51a5f40d8 ... (total length 6467)
  ```

Related Commands

`getDbGetMode`

uniquify

```
uniquify
[-help]
{[design | module] | -new_master hinsts} [-exclude modules]
[-verbose]
```

Supports regrouping of master-clone partitions inside the non-unique modules. This command uniquifies or regroups the master/clones (one master module used by two or more `hinst` clones) by creating unique modules for each `hinst` clone, as needed. In all the cases, the new module names are generated by appending the '`_number`' suffix (such as `_1` and `_2`) sequentially to the original module name for creating a unique module name.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `uniquify` parameter. For a detailed description of the command and all its parameters, use the man command: `man uniquify`
`design` `|` *`module`*	Regroups the master/clones within the *`design`* or *`module`* and recursively below it. This assures that all the `hinsts` inside those would have their own unique module, except for the modules specified using the `-exclude` parameter. In case multiple `hinsts` are referenced to the specified `module`, those will remain as clones. *Type:* Mandatory
`-new_master` *`hinsts`*	Splits the `hinst` clones of a master module into the clones of a new master. The new master is also regrouped, except for the modules specified using the `-exclude` parameter. It may also happen that one of such `hinst` clones is forced to have a new master (refer to the **Examples** section). In case a `hinst` is not a clone of the same module, an error message is displayed. *Type:* Mandatory
`-exclude` *`modules`*	Excludes the specified modules from unification or regrouping. Use this parameter if a specific sub-module is a partition, which will be implemented and shared only once. *Type:* Optional
`[-verbose]`	Reports the `hinst` clones (if any), which are regrouped. *Type:* Optional

Examples

- The following command uniquifies the whole design, where the `h1`, `h2`, and `h3 hinsts` are clones of the `mod1` module, and the `h4` and `h5 hinsts` (clones of the `mod2` module) are inside `mod1`:
  ```
  uniquify top -verbose
  #hinst name module name
  h1 mod1 (unchanged)
  h2 mod1_1
  h3 mod1_2
  h1/h4 mod2 (unchanged)
  h1/h5 mod2_1
  h2/h4 mod2_2
  h2/h5 mod2_3
  h3/h4 mod2_4
  h3/h5 mod2_5
  ```
  Note that the first clone of each module is not modified. So, `h1` still uses `mod1`, and `h1`/`h4` still use `mod2`. However, these are still reported to see the `hinst` clone, which forces a new module for the other `hinst` clones.
- Using the same scenario as in the previous example, the following command uniquifes the design below `mod1`:
  ```
  uniquify mod1 -verbose
  #hinst name module name
  h1/h4 mod2 (unchanged)
  h1/h5 mod2_1
  h2/h4 mod2 (unchanged)
  h2/h5 mod2_1
  h3/h4 mod2 (unchanged)
  h3/h5 mod2_1
  ```
  Note that only the design inside `mod1` is uniquified. So, `h1`, `h2`, and `h3` are still the clones of `mod1`, and there are still master/clones grouping between `h1`, `h2`, and `h3`. For instance, all the `h4`s use `mod2`, and all the `h5`s use `mod2_1`.
- Using the same scenario as in the first example, the following command splits `h1` and `h2` into a new master. This is normally done, if `h1`, `h2`, and `h3` are partitions and need a different implementation than `h3`. It is recommended not to uniquify `mod2`, if it is a lower-level partition, which will be implemented once and shared by all `h1`, `h2`, and `h3`.
  ```
  uniquify -new_master {h1 h2} -exclude mod2 -verbose
  #hinst name module name
  h1 mod1_1
  h2 mod1_1
  ```
  Note that `h3` is still using `mod1`, and the `mod2 hinsts` inside `mod1` and `mod1_1` (such as `h4` and `h5`) are still the clones of `mod2`. If there is also a `mod3` inside `mod1`, it will also be uniquified, but nothing inside `mod2` is uniquified.
- Using the same scenario as in the first example, the following command splits `h1` and `h4` into a new master. This implicitly forces `h1` to be uniquified from `h2` and `h3`.
  ```
  uniquify -new_master {h1/h4} -verbose
  #hinst name module name
  h1 mod1_1 (forced by lower level change)
  h1/h4 mod2_1
  ```
  If there is a `mod4` inside `mod2_1`, it will also be uniquified, but it will not be shared with `mod2`.

uu2dbu

```
uu2dbu
[help]
value
[-unit {100 | 200 | 1000 | 2000 | 10000 | 20000}]
[-unit number]
{value | {value_list}}
```

Takes an arbitrary list of values and returns it in the same form, with each user unit floating point value converted to its database integer value equivalent.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `uu2dbu` parameter. For a detailed description of the command and all its parameters, use the man command: `man uu2dbu`
`-unit number`	Specifies the convert factor by which to multiply the user unit value. This is an optional parameter. *Value:* One of `100`, `200`, `1000`, `2000`, `10000`, or `20000` Default: Uses the LEF `UNITS DATABASE MICRONS` value from the Innovus database technology file, or the OpenAccess equivalent.
`{value | {value_list}}`	
	Specifies the value or value list to convert. You can specify a single value, or an arbitrary list of values, including points, rectangles and coordinates. This is a mandatory parameter. *Type:* Float

Examples

- The following command converts the user unit value `30` into its equivalent database integer value, based on a convert factor of `2000`:
  ```
  uu2dbu -unit 2000 30
  ```
  The software returns the following information:
  ```
  60000
  ```
- The following command converts the user unit values for the list of numbers {`30 40`} into their equivalent database unit values, based on a convert factor of `200`:
  ```
  uu2dbu -unit 200 {30 40}
  ```
  The software returns the following information for the list of numbers. The list is implied, but not shown.
  ```
  6000 8000
  ```
- The following command converts the value to the database unit value using the database unit of the loaded database:
  ```
  uu2dbu 0.5
  ```
  The output of this command is:
  ```
  1000
  ```
  Post running the above command, the following command returns the database unit value of the loaded database as `2000`:
  ```
  dbget head.dbUnits
  ```
  Note that if the value cannot be converted to an integer database value, an error message will be displayed, as shown below:
  ```
  uu2dbu -unit 2000 0.000005
  **ERROR: (IMPSYC-1295): Cannot convert to integer database unit.
  ```

Related Commands

`dbu2uu`

Basic Database Access Tcl Global Variables

- restore_db_version
- layerNameNoAbbreviation
- mustjoinallports_is_one_pin

restore_db_version

```
restore_db_version string
```

Type: String

Default: `""`

Restores the tool database version that generates the current design. This command was renamed from `dbRestoreDbVersion`.

layerNameNoAbbreviation

```
layerNameNoAbbreviation {0 | 1}
```

Type: Boolean

Default:

- `1` for N7 designs
- `0` for rest all designs

By default `dbGet layer.name` and most command `-layer layer` options allow abbreviations such as M1 and M2, for routing layers such as 1 and 2. If this global variable is set to `1`, `dbGet layer.name` returns the full layer names rather than an abbreviation, and the M1 and M2 abbreviations are not allowed for `-layer` input.

This global is intended to be set when a technology has real layer names like M1, M2, that do not correspond to routing layers 1, 2. Setting this global to 1 will disable the M<num> abbreviations that may cause confusion, and only allow full layer names, or routing layer numbers.

Example

```
dbGet head.layers.extName                  //.extName is full layer name
Metal1 Metal2 Metal3 Metal4 Metal5 Metal6 Via12 Via23 Via34 Via45 Via56 OVERLAP

dbGet head.layers.name                     //.name is abbreviated layer name
M1 M2 M3 M4 M5 M6 Via12 Via23 Via34 Via45 Via56 OVERLAP

add_shape -rect {0 0 5 5} -layer Metal1  //full name works

add_shape -rect {0 0 5 5} -layer M1     //abbreviated name work

add_shape -rect {0 0 5 5} -layer 1      //routing number works

set layerNameNoAbbreviation 1 //set layer name returned with full layer name instead of abbreviation
1

dbGet head.layers.name                     //.name is now full layer name
Metal1 Metal2 Metal3 Metal4 Metal5 Metal6 Via12 Via23 Via34 Via45 Via56 OVERLAP

add_shape -rect {0 0 5 5} -layer Metal1   //full name works

add_shape -rect {0 0 5 5} -layer M1      //abbreviated name fails

**ERROR: (IMPSPR-162): Incorrect usage for command 'add_shape'. Invalid layer name: M1

add_shape -rect {0 0 5 5} -layer 1       //routing number works
```

mustjoinallports_is_one_pin

```
mustjoinallports_is_one_pin {true | false}
```

Type: Boolean

Default: `true`

If `false`, the LEF MUSTJOINALLPORTS property on a LEF PIN automatically creates the MUSTJOIN pins for each PORT. If `true`, the MUSTJOIN pins are not created, but the router must be connected to every port with a single wire or single-via that physically overlaps all the ports.

4

Bus Plan Commands

- createBusGuide
- deleteBusGuide
- deselectBusGuide
- resetBusGuideMultiColors
- selectBusGuide
- selectBusGuideSegment
- setBusGuideMultiColors
- update_bus_guide

createBusGuide

```
createBusGuide
[-help]
[-type {hard | soft}]
-netGroup netGrpName
-layer {id | id1:id2}
{[{-centerLine {x1 y1 x2 y2} -width value} [-beginExt value] [-endExt value]] | -rect
{x1 y1 x2 y2}}
```

Creates bus guides associated with net groups, to guide routing for all the nets of the specified net group. You can create horizontal, vertical, and 45-degree bus guide segments. 45-degree bus guides are required for flip chip routing.

Use this command after defining a net group that will be associated with the new bus guide segment.

After a bus guide is created, you can modify its layer or width later by using the `update_bus_guide` command.

Parameters

`-help`	Prints a brief description that includes type and default information for each `createBusGuide` parameter. For a detailed description of the command and all of its parameters, use the man command: `man createBusGuide`.
`-beginExt` *value*	Specifies the value, in microns, by which the bus guide segment is to be extended in the beginning.
`-centerLine` *x1 y1 x2 y2*	
	Specifies the coordinates of the center line of the bus guide segment. The direction of the guide segment is derived from the `x1 y1 x2 y2` coordinate values. **Note:** Non-orthogonal bus guide segments are not supported.
`-endExt` *value*	Specifies the value, in microns, by which the bus guide segment is to be extended in the end.
`-layer {`*id* `|` *id1*`:`*id2*`}`	Specifies the routing layer constraint honored by the router. ⓘ Routing done on non-preferred routing layers or on layers which are not part of the specified layer constraint can have high cost implications
`-netGroup` *netGrpName*	Specifies the name of the net group that will be associated with the new bus guide segment that you are creating.
`-rect {`*x1 y1 x2 y2*`}`	Specifies the lower-left and upper-right coordinates of the rectangle formed by the bus guide segment.

`-type {hard | soft}`	Specifies the constraint type of the bus guide: • `hard` - After running Early Global Route (eGR), NanoRoute (NR), or NanoRoute High Frequency Router (NRHF), the route obeys the path of the bus guide. • `soft` - After running eGR, NR, or NRHF, the bus guide only guides the route path. The tool can route the net out of the bus guide. *Default*: `soft` (If you don't specify the `-type` parameter to define the constraint type, the value of constraint type is taken as `soft`.)
`-width` *value*	Specifies the width, in microns, of the bus guide segment that you are creating.

Examples

- The following command creates a horizontal bus guide segment whose width is `120` microns, and layer constraint is from METAL4 to METAL8:
  ```
  createBusGuide -netGroup netGroup1 -centerLine 4421.8 10749.36 4960.8 10749.36 -width 120 -layer METAL4:METAL8
  ```
- The following command creates a bus guide segment whose lower-left coordinate is at pt {`464 1240`} and upper-right coordinate is at pt {`528 1260`} for the net group `test` and with a layer constraint of Metal1 to Metal4:
  ```
  createBusGuide -netGroup test -layer {1:4} -rect {464 1240 528 1260}
  ```
- The following command creates a 45-degree bus guide segment:
  ```
  createBusGuide -netGroup test3Group -centerLine 400 500 450 550 -width 20 -layer {METAL7:METAL8}
  ```

Related Information

- `createNetGroup`
- `deleteBusGuide`
- `selectBusGuide`
- `update_bus_guide`
- Bus Planning chapter in the *User Guide*
  - *Creating a Bus Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Bus Guide*

deleteBusGuide

```
deleteBusGuide
[-help]
[-area x1 y1 x2 y2]
[-netGroup {netGroup | {list_of_net_groups}}]
[-layer {id | id1:id2}]
[-direction {H | V}]
[-all]
```

Deletes a bus guide.

If you do not specify any option, by default, the command delete all the bus guide segments.

Use this command after creating a bus guide segment.

Parameters

`-all`	Deletes all bus guide segments in the design.
`-area` *`x1y1x2y2`*	Specifies the coordinates of the area from which the bus guide segments are deleted.
`-direction {H | V}`	Specifies the direction of the bus guide segment to delete. This can be `H` (horizontal) or `V` (vertical).
`-help`	Outputs a brief description that includes type and default information for each `deleteBusGuide` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deleteBusGuide`.
`-layer {`*`id`* `|` *`id1`*`:`*`id2`*`}`	Specifies the layer or layer range from which bus guide segments are deleted. The command deletes bus guide segments with the exact specified layer information and not a subset of it. For example, for a bus guide segment with layer constraint `2:5` if you specify `-layer 5`, the segment is not deleted. To delete this guide segment you need to specify `-layer 2:5` completely.
`-netGroup {`*`netGroup`*`|` `{`*`list_of_net_groups`*`}}`	Specifies the name of the net group or list of net groups from which the bus guide segments are deleted. You can also use wildcard (* or ?) for specifying the net group name.

Example

The following command deletes all the bus guide segments associated with the net group `abcBus` that has layer constraint from `Metal 3` to `Metal 7`:

```
deleteBusGuide -netGroup abcBus -layer 3:7
```

Related Information

- `createBusGuide`
- `update_bus_guide`
- Bus Planning chapter in the *User Guide*
  - *Creating a Bus Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Bus Guide*

deselectBusGuide

```
deselectBusGuide
[-help]
[-area x1 y1 x2 y2]
[-netGroup {netGroup | {list_of_net_groups}}]
[-layer {id | id1:id2}]
[-direction {H | V}]
[-all]
```

Deselects a bus guide segment.

If you do not specify any option, by default, the command deselects all the bus guide segments.

Use this command after selecting a bus guide segment.

Parameters

`-all`	Deselects all the bus guide segment in the design.
`-area` *`x1y1x2y2`*	Specifies the coordinates of the area from which the bus guide segments are deselected.
`-direction {H | V}`	Specifies the direction of the bus guide segment to deselect. This can be `H` (horizontal) or `V` (vertical).
`-help`	Outputs a brief description that includes type and default information for each `deselectBusGuide` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deselectBusGuide`.
`-layer {`*`id`* `|` *`id1`*`:`*`id2`*`}`	Specifies the layer or layer range from which bus guide segments are deselected. The command deselects bus guide segments with the exact specified layer information and not a subset of it. For example, for a bus guide segment with layer constraint `2:5` if you specify `-layer 5`, the segment is not deselected. To deselect this guide segment you need to specify `-layer 2:5` completely.
`-netGroup {`*`netGroup`*`|` `{`*`list_of_net_groups`*`}}`	Specifies the name of the net group or list of net groups from which the bus guide segments are deselected. You can also use wildcard (* or ?) for specifying the net group name.

Example

The following command deselects all the bus guide segments associated with the net group `abcBus` that has layer constraint from metal `3` to metal `7`:

```
deselectBusGuide -netGroup abcBus -layer 3:7
```

Related Information

- `selectBusGuide`
- `selectBusGuideSegment`
- Bus Planning chapter in the *User Guide*
  - *Creating a Bus Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Bus Guide*

resetBusGuideMultiColors

`resetBusGuideMultiColors`

Clears the highlighted bus guide colors.

Use this command after `setBusGuideMultiColors` has been used to color the bus guide(s).

Parameters

None

Related Information

- `setBusGuideMultiColors`
- Bus Planning chapter in the *User Guide*
  - *Highlighting and Dehighlighting the Bus Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Bus Guide*

selectBusGuide

```
selectBusGuide
[-help]
[-all]
[-area {x1 y1 x2 y2} ]
[-direction {H | V}]
[-layer { id | id1 : id2 }]
[-netGroup { netGroup | { list_of_net_groups }}]
```

Selects a bus guide segment.

Note: If you do not specify any option, by default, the command selects all the bus guide segments.

Use this command after creating a bus guide segment.

Parameters

`-all`	Selects all the bus guide segments in the design.
`-area {x1 y1 x2 y2}`	Specifies the coordinates of the area from which the bus guide segments are selected.
`-direction {H | V}`	Specifies the direction of the bus guide segment to select. This can be `H` (horizontal) or `V` (vertical).
`-help`	Outputs a brief description that includes type and default information for each `selectBusGuide` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man selectBusGuide`.
`-layer {id | id1: id2}`	Specifies the metal layer or layer range from which bus guide segments are selected. The command selects bus guide segments with the exact specified layer information and not a subset of it. For example, for a bus guide segment with layer constraint `2:5` if you specify `-layer 5`, the segment is not selected by the command. To select this guide segment you need to specify `-layer 2:5` completely.
`-netGroup {netGroup | {list_of_net_groups}}`	
	Specifies the name of the net group or list of net groups from which the bus guide segments are selected. You can also use wildcard (* or ?) for specifying the net group name.

Example

The following command selects all the horizontal (`H`) bus guide segments associated with the net group `group1`:

```
selectBusGuide -netGroup group1 -direction H
```

Related Information

- `createBusGuide`
- `deleteBusGuide`
- `deselectBusGuide`
- `selectBusGuideSegment`
- `update_bus_guide`
- Bus Planning chapter in the *User Guide*
  - *Creating a Bus Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Bus Guide*

selectBusGuideSegment

```
selectBusGuideSegment
[-help]
[-box x1 y1 x2 y2]
[-layer {id | id1:id2}]
[-netGroup {netGroup}]
```

Selects a bus guide segment with its specified bounding box. This command is mainly used for recording the bus guide segment selection operation in the software's command file.

Use this command after creating a bus guide segment.

Parameters

`-box x1y1x2y2`	Specifies the coordinates of the bus guide box to select.
`-help`	Outputs a brief description that includes type and default information for each `selectBusGuideSegment` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man selectBusGuideSegment`.
`-layer {id | id1:id2}`	Specifies layer or layer range from which the bus guide segment is to be selected.
`-netGroup {netGroup| {list_of_net_groups}}`	Specifies the name of the net group or list of net groups from which the bus guide segment is to be selected.

Example

The following command selects the bus guide segment that has bounding box coordinates `x1` = `15150.7`, `y1` = `7061.0`, `x2` = `15450.7`, `y2` = `11230.0`, layer constraint on metal `3`, associated with the net group `busGroup`.

```
selectBusGuideSegment -netGroup busGroup -box 15150.7 7061.0 15450.7 11230.0 -layer 3
```

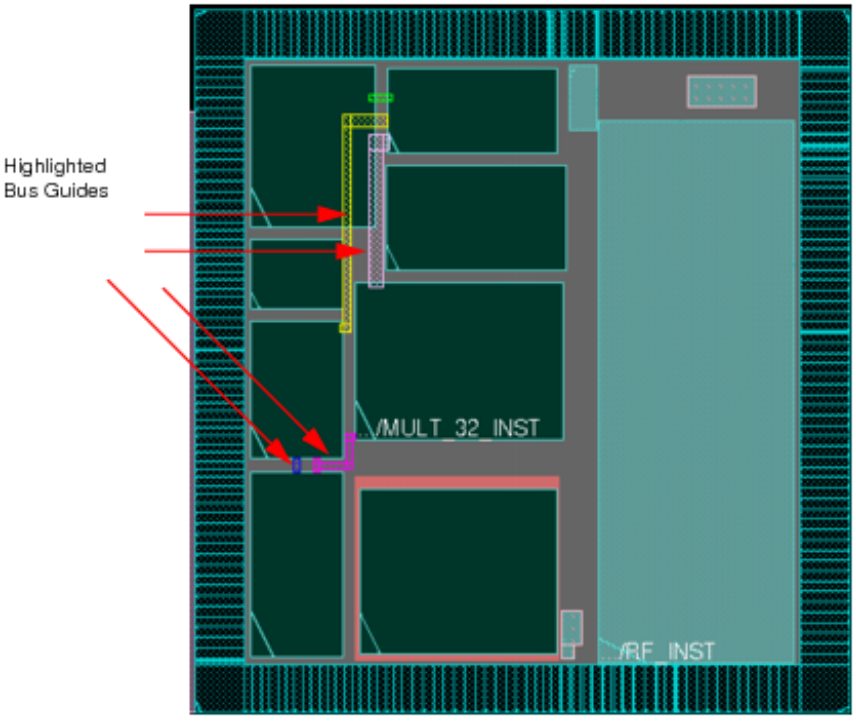
Related Information

- `createBusGuide`
- `deleteBusGuide`
- `deselectBusGuide`
- `selectBusGuide`
- `update_bus_guide`
- Bus Planning chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
    - *Bus Guide*

setBusGuideMultiColors

`setBusGuideMultiColors`

Colors all the bus guides. Use this command to make it easy to distinguish between different bus guide objects in the design.



Specify `resetBusGuideMultiColors` to clear the bus guide colors.

Parameters

None

Related Information

- `resetBusGuideMultiColors`
- Bus Planning chapter in the *User Guide*
  - *Highlighting and Dehighlighting the Bus Guide*
- Edit Menu chapter in the *Menu Reference*

- *Bus Guide*

update_bus_guide

```
update_bus_guide
[-help]
[-avoidObstacles]
[-layer_horizontal {id | id1:id2}]
[-layer_vertical {id | id1:id2}]
[-no_snap]
[-resize_direction {low high}]
[-type {hard | soft}]
[-width value | -net_group name | {{-rule rulename | {-wire_width value -
wire_spacing value}} [-with_shield]}]
```

Updates the selected bus guide as specified. The bus guide can expand automatically with pre-route or routing blockage.

Use the `update_bus_guide` command when you need to modify the layer or width information of an existing bus guide. Before using the `update_bus_guide` command, select the bus guide with the `selectBusGuide` command.

Parameters

`-help`	Prints a brief description that includes type and default information for each `update_bus_guide` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man update_bus_guide`
`-avoidObstacles`	Specifies whether to avoid pre-routes and blockages (detour). If this option is not specified, the bus guide is expanded with the pre-route and routing blockage.
`-layer_horizontal {id | id1:id2}`	Specifies the horizontal layer *id* or a layer range. Use this parameter to change the horizontal layer of the selected bus guide.
`-layer_vertical {id | id1:id2}`	Specifies the vertical layer *id* or a layer range. Use this parameter to change the vertical layer of the selected bus guide.
`-net_group name`	Specifies the net group name. The net group you specify must be an existing net group.

`-no_snap`	Specifies whether or not to snap the bus guide.
`-resize_direction {low high}`	Specifies the resize direction as follows: • `low`: Expands the selected bus guide from the top or right edge. • `high`: Expands the selected bus guide from the bottom or left edge. If this option is not specified, the bus guide is expanded from the center line.
`-rule` *`rulename`*	Specifies the rule name or default. Use this parameter to change the selected bus guide width as per the wire width or wire spacing defined in the specified LEF rule. If *`rulename`* is not specified, default LEF rule is used. **Note**: This parameter cannot be used with the `-width`, `-wire_width`, or `wire_spacing` parameters.
`-type {hard | soft}`	Specifies the new constraint type of the bus guide: • `hard` - After running Early Global Route (eGR), NanoRoute (NR), or NanoRoute High Frequency Router (NRHF), the route obeys the path of the bus guide. • `soft` - After running eGR, NR, or NRHF, the bus guide only guides the route path. The tool can route the net out of the bus guide.
`-width` *`value`*	Specifies bus guide width. Use this parameter to change the selected bus guide width to the specified value. **Note**: This parameter cannot be used with the `-rule`, `-wire_width`, or `wire_spacing` parameters.
`-wire_spacing` *`value`*	Specifies wire spacing. Use this parameter to change the selected bus guide width as per the specified wire spacing value. **Note**: This parameter cannot be used with either the `-rule` parameter or the `-width` parameter.
`-wire_width` *`value`*	Specifies wire width. Use this parameter to change the selected bus guide width as per the specified wire width value. **Note**: This parameter cannot be used with either the `-rule` parameter or the `-width` parameter.

`-with_shield`	Adds shield wire base on the shield net property defined by the following `setEditMode` options: `-outer_shield_spacing`, `-outer_shield_width`, and `-shield`. When specified with: • `-wire_width` and `-wire_spacing`, the tool considers only the outer shielding nets. • `-rule`, the tool considers the shielding nets between wires.

Example

- The following command updates the width of the selected bus guide as per the NDR *doublewidth* defined in LEF. It also changes the horizontal layer of the selected bus guide to 3 and considers the shielding nets between wires.
```
update_bus_guide -rule doublewidth -layer_horizontal 3 -with_shield
```

Related Information

- `createBusGuide`
- `deleteBusGuide`
- `selectBusGuide`
- `setEditMode`
- Bus Planning chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Bus Guide*

5

Clock Tree Synthesis Commands

- add_clock_tree_source_group_roots
- assign_clock_tree_source_groups
- calculate_ccopt_cannot_clone_reason
- ccopt_add_exclusion_drivers
- ccopt_design
- ccopt_pro
- check_ccopt_clock_tree_convergence
- clock_opt_design
- close_ctd_win
- commit_ccopt_clock_tree_route_attributes
- connect_clock_tree_mesh_drivers
- convert_lib_clock_tree_latencies
- create_ccopt_clock_spine
- create_ccopt_clock_tree
- create_ccopt_clock_tree_source_group
- create_ccopt_clock_tree_spec
- create_ccopt_flexible_htree
- create_ccopt_generated_clock_tree
- create_ccopt_macro_model_spec
- create_ccopt_preferred_cell_stripe
- create_ccopt_skew_group
- create_route_type
- cts_refine_clock_tree_placement
- ctd_save_histogram
- ctd_save_view
- ctd_show_skew_group
- ctd_trace
- ctd_win
- delete_ccopt_clock_spines
- delete_ccopt_clock_tree_source_group

- delete_ccopt_clock_tree_spec
- delete_ccopt_clock_trees
- delete_ccopt_flexible_htrees
- delete_ccopt_preferred_cell_stripe
- delete_ccopt_skew_groups
- delete_clock_tree_repeaters
- delete_route_type
- get_ccopt_clock_spines
- get_ccopt_clock_tree_capacitance
- get_ccopt_clock_tree_cells
- get_ccopt_clock_tree_nets
- get_ccopt_clock_tree_sinks
- get_ccopt_clock_tree_slew
- get_ccopt_clock_tree_source_groups
- get_ccopt_clock_trees
- get_ccopt_dag_traversal
- get_ccopt_delay_corner
- get_ccopt_effective_max_capacitance
- get_ccopt_flexible_htrees
- get_ccopt_preferred_cell_stripe
- get_ccopt_property
- get_ccopt_skew_group_delay
- get_ccopt_skew_group_path
- get_ccopt_skew_groups
- get_ctd_win_id
- get_ctd_win_title
- get_lib_clock_tree_path_delay
- gui_zoom_ctd
- merge_clock_cells
- modify_ccopt_skew_group
- preserve_ccopt_port
- report_ccopt_cell_filtering_reasons
- report_ccopt_cell_halo_violations
- report_ccopt_clock_tree_convergence
- report_ccopt_clock_tree_drv

- report_ccopt_clock_tree_structure
- report_ccopt_clock_trees
- report_ccopt_pin_insertion_delays
- report_ccopt_preserved_clock_tree_ports
- report_ccopt_skew_groups
- report_ccopt_worst_chain
- reset_all_ccopt_preserved_clock_tree_ports
- reset_ccopt_config
- reset_ccopt_preserved_clock_tree_port
- reset_ccopt_routing_state
- restore_ccopt_config
- route_ccopt_clock_tree_nets
- route_ccopt_flexible_htrees
- save_ccopt_config
- set_ccopt_preserved_clock_tree_port
- set_ccopt_property
- set_ctd_win_title
- show_ccopt_cell_name_info
- synthesize_ccopt_flexible_htrees
- unset_ccopt_property
- update_ccopt_clock_tree
- update_clock_latencies
- update_clock_tree_source_latency
- update_clock_tree_spec_annotations

add_clock_tree_source_group_roots

```
add_clock_tree_source_group_roots
[-help]
-cell driverCell
[-locations {{x1 y1} {x2 y2}...}]
-name sourceGroupName
-pin rootPin
[-grid {columns rows} [[-adjust_grid_for_aspect_ratio]
[-grid_rect {lx ly ux uy}]
[-grid_search_area {width height}]
[-grid_exclusion_rects {{lx1 ly1 ux1 uy1} {lx2 ly2 ux2 uy2}...}]]]
```

This command updates the design and clock tree specification to create a multiple-tree "source group" below the specified driving root pin. A source group is a set of multiple, logically equivalent, clock trees that drive a collection of clock tree sinks. The

actual allocation of sinks to specific source group roots is chosen during clock tree synthesis.

This command performs the following tasks:

- Inserts the instances that will become the clock tree roots into the netlist. Their input pins will be driven by the root pin and the fanout originally driven by the root pin will be rewired to the output of a single root instance.
- Defines clock trees at the outputs of each of the newly inserted root instances.
- Defines a clock source group to combine the clock trees.

You can insert a grid of tap buffers and also omit a tap buffer at grid points that are blocked. For this, you can specify grid coordinates, a two-value pair `{x y}`, that place the source group driver at that location.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `add_clock_tree_source_group_roots` parameter. For a detailed description of the command and all of its parameters, use the man command: `man add_clock_tree_source_group_roots`.
`-adjust_grid_for_aspect_ratio`	
When specified, exchanges rows and columns to match the aspect ratio.	
`-cell` *`driverCell`*	Specifies the buffer cell to use for new source group # roots (`libCell/ptnCell`). This is a required parameter.
`-grid {columns rows}`	
	Specifies a grid of target locations. By default, the command attempts to create a full grid consisting of columns and rows. However, if a particular grid target point falls within an exclusion rectangle, specified using the `-grid_exclusion_rects` parameter, then that grid point will be skipped. If the `-adjust_grid_for_aspect_ratio` parameter is specified, the columns and rows will be swapped if doing so better matches the aspect ratio of the grid rectangle area. **Note**: The `-grid` and `-locations` parameters can be used together. For example, you can specify a grid, and then use `-locations` to add a few additional taps in specific floor-plan locations.
`-grid_exclusion_rects {{lx1 ly1 ux1 uy1} {lx2 ly2 ux2 uy2}...}`	
	Provides the coordinates of rectangular areas that should not contain source group roots. Any grids that fall within the exclusion rectangle will be ignored.
`-grid_rect {lx ly ux uy}`	

	Specifies the area that the grid should cover. This parameter provides the overall rectangle intended to be driven by the taps requested by the `-grid` parameter. **Note**: This parameter does not specify the bounding box of the grid taps. Basically the overall `-grid_rect` rectangle is cut up into `m x n` smaller rectangles, and a tap is targeted at the center of each smaller rectangle. to understand more clearly, consider the below example. If you have `-grid {2 2}` and `-grid_rect {0 0 100 100}`, the corresponding target locations will be `{{25 25} {25 75} {75 25} {75 75}}`. This is different than if the `-grid_rect` were the bounding box of tap locations, in which case it would be `{{0 0} {0 100} {100 0} {100 100}}`).
`-grid_search_area {width height}`	
	Specifies the size of the search area around each grid point.
`-locations {{x1 y1} {x2 y2}...}`	
	Specifies a list of (`x,y`) locations for the new roots that are to be created. The specified roots are inserted at the nearest legal location. **Note**: This parameter takes a list of points, not rectangles. A specified point will be taken as a tap target location, and the software will search for the nearest legal location. If the `-grid_search_area {width height}` parameter is specified, the search will be limited to the points specified using the `-grid` parameter and not points specified using the `-locations` parameter. If it is not specified, the software will search as far as needed to find a legal location for a tap. If a legal location cannot be found, either within the specified search area, or anywhere in the floorplan, the tap will be added at the target location and not legalized. If all taps can be inserted at legal locations, they will be set to locked for CTS placement status. If one or more taps cannot be legalized, the command will print a warning and all taps will be left in simple placed status. **Note**: The behavior when a tap cannot be legalized, and for whether taps are locked applies to all taps created by this command - from the `-grid` and/or `-locations` parameters.
`-name` *prefix*	Specifies the name of the source group to be created for the roots. The name should be unique. The multi-tap roots will be named using this string as a prefix. For example, `sourceGroupName_0`, `sourceGroupName_1`..`sourceGroupName_n`. **Note**: • The skew group name will be `<`*prefix*`>_skew_group`. • The clock tree name for a tap will be `<`*prefix*`>_<index>`, and `<`*prefix*`>_<index>_inst` for the corresponding cell instance. This is a required parameter.
`-pin` *rootPin*	Specifies the name of the root pin that will drive the multi-tap root inputs. The pin must be an output pin and must drive a net. The entire clock structure, including clock gates, flops, and so on that are to be driven by the multi-tap roots should originally be connected below this pin. If there are multiple drivers on the root net, any one of their output pins may be specified as the `-pin` parameter. This is a required parameter.

Related Commands

- assign_clock_tree_source_groups
- create_ccopt_clock_tree_source_group
- delete_ccopt_clock_tree_source_group
- get_ccopt_clock_tree_source_groups

assign_clock_tree_source_groups

```
assign_clock_tree_source_groups
[-help]
[-no_merge]
```

When used on designs with clock tree source group definitions, this command assigns clock sinks to the source group roots - multi-tap assignment - and then performs the cloning and rewiring necessary to distribute the sinks among the source group roots.

Note that the source group assignment process normally happens automatically during the early stages of clock tree synthesis. However, on large designs, the full clock tree synthesis step can take some time. The purpose of this command is to provide an easier way to get early feedback about the multi-tap assignment process, visualize the result, adjust source group definitions, and so on.

By default, the command will do initial clock gate merging and then clone as necessary to implement the assignment. However, the merging step can be disabled by specifying the `-no_merge` option. If no source groups are defined, the command will do nothing.

After running the command, you can review the log file messages, or check the result in the database either by using the Tcl commands, clock tree debugger, and so on.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `assign_clock_tree_source_groups` parameter. For a detailed description of the command and all of its parameters, use the man command: `man assign_clock_tree_source_groups`.
`-no_merge`	Specifies that clock gates should not be merged.

Related Commands

- `add_clock_tree_source_group_roots`
- `create_ccopt_clock_tree_source_group`
- `delete_ccopt_clock_tree_source_group`
- `get_ccopt_clock_tree_source_groups`

calculate_ccopt_cannot_clone_reason

```
calculate_ccopt_cannot_clone_reason
[-help]
-inst instName
```

Returns a Tcl list of reasons why the specified instance cannot be cloned in CTS. An empty list indicates the given instance can be cloned. This command is similar to the CCOpt property, `cannot_clone_reason` but it can be used before running `ccopt_design`.

It differs from the property, `cannot_clone_reason` in that it always calculates the reason as compared to just retrieving a saved property value from the last time that cloning happened.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `calculate_ccopt_cannot_clone_reason` parameter. For a detailed description of the command and all of its parameters, use the man command: `man calculate_ccopt_cannot_clone_reason`.
`-inst` *instName*	Specifies the name of the instance for which you want to know the reason for being uncloneable.

Related Information

- The `cannot_clone_reason` property.
  This property provides a list of reasons to explain why the cell could not be cloned during clustering. For more information on above property, see the CCOpt Properties chapter in the *Innovus User Guide* or run the following command:

  ```
  get_ccopt_property -help cannot_clone_reason
  ```

- `ccopt_design`
- `clock_opt_design`

ccopt_add_exclusion_drivers

```
ccopt_add_exclusion_drivers
[-help]
[-lib_cell list_library_cells]
```

Removes the excluded sinks from the clock trees by adding exclusion drivers above these sinks. Use this command after running the `create_ccopt_clock_tree_spec` command. Alternatively, you can use the `add_exclusion_drivers` property to remove the exclude sinks from the clock tree. It is recommended that you use one of the two, either the command or the property, to exclude sinks from the clock trees.

For details of this property, see the "Related Properties" section below.

The names of the newly added drivers start with '`CTS_cse`', so that they can be easily identified. This functionality is useful because it allows you to fix hold violations on clock nets. GigaOpt does not fix hold violations on clock nets. For example, if a pin, D, of a flop is a clock sink then the net attached to the D pin is a clock net. Any hold violations will not be fixed by GigaOpt. However, by adding an exclusion buffer above the D pin, the net between the buffer's output pin and the D pin of the flop becomes a non-clock net and any hold violations will be fixed by GigaOpt. Therefore, this new functionality allows previously unfixable hold violations to be fixed.

You can use the following command to view the exclusion drivers that have been added by CCOpt:

```
get_ccopt_clock_tree_sinks *CTS_cse*
```

Parameters

`-help`	Outputs a brief description that includes type and default information for each `ccopt_add_exclusion_drivers` parameter. For a detailed description of the command and all of its parameters, use the man command: `man ccopt_add_exclusion_drivers`.
`-lib_cell`	Specifies a particular buffer or inverter to use as an exclusion driver. If this parameter is not specified or if the property, `add_exclusion_drivers`, is used to enable this functionality, then CCOpt will choose small area drivers from the CTS cell lists to use as exclusion drivers. CCOpt will either add pairs of inverters, or single buffers as exclusion drivers. While choosing the cell to use, CCOpt will choose either inverters or buffers depending on the value specified using the `use_inverters` property. This property specifies whether clock tree synthesis should use inverters or buffers when balancing the clock trees. For more information about the `use_inverters` property, use the following command: `get_ccopt_property -help use_inverters`

Related Properties

- `add_exclusion_drivers`
  This is a user-settable property. Setting this property to `true` before running `ccopt_design` or `ccopt_design -cts` will cause CCOpt to add extra drivers above exclude pins to remove them from the clock tree.

  Normally an exclusion buffer or a pair of inverters are added above an exclude sink to remove them from the clock tree. A new clock sink will be created on the input to the newly added driver. If there is a suitable pre-existing driver above the exclude pin, then no extra driver will be added – the new clock sink will be created on the input to the pre-existing driver.
- `effective_sink_type`
  This is a read-only property. It indicates how CCOpt will treat a given pin, taking into account any `implicit_sink_type` and `sink_type` settings.

- `implicit_sink_type`
  This is a read-only property. It Indicates the type of sink that CCOpt classified this pin as. The possible values are `exclude`, `ignore`, `stop`, and `empty string ""`. Instances of cell pins that are data pins, scan input pins, or scan enable pins will have `implicit_sink_type=exclude`. This means that unless there is an explicit `sink_type` set for that pin, either `ignore` or `stop`, they will be removed from the clock tree. This property takes into account the settings done using the sink_tpe property. Setting a non-default value for the `sink_type` property will override the `implicit_sink_type` property.
- `sink_type`
  This is a user-settable property. It specifies the type of sink that the pin represents. Valid values are `auto`, `through`, `stop`, `ignore`, `min`, and `exclude`. To exclude a clock sink that is not already `implicit_exclude`, you can set this property to `exclude`. This will change the `effective_sink_type` property to `exclude`, and the sink will be removed from the clock tree.

  For more information about the above properties, use the following command:
  `get_ccopt_property -help property_name`

Related Information

- `ccopt_design`
- `clock_opt_design`
- `create_ccopt_clock_tree_spec`
- `get_ccopt_clock_tree_sinks`
- The following topic in the *Innovus User Guide*.
  - Shared Clock and Data Concerns in CTS

ccopt_design

```
ccopt_design
[-help]
[-check_prerequisites]
[-ckSpec]
[-cts]
[-expandedViews]
[-num_paths number_of_paths]
[-outDir dirname]
[-prefix fileNamePrefix]
[-timing_debug_report]
```

Performs clock concurrent optimization (CCOpt) on the current loaded design in Innovus. CCOpt optimizes both the clock tree and the datapath to meet global timing constraints.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `ccopt_design` parameter. For a detailed description of the command and all of its parameters, use the man command: `man ccopt_design`.
`-check_prerequisites`	Checks that all the prerequisites for running clock tree synthesis are fulfilled without actually doing CTS. For example, it checks that the design is loaded, and the options set by that the user are correct.
`-ckSpec`	Runs the design using the legacy FE-CTS specification file. This mode is a compatibility layer to enable the use of CCOpt technology with existing FE-CTS specification flows. It enables a translation step that transforms constructs in the FE-CTS specification file into a CCOpt clock tree specification tcl script. **Note**: This parameter is used with the `-cts` parameter to specify that CTS part of CCOpt will be performed using the legacy FE-CTS specification file.
`-cts`	Turns off clock concurrent optimization, and performs only clock tree synthesis (CTS) using the CCOpt engine. It does not perform any datapath optimization or useful skew. This parameter can be used with or without the `-ckSpec` parameter. When used with the `-ckSpec` parameter, it specifies that the CTS part of CCOpt will be performed using the legacy FE-CTS specification file.
`-expandedViews`	This parameter is not interpreted in CCOpt but passed directly to `optDesign`. The parameter is same as `optDesign -expandedViews` parameter, which is specified to generate detailed view-specific timing reports at the end of the `optDesign` command run.
`-num_paths` *number_of_paths*	Specifies the number of paths generated in the timing reports. By default, all timing paths are reported.
`-outDir` *dirname*	This parameter specifies the directory where the software writes timing reports that are generated after timing optimization. The default directory is `./timingReports`
`-prefix` *fileNamePrefix*	Specifies a prefix for `optDesign` report file names. *Default*: `DesignName_DesignStage` , where `DesignStage` is preCTS, postCTS, or postRoute
`-timing_debug_report`	When specified, generates a machine-readable timing debug report after completing CTS. This report is used for debugging timing results using the timing debug feature in Innovus. The number of paths to be reported can be specified using the `-num_paths` parameter.

Example

- The following command runs a complete CCOpt optimization:
  ```
  ccopt_design
  ```

Related Information

- `clock_opt_design`
- `create_ccopt_clock_tree_spec`
- `optDesign`
- The following topic in the *Innovus User Guide*
  - Flow and Quick Start for CTS

ccopt_pro

```
ccopt_pro
[-help]
[-enable_average_id_reduction {true | false}]
[-enable_downsizing_pass {true | false}]
[-enable_drv_fixing {true | false}]
[-enable_drv_fixing_by_rebuffering {true | false}]
[-enable_refine_place {true | false}]
[-enable_routing_eco {true | false}]
[-enable_skew_fixing {true | false}]
[-enable_skew_fixing_by_rebuffering {true | false}]
```

Repairs clock tree design rule violations (DRVs) and skews that have occurred in the clock tree across a routing step. DRVs and skews can be fixed either by sizing existing clock instances or by adding buffers. Use this command to enable or disable average insertion delay reduction, global cell downsizing pass, refine placement of design, ECO routing, legalizer sharing, and the timing update step.

This command can be run at the following stages:

- After `ccopt_design` – when only the clock is routed
- After `routeDesign` , but before `optDesign -postRoute` – when both clock and datapath are routed.

By default a post `ccopt_design` call to this command will fix DRVs and skew violations by resizing instances. It will also fix persistent DRVs by adding buffers. However, when called post `routeDesign`, this command is less disruptive by default and will only

perform resizing to fix DRVs. The `ccopt_pro` log will report a list of "active optimizations" for clarification of what steps will be executed.

By default, the command will modify the clock to reduce violations and then call `refinePlace` and `ecoRoute` to patch up the modified routes. Both `refinePlace` and `ecoRoute` parts may be disabled to reduce runtime for user experimentation. But they will need to be called eventually, either with a final call to `ccopt_pro` or manually. Without them, timing measurements will be inaccurate.

Note:

- This command is integrated within the `ccopt_design` command and is run after clock routing. It can be identified in the `ccopt_design` log with a search for "PostConditioning....".
- This command is integrated within the `optDesign -postRoute` command. It can be identified in the `optDesign` log with a search for "PRO...".
- If this command is called after running `ccopt_design` (when only the clock is routed), it will also call `earlyGlobalRoute` on the datapath nets.
- The `ccopt_pro` command and the clock DRV fixing performed by running `optDesign -postRoute` do not work when signoff effort-level for extraction is specified using `setExtractRCMode -effortLevel signoff`. The signoff effort-level for extraction should be used only for ECO flows, after postRoute optimization.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `ccopt_pro` parameter. For a detailed description of the command and all of its parameters, use the man command: `man ccopt_pro`.
`-enable_average_id_reduction {true | false}`	Enables or disables average insertion delay reduction.
`-enable_downsizing_pass {true | false}`	Enables or disables a global cell downsizing pass.
`-enable_drv_fixing {true | false}`	Enables or disables DRV fixing.
`-enable_drv_fixing_by_rebuffering {true | false}`	Enables or disables DRV fixing by adding buffers.
`-enable_refine_place {true | false}`	Enables or disables refine placement of design. Disabling this may result in instance overlaps.
`-enable_routing_eco {true | false}`	Enables or disables the ECO routing step, which corrects routing for new instance locations. Disabling this will leave inaccurate routing in place and lead to timing inaccuracy.
`-enable_skew_fixing {true | false}`	Enables or disables fixing of clock tree skew. **Note the following**: • When `-enable_skew_fixing` is set to `false` then whether the `-enable_skew_fixing_by_rebuffering` parameter is specified as `true` or `false`, there is no skew fixing during `ccopt_pro`. • When `-enable_skew_fixing` is set to `true`, while `-enable_skew_fixing_by_rebuffering` parameter is set to `false`, the software resizes cells to fix skew. • When `-enable_skew_fixing` is set to `true`, while `-enable_skew_fixing_by_rebuffering` parameter is set to `true`, the software not only resizes cells but also does rebuffering to fix skew.
`-enable_skew_fixing_by_rebuffering {true | false}`	Enables or disables fixing of clock tree skew by adding buffers. This will only help with lengthening short paths.

Related Commands

- ccopt_design
- clock_opt_design
- ecoRoute
- earlyGlobalRoute
- refinePlace
- routeDesign
- optDesign
- setExtractRCMode

check_ccopt_clock_tree_convergence

```
check_ccopt_clock_tree_convergence
[-help]
[-warnthreshold thresholdValue]
```

This command is used to trigger a warning message for sinks that have more than the specified number of convergence paths from the clock sources to the sink. This check is performed at the end of clock tree specification, for complex convergence above sinks that might cause large runtimes in CCOpt as it attempts to balance these trees.

When this command is specified, any sinks that are found to have more than the specified *thresholdvalue* of paths from the clock sources to the sink are reported in a warning message from the software. For example:

```
WARN: (IMPCCOPT-4338): There are 17 clock paths leading to primaryout. This may cause long CCOpt runtimes. Type
'man IMPCCOPT-4338' for more detail.
```

The following detailed information is provided when you type `man IMPCCOPT-4338`:

```
The design contains a lot of converging clock paths leading to some sinks. This may cause long CCOpt runtimes.
The sinks with the most paths leading to them can be found with report_ccopt_clock_tree_convergence. See CCOpt
Clock Tree Specification under Clock Concurrent Optimization in the User Guide for more information.
```

Note: The warning is issued if the number of clock paths to any sink is greater than the threshold value of paths specified by the command `check_ccopt_clock_tree_convergence`.

To view a summary of the convergence above clock sinks and to identify the names of the sinks with the highest number of clock network paths, run the `report_ccopt_clock_tree_convergence`command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `check_ccopt_clock_tree_convergence` parameter. For a detailed description of the command and all of its parameters, use the man command: `man check_ccopt_clock_tree_convergence`.
`-warnthreshold` *thresholdValue*	Triggers a warning message for sinks with more than the specified number of paths. *Default*: `100`

Related Information

- `create_ccopt_clock_tree_spec`
- `report_ccopt_clock_tree_convergence`
- The following topic in the *Innovus User Guide*
  - Clock Tree Convergence Reporting

clock_opt_design

```
clock_opt_design
[-help]
[-check_cts_config]
[-cts]
[-expandedViews]
[-num_paths number_of_paths]
[-out_dir dirname]
[-prefix fileNamePrefix]
[-timing_debug_report]
```

The `clock_opt_design` command is used in place of `ccopt_design` when the input DB is place_opt_design V2 (PODV2 or pod-turbo). It performs clock concurrent optimization (CCOpt) on the current loaded design in Innovus. CCOpt optimizes both the clock tree and the datapath to meet global timing constraints. The only difference between this flow and the `ccopt_design` flow is that `clock_opt_design` has legalization at the beginning to remove overlapping caused by PODV2 optimization.

In this flow, the commands, `place_opt_design` and `clock_opt_design` are used in combination. This means that the `clock_opt_design` command can only be used with the `place_opt_design V2 (PODV2)`, which is the `place_opt_design` command that is run after enabling the PODV2 flow.

For details of the flow, see The clock_opt_design Flow.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `clock_opt_design` parameter. For a detailed description of the command and all of its parameters, use the man command: `man clock_opt_design`.
`-check_cts_config`	Checks that all the prerequisites for running clock tree synthesis are fulfilled without actually doing CTS. For example, it checks that the design is loaded, and the options set by that the user are correct. There are no changes made to the design
`-cts`	Turns off clock concurrent optimization, and performs only clock tree synthesis (CTS) using the CCOpt engine. It does not perform any datapath optimization or useful skew.
`-expandedViews`	This parameter is not interpreted in CCOpt but passed directly to `optDesign`. The parameter is same as `optDesign -expandedViews` parameter, which is specified to generate detailed view-specific timing reports at the end of the `optDesign` command run.
`-num_paths` *`number_of_paths`*	Specifies the number of paths generated in the timing reports. By default, all timing paths are reported.
`-out_dir` *`dirname`*	This parameter specifies the directory where the software writes timing reports that are generated after timing optimization. The default directory is `./timingReports`
`-prefix` *`fileNamePrefix`*	Specifies a prefix for `optDesign` report file names. *Default*: `DesignName_DesignStage` , where `DesignStage` is preCTS, postCTS, or postRoute
`-timing_debug_report`	When specified, generates a machine-readable timing debug report after completing CTS. This report is used for debugging timing results using the timing debug feature in Innovus. The number of paths to be reported can be specified using the `-num_paths` parameter.

Examples

- The following command runs a complete CCOpt optimization for the podV2 flow:
  ```
  clock_opt_design
  ```

Related Information

- `ccopt_design`
- `create_ccopt_clock_tree_spec`
- `optDesign`
- `place_opt_design`
- The following topic in the *Innovus User Guide*.
  - Flow and Quick Start for CTS

close_ctd_win

```
close_ctd_win
[-help]
[-all | -id WindowIDName]
```

Closes open CTD windows. When no parameter is specified, the command closes the current active CTD window.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `close_ctd_win` parameter. For a detailed description of the command and all of its parameters, use the man command: `man close_ctd_win`.
`-all`	Closes all open CTD windows.
`-id` *`WindowIDName`*	Closes the CTD window with the specified window ID. **Note**: The parameters, `-all` and `-id` are mutually exclusive.

Examples

- The following command closes all CTD windows in the current session:

  ```
  close_ctd_win -all
  ```

- The following command closes the CTD window with the ID, `user_id2`:

  ```
  close_ctd_win -id user_id2
  ```

Related Information

- `create_ccopt_clock_tree_spec`
- `ctd_trace`
- `ctd_win`
- `get_ctd_win_id`
- `get_ctd_win_title`
- `gui_zoom_ctd`
- Clock Menu chapter in the *Innovus Menu Reference*

commit_ccopt_clock_tree_route_attributes

```
commit_ccopt_clock_tree_route_attributes
[-help]
[-verbose]
```

The command traces the clock tree as defined by the CCOpt spec and sets net attributes to allow early-global-route to more predictably model clock nets. It sets `isClock` and `isCTSClock` on clock tree nets in the database. It applies routing preferences such as: `top_preferred_layer`, `bottom_preferred_layer`, `non_default_rule`, and `shield_net` based on the "`route_type`" CCOpt properties.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `commit_ccopt_clock_tree_route_attributes` parameter. For a detailed description of the command and all of its parameters, use the man command: `man commit_ccopt_clock_tree_route_attributes`.
`-verbose`	Prints the layer constraint summary for CCOpt nets after attribute application.

Related Information

For detailed information, see the CCOpt Properties chapter in the *Innovus User Guide*.

You can also view property help by running the following command:

```
get_ccopt_property -help property name
```

connect_clock_tree_mesh_drivers

```
connect_clock_tree_mesh_drivers
[-help]
[-driver_pins pin_names]
[-driver_source_group source_group_name]
```

Uses the specified set of drivers and merges the nets at their outputs to form a single multi-driven net. To run this command successfully, one of the two options, `-driver_pins` or `-driver_source_group` must be specified.

It is possible that some or all of the driver pin nets are already multi-drive. You might be repeating the same command, or you could be merging additional drivers into an already multi-drive net. Running the command twice does nothing the second time. If the net of any driver pin is *already* multi-drive, the pin list will be interpreted to include all drivers of that net.

The command returns the resulting merged net through a Tcl return value.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `connect_clock_tree_mesh_drivers` parameter. For a detailed description of the command and all of its parameters, use the man command: `man connect_clock_tree_mesh_drivers`.
`-driver_pins` *`pin_names`*	Specifies a list of driver output pins whose nets will be merged to form a multi-drive mesh net. **Note**: This parameter is optional if the `-driver_source_group` parameter is specified.
`-driver_source_group` *`source_group_name`*	Specifies the source group for the list of driver output pins. Only the root pins of the specified source group will be included in the list of driver output pins. **Note**: This parameter is optional if the `-driver_pins` parameter is specified.

Related Command

- `add_clock_tree_source_group_roots`

convert_lib_clock_tree_latencies

```
convert_lib_clock_tree_latencies
[-help]
[-latency_file_prefix prefix_name]
[-pins pin_list]
[-views view_list]
[-override_existing_latencies | -sum_existing_latencies]
[-sum_existing_latencies | -override_existing_latencies_pins pin_list]
[-override_existing_latencies | -sum_existing_latencies_pins pin_list]
[-sum_existing_latencies | -sum_existing_latencies_pins pin_list]
[-override_existing_latencies | -override_existing_latencies_pins pin_list]
```

Converts the Liberty `max_clock_tree_path` and `min_clock_tree_path` (MCTP) data to per-pin clock latency adjustments.

Note:

- Run the command before running the `create_ccopt_clock_tree_spec` command so that the the output of `convert_lib_clock_tree_latencies` command impacts the clock tree specification that is created.
- Before running this command, ensure that the analysis type is set as `onChipVariation` in the following manner:
  `setAnalysisMode -analysisType onChipVariation`

By default, the conversion will consider all instance pins with MCTP data in all analysis views. Pins which have any existing network latency, differing from the corresponding clock network latency will be skipped completely for all clocks and for all views – this avoids the conversion process from interfering with any SDC-specified pin latencies. Alternatively, existing latency offsets can be summed or overridden for all or names pins using the appropriately named override and sum option. Additionally, the `-pins` option can be used to consider only a specified list of pins instead of all pins with MCTP data.

By default, the conversion will internally apply the resulting `set_clock_latency` commands to the in memory timing constraints. Alternatively the `-latency_file_prefix` parameter can be specified to have one file per analysis view written containing the `set_clock_latency` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `convert_lib_clock_tree_latencies` parameter. For a detailed description of the command and all of its parameters, use the man command: `man convert_lib_clock_tree_latencies`.
`-latency_file_prefix` *prefix_name*	
	When specified, the software, instead of applying changes, writes the commands to one latency file per analysis view. Any pre-existing files are overwritten. For example, *<prefix><analysis_view_name>.sdc* **Note**: The prefix may contain directory name components. New directories should be created, as required.
`-override_existing_latencies`	

	When specified, includes all pins with existing latency offsets and overwrites the existing latency offsets. **Note**: • The `-override_existing_latencies` and `-sum_existing_latencies` parameters are mutually exclusive. • The `-override_existing_latencies` and `-sum_existing_latencies_pins` parameters are mutually exclusive. • The `-override_existing_latencies` and `-override_existing_latencies_pins` parameters are mutually exclusive.
`-override_existing_latencies_pins` *pin_list*	
	Includes only the specified list of pins with existing latency offsets and overwrites the existing latency offsets. **Note**: • The `-override_existing_latencies_pins` and `-override_existing_latencies` parameters are mutually exclusive. • The `-override_existing_latencies_pins` and `-sum_existing_latencies` parameters are mutually exclusive. • The software issues an error if a pin is specified in both `-override_existing_latencies_pins` and `-sum_existing_latencies_pins` parameters.
`-pins` *pin_list*	
	Specifies the clock endpoint pins to consider. By default, the software considers all clock endpoints with the library `max/min_clock_tree_path` attributes. **Note**: The software issues an error if a pin is specified in either the `-sum_existing_latencies_pins` or the `-override_existing_latencies_pins` list, but the pin is not included in the list of pins specified using the `-pins` parameter.
`-sum_existing_latencies`	
	When specified, includes all pins with existing latency offsets and sums the existing latency offset with the library-specified offset. **Note**: • The `-sum_existing_latencies` and `-override_existing_latencies` parameters are mutually exclusive. • The `-sum_existing_latencies` and `-override_existing_latencies_pins` parameters are mutually exclusive. • The `-sum_existing_latencies` and `-sum_existing_latencies_pins` parameters are mutually exclusive.
`-sum_existing_latencies_pins` *pin_list*	
	Includes only the specified list of pins with existing latency offsets and sums the existing latency offset with the library-specified offset. **Note**: • The `-sum_existing_latencies_pins` and `-sum_existing_latencies` parameters are mutually exclusive. • The `-sum_existing_latencies_pins` and `-override_existing_latencies` parameters are mutually exclusive. • The software issues an error if a pin is specified to both `-sum_existing_latencies_pins` and `-override_existing_latencies_pins` parameters.

-views *view_list*	
	Specifies the analysis views to operate on. By default the software operates on all active analysis views.

Examples:

Example 1 – Two clocks, existing clock network latency, no pre-existing latency override

```
Circuit

ck1 ---> +-----+

         | mux | -----> icg --+--> flop1/CK   (regular flop pin without MCTP data)

ck2 ---> +-----+              +--> macro1/CK  (pin with MCTP data, of 0.222R/0.232F at 0.125 transition time)

                              +--> macro2/CK  (pin with MCTP data, of 0.333R/0.343F at 0.125 transition time)

SDC

  create_clock -period 1.0 [get_ports ck1]

  create_clock -period 1.0 [get_ports ck2]

  set_clock_transition 0.125 [get_clocks ck1]

  set_clock_transition 0.125 [get_clocks ck2]

  set_clock_latency 1.5 [get_clocks {ck1}]

  set_clock_latency 2.0 [get_clocks {ck2}]

Command and log output

convert_lib_clock_tree_latencies -latency_file_prefix lat_ -sum_existing_latencies -views
default_analysis_view_setup

Converting library clock tree path delays to clock latencies for analysis_view default_analysis_view_setup

Found 2 of 3 clock endpoints with library clock tree path data

+-----------------------------------------------------------------+

| Clock endpoint         | Status                                 |

|------------------------+----------------------------------------|

| macro1/ck              | converted                              |

| macro2/ck              | converted                              |

+-----------------------------------------------------------------+

Writing latencies to file 'lat_default_analysis_view_setup.sdc'

Output file (the comments should not be written in the file)

  set_clock_latency 1.278 [get_pins {macro1/CK}] -clock [get_clocks {ck1}] -rise /1.5-0.222=1.278

  set_clock_latency 1.268 [get_pins {macro1/CK}] -clock [get_clocks {ck1}] -fall /1.5-0.232=1.268

  set_clock_latency 1.778 [get_pins {macro1/CK}] -clock [get_clocks {ck2}] -rise /2.0-0.222=1.778

  set_clock_latency 1.768 [get_pins {macro1/CK}] -clock [get_clocks {ck2}] -fall /2.0-0.232=1.768

  set_clock_latency 1.167 [get_pins {macro2/CK}] -clock [get_clocks {ck1}] -rise /1.5-0.333=1.167

  set_clock_latency 1.157 [get_pins {macro2/CK}] -clock [get_clocks {ck1}] -fall /1.5-0.343=1.157
```

```
  set_clock_latency 1.667 [get_pins {macro2/CK}] -clock [get_clocks {ck2}] -rise /2.0-0.333=1.667
  set_clock_latency 1.657 [get_pins {macro2/CK}] -clock [get_clocks {ck2}] -fall /2.0-0.343=1.657
```

Example 2 – User-specified pin list

```
Circuit
  ck1 --> icg --+--> flop1/CK   (regular flop pin without MCTP data)
                +--> macro1/CK  (pin with MCTP data, of 0.222R/0.232F at 0.125 transition time)
                +--> macro2/CK  (pin with MCTP data, of 0.333R/0.343F at 0.125 transition time)
SDC
  create_clock -period 1.0 [get_ports ck1]
  set_clock_transition 0.125 [get_clocks ck1]
Command and log output
convert_lib_clock_tree_latencies -latency_file_prefix lat_ -pins {flop1/CK macro2/CK flop3/D} -views
default_analysis_view_setup
Converting library clock tree path delays to clock latencies for analysis_view default_analysis_view_setup
Found 2 of 3 clock endpoints with library clock tree path data
+-------------------------------------------------------------------+
| Clock endpoint         | Status                                   |
|------------------------+------------------------------------------|
| flop1/ck               | skipped, no library clock tree path data  |
| macro2/ck              | converted                                |
| flop3/D                | skipped, not a clock endpoint            |
+-------------------------------------------------------------------+
Writing latencies to file 'lat_default_analysis_view_setup.sdc'
Output file
  set_clock_latency -0.333 [get_pins {macro2/CK}] -clock [get_clocks {ck1}] -rise
  set_clock_latency -0.343 [get_pins {macro2/CK}] -clock [get_clocks {ck1}] -fall
```

Example 3 – Multiple analysis views

```
Command and log output
convert_lib_clock_tree_latencies -latency_file_prefix lat_ -pins {flop1/CK macro2/CK}
Converting library clock tree path delays to clock latencies for analysis_view 'setup1'
Found 2 of 3 clock endpoints with library clock tree path data
+-------------------------------------------------------------------+
| Clock endpoint         | Status                                   |
|------------------------+------------------------------------------|
| macro1/ck              | skipped, existing latency found          |
| macro2/ck              | converted                                |
  +-------------------------------------------------------------------+
```

```
Converting library clock tree path delays to clock latencies for analysis_view 'setup2'
Found 2 of 3 clock endpoints with library clock tree path data
+-------------------------------------------------------------------------+
| Clock endpoint          | Status                                        |
|-------------------------+-----------------------------------------------|
| macro1/ck               | skipped, existing latency found in other view |
| macro2/ck               | converted                                     |
+-------------------------------------------------------------------------+
Writing latencies to file 'lat_setup1.sdc'
Writing latencies to file 'lat_setup2.sdc'
```

Related Information

- create_ccopt_clock_tree_spec
- get_lib_clock_tree_path_delay
- set_clock_latency
- setAnalysisMode
- create_clock
- The following topic in the *Innovus User Guide*
  - Converting Library Path Delays to Clock Latencies

create_ccopt_clock_spine

```
create_ccopt_clock_spine
[-help]
[-hard]
-name spinename
[-x x]
[-xmax xmax]
[-xmin xmin]
[-y y]
[-ymax ymax]
[-ymin ymin]
```

Creates a CCOpt spine object. A spine is a single vertical or horizontal line under which clock instance pins that exist in the prects netlist can be arranged. Spines must be either vertical with a single X value or horizontal with a single Y value. For example, to create a vertical spine, you need to specify a target X value, and a Y range. All distances are in microns. For example:

```
create_ccopt_clock_spine -name spineA -x 1000.0 -ymin 500.0 -ymax 1500.0
```

The software displays the following message:

```
Created clock spine: spineA
```

Note: Only those pins whose CCOpt property, `lock_on_clock_spine`, contains the name of the spine will be aligned to that spine. This property specifies a set of clock spine names restricting where the pin should be located. If the set contains more than one item, the pin may be placed on any of the specified clock spines. For more information about how this property is used, type the following:

```
get_ccopt_property -help lock_on_clock_spine
```

A spine may be specified as "hard", in which case it defines a rectangle, and cells will be moved out of this area, if possible, to make way for cells locked to the spine. For example:

```
create_ccopt_clock_spine -name spineB -hard -y 2000.0 -xmin 0.0 -xmax 3000.0 -ymin 1990.0 -ymax 2010.0
```

Note: When specifying a hard spine, the target X or Y value must lie between the specified edges.

⚠ This command is a part of the CCOpt spine functionality, which is a limited-access feature in this release. It is enabled by a variable specified through the `setLimitedAccessFeature` command. To use this feature, contact your Cadence representative to explain your usage requirements, and make sure this feature meets your needs before deploying it widely.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_ccopt_clock_spine` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_ccopt_clock_spine`.
`-hard`	If specified, creates a hard clock spine. This creates a rectangle and it causes other cells to be pushed out of the spine region bounds to make way for the cells locked to the spine.
`-name`	Specifies the user-defined identifier for this clock spine. This parameter is required.
`-x` *x*	Specifies the X value of a vertical clock spine's target line.
`-xmax` *xmax*	Specifies the maximum X value of objects on a horizontal spine, and the right edge of the hard spine area.
`-xmin` *xmin*	Specifies the minimum X value of objects on a horizontal spine, and the left edge of the hard spine area.
`-y` *y*	Specifies the Y value of a horizontal clock spine's target line.
`-ymax` *ymax*	Specifies the maximum Y value of objects on a vertical spine, and the top edge of the hard spine area.
`-ymin` *ymin*	Specifies the minimum Y value of objects on a vertical spine, and the bottom edge of the hard spine area.

Example

- The following command creates a hard, horizontal spine, with the specified target Y value, and the four edges of the rectangle to be cleared:

  ```
  create_ccopt_clock_spine -name -hard spineB -y 2000.0 -xmin 0.0 -xmax 3000.0 -ymin 1500.0 -ymax 2500.0
  ```

  The software displays the following message:

  ```
  Created clock spine: spineB
  ```

Related Information

- `create_ccopt_clock_tree`
- `delete_ccopt_clock_spines`
- `get_ccopt_clock_spines`
- `get_ccopt_property`
- The following properties in the CCOpt Properties chapter of the *Innovus User Guide:*
  - `associated_row`
  - `bounds`
  - `center_line`
  - `hard`
  - `is_vertical`
  - `lock_on_clock_spine`
  - `legalized_on_clock_spine`

create_ccopt_clock_tree

```
create_ccopt_clock_tree
[-help]
[-name clockname]
[-no_skew_group]
-source pin
[-stop_at_sdc_clock_roots]
```

Defines a new clock tree within the design. You can define more than one clock tree and ordering of clock tree definitions is important. CCOpt can only optimize defined clock trees. When you run this command, CCOpt runs an algorithm known as clock tree extraction. Clock tree extraction examines the design starting at the root of a clock tree and decides which pins in the circuit are clock sinks for that tree and its clock gating and logical structure.

The `create_ccopt_clock_tree` command tells CCOpt that a part of the circuit is a clock tree. CCOpt then synthesizes the clock tree when you run the `ccopt_design` command. CCOpt treats clock trees and datapaths differently for power analysis purposes.

By default, defining a clock tree also creates associated skew groups.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_ccopt_clock_tree` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_ccopt_clock_tree`.
`-name clockname`	
	Specifies the user-defined identifier for the clock tree. This name should be unique. If this parameter is not specified, the software uses the name of the source pin by default.
`-no_skew_group`	
	Specifies that no default skew group will be defined for the sinks in the clock tree. If any skew groups are required, they will need to be defined manually using the `create_ccopt_skew_group` command.
`-source pin`	
	Specifies the name of the source pin of the clock tree in the design.
`-stop_at_sdc_clock_roots`	
	Specifies that clock tree definition should stop searching for parts of the clock tree through the root of SDC clocks.

Example

The following command creates a clock tree named "ck1" by tracking the clock path from the pin "clk":

```
create_ccopt_clock_tree -name ck1 -source clk
```

Related Information

- `ccopt_design`
- `clock_opt_design`
- `delete_ccopt_clock_trees`
- `get_ccopt_clock_trees`
- `report_ccopt_clock_trees`
- `update_ccopt_clock_tree`
- The following topic in the *Innovus User Guide*
  - Clock Tree Concepts and Specifications

create_ccopt_clock_tree_source_group

```
create_ccopt_clock_tree_source_group
[-help]
-clock_trees clock_tree_list
-name clock_tree_source_group_name
```

Creates a clock tree source group. These source groups are used when a clock tree has multiple equivalent roots, and you want CTS to automatically swap sinks between roots to reduce insertion delay. It is required for flows where we are connecting one logical clock tree to multiple tap-off points from a top level H-tree or a mesh.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_ccopt_clock_tree_source_group` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_ccopt_clock_tree_source_group`.
`-clock_trees` *clock_tree_list*	Specifies a Tcl list of clock tree names that should be grouped together. CTS will attempt to swap sinks between these clock trees in order to balance the insertion delay under each clock tree's root and may clone clock logic to help this. This parameter is required. **Note:** This Tcl list is the same as that defined previously using the `create_ccopt_clock_tree` command.
`-name` *clock_tree_source_group_name*	
	Specifies the user-defined name for the new clock tree source group. The name must be unique. This parameter is required.

Example

- The following command creates a clock tree source group containing three clock trees: clk1, clk2 and clk3:
  ```
  create_ccopt_clock_tree_source_group -name clk_grp -clock_trees {clk1 clk2 clk3}
  ```

Related Information

- `add_clock_tree_source_group_roots`
- `assign_clock_tree_source_groups`
- `create_ccopt_clock_tree_spec`
- `create_ccopt_clock_tree`
- `ccopt_design`
- `clock_opt_design`
- `delete_ccopt_clock_tree_source_group`
- `get_ccopt_clock_tree_source_groups`
- Clock Tree Synthesis chapter in the *Innovus User Guide*

create_ccopt_clock_tree_spec

```
create_ccopt_clock_tree_spec
[-help]
[-file filename]
[-keep_all_sdc_clocks]
[-views viewList]
```

Creates a clock tree network with associated skew groups and other clock tree synthesis (CTS) configuration settings such as ignore pins, case analysis, maxTrans, and so on based on a multi-mode timing configuration in the common timing engine (CTE).

When you run this command, one skew group will be created for each SDC clock in each constraint mode. Each such skew group will be able to take a skew target from any delay corner, but will have this target set to “ignore” unless there is an active analysis view that links the constraint mode for the skew group to that delay corner. The output of this command is a sequence of Innovus Tcl commands that can be edited manually.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_ccopt_clock_tree_spec` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man create_ccopt_clock_tree_spec`.

`-file` *`filename`*	Specifies the name for the clock tree specification script file. Providing the filename writes out the specified file. The file is not executed. To execute the file, run the following:
	`create_ccopt_clock_tree_spec -file spec.tcl`
	`source spec.tcl`
	Note: You can execute the file only once. If you run the above again, the software gives an error message:
	`source spec.tcl`
	`Cannot run clock tree spec: clock trees are already defined.`
	The already loaded specification file can be removed by using either of the following commands:
	• `delete_ccopt_clock_tree_spec`
	• `reset_ccopt_config`
	Note: If this parameter is not specified, the clock trees are defined by directly running clock tree extraction without the creation of a clock tree extraction script.

`-keep_all_sdc_clocks`	
	Specifies that all clocks and clock trees should be kept in 1:1 ratio, wherever possible. This means that CCOpt will maintain generated clock names for clock trees, even where these are functionally redundant, for example, clocks defined on buffer outputs. By default, CCOpt will not define these redundant clock trees.
`-views` *`viewList`*	Specifies the Tcl list of analysis view names. By default, the software considers all active analysis views.

Example

- The following example shows how running `create_ccopt_clock_tree_spec` defines clock trees by directly running clock tree extraction instead of creating a clock tree extraction script:

```
get_ccopt_clock_trees *
create_ccopt_clock_tree_spec
get_ccopt_clock_trees *
m_clk m_digit_clk m_dsram_clk m_ram_clk m_rcc_clk m_spi_clk refclk<1> refclk
```

Related Information

- `ccopt_design`
- `clock_opt_design`
- `create_ccopt_clock_tree`
- `delete_ccopt_clock_tree_spec`
- `reset_ccopt_config`
- Clock Tree Synthesis chapter in the *Innovus User Guide*

create_ccopt_flexible_htree

```
create_ccopt_flexible_htree
[-help]
[-adjust_sink_grid_for_aspect_ratio {true | false}]
-final_cell base_cell
[-hv_balance {true | false}]
[-image_directory dirName]
[-inverting]
[-layer_density value]
[-max_driver_distance value]
[-max_root_distance value]
[-mode {drv | distance}]
-name flexibleHtreeName
[-omit_symmetry string]
[-partition_boundary_polarity {non_inverting | inverting | ignore}]
[-partition_groups {{partition ...} [max_boundary_net_length] [inverting | non_inverting] [subtree] ...}]
[-power_weight value]
-pin {pin | port}
[-sink_grid {columns rows}]
[-sink_grid_box {xmin ymin xmax ymax}]
[-sink_grid_exclusion_zones {{xmin ymin xmax ymax} ...}sink_grid_exclusion_zones]
[-sink_instance_prefix prefixName]
[-sink_grid_sink_area {width height}]
[-sinks {{pin_name | {xmin ymin xmax ymax}} ...}]
[-stop_at_sdc_clock_roots]
-trunk_cell base_cell
```

Creates a design object for a flexible H-tree, under the specified pin. The H-tree is adjusted to consider placement and routing blockages, and any power routing.

You can specify this command in a sequence to create multiple flexible H-trees.

Note: This command does not synthesize flexible H-trees. After creating or defining flexible H-trees using this command, use the `synthesize_ccopt_flexible_htrees` command to synthesize all the flexible H-trees created using this command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_ccopt_flexible_htree` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_ccopt_flexible_htree`.
`-adjust_sink_grid_for_aspect_ratio {true | false}`	
	When set to true, adjusts the sink grid for aspect ratio. *Default*: `true`
`-image_directory dirName`	

	Specifies the name of the directory to which the images generated by the H-tree synthesis algorithm are written. The images are PNG files. Color coding of images is provided below: White: Unobstructed edges of the synthesis grid Red: Grid points that are blocked for trunk cell placement in all modules Orange: Grid points that are blocked for final cell placement in all modules Red orange: Grid points that are blocked for trunk and final cell placement in all modules Yellow circle: The grid point of the source Yellow points: Candidate grid points of H-tree sinks Yellow rectangle: The sink area containing target grid point candidates of H-sinks, adjusted to the synthesis grid Brown: If specified, the sink grid box snapped to the synthesis grid Green/blue: The edges of the synthesized H-tree Purple: H-tree repeaters Cyan: Partition boundaries in hierarchical designs
`-final_cell` *`base_cell`*	
	Specifies the library cell (buffer or inverter) to use for the ends of the clock tree.
`-hv_balance {true | false}`	
	When set to `true`, specifies that the horizontal and vertical metal wires can only be balanced against other wires of the same orientation. When set to `false`, specifies that any wire can be balanced against any other wire. Default: `true`
`-inverting`	
	Specifies whether the tree will invert its input or not. By default, this parameter is not enabled.
`-layer_density` *`value`*	
	Specifies the layer density used to compute the parasitics for timing estimates of H-tree nets.
`-max_driver_distance` *`value`*	
	When specified, the software ignores DRVs and uses the given value as the maximum length of the nets connecting the H-tree drivers.
`-max_root_distance` *`value`*	
	When specified, the software ignores DRVs and uses the given value as the maximum length of the net connecting the root and the first driver of the H-tree. This value can only be specified if the `-max_driver_distance` parameter is also specified.
`-mode {drv | distance}`	

	Specifies the driver insertion mode. When set to `drv`, drivers are inserted to avoid DRVs. When set to `distance`, the parameters, `-max_driver_distance` and `-max_root_distance` determine the maximum net lengths allowed. Transitions and delays are not computed in this case. *Default*: `drv` **Note**: When this parameter is set to `distance`, the `-max_driver_distance` and `-max_root_distance` parameters must be specified.
`-name` *flexibleHtreeName*	
	Specifies the name of the flexible H-tree to be created. This parameter is required.
`-omit_symmetry` *string*	
	Specifies that no symmetry drivers and/or branches should be added to balance the pin load of nets in the flexible H-tree. Symmetry drivers are drivers that are added to balance pin capacitances at branch points. For example, if buffer pairs are inserted at branch points, one of these buffers may not drive any fanout and is inserted to match other buffer pairs at the same level of the flexible H-tree. Omitting symmetry drivers can reduce the power of the H-tree but increase its skew. Drivers that are added to terminate symmetry branches do not count as symmetry drivers and are never omitted. Similarly, symmetry branches are branches that are needed to balance the wire load at branch points. They are added to match other branchpoints at the same level of the flexible H-tree. Symmetry branches are always terminated with drivers, which are not counted as symmetry drivers. Possible values for this parameter are: `false`: Add symmetry branches and drivers `true`: Omit both symmetry drivers and branches `drivers`: Omit symmetry drivers `branches`: Omit symmetry branches `{drivers branches}`: Omit both symmetry drivers and branches `{}`: Add symmetry branches and drivers By default, symmetry branches and buffers are added.
`-partition_boundary_polarity {non_inverting | inverting | ignore}`	
	Specifies the polarity of the clock signal with regard to the source of the flexible H-tree when entering partitions. The following values are allowed: • `non_inverting`: When specified, allows only non-inverting polarity at partition entry points. • `inverting`: When specified, allows only inverting polarity at partition entry points. • `ignore`: When specified, allows inverting and non-inverting polarity at partition entry points. *Default*: `non_inverting`
`-partition_groups {{`*partition* `...} [max_boundary_net_length] [inverting | non_inverting] [`*subtree*`] ...}`	

	Specifies the groups in which partitions are clustered in channelless designs. Nested lists imply allowed crossings between groups. Each group has zero or one input port and each partition must only be specified once. Optionally, a maximum pre-route net length from the boundary of a partition group can be specified. The maximum pre-route net length must be specified as the second parameter of a nested partition group. Optionally, the next argument specifies the clock phase when entering the partition group in relation to the root pin of the H-tree. Allowed values are 'inverting' and 'non_inverting'. If no value is specified, the clock phase is unconstrained. The clock phase is unconstrained when crossing the boundaries of partitions within the same group. All specified clock phases must be either inverting or non_inverting. For example, if the following partitions are specified: `{{A} {{C D} 50 non_inverting {{E F}} {{G}}}}` It means the following: • The H-tree starts in partition A and descends into group C/D. • From partition group C/D, the tree descends into groups E/F and G. • Any clustering of sinks inside the C/D and E/F groups is allowed, potentially crossing internal partition boundaries several times. • Partition A has no clock input port and one clock output port • Partition group C/D has one clock input port and two clock output ports • Partition group E/F and G have one clock input port and no clock output port • The maximum net length from the entry point into partition group C/D is 50um (pre-route) • The clock phase is non_inverting when entering parition group C/D. The clock phase is unconstrained when entering other partition groups Default: `{}`
`-power_weight` *value*	
	Specifies the power versus insertion delay trade-off. Valid values are between 0 and 1, specifying the weight that is put on power optimization during the synthesis of flexible H-trees. *Default*: `1`
`-pin {pin | port}`	
	Specifies the pin under which to create the flexible H-tree. This parameter is required.
`-sink_grid {columns rows}`	
	Specifies the columns and rows of a grid of H-tree sinks. The columns and rows may be swapped if the `-adjust_sink_grid_for_aspect_ratio` parameter is set to `true`. **Note**: This parameter is not supported in master/clone designs.
`-sink_grid_box {xmin ymin xmax ymax}`	
	Specifies the area of the box that the grid of H-trees should cover.
`-sink_grid_exclusion_zones {{`*xmin ymin xmax ymax*`} ...}`	
	Specifies zones from which sinks of the sink grid are excluded if the zones completely cover the respective sink area.
`-sink_grid_sink_area {width height}`	

	Specifies the approximate size of the rectangle describing valid locations for final cells, specified using the `-final_cell` parameter, per H-tree sink in the grid.
`-sink_instance_prefix` *prefixName*	
	Specifies the prefix used for instance names of final cells, specified using the `-final_cell` parameter and the sinks specified using the `-sinks` parameter.
`-sinks {`*pin_name* `|` `{` *xmin ymin xmax ymax* `}}`	
	Specifies approximate rectangular sink regions for the buffer locations at the end of the tree, which are specified using the `-final_cell` parameter or pins. The specified pins must be in the clock tree. Nested lists imply clustering relationships.
`-stop_at_sdc_clock_roots`	
	Specifies that the clock tree definition should stop searching for parts of the clock tree at the root of SDC clocks.
`-trunk_cell` *base_cell*	
	Specifies that this cell should be used for the buffers inside the flexible H-tree. This parameter is required.

Examples

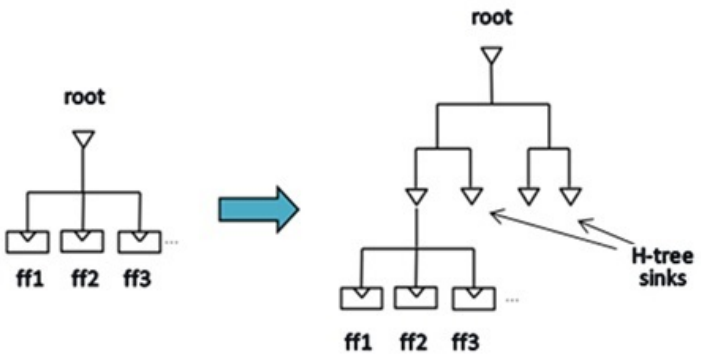
- The following command creates a flexible H-tree for four sinks under the the pin "`root/Y`". The sinks of the H-tree will be of type `BUFX16` and placed approximately inside the specified sink rectangles. Other buffers added to the H-tree will be of type `BUFX8`. No symmetry buffers are added to balance the pin load of nets. The flexible H-tree is then synthesized using the `synthesize_ccopt_flexible_htrees` command.

  ```
  create_ccopt_flexible_htree -name tree0 -pin root/Y -sinks { {200 200 300 300} {200 700 300 800} {700 200 800 300} {700 700 800 800} } -final_cell BUFX16 -trunk_cell BUFX8 -omit_symmetry drivers
  synthesize_ccopt_flexible_htrees
  ```

- The following commands create two flexible H-trees under the pins "`rootA/Y`" and "`rootB/Y`". The sinks will be of the type `INVX8` and will be placed in the specified sink grids. Other buffers added to the H-trees will be of type `INVX12`. Symmetry buffers are omitted to balance the pin load of nets. The flexible H-trees are then synthesized using the `synthesize_ccopt_flexible_htrees` command.

  ```
  create_ccopt_flexible_htree -name htreeA -pin rootA/Y -sink_grid {2 2} -final_cell INVX8 -trunk_cell INVX12 -omit_symmetry drivers
  create_ccopt_flexible_htree -name htreeB -pin rootB/Y -sink_grid {4 4} -final_cell INVX8 -trunk_cell INVX12 -omit_symmetry drivers
  synthesize_ccopt_flexible_htrees
  ```

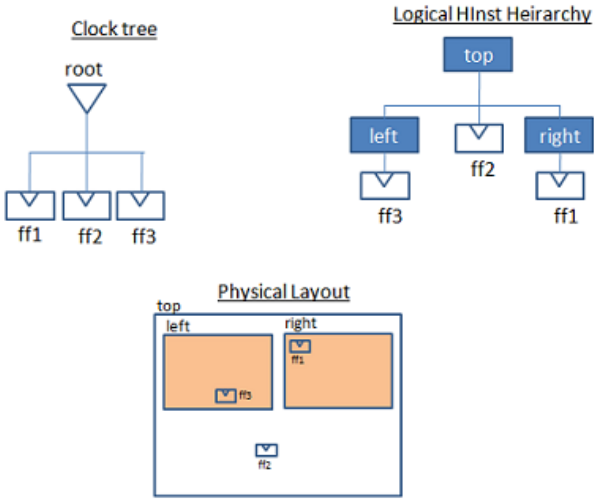
Note: During flexible H-tree synthesis a number of buffers (or inverters) are created. In designs with no fence regions, every instance in the flexible H-tree, including all H-tree sinks, are assigned to the top-level hInst. In this case, the `create_ccopt_flexible_htree` command moves the original fanout of the root pin of the flexible H-tree to a single H-tree sink as shown in the image below.



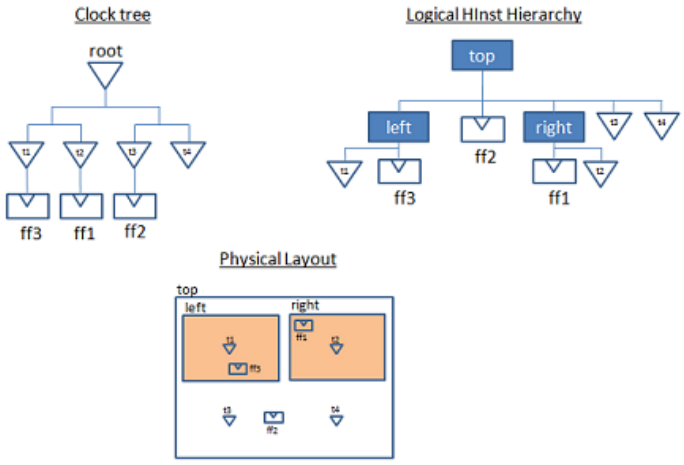
In designs where there are fence regions, the command chooses hInsts for instances in the flexible H-tree considering fence region constraints. The fanout of the root pin of the flexible H-tree is connected to H-tree sinks that best match the hInst to which a fanout instance belongs.

The images below illustrate a clock tree with three clock tree sinks, ff1, ff2, and ff3 and three hInsts, "top", "left", and "right". The hInsts "left" and "right" are fenced. Fenced regions are shown in pale orange. Sink ff1 is in hInst "right", ff2 is in the top-level hInst, and ff3 is in hInst "left". The hInsts "left" and "right" are children of the top-level hInst. After flexible H-tree synthesis there are four H-tree sink instances, t1, t2, t3, and t4 laid out in a square such that t1 is in the fence region for the hInst "left", t2 is in the fence region for the hInst "right", and the other two are in hInst "top". The flops have been connected to H-tree sinks that match their fence constraints. Note that when running `ccopt_design` after creating the flexible H-tree, multi-tap CTS may reconnect ff2 to t4 rather than t3.

Before H-tree Synthesis



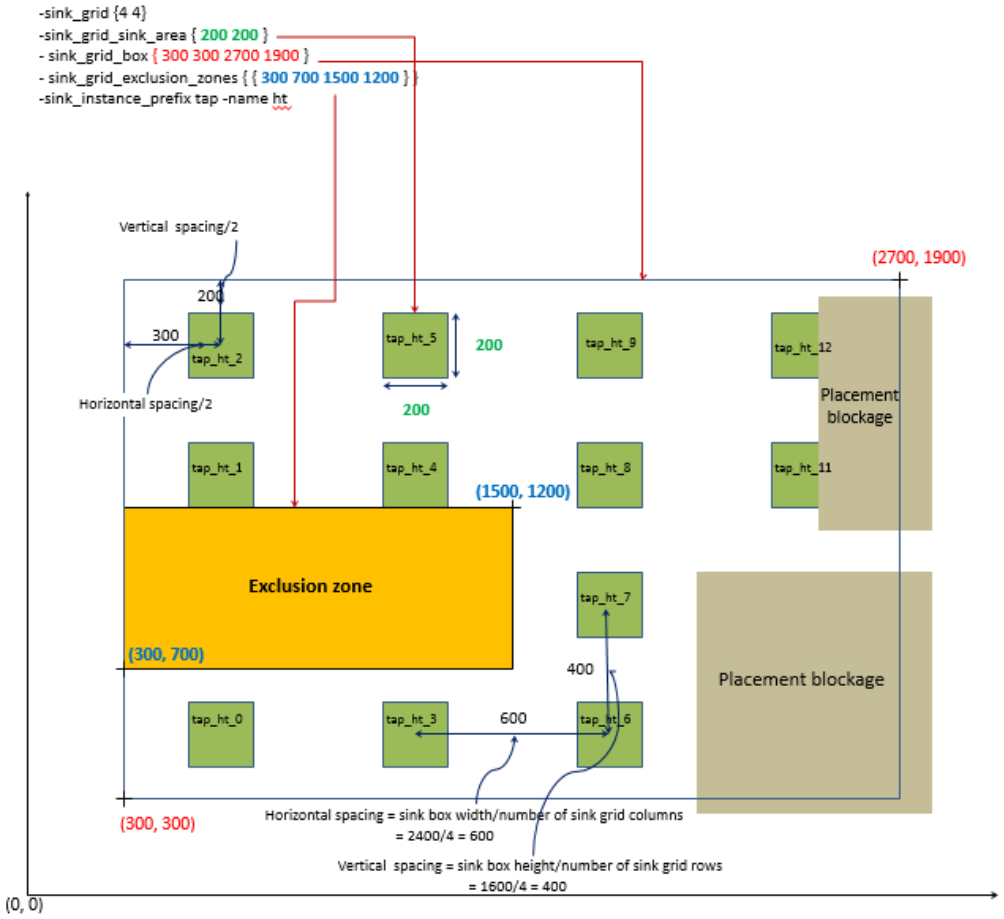
After H-tree Synthesis



Using the Sink Grid Parameters

The diagram below illustrates the usage of the sink grid parameters of this command. Note the following:

- The sink grid is specified as {4 4}, which means it will have four columns and four rows
- No H-tree sinks are created for sink areas that are completely covered by placement blockages or exclusions zones
- H-tree sink instances with the name specified in the diagram will be placed inside the unblocked parts of the sink areas
- The sink grid is adjusted by snapping it to the synthesis grid, which means that the specified sink areas are treated only as a guide



Related Properties

You can also set the following properties on the flexible H-tree design objects by using the `set_ccopt_property` command:

- `adjust_sink_grid_for_aspect_ratio`
- `final_cell`
- `flexible_htree`
- `flexible_htree_placement_legalization_effort`
- `htree_sinks`
- `hv_balance`
- `image_directory`
- `inverting`
- `layer_density`
- `mode`
- `omit_symmetry`

- `partition_boundary_polarity`
- `pin`
- `power_weight`
- `sink_grid`
- `sink_grid_sink_area`
- `sink_grid_box`
- `sink_grid_exclusion_zones`
- `sink_instance_prefix`
- `stop_at_sdc_clock_roots`
- `trunk_cell`

For detailed information about all public CCOpt properties, see the CCOpt Properties chapter in the *Innovus User Guide*.

You can also use the following command:

`get_ccopt_property -help` *`propertyName`*

Related Information

- `ccopt_design`
- `clock_opt_design`
- `delete_ccopt_flexible_htrees`
- `get_ccopt_flexible_htrees`
- `get_ccopt_property`
- `synthesize_ccopt_flexible_htrees`
- `set_ccopt_property`
- The following topic in the *Innovus User Guide*
  - Flexible H-Tree and Multi-Tap Clock Flow
- For details of the use model for this feature, see the Flexible H-tree and Multi-Tap Clock Flow in Innovus Application Note on the Cadence Online Support website.

create_ccopt_generated_clock_tree

```
create_ccopt_generated_clock_tree
[-help]
[-generated_by pins]
[-name clockname]
[-parents parents]
[-search_method method]
-source pin
[-stop_at_sdc_clock_roots]
```

Defines a generated clock tree within the design. Use this command if you want to balance the sinks in the generated tree against those in the parent trees. CCOpt accomplishes this action by pulling up any sinks in the parent trees that contribute directly to the generated clock tree. The insertion delay under such sinks in the parent trees will be set to the insertion delay of the generated clock tree plus the delay between the sink pin and the source of the generated clock tree.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_ccopt_generated_clock_tree` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_ccopt_generated_clock_tree`.
`-generated_by pins`	Specifies the sink pins in the parent clock trees for which the delay between the parent and generated clock trees should be considered. If this parameter is not specified, all possible timing paths between sinks in the parent tree and the source of the generated clock tree will be considered. This parameter is mutually exclusive with the `-search_method` parameter.
`-name clockname`	Specifies the user-defined identifier for the generated clock tree. This name should be unique. If this parameter is not specified, the software uses the name of the source pin by default.
`-parents parents`	Specifies the parent clock trees for the generated clock trees. This parameter should be specified if the parents are ambiguous.

`-search_method {same_instance multi_stage}`	Specifies the search methdology to be used when tracing the path to the generator inputs.
	`multi_stage`
	This is the default method, and will select the current behavior of the software. When this method is specified, the software searches backwards through all the delay paths leading into the clock root pin, looking for pre-existing clock sinks. Every such clock sink found is a candidate generator input for the generated clock tree. This includes clock sinks that lie on different instances to the root pin - where multiple stages lie between the clock sink and the root pin. Additional filtering on the candidate generator inputs is applied by the `-parents` parameter to avoid producing cycles in the clock tree structure. Once the set of generator inputs is finalized, the delay path between each finalized generator input and the clock root pin is imported into the CCOpt clock tree data stuctures. If the generator input lies on a different instance of the clock root pin, ClockGeneratorPathNode objects are created.
	`same_instance`
	This method is similar to the `multi_stage` method, except that the software restricts its search to consider only clock sinks that lie on the same instance as the generated clock tree's root pin. This means that the software does not traverse net arcs looking for generator input clock sinks. Consequently, ClockGeneratorPathNode objects will not be needed to model any generator path.
	Note:
	• This functionally is identical to specifying the `-generated_by` parameter that lists all the clock sinks on the same instance as the clock root pin. However, it avoids the need to explicitly identify those clock sinks.
	• Irrespective of the search method, if no candidate generator clock sink is found, or if there are no valid generator inputs discovered in the set of clock sinks, creation of the generated clock tree fails with an error.
	The software uses `multi_stage` method by default.
	This parameter is mutually exclusive with the `-generated_by` parameter.

`-source pin`	Specifies the name of the source pin of the generated clock tree. This is a required parameter.
`-stop_at_sdc_clock_roots`	Specifies that clock tree definition should stop searching for parts of the clock tree through the root of the SDC clocks.

Example

- The following command defines a generated clock tree `clk_div_2`, which is generated from the `gen_clk/CK` clock pin, using the `gen_clk.QN` pin as the source pin:
```
create_ccopt_generated_clock_tree -name clk_div_2 -source gen_clk/QN -generated_by gen_clk/CK
```

Related Information

- `ccopt_design`
- `clock_opt_design`
- `create_ccopt_clock_tree`
- `create_ccopt_clock_tree_spec`
- The following topics in the *Innovus User Guide*
  - Automatic Clock Tree Specification Creation
  - Manual Setup and Adjustment of the Clock Tree Specification

create_ccopt_macro_model_spec

```
create_ccopt_macro_model_spec
[-help]
[-ckSpec]
[-ediCtsSpecForMacroModels fileName]
[-file filename]
```

This command is used to view, or edit, the pin insertion delays for CCOpt derived from the macro model specification. The macro model information for CCOpt can be retrieved by providing the name of the specification file that contains the macro model information .This command allows the macro model to CCOpt pin insertion delay conversion in a way that gives you the pin insertion delay in a TCL format that you can edit and then integrate into your scripts.

Note: To run this command, ensure that the design is loaded and the specification file is available. Once the macro model specification file is created, to use this file, you only need to source it after the running the `create_ccopt_clock_tree_spec` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_ccopt_macro_model_spec` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_ccopt_macro_model_spec`.
`-ckSpec`	Uses the CCOpt CTS specification file for macro model information.
`-ediCtsSpecForMacroModels fileName`	
	Specifies the legacy FE-CTS specification file for macro model information.
`-file filename`	Exports the CCOpt macro model spec to the specified file.

Example

- The following command is used to create the macro model specification file, CCOPT_MM, using the legacy FE-CTS specification file, `CTSSPEC`, in the CCOpt-CTS and CCOpt flow:

  ```
  create_ccopt_macro_model_spec -ediCtsSpecForMacroModels CTSSPEC -file CCOPT_MM
  ```

- The following command is used to create the macro model specification file, CCOPT_MM, in the CCOpt-CTS flow driven directly by the legacy FE-CTS specification file (`CTSSPEC`):

  ```
  create_ccopt_macro_model_spec -ediCtsSpecForMacroModels CTSSPEC -file CCOPT_MM -ckSpec
  ```

Related Commands

- `ccopt_design`
- `clock_opt_design`
- `create_ccopt_clock_tree_spec`

create_ccopt_preferred_cell_stripe

```
create_ccopt_preferred_cell_stripe
[-help]
-bbox {xmin xmax ymin ymax}
-cells list_of_cells
-name name
```

This command creates an object that corresponds to a single preferred_cell_stripe. The object represents a constraint where CCOpt tries to place instances of the specified cells within the specified bounding box of the stripe. When several of these objects are created, CTS tries to place instances within the bounding boxes of the stripes, choosing the stripe that yields the best quality of result.

The object must be created before running `ccopt_design`. This command is used when you want to place large CTS cells, which induce a large IR drop, close to the power switches that are often arranged in repeated vertical columns on the chip.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_ccopt_preferred_cell_stripe` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_ccopt_preferred_cell_stripe`.
`-bbox bbox`	
	Specifies the area of the bounding box. Cell instances within this box will be preferentially placed on the stripes, if their library cell appears in the cell list associated with the stripes instance.
`-cells list_of_cells`	
	Specifies the list of library cells that should be placed on preferred cell stripes.
`-name name`	
	Specifies the user-defined identifier for these preferred cell stripes.

Related Information

- `ccopt_design`
- `delete_ccopt_preferred_cell_stripe`
- `get_ccopt_preferred_cell_stripe`
- The following properties are related to the cell preferred stripes feature. For detailed information about these properties, see the CCOpt Properties chapter in the *Innovus User Guide:*
  - `stripe_bbox`
  - `stripe_cells`

create_ccopt_skew_group

```
create_ccopt_skew_group
[-help]
[-constrains cts | ccopt_initial | ccopt | subset_of_values | all | none]
[-from_clocks clock_names]
[-from_constraint_modes constraint_mode_names]
[-from_delay_corners delay_corner_names]
-name skew_group_name
[-rank rank]
[-sinks pins | -shared_sinks pins | -exclusive_sinks pins | -auto_sinks | -filtered_auto_sinks pins | -
balance_skew_groups skew_groups]
[-sources pins | -balance_skew_groups skew_groups]
[-target_insertion_delay value]
[-target_skew value]
```

Creates a new skew group within the design. CCOpt balances clock trees according to skew group requirements. Balancing between multiple trees can be achieved using the `-sources` parameter. The sinks to be balanced can either be specified explicitly using the `-shared_sinks` or `-exclusive_sinks` parameters or can be automatically inferred by using the `-auto_sinks` parameter.

Every skew group is assigned a rank, accessible through the `exclusive_sinks_rank` property:

- **Shared skew groups** always have an `exclusive_sinks_rank` value of zero.
- **Exclusive skew groups** always have an `exclusive_sinks_rank` value greater than zero. When an exclusive skew group is created, CCOpt assigns that exclusive skew group a rank one greater than the highest existing skew group.

The rank of a skew group determines whether a member pin is an active sink in that skew group or not. A pin is only an active sink in the skew group(s) with the highest rank, out of all the skew groups to which the pin belongs. An active sink is a pin that will be balanced against other active sinks in the same skew group.

For example, consider the following sequence of commands:

1. `create_ccopt_skew_group -name SG1 -sources top -shared_sinks [get_ccopt_clock_tree_sinks */CK]`
2. `create_ccopt_skew_group -name SG2 -sources top -exclusive_sinks [get_ccopt_clock_tree_sinks *XYZ*/CK]`
3. `create_ccopt_skew_group -name SG3 -sources top -exclusive_sinks [get_ccopt_clock_tree_sinks *XYZ_01*/CK]`

After running the first command, a single skew group `SG1` is created. This is a shared skew group, so has an `exclusive_sinks_rank` value of zero. All the "`CK`" pins in the design are members of this skew group. In addition, all the "`CK`" pins are active sinks in skew group `SG1`. This is because `SG1` is the highest ranked skew group so far, even though it has a rank of zero.

The second command defines an exclusive skew group `SG2`. This is given an `exclusive_sinks_rank` value of 1, which is one higher than the current highest rank of 0. Sinks that match the pattern `*XYZ*/CK` are now members of both `SG1` and `SG2`. However, they are only active sinks in `SG2`, which is the highest ranked parent skew group till now. Sinks that don't match this pattern remain active pins in `SG1`.

The third command defines another exclusive skew group, `SG3`. This is given an `exclusive_sinks_rank` value of 2, which is one higher than the current highest rank of 1. Sinks that match the pattern `*XYZ_01*/CK` (which must also be members of `SG2`, since those sinks would also match the pattern `*XYZ*/CK`) are now members of skew groups `SG1`, `SG2`, and `SG3`. However, they are only active sinks in `SG3`, which is the highest ranked parent skew group. Sinks that matched the pattern `*XYZ*/CK` but not `*XYZ_01*/CK` are members of both `SG1` and `SG2` but only active sinks of `SG2`. Sinks that don't match the pattern `*XYZ*/CK` are members of `SG1` and active sinks in `SG1`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_ccopt_skew_group` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_ccopt_skew_group`.
`-auto_sinks`	When specified, determines the target sinks for the skew group based on the input pins reachable from the source.
`-balance_skew_groups` *skew_groups*	
	Specifies that the new skew groups should be created by merging the specified skew groups in order to balance them together as a single skew group. **Note**: The sources of skew groups are specified using the `-sources` parameter. However, if the `-sources` parameter is not specified and you are using the `-balance_skew_groups` parameter, then the sources are the sources of the skew groups that are specified using this parameter.
`-constrains cts | ccopt_initial | ccopt | subset_of_values | all | none`	
	Specifies whether the skew group should constrain `cts, ccopt_initial, ccopt, a subset of these values, all`, or `none`. If the value includes cts, the created skew group will constrain `cts`, if the value includes `ccopt_initial`, the created skew group will be used to constrain CCOpt's initial solution.If the value includes `ccopt`, the created skew group will constrain CCOpt's final solution.
`-exclusive_sinks` *pins*	
	Specifies the input pins in the design that CCOpt should consider as members of the new skew group. This will give the new skew_group an `exclusive_sinks_rank` 1 greater than any other currently defined skew group, effectively removing the pins from all skew_groups that they currently belong to. If both "`shared_sinks`" and "`exclusive_sinks`" are not specified, CCOpt will use the set of sinks reachable from the source if the `auto_sinks` switch is provided, or no sinks otherwise.
`-filtered_auto_sinks` *pins*	
	Specifies that the target sinks should be determined automatically from the subset provided. The target sinks for the skew group are determined based on the input pins reachable from the source. However, of these pins, only those are selected that are also present in the list of pins provided as a filter.
`-from_clocks` *clock_names*	
	Specifies the name of the clock for which this skew group was extracted.
`-from_constraint_modes` *constraint_mode_names*	
	Specifies the constraint mode for which this skew group was extracted.
`-from_delay_corners` *delay_corner_names*	
	Specifies the delay corners associated with the constraint modes for which this skew group was extracted.
`-name` *skew_group_name*	
	Specifies the user-defined name for the new skew group. The name must be unique.
`-rank` *rank*	Specifies the exclusive_sinks_rank for this skew group.
`-shared_sinks` *pins*	

	Specifies the input pins in the design that CCOpt should consider as members of the new skew group but are also shared with other skew groups. These pins are members of both the original skew group and the new skew group. This implies a `exclusive_sinks_rank` of 0.
`-sinks` *pins*	Specifies the input pins in the design that CCOpt should consider as members of the new skew group.
`-sources` *pins*	Specifies the source pins of the skew group.
`-target_insertion_delay` *value*	
	Specifies the target insertion delay to be used for the skew group. CCOpt attempts to keep the latency of all sinks within half the target skew on either side of this delay. For example, `create_ccopt_skew_group -name b1 -sources b1 -exclusive_sinks {A/CK B/CK G/CK} -target_insertion_delay 0.3ns -target_skew 0.1ns`
`-target_skew` *value*	

	Specifies the skew that the clock tree synthesis should aim for. It sets the `target_skew` property for the created skew group (`set_ccopt_property target_skew` *value*). If this parameter is not set, the software will use the default value that it gets from `get_ccopt_property target_skew`.
	For example:
	`get_ccopt_property target_skew`
	The software returns the following value:
	`2`

Example

- The following command creates the specified skew group, `clk_disable`, for the specified source and exclusive sinks:

```
create_ccopt_skew_group -name clk_disable -sources clk -exclusive_sinks {mod/ff1/CK mod/icg/CK}
```

Related Information

- `get_ccopt_property`
- `get_ccopt_skew_groups`
- `get_ccopt_skew_group_delay`
- `get_ccopt_skew_group_path`
- `delete_ccopt_skew_groups`
- `modify_ccopt_skew_group`
- `report_ccopt_skew_groups`
- `set_ccopt_property`
- The following topics in the *Innovus User Guide*

- Clock Trees and Skew Groups
- Automatic Clock Tree Specification Creation
- Manual Setup and Adjustment of the Clock Tree Specification

create_route_type

```
create_route_type
[-help]
[-em_ndr_dist float]
[-em_ndr_rule rule_name]
[-min_stack_layer layer]
-name string
[-one_side_spacing_range bottomLayerNum:topLayerNum]
[-prefer_multi_cut_via]
[-shield_side {one_side | both_side}]
[-shield_tap_instance_insertion_effort {none | standard | high}]
[-stack_distance float]
[[-non_default_rule ndr_name] [-shield_net net_name] [-top_preferred_layer layer] [-bottom_preferred_layer layer]
[-preferred_routing_layer_effort {low | medium | high}]]
[-mask mask_number [-layer_mask_range bottomLayerNum:topLayerNum]]
```

Creates a new route type and sets the routing properties for the nets. This command lets you create the standard `route_type` that defines a collection of routing attributes such as non-default rule (NDR), shield net, preferred layers and so on that can then be applied to either all leaf or all trunk nets for a particular clock tree. The type of route_type object created depends on the parameters you specify with this command. A standard route_type object is created if you specify the `-non_default_rule, -shield_net, -top_preferred_layer` or `-preferred_routing_layer` parameters.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_route_type` parameter. For a detailed description of the command and all of its parameters, use the man command: man `create_route_type`
`-bottom_preferred_layer` *`layer`*	Specifies the preferred lowest routing layer. This attribute is a soft limit, which means, NanoRoute might use a layer below the specified layer if necessary to complete routing. A layer name can be any real layer name, for example, from LEF or OpenAccess. Routing layer names can also be abbreviated as 1 or metal1 for the first routing layer, 2 or metal2 for the second routing layer, and so on. Note, the software will check for real layer names first. So, if metal1 matches a real layer name it might not be the first routing layer. If not specified, the router will freely route the net starting on the first routing layer, with no preference for higher layers.
`-em_ndr_dist` *`float`*	Specifies the distance related to the output pin, when `em_ndr_rule` is applied to the net. When routing outside this distance range, the router either uses regular wire or regular non-default routing (NDR), if the regular NDR setting is enabled in this net. **Note:** When the `-em_ndr_rule` option is used with the command, the `-em_ndr_rule` option becomes mandatory.
`-em_ndr_rule` *`rule_name`*	Specifies the EM NDR rule to associate with this route type. When routing within the distance specified in `-em_dist` from output pin, router will use this ndr rule to route. *Default:* No EM NDR rule
`-layer_mask_range` *`bottomLayerNum:topLayerNum`*	Specifies the layer range that the mask constrain should be applied on. For example, 3:4 means that the constrain should be applied on metal3 and metal4.
`-mask mask_number`	Specifies the mask number in allowed range (min=0, max=3).
`-min_stack_layer` *`layer`*	Specifies that the output pins of the net use stacked vias from the pin to the given *`layer`* and the rest of the connections start from the mentioned *`layer`*.
`-name` *`string`*	Specifies the name of the route type object. This is required.
`-non_default_rule` *`ndr_name`*	Specifies the non-default routing (NDR) rule to associate with this route_type. Default is no NDR rule.
`-one_side_spacing_range` *`bottomLayerNum:topLayerNum`*	Specifies the layer range that one side spacing constrain should be applied on. By default, NDR spacing is applied to both sides of an NDR net or wire. Use this option to specify the layer range for wires that have only one neighboring wire with minimum spacing, which means only one side needs to follow larger NDR spacing.
`-prefer_multi_cut_via`	Specifies whether or not to use the multi-cut vias for connections at the net driver pins.

`-preferred_routing_layer_effort {low | medium | high}`	Specifies whether the preferred routing layer effort will be low, medium or high. This parameter determines how flexible the NanoRoute router is when setting layer limits for routing specified nets. Use this parameter with the `-top_preferred_routing_layer` and `-bottom_preferred_routing_layer` parameters. Specify `medium` or `high` routing layer effort level to make NanoRoute less flexible in setting layer limits for routing specified nets. NanoRoute will obey the preferred routing layer range more strictly. Specify `low` to make NanoRoute more flexible in setting layer limits for routing specified nets. *Default*: `high`
`-shield_net` *net_name*	Specifies a special net to use as a shield for a critical or high-speed net. Typically, a shielded net is routed before routing other net. You can specify only one shield net. However, you cannot control which side is used by the special net that is routed on the same layer as the signal routing. The shield terminates near the pin. If not specified, the net is not shielded.
`-shield_side {one_side | both_side}`	Specifies whether to perform one sided or two sided shielding for the route type specified.
`-shield_tap_instance_insertion_effort {none | standard | high}`	Specifies the clock route type for clock shielding of the high and low frequencies through the `high` and `standard` values respectively. The main difference between these values is related to the ground that the tie cell instance/via uses to achieve shield segments of the electrical connection.
`-stack_distance` *float*	Specifies that the cut distance of cuts on adjacent layers in the stacked vias are defined in `-min_stack_layer`.
`-top_preferred_layer` *layer*	Specifies the top-preferred layer for the route type.

Example

For example, use the following commands to create the standard route_types that specify the routing attributes we want to assign to particular nets:

```
create_route_type -name RT1 -non_default_rule NDR1
create_route_type -name RT2 -non_default_rule NDR2
create_route_type -name RT3 -non_default_rule NDR3 -top_preferred_layer M4 -bottom_preferred_layer M3
create_route_type -name RT4 -non_default_rule NDR4 -preferred_routing_layer_effort high -top_preferred_layer M4 -
bottom_preferred_layer M3
```

cts_refine_clock_tree_placement

```
cts_refine_clock_tree_placement
[-help]
```

Performs clock placement refinement with cell halos. Use this command at the end of CTS to ensure that all clock cells are placed legally and obey cell halo and density constraints. You can use `report_ccopt_cell_halo_violations` to report any clock halo constraint violations. Then run `cts_refine_clock_tree_placement` to fix any reported violations. You can re-run the `report_ccopt_cell_halo_violations` command to verify that these instances have been fixed.

Parameter

`-help`	Outputs a brief description for the `cts_refine_clock_tree_placement` command parameter. For a detailed description of the command, use the man command: `man cts_refine_clock_tree_placement`.

Example

The following set of commands show that halo violations are reported with `report_ccopt_cell_halo_violations` command, then the `cts_refine_clock_tree_placement` command performs clock placement refinement with cell halos and then `report_ccopt_cell_halo_violations` is re-run to verify that the instances have been fixed:

```
report_ccopt_cell_halo_violations

Clock Cell Halo Rule Check in Summary
=====================================

-----------------------------------------------------
Clock Instances with cell_halo Failed instances
-----------------------------------------------------
clock 6 3
-----------------------------------------------------

Total number of violating instances: 3

Clock Cell Halo Rule Violations
===============================

-------------------------------------------------------------------------------------------------------
No. Cell Instance In clock Cell Halo (x, y) Violating instance From clock Intrusion (x, y)
-------------------------------------------------------------------------------------------------------
1 CLKBUFX12 CTS_ccl_BUF_clock_G0_L1_1 clock (15, 0) CTS_ccd_BUF_clock_G1_L2_5 clock (3.22, 3.69)
2 CLKBUFX12 CTS_ccl_BUF_clock_G0_L2_6 clock (15, 0) CTS_cdb_BUF_clock_G0_L3_4 clock (2.3, 3.69)
3 CLKBUFX12 CTS_csk_BUF_clock_G1_L2_1 clock (15, 0) CTS_ccd_BUF_clock_G1_L3_5 clock (2.12, 3.69)
-------------------------------------------------------------------------------------------------------

cts_refine_clock_tree_placement

Calling refine place with halo constraints.
Have found 0 CTS cell-density overrides:
Refine Place commencing.
Leaving CCOpt scope...
*
* Starting clock placement refinement...
*
* First pass: Refine non-clock instances...
```

```
*
*** Starting refinePlace (0:00:21.6 mem=897.0M) ***
Total net bbox length = 1.613e+04 (9.844e+03 6.288e+03) (ext = 6.967e+01)
Density distribution unevenness ratio = 14.001%
Move report: Detail placement moves 0 insts, mean move: 0.00 um, max move: 0.00 um
Runtime: CPU: 0:00:00.0 REAL: 0:00:00.0 MEM: 899.0MB
Summary Report:
Instances move: 0 (out of 255 movable)
Mean displacement: 0.00 um
Max displacement: 0.00 um
Total net bbox length = 1.613e+04 (9.844e+03 6.288e+03) (ext = 6.967e+01)
Runtime: CPU: 0:00:00.0 REAL: 0:00:00.0 MEM: 899.0MB
*** Finished refinePlace (0:00:21.6 mem=899.0M) ***
*
* Second pass: Refine clock instances...
*
*** Starting refinePlace (0:00:21.9 mem=899.0M) ***
Total net bbox length = 1.613e+04 (9.844e+03 6.288e+03) (ext = 6.967e+01)
Density distribution unevenness ratio = 0.000%
Move report: Detail placement moves 28 insts, mean move: 4.75 um, max move: 11.98 um
Max move on inst (CTS_ccd_BUF_clock_G1_L2_5): (73.14, 29.52) --> (68.54, 22.14)
Runtime: CPU: 0:00:00.0 REAL: 0:00:00.0 MEM: 899.0MB
Summary Report:
Instances move: 28 (out of 333 movable)
Mean displacement: 4.75 um
Max displacement: 11.98 um (Instance: CTS_ccd_BUF_clock_G1_L2_5) (73.14, 29.52) -> (68.54, 22.14)
Length: 7 sites, height: 1 rows, site name: tsm12site, cell type: CLKBUFX8
Total net bbox length = 1.621e+04 (9.903e+03 6.304e+03) (ext = 6.967e+01)
Runtime: CPU: 0:00:00.0 REAL: 0:00:00.0 MEM: 899.0MB

*** Finished refinePlace (0:00:21.9 mem=899.0M) ***
*
* Moved 7 and flipped 4 of 78 clock instance(s) during refinement.
* The largest move was 11.98 microns for CTS_ccd_BUF_clock_G1_L2_5.
*
* Finished with clock placement refinement.
*

Leaving CCOpt scope done.
Refine Place finished.

Summary of Movements
Total moves = 6
Max Move : inst : CTS_ccd_BUF_clock_G1_L2_5 has moved from ( 73.140um , 29.520um )
to ( 68.540um , 22.140um ) distance = 11.98um.
Average Move = 6.6
6
report_ccopt_cell_halo_violations

Clock Cell Halo Rule Check in Summary
====================================

-----------------------------------------------------
Clock Instances with cell_halo Failed instances
-----------------------------------------------------
clock 6 0
-----------------------------------------------------
```

```
Total number of violating instances: 0

Clock Cell Halo Rule Violations
==============================

-----------------------------------------------------------------------------------------------
No. Cell Instance In clock Cell Halo (x, y) Violating instance From clock Intrusion (x, y)
-----------------------------------------------------------------------------------------------
(empty table)
-----------------------------------------------------------------------------------------------
```

Related Commands

- report_ccopt_cell_halo_violations
- refinePlace

ctd_save_histogram

```
ctd_save_histogram
[-help]
[-csv]
-file fileName
[-id windowID]
```

Saves the CTD histogram to a file. By default, the file type is PNG. You can also specify that the file be saved in the CSV format.

Before using this command to save the histogram view, first open and configure the histogram in the GUI, including skew group and delay corners.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `ctd_save_histogram` parameter. For a detailed description of the command and all of its parameters, use the man command: man `ctd_save_histogram`.
`-csv`	Saves the histogram in CSV format. This parameter is optional. By default, the histogram is saved as a PNG file.
`-file` *fileName*	Specifies the name of the file where you want to save the histogram. This parameter is required.
`-id` *windowIDName*	Specifies the ID of the active CTD window. The default value is the ID of the current active window.

Example

The following command saves the histogram in the `histogram1.csv` file.

```
ctd_save_histogram -file histogram1 -csv
```

Related Information

- `create_ccopt_clock_tree_spec`
- `close_ctd_win`
- `ctd_win`
- `get_ctd_win_id`
- `get_ctd_win_title`
- `gui_zoom_ctd`
- `set_ctd_win_title`
- Clock Menu chapter of the *Innovus Menu Reference.*

ctd_save_view

```
ctd_save_view
[-help]
-file fileName
[-id windowID]
[-legend_on]
[-type {PNG | GIF}
```

Saves the current snapshot of the clock tree viewer into a file.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `ctd_save_view` parameter. For a detailed description of the command and all of its parameters, use the man command: man `ctd_save_view`.
`-file fileName`	Specifies the name of the file where you want to dump the display image. This parameter is required.
`-id windowIDName`	Specifies the window ID whose display is to be saved. The default value is the returned value of command `get_ctd_win_id`.
`-legend_on`	Saves the whole window including the panels that are open.
`-type {PNG | GIF}`	Specifies whether the format of the image should be GIF or PNG. The default format is PNG.

Example

The following command saves the view snapshot in the `snapshot1.png` file.

```
ctd_save_view -file snapshot1 -type PNG
```

Related Information

- `create_ccopt_clock_tree_spec`
- `close_ctd_win`
- `ctd_win`
- `get_ctd_win_id`
- `get_ctd_win_title`
- `gui_zoom_ctd`
- `set_ctd_win_title`
- Clock Menu chapter of the *Innovus Menu Reference.*

ctd_show_skew_group

```
ctd_show_skew_group
[-help]
[-id window_id_name]
-skew_group skew_group_name
```

Shows the specified skew group in the CTD window.

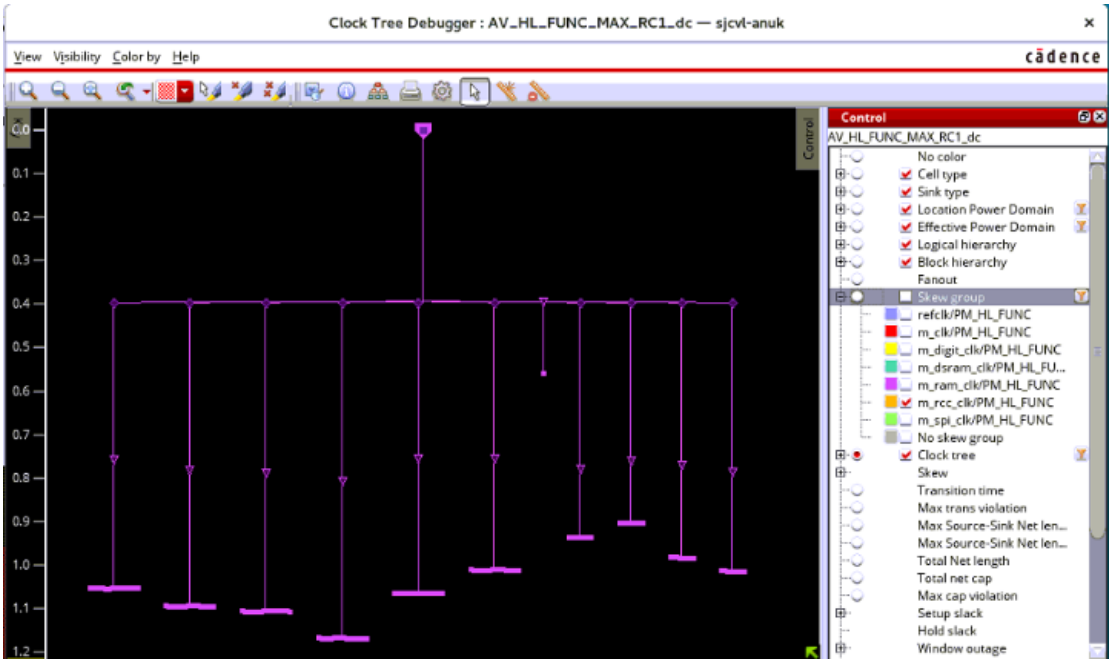
Parameters

`-help`	Outputs a brief description that includes type and default information for each `ctd_show_skew_group` parameter. For a detailed description of the command and all of its parameters, use the man command: man `ctd_show_skew_group`.
`-id window_id_name`	Specifies the CTD window ID. The default value is the ID of the current active CTD window.
`-skew_group skew_group_name`	Specifies the name of the skew group. This is a required parameter.

Example

Use the following command to show the specified skew group in the CTD window:

```
ctd_show_skew_group -skew_group m_rcc_clk/PM_HL_FUNC
```



Related Information

- `create_ccopt_clock_tree_spec`
- `ctd_win`
- `close_ctd_win`
- `get_ctd_win_id`
- `get_ctd_win_title`
- `gui_zoom_ctd`
- `set_ctd_win_title`
- The following topics in the *Innovus Menu Reference*
  - Clock Tree Debugger-View Menu
  - Control Panel-Viewing Skew Groups

ctd_trace

```
ctd_trace
[-help]
[-color colorname]
[-file fileName]
[-from rootclock]
[-index colorindex]
[-to sink | -through instancename]
```

Highlights the clock tree path from the clock root to the pin or from one pin to the other if the two pins are on the same path using the specified color or color index. The highlighted clock tree path can be viewed in the Clock Tree Viewer of the CCOpt Clock Tree Debugger (CTD). You can also use the Highlight Clock Path option in the context menu of the CTD to highlight the complete clock tree path of a selected pin to the clock root.

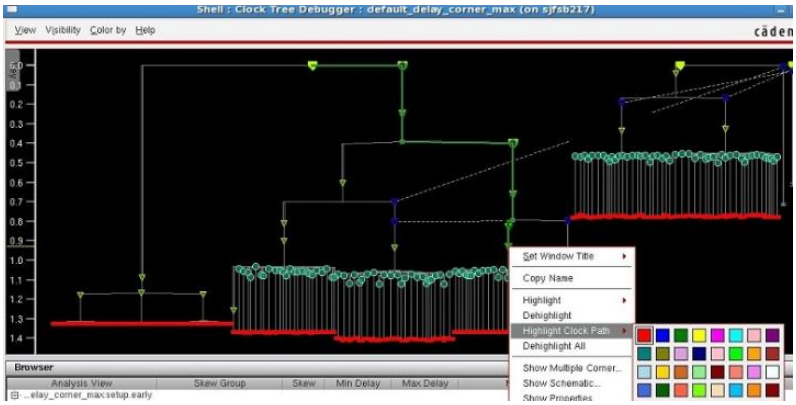
Parameters

`-help`	Outputs a brief description that includes type and default information for each `ctd_trace` parameter. For a detailed description of the command and all of its parameters, use the man command: man `ctd_trace`.
`-color` *`colorname`*	Specifies the color to be used for highlighting the clock tree path that is being traced. *Default*: red

`-file` *`fileName`*	Specifies the name of the file in which the physical information for each instance and net in the clock path being traced will be dumped.
	The following information is saved in the file:
	`Name Cell Status Location Wire Length`
	`CLK_ROOT PAD Fixed (20,40)`
	`net_1 don't_touch 12.5`
	`Buf_1 BUFX1 placed (100,180)`
	`net_2 35`
	`Buf_2 BUFX2 placed (200, 120)`
	Name is the instance name or the net name
	Cell is the instance cell type. This information is only for instances
	Status is the place status for an instance and constrain status for a net. For dont_touch nets, the status is '`dont_touch`', otherwise this column is left blank
	Location is the physical location of the instance or the coordinate of that instance
	Wire Length is only for nets

`-from` *`clockroot`*	Specifies the instance name as the starting point of the highlight. If this parameter is not specified, the starting point of the highlight is the clock root of the specified sink.
`-index` *`colorindex`*	Specifies the range of colors that can be used to highlight the clock tree path. Minimum is 1 and maximum is 32. The range corresponds to the colors in the color picker provided in the *Highlight Clock Path* option in the context menu of the Clock Tree Viewer. The number 1 means red, 2 means blue, and so on. An example is shown in the image below. *Default:* 1 or red.
`-through` *`instancename`*	Specifies the instance or pin through which the path should be traced.
`-to` *`sink`*	Specifies the end point of the highlighted path.

The image below shows the clock tree path highlighted in green, which corresponds to the color index 3. It also shows the color picker with the range of colors included in the color index.

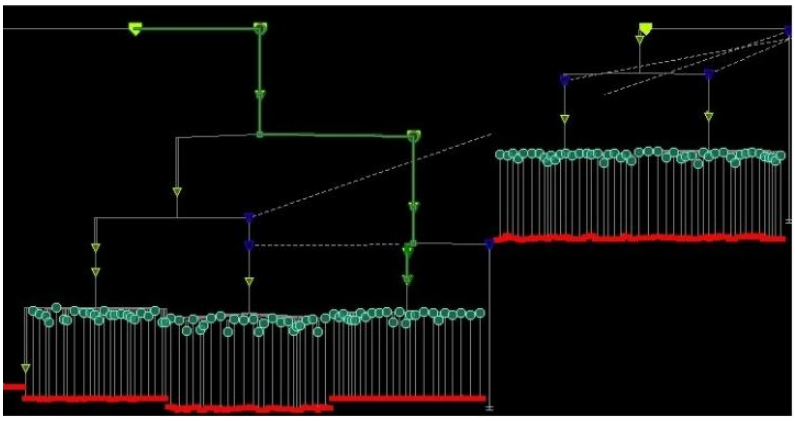


Example

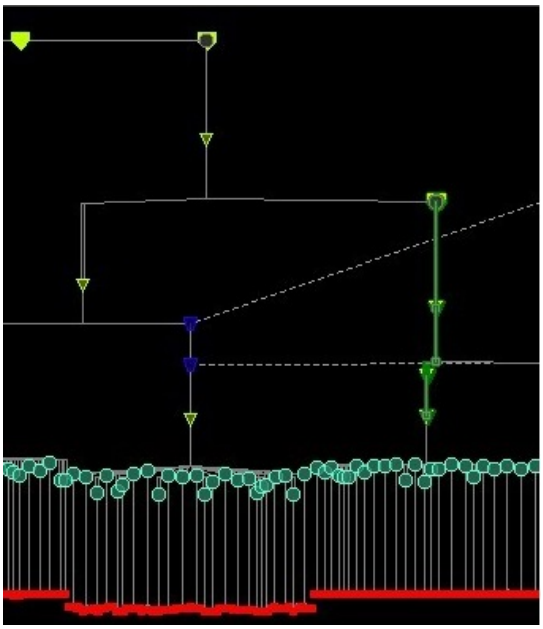
The following command highlights the clock tree path for the specified pin, `ff1`, from the clock root in green color:

- `ctd_trace -to ff1 -index 3`

  The CTD shows the following:

- The following command highlights the path between two sinks that are on the same path, in green color:
  `ctd_trace -to ff1 -from u1 -index 3`



Related Information

- `create_ccopt_clock_tree_spec`
- `ctd_win`
- `close_ctd_win`
- `get_ctd_win_id`
- `get_ctd_win_title`
- `gui_zoom_ctd`
- `set_ctd_win_title`
- The following topic in the *Innovus Menu Reference*
  - Context Menu of Clock Tree Debugger

ctd_win

```
ctd_win
[-help]
[-id windowIDName]
[-include_reporting_only_skew_groups]
[-title CTDWindowID]
[-unit_delay]
```

Opens a Clock Tree Debugger (CTD) window with the user-defined window ID and title.

When run without specifying the ID and title, the command opens a CTD window with a system default window title. By default, the window title is the name of the current analysis corner, which shows in the CTD window. For example, "Clock Tree Debugger - func_rcbest".

By default, all CTD windows have their unique window ID. The mechanism of the default window ID is `ctdMain_0, ctdMain_1, ctdMain_2, ctdMain_3`....and so on. So, by default, the window ID of a new opened Clock Tree Debugger window is `ctdMain_0`.

The information in the title comprises the following:

```
Session title: Window type: Window title
```

where,

Window type is always “Clock Tree Debugger”.

If the user-specified session title is “`my_test.tcl`”, and the default primary analysis corner is “`primary_corner`”, then with `ctd_win`, the total set of the window title line will be:

```
my_test.tcl : Clock Tree Debugger : primary_corner
```

Parameters

`-help`	Outputs a brief description that includes type and default information for each `ctd_win` parameter. For a detailed description of the command and all of its parameters, use the man command: man `ctd_win`.
`-id windowIDName`	Opens a Clock Tree Debugger window with a user-specified window ID. By default, the window IDs will be `ctdMain_0`, or `ctdMain_1`, and so on.
`-include_reporting_only_skew_groups`	When this parameter is specified, the GUI displays the reporting-only skew groups. A reporting-only skew group is a skew group whose `constrains` property is set to `none`. Such a skew group imposes no clock balancing constraint and is not considered by CTS. By default, this setting if off, which means the reporting-only skew groups are omitted from the CTD. **Note**: Once the CTD is started, you cannot add or remove the reporting-only skew groups from those that are displayed. Start a new CTD window to specify the skew group filtering.

`-title` *`CTDWindowID`*	Specifies the user-defined title for the CTD window. Open a Clock Tree Debugger window with a user specified title. For example, if you specify `my_title` as the window title, when session title is `my_test.tcl` and the primary analysis corner is `primary_corner`, then the total set of the window title line will be: `my_test.tcl : Clock Tree Debugger : my_title` **Note**: The window's ID if not specified using the `-id` parameter, will be system default ID. To view the default window ID, use the `get_ctd_win_id` command.

`-unit_delay`

Opens the CTD in the unit delay mode. The unit delay lets you view the depth of clock tree objects easily. This is useful when you have large designs with deep gate-level trees. When debugging clock trees, you can use this view to check how the clock trees are balanced by looking at the depth of the clock trees. Although, the unit delay mode can also be enabled using the *Unit delay* submenu in the *Visibility* menu of the CTD but this is only valid after running clock tree synthesis, and you still need to invoke timing calculation and extraction, which are time consuming. You can use this parameter to open the CTD in unit delay mode even if the design is not placed or without performing CTS.

When the CTD is opened in this mode, the following options in CTD menus are disabled:

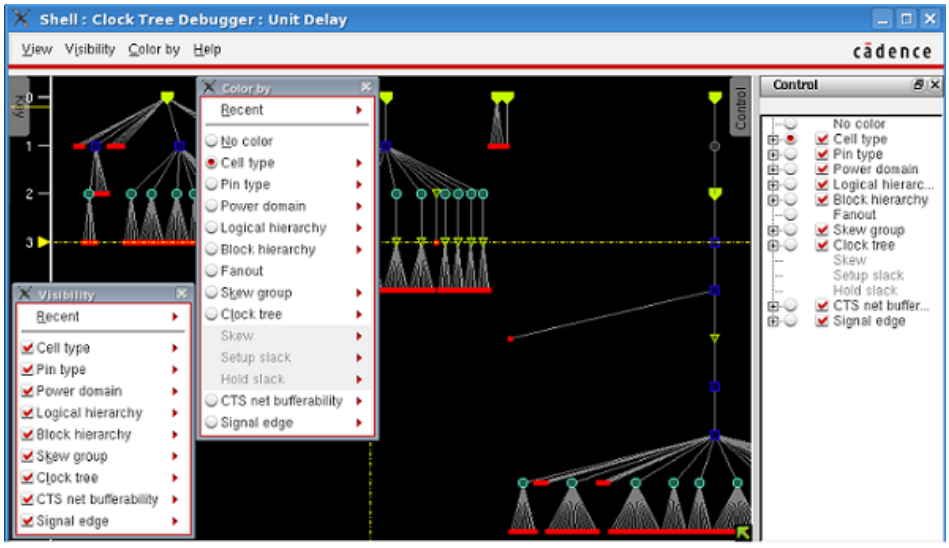
Visibility menu:

- Timing windows
- Delays
- Unit delay

Color by menu and the *Control* panel:

- Transition time
- Total net cap
- Max cap violation
- Skew
- Setup slack
- Hold slack
- Timing windows

This is shown in below image.



Examples

- The following command creates a CTD window ID with the default name, `ctdMain_0`:

  ```
  ctd_win
  ```

  When you use the `get_ctd_win_id` command to list the window ID, you get the above window ID:

  ```
  get_ctd_win_id
  ctdMain_0
  ```

- The following command creates a CTD window ID with the user-specified name, `my_test`:

  ```
  ctd_win -id my_test
  ```

  Now, when you use the `get_ctd_win_id` command to list all window IDs, you get the above window IDs in the list:

  ```
  get_ctd_win_id -all
  ctdMain_0 my_test
  ```

  If you use the `ctd_win` command with an existing window ID, it will pop up the corresponding CTD windows instead of creating a new one. This command will open the existing window, `my_test`.

  ```
  ctd_win -id my_test
  ```

- The following command creates a CTD window ID, `my_test` and CTD window title of `myTest.tcl`:

  ```
  ctd_win -id my_test -title myTest.tcl
  ```

  When you use the `-detail` parameter of the `get_ctd_win_id` command, the following information is returned:

  ```
  get_ctd_win_id -detail
  my_test "Clock Tree Debugger:myTest.tcl: func_rcbest"
  ```

  where, `my_test` is the specified window ID, window type is Clock Tree Debugger, window title is `myTest.tcl`, and `func_rcbest` is the name of the current analysis corner.

Related Information

- `create_ccopt_clock_tree_spec`
- `close_ctd_win`
- `get_ctd_win_id`
- `get_ctd_win_title`
- `gui_zoom_ctd`
- `set_ctd_win_title`
- Clock Menu chapter of the *Innovus Menu Reference.*

delete_ccopt_clock_spines

```
delete_ccopt_clock_spines
[-help]
pattern
[-regexp]
```

This command deletes all clock spines matching the specified pattern. This will also delete any placement groups or regions associated with the spine.

> ⚠ This command is part of CCOpt spine functionality, which is a limited-access feature in this release. It is enabled by a variable specified through the `setLimitedAccessFeature` command. To use this feature, contact your Cadence representative to explain your usage requirements, and make sure this feature meets your needs before deploying it widely.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `delete_ccopt_clock_spines` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_ccopt_clock_spines`.
`pattern`	Specifies the patterns for clock spines to delete. This parameter is required.
`-regexp`	Specifies whether or not to use regular expression matching.

Example

- The following set of commands are used to delete and retrieve information for clock spines spineA and spineB that are created using the `create_ccopt_clock_spine` command:
  The following command deletes `spineA`:
  ```
  delete_ccopt_clock_spines spineA
  ```

  Now when you retrieve the list of spines, the software returns the following information:

  ```
  get_ccopt_clock_spines spine*
  spineB
  ```

Related Commands

- `create_ccopt_clock_tree`
- `create_ccopt_clock_spine`
- `get_ccopt_clock_spines`
- `get_ccopt_property`

delete_ccopt_clock_tree_source_group

```
delete_ccopt_clock_tree_source_group
[-help]
pattern
[-regexp]
```

This command deletes all clock tree source groups matching the specified pattern.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `delete_ccopt_clock_tree_source_group` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_ccopt_clock_tree_source_group`.
`pattern`	Specifies the patterns for clock tree source groups to delete. This parameter is required.
`-regexp`	Specifies whether or not to use regular expression matching.

Examples

- Use the following command to delete all clock tree source groups starting with `"clk"`:

  ```
  delete_ccopt_clock_tree_source_group clk*
  ```

- When you check for clock tree source groups starting with `"clk"`, the software returns nothing.

  ```
  get_ccopt_clock_tree_source_groups clk*
  nothing returned
  ```

Related Commands

- create_ccopt_clock_tree_source_group
- create_ccopt_clock_tree_spec
- create_ccopt_clock_tree
- ccopt_design
- clock_opt_design
- get_ccopt_clock_tree_source_groups

delete_ccopt_clock_tree_spec

```
delete_ccopt_clock_tree_spec
[-help]
[-preserve_sink_insertion_delays]
```

This command deletes all the skew group definitions and other clock tree synthesis configuration information. This command fully resets any configuration except for any global CCOpt mode settings. However, you can preserve the pin insertion delays on clock tree sinks by specifying the `-preserve_sink_insertion_delays` parameter.

Parameter

`-help`	Outputs a brief description of the `delete_ccopt_clock_tree_spec` parameter.
`-preserve_sink_insertion_delays`	When specified, preserves pin insertion delays currently configured on clock tree sinks.

Example

- Use the following command to get a list of clock trees:

  ```
  get_ccopt_clock_trees *
  m_clk m_digit_clk m_dsram_clk m_ram_clk m_rcc_clk m_spi_clk refclk<1> refclk
  ```

- Use the following command to get a list of skew groups:

  ```
  get_ccopt_skew_groups *
  m_clk/PM_HL_FUNC m_digit_clk/PM_HL_FUNC m_dsram_clk/PM_HL_FUNC m_ram_clk/PM_HL_FUNC m_rcc_clk/PM_HL_FUNC
  m_spi_clk/PM_HL_FUNC refclk/PM_HL_FUNC
  ```

- Use the following command to delete all CTS configuration:

  ```
  delete_ccopt_clock_tree_spec
  ```

- When you check for clock trees and skew groups after deleting the CTS configuration, the software returns nothing.

  ```
  get_ccopt_clock_trees *
  nothing returned
  get_ccopt_skew_groups *
  nothing returned
  ```

Related Commands

- ccopt_design
- clock_opt_design
- create_ccopt_clock_tree
- create_ccopt_clock_tree_spec
- reset_ccopt_config

delete_ccopt_clock_trees

```
delete_ccopt_clock_trees
[-help]
pattern
[-regexp]
```

This command deletes all clock trees matching the specified pattern. It makes no changes to the design.

Note: It is recommended to use the `delete_ccopt_clock_tree_spec` command to reset the entire CTS configuration. It removes clock tree definition from the in-memory clock tree specification.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `delete_ccopt_clock_trees` parameter. For a detailed description of the command and all of its parameters, use the man command: man `delete_ccopt_clock_trees`.
`pattern`	Specifies the patterns for clock trees to delete. This parameter is required.
`-regexp`	Specifies whether or not to use regular expression matching.

Example

- The following command deletes all clock trees whose names start with "ref":
  ```
  delete_ccopt_clock_trees ref*
  ```

  The software gives the following message:
  ```
  Un-defining clock tree refclk.
  ```

Related Information

- `ccopt_design`
- `clock_opt_design`
- `create_ccopt_clock_tree`
- `get_ccopt_clock_trees`
- `report_ccopt_clock_trees`
- `update_ccopt_clock_tree`
- The following section in the *Innovus User Guide*
  - Clock Tree Concepts and Specifications

delete_ccopt_flexible_htrees

```
delete_ccopt_flexible_htrees
[-help]
pattern
[-regexp]
```

This command deletes all flexible H-trees matching the specified pattern.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `delete_ccopt_flexible_htrees` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_ccopt_flexible_htrees`.
`pattern`	Specifies the patterns for flexible H-trees to delete. This parameter is required.
`-regexp`	Specifies whether or not to use regular expression matching.

Example

- The following command deletes all flexible trees whose names start with "`htree`":

  ```
  delete_ccopt_flexible_htrees htree*
  ```

Related Information

- `create_ccopt_flexible_htree`
- `get_ccopt_flexible_htrees`
- `get_ccopt_property`
- `synthesize_ccopt_flexible_htrees`
- `set_ccopt_property`
- The following topic in the *Innovus User Guide*
  - Flexible H-Tree and Multi-Tap Clock Flow
- For details of the use model for this feature, see the Flexible H-tree and Multi-Tap Clock Flow in Innovus Application Note on the Cadence Online Support website.

delete_ccopt_preferred_cell_stripe

```
delete_ccopt_preferred_cell_stripe
[-help]
pattern
[-regexp]
```

Deletes all preferred cell stripes whose names match the specified pattern. Use this command to delete any incorrectly created preferred cell stripes objects so that they may be recreated using the `create_ccopt_preferred_cell_stripe` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `delete_ccopt_preferred_cell_stripe` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_ccopt_preferred_cell_stripe`.
`pattern`	Specifies the patterns for preferred cell stripes to delete. This parameter is required.
`-regexp`	Specifies whether or not to use regular expression matching.

Example

- Use the following command to delete all preferred cell stripes starting with "stripe":
  ```
  delete_ccopt_preferred_cell_stripes stripe*
  ```

Related Commands

- `ccopt_design`
- `clock_opt_design`
- `create_ccopt_preferred_cell_stripe`
- `get_ccopt_preferred_cell_stripe`

delete_ccopt_skew_groups

```
delete_ccopt_skew_groups
[-help]
[-constrains cts | ccopt_initial | ccopt]
pattern
[-regexp]
```

This command deletes all skew groups matching the specified pattern. It makes no changes to the design.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `delete_ccopt_skew_groups` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_ccopt_skew_groups`.
`-constrains cts | ccopt_initial | ccopt`	
	Specifies whether `cts`, `ccopt_initial`, or `ccopt` skew groups are to be deleted.
`pattern`	Specifies the patterns for skew groups to delete. This parameter is required.
`-regexp`	Specifies whether or not to use regular expression matching.

Examples

- The following command deletes all skew groups associated with the clock, `m_digit_clk`:
  ```
  delete_ccopt_skew_groups m_digit_clk*
  ```

- The following command deletes all skew groups whose name contains the word "`ram`":
  ```
  delete_ccopt_skew_groups ram -regexp
  ```

Related Information

- `get_ccopt_property`
- `get_ccopt_skew_groups`
- `get_ccopt_skew_group_delay`
- `get_ccopt_skew_group_path`
- `create_ccopt_skew_group`
- `modify_ccopt_skew_group`
- `report_ccopt_skew_groups`
- `set_ccopt_property`
- The following topic in the *Innovus User Guide*
  - Clock Trees and Skew Groups

delete_clock_tree_repeaters

```
delete_clock_tree_repeaters

[-help]
[-force]
```

This command removes buffers and inverter pairs from the clock trees. If used without specifying any parameters, it deletes all repeaters except those that have fixed or dont_touch attributes that are assigned either by the user or by `ccopt_design`. To delete all repeaters, specify the `-force` parameter.

Note: After running the command, there may be remaining inverters in the clock tree that are required to preserve an inversion. The remaining inverters may not be the same as the ones that were originally introduced for inversion, which means that the naming can differ.

Parameter

`-help`	Prints a brief description that includes type and default information for each `delete_clock_tree_repeaters` parameter. For a detailed description of the command and its parameter, use the man command: `man delete_clock_tree_repeaters`.
`-force`	Deletes all repeaters even if they have fixed or dont_touch attributes defined.

Related Command

- `ccopt_design`
- `clock_opt_design`

delete_route_type

```
delete_route_type
[-help]
-name string
```

This command deletes the specified route type object created using the `create_route_type` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `delete_route_type` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_route_type`.
`-name string`	Specifies the name of the route type object to be deleted. This parameter is required.

Examples

- The following command creates a route type for a non-default rule:
  ```
  create_route_type -name RType1 -non_default_rule NDR1
  ```
- The following command deletes the route type that was previously created for the NDR::
  ```
  delete_route_type -name RType1
  ```

get_ccopt_clock_spines

```
get_ccopt_clock_spines
[-help]
pattern
[-regexp]
```

This command returns a list of clock spines whose names match the specified pattern.

⚠ This command is part of CCOpt spine functionality, which is a limited-access feature in this release. It is enabled by a variable specified through the `setLimitedAccessFeature` command. To use this feature, contact your Cadence representative to explain your usage requirements, and make sure this feature meets your needs before deploying it widely.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_ccopt_clock_spines` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_ccopt_clock_spines`.
pattern	Returns a list of clock spines that match this pattern. It is a required parameter.
`-regexp`	When specified, treats all patterns as full Tcl regular expressions. If not specified, pattern is treated as a wildcard pattern.

Example

- The following command returns the list of all clock spines that have "spine" in their names:

  ```
  get_ccopt_clock_spines spine*
  spineA spineB
  ```

Related Commands

- `create_ccopt_clock_tree`
- `create_ccopt_clock_spine`
- `delete_ccopt_clock_spines`
- `get_ccopt_property`

get_ccopt_clock_tree_capacitance

```
  get_ccopt_clock_tree_capacitance
[-help]
pin | port
[-delay_corner corner]
[-delay_type {early | late}]
[-edge {rise | fall | both}]
[-load | -wire]
```

This command retrieves the capacitance on the specified pin. When you run the command without specifying the type of capacitance to be retrieved, load or wire, then the command retrieves the total capacitance on a pin, which includes both the wire capacitance of nets connected to the pin, and the load capacitance of pins connected to those nets. Because the capacitance varies depending on the timing data used to calculate delays, this command provides options such as rise, fall, early, and late that let you specify the exact timing criteria to use when calculating capacitances.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_ccopt_clock_tree_capacitance` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_ccopt_clock_tree_capacitance`.
`-delay_corner` *corner*	Specifies the delay corner of the specified pin for which capacitance information is provided. By default, if delay corner is not specified, CCOpt uses the CTS primary corner as configured by the `set_ccopt_property primary_delay_corner <corner>` command. If this is not set, it will use the default setup analysis view as configured by the `set_default_view -setup <view>` command.
`-delay_type {early | late}]`	Specifies whether to use early or late half delays. When `early` is specified, CCOpt uses only early path timing data to calculate capacitances, for the clock edge specified. When `late` is specified, CCOPT uses only late path timing data to calculate capacitances, for the clock edge specified. By default, if neither early or late are specified, CCOpt will use late half delays to calculate capacitances.
`-edge {rise | fall | both}`	
	Specifies the edge of the specified pin for which capacitance is to be calculated. When `fall` is specified, CCOpt uses only `falling` clock edge timing data to calculate capacitances. When `rise` is specified, CCOpt uses only `rising` clock edge timing data to calculate capacitances. By default, if neither `rise` or `fall` are specified or if both is specified, CCOpt uses both rising and falling clock edge timing data to calculate capacitances.
`-load | -wire`	When `-load` is specified, the command returns the driven load capacitance value of the specified pin. When `-wire` is specified, the command returns the driven wire capacitance value of the specified pin. The two options are mutually exclusive.
pin | *port*	Specifies the pin or port for which the load capacitance value is to be returned.This is a required parameter.

Examples

- Use the following command to return the list of clock tree cells:
  ```
  get_ccopt_clock_tree_cells 770 -regexp
  RESULTS_CONV_INST/upper770/g54 ...
  ```
- Use the following command to return the pins for the specified object:
  ```
  get_property [get_pins -of_object RESULTS_CONV_INST/upper770/g54] hierarchical_name
  RESULTS_CONV_INST/upper770/g54/ZN RESULTS_CONV_INST/upper770/g54/I
  ```
- Use the following command to return the capacitance information for the specified pin:
  ```
  get_ccopt_clock_tree_capacitance RESULTS_CONV_INST/upper770/g54/ZN
  ```
  The software returns the following:
  ```
  0.0160777831078
  ```
- Use the following command to return the wire capacitance information for the specified pin:
  ```
  get_ccopt_clock_tree_capacitance TEST_CONTROL_INST/g145/ZN -wire
  ```
  The software returns the following:
  ```
  0.00874637126923
  ```

Related Commands

- get_ccopt_clock_tree_cells
- get_pins
- set_ccopt_property
- set_default_view

get_ccopt_clock_tree_cells

```
get_ccopt_clock_tree_cells
[-help]
pattern
[-in_clock_trees list_of_trees]
[-net_types leaf | trunk | top]
[-node_types buffer | inverter | clock_gate | logic | source | generator | all]
[-not_in_clock_trees list_of_trees]
[-regexp]
```

This command returns a list of clock tree cells - cell instances that form a part of the clock tree network - whose names match the specified pattern. By default, the command returns all clock tree cells with matching names, but you can use optional parameters to filter the list based on cell type and clock tree membership.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_ccopt_clock_tree_cells` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_ccopt_clock_tree_cells`.
`-in_clock_trees` *`list_of_trees`*	
	Specifies a Tcl list of clock trees whose cells are to be included in the list. If not specified, all the cells of all the clock trees in the design that satisfy the other conditions of the command will be returned.
`-net_types leaf | trunk | top`	
	Only returns nodes driving nets of the specified type(s). All clock tree nets fall into one of three categories: • leaf nets - nets connected to sinks • top nets - nets with a high transitive fanout, exceeding the threshold specified using the `set_ccopt_property routing_top_min_fanout` CCOpt property • trunk nets - all other nets in the clock tree that are neither leaf nor top nets By default, all net types are returned: `leaf, top, and trunk`.
`-node_types buffer | inverter | clock_gate | logic | source | generator | all`	
	Specifies the type of cells returned by the command. If not specified, all types of cells that satisfy the other conditions of the command will be returned. Valid values for *`type`* are: "buffer", "clock_gate", "inverter", "logic", "source", "generator", and "all".
`-not_in_clock_trees` *`list_of_trees`*	
	Specifies a Tcl list of clock tree whose cells are to be excluded from the list. If not specified, all the cells that satisfy the other conditions of the command will be returned.
`pattern`	Returns clock tree cells that match this pattern. If not specified, all the clock tree cells that satisfy the other conditions of the command will be returned. **Note**: The wildcard pattern, (*) matches zero or more characters. You can also use the question mark (?), which matches exactly one character.
`-regexp`	When specified, treats all patterns as full Tcl regular expressions. If not specified, pattern is treated as a wildcard pattern.

Examples

- The following command returns the list of all clock tree cells that start with "TEST", which is a wildcard match:

```
get_ccopt_clock_tree_cells TEST*
TEST_CONTROL_INST/g138 TEST_CONTROL_INST/g140 TEST_CONTROL_INST/g142 TEST_CONTROL_INST/g139
TEST_CONTROL_INST/g141 TEST_CONTROL_INST/g137
```

- The following command returns the list of all clock tree cells that contain the word "TEST", which is a Tcl pattern match:

```
get_ccopt_clock_tree_cells TEST -regexp
TEST_CONTROL_INST/g138 TEST_CONTROL_INST/g140 TEST_CONTROL_INST/g142 TEST_CONTROL_INST/g139
RAM_256x16_TEST_INST/CPF_LS_160_m_dsram_clk TEST_CONTROL_INST/g141 TEST_CONTROL_INST/g137
```

- The following command returns the list of all clock tree cells in either of the clock trees, `M_CLK or M_DIGIT_CLK:`

```
get_ccopt_clock_tree_cells * -in_clock_trees {m_clk m_digit_clk}
TEST_CONTROL_INST/g141 TEST_CONTROL_INST/g137 RESULTS_CONV_INST/CPF_LS_159_m_clk DMA_INST/CPF_LS_158_m_clk
SPI_INST/RC_CG_HIER_INST17/RC_CGIC_INST RESULTS_CONV_INST/RC_CG_HIER_INST7/RC_CGIC_INST
RESULTS_CONV_INST/RC_CG_HIER_INST6/RC_CGIC_INST RESULTS_CONV_INST/RC_CG_HIER_INST3/RC_CGIC_INST
RESULTS_CONV_INST/RC_CG_HIER_INST2/RC_CGIC_INST RESULTS_CONV_INST/RC_CG_HIER_INST1/RC_CGIC_INST
DMA_INST/RC_CG_HIER_INST0/RC_CGIC_INST
```

- The following command returns the list of all clock trees except for cells in either of the clock trees, `M_CLK or M_DIGIT_CLK:`

```
get_ccopt_clock_tree_cells * -not_in_clock_trees {m_clk m_digit_clk}
TEST_CONTROL_INST/g138 SPI_INST/RC_CG_HIER_INST16/RC_CGIC_INST TEST_CO...
```

- The following command gets all inverters in any clock tree:

```
get_ccopt_clock_tree_cells *  -node_types inverter
RESULTS_CONV_INST/g18694 RESULTS_CONV_INST/upper770/g54 RESULTS_CONV_INST/upper1336/g54
RESULTS_CONV_INST/lower770/g54 RESU...
```

- The following command gets all inverters in `M_RCC_CLK clock tree:`

```
get_ccopt_clock_tree_cells *  -node_types inverter -in_clock_trees m_rcc_clk
RESULTS_CONV_INST/g18694
```

Related Information

- `create_ccopt_clock_tree`
- `delete_ccopt_clock_trees`
- `set_ccopt_property`
- The following topic in the *Innovus User Guide*
  - Generating Reports in Clock Tree Synthesis

get_ccopt_clock_tree_nets

```
get_ccopt_clock_tree_nets
[-help]
pattern
[-in_clock_trees list_of_trees]
[-net_types leaf | trunk | top]
[-not_in_clock_trees list_of_trees]
[-regexp]
```

This command returns a list of clock tree nets whose names match the specified pattern. By default, the command returns all clock tree nets with matching names, but you can use optional parameters to filter the list based on net type and clock tree membership. Valid values for net types are `leaf, trunk,` and `top`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_ccopt_clock_tree_nets` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_ccopt_clock_tree_nets`.
`-in_clock_trees` *`list_of_trees`*	Specifies a TCL list of clock trees whose nets are to be included in the list. If not specified, all the nets of all the clock trees in the design that satisfy the other conditions of the command will be returned.
`-net_types leaf | trunk | top`	
	Only returns nets of the specified type(s). All clock tree nets fall into one of three categories: • leaf nets - nets connected to sinks • top nets - nets with a high transitive fanout, exceeding the threshold specified using the `set_ccopt_property routing_top_min_fanout` CCOpt property • trunk nets - all other nets in the clock tree that are neither leaf nor top nets By default, all net types are returned: `leaf, top, and trunk`.
`-not_in_clock_trees` *`list_of_trees`*	
	Specifies a TCL list of clock trees whose nets are to be excluded from the list. If not specified, all the nets that satisfy the other conditions of the command will be returned.
`pattern`	Returns clock tree nets that match this pattern. If not specified, all the nets that satisfy the other conditions of the command will be returned.
`-regexp`	When specified, treats all patterns as full TCL regular expressions. If not specified, pattern is treated as a wildcard pattern.

Examples

- The following command returns the list of all clock tree nets in all clock trees:
  ```
  get_ccopt_clock_tree_nets *
  refclk clk TEST_CONTROL_INST/clk_CPF_LS_DUP_101 TEST_CONTROL_INST/n_0 TEST_CONTROL_INST/n_1 m_tdsp_clk
  TDSP_CORE_INST/clk_CPF_LS_DUP_162 TDSP_CORE_INST/TDSP_CORE_MACH_INST
  ```

- The following command returns the list of all trunk type nets in all clock trees:
  ```
  get_ccopt_clock_tree_nets * -net_types trunk
  refclk clk TEST_CONTROL_INST/clk_CPF_LS_DUP_101 TEST_CONTROL_INST/n_1 m_tdsp_clk m_spi_clk m_rcc_clk
  RESULTS_CONV_INST/rcc_clk_CPF_LS_DUP_161 RESULTS_CONV_INST/upper770/n_4 RESULTS_CONV_INST/upper1336/n_4
  RESULTS_CONV_INST/lower770/n_4 RESULTS_CONV_INST/lower1336/n_4 RESULTS_CONV_INST/n_1390
  RESULTS_CONV_INST/n_1389 RESULTS_CONV_INST/n_1394 RESULTS_CONV_INST/n_1393 RESULTS_CONV_INST/n_1392
  RESULTS_CONV_INST/n_1391 m_dsram_clk
  ```

- The following command returns the list of all nets whose names start with the word "TEST":
  ```
  get_ccopt_clock_tree_nets TEST*
  TEST_CONTROL_INST/clk_CPF_LS_DUP_101 TEST_CONTROL_INST/n_0 TEST_CONTROL_INST/n_1
  ```

- The following command returns the list of all nets that contain the word "TEST":
  ```
  get_ccopt_clock_tree_nets TEST -regexp
  TEST_CONTROL_INST/clk_CPF_LS_DUP_101 TEST_CONTROL_INST/n_0 TEST_CONTROL_INST/n_1
  RAM_256x16_TEST_INST/wr_CPF_LS_DUP_160
  ```

Related Information

- `create_ccopt_clock_tree`
- `delete_ccopt_clock_trees`
- `set_ccopt_property`
- The following topic in the *Innovus User Guide*
  - Generating Reports in Clock Tree Synthesis

get_ccopt_clock_tree_sinks

```
get_ccopt_clock_tree_sinks
[-help]
pattern
[-in_clock_trees list_of_trees]
[-not_in_clock_trees list_of_trees]
[-regexp]
```

This command returns a list of clock tree sinks whose names match the specified pattern. By default, the command returns all clock tree sinks with matching names, but you can use optional parameters to filter the list based on the clock tree membership.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_ccopt_clock_tree_sinks` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_ccopt_clock_tree_sinks`.
`-in_clock_trees list_of_trees`	
	Specifies a TCL list of clock trees whose sinks are to be included in the list. If not specified, all the sinks of all the clock trees in the design that satisfy the other conditions of the command will be returned.
`-not_in_clock_trees list_of_trees`	
	Specifies a TCL list of clock trees whose sinks are to be excluded from the list. If not specified, all the sinks that satisfy the other conditions of the command will be returned.
pattern	Returns clock tree sinks that match this pattern. If not specified, all the sinks that satisfy the other conditions of the command will be returned.
`-regexp`	When specified, treats all patterns as full TCL regular expressions. If not specified, pattern is treated as a wildcard pattern.

Examples

- The following command returns the list of all clock tree sinks in all clock trees:

```
get_ccopt_clock_tree_sinks *
{TDSP_CORE_INST/TDSP_CORE_MACH_INST/tdsp_state_reg[2]/CP}
{TDSP_CORE_INST/TDSP_CORE_MACH_INST/tdsp_state_reg[1]/CP}
{TDSP_CORE_INST/TDSP_CORE_MACH_INST/tdsp_state_reg[0]/CP}...
```

- The following command returns the list of all clock tree sinks in clock whose name starts with "TEST":

```
get_ccopt_clock_tree_sinks TEST*
TEST_CONTROL_INST/g137/A1
```

- The following command returns the list of all clock tree sinks in clock trees whose names contain "TEST":

```
get_ccopt_clock_tree_sinks TEST -regexp
TEST_CONTROL_INST/g137/A1 RAM_128x16_TEST_INST/RAM_128x16_INST/CLK RAM_256x16_TEST_INST/RAM_256x16_INST/CLK
```

Related Information

- `create_ccopt_clock_tree`
- `delete_ccopt_clock_trees`
- `set_ccopt_property`
- The following topic in the *Innovus User Guide*
  - Generating Reports in Clock Tree Synthesis

get_ccopt_clock_tree_slew

```
get_ccopt_clock_tree_slew
[-help]
pin
[-delay_corner corner]
[-delay_type {early | late}]
[-edge {rise | fall | both}]
```

This command returns the slew information for the user-specified pin. The command is usually used to collect slew data on a clock tree, for example, to create custom histograms.

Note: A clock tree specification is required before running `get_ccopt_clock_tree_slew`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_ccopt_clock_tree_slew` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_ccopt_clock_tree_slew`.
`-delay_corner` *corner*	
	Specifies the delay corner of the specified pin to be queried for slew information. By default, if delay corner is not specified, CCOpt uses the CTS primary corner as configured by the `set_ccopt_property primary_delay_corner` *corner* command. If this is not set, it will use the default setup analysis view as configured by the `set_default_view -setup` *view* command.
`-delay_type {early | late}]`	
	Specifies whether to use early or late half delays. When `early` is specified, CCOpt uses only early path timing data to calculate slew, for the clock edge specified. When `late` is specified, CCOpt uses only late path timing data to calculate slew, for the clock edge specified. By default, if neither early or late are specified, CCOpt will use late half delays to calculate slew information.
`-edge { rise | fall | both }`	
	Specifies the edge of the specified pin for which slew information is to be calculated. When `fall` is specified, CCOpt uses only `falling` clock edge timing data to calculate slew. When `rise` is specified, CCOpt uses only `rising` clock edge timing data to calculate slew. By default, if neither rise or fall are specified, CCOpt uses both rising and falling clock edge timing data to calculate slew.
pin	Specifies the pin for which the slew information is provided. This is a required parameter.

Examples

- Use the following command to get a list of clock tree cells:
  ```
  get_ccopt_clock_tree_cells *
  PLLCLK_INST TEST_CONTROL_INST/CPF_LS_101_clk TEST_CONTROL_INST/g145 TEST_CONTROL_INST/g144
  TEST_CONTROL_INST/g136...
  ```

- Use the following command to get the list of pins in the specified clock tree:
  ```
  get_pins TEST_CONTROL_INST/g145/*
  TEST_CONTROL_INST/g145/ZN TEST_CONTROL_INST/g145/A2 TEST_CONTROL_INST/g145/A1
  0x5d5
  ```
- Use the following command to get the slew value for the specified pin:
  ```
  get_ccopt_clock_tree_slew Inst1/A
  ```
  The software returns the following value for the slew for the pin A:
  ```
  0.2228
  ```
  Note: To confirm that the instance is part of the clock network, use `get_ccopt_clock_tree_cells <inst>`
- Use the following command to get the slew value for the specified pin in the specified clock tree:
  ```
  get_ccopt_clock_tree_slew TEST_CONTROL_INST/g145/ZN
  ```
  The software returns the following value for the slew for the pin ZN:
  ```
  0.2228
  ```

Related Commands

- create_ccopt_clock_tree
- delete_ccopt_clock_trees
- get_ccopt_clock_tree_cells
- get_pins
- reportDelayCalculation
- set_ccopt_property
- set_default_view

get_ccopt_clock_tree_source_groups

```
get_ccopt_clock_tree_source_groups
[-help]
pattern
[-regexp]
```

This command returns a list of clock tree source group objects matching the supplied pattern.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_ccopt_clock_tree_source_groups` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_ccopt_clock_tree_source_groups`.
`pattern`	Specifies the patterns for clock tree source groups to return.
`-regexp`	Specifies whether or not to use regular expression matching.

Examples

- The following command returns the list of all clock tree source groups:

  ```
  get_ccopt_clock_tree_source_groups *
  top_clk_group test_sig_group
  ```

- The following command returns the list of all clock tree source groups that contain the word `clk`, which is a Tcl pattern match：

  ```
  get_ccopt_clock_tree_source_groups clk -regexp
  top_clk_group
  ```

- The following command returns the list of all clock tree source groups that start with `"t"`, which is a wildcard match：

  ```
  get_ccopt_clock_tree_source_groups t*
  top_clk_group test_sig_group
  ```

Related Commands

- create_ccopt_clock_tree_source_group
- create_ccopt_clock_tree_spec
- create_ccopt_clock_tree
- ccopt_design
- clock_opt_design
- delete_ccopt_clock_tree_source_group

get_ccopt_clock_trees

```
get_ccopt_clock_trees
[-help]
pattern
[-regexp]
```

This command returns the names of the clock trees whose names match the specified pattern.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_ccopt_clock_trees` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_ccopt_clock_trees`.
`pattern`	Returns a list of clock trees that match this pattern. The wildcard pattern, (*) matches zero or more characters. You can also use the question mark (?), which matches exactly one character.
`-regexp`	When specified, treats all patterns as full Tcl regular expressions. If not specified, pattern is treated as a wildcard pattern.

Examples

- The following command returns the list of all clock trees:

  ```
  get_ccopt_clock_trees *
  m_clk m_digit_clk m_dsram_clk m_ram_clk m_rcc_clk m_spi_clk refclk<1> refclkv
  ```

- The following command returns the list of all clock trees that contain the word `ram_clk`, which is a Tcl pattern match：

  ```
  get_ccopt_clock_trees ram_clk -regexp
  m_dsram_clk m_ram_clk
  ```

- The following command returns the list of all clock trees that start with `"m"`, which is a wildcard match：

  ```
  get_ccopt_clock_trees m*
  m_clk m_digit_clk m_dsram_clk m_ram_clk m_rcc_clk m_spi_clk
  ```

Related Information

- `ccopt_design`
- `clock_opt_design`
- `create_ccopt_clock_tree`
- `delete_ccopt_clock_trees`
- `report_ccopt_clock_trees`
- The following topics in the *Innovus User Guide*
  - Clock Tree Concepts and Specifications
  - Generating Reports in Clock Tree Synthesis

get_ccopt_dag_traversal

```
get_ccopt_dag_traversal
[-help]
[-fanin]
[-fanout]
[-of_insts instances]
[-of_pins pins]
[-skew_group skew_group_name]
[-skip buffers inverters drivers clock_gates logics generators generated_roots all]
[-transitive]
```

This command traverses the clock tree network DAG, starting at the specified pins and traversing to their fanin or fanout, returning a list of the resulting pins. Instance types can be skipped over, and can be restricted to a specified skew group. The clock tree network is modeled as a DAG (Directed Acyclic Graph) to accommodate re-convergence and overlap among clock trees and skew groups.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_ccopt_dag_traversal` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_ccopt_dag_traversal`.
`-fanin`	Returns the output clock network pins of the fanin of the specified pins.
`-fanout`	Returns the input clock network pins of the fanout of the specified pins.
`-of_pins`	Specifies a list of pins to start traversing from.
`-of_insts`	Specifies a list of instances to start traversing from. If -fanin is specifed, will start from the input pins of the instances. If -fanout is specifed, will start from the output pins.
`-skew_group skew_group_name`	If specified, restricts traversal to instances in the skew group.
`-skip` *types*	Specifies the types of nodes that should be skipped past to their fanin/fanout during the traversal. You can specify one or more of the following: • buffers • inverters • drivers (equivalent to buffers and inverters) • clock_gates • logics • generators • generated_roots • all (equivalent to listing all the above, and also equivalent to `-transitive`)
`-transitive`	Skip all types and get sinks (if -fanout) or clock roots (if -fanin). This is equivalent to the `-skip all` option.

Examples

```
get_ccopt_dag_traversal -fanout -of_insts CTS_ccd_BUF_clock_G0_L1_1

get_ccopt_dag_traversal -fanout -skip buffers -of_insts CTS_ccd_BUF_clock_G0_L1_1

get_ccopt_dag_traversal -fanin -skip {buffers generators logics} -of_pins {CTS_ccd_BUF_clock_G0_L1_1/A
CTS_cdb_BUF_clock_G0_L1_2/A}

set pins [get_ccopt_dag_traversal -transitive -skew_group skewgroup1 -fanin -of_pins d1111a/CK]

foreach pin $pins  {

  Puts "Got pin $pin"
}
```

get_ccopt_delay_corner

```
get_ccopt_delay_corner
[-help]
```

This command returns the name of the CCOpt primary-half delay corner, which is the main corner that CTS uses for balancing clock trees. This delay corner is usually the first setup corner. This command is useful if you are trying to determine why the balancing constraints have not been met.

The primary delay corner can be set using the `primary_delay_corner` property. However, if you set this property to {}, which is the default value, then you can use the `get_ccopt_delay_corner` command to retrieve the name of the corner that was picked by default.

Parameter

`-help`	Outputs a brief description of `get_ccopt_delay_corner` parameter.

Example

- The following command returns the name of the CCOpt primary-half corner:

  ```
  get_ccopt_delay_corner
  AV_HL_FUNC_MAX_RC1_dc:setup
  ```

Related Information

- `set_ccopt_property`
- The following topic in the *Innovus User Guide*
  - Generating Reports in Clock Tree Synthesis
- The following property:
  `primary_delay_corner`
  For details, see the CCOpt Properties chapter in the *Innovus User Guide.*

get_ccopt_effective_max_capacitance

```
get_ccopt_effective_max_capacitance
[-help]
pin | port
[-delay_corner corner]
[-delay_type {early | late}]
[-source]
[-value]
```

Returns the value of the frequency-dependent effective maximum capacitance constraint that the software will apply at a given pin in the clock tree. By default, the software returns the following information:

- The actual value of the maximum effective capacitance or constrained maximum capacitance at the specified pin
- The source of the constraint

To annotate a clock period at a point in the clock tree, a new property called `clock_period` is provided. To aid debugging of clock period propagation, the effective or the computed clock period will be accessible for any given pin in the clock tree by querying the read only property, `effective_clock_period`. This property lists the set of pins that define the fastest `effective_clock_period` at a given pin. Details of these properties are provided below.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_ccopt_effective_max_capacitance` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_ccopt_effective_max_capacitance`.
`-delay_corner` *corner*	Specifies the delay corner of the specified pin for which effective maximum capacitance information is provided. By default, if delay corner is not specified, CCOPT uses the CTS primary corner as configured by the `primary_delay_corner` *delay_corner_name* property of the `set_ccopt_property` command. If this is not set, it will use the default setup analysis view as configured by the command, `set_default_view -setup` *newDefaultSetupView*.
`-delay_type {early | late}`	Specifies whether to use early or late half delays. When `early` is specified, CCOPT uses only early path timing data to calculate effective maximum capacitance, for the specified clock edge. When `late` is specified, CCOPT uses only late path timing data to calculate effective maximum capacitance, for the specified clock edge. By default, if neither `early` or `late` are specified, CCOPT will use late half delays to calculate effective maximum capacitance.
pin | *port*	Specifies the pin for which the effective maximum capacitance value is to be returned. This is a required parameter.

`-source`	Reports the source of the effective maximum capacitance. It is a boolean type. The reported source may be any of the following: • `library_or_sdc`: Specifies that the most constraining capacitance came from a non-frequency dependent library constraint on a library cell pin or SDC override. This can also apply to clock roots based on the `source_driver` property where a clock root can be modeled as a specific type of library cell. • `freq_dep_library`: Specifies that the most constraining capacitance value came from a frequency-dependent library constraint. As per `library_or_sdc`, this can also apply to clock roots via library cell modeling. • `target_max_capacitance`: Specifies that the most constraining capacitance value came from an explicit `target_max_capacitance` property value. • `source_max_capacitance`: Specifies that the most constraining capacitance value came from the `source_max_capacitance` property. • `computed_source_max_capacitance`: Specifies that the most constraining capacitance value was computed by CCOpt for a clock root based on a library cell and a transition target. For more information about the above properties, use the following command: `get_ccopt_property -help property_name`
`-value`	Reports the effective maximum capacitance value of the specified pin.

Examples

- Use the following command to return the list of clock tree cells:
  ```
  get_ccopt_clock_tree_cells TEST*
  TEST_CONTROL_INST/CPF_LS_101_clk TEST_CONTROL_INST/g145 TEST_CONTROL_INST/g144 TEST_CONTROL_INST/g136 ...
  ```
- Use the following command to return the pins for the specified object:
  ```
  get_pins -of_object TEST_CONTROL_INST/g139
  TEST_CONTROL_INST/g139/A1 TEST_CONTROL_INST/g139/A2 TEST_CONTROL_INST/g139/B TEST_CONTROL_INST/g139/ZN
  ```
- Use the following command to return the source of the effective maximum capacitance value:
  ```
  get_ccopt_effective_max_capacitance TEST_CONTROL_INST/g139/ZN -source
  ```
  The software returns the following:
  ```
  library_or_sdc
  ```
- Use the following command to return the effective maximum capacitance value:
  ```
  get_ccopt_effective_max_capacitance TEST_CONTROL_INST/g139/ZN -value
  ```
  The software returns the following:
  ```
  0.0853199996948
  ```

Related Information

- `get_ccopt_property`
- `get_ccopt_clock_tree_cells`
- `get_pins`
- `set_ccopt_property`
- `set_default_view`
- The following properties:
  - `clock_period`
  - `effective_clock_period`
  - `effective_clock_period_sources`

For details, see the CCOpt Properties chapter in the *Innovus User Guide.*

get_ccopt_flexible_htrees

```
get_ccopt_flexible_htrees
[-help]
pattern
[-regexp]
```

This command returns the names of the flexible H-trees whose names match the specified pattern.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_ccopt_flexible_htrees` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_ccopt_flexible_htrees`.
`pattern`	Returns a list of flexible H-trees that match this pattern. It is a required parameter.
`-regexp`	When specified, treats all patterns as full Tcl regular expressions. If not specified, pattern is treated as a wildcard pattern.

Example

- The following command returns the list of all flexible H- trees that contain the word `htree`, which is a Tcl pattern match:

  ```
  get_ccopt_flexible_htrees htree -regexp
  htreeA htreeB
  ```

Related Commands

- `create_ccopt_flexible_htree`
- `delete_ccopt_flexible_htrees`
- `get_ccopt_property`
- `synthesize_ccopt_flexible_htrees`
- `set_ccopt_property`

get_ccopt_preferred_cell_stripe

```
get_ccopt_preferred_cell_stripe
[-help]
pattern
[-regexp]
```

Returns a list of preferred cell stripes objects matching the supplied pattern.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_ccopt_preferred_cell_stripe` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_ccopt_preferred_cell_stripe`.
`pattern`	Specifies the pattern for preferred cell stripes to return.
`-regexp`	Specifies whether or not to use regular expression matching.

Example

- The following command returns the list of all preferred cell stripes:

  ```
  get_ccopt_preferred_cell_stripe *
  stripes1 stripes2
  ```

Related Commands

- `create_ccopt_preferred_cell_stripe`
- `delete_ccopt_preferred_cell_stripe`

get_ccopt_property

```
get_ccopt_property
[-help]
name
[-cell library_cell_name]
[-clock_tree clock_tree_name]
[-clock_tree_source_group source_group_name]
[-delay_corner corner_name]
[-flexible_htree flexibleHtreeName]
[-help property_name]
[-inst instance_name]
[-lib_pin lib_pin_name]
[-list]
[-net net_name]
[-net_type leaf | trunk | top]
[-pin pin_name]
[-power_domain domain_name]
[-skew_group skew_group_name]
[-early | -late]
[-rise | -fall]
[-min | -max]
```

Retrieves the value of the specified CCOpt property. It can be used in the following ways:

1. To retrieve the value of a property. Some properties are global, and others are specific to a particular object. The command returns the value of the specified property when you use `get_ccopt_property property_name`.
   For example, use the following command to return the value for the maximum spatial density of clock tree cells.
   ```
   get_ccopt_property cell_density
   ```
   The software returns the following information:
   ```
   1
   ```
2. To retrieve a list of all the available CCOpt properties - in alphabetical order, run the following command:
   ```
   get_ccopt_property -help *
   ```
   The software displays the following information:
   ```
   add_driver_cell
   additional_buffer_depth
   additional_buffer_depth_skew_group_fraction
   add_exclusion_drivers
   adjacent_rows_legal

   ...............................
   ```
3. To retrieve a list of all the available properties that start with a word or a letter, for example "a", along with their brief descriptions, run the following command:
   ```
   get_ccopt_property -help a*
   ```
   The software displays the following information:
   ```
   add_driver_cell                              # Specify a driver cell to add for clock tree after root.
   add_exclusion_drivers                        # Setting this property true causes CCOpt to add extra
   drivers above exclude pins to remove them from the clock tree.
   additional_buffer_depth                      # Relaxes the max_buffer_depth constraint by an additional N
   stage depths, but only for the shortest paths.
   additional_buffer_depth_skew_group_fraction  # When using additional_buffer_depth, apply the relaxed
   ```

```
constraint to specified fraction of the skew group's shortest paths.
```

Note: If the wildcard that is used matches many properties, as seen in above example, you will see a brief description of each property. However, if the wildcard that is used matches a single option, for example, `balance_*`, then the detailed description of the property will be displayed as shown in below example.

4. The `get_ccopt_property` command is also used to query detailed information about a specified property when you use, `get_ccopt_property -help` *`property_name`* . The information displayed by the tool includes the usage, default value, valid values, and the object or objects that the property applies to. For example, the following command returns the information for the `balance_mode` property.

   ```
   get_ccopt_property -help clock_source_cells
   ```

   or

   ```
   get_ccopt_property -help clock_sou*
   ```

   The software displays the following information:

   ```
   Specifies the cells available for CTS to size clock sources if the property 'size_clock_source' is set to
   true. If none are specified CCOpt will choose cells from the libraries.

   Cell names may be specified as a Tcl list of names, or as a Tcl list of patterns to be expanded to match
   names.

   If set explicitly, CCOpt will ignore any don't use settings for the cells specified.

   Different cells may be specified for clock trees or power domains. Only clock sources that are buffers,
   inverters, logic and clock gating cells with a single output can be resized.

   Valid values: a list of library cell names, or a list of patterns to expand to library cell names

   Default: {}

   Optional applicable arguments: "-clock_tree <name>" and "-power_domain <name>"
   ```

For detailed information about how to use CCOpt properties, see the "CCOpt Property System" section in Clock Tree Synthesis chapter in *Innovus User Guide*. For detailed description of all public CCOpt properties, see the CCOpt Properties chapter in *Innovus User Guide.*

Parameters

`-help`	Outputs a brief description for the `get_ccopt_property` command. For a detailed description of the command, use the man command: `man get_ccopt_property`.
`-cell` *`library_cell_name`*	
	Specifies the library cell to which the specified property applies.
`-clock_tree` *`clock_tree_name`*	
	Specifies the name of the clock tree whose property is to be returned.
`-clock_tree_source_group` *`source_group_name`*	
	Specifies the name of the clock tree source group whose property is to be returned.
`-delay_corner` *`corner_name`*	
	Specifies the name of the delay corner whose property is to be returned.

`-early`	Specifies that the property applies to the early signal.
`-fall`	Specifies that the property applies to the fall signal.
`-flexible_htree` *flexibleHtreeName*	
	Specifies the name of the flexible H-tree whose property is to be returned.
`-help` *property_name*	Gets help for the specified property. All other options are ignored.
`-inst` *instance_name*	Specifies the instance to which the property applies.
`-late`	Specifies that the property applies to the late signal.
`-lib_pin` *lib_pin_name*	Specifies the library pin to which the specific property applies.
`-list`	Displays property values as a list.
`-max`	Specifies that the maximum value for the property be returned.
`-min`	Specifies that the minimum value for the property be returned.
name	Specifies the name of the property whose value is to be returned. This parameter is required.
`-net` *net_name*	Specifies the name of the net level to which the specified property applies.
`-net_type leaf | trunk | top`	
	Specifies the net type to which the property applies. It could be leaf net, trunk net, or top net.
`-pin` *pin_name*	Specifies the pin object whose property is to be returned.
`-power_domain` *domain_name*	
	Specifies the power domain to which the property applies.
`-rise`	Specifies that the property applies to the rise signal.
`-skew_group` *skew_group_name*	
	Specifies the name of the skew group to which the specified property applies.

Examples:

- The following command retrieves the value of the `use_inverters` property:

  ```
  get_ccopt_property use_inverters
  ```

  The software returns the following:

  ```
  auto
  ```

- The following command gets detailed help for the specified property:

  ```
  get_ccopt_property -help use_inverters
  ```

  The software displays the following information:

  ```
  Usage: get_ccopt_property use_inverters:
  Specifies whether clock tree synthesis should prefer to use inverters rather
  than buffers when balancing the clock tree. If set to true, CTS will use
  inverters for the clock tree balancing process. If set to false, CTS will use
  the minimum number of levels of inverters required to maintain logical
  correctness. If set to auto (the default) CTS will use what it considers to be
  the best combination of buffers and inverters to get optimal quality of
  results.

  Valid values: auto true false

  Default: auto

  Optional applicable arguments: "-clock_tree <name>".
  ```

Related Information

- `unset_ccopt_property`
- `set_ccopt_property`
- For details about the key considerations for setting CCOpt properties, getting properties from Tcl, getting a list of properties and their detailed descriptions, key/value pairs, and defaults for key/value pairs, see the The CCOpt Property System in the *Innovus User Guide*.
- CCOpt Properties chapter in the *Innovus User Guide*

get_ccopt_skew_group_delay

```
get_ccopt_skew_group_delay
[-help]
[-check_type {setup | hold}]
[-delay_corner delay_corner]
[-delay_type {early | late}]
[-edge {rise | fall}]
[-exclude_pin_insertion_delays]
-skew_group skew_group_name
[-skew | -longest | -shortest ]
[-path_length distance_metric | -summarize_stage_depth cell_type_list]
[-all_active_sinks | -to pin | -from pin | -through pin]
[-virtual_delays_only | -real_delays_only]
[-wire_delays_only | -cell_delays_only]
```

By default, this command returns the longest source-to-sink delay for the specified skew group. Alternatively, you can use `-shortest` to return the shortest source-to-sink delay, or `-skew` to return the difference between the shortest and longest source-to-sink delay.

Optional arguments let you restrict the set of pins considered. For example, use `-to` to return the longest delay to the specified sink pin, use `-delay_type` and `-delay_corner` to specify how delays are calculated, and use `-wire_delays_only` to specify which delays are considered.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_ccopt_skew_group_delay` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_ccopt_skew_group_delay`.
`-all_active_sinks`	Specifies that the result should be a tcl dict of pairs of sink and delay, describing delays for all the active sinks of the skew group. For example, `{sink_a latency_a sink_b latency_b ... sink_n latency_n}`.
`-cell_delays_only`	Specifies that CCOpt only considers cell delays when calculating delays and ignores wire delays.
`-check_type {setup | hold}`	
	Specifies whether to use setup or hold timing as the check type to use for the specified skew group. If a check type is specified, but the `-delay_type` parameter is not specified, the software behaves in the following manner: • `-check_type -setup` will mean `-delay_type -late` • `-check_type -hold` will mean `-delay_type -early` **Note**: If a check type is not specified, the software will always look at what the views permit, preferring the “setup” view if there is a choice.
`-delay_corner delay_corner`	

	Specifies the delay corner to use when calculating delays. If this parameter is not specified, the software will use the delay corner specified using the CCOpt property, `primary_delay_corner`. However, if no valid delay corner is found, the software will error out. **Note**: For detailed help on the CCOpt property, `primary_delay_corner`, run the following command: `get_ccopt_property -help primary_delay_corner`
`-delay_type {early | late}`	
	Specifies whether to use early or late half delays. When `early` is specified, CCOpt uses only early path timing data to calculate delay, for the clock edge specified. When `late` is specified, CCOpt uses only late path timing data to calculate delay, for the clock edge specified. **Note**: If the delay type is not specified but the check type is specified as “setup”, the software will consider the delay type as “late”. If the check type is “hold”, the software will consider the delay type as “early”.
`-edge {rise | fall}`	Specifies the edge of the specified pin for which delay information is to be calculated. When `fall` is specified, CCOpt uses only `falling` clock edge timing data to calculate delay. When `rise` is specified, CCOpt uses only `rising` clock edge timing data to calculate delay.
`-exclude_pin_insertion_delays`	Specifies that the delay should exclude pin insertion delays.
`-from` *pin*	Specifies the pin to query delays from. This means that only paths from the given pin to a sink of the skew group will be considered.
`-longest`	Specifies that CCOpt will return the delay of the longest source-to-sink path within the skew group. This is the default option.

`-path_length {half_perim pin_to_pin_steiner routed}`	Specifies that CCOpt will report by the distance from the root measured using the specified distance metric. You can choose to report for any of the following metrics:
	• `half_perim`: The sum of the Manhattan distances between terminals on the path. This is not blockage-aware and is suitable for unbuffered clock trees.
	• `pin_to_pin_steiner`: The sum of lengths calculated using a pin-to-pin steiner route. This is blockage-aware and is suitable for unbuffered clock trees.
	• `routed`: The sum of pin-to-pin paths calculated on the full steiner for each net. This is blockage-aware but is not suitable for unbuffered clock trees.
	The routed metric is best post-CTS as it measures the actual clock path. The other metrics are good for pre-CTS although they can also be calculated post CTS. For pre-CTS, the `half_perim` metric is fast to calculate while the other metrics are slow.
	Example:
	`get_ccopt_skew_group_delay -skew_group skew_group:clk/default_constraint_mode -longest -path_length half_perim 1177.8065`
	`get_ccopt_skew_group_delay -skew_group skew_group:clk/default_constraint_mode -longest -path_length half_perim -to pin:nastyclock/flop001/CK 930.0045`
	Note:
	• You can only specify one metric at a time.
	• You have to specify a skew group and that the length reported will be from the skew group sources to the skew group sinks. This may not cover the entire clock tree. The calculated length will take account of any cells on the clock path, which may increase the path length. The behavior is consistent with delay. The -longest, -shortest, and -skew options work the same. This means that the software looks at all skew group sinks and finds the one with the longest/shortest distance. This sink will not always be the same with the three metrics.
	• This parameter is mutually exclusive with the `-summarize_stage_depth` parameter.

`-real_delays_only`	Specifies that CCOpt only considers real delays in the design. It will not include the virtual delays added by trial CTS to resolve skew constraints.
`-shortest`	Specifies that CCOpt will return the delay of the shortest source-to-sink path within the skew group.
`-skew`	Specifies that CCOpt will return the difference between the shortest and longest source-to-sink delays within the skew group.
`-skew_group` *skew_group_name*	
	Specifies the name of the skew group for which the delay information will be returned. This is a required parameter.
`-summarize_stage_depth` *cellTypeList*	

	Specifies that the command will consider the number of cells of the specified type(s) in the tree, instead of considering delay. Possible values are buffer, inverter,logic, clock_gate, and generator. You can also specify * to include all cell types.
	For example, the following command tells you the number of buffers and inverters along the specified path type in the skew group, `PM_HL_FUNC`.
	`get_ccopt_skew_group_delay -summarize_stage_depth {buffer inverter} -skew_group m_dsram_clk/PM_HL_FUNC`
	Note: The * setting is not used in combination with any other cell type. For example, the below is not a valid setting.
	`-summarize_stage_depth {* buffer}`
	Note: This parameter is mutually exclusive with the `-path_length` parameter.

`-through` *pin*	Specifies the pin to query delays through. This means that only paths from a source of the given skew group, through this pin, and to a sink of the given skew group, will be considered.
`-to` *pin*	Specifies the pin to query delays to. This means that only paths from a source of the given skew group to this pin will be considered.
`-virtual_delays_only`	Specifies that CCOpt will return the skew, shortest path delay, or longest path delay only considering the virtual path delays (added by trial balancing) in the design and any user pin insertion delays applied before CTS. It will not include real delays added by clustering buffers and existing cells in the design.
`-wire_delays_only`	Specifies that CCOpt will return the skew, shortest path delay, or longest path delay only considering the wire delays in the design. (that is, ignoring cell delays)

Examples

- Use the following command to list all skew groups in all clocks:

  ```
  get_ccopt_skew_groups *
  m_clk/PM_HL_FUNC m_digit_clk/PM_HL_FUNC m_dsram_clk/PM_HL_FUNC m_ram_clk/PM_HL_FUNC m_rcc_clk/PM_HL_FUNC
  m_spi_clk/PM_HL_FUNC refclk/PM_HL_FUNC
  ```

- Use the following command to list the sinks in the specified skew group:

  ```
  get_ccopt_property sinks  -skew_group m_clk/PM_HL_FUNC
  TDSP_DS_CS_INST/g395/A1 ROM_512x16_0_INST/CLK SPI_INST/present_state_reg[0]/CP SPI_INST/dflag_reg/CP S..
  ```

- Use the following command to get the longest source to sink delay for the specified sink:`get_ccopt_skew_group_delay -skew_group m_clk/PM_HL_FUNC -to SPI_INST/dflag_reg/CP`

- Use the following command to get the number of buffers and inverters along the specified path type in the skew group, `PM_HL_FUNC`:

  ```
  get_ccopt_skew_group_delay -summarize_stage_depth {buffer inverter} -skew_group  m_dsram_clk/PM_HL_FUNC
  ```

  The software returns the following value:

  ```
  4
  ```

Related Information

- `get_ccopt_skew_groups`
- `get_ccopt_property`
- `get_ccopt_skew_group_path`
- `delete_ccopt_skew_groups`
- `modify_ccopt_skew_group`
- `report_ccopt_skew_groups`
- `set_ccopt_property`
- The following topics in the *Innovus User Guide*
  - Generating Reports in Clock Tree Synthesis
  - Manual Setup and Adjustment of the Clock Tree Specification

get_ccopt_skew_group_path

```
get_ccopt_skew_group_path
[-help]
[-check_type {setup | hold}]
[-delay_corner delay_corner]
[-delay_type {early | late}]
[-show_generator_paths]
-skew_group skew_group_name
[-longest  | -shortest]
[-summarize_stage_depth cell_type_list]
[-virtual_delays_only  | -real_delays_only]
[-wire_delays_only  | -cell_delays_only]
[-sink pin | -below pin]
```

This command returns the path delay information for the specified skew group. By default, this command returns the longest source-to-sink path in the specified skew group, which is the path with the largest delay. Paths are returned as a list of pins. You can use `-shortest` to return the shortest source-to-sink path rather than the longest.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_ccopt_skew_group_path` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_ccopt_skew_group_path`
`-below pin`	Specifies that CCOpt will only consider paths below this pin.
`-cell_delays_only`	Specifies that CCOpt will only consider cell delays (ignoring wire delays). By default both cell and wire delays are used to calculate delays.
`-check_type {setup | hold}`	
	Specifies whether the check type for the specified skew group should be setup or hold. If a check type is specified, but the `-delay_type` parameter is not specified, the software behaves in the following manner: • `-check_type -setup` will mean `-delay_type -late` • `-check_type -hold` will mean `-delay_type -early` **Note**: If a check type is not specified, the software will always look at what the views permit, preferring the “setup” view if there is a choice.
`-delay_corner delay_corner`	

	Specifies the delay corner to use when calculating delays. (use `all_delay_corners` to see all configured delay corners) If this parameter is not specified, the software will use the delay corner specified using the CCOpt property, `primary_delay_corner`. However, if no valid delay corner is found, the software will error out. **Note**: For detailed help on the CCOpt property, `primary_delay_corner`, run the following command: `get_ccopt_property -help primary_delay_corner`
`-delay_type {early | late}`	
	Specifies whether to use early or late half delays. When `early` is specified, CCOpt uses only early path timing data to calculate path delay, for the clock edge specified. When `late` is specified, CCOpt uses only late path timing data to calculate path delay, for the clock edge specified. **Note**: If the delay type is not specified but the check type is specified as “setup”, the software will consider the delay type as “late”. If the check type is “hold”, the software will consider the delay type as “early”.
`-longest`	Specifies that CCOpt will return the longest path delay within the skew group. This is the default option.
`-real_delays_only`	Specifies that CCOpt will only consider the real path delays in the design. It will not include the virtual path delays added by trial CTS to resolve skew constraints.
`-shortest`	Specifies that CCOpt will return the shortest source-to-sink path within the skew group (that is, the path with the smallest source-to-sink delay). By default, this command returns the longest path.
`-show_generator_paths`	Specifies that CCOpt follows generated clock tree paths when determining the longest source-to-sink path. By default generated clock tree paths are not followed.
`-sink` *pin*	Specifies that CCOpt will return the longest path to this pin for the specified skew group.
`-skew_group` *skew_group_name*	
	Specifies the name of the skew group for which path information will be returned. This is a required parameter.
`-summarize_stage_depth` *cell_type_list*	

	Specifies that the command will consider the number of cells of the specified type(s) in the tree, instead of considering delay. Possible values are buffer, inverter,logic, clock_gate, and generator. You can also specify * to include all cell types.
	For example, the following command shows you the path with most buffers and inverters in the skew group, `PM_HL_FUNC`.
	`get_ccopt_skew_group_path -summarize_stage_depth {buffer inverter} -longest -skew_group m_dsram_clk/PM_HL_FUNC`
	Note: The * setting is not used in combination with any other cell type. For example, the below is not a valid setting.
	`-summarize_stage_depth {* buffer}`

`-virtual_delays_only`	Specifies that CCOpt will return the shortest path delay or longest path delay only considering the virtual path delays (added by trial balancing) in the design and any user pin insertion delays applied before CTS. It will not include real path delays added by clustering buffers and existing cells in the design.
`-wire_delays_only`	Specifies that CCOpt will only consider wire delays (ignoring cell delays). By default both cell and wire delays are used to calculate delays.

Examples

- Use the following command to retrieve the list of skew groups:
  ```
  get_ccopt_skew_groups *
  m_clk/PM_HL_FUNC m_digit_clk/PM_HL_FUNC m_dsram_clk/PM_HL_FUNC m_ram_clk/PM_HL_FUNC m_rcc_clk/PM_HL_FUNC
  m_spi_clk/PM_HL_FUNC refclk/PM_HL_FUNC
  ```
- Use the following command to retrieve the paths with most buffers and inverters in the skew group, `PM_HL_FUNC`:
  ```
  get_ccopt_skew_group_path -summarize_stage_depth {buffer inverter} -longest -skew_group
  m_dsram_clk/PM_HL_FUNC
  ```

```
TEST_CONTROL_INST/AZ_cid_BUF_m_dsram_clk_G0_L1_1/Z RAM_256x16_TEST_INST/CPF_LS_160_m_dsram_clk/I
RAM_256x16_TEST_INST/CPF_LS_160_m_dsram_clk/Z
TDSP_CORE_INST/TDSP_CORE_MACH_INST/RC_CG_HIER_INST29/AZ_csf_BUF_m_dsram_clk_G0_L3_1/I
TDSP_CORE_INST/TDSP_CORE_MACH_INST/RC_CG_HIER_INST29/AZ_csf_BUF_m_dsram_clk_G0_L3_1/Z
TDSP_CORE_INST/TDSP_CORE_MACH_INST/RC_CG_HIER_INST29/AZ_csf_BUF_m_dsram_clk_G0_L4_1/I
TDSP_CORE_INST/TDSP_CORE_MACH_INST/RC_CG_HIER_INST29/AZ_csf_BUF_m_dsram_clk_G0_L4_1/Z
TDSP_CORE_INST/TDSP_CORE_MACH_INST/RC_CG_HIER_INST29/AZ_csf_BUF_m_dsram_clk_G0_L5_1/I
TDSP_CORE_INST/TDSP_CORE_MACH_INST/RC_CG_HIER_INST29/AZ_csf_BUF_m_dsram_clk_G0_L5_1/Z
TDSP_CORE_INST/TDSP_CORE_MACH_INST/RC_CG_HIER_INST29/AZ_csf_BUF_m_dsram_clk_G0_L6_1/I
TDSP_CORE_INST/TDSP_CORE_MACH_INST/RC_CG_HIER_INST29/AZ_csf_BUF_m_dsram_clk_G0_L6_1/Z
RAM_256x16_TEST_INST/RAM_256x16_INST/CLK
```

Related Information

- `get_ccopt_skew_groups`
- `get_ccopt_property`
- `get_ccopt_skew_group_delay`
- `delete_ccopt_skew_groups`
- `modify_ccopt_skew_group`
- `report_ccopt_skew_groups`
- `set_ccopt_property`
- The following topics in the *Innovus User Guide*
  - Generating Reports in Clock Tree Synthesis
  - Manual Setup and Adjustment of the Clock Tree Specification

get_ccopt_skew_groups

```
get_ccopt_skew_groups
[-help]
pattern
[-regexp]
```

This command returns a list of skew groups whose names match the specified pattern.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_ccopt_skew_groups` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_ccopt_skew_groups`
`pattern`	Returns skew groups that match this pattern. This is a required parameter.
`-regexp`	When specified, treats all patterns as full Tcl regular expressions. If not specified, pattern is treated as a wildcard pattern.

Examples

- The following command returns all skew groups:
  ```
  get_ccopt_skew_groups *
  m_clk/PM_HL_FUNC m_digit_clk/PM_HL_FUNC m_dsram_clk/PM_HL_FUNC m_ram_clk/PM_HL_FUNC m_rcc_clk/PM_HL_FUNC
  m_spi_clk/PM_HL_FUNC refclk/PM_HL_FUNC
  ```
- The following command returns all skew groups associated with the clock, `m_digit_clk`:
  ```
  get_ccopt_skew_groups m_digit_clk*
  m_digit_clk/PM_HL_FUNC
  ```
- The following command returns all skew groups whose name contains the word "ram":
  ```
  get_ccopt_skew_groups ram -regexp
  m_dsram_clk/PM_HL_FUNC m_ram_clk/PM_HL_FUNC
  ```

Related Information

- `create_ccopt_skew_group`
- `get_ccopt_property`
- `get_ccopt_skew_group_delay`
- `get_ccopt_skew_group_path`
- `delete_ccopt_skew_groups`
- `modify_ccopt_skew_group`
- `report_ccopt_skew_groups`
- `set_ccopt_property`
- The following topics in the *Innovus User Guide*

- Generating Reports in Clock Tree Synthesis
- Manual Setup and Adjustment of the Clock Tree Specification

get_ctd_win_id

```
get_ctd_win_id
[-help]
[-all]
[-detail]
```

Retrieves the IDs of the CTD windows. If no parameters are specified, the command retrieves the window ID of the current active CTD window. For example, if you create a window with the ID, `user_id`, and then run this command without specifying any parameters, the software displays the following information:

```
ctd_win -title my_title -id user_id
get_ctd_win_id
user_id
```

Note: Before using this command, ensure that a CTD window is open. You can open a CTD window using the `ctd_win` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_ctd_win_id` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_ctd_win_id`
`-all`	Retrieves the IDs of all CTD windows.
`-detail`	Retrieves the IDs of CTD windows, with a mapping to their titles.

Examples

- The following command retrieves the ID of all CTD windows in the current session:

  ```
  get_ctd_win_id -all
  ```

  user_id1 user_id2 user_id3

- The following command retrieves the IDs of CTD windows, with a mapping of their titles:

  ```
  get_ctd_win_id -detail
  ```

  ```
  user_id2 : Clock Tree Debugger : title2
  ```

  where, `user_id2` is the window ID, Clock Tree Debugger is the window type, and `title2` is the mapping title for this window.

Related Information

- `create_ccopt_clock_tree_spec`
- `close_ctd_win`
- `ctd_trace`
- `get_ctd_win_title`
- `gui_zoom_ctd`
- `set_ctd_win_title`
- Clock Menu chapter in the *Innovus Menu Reference.*

get_ctd_win_title

```
get_ctd_win_title
[-help]
[-all | -id WindowIDName]
```

Retrieves the titles of the CTD windows. If no parameters are specified, the command retrieves the title of the current active CTD window.

Note: Before using this command, ensure that a CTD window is open. You can open a CTD window using the `ctd_win` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_ctd_win_title` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_ctd_win_title`
`-all`	Retrieves the IDs of all CTD windows.
`-id` *`WindowIDName`*	Retrieves the title of the CTD window with the specified ID. **Note**: The parameters, `-all` and `-id` are mutually exclusive.

Examples

- The following command retrieves the titles of all CTD windows in the current session:

  ```
  get_ctd_win_title -all
  ```

  ```
  {user_id1 : Clock Tree Debugger : title1} {user_id2 : Clock Tree Debugger : title2} {user_id3 : Clock Tree Debugger : title3}
  ```

- The following command retrieves the title of the CTD window with the ID, `user_id2`:

  ```
  get_ctd_win_title -id user_id2
  ```

  ```
  user_id2 : Clock Tree Debugger : title2
  ```

  where, `user_id2` is the window ID, Clock Tree Debugger is the window type, and `title2` is the mapping title for this window.

Related Information

- `create_ccopt_clock_tree_spec`
- `close_ctd_win`
- `ctd_trace`
- `get_ctd_win_id`
- `gui_zoom_ctd`
- `set_ctd_win_title`

- Clock Menu chapter in the *Innovus Menu Reference.*

get_lib_clock_tree_path_delay

```
get_lib_clock_tree_path_delay
[-help]
-base_pin base_pin_name
[-delay_type {max | min}]
[-edge {rise | fall}]
[-mode analysis_mode_name]
[-power_domain power_domain_name]
-transition transition_time
-view analysis_view_name
```

This command retrieves information about the Liberty `max_clock_tree_path` and `min_clock_tree_path` delay attributes for a library pin. You can then use Tcl to apply the `set_clock_latency` command options, as required. The value returned by the `get_lib_clock_tree_path_delay` command is in the default system units, such that the user can pass to the `set_clock_latency` command directly. The same units are used for the `-transition` value.

If the library pin does not have a corresponding `min/max_clock_tree_path` delay attribute, the command returns a value of `0`. If other errors are encountered, the command fails.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_lib_clock_tree_path_delay` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_lib_clock_tree_path_delay`
`-base_pin` *`base_pin_name`*	Specifies the name of the base pin for which the `min/max_clock_tree_path` attribute value is to be retrieved. This is a required parameter.
`-delay_type {max | min}`	Specifies whether to use `max` or `min` delays. You can specify either `max` or `min` to retrieve the `max_clock_tree_path` and `min_clock_tree_path` delay values, respectively. *Default*: `max`
`-edge {rise | fall}`	Specifies whether the delay should be retrieved for the rising edge or the falling edge of the signal. *Default*: `rise`
`-mode` *`analysis_mode_name`*	Specifies the Liberty timing library mode value to be used while retrieving the delay attributes for the library pin. If no mode definition is available, the default look-up table (LUT) is used. The detailed use model is provided below. **User-Specified Item**: **Software Behavior** • mode + delay_type: Use the specified mode and the specified delay_type LUT • mode: Use the specified mode and the first max LUT • delay_type: Use the first specified delay type LUT • No mode and no delay type: Use the first max LUT For more information about setting modes, see `set_mode`.
`-power_domain` *`power_domain_name`*	Specifies the name of the power domain. This parameter is only required to be specified when the same base cell has different models between different power domains.
`-transition` *`transition_time`*	Specifies the input transition time. This is a required parameter.
`-view` *`analysis_view_name`*	Specifies the analysis view name. This is a required parameter.

Examples:

- Use the following command to retrieve the value of `max_clock_tree_path` delay at the rising edge of the specified pin:
  `get_lib_clock_tree_path_delay -base_pin macro_lib_cell/CLK -view setup_func_m40 -transition 0.1`
- Use the following command to retrieve the value of `min_clock_tree_path` delay at the falling edge of the specified pin:
  `get_lib_clock_tree_path_delay -base_pin macro_lib_cell/CLK -view setup_func_m40 -delay_type min -edge fall -transition 0.1`
- Use the following command to retrieve the value of `max_clock_tree_path` delay when `macro_lib_cell` is specified only in the `setup_func_m40c` view of `PD_TOP` power domain:
  `get_lib_clock_tree_path_delay -base_pin macro_lib_cell/CLK -view setup_func_m40 -power_domain PD_TOP -transition 0.1`

Related Command

- `set_clock_latency`

gui_zoom_ctd

```
gui_zoom_ctd
[-help]
[-id string]
[-in | -out | -selected]
[-vertical_range {ymin ymax}]
```

Zooms into or out of the CTD window.

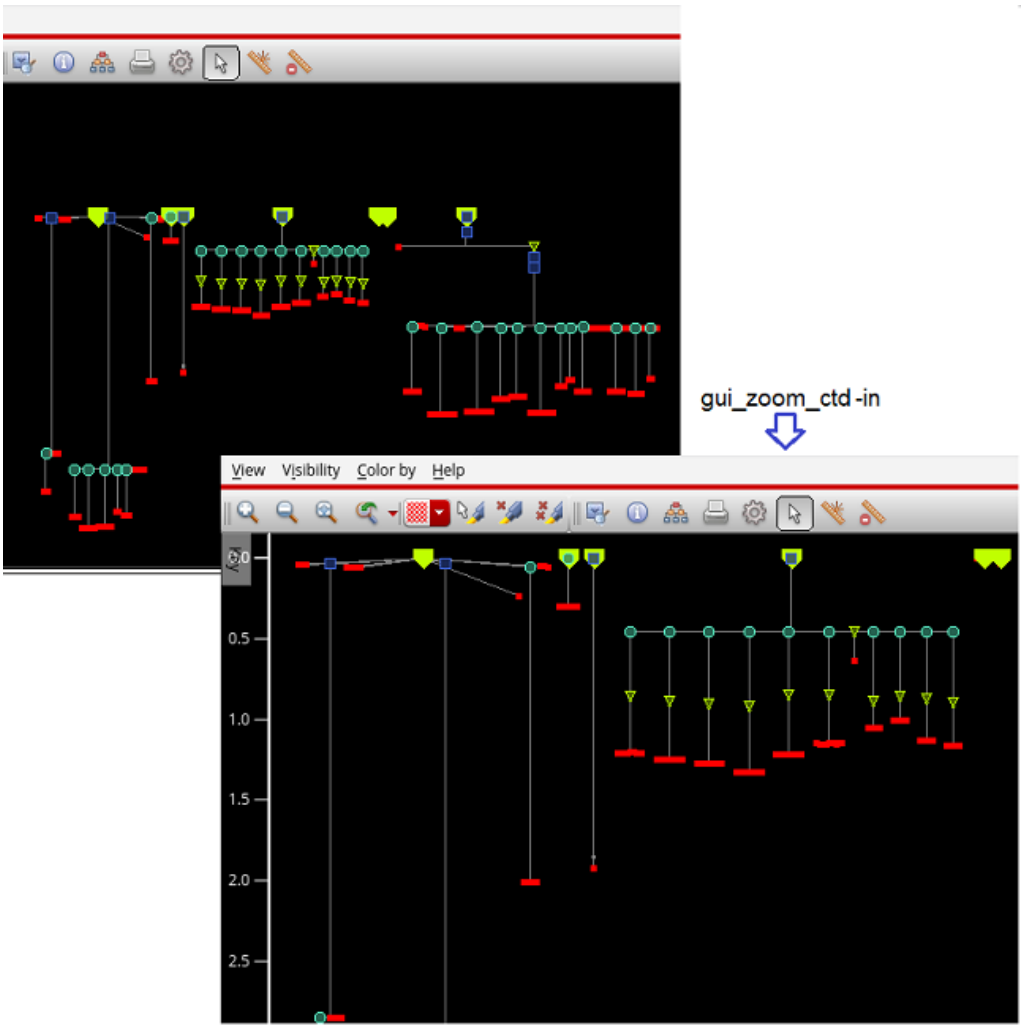
Parameters

`-help`	Prints a brief description of the `gui_zoom_ctd` command. For a detailed description of the command, use the `man` command: `man gui_zoom_ctd`
`-id string`	Specifies the window ID of the CTD window in which to zoom in or zoom out.
`-in`	Zoom into the CTD window.
`-out`	Zoom out of the CTD window.
`-selected`	Zoom to enclose all selected objects.
`-vertical_range {ymin ymax}`	Zoom to a specific vertical latency range. If CTD is in timing-based mode, the window will zoom to the clock tree parts where latency is between *ymin* and *ymax*, and the unit is the same as global timing unit. If CTD is in unit delay mode, the window will zoom to the clock tree parts where stage depth is between *ymin* and *ymax*. For example, if CTD is in timing-based mode, then the command, `gui_zoom_ctd -vertical_range {0 1.5}` will zoom to the clock tree parts where latency is between 0 and 1.5, the unit is the same as `get_time_unit`. If CTD is in unit delay mode, then the command, `gui_zoom_ctd -vertical_range {0 1.5}` will zoom to the clock tree parts where stage depth is between 0 and 1.5. **Note**: The window only zooms in vertical.

Examples

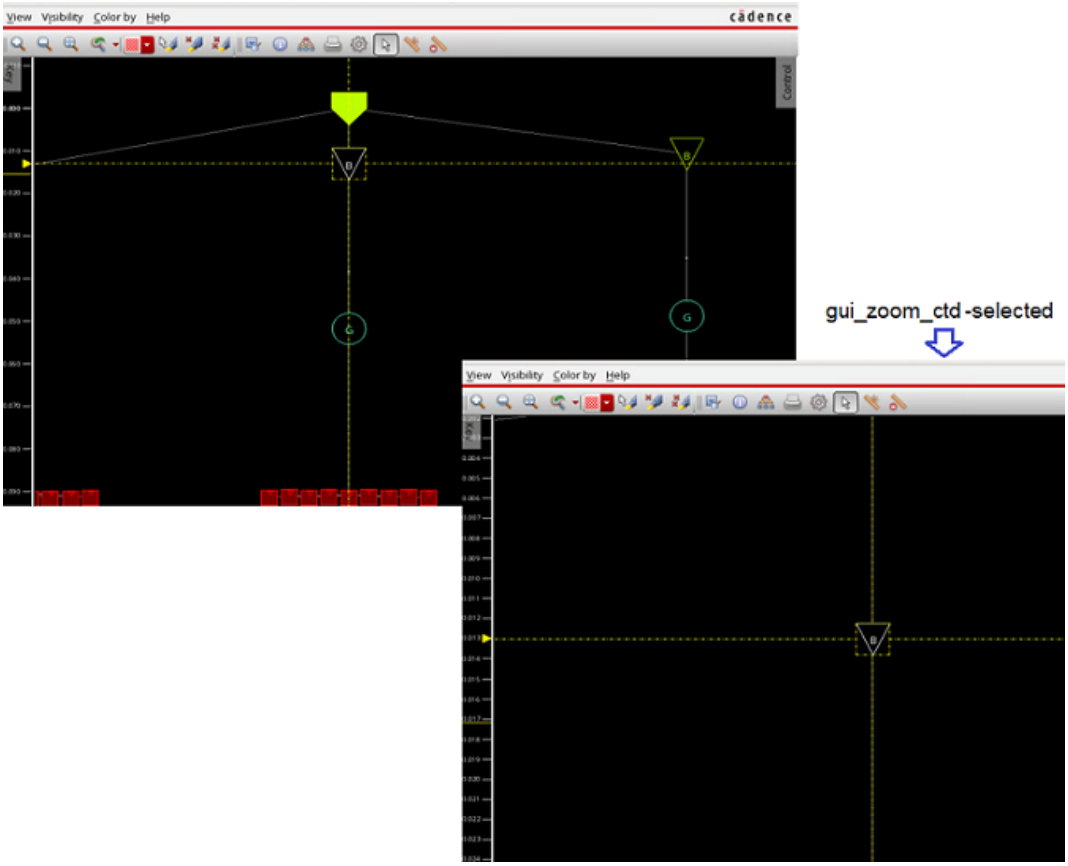
Use the following command to zoom in to the CTD:

```
gui_zoom_ctd -in
```



Select an instance and use the following command to zoom into the selected instance:

```
gui_zoom_ctd -selected
```



Related Information

- close_ctd_win
- ctd_trace
- ctd_win
- get_ctd_win_title
- get_ctd_win_id
- set_ctd_win_title
- Clock Menu chapter of the *Innovus Menu Reference*.

merge_clock_cells

```
merge_clock_cells
[-help]
[-only_above_flops]
[-only_clock_gates]
[-only_clock_logics]
```

When specified without any parameters, the command merges both clock gates and clock logic. You can specify the parameters, `-only_clock_gates` or `-only_clock_logics` to merge only clock gates or clock logics, respectively. The command ignores the settings of the `cts_merge_clock_logic and cts_merge_clock_gates` properties.

The log will show the names of instances that will be merged by saying, "Merging clock gates...", or "Merging clock logics", or "Merging clock gates and clock logics".

The summary tables in the log will list the number of instances that are merged. Running this command also sets the `cannot_merge_reason` property on the instances that could not be merged so the summary tables in the log will also provide reasons for why some instances could not be merged. This is shown in the "Examples" section below.

For details of all the above-mentioned properties, see the CCOpt Properties chapter of the *Innovus User Guide*.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `merge_clock_cells` parameter. For a detailed description of the command and all of its parameters, use the man command: `man merge_clock_cells`
`-only_above_flops`	Specifies that CCOpt should merge clock gates that only drive flops and not the ones that drive RAMs, latches, black boxes, or any other clock tree sink that is not a flop.
`-only_clock_gates`	Specifies that only clock gates should be merged and not clock logics.
`-only_clock_logics`	Specifies that only clock logics should be merged and not clock gates.

Examples

- When you run the following command, the summary table that is generated is shown below:
  ```
  merge_clock_cells -only_clock_gates
  ```

  Summary of the merge of duplicate siblings
  ```
  ------------------------------------------------------------
  Description                          Number of occurrences
  ------------------------------------------------------------
  Total clock gates                              2
  Globally unique enables                        1
  Potentially mergeable clock gates              1
  Actually merged clock gates                    0
  ------------------------------------------------------------
  ------------------------------------------------------
  Description                       Number of occurrences
  ------------------------------------------------------
  Total clock logics                         2
  Actually merged clock logics               0
  ------------------------------------------------------
  ------------------------------------------------------------
  Cannot merge reason                  Number of occurrences
  ------------------------------------------------------------
  UniqueUnderParent                              1
  ClockLogicMergingDisabledOnTree                2
  ------------------------------------------------------------
  ```
  In the above example, the first table is for clock gates – there are two in this design. No clock gates were merged.
  The second table is for clock logics – there are two in this design. No clock gates were merged.
  The third table provides the reasons as to why the instances could not be merged. Logic merging has been disabled because of the –only_clock_gates parameter. If the logics cannot be merged, then the clock gates are not siblings and cannot be merged.

- When you run the following command, the summary table that is generated is shown below.
  ```
  merge_clock_cells
  ```

  ```
  Summary of the merge of duplicate siblings
  ------------------------------------------------------
  Description                        Number of occurrences
  ------------------------------------------------------
  Total clock gates                           2
  Globally unique enables                     1
  Potentially mergeable clock gates           1
  Actually merged clock gates                 1
  ------------------------------------------------------
  ------------------------------------------------
  Description                      Number of occurrences
  ------------------------------------------------
  Total clock logics                        2
  Actually merged clock logics              1
  ------------------------------------------------
  ---------------------------------------------
  Cannot merge reason     Number of occurrences
  ---------------------------------------------
  UniqueUnderParent                1
  ---------------------------------------------
  ```

In the above example, the clock logics got merged - one logic got merged with its sibling logic. This means that the clock gates became siblings and were merged.

Related Information

- `get_ccopt_property`
- `set_ccopt_property`
- The following properties:
  - `clock_gating_only_optimize_above_flops`
  - `cannot_merge_reason`

For retrieving information about the above properties in the software, use the following command:
`get_ccopt_property -help property_name`

modify_ccopt_skew_group

```
modify_ccopt_skew_group
[-help]
[-make_exclusive]
-skew_group skew_group_name
[-add_sinks pins | -remove_sinks pins]
[-add_ignore_pins pins | -remove_ignore_pins pins]
```

This command is used internally to add or remove sinks in existing skew groups produced automatically by the `create_ccopt_skew_group` command, but you can also use it to modify existing skew_group objects. Sinks and ignore pins can be added or removed using the appropriate switches.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `modify_ccopt_skew_group` parameter. For a detailed description of the command and all of its parameters, use the man command: `man modify_ccopt_skew_group`
`-add_ignore_pins pins`	Specifies the ignore pins to be added to the skew group.
`-add_sinks pins`	Specifies the sink pins to be added to the skew group.
`-make_exclusive`	When specified, sets the `exclusive_sinks_rank` property of the specified skew group (using the -skew_group parameter) to a value of one more than the current exclusive_sinks_rank of all skew groups sharing the current skew group mode.
`-remove_ignore_pins pins`	Specifies the ignore pins to be removed from the skew group.
`-remove_sinks pins`	Specifies the sink pins to be removed from the skew group.
`-skew_group skew_group_name`	Specifies the user-defined identifier for the skew group to be modified.

Example

- The following command specifies the sink pins to be added to the specified skew group:
  ```
  modify_ccopt_skew_group -skew_group clkA -add_sinks f1/ck
  ```

Related Information

- `create_ccopt_skew_group`
- `get_ccopt_property`
- `get_ccopt_skew_groups`
- `get_ccopt_skew_group_delay`
- `get_ccopt_skew_group_path`
- `delete_ccopt_skew_groups`
- `report_ccopt_skew_groups`
- `set_ccopt_property`
- The following topics in the *Innovus User Guide*
  - Automatic Clock Tree Specification Creation
  - Manual Setup and Adjustment of the Clock Tree Specification

preserve_ccopt_port

```
  preserve_ccopt_port
[-help]
{[portname [-location {x y} | -reset ]] | -reset_all  | [-list  [-file filename]] | -tcl_list}
```

This command preserves module ports for the flexible h-tree feature. It can be used on any module port in the clock tree. It will preserve the specified module ports in both, clock tree synthesis and flexible h-tree synthesis. Any sinks connected downstream of the ports will remain downstream, and only those ports will be considered downstream.

Note: Currently, physical locations are only supported for flexible h-tree synthesis, and only on partition boundaries, although they will be accepted on all other module ports. Also, ports belonging to hInsts that have siblings but are not defined as partitions are not supported.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `preserve_ccopt_port` parameter. For a detailed description of the command and all of its parameters, use the man command: `man preserve_ccopt_port`
`-file` *`filename`*	Specifies the name of the preserved port list report.
`-list`	Prints a list of currently preserved ports.
`-location`	Provides the location of the preserved port specified using the *`portname`* parameter.
`portname`	Specifies the name of the port.
`-reset`	Resets the preservation settings for the preserved port specified using the *`portname`* parameter.
`-reset_all`	Resets all port preservation settings.
`-tcl_list`	Returns a tcl list of currently preserved ports. **Note**: The parameters, `-list` and `-tcl_list` are mutually exclusive.

Examples

- Use the following command to mark a port on a partition boundary as preserved and set the location:
  ```
  preserve_ccopt_port m0/ck -location {100 100}
  ```
- Use the following command to see the report, which shows both the port that was set and its sibling:
  ```
  preserve_ccopt_port -list
  Port m0/ck fixed at (100.000,100.000)
  Sibling port m1/ck fixed at (900.960,100.000)
  ```
- Use the following command to view the report of preserved ports as a tcl list:
  ```
  preserve_ccopt_port -tcl_list
  ```
  The software returns the following information:
  ```
  {m0/ck 100.0 100.0} {m1/ck 900.96 100.0}
  ```

- Use the following command to reset the port, in this case by specifying the sibling:
  ```
  preserve_ccopt_port -reset m1/ck
  ```
  Now both ports have been cleared
  ```
  preserve_ccopt_port -list
  ```
  The software returns nothing.

Related Commands

- create_ccopt_flexible_htree
- delete_ccopt_flexible_htrees
- get_ccopt_flexible_htrees
- synthesize_ccopt_flexible_htrees

report_ccopt_cell_filtering_reasons

```
report_ccopt_cell_filtering_reasons
[-help]
[-cell_type buffer | inverter | logic | delay | clock_gate]
[-clock_trees {string1 string2 ...}]
[-file file_name]
[-power_domain power_domain_name]
[-tcl_list]
```

Reports the reasons for filtering the specified cell types. The report can be saved as a file by specifying the `-file` parameter of the command or it can be viewed as a Tcl list by specifying the `-tcl_list` parameter. The output of the command is also included in the log.

In the report, "all" is used for the Clock tree names where the reason is common to all clock_trees. Some of the reasons for filtering that are reported and their brief descriptions are provided below:

- `Unbalanced rise/fall delays` : The difference between rise and fall delays for the cell is bigger than 20%, or any of rise or fall timing arcs is missing in the library.
- `Too tall`: The cell height is bigger than the maximum cell height specified using the `max_cell_height` property.
- `Wrong power context` : The cell does not match the power context requirements. For example, do not have same power context on all pins.
- `Not available in all views` : The cell is not available in all analysis views because the library does not contain all the specified views (corner cases) in the design for that specific cell. To avoid this filtering, check the libraries and the MMMC configuration to see why the cell is not available in some views.
- `Always on but not in cpf`: The cell is marked as always on but does not match the CPF description. You can avoid this filtering by checking/updating the CPF.
- `Weak max cap cell`: The cell is unable to drive enough capacitance at certain clock frequencies. This check is controlled by the CCOpt property, `frequency_dependent_max_cap_usability_check_max_cap_fanout_factor`. The default value of this property is 4. The cells that cannot drive four instances of that cell are filtered. You can reduce this value to avoid the filtering, for example, by relaxing the requirement so that the cell only has to be able to drive one or two instances. Alternatively, since this is controlled by the period on the clock_tree, you can check the `effective_clock_period` property to see if those are correct.
- `Library trimming`: The cell was removed automatically because it is an inefficient cell. The cell was removed as an inefficient cell to reduce the library size and save runtime. To avoid removing such cells, disable library trimming.
- `Can't use due to library mismatch`: Check the library data for the cell - both timing and physical data for the cell are needed. There is a mismatch between LEF and liberty libraries because the cell is missing in one of them.
- `Invalid for balancing`: The cell is filtered out because it does not match several predefined properties for balancing, such as: 1) cell delay is too small; 2)cell has an invalid (rise/fall) timing arc.
- `No test enable signal`: The clock gate cell is filtered out because it does not have a test enable input.
- `Has test enable signal`: The clock gate cell is filtered out because it has a test enable input.
  Note: The above two cell filtering reasons are context dependent. The software filters all clock gates, reporting the clock gates with test enables as one set and the clock gates without test enables as another set. A clock gate either has a test enable or it does not have one, so all clock gates are listed in one of the two sets. However, if you need a clock gate with no test enable signal, but there are no such clock gates, you can use a clock gate that has a test enable signal. But the other way around is not possible, which means if you need a clock gate with test enable signal but there is no such clock gate, you cannot use one that does not have a test enable signal.

- `Has observable output`: Clock gates without observability outputs are preferred, if they are available. Normally any cells with observability outputs will be filtered out. But if all the available cells have observability outputs they will not be filtered.
- `Edge sensitive`: The clock gate cell is filtered out because it is not positive edge sensitive.
- `Cannot be legalized`: The cell is filtered because it cannot physically be legalized in the power domain.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_ccopt_cell_filtering_reasons` parameter. For a detailed description of the command and all of its parameters, use the man command: man `report_ccopt_cell_filtering_reasons`.
`-cell_type buffer | inverter | logic | delay | clock_gate`	
	Restrict the report the specified library cell type.
`-clock_trees {string1 string2 ...}`	
	Restricts the report to the specified clock trees
`-file` *file_name*	Specifies that a copy of the contents of the report will be saved to the specified file name.
`-power_domain` *power_domain_name*	Restricts the report to the specified power domain.
`-tcl_list`	Returns the report as Tcl formatted data, and formats the `-file` as tcl, if specified.

Examples

- Use the following command to filter reasons for clock gate cell types:

```
report_ccopt_cell_filtering_reasons -cell_type clock_gate

Filtering reasons for cell type: clock gate cell
================================================

-------------------------------------------------------------------------------------------------
Clock trees Power domain   Reason                  Library cells
-------------------------------------------------------------------------------------------------
{ clk4 }    auto-default   No test enable signal  { TLATNCAX12 TLATNCAX16 TLATNCAX2 TLATNCAX20
                                                    TLATNCAX3 TLATNCAX4 TLATNCAX6
                                                    TLATNCAX8 }
{ clk4 }    auto-default   Has test enable signal { TLATNTSCAX12 TLATNTSCAX16 TLATNTSCAX2
                                                    TLATNTSCAX20 TLATNTSCAX3 TLATNTSCAX4
                                                    TLATNTSCAX6 TLATNTSCAX8 }
-------------------------------------------------------------------------------------------------
```

- Use the following command to filter reasons for buffer cell types:

```
report_ccopt_cell_filtering_reasons -cell_type buffer

Example output format:

Filtering reasons for cell type: buffer
=======================================

-----------------------------------------------------------------------------
Clock trees   Power domain   Reason             Library cells
-----------------------------------------------------------------------------
all           AO             Library trimming  { BUFFD12BWP BUFFD2BWP BUFFD3BWP
                                                   BUFFD4BWP BUFFD6BWP BUFFD8BWP
                                                   PTBUFFD2BWP }
all           PLL            Unbalanced        { BUFFD12BWP BUFFD2BWP BUFFD3BWP
                             rise/fall delays    BUFFD4BWP BUFFD6BWP BUFFD8BWP
                                                 PTBUFFD2BWP }
-----------------------------------------------------------------------------
```

- Use the following command to filter reasons for inverter cell types:

```
report_ccopt_cell_filtering_reasons -cell_type inverter

Example output format:

Filtering reasons for cell type: inverter
=========================================

-----------------------------------------------------------------------------
Clock trees   Power domain   Reason             Library cells
-----------------------------------------------------------------------------
all           AO             Unbalanced         { INVD12BWP INVD16BWP INVD1BWP
                             rise/fall delays     INVD2BWP INVD3BWP INVD4BWP
                                                  INVD6BWP INVD8BWP }
all           PLL           Library Trimming    { INVD12BWP INVD16BWP INVD1BWP
                                                  INVD2BWP INVD3BWP INVD4BWP
                                                  INVD6BWP}
-----------------------------------------------------------------------------
```

Related Information

- The following properties:
  - `max_cell_height`
  - `frequency_dependent_max_cap_usability_check_max_cap_fanout_factor`
  - `effective_clock_period`

  For details, see the CCOpt Properties chapter in the *Innovus User Guide*.
- The following topic in the *Innovus User Guide*
  - Generating Reports in Clock Tree Synthesis

report_ccopt_cell_halo_violations

```
report_ccopt_cell_halo_violations
[-help]
[-add_markers | -clear_markers]
[-file fileName]
[-num number]
[-summary]
```

This command is used to report clock instances with clock halo violations.

By default, when the command is specified without any arguments, it will check for halo violations for clock halos defined using all of the following CCOpt property pairs:

- `cell_halo_x` / `cell_halo_y`
- `cell_density` / `adjacent_rows_legal`
- `cell_halo_sites` / `cell_halo_rows`

The report can be viewed as a text file and in the Violation Browser within the Innovus GUI.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_ccopt_cell_halo_violations` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_ccopt_cell_halo_violations`
`-add_markers`	Adds any detected clock halo violations to the Violation Browser.
`-clear_markers`	Removes any markers added to the Violation Browser by the `report_ccopt_cell_halo_violations` command.
`-file` *`fileName`*	Specifies the name of the file in which the contents of the report will be saved.
`-num` *`number`*	Specifies the number of violations to report for each clock.
`-summary`	Prints a report providing information about the instances with cell halos and the instances with cell halo violations or "Failed instances". This report contains a summary table titled, "Clock Cell Halo Rule Check in Summary".

Example

The following command prints the summary of the cell halo violation report:

```
report_ccopt_cell_halo_violations -summary
```

The software prints the following report:

```
Clock Cell Halo rule Check in Summary
=====================================

-------------------------------------------------------
Clock       Instances with cell_halo Failed Instances
-------------------------------------------------------
```

```
funcclk1            4                   3
funcclk2            4                   3
funcclk3            4                   2
-------------------------------------------------------
total number of violating instances: 8
```

Use the following command to view the detailed "Clock Cell Halo Rule Violations" table.

```
report_ccopt_cell_halo_violations
```

The software prints the following report:

```
Clock Cell Halo Rule Violations
*******************************
```

No.	Cell	Instance	In clock	Power domain	Cell Halo (x, y)	Violating instance	From clock	Intrusion (x, y)
1	INVD1BWP	RESULTS_CONV_INST/g18692	m_rcc_clk	TD8PCore	(3, 0)	RESULTS_CONV_INST/RC_CG_HIER_INST8/RC_CGIC_INST	m_rcc_clk	(2.94, 1.26)
2	INVD1BWP	RESULTS_CONV_INST/g18693	m_rcc_clk	TD8PCore	(3, 0)	RESULTS_CONV_INST/RC_CG_HIER_INST9/RC_CGIC_INST	m_rcc_clk	(2.94, 1.26)
3	INVD1BWP	RESULTS_CONV_INST/g18688	m_rcc_clk	TD8PCore	(3, 0)	RESULTS_CONV_INST/RC_CG_HIER_INST8/RC_CGIC_INST	m_rcc_clk	(2.58, 1.26)

Note: The "Power Domain" column helps to understand where the halo value comes from in the case where the halo has been set per power domain.

Use the following set of commands to view the clock halo violations in the Violation Browser

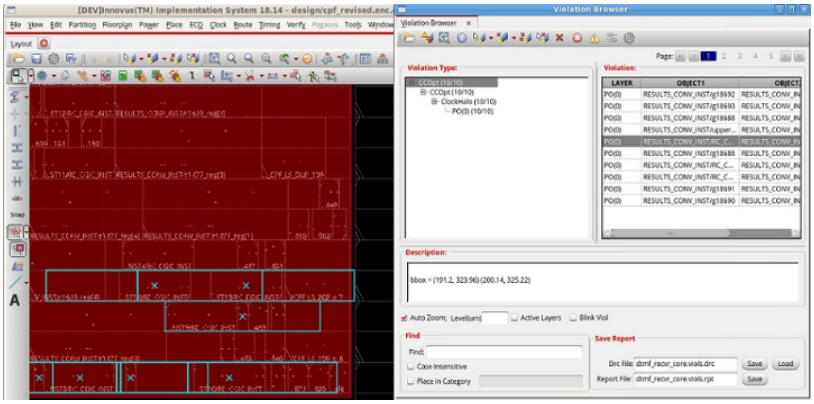
Create cell halo violations using the following property:

```
set_ccopt_property cell_halo_x 3
```

Use the following command to add the detected clock halo violations to the Violation Browser and display them in the GUI:

```
report_ccopt_cell_halo_violations -add_markers
```

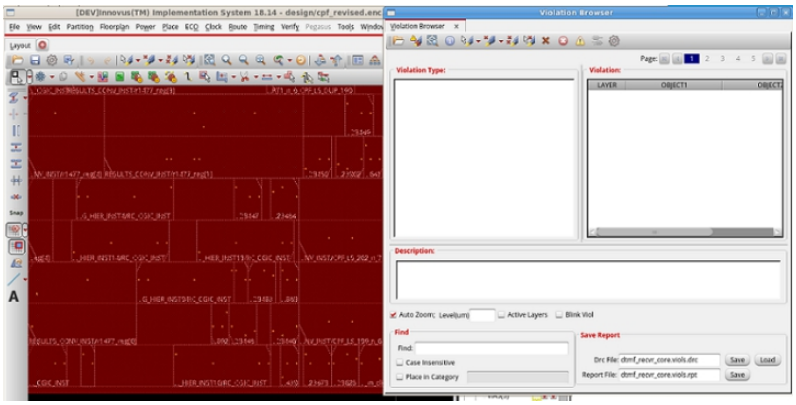
You will see the following:



Use the following command to clear the clock halo violations in the Violation Browser and delete markers in the GUI:

```
report_ccopt_cell_halo_violations -clear_markers
```

You will see the following:



Related Information

The following properties:

- `cell_halo_rows`
- `cell_halo_sites`
- `cell_halo_x`
- `cell_halo_y`

For details, see the CCOpt Properties chapter in the *Innovus User Guide*.

The following topic in the *Innovus User Guide*

- Clock Cell Halo Violations Reporting

report_ccopt_clock_tree_convergence

```
report_ccopt_clock_tree_convergence
[-help]
[-file fileName]
```

This command is used to report a summary of the convergence above clock sinks and to identify the names of sinks with the highest number of clock path networks. If there are sinks with many paths leading to them, there can be a considerable impact on runtime. Reducing the complexity of the logic, or excluding sinks that need not have timing constraints from the clock trees can help improve runtime. An example of the report is shown below.

```
Convergence above clock sinks
=============================
-----------------------------------------------------------------
Number of paths to sink PIOs DFFs  Other sinks Total
-----------------------------------------------------------------
1                       1    10118 0          10119
8                       54   2134  183        2371
16                      13   311   18         342
32                      2    0     0          2
700                     3    164   5          172
1400                    5    0     0          5
-----------------------------------------------------------------
Sinks with most paths leading to them
=====================================
---------------------------------
Sink             Number of paths
---------------------------------
Owest/nout[0]    1400
Owest/nout[1]    1400
Owest/nout[2]    1400
Owest/nout[3]    1400
Owest/nout[4]    1400
---------------------------------
---------------------------------
```

In the above sample report, 5 primary IOs have 1400 paths leading to them from the clock sources.The report also shows a table with the names of sinks with most paths leading to them. In the "Sinks with most paths leading to them" section of the report, the five sinks, `Owest/nout[n]`, have 1400 paths leading to them. These sinks are likely to be problematic and need to be checked.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_ccopt_clock_tree_convergence` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_ccopt_clock_tree_convergence`
`-file` *`fileName`*	Specifies the file name in which the report will be saved.

Related Information

- `check_ccopt_clock_tree_convergence`
- `create_ccopt_clock_tree_spec`
- The following topic in the *Innovus User Guide*
  - Clock Tree Convergence Reporting

report_ccopt_clock_tree_drv

```
report_ccopt_clock_tree_drv
[-help]
[-clock_trees {list_of_clock_trees}]
[-delay_corners list_of_delay_corner_names]
[-drv_types {transition max_cap max_source_to_sink_net_length}]
[-early]
[-exclude_violation_types {dont_touch_nets pre_routed_nets}]
[-file file_name]
[-late]
[-num_nets num | all]
[-views {list_of_views}]
```

This command is used to report design rule violations (DRVs) in the clock network.

The report consists of three sections:

1. Constraint summary: Lists all the implicit and explicit targets that CTS is targetting, along with which corners are being constrained.
2. Violation summary: Counts nets in the DAG with what types of violations exist.
3. Top N detailed net violation listing: Outputs a detailed list of nets that have violations.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_ccopt_clock_tree_drv` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_ccopt_clock_tree_drv`
`-clock_trees {`*`list_of_clock_trees`*`}`	Restricts the report to the specified clock trees. By default, it reports DRVs for all clock trees.
`-delay_corners` *`list_of_delay_corner_names`*	Filters the report for a specific constrained delay corner. By default, it reports for all delay corners.
`-drv_types {transition max_cap max_source_to_sink_net_length}`	Filters to choose the types of violation to report on. By default, all types of violations are reported.
`-early`	Reports only the early half of the delay corners.
`-exclude_violation_types {dont_touch_nets pre_routed_nets}`	Specifies certain nets to be excluded if they are known to be not fixable. By default, all nets are included. You can choose to exclude dont_touch nets and pre-routed nets.
`-file file_name`	Saves a copy of the contents of the report to the specified file name. *Default*: `{}`
`-late`	Reports only the late half of the delay corners.
`-num_nets num | all`	Specifies the number of nets for which the violations should be reported. You can specify a number or specify all to report for all nets. By default, violations are rpeorted for 10 nets.
`-views {`*`list_of_views`*`}`	Restricts the report to the delay corners associated with the specified views.

Example

Use the following command to report the violations for all nets in the specified report file:

```
report_ccopt_clock_tree_drv -num_nets all -out_file report_file1
```

For sample report, see Clock Tree Network DRV Reporting.

Related Information

- `create_ccopt_clock_tree_spec`
- Clock Tree Synthesis chapter in the *Innovus User Guide*.

report_ccopt_clock_tree_structure

```
report_ccopt_clock_tree_structure
[-help]
[-check_type {setup | hold}]
[-clock_trees {list_of_clock_trees}]
[-delay_corner corner_name]
[-delay_type {early | late}]
[-expand_below_logic]
[-expand_generated_clock_trees {independently | inline | independently_and_inline}]
[-include_reporting_only_skew_groups]
[-file file_name]
[-show_sinks]
[-update_timing]
```

Reports the structure of the clock network as a text report. When the command is run without specfying the `-update_timing` parameter, the report format is as follows:

- The summary information is added below the name of the clock tree
- The slew, capacitance, and co-ordinate information is not included
- The input and output pin information is merged in a single line

For details, see the examples below.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_ccopt_clock_tree_structure` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_ccopt_clock_tree_structure`
`-check_type {setup | hold}`	Specifies whether setup or hold views are used for the report. By default, setup view is used for the report.
`-clock_trees {list_of_clock_trees}`	Restricts the report to the specified clock trees.
`-delay_corner corner_name`	Reports in the specified delay corner. By default, the software will use the primary-half delay corner that is set using the CCOpt property, `primary_delay_corner`. Use the `get_ccopt_delay_corner` command to retrieve the name of the CCOpt primary-half delay corner.
`-delay_type {early | late}`	Reports in the specified delay type, early or late. When late is specified, only the late half of the delay corners are reported. When early is specified, only the early half of the delay corners are reported. By default, late half of the delay corners are reported.

`-expand_below_logic`	When this parameter is specified, the subtree below each multi-input logic will be printed in its entirety every time it is encountered. When this parameter is not specified, then by default, the clock tree below a multi-input clock logic is reported only once, which is at the first place it is encountered in the report. Subsequent visitations of the subtree will simply note the presence of an omitted subtree, for example, "...71 `fanout omitted`". **Note**: For clock networks with ladders of reconverging clock logic, this can produce large volumes of output.
`-expand_generated_clock_trees {independently | inline | independently_and_inline}`	
	Specifies how generated clock trees should be displayed. The following options are available: `independently`: display generated clock trees at the top level. This is the default display. `inline` : display generated clock trees as fanout of their generator inputs `independently_and_inline`: display generated clock trees both at the top level and as fanout of their generator inputs
`-file` *file_name*	Saves a copy of the report in the specified file. *Default*: `{}`
`-include_reporting_only_skew_groups`	When specified, the report additionally includes the reporting-only skew groups. A reporting-only skew group is a skew group whose `constrains` property is set to `none`. Such a skew group imposes no clock balancing constraint and will not be considered by CTS. By default, the reporting-only skew groups are not included in the report.
`-show_sinks`	Includes all clock network sinks in the report. This parameter controls whether the clock network sinks, which are the non-generator flops/latches, are included in the report or not. By default, these are not included in the interest of minimizing the report output. Instead, a count of the number of sinks is displayed,for example. `"... 209 sinks omitted"`. When this parameter is specified, per-skew group latencies for each sink are included. **Note**: The report identifies macros in the clock tree. This means that if your clock sink is a macro, the report will show “macro sink” instead of just “sink”.
`-update_timing`	When specified, the report includes detailed timing information including pin slew, capacitance, and location along with the clock tree structure.

Examples

- Use the following command to report the structure of the clock network for the specified clock trees, `m_clk` and `m_digit_clk`:

  ```
  report_ccopt_clock_tree_structure -clock_trees {m_clk m_digit_clk}
  ```

  A part of the report is shown below.

  ```
  Clock tree m_clk:
  Total FF: 115
  ```

```
Max Level: 5
  (L1) TEST_CONTROL_INST/g137/ZN (ND2D1BWP)
  \_  (L1) port TEST_CONTROL_INST/m_clk
       \_ ... (21 sinks omitted)
       \_  (L2) DMA_INST/CPF_LS_158_m_clk/I -> Z (LVLHLD2BWP)
        |   \_ ... (8 sinks omitted)
        |   \_  (L3) DMA_INST/RC_CG_HIER_INST0/RC_CGIC_INST/CP -> Q (CKLNQD1BWP)
        |       \_ ... (8 sinks omitted)
        \_  (L2) RESULTS_CONV_INST/CPF_LS_159_m_clk/I -> Z (LVLHLD2BWP)
        |   \_ ... (15 sinks omitted)
        |   \_  (L3) RESULTS_CONV_INST/RC_CG_HIER_INST1/RC_CGIC_INST/CP -> Q (CKLNQD1BWP)
        |   |   \_ ... (9 sinks omitted)
        |   \_  (L3) RESULTS_CONV_INST/RC_CG_HIER_INST2/RC_CGIC_INST/CP -> Q (CKLNQD1BWP)
        |   |   \_ ... (16 sinks omitted)
```

.....

```
Clock tree m_digit_clk:
 Total FF: 9
 Max Level: 3
   (L1) TEST_CONTROL_INST/g141/ZN (IOA21D1BWP)
   \_  (L1) port TEST_CONTROL_INST/m_digit_clk
        \_ ... (9 sinks omitted)
```

- Use the following command to report the structure of the clock network inlcuding the timing information for the specified clock tree, `m_clk`:

```
report_ccopt_clock_tree_structure -clock_trees {m_clk} -update_timing
```

```
Clock tree m_clk:
TEST_CONTROL_INST/g137/ZN root output at (273.210,301.000), lib_cell ND2D1BWP, level 1, slew 9.719ns, wire_cap
0.096pF, load_cap 0.452pF
\_ port TEST_CONTROL_INST/m_clk port output, preserved by {sdc_constraint}, level 1, slew 0.000ns, wire_cap
0.000pF, load_cap 0.000pF
   \_ ... (21 sinks omitted)
   \_ DMA_INST/CPF_LS_158_m_clk/I logic input at (169.210,300.720), lib_cell LVLHLD2BWP, level 2, slew 9.720ns
   | DMA_INST/CPF_LS_158_m_clk/Z logic output at (169.770,300.860), lib_cell LVLHLD2BWP, level 2, slew
0.431ns, wire_cap 0.008pF,
     load_cap 0.006pF
   | \_ ... (8 sinks omitted)
   | \_ DMA_INST/RC_CG_HIER_INST0/RC_CGIC_INST/CP cgate input at (115.450,305.760), lib_cell CKLNQD1BWP, level
3, slew 0.431ns
   |    DMA_INST/RC_CG_HIER_INST0/RC_CGIC_INST/Q cgate output at (116.290,305.900), lib_cell CKLNQD1BWP, level
3, slew 0.298ns,
        wire_cap 0.004pF, load_cap 0.005pF
   |    \_ ... (8 sinks omitted)
   \_ RESULTS_CONV_INST/CPF_LS_159_m_clk/I logic input at (199.170,320.880), lib_cell LVLHLD2BWP, level 2,
slew 9.720ns
   | RESULTS_CONV_INST/CPF_LS_159_m_clk/Z logic output at (199.730,321.020), lib_cell LVLHLD2BWP, level 2,
slew 0.714ns,
     wire_cap 0.021pF, load_cap 0.015pF
```

- Use the following command to report the structure of the clock network with generated clock trees expanded inline for the specified clock trees, `m_clk` and `m_digit_clk`:

```
report_ccopt_clock_tree_structure -clock_trees {m_clk m_digit_clk} -expand_generated_clock_trees inline
```

```
Clock tree m_clk:
Total FF: 115
```

```
Max Level: 5
(L1) TEST_CONTROL_INST/g137/ZN (ND2D1BWP)
\_  (L1) port TEST_CONTROL_INST/m_clk
   \_ ... (21 sinks omitted)
   \_   (L2) DMA_INST/CPF_LS_158_m_clk/I -> Z (LVLHLD2BWP)
   |    \_ ... (8 sinks omitted)
   |    \_ (L3) DMA_INST/RC_CG_HIER_INST0/RC_CGIC_INST/CP -> Q (CKLNQD1BWP)
   |        \_ ... (8 sinks omitted)
   \_   (L2) RESULTS_CONV_INST/CPF_LS_159_m_clk/I -> Z (LVLHLD2BWP)
   |    \_ ... (15 sinks omitted)
   |    \_ (L3) RESULTS_CONV_INST/RC_CG_HIER_INST1/RC_CGIC_INST/CP -> Q (CKLNQD1BWP)
   |   |    \_ ... (9 sinks omitted)
   |    \_ (L3) RESULTS_CONV_INST/RC_CG_HIER_INST2/RC_CGIC_INST/CP -> Q (CKLNQD1BWP)
   |   | \_ ... (16 sinks omitted)
   |    \_ (L3) RESULTS_CONV_INST/RC_CG_HIER_INST3/RC_CGIC_INST/CP -> Q (CKLNQD1BWP)
   |   |    \_ ... (16 sinks omitted)
   |    \_ (L3) RESULTS_CONV_INST/RC_CG_HIER_INST6/RC_CGIC_INST/CP -> Q (CKLNQD1BWP)
   |   |    \_ ... (6 sinks omitted)
   |    \_ (L3) RESULTS_CONV_INST/RC_CG_HIER_INST7/RC_CGIC_INST/CP -> Q (CKLNQD1BWP)
   |        \_ ... (8 sinks omitted)
   \_   (L2) SPI_INST/RC_CG_HIER_INST17/RC_CGIC_INST/CP -> Q (CKLNQD1BWP)
         \_ ... (8 sinks omitted)

Clock tree m_digit_clk:
Total FF: 9
Max Level: 3
(L1) TEST_CONTROL_INST/g141/ZN (IOA21D1BWP)
\_   (L1) port TEST_CONTROL_INST/m_digit_clk
     \_ ... (9 sinks omitted)
```

Related Information

- `get_ccopt_delay_corner`
- The following property:
  - `primary_delay_corner`

  For details, see the CCOpt Properties chapter in the *Innovus User Guide*.
- The following topic in the *Innovus User Guide*
  - Clock Tree Network Structure Reporting

report_ccopt_clock_trees

```
  report_ccopt_clock_trees
[-help]
[-clock_trees {list_of_clock_trees}]
[-delay_corners {list_of_delay_corners} | -views {list_of_views}]
[-file file_name]
[-histograms]
[-list_special_pins]
[-no_invalidate]
[-num_cap_violating_pins number_of_pins]
[-num_fanout_violating_nets number_of_nets]
[-num_length_violating_nets number_of_nets]
[-num_resistance_violating_nets number_of_nets]
[-num_slew_violating_pins number_of_pins]
[-skip_timing_update]
[-summary]
[-tcl_list]
[-early | -late]
```

This command reports a summary of all defined clock trees. In particular, this report provides a summary of numbers of clock gates at different depths in each clock tree. Use the `report_ccopt_clock_trees` command to monitor the amount of clock gating used in the design and the buffering used in the clock tree. You can run this command any time after defining or extracting clock trees.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_ccopt_clock_trees` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_ccopt_clock_trees`
`-clock_trees {list_of_clock_trees}`	
	Specifies that the report will be restricted to the specified clock trees.
`-delay_corners {list_of_delay_corners}`	
	Specifies that the report will be restricted to the specified delay corners. **Note**: The `-delay_corners` and `-views` parameters are mutually exclusive.
`-early`	Specifies that only the early half of the delay corners will be reported.
`-file file_name`	Specifies that a copy of the contents of the report will be saved to the specified file name.
`-histograms`	Specifies that histograms of key information will be displayed.
`-late`	Specifies that only the late half of the delay corners will be reported.
`-list_special_pins`	Specifies that the report will list individual pins that are stop, ignore, min, or other special type of pins.

`-no_invalidate`	Specifies that the timing engine should not be invalidated. The timing engine is used to calculate the timing data that is displayed in the report. By default, this is invalidated when you generate the report, so all the timing data is recalculated. This increases the time taken to generate the report. Use the `-no_invalidate` option to reuse the existing timing data for the report.
`-num_cap_violating_pins` *number_of_pins*	
	Specifies the maximum number of capacitance violating pins reported in the "Max capacitance violation summary across all clock trees" table of the report. By default, the software reports the top 10 violations.
`-num_fanout_violating_nets` *number_of_nets*	
	Specifies the maximum number of violating fanout nets reported in the "Max fanout violation summary across all clock trees" table of the report. By default, the software reports the top 10 violations.
`-num_length_violating_nets` *number_of_nets*	
	Specifies the maximum number of `max_source_to_sink_net_length` violating nets reported in the "Max length violation summary across all clock trees" table of the report. By default, the software reports the top 10 violations.
`-num_resistance_violating_nets` *number_of_nets*	
	Specifies the maximum number of `max_source_to_sink_net_resistance` violating nets reported in the "Max resistance violation summary across all clock trees" table of the report. By default, the software reports the top 10 violations.
`-num_slew_violating_pins` *number_of_pins*	
	Specifies the maximum number of worst slew violating pins that are reported in the "Slew violation summary across all clock trees" table. By default, the software reports the top 10 violations.
`-skip_timing_update`	Specifies that the timing engine should not be updated. The timing engine is used to calculate the timing data that is displayed in the report. By default, when you generate the report, the timing data is recalculated. This increases the time taken to generate the report. Specify this parameter to skip calculation of timing data.
`-summary`	Specifies that just the summary table should be displayed in the report.
`-tcl_list`	Specifies that the report will be created as Tcl data instead of a text report.
`-views {`*list_of_views*`}`	Restricts the report to the delay corners associated with the specified views. **Note**: The `-delay_corners` and `-views` parameters are mutually exclusive.

Examples

Example1: **Clock Timing Summary table**

```
Clock Timing Summary:
=====================

Target and measured clock slews (in ns):

-------------------------------------------------------------------------------------------------------------------------------------------------
Clock tree   Timing Corner                          Worst Rising  Worst Falling  Worst Rising  Worst Falling  Leaf Slew      Leaf Slew  Trunk Slew     Trunk Slew
                                                    Leaf Slew     Leaf Slew      Trunk Slew    Trunk Slew     Target Type    Target     Target Type    Target
-------------------------------------------------------------------------------------------------------------------------------------------------
m_clk        AV_HL_FUNC_MAX_RC1_dc:setup.early      9.673         10.980         9.668         10.974         ignored        -          ignored        -
m_clk        AV_HL_FUNC_MAX_RC1_dc:setup.late       9.706         10.990         9.701         10.985         auto computed  -          auto computed  -
m_clk        AV_HL_FUNC_MIN_RC1_dc:hold.early       4.361         4.325          4.349         4.310          ignored        -          ignored        -
m_clk        AV_HL_FUNC_MIN_RC1_dc:hold.late        4.379         4.328          4.368         4.313          ignored        -          ignored        -
m_digit_clk  AV_HL_FUNC_MAX_RC1_dc:setup.early      0.214         0.244          -             -              ignored        -          ignored        -
m_digit_clk  AV_HL_FUNC_MAX_RC1_dc:setup.late       0.229         0.246          -             -              auto computed  -          auto computed

* - indicates that target was not met.

auto extracted - target was extracted from SDC.
auto computed - target was computed when balancing trees.
```

Example2: Slew violation summary table across all clock trees for top 10 violations

```
Slew violation summary across all clock trees - Top 10 violating pins:
======================================================================

Target and measured clock slews (in ns):

-------------------------------------------------------------------------------------------------------------------------------------
Half corner                       Violation  Slew    Slew      Dont   Ideal  Target           Pin
                                  amount     target  achieved  touch  net?   source
                                                               net?
-------------------------------------------------------------------------------------------------------------------------------------
AV_HL_FUNC_MAX_RC1_dc:setup.late  10.171     0.814   10.985    N      N      liberty explicit  ARB_INST/present_state_reg[2]/CP
AV_HL_FUNC_MAX_RC1_dc:setup.late  10.171     0.814   10.985    N      N      liberty explicit  ARB_INST/present_state_reg[1]/CP
AV_HL_FUNC_MAX_RC1_dc:setup.late  10.171     0.814   10.985    N      N      liberty explicit  ARB_INST/dma_grant_reg/CP
AV_HL_FUNC_MAX_RC1_dc:setup.late  10.171     0.814   10.985    N      N      liberty explicit  ARB_INST/present_state_reg[0]/CP
AV_HL_FUNC_MAX_RC1_dc:setup.late  10.171     0.814   10.985    N      N      liberty explicit  ARB_INST/tdsp_grant_reg/CP
AV_HL_FUNC_MAX_RC1_dc:setup.late  10.171     0.814   10.985    N      N      liberty explicit  SPI_INST/present_state_reg[2]/CP
AV_HL_FUNC_MAX_RC1_dc:setup.late  10.171     0.814   10.985    N      N      liberty explicit  SPI_INST/present_state_reg[1]/CP
AV_HL_FUNC_MAX_RC1_dc:setup.late  10.171     0.814   10.985    N      N      liberty explicit  SPI_INST/dflag_reg/CP
AV_HL_FUNC_MAX_RC1_dc:setup.late  10.171     0.814   10.985    N      N      liberty explicit  SPI_INST/present_state_reg[0]/CP
AV_HL_FUNC_MAX_RC1_dc:setup.late  10.171     0.814   10.985    N      N      liberty explicit  SPI_INST/RC_CG_HIER_INST17/RC_CGIC_INST/CP
-------------------------------------------------------------------------------------------------------------------------------------

Target sources:
auto extracted - target was extracted from SDC.
auto computed - target was computed when balancing trees.
explicit - target is explicitly set via cts_target_max_transition_time property.
pin explicit - target is explicitly set for this pin via cts_pin_target_max_transition_time property.
liberty explicit - target is explicitly set via max_transition from liberty library.

Found 0 pins on nets marked dont_touch that have slew violations.
Found 0 pins on nets marked dont_touch that do not have slew violations.
Found 0 pins on nets marked ideal_network that have slew violations.
Found 0 pins on nets marked ideal_network that do not have slew violations.
```

Example3: Max capacitance violation summary table for top 4 violations

```
Max capacitance violation summary across all clock trees - Top 2 violations:
============================================================================

Target and measured capacitances (in pF):

-------------------------------------------------------------------------------------------------------------------------------------
--
Half corner                       Violation Capacitance Capacitance Target                Pin
                                  amount    target      achieved    source
-------------------------------------------------------------------------------------------------------------------------------------
--
AV_HL_FUNC_MAX_RC1_dc:setup.late 0.505    0.043       0.548    library_or_sdc_constraint
TEST_CONTROL_INST/g137/ZN
AV_HL_FUNC_MAX_RC1_dc:setup.late 0.127    0.041       0.169    library_or_sdc_constraint
TEST_CONTROL_INST/g142/ZN
-------------------------------------------------------------------------------------------------------------------------------------
--
```

```
Target sources:
target_max_capacitance_property - the target was set in the target_max_capacitance property
library_or_sdc_constraint - the non-frequency-dependent target was set in a library file or by an SDC constraint
frequency_dependent_library_or_sdc_constraint - the frequency-dependent target was set in a library file or by an
SDC constraint
computed_from_slew_target - the target was calculated based on the slew target at a clock root
clock_root_forced - the target was forced to be zero at a clock root


Found a total of 0 clock tree nets with max resistance violations.
Found a total of 0 clock tree nets with max length violations.
Found a total of 0 clock tree nets with max fanout violations.
Found a total of 279 clock tree pins with a slew violation.
```

Example4: Count of violations table

```
Count of violations across all clock trees:
==========================================

-------------------------------------------------------------------------------
Clock Tree  #Max capacitance #Max resistance #Max length #Max fanout #Slew
Name        violations        violations     violations  violations  violations
-------------------------------------------------------------------------------
m_clk        1               0              0           0           30
m_digit_clk  0               0              0           0           0
m_dsram_clk  1               0              0           0           2
m_ram_clk    1               0              0           0           2
m_rcc_clk    0               0              0           0           69
m_spi_clk    0               0              0           0           0
refclk       0               0              0           0           0
refclk<1>    1               0              0           0           176
-------------------------------------------------------------------------------
Total        4               0              0           0           279
-------------------------------------------------------------------------------

Note the above table per clock tree is based on CCOpt clock tree view. The violations are counted across half
corners.

Found a total of 4 clock tree pins with max capacitance violations.
Found a total of 4 max capacitance violations.
Total violation amount 0.838pF.
```

Example5: Clock Sink Summary Report

```
Clock Sink Summary:
===================

-------------------------------------------------------------------------------------------------------------------------------
Clock Tree  Explicit      Implicit      Explicit     Implicit     Explicit    Implicit    Posedge  Negedge  Memory  Enable  Non enable  Other
Name        exclude pins  exclude pins  ignore pins  ignore pins  stop pins   stop pins   Flops    Flops    Clock   Latch   Latch       Sinks
                                                                                                          Pins    Sinks   Sinks
-------------------------------------------------------------------------------------------------------------------------------
m_clk             0             0             1            0            0           0        113        0       0       0           0       2
m_digit_clk       0             0             0            0            0           0          9        0       0       0           0       0
m_dsram_clk       0             0             0            0            0           0          0        0       1       0           0       0
m_ram_clk         0             0             0            0            0           0          0        0       1       0           0       0
m_rcc_clk         0             0             0            0            0           0        129        0       0       0           0       0
m_spi_clk         0             0             0            0            0           0         12        0       0       0           0       0
refclk            0             0             1            0            0           0        258        0       0       0           0       1
refclk<1>         0             0             1            0            0           0        258        0       0       0           0       1
-------------------------------------------------------------------------------------------------------------------------------
```

```
Clock Sink Summary across all clock trees:
==========================================

-------------------------------------------------------------------------------------------------------------------------
Explicit      Implicit      Explicit      Implicit      Explicit    Implicit    Posedge   Negedge   Memory   Enable   Non enable   Other
exclude pins  exclude pins  ignore pins   ignore pins   stop pins   stop pins   Flops     Flops     Clock    Latch    Latch        Sinks
                                                                                                   Pins     Sinks    Sinks
-------------------------------------------------------------------------------------------------------------------------
     0             0             3             0             0           0         521       0         2        0        0            3
-------------------------------------------------------------------------------------------------------------------------
```

Related Information

- ccopt_design
- clock_opt_design
- create_ccopt_clock_tree
- delete_ccopt_clock_trees
- get_ccopt_clock_trees
- update_ccopt_clock_tree
- The following topic in the *Innovus User Guide*
  - Generating Reports in Clock Tree Synthesis

report_ccopt_pin_insertion_delays

```
  report_ccopt_pin_insertion_delays
[-help]
[-bin_size string]
[-check_type {setup | hold}]
[-clock_trees {string1 string2 ...}]
[-delay_corner delay_corner_name]
[-delay_type {early | late}]
[-file file_name]
[-skew_groups {string1 string2 ...} | -include_reporting_only_skew_groups]
[-sources {string1 string2 ...}]
```

Reports pin insertion delays at clock tree sinks. By default, the report visits all the clock tree sinks, and checks for pin insertion_delay at each sink. Statistics about the distribution of pin insertion_delay values are generated and reported. The report provides a quick overview of the distribution and nature of the pin insertion delays found, which can be used to identify the problem areas such as excessively large skews, surprisingly numerous skews, or large numbers of small skews, and so on.

The `-clock_trees` and `-skew_groups` parameters are used to restrict the scope of the report as a subset of the clock tree sinks, which means that when the `-clock_trees` parameter is specified, only those sinks that are under the listed clock trees are selected, and when the `-skew_groups` parameter is specified, only those sinks that are under the listed skew groups are selected. By default, the report analyzes the pin insertion delays that are found under the CCOpt primary half corner. The `-delay_corner`, `-check_type`, and `-delay_type` parameters permit the selection of another half corner to report against.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_ccopt_pin_insertion_delays` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_ccopt_pin_insertion_delays`
`-bin_size string`	Specifies a bin (bucket) size for the report histograms. By default the software uses the histogram bin size specified using the CCOpt property, pin_insertion_delay_histogram_bin_size. Use this parameter to override this default bin size value. For details, see CCOpt Properties in the *Innovus User Guide*.
`-check_type {setup | hold}`	
	Specifies the check type for the specified skew group to be reported. You can specify either `setup` or `hold`. If a check type is specified, but the `-delay_type` parameter is not specified, the software behaves in the following manner: • `-check_type -setup` will mean `-delay_type -late` • `-check_type -hold` will mean `-delay_type -early` **Note** : If a check type is not specified, the software will always look at what the views permit, preferring the “setup” view if there is a choice.
`-clock_trees {string1 string2 ...}`	

	Restricts the report to the specified clock trees. This means that only those sinks that are under the specified clock trees are selected for reporting. *Default*: all defined clock trees
`-delay_corner` *`delay_corner_name`*	
	Reports in the specified delay corner. By default, the software will use the primary-half delay corner that is set using the CCOpt property, `primary_delay_corner` .
`-delay_type {early | late}`	
	Reports in the specified delay type, `early` or `late`. When `late` is specified, only the late half of the delay corners are reported. When `early` is specified, only the early half of the delay corners are reported. *Default:* `late`
`-file` *`file_name`*	Specifies that a copy of the contents of the report be saved to the specified file name.
`-include_reporting_only_skew_groups`	When specified, the report additionally includes the reporting-only skew groups. A reporting-only skew group is a skew group whose `constrains` property is set to `none`. Such a skew group imposes no clock balancing constraint and will not be considered by CTS. By default, the reporting-only skew groups are not included in the report. If a reporting-only skew group is explicitly specified using the `-skew_groups` parameter, then that skew group is always included in the report output. **Note**: The `-include_reporting_only_skew_groups` and `-skew_groups` parameters are mutually exclusive. If both these parameters are specified together, the software errors out.
`-skew_groups {string1 string2 ...}`	
	Restricts the report to the specified skew groups. This means that only those sinks that are under the specified skew groups are selected for reporting. **Note**: The `-include_reporting_only_skew_groups` and `-skew_groups` parameters are mutually exclusive. If both these parameters are specified together, the software errors out. *Default*: all defined skew groups

`-sources {string1 string2 ...}`	Restricts the report to skew group sink pins that have a pin insertion delay value from one of the listed sources.
	This `-sources` list of categories takes wildcard values that are expanded to matching categories.
	For example:
	`report_ccopt_pin_insertion_delays -sources u*`
	This is equivalent to:
	`report_ccopt_pin_insertion_delays -sources {useful_skew user}`

Example

The following command reports the pin insertion delays at all clock tree sinks.

```
report_ccopt_pin_insertion_delays
```

Note: The CCOpt pin insertion delays are +ve valued if they produce an advance, and -ve valued if they produce a delay. This can be viewed in the report output shown below.

```
Positive (advancing) pin insertion delays
=========================================

Found 1 advances (2.041% of 49 clock tree sinks)

-------------------------------------------
From (ns)    To (ns)    Count
-------------------------------------------
0.000         0.010     1
-------------------------------------------

Mean              : 0.006ns
Std.Dev           : 0.000ns

Smallest advance  : 0.006ns at d2211a/CK
Largest advance   : 0.006ns at d2211a/CK

Negative (delaying) pin insertion delays
========================================

Found 23 delays (46.939% of 49 clock tree sinks)

-------------------------------------------
From (ns)    To (ns)   Count
-------------------------------------------
-0.120      -0.110     1
-0.110      -0.100     0
-0.100      -0.090     1
-0.090      -0.080     2
-0.080      -0.070     3
-0.070      -0.060     2
-0.060      -0.050     4
-0.050      -0.040     4
-0.040      -0.030     3
-0.030      -0.020     1
-0.020      -0.010     2
-------------------------------------------

Mean             : -0.056ns
Std.Dev          : 0.025ns

Smallest delay   : -0.014ns at d3211a/CK
Largest delay    : -0.110ns at d1111d/CK
```

Related Information

- The following topic in the *Innovus User Guide*
  - Generating Reports in Clock Tree Synthesis

report_ccopt_preserved_clock_tree_ports

```
  report_ccopt_preserved_clock_tree_ports
[-help]
[-out_file filename]
[-tcl_list]
```

This command reports all currently preserved ports set using the `set_ccopt_preserved_clock_tree_port` command. You can choose to save the information in a file or view it as a tcl list by using the relevant parameter.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_ccopt_preserved_clock_tree_ports` parameter. For a detailed description of the command, use the man command: `man report_ccopt_preserved_clock_tree_ports`
`-out_file filename`	Specifies the name of the preserved port list report file.
`-tcl_list`	Returns a tcl list of currently preserved ports.

Example

- Use the following command to view the report of preserved ports as a tcl list:
  ```
  report_ccopt_preserved_clock_tree_ports -tcl_list
  ```
  The software returns the following information:
  ```
  {m0/ck 100.0 100.0} {m1/ck 900.96 100.0}
  ```

Related Commands

- `reset_ccopt_preserved_clock_tree_port`
- `reset_all_ccopt_preserved_clock_tree_ports`
- `set_ccopt_preserved_clock_tree_port`

report_ccopt_skew_groups

```
report_ccopt_skew_groups
[-help]
[-delay_corners {list_of_delay_corners} | -views {list_of_views}]
[-exclude_pin_insertion_delay]
[-exclude_source_latency]
[-file file_name]
[-from sources]
[-histograms]
[-include_voltage {on | off | auto} | -summary]
[-no_invalidate]
[-paths number_of_paths]
[-skew_groups list_of_skew_groups | -include_reporting_only_skew_groups]
[-skip_timing_update]
[-summarize_stage_depth cell_type_list]
[-tcl_list]
[-through list_of_pins]
[-to list_of_sinks]
[-early | -late ]
```

This command displays information about skew and insertion delay in skew groups. By default, the `report_ccopt_skew_groups` command displays two summary tables at the beginning of the report. The first table lists the insertion delays and skews (along with target values) for each skew group in each timing corner. The second table lists the start and end points of the min and max paths in the skew group. The remainder of the report contains detailed min and max path information for each skew group, in each timing corner.

You can customize the columns included in the skew group report by configuring `set_ccopt_property skew_group_report_columns`.

You can customize the size of the histogram ranges by configuring `set_ccopt_property skew_group_report_histogram_bin_size`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_ccopt_skew_groups` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_ccopt_skew_groups`
`-delay_corners list_of_delay_corners`	
	Specifies that the report will be restricted to the specified delay corners. **Note**: The `-delay_corners` and `-views` parameters are mutually exclusive.
`-early`	Specifies that only the early half of the delay corners will be reported.
`-exclude_pin_insertion_delay`	
	Specifies that pin insertion delay should be excluded from the report.
`-exclude_source_latency`	Specifies that source latency of the clocks should be excluded from the report.

`-file` *file_name*	Specifies that a copy of the contents of the report will be saved to the specified file name.
`-from` *sources*	Specifies the list of sources of the skew group to consider for the report. If specified, only paths beginning at one of the sources specified will be reported.
`-histograms`	Specifies that histograms of key information will be displayed.
`-include_voltage {on | off | auto}`	Prints the voltage column. The following options are available: • `on`: always prints the extra voltage column • `off`: the extra voltage column is not printed • `auto`: only prints the extra voltage column when voltages are different **Note**: This parameter is mutually exclusive with the `-summary` parameter. This is because path delays are not displayed in the summary tables. *Default*: `auto`
`-include_reporting_only_skew_groups`	When specified, the report additionally includes the reporting-only skew groups. A reporting-only skew group is a skew group whose `constrains` property is set to `none`. Such a skew group imposes no clock balancing constraint and is not considered by CTS. By default, the reporting-only skew groups are not included in the report. If a reporting-only skew group is explicitly specified using the `-skew_groups` parameter, then that skew group is always included in the report output. **Note**: The `-include_reporting_only_skew_groups` and `-skew_groups` parameters are mutually exclusive. If both these parameters are specified together, the software errors out.
`-late`	Specifies that only the late half of the delay corners will be reported.
`-no_invalidate`	Specifies that the timing engine should not be invalidated. The timing engine is used to calculate the timing data that is displayed in the report. By default, this is invalidated when you generate the report, so all the timing data is recalculated. This increases the time taken to generate the report. Use the `-no_invalidate` option to reuse the existing timing data for the report.
`-paths` *number_of_paths*	Specifies the number of paths to report on for each timing corner. *Default*: `1`
`-skew_groups` *list_of_skew_groups*	
	Restricts the report to the specified skew groups. By default, all defined skew groups are displayed.
`-skip_timing_update`	Specifies that the timing engine should not be updated. The timing engine is used to calculate the timing data that is displayed in the report. By default, when you generate the report, the timing data is recalculated. This increases the time taken to generate the report. Specify this parameter to skip calculation of timing data.
`-summarize_stage_depth` *cell_type_list*	

	Specifies that the command will consider the number of cells of the specified type(s) in the tree, instead of considering delay while reporting. Possible values are buffer, inverter, logic, clock_gate, and generator. You can also specify `*` to include all cell types.
	The following command shows a report of the skew groups with counts of the buffers and inverters, instead of the delay.
	`report_ccopt_skew_groups -summarize_stage_depth {buffer inverter}`
	Note: The `*` setting is not used in combination with any other cell type. For example, the below is not a valid setting.
	`-summarize_stage_depth {* buffer}`

`-summary`	Specifies that just the summary table should be displayed in the report.
`-tcl_list`	Specifies that the report will be created as Tcl data instead of a text report.
`-through` *list_of_pins*	Provides a list of pin objects representing points that all # paths reported must pass through. The paths must pass through one of the pins specified.
`-to` *list_of_sinks*	Provides a list of pin objects representing all the sinks of skew groups to consider for the report. If specified, only paths terminating at one of the sinks specified will be reported.
`-views {`*list_of_views*`}`	Restricts the report to the delay corners associated with the specified views. **Note**: The `-delay_corners` and `-views` parameters are mutually exclusive.

Examples

Example1: Use the following command to display the skew group information as a summary table in the report:
`report_ccopt_skew_groups -summary`

```
Skew Group Structure:
=====================

-------------------------------------------------------------------------
Skew Group                Sources    Constrained Sinks    Unconstrained Sinks
-------------------------------------------------------------------------
m_clk/PM_HL_FUNC              1              114                    1
m_digit_clk/PM_HL_FUNC        1                9                    0
m_dsram_clk/PM_HL_FUNC        1                1                    0
m_ram_clk/PM_HL_FUNC          1                1                    0
m_roc_clk/PM_HL_FUNC          1              129                    0
m_spi_clk/PM_HL_FUNC          1               12                    0
refclk/PM_HL_FUNC             1              258                    1
-------------------------------------------------------------------------
Skew Group Summary:
===================

-----------------------------------------------------------------------------------------------------------------------------------------------
Timing Corner                        Skew Group              ID Target  Min ID  Max ID  Avg ID  Std.Dev.ID  Skew Target Type  Skew Target  Skew  Skew window occupancy
-----------------------------------------------------------------------------------------------------------------------------------------------
AV_HL_FUNC_MAX_RC1_dc:setup.early    m_clk/PM_HL_FUNC            -   0.039   3.924   2.851   1.390       ignored           -            3.884       -
                                     m_digit_clk/PM_HL_FUNC      -   0.000   0.001   0.001   0.000       ignored           -            0.000       -
                                     m_dsram_clk/PM_HL_FUNC      -   1.930   1.930   1.930   0.000       ignored           -            0.000       -
                                     m_ram_clk/PM_HL_FUNC        -   0.031   0.031   0.031   0.000       ignored           -            0.000       -
                                     m_roc_clk/PM_HL_FUNC        -   0.650   1.377   1.236   0.105       ignored           -            0.727       -
                                     m_spi_clk/PM_HL_FUNC        -   0.301   0.303   0.302   0.001       ignored           -            0.003       -
                                     refclk/PM_HL_FUNC           -   1.446   2.683   2.311   0.417       ignored           -            1.237       -
AV_HL_FUNC_MAX_RC1_dc:setup.late     m_clk/PM_HL_FUNC         none   0.039   3.932   2.857   1.393       auto computed     -            3.893       -
                                     m_digit_clk/PM_HL_FUNC   none   0.001   0.001   0.001   0.000       auto computed     -            0.000       -
-----------------------------------------------------------------------------------------------------------------------------------------------
* - indicates that target was not met.

Skew Group Min/Max path pins:
=============================

-----------------------------------------------------------------------------------
Timing Corner                        Skew Group              Min ID    PathID    Max ID    PathID
-----------------------------------------------------------------------------------
AV_HL_FUNC_MAX_RC1_dc:setup.early    m_clk/PM_HL_FUNC        0.039     -         3.924     -
-    min ARB_INST/dma_grant_reg/CP
-    max RESULTS_CONV_INST/high_mag_reg[13]/CP
                                     m_digit_clk/PM_HL_FUNC  0.000     -         0.001     -
-    min DIGIT_REG_INST/digit_out_reg[0]/CP
-    max DIGIT_REG_INST/digit_out_reg[6]/CP
                                     m_dsram_clk/PM_HL_FUNC  1.930     -         1.930     -
-    min RAM_256x16_TEST_INST/RAM_256x16_INST/CLK
-    max RAM_256x16_TEST_INST/RAM_256x16_INST/CLK
                                     m_ram_clk/PM_HL_FUNC    0.031     -         0.031     -
-    min RAM_128x16_TEST_INST/RAM_128x16_INST/CLK
-    max RAM_128x16_TEST_INST/RAM_128x16_INST/CLK
                                     m_roc_clk/PM_HL_FUNC    0.650     -         1.377     -
-    min RESULTS_CONV_INST/go_reg/CP
-    max RESULTS_CONV_INST/r941_reg[15]/CP
                                     m_spi_clk/PM_HL_FUNC    0.301     -         0.303     -
```

Note:

- The “Timing Corner” column specifies the delay corner, timing check type (setup or hold) and derate type (early or late) reporting conditions for the skew group. All *early corners are reported using early clock derates and all *late corners are reported using late clock derates. A skew group row with no timing column specification is using the timing corner specified above it.
- The "Skew Target Type" column shows either "explicit", “auto computed” or “ignored”. Only the timing corner(s) with "explicit" or "auto computed" are actually considered during clock tree construction, and only these corners will have a numeric skew target in the “Target Skew” column. Generally, only timing corners with late derates for a delay corner are activated for clock tree construction.
- The "Skew window occupancy" column is defined as the percentage of sinks in the skew group that fall within the user-defined skew target. It is computed by finding a min and max latency window in which the largest proportion of sinks fall. The intent of this column is to indicate, when the skew target is not met, some measure of how badly it has not been met by showing the largest proportion of sinks that occupy the desired skew target.

Example2: Use the following command to shows a report of the skew groups with counts of the buffers and inverters, instead of the delay:

```
report_ccopt_skew_groups -summarize_stage_depth {buffer inverter} -skew_groups m_dsram_clk/PM_HL_FUNC

Timing for timing corner AV_HL_FUNC_MIN_RC1_dc:hold.late, max clock_path:
=========================================================================

 PathID : 8
 Path type : skew group m_dsram_clk/PM_HL_FUNC (path 1 of 1)
 Start : TEST_INST/g139/ZN
 End : RAM_TEST_INST/RAM_INST/CLK
 Counted cell types : buffer inverter
 Count : 0

 ---------------------------------------------------------------------------------------------------------------------
```

```
-
 Name                       Lib cell  Event Incr Count Slew Cap    Location Distance Fanout Status Clockpath
trace
                                                    (ns) (pF)      (um)
 ---------------------------------------------------------------------------------------------------------------
-
 TEST_INST/g139/ZN         - IOA21D1BWP rise  -    0    0.131 0.015 (139.930,130.200)   -       1
 RAM_TEST_INST/LS_m_clk/I  - LVLHLD2BWP rise  0    0    0.131 -     (250.130,127.960)  112.440  -
 ---------------------------------------------------------------------------------------------------------------
-
```

Example3: Use the following command to print the voltage column in the report:

```
report_skew_groups -to icache_immu_im0_itlb_tlb_mem_tlb00_upper_mem/rd_c1clk -skew_group skew_group:L2/func -late
-min -include_voltage on
```

```
A part of the output table is shown below.

-------------------------------------------------------------------------------------------------------------
--
Name Lib cell                  Event Incr     Arrival  Slew   Cap    Voltage Location       Distance Fanout
Status
                                     (ns)       (ns)    (ns)   (pF)     (V)      (um)
-- Clockpath trace ------------------------------------------------------------------------------------------
--
e300_sysclk_in
-    -                         rise    -       0.000    0.025  0.003   0.9    (0.000,48.420)   -       1
CTS_cdb_inv_05321/I
- CKN_NOM_D8_H210_L6_P51_UL   rise    0.002   0.002    0.025  -       0.9    (3.034,49.455)   4.069   -
CTS_cdb_inv_05321/ZN
- CKN_NOM_D8_H210_L6_P51_UL   fall    0.006   0.008    0.006  0.001   0.9    (3.170,49.385)   0.206   1
CTS_cdb_inv_05322/I
- CKN_NOM_D3_H210_L6_P51_UL   fall    0.000   0.008    0.006  -       0.9    (3.119,49.035)   0.401   -
```

Related Information

- create_ccopt_skew_group
- get_ccopt_property
- get_ccopt_skew_groups
- get_ccopt_skew_group_delay
- get_ccopt_skew_group_path
- delete_ccopt_skew_groups
- modify_ccopt_skew_group
- set_ccopt_property
- The following topic in the *Innovus User Guide*
  - Skew Group Reporting

report_ccopt_worst_chain

```
report_ccopt_worst_chain
[-help]
[-check_type {setup | hold}]
[-file filename]
[-through pin | -through_file filename]
[-view viewname]
```

This command displays information about the worst chain, which is the chain that contains the path with the worst negative slack or WNS in the design. CCOpt is able to move slack between paths, so the fundamental timing constraint is not from one flop to the next, but chains of flop–to–flop paths. At each flop, timing can be shifted before or after the flop in order to make the physical optimization problem easier.

The command reports a chain as a series of nodes linked together by timing paths. Additionally, it contains information to let you determine how well the chain has been optimized. This report contains the following information:

- The timing chain containing the worst flop-to-flop path, which is identified in the report output by the string, "*WNS*"
- Timing path and constraint information to show why that WNS cannot be improved
- The path group name for each path

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_ccopt_worst_chain` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_ccopt_worst_chain`
`-check_type setup | hold`	
	Specifies whether setup views or hold views are used for the report. *Default*: By default, only the worst chain for setup views is reported.
`-file filename`	Specifies that a copy of the contents of the report will be saved to the specified file name.
`-through pin`	Specifies the path through which the worst chain is being reported. When this parameter is specified, the chain containing the given pin or port is reported. The timing path which contains the `-through` pin is labelled with “*SEL*”. When this parameter is not specified, the chain with the WNS timing path is reported. The WNS timing path in the chain is labelled “*WNS*”.
`-through_file filename`	
	Creates a worst chain report for each pin or port in the specified file. **Note**: The parameters, `-through` and `-through_file` are mutually exclusive.
`-view viewname`	Specifies the view for which the worst chain is being reported.

Example

Following is an example of the report output:

```
report_ccopt_worst_chain
```

```
,-o PM_INST/power_switch_enable_reg_reg
| |     .../CP @(unknown) constraint=(unknown) chosen=(unknown)
| |           location=(209.510,145.600) slew=(launch: 0.000ns, capture: 0.000ns)
| |   slack 1.036ns .../Q -> .../D (distance: 42.280um)
| |               delays=(launch: 0.000ns, datapath: 0.964ns, capture: 0.000ns, adjust: 2.000ns)
| |               launch/capture clock m_clk in analysis view AV_HL_FUNC_MAX_RC1
| |               path group reg2reg
`-o PM_INST/power_switch_enable_reg_reg
  |     .../CP @(unknown) constraint=(unknown) chosen=(unknown)
  |           location=(209.510,145.600) slew=(launch: 0.000ns, capture: 0.000ns)
  |   slack -2.476ns .../Q -> .../NRESTORE (distance: 314.740um)
  |                delays=(launch: 0.000ns, datapath: 4.476ns, capture: 0.000ns, adjust: 2.000ns)
  |                launch clock m_clk in analysis view AV_HL_FUNC_MAX_RC1
  |                capture clock refclk in analysis view AV_HL_FUNC_MAX_RC1
  |                path group reg2reg
  o TDSP_CORE_INST/DECODE_INST/ir_reg[8]
  |     .../CP @(unknown) constraint=(unknown) chosen=(unknown)
  |           location=(393.070,150.640) slew=(launch: 0.000ns, capture: 0.000ns)
  |   *WNS* -14.670ns .../Q -> .../D (distance: 454.300um)
  |                 delays=(launch: 1.000ns, datapath: 16.670ns, capture: 1.000ns, adjust: 2.000ns)
  |                 launch/capture clock refclk in analysis view AV_HL_FUNC_MAX_RC1
  |                 path group reg2reg
,-o TDSP_CORE_INST/EXECUTE_INST/acc_reg[7]
| |     .../CP @(unknown) constraint=(unknown) chosen=(unknown)
| |           location=(328.810,125.440) slew=(launch: 0.000ns, capture: 0.000ns)
| |   slack -11.200ns .../Q -> .../D (distance: 334.320um)
| |                 delays=(launch: 1.000ns, datapath: 16.670ns, capture: 1.000ns, adjust: 2.000ns)
| |                 launch/capture clock refclk in analysis view AV_HL_FUNC_MAX_RC1
| |                 path group reg2reg
`-o TDSP_CORE_INST/EXECUTE_INST/acc_reg[7]
```

Related Information

- ccopt_design
- clock_opt_design
- The following topics in the *Innovus User Guide*
  - Worst Chain Reporting
  - Concept of Chains in CTS

reset_all_ccopt_preserved_clock_tree_ports

```
  reset_all_ccopt_preserved_clock_tree_ports
[-help]
```

This command resets all preservation settings that have been specified using the `set_ccopt_preserved_clock_tree_port` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `reset_all_ccopt_preserved_clock_tree_ports` parameter. For a detailed description of the command, use the man command: `man reset_all_ccopt_preserved_clock_tree_ports`

Example

- Use the following command to reset all the ports:
  ```
  reset_all_ccopt_preserved_clock_tree_ports
  ```

Related Commands

- `reset_ccopt_preserved_clock_tree_port`
- `report_ccopt_preserved_clock_tree_ports`
- `set_ccopt_preserved_clock_tree_port`

reset_ccopt_config

```
reset_ccopt_config
[-help]
[-preserve_sink_insertion_delays]
```

Resets the CCOpt configuration. The command functions in the same manner as the `delete_ccopt_clock_tree_spec` command, but also takes out all values set using the `set_ccopt_property` command. This command basically wipes out all CCOpt-specific data in Innovus. However, you can preserve the pin insertion delays on clock tree sinks by specifying the `-preserve_sink_insertion_delays` parameter.

Parameter

`-help`	Outputs a brief description of the `reset_ccopt_config` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reset_ccopt_config`
`-preserve_sink_insertion_delays`	When specified, preserves pin insertion delays currently configured on clock tree sinks.

Example

- The following command resets the CCOpt configuration:
  `reset_ccopt_config`

Related Commands

- `delete_ccopt_clock_tree_spec`
- `restore_ccopt_config`
- `save_ccopt_config`
- `set_ccopt_property`

reset_ccopt_preserved_clock_tree_port

```
  reset_ccopt_preserved_clock_tree_port
[-help]
{portname}
```

This command resets the preservation settings for the specified preserved port that have been specified using the `set_ccopt_preserved_clock_tree_port` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `reset_ccopt_preserved_clock_tree_port` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reset_ccopt_preserved_clock_tree_port`
portname	Specifies the name of the port.

Example

- Use the following command to reset the port:
  ```
  reset_ccopt_preserved_clock_tree_port m1/ck
  ```

Related Commands

- `report_ccopt_preserved_clock_tree_ports`
- `reset_all_ccopt_preserved_clock_tree_ports`
- `set_ccopt_preserved_clock_tree_port`

reset_ccopt_routing_state

```
reset_ccopt_routing_state
[-help]
[-excluded {netname1 netname2.....}]
[-no_delete_routes]
[-reset_preroutes]
```

Resets the status of the CCOpt clock nets to "routed". The default behavior of the software is to then delete these nets, but you can specify the `-no_delete_routes` parameter to stop these nets from being deleted. The purpose of this command is to reset the routing state post NanoRoute, so that it can be rerun later using the `route_ccopt_clock_tree_nets` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `reset_ccopt_routing_state` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reset_ccopt_routing_state`
`-excluded {netname1 netname2.....}`	Specifies the list of nets whose routing state is not to be reset.
`-no_delete_routes`	Specifies that the nets whose status is set to routed should not be deleted.
`-reset_preroutes`	When specified, resets the status of pre routes. By default, the pre-routes will be skipped.

Example

- The following command resets the routing state of clock nets without deleting them and excluding the specified net:

  ```
  reset_ccopt_routing_state -no_delete_routes -excluded {clock1111}
  ```

Related Command

- `route_ccopt_clock_tree_nets`

restore_ccopt_config

```
restore_ccopt_config
[-help]
input_path
```

Reads the CCOpt configuration from the specified directory path. The CCOpt configurations are saved using the `save_ccopt_config` command.

Note: The CCOpt configuration is not restored if the `restoreDesign -mmmcFile` option is set. In this case, the software issues the following warning:

```
**WARN: (IMPIMEX-4016): The saved CCOpt configuration will not be restored, as the -mmmcFile option has been set.
To restore the CCOpt configuration try restoring the design again without using the -mmmcFile option.
```

Parameters

`-help`	Outputs a brief description that includes type and default information for each `restore_ccopt_config` parameter. For a detailed description of the command and all of its parameters, use the man command: `man restore_ccopt_config`
`input_path`	Specifies the user-defined directory path from where to read the CCOpt configuration.

Examples

- Use the following command to write the configuration scripts to the `./az_ccopt dir`:
  ```
  save_ccopt_config ./az_ccopt
  ```
- Use the following command to clear the configuration:
  ```
  reset_ccopt_config
  ```
- Use the following command to restore the saved configuration by sourcing all the configuration scripts in the `./az_ccopt dir`:
  ```
  restore_ccopt_config ./az_ccopt
  ```

Related Commands

- `save_ccopt_config`
- `reset_ccopt_config`

route_ccopt_clock_tree_nets

```
route_ccopt_clock_tree_nets
[-help]
[-mesh]
```

Performs routing of CCOpt clock tree nets using the CCOpt route guides. Use this command after running `ccopt_design`.

CCOpt uses internal clock routes during the early stages of CTS. It then generates route guides that are passed to NanoRoute for final clock tree routing. This is done by default for full CTS runs. If, for some reason, you have disabled final clock tree routing in CCOpt, then this command can be used to complete the clock tree routing based on the CCOpt route guides. When the `-mesh` parameter is specified, the command only routes the mesh nets.

Note: Before running this command, ensure that the CCOpt data structures are in place.

Parameters

`-help`	Outputs a brief description for `route_ccopt_clock_tree_nets` parameter. For a detailed description of the command and all of its parameters, use the man command: `man route_ccopt_clock_tree_nets`
`-mesh`	Specifies that only mesh nets should be routed.

Related Commands

- `ccopt_design`
- `clock_opt_design`

route_ccopt_flexible_htrees

```
route_ccopt_flexible_htrees
[-help]
[nets net+]
```

Routes CCOpt flexible h-tree nets as a separate step after running the `synthesize_ccopt_flexible_htrees` command. The following use models are supported:

1. Routing during H-tree syntheses:
    a. `create_ccopt_flexible_htree`
    b. `synthesize_ccopt_flexible_htrees`
2. Routing after H-tree synthesis (block level)
    a. `create_ccopt_flexible_htree`
    b. `synthesize_ccopt_flexible_htrees -use_estimated_routes`
    c. `route_ccopt_flexible_htrees`

Note: When using this command, the software expects the input routes of nets in the flexible H-tree to be fully connected, as it is the case for routes generated by `synthesize_ccopt_flexible_htrees -use_estimated_routes`. If this expectation is not met, then unexpected results such as open nets may occur and you should check for connectivity-related warnings in the log file.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `route_ccopt_flexible_htrees` parameter. For a detailed description of the command and all of its parameters, use the man command: `man route_ccopt_flexible_htrees`
`-nets net+`	Specifies a list of flexible h-tree nets to route. The specified nets must all have been created by the `synthesize_ccopt_flexible_htrees` command. By default, the software automatically determines all the flexible h-tree nets that need to be routed.

Related Commands

- `create_ccopt_flexible_htree`
- `synthesize_ccopt_flexible_htrees`

Related Properties

You can also set the following properties on the flexible H-tree design objects by using the `set_ccopt_property` command:

- `adjust_sink_grid_for_aspect_ratio`
- `final_cell`
- `flexible_htree`
- `flexible_htree_placement_legalization_effort`

- htree_sinks
- hv_balance
- image_directory
- inverting
- layer_density
- mode
- omit_symmetry
- partition_boundary_inverting
- pin
- power_weight
- sink_grid
- sink_grid_sink_area
- sink_grid_box
- sink_grid_exclusion_zones
- sink_instance_prefix
- stop_at_sdc_clock_roots
- trunk_cell

For detailed information about all public CCOpt properties, see the CCOpt Properties chapter in the *Innovus User Guide*. You can also use the following command:

```
get_ccopt_property -help propertyName
```

save_ccopt_config

```
save_ccopt_config
[-help]
output_path
```

Writes the CCOpt configuration to the specified directory.

Note: The content of the files generated using this command is proprietary. The files may not be in any particular format, or even human-readable, and are subject to change. The only supported use of the output of `save_ccopt_config` command is as input to the `restore_ccopt_config` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `save_ccopt_config` parameter. For a detailed description of the command and all of its parameters, use the man command: `man save_ccopt_config`
`output_path`	Specifies the user-defined directory to which the CCOpt configuration scripts will be saved.

Example

- The following command writes the CCOpt configuration scripts to the `./az_ccopt dir`:
  ```
  save_ccopt_config ./az_ccopt
  ```

Related Commands

- `restore_ccopt_config`
- `reset_ccopt_config`

set_ccopt_preserved_clock_tree_port

```
  set_ccopt_preserved_clock_tree_port
[-help]
{[portname [-location {x y}]]}
```

This command preserves module ports for the flexible h-tree feature. It can be used on any module port in the clock tree. It will preserve the specified module ports in both, clock tree synthesis and flexible h-tree synthesis. Any sinks connected downstream of the ports will remain downstream, and only those ports will be considered downstream.

Note: Currently, physical locations are only supported for flexible h-tree synthesis, and only on partition boundaries, although they will be accepted on all other module ports. Also, ports belonging to hInsts that have siblings but are not defined as partitions are not supported.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_ccopt_preserved_clock_tree_port` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_ccopt_preserved_clock_tree_port`
`-location`	Provides the location of the preserved port specified using the *portname* parameter.
portname	Specifies the name of the port.

Example

- Use the following command to mark a port on a partition boundary as preserved and set the location:
  ```
  set_ccopt_preserved_clock_tree_port m0/ck -location {100 100}
  ```

Related Commands

- report_ccopt_preserved_clock_tree_ports
- reset_all_ccopt_preserved_clock_tree_ports
- reset_ccopt_preserved_clock_tree_port

set_ccopt_property

```
set_ccopt_property
[-help]
name
value
[-cell library_cell_name]
[-clock_tree clock_tree_name]
[-clock_tree_source_group source_group_name]
[-delay_corner delay_corner_name]
[-flexible_htree flexibleHtreeName]
[-help propertyname]
[-inst instance_name]
[-lib_pin pin_name]
[-net net_name]
[-net_type leaf | trunk | top]
[-pin pin_name]
[-power_domain domain_name]
[-skew_group skew_group_name]
[-early | -late]
[-rise | -fall]
[-min | -max]
```

This command is used to set the values of various CCOpt object properties.

There are some properties that cannot be set by you because these are read-only properties. This means that you can obtain information about these properties by using the `get_ccopt_property` command, but you cannot change that information. However, there are writable properties that can be modified. These properties let you configure different aspects of CCOpt behavior. This command is used to set the writable CCOpt properties.

To set a property, you need to specify the property name, value, object type, and object pattern. The following syntax are acceptable:

1. `set_ccopt_property <name> <value> [-obj <obj_pattern>]`
2. `set_ccopt_property <name> [-obj  <obj_pattern>] <value>`
3. `set_ccopt_property [-obj <obj_pattern>] <name> <value>`

Where,

name is the name of the property to be set

value is the new value to which you want to set the property

For example, all three commands below are equivalent:

1. `set_ccopt_property target_skew 0.1 -skew_group sg1`
2. `set_ccopt_property target_skew -skew_group sg1  0.1`
3. `set_ccopt_property -skew_group sg1 target_skew 0.1`

For details about the key considerations for setting CCOpt properties, getting properties from Tcl, getting a list of properties and their detailed descriptions, key/value pairs, and defaults for key/value pairs, see the "CCOpt Properties and Configuration" section in the Clock Tree Synthesis chapter of the *Innovus User Guide*. For detailed descriptions about all the public CCOpt properties, see the CCOpt Properties chapter of the *Innovus User Guide*.

Parameters

`-help`	Outputs a brief description for the `set_ccopt_property` command. For a detailed description of the command, use the man command: `man set_ccopt_property`
`-cell` *`library_cell_name`*	Specifies the library cell to which the property applies.
`-clock_tree` *`clock_tree_name`*	Specifies the clock tree to which the property applies.
`-clock_tree_source_group` *`source_group_name`*	Specifies the clock tree source group to which the property applies.
`-delay_corner` *`delay_corner_name`*	Specifies the delay corner to which the property applies.
`-early`	Specifies that the property applies to the early signal.
`-fall`	Specifies that the property applies to the fall signal.
`-flexible_htree` *`flexibleHtreeName`*	Specifies the flexible H-tree to which the property applies.
`-help` *`propertyname`*	Provides help for the specified property. Other options are ignored.
`-inst` *`instance_name`*	Specifies the instance to which the property applies. This parameter enables properties to be added to instances.
`-late`	Specifies that the property applies to the late signal.
`-lib_pin` *`pin_name`*	Specifies the library pin to which the property applies.
`-max`	Specifies the maximum value for the property.
`-min`	Specifies the minimum value for the property.
`name`	Specifies the name of the property to be set.
`-net` *`net_name`*	Specifies the net level to which the property applies.
`-net_type` *`leaf	trunk	top`*	Specifies the net type to which the property applies. It could be leaf net, trunk net, or top net.
`-pin` *`pin_name`*	Specifies the pin to which the property applies.
`-power_domain` *`domain_name`*	Specifies the power domain to which the property applies.
`-rise`	Specifies that the property applies to the rise signal.
`-skew_group` *`skew_group_name`*	Specifies the skew group to which the property applies.
`value`	Specifies the new value to which to set the property.

Examples

- The following commands will cause all ‘trunk’ type clock tree nets to be routed using route_type ‘`CTS_2w2s`’ and all ‘leaf’ type clock tree nets to be routed using route_type ‘`CTS_1w2s`’:

  ```
  set_ccopt_property route_type -net_type trunk CTS_2w2s
  set_ccopt_property route_type -net_type leaf CTS_1w2s
  ```

- The following command will cause all ‘trunk’ clock tree nets for clock trees matching pattern “`MAIN`*” to be routed with route_type ‘`NDR_xtra_wide`’:

  ```
  set_ccopt_property route_type -net_type trunk -clock_tree MAIN* NDR_xtra_wide
  ```

- The following command sets the net type to `leaf`:

  ```
  set_ccopt_property -net_type leaf
  ```

- The example below specifies that clock tree synthesis should use inverters when balancing clock tree ct1:

  ```
  set_ccopt_property use_inverters -clock_tree ct1 true
  ```

- Some properties are global, and others are specific to a particular object. Some examples are provided below.
  - The `adjacent_rows_legal` property is global. This means that you can set this property using the following command:

    ```
    set_ccopt_property adjacent_rows_legal true/false
    ```

  - Some properties can be set on particular objects. For example, minimum number of transitive fanout for a net to be routed as a top net can be set for a clock tree:

    ```
    set_ccopt_property -clock_tree clk500m routing_top_min_fanout 10000
    ```

  - Some other properties can be set per object AND per early/late path. For example, the `insertion_delay` property can be set in the following manner:

    ```
    set_ccopt_property insertion_delay 0.8 -pin pinA -early
    set_ccopt_property insertion_delay 0.7 -pin pinA -late
    ```

Related Information

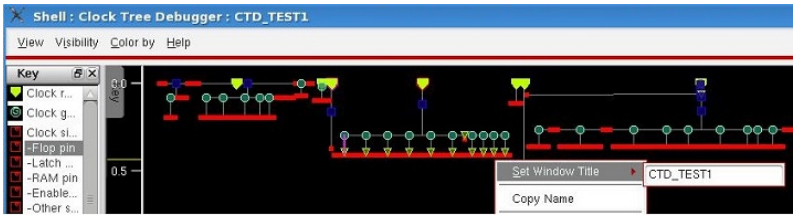
- `get_ccopt_property`
- `unset_ccopt_property`
- For details about the key considerations for setting CCOpt properties, getting properties from Tcl, getting a list of properties and their detailed descriptions, key/value pairs, and defaults for key/value pairs, see The CCOpt Property System topic in the *Innovus User Guide*.
- For detailed descriptions of all the public CCOpt properties, see the CCOpt Properties chapter in the *Innovus User Guide*.

set_ctd_win_title

```
set_ctd_win_title
[-help]
title
[-id WindowIDName]
```

Specifies the title of the CTD window. By default, this command is used to set the title of the current active CTD window either if a unique ID has not been specified for this window or if you want to change the existing window ID.

You can also set the window title from the context menu of the Clock Tree Viewer in the CTD. Click Set Window Title, type the title for the CTD session, and press enter. The new title appears in the top bar of the CTD. This is shown in below image.



Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_ctd_win_title` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_ctd_win_title`
`-id` *`WindowIDName`*	Specifies the ID of the window whose title is to be set. If there are several CTD windows open in the current session, with window IDs, `user_id1, user_id3, user_id3`, and so on, with corresponding window titles, `title1, title2, title3`, and so on, then the following command will only change the `title1` of window with ID, `user_id1`, to `my_new_title`. `set_ctd_win_title my_new_title -id user_id1`
`title`	Specifies the title to be given to the specified window. This parameter is required.

Examples

- The following command sets the window title of the current CTD window, `my_title`, to `my_new_title`:

  ```
  set_ctd_win_title my_new_title
  ```

  The active CTD window title will be changed from:

  ```
  my_test.tcl : Clock Tree Debugger : my_title
  ```

  to:

  ```
  my_test.tcl : Clock Tree Debugger : my_new_title
  ```

- The following command retrieves the window IDs of CTD windows, with a mapping of their window titles:

  ```
  get_ctd_win_id -detail
  ```

  ```
  user_id2 : Clock Tree Debugger : title2
  ```

where, `user_id2` is the window ID, Clock Tree Debugger is the window type, and title2 is the mapping title for this window.

Related Information

- `create_ccopt_clock_tree_spec`
- `ctd_win`
- `ctd_trace`
- `get_ctd_win_id`
- `get_ctd_win_title`
- `gui_zoom_ctd`
- Clock Menu chapter in the *Innovus Menu Reference.*

show_ccopt_cell_name_info

```
show_ccopt_cell_name_info
[-help]
```

Returns the list of cell name codes. All new cell instances created by CCOpt have an identifying code in their name to let you determine why that cell was created. This command lists all available codes and their meanings.

Parameters

`-help`	Outputs a brief description for the `show_ccopt_cell_name_info` parameter. For a detailed description of the command, use the man command: `man show_ccopt_cell_name_info`

Example

- The following command returns a list of all available codes and their meanings:

```
show_ccopt_cell_name_info

Creators:
CTSG Unnamed - this should not be used any more.
cuk Cts: Unknown creator, will not appear in the netlist.
ccl_a Cts: Created during clustering by the agglom clustering algorithm.
cbi Cts: Created by the swapping buffers and inverters for power.
cex Cts: Existing cells in the clock tree which cannot be removed.
coi Cts: Cells created as a result of cancelling out inversions
lbl Cts: Created by the clustering process to meet the slew target
ccl Cts: Created by the clustering process to meet the slew target
ccd Cts: Created by clustering for balancing the tree - these cells are not necessary to meet the slew
target.
cci Cts: Created by the clustering process to fix inversion.
csf Cts: Created by the CTS slew fixing step in cases where clustering did not meet the slew target.
cms Cts: Created during the process of physically moving clock gates to improve their enable timing.
cid Cts: Created by CTS on the outputs of weak driving cells to reduce insertion delay.
cdb Cts: Created by CTS to balance the delays in the clock tree.
cdbw Cts: Created by CTS to balance the delays in the clock tree.
cwb Cts: Created by CTS to balance the wire delays in the clock tree.
cfo Cts: Created by CTS to reduce fanout skew.
csk Cts: Created by the CTS skew fixing step to finely balance the clock tree.
cmf Cts: Created by CTS to buffer long nets.
cbc Cts: Created by the clock tree conditioning step to clone off sub trees that cannot optimized by gated
synthesis, such as those parts above RAMs, black boxes, and lockup latches.
css Cts: Created by the early cloning of simple sink allocations.
cdc Cts: A clock driver created by adding driver cell process for property add_driver_cell.
cpd Cts: A clock driver created below an input port or above an output port specified by property
add_port_driver.
ccg Cts: A clock gate created by one of the gated synthesis algorithms.
cse Cts: A clock driver created above exclude pins to remove them from the clock tree.
cfh Cts: A clock driver created as part of a flexible H-tree.
cat Cts: Created by the add_clock_tree_source_group_roots command
cpc_drv Cts: A clock driver created by post conditioning.
cpc_sk Cts: A clock driver created by post conditioning.
PRO Post-route optimization.
PRO_drv Post-route optimization.
PRO_sk Post-route optimization.
sfc Cloned by slew fixing.
ccc Cts: Created by the clone_clock_cells command.
grb Cts: A clock driver created by global route buffering.
sgb Cts: A clock driver created by source group balancing.
idc Cloned to reduce insertion delay
vgb Cts: A clock driver created by VG buffering.
USK skewClock
cff Cts: A clock driver created by fanout fixing.
```

synthesize_ccopt_flexible_htrees

```
synthesize_ccopt_flexible_htrees
[-help]
[-block_master_clone]
[-dry_run]
[-spec_file fileName]
[-use_estimated_routes | -trial_mode {true | false | placement_only | routing_only}]
```

This command is used to synthesize all flexible H-trees that have been created but not previously synthesized, in one go. The flexible H-trees are created using the `create_ccopt_flexible_htree` command. The `synthesize_ccopt_flexible_htrees` command helps reduce the runtime by synthesizing all flexible H-trees in one go as compared to if the flexible H-trees were synthesized one at a time. This is because when you use this command, nanoRoute is called only once to route all flexible H-trees at the same time.

Note: You must run the `synthesize_ccopt_flexible_htrees` command to synthesize any flexible H-trees previously created using the `create_ccopt_flexible_htree` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `synthesize_ccopt_flexible_htrees` parameter. For a detailed description of the command and all of its parameters, use the man command: `man synthesize_ccopt_flexible_htrees`
`-block_master_clone`	When specified, blocks master/clone partitions for H-tree instances and nets. If this parameter is not specified, and the master/clone partitions do not contain H-tree sink pins, then the software will error out with the following error: `**ERROR: (IMPCCOPT-1506): Flexible H-tree 'htree' uses a sink grid, which is not supported on master/clone designs unless master/clone partitions are blocked during flexible H-tree synthesis. If applicable, master/clone partitions can be blocked using synthesize_ccopt_flexible_htrees -block_master_clone.`
`-dry_run`	Runs the synthesis algorithm but do not insert the H-tree into the database.
`-spec_file` *fileName*	Specifies the name of the clock tree specification script file. When the file name is specified, the clock tree specification file is not amended to reflect the new state of the clock tree. Instead, the amendments, which are in the form of Tcl commands, are written to the file specified using this parameter.
`-trial_mode {true | false | placement_only | routing_only}`	When set, enables trial routing and placement mode for flexible H-tree synthesis. The following settings are available: `true`: Run routing and detailed placement in trial mode. `false`: Run neither routing nor detailed placement in trial mode. This is the default setting. `placement_only`: Only run detailed placement in trial mode. `routing_only` : Only run routing in trial mode. **Note**: The `-trial_mode` and `-use_estimated_routes` parameters are mutually exclusive. *Default*: `false`
`-use_estimated_routes`	Specifies that NanoRoute should not be called. So, the routing estimates are left in the netlist. **Note**: The `-trial_mode` and `-use_estimated_routes` parameters are mutually exclusive.

Example

- The following set of commands are used to synthesize a flexible H-tree under the pins "`rootC/Y`". The sinks will be of the type `INVX9` and will be placed in the specified sink grid. Other buffers added to the H-tree will be of type `INVX13`. The flexible H-tree is then synthesized using the `synthesize_ccopt_flexible_htrees` command.

```
create_ccopt_flexible_htree -name htreeC -sink_grid {6 6} -final_cell INVX9 -trunk_cell INVX13 -pin
rootC/Y
synthesize_ccopt_flexible_htrees
```

Related Information

- The following properties:
  Note: You can also set the following properties on the flexible H-tree design objects by using the `set_ccopt_property` command:
  - `adjust_sink_grid_for_aspect_ratio`
  - `final_cell`
  - `flexible_htree`
  - `flexible_htree_placement_legalization_effort`
  - `htree_sinks`
  - `hv_balance`
  - `image_directory`
  - `inverting`
  - `layer_density`
  - `mode`
  - omit_symmetry
  - `partition_boundary_inverting`
  - `pin`
  - `power_weight`
  - `sink_grid`
  - `sink_grid_sink_area`
  - `sink_grid_box`
  - `sink_grid_exclusion_zones`
  - `sink_instance_prefix`
  - `stop_at_sdc_clock_roots`
  - `trunk_cell`

  For detailed information about all public CCOpt properties, see the CCOpt Properties chapter in the *Innovus User Guide*.
- `create_ccopt_flexible_htree`
- `delete_ccopt_flexible_htrees`
- `get_ccopt_flexible_htrees`
- `get_ccopt_property`
- `set_ccopt_property`
- The following topic in the *Innovus User Guide*
  - Flexible H-Tree and Multi-Tap Clock Flow
- For details of the use model for this feature, see the Flexible H-tree and Multi-Tap Clock Flow in Innovus Application Note

on the Cadence Online Support website.

unset_ccopt_property

```
unset_ccopt_property
[-help]
name
[-cell library_cell_name]
[-clock_tree clock_tree_name]
[-clock_tree_source_group source_group_name]
[-delay_corner delay_corner_name]
[-flexible_htree flexibleHtreeName]
[-help property_name]
[-inst instance_name]
[-lib_pin pin_name]
[-net net_name]
[-net_type leaf | trunk | top]
[-pin pin_name]
[-power_domain domain_name]
[-skew_group skew_group_name]
[-early | -late]
[-rise | -fall]
[-min | -max]
```

This command is used to reset the CCOpt object properties to their default values.

The `-help` parameter of the command when used with the property name is used to retrieve detailed help on the property (`unset_ccopt_property property_name -help`). For example, run the following command to view detailed help on the `use_inverters` property:

```
unset_ccopt_property use_inverters -help
```

The software displays the following information:

```
Specifies whether clock tree synthesis should prefer to use inverters rather
than buffers when balancing the clock tree. If set to true, CTS will use
inverters for the clock tree balancing process. If set to false, CTS will use
the minimum number of levels of inverters required to maintain logical
correctness. If set to auto (the default) CTS will use what it considers to be
the best combination of buffers and inverters to get optimal quality of
results.
```

Default: `auto`

Valid values: `auto true false`

Applies to: `Global, clock_tree`

Note: For detailed description of all public CCOpt properties, see the CCOpt Properties chapter in the *Innovus User Guide.*

Parameters

`-help`	Outputs a brief description for the `unset_ccopt_property` command. For a detailed description of the command, use the man command: `man unset_ccopt_property`
`-cell` *`library_cell_name`*	Specifies the library cell to which the property applies.

`-clock_tree` *clock_tree_name*	
	Specifies the clock tree to which the property applies.
`-clock_tree_source_group` *source_group_name*	
	Specifies the clock tree source group to which the property applies.
`-delay_corner` *delay_corner_name*	
	Specifies the delay corner to which the property applies.
`-early`	Specifies that the property applies to the early signal.
`-fall`	Specifies that the property applies to the fall signal.
`-flexible_htree` *flexibleHtreeName*	
	Specifies the flexible H-tree to which the property applies.
`-help` *property_name*	Provides help for the specified property. All other options are ignored.
`-inst` *instance_name*	Specifies the instance to which the property applies. This parameter enables properties to be added to instances.
`-late`	Specifies that the property applies to the late signal.
`-lib_pin` *pin_name*	Specifies the library pin to which the property applies.
`-max`	Specifies the maximum value for the property.
`-min`	Specifies the minimum value for the property.
name	Specifies the name of the property to unset.
`-net` *net_name*	Specifies the net level to which the property applies.
`-net_type` *leaf | trunk | top*	
	Specifies the net type to which the property applies. It could be leaf net, trunk net, or top net.
`-pin` *pin_name*	Specifies the pin to which the property applies.
`-power_domain` *domain_name*	
	Specifies the power domain to which the property applies.
`-rise`	Specifies that the property applies to the rise signal.
`-skew_group` *skew_group_name*	Specifies the skew group to which the property applies.

Examples

- Use the following command to return the default value of the post_conditioning property:
  ```
  get_ccopt_property post_conditioning
  ```
  The software returns the following information:
  ```
  1
  ```
- Use the following command to set the property to the new value:
  ```
  set_ccopt_property post_conditioning 0
  ```
- Use the following command to check the new value of this property:
  ```
  get_ccopt_property post_conditioning
  ```
  The software returns the following information:
  ```
  0
  ```
- Use the following command to unset the property:
  ```
  unset_ccopt_property post_conditioning
  ```
- Use the following command to check the value of this property now:
  ```
  get_ccopt_property post_conditioning
  ```
  The software returns the following information:
  ```
  1
  ```

Related Information

- `get_ccopt_property`
- `set_ccopt_property`
- For details about the key considerations for setting CCOpt properties, getting properties from Tcl, getting a list of properties and their detailed descriptions, key/value pairs, and defaults for key/value pairs, see the The CCOpt Property System in the *Innovus User Guide*.
- For detailed descriptions of all the public CCOpt properties, see the CCOpt Properties chapter in the *Innovus User Guide*.

update_ccopt_clock_tree

```
update_ccopt_clock_tree
[-help]
clock_tree
[-generated_by sink_pins]
```

Updates the specified clock tree. This command converts a normal (non-generated) clock tree into a generated clock tree. It is intended for internal use only, and should never be required in a CCOpt flow, because the `create_ccopt_clock_tree_spec` command issues `create_ccopt_clock_tree` and `create_ccopt_generated_clock_tree` commands in the correct order to ensure that the right type of clock trees are created. This command is only required when you are translating a legacy FE-CTS clock tree specification for use with CCOpt, which is done using the `-ckSpec` option of the `ccopt_design` command. This is because, in this case, the legacy FE-CTS clock specification may be in a different order than that required by CCOpt.

This command can be seen in the output of `ccopt_design -ckspec -cts -genScriptOnly filename`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `update_ccopt_clock_tree` parameter. For a detailed description of the command and all of its parameters, use the man command: `man update_ccopt_clock_tree`
`clock_tree`	Specifies the name of the clock tree to update. This parameter is required.
`-generated_by` *`sink_pins`*	Specifies the sink pins in the parent clock trees from which this clock tree is generated.

Example

- The following command updates the specified clock tree:
  ```
  update_ccopt_clock_tree m_clk
  ```

Related Commands

- `ccopt_design`
- `clock_opt_design`
- `delete_ccopt_clock_trees`
- `get_ccopt_clock_trees`
- `report_ccopt_clock_trees`

update_clock_latencies

```
update_clock_latencies
[-help]
```

Updates the IO latencies for clock sinks specified using the `consider_during_latency_update` property. You can set the following values for this property:

```
set_ccopt_property consider_during_latency_update true | auto | false
```

- `true`: Consider this pin during latency update.
- `auto`: Consider this pin during latency update only if it is an active sink of a non-reporting only sink group. This is the default value.
- `false`: Do not consider this pin during latency update. This setting is appropriate for sinks that have a large user pin insertion delay applied.

When the above property is set, use the `update_clock_latencies` command to update the IO latencies.

Parameters

`-help`	Outputs a brief description of the `update_clock_latencies` command. For a detailed description of the command, use the man command: `man update_clock_latencies`

Related Information

The following property:

- `consider_during_latency_update`

For details, see the CCOpt Properties chapter in the *Innovus User Guide*.

You can also use the following command:

`get_ccopt_property -help` *`propertyName`*

update_clock_tree_source_latency

```
update_clock_tree_source_latency
[-help]
-clock_tree_source_groups source_group_list
```

Stores clock arrival and transition times at the clock tree source group roots in preparation for multi-tap CTS.

This command may be helpful for configuring arrival times for clock structures in which a mesh drives the source group roots. With other structures such as H-tree, which have a single driver per net, the source group roots are usually defined as generated clock trees and CTS will automatically propagate the clock arrival times from the clock source, through the drive structure, up to the inputs of the source group roots. Flexible H-tree synthesis takes care of all this automatically. However for mesh applications, the CTS clock specification, clock trees defined by the `create_ccopt_clock_tree` and `create_ccopt_generated_clock_tree` commands, cannot include nets with multiple drivers, so CTS cannot automatically propagate arrival times through the clock mesh. For these structures, the source group is typically defined as a collection of non-generated clock trees, and the arrival times are explicitly annotated at their roots before the start of CTS.

This command simplifies the process by querying the timing engine for arrivals and transitions and setting the `source_latency` and `source_input_max_trans` properties at the roots of the specified source group or groups.

Note: The `update_clock_tree_source_latency` command queries the timing graph to get arrival times and transitions. If timing is not already available, the command will first perform a timing graph update, which may take some time. Also, the command assumes that for a particular source group there is a single common clock phase present at all roots. If there is no common clock phase at all roots, or if there are multiple common clocks phases, the command will issue an error message and quit. Also, if a specified source group includes any generated clock trees, the command will issue an error message and quit.

Parameters

`-help`	Outputs a brief description of the `update_clock_tree_source_latency` command. For a detailed description of the command, use the man command: `man update_clock_tree_source_latency`
`-clock_tree_source_groups source_group_list`	
	The list of clock tree source groups for whose clock roots the latencies will be stored. This parameter is required to be specified.

Example

The following command updates the latencies for a single source group, `four_taps`. In this case, timing is already available and no timing graph update is required.

```
update_clock_tree_source_latency -clock_tree_source_groups four_taps

Updating clock tree source latencies for source group 'four_taps'.

Found common clock 'clock' at all source group roots.

Set source timing for 4 roots of source group 'four taps'.

dly_abc_worst:

latency: late rise: [0.565ns, 0.602ns], late fall: [0.816ns, 0.852ns], early rise: 0.000ns, early fall: 0.000ns
transition: late: [0.119ns, 0.120ns], early: [0.119ns, 0.120ns]
```

Related Information

- `create_ccopt_clock_tree`
- `create_ccopt_generated_clock_tree`
- The following properties:
  - `source_latency`
  - `source_input_max_trans`

  For detailed information about the above properties, see the CCOpt Properties chapter in the *Innovus User Guide*. You can also view the property help by running the following command:
  `get_ccopt_property -help property name`

update_clock_tree_spec_annotations

```
update_clock_tree_spec_annotations
[-help]
[-file file_name]
```

Updates all ideal nets, transition and delay annotations to match active timing constraints and removes all prior annotations from the clock tree specification. When this command is specified, all prior annotations are removed from the clock tree specification and the `set_annotated_transition` and `set_annotated_delay` values are mapped to the `annotated_transition` and `annotated_delay_to` property settings. The updated settings can be written to a file using the `-file` option or they can be applied immediately.

The following properties are provided to model the data captured by `set_ideal_network`, `set_annotated_delay`, `set_annotated_transition`, `set_ideal_latency` and `set_ideal_transition`:

- **ideal_net**

  If set the clock tree timing engine will consider this net as ideal.

  Default: `false`
  Required argument: `net`

- **annotated_delay_to**

  Overrides any clock tree timing engine-computed cell arc or net arc delays to the specified pin, in a manner similar to that of SDC set_annotated_delay command.

  For **cell arcs**: The format is dict where the keys are pins and the values are a rise fall delay pair. For example:

  ```
  "<from pin 1> {<from rise delay 1> <from fall delay 1>} <from pin 2> {<from rise delay 1> <from fall delay 2>}"
  ```

  For **net arcs**: The format is dict where the keys are pins and the values are the delay. For example:

  ```
  "<from pin 1> <delay 1> <from pin 2> <delay 2>}"
  ```

  For convenience, set the property using a single value that will apply to all available delays from pins and rise/fall from edges. For example, "0.100".

  Default: `""`

  Applicable arguments: `"-delay_corner <name>", "-pin <name>", "-early", "-late", "-rise" and "-fall". Required: "-pin <name>".`

- **annotated_transition**

  Overrides any clock tree timing engine-computed transition at the specified pin, in a manner similar to that of SDC `set_annotated_transition` command.

  Default: `auto`

  Applicable arguments: `"-delay_corner <name>", "-pin <name>", "-early", "-late", "-rise" and "-fall". Required: "-pin <name>".`

Parameters

`-help`	Outputs a brief description of the `update_clock_tree_spec_annotations` command. For a detailed description of the command, use the man command: `man update_clock_tree_spec_annotations`.
`-file` *`file_name`*	Writes commands to the specified file instead of applying the changes in memory.

Example

Use the set of commands listed below to define a transition annotation at `pin b1/I`, delay annotation at `net_b1`, update the annotations from CTE to clock tree timing engine (CTTE), and verify that the annotation is corrected applied at clock tree timing engine (CTTE).

```
set_interactive_constraint_modes [all_constraint_modes]
set_propagated_clock [all_clocks]

create_ccopt_clock_tree -name clk -source clk -no_skew_group
create_ccopt_skew_group -name clk/default_constraint_mode -sources clk -auto_sinks
```

Annotate transition at b1/I pin using the below command:

```
set_annotated_transition 1.234 [get_pins {b1/I}]
```

Annotate delay at net_b1 net using the below command:

```
set_annotated_delay 2.345 -net -from [get_pins {b1/Z}] -to [get_pins {b2_2/I}]
```

Update the annotations from CTE to CTTE using the below command:

```
update_clock_tree_spec_annotations
```

The log shows the following information:

```
<CMD> update_clock_tree_spec_annotations
Ignoring AAE DB Resetting ...
Updating timing graph...

Leaving CCOpt scope - BuildTimeGraph...
AAE DB initialization (MEM=1059.9 CPU=0:00:00.2 REAL=0:00:00.0)
################################################################################
# Design Stage: PreRoute
# Design Name: top
# Design Mode: 90nm
# Analysis Mode: MMMC Non-OCV
# Parasitics Mode: No SPEF/RCDB
# Signoff Settings: SI Off
################################################################################
Calculate delays in Single mode...
Start delay calculation (fullDC) (1 T). (MEM=1067.43)
siFlow : Timing analysis mode is single, using late cdB files
Total number of fetched objects 8
AAE_INFO: Total number of nets for which stage creation was skipped for all views 0
End delay calculation. (MEM=1141.94 CPU=0:00:00.0 REAL=0:00:00.0)
End delay calculation (fullDC). (MEM=1130.4 CPU=0:00:00.4 REAL=0:00:01.0)
Leaving CCOpt scope - BuildTimeGraph done. (took cpu=0:00:00.7 real=0:00:00.9)
Updating timing graph done.
Updating latch analysis...
Leaving CCOpt scope - Updating latch analysis...
```

```
Leaving CCOpt scope - Updating latch analysis done. (took cpu=0:00:00.0 real=0:00:00.0)
Updating latch analysis done.
Clock tree spec update: Transition and delay annotations were updated to match active timing constraints.
```

Verify that the annotations are correctly updated using the below commands:

```
get_ccopt_clock_tree_slew b1/I
<CMD> get_ccopt_clock_tree_slew b1/I
Clock tree timing engine global stage delay update for default_delay_corner_max:setup.late...
Clock tree timing engine global stage delay update for default_delay_corner_max:setup.late done. (took
cpu=0:00:00.2 real=0:00:00.2)
1.234
```

Related Commands

- get_ccopt_clock_tree_slew
- create_ccopt_clock_tree
- create_ccopt_skew_group
- get_pins
- set_annotated_delay
- set_annotated_transition
- set_interactive_constraint_modes
- set_propagated_clock

6

Conformal Commands

- runCLP

runCLP

```
runCLP
[-cpf fileName]
[-cmd cmdFile]
[-extraLib lib …]
[-extraVlog fileName …]
[-post_synthesis]
[-setupOnly]
[-useEEQCellWithLibertyInfo]
[-user_config ConfigurationFile]
```

Verifies the current design with the power intent file (CPF and 1801) using Conformal Low Power (CLP).

To use this command, specify the path to the CLP (lec) installation before running the Innovus software, and the CLP license is also required. You can use this command after loading a design.

By default, Innovus software generates the following files, then runs CLP

- Netlist (clp_input/feclp.v)
- CPF/1801 file (feclp.cpf or feclp.upf)
- dofile (clp_input/feclp.tcl)

Parameters

`-extraLib` *lib …*	Specifies a list of extra libraries. *Data_type*: string, optional
`-extraVlog` *fileName …*	Specifies a list of extra Verilog files. *Data_type*: string, optional
`-cpf` *fileName*	Specifies the CPF file to use as input to the verification tool. *Default:* `./feclp.cpf`, generated by the Innovus software. *Data_type*: string, optional
`-cmd` *cmdFile*	Specifies the Tcl file to use as input to the verification tool. *Default*: `./clp_input/feclp.tcl`, generated by the Innovus software. *Data_type*: string, optional
`-post_synthesis`	Supports the post_synthesis mode. *Data_type*: bool, optional
`-setupOnly`	Generates input files, but does not run the verification tool. *Data_type*: bool, optional
`-useEEQCellWithLibertyInfo`	Uses an EEQ cell with timing library definition as the master cell of a leaf instance. *Data_type*: bool, optional
`-user_config` *ConfigurationFile*	Specifies the user configuration file. *Data_type*: string, optional

7

Delay Calculation Commands and Global Variables

- Delay Calculation Commands
- Delay Calculation Global Variables

Delay Calculation Commands

- addDeCap
- addDeCapCellCandidates
- cleanupExcludeNet
- clearDeCapCellCandidates
- deleteDeCap
- getDelayCalMode
- reportDeCap
- reportDeCapCellCandidates
- reportDelayCalculation
- saveExcludeNet
- saveSignalStormConstraint
- setDelayCalMode
- translateSNDCSetupFile
- writeSetLoad

addDeCap

```
addDeCap
[-help]
-totCap total_cap_in_fF
[-cells cellName]
[-addFixAttr]
[-area x1 y1 x2 y2 | -exclude {{x1 y1 x2 y2} ...}]
[-effort [low | high]]
[-prefix prefixName]
[-noFixDRC]
[-log logName]
[-maxNrIter number]
[-fromFile fileName]
[-pgNet netName]
```

Adds the specified total decoupling capacitance to the design. The software chooses from the available decoupling capacitance cell candidates, and adds enough cells until their combined total capacitance value equals the user-specified value.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `addDeCap` parameter. For a detailed description of the command and all of its parameters, use the man command: `man addDeCap`.
`-addFixAttr`	Specifies that the decap cell being placed is Fixed. Use this parameter after inserting decoupling capacitance.
`-area x1 y1 x2 y2`	Defines an area within the design in which to add the decoupling capacitance. *Default:* Entire design area x1: Specifies the area's lower left point on the x axis. y1: Specifies the area's lower left point on the y axis. x2: Specifies the area's upper right point on the x axis. y2: Specifies the area's upper right point on the y axis.

`-cells` *cellName …*	Specifies the cells to use for decoupling capacitance insertion. The specified cells must first be defined using the `addDeCapCellCandidates` command. *Default:* Any cell previously defined with the `addDeCapCellCandidates` command. **Note**: The `-cells` parameter is order dependent.
`-effort [low | high]`	Specifies the approach to use when inserting decoupling capacitance. *Default:* `low` Specify one of the following: Uses a homogeneous approach to add the decoupling capacitance. That is, the software treats all areas the same when adding decoupling capacitance cells. Analyzes the peak current of the instances and tries to add more cells to the areas with high peak current density.
`-exclude {`*x1 y1 x2 y2*`} …`	Defines a blockage area within which decoupling capacitance cannot be inserted. You can specify one or more blockage areas. **Note:** The parameters `-exclude` and `-area` are mutually exclusive. When both parameters are specified on the command line, the tool throws an error.
`-noFixDRC`	Disables correction of DRC errors.
`-fromFile` *fileName*	Performs decoupling capacitance insertion using the information in the specified log file (created from a previous `addDeCap` session). This file can contain information for adding decoupling capacitance cells and for deleting existing decoupling capacitance cells.

`-log` *logName*	Outputs a log file with the specified name that contains information on where the decoupling capacitance cells were added. It can also contain information on where cells were deleted due to DRC violations. The log file can be used for tracking the location of decoupling capacitance and for debugging purposes. It can also be used to insert decoupling capacitance cells in a different session. The log file is output in the following format: `ADD` *cellName locationX locationY* [*orientation*] [*instanceName*] `DEL` *instanceName* It contains the cell name and the location where an instance of it was placed. It can also contain the orientation of the instance, and a name for the instance, if that name has not been used already. For example, the following line shows that a new instance of `CELL_8` with the name `Instance100` was added at the location 100 200, with an orientation of R0: `ADD CELL_8 100 200 R0 Instance100`
`-maxNrIter` *number*	Specifies the iteration for inserting decap if there is DRC. *Default*: 10
`-prefix` *prefixName*	Defines the prefix to use for the decoupling capacitance cells. *Default:* `DECAP`
`-pgNet` *netName*	Adds decap cells on the specified power rail. *Default:* Add decap cells irrespective of power nets.
`-totCap` *total_cap_in_fF*	Specifies the total decoupling capacitance to add to the design, in femtofarads (fF). The software uses the specified decoupling cell candidates and adds cells until their combined capacitance value equals *capacitance*.

Command Order

Use this command after defining decoupling capacitance cell candidates with the `addDecapCellCandidates` command.

Examples

- The following command adds 1000 fF of capacitance to the design using `DECAP10` and `DECAP8` cells, in that order:

  ```
  addDeCap -totCap 1000 -cells DECAP10 DECAP8
  ```

- The following command adds 6367.341 fF of capacitance to the specified area within the design, and marks the placement status of all instances as `Fixed`:

  ```
  addDeCap -totCap 6367.341 -area 600 600 900 900 -addFixAttr
  ```

Related Information

- Placing the Design chapter in the *Innovus User Guide*
  - Adding Decoupling Capacitance

Related Information

- `deleteDeCap`
- `reportDeCap`

addDeCapCellCandidates

```
addDeCapCellCandidates
[-help]
{cellName capacitance | -fromFile fileName}
```

Defines the cells that can be used by the `addDeCap` command to insert decoupling capacitance into the design. Use this command to define the candidate cells before using the `addDeCap` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `addDeCapCellCandidates` parameter. For a detailed description of the command and all of its parameters, use the man command: man `addDeCapCellCandidates`.
`cellName capacitance`	Specifies the name and capacitance value in femtofarads (fF) for the cell candidate.
`-fromFile` *`fileName`*	Defines the cells and capacitance values contained in the specified file as decoupling capacitance cell candidates.

Example

The following commands define two decoupling capacitance cell candidates: `DECAP10` has a capacitance value of 10fF, and `DECAP8` has a capacitance value of 5fF.

```
addDeCapCellCandidates DECAP10 10
addDeCapCellCandidates DECAP8 5
```

Related Information

- Placing the Design chapter in the *Innovus System User Guide*
  - Adding Decoupling Capacitance
- `addDeCapCellCandidates`
- `clearDeCapCellCandidates`
- `reportDeCapCellCandidates`

cleanupExcludeNet

```
cleanupExcludeNet [-help]
```

Clears excluded net settings that were specified with the delaycal_exclude_net global variable. Any nets that were previously excluded from delay calculation using the `delaycal_exclude_net` variable are now included in delay calculation. You do not have to rebuild the timing graph to include the previously-excluded nets. Use this command after specifying the nets that are to be excluded from delay calculation.

Parameter

`-help`	Outputs a brief description of the `cleanupExcludeNet` command. For a detailed description of the command, use the man command: `man cleanupExcludeNet`.

Example

The following command clears excluded nets such that previously excluded nets are now included again in delay calculation:

```
cleanupExcludeNet
```

clearDeCapCellCandidates

```
clearDeCapCellCandidates
[-help]
```

Clears all available cells for decoupling capacitance insertion. You can use this command to clear previously defined decoupling cell candidates specified using the addDeCapCellCandidates command.

Parameter

`-help`	Outputs a brief description of the `clearDeCapCellCandidates` command. For a detailed description of the command, use the man command: man `clearDeCapCellCandidates`.

Example:

The following command clears all available decoupling cell candidates:

```
clearDeCapCellCandidates
```

Related Information

- Placing the Design chapter in the *Innovus System User Guide*
  - Adding Decoupling Capacitance
- `addDeCapCellCandidates`
- `reportDeCapCellCandidates`

deleteDeCap

```
deleteDeCap
[-help]
```

Deletes all decoupling capacitance cells. You can use this command to delete decoupling capacitance cells that you added using the addDeCap command.

Parameter

`-help`	Outputs a brief description of the `deleteDeCap` command. For a detailed description of the command, use the man command: `man deleteDeCap`.

Example

The following command deletes all of the decoupling capacitance cells in a design:

```
deleteDeCap
```

Related Information

- Placing the Design chapter in the *Innovus System User Guide*
  - Adding Decoupling Capacitance
- `addDeCap`
- `reportDeCap`

getDelayCalMode

```
getDelayCalMode
{-help]
[-combine_mmmc]
[-enable_high_fanout]
[-enable_low_memory_mode]
[-enable_quiet_receivers_for_hold]
[-equivalent_waveform_model]
[-ewm_type]
[-honorSlewPropConstraint]
[-ignoreNetLoad]
[-pessimistic_base_timing]
[-reportOutBound]
[-SIAware]
[-slewOutBoundLimitHigh]
[-slewOutBoundLimitLow]
[-socv_accuracy_mode]
[-socv_lvf_mode]
[-socv_use_lvf_tables]
[-quiet]
[-nonDefault]
```

Returns the current settings of the `setDelayCalMode` command. The following information is displayed:

- Parameter name
- Current value of the parameter
- Type (Boolean, string, and so on)
- Information on whether the parameter value is user-defined or default.

If you do not specify any parameter, the software displays current settings of all the `setDelayCalMode` command parameters.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setDelayCalMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man getDelayCalMode`.
`parameter_names`	Displays information for the specified parameters. See `setDelayCalMode` command for descriptions of the parameters that you can specify.
`-quiet`	Displays the current settings of the specified parameters in Tcl list format. If you specify `-quiet` without any other parameters, the software displays the current settings of all the `setDelayCalMode` parameters in Tcl list format.

Examples

- The following command displays the current settings of the `setDelayCalMode -combine_mmmc` parameter:
  ```
  getDelayCalMode -combine_mmmc
  ```
  The following information is displayed:
  ```
  -combine_mmmc early_late # enums={none early_late}, default=early_late
  early_late
  ```
- The following command displays the current settings of all the `setDelayCalMode` parameters:
  ```
  getDelayCalMode -quiet
  ```

Related Commands

- `setDelayCalMode`
- `reportDelayCalculation`
- `report_timing`

Related Information

- "Base Delay Analysis" chapter in the *Innovus System User Guide*

reportDeCap

```
reportDeCap
[-help]
-area x1 y1 x2 y2
```

Reports placed decoupling capacitance cells in the specified area.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `reportDeCap` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reportDeCap`.
`-area x1 y1 x2 y2`	Defines the area for which to report placed decoupling capacitance cells.

Example

The following command reports placed decoupling capacitance cells in the specified area:

```
reportDeCap -area 250 250 1000 1000
```

Related Information

- Placing the Design chapter in the *Innovus User Guide*
  - Adding Decoupling Capacitance
- `addDeCap`
- `deleteDeCap`

reportDeCapCellCandidates

```
reportDeCapCellCandidates
[-help]
[-file outputFileName]
```

Reports the available cells for decoupling capacitance insertion. You use this command to report decoupling capacitance candidates previously specified using the addDeCapCellCandidates command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `reportDeCapCellCandidates` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reportDeCapCellCandidates`.
`-file` *filename*	Outputs a list of the available cells for decoupling capacitance insertion to the specified file. This file can then be used by the `addDeCapCellCandidates` command to define cell candidates in a different session, or after cell candidates have been deleted.

Example

Following is a sample report generated by using the `reportDeCapCellCandidates` command:

```
############################################################
# Generated by: Cadence Innovus
# OS:         Linux x86_64(Host ID icdopt70s)
# Generated on: Tue Jan 25 11:47:55 2011
# Design:     dtmf_chip
# Command:     reportDeCapCellCandidates
############################################################
#The available decoupling capaictance cells:
#--------------------------
#Cell Name    Cap (fF)
#--------------------------
DECAP1 10
DECAP8 5
#--------------------------
```

Related Information

- Placing the Design chapter in the *Innovus System User Guide*
    - Adding Decoupling Capacitance
- `addDeCapCellCandidates`
- `clearDeCapCellCandidates`

reportDelayCalculation

```
reportDelayCalculation
[-help]
[-active_arcs]
-from string
[-max]
[-min]
[-outfile filename ]
[-show_all_attackers]
[-si]
[-thresholds]
-to string
[-view string ]
[-voltage]
[-waveform]
```

Reports the delay calculation information for a cell or net timing arc. You can use this command to report delay calculation details for the specified timing arc. The delay calculation report may be useful for debugging timing or for verifying timing data.

You can run the `report_timing` command before running `reportDelayCalculation` to have the correct timing window and aggressor slew.

The `reportDelayCalculation` command output shows 3-sigma multiplier, instead of single sigma. This ensures that the sigma representation for both the delay and transition time is the same.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `reportDelayCalculation` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reportDelayCalculation`.
`-active_arcs`	Disables reporting of all the non-active parallel delay arcs and reports only the valid check arcs.
`-from` *string*	Specifies the name of the From port or pin. If the From port or pin is output of a cell and To port or pin is input of a cell, delay is calculated for the net and the delay type in the report is 'net delay.' If the From port or pin is input of a cell and To port or pin is output of a cell, delay is calculated for the cell and the delay type in the report is 'cell delay.'
`-max`	Indicates that the delay calculation report is for the worst-case operating conditions in min-max mode. If neither `min` nor `max` arguments are specified, then the default mode is set to `max`.
`-min`	Indicates that the delay calculation report is for the best-case operating conditions in min-max mode. If neither `min` nor `max` arguments are specified, then the default mode is set to `max`.
`-outfile` *filename*	Specifies the name of the output file in which to save the delay calculation report.
`-si`	Reports signal integrity (SI) related information.
`-show_all_attackers`	Allows you to report all the small attackers.
`-thresholds`	Displays the rise delay, rise slew, fall delay, fall slew, and slew delay threshold values for the From and To pins.
`-to` *string*	Specifies the name of the To port or pin. If the From port or pin is output of a cell and To port or pin is input of a cell, delay is calculated for the net and the delay type in the report is 'net delay.' If the From port or pin is input of a cell and To port or pin is output of a cell, delay is calculated for the cell and the delay type in the report is 'cell delay.'
`-view` *string*	Specifies the view name for which to print the delay calculation report when using the multi-mode multi-corner (MMMC) mode.

`-voltage`	Displays the voltage of the cell or net timing arc.
`-waveform`	Reports waveforms of the specified pins.

Examples

- The following command shows the cell delay calculation report for the From pin I1/A and To pin I1/Y:

```
reportDelayCalculation -from I1/A -to I1/Y
From pin : I1/A
To Pin : I1/Y
Cell : S2INVV1D1
Library : test
Arc sense : negative unate
Delay type : cell delay
----------------------------------------
RC Summary for net N2
----------------------------------------
Number of capacitance : 7
Net capacitance : 0.026057 pF
Total rise capacitance: 0.027172 pF
Total fall capacitance: 0.027169 pF
Number of resistance : 7
Total resistance : 1094.135127 Ohm
----------------------------------------
Rise Fall
--------------------------------------
Input transition time : 0.118700 ns 0.151800 ns
Effective capacitance : 0.026337 pF 0.025975 pF
Cell delay : 0.325700 ns 0.280800 ns
Output transition time : 0.298600 ns 0.204500 ns
--------------------------------------
```

- The following command shows the net delay calculation report for the from pin ff1/Q and to pin A1/buf1/A with waveforms:

```
tempus> reportDelayCalculation -from ff1/Q -to A1/buf1/A -waveform
```

```
From pin : ff1/Q
Cell : DFFX2
Library : gpdk045bc
To pin : A1/buf1/A
Cell : BUFX2
Library : gpdk045bc
Delay type: net

-------------------------------------------------------------
From pin waveforms
-------------------------------------------------------------
Rise Waveform : { 0.00e+00 0.00000 5.35e-10 0.05400 1.07e-09 0.10800 1.61e-09 0.16200
2.14e-090.21600 2.68e-09 0.27000 3.21e-09 0.32400 3.75e-09 0.37800 4.28e-09 0.43200
4.82e-09 0.486005.35e-09 0.54000 5.89e-09 0.59400 6.42e-09 0.64800 6.96e-09 0.70200
7.50e-09 0.756008.03e-09 0.81000 8.57e-09 0.86400 9.10e-09 0.91800 9.64e-09 0.97200
1.02e-08 1.02600 1.07e-08 1.08000 }

Fall Waveform : { 0.00e+00 1.08000 3.69e-10 1.02600 7.39e-10 0.97200 1.11e-09 0.91800
1.48e-09 0.86400 1.85e-09 0.81000 2.22e-09 0.75600 2.59e-09 0.70200 2.96e-09 0.64800
3.32e-09 0.59400 3.69e-09 0.54000 4.06e-09 0.48600 4.43e-09 0.43200 4.80e-09 0.37800
5.17e-09 0.32400 5.54e-09 0.27000 5.91e-09 0.21600 6.28e-09 0.16200 6.65e-09 0.10800
7.02e-09 0.05400 7.39e-09 -0.00000 }
-------------------------------------------------------------
To pin waveforms
-------------------------------------------------------------
Rise Waveform : { 2.73e-14 0.00000 5.35e-10 0.05318 1.07e-09 0.10793 1.61e-09 0.16125
2.14e-09 0.21587 2.68e-09 0.26930 3.21e-09 0.32383 3.75e-09 0.37734 4.28e-09 0.43179
4.82e-09 0.48538 5.35e-09 0.53975 5.89e-09 0.59341 6.42e-09 0.64772 6.96e-09 0.70144
7.50e-09 0.75570 8.03e-09 0.80946 8.57e-09 0.86368 9.10e-09 0.91748 9.64e-09 0.97166
1.02e-08 1.02549 1.07e-08 1.07965 }

Fall Waveform : { 2.38e-14 1.08000 3.69e-10 1.02719 7.39e-10 0.97209 1.11e-09 0.91911
1.48e-09 0.86416 1.85e-09 0.81104 2.22e-09 0.75623 2.59e-09 0.70298 2.96e-09 0.64828
3.32e-09 0.59493 3.69e-09 0.54033 4.06e-09 0.48688 4.43e-09 0.43237 4.80e-09 0.37885
```

```
5.17e-09 0.32440 5.54e-09 0.27081 5.91e-09 0.21643 6.28e-09 0.16279 6.65e-09 0.10846
7.02e-09 0.05476 7.39e-09 0.00048 }
------------------------------------------------------------


------------------------------------------------------------
RC Summary for net n2
------------------------------------------------------------
Number of capacitance : 19
Net capacitance : 0.714975 pF
Total rise capacitance: 0.716177 pF
Total fall capacitance: 0.716179 pF
Number of resistance : 24
Total resistance : 101.624695 Ohm
------------------------------------------------------------
Rise Fall
------------------------------------------------------------
Net delay : 0.002500 ns 0.002300 ns
From pin transition time: 8.565800 ns 5.910500 ns
To pin transition time : 8.568400 ns 5.913000 ns
------------------------------------------------------------
```

- The following command shows the attacker status in the SI delay calculation report output (when `setSIMode -enable_delay_report` is set to `true`):

```
reportDelayCalculation -si -from seg3/u5/Y -to seg3/u6/A

From pin : seg3/u5/Y
Cell : INV
Library : cell_w
To pin : seg3/u6/A
Cell : INV
Library : cell_w
Base or SI: SI delay
Delay type: net

------------------------------------------------------------
RC Summary for net seg3/n5
------------------------------------------------------------
Number of capacitance : 17
Net capacitance : 0.293534 pF
Total rise capacitance: 0.320849 pF
```

```
Total fall capacitance: 0.320780 pF
Number of resistance : 17
Total resistance : 567.671387 Ohm
-------------------------------------------------------------
SI information for rise
-------------------------------------------------------------------------------
Victim timing window : 1.0743 ns 2.7024 ns CLK_W_3
-------------------------------------------------------------------------------
Attacker Status Direction Slew Coupling cap Fanout Voltage Bump Timing window Clock
-------------------------------------------------------------------------------
seg2/n5 INF Fall 0.303667 ns 0.138759 pF 1 1.080 v 0.198396 v 1.1917 ns 2.8626 ns
CLK_W_2
seg1/n5 INF Fall 0.314333 ns 0.138787 pF 1 1.080 v 0.182412 v 1.3881 ns 3.0723 ns
CLK_W_1
-------------------------------------------------------------------------------
SI information for fall
-------------------------------------------------------------------------------
Victim timing window : 1.2016 ns 5.0700 ns CLK_W_3
-------------------------------------------------------------------------------
Attacker Status Direction Slew Coupling cap Fanout Voltage Bump Timing window Clock
-------------------------------------------------------------------------------
seg2/n5 INF Rise 0.550667 ns 0.138759 pF 1 1.080 v 0.138780 v 1.0887 ns 2.0217 ns
CLK_W_2
seg1/n5 INF Rise 0.800167 ns 0.138787 pF 1 1.080 v 0.107892 v 1.2751 ns 2.3031 ns
CLK_W_1
-------------------------------------------------------------------------------
Model types
-------------------------------------------------------------------------------
Net Term Cell Model
-------------------------------------------------------------------------------
seg3/n5 seg3/u5/Y INV NLDM
seg3/n5 seg3/u6/A INV NLDM
seg2/n5 Y INV NLDM
seg1/n5 Y INV NLDM
-------------------------------------------------------------------------------
Delays Rise Fall
-------------------------------------------------------------
Incr Net Delay : 0.565900 ns 0.093700 ns
Net delay (mean) : 0.696000 ns 0.188900 ns
(3-sigma) : 0.000000 ns 0.000000 ns
Timing Derate (mean) : 1.000000 1.000000
(sigma) : 1.000000 1.000000
Derated Net delay (mean) : 0.696000 ns 0.188900 ns
(3-sigma) : 0.000000 ns 0.000000 ns
```

```
------------------------------------------------------------

------------------------------------------------------------
Transitions Rise Fall
------------------------------------------------------------
Input transition (mean) : 0.482600 ns 0.234800 ns
(sigma) : 0.000000 ns 0.000000 ns
Output transition (mean) : 0.622200 ns 0.274100 ns
(sigma) : 0.000000 ns 0.000000 ns
------------------------------------------------------------
Abbreviative attacker status and filtered reason:
------------------------------------------------------------
SY : Synchronous
INF: Attacker with infinite timing window
PE : Filtered due to physical exclusion
UF : Filtered by user
CN : Filtered because it is a constant net
SB : Filtered due to small bump
TA : Filtered because of timing window does not overlap
LC : Filtered due to logical correlation
ACT: Attacker is Active.
------------------------------------------------------------
```

Related Commands

- report_timing
- setDelayCalMode
- setSIMode

saveExcludeNet

```
saveExcludeNet [-help] -file fileName
```

Reports the list of nets that are excluded from delay calculation. This includes the nets excluded by the delaycal_exclude_net variable and the nets that exceed the default value of 1,000 pins. You can use this command to save a file listing excluded nets previously defined using the `delaycal_exclude_net` variable.

Use this command after performing delay calculation.

Parameters

`-help`	Outputs a brief description that includes type and default information for the `saveExcludeNet` parameter. For a detailed description of the command and its parameter, use the man command: `man saveExcludeNet`.
`-file` *fileName*	Specifies the name of the excluded net file.

Example

- The following command saves the list of excluded nets to a file named `myfile.exc`:

  ```
  saveExcludeNet -file myfile.exc
  ```

saveSignalStormConstraint

```
saveSignalStormConstraint
[-help]
-outfile fileName
```

Translates constraint information from the Innovus software, such as boundary slews and loads, into signalStorm constraint syntax, and outputs it into a specified file. This file can then be included in, or sourced by, a signalStorm script to run standalone signalStorm. Use this command after loading the netlist and timing constraints.

Parameters

`-help`	Outputs a brief description that includes type and default information for the `saveSignalStormConstraint` parameter. For a detailed description of the command and its parameter, use the man command: `man saveSignalStormConstraint`.
`-outfile` *`fileName`*	Specifies the file to which to output the constraint information.

Examples

The following command saves a file containing Signal Storm constraint information:

```
saveSignalStormConstraint -outfile signalStorm.sndc
```

setDelayCalMode

```
setDelayCalMode
[-help]
[-accuracy_level level]
[-advanced_pincap_mode {0 | 1 | 2}]
[-advanced_node_pin_cap_settings {true | false}]
[-combine_mmmc {none | early_late | early_late_corner}]
[-early_irdrop_data_type {best_average | worst | best | average | worst_average}]
[-reset]
[-enable_high_fanout {true | false}]
[-enable_input_slew_sensitivity_on_constraint {true | false}]
[-enable_quiet_receivers_for_hold {true | false}]
[-equivalent_waveform_model {none | no_propagation | propagation}]
[-equivalent_waveform_model_for_timing_check {true|false}]
[-ewm_type {moments | simulation}]
[-honorSlewPropConstraint {true | false}]
[-irdrop_data_type {best_average | worst | best | average | worst_average}]
[-irDrop_window_based {none | late | early | both}]
[-late_irdrop_data_type {best_average | worst | best | average | worst_average}]
[-library_interpolation_mode {linear | non_linear}]
[-ignoreNetLoad {true | false}]
[-reportOutBound {true | false}]
[-SIAware {true | false}]
[-signoff_alignment_settings {true | false}]
[-skip_slew_merge_from_disabled_path {false | true}]
[-slewOutBoundLimitHigh value]
[-slewOutBoundLimitLow value]
[-socv_accuracy_mode {low | medium | high | ultra}]
[-socv_lvf_mode {moments | early_late}]
[-socv_machine_learning_level {0|1}]
[-socv_use_lvf_tables {all | delay | slew | constraint}]
```

Sets global parameters for delay calculation. Use this command to specify any required delay calculation mode settings before running any commands that perform delay calculation, such as timing analysis and optimization. The parameters specified with the `setDelayCalMode` command are then used automatically whenever delay calculation takes place.

Use the `getDelayCalMode` command to return the current settings for the `setDelayCalMode` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setDelayCalMode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setDelayCalMode`.
`-accuracy_level` *level*	Sets the accuracy level for delay calculations. You can use the `-accuracy_level` parameter to enable new enhancements for different process nodes (without using the `setDesignMode -process` parameter). **Note**: However, you can set the `setDesignMode -process` parameter to enable the delay accuracy level equivalent to the values given below: setDesignMode -process 16 == setDelayCalMode -accuracy_level 0 setDesignMode -process 10 == setDelayCalMode -accuracy_level 1 setDesignMode -process 7 == setDelayCalMode -accuracy_level 2 setDesignMode -process 5 == setDelayCalMode -accuracy_level 3 setDesignMode -process 3 == setDelayCalMode -accuracy_level 4 *Default*: 0, *Min*: 0, *Max*: 4

`-advanced_pincap_mode {0 | 1 | 2}`	Simulates the current stage and receiver stages together, such that the back miller effect is modeled dynamically. You can specify the following options: • `0`: Receiver current sources are computed using ECSM/CCS capacitance tables with the estimated input slew. • `1`: Receivers are no longer modeled as simple non-linear current source models. This results in increasing the delay calculation accuracy significantly as the modeling back miller effect improves. • 2: Uses advanced simulation-based approach to model the receiver back miller effect. *Default*: `0`
`-advanced_node_pin_cap_settings {true | false}`	Enables advanced node pin cap settings. *Default*: `false`

`-combine_mmmc {none | early_late | early_late_corner}`	Specifies whether delay calculation runs are to be combined for delay calculation simulations. This parameter provides user control for improving the run time of AAE for all analysis types by combining runs for early and late simulations into a single simulation. The views with common delay corners may also be combined into a single delay calculation simulation for added run time improvement. You can specify one of the following options: • `none`: Combines any delay calculation runs. Individual delay calculation simulations are performed for early and late simulations. This may result in significant CPU performance degradation due to the increased number of simulation runs performed. • `early_late`: Combines early and late simulations of a single corner into one delay calculation simulation. This will improve CPU performance by avoiding unnecessary delay calculation simulations. • `early_late_corner`: Combines early and late simulations of a single corner, and also combines views with common delay corners into a single delay calculation simulation. For example, the software will combine views with different `set_case_analysis` statements in one single simulation. This can result in a significant improvement in the CPU performance by avoiding unnecessary delay calculation simulations. *Default*: `early_late_corner`
`-early_irdrop_data_type {best_average | worst | best | average | worst_average}`	Allows you to set the early IR drop EIV values for different EIV voltages. *Default*: `worst`

`-enable_high_fanout {true | false}`	When this parameter is set to true, the default net delay will be annotated on high fanout nets. A net will be treated as a high fanout net when the fanout is more than the limit specified using the `delaycal_use_default_delay_limit` global variable. The default value of this variable is `1000` fanouts. The default net delay is also specified using this variable. The default net delay is 1ns. The cell delay will use the lumped capacitance that is specified using the `delaycal_default_net_load` global variable for table lookup. The default net load is `0.5pF`. The driver slew will use the default value of `0ps`, which can be changed by using the `delaycal_input_transition_delay` global variable and there will be no slew degradation along the interconnect. The default value of this parameter is `false`.
`-enable_input_slew_sensitivity_on_constraint {true | false}`	Enables the input slew sensitivity on constraints. When set to `true`, the input slew sensitivities at constrained and related pins will be considered by timing check sigma calculation. When set to `false`, the timing check arc is derived based on the nominal or mean slews only. *Default*: `false`
`-enable_quiet_receivers_for_hold {true | false}`	When set to `true`, the software forces some receivers as static (“quiet”). To compute the worst-case delays for early analysis, the software assumes that the receivers are not switching. This yields more conservative early delays. When set to `false`, the software assumes receivers may switch. *Default*: `false`

`-equivalent_waveform_model {none | no_propagation | propagation}`	Controls the equivalent waveform model to be used. You can specify one of the following options:
	• `none`: When this option is specified, the waveform shape beyond slew measurement points does not impact delays. This mode is equivalent to setting the following options in the previous releases:
	`setDelayCalMode -equivalent_waveform_model_type none`
	`setDelayCalMode -equivalent_waveform_model_propagation false`
	• `no_propagation`: In this mode, the impact of receiver waveform shape on the next stage delay is estimated and adjusted the net delay of the current stage. This mode is equivalent to setting the following options in the previous releases:
	`setDelayCalMode -equivalent_waveform_model_type ecsm`
	`setDelayCalMode -equivalent_waveform_model_propagation false`
	• `propagation`: When specified, the impact of input waveform shape is considered during delay calculation and the waveforms are propagated across each stage of the entire path.This mode is equivalent to setting the following options in the previous releases:
	`setDelayCalMode -equivalent_waveform_model_type ecsm`
	`setDelayCalMode -equivalent_waveform_model_propagation true`

	Default: `none`
`-equivalent_waveform_model_for_timing_check {true | false}`	Models the waveform distortion effect for timing check value look ups at both the data and clock pins. The model uses simulations to convert the distorted waveforms into equivalent waveforms (pre-driver waveforms in the libraries) by matching the pin CCB output waveforms. This feature requires the CCSN library. *Default*: `false`
`-ewm_type {moments | simulation}`	Specifies the equivalent waveform model (EWM) to be used. The options are described below: • `moments`: Enables moment-based equivalent waveform model. • `simulation`: Enables simulation-based equivalent waveform model. The simulation-based EWM is required for predicting delay in noisy waveforms. The `simulation` option requires noise models (ECSMN/CCSN). If noise models are not available, then the software defaults to moments-based EWM. It is recommended to use simulation-based EWM as it is more accurate than moment-based EWM. *Default*: `moments`

`-honorSlewPropConstraint {true | false}`	Determines whether to propagate the slew from the disabled timing arcs to the output pin. When this parameter is set to false, the slew from the disabled timing arc affects the delay of the next stage. Setting this parameter to false results in more accurate slew computation for the disabled timing arcs but can take longer run time. When set to true, the timing constraints defined using commands such as `set_disable_timing` and `set_case_analysis` impact the slew propagation, and the slew from the disabled timing arcs is not propagated. *Default* : `true`
`-irdrop_data_type {best_average | worst | best | average | worst_average}`	Allows you to set the early/late IR drop EIV values for different EIV voltages. *Default*: `worst`
`-irDrop_window_based {none | late | early | both}`	Searches window-based EIV values from the EIV database. *Default*: `both`
`-late_irdrop_data_type {best_average | worst | best | average | worst_average}`	Allows you to set the late IR drop EIV values for different EIV voltages. *Default*: `worst`
`-library_interpolation_mode {linear | non_linear}`	Controls library scaling mode - `linear` or `non_linear` - for the interpolation flow. *Default*: `linear`

`-ignoreNetLoad {true | false}`	When set to true, this parameter removes the default system requirement for having RC parasitics available for delay calculation. This parameter is used to get initial design timing when the design is not in a proper state to generate extracted results and an external SPEF source of parasitics is not available. The results obtained by enabling this parameter are equivalent to those obtained using a zero-cap, zero-resistance wire-load model for estimating delays. *Default* : `false`
`-reportOutBound {true | false}`	Generates a report that contains a list of index values (input transition) in the delay tables that are beyond the index range. The index values detected by this parameter can cause inaccurate delay values to be calculated. The report generated by this parameter can be used for debugging. When this parameter is set to `true`, the extrapolation information will be dumped into the following two files: • <design-name>.dc.outbound.slew.early.<view-name>.txt • <design-name>.dc.outbound.slew.late.<view-name>.txt The software writes a warning message in the log files noting that there are input transition values that are out of the index range of the timing table. The warning message also provides the file name for viewing the detail report. When set to `true`, the `-reportOutBound` parameter will generate reports that can be controlled using the `setOutboundReport` command. *Default*: `false`

`-reset`	Resets parameters to their default values. The `-reset` parameter *must* be the first parameter specified. If you specify `-reset` by itself, the software resets all `setDelayCalMode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. **Note**: Any values that do not already match the default value and are reset will be reported.
`-SIAware {true | false}`	Enables SIAware delay calculation that also includes cross-talk induced delays. *Default*: `false` **Note:** SIAware delay calculation requires the design to be in Multi-Mode, Multi-Corner (MMMC) Mode and the analysis type to be set to on-chip variations. **Note**: The `timeDesign` and `optDesign -postRoute` commands will set the `-siAware` option to `true` to perform SI analysis. To disable SI analysis at the post-route stage you must explicitly set `-siAware` to `false`, so that base analysis is performed.
`-skip_slew_merge_from_disabled_path {false | true}`	Avoids merging slews from unconstrained, segmented paths and black-box output pin. When set to `true`, the software does not allow merging transitions from disabled paths. *Default*: `false`
`-signoff_alignment_settings {true | false}`	Enables signoff alignment settings. *Default*: `false`

`-slewOutBoundLimitHigh` *value*	Limits the maximum slew used in the analysis to the specified value. *Min*: 1 *Max*: 3.40282346638529e *Data type*: float *Default*: 3.40282346638529e+38
`-slewOutBoundLimitLow` *value*	Limits the minimum slew used in the analysis to the specified value. *Min*: 0.001 *Max*: 1 *Data type*: float *Default*: 0.5
`-socv_accuracy_mode {low | medium | high | ultra}`	Allows the ability to control levels of accuracy. This parameter specifies level-based accuracy effort for delays and slews. You can use one of the following options: • `low`: Ignores correlation between slews and delays. This is the default setting. • `medium`: Models correlation between immediate slew and delay. • `high`: Considers skewness of non-gaussian distribution into account for computing quantiles. The skewness is local to one stage only. • `ultra`: Considers skewness of non-gaussian distribution, and also propagates skewness to the next stage in PBA analysis.
`-socv_lvf_mode {moments | early_late}`	Controls interpretation of LVF data. *Default*: `moments`

`-socv_machine_learning_level {0 | 1}`	Enables the SOCV machine learning mode. For accurate SOCV analysis, the software creates several statistical models, which are not available with standard LVF library, such as correlation between cells delay and output slew, conversion of moments from late/early sigma tables, and effects of waveform distortion on sigma and mean shift. When the `-socv_machine_learning_level` parameter is set to 1, the software enables several machine learning algorithms to generate such models internally. *Default*: 0
`-socv_use_lvf_tables {all | delay | slew | constraint}`	Controls which type of variations are considered during analysis. *Default*: `all`

Examples

- The following command enables waveform propagation along the path for equivalent waveform model:

  ```
  setDelayCalMode -equivalent_waveform_model propagation
  ```

- If you have set the value of the `-SIAware` parameter to `true`, use the following command to reset the value of the parameter to the default value, which is `false`, and whch disables SI analysis during delay calculation:

  ```
  setDelayCalMode -reset -SIAware
  ```

  Now when you retrieve the value of the parameter using the `getDelayCalMode` command, the software returns the following:

  ```
  getDelayCalMode -SIAware
  -SIAware false # enums={true false}, default=false
  false
  ```

Related Information

- "Base Delay Analysis" chapter in the *Innovus System User Guide.*
- `getDelayCalMode`

translateSNDCSetupFile

```
translateSNDCSetupFile
[-help]
sndcSetupFile
outputFile
```

Uses a signalStorm delay calculator setup file as input and converts the transition time and capacitative loading portions to timing constraints commands, such as `set_annotated_transition` and `set_load`. The timing constraints commands are included in the specified output file. This command is useful only if you are using standalone signalStorm delay calculation.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `translateSNDCSetupFile` parameter. For a detailed description of the command and all of its parameters, use the man command: `man translateSNDCSetupFile`.
`outputFile`	Specifies the name of the output file, which contain the timing constraints commands.
`sndcSetupFile`	Specifies the name of the signalStorm delay calculator setup file that will be converted.

Example

The following command uses a signalStorm delay calculator setup file named `sndc.setup` and generates the `constraints.rpt` file, which contains the timing constraints commands:

```
translateSNDCSetupFile sndc.setup constraints.rpt
```

writeSetLoad

```
writeSetLoad
[-help]
[-direct]
[-excludeZeroCap]
[-includePinCap]
[-wire_load]
[-view viewName]
-file outFile
```

Writes the capacitive loading on each net to an output file. By default, the capacitance value does not include pin capacitance. You can set the load using the `set_load` command or else the software uses the load defined in the SPEF file. The latter is preferred for transferring parasitic information since the `set_load` command does not give full information regarding RCs.

You can use this command for dumping capacitance values for loading into third party tools. You can also use it for debugging, to examine cap load values for nets. But you can achieve this by other means such as using `report_net` and `get_property` commands, which does not require you to dump a file.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `writeSetLoad` parameter. For a detailed description of the command and all of its parameters, use the man command: `man writeSetLoad`.
`-direct`	Refers the ports directly for which the capacitive loading will be written. This option removes the dependency of using the get_nets command.
`-excludeZeroCap`	Excludes the nets not annotated in the design while writing out the capacitive loading on each net to the output file.
`-file outFile`	Specifies the name of the output file.
`-includePinCap`	Includes the pin capacitance while writing out the capacitive loading on each net to the output file.
`-view viewName`	Specifies the view name to use for writing out the capacitive loading.
`-wire_load`	Adds option '-wire_load' to "set_load" (i.e. set_load -wire_load) only on the output (out) and bi-directional (inout) ports in the output file.

Example

The following command writes the capacitive load in the report file `load.rpt`:

```
writeSetLoad -file load.rpt
```

The following report is generated:

```
set_load 1e-05 [get_nets {sig_out4000}]

set_load 0.00011 [get_nets {sig_in1000}]

set_load 0.00011 [get_nets {sig_in1001}]

set_load 0.00015 [get_nets {node_out6000}]
```

Delay Calculation Global Variables

- delaycal_default_net_delay
- delaycal_default_net_load
- delaycal_input_transition_delay
- delaycal_rd_rnet_fraction_threshold
- delaycal_support_min_max_pin_cap
- delaycal_support_rise_fall_pin_cap
- delaycal_support_wire_load_model
- delaycal_use_default_delay_limit

delaycal_default_net_delay

`delaycal_default_net_delay` *`delay_value`*

Type: String

Default: `1000ps`

Sets the default net delay for delay calculation. The software uses this default value for nets that exceed 1,000 terminals. Delay calculation is performed for nets with fewer than 1,000 terminals.

Example

- The following global variable sets the net delay to `3 ns` for nets that exceed 1,000 terminals:

  ```
  set delaycal_default_net_delay 3ns
  ```

  The software displays the following information:

  ```
  Set Default Net Delay as 3000 ps.
  ```

Related Information

- "Base Delay Analysis" chapter in the *Innovus System User Guide*

Related Global Variables

- delaycal_default_net_load
- delaycal_input_transition_delay
- delaycal_use_default_delay_limit

delaycal_default_net_load

`delaycal_default_net_load` *`load_value`*

Type: String

Default: `0.5pF`

Sets the default net load for delay calculation. You can use this global variable to specify the default net load value that is used for nets that exceed 1,000 terminals.

Example

- The following global variable sets the net load to `2 pF` for nets that exceed 1,000 terminals:

  ```
  set delaycal_default_net_load 2pF
  ```

  The software displays the following information:

  ```
  Set Default Net Load as 2 pF.
  ```

Related Information

- "Base Delay Analysis" chapter in the *Innovus System User Guide*

Related Global Variables

- `delaycal_default_net_delay`
- `delaycal_input_transition_delay`
- `delaycal_use_default_delay_limit`

delaycal_input_transition_delay

`delaycal_input_transition_delay`

Type: String

Default: `4ps`

Sets the default input transition delays and the ideal clock transition time for delay calculation.

The default transition will be used at all places where transition is not defined (either by `set_input_transition`, `set_annotated_transition`, or any driving cell). For all the ports/pins that do not have any driving cell in fanin (or `set_input_transition`/`set_annotated_transition` settings) will also use this default slew value.

Note: It is mandatory to specify time units for this command. Valid time units are ps, ns, etc.

In addition, the default input transition value is applied to those pins to which the slew was not propagated. For example:

- Tiehi or tielo pins
- Macro output pins, which do not have a related input pin

Note: A macro output pin, which has a related input pin will most likely have slew propagation from the related input pin to the output pin.

- Macro output pins for which all the timing arcs are disabled
- Input ports without a user-specified transition value

Example

- The following global variable sets the input transition delay to 300ps for nets that exceed 1,000 terminals:

  ```
  set delaycal_input_transition_delay 300ps
  ```

  The software displays the following information:

  ```
  Set Input Pin Transition Delay as 300 ps.
  ```

Related Information

- "Base Delay Analysis" chapter in the *Innovus System User Guide*

Related Global Variables

- delaycal_default_net_delay
- delaycal_default_net_load
- delaycal_use_default_delay_limit

delaycal_rd_rnet_fraction_threshold

`delaycal_rd_rnet_fraction_threshold`

Type: Double

Default: `0.000000`

Specifies the ratio of drive resistance to net resistance at which special handling of early path delays is invoked. When active, slews across early delay path nets are not degraded.

Example

- The following global variable specifies the Rd / Rnet threshold to be `1.0` for which early net slews are not degraded:

  ```
  set delaycal_rd_rnet_fraction_threshold 1.0
  ```

delaycal_support_min_max_pin_cap

```
delaycal_support_min_max_pin_cap {0 | 1}
```

Type: Integer

Default: `1`

Controls whether SignalStorm delay calculation respects Liberty pin capacitance ranges. A "`0`" setting is only required for correlation with the feDC delay engine.

Example

- The following global variable disables SignalStorm delay calculation support for Liberty pin capacitance ranges:
  ```
  set delaycal_support_min_max_pin_cap 0
  ```

Related Global Variable

- `delaycal_support_rise_fall_pin_cap`

delaycal_support_rise_fall_pin_cap

```
delaycal_support_rise_fall_pin_cap {0 | 1}
```

Type: Integer

Default: `1`

Controls whether feDC or SignalStorm delay engines respect Liberty `rise_capacitance` and `fall_capacitance` attributes.

Note: This global variable is for backward compatibility.

Example

- The following global variable disables feDC and SignalStorm delay calculation support for Liberty `rise_capacitance` and `fall_capacitance` attributes:

  ```
  set delaycal_support_rise_fall_pin_cap 0
  ```

Related Global Variable

- `delaycal_support_min_max_pin_cap`

delaycal_support_wire_load_model

```
delaycal_support_wire_load_model {0 | 1}
```

Type: Integer

Default: `0`

Enables support of Liberty wire-load models and related SDC commands.

Related Information

- "Base Delay Analysis" chapter in the *Innovus System User Guide.*

Related Commands

- `report_wire_load`
- `reset_wire_load_mode`
- `reset_wire_load_model`
- `reset_wire_load_selection_group`
- `set_wire_load_mode`
- `set_wire_load_selection_group`

delaycal_use_default_delay_limit

`delaycal_use_default_delay_limit`

Type: Integer

Default: `1000`

Sets the pin number threshold for nets for delay calculation. Nets that have more than the specified number of pins are excluded from delay calculation, and use the default delay values instead.

Net default delay values are:

- Net delay: `1 ns`
- Transition time: `0 ps`
- Net load: `0.5 pF`

Note: In post route stage, default delay calculation does not honor this variable. Slews and delays are computed based on actual parasitics.

To change the default delay values, set the following global variables:

- `delaycal_default_net_delay`
- `delaycal_default_net_load`
- `delaycal_input_transition_delay`

Example

- The following global variable sets the pin threshold number for nets to 2,000:

  ```
  set delaycal_use_default_delay_limit 2000
  ```

  The software displays the following information:

  ```
  Set Using Default Delay Limit as 2000.
  ```

Nets that exceed pin count of 2,000 are excluded from delay calculation and use the default delay values.

Related Information

- "Base Delay Analysis" chapter in the *Innovus System User Guide*

Related Global Variables

- delaycal_default_net_delay
- delaycal_default_net_load
- delaycal_input_transition_delay

8

Flip Chip Commands and Global Variables

- Flip Chip Commands
- Flip Chip Global Variables

Flip Chip Commands

- addBumpConnectTargetConstraint
- assignBump
- assignPGBumps
- assignSigToBump
- changeBumpMaster
- checkBondPadSpacing
- checkBump
- ciopLoadBumpColorMapFile
- copy_bump
- create_bump
- createSignalPin
- deleteAloFiller
- deleteBumpConnectTargetConstraint
- deleteBumps
- deselect_bump
- edit_bump_name
- editBumpConnectTargetConstraint
- fcroute
- findPinPortNumber
- fixBondPad
- get_physical_info
- getBondPad
- getFlipChipMode
- ioInstOverlapCheck
- placeBondPad
- readFlipChipProperty
- readIoUpdate
- readPackage

- reportProbePins
- reportSpecialRoute
- routePointToPoint
- select_bump
- setBumpFixed
- setBumpPlacementStatus
- setFlipChipMode
- setProbePin
- setSpecialRouteType
- spaceBondPad
- staggerBondPad
- swapSignal
- unassignBump
- unfixBondPad
- unfixBump
- unsetProbePin
- verifyFlipChipRoutingConstraints
- viewBumpConnection
- writeFlipChipProperty

addBumpConnectTargetConstraint

```
addBumpConnectTargetConstraint
[-help]
{-bump list_of_bumps | -selected }
{ {-instName instance_name
      [ {-pinName pin_name | -netName net_name}
        [ -portNum value ]]}
    | -PGConnectType {ioring corering stripe iopin}}
```

Adds a string property in one or multiple bumps. If any of the specified/selected bump has a property value already, Innovus checks whether the new property value is exactly the same as the existing one. If not, the new property value is concatenated to the existing property string and separated by a space.

Using this command, you can add either a complete port numbering property (`inst:pin:port`) or an incomplete property, such as `inst` or `inst:pin` to PG bumps.

Parameters

`-help`	Prints a brief description that includes type and default information for each `addBumpConnectTargetConstraint` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man addBumpConnectTargetConstraint`
`-bump` *`list_of_bumps`* | `-selected`	
	Adds bump connect target constraint on specified or selected bumps. The option `-bump` can accept wildcards and `-selected` can have multiple bumps selected. In the multiple bump case, the same property is added onto all the bumps.
`-instName` *`instance_name`*	
	Specifies the connect target instance. If only `-instName` is specified and `-netName` or `-pinName` is not specified, the tool automatically finds the most suitable geometries in the instance specified by *`instance_name`*.
`-netName net_name`	Specifies the connect target net name. **Note:** This option cannot be used with `-pinName`.
`-PGConnectType {ioring corering stripe iopin}`	
	Specifies the PG connect type. Connection type can be I/O Ring (`ioring`), Core Ring (`corering`), Stripe (`stripe`), or I/O Pin (`iopin`).
`-pinName` *`pin_name`*	
	Specifies the connect target pin name. **Note:** This option cannot be used with `-netName`.
`-portNum value`	Specifies the connect target port number.

Examples

- The following command adds `bump_connect_target` property onto `Bump_1` on `VDD` net. Then `Bump_1` pairs with `PORT1` under the PIN connecting to `VDD` net of arealo `IOPADS_INST/esd`.
  ```
  addBumpConnectTargetConstraint -bump Bump_1 -instName IOPADS_INST/esd -netName VDD -portNum 1
  ```
- The following command adds `bump_connect_target` property onto `Bump_1`. Then `Bump_1` pairs with `PORT2` under the PIN whose name is `vdd!` of instance `DTMF_INST/ PLLCLK_INST`. `PORT2` is required with PORT `CLASS BUMP` attribute in LEF.
  ```
  addBumpConnectTargetConstraint -bump Bump_1 -instName DTMF_INST/PLLCLK_INST -pinName vdd! -portNum 2
  ```

Related Information

- `deleteBumpConnectTargetConstraint`
- `editBumpConnectTargetConstraint`
- `findPinPortNumber`
- Flip Chip Methodologies chapter of the *User Guide*
  - *Port Numbering*

- Tools Menu chapter in the *Menu Reference*
  - *Flip Chip*

assignBump

```
assignBump
[-help]
[{-area x1 y1 x2 y2 | -selected }]
[-constraint_file file_name]
[-exclude_region llx lly urx ury ...]
[-maxDistance distance]
[-multiBumpToMultiPad]
[{[[-pgnet net_list] | [-exclude_pgnet net_list]][-pginst instance_list]} [-pgonly]]
```

Assigns the bumps closest to the I/O cells, using euclidean distance, and adds bump connection target property onto bump. These bumps can be signal bumps specified in the Verilog netlist or power/ground (PG) bumps specified in the command line. From the 14.1 release, `assignBump` adds bump connect target to bumps by default during bump assignment.

This command is for flip chip designs. Bumps must be created or loaded from the I/O assignment file.

Use this command after the design is placed.

Note: The `assignBump` command does not affect any existing assignment. Hence, it will not reassign any existing power/ground assignments.

Support for CLASS BUMP Property

The `assignBump` command supports the `CLASS BUMP` of `PORT` in `LEF`. Using the `LEF` 5.7 syntax, you can add the `CLASS BUMP` property in the `PORT` of the hard macro `LEF`. With this property, `fcroute` is able to route from these ports to the assigned bump.

The `assignBump` command supports I/O pads or hard macros that have pins with multiple geometries. For the following hard macro definition in LEF, `assignBump` assigns three bumps for `block1`, including two bumps for pin `signal1` and one for pin `powerpin`.

:

```
    MACRO block1
        PIN signal1
            PORT
                CLASS BUMP ;
                LAYER … ;
                RECT … ;
            END
            PORT
                CLASS BUMP ;
                LAYER … ;
                RECT … ;
            END
        END signal1
        PIN powerpin
            PORT
                CLASS BUMP ;
                LAYER … ;
                RECT … ;
```

```
        END
      END powerpin
  END block1
```

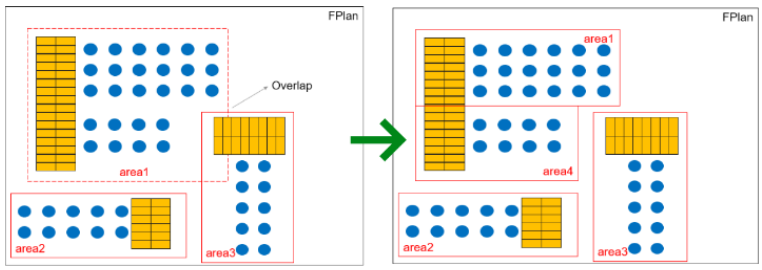
`fcroute` also supports routing for this kind of assignment. In the above example, routing happens from the two signal bump to the two port geometries and one power bump to the powerpin geometry with `CLASS BUMP` property.

The behavior of `CLASS BUMP` is summarized as below:

- If the `CLASS BUMP` property is not defined in any cell, `assignBump` assigns one bump to each IO pin and `fcroute` works normally.
- If any pin port in `CLASS PAD AREAIO` or `CLASS BLOCK` cells contains the `CLASS BUMP` property, `assignBump` uses the property for assignment.
- Each pin port with the `CLASS BUMP` property will have a bump assigned. This means a pin can have multiple bumps assigned to it if it has more than one port defined as CLASS `BUMP`. `fcroute` honors this property and routes the port and its assigned bump. A port with `CLASS BUMP` attribute with multiple geometries is considered as one object for assignment and routing.

Parameters

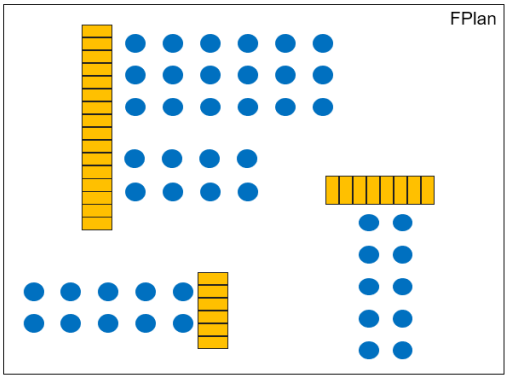
`-help`	Outputs a brief description that includes type and default information for each `assignBump` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man assignBump`
`-area` *`x1 y1 x2 y2`*	Assigns objects in the area specified by the *`x1`*, *`y1`*, *`x2`*, and *`y2`* coordinates. • If the specified area includes only bumps, `assignBump` assigns the selected bumps to IO cells based on the total shortest distance. • If the specified area includes only IO cells, `assignBump` assigns the selected IO cells to bumps based on the total shortest distance. • If the specified area includes both IO cells and bumps, then `assignBump` assigns the selected IO cells to the selected bumps. If the number of bumps are not enough, `assignBump` issues a warning. **Note:** The selected objects must be same as the ones selected by the command `windowSelect`. For designs with double pad rows, the following conditions must be kept in mind while defining area with `assignBump -area`: • All IO pads inside the area should be regular and should be placed in either two rows or two columns. If the pads are irregular, the tool will turn off double pad row assignment automatically and use the traditional optimization algorithm, based on minimal global length. • Define areas in such a way that there is only one double pad row in one area. • If two areas overlap, it means there are now two double pad rows in one area. The results of bump assignment in such situations might not be the most optimal. To get the most optimal results, split one of the areas into two to avoid overlap as shown below:



Now, you can assign bumps using the following commands:

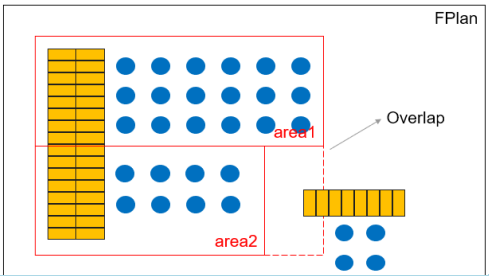
```
assignBump -area area1
assignBump -area area2
assignBump -area area3
assignBump -area area4
```

Note that for assigning bumps in a design with single pad rows, you can use `assignBump` without the `-area` option.



In the following example, the double pad area should be split to avoid overlaps as shown below. The rest of the bumps can be assigned without using the `-area` option:

```
assignBump -area area1
assignBump -area area2
assignBump
```



`-constraint_file` *file_name*	Specifies the path to the file containing bump assignment constraints. At present, the following types of bump assignment constraints are supported: • `SHARE_FIND_PORT` constraint to filter unnecessary ports • `ASSIGN_ANALOG_PG_NETS` constraint to specify which signal nets are analog PG nets • `SHARE_IGNORE_*` and `ASSIGN_IGNORE_*` constraints to exclude instances, macros, pins, or nets for assignment. • `ASSIGN_PAD2BUMP_RATIO` constraint to specify the pad to bump ratio per net, macro, or instance. For more information on specifying bump assignment constraints, refer to the "Using Bump Assignment Constraints" section of the Flip Chip Methodologies chapter of the *User Guide.*
`-exclude_pgnet` *net_list*	Specifies the names of the power/ground nets that must be excluded from bump assignment. Wild cards are supported in net names. The net names should be enclosed in double quotation marks (" ") or braces {}.
`-exclude_region` *llx lly urx ury ...*	Excludes bumps in the area specified by the lower-left x (*llx*), lower-left y (*lly*), upper-right x (*urx*), and upper-right y (*ury*) coordinates. You can specify multiple regions, which may be disjoint.
`-maxDistance` *distance*	Specifies the maximum distance, in microns, to look for a bump connection.
`-multiBumpToMultiPad`	Specifies that multiple pads need to be assigned to multiple bumps. This parameter is useful when you need to assign multiple pads on one net to multiple bumps based on shortest distance.
`-pginst` *instance_list*	Specifies the names of power/ground instances to which the power bumps are assigned. The instance names should be enclosed in double quotation marks (" ") or braces {}.
`-pgonly`	Assigns only power/ground nets to bumps. When this option is specified, signals are not assigned to bumps. **Note:** Option `-pginst` or `-pgnet|-exclude_pgnet` is required for option `-pgonly`.
`-pgnet` *net_list*	Specifies the names of the power/ground nets to which the bumps are assigned. The net names should be enclosed in double quotation marks (" ") or braces {}. By default, bumps are assigned to all pads whose ports are connected to power/ground nets.
`-selected`	Assigns selected bumps to the closest I/O instances or selected I/O instances to the closest bumps. • If only bumps are selected, `assignBump` assigns the selected bumps to IO cells based on the total shortest distance. • If only IO cells are selected, `assignBump` assigns the selected IO cells to bumps based on the total shortest distance. • If both IO cells and bumps are selected, `assignBump` assigns the selected IO cells to the selected bumps. If there are not enough bumps, `assignBump` issues a warning. **Note:** The `-selected` parameter works with other options of `assignBump`, except `-area` and `-exclude_region`.

Examples

- The following command assigns bumps to the power and ground nets, `VDD` and `VSS`, within the area specified by `600.0` (`llx`), `960.0` (`lly`), `2600.0` (`urx`), and `4100.0` (`ury`) coordinates.

  ```
  assignBump -pgnet {VDD VSS} -area {600.0 960.0 2600.0 4100.0}
  ```

- The following command assigns multi-PG pads to bumps:

  ```
  assignBump -pgonly -pgnet {VDD VSS}
  ```

Related Information

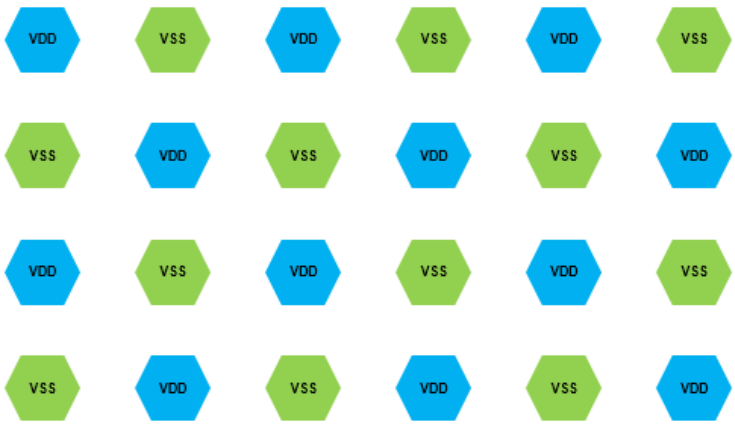
- `assignPGBumps`
- `assignSigToBump`
- `checkBump`
- `create_bump`
- `unassignBump`
- Flip Chip Methodologies chapter of the *User Guide*
- Tools Menu chapter of the *Menu Reference*
  - *Flip Chip*

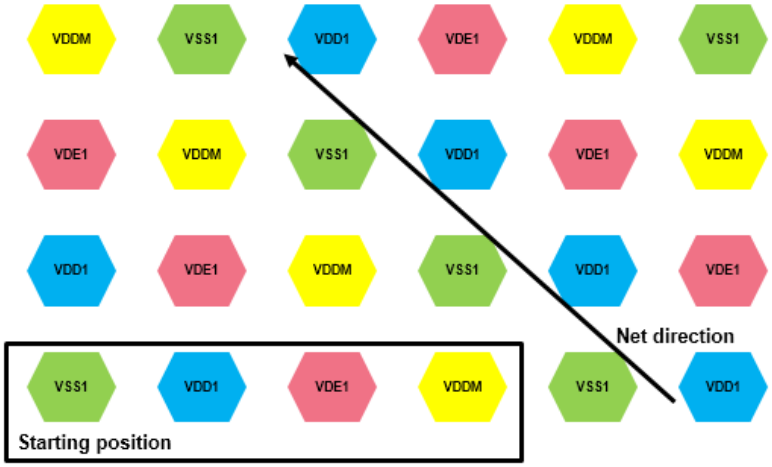
assignPGBumps

```
assignPGBumps
[-help]
[-connectType {ioring | corering | stripe | iopin}]
-nets {name_list}
[-selected | -floating | -bumps {bump_name_list}]
[-V | -H | -checkerboard [-square {width height}]]
```

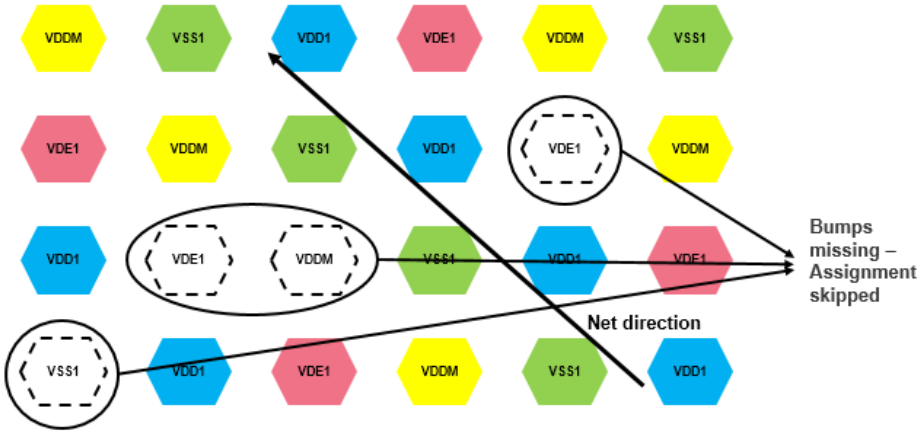
Assigns power and ground bumps to connect flip chip I/O pins and adds property value onto bumps. Use this command after bumps are created. It overwrites existing assignments for selected bumps.

Parameters

`-help`	Prints a brief description that includes type and default information for each `assignPGBumps` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man assignPGBumps`
`-bumps {bump_name_list}`	Specifies the names of the bumps to be assigned. The names must be enclosed in braces. Wildcards are supported in bump names. **Note**: This parameter cannot be used with `-selected` and `-floating`.
`-checkerboard`	Specifies that the checkerboard style of bump assignment will be used if multiple nets are defined using the `-nets` parameter:  The `-checkerboard` parameter is typically used for a regular (rectangular) bump array. To apply this pattern for an irregular (rectilinear) bump array, first form a regular array by creating virtual bumps in the areas where there are no bumps. Next, assign the bumps using the checkerboard pattern and then remove the virtual bumps. **Notes** • The checkerboard style can be used with more than two nets, as shown below: `assignPGBumps -nets {VSS1 VDD1 VDE1 VDDM} -checkerboard -selected`

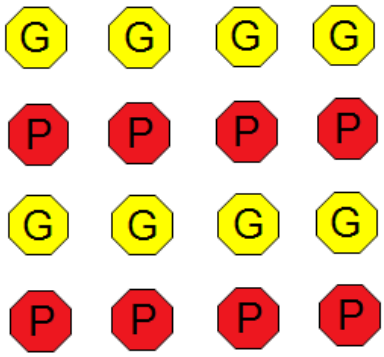
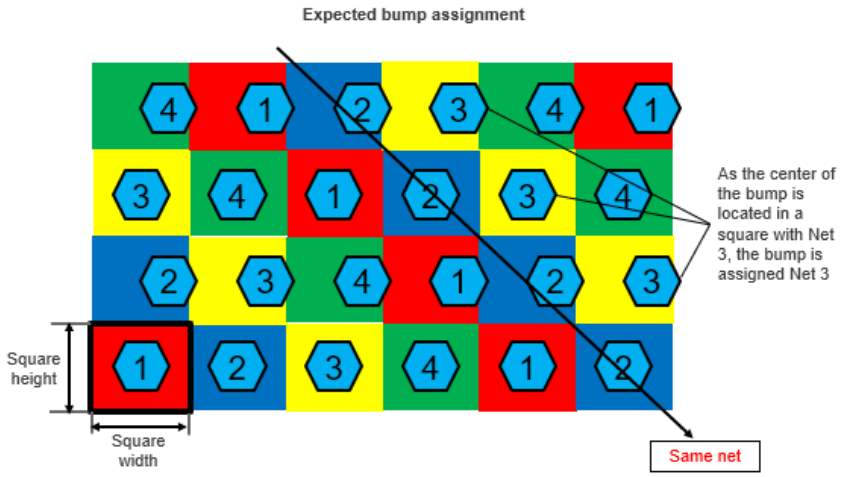


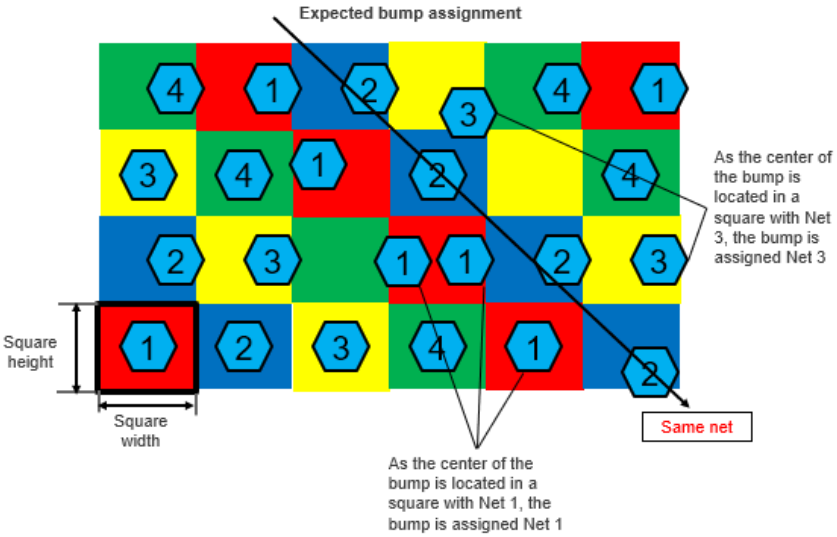
The `-checkerboard` option for multiple nets can be used in a regular bump matrix with the same bump cell for all bumps and the same X/Y pitch between bumps. If bumps are missing in the pattern, the tool skips assignment to maintain the diagonal, staggered net direction.



- The checkerboard style can support irregular bump assignment when used with the `-square` parameter. For details, see the description of the `-square` parameter.

`-connectType {ioring | corering | stripe | iopin}`	
	Specifies the bump connect type when assigning power/ground bumps. Connection type can be I/O Ring, Core Ring, Stripe, or I/O Pin.
`-floating`	Assigns all floating bumps.

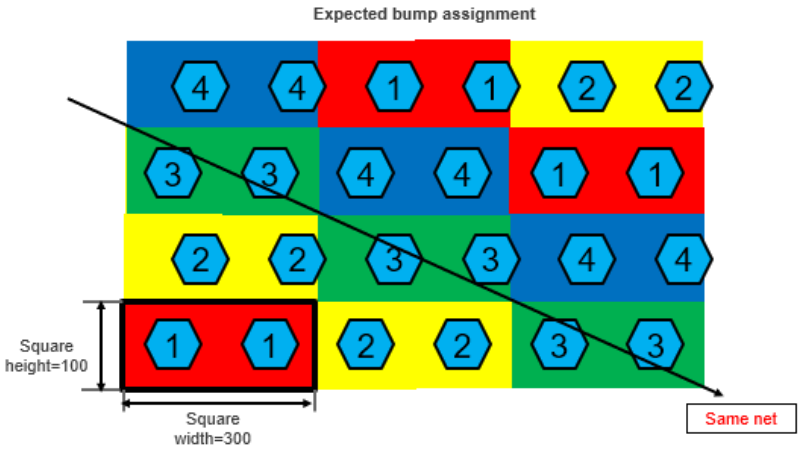
`-H`	Specifies that the horizontal style of bump assignment will be used if multiple nets are defined using the `-nets` parameter: 
`-nets {`*`name_list`*`}`	Specifies the net names. Net names must be enclosed in braces.
`-selected`	Assigns selected bumps.
`-square {`*`width height`*`}`	Specifies the width and height of a square in the checkerboard style of bump assignment. Use this parameter if you want to apply the checkerboard style of bump assignment for an irregular bump matrix. When you specify the `-square` parameter, the tool creates a virtual checkerboard made of squares of the specified dimensions and then assigns the squares with the specified nets. The square to which a bump belongs is determined based on the location of its center. The tool then assigns the corresponding net to the bump. For example, the following command creates a virtual checkerboard with squares of width 150 and height 100, assigns the squares with the Nets 1, 2, 3, and 4, and assigns the nets to bumps based on the square on which their center is located: `assignPGBumps -nets {1 2 3 4} -checkerboard -selected -square {150 100}` **Case A**  As shown in Case B below, the assignment is dependent on the location of the center of the bumps. **Case B**



The assignment also changes if you change the dimensions specified for the checkerboard squares. For example, the following command creates a virtual checkerboard with squares of width 300 and height 100.

```
assignPGBumps -nets {1 2 3 4} -checkerboard -selected -square {300 100}
```

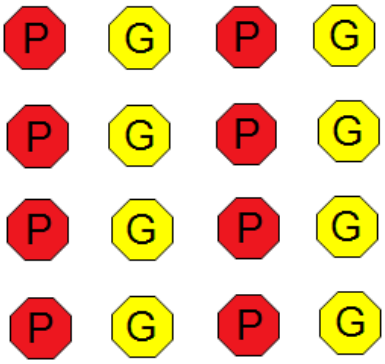
Case C



Note that although the bump locations are same in both Cases A and C, the assignments in Case C are different from Case A because of the change in square dimensions.

Notes

- The `-square` parameter can be used only if `-checkerboard` is specified.
- The `-square` parameter can be used for selected as well as floating bumps.

`-V`	Specifies that the vertical style of bump assignment will be used if multiple nets are defined using the `-nets` parameter: 

Example

- The following command assigns selected bump to the power net `VDD`:
  ```
  assignPGBumps -nets VDD -selected
  ```
- The following command assigns only the specified bumps to the power net `VDD`:
  ```
  assignPGBumps -nets VDD -bumps {Bump_1 Bump_2 Bump_3}
  ```

- The following command assigns floating bumps to the specified nets in the horizontal style:
  ```
  assignPGBumps -H -floating -nets {VSS1 VDD1 VDE1}
  ```
- The following command assigns selected power/ground bumps in the checkerboard style to the VDD and VSS nets:
  ```
  assignPGBumps -selected -nets {VDD VSS} -checkerboard
  ```
- The following command assigns all the floating bumps in checkerboard pattern to the VDD and VSS nets:
  ```
  assignPGBumps -floating -nets {VDD VSS} -checkerboard
  ```

Related Information

- `assignBump`
- `assignSigToBump`
- `unassignBump`
- Flip Chip Methodologies chapter of the *User Guide*
- Tools Menu chapter in the *Menu Reference*
  - *Flip Chip*

assignSigToBump

```
assignSigToBump
[-help]
{ {{-net net_name | {-top_pin port [-bump_pin bump_pin_name]}} {-bumps bump_name_list | -selected}} }
```

Assigns selected or specified bumps to the specified net or pin. The command supports signal, power, as well as ground nets. Feedthrough bumps with multiple pins are also supported.

Parameters

`-help`	Prints a brief description that includes type and default information for each `assignSigToBump` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man assignSigToBump`
`-bump_pin` *`bump_pin_name`*	Specifies the name of the bump pin that has to be assigned to the specified top pin. Use this option for feedthrough bumps, which have multiple pins. This parameter can be used only with the `-top_pin` parameter.
`-bumps` *`bump_name_list`*	Specifies the bumps to be assigned to the net specified with `-net`. Use braces to enclose the bump names.
`-net` *`net_name`*	Specifies the signal or PG net to which bumps are to be assigned.
`-selected`	Assigns the selected bumps to the specified net.
`-top_pin` *`port`*	Specifies the name of the top-level pin to which bumps are to be assigned. After assignment, the pin name of the bump is the top-level pin name and the net name of the bump is the net name of the specified pin. This parameter cannot be used with the `-net` parameter.

Examples

- The following command assigns `bump_1` and `bump_2` to the signal net `sig[1]`:
  ```
  assignSigToBump -net sig[1] -bumps {bump_1 bump_2}
  ```
- The following command assigns `bump_1` to the power net `VDD`:
  ```
  assignSigToBump -net VDD -bumps bump_1
  ```
- The following command assigns the selected bumps to the ground net `VSS`:
  ```
  assignSigToBump -net VSS -selected
  ```
- The following command assigns the `PIN_1` pin of the `bump1` feedthrough bump to the top-level pin, `CLK_IN`:
  ```
  assignSigToBump -bumps bump1 -top_pin CLK_IN [-bump_pin PIN_1]
  ```

Related Information

- `assignBump`
- `assignPGBumps`
- `unassignBump`
- Flip Chip Methodologies chapter of the *User Guide*
- Tools Menu chapter in the *Menu Reference*
  - *Flip Chip*

changeBumpMaster

```
changeBumpMaster
[-help]
-bumpMasterName bumpMasterName
[-fromBumpMasterName bumpMasterName ]
[-location_type {cell_center cell_lowerleft geometry_center geometry_lowerleft}]
{-netName netName | -allBumps | -bump_name bump_list | -selected}
```

Enables you to replace the cell master for specified bumps.

Parameters

`-help`	Prints a brief description that includes type and default information for each `changeBumpMaster` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man changeBumpMaster`
`-allBumps`	Specifies that all bumps will use the new cell master specified using the `-bumpMasterName` parameter.
`-bumpMasterName bumpMasterName`	
	Specifies the name of the new cell master.
`-bump_name bump_list`	
	Specifies the list of bumps for which you want to change the bump master. Use this parameter if you want to change the cell master of specific bumps, which may or may not have the same cell master originally.
`-fromBumpMasterName bumpMasterName`	
	Specifies the name of the old cell master. Use this parameter to replace the cell master of bumps using a specific bump master.
`-location_type {cell_center cell_lowerleft geometry_center geometry_lowerleft}`	
	Specifies the location type. *Default*: `cell_lowerleft`
`-netName netName`	Specifies that all bumps connected to the specified net will use the new cell master. For example, if the *netName* is VDD, all connected bumps will use the cell master specified using the `-bumpMasterName` parameter.
`-selected`	Changes the bump master of selected bumps to the new cell master as specified using the `-bumpMasterName` parameter.

Examples

- The following command changes the cell master of selected bumps from `BUMPCELL` to `BGA_PAD`:
  ```
  changeBumpMaster -bumpMasterName BGA_PAD -selected -fromBumpMasterName BUMPCELL
  ```
  Note: The change happens only for those of the selected bumps that originally belonged to `BUMPCELL`. If a selected bump belongs to a cell master other than `BUMPCELL`, its cell master is not changed.
- The following command changes the cell master of all the bumps connected to the net `int` to `BUMP_PWR`:
  ```
  changeBumpMaster -bumpMasterName BUMP_PWR -netName int
  ```
- The following command changes the cell master of all the bumps in the design to `BUMP_PWR`:
  ```
  changeBumpMaster -bumpMasterName BUMP_PWR -allBumps
  ```
- The following command changes the the cell master of all the bumps connected to the net `int` to `BUMP_PWR` and specifies their location type as `geometry_center`:
  ```
  changeBumpMaster -bumpMasterName BUMP_PWR -netName int -location_type geometry_center
  ```

Related Information

- Flip Chip Methodologies chapter in the *User Guide*

checkBondPadSpacing

```
checkBondPadSpacing [-help]
```

Checks for bond pad spacing violations after using the `spaceBondPad` command. You can use this command at any time after using the `spaceBondPad` command.

Parameters

`-help`	Prints a brief description of the `checkBondPadSpacing` command. For a detailed description of the command, use the `man` command: `man checkBondPadSpacing`

Related Commands

- `fixBondPad`
- `getBondPad`
- `placeBondPad`
- `spaceBondPad`
- `staggerBondPad`
- `unfixBondPad`

checkBump

```
checkBump
[-help]
[-bumpToPinOverlap inst+]
[-outfile fileName]
[-resetViolationMarker]
[-select_violation_bumps]
[-selected | -area {x1 y1 x2 y2}]
[-relative_type {embedded_bump inst_pin_port} [[-relative_object {inst_pin_port_list}] [-relative_assignment ] [-
relative_offset x y]]]
[ [[-relative_type {embedded_bump inst_pin_port}] [-bumpToPinPitch {pitch_value {[bump_name] [inst_pin_name [pin_x
pin_y]]}}]]
  | -nets net+ ]
[-bumpToEdgePitch pitch_value [-edgeOffset {offset [cornerOffset]}]]
[ [[-bumpPitch {pitch_value [bump_name]}]
   [-bumpToPinPitch {pitch_value {[bump_name] [inst_pin_name [pin_x pin_y]]}}]]
  | -bumpMaxPitch max_pitch_value | -bumpToEdgePitch pitch_value]
[-onBumpGrid [-onBumpGridTolerance bump_grid_name value]]
```

Checks the legality of the bump assignment and creates a report, which is either displayed in the console window or written to a file. Use this command for bump overlap and pitch checking. The `checkBump` command can be used after bumps are assigned with the `assignBump` command or defined in the I/O file with the signal name but without the `-fixed` constraint.

Bump-to-Bump Overlap Checking

`checkBump` checks bump overlaps, including overlap of dummy bumps, by default. If a bump overlaps with others, `checkBump` places violation markers on the center of the overlapped bumps and reports the total number of overlapped bumps.

The violation markers for bump overlap can be browsed in Violation Browser.

⚠ `checkBump`:

- Checks overlaps between regular bumps by checking for pin overlaps on bumps.
- Checks overlaps between dummy bumps by checking for OBS overlap.
- Checks overlaps between dummy bumps and regular bumps by checking for OBS and bump pin overlap.

Bump-to-Pin Overlap Checking

If `-bumpToPinOverlap` is used, `checkBump` checks for overlaps between bump shapes, including dummy bumps, and pins of the specified block. If the bump shape and the block pins are at the same location but on different layers, the tool does not consider them to be overlapping. However, if the bump shapes and the block pins are at the same location and on the same layer, they are reported as overlapping, as shown below:

Bump Pitch Checking

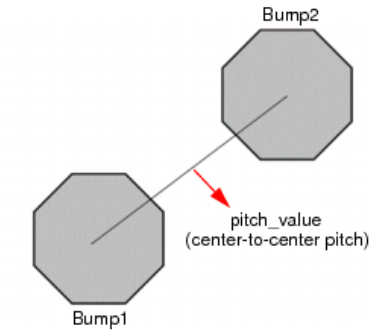
`checkBump` provides a bump pitch checking feature to record bumps that violate the bump pitch. The bump pitch checking feature has multiple options:

- `-bumpPitch` for checking the center-to-center pitch between bumps. The bumps that have a smaller center-to-center distance than the specified *`pitch_value`* are marked as spacing violations.
- `-bumpMaxPitch` for checking whether the center-to-center pitch between neighboring bumps violates the maximum spacing constraint.
- `-bumpToEdgePitch` for checking the pitch from the bump center to the guard or die edge.
- `-bumpToPinPitch` for checking the pitch between bump and pins of hard macros. The bumps and instance pin pairs that have a smaller bump_center-to-pin_center or bump_center-to-specified_pin_geometry_location distance than the *`pitch_value`* are marked as spacing violations.

In all these cases, bumps that violate the pitch request are marked as spacing violations and can be checked using the Violation Browser.

Bump-to-Bump Pitch Checking

For bump to bump pitch checking, the center-to-center bump pitch is needed. The center of a bump is calculated using the bump cell geometry at the topmost RDL layer.



Note: Only bump cell with single geometry at one RDL layer is supported. For multiple-layer bump cell, the topmost layer in the bump cell is the topmost RDL layer. If no geometry is defined at that layer, the geometry on the second topmost layer is used and a warning is issued:

No geometry at XX layer found; the geometry at XXX layer is used for bump center calculation.

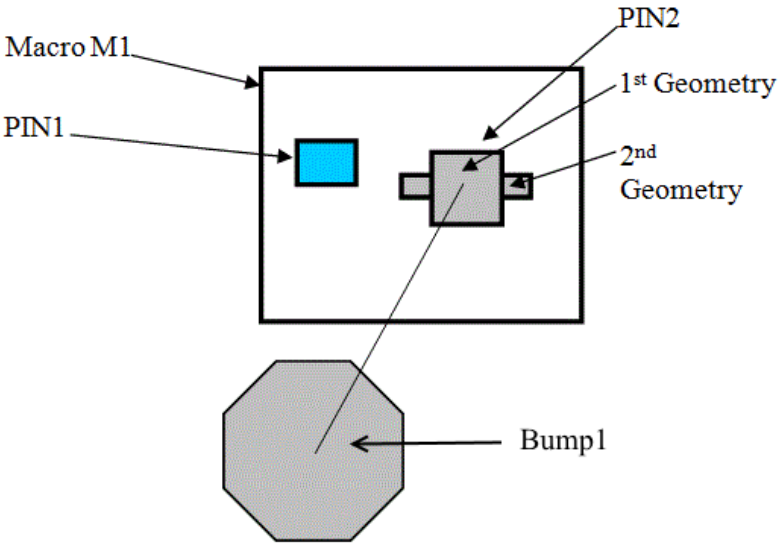
If the check is between a multiple-layer bump and a single-layer bump, the center is calculated using the geometries on the topmost layer in each cell. For example, in the following diagram, center-to-center pitch between `Bump1` and `Bump2` is calculated using the geometries on the top metal in `Bump1` and the second top metal layer in `Bump2`.

If there is no common layer in two bumps, there is no center-to-center pitch to be checked. For example, there is no violation to be reported for `Bump2` and `Bump3` in the above diagram.

Bump-to-Pin Pitch Checking

In some designs with IP macros, bumps have already been embedded inside the macro and are considered as macro pins. In this case, `checkBump` checks pitch between the bump center and the pin center. Bump center is defined in the same way as in the bump-to-bump case. Pin center calculation for pitch checking can vary as follows:

- Pin has multiple geometries on common layer
  `checkBump` selects all geometries of the pin if its layer is equal to bump's layer. For example, if bump only has topmost geometry, `checkBump` selects all the geometries on the topmost layer for pin. If the bump has both topmost and second layer geometry, `checkBump` selects the geometries for both layers. If no valid geometry of pin is found, `checkBump` skips the pin pitch checking for that pin.
  In the example below, the tool checks one by one the pitch for each geometry of `PIN2` on `Bump1`'s layer to see if there is a violation. That is, `checkBump` calculates the distance from the center of 1st geometry of `PIN2` to the center of `Bump1`. Then, it calculates distance from the center of 2nd geometry of `PIN2` to center of `Bump1`. For each check, `checkBump` reports a violation if the distance is less than `100`.
  ```
  checkBump -bumpToPinPitch {100 {Bump1 M1/PIN2}}
  ```



- Pin has multiple geometries and a particular geometry location is specified
  In case of multiple geometries, `checkBump` allows you to specify a location `[ pin_x pin_y ]`. If this location overlaps with a geometry, the pitch to this geometry is checked. `checkBump` calculates the distance from the center of this geometry to the bump center.

Note: `checkBump` does not support pin-to-pin pitch checking because such violations can only be fixed at LEF level which is beyond the flipchip tool capability. So in the following command:

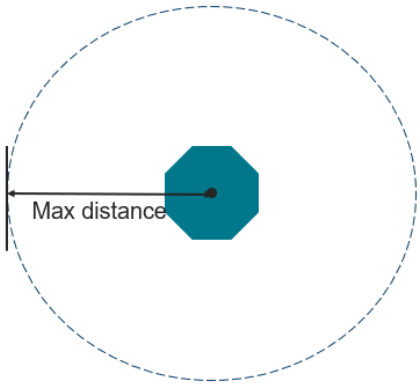
```
checkBump -bumpToPinPitch {100 {Bump1 M1/PIN1 M1/PIN2}}
```

`checkBump` checks `Bump1` and `M1/PIN1`; and `Bump1` and `M1/PIN2`; but not `M1/PIN1` and `M1/PIN2`.

Parameters

`-help`	Prints a brief description that includes type and default information for each `checkBump` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man checkBump`
`-area {x1 y1 x2 y2}`	Checks constraints for the bumps in the specific area. The unit is microns. The bumps selected by `-area` should be the same as the ones selected by the command `windowSelect`. If `-area` is used with `-bumpPitch`, `checkBump` checks the bump pitch only for the bumps in the specific area. If `-area` is specified without `-bumpPitch`, `checkBump` performs the default checks for the bumps in the specific area. **Note:** This parameter cannot be used with with `-bumpToPinPitch` or `-selected`.
`-bumpMaxPitch max_pitch_value`	

Checks whether the center-to-center pitch between neighboring bumps exceeds the *max_pitch_value*, which is specified in microns. For any bump, the search area is defined by a circle with a radius of *max_pitch_value* and the bump center as its center.

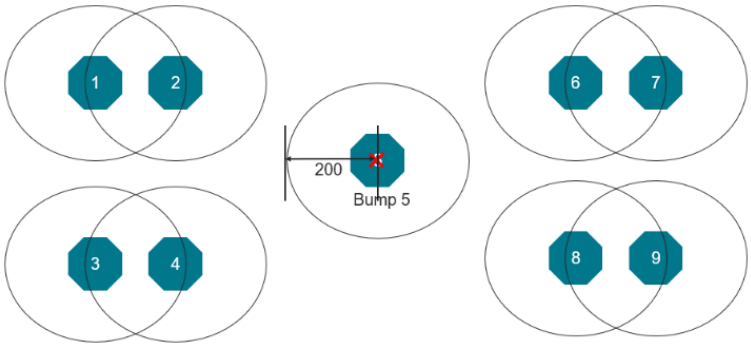


Search Area

If another bump is detected within a bump's search area, the bump is DRC free. If no other bump is detected in a bump's search area, the bump is marked with a maximum spacing violation and a warning message is displayed.

In the example below, only Bump 5 violates the maximum spacing constraint and is flagged with a maximum spacing violation:

```
checkBump -bumpMaxPitch 200
```



The `-bumpMaxPitch` parameter works on:

- all bumps in the design by default.
- selected bumps if `-selected` is specified.
- bumps in a specific area if `-area` is specified.
- same-net bumps if `-nets` is specified.

Note: The `-bumpMaxPitch` parameter cannot be specified along with the `-bumpPitch` or the `-bumpToEdgePitch` parameter.

`-bumpPitch {`*pitch_value* `[`*bump_name*`]}`

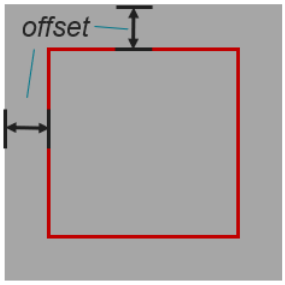
	Checks the center-to-center pitch between bumps. Here: • *pitch_value* represents the center-to-center pitch value in microns. • *bump_name* is the name of the bump. You can check pitch between two or more bumps. Wildcards can be used to specify bump names. If more than two bumps are specified, bump pitch is checked for all possible pairings. In the following example, checkBump checks center-to-center pitch between B1 and B2; B1 and B3; B2 and B3: `checkBump -bumpPitch {100 {B1 B2 B3}}` Any illegal pitch is logged as "Bump pitch between B1 and B2 XX < 100." Specifying bump names is optional. If bump names are specified, -bumpPitch works on the specified bumps. If bump names are not specified, -bumpPitch works on: ◦ selected bumps if -selected is specified. ◦ bumps in a specific area if -area is specified. ◦ all bumps in the design, if neither -selected nor -area is specified. **Note:** You can specify pitch checking multiple times in this option.
-bumpToEdgePitch *pitch_value*	
	Checks whether the pitch from the bump center to the guard or die edge is within the specified pitch value. If the detected pitch is less than the specified value, a warning is displayed and the bump is flagged with a spacing violation. *pitch_value* is specified in microns. By default, if -bumpToEdgePitch is specified, the tool checks the pitch from the bump center to the die edge. You can additionally specify -edgeOffset with this parameter to define a guard shape. If -edgeOffset is specified, the tool checks the distance from the bump center to the guard edge defined by -edgeOffset. If the bump center is outside the guard edge, the center-to-guard pitch will be a negative number, which is not allowed. The tool issues a warning in such cases. Check the **Examples** section to see how the -bumpToEdgePitch parameter can be used. The -bumpToEdgePitch parameter works on: • all bumps in the design by default. • selected bumps if -selected is specified. • bumps in a specific area if -area is specified. • bumps with specific nets if -nets is specified. **Note**: The -bumpToEdgePitch parameter cannot be specified along with the -bumpPitch or the -bumpMaxPitch parameter.
-bumpToPinOverlap *inst+*	

	Checks if the bump overlaps with the pins of the specified block. If the bump shape and the block pins are at the same location but on different layers, the tool does not consider them to be overlapping. However, if the bump shapes and the block pins are at the same location and on the same layer, they are reported as overlapping. The `-bumpToPinOverlap` parameter works on: • all bumps, by default. • selected bumps, if `-selected` is specified. • bumps in the specified area, if `-area` is specified. • bumps with the specified nets, if `-net` is specified. *`inst+`* should be of type block. Overwise, the tool reports an error.
`-bumpToPinPitch {`*`pitch_value`* `{[`*`bump_name`*`][`*`inst_pin_name`* `[`*`pin_x pin_y`*`]]}}`	
	Checks the center-to-center pitch between bumps and pins of hard macros. Here: • *`pitch_value`* represents the center-to-center pitch value in microns. • *`bump_name`* is the name of the bump. • *`inst_pin_name`* is the name of the instance pin. • *`pin_x`* and *`pin_y`* specify the pin geometry. **Note**: You can specify pitch checking multiple times in this option. Wildcards can be used in bump and pin names.
`-edgeOffset {`*`offset`* `[`*`cornerOffset`*`]}`	

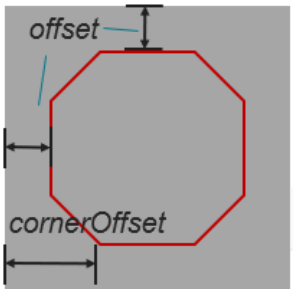
Defines the guard shape if it is different from the die boundary. The `-edgeOffset` parameter can be used only if the `-bumpToEdgePitch` parameter is specified.

You can specify either one or two values with the `-edgeOffset` parameter:

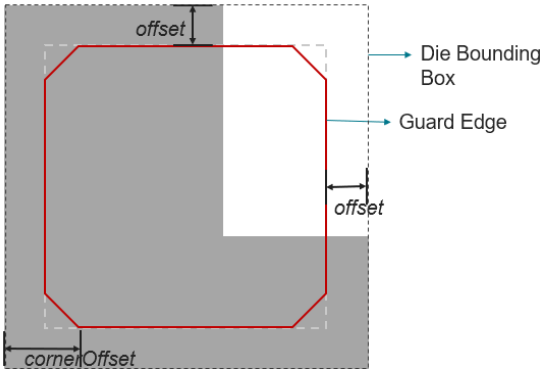
- If only one value (*`offset`*) is specified, the guard edge shape is a rectangle with the die boundary to guard edge distance being equal to the specified `offset` value.

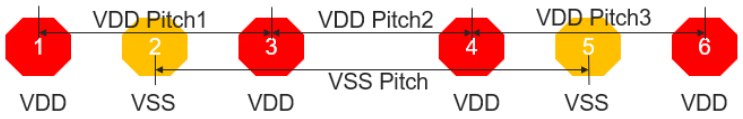


- If two values (*`offset`* and *`cornerOffset`*) are specified, the guard edge shape is an octagon with 45-degree exterior angles. Here, the guard edge to the die boundary distance is the specified *`offset`* value and the distance from the die boundary to the lower-left point of the guard edge is the specified *`cornerOffset`* value.



In a rectilinear design, you can use the bounding box of the die as a reference for specifying the offset for the guard edge.



`-nets net+`	Checks constraints for the bumps with the specified nets. If `-bumpMaxPitch` is specified along with `-nets`, the tool checks for the maximum pitch constraint only between bumps belonging to the same net. For instance, in the example given below, if you specify the command `checkBump -bumpMaxPitch 300 -nets {VDD VSS}`, the tool checks: • Checks the `VDD Pitch1` between bump1 and bump3, `VDD Pitch2` between bump3 and bump4, and `VDD pitch3` between bump4 and bump6 • Checks the `VSS Pitch` between bump2 and bump5.  If `-bumpMaxPitch` is not specified, the tool performs default checks for the bumps with the specified nets. **Note**: The `-nets` parameter cannot be used along with the `-bumpToPinPitch` or `-relative_type` parameter.
`-onBumpGrid`	Checks if the bumps are on bump grids. This option cannot be used with any of the other parameters, except `-onBumpGridTolerance` and `-outfile`. The command errors out in the following cases: • If there is no bump grid in the DB - In this case, use the `create_bump_grid` command to create a bump grid. • If there are multiple bump grids on a single layer The command reports an error but keeps checking in the following case: • If there are multiple bump grids in different layers for one bump Warnings are reported in the following cases: • If there is no bump grid on the bump's layer • If a bump is off the bump grid - The bump is also marked with a DRC marker.
`-onBumpGridTolerance bump_grid_name value`	

	Specifies the tolerance value to be used when checking if bumps are on the specified bump grid. If this parameter is specified, the tool checks the Euclidean distance between the bump center and the specified grid and reports a violation only if the distance is more than the specified tolerance value. ⚠ • The `-onBumpGridTolerance` option can be specified only when `-onBumpGrid` is specified. • The value specified with `-onBumpGridTolerance` should be smaller than half of the Euclidean distance between two diagonal bump grid points. That is: $value < \frac{\sqrt{(bump_grid_pitch_x)^2+(bump_grid_pitch_y)^2}}{2}$ Otherwise, no violation can be reported.
`-outfile` *fileName*	Specifies the report filename. If no filename is specified, the report is printed to the console window.

`-relative_assignment`	Checks whether the net assignment of the target bump is equivalent to corresponding relative object. If not, it issues an error message and places a violation marker at the geometry center of the target bumps:
	• For `-relative_type embedded bump`, the following error message is displayed:
	`The assignment of $bump $net is not connected to the pin of the relative object $blockName/$pinName $expectedNet.`
	• For `-relative_type inst_pin_port`, the following error message is displayed:
	If `-relative_type` is specified as `embedded_bump` and there is no bump in the region being checked, `checkBump` does not check further for relative assignment and issues the following error message:
	`There is no bump overlapping with the relative object $blockName/$pinName located at {x2 y2} with relative offset {x y}. So stop checking the relative assignment for this relative object.`
	Note: This option can be used only if `-relative_type` is specified.

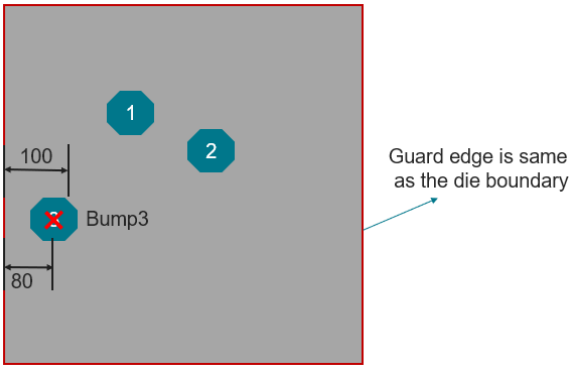
`-relative_object {`*inst_pin_port_list*`}`	

	Specifies the inst_pin_port list and supports wild cards. The format is `{inst:pin:port ...}` and `port` is of type integer. If `-relative_object` is specified, `checkBump` checks whether the geometry center of the target bumps is at the same location as the geometry center of the specified port. If not, it issues an error message and places a violation marker on the geometry center of the target bumps. In addition: • If no bump overlaps with the specified port, `checkBump` displays the message that there are no bumps in the design overlapping with the specified port. • If the specified ports do not exist, an error message is displayed. **Note**: When used with `-selected` or `-area`, the `relative_object` option works only on the specified bumps by selection or area.
`-relative_offset x y`	Specifies the offset of bump location compared with the corresponding relative object. This option can be used only if `-relative_type` is specified. *Default*: `{0 0}`
`-relative_type {embedded_bump inst_pin_port}`	
	Specifies the type of relative object: • `embedded_bump`: If this option is specified, `checkBump` checks whether the geometry center of the target bumps is at the same location as the geometry center of the overlapping embedded bumps. If not, it issues an error message and places a violation marker on the geometry center of the target bumps. In addition: ◦ If no bump overlaps with the embedded bumps, `checkBump` displays an error message as follows: `There is no bump overlapping with the relative object $blockName/$pinName located at {x2 y2} with relative offset {x y}.` ◦ If more than one bump is located at the region being checked, all bumps violating the embedded bump are reported. A summary of the number of the bumps violating the embedded bump is displayed. Violation markers are placed on the embedded bump as well as the bumps violating the embedded bump. The violations can be viewed in Violation Browser. ◦ If there are no embedded bumps, an error message is displayed. **Note**: As this check happens on the entire design, you cannot use the `-selected` or `-area` options if `-relative_type embedded_bump` is specified. • `inst_pin_port`: If the type is `inst_pin_port`, you must specify the `-relative_object {inst_pin_port_list}` option.
`-resetViolationMarker`	
	Resets violation markers.
`-select_violation_bumps`	
	Selects automatically all the bumps that are violating any check rules.

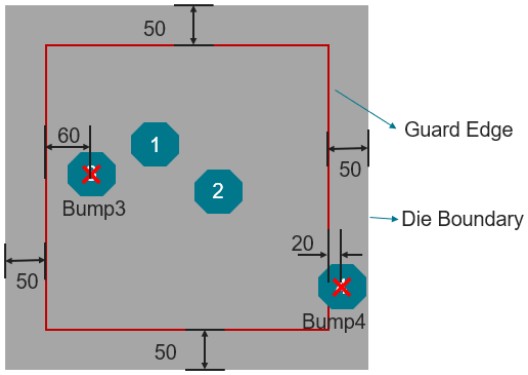
`-selected`	Checks constraints for selected bumps. If `-selected` is used with `-bumpPitch`, `checkBump` checks the bump pitch only for the selected bumps. If `-selected` is specified without `-bumpPitch`, `checkBump` performs the default checks for the selected bumps. **Note**: This parameter cannot be used with with `-bumpToPinPitch` or `-area`.

Example

- The following command checks the center-to-center pitch for all bumps. If the center-to-center distance between two bumps is smaller than 100, it reports a violation:
  ```
  checkBump -bumpPitch 100
  ```
- The following command checks the center-to-center pitch between bump_1 and bump_2:
  ```
  checkBump -bumpPitch {100 {bump_1 bump_2}}
  ```
- The following command checks the center-to-center pitch for the selected bumps:
  ```
  checkBump -bumpPitch 100 -selected
  ```
- The following command checks the center-to-center pitch for bumps in the specified area:
  ```
  checkBump -bumpPitch 100 -area {100 400 600 900}
  ```
- The following command checks whether the bump center to die edge pitch is less than 100 microns:
  ```
  checkBump -bumpToEdgePitch 100
  ```
  In the design below, Bump3 is flagged with a spacing violation and a warning message is displayed:

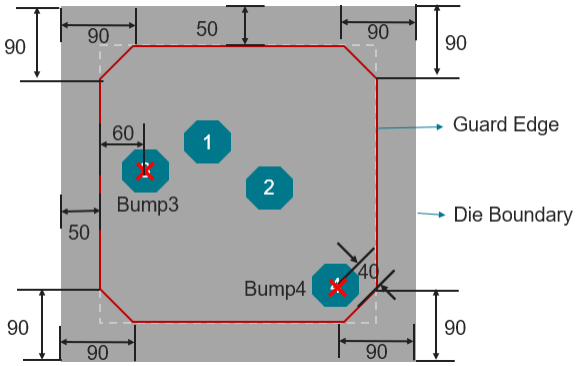
- The following command checks whether the bump center to guard edge pitch is less than 100 microns. The guard edge is offset from the die boundary by 50 microns:
  ```
  checkBump -bumpToEdgePitch 100 -edgeOffset 50
  ```
  In the design below, Bump3 and Bump4 are flagged with spacing violations and warnings are displayed:



- The following command checks whether the bump center to guard edge pitch is less than 100 microns. The guard edge is offset from the die boundary by 50 microns and the guard corners are offset by 90 microns:

  ```
  checkBump -bumpToEdgePitch 100 -edgeOffset {50 90}
  ```

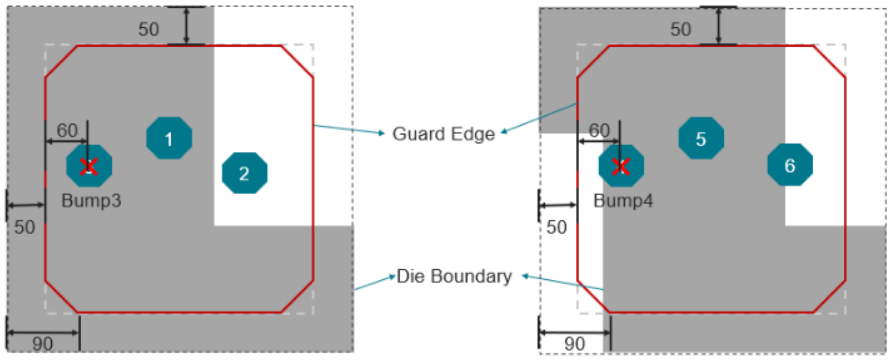
  In the design below, Bump3 and Bump4 are flagged with spacing violations and warnings are displayed:



- The following command checks whether the bump center to guard edge pitch is less than 100 microns. The guard edge is offset from the die boundary by 50 microns and the guard corners are offset by 90 microns:

  ```
  checkBump -bumpToEdgePitch 100 -edgeOffset {50 90}
  ```

  In the rectilinear designs below, Bump3 and Bump4 are flagged with spacing violations and warnings are displayed. However, Bumps2 and Bump6 are DRC free even though they are outside the die boundary because they are still within the guard shape:



- The following command checks the bump assignment and sends the report to `test.rpt`:

  ```
  checkBump -outfile test.rpt
  ```

  A sample report file is as follows:

  ```
  ##############################################################
  ```

```
# Generated by: Cadence Innovus 21.10-d226_1
# OS: Linux x86_64(Host ID noivl-leenap)
# Generated on: Tue Dec 1 13:22:12 2020
# Design: DTMF_CHIP
# Command: checkBump -outfile test.rpt
############################################################

*** The statistics information for bumps ***
Bump_23_2_2 Assigned port_pad_data_out[9] IOPADS_INST/Ptdspop09
Bump_22_1_2 Assigned port_pad_data_out[10] IOPADS_INST/Ptdspop10
Bump_61_0_6 Assigned port_pad_data_out[11] IOPADS_INST/Ptdspop11
Bump_14_3_1 Assigned port_pad_data_out[14] IOPADS_INST/Ptdspop14
Bump_37_6_3 Assigned port_pad_data_in[4] IOPADS_INST/Ptdspip04
Bump_74_3_7 Assigned port_pad_data_in[9] IOPADS_INST/Ptdspip09
Bump_32_1_3 Assigned port_pad_data_in[13] IOPADS_INST/Ptdspip13
Bump_62_1_6 Assigned vcom IOPADS_INST/Pvcomop
Bump_83_2_8 Assigned scan_out_2 IOPADS_INST/Pscanout2op
Bump_12_1_1 Assigned port_pad_data_out[5] IOPADS_INST/Ptdspop05
Bump_66_5_6 Assigned port_pad_data_out[6] IOPADS_INST/Ptdspop06
Bump_4_3_0 Assigned port_pad_data_out[12] IOPADS_INST/Ptdspop12
Bump_51_0_5 Assigned port_pad_data_in[7] IOPADS_INST/Ptdspip07
Bump_13_2_1 Assigned port_pad_data_in[3] IOPADS_INST/Ptdspip03
Bump_71_0_7 Assigned port_pad_data_in[6] IOPADS_INST/Ptdspip06
Bump_44_3_4 Assigned port_pad_data_in[12] IOPADS_INST/Ptdspip12
Bump_89_8_8 Assigned port_pad_data_in[14] IOPADS_INST/Ptdspip14
Bump_52_1_5 Assigned port_pad_data_in[15] IOPADS_INST/Ptdspip15
Bump_26_5_2 Unassigned
Bump_35_4_3 Assigned port_pad_data_out[4] IOPADS_INST/Ptdspop04
Bump_41_0_4 Assigned port_pad_data_out[7] IOPADS_INST/Ptdspop07
Bump_1_0_0 Assigned port_pad_data_out[13] IOPADS_INST/Ptdspop13
Bump_73_2_7 Assigned port_pad_data_in[5] IOPADS_INST/Ptdspip05
Bump_63_2_6 Assigned port_pad_data_in[8] IOPADS_INST/Ptdspip08
Bump_55_4_5 Assigned port_pad_data_in[11] IOPADS_INST/Ptdspip11
Bump_88_7_8 Assigned pllrst IOPADS_INST/Ppllrstip
Bump_3_2_0 Assigned ibias IOPADS_INST/Pibiasip
Bump_34_3_3 Unassigned
Bump_42_1_4 Unassigned
Bump_72_1_7 Unassigned
Bump_5_4_0 Assigned port_pad_data_out[8] IOPADS_INST/Ptdspop08
Bump_19_8_1 Assigned port_pad_data_out[15] IOPADS_INST/Ptdspop15
Bump_85_4_8 Assigned port_pad_data_in[10] IOPADS_INST/Ptdspip10
Bump_46_5_4 Unassigned
Bump_47_6_4 Assigned refclk IOPADS_INST/Prefclkip
Bump_84_3_8 Assigned reset IOPADS_INST/Presetip
Bump_25_4_2 Unassigned
Bump_48_7_4 Unassigned
Bump_24_3_2 Unassigned
Bump_33_2_3 Unassigned
Bump_11_0_1 Unassigned
Bump_75_4_7 Unassigned
Bump_43_2_4 Assigned spi_data IOPADS_INST/Pspidip
Bump_82_1_8 Assigned spi_fs IOPADS_INST/Pspifsip
Bump_76_5_7 Assigned scan_in_2 IOPADS_INST/Pscanin2ip
Bump_27_6_2 Assigned vcop IOPADS_INST/Pvcopop
Bump_7_6_0 Assigned scan_out_1 IOPADS_INST/Pscanout1op
Bump_6_5_0 Assigned tdigit[0] IOPADS_INST/Ptdigop0
Bump_17_6_1 Unassigned
Bump_79_8_7 Unassigned
```

```
Bump_2_1_0 Assigned int IOPADS_INST/Pintip
Bump_64_3_6 Assigned test_mode IOPADS_INST/Ptestmdip
Bump_67_6_6 Assigned scan_clk IOPADS_INST/Pscanckip
Bump_31_0_3 Assigned scan_en IOPADS_INST/Pscanenip
Bump_78_7_7 Assigned tdigit[1] IOPADS_INST/Ptdigop1
Bump_16_5_1 Assigned tdigit[2] IOPADS_INST/Ptdigop2
Bump_10_9_0 Assigned tdigit[3] IOPADS_INST/Ptdigop3
Bump_30_9_2 Assigned tdigit[6] IOPADS_INST/Ptdigop6
Bump_29_8_2 Unassigned
Bump_57_6_5 Unassigned
Bump_81_0_8 Assigned scan_in_1 IOPADS_INST/Pscanin1ip
Bump_86_5_8 Assigned port_pad_data_in[0] IOPADS_INST/Ptdspip00
Bump_21_0_2 Assigned port_pad_data_in[1] IOPADS_INST/Ptdspip01
Bump_8_7_0 Assigned tdigit[4] IOPADS_INST/Ptdigop4
Bump_18_7_1 Assigned tdigit[5] IOPADS_INST/Ptdigop5
Bump_45_4_4 Assigned tdigit[7] IOPADS_INST/Ptdigop7
Bump_40_9_3 Unassigned
Bump_38_7_3 Unassigned
Bump_49_8_4 Unassigned
Bump_87_6_8 Unassigned
Bump_56_5_5 Unassigned
Bump_53_2_5 Assigned port_pad_data_in[2] IOPADS_INST/Ptdspip02
Bump_20_9_1 Assigned tdigit_flag IOPADS_INST/Ptdigfgop
Bump_28_7_2 Assigned port_pad_data_out[0] IOPADS_INST/Ptdspop00
Bump_15_4_1 Unassigned
Bump_9_8_0 Unassigned
Bump_60_9_5 Unassigned
Bump_58_7_5 Unassigned
Bump_69_8_6 Unassigned
Bump_54_3_5 Unassigned
Bump_65_4_6 Unassigned
Bump_59_8_5 Assigned port_pad_data_out[2] IOPADS_INST/Ptdspop02
Bump_39_8_3 Assigned port_pad_data_out[3] IOPADS_INST/Ptdspop03
Bump_70_9_6 Assigned port_pad_data_out[1] IOPADS_INST/Ptdspop01
Bump_77_6_7 Unassigned
Bump_50_9_4 Unassigned
Bump_68_7_6 Unassigned
Bump_36_5_3 Unassigned
Bump_80_9_7 Unassigned
Bump_90_9_8 Unassigned
Bumps with signal assigned: 57 (with bump_connect_target: 0);
Bumps with power assigned: 0 (with bump_connect_target: 0);
Bumps with ground assigned: 0 (with bump_connect_target: 0);
Bumps unassigned: 33;
Total bumps: 90;
Signal/PG ratio: NA;
Min center-to-center bump pitch: 100.000
*** End the statistics information for bumps ***


*** Starting Check Bump Overlap ...
*** End Check Bump Overlap ***
Bumps overlap: 0
```

- The following command checks whether bumps are on bump grids:

  ```
  checkBump -onBumpGrid -outfile bump.rpt
  ```

  When `-onBumpGrid` is specified, other checks are not started. Therefore, the *Bump On Bump Grid* section is the only section in the report file:

```
#...header...#
*** Starting Bump On Bump Grid ...
'Bump_295’ is off bump grid “Grid_A”.
'Bump_296’ is off bump grid “Grid_B”.
'Bump_297’ needs to be checked on multiple conflicted bump grids: “Grid_A” on “Layer10” and “Grid_B” on
“Layer11”.
‘Bump_298’ won’t be checked. No bump grid on the bump layer: Layer8, Layer9.
*** End Check Bump On Bump Grid ***
Bumps off Bump Grid: 2
Bumps on Multiple Conflicted Bump Grids: 1
Bumps with No Bump Grid:1
```

Related Information

- assignBump
- create_bump
- create_bump_grid

ciopLoadBumpColorMapFile

```
ciopLoadBumpColorMapFile
[-help]
bumpColorFile
```

Loads the bump color map file to change the color of assigned bumps. The command supports all the colors specified in the Qt color table. A list of recognized color names can be found at `http://www.w3.org/TR/SVG/types.html#ColorKeywords`.

The bump color map file supports wildcards for ease of use. For example, instead of specifying `tdigit[1] green`, `tdigit[2] green` and so on, you can specify `tdigit* green` in the bump color map file to color all `tdigit` bumps green.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `ciopLoadBumpColorMapFile` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man ciopLoadBumpColorMapFile`
`bumpColorFile`	Specifies the name of the bump color map file to load.

Example

- The following command loads the bump color map file `BumpColor.Map` and assigns colors to bumps in the design:

  ```
  ciopLoadBumpColorMapFile BumpColor.Map
  ```

  Output:

  ```
  port_pad_data_out[8] wheat
  tdigit* green
  ```

Related Information

- `assignBump`

copy_bump

```
copy_bump
[-help]
[-assigned]
[-name_format {format}]
-shift {X Y}
{-bumps {bump_list} | -selected }
```

Copies bumps to the specified location with the same pitch constraint. By default, the target bumps after copy are unassigned and named with the default format `Bump_%i` by the tool. Here, `%i` refers to the index of the bump, which starts at one more than the number of existing bumps in the design. Use the option `-name_format` to name the copied bumps. If there are any name collisions, an error is given and the bumps are not copied.

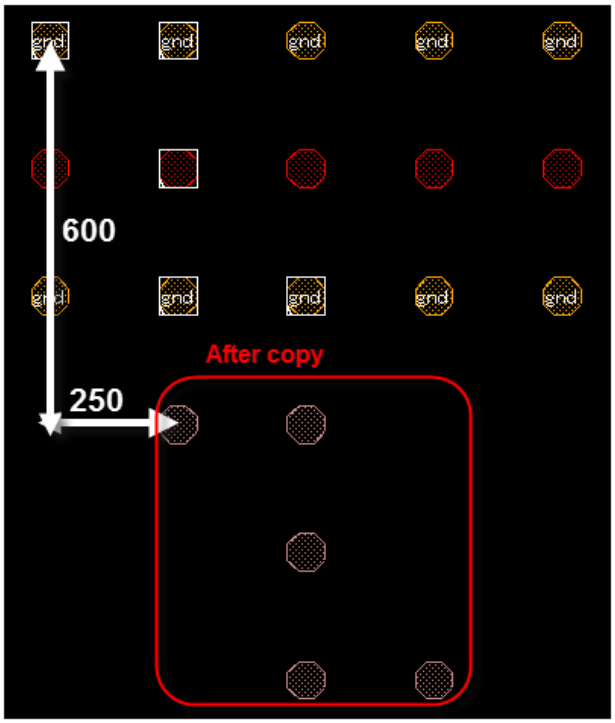
Parameters

`-help`	Prints a brief description that includes type and default information for each `copy_bump` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man copy_bump`
`-assigned`	Copies the bumps and their assigned signals.
`-bumps {`*`bump_list`*`}`	Specifies names of the bumps to be copied. You can use wildcards in the bump list.
`-name_format {`*`format`*`}`	
	Specifies the name format for the target bumps. The format is similar to the `printf` format, except that it uses `%i` for index and `%o` for original name of source bump. `%i` mean the index of bump, which starts at one more than the number of existing bumps in the design.
`-selected`	Copies only the selected bumps.
`-shift {X Y}`	Specifies the distance in microns from the source location to the target location in X/Y direction

Examples

- The following command copies the selected bumps, `Bump_1` to `Bump_5`, and pastes them with a `{250 -600}` shift. The copied bumps are named `Bump_1_copy`, `Bump_2_copy`, and so on.

  ```
  copy_bump -selected -shift {250 -600}  -name_format {%o_copy}
  ```



- Assuming that the number of bumps in the design is `15`, the following command copies the selected bumps, `Bump_1` to `Bump_5`, and pastes them with a `{250 -600}` shift. The copied bumps are named `Bump_1_copy_16`, `Bump_2_copy_17`, ..., and `Bump_5_copy_20`:

  ```
  copy_bump -selected -shift {250 -600} -name_format {%o_copy_%i}
  ```

- Assuming that the number of bumps in the design is `15`, the following command copies the selected bumps, `Bump_1` to `Bump_5`, and pastes them with a `{250 -600}` shift. The copied bumps are named `Bump_16`, `Bump_17`, ..., and `Bump_20`, according to the default name format:

  ```
  copy_bump -selected -shift {250 -600}
  ```

Related Information

- `create_bump`
- `edit_bump_name`
- `select_bump`
- `deselect_bump`
- `deleteBumps`
- Flip Chip Methodologies chapter in the *User Guide*

create_bump

```
create_bump
[-help]
[-allow_outside_of_die]
[-allow_overlap_control {keep_all keep_existing_bumps keep_new_bumps}]
-cell bumpcell_name
[-check_drc]
[-edge_spacing {left bottom right top}]
[-exclude_bump_area {rect_list}]
[ [-keep_out_region width height] | [-keep_out_pitch value] ]
[-loc x y]
[-loc_type {cell_center | cell_lowerleft | geometry_center | geometry_lowerleft}]
[-name_format {string, %i, %r, %c}]
[-orientation {R0 R90 R180 R270 MX MX90 MY MY90}]
[-respect_placement_blockage]
[-return_bumps_name]
[-start_index index]
[-pitch x y [-pattern_full_chip | -pattern_side {side width} | -pattern_array {row column} | -pattern_ring width | -
pattern_center {row column}]]
[-stagger_type type [-stagger_offset value]]
[-relative_type {embedded_bump inst_pin_port block}
  [[-relative_object {inst_pin_port_list}]
   [-relative_block inst+] [-relative_block_constraint fileName]
   [-relative_assignment] [-relative_offset x y]]]
[{-pattern_full_chip  | -pattern_array {row column} | -pattern_center {row column}} [-start_from {lower_left
upper_left lower_right upper_right}]]
```

Creates bumps and instances of bump cells based on the specified pattern. Use one of the following parameters to specify the bump pattern in the chip:

- `-pattern_full_chip`
- `-pattern_side {side width}`
- `-pattern_array {row column}`
- `-pattern_ring width`
- `-pattern_center {row column}`

If none of the above patterns is specified, `create_bump` creates a single bump at the specified location.

`create_bump` also enables you to create bumps based on pin location.

Note - During bump placement trials, if you click the *Undo* button (or use the `undo` command) after running `create_bump`, the bump floorplan will return to the state it was in before `create_bump`. If you then click the *Redo* button (or use the `redo` command), the bump floorplan will reapply the changes made by `create_bump`.

Embedded Bumps

Bumps are usually placed in the top design. However, some blocks may also include bumps. Such bumps are called embedded bumps. When implementing the block design, use the following special keywords in the LEF definition for embedded bumps.

- Add following property keywords in the head of tech LEF file

```
PROPERTYDEFINITION
    LAYER LEF58_TYPE STRING ;
END PROPERTYDEFINITIONS
```

- Add PASSIV LAYER definition after AP(top) layer

```
LAYER PASSIV
  TYPE CUT ;
  PROPERTY LEF58_TYPE "TYPE PASSIVATION ; " ;
END PASSIV
```

- Add a same geometry on PASSIV layers with the bump pins

```
MACRO BUMP
CLASS COVER BUMP ;
  …
    PIN A
  …
      PORT
        LAYER PASSIV ;
        POLYGON 27.985 0 67.565 0 95.55 27.985 95.55 67.565 67.565 95.55 27.985 95.55 0 67.565 0 27.985 ;
        LAYER AP ;
        POLYGON 27.985 0 67.565 0 95.55 27.985 95.55 67.565 67.565 95.55 27.985 95.55 0 67.565 0 27.985 ;
      END
    END A
  END BUMP
```

After the block design is finished, the bumps are dumped out as one port of the pin with PASSIV layer and top layer (AP) as similar as below, tool could check if the port is the embedded bump by PASSIV layer from the top design.

```
PIN DQ0
    DIRECTION INOUT ;
    USE SIGNAL ;
    PORT
      LAYER AP ;
      POLYGON 759.9810 366.7830 799.5610 366.7830
              827.5460 338.7980 827.5460 299.2180
              799.5610 271.2330 759.9810 271.2330
              731.9960 299.2180 731.9960 338.7980 ;
      LAYER PASSIV ;
      POLYGON 759.9810 366.7830 799.5610 366.7830
              827.5460 338.7980 827.5460 299.2180
              799.5610 271.2330 759.9810 271.2330
              731.9960 299.2180 731.9960 338.7980 ;
END
    PORT
…
END
…
END DQ0
```

If you are creating an embedded bump, `create_bump` automatically creates a bump in the top design with the correct net assignment at the same location as the embedded bump.

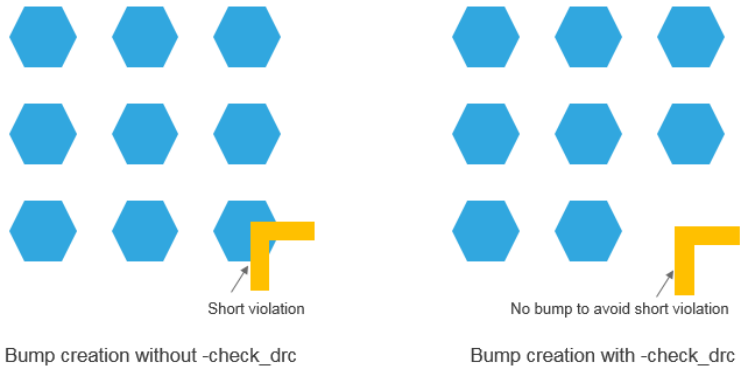
Dummy Bumps

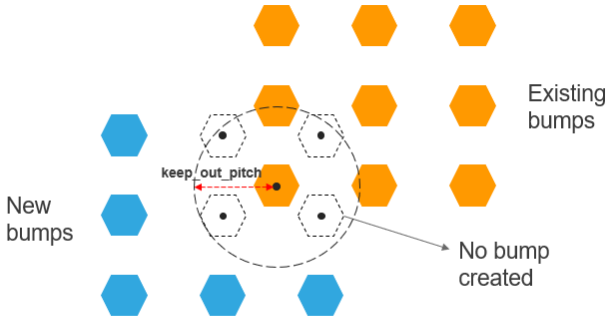
You can also use `create_bump` to create mechanical dummy bump cells without pin shapes. For details, see the Dummy Bump Creation section in the *Innovus User Guide*.

Parameters

`-help`	Prints a brief description that includes type and default information for each `create_bump` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man create_bump`
`-allow_outside_of_die`	Allows bumps to be created outside of die area. When this option is specified, the tool ignores floorplan constraints when creating a bump array pattern. This option can be used only if `-pattern_array` and `-location` are also specified. *Data_type*: bool, optional
`-allow_overlap_control {keep_all keep_existing_bumps keep_new_bumps}`	

	Specifies the overlap control type. The allowed values for the `-allow_overlap_control` parameter are: • `keep_all` - Keeps all overlapping bumps. • `keep_existing_bumps` - Keeps the existing bumps before calling `create_bump` if there is an overlap. If this option is specified, `create_bump` skips creating a bump if there is an overlap and continues creating the remaining bumps. A warning message is displayed when bump creation is skipped to avoid overlap. • `keep_new_bumps` - Removes the existing bumps before calling `create_bump`, places the newly created bump if there is overlap, and then continues creating the remaining bumps. A warning message is displayed when deleting the existing bumps to avoid overlap. If the status of the existing bumps to be deleted is covered or fixed, a warning is displayed and the new bumps overlapping with the covered or fixed old bumps are not kept. ⚠ `create_bump`: • Checks overlaps between regular bumps by checking for pin overlaps on bumps. • Checks overlaps between dummy bumps by checking for OBS overlap. • Checks overlaps between dummy bumps and regular bumps by checking for OBS and bump pin overlap.

`-check_drc`	Checks DRC rules and does not create a bump at any location where there is a violation. A suitable warning message is also issued. For example, as indicated in the following diagram, no bump is created at the location where a short violation is detected when `-check_drc` is specified with `create_bump`.  In addition, a warning such as the following is displayed: `**WARN: (IMPSIP-9068): New bump at (0.000, 75.000) has short violation and skipped.` Currently, `create_bump -check_drc` checks for the following types of violations: • Short violation • Min spacing violation • Cut spacing violation • EOL (end of line) violation
`-cell` *`bumpcell_name`*	Specifies the name of the bump cell, which is used to create bumps.
`-edge_spacing {`*`left bottom right top`*`}`	
	Specifies the minimum distances in microns to the edge of the chip from the lower left or center of the outermost bumps. The `-loc_type` value determines which point of the bump (lower-left or center of the cell or geometry) is used to compute the distance to the edge. The arrays start at the left, bottom location and extend to the right and top.
`-exclude_bump_area {`*`rect_list`*`}`	
	Specifies the area to be excluded from bump placement. No bump geometry is allowed inside the specified area. It supports multiple rectangles, such as `-exclude_bump_area {`*`llx1 lly1 urx1 ury1 llx2 lly2 urx2 ury2 ...`*`}`.

`-keep_out_pitch value`	Defines a circular keep-out area with the bump center as the center and the specified pitch value as the radius. By default, the tool will not create a bump at a location if an existing bump violates the center-to-center pitch at that location, as shown below.  Expected bump creation If the bump-center-to-bump-center pitch at a location is smaller than the specified keep-out pitch value, the tool displays a warning. Whether the tool then retains the existing bumps in the overlap region or replaces them with the new bumps depends on the `-allow_overlap_control` parameter setting: • If `-keep_out_pitch` is specified but `-allow_overlap_control` is not specified, the tool keeps the existing bumps and does not add new bumps in the overlap region. This is the same behavior as when `-keep_out_pitch` is specified along with `-allow_overlap_control keep_existing_bumps`. • If `-keep_out_pitch` is specified along with `-allow_overlap_control keep_new_bumps`, the tool keeps the newly added bumps and deletes the existing bumps. If the bump-center-to-bump-center pitch equals the specified keep-out pitch value, the tool creates bumps without any violations.
`-keep_out_region width height`	
	Defines a dynamic, keep-out region where bumps are not going to be created if there are existing bumps in that region. The keep-out region is a box area of the specified width and height, and its center is the same as the bump center. By default, if you specify `-keep_out_region` and there are existing bumps in the region, the tool displays a warning and keeps the existing bumps in the overlap area. Specify `-allow_overlap_control keep_new_bumps` if you want to keep the new bumps and delete existing ones in the case of overlap in the specified keep-out region.

`-loc x y`	Specifies the *x* and *y* coordinates of the start point for bump creation based on location type. The unit is micron. For different bump patterns, the `-loc` coordinates have different meanings for the start point: • For `-pattern_full_chip`, `-pattern_side {bottom width}`, `-pattern_side {left width}`, `-pattern_array {row column}`, `-pattern_ring width`, the start point is the location of the lower-left bump in the bump pattern. • For `-pattern_side {top width}`, the start point is the location of the upper-left bump in the bump pattern. • For `-pattern_side {right width}`, the start point is the location of the lower-right bump in the bump pattern. • For `-pattern_center {row column}`, the start point is the center of the bump pattern. If the location violates the spacing rule defined by the option `-edge_spacing`, the tool issues a warning but still uses the `-loc` value for the start point.
`-loc_type {cell_center | cell_lowerleft | geometry_center | geometry_lowerleft}`	
	Specifies the location type for bump creation. The default type is `cell_center`.
`-name_format {string, %i, %r, %c}`	
	Specifies the format for the bump name like a `printf` format, except that it uses `%i` for index, `%r` for row number, and `%c` for column number. The index starts at one more than the number of existing bumps in the design, unless `-start_index` is specified and increments for each bump created. This means if there are no bumps in the design, the index starts at `1` and if there are 100 bumps in the design, the index starts at 101. The default name format is `Bump_%i`. Therefore, if there are no bumps in the design, the tool generates bumps named `Bump_1`, `Bump_2`, and so on, by default. Some examples of the possible format are: • `-name_format prefix_%i_string_%r_%c_suffix` • `-name_format %i` • `-name_format bumpname` If there are any name collisions, an ERROR occurs and the bump pattern is not created.
`-orientation {R0 R90 R180 R270 MX MX90 MY MY90}`	

	Specifies the orientation of the bump for bump generation. Use this option if you are creating a pillar bump. A pillar bump can be rectangular and you may rotate the pillar bump by 90/180/270 degrees to alleviate package and chip routing problem. *Default:* `R0` **Note**: If you are creating a regular C4 bump, which is either octagonal or square in shape, rotation does not change its footprint in silicon.
`-pattern_array {`*`row column`*`}`	
	Creates an array of bumps. The bump starts at the coordinates specified with `-loc`, and extends to the number of *`row`* and *`column`* values given from the start corner specified using `-start_from`.
`-pattern_center {`*`row column`*`}`	
	Creates an array of bumps that is centered on the center of the chip and that extends for the *`row`* and *`column`* values given. The center point can optionally be set directly with `-loc`. The starting corner of the array can be specified using the `-start_from` parameter.
`-pattern_full_chip`	Creates an array of bumps covering the chip. The array starts at the closest to boundary point set by `-edge_spacing` at the corner specified by `-start_from` and extends until within `-edge_spacing` of the chip boundary for the opposite sides. You can optionally directly specify the `-loc` value for the starting point, in which case `-edge_spacing` affects only the two opposite edges of the array. For examples, see the Examples section below.
`-pattern_ring` *`width`*	Creates a ring of bumps around the boundary of the chip that is *`width`* bumps wide, where *`width`* is an integer value. The ring extends until within `-edge_spacing` of the chip boundary, starting at the lower-left value allowed by `-edge_spacing`. You can optionally directly specify the `-loc` value for the lower-left bump location, in which case `-edge_spacing` only affects the right and top edges of the array. The `-pitch` *`x`* value is the spacing between bumps inside one ring, and the `-pitch y` value is the spacing between two rings. For examples, see the Examples section below.
`-pattern_side {`*`side width`*`}`	
	Creates an array of bumps along one side of the chip. The array is the length of the side, and *`width`* bumps wide, where *`width`* is an integer value. The legal *`side`* names are: `left`, `top`, `right`, `bottom`. The array starts at the left/bottom controlled by `-edge_spacing`, and extends until within `-edge_spacing` of the top/right sides. You can optionally directly specify the `-loc` value for the lower-left bump location for bottom/left sides, the upper-left bump for the top side, or lower-right for the right side. For examples, see the **Examples** section below.

`-pitch x y`	Specifies the distance between bump centers in microns. `-pitch {x y}` is interpreted in different ways for different bump patterns.
	• For `full_chip`, bottom_side row, top_side row, left_side column, right_side column, array {row column}, and center {row column}:
	  ◦ `-pitch x` means the distance between the bump centers in a row or horizontal.
	  ◦ `-pitch y` means the distance between the bump centers in column or vertical.
	• For `ring ring_number`:
	  ◦ `-pitch x` means the distance between the bump centers in the ring.
	  ◦ `-pitch y` means the distance between rings from the bump center.

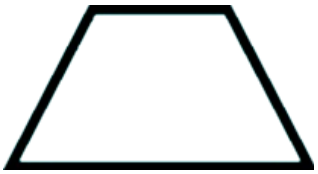
`-relative_assignment`	Assigns the corresponding net of the relative object to the bump being created. This option can be used only if `-relative_type` is specified. If `relative_assignment` is specified and `-relative_type` is: • `embedded_bump`: `create_bump` searches the top netlist for the net connected to the pin of the embedded bump and assigns it to the created bump. • `inst_pin_port`: You must also specify `-relative_object {inst_pin_port_list}`. `create_bump` then searches the top netlist for the net connected to the specified pin and assigns it to the created bump. • `block`: You must also specify `-relative_block {inst+}`. `create_bump` then searches the top netlist for the net connected to the top layer pins of the specified blocks and assigns it to the created bump. **Note**: This option is used with `-cell`, `-start_index`, `-name_format`, `-orientation`, `-allow_overlap_control`, `-relative_type`, `-relative_object` and `-relative_offset`. An error message is displayed if it is used with any other option.
`-relative_block inst+`	Specifies the block list for creating top-level bumps based on block pins on the top layer. The `-relative_block` parameter must be specified when `-relative_type` is set to `block`. The `-relative_block` parameter supports wildcards. For example, you can specify `-relative_block *`.
`-relative_block_constraint fileName`	

	Specifies the filename of the block constraint to be used for creating top-level bumps based on block pins on the top layer. The `-relative_block_constraint` parameter is optional and can be used only when `-relative_type block` is specified.
	
	`create_bump` parses the constraint file specified with `-relative_block_constraint` and creates bumps based on the instance pins specified with `-relative_block`. The top-level bump is created at the center of the block pin geometries. Note that bumps are created on a block pin only if it is on the top layer of a design.
	
	The constraint format for bumps based on a block pin is as follows:
	
	`BUMP_ON_BLOCK_PIN`
	
	`MACRO` *`marco_name_list`*
	
	`PIN [`*`pin_name_list`*`] [SINGLE|MULT]`
	
	`PORT [SINGLE|MULT]`
	
	`GEOMETRY_SHAPE RECT/POLY`
	
	`GEOMETRY_SHORT_EDGE` *`min_value`* `[:`*`max_value`*`]`
	
	`GEOMETRY_LONG_EDGE` *`min_value`* `[:`*`max_value`*`]`
	
	`NET` *`net_list_name`*
	
	`END BUMP_ON_BLOCK_PIN`
	
	Note: If a constraint file is specified, the tool creates bumps only on the block pins that are specified in the constraint file. If you do not specify a constraint file, the tool creates a single bump for each block pin on the top layer for the specified blocks. If a constraint file is specified but is empty, the tool does not create any bumps because no block pin is specified in the constraint file.
	
	To see the use model and examples of the `BUMP_ON_BLOCK_PIN` constraint, refer to the "Bump Creation" section in the Flip Chip Methodologies chapter in the *User Guide*.

`-relative_object {`*`inst_pin_port_list`*`}`	

	Specifies the inst_pin_port list and supports wild cards. The format is `{inst:pin:port ...}` and `port` is of type integer. When the `-relative_object` option is specified, bumps are created by default at the geometry center of the specified ports. The bump is named `$blockName-$pinName-Port_$portNumber-embeddedBump_$index`, where: • `$blockName` is the name of the block in the top design which has the corresponding embedded bump. • `$pinName` is the name of the pin in LEF which has the corresponding embedded bump. • `$portNumber` is the number of the port on which the created bump is based. **Example**: `Avd_test_1-DQ0-Port_1-embeddedBump_1` You can reset the default settings by using the `-start_index` and `-name_format` options. **Note**: This option is used with `-cell`, `-start_index`, `-name_format`, `-orientation`, `-allow_overlap_control`, `-relative_type`, `-relative_assignment` and `-relative_offset`. An error message is displayed if it is used with any other option.
`-relative_offset x y`	Specifies the offset of bump location compared with the relative object. The geometry center of the created bump is the geometry center of the relative object plus the offset. This option can be used only if `-relative_type` is specified. **Note**: This option is used with `-cell`, `-start_index`, `-name_format`, `-orientation`, `-allow_overlap_control`, `-relative_type`, `-relative_object` and `-relative_assignment`. An error message is displayed if it is used with any other option.
`-relative_type {embedded_bump inst_pin_port block}`	

	Specifies the type of the relative object: • `embedded_bump`: When the `embedded_bump` type is specified, `create_bump` automatically creates a bump at the same location as the embedded bump. The bump is named *`$blockName/$pinName/embeddedBump_$index`*, where: ◦ *`$blockName`* is the name of the block in the top design that has the corresponding embedded bump. ◦ *`$pinName`* is the name of the pin in LEF that has the corresponding embedded bump. You can reset the default settings by using the `-start_index` and `-name_format` options. If there are already overlapping bumps for some embedded bumps, `create_bump` skips creating bumps for those embedded bumps but continues to create bumps for remaining embedded bumps and issues the following warning message. `WARNING (#): There is(are) overlap bump(s) with the embedded bump $blockName/$pinName located at {x2 y2} with relative offset {x y}. Skip creating bump for it.` After all bumps are created, `checkBump` displays a summary of the number of embedded bumps for which the tool failed to create bumps. • `inst_pin_port`: When the `inst_pin_port` type is specified, `create_bump` creates bumps as per the *`inst_pin_port_list`* you specify with `-relative_object`. Therefore, if you specify `-relative_type` as `inst_pin_port`, you must specify the `-relative_object {`*`inst_pin_port_list`*`}` option. • `block`: When the `block` type is specified, `create_bump` automatically creates bumps at the same location as that of the specified block pin, which must be on the top routing layer of the design. The `-relative_block` parameter must be specified when `-relative_type` is set to `block`.
`-respect_placement_blockage`	Specifies that the bumps being created, including dummy bumps, should not overlap any placement blockages.
`-return_bumps_name`	Returns the names of the bumps generated. If `create_bump` fails to generate any bumps, the tool returns an empty string.
`-stagger_offset` *`value`*	Specifies the offset of bumps between rows or columns in the generated bump pattern. Its value is specified in microns. This option is used with `-stagger_type`.
`-stagger_type {odd_rows odd_columns even_rows even_columns trapezoid }`	

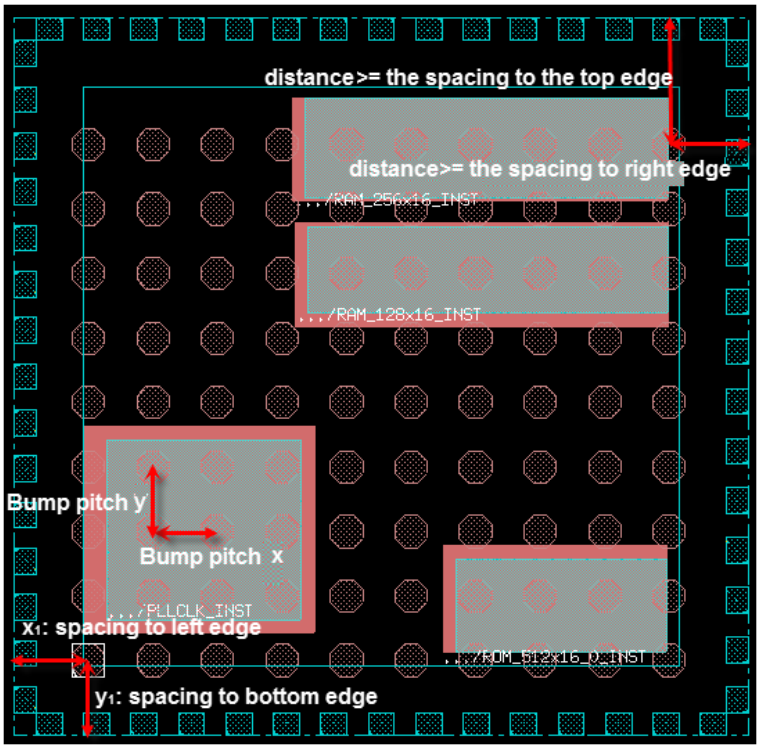
	Specifies the stagger type pattern. The `-stagger_type` parameter cannot be used with `-pattern_ring`. You can use the `-stagger_offset` parameter along with `-stagger_type` to specify the offset of bumps between rows or columns in the generated bump pattern. The allowed values for the `-stagger_type` parameter are: • `odd_rows` - Creates a bump array with the odd rows shifted to the right by the specified stagger offset value. • `odd_columns` - Creates a bump array with the odd columns shifted up by the specified stagger offset value. • `even_rows` - Creates a bump array with the even rows shifted to the right by the specified stagger offset value. • `even_columns` - Creates a bump array with the even columns shifted up by the specified stagger offset value. • `trapezoid` - Creates bumps are created in a trapezium pattern. This stagger type is supported only with `-pattern_side`.  For examples of how `-stagger_offset` affects different `-stagger_type` patterns, refer to the Examples section below.
`-start_from {lower_left upper_left lower_right upper_right}`	
	Specifies the starting corner for creating an array of bumps. This option can be used only if `-pattern_array`, `-pattern_center`, or `-pattern_full_chip` is specified. **Note**: Bumps are always indexed by row first, irrespective of the `-start_from` value. For examples of using `-start_from`, refer to the Examples section below. *Default*: `lower_left`
`-start_index` *index*	Specifies the starting number of the index for bumps to be created.

Examples

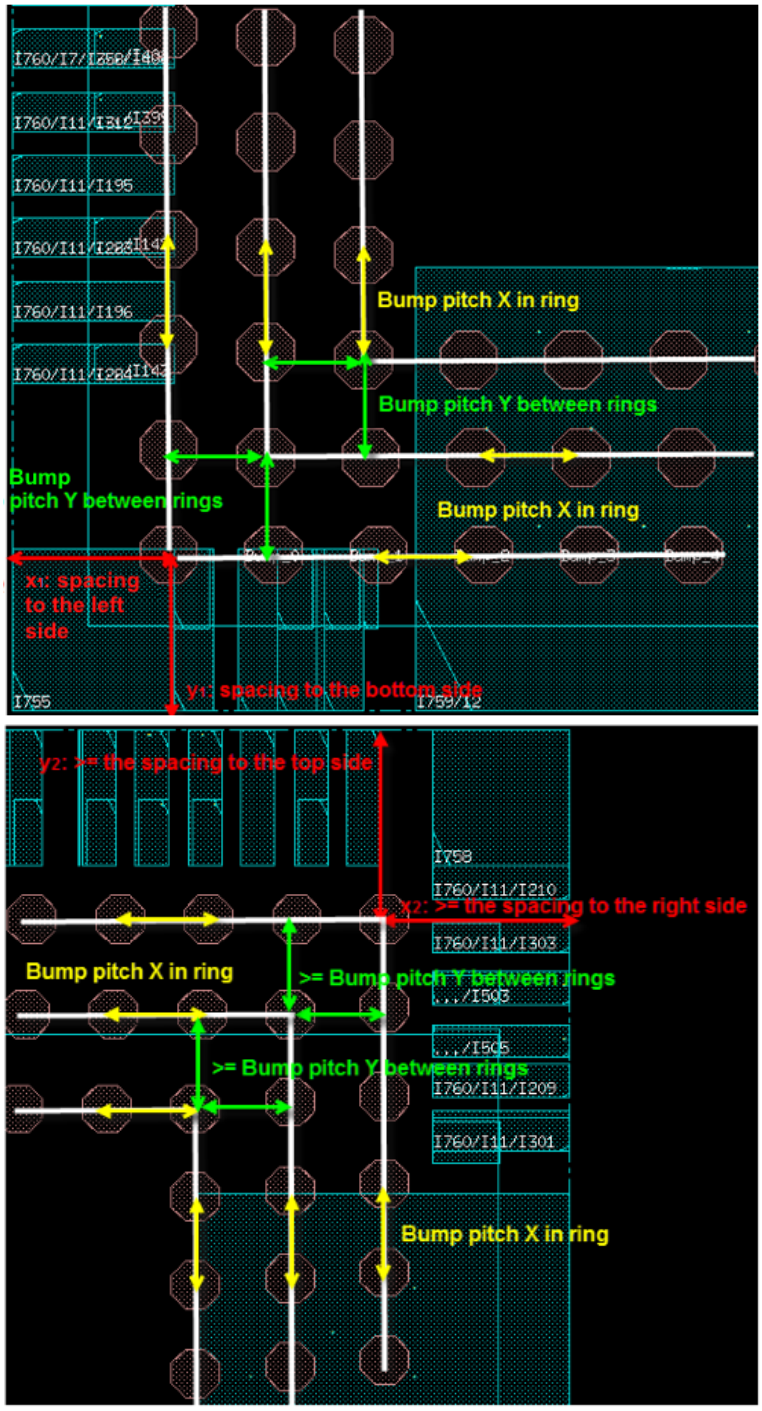
- The following command creates a single bump at the lower-left corner of the bump cell box:
  ```
  create_bump -cell BUMPCELL -loc_type cell_lowerleft -loc {0 0}
  ```
- The following command uses the default location type setting and creates an array of bumps with bump cell that has the same

size as the bump bounding box for the macro BUMPCELL.

```
create_bump -cell BUMPCELL -pitch {x y} -edge_spacing {x1 y1 x2 y2} -pattern_full_chip
```

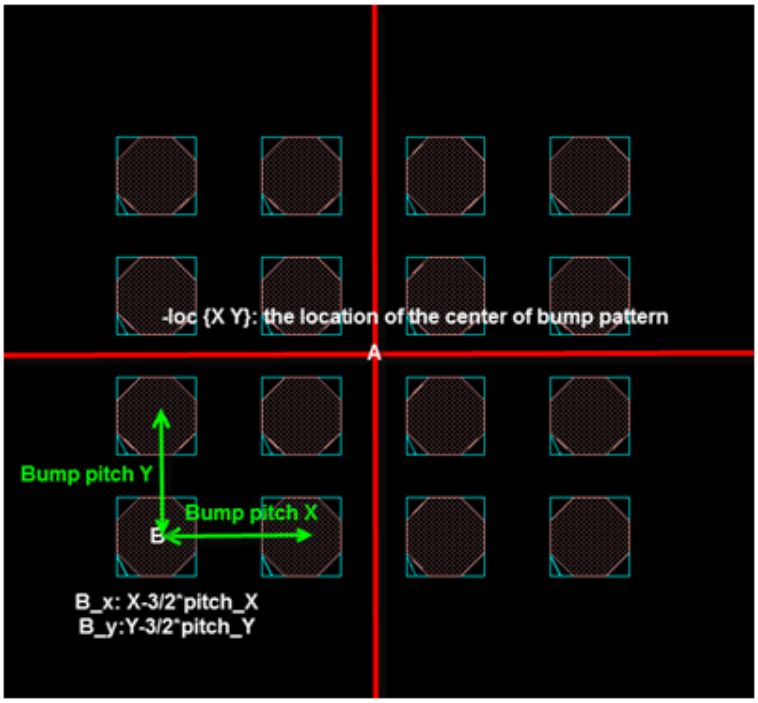


- The following command creates a full bump matrix on the chip with the spacing constraint {`400 500 450 600`} and bump pitch {`80 120`}. The names of the created bumps honor the default naming format `Bump_%i`, where `%i` means the index:
  ```
  create_bump -cell BUMPCELL -pitch {80 120} -edge_spacing {400 500 450 600} -pattern_full_chip
  ```
- The following command creates a ring of bumps around the boundary of the chip that is 3 bumps wide:
  ```
  create_bump -cell BUMPCELL -pitch {X Y} -edge_spacing {x1 y1 x2 y2} -pattern_ring 3
  ```

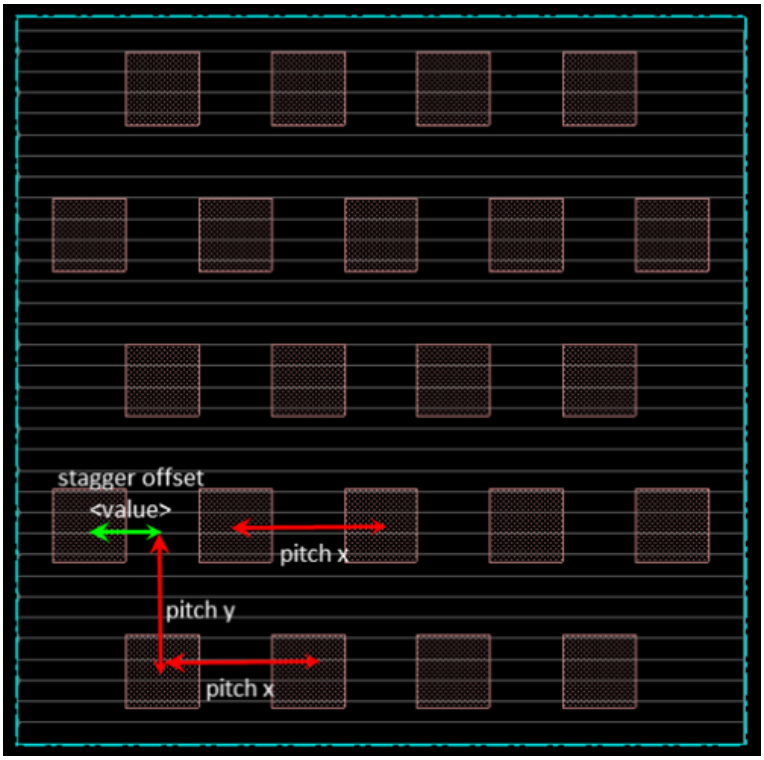


- The following command creates a 4x4 array of bumps that is centered on the point defined by `-loc`:

  ```
  create_bump -cell BUMPCELL -pitch {X Y} -loc {X Y} -pattern_center {4 4}
  ```

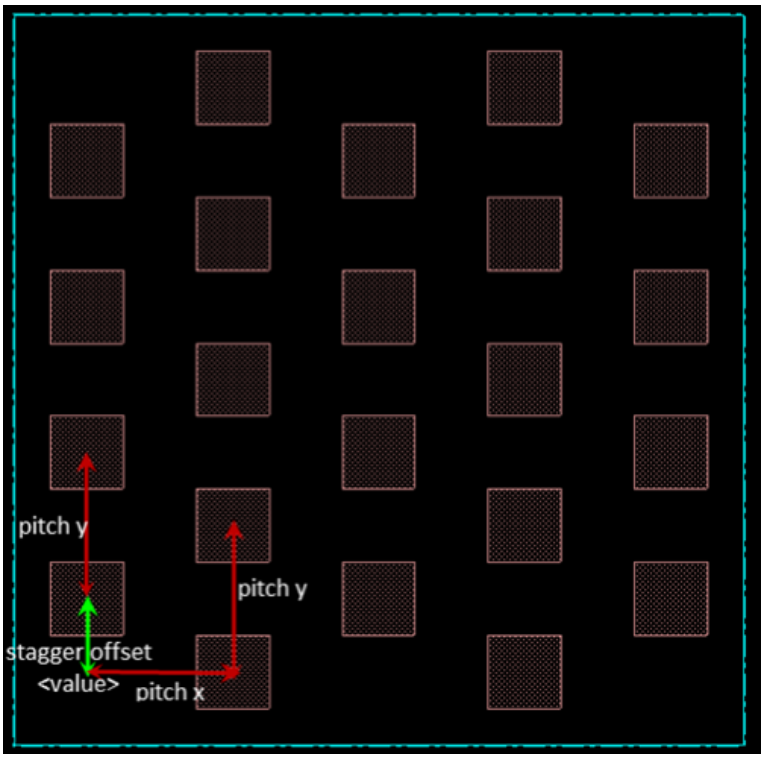


- The following example shows how `-stagger_offset` can be used to create a 5X5 array of bumps in `-stagger_type odd_rows` stagger pattern:
  `create_bump -cell BUMPCELL -loc{loc_x loc_y} -pitch {pitch_x pitch_y} -pattern_array {5 5} -stagger_type odd_rows -stagger_offset value`



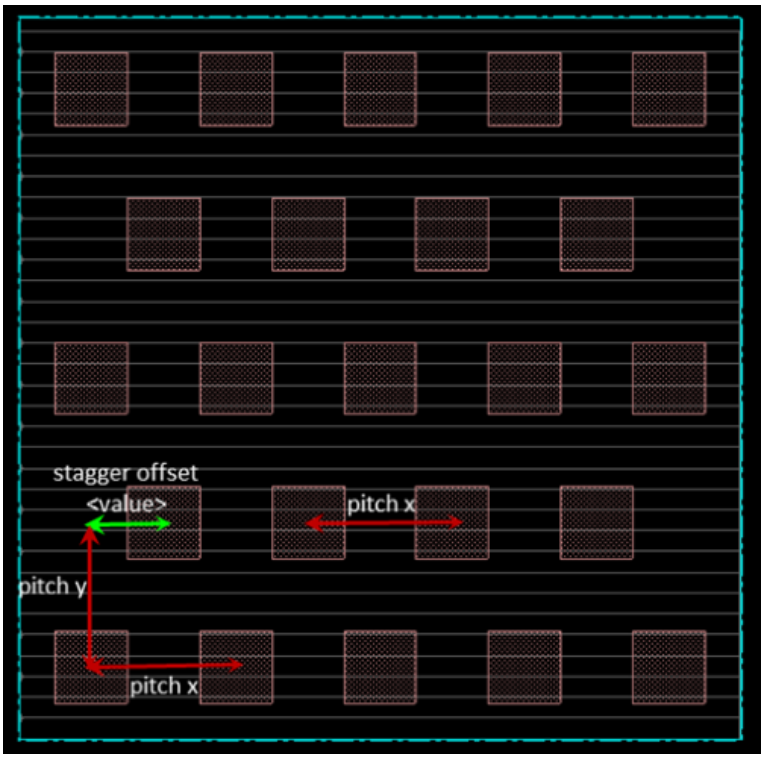
- The following example shows how `-stagger_offset` can be used to create a 5X5 array of bumps in `-stagger_type odd_columns` stagger pattern:

  `create_bump -cell BUMPCELL -loc{`*`loc_x loc_y`*`} -pitch {`*`pitch_x pitch_y`*`} -pattern_array {5 5} -stagger_type odd_columns -stagger_offset `*`value`*



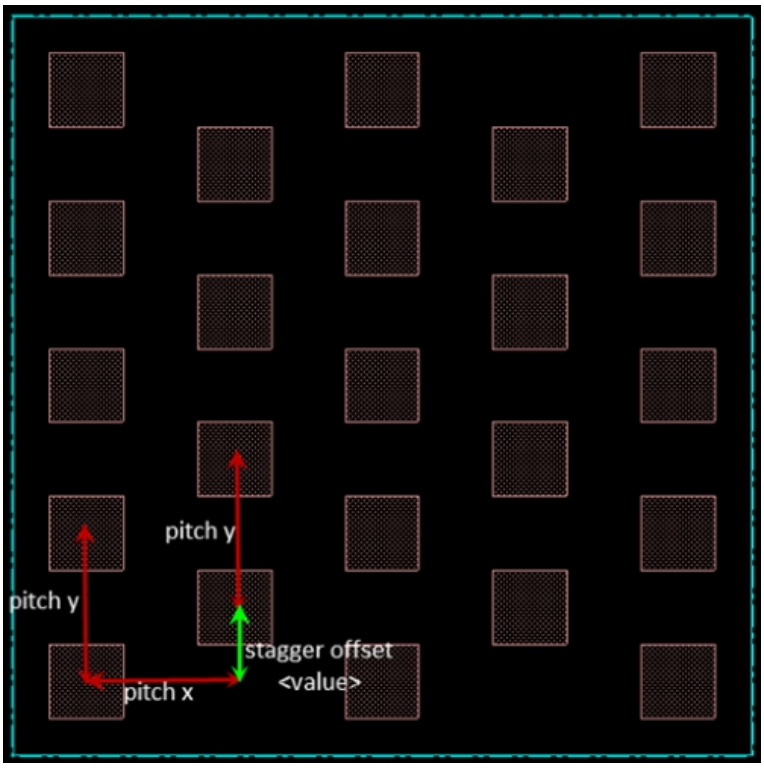
- The following example shows how `-stagger_offset` can be used to create a 5X5 array of bumps in `-stagger_type even_rows` stagger pattern:

  `create_bump -cell BUMPCELL -loc{`*`loc_x loc_y`*`} -pitch {`*`pitch_x pitch_y`*`} -pattern_array {5 5} -stagger_type even_rows -stagger_offset `*`value`*

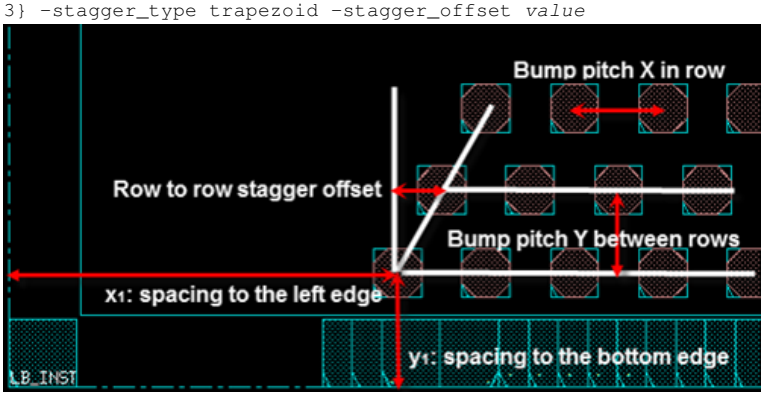


- The following example shows how `-stagger_offset` can be used to create a 5X5 array of bumps in `-stagger_type even_columns` stagger pattern:

  ```
  create_bump -cell BUMPCELL -loc{loc_x loc_y} -pitch {pitch_x pitch_y} -pattern_array {5 5} -stagger_type
  even_columns -stagger_offset value
  ```

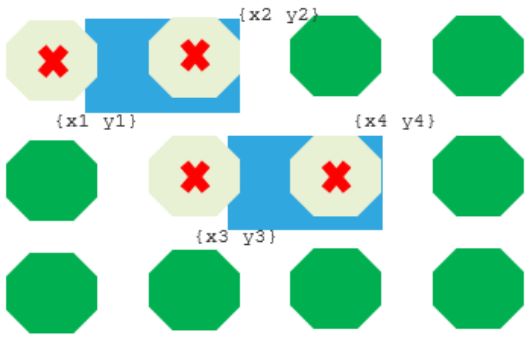


- The following example shows how `-stagger_offset` can be used with `-stagger_type trapezoid` stagger pattern to create a 3-row bump pattern on the bottom side:
  `create_bump -cell BUMPCELL -edge_spacing {left bottom right top} -pitch {pitch_x pitch_y} -pattern_side {bottom 3} -stagger_type trapezoid -stagger_offset value`



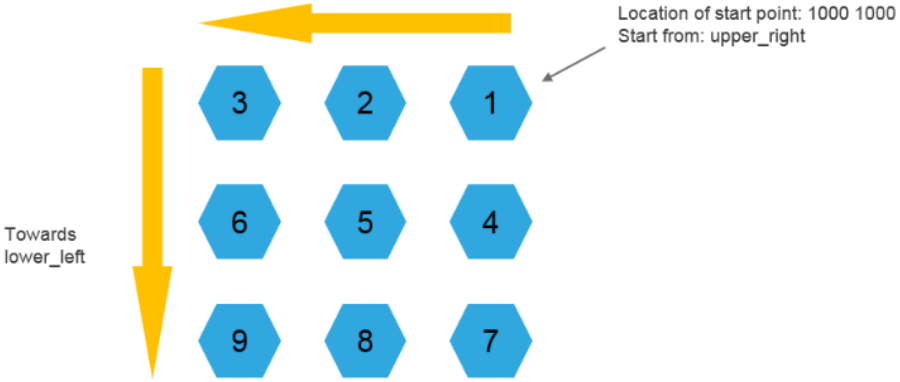
- The following command creates a 3X4 array of bumps but does not place bumps in the:
  - rectangular area with lower left corner at {x1 y1} and upper right corner at {x2 y2}
  - rectangular area with lower left corner at {x3 y3} and upper right corner at {x4 y4}

```
create_bump -cell BUMPCELL -loc {x y} -pattern_array {3 4} -exclude_bump_area {x1 y1 x2 y2 x3 y3 x4 y4}
```



- The following command creates a 3X3 array of bumps with the upper-right corner of the array starting at point (1000, 1000):

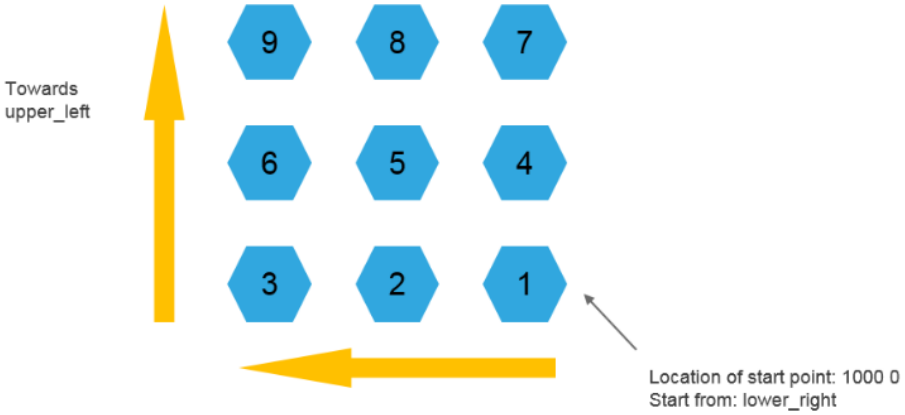
```
create_bump -pattern_array {3 3} -loc 1000 1000 -cell BUMP  -pitch 200 200 -start_from upper_right
```



The array starts from the upper right and continues towards lower left.

- The following command creates a 3X3 array of bumps with the lower-right corner of the array starting at point (1000, 0):

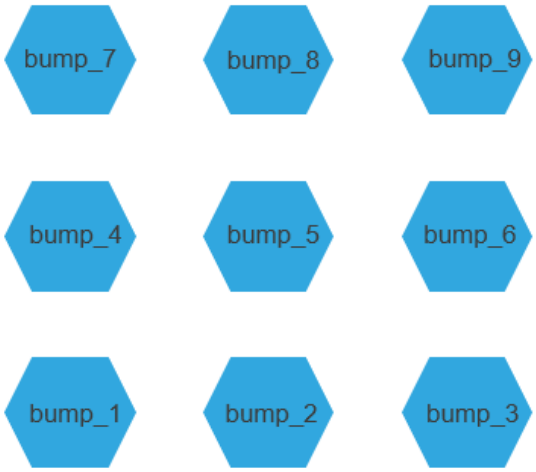
```
create_bump -pattern_array {3 3} -loc 1000 0 -cell BUMP -pitch 200 200 -start_from lower_right
```



The array starts from the lower right and continues towards upper left.

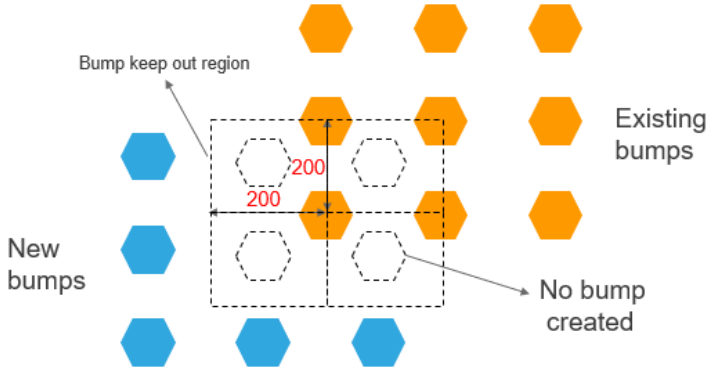
- The following command creates a 3X3 array of bumps at the specified location and returns the names of the bumps generated:

```
create_bump -pattern_array {3 3} -loc 0 1000 -cell BUMP  -pitch 200 200 -return_bumps_name
> bump_1 bump_2 bump_3 bump_4 bump_5 bump_6 bump_7 bump_8 bump_9
```



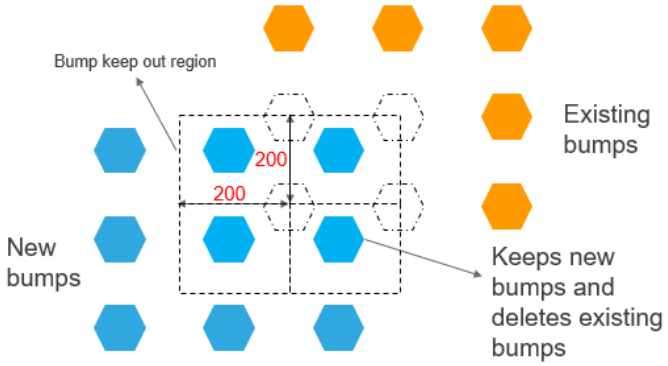
- The following command creates a 3X3 bump array and does not add new bumps in the specified keep-out region in the case of an overlap:

```
create_bump -pattern_array {3 3} -loc 200 200 -cell BUMP -pitch 200 200 \
-keep_out_region {200 200}
```



- The following command creates a 3X3 bump array and adds new bumps in the specified keep-out region in the case of an overlap:

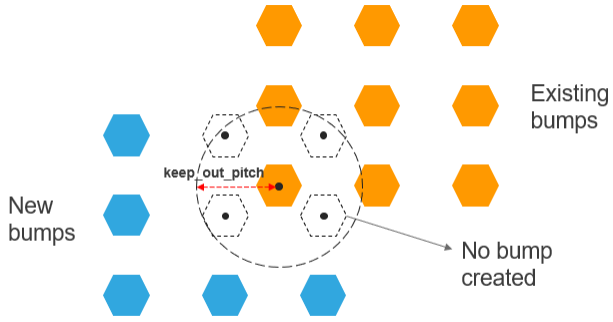
```
create_bump -pattern_array {3 3} -loc 200 200 -cell BUMP -pitch 200 200 \
-keep_out_region {200 200} -allow_overlap_control keep_new_bumps
```



- The following command creates a 5X5 bump array and allows bumps to be created outside of the die area:

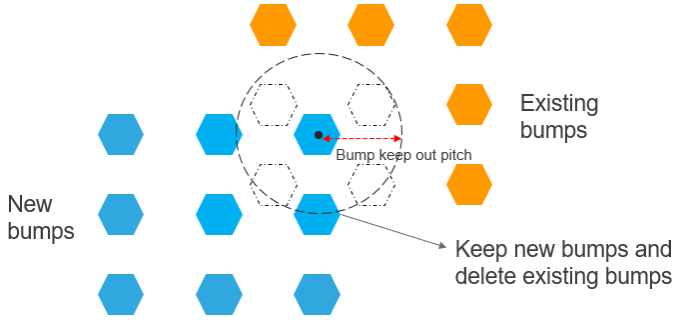
```
create_bump -cell BUMPCELL -start_from upper_right -loc 0 0 -pitch 100 100  -pattern_array {5 5} -
allow_outside_of_die
```

- The following command parses the `1.const` constraint file and creates bumps based on the specified instance pins:
  `create_bump -cell BUMPCELL -relative_type block -relative_block {`*BLOCK_LIST*`} -relative_block_constraint 1.const -relative_assignment`
- The following command creates a 3X3 bump array and does not add new bumps in the circular area defined by the specified keep-out pitch in the case of an overlap:
  `create_bump -pattern_array {3 3} -loc 200 200 -cell BUMP -pitch 200 200 -keep_out_pitch 200`



Expected bump creation

- The following command creates a 3X3 bump array and adds new bumps in the circular area defined by the specified keep-out pitch in the case of an overlap:
  ```
  create_bump -pattern_array {3 3} -loc 200 200 -cell BUMP -pitch 200 200 \
   -keep_out_pitch 200 -allow_overlap_control keep_new_bumps
  ```



Expected bump creation

Related Information

- `assignBump`
- `changeBumpMaster`
- `checkBump`
- `copy_bump`
- `deleteBumps`
- `deselect_bump`
- `redo`
- `select_bump`
- `unassignBump`

- undo
- Flip Chip Methodologies chapter in the *User Guide*
- Tools Menu chapter in the *Menu Reference*
    - *Flip Chip*

createSignalPin

```
createSignalPin
[-help]
[-ignore_floating_pins]
{-inst inst_name_list | -pins pin_name_list}
```

Promotes signal pins in the IO pad/block to the top-level design as a physical pin for specified instances or pins. The instance's cell type should be either CLASS PAD AREAIO or CLASS BLOCK. The pins to be promoted should have geometry on the top layer. You can choose to ignore floating pins.

Parameters

`-help`	Prints a brief description that includes type and default information for each `createSignalPin` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man createSignalPin`
`-ignore_floating_pins`	Ignores the floating pins. If this parameter is not specified when promoting pins to the top level, the tool converts the floating pins in the instance to pgTerms because these pins are not logically connected to any terminal on the top level.
`-inst` *`inst_name_list`*	Specifies the names of instances for which pins need to be created. **Note**: This option cannot be used with `-pins`.
`-pins` *`pin_name_list`*	Lists specific pins that are to be promoted to the top level. The specified pins should be hierarchical. **Note**: This option cannot be used with `-inst`.

Examples

- The instance `lna_inst` has a signal pin called `data1_i`. The pin `data1_i` has a port geometry shape on the top layer. The lef for the `lna` cell is as follows:

```
MACRO lna

CLASS BLOCK ;

……

  PIN data1_i

    DIRECTION INPUT ;

    USE SIGNAL ;

    PORT

        CLASS BUMP ;

        LAYER METAL8 ;

        RECT   10.00 10.00   20.00 100.00 ;

    END

  END data1_i

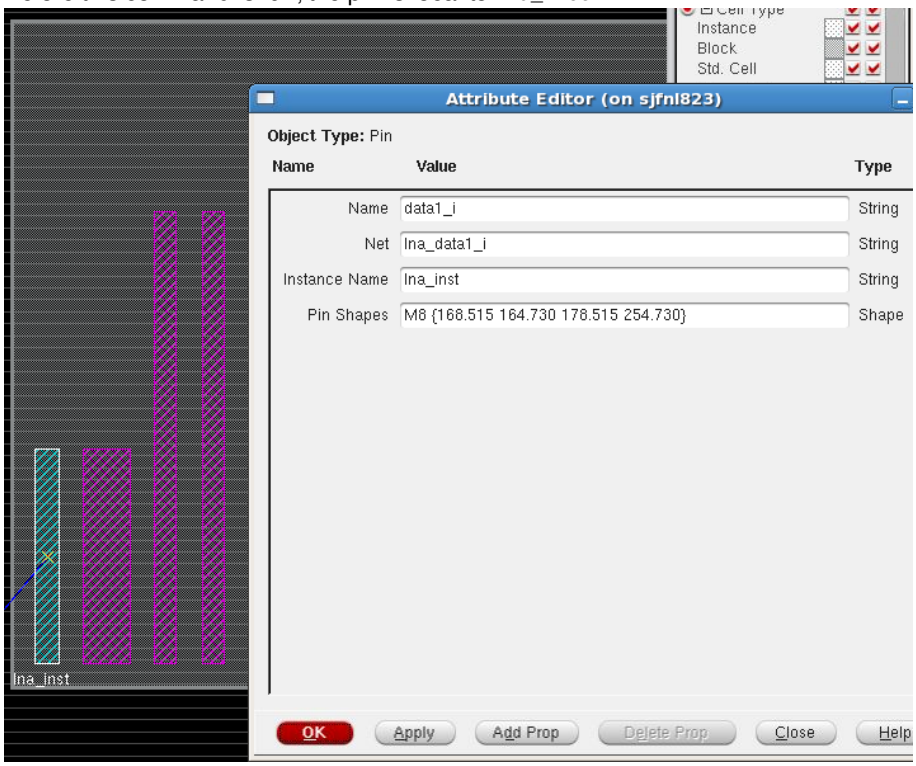
….

END lna
```

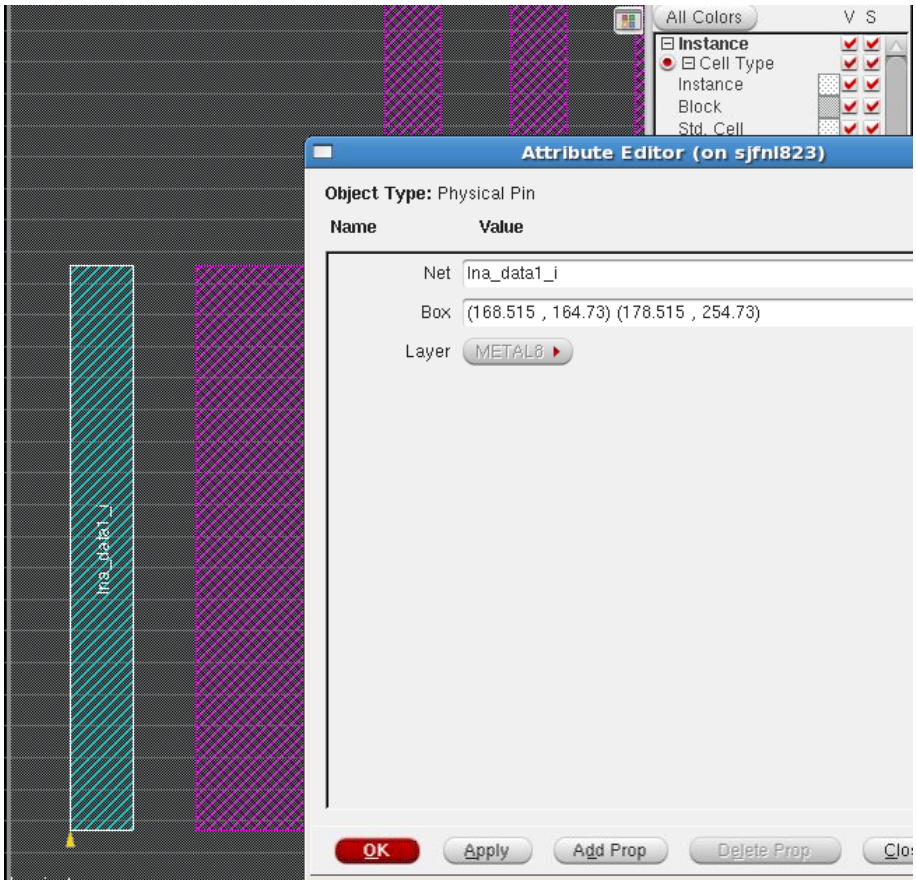
The following command creates a signal pin for the instance `lna_inst`:

```
createSignalPin -inst lna_inst
```

- Before this command is run, the pin is local to `lna_inst`:



- After the command is run, the pin is promoted:



- The following command promotes the specified pins to the top level:

```
createSignalPin -pins {lna_inst/data1_i lna_inst/data2_i phy/QS_PADP}
```

deleteAIoFiller

```
deleteAIoFiller
[-help]
-cell fillerCellName
[-prefix prefix]
-aioRowCluster aioRowClusterName | -allAIORowCluster
[-inst fillerInstanceName]
```

Deletes area I/O filler cell instances. Use this command after using the `addAIoFiller` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `deleteAIoFiller` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man deleteAIoFiller`
`-aioRowCluster` *`aioRowClusterName`*	Specifies the area I/O row cluster name where all area I/O filler cells will be deleted. **Note:** This parameter overrides `-allAIORowCluster`, i.e If you specify both, `-aioRowCluster` *`aioRowClusterName`* and `-allAIORowCluster` parameters, then only `-aioRowCluster` *`aioRowClusterName`* is valid.
`-allAIORowCluster`	Deletes all area I/O filler cells from all area I/O row clusters.
`-cell` *`fillerCellName`*	Specifies the name of the area I/O filler cell to delete.
`-inst` *`fillerInstanceName`*	Specifies the name of the area I/O filler instance name to delete.
`-prefix` *`prefix`*	Specifies the prefix name of the area I/O filler cell instance to be removed.

Example

- The following command deletes a filler cell from all area I/O row clusters:
  ```
  deleteAIoFiller -cell AO_123 -allAIORowCluster
  ```

Related Information

- `addAIoFiller`

deleteBumpConnectTargetConstraint

```
deleteBumpConnectTargetConstraint
[-help]
{-bump list_of_bumps | -selected | -all }
[-instName instance_name [-pinName pin_name [-portNum value]]]
```

Deletes existing properties on specified, selected, or all bumps. If a bump has multiple values for the property, use `-instName`, `-pinName` and `-portNum` to filter out the matched property value for deletion. If only `-instName` is specified, then all property values for the pad are deleted. If only `-instName` and `-pinName` are specified, then all property values for the pin are deleted. If the specified property value cannot be found for a bump, the tool prints an error message for the bump only and continues to process other bumps.

Parameters

`-help`	Prints a brief description that includes type and default information for each `deleteBumpConnectTargetConstraint` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man deleteBumpConnectTargetConstraint`
`-bump list_of_bumps | -selected | -all`	
	Deletes bump connect target constraint on specified, selected, or all bumps. The `-bump` option can accept wild cards and `-selected` can have multiple bumps selected.
`-instName instance_name`	Deletes the bump connect target constraint related to the specified instance.
`-pinName pin_name`	Deletes the bump connect target constraint related to the specified pin. **Note:** This option can be used only if you specify `-instName`.
`-portNum value`	Deletes the bump connect target constraint related to the specified port number. **Note:** This option can be used only if you specify `-instName` and `-pinName`.

Examples

- The following command removes `bump_connect_target` property on `Bump_1` which pairs with `VDD` PIN of instance `IOPADS_INST/esd`.

  ```
  deleteBumpConnectTargetConstraint -bump Bump_1 -instName IOPADS_INST/esd -pinName VDD
  ```

- The following command removes all `bump_connect_target` properties on `Bump_1`.

  ```
  deleteBumpConnectTargetConstraint -bump Bump_1
  ```

Related Information

- `addBumpConnectTargetConstraint`
- `editBumpConnectTargetConstraint`
- `findPinPortNumber`
- Flip Chip Methodologies chapter of the *User Guide*
  - *Port Numbering*

- Tools Menu chapter in the *Menu Reference*
  - *Flip Chip*

deleteBumps

```
deleteBumps
[-help]
[-all]
[-bumps bump+]
[-floating]
[-overlap_areaio]
[-overlap_blockages]
[-overlap_keepout bumpCellName]
[-overlap_macro]
[-selected]
```

```
Removes bumps from the design. Use this command anytime after creating a bump.
```

Deletion of assigned bumps is also supported. If any of the bumps being deleted is assigned, `deleteBumps` displays a WARNING message and deletes all specified non-routed bumps, assigned as well as unassigned.

If you click *Undo* after running `deleteBumps`, the changes made by `deleteBumps` are cancelled out and the following bump properties are recovered:

- Name
- Location
- Port number properties
- Fixed status
- Placement status

On clicking *Redo*, the bump floorplan will revert to the status before undo.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `deleteBumps` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man deleteBumps`
`-all`	Deletes all bumps.
`-bumps bump+`	Specifies the names of the bumps to be deleted. You can use wildcards in the bump name list.
`-floating`	Deletes bumps which are not assigned to any nets.
`-overlap_areaio`	Deletes bumps that overlap with the area I/O cells. You must first select the area I/O cells in the design display area, then use this command to delete the bumps.
`-overlap_blockages`	Deletes all bumps that overlap routing blockages on the same layer. For example, if a bump on layer M7 overlaps a routing blockage on layer M7, the bump is deleted when you use this parameter.
`-overlap_keepout`	Deletes specified bumps that overlap the bump keep out zone.
`-overlap_macro`	Deletes all bumps that overlap the selected macros. You must first select the macros in the design display area, then use this command to delete the bumps.
`-selected`	Deletes selected bumps. You must first select the bumps in the design display area, then use this command to delete the bumps.

Example

- The following command deletes all non-routed bumps, assigned as well as unassigned, in the current design:
  ```
  deleteBumps -all
  ```
- The following command deletes all selected bumps, whether assigned or unassigned:
  ```
  deleteBumps -selected
  ```

Related Information

- `assignBump`
- `changeBumpMaster`
- `copy_bump`
- `create_bump`
- `deselect_bump`
- `redo`
- `select_bump`
- `unassignBump`
- `undo`
- Flip Chip Methodologies chapter in the *User Guide*
- Tools Menu chapter in the *Menu Reference*
  - *Flip Chip*

deselect_bump

```
deselect_bump
[-help]
[-alternate {row column}]
[-area {{x1 y1 x2 y2} ...}]
[-assigned]
[-bump_cell {cell_list}]
[-bumps {bump_list}]
[-floating]
[-max_distance_to_side distance]
[-nets {net_list}]
[-side {top bottom left right}]
[-start_lower_left]
[-type {signal power ground}]
```

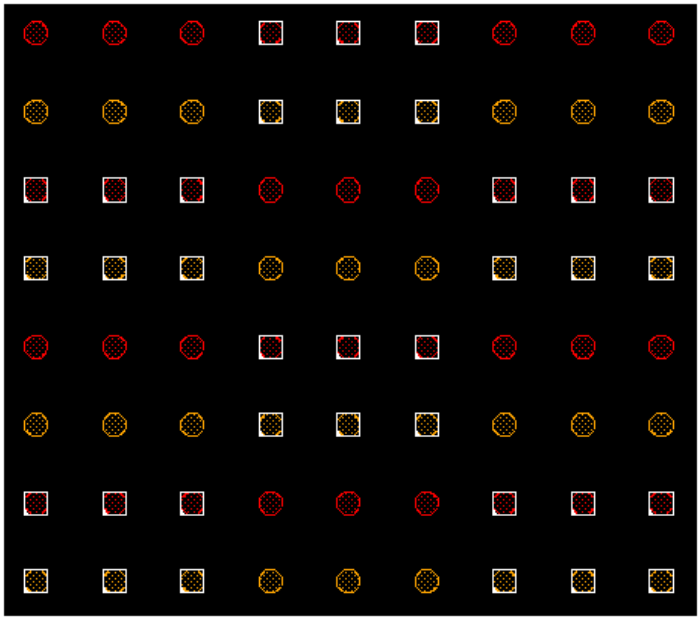
Enables you to deselect bumps in different ways. For example, you can deselect bumps by type or by net. By default, deselects all bumps in the design.

Parameters

`-help`	Prints a brief description that includes type and default information for each `deselect_bump` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man deselect_bump`
`-alternate {row column}`	Specifies the alternate bump pattern. You have to select an area to be alternate-deselected before using this option.
`-area {{x1 y1 x2 y2} ...}`	Specifies one or more areas from which bumps are to be deselected.
`-assigned`	Deselects only the bumps that are assigned.
`-bump_cell {cell_list}`	
	Specifies names of bump cells and the bumps with these bump cells are deselected. You can use wildcards in the bump cell list.
`-bumps {bump_list}`	Specifies names of the bumps to be deselected. You can use wildcards in the bump list.
`-floating`	Deselects only unassigned or floating bumps.
`-max_distance_to_side distance`	
	Specifies the maximum distance away from the side in microns. When specified, only the bumps with geometries completely in that region are deselected. By default, half the width/height of the die is used as the maximum distance to each side. For `distance`, minimum value is `0` and maximum is `1e+20`.
`-nets {net_list}`	Specifies names of nets and the related bumps are deselected. You can use wildcards in the net list.
`-side {top bottom left right}`	
	Specifies the side relative to which bumps are deselected.
`-start_lower_left`	
	Specifies the start point of alternate deselection. If it is not specified, the lower-left point is considered the start point of alternate selection. **Note**: This parameter can be used only with `-alternate`.
`-type {signal power ground}`	
	Specifies the type of bumps to be deselected.

Examples

- The following command deselects all bumps in the design:
  ```
  deselect_bump
  ```
- The following command deselects the bottom edge of bump geometry no more than 500 microns away from the top side:
  ```
  deselect_bump -side top -max_distance_to_side  500
  ```
- After selecting all the bumps in a design or in a specific area, use the following command to alternately deselect a `2x3` bump pattern:
  ```
  deselect_bump -alternate {2 3}
  ```
  The resulting deselection pattern is shown below. Note that as `-start_lower_left` is not specified, the bumps in the the lower-left corner are not deselected (that is, they are alternate selected).



Related Information

- `select_bump`

edit_bump_name

```
edit_bump_name
[-help]
-from current_pattern
[-selected]
-to new_pattern
```

Modifies the names of selected or specified bumps according to the specified pattern.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `edit_bump_name` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man edit_bump_name`
`-from` *current_pattern*	
	Specifies the name of a single bump or a pattern for the names of the bumps to be edited. The pattern is a regular expression, such as `{Bump_\d+}` or `{Bump_(\d)(\w+)}`, which can be referred in TCL. When specifying a pattern, the opening and closing braces are required.
`-selected`	Modifies the names of selected bumps.
`-to` *new_pattern*	
	Specifies the new name or name format for target bumps. When specifying a name format, the opening and closing braces are required.

Examples

- The following command modifies the names of all bumps that have the pattern `Bump_\d+`, such as `Bump_1` and `Bump_12`, to bumps with pattern `signal_\d+`, such as `signal_1` and `signal_12`:
  ```
  edit_bump_name -from {Bump_} -to {signal_}
  ```
- The following command modifies the name of the `Bump_1_1_1` bump to `signal_bump`:
  ```
  edit_bump_name -from {Bump_1_1_1} -to {signal_bump}
  ```
- The following command modifies the names of all bumps that have the pattern `Bump_(\d)_(\d)_(\d)` to bumps named `1_1_1_signal`, `1_1_2_signal`, and so on:
  ```
  edit_bump_name -from {Bump_(\d)_(\d)_(\d)} -to {\1_\2_\3_signal}
  ```

Related Information

- changeBumpMaster
- copy_bump
- create_bump
- deleteBumps
- deselect_bump
- select_bump

editBumpConnectTargetConstraint

```
editBumpConnectTargetConstraint
[-help]
{-bump list_of_bumps | -selected }
{-instName instance_name
  {[-toInstName instance_name]
   [-pinName pin_name {[-toPinName pin_name]
   [{-portNum value -toPortNum value}]}]}}
```

Modifies the property values on bumps. You can change the instance name, pin name, and port name on specified or selected bumps.

Parameters

`-help`	Prints a brief description that includes type and default information for each `editBumpConnectTargetConstraint` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man editBumpConnectTargetConstraint`
`-bump list_of_bumps | -selected`	Edits the bump connect target constraint on specified or selected bumps. The `-bump` option accepts wild cards and `-selected` can have multiple bumps selected.
`-instName instance_name`	Specifies the name of the instance for which you want to change the bump connect target constraint.
`-pinName pin_name`	Specifies the name of the pin for which you want to change the bump connect target constraint.
`-portNum value`	Specifies the port number for which want to change the bump connect target constraint.
`-toInstName instance_name`	Specifies the instance name to change to in the bump connect target constraint.
`-toPinName pin_name`	Specifies the pin name to change to in the bump connect target constraint.
`-toPortNum value`	Specifies the port number to change to in bump connect target constraint.

Example

- The following command modifies `bump_connect_target` property on `Bump_1` to change pairing between `Bump_1` and instance `IOPADS_INST/esd` from `PORT1` to `PORT2`. `PORT1` and `PORT2` are under `VDD` PIN of instance `IOPADS_INST/esd`.

  ```
  editBumpConnectTargetConstraint -bump Bump_1 -instName IOPADS_INST/esd -pinName VDD -portNum 1 -toPortNum 2
  ```

Related Information

- `addBumpConnectTargetConstraint`
- `deleteBumpConnectTargetConstraint`
- `findPinPortNumber`
- Flip Chip Methodologies chapter of the *User Guide*
  - *Port Numbering*
- Tools Menu chapter in the *Menu Reference*
  - *Flip Chip*

fcroute

```
fcroute
[-help]
-type {power | signal}
[-area {x1 y1 x2 y2}]
[-connectInsideArea]
[-connectTsvToBump] | [-connectTsvToPad] | [-connectTsvToRingStripe]
[-constraintFile filename]
[-deleteExistingRoutes]
[-designStyle {aio | pio}]
[-doubleBendRoute]
[-eco | -incremental]
[-extraConfig fileName]
[-globalOnly]
[-jogControl {preferWithChanges | preferSameLayer | preferDifferentLayer}]
[-keepDRC]
[-layerChangeBotLayer layerName]
[-layerChangeTopLayer layerName]
[-minEscapeDistance unit]
[-msgRate int]
[-nets {net_name_list | <filename | ~<filename} | -selected_bump]
[-overflowMap]
[-route_pg_style {none finger mesh}]
[-routeWidth real]
[-spreadWiresFactor value]
[-straightConnections [[straightWithDrcClean] [straightWithChanges]]
[-subclass subclass_string]
[-verbose]
```

Specifies that power routing and signal routing recognize the bumps specified in a flip chip design.

- The command supports all the `LEF 5.7` syntax to determine which power/ground pin shape on the I/O driver cell must be connected to a bump. For more information, see the "Performing Area I/O Placement" section in the Data Preparation chapter of the *Innovus User Guide*.
- The command supports mixed mode designs in the area I/O (AIO) and peripheral I/O (PIO) mode.
- The command, by default, reads the FE database to get all information. The default value of the variable, `set rda_global` (`srouteUseFEDB`) must be set to `1`. For the command to read information from the LEF file, the `set rda_global` (`srouteUseFEDB`) variable must be set to `0`.

Use this command after importing the design.

You can perform signal routing (`-type signal`) using the AIO or PIO routing styles. Each routing style supports a specific set of `fcroute` command parameters. The following table lists the routing style support for each parameter.

Note: The following table might not be readable when viewed with the Unix `man` command. Check the `fcroute` command description in the *Innovus Text Command Reference* to view the table.

fcroute Parameters	**-designStyle**	
	aio	**pio**
`-area {x1 y1 x2 y2}`	X	X1
`-connectInsideArea`	X	
`-connectTsvToBump`	X	

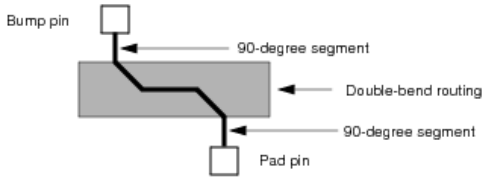
`-connectTsvToPad`	X	
`-connectTsvToRingStripe`	NA	NA
`-constraintFile filename`	X	X
`-deleteExistingRoutes`	X	X
`-doubleBendRoute`	X	X
`-eco`	X	X
`-extraConfig` *fileName*	X	X
`-globalOnly`		X
`-incremental`		X
`-jogControl {preferWithChanges | preferSameLayer | preferDifferentLayer}`	X	
`-keepDRC`	X	X
`-layerChangeBotLayer` *layerName*	X	X
`-layerChangeTopLayer` *layerName*	X	X
`-minEscapeDistance` *unit*	X	X
`-msgRate` *int*	X	X
`-nets {`*net_name_list* `| <filename | ~<filename}`	X	X
`-overflowMap`		X
`-routeWidth` *real*	X	X
`-selected_bump`	X	X
`-spreadWiresFactor` *value*	X	X
`-straightConnections [straightWithDrcClean] [straightWithChanges]`	NA	NA
`-type {power | signal}`	NA	NA
`-verbose`	X	X

X[1]	When run incrementally, the `fcroute` PIO mode partially supports area-based routing in the specified coordinates (`fcroute -area {`*x1 y1 x2 y2*`} -incremental`) for the area in which the net is routed.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `fcroute` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man fcroute`

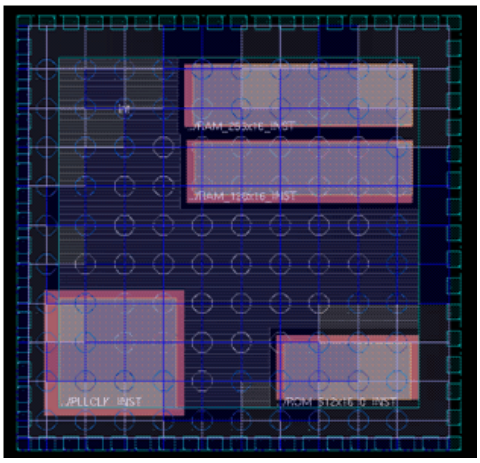
`-area {x1 y1 x2 y2}`	Specifies the coordinates for area I/O routing. You must enclose the x and y coordinates in curly braces and separate them with a space. **Notes**: • A warning message is issued if you specify area constraints in the flip chip route PIO mode. (`fcroute -designStyle pio`) • If `-route_pg_style` is specified as `finger`, the PG finger is routed in the area specified with `-area`. If `-route_pg_style` is not specified or specified as `none`, area I/O routing is done within the coordinates specified with `-area`.
`-connectInsideArea`	Specifies that connections from all sources within the specified area can connect only to targets that are also inside that area. **Note:** This parameter is only available if you also specify the `-area` parameter. *Default:* If you do not specify this option the software makes connections from all sources within the specified area to targets both inside and outside the specified area.
`-connectTsvToBump`	Connects TSVs to signal bumps. Use the `-connectTsvToBump` parameter with `-type signal`. This parameter cannot be used along with -connectTsvToPad and `-connectTsvToRingStripe`, which provide other routing options. **Note**: Different layers have different types of bumps. It is recommended that you route the bump on the back side and front side separately using the `srouteExcludeBumpType` statement in the extra configuration file.
`-connectTsvToPad`	Connects TSVs to IO pads. Use the `-connectTsvToPad` parameter with `-type signal`.
`-connectTsvToRingStripe`	Connects TSV with power net to the power mesh. Use the `-connectTsvToRingStripe` parameter with `-type power`. For more information, see the "*TSV/Bump/Back Side Metal Modeling in Innovus*" section in the Design Methodology for 3D IC with Through Silicon Via chapter of the *Innovus User Guide*.
`-constraintFile filename`	
	Specifies the file that contains constraints for flip chip routing. For more information, see "Useful Constraints for Flip Chip Routing" in the Flip Chip Methodologies chapter of the *Innovus User Guide.*
`-deleteExistingRoutes`	
	Specifies that the software remove existing connections when you use the `fcroute` command multiple times. *Default:* The software maintains existing connections each time you issue the `fcroute` command. **Note:** If you specify the `-area` parameter, existing routes are always preserved, even if you also specify the `-deleteExistingRoutes` parameter.
`-designStyle {aio | pio}`	
	Selects between area I/O and peripheral I/O routing styles. The area I/O (`aio`) mode supports area-based routing (`fcroute -area`). However, the peripheral I/O (`pio`) mode partially supports area-based routing only if you set the `-incremental` parameter. You can run fcroute in the PIO/AIO mode when pre-routes are already in the design, without deleting or re-routing the pre-routes. By default, this feature is true for `fcroute -type signal -designStyle aio`. For the `pio` mode, you should add the `-incremental` parameter to enable this feature. *Default*: `pio`

`-doubleBendRoute`	Allows double-bend routing on segments between a bump pin and a pad pin.  **Note:** Segments that connect directly to a bump pin or pad pin are always routed at 90 degrees. *Default*: Off
`-eco`	Enables ECO mode flip chip routing. When you specify this parameter, `fcroute` automatically performs typical ECO routing steps, such as deleting existing routing results or re-routing affected nets, whenever there is an ECO change. For more information, see the ECO Routing section in the "Flip Chip Methodologies" chapter of the *Innovus User Guide*. **Note:** The `-eco` parameter cannot be used along with `-incremental`.
`-extraConfig` *filename*	Specifies the name of an extra configuration file. This file contains additional fcroute commands that provide more control over the routes to bumps. For example, `fcroute -type signal -designStyle PIO -extraConfig mixmode.config` For more information, see "Useful Extra Configurations for Flip Chip Routing" in the Flip Chip Methodologies chapter of the *Innovus User Guide.*
`-incremental`	Runs the PIO mode incrementally when pre-routes are already in the design, without deleting or re-routing the pre-routes. **Note:** The `-incremental` parameter cannot be used along with `-eco`.
`-globalOnly`	Executes the global router to provide you with the routability statistics. When this option is specified, `fcroute` displays how the global router distributes routing resources and finishes routing with a minimum routing width. This is helpful for optimizing mode.
`-jogControl {preferWithChanges | preferSameLayer | preferDifferentLayer}`	
	Specifies that jogs are allowed during routing to avoid DRC violations. You must specify either the `-jogControl` parameter or the `-straightConnections` parameter. If you specify the `-jogControl` parameter, you can also specify one of the following options: • `preferWithChanges`: Specify this option if you prefer the software to make straight connections between targets, and to change layers instead of jog to avoid DRC violations. If you do not specify this option, the route uses both layer changes and jogging to avoid DRC violations. • `preferSameLayer`: If the route must jog to avoid a DRC violation, the jog occurs on the same layer whenever possible. This can result in routing in the non-preferred direction. • `preferDifferentLayer`: If the route must jog to avoid a DRC violation, the jog occurs on the layer that is in the preferred routing direction whenever possible.

`-keepDRC`	If specified, the command `clearDrc` will not run implicitly when executing the `fcroute` command, hence retaining the DRC violations. Otherwise, the command `clearDrc` runs implicitly when executing the `fcroute` command by default, purging all the violation markers.
`-layerChangeBotLayer` *layerName*	

	Specifies the bottom-most metal layer number / `LEF` layer names that the software can use when routing bumps.
	Default: If you do not specify this parameter, all layers are used.
	For example, you can specify the metal layer numbers, `8` and `2`, as follows:
	`fcroute -type power -routeWidth 4.0 -layerChangeTopLayer 8 -layerChangeBotLayer 2`
	You can also specify the `LEF` layer names, `METAL7` and `METAL1`, as follows:
	`fcroute -type power -routeWidth 4.0 -layerChangeTopLayer METAL7 -layerChangeBotLayer METAL1`

`-layerChangeTopLayer` *layerName*	
	Specifies the top-most metal layer number / `LEF` layer names that the software can use when routing bumps. *Default:* If you do not specify this parameter, all layers are used.
`-msgRate` *int*	Specifies the interval at which progress messages are generated. If you specify `1`, the software generates every progress message; if you specify `2`, the software generates every other progress message, and so on. *Default:* `0`
`-minEscapeDistance` *value*	
	Specifies the minimum distance from the edge of the pin to the edge of the wire segment.
`-nets {`*net_name_list* `| <filename | ~<filename}`	
	Specifies the nets to be connected or excluded from connection. *Default:* If you do not specify the `-nets` parameter, the software connects all power and ground nets in the design. Use one of the following options: • *net_name_list*: Specifies names of the nets to be connected. To specify more than one net, enclose the net names in braces and separate the names with a space. **Note:** As the `-nets` option accepts a maximum of 4096 string characters directly, use the *net_name_list* option if the number of nets to be connected is not very high. If a large number of nets need to be connected, use the `<filename` option or `~<filename` option with `-nets`. • `<filename`: Specifies the name of a text file containing the list of nets to be connected. Use this option if the number of nets to be connected is high. In the text file, separate net names with commas or the newline, space, or tab character. Wildcards can be used when specifying net names in the text file. • `~<filename`: Excludes the nets in the specified text file during flip chip routing. Use this option if the number of nets to be excluded is much lower than the number of nets to be connected. Wildcards can be used when specifying net names in the text file.

`-overflowMap`	Displays the thermal map using the width and spacing parameters specified in the extra configuration file. To display the thermal map in the floorplan view in the GUI, select *All Colors* which opens the Color Preferences form, then select *GCell Overflow*. **Note:** The routing congestion can also be displayed using the `setLayerPreference` command. (`setLayerPreference gcellOvflow -isVisible 1`) 
`-route_pg_style {none finger mesh}`	
	Specifies the routing pattern for flip chip route power routing: • `none`: For normal PG routing • `finger`: For PG finger routing style, in which you need to specify the routing direction. This option must be used with `fcroute -area` option. PG finger routing improves usability and provides better IR drop by lowering rail resistance. • `mesh`: For mesh-style PG routing on the redistribution layer (RDL). This option is used to increase PG density. If you specify PG mesh routing, you must specify the main width of the PG mesh by using the `setFlipChipMode -pg_mesh_main_width` parameter. *Default:* `none`
`-routeWidth` *real*	Connects wires to bumps using the specified width only, regardless of the size of the bump or the source to which the bump connects. *Default:* When using the PIO routing style, FCroute determines the width of the route as the bump width or I/O cell pin width that is connected to the bump, whichever is smaller. When using the AIO routing style, FCroute does not restrict the size of the RDL route to the width of the I/O cell pin and can route a wire that is larger in width than the specified I/O cell pin width. To override the above default setting for AIO routing style, you can use the `srouteUseSpecifiedWidthForTopLayer TRUE` and `srouteRouteWidthForLowerLayer` *db_unit* variables in the extra configuration file. This forces `fcroute` to use user-defined width for the top layer connecting to the bump and the lower layer. **Note:** `The fcroute` command considers the route width value from the constraint file if it is higher than `fcroute -routeWidth`, to enable splitting of wires.
`-selected_bump`	Routes the selected bumps. The selected bumps can be either power bumps or signal bumps.

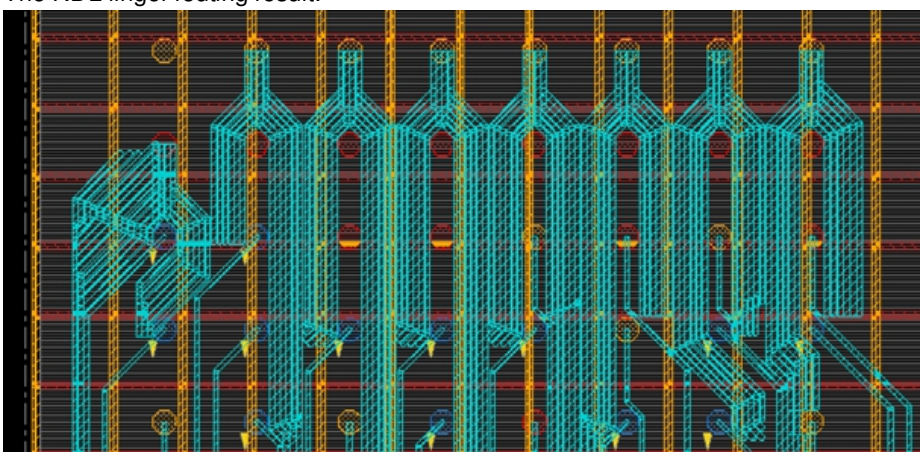
`-spreadWiresFactor` *`value`*	Specifies automatic spreading of wires during bump routing in order to prevent any SI violations. The *`value`* is applied as a multiple of the minimum spacing that is required.
`-straightConnections [straightWithDRCClean] [straightWithChanges]`	
	Specifies that only straight connections are made between targets and that jogs are not allowed during routing. This option is available only with `fcroute -type power`. You must specify either the `-straightConnections` parameter or the `-jogControl` parameter. If you specify the `-straightConnections` parameter, you can also specify one or both of the following options: • `straightWithDRCClean`: Leaves a route open if a straight connection cannot reach a target without causing a DRC violation. If you do not specify this option, the software makes a straight connection to a target, even if the route creates a DRC violations. • `straightWithChanges`: Permits the route to change to another layer to avoid DRC violations. If you do not specify this option, the software makes only straight connections on the same layer.
`-subclass` *`subclass_string`*	
	Assigns specified string as a subclass name to the wires and vias created.
`-type {power | signal}`	Specifies whether power or signal bumps are to be routed. **Note:** For `-type power`: • If `-route_pg_style` is specified as `finger`, the power bumps are routed in the finger routing style. • If `-route_pg_style` is not specified or specified as `none`, the power bumps are routed in normal PG routing style.
`-verbose`	Specifies that each progress message appears in the log file and the console.

Example

- The following set of commands can be used to route RDL finger in the South direction:

```
setFlipChipMode -route_pg_style finger
setFlipChipMode -finger_direction S
setFlipChipMode -finger_max_width 13
setFlipChipMode -finger_min_width 10
setFlipChipMode -finger_target_mesh_layer_range M4:M6
fcroute -type power -area {0 0 1446 1405}
```

  The RDL finger routing result:



Related Information

- addNetToNetGroup
- createNetGroup
- getFlipChipMode
- reportSpecialRoute
- routePointToPoint
- setFlipChipMode
- verifyFlipChipRoutingConstraints
- Flip Chip Methodologies chapter in the *User Guide*
- Tools Menu chapter in the *Menu Reference*
  - *Flip Chip*
  - *TSV*

findPinPortNumber

```
findPinPortNumber
[-help]
[-area x1 y1 x2 y2 ]
[-bumps bump_list]
[-geometry_height {min [max]}]
[-geometry_width {min [max]}]
[-layer string]
[-selected]
[-cellName cell_name | -instName instance_name ]
{-pinName pin_name | -netName  net_name }
```

Determines suitable ports available for a bump. The command has the following functions:

- Check if the IO pad has a standard LEF definition.
- Suggest suitable ports for flip chip routing on the specified instance based on the pin name or nets in a specified area.
- Filter ports by layer, geometry size, and area. Only the ports that match the specified filter rules are returned by `findPinPortNumber`.
- Consider routing between bumps and ports to return the combinations of `{bump:inst:pin:port}` if the bumps for which ports are to be found are specified.

`findPinPortNumber` returns a port number string list. You can use `set result [findPinPortNumber]` or `catch {findPinPortNumber} result` to get the result.

The following commands provide additional support for the port numbering feature. Use these commands to add, edit, and delete bump properties easily:

- `addBumpConnectTargetConstraint`
- `deleteBumpConnectTargetConstraint`
- `editBumpConnectTargetConstraint`

Parameters

`-help`	Prints a brief description that includes type and default information for each `findPinPortNumber` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man findPinPortNumber`
`-area` *`x1 y1 x2 y2`*	Specifies the area in which the command will be used. `findPInPortNumber` returns port numbers overlapping with the specified area.
`-bumps` *`bump_list`*	Finds suitable ports for the specified bumps based on the smallest distance. This option supports only 1:1 pairing between ports and bumps. Therefore, if the number of ports is different from the number of bumps, pairing is based on the smaller number. `findPinPortNumber` allows empty bumps or bumps assigned to the same net as the pin. If the assigned net is different from the net with which the pin is connected, `findPinPortNumber` issues an error message and quits.
`-cellName` *`cell_name`*	Finds the pin port number of the instance of the specified cell. Wild cards are supported in *`cell_name`*.
`-geometry_height {`*`min`* `[`*`max`*`]}`	
	Specifies the minimum and maximum values for the height of the target port's geometry. If you specify only one value, the height is taken as equal to the specified value. Only ports that match the specified height criterion are returned by `findPinPortNumber`.
`-geometry_width {`*`min`* `[`*`max`*`]}`	
	Specifies the minimum and maximum values of the width of the target port's geometry. If you specify only one value, the width is taken as equal to the specified value. Only ports that match the specified width criterion are returned by `findPinPortNumber`.
`-instName` *`instance_name`*	Finds pin port number of the specified instance. Wild cards are supported in *`instance_name`*.
`-layer` *`string`*	Filters target ports by layer. If you specify this parameter, only ports on the specified layer are returned. You can specify either layer name or layer ID, such as `M8`.
`-pinName` *`pin_name`* `|` `-netName` *`net_name`*	
	Finds the port number on the specified pin or on the pin connected to the specified net.

`-selected`	Finds suitable ports for the selected instance and bumps:
	• If only bumps are selected, `findPinPortNumber` first finds the ports satisfying the specified filtering rules and then considers the routing between these ports and the selected bumps.
	• If only IO instances are selected:
	  ◦ If only one IO instance is selected, `findPinPortNumber` finds the ports satisfying the specified filtering rules.
	  ◦ If more than one IO instance is selected, `findPinPortNumber` issues an error message stating that only a single IO instance is supported for this option and then quit.
	• If both IO instances and bumps are selected:
	  ◦ If only one IO instance is selected, `findPinPortNumber` first finds the ports satisfying the specified filtering rules and then considers the routing between these ports and the selected bumps.
	  ◦ If more than one IO instance is selected, `findPinPortNumber` issues an error message stating that only a single IO instance is supported for this option and then quit.

Example

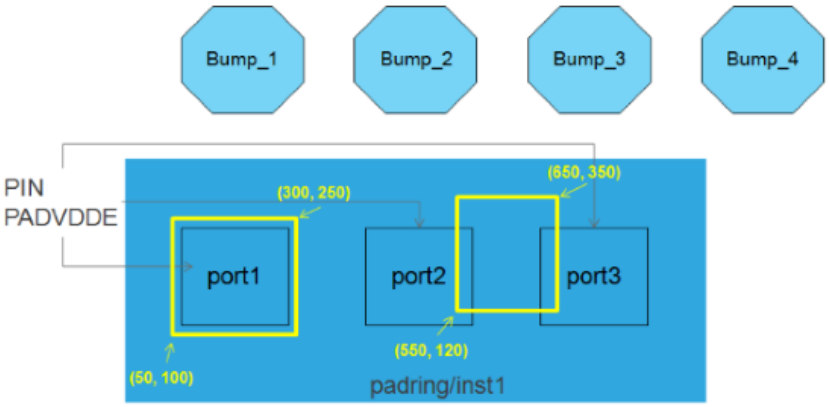
- The following command suggests port numbers on instances named `IOPADS_INST/ esd*` for the pins connected to net `VDD`.

  ```
  findPinPortNumber -instName IOPADS_INST/esd* -netName VDD
  ```

  The tool returns a list of suitable port numbers as follows:

  ```
  IOPADS_INST/esd:VDD:1 IOPADS_INST/esd1:VDD:1
  ```

- The diagram below depicts an IO instance `padring/inst1` with three ports for `PIN PADVDDE` on the same layer and with the same size. The bumps `Bump_1`, `Bump_2`, `Bump_3`, and `Bump_4` are empty.



  - The following command finds ports of the pin `PADVDDE` on the instance `padring/inst1` that meet the specified area constraint:

    ```
    findPinPortNumber -instName padring/inst1 -pinName PADVDDE -area "50 100 300 250"
    ```

    For the instance depicted in the diagram above, the command returns `{padring/inst1:PADVDDE:1}`.

  - The following command finds ports of the pin `PADVDDE` on the instance `padring/inst1` that meet the specified area constraint:

    ```
    findPinPortNumber -instName padring/inst1 -pinName PADVDDE -area “550 120 650 350”
    ```

    For the instance depicted, the command returns `{padring/inst1:PADVDDE:2 padring/inst1:PADVDDE:3}`. Both `port2` and `port3` are returned as they overlap with the specified area constraint.

  - The following command finds suitable ports of the pin `PADVDDE` on the instance `padring/inst1` for the specified bump:

    ```
    findPinPortNumber -instName padring/inst1 -pinName PADVDDE -bumps Bump_2
    ```

    For the instance depicted, the command returns `{Bump_2:padring/inst1:PADVDDE:2}`.

  - The following command finds suitable ports of the pin PADVDDE on the instance padring/inst1 for the specified bumps:

    ```
    findPinPortNumber -instName padring/inst1 -pinName PADVDDE -bumps {Bump_1 Bump_2 Bump_3 Bump_4}
    ```

    For the instance depicted, the command returns `{Bump_1:padring/inst1:PADVDDE:1 Bump_2:padring/inst1:PADVDDE:2 Bump_3:padring/inst1:PADVDDE:3}`.

Related Information

- `addBumpConnectTargetConstraint`
- `deleteBumpConnectTargetConstraint`
- `editBumpConnectTargetConstraint`
- Flip Chip Methodologies chapter of the *User Guide*
  - *Port Numbering*
- Tools Menu chapter in the *Menu Reference*
  - *Flip Chip*

fixBondPad

```
fixBondPad
[-help]
{{-ioInstName InstName [-pinName pinName]}
 | -selected}
```

Assigns a fixed status to a specified bond pad. This restricts other bond pad commands from modifying the bond pad.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `fixBondPad` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man fixBondPad`
`-ioInstName` *instName*	Specifies the name of the I/O instance whose bond pad will be assigned as fixed.
`-pinName` *pinName*	Specifies the name of the pin inside the instance.
`-selected`	Assigns a fixed status to a selected bond pad. You must first select the bond pad in the design display area, then use this parameter to assign the fixed status.

Example

- The following command assigns a fixed status to the bond pad in I/O instance `pad_addr[30]`:
  ```
  fixBondPad -ioInstName {pad_addr[30]}
  ```

Related Information

- checkBondPadSpacing
- getBondPad
- placeBondPad
- spaceBondPad
- staggerBondPad
- unfixBondPad

get_physical_info

```
get_physical_info
[-help]
{-object string | -selected }
[-highlight]
[-shapes {RING STRIPE FOLLOWPIN IOWIRE COREWIRE BLOCKWIRE PADRING BLOCKRING FILLWIRE FILLWIREOPC DRCFILL None}]
[-subclass {subclass_name_list}]
[-type {connection | routing_path}]
```

Gets the physical connection or routing path between IO pads and bumps. For example, you can select a bump or specify bump name to get information about connected IO pad. Alternatively, you can select or specify an IO pad and get information about connected bumps. You can specify IO pads by pad name, instance:pin combination, or instance:pin:port combination.

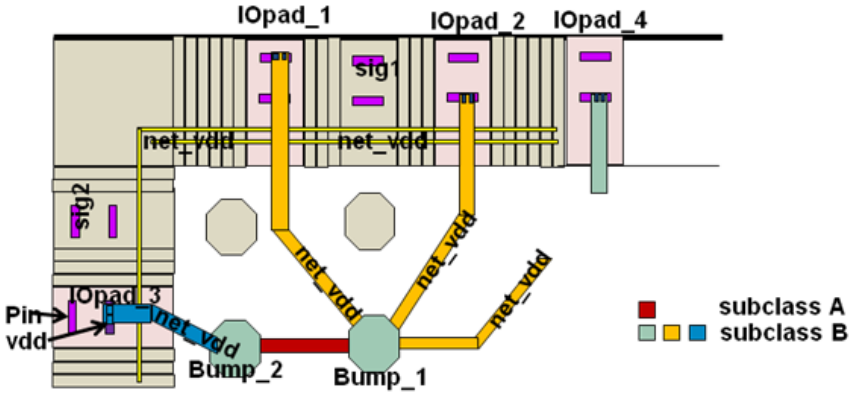
Use this command to check connections after RDL routing, either manual or through `fcroute`.

Parameters

`-help`	Prints a brief description that includes type and default information for each `get_physical_info` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man get_physical_info`
`-highlight`	Highlights the routing path.
`-object string`	Specifies the object from which to get the physical information. For bump, specify the complete bump name.For IO pad, specify one of the following: • *instance_name* • *instance_name*:*pin_name* • *instance_name*:*pin_name*:*port_number*
`-selected`	Gets the physical information for the selected object.
`-subclass {subclass_name_list}`	Specifies the subclass name of the wires and vias to be checked. You can specify multiple subclass names enclosed in braces. You can also use wildcards while specifying subclass names.
`-shapes {RING STRIPE FOLLOWPIN IOWIRE COREWIRE BLOCKWIRE PADRING BLOCKRING FILLWIRE FILLWIREOPC DRCFILL None}]`	
	Specifies the shapes of the wires and vias to be checked. The default value is `IOWIRE`.
`-type {connection | routing_path}`	
	Specifies the type of the physical information to return. The default type is `connection`. • `connection`: Returns the objects that are physically connected to the object you have specified or selected. For example, if you have selected an IO pad, it returns the names of the bump that are connected with the IO pad. If you have selected or specified a bump, it returns the IO pads connected to the bump, using the format *`ioInst:pin:port`*. • `routing_path`: Returns the pointer list of the wires and vias that are physically connected to the object you have selected or specified.

Examples

Consider the following design.



For this design, `get_physical_info` reports information as follows:

- `get_physical_info -object {IOpad_1:vdd:1}`
  Returns `{Bump_1}`
- `get_physical_info  -type connection -object {Bump_1}`
  Returns `{IOpad_1:vdd:1 IOpad_2:vdd:2  Bump_2}`
- `get_physical_info -object {Bump_1} -subclass B`
  Returns `{IOpad_1:vdd:1 IOpad_2:vdd:2}`
- `get_physical_info -type routing_path -object {Bump_2}`
  Returns `{0x2aaaba6c0ab0 0x2aaaba6c0b40 0x2aaaba6c0b70}`

Related Information

- `fcroute`
- Flip Chip Methodologies chapter in the *User Guide*

getBondPad

```
getBondPad
[-help]
-ioInstName InstName
[-pinName pinName]
```

Gets the current stagger position of a bond pad on a specified I/O instance.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `getBondPad` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man getBondPad`
`-ioInstName instName`	Specifies the name of the I/O instance whose bond pad position is reported.
`-pinName pinName`	Specifies the name of the pin inside the instance.

Example

- The following command gets the current stagger position of the bond pad on the I/O instance `testmacro` with pin `Z2`:
  ```
  getBondPad -ioInstName testmacro -pinName Z2
  ```
  Returns:
  ```
  Pin Name = Z2, Bond Pad Name = testmacro_PAD1, Position = 0, Fixed = 0
  ```

Related Information

- checkBondPadSpacing
- fixBondPad
- placeBondPad
- spaceBondPad
- staggerBondPad
- unfixBondPad

getFlipChipMode

```
getFlipChipMode
[-help]
[-allow_layer_change]
[-auto_pairing_file]
[-bump_use_oct_shape]
[-check_bump_access_directions]
[-compaction]
[-connectPowerCellToBump]
[-constraintFile]
[-drop_via_on_all_geometries]
[-drop_via_on_power_mesh]
[-extraConfig]
[-finger_direction]
[-finger_max_width]
[-finger_min_width]
[-finger_target_mesh_layer_range]
[-honor_bump_connect_target_constraint]
[-ignore_pad_type_check]
[-layerChangeBotLayer]
[-layerChangeTopLayer]
[-lower_layer_prevent_45_routing]
[-lower_layer_route_width]
[-merge_nearby_pin]
[-multi_pad_routing_style]
[-multipleConnection]
[-pg_mesh_direction]
[-pg_mesh_main_width]
[-pg_mesh_max_width]
[-pg_mesh_width]
[-prevent_diagonal_wire_access_bumps]
[-prevent_via_under_bump]
[-prevent_via_under_bump_extension]
[-route_pg_style]
[-route_style]
[-routeWidth]
[-via_abut_bump]
[-via_to_pad_honor_min_spacing]
[-wire_to_pad_honor_min_spacing
[-quiet]
[-nonDefault]
```

Returns information about the specified `setFlipChipMode` parameter in the Innovus log file and the console. The following information is returned:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the software returns information for all of the `setFlipChipMode` mode parameters.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `getFlipChipMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man getFlipChipMode`
`parameter_names`	Returns information for the specified parameters. You can specify one or more parameters. See `setFlipChipMode` for descriptions of the flip chip mode parameters you can specify.
`-quiet`	Returns the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software returns the current settings of all `setFlipChipMode` parameters in Tcl list format.
`-nonDefault`	Displays the current settings for the parameters that have nondefault settings.

Related Information

- `addBumpConnectTargetConstraint`
- `fcroute`
- `placePIO`
- `setFlipChipMode`
- `sroute`
- `verifyFlipChipRoutingConstraints`
- Flip Chip Methodologies chapter in the *User Guide*
- Tools Menu chapter of the *Menu Reference*
  - *Flip Chip*

ioInstOverlapCheck

```
ioInstOverlapCheck [-help]
```

Checks to see if any area I/O instance is overlapped with another area I/O instance. If an overlap occurs, a warning message is dumped to the console window, and the overlapped instances are highlighted in the work area.

Parameters

`-help`	Prints a brief description of the `ioInstOverlapCheck` command. For a detailed description of the command, use the `man` command: `man ioInstOverlapCheck`

placeBondPad

```
placeBondPad
[-help]
{-ioInstName instName | -selected}
[-pad padName]
[-pinName pinName]
[-position {i | m | o}]
[-fix]
```

Places a bond pad on a specified I/O instance. It defines the stagger position of a specified bump on a single or selected set of I/O instances.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `placeBondPad` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man placeBondPad`
`-fix`	Assigns a fixed status to the specified bond pad. This restricts other bond pad commands from modifying the bond pad.
`-ioInstName` *name*	Specifies the name of the I/O instance of where the bond pad will be placed.
`-pad` *padName*	Specifies the name of the bond pad.
`-pinName` *pinName*	Specifies the name of the pin inside the instance.
`-position {I | M | O}`	Defines the stagger position of the specified bond pad on the specified I/O instance. You can stagger the position of the bond pad using the inner (`I`), middle (`M`), and outer (`O`) positions.
`-selected`	Places a selected bond pad on the specified I/O instance. You must first select the bond pad in the design display area, then use this parameter to place the bond pad on the specified I/O instance.

Related Information

- checkBondPadSpacing
- fixBondPad
- getBondPad
- spaceBondPad
- staggerBondPad
- unfixBondPad

readFlipChipProperty

```
readFlipChipProperty
[-help]
fileName
[-onlyBumpConnectTargetConstraint]
```

Reads the text file containing flip chip property information. The `readFlipChipProperty` command prints an error message if the following errors occur:

- Type error of property name
- Instance/pin not found
- Metal layer not found
- Size is zero or minus

Parameters

`-help`	Prints out the command usage that includes type and default information for each `readFlipChipProperty` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man readFlipChipProperty`
`fileName`	Specifies the bump and IO pad property text file.
`-onlyBumpConnectTargetConstraint`	Reads only the bump connect target constraints.

Example

The following command reads only the bump connect target constraints from the `test2` input file:

```
readFlipChipProperty test2 -onlyBumpConnectTargetConstraint
```

Related Information

- `readIoUpdate`
- `writeFlipChipProperty`
- Flip Chip Methodologies chapter of the *User Guide*
- Tools Menu chapter in the *Menu Reference*
  - *Flip Chip*

readIoUpdate

```
readIoUpdate
[-help]
[-checkOnly]
inputFile
```

Reads a simple ASCII file which contains the bump and instance information.

The input die abstract file is a simple ASCII file that consists of bump and I/O instance information only; these are the only two objects that can be modified by SiP Layout.

- If a bump is defined in the db and not defined in the input file, the bump gets deleted from the db.
- If the instance names in the input file and db do not match, an error is displayed.
- If the nets referenced in the bump section in the input file do not exist in the db, an error is displayed.

Parameters

`-help`	Prints out the command usage that includes type and default information for each `readIoUpdate` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man readIoUpdate`
`-checkOnly`	Reports differences between the input file and the design in memory. This is an optional parameter. This parameter only generates a report and does not update the die.
`inputFile`	Specifies a file containing the state of I/O drivers and bumps (placement and assignments). This is a required parameter. The information from this file is updated in the database.

Related Information

- `readFlipChipProperty`
- `readPackage`
- `writeFlipChipProperty`
- Flip Chip Methodologies chapter in the *User Guide*
- Tools Menu chapter in the *Menu Reference*
  - *Flip Chip*

readPackage

```
readPackage
[-help]
packageFile
```

Reads the package file dumped out by the System-in-Package (SiP) layout in XML format and displays the package ball/finger location in the Innovus floorplan view. The command also displays the flight lines from the IO pads to fingers for a wire-bond design or from bumps to package balls for a flip chip design.

The package overlay schema dumped out by SiP includes bond finger, package balls, and many other package objects. `readPackage` displays all the package objects for a wire-bond or flip chip design, such as BGA_BALL, BOND_FINGER, BONDWIRE and ROUTE, in the Innovus main window. The exact information displayed depends on the package overlay file dumped out by SiP. You can turn on or turn off the display of specific package objects using the Custom tab of the Color Preferences form in Innovus. You can view attributes of a selected package object using Attribute Editor.

Parameters

`-help`	Prints out the command usage that includes type and default information for each `readPackage` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man readPackage`
`packageFile`	Specifies the name of the package file.

Related Information

- `readIoUpdate`
- Flip Chip Methodologies chapter in the *User Guide*

reportProbePins

```
reportProbePins [-help]
```

Generates a report of all probe pins that are used for package-level design and testing. If no probe points exist, the software returns the following message:

```
"No Probe Points Defined"
```

Note: You can set or delete probe points using the `setProbePin` or `unsetProbePin` commands, respectively.

Parameters

`-help`	Prints a brief description of the `reportProbePins` command. For a detailed description of the command, use the `man` command: `man reportProbePins`

Example

- The following command generates a report of all probe pins:
  `reportProbePins`

Related Information

- `setProbePin`
- `unsetProbePin`

reportSpecialRoute

```
reportSpecialRoute
[-help]
reportName
[-center_line]
[-net netName]
[[ [-by_layer]
   [-highlight45layer layerName]
   [-lengththreshold value]
   [-resfile resFileName]
   [-selected]
   [-summary]
   [-wire_segment]
   [-wirelength]   ]
 | [-interposer_route
   [[-exclude_nets netName]
    [-interposer_layers layerName]
    [-interposer_route_constraint_file ircFileName]]]]
```

Reports the length of all RDL routes (SPECIALNETS) created with the `fcroute` command. This includes support for Manhattan and 45-degree routes. You can report the wire length of each SPECIALNET and report the SPECIALNETS that are greater in length than the specified threshold value.

Use the `-by_layer` and `-wire_segment` options to report the width and length of the special nets per layer and wire segment.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `reportSpecialRoute` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man reportSpecialRoute`

`-by_layer`	Reports on layers of each special net. Here's a sample format of the report when the `-by_layer` option is specified:
	`NET: port_pad_data_in[0] TOTAL SEGMENTS 4 WIRE LENGTH 303.4980 VIAS 1`
	`LAYER 8(METAL8) TOTAL LENGTH 303.4980`
	`IOWIRE SEGMENTS 4 WIRE LENGTH 303.4980 VIAS 1`

`-center_line`	Specifies that routing length is calculated by center line. That is, the sum of wire center line is used to calculate wire length. Without this parameter, the routing length is calculated as sum of wire segments. Sum of wire center line Sum of wire segments
`-exclude_nets netName`	
	Specifies the net names to be excluded. This option can be used only if `-interposer_route` is specified.
`-highlight45layer layerName`	

	Highlights (in yellow) all nets with 45-degree routing on the specified layer in the design display area.
`-interposer_layers` *layerName*	
	Specifies the interposer layers. Use this option to report the maximum and minimum routing length from tap via to tap via and from bump to tap via by layer. This option can be used only if `-interposer_route` is specified.

`-interposer_route`	Reports the interposer route statistics, including maximum, minimum, and average wire length of the interposer routing. `reportSpecialRoute` reads the constraints defined by `setIntegRouteConstraint` to decide which nets are in one group. The `setIntegRouteConstraint` constraints are either obtained from the DB or can be specified through a file by using the `-interposer_route_constraint_file` option.
	The report format is as shown below:
	`### Header ###`
	`### Total Nets of Interposer Route ###`
	`Total nets: <total nets number>`
	`Layer: Layer_name`
	`Net with Max wire length: <net name + net length>`
	`Net with Min wire length: <net name + net length>`
	`Average wire length:`
	
	`### Net Group <group_name1> ###`
	`Total nets: <nets number in group>`
	`Layer: Layer_name`
	`Net with Max wire length:`
	`<net name> :`
	`net length :`
	`Euclidean distance:`
	`Manhattan distance:`
	`Net with Min wire length:`
	`<net name> :`
	`net length :`
	`Euclidean distance:`
	`Manhattan distance:`
	`Average wire length:`
	`Nets name:`
	
	`### Net Group <group_name2> ###`
	`Total nets: <nets number in group>`
	`Layer: Layer_name`
	`Net with Max length: <net name + net length>`

	... Net with Min length: <net name + net length> ... Average wire length: Nets name: ... The *group_name* of the nets is obtained from `setIntegRouteConstraint -name group_name`. If `-name` is not specified, the tool uses a number to mark the net group. For example: `### Net Group <1> ###`
`-interposer_route_constraint_file` *ircFileName*	
	Specifies the name of the interposer route constraint file. This option can be used only if `-interposer_route` is specified.
`-lengththreshold` *value*	
	Reports on the `SPECIALNETS` that are greater in length than the specified value. These values are listed in the output file specified by *reportName* and the routes are highlighted in the GUI.
`-net` *netName*	Reports on the wire length of each `SPECIALNET` fora specified net. **Note:** You must use this parameter in conjunction with the `-wirelength` or `-lengththreshold` parameters.
reportName	Specifies the report file name.
`-resfile` *resFileName*	
	Specifies the resistance file name (`fcroute.res`) generated by `fcroute` if `srouteFCReport` *file name* is specified in the config file and highlights the violated nets on GUI when the resistance does not match the max resistance constraint defined in the `fcroute` constraint file.
`-selected`	Reports the length of the selected wire segments for each special net.

`-summary`	Summarizes the wire length and VIAs for special nets. Here's a sample format of the report when `-summary` is specified:
	
	`----------`
	
	`----------`
	`Report wire length and VIAs per layer`
	`----------`
	` 8(METAL8)  Total wire segments 15 Total length 1243.527000`
	`  ----------`
	`    #Shape           Wire segment    Length`
	`     Horizontal      5                 297.6950`
	`     Vertical        5                 466.4500`
	`     45degree        5                 479.3820`
	`  ----------`
	`cutlayer VIA78 Total VIAs: 4 Total Cuts: 1360`
	
	`----------`
	
	`----------`
	`Report wire length per shape for all layers`
	`----------`
	` #Shape           Wire segment    Length`
	`  Horizontal      5                 297.6950`
	`  Vertical        5                 466.4500`
	`  45degree        5                 479.3820`
	`----------`
	
	`Total Wire Length of All Special Nets: 1243.5270`

`-wire_segment`	Reports on the wire segments of each special net. Here's a sample format of the report when `-wire_segment` is specified:
	
	`NET: port_pad_data_in[1] TOTAL SEGMENTS 4 WIRE LENGTH 251.9120 VIAS 1`
	`WIRESEGMENT 1 SHAPE Vertical LENGTH 110.7050 674.7200 880.7200 686.7200 991.4250`
	`WIRESEGMENT 2 SHAPE 45degree LENGTH 76.3320 620.7450 985.4250 686.7200 1051.4000`
	`WIRESEGMENT 3 SHAPE Horizontal LENGTH 38.3550 588.3900 1039.4000 626.7450 1051.4000`
	`WIRESEGMENT 4 SHAPE Vertical LENGTH 26.5200 588.3900 1039.4000 597.3900 1065.9200`
	`IOWIRE SEGMENTS 4 WIRE LENGTH 251.9120 VIAS 1`

`-wirelength`	Reports on the wire length of each `SPECIALNET`.

Example

- The following command reports the wire length of each `SPECIALNET` and lists it in the `spwire.rpt` report file:
  `reportSpecialRoute spwire.rpt -wirelength`

Example report file:

```
############################################################
#  Generated by:      Cadence Innovus 15.10-b024_1
#  OS:                Linux x86_64(Host ID rlno-leenap)
#  Generated on:      Tue Apr  7 13:28:46 2015
#  Design:            DTMF_CHIP
#  Command:           reportSpecialRoute spwire.rpt -wirelength
############################################################

NET: VDD TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0


NET: VSS TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0


NET: ibias TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0


NET: int TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0


NET: pllrst TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0


NET: port_pad_data_in[0] TOTAL SEGMENTS 4 WIRE LENGTH 303.4980 VIAS 1
       WIDTH   12.0000    LENGTH      303.4980
       IOWIRE SEGMENTS 4 WIRE LENGTH 303.4980 VIAS 1


NET: port_pad_data_in[10] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0


NET: port_pad_data_in[11] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0


NET: port_pad_data_in[12] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0


NET: port_pad_data_in[13] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0


NET: port_pad_data_in[14] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0


NET: port_pad_data_in[15] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0
```

```
NET: port_pad_data_in[1] TOTAL SEGMENTS 4 WIRE LENGTH 251.9120 VIAS 1
       WIDTH   12.0000    LENGTH      225.3920
       WIDTH    9.0000    LENGTH       26.5200
       IOWIRE SEGMENTS 4 WIRE LENGTH 251.9120 VIAS 1

NET: port_pad_data_in[2] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: port_pad_data_in[3] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: port_pad_data_in[4] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: port_pad_data_in[5] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: port_pad_data_in[6] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: port_pad_data_in[7] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: port_pad_data_in[8] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: port_pad_data_in[9] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: port_pad_data_out[0] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: port_pad_data_out[10] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: port_pad_data_out[11] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: port_pad_data_out[12] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: port_pad_data_out[13] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0
```

```
NET: port_pad_data_out[14] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: port_pad_data_out[15] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: port_pad_data_out[1] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: port_pad_data_out[2] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: port_pad_data_out[3] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: port_pad_data_out[4] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: port_pad_data_out[5] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: port_pad_data_out[6] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: port_pad_data_out[7] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: port_pad_data_out[8] TOTAL SEGMENTS 3 WIRE LENGTH 435.8845 VIAS 1
       WIDTH   12.0000    LENGTH      269.2945
       WIDTH    9.0000    LENGTH      166.5900
       IOWIRE SEGMENTS 3 WIRE LENGTH 435.8845 VIAS 1

NET: port_pad_data_out[9] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: refclk TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: reset TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: scan_clk TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0
```

```
NET: scan_en TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: scan_in_1 TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: scan_in_2 TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: scan_out_1 TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: scan_out_2 TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: spi_data TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: spi_fs TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: tdigit[0] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: tdigit[1] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: tdigit[2] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: tdigit[3] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: tdigit[4] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: tdigit[5] TOTAL SEGMENTS 4 WIRE LENGTH 252.2325 VIAS 1
       WIDTH   12.0000    LENGTH      213.2325
       WIDTH    9.0000    LENGTH       39.0000
       IOWIRE SEGMENTS 4 WIRE LENGTH 252.2325 VIAS 1

NET: tdigit[6] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: tdigit[7] TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0
```

```
NET: tdigit_flag TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: test_mode TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: vcom TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

NET: vcop TOTAL SEGMENTS 0 WIRE LENGTH 0.0000 VIAS 0

Total Wire Length of All Special Nets: 1243.5270
```

Related Information

- fcroute
- reportWire

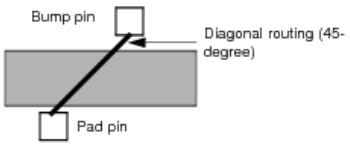
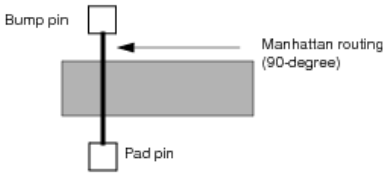
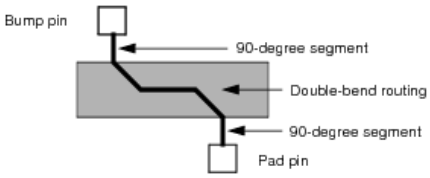
routePointToPoint

```
routePointToPoint
[-help]
[-constraintFile filename]
[-drop_via_in_trace]
[-net {[netName | -1] [layerName | -1] [(x y)]}]
[-pin {[instName | -1] [pinName | -1] [(x y)]}]
[-routeLayer {bot[:top[:step]] [,bot[:top[:step]]]}]
[-routeStyle {manhattan | doubleBend | diagonal}]
[-spacing spacingValue]
[-split {layerBase maxWidth:value [gap:value] [style:RIVER|MESH]}]
[-subclass subclass_name]
[-width widthValue]
[{[-useExactLoc] [-guidePoint {{x1 y1} {x2 y2}...} [-keep_partial]]} [-offset {x y}]]
```

Specifies routing constraints to perform point-to-point routing between I/O pad pins and bumps, and wires and bumps for SPECIALNETS (only) that are defined in the DEF file.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `routePointToPoint` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man routePointToPoint`
`-constraintFile` *`fileName`*	Specifies the name of the file that contains the nets defined in the DEF file.
`-drop_via_in_trace`	Drops all possible vias in the trace.
`-guidePoint {{x1 y1} {x2 y2}...}`	
	Specifies guiding points for routing. *`x`* and *`y`* values are real values in micron. You can specify up to 256 guiding points. The P2P router routes through all the usable guiding points in an order of distance to/from target. This feature allows you to control P2P router more precisely on pin selection.
`-keep_partial`	Keeps routed segments when routing with guide points fails partially. **Note:** This parameter can be used only with `-guidePoint`.
`-net {netName | -1 [layerName | -1] (x y)}`	
	Specifies net options, such as net name, net layer, and location coordinates. The *`layerName`* can now also be the TSV layer name.
`-offset {x y}`	Defines the maximum search offset from the original user click location. **Note:** This parameter can be used only with `-useExactLoc`.
`-pin {[instName | -1] [pinName | -1] [(x y)]}`	
	Specifies the pin options, such as the instance name, the pin name, or the location coordinates.
`-routeLayer {bot[:top[:step]] [,bot[:top[:step]]]}`	

	Specifies the route layer.
`-routeStyle [manhattan | doubleBend | diagonal]`	
	Specifies the routing style options for performing the point-to-point routing. • `diagonal`  • `manhattan`  • `doubleBend` 
`-spacing` *`spacingValue`*	Specifies the route spacing, in micrometers.
`-split {layerBase maxWidth:`*`value`* `[gap:`*`value`*`] [style:RIVER|MESH]}`	

	Splits the wire segment based on the maximum width and splitting style.
	The router gets the maximum width value from the layer information; the router does not split the wire segment when the wire width does not exceed the layer's maximum width.
	• `maxWidth:`*`value`*: `Specifies the maximum width for splitting.`
	• `gap:`*`value`*: Specifies the splitting gap.
	• `style:RIVER|MESH`:Specifies the splitting style:
	  ◦ `RIVER`: Allows only one crossing of the split wire.
	  ◦ `MASH`: Allows crossing of all of split wires and also drops all the crossing sections.

`-subclass` *`subclass_name`*	Assigns the specified string as a subclass name to the wires and vias created.

`-useExactLoc`	Uses the exact location that you click to perform routing. You can use the optional -offset parameter to define the maximum search offset from the original user click location. If the location does not fulfill the pin access requirements, P2P router performs an auto search to find a new access point. **Note:** If the `-useExactLoc` parameter is not specified, P2P finds the start end locations on the user-clicked object on its own.
`-width` *widthValue*	Specifies the route width, in micrometers.

Examples

The following command performs diagonal style point-to-point routing between the I/O pad pin `newIo_2` and bump `Bump_41_0_4` on route layer `M8`, having a default route width of `0.44` micrometers, and a spacing of `0.46` micrometers:

```
routePointToPoint -routeLayer M8:M8 -width 0.44 -spacing 0.46 -routeStyle diagonal -pin {newIo_2 -1 (-21.3115
535.932)} -pin {Bump_41_0_4 port_pad_data_in[10] (76.507 482.032)}
```

Related Information

- `fcroute`
- Flip Chip Methodologies chapter in the *User Guide*
  - P2P Router

select_bump

```
select_bump
[-help]
[-alternate {row column}]
[-area {{x1 y1 x2 y2} ...}]
[-assigned]
[-bump_cell {cell_list}]
[-bumps {bump_list}]
[-floating]
[-max_distance_to_side distance]
[-nets {net_list}]
[-side {top bottom left right}]
[-start_lower_left]
[-type {signal power ground}]
```

Enables you to select bumps in different ways. For example, you can select bumps by type or by net. By default, selects all bumps in the design.

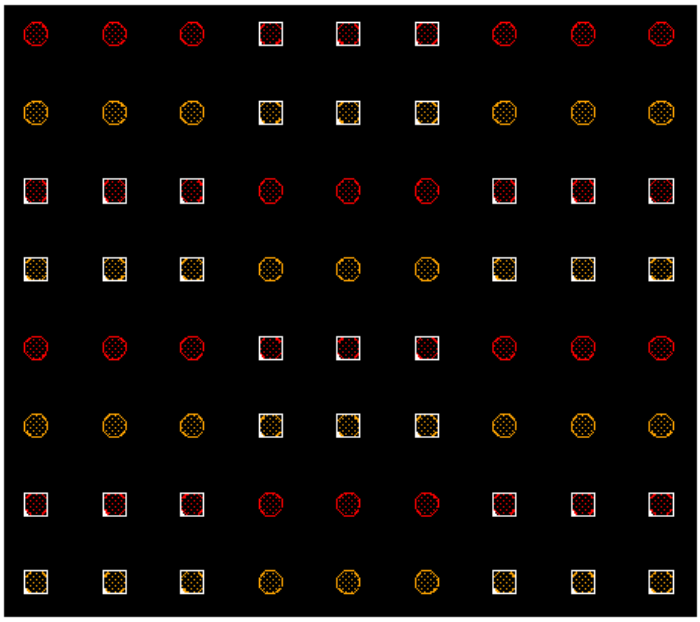
Parameters

`-help`	Prints a brief description that includes type and default information for each `select_bump` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man select_bump`
`-alternate {row column}`	Specifies the alternate bump pattern. You have to preselect the area in which the bumps are to be alternate-selected before using this option.
`-area {{x1 y1 x2 y2} ...}`	Specifies one or more areas from which bumps are to be selected.
`-assigned`	Selects only the bumps that are assigned.
`-bump_cell {cell_list}`	Specifies names of bump cells and the bumps with these bump cells are selected. You can use wildcards in the bump cell list.
`-bumps {bump_list}`	Specifies names of the bumps to be selected. You can use wildcards in the bump list.
`-floating`	Selects only unassigned or floating bumps.
`-max_distance_to_side distance`	
	Specifies the maximum distance away from the side in microns. When specified, only the bumps with geometries completely in that region are selected. By default, half the width/height of the die is used as the maximum distance to each side. For *distance*, minimum value is `0` and maximum is `1e+20`.
`-nets {net_list}`	Specifies names of nets and the related bumps are selected. You can use wildcards in the net list.
`-side {top bottom left right}`	
	Specifies the side relative to which bumps are selected.
`-start_lower_left`	
	Specifies the start point of alternate selection. If it is not specified, the lower-left point is considered the start point of alternate deselection. **Note**: This parameter can only be used with `-alternate`.
`-type {signal power ground}`	
	Specifies the type of bumps to be selected.

Examples

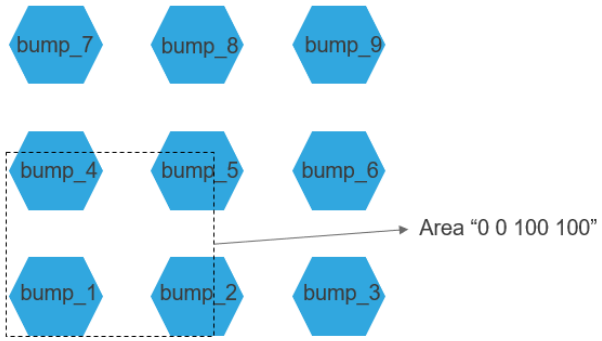
- The following command selects all bumps in the design:
  ```
  select_bump
  ```
- The following command selects the bottom edge of bump geometry no more than 500 microns away from the top side:
  ```
  select_bump -side top -max_distance_to_side  500
  ```
- After selecting all the bumps in a design or in a specific area, use the following command to alternately select a 2x3 bump pattern starting from the lower left corner:
  ```
  select_bump -alternate {2 3} -start_lower_left
  ```

The resulting selection pattern is shown below.



- The following command selects four bumps, `bump_1`, `bump_2`, `bump_4`, and `bump_5`, as shown below:

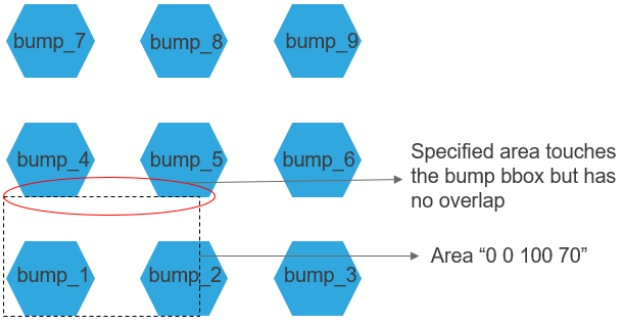
```
select_bump -area {0 0 100 100}
```



Bumps generated by `create_bump`

- The following command selects two bumps, `bump_1` and `bump_2`, as shown below. Note that although the specified area touches the bump bounding box (bump bbox), there is no overlap:

```
select_bump -area {0 0 100 70}
```



Bumps generated by `create_bump`

Related Information

- `copy_bump`
- `deleteBumps`
- `deselect_bump`
- `edit_bump_name`
- Flip Chip Methodologies chapter in the *User Guide*
- Tools Menu chapter in the *Menu Reference*
  - *Flip Chip*

setBumpFixed

```
setBumpFixed
[-help]
{-allBumps | -byBumpName {list} | -byBumpSite {list}}
```

Fixes a specified group of assigned bumps to keep them from being reassigned. The software updates the DEF file by adding the `+FIXEDBUMP` attribute to the specified group of assigned bumps to indicate that they are now in a fixed state. You can use the `defOut` command to view the bumps with the `+FIXEDBUMP` attribute. Use this command after bumps are assigned with the `assignBump` command or defined in the I/O file with the signal name.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setBumpFixed` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setBumpFixed`.
`-allBumps`	Fixes all of the assigned bumps.
`-byBumpName list`	Fixes the assigned bumps that are specified in the bump name *`list`*.
`-byBumpSite list`	Fixes the assigned bumps that are specified in the bump site *`list`*.

Example

- The following command fixes all of the assigned bumps:
  ```
  setBumpFixed -allBumps
  ```

Related Information

- `assignBump`
- `defOut`
- `setBumpPlacementStatus`
- `unfixBump`
- Flip Chip Methodologies chapter in the *User Guide*
- Tools Menu chapter in the *Menu Reference*
  - *Flip Chip*

setBumpPlacementStatus

```
setBumpPlacementStatus
[-help]
[-bumpName Name | -selected]
PLACED | FIXED | COVER
```

Sets the bump placement status.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setBumpPlacementStatus` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setBumpPlacementStatus`.
`-bumpName Name`	Specifies the name of the bump to set the placement status.
`PLACED | FIXED | COVER`	Specifies the bump placement status.
`-selected`	Specifies to set the bump placement status for the selected bump.

Example

- The following command sets the placement status of the bump `Bump_53_2_5` to `FIXED`:
  `setBumpPlacementStatus FIXED -bumpName Bump_53_2_5`

  Note: You can also double-click the bump to change the placement status from `PLACED` to `FIXED`.

Related Information

- `setBumpFixed`
- `unfixBump`
- Flip Chip Methodologies chapter in the *User Guide*
- Data Preparation chapter in the *User Guide*
  - *Defining BUMP CELL Placement Status*
- Tools Menu chapter in the *Menu Reference*
  - *Flip Chip*

setFlipChipMode

```
setFlipChipMode
[-help]
[-reset]
[-allow_layer_change {true | false}]
[-auto_pairing_file fileName]
[-bump_use_oct_shape {true | false}]
[-check_bump_access_directions {true|false}]
[-compaction {true | false}]
[-connectPowerCellToBump {true | false}]
[-constraintFile fileName]
[-drop_via_on_all_geometries {true | false}]
[-drop_via_on_power_mesh layerName]
[-extraConfig fileName]
[-finger_direction {N|E|S|W}]
[-finger_max_width real_value]
[-finger_min_width real_value]
[-finger_target_mesh_layer_range {topLayer [bottomLayer]}]
[-honor_bump_connect_target_constraint {true | false}]
[-ignore_pad_type_check {true | false}]
[-layerChangeBotLayer layerName]
[-layerChangeTopLayer layerName]
[-lower_layer_prevent_45_routing {true | false}]
[-lower_layer_route_width value]
[-merge_nearby_pin {true | false}]
[-multi_pad_routing_style {default|serial|star}]
[-multipleConnection {multiPadsToBump | multiBumpsToPad | default}]
[-pg_mesh_direction {default | horizontal | vertical}]
[-pg_mesh_main_width real_value]
[-pg_mesh_max_width real_value]
[-pg_mesh_width value]
[-prevent_diagonal_wire_access_bumps {true | false}]
[-prevent_via_under_bump {true | false}]
[-prevent_via_under_bump_extension real_value]
[-route_pg_style {none | finger | mesh}]
[-route_style {manhattan | 45DegreeRoute}]
[-routeWidth real_value]
[-via_abut_bump {true | false}]
[-wire_to_pad_honor_min_spacing float_value]
```

Loads data for `placePIO` and `fcroute` commands. Both, `placePIO -optIOs -assignBump -noRandomPlacement` and `fcroute` commands use the same data for I/O optimization, bump assignment, and flip chip routing.

> You need to first execute `setFlipChipMode` command followed by `placePIO`, and `fcroute` commands.

Use the `getFlipChipMode` command to return the current settings for the `setFlipChipMode` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setFlipChipMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setFlipChipMode`
`-reset`	Resets parameters to their default values. The `-reset` parameter *must* be the first parameter specified. If you specify `-reset` by itself, the software resets all `setFlipChipMode` parameters to their default values. Any values that do not already match the default value and are reset will be reported. If you specify parameters after `-reset`, the software resets only those parameters to their default values.
`-allow_layer_change {true | false}`	

	Turns on the layer change feature, when set to `true`. Only the PIO mode supports this feature. Both Manhattan and 45-degree routing support this constraint.
	Different setting for routing layers will have the different behaviors as shown in the following examples. Assume the top RDL is `TOP_RDL` and the second top RDL is `2nd_RDL`:
	• Use `TOP_RDL` as much as possible and `2nd_RDL` is used only when a single layer cannot finish routing in case of cross-over:
	`-layerChangeBotLayer TOP_RDL -layerChangeTopLayer TOP_RDL`
	• Freely change layers so that the tool will use the layer resources by its intelligence
	`-layerChangeBotLayer 2nd_RDL -layerChangeTopLayer TOP_RDL`
	Default : `false`

`-auto_pairing_file` *fileName*	

	Makes `fcroute` dump out auto-pairing information. This enables you to check whether or not `fcroute` auto pairing and optimize pairing result based on the port number property.
	Depending on the design, the format of the output file can be as follows:
	• If the connection between the bump and the instance is known, the format is:
	`addBumpConnectTargetConstraint -bump bumpName -instName instName`
	• If the connection between the bump and the instance pin is known, the format is:
	addBumpConnectTargetConstraint -bump bumpName -instName instName -pinName pinName
	• If the connection between the bump and the pin port of the instance is known, the format is:
	`addBumpConnectTargetConstraint -bump bumpName -instName instName -pinName pinName -portNum portNumber`
	See Examples section for an example.

`-bump_use_oct_shape {true | false}`	

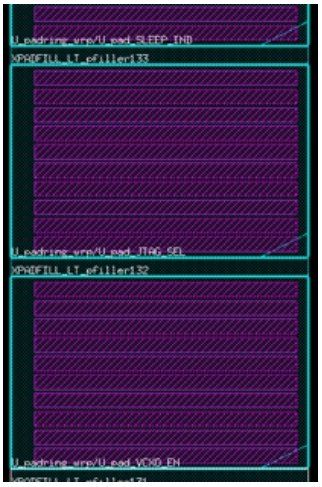
	Controls how `fcroute` handles special bumps with multiple shapes:
	• When this parameter is set to `true`, the tool uses an octagonal shape to mimic the special bump.
	• When this parameter is set to `false`, the tool:
	  ◦ Uses the bump's exact LEF description if the bump is simple enough.
	  ◦ May replace the bump shape with an octagonal shape if the bump is not simple according to the tool's internal criterion.
	Note: When the special bump is replaced with an octagonal shape, the routing quality may be improved, but there may be DRC violations.
	Default : `false`

`-check_bump_access_directions {true | false}`	
	Specifies whether `fcroute` should check the access direction of bump cells. Use this parameter if you have an octagonal bump pad structure with a pre-drawing wire for outside access, where the pre-drawing wire is a pin in the bump LEF. When `-check_bump_access_directions` is set to `true`, `fcroute` checks the bump LEF to detect the bump access direction before routing. Then, `fcroute` connects the pad to the bump pre-routed wire based on this direction. If the pre-routed wire is a 45-degree wire, the `manhattan` routing style is not supported. Note that one bump cell can have multiple access directions. **Note**: The `-check_bump_access_directions` parameter is applicable only for `fcroute -type signal`. If this option is used for `fcroute -type power`, `fcroute` will ignore the setting and display a warning. *Default*: `false`
`-compaction {true | false}`	
	Turns on compaction routing, when set to `true`. In the flip chip post-route stage, the compaction function pushes the routing as close as possible to bumps in order to leave more routing resource for subsequent routing steps, such as P/G bump connections or general power routing. **Note**: This feature cannot be used with resistance driven and diffpair routings because the compaction action may change the wire length and routing pattern. This feature does not support two-layer RDL routing. *Default* : `false`
`-connectPowerCellToBump {true | false}`	

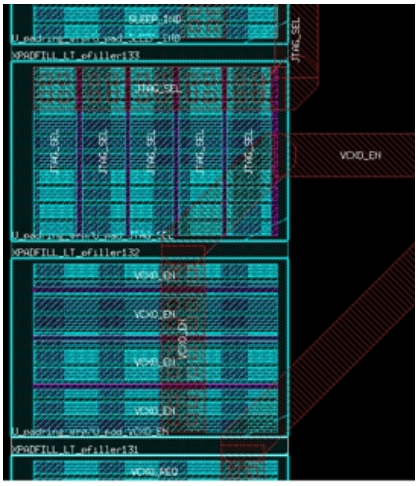
	Connects all power bumps to the I/O cell pin. It does not connect to a power or ground stripe or ring.
	When used with the `fcroute -type signal` parameter, you can connect power bumps to I/O power pads. To make this type of connection, you need the following information:
	• The Verilog netlist must contain the I/O power pad, for example:
	VDDCELL VDD_INST (.PAD());
	• The LEF macro pin must contain the USE POWER statement, for example:
	MACRO VDDCELL
	PIN PAD
	USE POWER;
	To improve the quality of routing in situations where there may be many power bumps in the design, you can pair power bumps to power cells by using this parameter in conjunction with the `fcroute -constraintFile` parameter. For example:
	`setFlipChipMode -connectPowerCellToBump true -constraintFile pair.const`
	where *pair.const* specifies the following information:
	`### Syntax: net pad [pad2] bump [bump2] ###`
	`PAIR`
	`VDD IOPADS_INST/esd Bump_90_9_8`
	`END PAIR`
	Default: `false`

`-constraintFile fileName`	
	Specifies the file that contains constraints for flip chip routing. For more information, see "*Useful Constraints for Flip Chip Routing*" in the Flip Chip Methodologies chapter of the *Innovus User Guide.*
`-drop_via_on_all_geometries {true | false}`	

Drops vias on all defined geometries. Consider the pin shape below.

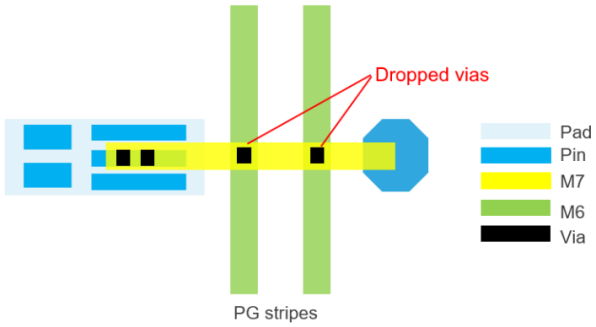


For the given shape, the routing result will be as follows when `-drop_via_on_all_geometries` is set to `true`:



Default: false

`-drop_via_on_power_mesh` *layerName*

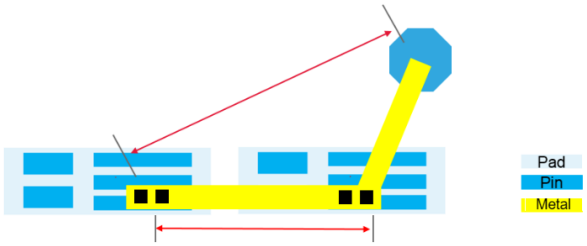
	Specifies the number or name of the metal layer that can be used by the tool for dropping a via on a PG mesh. `fcroute` automatically drops a via on the PG mesh on the specified metal layer if the PG RDL route crosses the PG mesh. For example:
	`setFlipChipMode -drop_via_on_power_mesh M6`
	`setFlipChipMode -connectPowerCellToBump true`
	`fcroute -type signal -designStyle pio`
	 
	
	The via is dropped only if there is no DRC violation due to the via drop.
	Note: The `-drop_via_on_power_mesh` parameter does not affect the routing results from PG bumps to pad.
	Default: `""`

`-extraConfig` *`fileName`*	Specifies the name of an extra configuration file that uses options defined in `sroute`.
`-finger_direction {N|E|S|W}`	
	Specifies the main expansion direction for PG finger routing. The default direction is North. *Default*: `N`
`-finger_max_width` *`real_value`*	
	Specifies the maximum routing width for one wire segment of the PG finger. By default, there is no constraint on the max width; `fcroute` uses `MAXWIDTH` in LEF. If this constraint is not specified in LEF, `fcroute` uses the maximum width possible. If there is a conflict between the user-specified constraint and the LEF rule, the LEF rule is used as the maximum width. *Default*: `0`
`-finger_min_width` *`real_value`*	

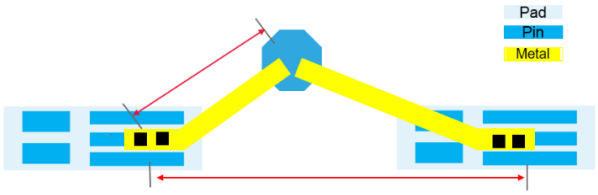
	Specifies the minimum routing width for one wire segment of the PG finger. The default is the RDL minimum width defined in LEF. If there is a conflict between the user-specified constraint and the LEF rule, the LEF rule is used as the minimum width. *Default*: `0`
`-finger_target_mesh_layer_range {`*`topLayer`* `[`*`bottomLayer`*`]}`	
	Specifies the target mesh layer range of the power mesh that will be connected for finger routing. If not specified, no layer of the power mesh will be connected. *Default*: ""
`-honor_bump_connect_target_constraint {true | false}`	
	Enables port numbering routing, when set to `true`. *Default*: `false`
`-ignore_pad_type_check {true | false}`	
	Specifies whether or not CLASS PAD cells are supported in the flip chip flow. When set to `true`, this option equals with following settings: • `set flipchip_allow_class_pad true` • `PADANYTYPE` in the `fcroute` constraint file • `srouteGrouteCheckPadClass FALSE` in the `fcroute` extra configuration file. *Default*: `true`
`-layerChangeBotLayer` *`layerName`*	
	Specifies the bottom-most metal layer that the software can use when routing bumps. You can specify the layer either by layer number or layer name.
`-layerChangeTopLayer` *`layerName`*	
	Specifies the top-most metal layer that the software can use when routing bumps. You can specify the layer either by layer number or layer name.
`-lower_layer_prevent_45_routing {true | false}`	
	Prevents 45-degree routing for lower layer, when set to `true`. *Default*: `false`
`-lower_layer_route_width` *`value`*	
	Specifies a different width value for the lower routing layer. The unit is in microns. *Default* = 0, *Min* = 0.000000, *Max* = 10000000000.000000
`-merge_nearby_pin {true | false}`	
	Merges pins that are close but not touching. *Default*: `false`
`-multi_pad_routing_style {default | serial | star}`	
	Specifies the routing style for multi-pad routing:

- `default` - The flip chip router (`fcroute`) connects a bump to multiple pads with multiple different wires as shown below:

- `serial` - `fcroute` connects a bump first to one pad and then to the other pad, as shown below, provided that the distance from the bump to either pad is more than the distance between the two pads:

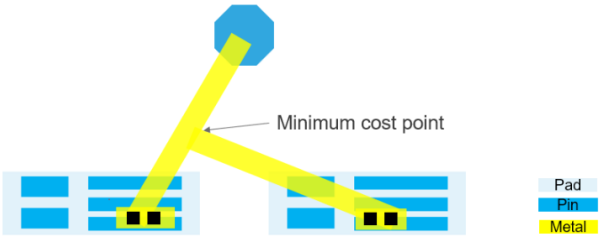


If the distance from the bump to any pad is shorter than the distance between the two pads, `fcroute` reverts to the default behavior of multiple wire connection even if `-multi_pad_routing_style` is set to `serial`.



This mode supports only two-pad routing.

- `star` - `fcroute` connects a bump first to the closest pad. It then finds a minimum cost point on its routing path and connects this point to another pad as shown below:



The minimum cost point could be anywhere on the first routing path, including its beginning and end.

Both the `serial` and `star` modes are useful for saving routing resources in complex designs as they share the routing between pads. However, the `serial` mode supports only two-pad routing. Use the `star` mode if you want to route more than two pads.

Note: The `setFlipChipMode -multi_pad_routing_style` works only if the `setFlipChipMode -multipleConnection multiPadsToBump` is specified. For example, to use `star` mode, you should specify the following commands before running `fcroute`:

```
setFlipChipMode -multipleConnection multiPadsToBump
setFlipChipMode -multi_pad_routing_style star
```

Default: `default`

`-multipleConnection {multiPadsToBump | multiBumpsToPad | default}`

Specifies routing connections between multiple pads and bumps.

- `multiPadsToBump` - Enables routing from multiple pads to one bump in parallel. This parameter is supported by both, `fcroute` AIO and PIO routing styles.
- `multiBumpsToPad` - Enables routing from multiple bumps to one pad in parallel. This parameter is supported by both, `fcroute` AIO and PIO routing styles.
- `default` - Enables routing from one bump to one pad in parallel. This means there is no multiple connection.

Default: `default`

`-pg_mesh_direction {default | horizontal | vertical}`

Specifies the main routing direction of the PG mesh. You can specify one of the following values for this option:

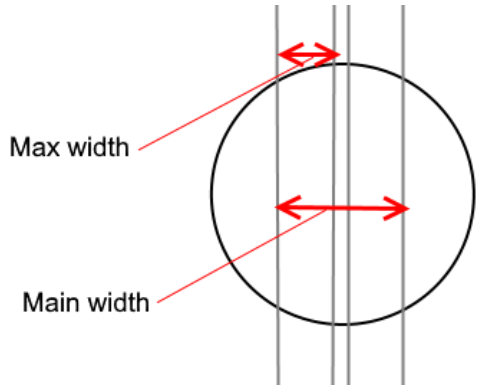
- `default` - Uses the preferred routing direction defined in the Tech LEF.
- `horizontal` - Uses the horizontal routing direction.
- `vertical` - Uses the vertical routing direction.

The `-pg_mesh_direction` parameter can be used only if `-route_pg_style mesh` is specified.

Default: `default`

`-pg_mesh_main_width` *`real_value`*

Specifies the main width of the PG mesh. This option can be used only if `-route_pg_style mesh` is specified.

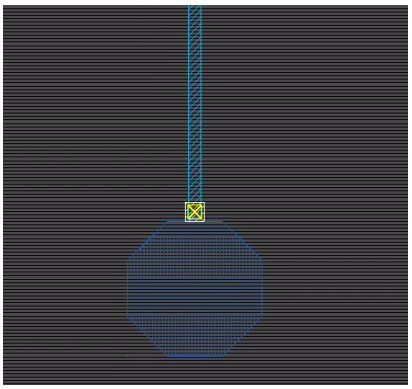
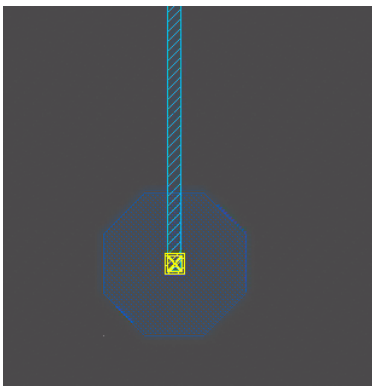


Default: `0`; *Min*: `0`; *Max*: `10000000000`

`-pg_mesh_max_width` *`real_value`*

	Specifies the maximum width of the routing wire in a PG mesh. This option can be used only if `-route_pg_style mesh` is specified. By default, there is no constraint on the maximum width. `fcroute` uses `MAXWIDTH` in the Tech LEF. If there is a conflict between the value you specify and the LEF rule, the tool uses the LEF rule as the maximum width. *Default*: `0`; *Min*: `0`; *Max*: `10000000000`
`-pg_mesh_width` *value*	Specifies the exact routing width of the PG mesh. This option can be used only if `-route_pg_style mesh` is specified. If only one value is specified with `-pg_mesh_width`, the value is applicable for the width of both horizontal and vertical wires. If two values are specified, the first value is applicable for the width of horizontal wires and the second value is applicable for the width of vertical wires. *Default:* `""`
`-prevent_diagonal_wire_access_bumps {true | false}`	
	Specifies whether or not diagonal wires can be used to access bumps. *Default:* `false`
`-prevent_via_under_bump {true | false}`	
	Specifies that vias are generated next to the bump instead of directly on the bump. If you do not specify this parameter, vias generated connect directly to the bump. This option is available for both signal and power bumps (`fcroute -type signal` and `fcroute -type power`, respectively.) *Default:* `false`
`-prevent_via_under_bump_extension` *real_value*	
	Specifies a spacing value to define an extension area around the bump in which vias should not be dropped. The spacing value must be greater than or equal to `0`. This parameter can be used only if `-prevent_via_under_bump` is set to `true`. Check the **Examples** section to see how this parameter can be used in conjunction with `prevent_via_under_bump` to control the spacing around bumps when vias are dropped. *Default:* `0`
`-route_style {manhattan | 45DegreeRoute}`	
	Determines the type of routing, either `manhattan` or `45DegreeRoute`. *Default*: `45DegreeRoute` **Note:** `45DegreeRoute` does not work in conjunction with the following `fcroute` parameter: • `-doubleBendRoute`
`-route_pg_style {none | finger | mesh}`	

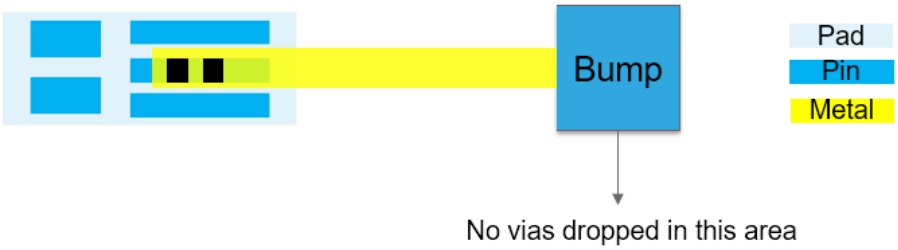
	Specifies the routing pattern for `fcroute -type power` routing: • `none`: For normal PG routing • `finger`: For PG finger routing style, in which you need to specify the routing direction. This option must be used with `fcroute -area` option. • `mesh`: For mesh-style PG routing on the redistribution layer (RDL). This option is used to increase PG density. If you specify PG mesh routing, you must specify the main width of the PG mesh by using the `-pg_mesh_main_width` parameter. *Default:* `none`
`-routeWidth` *`real_value`*	Specifies the routing width. Wires are connected to bumps using the specified route width not wider than the I/O pad pin. *Default*: 0
`-via_abut_bump {true | false}`	

	Specifies that via should abut the bump only if routing can be done successfully. • With `-via_abut_bump` set to `true`  • Without `-via_abut_bump` set to `true`  In this case, via is dropped inside bump as shown above. This is the default setting. *Default*: `false`
`-wire_to_pad_honor_min_spacing` *`float_value`*	
	Specifies the minimum spacing between the bump/pad and a wire. *Default*: 0

Examples

- The following examples illustrate how the `-prevent_via_under_bump` and `-prevent_via_under_bump_extension` parameters can be used to control the spacing around bumps when vias are dropped:
  - In the example below, no vias are dropped directly under the bump as `-prevent_via_under_bump` is set to `true`:

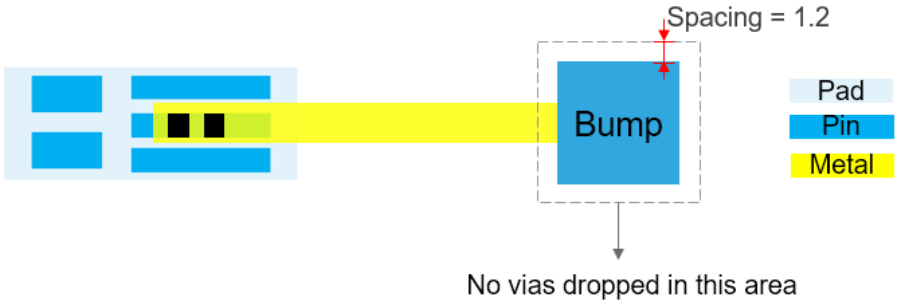
```
> setFlipChipMode -prevent_via_under_bump true
> fcroute -type signal -designStyle pio
```



Additionally, if `-prevent_via_under_bump_extension` is set to `0`, the same result is achieved.

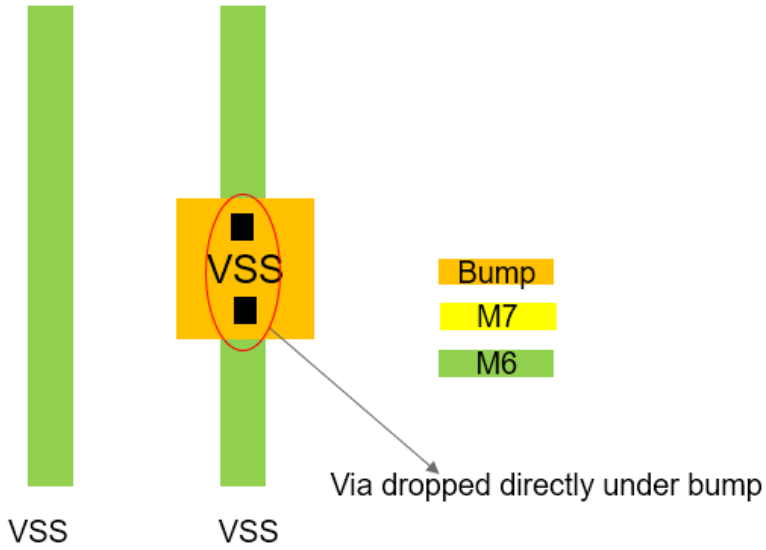
- In the example below, no vias are dropped in the extension area defined around the bump:

```
> setFlipChipMode -prevent_via_under_bump true
> setFlipChipMode -prevent_via_under_bump_extension 1.2
> fcroute -type signal -designStyle pio
```



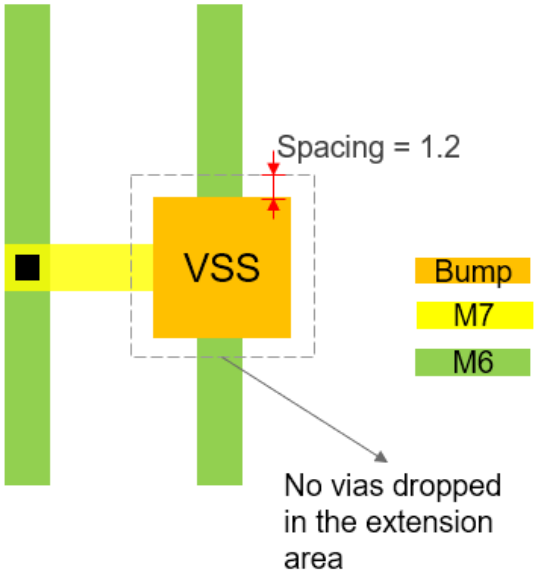
- In the example below, vias are dropped directly under the power bumps as `-prevent_via_under_bump` is set to `false`:

```
> setFlipChipMode -prevent_via_under_bump false
> fcroute -type power
```



- In the example below, no vias are dropped in the extension area defined around the power bumps:

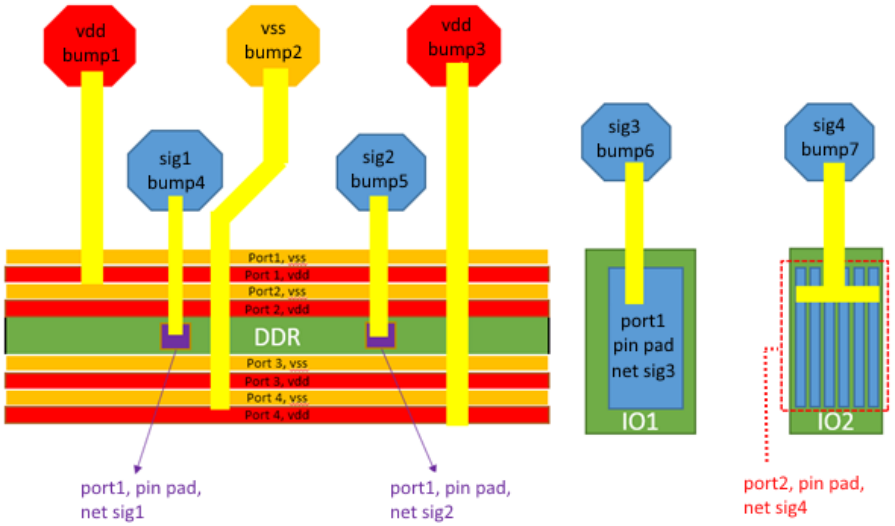
```
> setFlipChipMode -prevent_via_under_bump true
> setFlipChipMode -prevent_via_under_bump_extension 1.2
> fcroute -type power
```



- The following command specifies the flip chip constraints in the constraint file `CFG/ res.constr` enabling routing from a single bump to single power cell in parallel, having a route width of `10` micrometers on the metal layer `8`:

```
setFlipChipMode -connectPowerCellToBump true \
-constraintFile CFG/res.constr \
-extraConfig CFG/ex.cfg \
-layerChangeBotLayer 8 \
-layerChangeTopLayer 8 \
-multipleConnection default \
-routeWidth 10
```

- Consider the connection displayed below.

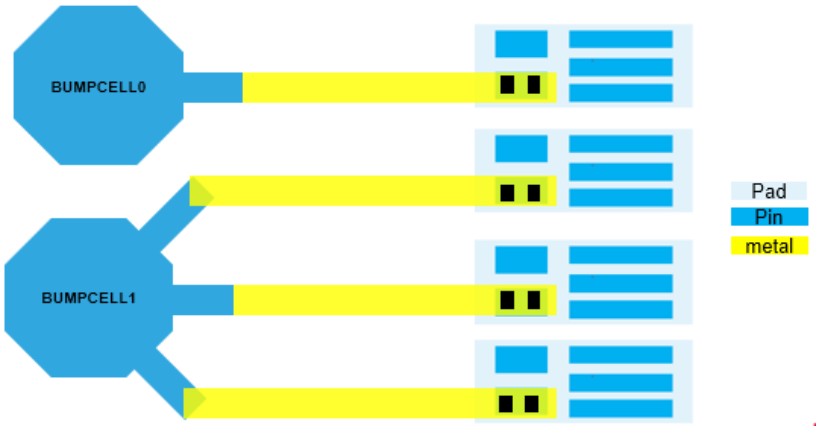


For this connection, the `setFlipChipMode -auto_pair_file pair.rpt` command generates the following report:
In `pair.rpt` file:

```
addBumpConnectTargetConstraint -bump bump1 -instName DDR -pinName vdd -portNum 1
addBumpConnectTargetConstraint -bump bump2 -instName DDR -pinName vss -portNum 4
addBumpConnectTargetConstraint -bump bump3 -instName DDR -pinName vdd -portNum 4
addBumpConnectTargetConstraint -bump bump4 -instName DDR -pinName pad -portNum 1
addBumpConnectTargetConstraint -bump bump5 -instName DDR -pinName pad -portNum 1
addBumpConnectTargetConstraint -bump bump6 -instName IO1 -pinName pad -portNum 1
addBumpConnectTargetConstraint -bump bump7 -instName IO2 -pinName pad -portNum 2
```

- In the example below, `fcroute` checks the bump LEF to detect the bump access directions and then connects the pads to the bump pre-routed wires based on the detected directions:

```
> setFlipChipMode -check_bump_access_directions true
> fcroute -type signal -designStyle pio
```



Related Information

- `addBumpConnectTargetConstraint`
- `getFlipChipMode`
- `fcroute`
- `placePIO`
- `sroute`
- `verifyFlipChipRoutingConstraints`
- Flip Chip Methodologies chapter in the *User Guide*
- Tools Menu chapter in the *Menu Reference*
  - *Flip Chip*
  - *TSV*

setProbePin

```
setProbePin
[-help]
{{-instName instName -pinName pinName}
 | -bump bumpName}}
```

Sets a pad or bump as a probe point. Probe points are used for package-level design and testing. To delete or report on probe points, use the `unsetProbePin` or `reportProbePins` commands respectively.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setProbePin` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setProbePin`
`-bump bumpName`	Specifies the instance name of the bump.
`-instName InstName`	Specifies the name of the I/O instance.
`-pinName pinName`	Specifies the name of the pin on the I/O instance that will be set as a probe point.

Example

- The following command sets pin `PAD` of I/O instance `Bump_1` as a probe point:

  ```
  setProbePin -instName Bump_1 -pinName PAD
  ```

Related Information

- `reportProbePins`
- `unsetProbePin`

setSpecialRouteType

```
setSpecialRouteType
[-help]
-stopTypeList swire_type_list
[-stopLayer Metal_layer]
-setType
{BLOCKRING|BLOCKAGEWIRE|BLOCKWIRE|COREWIRE|DRCFILL|FILLWIREOPC|FILLWIRE|FOLLOWPIN|IOWIRE|NOTYPE|PADRING|RING|STRIPE}
{-bumpCell bumpCellList | -instPin {{inst1 pin1} {inst2 pin2}...}}
```

Sets special routing wires to a particular shape type so that they can be written out into a DEF file using the `-specialShape` parameter of the `defOutBySection` command. This makes data filtering easier by enabling customized selection and output of special routing shape types. The output data is very specific and can be easily managed as storage size is small.

The `setSpecialRouteType` command works by tracing special wires and vias that touch bumps or instance pins through to a stop-layer or special wire stop type. All the traced special wires and vias are marked with a particular wire type. The `defOutBySection -specialShape` command is then used to write out the special type routes and vias.

Note: Use the `setSpecialRouteType` and `defOutBySection` commands in sequence. Save the design before you modify the shape types so that you can recover the original design with `defIn`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setSpecialRouteType` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setSpecialRouteType`
`-bumpCell` *`bumpCellList`*	Specifies the list of bump cells. If this option specified, the tracing starts from any special wires and vias that touch the named bump cells. **Note:** This option cannot be used with `-instPin`.
`-instPin` `{{`*`inst1 pin1`*`}` `{`*`inst2 pin2`*`}...}`	Specifies a list of pairs of instances and their pins. If this option specified, the tracing starts from any special wires and vias that touch the named instance pin pairs. **Note:** This option cannot be used with `-bumpCell`.
`-setType` `{BLOCKRING|BLOCKAGEWIRE|BLOCKWIRE|COREWIRE|DRCFILL|FILLWIREOPC|FILLWIRE|FOLLOWPIN|IOWIRE|NOTYPE|PADRING|RING|STRIPE}`	
	Sets the marked special wire to the specified wire type (DEF SPECIAL ROUTE SHAPE type). *Default*: `IOWIRE`
`-stopLayer` *`Metal_layer`*	Specifies the metal layer at which tracing should stop.
`-stopTypeList` *`swire_type_list`*	Specifies the special-wire types used in tracing. When a special wire with a shape type that is not included in the list is encountered, tracing stops. The legal list of *`swire_type_list`* are the DEF SPECIAL ROUTE SHAPE types: `BLOCKRING`, `BLOCKAGEWIRE`, `BLOCKWIRE`, `COREWIRE`, `DRCFILL`, `FILLWIREOPC`, `FILLWIRE`, `FOLLOWPIN`, `IOWIRE`, `NOTYPE`, `PADRING`, `RING`, `STRIPE`.

Example

- The following command traces special wires and vias that touch the named bump cells. Tracing stops when a special wire of type other than `IOWIRE` is encountered. The marked special wire is set to `IOWIRE`.

```
setSpecialRouteType -bumpCell {bc*} -stopTypeList {IOWIRE} -setType IOWIRE
```

Related Information

- defIn
- defOutBySection

spaceBondPad

```
spaceBondPad
[-help]
[-spread]
```

Spaces bond pads and the corresponding area I/O instances based on the constraints returned by the `getCellDist()` Tcl script.

The command distributes excess space until all the bond pads reach their maximum spacing constraint returned by the `getCellDist()` Tcl script or until the time when no extra space is left between the instances.

The initial distribution of space occurs for the clusters of instances that have the first or the last bond pads in the row. The remaining iterations equally distribute the remaining space between each I/O. The I/O instances that are in the end are placed at minimum spacing to the die edges, both on the left and right sides of the die.

The I/O pads will snap to I/O sites and the corresponding bond pads (bumps) will shift accordingly.

You can define the constraints in the `getCellDist()` Tcl script as follows:

```
proc getCellDist { xp yp xn yn padPosition padCount side ioRow staggerPos ioName }
```

Here:

- *xp yp* - Specifies the x and y coordinates of a previous bond pad having the same stagger position. If first, the location is 0, 0.
- *xn yn* - Specifies the x and y coordinates of the previous neighboring bond pad having any stagger position. If first, the location is 0, 0.
- *padPosition* - Specifies the position of the bond pad. The position can be i (inner), m (middle), or o (outer).
- *padCount* - Specifies the current bond pad count for bond pads at the same stagger position. For example, 1 for the first inner bond pad, 2 for the second inner bond pad, and so on.
- *side* - Specifies the n (North), s (South), e (East), or w (West) side of the bond pad.
- *ioRow* - Specifies the I/O row.
- *staggerPos* - Specifies the stagger type.
- *ioName* - Specifies the name of the I/O.

Note:

- The bond pad spacing for North and South is done from left to right.
- The bond pad spacing for East and West is done from bottom to top.
- The bond pad spacing for each side and ring is done independently.

After using the `spaceBondPad` command, you can use the `checkBondPadSpacing` command to check the bond pad spacing.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `spaceBondPad` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man spaceBondPad`
`-spread`	Removes any extra spacing that may exist between I/O cells, which were placed based on the rules defined in the `getCellDist()` Tcl script, and evenly distirbutes the extra space between all of the I/O cells in that row. You can use this parameter if there are large gaps (spaces) between I/O cells.

Related Information

- checkBondPadSpacing
- fixBondPad
- getBondPad
- placeBondPad
- staggerBondPad
- unfixBondPad

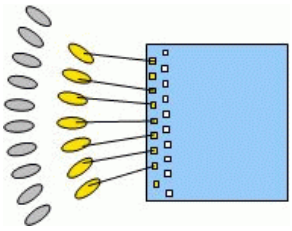
staggerBondPad

```
staggerBondPad
[-help]
{-startIoInstName instName [-endIoInstName instName] | -ring number |
-side {e | w | n | s} [-ring number] | -all}
{-pattern pattern_string | -startStaggerPosition {i | m | o}}
[-pad position padName position padName...]
[-pinName pinName]
```

Generates a staggered wirebond pad pattern on top of an existing I/O ring. In this design methodology, bond pads are separate LEF macro definitions from the I/O driver cells, but are placed on top of the I/O driver cells to form the combined bond pad and I/O driver circuitry.

The command only supports `CLASS PAD` instances.

To produce the staggered pattern, multiple bond pad cells are defined in the LEF for each I/O cell, each with its bond pad in a specific location corresponding to an outer, middle, or inner stagger position. The bond pads are defined such that when placed on the I/O cell, any one of them will connect to the I/O cell pin with no or minimal RDL routing.



Innovus generates a stagger pattern by selecting the outer, middle, or inner stagger bond pad to place on each I/O cell based on the location of that I/O instance in the I/O ring and the pattern that is being generated. If the order of the I/O cells is changed, or if cells are added or deleted, the stagger pattern may be regenerated. Because the bond pad macros are separately defined, they can be easily swapped with bond pads of different stagger positions without changing the logic of the design.

To define the relationship between the bond pads and I/O cells in the LEF file,
the following properties must be defined in each LEF file where they are referenced:

```
PROPERTYDEFINITIONS

     PIN bondPadOuter STRING;

     PIN bondPadMiddle STRING;

     PIN bondPadInner STRING;

     PIN ioCellOriginX REAL;

     PIN ioCellOriginY REAL;

     MACRO ioCellOffsetX REAL;

     MACRO ioCellOffsetY REAL;

 END PROPERTYDEFINITIONS
```

The LEF macros define the property values as follows:

```
MACRO IOCELL

     CLASS PAD ;

     ...

     SIZE 50.0 BY 225 ;

     ...

     PIN PAD
```

```
        PROPERTY bondPadOuter "BUMPA BUMPA1 BUMPD" ;
        PROPERTY bondPadMiddle "BUMPB" ;
        PROPERTY bondPadInner "BUMPC" ;
        PROPERTY ioCellOriginX 25.0 ;
        PROPERTY ioCellOriginY 0.0 ;
    END PAD
 END IOCELL


MACRO BUMPA
    CLASS COVER BUMP
    ...
    PROPERTY ioCellOffsetX -40 ;
    PROPERTY ioCellOffsetY -70 ;
 END BUMPA
```

Innovus automatically names the stagger bond pads as:

*IoCellInstName*_PAD*#*

Here:

- *IoCellInstName* - Specifies the instance name of the I/O cell on which the bond pad is being placed.
- *#* - Specifies the incremented bond pad number. The number `1` is used for all single pad I/O cells. This number is incremented when there are multiple bond pads on a single I/O cell.
  For example, if I/O cell `CKdiff` contained two pins, two bond pads would be placed on this cell and named as `CKdiff_PAD1` and `CKdiff_PAD2`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `staggerBondPad` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man staggerBondPad`
`-all`	Specifies that bond pads will be placed on all I/O instances.
`-endIoInstName` *instName*	Specifies the name of the last I/O instance of where bond pads will be placed. You can use this parameter in conjunction with the `-startIoInstName` parameter. **Note:** Bond pads are placed in a clock-wise pattern from the starting I/O instance (`-startIoInstName`) to the last I/O instance (`-endIoInstName`).
`-pad` *positionpadNamepositionpadName...*	Specifies which bond pad and position to use when multiple bond pads are specified in the LEF file for a given inner (i), middle (m), or outer (o) position. For example, If there are three outer bond pads specified in the LEF file, you can use this parameter to choose which bond pad to use as the outer bond pad. *Default*: The software uses the first bond pad in the list of bond pads for a given position.
`-pattern` *pattern_string*	Specifies the stagger pattern of the bond pads on the I/O instances. *pattern_string* specifies any combination of `i` (inner), `m` (middle), and `o` (outer).
`-pinName` *pinName*	Specifies the name of the pin inside the instance. If the instance specified by the `-startIoInstName` parameter contains multiple pins, you use this parameter to specify which pin to start with.
`-ring` *number*	Specifies which ring to operate on with ring 1 being the outer most ring. The ring number is incremented by one as you get closer to the core. For a three-ring design, the inner most ring is ring 3.
`-side {e | w | n | s}`	Specifies the geographical side of the die. You can specify the East (`e`), West (`w`), North (`n`), or South (`s`) side of the die.
`-startIoInstName` *instName*	Specifies the name of the first I/O instance of where bond pads will be placed. You can use this parameter in conjunction with the `-endIoInstName` parameter. If multiple pins exist in the instance, you can use the `-pinName` parameter to specify which pin to start with. **Note:** Bond pads are placed in a clock-wise pattern from the starting I/O instance (`-startIoInstName`) to the last I/O instance (`-endIoInstName`) if applicable.
`-startStaggerPosition {i | m | o}`	Specifies the starting stagger position of the specified bond pad (`-pad`) on the specified I/O instance (`-startIoInstName`). You can start the stagger position of the bond pad using the inner (`i`), middle (`m`), or outer (`o`) position. The default stagger position sequence is inner (`i`), middle (`m`), and outer (`o`). If you start with a stagger position of middle (`m`), the next bond pad is placed in an outer (`o`) stagger position followed by a bond pad placed in an inner (`i`) stagger position.

Example

- The following command places bond pads on all I/O instances using a stagger pattern of inner, outer, and middle and using bond pad `BUMPA1` at the outer position:

  ```
  staggerBondPad -all -pattern iom -pad o BUMPA1
  ```

- The following command places bond pads on the I/O instances of the outer most ring (East side) starting with a bond pad placed in the middle stagger position:

  ```
  staggerBondPad -side e -startStaggerPosition m -ring 1
  ```

- The following command places bond pads from I/O instance `abc` to I/O instance `xyz`, starting with pin `PAD` of I/O instance `abc` and using an inner, inner, outer stagger pattern:

  ```
  staggerBondPad -startIoInstName abc -endIoInstName xyz -pinName PAD -pattern iio
  ```

Related Information

- `checkBondPadSpacing`
- `fixBondPad`
- `getBondPad`
- `placeBondPad`
- `spaceBondPad`
- `unfixBondPad`

swapSignal

```
swapSignal
[-help]
[[-from {nameList} -to {nameList}] | -circle_offset value]
```

Swaps signal between selected instances (I/O pads or bumps).

To swap bump assignments in a circle, determine the value of offset you want to use. Assuming that the value of offset is `N`, the number of the selected bumps is `M`, the order of the input bump is `$index`, and the name of the bump with `$index` is `bump_$index`, you can swap bump assignments in a circle by using the the following command:

```
swapSignal -from {bump_1 bump_2 bump_3 ... bump_$index} -to {bump_[1+N] bump_[2+N] bump_[3+N]... bump_[$index+N-M]}
```

Parameters

`-help`	Prints a brief description of the `swapSignal` command. For a detailed description of the command, use the `man` command: `man swapSignal`
`-circle_offset value`	Specifies the offset value to swap bumps in circle. The value should be be positive and less than the number of the selected bumps. For example, if you select the bumps bump1, bump2, bump3, bump4 and bump5 in the GUI and specify `-circle_offset` as 2, the result of signal swapping will be as follows: bump1(S1) bump2(S2) bump3(S3) bump4(S4) bump5 (S5) -> bump1(S4) bump2(S5) bump3(S1) bump4(S2) bump5 (S3)
`-from nameList`	Specifies the names of bumps from which signal is to be swapped.
`-to nameList`	Specifies the names of bumps to which signal is to be swapped.

Examples

- The bumps `Bump_74_3_7` and `Bump_23_2_2` are selected. The following command swaps the signal between them:
  `swapSignal`
- The bumps `Bump_100_9_9`, `Bump_90_9_8`, `Bump_80_9_7`, and `Bump_70_9_6` are selected. The following command swaps the bump assignments between them in a circle with an offset value of `1`:
  `swapSignal -circle_offset 1`

Related Information

- `assignBump`
- `unassignBump`
- `viewBumpConnection`
- Flip Chip Methodologies chapter in the *User Guide*
- Tools Menu chapter in the *Menu Reference*
  - *Flip Chip*

unassignBump

```
unassignBump
[-help]
{-allBumps | -byBumpName {list} | -byBumpSite {list} | -selected}
```

Removes the Verilog signal names from bumps, which removes the connection between bumps and I/O pins. The original connection will remain in the Verilog and is connected when the bump is reassigned. If the bump is assigned, it will appear in the DEF `SPECIALNETS` section with the signal (net) name. Use this command after bumps are assigned with the `assignBump` command or defined in the I/O file with the signal name, but without the `-fixed` constraint.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `unassignBump` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man unassignBump`
`-allBumps`	Removes all the connections between bumps and I/O pins and removes the signal assignment of the bumps.
`-byBumpName list`	Removes only the connections between bumps and I/O pins contained in the bump name list.
`-byBumpSite list`	Removes only the connections between bumps and I/O pins contained in the site list.
`-selected`	Specifies to unassign the bump(s) selected in the design in the GUI.

Examples

- The following command unassigns all of the bumps:
  ```
  unassignBump -allBumps
  ```
- The following command unassigns the (named) bumps `bumpA` and `bumpB`:
  ```
  unassignBump -byBumpName {bumpA bumpB}
  ```

Related Information

- `assignBump`
- `checkBump`
- `deleteBumps`
- Flip Chip Methodologies chapter of the *User Guide*
- Tools Menu chapter of the *Menu Reference*
  - *Flip Chip*

unfixBondPad

```
unfixBondPad
[-help]
{-ioInstName InstName [-pinName pinName] | -selected}
```

Unassigns the fixed status of a specified bond pad.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `unfixBondPad` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man unfixBondPad`
`-ioInstName` *instName*	Specifies the name of the I/O instance whose bond pad's fixed status will be unassigned.
`-pinName` *pinName*	Specifies the name of the pin.
`-selected`	Unassigns the fixed status of a selected bond pad. You must first select the bond pad in the design display area, then use this parameter to unassign the fixed status.

Example

- The following command unassigns the fixed status of the bond pad on I/O instance `pad_addr[30]`:

  ```
  unfixBondPad -ioInstName {pad_addr[30]}
  ```

Related Information

- `checkBondPadSpacing`
- `fixBondPad`
- `getBondPad`
- `placeBondPad`
- `spaceBondPad`
- `staggerBondPad`

unfixBump

```
unfixBump
[-help]
{-allBumps | -byBumpName {list}}
```

Removes the bump assignment status from a specified list of assigned bumps that were set with the `setBumpFixed` command. The software updates the DEF file by removing the `+FIXEDBUMP` attribute from the specified group of assigned bumps to indicate that they are now in an unfixed state.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `unfixBump` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man unfixBump`
`-allBumps`	Removes the bump assignment status from all bumps.
`-byBumpName {list}`	Removes the bump assignment status from a specified list of assigned bumps.

Examples

- The following command removes the bump assignment status from all bumps:
  ```
  unfixBump -allBumps
  ```
- The following command removes the bump assignment status from bumps `bumpA` and `bumpB`:
  ```
  unfixBump -byBumpName {bumpA bumpB}
  ```

Related Information

- `assignBump`
- `defOut`
- `setBumpFixed`
- Flip Chip Methodologies chapter in the *User Guide*
- Tools Menu chapter in the *Menu Reference*
  - *Flip Chip*

unsetProbePin

```
unsetProbePin
[-help]
{-instName InstName -pinName pinName}
| -bump bumpName
```

Unsets a pad or bump as a probe point. Probe points are used for package-level design and testing. To set or report on probe points, use the `setProbePin` or `reportProbePins` commands respectively.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `unsetProbePin` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man unsetProbePin`
`-bump bumpName`	Specifies the instance name of the bump.
`-instName InstName`	Specifies the name of the I/O instance.
`-pinName pinName`	Specifies the name of the pin on the I/O instance that will be unset as a probe point.

Example

- The following command unsets the `PAD` pin of `Bump_1` as a probe point:

  ```
  unsetProbePin -instName Bump_1 -pinName PAD
  ```

Related Information

- `reportProbePins`
- `setProbePin`

verifyFlipChipRoutingConstraints

```
verifyFlipChipRoutingConstraints
[-help]
[-routeStyle {manhattan | 45DegreeRoute}]
```

Reports the status of bump placement and routing contsraints. Use this command to check whether the flip chip routing result meets the settings in the constraints file.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `verifyFlipChipRoutingConstraints` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man verifyFlipChipRoutingConstraints`.
`-routeStyle {manhattan | 45DegreeRoute}`	Specifies the route style to check, which could be `manhattan` or `45DegreeRoute`.

Example

- The following command checks for flip chip routing violations of the Manhattan route style:
  ```
  verifyFlipChipRoutingConstraints -routeStyle manhattan
  ```

Related Information

- `fcroute`
- `getFlipChipMode`
- `setFlipChipMode`
- Flip Chip Methodologies chapter of the *User Guide*
- Tools Menu chapter of the *Menu Reference*
  - *Flip Chip*

viewBumpConnection

```
viewBumpConnection
[-help]
[-honor_color]
[-multiBumpsToPad]
[-multiPadsToBump]
[-remove]
[[[-net net_list] [-bump bump_list] [-io_inst io_inst_list] [-selected]]|[[-bumpType {all | power | signal}] [-target
{pad | ring}]]]
```

Displays the connection as a flightline between a power/ground (PG) bump and a PG I/O pad, during signal bump assignment. You can remove the flightlines between the bumps and I/O pads by using `viewBumpConnection -remove`.

For both signal and power pins, `viewBumpConnection`, by default, displays a long pin connection with the correct location, instead of the center of the pin.

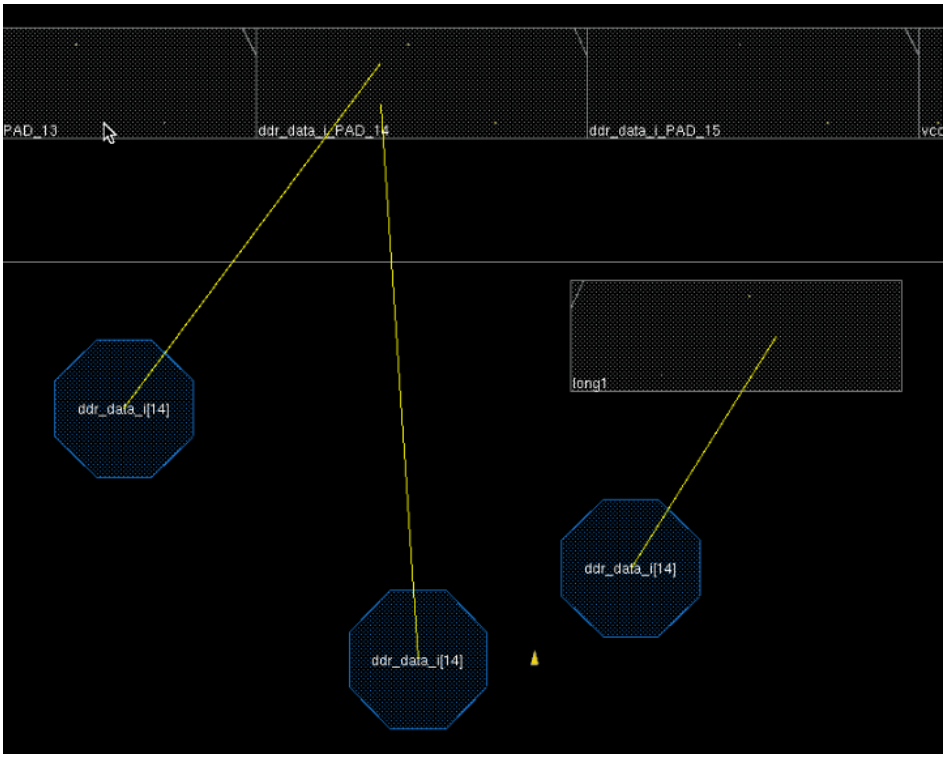
Note: `viewBumpConnection` honors the port tag information in bumps for both signal and power flightline for multiple pad and multiple bump routing.

Parameters

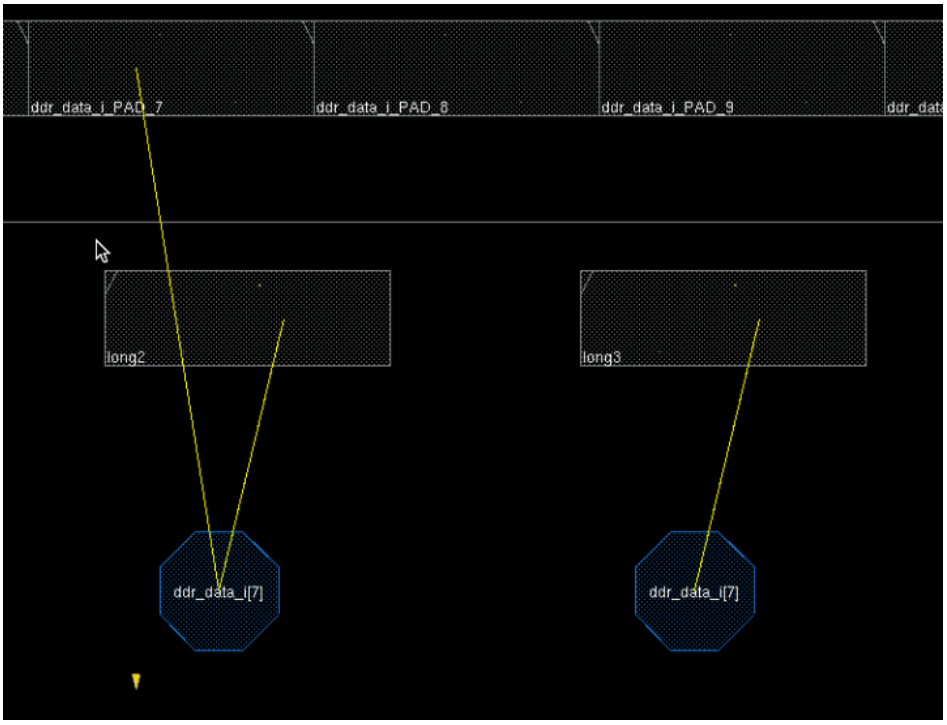
`-help`	Prints a brief description that includes type and default information for each `viewBumpConnection` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man viewBumpConnection`
`-bump` *`bump_list`*	Specifies bumps for which flightlines are to be displayed.
`-bumpType {all | power | signal}`	
	Specifies the connecting bump type, which could be: • `all` - All the bump types • `power` - Power bump • `signal` - Signal bump *Default:* `all`
`-honor_color`	Colors flightlines according to the settings defined with the `ciopLoadBumpColorMapFile` command.
`-io_inst {`*`io_inst_list`*`}`	
	Specifies IO instances or blocks for which flightlines are to be displayed. Specified IO instances should be CLASS PAD AREAIO instances. Specified blocks should have at least one pin port in CLASS BLOCK cell containing the CLASS BUMP property.
`-multiBumpsToPad`	
	Displays connections from multiple bumps to one pad. Note: If neither `-multiBumpsToPad` or `-multiPadsToBump` is specified, the `viewBumpConnection` command will display only one-to-one connections.
`-multiPadsToBump`	
	Displays connections multiple pads to one bump. Note: If neither `-multiBumpsToPad` or `-multiPadsToBump` is specified, the `viewBumpConnection` command will display only one-to-one connections.
`-net` *`net_list`*	Specifies nets for which flightlines are to be displayed.
`-remove`	Removes the bump connections.
`-selected`	Displays flightlines for selected bumps and IO instances in bold.
`-target {pad | ring}`	
	Specifies the connecting target, which could be a `pad` or a `ring`.

Examples

- The following command displays flightlines for bumps `Bump_1` and `Bump_2`:
  ```
  viewBumpConnection -bumps {Bump_1 Bump_2}
  ```
- The following command displays connections from multiple bumps to one pad:
  ```
  viewBumpConnection -multiBumpsToPad
  ```



- The following command displays connections from multiple bumps to one pad:
  `viewBumpConnection -multiPadsToBump`



Related Information

- `ciopLoadBumpColorMapFile`
- `fcroute`
- `setFlipChipMode`
- Flip Chip Methodologies chapter of the *User Guide*
  - *Viewing Flip Chip Flightlines*
- Tools Menu chapter of the *Menu Reference*
  - *Flip Chip*

writeFlipChipProperty

```
writeFlipChipProperty
[-help]
fileName
```

Saves the bump and pad properties in the bump and IO pad property file. The property file is a manually editable text file.

Parameters

`-help`	Prints out the command usage that includes type and default information for each `writeFlipChipProperty` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man writeFlipChipProperty`
`fileName`	Specifies the bump and pad property file name.

Example

The following command saves the bump and pad properties in a file named `test2`:

```
writeFlipChipProperty test2
```

Related Information

- `readIoUpdate`
- `readFlipChipProperty`
- Flip Chip Methodologies chapter of the *User Guide*
- Tools Menu chapter in the *Menu Reference*
  - *Flip Chip*

Flip Chip Global Variables

- flipchip_allow_routed_bump_edit

flipchip_allow_routed_bump_edit

```
flipchip_allow_routed_bump_edit {0 | 1}
```

Type: Integer

Default: `1`

Specifies whether the tool should check routing status of bumps before allowing manipulation. By default, the tool ignores routing status and allows manipulation of routed bumps. Manipulation actions include moving, deleting, re-assigning, and swapping. Set this variable to `0` if you want the tool to check routing status of bumps and disallow manipulation of routed bumps.

The `flipchip_allow_routed_bump_edit` variable directly affects `dbSetFPlanBox`. If this variable is set to `1`, a routed bump can be set to a new box (moving action is implemented by changing the object's box). All commands that call `dbSetFPlanBox` internally are also affected by this variable.

Related Commands

- `assignBump`
- `assignSigToBump`
- `copy_bump`
- `deleteBumps`
- `swapSignal`
- `unassignBump`
- `alignObject`
- `dbSetFPlanBox`
- `flipOrRotateObject`
- `spaceObject`

9

Floorplan Commands

- add_ndr
- add_shape
- add_text
- addAloFiller
- addAlORow
- addDummyBoundaryWires
- addHaloToBlock
- addInstToInstGroup
- addIoFiller
- addIoInstance
- addIoRowFiller
- addModuleToFPlan
- addObjFPlanCutBox
- addRoutingHalo
- addSizeBlockage
- adjustFPlanChannel
- alignObject
- changeFloorplan
- changeIoConstraints
- check_macro_place_constraint
- check_ufc
- checkFPlan
- checkFPlanSpace
- checkMacroLLOnTrack
- convertFenceToLef
- copyObject
- create_bump_grid
- create_pg_model_for_macro_place
- create_relative_floorplan
- createAbuttedFPlan
- createDensityArea
- createExclusiveGroups
- createFence
- createGuide
- createInstGroup
- createIoRow
- createLogicHierarchy
- createPGPin

- createPhysicalPin
- createPlaceBlockage
- createRegion
- createRouteBlk
- createRouteBlkAlongTrack
- createRow
- createSoftGuide
- createSpecialDrcRegion
- createSpecialMarkLayerShape
- createStairwayBoundary
- cutRectilinearInst
- cutRow
- delete_bump_grid
- delete_relative_floorplan
- deleteAllDensityAreas
- deleteAllFPObjects
- deleteAllInstGroups
- deleteAllPowerPreroutes
- deleteAllSignalPreroutes
- deleteExclusiveGroups
- deleteFPObject
- deleteHaloFromBlock
- deleteInstFromInstGroup
- deleteInstGroup
- deleteIoFiller
- deleteIoInstance
- deleteIoRowFiller
- deleteNetWeight
- deletePGPin
- deletePlaceBlockage
- deleteRouteBlk
- deleteRoutingHalo
- deleteRow
- deleteSelectedFromFPlan
- deleteSizeBlockage
- deselectAll
- deselectGroup
- deselectInst
- deselectInstByCellName
- deselectInstOnNet
- deselectIOPin
- deselectNet
- display_obj_connectivity

- exportNdr
- finishFloorplan
- fix_boundary_overlaps
- fix_floorplan
- fixAllIos
- flipModule
- flipOrRotateObject
- floorPlan
- generate_fence
- get_macro_place_constraint
- get_snap_grid_info
- get_trace_obj_connectivity_mode
- getDrawView
- getFinishFPlanMode
- getFPlanMode
- getIoFlowFlag
- getNetWeight
- getObjFPlanBoxList
- getObjFPlanPolygon
- getRailPrototypeMode
- getResizeFPlanMode
- initCoreRow
- insert_physical_cell
- legalizeFPlan
- loadFPlan
- loadIoFile
- modify_ndr
- move_obj
- moveGroupPins
- moveMacroInsideModule
- pasteObject
- placeAIO
- placePadIO
- placePIO
- predict_floorplan
- queryFPlanObject
- refine_macro_place
- refineMacro
- report_narrow_channel
- report_obj_connectivity
- reportNetGroup
- reportSeedConnection
- reportSelect

- reportUnsnapBlocks
- reset_macro_place_constraint
- resizeFloorplan
- save_global
- saveFPlan
- saveIoFile
- select_row
- selectGroup
- selectInst
- selectInstByCellName
- selectInstOnNet
- selectIOPin
- selectNet
- selectPGPin
- selectRouteBlk
- set_macro_place_constraint
- set_trace_obj_connectivity_mode
- setBottomIoPadOrient
- setDrawView
- setFinishFPlanMode
- setFixedBlockSize
- setFlipping
- setFPlanMode
- setFPlanRowSpacingAndType
- setInstancePlacementStatus
- setInstGroupPhyHier
- setIoFlowFlag
- setIoRowMargin
- setObjFPlanBox
- setObjFPlanBoxList
- setObjFPlanPolygon
- setRailPrototypeMode
- setResizeFPlanMode
- setResizeLine
- setRouteBlkDefaultLayer
- setSelectedDensityArea
- setSelectedObstruct
- setSelectedRouteBlk
- setSelectedStripBoxShape
- setSelectedStripBoxState
- shiftOrigin
- snapFPlan
- snapFPlanIO

- spaceIoInst
- spaceObject
- specifyNetWeight
- stretchRows
- swapPins
- trace_obj_connectivity
- unfixAllIos
- unplaceAllBlocks
- unplaceAllGuides
- unplaceAllInsts
- unplaceGuide
- unplaceGuideConstraints
- unsetFixedBlockSize
- update_floorplan_for_macro_place
- write_macro_place_constraint
- writeFPlanScript
- add_interposer_route_block
- create_interposer_route_block

add_ndr

```
add_ndr
[-help]
[-add_via {via_name1 via_name2 ...}]
[-exclude_backside_via]
[-generate_via]
[-hard_spacing]
[-init string]
[-min_cut {layer1[:layer2] min_cut ...}]
[-spacing string | -spacing_multiplier string]
[-via {via_name1 via_name2 ...}]
[-width string | -width_multiplier string]
{-name ruleName}
[-use_via_cut_class string]
```

Adds a non-default rule for objects. You can also remove vias from the NDR. The command checks the `generateVias` mode and if `setGenerateViaMode` is set to `-auto true`, the generated vias is used irrespective of the `-generate_via` option. You can use the command at any point in the design flow.

Parameters

`-help`	Prints a brief description that includes type and default information for each `add_ndr` parameter. For a detailed description of the command and all of its parameters, use the man command: `man add_ndr`
`-add_via {`*`via_name1 via_name2 ...`*`}`	
	Adds vias to the existing via list. You can use this parameter to add DFM vias or other custom vias.
`-exclude_backside_via`	Prevents inclusion of non-route or backside vias in a newly defined non-default rule.
`-generate_via`	
	Auto-generates vias with the generateVia command. **Note:** If the NDR specified by you is with metal wider than the default rule, it is recommended to use this option for getting better routing results.
`-hard_spacing`	Sets the non-default rule as HARDSPACING.
`-init` *`string`*	Specifies the name of an existing non-default rule from which all values are initialized. **Note:** Other options will over-write these values.
`-min_cut {layer1[:layer2]` *`min_cut ...`*`}`	
	Specifies the minimum cut limit for via layers.
`-name` *`ruleName`*	Specifies the name of the non-default rule that is to be added.
`-spacing {layer1[:layer2]` *`spacing ...`*`}`	
	Specifies the spacing for routing layers.
`-spacing_multiplier {layer1[:layer2]` *`spacing_multiplier`*`}`	
	Spacing multiplier for routing layers.
`-use_via_cut_class` *`string`*	

	Specifies multiple layer definition and `cut_class` list to support the LEF/DEF property. The value must be given in the format of either `{layer cut_class}` or `{layer cut_class nr_row nr_col}`. Refer to the examples below:
	`add_ndr -name new_1 -use_via_cut_class {V1 VX}`
	`add_ndr -name new_2 -use_via_cut_class {V1 VX 2 1}`
	`add_ndr -use_via_cut_class {{V1 VX 2 1} {V2 VH 2 1} .....}`

`-via {`*`via_name1 via_name2 ...`*`}`	
	Resets and adds the list of via or via rule generate name.
`-width {layer1[:layer2]` *`width ...`*`}`	
	Specifies the width for routing layers.
`-width_multiplier {layer1[:layer2]` *`width_multiplier ...`*`}`	
	Width multiplier for routing layers.

Examples

- The following command creates a rule that requires extra spacing. All other values are the same as the default rule.
  ```
  add_ndr -name default_2x_space -spacing {metal1:metal4 0.2 metal5:metal6 0.4 metal7 0.6}
  ```
- The following command creates a new wide-wire routing rule (double wide on the Mx routing layers, but not the wide layers). Auto-generate vias because the width changed, so the default vias are not correct.
  ```
  add_ndr -name Mx_2x_wide -width {metal1:metal4 0.20} -generate_via
  ```
- The following command uses existing wide-wire rule, but require 2 cut vias for the Vx layers (via1 to via3)
  ```
  add_ndr -name Mx_2x_wide_Vx_2cut -init Mx_2x_wide -min_cut {via1:via3 2}
  ```

Related Information

- modify_ndr
- exportNdr
- setGenerateViaMode
- generateVias
- Floorplanning the Design chapter in the *Innovus User Guide*

add_shape

```
add_shape
[-help]
-layer {layerNameOrPointer}
[-net {netName}]
[-shape {None RING STRIPE FOLLOWPIN IOWIRE COREWIRE BLOCKWIRE FILLWIRE BLOCKAGEWIRE PADRING BLOCKRING DRCFILL FILLWIREOPC}]
[-shield_net {netNameOrPointer}]
[-status {ROUTED FIXED COVER SHIELD NOSHIELD}]
[-user_class className]
{-rect {x1 y1 x2 y2} | -patch {x1 y1 x2 y2} | -polygon {x1 y1 x2 y2 ...xn yn} |
[-pathSeg {x1 y1 x2 y2} -width value [-beginExt value -endExt value]]}
```

The command `add_shape` adds `DEF SPECIALNETS` wiring shapes to the database. You can use the command at any point in the design flow.

Parameters

`-help`	Prints a brief description that includes type and default information for each `add_shape` parameter. For a detailed description of the command and all of its parameters, use the man command: `man add_shape`
`-beginExt` *value*	Specify the extension beginning value in microns. *Default*: The default extension for a pathSeq is 0.
`-endExt` *value*	Specify the extension beginning value in microns. *Default*: The default extension for a pathSeq is 0.
`-layer {`*layerNameOrPointer*`}`	
	Name of the layer or layer pointer.
`-net` *netName*	Name of the net.
`-pathSeg {`*x1 y1 x2 y2*`}`	
	The beginning and ending coordinates for the path segment.
`-patch {`*x1 y1 x2 y2*`}`	
	Specifies the coordinates of the lower-left and upper-right points of the rectangle for creating the patch wires.
`-polygon {`*x1 y1 x2 y2 ...xn yn*`}`	
	The point list of the polygon.
`-rect {`*x1 y1 x2 y2*`}`	
	Specifies the coordinates of the lower-left and upper-right points of the rectangle for creating the horizontal and vertical wires.
`-shape {None RING STRIPE FOLLOWPIN IOWIRE COREWIRE BLOCKWIRE FILLWIRE BLOCKAGEWIRE PADRING BLOCKRING DRCFILL FILLWIREOPC}`	
	Defines the `SPECIALNETS` wiring shape. *Default*: `None`
`-shield_net {`*netNameOrPointer*`}`	
	The name or pointer of the net to be shielded by the via instance. The `-shield_net` parameter is only allowed when `-status` SHIELD is used.
`-status {ROUTED FIXED COVER SHIELD NOSHIELD}`	
	Defines the `SPECIALNETS` wiring status. *Default*: `ROUTED`.
`-width` *value*	The width in microns.
`-user_class`	Defines the attribute class by user.

Example

- `-pathSeg`: The command would add a `pathSeg` to the net with the name `VSS` on layer `METAL1` at the co-ordinates (100, 100, 250 ,250) with a width of 2.5 microns and the begin and end extensions of 1.25 microns.
  Note: Typically, the begin and end extension values are zero for most `DEF SPECIALNETS` wiring, but can be any value that would result in the end of the wire falling on the manufacturing grid.
  ```
  add_shape -net VSS -layer METAL1 -pathSeg {100 100 250 250} -width 2.5 -beginExt 1.25 -endExt 1.25
  ```
- `-polygon`: The command would add a polygon shaped wire to the net with the name `VSS` on layer `METAL1` as a polygon with coordinates specified.

```
add_shape -net VSS -layer METAL1 -polygon {{25.05 25.115} {25.115 25.05} {25.115 24.95} {5.05 4.885} {4.95 4.885} {4.885 4.95} {4.885 5.05} {24.95 25.115}}
```

- This command would add a polygon shape swire to snet `VSS` on layer `METAL3` having coordinates similar to the swire belonging to the net `VSS` on layer `METAL1`.

```
add_shape -polygon [dbGet [dbGet [dbGet top.nets.name VSS -p ].swires.layer.name M1 -p2 ].polyPts -i 0] -net VSS -layer METAL3
```

Related Information

- DEF Syntax chapter in the *LEF/DEF Language Reference*

add_text

```
add_text
[-help]
[-alignment {centerCenter centerLeft centerRight lowerCenter lowerLeft lowerRight upperCenter upperLeft upperRight}]
[-drafting {true | false}]
[-font {euroStyle gothic math roman script stick fixed swedish milSpec}]
[-height value]
-label {string}
[-layer {layerNameOrPointer}]
[-oaPurpose {string}]
[-orient {MX MX90 MY90 R0 R180 R270 R90}]
-pt {x y}
```

Adds text to the specified custom layer. You can use the `add_text` command at any point in the design flow. The text added by the command is to annotate comment text and does not show up in a `DEF` output.

Parameters

`-help`	Prints a brief description that includes type and default information for each `add_text` parameter. For a detailed description of the command and all of its parameters, use the man command: `man add_text`
`-alignment {centerCenter centerLeft centerRight lowerCenter lowerLeft lowerRight upperCenter upperLeft upperRight}`	
	Specifies the alignment of the text.
`-drafting {true | false}`	
	Indicates if the text is always displayed.
`-font {euroStyle gothic math roman script stick fixed swedish milSpec}`	
	Specifies the font style of the text. *Default*: `stick`
`-height value`	Specifies the height of the text, in microns.
`-label {string}`	
	Specifies the text string.
`-layer {layerNameOrPointer}`	
	Specifies the layer name or the layer pointer.
`-oaPurpose {string}`	
	Specifies a user-defined purpose name for OpenAccess text layer purpose pair support. Only values that exist in the library's tech graph are allowed. If the specified name is not a legal purpose name, a warning is displayed and the text is treated as drawing purpose. *Default:* `drawing`
`-orient {MX MX90 MY90 R0 R180 R270 R90}`	
	Specifies the orientation of the text.
`-pt {x y}`	
	Specifies the beginning coordinates of the text. It is specified in microns.

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

addAIoFiller

```
addAIoFiller
[-help]
{-cell fillerCellName | -cellList {fillerCellName1 fillerCellName2 ...}}
[-prefix prefix]
[-aioRowCluster aioRowClusterName | -allAIORowCluster]
[-setPreplaced]
[-onlyESD]
[-onlyShoulder]
[-onlyGap]
[-powerDomain powerDomainName]
```

Adds area I/O filler cell instances. Use this command during or after floorplanning. You can also use the command after the placement of manually created area I/Os.

The command,

- By default, adds area I/O filler cells to all the area I/O clusters if you do not specify the `-aioRowCluster` and `-allAIORowCluster` parameters.
- Adds filler cells in all the blank sites of the specified area I/O row clusters if you do not specify the `-onlyESD`, `-onlyGap` and `-onlyShoulder` parameters.
- Uses any LEF `CLASS PAD` or `CLASS PAD AREAIO` for adding the area I/O filler cells. By default the command uses LEF `CLASS PAD SPACER` to add filler cells, and also adds filler cells using any LEF `CLASS PAD` displaying a warning in Innovus.
- Does not allow overlapping of filler cells and places non-overlapping filler cells.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `addAIoFiller` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man addAIoFiller`
`-aioRowCluster` *aioRowClusterName*	
	Specifies the area I/O row cluster name. AIO filler cells are added only to this cluster. *Data_type*: string, optional
`-allAIORowCluster`	Adds AIOFiller cells to all the area I/O clusters. *Data_type*: bool, optional
`-cell` *fillerCellName*	Specifies the name of the area I/O filler cell to be added. *Data_type*: string, optional
`-cellList {`*fillerCellName1 fillerCellName2 ...*`}`	
	Specifies the list of area I/O filler cells that fill the gaps in I/O row clusters. Adds the largest filler cell first, followed by smaller filler cells. *Data_type*: string, optional
`-onlyESD`	Specifies to add ESD (electrostatic discharge) cells with the filler cell name in the area I/O row clusters. *Data_type*: bool, optional
`-onlyGap`	Specifies to add the filler cells with the filler cell name to fill all the gaps in the area I/O row clusters. *Data_type*: bool, optional
`-onlyShoulder`	Specifies to add shoulder cells with the filler cell name in the area I/O row clusters. *Data_type*: bool, optional
`-powerDomain` *powerDomainName*	
	Specifies the name of the power domain in which the area I/O filler cells are to be added. *Data_type*: string, optional
`-prefix` *prefix*	Specifies the prefix name to be appended for the added area I/O filler cell instance. *Data_type*: string, optional
`-setPreplaced`	Sets the status of the area I/O filler cells as preplaced. *Data_type*: bool, optional

Example

- The following command fills all area I/O row clusters on `ESD`, `shoulder`, and `gap` sites with a filler cell `A0_123` with preplaced status:
  ```
  addAIoFiller -cell AO_123 -setPreplaced
  ```
- The following command adds area I/O filler cells,`IOFILL` and `IOFILL_HALF`,to fill all the gaps in all area I/O row clusters:
  ```
  addAIoFiller -allAIORowCluster -onlyGap -cellList { IOFILL IOFILL_HALF }
  ```

Related Information

- `addAIORow`
- `addIoFiller`
- `addIoInstance`
- Floorplanning the Design chapter in the *Innovus User Guide*

addAIORow

```
addAIORow
[-help]
-site site_name
{-bump bumpId | -loc llx lly}
[-dx x_dist] [-dy y_dist]
[-orient {R0|R90|R180|R270|MX|MX90|MY|MY90}]
[-H | -V]
[-esd esites_no...]
[-shoulder site_no...]
[-cluster_halo space]
[-num num]
[-abutRow]
[-rowName name]
[-noSnap]
[-soft]
[{-ptnArrX elem1 | -ptnArrY elem2} {-pitchX dist | -pitchY dist}]
```

Adds area I/O rows. Use this command during or after floorplanning.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `addAIORow` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man addAIORow`
`-abutRow`	When creating two I/O rows that abut each other, this option specifies that one of them will be in a flip orientation to the other. *Data_type*: bool, optional
`-bump bumpId`	Specifies the bump ID that is used to specify the start point of the row area. The `-bump` parameter creates IO rows from the center point of the specified bump. For example, when you specify -bump 17_1_1, the row will be created from the center point of this bump. The length of row is controlled by other options. *Data_type*: string, required
`-cluster_halo space`	
	Specifies the distance, in microns, around the boundary of a row cluster to prevent other cells from being placed near it. *Data_type*: float, optional
`-dx x_dist`	Offsets the reference point, in microns, to the first I/O row on the `x` axis. *Data_type*: float, optional
`-dy y_dist`	Offsets the reference point, in microns, to the first I/O row on the `y` axis. *Data_type*: float, optional
`-esd esites_no...`	Reserves the sites in the row as electrostatic discharge cells (ESD). For example: `-esd 2,7` **Notes:** • Use only when you create cluster rows with multiple sites. • Do not use the `-soft` option with this option. *Data_type*: string, optional
`-H | -V`	Specifies the either a **H**orizontal or a **V**ertical row. *Data_type*: bool, optional
`-loc llx lly`	Specifies the lower left `x` and `y` coordinate reference points, in microns, of the new I/O row. *Data_type*: point, required

`-noSnap`	Adds the area IO row at the exact location specified by the `addAIORow -loc` coordinates rather than snapping the row to the standard cell placement grid. *Data_type*: bool, optional
`-num` *num*	Specifies the number of sites. *Data_type*: int, optional
`-orient {R0|R90|R180|R270|MX|MX90|MY|MY90}`	
	Specifies the orientation for the row. See *Orientation Key* below for more information. *Default*: R0 *Data_type*: enum, optional
`-ptnArrX` *elem1*	Specifies the x dimensions of the row cluster array size. For example, to create a 5 x 8 row array, `-ptnArrX 5 -ptnArrY 8` *Data_type*: int, optional
`-ptnArrY` *elem2*	Specifies the y dimensions of the row cluster array size. For example, to create a 4 x 6 row array, `-ptnArrX 4 -ptnArrY 6` *Data_type*: int, optional
`-pitchX` *dist*	Specifies the distance between bump centers, in microns, in the `x` direction. *Data_type*: float, optional
`-pitchY` *dist*	Specifies the distance between bump centers, in microns, in the `y` direction. *Data_type*: float, optional
`-rowName` *name*	Specifies a user-defined instance name for an added IO Row or a prefix for multiple rows, as in a matrix. If only one row is added, the name is used exactly as specified. If multiple rows are added, a number prefix is appended to the name. *Data_type*: string, optional
`-site` *siteName*	Specifies the site name for the area I/O row. *Data_type*: string, required
`-shoulder` *site_no...*	Reserves the sites in a row as shoulder cells. For example: `-shoulder 1,10` **Notes:** • Use only when you create cluster rows with multiple sites. • Do not use `-soft` option with this option. *Data_type*: string, optional
`-soft`	Creates a reef or cluster that can be collapsed if no I/O cells are placed in the available spaces within the reef or cluster. Use this parameter if the cluster site is not fully placed so that standard cells can be placed in these spaces if I/O cells are not placed there. **Note:** Do not use the `-esd` or `-shoulder` options with this option. *Data_type*: bool, optional

Orientation Key

The following table is a key to row orientation:

Value	Definition
`R0`	No rotation
`MX`	Mirror through X axis

`MY`	Mirror through Y axis
`R180`	Rotate counter-clockwise 180 degrees
`MX90`	Mirror through X axis and rotate counter-clockwise 90 degrees
`R90`	Rotate counter-clockwise 90 degrees
`R270`	Rotate counter-clockwise 270 degrees
`MY90`	Mirror through Y axis and rotate counter-clockwise 90 degrees

Examples

- The following command creates an I/O row with site name `IO1` at the location related to the center of the bump selected in the design window. There is a 5 x 7 row array and each element in the array is separated by `100` µm in both the `x` and `y` axes. Each row has `8` sites.
  ```
  addAIORow -site IO1 -bump 0x33457a -H -num 8 -ptnArrX 5 -ptnArrY 7 -pitchX 100 -pitchY 100
  ```
- The following command creates a soft I/O row cluster with a `15.0` micron halo around it:
  ```
  addAIORow -site IO3 -loc 0.0 0.0 -dx 230 -dy 58.8 -num 7 -orient R0 -V -soft -abutRow -cluster_halo 15.0
  ```
- The following command creates an IO Row with a user-defined name:
  ```
  addAIORow -site IO1 -rowName newrow -loc 0 0 -dx 0 -orient R0 -V -num 1
  ```

Related Information

- `addAIoFiller`
- `addIoFiller`
- `addIoInstance`
- Floorplanning the Design chapter in the *Innovus User Guide*

addDummyBoundaryWires

```
addDummyBoundaryWires
[-help]
-layers layerName | { layerNameList ... }
[-masks maskNumber | { maskNumberList ... }]
[-shape {blockring | blockagewire | blockwire | corewire | drcfill| fillwireopc | fillwire | followpin | iowire | none | padring | ring |
stripe} ]
[-shortWires wireLength | { wireLengthList ... }]
[-space spaceValue | { spaceValueList ... }]
```

Add wires on the first and last tracks in the direction parallel to preferred routing direction with specified layers.

Note: DRC rules define forbidden regions near the design boundary edges that are parallel to the preferred routing direction of a layer. Any wires in these regions cannot project to the design boundary directly. Using the `addDummyBoundaryWires` command you can add wires that shadow the design boundary such that no routing wires in the forbidden regions can project to the design boundary. This helps to avoid DRC violations.

Note: The `addDummyBoundaryWires` command enables long dummy wires to honor route blockages by default.

Note: The `addDummyBoundaryWire` command honors routing blockages and adds dummy wires along the routing blockages when the track box overlaps with them. The dummy wires do not cross partial routing blockages around the design boundary. It keeps the wires at the boundary and discontinues them when there is a routing blockage.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `addDummyBoundaryWires` parameter. For a detailed description of the command and all of its parameters, use the man command: `man addDummyBoundaryWires`
`-layers layerName | { layerNameList ... }`	

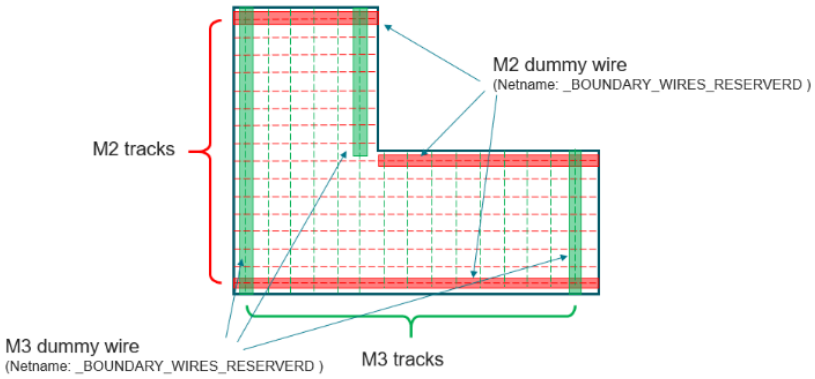
	Specifies the layers on which the dummy boundary wires are added.
	Note: The net name of the dummy wires is "`_BOUNDARY_WIRES_RESERVED`".
	Note: You can specify the *layerName* as a number, abbreviation or as a layer name defined in LEF. For example, *layerName* can be specified as 2, M2 or Metal2.
	`addDummyBoundaryWires -layers {metal2 metal3} //full name works`
	`addDummyBoundaryWires -layers {M2 M3} //abbreviated name works`
	`addDummyBoundaryWires -layers {2 3} ///layer number works`
	Data_type: string, required

`-masks maskNumber | { maskNumberList ... }`	Specifies one or multiple mask numbers which short dummy boundary wires snap to. The number of specified mask values should match the number of the specified layers in option layers. *Data_type*: string, optional
`-shape {blockring | blockagewire | blockwire | corewire | drcfill| fillwireopc | fillwire | followpin | iowire | none | padring | ring | stripe}`	
	Specifies the shape of the dummy boundary wires. *Default:* `none` *Data_type*: string, optional
`-shortWires wireLength | { wireLengthList ... }`	Specifies one or multiple set of lengths of short dummy boundary wires and space between them. The number of specified shortWires values should be twice of the number of specified layers in option layers. *Data_type*: string, optional
`-space spaceValue | { spaceValueList ... }`	Specifies one or multiple space values to design boundary. **Note**: The space value is the distance between the design boundary and the start or end of the wire for 90 corner. The space value does not apply to 270 corner. *Default:* `0` *Data_type*: string, optional

Examples

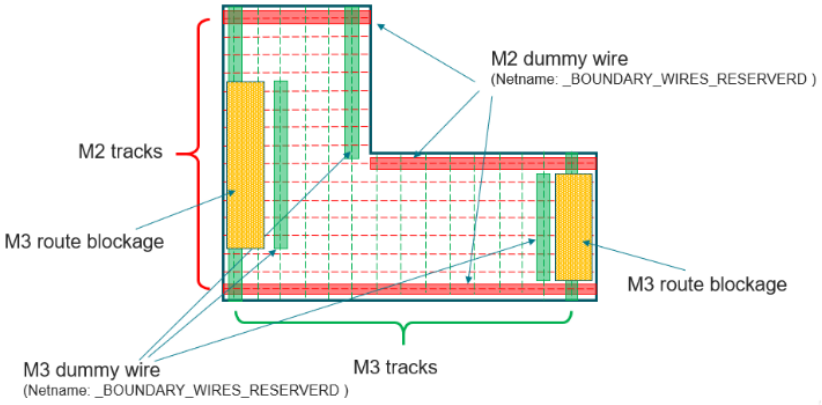
- The following command adds dummy wires on layers metal2 and metal3:

```
addDummyBoundaryWires -layers {M2 M3}
```



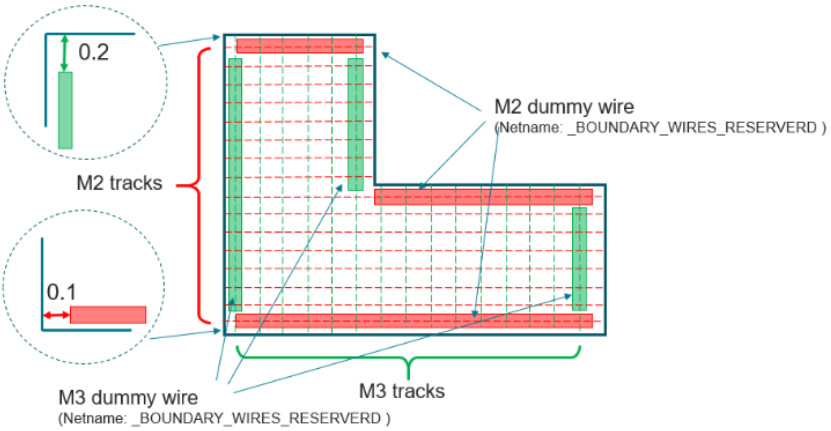
- The following command adds dummy wires on layers metal2 and metal3. Dummy wires will honor route blockages by default on Metal3 :

```
addDummyBoundaryWires -layers {M2 M3}
```



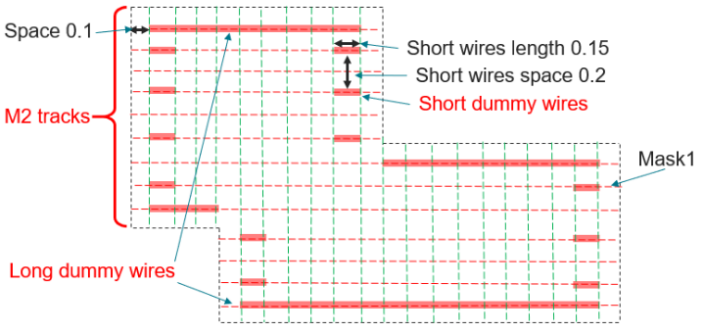
- The following command adds dummy wires with space value on layers metal2 and metal3:

```
addDummyBoundaryWires -layers {M2 M3} -space {0.1 0.2}
```



- The following command adds long dummy wires with specified space and short dummy wires with specified length, space and mask on layers metal2:

```
addDummyBoundaryWires -layers M2 -space 0.1 -shortWires {0.15 0.2} -masks 1
```



Related Information

- `legalizeFPlan`
- Floorplanning the Design chapter in the *Innovus User Guide*

addHaloToBlock

```
addHaloToBlock
[-help]
left bottom right top
[-fromInstBox]
[-snapToSite]
[-ori orientation]
[inst_name |-allMacro | -allBlackBox | -allCommitPtn | -allBlock | -allIOPad | -cell name]
```

Adds a halo to a block. A halo is an area that prevents the placement of blocks and standard cells within the specified halo distance from the edges of a hard macro, black box, or committed partition in order to reduce congestion. A block halo value is specified based on the current block orientation. Use this command during or after floorplanning. You can use the `deleteHaloFromBlock` command to delete halos.

Parameters

`-allBlackBox`	Specifies halo values for all black boxes. *Data_type*: bool, optional
`-allBlock`	Specifies halo values for all hard macros, black boxes, and committed partitions. *Data_type*: bool, optional
`-allCommitPtn`	Specifies halo values for all committed partitions. *Data_type*: bool, optional
`-allIOPad`	Specifies halo values for all IO pads. *Data_type*: bool, optional
`-allMacro`	Specifies halo values for all hard macros. *Data_type*: bool, optional
`-cell` *name*	Specifies halo values for all instances of a cell. *Data_type*: string, optional
`-fromInstBox`	Creates a block halo, based on the instance boundary, even when there is an overlap layer defined in the block cell or macro definition. **Note:** This parameter is for backward compatibility only. Cadence does not recommend the use of this parameter because there can be problems with the defin flow if this parameter is specified and if the block cell or the macro has overlap layers. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `addHaloToBlock` parameter. For a detailed description of the command and all of its parameters, use the man command: `man addHaloToBlock`
inst_name	Specifies the block where the halo is to be added. You can use wildcards (* or ?) for specifying instance names. *Data_type*: string, optional
left bottom right top	
	Specifies the halo width on 4 sides. These values are floats, units are microns. Halo deltas are specified in relation to a block in R0 orientation (`left` is applied to the left edge of block oriented R0). For non-R0, the halo deltas are rotated/flipped according to the blocks's orientation (for block oriented MY, the `left` halo delta specifies the width of the halo along the *right* side of the inst in the floorplan). • `left`: the width of the halo along the left edge of insts having halos added. • `bottom`: the width of the halo along the bottom edge of objects having halos added. • `right`: the width of the halo along the right edge of objects having halos added. • `top`: the width of the halo along the top edge of objects having halos added. *Data_type*: box, required
`-ori` *orientation*	
	Specifies the orientation value of the instances to which the block halo values are applied. The orientation value can be R0, R90, R180, R270, MX, MX90, MY, or MY90. **Note**: If the orientation is not set, the halo will be added with the current orientation of the macro. *Data_type*: string, optional
`-snapToSite`	Snaps the halo box to a site. *Data_type*: bool, optional

Example

The following command adds a halo around block `xy_inst`:

```
addHaloToBlock 10 20 30 40 xy_inst
```

If `xy_inst` is R0, the halo width is 10 along the left edge and 30 along the right edge. If the block is MY, the halo width is 30 along the *left* edge in the floorplan, and 10 along the *right* edge in the floorplan.

The following command adds a halo around all insts with cells of type `myDRAMCell`:

```
addHaloToBlock 10 20 30 40 -cell myDRAMCell -ori MY
```

If the inst is oriented R0, then the *left* edge width of the halo is 30 in the floorplan, and the *right* edge width of the halo is 10.

Related Information

- `deleteHaloFromBlock`
- Floorplan Menu chapter in the *Innovus Menu Reference*
  - Edit Halo

addInstToInstGroup

```
addInstToInstGroup
[-help]
group_name
object_name
```

Adds a hierarchical instance, an instance or list of instances, or a group to a specified group. A group is a user-defined pseudo hierarchy and is created to contain instances and/or other groups. Use this command after creating a group name.

Notes:

- When you add a member to a group whose constraint is a fence or a region, the size of the bounding box of the group does not change.
- If the group being added is also a power domain, a warning is displayed and the command is not executed.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `addInstToInstGroup` parameter. For a detailed description of the command and all of its parameters, use the man command: `man addInstToInstGroup`
group_name	Specifies the name of the created group. *Data_type*: string, required
object_name	Specifies the name of the object, another group, hierarchical instance or leaf instance. **Note:** You can specify an instance names list in the { *inst_name_list* } format. The names list option is used for instances only and NOT for group or hierarchical instances. *Data_type*: string, required

Examples

- The following command adds hierarchical instance to group adder1:
  ```
  addInstToInstGroup adder1 SH28/I12/I44
  ```
- The following command adds group subadder to group adder1:
  ```
  addInstToInstGroup adder1 subadder
  ```
- The following command adds the instances inst_I1, inst_I2, and inst_I3 to group inst_group:
  ```
  addInstToInstGroup inst_group {inst_I1 inst_I2 inst_I3}
  ```

Related Information

- `createInstGroup`
- `deleteAllInstGroups`
- `deleteInstFromInstGroup`
- `deleteInstGroup`
- `createExclusiveGroups`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Grouping Instances

addIoFiller

```
addIoFiller
[-help]
-cell {name_list}
[-fillAnyGap]
[-fillerOrient R0 | R90 | R180 | R270 | MX | MX90 | MY | MY90]
[-from coord]
[-prefix prefix]
[-side {top | bottom | left | right}]
[-to coord]
[-row rowNumber]
[-useSmallIoHeight]
[-logic [-deriveConnectivity]]
```

Adds I/O instances in the I/O box. The I/O instances are added between the gap of existing I/O pad instances where the gap is large enough for the I/O instance. Use this command during or after floorplanning.

Note: Before using `addIoFiller`, run the `globalNetConnect` command to provide global-net-connection rules for supply pins of the added fillers. Without these rules, the built-in design-rule checks of `addIoFiller` will not be accurate.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `addIoFiller` parameter. For a detailed description of the command and all of its parameters, use the man command: `man addIoFiller`
`-cell {name_list}`	Specifies the name list of the I/O filler cells to be added. *Data_type*: list, required
`-deriveConnectivity`	Derives connectivity for I/O fillers. *Data_type*: bool, optional
`-fillAnyGap`	Forces the I/O filler instance into a gap even though the gap (clearance) is not large enough. *Default*: If you do not specify this parameter, it does not force a filler instance. *Data_type*: bool, optional
`-fillerOrient R0 | R90 | R180 | R270 | MX | MX90 | MY | MY90`	
	Specifies the orientation value of the filler cell. The orientation value can be R0, R90, R180, R270, MX, MX90, MY, or MY90. *Default*: R0 *Data_type*: string, optional
`-from coord`	Specifies the starting offset from the edge for adding fillers. The values are in micrometers. **Note:** The `-from` and `-to` parameters are intended to be used with the `-side` parameter to indicate a range. This command is applicable to peripheral I/O only; therefore, it is only one dimensional. *Data_type*: float, optional
`-logic`	Enables the creation of I/O filler instances for non-physical I/O cells. The added IO fillers are not marked as physical only cells. They are saved in netlist if user does saveNetlist. *Data_type*: bool, optional
`-prefix prefix`	Specifies the prefix name for the new I/O filler instances. *Default*: `FILLER` *Data_type*: string, optional
`-row rowNumber`	Specifies the row number for inserting fillers in a multiple I/O row design. *Minimum*: 0 *Maximum*: 1 *Data_type*: int, optional

`-side {top | bottom | left | right}`	
	Specifies the side as top side, bottom side, left side, and right side of the I/O box. *Default*: If you do not specify this parameter, all sides are specified. *Data_type*: string, optional
`-to` *coord*	Specifies the ending offset from the edge for adding fillers. The values are in micrometers. **Note:** The `-from` and `-to` parameters are intended to be used with the `-side` parameter to indicate a range. This command is applicable to peripheral I/O only; therefore, it is only one dimensional. *Data_type*: float, optional
`-useSmallIoHeight`	Specifies that the filler cell should be inserted between I/O pads only when the smallest I/O pad has the same height as the height of the filler cell. For example, consider that there are I/O pads A, B, C, D, and E. Out of these, A, B, and E are the same height, and C and D have a height double that of A, B, and E. Assume that you want to insert an I/O filler cell F, which has the same height as A, B, and E. If you do not specify the `-useSmallIoHeight` parameter, the cell F will be inserted between A and B, B and C, C and D, and D and E. However, if you specify the `-useSmallIoHeight` parameter, the cell F will be inserted between A and B, B and C, and D and E. The cell will not be inserted between C and D, because both C and D have a height different from F. *Data_type*: bool, optional

Examples

- The following command adds I/O instances to the I/O pad area on all sides wherever there is space between existing I/O pads. The added I/O pad filler instance names have a prefix PAD8.
  ```
  addIoFiller -cell PAD8 -prefix PAD8
  ```
- The following command adds I/O instances to the I/O pad area on the top side from 200 micrometers. The added I/O pad filler instance names have a prefix PAD8.
  ```
  addIoFiller -cell PAD8 -prefix PAD8 -side top -from 200
  ```
- The following command adds I/O instances to the I/O pad area on the left side to 300 micrometers. The added I/O pad filler instance names have a prefix PAD8.
  ```
  addIoFiller -cell PAD8 -prefix PAD8 -side left -to 300
  ```

Related Information

- `deleteIoFiller`
- `addAIoFiller`
- `addAIORow`
- `addIoInstance`
- `globalNetConnect`
- Floorplanning the Design chapter in the *Innovus User Guide*

addIoInstance

```
addIoInstance
[-help]
[-cell {name1 name2 ...}]
-inst {inst1 inst2 ...}
{{-refInst inst_name [-ccw]} | {-selected [-ccw]} |
{-repeat num [-skip num]
[-inSelected | [-ioRing num] -side {N | W | S | E} | -ioRow name]} | -loc llx lly }
[-spacing val]
[-orientation orient]
[-spread]
```

Specifies constraints for insertion of power and ground I/O instances in the I/O ring or in the I/O rows. This command inserts physical IO cells into design following constraints. It is used during floorplanning. If you do not specify any of the following parameters, the command `addIoInstance` adds I/Os for all I/O cells, on all sides and for all I/O rings.

- `-ioRing num`
- `-side`
- `-ioRow name`

Parameters

`-ccw`	Specifies the counter-clockwise direction to add the new I/O instance. *Data_type*: bool, optional
`-cell {name1 name2 ...}`	Specifies one or more cells. Adds one I/O for each cell that was specified. *Data_type*: string, optional
`-help`	Outputs a brief description that includes type and default information for each `addIoInstance` parameter.For a detailed description of the command and all of its parameters, use the man command: `man addIoInstance`
`-inSelected`	Repeatedly inserts new I/O instance(s) within the selected I/O cell(s). *Data_type*: bool, required
`-inst {inst1 inst2 ...}`	Specifies the instance name prefix for the new I/O. This must match with the number of cells that are specified. *Data_type*: string, required
`-ioRing num`	Specifies the I/O ring number in which the new I/O instance is added. *Default*: 0 *Data_type*: int, optional
`-ioRow name`	For the new I/O row flow, adds I/Os on the specified I/O row. *Data_type*: string, optional
`-loc x y`	Adds the new I/O instance at the location specified by *x y* coordinates. **Note**: You cannot use this parameter in conjunction with the `-refInst` or `-selected` parameters. *Data_type*: point, required

`-orientation` *orient*	Specifies the orientation of the new I/O instance. You can specify one of the following values: • `R0` • `R90` • `R180` • `R270` • `MX90` • `MY90` • `MX` • `MY` *Default*: `R0` Use this parameter with `-cell` and `-loc` parameters. **Note**: When specifying the `-refInst` or `-selected` parameters, the software uses the orientation from those instances. See Orientation Key for more information. *Data_type*: string, optional
`-refInst` *inst_name*	Specifies the name of the reference instance and adds the new I/O instance above or below the reference instance in clockwise direction. *Default*: Clockwise direction of the reference instance. *Data_type*: string, optional
`-repeat` *num*	Repeatedly adds new I/O instances every number (num) of cells that you specify. *Default*: Adds instances in the clockwise direction of the reference instance. *Data_type*: int, required
`-selected`	If `-repeat` is specified, this parameter adds the new I/O instance(s) in the selected cell(s) in clockwise direction of the selected instance. If `-repeat` is not specified, this parameter changes the selected cells as referenced cells and adds new I/O instances in the referenced cells, in the default clockwise direction. *Data_type*: bool, required
`-side {N | W | S | E}`	Specifies the side on which the new I/O instance is added. *Default*: `N` *Data_type*: string, optional
`-skip` *num*	Skips the number (num) of cells specified, before adding the first I/O instance. This happens only when `-repeat` num is specified. *Data_type*: int, optional
`-spacing` *val*	Specifies the spacing value, in microns, between the I/O instances. *Data_type*: float, optional
`-spread`	Evenly replaces all the I/O cells on the same row where the new I/O instance was added. This prevents overlapping of I/O cells. If you do not specify this parameter, the command still adds the new I/Os even if overlapping occurs. **Note**: The `addIoInstance` command works for both the I/O flows, the old flow and the new row based pad placement flow. In the new flow, each time you add a new I/O instance, the I/Os on the I/O rows with the existing constraints are replaced. In this situation, if all the existing I/Os do not have any specific constraint defined, then the replacement of the I/Os also resolves overlapping each time when the I/O cell is added, evading the usage of `-spread` option. *Data_type*: bool, optional

Example

- The following command adds new I/O instance il to cell BUMPCELL at lower-left x location 100 and lower-left y location 100:
  ```
  addIoInstance -inst il -cell BUMPCELL -loc 100 100
  ```
- The following command inserts the I/O instance newIo for the cell PVDD on the West side of the I/O box, skipping 3 I/O's (from below) on the West side and repeating the insertion of I/O's every 2 instances.

```
addIoInstance -cell PVDD -repeat 2 -inst newIo -side W -skip 3 -spread
```

Related Information

- deleteIoInstance
- Floorplanning the Design chapter in the *Innovus User Guide*

addIoRowFiller

```
addIoRowFiller
[-help]
-cell {fillerCellNameList ...}
[-logic]
[-prefix prefix]
[[-ioRow name | -powerDomain powerDomainName]
[-fillerOrient R0 | R90 | R180 | R270 | MX | MX90 | MY | MY90]]
[-from coord] [-to coord]
[-fillAnyGap]
[-useSmallIoHeight]
[-ignoreSiteType]
```

Adds I/O filler cells in the I/O rows. The I/O filler cells are added between the gap of existing I/O pad instances where the gap is large enough for the I/O cell. Use this command during or after floorplanning.

Parameters

`-cell {`*`fillerCellNameList ...`*`}`	
	Specifies the name of the filler cell(s) to add in the I/O row. *Data_type*: string, required
`-fillAnyGap`	Forces the I/O filler instance into a gap even though the gap (clearance) is not large enough. *Default*: If you do not specify this parameter, it does not force a filler instance. *Data_type*: bool, optional
`-fillerOrient R0 | R90 | R180 | R270 | MX | MX90 | MY | MY90`	
	Specifies the orientation value of the filler cell. The orientation value can be R0, R90, R180, R270, MX, MX90, MY, or MY90. *Default*: `R0` *Data_type*: string, optional
`-from` *`coord`*	Specifies the coordinates of a specified starting range where I/O filler cells are added. The values are in micrometers. *Data_type*: float, optional
`-help`	Outputs a brief description that includes type and default information for each `addIoRowFiller` parameter. For a detailed description of the command and all of its parameters, use the man command: `man addIoRowFiller`
`-ignoreSiteType`	Ignores the I/O row site type. *Data_type*: bool, optional
`-ioRow` *`name`*	Specifies the name of the I/O row to add the filler cell. *Data_type*: string, optional
`-logic`	Adds logical I/O fillers to net list in top module. *Data_type*: bool, optional
`-powerDomain` *`powerDomainName`*	
	Adds I/O filler instances to the I/O rows associated with the specified power domain. *Data_type*: string, optional
`-prefix` *`prefix`*	Specifies the prefix name to be appended for the added I/O row filler cell. *Data_type*: string, optional
`-to` *`coord`*	Specifies the coordinates of a specified ending range where I/O row filler cells are added. The values are in micrometers. *Data_type*: float, optional
`-useSmallIoHeight`	Specifies that the filler cell should be inserted between I/O pads only when the smallest I/O pad has the same height as the height of the filler cell. For example, consider that there are I/O pads A, B, C, D, and E. Out of these, A, B, and E are the same height, and C and D have a height double that of A, B, and E. Assume that you want to insert an I/O filler cell F, which has the same height as A, B, and E. If you do not specify the `-useSmallIoHeight` parameter, the cell F will be inserted between A and B, B and C, C and D, and D and E. However, if you specify the `-useSmallIoHeight` parameter, the cell F will be inserted between A and B, B and C, and D and E. The cell will not be inserted between C and D, because both C and D have a height different from F. *Data_type*: bool, optional

Example

- The following command adds I/O instances to the I/O pad area on all sides wherever there is space between existing I/O pads. The added I/O pad filler instance names have a prefix PAD8.
  ```
  addIoRowFiller -cell PAD8 -prefix PAD8
  ```

Related Information

- `createIoRow`
- `deleteIoRowFiller`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Performing I/O Row Based Pad Placement

addModuleToFPlan

```
addModuleToFPlan
[-help]
hInstName
[add_or_remove]
```

Use this command to display modules in the floorplan view whose instance count is smaller than the `setPreference MinFPModuleSize` value. By default, modules whose instance count is less than `MinFPModuleSize` are not displayed (default is 1). The `MinFPModuleSize` variable can be set to a larger value to focus only on larger modules which require floorplanning. You can then display specific smaller modules using this command. For example, use `addModuleToFPlan` if you need to interactively floorplan small regions or fences to control placement of small modules of logic.

Note: You can change the *Min. floorplan Module Size* option in the *Preferences* form in the *Display* menu and import the design. Alternatively, you can use the `setPreference MinFPModuleSize value` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `addModuleToFPlan` parameter. For a detailed description of the command and all of its parameters, use the man command: `man addModuleToFPlan`
hInstName	Specifies the name of the hierarchical instance. *Data_type*: string, required
add_or_remove	Adds or removes the specified hierarchical instance. Specify 1 to add or 0 to remove. *Data_type*: int, optional

Example

- The following command adds hierarchical instance SH99 in the floorplan view:
  ```
  addModuleToFPlan SH99 1
  ```
- The following command only displays modules with 200 instances or more:
  ```
  setPreference MinFPModuleSize 200
  ```

  If module mcore0/a0 has less than 200 instances you can display it using:
  ```
  addModuleToFPlan mcore0/a0 1
  ```

  To disable its display:
  ```
  addModuleToFPlan mcore0/a0 0
  ```

Related Information

- `setPreference`
- Floorplanning the Design chapter in the *Innovus User Guide*

addObjFPlanCutBox

```
addObjFPlanCutBox
[-help]
type
name
boxlist
```

Specifies the rectilinear shape of floorplan objects by describing the cut out area. The floorplan objects include block top cell, module constraint (guide, region, fence), layershape, pin, net, group, and instance. The command creates a rectilinear cut box for an object. A rectilinear object is comprised of two or more boxes. To use this command, divide the required cut out area into one or more bounding boxes and specify the coordinates of each bounding box in the command. Once you run the command, the rectilinear cut area disappears from the object after a redraw. You can use the command at any point in the design flow.

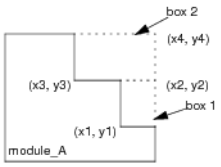
Parameters

`-help`	Outputs a brief description that includes the type and default information for each `addObjFPlanCutBox` parameter. For a detailed description of the command and all of its parameters, use the man command: `man addObjFPlanCutBox`
`name`	Specifies the name of the object for type. *Data_type*: string, required
`type`	Specifies the type of object for name. This can be: • `Cell` • `Group` • `Instance` • `Layershape` • `Module` • `Net` • `Pin` *Data_type*: string, required
`boxList`	Specifies the rectilinear shape of an object. The box list must be specified in the following format: `{{x1 y1 x2 y2} ....}` *Data_type*: list, required

Example

The following example defines the rectilinear shape of the module module_A by specifying 2 rectilinear cut boxes defined by coordinates (x1, y1), (x2, y2), (x3, y3), and (x4, y4).

```
addObjFPlanCutBox Module module_A {{x1 y1 x2 y2} {x3 y3 x4 y4}}
```



Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

addRoutingHalo

```
addRoutingHalo
[-help]
{ {-bottom bottomLayer -space haloValue -top topLayer} | -lithoHalo }
{-allBlocks | -block blockNameList | -cell cellNameList | -inst instanceName | -designHalo }
```

Adds a routing halo for blackboxes, hard macros, or block-level designs. A routing halo significantly reduces the possibility of long wire routing within the specified area, thus helping reduce signal integrity violations at the top- and/or block-level designs. A routing halo is honored only by the signal router. The signal router treats routing on the specific routing layers in the routing halo area as very high cost routing. However, perpendicular routing or straight connections to pins are acceptable. The special router does not honor routing halos. To block special routing, use routing blockages instead. You can specify a routing halo for:

- hard macros or blackboxes
- block-level designs
- partitions

For the bottom-up hierarchical flow, use this command to specify routing halo for a block at the top-level design or at the block-level design. The routing halo can be stored as an instance and/or design property and can be viewed through the attribute editor for the instance block. The routing halo can be saved to or restored from a floorplan file. Use this command after importing a design.

Notes:

- For top-down hierarchical flow, specify routing halo for a partition through the `definePartition` command or through the *Specify Partition* form.
- A halo is not allowed on std-cells (cells with CLASS CORE SITE) and is written out as ROUTEHALO in DEF. A design halo is written out as a DEF design FE_ROUTING_HALO property and a litho halo is written out as a DEF component LITHOHALO property.

Parameters

`-allBlocks`	Specifies that the routing halo should be added to all blocks in the design. *Data_type*: bool, required
`-block` *blockNameList*	Specifies that the routing halo should be added to the specified blocks. A block name must be an instance name and not a cell master name. The specified blocks should be separated by a space. **Note:** A halo is not allowed on std-cells (cells with CLASS CORE SITE) and is written out as ROUTEHALO in DEF. A design halo is written out as a DEF design FE_ROUTING_HALO property and a litho halo is written out as a DEF component LITHOHALO property. *Data_type*: list, required
`-bottom` *bottomLayer*	Specifies the bottom layer for the routing halo. **Note:** Even though the `-bottom`, `-space`, and `-top` parameters are optional, it is important to specify all of them together. *Data_type*: string, required
`-cell` *cellNameList*	Specifies that the routing halo should be added to all insts of the specified cell names. *Data_type*: list, required
`-designHalo`	Specifies that the routing halo should be added inside design boundary. **Note:** The `-designHalo` parameter cannot be specified with the `-lithoHalo` parameter. These parameters are mutually exclusive. *Data_type*: bool, required
`-help`	Outputs a brief description that includes the type and default information for each `addRoutingHalo` parameter. For a detailed description of the command and all of its parameters, use the man command: `man addRoutingHalo`
`-inst` *instanceName*	Specifies that the routing halo should be added to the specified instance. This parameter can be useful for adding halo for I/O cells. *Data_type*: list, required
`-lithoHalo`	Adds a litho halo to the instances on each routing layer with `LEF LITHOMACROHALO` defined. **Notes:** • The `-lithoHalo` parameter cannot be specified with the `-designHalo` parameter. These parameters are mutually exclusive. • The `-lithoHalo` parameter requires some LEF LITHOMACROHALO values to be set. • A halo is not allowed on std-cells (cells with CLASS CORE SITE) and is written out as ROUTEHALO in DEF. A design halo is written out as a DEF design FE_ROUTING_HALO property and a litho halo is written out as a DEF component LITHOHALO property. • *Data_type*: bool, required
`-space` *haloValue*	Specifies the routing halo value in microns. **Note:** Even though the `-bottom`, `-space`, and `-top` parameters are optional, it is important to specify all of them together. *Minimum*: 0 *Maximum*: 1.79769313486232e+308 *Data_type*: float, required
`-top` *topLayer*	Specifies the top layer for the routing halo. **Note:** Even though the `-bottom`, `-space`, and `-top` parameters are optional, it is important to specify all of them together. *Data_type*: string, required

Examples

- The following command adds a routing halo of 10 microns to all blocks and specifies that the top and the bottom layers for the routing halo are Metal7 and Metal1, respectively.
  ```
  addRoutingHalo -space 10 -top Metal7 -bottom Metal1 -allBlocks
  ```
- The following command adds a routing halo of 5 microns to the specified blocks and specifies that the top and the bottom layers for the routing halo are Metal2 and Metal1, respectively.
  ```
  addRoutingHalo -block DTMF_INST/RAM_256x16_TEST_INST/RAM_256x16_INST DTMF_INST/RAM_128x16_TEST_INST/RAM_128x16_INST -space 5 -top
  ```

```
Metal2 -bottom Metal1
```

Related Information

- deleteRoutingHalo
- Floorplanning the Design chapter in the *Innovus User Guide*
- Floorplan Menu chapter in the *Innovus Menu Reference*
  - Edit Halo - Routing Halo

addSizeBlockage

```
addSizeBlockage
[-help]
[-name blockageName]
[-isResizeable]
-box x1 y1 x2 y2
```

Adds a size blockage object that controls the behavior of the resize line under a covered area. The object area prevents floorplan objects in this area from being resized and maintains the alignment and minimum space between these objects during floorplan resize. Size blockage is special kind of blockage that is only honored by the `resizeFloorplan` command. By default resize line can not go through size blockage, which means, objects under size blockage stay fixed after floorplan resize. You can use this command before using the `resizeFloorplan` command.

Parameters

`-box x1 y1 x2 y2`	Specifies the lower left x coordinate, lower left y coordinate, upper right x coordinate, and upper right y coordinate respectively of the size blockage area. *Data_type*: box, required
`-help`	Outputs a brief description that includes type and default information for each `addSizeBlockage` parameter. For a detailed description of the command and all of its parameters, use the man command: `man addSizeBlockage`
`-isResizeable`	Specifies that the size blockage can be resized, however alignment and the minimum space between the objects in the blockage area will be maintained during floorplan resize. When this options is specified, the resize line can go through size blockage. But it can not turn direction inside blockage. The benefit of using this option is if objects (for example, macros) under size blockage are aligned before calling `resizeFloorplan`, they will stay aligned after the command. *Data_type*: bool, optional
`-name blockageName`	Specifies the name of the size blockage. *Data_type*: string, optional

Example

The following command creates a size blockage.

```
addSizeBlockage -name sizeBlkg1 -box 400 400 500 500
```

Related Information

- `deleteSizeBlockage`
- `getResizeFPlanMode`
- `resizeFloorplan`
- `setResizeFPlanMode`
- `setResizeLine`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Resizing the Floorplan

adjustFPlanChannel

```
adjustFPlanChannel
[-help]
[-channelUtil utilizationValue]
[-constraints constraint_file]
[-moveOnly]
[-report output_file]
[-reportOnly [-report_low_utilized_channels value]
  [-type {all fenceToFence fenceToCore fenceToMacro macroToMacro macroToCore}]]
```

Automatically adjusts the channel width between objects to prevent potential routing congestion. You can use this command after importing the design, loading a completed floorplan, and then running the `place_design` and `earlyGlobalRoute` commands.

Parameters

`-channelUtil utilizationValue`	
	Specifies the maximum utilization of routing tracks in a channel. *Data_type*: float, optional
`-constraints constraint_file`	
	Specifies the file that contains constraints for floorplan objects. A constraint file is a text file that can contain a list of seeds to use when generating fences and module guides, and basic relative, spacing and orientation constraints to follow during macro placement. *Data_type*: string, optional
`-help`	Outputs a brief description that includes type and default information for each `adjustFPlanChannel` parameter. For a detailed description of the command and all of its parameters, use the man command: `man adjustFPlanChannel`
`-moveOnly`	Specifies that only move-only adjustment is allowed. *Data_type*: bool, optional
`-report output_file`	Generates a report that contains the channel congestion, targeted channel adjustment, and final real floorplan adjustment. *Default*: `topcell.channel` *Data_type*: string, optional
`-reportOnly`	Generates a report of the channel congestion without adjusting the floorplan. *Data_type*: bool, optional
`-report_low_utilized_channels value`	
	Reports channel congestion whose track utilization is lower than the user-specified threshold value. *Minimum*: 0 *Maximum*: 1.0 *Data_type*: float, optional
`-type {all fenceToFence fenceToCore fenceToMacro macroToMacro macroToCore}`	
	Reports channel congestion for specified type. You can specify more than one type at a time. *Default*: `all` *Data_type*: enum_list, optional

Example

- The following command generates a report of only fence-to-fence congestion, without adjusting the floorplan:
  ```
  adjustFPlanChannel -reportOnly -type {fenceToFence}
  ```
- The following command generates a channel congestion report without adjusting the floorplan:
  ```
  adjustFPlanChannel -reportOnly -report reportfile
  ```

  The following report is generated:

  ```
  Channel (315.48 357.67 361.60 511.57)
  Left: CORE
  Right: DTMF_INST/PLLCLK_INST
  Congested utilization: 200/185 (108.1% V)
  Expected channel width increase: 6.48 um

  Channel (315.48 511.57 351.33 646.17)
  Left: CORE
  Right: DTMF_INST/RAM_128x16_TEST_INST/RAM_128x16_INST
  Congested utilization: 128/117 (109.4% V)
  Expected channel width increase: 5.51 um

  Channel (466.52 646.17 792.28 670.01)
  Bottom: DTMF_INST/RAM_128x16_TEST_INST/RAM_128x16_INST
  Top: DTMF_INST/ROM_512x16_0_INST
  Congested utilization: 52/42 (123.8% H)
  Expected channel height increase: 7.38 um
  ```

Related Information

- checkDesign
- place_design
- earlyGlobalRoute
- Floorplanning the Design chapter in the *Innovus User Guide*

alignObject

```
alignObject
[-help]
-side {right | left | center | top | bottom | middle}
[-referToFirst]
[-mix]
```

Aligns objects (instances, modules, or blockages) vertically or horizontally in the floorplan. Use this command after selecting two or more objects in the design display area.

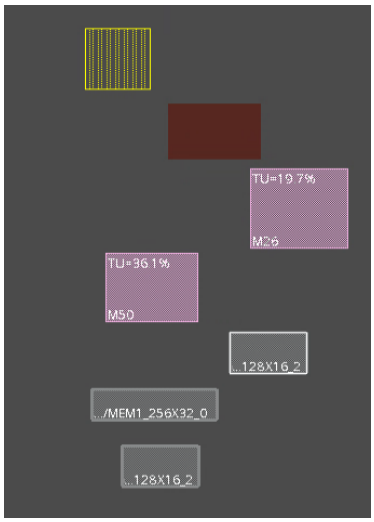
Parameters

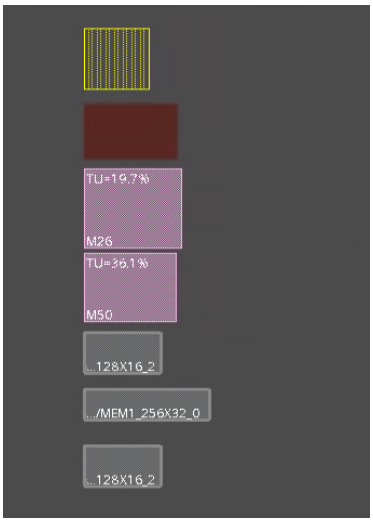
`-help`	Outputs a brief description that includes the type and default information for each `alignObject` parameter. For a detailed description of the command and all of its parameters, use the man command: `man alignObject`
`-mix`	Considers all the selected objects together while aligning them. *Data_type:* bool, optional
`-referToFirst`	If you specify this option, the first selected object is the reference object. *Data_type:* bool, optional
`-side {right | left | center | top | bottom | middle}`	
	Specifies the vertical and horizontal alignment for the selected objects. Choose one of the following: • `right`: Specifies vertical right alignment. • `left`: Specifies vertical left alignment. • `center`: Specifies vertical center alignment. • `top`: Specifies horizontal top alignment. • `bottom`: Specifies horizontal bottom alignment. • `middle`: Specifies horizontal middle alignment. *Data_type:* string, required

Example

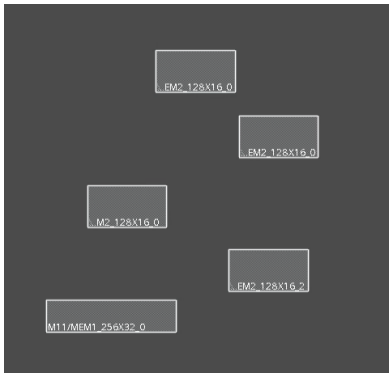
- The following command aligns the selected objects (Block) vertically to the left:
  ```
  alignObject -side left
  ```
  Before: After:

- The following command aligns the selected objects (Blockage、Module、Block) vertically to the left：
  ```
  alignObject -side left -mix
  ```
  Before: After：

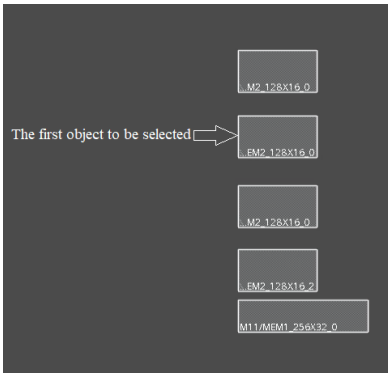




- The following command aligns the selected objects (Block) vertically to the left：
  ```
  alignObject -side left -referToFirst
  ```
  Before：



After：



Related Information

- `shiftOrigin`
- `spaceObject`
- `flipOrRotateObject`
- Floorplanning the Design chapter in the *Innovus User Guide*

changeFloorplan

```
changeFloorplan
[-help]
[-noSnapToGrid]
[[-coreToLeft value][-coreToBottom value][-coreToRight value][-coreToTop value] |
[-coreToEdge {left bottom right top}]]
```

Changes the distances between the core box and the die box on the sides you specified. Use this command after specifying the floorplan data or running an initial floorplan.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `changeFloorplan` parameter. For a detailed description of the command and all of its parameters, use the man command: `man changeFloorplan`
`-coreToBottom value`	Specifies the distance from the bottom of the core box to the bottom of the die box. *Data_type*: float, optional
`-coreToEdge {left bottom right top}`	
	Specifies the distances between the core box and die box on each side (left, bottom, right, and top). *Data_type*: box, optional
`-coreToLeft value`	Specifies the distance from the left of the core box to the left of the die box. *Data_type*: float, optional
`-coreToRight value`	Specifies the distance from the right of the core box to the right of the die box. *Data_type*: float, optional
`-coreToTop value`	Specifies the distance from the top of the core box to the top of the die box. *Data_type*: float, optional
`-noSnapToGrid`	Specifies that the core box (or die box) boundary will not be snapped to the nearest metal pitch even if the specified die/ core box size is not an integer multiple of the smallest metal pitch. This prevents the input floorplan boxes from being changed. *Data_type*: bool, optional

Example

- The following command changes the distance from the bottom of the core box to the bottom of the die box to 200:
  ```
  changeFloorplan -coreToBottom 200
  ```

Related Information

- `saveDesign`
- `saveFPlan`
- Floorplanning the Design chapter in the *Innovus User Guide*

changeIoConstraints

```
changeIoConstraints
[-help]
[-forceIt]
[-ioOrder {clockwise | counterclockwise | default} | -row string]
[-spacing num | -removeSpacing]
[-endSpace num | -removeEndSpace]
[-firstCellSpacing float| -removeFirstCellSpacing]
[-removeAll]
```

Changes the constraints of an I/O row defined in the I/O constraint file. If you do not specify the name of the I/O row, the command modifies all the I/O rows. You can use this command to move a IO pad .

Parameters

`-endSpace num`	Specifies the space, in µmeters, between the corner pad and the last I/O pad for the specified side of the row. *Default*: `0` *Data_type*: float, optional
`-firstCellSpacing float`	
	Specifies the distance, in µmeters, from the first cell to the beginning of the I/O row, based on the I/O order. *Data_type*: float, optional
`-forceIt`	Forces the I/O constraints to run on a non I/O row flow. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes type and default information for each `changeIoConstraints` parameter. For a detailed description of the command and all of its parameters, use the man command: `man changeIoConstraints`.
`-ioOrder {clockwise | counterclockwise | default}`	
	Changes the I/O placement order on all the rows. **Note:** This parameter is ignored if you specify a row name. *Default*: `default` *Data_type*: string, optional
`-removeAll`	Removes all the existing constraints on the I/O's. Evenly distributes all the I/O's in the rows. *Data_type*: bool, optional
`-removeEndSpace`	Removes the end space of the row. *Data_type*: bool, optional
`-removeFirstCellSpacing`	
	Removes the spacing of the first cell. *Data_type*: bool, optional
`-removeSpacing`	Remove the spacing constraint from I/O cells on a row.
`-row string`	Specifies the name of the row to modify the I/O constraints. *Default*: 1 *Data_type*: string, optional
`-spacing num`	Specifies the global spacing, in µmeters, for all the I/O's on a row. **Note:** `changeIoConstraints -ioOrder -spacing num` does not modify the spacing of corner I/O pads. *Default*: 0 *Data_type*: float, optional

Example

- The following command removes the spacing constraint from all I/O rows defined in the I/O constraint file:
  ```
  changeIoConstraints -removeSpacing
  ```

Related Information

- `createIoRow`
- `snapFPlanIO`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Performing I/O Row Based Pad Placement
- Data Preparation chapter in the *Innovus User Guide*
  - Creating an I/O Assignment File

check_macro_place_constraint

```
check_macro_place_constraint
[-help]
```

Checks the constraints of macros which will be honored during the macro placer. To define the constraints of macros, use the `set_macro_place_constraint` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `check_macro_place_constraint` parameter. For a detailed description of the command and all of its parameters, use the man command: `man check_macro_place_constraint`

Example

- The following command checks the constrains of macros:
  ```
  check_macro_place_constraint
  ```

Related Information

- `create_pg_model_for_macro_place`
- `get_macro_place_constraint`
- `reset_macro_place_constraint`
- `set_macro_place_constraint`
- `write_macro_place_constraint`
- Floorplanning the Design chapter in the *Innovus User Guide*
- Using the Mixed Placer chapter in the *Innovus User Guide*

check_ufc

```
check_ufc
[-help]
fileName
[-include_all_rules]
[-report_file string]
[-report_unplaced_block]
[-selected_rules string]
```

Checks the floorplan based on the user specified UFC rules defined in the UFC file and reports violations. These violations can be seen in the *Violation Browser*.

Note: The Unified Floorplan Constraints (UFC) methodology is a powerful Innovus capability that can define several rules on floorplan objects, check specified rules, and report the violations with rule name as well as severity. The violations can be shown in the violation browser. UFC also provides the ability to automatically fix specific violations. It can add routing blockages, snap macro location, adjust core shape to fix corresponding violations. In addition, UFC also provides the ability to exclude rules from checking for specified objects.

Usage Model:

- Load the Design.
- Create a UFC file with floorplan rules.
- Run UFC checking. (`check_ufc`)
- Fix reported violations. (`fix_floorplan`)

Note: The check_ufc command honors the `setFPlanMode -no_cut_row` parameter that specifies if rows are not required to be cut before operations. With `setFPlanMode -no_cut_row false` or not specified, the check_ufc command will work based on the current row shape. If there are rules for rows in the UFC file and the rows are not cut before, there will be many wrong violations reported out. So the rows for objects like insts, place blockages are expected to cut out before using the `check_ufc` command. If rows are not cut before the check_ufc command is run, with `setFPlanMode -no_cut_row true`, the tool identifies the rows have not been cut before. It calls the `cutRow` command to get a cut row shape during the `check_ufc` command. The `cutRow` command cuts the rows for insts, hinsts, place blockages, route blockages, power domains, groups, or bumps by default. The check_ufc command will work based on the cut row shape. The actual row shape is not changed after these operations. You can still get the expected check result for row's rule without running the `cutRow` before.

Parameters

fileName	Specifies the name of file containing the rules or constraints. *Data_type*: string, required
-help	Prints a brief description that includes type and default information for each `check_ufc` parameter. For a detailed description of the command and all of its parameters, use the man command: `man check_ufc`
-include_all_rules	Enables the tool to report the summary of violations for all rules. *Data_type*: bool, optional
-report_file *string*	Specifies the name of the file that saves the report summary. *Data_type*: string, optional
-report_unplaced_block	Enables the tool to show the number of unplaced macros in summary. *Data_type*: bool, optional

-selected_rules *string*	Specifies that only the selected rules will be checked.
	Note: The `-selected_rules` parameter supports the variable format for specifying the rule list.
	For example, you can define a rule in the UFC file and then use the `check_ufc` command to check the rule.
	`set sanity_rules 'A C'`
	`check_ufc file.ufc -selected_rules $sanity_rules`
	Here, rule A and C will be checked
	Data_type: string, optional

Example

- The following command checks the floorplan based on the *DMTB_ENC.ufc* file that contains the user specified UFC rules:

```
check_ufc DMTB_ENC.ufc
```

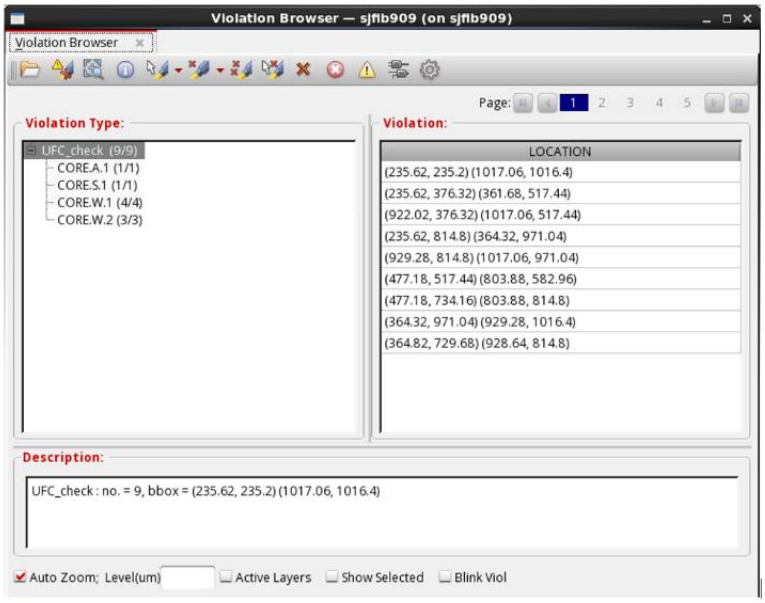
The input UFC file is like this:

```
set_area_rule -name CORE.A.1 -obj [available_sites] {.area < 500} \
  -description {The area of available sites should be lesser than 500}
set_width_rule -name CORE.W.1 -obj [available_sites] -direction horizontal -type simple {.width > 200} \
  -description {The horizontal simple width of available sites must be larger than 200}
set_width_rule -name CORE.W.2 -obj [available_sites] -direction vertical -type simple {.width > 100} \
  -description {The vertical simple width of available sites must be larger than 100}
set_spacing_rule -name CORE.S.1 -obj [base_cells -if {.name ==ram_256x16A}] -ref [base_cells -if {.name == rom_512x16A}] -direction
vertical {on_grid (.spacing-0.1,0.2)} \
  -description {The vertical spacing between macro ram_256x16A and rom_512x16A must be 0.2*n + 0.1}
set_spacing_rule -name CORE.S.2 -obj [base_cells -if {.name ==rom_512x16A}] -ref [base_cells -if {.name == ram_128x16A}] -direction
vertical {.spacing > 10} \
  -description {The vertical spacing between macro rom_512x16A and ram_128x16A must larger than 10}
```

Following is an example of report summary of UFC checking after running the check_ufc command. The violations can be seen in the Violation Browser.

```
--------------------------Summary of UFC Checking ---------------------------------------
RuleName Count  Description                                                          ViolatedObjects
........ .....  ...........................                                          ......................
CORE.A.1   1    The area of available sites should be lesser than 500                available_sites
CORE.S.1   1    The vertical spacing between macro ram_256x16A and rom_512x16A must be 0.2*n + 0.1
 DTMF_INST/RAM_256x16_TEST_INST/RAM_256x16_INST
CORE.W.1   4    The horizontal simple width of available sites must be larger than 200       available_sites
CORE.W.2   3    The vertical simple width of available sites must be larger than 100         available_sites
```

The violations can be seen in the Violation Browser.



- The following command checks the floorplan based on the *DMTB_ENC.ufc* file and includes all the rules:

```
check_ufc -include_all_rules
```

Following is an example of report summary:

```
--------------------------Summary of UFC Checking ---------------------------------------
RuleName Count  Description                                                          ViolatedObjects
```

```
........ .....  ............................                                                  ........................
CORE.A.1   1    The area of available sites should be lesser than 500                        available_sites
CORE.S.1   1    The vertical spacing between macro ram_256x16A and rom_512x16A must be 0.2*n + 0.1
 DTMF_INST/RAM_256x16_TEST_INST/RAM_256x16_INST
CORE.W.1   4    The horizontal simple width of available sites must be larger than 200        available_sites
CORE.W.2   3    The vertical simple width of available sites must be larger than 100          available_sites
CORE.S.2   0    The vertical spacing between macro rom_512x16A and ram_128x16A must be larger then 10
```

- The following command checks the floorplan based on the *DMTB_ENC.ufc* file and includes all the rules:
  ```
  check_ufc -include_all_rules -report_unplaced_block
  ```

  Following is an example of report summary:

  ```
  -------------------------Summary of UFC Checking ---------------------------------------
  RuleName Count  Description                                                               ViolatedObjects
  ........ .....  ............................                                              ........................
  CORE.A.1   1    The area of available sites should be lesser than 500                     available_sites
  CORE.S.1   1    The vertical spacing between macro ram_256x16A and rom_512x16A must be 0.2*n + 0.1
   DTMF_INST/RAM_256x16_TEST_INST/RAM_256x16_INST
  CORE.W.1   4    The horizontal simple width of available sites must be larger than 200    available_sites
  CORE.W.2   3    The vertical simple width of available sites must be larger than 100      available_sites
  CORE.S.2   0    The vertical spacing between macro rom_512x16A and ram_128x16A must be larger then 10
  **WARN: (IMPFP - 10100): There are 1 unplaced blocks. { DTMF_INST/PLLCLK_INST}. Tool doesn't check and fix violations for these blocks.
  ```

Related Information

- fix_floorplan
- Floorplanning the Design chapter in the *Innovus User Guide*

checkFPlan

```
checkFPlan
[-help]
[-outFile fileName]
[-reportUtil]
```

Checks the quality of the floorplan to detect potential problems before the design is passed on to other tools. This highlights the cells in the design display area with violation markers, where applicable. Use this command after specifying the floorplan data or running an initial floorplan. Run `checkFPlan` on your final floorplan file.

Notes:

- Parameters that you specify with the `setFPlanMode` command are used automatically whenever you run the `checkFPlan` command.
- Innovus honors the FinFET grid and automatically checks for the FinFET definition. If it detects a FinFET definition it enables support and sets the placement grid as FinFET grid by default. Since the height of all placeable objects must be a multiple of FinFET grid and the origin of these objects must be snapped to the FinFET grid. The checkFPlan command checks if all the supported objects (die box, core box, standard cell rows, I/O rows, blocks, IO pads, and standard cells) are aligned with FinFET grid and reports DRC violations.
- The `checkFPlan` command can check if the row's orientation is correct according to related tech site's SYMMETRY definition. It flags when Y SYMMETRY is missing from techlef CORE site definition.
- The `checkFPlan` command checks the pin snapping result.
  - For min width pins, it checks whether the pin center is snapped to the routing track with the same color.
  - For fat pins, it checks whether the pin center is snapped to the routing track and decides if the pin center should snap to the same or opposite colored routing track.
- The `checkFPlan` command checks if power stripes are on routing track for 10nm DPT layers (Metal1, Metal2, Metal3). It also checks power vias upper and lower metal shapes if they are not entirely covered by other power stripes on the same layers. Additionally it also checks if multi-height rows conflict with PG rails. In case of a conflict, a violation is reported.
- You can use the `checkFPlan` command to check if there are any existing overlaps between group boxes and power domain boxes as the box for a group should not physically overlap with the box for a power domain.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `checkFPlan` parameter. For a detailed description of the command and all of its parameters, use the man command: `man checkFPlan`
`-outFile` *`fileName`*	Outputs detailed information for the specified checks. *Data_type:* string, optional
`-reportUtil`	Reports target utilization (TU) and effective utilization (EU) for the entire design, fences, and regions (partitions, power domains, and regular fences). *Data_type:* bool, optional

Example

- The following command checks the floorplan and writes the detailed information to the details.txt file. It also reports the final core utilization:
  ```
  checkFPlan -outFile details.txt -reportUtil
  ```

Related Information

- `setFPlanMode`
- `check_design`
- `checkFPlanSpace`
- Floorplanning the Design chapter in the *Innovus User Guide*

checkFPlanSpace

```
checkFPlanSpace
[-help]
[-outfile filename]
[-constraints filename | -trackUtil layerID value layerID value ...]
[-clearMarker]
```

Checks the floorplan for spacing rule violations and saves the violation information in a report. A marker is displayed at each violation spot. You can use the command at any point in the design flow.

Parameters

`-clearMarker`	Clears all space violation markers. *Data_type*: bool, optional
`-constraints filename`	Specifies the file that contains the spacing rules. A constraint file is a text file that can contain a list of seeds to use when generating fences and module guides, and basic relative, spacing and orientation constraints to follow during macro placement. *Data_type*: string, optional
`-help`	Outputs a brief description that includes type and default information for each `checkFPlanSpace` parameter. For a detailed description of the command and all of its parameters, use the man command: `man checkFPlanSpace`
`-outfile filename`	Writes the spacing rule violation information to the specified file. *Data_type*: string, optional
`-trackUtil layerID value layerID value ...`	
	Specifies the track utilization value on each routing layer from bottom to top, for example, `Metal1 value Metal1 value`. If the layer track utilization value is not specified, the default value is used. **Note:** The `checkFPlanSpace` command supports the model based space checker. It takes user provided track utilization numbers and technology information stored in the LEF file to estimate required minimum channel width between placed macros. This estimated minimum channel width (demand) is then compared to actual channel width (supply) in floorplan. A violation marker is displayed in GUI over the channel in which the supply is less than the demand. *Data_type*: list, optional

Example

- The following command checks the floorplan for spacing rule violations and saves the violation information in a `details.txt` report file;
  ```
  checkFPlanSpace -outFile details.txt
  ```

Related Information

- `check_design`
- `checkFPlan`
- Floorplan Menu chapter in the *Innovus Menu Reference*
  - Check Floorplan Space Rule

checkMacroLLOnTrack

```
checkMacroLLOnTrack
[-help]
-useLayerOffset
-useM2M3Track
```

By default, checks whether the lower left coordinates (llx, lly) of hard macros are at the intersection of Metal1/Metal2 tracks, and reports an error in Innovus if they are not. This is based on the assumption that Metal1 and Metal2 are both specified in opposite directions in a 65nm process. You can use this command after the floorplan is ready and want to make sure hard macro is on tracks.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `checkMacroLLOnTrack` parameter. For a detailed description of the command and all of its parameters, use the man command: `man checkMacroLLOnTrack`
`-useLayerOffset`	Checks pin alignment in a block against the layer offset of the design library. *Data_type*: bool, optional
`-useM2M3Track`	Checks the location of the macro against Metal2/Metal3 tracks if both Metal1 and Metal2 are specified in the same horizontal direction in a 40nm process. *Data_type*: bool, optional

Example

- The following command checks whether the lower left coordinates (llx, lly) of hard macros are at the intersection of Metal1/Metal2 tracks:
  ```
  checkMacroLLOnTrack -useLayerOffset -useM2M3Track
  ```

Related Information

- `check_macro_place_constraint`
- `checkFPlan`
- `checkFPlanSpace`
- Floorplanning the Design chapter in the *Innovus User Guide*

convertFenceToLef

```
convertFenceToLef
[-help]
hinst
[ptnName]
```

Converts a specified hierarchical instance or partition name to a LEF block. Use this command after floorplanning the design.

Note: This command runs the `savePartition` command for the specified partition as if it was at the top-level. This will overwrite any partition or hierarchical instance that is specified in the .lef file in the top partitioned subdirectory. In addition, the *ptnname.def* file will overwrite the partition's *ptn.def* subdirectory file.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `convertFenceToLef` parameter. For a detailed description of the command and all of its parameters, use the man command: `man convertFenceToLef`
hinst	Specifies the hierarchical instance cell name. *Data_type*: string, required
ptnName	Specifies the partition name. If this is not specified, the command uses the hierarchical instance cell name as the partition name. *Data_type*: string, optional

Example

The following command converts the results_conv hierarchical instance to a LEF block.

```
convertFenceToLef results_conv
```

Related Information

- `savePartition`
- Floorplanning the Design chapter in the *Innovus User Guide*

copyObject

```
copyObject
[-help]
```

Copies the selected placement or routing blockages. Use the `pasteObject` command to paste the copied object to the required location. You can use the `copyObject` command any time after floorplanning the design. You can also copy selected placement and routing blockages from the GUI by using the Copy tool. To access the Copy tool, choose *Edit - Copy* from the menu bar or press `F3` after clicking the *Copy* widget ( ) on the toolbar.

Note: The `copyObject` command currently works only for placement and routing blockages. Use the `editCopy` command to copy wires or vias.

Parameters

`-help`	Prints a brief description of the `copyObject` command. For a detailed description of the command, use the man command: `man copyObject`

Example

- The following commands copy and paste the selected placement and routing blockages:

  ```
  copyObject
  pasteObject -loc 10 20
  ```

Related Information

- `pasteObject`
- `editCopy`
- Floorplanning the Design chapter in the *Innovus User Guide*

create_bump_grid

```
create_bump_grid
[-help]
-grid_name string
-layers string
-origin {x y}
-pitch {x y}
```

Creates a bump grid. A bump grid is a grid for bump/TSV placement that helps in guiding bump placement and bump alignment.

Note:

- When you use the `savePartition` command, the tool recalculates the offset so that a bump can be aligned after assemble.
- The `saveFPlan` command saves the bump grid definition into *.fp file.
- The `loadFPlan` command loads the bump grid file while reading the design.

Parameters

`-grid_name string`	Specifies the bump grid name. *Data_type*: string, required
`-help`	Outputs a brief description that includes the type and default information for each `create_bump_grid` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_bump_grid`
`-layers string`	Specifies the layer names. You can specify the layer of a relative bump pin for this option. *Data_type*: string required
`-origin {x y}`	Specifies the coordinates of the origin of the bump grid. *Data_type*: point, required
`-pitch {x y}`	Specifies the spacing between two bumps. The first value (x) defines the horizontal spacing and the second value (y) defines the vertical spacing. *Data_type*: point, required

Example

- The following command creates a bump grid named myGridFront_9x9:
  ```
  create_bump_grid -grid_name myGridFront_9x9 -layers {UBM UBM1} -origin {50 50} -pitch {9 9}
  ```

Related Information

- `delete_bump_grid`
- Floorplanning the Design chapter in the *Innovus User Guide*

create_pg_model_for_macro_place

```
create_pg_model_for_macro_place
[-help]
{-file string}
```

Enables you to create pg models for the concurrent macro placement. This command saves PG modeling information to a Tcl file from the existing floorplan. This file is used to generate better congestion and wire-length modeling while placing the macros. You can use this command after the existing floorplan has been power planned.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `create_pg_model_for_macro_place` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_pg_model_for_macro_place`
`-file string`	Specifies the name of the file to which PG model details are written. *Data_type*: string, required

Example

After restoring the initial DB with power mesh, the following command can be used to create pg models for macro placement;

```
create_pg_model_for_macro_place -file golden_mimic_power_mesh.tcl
```

Related Information

- `check_macro_place_constraint`
- `get_macro_place_constraint`
- `reset_macro_place_constraint`
- `set_macro_place_constraint`
- `write_macro_place_constraint`
- Floorplanning the Design chapter in the *Innovus User Guide*
- Using the Mixed Placer chapter in the *Innovus User Guide*

create_relative_floorplan

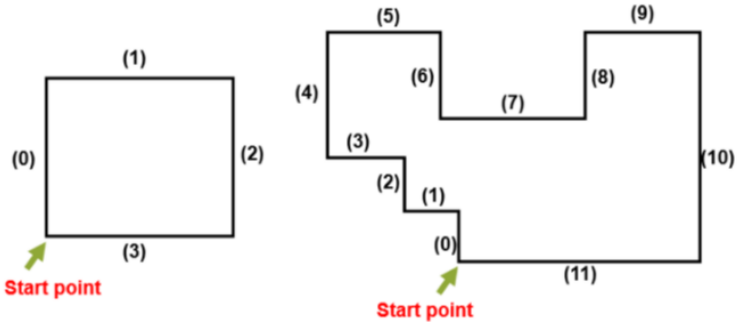
```
create_relative_floorplan
[-help]
{{{-place obj_name_list | -reshape obj_name_list} [-ref ref_obj_name_list]}}
[-place obj_name_list {-ref_type {{object core_boundary die_boundary}...}
 -horizontal_edge_separate {{ref_edge_horizontal y_offset obj_edge_horizontal}...}
 -vertical_edge_separate {{ref_edge_vertical x_offset obj_edge_vertical}...}}
[[-bbox {{none target reference both}...}]
 [-orient {{R0 R90 R180 R270 MX MY MX90 MY90}...}]]]
[-reshape obj_name_list { -dimension {{width height}...} }
[ [-fixed_edges {{edge1 edge2}...}] [-density {value}] [-offset {value}]]]
```

The `create_relative_floorplan` command captures and defines the placement relationship of floorplan objects independently from the actual coordinates in a floorplan, and resizes modules or blackboxes based on other floorplan objects, even outside the core boundary. You can use this command after importing the design.

Note: The `create_relative_floorplan` command accepts an array of objects. You can capture and define the placement relationship of an array of floorplan objects using just one command.

Note: You can use `writeFPlanScript -sections relativefplan` to save relative floorplan info.

Note: The edges are numbered as 0,1,2,3... from the low left corner in the clockwise direction. The left edge closest to the origin that has the smallest x,y coordinates is taken as the first zero edge. An even edge number donates a vertical edge and an odd edge number donates a horizontal edge. The following figure shows how to count the edges for a rectangle and rectilinear object.



Parameters

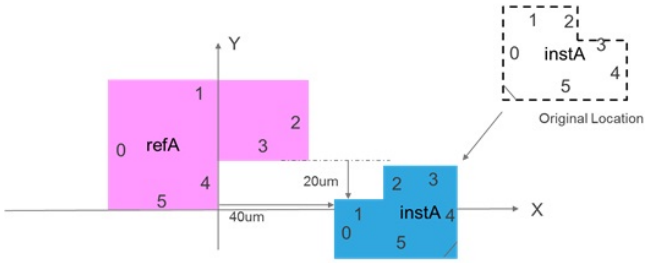
`-bbox {none target reference both}`	
	Specifies whether to use a bounding box instead of the rectilinear shape of target or reference object. *Default*: `none` *Data_type:* enum_list, optional
`-density value`	Specifies the target utilization (TU). By default, reshaping the object will keep the current TU. *Data_type:* string, optional
`-dimension {width height}`	Specifies the width or height of the referenced object as the reference dimension. *Default*: `width` *Data_type:* enum_list, optional
`-fixed_edges {edge1 edge2}`	Specifies that the reshape object will be stretched while fixing two perpendicular edges. *Default*: `{-1 0}` *Data_type:* string, optional
`-horizontal_edge_separate {ref_edge_horizontal y_offset obj_edge_horizontal}`	
	Specifies the vertical spacing between the horizontal edge of the target and reference objects. *Data_type:* string, optional
`-help`	Outputs a brief description that includes the type and default information for each `create_relative_floorplan` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_relative_floorplan`
`-offset value`	Specifies the offset value from the reference object in the specified dimension. *Data_type:* string, optional
`-orient {R0 R90 R180 R270 MX MY MX90 MY90}`	
	Specifies the orientation of the target object. *Default*: `R0` *Data_type:* enum_list, optional
`-place obj_name_list`	Specifies the target object to place. It can be hInst, inst, group, power_domain, pin_guide, blockages, or port. *Data_type:* string, optional
`-ref ref_obj_name_list`	Specifies the reference object name. It can be hInst, inst, group, power_domain, pin_guide, blockages, port, or pin. It cannot be used with `-ref_type core_boundary` or `-ref_type die_boundary`. *Data_type:* string, optional
`-ref_type {object core_boundary die_boundary}`	
	Specifies the type of reference. *Default:* `object` *Data_type:* enum_list, optional
`-reshape obj_name_list`	Specifies the target object to reshape. It can be hInst,group, power_domain,pin_guide, or blockages. *Data_type:* string, optional
`-vertical_edge_separate {ref_edge_vertical x_offset obj_edge_vertical}`	
	Specifies the horizontal spacing between the vertical edge of the target and reference objects. *Data_type:* string, optional

Examples

Placing an Instance According to a Rectilinear Object

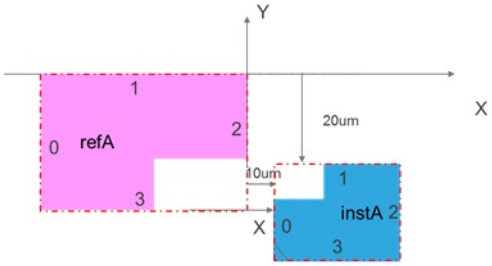
- The following command places an object with respect to a rectilinear object:

```
create_relative_floorplan -place instA -ref_type object -ref refA -orient MY \
        -horizontal_edge_separate {3 -20 1} -vertical_edge_separate {4 40 0}
```



- The following command places an object with respect to the bounding box of a rectilinear object:

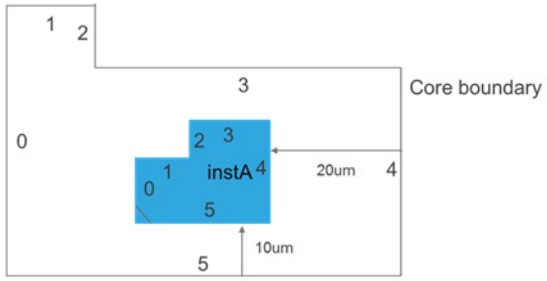
```
create_relative_floorplan -place instA -ref_type object -ref refA
        -horizontal_edge_separate {1 -20 1} -vertical_edge_separate {2 10 0} -bbox both
```



Placing an Instance According to the Core Boundary

- The following command places an object with respect to the core boundary:

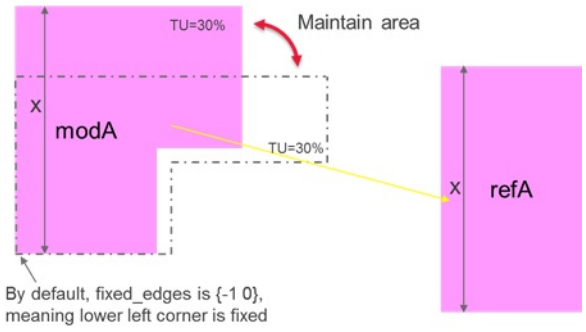
```
create_relative_floorplan -place instA -ref_type core_boundary \
       -horizontal_edge_separate {5 10 5} -vertical_edge_separate {4 -20 4}
```



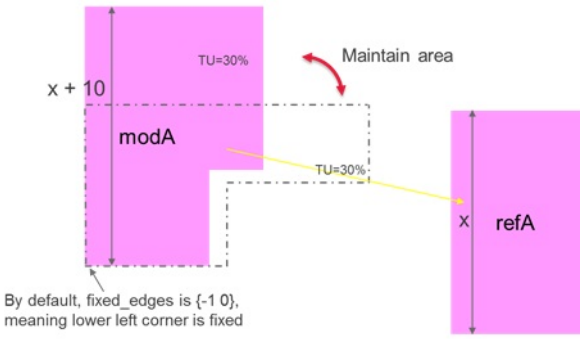
Reshaping an Instance in a Reference Dimension while Maintaining its Current Area

The following commands reshape an object based on a specified height while maintaining its current area:

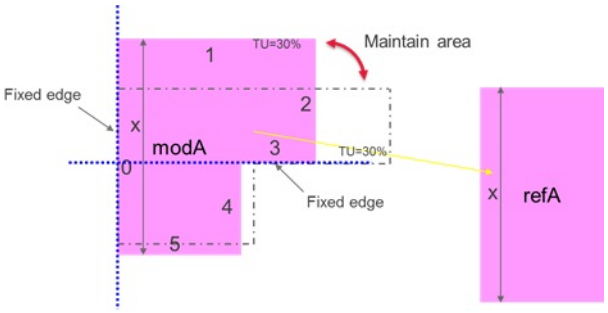
- `create_relative_floorplan -reshape modA -ref refA -dimension height`



- `create_relative_floorplan -reshape modA -ref refA -dimension height -offset 10`

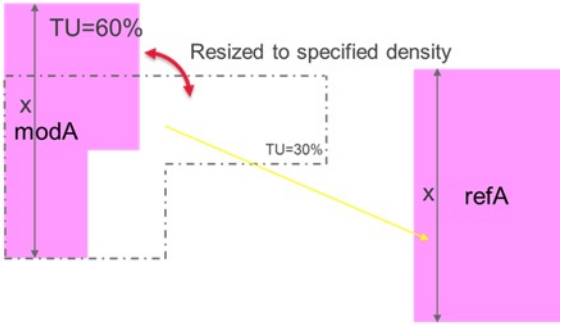


- `create_relative_floorplan -reshape modA -ref refA -fixed_edges {0 3} -dimension height`



Reshaping an Instance while Maintaining a Target Utilization Value

- The following command reshapes an object based on a specified height maintaining a target utilization value:
  `create_relative_floorplan -reshape modA -ref refA  -dimension height -density 0.6`



Placing multiple floorplan objects

- The following command places obj1, obj2, and obj3:
  `create_relative_floorplan -place {obj1 obj2 obj3} -ref {ref1 ref2 ref3} -ref_type {reftype1 reftype2 reftype3} -`

```
horizontal_edge_separate {-1 0 -1,-1 0 -1,-1 0 -1} -vertical_edge_separate {0 -100 2,0 -100 2,0 -100 2} -bbox {both target target} -
orient {MX90 R90 R90}
```

Related Information

- delete_relative_floorplan
- Floorplanning the Design chapter in the *Innovus User Guide*

createAbuttedFPlan

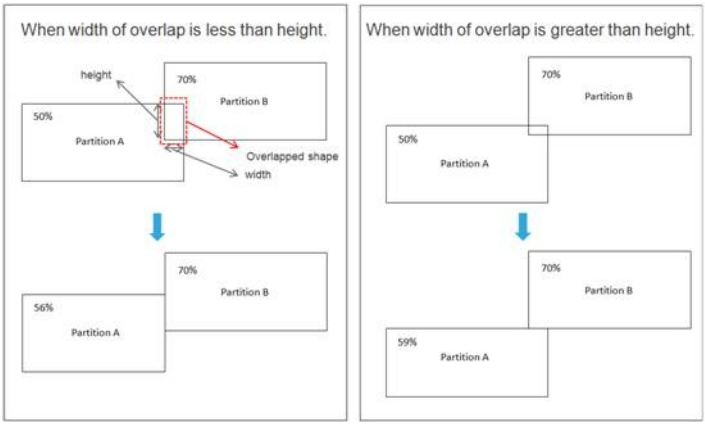
```
createAbuttedFPlan
[-help]
[-cutToFixOverlap]
[-honorPlaceBlockage blockageName]
[-ptnToCoreGap distance]
[-ptnToPtnGap distance]
{-hinsts hierarchicalInstanceList | -selected | -all}
```

Creates an abutted floorplan from an existing floorplan by fixing gaps between:

- Partitions
- Partition and core boundary
- Partition and macro when there is no overlap between the partition and the macro

Note:

- The `createAbuttedFPlan` command honors place blockage when the blockage has no overlap with the partition. It does not honor doughnut and master/clone partitions.
- When you use the `createAbuttedFPlan` command, partitions grow. The order in which partitions grow is top edge,bottom edge,left edge,right edge. Also, some partitions shrink when the `-ptnToPtnGap` option is used. The order in which partitions shrink is left edge, top edge, right edge, bottom edge.
- When there is an overlap between two partitions, if the width of the overlapped shape is less than the height, the partition with the lower TU shrinks horizontally. However, if the width of overlapped shape is bigger than the height, the partition with the lower TU shrinks vertically.

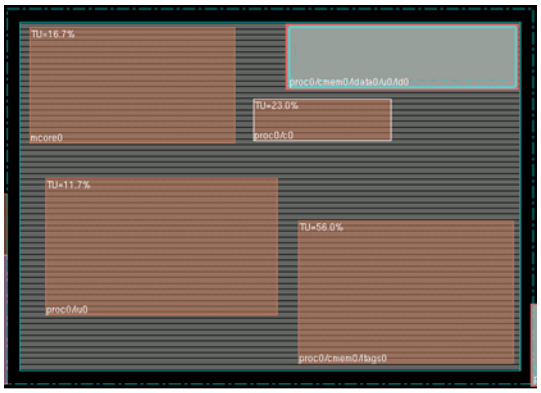


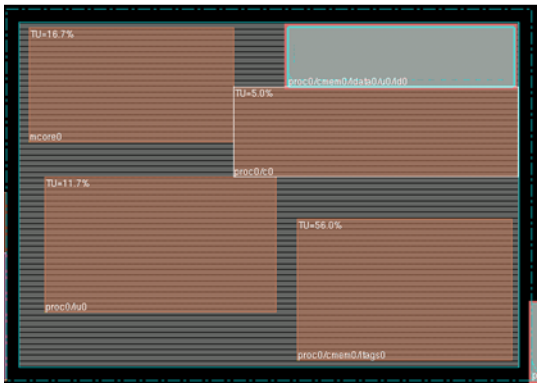
Parameters

`-help`	Outputs a brief description that includes the type and default information for each `createAbuttedFPlan` parameter. For a detailed description of the command and all of its parameters, use the man command: `man createAbuttedFPlan`
`-all`	Specifies to grow all partitions in the design. *Data_type*: bool, optional
`-cutToFixOverlap`	Cuts the overlapped partitions, if specified. Otherwise, the tool will shrink the partitions instead of cutting. *Data_type*: bool, optional
`-hinsts` *hierarchicalInstanceList*	Specifies a list of hierarchical instances that will be modified. *Default*: All partitions in the design. *Data_type*: string, optional
`-honorPlaceBlockage` *blockageName*	Specifies to fill gap upto the placement blockage boundary only. If a blockage name is specified, it honors only the specified blockage name. *Default*: Does not honor placement blockage. *Data_type*: string, optional
`-ptnToCoreGap` *distance*	Specifies the required distance (in micron) between a partition boundary and and a core boundary after growing the partition. *Default*: 0 micron *Data_type*: float, optional
`-ptnToPtnGap` *distance*	Specifies the required distance (in micron) between the partitions after growing the partitions. *Default*: 0 micron *Data_type*: float, optional
`-selected`	Specifies to grow selected partitions. *Data_type*: bool, optional

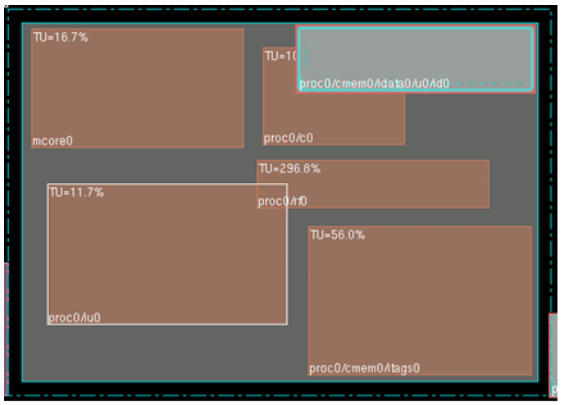
Examples

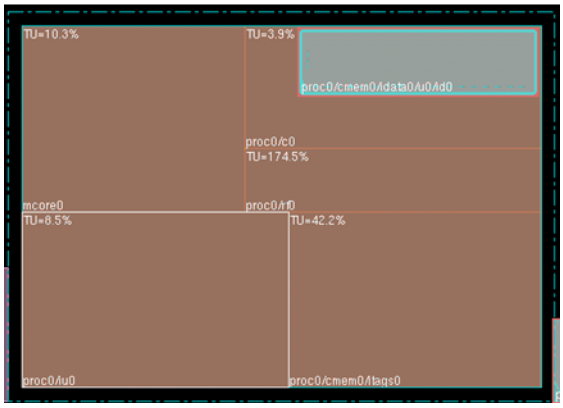
- The following command grows the selected partition
  ```
  createAbuttedFPlan -selected
  ```





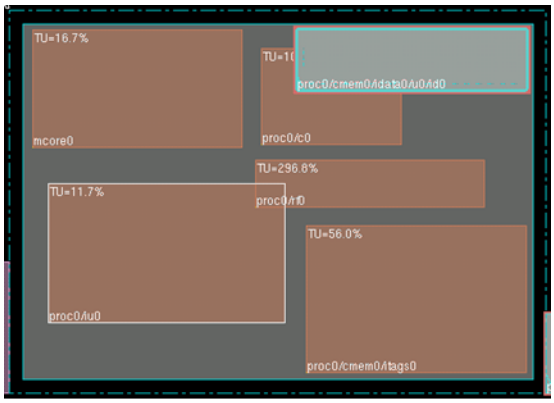
- If a partition overlaps another partition, the partition with higher target utilization keeps the original shape. The other partition shrinks to remove the overlap.
  The following command keeps the originals shape of proc0/rf0 and shrinks the other partitions to remove the overlap.
  ```
  createAbuttedFPlan -all
  ```

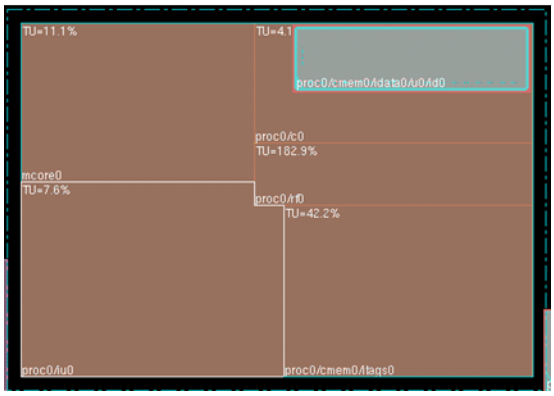




- The following command cuts the proc0/iu0 partition and keeps the original shape of the proc0/rf0 partition, which has higher utilization, to remove the overlap.

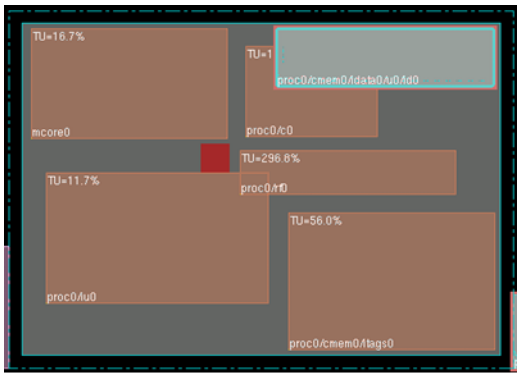
```
createAbuttedFPlan -cutToFixOverlap -all
```

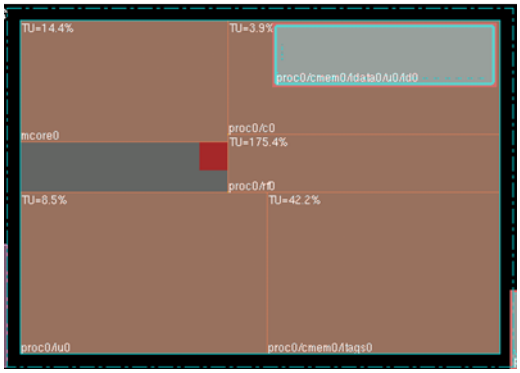




- The following command honors the place blockage which is not overlapping the partition by default.

```
createAbuttedFPlan -all -honorPlaceBlockage all
```





- The following command keeps the gap between two partitions, and between partitions and the core boundary after growing partition.
  ```
  createAbuttedFPlan -ptnToCoreGap 5 -ptnToPtnGap 5 -all -cutToFixOverlap
  ```

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

createDensityArea

```
createDensityArea
[-help]
box
density
[-name obj_name]
```

Creates a density screen area. A density screen area is a floorplan object used to assign standard cell placement density in the design area. The createDensityArea command can be used to create partial placement blockages even outside the core boundary. Use this command after importing the design.

Note: After creating the density screen area, you can double-click on the object in the design display area to open the Attribute Editor to change the density value.

Parameters

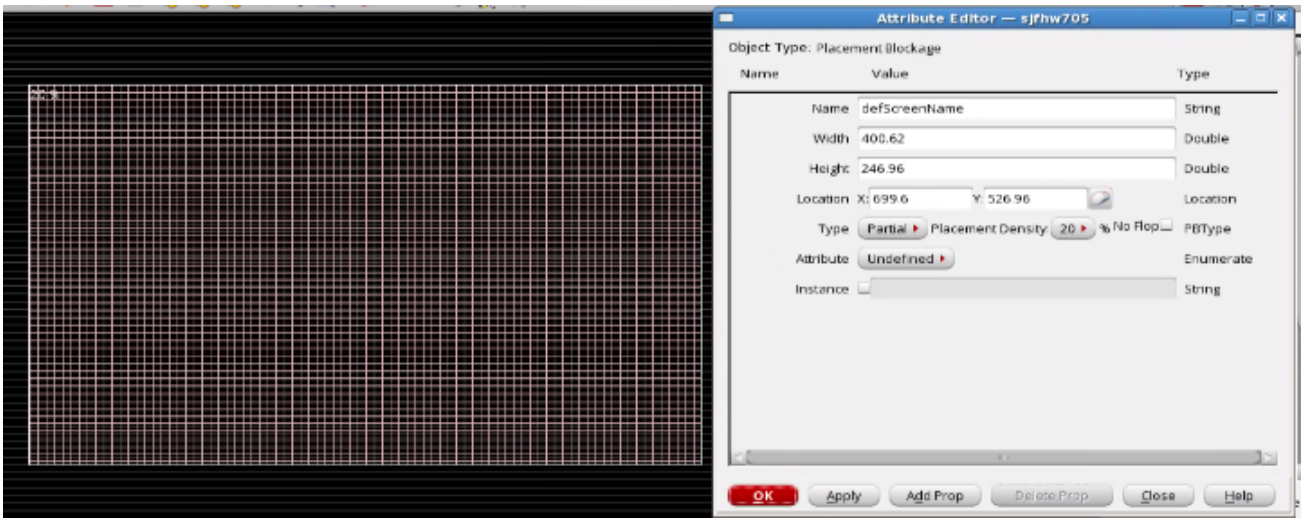
density	Specifies the maximum standard cell placement density percentage allowed in the area. Valid values are in increments of 5: 100, 95, 90, 85, 80, 75, 70, 65, 60, 55, 50, 45, 40, 35, 30, 25, 20, 15, 10, 5, 0. For example, setting a density percentage of 75 percent means that up to 75 percent of the defined density area will be used for placement. **Note:** If you enter any number that is not valid, the percentage defaults to the nearest valid value. *Maximum*: 100 *Minimum*: 0 *Data_type*: float, required
-help	Outputs a brief description that includes the type and default information for each `createDensityArea` parameter. For a detailed description of the command and all of its parameters, use the man command: `man createDensityArea`
box	Specifies the box in the coordinates of the density area. • *llx*: Specifies the lower left x coordinate of the density area. • *lly*: Specifies the lower left y coordinate of the density area. • *urx*: Specifies the upper right x coordinate of the density area • *ury*: Specifies the upper right y coordinate of the density area. *Data_type*: box, required
-name *obj_name*	Specifies the name of the density screen area. *Data_type*: string, optional

Example

The following command creates a density screen area with placement density of 20%:

```
createDensityArea 699.6 526.96 1100.22 773.92 20
```

Before placement:



After placement:

Related Information

- `deleteAllDensityAreas`
- `setSelectedDensityArea`
- Floorplanning the Design chapter in the *Innovus User Guide*

createExclusiveGroups

```
createExclusiveGroups
[-help]
[group_name]
[-gap float]
[-min_gap float]
[-exclusiveGroups string | -mutuallyExclusiveGroups string]
```

Creates exclusive regions. Exclusive regions specified for a particular group cannot be placed/overlapped with it but they can overlap on each other/ any other region in the core area.

Sample Script

Following is a sample script to create exclusive groups:

```
createInstGroup $group_name
addInstToInstGroup $group_name {insts}
createInstGroup $group_name1
addInstToInstGroup $group_name1 {insts}
createExclusiveGroups $group_name -exclusiveGroups {$groups_name1}
```

Parameters

`-exclusiveGroups string`	Specifies the groups/regions that cannot be placed/overlapped with the specified group name. For Example: If regions B,C,D are defined as exclusive regions to A. Instances from B,C,D can never be placed on A. Other regions which are not defined exclusive to region A can be placed or overlapped with region A. *Data_type*: string, optional
`-gap float`	Specifies the value of the gap, in microns, that is maintained among exclusive groups. *Default*: 5 (microns) *Data_type*: float, optional
`group_name`	Specifies the name of the created group. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `createExclusiveGroups` parameter. For a detailed description of the command and all of its parameters, use the man command: `man createExclusiveGroups`
`-min_gap float`	Specifies the value of the minimum gap, in microns, that is maintained among exclusive groups. *Data_type*: float, optional
`-mutuallyExclusiveGroups string`	Specifies the groups which need to be excluded with each other. You can independently use this option to specify and create multiple mutually exclusive groups. **Note:** If a group has already been defined as an exclusive groups, then a second setting for this group using the `createExclusiveGroups` command will give display an error that suggests you to reset the exclusive groups attributes to default value. If you want to create mutually exclusive groups, then reset the exclusive groups attributes using the `deleteExclusiveGroups` command and then create the mutually exclusive groups using the `createExclusiveGroups -mutuallyExclusiveGroups` command. *Data_type*: string, optional

Examples

- The following commands define `gr_300` as an exclusive region to gr_200:

```
createInstGroup gr_200
addInstToInstGroup gr_200 DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/acc_reg*
createInstGroup gr_300
addInstToInstGroup gr_300 DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/top_reg_*
createExclusiveGroups gr_200 -exclusiveGroups gr_300
```

- The following commands defines instance groups A, B, C to be mutually exclusive groups with a space of 10 microns between each other.

```
createExclusiveGroups -mutuallyExclusiveGroups {A B C} -gap 10
```

Related Information

- `deleteExclusiveGroups`
- `createInstGroup`
- `addInstToInstGroup`
- Floorplanning the Design chapter in the *Innovus User Guide*

createFence

```
createFence
[-help]
obj_name
box
[-floating]
```

Creates a fence for a module or a group. Use this command after importing the design. To delete fences, use the `unplaceGuide` command.

Parameters

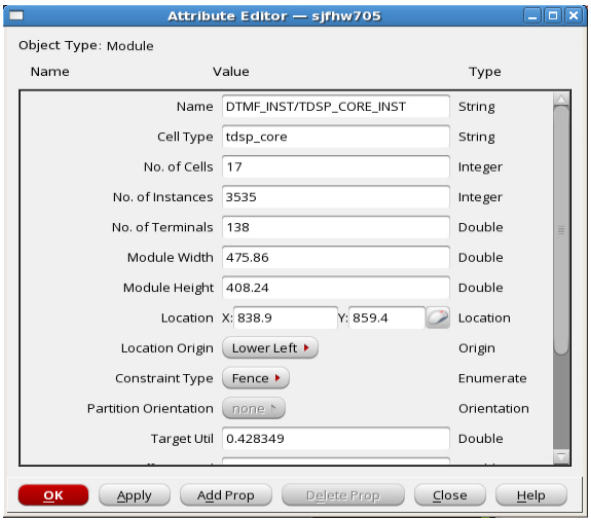
box	Specifies the constraint box of the box. • *llx*: Specifies the lower left x coordinate of the fence area. • *lly*: Specifies the lower left y coordinate of the fence area. • *urx*: Specifies the upper right x coordinate of the fence area. • *ury*: Specifies the upper right y coordinate of the fence area. *Data_type*: box, required
-floating	Creates a floating fence. The floating region/fence constraint allows the entire region/fence to move as a cluster across the core area. *Data_type*: bool, optional
-help	Outputs a brief description that includes the type and default information for each `createFence` parameter. For a detailed description of the command and all of its parameters, use the man command: `man createFence`
obj_name	Specifies the name of the module or group to be fenced. *Data_type*: string, required

Example

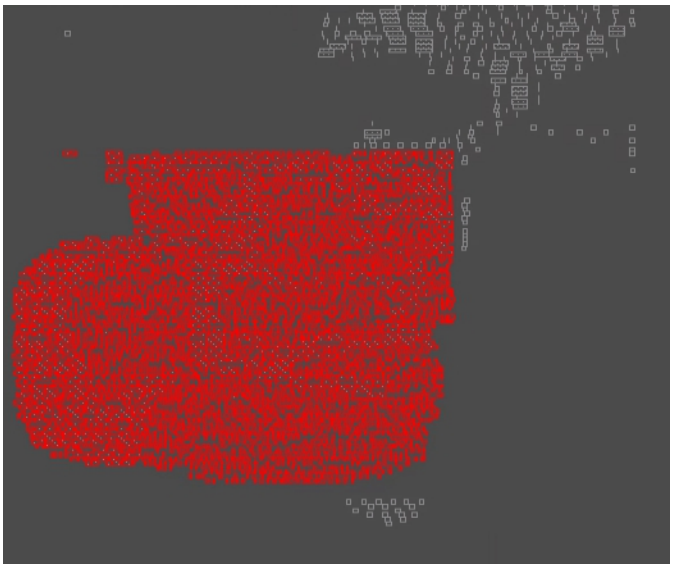
The following command places the module guide of DTMF_INST/TDSP_CORE_INST in the floorplan view and changes its status to Fence:

```
createFence DTMF_INST/TDSP_CORE_INST 838.9 859.4 1314.76 1267.64
```

Before placement (Floorplan view):



After placement (Physical view) :



Related Information

- `unplaceGuide`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Module Constraint Types
- *Floorplan Menu chapter in the Innovus Menu Reference*

createGuide

```
createGuide
[-help]
obj_name
box
```

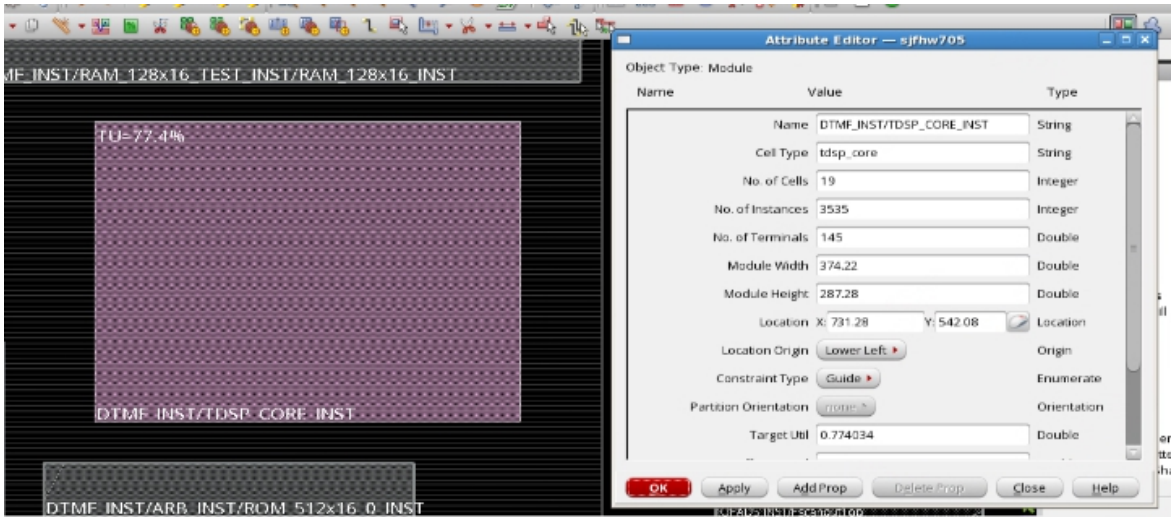
Creates a guide for a module or a group. Use this command after importing the design.

Parameters

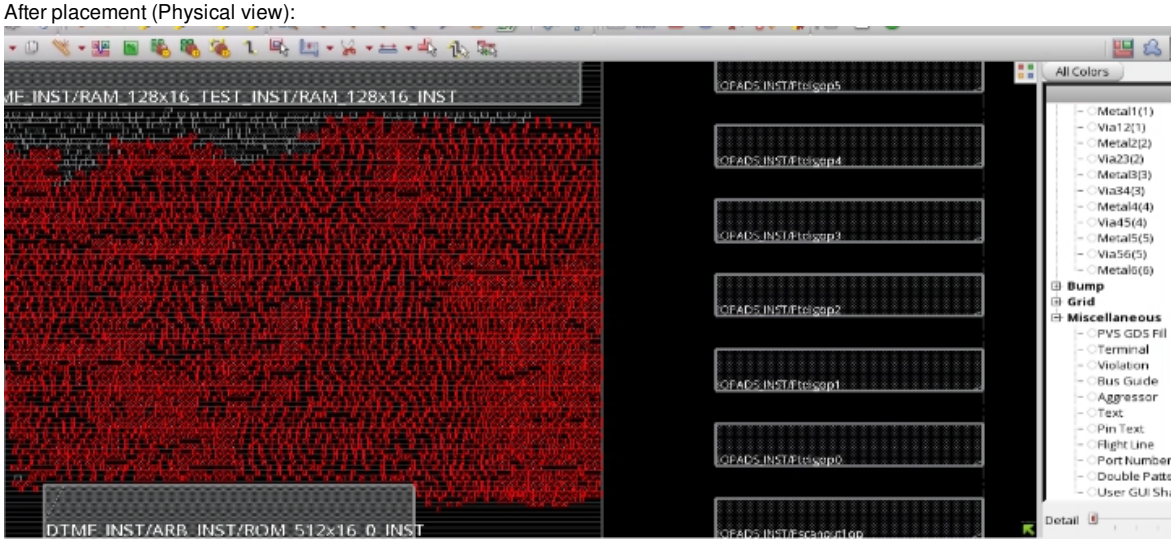
`-help`	Outputs a brief description that includes the type and default information for each `createGuide` parameter. For a detailed description of the command and all of its parameters, use the man command: `man createGuide`
obj_name	Specifies the name of the module or group to be guided. *Data_type*: string, required
box	Specifies the constraints of the box. • *llx*: Specifies the lower left x coordinate of the guide area. • *lly*: Specifies the lower left y coordinate of the guide area. • *urx*: Specifies the upper right x coordinate of the guide area. • *ury*: Specifies the upper right y coordinate of the guide area. *Data_type*: box, required

Examples

- The following command creates a guide for module of DTMF_INST/TDSP_CORE_INST:
  ```
  createGuide DTMF_INST/TDSP_CORE_INST 731.28 542.08 1105.5 829.36
  ```
  Before placement (Floorplan view) :



  After placement (Physical view):



- The following command creates a guide for group adder1:
  ```
  createGuide adder1 834.2000 1012.0000 2498.4000 3087.0000
  ```

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*
  - Module Constraint Types

createInstGroup

```
createInstGroup
[-help]
group_name
[-isPhyHier]
[-guide box | -region box | -fence box | -softGuide ]
[[-ar num] -density num ]
[-softGuide ]
[-floating | -softGuide ]
[-floating | -guide box ]
```

Creates a new instance group, even outside the core boundary. Use this command after importing the design.

Note: Instance groups are used to create a new logical hierarchy unit, and normally all power domains will associate with an instance group which always have a constraint fence. The constraint fence means standard cell belonging to this group cannot be placed outside and only standard cell belong to this group can be placed inside. Cadence recommends that this physical-logical coherence should not be violated. The coherence requirement does not only apply for instance groups, but also pertains to all hinsts and instance groups as long as they have constraint fence.
In other words, if fence A belongs to fence B in logical, then fence A must be placed inside fence B.

Parameters

`-ar num`	Specifies the aspect ratio. If this is not specified, 1 is the default. *Data_type*: float, optional
`-density num`	Specifies a placement density percentage. Use the decimal format, where 0.75 is for 75% density. *Data_type*: float, optional
`-fence box`	Specifies the coordinates of the box (*x1 y1 x2 y2*) for a *fence*. • `x1`: Specifies the lower left x coordinate of the box. • `y1`: Specifies the lower left y coordinate of the box. • `x2`: Specifies the upper right x coordinate of the box. • `y2`: Specifies the upper right y coordinate of the box. **Note**: Since a fence should not cross multiple power domains, the `createInstGroup` command should only be used to group objects in the same power domain (otherwise the design may become unplaceable). *Data_type*: box, optional
`-floating`	Creates a floating instance group. **Note:** It does not support rectilinear shapes. **Note:** You can use the `-floating` parameter with the `-fence` or `-region` parameters only. *Data_type*: bool, optional

group_name	Specifies the name of the created group. A group is a user-defined group name and is created to contain hierarchical instances, instances (leaf instances), or other groups.
	Note: In the nested partitions scenario, to add a module to be within a module, you must include the hinst name of the module in which you want the new module to be in, within the group name.
	For example,
	`createInstGroup i_a/bc`
	`addInstToInstGroup i_a/bc i_a/b`
	`addInstToInstGroup i_a/bc i_a/c`
	Note: Incorrect groupings can cause violations.
	Data_type: string, required

`-guide box`	

	Specifies the coordinates of the box (*x1 y1 x2 y2*) for a *guide*. • `x1`: Specifies the lower left x coordinate of the box. • `y1`: Specifies the lower left y coordinate of the box. • `x2`: Specifies the upper right x coordinate of the box. • `y2`: Specifies the upper right y coordinate of the box. *Data_type*: box, optional
`-help`	Outputs a brief description that includes the type and default information for each `createInstGroup` parameter. For a detailed description of the command and all of its parameters, use the man command: `man createInstGroup`
`-isPhyHier`	Specifies that the created group is a physical hierarchy. By default, the created group is not saved back to the netlist. *Data_type*: bool, optional
`-region` *box*	Specifies the coordinates of the box (*x1 y1 x2 y2*) for a *region*. • `x1`: Specifies the lower left x coordinate of the box. • `y1`: Specifies the lower left y coordinate of the box. • `x2`: Specifies the upper right x coordinate of the box. • `y2`: Specifies the upper right y coordinate of the box. **Note**: Since a fence should not cross multiple power domains, the `createInstGroup` command should only be used to group objects in the same power domain (otherwise the design may become unplaceable). *Data_type*: box, optional
`-softGuide`	Specifies that the created group is a soft guide constraint. *Data_type*: bool, required

Examples

- The following command creates group adder1:
  ```
  createInstGroup adder1
  ```
- The following command creates group gusher_4 and its placement guide at the above area location:
  ```
  createInstGroup gusher_4 -guide 2345.3 9962.1 4930.0 18244.8
  ```
- The following command creates group adder2 with 80% of the area occupied by the instance area with an aspect ratio of 2:
  ```
  createInstGroup adder2 -density .8 -AR 2
  ```

Related Information

- `addInstToInstGroup`
- `deleteAllInstGroups`
- `deleteInstFromInstGroup`
- `deleteInstGroup`
- `setInstGroupPhyHier`
- `createExclusiveGroups`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Grouping Instances

createIoRow

```
createIoRow
[-help]
{-site site_name [{-side {N | W | S | E}
[-rowMargin value]
[-beginOffset value]
[-endOffset value | -length len | -nrSites nr] } |
{-corner {BL | BR | TR | TL} [-xOffset value] [-yOffset value]}}]
[-orientation {R0 | R90 | R180 | R270}]
[-name row_name]} | {-deriveByCells} | {-deriveBySelection}
```

Creates new I/O rows and edits the existing I/O rows. You can use this command in the floorplanning stage.

Note: Before creating I/O rows for I/O row based pad placement, you need to set the I/O row flag using the `setIoFlowFlag` command.

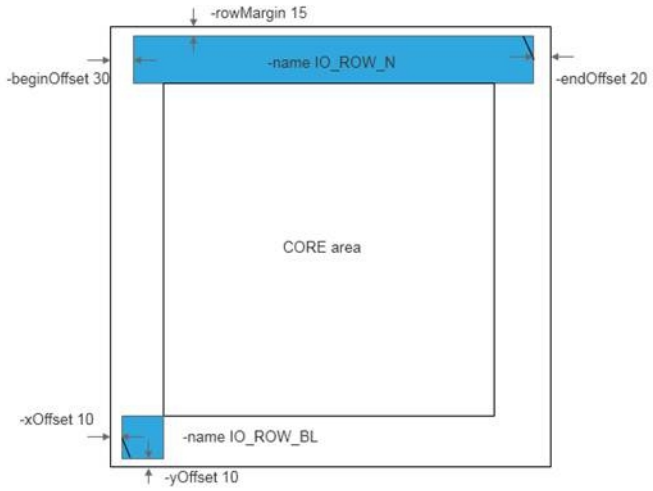
Parameters

`-beginOffset value`	Specifies the starting location of the I/O row from the die edge. • For a horizontal row, this value is the lower X value. • For a vertical row, this value is the lower Y value. *Default*: 0, starting from the die edge. *Data_type*: float, optional
`-corner {BL | BR | TR | TL}`	
	Specifies the location of the corner cell. This can be: • `BL`: Bottom Left • `BR`: Bottom Right • `TR`: Top Right • `TL`: Top Left *Default*: `BL` *Data_type*: string, optional
`-deriveByCells`	Creates an I/O row for all the I/O cells in the design. *Data_type*: bool, required
`-deriveBySelection`	Creates an I/O row only for the selected I/O cells. **Note:** When you specify this parameter, some I/O rows may overlap depending on the I/O cell distribution. *Data_type*: bool, required
`-endOffset value`	Specifies the end location of the I/O row from the die edge. • For a horizontal row, this value is the higher X value. • For a vertical row, this value is the higher Y value. *Default*: 0, ending from the die edge. *Data_type*: float, optional
`-help`	Outputs a brief description that includes type and default information for each `createIoRow` parameter. For a detailed description of the command and all of its parameters, use the man command: `man createIoRow.`
`-length len`	Specifies the length of the row, in µmeters, starting from the die edge or `-beginOffset value`. *Data_type*: float, optional
`-name row_name`	Specifies the name of the I/O row to be created. *Data_type*: string, optional

`-nrSites` *nr*	Specifies the number of sites to define the length of the row. **Note:** You can either specify `-endOffset` or `-length` or `-nrSites` each time you run the `createIoRow` command. *Data_type*: int, optional
`-orientation {R0 | R90 | R180 | R270}`	
	Specifies the orientation of the created row. If you do not specify this parameter, the default orientation for each side starting from the row on the south side is R0, rotate R90 for the next side, and so on... as follows: • R0 - South West • R90 - South East • R180 - North East • R270 - North West The default order of orientation is South, East, North, and West. *Data_type*: string, optional
`-rowMargin` *value*	Specifies the distance, in µmeters, from the row edge to the die edge. The row edge is `top' for a North side row, `bottom' for a South side row, `left' for a West side row, and `right' for an East side row. *Data_type*: float, optional
`-side {N | W | S | E}`	Specifies the side on which the row is to be created. If you do not specify the side, by default, one row is created on all sides (South, East, North, and West). *Data_type*: string, optional
`-site` *site_name*	Specifies the name of the site defined in the LEF library. *Data_type*: string, required
`-xOffset` *value*	Specifies the distance, in µmeters, from the left or right edge of the corner row to the die edge. *Data_type*: float, optional
`-yOffset` *value*	Specifies the distance, in µmeters, from the top or bottom edge of the corner row to the die edge. *Data_type*: float, optional

Example

- The following command creates I/O rows based on the existing pad placement.
  ```
  createIoRow -deriveByCells
  ```
- The following command creates I/O row at north side with offset 30 from left side, offset 20 from right side and row margin 15 from top side.
  ```
  createIoRow -name IO_ROW_N -beginOffset 30 -endOffset 20  -rowMargin 15 -side N -site IO -orientation R180
  ```
- The following command creates I/O row at bottom left corner to place corner IO with offset 10 in x and y direction.
  ```
  createIoRow -name IO_ROW_BL  -xOffset  10  -yOffset  10  -corner BL -site corner -orientation R0
  ```



Related Information

- `addIoRowFiller`
- `deleteIoRowFiller`
- `setIoFlowFlag`
- `getIoFlowFlag`
- Floorplan Menu chapter in the *Innovus Menu Reference*
  - Create I/O Row
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Performing I/O Row Based Pad Placement
- Data Preparation chapter in the *Innovus User Guide*
  - Creating an I/O Assignment Flow

createLogicHierarchy

```
createLogicHierarchy
[-help]
-cell string
[-commit]
-newHinst string
[-objects string]
[-selected]
```

Creates a new logical hierarchy for the specified hierarchical instances. The `createLogicHierarchy` command changes the netlist, CPF, SDC, Scan DEF, and floorplan constraints for the newly created hierarchy.

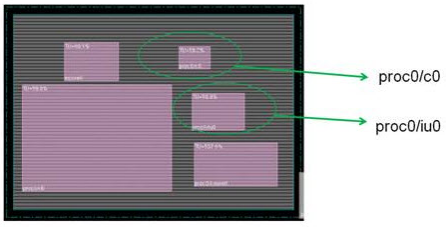
Parameters

`-help`	Outputs a brief description that includes the type and default information for each `createLogicHierarchy` parameter. For detailed description of the command and all its parameters, use the man command: `man createLogicHierarchy`.
`-cell string`	Specifies the new cell name. *Data_type*: string, optional
`-commit`	Specifies the commit group result. *Data_type*: bool, optional
`-newHinst string`	Specifies the new hier inst name. *Data_type*: string, required
`-objects string`	Specifies the list of hinst and/or instance groups. *Data_type*: string, optional
`-selected`	Specifies that grouping is done for the current selected hinsts or instant groups. *Data_type*: bool, optional

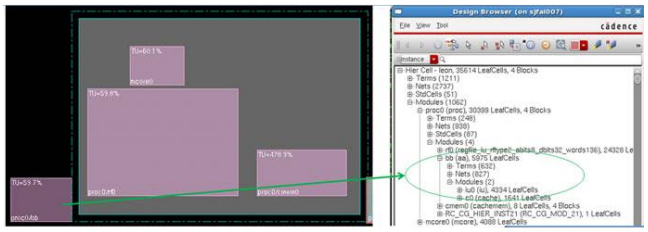
Examples

- After using the following command, you can create a new Hinst name `proc0/bb` whose cell name is `aa` and add module `proc0/c0` and `proc0/iu0` to Hinst `proc0/bb`.

  ```
  createLogicHierarchy -commit -cell aa -newHinst proc0/bb -objects {proc0/c0 proc0/iu0}
  ```

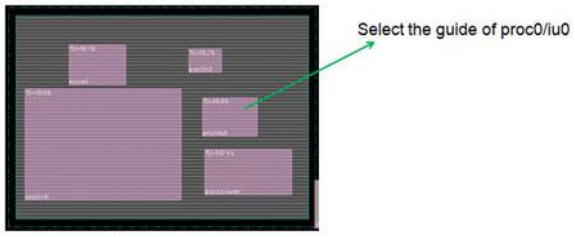
  

  The module `proc0/c0` and `proc0/iu0` will be moved to group `proc0/bb`.

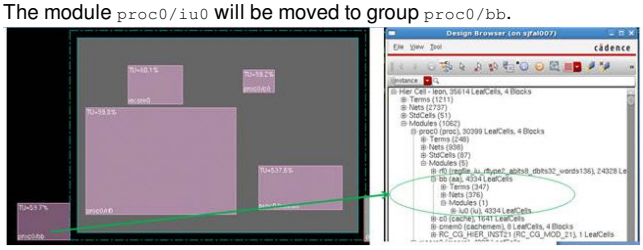
  

- After using the following command, you can create a new Hinst name `proc0/bb` whose cell name is `aa` and add selected module (it is `proc0/iu0` in this case) to Hinst `proc0/bb`.

```
createLogicHierarchy -commit -cell aa -newHinst proc0/bb -selected
```



The module `proc0/iu0` will be moved to group `proc0/bb`.



Note: It will not create group result without `-commit` option of `createLogicHierarchy` command.
Without the `-commit` option, the netlist of the design is not changed.

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*
  - Adding Logical Hierarchy Without Creating Additional Hierarchy
  - Logical Hierarchy Manipulation

createPGPin

```
createPGPin
[-help]
{-onDie {-selected | -net netName}
[-width float]
[-length float]} |
{pgPinName [-net netName]
[-geom layerName llx lly urx ury]}
[-dir {undefined | input | output}]
```

Creates a power/ground pin as per the specified coordinates of the physical shape. You can use this command to create a power/ground pin without having to first create a power stripe with the `addStripe` command.

If the `-geom` parameter is not specified, only the logical pin is created.
If the `-geom` parameter is specified, the physical pin is also created.

If a logical power/ground pin exists before this command is run, the net name will be ignored if it is specified.
If a logical power/ground pin does not exist before this command is run:

- If a net has been specified with the -net parameter, the power/ground pin will be created and attached to the net specified.
- If a net name has not been specified, the net name is assumed to be the same as the pin name. In this case, the power/ground pin will be created and attached to that nets.

Parameters

`-dir {undefined | input | output}`	
	Specifies the direction of the power/ground pin. You can specify one of the following: • `undefined`: Specifies that the direction of the pin is not defined. • `input`: Specifies that the direction of the pin is input. • `output`: Specifies that the direction of the pin is output. *Data_type:* enum, optional
`-geom layerName llx lly urx ury`	
	Specifies the geometry of the physical pin. • *`layerName`*: Specifies the layer on which the power/ground pin will be created. The *`layerId`* can range from 1 to n, where, the maximum value for n is the top most routing layer. **Note:** Layers can also be specified using layer names (for example, Metal1, Metal2, Metal3) or the LEF layer names. • *`llx`*: Specifies the lower-left x coordinate, in microns, of the power/ground pin. • *`lly`*: Specifies the lower-left y coordinate, in microns, of the power/ground pin. • *`urx`*: Specifies the upper-right x coordinate, in microns, of the power/ground pin. • *`ury`*: Specifies the upper-right y coordinate, in microns, of the power/ground pin. *Data_type:* string, optional
`-help`	Outputs a brief description that includes the type and default information for each *`createPGPin`* parameter. For a detailed description of the command and all of its parameters, use the man command: `man createPGPin`
`-length float`	Specifies the length of the power/ground pin. *Default*: Same as the width *Data_type:* float, optional
`-net netName`	Specifies the name of the net to which the power/ground pin will be attached. *Data_type:* string, optional
`-onDie`	Creates power/ground pin on the die. *Data_type:* bool, optional
`pgPinName`	Specifies the name of the power/ground pin. *Data_type:* string, optional
`-selected`	Creates power/ground pin on the die, for all selected stripes. *Data_type:* bool, optional
`-width float`	Specifies the width of the power/ground pin. *Default*: width of the stripe. *Data_type:* float, optional

Examples

- This example is for the case where a logical pin name has not been defined prior to running the command and a net name has been specified.
  The following command creates a power/ground pin named pgpin_A on layer Metal2 and attaches the pin to the net net_A. The pin coordinates are 7 (llx), 3 (lly), 17 (urx), and 23 (ury).
  ```
  createPGPin -geom Metal2 7 3 17 23 -net net_A pgpin_A
  ```
- This example is for the case where a logical pin name has not been defined prior to running the command and a net name has not been specified.
  The following command creates a power/ground pin named pgpin_B on layer Metal3 and attaches the pin to the net pgpin_B. The pin coordinates are 7 (llx), 3 (lly), 17 (urx), and 23 (ury). As the net name was not specified, the net name is assumed to be the same as the pin name.

```
createPGPin -geom Metal3 7 3 17 23 pgpin_B
```

- This example is for the case where a logical pin name has already been defined prior to running the command.
  The following command creates a power/ground pin named pgpin_C on layer Metal5. The pin coordinates are 7 (llx), 3 (lly), 17 (urx), and 23 (ury). In this case, the logical pin name was already defined prior to running the command, and so a net name is not required. Even if a net name were specified, it would have been ignored.
  ```
  createPGPin -geom Metal5 7 3 17 23 pgpin_C
  ```
- This example is for the case where the net name is not specified and the -geom parameter has not been specified.
  The following command creates a logical pin names pgpin_D. The net name is assumed to be the same as the pin name and no physical pin is created.
  ```
  createPGPin pgpin_D
  ```
- This example is for the case where the pin is created on the die attached to net vdd with a width of 2 and a length of 3.
  ```
  createPGPin -onDie -net vdd -width 2 -length 3
  ```

Related Information

- addStripe
- deletePGPin
- Power Menu chapter in the *Innovus Menu Reference*
  - Create P/G Pin

createPhysicalPin

```
createPhysicalPin
[-help]
pinName
[-allowOutsideBoundary]
[-net netName]
[-samePort]
{{ -layer layerName {-rect {x1 y1 x2 y2} | -polygon {x1 y1 x2 y2 ... xn yn}}}}
```

Adds a new physical pin-shape to a top-level term (DEF PIN). You can use this command to add more PG (power/ground) pin-shapes to an existing PG term. It will also create a new PG term if there is not already a PG term.

Note: It can add shapes to top-level signal terms, but this is not recommended. Multiple shapes for a signal term may cause flow problems (the router will not connect the different pin shapes, etc.) and should only be done with expert knowledge of the flow implications. It will not create a new signal term (You must use ECO commands to add a new signal term).

Parameters

`-allowOutsideBoundary`	Do not check whether the specified rectangle is inside the design boundary. *Data_type:* bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `createPhysicalPin` parameter. For a detailed description of the command and all of its parameters, use the man command: `man createPhysicalPin`
`-layer layerName`	Specifies the LEF (or OA) layer name, or the routing layer index value. For example, 1 for first routing layer, 2 for second routing layer. *Data_type:* string, optional
`-net netName`	Specifies the name of the net. This is ignored if a top-level term named *pinName* already exists. *Data_type:* string, optional
pinName	Specifies the name of the top-level term (DEF PIN) to add shapes to. If it does not exist, then `-net netName` is used to find a top-level term instead. If *netName* does not already have a top-level term, and it is a PG net, then a new term named *pinName* will be created for *netName*. *Data_type:* string, required
`-polygon {x1 y1 x2 y2 ... xn yn}`	Specifies the coordinates of each vertex of the shape. *Data_type:* (point)+, optional
`-rect {x1 y1 x2 y2}`	Specifies the coordinates of the rectangular shape. • *x1* Specifies the lower-left x coordinate, in microns, of the physical pin. • *y1*: Specifies the lower-left y coordinate, in microns, of the physical pin. • *x2*: Specifies the upper-right x coordinate, in microns, of the physical pin. • *y2*: Specifies the upper-right y coordinate, in microns, of the physical pin. *Data_type:* rect, optional
`-samePort`	Specifies that the physical pin shapes will be added to the first DEF PORT. If this parameter is not specified, each pin shape is created on a separate DEF port. Normally each PG shape should be on a separate port so each one is connected separately to the PG mesh at the next level up. *Data_type:* bool, optional

Examples

- The following command adds a physical pin-shape to the top-level term named VSS (e.g. DEF PIN named VSS).
  ```
  createPhysicalPin VSS -layer Metal2 -rect {7 3 17 23}
  ```

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

createPlaceBlockage

```
createPlaceBlockage
[-help]
[-pushdown]
{{[-box {x1 y1 x2 y2} | -polygon {{x1 y1} {x2 y2}...}
  | -boxList {{x1 y1} {x2 y2}...}
  | -allPartition | -allMacro ]
  [-inst inst_name | -hinst hinst_name] }
  [-name place_blockage_name | -prefixOn ]
  [{-type {hard soft partial macroOnly}}
  [-density value] [-excludeFlops ]]
  [-noCutByCore]
  [-snapToSite] }
[ {-inst inst_name | -hinst hinst_name | -allPartition | -allMacro }
  [-cover ]
  [-innerRingBySide {x1 y1 x2 y2} | -innerRingByEdge {edge1 edge2 edge3 ...}]
  [-outerRingBySide {x1 y1 x2 y2} | -outerRingByEdge {edge1 edge2 edge3 ...}] ]
```

Creates cell placement blockages that can be placed. A placement blockage is a floorplan object used to block standard cell placements. Use this command after importing the design.

Parameters

`-allMacro`	For each macro creates a placement blockage covering that macro. *Data_type*: bool, optional
`-allPartition`	For each partition creates a placement blockage covering that partition. *Data_type*: bool, optional
`-box {x1 y1 x2 y2}`	Specifies the coordinates of the blockage area. • `x1`: Specifies the lower left x coordinate of the obstruction area. • `y1`: Specifies the lower left y coordinate of the obstruction area. • `x2`: Specifies the upper right x coordinate of the obstruction area. • `y2`: Specifies the upper right y coordinate of the obstruction area. **Note:** You can enclose the coordinate values (*x1...y2*) in double-quotes, curly-braces, [list ...], or just separate them with spaces. *Data_type*: box optional
`-boxList {{x1 y1 x2 y2} ...}`	Specifies the coordinates of a list of blockage areas. *Data_type*: box_list, optional
`-cover`	By default the parameters (`-outerRingByEdge` and `-outerRingBySide`) used to create placement blockages around the outside of the object, create toroidal placement blockages. The `-cover` parameter combined with these parameters creates a solid placement blockage, otherwise the `-cover` parameter does nothing. *Data_type*: bool, optional
`-density value`	Specifies the maximum density allowed for partial or soft blockages. The value is in steps of 5 between 5 and 100, with units of percent. For example, a partial placement value of 75 means that up to 75 percent placement density is allowed in the area. **Note**: The `-density` parameter can only be used in combination with the `-type partial` or `-type soft` parameter. *Minimum*: 5 *Maximum*: 100 *Data_type*: inst, optional

`-excludeFlops`	Specifies that flops and latches will not be placed in the blockage during global placement. **Note:** The `-excludeFlops` parameter can only be used with the `-type partial` and `-density` parameters. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `createPlaceBlockage` parameter. For a detailed description of the command and all of its parameters, use the man command: `man createPlaceBlockage`
`-hinst` *hinst_name*	Specifies the name of the hierarchical instance on which the placement blockage is to be applied. *Data_type*: (hInst), optional
`-innerRingByEdge {`*edge1 edge2 edge3 ...*`}`	
	Creates a hollow ring of placement blockage around the inside of the object(s) specified. *edgeN* is the width in microns of the blockage created inside the Nth edge. *edge1* is the vertical edge starting at the left corner of the leftmost edge with the lowest y coordinate as placed, and proceeding clockwise through the other edges. **Note** : • The `-inst` , `-hinst` , `-allMacro`, or `-allPartition` parameters must be specified with this parameter. • There must be an *edgeN* specified for every edge of the object(s). • Enclose the *edgeN*s in double-quotes, curly braces, or [list ...]. *Data_type*: string, optional
`-innerRingBySide {`*x1 y1 x2 y2*`}`	
	Creates a hollow ring of placement blockage around the inside of the object(s) specified. *bottom* is the width in microns of the blockage created on bottom edges, etc. **Note**: • The `-inst`, `-hinst`, `-allMacros`, or `-allPartition` parameters must be specified with this parameter. • `The -innerRingBySide` parameter can be used with rectilinear objects. • Enclose the *bottom...left* values in double-quotes, curly braces, or [list ...] *Data_type*: box, optional
`-inst` *inst_name*	Specifies the name of the instance on which the placement blockage is to be applied. *Data_type*: (inst), optional
`-` `name` *place_blockage_name*	Specifies the name of the placement blockage. **Note:** Placement blockage names need not be unique. **Note:** If neither `-name` nor `-prefixOn` parameters are specified, then the created place blockage is named `defScreenName`. *Data_type*: string, optional
`-noCutByCore`	Specifies that placement blockages should not be truncated at the core boundary. Placement blockage will always be truncated at the die boundary. *Data_type*: bool, optional
`-prefixOn`	Specifies that the placement blockages should be named with a prefix defined by `getFPlanMode -defaultBlockageNamePrefix` attribute. Names are of the form `prefix_<int>` *Data_type*: bool, optional
`-outerRingByEdge {`*edge1 edge2 edge3 ...*`}`	

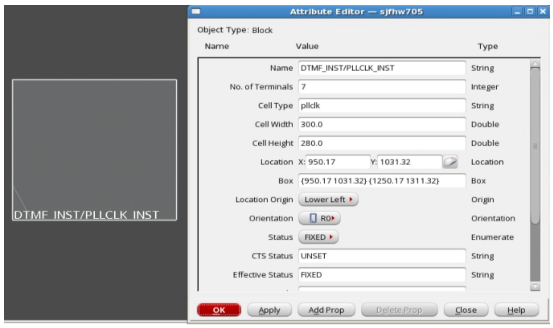
	Creates a hollow ring of placement blockage around the outside of the object(s) specified. *edgeN* is the width in microns of the placement blockage along the Nth edge. *edge1* is the vertical edge starting at the left corner of the leftmost edge with the lowest y coordinate as placed, and proceeding clockwise through the other edges. **Note**: • The `-inst`, `-hinst` , `-allMacros`, or `-allPartition` parameters must be specified with this parameter. • There must be an *edgeN* specified for every edge of the object(s). • By default, a toroid of blockage is created around the specified object(s). Use the `-cover` parameter to create a solid blockage over the objects with the specified additional halo of blockage. • Enclose the *edgeN*s in double-quotes, curly braces, or [list ...]. *Data_type*: string, optional
`-outerRingBySide {x1 y1 x2 y2}`	
	Creates a hollow ring of placement blockage around the outside of the object(s) specified. `bottom` is the width in microns of the placement blockage along the bottom edge(s) of object(s) specified, etc. **Note**: • The `-inst`, `-hinst`, or `-allPartition` parameters must be specified with this parameter. • The `-outerRingBySide` parameter can be used with rectilinear objects. • By default, a toroid of blockage is created around the specified object(s). Use the -cover parameter to create a solid blockage over the objects with the specified additional halo of blockage. • Enclose the *bottom...left* in double-quotes, curly braces, or [list ...]. *Data_type*: box, optional
`-polygon {{x1 y1} {x2 y2} ...}`	
	Specifies the polygon vertices of the blockage, in microns. *Data_type*: point_list, optional
`-pushdown`	Set pushdown attribute for blockages. A blockage with pushdown attribute is pushed down into the block from the top level of the design. **Note:** A normal routing blockage is just used to check for overlaps with routes. A placement blockage with pushdown attribute in Smart Hier flow is used to restrict boundary area for placement and assemble design does not bring it back. *Data_type*: bool, optional
`-snapToSite`	Snaps the newly created placement blockage to a row site. *Data_type*: bool, optional
`-type {hard | soft | partial | macroOnly}`	

	Specifies the type of blockage to be created. • `hard`: Neither standard cell instances nor macros may be placed in the blockage. This is the default. • `soft`: Only buffers, inverters, isolation cells, clock gates, tie cells and level shifters are allowed to be placed inside the soft blockages. All other cells are placed outside the soft placement blockage. You can optionally specify cells and instances that are allowed or forbidden to be placed inside soft blockages by using the `specifySelectiveBlkgGate` or `unspecifySelectiveBlkgGate` commands. • `partial`: In conjunction with the `-density` parameter alone or with the `-density` and `-excludeFlops` parameters together, creates a placement blockage that has a maximum density as specified (and sometimes excludes flops and latches too). • `macro_only`: Enables the `proto_design` command to keep macros out of the placement blockage; however, it enables standard cells to be placed inside the box as if no blockage is present. Macro-Only blockage is represented as "0" percent partial blockage in floorplan file, DEF file, and in Innovus DB. *Default*: `hard` **Note**: Alternatively, you can create different types of placement blockages using the *Set Placement Blockage Options* form or after creating the placement blockage, you can double-click on the object in the design display area to open the *Attribute Editor* and select from four blockage types. *Data_type*: enum, optional

Example

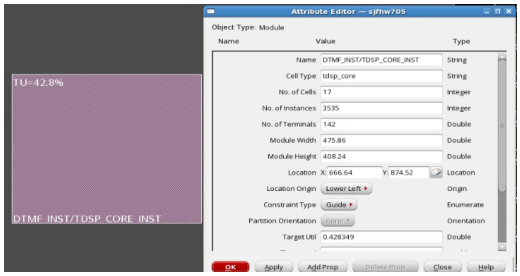
- The following command creates a soft blockage, softBlockage2 in the design area:
  ```
  createPlaceBlockage -type soft -box { { 3442.3600 3739.2000 3511.6500 5716.6300 } } -name softBlockage2
  ```

- The following command creates a hard blockage covers the block instance of DTMF_INST/PLLCLK_INST:
  ```
  createPlaceBlockage -type hard -inst DTMF_INST/PLLCLK_INST -cover
  ```
  Before:

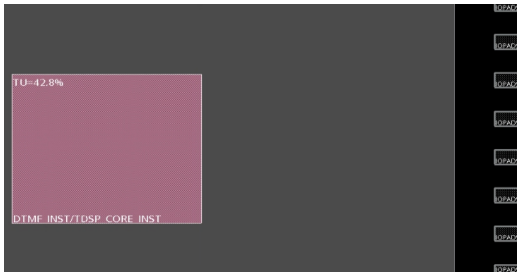
  

  After:

- The following command creates a hard blockage for hierarchical instance (Module) of DTMF_INST/TDSP_CORE_INST:
  ```
  createPlaceBlockage -type hard -hinst DTMF_INST/TDSP_CORE_INST -cover
  ```
  Before:

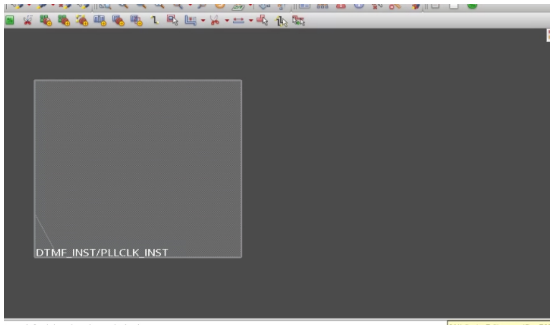
  After:

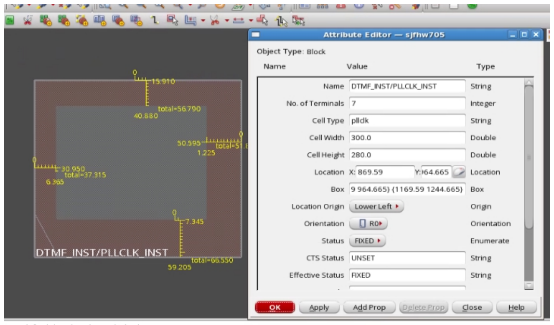
- The following command creates a hollow ring of placement blockage inside of the object:

```
createPlaceBlockage -type hard -inst DTMF_INST/PLLCLK_INST -innerRingByEdge {30 40 50 60}
```

Before:

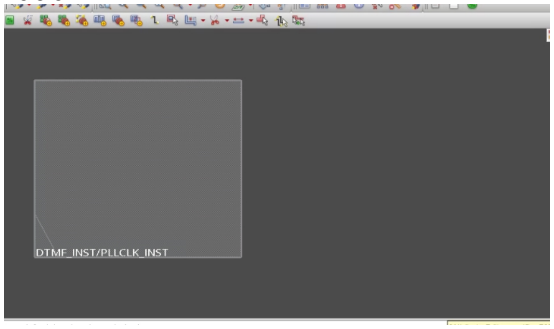


After:

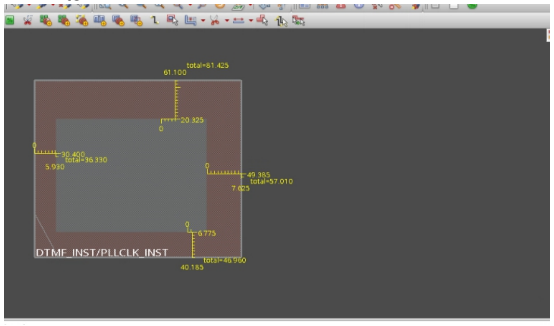



```
createPlaceBlockage -type hard -inst DTMF_INST/PLLCLK_INST -innerRingBySide {30 40 50 60}
```

Before:

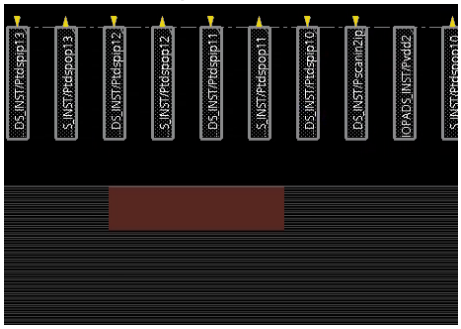


After:

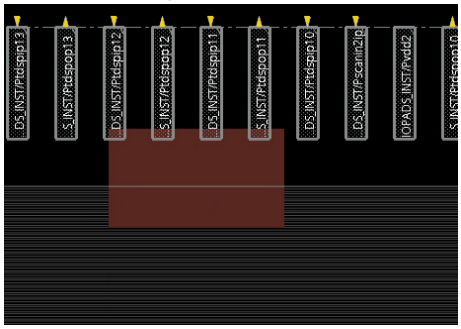


- The following command shows the functionality of the option `noCutByCore`:

```
createPlaceBlockage -box {952 1690 1291 1899}
```



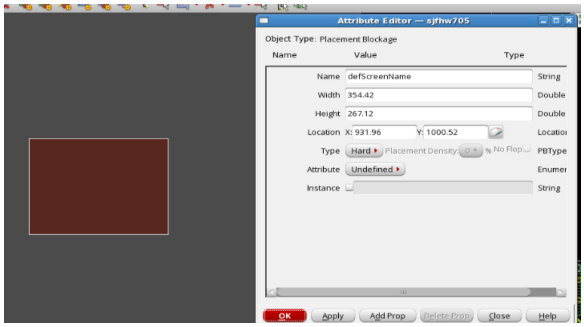
```
createPlaceBlockage -box {952 1690 1291 1899} -noCutByCore
```



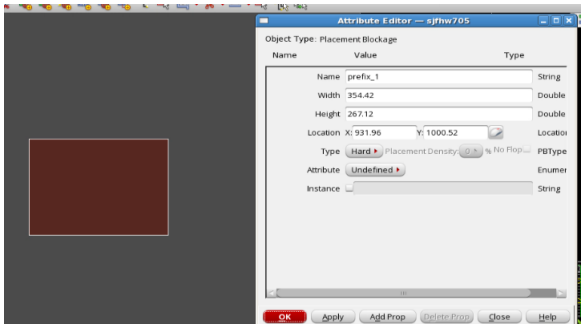
- The following command shows the functionality of the option prefixOn:

setFPlanMode -defaultBlockageNamePrefix prefix

createPlaceBlockage -box {932 1004 1286 1265}



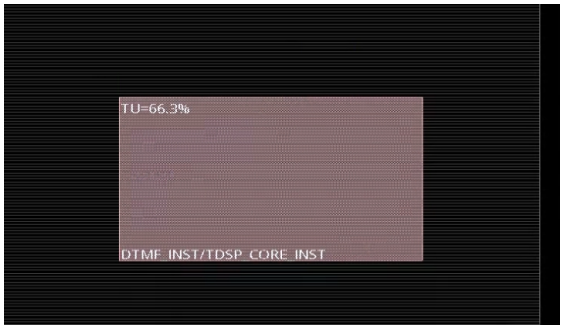
createPlaceBlockage -box {932 1004 1286 1265} -prefixOn

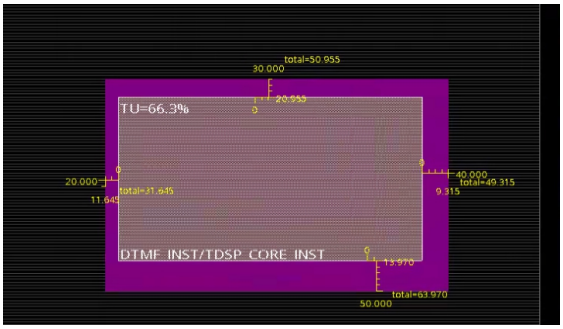


- The following command creates a hollow ring of placement blockage outside of the object:

```
createPlaceBlockage -hinst DTMF_INST/TDSP_CORE_INST -innerRingByEdge {20 30 40 50}
```

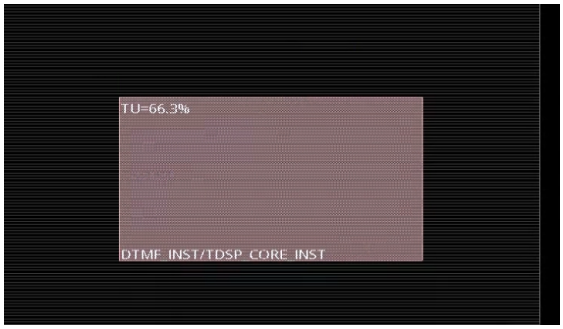
Before: After:

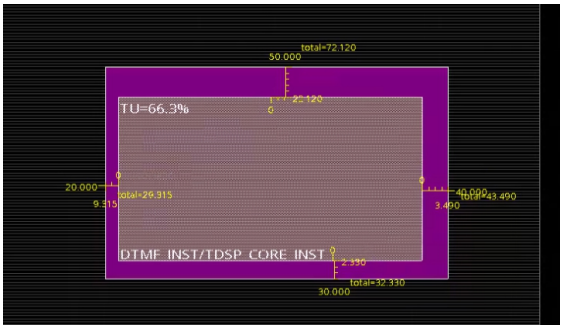





```
createPlaceBlockage -hinst DTMF_INST/TDSP_CORE_INST -innerRingBySide {20 30 40 50}
```

Before: After:





Related Information

- `setFPlanMode`
- `deletePlaceBlockage`
- "Guiding Placement With Blockages" in the *Innovus User Guide*.
- *Create Placement Blockage* form in the *Innovus Menu Reference*.

createRegion

```
createRegion
[-help]
obj_name
box
[-floating]
```

Creates a region for a module or a group. Use this command after importing the design. To delete regions, use unplaceGuide.

Parameters

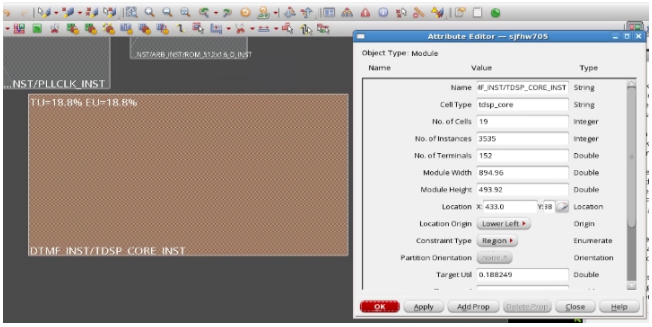
box	Specifies the constraint box in `llx lly urx ury`: • `llx`: Specifies the lower left x coordinate of the region area. • `lly`: Specifies the lower left y coordinate of the region area. • `urx`: Specifies the upper right x coordinate of the region area. • `ury`: Specifies the upper right y coordinate of the region area. *Data_type*: box, required
`-floating`	Creates a floating region. The floating region/fence constraint allows the entire region/fence to move as a cluster across the core area. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `createRegion` parameter. For a detailed description of the command and all of its parameters, use the man command: `man createRegion`
`obj_name`	Specifies the name of the module or the group. *Data_type*: string required

Example

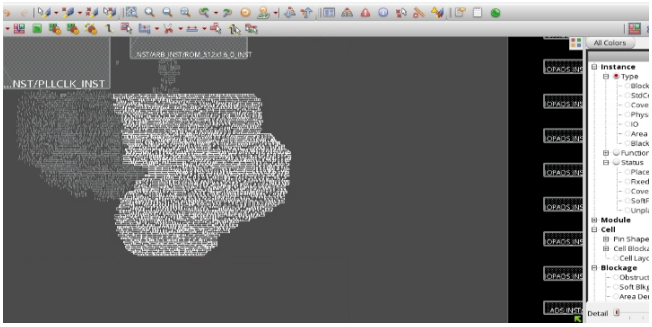
The following command creates a module guide in the floorplan view with status Region:

```
createRegion DTMF_INST/TDSP_CORE_INST 433.0 793.88 1327.96 1287.8
```

before placement (Floorplan view):



After placement (Physical view):



Related Information

- `unplaceGuide`
- Floorplanning the Design chapter in the *Innovus User Guide*
- Floorplan Menu chapter in the *Innovus Menu Reference*

createRouteBlk

```
createRouteBlk
[-help]
[-cutLayer layerName | {layerNamelist ...} | all]
[-drcRegionLayer layerName| {layerNamelist ...} | all]
[-fills]
[-inst name]
[-layer layerName | {layerNamelist ...} | all]
[-mask integer]
[-noWrongWay]
[-pushdown]
[-trimMetalLayer layerName | {layerNamelist ...} | all]
[-name blk| -prefixOn]
{-box {x1 y1 x2 y2} | -cover | -polygon {{x1 y1} {x2 y2}...} | -boxList {{x1 y1} {x2 y2}...}}
[-exceptpgnet | -pgnetonly  | -partial integer]
[-spacing float | -designRuleWidth float]
```

Creates a routing blockage object. The object area prevents routing of specified metal layers, signal routes, and hierarchical instances in this area. Use this command during partition floorplanning.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `createRouteBlk` parameter. For a detailed description of the command and all of its parameters, use the man command: `man createRouteBlk`

`-box {x1 y1 x2 y2}`	Specifies the coordinates of the blockage area.
	• *x1*: Specifies the lower left x coordinate of the blockage area.
	• *y1*: Specifies the lower left y coordinate of the blockage area.
	• *x2*: Specifies the upper right x coordinate of the blockage area.
	• *y2*: Specifies the upper right y coordinate of the blockage area.
	Data_type: box, required
	For example, the following command creates a routing blockage object on metal layers 1, 4, and 5:
	`createRouteBlk -box {0 0 100 100} -layer {metal1 metal4 metal5}`
	Note: You can also add variables to specify the blockage area. For example,
	`set box "0 0 100 100"`
	`createRouteBlk -box $box`

`-boxList {{x1 y1} {x2 y2}...}`	
	Specifies the coordinates of a list of blockage areas. *Data_type*: point_list, required
`-cover`	Specifies that a routing blockage of the same size as the specified block instance (`-inst name`) will be created on top of the instance. **Note:** The `-inst` parameter must be specified with this parameter. *Data_type*: bool, required
`-cutLayer layerName | {layerNamelist ...} | all`	

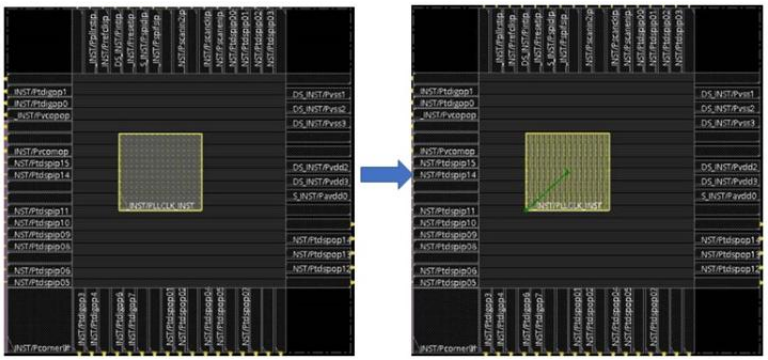
	Specifies the cut layer, list of cut layers, or all cut layers on which the routing blockage is to be applied. To specify cut layer between the first layer (e.g. metal1) and second layer (e.g. metal2), use the second layer name (e.g. metal2). For example, `-cutLayer metal2`. Similarly, if you want to create blockage on cut layer between the second layer (e.g. metal2) and third layer (e.g. metal3), use the third layer name (e.g. metal3). For example, `-cutlayer metal3`, and so on. **Note:** The values specified with this parameter will override any default blockage layers specified with the `setRouteBlkDefaultLayer` command. *Default*: If you do not specify this parameter, the Innovus software will check if any default blockage layers have been specified with the `setRouteBlkDefaultLayer` command. If yes, the layers specified with the `setRouteBlkDefaultLayer` command will be used. If no, the default is metal3 if the number of layers is greater than 3, or metal2 if the number of layers is less than or equal to 3. *Data_type*: list, optional
`-designRuleWidth` *`float`*	Specifies the effective width of the blockage for the purposes of spacing calculations. If you specify `-designRuleWidth`, you cannot specify `-spacing`. **Type:** The *`effectiveWidth`* is a real number greater than 0, in microns. *Data_type*: float, optional
`-drcRegionLayer` *`layerName`*`| {`*`layerNamelist ...`*`} | all`	
	Specifies the drc region layers, list of drc region layers, or all drc region layers on which the routing blockage is to be applied. *Data_type*: list, optional
`-exceptpgnet`	Specifies that the routing blockage is to be applied on a signal net routing and not on power or ground net routing. Use this option to block signal routing above or around a sensitive block to avoid noise from nearby signal nets but still allow power connections to go through the blockage. **Note:** Blocking the signal net routing helps in avoiding crosstalk or coupling caused by signal routes. *Data_type*: bool, optional
`-fills`	Specifies that the routing blockage is to be applied on metal fills. *Data_type*: bool, optional
`-inst` *`name`*	Specifies the name of the instance on which the routing blockage is to be applied. **Note:** When the `-cover` parameter is specified, the `-inst` parameter must also be specified. However, when the `-inst` parameter is specified then specifying the `-cover` option is optional. *Data_type*: string, optional
`-layer` *`layerName`* `|` {*`layerNamelist ...`*`} | all`	
	Specifies the layer, list of layers, or all layers on which the routing blockage is to be applied. **Note:** The values specified with this parameter will override any default blockage layers specified with the `setRouteBlkDefaultLayer` command. *Default*: If you do not specify this parameter, the Innovus will check if any default blockage layers have been specified with the `setRouteBlkDefaultLayer` command. If yes, the layers specified with the `setRouteBlkDefaultLayer` command will be used. If no, the default is wire3 if the number of layers is greater than 3, or wire2 if the number of layers is less than or equal to 3. **Note**: You must specify the *layerName* as a number or as a layer name defined in LEF. For example, *layerName* can be specified as 2 or metal2. *Data_type*: list, optional

`-mask integer`	Blocks routing layers with specified masks. Layers with other masks can still be routed through the blockage. **Note:** • The default value of the `-mask` parameter is 0. This implies that all the specified layers will be blocked. • If the `-mask` parameter is specified with an integer value greater than 0, then both layers with specified mask and uncolored layers are blocked. Example. If `-mask 1` is specified, it blocks mask 1 and uncolored routes. • If more than one layers are associated with a mask, the mask is applied to all of them. • If the `-mask` parameter is specified, the tool checks if the mask is suitable for all of the specified layers. If not, it displays an ERROR message and no blockages are created. The number of arguments specified with the –`mask` parameter must be same as the number of arguments specified with the `-layer`, `-cutLayer` and `-trimMetalLayer` parameters. • The `setNanoRouteMode` and `verify_drc` commands honor the `-mask` parameter. *Default*: 0 *Data_type*: int, optional
`-name blk`	Specifies the layer name of the route blockage. *Data_type*: string, optional
`-noWrongWay`	When this parameter is specified, wrong way routing is avoided in the region. Both router and verify_drc will report DRC if any jogs are present. *Data_type*: bool, optional
`-partial integer`	Specifies the density for partial routing blockages. It causes the global router to only use up to the specified percent of the routing resource on the layer in the blockage area, so the global router will see higher congestion and put fewer routes on that layer in the area. For example, `-partial 75` means only 75% of routing resource in a gcell is available for global routing, and this will affect the final demand/supply values in the congestion map after routing. It is an integer in the range 0 to 100. This can be useful if the router consistently creates too much congestion in a specific area. If you add a partial routing blockage in that area you can force the router to end up with less “real congestion” in that area. When there are overlapping partial routing blockages, the lowest density percentage of the blockages will be used in the overlapping region. For example, when a partial routing blockage with 75 percent density overlapping with another partial routing blockage with 50 percent density, the density of the overlapping region will be 50 percent. **Note:** You can attach a partial routing blockage to a block instance with `-inst` if the congestion is related to the boundary and pins of a specific block. **Note:** The `-designRuleWidth`, `-exceptpgnet`, `-pgnetonly`, and `-spacing` parameters are not allowed for partial routing blockages. *Minimum*: 0 *Maximum*: 100 *Data_type*: int, optional

`-pgnetonly`	Specifies that the routing blockage is to be applied only on power or ground net special routes and not on signal nets. This option affects only commands that create special routes,such as `addRing`, `addStripe`, or `sroute`, when they are used for PG nets. NanoRoute, which is used to connect tie-high or tie-low connections or to connect to secondary standard-cell power-pin connections, is also not affected by this option. Use this option during power planning to prevent power routes from getting too close to block edges and, as a result, blocking signal pin access or causing congestion in narrow channels. **Note:** This option cannot be used with `-exceptpgnet`. *Data_type*: bool, optional
`-prefixOn`	Specifies that the routing blockages should be named with a prefix. *Data_type*: bool, optional
`-polygon {{x1 y1} {x2 y2}...`	Specifies the polygon vertices of the blockage, in microns. **Note:** Although `createRouteBlk` can be be used to create triangular blockages, metal fill does not currently support triangular route blockages. *Data_type*: point_list, required
`-pushdown`	Set pushdown attribute for blockages. A blockage with pushdown attribute is pushed down into the block from the top level of the design. **Note:** A normal routing blockage is just used to check for overlaps with routes. A route blockage with pushdown attribute is treated as a real wire shape from above with full DRC checks. *Data_type*: bool, optional
`-spacing float`	Specifies the minimum spacing allowed between the blockage and any other routing shape. If you specify `-spacing`, you cannot specify `-deignRuleWidth`. **Type:** The float is a real number greater than or equal to 0, in microns. **Note:** When the blockage is of FILL type, the spacing property does not work. In that case, metal fill can still be abutted to the fill blockage. *Data_type*: float, optional
`-trimMetalLayer layerName | {layerNamelist ...} | all`	
	Specifies the trim metal layer, list of trim metal layers, or all trim metal layers on which the routing blockage is to be applied. For example, the following command command creates routing blockages in trim metal layers Metal1 and Metal2. `createRouteBlk -trimMetalLayer {metal1 metal2 …}` *Data_type*: list, optional

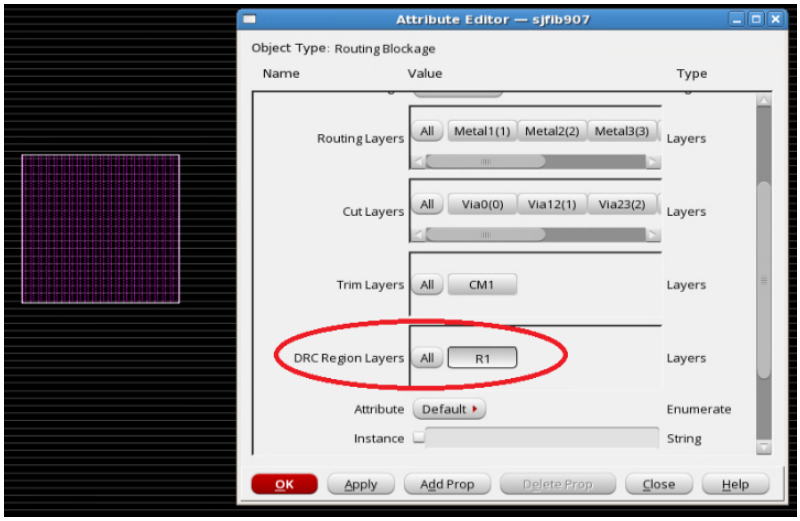
Example

- The following command creates a routing blockage object to block metal3 routing in this area:
  ```
  createRouteBlk -box {2353.8000 7123.000 2653.8000 8298.2000} -layer metal3
  ```
- The following command creates a routing blockage object on metal layers 1, 4, and 5:
  ```
  createRouteBlk -box {0 0 100 100} -layer {metal1 metal4 metal5}
  ```
- The following command creates a routing blockage object on metal layers 1, 4 and blocks the signal net routing:
  ```
  createRouteBlk -box {10 18 1782 2300} -layer {metal1 metal4} -exceptpgnet
  ```
- The following command creates a routing blockage object having polygon vertices (10, 10), (10, 15), (15, 15), (15, 20), (20, 20), (20, 10) on the metal layer 1:
  ```
  createRouteBlk -polygon {10 10 10 15 15 15 15 20 20 20 20 10} -layer metal1
  ```
- The following command creates a routing blockage on top of the the specified block instance that is of the same size the instance.
  ```
  createRouteBlk -inst DTMF_INST/PLLCLK_INST -cover
  ```



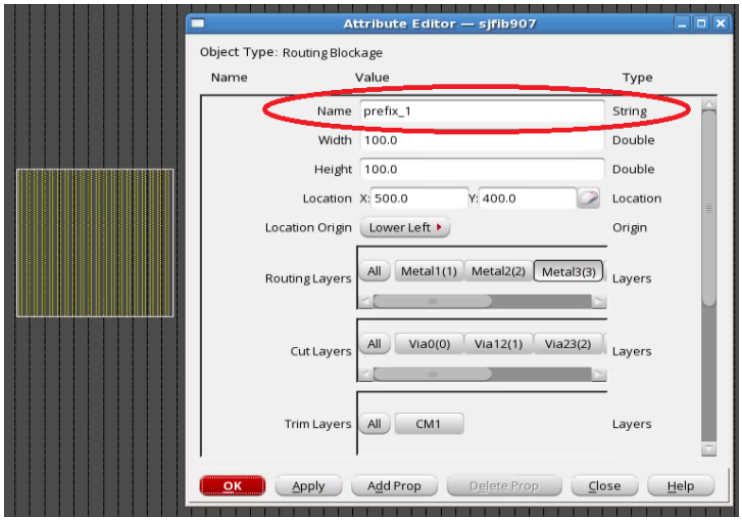
- The following command creates a routing blockage object on drc region layer:

```
createRouteBlk -box {500 400 600 500} -drcRegionLayer R1
```



- The following command creates a routing blockage object with a prefix name:
  set prefix name：`setFPlanMode -defaultBlockageNamePrefix prefix`

```
createRouteBlk -box {500 400 600 500} -prefixOn
```



- The following command creates a routing blockage on metal fills only:

```
createRouteBlk -box {500 400 600 500} -fills
```

- The following command creates a routing blockage on power or ground net special routes only and not on signal nets.

```
createRouteBlk -box {0 0 100 100} -pgnetonly
```

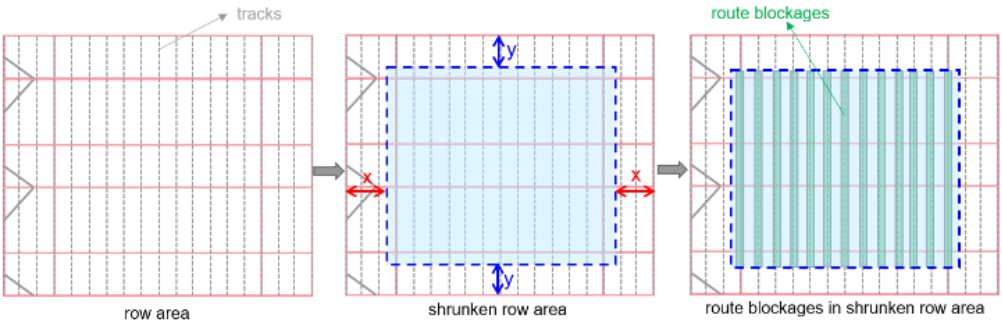
Related Information

- deleteRouteBlk
- setRouteBlkDefaultLayer
- Floorplanning the Design chapter in the *Innovus User Guide*

createRouteBlkAlongTrack

```
createRouteBlkAlongTrack
[-help]
-align_track string
[-consider_cells_as_macros string]
[-consider_macros_as_cells string]
[-edge_shrink {x y}]
-layer string
[-no_cut_row]
```

Creates routing blockages that are aligned to specified tracks which are in the shrunken row area. Use this command during partition floorplanning.

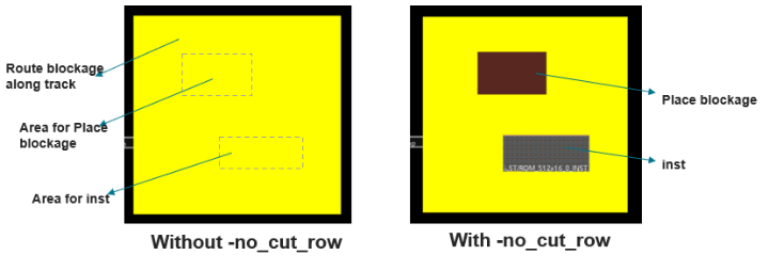


Note:

- Routing blockages are also created with align to the track which is near to the shrunken row area with less than half of layer width.
- Routing blockages honor rectilinear row area.

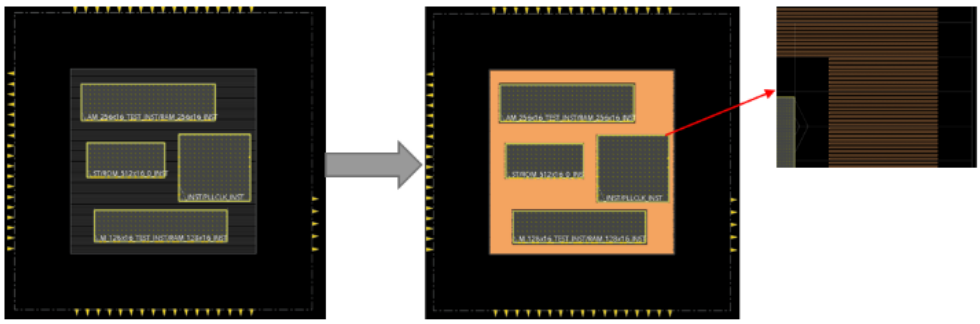
Parameters

`-align_track string`	Specifies the track layer to which the route blockages align to. *Data_type*: string, required
`-help`	Prints a brief description that includes the type and default information for each `createRouteBlkAlongTrack` parameter. For a detailed description of the command and all of its parameters, use the man command: `man createRouteBlkAlongTrack`
`-consider_cells_as_macros string`	
	Specifies the names of the cells that are considered as macros. Blockages are not created over them. *Data_type*: string, optional
`-consider_macros_as_cells string`	
	Specifies the names of the cells that are considered as std cells. Blockages are created over them. *Data_type*: string, optional
`-edge_shrink {x y}`	Shrinks row area from the edge in X and Y direction. *Data_type*: point, optional
`-layer string`	Specifies the name of the layer on which the tool creates routing blockages for steam out. *Data_type*: string, required

`-no_cut_row`	Specifies whether the blockages to be added should be based on the no-cut row site area even if the row site area is not cut in the design. • Without the `-no_cut_row` parameter, the tool will create route blockages based on the current row area. So the rows for objects like insts, place blockages are expected to be cut out before running the `createRouteBlkAlongTrack` command to get expected results. • With the `-no_cut_row` parameter, the tool will identify the rows have not been cut before and will call the `cutRow` command to get a cut row shape during the action. The route blockage is created based on the cut row shape. The actual row shape will not be changed. • The `setFPlanMode -no_cut_row true` command has the same use with this option. Example: In the following illustration, the rows are not cut before. Now, without the `-no_cut_row` parameter, the tool will create a route blockage over the place blockage and the inst. With the `-no_cut_row` parameter, the tool will not create route blockage over them.  *Default:* false *Data_type*: bool, optional

Example

- The following command creates a routing blockage on layer M2. The route blockages align to track layer M2:
  ```
  createRouteBlkAlongTrack -layer M2 -align_track M3 -edge_shrink {5 5}
  ```



Related Information

- `createRouteBlk`
- `deleteRouteBlk`
- Floorplanning the Design chapter in the *Innovus User Guide*

createRow

```
createRow
[-help]
-site siteName
[-area {x1 y1 x2 y2} | -boxList {{x1 y1} {x2 y2} ...} | -polygon {{x1 y1} {x2 y2}...}]
[-spacing distance]
[-limitInCore]
[-noAbut | -noAbut1st]
[-flip1st]
[-noCheck]
[-noFlip]
```

Creates rows for the specified site. The row boundary can be defined by core area or the area that you specify. This command supports the creation of overlapping rows. This command can create only horizontal rows. By default, alternate rows are flipped and all rows are abutted (no spaces between rows).

The following points apply to the usage of this command:

- Rows can be created both inside and outside the core area, but within the die.
- All new rows created in an area must be an integer multiple of any existing rows in the same area.
- Non-integer multiple height rows cannot be created outside power domains.

Use this command after importing a design.

Parameters

`-area {x1 y1 x2 y2}`	
	Create rows in the specified rectangle. (Default is within the core box.) The values are floats, units are microns. Rows are truncated at the die box boundary. Either a tcl list or 4 separate parameters works. • `x1`: Specifies the lower left x coordinate of the area. • `y1`: Specifies the lower left y coordinate of the area. • `x2`: Specifies the upper right x coordinate of the area. • `y2`: Specifies the upper right y coordinate of the area. *Data_type*: box, optional
`-boxList {{x1 y1} {x2 y2} ...}`	
	The X and Y values are floats, units are microns. There must be an even number of points specified. Pairs of points are used to determine a set of rectangles, where the first point of the pair is LLX, LLY, the second point is URX, URY. Rectangles need not be overlapping. Rows are created in the union of the rectangles defined. Note that for overlapping rectangles, rows are created continuously starting at the first (bottom) row (of the lowest rectangle). For non-overlapping rectangles, rows are created independently starting at the bottom of each rectangle. *Data_type*: point_list, optional
`-flip1st`	Controls the orientation of the first (bottom) row, which is R0 by default. When `-flip1st`, the first row's orientation is MX. See `-noFlip`. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `createRow` parameter. For a detailed description of the command and all of its parameters, use the man command: `man createRow`
`-limitInCore`	By default (none of `-area, -polygon, -boxList` specified) rows are created in the core box. When one of `-area, -polygon, -boxList` is specified, by default rows are created within the intersection of the die box and the specified rectilinear shape. When one of `-area, -polygon, -boxList` and `-limitInCore` is specified, rows are created within the intersection of the core box and the specified rectilinear shape. *Data_type*: bool, optional
`-noAbut`	Controls where spaces are inserted between rows. By default, spaces are inserted between pairs of rows. When `-noAbut` is enabled spaces are inserted between every row. See `-spacing` and `-noAbut1st`. *Default*: The rows are abutted by default. *Data_type*: bool, optional

`-noAbut1st`	Controls where the first space between rows is inserted. Default is that the first space is inserted after the first (lowest) *pair* of rows, and subsequently after every pair of rows (space inserted between 2nd and 3rd rows, between 4th and 5th rows, etc). When `-noAbut1st`, the first space is inserted after the first (lowest) row, and subsequently after every pair of rows (space inserted between 1st and 2nd rows, then 3rd and 4th rows, etc). See `-spacing` and `-noAbut`. *Data_type*: bool, optional
`-noCheck`	Specifies that the row should not be aligned to the existing row and should support redo undo. *Data_type*: bool, optional
`-noFlip`	By default rows have alternating orientation `(...MX R0 MX R0...)`, or when `-no_flip_rows` is specified rows have identical orientation (one of: `...R0 R0 R0...`; or `...MX MX MX...`). See `-flip1st`. *Data_type*: bool, optional
`-polygon {{x1 y1} {x2 y2}...}`	The X and Y values are floats, units are microns. There must be an even number of points specified. The points specify a polygon in which rows are created. Each pair of points specifies one edge of the polygon, so { X1 Y1 } and { X2 Y2 } specify one edge, { X2 Y2 } { X3 Y3 } specify the next edge, etc. The last edge is specified by { Xlast Ylast } { X1 Y1 }. In the degenerate case (multiple rectilinear areas touching at corners) rows will be created continuously starting at the lowest point. *Data_type*: point_list, optional
`-site siteName`	Specifies the name of the site to be used for creating the rows. (Objects not allowed.) *Data_type*: bool, optional
`-spacing`	Specifies the width of the space inserted between certain rows. The first (lowest) row is always inserted along the bottom edge of the area having rows added. If `-noAbut` is specified, then space is inserted between every subsequent row. The default is space is inserted between every other row. If `-noAbut1st` is specified, then space is inserted after the first row (and then the third etc). The default is that space is inserted after the second row (and then the 4th etc). Note that `-noAbut` and -noAbut1st are mutually exclusive. *Default*: 0.0 um *Data_type*: float, optional

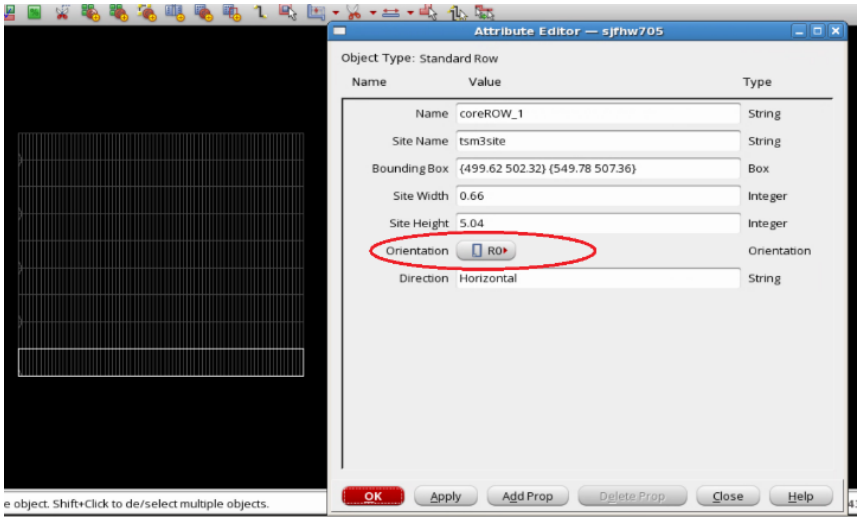
Example

- The following command creates rows for the site lt3site within the area defined by the coordinates 376.7, 577.57, 688.94, 725.68, maintaining a spacing of 1 micron between the rows, without the first row being abutted:

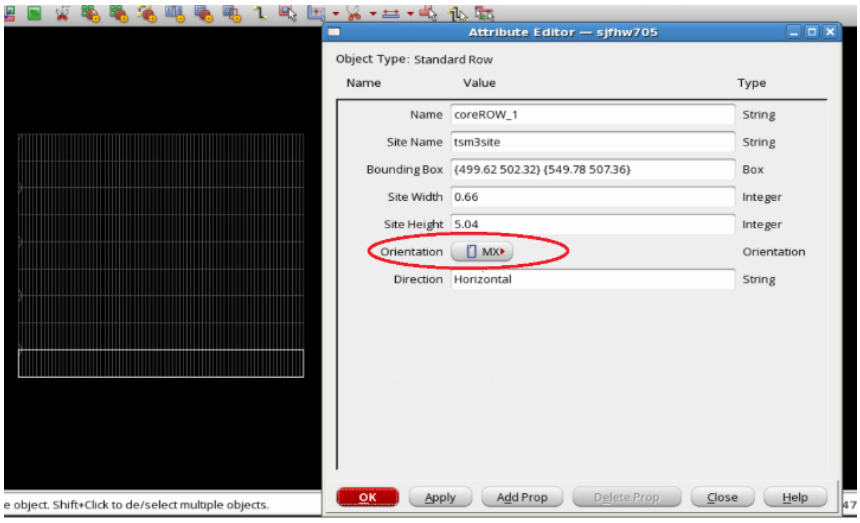
```
createRow -site lt3site -area 376.7 577.57 688.94 725.68 -spacing 1.0 -noAbut1st
```

- The following commands and images show the functionality of the option `flip1st` :

```
createRow -area {500 500 550 550} -site tsm3site :
```

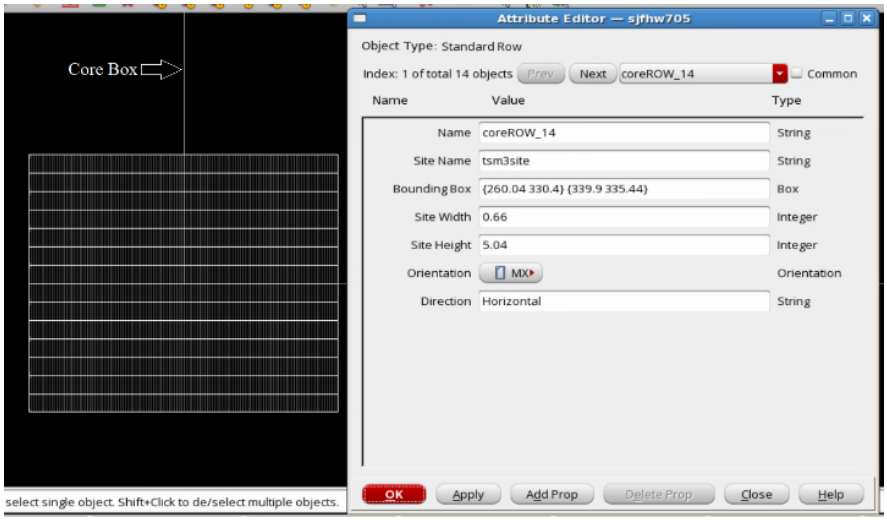



```
createRow -area {500 500 550 550} -site tsm3site -flip1st :
```

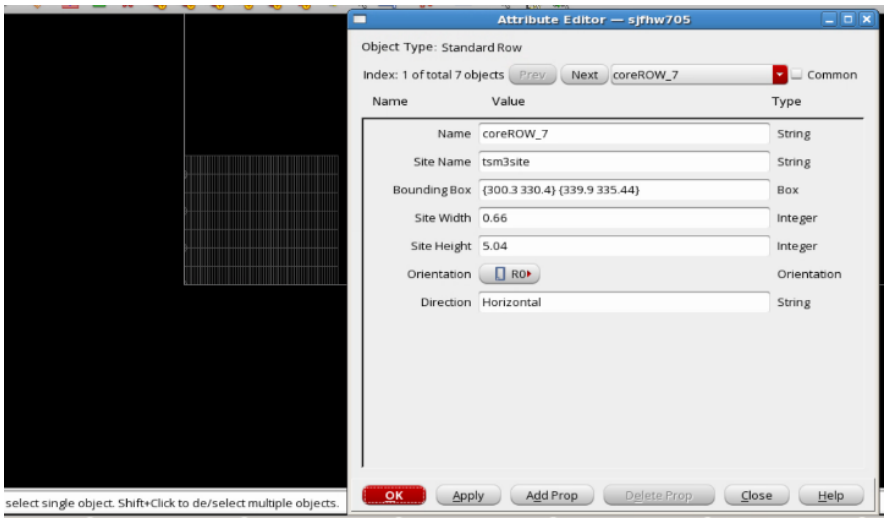


- The following commands and images show the functionality of the option `limitInCore` :

```
createRow -area {260 260 340 340} -site tsm3site:
```

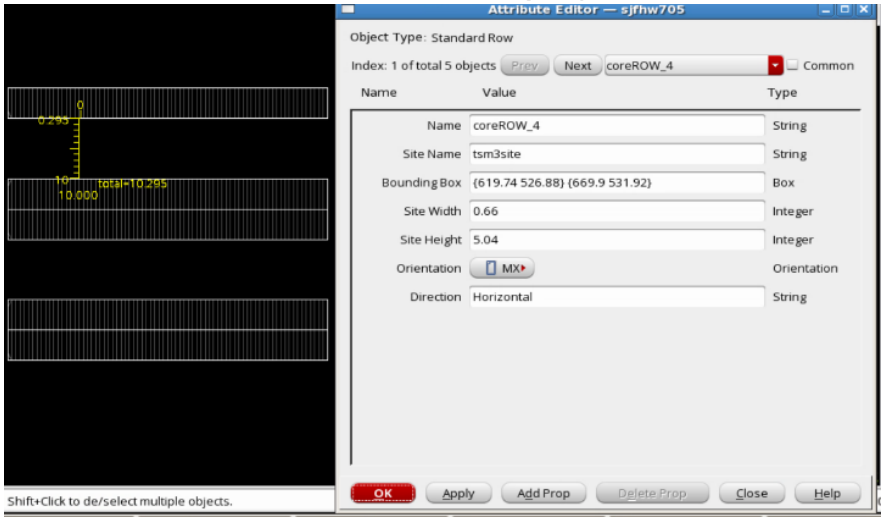



```
createRow -area {260 260 340 340} -site tsm3site -limitInCore:
```

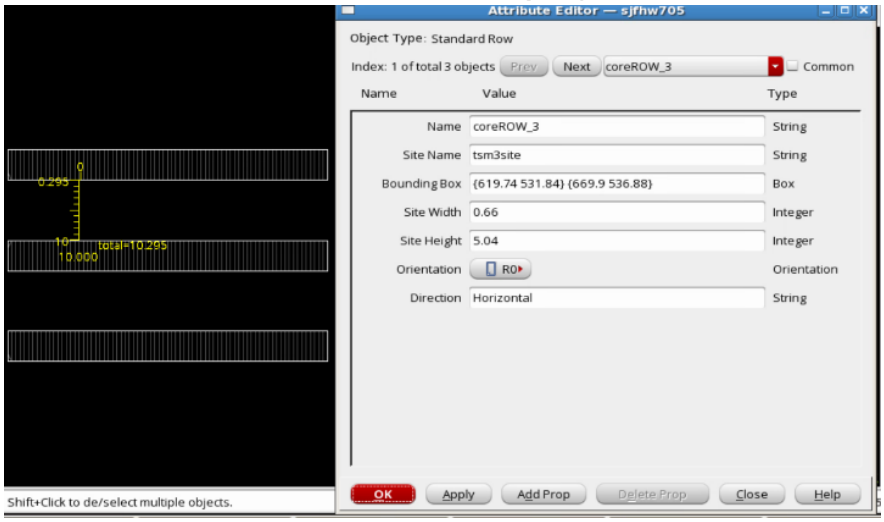


- The following commands and images show the capabilities of the options `spacing`, `noAbut` and `noAbut1st`:

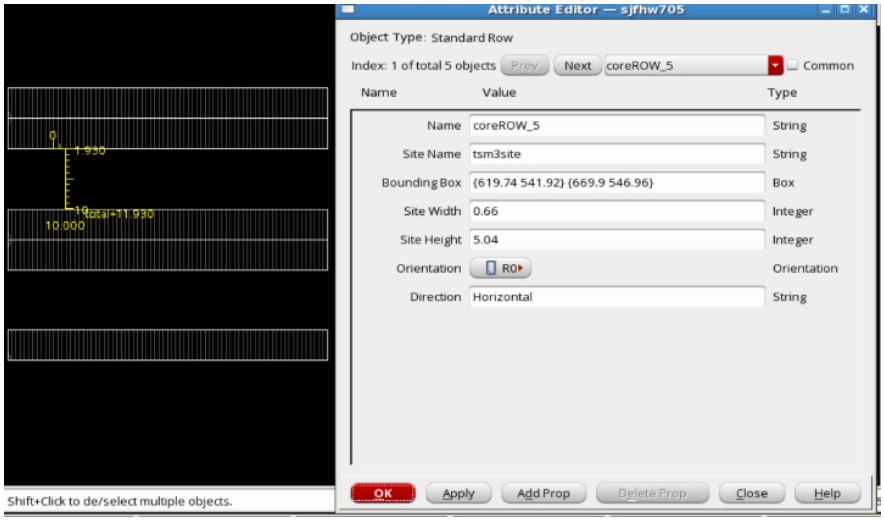
```
createRow -area {620 500 670 550} -site tsm3site -spacing 10:
```




```
createRow -area {620 500 670 550} -site tsm3site -spacing 10 -noAbut:
```

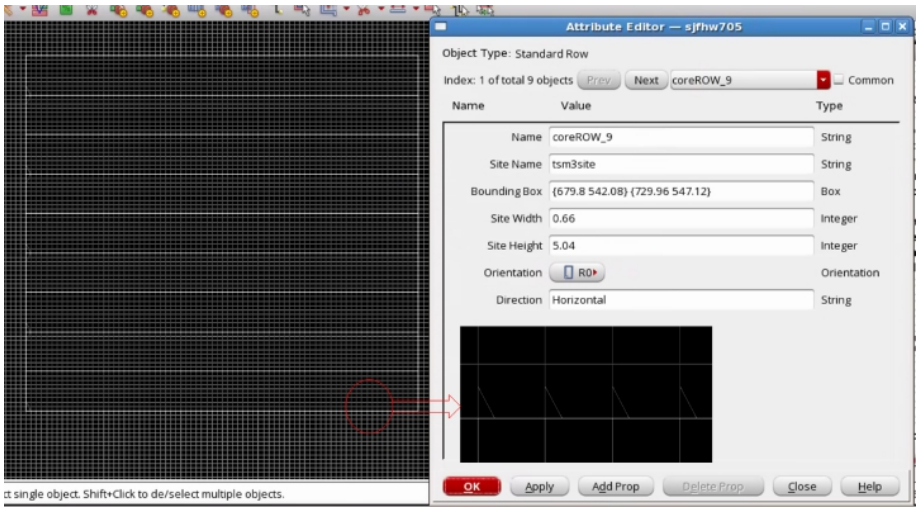


```
createRow -area {620 500 670 550} -site tsm3site -spacing 10 -noAbut1st:
```

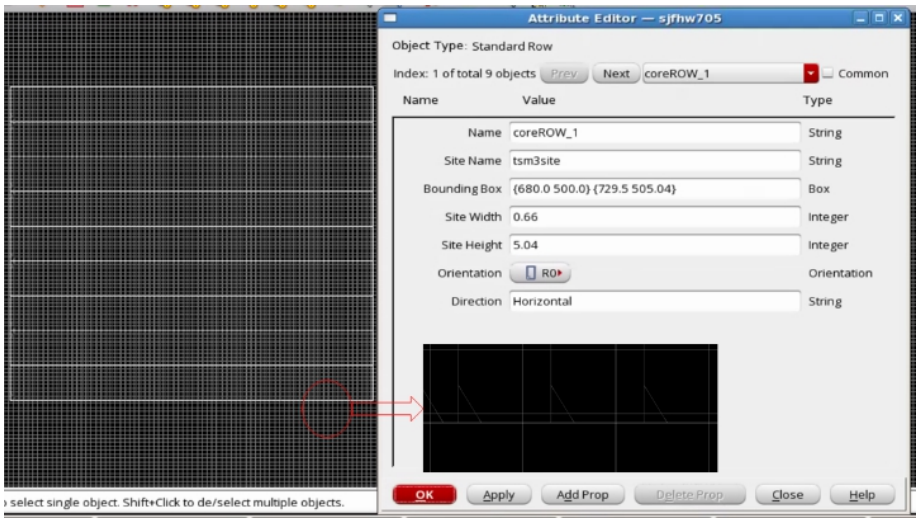


- The following commands and images show the capabilities of the options `noCheck` (The placement grid is opened):

```
createRow -area {680 500 730 550} -site tsm3site :
```

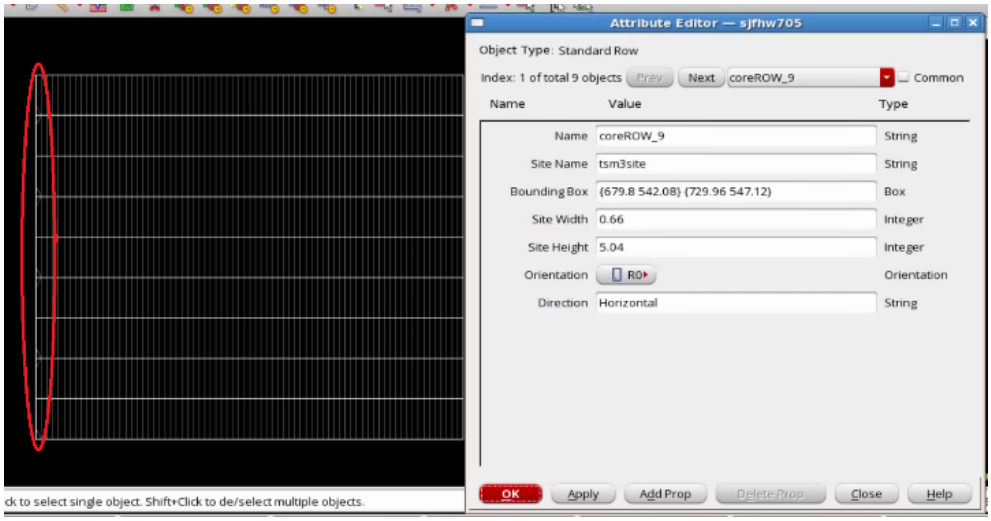



```
createRow -area {680 500 730 550} -site tsm3site -noCheck :
```

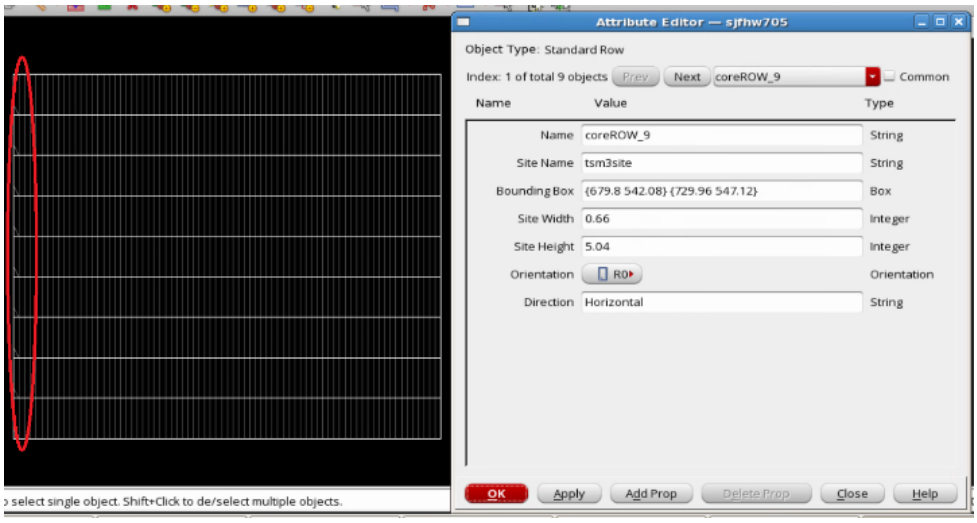


- The following commands and images show the capabilities of the option `noFlip`:

```
createRow -area {680 500 730 550} -site tsm3site :
```




```
createRow -area {680 500 730 550} -site tsm3site -noFlip:
```



Related Information

- `deleteRow`
- `cutRow`
- `select_row`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Using Vertical Rows

createSoftGuide

```
createSoftGuide
[-help]
name
```

Creates a soft guide for a module or a group. Use this command after importing a design.

Parameters

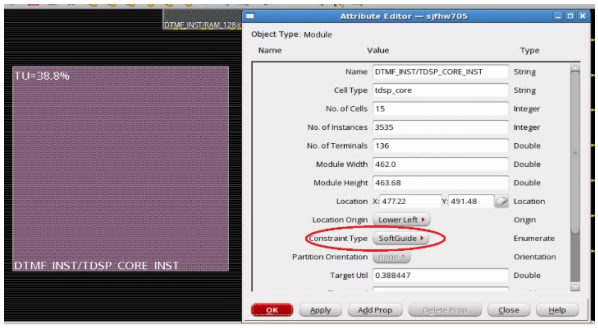
`-help`	Outputs a brief description that includes the type and default information for each `createSoftGuide` parameter. For a detailed description of the command and all of its parameters, use the man command: `man createSoftGuide`
name	Specifies the name of module or group. *Data_type*: string, required

Examples

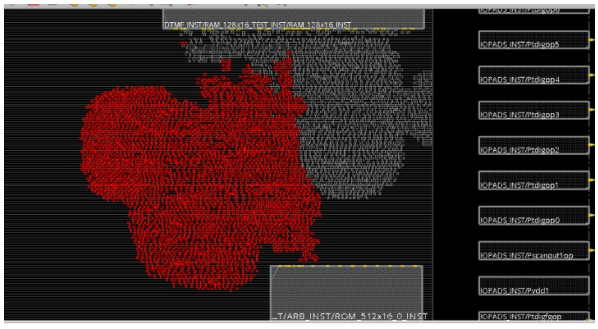
- The following command creates a soft guide for hierarchical instance DTMF_INST/TDSP_CORE_INST:

  `createSoftGuide DTMF_INST/TDSP_CORE_INST`:

  `Before placement`:

  `After placement`:

- The following command creates a soft guide for group adder1:

  `createSoftGuide adder1`

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*
- *Floorplan Menu chapter in the Innovus Menu Reference*

createSpecialDrcRegion

```
createSpecialDrcRegion
[-help]
[-extendOuterX float]
[-extendOuterY float]
[-extendX float]
[-extendY float]
[-insts string]
-layer string
```

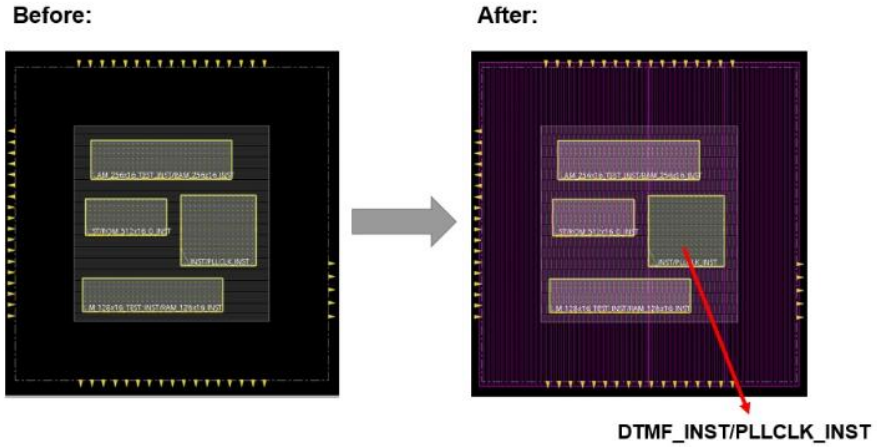
DRC region layers define a special masterslice layer that is used to define the areas of a region on which a set of rules defined in the metal, cut, and/or trim metal layers with the REGION property would be applied. The `createSpecialDrcRegion` command can be used to add drc region blockages in or around the die area except specified macros.

Parameters

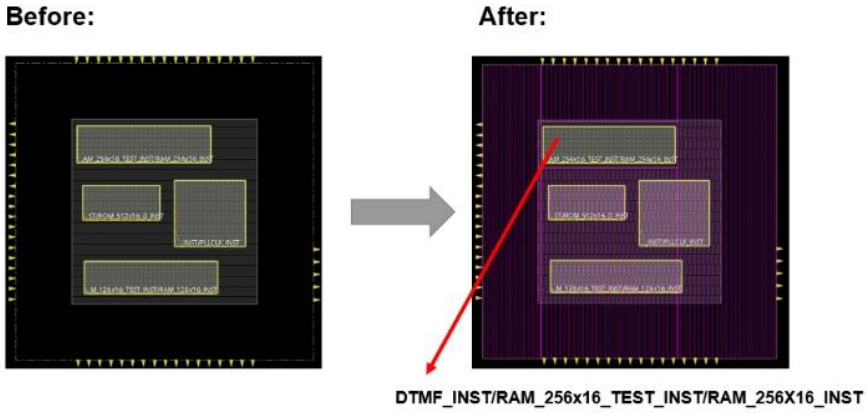
`-extendOuterX` *float*	Expands the routing blockage in X direction. *Data_type*: float, optional
`-extendOuterY` *float*	Expands the routing blockage in Y direction. *Data_type*: float, optional
`-extendX` *float*	Expands the non-routing blockage in X direction. *Data_type*: float, optional
`-extendY` *float*	Expands the non-routing blockage in X direction. *Data_type*: float, optional
`-help`	Outputs a brief description that includes the type and default information for each `createSpecialDrcRegion` parameter. For a detailed description of the command and all of its parameters, use the man command: `man createSpecialDrcRegion`
`-insts` *string*	Specifies the name of the hierarchical instance that are to be cut out. **Note:** Wild card characters are not supported while specifying hierarchical instance names. **Note:** All blocks are included if `-insts` is not specified or when `-insts{}` is specified. *Data_type*: string, optional
`-layer` *string*	Specifies the names of the layers on which the routing blockage is created. *Data_type*: string, optional

Example

- The following command creates a routing blockage on R1 that blocks out `DTMF_INST/PLLCLK_INST`:
  `createSpecialDrcRegion -layer R1 -extendOutterX 10 -extendOutterY 20 -insts DTMF_INST/PLLCLK_INST`



- The following command creates a routing blockage on R1 that blocks out `DTMF_INST/RAM_256x16_TEST_INST/RAM_256x16_INST`:
  `createSpecialDrcRegion -layer R1 -extendX 10 -extendY 20 -insts DTMF_INST/RAM_256x16_TEST_INST/RAM_256x16_INST`



Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

createSpecialMarkLayerShape

```
createSpecialMarkLayerShape
[-help]
[-cutOutBlockInsts string]
[-extendOuterX float]
[-extendOuterY float]
[-extendX float]
[-extendY float]
-layer string
```

Creates a mark layer shape, using special route. This command enables you to apply one set of rules for the entire design in Innovus by connecting the macro pins around halo of IP blocks. The IPs are selected based on a list which provides the instance name.

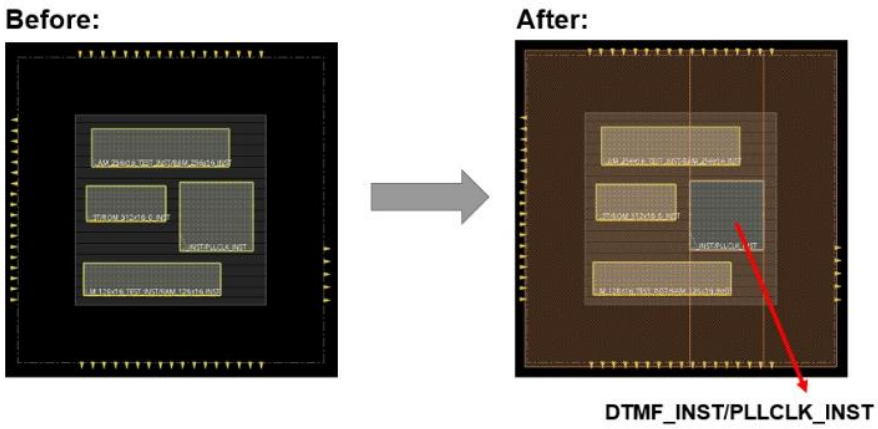
Parameters

`-cutOutBlockInsts` *string*	Specifies the name of the instances that are to be cut out. **Note:** All blocks are cut out if `-cutOutBlockInsts` is not specified however, if `-cutOutBlockInsts {}` is specified then all blocks are NOT cut as this pattern cannot match anything. *Data_type*: string, optional
`-extendOuterX` *float*	Expands the mark layer shape in X direction. *Data_type*: float, optional
`-extendOuterY` *float*	Expands the mark layer shape in Y direction. *Data_type*: float, optional
`-extendX` *float*	Expands the no-mark layer shape in X direction. *Data_type*: float, optional
`-extendY` *float*	Expands the no-mark layer shape in Y direction. *Data_type*: float, optional
`-help`	Outputs a brief description that includes the type and default information for each `createSpecialMarkLayerShape` parameter. For a detailed description of the command and all of its parameters, use the man command: `man createSpecialMarkLayerShape`
`-layer` *string*	Specifies the names of the layers on which the mark layer shape is to be applied. *Data_type*: string, required

Example

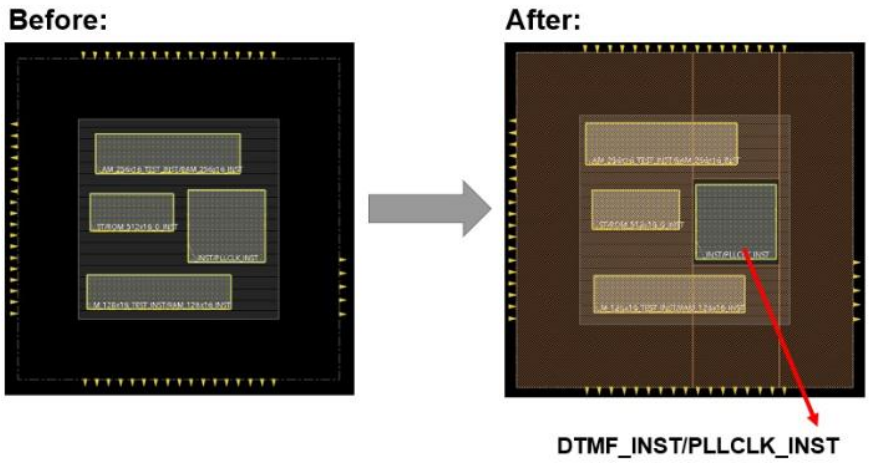
- The following command creates a mark layer shape on M2_P48 that blocks out `DTMF_INST/PLLCLK_INST` and expands the marker layer shape by 10 in X direction and by 20 in Y direction:

```
createSpecialMarkLayerShape -cutOutBlockInsts DTMF_INST/PLLCLK_INST -extendOuterX 10 -extendOuterY 20 -layer M2_P48
```



- The following command creates a mark layer shape on M2_P48 that blocks out `DTMF_INST/PLLCLK_INST` and expands the no-mark layer shape by 10 in X direction and by 20 in Y direction:

```
createSpecialMarkLayerShape -cutOutBlockInsts DTMF_INST/PLLCLK_INST -extendX 10 -extendY 20 -layer M2_P48
```



Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

createStairwayBoundary

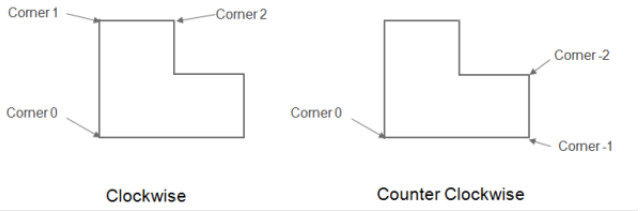
```
createStairwayBoundary
[-help]
-dx value
-dy value
[-growNeighborPtnDir {x y}]
-ptn partition_name
-step number_of_steps
{-corner {LL LR UL UR} | -vertex number_of_vertex}
```

Creates small stairway rectilinear edges at a specified partition corner to improve congestion.

Note: This command does not support master/clones.

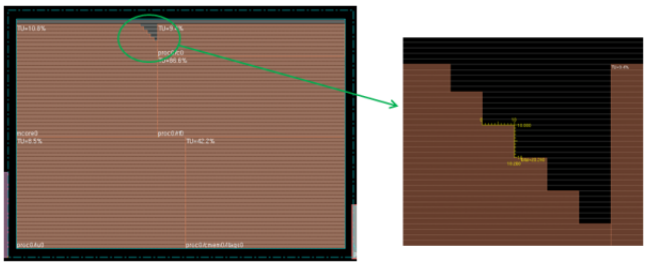
Note: The `-dx`, `-dy`, `-ptn`, `-step`, and `-corner/-vertex` options should be set by default. The partition will snap to the row automatically.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `createStairwayBoundary` parameter. For a detailed description of the command and all of its parameters, use the man command: `man createStairwayBoundary`
`-corner {LL LR UL UR}`	Specifies partition corner where a stairway will be created. • `UL`: Upper left • `LL`: Lower left • `LR`: Lower right • `UR`: Upper right *Data_type*: enum, optional
`-dx value`	Specifies step width. *Data_type*: float, required
`-dy value`	Specifies step height. *Data_type*: float, required
`-growNeighborPtnDir {x y}`	
	Specifies the direction of the target partition in which the neighbor partition will grow to fill the stairway gap between the target partition and the neighbor partition. If this option is not specified, no abutted partition will grow. *Data_type*: enum, optional
`-ptn partition_name`	Specifies the name of the target partition. *Data_type*: string, required
`-step number_of_steps`	Specifies the number of steps to be created in stairway. *Data_type*: int, required
`-vertex number_of_vertex`	Specifies partition vertex where stairway will be created. Vertex starts from the lower-left corner of the partition clockwise. The first vertex is 0. A vertex can be negative, and for a negative vertex, corner numbering starts at 0 from the lower-left corner of a partition counter clockwise.  *Data_type*: int, optional

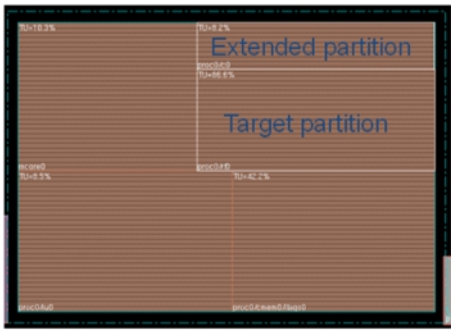
Examples

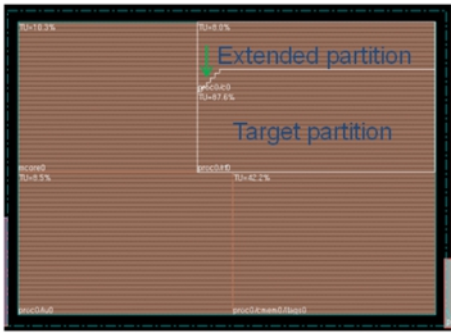
- The following command creates five-stairway rectilinear edges. The width and height of steps are 10.
  ```
  createStairwayBoundary -dx 10 -dy 10 -ptn mcore -step 5 -corner UR
  ```



- The following command sets `-growNeighborPtnDir y`, and the partition next to the target partition in the Y direction extends.

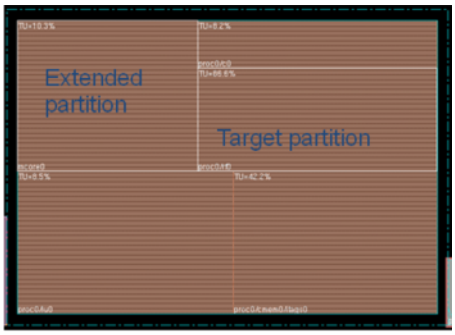
```
createStairwayBoundary -dx 10 -dy 10 -ptn regfile_iu_rftype2_abits8_dbits32_words136  -step 5 -corner UL -growNeighborPtnDir y
```

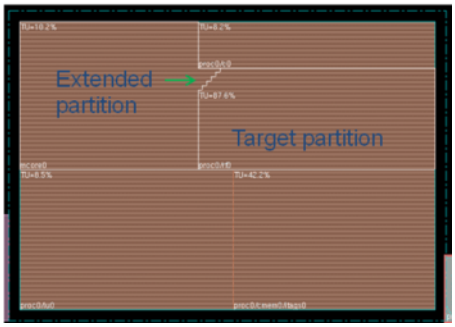




- The following command sets `-growNeighborPtnDir x`, and the partition next to the target partition in the X direction extends.

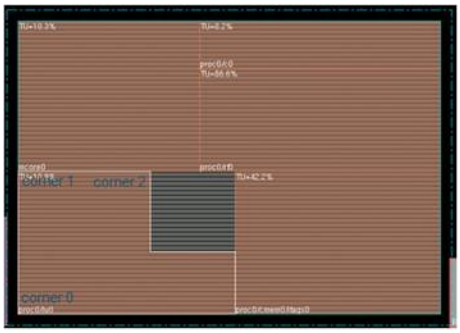
```
createStairwayBoundary -dx 10 -dy 10 -ptn regfile_iu_rftype2_abits8_dbits32_words136  -step 5 -corner UL -growNeighborPtnDir x
```

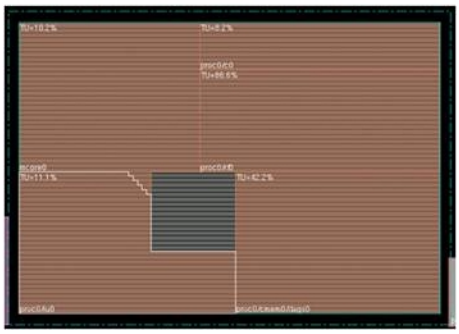




- The following command changes the vertex 2 to a stairway.

```
createStairwayBoundary -dx 10 -dy 10 -ptn iu  -step 5 -vertex 2 -growNeighborPtnDir y
```





Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

cutRectilinearInst

```
cutRectilinearInst
[-help]
-inst instName
{{-refInst instName [-spacingX xvalue] [-spacingY yvalue]} |
{-cutBox {x1 y1 x2 y2}} | {-corner cornerValue [-x value -y value]}}
[-pushOut sideEdgeName]
```

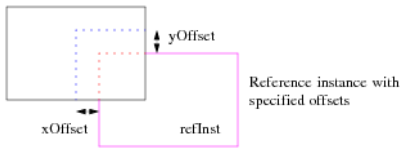
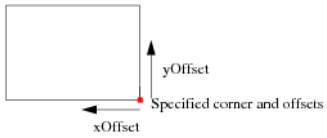
Cuts partition or black box boundaries to rectilinear instances that fit blocks in a given die, without modifying the existing block area. The command performs the rectilinear cut based on the following criteria that you specify:

- The name of a hierarchical instance.
- The name of a referenced instance.
- The name of corner where the cut is to be defined.
- The x and y offset values, in microns, from a specific corner or an overlapping area.
- The name of a side or edge that is to be pushed out, to maintain the existing block area.

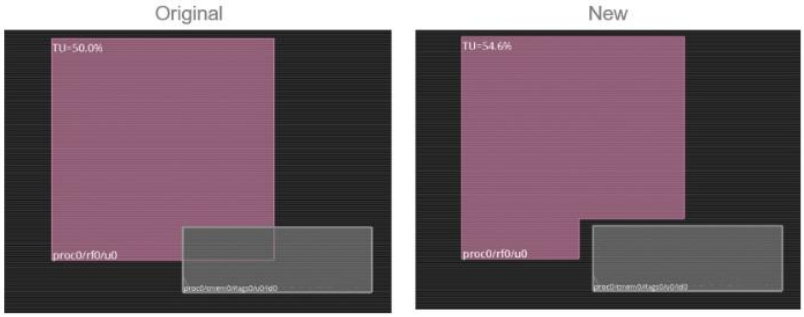
You can use the command at any point in the design flow

Parameters

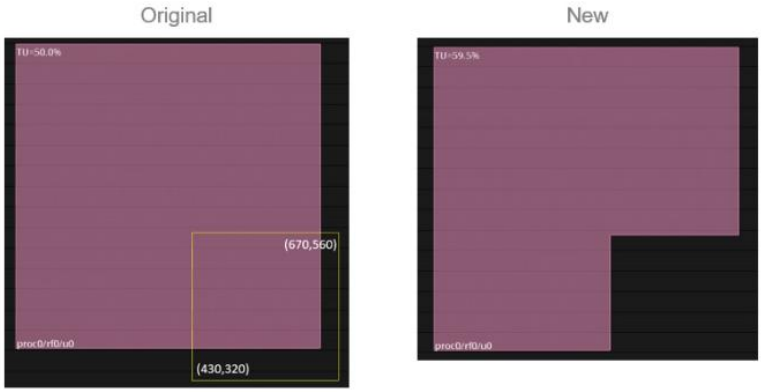
`-corner` *`cornerValue`*	Specifies the name of a corner from where the cut is to be defined. This can be lx, ly, ux, uy coordinates, or edge number in case of a rectilinear corner. Edge numbers are counted clock-wise from the lower left corner starting with 0. *Data_type*: string, optional
`-cutBox`	Specifies the rectilinear box that is to be cut. • *`x1`*: Specifies the lower left x coordinate of the cut area. • *`y1`*: Specifies the lower left y coordinate of the cut area. • *`x2`*: Specifies the upper right x coordinate of the cut area. • *`y2`*: Specifies the upper right y coordinate of the cut area. *Data_type*: box, optional
`-help`	Outputs a brief description that includes the type and default information for each `cutRectilinearInst` parameter. For a detailed description of the command and all of its parameters, use the man command: `man cutRectilinearInst`
`-inst` *`instName`*	Specifies the name of a hierarchical instance. The instance can be a module, an uncommitted partition or a black box. *Data_type*: (inst|hinst|group)+, required
`-pushOut` *`sideEdgeName`*	Specifies the x offset value, in microns, from a specified corner. Corner numbers are counted clock-wise from the lower left corner starting with 0. This can be: • top • bottom • left • right • rectilinear edge number **Note**: If you do not specify a side or edge, the `cutRectilinearInst` command will not maintain the existing area and all the sides or edges are modified by default. *Data_type*: string, optional
`-refInst` *`instName`*	Specifies the name of a reference instance. The instance can be a module, an uncommitted partition, a black box or a hard macro. *Data_type*: (inst|hinst|group)+, required

`-spacingX` *`xValue`*	Specifies the x offset value, in microns, of the overlapping area between the specified block and the reference object. *Default*: 0  *Data_type*: float, optional
`-spacingY` *`yValue`*	Specifies the y offset value, in microns, of the overlapping area between the specified block and the reference object. *Default*: 0 *Data_type*: float, optional
`-x` *`xValue`*	Specifies the x offset value, in microns, from a specified corner. Corner numbers are counted clock-wise from the lower left corner starting with 0.  *Data_type*: float, optional
`-y` *`yValue`*	Specifies the y offset value, in microns, from a specified corner. *Data_type*: float, optional

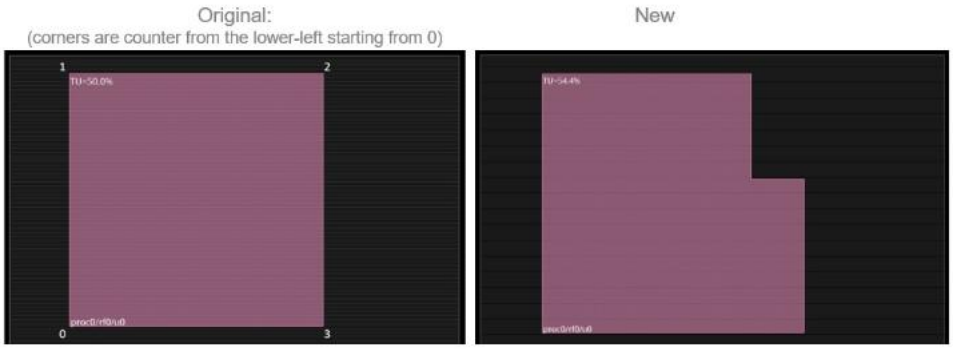
Example

- The following command cuts away the area of proc0/rf0/u0 which overlaps proc0/cmem0/itags0/u0/id0, plus an additional 30um in the X-direction and 15um in the Y-direction resulting in the shape below:
  ```
  cutRectilinearInst -inst proc0/rf0/u0 -refInst proc0/cmem0/itags0/u0/id0 -spacingX 30 -spacingY 15
  ```
  

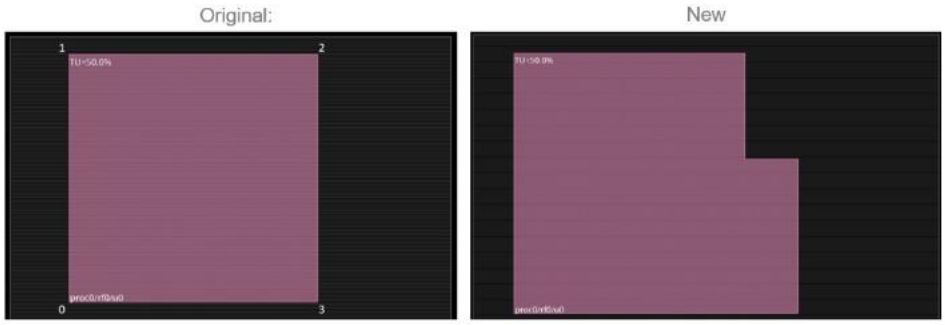
- The following command cuts away the intersection of the module and cut box resulting in the shape below:
  ```
  cutRectilinearInst -inst proc0/rf0/u0 -cutBox 430 320 670 560
  ```



- The following command ccuts the module at corner number 2. The cut is 100um in the X-direction and 200um in the Y-direction:
  ```
  cutRectilinearInst -inst proc0/rf0/u0 -corner 2 -x 100 -y 200
  ```



- The command following cuts the module at corner number 2. The cut is 100um in the X-direction and 200um in the Y-direction. The area is maintained by pushing out the left edge (notice utilization of 50% is maintained):
  ```
  cutRectilinearInst -inst proc0/rf0/u0 -corner 2 -x 100 -y 200 -pushOut left
  ```



Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*.

cutRow

```
cutRow
[-help]
[-area box | -object string | -selected | -fromIo {left bottom right top}]
[-halo float| {-leftGap float| -rightGap float | -topGap float | -bottomGap float}]
[-site name]
[-keepCell]
```

Cuts site rows within an area or that intersect with the selected object(s). Use this command after rows have been created. Only insts, hinsts, place blockages, route blockages, power domains, groups, or bumps can participate in row splitting.

Note: The default behavior is to cut rows that intersect with placement blockages and macros.

Parameters

`-area box`	Rows will be cut within the specified box. `box` is a tcl list containing 4 items. The items are, in order: `LLX, LLY, URX, URY`. **Note:** The `-halo` parameter cannot be applied `-area` used. *Data_type*: box, optional
`-bottomGap float`	Specifies the additional gap to be provided on the bottom side of the specified or selected object. *Data_type*: float, optional
`-fromIo {left bottom right top}`	
	Cuts core rows to keep them away from I/O cells in the minimum distance specified. `Left` means for IO cells on left side of die box, and so forth. *Data_type*: box, optional
`-halo float`	Specifies additional space to be provided on the top, bottom, left, and right sides of the objects causing rows to be cut. Rows are cut to the expanded boundary. The same value is used for all sides. *Data_type*: float, optional
`-help`	Outputs a brief description that includes the type and default information for each `cutRow` parameter. For a detailed description of the command and all of its parameters, use the man command: `man cutRow`
`-keepCell`	Specifies that all cells that are placed inside the cut row area will not be unplaced. *Default*: All cells inside the cut rows will be unplaced. *Data_type*: bool, optional
`-leftGap float`	Specifies the additional gap to be provided on the left side of the specified or selected object. *Data_type*: float, optional
`-object string`	Specifies the name of an intersecting object. Rows that intersect this object are cut. Only one name may be specified. *Data_type*: string, optional
`-rightGap float`	Specifies the additional gap to be provided on the right side of the specified or selected object. *Data_type*: float, optional
`-selected`	Specifies that the rows that intersect with the selected object(s) will be cut. **Note:** Only supports inst, hinst, power domain, group, place blockage, route_blockage, and bump as selected object(s). *Data_type*: bool, optional
`-site name`	Rows containing site `name` will be cut. *Default*: All rows in the core box are cut. *Data_type*: string, optional
`-topGap float`	Specifies the additional gap to be provided on the top side of the specified or selected object. *Data_type*: float, optional

Example

- The following command cuts the rows containing sites of type `SITE_FE283001` in the specified area:
  ```
  cutRow -site SITE_FE283001 -area 124.975 138.123 237.378 223.38
  ```
- The following command cuts the rows intersecting with selected objects, with a 0.5 micron halo:
  ```
  cutRow -selected -halo 0.5
  ```
- The following command cuts the rows intersecting with the object named `fuu`, with additional bottom gap of 0.5 microns:
  ```
  cutRow -object fuu -bottomGap 0.5
  ```

Related Information

- `deleteRow`
- `select_row`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Creating and Editing Rows

delete_bump_grid

```
delete_bump_grid
[-help]
[-grid_names string | -all ]
```

Removes the bump grid from the design. Use this command anytime after creating a bump grid.

Parameters

`-all`	Deletes all the bump grids. *Data_type*: bool, optional
`-grid_names` *string*	Specifies the name(s) of the bump grid to be deleted. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `delete_bump_grid` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_bump_grid`

Example

- The following command deletes all the bump grids in the current design:
  ```
  delete_bump_grid -all
  ```
- The following command deletes the `myGridFront_9x9` named bump grid:
  ```
  delete_bump_grid -grid_names {myGridFront_9x9}
  ```
- The following command deletes multiple bump girds:
  ```
  delete_bump_grid -grid_names {myGridFront_9x9 myGridFront_25x25}
  ```

Related Information

- `create_bump_grid`
- Floorplanning the Design chapter in the *Innovus User Guide*

delete_relative_floorplan

```
delete_relative_floorplan
[-help]
{obj_name_list | -all}
```

Removes the relative floorplan information from the database. It deletes all relative floorplan constraints, all relative constraints for the specified object(s), or removes the last executed relative floorplan action from the database. Use this command after running `create_relative_floorplan`.

Notes:

- The arrows showing relative constraints for the specified objects are removed from the floorplan display.
- The `delete_relative_floorplan` command accepts an array of objects.

Parameters

`-all`	Delete all relative floorplan constraints. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes type and default information for each `delete_relative_floorplan` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_relative_floorplan`
obj_name_list	Deletes all relative constraints for the specified object(s). **Note**: If you do not specify *obj_name_list*, this command removes the last executed relative floorplan action from the database. *Data_type*: string, optional

Example

- The following command deletes obj1, obj2, and obj3 from the database:
  ```
  delete_relative_floorplan {obj1 obj2 obj3}
  ```

Related Information

- `create_relative_floorplan`
- Floorplanning the Design chapter in the *Innovus User Guide*

deleteAllDensityAreas

```
deleteAllDensityAreas
[-help]
```

Removes all partial placement blockages from the floorplan. Use this command after importing the design.

Parameters

`-help`	Outputs a brief description of the `deleteAllDensityAreas` command. For a detailed description of the command, use the man command: `man deleteAllDensityAreas`

Related Information

- `createDensityArea`
- `setSelectedDensityArea`
- Floorplanning the Design chapter in the *Innovus User Guide*

deleteAllFPObjects

```
deleteAllFPObjects
[-help]
```

Removes all floorplan objects. The density screen, obstruction, power strips, and route guides objects are deleted. The guides and block guides are cleared from the design area. The physical instance groups are also cleared from the core area. Use this command after importing the design.

Parameters

`-help`	Outputs a brief description of the `deleteAllFPObjects` command. For a detailed description of the command, use the man command: man deleteAllFPObjects

Related Information

- `deleteSelectedFromFPlan`
- Floorplanning the Design chapter in the *Innovus User Guide*
- Floorplan Menu chapter in the *Innovus Menu Reference*
  - Clear Floorplan

deleteAllInstGroups

```
deleteAllInstGroups
[-help]
```

Removes all instance groups. Instance groups are user-created and contain instances and/or other groups. Use this command after importing the design.

Parameters

`-help`	Outputs a brief description of the `deleteAllInstGroups` command. For a detailed description of the command, use the man command: `man deleteAllInstGroups`

Examples

- The following command removes all instance groups:
  ```
  deleteAllInstGroups
  ```

Related Information

- `addInstToInstGroup`
- `createInstGroup`
- `deleteInstFromInstGroup`
- `deleteInstGroup`
- `createLogicHierarchy`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Grouping Instances

deleteAllPowerPreroutes

```
deleteAllPowerPreroutes
[-help]
```

Removes all power preroutes from the floorplan. Use this command after importing the design.

Parameters

None

`-help`	Outputs a brief description of the `deleteAllPowerPreroutes` command. For a detailed description of the command, use the man command: `man deleteAllPowerPreroutes`

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

deleteAllSignalPreroutes

```
deleteAllSignalPreroutes
[-help]
```

Removes all signal preroutes from the floorplan. Use this command after importing the design.

Parameters

`-help`	Outputs a brief description of the `deleteAllSignalPreroutes` command. For a detailed description of the command, use the man command: `man deleteAllSignalPreroutes`

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

deleteExclusiveGroups

```
deleteExclusiveGroups
[-help]
[-all | group_name]
```

Deletes existing exclusive regions for groups. You can use this command after creating exclusive groups. It restores all attributes of exclusive groups

Parameters

`-all`	Deletes all exclusive regions. When specified, the tool resets all existing exclusive groups attributes to their default values. *Data_type*: bool, optional
group_name	Specifies the name of the group to delete. The tool deletes exclusive regions of the specified groups and all other exclusive groups that have been created together. You can specify one or multiple groups after the command. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `deleteExclusiveGroups` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deleteExclusiveGroups`

Examples

- The following command deletes all exclusive regions of groups (A,B,C,D,E,F).
  ```
  deleteExclusiveGroups -all
  ```
- The following command deletes exclusive regions of groups A and F.
  ```
  deleteExclusiveGroups {A F}
  ```
- The following command deletes exclusive regions of groups (A,B,C).
  ```
  deleteExclusiveGroups B
  ```

Related Information

- `createExclusiveGroups`
- Floorplanning the Design chapter in the *Innovus User Guide*

deleteFPObject

```
deleteFPObject
[-help]
objectType
objectName
```

Deletes the specified floorplan object by name. You can use the command at any point in the design flow.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `deleteFPObject` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deleteFPObject`
`objectName`	Specifies the name of the object for the *`objectType`*. To get *`objectName`*, you can use *`dbObjToName objPtr`* where *`objPtr`* is the target object db pointer. *Data_type*: string, required
`objectType`	Specifies the type of object for the *`objectName`*. The object type can be any of the following: • Placement Blockage (`LayerShape`) • Partial Placement Blockage (`LayerShape`) • Routing Blockage (`LayerShape`) • Partition Feedthrough (`PtnFeed`) • Partition Pin Blockage (`PtnPinBlk`) *Data_type*: string, required

Examples

- The following command deletes the placement blockage (658300,824180,874060,1048640) from the current design:
  ```
  deleteFPObject LayerShape (658300,824180,874060,1048640)
  ```
  Note: You can get `"objectName"` by using `[dbObjToName objPtr]`, where `objPtr` is the db pointer to the placement blockage object.

  This is similar to,
  ```
  deleteFPobject LayerShape [dbObjToName objPtr]
  ```
- The following command deletes the Partial Placement Blockage from the current design:
  ```
  deleteFPObject LayerShape (1079960,1006880,1211620,1111280)
  ```
- The following command deletes the Routing Blockage from the current design:
  ```
  deleteFPObject LayerShape (628350,905550,857290,1057010)
  ```
- The following command deletes the Partition Feedthrough from the current design:
  ```
  deleteFPObject PtnFeed (980200,912920,1298040,954140)
  ```
- The following command delete Partition Pin Blockage from the current design:
  ```
  deleteFPObject PtnPinBlk (970780,947630,1007150,1012280)
  ```

Related Information

- `dbObjToName`
- Floorplanning the Design chapter in the *Innovus User Guide*

deleteHaloFromBlock

```
deleteHaloFromBlock
[-help]
[inst_name | -allMacro | -allBlackBox | -allCommitPtn  | -allBlock | -allIOPad | -cell name]
```

Removes block halo values for specific blocks or for all blocks. Use this command after running `addHaloToBlock`.

Parameters

`-allBlackBox`	Specifies the removal of halo values for all black boxes. *Data_type*: bool, optional
`-allBlock`	Specifies the removal of halo values for all blocks. *Data_type*: bool, optional
`-allCommitPtn`	Specifies the removal of halo values for all committed partitions. *Data_type*: bool, optional
`-allIOPad`	Specifies the removal of halo values for all IO pads. *Data_type*: bool, optional
`-allMacro`	Specifies the removal of halo values for all hard macros. *Data_type*: bool, optional
`-cell name`	Specifies the removal of halo values for all instances of a cell. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `deleteHaloFromBlock` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deleteHaloFromBlock`
inst_name	Specifies the block where the halo is to be deleted. You can use wildcards (* or ?) for specifying instance names. *Data_type*: string, optional

Example

The following command removes block halo values from a hard macro named cube32:

```
deleteHaloFromBlock cube32
```

Related Information

- `addHaloToBlock`
- Floorplanning the Design chapter in the *Innovus User Guide*
- Floorplan Menu chapter in the *Innovus Menu Reference*
  - Edit Halo

deleteInstFromInstGroup

```
deleteInstFromInstGroup
[-help]
group_name
object_name
```

Removes a hierarchical instance, an instance or list of instances, or a group from a specified group. You can pass a list of instances and instance pointers to the `deleteInstFromInstGroup` command. Use this command after importing the design.

Parameters

group_name	Specifies the name of the created group. *Data_type*: (group), required
-help	Outputs a brief description that includes the type and default information for each `deleteInstFromInstGroup` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deleteInstFromInstGroup`
object_name	Specifies the name of the object, another group, hierarchical instance, or leaf instance name. You can use wildcards (*?) with this parameter. **Note:** You can specify an instance names list in the { *inst_name_list* } format. The names list option is used for instances only and NOT for group or hierarchical instances. *Data_type*: (inst|hInst|group)*, required

Examples

- The following command removes hierarchical instance MEM_DSCAN from group adder1:
  ```
  deleteInstFromInstGroup adder1 MEM_DSCAN
  ```
- The following command removes group subadder from group adder1:
  ```
  deleteInstFromInstGroup adder1 subadder
  ```
- The following command removes all instance names that begin with the letter G from group adder1:
  ```
  deleteInstFromInstGroup adder1 G*
  ```

Related Information

- `addInstToInstGroup`
- `createInstGroup`
- `deleteAllInstGroups`
- `deleteInstGroup`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Grouping Instances

deleteInstGroup

```
deleteInstGroup
[-help]
group_name
```

Removes a group that was created. Use this command after creating an instance group name.

Note: If the group being removed is also a power domain, a warning will be displayed and the command will not be executed.

Parameters

group_name	Specifies the name of the user created instance group name. *Data_type*: string, required
-help	Outputs a brief description that includes the type and default information for each `deleteInstGroup` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deleteInstGroup`

Example

The following commands create and remove the instance group ADDER1:

```
createInstGroup ADDER1
deleteInstGroup ADDER1
```

Related Information

- `addInstToInstGroup`
- `createInstGroup`
- `deleteAllInstGroups`
- `deleteInstFromInstGroup`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Grouping Instances

deleteIoFiller

```
deleteIoFiller
[-help]
[-cell {name_list}]
[-from coord]
[-prefix prefix]
[-ring num]
[-side {top | bottom | left | right}]
[-to coord]
[-logic [-deriveConnectivity]]
```

Deletes I/O filler cell instances from the design. Use this command after filler cells have been added using the `addIoFiller` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `deleteIoFiller` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deleteIoFiller`
`-cell {name_list}`	Specifies the name list of the I/O filler cells to be removed. *Data_type*: list, optional
`-deriveConnectivity`	Deletes connectivity for I/O fillers. *Data_type*: bool, optional
`-from coord`	Specifies the coordinates of the starting location from where I/O filler cells are to be removed. The values are in micrometers. *Data_type*: float, optional
`-logic`	Deletes non-physical logical I/O filler cells created by the addIoFiller command. *Data_type*: bool, optional
`-prefix prefix`	Specifies the prefix of the names of I/O filler instances to be removed. *Data_type*: float, optional
`-ring num`	Specifies the number of the ring from which I/O filler cells are to be removed. Use this parameter if you want to delete I/O filler cell instances from only a specific I/O ring in a multiple I/O ring design. *Data_type*: int, optional
`-side {top | bottom | left | right}`	
	Specifies the side as top side, bottom side, left side, and right side of the I/O box from which the I/O filler cells are to be removed. *Default*: If you do not specify this parameter, all sides are specified. *Data_type*: string, optional
`-to coord`	Specifies the coordinates of the ending location till which I/O filler cells are to be removed. The values are in micrometers. *Data_type*: float, optional

Related Information

- `addIoFiller`
- Floorplanning the Design chapter in the *Innovus User Guide*

deleteIoInstance

```
deleteIoInstance
[-help]
-instName {name1 name2 ...}
```

Deletes the specified I/O instance. You can use this command at any point in the design flow.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `deleteIoInstance` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deleteIoInstance`
`-instName {name1 name2 ...}`	
	Specifies the name(s) of I/O instance(s) to be deleted. *Data_type*: list, required

Example

- The following command deletes the specified I/O instance:
  `deleteIoInstance -instName` newIo

Related Information

- `addIoInstance`
- Floorplanning the Design chapter in the *Innovus User Guide*

deleteIoRowFiller

```
deleteIoRowFiller
[-help]
-cell {fillerCellNameList ...}
[-logic]
[-prefix prefix]
[-ioRow name]
[-from coord] [-to coord]
```

Deletes the I/O row filler cells. You can use this command at any point in the design flow.

Parameters

`-cell {fillerCellNameList ...}`	
	Specifies the name of the filler cell(s) to delete from the I/O row. *Data_type*: string, optional
`-from coord`	Specifies the coordinates of a specified starting range where I/O filler cells are to be deleted. The values are in micrometers. *Data_type*: float, optional
`-help`	Outputs a brief description that includes the type and default information for each `deleteNetWeight` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deleteNetWeight`
`-ioRow name`	Specifies the name of the I/O row to delete the filler cell. *Data_type*: string, optional
`-logic`	Deletes non-physical I/O fillers. *Data_type*: bool, optional
`-prefix prefix`	Specifies the prefix name to be appended to the I/O row filler cell to be deleted. *Data_type*: string, optional
`-to coord`	Specifies the coordinates of a specified ending range where I/O row filler cells to be deleted. The values are in micrometers. *Data_type*: float, optional

Example

- The following command deletes the specified I/O instance.
  ```
  deleteIoRowFiller -cell PAD8 -prefix PAD8
  ```

Related Information

- `createIoRow`
- `addIoRowFiller`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Performing I/O Row Based Pad Placement

deleteNetWeight

```
deleteNetWeight
[-help]
{-all | netName ...}
```

Removes the net weight values on the specified nets. You can use this command after running the `specifyNetWeight` command.

Parameters

`-all`	Specifies the removal of net weight values on all nets. *Data_type*: bool, required
`-help`	Outputs a brief description that includes the type and default information for each `deleteNetWeight` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deleteNetWeight`
netName	Specifies the net names, separated by spaces, for which the net weight values are removed. *Data_type*: string, required

Related Information

- `specifyNetWeight`
- `getNetWeight`
- Floorplanning the Design chapter in the *Innovus User Guide*

deletePGPin

```
deletePGPin
[-help]
[-area {x1 y1 x2 y2}]
[-layer layer_id | {list_of_layer}]
[-net net_name]
[-pg_port]
[-all | -selected ]
```

Deletes power/ground pins as per the specified parameters and reports the number of deleted power/ground pins. You can use this command at any point in the design flow.

Note: The `deletePGPin` command only deletes the pin shapes and fterms created using the `createPGPin` command. If the power/ground pins are added using the `loadFPlan` command, then the `deletePGPin` command only deletes the pin shapes and keeps the pg pins.

If there is no power/ground pin and pin box definition in the .fp.gz file, and you add the power/ground pins using the `createPGPin` command, then Innovus will automatically add the pg fterm for it. In such a case, the `deletePGPin` command deletes both pin shapes and pg pins.

Parameters

`-all`	Deletes all the power/ground pins. *Data_type*: bool, optional
`-area x1 y1 x2 y2`	
	Specifies a bounding box for the area in which power/ground pins are to be deleted. *Data_type*: box, optional
`-help`	Outputs a brief description that includes the type and default information for each `deletePGPin` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deletePGPin`
`-layer layer_id | {list_of_layer}`	
	Specifies the layer(s) on which the power/ground pins are to be deleted. **Note**: Layers can be specified using the LEF layer names or layer ID numbers. *Data_type*: string, optional
`-net net_name`	Specifies the name of the net whose attached power/ground pins are to be deleted. *Data_type*: (net)*, optional
`-pg_port`	Deletes pg ports with pg shape or deletes pg ports based on the net name specified with the `-net` option. *Data_type*: bool, optional
`-selected`	Deletes the selected power/ground pins. *Data_type*: bool, optional

Related Information

- `createPGPin`
- `loadFPlan`
- `addStripe`
- Floorplanning the Design chapter in the *Innovus User Guide*

deletePlaceBlockage

```
deletePlaceBlockage
[-help]
[-all | -type { hard | soft | partial | macroOnly}]
[-all | obs_name ...]
```

Removes all or individual placement obstructions from the floorplan. Use this command after importing the design.

Parameters

`-all`	Specifies that all placement obstructions are removed from the floorplan. *Data_type*: bool, required
`-help`	Outputs a brief description that includes the type and default information for each `deletePlaceBlockage` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deletePlaceBlockage`
obs_name	Specifies the name of an obstruction or a group of obstructions, with each obstruction name separated by a space. *Data_type*: list, required
`-type {hard | soft | partial | macroOnly}`	
	Specifies the type of blockage to delete. *Default*: `hard` *Data_type*: enum, required

Example

The following command deletes obstructions CTSOBS1 and ABCOBS:

```
deletePlaceBlockage CTSOBS1 ABCOBS
```

Related Information

- `createPlaceBlockage`
- `setFPlanMode`
- Floorplanning the Design chapter in the *Innovus User Guide*

deleteRouteBlk

```
deleteRouteBlk
[-help]
[-box {x1 y1 x2 y2} | -name blk | -all ]
[-layer layerName | {layerNameList} | all]
[-cutLayer layerName | {layerNameList} | all]
[-drcRegionLayer layerName | {layerNameList} | all]
[-trimMetalLayer layerName | {layerNameList} | all]
[-type {all | routes | fills | slots | partial}]
```

Deletes a routing blockage object. Use this command during partition floorplanning.

Note: The `deleteRouteBlk` command matches the layer name and the layer type while deleting routing blockages. If the layer name and the layer type do not match, the specified layer is not deleted and a warning about the mismatch is displayed. This rule is honored by standard layers, cutLayers, drcRegionLayers, and trimMetalLayers. When a list of layers is specified, only the layers matching with the specified layer type are deleted.

Parameters

`-all`	Removes all route blockage objects in the floorplan. *Data_type*: bool, required
`-box {x1 y1 x2 y2}`	
	Specifies the coordinates of the bounding box of the blockage area. *Data_type*: box, optional
`-cutLayer layerName | {layerNameList} | all`	
	Specifies the cut layer, list of cut layers, or all cut layers on which the routing blockage is to be deleted. *Default*: If you do not specify this parameter, it deletes the routing blockage object on all cut layers. *Data_type*: list, optional
`-drcRegionLayer layerName | {layerNameList} | all`	
	Specifies the drc region layer, list of drc region layers, or all drc region layers on which the routing blockage is to be deleted. *Default*: If you do not specify this parameter, it deletes the routing blockage object on all drc region layers. *Data_type*: list, optional
`-help`	Outputs a brief description that includes the type and default information for each `deleteRouteBlk` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deleteRouteBlk`
`-layer layerName | {layerNameList} | all`	
	Specifies the layer, list of layers, or all layers on which the routing blockage is to be deleted. *Default*: If you do not specify this parameter, it deletes the routing blockage object on all layers. *Data_type*: list, optional
`-name blk`	Specifies the name of the route blockage. You can specify wild cards as part of the route blockage name. *Data_type*: list, required
`-trimMetalLayer layerName | {layerNameList} | all`	
	Specifies the trim metal layer, list of trim metal layers, or all trim metal layers on which the routing blockage is to be deleted. *Data_type*: list, optional
`-type {all | routes | fills | slots | partial}`	
	Specifies the type of routing blockage to delete. The following options are available: • `all`: Deletes all types of blockages. • `routes`: Deletes the blockages that block routing. • `fills`: Deletes the blockages that block metal-fill. • `slots`: Deletes the blockages that block slots. • `partial`: Deletes partial routing blockages. Default: `all` *Data_type*: enum, optional

Examples

- The following commands create a blockage on all metal layers and deletes only on metal3:

```
createRouteBlk -box 11.2 164.07 95.005 319.205 -layer all
deleteRouteBlk -box 11.2 164.07 95.005 319.205 -layer 3
```

- The following commands create a blockage on all metal layers with the specific name BLK and deletes it.

```
createRouteBlk -box 11.2 164.07 95.005 319.205 -layer all -name BLK
deleteRouteBlk -name BLK
```

- The following commands create a blockage on all metal layers and deletes it.

```
createRouteBlk -box 400 400 500 500 -layer all
deleteRouteBlk -box 400 400 500 500
```

- The following commands create a blockage on metal3 and deletes it.

```
createRouteBlk -box 400 400 500 500 -layer 3
deleteRouteBlk -box 400 400 500 500
```

Related Information

- createRouteBlk
- setRouteBlkDefaultLayer
- Floorplanning the Design chapter in the *Innovus User Guide*

deleteRoutingHalo

```
deleteRoutingHalo
[-help]
[-lithoHalo]
{-allBlocks | -block blockNameList | -cell cellNameList | -inst instanceName | -designHalo}
```

Deletes a routing halo for a blackbox, hard macro, or block-level design. Use this command after specifying a routing halo for a blackbox, hard macro, or block-level design. You can use the `addRoutingHalo` command to add routing halos.

Parameters

`-allBlocks`	Deletes the routing halo for all blocks and black boxes. *Data_type*: bool, required
`-block` *blockNameList*	Deletes the routing halo for the specified blocks. *Data_type*: list, required
`-cell` *cellNameList*	Deletes the routing halo for the specified cells. *Data_type*: list, required
`-designHalo`	Deletes the routing halo at the design level. *Data_type*: bool, required
`-help`	Outputs a brief description that includes the type and default information for each `deleteRoutingHalo` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deleteRoutingHalo`
`-inst` *instanceName*	Specifies that the routing halo should be added to the specified instance. Use this option to delete the halo of an I/O pad instance. *Data_type*: list, required
`-lithoHalo`	Deletes the litho routing halo. *Data_type*: bool, optional

Example

- The following command delete the routing halo for all blocks and black boxes .
  ```
  deleteRoutingHalo -allBlocks
  ```

Related Information

- `addRoutingHalo`
- Floorplanning the Design chapter in the *Innovus User Guide*
- Floorplan Menu chapter in the *Innovus Menu Reference*
  - Edit Halo - Routing Halo

deleteRow

```
deleteRow
[-help]
{-all | -selected | -site string| row}
```

Deletes the specified row(s). Use this command after importing a design.

Parameters

`-all`	Specifies that all rows will be deleted. *Data_type*: bool, required
`-help`	Outputs a brief description that includes the type and default information for each `deleteRow` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deleteRow`
row	Specifies the name or id of the row(s) to be deleted. *Data_type*: list, required
`-selected`	Specifies that the selected rows should be deleted. *Data_type*: bool, required
`-site string`	Specifies the site for which rows will be deleted. *Data_type*: string, required

Example

The following command deletes rows for the site site0A10.

```
deleteRow -site site0A10
```

Related Information

- `select_row`
- `cutRow`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Creating and Editing Rows

deleteSelectedFromFPlan

```
deleteSelectedFromFPlan
[-help]
```

Deletes the currently selected floorplan objects. You can use this command at any point in the design flow.

Parameters

`-help`	Outputs a brief description of the `deleteSelectedFromFPlan` command. For a detailed description of the command, use the man command: `man deleteSelectedFromFPlan`

Related Information

- `deleteAllFPObjects`
- Floorplanning the Design chapter in the *Innovus User Guide*

deleteSizeBlockage

```
deleteSizeBlockage
[-help]
[-name blockageName]
[-selected]
[-all]
```

Deletes a size blockage object. You can use this command after resizing the floorplan.

Parameters

`-all`	Deletes all the size blockages. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes type and default information for each `deleteSizeBlockage` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deleteSizeBlockage`
`-name blockageName`	
	Specifies the name of the size blockage that is to be deleted. *Data_type*: string, optional
`-selected`	Deletes the selected size blockage. *Data_type*: bool, optional

Example

- The following command creates a size blockage and deletes it:

```
addSizeBlockage -name sizeBlkg1 -box 400 400 500 500
deleteSizeBlockage -name sizeBlkg1
```

Related Information

- `addSizeBlockage`
- `getResizeFPlanMode`
- `resizeFloorplan`
- `setResizeFPlanMode`
- `setResizeLine`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Resizing the Floorplan

deselectAll

```
deselectAll
[-help]
```

Deselects all selected nets. Once the nets are deselected, they are no longer highlighted in the design display window. Use this command after selecting one or more nets.

Parameters

`-help`	Outputs a brief description of the `deselectAll` command. For a detailed description of the command, use the man command: `man deselectAll`

Related Information

- `selectNet`
- Floorplanning the Design chapter in the *Innovus User Guide*

deselectGroup

```
deselectGroup
groupName
[-help]
```

Deselects a group. Use this command after selecting a group.

Parameters

groupName	Specifies the name of the group. *Data_type*: string, required
-help	Outputs a brief description that includes the type and default information for each `deselectGroup` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deselectGroup`

Related Information

- `selectGroup`
- Floorplanning the Design chapter in the *Innovus User Guide*

deselectInst

```
deselectInst
[-help]
instName
```

Deselects the specified instance. Once an instance is deselected, it is no longer highlighted in the design display window. Use this command after selecting one or more instances.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `deselectInst` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deselectInst`
`instName`	Specifies the name(s) of the instance(s). *Data_type*: (inst)+, required

Examples

- The following command deselects the inst named inst_1:
  ```
  deselectInst inst_1
  ```
- The following command deselects the insts named inst_2 and inst_3:
  ```
  deselectInst {inst_2 inst_3}
  ```

Related Information

- `selectInst`
- Floorplanning the Design chapter in the *Innovus User Guide*

deselectInstByCellName

```
deselectInstByCellName
[-help]
cellName
```

Deselects an instance by cell name. Use this command after selecting an instance by cell name.

Parameters

cellName	Specifies the name of the cell. *Data_type*: (libCell/ptnCell|vCell|topCell)+, required
-help	Outputs a brief description that includes the type and default information for each `deselectInstByCellName` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deselectInstByCellName`

Related Information

- `selectInstByCellName`
- Floorplanning the Design chapter in the *Innovus User Guide*

deselectInstOnNet

```
deselectInstOnNet
[-help]
netName
```

Deselects an instance on a net. Use this command after selecting an instance on a net.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `deselectInstOnNet` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deselectInstOnNet`
netName	Specifies the name of the net. *Data_type*: (net)*, required

Related Information

- `selectInstOnNet`
- Floorplanning the Design chapter in the *Innovus User Guide*

deselectIOPin

```
deselectIOPin
[-help]
pinName
```

Deselects an I/O pin. Use this command after selecting an I/O pin.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `deselectIOPin` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deselectIOPin`
pinName	Specifies the name of the I/O pin. *Data_type*: string, required

Related Information

- `selectIOPin`
- Floorplanning the Design chapter in the *Innovus User Guide*

deselectNet

```
deselectNet
[-help]
[netName]
[-clock | -nonDefaultRule | -shield ]
```

Deselects the specified net. Once a net is deselected, it is no longer highlighted in the design display window. Run this command after selecting one or more nets.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `deselectNet` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deselectNet`
netName	Specifies the name of the net you want to deselect. You cannot specify more than one net. You can, however, use a wild card (*). *Data_type*: (net)*, optional
`-clock`	Deselects all nets that have a DEF attribute + USE CLOCK. *Data_type*: bool, optional
`-nonDefaultRule`	Deselects all nets that have a DEF attribute + NONDEFAULTRULE. *Data_type*: bool, optional
`-shield`	Deselects all nets that have a DEF attribute + SHIELDNETS. *Data_type*: bool, optional

Examples

- The following command deselects the net named net_1:
  ```
  deselectNet net_1
  ```
- The following command deselects all clock nets:
  ```
  deselectNet -clock
  ```

Related Information

- `deselectAll`
- `selectNet`
- Floorplanning the Design chapter in the *Innovus User Guide*

display_obj_connectivity

```
display_obj_connectivity
[-help]
[-direction {in|out|all}]
[-level level]
[-line_to_ports]
[-min_connection number]
{-insts string | -ports string | -selected | -reset }
[-through_registers [-all_registers ]]
```

Displays the connections of specified macros, selected macros, or specified ports. You can also display the input and output connectivity separately. By default it displays both.

Parameters

`-all_registers`	Displays all registers found during the analysis, even those which do not drive to other macros. **Note:** This parameter must be used with the `-through_registers` option. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes type and default information for each `display_obj_connectivity` parameter. For a detailed description of the command and all of its parameters, use the man command: `man display_obj_connectivity`
`-direction {in|out|all}`	Specifies the pin direction of objects to be displayed. *Default*: `all` *Data_type*: string, optional
`-insts string`	Specifies the macros for which the connectivity is displayed. *Data_type*: string, optional
`-level level`	Specifies the trace level to display. *Maximum Level*: 10 *Minimum Level*: 1 *Default*: 1 *Data_type*: int, optional
`-line_to_ports`	Displays the display paths from/to primary ports. *Data_type*: bool, optional
`-min_connection number`	Specify the trace minimum connection. *Default*: 1 *Data_type*: int, optional
`-ports string`	Specifies the IO ports for which the connectivity is displayed. *Data_type*: string, optional
`-reset`	Clears the trace macro display. *Data_type*: bool, optional
`-selected`	Displays the connectivity of selected macros. *Data_type*: bool, optional
`-through_registers`	Displays the flight lines going through registers. *Data_type*: bool, optional

Example

The following command displays the connections for blk1 upto level 3:

```
trace_obj_connectivity -insts blk1 -level 3
```

Related Information

- `trace_obj_connectivity`
- `set_trace_obj_connectivity_mode`
- `get_trace_obj_connectivity_mode`
- Floorplanning the Design chapter in the *Innovus User Guide*
- Floorplan Menu chapter in the *Innovus Menu Reference*

exportNdr

```
exportNdr
[-help]
ndrRuleName
{-def defFileName | -lef lefFileName}
```

Exports/saves the new rule that was created with the `add_ndr` command into a LEF/DEF file. You can use this command after creating a non-default rule.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `exportNdr` parameter. For a detailed description of the command and all of its parameters, use the man command: `man exportNdr`
ndrRuleName	Specifies the name of a non-default rule. *Data_type*: string, required
`-def defFileName`	
	Specifies the name of a DEF file, to which the non-default rule is exported. *Data_type*: string, required
`-lef lefFileName`	
	Specifies the name of a LEF file, to which the non-default rule is exported. *Data_type*: string, required

Example

The following command exports the non-default rule, testndr to the LEF file, all.lef.

```
exportNdr testndr -lef all.lef
```

Related Information

- `add_ndr`
- `modify_ndr`
- Floorplanning the Design chapter in the *Innovus User Guide*
- Using the NanoRoute Router chapter in the *Innovus User Guide*
  - Using Non-Default Rules

finishFloorplan

```
finishFloorplan
[-help]
[-mask integer]
{ [-autoHalo | [-autoHaloBasedOnPitch [-margin value]] | -addHalo width |
[-fillPlaceBlockage blkgType maxGap
 [-density value] [-excludeFlops] [-deadArea float] [-honor_placement_density]] |
[-fillRouteBlockage maxGap [-layer layerID]
[-cutLayer cutLayerID]
[-fills ] [-exceptpgnet ]
[-spacing spaceValue | -designRuleWidth width] [-deadArea maxDeadArea]] |
-addCornerBlockage string |
[-drcRegionLayer layer+ [-edgeExtend {x y}] [-edgeShrink {x y}]]| -addBoundaryEOLBlockage | -addBoundaryBlockage | -
addRowBasedRegionBlockage| -undo ]
[-area {x1 y1 x2 y2}]
[-namePrefix name] }
```

Performs advanced placement-related refinements to a floorplan, to produce a more polished floorplan. The `finishFloorplan` command, can be used on any floorplan, including third-party generated floorplans. You can use the `setFinishFPlanMode` command to set the active objects and specify the channel direction for the `finishFloorplan` command to use. The `setFinishFPlanMode` parameters affects the behavior the `finishFloorplan` command.

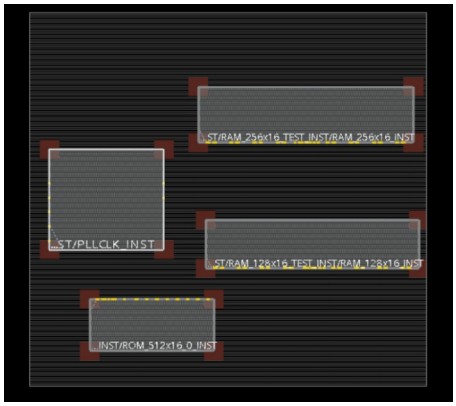
Note: You can access the functionality of the `finishFloorplan` command using the `finishFPlan` alias.

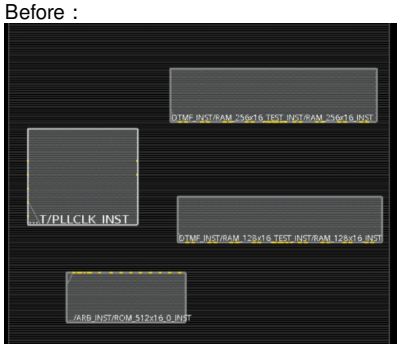
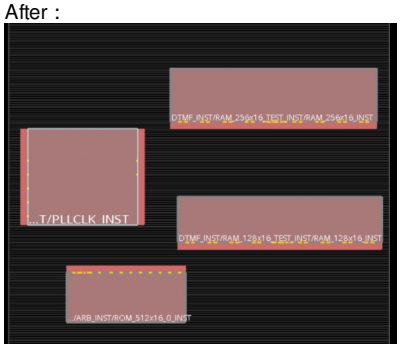
Note: The `finishFloorplan` command honors the settings of the `-no_cut_row` parameter of the `setFPlanMode` command that specifies if rows are not required to be cut before other operations. When the `-no_cut_row` parameter is disabled, the `finishFloorplan` command adds blockages around the current row/non-row area in the design when the "`-activeObj row`" or "`-drcRegionObj nonRowArea`" options of the `setFinishFPlanMode` command are set. If rows are not cut before other operations but the `-no_cut_row` parameter is enabled, the `finishFloorplan` command can work as cut row flow to add blockages around row/non-row area which satisfy row constraints.

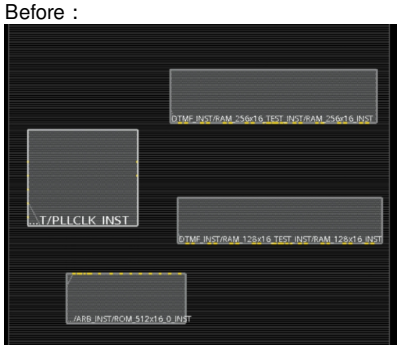
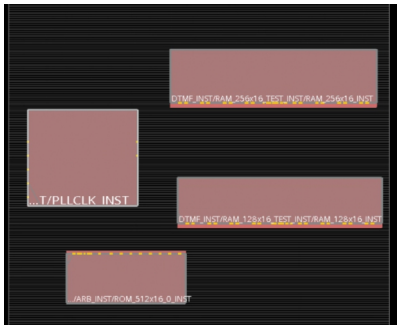
Parameters

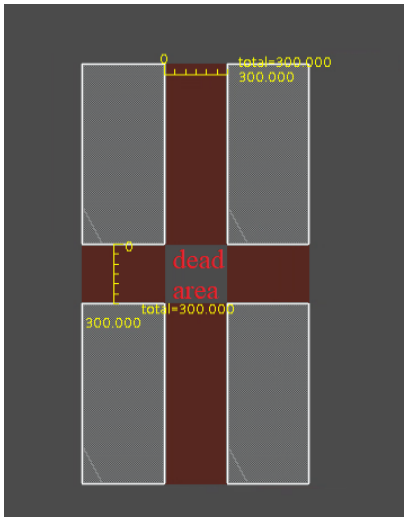
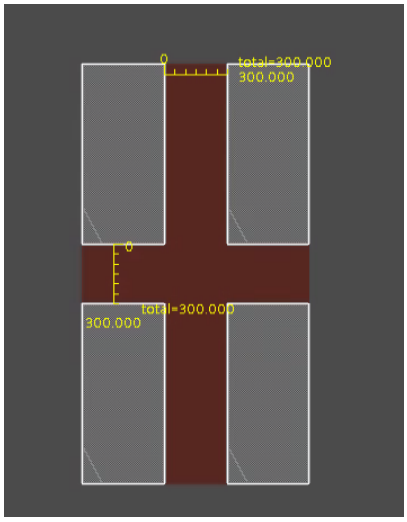
`-addBoundaryBlockage`	Adds boundary routing blockages with "BOUNDARYBLOCKAGE" property around active object boundary. This parameter can only work with die as an active object. For each layer, boundary blockage is perpendicular to the preferred routing direction. *Data_type*: bool, optional

`-addBoundaryEOLBlockage`	Adds end-of-line(EOL) routing blockages with "COREEOLBLOCKAGE" property around active object boundary. This parameter can only work with rows as active objects. For each layer, EOL routing blockage is perpendicular to the preferred routing direction.
	Note: A Core EOL blockage rule is used to specify line-end blockages formed from the core edges. The EOL Blockage is defined in LEF as below:
	`PROPERTY LEF58_COREEOLBLOCKAGE "COREEOLBLOCKAGE outwardSize inwardSize [SIDEEXTENSION sideExtension] ;" ;`
	`COREEOLBLOCKAGE outwardSize inwardSize SIDEEXTENSION sideExtension`
	Defines the line-end blockages formed from the core edges in the orthogonal direction of the preferred routing direction of the layer of a design. The blockage is formed by going outward from the core by outwardSize and inward by inwardSize. If SIDEEXTENSION is defined, the blockage is extended by sizeExtension along the direction of the core edge on both sides. These blockages prevent any line-end of a wire end inside it. If a wire goes through the blockage and just touches the far end edge, it is still a violation. If a wire just touches an edge without extending inside the blockage, it is fine. .
	Data_type: bool, optional

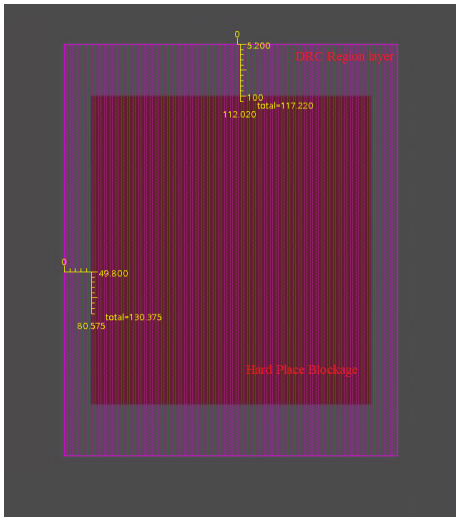
`-addCornerBlockage x y`	Specifies the width and height of the hard placement blockage that is automatically added around the corner. Example: `finishFloorplan -addCornerBlockage 50 50` Before： After：  *Data_type*: string, optional
`-addHalo width`	Adds halo of the specified width. *Data_type*: float, optional
`-addRowBasedRegionBlockage`	When specified, it adds row based region blockages. This parameter adds region layer blockages with “ROWBASED” property in some row areas. **Note:** The row area is determined by standard cells with both such row sites and OBS on the region layer. *Data_type*: bool, optional
`-area {x1 y1 x2 y2}`	Specifies the lower left x coordinate, lower left y coordinate, upper right x coordinate, and upper right y coordinate respectively of the blockage area. *Data_type*: box, optional

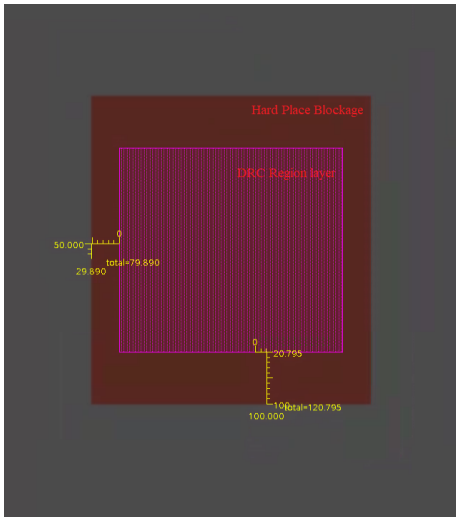
`-autoHalo`	Automatically adds halos around every macro in design. For each macro edge, the halo thickness is proportional to pin density on that edge. Example: `finishFloorplan -autoHalo` Before：  After：  *Data_type*: bool, optional

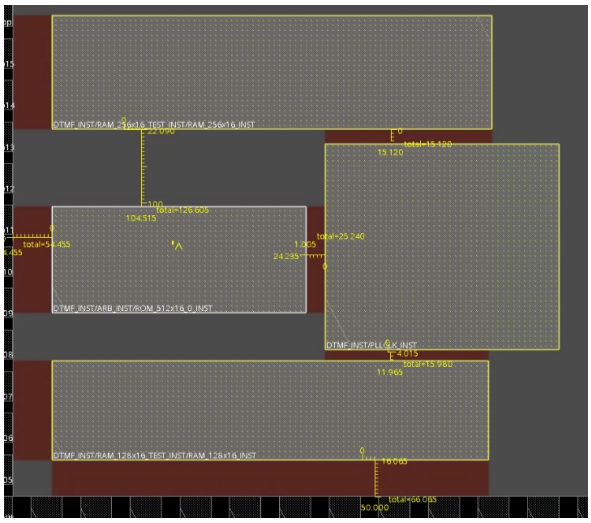
`-autoHaloBasedOnPitch`	Automatically add halos based on the number of pins, routing layer, and layer pitch on the edge. Example: `finishFloorplan -autoHaloBasedOnPitch` Before：  After：  *Data_type*: bool, optional
`-cutLayer` *cutLayerID*	Specifies the cut layer ID on which the routing blockage is to be applied. *Data_type*: string, optional

`-deadArea` *float*	Specifies the maximum value of dead area.
	Note: A dead area is an island with placeable rows that has a narrow channel connect to main core area. For example, the placeable rows surrounded by macros.
	Example：
	`finishFloorplan -fillPlaceBlockage hard 300`
	 
	`finishFloorplan -fillPlaceBlockage hard 300 -deadArea 90000`
	 
	Data_type: float, optional

`-density` *value*	Specifies the density of the partial placement blockage. *Data_type*: string, optional

`-designRuleWidth width`	Specifies the effective width of the routing blockage. The value is a real number greater than 0. *Data_type*: float, optional
`-drcRegionLayer layer+`	Specifies the region layer name list. *Data_type*: string, optional
`-edgeExtend {x y}`	Specifies that the DRC region can be extended from the specified edge. The `-edgeExtend` parameter is used for creating DRC region blockages by extending from the specified drcRegion objects. **Note:** • The specified value for option `-edgeExtend` should be equal to or greater than zero. If the specified value is less than zero, treat it as zero. • For X/Y direction, if the `-edgeExtend` and `-edgeShrink` options are specified at the same time, then Innovus compares the value first and then extends or shrinks. Example, `-edgeExtend {3 4}` and `-edgeShrink {2 5}` are specified at the same time, tool will extend 1(3-2) in X-direction and Shrink 1(5-4) in Y-direction. Example: `setFinishFPlanMode -drcRegionObj hardBlkg` `finishFloorplan -drcRegionLayer R1 -edgeExtend {50 100}`  *Data_type*: point, optional

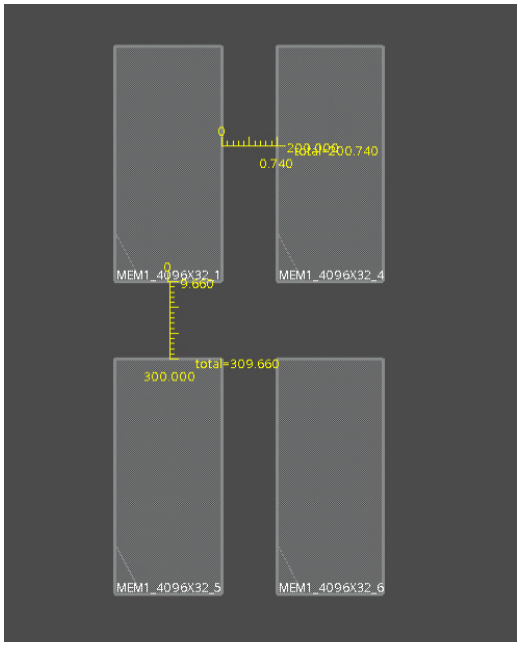
`-edgeShrink {x y}`	Specifies that the DRC region can be shrunk from the specified edge. The `-edgeShrink` parameter is used for creating DRC region blockages by shrinking the specified drcRegion objects. **Note:** • The specified value for option `-edgeShrink` should be equal to or greater than zero. If the specified value is less than zero, treat it as zero. • For X/Y direction, if the `-edgeExtend` and `-edgeShrink` options are specified at the same time, then Innovus compares the value first and then extends or shrinks. Example, `-edgeExtend {3 4}` and `-edgeShrink {2 5}` are specified at the same time, tool will extend 1(3-2) in X-direction and Shrink 1(5-4) in Y-direction. Example: `setFinishFPlanMode -drcRegionObj hardBlkg` `finishFloorplan -drcRegionLayer R1 -edgeShrink {50 100}`  *Data_type*: point, optional
`-exceptpgnet`	Specifies that the routing blockage is to be applied on a signal net routing and not on power or ground net routing. *Data_type*: bool, optional
`-excludeFlops`	Specifies that flops and latches will be excluded from the partial placement blockage. **Note**: The `-density` and `-excludeFlops` options must be used with the `-fillPlaceBlockage` parameter. *Data_type*: bool, optional
`-fillPlaceBlockage blkgType maxGap`	
	Adds placement blockages of the specified blockage type (`blkgType`) to fill macro-to-macro, macro-to-boundary, macro-to-OBS, OBS-to-OBS, and boundary-to-OBS gaps up to a specified size (`maxGap`). • For `blkgType`, you can specify soft, hard, or partial. • For `maxGap`, specify a positive value, in microns. *Default*: `-autoPlaceBlockage` Example1: `finishFloorplan -fillPlaceBlockage hard 50`



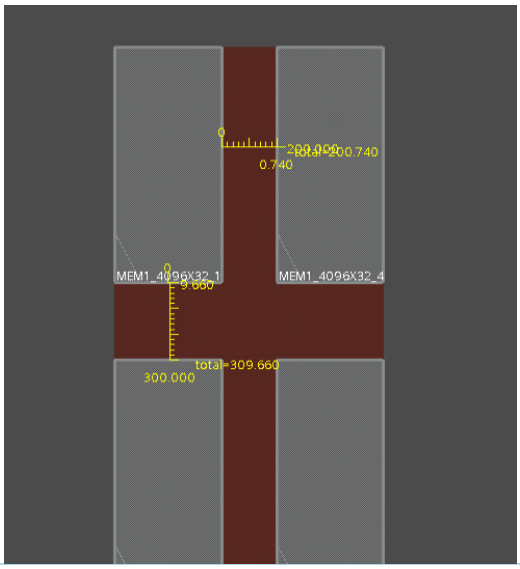
Example2: `setFinishFPlanMode -activeObj {macro hardblkg}`

`finishFloorplan -fillPlaceBlockage hard 300`

Before:

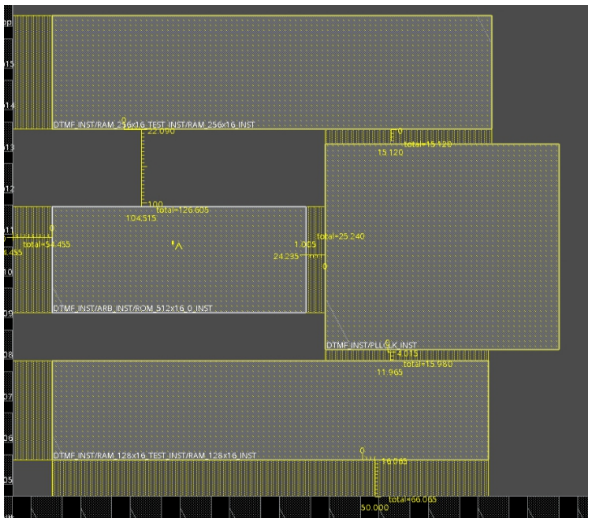


After:



`-fillRouteBlockage` *maxGap*

Adds routing blockage upto a specified size (maxGap)
Example: `finishFloorplan -fillRouteBlockage 50`



Data_type: float, optional

`-fills`

Specifies that the routing blockage is to be applied on metalfills.

Data_type: bool, optional

`-help`

Outputs a brief description that includes type and default information for each `finishFloorplan` parameter.

For a detailed description of the command and all of its parameters, use the man command:
`man finishFloorplan`.

`-honor_placement_density`	Adds standard cell density check when adding placement blockages. **Note:** • If option the `-density` parameter is not specified with the "`finishFloorplan -fillPlaceBlockage soft`" command, the density of the created soft blockage is 100. • If there are standard cells in the channel, partial blockages will be created with the density of the current standard cell in the channel when the following command is used: `finishFloorplan -fillPlaceBlockage hard/soft -honor_placement_density` • If there are standard cells in the channel and the specified density for the partial blockage is less than the density of the current standard cell in the channel, partial blockages will be created with the density of the current standard cell in the channel with the "`finishFloorplan -fillPlaceBlockage partial -density`" command. • If there are standard cells in the channel and the specified density for partial blockage is larger than the density of the current standard cell in the channel, partial blockages will be created with the specified density when the "`finishFloorplan -fillPlaceBlockage partial -density`" command is called. *Data_type*: bool, optional
`-layer` *layerID*	Specifies the layer ID of the route blockage. *Data_type*: string, optional
`-margin` *value*	Specifies the pitch margin when halos are added using `-autoHaloBasedOnPitch`. *Default*: `0.1` *Data_type*: float, optional

`-mask` *integer*	Specifies the mask number used when adding routing blockages and creating DRC region.
	• The `-mask` parameter enables the `finishFloorplan` command to block routing layers with specified masks when adding routing blockages and creating DRC region. Layers with other masks can still be routed through the blockage.
	• The `-mask` parameter must be used with one or more of `-layer`, `-cutLayer` and `-drcRegionLayer` parameters.
	• Only one mask can be specified. The default value of the `-mask` parameter is 0. This implies that all the specified layers will be blocked. Uncolored routes are always blocked.
	• If the `-mask` parameter is specified with an integer value greater than 0, then both layers with specified mask and uncolored layers are blocked.
	Example. If `-mask 1` is specified, it blocks mask 1 and uncolored routes.
	• If more than one layers are associated with a mask, the mask is applied to all of them.
	• If the `-mask` parameter is specified, the tool checks if the mask is suitable for all of the specified layers. If not, it displays an ERROR message and no blockages are created. The number of arguments specified with the –`mask` parameter must be same as the number of arguments specified with the `-layer`, `-cutLayer` and `-drcRegionLayer` parameters.
	• The `setNanoRouteMode` and `verify_drc` commands honor the `-mask` parameter.
	Example:
	• The following command creates routing blockages for blocking mask 2 of M1 layer.
	`finishFloorplan -layer M1 -mask 2 -fillRouteBlockage 400`
	• The following command(s) DRC region for mask 1 of PIO layer in core to die area.
	`setFinishFPlanMode -drcRegionObj {coreSpacing}`
	`finishFloorplan -drcRegionLayer PIO -mask 1 -edgeExtend {0.083 0}`
	Default: `0`
	Data_type: int, optional

`-namePrefix` *name*	Specifies the name prefix for the added blockage to avoid naming conflict with the existing one. *Data_type*: string, optional
`-spacing` *spaceValue*	Specifies the minimum spacing between layers within the routing blockage. The value is a real number greater than or equal to 0. *Data_type*: float, optional

`-undo`	Reverses the previous run of `finishFloorplan`. *Data_type*: bool, optional

Example

- The following command creates a dead area with partial placement blockage of gap of 5 microns and density 60:
  ```
  finishFloorplan -deadArea 100 -fillPlaceBlockage partial 5 -density 60
  ```

Related Information

- `getFinishFPlanMode`
- `setFinishFPlanMode`
- Floorplanning the Design chapter in the *Innovus User Guide*.
  - Finishing the Floorplan

fix_boundary_overlaps

```
fix_boundary_overlaps
[-help]
[-move_area {x1 y1 x2 y2}]
[-groups {string1 string2 ...}]
[-hinsts {string1 string2 ...}]
[-fix_area {x1 y1 x2 y2}]
```

Resolves the overlaps between macros or hierarchical instances (hinsts). This command can also be used to resolve overlaps between instance groups.

Parameters

`-help`	Prints a brief description that includes type and default information for each `fix_boundary_overlaps` parameter. For a detailed description of the command and all of its parameters, use the man command: `man fix_boundary_overlaps`
`-fix_area {x1 y1 x2 y2}`	Specifies the coordinates of the area in which overlap needs to be resolved. *Data_type*: box, optional
`-groups {string1 string2 ...}`	Specifies the prefixed group names. Use this parameter if you want to resolve overlaps between only specified groups. *Data_type*: list, optional
`-hinsts {string1 string2 ...}`	Specifies the prefixed hinst names. Use this parameter if you want to resolve overlaps between only specified hinsts. *Data_type*: list, optional
`-move_area {x1 y1 x2 y2}`	Specifies the area where the floorplan objects can be moved when resolving overlaps. *Data_type*: box, optional

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

fix_floorplan

```
fix_floorplan
[-help]
-file fileName
-type {ufc tcic}
[-selected ]
[-report_file string]
[-report_unplaced_block ]
[-break_at_conflict ]
[-honor_snap_block_grid ]
[-selected_rules rule_name_list]
```

Fixes the detected violations as part of the UFC methodology by adding routing blockages around macro, snapping macro location, or re-shaping core shape. The UFC methodology also provides the capability to automatically fix the violations with imported UFC rules.

Note: The Unified Floorplan Constraints (UFC) methodology is a powerful Innovus capability that can define several rules on floorplan objects, check specified rules, and report the violations with rule name as well as severity. The violations can be shown in the violation browser. UFC also provides the ability to automatically fix specific violations. It can add routing blockages, snap macro location, adjust core shape to fix corresponding violations. In addition, UFC also provides the ability to exclude rules from checking for specified objects.

Usage Model:

- Load the Design.
- Create a UFC file with floorplan rules.
- Run UFC checking. (check_ufc)
- Fix reported violations. (fix_floorplan)

You can use the `fix_floorplan` command to fix the detected violations as part of the UFC methodology. It fixes violations by:

- Adding routing blockages around macros
- Snapping macro locations
- Re-shaping core shapes

The UFC methodology also provides the capability to automatically fix the violations with imported UFC rules.

Note: The `fix_floorplan` command tries to fix all the detected floorplan violations by default, however, it currently does not support fixing die size and core size violations. If a floorplan has rule conflicts, then the results after the fix may not be perfect. Cadence recommends you ensure that the rules used are not conflicting and that the die or core size satisfy the rules. Cadence also recommends you contact your Cadence representative if your constraints file comes from a third party, but you want to use the `fix_floorplan` command to fix the violations.

Note: The `fix_floorplan` command honors the `setFPlanMode -no_cut_row` parameter that specifies if rows are not required to be cut before operations. With `setFPlanMode -no_cut_row false` or not specified, the `fix_floorplan` command will work based on the current row shape. If there is any violation for rows, the `fix_floorplan` command will run the `cutRow` command to fix. With `setFPlanMode -no_cut_row true`, the tool identifies that the rows should not be cut. The `fix_floorplan` command will fix the row violations by adding place blockages instead of cutting row. The place blockages will be added to the row area that will be cut out and kept after fix_floorplan to satisfy the row constraint. The actual row shape will not be changed after the `fix_floorplan` command.

Parameters

`-break_at_conflict`	Enables the tool to exit from fixing violations when conflict of rules is encountered. You must fix the conflict of rules and then re-run the `fix_floorplan` command. *Data_type*: bool, optional
`-file` *fileName*	Specifies the name of file containing the rules or constraints. *Data_type*: string, required
`-help`	Prints a brief description that includes type and default information for each `fix_floorplan` parameter. For a detailed description of the command and all of its parameters, use the man command: `man fix_floorplan`
`-honor_snap_block_grid`	When specified, the `fix_floorplan` command honors the block snap grid defined by the `setFPlanMode -snapBlockGrid` command. *Data_type*: bool, optional
`-report_file` *string*	Specifies the name of the file that saves the report summary. *Data_type*: string, optional
`-report_unplaced_block`	Enables the tool to show the number of unplaced macros in summary. *Data_type*: bool, optional
`-selected`	Specifies that only the selected objects can be moved to fix violations. **Note:** The `fix_floorplan` command currently only support selected macros. *Data_type*: bool, optional
`-selected_rules` *rule_name_list*	When specified, the `fix_floorplan` command only fixes violations for specified rules. The `-selected_rules` parameter accepts rule names, rule names list, variables, and wildcards. **Note:** When this option is not specified, the `fix_floorplan` command fixes all violations by default. *Data_type*: bool, optional
`-type {ufc tcic}`	Specifies the type of input file. The `fix_floorplan` command supports ufc and tcic files. *Data_type*: enum, required

Example

- The following command fixes the ufc violations based on the rules defined in the DTMF1.ufc file and writes the report to the fix_cmds.tcl file.
  ```
  fix_floorplan -type ufc -file DTMF1.ufc -report_file fix_cmds.tcl
  ```
- The following command fixes the ufc violations for Rule A and Rule B only.
  ```
  fix_floorplan -type tcic -file test.tcic -selected_rules "RuleA RuleB"
  ```

Related Information

- `check_ufc`
- Floorplanning the Design chapter in the *Innovus User Guide*

fixAllIos

```
fixAllIos
[-help]
[-pinOnly | -cellOnly | -incAreaIo]
```

Changes the status of all I/O pins, I/O cells, or CLASS PAD AREAIO cells to a FIXED state to keep them from being reassigned. Use this command after importing the design.

Parameters

`-cellOnly`	Changes the status of all I/O cells to a FIXED state. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `fixAllIos` parameter. For a detailed description of the command and all of its parameters, use the man command: `man fixAllIos`
`-incAreaIo`	Changes the status of all CLASS PAD AREAIO cells to a FIXED state. *Data_type*: bool, optional
`-pinOnly`	Changes the status of all I/O pins to a FIXED state. *Data_type*: bool, optional

Examples

- The following command changes the status of all I/O pins and I/O cells to a FIXED state:
  ```
  fixAllIos
  ```
  Note: This is the default functionality.
- The following command changes the status of all I/O pins to a FIXED state:
  ```
  fixAllIos -pinOnly
  ```
- The following command changes the status of all I/O cells to a FIXED state:
  ```
  fixAllIos -cellOnly
  ```
- The following command changes the status of all I/O pins, I/O cells, and all CLASS PAD AREAIO cells to a FIXED state:
  ```
  fixAllIos -incAreaIo
  ```

Related Information

- `unfixAllIos`
- Floorplanning the Design chapter in the *Innovus User Guide*.

flipModule

```
flipModule
[-help]
moduleName
{X | Y}
```

Flips all hierarchical instances in the specified module, with respect to the referenced X axis or Y axis of the module. Use this command after importing the design.

Parameters

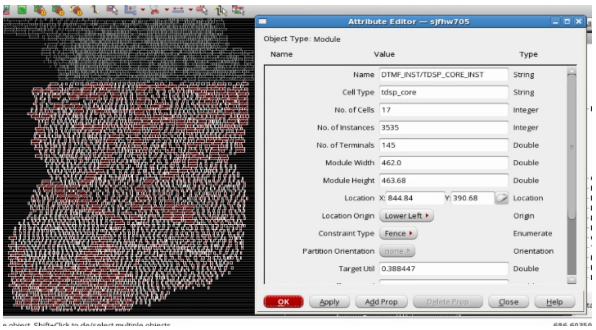
`-help`	Outputs a brief description that includes the type and default information for each `flipModule` parameter. For a detailed description of the command and all of its parameters, use the man command: `man flipModule`
`moduleName`	Specifies the name of the module whose hierarchical instances are to be flipped. *Data_type*: string, required
*`X	Y`*	Specifies the X or Y axis of the module to be used as a reference to flip all hierarchical instances. This referenced X or Y axis is normally at the center of the module. *Data_type*: enum, required

Example

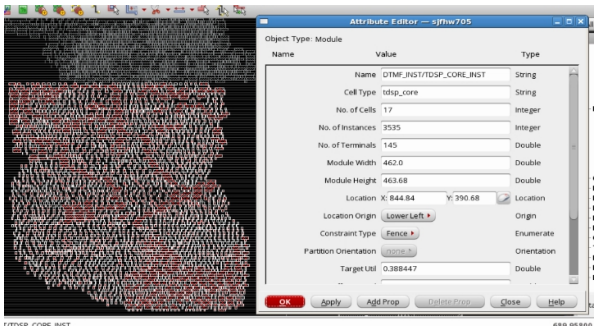
The following command flips all hierarchical instances in the module DTMF_INST/TDSP_CORE_INST from the right side of the referenced Y axis to the left and from the left side to the right, and flips all instances mirrored through it's own Y axis as well.

flipModule DTMF_INST/TDSP_CORE_INST Y

Before:



After：



Related Information

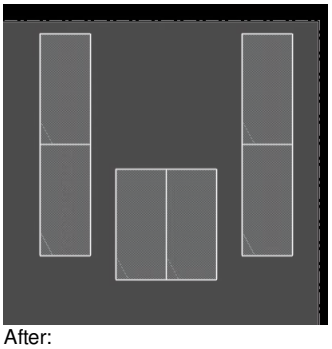
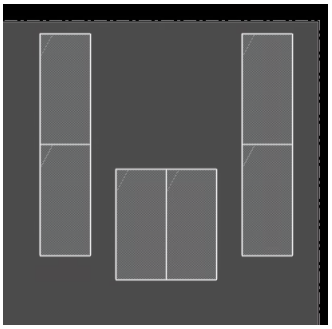
- Floorplanning the Design chapter in the *Innovus User Guide*
- Floorplan Menu chapter in the *Innovus Menu Reference*

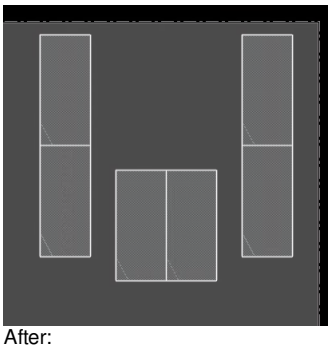
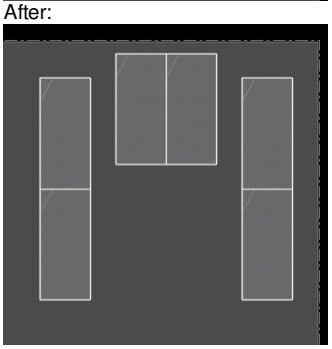
flipOrRotateObject

```
flipOrRotateObject
[-help]
[-keepRelative]
{-flip {MX | MY} | -rotate {R90 | R180 | R270}}
[-group]
[-name object]
```

Flips or rotates the selected objects. Alternatively, you can use the "*Flip/Rotate Selected Objects*" form in the Innovus GUI, to flip or rotate an instance. Use this command after importing the design.

Parameters

`-flip {MX | MY}`	
	Flips the selected object(s) through x axis or y axis. Example: `flipOrRotateObject -flip MX` Before:  After:  *Data_type*: string, required
`-help`	Outputs a brief description that includes the type and default information for each `flipOrRotateObject` parameter. For a detailed description of the command and all of its parameters, use the man command: `man flipOrRotateObject`

`-group`	Flips or rotates the bounding box which encloses all the selected objects. Example: `flipOrRotateObject -flip MX -group` Before:  After:  *Data_type*: bool, optional
`-keepRelative`	Calls the `create_relative_floorplan` command. With this option enabled, when the anchor objects are rotated, the relative objects are rotated and updated automatically. *Data_type*: bool, optional
`-name object`	Specifies the name of the object you want to flip or rotate. **Note**: The objects can be modules, groups, placement blockage, routing blockage, hard Macros, special routings, special wires, and special vias (including 45 degree). *Data_type*: string, required
`-rotate {R90|R180|R270}`	
	Rotates the selected object(s) by 90 or 180 or 270 degrees. *Data_type*: string, required

Example

The following command flips the selected object(s) through the x axis:

```
flipOrRotateObject -flip MX
```

Related Information

- `flipchip_allow_routed_bump_edit`
- `create_relative_floorplan`
- Floorplanning the Design chapter in the *Innovus User Guide*

floorPlan

```
floorPlan
[-help]
[-adjustToSite]
[-coreMarginsBy {io | die}]
[-dieSizeByIoHeight {min |max}]
[-flip {f | s | n}]
[-fplanOrigin {center | llcorner}]
[-noResize]
[-noSnapToGrid]
[-overlapSameSiteRow]
{-b {die_box io_box core_box} |
 -s {W H Left Bottom Right Top}  |
 -d {W H Left Bottom Right Top}  |
 -r {aspectRatio [rowDensity [Left Bottom Right Top]]}|
 -su {aspectRatio [stdCellDensity [Left Bottom Right Top]]}|
 -keepShape util}
[-site name | -siteOnly name]
```

Specifies the floorplan dimensions by size; or by die, I/O, or core coordinates. The `floorPlan` command initializes the floorplan and calls the `add_tracks` command to create new routing tracks. By default the `add_tracks` command calculates the optimum spacing between tracks and ignores the pitch values in the tech LEF. When you use the *Specify Floorplan* form (or specify the floorplan through the `floorPlan` command), the floorplan is resized automatically – relatively floorplan constraints are automatically derived on the fly for blocks, fixed standard cells, fixed pre-routes, and blockages.

The floorplan is linearly adjusted as follows:

- Spacing among blocks is evenly adjusted
- The size of the modules and black boxes is evenly adjusted.
- The fixed pre-routes and I/Os are automatically adjusted.

Note: The `floorPlan` command checks if the specified IO box and core box is within limits of die box and displays an error message if they are outside the die box.

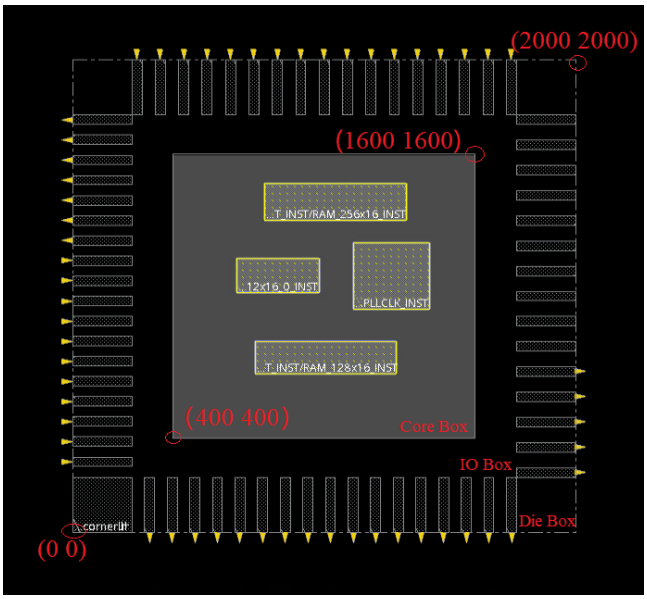
Parameters

`-adjustToSite`	Adjusts the width of the die area such that it is an integer multiple of the width of the IO site or core site. *Data_type*: bool, optional
`-b {die_box io_box core_box}`	

Specifies all three boxes of the floorplan, `die_box`, `io_box`, `core_box`.

The following command creates floorplan by specifying a core box size of 1200 x 1200,io box size of 1600 x 1600 and die box size of 2000 x 2000.

```
floorPlan -b {0 0 2000 2000 200 200 1800 1800 400 400 1600 1600}
```



Data_type: list, required

`-coreMarginsBy {io | die}`

Specifies whether the core margins are calculated using the core-to-IO boundary or the core-to-die boundary.

Default: `io`

Data_type: enum, optional

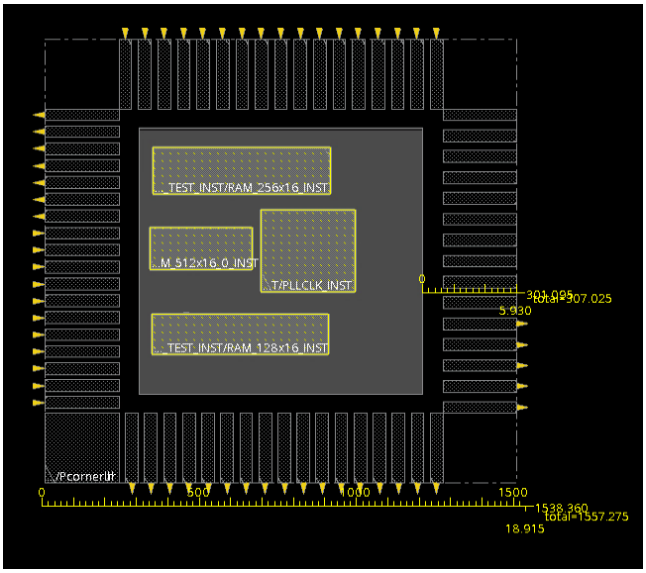
`-d {W H Left Bottom Right Top}`

Specifies the die size and the spacing, in micrometers, between the die edge, which is the margin between the outside edge of the die.

- *W* : Specifies the die's width value.
- *H* : Specifies the die's height value.
- *Left* : Specifies the margin from the outside edge of the core to the left of the I/O boundary.
- *Bottom* : Specifies the margin from the outside edge of the core to the bottom of the I/O boundary.
- *Right* : Specifies the margin from the outside edge of the core to the right of the I/O boundary.
- *Top* : Specifies the margin from the outside edge of the core to the top of the I/O boundary.

The following command creates floorplan by specifying a die box size of 1500 x 1500,and the spacing between core edge to each die edge of 300.

```
floorPlan -d {1500 1500 300 300 300 300}
```



Data_type: list, required

`-dieSizeByIoHeight {min | max}`

Specifies whether the maximum I/O height or the minimum I/O height should be used for die size calculation.

Default: `min`

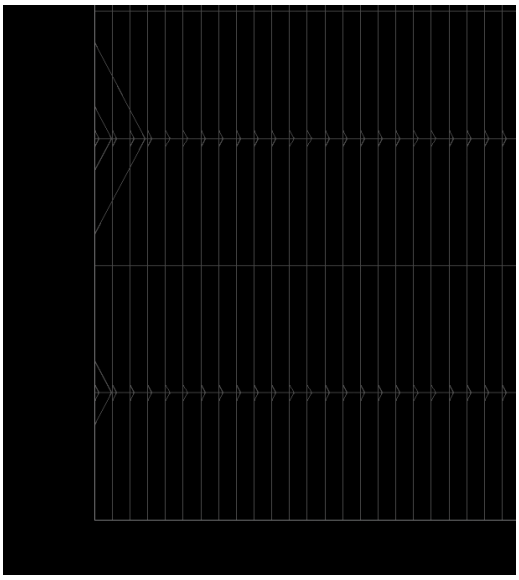
Data_type: enum, optional

`-flip {f | s | n}`

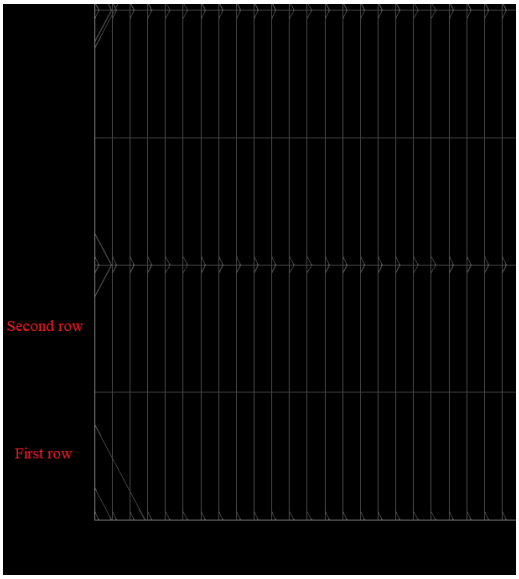
Specifies the orientation of the bottom row in the core area.

- `f`: Specifies that the first row flips from the bottom up.
- `n`: Specifies no row flipping.
- `s`: Specifies that the second row flips from the bottom up.

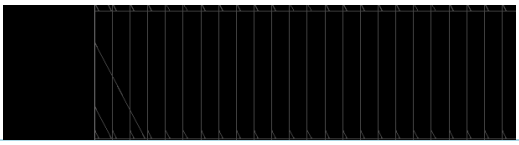
```
floorPlan -b {0 0 2000 2000 200 200 1800 1800 400 400 1600 1600} -flip f
```



```
floorPlan -b {0 0 2000 2000 200 200 1800 1800 400 400 1600 1600} -flip s
```




```
floorPlan -b {0 0 2000 2000 200 200 1800 1800 400 400 1600 1600} -flip n
```

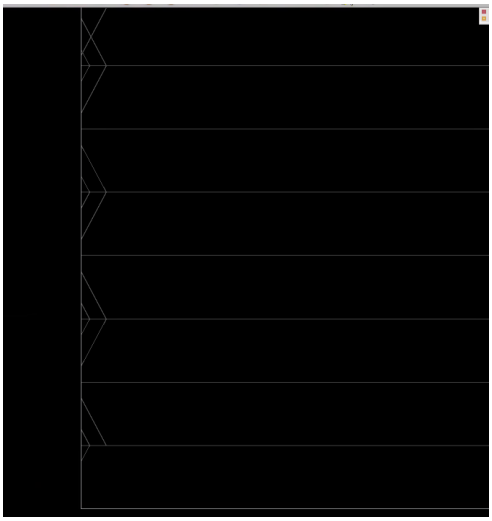
	
`-fplanOrigin {center | llcorner}`	
	Specifies whether the origin of the floorplan should be at the center or at the lower left corner. *Default*: `llcorner` (lower left corner) *Data_type:* enum, optional
`-help`	Outputs a brief description that includes the type and default information for each `floorplan` parameter. For a detailed description of the command and all of its parameters, use the man command: `man floorplan`
`-keepShape` *util*	Specifies the target utilization after resize. *Data_type:* float, required
`-noResize`	Checks if there is a need to do floorplan if everything is the same. *Data_type*: bool, optional
`-noSnapToGrid`	Specifies that the core box (or die box) boundary will not be snapped to the nearest metal pitch even if the if the specified die/ core box size is not an integer multiple of the smallest metal pitch. This prevents the input floorplan boxes from being changed. *Data_type*: bool, optional
`-overlapSameSiteRow`	

Specifies that same-site rows can overlap with one another. Single height rows must not overlap. Only double or multiple-height rows can overlap, and in sections that are multiple of single height any other overlap section size will cause the rows to be ignored.

Example: `floorPlan -s {1000 1000 300 300 300 300} -overlapSameSiteRow`

Before:

After:



Data_type: bool, optional

`-r {aspectRatio [rowDensity [Left Bottom Right Top]]}`

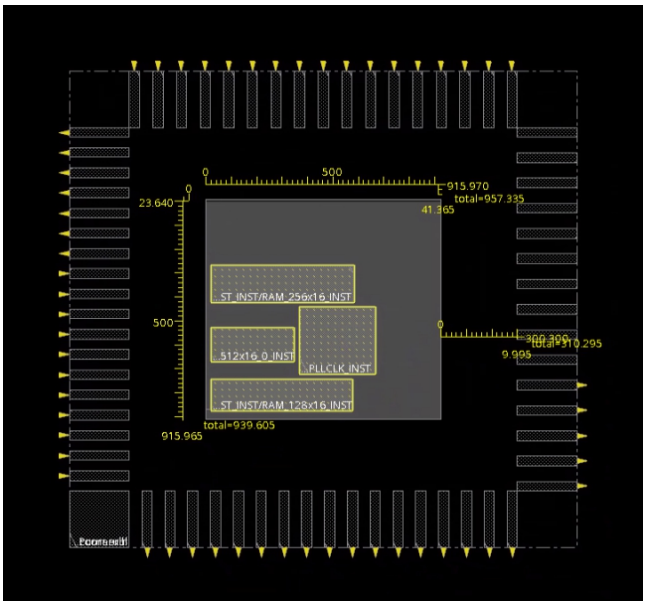
Specifies the chip's core dimensions as the ratio of the height divided by the width. If a value of 1.0 is used, a square chip is defined. A value of 2.0 will define a
rectangular chip with height dimension that is twice the width dimension.

- *aspectRatio*: Specifies the aspect ratio value.
- *rowDensity:* Specifies a row density value.

  ```
  rowDensity = (std area + block/macro area) / core area
  ```
- *Left*: Specifies the margin from the outside edge of the core box to the left.
- *Bottom*: Specifies the margin from the outside edge of the core box to the bottom.
- *Right*: Specifies the margin from the outside edge of the core box to the right.
- *Top*: Specifies the margin from the outside edge of the core box to the top.

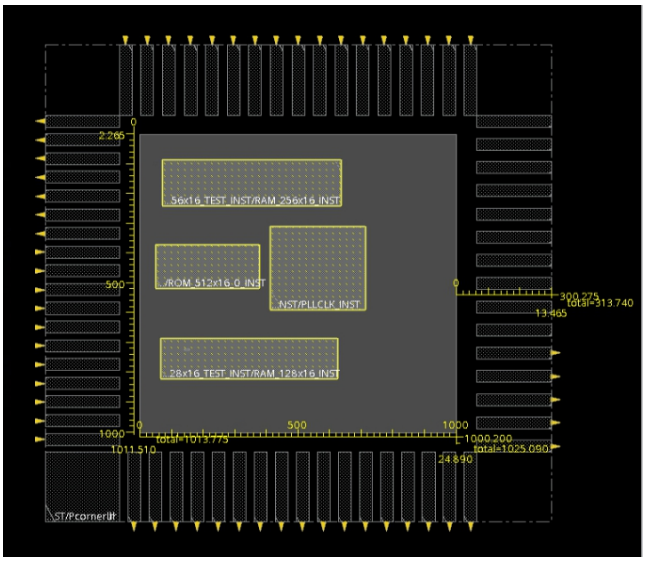
The following command creates floorplan by specifying the core dimensions aspect ratio of 1.0,row density of 0.5,and the spacing between core edge to each io box edge of 300.

```
floorPlan -site tsm3site -r {1.0 0.5 300 300 300 300} -coreMarginsBy io
```



Data_type: list, required

-s {W H Left Bottom Right Top}

	Specifies the core size and the spacing, in micrometers, between the core edge, which is the margin between the outside edge of the core (head) box. • *W*: Specifies the core box's width value. • *H*: Specifies the core box's height value. • *Left*: Specifies the margin from the outside edge of the core box to the left. • *Bottom*: Specifies the margin from the outside edge of the core box to the bottom. • *Right*: Specifies the margin from the outside edge of the core box to the right. • *Top*: Specifies the margin from the outside edge of the core box to the top. The following command creates floorplan by specifying a core size of 1000 x 1000, and the spacing between core edge to each die edge of 300. `floorPlan -site tsm3site -s {1000 1000 300 300 300 300} -coreMarginsBy die`  *Data_type*: list, required
`-site` *name*	Specifies a core row site. *Data_type*: (site), optional
`-siteOnly` *name*	Creates rows only for the specified site. *Data_type*: (site), optional
`-su {`*aspectRatio* `[`*stdCellDensity* `[`*Left Bottom Right Top*`]]}`	

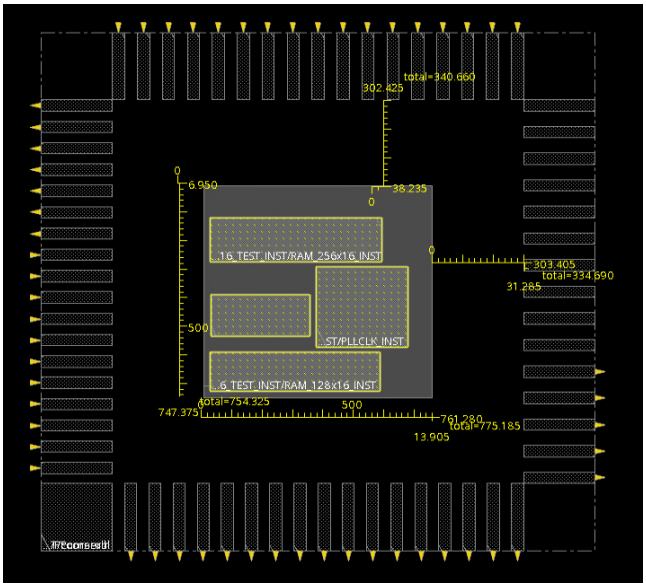
Determines the core and module sizes by standard cell density.

- *aspectRatio*: Specifies the aspect ratio value.
- *stdCellDensity*: Specifies a standard cell density value.

  ```
  stdCellDensity =  std cell area/(core area - block/macro area)
  ```
- *Left*: Specifies the margin from the outside edge of the core box to the left.
- *Bottom*: Specifies the margin from the outside edge of the core box to the bottom.
- *Right*: Specifies the margin from the outside edge of the core box to the right.
- *Top*: Specifies the margin from the outside edge of the core box to the top.

The following command creates floorplan by specifying the core dimensions aspect ratio of 1.0,standard cell density of 0.5,and the spacing between core edge to each io box edge of 300.

```
floorPlan -su {1.0 0.5 300 300 300 300}
```



Data_type: list, required

Examples

- The following command creates a new floorplan containing all the sites in the original floorplan

  ```
  floorPlan -s {...}
  ```
- The following command automatically shrinks or expands the block-level floorplan boundary, proportionally, in the X and Y directions, trying to meet the specified target core utilization 75.

  ```
  floorPlan -keepShape 75
  ```

Related Information

- `add_tracks`
- `checkFPlan`
- `floorPlan`
- `getFPlanMode`
- `loadFPlan`
- `initCoreRow`
- `saveFPlan`
- *Specify Floorplan* form in the *Innovus Menu Reference*.
- Floorplanning the Design chapter in the *Innovus User Guide*.

generate_fence

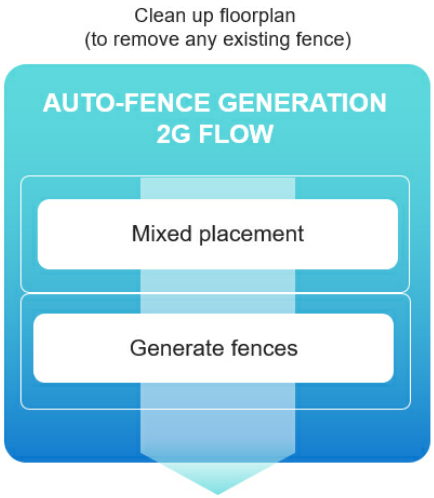
```
generate_fence
[-help]
[-flow {2G}]
[-for_partition]
[-rectangle_shape]
[-target_util float]
[ [-hInst {hInst(s)}] | [-module {module(s)}] | [-inst_group {instGroup(s)}] ]
```

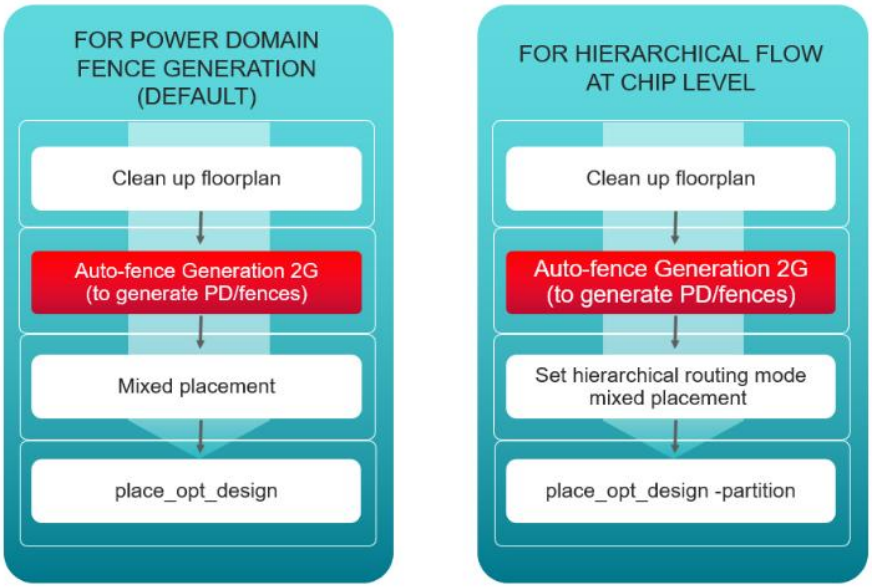
Automatically draws partition fences that enclose all their children FlexModel guides. It helps to improve the usability of the prototyping flow. The recommended use model for automatically generating fences is as follows:

1. Run `proto_design` to place FlexModel guides/fences.
2. Manually adjust/move FlexModels to shorten timing paths.
3. Run the following commands to analyze the floorplan and check timing. Repeat step 2 and 3 if needed
   a. `place_opt_design`
   b. `earlyGlobalRoute`
   c. `timeDesign -proto`
   d. `load_timing_debug_report -proto`
4. Select FlexModels for partitions and define partitions.
5. Run `generate_fence` to create fences that enclose all their children FlexModel guides.

Note: If no option is specified then the command draws fences for all existing defined partitions.

Parameters

`-flow {2G}`	Enables the Auto-Fence Generation 2G flow to draw the fence shapes automatically. When this flow is enabled, the mixed-placement capability is leveraged to give optimal module placement distribution. A fence is generated for each target module after considering target utilization, boundary gaps and I/Os. The Auto-Fence Generation 2G flow involves the following key steps:  • Running flat global mixed placement • Creating the fence based on the module cells placement distribution and control the utilization. • Polishing the fence so not to block any channel or I/O access *Data_type*: float, optional

`-for_partition`	When specified, it enables the partition fence generation capability. When this option is not specified, the power domain fence generation capability is enabled by default.  *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `generate_fence` parameter. For a detailed description of the command and all of its parameters, use the man command: `man generate_fence`
`-hInst {hInst(s)}`	Specifies the list of hierarchical instance names to draw fences. This option can be used if you only want to create a fence for a specific master or clone. You can use wildcards. **Note:** It can not be used for both master and clones. *Data_type*: string, optional
`-inst_group {instGroup(s)}`	Specifies list of instance group names to draw fences. You can use wildcards. *Data_type*: string, optional
`-module {module(s)}`	Specifies list of partition modules to draw fences. This option should be used for master/clone partitions. *Data_type*: string, optional
`-rectangle_shape`	When specified, only rectangle shaped fences are generated. *Data_type*: bool, optional
`-target_util string`	Specifies the fence target utilization value. *Default*: Core Utilization *Min*: 1e-07 *Max*: 1 *Data_type*: float, optional

Example

- The following command enables the auto-fence generation 2G flow and creates fences for the $PowerDomainNameList group.
  ```
  generate_fence -flow 2G -inst_group $PowerDomainNameList
  ```
- The following command enables the auto-fence generation 2G flow and creates fences for specified hierarchical instance(s).
  ```
  generate_fence -flow 2G -hInst $hInstNameList -for_partition
  ```

Related Information

- `place_opt_design`
- `earlyGlobalRoute`
- `load_timing_debug_report`
- `timeDesign`
- Partitioning the Design chapter in the *Innovus User Guide*
- Floorplanning the Design chapter in the *Innovus User Guide*

get_macro_place_constraint

```
get_macro_place_constraint
[-help]
[-macro_corner_keepout]
[[-array array_name] |
[-inst string  -orientation ] |
[-cell string  -cell_obs ] |
[-track_adjustment ] |
[-max_io_pin_group_keep_out ] |
[-forbidden_space_to_core ] |
[-forbidden_space_to_macro ] |
[-min_space_to_core ] |
[-min_space_to_macro ] |
[-parallel_run_length ] |
[-horizontal_stacking ] |
[-vertical_stacking ] |
[-honor_strict_spacing_constraint ] |
[-avoid_abut_macro_edge_with_pins ] |
[-same_length_site ] |
[-pg_resource_model ] |
[-non_default ] |
[-halo_sharing ]]
```

Gets the current value of the macro constraints that are honored by the macro placer. To define the constraints of macros, use the `set_macro_place_constraint` command.

Note: The report displayed when the -array, -cell/-cell_obs, or -inst/-orientation parameters are used can result in long lines on the screen that are supported by ellipses (…) indicating that more information about the macro constraint follows. If you want to get all the information together, Cadence recommends using the `write_macro_place_constraint` command to save all the information into a file.

Parameters

`-array` *array_name*	Specifies the macro array name. *Data_type*: string, optional
`-avoid_abut_macro_edge_with_pins`	When specified, it returns if abutting macro edges with pins must be avoided even if the macro distance is within the forbidden space setting. *Data_type*: bool, optional
`-cell_obs` *string*	When specified, it returns the cell obstruction value. *Data_type*: bool, optional
`-cell` *string*	Specifies constraints for all instances of the specified cells. *Data_type*: string, optional
`-forbidden_space_to_core`	When specified, it returns the forbidden space to core value. *Data_type*: bool, optional
`-forbidden_space_to_macro`	When specified, it returns the forbidden space to macro value. *Data_type*: bool, optional
`-halo_sharing`	When specified, it returns the halo sharing mode. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `get_macro_place_constraint` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_macro_place_constraint`

`-honor_strict_spacing_constraint`	When specified, it returns if the forbidden spacing and minimum spacing constraints must be honored strictly. *Data_type*: bool, optional
`-horizontal_stacking`	When specified, it returns the horizontal stacking constraint. *Data_type*: bool, optional
`-inst` *string*	Specifies the macros on which the constraints are set. *Data_type*: string, optional
`-macro_corner_keepout`	When specified, it returns the macro corner keepout value. *Data_type*: bool, optional
`-max_io_pin_group_keep_out`	When specified, it returns the max height of the I/O pin blockages. *Data_type*: bool, optional
`-min_space_to_core`	When specified, it returns the minimum space to core value. *Data_type*: bool, optional
`-min_space_to_macro`	When specified, it returns the minimum space to macro value. *Data_type*: bool, optional
`-non_default`	When specified, it displays the current settings which are not the default settings. *Data_type*: bool, optional
`-orientation`	When specified, it returns the valid orientations. *Data_type*: bool, optional
`-parallel_run_length`	When specified, it returns the parallel run length value. *Data_type*: bool, optional
`-pg_resource_model`	When specified, it returns the pg resource model details. *Data_type*: bool, optional
`-same_length_site`	When specified, it returns the the minimum number of sites between the vertical edge of two macros. *Data_type*: bool, optional
`-track_adjustment`	When specified, it returns the track adjustment value. *Data_type*: bool, optional
`-vertical_stacking`	When specified, it returns the vertical stacking constraint. *Data_type*: bool, optional

Example

- The following command gets the current value of the macro constraints that are honored by the macro placer.:
  ```
  get_macro_place_constraint
  ```

Related Information

- `check_macro_place_constraint`
- `create_pg_model_for_macro_place`
- `reset_macro_place_constraint`
- `set_macro_place_constraint`
- `write_macro_place_constraint`
- Floorplanning the Design chapter in the *Innovus User Guide*
- Using the Mixed Placer chapter in the *Innovus User Guide*

get_snap_grid_info

```
get_snap_grid_info
[-help]
[-origin]
[-pitch_x]
[-pitch_y]
-type {manufacturing | inst | placement | userdefine | layertrack | finfetInst |finfetmanufacturing | finfetplacement}
```

Gets information on the current snap grid settings.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `get_snap_grid_info` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_snap_grid_info`
`-origin`	Specify to get the original point of the grid. *Data_type*: bool, optional
`-pitch_x`	Specify to get the X-direction pitch of the grid. *Data_type*: bool, optional
`-pitch_y`	Specify to get the Y-direction pitch of the grid. *Data_type*: bool, optional
`-type {manufacturing | inst | placement | userdefine | layertrack | finfetInst |finfetmanufacturing | finfetplacement}`	
	Specify the grid type for which the information is required. The grid types can be: • `manufacturing`: Manufacture Grid • `inst`: Instance Grid • `placement`: Placement Grid • `userDefine`: User-defined Grid • `LayerTrack`: LayerTrack Grid • `finfetManufacturing`: FinFET Manufacture Grid • `finfetInst`: FinFET Instance Grid • `finfetPlacement`: FinFET Placement Grid *Data_type*: enum, required

Examples

The following commands displays the snap grid settings for the placement grid:
get_snap_grid_info -type placement

For example:

```
{origin {0.0000000000,0.0000000000}} {pitch_x 0.6600000000} {pitch_y 0.5600000000}
```

Related Information

- `setSnapGrid`
- `setFPlanMode`
- Floorplanning the Design chapter in the *Innovus User Guide*

get_trace_obj_connectivity_mode

```
get_trace_obj_connectivity_mode
[-help]
[-macro_pins]
[-max_fanin_fanout]
[-register_inputs]
[-register_outputs]
[-quiet]
```

Returns information about the specified `set_trace_obj_connectivity_mode` parameter in the Innovus log file and the console. The following information is returned:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

Note: If you do not specify a parameter, the software displays values for all of the `set_trace_obj_connectivity_mode` parameters.

You can use this command after importing the design.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_trace_obj_connectivity_mode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man` get_trace_obj_connectivity_mode
`parameter_names`	Displays information for the specified parameters. You can specify one or more parameters. See `set_trace_obj_connectivity_mode` for descriptions of the parameters you can specify.
`-quiet`	Displays the current settings for the specified parameters in Tcl list format only.

Example

The following command displays the values for all of the `set_trace_obj_connectivity_mode` parameters:

```
get_trace_obj_connectivity_mode
```

Related Information

- `trace_obj_connectivity`
- `set_trace_obj_connectivity_mode`
- `display_obj_connectivity`
- `all_fanin`
- `all_fanout`
- Floorplanning the Design chapter in the *Innovus User Guide*
- Floorplan Menu chapter in the *Innovus Menu Reference*

getDrawView

```
getDrawView
[-help]
```

Returns the design view that was set with the `setDrawView` command. You can use this command after `setDrawView`.

Parameters

`-help`	Outputs a brief description of the `getDrawView` command. For a detailed description of the command, use the `man` command: `man getDrawView`

Related Information

- `setDrawView`
- Floorplanning the Design chapter in the *Innovus User Guide*

getFinishFPlanMode

```
  getFinishFPlanMode
[-help]
[-abuttedEdgeAsNonRowArea]
[-activeObj]
[-direction]
[-drcRegionObj]
[-override]
[-quiet]
```

Returns information about the specified `setFinishFPlanMode` parameter in the Innovus log file and the console. The following information is returned:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

Note: If you do not specify a parameter, the software displays values for all of the `setFinishFPlanMode` parameters.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `getFinishFPlanMode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man getFinishFPlanMode`.
`-parameter_names`	Displays information for the specified parameters. You can specify one or more parameters. See `setFinishFPlanMode` for descriptions of the parameters you can specify.
`-quiet`	Displays the current settings for the specified parameters in Tcl list format only.

Example

- The following command returns information about the `setFinishFPlanMode` parameters in the Innovus log file and the console:
  `getFinishFPlanMode`

Related Information

- `finishFloorplan`
- `setFinishFPlanMode`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Finishing the Floorplan

getFPlanMode

```
getFPlanMode
[-help]
[-autoSyncMasterClone]
[-cellsForExtraSites]
[-checkTypes]
[-cutOffPlaceBlockageOutsideDie]
[-cutOffRouteBlockageOutsideDie]
[-defaultBlockageNamePrefix]
[-defaultPowerDomainSite]
[-defaultRowPatternSite]
[-defaultTechSite]
[-enableRectilinearDesign]
[-extraSites]
[-firstRowSiteIndex]
[-includeIoWhenInitArea]
[-initAllCompatibleCoreSiteRows]
[-keepRowsWhenMovingPowerDomain]
[-lastRowSiteIndex]
[-maxIoHeight]
[-minimumSites]
[-move_macros_with_constraint]
[-move_preplaced_std_cell_only]
[-move_std_cell_with_constraint]
[-narrowChannelThreshold]
[-no_cut_row]
[-powerRailLayer]
[-rowHeightIncrementCornerToCorner]
[-rowHeightIncrementIncornerToCorner]
[-rowHeightIncrementIncornerToIncorner]
[-rowHeightMultiple]
[-rowSiteHeight]
[-rowSiteWidth]
[-snap_all_corners_to_grid]
[-snapBlockGrid]
[-snapConstraintGrid]
[-snapCoreGrid]
[-snapDieGrid]
[-snapIoGrid]
[-snapPlaceBlockageGrid]
[-snapPlaceBlockageType]
[-user_define_grid]
[-quiet]
[-nonDefault]
```

Returns information about the specified `setFPlanMode` parameter in the Innovus log file and the console. The following information is returned:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

You can use this command after importing the design.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `getFPlanMode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man getFPlanMode`
`parameters`	Returns information for the specified parameters. You can specify one or more parameters. See `setFPlanMode` for descriptions of the parameters you can specify.
`-quiet`	Displays the current settings for the specified parameters in Tcl list format only.

Related Information

- `setFPlanMode`
- `floorPlan`
- Floorplanning the Design chapter in the *Innovus User Guide*

getIoFlowFlag

```
getIoFlowFlag
[-help]
```

Gets the current I/O row flow flag setting. A value of "0" means traditional I/O flow where as a value of 1 means I/O row flow. To set the I/O row flow flag, use the `setIoFlowFlag` command.

Parameters

`-help`	Outputs a brief description of the `getIoFlowFlag` command. For a detailed description of the command, use the man command: `man getIoFlowFlag`

Example

- The following command displays the current I/O row flow flag settings:
  ```
  getIoFlowFlag
  ```

Related Information

- `createIoRow`
- `setIoFlowFlag`
- Floorplanning the Design chapter in the *Innovus User Guide*
- Floorplan Menu chapter in the *Innovus Menu Reference*
  - Create I/O Row

getNetWeight

```
getNetWeight
[-help]
{-all | netName ... }
```

Retrieves the net weight values for the specified nets. You can use this command after running the `specifyNetWeight` command.

Parameters

`-all`	Retrieves the net weight value for all nets. *Data_type*: bool, required
`-help`	Outputs a brief description that includes the type and default information for each `getNetWeight` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man getNetWeight`
netName	Retrieves the net weight value for all specified nets. *Data_type*: string, required

Related Information

- `specifyNetWeight`
- `deleteNetWeight`
- Floorplanning the Design chapter in the *Innovus User Guide*

getObjFPlanBoxList

```
getObjFPlanBoxList
[-help]
type
name
```

Retrieves a box list of the specified rectilinear object that was created earlier with the `setObjFPlanBoxList` command. This command can also be used to retrieve a box list of a rectilinear block instance from the PEF overlap layer. A box list comprises two or more boxes. You can use this command after setting the rectilinear shape of a block instance with the `setObjFPlanBoxList` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `getObjFPlanBoxList` parameter. For a detailed description of the command and all of its parameters, use the man command: `man getObjFPlanBoxList`
name	Specifies the name of the object for object type. *Data_type*: string, required
type	Specifies the type of object (`Cell | Group | Instance | Layershape | Module | Net | Pin`) for name. *Data_type*: string, required

Example

The following command retrieves the rectilinear boundary for module xyz.

```
getObjFPlanBoxList Module xyz
```

Related Information

- `setObjFPlanBoxList`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Defining the Bounding Box

getObjFPlanPolygon

```
getObjFPlanPolygon
[-help]
type
name
```

Retrieves polygon coordinates of the specified rectilinear object that was created earlier with the `setObjFPlanPolygon` command. You can use this command after setting the polygon coordinates of a rectilinear block with the `setObjFPlanPolygon` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `getObjFPlanPolygon` parameter. For a detailed description of the command and all of its parameters, use the man command: `man getObjFPlanPolygon`
name	Specifies the name of the object for type. *Data_type*: string, required
type	Specifies the type of object for objectName. This can be: • Cell • Group • Instance • Layer shape • Module *Data_type*: string, required

Example

The following command retrieves the polygon coordinates of module xyz.

```
getObjFPlanPolygon module xyz
```

Related Information

- `setObjFPlanPolygon`
- Floorplanning the Design chapter in the *Innovus User Guide*

getRailPrototypeMode

```
getRailPrototypeMode
[-help]
[-domain]
[-railModel]
[-totalPower]
[-quiet]
```

Returns information about the specified `setRailPrototypeMode` parameter in the Innovus log file and the console. The following information is returned:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

Note: If you do not specify a parameter, the software displays values for all of the `setRailPrototypeMode` parameters.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `getRailPrototypeMode` parameter. For a detailed description of the command and all of its parameters, use the man command `man getRailPrototypeMode`.
`-parameter_names`	Displays information for the specified parameters. You can specify one or more parameters. See `setRailPrototypeMode` for descriptions of the parameters you can specify.
`-quiet`	Displays the current settings for the specified parameters in Tcl list format only.

Related Information

- `setRailPrototypeMode`
- Floorplanning the Design chapter in the *Innovus User Guide*

getResizeFPlanMode

```
getResizeFPlanMode
[-help]
[-congAware]
[-honorHalo]
[-ioFix]
[-ioMoveWithEdge]
[-ioProportional]
[-maintainResourceRatioAfterResize]
[-proportional]
[-shiftBased]
[-shrinkFence]
[-snapToTrack]
[-quiet]
```

Returns information about the specified `setResizeFPlanMode` parameter in the Innovus log file and the console. The following information is returned:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

Note: If you do not specify a parameter, the software displays values for all of the `setResizeFPlanMode` parameters.

You can use this command after importing the design.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `getResizeFPlanMode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man getResizeFPlanMode`
parameter_names	
	Displays information for the specified parameters. You can specify one or more parameters. See `setResizeFPlanMode` for descriptions of the parameters you can specify.
`-quiet`	Displays the current settings for the specified parameters in Tcl list format only.

Related Information

- `resizeFloorplan`
- `setResizeLine`
- `setResizeFPlanMode`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Resizing the Floorplan

initCoreRow

```
initCoreRow
[-help]
[-powerDomain string]
```

Regenerates rows for the core area and the power domains in a design. The rows are based on the current row parameters instead of using the `floorPlan` command. You can use this command after importing the design.

Note: In a common flow, Innovus goes over all technical sites used in the netlist and creates rows for these sites. It chooses the shortest tech site in the design (with the smallest height and width) as the default-technical-site. All other used tech-sites with integer-multiple height/width are considered as compatible.

Parameters

`-help`	Outputs a brief description of the `initCoreRow` command. For a detailed description of the command, use the `man` command: `man initCoreRow`
`-powerDomain string`	Specifies the power domain for which rows are to be created. *Data_type*: string, optional

Related Information

- `floorPlan`
- Floorplanning the Design chapter in the *Innovus User Guide*

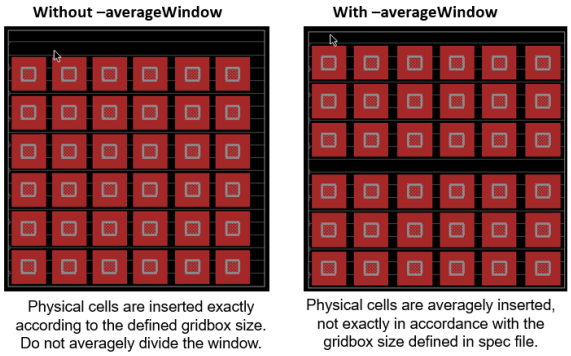
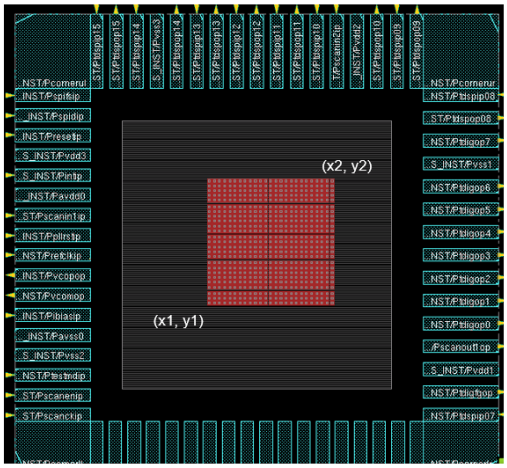
insert_physical_cell

```
insert_physical_cell
[-help]
[-cell string]
[-genTemplateOnly string]
[-inputFile filename]
[-insertRowAreaBoundary]
[-averageWindow | { [-insertNearBoundary ] [-box {x1 y1 x2 y2}] [-minimalNumberOfTCD integer]}]
[-verifyWindow {x y}[ [-verifyOnly ][-xStep float] [-yStep float] ] ]
```

Provides a flexible way to insert and check physical cells. It can be used to assign a specific area for physical insertion, not only the die area.

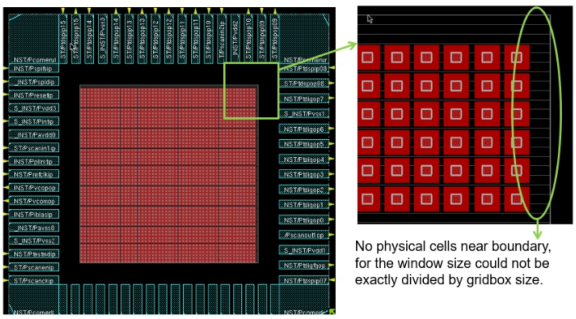
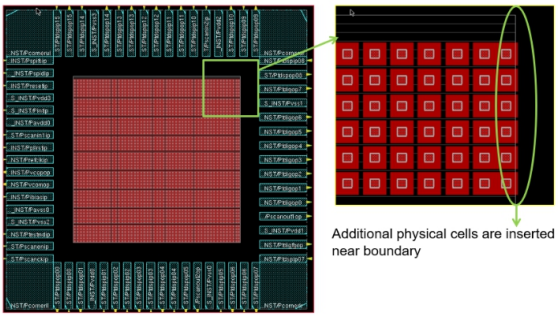
Note: The `insert_physical_cell` command is suitable for inserting physical cells as a pattern. The class of these cells must be block and not core. The `insert_physical_cell` command is not when the class of the physical cells is core and there is no requirement for inserting cells in a pattern.

Parameters

`-averageWindow`	When specified, it changes the window calculation approach to average. The size of windows for inserting cell is the average of the core box. The insertion will be done not exactly in accordance with the size defined in spec file.  Physical cells are inserted exactly according to the defined gridbox size. Do not averagely divide the window. Physical cells are averagely inserted, not exactly in accordance with the gridbox size defined in spec file. *Data_type*: bool, optional
`-box {x1 y1 x2 y2}`	Specify the bounding box of the physical cell in `x1 y1 x2 y2`. For example, the following command inserts the cell DTCDCELL in the (*x1 y1 x2 y2)* box. `insert_physical_cell -cell DTCDCELL -inputFile SPECFILE -box {x1 y1 x2 y2}`  *Data_type*: box, optional
`-cell string`	Specify the physical cell name. *Data_type*: string, optional

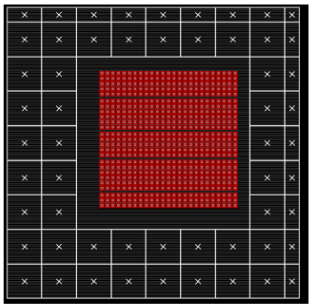
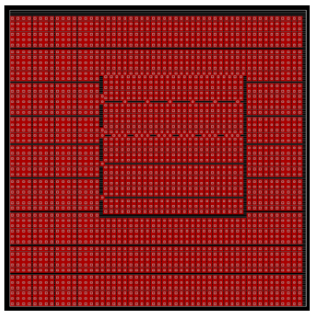
`-genTemplateOnly` *string*	Generates a template file to guide how to define spec file for DTCD, ICOVL and LinerCell. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `insert_physical_cell` parameter. For a detailed description of the command and all of its parameters, use the man command: `man insert_physical_cell`

`-inputFile` *filename*	Specify the spec file for physical cells.
	Note: The inserted physical cells must be specified in a spec file. This file specifies the information related to the dimensions and spacing physical cells. The tool will insert physical cells according to these specifications.
	Here is an example of a spec file:
	`**********`
	`DTCD Cell cell_name`
	`WinSizeX  # x dimension of DTCD insertion grid box`
	`WinSizeY  # y dimension of DTCD insertion grid box`
	`MacroSpacing  # spacing between DTCD cell and macro`
	`DTCDSpacing  # spacing between DTCD cells`
	`PlaceBlockSizeX  # x dimension of padded placement blockage around DTCD cell`
	`PlaceBlockSizeY  # y dimension of padded placement blockage around DTCD cell`
	`RouteBlockSizeX  # x dimension of padded routing blockage around DTCD cell`
	`RouteBlockSizeY  # y dimension of padded routing blockage around DTCD cell`
	`End`
	`**********`
	Data_type: string, optional

`-insertNearBoundary`	Inserts physical cells near boundary.
	Note: When the window size is not exactly divided by gridbox size, you may not get physical cells near boundary.
	For example:
	`insert_physical_cell -cell DTCDCELL -inputFile SPECFILE`
	 
	
	`insert_physical_cell -cell DTCDcell -inputFile SPECFILE -insertNearBoundary`
	 
	
	Data_type: bool, optional

`-insertRowAreaBoundary`	Inserts LinerCell around row area boundary. *Data_type*: bool, optional

`-minimalNumberOfTCD` *integer*	Specifies the the minimal number of DTCD that must be inserted. If the current windows size definition (*WinSizeX* and *WinSizeY*) cannot fit this requirement, the window size is shrunk automatically. *Data_type*: int, optional

`-verifyOnly`	Only verifies the physical cells to inserted. It does not actually insert them.
	Note: The `-verifyWindow`, `-xStep`, and `-yStep` parameters are used to specify the size of the window to verify.
	For example:
	`insert_physical_cell -cell DTCDCELL -inputFile SPECFILE -verifyWindow 100 100 -xStep 100 -yStep 100 -verifyOnly`
	 
	Only verify the physical cell in gridbox, do not insert.
	`insert_physical_cell -cell DTCDCELL -inputFile SPECFILE -verifyWindow 100 100 -xStep 100 -yStep 100`
	 
	Verify and insert the physical cells
	Data_type: bool, optional

`-verifyWindow {x y}`	Specifies the size of window for verifying the physical cells to be inserted. *Data_type*: point, optional
`-xStep` *float*	Specifies the x step of the verify window. *Data_type*: float, optional
`-yStep` *float*	Specifies the y step of the verify window. *Data_type*: float, optional

Example

- The following command inserts the cell DTCDCELL in the *(x1 y1 x2 y2)* box based on the spec specified in the SPECFILE.
  `insert_physical_cell -cell DTCDCELL -inputFile SPECFILE -box {x1 y1 x2 y2}`
- The following command only verifies the cell DTCDCELL in the grid box. It does not insert the cell.
  `insert_physical_cell -cell DTCDCELL -inputFile SPECFILE -verifyWindow 100 100 -xStep 100 -yStep 100 -verifyOnly`

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

legalizeFPlan

```
legalizeFPlan
[-help]
[-checkOri]
[-checkSite]
```

Legalizes partition locations according to standard cell row orientations and specific design constraints. You can use this command after specifying the floorplan.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `legalizeFPlan` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man legalizeFPlan`
`-checkOri`	Specifies that all partitions snap to the closest `R0` row on the grid. For this rule, the Innovus software assumes that standard cell rows start with an `R0` orientation and continue in an `R0`-to-`MX` pattern. *Data_type*: bool, optional
`-checkSite`	Specifies that partitions be placed on grids so that they are legal for all design constraints. *Data_type*: bool, optional

Example

The following command legalizes all the partition locations such that they are snapped to the closest `R0` rows:

```
legalizeFPlan -checkOri
```

Related Information

- `checkFPlan`
- Floorplanning the Design chapter in the *Innovus User Guide*

loadFPlan

```
loadFPlan
file
[-help]
[-noEqualizePtnHInst]
[-objType {macro | pin | bndry | special_route | pin_constraint}]
[-xml]
```

Loads a floorplan file. You can use this command after importing the design.

Note:

- When you run the `loadFPlan` command, the Innovus software removes any existing floorplan information and reads in the new information. The `loadFPlan` command honors the blackbox min/max aspect ratio information in the floorplan information file.
- Blocks and instances loaded with this command are set as preplaced.
- The `loadFPlan` command loads the user-specified seed information by default.
- The `loadFPlan` command restores the information related to blockages, tracks, and pin shapes from the *.fp file.

Parameters

file	Specifies the name of the floorplan file that was saved. *Data_type*: string, required
-help	Outputs a brief description that includes type and default information for each `loadFPlan` parameter. For a detailed description of the command and all of its parameters, use the man command: `man loadFPlan.`
-noEqualizePtnHInst	Disables the snapping capability of the clone partitions to the power grid, such that clones do not necessarily have the same row structure and pattern as their master. *Data_type*: bool, optional
-objType {macro | pin | bndry | special_route | pin_constraint}	
	Specifies the objects section in the floorplan file that will be updated. The sections can be macro, pin, bndry, special_route, or pin_constraint. **Note**: The `loadFPlan` command supports routing blockages on master slice layers. *Data_type*: string, optional
-xml	Reads the floorplan information from the XML format floorplan file. *Data_type*: bool, optional

Example

- The following command loads the floorplan file myFPlan.fp, which was saved in an earlier session:
  ```
  loadFPlan myFPlan.fp
  ```
- The following command loads only the pin constraints data in the present design with other floorplan objects remaining unchanged.
  ```
  loadFPlan myFPlan.fp -objType {pin_constraint}
  ```

Related Information

- saveFPlan
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Module Constraint Types
  - Viewing the Floorplan

loadIoFile

```
loadIoFile
[-help]
file_name
[-noAdjustDieSize]
[-specifiedIosOnly]
[-ECO [-padsOnly | -bumpsOnly]]
```

Loads an I/O assignment file. You can use this command after importing the design.

Notes:

- By default, the `loadIoFile` command will automatically adjust the die size to accommodate all the IOs. If this command is used after floorplanning, it would change the die size defined in the floorplan file. If you do not want the die size to be automatically adjusted, use the `-noAdjustDieSize` parameter.
- During I/O inst placement, the *space* value determines the distance between the current inst and the previous inst. Earlier, while honoring `iopad` spacing, the `loadIoFile` command considers the previous inst to be the inst that appears before the current inst when insts are placed according the io_order specified in the I/O assignment file .

Parameters

`-bumpsOnly`	Loads only bump ECO operations in specified IO file. *Data_type*: bool, optional
`-ECO`	Allows I/O ECO change by loading a new I/O file. In a chip-package co-design situation, when I/O Planner (IOP) utility in SiP is used for modifying the I/O and bump information, the modified information needs to be passed to Innovus. When `-ECO` is specified, `loadIoFile` overwrites the existing setting for I/Os and bumps with the information from the new I/O file. *Data_type*: bool, optional
file_name	Specifies the name of the I/O assignment file. *Data_type*: string, required
`-help`	Outputs a brief description that includes the type and default information for each `loadIoFile` parameter.For a detailed description of the command and all of its parameters, use the `man` command: `man loadIoFile`
`-noAdjustDieSize`	Specifies that the die size should not be adjusted automatically. *Data_type*: bool, optional
`-padsOnly`	Loads only IO pad ECO operations in the specified IO file. *Data_type*: bool, optional
`-specifiedIosOnly`	Loads only the IO in the specified IO file. This parameter enables you to place IO cells listed in the I/O assignment file without deleting or moving the IO cells not listed in I/O assignment file. *Data_type*: bool, optional

Example

The following command loads the I/O assignment file `myPinFile.io`:

```
loadIoFile myPinFile.io
```

Related Information

- `saveIoFile`
- Floorplanning the Design chapter in the *Innovus User Guide*
- Data Preparation chapter in the *Innovus User Guide*
  - Generating the I/O Assignment File

modify_ndr

```
modify_ndr
[-help]
[-add_via {via_name1 via_name2 ...}]
[-generate_via]
[-hard_spacing {0|1}]
[-min_cut {layer1[:layer2] min_cut ...}]
-name ruleName
[-spacing {layer1[:layer2] spacing ...}]
[-use_via_cut_class string]
[-via {via_name1 via_name2 ...}]
[-width {layer1[:layer2] width ...}]
```

Modifies a non-default rule created by the add_ndr command, or coming in from DEF or the OpenAccess design data.

Note: Any NONDEFAULTRULE defined in LEF or the OA techfile cannot be modified by this command. You must modify the LEF or OA techfile directly if you need to change them.

This command should normally be used before any routing exists in the design. If there is any existing routing using this NDR, then the changes may cause new DRC violations. In some cases ecoRoute may be able to repair the routing (for example just changing the vias can often be repaired, and just adding new via choices should not need any repair), but in many cases you will need to reroute the entire design (e.g. when changing the width or spacing) to get good results.

If `setGenerateViaMode -auto` is `true`, then vias are automatically generated for the modified NDR based on the `setGenerateViaMode` settings. See the `setGenerateViaMode` command for more details.

Parameters

`-help`	Prints a brief description that includes type and default information for each `modify_ndr` parameter. For a detailed description of the command and all of its parameters, use the man command: `man modify_ndr`
`-add_via {`*`via_name1 via_name2 ...`*`}`	
	Specifies a list of vias to add to the existing NDR via list. You can use this parameter to add DFM vias or other custom vias.
`-generate_via`	Auto-generates vias with the generateVias command. This is intended for debugging, and not used generally as it only adds vias for the current session. If you save, and later restore the design in a new session, the added vias are not added to the NDR via list any longer. You can use this option to do experiments in the current session, or write out a LEF or DEF file to put the modified NDR into the LEF or OA techfile. **Note:** If setGenerateViaMode -auto is true, then vias are always auto-generated even when -generate_via is not given. This is the normal usage behavior if you want to use the auto-generated vias for the rest of the flow (e.g. after save, and then restoring the design in a new session).
`-hard_spacing {0|1}`	Sets the non-default rule as HARDSPACING.
`-min_cut {`*`layer1`*`[:`*`layer2`*`] `*`min_cut ...`*`}`	
	Specifies the minimum cut limit for via layers.
`-name `*`ruleName`*	Specifies the name of the non-default rule to be modified.
`-spacing {`*`layer1`*`[:`*`layer2`*`] `*`spacing ...`*`}`	
	Specifies the spacing for the routing layers.
`-use_via_cut_class `*`string`*	Specifies the layer and `cut_class` list to support the LEF/DEF property. The value must be given in the format of either `{layer cut_class}` or `{layer cut_class nr_row nr_col}`. Refer to the examples below: modify_ndr -name new_1 -use_via_cut_class {V1 VX} modify_ndr -name new_2 -use_via_cut_class {V1 VX 2 1}
`-via {`*`via_name1 via_name2 ...`*`}`	
	Resets the via list to empty, and then adds the list of named vias to the NDR via list.
`-width {`*`layer1`*`[:`*`layer2`*`] `*`width ...`*`}`	
	Specifies the width for the routing layers.

Examples

- The following command creates a rule that requires double width and double spacing. All other values are the same as the default rule. `add_ndr` is like modify_ndr for via generation. If `setGenerateViaMode -auto` is true, then vias are automatically generated that match the different layer widths. If not, the vias from the default rule are used, which is not correct for wires that are twice the default width.
  ```
  add_ndr -name NDR_2w2s -spacing {metal1:metal4 0.2 metal5:mmetal6 0.4 metal7 0.6} -width {metal1:metal4 0.2 metal5:metal6 0.4 metal7 0.6}
  ```
- The following command modifies the width and spacing values in non-default rule NDR_2w2s created by the command. above. If `setGenerateViaMode -auto` is true, vias are automatically generated with matching widths and added to the NDR via-list. If not, then the original vias are not changed, which can be wrong for different widths.
  ```
  modify_ndr -name NDR_2w2s -spacing {metal1:metal4 0.4 metal5:metal6 0.8 metal7 0.9} -width {metal1:metal4 0.4 metal5:metal6 0.8 metal7 0.9}
  ```
- The following command auto-generates vias for rule NDR_2w2s with the `generateVias` command to get vias that match the wider wire widths, even if `setGenerateViaMode -auto` is false. This is only useful for debugging in the current session, or to write a LEF file with the NDR to add back into the LEF or OA techfile. If you want to use the auto-generated vias for the rest of the flow, you should set `setGenerateViaMode` -auto to true.
  ```
  modify_ndr -name NDR_2w2s -generate_vias
  ```
- The following command modifies existing rule NDR_2w2s to specify MINCUTS to 2 for the Vx layers (via1 to via3).
  ```
  modify_ndr -name NDR_2w2s -min_cut {via1:via3 2}
  ```

Related Information

- `add_ndr`
- `exportNdr`
- `setGenerateViaMode`
- `generateVias`
- Floorplanning the Design chapter in the *Innovus User Guide*

move_obj

```
  move_obj
[-help]
[-group]
{ -point {x y} | { -direction {up down left right} { -distance value | -to {core_box die_box} }}}
{ obj_list | -selected }
```

Moves objects (instances, modules, groups, placement blockage, routing blockage, hard macros, special routings, and I/O pins) vertically or horizontally by a specified distance. The objects can be moved to the die or core box boundary, as specified. You can use this command after selecting one or more objects in the design display area.

Parameters

`-direction {up down left right}`	
	Specifies the vertical and horizontal movement for the selected objects. Choose one of the following: • `up`: Specifies an upward move. • `down`: Specifies a downward move. • `left`: Specifies a left move • `right`: Specifies a right move. *Data_type*: enum, optional
`-distance` *value*	Specifies the distance, in micrometers, that the objects will be moved in the specified direction. *Data_type*: float, optional
`-group`	Moves all the specified or selected objects as a group. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `move_obj` parameter. For a detailed description of the command and all of its parameters, use the man command: `man move_obj`
obj_list	Specifies the list of objects to move. *Data_type*: (any_object)+, optional
`-point {`*x y*`}`	Specifies the lower left point that the objects will be moved to. *Data_type*: point, optional
`-selected`	Moves the selected object. *Data_type*: bool, optional
`-to {core_box die_box}`	
	Moves the object to the core_box or the die_box boundary, as specified. *Data_type*: enum, optional

Examples

- The following command moves each of selected objects to the left of core boundary:
  ```
  move_obj -selected -direction left -to core_box
  ```
- The following command moves the bounding box of all selected objects to the left of core boundary:
  ```
  move_obj -selected -direction left -to core_box -group
  ```
- The following command moves each of the selected objects to the left with 50um. The same as with option -group:
  ```
  move_obj -selected -direction left -distance 50
  ```
- The following command moves each of the selected objects' low left coordinates to the specified point:
  ```
  move_obj -selected -point {330 410}
  ```
- The following command moves the bounding box's low left coordinates of all selected objects to the specified point:
  ```
  move_obj -selected -point {330 410} -group
  ```

Related Information

- `pasteObject`
- `editCopy`
- `copyObject`
- Floorplanning the Design chapter in the *Innovus User Guide*

moveGroupPins

```
moveGroupPins
[-help]
[-depth float]
[-end_location {x y}]
[-expand_spacing_factor float]
[-layer string]
-loc {x y}
[-noFixed]
[-spacing float]
[-spread_type string]
[-start_location {x y}]
[-width float]
[-withOverlap]
```

Changes the pin layer, the pin size, pin status, resolves pin overlap, and moves a selected pin or pin group to a specific location. You can use this command after selecting the pin(s) in the move mode (selecting pins in the design display area with the *Move/Resize/Reshape* tool widget).

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `moveGroupPins` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man moveGroupPins`
`-depth float`	Specifies the depth of the pin or pin group, in micrometers. *Data_type*: float, optional
`-end_location {x y}`	Specifies the coordinates where the pins should end when the pins are spread along a range. *Data_type*: point, optional
`-expand_spacing_factor float`	Specifies the value that will be multiplied to the current spacing in tracks. *Data_type*: float, optional
`-layer string`	Specifies the number of the metal layer for the pin or pin group. *Data_type*: string, optional
`-loc {x y}`	Specifies the new x and y location for the pin or pin group. *Data_type*: point, required
`-noFixed`	Specifies that the moved pins or group of pins are not set to the *fixed* status. *Data_type*: bool, optional
`-spacing float`	Specifies the spacing between pins for multi-pin editing. *Data_type*: float, optional
`-spread_type string`	Specifies how spacing between pins is managed while moving grouped-pins. *Data_type*: string, optional
`-start_location {x y}`	Specifies the coordinates where the pins should start when the pins are spread along a range. *Data_type*: point, optional
`-width float`	Specifies the width of the pin or pin group, in micrometers. *Data_type*: float, optional
`-withOverlap`	Specifies that moved pin(s) can overlap. *Data_type*: bool, optional

Example

The following command moves the selected pin group to a new location (10,20):

```
moveGroupPins -loc 10 20
```

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*
  - Editing Pins

moveMacroInsideModule

```
moveMacroInsideModule
[-help]
{-hInst {hinstName(s)} | -all}
```

Moves all hard macros that belong to the specified module into its boundary. You use this command after the module is placed in the core area.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `moveMacroInsideModule` parameter. For a detailed description of the command and all of its parameters, use the man command: `man moveMacroInsideModule`.
`-all`	Moves all hard macros that belong to hierarchical modules. *Data_type*: bool, optional
`-hInst {hinstName}`	Specifies the name of the hierarchical instance. You can use wildcards (*?) with this parameter. *Data_type*: string, optional

Example

The following command moves all hard macros that belong to modules that have hierarchical instance names that match DTMF_INST/*_TEST_INST into its boundary:

```
moveMacroInsideModule -hInst DTMF_INST/*_TEST_INST
```

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

pasteObject

```
pasteObject
[-help]
[-loc {x y}]
```

Pastes copied placement or routing blockages at the specified location. Use the `copyObject` command to copy the required object. Use this command any time after floorplanning the design. You can also copy and paste selected placement and routing blockages from the GUI by using the Copy tool. To access the Copy tool, choose *Edit - Copy* from the menu bar or press `F3` after clicking the *Copy* widget ( ) on the toolbar.

Parameters

`-help`	Prints a brief description of the `pasteObject` command. For a detailed description of the command, use the `man` command: `man pasteObject`
`-loc {x y}`	Specifies the location for pasting the object copy. *Data_type*: point, optional

Example

- The following commands copy and paste the selected placement and routing blockages:

```
copyObject
pasteObject -loc 10 20
```

Related Information

- `copyObject`
- `editCopy`
- Floorplanning the Design chapter in the *Innovus User Guide*

placeAIO

```
placeAIO
[-help]
[-onlyAIO]
[-assignBump]
[-maxDistance distance]
[-fast]
[-packing]
[-ignoreAIOByName {list}]
[-ignoreAIOByCellName {list}]
[-hardFence]
```

Places I/O driver cells. The area I/O Library must be loaded when importing the design. Area I/O rows are also required.

Note: Once you run the `placeAIO -onlyAIO` command to place the area I/Os, you can run `placePIO -cellList` to specify the cell(s) that must be placed on the periphery.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `placeAIO` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man placeAIO`
`-assignBump`	If you have unassigned bumps for area I/O instance connections, this connects area I/O instances to the nearest unassigned bumps. This option requires that bump cells and bump locations are already loaded. **Note:** You can also assign bumps after area I/O placement by using the `assignBump` command. *Data_type*: bool, optional
`-fast`	Runs area I/O placement in the fast mode, trading speed for quality. *Data_type*: bool, optional
`-hardFence`	If specified, restricts the area I/O placement to the power domain fence to which the area I/O's belong to. *Data_type*: bool, optional
`-ignoreAIOByCellName {list}`	Ignores all area I/O instances that instantiate from the cells specified in the list, during placement. For example, if a cell `PDI` has 2 AIO instances, `test_in_1` and `test_in_2`, this parameter ignores both the instances during placement. *Data_type*: string, optional
`-ignoreAIOByName {list}`	Ignores all the area I/O cells specified in the list, during placement. Specify this parameter if you already placed the area I/O cells on the periphery and you want to avoid placing the same cells in the core. *Data_type*: string, optional
`-maxDistance value`	Specifies the maximum allowable distance between the I/O driver cells and the bumps. *Data_type*: int, optional
`-onlyAIO`	Places only area I/O instances. If this option is not specified, the command will also place all standard cell instances and blocks. *Data_type*: bool, optional
`-packing`	Specifies the packing of I/O driver cells within each cluster of rows. This option is only applicable to designs with clusters of I/O rows. *Data_type*: bool, optional

Examples

- The following command places area I/O instances and all standard cell instances, and assign bumps for all chip I/O pins:
  ```
  placeAIO -assignBump
  ```
- The following command places only area I/O cells:
  ```
  placeAIO -onlyAIO
  ```

Related Information

- placePIO
- Floorplanning the Design chapter in the *Innovus User Guide*

placePadIO

```
placePadIO
[-help]
[-rows numOfRow]
[-maxIOHeight]
```

Places the I/O pads evenly from one row to multiple rows. You can use this command after design initialization when the Innovus software automatically places the I/O pads in one row.

Parameters

`-help`	Outputs a brief description that includes type and default information for each placePadIO parameter. For a detailed description of the command and all of its parameters, use the man command: `man placePadIO.`
`-rows numOfRows`	Specifies the number of I/O rows needed for the placement. *Data_type*: int, optional
`-maxIOHeight`	Specifies the maximum height used to calculate the head box. By default, this command uses the minimum height. *Data_type*: bool, optional

Example

- The following command places I/O pads in two I/O rows:
  ```
  placePadIO -row 2
  ```

Related Information

- `placePIO`
- `placeAIO`
- `setIoRowMargin`
- Floorplanning the Design chapter in the *Innovus User Guide*

placePIO

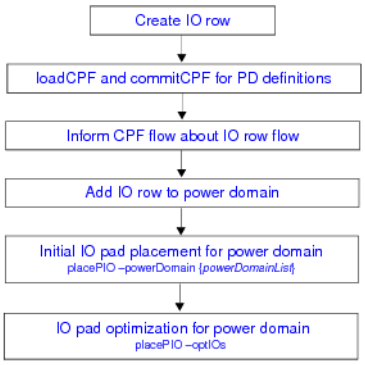
```
placePIO
[-help]
[-assignBump]
[-optIOs]
[-overflowMap]
[-maxIOHeight]
[-ioFile fileName]
[-rdlConstraintFile fileName]
[-noRandomPlacement]
[-extraConfig fileName]
[-cellList {cellList}]
[-instList {instList}]
[-powerDomain powerDomainList]
```

Places CLASS PAD AREA IO cells on the die boundary (periphery) in random order. Use this command after the design is loaded.

- The command, by default, honors pre-placed or `FIXED` area I/Os in the design and does not modify their placement when executed.
- The command can place IO pads onto the IO rows that belong to a power domain and optimize the locations of the IO pads for the power domain.

  ```
  placePIO -powerDomain {VCCQ_MC0 VCCQ_U VCCQ2_E VCCQ_L VCCQ2_H VCCQ_D VCCQ2_C}
  placePIO -optIOs
  ```

  The I/O row information is retrieved from the power domain itself. IO rows need to be created and added to power domain before the placement and optimization. Run `placePIO -powerDomain {powerDomainList}` for a random placement then use `placePIO -optIOs` for optimized result.

  You can use this function to place IO pads in a design that has multiple power domains with IO pads. Innovus recognizes the relationship between the IO pads, IO rows, and power domains and so placement and optimization are automatic.
- The command reads options specified in the `setFlipChipMode` command before placing the CLASS PAD AREA IO cells on the die boundary. It retrieves the `rdlconstraintfile` and `sroute` config file information from the `setFlipChipMode -constraintFile` and `setFlipChipMode -extraConfig` command options.
- During bump assignment, the command uses resistance constraints that are specified in the `rdlconstraintfile` for net(s). To view a constraint file, see "Examples and Report Files" in Flip Chip Methodologies chapter of the *Innovus User Guide*.
- The `placePIO -optIos -assignBump` command also reads `setFlipChipMode -routeWidth` value before reassigning the signal bumps in the design.
- When the I/O row flow is enabled (`setIoFlowFlag 1`), the `placePIO` command restricts I/O placement in the rows created previously, instead of all possible locations. However, you need to ensure that the I/O rows are created outside the I/O box.

  Command flow:

  ```
  setIoFlowFlag 1
  createIoRow
  placePIO
  ```

Note: IO cells must be defined as CLASS PAD on the boundary of the design. The peripheral I/O Library must be loaded when importing the design. Peripheral I/O rows are also required.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `placePIO` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man placePIO`.
`-assignBump`	Allows signal bumps to be reassigned to improve routing if the I/O cells have been fixed. *Data_type*: bool, optional
`-cellList {instlist}`	Specifies which area I/O cell(s) must be placed on the periphery. Once you run `placePIO -cellList` to grab the area I/O cells for placement on the periphery, you can run `placeAIO -ignoreAIOByName {list}` to avoid the placement of these cells in the core area. *Data_type*: string, optional
`-extraConfig fileName`	Specifies a file containing user-defined placement options written in `sroute` syntax. For more information, see "Extra Configuration File Options for fcroute" in `fcroute`. *Data_type*: string, optional
`-ioFile fileName`	Specifies the name of a file to control the placement of the I/O cells. It supports the version 3.0 I/O file format. *Data_type*: string, optional
`-instList {instList}`	Specifies which area I/O instance(s) must be grabbed for placement on the periphery. Once you run `placePIO -instList` to place the area I/O instances on the periphery, you can run `placeAIO -ignoreAIOByName {list}` to avoid the placement of these instances in the core area. *Data_type*: string, optional
`-maxIOHeight`	Specifies the maximum height for the I/O row. *Data_type*: bool, optional
`-noRandomPlacement`	Without options, the `placePIO` command randomly places all I/O cells on the periphery. This option inhibits this behavior and optimizes pads without initial placement. *Data_type*: bool, optional
`-optIOs`	Allows I/O cells to be moved to improve routing if the bumps have been fixed, optimizing the initial placement. *Data_type*: bool, optional

`-overflowMap`	Displays the congestion map using the width and spacing parameters specified in the extra configuration file. To display the congestion map, run the following command to specify the options for the extra configuration file:
	`placePIO -noRandomPlacement -overflowMap -extraConfig scripts/global.cfg`
	where:
	`scripts/global.cfg`:
	`srouteBottomLayerLimit 8 /* Assume the intended routing layer range is M8 to M9 */`
	`srouteTopLayerLimit 9`
	`srouteRouteWidth 10000 /* n is the routing width, in db unit - 5u */`
	Data_type: bool, optional

`-powerDomain powerDomainName`	Specifies the name of the power domain that consists of peripheral I/Os. The I/O row information can now be retrieved from the power domain itself. *Data_type*: string, optional
`-rdlConstraintFile fileName`	Specifies a file with constraints to control the placement of the I/O cells. The command uses resistance constraints specified in this file, for cell placement. To view a constraint file, see "Examples and Report Files" in Flip Chip Methodologies chapter of the *Innovus User Guide*. *Data_type*: string, optional

Example

- The following command places CLASS PAD AREA IO cells on the die boundary (periphery) and allows I/O cells to be moved to improve routing if the bumps have been fixed, optimizing the initial placement:
  `placePIO -optIOs`

Related Information

- `placeAIO`
- `placePadIO`
- `setIoRowMargin`
- Floorplanning the Design chapter in the *Innovus User Guide*
- Flip Chip Methodologies chapter in the *Innovus User Guide*
  - Useful Constraints for Flip Chip Routing
- Tools Menu chapter in the *Innovus Menu Reference*
  - Flip Chip Route - Advanced - Routing Constraints GUI

predict_floorplan

```
  predict_floorplan
[-help]
[-allow_illegal_macro]
[-enable_cpg]
[-reshape [[-post_reshape_tcl string]]]
[{-reshape | -skip_generate_fence }]
```

Generates an initial floorplan that can be used as a starting point for making the final floorplan. It can generates fences on power domains automatically, and then does the placement for all instances. If the design has macros/blocks, the `predict_floorplan` command calls the mixed placer to place the macros/stdcells, however, the design has only stdcells, then the `place_opt_design` command is called.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `predict_floorplan` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man predict_floorplan`
`-allow_illegal_macro`	Allows the tool to continue even when macros are not legally placed. *Default*: false *Data_type*: bool, optional
`-enable_cpg`	Enables CPG for a design which already has a floorplan with macro locations. *Data_type*: bool, optional
`-post_reshape_tcl` *string*	When specified it allows the tool to load user specified floorplan objects (like tracks) after the floorplan has been reshaped. **Note**: This option must be use with the -reshape parameter. *Data_type*: string, optional
`-reshape`	Enables the floorplan to be enlarged in order to fit all instances. *Data_type*: bool, optional
`-skip_generate_fence`	When specified, auto-fence generation is skipped. *Data_type*: bool, optional

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

queryFPlanObject

```
queryFPlanObject
[-help]
[-pd]
```

Prints the properties of selected objects to the console. This command supports only instances and modules. You can use this command after selecting one ore more instances or modules.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `queryFPlanObject` parameter. For a detailed description of the command and all of its parameters, use the man command: `man queryFPlanObject`
`-pd`	Use this option to output power domain information, as well as check if the objects belong to a power domain. *Data_type*: bool, optional

Example

The following commands select and print property information for module `modA`:

```
selectInst modA
queryFPlanObject -pd
```

Related Information

- `selectInst`
- Floorplanning the Design chapter in the *Innovus User Guide*

refine_macro_place

```
refine_macro_place
[-help]
[-verbose ]
```

Legalizes the macros based on constraints such as halo and blockages, forbidden spacing, min-space, etc. To define the constraints of macros, use the `set_macro_place_constraint` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `refine_macro_place` parameter. For a detailed description of the command and all of its parameters, use the man command: `man refine_macro_place`
`-verbose`	Writes the macro movement details to a tcl file. *Data_type*: bool, optional

Example

The following command specifies the minimum space between macro-to-core boundary as 5 and the forbidden space between macro-to-core boundary as 2.

```
set_macro_place_constraint -forbidden_space_to_core 2 -min_space_to_core 5
refine_macro_place
```

Related Information

- `set_macro_place_constraint`
- `setPlaceMode`
- Floorplanning the Design chapter in the *Innovus User Guide*
- Using the Mixed Placer chapter in the *Innovus User Guide*

refineMacro

```
refineMacro
[-help]
{{-area {x1 y1 x2 y2} [-selected]}| -permutePack | -markStep | -restoreStep step_number
       |-restoreMark mark_number}
[-adjustPack]
[-ioPinClearance]]
```

Refines the placement of macros in the floorplan. By default, the `refineMacro` command performs global incremental macro adjustment. You can also adjust specific macro packs, or all of the macros in a specific area of the design. You can use `refineMacro` command after creating an initial floorplan and analyzing the results. Macros must have a status of PLACED in the database in order to use this command.

Parameters

`-adjustPack`	Adjusts the way that macros packs are placed, relative to their locations. For example, an array of 16 macros might be placed in four rows of four macros (4 x 4), if they are near the top or bottom of the chip. This might change to eight rows of two macros (8 x 2), if the macros are moved near the left or right edge of the chip. *Data_type*: bool, optional
`-area` *x1 y1 x2 y2*	Adjusts the placement of any macros within the specified area to reduce empty area between them. If you also specify `-selected`, the selected macros are moved to the specified area and their position is adjusted along with any macros already within the area. *Type:* Float, specified in microns
`-help`	Outputs a brief description that includes type and default information for each `refineMacro` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man refineMacro`.
`-ioPinClearance`	Leaves an internally calculated amount of clearance space between placed macros and block I/O pins. Use this parameter to ensure that pins are accessible to the router. *Data_type*: bool, optional
`-markStep`	Marks an intermediate macro refinement step to save. When you perform incremental macro refinement, each adjustment is considered a step. By default, all incremental steps done with the `refineMacro` command are saved and listed in the log file with a number. Since the manal refinement steps (such as move, rotate, and flip) are not saved by default, you can use this parameter to mark manual refinement steps, or any other important step that you want to save. **Note:** Innovus saves up to 1000 incremental refinement steps in the database and 20 marks before overwriting them. *Data_type*: bool, optional
`-permutePack`	Adjusts the placement of a selected pack of macros to reduce empty area between them. A pack is a set of macros from the same seed that have similar sizes and aspect ratios and have been grouped together. You must select the macro pack in the main window before using this parameter. You can select one macro to select the entire pack. Macro pack information is stored temporarily in the database. If you quit the Innovus session, the information is lost. *Data_type*: bool, optional
`-restoreMark` *mark_number*	Returns the design to the state it was in after the specified marked step (`-markStep`). For example, to return the design to the state it was in after the step saved as `mark2`, specify: `refineMacro -restoreMark 2` You can find the mark numbers listed in the log file. *Data_type*: int, optional

`-restoreStep` *step_number*	Returns the design to the state it was in after the specified intermediate non-marked step.
	For example, to return the design to the state it was in after step 4, specify:
	`refineMacro -restoreStep 4`
	To return the design to the state it was in after the previous adjustment, specify:
	`refineMacro -restoreStep -1`
	You can find the step numbers listed in the log file.
	Data_type: int, optional

`-selected`	Moves the selected macros to the specified area and adjusts their position along with any macros already within the area. **Note:** You cannot use this parameter without specifying the `-area` parameter. *Data_type*: bool, optional

Example

The following command moves the selected macros to a new area:

```
refineMacro -selected -area 100 100 200 300
```

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

report_narrow_channel

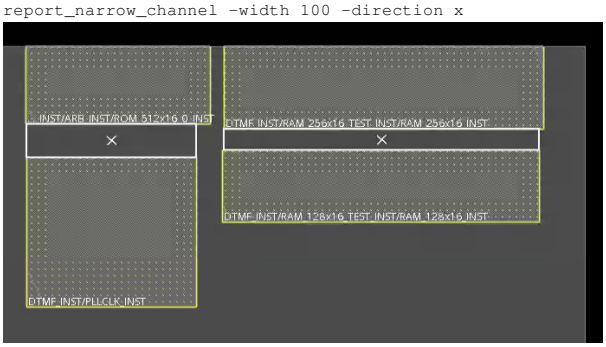
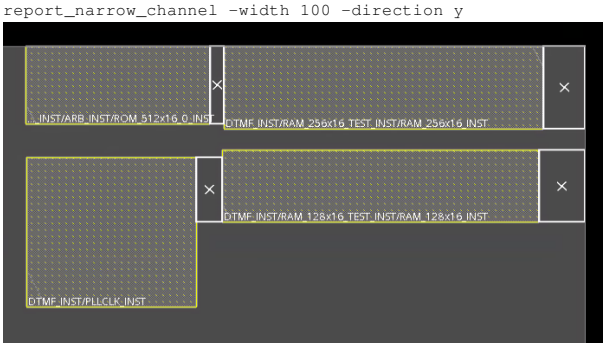
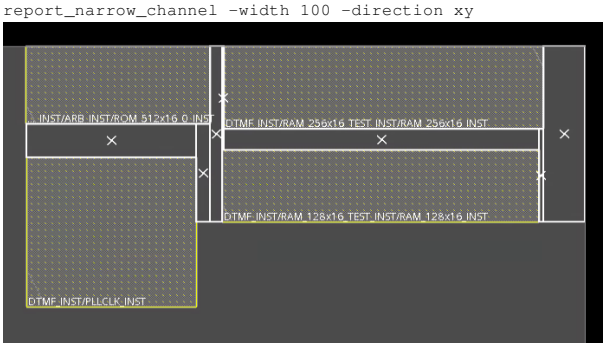
```
report_narrow_channel
[-help]
{-width width}
[-ignore_placement_blockage ]
[-no_merge_channel_check ]
[-direction {x y xy}]
[-active_objects {macro macroHalo ioPad ioCell core fence hardBlkg softBlkg partialBlkg routeBlkg row}] }
```

Creates a report that contains a list of narrow channels.

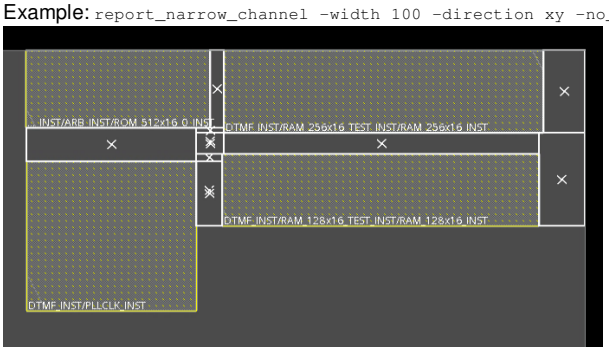
Note: The `report_narrow_channel` command returns channel boxes as its return value.

Parameters

`-active_objects {macro macroHalo ioPad ioCell core fence hardBlkg softBlkg partialBlkg routeBlkg row}`	
	Specifies the active objects for narrow channel checking. *Default*: `"core macro macroHalo hardBlkg"` *Data_type*: enum_list, optional
`-help`	Outputs a brief description that includes the type and default information for each `report_narrow_channel` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_narrow_channel`

`-direction {x y xy}`	Specifies the direction for checking narrow channels. You can specify x, y, or xy.
	Default: xy
	Example:
	`report_narrow_channel -width 100 -direction x`
	 
	
	`report_narrow_channel -width 100 -direction y`
	 
	
	`report_narrow_channel -width 100 -direction xy`
	 
	
	Data_type: enum, optional

`-ignore_placement_blockage`	When specified, the placement blockages are ignored while computing channel widths. *Data_type*: bool, optional

`-no_merge_channel_check`	Specifies that the narrow channel error boxes will be merged into contiguous error boxes. Example: `report_narrow_channel -width 100 -direction xy -no_merge_channel_check`  *Data_type*: bool, optional
`-width` *width*	Specifies the width for checking the channels. *Maximum*: `100000` *Minimum*: `0` *Data_type*: float, optional

Example

The following command checks for channels with width 1.0 and ignores placement blockages:

```
report_narrow_channel -width 1.0 -ignore_placement_blockage
```

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

report_obj_connectivity

```
report_obj_connectivity
[-help]
[-direction {in|out|all}]
[-file file_name]
[-level level]
[-no_split]
[-through_registers]
[-to_ports]
[-insts inst+] | [-ports port+] | [-selected]
```

Reports the connections of specified macros, selected macros, or specified ports. You can also report the input and output connectivity separately. By default it reports both.

Parameters

`-direction {in|out|all}`	Specifies the pin direction of objects to be reported. *Default*: `all` *Data_type*: string, optional
`-file file_name`	Specifies the name of the output report file in which the trace macro results will be saved. *Default*: `<top_module_name>.tmcrpt` *Data_type*: string, optional
`-help`	Outputs a brief description that includes type and default information for each `report_obj_connectivity` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_obj_connectivity`
`-insts inst+`	Specifies the macros to be reported. *Data_type*: string, optional
`-level level`	Specifies the trace level. *Maximum*: 10 *Minimum*: 1 *Default*: 5 *Data_type*: int, optional
`-no_split`	Reports a single object per line.
`-ports port+`	Specifies the IO ports to be reported. *Data_type*: string, optional
`-through_registers`	Reports the through registers between selected objects. *Data_type*: bool, optional
`-to_ports`	Reports the connectivity from the selected objects to IO ports. *Data_type*: bool, optional
`-selected`	Reports the selected macros or IO ports. *Data_type*: bool, optional

Example

The following command reports the selected macros or IO ports:

```
report_obj_connectivity -selected
```

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*
- Floorplan Menu chapter in the *Innovus Menu Reference*

reportNetGroup

```
reportNetGroup
[-help]
[-outfile fileName]
```

Creates a report that contains net groups and their nets. You can use this command after importing the design.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `reportNetGroup` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man reportNetGroup`
`-outfile fileName`	Specifies the report filename. If no filename is specified, the report is printed to the console window. *Data_type*: string, optional

Example

Check the bump assignment and send the report to `netGrp.rpt`:

```
reportNetGroup -outfile netGrp.rpt
```

The output format is the same as the NetGroup section of the floorplan file:

```
# NetGroup Report File
NetGroup: netGroupName_one
    GroupNet net_name_01_in_netGroupName_one
    GroupNet net_name_02_in_netGroupName_one
NetGroup: netGroupName_two
    GroupNet net_name_01_in_netGroupName_two
    GroupNet net_name_02_in_netGroupName_two
```

Related Information

- `createNetGroup`
- `addNetToNetGroup`
- `deleteNetGroup`
- `deleteNetFromNetGroup`
- Partitioning the Design chapter in the *Innovus User Guide*
- Floorplanning the Design chapter in the *Innovus User Guide*

reportSeedConnection

```
reportSeedConnection
[-help]
[-constraints filename]
[-excNetFile filename]
[-file outfile]
```

Reports the number of connections between user-specified seeds.

Parameters

`-constraints` *filename*	Specifies the name of the input constraints file containing the seed definition. *Data_type*: string, optional

`-excNetFile` *filename*	Specifies the name of the exclude net file to use during the calculation of seed connections. The excNetFile format should be:
	`#any comment`
	`netName, netName,...`
	`netName, netName,...`
	Data_type: string, optional

`-file` *filename*	Specifies the name of the output report file in which the information about the number of connections between user-specified seeds is written. *Data_type*: string, optional
`-help`	Outputs a brief description that includes type and default information for each `reportSeedConnection` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reportSeedConnection`.

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

reportSelect

```
reportSelect
[-help]
-file filename
```

Reports the properties of one or more selected objects in the design display area in the main console window and the Innovus log file. You can use this command after selecting one or more objects in the design display area.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `reportSelect` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man reportSelect`
`-file filename`	Specifies the name of the output file that reports the properties of the selected object. *Data_type*: string, optional

Example

The following command generates a report file `object.rpt`, which displays the properties of the selected object:

```
reportSelect -file object.rpt
```

The following is the output format of the report file:

```
Object Type : IO Cell
Name : IOPADS_INST/Ptdspip00
No. of Terminals : 2
Cell Type : PDIDGZ
Cell Width : 40.0
Cell Height : 235.0
Location : 253.78 0.0
Location Origin : Lower Left
Orientation : R0
Status : PLACED
Routing Halo : None
InstGroup : None
```

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*
- Floorplan Menu chapter in the *Innovus Menu Reference*

reportUnsnapBlocks

```
reportUnsnapBlocks
[-help]
```

Reports unsnapped blocks based on the snap preferences (as defined in the *Floorplan Tab* of the *Preferences* form). You can use this command after importing the design.

Parameters

`-help`	Outputs a brief description of the `reportUnsnapBlocks` command. For a detailed description of the command, use the man command: `man reportUnsnapBlocks`

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*
- Floorplan Menu chapter in the *Innovus Menu Reference*

reset_macro_place_constraint

```
reset_macro_place_constraint
[-help]
[-array]
[-avoid_abut_macro_edge_with_pins]
[-cell_obs]
[-cpg]
[-forbidden_space_to_core]
[-forbidden_space_to_macro]
[-honor_strict_spacing_constraint]
[-horizontal_stacking]
[-instance_orientation]
[-macro_corner_keepout]
[-max_io_pin_group_keep_out]
[-min_space_to_core]
[-min_space_to_macro]
[-parallel_run_length]
[-pg_resource_model]
[-same_length_site]
[-track_adjustment]
[-vertical_stacking]
```

Resets the macro constraints that are honored by the macro placer. To define the constraints of macros, use the `set_macro_place_constraint` command.

Parameters

`-array`	Deletes all the array constraints. *Data_type*: bool, optional
`-avoid_abut_macro_edge_with_pins`	
	Specifies that macro edges with pins should not be abutted even when the macro distance is within the forbidden spacing setting. *Data_type*: bool, optional
`-cell_obs`	Deletes all the cell obstruction constraints. *Data_type*: bool, optional
`-cpg`	Deletes all the cpg constraints. *Data_type*: bool, optional
`-forbidden_space_to_core`	Resets the forbidden space to core. *Data_type*: bool, optional
`-forbidden_space_to_macro`	Resets the forbidden space to macro. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `reset_macro_place_constraint` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reset_macro_place_constraint`
`-honor_strict_spacing_constraint`	Specifies that forbidden spacing and minimum spacing constraints should be honored strictly. *Data_type*: bool, optional
`-horizontal_stacking`	Resets the maximum horizontal stack depth and the minimum horizontal spacing. *Data_type*: bool, optional
`-instance_orientation`	Deletes all the instance orientations. *Data_type*: bool, optional

`-macro_corner_keepout`	Resets the macro corner keepout constraints. *Data_type*: bool, optional
`-max_io_pin_group_keep_out`	Deletes the macro-only blockages which are automatically added to IO pins and resets the maximum height or width of macro-only blockages to 0. *Data_type*: bool, optional
`-min_space_to_core`	Resets the minimum space to core. *Data_type*: bool, optional
`-min_space_to_macro`	Resets the minimum space to macro. *Data_type*: bool, optional
`-parallel_run_length`	Resets the parallel run length to enable space checking. *Data_type*: bool, optional
`-pg_resource_model`	Deletes the pg resource model constraints. *Data_type*: bool, optional
`-same_length_site`	Resets the minimum difference site number. *Data_type*: bool, optional
`-track_adjustment`	Deletes all the track adjustment. *Data_type*: bool, optional
`-vertical_stacking`	Resets the maximum vertical stack depth and the minimum vertical spacing. *Data_type*: bool, optional

Example

The following command deletes all the pg resource model constraints:

```
reset_macro_place_constraint -pg_resource_model
```

Related Information

- `set_macro_place_constraint`
- `check_macro_place_constraint`
- `get_macro_place_constraint`
- `write_macro_place_constraint`
- `refine_macro_place`
- Floorplanning the Design chapter in the *Innovus User Guide*
- Using the Mixed Placer chapter in the *Innovus User Guide*

resizeFloorplan

```
resizeFloorplan
[-help]
[-xSize xAxisSize]
[-ySize yAxisSize]
[-xPercent xAxisPercent]
[-yPercent yAxisPercent]
[-undo]
[-pushCore]
[-forceResize]
```

Resizes the floorplan while maintaining the relative locations of the existing floorplan. You can also set global parameters for the `resizeFloorplan` by using the `setResizeFPlanMode` command. Parameters that you specify with `setResizeFPlanMode` are then used automatically whenever you run the `resizeFloorplan` command or use the *Resize Floorplan* form. You can use this command after importing the design. When run on a block design, the `resizeFloorplan` command keeps physical PG pins during resize and trims the PG pin geometries that fall out of the chip boundary after resize.

Note: You can use the `addSizeBlockage` command to add a size blockage object that controls the behavior of the resize line under a covered area.

Parameters

`-forceResize`	Resizes the floorplan even if the target size can not be met. However, the floorplan is resized as near as possible to the specified size. *Default*: If you do not specify this parameter, the floorplan is not resized if the target size can not be met. **Note**: This parameter can be used only when the `setResizeFplanMode -shiftBased` parameter has been set to `true`. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes type and default information for each `resizeFloorplan` parameter. For a detailed description of the command and all of its parameters, use the man command: `man resizeFloorplan`
`-pushCore`	Pushes the core when the design is I/O bound. *Data_type*: bool, optional
`-undo`	Revert to the floorplan that existed before the command was run. *Data_type*: bool, optional
`-xPercent xAxisPercent`	Specifies the percentage of how much the floorplan should be expanded or shrunk in the x direction. A positive value expands the floorplan and a negative value shrinks the floorplan. *Data_type*: float, optional
`-yPercent yAxisPercent`	Specifies the percentage of how much the floorplan should be expanded or shrunk in the y direction. A positive value expands the floorplan and a negative value shrinks the floorplan. *Data_type*: float, optional
`-xSize xAxisSize`	Specifies how much the floorplan should be expanded or shrunk in the x direction. A positive value expands the floorplan and a negative value shrinks the floorplan. This value is in microns. **Note**: If both `-xSize` and `-ySize` parameters are specified, the resize order is the same as the order of the parameters. *Data_type*: float, optional
`-ySize yAxisSize`	Specifies how much the floorplan should be expanded or shrunk in the y direction. A positive value expands the floorplan and a negative value shrinks the floorplan. This value is in microns. *Data_type*: float, optional

Example

The following command resizes the floorplan as follows: shrinks the floorplan in shift based mode by -50 microns in the x direction and -40 microns in the y direction.

```
resizeFloorplan -xSize -50 -ySize -40
```

Related Information

- `addSizeBlockage`
- `deleteSizeBlockage`
- `getResizeFPlanMode`
- `setResizeFPlanMode`
- Floorplan Menu chapter in the *Innovus Menu Reference*
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Resizing the Floorplan

save_global

```
save_global
[-help]
file_name
```

Saves the current global variable settings in a file. You can use this command after setting the global variable.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `save_global` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man save_global`
`file_name`	Specifies the file in which the global variable setting should be saved. *Data_type*: string, required

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

saveFPlan

```
saveFPlan
file
[-help]
[-noName]
[-objType {macro | pin | bndry | special_route | pin_constraint}]
[-xml]
```

Saves the floorplan information to a file. The command saves all the floorplan blocks that are placed/fixed/covered, in the floorplan file. When you save a floorplan, the Innovus software saves all objects which can be saved and any specific properties which are affected if the data is saved. By default, the floorplan information is saved as a Version 8 floorplan file. After you save a floorplan, the Innovus software creates the following files:

- A general floorplanning file with the extension .fp
- A power route data file with the extension .fp.spr
  Note: If there is an entry in the IO Cell Libraries field in the Design Import form, a third file is created with the extension `.fp.areaio`.

Note:

- The `saveFPlan` command honors the blackbox min/max aspect ratio information in the floorplan information file.
- The `saveFPlan` command saves the net attributes including non-default attributes of a net, like weight, bottomPreferredRoutingLayer, or detour. You can use this command after importing the design.
- The `saveFPlan` command saves the user-specified seed information by default.
- The `saveFPlan` command saves the information related to blockages, tracks, and pin shapes into the *.fp file.
- The `saveFPlan` command saves feedthrou-bumps into the .fp.areaio file. The format is as below:
  Bump: [BumpName] [BumpCellName] [x coord] [y coord] [BumpPin_1] [PinName_1] [BumpPin_2] [PinName_2]

Parameters

file	Specifies the name of floorplan file to be saved. *Data_type*: string, required
`-help`	Outputs a brief description that includes type and default information for each `saveFPlan` parameter. For a detailed description of the command and all of its parameters, use the man command: `man saveFPlan`.
`-noName`	Specifies that floorplan object names are not output to the floorplan file when you save the floorplan. *Data_type*: bool, optional
`-objType {macro | pin | bndry | special_route | pin_constraint}`	
	Specifies the objects section in the floorplan file that will be updated. The sections can be: • `macro` • `pin` • `bndry` • `special_route` • `pin_constraint` **Note:** The `saveFPlan` command supports routing blockages on master slice layers. *Data_type*: string, optional
`-xml`	Saves the floorplan information in the XML format floorplan file. *Data_type*: bool, optional

Example

The following command writes the current floorplan information to the file `myFloorPlan.fp`:

```
saveFPlan myFloorPlan.fp
```

Related Information

- `loadFPlan`
- `loadPtnPin`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Saving and Loading Floorplan Data

saveIoFile

```
saveIoFile
[-help]
[-v2]
[-locations | -byOrder]
[-includeCellName]
[-only_selected_bump]
[-temp [-ioOrder {default | clockwise | counterclockwise}]]
[-relativeOrient]
fileName
```

Saves the current I/O information to a Version:3 file by default. It also provides you the option to save the I/O information in a Version:2 file for 6.2 and earlier releases of the software. The saved file can be used to specify the I/O Assignment File in the Design Import form. This is used to save the I/O pin or pad order to a file after assigning pins or pads by loading a floorplan file or DEF data. This I/O Assignment file is rule-based, and the keyword in the I/O file is Version:3. You can use this command after floorplanning the design.

Note: The `saveIoFile` command saves the bump connect target information in the I/O file by default.

Note: The `saveIoFile` command will write all the information of I/O pins and I/O pads if the design has both IO pins and IO pads.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `saveIoFile` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man saveIoFile`
`-byOrder`	Specifies the order of the I/O pad placement locations in the I/O file. *Data_type*: bool, optional
`-includeCellName`	Specifies that the cell name of the I/O instance must be included in the I/O file. *Data_type*: bool, optional
`-ioOrder {default | clockwise | counterclockwise}`	
	Specifies the order of placing the I/O pins in the I/O template file. You can choose one of the following: • `default`: Specifies the default pin order. For a vertical edge, the default pin order is from the bottom to the top. For a horizontal edge, the default pin order is from the left of the right. • `clockwise`: Specifies the pin order in the clockwise direction. • `counterclockwise`: Specifies the pin order in the clockwise direction. **Note:** For I/O cells, the saveIoFile command will write out only the default I/O order. *Data_type*: string, optional
`fileName`	Specifies the name of the pin order file to be written to. *Data_type*: string, required
`-locations`	Generates an I/O file that has placement locations rather than a sequence or order of the I/O pads. Placement co-ordinates are relative to the design boundary and are not absolute locations. *Default:* If you do not specify this parameter, it generates an order file. *Data_type*: bool, optional
`-only_selected_bump`	Saves only the selected bumps to the IO file. By default, `saveIoFile` dumps out embedded bumps to the IO file. When the `-only_selected_bump` option is specified, only selected bumps are dumped out and embedded bumps are ignored. *Data_type*: bool, optional
`-relativeOrient`	Specifies an I/O orientation relative to the bottom (south) side. *Data_type*: bool, optional
`-temp`	Performs initial I/O placement by randomly distributing the I/Os evenly. Writes out an I/O template file with evenly spaced I/O pins, after design import. Once the I/O template file is generated, you can modify the I/O file manually and reload the file in Innovus. *Data_type*: bool, optional
`-v2`	Saves the current I/O information to a Version: 2 file for 6.2 and earlier releases of the software. If you specify this parameter for saving an I/O file consisting of rectilinear design for I/O pads, Innovus displays an error message. This is because the rectilinear design for I/O pads is supported only in the current release version of Innovus. *Default*: If you do not specify this parameter, the I/O information is saved in a Version: 3 file. *Data_type*: bool, optional

Example

- The following command writes the current I/O pad and pin information to Version: 3 file:
  ```
  saveIoFile myIoPlan.io
  ```
- The following command writes the current I/O pad and pin information to a Version: 2 file and specifies the clockwise placement of the I/O pins in the I/O template file:
  ```
  saveIoFile -v2 -ioOrder clockwise -template myIoPlan.io
  ```

Related Information

- `loadIoFile`
- Floorplanning the Design chapter in the *Innovus User Guide*
- Data Preparation chapter in the *Innovus User Guide*
  - Generating the I/O Assignment File

select_row

```
select_row
[-help]
[-all | -name string | -site string]
```

Selects the specified row(s), enabling you to issue an additional command for the row. Use this command after importing a design.

Parameters

`-all`	Specifies that all rows will be selected. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `select_row` parameter. For a detailed description of the command and all of its parameters, use the man command: `man select_row`
`-name string`	Specifies the name or id of the row(s) to be selected. *Data_type*: string, optional
`-site string`	Specifies the site for which rows will be selected. *Data_type*: string, optional

Example

The following command selects rows from the site site0A10.

```
select_row -site site0A10
```

Related Information

- `deleteRow`
- `cutRow`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Creating and Editing Rows

selectGroup

```
selectGroup
[-help]
groupName
```

Selects a group. You can use this command after importing the design.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `selectGroup` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man selectGroup`
`groupName`	Specifies the name of the group. *Data_type*: string, required

Related Information

- `deselectGroup`
- Floorplanning the Design chapter in the *Innovus User Guide*

selectInst

```
selectInst
[-help]
instName | {list_of_instNames}
```

Selects an instance and highlights it in the design display window. You can use this command after importing or restoring the design.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `selectInst` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man selectInst`
instName	Specifies the name of one or more than one insts. You can use wildcards (*) to specify inst names. *Data_type*: (inst)+, required

Example:

- The following command selects nets and highlights them in the design display window:
  ```
  selectInst {reg1 reg2}
  ```

Related Information

- `queryFPlanObject`
- `deselectInst`
- Floorplanning the Design chapter in the *Innovus User Guide*

selectInstByCellName

```
selectInstByCellName
[-help]
cellName
```

Selects instances and hierarchical instances by cell name. When hierarchical instances are selected, child instances are also highlighted. You can use this command after importing the design.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `selectInstByCellName` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man selectInstByCellName`
`cellName`	Specifies the name of the cell. *Data_type*: (libCell/ptnCell|vCell|topCell)+, required

Related Information

- `deselectInstByCellName`
- Floorplanning the Design chapter in the *Innovus User Guide*

selectInstOnNet

```
selectInstOnNet
[-help]
netName
```

Selects an instance on a net. You can use this command after importing the design.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `selectInstOnNet` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man selectInstOnNet`
`netName`	Specifies the name of the net. *Data_type*: (net)*, required

Related Information

- `deselectInstOnNet`
- Floorplanning the Design chapter in the *Innovus User Guide*

selectIOPin

```
selectIOPin
[-help]
pinName
```

Selects an I/O pin. You can use this command after importing the design.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `selectIOPin` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man selectIOPin`
`pinName`	Specifies the name of the I/O pin. *Data_type*: string, required

Related Information

- `deselectIOPin`
- Floorplanning the Design chapter in the *Innovus User Guide*.

selectNet

```
selectNet
[-help]
{netName | {list_of_netNames} | -allDefClock | -clock | -nonDefaultRule | -shield}
```

Selects a net and highlights it in the design display window. You can use this command after importing or restoring the design.

Parameters

`-allDefClock`	
	Selects all nets that have a DEF attribute `+ USE CLOCK` **Note:** The `-allDefClock` parameter is for backward compatibility only. Cadence recommends that you use the `-clock` parameter to select all nets that have a DEF attribute `+ USE CLOCK` *Data_type*: bool, optional
`-clock`	Selects all clock nets. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `selectNet` parameter. For a detailed description of the command and all of its parameters, use the man command: `man selectNet`
netName | {*list_of_netNames*}	
	Specifies the name of one or more than one nets. You can use wildcards (*) to specify net names. *Data_type*: (net)*, optional
`-nonDefaultRule`	
	Selects all nets that have a DEF attribute `+ USE NONDEFAULTRULE` *Data_type*: bool, optional
`-shield`	Selects all nets that have a DEF attribute `+ USE SHIELDNETS` *Data_type*: bool, optional

Example:

- The following command selects nets and highlights them in the design display window:
  ```
  selectNet {net1 net2 net3}
  ```

Related Information

- `deselectNet`
- Floorplanning the Design chapter in the *Innovus User Guide*

selectPGPin

```
selectPGPin
[-help]
[-area llx lly urx ury]
[-net netName]
[-layer layerId | {layerIdList}]
[-all]
```

Selects power/ground pins as per the specified parameters and reports the number of selected power/ground pins. You can use this command after importing the design.

Parameters

`-all`	Selects all the power/ground pins. *Data_type*: bool, optional
`-area llx lly urx ury`	
	Specifies a bounding box for the area in which power/ground pins are to be selected. *Data_type*: box, optional
`-help`	Outputs a brief description that includes the type and default information for each `selectPGPin` parameter. For a detailed description of the command and all of its parameters, use the man command: `man selectPGPin.`
`-layer layerId | {layerIdList}`	
	Specifies the layer(s) on which the power/ground pins are to be selected. **Note:** Layers can be specified using the LEF layer names or layer ID numbers. *Data_type*: string, optional
`-net netName`	Specifies the name of the net whose attached power/ground pins are to be selected. *Data_type*: (net)+, optional

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*.

selectRouteBlk

```
selectRouteBlk
[-help]
[-box llx lly urx ury]
name
[-layer layerId | layerIdlist ...]
```

Selects the routing blockages based on the following specifications:

- The coordinates of the bounding box
- The name of the blockage
- The layer(s) or the via(s). The command supports selection on multiple layers and/or vias.

If you don't specify the box coordinates, [*llx lly urx ury*], the command selects the first routing blockage by name and layer.

Use this command during partition floorplanning.

Parameters

`-box llx lly urx ury`	
	Specifies the coordinates of the blockage area. • *llx*: Specifies the lower left x coordinate of the blockage box. • *lly*: Specifies the lower left y coordinate of the blockage box. • *urx*: Specifies the lower right x coordinate of the blockage box. • *ury*: Specifies the lower right y coordinate of the blockage box. *Data_type*: box, optional
`-help`	Outputs a brief description that includes the type and default information for each `selectRouteBlk` parameter. For a detailed description of the command and all of its parameters, use the man command: `man selectRouteBlk.`
`-layer layerId | layerIdlist ...`	
	Specifies the layer(s) and/or via(s). For layer names, specify the number. For example 3. For via names specify the lowercase character v followed by the via number. For example, v3. **Note:** The `-layer` parameter supports masterslice layers with type `REGION`. *Data_type*: string, optional
name	Specifies the name of the routing blockage. All the routing blockages with the specified name get selected. You can use wildcards (* or ?) for specifying the names of routing blockages. *Data_type*: string, required

Examples

- The following command selects all the routing blockages with the name blk.
  ```
  selectRouteBlk blk
  ```
- The following command selects the routing blockages with names blk1, blk2, and so on.
  ```
  selectRouteBlk blk*
  ```
- The following command selects the routing blockage blk1 that lies in the bounding box with the coordinates 17, 223, 1117, and 1223. The commands selects the routing blockage on layer 1 and layer 2.
  ```
  selectRouteBlk -box 17 223 1117 1223 blk1 -layer 1 2
  ```
- The following command selects the routing blockage blk1 on layer 3 and 4.
  ```
  selectRouteBlk blk1 -layer 3 4
  ```
- The following command selects the routing blockage blk2 that lies in the bounding box with the coordinates 17, 223, 1117, and 1223. The commands selects the routing blockage on layer1 and via 2.

```
selectRouteBlk -box 17 223 1117 1223 blk2 -layer 1 v2
```

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

set_macro_place_constraint

```
set_macro_place_constraint
[-help]
[array elements]
[-array array_name [-common_hier name][-valid_group_orients {R0 MX MY}]
  [-place_order {horizontal vertical}]]
[{-insts string | -all_macros } {-orientation string} [-common_hier name]]
[-cells string {-cell_obs string | -track_adjustment string}]
[-cpg string]
[[-max_io_pin_group_keep_out {depth_value [side_value]}]
[-forbidden_space_to_core {unified_value | horizontal_value vertical_value}]
[-forbidden_space_to_macro {unified_value | horizontal_value vertical_value}]
[-min_space_to_core {unified_value | horizontal_value vertical_value}]
[-min_space_to_macro {unified_value | horizontal_value vertical_value}]
[-parallel_run_length float]
[-horizontal_stacking {max_macro_stack_length min_space_between_macro_stack max_space_between_macros}]
[-vertical_stacking {max_macro_stack_length min_space_between_macro_stack max_space_between_macros}]
[-honor_strict_spacing_constraint {true | false}]
[-avoid_abut_macro_edge_with_pins {true | false}]
[-same_length_site integer]
[-pg_resource_model {M1 value1 M2 value2 …}]
[-macro_corner_keepout {double_endcap_width double_endcap_height}]
[-halo_sharing {true | false}]]
```

Enables you to specify the constraints for placing the macros and standard cells concurrently. These constraints can be used to place macros in an array (or matrix) and define the relative placement of each macro. The whole array, treated like one macro, can be flipped or mirrored (R0, MX, MY, R180). You can use the new `set_macro_place_constraint` command to specify:

- Macro array constraints
- Group aligning constraints
- Macro-to-macro and macro-to-core boundary spacing constraints
- Macro orientation constraints
- Over macro power routing modeling
- Maximum stacking length
- Reserved space for pin routing
- I/O Pin Keep-out

Note: For more information on the mixed placement constraints, see the "Mixed Place Constraints" section of the Using the Mixed Placer chapter in the *Innovus User Guide*.

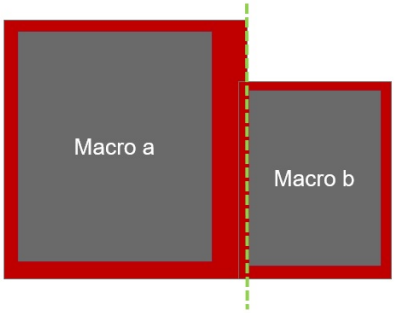
Parameters

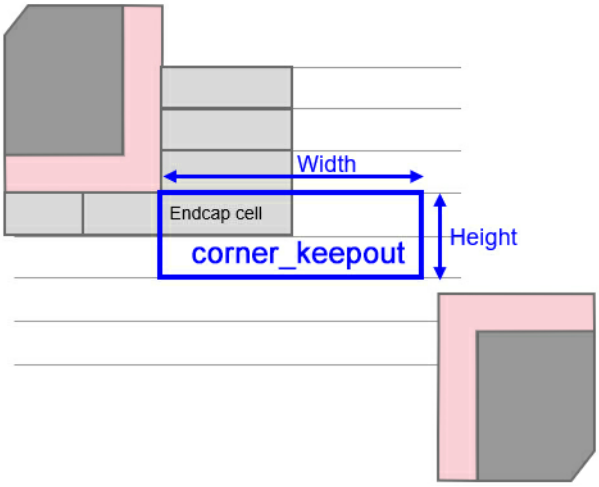
`-all_macros`	Specifies that all macros should honor the constraints. This option makes sense when using the `-orientation` option. *Data_type*: bool, optional
array_elements	Specifies the instance members information of a row or column of the array in the following format: *inst_name*:*orientation* [*spacing inst_name*:*orientation*] + **Note:** The *array_elements* should be specified with the `-array` parameter. *Data_type*: string, optional
`-array` *array_name*	Specifies the macro array name. *Data_type*: string, optional

`-avoid_abut_macro_edge_with_pins {true | false}`	When set to true, the `set_macro_place_constraint` command does not abut macro edges with pins even if the macro distance is within the forbidden space setting. Macros will be separated by the `min_space_to_macro` distance. *Default*: `true` *Data_type*: enum, optional
`-cell_obs {{Layer1_Name PowerStripeWidth SpacingBetweenStripes} {Layer2_Name ….}}`	

	Specifies the layer width and spacing to add cell obstruction on each macro to model power stripes . During the concurrent macro placement the power mesh does not exist. This parameter allows to mimic the number of routing tracks occupied by the real power mesh over the macros to make the congestion analysis more accurate during the concurrent macro placement.
	The format is:
	`cell_obs{{Layer1_Name PowerStripeWidth SpacingBetweenStripes} {Layer2_Name ….}}`
	Example:
	`set_macro_place_constraint -cells <cell_name>  -cell_obs {{M5 0.24 1.251} {M8 0.64 8.409} {M9 1.2 7.49} }`
	Note: This setting is generated automatically by the `create_pg_model_for_macro_place` command. It is highly recommended to use this command to define the `-cell_obs` parameter.
	Data_type: string, optional

`-cells string`	Used in combination with the -cell_obs command to specifies the constraints for all instances of the specified cells. *Data_type*: string, optional
`-common_hier name`	Specifies the common logic hierarchy name for `inst_name` in `array_elements`, so the `inst_name` can be specified with short-hand format. *Data_type*: string, optional
`-cpg string`	Specifies the cpg guide of one instance. *Data_type*: string, optional
`-forbidden_space_to_core {unified_value | horizontal_value vertical_value}`	
	Specifies the value for forbidden spacing between macro to core boundary. If the space between the macro and core is smaller than this value, then the macros are abutted. **Note:** • If only one unified value is specified, both horizontal and vertical spacings use the same value. If two values are specified, the first value is for the horizontal spacing, the second value is for the vertical spacing. • In case the `-min_space_to_core` parameter has been specified, then the value specified for the `-forbidden_space_to_core` parameter should be less than the value specified for the `-min_space_to_core` parameter. *Minimum=0* *Maximum=100000* *Default:* `0` *Data_type*: string, optional
`-forbidden_space_to_macro {unified_value | horizontal_value vertical_value}`	

	Specifies the value for forbidden spacing between macro to macro boundary. If the space between the macros is smaller than this value, then the macros are abutted. **Note:** • If only one unified value is specified, both horizontal and vertical spacings use the same value. If two values are specified, the first value is for the horizontal spacing, the second value is for the vertical spacing. • In case the `-min_space_to_macro` parameter has been specified, then the value specified for the `-forbidden_space_to_macro` parameter should be less than the value specified for the `-min_space_to_macro` parameter. *Minimum=0* *Maximum=100000* *Default:* `0` *Data_type*: string, optional
`-halo_sharing {true | false}`	When enabled, abutted macros can share halos between them. **Note:** When the spacing between two macros is less than the forbidden-space-to-macro, the two macros are abutted. Since a single halo width instead of double width is enough to meet the physical rules, abutted macros can share halos between them using the new `-halo_sharing` parameter. **Note:** The halos of the abutted macros can overlap but macros cannot overlap with the halo of the other macro.  *Default:* `false` *Data_type*: enum, optional
`-help`	Outputs a brief description that includes the type and default information for each `set_macro_place_constraint` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_macro_place_constraint`
`-honor_strict_spacing_constraint{true | false}`	
	When set to true, it specifies that the forbidden spacing and minimum spacing constraints must be honored strictly. *Data_type*: enum, optional
`-horizontal_stacking {`*`max_macro_stack_length  min_space_between_macro_stack  max_space_between_macros`*`}`	

	Specifies the maximum horizontal stack depth and the minimum horizontal spacing. • *max_macro_stack_length:* Specifies that the macro stacking length should not be more than the specified value. • *min_space_between_macro_stack:* Specifies the spacing between different macro stacks. • *max_space_between_macros*: Specifies the maximum space between macros. When the macro spacing is less than the specified value, the macros are counted in the stacking length. For example, if the max_space_between_macro value is 9 then the macros will be considered as packed if the space between the macros is between 0 and 9, however, if the real space is 10 then nothing will happened. *Data_type*: string, optional
`-insts string`	Specifies the macros on which the orientation constraints are set. Refer to the `-orientation` for more information.
`-macro_corner_keepout {double_endcap_width double_endcap_height}`	
	Specifies the macro corner spacing constraints. It ensures that there is enough space between the macro corners for inserting the endcap cells properly. **Note:** The corner keepout area starts at the corner of the macro halo.  **Note:** The macro corner spacing constraints specified with this parameter are honored by the `refine_macro_place` and `check_macro_place_constraint` commands. However, if honoring the corner keepout constraint is difficult, corner abutting is allowed. **Note:** The tool reserves either the horizontal keepout larger than double corner cell width or vertical keepout larger than double corner cell height *Default*: {0,0} *Data_type*: string, optional

`-max_io_pin_group_keep_out {depth_value [side_value]}`	Specifies the maximum height and (optionally) side extension to automatically add macro-only blockages to IO pins.
	Note: The `-max_io_pin_group_keep_out` parameter automatically coordinates the size of the macro-only blockages based on the pin density and number for each group of IO pins.
	Note: Side value is optional. If you do not specify the side value, the `set_macro_place_constraint` command follows the default method to calculate the side (side = ¼ depth). If you specify the side value, the max IO group needs to follow the specified depth and side distance. The blockages of smaller IO groups will be derived automatically, keeping the same *depth/side* proportion (as described for the max IO group)
	Innovus supports the following formats:
	`set_macro_place_constraint -max_io_pin_group_keep_out depth_value`
	`set_macro_place_constraint -max_io_pin_group_keep_out {depth_value}`
	`set_macro_place_constraint -max_io_pin_group_keep_out {depth_value side_value}`
	Note: The depth and side values are specified in microns.
	Minimum=0
	Maximum=100000
	Data_type: string, optional

`-min_space_to_core {unified_value | horizontal_value vertical_value}`	
	Specifies the value for the minimum spacing between macro and core boundary. If the space between the macro and core is smaller than this value and larger than the forbidden space, then the macros are pushed away from the core to minimum space. **Note:** If only one unified value is specified, both horizontal and vertical spacings use the same value. If two values are specified, the first value is for the horizontal spacing, the second value is for the vertical spacing. *Minimum=0* *Maximum=100000* *Default:* `0` *Data_type*: string, optional
`-min_space_to_macro {unified_value | horizontal_value vertical_value}`	

	Specifies the value for the minimum spacing between macro to macro boundary. If the space between the macros is less than this value, then the macros are aligned.
	Note: If only one unified value is specified, both horizontal and vertical spacings use the same value. If two values are specified, the first value is for the horizontal spacing, the second value is for the vertical spacing.
	For Example:
	• In the following command, both horizontal and vertical spacing use the same value.
	`set_macro_place_constraint -min_space_to_macro 10`
	• In the following command, the first value is for horizontal spacing, the second value is for vertical spacing.
	`set_macro_place_constraint -min_space_to_macro {10 20}`
	Minimum=0
	Maximum=100000
	Data_type: string, optional

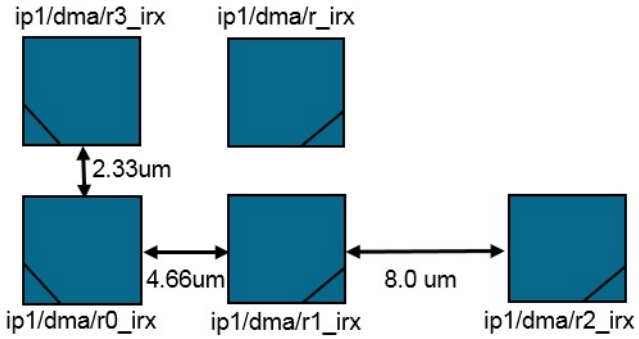
`-orientation string`	Specifies the allowed orientation for the macros. The macro instances on which you want to constrain the orientations can be specified by -insts or -all_macros **Note:** Specified orientation overwrites the previous definition. Example: `set_macro_place_constraint -all_macros -orientation {R90 MX90 MY90 R270}` *Data_type*: string, optional

`-parallel_run_length` *float*	Specifies the parallel run length to enable space checking. When the parallel run length of macros is larger than the specified parallel run length value, macro-to-macro space checking will be enabled. **Note:** When the `-parallel_run_length` parameter is specified along with the `-horizontal_stacking` or `-vertical_stacking` parameters, only the macros with parallel run length >= the specified value will be counted in the stacking length. *Minimum=0* *Maximum=100000* *Data_type*: float, optional
`-pg_resource_model {`*M1 value1 M2 value2 …*`}`	
	Specifies the pg resource model for concurrent macro placement. During the concurrent macro placement the power mesh does not exist. This parameter allows you to mimic the number of routing tracks occupied by the real power mesh to make the congestion analysis more accurate during the concurrent macro placement. **Note:** This setting is generated automatically by the `create_pg_model_for_macro_place` command. It is highly recommended to use this command to define the `-pg_resource_model` parameter. *Data_type*: string, optional
`-place_order {horizontal | vertical}`	Specifies the place order in row (horizontal) or column (vertical) first for macro array. *Default*: `horizontal` *Data_type*: enum, optional
`-same_length_site` *integer*	Specifies the minimum number of sites between the vertical edge of two macros. *Minimum=0* *Maximum=100000* *Data_type*: int, optional
`-track_adjustment {{`*layer1 density1*`} {`*layer2 ….*`}}…`	Specifies the layer density to add track adjustment on each macro. For each routing layer and each macro, if the PG density over a macro is lower than the standard cell area then it increases the track resource in macro area to mimic the lower PG density. **Note:** This setting is generated automatically by the `create_pg_model_for_macro_place` command. It is highly recommended to use this command to define the `-cell_obs` parameter. *Data_type*: string, optional
`-valid_group_orients {R0 MX MY}`	Specifies the valid orientations for the whole group. *Data_type*: enum, optional
`-vertical_stacking {`*max_macro_stack_length min_space_between_macro_stack max_space_between_macros*`}`	
	Specifies the maximum vertical stack depth and the minimum vertical spacing. • *max_macro_stack_length:* Specifies that the macro stacking length should not be more than the specified value. • *min_space_between_macro_stack:* Specifies the spacing between different macro stacks. • *max_space_between_macros*: Specifies the maximum space between macros. When the macro spacing is less than the specified value, the macros are counted in the stacking length. For example, if the max_space_between_macro value is 9 then the macros will be considered as packed if the space between the macros is between 0 and 9, however, if the real space is 10 then nothing will happened. *Data_type*: string, optional

Example

- The following command creates a macro array, ram_array, and defines it constraints.

```
set_macro_place_constraint -array ram_array {{ip1/dma/r0_irx:R0 4.66 ip1/dma/r1_irx:MY 8.0 ip1/dma/r2_irx:MY} 2.33 {ip1/dma/r3_irx:R0 4.66 ip1/dma/r_irx:MY}} -valid_group_orients {R0 MY}
```



Related Information

- check_macro_place_constraint
- create_pg_model_for_macro_place
- get_macro_place_constraint
- reset_macro_place_constraint
- write_macro_place_constraint
- refine_macro_place
- Floorplanning the Design chapter in the *Innovus User Guide*
- Using the Mixed Placer chapter in the *Innovus User Guide*

set_trace_obj_connectivity_mode

```
set_trace_obj_connectivity_mode
[-help]
[-reset]
[-macro_pins list_of_pins]
[-max_fanin_fanout number]
[-register_inputs list_of_pins]
[-register_outputs list_of_pins]
```

Controls certain aspects of how the connections of specified macros, selected macros, or specified ports are traced. It enables you to set the trace mode such as register pins, macro pins, and maximum fanin and fanout. Use the `get_trace_obj_connectivity_mode` command to return the current settings for the `set_trace_obj_connectivity_mode` command. The `set_trace_obj_connectivity_mode` parameters affects the behavior the `trace_obj_connectivity` command. You can use this command after importing the design and before using the `trace_obj_connectivity` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_trace_obj_connectivity_mode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man` set_trace_obj_connectivity_mode
`-macro_pins` *list_of_pins*	Specifies the macro pin name patterns. *Default*: " " *Data_type*: string
`-max_fanin_fanout` *number*	Specifies the maximum fanin/fanout number for a pin. While tracing the connectivity, a pin will be ignored if its fanin/fanout number is larger than the specified number. *Default*: 1000 *Data_type*: int
`-register_inputs` *list_of_pins*	Specifies the register input pin names. *Default*: " " *Data_type*: string
`-register_outputs` *list_of_pins*	Specifies the register output pin names. *Default*: " " *Data_type*: string
`-reset`	Restores all parameters modified using `set_trace_obj_connectivity_mode` to their default settings. The `-reset` parameter must be the first parameter specified. If you specify -reset by itself, the software resets all `set_trace_obj_connectivity_mode` parameters to their default values. If you specify parameters after -reset, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported.

Example

The following command set the maximum fanin/fanout number for pins to 200:

```
set_trace_obj_connectivity_mode -max_fanin_fanout 200
```

Related Information

- `trace_obj_connectivity`
- `get_trace_obj_connectivity_mode`
- `display_obj_connectivity`
- `all_fanin`
- `all_fanout`
- Floorplanning the Design chapter in the *Innovus User Guide*
- Floorplan Menu chapter in the *Innovus Menu Reference*

setBottomIoPadOrient

```
setBottomIoPadOrient
[-help]
orient
```

Changes the orientation for the bottom pad (South side). Changing the bottom pad orientation changes the pad relationship of the East, North, and West side pads. You can use this command after importing the design.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `setBottomIoPadOrient` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setBottomIoPadOrient`
orient	Specifies the orientation. The possible orientation values are R0, R90, R180, and R270. *Data_type*: string, required

Example

The following command changes the orientation of the bottom I/O pad instances from R0 to R270 for the bottom pad (South side):

```
setBottomIoPadOrient R270
```

As a result, the respective orientation of the I/O pad instances change on the East (R270 to R180), North (R180 to R90), and West (R90 to R0) sides.

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

setDrawView

```
setDrawView
[-help]
mode
```

Sets the design view in the design display area. You can use this command after importing or restoring the design. To see the current view use `getDrawView`.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `setdrawView` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setDrawView`
mode	Sets the mode. • `amoeba`: Sets the view to Amoeba view. This view displays the outline of the modules and submodules after placement, showing physical locality of the module. • `fplan`: Sets the view to floorplan view. This view displays the hierarchical module and block guides, connection flightlines, and floorplan objects, including block placement, and power and ground nets. • `place`: Sets the view to physical view. This view displays the detailed placements of the module's blocks, standard cells, nets, and interconnects. *Data_type*: string, required

Related Information

- `getDrawView`
- Floorplanning the Design chapter in the *Innovus User Guide*

setFinishFPlanMode

```
setFinishFPlanMode
[-help]
[-reset]
[-abuttedEdgeAsNonRowArea {true|false}]
[-activeObj {macro|macroHalo|core|iopad|iocell|fence|hardblkg|softblkg|partialblkg|routeblkg|routingblkg|row}]
[-direction {x|y|xy}]
[-drcRegionObj {macro|macroHalo|hardBlkg|minGap|coreSpacing|nonRowArea}]
[-override {true|false}]
```

Set the active objects and specifies the channel direction for the `finishFloorplan` command to use whenever you perform advanced placement-related refinements to a floorplan. Use the `getFinishFPlanMode` command to return the current settings for the `setFinishFPlanMode` command. The `setFinishFPlanMode` parameters affects the behavior the `finishFloorplan` command.

Parameters

`-abuttedEdgeAsNonRowArea {true|false}`

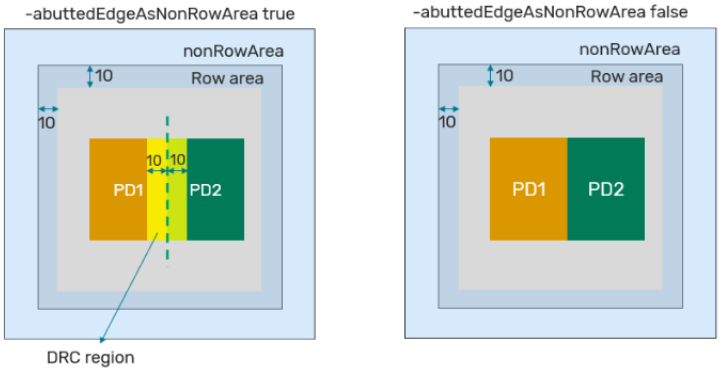
Specifies whether the abutted edge of a power domain should treated be as nonRowArea.

- `true`: The abutted edge of the power domain is treated as nonRowArea with size 0 by default. The tool creates a
- `false`: The abutted edge of the power domain is excluded from nonRowArea. The tool then does not create a DR

Note: This option can only be specified when the `-drcRegionObj nonRowArea` parameter option is also specified.

For example:

```
setFinishFPlanMode -drcRegionObj nonRowArea -abuttedEdgeAsNonRowArea {true|false}
finishFloorplan -drcRegionLayer M1 -edgeExtend {10 10}
```



Default: `true`

Data_type: bool

`-activeObj {macro|macroHalo|core|iopad|iocell|fence|hardblkg|softblkg|partialblkg|routeblkg|routingblkg|row}`

	Specifies the active object list. The boundaries of the specified active objects are looked at while deciding where to objects can be: • `core` • `macro` • `macroHalo` • `ioCell` • `ioPad` • `fence` • `hardblkg` • `softblkg` • `partialblkg` • `routeblkg` • `routingblkg` • `row` *Default*: `core macro macroHalo` *Data_type*: enums={macro macroHalo core iopad iocell fence hardblkg softblkg partialblkg routeblkg row}_list
`-help`	Outputs a brief description that includes the type and default information for each `setFinishFPlanMode` parameter. For a detailed description of the command and all of its parameters, use the man command `man setFinishFPlanMod`
`-direction {x | y | xy}`	Specifies the direction in which the blockage is added. You specify one of the following options: • `x`: Fills the channel whose height is less than width. • `y`: Fills the channel whose width is less than height. • `xy`: Fills every channel whose smallest size (in X or Y direction) is within Max Gap range. *Default*: `xy` *Data_type*: enums={x y xy}
`-drcRegionObj {macro|macroHalo|hardBlkg|minGap|coreSpacing|nonRowArea}`	
	Specifies the drc region object list. The boundaries of the specified drc region objects are looked at while deciding w • `coreSpacing` • `hardBlkg` • `macro` • `macroHalo` • `minGap` • `nonRowArea` *Default*: `macro macroHalo hardBlkg minGap coreSpacing` *Data_type*: enums={macro macroHalo hardBlkg minGap coreSpacing nonRowArea}_list
`-override {true | false}`	
	Overrides old blockages that overlap with the new blockages. *Default*: `false` *Data_type*: bool
`-reset`	Restores all parameters modified using `setFinishFPlanMode` to their default settings. The `-reset` parameter must be to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their defa

Example

- The following commands specify the region layer name list for creating a region layer for the specified DRC region object:

```
setFinishFPlanMode -drcRegionObj macro
finishFloorplan -drcRegionLayer R1
```

Related Information

- `finishFloorplan`
- `getFinishFPlanMode`
- "Finishing the Floorplan" in the *Innovus User Guide*
- "Finish Floorplan" in the *Innovus Menu Reference*

setFixedBlockSize

```
setFixedBlockSize
[-help]
name_list
```

Specifies that during floorplan resizing, the size of the specified modules and/or blackboxes should not change. Use this command before specifying the floorplan.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `setFixedBlockSize` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setFixedBlockSize`
`name_list`	Specifies that the size of the *specified* modules and/or blackboxes should not change during floorplan resizing. Provide a space-separated list of instances. *: Specifies that the size of *all* the modules and/or blackboxes should not change during floorplan resizing. *Data_type*: list, required

Example

The following command specifies that the size of `instanceA` and `instanceB` should not change during floorplan resizing.

```
setFixedBlockSize instanceA instanceB
```

Related Information

- `unsetFixedBlockSize`
- Floorplanning the Design chapter in the *Innovus User Guide*

setFlipping

```
setFlipping
[-help]
flip_type
```

Specifies the orientation of the bottom row in the core area. The command recreates all the floorplan rows. You can use this command after importing the design.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `setFlipping` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setFlipping`
flip_type	Specifies the type of orientation. • `f`: Specifies that the first row flips from the bottom up. • `n`: Specifies that no row flipping is done. • `s`: Specifies that the second row flips from the bottom up. **Tip:** To get the flipping value set by the `setFlipping` command, use the following: `dbGet top.fplan.flipRows` *Data_type*: string, required

Examples

- If the existing orientation of the bottom row is Mx and the second row from bottom is R0, then the following command changes the second row orientation to Mx and changes the orientation of the other rows accordingly (Bottom row orientation will be R0).
  ```
  setFlipping s
  ```
- If the existing orientation of the bottom row is R0 and second row from bottom is Mx, then the following command changes the first row orientation to Mx and changes the orientation of the other rows accordingly (second from bottom row orientation will be R0):
  ```
  setFlipping f
  ```

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

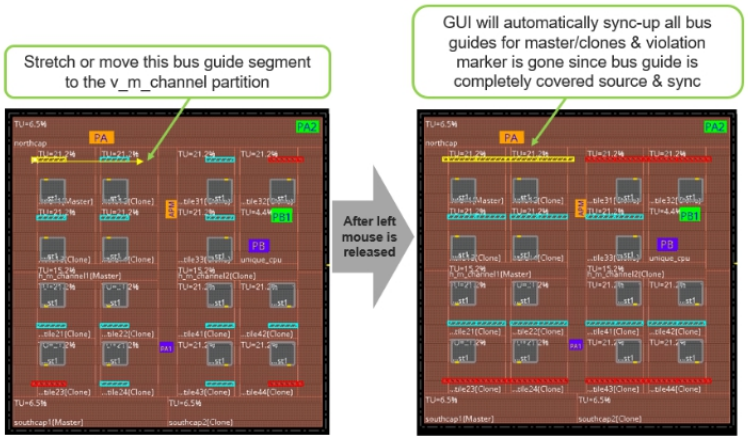
setFPlanMode

```
setFPlanMode
[-help] [-reset]
[-autoSyncMasterClone {busGuides|constraints|stdcells|macros}]
[-cellsForExtraSites cell_name]
[-checkTypes {basic|fence|oddEvenSiteRow|macroPin|color|alignmentFollowingPin|alignmentPartitionClone|
  busGuideConnectivity|feedThrough|partitionInPartition|multiLayerPin|powerDomain|
  partition|place|sameLengthSite|narrowChannel|blockOnly|unifiedVddOnBottom|all}]
[-cutOffPlaceBlockageOutsideDie {true|false}]
[-cutOffRouteBlockageOutsideDie {true|false}]
[-defaultBlockageNamePrefix string]
[-defaultPowerDomainSite {true|false}]
[-defaultRowPatternSite string]
[-defaultTechSite string]
[-enableRectilinearDesign {true|false}]
[-extraSites site_name]
[-firstRowSiteIndex int]
[-includeIoWhenInitArea {true|false}]
[-initAllCompatibleCoreSiteRows {true|false}]
[-keepRowsWhenMovingPowerDomain {true|false}]
[-lastRowSiteIndex int]
[-maxIoHeight {true|false}]
[-minimumSites value]
[-move_macros_with_constraint {guide|region|fence|none|all}]
[-move_preplaced_std_cell_only {true|false}]
[-move_std_cell_with_constraint {guide|region|fence|none|all}]
[-narrowChannelThreshold value]
[-no_cut_row {true|false}]
[-powerRailLayer string]
[-rowHeightIncrementCornerToCorner value]
[-rowHeightIncrementIncornerToCorner value]
[-rowHeightIncrementIncornerToIncorner value]
[-rowHeightMultiple value]
[-rowSiteHeight {any|odd|even|computed}]
[-rowSiteWidth {any|odd|even}]
[-skipViolations string]
[-snap_all_corners_to_grid {true|false}]
[-snapBlockGrid {manufacturing|inst|placement|userDefine|LayerTrack|finfetInst|finfetManufacturing|finfetPlacement}]
[-snapConstraintGrid {manufacturing|inst|placement|userDefine|LayerTrack|finfetInst|finfetManufacturing|finfetPlacement}]
[-snapCoreGrid {manufacturing|inst|placement|userDefine|LayerTrack|finfetInst|finfetManufacturing|finfetPlacement}]
[-snapDieGrid {manufacturing|inst|placement|userDefine|LayerTrack|finfetInst|finfetManufacturing|finfetPlacement}]
[-snapIoGrid {manufacturing|inst|placement|userDefine|LayerTrack|finfetInst|finfetManufacturing|finfetPlacement}]
[-snapPlaceBlockageGrid {manufacturing|inst|finfetInst|finfetManufacturing|userDefine}]
[-snapPlaceBlockageType {closest|upsize|sequential}]
[-user_define_grid {x_offset y_offset x_pitch y_pitch}]
```

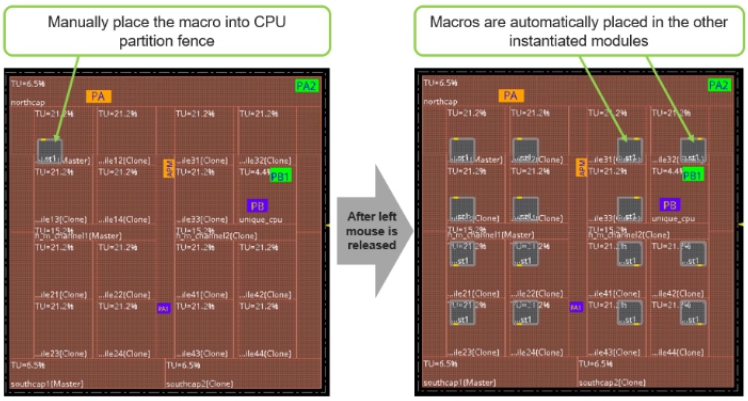
Sets the global parameters of the check floorplan feature. Use the `getFPlanMode` command to return the current settings for the `setFPlanMode` command. The `setFPlanMode` parameters affects the behavior the `checkFPlan` command. You can use this command after importing the design and before using the `checkFPlan` command.

Parameters

`-autoSyncMasterClone {busGuides|constraints|stdcells|macros}`	
	Specifies the objects that automatically sync up between master and clone partitions during add, delete, edit, move, flip, rotate, unplace, or stretch operations. The sync-up object can be bus guides, standard cells, macros, constraints (fence/guide/region), placement blockages, routing blockages, or special wires. • `busGuides`: Any change (add, delete, edit, move, or stretch) for busGuides in master will sync-up to other hinsts that have the same module. For example, `setFPlanMode -autoSyncMasterClone {busGuides}`



- `constraints`: Any change (add, delete, move, reshape) in fence/guide/region of master will sync-up to other hinsts that have the same module.
  - When constraints is not specified, tool will not sync-up any change of fence/guide/region between master and clones.
  - When constraints is specified, tool will auto sync-up any change of fence/guide/region between master and clones when doing moving/stretching/add/delete/edit.
- `stdcells`: Any change (add, delete, edit, move, flip, rotate, or unplace) in standard cells will sync-up to other hinsts that have the same module.
  For example, `setFPlanMode -autoSyncMasterClone {stdcells}`
- `macros`: Any change (add, delete, edit, move, flip, rotate, or unplace) in hard macros in master will sync-up to other hinsts that have the same module.
  For example, `setFPlanMode -autoSyncMasterClone {macros}`



Default: `" "`

Data_type: `enums={busGuides constraints stdcells macros}_list`

`-cellsForExtraSites` *cell_name*	Specifies the cell names for which the additional sites will be created. These sites can be used up later in the flow. *Default*: `" "` *Data_type:* string

```
-checkTypes {basic|fence|oddEvenSiteRow|macroPin|color|alignmentFollowingPin|alignmentPartitionClone|
  busGuideConnectivity|feedThrough|partitionInPartition|multiLayerPin|powerDomain|
  partition|place|sameLengthSite|narrowChannel|blockOnly|unifiedVddOnBottom|all}
```

	Specifies the check types for the `checkFPlan` command. • `basic`: Checks the basic quality of the floorplan to detect potential problems before the design is passed on to other tools. • `fence`: Checks the fences. • `oddEvenSiteRow`: Checks odd or even site row. It checks if the current row fits to the setting of `setFPlanMode -rowSiteWidth` and `-rowSiteHeight`. • `macroPin`: Checks if the macro pins are off track. • `color`: Checks the color issues of all floorplan objects, such as pins, preRoutes, prePlacedInst, etc. • `alignmentFollowingPin`: Checks if the centerline of the followpin is the same as the row's border and matches the power attribute. • `alignmentPartitionClone`: Checks that partition clones are aligned with their master on the power mesh. This checks that the master and clones partitions are inside a core boundary, and ensures that the partition fence instances sizes match the master instance. • `busGuideConnectivity`: Checks incorrect overlapping and connectivity. • `feedThrough`: Checks feedthrough buffer insertion. • `partitionInPartition`: Checks if child partitions reside inside parent partitions. • `multiLayerPin`: Checks multi layer pins. • `powerDomain`: Checks the power domains. • `partition`: Checks the partitions. • `place`: Checks the placement. • `sameLengthSite`: Checks the same length site rule where, in a group of continuous poly stripes with the same length, the number of sites must be >= N.(N is specified by the `-minimumSites` option.) • `narrowChannel`: Checks for narrow channels. • `blockOnly`: Checks for performance most frequently used in block implementation. Includes: basic, oddEvenSiteRow, macroPin, and color. • `unifiedVddOnBottom`: Checks whether all the cells in the design are with the same vdd on bottom. • `all`: Performs all checks on the floorplan to detect potential problems before the design is passed on to other tools. *Default*: `basic` *Data_type:* enums={basic fence oddEvenSiteRow macroPin color alignmentFollowingPin alignmentPartitionClone busGuideConnectivity feedThrough partitionInPartition multiLayerPin powerDomain partition place sameLengthSite narrowChannel blockOnly unifiedVddOnBottom all}_list
`-cutOffPlaceBlockageOutsideDie {true|false}`	When creating placement blockages, it cuts the part outside of die and snaps it by default. *Default*: `true` *Data_type:* bool
`-cutOffRouteBlockageOutsideDie {true|false}`	When creating routing blockages around inst/Hinst/Partition, it cuts the part outside of die and snaps it by default. If this option is set to false, then the outside part is not cut and snapped. *Default*: `true` *Data_type:* bool
`-defaultBlockageNamePrefix` *string*	Specifies the name prefix for placement blockage and routing blockage. *Default*: " " *Data_type:* string

`-defaultPowerDomainSite {true | false }`	Creates rows based on the default power domain site instead of the the default site of the design. *Default*: `false` *Data_type:* bool
`-defaultRowPatternSite string`	Specifies the default ROWPATTERN site. **Note:** Defined in the LEF, a ROWPATTERN site is a set of previously defined sites and their orientations. This site is defines a placement site in the design. *Default*: " " *Data_type:* string
`-defaultTechSite string`	Specifies a site as default technical site. **Note**: After setting a default technical site, the auto-calculating default-technical-site functionality is disabled. **Note**: The `saveFPlan`, `loadFPlan`, `initCoreRow` and `checkFPlan` commands honor the default technical site specified using the `setFPlanMode -defaultTechSite` parameter. *Default*: auto-calculated default-technical-site by tool. *Data_type:* string
`-enableRectilinearDesign {true | false}`	Specifies whether the design boundary can be changed to rectilinear shape. *Default*: `false` *Data_type:* bool
`-extraSites site_name`	Specifies the name for the additional sites that will be created for cells which are not used in the netlist by default. These sites can be used up later in the flow. **Note**: Sites for all specified cells should be defined in the LEF. *Default*: `" "` *Data_type:* string
`-firstRowSiteIndex int`	Specifies the index number in default row site pattern as the first row for creating hybrid rows. For example, *Default*: 0 *Data_type:* int
`-help`	Outputs a brief description that includes the type and default information for each `setFPlanMode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setFPlanMode.`
`-includeIoWhenInitArea {true|false}`	Includes IO cells when calculating the area of modules. *Default*: `false` *Data_type:* bool
`-initAllCompatibleCoreSiteRows {true|false}`	When enabled, initCoreRow creates SITES which are used by all cells in the library. *Default*: `false` *Data_type:* bool
`-keepRowsWhenMovingPowerDomain {true|false}`	Specifies whether to keep all existing rows in power domain when moving, resizing or reshaping a power domain on GUI or by command. *Default*: `false` *Data_type:* bool
`-lastRowSiteIndex int`	Specifies the index number in default row site pattern as the last row for creating hybrid rows. *Default*: -1 *Data_type:* int

`-maxIoHeight {true|false}`	When specified, the maximum IO height is used to calculate a die box with IO placement. By default, the minimum IO height is used. *Default*: `false` *Data_type:* bool
`-minimumSites` *value*	Specifies the number of minimal sites (N) that the `checkFPlan` command will check for the same length site rule. When the `sameLengthSite` option of the `-checkTypes` parameter is specified, this parameter is required. *Default*: `0` *Data_type:* int
`-move_macros_with_constraint {guide|region|fence|none|all}`	
	Specifies that the macros move with the constraint modules at the same time. *Default*: `fence` *Data_type:* `enums={guide region fence none all}_list`
`-move_preplaced_std_cell_only {true|false}`	Specifies that only the pre-placed standard cells can move with the constraint modules at the same time. *Default*: `false` *Data_type:* bool
`-move_std_cell_with_constraint {guide|region|fence|none|all}`	
	Specifies that standard cells move with the constraint modules at the same time. *Default*: `none` *Data_type:* `enums={guide region fence none all}_list`
`-narrowChannelThreshold` *value*	Reports the narrow channels whose width (in micros) is smaller than the specified value. When the `narrowChannel` option of the `-checkTypes` parameter is specified, this parameter is required. *Default*: `0` *Data_type:* float
`-no_cut_row {true|false}`	Specifies that the rows are not required to be cut before other operations. The following commands honor this mode setting: • `createRouteBlkAlongTrack` • `check_ufc` • `fix_floorplan` • `finishFloorplan` With the `-no_cut_row` parameter set to `false` or not specified, the tool will run above the mentioned commands based on the current row shape. So the rows for objects like insts, place blockages are expected to be cut out before running these commands to get expected results. If rows are not cut before run these commands are run and the `-no_cut_row` parameter is set to `true`, the tool will identify the rows that have not been cut before running these commands. It calls the `cutRow` command to get a cut row shape during these commands action. The `cutRow` command cuts the rows for insts, hinsts, place blockages, route blockages, power domains, groups, or bumps by default. These commands will work based on the cut row shape. But the actual row shape is not changed after these operations. *Default*: `false` *Data_type:* bool
`-powerRailLayer` *string*	Specifies the layers used for calculating the power/ground on the bottom attribute of the cell or techsite. By default, the lowest metal layer is used. *Default*: " " *Data_type:* string

`-reset`	Restores all parameters modified using `setFPlanMode` to their default settings. The `-reset` parameter must be the first parameter specified. If you specify -reset by itself, the software resets all `setFPlanMode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported.
`-rowHeightIncrementCornerToCorner` *`CornerToCornerValue`*	
	Specifies the offset value for user defined row height constraint with corner to corner edge. *Default*: 0 *Data_type:* int
`-rowHeightIncrementIncornerToCorner` *`IncornerToCornerValue`*	
	Specifies the offset value for user defined row height constraint with incorner to corner edge. *Default*: 0 *Data_type:* int
`-rowHeightIncrementIncornerToIncorner` *`IncornerToIncornerValue`*	
	Specifies the offset value for user defined row height constraint with incorner to incorner edge. *Default*: 0 *Data_type:* int
`-rowHeightMultiple` *`multipleHeightValue`*	Specifies the pitch value for user defined row height constraint. *Default*: 0 *Data_type:* int
`-rowSiteHeight {any|odd|even|computed}`	Specifies the odd, even, or user defined row height constraints. **Note:** For a hybrid row design, specify the `computed` option for the `-rowSiteHeight` parameter to specify the row constraints. *Default*: `any` *Data_type:* enums={any odd even computed}
`-rowSiteWidth {any|odd|even}`	Specifies the odd even row width constraints. *Data_type:* enums={any odd even}

`-skipViolations` *`string`*	Skips reporting and showing the violation(s) associated with the specified message ID(s) in the violation browser.
	Note: You can specify a list of message IDs with the `-skipViolations` parameter. It also supports wildcards.
	• If the `-skipViolations` parameter is not specified, the tool reports all the violations checked by the `checkFPlan` command in the terminal/.log/.logv and shows them in the violation browser if there is marker box.
	• If the `-skipViolations` parameter is specified, the tool does not report the violations in terminal/.log/.logv and also does not show them in the violation browser even through there is marker box.
	For example, the following command does not report the violations for IMPFP-7236.
	`setFPlanMode -checkTypes basic -skipViolations IMPFP-7236`
	`checkFPlan`
	Default: `" "`
	Data_type: string

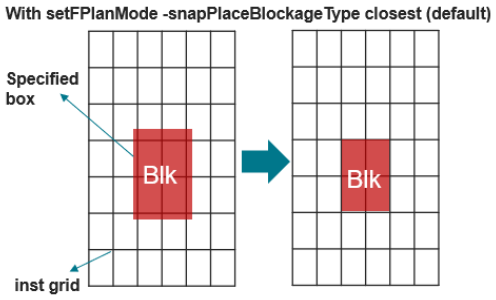
`-snap_all_corners_to_grid {true|false}`	Snaps all corners of an object to the grid. It defines the snapping behavior for the kinds of objects: constraint, ptnCut and black box. **Note**: This option only works when the snapping grid is user defined grid or FinFET grid. • When the snapping grid is ManufactureGrid, the object is snapped to the closest grid. • When the snapping grid is InstanceGrid and PlacementGrid, the object is moved to first snap the LL corner to the closest location and then to snap the UR corner to closest location. • When the snapping grid is user defined grid, the object is moved to snap the LL corner to the closest location. Then if the `-snap_all_corners_to_grid` parameter set to true, the object is moved to snap the UR corner to the closest location. • When the snapping grid is FinFET grid and the `-snap_all_corners_to_grid` parameter set to true, all corners of the object are snapped to the closest grid. However, if the `-snap_all_corners_to_grid` parameter set to false, only the LL corner of the object is snapped to the closest location. *Default*: `false` *Data_type:* bool
`-snapBlockGrid {manufacturing | inst | placement | userDefine | LayerTrack | finfetInst | finfetManufacturing | finfetPlacement }`	
	Specifies the block snap rules. It can be set to snap to: • `manufacturing`: Manufacture Grid • `inst`: Instance Grid • `placement`: Placement Grid • `userDefine`: User-define Grid • `LayerTrack`: LayerTrack Grid • `finfetManufacturing`: FinFET Manufacture Grid • `finfetInst`: FinFET Instance Grid • `finfetPlacement`: FinFET Placement Grid *Default*: `manufacturing` **Note**: For FinFET designs, the default is `finfetManufacturing`. *Data_type:* enums={manufacturing inst placement userDefine LayerTrack finfetInst finfetManufacturing finfetPlacement}
`-snapConstraintGrid {manufacturing | inst | placement | userDefine | LayerTrack | finfetInst | finfetManufacturing | finfetPlacement }`	
	Specify the constraint snap rules. It can be set to snap to: • `manufacturing`: Manufacture Grid • `inst`: Instance Grid • `placement`: Placement Grid • `userDefine`: User-define Grid • `LayerTrack`: LayerTrack Grid • `finfetManufacturing`: FinFET Manufacture Grid • `finfetInst`: FinFET Instance Grid • `finfetPlacement`: FinFET Placement Grid *Default*: `inst` **Note**: For FinFET designs, the default is `finfetInst`. *Data_type:* enums={manufacturing inst placement userDefine LayerTrack finfetInst finfetManufacturing finfetPlacement}
`-snapCoreGrid {manufacturing | inst | placement | userDefine | LayerTrack | finfetInst | finfetManufacturing | finfetPlacement }`	

	Specifies the core snap rules. It can be set to snap to: • `manufacturing`: Manufacture Grid • `inst`: Instance Grid • `placement`: Placement Grid • `userDefine`: User-define Grid • `LayerTrack`: LayerTrack Grid • `finfetManufacturing`: FinFET Manufacture Grid • `finfetInst`: FinFET Instance Grid • `finfetPlacement`: FinFET Placement Grid *Default*: `placement` **Note**: For FinFET designs, the default is `finfetPlacement`. *Data_type:* enums={manufacturing inst placement userDefine LayerTrack finfetInst finfetManufacturing finfetPlacement}
`-snapDieGrid {manufacturing | inst | placement | userDefine | LayerTrack | finfetInst | finfetManufacturing | finfetPlacement }`	
	Specifies the die snap rules. It can be set to snap to: • `manufacturing`: Manufacture Grid • `inst`: Instance Grid • `placement`: Placement Grid • `userDefine`: User-define Grid • `LayerTrack`: LayerTrack Grid • `finfetManufacturing`: FinFET Manufacture Grid • `finfetInst`: FinFET Instance Grid • `finfetPlacement`: FinFET Placement Grid *Default*: `placement` **Note**: For FinFET designs, the default is `finfetPlacement`. *Data_type:* enums={manufacturing inst placement userDefine LayerTrack finfetInst finfetManufacturing finfetPlacement}
`-snapIoGrid {manufacturing | inst | placement | userDefine | LayerTrack | finfetInst | finfetManufacturing | finfetPlacement }`	
	Specifies the IO snap rules. It can be set to snap to: • `manufacturing`: Manufacture Grid • `inst`: Instance Grid • `placement`: Placement Grid • `userDefine`: User-define Grid • `LayerTrack`: LayerTrack Grid • `finfetManufacturing`: FinFET Manufacture Grid • `finfetInst`: FinFET Instance Grid • `finfetPlacement`: FinFET Placement Grid *Default*: `manufacturing` **Note**: For FinFET designs, the default is `finfetManufacturing`. *Data_type:* enums={manufacturing inst placement userDefine LayerTrack finfetInst finfetManufacturing finfetPlacement}

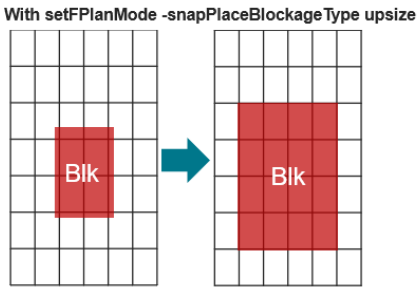
`-snapPlaceBlockageGrid {manufacturing|inst|finfetInst|finfetManufacturing|userDefine}`	
	Specifies the placement blockage snap rules. • `manufacturing`: Manufacture Grid • `inst`: Instance Grid • `userDefine`: User-define Grid • `finfetManufacturing`: FinFET Manufacture Grid • `finfetInst`: FinFET Instance Grid *Default*: `inst` *Data_type:* enums={manufacturing inst finfetInst finfetManufacturing userDefine}
`-snapPlaceBlockageType {closest|upsize|sequential}`	

Specifies a snapping type for placement blockages. The snapping types can be:

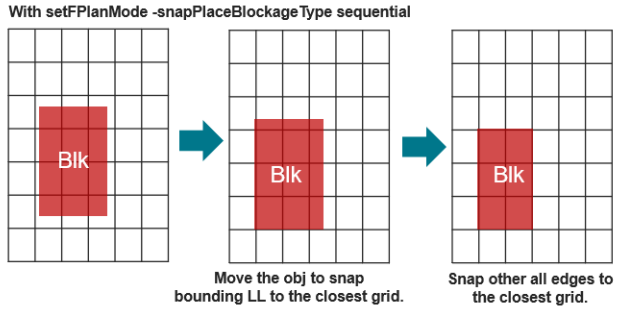
- `closest`: The tool snaps all corners of the placement blockage to the closest specified grid, regardless of which direction could increase or decrease the blockage size.



- `upsize`: The tool snaps all corners of the placement blockage to the corresponding closest specified grid in the direction which would increase the objects' size.



- `sequential`: The tool moves the placement blockage to snap the low-left corner to the closest specified grid, then snaps the upper-right corner to the closest grid.



This setting is honored by the following commands:

- `setObjFPlanBox`
- `setObjFPlanBoxList`
- `setObjFPlanPolygon`
- `createPlaceBlockage {-box|-boxList|-polygon}`
- `snapFPlan -placeBlk`

Default: `closest`

Data_type: enums={closest upsize sequential}

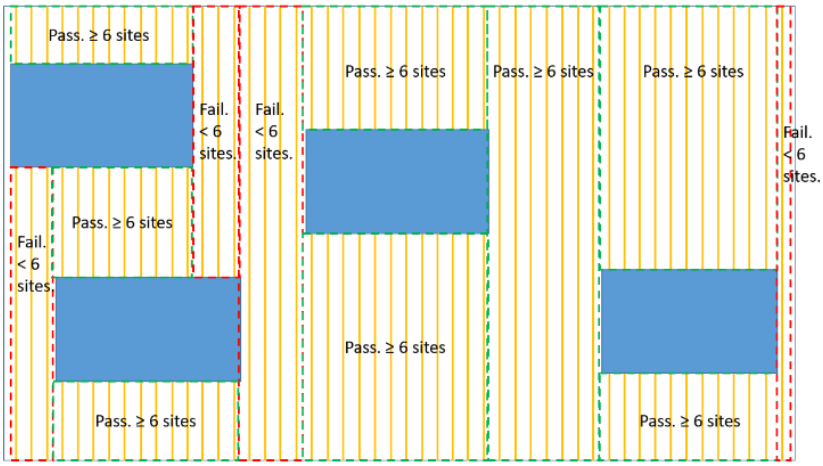
`-user_define_grid {x_offset y_offset x_pitch y_pitch}`

	Specifies the user-defined grid. The format is {*x_offset y_offset x_pitch y_pitch*}. Where, *x_offset y_offset* is the offset of the user-defined grid. *x_pitch* separates the vertical grid lines. *y_pitch* separates the horizontal grid lines. **Note:** The values are in microns. *Default*: {0 0 0 0} *Data_type:* box

Examples

In the following, the command checks the same length site rule in a group of continuous poly stripes with the same length. The number of minimal sites it check is set to 6 using the `-minimumSites` parameter.

```
setFPlanMode -checkTypes sameLengthSite -minimumSites 6
```



Related Information

- `saveFPlan`
- `loadFPlan`
- `initCoreRow`
- `checkFPlan`
- `floorPlan`
- `getFPlanMode`
- Floorplanning the Design chapter in the *Innovus User Guide*

setFPlanRowSpacingAndType

```
setFPlanRowSpacingAndType
[-help]
spacing
type
```

Specifies the standard row spacing values and row pattern type. You can use this command after importing the design.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `setFPlanRowSpacingAndType` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setFPlanRowSpacingAndType`
spacing	Specifies the standard row spacing, in micrometers. This must be a positive value. *Default*: 0 *Data_type*: float, required
type	Specifies the row spacing type. Specify 1 for every row, or 2 for every other row to apply the spacing value. *Data_type*: int, required

Example

The following command specifies a row spacing value of 1.1 micrometers for every other row:

```
setFPlanRowSpacingAndType 1.1 2
```

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*
  - `Standard Row Spacing

setInstancePlacementStatus

```
setInstancePlacementStatus
[-help]
{-allHardMacros | -allPtnBlks |-allBlackBoxes | -name instName(s)}
[-status {unplaced | fixed | placed | cover | softFixed}]
```

Changes the placement attributes status for all hard macros, all partition blocks, all blackboxes, or selected instances. This allows you to fix all blocks so they are not moved by `place_design`.

Parameters

`-allHardMacros`	Applies the status option only to all hard macros. *Data_type*: bool, required
`-allPtnBlks`	Applies the status option only to all partition blocks. *Data_type*: bool, required
`-allBlackBoxes`	Applies the status option only to all blackboxes. *Data_type*: bool, required
`-help`	Outputs a brief description that includes type and default information for each `setInstancePlacementStatus` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setInstancePlacementStatus`
`-name instName(s)`	Applies the status options to the specified instance(s). You can use a wildcard to specify multiple instances. *Data_type*: string, required
`-status {unplaced | fixed | placed | cover` | softFixed}	
	Specifies the change in placement status for instances. • `unplaced`: Specifies the change in placement status to be an *unplaced* status. The unplaced component does not have a location. • `fixed`: Specifies the change in placement status to a *fixed* status. The fixed component has a location and cannot be moved by automatic tools, but can be moved using interactive commands. • `placed`: Specifies the change in placement status to a *placed* status. The placed component has a location, and can be moved using automatic tools and interactive commands. • `cover`: Specifies the change in placement status to a *cover* status. The cover component has a location, but cannot be moved by automatic tools or interactive commands. • `softFixed`: Specifies the change in placement status to a *softFixed* status. The softFixed placement status means that instances cannot be moved by global placement and can only be moved by the legalization step of detail placement. Instances with this status can also be upsized by optimization. *Data_type*: enum, required

Example

The following commands set all hard macros and blackboxes to `fixed`. By setting this status, `place_design` can run without moving these fixed blocks.

```
setInstancePlacementStatus -allHardMacros -status fixed
setInstancePlacementStatus -allBlackBoxes -status fixed
```

Related Information

- `place_design`
- Floorplanning the Design chapter in the *Innovus User Guide*

setInstGroupPhyHier

```
setInstGroupPhyHier
[-help]
groupName
```

Sets a pre-existing instance group into a physical hierarchy where you can generate a corresponding netlist. You can use this command after creating a group name.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `setInstGroupPhyHier` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setInstGroupPhyHier`
`groupName`	Specifies the name of the created group. *Data_type*: string, required

Example

The following commands create and set the instance group `adder1` as a physical hierarchy:

```
createInstGroup adder1
setInstGroupPhyHier adder1
```

Related Information

- `createInstGroup`
- Floorplanning the Design chapter in the *Innovus User Guide*

setIoFlowFlag

```
setIoFlowFlag
[-help]
{0 | 1}
```

Sets the flag to use I/O row flow for pad placement. To view the current flag setting, use the `getIoFlowFlag` command. You can use this command in the floorplanning stage. You can use this commands before creating I/O rows for I/O row based pad placement.

Parameters

`{0 | 1}`	Specify 1, to use the I/O row flow. Specify 0, to use the normal I/O flow. *Data_type*: int, required
`-help`	Outputs a brief description that includes type and default information for each `setIoFlowFlag` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setIoFlowFlag.`

Examples

- The following command enables you to start using the I/O row flow in Innovus:
  ```
  setIoFlowFlag 1
  ```

Related Information

- `getIoFlowFlag`
- `createIoRow`
- Floorplan Menu chapter in the *Innovus Menu Reference*
  - Create I/O Row
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Performing I/O Row Based Pad Placement
- Data Preparation chapter in the *Innovus User Guide*
  - Creating an I/O Assignment Flow

setIoRowMargin

```
setIoRowMargin
[-help]
side
row
margin
```

Sets the distance from the die boundary edge to the I/O row starting edge location. You can use this command for multiple I/O rows.

Note: You can use the *Edit I/O Ring* form to specify I/O pad rings and row margins for multiple rows. Alternatively, to achieve the same using text commands, you must first use the `setIoRowMargin` command to set the distance from the die boundary edge to start of each row and then use the `placePadIO` command to place the I/O pads evenly between these rows.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `setIoRowMargin` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setIoRowMargin`
side	Specifies the side of the I/O row margin, where n is the North side, w is the West side, s is the South side, and e is the East side. *Data_type*: string, optional
margin	Specifies the distance between the die boundary edge to the I/O row edge, in microns. *Data_type*: float, optional

row	Specifies the I/O row number. This value can be up to 8 rows. You can also use this value in conjunction with the `placePadIO` and `placePIO -optIOs` commands to specify the number of rows for placing peripheral I/O cells for the CLASS PAD and CLASS PAD AREAIO flows respectively.
	• To create and optimize multiple I/O Rows in a CLASS PAD flow, use the following commands:
	`setIoRowMargin row`
	`placePadIO`
	• To create and optimize multiple I/O Rows in a CLASS PAD AREAIO flow, use the following commands:
	`setIoRowMargin rowNumber`
	`placePIO -optIOs`
	Maximum: 1000
	Minimum: 0
	Data_type: int, optional

Example

The following command sets the North side of the second I/O row starting edge value at 300 microns from the die boundary edge to the second I/O row edge:
`setIoRowMargin n 2 300`

Related Information

- `placePadIO`
- `placePIO`
- Floorplanning the Design chapter in the *Innovus User Guide*
- Floorplan Menu chapter in the *Innovus Menu Reference*
  - Edit I/O Ring

setObjFPlanBox

```
setObjFPlanBox
[-help]
objectType
objectName
llx lly urx ury
```

Defines the bounding box of a specified object, even outside the core boundary. You can use this command after importing the design.

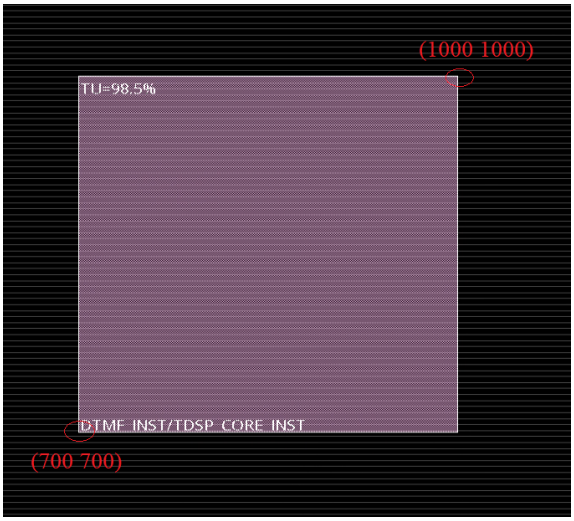
Parameters

`-help`	Outputs a brief description that includes the type and default information for each `setObjFPlanBox` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setObjFPlanBox`
`objectName`	Specifies the name of the object for *`objectType`*. *Data_type*: string, required
`objectType`	Specifies the type of object for objectName. This can be any of the following: • Bump • Cell • Group • Instance • I/O cell • I/O pin • Layershape • Module • Net • Partition cut • Partition feedthrough • Partition pin block • Pin • Pin guide • Row cluster • Standard row *Data_type*: string, required
`llx`	Specifies the lower left x coordinate of the object. *Data_type*: float, required
`lly`	Specifies the lower left y coordinate of the object. *Data_type*: float, required
`urx`	Specifies the upper right x coordinate of the object. *Data_type*: float, required
`ury`	Specifies the upper right y coordinate of the object. *Data_type*: float, required

Example

The following command specifies a bounding box for module DTMF_INST/TDSP_CORE_INST at a lower left x coordinate of 700.00, a lower left y coordinate of 700.00, and upper right x coordinate of 1000.00, and an upper right y coordinate of 1000.00:

```
setObjFPlanBox Module DTMF_INST/TDSP_CORE_INST 700.00 700.00 1000.00 1000.00
```



Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*
  - Defining the Bounding Box

setObjFPlanBoxList

```
setObjFPlanBoxList
[-help]
{Cell | Group | Instance | Layershape | Module}
objectName
boxList
```

Defines the rectilinear shape of an object even if the object lies outside the core area. A rectilinear object comprises of two or more boxes. You can use this command after importing the design.

Parameters

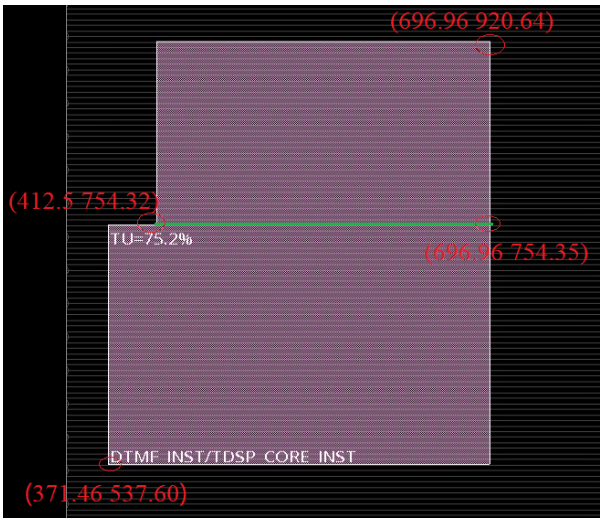
`-help`	Outputs a brief description that includes the type and default information for each `setObjFPlanBoxList` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setObjFPlanBoxList`
`objectName`	Specifies the name of the object for object type. *Data_type*: string, required
{`Cell | Group | Instance | Layershape | Module`}	
	Specifies the type of object for objectName. *Data_type*: enum, required
`boxList`	Specifies the rectilinear shape of an object. • *`llx1`*: Specifies the lower left x coordinate of the first box in the box list. • *`lly1`*: Specifies the lower left y coordinate of the first box in the box list. • *`llx2`*: Specifies the lower left x coordinate of the second box in the box list. • *`lly2`*: Specifies the lower left y coordinate of the second box in the box list. • *`urx1`*: Specifies the upper right x coordinate of the first box in the box list. • *`ury1`*: Specifies the upper right x coordinate of the first box in the box list. • *`urx2`*: Specifies the upper right x coordinate of the second box in the box list. • *`ury2`*: Specifies the upper right y coordinate of the second box in the box list. • *`llxn`*: Specifies the lower left x coordinate of the nth box in the box list. • *`llyn`*: Specifies the lower left y coordinate of the nth box in the box list. • *`urxn`*: Specifies the upper right x coordinate of the nth box in the box list. • *`uryn`*: Specifies the upper right y coordinate of the nth box in the box list. *Data_type*: list, required

Example

The following command defines a rectilinear boundary for module DTMF_INST/TDSP_CORE_INST. The rectilinear boundary is made up of two bounding boxes:

1. (371.46, 537.60) (696.96, 754.35)
2. (412.5, 754.32) (696.96, 920.64)

```
setObjFPlanBoxList Module DTMF_INST/TDSP_CORE_INST 371.46 537.60 696.96 754.35 412.5 754.32 696.96 920.64
```



Related Information

- `getObjFPlanBoxList`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Defining the Bounding Box

setObjFPlanPolygon

```
setObjFPlanPolygon
[-help]
{Cell | Group | Instance | Layershape | Module }
objectName
coordList
```

Specifies a rectilinear object with polygon coordinates. The rectilinear object can take any shape depending on the number of x and y coordinates you specify. You can use this command after importing the design.

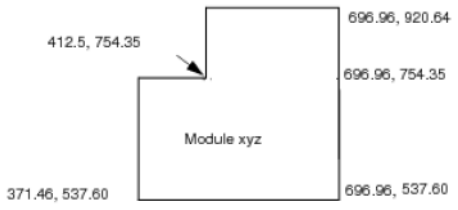
Parameters

`-help`	Outputs a brief description that includes the type and default information for each `setObjFPlanPolygon` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setObjFPlanPolygon`
objectName	Specifies the name of the object for objectType. *Data_type*: string, required
{`Cell	Group	Instance	Layershape	Module`}	
	Specifies the type of object for objectName. This can be: • Cell • Group • Instance • Layershape • Module *Data_type*: enum, required
coordList	Specifies the corner coordinates, {*x1 y1 x2 y2 .....*}, of the rectilinear object. The rectilinear object can take any shape depending on the number of x and y coordinates you specify. *Data_type*: list, required

Example

The following command defines a rectilinear object - module xyz with five pairs of corner coordinates.

```
setObjFPlanPolygon module xyz 371.46 537.60 371.46 754.35 412.5 754.35 412.5 920.64 696.96 920.64 696.96 537.60
```



Related Information

- `getObjFPlanPolygon`
- Floorplanning the Design chapter in the *Innovus User Guide*

setRailPrototypeMode

```
setRailPrototypeMode
[-help]
[-reset]
[-domain {{PD1 float net1} {PD2 float net2}..}]
[-railModel {virtual | existing}]
[-totalPower float]
```

Sets the mode for Power Aware Congestion Estimation (PACE). This helps in estimating the routability by taking into account power rail effect so that congestion estimation can be done without actually building the power plan. It supports multiple power domains specified in floorplan.

Note: `setRailPrototypeMode` command must be called before any congestion estimation command is called for it to take effect.

Parameters

`-domain {{PD1 float net1} {PD2 float net2}...}`	
	Specifies the power, in milliwatts, assigned for nets in power domains. **Note:** If the total power or the domain power is not specified, the `setRailPrototypeMode` command will estimate it internally based on the instance area. *Default*: `" "` (empty string) *Data_type*: string
`-help`	Outputs a brief description that includes the type and default information for each `setRailPrototypeMode` parameter. For a detailed description of the command and all of its parameters, use the man command `man setRailPrototypeMode`.
`-railModel {virtual | existing}`	
	Specifies the power grid to use. • `virtual`: Use routing blockages to model virtual power grid. These routing blockages are removed after congestion analysis is done. • `existing`: Use the current power grid. *Default*: `existing` *Data_type*: enums={virtual existing}
`-reset`	Restores all parameters modified using `setRailPrototypeMode` to their default settings. The `-reset` parameter must be the first parameter specified. If you specify -reset by itself, the software resets all `setRailPrototypeMode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported.
`-totalPower float`	
	Specifies a target value for the power, in milliwatts, consumed by the design. *Default*: `0` *Data_type*: float

Related Information

- `getRailPrototypeMode`
- Floorplanning the Design chapter in the *Innovus User Guide*

setResizeFPlanMode

```
setResizeFPlanMode
[-help]
[-reset]
[-honorHalo {true | false}]
[-maintainResourceRatioAfterResize {true | false}]
[-shrinkFence {true | false}]
[-snapToTrack {true | false}]
[-shiftBased {true | false} | -proportional {true | false} |
-congAware {true | false}]
[-ioProportional {true | false} | -ioMoveWithEdge {true | false} |
-ioFix {true | false}]
```

Controls certain aspects of how floorplan resizes the design. Use the `getResizeFPlanMode` command to return the current settings for the `setResizeFPlanMode` command. The `setResizeFPlanMode` parameters affects the behavior the `resizeFloorplan` command. You can use this command after importing the design and before using the `resizeFloorplan` command. You can resize the space among floorplan objects in three ways - proportional spacing, shift-based spacing, or congestion-based spacing.

Parameters

`-congAware {true | false}`	
	Resizes and shifts the floorplan objects by estimating the congestion for the floorplan and automatically deciding where to draw a resize line to avoid the congested area. *Default*: `false` *Data_type*: bool
`-help`	Outputs a brief description that includes the type and default information for each `setResizeFPlanMode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setResizeFPlanMode.`
`-honorHalo {true | false}`	
	Honors placement halo and preserves the space between macros held by halo after the floorplan resize. With this option set to true, if the original placement halo does not overlap with other macros or placement halo, after `resizeFP`, it will not overlap with other macros or placement halo. *Default*: true *Data_type*: bool
`-ioFix {true | false}`	
	Fixes I/Os at current location. *Default*: false *Data_type*: bool
`-ioMoveWithEdge {true | false}`	
	Moves pins on the moved edge orthogonally to edge direction so pins stay on edge after movement. The pins on other edges are not touched. *Default*: false *Data_type*: bool
`-ioProportional {true | false}`	
	Resizes the space among I/Os proportionally. If you set this parameter to true, the I/Os are distributed proportional to the original I/O space. If you set this parameter to false, the I/Os are distributed evenly. **Note**: By default, the overlapping I/Os are spaced proportionally. *Default*: true *Data_type*: bool
`-maintainResourceRatioAfterResize {true | false}`	

	When enabled, it maintains the resource ratio after `resizeFloorplan` in shift-based mode. *Default*: false *Data_type*: bool
`-proportional {true | false}`	
	Resizes the space among floorplan objects proportionally. Default: false *Data_type*: bool
`-reset`	Restores all parameters modified using `setResizeFPlanMode` to their default settings. The `-reset` parameter must be the first parameter specified. If you specify -reset by itself, the software resets all `setResizeFPlanMode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported.
`-shiftBased {true | false}`	
	Shifts floorplan objects at appropriate location(s) without changing the proportional spacing. You can specify resize lines to control the areas of floorplan that will be expanded or shrunk. To set resize lines, use the `setResizeLine` command. If you do not specify resize line(s), the floorplan is resized automatically - that is, the resize lines are derived automatically based on the current floorplan and the specified resize values. *Default*: true *Data_type*: bool
`-shrinkFence {true | false}`	
	Enables resized lines to go through fences and regions in floorplan. With this option the `resizeFloorplan` command, automatically resizes fences, regions, and guides inside the floorplan while maintaining their minimum size after the shrink. Default: false *Data_type*: bool
`-snapToTrack {true | false}`	
	Snaps resize values (shrink/expand) of the floorplan to a multiple integer of the metal layer pitch. For example, if the horizontal metal pitch is 1.568 microns and you want to shrink the floorplan by 8 microns in y direction, the actual shrink value is 7.84 microns, the nearest multiple integer of the metal pitch. The x direction resize value snaps to the first vertical layer pitch and the y direction resize value snaps to the first horizontal layer pitch. *Default*: true *Data_type*: bool

Related Information

- `resizeFloorplan`
- `setResizeLine`
- `getResizeFPlanMode`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Resizing the Floorplan

setResizeLine

```
setResizeLine
[-help]
-clearAll
[-direction {H|V}]
[points]
[-width float]
```

Sets the resize lines for shift-based floorplan resize option of the `resizeFloorplan` command. The resize lines can be non-continuous though they must be orthogonal. If resize lines are specified, the floorplan will be resized between the area specified by the resize lines (for shift-based floorplan resize). To shrink or expand the floorplan in the horizontal direction, specify a vertical resize line. To shrink or expand the floorplan in the vertical direction, specify a horizontal resize line.

Each resize line can have multiple segments (for example, horizontal, vertical, and again horizontal) but it can only be applied to one direction. If you want to shrink or expand the floorplan in both directions, specify resize lines for each direction. You can create multiple resize lines. The resize lines are removed once the `resizeFloorplan` command is run. You can use this command after importing the design.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setResizeLine` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setResizeLine`
`-clearAll`	Clears all resize lines. *Data_type*: bool, optional
`-direction {H|V}`	Specifies whether the resize line is horizontal (H) or vertical (V). *Data_type*: string, optional
points	Specifies the resize line point list. *Data_type*: pt list, optional
`-width` *float*	Specifies the width of the resize line in microns. A positive value expands the floorplan and a negative value shrinks the floorplan. *Data_type*: float, optional

Examples

- The following command sets a resize line with a width of 20 microns in the horizontal direction. As the width has a positive value, the floorplan will expand. The resize line is horizontal and will, therefore, expand the floorplan in a vertical direction.
  ```
  setResizeLine -direction H -width 20 (1924 3476) (3807 3476)
  ```
- The following command sets a resize line with a width of -20 microns in the horizontal direction. As the width has a negative value, the floorplan will shrink. The resize line is horizontal and will, therefore, shrink the floorplan in a vertical direction.
  ```
  setResizeLine -direction H -width -20 (1924 3476) (3807 3476)
  ```
- The following command sets a resize line with a width of 20 microns in the vertical direction. As the width has a positive value, the floorplan will expand. The resize line is vertical and will, therefore, expand the floorplan in a horizontal direction.
  ```
  setResizeLine -direction V -width 20 (2761 2215) (2761 3810)
  ```
- The following command sets a resize line with a width of -20 microns in the vertical direction. As the width has a negative value, the floorplan will shrink. The resize line is vertical and will, therefore, shrink the floorplan in a horizontal direction.
  ```
  setResizeLine -direction V -width -20 (2761 2215) (2761 3810)
  ```
- The following command sets a resize line with a width of -20 microns in the vertical direction. As a * has been specified for one of the coordinates, the value of that coordinate will be automatically derived.
  ```
  setResizeLine -direction V -width -20 (2761 *) (2761 3810)
  ```
- The following command creates a resize line with multiple segments.
  ```
  setResizeLine -direction V -width -20 (2643 1744) (2643 3411) (3382 3411) (3382 4117)
  ```

Related Information

- `addSizeBlockage`
- `deleteSizeBlockage`
- `getResizeFPlanMode`
- `resizeFloorplan`
- `setResizeFPlanMode`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Resizing the Floorplan

setRouteBlkDefaultLayer

```
setRouteBlkDefaultLayer
[-help]
{{[-layer layerName | {layerNameList} | all]
[-cutLayer layerName | {layerNameList} | all]
[-drcRegionLayer layerName | {layerNameList} | all]
[-trimMetalLayer layerName | {layerNameList} | all] } | {-allMetalCut }}
```

Specifies the default layers for routing blockages, for both metal layers and cut layers. The default layers set with this command are applicable for all routing blockages subsequently created using the GUI or the `createRouteBlk` command. Use this command before creating routing blockages.

Note: If any layers are specified with the `createRouteBlk` command, the layers specified with the `setRouteBlkDefaultLayer` command are ignored.

Parameters

`-allMetalCut`	Specifies that default routing blockages will be created for all metal layers and all cut layers. *Data_type*: bool, required
`-cutLayer layerName | {layerNameList} | all`	
	• *layerName*: Specifies that default routing blockages will be created for the specified cut layers. • `all`: Specifies that default routing blockages will be created for all cut layers. *Data_type*: list, optional
`-drcRegionLayer layerName | {layerNameList} | all`	
	• *layerName*: Specifies that default routing blockages will be created for the specified drc region layers. • `all`: Specifies that default routing blockages will be created for all drc region layers. *Data_type*: list, optional
`-help`	Outputs a brief description that includes the type and default information for each `setRouteBlkDefaultLayer` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setRouteBlkDefaultLayer`
`-layer layerName | {layerNameList} | all`	
	• *layerName*: Specifies that default routing blockages will be created for the specified metal layers. • `all`: Specifies that default routing blockages will be created for all metal layers. *Data_type*: list, optional
`-trimMetalLayer layerName | {layerNameList} | all`	
	• *layerName*: Specifies that default routing blockages will be created for the specified trim metal layers. • `all`: Specifies that default routing blockages will be created for all trim metal layers. *Data_type*: list, optional

Example

The following command sets default routing blockages for the metal layers1, 2, and 3 and the cut layers 3 and 5.

```
setRouteBlkDefaultLayer -layer 1 2 3 -cutLayer 3 5
```

Related Information

- `createRouteBlk`
- `deleteRouteBlk`
- Floorplanning the Design chapter in the *Innovus User Guide*

setSelectedDensityArea

```
setSelectedDensityArea
[-help]
x1 y1 x2 y2
density
name
{Partial Soft Hard}
{Instance Pushdown Undefined}
```

Specifies the attributes for the selected density screen area(s). After importing the design and selecting the density area, if required, you can use this command to change the density area's location, type, or density.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `setSelectedDensityArea` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setSelectedDensityArea`
x1 y1 x2 y2	
	Specifies the coordinates of the density screen area. *Data_type*: float, required
density	Specifies the standard cell placement density percentage. The valid values are in increments of 5: 0, 5, 10, ...100. For example, setting a density percentage of 75 percent means that up to 75 percent of the defined density area will be used for placement. **Note**: If you enter any number that is not valid, the percentage defaults to the nearest valid value. *Maximum:* 100 *Minimum:* 0 *Data_type*: float, required
name	Specifies the name of the density screen area. *Data_type*: string, required
`{Partial Soft Hard}`	
	Specifies the hardness of the density screen area. *Data_type*: enum, optional
`{Instance Pushdown Undefined}`	
	Specifies the relationship of the density screen area to the design. • `Instance`: The density screen area is associated with the current design. • `Pushdown`: The density screen area is pushed down from the hierarchy. • `Undefined`: The relationship of the density screen area to the design is undefined. *Default*: `Undefined`. *Data_type*: enum, optional

Example

The following command sets a soft density screen area, softDensity2, of the current design in the specified design area:

```
setSelectedDensityArea 100.0 100.0 200.0 200.0 75 softDensity2 Soft Instance
```

Related Information

- `deleteAllDensityAreas`
- `createDensityArea`
- Floorplanning the Design chapter in the *Innovus User Guide*

setSelectedObstruct

```
setSelectedObstruct
[-help]
x1 y1 x2 y2
ObstructName
{Instance Pushdown Undefined}
```

Sets the placement blockage attributes, like the area, name, and type, for the selected placement blockage (only). Use this command after importing the design and selecting the placement blockage.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `setSelectedObstruct` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setSelectedObstruct`
`x1 y1 x2 y2`	Specifies the coordinates of the placement blockage area. *Data_type*: float, required
`ObstructName`	Specifies the name of the placement blockage. *Data_type*: string, required
{`Instance Pushdown Undefined`}	
	Specifies the type of placement blockage. • `Instance`: The placement blockage is associated with the current design. • `Pushdown`: The placement blockage is pushed down from the hierarchy. • `Undefined`: The type of placement blockage is undefined. *Default*: `Undefined` *Data_type*: enum, required

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

setSelectedRouteBlk

```
setSelectedRouteBlk
[-help]
llx lly urx ury
name
layers
[{Undefined | Fills | Slots | Pushdown | EXCEPTPGNET | PGNETONLY | ALLNET}]
[spacing]
```

Sets a routing blockage for an attribute of the selected routing blockage. Use this command after importing the design and selecting the routing blockage.

Parameters

`{Undefined | Fills | Slots | Pushdown | EXCEPTPGNET | PGNETONLY | ALLNET}`	
	Specifies the attribute of the routing blockage. This can be: • `Undefined` • `Fills` • `Slots` • `Pushdown` • `EXCEPTPGNET` • `PGNETONLY` • `ALLNET` *Default*: `Undefined` *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `setSelectedRouteBlk` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setSelectedRouteBlk`
`layers`	Specifies the layer(s) and/or via(s). • For layer names, specify the number. For example, 3 • For via names, specify the uppercase character V followed by the via number. For example, V3 **Note:** This parameter supports masterslice layers with type `REGION`. *Data_type*: string, required
`llx`	Specifies the lower left x coordinate of the bounding box. *Data_type*: float, required
`lly`	Specifies the lower left y coordinate of the bounding box. *Data_type*: float, required
`name`	Specifies the name of the routing blockage. *Data_type*: string, required
`urx`	Specifies the upper right x coordinate of the bounding box. *Data_type*: float, required
`ury`	Specifies the upper right y coordinate of the bounding box. *Data_type*: float, required
`spacing`	Specifies the minimum spacing between layers within the routing blockage. The value is a real number greater than or equal to 0. *Data_type*: string, optional

Examples

- The following command sets the routing blockage for fills in routing blockage blk2 that lies in the bounding box with the coordinates 17, 223, 1117, and 1223, on metal layer1 and via 2.

```
setSelectedRouteBlk 17 223 1117 1223 blk2 {1 V2} Fills
```

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

setSelectedStripBoxShape

```
setSelectedStripBoxShape
[-help]
index
{None RING STRIPE FOLLOWPIN IOWIRE COREWIRE BLOCKWIRE FILLWIRE PADRING DRCFILL FILLWIREOPC}
```

Sets the shape of the selected strip box. You can use this command after importing the design and selecting a stripe box (special routing).

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `setSelectedStripBoxShape` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setSelectedStripBoxShape`
index	Specifies which selected strip box shape needs to be set. You can specify one of the following: • `all`: Specifies that the shape of all the selected strip boxes must be set. • `integer`: Specifies that the shape of only the selected strip box must be set. For example, if you have 4 strip boxes selected, an index value, 2, specifies that the shape of the third selected strip box must be set. *Data_type*: string, required
`{None RING STRIPE FOLLOWPIN IOWIRE COREWIRE BLOCKWIRE FILLWIRE PADRING DRCFILL FILLWIREOPC}`	
	Specifies the shape of the selected strip box. You can specify one of the following values: • `None` • `RING` • `STRIPE` • `FOLLOWPIN` • `IOWIRE` • `COREWIRE` • `BLOCKWIRE` • `FILLWIRE` • `PADRING` • `DRCFILL` • `FILLWIREOPC` *Data_type*: enum, required

Examples

- If you selected 5 strip boxes, the following command sets the shape of the fifth strip box to RING.
  ```
  setSelectedStripBoxShape 4 RING
  ```
- If you selected 4 strip boxes, the following command sets the shape of all the 4 strip boxes to RING.
  ```
  setSelectedStripBoxShape all RING
  ```

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

setSelectedStripBoxState

```
setSelectedStripBoxState
[-help]
index
{ROUTED FIXED COVER SHIELD}
[shield_net]
```

Sets the state of the selected strip box. You can use this command after importing the design and selecting a stripe box.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `setSelectedStripBoxState` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setSelectedStripBoxState`
index	Specifies which selected strip box state needs to be set.You can specify one of the following: • `all`: Specifies that the state of all the selected strip boxes must be set. • `integer`: Specifies that the state of only the selected strip box must be set.For example, if you have 4 strip boxes selected, an index value, 2, specifies that the state of the third selected strip box must be set. *Data_type*: string, required
`{ROUTED FIXED COVER SHIELD}`	
	Specifies the state of the selected strip box. You can specify one of the following values: • `ROUTED` • `FIXED` • `COVER` • `SHIELD` *Data_type*: enum, required
shield_net	Specifies the shield net. If you are setting the state of the selected strip box as SHIELD, you must specify a shield net. *Data_type*: string, optional

Examples

- If you selected 5 strip boxes, the following command sets the state of the fifth strip box to FIXED.
  ```
  setSelectedStripBoxState 4 FIXED
  ```
- If you selected 4 strip boxes, the following command sets the state of all the 4 strip boxes to ROUTER.
  ```
  setSelectedStripBoxState all ROUTED
  ```

Related Information

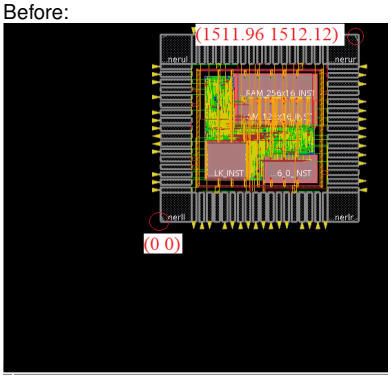
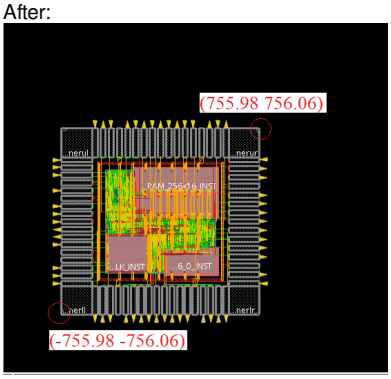
- Floorplanning the Design chapter in the *Innovus User Guide*

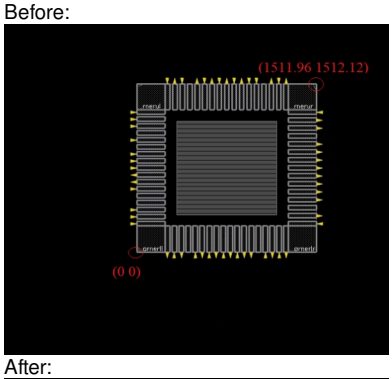
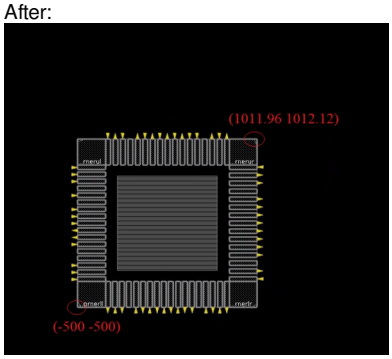
shiftOrigin

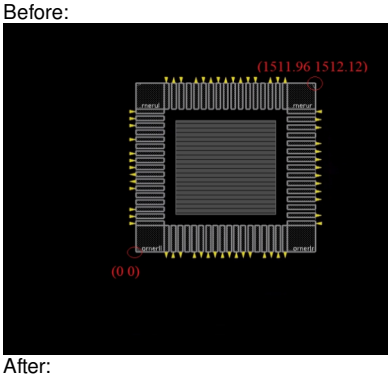
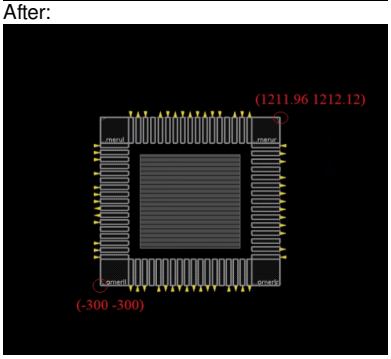
```
shiftOrigin
[-help]
{-center | -lowerLeft | -location x y | -delta x y}
```

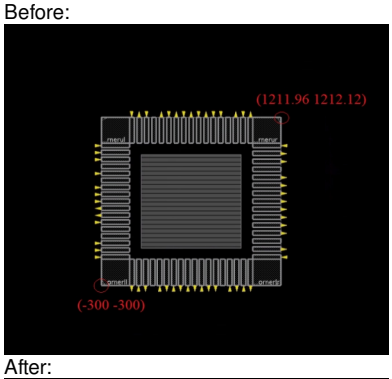
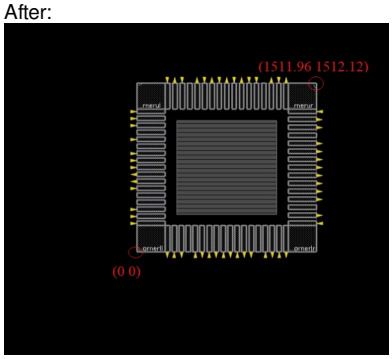
Shifts the entire design to a new location. All objects that are recorded in DEF are shifted to a new location. No change is made to the other attributes of objects except the location. All properties and characters such as orientation and placement status remain the same after the shift.

Parameters

`-center`	Shifts the origin point of the entire design to the center point of the current floorplan box. Example: `shiftOrigin -center` Before:  After:  *Data_type*: bool, optional

`-delta x y`	Shifts the origin point of the entire design by the specified distance, *deltaX* and *deltaY* Example: `shiftOrigin -delta 500 500` Before:  After:  *Data_type*: point, optional
`-help`	Outputs a brief description that includes the type and default information for each `shiftOrigin` parameter. For a detailed description of the command and all of its parameters, use the man command: `man shiftOrigin`

`-location x y`	Shifts the origin point of the entire design to the specified location, `x y`. Example: `shiftOrigin -location 300 300` Before:  After:  *Data_type*: point, optional

`-lowerLeft`	Shifts the origin point of the entire design to the lower left point of current floorplan box. Example: `shiftOrigin -lowerLeft` Before:  After:  *Data_type*: bool, optional

Example

The following command moves the origin point to the center point of the current floorplan box:

```
shiftOrigin -center
```

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

snapFPlan

```
snapFPlan
[-help]
{{-all} | {-selected} |
{[-guide][-block][-stdCell][-ioPad][-areaIo][-ptnCore][-pinGuide][-routeBlk][-placeBlk][-macroPin][-pin][-pinBlk]}}
```

Snaps floorplan objects to the grid. You can use this command after importing the design.

Note: Innovus honors the FinFET grid and automatically checks for the FinFET definition. If it detects a FinFET definition, it enables support and sets the placement grid as FinFET grid by default. Since the height of all placeable objects must be a multiple of FinFET grid and the origin of these objects must be snapped to the FinFET grid. The snapFPlan command is used to snap blocks, I/O pads, standard cells to the FinFET grid. It honors user specified snapping rule for x direction. For y direction, it honors the FinFET grid (when the FinFET grid is horizontal in regular horizontal row design).

Parameters

`-all`	Specifies the snapping of all supported object types. *Data_type*: bool, required
`-areaIo`	Specifies the snapping of I/O cells. *Data_type*: bool, optional
`-block`	Specifies the snapping of floorplan blocks. *Data_type*: bool, optional
`-guide`	Specifies the snapping of floorplan guides, regions, and fences. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `snapFPlan` parameter. For a detailed description of the command and all of its parameters, use the man command: `man snapFPlan`
`-ioPad`	Specifies the snapping of I/O pads. *Data_type*: bool, optional
`-macroPin`	Specifies the snapping of macro pins to the correct color track. **Note:** The `snapFPlan` command only snaps signal pins and places a macro with macro pins on color track. Innovus snaps the port on the lowest metal layer which is close to macro origin to the color track. *Data_type*: bool, optional
`-pin`	Specifies the snapping of pins. *Data_type*: bool, optional
`-pinBlk`	Specifies the snapping of pin blockages. *Data_type*: bool, optional
`-pinGuide`	Specifies the snapping of pin guides. *Data_type*: bool, optional
`-placeBlk`	Specifies the snapping of placement blockages. *Data_type*: bool, optional
`-ptnCore`	Specifies the snapping of the partition core. *Data_type*: bool, optional
`-routeBlk`	Specifies the snapping of routing blockages. *Data_type*: bool, optional
`-selected`	Specifies the snapping of all supported object types that are selected in the design display area. *Data_type*: bool, required
`-stdCell`	Specifies the snapping of standard cells. *Data_type*: bool, optional

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

snapFPlanIO

```
snapFPlanIO
[-help]
[-selected]
[-userGrid | -toIoRow [-overlapRowOnly]]
```

Snaps I/O cells to a user-defined grid. Use this command after importing the design.

Note: You can specify the user-defined grid in the *Preferences - Floorplan* form in the View Menu of the Innovus GUI.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `snapFPlanIO` parameter. For a detailed description of the command and all of its parameters, use the man command: `man snapFPlanIO`
`-overlapRowOnly`	Snaps the I/O cell only onto an overlapping I/O row. If an I/O row is not found, the command is not executed. *Data_type*: bool, optional
`-selected`	Applies to the selected I/Os only. *Default*: snaps to all I/O cells. *Data_type*: bool, optional
`-toIoRow`	Snaps the I/O cell onto an overlapping I/O row first. If an I/O row is not found, places the I/O cell in any row having the same site. *Data_type*: bool, optional
`-userGrid`	Specifies the snapping of I/O pads to user-defined grids. *Default*: snaps I/O to manufacture grid. *Data_type*: bool, optional

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

spaceIoInst

```
spaceIoInst
[-help]
[-fixSide {left | right | top | bottom | horDistribute | vertDistribute}]
[-space value]
```

Spaces the selected I/O cells horizontally or vertically by a specified distance value, and can evenly distribute the spacing horizontally or vertically between two or more I/O cells. You can use this command after selecting two or more instances in the design display area.

Note: This command supports I/O row flow.

Parameters

`-fixSide`	Specifies the vertical and horizontal instance spacing. Choose one of the following: • `left`: Specifies horizontal left spacing. • `right`: Specifies horizontal right spacing. • `top`: Specifies vertical top spacing. • `bottom`: Specifies vertical bottom spacing. • `horDistribute`: Specifies an even horizontal distribution. • `vertDistribute`: Specifies an even vertical distribution. *Default:* The default value is `left`. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `snapIoInst` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man snapIoInst`
`-space value`	Specifies the spacing value, in micrometers. *Default:* The default value is 0 *Data_type*: float, optional

Example

The following command aligns the right-most highlighted I/O cell horizontally with a spacing of 5 micrometers:

```
spaceIoInst -fixSide right -space 5
```

Related Information

- `spaceObject`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Performing I/O Row Based Pad Placement

spaceObject

```
 spaceObject
[-help]
-fixSide {left | right | center | top | bottom | middle | distHorizontal | distVertical}
[-honorHalo]
[-space value [-isSpaceMeanLLtoLL ] ]
```

Spaces objects (instances, modules, or blockages) horizontally or vertically by a specified distance value, and can evenly distribute the spacing horizontally or vertically between three or more objects. You can use this command after selecting two or more objects in the design display area.

Note: To space I/O cells, use the `spaceIoInst` command.

Parameters

`-fixSide`	Specifies the vertical and horizontal object spacing. Choose one of the following: • `left`: Specifies that the left most object is fixed and all other objects are shifted horizontally to the right side of the fixed object, maintaining space defined in the `-space` option. • `right`: Specifies that the right most object is fixed and all other objects are shifted horizontally to the left side of the fixed object, maintaining space defined in the `-space` option. • `center`: Maintains the center point of the selected objects. It calculates the mid-point between the left most edge and right most edge of the selected objects and then spaces the blocks at the specified distance and centers them around this mid-point so that the left most edge and right most edge are at an equal distance from the mid-point. • `top`: Specifies that the top most object is fixed and all other objects are shifted vertically to the down side of the fixed object, maintaining space defined in the `-space` option. • `bottom`: Specifies that the bottom most object is fixed and all other objects are shifted vertically to the up side of the fixed object, maintaining space defined in the `-space` option. • `middle`: Maintains the center point of the selected objects. It calculates the mid-point between the top most edge and bottom most edge of the selected objects and then spaces the blocks at the specified distance and centers them around this mid-point so that the top most edge and bottom most edge are at an equal distance from the mid-point. • `distHorizontal`: Specifies an even horizontal distribution. • `distVertical`: Specifies an even vertical distribution. *Data_type*: enum, required
`-help`	Outputs a brief description that includes the type and default information for each `spaceObject` parameter. For a detailed description of the command and all of its parameters, use the man command: `man spaceObject`
`-honorHalo`	Honors placement halo. With this option on, selected objects are spaced by a given value measured from the edge of the halo and not from the edge of the object. *Data_type*: bool, optional
`-isSpaceMeanLLtoLL`	When specified it signifies that the spacing value denotes the distance from the Object A's lower left to Object B's lower left. By default, this parameter is false. This implies that the spacing value signifies the distance between the bounding of Object A and Object B. *Data_type*: bool, optional
`-space value`	Specifies the spacing value, in micrometers. *Data_type*: float, optional

Example

The following command fixes the bottom-most highlighted object and moves all the other objects vertically to the top, spacing the objects 5 micrometers apart from each other:

```
spaceObject -fixSide bottom -space 5
```

Related Information

- `spaceIoInst`
- Floorplanning the Design chapter in the *Innovus User Guide*

specifyNetWeight

```
specifyNetWeight
[-help]
netName ... netWeightValue
```

Specifies the priority weighting of a net. You can use this command after importing the design.

Note: Partition pin assignment honors `specifyNetWeight`. Pins that connect to a net that has a higher net weight are assigned before the pins that connect to a net with a lower net weight.

Note: The `setPlaceMode -place_global_clock_gate_aware` option has higher priority over `specifyNetWeight` and hence, if it is set then tool will not honor the weights on the clock net during the placement run as specified using the `specifyNetWeight` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `specifyNetWeight` parameter. For a detailed description of the command and all of its parameters, use the man command: `man specifyNetWeight`
`netName..`	Specifies the name(s) of an individual net. You can use wildcards (*?) with this parameter. *Data_type*: string, required
`netWeightValue`	Specifies the weighting priority of the net name, with values 512 (highest) through 1 (lowest), or 0 (don't care). *Default*: If you do not specify this parameter, the weight for nets is 2. The placer will try to shorten nets with higher weight, and give less priority to lower weights (ignoring nets with weight 0). The placer has a limit of 10 for the highest weight, and will treat any value above 10 as 10. The router will route higher weighted nets before lower weighted nets. *Data_type*: string, required

Examples

- The following commands set the priority net weighting for nets STRW to first, WE2N to second, and IOWIN to third:

  ```
  specifyNetWeight WE2N 9
  specifyNetWeight STRW 10
  specifyNetWeight IOWIN 8
  ```

- `specifyNetWeight` does not accept a Tcl list. You can list multiple nets by omitting the curly brackets:

  ```
  specifyNetWeight n1 n2 10
  specifyNetWeight n5 scan* 1
  ```

  You can also surround it with a foreach loop:

  ```
  foreach net [dbGet top.nets.name n*] {specifyNetWeight $net 10}
  ```

Related Information

- `setPlaceMode`
- `getNetWeight`
- `deleteNetWeight`
- Floorplanning the Design chapter in the *Innovus User Guide*

stretchRows

```
stretchRows
[-help]
{-left | -right | -left -right}
[-toCoreEdge]
```

Stretches selected rows. You can use this command after importing a design.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `stretchRows` parameter. For a detailed description of the command and all of its parameters, use the man command: `man stretchRows`
`-left`	Specifies that the left edge of all selected rows should be aligned to the left-most edge among all selected rows. *Data_type*: bool, optional
`-right`	Specifies that the right edge of all selected rows should be aligned to the right-most edge among all selected rows. *Data_type*: bool, optional
`-toCoreEdge`	Specifies that row edges should be aligned to the core boundary. *Data_type*: bool, optional

Example

The following command specifies that the left edge of all selected rows should be aligned to the left-most edge among all selected rows.

```
stretchRows -left
```

Related Information

- `deleteRow`
- `cutRow`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Creating and Editing Rows

swapPins

```
swapPins
[-help]
```

Switches the locations of two pins. You can use this command after selecting two pins in the same module in the design display area.

Parameters

`-help`	Outputs a brief description of the `swapPins` command. For a detailed description of the command, use the man command: `man swapPins`

Example

- The following commands show the method to swap two pins in command line:

  ```
  deselectAll
  selectPin pin1
  selectPin pin2
  swapPins
  ```

Related Information

- `deselectAll`
- `selectPin`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Editing Pins

trace_obj_connectivity

```
trace_obj_connectivity
[-help]
[-clear_trace]
[-level level]
[-mode {netlist_based timing_based}]
[-out_file file_name]
[-insts string] | [-ports string] | [-selected ] | [-in_files file_name]
```

Traces the connections of specified macros, selected macros, or specified ports. You can also trace the connections based on netlist or timing. It supports different levels. You can also set global parameters for the `trace_obj_connectivity` by using the `set_trace_obj_connectivity_mode` command. Parameters that you specify with `set_trace_obj_connectivity_mode` are then used automatically whenever you run the `trace_obj_connectivity` command or use the *Trace Macro* form. You can use this command after `proto_design`, `place_design`, or even after the whole design is completed.

Parameters

`-clear_trace`	Clears the existing trace result. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes type and default information for each `trace_obj_connectivity` parameter. For a detailed description of the command and all of its parameters, use the man command: `man trace_obj_connectivity`
`-in_files file_name`	Specifies the name of the file from which saved trace macro results are loaded. *Data_type*: string, optional
`-insts string`	Specifies the macros to be traced. *Data_type*: string, optional
`-level level`	Specifies the trace level. *Maximum Level*: 10 *Minimum Level*: 1 *Default*: 1 *Data_type*: int, optional
`-mode {netlist_based timing_based}`	
	Specifies the trace mode to use: • `netlist_based`: Specifies that connection of the macro will be traced based on the netlist. • `timing_based`: Specifies that connection of the macro will be traced based on advanced timing tcl scripts command `all_fanin` and `all_fanout`. *Data_type*: enum, optional
`-out_file file_name`	Specifies the name of the file in which the trace macro results are saved. *Default*: `<top_module_name>.tmcdb` *Data_type*: string, optional
`-ports string`	Specifies the names of the ports to be traced. *Data_type*: string, optional
`-selected`	Traces the selected macros or ports. *Data_type*: bool, optional

Example

The following command traces the connections for blk1 upto level 3:

```
trace_obj_connectivity -insts blk1 -level 3
```

Related Information

- `set_trace_obj_connectivity_mode`
- `get_trace_obj_connectivity_mode`
- `display_obj_connectivity`
- `all_fanin`
- `all_fanout`
- Floorplanning the Design chapter in the *Innovus User Guide*
- Floorplan Menu chapter in the *Innovus Menu Reference*

unfixAllIos

```
unfixAllIos
[-help]
[-incAreaIo]
```

Changes the status of all I/Os from `FIXED` to `PLACED`. If any I/O has a status other than fixed prior to running this command, that status will be retained. You can use this command after importing the design.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `unfixAllIos` parameter. For a detailed description of the command and all of its parameters, use the man command: `man unfixAllIos`
`-incAreaIo`	Changes the status of all `CLASS PAD AREAIO` cells from a `FIXED` state to a `PLACED` state. *Data_type*: bool, optional

Examples

- The following command changes the status of all I/O pins and I/O cells from a `FIXED` state to a `PLACED` state:
  ```
  unfixAllIos
  ```
- The following command changes the status of all I/O pins, I/O cells, and all `CLASS PAD AREAIO` cells from a `FIXED` state to a `PLACED` state:
  ```
  unfixAllIos -incAreaIo
  ```

Related Information

- `fixAllIos`
- Floorplanning the Design chapter in the *Innovus User Guide*
- Data Preparation chapter in the *Innovus User Guide*
  - Performing Area I/O Placement

unplaceAllBlocks

```
unplaceAllBlocks
[-help]
```

Unplaces all blocks in the floorplan. You can use this command after importing the design.

Parameters

`-help`	Outputs a brief description of the `unplaceAllBlocks` command. For a detailed description of the command, use the man command: `man unplaceAllBlocks`

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

unplaceAllGuides

```
unplaceAllGuides
[-help]
```

Clears all constraints (guides, regions, and fences) from the floorplan. You can use this command after importing the design.

Note: The `unplaceAllGuides` command does not delete instGroups.

Parameters

`-help`	Outputs a brief description of the `unplaceAllGuides` command. For a detailed description of the command, use the man command: `man unplaceAllGuides`

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

unplaceAllInsts

```
unplaceAllInsts
[-help]
```

Unplaces all instances from the floorplan including blobs and std cells whose status is PLACED. It changes the status from PLACED to UNPLACED. You can use this command after importing the design.

Note: The `unplaceAllInsts` command does not deal with the std. cells whose status is FIXED, SOFTFIXED, or COVER.

Parameters

`-help`	Outputs a brief description of the `unplaceAllInsts` command. For a detailed description of the command, use the man command: `man unplaceAllInsts`

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

unplaceGuide

```
   unplaceGuide
[-help]
obj_name
```

Unplaces hierarchical instances and instance groups from the floorplan. This command can also be used to delete fences and regions. Guides are hierarchical instances in the design and are guided in the floorplan. You can use this command after importing the design.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `unplaceGuide` parameter. For a detailed description of the command and all of its parameters, use the man command: `man unplaceGuide`
`obj_name`	Specifies the name of the module or group to be guided. *Data_type*: string, required

Examples

- The following command unplaces the guide MEM_DSCAN from the floorplan:
  ```
  unplaceGuide MEM_DSCAN
  ```
- The following command unplaces the instance group adder1 from the floorplan:
  ```
  unplaceGuide adder1
  ```

Related Information

- `createFence`
- `createRegion`
- Floorplanning the Design chapter in the *Innovus User Guide*

unplaceGuideConstraints

```
unplaceGuideConstraints
[-help]
```

Clears only the guide constraints from the floorplan. You can use this command after importing the design.

Parameters

`-help`	Outputs a brief description of the `unplaceGuideConstraints` command. For a detailed description of the command, use the man command: `man unplaceGuideConstraints`

Related Information

- Floorplanning the Design chapter in the *Innovus User Guide*

unsetFixedBlockSize

```
  unsetFixedBlockSize
[-help]
name_list
```

Specifies that during floorplan resizing, the size of the specified modules and/or blackboxes can change. Use this command to unset the fixed block size specified with the `setFixedBlockSize` command. You would use this command after setting the block size of modules and/or black boxes to fixed with the `setFixedBlockSize` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `unsetFixedBloxkSize` parameter. For a detailed description of the command and all of its parameters, use the man command: `man unsetFixedBlockSize`
`name_list`	Specifies that the size of the specified modules and/or blackboxes can change during floorplan resizing. Provide a space-separated list of instances. `*`: Specifies that the size of all the modules and/or blackboxes can change during floorplan resizing. *Data_type*: list, required

Example

The following command specifies that the size of instanceA and instanceB can change during floorplan resizing.

```
unsetFixedBlockSize instanceA instanceB
```

Related Information

- `setFixedBlockSize`
- Floorplanning the Design chapter in the *Innovus User Guide*

update_floorplan_for_macro_place

```
update_floorplan_for_macro_place
[-help]
[-init_rows]
[-keep_fixed_insts {inst1 inst2 ...}]
[-remove {all route_blockage place_blockage relative_floorplan power_domain insts_place wire boundary_constraint physical_inst drc PG_pin}]
```

Enables you to update the floorplan before defining constraints for macros with the `set_macro_place_constraint` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `update_floorplan_for_macro_place` parameter. For a detailed description of the command and all of its parameters, use the man command: `man update_floorplan_for_macro_place`
`-init_rows`	Creates core rows. *Data_type*: bool, optional
`-keep_fixed_insts {inst1 inst2 ...}`	
	Keeps the specified instances in the floorplan. It sets the instances which are not unplaced. *Data_type*: string, optional
`-remove {all route_blockage place_blockage relative_floorplan power_domain insts_place wire boundary_constraint physical_inst drc PG_pin}`	
	Removes the specified objects from the floorplan. *Data_type*: enum_list, optional

Example

- The following command initializes the core rows and deletes the placement blockages from the floorplan:

  ```
  update_floorplan_for_macro_place -init_rows -remove place_blockage
  ```

Related Information

- `create_pg_model_for_macro_place`
- `get_macro_place_constraint`
- `reset_macro_place_constraint`
- `set_macro_place_constraint`
- `write_macro_place_constraint`
- Floorplanning the Design chapter in the *Innovus User Guide*
- Using the Mixed Placer chapter in the *Innovus User Guide*

write_macro_place_constraint

```
write_macro_place_constraint
[-help]
[-cpg_scope {selected macro_only all}]
-out_file string
[-sections {array basic cpg pg_model_over_macros pg_resource_model}]
```

Enables you to write constraints into a TCL format file from the database.

Parameters

`-cpg_scope {selected macro_only all}`	
	Specifies the instances that are to be written in the CPG guide section. *Data_type*: enum, optional
`-help`	Outputs a brief description that includes the type and default information for each `write_macro_place_constraint` parameter. For a detailed description of the command and all of its parameters, use the man command: `man` write_macro_place_constraint
`-out_file string`	Specifies the name of the file to which the macro place constraints are written. *Data_type*: string, optional
`-sections {array basic cpg pg_model_over_macros pg_resource_model net_length net_resistance}`	
	Specifies the constraint sections to be written. • `array`: Writes macro array constraints. • `basic`: Writes non-default macro place constraints set by the `set_macro_place_constraint` command. • `cpg`: Writes out the guide file for initializing the orientation of the instances and placing them close to the specified location. • `pg_model_over_macros`: Writes PG modeling for macros. • `pg_resource_model`: Writes PG resource modeling for standard cell areas. **Note**: This option is used to model PG over the whole design if the `-pg_model_over_macros` option is not specified. **Note:** Only the basic constraints are dumped out if the -sections option is not specified. *Data_type*: enum_list, optional

Example

- The following command writes the basic macro place constraints to the xyz file:
  ```
  write_macro_place_constraint -cpg_score all -out_file xyz
  ```

Related Information

- `check_macro_place_constraint`
- `create_pg_model_for_macro_place`
- `get_macro_place_constraint`
- `set_macro_place_constraint`
- `reset_macro_place_constraint`
- Floorplanning the Design chapter in the *Innovus User Guide*
- Using the Mixed Placer chapter in the *Innovus User Guide*

writeFPlanScript

```
writeFPlanScript
[-help]
[-appendToFile]
[-compactRow]
-fileName fileName
[-no_snap_to_grid]
[-selected | -sections {boundary row placeBlockages routeBlockages pinBlockages groups constraints pins ioPad areaIO bump partitions blocks
globalNetConnect busSinkGroups blackboxes netGroupAndBusGuide relativefplan pinGroups pinGuides customShape coverCell}]
```

Allows you to write out the specified floorplan sections and source the output file after init_design. With this command the tool would write a file that you can examine, edit, and then source to selectively (or completely) define your floorplan. You can specify the floorplan section to write out. The `writeFPlanScript` command writes out a file with Tcl commands to recreate the floorplan. It can optionally write out specific floorplan sections listed with option -sections. The file includes commands to delete any existing data for each section if any exists.

Note: Unlike `saveFPlan`, the `writeFPlanScript` command does not include special-routing like P/G routing. You should use `defOutBySection -specialNetRouting` for that purpose.

Note: The `createBusGuide -type {hard | soft}` command supports soft/hard constraints to guide Innovus routers. With the hard constraint type, after running Early Global Route (eGR), NanoRoute (NR), or NanoRoute High Frequency Router (NRHF), the route obeys the path of the bus guide but with soft constraints the bus guide only guides the route path. The tool can route the net out of the bus guide. The `writeFPlanScript` command writes out these constraints types to the output file.

Parameters

`-appendToFile`	Appends new data to the file, by default it will write out to the file specified by `-fileName`. *Data_type*: bool, optional
`-compactRow`	When specified, it enables the `writeFPlanScript` command to write out the row-creation commands in a simple and compact way. *Data_type*: bool, optional
`-fileName` *fileName*	Writes the floorplan Tcl commands to the specified file. *Data_type*: string, required
`-help`	Outputs a brief description that includes the type and default information for each `writeFPlanScript` parameter. For a detailed description of the command and all of its parameters, use the man command: `man writeFPlanScript`
`-no_snap_to_grid`	Specifies that the object should not be snapped to the specified grid and keep the location of object the same before and after loading the dumped script. *Data_type*: bool, optional
`-sections {boundary row placeBlockages routeBlockages pinBlockages groups constraints pins ioPad areaIO bump partitions blocks globalNetConnect busSinkGroups blackboxes netGroupAndBusGuide relativefplan pinGroups pinGuides customShape coverCell}`	

	Specifies one or more specific floorplan sections to write out to the file. • `areaIO`: Writes out areaIO information. • `blackboxes`: Writes out location, shape, and orientation of black box instances. • `blocks`: Writes out placement and orientation of all blocks. • `boundary`: Writes out the floorplan boundary, including the boundary to IO and core box spacing, and the global row parameters required by the `floorPlan` command to regenerate the rows automatically. **Note**: The `floorPlan` command will auto-generate the rows also, unless "rows" is included with `-sections` to override them with the current rows. • `bump`: Writes out all the bumps. • `busSinkGroups`: Writes bus sink group information. • `constraints`: Writes out any region, fence, and guide constraints for groups or hinsts. • `coverCell`: Writes out cover cell information. • `customShape`: Writes out custom shape information. • `globalNetConnect`: Writes out all the global net connections. • `groups`: Writes out any group definitions (group names and list of instances). It also writes out the groups belonging to power domains. • `ioPad`: Writes out all the IO pads. • `netGroupAndBusGuide`: Writes out all the net groups and bus guides. • `partitions`: Writes out all the partitions. • `pinBlockages`: Writes out all pin blockages. • `pinGroups`: Writes out all the pin groups. • `pinGuides`: Writes out all the pin guides. • `pins`: Writes out all the external IO pins. • `placeBlockages`: Writes out placement/density blockages. • `relativefplan`: Writes out relative floorplan constraints and all the placement relationship of floorplan objects independently from the actual coordinates in a floorplan. **Note**: You can use `writeFPlanScript -sections relativefplan` to save relative floorplan info. • `routeBlockages`: Writes out routing blockages. • `row`: Writes out all the core and IO rows. *Data_type*: enum_list, optional
`-selected`	Specifies the floorplan object to write out. **Note:** The tool does not support to write out standard cells. *Data_type*: bool, optional

Example

- The following command writes out the user specified (row) section to the `writeFPlanScript.tcl` file.

  ```
  writeFPlanScript -sections {row} -fileName writeFPlanScript.tcl
  ```

  In writeFPlanScript.tcl, something like the following will be written out:

  ```
  # Standard Cell Row
  deleteRow -all
  set rdagFPlanRowSpaceType 2
  set rdagFPlanRowSpacing 0
  createRow -site tsm3site -area 335.280 335.440 1176.780 340.480 -flip1st -noCheck
  createRow -site tsm3site -area 335.280 340.480 1176.780 345.520 -noCheck
  createRow -site tsm3site -area 335.280 345.520 1176.780 350.560 -flip1st -noCheck
  createRow -site tsm3site -area 335.280 350.560 1176.780 355.600 -noCheck
  createRow -site tsm3site -area 335.280 355.600 1176.780 360.640 -flip1st -noCheck
  createRow -site tsm3site -area 335.280 360.640 1176.780 365.680 -noCheck
  ```

```
...
createRow -site tsm3site -area 335.280 1172.080 1176.780 1177.120 -flip1st -noCheck
fpSetDefaultTechSite -site tsm3site
```

- The following command by default writes out all the sections to the `writeFPlanScript.tcl` file

  ```
  writeFPlanScript -fileName writeFPlanScript.tcl
  ```

  In `writeFPlanScript.tcl`, something like the following will be written out:

  ```
  # floorplan boundary
  set ret [catch {floorplan -noResize -noSnapToGrid -b 0.000 0.000 1511.960 1512.120 235.000 235.000 1276.960 1277.120 335.280 335.440
  1176.960 1177.120 }]
  if {$ret != 0} {
  floorplan -noSnapToGrid -b 0.000 0.000 1511.960 1512.120 235.000 235.000 1276.960 1277.120 335.280 335.440 1176.960 1177.120
  }
  # Standard Cell Row
  deleteRow -all
  set rdagFPlanRowSpaceType 2
  set rdagFPlanRowSpacing 0
  createRow -site tsm3site -area 335.280 335.440 1176.780 340.480 -flip1st -noCheck
  createRow -site tsm3site -area 335.280 340.480 1176.780 345.520 -noCheck
  ...
  fpSetDefaultTechSite -site tsm3site
  # Placement obstruction
  deletePlaceBlockage -all
  # Routing obstruction
  deleteRouteBlk -all
  # Pin blockage
  deletePinBlkg -all
  # Instance group
  # Hierarchical Instance's constraint(Fence, Region or Guide)
  # Pin
  setPinAssignMode -pinEditInBatch true
  setPinAssignMode -pinEditInBatch false
  # IO pad
  dbPlaceInst [dbGetInstByName IOPADS_INST/Pibiasip] [uu2dbu 0.000] [uu2dbu 548.160] R270
  setInstancePlacementStatus -status fixed -name IOPADS_INST/Pibiasip
  ...
  # Area IO
  # Bump
  deleteBumps -all
  # Block placement
  unplaceAllBlocks
  deleteHaloFromBlock -allBlock
  placeInstance DTMF_INST/PLLCLK_INST 355.280 355.440 R0
  addHaloToBlock 20 20 20 20 DTMF_INST/PLLCLK_INST
  setInstancePlacementStatus -status fixed -name DTMF_INST/PLLCLK_INST
  ...
  # Global Net Connection
  clearGlobalNets
  globalNetConnect VDD -pin VDD -instanceBasename * -type pgpin -all
  globalNetConnect VSS -pin VSS -instanceBasename * -type pgpin -all
  ...
  applyGlobalNets
  # partition
  # netGroupAndBusGuide
  # relative floorplan
  ```

- The following command writes out the user specified (customShape) section to the `writeFPlanScript.tcl` file

  ```
  writeFPlanScript -sections {customShape} -fileName writeFPlanScript.tcl
  ```

  In `writeFPlanScript.tcl`, something like the following will be written out:

  ```
  # custom shapes
  delete_gui_object -shape
  add_gui_shape -layer <layer>  -line {x1 y1 x2 y2}
  add_gui_shape -layer <layer>  -polygon {x1 y1 x2 y2 …}
  ```

Related Information

- `saveFPlan`
- `defOutBySection`
- `createBusGuide`
- Floorplanning the Design chapter in the *Innovus User Guide*
  - Saving and Loading Floorplan Data

add_interposer_route_block

```
add_interposer_route_block
[-help]
-cell cell_name
-inst clone_name
-location {x y}
-orient {R0 R90 R180 R270 MX MY MX90 MY90}
```

Adds the clone blocks. It transforms the similar unrouted areas to a clone partition, which repeats the same routing pattern as the routed master block. This command enables you to replicate the routed master block multiple times with different orientations.

Note: Before using the `add_interposer_route_block` command, you must launch another Innovus session for loading the initial DB followed by using `create_interposer_route_block` command to create the routed master block partition by specified box. This is used to generate master block partition and top DB.

Parameters

`-cell cell_name`	Specifies the master cell name that needs to be repeated (string, required) *Data_type*: string, required
`-help`	Outputs a brief description that includes the type and default information for each `add_interposer_route_block` parameter. For a detailed description of the command and all of its parameters, use the man command: `man add_interposer_route_block`
`-inst clone_name`	Specifies the clone block name. *Data_type*: string, required
`-location {x y}`	Specifies the clone block's origin location. **Note:** The relative location between clone block's origin and the terms should exactly be the same with the master. Otherwise, the wire of clone blocks will be shift accordingly after assemble design. **Note:** The snapping grid of the created block honors the `setFPlanMode -snapBlockGrid` command settings. *Data_type*: point, required
`-orient {R0 R90 R180 R270 MX MY MX90 MY90}`	
	Specifies the orientation of the clone block compared with the master. *Data_type*: enum, required

Examples

- The following command replicates the routing pattern of the master cell (`master_block`) to the clone block (`clone_block_1`) at location x1 y1 at orientation R0.
  ```
  add_interposer_route_block -cell master_block -inst clone_block_1 -location {x1 y1} -orient R0
  ```
- The following command repeats replicating the routing pattern of the master cell (`master_block`) to the clone block (`clone_block_2`) at location x2 y2 at orientation R180.
  ```
  add_interposer_route_block -cell master_block -inst clone_block_2 -location {x2 y2} -orient R180
  ```

Related Information

- create_interposer_route_block
- "Repeating Routing Patterns in Interposer Designs" in the *Innovus User Guide*

create_interposer_route_block

```
create_interposer_route_block
[-help]
-box {x1 y1 x2 y2}
-cell cell_name
-dir dir_name
-inst master_name
```

Creates the master block. It transforms the routed domain to a master partition block that enrolls all the special routes.

Note: If the design does not have a routing pattern that can be treated as a master block, the functionality of the `create_interposer_route_block` command can be replaced by loading the LEF file of the master partition that has the same routing pattern using the `loadLefFile` command. After this, the `add_interposer_route_block` command can be used directly.

Parameters

`-box {x1 y1 x2 y2}`	Specifies the master block's bounding box. **Note**: The snapping grid of the created block honors the `setFPlanMode -snapBlockGrid` command settings. *Data_type*: box, required
`-cell cell_name`	Specifies the master cell name. *Data_type*: string, required
`-dir dir_name`	Specifies the directory name that will contain the partition information and files. The tool creates the directory and saves the information and files for the top and master block partition automatically after the `create_interposer_route_block` command is used. *Data_type*: string, required
`-help`	Outputs a brief description that includes the type and default information for each `create_interposer_route_block` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_interposer_route_block`
`-inst master_name`	Specifies the master block name. **Note:** The default orientation of the created master block is R0. *Data_type*: string, required

Examples

- The following command transforms the routed domain to a master partition block.
  ```
  create_interposer_route_block -cell cell_name -inst master_block -box {llx lly urx ury} -dir PTN
  ```

Related Information

- `add_interposer_route_block`
- `loadLefFile`
- "Repeating Routing Patterns in Interposer Designs" in the *Innovus User Guide*

10

General Commands and Global Variables

- General Commands
- General Global Variables

General Commands

- alias
- bindKey
- check_ccr
- check_syntax
- checkNetlist
- compare_release
- decrypt
- define_proc_arguments
- deleteDangling1b1Or0s
- deleteDanglingPort
- deselect_obj
- encMessage
- encrypt
- eval_common_ui
- find_global

- generateFFSetupFile
- get_message
- get_metric
- getAllLayers
- getBuildArch
- getCheckMode
- getCompressLevel
- getLayerPreference
- getPreference
- getVersion
- gui_get_legend
- gui_set_legend
- help
- is_common_ui_mode
- lminus
- man
- parse_proc_arguments
- placeCursor
- print_message
- Puts
- read_taf
- redirect
- rename_obj
- report_area
- report_command_mode
- report_disk
- report_hidden_usage

- report_message
- reportDanglingPort
- reportGateCount
- reportNetStat
- resume
- savePreference
- select_obj
- selectObjByProp
- set_global_always_keep
- set_message
- set_proc_verbose
- setCheckMode
- setCompressLevel
- setLayerPreference
- setLicenseCheck
- setLimitedAccessFeature
- setMessageLimit
- setPreference
- setTopCell
- setWindowPreference
- source
- summaryReport
- suppressMessage
- suspend
- time_info
- uiGetRecordObjByInfo
- unsetMessageLimit

- unsuppressMessage
- update_names
- viewLog
- vPuts
- win
- writeFlowTemplate

alias

```
alias
[-help]
new_cmd
old_cmd
[-args {{new old} ...}]
```

Create command aliases for any Tcl command. It is a wrapper around the built-in Tcl “interp alias” command. All command aliases are public, so partial command matching and using <tab> to complete a typed in command work on them. It also supports argument name aliases for procs that use `define_proc_arguments` and `parse_proc_arguments` for parsing the arguments, and for application commands. The argument aliases do not work for built-in Tcl commands. The argument aliases are private, so they are not shown by `-help`, and do not support partial-option completion or matching.

You can use this command to migrate an existing Tcl script when some of the commands or argument names have changed, but are otherwise compatible.

Parameters

`-help`	Prints a brief description that includes type and default information for each `bindkey` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man bindkey`
`new_cmd`	Specifies the new command name that is an alias for the existing *`old_cmd`* name. It can be the same name as the *`old_cmd`* if you only want to create new `-arg` aliases, and do not need a new command alias. *Data_type*: string, required
`old_cmd`	Specifies the existing old command name. *Data_type*: string, required
`-args {{`*`new old`*`} …}`	Specifies a list of new and old argument names to create aliases. You can give multiple *`new`* names for the same *`old`* name. The new aliases are private, and do not show up with `-help`. You can create more than one alias for the same *`old`* name. *Data_type*: string, optional

Example

The following command adds a new command name alias `new_read_sdf` and some arg name aliases for `read_sdf`:

```
>alias new_read_sdf read_sdf -args {
  {-ignore_error -continue_on_error}
  {-summary -report_summary}
  {-sum -report_summary}
}
```

Then, the following commands

```
>new_read_sdf in.sdf -ignore_error -summary
>new_read_sdf in.sdf -ignore_error -sum
>new_read_s in.sdf -ignore_error -sum
```

would behave the same as

```
>read_sdf in.sdf -continue_on_error -report_summary
```

Note: Tcl partial command matching works if the partial command is unique as shown in the above example, but the alias argument names are private, and will not support partial option matching, or appear in the `-help` output.

Related Topics

- `define_proc_arguments`
- `parse_proc_arguments`

bindKey

```
bindKey key cmd
[-help]
```

Enables you to create keyboard shortcuts.

You can use this command at any time after starting the Innovus software.

Parameters

`-help`	Prints a brief description that includes type and default information for each `bindkey` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man bindkey`
`key`	Specifies the key to be used as the shortcut. You can specify a: • Single Key-Specify any one of the following keys: ◦ `a - z` ◦ Function keys: use `F1 - F12` ◦ Arrow keys: use `Up Down Left Right` ◦ Number keys ◦ Unshifted punctuation keys: `. , / - =` ◦ Other keys: use `Space`, `Delete`, `Backspace`, or `Esc` • Shift Key Combination-Specify `Shift` as the first key followed by '-' followed by: ◦ `A - Z` ◦ One of the 'shifted' punctuation chars: `~@#$%^&*()_+` ◦ Function keys: use `F1 - F12` ◦ Arrow keys: use `Up Down Left Right` ◦ Other keys: use `Space`, `Delete`, `Backspace`, or `Esc` • Ctrl/Alt Key Combination-Specify `Ctrl` or `Alt` as the first key followed by '+' followed by any of the single keys above *Data_type*: string, optional
`cmd`	Specifies the command to be executed when key is pressed. *Data_type*: string, optional

Example

- Runs the `fit` command when you press `f` on your keyboard:

```
bindKey f "fit"
```

- The following example shows how to use `bindKey` when # and @ keys are pressed from GUI.

```
bindKey Shift-@ "setPreference StretchRectilinear 1; setPreference
StretchRestriction 1"
bindKey Shift-# "setPreference StretchRectilinear 0; setPreference
StretchRestriction 0"
```

Related Topics

- Binding Key section of View Menu in *Innovus Menu Reference*

check_ccr

```
check_ccr
[-help]
ccr_number
```

Enables users to check if a CCR (Cadence Change Request) created on their behalf is checked into the current release or not.

The command returns 1 if the specified CCR is integrated into this build and 0 if it is not.

- It includes all integrated CCRs that have the `Checked_In` or `Validated` status at the time of the build.
- It only includes master CCRs. Duplicate CCRs are not included.
- It currently lists all CCRs fixed in the current release stream or previous release. For example, checking in release version 19.11, will find all CCRs integrated into 19.10 through 19.11 that are in the current 19.11 release.

Parameters

`-help`	Prints a brief description that includes type and default information for each `check_ccr` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man check_ccr`
`ccr_number`	Specifies the CCR number to check. *Data_type*: string, required

Examples

- The following example checks if the CCR 123456 is checked into the Innovus release 18.13-s088_1.

```
innovus 1> getversion
18.13-s088_1

innovus 2> check_ccr 123456
1
```

check_syntax

```
check_syntax
[-help]
[file1.tcl file2.tcl ...]
[nagelfar.tcl options]
[-options]
[-out_file fileName]
```

Does the “lint type” checking of Innovus Tcl script files.

It finds many common Tcl errors, such as misspelled commands, using a Tcl variable before setting it.

The `check_syntax` command is a wrapper around the public-domain Tcl linter named nagelfar. Nagelfar automatically loads the Innovus syntax and plugin files and is therefore aware of Innovus commands and options. Nagelfar is available from http://sourceforge.net/projects/nagelfar. A recent version of nagelfar is included in the installation hierarchy at: `<install_dir>/share/nagelfar`.

You can check the nagelfar documentation in the installation directory for details about its options, the checks it performs, and the steps to turn on or off various checks.

To run this command, you should have tclsh in your search path.

Parameters

`-help`	Prints a brief description that includes type and default information for each check_syntax parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man check_syntax`
`file1.tcl` *`file2.tcl`*	Specifies the Tcl files to be checked. *Data_type*: string, optional
`nagelfar.tcl options`	Specifies the native nagelfar.tcl options. See the nagelfar docs in the installation directory for details. *Data_type*: string, optional
`-options`	Calls “nagelfar.tcl –help” to list out all the legal nagelfar options. This option cannot be used with any other options. *Data_type*: bool, optional
`-` `out_file` *`fileName`*	Sends the output to the specified file, rather than the default of stdout. *Data_type*: string, optional

Examples

- The following example show the usage of the `check_syntax` command for file `a.tcl`:

```
innovus> check_syntax a.tcl
Checking file a.tcl
Line 2: W Unknown command "typo_in_cmd_name"
Line 3: E Unknown variable "abc"
```

checkNetlist

```
checkNetlist
-outfile filename
[-includeSubModule]
```

Performs several checks on the design netlist and creates a report. The report provides the following information:

- Design Summary
- Design Statistics
- I/O Pad Summary
- Design Rule Checking
- Submodule Port Definition Checking (optional)

You can use this command after the design netlist is imported.

Parameters

`-outfile` *filename*	Specifies the name of the file where the report will be written. If no name is specified, the default filename is `checknetlist.rpt`. *Data_type*: string, optional
`-includeSubModule`	If this option is included, `checkNetlist` checks the port definitions for any submodules. *Data_type*: bool, optional

Example

Error conditions are indicated by an asterisk (*) preceding the reported statistic, such as

```
Number of Ports Connect to Core Cells *: 57

Number of Ports Connect to multiple Pads *: 0

Number of Floating Ports *: 0
```

Warnings, or possible errors are indicated by a question mark in front of the statistic, such as

```
Number of undefined Floating Pins ?: 0
```

The following example shows the information contained in the generated by `checkNetlist` report.

```
Design: TOPModule

------ Design Summary:
Total Standard Cell Number (cells) : 5908
Total Block Cell Number (cells) : 4
Total I/O Pad Cell Number (cells) : 0
Total Standard Cell Area ( um^2) : 133864.32
Total Block Cell Area ( um^2) : 366346.56
Total I/O Pad Cell Area ( um^2) : 0.00

------ Design Statistics:
Number of Instatnces : 5912
Number of Nets : 6217
Average number of Pins per Net : 3.69
Maximum number of Pins in Net : 541

------ I/O Pad summary
Number of Primary I/O Ports : 57
Number of Input Ports : 28
Number of Output Ports : 29
Number of Bidirectional Ports : 0
Number of Power/Ground Ports : 0
Number of Ports Connect to Core Cells *: 57
Number of Ports Connect to multiple Pads *: 0
Number of Floating Ports *: 0

------ Design Rule Checking:
Number of Output Pins connect to Power/Ground *: 0
Number of Input Floating Pins *: 1
Number of Output Floating Pins : 245
Number of InOut Floating Pins : 0
Number of UniPort Floating Pins : 0
Number of undefined Floating Pins ?: 0
Number of Multiple Driving Nets *: 1
Number of Parallel Driving Nets : 1
Number of Tristate Driving Nets : 0
Number of Input Floating Nets(No FanIn) *: 0
Number of Output Floating Nets(No FanOut) : 1
Number of Hign Fanout Nets (>50) : 3

------ Sub Module Port Definition Checking
* module arb has 3 port define error
* module accum_stat has 1 port define error
* module tdsp_core_glue has 128 port define error
```

Related Topics

- Data Preparation chapter of *Innovus User Guide*

compare_release

```
compare_release
[-help]
[-out_file file_name]
{-base_version release_version | -base_file file_name}
[-target_version release_version | -target_file file_name]
```

Compares two different releases and reports all the added, deleted, or modified commands, arguments, and global variables. It also reports any data-type changes for argument values or globals and any enum value changes.

Parameters

`-help`	Prints a brief description that includes type and default information for each `compare_release` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man compare_release`
`-base_file` *`file_name`*	Specifies the `.info` file of the older base release. *Data_type*: string, optional
`-base_version` *`release_version`*	Specifies the older base release, such as 16.10-p001_1, from *`install`*`/share/innovus/version_info`. *Data_type*: string, optional
`-out_file` *`file_name`*	Specifie the output report file. *Default*: `stdout` *Data_type*: string, optional
`-target_file` *`file_name`*	Specifies the target release .info file to be compared with the older base release .info file. *Data_type*: string, optional
`-target_version` *`release_version`*	Specifies the version, such as 16.20-p002_1, which is compared to the older base release. Looks in *`install`*`/share/innovus/version_info` for the correct `.info` file. *Default*: Current version *Data_type*: string, optional

Example

- The following command compares two releases by first finding the install directories, and the correct .info files to compare.

  ```
  >which innovus

  /tools/innovus18.11/bin/innovus
  ```

  The install directory above is /tools/innovus18.11 (for examples, the 18.1 ISR release).

  Suppose you also have the 18.1 production release in /tools/innovus.18.10. Find the .info files to compare:

  ```
  >ls /tools/innovus18.10/share/innovus/version_info

  innovus.18.10-p006_1.cui.info  innovus.18.10-p006_1.info

  >ls /tools/innovus18.11/share/innovus/version_info

  innovus.18.11-s080_1.cui.info  innovus.18.11-s080_1.info
  ```

  Then, run the `compare_release` command to compare either the legacy (.info.) or Common UI versions (.cui.info):

  ```
  innovus> compare_release \

  -base_file /tools/innovus18.10/share/innovus/version_info/innovus.18.10-
  p006_1.info \

  -target_file /tools/innovus18.11/share/innovus/version_info/innovus.18.11-
  s080_1.info \

  -out_file 18.11.diff
  ```

  The output file will show all the differences between the two releases.

decrypt

```
decrypt
[-help]
file_name
```

Decrypts and evaluates a Tcl file that was encrypted with the encrypt command.

Parameters

`-help`	Prints out the command usage.
`file_name`	Specifies the name of the Tcl file to be decrypted and evaluated. *Data_type*: string, required

Related Topics

- `encrypt`

define_proc_arguments

```
define_proc_arguments
[-help]
command_name
[-info string]
[-define_args string]
[-hide]
```

Defines the arguments and attributes of a Tcl procedure. The arguments and attributes are displayed when you view the help associated with the procedure.

The `define_proc_arguments` command, when used with the `parse_proc_arguments` command, enables you to validate command usage. For invalid commands, appropriate error messages are displayed.

Parameters

`-help`	Prints out the command usage.
`proc_name`	Specifies the name of an existing Tcl procedure. *Data_type*: string, optional
`-info` *`string`*	Provides help information for the procedure. *Data_type*: string, optional
`-define_args` *`string`*	Defines all possible arguments of the Tcl procedure. It can also be used to define the help text and constraints for individual arguments of the procedure and generates the help for command options. The variable *`arg_defs`* is a list of lists where each list element defines one argument. Each element in the list has the following format: `"arg_name" "option_help" "value_help" "data_type" "attributes"` *Data_type*: string, optional
	`arg_name`: Specifies the name of the argument.
	`option_help`: Provides a short description of the argument.
	`value_help`: Specifies the help information for the associated option values.

	data_type: (Optional) Used for validation of options. Supported data types are `string`, `list`, `boolean`, `int`, `integer`, `float`, or `one_of_string`. *Default:* `string`
	attributes: (Optional) Used for validation of options. The attributes can be: • `required`: This argument must be specified . This is mutually exclusive with `optional` and `internal`. • `optional`: Specifying this argument is optional. This is mutually exclusive with `required`. • `internal`: Disables the display of help information for the option in the procedure help. This argument is mutually exclusive with `required`. • `value_help`: Indicates that the valid values for `one_of_string` argument should be listed whenever argument help is shown. • `values {`*`allowable_values`*`}`: This argument must be used with the `one_of_string` data type. • `merge_duplicates` : Ensures that when the option appears more than once in a command, its values are concatenated into a list of values. The default behavior is that the right-most value specified for the option is used. • `bind_option` *`other_arg_name`*: Binds the current option with the option specified as the *`other_arg_name`* variable. • `mutual_exclusive_group` *`group_name`*: Defines a group name and includes the current option in the *`group_name`* group. All options in the same group are treated as mutually exclusive. • *Default:* `optional`
`-hide`	Hides the procedure and its body. *Data_type*: bool, optional

Examples

The following Tcl procedure uses the `define_proc_arguments` command for defining arguments and attributes, and for validating the command usage.

```
proc cmdProc {args} {

parse_proc_arguments -args $args results

foreach argname [array names results] {

puts "$argname = $results($argname)"

}

}

define_proc_arguments cmdProc -info "Command Description" \

-define_args \

{{-string "String help" "string_val" string optional}

{-integer "Int help" "int_value" int optional}

{-Bool "Bool_help" "bool_value" boolean optional}

{-list "List help" "list_value" list {optional bind_option -Bool}}

{-boolean_a "BoolA help" "" boolean {optional mutual_exclusive_group

g0}}

{-boolean_b "BoolB help" "" boolean {optional mutual_exclusive_group

g0}}

{-float_num "Float help" "float_value" float}

{-oosa "One of StringA help" "one_of_str_value" one_of_string

{optional value_help {values {v1 v2}}}}

{-oosb "One of StringB help" "one_of_str_value" one_of_string

{optional {values {v4 v5}}}}

}
```

Now, if the `cmdProc` procedure is invoked with the `-help` option at the command prompt, the following help information is displayed.

```
cmdProc -help
```

```
Usage: cmdProc # Command Description

[-string string_val] (String help)

[-integer int_value] (Int help)

[-list list_value] (List help, Bind_Option=-Bool)

[-boolean_a ] (BoolA help, Mutual_Exclusive_Group=g0)

[-boolean_b ] (BoolB help, Mutual_Exclusive_Group=g0)

[-float_num float_value] (Float help)

[-oosa one_of_str_value] (One of StringA help, Values={v1 v2 })

[-oosb one_of_str_value] (One of StringB help)
```

Related Topics

- parse_proc_arguments

deleteDangling1b1Or0s

```
deleteDangling1b1Or0s
-help
```

Deletes dangling 1'b1s or 1'b0s from the Verilog. They are either dangling nets assigned 1'b1 or 1'b0 or 1'b1/1'b0 connections that are not hierarchically connected to a leaf instance.

Parameters

None

Example

In the code below only BUFX2 is a leaf.

```
//
// Original Verilog
module hier4 (A,B);
input A;
input B;

BUFX2 u0 (.A(A));
BUFX2 u1 (.A(1'b1));
endmodule

module hier3 ();

hier4 u0 (.A(1'b1),.B(1'b0));
hier4 u1 (.A(1'b0),.B(1'b1));
endmodule

module hier2 ();

hier3 u0 ();
endmodule

module top ();

// Internal wires
wire a;
wire b;
wire c;
wire d;

assign a = 1'b1 ;
```

```
assign b = 1'b1 ;
assign c = 1'b0 ;
assign d = 1'b0 ;

hier2 u0 ();
BUFX2 u1 (.A(a));
BUFX2 u2 (.A(c));
endmodule

//
// Verilog after deleteDangling1b1Or0s is issued

module hier4 ( A, B);
input A;
input B;

BUFX2 u0 (.A(A));
BUFX2 u1 (.A(1'b1));
endmodule

module hier3 ();

hier4 u0 (.A(1'b1));
hier4 u1 (.A(1'b0));
endmodule

module hier2 ();

hier3 u0 ();
endmodule

module top ();

// Internal wires
wire a;
wire c;

assign a = 1'b1 ;
assign c = 1'b0 ;

hier2 u0 ();
BUFX2 u1 (.A(a));
BUFX2 u2 (.A(c));
endmodule
```

deleteDanglingPort

```
deleteDanglingPort
[-help]
[-ptn]
```

Deletes module ports that are disconnected on the outside.

Parameters

`-help`	Prints a brief description that includes type and default information for each `deleteDanglingPort` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deleteDanglingPort`
`-ptn`	Evaluate for deletion only ports on partition boundaries. *Data_type*: bool, optional

Examples

Delete all dangling ports on all partitions

```
deleteDanglingPort -ptn
```

deselect_obj

```
deselect_obj
[-help]
{[-all ] | [obj_list]}
```

Deselects the specified object or the list of objects from the Viewer. The object type can be any object type drawn on GUI such as instance wire, via instance.

Parameters

`-help`	Prints a brief description that includes type and default information for each `deselect_obj` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deselect_obj`
`-all`	Deselects all objects. *Data_type*: bool, optional
obj_list	Specifies the list of objects to be deselected. *Data_type*: (any_object)+, optional

Example

The following command deselects the instance name i1.

```
deselect_obj [dbGet top.insts.name i1 -p]
```

Related Topics

- `select_obj`

encMessage

```
encMessage
[-help]
msgType
valStr
```

Specifies whether to enable the display of warning, information, or debug messages.

Parameters

`-help`	Prints a brief description that includes type and default information for each `encMessage` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man encMessage`
`msgType`	Specifies the type of message you want to enable or disable. Type can be: • `warning` • `info` • `debug` *Data_type*: string, required
`valStr`	Controls the display of the messages. Use `0` to disable and `1` to enable messages. *Data_type*: string, required

Example

- The following command disables the display of all information messages:

  ```
  innovus 1> encMessage info 0
  ```

- By default, the tool displays the warning, information, or debug messages for each command executed. The following example shows the information that the tool displays after `selectRow` and `setDesignMode` commands are executed:

  ```
  innovus 2> selectRow 10 10 1
  selectRow finished.
  ```

  ```
  innovus 3> setDesignMode -process 45
  Applying the recommended capacitance filtering threshold values for 45nm process
  node: total_c_th=0, relative_c_th=1 and coupling_c_th=0.1.
  These values will be used by all post-route extraction engines, including
  TQuantus, IQuantus and Quantus QRC extraction.
  Capacitance filtering mode(-capFilterMode option of the setExtractRCMode) is
  'relAndCoup' for all engines.
  The accuracy mode for postRoute effortLevel low extraction will be set to 'high'.
  Default value for EffortLevel(-effortLevel option of the setExtractRCMode) in
  postRoute extraction mode will be 'medium' if Quantus QRC technology file is
  specified else 'low.
  ```

  You can suppress these information messages using the `encMessage` command. To suppress the information messages in log, specify `0` as the value for `info`. For example:

  ```
  innovus 4> encMessage info 0
  ```

  After the above setting, no message is displayed when commands are executed. However, this setting cannot be done for any specific command. To enable the display of information messages, specify `1` as the value for `info` as given below:

  ```
  innovus 5> encMessage info 1
  ```

encrypt

```
encrypt
[-help]
input_file_name
[-pragma]
[> file]
[-tcl  | -verilog ]
```

Encrypts a file in an nc-protect compatible format.

Parameters

`-help`	Prints out the command usage.
input_file_name	Specifies the file to be encrypted. *Data_type*: string, required
> file	Specifies the name of the encrypted file. By default, the encrypted file is printed to standard out (stdout). *Data_type*: redirect_operator, optional
`-pragma`	Specifies to only encrypt the text between the protect begin and protect end NC Protect pragmas. Encryption block must start with pragma protect comment. *Data_type*: bool, optional
`-tcl`	Specifies that the file to be encrypted is a Tcl file. This is the default option. *Data_type*: bool, optional
`-verilog`	Uses Verilog style comments for NC Protect pragmas. *Data_type*: bool, optional

Related Topics

- `decrypt`

eval_common_ui

```
eval_common_ui
[-help]
script
```

Runs a Stylus Common UI command in the legacy interpreter. This command is not required in the normal usage. However, it may be needed when there is a transition period, that some scripts are already converted into Common UI style, but you still need to run the flow in legacy. This command is intended as a temporary workaround as you transition to the Stylus Common UI and will be removed in a future release.

Most Stylus Common UI commands are available, but the commands that have behavior difference from legacy will not work properly (for example, write_db, read_db, read_netlist, read_physical, read_mmmc, init_design, and report_timing).

These commands will not work and give error messages:

Initialization commands (legacy uses globals and its own version of init_design)

- `read_mmmc`
- `read_netlist`
- `read_physical`
- `init_design`

Some timing commands that have different formatting and controls:

- `report_timing`
- `report_critical_insts`

Commands that only work with Stylus logging:

- `set_log_post_cmd`

The CUI only supports MMMC2 and does not support MMMC1 usage. So the following commands have limitations:

Command	Limitation
`read_db`	Can only read an MMMC2 DB, saved by either the legacy saveDesign or CUI write_db command. An MMMC1 DB will cause an error.

`write_db`	Can only save an MMMC2 DB (if the current DB is MMMC1 an error occurs)
`write_mmmc`	Not supported

See the legacy global `init_mmmc_version` for more info about how to load an MMMC2 file into legacy during initialization. You can also use `saveDesign -mmmc2` to convert a legacy MMMC1 DB in memory to a saved MMMC2 DB.

The Stylus Common UI commands run in a separate Tcl interpreter. Therefore, Tcl variables in the current interpreter are not available. That means the script should normally be inside " " rather than { } so that the Tcl variable substitution occurs in the current interpreter before passing the script to the Stylus Common UI Tcl interpreter.

If a TCL_ERROR is returned by any command inside a *script*, the *script* stops executing, and `eval_common_ui` returns a TCL_ERROR.

Parameters

`-help`	Prints a brief description that includes type and default information for each `eval_common_ui` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man eval_common_ui`.
`script`	Specifies one or more Stylus Common UI Tcl commands to run in the Stylus Common UI Tcl interpreter. *Data_type*: string, required

Examples

- The following command runs the finish_floorplan command in the CUI Tcl interpreter.

  ```
  eval_common_ui "finish_floorplan"
  ```

- If you want a Tcl var from the legacy Tcl interpreter available inside the CUI interpreter script, you should protect it with “” to force Tcl var substitution in the legacy interpreter like this:

  ```
  set inst i1
  eval_common_ui “get_db insts -if {.name == $inst}”
  ```

  The $inst will be substituted to result in ‘get_db insts -if {.name == i1}’ before eval_common_ui sees it and passes it to the CUI interpreter.

- If you want to use CUI Tcl vars inside the CUI interpreter, you should protect them with { } like this to avoid substitution in the legacy interpreter:

  ```
  eval_common_ui {set i [get_db insts -if {.base_cell.name == AND2X}] get_db $i
  .name}
  ```

  The Tcl var $i is only used inside the CUI interpreter, and not added to the legacy interpreter.

- If you have a few global Tcl vars in the legacy interp to pass into a common CUI script, you could add them in separate eval_common_ui calls like this:

  ```
  set glob1 test
  set glob2 test2
  eval_common_ui “set glob1 $glob1”   This command sets CUI glob1 to legacy $glob1
  eval_common_ui “set glob2 $glob12”  This command sets CUI glob2 to legacy $glob2
  eval_common_ui “source my.tcl”      CUI my.tcl can use both globals
  ```

Related Information

- init_mmmc_version
- saveDesign

find_global

```
find_global
[-help]
[pattern | -non_default ]
[[-tcl ] | [[-name ] [-value ] [-type ] [-default ] [-info ] [-range_or_enum ] [-
transient ]]]
```

Returns all the public variables. If you want to retrieve specific global variables, you can use patterns with this command.

Parameters

`-help`	Prints a brief description that includes type and default information for each `find_global` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man find_global`.
pattern	Specifies a string with wildcards that can be used to retrieve specific global variables. *Data_type*: string, optional
`-default`	Displays the default value of variables. If a pattern is specified, displays default value of all variables matching the specified pattern. *Data_type*: bool, optional
`-info`	Displays the description of variables. If a pattern is specified, displays description of all variables matching the specified pattern. *Data_type*: bool, optional
`-name`	Displays the name of all or specified global variables. *Data_type*: bool, optional
`-non_default`	Displays the list of variables whose values are not default. **Note**: If neither `pattern`, nor `-non_default` is specified, the tool displays all variables. *Data_type*: bool, optional

`-range_or_enum`	Displays the range or legal enum value list, if they exist, for all or specified global variables. *Data_type*: bool, optional
`-tcl`	Returns a tcl list that has every field value of the specified variables. If a pattern is not specified, displays this information for all global variables. *Data_type*: bool, optional
`-transient`	Displays whether the specified variable is transient. If a pattern is not specified, displays this information for all global variables. *Data_type*: bool, optional
`-type`	Displays the type (`Boolean`, `Integer`, `String`, `Enum`) of all or specified variables. *Data_type*: bool, optional
`-value`	Displays the current value of all or specified variables. *Data_type*: bool, optional

Example

- The following command returns all public variables and private variables with a non-default value:
  ```
  find_global
  ```
- The following command returns all public global variables (and private variables with a non-default value) beginning with `dbg`.
  ```
  find_global dbg*
  ```

  Output:

  ```
  dbgAllowInferredTriStateArcs 1 # (Integer, default=1, persistent)
  # Controls whether arcs from a tristate enable to a inferred as tristate
  # enable and disable arcs if they are not explicitly specified by the Liberty
  # timing_type attribute. This functionality is mainly to support older
  # modeling styles in legacy libraries.
  dbgConvertSetupHoldOnAsyncToRecRem 1 # (Integer, default=1, persistent)
  # Controls whether single-edged setup and hold checks on asynchronous pins
  ```

```
# in the Liberty library are inferred as recovery and removal checks. This
# is mainly to support older modeling styles in legacy libraries.
dbgPicoSecPerDBU 0.1 # (Double, default=0.1, persistent)
# Controls how timing library numbers are internally scaled to database
# units. The default is 1/10 pS per DBU. To represent very large delay
# numbers accurately this value can be increased as needed. This variable
# should be set before timing libraries are read.
dbgUseDuplicateTimeArc 1 # (Integer, default=1, persistent)
# This variable controls whether or not redundant timing arcs (no retain
# or conditional variance) are supported. This variable is obsolete and
# will be removed from future releases of the software.
```

- The following command returns information on a specific variable, `lefDefOutVersion`.

  ```
  find_global lefDefOutVersion
  ```

  The output is as follows:

  ```
  lefDefOutVersion "" # (String, default="", persistent)
  # Specify LEF/DEF output version. Possible string value are 5.3, 5.4, 5.5,
  # 5.6, 5.7 and 5.8.
  ```

- The following command returns names and default information for all global variables starting with the pattern `def`.

  ```
  find_global def* -name -default
  ```

  The output is as follows:

  ```
  defHierChar # (default="")
  defInShapeBasedDefFile # (default=0)
  defOutBumpAsInstance # (default=1)
  defOutLefNDR # (default=0)
  defOutLefVia # (default=0)
  defOutPolygonDieArea # (default=1)
  defOverlapWireReportFileName # (default="")
  ```

Related Topics

- For information on viewing global settings using the GUI, see "Set Global Variable" in the Tools Menu chapter of the *Innovus Menu Reference*.
- For complete list of public globals returned by `find_global`, see:

- "Appendix B: Tcl Globals" chapter in *Innovus Text Command Reference*.

generateFFSetupFile

```
generateFFSetupFile
[-help]
```

Starts the foundation flow on Innovus.

Parameters

`-help`	Outputs a brief description for `generateFFSetupFile`. For a detailed description of the command and all of its parameters, use the `man` command: `man generateFFSetupFile`.

Related Topics

- Design Implementation Flow chapter of the of the *Innovus User Guide*

get_message

```
get_message
-help
-id msgID
{-severity | -limit | -suppress | -count | -short | -long | | -on_error | -
stop_script_limit | -detail}
```

Gets the limit, severity, or suppression for a message.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_message` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_message`
`-count`	Returns the number of times this message has been issued this session. *Data_type*: bool, optional
`-detail`	Returns the message of specified id. *Data_type*: bool, optional
`-id msgID`	Specifies a single message ID, in the form "ENCFOO-123". *Data_type*: string, required
`-limit`	Returns the limit for the specified message as an integer. -1 is unlimited. 0 is suppressed. *Data_type*: bool, optional
`-long`	Returns the long message description, as used for the Description section in "`man msgID`" *Data_type*: bool, optional
`-on_error`	Returns the error action for the specified message. *Data_type*: bool, optional

`-severity`	Returns the severity, `error`, `warn`, `info`, or `default`, of the specified message as a string. *Data_type*: bool, optional
`-short`	Returns the short message description. Equivalent to "`help` *`msgID`*" . *Data_type*: bool, optional
`-stop_script_limit`	Returns the message limit allowed for every public command. Message limit set as -1 means unlimited. *Data_type*: bool, optional
`-suppress`	Returns `1` if the message is suppressed, `0` if not suppressed. *Data_type*: bool, optional

Related Topics

- `set_message` command

get_metric

```
get_metric
-help
[-branch list_of_patterns]
[-exclude_inherited]
[-id id_name]
[-names label_name]
pattern_name
[-pending]
[-raw]
```

Gets the current state of metric(s).

You can use this command to query the current state of any metric or metrics from the current specified list of metrics as well as a previous state.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_metric` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_metric`
`-branch` *`list_of_patterns`*	Specifies list of patterns to match snapshot branches. This will limit all output to metrics stored in snapshots whose branch matches the specified branch. This requires any inheritance to be re-run to exclude inherited values from non-matching snapshots. *Data_type*: string, optional
`-exclude_inherited`	Specifies not to return any inherited value. *Data_type*: bool, optional
`-id` *`id_name`*	Specifies the database ID to output. Default is current database. *Data_type*: string, optional
`-names` *`label_name`*	List of patterns to match snapshot names Name of label to get metrics from (wildcards supported); no label reports current metrics. *Data_type*: string, optional
`pattern_name`	Specifies the list of patterns to match metrics. Supports wildcard *. Use of * gets all metrics. *Data_type*: string, optional
`-pending`	Returns metrics from the pending (not yet created) snapshot. *Data_type*: bool, optional
`-raw`	Returns metrics as raw value (without normalization to defined units). *Data_type*: bool, optional

Related Topics

- encEnableMetric

getAllLayers

```
getAllLayers
[-help]
[{object display multi internal module metal color custom oaLayout}]
```

Returns a complete list of all layers and floorplan object settings. If you specify type, the software returns all the layers of the specified type. This command can be used at any stage in the design flow.

Parameters

`-help`	Prints a brief description that includes type and default information for each `getAllLayers` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man getAllLayers`
`{object display multi internal module metal color custom oaLayout}`	Specifies the type of the layer. Innovus supports nine types of layers, which can be specified as follows: • `object`: Returns object layers, which represent db objects, such as instances, modules, pins, and so on. • `display`: Returns display-only or view-only layers, including flightlines, rulers, and congestion. • `multi`: Returns multiple color layers, such as congestion and yield maps. • `internal`: Returns internal layers, such as bump layers, structured data path layers, etc. • `module`: Returns module layers, which are layers associated with hinsts. • `metal`: Returns wire/via layers, including metal/via, pin, and blockage. • `color`: Returns color layers. • `custom`: Returns custom layers, which are used to represent custom objects and GDSII data. • `oaLayout`: Returns oaLayout layers. *Data_type*: enum, optional

Example

Returns all metal layer names:

```
getAllLayers metal
```

Related Topics

- Getting Started chapter of the *Innovus User Guide*.

getBuildArch

```
getBuildArch [-help]
```

Reports whether the Innovus session is running in 32-bit or 64-bit mode.

Parameters

`-help`	Prints a brief description of the command.

getCheckMode

```
getCheckMode
[-all]
[-checkIlm]
[-extraction]
[-floorplan]
[-globalNet]
[-integrity]
[-io]
[-library]
[-mgrid]
[-netlist]
[-placement]
[-route]
[-sroute]
[-tapeOut]
[-timingGraph]
[-vcellnetlist]
```

Returns information about the specified `setCheckMode` parameter in the log file and in the console. The following information is returned:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the software displays information for all of the `setCheckMode` parameters.

Parameters

-parameter_names	
	Displays information for the specified parameters. You can specify one or more parameters. See `setCheckMode` for descriptions of the data checking parameters you can specify.

Examples

- The following command displays the current setting for the `-extraction` parameter:

  ```
  getCheckMode -extraction
  ```

  The software displays the following information:

  ```
  -extraction false # bool, default=false

  false
  ```

- The following command displays the current setting of the `-extraction` parameter in Tcl list format:

  ```
  getCheckMode -extraction -quiet
  ```

  The software displays the following information:

  ```
  false
  ```

- The following command displays the current settings of the `-floorplan` and `-placement` parameters:

  ```
  getCheckMode -floorplan -placement
  ```

  The software displays the following information:

  ```
  -floorplan false # bool, default=false

  -placement false # bool, default=false

  {floorplan false} {placement false}
  ```

- The following command displays the current settings of the `-floorplan` and `-placement` parameters in Tcl list format:

  ```
  getCheckMode -floorplan -placement -quiet
  ```

  The software displays the following information:

```
{floorplan false} {placement false}
```

Related Topics

- Importing and Exporting Designs chapter of the *Innovus User Guide*.

getCompressLevel

`getCompressLevel`

Displays the current compression speed level used for compressing large data files. Compression speed is controlled with the `setCompressLevel` command.

Related Topics

- `setCompressLevel`

getLayerPreference

```
getLayerPreference
[-help]
layer_name
{-isVisible | -isSelectable | -color | -lineWidth | -stipple | -stippleData}
```

Provides information about a single preference for a single object.

See `setLayerPreference` for descriptions of how to set layer preferences.

Parameters

`-help`	Prints a brief description that includes type and default information for each `getLayerPreference` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man getLayerPreference`
`-color`	Provides information about the color of the object. A color name or a numeric RGB value is returned. *Data_type*: bool, optional
`-isSelectable`	Provides information about whether the object is selectable. If a value of `1` is returned, the object is selectable. If a value of `0` is returned, the object is not selectable. *Data_type*: bool, optional
`-isVisible`	Provides information about whether the object is visible. If a value of `1` is returned, the object is visible. If a value of `0` is returned, the object is not visible. *Data_type*: bool, optional
`layer_name`	Provides the name of the layer or the object type. *Data_type*: string, required
`-lineWidth`	Provides information about the number of pixels in the border of the specified object. An integer value is returned. *Data_type*: bool, optional
`-stipple`	Provides information about the stipple pattern of the object. A stipple name is returned. *Data_type*: bool, optional
`-stippleData`	Provides information about the width and height of the stipple pattern. This information is in x-window bitmap format. *Data_type*: bool, optional

Example

Returns the color for `Metal1` objects:

```
getLayerPreference Metal1 -color
```

Related Topics

- `setLayerPreference`

getPreference

```
getPreference
[-help]
preference_name
```

Returns the current preference settings.

Parameters

`-help`	Prints out the command usage
preference_name	Specifies the name of a preference. *Data_type*: string, optional

Related Topics

- setPreference

getVersion

```
getVersion
[-help]
[-major]
```

Returns the Innovus software version number.

Parameters

`-help`	Prints a brief description of the command. For a detailed description of the command, use the `man` command: `man getVersion`
`-major`	Returns the major version and one digit after the decimal point. *Data_type*: bool, optional

Examples

- The following command returns the software version:

  ```
  innovus 7> getVersion
  20.10-d251_1
  ```

- The following command gives the major version of the software:

  ```
  innovus 8> getVersion -major
  20.1
  ```

gui_get_legend

```
gui_get_legend
[-help]
-type {inst_density pin_density timing_map metal_density route_congestion temperature}
[-display_range | -value_range | -show_legend_on_layout | -legend_location | -
enable_min_value | -enable_max_value]
```

Gets the current legend setting.

You can use the `gui_set_legend` and `gui_get_legend` commands to get the snapshot of different density for analysis.

Parameters

`-help`	Prints a brief description of the command. For a detailed description of the command, use the `man` command: `man gui_get_legend`
`-display_range`	Gets the display range
`-enable_max_value`	Gets the state of the max value of display range.
`-enable_min_value`	Gets the state of the min value of display range.
`-legend_location`	Gets the location of legend.
`-show_legend_on_layout`	Gets the state of legend on layout.
`-type {inst_density pin_density timing_map metal_density route_congestion temperature}`	Gets the setting for the specified legend type.
`-value_range`	Gets the value range.

Example

The following command sets the legend for the inst_density legend type:

```
> gui_set_legend -type inst_density -display_range {1 0.9} -show_legend_on_layout true
```

The following command gets the current legend setting for the `inst_density` legend type.

```
> gui_get_legend -type inst_density
{display_range {0.9 0.9}} {legend_location ne} {show_legend_on_layout true}
{value_range {0.75 1}}
```

Related Information

- `gui_set_legend`

gui_set_legend

```
gui_set_legend
[-help]
[-display_range {x y}]
[-enable_max_value {true false}]
[-enable_min_value {true false}]
[-legend_location {ne nw se sw}]
[-reset -display_range|-value_range|-show_legend_on_layout|-legend_location|-
enable_min_value|-enable_max_value]
[-show_legend_on_layout {true false}]
-type {inst_density pin_density timing_map metal_density route_congestion temperature}
[-value_range {x y}]
```

Specifies the legend setting.

You can use the `gui_set_legend` and `gui_get_legend` commands to get the snapshot of different density for analysis.

Parameters

`-help`	Prints a brief description of the command. For a detailed description of the command, use the `man` command: `man gui_set_legend`
`-display_range {x y}`	Specifies the display range in the {min max} format
`-enable_max_value {true false}`	Enables the max value of the display range.
`-enable_min_value {true false}`	Enables the min value of the display range.
`-legend_location {ne nw se sw}`	Specifies the location of legend.
`-reset -display_range|-value_range|-show_legend_on_layout|-legend_location|-enable_min_value|-enable_max_value`	Resets the settings of the specified option, `-reset -display_range|-value_range|-show_legend_on_layout|-legend_location|-enable_min_value|-enable_max_value`. You can reset all options by using `-reset all`.
`-show_legend_on_layout {true false}`	Specifies to show the legend on layout.
`-type {inst_density pin_density timing_map metal_density route_congestion temperature}}`	Specifies the setting for the specified legend type.
`-value_range {x y}`	Specifies the value range in the {min max} format.

Example

The following command sets the legend for the inst_density legend type:

```
> gui_set_legend -type inst_density -display_range {1 0.9} -show_legend_on_layout true
```

The following command gets the current legend setting for the `inst_density` legend type.

```
> gui_get_legend -type inst_density
{display_range {0.9 0.9}} {legend_location ne} {show_legend_on_layout true}
{value_range {0.75 1}}
```

Related Information

- gui_get_legend

help

```
help
[-help]
[-k keyword| -cmd commandName| -var] [pattern]
```

Displays information, such as command syntax, summary of messages, list of commands, based on the specified argument. Displays the following information:

- Syntax of a command or global variable.
- Summary of the message text based on the specified message ID.
- List of commands based on keywords.
- List of commands based on search string.

Command Syntax

To display the syntax of a command or a variable, use the following command:

```
help commandName
```

The command syntax output is written in the log file and displayed in the Innovus console.

Message Summary

To display a summary of the message text for a particular message ID, use the following command:

```
help msg_id
```

Shows the summary of message text for the message ID specified.

Keyword Searches

To perform a keyword search, use the following command:

```
help -k keyword
```

Shows a list of commands with the specified keyword.

Note: Keyword-based searches are case insensitive.

Partial String Searches

To perform a partial string search, use the following command:

`help *string*`

Shows all commands containing the specified string. The search results vary depending on whether you use wildcards (*) in the beginning, end, or middle of the string.

Note: Partial string searches are case insensitive.

Parameters

`-cmd` *commandName*	Displays help for the specified command name. If you do not specify a command name, all officially supported Innovus commands are displayed. *Data_type*: bool, optional
`-k` *keyword*	Searches for the specified keyword and displays all commands associated with that keyword. *Data_type*: bool, optional
pattern	Specifies command pattern, message id or keyword. *Data_type*: bool, optional
`-var`	Performs variable search only. *Data_type*: bool, optional

Examples

- The following command prints the command syntax for the `setRouteMode` command:
  ```
  help setRouteMode
  ```
- The following command shows the message summary for the `ENCSYC-139` message ID:
  ```
  help ENCSYC-139
  ```
- The following command displays all commands that include "place" as a keyword:
  ```
  help -k place
  ```
- The following commands show how wildcards can be used in string searches:
  - Use wildcards at the beginning of the string to get all commands that end with the

specified string:

```
innovus 3> help *_timing

 Multiple commands found:
  check_timing
  compare_model_timing
  get_constant_for_timing
  report_cell_instance_timing
  report_clock_timing
  report_timing
  reset_disable_timing
  set_disable_timing
  write_model_timing
```

- Use wildcards in the middle of the string when you are not sure what comes in the middle of the string:

```
innovus 4> help report*timing

 Multiple commands found:
  report_cell_instance_timing
  report_clock_timing
  report_timing
```

- Use wildcards at the end of the string to get all commands that begin with the specified string:

```
innovus 5> help report_tim*

 Multiple commands found:
  report_timing
  report_timing_derate
  report_timing_format
```

- Use wildcards at the beginning and end of the string to get a list of all the commands that contain the string:

```
innovus 6> help *timing*

 Multiple commands found:
  checkTimingLibrary
  checkWhatIfTiming
  check_timing
  compare_model_timing
  deleteWhatIfTimingAssertions
  deriveTimingBudget
  getWhatIfTimingAssertions
```

. . .

is_common_ui_mode

```
is_common_ui_mode
[-help]
```

Checks if the Stylus Common UI mode is enabled. This command is helpful if you are using both Common UI and Legacy UI, and you want to source their respective script based on the value returned by `is_common_ui_mode`. In such scripts, if `is_common_ui_mode` returns 1, Stylus Common UI commands will be sourced, and if it returns 0, Legacy UI commands will be sourced.

For example, you can write a script, which can be sourced irrespective of UI, as below.

```
if {[is_common_ui_mode]}

{
<Common UI commands>
}

else

{
<Legacy UI commands>
}
```

Parameters

None

Example

The following example checks it on the Innovus prompt:

```
@innovus> is_common_ui_mode

1

innovus> is_common_ui_mode

0
```

Return value `1` indicates that the Stylus Common UI mode is enabled.

lminus

```
lminus
[-exact]
list1
list2
```

Subtracts the specified elements in the second list (*list2*) from the first specified list (*list1*) .

Parameters

-exact	Searches for an exact match.
list1	Specifies the list from which elements must be removed.
list2	Specifies the list of elements to be subtracted.

Examples

- The following command subtracts the elements from list l2 from list l1.

```
innovus> set l1 {a a* ab2 b1 b2 c}
innovus> set l2 {a c}
innovus> lminus $l1 $l2
a* ab2 b1 b2
```

- The following command subtracts a list consisting of a* and c from the list of elements of l1.

```
innovus> lminus $l1 {a* c}
b1 b2
```

- The following command uses an exact search.

```
innovus> lminus -exact $l1 {a* c}
a ab2 b1 b2
```

man

```
man
command_name | msg_id
```

This command is a Tcl wrapper that calls the Linux ‘man’ command, so you can see man pages for Innovus commands, Innovus Tcl global variable, or other Linux commands. The man-page directories inside the Innovus installation are automatically added to the beginning of the Linux environment variable MANPATH so you will see Innovus man-pages before any other man-pages in your MANPATH.

The only exception is if the argument matches the msg_id format (e.g. several letters with a dash followed by an integer like ‘ABC-123’)., the Tcl wrapper directly returns the msg_id message you would see in the log file, along with any extended help description if it is available for that msg_id. These msg_id ‘man-pages’ are not included in the man-page directories, but are only available inside Innovus.

For all other arguments, it just calls the Linux ‘man’ command with the proper MANPATH, so you can use any Linux ‘man’ options as described in the Linux man-page for ‘man’.

For information on setting up MANPATH for Stylus Common UI commands directly from Linux, see Setting Up the Run-Time Environment in the *Getting Started* chapter of the *Innovus User Guide*.

Parameter

command_name	Indicates the name of the text command (or variable) for which the manual information is required.
msg_id	Specifies a message ID in the format like ‘ABC-123’. The output will show the log file message and any extended help description that can include likely causes, possible fixes, and possible workarounds for this msg_id.

Example

- The following command returns the manual description of the command `report_timing`:

  ```
  man report_timing
  ```

- The following command returns the manual description of the IMP`SYC-139` message ID. Some messages have extra man help, beyond what the original error or warning message says.

  ```
  man IMPSYC-139
  ```

parse_proc_arguments

```
parse_proc_arguments
[-help]
result_array
[forced_proc]
-args string
```

Supports argument validation within a Tcl procedure and enables the use of the `-help` option to view the help information associated with the procedure.

Procedures using the `parse_proc_arguments` command will validate the arguments. If the required arguments are missing or have invalid values, the `parse_proc_arguments` command generates a Tcl error and the calling procedure terminates.

The `parse_proc_arguments` command must be the first command called in the procedure. This command cannot be used outside of a procedure.

Parameters

`-help`	Prints out the command usage.
`-args string`	Specifies the list of arguments passed to the Tcl procedure. *Data_type*: string, required
result_array	Specifies the result array into which the parsed arguments are to be stored. *Data_type*: string, required
forced_proc	Specifies the name of the invoking procedure. *Data_type*: string, optional

Examples

The following Tcl procedure uses the `parse_proc_arguments` command for argument validation.

```
proc cmdProc {args} {

parse_proc_arguments -args $args results

foreach argname [array names results] {

puts "$argname = $results($argname)"
```

```
}

}


define_proc_arguments cmdProc -info "Command Description" \

-define_args \

{{-string "String help" "string_val" string optional}

{-integer "Int help" "int_value" int optional}

{-list "List help" "list_value" list {optional bind_option -Bool}}

{-boolean_a "BoolA help" "" boolean {optional mutual_exclusive_group g0}}

{-boolean_b "BoolB help" "" boolean {optional mutual_exclusive_group g0}}

{-float_num "Float help" "float_value" float}

{-oosa "One of StringA help" "one_of_str_value" one_of_string {optional value_help
{values {v1 v2}}}}

{-oosb "One of StringB help" "one_of_str_value" one_of_string {optional {values {v4
v5}}}}

}
```

Now, if the cmdProc procedure is invoked using the following command at the command prompt, argument validation takes place and the following errors are reported.

```
innovus> cmdProc -boolean_a -boolean_b

**ERROR: Options '-boolean_a -boolean_b' are mutually exclusive for 'cmdProc'.

Usage: cmdProc [-string string_val] [-integer int_value] [-list list_value] [-boolean_a
]

[-boolean_b ] [-float_num float_value] [-oosa one_of_str_value {v1 v2 }] [-oosb
one_of_str_value]]

**ERROR: Errors in 'cmdProc'.
```

The following command to invoke the cmdProc procedure will also return the errors shown below.

```
innovus> cmdProc -oosa v5

**ERROR:  The arguments specified (v5) in cmdProc -oosa are not valid. Refer to the
documentation for valid arguments.

Usage: cmdProc [-string string_val] [-integer int_value] [-list list_value] [-boolean_a
```

```
]

[-boolean_b ] [-float_num float_value] [-oosa one_of_str_value {v1 v2 }] [-oosb
one_of_str_value]]

**ERROR: Errors in 'cmdProc'.
```

placeCursor

```
placeCursor
[-help]
x y
```

Places the cursor at the specified location in the main window.

You can specify this command at any time after starting the Innovus software.

Parameters

`-help`	Prints a brief description that includes type and default information for each `placeCursor` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man placeCursor`
x	Specifies the x coordinate of the cursor location. *Data_type*: float, required
y	Specifies the y coordinate of the cursor location. *Data_type*: float, required

Example

The following command places the cursor at the coordinates (`133`, `277`).

```
placeCursor 133 277
```

print_message

```
print_message
-help
[-id msgID | formatString]
[args]
[-error]
[-warning]
```

Generates a message that is included in the `report_message` output if the message is designated as an error or a warning.

Parameters

args	Specifies the format argument value to print. *Data_type*: string, optional
-error	Specifies the message is an error. A prefix will be added. *Data_type*: bool, optional
formatString	Specify the format specifier like sprintf or Tcl's format command. *Data_type*: string, required
-help	Prints a brief description that includes the type and default information for each `print_message` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man print_message`.
-id *msgID*	Specifies the message id.
-warning	Specifies that the message is a warning. A prefix will be added. *Data_type*: bool, optional

Examples

The following commands generate an error and a message, which are included in the report_message output:

```
[DEV]innovus> print_message -error "A test %s." "from the user"
ERROR: (user): A test from the user.


innovus> print_message -warning "A test %s." "from the user"
WARN: (user): A test from the user.


innovus> print_message "A test %s." "from the user"
A test from the user.

innovus > report_message

*** Summary of all messages that are not suppressed in this session:
Severity      ID       Count          Summary
ERROR       no_id       1            A test %s.
ERROR       user        1            A test %s.
WARN        user        1            A test %s.
*** Message Summary: 1 warning(s), 2 error(s)
```

Related Topics

- report_message

Puts

```
Puts
[-help]
string
```

Outputs information to the screen (stdout) and Innovus log files (.log/.logv).

read_taf

```
read_taf
[-help]
[filename [-quiet]
```

Applies additional attribute data from Genus for the user-managed exchanges. It is not needed for managed exchanges after iSpatial (syn_opt -spatial) from Genus to Innovus or when using common-DB between Genus and Innovus. The 'taf' format is a private format between Genus and Innovus.

Parameters

`-help`	Prints a brief description that includes type and default information for each `read_taf` parameter.
filename	Specifies the name of file to read. *Data_type*: string, required
`-quiet`	When `-quite` is specified, the tool does not give a summary. *Data_type*: bool, optional

redirect

```
redirect
[-help]
[file_or_var_name]
[command]
[-tee]
[-stdin | -append]
[-stdin | -variable]
[-variable]
[-stderr | -stdin]
```

Redirects the output to a file or variable. You can also directly use redirect characters without explicit use of the redirect command for Innovus commands.

Any Tcl return value from *command* is returned by redirect. If redirect is used at the prompt, any echoing of the Tcl return at the prompt is not redirected.

You can also use the `redirect` command to write the contents directly to a gzip compressed file by ending the file name with .gz.

Parameters

`-help`	Prints a brief description that includes type and default information for each `redirect` parameter.
`file_or_var_name`	File or variable name for output to go. *Data_type*: string, optional
`command`	Command to be executed. *Data_type*: string, optional
`-append`	Appends stdout/stderr to file or variable. *Data_type*: bool, optional
`-stderr`	Redirects stderr. *Data_type*: bool, optional
`-stdin`	Redirects stdin. *Data_type*: bool, optional
`-tee`	Redirects output to both screen and file or variable. *Data_type*: bool, optional
`-variable`	Redirects to a Tcl variable. *Data_type*: bool, optional

It is also possible to use redirect characters similar to Unix usage, for Innovus commands, but not for built-in Tcl commands. This usage works for any Innovus command that does not explicitly register > or >> as options (if > or >> are visible in the man page or -help output for the command, it is explicitly registered). Note that the "source" command is overlaid by Innovus to add various verbose options, so it also supports these characters.

The table below shows the usage of redirect characters, and how to use & to indicate a Tcl var name.

Redirect Characters	Meaning
> <name>	Redirects stdout to the file <name>
>> <name>	Redirects stdout and appends to the file <name>

> &<name>	Redirects stdout to the Tcl var <name>
>> &<name>	Redirects stdout and appends to the Tcl var <name>

Examples

- Redirects the output to a new file timing.txt.
  ```
  redirect timing.txt "timeDesign -postRoute"
  ```
- Appends the output to the file test.out.gz in compressed gzip format.
  ```
  redirect test.out.gz "source test.tcl" -append
  ```
- Redirects the output to both screen and a file.
  ```
  redirect timing.txt "timeDesign -postRoute" -append -tee
  ```
- Redirects to the file a.log in compressed gzip format
  ```
  source test.out > a.log.gz
  ```
- Redirects and appends to the file a.log
  ```
  report_area >> a.log
  ```
- Redirects to the Tcl var a
  ```
  report_area > &a
  ```
- Redirects and appends to the Tcl var a
  ```
  report_area >> &a
  ```
- The `dbGet` and `get_property` commands return their values as TCL_RESULTS. To redirect the output to any file, you can use any of the following methods:
  a. You can redirect output of `dbGet` and `get_property` using the following commands:
  ```
  redirect {puts "[dbGet top.insts.name]"} > inst_name.rpt
  redirect {puts "[get_property [get_lib_cells F1] area]"} > property.rpt
  ```
  b. You can get the value of output stored in a variable using the following commands:
  ```
  redirect a {puts "[dbGet top.insts.name]"} -variable
  redirect b {puts "[get_property [get_lib_cells F1] area]"} -variable
  ```

rename_obj

```
  rename_obj
[-help]
[-escape_ok]
[-flexible]
inst|hInst|vCell|topCell|term|hTerm|hNet
new_name
```

Renames the `base_name` of a single design object, which could only be a `inst`, `hinst`, `design`, `module`, `port`, `hport`, or `hnet`. This command changes only the `base_name` of the specified objects, and cannot move those across the netlist hierarchy. This command also returns the new name of the renamed object.

The names used for SDC timing constraints can only be modified using this command, if they have been loaded into the memory. So, ensure that all the timing views are loaded using the `set_analysis_view` command before running the `rename_obj` command so that the SDC constraints are updated properly. The modified SDC files can be retrieved after renaming the objects.

The `rename_obj` command cannot update the names used inside a CPF `power_intent` file. This is because the CPF usage assumes that the CPF file is golden and should not be modified. To change the names used inside a CPF file, first update the CPF file manually, and then use the updated CPF file to run the `rename_obj` command.

An IEEE1801 `power_intent` file is allowed to be changed during the design flow. So, changing names using the `rename_obj` command will automatically update the IEEE1801 object names. You can use the `rename_obj` command to write out the new IEEE1801 file with the updated names.

Parameters

`-help`	Provides a brief description of the command that includes the type and default information for its parameters. For a detailed description of the command and all its parameters, use the man command: `man rename_obj`
`-escape_ok`	Allows `new_name` to have either the hierarchy chars '`/`' or the bus-bit chars '`[]`' in it, which will automatically be escaped. If this parameter is not specified, any set of '`/`' or '`[]`' chars in `new_name` will cause an error and the object will not be renamed. *Data_type*: bool, optional

`-flexible`	If `new_name` coincides with an existing name, this parameter appends a suffix (such as `_1` and `_2`) to differentiate it. If this parameter is not specified, a name collision in such a scenario causes an error, and the specified object is not renamed. *Data_type*: bool, optional
`inst|hInst|vCell|topCell|term|hTerm|hNet`	Specifies the object to be renamed. The object here must be an `inst`, `hinst`, design, `module`, `port`, `hport`, or `hnet`. If *`object`* is a `port` or `hport`, any `hnet` connected to it with the same `base_name` is also renamed to keep it matched (for example, renaming `hport:top/i1/clkin` also renames `hnet:top/i1/clkin`). If *`object`* is an `hnet` that is connected to a `port` or `hport` with the same `base_name`, an error occurs. In such a case, you must change the `port` or `hport` name to proceed.

	If an `hnet` name matches with its associated `net` name (for example, `get_db $my_hnet .net`), then the net name is also renamed on using the `rename_obj` command. For example, renaming `hnet:top/net1` also renames `net:top/net1`. Therefore, to rename a `net`, you need to rename the `hnet` with the same name. This command does not allow renaming a bus-bit object. So any `port`, `hport`, or `hnet` that has a `.base_name` like `busA[0]` (a bus-bit object), cannot be renamed, While a `.base_name` with a scalar name (like `busA\[0\]`) can be changed. *Data_type*: (`inst | hinst | module | design | port | hport | hnet`), required
new_name	Specifies the new `base_name` for the specified object. *Data_type*: string, optional

Examples

- The following command renames an `inst base_name` to `i2_new`:

```
rename_obj inst:top/i1/i2 i2_new
inst:top/i1/i2_new                #returned object with new name
```

- The following command renames an `inst base_name i2\/i3` with escaped hierarchy:

```
rename_obj {inst:top/i1/i2\/i3} i2/i3_new -escape_ok
inst:top/i1/i2\/i3_new            #returned inst object, no hierarchy change
```

- The following command renames a scalar `hport` name to remove escaped bus-bit chars:

```
rename_obj {hport:top/i1/busA\[0\]} busA_0
hport:top/i1/busA_0               #returned hport object
```

  Note: `hnet:top/i1/busA\[0\]` is also renamed to `hnet:top/i1/busA_0`.

- The following command renames a `port` name:

```
rename_obj port:top/clkin clkin_new
port:top/i1/clkin_new             #returned port object
```

  Note: Both `hnet:top/i1/clkin` and `net:top/i1/clkin` are renamed.

Related Information

`update_names`

report_area

```
report_area
[-help]
[-detail]
[-hierarchical_instance hinst_name]
[-include_physical]
[-min_area area_per_module]
[-min_count instance_count]
[-out_file filename]
[-show_leaf_cells]
[-show_msv_cells]
[-sort_by {name count area}]
[-table_style {horizontal vertical}]
[-summary | -depth depth_of_hierarchy]
```

Reports the combined standard cell area in each of the hierarchy modules and the top-level design. Also reports the cell area based on different cell types if the `-detail` option is specified. The command provides breakdown based on the cell type (such as .is_* attributes). The report represents the design hierarchy, and therefore, provides better readability.

Note: The units used in the report depend on the units used in the library. The area for each module is calculated from all the standard cells within each module using the LEF information.

Parameters

`-help`	Prints a brief description that includes type and default information for each `report_area` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_area`
`-depth` *depth_of_hierarchy*	Specifies the depth of hierarchy, all levels if not specified; the top of the hierarchy (design) is 1. *Data_type*: int, optional

`-detail`	Report the area in different cell type (buffer, inverter, combinational, flop, latch, clock_gate, macro, physical). *Data_type*: bool, optional
`-hierarchical_instance` *hinst_name*	Reports the area of the leaf instances in the specified hierarchical instance. You can use the -depth option to control the analysis depth of the specified hierarchical instance. *Data_type*: (hinst), optional
`-include_physical`	Specifies to include physical cells in the area calculation and report. By default, it reports only logical cells. *Data_type*: bool, optional
`-min_count` *instance_count*	Specifies the min instance count per module. The generated output will report the area of hinsts with number of instances greater than the specified `min_count value`. *Default*: `0` *Data_type*: float, optional
`-` `min_area` *area_per_module*	Specifies the min area per module. The generated output will report the area of hinsts with area greater than the specified min_area lib_units. *Default*: `0` *Data_type*: int, optional
`-out_file` *filename*	Specifies the name of the output file. *Data_type*: string, optional

`-show_leaf_cells`	Specifies to include the leaf cells at the design level and each hinst level. These are the instances whose parent is the hinst display. *Default*: `0` (Shows the hinst children in the report) *Data_type*: bool, optional
`-show_msv_cells`	Includes multiple supply voltage cells in the report. This option works with "`-table_style vertical`"
`-sort_by {name count area}`	Specifies the order of hinst names. *Default*: `name` *Data_type*: bool, optional
`-table_style {horizontal vertical}`	Specifies whether the base report should be expanded in the vertical or horizontal direction. *Default*: `horizontal` *Data_type*: enum, optional
`-summary`	It will print the area summary for the design. It will be equivalent as doing "`-depth 0`". *Data_type*: bool, optional

Examples

The following command shows the standard cell area in different base classes.

```
report_area -detail
```

Hinst Name	Module Name	Inst Count	Total Area	Combinational	Buffer	Inverter	Flop	Latch	Clock-Gate	Macro
dtmf_recvr_core		4283	37949.153	5223.91	9.526	262.836	2535.221	0.000	0.000	0.000
RESULTS_CONV_INST	results_conv	1331	2201.472	1126.49	0.000	79.38	995.602	0.000	0.000	0.000
TDSP_CORE_INST_MPY_32_INST_csa_tree_mul_61_28	csa_tree	623	1158.419	1115.024	0.000	43.394	0	0.000	0.000	0.000
TDSP_CORE_INST_EXECUTE_INST	execute_i	570	1251.382	504.151	0.000	28.048	719.183	0.000	0.000	0.000
TDSP_CORE_INST_ALU_32_INST	alu_32	466	308.593	742.644	0.000	36.515	0.000	0.000	0.000	0.000
sub_84_22	sub_unsigned_801	93	39.421	170.05	0.000	15.876	0.000	0.000	0.000	0.000
add_81_22	add_unsigned_799	77	28.016	170.755	0.000	0.529	0.000	0.000	0.000	0.000
inc_add_54_27_1	increment_unsigned	31	14.262	107.604	0.000	0	0.000	0.000	0.000	0.000
TDSP_CORE_INST_MPY_32_INST_final_adder_mul_61_28	add_unsigned	74	27.634	165.287	0.000	0	0.000	0.000	0.000	0.000
TDSP_CORE_INST_MPY_32_INST_addinc_add_62_54_8	add_signed_carry	76	28.573	163.523	0.000	0.529	0.000	0.000	0.000	0.000
TDSP_CORE_INST_TDSP_CORE_GLUE_INST_sll_120_49	shift_left_vlog_unsigned_393	92	51.014	128.772	0.000	3.175	0.000	0.000	0.000	0.000
TDSP_CORE_INST_TDSP_CORE_GLUE_INST_sll_121_36	shift_left_vlog_unsigned	56	30.828	94.55	0.000	1.058	0.000	0.000	0.000	0.000

The following command shows the leaf cells at the hinst level.

```
report_area -show_leaf_cells


          Hinst Name                                  Module Name                  Cell
Count  Cell Area  Total Area

-------------------------------------------------------------------------------------
-------------------------

dtmf_recvr_core
 4283  37949.153  41732.426

 (Leaf Cells)
995  32001.000  33406.000

 RESULTS_CONV_INST                                  results_conv
 1331   2201.472   3311.412

 TDSP_CORE_INST_MPY_32_INST_csa_tree_mul_61_28    csa_tree
623   1158.419   1618.517

 TDSP_CORE_INST_EXECUTE_INST                        execute_i
 570   1251.382   1613.524

 TDSP_CORE_INST_ALU_32_INST                         alu_32
466    779.159   1087.752

   sub_84_22                                        sub_unsigned_801
```

```
 93    185.926    225.347

  add_81_22                                          add_unsigned_799
 77    171.284    200.101

  inc_add_54_27_1                                    increment_unsigned
 31    107.604    121.866

 TDSP_CORE_INST_MPY_32_INST_final_adder_mul_61_28 add_unsigned
 74    165.287    192.921

 TDSP_CORE_INST_MPY_32_INST_addinc_add_62_54_8    add_signed_carry
 76    164.052    192.625

 TDSP_CORE_INST_TDSP_CORE_GLUE_INST_sll_120_49    shift_left_vlog_unsigned_393
 92    131.947    182.961

 TDSP_CORE_INST_TDSP_CORE_GLUE_INST_sll_121_36    shift_left_vlog_unsigned
 56     95.609    126.436
```

The following command shows the base report in the vertical direction.

```
report_area -detail -table_style vertical
```

```
 Hinst Name                  Module Name           Inst Count           Total Area
-------------------------------------------------------------------------------
dtmf_recvr_core                                        4283             37949.15
  (Buffer)                                                9                 9.53
  (Inverter)                                            491               262.84
  (Combinational)                                      3279              5223.91
  (Flop)                                                501              2535.22
  (Latch)                                                 0                 0.00
  (Clock-Gate)                                            0                 0.00
  (Macro)                                                 3             29917.66
  RESULTS_CONV_INST          results_conv              1331              2201.47
    (Buffer)                                              0                 0.00
```

(Inverter)	150	79.38
(Combinational)	981	1126.49
(Flop)	200	995.60
(Latch)	0	0.00
(Clock-Gate)	0	0.00
(Macro)	0	0.00

report_command_mode

```
report_command_mode
[-help]
[-filter list_of_commands]
{-non_default | -user}
```

Reports the *Mode options that do not match the default value or the options that have been manually set.

Parameters

`-help`	Prints a brief description that includes type and default information for each `report_command_mode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_command_mode`
`-filter` *list_of_commands*	Specifies the commands that will be reported by `report_command_mode`. *Data_type*: string, optional
`-non_default`	Reports all the *Mode options that do not match the default value. *Data_type*: bool, optional
`-user`	Reports all the *Mode options that have been manually set. *Data_type*: bool, optional

Examples

- The following command displays the mode options that do not match the default value for all the commands:

  ```
  report_command_mode -non_default
  ```

  Output:

  ```
  setDesignMode -topRoutingLayer 6
  setDesignMode -process 40
  setDelayCalMode -enable_high_fanout true
  ```

- The following command displays the mode options that do not match the default value for the `setNanoRouteMode` command:

  ```
  report_command_mode -non_default -filter setDesignMode
  ```

  Output:

  ```
  setDesignMode -topRoutingLayer 6
  ```

- The following command displays the mode options that do not match the default value for the `setDesignMode` command:

  ```
  report_command_mode -non_default -filter {setDesignMode}
  ```

  Output:

  ```
  setDesignMode -topRoutingLayer 6
  ```

report_disk

```
report_disk
[-help]
[-dir directory_name]
```

Reports the disk metrics of the current working directory and the /tmp directory. The output of this command allows you to investigate Mb/sec throughput of the disk and disk I/O requirements.

This command also reports the used and free disk space for the directory specified by the `-dir` parameter (default is /tmp).

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_disk` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_disk`.
`-dir` *`directory_name`*	Specifies the name of the directory in which disk metric information is written. The default directory is `/tmp`.

Examples

- The following is a sample output for `report_disk`:

```
>report_disk

DISK_INFO : 206.66 Mb/S   381G total   1.1G used   359G free   /env(TMPDIR)

{206.66 381G 1.1G 359G}
```

- The following command shows the usage of `report_disk -dir`:

```
>report_disk -dir .

DISK_INFO : 98.51 Mb/S   9.6T total   9.3T used   395G free   .

{98.51 9.6T 9.3T 395G}
```

report_hidden_usage

```
report_hidden_usage
in_files
```

Reports a summary of all the hidden application Tcl globals and mode options in the specified files. Hidden globals and mode options are normally used for temporary fixes or early access, and often disappear or change in a new release. This command will help to find out the hidden settings used in existing scripts that might not be needed in the new release.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_hidden_usage` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_hidden_usage`
in_files	Specifies input files to search for hidden usage. *Data_type*: string, required

Examples

- The following command reports the hidden settings in the floorplan.tcl and place.tcl files.

```
>report_hidden_usage {floorplan.tcl place.tcl} > hidden.rpt

#########################################################################

#  Generated by:    Cadence Innovus 18.10-a072_1

#  OS:                        Linux x86_64(Host ID sjfib146)

#  Generated on:    Tue Apr 17 22:47:31 2018

#  Design:                top
```

```
#  Command:           report_hidden_usage {floorplan.tcl place.tcl} > hidden.rpt
#########################################################################


### floorplan.tcl ###
-------------------------------------------------------------------------------
Line                         Hidden name                      Command
-------------------------------------------------------------------------------
14                           fpAllowXXX                       set fpAllowXXX true
34                           -XXX                             setFPlanMode -XXX 4


###place.tcl###
-------------------------------------------------------------------------------------
--
Line                        Hidden name                       Command
-------------------------------------------------------------------------------------
---
62                          -place_xxx                      setPlaceMode -place_xxx
true
```

report_message

```
report_message
[-help]
[[{-errors | -warnings | -all } [-count ]] | -suppressed ] [[-prefix prefix [-detail ]
[-summary ]] ]
[-detail | -count | -suppressed ] [-summary | -count | -suppressed ]
```

Reports the messages that have been issued since the tool was launched. When you run `report_message` without any argument, it prints a summary table and returns no Tcl value. However, when you provide an argument, it returns a list a of message IDs and does not print a table.

Parameters

`-help`	Prints a brief description that includes type and default information for each `report_message` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_message`
`-all`	Returns a list of all issued message IDs as a Tcl result. The result does not include suppressed message IDs. *Data_type*: bool, optional
`-count`	Returns the count of every message when `-errors`, `-warnings`, or `-all` is specified. For example, if an error appeared 20 times, command will return: `{<Error ID> 20}` *Data_type*: bool, optional
`-detail`	Returns a list of detail message of the specified prefix. *Data_type*: bool, optional
`-errors`	Returns a list of issued error message IDs as a Tcl result. *Data_type*: bool, optional
`-prefix` *prefix*	Returns a list of issued error message IDs matched with `-prefix` specified as a Tcl result. *Data_type*: string, optional
`-summary`	Returns the summary of the message that has the specified prefix. *Data_type*: bool, optional
`-suppressed`	Returns a list of suppressed message IDs as a Tcl result. *Data_type*: bool, optional
`-warnings`	Returns a list of issued warning message IDs as a Tcl result. *Data_type*: bool, optional

Examples

- The following command prints a summary table and returns no Tcl value.

```
report_message
```

Output:

```
*** Summary of all messages that are not suppressed in this session:
Severity           ID               Count              Summary
WARNING          ENCFP-2101         6                Shifter for %s to %s is not
defined in s...
WARNING          ENCFP-3961         1                techSite '%s' has no related
Cells in LE...
WARNING          ENCMSMV-1610       4                View %s domain %s operating
voltage %g d...
WARNING          ENCTS-3            692              Can't create min and max cell
relation f...
WARNING          ENCTS-419          119              Timing arc(s) mismatch found
in cell '%s...
ERROR            ENCTS-423          5                No library found for instance
'%s', cell...
ERROR            ENCTS-424          1                Missing library for some
instance found...
WARNING          ENCEXT-6202        1                In addition to the technology
file, capa...
WARNING          ENCSYT-3046        2                Unknown preference name "%s".
WARNING          ENCSYC-1592        2                No default tech site found for
power dom...
WARNING          ENCPP-4022         2                Option "-%s" is obsolete and
has been re...
WARNING          ENCSC-1001         2                Unable to trace scan chain
"%s". Check t...
WARNING          ENCSC-1020         1                Instance's output pin "%s/%s"
(Cell "%s"...
WARNING          ENCSC-1114         2                The scan chain "%s" is not
traced.
WARNING          ENCSC-1143         1                Unable to apply DEF ordered
sections for...
WARNING          ENCSC-1144         2                Scan chain "%s" was not traced
through. ...
WARNING          ENCOPT-3058        1003             Cell %s/%s has already a
dont_use attrib...
WARNING          ENCOAX-571         1                Property '%s' from OA is a
```

```
hierarchical ...
WARNING          ENCOAX-1257        1               Allowing saveDesign to create
OpenAccess...
WARNING          ENCTCM-77          1               Option "%s" for command %s is
obsolete a...
*** Message Summary: 1843 warning(s), 6 error(s)
```

- The following command reports all issue message IDs.

  ```
  report_message -all
  ```

- The following command reports all issue message IDs.

  ```
  report_message -suppressed
  ```

- The following command prints the user-defined errors.

  ```
  innovus > report_message
  *** Summary of all messages that are not suppressed in this session:
  Severity      ID       Count         Summary
  ERROR       no_id       1           A test %s.
  ERROR       user        1           A test %s.
  WARN        user        1           A test %s.
  *** Message Summary: 1 warning(s), 2 error(s)
  ```

reportDanglingPort

```
reportDanglingPort
-help
[-ptn]
-outfile fileName
```

Reports ports that are disconnected from both the inside and outside of a module, and ports that are connected within the module but disconnected outside the module.

Note: If the design has some dangling or unconnected ports, Innovus will add `FE_UNCONNECTED_*` nets for these dangling or unconnected ports in the design during optimization. You can report these dangling ports using the `reportDanglingPort` command.

Parameters

`-help`	Prints a brief description that includes type and default information for each `reportDanglingPort` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reportDanglingPort`
`-outfile`	Specifies the name of the report generated by this command. *Data_type*: string, required
`-ptn`	Specifies a partition only flag. *Data_type*: bool, optional

Example

The following command writes information on dangling reports to file `myreport`:

```
reportDanglingPort -outfile myreport
```

Related Information

- `deleteDanglingPort`
- `disconnectDanglingPort`

reportGateCount

```
reportGateCount
[-help]
[-hinst hinst_name]
[-level level]
[-limit gateCount]
[-stdCellOnly]
[-outfile fileName]
```

Reports the size of the imported design, measured in gate counts.

The software calculates the gate count for a module using the following equation:

```
gateCount = moduleArea / gateSize
```

Where:

- `moduleArea` is the area of the module. By default, this is the sum of the areas of all instances inside of the module, including standard cells, blocks, and I/O cells. If you specify the `-stdCellOnly` parameter, `moduleArea` is the sum of the areas of the standard cells inside the module only.
- `gateSize` is calculated as follows:

  ```
  gateSize = dbgStdCellHgt x dbgDBUPerIGU x dbgSitesPerGate
  ```

  Where:

  - `dbgStdCellHgt` is the standard cell row height.
  - `dbgDBUPerIGU` is the *M2* layer pitch.
  - `dbgSitesPerGate` is a user-defined global variable that determines the gate size the software assumes when calculating the gate count. For example, the default value of `3` means the assumed the gate size is equal to 1 standard cell row height and 3 *M2* layer pitch widths. You can change this value to adjust the assumed gate size for each specific design.

You can use the `reportGateCount` command after importing the design.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `reportGateCount` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man reportGateCount`.
`-level` *level*	Expands the module to the specified level. For example, if you specify `1`, CTS expands the specified module by one level. The gate counts of all "children" modules of the module that satisfy the `-limit` parameter are included in the gate count report. If you specify `2`, CTS expands the module by two levels. The gate counts of all "children" and "grandchildren" modules of the module are included in the gate count report. *Default*: Module is not expanded, and CTS reports only the gate count of the module itself. (This is equivalent to specifying a level of `0`.) *Data_type*: int, optional
`-limit` *gateCount*	Instructs CTS to ignore modules with gate counts below the specified limit. Such modules are ignored during expansion and reporting. *Default*: CTS ignores modules with 1000 or fewer gates during expansion and reporting. *Data_type*: int, optional
`-hinst` *hinst_name*	Specifies the hierarchical instance for which to generate a gate count report. *Default*: Generates a gate count report for the hierarchical instance corresponding to the top cell of the design. *Data_type*: string, optional
`-outfile` *fileName*	Writes out the gate count report to the specified file name. The default file extension is `.gateCount`. *Default*: Displays the gate count report *only* in the Innovus console. *Data_type*: string, optional
`-stdCellOnly`	Calculates gate count based only on standard cells within the specified module. Blocks and I/O cells within the module are ignored. *Data_type*: bool, optional

reportNetStat

```
reportNetStat
```

Reports statistical data of the design netlist such as number of cells, number of nets (including tri-nets and degen nets), number of pins, and number of IOs in the design and number nets with different number of terms. The primitives used in the design are also listed.

Example

```
*** Statistics for net list DTMF_CHIP ***
Number of cells      = 4263
Number of nets       = 4551
Number of tri-nets   = 0
Number of degen nets = 21
Number of pins       = 16490
Number of i/os       = 57

Number of nets with    1 terms = 21 (0.5%)
Number of nets with    2 terms = 2510 (55.2%)
Number of nets with    3 terms = 771 (16.9%)
Number of nets with    4 terms = 510 (11.2%)
Number of nets with    5 terms = 223 (4.9%)
Number of nets with    6 terms = 108 (2.4%)
Number of nets with    7 terms = 76 (1.7%)
Number of nets with    8 terms = 46 (1.0%)
Number of nets with    9 terms = 30 (0.7%)
Number of nets with >=10 terms = 256 (5.6%)

*** 72 Primitives used:
Primitive ADDFHX1 (195 insts)
Primitive ADDHX1 (15 insts)
Primitive AND2X1 (279 insts)
Primitive AND3X1 (1 insts)
Primitive AND4X1 (8 insts)
Primitive AOI211X1 (65 insts)
Primitive AOI21X1 (282 insts)
```

resume

```
resume [-help]
```

Resumes a suspended script from the point where it had stopped.

Use the `suspend` and `resume` commands to debug your scripts interactively. The `suspend` command stops a script and restores access to the Innovus prompt. You can then type any command required for debugging. Whenever you want to resume the suspended script, type `resume` at the Innovus prompt.

Parameters

`-help`	Prints the usage of the `resume` command. For a detailed description of the command, use the `man` command: `man resume`.

Related Information

- `suspend`
- "Interrupting the Software" section of the Getting Started chapter in the *Innovus User Guide*

savePreference

```
savePreference
[-help]
[-include_window_font]
{[-directory {home current} | -file_name PreferenceFileName]}
```

Saves the preference file.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `savePreference` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man savePreference`.
`-directory {home current}`	Specifies the directory path.
`-file_name PreferenceFileName`	Specifies the saved filename.
`-include_window_font`	Specifies the window and font settings.

Related Information

- `getPreference`
- `setPreference`
- For more information on setting preferences from the GUI, see Set Preference form in the View Menu chapter in the *Innovus Menu Reference*.

select_obj

```
select_obj
[-help]
<any_object>+
```

Selects the object or the list of objects for putting into a set. The object type can be any object type selectable on the GUI such as inst and net.

You can add a new selection to an already selected object using the following command:

```
select_obj [concat [dbGet selected.] new_object_pointer]
```

Parameters

`-help`	Prints a brief description that includes type and default information for each `select_obj` parameter. For a detailed description of the command and all of its parameters, use the man command: `man select_obj`
`<any_object+`	Specifies the list of objects. **Note:** If the list contains the objects that cannot be selected, such as `head`, `layer`, and `ptn`, `select_obj` will not select them. The objects that cannot be selected are `head`, `layer`, `ptn`, `fPlan`, `prop`, `bus`, `vCell`, `viaRuleGenerate`, `layerRule`, `foreign`, `hNet`, `site`, `densityShape`, `shape`, `rackDef`, and `gCellGridDef`. *Data_type*: (any_object)+, required

Example

```
select_obj [dbGet -p top.insts.name i1]
```

Related Topics

- `deselect_obj`

selectObjByProp

```
selectObjByProp
[-help]
{Instance Pin Net Module InstanceGroup Bump}
expression
```

Logs the select operations performed through the Find/Select Object form (*View -- Find/Select Object*).

Parameters

`-help`	Outputs a brief description that includes type and default information for each `selectObjByProp` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man selectObjByProp`.
`{Instance Pin Net Module InstanceGroup Bump}`	
	Specifies the name of the object(s) on which the select operation is performed. *Data_type*: enum, required
expression	Specifies the selection criteria specified through the Find/Select Object form. *Data_type*: string, required

Examples

- The following command logs the select operation for all instances of type BLOCK in the *View - Find/Select Objects* form. In this case, the value of the Mode field was set to *Single* in the *Find/Select Objects* form.

  ```
  selectObjByProp Instance <Type>=<Block>
  ```

- The following command logs the select operation for all I/O pins that are on the layer Metal2 and whose status is not *placed*. In this case, the value of the Mode field was set to *And* in the *Find/Select Objects* form.

  ```
  selectObjByProp Pin {And(<Type>=<I/O Pin>,!<Status>=<Placed><layer>=<Metal2>)}
  ```

- The following command logs the select operation for all modules whose constraint type is Fence or Region. In this case, the value of the Mode field was set to *OR* in the *Find/Select*

Objects form.

```
selectObjByProp Module {OR(<Constraint Type>=<Fence>,<Constraint Type>=<Region>)}
```

- The following command logs the select operation for all pins of the modules DTMF_INST/TDSP_CORE_INST and DTMF_INST/ARB_INST that are on the top side and have a status of *placed*, *fixed*, or *cover.* In this case, the value of the Mode field was set to *Multiple* in the Find/Select Objects form.

```
selectObjByProp Pin And(And(!<Status>=<Unplaced>,<Side=
<Top>),OR(<Module>Match<DTMF_INST/TDSP_CORE_INST>,
<Module>Match<DTMF_INST/ARB_INST>))
```

Related Topics

- For information on *Find/Select Objects* form, see "Find/Select Object" in the View Menu chapter of the *Innovus Menu Reference*.

set_global_always_keep

```
set_global_always_keep
[-help]
variableName
```

Saves the specified UDM variable to the .global file.

Parameters

`-help`	Prints a brief description that includes type and default information for each `set_global_always_keep` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_global_always_keep`
`variableName`	Specifies the variable name. *Data_type*: string, required

Example

The following command will save the UDM variable `timing_disable_internal_inout_cell_paths` with the value saved by the user:

```
> set_global_always_keep timing_disable_internal_inout_cell_paths
```

Then, after `saveDesign`, the following line will appear the in the `.global` file:

```
set_global timing_disable_internal_inout_cell_paths true
```

Related Information

- `saveDesign`

set_message

```
set_message
-help
-id <list_of_msg_ids>
[-severity {warn error info reset} | -limit <number> | -no_limit | -suppress | -
unsuppress | -on_error {msg_only exit stop_script | -stop_script_limit limitNumber}]
```

Specifies to change the message severity levels, `INFO, WARNING` or `ERROR`, at the beginning of the message. This command allows you to control the messages that are displayed for a particular design. For example, if there are some warnings that are critical for the design, you can change the message severity to `ERROR`. Similarly, if there are messages that you want to ignore, you can change the message severity to `INFO`.

An example of an ERROR message is:

**ERROR: (GLOBAL-105): Unknown global 'timing_enable_socv_analysis' specified.

All ERROR and WARN message begin with a header like:

**ERROR: (<msg_id>):

**WARN: (<msg_id>):

The <msg_id> format is: <prefix>-<num> ("GLOBAL" is the prefix, and "105" is the num in the example above).

You can use the msg_id to change how that specific message is handled using the options below.

Parameters

-help	Outputs a brief description that includes type and default information for each `set_message` parameter.
`-id` *list_of_msg_ids*	Specifies the list of message IDs that require a change. If you do not specify any message IDs, the change is applied to all message IDs. *Data_type*: string, optional

`-limit` *`number`*	Sets the message limit to *`number`* per every public command. The default limit is 20, so any specific message will be written out 20 times for one command and then further output of that message will be disabled (along with an additional message saying the message limit has been exceeded). If the number is set to 0, the message will be completely suppressed for the next public command that triggers the message. You can unsuppress the message by setting the number greater than 0. *Data_type*: int, optional
`-no_limit`	Removes any existing limit on the selected messages. **Note:** Suppressed messages remain suppressed until the `-unsuppress` option is applied. *Data_type*: bool, optional
`-on_error {msg_only exit stop_script}`	Specifies an extra action for error messages (not for warn or info messages). If no message is specified, the action applies to all error messages. • `msg_only`: Prints error message only, and the command proceeds as it normally would. This is the default. • `exit`: Exits immediately and returns to Linux. • `stop_script`: Waits for the current Tcl command to finish, then stops executing any Tcl script, and returns to the interactive Tcl prompt. Note, if a message severity is changed from error to either warn or info, it will no longer trigger this extra action. *Data_type*: enum, optional

`-severity {warn error info reset}`	Specifies the new severity level for the message being modified. The `reset` argument sets the message severity back to its original state. Normal INFO messages do not have a message ID, but if a warn or error message severity is changed to info, the INFO message will include the message ID in the message header. For example, the INFO header might look like: INFO: (MSG-123): ... *Data_type*: enum, optional
`-stop_script_limit` *limitNumber*	Sets the message limit per every public command. If you set `-stop_script_limit 10`, at the 10th time the message being triggered for one command, the tool will wait for current Tcl command to finish, then stop executing any Tcl script, and return to the interactive Tcl prompt. The default value is `-1`, which is unlimited. *Data_type*: int, optional
`-suppress`	Suppresses the message completely. There is no warning about exceeding message limits given. *Data_type*: bool, optional
`-unsuppress`	Unsuppresses the message. *Data_type*: bool, optional

Examples

- The following command changes two messages from `ERROR` to `WARN`:
  ```
  set_message -id {IMPLF-81 ENCLF-95} -severity warn
  ```
- The following command returns all messages to their original severity:
  ```
  set_message -severity reset
  ```
- The following command resets the specified message to its original severity:
  ```
  set_message -id IMPLF-81 -severity reset
  ```

Related Topics

- `setMessageLimit` command
- `get_message` command

set_proc_verbose

```
set_proc_verbose
[-help]
{[procedure_name [-quiet ]] | [-report ]}
```

Makes a procedure verbose, when the procedure is called for execution.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_proc_verbose` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_proc_verbose`.
procedure_name	Specifies the name of the procedure to be made verbose. *Data_type*: string, required
`-quiet`	Reverts the procedure back from being verbose. *Data_type*: bool, optional
`-report`	Reports procedures that are verbose. *Data_type*: bool, optional

Examples

- The following commands show the usage of the `set_proc_verbose` command and its options for the `fplan_boxes` procedure.

  The code within the `fplan_boxes` procedure is given below:

  ```
  innovus 1> proc fplan_boxes {} {
  + puts "Floor Plan Boxes:"
  + puts [dbGet top.fplan.boxes]
  + }
  ```

The following command executes the `fplan_boxes` procedure. Please note that the execution result simply shows the coordinates of the floor plan boxes.

```
innovus 2> fplan_boxes

Floor Plan Boxes:

{{0.0 0.0 1511.96 1512.12}}
```

The following commands makes the `fplan_boxes` procedure verbose:

```
innovus 3> set_proc_verbose fplan_boxes

fplan_boxes
```

After the `fplan_boxes` procedure is made verbose and is executed, the code of the `fplan_boxes` procedure is displayed:

```
innovus 5> fplan_boxes

#@ Begin verbose proc fplan_boxes

@proc(fplan_boxes) 1: puts "Floor Plan Boxes:"

Floor Plan Boxes:

@proc(fplan_boxes) 2: dbGet top.fplan.boxes

@proc(fplan_boxes) 3: puts [dbGet top.fplan.boxes]

{{0.0 0.0 1511.96 1512.12}}

#@ End verbose proc fplan_boxes
```

You can use the `-report` option to see the verbose procedures:

```
innovus 7> set_proc_verbose -report

innovus 8> fplan_boxes
```

If you have made your procedure verbose and now you want to make it non-verbose, you can use the `-quite` option. The command below makes the `fplan_boxes` procedure non-verbose:

```
innovus 6> set_proc_verbose fplan_boxes -quiet

fplan_boxes
```

setCheckMode

```
setCheckMode
[-help]
[-reset]
[-all {true | false}]
[-checkIlm {true | false}]
[-extraction {true | false}]
[-floorplan {true | false}]
[-globalNet {true | false}]
[-integrity {true | false}]
[-io {true | false}]
[-library {true | false}]
[-mgrid {true | false}]
[-netlist {true | false}]
[-placement {true | false}]
[-route {true | false}]
[-sroute {true | false}]
[-tapeOut {true | false}]
[-timingGraph {true | false}]
[-vcellnetlist {true | false}]
```

Sets the data checks that the software performs.

If the software finds missing or incorrect data during the check, it

- Does not proceed with the functionality that would be affected by the incomplete or incorrect data
- Generates an error message in the Innovus console

Use the `getCheckMode` command to display the current settings for the `setCheckMode` command.

Use the `setCheckMode` command immediately before importing the design, or before performing a functionality.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setCheckMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setCheckMode`.
`-all {true | false}`	Performs all data checks. *Default*: `false` *Data_type*: bool, optional
`-checkIlm {true | false}`	Checks if no instances are inserted inside an ILM and whether the ILM boundary nets are consistent between flattened and unflattened states. *Default:* `false` *Data_type*: bool, optional
`-extraction {true | false}`	Checks that all data required for extraction are correct. **Note:** Although this parameter instructs the Innovus software to check that routing is complete, this parameter does *not* check wiring for DRC violations. *Default*: `false` *Data_type*: bool, optional
`-floorplan {true | false}`	Checks the floorplan. *Default:* `false` *Data_type*: bool, optional
`-globalNet {true | false}`	Checks instance pin and power/ground pin net connections. *Default:* `false` *Data_type*: bool, optional
`-integrity {true | false}`	Checks integrity. *Default:* `false` *Data_type*: bool, optional

`-io {true | false}`	Checks the I/O pads. *Default:* `false` *Data_type*: bool, optional
`-library {true | false}`	Checks the physical (LEF) and timing libraries. *Default:* `false` *Data_type*: bool, optional
`-mgrid {true | false}`	Checks that geometries are on the manufacturing grid. **Note:** The software does not stop this check, even if there are problems in the DEF file. The software generates an error message if it finds any problems in the DEF file. *Default:* `false` *Data_type*: bool, optional
`-netlist {true | false}`	Checks the netlist. *Default:* `false` *Data_type*: bool, optional
`-placement {true | false}`	Checks the placement of the design. *Default:* `false` *Data_type*: bool, optional
`-reset`	Resets parameters to their default values. The `-reset` parameter *must* be the first parameter specified. If you specify `-reset` by itself, the software resets all `setCheckMode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported. *Data_type*: bool, optional

`-route {true | false}`	Checks whether power and ground pins are connected to power and ground nets before power routing. The software also checks whether all tie high and tie low pins are connected to global power and ground nets before detailed routing. *Default*: `false` *Data_type*: bool, optional
`-sroute {true | false}`	Checks special routes. *Default:* `false` *Data_type*: bool, optional
`-tapeOut {true | false}`	Runs the software in tapeout mode instead of in protyping mode. In tapeout mode, the software stops running if it detects errors that would seriously affect the overall integrity of results. In prototyping mode, the software does not stop if it detects such errors. *Default:* `false` *Data_type*: bool, optional
`-timingGraph {true | false}`	Checks library cells for timing information. *Default:* `false` *Data_type*: bool, optional
`-vcellnetlist {true | false}`	Checks for tie-high and tie-low output pins when reading the Verilog netlist. The software stops the flow if tie-high or tie-low output pins are found because connecting an output signal pin to a power or ground creates a potential short circuit. If all of the tie-high or tie-low output pins are actually the driver pins for power or ground nets, specify `-vcellnetlist false`, so that the flow can continue. **Note:** To turn off the checking, you *must* set this parameter before initializing the design. *Default:* `true` *Data_type*: bool, optional

Examples

- The following command turns on all data-checking modes except the pre-power-routing data checking:

  ```
  setCheckMode -all true -route false
  ```

- The following command checks a DEF file for objects that are off the manufacturing grid.

  ```
  setCheckMode -mgrid true
  ```

  The software generates error messages such as the following:

  ```
  **ERROR: Line 56: OffMGrid: Track start value (566) is not on Manufacturing Grid.

  **ERROR: Line 446: OffMGrid: Via (-10000, -10000) (10006, 10006) named via3Array_1
  is not on Manufacturing Grid.

  **ERROR: Line 902: OffMGrid: Component location (130686, 171366) of inst
  iblockN20/iregNS/yi_reg_1 is not on Manufacturing Grid.
  ```

- The following command resets the `-mgrid` parameter to its default value:

  ```
  setCheckMode -reset -mgrid
  ```

- The following command resets all `setCheckMode` parameters to their default values:

  ```
  setCheckMode -reset
  ```

Related Topics

- `getCheckMode`

setCompressLevel

`setCompressLevel` *value*

Specifies the compression level to use when compressing large data files. A higher compression level creates a more compressed file, but at the cost of speed. A lower compression level creates a less compressed file, but speeds up the process. By default, the software uses the `gzip` default value, that is 6, which results in high compression at the cost of speed.

Parameters

`-help`	Prints a brief description that includes type and default information for each `setCompressLevel` parameter. For a detailed description of the command and all of its parameters, use the man command: man `setCompressLevel`
value	Specifies the compression level to use when compressing files. You can specify -1 or values between 1 and 9, where: `1` Creates the least compressed files at the highest speed. `9` Creates the most compressed files at the lowest speed. -1 Returns to the `gzip` default value, that is 6 (`Z_DEFAULT_COMPRESSION`). *Data_type*: int, required

Examples

- The following command sets the highest compression level and results in the most compressed data:
  ```
  setCompressLevel 9
  ```
- The following command resets the compression level to the gzip default, that is 6.
  ```
  setCompressLevel -1
  ```

Related Topics

- `getCompressLevel`

setLayerPreference

```
setLayerPreference
[-help]
layer_name
[-isVisible {1 | 0}]
[-isSelectable {1 | 0}]
[-color {color_name | color_value}]
[-lineWidth {lw}]
[-stipple stipple_name | -stippleData width height "stipple_data"]
```

Sets a new preference for the specified object or layer.

See `getLayerPreference` for descriptions of how to display layer preferences.

Parameters

`-help`	Prints a brief description that includes type and default information for each `setLayerPreference` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setLayerPreference`
`-color`	Specifies a color for the object. Specify either a color name or a numeric RGB value. *Date_type*: string, optional
`-isSelectable`	Specifies whether the object is selectable. Specify `1` to make the object selectable. Specify `0` to make the object non-selectable. *Date_type*: enum, optional

`-isVisible`	Specifies whether the object is visible. Specify `1` to make the object visible. Specify `0` to make the object invisible.
	Examples
	• You can turn off the visibility of all custom layers by using the following command:
	`setLayerPreference customLayers -isVisible 0`
	• You can turn off display of flightlines by using the following command:
	`setLayerPreference flightLine -isVisible 0`
	Date_type: enum, optional

layer_name	Specifies the name of the layer or the object type. The valid layer names are: • layer number • `#AVio`, `#Cont`, `#ContPin`, `#ContRB`, `#Pin`, `#RB`, `#RTrk`, `#Track`, and `M#Vio`, where `#` is the number representing layer. • `MGroup*` for controlling preference for by group, where * represents a group such as Pin or Track. The valid object types are `blackBox`, `block`, `blockHalo`, `bump`, `bump1`, `bump2`, `bump3`, `cellBlkgObj`, `channel`, `clkTree`, `congest`, `congestH`, `congestObj`, `congestV`, `datapath`, `fence`, `gcellOvflow`, `guide`, `hinst`, `inst`, `ioSlot`, `ioRow`, `macroSitePattern`, `metalFill`, `mGrid`, `net`, `netRect`, `nonPrefTrackObj`, `obstruct`, `pinObj`, `pinText`, `power`, `powerNet`, `ptnFeed`, `ptnPinBlk`, `pwrdm`, `pwrdm1`, `pwrdm2`, `pwrdm3`, `layerBlk`, `region`, `routeGuide`, `ruler`, `screen`, `select`, `sitePattern`, `stdRow`, `term`, `text`, `trackObj`, `userGrid`, and `violation`.
`-lineWidth`	Specifies the number of pixels in the border of the specified object. Specify this value as integer. *Date_type*: int, optional

`-stipple`	Specifies information about the stipple pattern of the object. The valid stipple names are `None`, `Horizontal`, `Vertical`, `Grid`, `Slash`, `Backslash`, `Cross`, and `Brick`. *Date_type*: string, optional
`-stippleData`	Specifies the stipple information for the object. Specify this information in x-window bitmap format. *Date_type*: string, optional

Examples

- The following command sets a layer preference for the module object in green with dotted stipple pattern (8 by 4 bitmap with dot at every 4 pixels):
  ```
  setLayerPreference hinst -color green -stippleData 8 4 "0x00 0x11 0x00 0x00"
  ```
- The following command turns off the visibility of the track for Metal1:
  ```
  setLayerPreference Metal1Track -isVisible 0
  ```
- The following command turns on the selectability of the track for Metal1:
  ```
  setLayerPreference Metal1Track -isSelectable 1
  ```
- The following command turns off selectability for all tracks:
  ```
  setLayerPreference MetalGroupTrack -isSelectable 0
  ```

Related Topics

- `getLayerPreference`

setLicenseCheck

```
setLicenseCheck
[-help]
[-checkout lic]
[-default]
[-existOnServer]
[-optionList "lic1 lic2 ..."]
[-status]
[-wait time_in_minutes]
```

Manages licenses for product options. Specifies licenses to check out, license wait time, and the status of all product option licenses.

Note: This command does not manage multi-CPU licenses. For information on the commands that manage multi-CPU licenses, see the "*Multiple-CPU Processing Commands*" chapter.

Parameters

`-help`	Prints a brief description that includes type and default information for each `setLicenseCheck` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setLicenseCheck`.
`-checkout lic`	Checks out the allowed optional license immediately. *Date_type*: string, optional
`-default`	Restores the default. values for this command. *Date_type*: bool, optional

`-existOnServer`	Prints out all licenses that exist on the license server. The following information is included for each license. • License name • Product number • Product name • License string • Version *Date_type*: bool, optional
`-optionList "`*`lic1 lic2 ...`*`"`	Specifies an ordered list to limit dynamic optional license check-out. Use `-status` to see legal names. Dynamic licenses are not checked out until they are needed. If you specify a license that is not allowed with your base license, or a license that is not available, the software does not check out that license and instead issues a warning message. If you specify more than one license, begin and end the option list with double quotation marks or braces. *Date_type*: string, optional
`-status`	Prints the current status of all licenses. The following information is included: • The name of the base license that is checked out • The current wait time • A list of the allowed license options • A list of the available license options • A list of the currently checked-out license options. *Date_type*: bool, optional
`-wait` *`time_in_minutes`*	Specifies the amount of time the system waits for a license to become available. If the license is available in less than the specified wait time, the system checks out the next needed license without waiting. *Default:* `0` (no wait time) *Value range:* `0` to `10,000` *Date_type*: int, optional

Examples

- The following command outputs a table that provides information about the licenses that exist on the license server:
  ```
  setLicenseCheck -existOnServer
  ```
  The system prints the following information:
  ```
  Name          Prod #    Product Name                          License string Version
  invs_hier     INVS40    Innovus Hierarchical Design Option    Innovus_Hier_Opt 15.1
  invs_ms       INVS30    Innovus Mixed Signal Option           Innovus_MS_Opt 15.1
  invs_3d_ic    INVS60    Innovus 3D-IC Option                  Innovus_3D_IC_Opt 15.1
  invs_20nm     INVS20    Innovus 20nm Option                   Innovus_20nm_Opt 15.1
  invs_10nm     INVS10    Innovus 10nm Option                   Innovus_10nm_Opt 15.1
  ..........................................................................
  ..........................................................................
  ```
- The following command specifies that licenses for Tempus-L and Tempus-XL may be checked out. Tempus-L licenses are checked out before Tempus-XL licenses.
  ```
  setLicenseCheck -optionList "tpsl tpsxl"
  ```
- The following command sets the wait time to 5 minutes:
  ```
  setLicenseCheck -wait 5
  ```

Related Topics

- Product and Licensing Information chapter in the *Innovus User Guide*
- Getting Started chapter in the *Innovus User Guide*

setLimitedAccessFeature

```
setLimitedAccessFeature
[-help]
variableName
1 | 0
```

Enables or disables a limited access feature.

The Innovus includes certain limited access features that showcase advanced technology integrated in the software. These features have been internally qualified at Cadence but have had only limited customer testing.

> Limited access features are enabled by a variable specified through the `setLimitedAccessFeature` command. To use these limited access features, contact your Cadence representative to qualify your usage and to make sure it meets your needs before deploying it widely.

Parameters

`-help`	Prints a brief description that includes type and default information for each `setLimitedAccessFeature` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setLimitedAccessFeature`
variableName	Specifies the name of the variable that enables the limited access feature. *Data_type*: string, required
1 | 0	Specifying a value of `1` enables the feature. Specifying a value of `0` disables the feature. *Data_type*: bool, required

setMessageLimit

```
setMessageLimit
[-help]
numberToLimit
[prefix [ID1, ID2, ...]]
```

Limits error and warning messages to a specified number per unique ID. Any occurrences beyond the limit are suppressed. The default limit is 20 per message ID. The default limit is applicable only for `ERROR*` and `WARNING*` messages; there is no default limit for `verify*`, `check*`, and `INFO*` messages.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setMessageLimit` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setMessageLimit`.
`numberToLimit`	Limits error and warning messages to a specified number per unique ID. Any occurrences beyond the limit are suppressed. The default limit is 20 per message ID. The default limit is applicable only for ERROR* and WARNING* messages; there is no default limit for verify*, check* and INFO* messages. *Default*: 20 *Data_type*: int, required
`prefix [ID1, ID2, ...]`	Specifies the number per unique ID of specified prefix. *Data_type*: string, optional

Example

- The following command sets a limit of 10 messages per unique ID for all error and warning messages:
  ```
  setMessageLimit 10
  ```
- The following command sets a limit of 2 for the TCLCMD-917 error message.
  ```
  setMessageLimit 2 TCLCMD 917
  ```

After 2 occurences of TCLCMD-917, which is the same as the limit set, the system announces that the limit has been reached.

```
[DEV]innovus 4> set_propagated_clock CLK1

**ERROR: (TCLCMD-917): Cannot find 'ports, pins, or clocks' that match 'CLK1'

[DEV]innovus 5> set_propagated_clock CLK1

**ERROR: (TCLCMD-917): Cannot find 'ports, pins, or clocks' that match 'CLK1'

[DEV]innovus 6> set_propagated_clock CLK1

Message <TCLCMD-917> has exceeded the message display limit of '2'.
setMessageLimit/set_message_limit sets the limit.
unsetMessageLimit/unset_message_limit can be used to reset this.
```

Related Topics

- `set_message` command
- `get_message` command

setPreference

```
setPreference
[-help]
preference_name
value
```

Defines preferences, such as, commands issued during a session can be logged to a log or the screen.

When writing to a log and screen, the commands are preceded by <CMD>.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setPreference` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setPreference`.
`preference_name`	The name of the preference variable you want to set. To see a list of preferences, select ***View - Set Preference*** from the menu, then click *Save*. In the Save Preferences form, select the required save option and click *OK*. If you select the default *Save to Home Directory* option, the list of preferences and their values are saved in `.enc` file in your home directory. *Data_type*: string, required
`value`	The value for the preference. For example, the `CmdLogMode` preference has the following values. `0`--write to command only `1`--write to command and log (default) `2`--write to command, log, and screen *Data_type*: string, required

Preference Variables

The following table lists the preference variables you can set using the `setPreference` command and their GUI equivalent in the Preferences form:

Note: The following table might not be readable when viewed with the Unix man command. Check the `setPreference` command description in the *Innovus Text Command Reference* to view the table.

Variable Name	GUI Equivalent in Preferences Form	
	Tab Name	Option Name
`DesignName`	***Design***	*Design Name*
`DesignHierChar`		*Design Hierarchical Character*
`DEFHierChar`		*DEF Hierarchical Character*
`PDEFBusDelim`		*PDEF Bus Delimiter*
`CmdLogMode`		*Command Log Mode* ◦ *Command Only* ◦ *Command and Log* ◦ *Command, Log, and Screen*
`LogTypeInCmd`		*Log Type-In Command*
`logviewer`		*Invoke viewLog at Start Up*
`MouseGestures`		*Enable Mouse Gestures*

ShowFPObjInPlace	***Display***	*Show Floorplan Objects in Physical View*
ShowAllFence		*Show All Constraint*
DisplayPtnPin		*Display Partition Pin*
ShowAttrPopup		*Show Attributes Popup*
ShowUnplacedInst		*Display Unplaced Instances*
ShowRectilinearPad		*Show Rectilinear Pad Cell*
DisplayCellPin		*Display Instance Pin*
ShowLefLayerName		*Show Lef Layer Name*
AutoRedraw		*Auto-redraw after Layer Change*
EnlargeLogicalPin		*Enlarge Logical Pin In Fit View*
InstDisplayThreshold		*Instance Display Threshold*
WireDisplayThreshold		*Wire Display Threshold*
WireDisplayCheck2D		*Apply Threshold in 2D*
DrawInstanceByStatus		*Draw Instance by Placement Status*
DrawWireByStatus		*Draw Wire by Wire Status*
percentageOfPan		*Percentage of Window to Pan*
scaleOfZoom		*Scale of Zoom In/Out*
zoomPrevCount		*Number of Previous Zoom Saved*
MinFPModuleSize		*Min. Floorplan Module Size*
ShowParentModule		*Show Parent Module With Level*
ShowChildPartition		*Show Child Partition With Level*
ShowParentPartition		*Show Parent Partition With Level*

`AutoDetailDisplay`		*Automatic Detail Display for Large Design* ◦ Special Wire ◦ Regular Wire
`DetailDisplayFactor`		*Factor*
`DoubleClickTime`		*Double Click Time*
`DisplayRelFPlan`	***Edit***	*Honor Relative Floorplan Constraints*
`SingleLayerMode`		*Single Layer Mode For Routing Blockage, Ptn Pin Blockage and Ptn Feed*
`ShowDelBox`		*Show Delete Confirmation Box*
`StrecthRestriction`		*Box Stretch Restrictions* ◦ *No Restriction* ◦ *Maintain Area* ◦ *Maintain Aspect Ratio*
`StretchRectilinear`		*Rectilinear Stretch Restrictions* ◦ *No Restriction* ◦ *Maintain Area*
`MoveRestriction`		*Move Restrictions* ◦ *No Restrictions* ◦ *Orthogonal*
`clone_row_orientation_snapping`		*Clones snapping to same master row orientation*

SelectByArea	***Selection***	*Select by Area* ◦ *Enclosed by Box* ◦ *Intersecting Box*
SelectByLine		*Select by Line*
SelectStickyMode		*Sticky Mode*
SelectNetWhenSelectPin		*Hilite Net When Selecting a Pin*
HiliteNetWhenQueryObj		*Hilite Nets When Querying an Instance*
HiliteShapeWhenSelectNet		*Hilite Pin Shapes When Selecting a Net*
QueryWireNet		*Query Nets When Wires are Selected*
QuerySkipInst		*Skip Objects by Auto Query* ◦ *Cell*
QuerySkipInstObs		◦ *Obs in Cell*
QuerySkipInstPin		◦ *Pin in Cell*
QuerySkipRegular		◦ *Regular Wire*
QuerySkipSpecial		◦ *Special Wire*
WinSelectMargin		*Window Select Margin*
HighlightColorNumber		*Number of Highlight Color*
MinFlightLineWidth	***Flightline***	*Minimum Flightline Width*
MaxFlightLineNetTerms		*Maximum Flightline Net Terms*
DisplayPlaceFlightLine		*Show Flightline In Physical View*
ShowNumberBlockConnection		*Show Number of Connections For Block*
FlightLineInMove		*Draw Flightline When Moving Instance(s)*
ShowConnectionInOutNumber		*Show Input/Output Connection Number*

ShowFlightLineTermMark	*Show Pin's Input/Output Mark*
ShowBothInputConnection	*Show Both Input Pins Connection*
ShowNetWeightConnection	*Show Connection With Netweight 0*
ShowFlightLineThruPtnPin	*Show Flightlines Through Partition Pins*
noInsideMacro	*Don't Show Flightline to Inside Macro*
SkipBufferFlightline	*Skip Buffer/Invert Cell*
NoClockFlightline	*Don't Show Clock Net*
OnlyClockFlightline	*Show Clock Net Only*
DrawFlightlineLast	*Draw Flightline on Top of Selection*
OnlyBundleNetFlight	*Only Show Net Bundle Flight Line*
ExclusiveFlight	*Show Connection Only Between Selected Objects*
ConnectionColorType	*Single Color* *Different Color by Type of Connections* *Different Color by Number of Connections*
InputConnectionColor	• *Different Color by Type of Connections: Input* • *Different Color by Number of Connections: Small*
OutputConnectionColor	• *Different Color by Type of Connections: Output* • *Different Color by Number of Connections: Medium*

`InoutConnectionColor`	• *Different Color by Type of Connections: Inout* • *Different Color by Number of Connections: Large*
`MixtureConnectionColor`	• *Single Color: <color>* • *Different Color by Type of Connections: Inout*
`ShowConnectionWithWidth`	*Different Width by Number of Connections*
`FLWidthThresholdLow`	*Number of Connections* ◦ *Small* - Fewer than specified low threshold
`FLWidthThresholdHigh`	◦ *Medium* - Equal to or between than specified low and high thresholds ◦ *Large* - Greater than specified high threshold
`NoInstFlightLine`	*Don't Show Connection to Unselected Object* ◦ *Instance*
`NoBlockFlightLine`	◦ *Block*
`NoModuleFlightLine`	◦ *Module*
`NoBlackBlobFlightLine`	◦ *Black Blob*
`NoBlackBoxFlightLine`	◦ *Black Box*

InstanceText	***Text***	*Instance Text* ◦ *Instance Name* ◦ *Master Name*
ShowNetFullName		*Show Full Net Name*
ShowNetNameWithLayerColor		*Show Net Name With Layer Color*
ShowModuleText		*Object Text Display* ◦ *Module*
ShowAmoebaModuleText		◦ *Module in Amoeba*
ShowRowSiteText		◦ *Row Site*
ShowIoPadText		◦ *IO Pad*
ShowInstanceText		◦ *Instance*
ShowInstancePinText		◦ *Instance Pin*
ShowBlackBlobText		◦ *Black Blob*
ShowIoPinText		◦ *IO Pin*
ShowGroupText		◦ *Group*
ShowBumpText		◦ *Bump*
ShowClockTreeText		◦ *Clock Tree*
ShowChannelText		◦ *Channel*
ShowLefPortNumText		◦ *Lef Port Num*

ShowMacroSitePtnText		◦ *Macro Site Ptn*
ShowSIPFingerText		◦ *SIP Finger*
ShowNetText		◦ *Net*
TextDisplaySize		*Text Display Size* ◦ Small ◦ Medium ◦ Large ◦ Auto
TermCrossPix		*Term Cross Symbol Size*

Note: Options on the ***Windows*** tab of the Preferences form map to the `setWindowPreference` command instead of the `setPreference` command.

The following table lists the preference variables you can set using the `setPreference` command and their type, range/default/sample value.

Note: The following table might not be readable when viewed with the Unix man command. Check the `setPreference` command description in the *Innovus Text Command Reference* to view the table.

Variable Name	Type	Range/Default/Sample Value (Default values in bold)
DesignName	String	/spc/spc4/FE_tests_7.1/designs.. ../dtmf_pso/dtmf_fpd.enc.dat
DesignHierChar	String	**/**
DEFHierChar	String	**/**
PDEFBusDelim		**[]**
LecDofile	Boolean	**0 - False** 1 - True

CmdLogMode	Enum	*0 – Command Only* **1 – *Command and Log*** *2 – Command, Log, and Screen*
LogTypeInCmd	Boolean	**0 – False** 1 – True
logviewer	Boolean	**0 – False** 1 – True
MouseGestures	Boolean	0 – False **1 – True**
ShowFPObjInPlace	Boolean	**0 – False** 1 – True
ShowAllFence	Boolean	**0 – False** 1 – True
DisplayPtnPin	Boolean	0 – False **1 – True**
ShowAttrPopup	Boolean	0 – False **1 – True**
ShowUnplacedInst	Boolean	**0 – False** 1 – True
ShowRectilinearPad	Boolean	**0 – False** 1 – True
SpeedLevel	Enum	0 – Full detail 4 – Full speed
DisplayCellPin	Boolean	**0 – False** 1 – True

ShowLefLayerName	Boolean	0 – False **1 – True**
AutoRedraw	Boolean	0 – False **1 – True**
EnlargeLogicalPin	Boolean	**0 – False** 1 – True
InstDisplayThreshold	Real	**0.0**
WireDisplayThreshold	Integer	**0**
WireDisplayCheck2D	Boolean	0 – False **1 – True**
percentageOfPan	Enum	10, 20, 30, 40, 50, 60, 70, 80, 90, **95**, 100
scaleOfZoom	Real	**2.0**
zoomPrevCount		**1** to 6
MinFPModuleSize	Integer	**100**
ShowParentModule	Integer	**0**
ShowChildPartition	Integer	**0**
ShowParentPartition	Integer	**0**
AutoDetailDisplay	Boolean	**0 – False** 1 – True
DetailDisplayFactor	Integer	**16**
DisplayRelFPlan	Boolean	0 – False **1 – True**
SingleLayerMode	Boolean	**0 – False** 1 – True

ShowDelBox	Boolean	**0 – False** 1 – True
StrecthRestriction	Enum	**0 – No Restriction** 1 – Maintain Area 2 – Maintain Aspect Ratio
StretchRectilinear		**0 – No Restriction** 1 – Maintain Area
MoveRestriction		**0 – No Restrictions** 1 – Orthogonal
clone_row_orientation_snapping	Boolean	**0 – False** 1 – True
SelectByArea	Boolean	**0 – Enclosed by Box** 1 – Intersecting Box
SelectByLine	Boolean	**0 – False** 1 – True
SelectStickyMode	Boolean	**0 – False** 1 – True
SelectNetWhenSelectPin	Boolean	**0 – False** 1 – True
HiliteNetWhenQueryObj	Boolean	0 – False **1 – True**
HiliteShapeWhenSelectNet	Boolean	**0 – False** 1 – True
QueryWireNet	Boolean	**0 – False** 1 – True

QuerySkipInst	Boolean	**0 – False** 1 – True
QuerySkipInstObs	Boolean	**0 – False** 1 – True
QuerySkipInstPin	Boolean	**0 – False** 1 – True
QuerySkipRegular	Boolean	**0 – False** 1 – True
QuerySkipSpecial	Boolean	**0 – False** 1 – True
WinSelectMargin	Integer	**8**
HighlightColorNumber	Enum	**64**, 128, 256
MinFlightLineWidth	Integer	**1**
MaxFlightLineNetTerms	Integer	**500**
DisplayPlaceFlightLine	Boolean	0 – False **1 – True**
ShowNumberBlockConnection	Boolean	**0 – False** 1 – True
FlightLineInMove	Boolean	0 – False **1 – True**
ShowConnectionInOutNumber	Boolean	**0 – False** 1 – True
ShowFlightLineTermMark	Boolean	**0 – False** 1 – True

ShowBothInputConnection	Boolean	**0 – False** 1 – True
ShowNetWeightConnection	Boolean	0 – False **1 – True**
ShowFlightLineThruPtnPin	Boolean	**0 – False** 1 – True
noInsideMacro	Boolean	**0 – False** 1 – True
SkipBufferFlightline	Boolean	**0 – False** 1 – True
NoClockFlightline	Boolean	**0 – False** 1 – True
OnlyClockFlightline	Boolean	**0 – False** 1 – True
DrawFlightlineLast	Boolean	**0 – False** 1 – True
OnlyBundleNetFlight	Boolean	**0 – False** 1 – True
ExclusiveFlight	Boolean	0 – False **1 – True**
ConnectionColorType	Enum	**0 – Single Color** 1 – Different Color by Type of Connections 2 – Different Color by Number of Connections
InputConnectionColor		**green**

OutputConnectionColor		**yellow**
InoutConnectionColor		**pink**
MixtureConnectionColor		**blue**
ShowConnectionWithWidth	Boolean	**0 – False** 1 – True
FLWidthThresholdLow	Integer	**20**
FLWidthThresholdHigh	Integer	**80**
NoInstFlightLine	Boolean	**0 – False** 1 – True
NoBlockFlightLine	Boolean	**0 – False** 1 – True
NoModuleFlightLine	Boolean	**0 – False** 1 – True
NoBlackBlobFlightLine	Boolean	**0 – False** 1 – True
NoBlackBoxFlightLine	Boolean	**0 – False** 1 – True
InstanceText		**Instance** Master
ShowNetFullName	Boolean	**0 – False** 1 – True
ShowNetNameWithLayerColor	Boolean	**0 – False** 1 – True
ShowModuleText	Boolean	0 – False **1 – True**

ShowAmoebaModuleText	Boolean	0 – False **1 – True**
ShowRowSiteText	Boolean	0 – False **1 – True**
ShowIoPadText	Boolean	0 – False **1 – True**
ShowInstanceText	Boolean	0 – False **1 – True**
ShowInstancePinText	Boolean	0 – False **1 – True**
ShowBlackBlobText	Boolean	0 – False **1 – True**
ShowIoPinText	Boolean	0 – False **1 – True**
ShowGroupText	Boolean	0 – False **1 – True**
ShowBumpText	Boolean	0 – False **1 – True**
ShowClockTreeText	Boolean	0 – False **1 – True**
ShowChannelText	Boolean	0 – False **1 – True**
ShowLefPortNumText	Boolean	0 – False **1 – True**

ShowMacroSitePtnText	Boolean	0 – False **1 – True**
ShowSIPFingerText	Boolean	0 – False **1 – True**
ShowNetText	Boolean	0 – False **1 – True**
TextDisplaySize	Enum	**s**, m, l, a
TermCrossPix	Enum	0, **4**, 6, 8

Examples

- The following command writes log to command, log, and screen:
  ```
  setPreference CmdLogMode 2
  ```
- The following command sets the time between clicks in a double-click operation to 500 milliseconds:
  ```
  setPreference DoubleClickTime 500
  ```

Related Topics

- getPreference
- For more information on setting preferences from the GUI, see Set Preference form in the View Menu chapter in the *Innovus Menu Reference*.

setTopCell

`setTopCell` *`cellName`*

Switches the design to the partition specified by the *`cellName`*.

Note: Cadence recommends that you use the same name for the top cell and the partition.

You can use this command only after you commit the partition.

Parameters

`cellName`	The name of the top cell that indicates a different partition. *Data_type*: string, required

Related Topics

- Hierarchical Flow Capabilities chapter in the Innovus User Guide
  - Partitioning the Design - Restoring the Top-Level Floorplan with Partition Data

setWindowPreference

```
setWindowPreference
[-help]
window_name
{on | off}
```

Enables or disables windows (panels) from the Innovus main display. You can save these window settings in the .enc file in your home directory. The settings are automatically loaded when invoking the Innovus main window.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setWindowPreference` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setWindowPreference`.
`window_name`	Specifies the name of the window you want to turn on or off in the Innovus main display. You can select from the following: • `properties` -- Controls the Attribute Viewer panel. • `status` -- Controls the message bar display, which shows object identifier or coordinate information. • `toolbox` -- Controls the tool widget display. • `viewColor` -- Controls the visibility, selectability, and color display of various attributes. • `world` -- Controls the satellite window display. *Data_type*: string, required
`{on | off}`	Specifies whether window shoud be turned on or off. *Data_type*: string, required

Example

The following command turns on the tool widget display:

```
setWindowPreference toolbox on
```

source

```
source
[-help]
filename
[-quiet  | -verbose ] [-quiet  | -echo ]
```

Reads a file and executes the commands in the file (scripts). You can source an encrypted file created by the `encrypt` command. To be sourced, the file must have suffix “p” added to the file name, for example, test.tcl being encrypted as test.tclp.

Parameters

`-help`	Describes the command usage.
filename	Specifies the name of the file to source. *Data_type*: string, required
`-quiet`	Disable verbose source. *Data_type*: bool, optional
`-verbose`	Enable verbose source. *Data_type*: bool, optional
`-echo`	Enable verbose source. *Data_type*: bool, optional

With no `-quiet, -verbose, -echo` argument, file is sourced according to the `enc_source_verbose` global.

Note that the setting of `-quiet, -verbose, -echo` argument only affects the verbosity of sourcing the outermost file. If that file contains nested `source` commands, the verbosity of those nested commands is controlled by their associated `[-quiet | -verbose | -echo]` arguments or the `enc_source_verbose` global if no arguments.

Related Topics

- `enc_source_verbose`
- `enc_source_continue_on_error`
- `enc_source_echo_filename`

summaryReport

```
summaryReport
[-help]
[-outdir directoryName]
[-noText | -noHtml]
[-outfile fileName | -noText]
[-noHtml | -browser] [-exclude_base_cell_for_density cell_name_list | -
exclude_inst_prefix list_of_prefixes]
```

Reports statistics for the entire design, or a selected object in the design. A report on the entire design includes statistics for the following categories:

- General design information
- General library information
- Netlist information
- Timing information
- Floorplan/Placement information

To display information about a selected object in the design, you must select the object first before using `summaryReport`. The information contained in the report varies depending on the object you select. You can select a module, an instance, or a net.

You can use this command at any time after importing a design.

Parameters

`-browser`	Opens a browser and displays the HTML version of the report. *Data_type*: bool, optional
`-exclude_base_cell_for_density` *cell_name_list*	Specifies the list of cells to be excluded from the density report. You can use the wildcard, *, with this parameter. *Data_type*: (libCell/ptnCell)+, optional
`-exclude_inst_prefix` *list_of_prefixes*	Specifies the list of prefixes to map all instance names to be excluded from the density report. *Data_type*: string, optional

`-noHtml`	Generates only a text version of the report. **Note:** If you specify `-noHtml`, you cannot specify `-noText`. *Default:* The report is generated in both HTML and text format. *Data_type*: bool, optional
`-noText`	Generates only an HTML version of the report. Some statistics in the sections on the main HTML report page include links to more detailed HTML report pages. For example, if you click on the *# Layers* link under the *General Library Information* section of the HTML report, the software displays a detailed HTML report on the layers in the design. **Note:** If you specify `-noText`, you cannot specify `-noHtml`. *Default:* The report is generated in both HTML and text format. *Data_type*: bool, optional
`-outdir` *directoryName*	Specifies the directory in which the HTML and text reports are saved. If `-noText` is specified, saves only the HTML version of the report to the specified directory. The `-outdir` parameter cannot be used with `-noHtml`. *Default:* `summaryReport` *Data_type*: string, optional
`-outfile` *fileName*	Writes the text version of the report to the specified file. **Note:** You can specify this option only if you specify `-noHtml`. *Default:* If you specify `-noHtml` and do not specify `-outFile`, the software writes the text version of the report to a file named *design_name*`.main.htm.ascii` in the `summaryReport` folder. *Data_type*: string, optional

Examples

Examples

- The following command reports statics for the entire design and generates a text version of the results called `summaryReport.rpt`:

  ```
  summaryReport -noHtml -outfile summaryReport.rpt
  ```

  The following is a sample of the resulting text report:

  ```
  ==============================

  General Design Information

  ==============================

  Design Status: Routed

  Design Name: dtmf_chip

  # Instances: 7780

  # Hard Macros:

  ------------------------------

  Macro Cells in Netlist

  ------------------------------

  Macro Name Instance Count Area (um^2) Area Percentage in Core

  ram_256x16A 1 113204.162 26.353%

  rom_512x16A 1 68789.789 16.014%

  pllclk 1 41054.915 9.557%
  ```

```
ram_128x16A 1 100352.611 23.361%

# Std Cells:

------------------------------

Standard Cells in Netlist

------------------------------

Cell Type Instance Count Area (um^2)

XOR2X4 2 113.0976

XOR2X2 7 232.8480

XOR2X1 305 8116.4160

XNOR2X4 8 452.3904

XNOR2X2 8 292.7232

XNOR2X1 146 3885.2352

TLATNX1 1 36.5904

SEDFFX4 8 931.3920

SEDFFX1 124 11136.7872

SDFFSHQXL 1 79.8336

SDFFSHQX4 7 745.1136

.
```

```
.
.

# Pads:

------------------------------

IO Cells in Netlist

------------------------------

I/O Name Instance Count

PCORNERDG 4

PDIDGZ 26

PDO04CDG 27

PVDD1DGZ 5

PVSS1DGZ 5

# Net: 8641

# Special Net: 4

# IO Pins:

------------------------------

Issued IO Information

------------------------------
```

```
# Unplaced IO Pin 0

# Floating IO 0

# IO Connected to Non-IO Inst 0 53

# Pins:

------------------------------

Correctness of Pin Connectivity for All Instances

------------------------------

# Floating Terms 0

# Output Term Marked Tie Hi/Lo 0

# Output Term Shorted to PG Net 0 26897

.
.
.
```

- The following command also reports statics for the entire design, but creates both a text version and an HTML version of the results. Both versions of the report are saved in the `myReport` directory. The text version is named `dtmf_chip.main.htm.ascii` whereas the HTML report is saved as *`design_name`*`.main.htm`.

  ```
  summaryReport -outdir myReport
  ```

Related Topics

- For an example of an HTML report, see "HTML Summary Report" in the File Menu chapter of the *Innovus Menu Reference*.

suppressMessage

```
suppressMessage
[-help]
prefix numId [numId2 … numIdn]
```

Suppresses error or warning messages identified by a unique ID.

Note: This command suppresses the specified message regardless of the setting specified by the `setMessageLimit` command. For example, consider the case where you have used the `setMessageLimit` command to specify that a message should be suppressed after a fixed number of occurrences. If you now run the `suppressMessage` command on this message, the message will be suppressed, irrespective of the number of times it occurs.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `supressMessage` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man suppressMessage`.
`prefix` *`numId`* `[`*`numId2 … numIdn`*`]`	Specifies the prefix for the error or warning message. *`numId`: Specifies the unique numerical ID of the message to suppress.* *`numId2 … numIdn`: Specifies additional numerical IDs for the same prefix.* *Data_type*: string, optional

Examples

- The following command suppresses the warning messages with the message ID SI-2190:
  ```
  suppressMessage SI 2190
  ```
- This example suppresses *SOCLF-200* warning messages during design import:
  ```
  suppressMessage SOCLF 200
  ```
- This example suppresses SOCLF-80 and SOCLF-200 warning messages during design import:
  ```
  suppressMessage SOCLF 80 200
  ```
- Alternatively, you can explicitly name each message. This example also suppresses *SOCLF-80 and SOCLF-200* warning messages during design import:
  ```
  suppressMessage SOCLF-80 SOCLF-200
  ```
- This example suppresses *SOCLF-80, SOCLF-200, SOCPTN-160, and SOCPTN-400* warning messages:
  ```
  suppressMessage SOCLF 80 200 SOCPTN 160 400
  ```
- Alternatively, you can explicitly name each message. This example also suppresses SOCLF-80, SOCLF-200, SOCPTN-160, and SOCPTN-400 warning messages.
  ```
  suppressMessage SOCLF-80 SOCLF-200 SOCPTN-160 SOCPTN-400
  ```
- This example suppresses all messages from the *SOCLF* catalog during design import:
  ```
  suppressMessage SOCLF
  ```
- Multiple parameters can be used in combination on a single line. This example suppresses all *SOCLF* messages and *SOCECO-90* messages.
  ```
  suppressMessage SOCLF SOCECO-90
  ```
- The following command will suppress the SOCLF-8 message.
  ```
  suppressMessage SOCLF 8*
  ```
  * (asterisk) is not allowed in the suppressMessage command.

Related Topics

- setMessageLimit

suspend

```
suspend [-help]
```

Suspends your script and returns to the Innovus prompt.

The `suspend` command gives you more flexibility in managing and debugging your scripts. You can insert multiple `suspend` commands in a script. The `suspend` command can also be used in nested scripts. When the script reaches a point where you have inserted the `suspend` command, it stops and restores access to the Innovus prompt. You can then type any command required for debugging.

Whenever you want to resume your script, just type `resume` at the Innovus prompt.

Parameters

`-help`	Prints the usage of the `suspend` command. For a detailed description of the command, use the `man` command: `man suspend`.

Related Topics

- `resume`
- "Interrupting the Software" section of the Getting Started chapter in the *Innovus User Guide*

time_info

```
time_info
[-help]
[stage_name]
[-current]
[-report]
[-stamp]
[-table metric_table_id]
```

Reports and tracks runtime and memory performance.

When the `time_info` command is used without any name, an entry for the stage is created in the table named as "undefined". When the command is used multiple times without any stage name, the stage name is incremented, for example undefined1, undefined2, undefined3. This will ensure that each stage name in the table is unique.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `time_info` parameter. For a detailed description of the command and all of its parameters, use the man command: `man time_info`.
stage_name	Specifies the name of the stage.
`-current`	Reports the current stage.
`-report`	Reports all stages up to this point
`-stamp`	Creates a time stamp.
`-table` *metric_table_id*	Specifies metrics table ID to support multiple tables. *Default*: 'default'

Examples

- The following command shows the usage of the `-current` parameter:

```
[DEV]innovus 4> time_info -current
=====================================================
CPU Runtime: 0:00:52.5
Wall Runtime: 0:03:41
Memory Usage: 1387.4 MBytes
Peak Memory Usage: 1505.3 MBytes
Current Time: 12:17:13 (Jul07)
=====================================================
```

- The following command shows the usage of -report:

```
@[DEV]innovus 6> time_info -report
Total Wall | Stage Wall | Wall % | Date - Time     | Memory   | Stage
---------------+---------------+-----------+------------------+-----------+--
--------------------
0:00:00    | 0:00:00    | 0.0%   | 8:27:08 (Jul07) | 824.3 MB | init
---------------+---------------+-----------+------------------+-----------+--
--------------------
Number of threads: 1 * 1 (id: default)
```

uiGetRecordObjByInfo

```
uiGetRecordObjByInfo
[-help]
[-layer layer_value]
[-name object_name]
-objType {viaInst sViaInst sWire wire pWire pBlkg}
-rect {x1 y1 x2 y2}
```

Returns the point of an object based on specific layer/name/area.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `uiGetRecordObjByInfo` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man uiGetRecordObjByInfo`.
`-layer layer_value`	Specifies the layer value or layer range. *Data_type*: string, optional
`-name object_name`	Specifies the object name of net name for wire/sWire/pWire/viaInst/sViaInst. *Data_type*: string, optional
`-objType {viaInst sViaInst sWire wire pWire pBlkg}`	Specifies the object type. *Data_type*: enum, required
`-rect {x1 y1 x2 y2}`	Specifies the coordinates of the object bounding-box. *Data_type*: box, required

Example

The following command returns the point of wire which area is `{365.83 989.66 384.59 989.94}`, layer value is `3`, net name is `DTMF_INST/TDSP_CORE_INST/MPY_32_INST/ab_b[6]`.

```
uiGetRecordObjByInfo -objType wire -rect {365.83 989.66 384.59 989.94} -layer 3 -name
{DTMF_INST/TDSP_CORE_INST/MPY_32_INST/ab_b[6]}
```

unsetMessageLimit

```
unsetMessageLimit
[-help]
[prefix [ID1, ID2, ...]]
```

Removes the default or user-specified limit on error and warning messages. All errors and warnings are output to log.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `unsetMessageLimit` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man unsetMessageLimit`.
prefix [*ID1*, *ID2*...]	Removes the default or user-specified limit on error and warning messages with prefix. If no prefix is specified, the command removes the default or user-specified limit on error and warning messages. All errors and warnings are output to log. *Data_type*: string, optional

Example

The following command removes the limit on the number of error or warning messages to display:

```
unsetMessageLimit
```

Related Topics

- `setMessageLimit`

unsuppressMessage

```
unsuppressMessage
prefix numId [numId2 … numIdn]
```

Reverses the effect of the `suppressMessage` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `supressMessage` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man suppressMessage`.
`prefix numId [numId2 … numIdn]`	Specifies the prefix for the error or warning message. *`numId`*: Specifies the unique numerical ID of the message to unsuppress. *`numId2 … numIdn`*: Specifies additional numerical IDs for the same alpha prefix. *Data_type*: string, required

Examples

- The following command unsuppresses the warning messages with the message ID `SI-2190`:

  ```
  unsuppressMessage SI 2190
  ```

- This example suppresses the occurrence of the messages *SOCECO-308* and *SOCECO-310* during *optDesign -preCts*. For the rest of the flow, *SOCECO-308* is displayed but *SOCECO-310* continues to be suppressed.

  ```
  suppressMessage SOCECO 308 310

  optDesign -preCts

  unsuppressMessage SOCECO-308
  ```

- Multiple parameters can be used in combination on a single line. This example unsuppresses all *SOCECO* messages and *SOCLF-80* messages.

  ```
  unsuppressMessage SOCLF 80 SOCECO
  ```

Related Topic

- suppressMessage

update_names

```
update_names
[-help]
[-allowed chars]
[-change_modules hinsts_modules [-local]]
[-design]
[-first_restricted chars]
[-honor_dont_touch]
[-hport]
[-inst]
[-last_restricted chars]
[-log_file string [-append_log]]
[-map {{from to}...}]
[-max_length integer]
[-module]
[-net]
[-no_verbose]
[-nocase]
[-phys_inst]
[-port]
[-prefix string [-name_collision]]
[-random [-random_map_dir out_dir]]
[-regexp]
[-replace_str string]
[-reserved_words string]
[-restricted string]
[-suffix string [-name_collision]]
[-system_verilog]
[-verilog]
[-vhdl]
```

Updates the inst, net, phys_insts, port, module, and design names at each level of hierarchy. The inst and net names are local to only one level of hierarchy, as seen by Verilog inside a module.

If a net or port name is a part of a bus definition, then only the base bus name is updated, and the bus-bits are not modified. So for example, a net named 'busA[0]' specifies that the base bus name is 'busA', which can be changed to 'busB[0]'. However, the '[0]' part will not be changed.

If a net or port name is an escaped scalar name, then all the characters can be modified. So in such a case, an escaped Verilog name like '\busA[0] ' can be changed to 'busA_0_' using various matching options available as shown in the Examples section below.

You can restrict the changes to just one name type using parameters, such as `-inst`, `-net`, `-port`, `-`

`module`, or `-design`. Otherwise, the requested change is applied to all the name types.

By default, there is no restriction on the length of the new names (see `-max_length`).

Each name is checked, and the characters or strings are removed, replaced, or added according to the parameters explained below. Then the final name is checked for collisions with any existing names. If the new name collides with an existing name, a suffix such as _1 and _2 is appended until it is unique.

The old and new names for each modified name are reported.

Note: Names used for SDC timing constraints can only be modified if they are loaded into memory. So you must make sure all the timing views are loaded with `set_analysis_view` before running this command.

Note: There is a limitation that names used inside a CPF `power_intent` file cannot be updated. CPF usage assumes that the CPF file is golden and should not be modified. If you need to change names used inside a CPF file, you need to update the CPF file manually and then read in the new CPF file after running `update_names`. In contrast, an IEEE1801 `power_intent` file is allowed to change during the design flow, so changing any pin, instance or net names with `update_names` will automatically update the IEEE 1801 names. You can write out the new IEEE1801 file with the updated names.

Parameters

`-allowed` *chars*	Specifies the characters that are allowed in the names. Any characters that are not in the allowed list will be removed in the final name. The minimum specification is 10 characters. To allow all the letters from a to z in capital and lower case letters, you must specify all of them. Use `-regexp` to allow dash "-" as a character range indicator. Type: string, optional
`-append_log`	Appends the report to `<out_file>` specified with the `-log_file` option. If you omit this option, `<out_file>` is overwritten. You can specify this option only if you have specified the `-log_file` option. Type: bool, optional

`-change_modules` *`hinsts_modules`*	Limits the name changes to the objects inside the specified list of modules or modules of the `hinsts`. The changes are applied recursively down the hierarchy below each module, unless the `-local` parameter is specified. The current design is used by default, if this parameter is not used. Type: `(hInst|vCell|topCell)+`, optional
`-design`	Changes only the design module name. Type: bool, optional
`-first_restricted` *`chars`*	Specifies the characters that cannot be used in the beginning of the name. Those will be removed, including repeated characters. So `-first_restricted {_#%$}` will change _$abc to just abc. Type: string, optional
`-help`	Prints the command usage.
`-honor_dont_touch`	Prevents renaming of the objects, for which the `dont_touch` attribute is set.
`-hport`	Changes the port names (e.g. Verilog input, output, or inout names) of the intermediate modules for all hinsts, including bus port names. It also modifies the same implicit net name connected to the port to match the new port name. It does not modify the port names of library cells or the top-level design. Type: bool, optional
`-inst`	Changes only the Verilog instance names at each level of hierarchy. Type: bool, optional
`-last_restricted` *`chars`*	Specifies the characters that cannot be used at the ending of a name. Those will be removed, including repeated chars. Type: string, optional

`-local`	Turns off the recursive descent, so that changes become local to the module or design. The objects defined in the design/module can be modified (for example, design/module, `port`/`hport`, `inst`, or `net` names), but sub-module names and `hport` names of sub-modules cannot be modified. Type: bool, optional
`-log_file` *out_file*	Specifies a log file for the output report that shows the old name and the new name for any name changes. Type: string, optional
`-map {{`*from to*`}...}`	Takes a list of string pairs and maps the first string to the second string in each pair. You need to enclose each pair in braces. To be compatible with Genus usage, double quotes are optionally allowed around the strings. For example, both `-map {{"from" "to"} {"AND" "and"}}` and `-map {{from to} {AND and}}` will result in the same output. Type: string, optional
`-max_length` *int*	Limits the length of the changed name to the specified number of characters. If the name exceeds the limit, the additional characters will be truncated from the end. If the new name is not unique, the original name is further trimmed back to allow the name with a suffix, such as 1, 2, etc. to make it unique and within the specified length limit. Note, the truncation happens after all other changes. This means the truncated names may now collide with -vhdl, -verilog, or -system_verilog reserved names, or end with a char from the -last_restricted set. If that happens, you will have to rerun the command with the same options and either a longer -max_length or no -max_length. Type: Integer, optional
`-module`	Changes all module names other than the design module name. Type: bool, optional

`-name_collision`	Adds only the `-prefix` or `-suffix` values, if a name change collides with an existing name. This parameter can be used only when either the `-prefix` or `-suffix` parameter is used.
`-net`	Changes the Verilog net names at each level of hierarchy (e.g. Verilog wire names). This will not modify net names that correspond to Verilog port definitions (e.g. Verilog input, output or inout names). If you want to modify those net names, use the -port option. Type: bool, optional
`-no_verbose`	Suppresses the output generated from `update_names`. Type: bool, optional
`-nocase`	Changes names that collide if the case is ignored. For example, if both net1 and Net1 exist, one of them will be changed to resolve the collision by appending a suffix, such as _1 or _2. Type: bool, optional
`-phys_inst`	Changes only those physical inst names (like filler cells) that do not appear normally in the Verilog output. When this parameter is specified, the module-scoping parameters (like `-change_modules`) are ignored, so all the physical inst names are checked. Type: bool, optional
`-port`	Changes the port names (e.g. Verilog input, output, or inout names) of the top-level module for the design, including bus port names. It also modifies the same implicit net name connected to the port to match the new port name. It does not modify the port names of library cells or intermediate modules. Type: bool, optional
`-prefix` *string*	Adds a prefix to the names that are changed.

`-random`	Makes all the modified names random. It obfuscates the design names so that the test cases can be shared without exposing the real netlist names. This parameter cannot be used with any other parameter that affects names. It can be used with `-inst`, `-net`, `-hport`, `-design`, and `-port` to limit the types of names to be modified.

`-random_map_dir` *`out_dir`*	Specifies the `out_dir` (output directory) for the map-file of the old and new random names to be written for each name type that is modified inside the output directory. This parameter requires the `-random` parameter. It is useful in the cases when there are side files that have names to be mapped and are not supported by `update_names`. The following files are written:
	`inst.map for inst and hinst names`
	`net.map for net and hnet names`
	`pin.map for hport and pin names`
	`port.map for top-level port names`
	`module.map for module names`
	The format is *`old_name new_name`* on each line for every name. For example, `inst.map` may look like:
	`blockA/blockB/regC i1/i2/i3`
	`blockA/regD i1/i3`
	…

`-regexp`	Allows specifying character ranges with a dash "-" in the `-allowed` option. This is similar to regexp single character matching usage. For example, to allow all alphanumerical characters, along with _ and $ as legal chars, use: -allowed {a-zA-Z0-9_$} Type: bool, optional
`-replace_str` *`string`*	Specifies the replacement string for `-restricted` or `-reserved_words` strings. The default value for this option is "_". Type: string, optional

`-reserved_words` *string*	Specifies a list of words to be avoided (reserved words), like 'begin end'. If a complete name exactly matches one of the reserved words (not just a sub-string of the name), then by default, the name will be changed with the addition of an `_1`, `_2`, etc. suffix. If you also specify the `-replace_string` *rep_str* parameter, then the reserved words will be replaced by *rep_str* instead, and the `_1`, `_2`, etc. will be appended, if there are any collisions. Type: string, optional
`-restricted` *string*	Specifies a list of strings that cannot be used inside a name. Each string is replaced with the `-replace_str` string. For example, `-restricted {ab bc}` will replace all the `ab` and `bc` strings inside the names. Type: string, optional
`-suffix` *string*	Adds a suffix to the names that are changed. **Note:** If the length of `-suffix` exceeds the `-max_length` value, the suffix is truncated.
`-system_verilog`	Replaces SystemVerilog reserved words. If a complete name exactly matches one of the reserved words (not just a sub-string of the name), then the name will be changed. Type: bool, optional
`-verilog`	Replaces Verilog reserved words. If a complete name exactly matches with one of the Verilog reserved words (not only a sub-string of the name), then the name will be changed. Type: bool, optional

`-vhdl`	Fixes names to meet VHDL identifier requirements. It replaces VHDL reserved words. If a complete name exactly matches one of the VHDL reserved words (not only a sub-string of the name), then the name will be changed. It also implicitly uses –first_restricted “_0123456789” to remove any leading _ or digit chars, and –last_restricted “_” to remove any trailing _ chars, so you cannot use –vhdl together with –first_restricted or –last_restricted. Finally, this option will remove any repeated _ chars in the middle of any name, and replace it with a single _ char because VHDL does not allow repeated _ chars. Type: bool, optional

Examples

- The following command replaces the `p1` inst name with the `prefix_p1` prefix without updating all the `p1` sub-strings (for example, without changing `insts` named `p12` or `op1`):
  ```
  update_names -inst -reserved_words {p1} -replace_string {prefix_p1}
  ```
- The following example replaces sub-string "no" with "NO", and all brackets "[" and "]" with underscore "_" in all net names:
  ```
  innovus 1> update_names -net -map {{no NO} {[ _} {] _}}
  ```
  Optional double-quotes are ignored, so this has the same result:
  ```
  innovus 1> update_names -net -map {{"no" "NO"} {"[" "_"} {"]" "_"}}
  ```
  So, a Verilog net name that requires escaping like:
  `\test[0]nox` would change to `test_0_NOx`
  Note, neither the Verilog or DEF name requires escaping anymore, so no \ would be written to Verilog or DEF for these names. A “real” bus name like
  `no[0]` would just be changed to `NO[0]`
  because the `[0]` are bus-bit chars and values that are never changed.
- The following example replaces any sub-strings that match ab, bc, or ca with "@" in all object names:
  ```
  innovus 1> update_names -restricted "ab bc ca" -replace_str "@"
  ```
- The following command changes the brackets "[" and "]" in all names to "_". It does not change real bus-bit chars for nets.

```
innovus 1> update_names -restricted {[ ]} -replace_str "_"
```

Note: You should use {} to protect the [] from being interpreted by Tcl.

- The following example specifies the maximum length of all module names to be 12 characters.

  ```
  innovus 1> update_names -max_length 12 -module
  ```

- The following example fixes case-insensitive collisions. Suppose you have two insts named inst73 and INST73. This command renames one of the insts by appending _<num> to one of the names for avoiding a naming conflict, when casing is ignored.

  ```
  innovus 1> update_names -inst -nocase
  ```

- The following example allows all capital and lower case letters, numbers, underscores, backslash, and brackets. Any other character will be removed.

  ```
  innovus 1> update_names -allowed
  {ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789_\[]}
  ```

  If you want to use character ranges, you can use `-regexp`:

  ```
  innovus 1> update_names -regexp -allowed {a-zA-Z0-9_\[]}
  ```

- If you want to change names that collide with reserved Verilog words, then use the following command:

  ```
  innovus 1> update_names -verilog
  ```

  Names that exactly match a Verilog reserved words, like "input" will be replaced with "input_1". This does not replace sub-strings in names that match "input", so "my_input" would not be modified.

Related Topics

`rename_obj`

viewLog

```
viewLog
[-help]
[-file logFileName]
```

Opens up a log file in a separate console window for viewing within the Innovus.

The log file viewer within the Innovus opens the current log file by default, and updates it in real time. If you specify a log file name, the software opens up the specified log file.

The log file displayed contains `[+]` and `[-]` markers that expand and collapse command information, so that you can control how much detail you want to read. Log messages are color coded for easier identification: blue messages are warnings, red messages are errors.

You can access documentation for commands in the log file that are underlined. Click on the command in the file, and the software opens an HTML window displaying the *Innovus Text Command Reference* documentation for that command.

You also can access the log file viewer from the Tools Menu - *Log Viewer*.

Note: You can specify `viewLog` more than once to view multiple log files in separate console windows simultaneously. However, you cannot open multiple versions of the current log file.

Parameters

`-file` *`logFileName`*	Specifies the name of the log file to open. *Data_type*: string, optional
`-help`	Outputs a brief description that includes type and default information for each `viewLog` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man viewLog`.

Related Topic

- Tools Menu chapter in *Menu Reference*.

vPuts

```
vPuts
[-help]
[string]
[-nonewline]
```

Outputs information to the verbose log file (innovus.logv) and the standard output.

If the verbose log file is not available (invoked with `innovus -no_logv`), the `vPuts` command outputs only to the standard output.

Parameters

`-help`	Prints a brief description of the command. For a detailed description of the command, use the `man` command: `man vPuts`
`string`	Specifies the output string.
`-nonewline`	When you specify this option, the command does not output a newline character after the output string. *Data_type*: bool, optional

Related Topics

- `innovus`

win

```
win
[-help]
[on | off]
```

Opens the Innovus main window. This is usually done after executing the command `innovus -init` *`tclCommandFile`*.

If you run the `win` command when the Innovus main window is already open, the tool retains the current zoom level and displays the main window without refreshing it. For example:

```
innovus 23> zoomBox 94 -640 95 -645
innovus 24> win
```

This displays the main window with the design zoomed to the coordinates "`94 -640 95 -645`".

Note: If you started the Innovus software with the `innovus -nowin` command, you cannot then use the `win` command to display the Innovus main window.

Parameters

`-help`	Prints a brief description that includes type and default information for each `win` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man win`
`on | off`	Specifies whether to switch on or off the window. *Data_type*: string, optional

Related Topic

- `innovus`

writeFlowTemplate

```
writeFlowTemplate
-help
[-directory directory_name]
```

Copies the Innovus Foundation flows templates into the directory you specify, or to the current directory if you do not specify one with the `-directory` parameter.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `writeFlowTemplate` parameter. For a detailed description of the command and all of its parameters, use the man command: `man writeFlowTemplate`
`-directory` *`directory_name`*	Specifies the directory in which to save the templates. *Data_type*: string, optional

General Global Variables

- auto_file_dir
- auto_file_prefix
- enable_legacy_metric
- enc_partial_cmd_argument_matching
- enc_print_full_message_summary
- enc_source_continue_on_error
- enc_source_echo_filename
- enc_source_verbose
- enc_source_verbose_cmd_print_length_limit
- enc_source_verbose_output
- enc_tcl_error_info
- enc_tcl_return_display_limit
- encEnableMetric
- gui_verbose
- highFreqInterposerFlow
- load_netlist_ignore_undefined_cell
- metric_advanced_url_endpoint
- metric_capture_3d_hotspots
- metric_capture_depth
- metric_capture_design_image
- metric_capture_design_image_blockages
- metric_capture_design_image_blockages_threshold
- metric_capture_design_image_power_intent
- metric_capture_design_image_route_drc
- metric_capture_max_drc_markers

- metric_capture_min_count
- metric_capture_overwrite
- metric_capture_pba_tns_histogram
- metric_capture_per_view
- metric_capture_reg2reg_metrics
- metric_capture_timing_analysis_mode
- metric_capture_timing_path_groups
- metric_capture_timing_paths
- metric_capture_tns_histogram
- metric_capture_tns_histogram_buckets
- metric_capture_tns_histogram_max_slack
- metric_capture_tns_histogram_paths
- metric_capture_vth_per_power_domain
- metric_category_default
- metric_current_run_id
- metric_enable
- metric_summary_metrics
- restore_db_directory
- restore_db_file_check
- restore_db_stop_at_design_in_memory
- restore_db_tool
- script_search_path
- soft_stack_size_limit

auto_file_dir

`auto_file_dir` *directoryName*

Type: String

Default: ./

Specifies the directory to store files/sub-directories 'auto-generated' by the tool. This includes report files or other output files with default names that are automatically created by some commands in the current run directory. Default names that have been overridden with a name chosen by the user, are not affected. This allows multiple simultaneous runs from the same run directory to direct output files to different result directories to avoid collisions.

Related Topics

- `auto_file_prefix`

auto_file_prefix

`auto_file_prefix`

Type: String

Default: ""

Specifies a prefix to be applied to all files 'auto-generated' by the tool. This includes report files or other output files with default names that are automatically created by some commands in the current run directory. So a default name like "cmd.report" will become "<prefix>cmd.report". Default names that have been overridden with a name chosen by the user, are not affected. This allows multiple simultaneous runs from the same run directory to direct default output files to different locations to avoid collisions.

Related Topics

- `auto_file_dir`

enable_legacy_metric

```
enable_legacy_metric {true | false}
```

Type: Boolean

Default: `false`

Enables the legacy `get_metric` command. If set to `true`, the `get_metric` command used is old legacy.

Related Topics

- `get_metric`

enc_partial_cmd_argument_matching

enc_partial_cmd_argument_matching

Type: enum

Choices: `error`, `warn`, `quiet`

Default: `quiet`

Enables partial command argument matching.

- `error`: Does not allow partial argument matching. System will issue errors when command arguments are uniquely matched from a partial specification.
- `warn`: Allows partial argument matching. System will issue warnings when command arguments are uniquely matched from a partial specification.
- `quiet`: Allows partial argument matching without any warnings or errors.

enc_print_full_message_summary

```
enc_print_full_message_summary
```

Type: boolean

Default: `false`

Controls the format of the message summary. If `true`, prints full message, otherwise prints truncated message.

enc_source_continue_on_error

`enc_source_continue_on_error` *value*

Type: Boolean

Default: `false`

Enables or disables `source` command to continue the script on tcl error. When set to `true`, the `source` command will ignore tcl errors and continue processing the script files. When set to `false`, a tcl error will halt the script.

Example

In the following example, the Tcl File (test.tcl) has an error i.e. no option specified after hyphen (-) for the `setDesignMode` command in the script.

test.tcl content:

```
::::::::::::

puts "Testing Script for Errors"

setDesignMode -

puts "End Testing Script for errors"

::::::::::::
```

If `enc_source_continue_on_error` is set to `false`, and `setDesignMode` is specified with hyphen only (without the option name after the hyphen), the script will end and will not source other commands mentioned after `setDesignMode`.

```
innovus 3> set enc_source_continue_on_error false

false

innovus 4> source test.tcl

Testing Script for Errors

Usage: setDesignMode [-help] [-reset] [-addPhysicalCell {hier|flat}] [-congEffort
{low|medium|high|auto}] [-dual_rail_via_pitch <min_via_pitch> <min_filler_via_pitch>
<cell_boundary_spacing>] [-earlyClockFlow {true|false}] [-expressRoute {true|false}]
[-flowEffort {express|standard|extreme}] [-ignore_followpin_vias {true|false}] [-
pessimisticMode {true|false}] [-powerEffort {none|low|high}] [-process <integer>] [-
slackWeighting {unityWeighting|viewBasedWeighting}]
```

```
**ERROR: (IMPTCM-48):   ""-"" is not a legal option for command ""setDesignMode"".
Either the current option or an option prior to it is not specified correctly.
```

If `enc_source_continue_on_error` is set to `true`, and `setDesignMode` is specified with hyphen only (without the option name after the hyphen), the script will throw an error and continue to source the other commands without exiting.

```
innovus 5> set enc_source_continue_on_error true

true

innovus 6> source test.tcl

Testing Script for Errors

Usage: setDesignMode [-help] [-reset] [-addPhysicalCell {hier|flat}] [-congEffort
{low|medium|high|auto}] [-dual_rail_via_pitch <min_via_pitch> <min_filler_via_pitch>
<cell_boundary_spacing>] [-earlyClockFlow {true|false}] [-expressRoute {true|false}]
[-flowEffort {express|standard|extreme}] [-ignore_followpin_vias {true|false}] [-
pessimisticMode {true|false}] [-powerEffort {none|low|high}] [-process <integer>] [-
slackWeighting {unityWeighting|viewBasedWeighting}]

**ERROR: (IMPTCM-48):   ""-"" is not a legal option for command ""setDesignMode"".
Either the current option or an option prior to it is not specified correctly.

**ERROR: (IMPSE-110):   File 'test.tcl' line 2: errors out.

End Testing Script for errors
```

Related Topics

- source

enc_source_echo_filename

`enc_source_echo_filename` *value*

Type: *Boolean*

Default: `0`

When this setting is enabled, the log file will display the file name being sourced before the actually sourcing the file. By default, name of the file being sourced is not displayed in the log file.

Related Topics

- `source`

enc_source_verbose

`enc_source_verbose` *value*

Type: Integer

Default: `0`

Controls whether scripts are sourced in quiet or verbose mode:

- By default (`0`), files are sourced in quiet mode. This is equivalent to `source` *filename*`-quiet`.
- If you set this variable to `1`, files are sourced in verbose mode. This is equivalent to `source` *filename*`-verbose`.

Related Topics

- `source`

enc_source_verbose_cmd_print_length_limit

`enc_source_verbose_cmd_print_length_limit` *value*

Type: Integer

Default: `-1`

Limits the command printed to stdout and the .log file to the specified number of characters for verbose script source.

Related Commands

- `source`

enc_source_verbose_output

`enc_source_verbose_output`

Type: `enum`

Choices: `debug, echo, echo_comments`

Default: `echo`

Used to control the verbosity of the `source -verbose` output.

- If you specify `echo`, the output will only include the command lines.
- If you specify `debug`, the output will include file name, line number, and command. This option is used to debug problems in scripts.
- If you specify `echo_comments`, the output will include commands, comments and empty lines

Examples

- By default, source -verbose prints only the real command line start with #:

```
[DEV]innovus 3> source -verbose a.tcl
### Start verbose source output (echo mode) for 'a.tcl' ...
# set a b
### End verbose source output for 'a.tcl'.
```

- With `set enc_source_verbose_output echo_comments`, all file contents, including comment lines and empty lines, are printed. Comment lines start with # and command lines start with @:

```
[DEV]innovus 5> source -verbose a.tcl
### Start verbose source output (echo_comments mode) for 'a.tcl' ...

#comment line1\
  comment line2

#line 6 is empty line

@set a b
### End verbose source output for 'a.tcl'.
```

Related Topics

source

enc_tcl_error_info

```
enc_tcl_error_info {true | false}
```

Type: Boolean

Default: `false`

When set to `true`, `enc_tcl_error_info` prints the error information if there is an issue in the Tcl script.

enc_tcl_return_display_limit

```
enc_tcl_return_display_limit value
```

Type: Integer

Default: `10000`

You can use this variable to set the number of characters displayed on the xterm window (stdout) and the .log file. The Tcl return value itself is not truncated. A value of `-1` indicates that there is no display limit. The Tcl return value of the history command will not be truncated by setting this variable.

Example

- In the example below, the tcl return values of `getDesignMode` command options are not truncated. However, the remaining part is truncated to 3 characters.

```
innovus 1> set enc_tcl_return_display_limit 3
innovus 2> getDesignMode
-addPhysicalCell hier                  # enums={hier flat}, default=hier
-congEffort auto                       # enums={low medium high auto},
default=auto
-dual_rail_via_pitch {}                # string, default="""
-earlyClockFlow false                  # bool, default=false
-expressRoute false                    # bool, default=false
-flowEffort standard                   # enums={express standard extreme},
default=standard
-ignore_followpin_vias false           # bool, default=false
-pessimisticMode false                 # enums={true false}, default=false
-powerEffort none                      # enums={none low high}, default=none
-process 90                            # int, default=90, min=3, max=250
-slackWeighting unityWeighting         # enums={unityWeighting
viewBasedWeighting}, default=unityWeighting
{ad... truncated to 3 characters (use the enc_tcl_return_display_limit variable to
control the length before truncation).
```

- In the example below, the value of `enc_tcl_return_display_limit` is set to `-1`. As a result, there is no display limit.

```
innovus 3> set enc_tcl_return_display_limit -1
-1
innovus 4> getDesignMode
-addPhysicalCell hier # enums={hier flat}, default=hier
-congEffort auto # enums={low medium high auto}, default=auto
-dual_rail_via_pitch {} # string, default=""
-earlyClockFlow false # bool, default=false
-expressRoute false # bool, default=false
-flowEffort standard # enums={express standard extreme}, default=standard
-ignore_followpin_vias false # bool, default=false
-pessimisticMode false # enums={true false}, default=false
-powerEffort none # enums={none low high}, default=none
-process 90 # int, default=90, min=3, max=250
-slackWeighting unityWeighting # enums={unityWeighting viewBasedWeighting},
default=unityWeighting

{addPhysicalCell hier} {congEffort auto} {dual_rail_via_pitch {}} {earlyClockFlow
false} {expressRoute false} {flowEffort standard} {ignore_followpin_vias false}
{pessimisticMode false} {powerEffort none} {process 90} {slackWeighting
unityWeighting}
```

encEnableMetric

`encEnableMetric` *value*

Type: Integer

Default: `1000`

Controls design metrics display.

Related Topics

- `get_metric`

gui_verbose

```
gui_verbose {all db none}
```

Type: enum {all db none}, read/write

Default: `db`

Controls which GUI actions are logged. You can select:

- `all`: Logs all GUI actions.
- `db`: Logs the GUI actions that affect the DB, such as `gui_select`, but does not capture display-only actions such as `gui_dim_foreground`.
- `none`: Does not log any GUI actions. So, the .log and .cmd files are incomplete.

highFreqInterposerFlow

```
highFreqInterposerFlow {0 | 1}
```

Type: Boolean

Default: `0`

Specifies the flow as a high frequency interposer flow. When set to `1`, `setIntegRouteConstraint` will auto turn off the options which are not suitable for interposer routing.

Related Topics

- `setIntegRouteConstraint`

load_netlist_ignore_undefined_cell

```
load_netlist_ignore_undefined_cell { 1 | 0 }
```

If `false`, a Verilog reference to an undefined cell (for example, a Verilog module name that has no matching library cell and no Verilog module definition) causes an error and an exit after parsing the netlist. If `true`, an empty Verilog module is created with a warning. The default value is `true` in Innovus to allow repair of 'dirty netlists'. An empty module will become an hinst or inst depending on the value of `setImportMode -keepEmptyModule`.

To set this variable, use the `set` command.

Datatype: boolean

Default: `1`

metric_advanced_url_endpoint

`metric_advanced_url_endpoint` *value*

Default: The default value of this string variable is not set.

Provides pointer to advanced metric server URL to store the snapshot information.

metric_capture_3d_hotspots

```
metric_capture_3d_hotspots {true | false}
```

Type: Boolean

Default: `true`

Captures 3d hotspot metrics.

metric_capture_depth

`metric_capture_depth` *integer*

Default: `0`

Specifies the depth for capturing hinst design and power metrics.

metric_capture_design_image

`metric_capture_design_image` *`string`*

Default: `true`

Captures the design image.

metric_capture_design_image_blockages

```
metric_capture_design_image_blockages {true | false}
```

Default: `true`

Captures blockages in the design image.

metric_capture_design_image_blockages_threshold

`metric_capture_design_image_blockages_threshold` *`threshholdValue`*

Type: Integer

Default: `100`

Threshold for capturing blockages in metric image.

metric_capture_design_image_power_intent

```
metric_capture_design_image_power_intent {true | false}
```

Default: `true`

Captures power intent objects in the design image.

metric_capture_design_image_route_drc

```
metric_capture_design_image_route_drc {true | false}
```

Default: `true`

Captures route DRC markers in the design image.

metric_capture_max_drc_markers

`metric_capture_max_drc_markers` *`max_number`*

Default: `100000`

Specifies the maximum number of DRC markers to include in metric image.

metric_capture_min_count

`metric_capture_min_count` *integer*

Default: `1000`

Captures the minimum instance count for capturing hinst design and power metrics.

metric_capture_overwrite

```
metric_capture_overwrite {true | false}
```

Default: `false`

Overwrites pending metrics during the `createSnapshot` category capture.

metric_capture_pba_tns_histogram

```
metric_capture_pba_tns_histogram {true | false}
```

Default: `false`

Captures the PBA histogram data for TNS.

metric_capture_per_view

```
metric_capture_per_view {true | false}
```

Default: `true`

Captures timing metrics per analysis_view.

metric_capture_reg2reg_metrics

```
metric_capture_reg2reg_metrics [true | false}
```

Default: `false`

Forces the capture of reg2reg metrics regardless of path groups.

metric_capture_timing_analysis_mode

```
metric_capture_timing_analysis_mode {gba | ipba}
```

Type: Enum
Default: gba

Controls the timing analysis mode for metric capture during create_snapshot categories setup and hold; gba, ipba.

- `gba`: graph based analysis
- `ipba`: path based analysis, retime_method == path_slew_propagation, retime_mode == exhaustive, timing_path_based_exhaustive_pba_bounded_mode == true

metric_capture_timing_path_groups

```
metric_capture_timing_path_groups path_group
```

Default: The default value of this string variable is not set.

Controls the path groups for which detailed timing path information (full timing path report and display) is to be captured . By default, it is empty and the tool captures all and reg2reg. If the global is set to a list of groups, the tool will use the listed groups. The related metric name is `timing.setup.paths.path_group:<group>`.

metric_capture_timing_paths

```
metric_capture_timing_paths integer
```

Default: `10`

Specifies the number of paths to capture for detailed display.

metric_capture_tns_histogram

```
metric_capture_tns_histogram {true | false}
```

Default: `true`

Capture histogram data for TNS.

metric_capture_tns_histogram_buckets

`metric_capture_tns_histogram_buckets` *integer*

Default: `50`

Specifies the number of buckets for the TNS histogram.

metric_capture_tns_histogram_max_slack

`metric_capture_tns_histogram_max_slack` *`maximum_slack_value`*

Default: `0`

Specifies the maximum slack value for the TNS histogram. You can change its value if you need to shift the y axis.

metric_capture_tns_histogram_paths

`metric_capture_tns_histogram_paths` *integer*

Default: `10000`

Specifies the number of paths to capture for the TNS histogram.

metric_capture_vth_per_power_domain

```
metric_capture_vth_per_power_domain [true | false}
```

Default: `false`

Captures vth metrics per power domain.

metric_category_default

`metric_category_default` *`metric_category`*

Default: `design`

Specifies to capture the default metric category if none are provided.

metric_current_run_id

`metric_current_run_id`

Default: The default value of this string variable is not set.

Provides the current unique run ID returned by the advanced metric server.

It contains the identifier used with the advanced metric server so that multiple uploads of the data (such as, data upload after fplan, after prects, after cts) are seen as a single execution of the tools. The identifier is created by the advanced metric server so that it is unique and is stored in `metric_current_run_id`.

metric_enable

```
metric_enable {true | false}
```

Default: `false`

Enables the metric snapshot capture.

metric_summary_metrics

```
metric_summary_metrics
```

Default: The default value of this string variable is not set.

Lists metrics to report to log file during `createSnapshot`.

restore_db_directory

```
restore_db_directory directory_name
```

Type: `string`

Default: `""`

Specifies the directory of the current loaded database.

restore_db_file_check

```
restore_db_file_check {true | false}
```

Type: `boolean`

Default: `true`

By default, `restoreDesign` will check all the files inside the saved DB directory. If any file is edited, renamed, or deleted, the tool will give an error. However, `restoreDesign` will not perform this check if `restore_db_file_check` is set to `false`.

restore_db_stop_at_design_in_memory

```
restore_db_stop_at_design_in_memory { 0| 1}
```

Type: Boolean

Default: `1`

By default, tool gives error and stops if you call `restoreDesign` more than once in the same Innovus session unless `restore_db_stop_at_design_in_memory` is set to `0`. Using `restoreDesign` twice in one Innovus session is not recommended as settings from the first database may not be cleaned up properly and can cause problems in the second database.

restore_db_tool

```
restore_db_tool
```

Type: String

Default: Not set

Indicates the name of the tool that created the restored db on the disk database.

script_search_path

`script_search_path`

Default: ".". The default value represents the current application run directory.

Provides a set of directories for the software to search for files that are sourced using the Tcl `source` command. The software searches each search path directory for the specified file.

soft_stack_size_limit

`soft_stack_size_limit`

Default: `10`

Specifies the soft stack size limit in mbytes. When the tool is launched, the the soft stack size limit is auto-enlarged to 0.2% RAM. If 0.2%RAM is larger than the hard limit, the tool sets it to the hard limit.

11

GUI Commands

- add_gui_marker
- add_gui_shape
- add_gui_text
- clearAllRulers
- createRuler
- createSnapshot
- createUserBundleNet
- db_browser
- dehighlight
- delete_gui_object
- deleteUserBundleNet
- deleteWorkspace
- deselectModule
- deselectPin
- display_temperature_result
- displayUserBundleNet
- dumpPictures
- dumpToGIF
- editCopy
- fit
- get_visible_bumps
- get_visible_netGroups

- get_visible_nets
- gui_attach_to_cursor
- gui_clear_trace_flightline
- gui_close_cell_view
- gui_deselect
- gui_dim_foreground
- gui_dump_picture
- gui_group_hinst
- gui_open_cell_view
- gui_report_trace_flightline
- gui_save_flightline
- gui_schematic_load_view
- gui_schematic_save_view
- gui_select
- gui_show_edge_number
- gui_trace_flightline
- gui_ungroup_hinst
- highlight
- highlight_pin
- highlight_pin_connection
- lineSelect
- list_gui_marker
- loadWorkspace
- open_schematic
- pan
- panCenter
- panPage

- read_temperature_map_file
- redo
- redraw
- register_gui_edit_callback
- remove_gui_marker
- report_gui_edit_callback
- resetMultiColorsHier
- saveColorPreference
- saveHInstColor
- saveWorkspace
- select_highlighted
- selectModule
- selectPin
- set_object_color
- set_power_rail_display
- set_power_rail_layers_nets
- set_visible_bumps
- set_visible_netGroups
- set_visible_nets
- setDefaultWorkspace
- setHInstColorId
- setMultiColorsHier
- setSelectedPtnCut
- setSelectedPtnFeedthrough
- setSelectedPtnPinBlk
- setSelectedPtnPinGuide
- setSelectedWireState

- setSelHInstColor
- ui_view_box
- uiAdd
- uiDelete
- uiFind
- uiGet
- uiSet
- uiSetTool
- undo
- unregister_gui_edit_callback
- viewLast
- viewNext
- viewSnapshot
- windowDeselect
- windowSelect
- windowToggleSelect
- zoomBox
- zoomIn
- zoomOut
- zoomSelected
- zoomTo

add_gui_marker

```
add_gui_marker
[-help]
-color <string>
-name <string>
-pt {x y}
-type {X TICK STAR}
```

Places a marker to indicate location on the design layout.

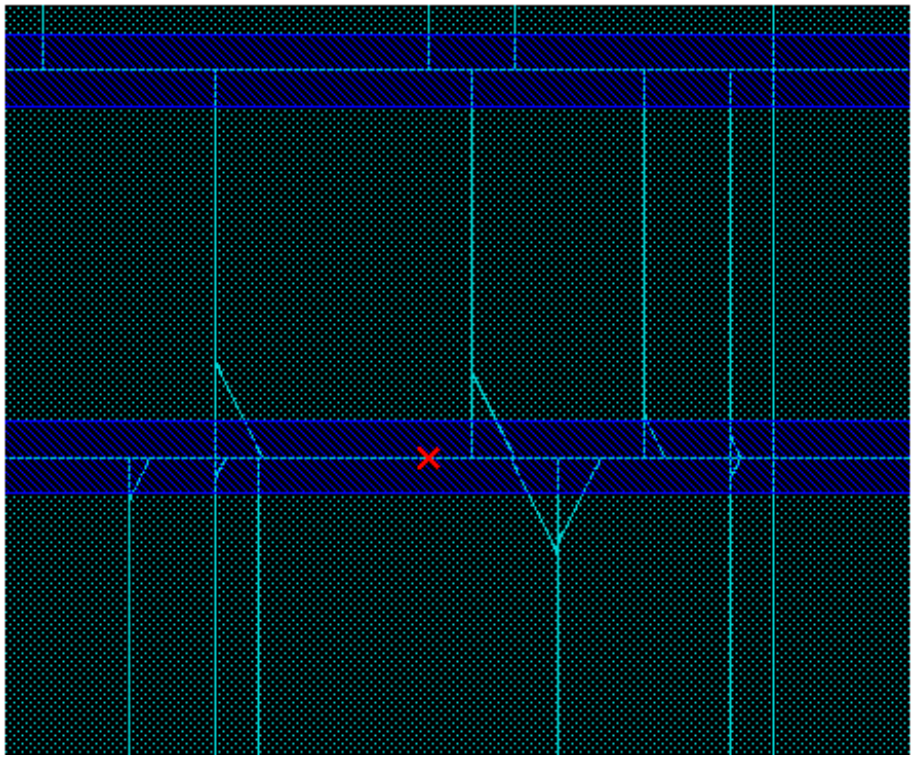
Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-color <string>`	Specifies the color of the marker. You can specify 16 colors for the marker. These are: `red blue green yellow magenta cyan pink orange brown purple violet teal olive gold maroon wheat`
`-name <string>`	Specifies the name of the marker.
`-pt {x y}`	Specifies the location `{x y}` at which the marker will be placed on the design layout.
`-type {X TICK STAR}`	Specifies the shape of the marker. There are three shapes for the markers: X, TICK, and STAR.

Example

- The following command shows that a X type marker of color RED has been placed at the location {100 100}.

```
add_gui_marker -name aa -color red -pt {100 100} -type X
```



Related Information

- `list_gui_marker`
- `remove_gui_marker`

add_gui_shape

```
add_gui_shape
[-help]
[-layer layerName]
[-name guiShapeName]
[-width width]
{-rect {x1 y1 x2 y2} | -line {x1 y1 x2 y2 ...} | -polygon {x1 y1 x2 y2 ...}}
[-line {x1 y1 x2 y2 ...} [-arrow ] ]
```

Adds a shape to the default or specified custom layer. You can use the `add_gui_shape` command at any point in the design flow. The shapes added by the command annotate comment shapes and do not show up in the DEF output.

Note: In OpenAccess flows, the shapes added with the `add_gui_shape` command are stored in the OpenAccess cellview. However, the shapes are stored in a private structure that is accessible only to Innovus. To create shapes that are visible in Virtuoso as well, use the `add_shape` command instead.

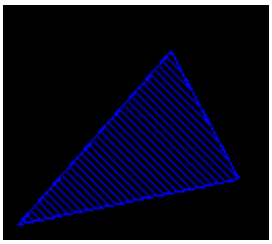
Parameters

`-help`	Prints a brief description that includes type and default information for each `add_gui_shape` parameter. For a detailed description of the command and all of its parameters, use the man command: `man add_gui_shape`
`-arrow`	Draws an arrow in the middle of the line segment to indicate the start to end direction. This parameter can only be used with `-line`.
`-layer` *layerName*	Specifies the GUI layer name to use or create for the shape. If the specified layer does not exist, it is created automatically. If a layer is not specified, the tool automatically creates a layer with the name `defaultLayer` and adds the shape to it.
`-line {`*x1 y1 x2 y2 ...*`}`	Creates a line, with *x1 y1 x2 y2* ... specifying the coordinates for the line segments. The values are specified in microns.
`-name` *guiShapeName*	Specifies the name of the GUI shape.
`-polygon {`*x1 y1 x2 y2 ...*`}`	Creates a polygon shape, with *x1 y1 x2 y2* ... specifying the coordinates for the polygon segments. The values are specified in microns.
`-rect {`*x1 y1 x2 y2*`}`	Specifies the coordinates of the lower-left and upper-right points of the rectangle. The values are specified in microns.
`-width` *width*	Specifies the width of the shape border in pixels. The minimum (default) value is `1` and the maximum is `7`.

Examples

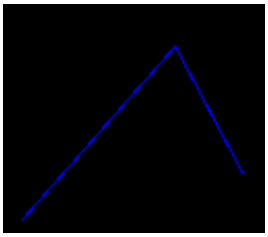
- The following command adds a triangular shape in layer `1`:

```
add_gui_shape -layer 1 -polygon {95.841 -645 96.592 -644.147 96.926 -644.777}
```



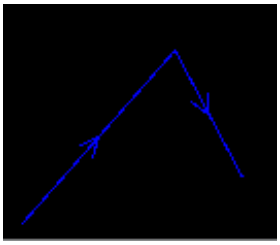
- The following command adds a line shape with two segments:

```
add_gui_shape -layer 1 -line {95.841 -645 96.592 -644.147 96.926 -644.777}
```



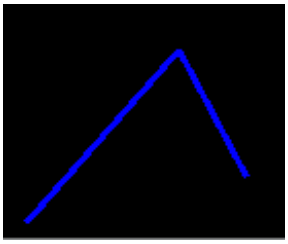
- The following command adds a line shape with two segments and direction arrows:

```
add_gui_shape -layer 1 -line {95.841 -645 96.592 -644.147 96.926 -644.777} -arrow
```



- The following command adds a line shape of 3-pixel width:

```
add_gui_shape -layer 1 -line {95.841 -645 96.592 -644.147 96.926 -644.777} -width
3
```



Related Information

- add_gui_text
- add_shape
- add_text
- The Main Window chapter of *Menu Reference*
    - *Wire Edit Toolbar*

add_gui_text

```
add_gui_text
[-help]
-label text
-layer layerName
{-box {x1 y1 x2 y2} | {{-height height | -fixed_height fixHeight} -pt {x y} [-orient
direction]}}
[-box {x1 y1 x2 y2} [-no_bbox]]
```

Adds text (annotation) to the specified custom layer. You can use the `add_gui_text` command at any point in the design flow. The text added by the command annotates comment text and does not show up in a DEF output.

Note: In OpenAccess flows, any text added with the `add_gui_text` command is stored in the OpenAccess cellview. However, the text is stored in a private structure that is accessible only to Innovus. To create text that is visible in Virtuoso as well, use the `add_text` command instead.

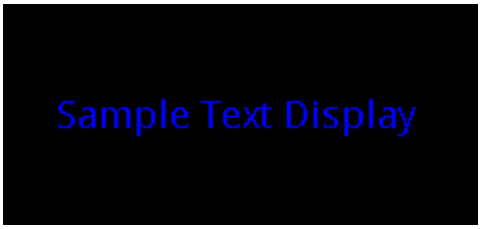
Parameters

`-help`	Prints a brief description that includes type and default information for each `add_gui_text` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man add_gui_text`
`-box {x1 y1 x2 y2}`	Specifies the coordinates of the rectangle in which text will be added. The coordinate values are specified in microns. When this parameter is used, the tool figures out the correct size and orientation to be used to fit the text into the box. The text is center justified in the box. Use this parameter to control the text orientation and size.
`-height height`	Specifies the height of the text in microns.
`-fixed_height fixHeight`	
	Specifies the height in pixels. *`fixHeight`* should be an integer value.
`-label text`	Specifies the text to be displayed. If your text has spaces, enclose it in quotation marks or braces.
`-layer layerName`	Specifies the GUI layer name to use or create for the text. If the specified layer does not exist, it will be automatically created.
`-no_bbox`	Specifies that bounding box should not be drawn when text is displayed at the specified box coordinates. This parameter can be used only when `-box` is specified.
`-orient direction`	Specifies the orientation of the text. Possible values are: `MX MX90 MY MY90 R0 R90 R180 R270` This option cannot be used with `-box`.
`-pt {x y}`	Specifies the origin coordinate of the text (microns).

Examples

- The following command adds the text label `Sample Text Display` with size of 18 pixels on layer `1` at the specified point coordinates:

  ```
  add_gui_text -layer 1 -label "Sample Text Display" -fixed_height 18 -pt {0 250}
  ```

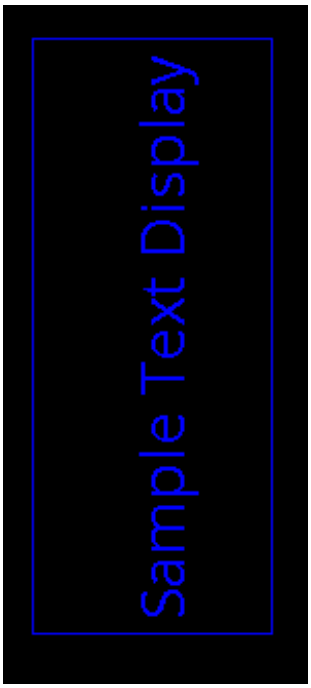
  

- The following command adds the text label `Sample Text Display` on layer `1` in a box with the specified coordinates. As the length of the box is more than the width, the text is displayed horizontally:

  ```
  add_gui_text -layer 1 -box {0 250 500 450} -label "Sample Text Display"
  ```


- The following command adds the text label `Sample Text Display` on layer `1` in a box with the specified coordinates. As the height of the box is more than the width, the text is displayed vertically:

  ```
  add_gui_text -layer 1 -box {0 250 200 750} -label "Sample Text Display"
  ```



- The following command adds the text label `Sample Text` Display on layer `1` at the specified box coordinates. However, the bounding box is not displayed:

```
add_gui_text -layer 1 -box {0 250 500 450} -no_bbox -label "Sample Text Display"
```

Related Information

- `add_gui_shape`
- `add_shape`
- `add_text`
- The Main Window chapter of *Menu Reference*
  - *Wire Edit Toolbar*

clearAllRulers

```
clearAllRulers -help
```

Removes all ruler lines from the main window. You can also clear the ruler lines with the *Clear All Rulers* icon on the toolbar in the main window.

Related Information

- `createRuler`
- The Main Window chapter of *Menu Reference*
  - *Toolbar Widgets*
- Tools Menu chapter of *Menu Reference*
  - *Create Ruler*
  - *Clear All Rulers*

createRuler

```
createRuler
[-help]
{{-center {x y} -radius value} | -coordinates {x1 y1 x2 y2 ...}}
```

Adds a ruler. For an orthogonal ruler, use `-coordinates` to specify ruler segments. For a circular ruler, use `-center` and `-radius`.

Parameters

`-help`	Prints a brief description that includes type and default information for each `createRuler` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man createRuler`
`-center {x y}`	Specifies the coordinates for the center of a circular ruler.
`-coordinates {x1 y1 x2 y2 ...}`	Specifies the beginning and ending coordinates for the ruler line segments. The values are in microns.
`-radius value`	Specifies the radius of a circular ruler. The value is of type float.

Example

- The following command creates a circular ruler of radius 500 with the center at `{0 0}`:

  ```
  createRuler -center {0 0} -radius 500
  ```

Related Information

- `clearAllRulers`
- The Main Window chapter of *Menu Reference*
  - *Toolbar Widgets*

- Tools Menu chapter of *Menu Reference*
  - *Create Ruler*
  - *Clear All Rulers*

createSnapshot

```
createSnapshot
[-help]
-dir string
-name string
[-overwrite]
```

Creates snapshots of the design for all three views (Floorplan, Amoeba, and Placement) at any stage of the design flow. The command creates GIF images, for all the design views, on the same scale as can be seen in the software. It can also save QoR parameters like WNS,TNS, Placement Overlap, and Congestion at any stage of the design flow. Once the snapshot is created, you can view it using `viewSnapshot`.

Parameters

`-help`	Prints a brief description that includes type and default information for each `createSnapshot` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man createSnapshot`.
`-dir` *`string`*	Specifies the directory in which you want to save the snapshot.
`-name` *`string`*	Specifies the name of snapshot and the files will be created, like `ss_string_view.gif` for all three views.
`-overwrite`	Overwrites the existing snapshot with new one.

Note: This command will not work in the `-nowin` mode.

Examples

- The following command will create a design snapshot in the directory `/work/user1`. Before creating the snapshot, it will create the following files:
  - `ss_cts.fplan.gif` for floorplan view

  - `ss_cts.amoeba.gif` for amoeba view
  - `ss_cts.place.gif` for place view

    ```
    createSnapshot -dir /work/user1 -name cts
    ```

- The following command will create a snapshot under the name `cts`. If there exists a snapshot under this name in the directory `/work/user1`, it will overwrite it.

  ```
  createSnapshot -dir /work/user1 -name cts -overwrite
  ```

Related Information

- `dumpPictures`
- `dumpToGIF`
- `gui_dump_picture`
- `viewSnapshot`
- Tools Menu chapter of *Menu Reference*
  - *Create Snapshot*
  - *View Snapshot*

createUserBundleNet

```
createUserBundleNet
[-help]
name
-nets listOfNets
-object1 string
-object2 string
```

Creates a bundle of specified nets that can be displayed as flightlines. Bundled nets help you with data flow analysis before and after layout. Net bundles can be created and viewed in floorplan and amoeba views. Each custom net bundle displays the number of nets in it and works for macros, blobs, and giudes.

Using `createUserBundleNet`, you can create custom bundled nets that allow you to define your own net bundles as per your requirements. This is useful while debugging physical properties of interconnects within a complicated hierarchical design because you can define start and end points of a net bundle. As a result, you can create and display only relevant flightlines, instead of looking at all the flightlines of specified nets.

Note: One net can be part of multiple net bundles simultaneously.

You can display flightline net bundles using the `displayUserBundleNet` command and delete them using the `deleteUserBundleNet` command.

Note: You can choose to display flightlines for only bundled nets by using the *Show Net Bundle FlightLine Only* option in the Preferences form. If this option is not selected, the net bundle flightlines are displayed along with other selection-based flightlines by default.

Parameters

`-help`	Prints a brief description that includes type and default information for each `createUserBundleNet` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man createUserBundleNet`
`name`	Specifies the name of the custom net bundle.
`-nets` *`listOfNets`*	Specifies the list of nets to be included in the net bundle. You can use patterns with wildcards to specify net names. All the nets that match the specified pattern are put in the same bundle.
`-object1` *`string`*	Specifies the name of object which is the start point of the nets. The object type includes module, block, black box, and IO pad.
`-object2` *`string`*	Specifies the name of object which is the end point of the nets. The object type includes module, block, black box, and IO pad.

Examples

- The following command creates a bundle called `bundle_1`, which contains flightlines of nets `net_1` and `net_2`:

  ```
  createUserBundleNet bundle_1 -nets {net_1 net_2} -object1 Module1 -object2 Module2
  ```

  You can then use the `displayUserBundleNet` command to display `bundle_1` flightlines as follows:

  ```
  displayUserBundleNet -netBundle bundle_1
  ```

Related Information

- `deleteUserBundleNet`
- `displayUserBundleNet`
- View Menu chapter of the *Menu Reference*
    - *Preferences - Flightline*

db_browser

```
db_browser
[-help]
obj_list | head | top | selected
```

Launches the database browser (DB Browser). The DB browser returns information about one of the following:

- Specified object list
- Root of the database
- Top cell of the design
- Selected object

Parameters

`-help`	Prints a brief description that includes type and default information for each `db_browser` parameter. For a detailed description of the command and all of its parameters, use the man command: `man db_browser`
`obj_list` | `head` | `top` | `selected`	Specifies the objects whose attributes need to be displayed in the browser: • *`obj_list:`* Displays attributes of the list of specified objects in the browser. Use the *Show next object in list* and the *Show last object in list* buttons in the browser to traverse through the list. • `head`: Displays technology information about the root of the database in the browser. • `top`: Displays attributes of the top cell of the design in the browser • `selected`: Displays attributes of the objects you have selected in the main window. Use the *Show next object in list* button in the DB Browser to move to the next object in the sequence.

Examples

- The following command launches the DB Browser and returns attributes of the top cell of the design.

```
db_browser top
```

Related Information

- Edit Menu chapter in the *Innovus Menu Reference*
    - DB Browser

dehighlight

```
dehighlight
[-help]
[objectList]
[-all | -index indexValue | -original | -select]
```

Removes highlights from all the highlighted objects at any stage of the design flow. You can choose to dehighlight all objects, selected objects, objects highlighted with original color, or objects with specified index values.

Parameters

`-help`	Prints a brief description that includes type and default information for each `dehighlight` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man dehighlight`
`-all`	Specifies that all objects will be dehighlighted.
`-index` *`indexValue`*	Specifies that the objects with the index value provided will be dehighlighted. The index value is the value of the Highlight Set as specified through the *Edit - Highlight Selected* menu or through the `highlight` command. The *Edit - Highlight Selected* menu provides eight highlight patterns to choose from. The value of index should, therefore, be a number be from 1 to 8. As an example, if you specify a value of 3 for index, all objects that are highlighted as Highlighted Set 3 will be dehighlighted.
`objectList`	Specifies the object pointer list for dehighlighting.
`-original`	Specifies that the objects highlighted with original color should be dehighlighted.
`-select`	Specifies that all selected objects should be dehighlighted.

Examples

- The following command dehighlights all highlighted objects.

  ```
  dehighlight
  ```

- The following command dehighlights all highlighted objects that have been highlighted as Highlight Set 4 through the Edit - Highlight Selected menu or through the `highlight` command.

  ```
  dehighlight -index 4
  ```

- The following command dehighlights all highlighted objects that are currently selected in the display window.

  ```
  dehighlight -select
  ```

Related Information

- `highlight`
- `highlight_pin`
- View Menu chapter of the *Menu Reference*:
  - *Highlight Selected*
  - *Clear Highlight*
  - *Edit Highlight Color*

delete_gui_object

```
delete_gui_object
[-help]
[-shape | -text | -marker | -all | -selected ]
```

Deletes specified or selected GUI objects.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command, use the `man` command: `man delete_gui_object`
`-all`	Deletes all GUI objects.
`-marker`	Deletes GUI markers.
`-selected`	Deletes selected GUI objects.
`-shape`	Deletes GUI shapes.
`-text`	Deletes GUI text.

Related Information

- `add_gui_marker`
- `add_gui_shape`
- `add_gui_text`

deleteUserBundleNet

```
deleteUserBundleNet
[-help]
name
```

Deletes the specified flightline net bundle. If a net is part of multiple net bundles and you delete one of the net bundles, other net bundles containing the net are not affected.

You can create flightline net bundles using the `createUserBundleNet` command and display them using the `displayUserBundleNet` command.

Note: `deleteUserBundleNet` removes the specified net bundle. If you want to retain the net bundle but just want to clear its flightlines from the display area, use the `-remove` option of `displayUserBundleNet` instead.

Parameters

`-help`	Prints a brief description that includes type and default information for each `deleteUserBundleNet` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man deleteUserBundleNet`
`name`	Specifies the name of the flightline net bundle to be removed.

Examples

- The following command deletes the flightline net bundle `bundle_1`:

  ```
  deleteUserBundleNet bundle_1
  ```

Related Information

- `createUserBundleNet`
- `displayUserBundleNet`
- View Menu chapter of the *Menu Reference*
    - *Preferences - Flightline*

deleteWorkspace

```
deleteWorkspace
[-help]
-name workspaceName
[-dir directory]
```

Deletes the specified workspace. Use this command at any stage in the design flow to remove a workspace that you do not need any longer.

Parameters

`-help`	Prints a brief description that includes type and default information for each `deleteWorkspace` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man deleteWorkspace`
`-name` *`workspaceName`*	Specifies the name of the workspace to be deleted.
`-dir` *`directory`*	Specifies the directory from which the workspace is to be deleted. The default search order to access directories for deleting workspaces is as below: • Current working directory • User home directory

Example

The following example deletes the `Test` workspace from the current working directory.

```
deleteWorkspace -name Test
```

Related Information

- `loadWorkspace`
- `saveWorkspace`
- `setDefaultWorkspace`
- Windows Menu chapter of the *Menu Reference*
    - *Workspaces*

deselectModule

```
deselectModule
[-help]
name
```

Deselects the specified module.

Parameters

`-help`	Prints a brief description that includes type and default information for each `deselectModule` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man deselectModule`
name	Specifies the name of the module to be deselected.

Example

The following example deselects the `DTMF_INST` module.

```
deselectModule DTMF_INST
```

Related Information

- `selectModule`

deselectPin

```
deselectPin
[-help]
name
```

Deselects the specified pin.

Parameters

`-help`	Prints a brief description that includes type and default information for each `deselectPin` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man deselectPin`
`name`	Specifies the name of the pin to be deselected.

Example

The following example deselects the `swack` pin.

```
deselectPin swack
```

Related Information

- `selectPin`

display_temperature_result

```
  display_temperature_result
[-help]
-layer <string>
-reset
```

Displays the tile-based temperature map file generated by Sigrity Celsius.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-layer <string>`	Displays the temperature map for the specified layer. **Example**: `display_temperature_result -layer Metal6`
`-reset`	Resets the temperature map display.

displayUserBundleNet

```
displayUserBundleNet
[-help]
[-keepExisting]
-netBundle bundleName
[-remove]
```

Displays flightlines for the specified net bundle. Bundled nets help you with data flow analysis before and after layout. Net bundles can be created and viewed in floorplan and amoeba views. When you run the `displayUserBundleNet` command, the software switches automatically to the net bundle mode and clears flightlines based on selection.

You can create flight line net bundles using the `createUserBundleNet` command and delete them using the `deleteUserBundleNet` command.

Note: When selecting objects, you can choose to display flightlines for only bundled nets by using the *Only Show Net Bundle Flight Line* option in the Preferences form. If this option is not selected, the net bundle flightlines are displayed along with other flightlines.

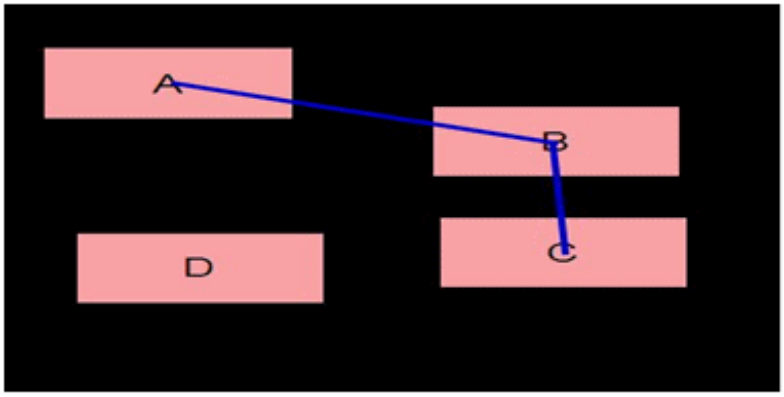
Parameters

`-help`	Prints a brief description that includes type and default information for each `displayUserBundleNet` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man displayUserBundleNet`
`-keepExisting`	Retains existing net bundle flightlines while displaying the flightline of specified net bundles. If this option is not specified, the command clears existing flightlines before displaying specified net bundle flightlines.
`-netBundle` *`bundleName`*	Specifies the name of the net bundle whose flightlines you want to display. You can specify more than one net bundle.
`-remove`	Clears flightlines of specified net bundle from the display area. If no net bundle name is specified, all flightlines based on net bundles are removed. **Note:** This option only clears the flightlines of the net bundle from the display area. If you want to remove the net bundle itself, use the `deleteUserBundleNet` command.

Examples

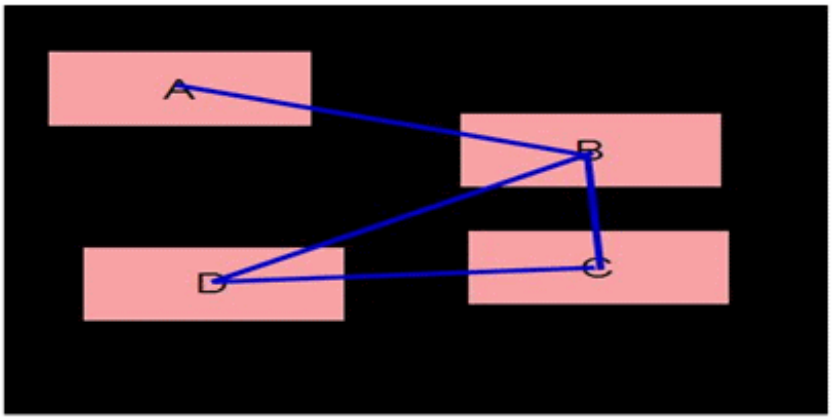
- The following command displays flightlines of the net bundle `bundle_1`. Here, `bundle_1` contains `net_1`, which is just between A and B, and `net_2`, which is just between B and C:

  ```
  displayUserBundleNet -netBundle bundle_1
  ```

- The following command displays flightlines of the net bundle `bundle_2` while retaining flightlines of `bundle_1`. Here, `bundle_2` contains `net_3`, which is among B, C, and D:

  ```
  displayUserBundleNet -netBundle bundle_2 -keepExisting
  ```

  Note: The same flightlines could also be displayed in a single step by using the following command:

  ```
  displayUserBundleNet -netBundle {bundle_1 bundle_2}
  ```

Related Information

- `createUserBundleNet`
- `deleteUserBundleNet`
- View Menu chapter of the *Menu Reference*
    - *Preferences - Flightline*

dumpPictures

```
dumpPictures
[-help]
-dir string
[-fullScreen]
-prefix string
```

Dumps images at various stages of the Innovus flow (such as congestion map, cell density map, floorplan view, colored hierarchy map), when executed in batch mode. This command dumps the following images in the specified directory in the `.gif` format:

1. `ctd`
2. `congestion`
3. `density`
4. `floor`
5. `placement`
6. `route_all`
7. `route_C4`
8. `route_C5`
9. `route_C6`
10. `route_M1`
11. `route_M2`
12. `route_M3`
13. `specialroute`
14. `violations`

Parameters

`-help`	Prints a brief description that includes the type and default information for each `dumpPictures` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man dumpPictures`
`-dir` *`string`*	Specifies the output directory.
`-fullScreen`	Maximizes the window.
`-prefix` *`string`*	Specifies the file prefix.

Example

The following command dumps the full screen images at the various stages of the Innovus flow to the `TEST` directory:

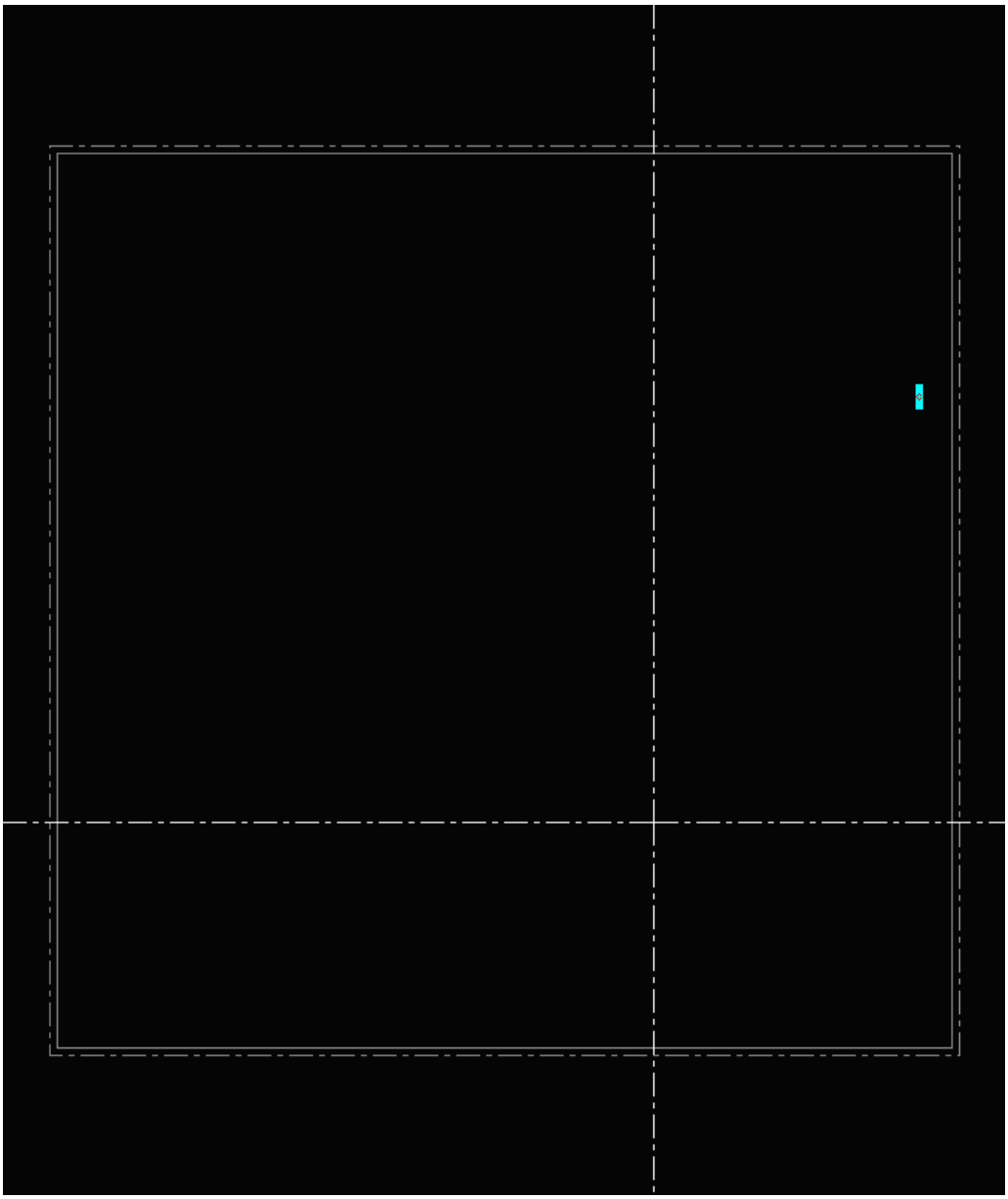
```
dumpPictures -dir TEST -prefix [dbGet top.name] -fullScreen
```

The following images are dumped in the `TEST` directory:

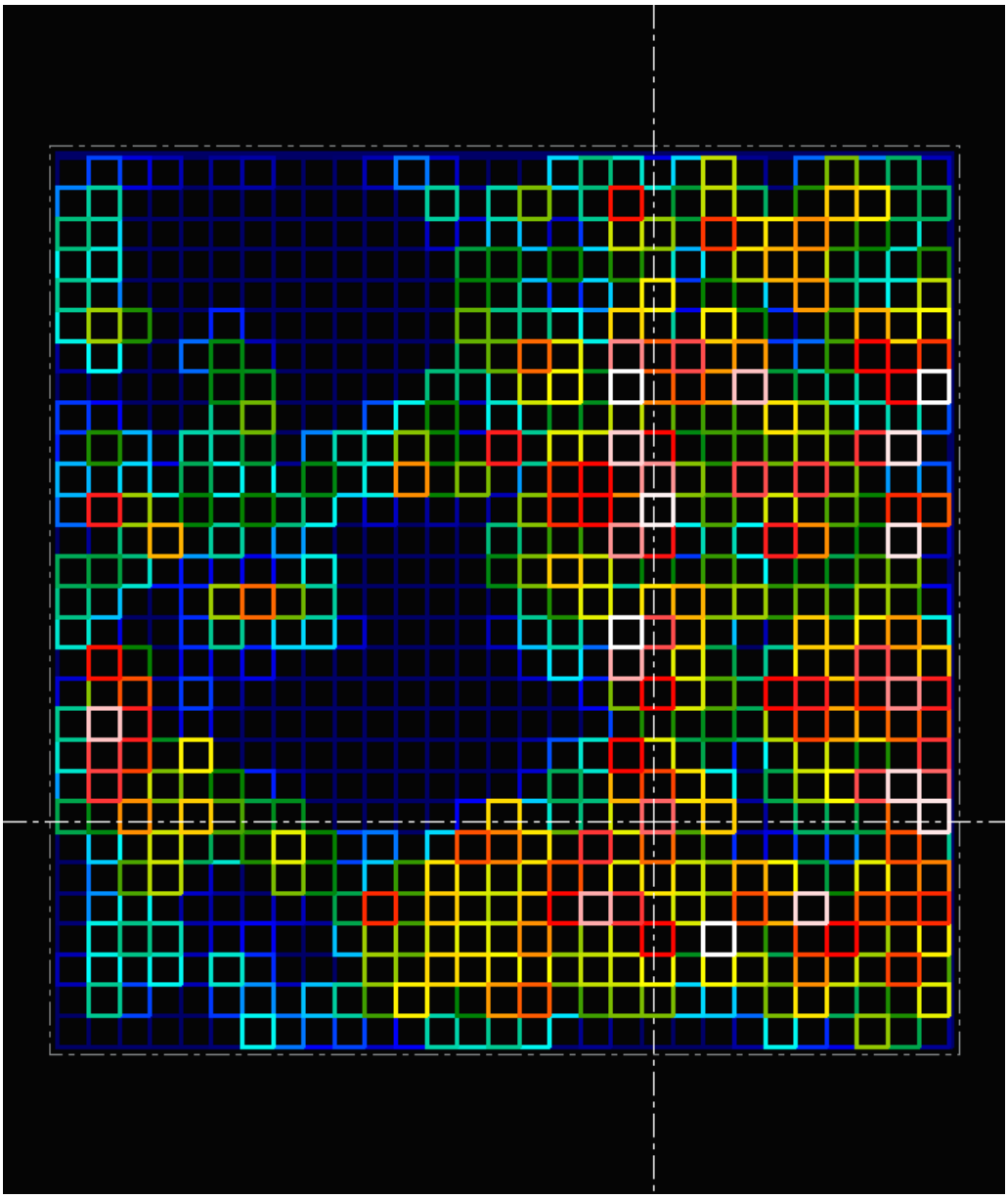
1. `congestion`
2. `density`
3. `floor`
4. `placement`
5. `route_all`
6. `route_C4`
7. `route_C5`
8. `route_C6`
9. `route_M1`
10. `route_M2`

11. `route_M3`
12. `specialroute`
13. `violations`

Following is the `congestion` image:



Following is the `density` image:



Related Information

- createSnapshot
- dumpToGIF
- gui_dump_picture

dumpToGIF

```
dumpToGIF
[-help]
filename
[-honor_umask]
```

Saves a snapshot of the current screen to a GIF file with the specified name in the current directory. You can use this command at any stage of the design flow.

Parameters

`-help`	Prints a brief description that includes type and default information for each `dumpToGIF` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man dumpToGIF`.
`filename`	Specifies the name of the output file as a string.
`-honor_umask`	Honors the umask settings while saving the GIF file. If you specify this option, the permission of the saved GIF file will be set by subtracting the umask value from the full permission value. For example, if the full permission setting of a file is `0666` and the umask setting is `0022`, the tool saves the GIF file with the permission setting of `0666 - 0022`, that is `0644`.

Example

The following example captures the current screen and saves it to *`test1.gif`* in the current directory.

```
dumpToGif test1.gif
```

Related Information

- `createSnapshot`
- `dumpPictures`
- `gui_dump_picture`
- Tools Menu chapter of *Menu Reference*
    - *Write To GIF File*

editCopy

```
editCopy
[-help]
[-objects string]
[-orientation {R0 R90 R180 R270 MX MX90 MY MY90}]
[-net net_name | -net_group string | -keep_net_name]
[-rotate_point {x y} | -times integer]
[-rotate_point {x y} | [dx dy ]]
```

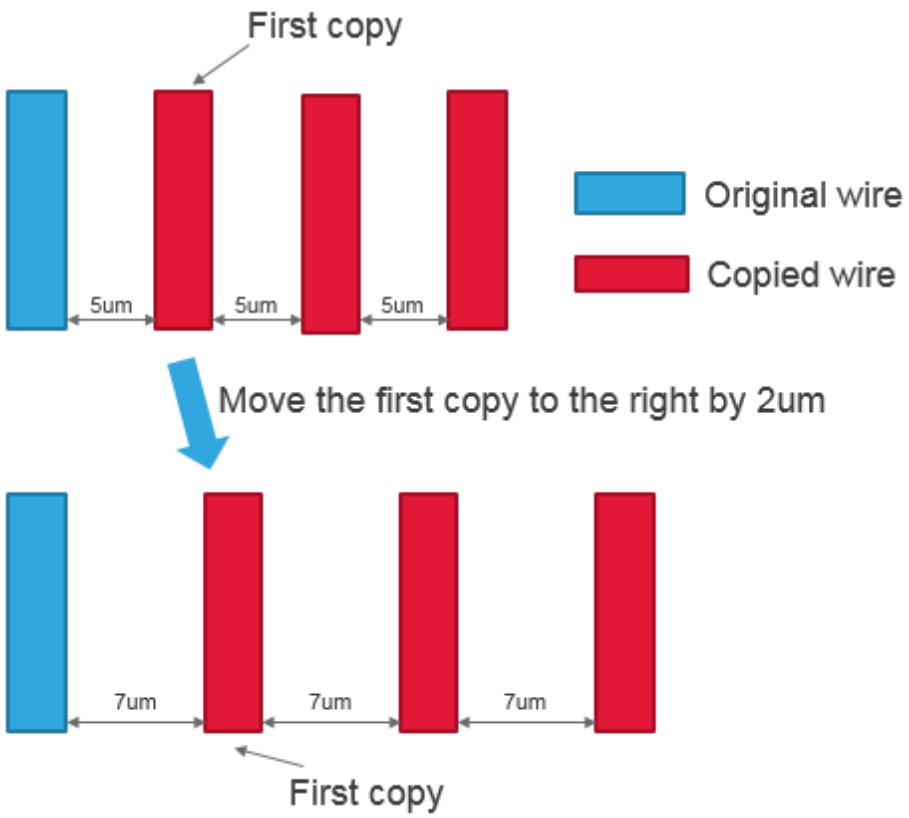
Copies selected or specified objects to the specified location. In the current release, the command supports the copying of bus guides, regular wires and vias, special wires and vias, as well as patch wires. When `setEditMode -snap_objects_to_track regular` (or `special` or `patch`) is set before using `editCopy`, copied wires of that type are snapped to track.

Note: Use the `copyObject` and `pasteObject` commands to copy placement and routing blockages.

Parameters

`-help`	Prints a brief description that includes type and default information for each `editCopy` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man editCopy`
dx	Specifies the dx distance to move the object copy in microns.
dy	Specifies the dy distance to move the object copy in microns.
`-keep_net_name`	Specifies that the net name from the original object should be retained after copying.

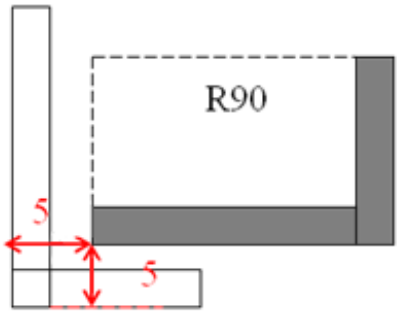
`-net` *net_name*	Specifies the net to which the copied object should be assigned. If neither `-net` nor `-keep_net_name` is specified and: • The copied wire connects to a single wire, the tool assigns the single wire's net to the copied wire. • The copied wire connects to multiple wires on the same net, the tool assigns the net to the copied wire. • The copied wire connects to multiple wires on different nets, the tool assigns `_NULL` to the copied wire and displays the following message: `**WARN: (IMPSPR-304): One target route connects to more than one routes on different nets, net for the target route might not be correctly assigned. Undo and try again to pick a proper location.`
`-net_group` *string*	Assigns the specified net group name to the new copy of the bus guide. You can specify only one net group name with the `-net_group` parameter. If the net group name you specify does not exist, the tool does not copy the bus guide and issues an error message. If you copy a bus guide without the `-net_group` parameter, the new bus guide retains the net group name of the original bus guide. Note that the `-net_group` parameter cannot be used for wires or vias. If you use it for copying a wire or via, the tool ignores the setting but continues the copy operation with a warning. Similarly, the `-net` and `-keep_net_name` parameters do not work for bus guides. If you copy a bus guide with `-net` or `-keep_net_name`, the tool ignores the settings, issues a warning, and copies the bus guide with the original net group name.
`-objects` *string*	Specifies the object pointers to be copied.
`-orientation {R0 R90 R180 R270 MX MX90 MY MY90}`	
	Specifies the orientation of object copy after pasting. Use this parameter to rotate the selected wire or via during copy/paste. For values `R0`, `R90`, `R180`, and `R270`, the wire or via is rotated by specified degrees around the lower left point. For values `MX`, `MX90`, `MY`, and `MY90`, the bounding box is mirrored.
`-rotate_point {x y}`	

	Rotates the copied wire(s) around the specified point `{x y}`, when specified along with the `-orientation` parameter. If the `-rotate_point` parameter is not specified, the copied wires are rotated by default around the lower-left corner of the bounding box of the original wires. If you specify the `-rotate_point` parameter without the `-orientation` parameter, the tool copies the object at the same location. This is the same as specifying the `-orientation {R0}` option. **Note**: The `-rotate_point` parameter cannot be used with the `-times` parameter.
`-times` *integer*	Specifies the number of times the object is to be copied. For example, if you specify 3, the selected wire is copied thrice. The distance between different copies of the wire is equal to the distance between the original wire and the first copy. If you move the first copy, the other copies are moved by the same distance:  **Note**: If DRC checking is turned on with `setEditMode -drc_on true`, all copied wires obey the DRC rules.

Examples

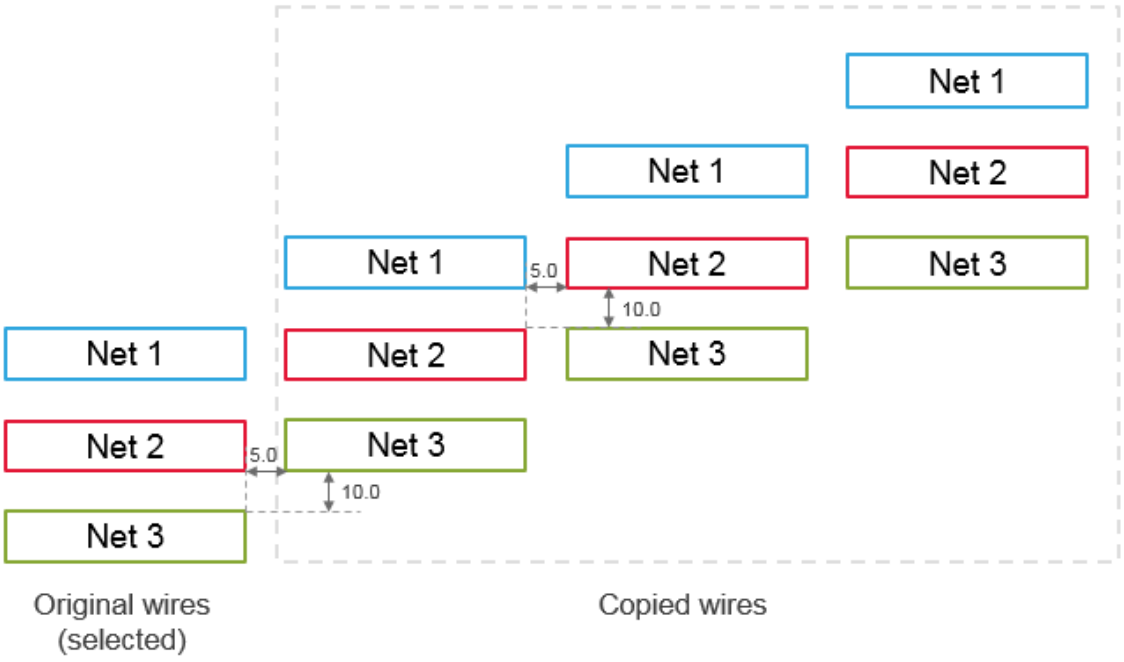
- The following command copies the specified wire 5 microns up and to the right of the original and rotates the wire by 90 degrees around the lower left corner:

  ```
  editCopy 5 5 -orientation R90 -objects wire_pointer
  ```



- The following command copies the selected wires 3 times and the distance between each copy set is as specified:

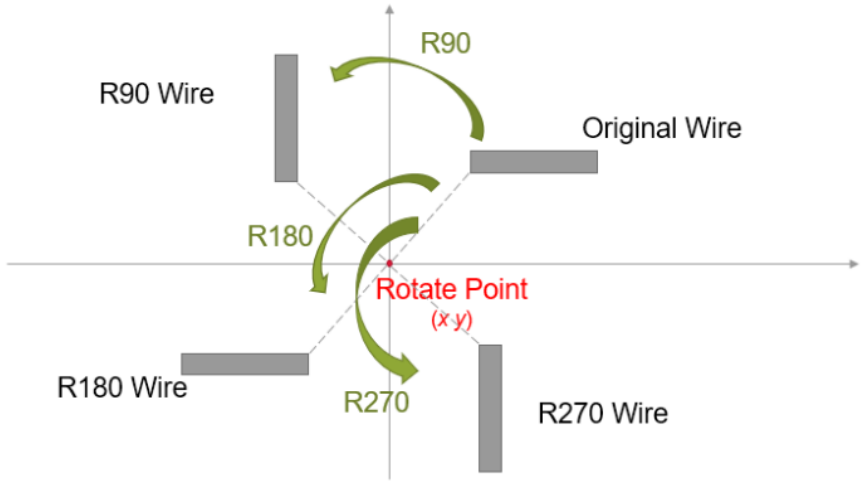
  ```
  editCopy 5.0 10.0 -times 3 -keep_net_name
  ```



- The following commands rotate the new copy of the selected wire by 90, 180, and 270 degrees, respectively, around the specified rotation point, *x y*:

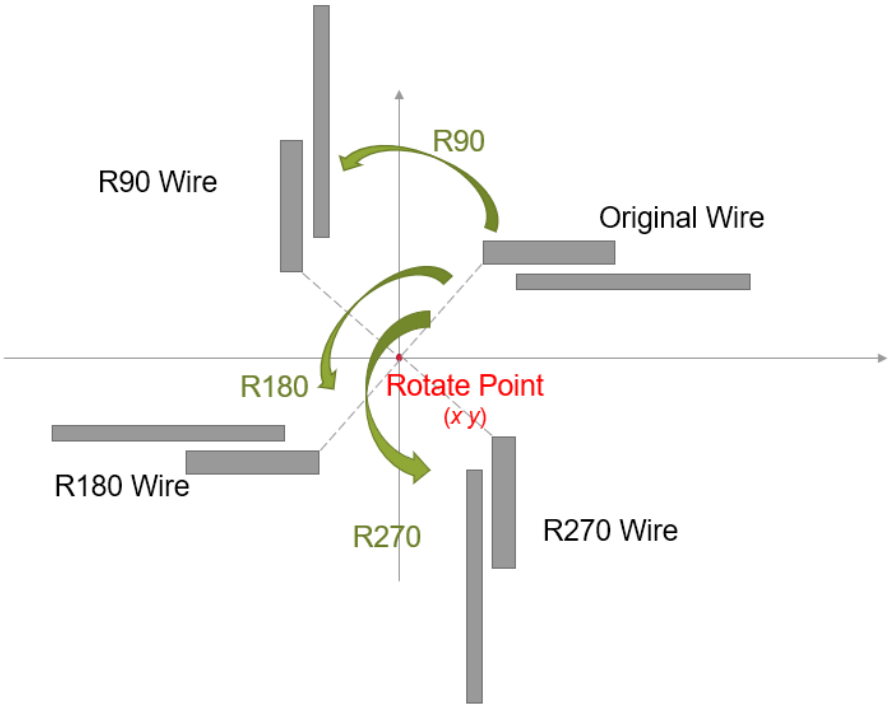
  ```
  editCopy -orientation R90 -rotate_point x y
  editCopy -orientation R180 -rotate_point x y
  ```

```
editCopy -orientation R270 -rotate_point x y
```



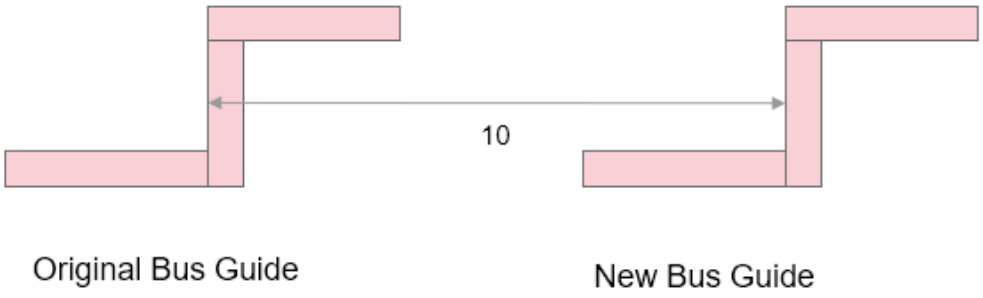
- The following commands rotate the new copy of the selected group of wires by 90, 180, and 270 degrees, respectively, around the specified rotation point, `x y`:

```
editCopy -orientation R90 -rotate_point x y
editCopy -orientation R180 -rotate_point x y
editCopy -orientation R270 -rotate_point x y
```



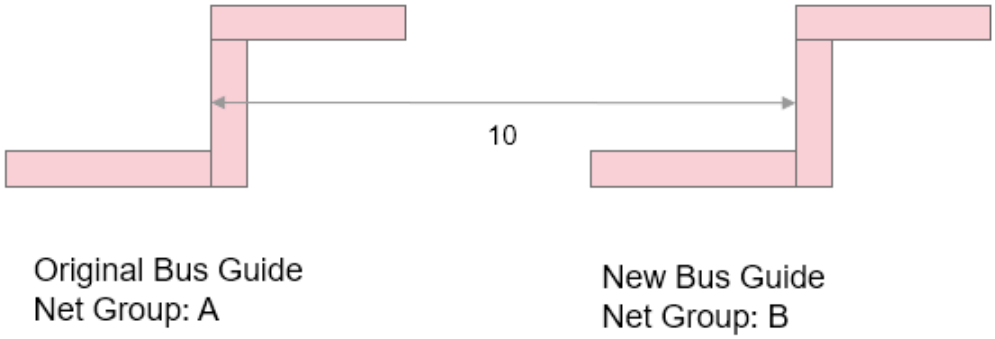
- The following command copies the selected bus guide 10 microns to the right of the original:

```
editCopy 10 0
```



- The following command copies the selected bus guide 10 microns to the right of the original and assigns the net group `B` to the copy:

```
editCopy 10 0 -net_group B
```



Related Information

- `copyObject`
- `editDuplicate`
- `editMove`
- `pasteObject`
- `setEditMode`
- Edit Menu chapter of the *Menu Reference*
  - *Copy* command
- Editing Wires chapter of the *User Guide*
  - *Copying Wires*

fit

```
fit [-help]
```

Fits the entire design in the viewable window. You can use this command at any stage of the design flow to view the design in entirety.

Parameters

`-help`	Prints a brief description of the `fit` command. For a detailed description of the command, use the `man` command: `man fit`.

Related Information

- `zoomIn`
- `zoomOut`
- View Menu chapter of the *Menu Reference*
  - *Fit*

get_visible_bumps

```
get_visible_bumps
[-help]
```

Returns all visible bump cells in the design if some bumps have been hidden in the design.

Parameters

`-help`	Prints a brief description that includes type and default information for each `get_visible_bumps` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man get_visible_bumps`

Examples

- The following command returns the name of all visible bumps in the designs:
  ```
  innovus 17> get_visible_bumps
  ```

Related Information

- `set_visible_bumps`

get_visible_netGroups

```
get_visible_netGroups
[-help]
```

Returns the names of the visible netGroups if some netGroups have been hidden in the design. If all netGroups are visible, this command has no effect.

Parameters

`-help`	Prints a brief description that includes type and default information for each `get_visible_netGroups` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man get_visible_netGroups`

Examples

- The following command returns the name of all visible netGroups in the designs:

  ```
  innovus 17> get_visible_netGroups
  test_ng
  ```

Related Information

- `set_visible_netGroups`

get_visible_nets

```
get_visible_nets
[-help]
```

Returns the names of the visible nets if some nets have been hidden in the design. If all nets are visible, this command has no effect.

Parameters

`-help`	Prints a brief description that includes type and default information for each `get_visible_nets` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man get_visible_nets`

Examples

- The following command returns the name of all visible nets in the designs:

```
innovus 17> get_visible_nets
ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/tpo_a_9_
```

Related Information

- `set_visible_nets`

gui_attach_to_cursor

```
gui_attach_to_cursor
[-help]
[ -selected  | objects ]
```

Attaches selected or specified objects to the cursor. The use model is as follows:

1. Select the object(s) to be moved by using the `gui_attach_to_cursor` command.
2. Zoom into the location where you want to place the object(s).
3. Click at the required location to place the selected object(s) there.

This command is useful during floorplanning when you may need to move multiple objects into the same zoomed in area.

Parameters

`-help`	Prints a brief description of the `gui_attach_to_cursor` command. For a detailed description of the command, use the `man` command: `man gui_attach_to_cursor`.
`objects`	Specifies the object pointer list.
`-selected`	Attaches the selected objects to the cursor.

Example

The following command attaches objects `A` and `B` to the cursor:

```
gui_attach_to_cursor {A B}
```

Now, move the cursor to the layout and click at the desired location to place objects `A` and `B` there.

Related Information

- gui_select
- zoomIn
- zoomOut
- zoomTo

gui_clear_trace_flightline

```
gui_clear_trace_flightline [-help]
```

Clears all flightlines traced through the `gui_trace_flightline` command.

Parameters

`-help`	Prints a brief description of the `gui_clear_trace_flightline` command. For a detailed description of the command, use the `man` command: `man gui_clear_trace_flightline`.

Related Information

- `gui_report_trace_flightline`
- `gui_save_flightline`
- `gui_trace_flightline`

gui_close_cell_view

```
gui_close_cell_view [-help]
```

Closes the Cell Viewer.

If the Cell Viewer is open at any stage of the design flow, you can use the `gui_close_cell_view` command to close it.

Parameters

`-help`	Prints a brief description of the `gui_close_cell_view` command. For a detailed description of the command, use the `man` command: `man gui_close_cell_view`.

Related Information

- `gui_open_cell_view`
- Tools Menu chapter of the *Menu Reference*
  - *Cell Viewer*

gui_deselect

```
gui_deselect
[-help]
{-all  |
-rect {x1 y1 x2 y2} |
-point {x y} |
-line {x1 y1 x2 y2}}
```

Deselects objects as per the parameters you specify. If `-all` is specified, deselects all selected objects.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_deselect` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_deselect`
`-all`	Deselects all objects.
`-line {x1 y1 x2 y2}`	Deselects objects through which the specified line passes.
`-point {x y}`	Deselects objects at the specified point.
`-rect {x1 y1 x2 y2}`	Deselects objects within the specified box coordinates.

Related Information

- deselect_obj
- gui_select
- lineSelect
- select_obj
- windowSelect
- The Main Window chapter in *Menu Reference*
    - *Tool Box Widgets*

gui_dim_foreground

```
gui_dim_foreground
[-help]
-lightness_level {default medium dark}
```

Dims the display area to the specified level. Use this command to view selected and highlighted objects clearly against the dimmed foreground. This command is especially useful when you are debugging.

The equivalent bindkey is F12.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_dim_foreground` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_dim_foreground`.
`-lightness_level {default medium dark}`	
	Specifies the brightness level of the foreground in the display area.

Related Information

- gui_select
- highlight
- View Menu chapter in *Menu Reference*
    - *Dim Background*

gui_dump_picture

```
gui_dump_picture
[-help]
filename
[-format {BMP GIF JPG JPEG PNG PBM PGM PPM XBM XPM}]
[-honor_umask]
[-width integer_value -height integer_value]
[-window string]
```

Saves a snapshot of the layout or specified window with the specified name in the current directory. You can specify the size as well as the format in which you want to save the snapshot. The command also prints out overlay maps with legends.

You can use this command at any stage of the design flow.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_dump_picture` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_dump_picture`

`filename`	Specifies the name of the picture file as a string. You can specify the file extension as well.
	
	Note: The file extension you specify does not control the format of the picture. The picture file is dumped in the PNG format by default or the format you explicitly specify with `-format`. For example, the following command saves the `testShot` picture file in the PNG format:
	`innovus 34> gui_dump_picture testShot.gif`
	
	`current_directory``@file testShot.gif`
	`testShot.gif: PNG image data, 540 x 688, 8-bit/color RGB, non-interlaced`

`-format {BMP GIF JPG JPEG PNG PBM PGM PPM XBM XPM}`	

	Specifies the format of the picture file. *Default*: PNG
`-height` *integer_value*	
	Specifies the height of the picture as an integral value. **Note**: The `-width` parameter is required when `-height` is specified. *Min*: 100 *Max*: 20000
`-honor_umask`	Honors the umask settings while saving the picture. If you specify this option, the permission of the saved picture file will be set by subtracting the umask value from the full permission value. For example, if the full permission setting of a file is `0666` and the umask setting is `0022`, the tool saves the picture file with the permission setting of `0666` - `0022`, that is `0644`.
`-width` *integer_value*	
	Specifies the width of the picture as an integral value. **Note**: The `-height` parameter is required when `-width` is specified. *Min*: 100 *Max*: 20000
`-window` *string*	Specifies the ID of the window that you want to dump. You can use the following IDs: • For Layout Viewer: `designview.win` • For Cell Viewer: `.uiCellView.win` If `-window` is not specified, the layout window is dumped out by default.

Examples

- The following command captures the layout window and saves it in the default PNG format with the name *test2* and the dimensions 1000 x 1000 in the current directory:

  ```
  gui_dump_picture test2 -width 1000 height 1000
  ```

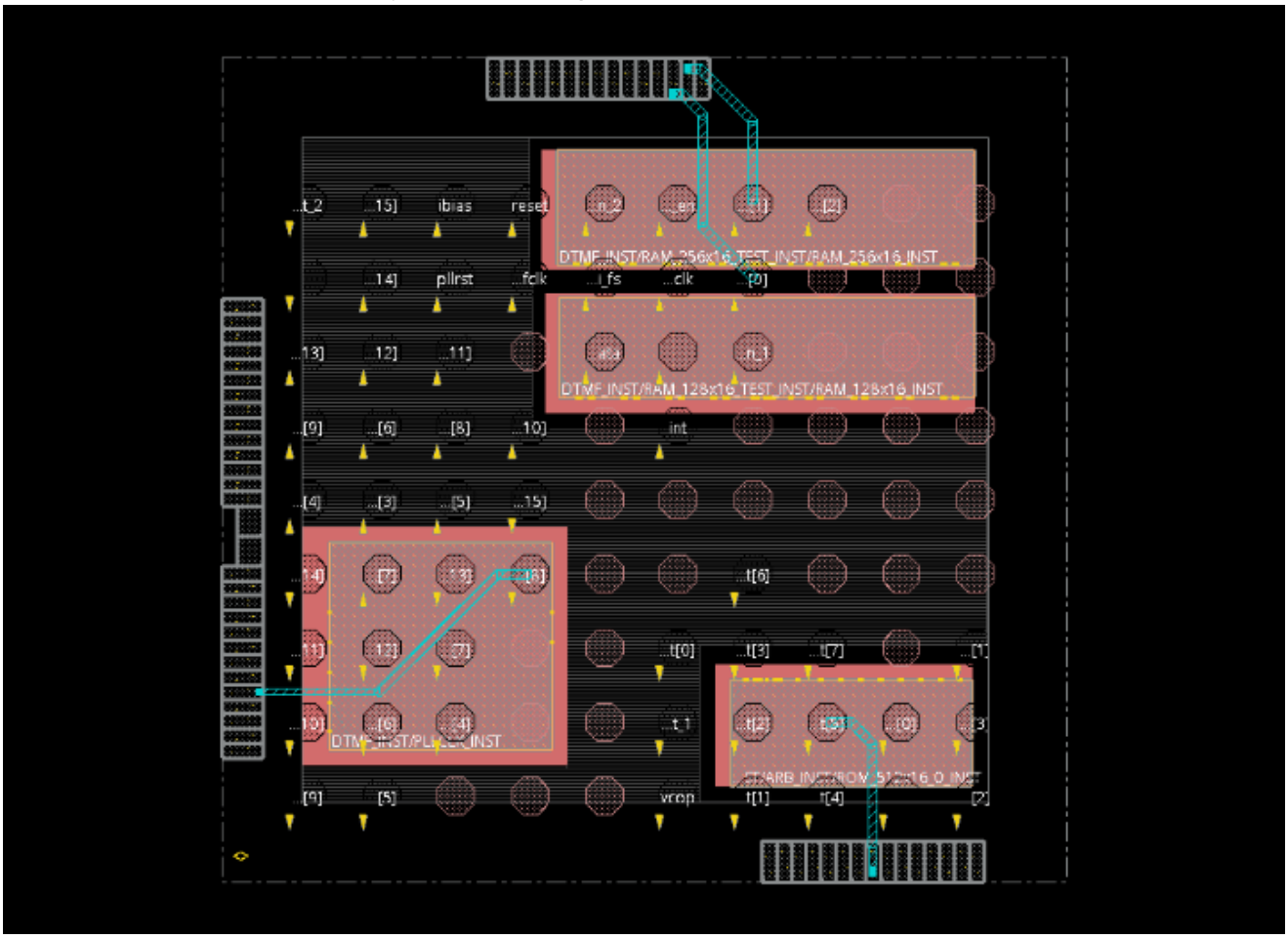
- The following command captures the layout window and saves it in the GIF format with the name *test1.gif* in the current directory:

  ```
  gui_dump_picture -format GIF test1.gif
  ```

- The following command captures the layout viewer window and saves it in PNG format with the name testShot.png in the current directory:

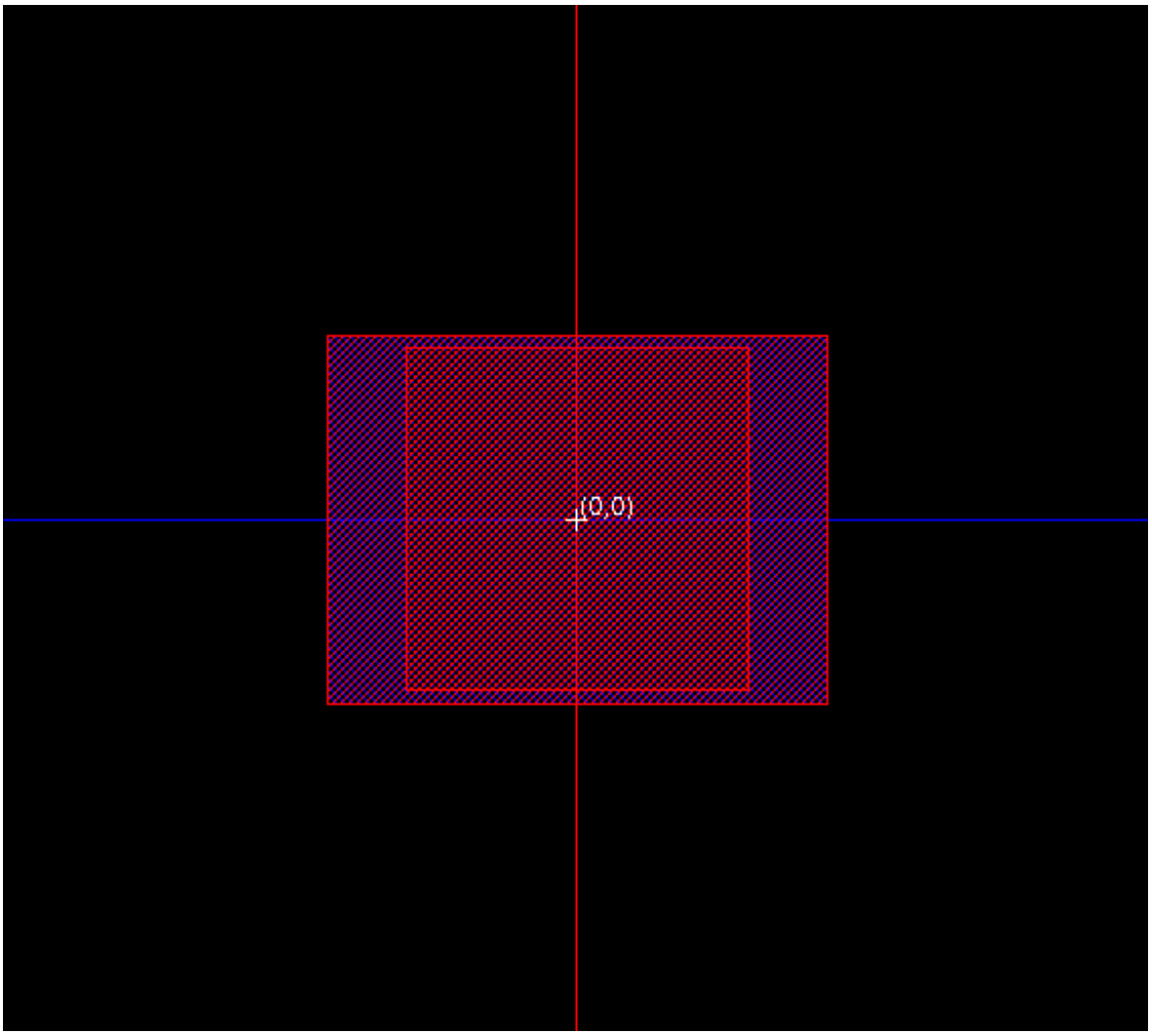
  ```
  gui_dump_picture testShot.png -window designview.win
  ```

  Note: You must have the layout viewer open.



- The following command captures the view in the cell viewer window and saves it in PNG format with the name testCell.png in the current directory:

```
gui_dump_picture testCell.png -window .uiCellView.win
```



Related Information

- createSnapshot
- dumpPictures
- dumpToGIF

gui_group_hinst

```
gui_group_hinst
[-help]
[-type {flex_model} | hname]
```

Moves one level up the hierarchy for the specified module guide and displays the next module. The `gui_group_hinst` command is equivalent to the *Group* widget on the toolbar.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_group_hinst` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_group_hinst`
`hname`	Specifies the name of the hierarchical instance to be grouped.
`-type {`*`flex_model`*`}`	Specifies the type of objects to be grouped. At present, only the `flex_model` type is supported.

Examples

- The following command goes one level up the specified module:
  ```
  gui_group_hinst DTMF_INST/TDSP_CORE_INST
  ```
- The following command groups all flexModels, including flex_module and flex_instgroup, up to the partition or instance group level.
  ```
  gui_group_hinst -type flex_model
  ```

Related Information

- gui_ungroup_hinst
- The Main Window chapter in *Menu Reference*
    - *Toolbar Widgets*

gui_open_cell_view

```
gui_open_cell_view [-help]
```

Opens the Cell Viewer. The Cell Viewer provides a convenient way to view the content of a cell, cell view, or partition without having to zoom into the design. It provides an artwork window for viewing the following objects:

- any LEF library cell
- any via cell
- any OpenAccess database cell view
- abstract view of any partition in the design
- power grid view

You can use the `gui_open_cell_view` command at any stage of the design flow.

Parameters

`-help`	Prints a brief description of the `gui_open_cell_view` command. For a detailed description of the command, use the `man` command: `man gui_open_cell_view`.

Related Information

- `gui_close_cell_view`
- Tools Menu chapter of the *Menu Reference*
  - *Cell Viewer*

gui_report_trace_flightline

```
gui_report_trace_flightline
[-help]
-file report_name
```

Saves information about the all flightlines traced through the `gui_trace_flightline` command to the specified file.

Parameters

`-help`	Prints a brief description of the `gui_report_trace_flightline` command. For a detailed description of the command, use the `man` command: `man gui_report_trace_flightline`.
`-file` *report_name*	Specifies the name of the report file.

Example

The following set of commands reports all traced flightlines:

```
innovus 3> gui_trace_flightline -inst  DTMF_INST/DMA_INST/present_state_reg_3 -sink -
sink_level 2 -sink_color pink
innovus 4> gui_trace_flightline -inst DTMF_INST/RESULTS_CONV_INST/r852_reg_5 -sink -
sink_level 2 -sink_color green
innovus 5> gui_report_trace_flightline -file file1
```

```
The tool saves the report as follows:
```

```
#######################################################
# Generated by: Cadence Innovus 18.11-e070_1
# OS: Linux x86_64(Host ID noi-leenap1)
# Generated on: Wed Jul 25 11:51:06 2018
# Design: DTMF_CHIP
# Command: gui_report_trace_flightline
#######################################################


Path0
```

```
Start: DTMF_INST/DMA_INST/present_state_reg_3/Q End:
DTMF_INST/DMA_INST/top_buf_flag_reg/SI
Start: DTMF_INST/DMA_INST/top_buf_flag_reg/Q End: DTMF_INST/TDSP_DS_CS_INST/i_4386/A

Path1
Start: DTMF_INST/DMA_INST/present_state_reg_3/Q End:
DTMF_INST/DMA_INST/top_buf_flag_reg/SI
Start: DTMF_INST/DMA_INST/top_buf_flag_reg/Q End:
DTMF_INST/TDSP_CORE_INST/TDSP_CORE_MACH_INST/samp_bio_reg/D

Path2
Start: DTMF_INST/DMA_INST/present_state_reg_3/Q End:
DTMF_INST/DMA_INST/top_buf_flag_reg/SI
Start: DTMF_INST/DMA_INST/top_buf_flag_reg/Q End: DTMF_INST/DMA_INST/read_spi_reg/SI

....

Path18
Start: DTMF_INST/RESULTS_CONV_INST/r852_reg_5/Q End:
DTMF_INST/RESULTS_CONV_INST/i_1901/B0
Start: DTMF_INST/RESULTS_CONV_INST/i_1901/Y End: DTMF_INST/RESULTS_CONV_INST/i_7337/A1

Path19
Start: DTMF_INST/RESULTS_CONV_INST/r852_reg_5/Q End:
DTMF_INST/RESULTS_CONV_INST/r852_reg_6/SI
Start: DTMF_INST/RESULTS_CONV_INST/r852_reg_6/Q End:
DTMF_INST/RESULTS_CONV_INST/i_1894/B0

Path20
Start: DTMF_INST/RESULTS_CONV_INST/r852_reg_5/Q End:
DTMF_INST/RESULTS_CONV_INST/r852_reg_6/SI
Start: DTMF_INST/RESULTS_CONV_INST/r852_reg_6/Q End:
DTMF_INST/RESULTS_CONV_INST/r852_reg_7/SI

...

Path39
Start: DTMF_INST/RESULTS_CONV_INST/r852_reg_5/Q End:
DTMF_INST/RESULTS_CONV_INST/i_198/AN
Start: DTMF_INST/RESULTS_CONV_INST/i_198/Y End: DTMF_INST/RESULTS_CONV_INST/i_9316/A

Path40
Start: DTMF_INST/RESULTS_CONV_INST/r852_reg_5/Q End:
DTMF_INST/RESULTS_CONV_INST/i_198/AN
Start: DTMF_INST/RESULTS_CONV_INST/i_198/Y End: DTMF_INST/RESULTS_CONV_INST/i_1455/B0
```

Related Information

- gui_clear_trace_flightline
- gui_save_flightline
- gui_trace_flightline

gui_save_flightline

```
gui_save_flightline
[-help]
-file file_name
[-netWindow]
```

Saves the flightline information of the selected instance to the specified file.

Parameters

`-help`	Prints a brief description of the `gui_save_flightline` command. For a detailed description of the command, use the `man` command: `man gui_save_flightline`.
`-file file_name`	Specifies the name of the file in which the flightline information will be saved.
`-netWindow`	Saves the contents of the Flightline Net Window.

Example

The following command saves the flightline information to a file named `flight1`:

```
gui_save_flightline -file flight1
```

Here's a sample report:

```
#######################################################
# Generated by: Cadence Innovus 18.10-d203_1
# OS: Linux x86_64(Host ID noi-leenap)
# Generated on: Fri Jan 12 09:39:35 2018
# Design: DTMF_CHIP
# Command: gui_save_flightline
#######################################################


DTMF_INST/PLLCLK_INST IOPADS_INST/Pvcopop 1
DTMF_INST/PLLCLK_INST IOPADS_INST/Pvcomop 1
DTMF_INST/PLLCLK_INST IOPADS_INST/Prefclkip 1
DTMF_INST/PLLCLK_INST IOPADS_INST/Ppllrstip 1
```

```
DTMF_INST/PLLCLK_INST IOPADS_INST/Pibiasip 1
```

Related Information

- gui_clear_trace_flightline
- gui_report_trace_flightline
- gui_trace_flightline

gui_schematic_load_view

```
gui_schematic_load_view
[-help]
-file file_name
[-name name]
```

Loads the specified schematic file.

⚠ This commands work only in the cone view (flattened mode).

Parameters

`-help`	Prints a brief description of the `gui_schematic_load_view` command. For a detailed description of the command, use the `man` command: `man gui_schematic_load_view`.
`-file` *`file_name`*	Specifies the name of the schematic file to be loaded.
`-name` *`name`*	Specifies a unique window name for the opened schematic view.

Related Information

- `gui_schematic_save_view`

gui_schematic_save_view

```
gui_schematic_save_view
[-help]
-file file_name
[-name string]
```

Saves the current schematic view into the specified file.

⚠ This commands work only in the cone view (flattened mode).

Parameters

`-help`	Prints a brief description of the `gui_schematic_save_view` command. For a detailed description of the command, use the `man` command: `man gui_schematic_save_view`.
`-file` *`file_name`*	Specifies the name of the file in which to save the schematic view.
`-name` *`string`*	Specifies the name of the schematic view.

Related Information

- `gui_schematic_load_view`

gui_select

```
gui_select
[-help]
{-point {x y} |
-line {x1 y1 x2 y2} |
-rect {x1 y1 x2 y2}}
[-toggle  | -append  | -next]
[-point {x y} [-next ]]
```

Selects objects as per the parameters you specify. Use `-point` to select the object at the specified coordinate, `-line` to select the objects through which the specified line passes, or `-rect` to select the objects falling within the specified box coordinates.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_select` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_select`
`-append`	Appends to the existing selection. This is equivalent to pressing the `Shift` key while selecting an object using the GUI.
`-line {x1 y1 x2 y2}`	Selects by line. Use this parameter to select the objects through which the specified line passes.
`-next`	Steps to next object under point. This is same as using space bar while selecting an object using the GUI.
`-point {x y}`	Selects the object at the specified point.
`-rect {x1 y1 x2 y2}`	Selects by box. Use this parameter to select the objects falling within the specified rectangular area.
`-toggle`	Toggles the selection. This is equivalent to pressing the `Ctrl` key while selecting an object using the GUI.

Examples

- The following command selects the objects that are totally enclosed within the box defined by the coordinates (`x = 10, y = 200`) and (`x = 100, y = 50`):
  ```
  gui_select -rect {10 200 100 50}
  ```
- The following command selects all the objects falling across the specified line coordinates:
  ```
  gui_select -line {168.654 994.049 688.129 436.224}
  ```
- The following command selects the objects crossed by the specified line and appends them to the current selection:
  ```
  gui_select -line {169.622 996.049 698.290 446.240} -append
  ```
- The following command toggles the selection on the objects in the specified bounding box. If the objects are previously selected, the command deselects them and vice versa:
  ```
  gui_select -rect {10 200 100 50} -toggle
  ```

Related Information

- `deselect_obj`
- `gui_deselect`
- `lineSelect`
- `select_obj`
- `windowSelect`
- The Main Window chapter in *Menu Reference*
  - *Tool Box Widgets*

gui_show_edge_number

```
gui_show_edge_number [-help]
[-objects string]
[-reset]
```

Displays edge numbers on the sides of the selected or specified objects in the GUI. Currently, you can display edge numbers on partitions and blocks. This feature is useful if you need to indicate the partition or block edge number in your script.

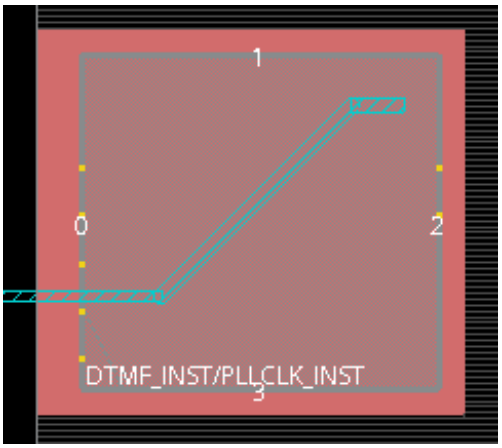
Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_show_edge_number` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_show_edge_number`
`-objects` *`string`*	Specifies the name of the object for which the edge numbers should be displayed. If you want to specify multiple object names, enclose them in braces or double quotation marks. Currently, you can display the edge numbers for partitions and blocks.
`-reset`	Clears the edge numbers from the specified or selected objects on the GUI. If you want to clear the edge numbers for all objects, use the `-reset` parameter with `-objects *`.

Examples

- The following command displays the edge numbers on the `DTMF_INST/PLLCLK_INST` block in the GUI:

  ```
  gui_show_edge_number -objects DTMF_INST/PLLCLK_INST
  ```



- The following command clears the edge numbers from all blocks:

  ```
  gui_show_edge_number -objects * -reset
  ```

Related Information

- `gui_select`
- `lineSelect`
- `windowSelect`

gui_trace_flightline

```
gui_trace_flightline
[-help]
-inst instance_name
[-max_inout_number value]
{[-sink  -sink_level value -sink_color string] | [-source  -source_level value -
source_color string]}
```

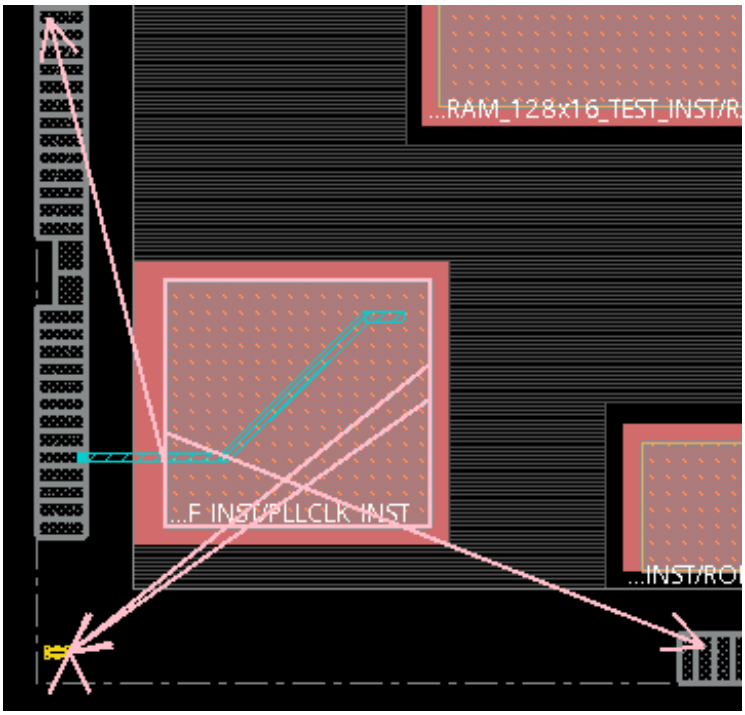
Traces flightlines for the specified instance from either the output pins (with `-sink`) or the input pins (with `-source`). The flightlines drawn with `gui_trace_flightline` are not dependent on instance selection. These flightlines are cleared only when you use `gui_clear_trace_flightline`.

Parameters

`-help`	Prints a brief description of the `gui_trace_flightline` command. For a detailed description of the command, use the `man` command: `man gui_trace_flightline`.
`-inst` *`instance_name`*	Specifies the name of the instance from which the flightline starts
`-max_inout_number` *`value`*	Specifies the maximum number of net terms. If the net has more terms than the specified number, the tool skips this net while tracing the flightline. *Default*: 50
`-sink`	Traces flightlines from the output pins of the specified instance. You must also specify `-sink_color` and `-sink_level` with this option. You cannot use this option with any of the `-source*` options.
`-sink_color` *`string`*	Specifies the color of the sink trace.
`-sink_level` *`value`*	Specifies the sink trace level.
`-source`	Traces flightlines from the input pins of the specified instance. You must also specify `-source_color` and `-source_level` with this option. You cannot use this option with any of the `-sink*` options.
`-source_color` *`string`*	Specifies the color of the source trace.
`-source_level` *`value`*	Specifies the source trace level.

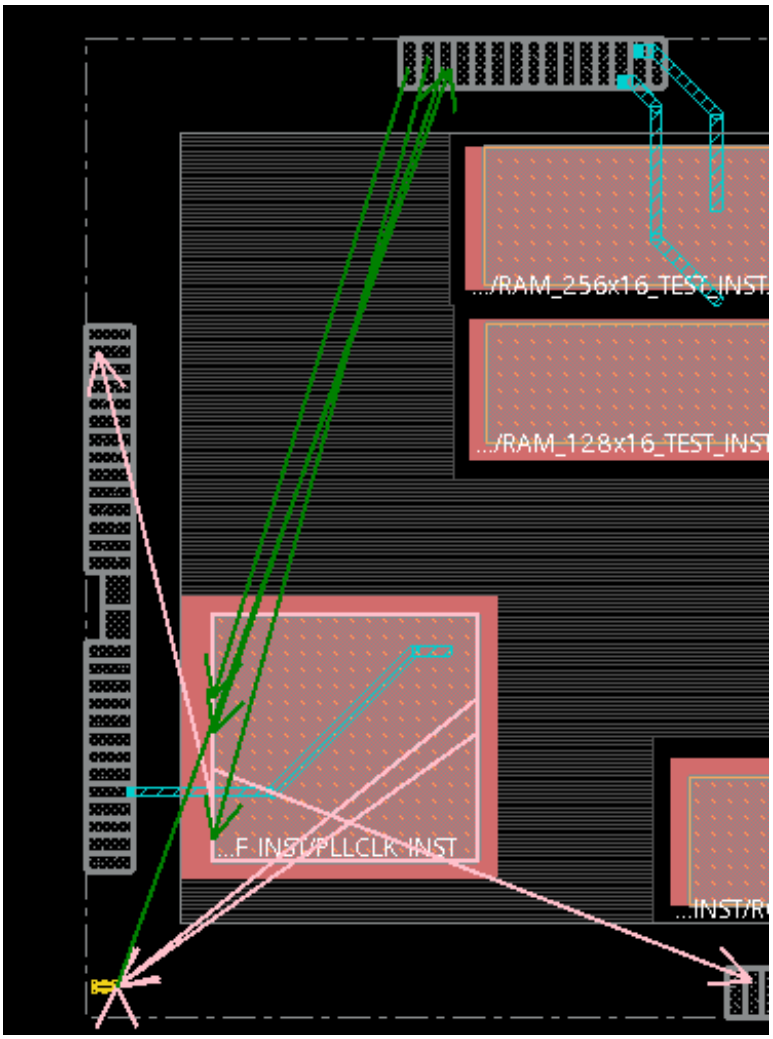
Examples

- The following command traces flightlines for the `DTMF_INST/PLLCLK_INST` instance from the output pins for `2` logic levels in the color `pink`:
  ```
  gui_trace_flightline -inst DTMF_INST/PLLCLK_INST -sink -sink_level 2 -sink_color pink
  ```



- The following command traces flightlines for the `DTMF_INST/PLLCLK_INST` instance from the input pins for `2` logic levels in the color `green`:

  ```
  gui_trace_flightline -inst DTMF_INST/PLLCLK_INST -source -source_level 2 -
  source_color green
  ```



Related Information

- gui_clear_trace_flightline
- gui_report_trace_flightline
- gui_save_flightline

gui_ungroup_hinst

```
gui_ungroup_hinst
[-help]
[-display_all_constraints]
[-type {flex_model} | hname]
```

Displays the submodules for the specified module guide. The `gui_ungroup_hinst` command is equivalent to the *Ungroup* widget on the toolbar.

If you try to ungroup a module that contains only standard cells and marcos but does not contain any submodules, the tool ignores the ungroup request and issues the following warning:

```
**WARN: (ENCSYT-3162): Cannot ungroup Module 'DTMF_INST/RAM_128x16_TEST_INST' because
it has no child.
```

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_ungroup_hinst` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_ungroup_hinst`
`-display_all_constraints`	Displays all constraints, such as fences, guides, regions, and soft guides, regardless of the current hierarchy. When this option is specified, you can view the child constraints.
hname	Specifies the name of the hierarchical instance (hinst) to be ungrouped.
`-type {`*flex_model*`}`	Specifies the object type to be ungrouped. At present, only the `flex_model` type is supported. If you specify `-type {`*flex_model*`}`, the command ungroups all hierarchical instances or parent hinst down to the flex_module and/or flex_instgroup level.

Examples

- The following command ungroups the specified module and displays its submodules.
  ```
  gui_ungroup_hinst DTMF_INST/TDSP_CORE_INST
  ```
- The following command ungroups all hierarchical instances or parent hinst down to the flex_module and/or flex_instgroup level:
  ```
  gui_ungroup_hinst -type {flex_model}
  ```

Related Information

- `gui_group_hinst`
- The Main Window chapter in *Menu Reference*
  - *Toolbar Widgets*

highlight

```
highlight
[-help]
[<any_object>+]
[-index indexValue | -auto_color  | -original]
[-color colorValue | -auto_color  | -original]
[-pattern patternValue | -original]
```

Highlights selected or specified objects with the Highlight Set available through the *View - Highlight Selected* menu. The value for the index corresponds to the Highlight Set that will be applied. For example, a value of 4 for the index specifies that all selected objects will be highlighted with the same color and pattern as available for the *Highlight Set #4* in the *View - Highlight Selected* menu. You can use this command at any stage in the design flow.

You can also provide a specific color and/or pattern to be used. All selected objects will be highlighted with the color and/or pattern that you specify.

If you specify a color and or pattern, the Highlight Set corresponding to the index value will be updated to use the color and pattern specified. For example, if you specify an index value of 5, color `green`, and pattern `backslash`, the *Highlight Set #5* will get updated to have the color `green` and the pattern `backslash`.

You can dehighlight the highlighted objects through the `dehighlight` command or through the *View - Clear Highlight* menu.

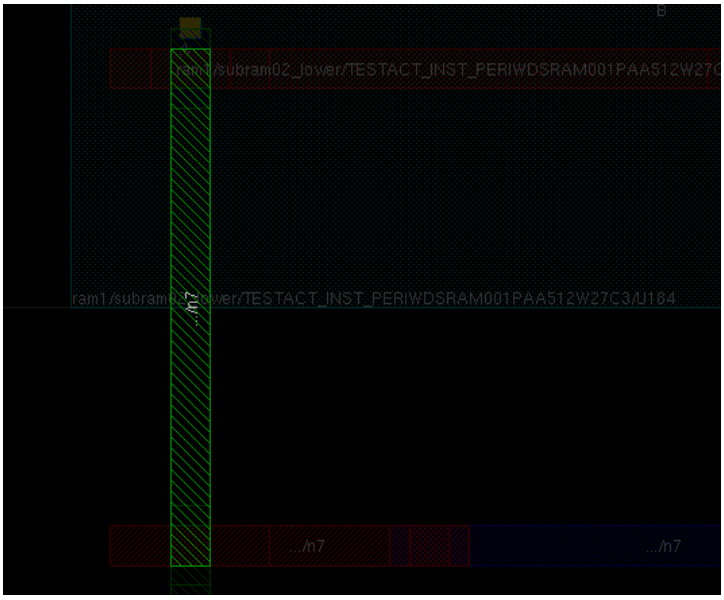
Note: If an object is selectable, you can highlight it using the `highlight` command.

Use the `highlight_pin` command to highlight shapes for specified SIGNAL/POWER pins.

Parameters

`-help`	Prints a brief description that includes type and default information for each `highlight` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man highlight`
any_object+	Specifies object pointer list for highlighting.

`-auto_color`	Specifies the object color automatically. When this parameter is specified, the selected object is highlighted with a different color each time you call the `highlight` command. **Note**: This parameter cannot be used with `-index` or `-color`.
`-index` *`indexValue`*	Specifies the index value for the specified object. All objects that are selected will be highlighted with the same color and pattern as defined in corresponding Highlight Set in the *View - Highlight Selected* menu As an example, if you specify a value of 3 for index, all objects that are selected will be highlighted with the same color and pattern as available for the *Highlight Set #3* in the *View - Highlight Selected* menu. Note that the maximum value is related to the *Number of Highlight Color* setting in the Edit Highlight Color form. If this value is 64, the maximum acceptable value for the `-index` option is 64. If set to 256, the maximum acceptable value for `-index` is 256. The minimum value is `1`.
`-color` *`colorValue`*	Specifies the color value for the specified object. The color value is the same as that available through the Edit Highlight Color form (*View - Edit Highlight Color*). You can also use the Red Green Blue (RGB) value in the following format: #*`V1V2V3`* where V1, V2, and V3 are the hexadecimal values for the red, green, and blue color components respectively. For example, you can specify the value `#CC66FF`. *Default:* The color defined for the Highlight Set corresponding to the index value is used.

`-original`	Highlights the object in original color. This option is useful if you want to check a specific object more closely with the background dimmed. An object highlighted in original color will stand out in the dimmed background:  **Note**: This parameter cannot be used with `-index`, `-color`, or `-pattern`.
`-pattern` *`patternValue`*	Specifies the pattern value for the specified object. The pattern value is the same as the Stipple Value available through the Edit Highlight Color form. *Default:* The pattern (Stipple Value) for the Highlight Set corresponding to the index value is used.

Examples

- The following command highlights all selected objects with the same color and pattern as defined for the *Highlight Set #4* in the *View - Highlight Selected* menu.

  ```
  highlight -index 4
  ```

- The following commands highlight the boxes named `fuu` and `bar` with the same color and pattern as defined for the *Highlight Set #4* in the *View - Highlight Selected* menu.

  ```
  highlight [ list [ dbget -p1 top.insts.name fuu ] [ dbget -p1 top.insts.name bar ] ] -index 4
  ```

  or

  ```
  highlight " [ dbget -p1 top.insts.name fuu ] [ dbget -p1 top.insts.name bar ] " -index 4
  ```

- The following command highlights just the box named `fuu` with the same color and pattern as defined for the *Highlight Set #4* in the *View - Highlight Selected* menu.

  ```
  highlight [ dbget -p1 top.insts.name fuu ] -index 4
  ```

- The following command highlights all selected objects with the color `green` and the pattern `dot4`. The index values is set to 5 and, therefore, the command will also change the definition of the *Highlight Set #5* (*View - Highlight Selected* menu) to have color `green` and pattern `dot4`.

  ```
  highlight -index 5 -color green -pattern dot4
  ```

- The following command highlights all selected objects with the color corresponding to the RGB hexadecimal value `#33FF66` and the pattern `horizontal`. The index values is set to 3 and, therefore, the command will also change the definition of the *Highlight Set #5* (*View - Highlight Selected* menu) to have color `#33FF66` and pattern `horizontal`.

  ```
  highlight -index 3 -color #33FF66 -pattern horizontal
  ```

Related Information

- `dehighlight`
- `highlight_pin`
- View Menu chapter of the *Menu Reference*:
  - *Highlight Selected*
  - *Clear Highlight*
  - *Edit Highlight Color*

highlight_pin

```
highlight_pin
[-help]
-inst string
-pin string
[-original | -index indexValue ]
[-original | -color colorValue ]
[-original | -pattern patternValue ]
```

Highlights the shapes of all specified SIGNAL/POWER instance pins with the Highlight Set available through the *Edit - Highlight Selected* menu. The value for the index corresponds to the Highlight Set that will be applied. For example, a value of 4 for the index specifies that all selected objects will be highlighted with the same color and pattern as available for the *Highlight Set #4* in the *Edit - Highlight Selected* menu.

You can also provide a specific color and/or pattern to be used. All specified pins will be highlighted with the color and/or pattern that you specify.

Note: If you specify a color and or pattern, the Highlight Set corresponding to the index value will be updated to use the color and pattern specified. For example, if you specify an index value of `5`, color `green`, and pattern `backslash`, the *Highlight Set #5* will get updated to have the color `green` and the pattern `backslash`.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `highlight_pin` parameter. For a detailed description of the command and all of its parameters, use the man command: `man highlight_pin`

`-color` *colorValue*	Specifies the color value for the specified object. The color value is the same as that available through the Edit Highlight Color form, which can be accessed by selecting *Edit - Edit Highlight Color*. You can also use the Red Green Blue (RGB) value in the following format: `#`*V1V2V3* where *V1*, *V2*, and *V3* are the hexadecimal values for the Red, Green, and Blue color components, respectively. For example, you can specify the value `#CC66FF`. *Default:* The color defined for the Highlight Set corresponding to the specified index value is used.
`-index` *indexValue*	Specifies the index value for the specified object. By default, all specified pins are highlighted with the same color and pattern as defined in the *Highlight Set* corresponding to the specified index value in the *Edit - Highlight Selected* submenu. For example, if you specify a value of 3 for index, all specified pins are highlighted with the same color and pattern as available for *Highlight Set #3* in the *Edit - Highlight Selected* submenu. **Note**: The *Edit - Highlight Selected* submenu provides eight highlight patterns. Therefore, the value of the index should be a number from 1 to 8.
`-inst` *string*	Specifies the name of instance.
`-original`	Highlights the instance pin in original color. This option is useful if you want to check a specific instance pin more closely with the background dimmed. An object highlighted in original color will stand out in the dimmed background. **Note**: This parameter cannot be used with `-index`, `-color`, or `-pattern`.
`-pattern` *patternValue*	Specifies the pattern value for the specified object. The pattern value is the same as the *Stipple* value available through the Edit Highlight Color form. *Default:* The pattern (*Stipple* value) defined for the Highlight Set corresponding to the specified index value is used.
`-pin` *string*	Specifies the name of SIGNAL/POWER pin to be highlighted. You can specify more than one pin name.

Examples

- The following command highlights the SIGNAL pins `E` and `D` of instance `U1` using the color yellow and the solid pattern:

  ```
  highlight_pin -inst U1 -pin {E D} -color yellow -pattern solid -index 1
  ```

- The following command highlights the SIGNAL pin `E` and the POWER pin `gnd` of instance `U1` using the color yellow and the solid pattern:

  ```
  highlight_pin -inst U1 -pin {E gnd} -color yellow -pattern solid -index 1
  ```

Related Information

- `dehighlight`
- `highlight`
- `highlight_pin_connection`
- View Menu chapter in the *Innovus Menu Reference*:
  - *Highlight Selected*
  - *Clear Highlight*
  - *Edit Highlight Color*

highlight_pin_connection

```
 highlight_pin_connection
[-help]
-from_pin pin
[-mode {wire_segment | whole_net | flight_line}]
-to_pin pin
[-with_arrow | -select_only]
[-net_color_index index_value | -select_only]
[-inst_color_index index_value | -select_only]
```

Highlights the full timing path or its segment in the text format while generating the sign-off timing report. Use this command to quickly debug the timing path (timing point information) used in the tool.

Parameters

`-help`	Prints a brief description that includes type and default information for each `highlight_pin_connection` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man highlight_pin_connection`
`-from_pin` *`pin`*	Specifies the name of the starting pin or IO port to be highlighted.
`-inst_color_index` *`index_value`*	Specifies the instance color index value (an integer value from 1 to 64) for the specified timing path.
`-mode {wire_segment | whole_net | flight_line}`	Specifies the draw mode between the starting and ending pins.
`-net_color_index` *`index_value`*	Specifies the net color index value (an integer value from 1 to 64) for the specified timing path.
`-select_only`	Selects only the specified path for highlighting.
`-to_pin` *`pin`*	Specifies the name of the ending pin or IO port to be highlighted.
`-with_arrow`	Specifies whether or not the connection is defined with an arrow.

Example

The following command highlights a line drawn from the specified input pin to output pin while generating the sign-off timing report:

```
highlight_pin_connection -from_pin ffa/Q -U7/A -mode flight_line
```

The timing report generated is shown below:

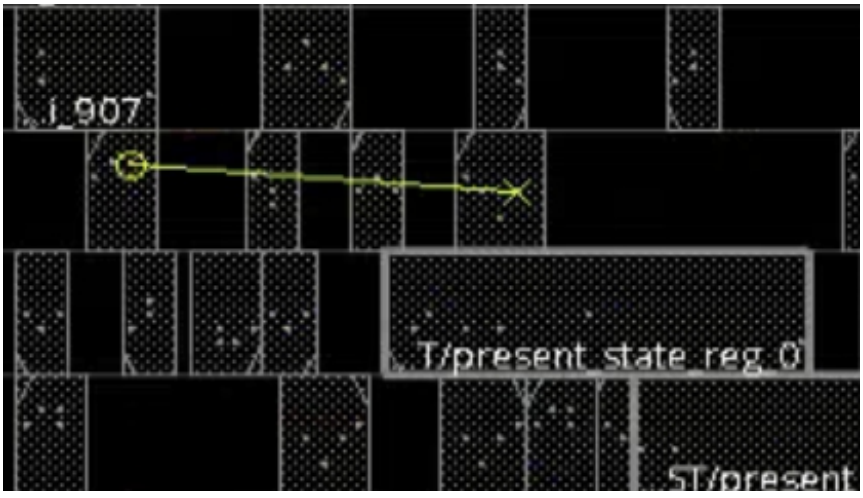
:::::::Timing Report::::::::::::

```
Startpoint: ffa (rising edge-triggered flip-flop clocked by CLK)
Endpoint: ffd (rising edge-triggered flip-flop clocked by CLK)
Path Group: CLK
Path Type: max
Point                          Fanout    Cap     Trans       Incr       Path
-----------------------------------------------------------------------------
clock CLK (rise edge)                                         0.00       0.00
clock network delay (propagated)                              4.90       4.90
ffa/CLK (DTC10)                                 0.30          0.10       5.00 r
ffa/Q (DTC10)                                   0.57          1.70       6.70 f
b (net)                          2     3.85
U7/A (IV110)                                    0.57          0.00       6.70 f
U7/Y (IV110)                                    1.32          0.84       7.55 r
QA (net)                        50    32.59
U12/A (NA310)                                   3.02          4.00      11.55 r
U12/Y (NA310)                                   2.47          4.04      15.58 f
n9 (net)                         3     8.87
U17/A (NA211)                                   2.47          0.00      15.58 f
U17/Y (NA211)                                   1.01          1.35      16.94 f
n14 (net)                        2     4.87
U23/A (IV120)                                   1.01          0.00      16.94 f
U23/Y (IV120)                                   0.51          0.37      17.30 r
n13 (net)                        1     2.59
U15/A2 (BF003)                                  0.51          0.00      17.31 r
U15/Y (BF003)                                   0.88          0.81      18.12 f
n12 (net)                        1    15.61
U16/B2 (BF003)                                  3.88          3.00      21.12 f
U16/Y (BF003)                                   2.46          0.99      22.11 r
```

```
  n7 (net)                          1      2.61
  U10/A (AN220)                                    2.46       0.00       22.11 r
  U10/Y (AN220)                                    0.46       1.04       23.15 r
  n15 (net)                         1      2.63
  ffd/D (DTN10)                                    0.46       0.00       23.15 r
  data arrival time                                                      23.15
 clock CLK (rise edge)                                          10.00      10.00
  clock network delay (propagated)                               3.50      13.50
  ffd/CLK (DTN10)                                                          13.50 r
  library setup time                                            -1.33      12.17
  data required time                                                     12.17
  ---------------------------------------------------------------------------------
  data required time                                                     12.17
  data arrival time                                                     -23.15
  ---------------------------------------------------------------------------------
  slack (VIOLATED)                                                      -10.98

:::::::END:::::::::::
```

The drawn layout along with the highlighted line is shown in the below picture:



lineSelect

```
lineSelect
[-help]
mode
ux1 uy1 ux2 uy2
```

Selects all objects that fall across the specified line coordinates. The `lineSelect` command is equivalent to the *Select by Line* check box in the *Selection* page of the Preferences form.

Innovus provides an easy way to toggle between window and line selection modes using the GUI:

1. The default selection mode is window.
2. Press the `Ctrl` key to switch to the line selection mode. With the `Ctrl` key pressed, draw a line across the objects you want to select. All the objects that the line crosses are selected.
3. When you release the `Ctrl` key, the selection mode switches back to window.

Note: This command works only in GUI mode.

Parameters

`-help`	Prints a brief description that includes type and default information for each `lineSelect` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man lineSelect`
mode	Specifies the action to be performed on the objects falling across the specified line coordinates. Choose one of the following modes: • `new`: Selects all objects crossed by the line. This mode is equivalent to the *Select by Line* check box in the *Selection* page of the Preferences form. • `delete`: Deletes all object selections crossed by the line. Use this mode if multiple objects are selected and you want to deselect some of them but retain the other selections. • `append`: Selects objects crossed by the line and appends them to the current selection. • `toggle`: Deselects objects in the bounding box if they are selected; selects objects if they are deselected.
ux1 uy1 ux2 uy2	
	Specifies the start and end points of the line.

Example

The following command selects all objects falling across the specified line coordinates.

```
lineSelect new 168.654 994.049 688.129 436.224
```

Related Information

- `windowSelect`
- `windowToggleSelect`

list_gui_marker

`list_gui_marker`

Gives a list of GUI markers in the design layout.

Parameters

`-help`	Prints a brief description of the `list_gui_marker` command. For a detailed description of the command, use the `man` command: `man list_gui_marker`.

Example

- The following command shows a list of GUI markers:
  `list_gui_marker`
  The following is displayed:
  `{name:aa  coordinate:(100.000 100.000)}`

Related Information

- `add_gui_marker`
- `remove_gui_marker`

loadWorkspace

```
loadWorkspace
[-help]
-name workspaceName
[-dir directory]
```

Loads the named workspace from the specified directory. A workspace contains customized main window settings including:

- Size and placement of window
- Specific orientation of toolbars
- Docking status
- Visibility settings

You can use the `loadWorkspace` command at any stage in the design flow to load a previously saved workspace. However, if you want Innovus to load a specific workspace automatically on starting, you need to first set that workspace as default using the `setDefaultWorkspace` command.

Parameters

`-help`	Prints a brief description that includes type and default information for each `loadWorkspace` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man loadWorkspace`
`-name` *`workspaceName`*	Specifies the name of the workspace to be loaded.
`-dir` *`directory`*	Specifies the directory from which the workspace is to be loaded. The default search order to access directories for loading workspaces is as below: • Current working directory • User home directory The `-dir` parameter is useful if you have workspaces with the same name in both the current working directory and your home directory. If you do not specify `-dir`, the software loads the *`workspaceName`* workspace from the current working directory.

Example

The following example loads the `MyWorkspace` workspace from the current working directory.

```
loadWorkspace -name MyWorkspace
```

Related Information

- `deleteWorkspace`
- `saveWorkspace`
- `setDefaultWorkspace`
- Windows Menu chapter of the *Menu Reference*
  - *Workspaces*

open_schematic

```
open_schematic
[-help]
[-name string]
[-objects <inst | net | instTerm | term | hinst>+ | -selected]
[-type {module cone}]
```

Opens the schematic view of the specified or selected objects.

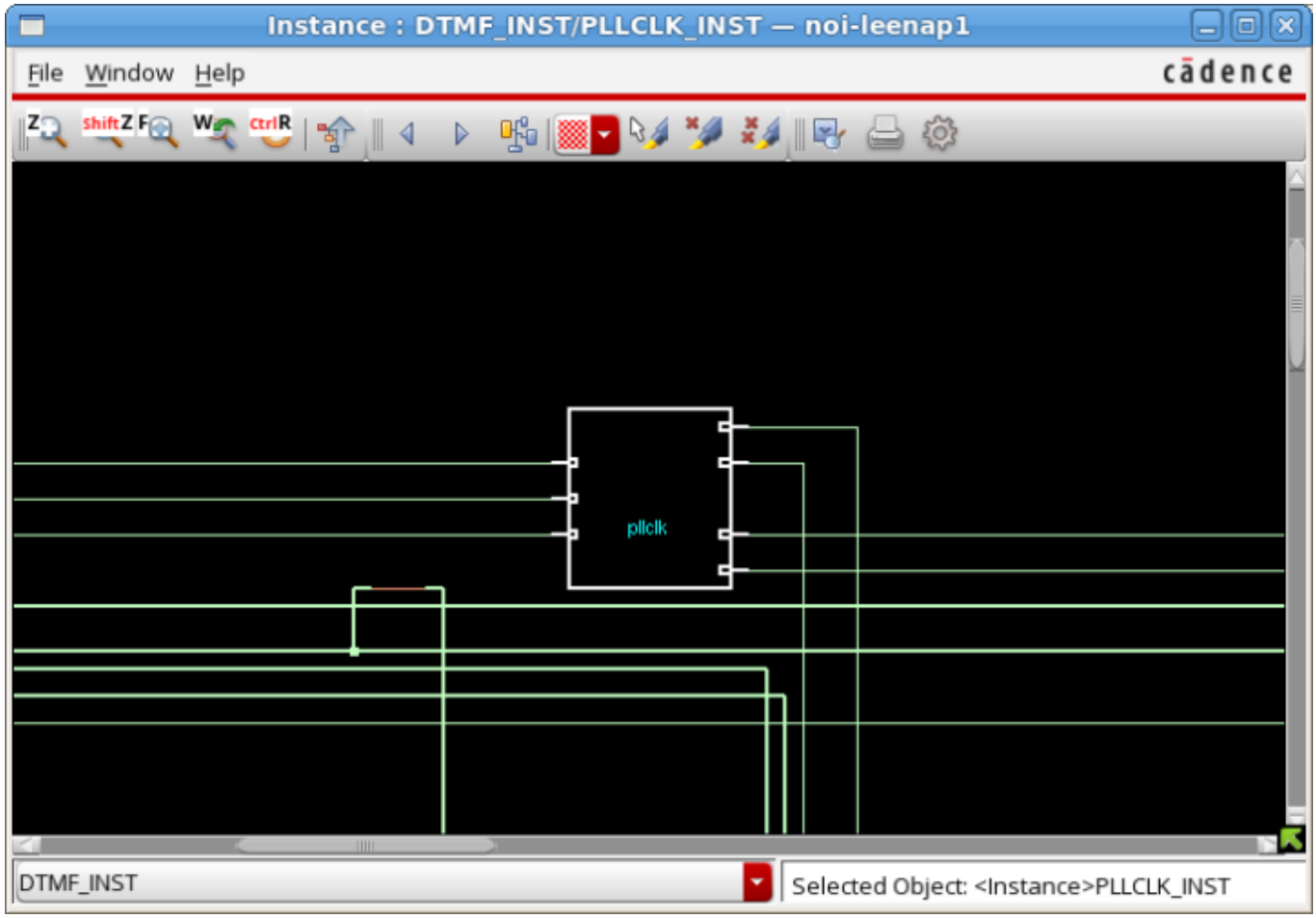
Parameters

`-help`	Prints out the command usage. For a detailed description of the command, use the `man` command: `man open_schematic`
`-name` *string*	Specifies a unique window name for the schematic view to be opened. *Data_type*: string, optional
`-objects <inst | net |` *instTerm* `|` *term* `|` *hinst*`>+`	
	Specifies the object(s) to be added in the schematic view. If multiple objects are specified, the Schematic Viewer can be opened only in the flattened mode (`-type cone`). As the default schematic mode is hierarchical, this means you must always specify `-type cone` when specifying multiple objects. **Note**: This option cannot be used with `-type module`. Data_type: (*inst*|*net*|*instTerm*|*term*|*hinst*)+, optional
`-selected`	Specifies that the selected object(s) are to be added in the schematic view. If multiple objects are selected, the Schematic Viewer can be opened only in the flattened mode (`-type cone`). As the default schematic mode is hierarchical, this means you must always specify `-type cone` when selecting multiple objects. **Note**: This option cannot be used with `-objects` or `-type module`. *Data_type*: bool, optional
`-type {module cone}`	Specifies the type of schematic to be opened. If you specify `cone`, the tool opens the Schematic Viewer to display schematic in the flattened mode. If you specify `module`, the tool opens the Schematic Viewer to display schematic in the hierarchical mode. *Default*: `module` *Data_type*: enum, optional

Examples

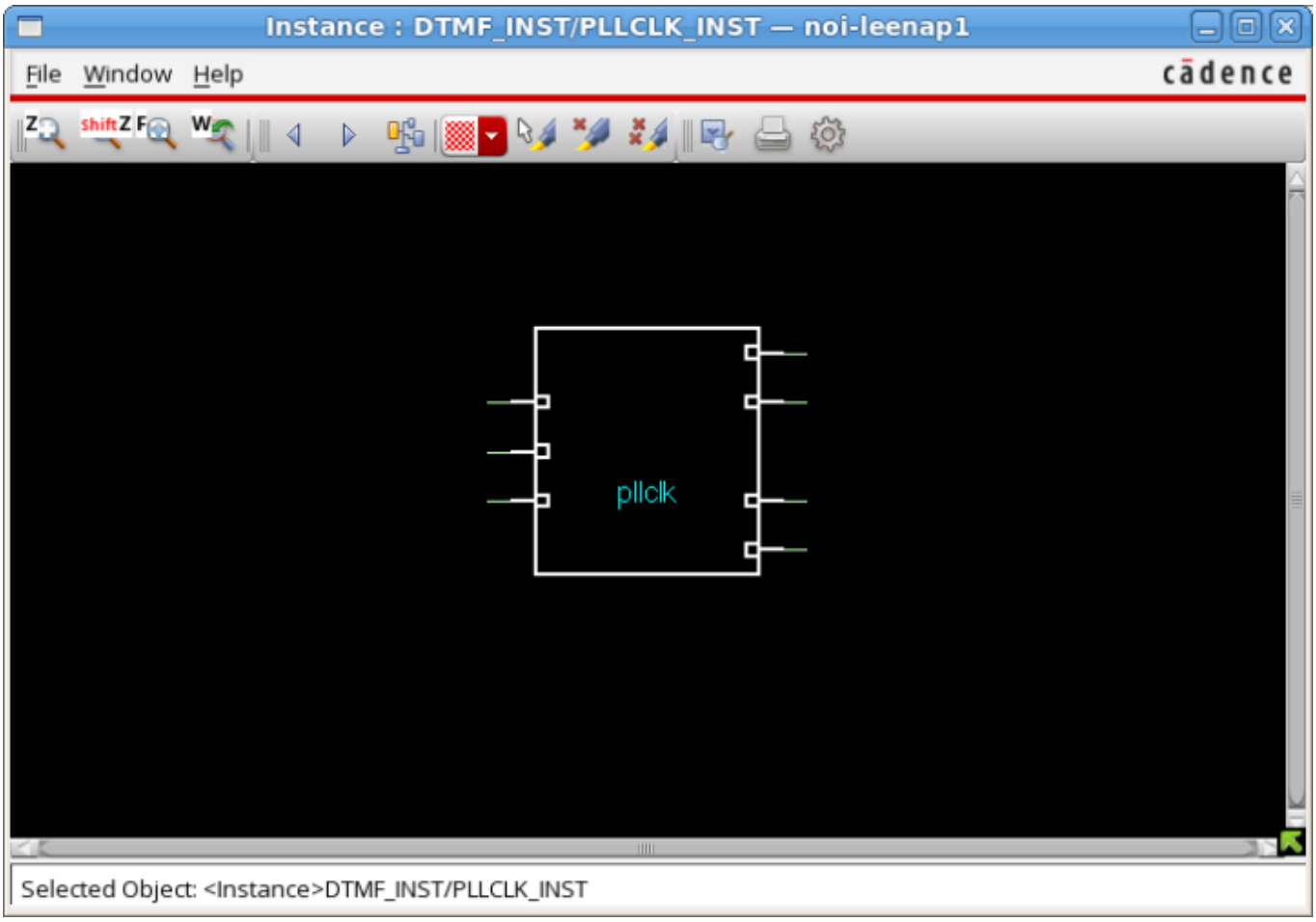
- The following command opens the schematic for the specified instance in the hierarchical mode:

```
open_schematic -objects DTMF_INST/PLLCLK_INST
```



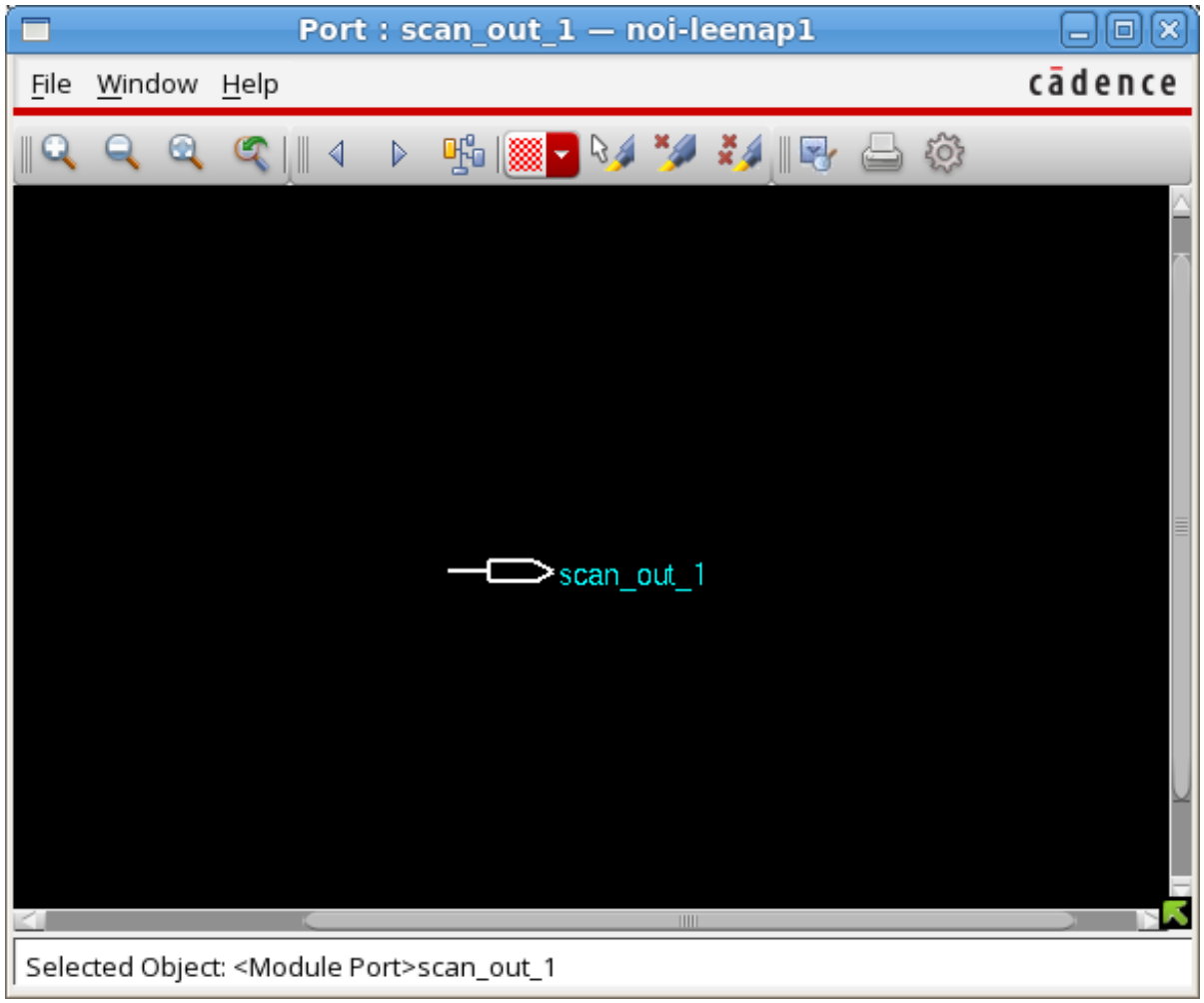
- The following command opens the schematic for the specified instance in the flattened mode:

```
open_schematic -objects DTMF_INST/PLLCLK_INST -type cone
```



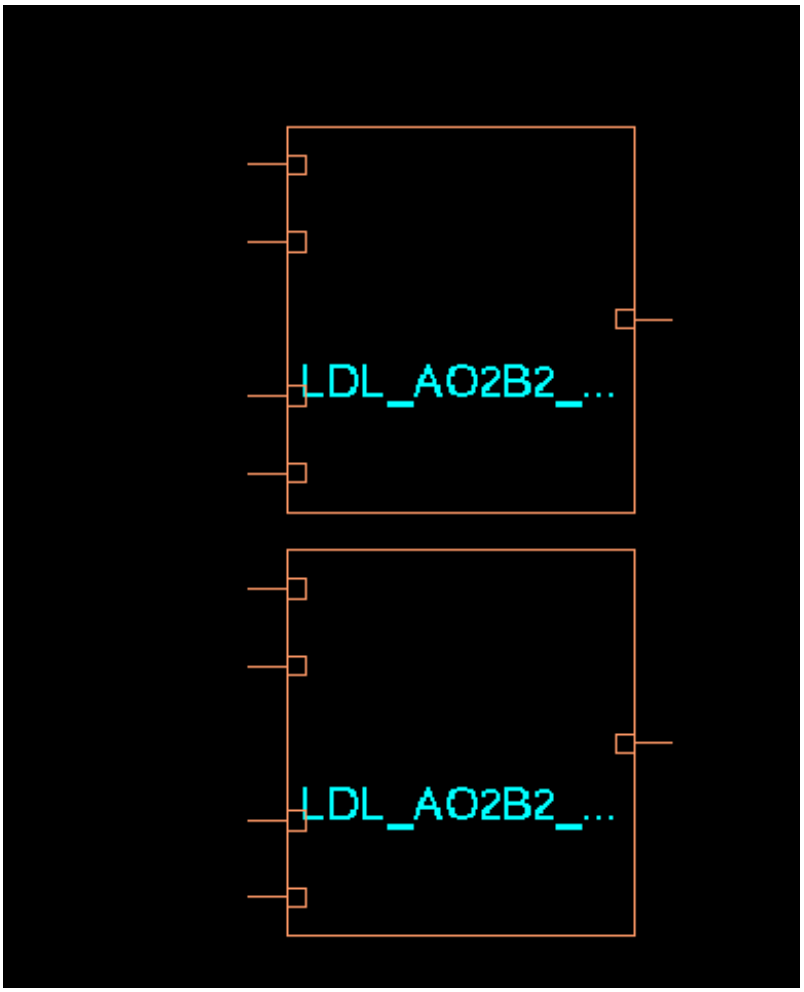
- The following command opens the schematic for the selected object in the flattened mode:

```
open_schematic -selected -type cone
```



- The following command opens the schematic for multiple selected objects in the flattened mode:

```
open_schematic -selected -type cone
```



Related Information

- `gui_schematic_load_view`
- `gui_schematic_save_view`
- Debugging Timing Results chapter in the *User Guide*
  - *Viewing Schematics*

pan

```
pan
[-help]
dx
dy
```

Specifies that the new center of the viewing window should shift from the current center of the viewing window by (*x*, *y*) microns. Use this command at any stage in the design flow to pan or move the design view in a specific direction.

Parameters

`-help`	Prints a brief description that includes type and default information for each `pan` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man pan`.
dx	Specifies that the design center should shift *dx* microns on the x-axis from the center of the viewing window. You can specify positive or negative values for *dx*. A positive value specifies shifting to the right on the x axis.
dy	Specifies that the design center should shift `dy` microns on the y axis values from the center of the viewing window. You can specify positive or negative values for *dy*. A positive value specifies shifting up on the y axis.

Example

The following example shifts the design center to the right by 200 microns and down by 300 microns from the center of the viewing window.

```
pan 200 -300
```

Related Information

- `panCenter`
- `panPage`
- View Menu chapter in the *Menu Reference*
  - *Pan*
- The Main Window chapter in the *Menu Reference*
  - *Mouse Operations*

panCenter

```
panCenter
[-help]
x y
```

Pans in the viewable window to the center point defined by the specified coordinates. Use this command at any stage of the design flow to view the design area around a specific point.

Parameters

`-help`	Prints a brief description that includes type and default information for each `panCenter` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man panCenter`.
`x y`	Specifies the x and y coordinates of the center point.

Example

The following command pans the viewable window to the center point (x=100, y=200)

```
panCenter 100 200
```

Related Information

- `pan`
- `panPage`
- View Menu chapter in the *Menu Reference*
  - *Pan*
- The Main Window chapter in the *Menu Reference*
  - *Mouse Operations*

panPage

```
panPage
[-help]
x y
```

Pans the viewable window in pages defined by the offsets. The size of each page is equal to the size of the current viewable window. Use this command at any stage of the design flow.

Parameters

`-help`	Prints a brief description that includes type and default information for each `panPage` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man panPage`.
`x y`	Specifies `x` and `y` offsets to indicate number of pages to pan. A positive value of *`x`* specifies a pan of *`x`* pages to the right. A positive number of *`y`* specifies a pan of *`y`* pages toward the top. You can specify integer values for *`x`* and *`y`*.

Examples

- The following command pans the display to the right by one page
  ```
  panPage 1 0
  ```
- The following command pans the display to the left by one page
  ```
  panPage -1 0
  ```

Related Information

- `pan`
- `panCenter`
- View Menu chapter in the *Menu Reference*
  - *Pan*
- The Main Window chapter in the *Menu Reference*
  - *Mouse Operations*

read_temperature_map_file

```
  read_temperature_map_file
[-help]
-file <string>
```

Loads the tile-based temperature map file generated by Sigrity Celsius to be used for displaying temperature.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-file <string>`	Specifies the name of the tile-based temperature map file generated by Sigrity Celsius.

redo

```
redo [-help]
```

Returns the design to the state it was in immediately prior to issuing the `undo` command. It can be used only to reinstate power planning, bump placement, and selected floorplanning and wire edit functions that were removed with the `undo` command.

You can use this command after you use the `undo` command for any of the following text commands:

- Selected bump placement commands:
  - `create_bump`
  - `deleteBumps`
- Selected floorplanning commands:
  - `alignObject`
  - `flipOrRotateObject`
  - `setObjFPlanBox`
  - `setObjFPlanBoxList`

  - setObjFPlanPolygon
  - snapFPlan
  - snapFPlanIO
  - spaceObject
- Selected power planning commands:
  - addRing
  - addStripe
  - editPowerVia
  - sroute
- All wire editor commands:
  - editAddFillet
  - editAddPoly
  - editAddRoute
  - editAddTrimMetal
  - editAddVia
  - editChangeLayer
  - editChangeNet
  - editChangeRule
  - editChangeStatus
  - editChangeVia
  - editCommitPoly
  - editCommitRoute
  - editCutWire
  - editDelete
  - editDeleteFillet
  - editDeselect

- editDuplicate
- editFixWideWires
- editMerge
- editMove
- editResize
- editRotate
- editSelect
- editSplit
- editTrim
- prepareForEcoRoute

Parameters

-help	Prints a brief description of the `redo` command. For a detailed description of the command, use the `man` command: `man redo`

Related Information

- undo
- Edit Menu chapter in the *Menu Reference*
  - *Undo*
  - *Redo*

redraw

```
redraw [-help]
```

Refreshes the display in the viewable window. This command performs the same function as the *Redraw* icon (CTRL+R keyboard shortcut). You can use this command at any stage in the design flow.

Parameters

`-help`	Prints a brief description of the `redraw` command. For a detailed description of the command, use the `man` command: `man redraw`.

register_gui_edit_callback

```
register_gui_edit_callback
[-help]
-command string
-edit_type {selection pre-move post-move}
```

Adds the callback function to the specified move action. In GUI, the move action involves the following three steps:

1. Click an object.
2. Move the object to the new location.
3. Click to place the object at the new location and complete the move.

The callback function is implemented based on the move action specified with the `-edit_type` parameter.

Parameters

`-help`	Prints the usage of the command. For a detailed description of the command, use the `man` command: `man register_gui_edit_callback`
`-command string`	Specifies the name of the callback function.
`-edit_type {selection pre-move post-move}`	Specifies the GUI edit function: • `selection` - Implements the callback function when an object is selected (after Step 1). • `pre-move` - Implements the callback function just before the object is placed at the new location (after Step 2, just before you change the coordinates in Step 3). • `post-move` - Implements the callback function after an object is moved (after Step 3, when the tool updates the DB records).

Example

Consider the following proc definition:

```
proc move_event_callback {args} {
     set objList $args
     foreach obj $objList {
           set type [dbGet $obj.objType]
           if {$type == "sdp"} {
                custom_snap_sdp $obj
           } elseif {$type == "inst"} {
                set cellName [dbGet $obj.cell.name]
                if {[regexp grf $cellName]} {
                     custom_snap_grf $obj
                }
           }
}
}
```

The following command adds the `move_event_callback` function to the selected object just before the move is completed:

```
register_gui_edit_callback -edit_type pre_move -command {move_event_callback [dbGet selected]}
```

Related Information

- report_gui_edit_callback
- unregister_gui_edit_callback

remove_gui_marker

```
remove_gui_marker
[-help]
{-name <string> | -all }
```

Removes a marker from the design layout.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-name <string>`	Specifies the name of the marker to be removed from the design layout.
`-all`	Removes all markers from the design layout.

Related Information

- `add_gui_marker`
- `list_gui_marker`

report_gui_edit_callback

```
report_gui_edit_callback
[-help]
[-edit_type {selection pre-move post-move}]
[-file string]
```

Reports the callback functions added to the specified move action. In GUI, the move action involves the following three steps:

1. Click an object.
2. Move the object to the new location.
3. Click to place the object at the new location and complete the move.

The callback function is implemented based on the move action specified with the `-edit_type` parameter.

Note that the `report_gui_edit_callback` command does not report the default callback functions.

Parameters

`-help`	Prints the usage of the command. For a detailed description of the command, use the `man` command: `man report_gui_edit_callback`
`-edit_type {selection pre-move post-move}`	Specifies the GUI edit function: • `selection` - Implements the callback function when an object is selected (after Step 1). • `pre-move` - Implements the callback function just before the object is placed at the new location (after Step 2, just before you change the coordinates in Step 3). • `post-move` - Implements the callback function after an object is moved (after Step 3, when the tool updates the DB records).
`-file string`	Specifies the name of the file.

Example

Consider the following proc definitions:

```
proc move_callback_1 {

     ...

}

proc move_callback_2 {

     ...

}

proc select_callback_1 {

     ...

}
```

- The following commands add the specified callback functions to the selected object based on the selection and on the post-move operations, as specified below:

  ```
  register_gui_edit_callback -edit_type post-move -command move_callback_1

  register_gui_edit_callback -edit_type post-move -command move_callback_2

  register_gui_edit_callback -edit_type selection -command select_callback_1
  ```

- The following command reports all the callback functions on the console:

  ```
  > report_gui_edit_callback

  ################################
  # GUI Callback Function Report
  ################################
  Post Move:
  move_callback_1
  move_callback_2

  Selection:
  select_callback_1
  ```

- The following command prints the callback functions on the post-move operation to the file `cb_move`:

  ```
  report_gui_edit_callback -edit_type post-move -file cb_move
  ```

  The contents of the file is as follows:

```
#########################################################################
#  Generated by:      Cadence Innovus 18.10-d133_1
#  OS:                Linux x86_64(Host ID sjfib212)
#  Generated on:      Sun Oct  8 19:51:35 2017
#  Design:            DTMF_CHIP
#  Command:           report_gui_edit_callback -edit_type post-move -file cb_move
#########################################################################
Post Move:
move_callback_1
move_callback_2
```

Related Commands

- register_gui_edit_callback
- unregister_gui_edit_callback

resetMultiColorsHier

```
resetMultiColorsHier
[-help]
[colorID [hInstName]]
```

Resets the color Id for all or the specified hierarchical instances.

Parameters

`-help`	Prints a brief description that includes type and default information for each `resetMultiColorsHier` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man resetMultiColorsHier`.
`colorID`	Specifies the color Id to which all hierarchical instances or the specified hierarchical instance will be reset. The valid values of color Id are from 0 to 29. The color value for the color id is the same as specified in the Module Color Preference form (accessed by clicking *Floorplan - Edit Floorplan - Color Module*). *Default*: If no color Id and no instance name is specified, all hierarchical instances are displayed with the color id `0`. This is the same behavior as the *Reset* button on the Module Color Preferences form.
`hInstName`	Specifies the name of hierarchical instance that should be displayed in the color corresponding to the specified color Id. *Default*: If no instance is specified, the color Id for all hierarchical instances is reset.

Example

- The following command resets the color Id for all hierarchical instances and their descendants to `3`:

  ```
  resetMultiColorsHier 3
  ```

- The following command resets the color Id for the hierarchical instance *`DTMF_INST`* and all its descendants to the color id 3:

```
resetMultiColorsHier 3 DTMF_INST
```

Related Information

- saveHInstColor
- setMultiColorsHier
- setSelHInstColor
- Floorplan Menu chapter in the *Menu Reference*
  - *Color Module*

saveColorPreference

```
saveColorPreference
[-help]
[-dir <string>]
[-include_customized]
-name <string>
```

Saves color preferences for the Innovus main display as specified in the Color Preferences form.

Parameters

`-help`	Prints a brief description that includes type and default information for each `saveColorPreference` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man saveColorPreference`
`-dir` *string*	Specifies the directory path. By default, the color preference file is saved in the current directory.
`-include_customized`	Includes the settings in the Customize Tab form within the Color Preferences form.
`-name` *string*	Specifies the name of the color preference file.

Related Information

- `setPreference`
- The Main Window chapter of the Menu Reference
  - *Color Preferences*
- View Menu chapter of the Menu Reference
  - *All Colors*

saveHInstColor

```
saveHInstColor
[-help]
fileName
```

Saves the current color setting for a hierarchical instance from the Module Color Preference form to a file. The Module Color Preference form is accessed by clicking *Floorplan - Edit Floorplan - Color Module*.

Parameters

`-help`	Prints a brief description that includes type and default information for each `saveHInstColor` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man saveHInstColor`.
`fileName`	Specifies the name of the file that stores 30 color settings starting with the `setLayerPreference` command. This is the same behavior as the Save button on the Module Color Preference form.

Example

The following command saves the current color setting for a hierarchical instance from the Module Color Preference form to a file named `hinstColorFile`:

```
saveHInstColor hinstColorFile
```

Related Information

- `resetMultiColorsHier`
- `saveColorPreference`
- `setHInstColorId`
- `setMultiColorsHier`
- `set_object_color`
- `setSelHInstColor`
- Floorplan Menu chapter in the *Menu Reference*
    - *Color Module*

saveWorkspace

```
saveWorkspace
[-help]
-name workspaceName
[-dir directory]
```

Saves the named workspace to the specified directory. You can then load the workspace whenever you want from the specified directory by using the `loadWorkspace` command. A workspace contains customized main window settings including:

- Size and placement of window
- Specific orientation of toolbars
- Docking status
- Visibility settings

Parameters

`-help`	Prints a brief description that includes type and default information for each `saveWorkspace` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man saveWorkspace`
`-name workspaceName`	Specifies the name of the workspace to be saved.
`-dir directory`	Specifies the directory in which the workspace is to be saved. *Default*: Current working directory

Example

The following example saves the current window settings to the `TestWorkspace` in the current working directory.

```
saveWorkspace -name TestWorkspace
```

Related Information

- `deleteWorkspace`
- `loadWorkspace`
- `setDefaultWorkspace`
- Windows Menu chapter of the *Menu Reference*
    - *Workspaces*

select_highlighted

```
select_highlighted
[-help]
[-type string]
```

Selects highlighted objects. If `-type` is specified, selects highlighted objects of the specified type.

Parameters

`-help`	Prints a brief description that includes type and default information for each `select_highlighted` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man select_highlighted`
`-type` *`string`*	Specifies the type of the highlighted objects to be selected. Options are `instance`, `wire`, and `via`.

Examples

- The following command selects all highlighted instances.
  ```
  select_highlighted -type instance
  ```

Related Information

- `gui_select`
- `highlight`

selectModule

```
selectModule
[-help]
name
```

Selects the specified module.

Parameters

`-help`	Prints a brief description that includes type and default information for each `selectModule` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man selectModule`
name	Specifies the name of the module to be selected.

Example

The following example selects the `DTMF_INST` module.

```
selectModule DTMF_INST
```

Related Information

- `deselectModule`

selectPin

```
selectPin
[-help]
name
```

Selects the specified pin.

Parameters

`-help`	Prints a brief description that includes type and default information for each `selectPin` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man selectPin`
`name`	Specifies the name of the pin to be selected.

Example

The following example selects the `swack` pin.

```
selectPin swack
```

Related Information

- `deselectPin`

set_object_color

```
set_object_color
[-help]
{[-object_name InstOrHInstName | -object_type typeList]
[-multicolor | -color_id colorID]
[-include_children {off | color | multicolor}]
[-reset]}
```

Sets the color for objects of the specified name or type. You can choose to include children when specifying object color for a hierarchical instance (HInst).

The `set_object_color` command is typically used during floorplanning to color objects that are currently displayed in the design display area. You can use this command to distinguish between objects of different type or between objects belonging to different instances, hierarchical instances, instance groups, or power domains.

Before using the `set_object_color` command, you should have:

- Loaded the floorplan
- Ungrouped objects so that only required objects are displayed in the design display area in the Floorplan or Physical view.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_object_color` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_object_color`.
`-color_id` *`colorID`*	Specifies the color ID from 0 to 29. The color value for the color ID is the same as specified in the Module Color Preference form (accessed by clicking *Floorplan - Edit Floorplan - Color Module*). *Default*: Color ID `0`
`-include_children {off | color | multicolor}`	Specifies whether the children of the hierarchical instance will be colored simultaneously when you specify a color for an HInst. You can set one of the following options: • `off`: The child instances will not be colored. • `color`: The child instances are colored the same color as their parent instance. • `multicolor`: The child instances are colored in multiple colors.
`-multicolor`	Uses multiple colors to color objects of the specified type, instance, or hierarchical instance.
`-object_name` *`InstOrHInstName`*	Specifies the name of the instance, hierarchical instance, power domain, or instance group whose objects are to be colored. You can specify an object list, such as `objectName {A,B}` for *`objName`*.
`-object_type` *`typeList`*	Specifies the list of type of objects that are to be colored, such as `BusGuide`, `Module`, `PowerDomain`, and `InstGroup`.
`-reset`	Resets the object color.

Examples

- The following command colors the `DTMF_INST/TDSP_CORE_INST` instance with color `3`, as specified in the Module Color Preference form:

```
set_object_color -object_name DTMF_INST/TDSP_CORE_INST -color_id 3
```

- The following command colors the `DTMF_INST/TDSP_CORE_INST` instance and its children with color `3`, as specified in the Module Color Preference form:

  ```
  set_object_color -object_name DTMF_INST/TDSP_CORE_INST -color_id 3 -
  include_children color
  ```

- The following command colors the children of the `DTMF_INST/TDSP_CORE_INST` instance in multiple colors:

  ```
  set_object_color -object_name DTMF_INST/TDSP_CORE_INST -include_children
  multicolor
  ```

Related Information

- `resetMultiColorsHier`
- `saveColorPreference`
- `saveHInstColor`
- `setHInstColorId`
- `setMultiColorsHier`
- `setSelHInstColor`
- Floorplan Menu chapter in the *Menu Reference*
  - *Color Module*

set_power_rail_display

```
set_power_rail_display
[-help]
[-enable_result_browser {true | false}]
[-enable_rlrp {true | false}]
[-enable_voltage_sources {true | false}]
[-filter_max max_value]
[-filter_min min_value]
[-legend {off ne nw se sw}]
[-plot plot_type]
[-range_max max_value]
[-range_min min_value]
[-reset_color_scale {true | false}]
[-filter_color {on | off | color_name}]
[-enable_percentage_range {true | false}]
[-thermal_layer_name thermal_layer]
[-tile_count_x value]
[-tile_count_y value]
[-tile_size_x value]
[-tile_size_y value]
[-customized_range {value1 value2 ... value9}]
[-grid {on | off}]
[-enable_auto_apply {true | false}]
```

Controls the display of the main window widgets, such as legend, result browser, voltage source, filter range, and RLRP path.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_power_rail_results` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_power_rail_results`.

`-customized_range {value1 value2 ... value9}`

Specifies the color scale range values for the 8 filters between the minimum and maximum values of the color gradient based (*Basic*) filter. The value list must contain 9 values.

The filter ranges for the 8 buckets are evenly divided by default. This parameter can be used to customize the filter ranges. You can either modify the color range using the GUI (Basic button) or the `-customized_range` parameter.

Example:

```
set_power_rail_display -plot ir -customized_range { 1 10 12 13 14 15 16 17 49 }
```

`-enable_auto_apply {true | false}`

Specifies to automatically redraw the layout after modifying the color scale range values. This parameter allows you to enable/disable the *Auto Apply for Color Scale* checkbox of the Power Rail Plots form using the Tcl script instead of the GUI option.

The default value of this parameter is `false`.

`-enable_percentage_range {true | false}`

Specifies to display the color gradients for the following plots by percentage in the GUI:

- IR Drop
- IVDD
- IVDN

The percentage value is based on the nominal voltage value of each rail/instance. It displays high percentage voltages in red and low percentage voltages in blue.

Default : `false`

`-enable_result_browser {true | false}`

When set to `true` it enables the result browser.

Default : `false`

`-enable_rlrp {true | false}`

	When set to `true` it enables the display of the RLRP path. The rail analysis must include RLRP analysis (`set_rail_analysis_mode -enable_rlrp_analysis true`) in order to have the required data for this functionality. *Default* : `false`
`-enable_voltage_sources {true | false}`	
	Displays voltage sources (power pads) using white pixels in the layout canvas. *Default* : `false`
`-filter_color {on | off | color_name}`	
	Allows to set the color of the filtered out resistors specified using the scissor filter range. By default, the filtered resistors are not displayed. If you want to view these resistors, you can use this parameter to enable the visibility of resistors and set desired colors for them. The valid values of this parameter are: • `on` – enables the visibility of the filtered out resistors. • `off` - disables the visibility of the filtered out resistors. • *`color_name`* - enables the visibility of filtered out resistors and set the color of resistors per the specified color name.
`-filter_max max_value`	
	Specifies the maximum value of the scissor filter range that will be plotted. You can use the scissor filter range to filter out the resistors and nodes that you do not want to view, and obtain statistics only for the instances in the specified filter range. Optional, but if specified, the `-plot` option is required, and requires both `-filter_min` and `-filter_max` to be specified.
`-filter_min max_value`	
	Specifies the minimum value of the scissor filter range that will be plotted. Optional, but if specified, the `-plot` option is required, and requires both `-filter_min` and `-filter_max` to be specified.
`-grid {on | off}`	

	Allows you to control the grid display when viewing the Voltage Source Current (VC) and Voltage across Package Model (VU) plots. The default value of this parameter is `on`. You can set this parameter to `off` to hide the grid view for better display and easier analysis of the plot data.
`-legend {off ne nw se sw}`	
	Displays the filter color and range details. When you set `set_power_rail_display -legend` to one of the location (`ne nw se sw`) , this opens the legend at the specified location on the design layout. Use `set_power_rail_display -legend off` to close the legend. *Default*: `off` .
`-plot` *plot_type*	
	Plots the specified plot type. See Plot Types for a description of each of the plot types.
`-range_max` *max_value*	
	Specifies the maximum value of the color scale. Optional, but if specified, the `-plot` option is required, and requires both `-range_min` and `-range_max` to be specified.
`-range_min` *min_value*	
	Specifies the minimum value of the color scale. Optional, but if specified, the `-plot` option is required, and requires both `-range_min` and `-range_max` to be specified.
`-reset_color_scale {true | false}`	
	Specifies to reset the maximum (`-range_max`) and minimum (`-range_min`) values of the color scale to the default plot data values. For example, when you specify `-plot ir`, the default value for `-range_min` is `0` and `-range_max` is `100`. You can modify the color scale, such as set `-range_min` to `0` and `-range_max` to `20`. To reset the color scale of the IR drop plot to default, you can specify the `-reset_color_scale` parameter.
`-thermal_layer_name` *thermal_layer*	

	Displays the specified layer of the temperature map. **Example**: `set_power_rail_display -thermal_layer_name VIA_7`
`-tile_count_x` *value*	
	Specifies the number of tiles in the x direction. This parameter is used to resize the tile power density plot. You can either specify the tiles by count (`tile_count_x` and `tile_count_y`) or specify the tiles by size (`tile_size_x` and `tile_size_y`). When you resize the tile power density (`tpd`) plot in the batch or GUI mode, the software automatically invokes the `report_power` command and resizes the plot. This feature allows you to resize the `tpd` plot without having to restore the power database (`power.db`) again.
`-tile_count_y` *value*	
	Specifies the number of tiles in the y direction. This parameter must be specified with the `tile_count_x` parameter.
`-tile_size_x` *value*	
	Specifies the horizontal dimension of each tile polygon in um. This parameter is used to resize the tile power density plot. You can either specify the tiles by count (`tile_count_x` and `tile_count_y`) or specify the tiles by size (`tile_size_x` and `tile_size_y`). When you resize the tile power density (`tpd`) plot in the batch or GUI mode, the software automatically invokes the `report_power` command and resizes the plot. This feature allows you to resize the `tpd` plot without having to restore the power database (`power.db`) again. The following is an example of specifying tiles by size: `set_power_rail_display -plot tpd -tile_size_x 100 -tile_size_y 100`
`-tile_size_y` *value*	
	Specifies the vertical dimension of each tile polygon in um. This parameter must be specified with the `tile_size_x` parameter.

Plot Types

Plot Type	Description
`ac`	Available/feasible capacitance (Farad) Displays the feasible decoupling capacitance that can be added in the area. The feasible capacitance in the area is calculated based on the filler cells placement. The program matches the placed filler cells with the decap cells available in the power-grid library and computes the feasible capacitance. The capacitance of the decap cell is stored in the power-grid library.
`cap`	Ggrid capacitance (Farad) Plots the capacitance of the extracted power-grid network.
`cvi1`	Customized File 1 Plots the user-specified instance voltage file.
`dd`	Decoupling capacitance (decap) density (Farad/um ^{2}) Plots decap density of the design. The decap density is the cell internal decoupling capacitance in the unit area (µm ^{2}).
`dr`	Decoupling capacitance required Reports the decoupling capacitance required to fix dynamic IR-drop violations. This option suggests how much decap needs to be added in a given area to fix the IR-drop violations. The voltage threshold used to determine the IR-drop violations for this analysis is the value specified by the `-threshold` option of the `set_pg_nets` command.
`effr`	Effective resistance (ohm) - instance based Display the effective resistance results for all instances.
`node_reff`	Effective resistance (ohm) - node based Display the effective resistance results for all nodes.
`ipc`	Instance Peak Current Display the peak current of each instance.

`ivdn`	Instance Voltage Drop (Volts) – net-based Plots the net-based instance voltage drop.
`ivdd`	Instance Voltage Drop (Volts) – domain-based Plots the domain-based instance voltage drop. For domain based, at least one power and one ground net must be selected.
`fd`	Filler cell density (1/µm ^{2}) Displays the filler cell density per unit area (µm ^{2}). The plot can be used to analyze the feasibility of adding decap cells by replacing the filler cells and not changing the placement of the logic cells.
`freq`	Frequency domain (Hz) Displays the associated clock frequency of all instances in the design. The instances associated with multiple clock domains are displayed using the fastest clock frequency. This plot can be used to analyze and debug the power distribution in the design.
`ip`	Instance total power (mW) The total power consumed by each instance is displayed. The total power is the sum of the internal, switching and leakage power of the instance. The power data is read from the power database and can be specified using the `-power_db` option or it is queried directly from memory, if power calculation is run during the session.
`ip_i`	Instance internal power (mW) The power consumed by charging and discharging of the internal interconnect and device capacitances of the instance.
`ip_l`	Instance leakage power (mW) The power consumed by devices when not switching.
`ip_s`	Instance switching power (mW) The power consumed by charging and discharging of the interconnect or output load.
`ipd_i`	Specifies internal power density in mW/um ^{2} .

`ipd_l`	Specifies leakage power density in mW/um ^{2} .
`ipd_s`	Specifies switching power density in mW/um ^{2} .
`ipd_t`	Specifies total power density in in mW/um ^{2} .
`ir`	IRdrop or voltage drop (Volts) Displays the IRdrop across each extracted power-grid segment in the design.
`jrms`	Specifies current density as j/jmax based on the Irms current. This plot is only supported for dynamic analysis. A jrms gif file will be saved in the state directory that shows where the Irms based EM violation occurred. This plot type allows you to verify the analysis results of "jrms" using the GUI.
`load`	Loading capacitance (Farad) Displays the output loading capacitance driven by the instance. This plot can be used to analyze and debug the regions with high switching power.
`pi`	Power-switch (gate) current violation: I/Idsat Displays the ratio of current through the power-switch and the saturation current (Idsat) of the power-switch. This plot can be used to analyze and debug whether the current through power-switches exceeds the saturation current characterized for the power-switch devices. The saturation current and on-resistance of the power-switches are characterized and stored inside the power-grid library of the power-switch cell. A ratio of greater than 1 means that the current requirement of the power-gated block can not be met with placed power-switch instance. Either the power-switch needs to be made larger or power-switch instances need to be added.
`ptpd`	Profiling Tile Power Density Displays the tile-based power density plot for the vector profiling flow. You can enable this plot using the `read_power_rail_results -profiling_power_file` *`file`* command. You can also view the power density for the different tile co-ordinates in the *Details* tab of the Result Browser.

`pv`	Voltage drop across power-switch (Volt) Displays the IRdrop across power-switch instances. This plot can be used to analyze and debug the IRdrop inside the power-gated block, e.g. if the IRdrop across power-switches is already high, the IRdrop inside the power-gated block will be much higher. In this case, the power-switch placement will have to be refined in order to resolve the IRdrop problem.
`rc`	Resistor current (Amp) Displays the current across power-grid resistor segments. This plot can be used to analyze and debug how the current flows from the voltage sources into the rest of the design, e.g. high current should flow from pads to the top-level routing level layers and then to the lower level metal layers. This plot can also be used to debug current density violations (`rj`).
`rlrp`	The total resistance of the instance along the least resistive path. Displays the total resistance of the instance along the least resistive path, meaning the resistance of the instance along the path from where it gets the most amount of current. This plot can be used to analyze and debug the power-grid integrity early in the design stage. An instance with high resistance is likely to have high static or dynamic IRdrop depending on the vector that is simulated later in the design cycle. This plot can be also used during decap optimization, since regions with high dynamic IRdrop and high resistance may not be improved upon by adding decaps, because the effectiveness of decoupling capacitance depends on the resistance of the power-grid network. The plot displays the RLRP paths for power (purple color) and ground (white color) nets in different colors for the identification of paths and ease of analysis. You can also highlight the least resistive path of the instance using the GUI and selecting the instance with left mouse button. The *RLRP Path* checkbox should be enabled.

`res`	Grid resistance (ohm) Displays the total resistance of the least resistive path from each node to voltage source. This plot can be used to view high resistance regions of the power-grid. It is useful during IRdrop and decap analysis to identify weak power-grid connectivity. The GUI also allows you to view the resistor-based RLRP path. To view the resistor-based RLRP path: 1. Select the *res* Rail plot, and the *RLRP Path* checkbox. 2. Select a resistor. The GUI highlights the least effective resistance path to the nearest voltage source, and opens the *Resistance Path* form.
`rj`	Resistor current density, J/Jmax. Displays the current density (current per unit area) violation on each power-grid segment using the ratio of calculated current density and maximum allowed current density. A ratio of greater than 1 means that the current density limit of the segment is violated. The current density limits are supplied by the foundry and can be read during analysis using the `set_rail_analysis_mode -em_models` command.
rs	Specifies the plot type for resistor sensitivity.
`sem_avg`	Average Current Density Displays average current limit violations in the design. This plot is enabled when you specify `verifyACLimit -method avg`.
`sem_peak`	Peak Current Density Displays peak current limit violations in the design. This plot is enabled when you specify `verifyACLimit -method peak`.
`sem_rms`	RMS Current Density Displays RMS current limit violations in the design. This plot is enabled when you specify `verifyACLimit -method rms`.
`slack`	Instance slack (Seconds) Displays worst instance slack. The slack data is queried from either the timing database or if the power database is specified using `-power_db` option, then the worst slack data is read from the power database. When using an external TWF, the slack data is read from the TWF.

`tc`	Tap current (Amp) Displays the current consumed by the instance devices or current taps. During rail analysis, it is based on the power consumption of the instance and is distributed inside the cell. The current distribution inside the cell is based on the extracted device netlist stored inside the power-grid library of the cell. The current distribution is generally based on the device width, length, and spice models. This plot can be used to analyze and debug IRdrop inside the macro.
`td`	Transition density (Hz) Displays the worst transition density (activity * frequency) of the instance. This plot can be used to analyze and debug activity distribution and high power density regions.
`thermal`	Instance thermal Displays the instance temperature distribution for the chip. The plot shows tile-based temperature for instances. When you click on an instance, the *Attribute Viewer* window opens and displays the instance temperature in the *Thermal* field.
`unc`	Unconnected segments Displays power-grid segments that are disconnected from the voltage source (power pads). This plot can be used to debug unusually high IRdrop in the region.
`vc`	Voltage source current (Amp) Displays current through the voltage sources (power pads). This plot can be used to analyze the current carrying capacity of the power-pad.
`vu`	Voltage drop across package (Volt) Displays voltage drop across package. If analysis is run with package model attached to the power pads, this plot displays the IRdrop due to package RLCK elements.

Examples

- The following command specifies that `div` data will be plotted without loading design DEF:

  ```
  set_power_rail_display\
  -plot div
  ```

- The following command enables the visibility of the filtered out resistors without changing the color:

  ```
  set_power_rail_display -filter_color on
  ```

- The following command enables the visibility of the filtered out resistors and also changes the color to red:

  ```
  set_power_rail_display -filter_color red
  ```

Related Information

- `set_power_rail_layers_nets`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_power_rail_layers_nets

```
set_power_rail_layers_nets
[-help]
[-design_opacity value]
[-rail_analysis_opacity value]
[-nets {{all_power 0/1}{all_ground 0/1}} |
 -enable_nets {names} |
 -disable_nets {names} ]
[-enable_switch_nets {all|net_names} |
 -disable_switch_nets {all|net_names} ]
[-enable_report_layers {all|layer_names} |
 -disable_report_layers {all|layer_names} |
 -layers {{name report0/1 visible0/1}{...}} ]
[-enable_visible_layers {all|layer_names} |
 -disable_visible_layers {all|layer_names} |
 -layers {{name report0/1 visible0/1}{...}} ]
[-enable_visible_sem_layers {all|layer_names} |
 -disable_visible_sem_layers {all|layer_names} ]
[-show_layer_net_setting_list {true | false}]
[-instance_voltage_method {Worst | Best | Avg | WorstAvg}]
[-instance_voltage_window {1| 0}]
[-via_size {1 | 2 | 3 | 4 | 5 | 6}]
```

Controls the visibility of the layers, the opacity of the rail analysis or design database, and the visibility of power/ground nets.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_power_rail_layers_nets` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_power_rail_layers_nets`.
`-design_opacity value`	

	Specifies the design opacity value for the design database. The opacity range is from 0-100%, the default value is `60`.
`-disable_nets {`*`names`*`}`	
	Hides the specified nets. This parameter is mutually exclusive to the `-enable_nets` and `-nets` parameters.
`-disable_report_layers {all|`*`layer_names`*`}`	
	Hides the reporting (user-defined layers to be used for processing) of layers. You can use `all` to hide all layers, or list the specific layers that are not to be displayed. This parameter is mutually exclusive to the `-enable_report_layers` and `-layers` parameters.
`-disable_switch_nets {all|`*`net_names`*`}`	
	Hides the specified switch nets. You can use `all` to hide all the switch nets, or list the specific switch nets that are not to be displayed. This parameter is mutually exclusive to the `-enable_switch_nets` parameter.
`-disable_visible_layers {all|`*`layer_names`*`}`	
	Specifies which layers should not be made visible when plotting. You can use `all` to hide all layers, or list the specific layers that are not to be displayed. This parameter is mutually exclusive to the `-enable_visible_layers` and `-layers` parameters.
`-disable_visible_sem_layers {all|`*`layer_names`*`}`	
	Specifies which signal EM layers should not be made visible when plotting. You can use `all` to hide all layers, or list the specific layers that are not to be displayed. This parameter is mutually exclusive to the `-enable_visible_sem_layers` parameter.
`-enable_nets {`*`names`*`}`	
	Displays only the specified nets. This parameter is mutually exclusive to the `-disable_nets` and `-nets` parameters.
`-enable_report_layers {all|`*`layer_names`*`}`	

	Specifies to select the user-defined layers to be used for processing. You can use `all` to select all layers, or list the specific layers that are to be selected. This parameter is mutually exclusive to the `-disable_report_layers` and `-layers` parameters.
`-enable_switch_nets {all|`*`net_names`*`}`	
	Displays the specified switch nets. You can use `all` to display all the switch nets, or list the specific switch nets that are to be displayed. This parameter is mutually exclusive to the `-disable_switch_nets` parameter.
`-enable_visible_layers {all|`*`layer_names`*`}`	
	Specifies which layers should be made visible when plotting. You can use `all` to display all layers, or list the specific layers that are to be made visible. This parameter is mutually exclusive to the `-disable_visible_layers` and `-layers` parameters.
`-enable_visible_sem_layers {all|`*`layer_names`*`}`	
	Specifies which signal EM layers should be made visible when plotting. You can use `all` to select all layers, or list the specific layers that are to be selected. This parameter is mutually exclusive to the `-disable_visible_sem_layers` parameter.
`-instance_voltage_method {Worst | Best | Avg | WorstAvg}`	
	Allows different methods of computing effective instance voltage during rail analysis. Using this option, you can write out the Best, Worst, or Average effective instance voltage between power and ground nets. The WorstAvg method computes the average instance voltage during each evaluation window defined by the IV Window field, and reports the worst average instance voltage out of all the windows. If IV Method is WorstAvg and IV Window is whole, then this method will report average voltage across all time steps in simulation. Default : `Worst`
`-instance_voltage_window {1| 0}`	

	Controls the EIV plot. EIV is obtained by processing the voltage waveforms and finding the worse-case effective voltage between the power and ground pins during a specific window. There are two options to view EIV plots: • `1` – both switching and timing windows (window EIV) • `0` – worst of the entire rail simulation (worst EIV)
`-layers {{name report0/1 visible0/1}{...}}`	
	Controls the visibility of layers. You can enable or disable the reporting and visibility of specific layer. For example, `-layer {{M1 1 1} {M2 0 0}` means enable reporting and visibility of layer M1, and disable reporting and visibility of layer M2 This parameter is mutually exclusive to the `-disable_report_layers, -disable_visible_layers, -enable_visible_layers,` and `-enable_report_layers` parameters.
`-nets {{all_power 0/1}{all_ground 0/1}}`	
	Controls the display of all the power and ground nets. When `all_power` is set to `0`, hides all power nets, and when set to `1`, displays all power nets. When `all_ground` is set to `0`, hides all ground nets, and when set to `1`, displays all ground nets. This parameter is mutually exclusive to the `-enable_nets` and `-disable_nets` parameters.
`-rail_analysis_opacity` *value*	
	Specifies the opacity value for the rail analysis output. The opacity range is from 0-100%, the default value is `100`.
`-show_layer_net_setting_list {true | false}`	

	Displays the current status (OFF/ON) of all layers and nets in the console. The following is an example of the output:
	`#-------------------------------#`
	`# Layers   Report    Visible #`
	`# RD         ON          ON #`
	`# M0         ON          ON #`
	`.....`
	`#-------------------------------#`
	`# PowerNets  Status #`
	`# VDD           ON #`
	`# VDDm          ON #`
	`#-------------------------------#`
	`# GroundNets Status #`
	`# VSS          ON #`
	`#-------------------------------#`

`-via_size {1 | 2 | 3 | 4 | 5 | 6}`	
	Specifies to modify and control the size of vias in the GUI. The default via size is `1`. The possible via size values are `1`, `2`, `3`, `4`, `5`, and `6`.

Examples

- The following command specifies the layers that will be visible:

```
set_power_rail_layers_nets \
 -enable_visible_layers{RD M1 V12 M2 V23 M3 V34 M4 V45 M5 V56 M6} \
 -enable_nets {vdd vdd_1}
```

- The following commands describe the use model for enabling and disabling switch nets. Here, consider a net `VDD` with three switch nets `VDD_sw0`, `VDD_sw1`, and `VDD_sw2`:
  - The following command hides the switch net VDD_sw0:
    set_power_rail_layers_net `-disable_switch_nets {VDD {VDD_sw0}}`
    Displays `VDD_sw1` and `VDD_sw2`.
  - The following command hides all the switch nets:

    ```
    set_power_rail_layers_net -disable_switch_nets {VDD {all}}
    ```

  - The following command displays the switch net `VDD_sw0`:

    ```
    set_power_rail_layers_net -enable_switch_nets {VDD {VDD_sw0}}
    ```

  - The following command displays all the switch nets:

    ```
    set_power_rail_layers_net -enable_switch_nets {VDD {all}}
    ```

Related Information

- `set_power_rail_display`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_visible_bumps

```
set_visible_bumps
[-help]
{-bump_cells bump_cell_name| -all}
```

Displays only the bumps with the specified bump cell names and hides all other bumps.

To redisplay all bumps, specify the `-all` parameter.

Parameters

`-help`	Prints a brief description that includes type and default information for each `set_visible_bumps` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_visible_bumps`
`-all`	Displays all bumps. ⚠ By default, all bumps are displayed. When you specify `set_visible_bumps bump_cell_name`, the tool displays only the bumps with the specified bump cell names and hides all other bumps. You can then use `set_visible_bumps -all` to redisplay all bumps.
`-bump_cells` *names*	Displays only the bumps with the specified bump cell names and hides all other bumps. You can specify multiple bump cell names.

Examples

- The following command displays only the bumps with bump cell as BUMPCELL:
  ```
  set_visible_bumps -bump_cells BUMPCELL
  ```
- The following command redisplays all bumps:
  ```
  set_visible_bumps -all
  ```

Related Information

- get_visible_bumps

set_visible_netGroups

```
set_visible_netGroups
[-help]
{netGroups | -all}
```

Displays only the specified netGroups and hides all other netGroups.

To display all netGroups, specify the `-all` parameter.

Parameters

`-help`	Prints a brief description that includes type and default information for each `set_visible_netGroups` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_visible_netGroups`
`-all`	Displays all the netGroups.
`netGroups`	Displays only the specified netGroups and hides all other netGroups. You can specify multiple netGroup names.

Examples

- The following command hides all netGroups except the specified one:
  ```
  set_visible_netGroups test_ng
  ```

Related Information

- `get_visible_netGroups`

set_visible_nets

```
set_visible_nets
[-help]
{nets | -all}
```

Displays only the specified nets and hides all other nets.

To display all nets, specify the `-all` parameter.

Parameters

`-help`	Prints a brief description that includes type and default information for each `set_visible_nets` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_visible_nets`
`-all`	Displays all nets.
nets	Displays only the specified nets and hides all other nets. You can specify multiple net names.

Examples

- The following command hides all nets except the specified net:
  ```
  set_visible_nets inst1/lower/net_1
  ```

Related Information

- `get_visible_nets`

setDefaultWorkspace

```
setDefaultWorkspace
[-help]
-name workspaceName
[-dir directory]
```

Sets specified workspace as the default. The default workspace is loaded automatically when the software starts. So if you want Innovus to load a workspace automatically at startup, you need to first save it as default.

Parameters

`-help`	Prints a brief description that includes type and default information for each `setDefaultWorkspace` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setDefaultWorkspace`
`-name` *`workspaceName`*	Specifies the name of the workspace to be set as default.
`-dir` *`directory`*	Specifies the location of the workspace to be set as default.

Example

The following example sets the `TestWorkspace` workspace in the current working directory as default.

```
setDefaultWorkspace -name TestWorkspace
```

Related Information

- `deleteWorkspace`
- `loadWorkspace`
- `saveWorkspace`
- Windows Menu chapter of the *Menu Reference*
    - *Workspaces*

setHInstColorId

```
setHInstColorId
[-help]
hInstName
colorID
[-descendant]
```

Specifies the color ID for a hierarchical instance.

Parameters

`-help`	Prints a brief description that includes type and default information for each `setHInstColorId` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setHInstColorId`
`hInstName`	Name of the hierarchical instance to which this color ID applies.
`colorID`	Specifies the color ID from `0` to `29`. The color value for the color id is the same as specified in the Module Color Preferences form (accessed by clicking *Floorplan - Edit Floorplan - Color Module*).
`-descendant`	Specifies that the color ID should be applied to the child/descendant instances as well. *Default*: If no ID is specified, the color id is applied only for the top-level instance.

Example

The following command applies color ID `3` to the hierarchical instance `topModule`:

```
setHInstColorId topModule 3
```

Related Information

- `resetMultiColorsHier`
- `saveColorPreference`
- `saveHInstColor`
- `set_object_color`
- `setMultiColorsHier`
- `setSelHInstColor`
- Floorplan Menu chapter in the *Menu Reference*
    - *Color Module*

setMultiColorsHier

```
setMultiColorsHier
[-help]
[colorID [hInstName]]
```

Specifies the color assignment for hierarchical instances based on the color Id from the Module Color Preference form (accessed by clicking *Floorplan - Edit Floorplan - Color Module*).

Parameters

`-help`	Prints a brief description that includes type and default information for each `setMultiColorsHier` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setMultiColorsHier`
`colorID`	Specifies an integer from `0` to `29` to set the starting color ID for the specified hierarchical instance. *Default*: If no ID is specified, the color ID `0` in the Module Color Preference form is applied to the selected hierarchical instances. This is the same behavior as using the *Auto* button on the Module Color Preference form.
`hInstName`	Specifies the name of the hierarchical instance whose descendants are set to the specified color ID. *Default*: if this parameter is not specified, the first hierarchical instance uses the specified color id; the second hierarchical instance uses the next color ID, and so on.

Examples

- The following command specifies the color Id 3 for the first hierarchical module, the color Id 4 for the second hierarchical module, and so on.

  ```
  setMultiColorsHier 3
  ```

- In the following command, only the descendants of `topModule` are displayed using the color ID `3`.

  ```
  setMultiColorsHier 3 topModule
  ```

Related Information

- `resetMultiColorsHier`
- `saveColorPreference`
- `saveHInstColor`
- `setHInstColorId`
- `set_object_color`
- `setSelHInstColor`
- Floorplan Menu chapter in the *Menu Reference*
  - *Color Module*

setSelectedPtnCut

```
setSelectedPtnCut
[-help]
X1 Y1 X2 Y2
PartitionName
```

Sets the area and the name for the selected partition cut area.

Parameters

`-help`	Prints a brief description that includes type and default information for each `setSelectedPtnCut` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setSelectedPtnCut`
X1 Y1 X2 Y2	Specifies the coordinates of the partition cut area.
PartitionName	Specifies the name of the partition cut area.

Related Information

- `setSelectedPtnFeedthrough`
- `setSelectedPtnPinBlk`
- `setSelectedPtnPinGuide`

setSelectedPtnFeedthrough

```
setSelectedPtnFeedthrough
[-help]
x1 y1 x2 y2
PtnFeedthroughName
{ layerID | layerIDList }
```

Sets the area, name, and layer ID for the selected partition routing feedthrough object.

Parameters

`-help`	Prints a brief description that includes type and default information for each `setSelectedPtnFeedthrough` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setSelectedPtnFeedthrough`
x1 y1 x2 y2	Specifies the coordinates of the routing feedthrough area.
PtnFeedthroughName	Specifies the name of the routing feedthrough.
layerID | *layerIDList*	Specifies the metal layer(s) that can route through the partition feedthrough.

Example

- The following command sets the routing feedthrough object, `ptnFeedthru2`, to allow routing on the layer `5` (Metal5) to cross over the specified area:

  ```
  setSelectPtnFeedthrough 100.0 100.0 150.0 150.0 ptnFeedthru2 5
  ```

Related Information

- `setSelectedPtnCut`
- `setSelectedPtnPinBlk`
- `setSelectedPtnPinGuide`

setSelectedPtnPinBlk

```
setSelectedPtnPinBlk
[-help]
x1 y1 x2 y2
PtnPinBlkName
{layerID}
```

Sets the area, name, and layer ID for the selected partition pin blockage.

Parameters

`-help`	Prints a brief description that includes type and default information for each `setSelectedPtnPinBlk` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setSelectedPtnPinBlk`
x1 y1 x2 y2	Specifies the coordinates of the partition pin blockage area.
PtnPinBlkName	Specifies the name of the partition pin blockage.
layerID	Specifies the metal layer(s) where partition pins are not to be created.

Example

- The following command sets the partition pin blockage, `ptnPinBlk2`, within the specified area and on the layers `4` (Metal4), `5` (Metal5), and `6` (Metal6):

  ```
  setSelectedPtnPinBlk 100.0 100.0 150.0 150.0 ptnPinBlk2 {4 5 6}
  ```

Related Information

- `setSelectedPtnCut`
- `setSelectedPtnFeedthrough`
- `setSelectedPtnPinGuide`

setSelectedPtnPinGuide

```
setSelectedPtnPinGuide
[-help]
x1 y1 x2 y2
ptnPinObj
layerID
space
[cellName]
```

Sets area, name, and layer Id for the selected pin guide object.

Parameters

`-help`	Prints a brief description that includes type and default information for each `setSelectedPtnPinGuide` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setSelectedPtnPinGuide`
x1 y1 x2 y2	Specifies the coordinates of the pin guide area.
ptnPinObj	Specifies the name of the net, net group, or pin group associated with this pin guide
layerID	Specifies the metal layer(s) for the pin guide.
space	Specifies spacing for the pin guide.
cellName	This parameter is required only if the object specified with the *ptnPinObj* parameter is a pin group. Specifies the name of the cell with which the pin group is associated.

Example

- The following command sets the partition pin guide, `ptnGuide2`, within the specified area and on the layers `2` (Metal2) and `3` (Metal3) with pin guide spacing as `0` and associated with the cell `cell1`:

  ```
  selectPtnPinGuide 100.0 100.0 150.0 150.0 pinGuide2 {2 3} 0 {cell1}
  ```

Related Information

- `setSelectedPtnCut`
- `setSelectedPtnFeedthrough`
- `setSelectedPtnPinBlk`

setSelectedWireState

```
setSelectedWireState
[-help]
index
value
```

Sets the state of the selected wire.

Parameters

`-help`	Prints a brief description that includes type and default information for each `setSelectedWireState` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setSelectedWireState`
index	Specifies which selected wire shape needs to be set.
value	Specifies the state to be set for the selected wire. You can specify one of the following values: • `ROUTED` • `FIXED` • `COVER` • `NOSHIELD` • UNKNOWN

Related Information

- The wire section in the Innovus Database Object Information manual
- Editing Wires chapter of the *User Guide*

setSelHInstColor

```
setSelHInstColor
[-help]
[colorID]
```

Specifies the color Id for the selected hierarchical instances.

Parameters

`-help`	Prints a brief description that includes type and default information for each `setSelHInstColor` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setSelHInstColor`
`colorID`	Specifies the color Id to apply to all selected hierarchical instances. The color value for the color id is the same as specified in the Module Color Preference form (accessed by clicking *Floorplan - Edit Floorplan - Color Module*) *Default*: If no Id is specified, the color Id `0` is applied to all the selected hierarchical instances.

Example

The following command applies color Id 5 to all the selected hierarchical instances:

```
setSelHInstColor 5
```

Related Information

- `resetMultiColorsHier`
- `saveColorPreference`
- `saveHInstColor`
- `setHInstColorId`
- `setMultiColorsHier`
- `set_object_color`
- Floorplan Menu chapter in the *Menu Reference*
  - *Color Module*

ui_view_box

```
ui_view_box [-help]
```

Reports the coordinates of the current area being viewed in the GUI.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command, use the `man` command: `man ui_view_box`

Example

The following command reports the coordinates of the current area being viewed in the GUI:

```
innovus 9> ui_view_box

93.367 -646.465 103.294 -638.724
```

Related Information

- `zoomBox`

uiAdd

```
uiAdd
[-help]
name
[-before name]
[-checked {true | false}]
[-command string]
[-disabled {true | false}]
[-enablevar string]
[-foreground string]
[-icon filename]
[-in name]
[-label string]
[-movable {true | false}]
[-newline {true | false}]
[-shortcut string]
[-tooltip string]
-type {menu | toolbar | submenu | command | check | radio | separator | toolbutton}
[-underline integer]
[-value string ]
[-variable string]
[-visiblevar string]
```

Adds a new interface element with the specified name of the specified type. Use this command to customize the Innovus graphical interface by adding menus, toolbars, status bar, and other interface elements, as required.

Parameters

`-help`	Prints a brief description that includes type and default information for each `uiAdd` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man uiAdd`

name	Specifies as a string the internal name of the interface element you want to add. **Note:** Choose the internal name of the element carefully as this is what is used internally by the software to identify the element for any further manipulation.
-before *name*	When specified, adds the new element before the element identified by *name*.
-checked {true | false}	Specifies whether or not the check box or radio button you are adding should be checked. **Note:** You can use the -checked parameter only if you are adding elements of type check or radio. *Default*: false
-command *string*	Specifies the Tcl command to be executed when the element you are adding is invoked. **Note:** You can use the -command parameter only if you are adding elements of type command, check, radio or toolbutton.
-disabled {true | false}	Specifies the state of the interface element you are adding. **Note:** You can use the -disabled parameter only if you are adding elements of type menu, submenu, command, check, radio or toolbutton. The parameter is NOT applicable for elements of type toolbar and separator. *Default*: false
-enablevar *string*	Represents the Tcl global variable for Enable status.
-foreground *string*	Specifies foreground color of the icon. **Note:** You can use the -foreground parameter only if you are adding an element of type toolbutton.

`-icon` *`filename`*	When adding a toolbutton, specifies the file name of the toolbutton icon. *`filename`* can be the full path of the icon file or an icon name supported by Innovus. The icon names used in Innovus can be queried using the `uiGet` command. For list of supported image formats for the icon file, see the "Supported Image Formats" section of the Customizing the User Interface chapter in the *Innovus User Guide*. **Note:** The `-icon` parameter is applicable only for elements of type `toolbutton`.
`-in` *`name`*	Specifies the name of the parent element. Use this parameter along with `-before` to specify the location of the element you are adding. By default, `-in` adds the element to the end of the parent element. Use `-before` to add the element at a specific location within the parent element. **Note:** If `-in` is not specified, the new element is created but is not added to any location.
`-label` *`string`*	Specifies the label to be displayed on the new element. **Note:** This parameter is NOT applicable for elements of type `separator`.
`-movable {true | false}`	When adding a toolbar, specifies whether or not it will be movable.
`-newline {true | false}`	When adding a toolbar, specifies whether or not the toolbar should be added as a new row. **Note:** The `-newline` parameter is applicable only for elements of type `toolbar`. *Default*: `false`
`-shortcut` *`string`*	Specifies the shortcut key for a command, check box, radio button, or toolbutton. **Note:** You can use the `-shortcut` parameter only if you are adding elements of type `command`, `check`, `radio`, or`toolbutton`.

`-tooltip` *`string`*	Specifies the tooltip text for the toolbar or toolbutton you are adding. **Note:** The `-tooltip` parameter is applicable only for elements of type `toolbar` and `toolbutton`.
`-type {menu | toolbar | submenu | command | check | radio | separator | toolbutton}`	
	Specifies the type of the interface element you want to add. Currently, elements of type `main` and `statusbar` are not supported.
`-underline` *`integer`*	Specifies the integer index of the character to be underlined in the element label. The index begins with `0` and should have a value less than or equal to the length of the element label. For instance, if you want the second letter of the element label to appear underlined, specify *`integer`* as `1`. **Note:** The `-underline` parameter is NOT applicable for elements of type `toolbar, separator,` and `toolbutton`. **Note:** This parameter can be used only if the `-label` parameter is also specified.
`-value` *`stringValue`*	When adding a radio button, specifies the value to be stored in the associated variable when the radio button is selected. **Note:** The `-value` parameter is applicable only for elements of type `radio`.
`-variable` *`string`*	When adding a check box or radio button, specifies the Tcl global variable for the item.
`-visiblevar` *`string`*	Represents the Tcl global variable for Visible status

Examples

- The following command will add a new menu labeled *ExampleMenu* in the main window:
  ```
  uiAdd exmpMenu -type menu -label ExampleMenu -in main
  ```
- The following command will add a new menu item labeled *ExampleCommand* in *ExampleMenu*:

```
uiAdd exmpCommand -type command -label ExampleCommand -in exmpMenu -command [list
puts Example Command]
```

- The following command adds a new, unmovable toolbar to the layout window:
  ```
  uiAdd newTB -type toolbar -in main.layout -movable false
  ```

Related Information

- uiDelete
- uiFind
- uiGet
- uiSet
- Customizing the User Interface chapter in the *User Guide*
  - *Creating a New Menu*
  - *Adding a Menu Element to an Existing Menu*
  - *Replacing an Existing Menu Element*
  - *Adding a New Toolbar or Toolbutton*

uiDelete

```
uiDelete
[-help]
name
```

Deletes an existing interface element with the specified name. If multiple interface elements are found with the same name, the first matched element is deleted. Use this command to customize the Innovus graphical interface by removing menus, menu items, and other interface elements that you do not require any longer.

Parameters

`-help`	Prints a brief description that includes type and default information for each `uiDelete` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man uiDelete`
`name`	Specifies the internal name of the interface element as a string. An error is returned if an interface element with the specified name does not exist. **Note:** Here, *`name`* represents the internal name assigned to the element when it was added to the interface and not the label that is displayed on the interface. If you specify the label instead of the name, the software will return an error.

Examples

- The following command will delete an interface element with the internal name *`Menu1`*:

  ```
  uiDelete Menu1
  ```

Related Information

- `uiAdd`
- `uiFind`
- `uiGet`
- `uiSet`
- Customizing the User Interface chapter in the *User Guide*
    - *Replacing an Existing Menu Element*

uiFind

```
uiFind
[-help]
[name]
[property {value}]
```

Enables you to retrieve an element's name by specifying a property-value combination.

Parameters

`-help`	Prints a brief description that includes type and default information for each `uiFind` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man uiFind`
`name`	Specifies the name of the parent element to define the search scope. If multiple elements are found with the same *`name`*, the first matched element is searched. If *`name`* is not specified, a global search is performed.
`property {value}`	Specifies the property-value combination that should be used as the search criteria for finding an element. In addition to all the properties listed in the `uiAdd` command, `-type` *`type`* is also supported. To see the list of properties supported by the `uiAdd` command, see `uiAdd`.

Examples

- The following command returns the names of all the menus in the main window:
  ```
  uiFind main -type menu
  ```
  The software returns the following:
  ```
  file view edit partition fplan power place eco clock route timing verify pvs tools
  windows flow help
  ```
- The following command returns the name of the element before the Help menu in the main window:

```
uiFind -before help
```

The software returns the following:

```
flow
```

- The following command returns the name of the element with a label "*CCOpt Clock Tree Debugger...*" on the menu in the main window:

  ```
  uiFind -label "CCOpt Clock Tree Debugger..."
  ```

  The software returns the following:

  ```
  _CCOpt_Clock_Tree_Debugger...
  ```

  You can then get any property of the element by using `uiGet`. For example:

  ```
  uiGet _CCOpt_Clock_Tree_Debugger... -command
  ```

  The software returns the following:

  ```
  ctdMainWin::createSetting
  ```

  Here, `ctdMainWin::createSetting` is the command that is invoked when the menu item labeled *CCOpt Clock Tree Debugger...* is clicked.

- The following command returns all movable toolbars in the main window:

  ```
  uiFind -movable true
  ```

  The software returns the following:

  ```
  tbFile tbEdit tbView tbTools tbReport tbToolBox we main.layout.selectbar
  ```

Related Information

- `uiAdd`
- `uiDelete`
- `uiGet`
- `uiSet`
- Customizing the User Interface chapter in the *User Guide*
  - *Querying and Configuring Interface Elements*

uiGet

```
uiGet
[-help]
[name]
[-quiet]
[ -before | -checked | -children | -command
  | -disabled | -dockwin | -enablevar | -foreground
  | -geometry | -icon | -in | -label | -menu
  | -message | -movable | -newline | -shortcut
  | -statusbar | -title | -toolbar | -tooltip
  | -type | -underline | -value | -variable
  | -visible | -visiblevar | -windowstatus ]
```

Retrieves the current value of the specified property of the element that is being queried. Using the `uiGet` command, you can query all the properties in the `uiAdd` command for each type. In addition, depending on the `type`, some special properties are supported too as described below:

- For elements of type `menu/submenu/toolbar`:
  - `uiGet name -children`
    Returns a list of all the child elements of the specified interface element.
- For `main` (main window):
  - `uiGet main -menu`
    Returns a list of all the menus in the main window.
  - `uiGet main.layout -toolbar`
    Returns the list of all the toolbars in the main window.
  - `uiGet main -statusbar`
    Returns the status bar in the main window.
  - `uiGet main -geometry`
    Returns the geometry of the main window in the format *width*x*height*+*x*+*y*.
  - `uiGet main -title`
    Returns the title of the main window.
  - `uiGet main -closecmd`
    Returns the close confirmation command of the main window.
  - `uiGet main -visible`

Returns the visibility status of the main window.

- For `statusbar`:
  - `uiGet name -message`
    Returns the current message of the status bar.
  - `uiGet name -visible`
    Returns the visibility status of the status bar.
  - `uiGet name -in`
    Returns the parent element of the status bar.

The statusbar also supports the `-enablevar` and `-visiblevar` parameters.

Parameters

`-help`	Prints a brief description that includes type and default information for each `uiGet` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man uiGet`
`name`	Specifies as a string the name of the interface element you want to query. If multiple widgets are found with the same *`name`*, the first matched widget is queried. Returns an error if there is no element with the specified name.
`-before` *`name`*	When specified, returns the element that occurs before the element identified by *`name`*.
`-checked`	Returns the on/off state of the specified check box or radio button. If the check box or radio button is selected, returns `true`. If it is deselected, returns `false`.
`-children`	Returns child elements of specified interface element. **Note:** You can use this parameter with elements of type `menu`, `submenu`, and `toolbar`.

`-command`	Returns the Tcl command to be executed when the specified element you are adding is invoked. **Note:** You can use the `-command` parameter only with elements of type `command, check, radio` or `toolbutton`.
`-disabled`	When specified, returns whether the specified interface element is enabled (`false`) or disabled (`true`). **Note:** You can use the `-disabled` parameter only with elements of type `menu, submenu, command, check, radio` or `toolbutton`. The parameter is NOT applicable for elements of type `toolbar` and `separator`.
`-dockwin`	Returns the dockwidget.
`-enablevar`	Represents the Tcl global variable for Enable status.
`-foreground`	Returns the foreground color of the icon for the specified toolbutton. **Note:** You can use the `-foreground` parameter only with elements of type `toolbutton`.
`-geometry`	Returns the width, height, and coordinates of the main window. **Note:** The `-geometry` parameter is applicable only for the main window, which is type `main`.
`-icon`	Returns the file name of the specified toolbutton. **Note:** The `-icon` parameter is applicable only for elements of type `toolbutton`.
`-in`	Returns the name of the parent element.
`-label`	Returns the label of the specified element. **Note:** This parameter is NOT applicable for elements of type `separator`.
`-menu`	Returns the names of the menus in the main window. **Note:** This parameter is applicable only for the main window, which is type `main`.

`-message`	Returns the message on the status bar. **Note:** This parameter is applicable only for the status bar.
`-movable`	Returns whether or not the specified toolbar is movable. **Note:** The `-movable` parameter is applicable only for elements of type `toolbar`.
`-newline`	Returns whether or not the specified toolbar starts in a new row. **Note:** The `-newline` parameter is applicable only for elements of type `toolbar`.
`-quiet`	Displays the current settings for the specified parameters in the Tcl list format.
`-shortcut`	Returns the shortcut key for a command, check box, radio button, or toolbutton. **Note:** You can use the `-shortuct` parameter only with elements of type `command`, `check`, `radio`, or `toolbutton`.
`-statusbar`	Returns the name of the main window's status bar. **Note:** This parameter is applicable only for the main window (`main`).
`-title`	Returns the text on the main window's title bar. **Note:** This parameter is applicable only for the main window (`main`).
`-toolbar`	Returns the toolbars in the window.
`-tooltip`	Returns the tooltip text for the specified toolbar or toolbutton. **Note:** The `-tooltip` parameter is applicable only for elements of type `toolbar` and `toolbutton`.
`-type`	Returns the type of the specified element.
`-underline`	Returns the integer index of the underlined character in the element label. **Note:** The `-underline` parameter is NOT applicable for elements of type `toolbar`, `separator`, and `toolbutton`.

`-value`	Returns the value stored in the associated variable of the specified radio button. **Note:** The `-value` parameter is applicable only for elements of type `radio`.
`-variable`	Returns the Tcl global variable for the specified check box or radio button.
`-visible`	Returns the visibility status of the element.
`-visiblevar`	Represents the Tcl global variable for Visible status

`-windowstatus`	Returns the widget window status. This option returns one of the following values, depending on the window status, assuming that tool was launched in the GUI mode:
	• `Minimized`
	• `Maximized`
	• `Normal`
	For example, if the main window is in the default window mode, and you run the following command:
	`uiGet main -windowstatus`
	the software returns:
	`Normal`
	If the window is in the background, you can use the following command to bring the window to the forefront:
	`uiSet main -visible 1`

Examples

- The following command returns the label on element named verify:

  ```
  uiGet verify -label
  ```

  The software returns the following:

  ```
  Verify
  ```

- The following command returns the names of all the menus in the main window:

  ```
  uiGet main -menu
  ```

  The software returns the following:

  ```
  file view edit partition fplan power place eco clock route timing verify pvs tools
  windows flow help
  ```

- The following command returns the names of all the toolbars in the main layout:

  ```
  uiGet main.layout -toolbar
  ```

  The software returns the following:

  ```
  tbFile tbEdit tbView tbTools tbReport tbToolBox we main.layout.selectbar
  ```

- The following command returns whether the Select bar in the main layout is movable:

  ```
  uiGet main.layout.selectbar -movable
  ```

- The following command returns the window status of the main window:

  ```
  uiGet main -windowstatus
  ```

  The software returns the following (assuming the window is in the maximized mode at the time):

  ```
  Maximized
  ```

Related Information

- uiAdd
- uiDelete
- uiFind
- uiSet
- Customizing the User Interface chapter in the *User Guide*
  - *Querying and Configuring Interface Elements*

uiSet

```
uiSet
[-help]
name
[-before name]
[-checked {true | false}]
[-command string]
[-disabled {true | false}]
[-enablevar string]
[-foreground string]
[-geometry string | -fullscreen {true | false}]
[-icon filename]
[-in name]
[-label string]
[-message string]
[-movable {true | false}]
[-newline {true | false}]
[-shortcut string]
[-statusbar string]
[-title string]
[-tooltip string]
[-underline integer]
[-value string ]
[-variable string]
[-visible {true | false}]
[-visiblevar string]
```

Sets the specified property of an element. Use this command to customize the Innovus graphical interface by configuring various interface elements, such as submenus and commands. The properties that can be set for a specific element depends on the `type` of that element. To see the list of properties that can be set for each type, see the "Accepted Properties for Each Type" section below.

For elements of type `menu, submenu, command, check, radio, separator, toolbar,` and `toolbutton`, the `uiSet` command works like the tk command *`widget`*`configure`. For the main window (`main`), it works like the tk command `wm`.

Parameters

`-help`	Prints a brief description that includes type and default information for each `uiSet` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man uiSet`
`name`	Specifies the name of the element whose property is to be set. An error is returned if specified *`name`* does not exist.
`-before` *`name`*	Sets the specified element before the element identified by *`name`*.
`-checked {true | false}`	Checks or unchecks the specified element. **Note:** You can use the `-checked` parameter only for elements of type `check` or `radio`.
`-command` *`string`*	Specifies the Tcl command to be executed when the specified element is invoked. **Note:** You can use the `-command` parameter only if you are adding elements of type `command, check, radio` or `toolbutton`.
`-disabled {true | false}`	Disables or enables the specified interface element. **Note:** You can use the `-disabled` parameter only for elements of type `menu, submenu, command, check, radio` or `toolbutton`. The parameter is NOT applicable for elements of type `toolbar` and `separator`.
`-enablevar` *`string`*	Sets the Tcl global variable for Enable status.
`-foreground` *`string`*	Specifies the foreground color of the icon of the specified interface element. **Note:** You can use the `-foreground` parameter only if you are adding an element of type `toolbutton`.
`-fullscreen {true | false}`	
	Maximizes the main window. This property can be set only for the element `main`. **Note:** This option cannot be used with `-geometry`.

`-geometry` *`string`*	Modifies the geometry of the specified window element.
`-icon` *`filename`*	Specifies the file name of the toolbutton icon. *`filename`* can be the full path of the icon file or an icon name supported by Innovus. The icon names used in Innovus can be queried using the `gui_get_ui` command. For list of supported image formats for the icon file, see the "Supported Image Formats" section of the Customizing the User Interface chapter in the *User Guide*. **Note:** The `-icon` parameter is applicable only for elements of type `toolbutton`.
`-in` *`name`*	Specifies the name of the parent element.
`-label` *`string`*	Specifies the label to be displayed on the element being modified. **Note:** This parameter is NOT applicable for elements of type `separator`.
`-message` *`string`*	Specifies the message to be displayed on the element being modified. **Note:** This parameter is applicable for elements of type `statusbar`.
`-movable {true | false}`	Specifies whether or not the element being modified should be movable. **Note:** The `-movable` parameter is applicable only for elements of type `toolbar`.
`-newline {true | false}`	Specifies whether or not the element being modified should be on a new row. **Note:** The `-newline` parameter is applicable only for elements of type `toolbar`.
`-shortcut` *`string`*	Specifies the shortcut key for a command, check box, radio button, or toolbutton. **Note:** You can use the `-shortcut` parameter only if you are modifying elements of type `command`, `check`, `radio`, or `toolbutton`.
`-statusbar` *`string`*	Sets the statusbar of the specified window element.

`-title` *string*	Specifies the window title.
`-tooltip` *string*	Specifies the tooltip text for the toolbar or toolbutton you are adding. **Note:** The `-tooltip` parameter is applicable only for elements of type `toolbar` and `toolbutton`.
`-underline` *integer*	Specifies the integer index of the character to be underlined in the element label. The index begins with `0` and should have a value less than or equal to the length of the element label. For instance, if you want the second letter of the element label to appear underlined, specify *integer* as `1`. **Note:** The `-underline` parameter is NOT applicable for elements of type `toolbar, separator,` and `toolbutton`.
`-value` *stringValue*	Specifies the value to be stored in the associated variable when the specified radio button is selected. **Note:** The `-value` parameter is applicable only for elements of type `radio`.
`-variable` *string*	When modifying a check box or radio button, specifies the Tcl global variable for the item.
`-visible {true | false}`	Specifies the visibility status of the element being modified.
`-visiblevar` *string*	Sets the Tcl global variable for Visible status.

Accepted Properties for a Specific Type

- For the main window (`main`), the following properties can be set:

  `-geometry {`*`widthxheight | widthxheight+x+y`*`}`

  `-title` *`title`*

  `-visible true | false`

- For a status bar (`statusbar`), the following property can be set:

  `-enablevar` *`string`*

  `-in` *`parentname`*

  `-message` *`msg`*

  `-visible true | false`

  `-visiblevar` *`string`*

- For elements of type `menu` or `submenu`, the following properties can be set:

  `-enablevar` *`string`*

  `-before` *`nextSiblingName`*`]`

  `-disabled true | false`

  `-in` *`parentName`*

  `-label` *`label`*

  `-underline` *`index`*

  `-visible true | false`

  `-visiblevar` *`string`*

- For elements of type `command`, `radio`, and `check`, the following properties can be set:

  `-disabled true | false`

  `-label` *`label`*

  `-underline` *`index`*

  `-command` *`commandName`*

  `-in` *`parentName`*

  `-before` *`nextSiblingName`*

  `-enablevar` *`string`*

  `-visible true | false`

  `-visiblevar` *`string`*

  `-shortcut` *`string`*

For `radio` and `check`, the `-checked` and `-variable` properties can also be set.

- For elements of type `toolbutton`, the following properties can be set:

```
-state normal | disabled
-label label
-tooltip toolTipText
-underline index
-foreground colorName
-icon iconName
-command commandName
-in parentName
-before nextSiblingName
-enablevar string
-visible true | false
-visiblevar string
-shortcut string
```

Examples

- The following command changes the label of the *Verify* menu in the main window to *NewVerify*:

```
uiSet verify -label NewVerify
```

- The following command maximizes the Innovus main window:

```
uiSet main -fullscreen
```

Related Information

- `uiAdd`
- `uiDelete`
- `uiFind`
- `uiGet`
- Customizing the User Interface chapter in the *User Guide*

- *Querying and Configuring Interface Elements*

uiSetTool

```
uiSetTool
[-help]
mode
```

Selects specified tool widget in the main window and enters interactive mode.

For more information on tool widgets, see the "Tool Widgets" section in the The Main Window chapter of the *Innovus Menu Reference*.

Parameters

`-help`	Prints a brief description that includes type and default information for each uiSetTool parameter. For a detailed description of the command and all of its parameters, use the man command: man uiSetTool.
mode {select | move | ruler | util | cut | obstruct | layerBlk | ptnFeed | ptnPinBlk | ptnPinGuide | addWire | addVia | cutWire | stretchWire | addPoly | addBusGuide | p2pRoute}	
	Specifies the interactive mode to enter. The following modes are supported: • `select` - Chooses the Select tool widget in the main window. • `move` - Selects the Move/Resize/Reshape tool widget in the main window. • `ruler` - Selects the Create Ruler tool widget in the main window. • `util` - Selects the Query Area Density tool widget in the main window. • `cut` - Selects the Cut Rectilinear tool widget in the main window. • `obstruct` - Selects the Create Placement Blockage widget in the main window. • `layerBlk` - Selects the Create Routing Blockage tool widget in the main window. • `ptnFeed` - Selects the Create Physical Feedthrough tool widget in the main window. • `ptnPinBlk` - Selects the Create Pin Blockage tool widget in the main window. • `ptnPinGuide` - Selects the Create Pin Guide tool widget in the main window. • `addWire` - Selects the Edit Wire tool widget in the main window. • `addVia` - Selects the Add Via tool widget in the main window. • `cutWire` - Selects the Cut Wire tool widget in the main window. • `stretchWire` - Selects the Move/Resize Wire tool widget in the main window. • `addPoly` - Selects the Add Polygon tool widget in the main window. • `addBusGuide` - Selects the Edit Bus Guide tool widget in the main window. • `p2pRoute` - Selects the Point to Point Route tool widget in the main window.

Example

The following command is used to enter the Create Ruler mode in the design window.

```
uiSetTool ruler
```

Related Information

- The Main Window chapter of the *Menu Reference*
    - *Tool Widgets*

undo

```
undo [-help]
```

Undoes the previous edit made with any bump placement, floorplanning, power planning, or wire edit commands listed below.

- Selected bump placement commands:
  - create_bump
  - deleteBumps
- Selected floorplanning commands:
  - alignObject
  - flipOrRotateObject
  - setObjFPlanBox
  - setObjFPlanBoxList
  - setObjFPlanPolygon
  - snapFPlan
  - snapFPlanIO
  - spaceObject
- Selected power planning commands:
  - addRing
  - addStripe
  - editPowerVia
  - sroute
- All wire editor commands:
  - editAddFillet
  - editAddPoly
  - editAddRoute
  - editAddTrimMetal

- editAddVia
- editChangeLayer
- editChangeNet
- editChangeRule
- editChangeStatus
- editChangeVia
- editCommitPoly
- editCommitRoute
- editCutWire
- editDelete
- editDeleteFillet
- editDeselect
- editDuplicate
- editFixWideWires
- editMerge
- editMove
- editRotate
- editSelect
- editSplit
- editResize
- editTrim
- prepareForEcoRoute

You can use the redo command to redo an edit that was reversed with undo.

Parameters

`-help`	Prints a brief description of the `undo` command. For a detailed description of the command, use the `man` command: `man undo`

Related Information

- `redo`
- Edit Menu chapter in the *Menu Reference*
  - *Undo*
  - *Redo*

unregister_gui_edit_callback

```
unregister_gui_edit_callback
[-help]
-command string
-edit_type {selection pre-move post-move}
```

Removes the callback function from the specified move action. In GUI, the move action involves the following three steps:

1. Click an object.
2. Move the object to the new location.
3. Click to place the object at the new location and complete the move.

The callback function is implemented based on the move action specified with the `-edit_type` parameter.

Parameters

`-help`	Prints the usage of the command. For a detailed description of the command, use the `man` command: `man unregister_gui_edit_callback`
`-command string`	Specifies the name of the callback function.
`-edit_type {selection pre-move post-move}`	Specifies the GUI edit function: • `selection` - Implements the callback function when an object is selected (after Step 1). • `pre-move` - Implements the callback function just before the object is placed at the new location (after Step 2, just before you change the coordinates in Step 3). • `post-move` - Implements the callback function after an object is moved (after Step 3, when the tool updates the DB records).

Example

Consider the following proc definition:

```
proc move_event_callback {

     set objList [dbGet selected]

     foreach obj $objList {

           set type [dbGet $obj.objType]

           if {$type == "sdp"} {

                custom_snap_sdp $obj

           } elseif {$type == "inst"} {

                set cellName [dbGet $obj.cell.name]

                if {[regexp grf $cellName]} {

                     custom_snap_grf $obj

                }

           }

}

}
```

The following command triggers the `move_event_callback` function on the selected object just before the move is completed:

```
register_gui_edit_callback -edit_type pre_move -command move_event_callback
```

The following command removes the callback to the `move_event_callback` function:

```
unregister_gui_edit_callback -edit_type pre_move -command move_event_callback
```

Related Information

- `register_gui_edit_callback`
- `report_gui_edit_callback`

viewLast

```
viewLast [-help]
```

Displays the last view window or the previous view. This command provides the same function as the Zoom Previous icon (bindkey *w*).

Parameters

`-help`	Prints a brief description of the `viewLast` command. For a detailed description of the command, use the `man` command: `man viewLast`.

Related Information

- `viewNext`
- `zoomTo`
- `zoomIn`
- `zoomOut`
- `zoomSelected`
- The Main Window chapter in the Menu Reference

viewNext

```
viewNext [-help]
```

Displays the next view window. This command provides the same function as the *Next* toolbar widget. You can use the `viewNext` command or the *Next* toolbar widget to switch from the current display to the next location in the *Go To Saved Area* list or the next zoom level in the zoom sequence saved in the design memory. The equivalent bindkey is `Y`. When you press the `Y` bindkey or execute the `viewNext` command repeatedly, the tool cycles through all saved views (saved areas and zoom levels).

Parameters

`-help`	Prints a brief description of the `viewNext` command. For a detailed description of the command, use the `man` command: `man viewNext`.

Related Information

- `viewLast`
- `zoomTo`
- `zoomIn`
- `zoomOut`
- `zoomSelected`
- The Main Window chapter in the Menu Reference

viewSnapshot

```
viewSnapshot
[-help]
-dir string
[-name string]
[-view string]
```

Opens a window `SnapShot Result` for viewing saved snapshots. Snapshots are selected based on the options: `-dir` and `-name`. The window displays design views controlled by the `-view` option and also shows QoR parameters (WNS,TNS, Placement Overlap, and congestion) saved at the time of creating the snapshot.

Parameters

`-dir` *`string`*	Specifies the directory of the saved snapshot, when opted alone it will show all snapshots saved in the directory.
`-help`	Outputs a brief description that includes type and default information for each `viewSnapshot` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man viewSnapshot`.
`-name` *`string`*	Specifies the name of snapshot to be viewed in the `SnapShot Result` window.
`-view` *`string`*	Specifies the view of the design you want to see in `SnapShot Result` window. You can see multiple views by listing them under this option. *Default:* The fplan view is shown in the `SnapShot Result` window.

Note: This command will not work in the `-nowin` mode.

Examples

- The following command will show all the snapshots present in the directory `/work/user1`, along with all QoR parameters (WNS, TNS, Placement Overlap, and Congestion). By default only the fplan view will be displayed.

  ```
  viewSnapshot -dir {/work/user1}
  ```

- The following command will show snapshots with the name `cts` along with all QoR parameters (WNS, TNS, Placement Overlap, and Congestion). By default only the fplan view will be displayed.

  ```
  viewSnapshot -dir {/work/user1} -name cts
  ```

- The following command will show snapshots with the name `cts` along with all QoR parameters (WNS, TNS, Placement Overlap, and Congestion). Only design views `cts`will be shown as opted under the -view option.

  ```
  viewSnapshot -dir {/work/user1} -name cts -view cts
  ```

- The following command will show all the snapshots present in the directory `/work/user1`along with all QoR parameters (WNS, TNS, Placement Overlap, and Congestion). All the views will be shown side by side.

  ```
  viewSnapshot -dir {/work/user1} -view {fplan amoeba place}
  ```

Related Information

- `createSnapshot`
- Tools Menu chapter of *Menu Reference*
  - *Create Snapshot*
  - *View Snapshot*

windowDeselect

```
windowDeselect
[-help]
[ux1] [uy1] [ux2] [uy2]
```

Deselects all objects within the specified area.

Note: This command works only in GUI mode.

Parameters

`-help`	Prints a brief description that includes type and default information for each `windowDeselect` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man windowDeselect`
`[ux1] [uy1] [ux2] [uy2]`	Specifies the coordinates of the window area to be deselected.

Example

The following command deselects objects that are totally enclosed by the area with coordinates defined as (`x=10`, `y=200`) and (`x=100`, `y=50`):

```
windowDeselect 10 200 100 50
```

Related Commands

- `lineSelect`
- `windowSelect`
- `windowToggleSelect`

windowSelect

```
windowSelect
[-help]
area
```

Selects all objects within the specified area.

Note: This command works only in GUI mode.

Parameters

`-help`	Prints a brief description that includes type and default information for each `windowSelect` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man windowSelect`
`area`	Specifies the coordinates of the window area to be selected. You can specify *`area`* using any of the following formats: • *`x1 y1 x2 y2`* Example: `windowSelect 10 10 20 20` • "*`x1 y1 x2 y2`*" Example: `windowSelect "10 10 20 20"` • {*`x1 y1 x2 y2`*} Example: `windowSelect {10 10 20 20}` • `[list 10 10 20 20]` Example: `windowSelect [list 10 10 20 20]` • `[dbGet $inst.box]` Example: `windowSelect [dbGet $inst.box]` Where `inst` is a variable that has already been set.

Example

The following command selects objects that are totally enclosed by the area with coordinates defined as (`x=10`, `y=200`) and (`x=100`, `y=50`):

```
windowSelect 10 200 100 50
```

Related Information

- `lineSelect`
- `windowDeselect`
- `windowToggleSelect`

windowToggleSelect

```
windowToggleSelect
[-help]
x1 y1 x2 y2
```

Toggles between selecting or deselecting objects that are enclosed by the area defined by the coordinates. Use this command to select or deselect all objects within a specific area.

Note: This command works only in GUI mode.

Parameters

`-help`	Prints a brief description that includes type and default information for each `windowToggleSelect` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man windowToggleSelect`
x1 y1 x2 y2	Specifies the coordinates of the window area to enclose the objects to be selected or deselected.

Example

The first command selects the objects that are totally enclosed by the area with coordinates defined as (`x1=10`, `y1=200`) and (`x2=100`, `y2=50`):

The second command deselects the objects that are totally enclosed by the same area:

```
windowToggleSelect 10 200 100 50

windowToggleSelect 10 200 100 50
```

Related Information

- `windowDeselect`
- `windowSelect`

zoomBox

```
zoomBox
[-help]
area
```

Zooms in the viewable window to the area that encloses the specified coordinates.

Parameters

`-help`	Prints a brief description that includes type and default information for each `zoomBox` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man zoomBox`.
`area`	Specifies the coordinates for the box representing the area into which you want to zoom.

Example

The following command zooms in the viewable window to the area enclosed by the lower-left coordinates (`450 531`) and the upper-right coordinates (`550 631`)

```
zoomBox 450 531 550 631
```

Related Information

- `viewLast`
- `viewNext`
- `zoomTo`
- `zoomIn`
- `zoomOut`
- `zoomSelected`
- View Menu chapter in the *Menu Reference*
    - *Zoom*

zoomIn

```
zoomIn [-help]
```

Zooms in the viewable window.

Parameters

`-help`	Prints a brief description of the `zoomIn` command. For a detailed description of the command, use the `man` command: `man zoomIn`.

Related Information

- `viewLast`
- `viewNext`
- `zoomBox`
- `zoomOut`
- `zoomSelected`
- `zoomTo`
- View Menu chapter in the *Menu Reference*
  - *Zoom*

zoomOut

```
zoomOut [-help]
```

Zooms out from the viewable window by two times.

Parameters

`-help`	Prints a brief description of the `zoomOut` command. For a detailed description of the command, use the `man` command: `man zoomOut`.

Related Information

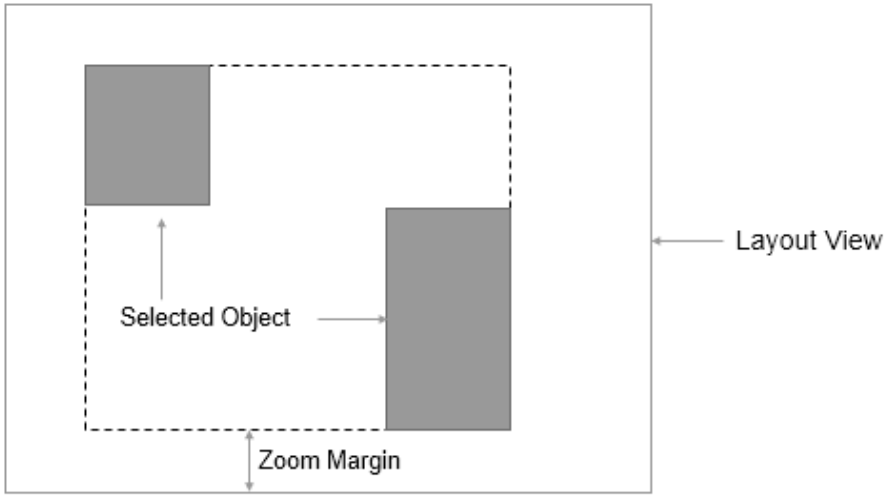
- `viewLast`
- `viewNext`
- `zoomBox`
- `zoomIn`
- `zoomSelected`
- `zoomTo`
- View Menu chapter in the *Menu Reference*
  - *Zoom*

zoomSelected

```
zoomSelected
[-help]
[-margin value]
```

Zooms in the viewable window to the area that encloses the selected objects.

Parameters

`-help`	Prints a brief description of the `zoomSelected` command. For a detailed description of the command, use the `man` command: `man zoomSelected`.
`-margin value`	Specifies the zoom margin ratio as an integral value. The zoom margin is the minimum spacing between the bounding box of the selected object(s) and the layout view.  The minimum value is `0` and the maximum value is `99`. *Default*: 50

Related Information

- `viewLast`
- `viewNext`
- `zoomBox`
- `zoomIn`
- `zoomOut`
- `zoomTo`
- View Menu chapter in the *Menu Reference*
  - *Zoom*

zoomTo

```
zoomTo
[-help]
x y
[-radius value]
```

Zooms in the viewable window to the point defined by the coordinates with a default radius of 150 microns.

You can also use the Select bar to specify the zoom area center and radius directly from the GUI.

Parameters

`-help`	Prints a brief description that includes type and default information for each `zoomTo` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man zoomTo`.
`-radius value`	Zooms in the viewable window to a radius of the specified value in microns, with the center at the point specified by *x* and *y*. The radius *value* is of type float. *Default*: 150 microns
x y	Zooms in the viewable window to the point defined by the coordinates, `x` and `y`.

Examples

- When the following command is run, the display zooms in to the point with x=100 and y=200. The display area is centered at this point and has the default radius of 150 microns:

  ```
  zoomTo 100 200
  ```

- When the following command is run, the display zooms in to point with x=100 and y=200. The display area is centered at this point and has a radius of 50 microns:

  ```
  zoomTo 100 200 -radius 50
  ```

Related Information

- `viewLast`
- `viewNext`
- `zoomBox`
- `zoomIn`
- `zoomOut`
- `zoomSelected`
- View Menu chapter in the *Menu Reference*
    - *Zoom*
- The Main Window chapter in the *Menu Reference*
    - *Select Bar*

12

Hierarchical Design Commands

- saveModel

saveModel

```
saveModel
[-help]
-dir string
[-ilm]
[-sdf]
[-spef]
[-stream [-mapFile string][-outputMacros]]
```

Saves block level design information for top level implementation. You can use this command after block implementation. The `saveModel` command saves the following data by default:

- LEF
- DEF
- Innovus DB (*.enc.dat)

For timing library (.lib) generation you need to use the `do_extract_model` command.

Parameters

`-dir` *string*	Specifies the name of the directory where you want to write the files. You can specify an additional level of hierarchy to define the model design stage, for example:
	`results_conv_model/postRoute/...`
	`results_conv_model/signoff/...`
	Data_type: string, required

`-help`	Outputs a brief description that includes type and default information for each `saveModel` parameter. For a detailed description of the command and all of its parameters, use the man command: `man saveModel`.
`-ilm`	Generates ILM models. *Data_type*: bool, optional
`-mapFile` *string*	Specifies the file used for layer mapping. This file is required for successful stream out, and is read by the next tool used in the design flow. *Data_type*: string, optional
`-outputMacros`	Writes LEF abstract information such as LEF pin geometries and obstructions for macros. Writes one text label per port within a LEF macro pin plus one text label per shape for feedthrough pins. For more information, see `streamOut`. *Data_type*: bool, optional
`-sdf`	Generates the delay data for each view in MMMC mode or for setup and hold modes. *Data_type*: bool, optional
`-spef`	Generates parasitic data if a design is at least routed. It invokes extraction, if not already extracted. For extraction, it honors the default, detailed or CCE mode user settings. *Data_type*: bool, optional
`-stream`	Generates Stream data, if the design is routed. *Data_type*: bool, optional

Examples

- The following command creates a directory called cts_condor_dma that contains the LEF (.lef and antenna.lef) files under the library directory, delay and parasitic information, and the design data of dma block:

  ```
  saveModel -sdf -spef -dir cts_condor_dma
  ```

Related Information

- write_lef_abstract
- defOut
- saveDesign
- write_sdf
- rcOut
- streamOut
- do_extract_model
- Partitioning the Design chapter in the *Innovus User Guide*

13

ILM and FlexILM Commands

- commit_module_model
- createInterfaceLogic
- flattenIlm
- getIlmMode
- getIlmType
- import_ilm_data
- read_ilm_eco_db
- reportIlmStatus
- setIlmMode
- setIlmType
- specifyIlm
- unflattenIlm
- unspecifyIlm
- update_partition
- write_ilm_eco_db

commit_module_model

```
commit_module_model
[-help]
[-copy_mmmc_latencies]
[-merged_mmmc_map_file mapFiles]
[-merged_mmmc_sdc_files mergedSdcFiles]
[-mmmc_file fullChipSdc]
[-mmmc_module_model_tag modelTag]
[-propagate_clocks]
[-cpf_file filename | -1801 filename]
```

Commits module models.

Parameters

`-copy_mmmc_latencies`	Copies clock latencies of all analysis views at the top-level design and applies them after assembling the design:
	When this option is specified, the command will do following after assembling the design:
	`foreach view [all_analysis_views] {`
	`update_analysis_view -name $view -latency_file $latency_map($view)`
	`}`
	Where `latency_map` has clock latency files for all views.

`-cpf_file` *`filename`*	Specifies the name of the full chip CPF file to be used.
`-help`	Outputs a brief description that includes the type and default information for each `commit_module_model` parameter. For a detailed description of the command and all of its parameters, use the man command: `man commit_module_model`
`-merged_mmmc_map_file` *`mapFiles`*	Specifies merge hierarchical mmmc map files.
`-merged_mmmc_sdc_files` *`mergedSdcFiles`*	Specifies SDC files for different dies.
`-mmmc_file` *`fullChipSdc`*	Specifies the name of the full chip MMMC view file.
`-mmmc_module_model_tag` *`modelTag`*	Specifies MMMC module model tag.

`-propagate_clocks`	Changes clock ideal to propagated clock. When this option is specified, you can run the following commands:
	`set_interactive_constraint_modes [all_constraint_modes -active]`
	`set_propagated_clock [all_clocks]`

`-1801` *`filename`*	Specifies the name of the IEEE 1801 file to be used.

Related Information

- set_interactive_constraint_modes
- set_propagated_clock
- update_analysis_view

createInterfaceLogic

```
createInterfaceLogic
[-help]
[-allowIlmEco]
[-ignorePorts ListOfPorts]
[-keepAll][-keepSelected]
[-modelType {timing si all}]
[-noInterClockPath]
[-overwrite]
[-removePowerGround]
[-dir dirName | -cellview {lib | cell | view}]
[ [-optStage {preCTS postCTS}] ]
```

Creates the specified directory containing ILM files with the appropriate level of extraction for specific model type such as timing, si, or for all models. For MMMC, ILM models will be generated based on constraint modes.

The `createInterfaceLogic` command creates a reduced set of instances representing only the interface paths, that is, paths from the I/O ports of the design to the first level of sequential instances (latches are considered sequential elements).

The command automatically modifies the module name with the following rule:

```
topModuleName+timestamp+$+moduleName
```

Parameters

`-help`	Outputs a brief description that includes type and default information for each `createInterfaceLogic` parameter. For a detailed description of the command and all of its parameters, use the man command: `man createInterfaceLogic`
`-allowIlmEco`	Allows ECO changes within the ILM block.
`-cellview {lib | cell | view}`	
	Specifies OA cell view information for generated ILMs. The `-cellview` option cannot be used in non-OA flow. For non-OA flow, you use the `-dir` option instead.

`-dir` *dirName*	Specifies the output directory name where ILMs file will be saved. If the design is not an OA design, this option must be specified.
`-keepAll`	Preserves the complete logic of block and hence does not do any logic pruning based on constraints or path groups. **Note:** Since the complete logic is preserved, so all the ILM views, that is, timing, CTS, and SI share the same model data.
`-ignorePorts` *ListOfPorts*	
	Specifies list of ports where these ports and the associated interface path(s) which are starting or ending at these ports will not be included in the ILM model.
`-keepSelected`	Preserves all selected instances and nets (in addition to those normally kept) when you create ILMs. Selected instances are those instances that drive the selected nets. You can select instances or nets using the `selectInst` or `selectNet` commands, then use `createInterfaceLogic -keepSelected`. By default, no additional instances and nets are preserved.
`-modelType {timing | si | all}`	
	Specifies the type of ILM model to be created. Default: `all` • `all`: Ensures that all models are written. • `timing`: Ensures that only the timing model is written for non-SI based timing, optimization, or clock tree commands. • `si`: Ensures that only the SI model is written that can be used for the SI based timing and optimization commands.
`-noInterClockPath`	Minimizes the clock latency at the upper-level CTS when the non-boundary flop in ILM has the maximum clock latency.
`-optStage {preCTS | postCTS}`	
	Specifies the optimization stage, preCTS or postCTS. This option should be specified both for ILM and flexILM creation.

`-overwrite`	Overwrites existing files.
`-removePowerGround`	Removes PG ports, PG nets, PG pin connections, and replaces PG net connections on signal pins with 1'b1 or 1'b0 .

Examples

- The following command generates both timing model and SI model in directory BlockA:
  ```
  createInterfaceLogic -dir BlockA
  ```
- The following command only generates timing model and saves it in directory BlockA:
  ```
  createInterfaceLogic -dir BlockA -modelType timing
  ```
- The following command completely replaces the library, cell and view in an OA flow.
  ```
  createInterfaceLogic -cellview {lib | cell | view} -overwrite
  ```

Note: In the current ILM flow, SDC constraints originally specified against the complete flat netlist for the design get filtered when being applied against the ILM view of the design.

Note: The `setIlmMode` command settings also affect the instances that `createInterfaceLogic` command will preserve to model the block.

Sample Summary Report

The following is a sample summary report generated at the end of the `createInterfaceLogic` command:

createInterfaceLogicSummary

Model	Reduced Instances	Reduced Registers
ilm_data	7153/7621 (93%)	174/285 (61%)
si_ilm_data	6793/7621 (89%)	160/285 (56%)

In this report, the reduction ratio in the `ilm_data` model is 93 percent which means that 7153 out the total 7621 instances for this block have been eliminated. Only 468 instances are written to the Verilog netlist for the `ilm_data` model out of which 111 instances are registers.

This summary report applies to a block using MMMC. Therefore, views with worst reduction ratio are displayed for each model.

Related Information

- `setIlmMode`
- *Creating ILMs for Shared Modules* topic in the Top-level Timing Closure Methodologies chapter of the *Innovus User Guide*.

flattenIlm

```
flattenIlm
[-help]
```

Flattens the interface logic models (ILMs) at the top level so that the entire design can be analyzed at the top. When you use this command, the software switches from blackbox view to ILM view.

If you do not run `flattenIlm`, the ILM netlist will not be visible to the Innovus software. You can switch between the ILM view and the blackbox view using the `flattenIlm` and `unflattenIlm` commands.

All super commands such as `timeDesign`, `place_design`, and `optDesign` flatten the ILMs internally and hence no explicit `flattenILM` is required. However, you should use this command for other timing related commands. For example, for the `report_timing` command, you need to run `flattenILM` before reporting timing. It would not work in the unflattened mode.

Example:

```
flattenIlm

report_timing
```

Parameters

`-help`	Outputs a brief description that includes the usage of the `flattenIlm` command.

Related Information

- `optDesign`
- `place_design`,
- `report_timing`
- `timeDesign`

getIlmMode

```
getIlmMode
[-filterInternalPath]
[-ilm]
[-ilm ilmName]
[-keepAsync {true|false}]
[-keepFlatten {true|false}]
[-keepHighFanoutPorts {true|false}]
[-keepInstInSdc {true|false}]
[-keepLoopBack {true|false}]
[-map {view|corner}]
[-maxNumInsts]
[-maxNumRegisters]
[-resetMap {view|corner}]
[-slackDriven]
[-top topViewName]
[-quiet]
```

Displays information about the specified `setIlmMode` parameter in the Innovus log file and in the Innovus console. Displays the following information.

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the software displays information for all of the `setIlmMode` parameters.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `getIlmMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man getIlmMode`
`parameter_names`	
	Displays information for the specified parameters. You can specify one or more parameters.
`-quiet`	
	Displays the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software displays the current settings of all `setILMMode` parameters in Tcl format.

Examples

- The following example displays the current setting for the `-keepAsync` parameter:

  ```
  getIlmMode -keepAsync
  ```

  The software displays the following information:

  ```
  -keepAsync false # bool, default=false

  false
  ```

- The following command displays the current settings for all `setIlmMode` parameters:

  ```
  getIlmMode
  ```

  The software displays the following information:

  ```
  -keepAsync false # bool, default=false
  ```

```
-keepHighFanoutPorts true # bool, default=true

-keepLoopBack false # bool, default=false

{keepAsync false} {keepHighFanoutPorts true} {keepLoopBack false}
```

- The following command displays the current setting for the `-keepHighFanoutPorts` parameter in Tcl list format only:
  ```
  getIlmMode -keepHighFanoutPorts -quiet
  ```
  The software displays the following information:
  ```
  true
  ```
- The following command reports which view of ILM, `blkA`, has been mapped to the top analysis view of `maxDelay`:
  ```
  getIlmMode -map view -top maxDelay -ilmCell blkA
  ```
- The following command reports all the corner mappings of ILM, `blkA`, that have been mapped to the top analysis corners:
  ```
  getIlmMode -map corner ilmCell blkA
  ```
- The following command reports all the view mappings for all ILMs:
  ```
  getIlmMode -map view
  ```
- The following command displays the current settings for all `setIlmMode` parameters in Tcl list format only:
  ```
  getIlmMode -quiet
  ```
  The software displays the following information:
  ```
  {keepAsync false} {keepHighFanoutPorts true} {keepLoopBack false}
  ```

Related Information

- `setIlmMode`

getIlmType

```
getIlmType
[-help]
[-model]
```

Displays information about the specified `setIlmType` in the Innovus log file and the Innovus console. Displays the following information:

- Parameter name
- Current value
- Type (boolean, string, and so on)
- The current value set by the user

If you do not specify a parameter, the Innovus software displays information for all of the `setIlmType` parameters.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `getIlmType` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man getIlmType`
`-model`	Outputs the model.

Examples

- The following example displays the current setting for the `-model` parameter:

  ```
  getIlmType -model
  ```

  The software displays the following information:

  ```
  timing
  ```

Related Information

- `setIlmType`

import_ilm_data

```
import_ilm_data
[-help]
[-cell block_name]
[-incr]
-model_type {timing | cts | si}
[-overwrite]
[-xtwf xtwf_file.spef]
{-dir directory_name | -cell_view lib cell view}
[-verilog cell_name.v [-cell_prefix cell_prefix -uniquify_netlist]]
[-sdc file_name -timing_view view_name]
[-spef spef_file.spef -rc_corner RC_corner_name]
[-optStage opt_stage]
```

Creates an ILM directory from existing data. It can only create non-merged models. This command is useful in situations where an ILM is to be created through external inputs or using data from third party models into a format that could be recognized by Innovus.

If the data files already exist in the specified directory, the tool stops and generates an error message. You can overwrite existing files by specifying `-overwrite`. You can add files to the directory by specifying `-incr`.

The command also creates a tcl script (cell_name_ILM.tcl) that you can source in your command file. The script contains the following command:

```
specifyIlm -cell cell_name -dir dir_name
```

Parameters

`-cell block_name`	Specifies the name of the block.
`-cell_prefix cell_prefix`	Specifies the cell prefix.
`-cell_view lib cell view`	Specifies the library name, cell name, and view name in the OpenAccess database format.
`-dir dir_name`	Stores the ILM data in the specified directory: the absolute directory path or the path relative to the current directory.

`-incr`	Lets you add files in addition to the existing files to the directory.
`-model_type {timing | cts | si}`	Specifies the type of ILM model, i.e timing, cts or si,.
`-help`	Outputs a brief description that includes the type and default information for each `import_ilm_data` parameter. For a detailed description of the command and all of its parameters, use the man command: man `import_ilm_data`.
`-optStage`	Specifies the optimization stage, preCTS or postCTS.
`-overwrite`	Overwrites existing files in the directory.
`-rc_corner` *RC_corner_name*	Specifies the RC corner name for MMMC analysis. It identifies the specified corner to be used with -spef.
`-sdc` *file_name*	Specifies an existing .sdc file that you want to include in the directory. It identifies the sdc file to be used with the specified view.
`-spef` *spef_name*`.spef`	Specifies an existing .spef file that you want to include in the directory. It identifies the spef file to be used with the specified corner.
`-timing_view` *view_name*	Specifies the view name for the ILM module. It identifies the specified view to be used with `-sdc`.
`-uniquify_netlist`	Uniquifies the input Verilog netlist. This ensures that there are no errors when ILMs are being loaded. Using this parameter, each sub-module for an ILM has a unique name, which is composed of the time stamp and its original name. For example, in top_sm WedJun31138212009$data_path, top_sm specifies the top-level design, WedJun31138212009 specifies the time stamp, and $data_path specifies the original module name. *Default*: `Off`
`-verilog` *netlist_name*`.v`	Specifies the verilog file you want to include in the ILM data directory.
`-xtwf` *xtwf_file.spef*	Specifies the .xtwf file that you want to include in the directory.

Examples

- The following commands create two module directories:

  ```
  import_ilm_data -cell blk1 -dir A -model_type timing -verilog V1.v -spef 1.spef -
  rc_corner corner1 -sdc sdcfile1 -timing_view view1


  import_ilm_data -cell blk2 -dir A -model_type si -verilog V2.v -spef 2.spef -
  rc_corner corner2 -sdc sdcfile2 -timing_view view2
  ```

- The command creates two tcl files in the current directory. The `blk1_ILM.tcl` file contains the following command:

  ```
  specifyIlm -dir A -cell blk1
  ```

  The `blk2_ILM.tcl` file contains the following command:

  ```
  specifyIlm -dir A -cell blk2
  ```

Related Information

- `specifyIlm`

read_ilm_eco_db

```
read_ilm_eco_db
[-help]
{-dir directoryName | -module_model_tag tagName}
```

Reads an ECO file after restoring a block-level design for non-iHDB flow.

- For iHDB flow, you should use `restore_module_model -add_ons {eco}` command to read the ECO file. Internally, `restore_module_model -add_ons {eco}` will source `<cellName>_eco.tcl` script that internally calls the `read_ilm_eco_db -module_module_tag` command.
- For non-iHDB flow, if you use `write_ilm_eco_db -module_model_tag`, you should use `read_ilm_eco_db -module_model_tag` to read an ECO file at block-level design.

Parameters

`-help`	Outputs a brief description of the `read_ilm_eco_db` command. For a detailed description of the command, use the man command: `man read_ilm_eco_db`
`-dir` *`directoryName`*	Specifies the directory name to write the ILM eco database. At the block-level design, if you use this command to read in the ECO changes, the command will automatically find out about the design name and only load/read the ECO file for this specific design.
`-module_model_tag` *`tagName`*	Specifies the tag name to use for iHDB. Use this option if `write_ilm_eco_db -module_model_tag` is used to save ECO file(s).

Related Topics

- restore_module_model
- write_ilm_eco_db

reportIlmStatus

```
reportIlmStatus
[-help]
```

Reports ILM status information including ILM block names, ILM data locations, and if the design is in the flattened or unflattened state.

You can use this command at any time in the flow, but it only reports ILM information if you have already specified ILMs in the design.

Parameters

`-help`	Outputs a brief description of the `reportIlmStatus` command. For a detailed description of the command, use the man command: `man reportIlmStatus`

Examples

The following command generates the ILM status report for `dtmf_recvr_core`:

```
reportIlmStatus

================================================================================
Current design: dtmf_recvr_core
Current view is ILM (after flattenIlm)
Current ILM data is for si mode.
================================================================================

Cell          Model-Type        Use-Type          Stage          Instance Name
--------------------------------------------------------------------------------
results_conv     All              ILM               -          RESULTS_CONV_INST
tdsp_core        All              ILM               -          TDSP_CORE_INST

ILM/flexILM Data Location :
--------------------------------------------------------------------------------
results_conv : .../libs/ilm/results_conv.ilm
tdsp_core : .../libs/ilm/tdsp_core.ilm
================================================================================
```

setIlmMode

```
setIlmMode
[-help]
[-reset]
[-filterInternalPath {true|false}]
[-keepAsync {true|false}]
[-keepHighFanoutPorts {true|false}]
[-keepInstInSdc {true|false}]
[-ilm ilmName]
[-ilmCell ilmCellName]
[-keepFlatten {true|false}]
[-keepLoopBack {true|false}]
[-map {view|corner}]
[-maxNumInsts maxInstances]
[-maxNumRegisters maxRegisters]
[-resetMap {view|corner}]
[-slackDriven {true|false}]
[-top topViewName]
```

Controls certain behaviors of ILM commands.

Use the `getIlmMode` command to display the current settings for the `setIlmMode` command.

You can set certain options through `setIlmMode` which can control how ILMs are created or the way top level is mapped to ILMs in case of mismatch between view/corner naming between block and top.

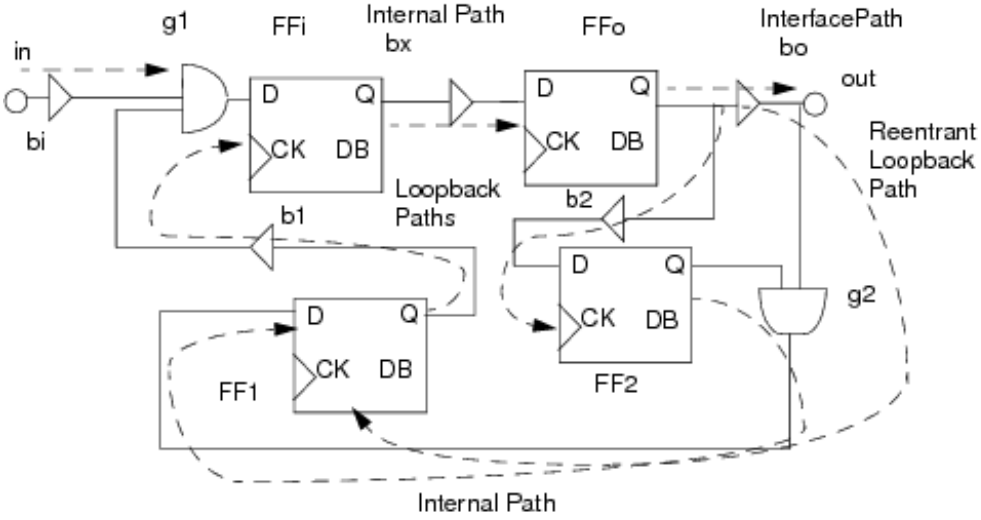
Parameters

`-help`	Outputs a brief description that includes type and default information for each `setIlmMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setIlmMode`.
`-filterInternalPath {true|false}`	Filters the internal path kept by ILM. When set to `true`, group paths are created for internal paths with slack adjustment of 9999. Therefore, all internal paths are filtered out from the timing report. *Default*: `false`

`-keepAsync {true | false}`	Keeps paths from asynchronous input ports when ILMs are created by `createInterfaceLogic`. For example, paths from an input port to the reset pin of interface sequential instances, but not to the internal sequential instances. *Default:* `false`
`-keepHighFanoutPorts {true | false}`	Keeps high-fanin outputs and high-fanout inputs when ILMs are created by `createInterfaceLogic`. *Default:* `true`
`-keepInstInSdc {true|false}`	Keeps instances which have constraints on it. *Default:* `false`
`-ilm ilmViewName |ilmCornerName`	Specifies the view or corner name of an ILM that is to be mapped to the top level.
`-ilmCell` *`ilmCellName`*	Specifies the cell name of an ILM that has view or corner mapping to the top level.
`-keepFlatten {true|false}`	Makes flow work under the flattenIlm mode. *Default:* `true`
`-keepLoopBack {true | false}`	Keeps loop back paths when ILMs are created by `createInterfaceLogic`. When set to `true`, then any path touching a net used for the ILM model is also included. This is done for better model accuracy, so that the actual capacitance on an output port could affect the delay on a net which feeds an internal path that is normally not included in the ILM model. This also degrades the reduction ratio (`#ILM_insts/#all_inst`) for the block. *Default:* `false`
`-map {view | corner}`	Maps a view or corner specified in an ILM to a view or corner of the top-design analysis. *Default*: `view`

`-maxNumInsts` *maxInstances*	Specifies the maximum number of instances. *Default*:1000
`-maxNumRegisters` *maxRegisters*	Specifies the maximum number of registers per port to be considered as high fanout ports. This setting will be honored by `-keepHighFanoutPorts`. *Default*: 40
`-reset`	Resets all or specified options to default value. Any values that do not already match the default value and are reset will be reported.
`-resetMap {view | corner}`	Resets view or corner mapping between an ILM and the top level. *Default*: `view`
`-slackDriven {true|false}`	Specifies slack driven ILM generation. *Default:* `false`
`-top` *topViewName* | *topCornerName*	Specifies the view or corner name of top design analysis while creating the mapping between the ILM and the top level. *Default*: ""

Loop-Back Paths



Examples

- The following command does not keep high fanout ports when ILMs are created by `createInterfaceLogic`:

  ```
  setIlmMode -keepHighFanoutPorts false
  ```

- The following command resets the `-keepHighFanoutPorts` parameter to its original value:

  ```
  setIlmMode -reset -keepHighFanoutPorts
  ```

- The following command undoes all the view mapping for ILM `modA`:

  ```
  setIlmMode -resetMap view -ilmCell modA
  ```

- The following command maps the view of `maxCnr_setup` of the ILM `blkA` to the top design analysis view of `rcMAX_setup`:

  ```
  setIlmMode -map view -top rcMAX_setup -ilmCell blkA -ilm maxCnr_setup
  ```

- The following command undoes all the corner mapping for ILM `blkA`:

  ```
  setIlmMode -resetMap corner -ilmCell blkA
  ```

Related Information

- `createInterfaceLogic`.
- `getIlmMode`
- Top-level Timing Closure Methodologies chapter of *Innovus User Guide*.

setIlmType

```
setIlmType
[-help]
[-model {timing | si}]
```

Controls the behavior of commands in the presence of ILMs.

ILM models can be an SI model or a non-SI model. In the flow, sometimes an explicit `flattenIlm` needs to be performed to perform the underlying operation. Based on the nature of operation, an appropriate model could be invoked/switched with the above command.

Use the `getIlmType` command to display the current settings for the `setIlmType` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setIlmType` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setIlmType`.
`-model {timing | si}`	Specifies the ILM model that is to be used by the `flattenIlm` command. All the timing-related commands require data to be in the flattened state (physical command require unflattened state). Design commands such as `optDesign` and `timeDesign` automatically set this parameter before flattening ILMs. However, the CTE commands and some clock commands, which can be run in the flattened mode require manual settings. • Use the `si` model for flattening the SI ILM model. • Use the `timing` model for flattening the non-SI related timing commands. **Note:** In case of non-merged models, use the `cts` model before running `flattenIlm` followed by the command which directly deals with the clocks. *Default*: `timing`

Examples

- The following command is used to report clock tree in the presence of ILMs:

  ```
  flattenIlm
  reportClockTree
  unflattenIlm
  ```

  The `unflattenIlm` command resets `-model` to its default value `timing` since the next command is not a clock command.

- The following command can be used to get timing reports containing the SI push-out delays on nets using the `-setIlmType -model` command:

  ```
  setIlmType -model si

  # Flattens to SI model

  flattenIlm

  # Turns on SI aware delay calculation

  setDelayCalMode -SIAware true

  # Adds incremental delay column (for SI push-out delays)
  #  in timing output:

  set_global_report_timing_format {instance arc cell fanout load slew delay
  incr_delay arrival}

  # Minimizes the width of the report such that it easily
  # fits into the screen without wrapping

  set_table_style -name report_timing -no_frame -indent 0

  report_timing
  ```

Note: You can also invoke the Global Timing Debugger (*Timing - Debug Timing - Generate*)

Related Information

- `flattenIlm`
- `getIlmType`
- `optDesign`
- `timeDesign`
- Top-level Timing Closure Methodologies chapter of *Innovus User Guide*.

specifyIlm

```
specifyIlm
[-help]
{[-cell cellName [-dir dirName]] | -cellview lib cell view}
```

Specifies ILM data directory for the specified block. You can run `specifyIlm` multiple times in the same session. Each time you specify `specifyIlm`, the software overwrites the previous setting for the block. If master/clones exist in the design, the cell name will have the name of the master partition.

Note: You can use this command only if the ILMs are unflattened (`unflattenIlm`) or the design is in black box view. You cannot change ILM settings in flattened or ILM view.

The syntax is order-dependent. You must specify `-cell` before you specify `-dir`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `specifyIlm` parameter. For a detailed description of the command and all of its parameters, use the man command: `man specifyIlm`
`-cell` *cellName*	Specifies the block name.
`-cellview` *lib cell view*	Specifies the library name, cell name, and view name in the OpenAccess database format.
`-dir` *dirName*	Specifies the directory for the files you want to read. **Note:** This parameter is optional. ⚠ For OA flow, the `-dir` option is not required and `specifyIlm` is used as follows: `specifyIlm -cell pll_fbdiv` The flow automatically finds the OA path when the `-dir` for `specifyIlm` is not set.

Examples

- The following commands create ilms and specify ilms for an OA design:

```
#create ilms for each block
createInterfaceLogic -cellview {OA_PTN_LIB tdsp_core abstract}
createInterfaceLogic -cellview {OA_PTN_LIB results_conv abstract}
#specify ilms in top level
specifyIlm -cell tdsp_core
specifyIlm -cell results_conv
```

- The following commands specifies directory `dir_dsp_core` for block `tdsp_core`, and directory `dir_onemore` for block `onemore`:

```
specifyIlm -cell tdsp_core -dir dir_dsp_core
specifyIlm -cell onemore -dir dir_onemore
```

- The following commands show how to reset the ILM directory from `dir_tdsp_core` to `dir2_tdsp_core`:

```
specifyIlm -cell tdsp_core -dir dir_tdsp_core
specifyIlm -cell tdsp_core -dir dir2_tdsp_core
```

  These commands are equivalent to the following commands, which include `unspecifyIlm`:

```
specifyIlm -cell tdsp_core -dir dir_tdsp_core
unspecifyIlm -cell tdsp_core
specifyIlm -cell tdsp_core -dir dir2_tdsp_core
```

Related Information

- `createInterfaceLogic`
- `unflattenIlm`
- `unspecifyIlm`
- Top-level Timing Closure Methodologies chapter of *Innovus User Guide*.

unflattenIlm

```
unflattenIlm
[-help]
```

Reverts the ILM back to its original hard macro view.

You can use this command after the `flattenIlm` command.

Note: `unflattenIlm` will take no effect if the design is already in the unflattened mode.

Many physical commands would not work in the flattened mode. They would need to be unflattened before performing the operation.

For example:

```
flattenIlm

report_timing

unflattenIlm
```

Parameters

`-help`	Outputs a brief description that includes the usage of the `unflattenIlm` command.

Example

The following command flattens all partitions and their ILM designs to the top-level. After reporting timing, the design is reverted back to the unflattened state with the `unflattenIlm` command.

```
flattenIlm

report_timing

unflattenIlm
```

Related Information

- `flattenIlm`
- `report_timing`
- Top-level Timing Closure Methodologies chapter of *Innovus User Guide*.

unspecifyIlm

```
unspecifyIlm
[-help]
-cell cellName
```

Unspecifies the ILM cell.

This is used to unspecify a previously specified ILM. This would be useful in situations where ILMs are to be removed from the design, to put them in non-ILM or black box mode.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `unspecifyIlm` parameter. For a detailed description of the command and all of its parameters, use the man command: `man unspecifyIlm`
`-cell` *`cellName`*	The name of the block to unspecify.

Related Information

- `specifyIlm`
- Top-level Timing Closure Methodologies chapter of *Innovus User Guide*.

update_partition

```
update_partition
[-pinLocation]
[-postECOSuffix postECOSuffix]
{-flexIlmECO}
{-goldenBlockDir directory}
-flexIlmDir directory
[-ecoDir directory]
[-fullNetlistWithECODir directory -flexILMECODir directory]
```

Creates a new partitioned output that contains the optimised blocks and top. It also performs ECO on the original block. The ECO output is saved as new Innovus database in the partitioned block. The ECO would happen only on `flexILM` partitions and not the regular blocks.

Parameters

`-ecoDir` *directory*	Specifies the scratch directory for partitioning and running distributed ECO job after optimization.
`-flexIlmDir` *directory*	Specifies the new partitioned output directory containing optimized blocks/top and ECO results.
`-flexIlmECO`	Performs ECO on FlexILM modules. **Note:** This is default.
`-flexILMECODir` *directory*	Specifies the postOpt FlexILM data directory.
`-fullNetlistWithECODir` *directory*	Specifies the directory to store full netllist ECO result.
`-goldenBlockDir` *directory*	Specifies the golden database directory for individual blocks.
`-pinLocation`	Updates new partition pin locations into the golden block DB. This option does not allow new added logic ports.
`-postECOSuffix` *postECOSuffix*	Specifies the suffix name that will be part of generated postECO DB name.

Examples

The following command will perform ECO on the block of `tdsp_core`, `ram_128x16_test` and `ram_256x16_test`.The ECO output is saved as new Innovus database with the suffix "`_postECO`" at the "`../flexILM_postOpt`" directory (which is defined by the `-flexIlmDir` option) for each specified partition block.

```
update_partition -flexIlmECO \
-goldenBlockDir ../../PTN/tdsp_core/tdsp_core_data.enc.dat \
-goldenBlockDir ../../PTN/ram_128x16_test/ram_128x16_test_data.enc.dat \
-goldenBlockDir ../../PTN/ram_256x16_test/ram_256x16_test_data.enc.dat \
-flexIlmDir ../flexIlm_postOpt -postECOSuffix postECO
```

After `update_partition`, a new dir of `flexIlm_postOpt` will be generated. Under this directory, `ecoDataSummary.rpt` will be generated to show summary for post ECO data and an example for `assembleDesign` to use post ECO data.

```
# Post ECO Data Summary
##########################################################################
##
TOP data: ../flexIlm_postOpt/dtmf_recvr_core
ram_256x16_test data:
../flexIlm_postOpt/ram_256x16_test/ram_256x16_test_postECO.enc.dat
ram_128x16_test data:
../flexIlm_postOpt/ram_128x16_test/ram_128x16_test_postECO.enc.dat
tdsp_core data: ../flexIlm_postOpt/tdsp_core/tdsp_core_postECO.enc.dat
##########################################################################
##
#To assemble postECO full netlist data, please run
assembleDesign -fe -topDir ../flexIlm_postOpt/dtmf_recvr_core\
-blockDir ../flexIlm_postOpt/ram_256x16_test/ram_256x16_test_postECO.enc.dat\
-blockDir ../flexIlm_postOpt/ram_128x16_test/ram_128x16_test_postECO.enc.dat\
-blockDir ../flexIlm_postOpt/tdsp_core/tdsp_core_postECO.enc.dat\
-mmmcFile <full chip viewDefinition file>
##########################################################################
##
```

Note: if there is no `-postECOSuffix` specified, a suffix of "`eco_<system time>`" will be added to the ECO database.

write_ilm_eco_db

```
write_ilm_eco_db
[-help]
-cell tcl_obj]
{-dir directoryName  | -module_model_tag tagName}
```

Saves netlist and placement information changes::

- Generates a binary ECO file that can be read/loaded back to a block-level design
- Generates the ECO TCL script with `read_ilm_eco_db -module_model_tag` for iHDB
- Output as corresponding ECO report file for viewing purpose

Parameters

`-help`	Outputs a brief description of the `write_ilm_eco_db` command. For a detailed description of the command, use the man command: `man write_ilm_eco_db`
`-cell` *`tcl_obj`*	Specifies a cell name for ECO information to be written out. If the specified cell name does not have any ECO change, both ECO and report files will be empty. If this option is not specified, the command will write ECO information for all ILMs specified in the design EVEN an ILM cell is not allowed for ECO. This to make sure accumulated ECO information will be preserved/saved as an ILM can be opened and then closed later during the flow. If there is no ECO change done to a particular cell, its ECO and report files will be empty.
`-dir` *`directoryName`*	Specifies a directory name where ECO files will be saved. All the ECO and/or report files should be saved under the convention *`dirName/cellName/cellName.eco`* and *`dirName/cellName/cellName.rpt`*, respectively.
`-module_model_tag` *`tagName`*	Specifies the model tag name where ECO file, ECO TCL, and its report will be saved. This option should be used for Stylus hierarchical DB.

Related Topics

- read_ilm_eco_db

14

Import and Export Commands and Global Variables

- Import and Export Commands
- Import and Export Global Variables

Import and Export Commands

- check_design
- checkDesign
- checkUnique
- createTrack
- defComp
- defIn
- defOut
- defOutBySection
- deleteModule
- deleteTrack
- disconnectDanglingPort
- dumpOutVias
- generateLef
- generateVias
- genPinText
- get_abstract_mode
- getCmdLogFileName
- getDesignMode
- getExportMode
- getGenerateViaMode
- getImportMode
- getLogFileName
- getStreamOutMode
- init_design
- loadDefFile
- loadLefFile
- read_name_mapping
- read_stream
- relink_db_files
- replaceLefMacro
- report_gates
- report_sequential
- restoreDesign
- run_abstract

- saveDesign
- saveNetlist
- set_abstract_mode
- set_cell_power_domain
- setDesignMode
- setExportMode
- setGenerateViaMode
- setImportMode
- setLibraryUnit
- setNet
- setStreamOutMode
- specify_lib
- streamOut
- trimDesign
- write_do_lec
- write_lef_abstract
- write_name_mapping

check_design

```
check_design
[-help]
[-out_file fileName]
{[-type {library netlist power_intent timing hierarchical pin_assign budget assign_statements place opt cts route signoff all}] [-no_check
]}
```

Checks that all prerequisites to run all or part of the hierarchical or block implementation flow are met.

The `check_design` command checks the preconditions for major flow steps before they are run. If there are errors identified by `check_design`, the current script will stop.check. Note that output to console and log file is controlled by message display limit. A complete list of errors is dumped to `-out_file`.

Some examples of problems that will cause design checks to fail include:

- Cells used in the design that cannot be legally placed
- Tracks misaligned with some or all cell pins
- In advanced processes, power nets that cannot be colored in the expected pattern once cells are placed
- Cell rows that are unblocked for placement but where pin layers are obstructed
- Cell rows that are unusable due to orientation and power rail misalignment
- Cells instantiated in the netlist that have no timing model defined
- Tracks blocked by power grid spacing constraints
- Instances not covered by a true timing path in any view
- ILM model with no clock tree defined
- No hold fixing cells are specified and delay cells are not usable
- No clock inverters, buffers, clock gates or logic cells are specified and the library cells are not usable for clock trees
- CTS cannot balance all skew groups without increasing insertion delay
- Preserved nets and cells in the clock tree that will violate other constraints because they cannot be resized or buffered.

Parameters

`-help`	Prints a brief description that includes a list of parameters, their usage, and their defaults. For a detailed description of the command and all of its parameters, use the `man` command: `man check_design`. No other args allowed with `-help`

`-type`	Specifies the category of the flow to be checked.
	The intent is to run `check_design` before each of the operation in the list. Each of the categories run several checks related to that operation.
	• `library`: Checks for the integrity of the library and for correct binding of all design instances to library cells or RTL modules. Individual checks are as follows:
	  ◦ For physical_only library, check_design will flag libcells that have only LEF but no .lib representation.
	  ◦ For logical_only library, check_design will flag libcells that have only .lib but no LEF representation.
	• `netlist`: Checks the netlist integrity at any point in the flow.
	  ◦ For undriven netlist, check_design will check the list of input/inout hpins/pins with no driver on the net connected to it (driver can be input port or output hpin/pin).
	  ◦ For multiple_driver netlist, check_design will check the list of input/inout pins or output/inout ports with more than one driver on their net)
	  ◦ For unloaded netlist, check_design will check the input/inout ports or output hpins/pins with no load (input/inout pins) in their fanout or hinsts/insts that have nothing connected to any output pin)
	  ◦ For constant netlist, check_design will check the output ports or input pins/hpins that are driven by a logical constant 0/1.
	  ◦ For feedthrough netlist, check_design will check the hinsts, where every output pin is directly driven by an input pin, w/o any logic in between).
	  ◦ For empty_modules, check_design will reports empty modules in the design.
	• `library`: Checks the library integrity and corrects binding of all design instances to library cells or RTL modules.
	• `power_intent`: Checks related to MSV setup including power domain/fence checks.
	• `timing`: Performs checks related to timing setup integrity. This check is run prior to running any operation that requires timing information. **Note**: The `check_timing` command should still be used for more exhaustive timing checks.
	• `hierarchical`: When you use the `hierarchical`, `pin_assign`, or `budget` options with the `-type` parameter, the `check_design` command checks for hierarchical partitions in the design. It performs checks related to the hierarchical flow. This type checks if the partition has a fence and if the fence is inside the core boundary. If not, it displays an error message. This check is run before running hierarchical design operations.
	• `pin_assign`: Checks related to pin assignment for partitioning flows. This check is run before pin assignment.
	• `budget`: Checks related to time budgeting for partitioning flows. This check is run before timing budgeting.
	• `assign_statements`: Performs checks for the netlist for assign statements. This check is run before exporting a netlist for signoff.
	• `place`: Checks related to placement such as PG rail alignment, pin access issues, missing followpin in the rails of standard cells, misaligned well tap cell, and missing termination cell. This check is run before placement.
	• `opt`: Checks related to optimization such as checks for available cells. This check is run before optimization.
	• `cts`: Checks related to clock tree definition such as constraint targets and route types. This is run before cts but after creating the clock tree spec. This check cannot run with pre-cts Innovus DB (just after place_opt_design). You need to set CCOpt option (target skew, target trans, etc) and also need to load the CCOpt spec file.
	• `route`: Checks related to routing such as congestion, placement, and pin access issues. The `-type` route cannot be run with pre-placed DB. This check is run before routing. It is useful to run check_drc ahead of time so the design markers are up to date.
	• `signoff`: Checks related to the signoff activities including extraction and ECOs. This check is run before TSO. To use this types, you need to set the Tempus path, else the tool will give an error.
	• `all`: Checks related to all categories.
	Note: `-type` or `-no_check` is required (except when using `-help` argument)

`-no_check`	Do not run any checks, echo the error messages associated with the checks that would be run for the specified `-type`. Run `check_design -no_check` to see the list of checks. `-type` or `-no_check` is required (except when using `-help` argument)

`-out_file` *fileName*	Specifies the output report file. Contents dumped to this file are in the Tcl format. Sourcing this file will create a TCL dictionary of the violations found by check_design.

Examples

- The following example shows the usage of the `-out_file` option:

```
> check_design -type "place cts route" -out_file checks.tcl

*** Summary of all messages that are not suppressed in this session:
Severity ID Count Summary
WARNING CHKCTS-16 1 Preferred routing layers for route_type ...
ERROR CHKCTS-1 3 CTS maximum transition target must be se...
ERROR CHKCTS-10 1 Route type(s) %s used for trunk or top c...
WARNING CHKCTS-21 1 Clock logic cells are not specified for ...
WARNING CHKCTS-46 1 Clock logic cells are not specified for ...
WARNING NRIG-1323 65 Net %s has bottom preferred routing laye...


> source checks.tcl

> dict keys $design_checks
cts route

> dict keys [dict get $design_checks place]

CHKCTS-1 CHKCTS-10 CHKCTS-16 CHKCTS-21 CHKCTS-46

# Each check as the following keys
> dict keys [dict get [dict get $design_checks cts] CHKCTS-1]

severity count violations

# For information regarding the check, use get_message:
> get_message -id CHKCTS-1 short

CTS maximum transition target must be set to a numeric value for delay_corner %s and delay_type %s for clock_tree %s and net_type %s.

# To query the violations, use the check dictionary:
> dict get [dict get [dict get $design_checks cts] CHKCTS-1] violations

{dc_0.9v_125c late clk top}
{dc_0.9v_125c late clk trunk}
{dc_0.9v_125c late clk leaf}
```

- The following command generates a list of errors that will prevent or seriously impact the running of the `place` step in the implementation flow.

```
check_design -type place -out_file place.checkDesign
```

```
############################################################
#  Generated by:     Cadence Innovus 17.12-e034_1
#  OS:               Linux x86_64(Host ID ip-172-19-134-167)
#  Generated on:     Mon Nov  6 05:53:12 2017
#  Design:           fuuBar
#  Command:          check_design -type place -out_file place.checkDesign
############################################################
#Design Check : place (12432915)
# ------------------------------------------------------------------------------
# Message: The via for net %s in DPT layer %s does not align with the tracks at %.4f %.4f.
#   Help:
# ------------------------------------------------------------------------------
dict set design_checks place IMPPP-4418 {
 count 12432915
 violations {
  {vddcx_2 M3 464.7360 2139.8625}
  {vddcx_2 M3 464.3520 2139.8625}
  {vddcx_2 M3 463.8720 2139.8625}
  {vddcx_2 M3 464.7360 2139.8825}
  ...
```

```
 }
}
#Design Check : place (7280)
# ------------------------------------------------------------------------------------------
# Message: The color of PG wire which is a non-default width %.4f wire for net %s in layer %s should have the opposite color with the track at %.4f %.4f.
#    Help:
# ------------------------------------------------------------------------------------------
dict set design_checks place IMPPP-4420 {
  count 7280
  violations {
    {0.0800 vssx_0 M0 202.3680 1095.6000}
    {0.0800 vddcx_2 M0 202.3680 1095.3000}
     ...
  }
}
```

Related Information

- check_timing
- setMessageLimit

checkDesign

```
checkDesign
[-help]
{ -all
   [-danglingNet [highlight]] | [[-io]
   [-netlist [-danglingNet [highlight]]]
   [-physicalLibrary]
   [-timingLibrary]
   [-powerGround]
   [-tieHiLo]
   [-floorplan]
   [-place]
   [-spef][-assigns]] }
[[-noText] [-outdir directoryName] [-browser]]
| [-noHtml [-outfile fileName]]
```

Checks for missing or inconsistent library and design data at any stage of the design and writes the results to a text and HTML report. This eliminates the need to perform these checks manually. By default, the HTML report is saved as *design_name*`.main.htm` whereas the text version of the report is saved as *design_name*`.main.htm.ascii`.

The Innovus Implementation System (Innovus) software checks the following data:

- I/Os
- Netlist
- Physical library
- Timing library
- Power and ground pins
- Tie-high and tie-low pins
- Floorplan
- Placement

Cadence recommends that you check libraries and data as follows:

- Perform I/O checking at any time. I/O problems might not impede any tool, but they might add to design problems.
- Perform netlist checking at any time after the design has been loaded.
- Perform physical library checking before floorplanning.
- Perform timing library checking before any timing-related operation (for example, timing driven placement or routing, timing optimization, clock-tree synthesis, and static timing analysis)
- Perform power/ground checking before routing and extraction, and verifying geometry and connectivity.
- Perform tie-high and tie-low checking before routing and extraction.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `checkDesign` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man checkDesign`.
`-all`	Performs all checks.
`-assigns`	Reports all assign statements in the netlist, including assigns to 1'b0/1'b1.
`-browser`	Opens a browser and displays the HTML version of the report. **Note:** This option cannot be used with `-noHtml`.
`-danglingNet [highlight]`	Checks the design for nets that have no instance or top-level terminals, and outputs their names into the report file. The `-danglingNet` parameter can be used only if the `-netlist` or `-all` parameter is specified. If you specify `highlight`, the software also highlights the "dangling" nets in the main window, after the `checkDesign` command finishes.

`-floorplan`	Checks the floorplan, generates error or warning messages, and reports the following information: • Off-grid horizontal and vertical tracks • Instances not snapped to row site • Unplaced I/O pins • Off Grid Power/Ground Pre-routes • Instances not on the manufacturing grid • Preroutes not on the manufacturing grid (error) • Regular preroutes not on tracks
`-io`	Checks I/O pads, generates warning messages, and reports the following information: • Unplaced I/O cells • Floating pins that belongs to I/O pads (warning) • I/O pins connected to non-I/O instances (directly connected to core cells) • Unplaced I/O pins • Floating I/O pins
`-netlist`	Checks the netlist, generates error or warning messages, and reports the following information: • Output pins tied to power/ground nets (for example, `BUFX1 U1 (.A(net1),.Y1(1'b0))`) • Input pins floating (warning) • Multiple driver nets (warning)
`-noHtml`	Generates only a text version of the report. *Default:* The report is generated in both HTML and text format.
`-noText`	Generates only an HTML version of the report. *Default:* The report is generated in both HTML and text format.
`-outdir` *directoryName*	Specifies the directory in which the HTML and text reports are saved. If `-noText` is specified, saves only the HTML version of the report to the specified directory. **Note:** The `-outdir` parameter cannot be used with `-noHtml`.
`-outfile` *fileName*	Writes the text version of the report to the specified file. **Note:** You can specify this option only if you specify `-noHtml`. *Default:* If you specify `-noHtml` but do not specify `-outfile`, the software writes the text version of the report to a file named *design_name*`.main.htm.ascii`.
`-physicalLibrary`	Checks the physical libraries and reports whether all cells have LEF views. The software generates error messages for the following conditions: • Cells not defined in LEF • Cells with missing dimensions • Pins with missing direction • Cells pins with missing geometry • Cell dimensions are not an integer multiple of the core site dimensions • A macro does not have fully covered obstruction • Fixed mask block have some pins not colored
`-place`	Checks the placement, generates error or warning messages, and reports the following information: • Number of instances with placement violations • Instances placed outside of the core area • Instances overlapping other instances • Instances overlapping placement blockages

`-powerGround`	Checks power and ground connections, generates warning messages, and reports the following information: • Power terminals connected to ground net • Ground terminals connected to power net • Floating power and ground terminals • Power and ground terminals connected to non-power and ground nets.
`-spef`	Checks spef.
`-tieHiLo`	Reports unconnected tie-high or tie-low terminals and tieHi/Lo pins and P/G pins connected to different nets. The system generates a warning message.
`-timingLibrary`	Checks whether the cells used in the design have been defined in the timing library. This option does not check for the presence of timing arcs. All physical cells are excluded from this check.

Examples

- The following command checks the design and generates a text version of the report called `checkDesign.ascii`:

  ```
  checkDesign -all -noHtml -outfile checkDesign.ascii
  ```

  The following is a sample of the resulting text report:

```
############################################################
#
#  Generated by:      Cadence Encounter 14.20-b038_1
#  OS:                Linux x86_64(Host ID rlno-leenap)
#  Generated on:      Thu Sep 25 16:03:18 2014
#  Design:            carve
#  Command:           checkDesign -all -noHtml -outfile
checkDesign.ascii
############################################################
#


========================
Design Stats
========================
Design Name: carve
    ----------------------------
    Cells used in the design
    ----------------------------
    LDL_AO22_3
    LDN_BUF_2
    LDN_BUF_HD_3
    LDL_AO2B2_S_1
    Number of cells used in the design  4

========================
Physical Library(LEF) Integrity Check
========================
    ----------------------------
    Cell with PG PIN missing
    ----------------------------
    E70G30096CU_ADD22_COVERMACRO
        ----------------------------
        Cell dimensions not interger multiple of its site
        ----------------------------
        Cell  Cell Dimension  Site Dimension
        E70G30096CU_ADD22_COVERMACRO  9540960 9540960  100000
100000
        LDL_FILL32RES
        LDL_FILL512RES
        LDL_FILL16RES
                LDL_FILL2RES
                LDL_FILL64RES
                LDL_FILL1024RES
                LDL_FILL1RES
                LDL_FILL128RES
                LDL_FILL256RES
                LDL_FILL4RES
                LDL_FILL8RES
                ATCPLL_MINI  932976 499884  30096 241164
                ATCPLL  932976 1499916  30096 241164
                WDAFUS33IOVPRGA  90288 286440  30096 241164
                WDAFUS33IOVDD33A  90288 286440  30096 241164
                TWF1STRBUFF
                TWF1STLBUFF
                TWF1SBRBUFF
                TWF1SBLBUFF
                Cells with missing LEF  0
                Cells with missing PG PIN  16
            Cells with missing dimension  0
            Cells dimensions not multiple integer of site  5
        Cells pin with missing direction: 0
        Cells pin with missing geometry: 0
        Cells PG Pins with missing geometry: 0

        ========================
        Timing information check
        ========================
            ----------------------------
            Cells with missing timing data
```

```
    ----- Nets missing Timing -----
    ------------------------------
    LDL_A022_3
    LDN_BUF_2
    LDN_BUF_HD_3
    LDL_A02B2_S_1
    Cells with missing Timing data  4

=========================
SPEF Coverage
=========================
Annotation to Verilog Netlist: 0%
Annotation to Physical Netlist: 0%

=========================
Top Level Netlist Check
=========================
Floating Ports: 0
Ports Connect to multiple Pads: 0
Ports connected to core instances: 0

=========================
Instance Pin Check
=========================
    Output pins connected to Power Ground net  0
    ------------------------------
    Instances with multiple input pins tied together
    ------------------------------
    Inst  Net  Term  Term...
    ram1/subram02
_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U189
ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/n7
C2  A
    ram1/subram02
_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U187
ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/n7
C2  A
    ram1/subram02
_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U184
ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/n7
C2  A
    ram1/subram02
_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U182
ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/n7
C2  A
    ram1/subram02
_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U170
ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/n7
C2  A
    Instances with input pins tied together  5
    TieHi/Lo term nets not connected to instance's PG terms  0
    ------------------------------
    Floating Instance terminals
    ------------------------------
    TieHi/TieLo  Term Type  Term Name  Inst Type  Inst Name
      Input  A1    ram1/subram02
_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U202
      Input  B1    ram1/subram02
_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U202
      Input  B2    ram1/subram02
_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U202
      Input  B    ram1/subram02
_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U189
      Input  C1    ram1/subram02
_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U189
      Input  B    ram1/subram02
_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U187
      Input  C1    ram1/subram02
_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U187
      Input  B    ram1/subram02
_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U184
      Input  C1    ram1/subram02
_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U184
      Input  C1    ram1/subram02
_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U182
      Input  B    ram1/subram02
_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U170
      Input  C1    ram1/subram02
_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U170
      Input  A    ram1/subram02
_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/TESTACT_WSRAM/ass
lgnbuf_709853
      Input  A    ram1/subram02
_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/FE_OFC1326_q02_10
_
      Input  A    ram1/subram02
_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/FE_OFC1329_q02_8_
    Floating Instance terminals  15
    Floating IO terms  0
    Tie Hi/Lo output terms floating  0
    Output term shorted to Power Ground net  0

=========================
Primitive Net DRC Check
=========================
Nets with tri-state driver: 0
Nets with parallel drivers: 0
Nets with multiple drivers: 0
    ------------------------------
    Undriven Net
    ------------------------------
    ram1/subram02
_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/tpo_a_9_
    ram1/subram02
_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/n7
```

```
    Nets with no driver (No FanIn)  2
Output Floating nets (No FanOut): 0
High Fanout nets (>50): 0

=========================
Sub Module Port Definition Check
=========================
Tie Hi/Lo instances connected to output: 0
Verilog nets with multiple drivers: 0

=========================
Dont Use Cells Used in Design
=========================
Dont use cells in design: 0

=========================
I/O Pin Check
=========================
Unplaced I/O Pins: 0
Floating I/O Pins: 0
I/O Pins connected to Non-IO Insts: 0

=========================
Unplaced IO Pads
=========================
Unplaced I/O Pads: 0

=========================
Power Ground Nets
=========================
VSS : Unrouted
VDD : Unrouted

=========================
Power/Ground Pin Connectivity
=========================
    Floating Power Ground terms  0
    Power/Ground pins connected to non Power/Ground net  0
    Power pin connected to Ground net  0
    Ground pin connected to Power net  0
        1) Number of error=0 & warning=0.

=========================
Top level Floorplan Check
=========================
Off-Grid Horizontal Tracks: 0
Off-Grid Vertical Tracks: 0
Placement grid on Mfg. grid: FALSE
User grid a multiple of Mfg. grid: FALSE
User grid a multiple of Mfg. grid: FALSE
Horizontal GCell Grid off Mfg. grid: 0
Vertical GCell Grid off Mfg. grid: 0
Core Row grid not a multiple of Mfg. grid: 0
AreaIO rows not on core-grid: 0
BlackBoxes Off Mfg. Grid: 0
Blocks Off Mfg. Grid: 0
BlackBoxes Off placement Grid: 0
Blocks off placement Grid: 0
Instances not snapped to row site: 0
Instances not on Mfg. Grid: 0
PrePlaced hard-macro pins not on routing grid: 0
Class COVER cell not snapped to manufacture grid: 0
Floating/Unconnected IO Pins: 0
Unplaced Io Pins: 0
IO Pin off Mfg. grid: 0
Overlapping IO pins: 0
Modules with off-grid Constraints: 0
Groups with off-grid Constraints: 0
Floating/Unconnected Ptn Pins: 0
Partition Pin off M. Grid: 0
Unplaced Partition Pins: 0
Partition Pin outside Partition box: 0
Overlapping Partition Pins: 0
Partition Pin Off-Track: 0
PartitionPower Domain off Grid: 0
PreRoute not on Mfg. grid: 0
Off Track Pre-Routes: 0
Off Grid Power/Ground Pre-routes: 0
```

Related Information

- Data Preparation chapter of the *Innovus User Guide*
  - Checking Designs
- Placing the Design chapter of the *Innovus User Guide*
  - Preparing for Placement

checkUnique

```
checkUnique
[-help]
[-verbose]
```

Checks the uniqueness of the netlist. Certain features, such as clock tree synthesis and IPO, require a module (a non-leaf cell) to be instantiated only once. If the netlist is unique, the result is `1`; otherwise, the result is `0`.

You can use the `checkUnique` command after importing a design.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `checkUnique` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man checkUnique`.
`-verbose`	Displays the names of modules instantiated more than once.

Example

The following command checks the uniqueness of the netlist and displays the names of any modules instantiate more than once:

```
innovus 4> checkUnique -verbose
```

The sample output is as shown below:

```
Non-leaf cell A instantiated 2 times.
0
```

createTrack

```
createTrack
[-help]
-dir direction
-layer layerNameList
-num numOfTrack
-start coordinate
-step step
```

Creates non-uniformed, customized tracks in selected design area. Typically, the routing grid is generated when the floorplan is initialized. The first track is located at an offset from the placement grid set by the OFFSET value for the layer in the LEF file. The track spacing is the PITCH value for the layer defined in LEF.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `createTrack` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man createTrack`.
`-dir direction`	Specifies track direction as a string.
`-start coordinate`	Specifies the starting coordinate to create the track. The unit for the coordinates is user unit (micron).
`-num numOfTrack`	Specifies the number of tracks to create for the grid. You cannot specify `0 numtracks`.
`-step step`	Specifies the spacing between the tracks.
`-layer layerNameList`	Specifies the routing layer used for the tracks. You can specify more than one layer.

defComp

```
defComp
[-help]
fileName
[-bump]
[-defAsGolden]
[-ignoreLayerFrom layer]
[-ignoreLayerTo layer]
[-ignoreLogicChange]
[-metalShapeDiff]
[-placeBlockage]
[-reportFile reportFile]
[-row]
[-snetPin]
[-unit units_value]
[-ignoreCellPlace lef_macro_classes_list
          [-checkCellPrefix prefix_list]]
```

Compares a DEF file with the physical design data in the Innovus database and reports any differences. You can use the `defComp` command after importing a design.

For example, after you perform an Engineering Change Order (ECO) and make changes to a DEF file, you can use the `defComp` command to compare the new file to the information stored in the Innovus database. Use the following command to compare physical design data:

```
defComp defFile -reportFile fileName
```

The default filename is `defPhyDiff.rpt`

The report file includes the following information:

- `VERSION` statement
  ```
  VERSION num
  ```
- `UNITS` statement
  ```
  UNITS num
  ```
- `ADDINST` statement
  ```
  ADDINST instName cellName x y orientation
  ```
  Note: The report provides the connectivity of added instances in the `NETS` section described below.
- `DELINST` statement
  ```
  DELINST instName cellName x y orientation
  ```
- `CHANGECELL` statement
  ```
  CHANGECELL instName newCellName oldCellName
  ```
  The master cell of the instance *instName* is changed from *oldCellName* to *newCellName*. The coordinate, orientation, and connectivity of the instance are not changed. If a master cell is changed along with any of the coordinates, orientation, or connectivity of the instance, Innovus considers this change as a deletion and addition of the instance. Rather than including a `CHANGECELL` statement, the file contains one pair of `DELINST` and `ADDINST` statements.
- `MOVEINST` statement
  ```
  MOVEINST instName [COORD oldX oldY newX newY] [ORIENT oldOrientation newOrientation]
  ```
  The statement contains the `COORD` phrase if the instance *instName* has moved, and the `ORIENT` phrase if the instance has changed orientation.
- `ADDNET` statement
  ```
  ADDNET netName
  ```
- `DELNET` statement
  ```
  DELNET netName
  ```
- `ADDPIN` statement
  ```
  ADDPIN netName instName pinName
  ```
- `DELPIN` statement
  ```
  DELPIN netName instName pinName
  ```
- `CHANGEROUTE` and `ENDCHANGEROUTE` statements
  ```
  CHANGEROUTE netName
  ```

```
ENDCHANGEROUTE
```

These statements mark the beginning and end of the CHANGEROUTE section that contains changes on the routing segment on net *netName*. The wire change statements are included between the CHANGEROUTE and ENDCHANGEROUTE statements.

- POWERROUTE and ENDPOWERROUTE statements

  ```
  POWERROUTE netName
  ENDPOWERROUTE
  ```

  These two statements mark the beginning and the end of the POWERROUTE section that contains the power routing differences between two DEF files. The wire change statements are included between the POWERROUTE and ENDPOWERROUTE statements.

- CHANGEAREA statement

  ```
  CHANGEAREA llx lly urx ury
  ```

  The bounding-box of any added or deleted wire shapes (regular, patch-wire, or special-wires including wire extensions) or vias (bbox across all layers) is merged into the CHANGEAREA box. For added/deleted instances, the bounding box of the placement overlap area (LEF SIZE) is used. That means any PIN or OBS shape that sticks outside the placement overlap area are not included in the bbox. COVER cells with no placement area use the bounding box of all the PIN and OBS shapes in the cell.

- Wire change statements
  The ADDWIRE, DELWIRE, ADDVIA, and DELVIA statements appear between the CHANGEROUTE and ENDCHANGEROUTE statements, orPOWERROUTE and ENDPOWERROUTE statements.

- ADDWIRE statement

  ```
  ADDWIRE layerName width x1 y1 x2 y2
  ```

- DELWIRE statement

  ```
  DELWIRE layerName width x1 y1 x2 y2
  ```

- ADDVIA statement

  ```
  ADDVIA [botLayerName bx1 by1 bx2 by2] topLayerName tx1 ty1 tx2 ty2
  ```

  Note: For turnvias, Innovus reports *topLayerName* only.

- DELVIA statement

  ```
  DELVIA [botLayerName bx1 by1 bx2 by2] topLayerName tx1 ty1 tx2 ty2
  ```

  Note: For turnvias, Innovus reports *topLayerName* only.

- ADDOBS statement

  ```
  ADDOBS layerName x1 y1 x2 y2
  ```

- DELOBS statement

  ```
  DELOBS layerName x1 y1 x2 y2
  ```

Parameters

`-help`	Prints a brief description that includes the type and default information for each `defComp` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man defComp`.
`-bump`	Checks only bump-related objects, such as bump and areaio instance in the COMPONENTS section, bump assignment, and special routing in the SPECIALNETS section.The information reported includes: • Added, moved, and deleted bumps • Added, changed, and deleted routing • Added, changed, and deleted net assignments **Note:** When this option is specified, `defComp` suppresses checks on default objects, such as standard cells and blocks, and only checks bump-related objects.
`-checkCellPrefix` *prefix_list*	Checks instances whose instance name begins with the specified prefix. You can use this parameter only if you also specify the `-ignoreCellPlace` parameter. If you specify more than one prefix, you must enclose the list in braces, for example, `{CORE PAD}`. Default: All LEF `MACRO` classes specified by the `-ignoreCellPlace` parameter are ignored.
`-defAsGolden`	Treats the DEF file as the golden file when comparing the DEF file to the data in the database. By default, the software compares the DEF to the data in the database and reports the differences relative to the database. For example, `ADDINST` in the report file means the instance is not in the database, but in the DEF file. When you specify `-defAsGolden`, the software reports the differences relative to the DEF file. Therefore, if the instance is not in the database, but is in the DEF file, `DELINST` is written in the report file.
fileName	Specifies the name of the DEF file you want to compare with the physical design data in the database.
`-ignoreCellPlace` *lef_macro_classes_list*	Ignores the specified cell classes, for example `{CORE PAD}`.
`-ignoreLayerFrom` *layer*	Ignores the specified metal or cut layer, and all metal and cut layers above. You can specify this parameter with the `-ignoreLayerTo` parameter to select a layer range. You must specify metal layers as `Metal1`, `Metal2`, `Metal3`, and so on, and cut layers as `V1`, `V2`, `V3`, and so on. *Default:* All layers are compared.
`-ignoreLayerTo` *layer*	Ignores the specified metal or cut layer, and all metal and cut layers below. You can specify this parameter with the `-ignoreLayerFrom` parameter to select a layer range. You must specify metal layers as `Metal1`, `Metal2`, `Metal3`, and so on, and cut layers as `V1`, `V2`, `V3`, and so on. *Default:* All layers are compared.
`-ignoreLogicChange`	Ignores logic changes when comparing a DEF file with the physical design data in the Innovus database.
`-metalShapeDiff`	Without the option, the command will report the same routing segment or via with one pair of line as difference - one with "ADDWIRE/ADDVIA" and one with "DELWIRE/DELVIA" - in the report file if it is on a different net or if its routing type has changed from special to regular or vice versa between db and def file. With the option, the command will not report such difference in report file.
`-placeBlockage`	Compares placement blockage and block halo equivalence.
`-reportFile` *fileName*	Specifies the name of the report containing the results of the comparison. *Default:* `defPhyDiff.rpt`
`-row`	Compares row and site equivalence, name, location, and orientation.
`-snetPin`	Reports any pins that have changed between the NETS and SPECIALNETS sections.
`-unit` *units_value*	Uses the specified `UNITS` value (typically, the `UNITS` value in the DEF file) to scale all distance values in the report file. *Default:* Uses the current database unit (DBU) value in the database, derived from the LEF information, to scale distance values.

Example

- The following command compares data in the `myDEF` file with that in the database, and writes the results to `myResultsFile.rpt`:
  ```
  defComp myDEF -reportFile myResultsFile.rpt
  ```
- The following command checks rows, placement blockages, and block halos in `myDEF`. This example checks all instances except class `CORE` and `PAD`; however, it checks `CORE` and `PAD` cells whose instance name prefix is `PRE`. This example also checks all wires and vias except for those on and between layers `Metal2` and `Metal6`, and writes the comparison results to `myResultsFile.def`.
  ```
  defComp -row -placeBlockage -ignoreCellPlace {CORE PAD} -checkCellPrefix PRE -ignoreLayerFrom Metal2 -ignoreLayerTo Metal6 -reportFile
  myResultsFile.def myDEF
  ```
- The following report is generated by the command `defComp -def a.def -reportFile defCompArea.out`:
  ```
  ############################################################
  #  Generated by:      Cadence Innovus 15.20-d132_1
  #  OS:                Linux x86_64(Host ID vlsj-daven)
  #  Generated on:      Fri Oct  2 14:17:41 2015
  #  Design:            top
  #  Command:           defComp -def a.def -reportFile defCompArea.out
  ############################################################
  VERSION 1
  UNITS 2000
  MOVEINST i1/i2 COORD 211200 252000   204600 252000
  ADDINST i1/new_inst AND2X1 100320 100800 FS
  DELINST i1/del_inst BUFX1  188760 252000 N

  CHANGEROUTE dOut2[1]
    DELWIRE METAL3 280 248920 51160 249480
    ADDWIRE METAL3 180 251110 51060 251670
  ENDCHANGEROUTE

  DELPIN VSS i1/spare_1 A
  ADDPIN VSS i1/spare_2 A

  POWERROUTE VSS
    DELWIRE METAL1 94080 266320 270240 268320
    ADDWIRE METAL1 94680 281970 270840 283970
  ENDPOWERROUTE
  CHANGEAREA 180 100800 270840 283970
  ```

defIn

```
defIn
[-help]
fileName
[-add_blockage]
[-allWarning]
[-components]
[-deleteBump]
[-deleteSpecialRoute]
[-ignore_drcfill]
[-ignore_pg_connectivity]
[-nets]
[-preserveShape]
[-removeHalfWireExtensionOnPin]
[-specialnets]
[-specialRouteUserClass user_class_name]
[-verilog_from_def_netlist_flow]
[-incremental_routing]
```

Loads the specified DEF file. `defIn` can be used at any step after importing a design. The typical usage is:

- Load a DEF file containing the floorplan saved by Innovus or other tool.
- Load a DEF file containing the scan chain information so that placement can do scan chain reorder.
- Load a DEF file containing the cell's placement and routing saved by Innovus or other tool.

Note: `defIn` ignores the shielding segment with self reference.

ⓘ When you load a DEF file, you must be at the same level in the hierarchy where the DEF file was saved.

The Innovus software loads the DEF file information into the database in several ways, depending on the object:

- Tracks, gcells, rows, die area
  The software deletes all of the existing objects in the database and reads in the new information.

  Note: The `defIn` command can read a rectilinear die area, but converts it to a rectangular die area with blockages at the cut off area.
- Blockages, fills, special nets
  The software compares the physical data (such as shape and layer) to the existing data in the database. If it finds the same data in the database, it ignores the data in the DEF file. If the data does not already exist, the software adds it to the database. For special nets, the software merges wire segments in the DEF file with any existing ones in the database that overlap and have the same attributes, such as layer, width, shape, and direction. The software does not delete any existing data.

  Note: As a lot of spacing rules in advanced nodes no longer just rely on wire width, `DESIGNRULEWIDTH` is not allowed for 20nm and below nodes.
- Nondefault rules, vias
  The software adds the objects to the database if they do not already exist there. If a nondefault rules definition with the same name already exists, the software ignores the one in the DEF file. If a via definition with the same name already exists, and the via is a fixed via, the software ignores the one in the DEF file. If a generated via definition with the same name already exists, the geometries are read, but the name might be changed. The software does not delete any existing objects of these types.

  The DEF reader checks the NDR rule only if its name is "DEFAULT" and checks whether all definition of the "DEFAULT" NDR match the LEF default rule. If the NDR rule name is not DEFAULT, the tool will give the following warning message:

  ```
  WARN: The definition of "s%" are different between the NDR Rule "DEFAULT" in DEF file and the implicit default rule in LEF File.
  ```

  Also, DEF reader will not create a "DEFAULT" NDR in DB, and give following warning message:

  ```
  WARN: Innovus does not allow user to specify an NDR rule named as "DEFAULT", ignore it when reading the DEF file and use LEF rule to replace it.
  ```
- Nets
  For each net in the DEF file, the software removes the existing regular routing (not special routing), then adds the routing from the DEF file for this net. If the net has a shielding segment with self reference, `defIn` will ignore the check. For example:

  ```
  SPECIALNETS 1 ;
  - VDD + SHIELD VDD ( 0 0 ) ( 0 100 ) ...
  ```

If the shielded net is not self referenced, `defIn` will check the shielding segment. For example:

```
SPECIALNETS 1 ;
- VDD + SHIELD net1 ( 0 0 ) ( 0 100 ) ...

NETS 1 ;
-nets + SHIELDNET VDD ...
```

- Pins
  For each pin in the DEF file, the software removes the existing physical information for the pin, then adds the physical information from the DEF file for this pin. If a pin does not already exist, and it is a power or ground pin, the software adds it to the database. If the pin is not a power or ground pin, the software generates an error message.
- Groups, regions
  If a group name in the DEF file matches an existing hierarchical instance name in the database, the region's boundary constraint is attached to the hierarchical instance, overriding any existing boundary constraint. The software checks whether the group members belong to the hierarchical instance, and generates a warning message if any do not belong. If a group name does not match any hierarchical instance name, the software creates the instance group with an attached region boundary constraint, if one exists. Regions that are not used by groups in the same DEF file are ignored. The software does not remove existing groups.

  Note: You can import DEF files containing wildcards (`*`) in the `GROUPS` statement. The `defIn` command does not support multiple groups per region.
- Components
  You cannot add new logical cells. You can add new physical cells, such as:
  - Cells with `CLASS PAD`
  - Cells with `CLASS COVER` and no signal pin
  - Instance with `+ SOURCE DIST`

You can use the `defIn` command after importing a design.

You can use the `defIn -verilog_from_def_netlist_flow` command only after using the `loadDefFile` command.

Note: Do not attempt to use the `defIn -verilog_from_def_netlist_flow` command in any other Tcl scripts.

> ***After loading the DEF netlist using the `defIn -verilog_from_def_netlist_flow` command, you can perform floorplanning, non-timing driven placement and routing, wire editing, and verification. You cannot use the DEF netlist flow for parasitic extraction, delay calculation, and timing-driven placement and routing effectively because the DEF names do not properly match the Verilog names used in timing constraints and timing analysis.***

Parameters

`-help`	Prints a brief description that includes the type and default information for each `defIn` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man defIn`
`-add_blockage`	Converts undefined instances (except COVER cells) into blockages.
`-allWarning`	Lists all warning messages issued when you load the DEF file. *Default:* Limits the number of warning messages to the first 1,000.
`-components`	Reads the `COMPONENTS` section of the DEF file and ignores other sections, except when the `-nets` or `-specialnets` parameters are specified.
`-deleteBump`	Deletes all bumps in the database before reading the DEF file.
`-deleteSpecialRoute`	Deletes all special routes in the database before reading the DEF file. This includes the RDL routing as well as the power and ground special routes in the design.
`-ignore_drcfill`	Ignores the special routing with DRCFILL shape attribute.
`-ignore_pg_connectivity`	Ignores the pg connectivity in the SPECIALNETS and NETS sections.
`fileName`	Specifies the name of the DEF file to load.
`-incremental_routing`	Prevents deletion of the whole routing of the signal net, and just adds the new wires and vias.
`-nets`	Reads the `NETS` section of the DEF file and ignores other sections, except when the `-components` or `-specialnets` parameters are specified.
`-preserveShape`	Prevents creation of symbolic center-line connected routing data and preserves the routing data as-is. The option will override the global environment variable `defInShapeBasedDefFile`.
`-removeHalfWireExtensionOnPin`	Truncates the half-width extension back to zero extension for all wire end points that overlap a pin. This is useful for DEF files created by `defOut -addHalfWireExtensionOnPin`, which forces all `NETS` section routing to use the default half-width wire extension. **Note:** Using this parameter can cause DRC violations.
`-specialRouteUserClass` *`user_class_name`*	
	Sets the specified user-class name on all the special routes read in from the DEF file.
`-specialnets`	Reads the `SPECIALNETS` and `VIAS` sections of the DEF file and ignores other sections, except when the `-components` or `-nets` parameters are specified.
`-verilog_from_def_netlist_flow`	Reconciles the names in the DEF and Verilog files generated by the `loadDefFile` command. This procedure is required if you want to retrieve any information that matches the original DEF file.

Example

- The following command loads the `TOPCHIP` DEF placement file:
  ```
  defIn TOPCHIP.def
  ```
- The following command loads the power grid from a DEF file:
  ```
  defIn -specialnets TOPCHIP.def
  ```

Related Information

- `defInShapeBasedDefFile`
- `defOverlapWireReportFileName`
- `loadDefFile`
- Importing and Exporting Designs chapter in the *Innovus User Guide*

  - "Updating Files During an Innovus Session"
- Placing the Design chapter of the *Innovus User Guide*
  - "ScanDEF-Based Reordering Approach"
- ECO Flows chapter in the *Innovus User Guide*
  - "Pre-Mask ECO Changes from a New DEF File"

defOut

```
defOut
[-help]
fileName
[-addHalfWireExtensionOnPin]
[-allLayers]
[-bumpAsPin]
[-cutRow]
[-cut_pg_rail]
[-earlyGlobalRoute]
[-flatten_via_pillar]
[-floorplan]
[-ioRow]
[-netlist]
[-noCoreCells]
[-noPrintWildCard]
[-noSpecialNet]
[-noStdCells]
[-noTracks]
[-no_virtual_trim]
[-outputMaskLayers listOfLayers]
[-scanChain]
[-selected]
[-skip_trimmetal_layers]
[-specialRouteUserClass user_class]
[-trimmetal_gap_fill]
[-unit unit_per_micron]
[-unplaced]
[-useEEQCellWithLibertyInfo]
[-usedVia]
[-verilog_from_def_netlist_flow]
[-withShield]
[-routing  [-routing_to_specialnet  | -wrongway_routing_as_specialroute ]]
```

Writes the specified information to a DEF file. By default, the `defOut` command writes floorplan, and all placed and unplaced standard cell information to the DEF file.

The `defOut` command does not support multiple groups per region. The `defOut` command supports rectilinear die area.

By default, the `defOut` command does not write out any `NONDEFAULTRULE` or `VIA` defined in the LEF file. To write out a LEF via, set the `defOutLefVia` global environment variable to `1`. To write out a nondefault rule, set the `defOutLefNDR` global environment variable to `1`.

You can use the `defOut` command after importing a design.

Note: You can select the DEF version for your output file by setting the `lefDefOutVersion` global environment variable to the version number. For example, to set the version number to 5.6, type the following command:

```
set lefDefOutVersion 5.6
```

Parameters

`-help`	Prints a brief description that includes the type and default information for each `defOut` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man defOut`.
`-addHalfWireExtensionOnPin`	
	Typically, signal routing in the `NETS` section uses the default half-width wire extensions everywhere except for zero extensions on pins. This parameter forces the zero-extension on pins to the default half-width extension, and makes the DEF compatible for older tools that do not support DEF zero wire extensions. **Note:** Using this parameter can cause DRC violations.
`-allLayers`	By default, only wires and vias on routing layers are written to the DEF. This option will write out wires, vias, and shapes on non-routing layers to the SPECIALNETS section. This includes the LEF MASTERSLICE layers, IMPLANT layers, and ROUTING or CUT layers with sub-types MIMCAP or PASSIVATION. These shapes might have been added with add_shape, or imported from DEF or OA directly.

`-bumpAsPin`	Writes bump cells as top-level pins in the `PINS` section. If a PORT in a block has a PASSIVATION layer, the tool treats it as an embedded bump. `defOut -bumpAsPin` outputs all the layer information for embedded bumps in the PIN section as for normal bumps. Use the `-bumpAsPin` parameter to generate a DEF file for use with Quantus gate-level extraction. A DEF file generated with this parameter cannot be read back into the Innovus software.
`-cut_pg_rail`	Cuts the pg rails.
`-cutRow`	Cuts rows against placement obstructions, block halos, or power domain fence minimum gaps. *Default:* Placement blockages are written to the DEF file.
`-earlyGlobalRoute`	Writes information about the wires generated by Early Global Router to the `NETS` section of the DEF file. This option implies the `-routing` parameter; therefore, if you specify this parameter, you do not need to specify `-routing`.
`fileName`	Specifies the DEF output file. If you add `.gz` as a suffix, the `defOut` command will automatically compress the generated file.
`-flatten_via_pillar`	Some advanced nodes that use SADP (self-aligned double patterning) use via-pillars. A via-pillar is a single via definition with two or more disjoint metal-shapes on the bottom (or top) routing layer of the via. This option will flatten the via pillar into several separate "normal" vias (e.g. a via with only a single metal shape on the top and bottom via layers). The multiple normal vias will be geometrically equivalent to the single via pillar. This can be useful for external tools that do not support a via pillar. **Note:** DEF created with this option should not be read back into Innovus. It should only be used for external tools that do not support via-pillars properly.
`-floorplan`	Writes the floorplan data to the DEF file. This information includes rows, chip size, `CLASS PAD` instances, `CLASS BLOCK` instances, and all placed standard cells.
`-ioRow`	Generates I/O rows based on I/O pad placement, and writes the I/O row information to the DEF file.
`-netlist`	Writes the netlist (that is, the routing connectivity information) to the DEF file. This option is typically used after the design has been placed. **Note:** Use `-netlist` with `-unplaced` if the design has not been placed.
`-noCoreCells`	Excludes core cell information from the DEF file to reduce the data size for import into APD.
`-noPrintWildCard`	Suppresses the use of the wildcard format to represent special net connectivity.
`-noSpecialNet`	Excludes special vias from the DEF file.
`-noStdCells`	Excludes placed and unplaced standard cell information from the DEF file. **Note:** • If you specify this parameter with the `-floorplan` or `-unplaced` parameters, it overrides their setting. • Fixed and cover instances are considered as pre-placed floorplan objects. This option does not apply to them.
`-noTracks`	Excludes output track and gcell statements to the DEF file.
`-no_virtual_trim`	Excludes the trim metal shapes on a virtual trim metal layer.
`-outputMaskLayers` *`listOfLayers`*	
	Specifies the output MASK data on the specified layers only. The valid values for this option are "metal", "cut", or a list of layers separated by a space character in a form of either a single layer or a range of layers. For example, `-outputMaskLayers {Metal1:Metal3 Via23}` You can also specify none as a value. When none is specified, `defOut` does not give any MASK data on any layer in the output.
`-routing`	Writes the routing information to the `NETS` section of the DEF file. If the `-routing` option is not specified, both power and signal vias are not written to the DEF file. **Note:** The `-routing` parameter implies the `-netlist` parameter; therefore, if you specify this parameter, you do not need to specify `-netlist`.

`-routing_to_specialnet`	Writes all regular NETS routing to the SPECIALNETS section instead. It converts all regular routing wires and vias that are normally written to the NETS section, to SPECIALNETS routing as RECT or VIA statements. This is intended to be read by analysis tools (such as RC extraction tools) that do not understand all the NETS routing properly. Some tools do not understand some newer DEF syntax, or how to interpret wrong-way routes that are wider than the default routing-rule for some advanced nodes due to WIDTH, WIDTHTABLE or SPANLENGTHTABLE rules with WRONGDIRECTION values. **Note**: Specialnet routing cannot be modified by the signal router. So, it should not be read back for any flows that will require optimization or future incremental or ECO routing. This option cannot be used with the `-earlyGlobalRoute` option.
`-scanChain`	Writes scan chain information to the DEF file. This option is mandatory to ensure that START/STOP instances of scan chain are preserved during timing/power closure when reloading the DEF in another session.
`-selected`	Writes out information for selected as follows: • If an instance or net is selected, writes only the DEF header and COMPONENT section for instances you have selected in the Innovus main window. If you specify the `-routing` parameter along with this parameter, the `defOut` command writes selected nets with their special routing and regular routing to the NETS section of the DEF file. • If a net is not selected but individual segments or vias are selected, writes only the selected vias to the DEF file. All LEF vias are excluded by default. If the `defOutLefVia` global variable is set to `1`, `defOut -selected` includes selected LEF vias when writing out selected vias. If the `-noSpecialNet` option is specified, special vias on the selected net will not be written out. • If a bump is selected, writes bump pin connection to the `SPECIALNETS` section of the DEF file. If a net is selected but bump pins are not individually selected, bump pin connection is not written. • If a blockage is selected in the main window, writes blockage information to the DEF file. • If a group is selected, all the instances in the selected groups are treated as selected and are writtten out.
`-skip_trimmetal_layers`	Specifies not to write trimmetal shapes in the FILLS section.
`-specialRouteUserClass` *user_class*	Specifies the user class for special route.
`-trimmetal_gap_fill`	Writes trimmetal gap fills to a dummy net _TRIMMETAL_FILLS_RESERVED in the SPECIALNETS section.
`-unit` *unit_per_micron*	Defines the units for the DEF file.
`-unplaced`	Writes information about unplaced standard cells to the DEF file.
`-useEEQCellWithLibertyInfo`	
	Specifies to use the EEQ cell with the timing library information as the master cell of a leaf instance.
`-usedVia`	Specifies that only used vias should be written to the DEF file. Unused vias are ignored. All LEF vias are also excluded by default. To include used LEF vias in the DEF file, set the `defOutLefVia` global variable is set to `1` before specifying `defOut -usedVia`. • If the `-routing` option is specified along with `-usedVia`, used signal vias are also written out. • If the `-noSpecialNet` option is specified along with `-usedVia`, used special vias are not written out. • If you specify both `-usedVia` and `-selected`, the `-usedVia` option is ignored and only selected vias are written to the DEF file. The following warning message is issued: `**WARN: (ENCDF-1039): Option -usedVia will be ignored because option -selected is specified. Only selected vias will be output.` • If neither `-selected` nor `-usedVia` is specified and the `defOutLefVia` variable is set to `0` (default), all DEF vias are written out. • If neither `-selected` nor `-usedVia` is specified and the `defOutLefVia` variable is set to `1`, all DEF and LEF vias are written out.
`-verilog_from_def_netlist_flow`	

	Reconciles the names in the DEF and Verilog files generated by the `loadDefFile` command. The `loadDefFile` command generates a flat Verilog file in which all of the special characters are escaped. Use this parameter to convert the escaped names back to the original names in the DEF file.
`-withShield`	By default, `defOut` does not write out the shielding routing on the power/ground net when outputting regular routing of selected nets in NETS section. When `-withShield` is specified, `defOut` writes out those shielding routing (if any) on the power/ground net which is in the SPECIALNETS section. The`-withShield` option must be used with the `-routing` and `-selected` options. **Note:** Multiple nets can be selected. If none of the selected nets have shielding routing, the `-withShield` option has no effect.
`-wrongway_routing_as_specialroute`	
	Writes wrong-way routing in the SPECIALNETS section. This option duplicates any wrong-way routing segments that require extra-width into the SPECIALNETS routing section as a RECT with +SHAPE DRCFILL. This option is intended to be used for tools (such as RC extraction tools) that do not understand how to increase the width of DEF wrong-way NETS routing properly. DEF requires wider widths for some wrong-way routes due to LEF DRC rules using the WIDTH, WIDTHTABLE, or SPANLENGTHTABLE statements along with WRONGDIRECTION values. These types of tools normally understand SPECIALNET RECT statements and can merge the SPECIALNET RECT on top of the NETS wrong-way routing segment and therefore use the correct width. The duplicate SPECIALNETS RECTs are given the +SHAPE DRCFILL keyword in order to make it easy to filter them out for tools that do not need them. When loading back the DEF with these DRCFILL, use `defIn -ignore_drcfill` to ignore these duplicate wrong-way direction segments.

Example

- The following command writes floorplanning data, and information on placed and unplaced standard cells to `TOPCHIP_SP.def`:
  ```
  defOut -floorplan -unplaced TOPCHIP_SP.def
  ```
- The following command also writes floorplanning data, and information on placed and unplaced standard cells:
  ```
  defOut test.def
  ```
- The following command writes floorplan and standard cell information to the DEF file. It does not write out unplaced standard cell information.
  ```
  defOut -floorplan
  ```
- The following command excludes all information on placed and unplaced standard cells from the DEF file:
  ```
  defOut -floorplan -noStdCells
  ```

  The resulting DEF file contains floorplan data only.
- The following command writes the shield wires that are shielding the selected nets to the DEF file:
  ```
  defOut -selected -routing -withShield test.def
  ```
- The following command writes bump cells as top-level pins in the PINS section of the `test.def` file:
  ```
  defOut -bumpAsPin test.def
  ```

  The PINS section in the test.def file is as follows:

```
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
PINS 102 ;
- iot_schm_2m16m_3v + NET iot_schm_2m16m_3v + DIRECTION OUTPUT + USE SIGNAL
+ SPECIAL + POLYGON ap ( 93300 65970 ) ( 65970 65970 )
( 65970 93300 ) ( 27330 93300 ) ( 27330 65970 ) ( 0 65970 )
( 0 27330 ) ( 27330 27330 ) ( 27330 0 ) ( 65970 0 )
( 65970 27330 ) ( 93300 27330 ) + FIXED ( 386690 2658905 ) N ;
- tck + NET tck + DIRECTION INPUT + USE SIGNAL
+ SPECIAL + POLYGON ap ( 93300 65970 ) ( 65970 65970 )
( 65970 93300 ) ( 27330 93300 ) ( 27330 65970 ) ( 0 65970 )
( 0 27330 ) ( 27330 27330 ) ( 27330 0 ) ( 65970 0 )
( 65970 27330 ) ( 93300 27330 ) + FIXED ( 53350 2658905 ) N ;
```

```
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
```

Related Information

- defHierChar
- defOutLefNDR
- defOutLefVia
- defOutPolygonDieArea
- defStreamOutCheckUncolored
- lefDefOutVersion
- Importing and Exporting Designs chapter in the *Innovus User Guide*
- Using the NanoRoute Router chapter in the *Innovus User Guide*
  - Using a Design Flow that Includes Astro or Apollo

defOutBySection

```
defOutBySection
[-help]
filename
[-addHalfWireExtensionOnPin]
[-bumpAsPin]
[-compPlacement]
[-earlyGlobalRoute]
[-fills]
[-groups [-noPrintGroupRegex]]
[-ioRow]
[-netRouting]
[-noComps]
[-noCoreCells]
[-noDieArea]
[-nonDefaultRules]
[-noNets]
[-noPrintWildCard]
[-no_virtual_trim]
[-outputMaskLayers layerNames]
[-pBlockages]
[-pins]
[-rBlockages]
[-rows]
[-scanChains]
[-selected]
[-specialNetRouting]
[-specialNets]
[-specialRouteUserClass user_class_list]
[-specialShape swire_type_list]
[-tracks]
[-unit unit_per_micron]
[-unplaced]
[-usedVia]
[-verilog_from_def_netlist_flow]
[-vias]
```

Writes information to the specified sections in a DEF file. By default, it writes DIEAREA (and associated FE_CORE_BOX* properties), and the netlist part (not the placement or routing part) of the COMPONENTS and NETS sections that must be explicitly suppressed with options below if you don't want them. Other sections or sub-sections are only added based on option settings.

By default, the `defOutBySection` command does not write out any via defined in the LEF file. If the `defOutLefVia` global environment variable is set to `1`, the `defOutBySection` command writes out the via defined in LEF file.

You can use the `defOutBySection` command after importing a design.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `defOutBySection` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man defOutBySection`
`-addHalfWireExtensionOnPin`	
	Forces the zero-extension on pins to the default half-width extension.
`-bumpAsPin`	Writes bump cells as top-level pins in the PINS section. Use this parameter to generate a DEF file for use with Quantus gate-level extraction. **Important**: A DEF file generated with this parameter cannot be read back into the Innovus software.
`-compPlacement`	Writes component placement and orientation information to the `COMPONENTS` section of the DEF file. The output is limited to instances with PLACED/FIXED/COVER status, unless `-unplaced` is also specified.
`-earlyGlobalRoute`	Writes wire information generated by the Early Global Router to the `NETS` section of the DEF file.
filename	Specifies the name of the DEF output file.

`-fills`	Writes metal fill information to the `FILLS` section of the DEF file.
`-groups`	Writes group information to the `GROUPS` section of the DEF file. When you use this parameter, Innovus also writes information about regions.
`-ioRow`	Generates I/O rows based on I/O pad placement and writes the I/O row information to the DEF file.
`-netRouting`	Writes routing data to the `NETS` section of the DEF file.
`-noComps`	Excludes component information from the DEF file.
`-noCoreCells`	Excludes core cell information from the DEF file to reduce the data size for import into APD.
`-noDieArea`	Excludes the DIEAREA and associated FE_CORE_BOX* properties (FE_CORE_BOX_LL_X, FE_CORE_BOX_UR_X, FE_CORE_BOX_LL_Y, FE_CORE_BOX_UR_Y). You can use this option if you do not need some sections, such as SCANCHAINS or NONDEFAULTRULES, to be consistent with the DIEAREA or core-box settings.
`-nonDefaultRules`	Writes non default rules (NDRs) to the NONDEFAULTRULES section of the DEF file. **Note**: The `-nonDefaultRules` parameter works only with `-noNets`. As the NETS section is always dumped by default, so the NONDEFAULTRULE section is also dumped by default. However, if `-noNets` is specified, the NONDEFAULTRULE section will be dumped only if `-nonDefaultRules` is also specified.
`-noNets`	Excludes net information from the DEF file. **Note**: If `-noNets` is specified, the `-netRouting` and `-earlyGlobalRoute` parameters are ignored.
`-noPrintGroupRegex`	Suppresses the use of regular expressions to represent instance names. You must also specify the `-groups` parameter.
`-noPrintWildCard`	Suppresses the use of the wildcard format to represent special net connectivity. If specified, the `-compPlacement` parameter is ignored. **Note**: The `-noPrintWildCard` parameter must be used along with the `-specialNets` parameter to be effective. If you specify `-noPrintWildCard` without `-specialNets`, the software ignores it. *Default:* If you do not specify this parameter, Innovus writes connectivity information in the `SPECIALNETS` section in wildcard format, if possible.
`-no_virtual_trim`	Excludes the trim metal shapes on a virtual trim metal layer.
`-outputMaskLayers` *layerNames*	Outputs MASK data on specified layers only.
`-pins`	Writes pin information to the `PINS` section of the DEF file.
`-pBlockages`	Writes placement blockage information to the `BLOCKAGES` section of the DEF file.
`-rBlockages`	Writes routing blockage information to the `BLOCKAGES` section of the DEF file.
`-rows`	Writes row information to the `ROWS` section of the DEF file.
`-scanChains`	Writes scan chain information to the `SCANCHAINS` section of the DEF file.
`-selected`	Writes out information for selected as follows: • If a net or instance is selected, writes only the DEF header and `COMPONENT` section for instances or nets you have selected in the Innovus main window. If you specify the `-specialNetRouting` parameter along with this parameter, the `defOutBySection` command writes selected nets with their special routing to the NETS section of the DEF file. • If a net is not selected but individual segments or vias are selected, writes only the selected vias to the DEF file. All LEF vias are excluded by default. If the defOutLefVia global variable is set to 1, `defOutBySection -selected` includes selected LEF vias when writing out selected vias. • If a bump is selected, writes bump pin connection to the `SPECIALNETS` section of the DEF file. If a net is selected but bump pins are not individually selected, bump pin connection is not written. • If a blockage is selected in the main window, writes blockage information to the DEF file. • If a group is selected, all the instances in the selected groups are treated as selected and are writtten out.

`-specialNets`	Writes the pin connections for special nets to the `SPECIALNETS` section of the DEF file. When specified along with `-selected`, writes out the pin connections for only selected nets. Does not write routing information.
`-specialNetRouting`	Writes routing information for special nets to the `SPECIALNETS` section of the DEF file, when specified with the `-specialNets` parameter. Use this parameter with `-selected` to write routing information of only selected nets. Use this parameter with `-specialRouteUserClass` to write routing information of only those special routes that have a `user_class` attribute from the specified list.
`-specialRouteUserClass` *`user_class_list`*	
	Writes out only the special routes with a user_class attribute from the specified list.
`-specialShape` *`swire_type_list`*	Writes special routing information with specified shape types. **Note:** Use `defOutBySection -specialShape` after you have used the `setSpecialRouteType` command to mark special wires and vias and set them to a specific shape type.
`-tracks`	Writes routing track information to the `TRACKS` and `GCELLGRID` statements of the DEF file.
`-unit` `unit_per_micron`	Defines the units for the DEF file.
`-unplaced`	Includes UNPLACED instances in the `COMPONENTS` section when combined with the `-compPlacement` option.
`-usedVia`	Specifies that only used vias should be written to the DEF file. Unused vias are ignored. All LEF vias are also excluded by default. To include used LEF vias in the DEF file, set the `defOutLefVias` global variable is set to `1` before specifying `defOutBySection -usedVia`. • If you specify both `-usedVia` and -selected, the `-usedVia` option is ignored and only selected vias are written to the DEF file. The following warning message is issued: **WARN: (ENCDF-1039): Option `-usedVia` will be ignored because option `-selected` is specified. Only selected vias will be output. • If neither `-selected` nor `-usedVia` is specified and the `defOutLefVias` variable is set to `0` (default), all DEF vias are written out. • If neither `-selected` nor `-usedVia` is specified and the `defOutLefVias` variable is set to `1`, all DEF and LEF vias are written out.
`-verilog_from_def_netlist_flow`	
	Reconciles the names in the DEF and Verilog files generated by the loadDefFile command. The loadDefFile command generates a flat Verilog file in which all of the special characters are escaped. Use this parameter to convert the escaped names back to the original names in the DEF file.
`-vias`	Writes via information to the `VIAS` section of the DEF file.

Example

- The following command writes component placement, pin, and special net information to the `test.def` file:

```
defOutBySection -compPlacement -pins -specialNets test.def
```

- In the following set of commands, `setSpecialRouteType` first traces special wires and vias that touch the named bump cells and sets them to `IOWIRE`. `defOutBySection` then writes the special routing with IOWIRE shape type to the `test1.def` file.

```
setSpecialRouteType -bumpCell {bc*} -stopTypeList {IOWIRE} -setType IOWIRE


defOutBySection -specialShape {IOWIRE} test1.def
```

- With only the VDD net selected in the main window, the following command writes the routing information without pin connections for only those special routes in VDD that have the user_class attribute names `attrA` or `attrB`.

```
defOutBySection -specialNetRouting -specialRouteUserClass {attrA attrB} -selected
```

Related Information

- `defOutLefNDR`
- `defOutLefVia`
- `lefDefOutVersion`
- `loadDefFile`
- Placing the Design chapter of the *Innovus User Guide*
  - Saving Scan Files

deleteModule

```
deleteModule
moduleName
[-help]
[-cell | -cells ]
```

Deletes a hierarchical instance and it's module and optionally, its cell.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `deleteModule` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man deleteModule`.
`-cell`	Deletes the module of the hierarchical instance. It only works when the specified hierarchical instance is the last one in the design that references this module.
`-cells`	Deletes all submodules that are hierarchically inside the module from which the given hierarchical instance *`moduleName`* is instantiated. If a submodule is multi-referenced and all sibling hierarchical instances instantiated from the submodule are hierarchically inside the *`moduleName`* hierarchical instance, the submodule will be deleted. Otherwise, the multi-referenced submodule will not be deleted.
`moduleName`	Specifies the hierarchical instance to delete.

Examples

- The following command deletes instance RESULTS_CONV_INST:
  ```
  deleteModule DTMF_INST/RESULTS_CONV_INST
  ```
- The following command deletes instance RESULTS_CONV_INST and deletes its cell:
  ```
  deleteModule DTMF_INST/RESULTS_CONV_INST -cell
  ```

deleteTrack

```
deleteTrack
[-help]
[-dir {X|Y}]
[-layer layerNameList]
```

Deletes a set of tracks in the database. Without arguments, it deletes all the tracks in the database.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `deleteTrack` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man deleteTrack`.
`-dir {X | Y}`	Specifies the direction of the track to be deleted. X indicates vertical lines; Y indicates horizontal lines. This is optional.
`-layer layerNameList`	Specifies the routing layer on which the tracks to be deleted. You can specify more than one layer. This is optional.

Examples

- The following command deletes the track between M1 and M2:
  ```
  deleteTrack -layer {M1 M2}
  ```
- The following command deletes the vertical tracks:
  ```
  deleteTrack -dir X
  ```

disconnectDanglingPort

```
disconnectDanglingPort
[-help]
[-keepInput]
[-keepOutput]
[-keepInout]
[-partitionOnly]
```

Disconnects dangling nets from ports in the whole design, from the top level to the bottom level. Use `reportDanglingPort` to report the dangling ports.

You can use the `disconnectDanglingPort` command after importing a design.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `disconnectDanglingPort` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man disconnectDanglingPort`.
`-keepInput`	Keeps the input dangling port connected.
`-keepOutput`	Keeps the output dangling port connected.
`-keepInout`	Keeps the inout dangling port connected.
`-partitionOnly`	Disconnects dangling nets in partitions only. *Default:* Disconnects all dangling nets.

Examples

- The following command disconnects dangling wires in partitions only:
  ```
  disconnectDanglingPort -partitionOnly
  ```

- The following set of commands first reports the dangling ports and then disconnects them:
  ```
  innovus > reportDanglingPort -outfile danglingport.rpt
  ```

  Sample content of the file `danglingport.rpt`:
  ```
  Port sel of module user_DFT_inst_BandA_0
  Port a of module user_DFT_inst_BandA_0
  Port b of module user_DFT_inst_BandA_0
  Port y of module user_DFT_inst_BandA_0

  innovus > disconnectDanglingPort
  Port sel of module user_DFT_inst_BandA_0 is disconnected...
  Port a of module user_DFT_inst_BandA_0 is disconnected...
  Port b of module user_DFT_inst_BandA_0 is disconnected...
  Port y of module user_DFT_inst_BandA_0 is disconnected...
  4 connected dangling ports are disconnected.
  ```

Related Information

- `deleteDanglingPort`
- `reportDanglingPort`

dumpOutVias

```
dumpOutVias
[-help]
[-autogen]
[-file filename]
[-all_via | -signal | -all_rule | -default | -ndr ]
```

Helps in debugging and testing. All the boolean options are default to false. If none is set, the vias are dumped out in LEF format.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `dumpOutVias` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man dumpOutVias`.
`-all_rule`	Dumps out vias from all routing rules. this option is of type boolean.
`-all_via`	Dumps out all vias. This option is of type boolean.
`-autogen`	Dumps out auto-generated vias. This option is of type boolean.
`-default`	Dumps out vias of default routing rule. This option is of type boolean.
`-file filename`	Specifies the output file name. This option is of type string. *Default*: `dump_via.out`
`-ndr`	Dumps out vias of nondefault routing rules. This option is of type boolean.
`-signal`	Dumps out signal vias. This option is of type boolean.

Example

The following command dumps out all signal vias to the default output file, `dump_via.out`:

```
dumpOutVias -signal
```

Here's the sample output:

```
#### Begin signal vias ####
VIA V01  # LEF
  LAYER POLY ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER M1L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER C01L ;
    RECT -0.033 -0.033 0.033 0.033 ;
END V01
VIA V01N  # LEF
  LAYER NDIFF ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER M1L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER C01L ;
    RECT -0.033 -0.033 0.033 0.033 ;
END V01N
VIA V01P  # LEF
  LAYER PDIFF ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER M1L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER C01L ;
    RECT -0.033 -0.033 0.033 0.033 ;
END V01P
VIA V1L2L  # LEF
  LAYER M1L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER M2L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER C1L2L ;
    RECT -0.033 -0.033 0.033 0.033 ;
END V1L2L
VIA V2L3L  # LEF
  LAYER M2L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER M3L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER C2L3L ;
    RECT -0.033 -0.033 0.033 0.033 ;
END V2L3L
VIA V3L4L  # LEF
  LAYER M3L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER M4L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER C3L4L ;
    RECT -0.033 -0.033 0.033 0.033 ;
.....................
.....................
.....................
END SP4V4L5LHVR1S5
VIA SP4V4L5LHVR3S5  # LEF
  LAYER M4L ; RECT -0.066 -0.033 0.264 0.033 ;
  LAYER M5L ; RECT -0.033 -0.066 0.033 0.066 ;
  LAYER C4L5L ;
    RECT -0.033 -0.033 0.033 0.033 ;
END SP4V4L5LHVR3S5
#### End signal vias ####
```

generateLef

```
generateLef
[-help]
outputLefFileName
[-clfFile clfFileName]
[-lefFile lefFileName]
[-lefFileList {technology.lef cell1.lef cell2.lef ...}]
[-noWE]
[-techFile technologyFileName]
```

Uses input data to create an optimized LEF file with generated vias for the NanoRoute® router.

Input data can include the following:

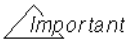
- Original or customized LEF technology files
- Milkyway technology file
- Milkyway CLF file

Note: Include sample vias for each cut layer in the LEF file. You must also include test via rules in the input in order for the output to contain the created vias.

- Scans default vias from input data to determine the following:
  - Minimum overhang
  - Width
  - Spacing
  - End-of-line overhang
- Generates auto-vias for default, power, and other generated explicit vias.
- Retains original naming convention for LEF macro reference. If there is a conflict, utilizes original names.
- Updates technology description with all vias (for example, adds multi-cut vias automatically if a `MINIMUMCUT` statement is provided).

Use the `generateLef` command before routing with the NanoRoute router.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `generateLef` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man generateLef`.
`-clfFile clfFileName`	Specifies a Milkyway CLF input file. You must also specify the `-techFile` parameter when you specify this parameter.
`-lefFile lefFileName`	Specifies a LEF input file.
`-lefFileList {technology.lef cell1.lef cell2.lef …}`	Specifies a list of LEF files to read into the software to use for generating vias with different orientations, extensions, and offsets.  The file can list only one filename per line. The first file listed must be the technology LEF file.
`-noWE`	Generates default vias for no wire extension routing.
`outputLefFileName`	Specifies the new LEF file to output.
`-techFile technologyFileName`	Specifies a Milkyway technology file. You must also specify the `-clfFile` parameter when you specify this parameter.

Examples

- The following command converts an old LEF file to an optimized LEF file for NanoRoute:
  ```
  generateLef -lefFile old.lef new.lef
  ```

- The following command converts a Milkyway technology file to an optimized LEF file for NanoRoute:
  ```
  generateLef -techFile mw.tf mw.lef
  ```
- The following command converts a CLF file to an optimized antenna LEF file for NanoRoute:
  ```
  generateLef -clfFile mw.clf ant.lef
  ```

Related Information

- Using the NanoRoute Router chapter in the *Innovus User Guide*
  - "Running the NanoRoute Router with the Innovus Text Commands"
  - "Using a Design Flow that Includes Astro or Apollo"

generateVias

```
generateVias
-help
[ -ndr_only ]
```

Auto-generates vias required by nanoroute for the default and non-default rules (NDRs). The vias are based on the wire-widths of the routing-rule and are created to meet all the DRC via enclosure rules with variations for good pin-access, variations of double-cut vias for good double-cut via utilization, and min-area vias required for stacked-vias that have no route connected to meet the min-area metal rule.

The command only creates vias in the current Innovus session; they are not saved and restored for usage in a later Innovus session. So this command is normally only used for experimentation, or possibly if you just want temporary extra DFM vias to be used for this one session. If you want to save the vias for usage by nanoroute in later Innovus sessions, use `setGenerateViaMode -auto true`. The command is not set to "on" by default as `setGenerateViaMode` is set to `false` by default.

The command requires correct DRC rules (e.g. enclosure, spacing, minimum-cut, min-area rules, etc), and at least one LEF VIARULE GENERATE statement (or OpenAccess standardViaDef) for every via layer used for routing (as required by OpenAccess). The cut enclosure values in the VIARULE GENERATE statement are not used except for old LEF technology files (LEF version 5.5 and older) that have no enclosure rules (this is not recommended usage, but is kept for backward compatability). The VIARULE cut-spacing values are ignored–only the DRC cut-spacing values are used.

No predefined vias are required (e.g LEF VIA DEFAULT, or NONDEFAULTRULE USEVIA or an OpenAccess validVias list for any routing ConstraintGroup are not needed).

Any predefined vias that do exist, are kept unless `-deleteViaBeforeGenerate` is used (see below). Generated vias are compared to predefined vias and if they are geometrically identical, that specific generated via is not kept so no duplicate vias are created.

Note: NDRs that have the same widths as the default rule will inherit the same vias, including any pre-defined vias.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `generateVias` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man generateVias`.
`-ndr_only`	Specifies vias generation for non-default rules only. When specified, it generates signal routing vias for all NDR (non-default rules) and skips generating via for default rule. It is always used for designs having full set of default rule vias, but not sufficient non-default rule vias. **Example** `generateVias -ndr_only` This generates vias for all NDRs, including the NDRs created by `add_ndr` or `createNdr` commands. You must use `setGenerateViaMode -auto true -ndr_only true` in the Innovus flow.

Example

- `generateVias`
  Will add auto-generated vias to any existing vias for all routing rules.

See also:

- `dumpOutVias` to write out the vias and look at them.
- `setGenerateViaMode` to generate vias persistently for multiple Innovus sessions.

genPinText

```
genPinText
[-format {assura dracula pvs}]
[-help]
[-layerMap fileName]
[-offset {x y}]
{[-nets {nets…}]
[-cells {cells…}]}
pin_text_file
```

Generates a file containing pin text information for nets and cells. You can read this file into Layout Versus Schematic (LVS) tools such as Dracula® and Assura™ for debugging problem areas.

You can use the `genPinText` command after you have finished placing, routing, and verifying your design in Innovus.

Note: Pin label is attached to physical pin shapes. If physical pin shapes do not exist, the pin text is labeled on any one instance pin shape. If both physical pin and instance pin shapes do not exist on the net, no pin text is generated in the output.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `genPinText` parameter. For a detailed description of the command and all of its parameters, use the man command: `man genPinText.`
`-cells {`*cells …*`}`	Specifies the list of leaf-level cells or top cell for which the pin text information is to be generated. The list of cells can be specified within braces or quotes. Additionally, pin text information can be output for all the cells using `"*"`. The `x y` values generated for the pin text are coordinates in user units local to the cell. You should not specify this parameter with the `-nets` parameter if you want to use this file with Dracula; however, you can specify both parameters if you use Assura or a third-party LVS tool.
`-format {assura dracula pvs}`	Specifies the output format.
`-layerMap` *fileName*	Specifies the name of the file that contains the mapping between LEF layers and layers defined in the LVS tool. The layer map file has the following format: `LEF_LAYER_NAME VERIF_LAYER_NAME` If you do not specify this parameter, Innovus assumes that the LEF layer names and LVS layer names are identical.
`-nets {`*nets …*`}`	Lists the name of the nets for which you want to generate pin text. You cannot use a wildcard (`*`) to specify all nets. You must use braces to enclose the list of nets. You should not specify this parameter with the `-cells` parameter if you want to use this file with Dracula; however, you can specify both parameters if you use Assura or a third-party LVS tool.
`-offset {x y}`	Specifies, in microns, the location of the design in the GDSII output file. When the GDSII output file is saved, the design shifts to the specified location.
pin_text_file	Specifies the pin text output file name. For information about the output file format see the Examples section below.

Examples

- The `genPinText` command creates a file containing one line of data for each cell or net you specify. The line has the following format:

  `name X=x Y=y ATTACH=verif_layer_name cell_name`

  The elements are

name	Specifies the net name (from the `-nets` parameter) or pin name (from the `-cell` parameter).
`X=`*x* `Y=`*y*	Specifies the x and y coordinates for the pin text.
`ATTACH=`*verif_layer_name*	Specifies the layer name used in the LVS tool.
cell_name	Specifies the cell name. The default is the top cell name.

- The following command creates a file, `pins.txt`, containing pin text information for `clk1` and `VDD`:

```
getPinText -nets {clk1 VDD} pins.txt
```

The output file would resemble the following:

```
clk1 X=0.0 Y=10.0 ATTACH=metal1 top_cell

VDD X=-10.0 Y=-10.0 ATTACH=metal2 top_cell

VDD X=20.0 Y=20.0 ATTACH=metal3 top_cell
```

- The following command creates a file, `hedtext.txt`, for cells `block1` and `block2`:
  ```
  genPinText -cells {block1 block2} hedtext.txt
  ```

  The output file would resemble the following:

  ```
  pin1b1 X=0.0 Y=0.0 ATTACH=metal1 block1
  pin2b1 X=1.0 Y=1.0 ATTACH=metal2 block1
  ```

  and so on, until all pins for `block1` are listed. Coordinates are local to `block1`.

  ```
  pin1b2 X=0.0 Y=0.0 ATTACH=metal2 block2
  pin2b2 X=1.0 Y=1.0 ATTACH=metal2 block2
  ```

  and so on, for all pins for `block2`.
- The following command creates a file, `pins.txt`, containing pin text information for `clk1`,`clk2`, and `rst` pins of the top cell `myTopCell`:
  ```
  getPinText -nets {clk1 clk2 rst} -cells {myTopCell} pins.txt
  ```

  The output file would resemble the following:

  ```
  clk1 X=204.27 Y=157.64 ATTACH=METAL2 myTopCell
  clk2 X=152.13 Y=0.28 ATTACH=METAL2 myTopCell
  rst X=0.33 Y=167.72 ATTACH=METAL2 myTopCell
  ```

get_abstract_mode

```
get_abstract_mode
[-help]
[-abstract_blockage_cut_around_pin]
[-antenna_connectivity]
[-antenna_diffusion_geom]
[-antenna_gate_geom]
[-blockage_detailed_layers]
[-boundary_layers]
[-cell_symmetry]
[-export_lef_version]
[-extract_layers_power]
[-extract_layers_signal]
[-extract_pin_layers_power]
[-extract_pin_layers_signal]
[-input_cell_type]
[-input_gds_layer_map_file]
[-input_lef_tech_file]
[-keep_temp_files]
[-pins_analog_names]
[-pins_clock_names]
[-pins_ground_names]
[-pins_output_names]
[-pins_power_names]
[-pins_text_pin_map]
[-pre_skill]
[-selected_cells]
[-site_name]
[-verbose]
```

Retrieves the current settings of all or specific `set_abstract_mode` parameters. The `set_abstract_mode` command sets options for the `run_abstract` command. For specified `set_abstract_mode` parameters, `get_abstract_mode` displays the following information in the Innovus log file and at the console:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the software displays information for all of the `set_abstract_mode` parameters.

Note: This command is under beta control. Contact your Cadence representative for more information related to this command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_abstract_mode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man get_abstract_mode`
`parameter_names`	Displays information for the specified parameters. You can specify one or more parameters. See `set_abstract_mode` for descriptions of the parameters you can specify.
`-quiet`	Displays the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software displays the current settings of all `set_abstract_mode` parameters in Tcl list format.

Example

The following command returns the cell type that will be selected for translation with abstract:

```
get_abstract_mode -input_cell_type
```

The output is as follows:

```
-input_cell_type std # enums={std io block}, default=std
```

`std`

Related Information

- `set_abstract_mode` command
- `run_abstract` command

getCmdLogFileName

```
getCmdLogFileName [-help]
```

Reports the name of the command log file.

You can use the `getCmdLogFileName` command at any time during an Innovus session.

Parameters

`-help`	Prints a brief description of the `getCmdLogFileName` command usage. For a detailed description of the command and all of its parameters, use the `man` command: `man getCmdLogFileName`.

Example

The following command gives the command log file name:

```
> getCmdLogFileName
innovus.cmd3
```

getDesignMode

```
getDesignMode
[-help]
[-addPhysicalCell]
[-backsideBottomRoutingLayer]
[-backsideTopRoutingLayer]
[-bottomRoutingLayer]
[-compressedPGDB]
[-congEffort]
[-dual_rail_via_pitch]
[-earlyclockFlow]
[-earlyPBAMode]
[-expressRoute]
[-flowEffort]
[-idealHoldFixing]
[-ignore_followpin_vias]
[-merge_trim_shapes]
[-node]
[-optimizationDensityScreenMargin]
[-pessimisticMode]
[-powerEffort]
[-process]
[-slackWeighting]
[-topRoutingLayer]
[-trim_grid_group]
[-quiet]
```

Returns the current settings of the `setDesignMode` command.

Note: The `setDesignMode` command is used to change the process technology dependent default settings globally for each application.

Parameter

`-help`	Outputs a brief description that includes type and default information for each `getDesignMode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man getDesignMode`.
`parameter_names`	Displays information for the specified parameters. See `setDesignMode` for descriptions of the `setDesignMode` parameters you can specify.
`-quiet`	Displays the current settings for the specified parameter in Tcl list format only. If you specify `-quiet` without a parameter, the software displays the current settings of all `setDesignMode` parameters in Tcl list format.

Examples

- The following command displays the current setting of `setDesignMode -process`:

  ```
  getDesignMode -process
  ```

  The software displays the following information:

  ```
  -process 90                # int, default=90, min=3, max=250
  ```

- The following command displays the current setting of `setDesignMode -process` in Tcl list format only:

  ```
  getDesignMode -process -quiet
  ```

  The software displays the following information:

  ```
  90
  ```

Related Information

- setDesignMode

getExportMode

```
getExportMode
[-help]
[-fullPinout]
[-implicitPortMapping]
[-quiet]
```

Retrieves the current settings of all or specific `setExportMode` parameters. The `setExportMode` command controls how the software writes out a Verilog netlist. For specified `setExportMode` parameters, `getExportMode` displays the following information in the Innovus log file and at the console:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, `getExportMode` displays information for all of the `setExportMode` parameters.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `getExportMode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man getExportMode`.
`parameter_names`	Displays information for the specified parameters. You can specify one or more parameters. See `setExportMode` for descriptions of the parameters you can specify.
`-quiet`	Displays the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software displays the current settings of all `setExportMode` parameters in Tcl list format.

Examples

- The following command displays the current setting of the `-fullPinout` parameter:
  ```
  getExportMode -fullPinout
  ```
  The software displays the following information:
  ```
  -fullPinout true # bool, default=false, user setting

  true
  ```
- The following command displays the current settings for all `setExportMode` parameters:
  ```
  getExportMode
  ```
  The software displays the following information:
  ```
  -fullPinout true # bool, default=false, user setting

  -implicitPortMapping false # bool, default=false

  {fullPinout true} {implicitPortMapping false}
  ```
- The following command displays the current setting of the `-fullPinout` parameter in Tcl list format only:
  ```
  getExportMode -fullPinout -quiet
  ```
  The software displays the following information:
  ```
  true
  ```

- The following command displays the current settings for all `setExportMode` parameters in Tcl list format only:

  ```
  getExportMode -quiet
  ```

  The software displays the following information:

  ```
  {fullPinout true} {implicitPortMapping false}
  ```

Related Information

- `setExportMode`

getGenerateViaMode

```
getGenerateViaMode
[-help]
[-auto]
[-ndr_only {true|false}]
[-quiet]
[-nonDefault]
```

Displays the information about the specified `setGenerateViaMode` parameter in the Innovus log file and in the Innovus console. Displays the following information:

- Option name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the software displays information for all of the `setGenerateViaMode` mode parameters.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `getGenerateViaMode` parameter. For a detailed description of the command and all of its parameters, use the man command: man `getGenerateViaMode`.
`-auto`	Enables auto via generation when design is loaded.
`-nonDefault`	Displays the current settings which are not default settings.
`-ndr_only {true|false}`	Generates vias for non default rules only. *Default:* `false`
`-quiet`	Displays the current settings in Tcl list format only.

Example

The following command gives the values of `-auto` and `-ndr_only` parameters that are set using the `setGenerateViaMode`:

```
> getGenerateViaMode
-auto true # bool, default=false, user setting
-ndr_only true # bool, default=false, user setting

{auto true} {ndr_only true}
```

Related Information

- `setGenerateViaMode`
- Design Implementation Flow chapter of *Innovus User Guide*.

getImportMode

```
getImportMode
[-help]
[-keepEmptyModule]
[-minDBUPerMicron]
[-syncReLativePath]
[-treatUndefinedCellAsBbox]
[-quiet]
```

Retrieves the current settings of all or specific `setImportMode` parameters. The `setImportMode` command controls how the software reads in the Verilog netlist and other design files. For specified `setImportMode` parameters, `getImportMode` displays the following information in the Innovus log file and at the console:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the software displays information for all of the `setImportMode` parameters.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `getImportMode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man getImportMode`.
`parameter_names`	Displays information for the specified parameters. You can specify one or more parameters. See `setImportMode` for descriptions of the parameters you can specify.
`-quiet`	Displays the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software displays the current settings of all `setImportMode` parameters in Tcl list format.

Examples

- The following command displays the current setting for the `-keepEmptyModule` parameter:
  ```
  getImportMode -keepEmptyModule
  ```
  The software displays the following information:
  ```
  -keepEmptyModule true # bool, default=true

  true
  ```
- The following command displays the current settings for all `setImportMode` parameters:
  ```
  getImportMode
  ```
  The software displays the following information:
  ```
  -minDBUPerMicron 0                # uint, default=0
  -syncRelativePath false           # bool, default=false
  -treatUndefinedCellAsBbox false   # bool, default=false

  {keepEmptyModule true} {minDBUPerMicron 0} {syncRelativePath false} {treatUndefinedCellAsBbox false}
  ```
- The following command displays the current setting for the `-keepEmptyModule` parameter in Tcl list format only:
  ```
  getImportMode -keepEmptyModule -quiet
  ```
  The software displays the following information:

```
true
```

- The following command displays the current settings for all `setImportMode` parameters in Tcl list format only:

  ```
  getImportMode -quiet
  ```

  The software displays the following information:

  ```
  {keepEmptyModule true} {minDBUPerMicron 0} {syncRelativePath false} {treatUndefinedCellAsBbox false}
  ```

Related Information

- Importing and Exporting Designs chapter in the *Innovus User Guide*
  - Handling Verilog Assigns
- Partitioning the Design chapter in *Innovus User Guide*
  - Defining Blackboxes
  - Blackbox Flow
- Using the NanoRoute Router chapter in the *Innovus User Guide*
  - Checking Your LEF Files

getLogFileName

```
getLogFileName [-help] [-fullPath]
```

Reports the name of the log file.

You can use the `getLogFileName` command at any time during an Innovus session.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `getLogFileName` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man getLogFileName`.
`-fullPath`	Reports the complete path to the log file.

Examples

- The following command reports the name of the log file for the current Innovus session:
  ```
  getLogFileName
  ```

  The software reports:

  ```
  innovus.log1003
  ```

- The following command reports the complete path to the log file for the current Innovus session:
  ```
  getLogFileName -fullPath
  ```

  The software reports:

  ```
  /icd/soclinx23/output/linux/07.10-e028_1/ph_46_08h34sec/innovus.log1003
  ```

Related Information

- `innovus`

getStreamOutMode

```
getStreamOutMode
[-help]
[-cellInstanceColor]
[-cellMasterColor]
[-cellNameUserPrefix]
[-cellNameUserSuffix]
[-convertRectToPath]
[-labelAllPinShape]
[-merge_trim_shapes]
[-mergeAppend]
[-oasisCompression]
[-oasisLayerName]
[-oasisSCellOffset]
[-pinTextOrientation]
[-reset]
[-removeNets]
[-SEcompatible]
[-SEvianames]e
[-snapToMGrid]
[-specifyViaName]
[-supportPathType4]
[-textSize]
[-uniquifyCellNamesPrefix]
[-version]
[-virtualConnection]
```

Displays the following information about the specified `setStreamOutMode` parameter in the log file and at the console. Displays the following information:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the software displays information for all of the `setStreamOutMode` parameters.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `getStreamOutMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man getStreamOutMode`.
`parameter_names`	Displays information for the specified parameters. You can specify one or more parameters. See `setStreamOutMode` for descriptions of the parameters you can specify.
`-quiet`	Displays the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software displays the current settings of all `setStreamOutMode` parameters in Tcl list format.

Examples

- The following command displays the current setting of the `-removeNets` parameter:

  ```
  getStreamOutMode -removeNets
  ```

  The software displays the following information:

  ```
  -removeNets {} # string, default=""

  {}
  ```

- The following command displays the current settings for all `setStreamOutMode` parameters:

```
getStreamOutMode
```

The software displays the following information:

```
-removeNets {} # string, default=""

-SEcompatible false # bool, default=false

-SEvianames false # bool, default=false

-snapToMGrid false # bool, default=false

-specifyViaName default # string, default=default

-supportPathType4 true # bool, default=true

-textSize 1 # float, default=1

-uniquifyCellNamesPrefix false # bool, default=false

-version 3 # int, default=3, min=1, max=32767

-virtualConnection true # bool, default=true

{removeNets {}} {SEcompatible false} {SEvianames false} {snapToMGrid false} {sp

ecifyViaName default} {supportPathType4 true} {textSize 1} {uniquifyCellNamesPr

efix false} {version 3} {virtualConnection true}
```

- The following command displays the current setting of the `-removeNets` parameter in Tcl list format only:
  ```
  getStreamOutMode -removeNets -quiet
  ```

  The software displays the following information:

  ```
  {}
  ```
- The following command displays the current settings for all `setStreamOutMode` parameters in Tcl list format only:
  ```
  getStreamOutMode -quiet
  ```

  The software displays the following information:

  ```
  {}
  ```

Related Information

- `setStreamOutMode`
- Importing and Exporting Designs chapter of *Innovus User Guide*.

init_design

```
init_design
[-help]
[-setup setup_view_list -hold hold_view_list]
```

Initializes a design using the Tcl globals. These globals are listed in the Related Globals section. The `init_design` flow replaces the configuration-file-based flow. `init_design` can be called only if design/library information is not already loaded. When calling `init_design`, you need to specify the `-setup` and `-hold` options depending on the following:

- If a view definition file (`view_definition.tcl`) is provided that properly initializes the system with the `set_analysis_view` command, `init_design` does not require `-setup` and `-hold`. However, if you do specify the active views using `-setup` and `-hold` and these conflict with the active view setting in the view definition file, the tool informs you about the conflict and then initializes the system with the active views specified in the `init_design` invocation.

  Note: If the view definition file is provided but it does not properly initialize the system with the `set_analysis_view` command (i.e the system is not in MMMC mode after sourcing the view definition file) and the `-setup` and `-hold` active views were not specified for `init_design`, the tool returns an error.
- If you are not providing a separate view definition file and are coding the MMMC objects inline in the script, the `-setup` and `-hold` options are required in the `init_design` invocation. If the `-setup` and `-hold` options are not specified, you receive the following warning:

  ```
  Active setup and hold analysis views were not provided by the -setup and -hold arguments to init_design. The system requires at least one active setup and hold analysis view to be declared before any timing commands can be run. If you need to run timing commands, you can add the -setup and -hold options to your init_design invocation. You can use the all_analysis_view command to identify the currently available views.
  ```

In Legacy mode, `eval_common_ui {read_mmmc, read_physical, read_netlist, init_design}` will give an error message, because Stylus initialization is not supported in Legacy.

Additionally, the `init_design` command:

- Initializes the default bounding box of square size with 70% utilization
- Creates default rows, the site of row depends on the height of the cell used in the netlist
- Creates track in the box based on the tech LEF

Parameters

`-help`	Prints a brief description that includes the type and default information for each `init_design` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man init_design`
`-hold` *`hold_view_list`*	Specifies the hold analysis view list. The system requires at least one active hold analysis view to be declared before the design can be initialized. You can use the `all_analysis_view` command to identify the currently available views. **Note:** Active hold analysis view should be provided in either the view definition file or by the `-hold` argument to `init_design`.
`-setup` *`setup_view_list`*	Specifies the setup analysis view list. The system requires at least one active setup analysis view to be declared before the design can be initialized. You can use the `all_analysis_view` command to identify the currently available views. **Note:** Active setup view should be provided in either the view definition file or by the `-setup` argument to `init_design`.

Examples

- The following set of commands initialize the design from a Verilog netlist using the top cell you specify (`myTopCell`):

```
set init_verilog ./myVerilog.v

set init_design_settop 1

set init_top_cell myTopCell

set init_design_netlisttype Verilog # optional, since Verilog is the detault

#set other initialization globals as required....

init_design
```

- The following set of commands load the design from a Verilog netlist. The top cell is determined automatically:

```
set init_verilog ./myVerilog.v

set init_design_settop 0

set init_design_netlisttype Verilog # optional, since Verilog is the detault

#set other initialization globals as required....

init_design
```

- The following set of commands initializes the design using the `myDesignLib/myTopCell/myViewName` cellview as the source of connectivity:

```
set init_oa_design_cell myTopCell

set init_oa_design_lib myDesignLib

set init_oa_design_view myViewName

set init_design_netlisttype OA

#set other initialization globals as required....

init_design
```

Related Information

- cds_lib
- fp_vertical_row
- init_abstract_view
- init_cpf_file
- init_design_netlisttype
- init_design_settop
- init_gnd_net
- init_import_mode
- init_io_file
- init_layout_view
- init_lef_file

- `init_mmmc_file`
- `init_mmmc_version`
- `init_oa_default_rule`
- `init_oa_design_cell`
- `init_oa_design_lib`
- `init_oa_design_view`
- `init_oa_foundry_rule`
- `init_oa_ref_lib`
- `init_oa_search_lib`
- `init_oa_special_rule`
- `init_pwr_net`
- `init_top_cell`
- `init_verilog`
- `read_power_intent`
- Importing and Exporting Designs chapter in the *Innovus User Guide*
  - The init_design Import Flow

loadDefFile

```
loadDefFile
[-help]
defFile
[-hier [-stub stubfile | -reflib {listOfRefLibs}]]
```

Reads a DEF file to build the in-memory database in Innovus. You can then continue to use other Innovus commands.

You can use the `loadDefFile` command after running the `loadLefFile` command.

When a list of OpenAccess reference libraries is specified, `loadLefFile` is not required before `loadDefFile`; otherwise, use this command after you load the library information.

> ***After loading the DEF netlist, you can perform floorplanning, non-timing driven placement and routing, wire editing, and verification. You cannot use the DEF netlist flow for parasitic extraction, delay calculation, and timing-driven placement and routing effectively because the DEF names do not properly match the Verilog names used in timing constraints and timing analysis.***

Parameters

`-help`	Prints a brief description that includes the type and default information for each parameter of the `loadDefFile` command. For a detailed description of the command and all of its parameters, use the `man` command: `man loadDefFile`.
`defFile`	Specifies the name of the DEF file to load.
`-reflib` *`listOfRefLibs`*	Specifies a list of OpenAccess reference libraries to load. When specified, the reference libraries are assumed to be correct by construction and `verilogAnnotate` is not run. This reference library is used for `def2oa`. Because the reference libraries are already provided, there is no need to use `loadLefFile` with this option.
`-hier`	Preserves the design hierarchy. When specified, creates a reference library by calling `lef2oa`. It will also create stub (`stub.v`) from pre-loaded LEF files. This stub will be used to run `verilogAnnotate` on reference libraries created by the `lef2oa` command. The reference library created this way is used by `def2oa`.
`-stub` *`stubfile`*	Specifies the name of the stub file to load. This option can be specified only with `-hier`. When specified, the stub file is used in `verilogAnnotate`. Stub is not created internally from LEF files.

Examples

- The following command loads the DEF file `myDef`:
  ```
  loadDefFile myDef
  ```
- The following command preserves the design hierarchy while loading the OpenAccess reference libraries and the DEF file `myDef.def`:
  ```
  loadDefFile -hier -reflib {reflib1 reflib2} mydef.def
  ```

Related Information

- `loadLefFile`
- Importing and Exporting Designs chapter in the *Innovus User Guide*

loadLefFile

```
loadLefFile
[file1 file2 ...]
[-help]
[-areaIo]
[-blackbox]
[-incremental]
```

Imports all the technology and library data that is specified in the LEF files into the Innovus environment. In addition, this command provides the location of the LEF files the routers need to access.

You can use the `loadLefFile` command at any time.

> The `loadLefFile` command checks your LEF file to make sure that layers are defined in the correct order (as described in the *LEF/DEF Language Reference*.) For example, routing layers must be defined alternately with cut layers:

```
LAYER METAL1
TYPE ROUTING
...
END METAL1


LAYER VIA12
TYPE CUT
...
END VIA12


LAYER METAL2
TYPE ROUTING
...
END METAL2
```

If you do not define layers in the correct order (for example, if you define all routing layers before cut layers, the Innovus software displays an error message.

Parameters

`-help`	Prints a brief description that includes the type and default information for each parameter of the `loadLefFile` command. For a detailed description of the command and all of its parameters, use the `man` command: `man loadLefFile`.
`-areaIo`	Treats the pad macro cell as an arealo cell. This option is used for versions of LEF prior to 5.5.
`-blackbox`	Treats the block macro cell as a blackbox. This option is used for versions of LEF prior to 5.5.
`file1 file2 ...`	Specify the names of LEF files to be loaded.
`-incremental`	Appends the antenna data into existing layers and macro cells. This is only used for appending LEF 5.4 antenna data.

Example

The following command imports the LEF file technology data:

```
loadLefFile sample.lef
```

Related Information

- `loadDefFile`
- Importing and Exporting Designs chapter in the *Innovus User Guide*

read_name_mapping

```
read_name_mapping
[-help]
in_file
[-debug]
```

Reads a name mapping file which records the mapping relationship between the original HDL (RTL bit level) names in Genus to the corresponding gate-level netlist names in Genus or Innovus for primary ports or reg/wire variables, and any logical phase inversions generated by Genus.

It can be used for Power Analysis using RTL switching activities and Formal Verification with Conformal LEC to facilitate key point mapping between the RTL names and the Innovus gate-level names. Innovus will not modify the incoming string <RTL_bit_level_name> and will attach this information to the corresponding netlist pin or port objects.

The input syntax of the file is as follows:

```
add mapped point RTL_bit_level_name netlist_inst_q_pin_or_port [-invert]
```

- *RTL_bit_level_name*: It is the hierarchical name of an RTL bit-level object corresponding to primary ports or reg/wire variables using "/" for the hierarchy delimiter
- *netlist_inst_q_pin_or_ports*: It is the hierarchical name of a gate-level instance q-pin or a port
- [-invert]: It indicates a logical phase inversion has been done during optimization for this pin or port

Parameters

-help	Prints a brief description that includes the type and default information for each read_name_mapping parameter. For a detailed description of the command and all of its parameters, use the man command: man read_name_mapping
-debug	Print debug info.
in_file	Name of input name mapping file.

Example

The following example show how different bits are merged into a multi-bit cell, so using simpler names is better.

```
add  mapped  point  IL_reg[1]      CDN_MBIT_reg/Q1      -invert
add  mapped  point  TRNS1_reg[0]   CDN_MBIT_reg/Q2
add  mapped  point  Vflag_reg      CDN_MBIT_reg/Q3
add  mapped  point  Zflag_reg      CDN_MBIT_reg/Q4
```

Example of the name mapping format used by Genus in presence of complex typed-names:

```
add mapped point clk clk //in
add mapped point sel sel //in
add mapped point in1[1] in1[1] //in
add mapped point in1[0] in1[0] //in
add mapped point out1[1] out1[1] //out
add mapped point out1[0] out1[0] //out
add mapped point out3/x[1] out3[x][1] //out
add mapped point out3/x[0] out3[x][0] //out
add mapped point isub/out1/x[1] isub_out1_reg[x][1]/Q
add mapped point isub/out1/x[0] isub_out1_reg[x][0]/Q
add mapped point out1[1] out1_reg[1]/Q
add mapped point out1[0] out1_reg[0]/Q
add mapped point out3/x[1] out3_reg[x][1]/Q
```

```
add mapped point out3/x[0] out3_reg[x][0]/Q
```

Related Information

- write_do_lec
- write_lec_files
- write_lec_directory_naming_style
- write_name_mapping

read_stream

```
read_stream
[-help]
-layer_map layer_map_files
{[-pvs_fill ] | [[in_files] [-cells cell_list [-offset {x y}]]]}
```

Reads in-stream files or a layer map and outputs a summary of the cell names read in, and any layer/data types that have data with no map_file entry. The data is automatically scaled during reading to the correct DB units based on the stream units.

- It reads in the metal fill stream data (with top cell name and offset) to overlay on the design. The fill data is displayed to database fill shapes (equivalent to reading DEF FILL data) to allow editing or changing the fill data.
  Reading in fill data is done once per session.
- It reads in-stream cells that have the same names as Innovus cell names to display the stream cell data (for example, overlay or toggle between stream shapes and LEF shapes for the cells).
  If a stream cell name is repeated, it replaces any previous data for that cell.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `read_stream` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_stream`.
`-cells cell_list`	Reads only the specified cells from GDS files.
in_files	Lists GDS and OASIS files.
`-layer_map layer_map_files`	Specifies the layer map file to map GDS layer to design layer
`-offset x y`	Specifies, in microns, the offset for the top-level fill cell.
`-pvs_fill`	Specifies to process the PVS FILL data in the read process.

Layer Map File Format

The `-layer_map` file maps stream layer/dtype data to Innovus layer and objects. Stream data does not have net ids, so you can only map stream data for display, or to fill, fill_opc or fill_blockage shapes.

The format for the layer map file is one line for each stream layer and dtype:

```
layer data_type layer_name object_type
```

Where

- *layer* is an stream layer number. It is an integer.
- *data_type* is a stream data_type number. It must be either an integer, or a range (e.g. 0-255).
- *layer_name* is normally must be a LEF (or OA tech) layer name. It can only be a “custom” layer name if the obj_type is “display” (e.g. this data is not on any of the LEF layers, but is useful for display purposes). Any custom layer data will be displayed like data created with add_gui_shape or add_gui_text. The *layer_name* is case-sensitive.
- *object_type* is one of these keywords:
  - `fill`: Equivalent to DEF FILL shapes
  - `fill_opc`: Equivalent to DEF FILL with + OPC shapes
  - `fill_blockage`: Equivalent to DEF BLOCKAGE + FILLS shapes
  - `display`: Is only for display purposes, and cannot be flattened.

  `fill`, `fill_opc`, and `display` can optionally be followed by
  `:mask:mask_num`
  where `mask_num` is the mask number for the shape for a multi-mask layer (equivalent of `DEF + MASK mask_num` for a shape).
  fill_blockage can optionally be followed by
  `:name:blockage_name`
  where *blockage_name* is the name given to all the blockages from this layer/dtype.

  Any text on the same layer/data_type will be mapped to the same DB layer as a text object on that layer. About the TEXT font and size, font 0 could be default (or other font decided by RD). Size could use the original size in GDS and scale it if dbu mismatch to Innovus.

The *object_type* keywords are case insensitive. The tool ignores blank lines or lines beginning with #.
A given layer/dtype combination can only appear once.

Example of Stream map-file for metal fill, via-fill, and fill blockages

```
#Stream map-file for metal fill, via-fill, and fill blockages
#only metal1 has two masks
#layer    dtype     layer_name     obj_type
1         1         metal1         fill:mask:1
1         2         metal1         fill:mask:2
51        1         cut1           fill
2         1         metal2         fill
52        1         cut2           fill
3         1         metal3         fill
…
101       1         metal1         fill_blockage:name:blkg1
101       2         metal1         fill_blockage:name:blkg2
101       3-255     metal1         fill_blockage
```

Example of Stream map-file for cell-data to display on top of LEF cell data

```
#Stream map-file for cell-data to display on top of LEF cell data
#Only routing layers and the cell-outline on custom layer “outline”
#are mapped. It does not show cut layers or layers below metal1.
#layer    dtype     layer_name     obj_type
1         0-255     metal1         display
2         0-255     metal2         display
...
#custom layer “outline” is created if it does not already exist
100       1         outline        display
```

Examples

Parse stream cells with specified layer map display stream cell data on top of LEF cell data. The report output is redirected to read_stream.rpt.

```
>read_stream {stdcells.gds memory.gds} -layer_map gds_layer.map > read_stream.rpt
```

Read in metal-fill data and overlay it on the design. The report will be in the log file.

```
>read_stream fill.gds -cell top_fill -layer_map gds_layer.map
```

Example of output report

```
#Cell names read in:
top_fill
fill_1
fill_2
…
#Layer/dtype data in the stream file with no map-file entry
1    10
4    1, 9
…
```

Related Information

- Map Files section of the Importing and Exporting Designs chapter of *Innovus User Guide*.

relink_db_files

```
relink_db_files
[-help]
-db_dir db_directory
-new_lib_dirs {new_lib_directory1 new_lib_directory2 ...}
```

Relinks symbolic links to library files inside a DB directory.

By default, `saveDesign` saves the location of library files as symbolic links under `<db_dir>/libs/...`. When a DB directory is copied to a new network where all the libraries are in a different location, or a new version of the library files are needed, you can use `relink_db_files` to change the symbolic links to new locations .

The `relink_db_files` command will search each `-new_lib_dirs` directory and the sub-directory tree below it for each library file name inside `<db_dir>`, and create a symbolic link to the new location. Only the base file name is used for the search (except for `qrcTechFile`). Therefore, two path names inside the new directory, such as `/timing/fast/std.lib` and `/timing/slow/std.lib`, will have duplicate names of "`std.lib`". The duplicate names are not supported and will cause an error. You can use the `restoreDesign` command to update all the LEF full-path names or use a new MMMC files with all the new .lib full-path names if duplicate file names need to be updated. If there are missing library file names, the tool will show an error.

The `relink_db_files` command supports `.lib`, `.lef`, and `qrcTechFile` library file types.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `relink_db_files` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man relink_db_files`
`-db_dir db_directory`	The current db directory which contains incorrect library symbolic links.
`-new_lib_dirs {new_lib_directory1 new_lib_directory2 ...}`	The new directories to search for library files.

Example

If the current LEF files are all under`/libs/lefs_v1.0/` (or sub-directories) and the current `.lib` and `qrcTechFiles` are all under`/libs/timing_v1.0/` (or sub-directories), you use the following command to update the LEF files to use `/libs/lefs_v2.0/`:

```
>relink_db_files -db_dir my_db -new_lib_dirs {/libs/lefs_v2.0 /libs/timing_v1.0}
```

Note: The command must find every library file. Therefore, you need to list the top of the old timing directory tree along with the top of the new LEF files directory tree.

Related Information

- `saveDesign`
- `restoreDesign`

replaceLefMacro

```
replaceLefMacro
[-macros {listOfMacros}]
lefFile
```

Replaces the current `MACRO` information in the design with the `MACRO` information in the specified LEF file. You can use the `replaceLefMacro` command to change the geometry information of a macro, such as boundary size, pin location, pin layer, and pin size. You cannot use the command to change the number or name of pins for a macro.

The `replaceLefMacro` command only reads and replaces the `MACRO` information from the LEF file; it does not use any other information in the file.

Note: In the following scenarios tool cannot replace the LEF file automatically.

- If you replaced the LEF macro file multiple times. In this case, the tool would not know which file to keep at the end.
- If the old LEF file that contains the original macro contains other MACRO definition. In this case, the old file cannot be removed by the tool. If the old file is not removed, the new file is appended to the list. As mentioned in the above point, the list may have multiple LEF files that contain the same MACRO but with different definition. When reading LEF files with multiple definitions, the tool keeps only the first one. As a result, in the next session the file that you want will not be loaded.

After trying different macro definitions, you need to replace the content of the macro definition manually. You can do this either by overwriting the one in original LEF file, or if the original LEF file contains only that macro, replacing the LEF file in the LEF file list.

Parameters

lefFile	Specifies the LEF file to read.
-macros {*listOfMacros*}	Replaces the information for the specified macros only. *Default:* Replaces the information for all macros in the LEF file.

report_gates

```
report_gates
[-help]
[-hinst hInstName]
[-out_file fileName]
[-sort {name count area leakage_power internal_power total_power switching_power}] [-power | -leakage_power ]
```

Reports the technology library cells that were implemented (and identifies their originating libraries), the area of the cell instances, and the break up of the instances into timing models, sequential cells, integrated clock-gating cells, inverters, buffers, and logic gate cells. Optionally power information can be reported.

The result is based on the cell_area calculated from LIB information. Use set_db use_area_from_lef true (default: false), if you want to get correlation between reports generated with `report_gates` command and `report_qor` or `report_utilization` commands. When a lef is read into Genus, the property "use_area_from_lef" is automatically set to `true`.

Timing models can refer to memory cells, IPs, and so on.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `report_gates` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_gates`
`-hinst hInstName`	Specifies the name of the hinst to collect the cell information. Only cells in the specified hinst will be considered.
`-leakage_power`	Provides leakage power statistic only.
`-out_file fileName`	Specifies the file in which report will be saved. If not specified, information is dumped in the log.
`-power`	Provides internal, switching, leakage and total power statistic.
`-sort {name count area leakage_power internal_power total_power switching_power}`	Specifies the sorting criteria to sort content of the tables to generate leakage_power/internal_power value only respected if the option -power is also specified.

Examples

```
>report_gates -sort name -out_file check.rpt
```

```
#####################################################################
#  Generated by:      Cadence Innovus 22.10-a047_1
#  OS:                Linux x86_64(Host ID sjcvl-swatip)
#  Generated on:      Mon Jun 20 09:53:59 2022
#  Design:            carve
#  Command:           report_gates -sort name -out_file check.rpt
#####################################################################
Library cell information:
 -------------------------------------------
| Cell          |  Instance  |  Area(um^2)  |
|-------------------------------------------|
| LDL_AO22_3    |          1 |        1.882 |
| LDL_AO2B2_S_1 |          5 |        9.409 |
| LDN_BUF_2     |          1 |        0.941 |
| LDN_BUF_HD_3  |          2 |        2.195 |
|-------------------------------------------|
| Total         |          9 |       14.427 |
 -------------------------------------------
Library cell type information:
 ------------------------------------------------------
| Cell Type        |   Instance   |        Area         |
|                  |              |       um^2  |   %   |
|------------------------------------------------------|
| Buffer           |            0 |       0.000 |   0.0 |
| Clock Gater      |            0 |       0.000 |   0.0 |
| Combinational    |            0 |       0.000 |   0.0 |
| Flop             |            0 |       0.000 |   0.0 |
| Inverter         |            0 |       0.000 |   0.0 |
| Latch            |            0 |       0.000 |   0.0 |
| Macro            |            0 |       0.000 |   0.0 |
| Multi-Bit Flop   |            0 |       0.000 |   0.0 |
| Physical Cell    |            9 |      14.427 | 100.0 |
|------------------------------------------------------|
| Total            |            9 |      14.427 | 100.0 |
 ------------------------------------------------------
```

report_sequential

```
report_sequential
[-help]
[-deleted]
[-full]
[-hinst hInst_names]
[-mapped]
[-optimized]
[-out_file <string>]
```

Reports information about sequentials the design. Sequentials are the elements that have a library is_sequential tag. These can be flops, latches, and clock gates.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `report_sequential` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_sequential`
`-deleted`	Generates a report with the deleted flops and the reason.
`-full`	Generates a report with the sequential name, whether it was inferred or instantiated, the type of sequential, and the type of reset.
`-hinst hInst_names`	Hinst for the report. By default, the design is reported.
`-mapped`	Generates a report with synthesis requirements modified to map the sequential.
`-optimized`	Generates a report with sequentials optimized to constants or are unloaded, but have not been removed.
`-out_file file_name`	File to dump the report. Default is stdout

Examples

- The following command reports the deleted flops:

```
innovus 1> report_sequential -deleted
Sequential elements: deleted flops
--------------------------------------

  Reason      Instance Name

--------------------------------------

unloaded       foo/bar/q_reg[15]

unloaded       foo/bar/q_reg[31]

unloaded       foo/bar/q_reg[47]

unloaded       foo/bar/q_reg[63]

unloaded       foo/bar/q_reg[79]

constant 0     moo/ff_reg_9
```

- The following command reports the optimized flops:

```
innovus 2> report_sequential -optimized
Sequential elements: optimized flops
-----------------------------------------

  Reason      Instance Name

-----------------------------------------

constant 0     RESULTS_CONV_INST/out_p1_reg[7]

constant 0     TDSP_CORE_INST/EXECUTE_INST/alu_cmd_reg[3]

constant 0     TDSP_CORE_INST/EXECUTE_INST/read_data_reg

constant 0     TDSP_CORE_INST/EXECUTE_INST/read_port_reg
```

restoreDesign

```
restoreDesign
[-help]
[design]
[-dynamic_view analysis_view]
[-hold_views hold_views]
[-leakage_view analysis_view]
[-noBinaryConstraint]
[-noTiming]
[-readGlobals tcl_file_name]
[-setup_views setup_views]
[{{session} | {-cellview lib cell view}} [-mmmcFile fileName] [-lef_files list_of_files]]
[-noTimingGraph]
```

Restores a saved database created by `saveDesign` in a previous session. It auto-detects the database type and loads it. An OpenAccess DB format is restored with `-cellview {<lib> <cell> <view>}` while a native Innovus DB format is read using `<session> <design>` for the DB directory name.

`saveDesign` optionally includes various types of data, including timing-graph data, RC extraction data. So, the state of the DB after `restoreDesign` depends on what was saved, and you may need to repeat RC extraction, timing analysis, etc. if that data was not saved.

If a design is saved with the newer Innovus version and you try to load the design in an older Innovus version, the tool will give an error and stop. The "older Innovus version" refers to the main version, such as 19.1 and 20.1. The second number after decimal does not make any difference. For example, if a design is saved in 20.12 and loaded in 20.11, it will not give any error and will work properly.

For OpenAccess usage, simple digital-only designs generally go back and forth with no special handling. But some types of Virtuoso custom data cannot be represented directly in the Innovus DB and are stored “on the side” by `restoreDesign` and geometrically equivalent data is visible in Innovus. That data is then restored by `saveDesign` when you write back to OA using the correct OA settings.

There are different types of OA methodologies and flows supported that are controlled by `setOaxMode`. You can also look at the Mixed Signal Interoperability Guide for more details on their effects on `saveDesign`/`restoreDesign`.

Note:

- Innovus does not support loading another DB while a design is already in memory.
- You should not modify the contents of a saved DB (*.enc.dat or *.db) at any time. If the contents of the saved DB are modified, restoring such modified DB might cause problems later or lead to DB corruption or crashes.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `restoreDesign` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man restoreDesign`
`-cellview lib cell view`	Specifies the library name, cell name, and view name for reading an OpenAccess DB design.
design	Specifies the top cell of the design saved in the *session* directory. If you specify any arbitrary cell name as the top cell name, the tool will automatically search the top cell name in DB directory.
`-dynamic_view analysis_view`	Specifies the analysis view for the dynamic power analysis.
`-hold_views hold_views`	Specifies the hold views for loading, rather than the hold views active during `saveDesign`.
`-leakage_view analysis_view`	Specifies the analysis view for leakage power analysis.

`-lef_files list_of_files`	Specifies the LEF files name for loading. Use this option if you want to replace the existing LEF files in the design with new LEF files instead. • This option is intended to allow upgrading to new versions of LEF files that are compatible with the old versions. The list of all LEF files used by this design must be given. • If any pin locations or cell boundaries or OBS shapes are changed for used cells, new DRC violations may occur. • If pins are added or removed from used cells, or used cells are removed, fatal errors may occur when loading the netlist. • If the removed pins and cells are unused in the design, warnings or errors may still occur if any library-specific data was stored for those cells or pins during the design flow (for example, dont_touch data) **Example** Existing LEFs used by `saveDesign`: `tech.lef, A.lef` You can change `A.lef` to `A_v2.lef` as given below: `restoreDesign place.enc.dat top -lef_files {tech.lef A_v2.lef}`
`-noBinaryConstraint`	Specifies not to load the binary timing constraint .tcz file even it is saved to disk in the previous session.
`-noTimingGraph`	Skips reading the timing graph even if it was saved to the disk by `saveDesign`.
`-mmmcFile`	Specifies the path name to a Multi-Mode/Multi-Corner (MMMC) configuration file (for example, the same file format used by `init_mmmc_file`) to replace the MMMC file saved by `saveDesign` (and all the corresponding .lib files, .sdc files, *_power_constraints.tcl etc. referenced by the MMMC file). If a new MMMC file is specified using `-mmmcFile`, the RC data saved by the previous session may mismatch with the new MMMC file and cause issues. If `restoreDesign -mmmcFile` is specified, by default, the tool will not load the RC data. You need to re-extract RC after the design is loaded. **Note**: CCOpt configuration will not be restored with `-mmmcFile` specified. You need to re-run `create_ccopt_clock_tree_spec` after `restoreDesign` to create a new CCOpt configuration against the new MMMC configuration.
`-noTiming`	Skips reading the MMMC file, timing libraries, and timing constraints. **Note:** Most designs that use LEF rely on the .lib information to define the bus order for MACROs that have bus bit terminals. Therefore, using the -`noTiming` option can result in the connectivity to bus ports of leaf level instances being reversed. When .lib information is not available, the default is for bus ports to be treated as descending (for example, [7:0]). When using Verilog modules or OpenAccess abstract views to define the bus order, the connectivity will still be correct even when `-noTiming` is specified. Timing libraries cannot be loaded in the same session after a design is read using the `-noTiming` option. You must use `restoreDesign` without the `-noTiming` option in a new session to load the design with timing libraries and constraints. If you save a design after `restoreDesign -noTiming`, it will not save timing information unless you use `saveDesign -addtiming`.
`-readGlobals tcl_file_name`	Specifies the UDM variables to be honored when restoring the design. You need to provide a tcl file stating the pre-defined UDM variable names to be honored.
session	Specifies the directory from which to load the DB files. If the *<session>* name ends with `.tar.gz` it is treated as a gzipped tar file (see `saveDesign -tgz` option) that is extracted into the directory first. So, `restoreDesign test.enc.dat.tar.gz` will create a directory named test with all the DB files inside it.
`-setup_views setup_views`	Specifies the setup views for loading. Rather than the setup views active during `saveDesign`.

Examples

- The following command loads the saved design files from the test.enc.dat directory:
  ```
  restoreDesign test.enc.dat top
  ```
- For physical-only editing, you can do this to speedup `restoreDesign`/`saveDesign` by ignoring the timing during `restoreDesign`, and adding it back during `saveDesign`.
  ```
  restoreDesign test1.enc.dat top -noTiming
  <physical-only editing>
  saveDesign test2.enc.dat -addtiming
  ```
  Then in a new session
  ```
  restoreDesign test2.enc.dat top     #has timing from test1.enc.dat
  ```
  If the logical netlist was modified during editing of test1.enc.dat, then reading test2.enc.dat may have unpredictable timing results.
- The following command reads the "designLib top placed" OpenAccess cellview but replaces the existing MMMC settings in the design with the settings in `new_mmmc.tcl`:
  ```
  restoreDesign -cellview "designLib top placed" -mmmcFile new_mmmc.tcl
  ```
- The following command reads the `placed.enc.dat` design but replaces the saved MMMC settings in the design with the settings in `new_mmmc.tcl`:
  ```
  restoreDesign placed.enc.dat -mmmcFile new_mmmc.tcl
  ```
  Then create a new CCOpt configuration:
  ```
  create_ccopt_clock_tree_spec
  ```

Related Information

- `create_ccopt_clock_tree_spec`
- `fill_setting_save`
- `init_mmmc_file`
- `saveDesign`
- `setOaxMode`
- Importing and Exporting Designs chapter in the *Innovus User Guide.*
- *Mixed Signal Interoperability Guide* for OpenAccess usage.

run_abstract

```
run_abstract
[-help]
-gds_files file_list
[-gen_script_only file.il]
[-lib_files file_list]
[-output_lef_file file.lef]
[-run_script_only file.il]
[-verilog_stub_files file_list]
```

Runs abstract to generate LEF from GDSII. The sequence of flow for the `run_abstract` command is as follows:

1. Checks if the executable “abstract” exists in the path. If not, it generates an error.
2. Generates “appropriate” configuration file.
3. Executes abstract with the configuration file.
4. Checks that the “abstract” exits with an exit code of “0”. If not, it generates an error.

The `run_abstract` command is run before loading a design.

The options for the `run_abstract` command are set using the `set_abstract_mode` command.

Note:

1. This command does not fully supports the block LEF abstract. Run 'ABSTRACT' standalone to generate LEF from block GDS.
2. This command is under beta control. Contact your Cadence representative for more information related to this command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `run_abstract` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man run_abstract`
`-gds_files file_list`	Lists the GDSII files that need to be translated **Note**: It is important that only GDSII files of the same type are listed together. For example, std-cells, IO cells, and block cells should be listed in separate runs.
`-gen_script_only file.il`	Generates the SKILL script that can be given to the abstract tool but does not execute it.
`-lib_files file_list`	Lists liberty (.lib) files. The .lib files can also be used to define pin directions by using the direction from the matching .lib cell and pin names.
`-output_lef_file file.lef`	Outputs the LEF file that will have the LEF macro abstracts but not technology file information.
`-run_script_only file.il`	Does not re-generate the script but just executes it with the specified file.
`-verilog_stub_files file_list`	Can be used to define the pin directions by using the direction from the matching Verilog module and pin names.

Related Information

- `set_abstract_mode`
- `get_abstract_mode`

saveDesign

```
saveDesign
[-help]
[-addTiming]
[-current_top_cell_only]
[-mmmc2]
[-noFill]
[-no_pvs_fill]
[-skip_file {metric cmd}]
[-no_wait out_file]
{{fileName } | {{{-cellview {libname cellname viewname} | -view viewname} [ -saveRestoreFile file_name] [-oaUsedLibsOnly | -oaLibs ]} }}
[-timingGraph [-noConstraint ]]
[[-user_path [-keep_input_path ]] ]
[-tcon ]
[-rc ]
[-libs ]
[-lib2ldb ]
[-def ]
[-verilog ]
[-tgz ]
```

Saves the complete design database in the native Innovus format if *fileName* is specified, or as an OpenAccess database (OA DB) if `-cellview` or `-view` are specified.

The native Innovus DB format in output directory must be used when you have LEF files for technology and physical library data in your original initialization script (e.g. `init_lef_file <files>`), while the OA DB output must be used if you use OA libraries (e.g. i`nit_oa_ref_lib <libs>` and `init_oa_search_libs <libs>`) during initialization.

There are different types of OA methodologies and flows supported that are controlled by `setOaxMode`. You can also look at the Mixed Signal Interoperability Guide for more details on their effects on `saveDesign`/`restoreDesign`.

You can save a design to the same location from which you restored the design. The new data will overwrite the previous data.

You can use the `saveDesign` command at any time after `init_design`.

By default, external file references used inside the `fileName.dat` directory (e.g. to library files, or SDC files that are not modified) are stored using a portable format that allows you to copy `fileName.dat` anywhere in the same network where the full path names are still valid, and then restore it with `restoreDesign`. This portable format is not supported for OA DBs that require the usage of a cds.lib file to find physical file locations.

Note:

- The `saveDesign` command does not save the state of the `timing_enable_simultaneous_setup_hold_mode` timing global variable. When the global is set to `true`, various physical design functions in the system are disabled. When the design is restored (using the `restoreDesign` command), the physical design commands are required to be active so that the system can be properly initialized.
- The new LEF will not be saved by `saveDesign` in the following scenarios.
  - If you replaced the LEF macro file multiple times. In this case, the tool would not know which file to keep at the end.
  - If the old lef file that contains the original macro contains other MACRO definition. In this case, the old file cannot be removed by the tool. If the old file is not removed, the new file is appended to the list. As mentioned in the above point, the list may have multiple lef files that contain the same MACRO but with different definition. When reading LEF files with multiple definitions, the tool keeps only the first one. As a result, in the next session the file that you want will not be loaded.

  After trying different macro definitions, you need to replace the content of the macro definition manually. You can do this either by overwriting the one in original LEF file, or if the original LEF file contains only that macro, replacing the LEF file in the LEF file list.
- You should not modify the contents of a saved DB (*.enc.dat or *.db) at any time. If the contents of the saved DB are modified, restoring such modified DB might cause problems later or lead to DB corruption or crashes.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `saveDesign` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man saveDesign`.

`-addTiming`	If you use `restoreDesign -noTiming` to restore the DB, the timing data is not loaded. This option allows you to copy the ignored timing data from the original `restoreDesign` location, and add it back during `saveDesign` like this:
	`restoreDesign -noTiming test1.enc.dat top`
	`<physical editing commands ...>`
	`saveDesign -addTiming test2.enc`
	This usage model is intended to allow faster physical-only editing, like DRC fixing, where the netlist is not modified. If you change the netlist, unpredictable results may occur when reading the test2 DB using timing data copied from test1.

`-current_top_cell_only`	Specifies to only write current design module and its submodules in the netlist. By default, all the modules from the input netlist will be saved even if they do not belong to current design.
`-cellview {`*`libname cellname viewname`*`}`	Saves the design in the OpenAccess database format using the specified lib, cell, and view names. The design is saved in the *lib/cell/view* OA directory. **Note**: Usually, the cell name is the same as the design top cell name. If the cell name specified is different from the top cell name, it means that the top module created in OpenAccess would have the new cell name instead of the original name.
`-def`	Saves a DEF file, in addition to the normal DB files in the DB directory. This parameter is not normally used for flat or block flows, and is not used by `restoreDesign`. It can be useful in a hierarchical flow, where you want to use `assembleDesign` to read only the netlist and the DEF files without opening up each design DB and writing out the Verilog and DEF files separately (see `-verilog`).
`fileName`	Specifies the name of the output file with a `restoreDesign` command in it to load the saved DB. The `.dat` extension is appended to the name to create an output directory to save the DB using native Innovus formats. So `saveDesign` test will create a file named `test`, with a command like this in it: `restoreDesign test.dat top` It will also create a directory named `test.dat` with all the DB files in it. You can then use source test to load the saved DB in a future session.
`-keep_input_path`	Saves the path same as the path in the input file.
`-libs`	Normally external file references, such as references to the LEF, .lib, SDC files are saved using a full-path name, symbolic links to the original files inside a libs sub-directory inside the DB directory. This option forces copies of the files, rather than symbolic links, in the libs sub-directory so there are no external file references. This makes the DB directory much larger, but it is suitable for sending to a different network where the library and external data files are not available. An OA DB will still have OA library references using the normal `cds.lib` methodology, but any other external file references will be copied inside the cellview directory. **Note**: Any future sessions that use `restoreDesign` and then `saveDesign` will have references to the files inside this design directory rather than to the original external locations. So, you must keep this DB directory around as long as any other DB references it (or you must use `saveDesign -libs` in the later session to force another local copy of the external files).
`-lib2ldb`	Converts the `.lib` timing libraries referenced in the MMMC file to the `.ldb` format and writes them in a libs sub-directory inside the output design directory. References to the original `.lib` files in the MMMC file are changed to the corresponding `.ldb` files inside the libs sub-directory. This makes the design directory much larger because it has all the timing library data inside it, and is not recommended for normal flows. Normally a user should setup the `.ldb` files in an external library directory and have the MMMC files use them directly to avoid this overhead. Using `.ldb` files will make `restoreDesign` faster when it reads the library data. So, this option is useful for “throw-away” debugging experiments that require repeated sessions at the cost of a one-time conversion, and carrying the library data with the design. See the `write_ldb` command for more information about the usage of .ldb files.
`-mmmc2`	Saves the complete design database in the MMMC2 format.
`-noConstraint`	Saves the timing graph data without the SDC constraints data, and requires the `-timingGraph` option. This is useful to reduce the disk size, and speedup `saveDesign`, `restoreDesign`. But the timing-graph data cannot be loaded by any other release version, and without the SDC files, the timing constraints would be lost if loaded by a different release version. **Note**: You can still load the timing-graph with the current release version, and then use `saveDesign` without the `-noConstraint` option, to regenerate the SDC files from the timing-graph and then use `restoreDesign` with a later release.

`-noFill`	Does not save any metal or via fill shapes inside the DB. This is useful if you did ECO routing, and the fill data is going to be regenerated anyway, so it is not worth saving the current fill data any more. All the fill data that would go to DEF FILLS section (floating fill data), or the DEF SPECIALNETS section with + SHAPE FILLWIRE or FILLWIREOPC will be ignored and not saved.
`-no_pvs_fill`	Does not save the PVS fill data file. In some PVS flows, a stream (GDS) file from PVS is saved inside the Innovus DB directory (see the PVS documentation for more details). During `restoreDesign` this file is not read but can be optionally loaded or used later in the flow. By default, if the PVS fill file exists, `saveDesign` will copy it from the `read_db` location to the new DB directory. If ECO changes to the current design have made the PVS fill file invalid, use this option to stop it from being copied.
`-no_wait` *`out_file`*	Saves the design in a separate process. You can continue executing new commands simultaneously while the design is being saved. This is useful when you want to save intermediate results without waiting for the save to complete. Simultaneous commands that modify the DB will have no effect on the saved design. The memory is shared unless you modify it, so there is little or no increase in the memory usage for small changes, but if you completely changed everything in the DB, you would temporarily have up to double the total memory for both processes to run. Each process may use multiple CPUs up to the limits set by `setMultiCpuUsage -localCpu`. Therefore, if the simultaneous commands use multiple local CPUs, you may temporarily have up to double the max local CPU usage until `saveDesign` is complete. The actual number of CPUs used by `saveDesign` varies with design size and the type of data being saved. So, you would need to experiment with your specific design to find the number of CPUs used, or set the `-localCpu` value temporarily lower to limit it. The output messages or error messages which normally go to stdout/stderr (e.g. the xterm) generated by `saveDesign` will be redirected to *`out_file`*. Any .log or .logv information is ignored and lost. You can check *`out_file`* to see if `saveDesign` completed successfully. If you give the `exit` command, the session will wait for `saveDesign -no_wait` to complete before exiting, while other commands will run simultaneously. This allows flow scripts that run multiple sessions to know any save from the previous session is complete before the next session uses it, without the need to poll or check *`out_file`*. **Example:** The following command saves the design into the `my_design.dat` directory, and any messages are written to `save.1`. `saveDesign my_design -no_wait save.1`
`-oaLibs`	Saves all OA libraries specified in cds.lib under the directory libname/cellname/viewname/libs/oa/. **Note**: This parameter works only in OA designs.
`-oaUsedLibsOnly`	Saves only the libraries listed in the `init_oa_ref_lib`, and `init_oa_search_lib`, which are referenced through the on-demand processing. On specifying this parameter, a new cds.lib file is created with only the specified libraries. The libraries are saved under the directory libname/cellname/viewname/libs/oa/. **Note**: This parameter works only in OA designs.
`-rc`	Saves RC extraction data, including data for incremental extraction. By default, Innovus does not automatically save RC extraction data to save disk space and run-time. The `-rc` parameter does not save RC extraction data if the design is not extracted or if you extract the data using the preRoute extraction engine.
`-saveRestoreFile` *`file_name`*	Specifies the file name that can be sourced to read back the OpenAccess Design. If you add this option, you can save a `*.inn` file and record the `restoreDesign` command. The `*.inn` file can be sourced to restore the OA design.
`-skip_file {metric cmd}`	Removes the specified db files. • If you save a design with the `-skip_file {metric}` option, the tool does not save file db `inn.um.gz` and .metric.gz. • If you save a design with the `-skip_file {cmd}` option, the tool does not save the `inn.cmd.gz` file. • If you save a design with the `-skip_file {metric cmd}` option, all `inn.um.gz`, `.metric.gz`, and `inn.cmd.gz` files cannot be saved.

`-tcon`	By default, if the SDC timing constraints have not been modified, a link to the original external SDC file is kept to reduce the disk size. If the SDC timing constraints were modified, then the SDC files are automatically stored in the DB directory. This option will force all SDC files to be copied locally inside the DB directory, even if they did not change. **Note**: Any future sessions that use `restoreDesign` and then `saveDesign` will have references to SDC files inside this design directory rather than to the original external location. So, you must keep this DB directory around as long as any other DB references it (or you must use `saveDesign` in the later session to force another local copy of the SDC files).
`-tgz`	Saves the design as a gzipped tar file. The `saveDesign` command saves `fileName.dat` and automatically turns on `-libs` and -`sdc` to force all external file references to be copied inside the DB directory (for example, .lib, .lef, .tcz files). Then it tars the directory and gzips it into a single portable Innovus design tar ball, by adding the `.tar.gz` extension to the `fileName.dat` name. The following commands results in a file named `myDesign.enc.dat.tar.gz`: `saveDesign myDesign.enc -tgz`
`-timingGraph`	Saves the timing graph data. When this parameter is specified, the -rc parameter is also automatically enabled. This means that the RC extraction data is also saved along with the timing graph data at the postRoute stage. At the preRoute stage, the RCs are not saved but all information is present to fully time the design without the need to rebuild the timing graph. Commands, such as report_timing and optDesign, that use timing analysis results use the saved timing graph. This can save significant CPU after `restoreDesign`. A user can get the same timing seen previously without waiting for the timing analysis to be repeated. **Note**: This feature is dependent on the build version and tool. Tempus cannot read a timing-graph written by Innovus. You must write and read the timing graph using the same tool and build. You can still load the design with a different tool or build, but the timing-graph is lost and you must repeat the timing analysis. The timing graph data must be fully built (e.g. with report_timing) before calling `saveDesign -timingGraph`.

`-user_path`	By default all external file references (such as. library files) are stored as absolute paths without any symbolic links. So, the DB directory is portable anywhere on the same network. This option stores external file references exactly as you entered it For example, full paths like /x/y or relative paths like ../x/y, ./x/y), and keeps any symbolic links used inside the path names.
	Example:
	Consider that there are two library files `a.lib` and `b.lib` under:
	`/global/project1/libs/mmmc`
	And two lef files c.lef and d.lef under:
	`/global/project1/libs/lef`
	There are two symlinks to the libs:
	`/usa/db1/libs1.0/mmmc/a.lib → /global/project1/libs/mmmc/a.lib`
	`/usa/libs/mmmc/b.lib → /global/project1/libs/mmmc/b.lib`
	And two symlinks to the lefs:
	`/usa/db1/libs1.0/lef/c.lef → /global/project1/libs/lef/c.lef`
	`/usa/libs/lef/d.lef → /global/project1/libs/lef/d.lef`
	Your current work dir is `/usa/db1.1/`. You have an init script like:
	`set init_mmmc_files ./mmmc.tcl`
	`set init_lef_file {/usa/db1/libs1.0/lef/c.lef ../libs/lef/d.lef}`
	`set init_verilog design.v`
	`init_design`
	Where mmmc.tcl has:
	`create_library_set -name test -timing {/usa/db1/libs1.0/mmmc/a.lib ../libs/mmmc/b.lib }`
	Then after saveDesign, **without -user_path**, the libs and lefs will save link to the real files:
	`> saveDesign db1.2`
	The `./mmmc.tcl` file is always stored inside the DB dir, so no external reference is used.

	By default, external file references are resolved to their fullpath name without any symlinks.
	The .lib files inside the mmmc.tcl file will be referenced as: `{/global/project1/libs/mmmc/a.lib /global/project1/libs/mmmc/b.lib}`
	The .lef files will be referenced as: `{/global/project1/libs/lef/c.lef /global/project1/libs/lef/d.lef}`
	This is portable and can be loaded by `restoreDesign` anywhere on the same network.
	`/usa/db1.2/libs1.2/mmmc/a.lib → /global/project1/libs/mmmc/a.lib`
	`/usa/db1.2/libs1.2/mmmc/b.lib → /global/project1/libs/mmmc/b.lib`
	`/usa/db1.2/libs1.2/lef/c.lef → /global/project1/libs/lef/c.lef`
	`/usa/db1.2/libs1.2/lef/d.lef → /global/project1/libs/lef/d.lef`
	
	With -user_path, the libs and lefs keep the symlinks of the current db:
	`> saveDesign db1.2 -user_path`
	The .lib files will be referenced as entered by the user: `{/usa/db1/libs1.0/mmmc/a.lib ../libs/mmmc/b.lib}`
	The .lef files will referenced as entered by the user: `{/usa/db1/libs1.0/lef/c.lef ../libs/lef/d.lef}`
	So `restoreDesign` must be run in the same directory as `saveDesign` was run, or in a directory with the same ./y and /x directory structure.
	`/usa/db1.2/libs1.2/mmmc/a.lib → /usa/db1/libs1.0/mmmc/a.lib`
	`/usa/db1.2/libs1.2/mmmc/b.lib → ../../../libs/mmmc/b.lib`
	`/usa/db1.2/libs1.2/lef/c.lef → /usa/db1/libs1.0/lef/c.lef`
	`/usa/db1.2/libs1.2/lef/d.lef → ../../../libs/lef/d.lef`

`-verilog`	Saves a Verilog file inside the DB directory, in addition to the normal DB files. This parameter is not normally used for flat or block flows, and is not used by `restoreDesign`. It can be used in a hierarchical flow, where you want to use `assembleDesign` to read only the Verilog and the DEF files without opening up each design DB and writing out the Verilog and DEF files separately (see `-def`).

`-view` *viewname*	Specifies the name of the OA view. This view will be saved in the same lib/cell/ directory that you used with `restoreDesign` or with init_design during initialization. This option provides a shortcut to save a view quickly in the same lib/cell structure that you read from.
	Example:
	`restoreDesign -cellview {designLib top layout}`
	`saveDesign -cellview {designLib top place}`
	`saveDesign -view place`
	Both the above `saveDesign` commands are equivalent.

Example

- The following command writes the current design into directory `test.enc.dat`:
  ```
  saveDesign test.enc
  ```

- If you already ran timing analysis, you can save some run time during read like this:
  ```
  report_timing #first time usage, builds timing-graph
  saveDesign test.enc -timingGraph
  ```
  Then in a new session
  ```
  restoreDesign test.enc.dat top
  ```
  report_timing #reports results, without repeating timing analysis

- For physical-only editing, you can do this to speedup restoreDesign/saveDesign by ignoring the timing during `restoreDesign`, and adding it back during `saveDesign`.
  ```
  restoreDesign test1.enc.dat top -no_timing
  <physical-only editing>
  saveDesign test2.enc -addTiming
  ```

  Then in a new session
  ```
  restoreDesign test2.enc.dat top #has timing from test1.enc.dat
  ```
  If the logical netlist was modified during editing of test1.enc.dat, then reading test2.enc.dat may have unpredictable timing results.

Related Information

- `assembleDesign`
- `fill_setting_save`
- `init_design`
- `init_oa_ref_lib`
- `init_oa_search_lib`
- `restoreDesign`
- `setMultiCpuUsage`
- `write_ldb`
- Importing and Exporting Designs chapter in the *Innovus User Guide.*
- Partitioning the Design chapter in the *Innovus User Guide.*
- Mixed Signal Interoperability Guide for OpenAccess usage.

saveNetlist

```
saveNetlist
fileName
[-help]
[-abstract]
[-applyMaskShift]
[-cellInstanceColor {true | false}]
[-cellMasterColor {true | false}]
[-derivePGPortDir]
[-excludeCellDef cellName]
[-excludeCellInst cellName]
[-excludeIlm]
[-excludeTopCellPGPort cellName]
[-exportTopPGNets]
[-followLogicalModuleOrder | -flat]
[-flattenBus]
[-ignoreFixedMask]
[-ilm]
[-includeBumpCell]
[-includePhysicalCell cellName]
[-includePhysicalInst]
[-includePowerGround]
[-keepAllBackslash]
[-lineLength characters_per_line]
[-maskShiftSuffix maskShiftSuffix]
[-omitFloatingPort]
[-phys]
[-removePowerGround]
[-replaceTieConnection]
[-replaceWorWandWithWire]
[-replace_inout_with_output_for_pg_ports]
[-topCell cellName]
[-topModuleFirst]
[-useCdsThruForAssign]
[-useEEQCellMap {masterCell EEQcell1 EEQcell2}]
[-useEEQCellWithLibertyInfo]
[-writeConstantForSignalConnection]
[-writeSupply0Supply1]
[-excludeLeafCell  | -onlyLeafCell  | -onlyStdCell  | -onlyMacro  | -module moduleName]
```

Writes a netlist file of the design. By default, a logical netlist is saved. The netlist is in hierarchical Verilog format only if the original netlist loaded was hierarchical. The Innovus software saves the netlist from the top level of the hierarchy to the bottom level of the hierarchy if the original netlist is specified from top to bottom. If you run this command after using `loadDefFile`, it bundles bus pins to create busses in Verilog. Saving the netlist is done after running placement with timing driven option, clock tree synthesis, scan group re-order, or IPO.

You can use this command after importing the design.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `saveNetlist` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man saveNetlist`.

`-abstract`	Writes an empty Verilog module with port declarations for the specified module. You can use the -abstract parameter with the following parameters: • `-includePowerGround` • `-excludeTopCellPGPort` • `-derivePGPortDir` • `-exportTopPGNets` • `-module` • `-phys` *Default*: Top module
`-applyMaskShift`	Adds a suffix to the cell name of a leaf instance based on the color assignments of the instance. **Note**: • The netlist must be unique or `-flat` must specified for the option to take effect. • The mask shifted netlist cannot be read back into Innovus, as the cell names with suffix do not actually exist in LEF and timing libraries.
`-cellInstanceColor {true | false}`	Specifies to use the instance-based color.
`-cellMasterColor {true | false}]`	Specifies to use the cell master-based color from the Innovus color engine.
`-derivePGPortDir`	Specifies whether to derive (vs. assume) directions of implicit PG ports in saving a physical netlist.
`-excludeCellDef` *`cellName`*	Excludes the module definition of the specified cells.
`-excludeCellInst` *`cellName`*	Excludes the instances of the specified cells from the netlist. You can exclude logical or physical instances. The *`cellName`* can be a single cell name or a list of cell names separated by a space and surrounded by braces (`{}`) or double quotation marks (`""`). For example: `-excludeCellInst {cell1 cell2}` or `"cell1 cell2"`.
`-excludeIlm`	Excludes the ILM module.
`-excludeLeafCell`	Writes I/O instances, macro or block instances, and standard cell instances to the netlist, but does not include leaf cell definitions in the netlist.
`-excludeTopCellPGPort` *`portName`*	Excludes specified power and ground port(s) from the top cell's module port list. The *`portName`* can be a single port name or a list of port names separated by a space and surrounded by braces (`{}`) or double quotation marks (`""`). For example: `-excludeTopCellPGPort {VSS VDD}` or `"VSS VDD"`.
`-exportTopPGNets`	Specifies to use top-cell PG nets as ports for the top module in a physical netlist created with the `-phys` or `-includePowerGround` option.
`fileName`	Specifies the name of the file where the netlist is written.
`-flat`	Writes a flattened Verilog netlist. *Default:* If you omit this parameter, and if the original netlist loaded was hierarchical, the netlist is written in hierarchical Verilog format.
`-flattenBus`	Writes a flattened-bus Verilog netlist. When this parameter is used, a bus bit, such as "A[0]" is output as "\A[0]". If such conversions conflict with existing object names, `saveNetlist` warns about such conflicts and returns an error: **WARN: (ENCDB-2156): Cannot rename bus-bit %s '%s' in cell '%s' because the name '%s' is already in use on %s in that cell. Change the design to remove the conflicting name or run the command without the `-flattenBus` option. **ERROR: (ENCSYC-1279): Cannot output flatten-bus netlist due to the reported object-name conflicts.

`-followLogicalModuleOrder`	Follows the logical module order. The order of the modules, written without `-includePowerGround`, in a hierarchical logical netlist is different from that, written with the option, in a hierarchical PG netlist. If `-followLogicalModuleOrder` is specified together with `-includePowerGround` (but not `-flat`) the order of the modules in a physical netlist will be the same as that in a logical netlist. **Note**: The same order is for non-leaf modules only.
`-ignoreFixedMask`	Ignores cell fixedMask when maskShift is applied.
`-ilm`	Writes out a netlist with ILM guts. **Note:** This parameter can be specified only in the `flattenIlm` state. If this parameter is specified in the unflatten state, the software returns an error message.
`-includeBumpCell`	Writes bump cell information to the netlist. You can also use `-includeBumpCell` with logical netlist. **Note:** This parameter is effective only when used with either the `-phys` or `-flat` parameters.
`-includePhysicalCell` *cellName*	Includes the physical instances of the specified cells in the netlist. With this parameter, 1'b1/1'b0-style, logical connections are used for tie-off pins. The *cellName* can be a single cell name or a list of cell names separated by a space and surrounded by braces ({}) or double quotes (""), for example, `-includePhysicalCell {cell1 cell2}` or `"cell1 cell2"`. *Default*: If you omit this parameter, no physical cell module definitions will be in the netlist file. **Note:** `-includePhysicalCell` creates a netlist that should not be read back into Innovus.
`-includePowerGround`	By default, PG connections in the original logical Verilog netlist are already written out (unless -flat is given). You can add "physical only" global PG net connections and "physical only" PG nets added by IEEE1801 to the Verilog output with this option. It adds PG nets inside each module and PG ports on each module in order to thread the PG connections to each pin. Some physical tools like LVS prefer to have these explicitly in the hierarchical Verilog.
`-includePhysicalInst`	Activates physical netlisting by including physical instances, such as fillers, in the netlist. With this parameter, GNC-based/physical connections, such as vss, are used for tie-off pins.
`-keepAllBackslash`	Keeps all backslashes in names.
`-lineLength` *characters_per_line*	Specifies the maximum number of characters per line for module port lists and instance port connections. When a line length exceeds the specified value, the software wraps the line at the next opportunity. You must specify a non-negative integer for the value. *Default:* `0` (The software wraps lines at the earliest opportunity.)
`-maskShiftSuffix` *maskShiftSuffix*	Adds a user-specified suffix to the cell name of a leaf instance instead of the tool's default suffix. The default suffix is `_<layer>_<mask>`. For example, BUFX1_M0_1 default suffix means the M0 color flipped from mask 0 to mask 1. The `-maskShiftSuffix` option works only when `-applyMaskShift` is used. **Note**: The netlist must be unique or `-flat` must specified for the option to take effect.
`-module` *moduleName*	Saves the specified module of the netlist.
`-omitFloatingPort`	Omits the unconnected instance ports from the saved netlist.
`-onlyLeafCell`	Writes only leaf cell definitions in the netlist. *Default:* If you do not specify this parameter, the software writes leaf cell definitions, standard cell instances, and macro/block instances in the netlist file.
`-onlyMacro`	Writes only macro or block cell definitions to the netlist. *Default:* If you do not specify this parameter, the software writes leaf cell definitions, standard cell instances, and macro/block instances in the netlist file.
`-onlyStdCell`	Writes only standard cell instances in the netlist. *Default:* If you do not specify this parameter, the software writes leaf cell definitions, standard cell instances, and macro/block instances in the netlist file.

`-phys`	Writes out physical cell instances, and inserts power and ground nets in the netlist. This is used for LVS and for designs with multiple supply voltages. Hierarchical power and ground nets are also supported. **Note:** `-phys` creates a netlist that should not be read back into Innovus.
`-removePowerGround`	Removes PG ports, PG nets, PG pin connections, and replaces PG net connections on signal pins with 1'b1 or 1'b0.
`-replaceTieConnection`	Replaces `1'b1` and `1'b0` constants with undriven nets in the output netlist. Use this parameter if you may want to save a Verilog netlist from Innovus without any `1'b*` constants. **Note:** By default, the `saveNetlist` command writes a Verilog netlist file of the design using `1'b*` constants to represent Assign nets and ports. However, subsequent verification steps that you use outside Innovus might not be able to handle these `1'b*` constants. In such a case, use `-replaceTieConnection`.
`-replaceWorWandWithWire`	Replaces wor/wand with wire statement.
`-replace_inout_with_output_for_pg_ports`	Changes the direction of inout P/G ports to output except for the ones defined by IEEE1801/UPF. Their directions remain unchanged.
`-topCell` *cellName*	Saves the specified module along with its submodules. If you use `saveNetlist -topCell`, tool only saves the top cell in the netlist file, and if you use `saveNetlist` without `-topCell`, the tool saves all modules in the netlist file.
`-topModuleFirst`	Writes out top module first, followed by the rest of the modules. **Note**: This feature is not available for physical netlist writing. As a result, the `-topModuleFirst` option will be ignored if any of the following parameters is also specified: `-phys`, `-includePowerGround`, and `-includePhysicalInst`.
`-useCdsThruForAssign`	Replaces the assign statement with cds_thru, the output netlist will not be able to read back by Innovus. This is only for third party tool.
`-useEEQCellMap {`*masterCell EEQcell1 EEQcell2*`}`	Defines the master cell and the EEQ cells that need to be mapped to the master cell {{masterCell EEQcell1 EEQcell2} ...}
`-useEEQCellWithLibertyInfo`	Specifies to use the EEQ cell with the timing library information as the master cell of a leaf instance.
`-writeConstantForSignalConnection`	Writes constant value for tie pin connection. When this option is specified, tool can assign the tie net name to related signal pin. If this option is not specified, tool assigns the real net name to the signal pin.
`-writeSupply0Supply1`	`-writeSupply0Supply1` must be specified together with `-phys` or `-includePowerGround` to take effect. Its default is off, with which a PG net is written as "supply0/supply1" only if it defined as such in the original Verilog. When on, PG nets are written as "supply0" or "supply1" except for implicit (see IEEE1801) PG nets, which are written as "wire" as needed.

Example

- The following command writes out the netlist in Verilog format of the current state of the design in Innovus to a file `mydesign_cts.v`:

  ```
  saveNetlist mydesign_cts.v
  ```

- The following command saves the netlist with `dtmf_recvr_core` as the top module to the `output.v` file:

  ```
  saveNetlist -topCell dtmf_recvr_core output.v
  ```

- The following command saves the physical netlist of ILM internals to the `output.v` file:

  ```
  saveNetlist -ilm -phys output.v
  ```

  Use this combination of options to pass on power or ground (P/G) information correctly to the netlist in low power ILM flow.

- The following example shows the netlist generated by `saveNetlist -abstract`.

  ```
  module test(in1, in2, out1, out2, clk1, clk2, clk3, se1, se2);
   input [3:0] in1;
   input [7:0] in2;
   input clk1, clk2, clk3, se1, se2;
   output [3:0] out1;
   output [7:0] out2;
  ```

```
endmodule
```

Related Information

- `flattenIlm`
- `loadDefFile`
- Importing and Exporting Designs chapter in the *Innovus User Guide.*
- Top-level Timing Closure Methodologies chapter in the *Innovus User Guide.*
  - Interactive Use of ILMs
- ECO Flows chapter in the *Innovus User Guide*
  - Pre-Mask ECO Changes from a New DEF File
  - Pre-Mask ECO Changes from an ECO File

set_abstract_mode

```
set_abstract_mode
[-help]
[-reset]
[-abstract_blockage_cut_around_pin layer_list]
[-antenna_connectivity {layer1 connect_layer layer2}]
[-antenna_diffusion_geom {{layer_name} {logical_expression}}]
[-antenna_gate_geom {{layer_name} {logical_expression}}]
[-blockage_detailed_layers layer_list]
[-boundary_layers layer_list]
[-cell_symmetry {R0 | X | Y | R90 | X Y | X R90 | Y R90 | X Y R90}]
[-export_lef_version {5.3 | 5.4 | 5.5 | 5.6 | 5.7 | 5.8}]
[-extract_layers_power layer_list]
[-extract_layers_signal layer_list]
[-extract_pin_layers_power layer_list]
[-extract_pin_layers_signal layer_list]
[-input_cell_type {std | io | block}]
[-input_gds_layer_map_file file.map]
[-input_lef_tech_file file.lef]
[-keep_temp_files {true|false}]
[-pins_analog_names pattern]
[-pins_clock_names pattern]
[-pins_ground_names pattern]
[-pins_output_names pattern]
[-pins_power_names pattern]
[-pins_text_pin_map pattern]
[-pre_skill file.il]
[-selected_cells pattern]
[-site_name site_names]
[-verbose {true | false}]
```

Sets options for the `run_abstract` command.

Use the `get_abstract_mode` command to return the current settings for the `set_abstract_mode` command.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `set_abstract_mode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_abstract_mode`
`-reset`	Resets all or specified options to default value. Any values that do not already match the default value and are reset will be reported.
`-abstract_blockage_cut_around_pin` *`layer_list`*	
	Specifies that obstruction should be cut around pin for standard cells. By default, `run_abstract` creates obstructions on pin shapes. • If `-input_cell_type` is `std`, the default value for `-abstract_blockage_cut_around_pin` is the list of all metal and via layer names. This means that abstract will not cover obstruction on pins for standard cells on all metal and via layers if `-abstract_blockage_cut_around_pin` is specified. • If `-input_cell_type` is `block` or `io`, the default value for `-abstract_blockage_cut_around_pin` is the list of all via layer names. This means that abstract will not cover obstruction on pins for blocks and io pads on all via layers if `-abstract_blockage_cut_around_pin` is specified. *Default:* ""
`-antenna_connectivity {`*`layer1 connect_layer layer2`*`}`	
	Specifies the layers for which connectivity is to be extracted. The syntax is `{`*`layer1 connect_layer layer2`*`}`. Example: `set_abstract_mode -antenna_connectivity {{poly V0 M1} {Diff V0 M1}}`. *Default:* ""
`-antenna_diffusion_geom {{`*`layer_name`*`} {`*`logical_expression`*`}}`	
	Specifies the diffusion layer, followed by the expression to remove the gate channel area from the diffusion region. Example: `set_abstract_mode -antenna_diffusion_geom {Diff_real {Diff andnot Poly}` *Default:* ""
`-antenna_gate_geom {{`*`layer_name`*`} {`*`logical_expression`*`}}`	
	Specifies the poly layer that forms a gate along with the layer expression to derive the gate area. This option also triggers the creation of antenna models. Example: `set_abstract_mode -antenna_gate_geom {{Gate} {Poly and Diff}}` *Default:* ""
`-blockage_detailed_layers` *`layer_list`*	Specifies the layers for which a detailed blockage model is to be created. The detailed blockage model generates blockages only at those places on the layer where there are real shapes in the cell. This only applies to the IO and block cells and not to standard cells. *Default*: ""
`-boundary_layers` *`layer_list`*	Specifies the drawn layers to extract the cell boundary. If not specified, abstract uses the prBounbdary object as the boundary size. By default, *`layer_list`* list is empty, and the prBoundary is used. The prBoundary comes from an explicit GDS layer that is used to compute the LEF SIZE (and OVERLAP OBS shapes for rectilinear boundaries). If there is no explicit GDS prBoundary layer, the boundary can be computed from the bounding box of geometries on this list of layers. **Note:** This option will not normally work for core standard cells because they almost always have diffusion and metal shapes extending outside the real prBoundary. These shapes are usually shared with the row above or below for the power rails. This option can be useful for blocks that have no prBoundary and where a little lost area due to worst-case bounding box approximations is acceptable. *Default*: ""

`-cell_symmetry {R0 | X | Y | R90 | X Y | X R90 | Y R90 | X Y R90}`	
	Specifies a property value for cell symmetry. The valid values for the property symmetry are R0, X, Y, R90, X Y, X R90, Y R90, and X Y R90. Example: `set_abstract_mode -cell_symmetry {X Y R90}` *Default:* `{X Y}`
`-export_lef_version {5.3 | 5.4 | 5.5 | 5.6 | 5.7 | 5.8}`	
	Specifies the version of LEF to be exported. The default version is the same as the version of the technology LEF file being imported. *Default*: `5.7`
`-extract_layers_power` *layer_list*	Specifies the layers through which the extractor extracts each power net. This is used to reduce the search space if abstract is taking too much run time for each cell. Usually not applicable to standard cells. For blocks, it is common to only extract the power nets on the top one or two metal layers. *Default*: ""
`-extract_layers_signal` *layer_list*	Specifies the layers through which the extractor extracts each net. This is used to reduce the search space if abstract is taking too much run time for each cell. Usually not applicable to standard cells. *Default*: ""
`-extract_pin_layers_power` *layer_list*	Determines the shapes, which are found by the extractor, to be turned into pins. This argument is valid only if the associated layer, geometry specification and connectivity have been provided in the technology LEF. Usually not applicable to standard cells. *Default*: ""
`-extract_pin_layers_signal` *layer_list*	Determines the shapes, which are found by the extractor, to be turned into pins. This argument is valid only if the associated layer, geometry specification and connectivity have been provided in the technology LEF. Usually not applicable to standard cells. *Default*: ""
`-input_cell_type {std | io | block}`	Selects the cell type being translated with abstract. The tool adjusts its settings on the basis of the selected cell type. *Default*: `std`
`-input_gds_layer_map_file` *file.map*	Specifies the file that maps GDS layer number/data type to LEF layer names in the Virtuoso format. *Default*: ""
`-input_lef_tech_file` *file.lef*	Specifies the technology LEF file. This is used to get all the layer names and their types, such as routing, cut and the layer order for extracting connectivity. *Default*: ""
`-keep_temp_files {true | false}`	Specifies to keep the temporary files/libraries that are created in the process after the `run_abstract` command finishes. *Default:* `false`
`-pins_analog_names` *pattern*	Identifies analog pin names based on label strings in the abstract to set the "USE ANALOG" property of LEF.
`-pins_clock_names` *pattern*	Identifies clock pin names based on label strings in the abstract to set the “USE CLOCK” property of LEF.
`-pins_ground_names` *pattern*	Identifies ground pins based on label strings in the abstract to set the “USE GROUND” property of LEF.
`-pins_output_names` *pattern*	Identifies output pins based on label strings in the GDS to set the direction to OUTPUT property of LEF.
`-pins_power_names` *pattern*	Identifies power pin names based on label strings in the abstract to set the “USE POWER” property of LEF.
`-pins_text_pin_map` *pattern*	Maps text labels to pins on nets. Specify the layer purpose pairs based on which you want the abstract generator to search for geometry of any given layer-purpose pair with a text label.
`-pre_skill` *file.il*	Specifies a skill file that will be executed in abstract before the auto-generated script is executed. Optional.
`-selected_cells` *pattern*	Selects cells to create abstract for using regex style expressions.

`-site_name` *site_names*	Specifies the site to be used for macros. This site is added to all the cells.
`-verbose {true|false}`	Specifies the verbose mode with extra print. *Default:* `false`

Examples

- The following command sets the values for the property symmetry:

```
> set_abstract_mode -cell_symmetry {R0, X, Y, R90}
> get_abstract_mode -cell_symmetry
-cell_symmetry {R0, X, Y, R90} # string, default="", user setting
{R0, X, Y, R90}
```

- The following command sets the version of LEF to be exported:

```
> set_abstract_mode -export_lef_version 5.3
> get_abstract_mode -export_lef_version
-export_lef_version 5.3 # enums={5.3 5.4 5.5 5.6 5.7 5.8}, default=5.7, user setting
5.3
```

Related Information

- `get_abstract_mode` command
- `run_abstract` command

set_cell_power_domain

```
set_cell_power_domain
[-help]
[-reset]
[-file file_name]
```

Defines domains for multi-VDD and multi-VSS cells. When reading an instance ASCII power file to perform power-rail analysis, if there are missing power values associated with ground pins, the software uses the information specified with the `set_cell_power_domain` command to derive domain association and distributes power to each ground net accordingly.

This domain information is stored in a side file that is used by power analysis to calculate power using the correct voltage for each domain. If you are specifying an external instance ASCII power file with pin information, this domain information is used to correctly distribute power to each pin.

Use this command before performing power and rail analysis.

Parameters

`-file file_name`	

	Specifies the name of the file containing the cell names and corresponding power-ground pairs.
	The format of the file is:
	`CELL: <cell name1>`
	`<power1> <gnd1>`
	`<power2> <gnd2>`
	`CELL: <cell name2>`
	`<power1> <gnd1>`
	`...`

`-help`	Outputs a brief description for the `set_cell_power_domain` parameter. For a detailed description of the command, use the `man` command: `man set_cell_power_domain`.
`-reset`	Resets all specified options back to default values.

Examples

- The following command specifies a file `domain_file` for the cell `mycell` having multiple power and ground pins (`VDD` at `1v` and `VDDm` at `0.84v`):
  ```
  set_cell_power_domain -file domain_file
  ```
  The following is a snippet from `domain_file`:
  ```
  CELL: mycell
  VDD_1v  VSS_1v
  VDDm_0.84v  VSSm_0.84v
  ```

Related Information

- `set_power_analysis_mode`
- `report_power`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

setDesignMode

```
setDesignMode
[-help]
[-reset]
[-addPhysicalCell {hier | flat}]
[-backsideBottomRoutingLayer layer]
[-backsideTopRoutingLayer layer]
[-bottomRoutingLayer layer]
[-compressedPGDB {true | false}]
[-congEffort {low|medium|high|auto}]
[-dual_rail_via_pitch min_via_pitch min_filler_via_pitch cell_boundary_spacing]
[-earlyClockFlow {true | false}]
[-earlyPBAMode {none | high}]
[-expressRoute {true | false}]
[-flowEffort {express | standard | extreme}]
[-idealHoldFixing {true|false}]
[-ignore_followpin_vias {true | false}]
[-merge_trim_shapes {{layers layer_name_list gap gap_value max_merge_num number_of_max_merge masks {mask_num ...}} ...}]
[-node
{N22|N12|N10|N7|N7Plus|N6|N5|N5PlusPlus|N4|N3|N3E|N2|S11|S10|S8|S7|S5|S4|S3|S2|G7|G5|ICF|I7|I5|P1277|P1278|P1280|C12|C7|unspecified}]
[-optimizationDensityScreenMargin margin]
[-pessimisticMode {true | false}]
[-powerEffort {none | low | high}]
[-process integer]
[-slackWeighting {unityWeighting|viewBasedWeighting}]
[-topRoutingLayer layer]
[-trim_grid_group group_name]
```

Specifies the process technology value. You can use this command to change the process technology dependent default settings globally for each application instead of setting several mode options.

With advanced technologies and new design, manufacturing techniques accurate delay calculation becomes more challenging. The main challenges are:

- Delay and slew interpretation of distorted waveforms.
- Waveforms are affected by:
  - Distributed RC
  - Back-miller current
  - Crosstalk and input noise
  - Different non-linear characteristics of gates
- Ultra low voltage applications

Tempus STA for advanced nodes has several capabilities for signoff timing accuracy at the respective nodes. The `setDesignMode` command automatically enables the node specific signoff STA setting required to meet the signoff STA accuracy challenges at the specific nodes.

Use the `getDesignMode` command to return the current settings for the `setDesignMode` command.

When you specify a process technology value using the `setDesignMode` command, the Innovus software assigns coupling capacitance threshold values to the RC extraction filters automatically. These values determine whether the coupling capacitances of the nets in a design will be lumped to the ground.

Note: In TQuantus extraction, the grounding of coupling capacitances also depends on the capacitance filtering mode set by the `-capFilterMode` parameter of the `setExtractRCMode` command.

The following table lists the coupling capacitance threshold values that are applied based on the specified process node. Note that the table might not be readable when viewed with the Unix man command. Check the `setDesignMode` command description in the *Innovus Text Command Reference* to view the table.

Process Node	Extraction Filters		
	coupling_c_th	relative_c_th	total_c_th
Above 130nm	3.0000	0.03	5.0000
130nm and below	0.4000	1	0.0000
90nm and below	0.2000	1	0.0000

65nm and below	0.1000	1	0.0000
45nm and below	0.1000	1	0.0000

Note: The filtering values set by the Innovus software, which are based on the process node setting, attempt to capture the most significant effect of coupling capacitances on SI analysis, and are strongly recommended. However, you can choose to specify different filtering thresholds using the `-coupling_c_th`, `-relative_c_th`, and `-total_c_th` parameters of the `setExtractRCMode` command.

In addition to the extraction filters, the software selects the extraction engine based on the process node. For 65nm and below process nodes, the software uses the TQuantus extraction engine, which provides scaling factors closer to 1 with better correlation to sign-off extraction. It also provides coupling capacitances to the wires on the layers above and below the victim wire.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `setDesignMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setDesignMode`
`-backsideBottomRoutingLayer` *layer*	Specifies the highest backside LEF layer name for global and detail routing. *Default*: `""`
`-backsideTopRoutingLayer` *layer*	Specifies the lowest backside LEF layer name for global and detail routing. *Default*: `""`
`-bottomRoutingLayer` *layer*	Specifies the lowest lef layer name for global and detail routing. For a special partial routing on the first routing layer in certain nodes, the routing enablement of that layer must be a global decision in the entire flow of the design. For example, if M0 or M1 routing is enabled, you need to set `setDesignMode -bottomRoutingLayer M0/1` from the init step. This bottom routing layer setting must not be changed in later stages of the flow. *Default*: `""`
`-compressedPGDB {true|false}`	Specifies whether the compressed PG feature is enabled. When it is set to `true`, `restoreDesign` will automatically compress the PG data while loading the design. You can use "dbGet top.isPGCompressed" to check if PG in the design is compressed. *Default*: `false`
`-congEffort {low|medium|high|auto}`	Specifies the congestion effort level. It controls placement as well the place_opt_design behavior. It is also possible to have other tuning for other steps in the flow. *Default:* `auto` Specify one of the following values: • `auto`: Automatically determines whether the design is congested and performs extra congestion driven effort for highly congested designs. • `high`: Runs more iterations of placement in an effort to achieve better congestion results. This parameter increases the run time. This also does congestion-aware buffering. When `-congEffort` is set to `high`, `optDesign` takes actions to be more congestion friendly and may call more iterations of placement to achieve better congestion results. It will also implicitly set `setPlaceMode -congEffort high` you have not set `setPlaceMode -conEffort`. However, if you have set `setPlaceMode -congEffort`, it will not override your setting. That is, the setting of "`setPlaceMode -congEffort`" will be honored. If you set both `setDesignMode -congEffort` and `setPlaceMode -congEffort`, the `placeDesign` and incremental placement will take your setting of `setPlaceMode -congEffort`. However for the flow, `optDesign` will take whatever tuning you have done for the `-congEffort` level. • `low`: Runs fewer iterations of placement to arrive quickly at a legal placement. This parameter might decrease placement quality. • `medium`: Runs placement on designs at a normal effort level.

`-reset`	Resets parameters to their default values. The `-reset` parameter *must* be the first parameter specified. If you specify `-reset` by itself, the software resets all `setDesignMode` parameters to their default values. Any values that do not already match the default value and are reset will be reported.
`-addPhysicalCell {hier | flat}`	
	Adds support for physical cells such as Add Filler, Add Well Tap, Add End Cap in a hierarchy that exists at the location of each of the newly added cells. *Default:* `hier`
`-dual_rail_via_pitch` *`min_via_pitch min_filler_via_pitch cell_boundary_spacing`*	
	This option is used when two horizontal power-rails on the first and third routing layer are used for standard-cells, and the min-spacing of stacked-vias between the dual-rails is controlled during placement and routing. • *`min_via_pitch`*: It is the minimum center-to-center pitch of the stacked-vias touching any active standard-cell (not a filler cell). • *`min_filler_via_pitch`*: It is the minimum center-to-center pitch of the stacked-vias touching only filler cells. • *`cell_boundary_spacing`*: Specifies how close a PIN or OBS shape must be to the cell boundary to block a stacked-via in the power rail at that point. The placer makes sure that there is room for a stacked-via as required, the global routers estimate the congestion cost before the stacked-vias are created, and the power-router adds them during the followpin routing as required.
`-earlyClockFlow {true|false}`	Enables the early clock flow inside `place_opt_design`. For the early clock flow inside `place_opt_design`, CCOpt creates a clock tree based on the initial clustering and annotates clock latencies for timing optimization. The flow also enables CCOpt ideal mode useful skew during the WNS/TNS fixing step inside `place_opt_design`. CCOpt skewing can be turned off by using `setOptMode -opt_skew_pre_cts false`. *Default*: `false`
`-earlyPBAMode {none | high}`	Specifies that early PBA mode should be used in Innovus during the `route_opt_design` run for delay calculation. This mode adjusts delay calculation settings to align to Tempus or `signoffTimeDesign` Path-Based Analysis (PBA) timing results. Early PBA mode is only supported in `route_opt_design` based optimization. You can enable early PBA mode before launching the `route_opt_design` command. This mode should only be used in Innovus, you must disable early PBA mode in Tempus, `signoffTimeDesign, and signoffOptDesign`. *Default*: `none`
`-expressRoute {true|false}`	Specifies to force routing-stage with the express flow, that is `-flowEffort express`.
`-flowEffort {express|standard|extreme}`	
	Guides the effort of every super-command. • `express`: Configures the flow to give the best turnaround time with good WNS/area correlation as compared to the standard flow. The flow is appropriate for prototyping. • `standard`: Configures the flow to give the best overall combination of quality of results and full-flow turnaround time. This flow is appropriate for the majority of designs. • `extreme`: Configures the flow to give the best quality of results at some cost in turnaround time. This flow is appropriate for designs where timing closure is challenging. *Default:* `standard` **Note:** The individual parameters of `set*Mode` commands have higher priority over the `setDesignMode -flowEffort` parameter.
`-idealHoldFixing {true|false}`	Enables the ideal hold fixing flow. *Default:* `false`

`-ignore_followpin_vias {true|false}`	This option directs the detailed placer, optimization, and signal routing to ignore any sViaInst (special-route vias) with the followpin shape. You can use this option if the two bottom routing layers both have a horizontal routing direction, and you use dual power-rails on those two layers on top of each other for the standard-cell power rails. In this usage, other commands and options will add as many sViaInsts as possible between the power-rails, and remove any sViaInsts that cause spacing violations between the sViaInst cut shapes and any cell or signal routing cut shapes. This gives the placer and router more freedom to get higher density, run faster, but still put as many sViaInsts as possible between the dual power-rails. When set to true, followpin vias are ignored during detailed placement, optimization, and routing. *Default:* `false`
`-merge_trim_shapes {{layers` *`layer_name_list`* `gap` *`gap_value`* `max_merge_num` *`number_of_max_merge`* `masks {`*`mask_num`* `...}} ...}`	
	Specifies the setting for trim shapes merge. • `layers`: You use this t specify the trim layer list which needs to merge trim shapes. • `Gap`: You use this setting when the end to end gap is less the value. • `max_merge_num`: You use this to specify the max trim shapes number that are merged. • `masks`: You use this to specify the trim shapes color that needs to be merged. *Default*: `""`
`-node {N22|N12|N10|N7|N7Plus|N6|N5|N5PlusPlus|N4|N3|N3E|N2|S11|S10|S8|S7|S5|S4|S3|S2|G7|G5|ICF|I7|I5|P1277|P1278|P1280|C12|C7|unspecified}`	
	Sets the design technology node. *Default*: `unspecified`
`-optimizationDensityScreenMargin` *`margin`*	Increases margin on the top of original density screen for optimization. *Default*: `0` *Min*: `0` *Max*: `1`
`-pessimisticMode {true|false}`	Specifies that delay calculation pessimistic mode should be used in Innovus during implementation. This mode adjusts delay calculation settings to maintain a degree of pessimism during implementation to help limit optimistic outliers during external timing signoff. **Note**: The pessimistic mode must not be used in Innovus if Tempus is the final signoff timing engine. *Default:* `false`
`-powerEffort {none|low|high}`	
	Specifies the effort to reduce the dynamic and leakage power. The tradeoff between dynamic and leakage power can also be controlled. • `none`: No power optimization is performed and power is not considered during timing optimization. • `low`: Power is considered during timing optimization and power optimization is performed in all flow stages. • `high`: Additional algorithms are used to minimize power consumption. Turnaround time with this option will be longer than using `low` or `none`. *Default:* `none`

`-process integer`	Specifies the process technology value to set for all the applications. Units in nanometers (nm). The `setDesignMode -node` and `-process options` to assign a tech node (such as N7) and a process number (such as 7) for the design. Innovus tools (placement, routing, etc) then use both the tech node and the process number to differentiate the designs and provide specific functions. It is important for the design to have a correct node and process information for the Innovus tools to work properly. *Type:* Integer *Range:* 2-250 **Note**: The process 2 is under the beta control. Contact your Cadence representative for more information related to this command. *Default:* 90
`-slackWeighting {unityWeighting|viewBasedWeighting}`	
	Specifies slack weighting to be used in the implementation flow. *Default*: `unityWeighting`
`-topRoutingLayer layer`	Specifies the highest lef layer name for global and detail routing. *Default*: `""`
`-trim_grid_group group_name`	Specifies the GROUP name of the set from which the trim metal grid will be used for placement and routing. *Default:* "" If the specified group name does not exist in LEF, the tool will return an ERROR. See the LEF documentation on the TRIMMETALTRACK keyword for more details.

Examples

- The following command sets the specified process technology value to 80:
  ```
  setDesignMode -process 80
  ```
- The following command resets the `-process` parameter to its default value:
  ```
  setDesignMode -reset -process
  ```
- The following command also resets the `-process` parameter to its default value:
  ```
  setDesignMode -reset
  ```
- The following commands forces the super commands to use their extreme-effort settings:
  ```
  setDesignMode -flowEffort extreme
  place_design or optDesign or routeDesign
  ```

Related Information

- `getDesignMode`
- `optDesign`
- `place_opt_design`
- `restoreDesign`
- `routeDesign`
- `setExtractRCMode`
- `setOptMode`
- `setPlaceMode`
- Design Implementation Flow chapter of *Innovus User Guide*
- Top-level Timing Closure Methodologies for iHDB Flow chapter of *Innovus User Guide*

setExportMode

```
setExportMode
[-help]
[-reset]
[-fullPinout {true | false}]
[-implicitPortMapping {true | false}]
```

Controls certain aspects of how the software writes out a Verilog netlist.

Use the `getExportMode` command to return the current settings for the `setExportMode` command.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `setExportMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setExportMode`.
`-fullPinout {true | false}`	
	Writes out all connected and unconnected pins in the netlist. *Default:* `false`
`-implicitPortMapping {true | false}`	
	Writes out the netlist with implicit port mapping (that is, ordered port connections or position-based port connections) for hierarchical instances. *Default:* `false`
`-reset`	Resets parameters to their default values. The `-reset` parameter *must* be the first parameter specified. If you specify `-reset` by itself, the software resets all `setExportMode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported.

Examples

Assume the following hierarchical module:

```
module

block (out1, out2, out3, A, B, C);

.

.

.

endmodule
```

- The following example shows how the `saveNetlist` command writes out an instance of the module `block` in the netlist by default:
  ```
  block i1(.out2(Y), .A(n1), .B(n2), .C(n3));
  ```

- The following command instructs the `saveNetlist` command to write out all connected and unconnected pins when it saves the netlist:
  ```
  setExportMode -fullPinout true
  ```

  The instance of module `block` is written in the saved netlist as follows:
  ```
  block i1(.out1(), .out2(Y), .out3(), .A(n1), .B(n2), .C(n3));
  ```

- The following command instructs the `saveNetlist` command to write out the netlist with implicit port mapping for hierarchical instances when it saves the netlist:
  ```
  setExportMode -implicitPortMapping true
  ```

The instance of the module `block` is written in the saved netlist as follows:

```
block i1( , Y, , n1, n2, n3);
```

- The following command resets the `-fullPinout` parameter to its default value:

  ```
  setExportMode -reset -fullPinout
  ```

- The following command resets all `setExportMode` parameters to their default values:

  ```
  setExportMode -reset
  ```

Related Information

- `getExportMode`
- `saveNetlist`

setGenerateViaMode

```
setGenerateViaMode
[-help]
[-reset]
[-auto {true|false}]
[-ndr_only {true|false}]
```

Auto-generates vias required by nanoroute for the default and non-default rules (NDRs). The vias are based on the wire-widths of the routing-rule and are created to meet all the DRC via enclosure rules with variations for good pin-access, variations of double-cut vias for good double-cut via utilization, and min-area vias required for stacked-vias that have no route connected to them to meet the min-area metal rule.

This command should be used before init_design, in which case the vias will be created during the initial netlist load. If you change the settings after loading the design, you will need to use saveDesign/restoreDesign to trigger the auto-generation of vias with the new settings. If you want to experiment with different settings inside one Innovus session, use generateVias instead.

The command requires correct DRC rules (e.g. enclosure, spacing, minimum-cut, min-area rules, etc), and at least one LEF VIARULE GENERATE statement (or OpenAccess standardViaDef) for every via layer used for routing. The cut enclosure values in the VIARULE GENERATE statement are not used except for old LEF technology files (LEF version 5.5 and older) that have no enclosure rules (this is not recommended usage, but is kept for backward compatability). The VIARULE cut-spacing values are ignored–only the DRC cut-spacing values are used.

No predefined vias are required (e.g LEF VIA DEFAULT, or NONDEFAULTRULE USEVIA is not necessary, and OpenAccess routing ConstraintGroup (NDR) definitions do not require any validVias list).

If there are predefined vias, the generated vias are compared to the predefined vias and if they are geometrically identical, that specific generated via is not kept so no duplicate vias are created. In that case a combination of pre-defined and auto-generated vias will be used by nanoroute.

The auto-generated vias are created as parameterized vias rather than fixed vias (e.g. VIA using VIARULE rather than RECT statements in DEF, or an OpenAccess standardVia rather than customVia).

Parameters

`-help`	Prints a brief description that includes the type and default information for each `setGenerateViaMode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setGenerateViaMode`.
`-reset`	Resets all or specified options to default value. Any values that do not already match the default value and are reset will be reported.
`-auto {true|false}`	Enables auto via generation when design is loaded. The option is of type bool. *Default*: false
`-ndr_only {true|false}`	Generates vias for non default rules only. *Default:* `false`

Example

- `setGenerateViaMode -auto true`
  Will add auto-generated vias to any existing vias for all routing rules when the design is loaded.

See also:

- `dumpOutVias` to write out the vias
- `generateVias` to generate vias in the current Innovus session
- Design Implementation Flow chapter of *Innovus User Guide*.

setImportMode

```
setImportMode
[-help]
[-reset]
[-discardFloatingVNets {true|false}]
[-keepEmptyModule {true| false}]
[-minDBUPerMicron DBUPerMicron]
[-syncRelativePath {true | false}]
[-timerMode {true|false}]
```

Controls aspects of how the software reads in the Verilog netlist and other design files.

Use the `getImportMode` command to return the current settings for the `setImportMode` command.

You can use the `setImportMode` command before importing a design.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `setImportMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setImportMode`.
`-discardFloatingVNets {true|false}`	
	If `true`, an hnet (Verilog wire statement) that has nothing connected to it, is deleted from the netlist. The default value is `false` in Innovus to allow for netlist repair, but `true` in Tempus to delete non-timed nets and reduce memory. *Default*: `false`
`-keepEmptyModule {true | false}`	
	Keeps empty modules. If set to false, the software converts the empty modules into special leaf cells that do not have a physical library associated with them. If the module is a hard macro, using a value of `true` or `false` produces the identical behavior: the software keeps the module. *Default*: `true` Note: Empty modules can be removed later from the design by using the `deleteEmptyModule` command.
`-minDBUPerMicron` *`DBUPerMicron`*	
	Sets the minimum database units per micron allowed when LEF files initialize the database units. You can specify a value for the minimum database units per micron allowed when LEF files initialize the database units. This option is intended for users with LEF files that have database units that are not fine enough for NanoRoute to run properly - when NanoRoute requires the LEF database units to be finer than the manufacturing grid. If you do not want to modify your LEF file, you can use this option to override the LEF database units value. The default value is 0, which means that the software uses the LEF file units. The software uses the units set in the LEF file, if the LEF units are greater than the minimum database units you specify. Conversely, if the units you specify are greater than the LEF units, the software uses the units you specify. The value you set is saved so the next time you open the design it will have the same database units. *Default:* 0
`-reset`	Resets parameters to their default values. The `-reset` parameter *must* be the first parameter specified. If you specify `-reset` by itself, the software resets all `setImportMode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported.
`-syncRelativePath {true | false}`	
	Synchronizes all relative paths in the top.globals file to the current working directory. Use this parameter if you want to restore the design without first changing your current working directory to the directory saved in the previous design session. *Default*: `false` (You must change your working directory to the directory saved in the previous design session before you restore the design.)
`-timerMode {true|false}`	When set to true, the tool loads only the Verilog and MMMC file (including timing constraint, timing lib, and RC extraction) during `restoreDesign`.

Examples

- The following command synchronizes all relative paths to the current working directory:
  ```
  setImportMode -syncRelativePath true
  ```
- The following command resets the `-syncRelativePath` parameter to its default value:
  ```
  setImportMode -reset -syncRelativePath
  ```
- The following command resets all `setImportMode` parameters to their default values:
  ```
  setImportMode -reset
  ```

Related Information

- `deleteEmptyModule`
- `getImportMode`
- `restoreDesign`
- Importing and Exporting Designs chapter in the *Innovus User Guide*
  - Handling Verilog Assigns
- Partitioning the Design chapter in *Innovus User Guide*
  - Defining Blackboxes
  - Blackbox Flow
- Using the NanoRoute Router chapter in the *Innovus User Guide*
  - Checking Your LEF Files

setLibraryUnit

```
setLibraryUnit
[-help]
[-cap capUnit]
[-time timeUnit]
```

Sets the time and capacitance unit values for constraints and report generation.

The timing constraints (SDC) files do not contain any unit specification for capacitance and time values. Hence, the units for these values are considered to be same as those specified in the timing libraries. The commands, e.g., `report_timing`, require consistent units to generate reports. If an inconsistency occurs in the timing libraries, the default unit of time (as ns) and capacitance (as pf) is used.

You can use this command when you need to override the library units, or force the setting of units when libraries have different units.

Note: You must set the `setLibraryUnit` command before the design and constraints are read.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `setLibraryUnit` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setLibraryUnit`.
`-cap` *capUnit*	Specifies the capacitance load unit in the timing library. *Default:* `1.0` pf
`-time` *timeUnit*	Specifies the time unit to be used for timing constraints. The valid values are 1ns, 1ps, 10ps, 100ps. *Default:* `1.0` ns

Example

- The following command sets the time unit for the design to `10`ps:
  ```
  setLibraryUnit -time 10ps
  ```

Related Information

- Timing Analysis chapter in the *Innovus User Guide*

setNet

```
setNet
[-help]
{-net netName | -file fileName}
[-setTermSpecial]
-type {regular | special}
```

Marks the specified nets and their related pins as either special nets and pins, or regular nets and pins.

When the Verilog list is first loaded into the design, all LEF pins except those with `USE POWER` and `USE GROUND` are considered "regular pins" by default, and are routed by Trial Route and NanoRoute. You can use this command to mark nets (and their pins) as special prior to routing to keep NanoRoute from attempting to route them. This is useful when you have nets that need to be routed by hand, or with other specialized routers.

If you later decide you want the nets to be routed by NanoRoute, you can use the `setNet` command to set them back to regular.

Note: The `setNet` command has no effect if the net is already routed.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `setNet` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setNet`.
`-file fileName`	Changes all nets (and their related pins) in the specified file to be either special or regular. The file must contain a list of net names, with one net name per line.
`-net netName`	Changes the specified net and its related pins to be either special or regular.
`-setTermSpecial`	Sets the term
`-type {regular | special}`	
	Specifies the type to which to change the net. `regular`: Changes the net and its pins to a regular net, so that NanoRoute will route them. `special`: Changes the net and its pins to a special net to keep NanoRoute from attempting to route them.

Example

The following command sets the net `myAnalogNet` and its related pins as special:

```
setNet -net myAnalogNet -type special -setTermSpecial
```

The NanoRoute router ignores this net when routing.

setStreamOutMode

```
setStreamOutMode
[-help]
[-cellInstanceColor {true|false}]
[-cellMasterColor {true|false}]
[-cellNameUserPrefix {prefix}]
[-cellNameUserSuffix {suffix}]
[-check_map_file {true|false}]
[-ignoreFixedMask {true|false}]
[-labelAllPinShape {true | false}]
[-mergeAppend {true|false}]
[-oasisCompression {true|false}]
[-oasisLayerName {true|false}]
[-oasisSCellOffset {true|false}]
[-pinTextOrientation {default | automatic}]
[-reset]
[-removeNets {list_of_nets}]
[-SEcompatible {true | false}]
[-SEvianames {true | false}]
[-snapToMGrid {true | false}]
[-specifyViaName {default | format_string}]
[-streamConvertRectToPath {true|false}]
[-streamVersion version_number]
[-supportPathType4 {true | false}]
[-textSize value]
[-uniquifyCellNamesPrefix {true | false}]
[-virtualConnection {true | false}]
```

Controls certain aspects of GDSII Stream files generated by the Innovus software.

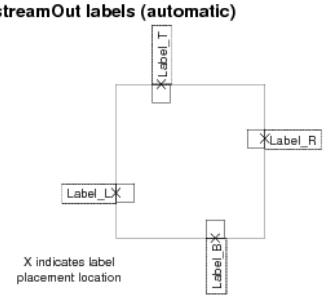
You can use the `setStreamOutMode` command any time before running the `streamOut` command.

Use the `getStreamOutMode` command to return current settings for the `setStreamOutMode` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setStreamOutMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setStreamOutMode`.
`-cellInstanceColor {true|false}`	
	Streams out the instance-based color. The instance mask shift is honored in `streamOut`. When the option is on and the `-merge` option is used, the merged gds files with mask shift suffix must be provided through the `-merge` option. Default: *false*
`-cellMasterColor {true|false}`	
	Streams out the cell-based color from the Innovus color engine. Default: *true*
`-cellNameUserPrefix {prefix}`	
	Adds a prefix to all cell names being written out. Uses the user-specified prefix to avoid potential conflict when integrating different macros, which are potentially two different cells (in two different libraries) with the same name.
`-cellNameUserSuffix {suffix}`	
	Use the user-specified suffix instead of appending color pattern to normal cell names. This applies only to the fixed color shift (flip) pattern (i.e. the single-shift, flip, pattern) applied to the design. *Default:* ""
`-cellNameUserSuffix {suffix}`	

	Use user-specified suffix instead of appending color pattern to normal cell names. This applies only to the fixed color shift (flip) pattern (i.e. the single-shift, flip, pattern) applied to the design. *Default:* ""
`-check_map_file {true|false}`	
	Checks the specified map file. If the objects exist in Innovus and but these objects are not mapped in the map file, the missed objects mapping and missed objects are printed in the log file. Tool checks only the main objects mapping. *Default*: `false`
`-ignoreFixedMask {true|false}`	Ignore macro FIXEDMASK keywords, enable color flip function.
`-labelAllPinShape {true | false}`	
	Labels all pin shapes, whether or not they are physically connected, when set to `true`. *Default*: `false`
`-mergeAppend {true|false}`	
	Merges the object shapes from the Innovus DB and object shapes from the external cell gdsii/oasis files. If the metal layers have multi-pattern shapes, the color information from the Innovus DB is used. For example: • If the instance color is not flipped, the instance object shapes are merged to the object shapes from the same name cell of external gdsii/oasis files. • If the instance color is flipped, the instance object shapes are merged to the object shapes from the same name cell (with mask shift suffix) of external gdsii/oasis files. If there is no color shift cell in the gdsii/oasis files, the same name cell is used from external gdsii/oasis files. *Default*: `false`
`-oasisCompression {true|false}`	
	CBLOCK compression is partial-selective encryption technology uses OASIS CBLOCK records to encrypt arbitrarily encapsulated sets of data records within an OASIS layout file. REPETITION compression supports an 'array' of cell placements, geometries, or text elements, instead of write each the element separate only different in position, this can save duplicate information for an OASIS layout file size. *Default*: `true`
`-oasisLayerName {true|false}`	
	Support LAYERNAME in the OASIS file. *Default*: `false`
`-oasisSCellOffset {true|false}`	Includes cell offset table to declare the byte offset from the beginning of the file (byte 0) to the corresponding CELL record location that appears in the file. *Default*: `false`
`-pinTextOrientation {default | automatic}`	

	Controls the pin text orientation, making it more readable. • `default` implies that the labels are centered in the pin shapes and have `R0` orientation and `LowerLeft` justification. • `automatic` implies that the labels for pin shapes that are abutted to the design boundary are centered on the edge of the pin shape. **streamOut labels (automatic)** 
`-removeNets {`*`list_of_nets`*`}`	
	Removes the specified nets from the output file. *Default:* `""` (empty string)
`-reset`	Resets parameters to their default values. The `-reset` parameter *must* be the first parameter specified. If you specify `-reset` by itself, the software resets all `setStreamOutMode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported.
`-SEcompatible {true | false}`	
	Output instance only if `FOREIGN` is specified. *Default:* `false`
`-SEvianames {true | false}`	
	Determines the naming convention used for vias declared in the LEF file. *Default:* `false` • Specify `true` to create unique names. • Specify `false` to retain LEF via names.
`-snapToMGrid {true | false}`	
	Specifies whether to snap the nets' wires to the manufacturing grid. *Default:* `false`
`-specifyViaName {default |` *`format_string`*`}`	
	Specifies the naming convention that the `streamOut` command uses for via cells declared in the DEF file. *Default:* `%t_VIA%u` (This default value is used so that the names of the via cells are compatible with versions 4.2 and earlier versions of the Innovus software.) **Note:** When the version of the GDS file is 3 (this is the default version), via names are truncated if they are longer than 32 characters. The software automatically appends a unique identifier to names if there is a name conflict. You can use any of the following characters to specify a format string to generate names that give you information about the cell contents. Separate the characters with underscores (_) to improve readability. You can list them in any order.

	• `%c` Specifies the number of columns. • `%l(lcu)` Specifies the via layers. For the lower layer, use `%l` or `%l(l)`. For the cut layer, use `%l(c)`. For the upper layer, use `%l(u)`. You can specify one, two, or all three layers. You must enclose the `l`, `c`, and `u` in parentheses, as in the following examples: `%l(c)`, `%l(lu)`. • `%n` Specifies the number of cut rows and columns that make up a cut array. • `%r` Specifies the number of cut rows. • `%t` Specifies the top structure name. • `%u` Specifies a unique number for the via. • `%v` Specifies the via name in the DEF file. For more information, see "Vias," in the DEF Syntax chapter of the *LEF/DEF Language Reference*.
`-streamConvertRectToPath {true|false}`	Converts DEF SPECIALNETS path/rectangle and FILLS shapes to GDSII/stream PATH data instead of BOUNDARY data when enabled. Conversion to path data can result in significantly smaller output database when there are many rectangle shapes (example: after metal fill is included). *Default:* `false`
`-streamVersion` *`version_number`*	Specifies the version for a created GDSII Stream file. If you use the `streamOut -merge` parameter, `streamOut` uses the highest version of the merged files, instead of the value specified by the parameter. *Default:* `3` *Minimum*:`1` *Maximum*: `32767`
`-supportPathType4 {true | false}`	
	Converts `PATHTYPE4` to `PATHTYPE0`. If you specify `-supportPathType4 false`, the software converts `PATHTYPE4` paths to `PATHTYPE0` when outputting the path. *Default:* `true`

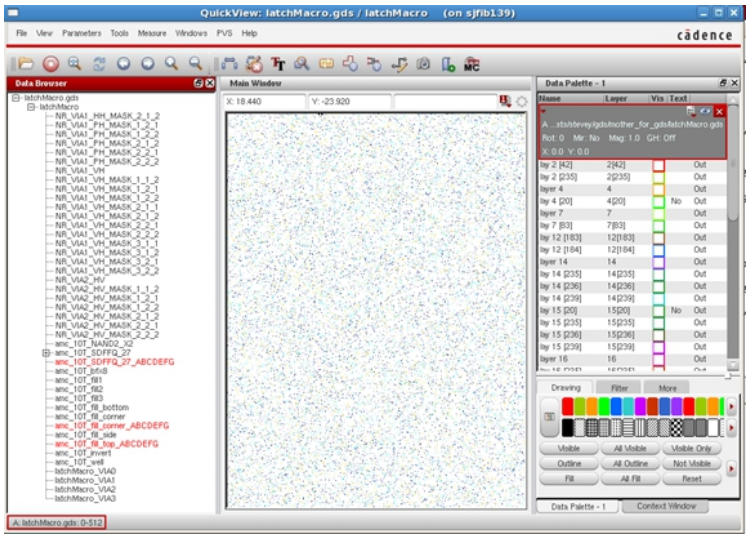
`-textSize` *`value`*	Changes the size of the text used in the text labels in the GDSII Stream file written by the Innovus software.
	Default: `1`
	• To increase the size of the text, specify a positive real number that is greater than 1, as in the following example:
	`setStreamOutMode -textSize 10.0`
	• To decrease the size of the text, specify a positive real number that is less than 1, as in the following example:
	`setStreamOutMode -textSize 0.5`
	Note: The `-textSize` parameter affects the size of only automatically created text. It does not affect text created through `add_gui_text`.

`-uniquifyCellNamesPrefix {true | false}`	
	Adds a prefix, instead of a suffix, to uniquified cell names. The prefix is the source filename. If this parameter is not specified, the software adds a suffix (also the source filename) to uniquified cell names.
`-virtualConnection {true | false}`	

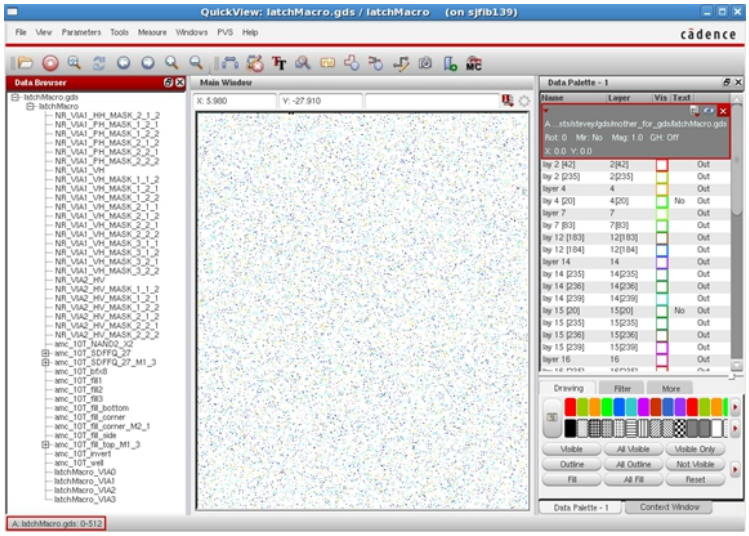
	Specifies whether a colon (`:`) label is appended to pin names for DEF pins with the `.extra`*N* syntax and to LEF pins with multiple ports (if `-outputMacros` is specified for LEF pin ports that have disjoint shapes). *Default:* `true` • Specify `true` to append a colon to pin names. • Specify `false` not to append a colon to pin names. **Note:** For more information, see the DEF Syntax chapter of the *LEF/DEF Language Reference*.

Examples

- The following command retains the via names defined in the LEF file:
  ```
  setStreamOutMode -SEvianames false
  ```
  When you type this command and run the `streamOut` command, the Innovus software does the following:
  - For `block1`, the software creates `VIA12Default`, `VIA12DoubleCut`, and so on.
  - For `block2`, the software creates `VIA12Default`, `VIA12DoubleCut`, and so on.
    If `block1` and `block2` are later merged into a top-level design, name collisions of the via masters occur. This result is acceptable if these are the same vias. However, if the vias are not the same, this command causes errors.
- The following command creates unique via names for vias defined in the LEF file:
  ```
  setStreamOutMode -SEvianames true
  ```
  When you type this command and run `streamOut`, the Innovus software does the following:
  - For `block1`, the software creates `block1_VIA1`, `block1_VIA2`, and so on.
  - For `block2`, the software creates `block2_VIA1`, `block2_VIA2`, and so on.
    If `block1` and `block2` are later merged, there is no risk of name collision between via masters.
- The following command generates names for via cells. The names identify the top structure name (`%v`), lower via layer (`%l(l)`), and DEF via name (`%v`):
  ```
  setStreamOutMode -specifyViaName %t_%v_%l(l)
  ```
- The following command resets the `-SEvianames` parameter to its default value:
  ```
  setStreamOutMode -reset -SEvianames
  ```
- The following command resets all `setStreamOutMode` parameters to their default values:
  ```
  setStreamOutMode -reset
  ```
- The following command specifies “ABCDEFG” as the suffix for the color flipped cells.
  ```
  setStreamOutMode -cellNameUserSuffix ABCDEFG
  ```
  When this command is used, “ABCDEFG” is suffixed for color flipped cells.



Without this command, suffix are automatically added for color flipped cells.



Related Information

- `add_gui_text`
- `getStreamOutMode`
- `streamOut`
- Importing and Exporting Designs chapter in the *Innovus User Guide*
- *DEF Syntax* chapter in *LEF/DEF Language Reference*

specify_lib

```
specify_lib
[-help] [<in_file <file1[file2] ...>>] [-reset]
```

Specify the name of the timing library file(s) for dynamic power analysis. This command allows you to run the dynamic power analysis flow without reading a dotlib into design database to reduce background memory consumption. You must have an external TWF file to run this flow as the timing engine will not be invoked from within the software. This flow is recommended for large designs where front-end processing of timing libraries is high. Some of the database query commands will not work if `specify_lib` is used.

This command works in conjunction with the `specify_def` and `specify_spef` commands.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `specify_lib` parameter.
in_file <file1[file2] ...	
	List of timing library files.
`-reset`	Resets all specified options back to default values.

Examples

- The following Tcl shows the use model of the `specify_lib` command that works in conjunction with the `specify_def/specify_spef` flow to perform dynamic power analysis:

```
read_lib -lef tech1.lef tech2.lef
specify_lib /../library1.lib /../library2.lib
specify_def adder_routed.def
specify_spef {test.spef }
read_twf test.twf2
read_verilog test2.v
set_top_module test
set_default_switching_activity -comb_clockgate_ratio 1 -period 2.01ns -input_activity 0.2 -icg_ratio 1
set_power_analysis_mode -default_supply 1.1 -method dynamic_vectorless
report_power
```

streamOut

```
streamOut
gdsFileName
[-attachInstanceName attributeNumber]
[-attachNetName attributeNumber]
[-attachNetProp {{prop_name1 attr_num1} {prop_name2 attr_num2........}}]
[-area {x1 y1 x2 y2} [-strict][-enclosed_instances_only]]
[-dieAreaAsBoundary]
[-format {stream oasis}[-outputInstanceName instName] [-outputNetName netName]]
[-help]
[-ignoreSubtype {NETTYPE SHAPE}]
[-libName libraryName]
[-mapFile mapFile]
[-mergeViaCellMapFile cell_list]
[-merge {listOfExternalGDSOASISFiles} [-noTSV tsv_name_list][-uniquifyCellNames]]
[-reportFile file_name]
[-mode {ALL | FILLONLY | NOFILL | NOINSTANCES}]
[-offset x y]
[-outputCriticalNetColorFromFileOnly sideFile]
[-outputMacros]
[-signoff_fill]
[-structureName structureName | -noStructureName]
[-units {100 | 200 | 1000 | 2000 | 10000 | 20000}]
```

Creates a GDSII Stream file of the current database. Writes a summary of errors and warnings to the log file.

You can use the `streamOut` command after the design is placed, but is most commonly used only after the design is routed (which may also include metal fill shapes).

- If you do not specify a map file, the Innovus software creates a generic map file, in GDSII Version 3 format. The generic file shows the mapping that was used, if none was specified. You must customize the file to make it appropriate for your design. You can change the version by using the `setStreamOutMode -version` parameter.
- You must specify all layers to stream out. Layers that are not specified in the map file are not included in the file created by this command.
  Note: A cut layer in the map file (for a via) can have both MASK and SIZE at the same time.
- If you specify the `-merge` parameter and you want to merge files with a version number greater than `3`, the software generates a file with the highest version number from the merged files.
- If you do not specify `-outputMacros`, `LEFPIN` and `LEFOBS` do not apply. If you do specify `-outputMacros` and `LEFPIN` and `LEFOBS` are not specified in the map file for the layers in the LEF macros, the GDSII structures for those macros will be empty.
- If `MAXVOLTAGE` and `MINVOLTAGE` constructs are there in the layermap, `streamOut` will output a max voltage label and a min voltage label, for each net, to the GDSII file.

Note: Pin label is attached to physical pin shapes. If physical pin shapes do not exist, the pin text is labeled on any one instance pin shape. If both physical pin and instance pin shapes do not exist on the net, no pin text is generated in the output.

Parameters

`-attachInstanceName` *attributeNumber*	Specifies the instance name value added to the attribute on SREF of the specified attribute number. The attribute number is an integer and its range is 1-127.
`-attachNetName` *attributeNumber*	Specifies that the net name will be added to the specified attribute number on the regular and special net wire shapes and vias SREF, but not on PIN shapes and pin vias. The attribute number is an integer and its range is 1-127.

`-attachNetProp {{prop_name1 attr_num1} {prop_name2 attr_num2........}}`	Specifies the attribute number on wiring shapes that will be used to store the value of the property associated nets in the design. For this option, you specify a list of property name and its attribute number. **Note:** Multiple property names can be specified. Only wiring shapes associated with nets that contain one or more of the specified property names will have values assigned to their attributes. **Example:** `-attachPropName { {prop1 100} {prop2 101} }` In the above example, the attribute number `100` will be used to store the value of the property `prop1`, and the attribute number `101` will be used to store the value of the property `prop2`. It is a list of lists with a pair of property and its attribute number.
`-area {x1 y1 x2 y2}`	Defines the area of the design to be output. Any shapes or instances enclosed, intersecting, or touching the specified area are output. So the data that is output can extend outside the specified area. By default the entire design is output.
gdsFileName	Specifies the GDSII Stream output file. Include the `.gz` extension to enable file compression (for example, `GDS_file.gds.gz`).
`-dieAreaAsBoundary`	Forces the `streamOut` command to write the die area as a polygon (that is, a boundary). Otherwise, the `streamOut` command writes the die area as a rectangle (that is, a box).
`-enclosed_instances_only`	Specifies to stream out only those instances in which the shapes are fully enclosed by the specified area.
`-format {stream oasis}`	Specifies the tape out file format.
`-help`	Outputs a brief description that includes type and default information for each `streamOut` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man streamOut`.
`-ignoreSubtype {NETTYPE SHAPE}`	Ignores the map file line containing the subtype keyword , and warning message is printed in the log file. For example, in the example below, with `-ignoreSubtype {NETTYPE SHAPE}` the line 61, 62, and 63 of the map file will be ignored. • Line61: M2 SPNET:NETYPE:POWER:MASK:2 32 12 • Line62: M2 SPNET:SHAPE:STRIPE:MASK:2 32 22 • Line63: M2 SPNET: NETYPE:GROUND:SHAPE:STRIPE:MASK:2 32 32 • Line64: M2 SPNET:MASK:2 32 2
`-libName` *libraryName*	Specifies the library to convert to GDSII Stream format. *Default:* `DesignLib`
`-mapFile` *mapFile*	Specifies the file used for layer mapping. This file is required for successful stream out, and is read by the next tool used in the design flow. Important If you do not specify a file with this parameter, the Innovus software will read a `streamOut.map` file if one exists. If a `streamOut.map`file does not exist, the software creates a generic map file that you must customize for your design. For information on the map file format, see "GDSII or OASIS Map File Format" in the Importing and Exporting Designs chapter of the *Innovus User Guide*.

`-merge {`*`listOfExternalGDSOASISFiles`*`}`	Specifies a single file or list of files to merge. Checks all merged files for name collisions, and generates a warning if any names are changed. Uses the following conventions for new names: • Replaces dot separators (`.`) with underscores (_). For example, `test.test_gds.gds` is renamed `test_test_gds_gds`. • Adds unique identifiers. For example, if more than one cell is named `test.test.gds.gds`, they are renamed `test_test_gds_gds_1`, `test_test_gds_gds_2`, and so on. Supports the mixed format of the GDSII and OASIS files. You need to specify the merged files with mixed format using the `-merge` option. The tool will identify the files format, covert them, and merge them to a stream out file. Compressed files are acceptable (include the `.gz` extension to enable file compression). The Innovus software automatically creates blackboxes when merging and writing macro files. It ignores any cells in the merge files that are not used in the design. If you specify more than one file, separate the file names with spaces and enclose the list of file names in braces (`{}`) or double quotation marks (`"`) You can use a wildcard to read all `.gds` files in a specified directory. The usage is as follows: `streamOut -merge` *`GDSIIdir`*`/*.gds` where *`GDSIIdir`* is the name of the directory that contains the `.gds` files. ***Use caution when using wildcards, as they might affect the order in which the software reads merge files.***
`-mergeViaCellMapFile` *`cell_list`*	Specifies the list of cells that need merge via cell from an external file.
`-mode {ALL | FILLONLY | NOFILL | NOINSTANCES}`	

	Identifies the layers to write.
	Specify one of the following:
	• `ALL`--Writes all layer information specified in mapFile as follows:
	  ◦ All instances
	  ◦ All via instances
	  ◦ All generated via cells
	• `FILLONLY`--Writes only fill layers specified in mapFile as follows:
	  ◦ No instances
	  ◦ Only fill via instances
	  ◦ Only generated fill via cells
	• `NOFILL`--Writes only non-fill layers specified in mapFile.
	• `NOINSTANCES`--Writes wiring (including vias) only; does not write out `COMPONENT` instances.

`-noStructureName`	Enables you merge to files without loading an Innovus database when the `-merge` parameter is also specified.
`-noTSV` *`tsv_name_list`*	Specifies a list of TSV via names that should not be written out.
`-offset` *`x y`*	Specifies, in microns, the location of the design in the GDSII output file. When the GDSII output file is saved, the design shifts to the specified location.

`-outputCriticalNetColorFromFileOnly` *`sideFile`*	Specifies the net list and the layer range through a side-file. The streamOut command outputs the object color on the specified layers and nets, Other objects remain gray in GDS.
	Here is an example of a side file:
	`<LAYERMAP>`
	`M5 PIN:MASK:1,NET:MASK:1,SPNET:MASK:1,VIA:MASK:1 35 271`
	`M5 PIN:MASK:2,NET:MASK:2,SPNET:MASK:2,VIA:MASK:2 35 272`
	`M6 PIN:MASK:1,NET:MASK:1,SPNET:MASK:1,VIA:MASK:1 36 271`
	`M6 PIN:MASK:2,NET:MASK:2,SPNET:MASK:2,VIA:MASK:2 36 272`
	`</LAYERMAP>`
	`<NETNAMELIST>`
	`TOP/NET1`
	`TOP/NET2`
	`...`
	`</NETNAMELIST>`

`-outputInstanceName` *`instName`*	Writes the instance name to the oasis file.
`-outputMacros`	Writes LEF abstract information such as LEF pin geometries and obstructions for macros. Writes one text label per port within a LEF macro pin plus one text label per shape for feedthrough pins only if `setStreamOutMode -labelAllPinShape` is not set to `true`. Specify this parameter to create GDSII output that contains the LEF macro structures as well as the design data. You can also use this parameter to debug LEF macro definitions against verification DRC rules. The output contains macro pins and obstructions. **Note:** `LEFPIN` and `LEFOBS` apply only when you specify this parameter. If you specify this parameter and `LEFPIN` and `LEFOBS` are not specified in the map file for the layers in the LEF macros the GDSII structures for those macros will be empty.
`-outputNetName` *`netName`*	Writes the net name to the oasis file.
`-signoff_fill`	Merges the signoff fill data attached to the current DB into the output GDS file and instantiates the signoff top fill cell at 0,0 R0 inside the top design cell. The signoff fill format must be GDS.
`-reportFile file_name`	Outputs cell name changed to the file.
`-strict`	Truncates the object outside the area.
`-structureName` *`structureName`*	
	Specifies the superstructure that consists of all geometries that the map file writes out. *Default:* Current design's top cell name
`-uniquifyCellNames`	Creates unique cell names if there are name collisions when the software merges files. Creates unique cell names if there are name collisions when the Innovus software merges files. ⓘ You must specify the `-merge` parameter if you specify this parameter. When you list the merge files, list the top-level file first, as the software uses the first name in the search path when renaming cells. When a cell name is repeated, the software renames the cell *`cellName_filename`*, where *`filename`* is the name of the merge file in which the cell name appears. The software changes the name of references to that cell in the merge file. The software changes the reference name if the referenced cell is defined in the same file, regardless of whether the cell is defined before or after the reference. If a reference in a merge file refers to a cell definition in a different merge file, then the reference is not changed in the GDSII Stream output file. *Default:* If you do not specify this parameter, the software ignores duplicate cell names.
`-units {100 | 200 | 1000 | 2000 | 10000 | 20000}`	Specifies the resolution for values in the GDSII file. Choose `100`, `200`, `1000`, `2000`, `10000`, or `20000`. *Default:* Units specified in LEF file

Examples

- The following command creates a GDSII Stream file (merging cells from the ios.gds file):

```
streamOut amba_usb.gds -mapFile streamOut.map -libName DesignLib
  -attachInstanceName 12 -merge {ios.gds}
  -attachNetName 15
  -units 2000 -mode ALL
  -dieAreaAsBoundary
```

- The following command creates blackboxes for cells not found in the merge libraries and writes macros only for those cells:

```
streamOut newFile.gds.gz -merge {myFile1 myFile2 myFile3} -outputMacros
```

- The following command merges cells needed in the design from all files in `myDirectory` with the current database:

```
streamOut myFile.gds.gz -merge myDirectory/*.gds
```

  If a merge file contains cells such as `A`, `B`, `C`, `D`, `X`, `Y`, and `Z`, and the design contains only cells `A`, `B`, and `D`, then the Innovus software merges only cells `A`, `B`, and `D`.

- The following example shows a stream out map file. The first column specifies the layer object name, and the second column specifies the layer object type. The third and fourth columns specify the GDS layer and data type to stream those objects, respectively. When the object name is "NAME", it indicates the text label for the object types specified in the second column.

```
==========original streamout.map file=============

M1        NET          1        0

M1        SPNET        2        0

M1        PIN          3        0

M1        LEFPIN       4        0

M1        FILL         5        0

M1        VIA          6        0

M1        VIAFILL      7        0

M1        LEFOBS       8        0

NAME      M1/NET       1        0 <--- text for nets

NAME      M1/SPNET     2        0 <--- text for special nets (i.e. power nets)

NAME      M1/PIN       3        0 <--- text for pins including top level IO pins

NAME      M1/LEFPIN    4        0 <--- text for LEF pins

================================================
```

Related Information

- `defStreamOutCheckUncolored`
- `run_pegasus_drc`
- `run_pvs_drc_rules`
- `setStreamOutMode`
- Importing and Exporting Designs chapter in the *Innovus User Guide*
  - Converting an Innovus Database to GDSII Stream or OASIS Format
  - About the GDSII Stream or OASIS Map File

trimDesign

```
trimDesign
[-help]
[-coreToBoundary distance]
[-dir output_dir_to_save_trimmed_design]
[-encryptName proc_name_to_encrypt]
[-name trim_design_name]
[-nets list_of_net_names]
[-placeOnly]
[-area {x1 y1 x2 y2} | -polygon list_of_points | -selectedRouteBlk ]
```

Allows you to specify the coordinates of a box to create a small portion of the design as an independent design. This trimmed down version of the design can be then used for investigation and analysis. When you share the trimmed design for investigation of a localized issue, you do not need to share the whole third-party IP information, thereby ensuring confidentiality of the third-party data.

While the trimmed design completely preserves the physical data, it does not preserve the timing and the power domain data. Working on a small portion of the design offers a faster turnaround time and a lesser memory footprint.

Notes:

- The `trimDesign` command retains the attributes and properties of all the standard cells and macros that are completely inside the trim box/area. All hard macros that overlap with the trim box/area are also preserved. It does not include any standard cells or macros that are completely outside the specified trim box/area.
- The `trimDesign` command preserves all overlapping rows, sites, and colors.
- The `trimDesign` command flattens all instances and net names. All instances in the trim box/areal lose their original logic hierarchy and appear ‘flat’ in the trimmed database. Any naming conflicts arising due to flattening are automatically resolved. For example, h1/h2/i1 in the full design will appear as i1 in the trimmed design. Also, h1/h4/i1 may appear as i1_uniquify1, to distinguish it from i1.
  Note: To resolve naming conflicts due to the loss of hierarchy, additional name uniquification is done. A mapping of original nets/instances names vs their trimmed design modified names, is written in a map file for book keeping purpose.
- The `trimDesign` command maintains the design parameters such as instance location/orientation, regular/special wire and its connectivity relative to the original full design.
- The `trimDesign` command retains the attributes and properties of all the nets having some instances inside the trim box/area. It also preserves (pushes down) the nets that are not connected to any instance in the trim box/area, but have some wires overlapping with trim box/area. Nets having all instances and wires outside of trim box/area are discarded.
- The `trimDesign` command creates ports for all wires crossing the boundary of the trimmed design.
- The `trimDesign` command preserves the tracks, attributes and colors of all wires that are completely inside the trim box/area. All wires that overlap with the trim box/area are also preserved. It does not include any wire that is completely outside the specified trim box/area.
- The `trimDesign` command preserves the Power/Ground/Special wires that overlap with the boundary of the trim box by cutting them against the boundary. It creates additional PG ports at cross points these wires.
- The `trimDesign` command discards fences, regions, guides, instance groups, and power domains.
- The `trimDesign` command saves the `.globals` files. The `.globals` file has the .lef list copied from original full design’s .globals file.
- The `checkPlace` command reports all violations that were there on instances in the box in the actual design. However, since hierarchical fences are not brought back, the trimmed design may not have the Region/Fence and Not-of-Fence Violations.
- The `verify_drc` command reports all violations that were there on nets in the box of actual design. However, since only the net wires crossing box boundary are preserved, the trimmed design may have some extra violations.

Parameters

`-help`	Prints a brief description that includes type and default information for each trimDesign parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man trimDesign`
`-area` *box_coordinates*	List of coordinates to represent a box for getting the trimmed design. **Note**: The specified box is snapped to the instance grid. All instances in the box lose their original logic hierarchy and appear flat in the new db. *Data_type*: string, required
`-coreToBoundary` *distance*	Specifies that any instance lying within the distance of coreToBoundary (in microns) to the trim box will be discarded. *Default*: 2 *Data_type*: string, optional
`-` `dir` *output_dir_to_save_trimmed_design*	Specifies the name of the directory in which the data related to the trimmed-design is saved. *Data_type*: string, optional
`-encryptName` *proc_name_to_encrypt*	Specifies that the names of instances and nets in the design will be encrypted when the trimmed design is saved. *Data_type*: string, optional
`-name` *trim_design_name*	Specifies the top-level name of the new trimmed design. *Data_type*: string, optional
`-nets` *list_of_net_names*	Species the nets that are to be completely enclosed by the specified shape by taking union of shape with net-boxes. *Data_type*: string, optional
`-placeOnly`	Specifies that only instances will be pushed. *Data_type*: bool, optional
`-polygon` *list_of_points*	List of coordinates to represent a polygon for getting the trimmed design. *Data_type*: string, optional
`-selectedRouteBlk`	Specifies that the selected routing blockages will be used to infer the box or polygon for getting the trimmed design. *Data_type*: string, optional

Examples

- The following command creates a trimmed design, `trim_db`, with the specified polygon coordinates and save the data in the DB file.

  ```
  trimDesign -dir DB -name trim_db -polygon {{500.0 700.0} {500.0 900.0} {500.0 900.0} {700.0 900.0} {700.0 900.0} {700.0 800.0} {700.0 800.0} {800.0 800.0} {800.0 800.0} {800.0 700.0} {800.0 700.0} {500.0 700.0} }
  ```

- The following command creates a trimmed design, based on the selected routing blockage:
  ```
  trimDesign -dir DB -name trim_db -selectedRouteBlk
  ```

- The following command uses a `get_new_name` procedure to encrypt the names on the instances and nets in the trimmed design `trim_db`:
  ```
  trimDesign -dir DB -name trim_db -area {300.0 200.0 800.0 700.0} -encryptName gen_new_name
  ```

  The following is a structure of the encryption procedure file. You can customize this procedure:

  ```
  proc gen_new_name {name} {
    global n
    if {![info exists n]} {
      set n 0
    }
    set new_name en_name__$n
    incr n
    return $new_name
  }
  ```

Related Information

- Importing and Exporting Designs chapter in the *Innovus User Guide*
  - Trimming the Design

write_do_lec

```
write_do_lec
[-help]
output_filename
[-checkpoint lec_checkpoint_file]
[-golden_design filename]
[-log_file filename]
[-no_exit]
[-pre_compare filename]
[-pre_exit filename]
[-pre_read filename]
[-revised_design filename]
[-verbose]
[-write_session lec_session_name]
{-hier | -flat }
> output_filename
```

Writes out the dofile for the Conformal® Logical Equivalence Checker tool.

For formal verification of the Innovus-generated netlists, the `write_do_lec` command generates, within Innovus, the Conformal LEC scripts (such as dofile). These scripts are used for equivalence checking. You can generate a flat or a hierarchical comparison script. The dofile written out by this command contains standard Conformal flow commands and you will need to add the DFT constraints if netlist is scan inserted.

The `write_do_lec` command creates a verification directory, if does not already exist, in which the name mapping information file will be created. The name mapping file will usually contain the mapping information of MBIT flops against its split/single bit candidate. By default, the name of this verification directory is `fv/invs/%s`. You can use `write_lec_directory_naming_style` to change the directory name.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `write_do_lec` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_do_lec`
`output_filename`	Specifies the name for the output file to be written.
`-checkpoint` *`lec_checkpoint_file`*	Specifies the name of the LEC checkpoint file to save at the end.
`-flat`	Creates a flat comparison script.
`-golden_design` *`filename`*	Specifies the user-written netlist file name to use as the golden design.
`-hier`	Creates a hierarchical comparison script.
`-log_file` *`filename`*	Specifies the name of the LEC log file.
`-no_exit`	Skips the exit command at end of dofile.
`-pre_compare` *`filename`*	Specifies an extra dofile to be sourced right before 'compare'.
`-pre_exit` *`filename`*	Specifies an extra dofile to be sourced right before 'exit'.
`-pre_read` *`filename`*	Specifies an extra dofile to be sourced right before reading library/design.
`-revised_design` *`filename`*	Specifies the user-written netlist file name to use as the revised design.
`-verbose`	Specifies whether the generated dofile will report verbosely.
`-write_session` *`lec_session_name`*	Specifies the name of the LEC session to save at the end.
`>` *`output_filename`*	Specifies the name for the output file to be written.

Related Information

- `write_lec_files`
- `write_lec_directory_naming_style`
- `write_name_mapping`

write_lef_abstract

```
write_lef_abstract
[-help]
fileName
[-add_obs_layers layer_list]
[-cutObsMinSpacing]
[-cutObsToExposeRouting distance_in_microns]
[-excludeObsLayers layer_list]
[-excludePinLayers layer_list]
[-design_boundary point_list]
[-extractBlockObs]
[-extractBlockPGPinLayers layer_number_list]
[-extractBlockSignalPinLayers layer_list]
[-extractWellLayerObs well_layer_list [-extractWellObsByRow margin]]
[-ignoreBumpOnPin {all pg signal}]
[-ioPadPin]
[-noCutObs]
[-property]
[-extractWellLayerObs well_layer_list [-mergeConnectedWellOnly ]]
[-obsSpacing {val layers [val2 layers ...]} [-obsSpacingPerLayer]]
[-specifyTopLayer layer_number [-obs_above_top_layer layer+]]
[-stripePin [-selected]]
[-stripePin [-portForEachStripePin]]
[-stripePin [-PGPinLayers layer_number_list]]
[-stripePin [-PGNets net+]]
[ -5.6 | -5.7 | -5.8 ]
```

Generates design abstract (LEF) information for the current routed block-level design. By default, `write_lef_abstract` copies all the current design PG and signal pins to the LEF and creates cut-outs in the obstructions (OBS shapes) around the pins. The output LEF file contains process antenna information, if you run `verify_antenna` before you run the `write_lef_abstract` command.

A routing blockage with a pushdown attribute (e.g. a DEF BLOCKAGE with + PUSHDOWN) is treated as an area of routing reserved for the level above this block. The LEF will have these blockages cut-out of the OBS.

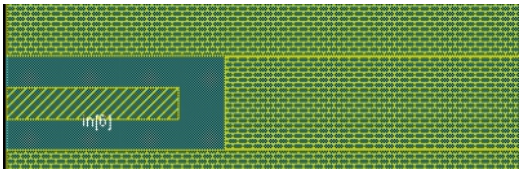
In a bottom-up flow (where the block was not created using the `createPartition` command), you normally create a simple LEF abstract that contains:

- one obstruction (`OBS`) shape per routing layer without cutting around pins
- signal pins located along the block boundary
- add power pins for the top metal layer PG stripes (see `-stripePin and -PgpinLayers` options)
- embedded bump pins shapes are also written out as LEF pin shapes

If the block used four routing layers out of a possible six layers, you would run the following command:

```
write_lef_abstract -noCutObs -stripePin -PGpinLayers 4 -specifyTopLayer 4
```

Parameters

`-help`	Prints a brief description that includes the type and default information for each `write_lef_abstract` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_lef_abstract`
`-5.6 | -5.7 | -5.8 | -6.0`	Specifies the LEF version number. This value overrides any value you set with the `lefDefOutVersion` variable. *Default:* The software uses the current `lefDefOutVersion` value. The default value for `lefDefOutVersion` is `5.8`.
`-add_obs_layers layer_list`	Writes the shapes on on the specified layers as OBS shapes in the LEF. These layers cannot be routing or cut layers.
`-cutObsMinSpacing`	Creates cut outs in the blockages around pins using minimum width to calculate the spacing. This option cannot be used with `-noCutObs`. 
`-cutObsToExposeRouting` *`distance_in_microns`*	Exposes routing within the specified halo distance to a LEF model.
`-design_boundary` *`point_list`*	Specifies a list of points defining the design boundary polygon for the output LEF (a rectangle would use 4 points). It is used instead of the real design boundary to compute the LEF SIZE, OBS shapes to block routing layers, and OBS shapes on the OVERLAP layer if it is non-rectangular. The LEF will also include any special routes that are outside the design boundary as OBS shapes unless they are converted to PIN shapes based on other options. It does not matter if PIN shapes are inside or outside the design boundary, they are always written out. This is intended for a block that only has bumps and/or top-level RDL routing to the bumps that are outside the "real" block boundary. The real block boundary can be specified with this option to shrink the SIZE and OBS shapes to just include the active area of the block, and let the bumps and RDL routing stick outside. An important limitation is that any instances or regular routing shapes (for example, created by `routeDesign`) outside the design boundary will be ignored. It is assumed that placement and routing blockages are used to prevent this outside the "real" block boundary.
`-excludeObsLayers` *`layer_list`*	Ignores the specified layer when saving OBS in the macro LEF file. You can use this parameter to set the layer as the following format: ◦ One layer: For example, if you specify `-excludeObsLayers {AP}`, the AP layer is not saved to OBS. ◦ Multiple layers: For example, if you specify `-excludeObsLayers {M7 M8}`, M7, M8 layers are not saved to OBS.
`-excludePinLayers` *`layer_list`*	Specifies not to create PIN shapes on the specified layers.
`-extractBlockObs`	Extracts all obstruction (`OBS`) information from the instantiated `CLASS BLOCK`, `CLASS PAD AREAIO`, and `CLASS RING` macros and writes out the obstructions to the output LEF file. `OBS` information is extracted for layers that are not already being completely covered under the normal routing layer range and for cut layers between routing layers that are completely covered.
`-extractBlockPGPinLayers` *`layer_number_list`*	

	Extracts the power and ground pin information from any `CLASS BLOCK`, `RING, PAD,` or `COVER` macros with pins on layers in the specified layer list and writes out the pin information to the LEF file.
	The layer names can be either names in the technology file like metal1, or technology independent names like 1 for the first routing layer, 2 for the second routing layer.
	For example, to create PG pins on layers `metal3` and `metal4`, use:
	`-extractBlockPGPinLayers {metal3 metal4}`
	Or
	`-extractBlockPGPinLayers {3 4}`

`-extractBlockSignalPinLayers` *`layer_list`*	Extract signal pins, connected to design term, from the BLOCK, RING, PAD, COVER macros on these layers.
`-extractWellLayerObs` *`well_layer_list`*	Extracts OBS from instances on the specified well layers. If you have DRC spacing value checks for the well layers in your LEF, you can copy the well layer OBS shapes out to the LEF abstract so the well shapes can be checked at the next level up.
`-extractWellObsByRow` *`margin`*	Extracts OBS from the bounding box of rows plus the margin on the specified well layers. You can create well layer OBS shapes in the LEF abstract for all the standard cell rows, with additional margin in microns. This is useful if the standard-cell LEF abstracts do not have well shapes in them but you want to see well OBS in the abstract. This option requires the `-extractWellLayerObs` option to indicate which layers to create.
`fileName`	Specifies the LEF output file.
`-ioPadPin`	Writes out the IO pads that are connected to the design pins as LEF pin shapes.
`-ignoreBumpOnPin {all pg signal}`	By default, pin shapes of bump cells (e.g. LEF MACRO CLASS BUMP) are written out as LEF pin shapes. This option can be used to suppress those LEF pin shapes depending on the type of net they are connected to.
`-mergeConnectedWellOnly`	Merges connected well shapes only, instead of merging all shapes into one bounding box.
`-noCutObs`	Creates a single obstruction shape for each routing layer, except routing blockages with the pushdown attribute are still cut-out. *Default:* Creates cut-outs in the LEF OBS for signal pins, and power and ground pins. This option cannot be used with `-cutObsMinSpacing`.
`-property`	Writes out all the user-defined design properties.
`-obsSpacing {`*`val layers`* `[`*`val2 layers ...`*`]}`	

	Writes out the LEF58_OBSSPACING property for the MACRO. This overrides the global USEMINSPACING OBS setting and sets the spacing required between routing and the OBS shapes of just this MACRO. This is equivalent to adding a SPACING *`value`* to every OBS RECT on the affected layers in the LEF. Note, it will only affect an OBS RECT that has no existing SPACING *`value`*.
	`val` = 'min' for min-spacing, and *`layers`* = {} for all layers.
	The order of the arguments is the order in the LEF file. In LEF, the last value for a given layer "wins" over previous values for that macro.
	For example, `-obsSpacing {min {} 1.0 {m2 m3} 2.0 {m5}}` would add this to the LEF:
	`PROPERTY LEF58_OBSSPACING "`
	`OBSSPACING MIN ;`
	`OBSSPACING 1.0 LAYER m2 LAYER m3 ;`
	`OBSSPACING 2.0 LAYER m5 ; " ;`
	So, OBS shapes in this MACRO on every layer would use SPACING <min-spacing> for that specific layer, then m2, m3 are changed to use SPACING 1.0, and finally m5 is changed to use SPACING 2.0.

`-obsSpacingPerLayer`	Writes out the spacing for each layer under the layer definition. This parameter does not work without `-obsSpacing`.
	Data_type: bool, optional
	To assign the value to all layers, you should set the layers as "{}". For example, if you specify `write_lef_abstract -obsSpacing {1.0 {}} -obsSpacingPerLayer`, it means the spacing of all layers is 1.0
	`Macro A`
	`  ...`
	`  OBS`
	`  Layer M1 SPACING 1.0 ;`
	`  RECT ...;`
	`  Layer M2 SPACING 1.0 ;`
	`  RECT ...;`
	`  ...;`
	`  END`
	`  ...`
	`END A`
	The following example sets the spacing of all layers as 1.0, except `M2`.
	`write_lef_abstract -obsSpacing {1.0 {} 2.0 {M2}} -obsSpacingPerLayer`
	`Macro A`
	`  ...`
	`  OBS`
	`  Layer M1 SPACING 1.0 ;`
	`  RECT ... ;`
	`  Layer M2 SPACING 2.0 ;`
	`  RECT ... ;`
	`  Layer M3 SPACING 1.0 ;`
	`  RECT ... ;`
	`  ...`
	`  END`
	`  ...`
	`END A`

`-obs_above_top_layer` *layer+*	Writes out OBS shapes rather than one large abstract OBS shape for special-routes on these layers that are not PIN shapes. The layers will have OBSSPACING FULLDRC added to the LEF so the OBS shapes on those layers are treated as 'real' shapes for DRC checking, and not abstract obstructions. This is intended for blocks that have some sparse special-routing above the blocks "normal" top-layer set with the `-specifyTopLayer` option (for example, it might be a few special-routes for clocks that match up with the chip clock distribution but must be visible inside the block). Only special-routes are currently supported.
`-PGNets` *net+*	Writes out PG stripes only on these PG nets. If you specify a net name that does not exist in the design, or the net is not a PG net, tool will give an error and stop.
`-PGpinLayers` *layer_number_list*	

	Writes out power and ground stripes on the specified layer numbers as power and ground pins.
	The layer names can be either names in the technology file like metal1, or technology independent names like 1 for the first routing layer, 2 for the second routing layer.
	For example, to create PG pins on layers metal3 and metal4, use:
	`-PGPinLayers {metal3 metal4}`
	Or
	`-PGPinLayers {3 4}`
	In the example below, the command will write out metal5 and metal6 stripe shapes, and any via5 cut-shapes on the PG nets to the LEF abstract as PIN shapes
	`-PGPinLayers {metal5 via5 metal6}`
	Note: You must specify the `-stripePin` parameter in order to use this parameter.
	Default: The top metal layer

`-portForEachStripePin`	Creates a separate LEF port for each pin shape created by `-stripePin`. This means that `verifyConnectivity` will check that every port is touching a PG routing shape at the next level. This is normally what users want for individual PG stripes on the top-layer of the block - each separate stripe shape should be connected to a top-level route or via somehow. This can help in detecting some missing power-mesh connections before running a complete IR drop analysis.
`-selected`	Writes out only the selected PG stripes as LEF pin shapes.
`-specifyTopLayer` *layer_number*	

	Creates obstructions (OBS shapes) covering the block only for layers up to the specified layer.
	The layer name can be either a name in the technology file like metal1, or a technology independent names like 1 for the first routing layer, 2 for the second routing layer.
	For example, to specify metal4 as the top layer, use:
	`-specifyToplayer metal4`
	Or
	`-specifyTopLayer 4`
	By default, all routing layers in the technology file are fully obstructed with OBS shapes across the entire block (possibly cut around pin shapes). If this block does not use some of the highest routing layers, you should use this option to limit the OBS shapes to below layer.

`-stripePin`	Writes out power and ground stripes on the top metal layer as power and ground pins. You can use this parameter in a bottom-up methodology, when you create abstracts for blocks that you will use at the next higher level of a hierarchical design. Only special-routes created by power-route commands will be output as pin shapes. That means special-routes that have a shape value of ring, stripe, followpin, iowire, corewire, blockwire, padring, and blockring are included. The shape value is the same as. DEF SPECIALNETS + SHAPE value or dbGet sWire .shape value. Special-routes with a shape value of blockagewire, drcfill, fillopc, fillwire, or notype are not output.

Example

Suppose you have a design that was routed on only the first 5 routing layers, even though the technology has higher metal layers.

You would use the following command to include any PG stripes on metal5 as pins in the block LEF to allow connecting to them from above, and to only create OBS for routing layers up to layer 5.

```
write_lef_abstract my.lef -specifyTopLayer 5 -stripePin -PgpinLayers 5
```

If your design also included a block inside that has PG pins on metal5 with no PG stripes in the current design covering them, you can include layer 5 PG pins from the internal blocks using the following command:

```
write_lef_abstract my.lef -specifyTopLayer 5 -stripePin -extractBlockPGPinLayers 5
```

If you also want to include metal4 PG stripes as pins, and metal4 PG pins from the block, you would use the following command:

```
write_lef_abstract my.lef -specifyTopLayer 5 -stripePin -Pgpinlayers {4 5} -extractBlockPGPinLayers {4 5}
```

If you want to include some sparse special-route shapes on metal6, metal7 (for example, for a few manually routed clock nets) , you would use the following command:

```
write_lef_abstract my.lef -specifyTopLayer metal5  -obs_above_top_layer {metal6 metal7}
```

Related Information

- `lefDefOutVersion`
- `routeDesign`
- `verify_antenna`
- Importing and Exporting Designs chapter in the *Innovus User Guide*
- Top-level Timing Closure Methodologies chapter in the *Innovus User Guide*
  - Example Top-Level Implementation Flow with ILMs

write_name_mapping

```
write_name_mapping
[-help]
[-bbox_pins]
[-hierarchical]
[-map_pins {input output all}]
[-pin_polarity]
[-enable_one_to_one_mapping |-report_power_format]
[-skip_clock_gate]
[-updated_only]
{output_filename | [[-output output_directory] -prefix filename_prefix]}
[-hdl | -initial | [-input_mapping_file mapping_file [-print_deleted][-include_new_iterms ]]]
```

Writes or updates the initial netlist key point names and corresponding current netlist key point names to a file. Mapping file consists of pin mapping information of all changes done during the flow (between comparison points) including polarity changes .

Parameters

`-help`	Prints a brief description that includes the type and default information for each `write_name_mapping` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_name_mapping`
`-bbox_pins`	Prints the pins of bbox in the output mapping file instead of inst names. • Prints bbox input pins if `-map_pins {input}` is specified. • Prints bbox output pins if `-map_pins {output}` is specified. • Prints both input and output pins if `-map_pins {all}` is specified.
`-enable_one_to_one_mapping`	When the value is set as `true`, only one of the clones is printed in the mapping file.
`output_filename`	Specifies the name of the output mapping file.
`-hdl`	Enables the generation of name mapping from current names to the names defined by the 'hdl_name' attribute. This will be used for power analysis.
`-hierarchical`	Specify if write out other post boundary optimizations mapping
`-initial`	Enables the generation of name mapping from current names to names defined by the 'initial_name' attribute. This will be used for verification of initial input netlist (post-synthesis) to any other netlist generated during the Innovus flow. The `write_do_lec` command in Innovus will call `write_name_mapping -initial` so that Conformal LEC do file and name mapping file are generated in one shot. The do file will source the name mapping file to facilitate key point mapping in LEC.
`-include_new_iterms`	When the value is set as `true`, the output mapping file writes the updated iterms as well as the new iterms that exist in Innovus but not in the input mapping file.
`-input_mapping_file` *`mapping_file`*	When provided Tool will read existing mapping file and write out updated mapping file.
`-map_pins {input output all}`	Specify type of pins to write out. *Default*: input
`-output` *`output_directory`*	Specify output directory
`-pin_polarity`	Specify if write out polarity information
`-prefix` *`filename_prefix`*	Used for file name prefix
`-print_deleted`	When `-print_deleted` is specified the mapping file will also print deleted instances.
`-report_power_format`	Specify if write out 1Pass power replay format.
`-skip_clock_gate`	Filters out ICGs that are not in RTL. If you set `-skip_clock_gate` to `true` with `-hdl`, the tool skips the clock gate pins except the pins with hdl_name. If `-skip_clock_gate` is set to `true` and `-hdl` is not specified, the tool skips all clock gate pins.
`-updated_only`	Skips printing the pin that does not have any change/update during optimizations/flow.

Related Information

- `read_name_mapping`
- `write_do_lec`
- `write_lec_files`
- `write_lec_directory_naming_style`

Import and Export Global Variables

- cds_lib
- check_design_across_hierarchy
- check_design_report_async_pins
- check_design_report_scan_pins
- check_design_threshold_fanout
- cvd
- defHierChar
- defInCheckMaskShifts
- defInShapeBasedDefFile
- defOutCompressVia
- defOutLefNDR
- defOutLefVia
- defOutPolygonDieArea
- defOverlapWireReportFileName
- defStreamOutCheckUncolored
- enc_save_binary_timing_constraints
- enc_save_timing_constraints_always
- fp_vertical_row
- ignore_max_via_stack_rule
- init_abstract_view
- init_check_output_pin_constant
- init_cpf_file
- init_design_netlisttype
- init_design_settop
- init_design_uniquify
- init_gnd_net
- init_ignore_pgpin_polarity_check
- init_import_mode
- init_io_file
- init_layout_view
- init_lef_check_antenna
- init_lef_check_mask_shifts
- init_lef_file
- init_mmmc_file
- init_mmmc_version
- init_oa_default_rule
- init_oa_design_cell
- init_oa_design_lib
- init_oa_design_view
- init_oa_foundry_rule
- init_oa_ref_lib

- init_oa_search_lib
- init_oa_special_rule
- init_original_verilog_files
- init_pwr_net
- init_top_cell
- init_verilog
- init_verilog_tolerate_port_mismatch
- inn_save_lef_ignore_abstracts
- lefdefInputCheckColoredShape
- lefDefOutVersion
- lefExtend2DCellShapes
- lefExtendCellObsShapeUnderTrim
- lefIgnoreLayerNames
- merge_multi_port_to_single_port
- readWriteLefCheckUncoloredShapes
- save_cmd_file_limit
- write_lec_dft_constraints
- write_lec_directory_naming_style
- write_lec_files
- writeLefAbstractCustomerHeader

cds_lib

`cds_lib` *`string`*

Type: String

Default: NULL

Specifies the library path as derived from `cds.lib` information. The `cds.lib` entries are available from Tcl in `cds_lib vars.cds_lib` is a system-defined, transient (non-persistent) variable. It should be treated as a *read-only* variable.

Related Information

- `init_design`
- `restoreDesign`

check_design_across_hierarchy

```
check_design_across_hierarchy {true | false}
```

Type: `Boolean`

Default: `true`

Checks for undriven/multi-driven/unloaded/constant pins/ports by checking across hierarchical boundaries. By default checks are across hierarchy.

Related Information

- `check_design_report_async_pins`
- `check_design_report_scan_pins`
- `check_design_threshold_fanout`

check_design_report_async_pins

```
check_design_report_async_pins {true | false}
```

Type: `Boolean`

Default: `true`

Checks async pins in a netlist design.

Related Information

- check_design_across_hierarchy
- check_design_report_scan_pins
- check_design_threshold_fanout

check_design_report_scan_pins

```
check_design_report_scan_pins {true | false}
```

Type: `Boolean`

Default: `true`

Checks scan pins in a netlist design.

Related Information

- check_design_across_hierarchy
- check_design_report_async_pins
- check_design_threshold_fanout

check_design_threshold_fanout

```
check_design_threshold_fanout {true | false}
```

Type: `Integer`

Default: `10000`

Checks for upper threshold in a netlist design.

Related Information

- check_design_across_hierarchy
- check_design_report_async_pins
- check_design_report_scan_pins

cvd

`cvd` *`string`*

Type: String

Default: NULL

Specifies the path name to the physical location of the lib/cell/view that was "restored" using `restoreDesign -cellview`. `cvd` is a system-defined, transient (non-persistent) global variable used in the OpenAccess flow. This should be treated as a *read-only* variable.

Related Information

- `restoreDesign`

defHierChar

`defHierChar` *`delimiter`*

Type: String

Default: `/`

Specifies the hierarchy delimiter character for the `defOut` and `defOutBySection` commands. For example, to set / as the delimiter, use the following code:

```
set defHierChar /
```

Note:

- You cannot unset this variable.
- The `defHierChar` is for the `defOut` and `defOutBySection` commands only. It does not change the hierarchy delimiter character of Innovus.

The hierarchy delimiter used appears in the output DEF File as shown below.

```
DIVIDERCHAR "/" ;
```

Related Information

- `defOut`
- `defOutBySection`

defInCheckMaskShifts

```
defInCheckMaskShifts {on | off | bypass}
```

Type: `Enum`

Default: `off`

Checks conflicts between DEF COMPONENT MASKSHIFT and MACRO MASKSHIFT values with the LEF FIXEDMASK and LAYERMASKSHIFT values, respectively, in the technology and MACRO sections. You can set the following values:

- `on`: When the variable is set as on, the the tool will check for the conflict. In case of a conflict, the tool will give an error and stop running.
- `off`: When the variable is set as off, the tool will not check for the conflict.
- `bypass`: When the variable is set as bypass, the tool will give and error message and will continue running.

Related Information

- `restoreDesign`

defInShapeBasedDefFile

```
defInShapeBasedDefFile {0 | 1}
```

Type: Integer

Default: `0`

Specifies that the input file is a shape-based DEF. DEF file generated by shape-based router may contain overlapping routing. By default, `defIn` drops any overlapping wire segment. Set this variable to `1` if you want `defIn` to consider overlapping wire segments.

If the router of a segment of a metal is connected with the two other segments of the same metal, `defInShapeBasedDefFile` will consider this as "overlapping" metals. For example, the metal5 segment of net A is connected with different length of other metal5 segments of net A. This connection will be considered as overlapping metals.

Related Information

- `defIn`

defOutCompressVia

```
defOutCompressVia {0 | 1}
```

Type: Integer

Default: 0

Compresses the via statement in SPECIALNETS section. This variable controls `defOut` to sort vias with two dimension array, so it can use `DO numX BY numY STEP stepX stepY` statement.

The via array information is only preserved in the `defIn` session, but it will not be kept by `saveDesign`/`restoreDesign`.

Related Information

- `defIn`
- `defOut`
- `restoreDesign`
- `saveDesign`

defOutLefNDR

```
defOutLefNDR {0 | 1}
```

Type: Integer

Default: `0`

Controls whether `defOut` writes out LEF non-default rule information in DEF. By default, `defOut` does not write out non-default rules defined in LEF.

Related Information

- `defOut`
- `defOutBySection`

defOutLefVia

```
defOutLefVia {0 | 1}
```

Type: Integer

Default: `0`

Controls whether `defOut` writes out LEF via information in DEF. By default, `defOut` does not write out vias defined in LEF. Set `defOutLefVia` to `1` to write out LEF vias in the DEF file.

Related Information

- `defOut`
- `defOutBySection`

defOutPolygonDieArea

```
defOutPolygonDieArea {0 | 1}
```

Type: Integer

Default: `1`

By default, `defOut` writes out polygon DIEAREA if the design is rectilinear. To write out rectangular DIEAERA along with blockages in cut-out area for a rectilinear design, set this variable to `0`.

Related Information

- `defOut`

defOverlapWireReportFileName

`defOverlapWireReportFileName` *`fileName`*

Type: String

Default: NULL

Reports overlapping wire segments in the specified file. The `defOverlapWireReportFileName` variable can be used along with the `defInShapeBasedDefFile` variable.

Related Information

- `defIn`
- `defInShapeBasedDefFile`

defStreamOutCheckUncolored

```
defStreamOutCheckUncolored {true 1 false 0 metal_only}
```

Type: enum

Default: `false`

`true` above N5 nodes and `metal_only` for N5 and below nodes

Specifies how to check and ignore uncolored shapes on DPT layers during DEF out and stream out.

- `true 1`: `defOut` and `streamOut` will go through all wires/vias/physical pins first. If the related layer has the definition for the "MASK" and "SAMEMASK" rules, tool checks the object if a color should be assigned.
- `false 0`: Ignores to check the uncolored object.
- `metal_only`: Only checks the color on the metal (including wires, physical pins, and top/bottom layer of vias).

Related Information

- `defOut`
- `streamOut`

enc_save_binary_timing_constraints

```
enc_save_binary_timing_constraints {true | false}
```

Type: Boolean

Default: `true`

If set to `true`, `saveDesign` will save timing constraints in binary format. The value of the global should be persistent in subsequent `saveDesign` and `restoreDesign` operations.

Related Information

- `restoreDesign`
- `saveDesign`

enc_save_timing_constraints_always

```
enc_save_timing_constraints_always {true | false}
```

Type: Boolean

Default: `false`

If `true`, `saveDesign` will copy SDC timing constraint files to the output directory rather than keep links to the original external SDC files. This is the equivalent to always using the `-tcon` option whenever `saveDesign` is called.

Related Commands

- `saveDesign`

fp_vertical_row

```
fp_vertical_row {0 | 1}
```

Type: Integer

Default: `0`

Specifies whether the rows in the floorplan are vertical or horizontal. By default, the rows in the floorplan are horizontal. Set this variable to `1` to enable vertical rows.

Example

- The following command sets the rows in the floorplan to vertical.
  ```
  set fp_vertical_row 1
  ```

Related Information

- `init_design`

ignore_max_via_stack_rule

```
ignore_max_via_stack_rule { true | false}
```

Type: Boolean

Default: `false`

The tech lef file contains `MAXVIASTACK` syntax by default. You can use `ignore_max_via_stack_rule` to turn off `MAXVIASTACK` syntax checking. When `ignore_max_via_stack_rule` is set to `true`, the `MAXVIASTACK` rule in the tech lef file is ignored.

init_abstract_view

`init_abstract_view` *`list_of_views`*

Type: String

Default: `abstract`

Specifies the OpenAccess view names that the tool should examine to find the equivalent LEF MACRO information (for example, PINS, OBS, FOREIGN). If this value is null (""), the tool uses `abstract` as the value. The `init_abstract_view` variable replaces the previous `rda_Input(ui_oa_abstractname)` configuration file variable.

The `init_abstract_view` is used in combination with `init_oa_ref_lib` and in the case where a Verilog netlist is read and views from the `init_abstract_view` list are used for on-demand loading for cells that were not processed from the libraries in the `init_oa_ref_lib` list. It is used by the `init_design` command.

Example

- The following command sets ABS1 as the OpenAccess view to be examined for equivalent LEF MACRO information.
  ```
  set init_abstract_view {ABS1}
  ```

Related Information

- `init_design`
- `init_oa_ref_lib`

init_check_output_pin_constant

```
init_check_output_pin_constant  {0 | 1 }
```

Type: Boolean

Default: `1`

Stops the Verilog reader if any output (or inout) signal pins have any 1'b0 or 1'b1 constants that would cause shorts to power/ground.

The default value of this boolean variable is `TRUE` or `1`.

Related Information

- `init_design`
- `setCheckMode`

init_cpf_file

`init_cpf_file` *file_path*

Type: String

Default: NULL

Specifies the path to the Common Power Format (CPF) file.

Related Information

- `init_design`

init_design_netlisttype

```
init_design_netlisttype netlist_source
```

Type: String

Default: `Verilog`

Specifies the source of the design netlist. You can specify either `Verilog` or `OA`. The `init_design_netlisttype` variable replaces the previous `rda_Input(ui_netlisttype)` configuration file variable.

Example

- The following command specifies Verilog as the source of the design netlist.
  ```
  set init_design_netlisttype {Verilog}
  ```

Related Information

- `init_design`

init_design_settop

```
init_design_settop value
```

Type: Integer

Default: `1`

Specifies whether the top cell name is automatically assigned (`0`) or is set by the user (`1`). This global variable replaces the previous `rda_Input(ui_settop)` configuration file variable.

Example

- The following command specifies that the top cell name is assigned automatically.
  ```
  set init_design_settop {0}
  ```

Related Information

- `init_design`

init_design_uniquify

`init_design_uniquify` *value*

Type: Integer

Default: `0`

Specifies whether the top is uniquified. When set to `1` (true), the design is uniquified during the read and flatten process. Optimization is not allowed on a non-unique design. For use in a master/clone partitioning flow, the value should be `0` (false). The `init_design_uniquify` global variable replaces the previous `rda_Input(ui_uniquify_netlist)` configuration file variable.

Example

- The following command specifies that the design should be uniquified automatically after the top cell is flattened.
  ```
  set init_design_uniquify {1}
  ```

Related Information

- `init_design`

init_gnd_net

`init_gnd_net` *`list_of_nets`*

Type: String

Default: NULL

Specifies the list of global ground nets used in the design. This global variable replaces the previous `rda_Input(ui_gndnet)` configuration file variable.

Example

- The following command specifies `VSS` as the ground net used in the design.
  ```
  set init_gnd_net {VSS}
  ```

Related Information

- `init_design`

init_ignore_pgpin_polarity_check

`init_ignore_pgpin_polarity_check`

Type: String

Default: NULL

Space separated list of leaf cell pin names to ignore polarity checking during `globalNetConnect` and CPF operations. Allows USE POWER pin to be connected to a USE GROUND net (or vice versa).

When specified, GNC/CPF are allowed to silently make connections to PG pins that are not of the same polarity as the PG net that they are being connected to. The list of pin names is only checked for the case where a polarity check failed and then can be ignored and connected in spite of the polarity mismatch.

Example

```
set init_ignore_pgpin_polarity_check "bias"
```

Related Information

- `init_design`

init_import_mode

`init_import_mode` *`import_mode_parameters`*

Type: `String`

Default: `NULL`

You can use this global variable to set the options of the `setImportMode` command (import mode parameters). The string with the options of the `setImportMode` command can be passed during the design initialization. This global variable replaces the previous `rda_Input(ui_import_mode)` configuration file variable.

Example

The following command passes the `-discardFloatingVNets true -keepEmptyModule true` options of the `setImportMode` command during initialization:
`set init_import_mode {-discardFloatingVNets true -keepEmptyModule true}`

Note:

- Empty modules and cells without physical cell definitions should not be converted into blackboxes and partitions.
- Empty modules should be retained.

Related Information

- `init_design`
- `setImportMode`

init_io_file

```
init_io_file file_name
```

Type: String

Default: NULL

Specifies the I/O constraint file, including path if the file is not in the current directory. This global variable replaces the previous `rda_Input(ui_io_file)` configuration file variable.

Example

- The following command specifies the `dtmf.io` file in the current directory as the I/O constraint file.

  ```
  set init_io_file {dtmf.io}
  ```

Related Information

- `init_design`

init_layout_view

```
init_layout_view list_of_views
```

Type: String

Default: NULL

Specifies the list of OpenAccess view names to be processed for cell layout viewing (GDSII equivalent). This global variable replaces the previous `rda_Input(ui_oa_layoutname)` configuration file variable.

The `init_layout_view` global is also used in the case where a Verilog netlist is read and views from the `init_layout_view` list are used in the Quick Abstract Inference flow to derive abstract information from layout views for cells that were not processed from the libraries in the `init_oa_ref_lib` list. It is used by the `init_design` command.

Example

- The following command specifies `LAY3` as the layout view to be imported.

  ```
  set init_layout_view {LAY3}
  ```

Related Information

- `init_design`
- `init_oa_ref_lib`

init_lef_check_antenna

```
init_lef_check_antenna {0 | 1}
```

Type: `Boolean`

Default: `1`

If you set `init_lef_check_antenna` to `true`, LEF or OA cells will have process antenna consistency checks done for LEF ANTENNA* data (or OA equivalent data) attached to their pins. Input pins with 0 gate-area, output pins with 0 diff-area, and inout pins with 0 gate-area or 0 diff-area values will generate warnings. If `false`, the tool will not perform consistency checking.

init_lef_check_mask_shifts

```
init_lef_check_mask_shifts {on | off | bypass}
```

Type: `Enum`

Default: `off`

Checks conflicts between the LEF FIXEDMASK and LAYERMASKSHIFT values in the technology and MACRO sections when reading in any LEF files. You can set the following values:

- `on`: When the variable is set as on, the the tool will check for the conflict. In case of a conflict, the tool will give an error and stop running.
- `off`: When the variable is set to off, the tool will not check for the conflict.
- `bypass`: When the variable is set as bypass, the tool will give and error message and will continue running.

Related Information

- `restoreDesign`

init_lef_file

```
init_lef_file list_of_files
```

Type: String

Default: NULL

Specifies the list of LEF files to be imported. This variable cannot be used with `init_oa_ref_lib`. The `init_lef_file` variable replaces the previous `rda_Input(ui_leffile)` configuration file variable.

Example

- The following command specifies `technology.lef` and `stdcell.lef` as the LEF files to be imported.
  ```
  set init_lef_file {technology.lef stdcell.lef}
  ```

Related Information

- `init_design`
- `saveDesign`

init_mmmc_file

```
init_mmmc_file file_name
```

Type: String

Default: NULL

Specifies a Tcl command script that contains the analysis view configurations for multi-mode multi-corner (MMMC) timing analysis and optimization. If this view definition file is not in the current directory, specify the complete path to the file. The `init_mmmc_file` global variable replaces the previous `rda_Input(ui_view_definition_file)` configuration file variable.

Example

- The following command specifies `mmmc.tcl` as the MMMC view definition file.

  ```
  set init_mmmc_file {mmmc.tcl}
  ```

Related Information

- `init_design`

init_mmmc_version

```
init_mmmc_version {1 2}
```

Type: Enum,

Default: `1`

Specifies the version of MMMC to use for a design. It must be set before `init_design`. If you try to change it after init_design, the tool will ignore the change and give a warning.

Related Topic

- `init_design`

init_oa_default_rule

`init_oa_default_rule` *ruleName*

Specifies a constraint group name at the start of Innovus session. The name provided would be used by Innovus for reading the place-and-route technology rules from the constraint group.

If the user specified constraint group is not found, Innovus outputs an error during `init_design`. If no value is specified for `init_default_oa_rule`, Innovus falls back to the default look-up mechanism for `LefDefaultRouteSpec` constraint group used by any place-and-route tool. Follow these guidelines while using this parameter:

- You can set this parameter only at the start of Innovus session .
- You will need to resolve conflicts in a design arising due to the two incompatible constraint groups used in two design runs in two different Innovus sessions.

The specified constraint group must have the following:

- Valid layers
- Valid vias
- Width
- Spacing

Related Information

- `init_design`

init_oa_design_cell

`init_oa_design_cell` *`cellname`*

Type: String

Default: NULL

Specifies the OpenAccess design cell name to be used. It is used by the `init_design` command.

This global variable replaces the previous `rda_Input(ui_oa_designCell)` configuration file variable.

Example

- The following command specifies `myOACell` as the OpenAccess database cell.
  ```
  set init_oa_design_cell {myOACell}
  ```

Related Information

- `init_design`

init_oa_design_lib

`init_oa_design_lib` *`libraryname`*

Type: String

Default: NULL

Specifies the OpenAccess design library name to be used. This global variable replaces the previous `rda_Input(ui_oa_designLib)` configuration file variable.

Example

- The following command specifies `myOALib` as the OpenAccess database library.
  ```
  set init_oa_design_lib {myOALib}
  ```

Related Information

- `init_design`

init_oa_design_view

`init_oa_design_view` *viewname*

Type: String

Default: NULL

Specifies the OpenAccess design view name to be used. This global variable replaces the previous `rda_Input(ui_oa_designView)` configuration file variable.

Example

- The following command specifies `myOAView` as the OpenAccess database view.
  ```
  set init_oa_design_view {myOAView}
  ```

Related Information

- `init_design`

init_oa_foundry_rule

`init_oa_foundry_rule` *`foundry_rule_name`*

Type: String

Default: NULL

Specifies the foundry constraint group to be used in OpenAccess. If your technology library contains multiple `foundry_innovus_XXXX` constraint groups, use the `init_oa_foundry_rule` variable to specify which one of them has to be read in during design initialization. This variable is used by the `init_design` command.

Example

- The following command specifies `foundry_innovus_2` as the foundry constraint group to be used during design initialization:

  ```
  set init_oa_foundry_rule {foundry_innovus_2}
  ```

Related Commands

- `init_design`

init_oa_ref_lib

```
init_oa_ref_lib list_of_OA_libs
```

Type: String

Default: NULL

Specifies the list of OpenAccess reference libraries to import. Each reference library is processed using the abstract view name list (`init_abstract_view`). The first library in the list indicates the library that will be used for the technology information unless `init_oa_design_lib` is specified. All cells from the specified libraries are read. It is important to include the standard cell library in the list to make sure that cells that are not in the current netlist are read and available for optimization. The `init_abstract_view` and `init_layout_view` lists specify which views of the cells are read. Used by the init_design command.

For example, if the reference library is `"lib1 lib2"`, and the abstract view name list is `"abstract abstract2"`, LEF MACRO information is processed for `lib1` with the `abstract` view. Then, for any cells in `lib1` that do not have `abstract` but do have `abstract2`, that view is processed for MACRO information. If a cell has both views, the first one is used. The process then is repeated for `lib2`.

The `init_oa_ref_lib` variable cannot be used with `init_lef_file`. `init_oa_ref_lib` replaces the previous `rda_Input(ui_oa_reflib)` configuration file variable.

Example

- The following command specifies `OAREFLIB1 OAREFLIB2` as the OpenAccess reference libraries.

  ```
  set init_oa_ref_lib {OAREFLIB1 OAREFLIB2}
  ```

Related Information

- `init_abstract_view`
- `init_design`
- `init_layout_view`
- `init_lef_file`
- `init_oa_design_lib`
- `saveDesign`

init_oa_search_lib

`init_oa_search_lib` *`list_of_OA_ref_libs`*

Type: String

Default: `*`

Specifies the list of OpenAccess reference libraries to be searched when cells are used in the Verilog netlist that are not included in the list of cells in the libraries specified in `init_oa_ref_lib` list. By default all libraries in the cds.lib are searched. The `init_abstract_view` and `init_layout_view` lists are used to indicate which view names are looked for during the search. Applies only when `init_verilog` is used to specify the connectivity to be read. Used by the `init_design` command.

Related Information

- `init_abstract_view`
- `init_design`
- `init_layout_view`
- `init_verilog`

init_oa_special_rule

`init_oa_special_rule` *name*

Type: String

Specifies the special rule to be used in OpenAccess to define the via search order for power routing. Typically used to specify a metal stack `LEFSpecialRouteSpec` type constraint group to read from the technology library if the library contains multiple metal stacks. If nothing is specified, then the constraint group with the name `LEFSpecialRouteSpec` is searched in OpenAccess. It is used by the `init_design` command.

Related Information

- `init_design`

init_original_verilog_files

`init_original_verilog_files` *name*

Type: String

Specifies a list of Verilog files to be read during initialization. `init_design` will set it to the list of input files given to it. The original input file names are retained through the flow for reference, and used by `write_do_lec` for the default golden netlist comparison file names.

Related Information

- `init_design`
- `write_do_lec`

init_pwr_net

`init_pwr_net` *`list_of_power_nets`*

Type: String

Default: NULL

Specifies the list of global power nets used in the design. This global variable replaces the previous `rda_Input(ui_pwrnet)` configuration file variable.

Example

- The following command specifies `VDD` as the power net in the design.
  ```
  set init_pwr_net {VDD}
  ```

Related Information

- `init_design`

init_top_cell

```
init_top_cell top_cell_name
```

Type: String

Default: NULL

Specifies the name of the top module. This variable is used only when reading Verilog netlist information. It replaces the previous `rda_Input(ui_topcell)` configuration file variable.

Example

- The following command specifies `TOP` as the top cell in the design.

  ```
  set init_top_cell {TOP}
  ```

Related Information

- `init_design`

init_verilog

`init_verilog` *`list_of_netlist_files`*

Type: String

Default: NULL

Specifies the gate-level Verilog netlist files to be read. This variable replaces the previous `rda_Input(ui_netlist)` configuration file variable.

An encrypted Verilog file is allowed to be specified for `init_verilog`. The file can be created by command `encrypt -verilog`. To be loaded, the encrypted Verilog must have suffix ".vp", like mydesign.vp.

Example

- The following command specifies `z650n3_n2n.enc.dat/z650n3.v` as the gate-level Verilog netlist file.

```
set init_verilog {z650n3_n2n.enc.dat/z650n3.v}
```

Related Information

- `encrypt`
- `init_design`

init_verilog_tolerate_port_mismatch

`init_verilog_tolerate_port_mismatch` *value*

Type: Integer

Default: `0`

Specifies whether new instance ports can be created during Verilog netlist parsing.

By default, this variable is set to 0. If the ports of the cell and its instantiated instance do not match, the Verilog parser stops processing and the tool errors out. If instead, you would like the tool to continue, set the `init_verilog_tolerate_port_mismatch` variable to `1`. When the variable value is set to `1`, the Verilog parser ignores the mismatch and adds the new port to the cell. This is intended to be used during prototyping when it may be alright to lose some connections in 'dirty netlists'. Warnings about mismatches will still be generated.

Set this variable before the Verliog file is read in.

Related Information

- `init_design`

inn_save_lef_ignore_abstracts

```
inn_save_lef_ignore_abstracts {True | False}
```

Type: Boolean

Default: False

Specifies whether to remember a list of unused LEF block macros (CLASS BLOCK or RING) so they are not read back with `restoreDesign`. If you have a large number of LEF blocks (for example, memories) that are not used in this design, this can speedup `restoreDesign` and reduce memory by skipping those LEF cells. Small blocks with less then 10 pins are still read in, because they are sometimes added later in the flow.

Related Information

- `restoreDesign`

lefdefInputCheckColoredShape

```
lefdefInputCheckColoredShape { true | false}
```

Type: Boolean

Default: `false`

Specifies to check and ignore colored shapes on colorless layers while reading the LEF or DEF data. When set to `true`, the tool errors out if a colored shape is on a colorless layer. When set to `false`, the tool gives an error message and ignores the color.

lefDefOutVersion

`lefDefOutVersion` *`version_num`*

Type: String

Default: `5.8`

Specifies LEF/DEF output version. Possible string values are `5.5`, `5.6`, 5.7, 5.8, and 6.0. The default is `5.8`.

Related Commands

- `defOut`
- `defOutBySection`
- `write_lef_abstract`

lefExtend2DCellShapes

```
lefExtend2DCellShapes { {layer extension_amount}… }
```

Type: String

Default: `""`

Specifies a list of layer and a value for creating obs shape for 2D pin/obs shapes. Cells could have 2D objects, which require larger line-end spacing. You can use `lefExtend2DCellShapes` to automatically add **SPACING 0 OBS** for routing during LEF in..

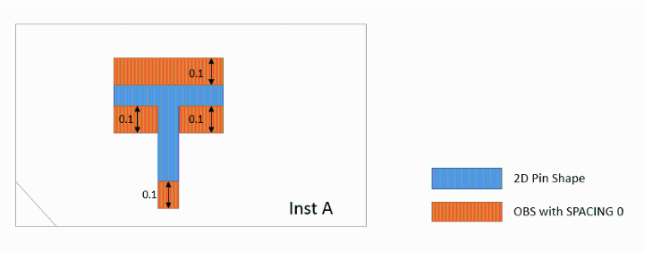
For any 2D shapes, you can extend edges that are perpendicular to the preferred routing direction of the rectilinear shape by specifying the `extension_amount` in `lefExtend2DCellShapes`.

Note: Cells would still require explicit edge constraint to prevent improper cell abutment.

Example

```
set lefExtend2DCellShapes {M1 0.1}
```

The direction of M1 is Vertical.



lefExtendCellObsShapeUnderTrim

```
lefExtendCellObsShapeUnderTrim listOfRoutingLayers
```

Type: String

Default: ""

Specifies a list of routing layers. On each specified *layer* the LEF OBS shapes that are overlapped partially by a shape on the TRIMMETAL layer (that trims the routing layer), will be extended in the preferred routing direction to be fully covered by the trim shape. You can use this global for some advanced node libraries to avoid trim/OBS short violations.

lefIgnoreLayerNames

`lefIgnoreLayerNames` *`layerNames`*

Type: String

Default: `NULL`

Specifies the layer list that needs to be ignored. If you do not specify any layers in this global, the tool load all layers. For example, if you specify `lefIgnoreLayerNames "TSV BSPM0"`, the tool will not load the definition for the TSV layer and BSPM0 layer when loading the tech LEF file. You should use this global to disable the layer list when loading an OA design.

If the disabled layers are used in the Tech LEF for via cell, via rule generation, NDR and property definition, the tool will ignore the definition when loading the Tech LEF file. If the disables layers are used in macro LEF, tool will stop reading the LEF file and give an error. If the disabled layers are used in the definition of the following objects in other design data, the tool will give error to skip this object:

- Wire/Via
- Marco Pin/OBS (Include bump)
- Blockage
- Bus guide
- Track
- Pin guide
- GUI shape/text
- Pin guide

merge_multi_port_to_single_port

```
merge_multi_port_to_single_port {true | false}
```

Type: `Boolean`

Default: `false`

When set to `true`, multiple ports in a LEF PIN are merged into a single port. However, the pin should not have the `MUSTJOINALLPORTS` attribute as it specifies that all of the ports must be connected individually.

readWriteLefCheckUncoloredShapes

`readWriteLefCheckUncoloredShapes {pin_only | on | off}`

Type: `enum`

Default: off

Specifies whether to check for uncolored LEF PIN and OBS shapes on multi-mask routing layers when reading or writing LEF files. If the check is on, LEF read or write commands will error out if there are any uncolored routing layer shapes (cut layers are not checked). If set to `pin_only`, only check LEF PIN shapes and ignore LEF OBS shapes.

- During `write_lef_abstract`, if the tool finds any uncolored physical pin and if `readWriteLefCheckUncoloredShapes` is set to `on`, the tool will print an error message, and will not write out uncolored shapes to the LEF file. However, if `readWriteLefCheckUncoloredShapes` is set to `off`, the tool will not check the uncolored object and dump all physical pins into the macro lef. In this case, the tool does not print any error message.
- When the tool reads the LEF file and finds any uncolored macro pin, and if `readWriteLefCheckUncoloredShapes` is set to `on`, the tool will print an error message and only read colored shapes into the design. However, if `readWriteLefCheckUncoloredShapes` is set to `off`, the tool will not check the uncolored object and will read the definition of all macros into the design. In this case, the tool does not print any error message.

The `readWriteLefCheckUncoloredShapes` global controls the behavior of following commands or flow:

- `write_lef_abstract`
- `restoreDesign` and init design flow
- `replaceLefMacro`

`readWriteLefCheckUncoloredShapes` does not impact `write_lef_library` and read/write oa tech.

Related Information

- `replaceLefMacro`
- `restoreDesign`
- `write_lef_abstract`
- `write_lef_library`

save_cmd_file_limit

`save_cmd_file_limit` *`limit`*

Type: `integer`

Default: `10`

The `saveDesign` command saves the complete .cmd history across multiple sessions. The `save_cmd_file_limit` variable specifies the file length limit (in units of Mb) that is saved in the inn.cmd.gz file inside the save directory. If it is set to 0, the command history is not saved.

Related Information

- `saveDesign`

write_lec_dft_constraints

`write_lec_dft_constraints` *`dictionary`*

Type: `String`

Default: `""`

This variable is a TCL dictionary that holds the LEC DFT pin constraints from the last LEC run by Genus. It is passed forward through the `write_design` command. The `write_do_le`c command will automatically include this constraint in the do file.

Related Information

- `write_do_lec`

write_lec_directory_naming_style

`write_lec_directory_naming_style` *`directory_name`*

Type: `String`

Default: `fv/invs/%s`

Specifies the directory name where the `write_do_lec` command will write verification files when the write_lec_files global is true. The directory will be created if it does not already exist, and will be overwritten if the existing directory has the same name. A `%s` in the string is replaced with the design name. For example, it will overwrite the directory for the same design but not for a different design. A `%d` in the string is replaced with a unique integer to avoid overwriting any existing directory.

Related Information

- `write_do_lec`
- `write_lec_files`

write_lec_files

`write_lec_files {true | false}`

Type: `Boolean`

Default: `true`

Specifies whether the `write_do_lec` command should write intermediate files to the verification directory specified by the `write_lec_directory_naming_style` global. If false, no files are written.

Related Information

- `write_do_lec`
- `write_lec_directory_naming_style`

writeLefAbstractCustomerHeader

`writeLefAbstractCustomerHeader` *`header_information`*

Type: `String`

Default: `""`

Writes LEF abstract files with the customer-specified header information.

15

Integrated Hierarchical Database Commands

- check_module_model
- create_module_model
- define_add_on_module_model
- import_module_model
- pack_module_model_libs
- report_module_model
- restore_module_model
- set_module_model
- update_module_model
- write_module_model_context

check_module_model

```
check_module_model
[-help]
-cell cell_name
[-out_file file_name]
-ref_tag tag_name
[-slack_threshold threshold] [-type {flexIlm ilm pnr}]
```

Checks and validates an ILM model to see if the current reduction ratio will cause any missing constraints at the hinst terminal or any slack difference. The `check_module_model` command compares/checks the target timing result with the reference timing result and generates a summary and detailed report.

Note: If a specified cell has more than one instantiated modules, all hinst terminals are checked and reported.

Usage Model:

- Generate a reference ILM model with the `createInterfaceLogic -keepAll` option and a target ILM model with user-specified settings.
- Restore the top-level design and set/commit target ILM model(s).
- Time the design.
- Call `check_module_model`

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `check_module_model` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man check_module_model`
`-cell cell_name`	Specifies the ILM cell name that will be validated. *Data_type*: string, required
`-out_file file_name`	Specifies the output report file. *Default:* `./check_module_model.rpt` *Data_type*: string, optional
`-ref_tag tag_name`	Specifies the reference tag name of the reference ILM model. *Data_type*: string, required
`-slack_threshold threshold`	Specified the slack threshold for reporting the number of mismatched slacks over the threshold. *Default:* `0.0005` *Data_type*: string, optional
`-type {flexIlm ilm pnr}`	Specifies the type of module model. The model type can be flexllm, ilm, or pnr. *Default*: `ilm` *Data_type*: string, optional

Example

- The following command check the tdsp_core cell and writes the detailed report in the tdsp_check_module_model.rpt file:
  ```
  check_module_model -cell tdsp_core -ref_tag ref_preCts -type ilm  -out_file tdsp_check_module_model.rpt
  ```

Related Information

- commit_module_model
- create_module_model
- report_module_model
- restore_module_model
- savePartition
- set_module_model
- update_module_model
- write_module_model_context
- Hierarchical and Prototyping Flow chapter in the *Innovus User Guide*
  - Integrated Hierarchical Database Flow

create_module_model

```
create_module_model
[-help]
[-allow_port_mismatch]
[-cell string]
[-phys_hier]
[-tag tag_name]
[-type {1801 boundary_model etm extraction_context flexIlm ilm latency lef mmmc pin_assign pnr pnr_shell scan_info spef
timing_context gds}]
```

Creates a module model. It honors the `set_module_model -default_create_options` command settings.

Parameters

`-allow_port_mismatch`	Creates a new updated LEF. It provides an ability to bring in an updated model with mis-matched ports to top-level design. **Note:** This parameter can be specified with the `lef` type (`-type`) of module models. *Data_type*: bool, optional
`-cell string`	Specifies the cell to save. By default, current top cell will be saved. **Note:** For 2D designs, the `-cell` parameter should be used for pnr type of module model only. **Note**: For 3D IC designs, the `-cell` parameter can be used to specify a list of partition names or design name to save for pnr, lef, or pre_pnr types of module models. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `create_module_model` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man create_module_model`
`-phys_hier`	Creates the whole physical hierarchy. *Data_type*: bool, optional
`-tag tag_name`	Specifies tag name of the saved model. **Note:** You cannot use 'latest' as a tag name as it is a reserved keyword for the latest tag. • If a model is created from an Innovus session that is opened by the `update_module_model` command, then the same tag name of the current loaded design will be used if this option is not specified. • This option is required if a block-level design has been opened explicitly. You only need to specify this tag option for one model type. For subsequently created model types, the same tag can be used if it has not been specified. • If the tag information is not available, the `create_module_model` command displays an error message. *Data_type*: string, optional
`-type {1801 boundary_model etm extraction_context flexIlm ilm latency lef mmmc pin_assign pnr pnr_shell scan_info spef timing_context gds}`	

	Specifies the type(s) of module model. The following is the list of the supported model types: • `1801`: UPF for power domain • `boundary_model`: Boundary model for timing analysis • `etm`: Extraction timing model • `extraction_context`: Extraction context model • `flexilm`: Flexible interface logic model • `gds`: Graphical design system model • `ilm`: Interface logic model • `latency`: Clock latency information • `lef`: LEF abstract • `mmmc`: Multi-mode multi-corner definition • `pin_assign`: SPEF file from RC extraction • `pnr`: Innovus DB • `pnr_shell`: Reduced pnr model where only the first level of logics is kept. • `scan_info`: Scan chain information • `spef`: SPEF file from RC extraction • `timing_context`: Timing context model **Note:** Use the `-type 1801` to save UPF associated with an ILM model so that you do not have to keep a track where is the corresponding UPF of an ilm. **Note:** You can specify a list of model types to create multiple types of module models at a time. **Note:** The `pnr_shell` model type is similar to the `flexilm` model but it only has first level of logics. **Note:** The `create_module_model -type spef` honors the `set_module_model -active_rc_corners` command to output SPEF(s) for the specified RC corners. By default, multi corner SPEFs are saved as gzip. The output SPEF(s) are saved at <iHDB_root>/<cell>/<tag>/spef/<topCell>.<cornerName>.spef.gz **Note:** If the `-rc_corner` parameter of the `rcOut` command is not specified, the name of the SPEF output file is <iHDB_root>/<cell>/<tag>/spef/<topCell>.spef.gz **Note:** You can use the `timing_context` or `extraction_context` add on types for creating a timing or extraction context model, respectively, for a specific cell or a master/clone instance. This context model can be loaded additionally with the design later in the flow. Default: `pnr` and `lef` *Data_type*: enum, optional

Example

- The following command creates ilm type of module model that has the postRoute tag.
  ```
  create_module_model -type ilm -tag postRoute
  ```

Related Information

- `check_module_model`
- `commit_module_model`
- `report_module_model`
- `restore_module_model`
- `savePartition`
- `set_module_model`
- `update_module_model`
- `write_module_model_context`
- Hierarchical and Prototyping Flow chapter in the *Innovus User Guide*
  - Integrated Hierarchical Database Flow

define_add_on_module_model

```
define_add_on_module_model
[-help]
[-commit_callback string]
-type string
{[-create_callback string]
[-set_callback string]
[-restore_callback string]
[-write_context_callback string]}
```

Allows you to define the add_on types or supplemental models that can be loaded additionally with an existing model or a design. These user-defined add_on types can be used with the `create_module_model`, `set_module_model`, `restore_module_model`, and `write_module_model_context` commands. You must provide a TCL file containing callback procedures for each applicable Integrated Hierarchical Database (iHDB) flow command that will be run when this command is called.

In the TCL file, the TCL callback procedures must be defined in the following format and must include the specified arguments:

```
proc <callBackTCLProcName> {dir cell tag args} {
…
}
```

Where,

`dir`, `cell`, and `tag` represent values that will be automatically derived by the iHDB capability.
`args` can be set using `set_module_model -default_commit_options {options} -type add_ons_type`

Parameters

`-commit_callback string`	Specifies the user-defined TCL procedure that is executed when the `commit_module_model` command is called with this `add_ons` type. *Data_type*: string, optional
`-create_callback string`	Specifies the user-defined TCL procedure that is executed when the `create_module_model` command is called with this `add_ons` type. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `define_add_on_module_model` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man define_add_on_module_model`
`-restore_callback string`	Specifies the user-defined TCL procedure that is executed when the `restore_module_model` command is called with this `add_ons` type. *Data_type*: string, optional
`-set_callback string`	Specifies the user-defined TCL procedure that is executed when the `set_module_model` command is called with this `add_ons` type. *Data_type*: string, optional
`-type string`	Specifies the newly added add_ons model type name. *Data_type*: string, required
`-write_context_callback string`	Specifies the user-defined TCL procedure that is executed when the `write_module_model_context` command is called with this `add_ons` type. *Data_type*: string, optional

Example

- Following is an example of defining the "attrs" add_ons model type. First this 'attrs' attribute must be defined with the define_add_on_module_model command with its associated callback TCL procedures:

```
source dump_attrs.tcl
set_module_model -default_commit_options {-fe -keepPinGeometry} -type pnr
define_add_on_module_model -type attrs -create_callback ihdb_create_attrs -set_callback ihdb_read_attrs
```

  Where `dump_attrs.tcl` contains:

```
proc ihdb_create_attrs {dir cell tag args} {   ;#This format is fixed

  write_block_attrs $dir/$cell/$tag/attrs/db.attrs

}

proc ihdb_read_attrs {dir cell tag args} {  ;# This format is fixed

  read_block_attrs $dir/$cell/$tag/attrs/db.attrs

}

…
```

- *Block Level*. The following command creates user defined attrs type of module model that has the *merge_attribute* tag.

```
create_module_model -type attrs -tag merge_attribute

create_module_model -tag merge_attribute
```

- *Top-Level*. The following commands configures and commits the *merge_attribute* module model using the `attrs` add-on associated with the `pnr` model type:

```
set_module_model -cell dcm -tag merge_attribute -type pnr -add_ons {attrs}
set_module_model -cell lsu -tag merge_attribute -type pnr -add_ons {attrs}
commit_module_model
```

Related Information

- `check_module_model`
- `commit_module_model`
- `create_module_model`
- `report_module_model`
- `restore_module_model`
- `savePartition`
- `update_module_model`
- `write_module_model_context`
- Hierarchical and Prototyping Flow chapter in the *Innovus User Guide*
  - Integrated Hierarchical Database Flow

import_module_model

```
import_module_model
[-help]
-cell string
[-idx_pair string]
-inputs string
-libs [-link_only]
[-rc_corner string]
[-tag string]
-type {lib lef qrc_tech qrc_conf}
```

Imports library files into the iHDB structure.

Parameters

`-cell` *string*	Specifies the cell to import. *Data_type*: string, required
`-help`	Outputs a brief description that includes the type and default information for each `import_module_model` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man import_module_model`
`-idx_pair` *string*	Specifies the idx pair list if the files are copied from the idx database. *Data_type*: string, optional
`-inputs` *string*	Specifies the full path of the input file to be imported. **Note:** The `import_module_model` command supports the import of Quantus™ Extraction Solution input files. *Data_type*: string, required
`-libs`	Specifies whether input file is a library file or not. **Note:** This option should be used for type lef, lib, qrc_tech, or qrc_conf. **Note:** The library files: • *.lib are saved at <iHDB>/<cell>/.libs/mmmc/ • *.lef are saved at <iHDB>/<cell>/.libs/lef/ • qrcTechFile are saved at <iHDB>/<cell>/.libs/mmmc/<rc_corner>/qrcTechFile • qrc.conf are saved at <iHDB>/<cell>/.libs/qx/qrc.conf **Note:** The following GDS related model types can be saved with `-libs` option: • `gds_map` • `stream_libs` • `stream_layermap` *Data_type*: bool, required
`-link_only`	Creates a link or actual hard copy. **Note:** If the `-link_only` parameter is specified, then a link pointing to the current library location is created. If is the `-link_only` parameter is not specified then the actual physical file will be there. *Data_type*: bool, optional
`-rc_corner` *string*	Specifies the list of RC corners. *Data_type*: string, optional
`-tag` *string*	Specifies a tag of a saved file. **Note:** The `-tag` and `-libs` parameters are mutually exclusive. *Data_type*: string, optional
`-type {lib lef qrc_tech qrc_conf}`	
	Specifies the type(s) of module model. The model type can be lib, lef, qrc_tech, or qrc_conf. Default: *Data_type*: string, required

Example

- The following command imports a LEF file:
  ```
  import_module_model -cell tdsp_core -libs -type lef -input_file
  /icd/hierflow2/DATA2/tdsp_core/init/pnr/tdsp_core.enc.dat/libs/lef/*.lef
  ```
- The following command imports streamOut layer map into cell top_die:
  ```
  import_module_model -cell top_die -inputs ./input_data/lvs_data/streamOut.layermap -libs -type steam_layermap
  ```

  This will be saved into <iHDB_root>/top_die/.libs/steam_layermap/steamOut.layermap

Related Information

- `check_module_model`
- `commit_module_model`
- `report_module_model`
- `restore_module_model`
- `savePartition`
- `set_module_model`
- `update_module_model`
- `write_module_model_context`
- Hierarchical and Prototyping Flow chapter in the *Innovus User Guide*
  - Integrated Hierarchical Database Flow

pack_module_model_libs

```
pack_module_model_libs
[-help]
[-cell cellName]
[-validate]
```

Packs libraries for the Integrated Hierarchical Database (iHDB) flow designs.

Notes:

- You must specify the default central directory name (`set_module_model -default_dir`), before using this command. In case default central directory is moved to a different location, the libraries can be packed again.
- The packed libraries are saved at *<default_model_data_directory>/<cell>/.libs.*

Parameters

`-cell` *cellName*	Specifies the cell name. All pnr model libraries of all tags under the specified cell will be linked to the packed libraries (*.libs)* saved by the `pack_module_model_libs` command. **Note:** The packed libraries are saved at *<default_model_data_directory>/<cell>/.libs.* *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `pack_module_model_libs` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man pack_module_model_libs`
`-validate`	Compares the checksum before and after packing libraries. It checks whether the libraries before and after packing are the same. **Note:** When the `-validate` parameter is specified, the `pack_module_model_libs` command will output *after_pack_libs.check_sum* and *before_pack_libs.check_sum* under the *.libs* directory for comparison. *Data_type*: bool, optional

Example

- The following command specifies the default central directory name (./DATA) where models data will be saved/loaded and then specifies the cell name. All pnr model libraries of all tags under the specified cell will be linked to the packed libraries (*.libs)* saved by the `pack_module_model_libs` command.

  ```
  set_module_model -default_dir DATA2
  pack_module_model_libs -cell tdsp_core -validate
  ```

  `Output:`
  This command will output the packed libraries at *`DATA2/tdsp_core/.libs`*. It will also create *`after_pack_libs.check_sum`* and *`before_pack_libs.check_sum`* under the *`.libs`* directory for comparison.

Related Information

- `set_module_model`
- Hierarchical and Prototyping Flow chapter in the *Innovus User Guide*
  - Integrated Hierarchical Database Flow

report_module_model

```
report_module_model
[-help]
[-active_rc_corners]
[-default_dir]
[-non_committed_hinst]
[-type {flexilm lef ilm pnr pnr_shell}]
```

Reports the module models that are currently loaded/committed at top-level design.

Parameters

`-active_rc_corners`	Reports the active RC corners. *Data_type*: bool, optional
`-default_dir`	Reports the current default module data directory. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `report_module_model` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_module_model`
`-non_committed_hinst`	Checks and reports the modules/cells that are available in the default module data directory but are not committed as hierarchical modules (hInst). *Data_type*: bool, optional
`-type {flexilm lef ilm pnr pnr_shell}`	Specifies the type of module model to be reported **Note:** The module models that are not committed are reported with the "init" type and tag. The module models that are committed are reported with the types and tags same as the requested types. *Data_type*: string, optional

Examples

- The following command reports the module models that are currently loaded/committed at top-level design:

```
report_module_model -type lef

Module       Type ReqType Tag     ReqTag    Dir
-------------------------------------------------------------------------------------
ptn_wrapper  lef  lef     preCts  preCts    /icd/demo/v4h/DATA2/ptn_wrapper/latest/lef


default directory : /icd/demo/v4h/DATA2
```

- The following command reports the ilm type of module models:

```
report_module_model -type ilm

Module       Type ReqType Tag         ReqTag      Dir
-----------------------------------------------------------------------------------------------
ptn_wrapper  ilm  ilm     my_postCts  my_postCts  /icd/demo/v4h/DATA2/ptn_wrapper/my_postCts/ilm
tdsp_core    ilm  ilm     my_preCts   my_preCts   /icd/demo/v4h/DATA2/tdsp_core/my_preCts/ilm

default directory : /icd/demo/v4h/DATA2
```

- The following command reports the list of active RC corners:

```
set_module_model -active_rc_corners {rcMin rcMax}
report_module_model -active_rc_corners
```

Related Information

- `check_module_model`
- `commit_module_model`
- `create_module_model`
- `restore_module_model`
- `savePartition`
- `set_module_model`
- `update_module_model`
- `write_module_model_context`
- Hierarchical and Prototyping Flow chapter in the *Innovus User Guide*
  - Integrated Hierarchical Database Flow

restore_module_model

```
restore_module_model
[-help]
[-dynamic_view string]
[-hold_views string]
[-leakage_view string]
[-lef_files string]
[-mmmcFile string]
[-noTiming]
[-noTimingGraph]
[-setup_views string]
[-tag tag_name]
{cell_name | -cell cell_name}
[options]
[-add_ons {1801 mmmc spef eco latency scan_info timing_context extraction_context pin_assign}]
```

Restores a design from a default module model repository. You can use this command only after the default central directory name where the module model data will be saved/loaded has been specified using the `set_module_model -default_dir` command.

The `restore_module_model` command provides you the following flexibilities:

- Ability to provide your own ECO script. The ECO script must be located at <cell>/<tag>/eco/<cell>_eco.tcl.
- Ability to read/restore a design of a specific tag but its ECO file can be loaded from a different tag.

This feature is useful when the ECO changes are done with an old netlist drop and a new netlist drop is saved with a different tag.

For example:

```
restore_module_model -cell tdsp_core -tag my_postcts -add_ons {anotherTag/eco}
```

Supported short-hand `restore_module_model` usage model:

```
restore_module_model <dir>/<cell>/<tag> [-add_ons {add_on_type}]
restore_module_model <cell>/<tag> [-add_ons {add_on_type}] ;# Local directory
restore_module_model <cell> -tag <tag> [-add_ons {add_on_type}] ;# Default root dir
```

Parameters

```
-add_ons {1801 mmmc spef eco latency scan_info timing_context extraction_context
pin_assign}
```

	Specifies the supplement model that can be loaded additionally with an existing model or a design. For example: • The following command restores the ptn_wrapper design that has my_extract_v1 tag and also bring in its extraction_context add_on model: `restore_module_model ptn_wrapper -tag my_extract_v1 -add_ons {extraction_context}` • The following commands show the location from where mmmc data will be loaded during `restore_module_model` according to usage of `-add_ons {mmmc}` ◦ `restore_module_model Ptn1 -tag XYZ` Uses view definition located inside .enc.dat in pnr directory of tag XYZ ◦ `restore_module_model Ptn1 -tag XYZ -add_ons {ABCD/mmmc}` Uses view definition from mmmc directory of tag ABCD ◦ `restore_module_model Ptn1 -tag XYZ -add_ons {mmmc}` Uses view definition from mmmc directory of tag XYZ **Note:** For more information about saving data in separate tags in the mmmc directory, see `saveTimingBudget -module_model_tag` *Data_type*: string, optional
cell_name | -cell *cell_name*	Specifies the module model name. **Note:** This parameter must be specified. *Data_type*: string, required
`-dynamic_view` *string*	Specifies the dynamic view. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `restore_module_model` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man restore_module_model`
`-hold_views` *string*	Specifies the hold views. **Note:** The `-setup_views` and `-hold_views` parameters specify the active analysis views for the loaded design. *Data_type*: string, optional
`-leakage_view` *string*	Specifies the leakage view. *Data_type*: string, optional
`-lef_files` *string*	Specifies the lef file(s). *Data_type*: string, optional
`-mmmcFile` *string*	Specifies the MMMC view file(s) *Data_type*: string, optional
`-noTiming`	Ignores timing libraries and MMMC configuration *Data_type*: bool, optional
`-noTimingGraph`	Specifies not to load timing graph. *Data_type*: bool, optional

options	Specifies the restoreDesign options. *Data_type*: string, optional
-setup_views *string*	Enables you to setup views. **Note:** The -setup_views and -hold_views parameters specify the active analysis views for the loaded design. *Data_type*: string, optional
-tag *tag_name*	Specifies the tag name of the flow. **Note:** If *tag_name* is not specified, 'latest' is used as the default tag value. *Data_type*: string, optional

Example

- The following command restores the dtmf_recvr_core module model that has the start tag.
  ```
  restore_module_model -cell dtmf_recvr_core -tag start
  ```
- The following commands restore the ptn_wrapper design along with its DEF extraction context to do hierarchical extraction:
  ```
  restore_module_model ptn_wrapper -tag my_extract_v1 -add_ons {extraction_context}
  #Load in the detailed LEF abstract of ram_128x16_test
  set_module_model -cell ram_128x16_test -type lef -tag my_extract_v1
  commit_module_model
  earlyGlobalRoute
  setExtractRCMode -engine postRoute
  extractRC
  rcOut -spef with_detailed_def.spef
  ```
-

Related Information

- check_module_model
- commit_module_model
- create_module_model
- report_module_model
- savePartition
- set_module_model
- update_module_model
- write_module_model_context
- Hierarchical and Prototyping Flow chapter in the *Innovus User Guide*
  - Integrated Hierarchical Database Flow

set_module_model

```
set_module_model
[-help]
[-allow_port_mismatch]
[-dir model_data_directory]
[-hold_views string]
[-phys_hier]
[-setup_views string]
[-tag flow_tag]
[-type {flexilm lef ilm pnr pnr_shell boundary_model }]
[-update]
{[[[[-cell moduleName] [-default_set_options string]] | [-top cell_name [-logical_only ]]]
[-add_ons {1801 etm latency pnr_view scan_info spef timing_context extraction_context}]] |
-default_dir default_model_data_directory |
-default_create_options default_option |
-default_commit_options string |
-active_rc_corners list_of_rc_corners }
```

Sets or configures specific block model(s) to top-level design.

Parameters

`-active_rc_corners list_of_rc_corners`	Specifies the list of RC corners. **Note:** For multi-corners, different applications have different ways to get or specify RC corners for outputting/reading SPEF (Standard Parasitic Exchange Format). In the Integrated Hierarchical Database (iHDB) flow, the `-active_rc_corners` parameter provides a uniform way to use RC corners. **Note:** The `create_module_model -type spef` honors the `set_module_model -active_rc_corners` command to output SPEF(s) for the specified RC corners. By default, multi corner SPEFs are saved as gzip. The output SPEF(s) are saved at <iHDB_root>/<cell>/<tag>/spef/<topCell>.<cornerName>.spef.gz *Data_type*: string, optional
`-add_ons {1801 etm latency pnr_view scan_info spef timing_context extraction_context}`	

	Specifies the supplement model that can be loaded additionally with an existing model or a design. The following is the list of the supported model types: • `1801`: UPF for power domain • `etm`: Extraction timing model • `latency`: Clock latency information • `pnr_view`: Full pnr physical DB • `scan_info`: Scan chain information • `spef`: SPEF file from RC extraction • `timing_context`: Timing context model • `extraction_context`: Extraction context model For example the following command specifies that the etm add on type is associated with the `-type lef` parameter. It uses LEF and timing model ETM (.lib) together at top-level design: `set_module_model -cell tdsp_core -type lef -add_ons {etm}` **Note:** The `1801` add on type can only be specified for `ilm` model type (`-type ilm`). **Note:** The `pnr_view` add on type can only be specified for `ilm` model type (`-type ilm`) where it will load in the full pnr physical DB on top of an ILM model so you can see all cells and routing. This can be used for debugging timing. **Note:** The `spef` add on type honors the `-active_rc_corners` parameter to read in SPEFs (Standard Parasitic Exchange Format) for the specified RC corners. **Note:** The `spef` add on type cannot be used with the `pnr_shell` model type. **Note:** With the `latency` and `scan_info` add on types, the `create_module_model` command can save the latency and scan chain data into iHDB. The `commit_module_model` command can bring back the latency data of an ILM data to the top-level design for pre-CTS stage, and be able to merge block scan chains back to top-level design. *Data_type*: bool, optional
`-allow_port_mismatch`	Allows model I/O ports mismatching from the original database. It replaces the new updated LEF and provides an ability to bring in an updated model with mis-matched ports to top-level design. **Note:** This parameter can be specified with the lef, flexllm, ilm, and pnr type (`-type`) of module models only. *Data_type*: string, optional
`-cell` *moduleName*	Specifies the cell name(s) to be loaded. You can specify a list of cell names. **Note:** The `-cell` parameter accepts wildcards as input. *Data_type*: string, optional

`-default_commit_options` *string*	Specifies the default commit options. These default settings are honored by the `commit_module_model` command.
	Use Model:
	`set_module_model -default_commit_options {`*listOfAssembleDesignOptions*`} -type pnr`
	`set_module_model -default_commit_options {`*listOfSetIlmTypeOptions*`} -type ilm`
	Example:
	`set_module_model -default_commit_options {-fe -keepPinGeometry} -type pnr`
	`set_module_model -default_commit_options {-model si} -type ilm`
	Data_type: string, optional

`-default_create_options` *default_options*	Specifies the default create options for a specific model type. These default settings are honored by the `create_module_model` command. **Note:** You must specify the `-type` parameter with this option. **Note:** While specifying the default create options, ensure that you specify the correct names of the options as per the selected Innovus interface. *Data_type*: string, optional
`-default_dir` *default_model_data_directory*	Specifies the default central directory name where the module models data will be saved/loaded. **Note:** If `set_module_model -default_dir` is not specified, the Innovus system does not set any directory by default. Later on, when you try to run the `restore_module_model` command, it displays an error message that the `-default_dir` parameter is not set. This option MUST be set before loading any Integrated Hierarchical Database (iHDB) design. *Data_type*: string, optional
`-default_set_options` *string*	Sets the default options. *Data_type*: string, optional
`-dir` *model_data_directory*	Specifies the directory where data of the specified cell can be found. **Note:** The `-dir` parameter will override the `-default_dir` setting for this cell. **Note:** The `-dir` parameter cannot be specified if the `-cell` parameter has been specified using wildcards. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `set_module_model` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_module_model`
`-hold_views` *string*	Specifies the hold views. **Note:** The `-setup_views` and `-hold_views` parameters allow you to specify the active analysis views for the loaded design. Innovus uses these settings to know which SDC constraints to use. By default, the active views will be based on the viewDefinition.file of pnr model (*.enc.dat). *Data_type*: string, optional
`-logical_only`	Specifies that only the logical data will be read. It controls whether to load DEF and LEF libraries or not. If `-logical_only` is specified, Innovus only loads Verilog/.lib but not DEF/LEF. *Data_type*: bool, optional
`-phys_hier`	Enables you to load the physical hierarchy for viewing purpose. *Data_type*: bool, optional
`-setup_views` *string*	Enables you to setup views. **Note:** The `-setup_views` and `-hold_views` parameters allow you to specify the active analysis views for the loaded design. Innovus uses these settings to know which SDC constraints to use. By default, the active views will be based on the viewDefinition.file of pnr model (*.enc.dat). *Data_type*: string, optional
`-tag` *flow_tag*	Specifies the tag name of the specified cell. It can be design stage/status name such as ‘prects’, ‘cts’, etc…. **Note:** The 'latest' tag will be used if this option is not specified. ‘latest’ can also be specified as tag name for referencing to the latest updated tag. **Note:** For a specific cell, you only need to specify this tag option for one model type. For subsequent model type setting of the same cell, same tag name will be used if it’s not specified. *Data_type*: string, optional

`-top cell_name`	Specifies the top module model. This is a light way to load the top-level Verilog netlist. When specified with `-type pnr`, the tool only loads the Verilog and DEF from the `pnr` model. **Note:** When the `-top` parameter is specified, the default model type (`-type`) is pnr.
`-type {flexilm ilm lef pnr pnr_shell boundary_model}`	Specifies the model type. The model type can be boundary_model, lef, flexllm, ilm, pnr, or pnr_shell. The following is the list of the most frequently used iHDB module model types: • `flexilm`: Flexible interface logic model • `ilm`: Interface logic model • `lef`: LEF abstract • `pnr`: Innovus DB • `pnrshell`: Reduced pnr model where only the first level of logics is kept. • `boundary_model`: Boundary model for timing analysis **Note:** You must specify the `-type` parameter. *Data_type*: enum, optional
`-update`	Specifies that the data of the specified cell will be loaded without doing any checking. *Data_type*: bool, optional

Example

- The following command specifies the default central directory name (./DATA) where models data will be saved/loaded.
  ```
  set_module_model -default_dir ./DATA
  ```
- The following command configures the tdsp_core module model using LEF and timing model ETM (.lib) together at the top-level design:
  ```
  set_module_model -cell tdsp_core -tag my_prects -type lef -add_ons {etm}
  ```
- The following example specifies the default options for type lef:
  ```
  set_module_model -type lef -default_create_options {-specifyTopLayer M10 -stripePin}
  ```

Related Information

- `check_module_model`
- `commit_module_model`
- `create_module_model`
- `report_module_model`
- `restore_module_model`
- `savePartition`
- `update_module_model`
- `write_module_model_context`
- Hierarchical and Prototyping Flow chapter in the *Innovus User Guide*
  - Integrated Hierarchical Database Flow

update_module_model

```
update_module_model
[-help]
[-cpu number_of_cpus]
[-distribute_job]
[-eco_report]
[-log_dir dir_name]
[-return_tag tag_name]
[-return_type {lef flexIlm ilm pnr}]
[-tag tag_name]
{[-distribute_run | -cell cell_name] [-interactive | -plugin_tcl tcl_commands | -gen_eco_only ]}
```

Invokes an independent session in interactive or batch mode at top-level design. The `update_module_model` command can be called at top-level design.

Note: When an independent session is invoked, the license(s) will be checked-out from the main Innovus session and checked in to the independent session. Once you exit the independent session, the license(s) will return back to main Innovus session.

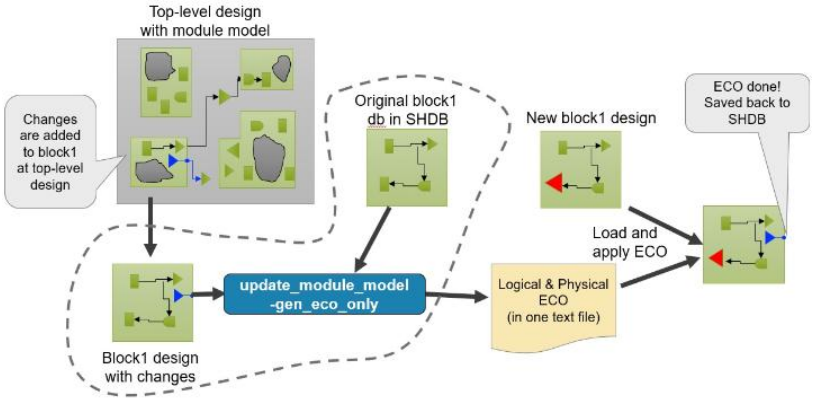
The `update_module_model` command:

- automatically updates the changes from top to block in an independent session.
- runs the user-provided plug-in commands in batch mode.
- automatically brings back the updated LEF models of the same tag to the top-level once the job is done.

Note: You should run the `create_module_model` command to save different types of models at the end of an independent session.

Parameters

`-cell` *cell_name*	Specifies which cell the command will update changes. *Data_type*: string, optional
`-cpu` *number_of_cpus*	Specifies the number of CPUs that will be used for the independent session. By default it uses the same of number of CPUs as its parent. *Data_type*: int, optional
`-distribute_job`	Enables distributed processing where more than one block design can be run in parallel. *Data_type*: bool, optional
`-distribute_run`	Enables you to execute all the specified `update_module_model` distributed jobs using a Makefile. *Data_type*: bool, optional
`-eco_report`	Specifies the output report file that contains the ECO change information. *Data_type*: bool, optional

`-gen_eco_only`	Generates an eco file that contains general netlist ECO and placement changes. This mode allows you to explicitly review the ECO before applying it to the block DB.  *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `update_module_model` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man update_module_model`
`-interactive`	Invokes independent session in interactive mode. *Data_type*: bool, optional
`-log_dir` *dir_name*	Specifies the log file directory name. *Data_type*: string, optional
`-plugin_tcl` *tcl_commands*	Specifies the TCL commands to be run in a batch independent session. *Data_type*: string, optional
`-return_tag` *tag_name*	Specifies the tag name that will be used to update the specified cell model at the top-level design. *Default*: The default return tag is the tag name of the current loaded block design. *Data_type*: string, optional
`-return_type {flexIlm ilm lef pnr}`	
	Specifies the model type to be brought back to top-level design. The return type can be lef, flexIlm, ilm, or pnr. *Default*: lef *Data_type*: enum, optional
`-tag` *tag_name*	Allows you to explicitly specify the tag name of the cell pnr model that will be loaded. If a tag is not specified, the `update_module_model` command loads the pnr model of the same tag with the currently loaded design (`restore_module_model`). For example, if the currently loaded design has tag *init*, then the `update_module_model` command will load the pnr model of tag *init* for the specified cell. It will display an error if the tag is not available. *Data_type*: string, optional

Example

- The following command opens an independent session in batch mode for the full DB of the cell tdsp_core, tag preCts:

```
update_module_model -cell tdsp_core -return_type ilm -plugin_tcl {
 route_eco
 create_module_model
}
```

Related Information

- `check_module_model`
- `commit_module_model`
- `create_module_model`
- `report_module_model`
- `restore_module_model`
- `savePartition`
- `set_module_model`
- `write_module_model_context`
- Hierarchical and Prototyping Flow chapter in the *Innovus User Guide*
  - Integrated Hierarchical Database Flow

write_module_model_context

```
write_module_model_context
[-help]
{-cell cellName | -master_clone_inst master_clone_inst}
[-tag tag_name]
[-type {extraction_context timing_context}]
[-type_specific_options type_specific_options]
```

Creates a context model for a specific cell or a master/clone instance. This context model can be loaded additionally with its design later in the flow.

Parameters

`-cell cellName`	Specifies the cell name(s) to be loaded. You can specify a list of cell names. *Data_type*: string, required
`-help`	Outputs a brief description that includes the type and default information for each `write_module_model_context` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_module_model_context`
`-master_clone_inst instance_name`	Specifies the instance name of a master/clone module. Use this option if the instance has clone(s). *Data_type*: string, optional
`-tag tag_name`	Specifies the output tag name. *Default*: latest *Data_type*: string, optional
`-type {extraction_context timing_context}`	Specifies the type of the generated context model. • `extraction_context`: Writes the extraction context information of specified modules/instances. • `timing_context`: Writes the timing context information of specified modules/instances. *Data_type*: enum, optional

`-type_specific_options`	Specifies the options of the type model. • For **`extraction_context`** type of model, the type specific options available are: `{-extension` *`distance`* `| -surrounding_tag` *`name`* `| -ground` *`name`*`}` ○ `-extension` *`distance`* Specifies the block halo distance (in microns) where routing data will be included in an output DEF file. *Default*: 10 ○ `-surrounding_tag` *`name`* Specifies the surrounding object tag name. *Default*: Tag name of the current loaded design. ○ `-ground` *`name`* Specifies the ground net that the surrounding nets will be tied to. • For **`timing_context`** type of model, the type specific options available are: `{-skip_interface_paths	-overwrite	-with_constraints}` ○ `-skip_interface_paths` Specify this option if the module is instantiated many times at the full-chip top design. It preserves the context data impacting only internal reg-reg paths. During analysis with this context data, interface paths will be unconstrained. ○ `-overwrite` Specify this option to overwrite the existing `timing_context` model. It overwrites the existing context data. ○ `-with_constraints` Specify this option to write the SDC file of each of the specified module inside the corresponding sub directories. This option is supported only in SMSC flow and for single instantiated modules. *Data_type*: string, optional

Example

- The following command creates an extraction context model for cell ptn_wrapper with routing data within 50 micron halo distance from ptn_wrapper LEF boundary:

```
restore_module_model dtmf_recvr_core -tag init -noTiming

# Ensure that routing to be cut around specified object should be a LEF
set_module_model -cell ptn_wrapper -type lef -tag my_extract_v1
commit_module_model -mmmc_file design/dtmf_recvr_core.enc.dat/viewDefinition.tcl

earlyGlobalRoute

write_module_model_context -cell ptn_wrapper -tag my_extract_v1 -type_specific_options {-extension 50 -ground vss1 init} -type
extraction_context
```

Related Information

- `check_module_model`
- `commit_module_model`
- `create_module_model`
- `report_module_model`
- `restore_module_model`
- `savePartition`
- `set_module_model`
- `update_module_model`
- Hierarchical and Prototyping Flow chapter in the *Innovus User Guide*
  - Integrated Hierarchical Database Flow

16

Interactive ECO Commands and Global Variables

- Interactive ECO Commands
- ECO Global Variables

Interactive ECO Commands

- addHierInst
- addInst
- addModulePort
- addNet
- attachDiode
- attachIOBuffer
- attachModulePort
- attachTerm
- bitblast_ports
- delete_feedthru_buffer
- deleteBNet
- deleteEmptyModule
- deleteInst
- deleteModulePort
- deleteNet
- deleteNotchFill
- deleteUninstantiatedModules
- detachModulePort
- detachTerm
- displayBufTree
- ecoAddRepeater
- ecoChangeCell
- ecoCompareNetlist
- ecoDefIn
- ecoDeleteRepeater
- ecoDesign
- ecoOaDesign
- ecoPlace
- ecoRemoveTiedInputs
- ecoSwapSpareCell
- endECO
- fillNotch
- fix_multi_drivers
- getEcoMode

- group
- initECO
- loadECO
- remove_assigns
- setEcoMode
- ungroup

addHierInst

```
addHierInst
[-help]
-cell cellName
{-hinst hinstName | -module moduleName}
```

Adds a hierarchical module to the design.

You can use this command after importing a design.

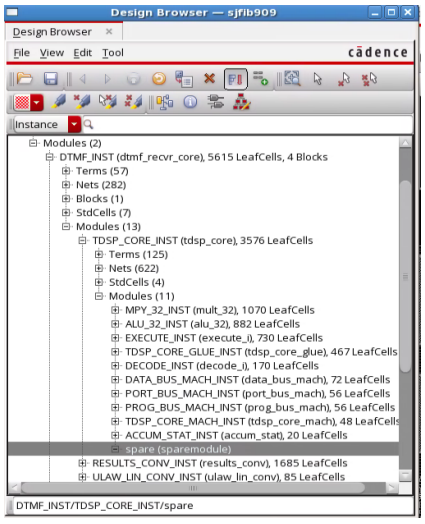
Parameters

`-help`	Outputs a brief description that includes the type and default information for each `addHierInst` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man addHierInst`
`-cell cellName`	Specifies the master of the module. Innovus creates a new, empty master if the cell name is new.
`-hinst hinstName`	Specifies the name of the module instance to add.
`-module moduleName`	Specifies the name of the module instance to add. **Note**: There is no difference between `-module` and `-hinst`. You should use `-module` for backward compatibility. It is recommended to use -hinst.

Examples

- The following command adds a hierarchical instance `DTMF_INST/TDSP_CORE_INST/spare` of module type sparemodule to the design, here DTMF_INST/TDSP_CORE_INST should be an existing hierarchical instance in the design:

  ```
  addHierInst -cell sparemodule -hinst DTMF_INST/TDSP_CORE_INST/spare
  ```



- The following commands create a new module `modA` and hinsts `inst_modA` and `inst_modA_clone` whose master cell is `modA`:

  ```
  addHierInst -cell modA -hinst inst_modA
  ```

```
addHierInst -cell modA -hinst inst_modA_clone
```

The resulting Verilog netlist would have:

```
module modA();

endmodule

modA inst_modA();

modA inst_modA_clone();
```

Related Information

- addModulePort

addInst

```
addInst
[-help]
[-dontSnapToPlacementGrid]
[-moduleBased verilogModule]
[-physical]
-cell cellName
-inst instName
[-loc {x y} [-ori {R0 | R90 | R180 | R270 | MX | MX90 | MY | MY90}]]
[-place_status placementStatus]
```

Adds an instance and places it in the design.

When `-inst` is specified, the instance is bound with the correct cell in the power domain.

When `-inst` and `-loc` are specified, the location also gives a power domain, which is checked against the power domain of the instance. If the power domains are different, a warning is issued.

In a MSV design, a cell specified using the `-cell` parameter should exist in libraries bound to the power domain of the newly added instance.

When `-loc` is specified along with the `-inst` parameter, the power domain given by the location should be consistent with the power domain of the newly added instance.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `addInst` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man addInst`
`-dontSnapToPlacementGrid`	If `-dontSnapToPlacementGrid` is specified, Innovus places the instance at the specified location even if the location is off the placement grid. This parameter is used with `-loc`. *Default*: false
`-moduleBased` *verilogModule*	Adds the new instance to the specified verilog module.
`-physical`	Places a physical instance without updating the netlist.
`-cell` *cellName*	Specifies the master of the instance.
`-inst` *instName*	Specifies the name of the instance to add and place. If the instance name contains special characters, you need to start the part of the instance name, which contains special characters, with a backslash '\' and end it with a space.
`-loc {`*x y*`}`	Specifies the location of the placed instance. *Default:* If you do not specify a location, Innovus places the instance at the design origin.
`-ori {R0 | R90 | R180 | R270 | MX | MX90 | MY | MY90}`	Specifies orientation of the placed instance. *Default:* If you do not specify an orientation, Innovus uses `R0`.
`-place_status` *placementStatus*	Specifies the placement status of the added instance. You can specify the following as the placement status: • `cover`: Specifies that the instance has a location and is a part of a cover macro. It cannot be moved by automatic tools or interactive commands. You must specify the instance's location and its orientation. • `placed`: The status of the added instance is placed. • `fixed`: The status of the added instance is fixed. • `soft_fixed`: The status of the added instance is soft_fixed. It cannot be moved by global placement and can only be moved by the legalization step of detail placement • `unplaced`: The status of the added instance is unplaced. *Default*: `unplaced` **Note:** This option is used along with the `-loc` option. It should not be used along with the `-moduleBased` option.

Examples

- The following command adds buffer instance i1/i2 at location 100, 200:
  ```
  addInst -cell BUF1 -inst i1/i2 -loc 100 200
  ```
- The following command adds buffer instance `insta` to the verilog module `HIER_2`:
  ```
  addInst -moduleBased HIER_2 -cell BUFX2_6 -inst insta
  ```
- The following command adds an instance with base name ant to hierarchy `DTMF_INST/TDSP_CORE_INST/TDSP_CORE_MACH_INST[0].g`:
  ```
  addInst -cell ANTENNA -inst{DTMF_INST/TDSP_CORE_INST/TDSP_CORE_MACH_INST[0].g/ant}
  ```
- The following command adds an instance with base name `TDSP_CORE_INST/TDSP_CORE_MATCH_INST[0].g/ant` to hierarchy `DTMF_INST`:
  ```
  addInst -cell ANTENNA -inst {DTMF_INST/TDSP_CORE_INST\/TDSP_CORE_MATCH_INST[9].g/ant}
  ```

Related Information

- deleteInst
- ecoPlace
- ecoSwapSpareCell
- loadECO

addModulePort

```
addModulePort
[-help]
moduleName
portName
portDir
[-bus n1:n2]
[-moduleBased]
```

Adds a port or bussed port to a module, specified as a module or a hierarchical instance.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `addModulePort` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man addModulePort`
`-bus n1:n2`	Adds a bussed port to the module. Specify the bus range (the beginning and end of the bus). Use integers to specify the range. If the specified bus already exists, tool will do bus bit resizing. The specified bus bit range must be adjacent to the first or the last bit of the existing bus. Also, it must have the same ascending or descending order as the existing bus.
`-moduleBased`	Specifies if the port is module based.
moduleName	Specifies the name of a hierarchical instance, or a Verilog module if `-moduleBased` is specified, to which you want to add a port. Specify "-" for the top hierarchical instance.
portDir	Specifies the port direction. You can specify the direction as: • `input` • `output` • `bidi`
portName	Specifies the name of the port to be added. The specified *portName* can be a new port name or any of the existing internal net names to which the port will connect in the module. Internal net means net inside the module without connections to any existing port of the module. If the new port name is the same as the existing net name in the module, the net will not connect the port. The new port name will be portName_1. Port Name cannot be the same as any existing port name of the same module. If a bus port is added, the bit port name cannot be the same as any net name in the module.

Examples

- The following command creates an input port `p1` on instance `i1/i2/i3`. The port `p1` must be a new port name in instance `i1/i2/i3`. Instance `i1/i2/i3` contains no nets name p1:
  ```
  addModulePort i1/i2/i3 p1 input
  ```
- The following set of commands adds a hierarchical block and then adds a bussed port to the block.
  ```
  addHierInst -cell i_block -hinst i_block1
  addModulePort i_block1 data output -bus 31:0
  ```
- The following command adds two input bits i1[8] and i1[9] to existing top-level bus port i1[0:7].
  ```
  addModulePort - i1 input -bus 8:9
  ```

Related Information

- `addHierInst`
- `deleteModulePort`

addNet

```
addNet
[-help]
netName
[-bus startID:endID]
[-moduleBased verilogModule]
[-power  | -ground ]
```

Adds a net to the design. The net can be logical or physical.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `addNet` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man addNet`
`-bus` *`startID:endID`*	Specifies the bit range for bus nets. The *`startID`* and *`endID`* indicate the first and last bits on the bus. You must separate the start and end IDs with a colon (:). If the specified bus does not exist, tool will create a bussed Verilog net. If the specified bus already exists, tool will do bus net resizing. The specified bus bit range must be adjacent to the first or last bit of the existing bus. Also, it must have the same ascending or descending order as existing bus. **Note**: PG bus net is not supported
`-ground`	Adds a ground net.
`-moduleBased` *`verilogModule`*	Adds the new net to the specified verilog module. If the module name contains special characters, you need to start the module name with a backslash '\' and end it with a space.
`netName`	Specifies the net to add. If the net name contains special characters, you need to start the part of the net name, which contains special characters, with a backslash '\' and end it with a space. **Note:** You must specify *`netName`* before you specify other parameters. The software issues an error message if you do not specify the *`netName`* first.
`-power`	Adds a power net.

Examples

- The following command adds net `i1/n1`:
  ```
  addNet i1/n1
  ```
- The following command adds one bit i1/n1[6] to existing bus i1/n1[2:5]:
  ```
  addNet i1/n1 -bus 6:6
  ```
- The following command adds three bits i1/n1[-1], i1/n1[0], i1/n1[1] to existing bus i1/n1[2:5]:
  ```
  addNet i1/n1 -bus -1:1
  ```
- The following command adds net with base name ant_net to hierarchy `DTMF_INST/TDSP_CORE_INST/TDSP_CORE_MACH_INST[0].g`:
  ```
  addNet {DTMF_INST/TDSP_CORE_INST/\TDSP_CORE_MACH_INST[0].g /ant_net}
  ```
  Here the "[" and "]" brackets between the backslash and the space are special characters.
- The following command adds net with base name `TDSP_CORE_INST/TDSP_CORE_MATCH_INST[0].g/ant_net` to hierarchy `DTMF_INST`:
  ```
  addNet {DTMF_INST/\TDSP_CORE_INST/TDSP_CORE_MATCH_INST[0].g /ant_net }
  ```
- The following command adds net with base name `ant_net[1]` to `module H[1]`:
  ```
  addNet {\ant_net[1] } -moduleBased {\H[1] }
  ```

Related Information

- deleteNet

attachDiode

```
attachDiode
[-help]
[-prefix prefix]
-diodeCell diodeCellName
-pin instName termName
[-loc x y
[-orient orient]]
```

Adds an antenna diode to a post-routed design. Innovus places the diode as close as possible to the specified pin.

When *instName* is specified, the new instance is bound with the correct cell in the power domain.

When `-loc` is specified, the location also gives a power domain, which is checked against the power domain of the pin. If the power domains are different, a warning is issued.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `attachDiode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man attachDiode`
`-diodeCell` *diodeCellName*	Specifies the name of the diode cell master
`-loc` *x y*	Specifies the location of the lower left corner of the diode.
`-orient` *orient*	Specifies orientation of the placed diode. To specify this parameter, you must also specify `-loc`.
`-pin` *instName termName*	Specifies the pin associated with the diode.
`-prefix` *prefix*	Specifies the prefix name of the diode cell instances. Innovus appends integers to differentiate the instances.

Example

- The following example adds a diode of type `D1` for `term2` on instance `i2`, at location `500`, `600`:

  ```
  attachDiode -diodeCell D1 -pin i2 term2 -loc 500 600
  ```

attachIOBuffer

```
attachIOBuffer
[-help]
[-baseName baseName]
[-dishonorUpfDomainBoundaryCheckForIsoLsStrategy]
[-excNetFile excNetFileName]
[-excludeClockNet]
[-port]
[-prePlace]
[-selNetFile selNetFileName]
[-skipRefinePlace]
[-suffix suffixName]
{[-in cellNameList] [-out cellNameList]}
[-status {placed fixed softfixed}]
```

Adds buffers/inverters to the I/O pins of a block and places the buffers/inverters near the I/O pins. When a top-level input (output) is connected to a power domain other than the default, the added buffer is matched to the hierarchy of the destination (source) cell and placed inside the power domain.

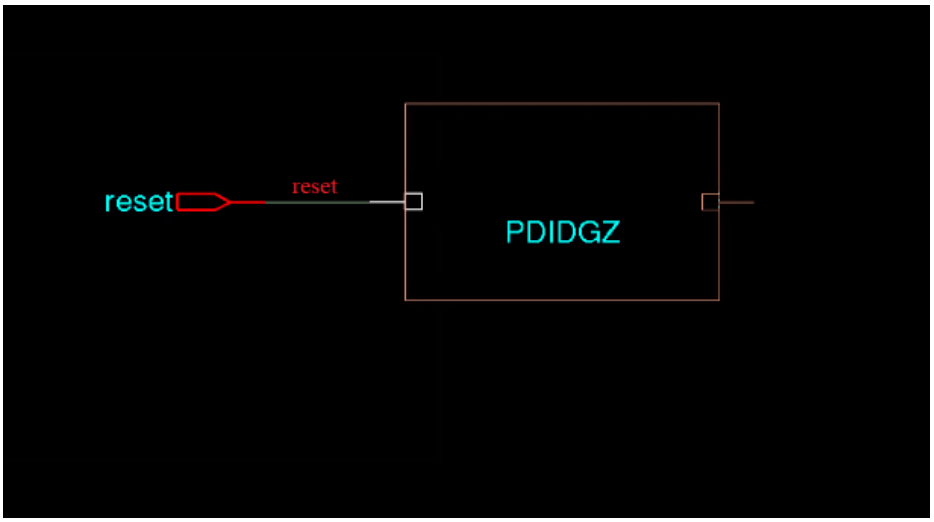
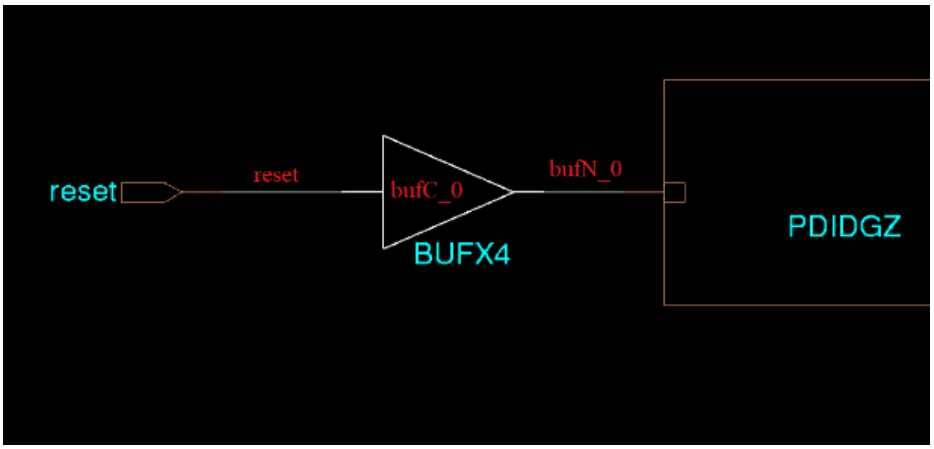
This command is power-domain aware. To take advantage of this feature when the netlist contains level shifters, load the shifter table before running this command.

Through the `-port` and `-suffix` parameters, you can control the instance and net names created when you use `attachIOBuffer`.

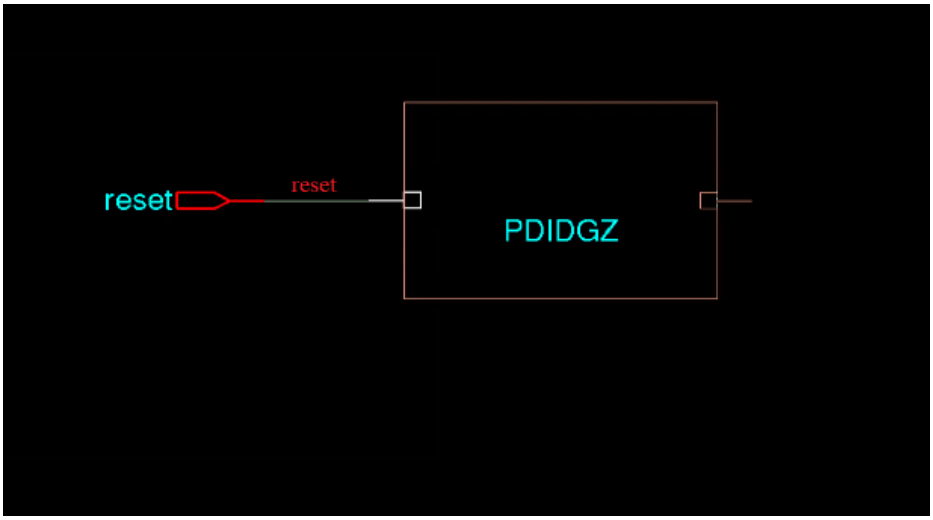
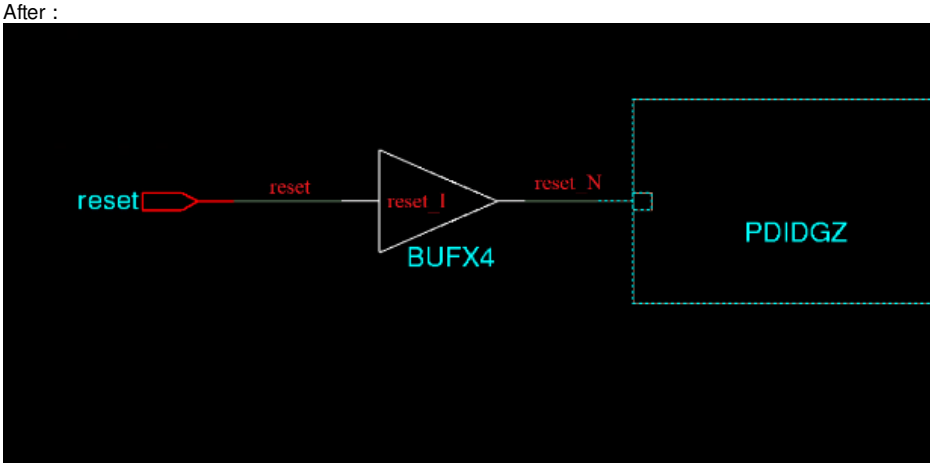
You can use this command after loading a design. Use the `-status fixed` parameter before you run `place_design`.

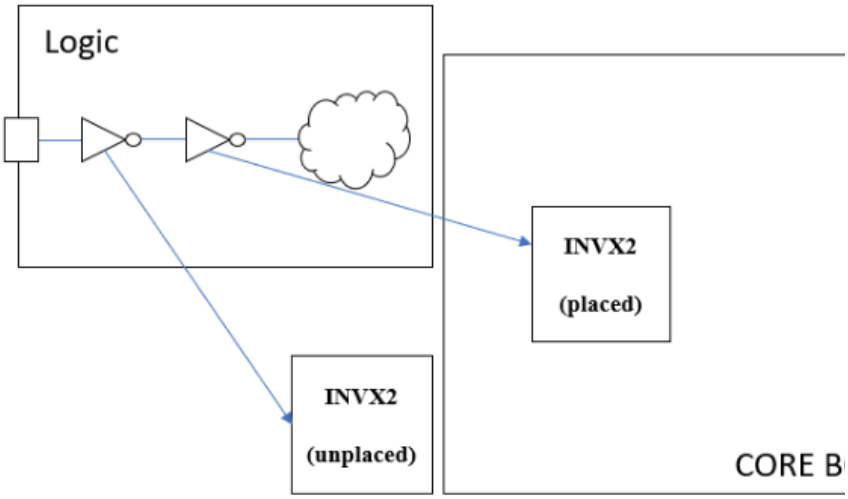
Parameters

`-help`	Outputs a brief description that includes the type and default information for each `attachIOBuffer` paran For a detailed description of the command and all of its parameters, use the `man` command: `man attachIOBuffer`

`-baseName` *baseName*	Specifies the base name of the new nets. For nets with the same base name, you should consider the h and add `1`. For example, if there are several nets with an `abc` base name, ending on `abc98`, then the first same base name is `abc99`. Example: `attachIOBuffer -baseName buf -in BUFX4` Before：  After： 
`-dishonorUpfDomainBoundaryCheckForIsoLsStrategy`	Dishonors power-domain boundary check for isolation/level-shifter instances.
`-excludeClockNet`	Does not connect the buffer on clock nets. You must trigger the clock net markings prior to running `atta` the `timeDesign` command. To query whether a given clock net is marked, use `dbIsNetClock` *netName*.
`-excNetFile` *excNetFileName*	Specifies the file that contains the names of nets to exclude from the buffer attachment operation.
`-in` *cellNameList*	Specifies the cell name or a list of cell names of the input (`-in`) buffer. If the specified cell is an inverter, `attachIOBuffer` command inserts an inverter pair. **Note:** If the `-in` and `-out` parameters are specified together, both input and output pins will be attached If only `-in` (or `-out`) option is specified, then only input (or output) pins will be attached buffer. If a list of c it will insert the first CPF compliant cell. It is recommended to give non-AO buffers/inverters before AO b the list.

`-out` *cellNameList*	Specifies the cell name or a list of cell names of the output (`-out`) buffer. If the specified cell is an inverte `attachIOBuffer` command inserts an inverter pair. **Note:** If the `-in` and `-out` parameters are specified together, both input and output pins will be attached If only `-in` (or `-out`) option is specified, then only input (or output) pins will be attached buffer. If a list of c it will insert the first CPF compliant cell. It is recommended to give non-AO buffers/inverters before AO b the list.

`-port`	Prepends the port name to the name of the net or instance created. Example: `attachIOBuffer -in BUFX4 -port` Before：  After： 

`-prePlace`	Specifies to legally place the inverter connected to the port and to leave the other inverter unplaced. Example: `attachIOBuffer -in INVX2 -prePlace` 
`-selNetFile` *selNetFileName*	Specifies the file that contains the names of nets to include in the buffer attachment operation.
`-skipRefinePlace`	Skips legalization of the new buffer/inverter (incremental placement needs to be run later to legalize it).
`-status {placed fixed softfixed}`	Specifies the placement status of the added buffer as placed/fixed/softfixed.
`-suffix` *suffixName*	Appends a string to name of the net or instance created.

Examples

- The following command attaches buffers `bufx4` to the I/O pins. The new nets have a basename of `abc`:
  ```
  attachIOBuffer -baseName abc -in bufx4
  ```
- The following command prepends the port name and appends the `_buf` string to the name of the instance and net created by `attachIOBuffer`:
  ```
  attachIOBuffer -port -suffix "buf"
  ```

Resulting instance name:

```
port1_I_buf
```

Resulting net name:

```
port1_N_buf
```

Related Information

- `place_design`
- `refinePlace`
- `timeDesign`

attachModulePort

```
attachModulePort
[-help]
moduleName
portName
netName
[-noNewPort]
```

Attaches a port in the specified hinst (or top level) to a net.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `attachModulePort`. For a detailed description of the command and all of its parameters, use the man command: man `attachModulePort`
`moduleName`	Specifies the module to which you want to attach the port. To specify the top module, enter '-'.
`netName`	Specifies the net on the module.
`portName`	Specifies the port on the module.
`-noNewPort`	Prevents creation of hierarchical ports if a module port is attached. If the module port does not connect to the net in a different hierarchy through the existing ports, the software displays an error message and the command stops running. *Default*: If you do not specify this parameter, the software creates hierarchical ports as required.

Examples

- The following commands create a port `p1` on hinst `i1/i2/i3` and a net `i1/i2/n1`. The `attachModulePort` command then connects the created port `p1` on hinst `i1/i2/i3` to the net `i1/i2/n1`:

  ```
  addModulePort i1/i2/i3 p1 input
  addNet i1/i2/n1
  attachModulePort i1/i2/i3 p1 i1/i2/n1
  ```

- The following command attaches port `in` on the top module to `net123`:

  ```
  attachModulePort - in net123
  ```

Related Information

- `addNet`
- `detachModulePort`

attachTerm

```
attachTerm
[-help]
[-moduleBased verilogModule]
[-noNewPort]
instName
termName
netName
[-port portName]
[-pin refInstName refPinName]
```

Attaches a terminal to a net. If the terminal already connects to a different net, the software first detaches the terminal from the current net, then attaches it to the new net.

It removes term order dependency while attaching multiple terms to a net, especially when both output and input terms are involved.

Note: Detaching a terminal that drives an output terminal of a module produces a Verilog violation at the output terminal if you use `detachTerm`. Instead, use `attachTerm` to attach the terminal to a new net. The `attachTerm` command automatically detaches the terminal from the net connecting to the output terminal, then attaches the terminal to the net you specify. This command also enables you to specify a hierarchical net name, in addition to a flat net name.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `attachTerm`. For a detailed description of the command and all of its parameters, use the man command: `man attachTerm`
instName	Specifies the instance containing the terminal. If the instance name does not exist, the software displays an error message and the command stops.
`-moduleBased` *verilogModule*	Attaches the terminal to the specified verilog module.
netName	Specifies the name of the net to attach to the terminal. If the net name does not exist, Innovus displays an error message and the command stops.The reserved names of 1'b1 and 1'b0 can be used to set a constant directly on the terminal rather than connect it to a net.
`-noNewPort`	Prohibits Innovus from creating hierarchical ports when it attaches a terminal. If the terminal cannot connect to the net through existing ports, Innovus displays an error message and the command stops. *Default:* If you do not specify this parameter, Innovus creates hierarchical ports as needed.
`-pin` *refInstName refPinName*	Specifies the pin *refPinName* on instance *refInstName* connected to the net that Innovus connects to the terminal. You cannot specify the `-port` parameter if you use this parameter.
`-port` *portName*	Specifies the hierarchical port used to connect the terminal with the net. If you specify this parameter, the hierarchical port must exist in the module that contains the instance. The hierarchical port must connect to the net. This parameter lets you use a specific port to maintain the same netlist topology, which simplifies equivalence checking later in the design flow. You cannot specify the `-pin` parameter if you use this parameter. *Default:* If you do not specify this parameter, Innovus uses existing ports or creates new hierarchical ports as necessary to connect the terminal to the net.
termName	Specifies the name of the terminal that Innovus connects to the specified net. If the terminal name does not exist, Innovus displays an error message and the command stops.

Examples

- The following command attaches terminal `in1` of instance `i1/i2/i3` to net `i1/i2/net26`:

  ```
  attachTerm i1/i2/i3 in1 i1/i2/net26
  ```

  If `i1/i2/i3` does not exist, or if `in1` is not a terminal of `i1/i2/i3`, or if `i1/i2/net26` does not exist, the software displays an error message and the command stops.

- The following command attaches terminal `in1` of instance `i1/i2/i3` to net `net27`, using hierarchical port `myPort`:

  ```
  attachTerm i1/i2/i3 in1 net27 -port myPort
  ```

  The `myPort` port must exist in the module definition for `i1/i2`, and `myPort` must already connect to `net27`. If this is not the case, the software displays an

error message and the command stops.

- The following command attaches terminal `in3` on instance `i1/i2/i3` through existing ports to `net28`:

  ```
  attachTerm -noNewPorts i1/i2/i3 in3 net28
  ```

  If ports do not exist, the software displays an error message and the command stops.
- The following command attaches terminal `Y` of instance `testInst` to the verilog module `hier_t3` using net `testnet`:

  ```
  attachTerm -moduleBased hier_t3 testInst Y testnet
  ```

Related Information

- `detachTerm`

bitblast_ports

```
bitblast_ports
[-help]
topCell|vCell|hInst+
[-bus busname]
```

Bitblasts the specified port bus (or hport bus) of the specified design, module, or module of the specified hinst, so they are scalars rather than bus-bits. This can be useful to allow optimization for flexibility such as changing the inversion of just some bits of the bus. The name of the bitblasted ports will follow the nomenclature specified by the bit_blasted_port_style global variable. The default style is:

%s_%d

where %s is the bus name, and %d is the bus-bit value. Each bus-bit that is renamed is reported for each module that is changed.

Names used for SDC timing constraints can only be modified if they are loaded into the memory. So, you must ensure that all the timing views are loaded with `set_analysis_view` before running this command. You can either write out the modified SDC files with the new names for later usage or write constraints and Tcl scripts in a way that does not use the original bus-bit names.

There is a limitation that port names used inside a CPF power_intent file cannot be changed. CPF usage assumes that the CPF file is golden and should not be modified. If you need to change names used inside a CPF file, you need to update the CPF file manually, and then read in the new CPF file after running `bitblast_ports`. In contrast, an IEEE1801 power_intent file is allowed to change during the design flow, so changing any port names with bitblast_ports will automatically update the IEEE 1801 names. You can write out the new IEEE1801 file with the updated names.

For non-uniquified netlist, if you bit-blast the hinst, the module and the cloned hinsts also get bit-blasted.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `bitblast_ports`. For a detailed description of the command and all of its parameters, use the man command: `man bitblast_ports`
`topCell|vCell|hInst+`	Specifies the list of design|module|hinst.
`-bus` *`busname`*	Bit-blasts just the specified bus (just “A” not “A[3:0]”), rather than all busses for the module.

Examples

- In the following example, the design top has two-bit input port AI:

```
AI[1:0]
@Innovus> bitblast_ports design:top
#New bitblasted names
Module: top
AI[0]      AI_0
AI[1]      AI_1
@Innovus> get_db ports .name
> AI_0 AI_1
```

- In the following example, hinst1 has two-bit port IN:

```
IN[1:0]
@Innovus> bitblast_ports hinst:top/hinst1 -bus IN
#New bitblasted names
Module: mod1
IN[0]      IN_0
IN[1]      IN_1
@Innovus> get_db hinst:top/hinst1 .hports.name
> hinst1/IN_0 hinst1/IN_1
```

- In the following example, design top has 2 two-bit input ports AI, BI:

```
AI[1:0] BI[1:0]
@Innovus> bitblast_ports design:top -bus AI
#New bitblasted names
Module: mod1
IN[0]      IN_0
IN[1]      IN_1
@Innovus> get_db ports .name
> AI_0 AI_1 BI[0] BI[1]
```

You can bit-blast all ports of two different hinsts:

```
>bitblast_ports {hinst:top/h1 hinst:top/h2} > bitblast.rpt
>
```

Related Information

- set_analysis_view

delete_feedthru_buffer

```
delete_feedthru_buffer
[-help]
[-dontDelDanglingPort]
[-non_preferred_keep_nets netNamePattern]
[-preferred_keep_nets netNamePattern]
{{-bufInst bufInstName} | {-instPrefix instPrefixName}
```

Removes the feedthrough buffer specified in the `-bufInst` option. It also removes all possible dangling hterms/hnets that are associated with the buffer removed. The `delete_feedthru_buffer` command supports multiple-instantiated buffer cells.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `delete_feedthru_buffer` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man delete_feedthru_buffer`
`-bufInst bufInstName`	Specifies the name of the feedthrough buffer to be removed.
`-dontDelDanglingPort`	Is used if you do not want to remove the dangling ports created by this command.
`-instPrefix instPrefixName`	Specifies the prefix name for the feedthrough buffer. It removes all buffer instances that were created by the `insertPtnFeedthrough` command if `insertPtnFeedthrough` also used this option to specify the exactly same `instPrefixName`. Therefore, you need to specify exactly the same instance name prefix in the `-instPrefix` option of both the commands. **Note**: The `-instPrefix` specified here must be unique to all instance names in the current design.
`-non_preferred_keep_nets netNamePattern`	Specifies the non-preferred keep net name pattern. The nets whose names match the pattern will not be used by buffer deletion.
`-preferred_keep_nets netNamePattern`	Specifies the preferred keep net name pattern. The nets whose names match the pattern will be used by buffer deletion.

Example

The following command deletes feedthrough buffer `i1/i2/FE_FEED_BUF_1`, and also keeps the dangling ports:

```
delete_feedthru_buffer -bufInst i1/i2/FE_FEED_BUF_1 -dontDelDanglingPort
```

Related Information

- `insertPtnFeedthrough`

deleteBNet

```
deleteBNet
[-help]
[-moduleBased moduleName]
{-bus busName | -busFromBit busBitName}
```

Given a bussed net or a single bit of a bussed net, logically removes the entire bussed net from a design.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `deleteBNet`. For a detailed description of the command and all of its parameters, use the man command: `man deleteBNet`
`-moduleBased moduleName`	Specifies the module name that contains the bus net which needs to be deleted. This parameter is optional.
`-bus busName`	Specifies the base name of the top-level bus net, or the base name of a bus net in verilog module when used with `-moduleBased` option
`-busFromBit busBitName`	Specifies the bus bit name of a top level or a non-top level bus net, or a bus bit name of a bus net in verilog module when used together with the `-moduleBased` option.

⚠ The `deleteBNet` command does not support wildcards.
It is not possible to delete a single bit of a bussed net. To change the bus width of an existing net, the net needs to be deleted and re-created with `addNet`.

Examples

Non-top level bus net

- The following command deletes the bus net x/y/z.

  ```
  deleteBNet -busFromBit x/y/z[2]
  ```

- The following command deletes a bus net from a verilog module, used with `-moduleBased` and `-busFromBit busBitName`.

  ```
  deleteBNet -busFromBit z[2] -moduleBased Y
  ```

- The following command deletes a bus net from a verilog module, used with `-moduleBased` and `-bus busName`.

  ```
  deleteBNet -bus z -moduleBased Y
  ```

Top level bus net

- The following command deletes the top-level bus net, using the option `-busFromBit busBitName`.
  ```
  deleteBNet -busFromBit x[2]
  ```
- The following command deletes the top-level bus net, using the option `-bus busName`.

  ```
  deleteBNet -bus x
  ```

Related Information

- `addNet`

deleteEmptyModule

```
deleteEmptyModule
[-help]
[-deleteEmpty1801Modules]
```

Deletes modules and their instantiations. Modules and their instantiations are deleted if the following criteria are met:

- The netlist is unique
- The module does not contain any instances, or the module contains only unconnected instances of other empty modules
- The instances of the module are not connected to other instances or ports

The `deleteEmptyModule` command does not delete used empty modules, where there exists at least one connected instance of the empty cell. For example, black boxes typically are empty, but they are still connected to other instances or ports.

At times, the lowest hierarchy is connected to the immediate higher-level hierarchy but the module is dangling and should be deleted. You can delete the dangling modules by first deleting the dangling ports followed by the deletion of the empty modules as given below:

```
deleteDanglingPort
```

```
deleteEmptyModule
```

Note: A design can contain empty modules for different reasons. Modules can become empty due to optimization processes. A design also can contain MCPO modules that were read in with the netlist. To convert empty modules into special leaf cells when reading in the netlist, see the `setImportMode` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `deleteEmptyModule`. For a detailed description of the command and all of its parameters, use the man command: `man deleteEmptyModule`
`-deleteEmpty1801Modules`	Deletes empty UPF modules.

Related Information

- `setImportMode`
- `deleteDanglingPort`

deleteInst

```
deleteInst
[-help]
listOfInst
[-moduleBased moduleName]
[-verbose]
```

Deletes an instance from a design. Innovus checks whether the instance is physical or logical, and deletes the instance accordingly. You can use deleteInst to delete the list of logical instances in one HECO session.

You can use wildcards (`*?`) to specify the instances you want the tool to delete.

As `deleteInst` supports deletion of a list of instance, you can get the list of instances using the `dbGet` command The correct way to use `dbGet` is to use TCL `join` command. The following example shows the correct usage of `deleteInst` command with `dbGet`:

```
deleteInst [ join [dbGet top.insts.name]]
```

Notes:

- If the `deleteInst` command deletes a constrained instance, the command will delete the timing graph to avoid other crashes because constraints would be referring to deleted objects and a message will be displayed.
- The `deleteInst` command cannot be used in the post-mask ECO. Please use `loadECO` *`ecofile`* `-postMask` to delete instances in the post-mask ECO.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `deleteInst`. For a detailed description of the command and all of its parameters, use the man command: man `deleteInst`
`listOfInst`	Specifies the list of the instances to delete.
`-moduleBased` *`moduleName`*	Specifies the name of the Verilog module.
`-verbose`	Prints out the deleted instance message.

Example

- The following command deletes instance `i1/i2`:
  ```
  deleteInst i1/i2
  ```
- The following command deletes all instances whose names begin with `i` inside the hierarchical instance `i1`:
  ```
  deleteInst i1/i*
  ```
- The following command deletes the instance `insta` from the verilog module `HIER_2`:
  ```
  deleteInst -moduleBased HIER_2 insta
  ```
- The following command deletes all instances in the `SPGM` module:
  ```
  deleteInst * -moduleBased SPGM
  ```

Related Information

- `addInst`
- `loadECO`

deleteModulePort

```
deleteModulePort
[-help]
moduleName
portName
[-bus startId:endId]
```

Disconnects the specified port name from its net and deletes the port. Individual bits of a bus cannot be deleted.

Parameters

`-bus startId:endId`	Specifies the startId and the endId of the bus bit. **Note**: • When a bit port or bit net is connected to something else, the bit will notbe removed. • Bus bit ports to be deleted must be adjacent to each other and adjacent to either msb or lsb of the existing bus.
`-help`	Outputs a brief description that includes the type and default information for each `deleteModulePort`. For a detailed description of the command and all of its parameters, use the man command: man `deleteModulePort`
moduleName	Specifies the name of the hierarchical instance whose port is to be deleted. Specify "-" for the top hierarchical instance.
portName	Specifies the name of the port to be deleted. Wildcards are not supported.

Examples

- The following command deletes port '`rst_i`' from the top hierarchical instance:
  ```
  deleteModulePort - rst_i
  ```
- The following command deletes all bits of the bus port '`data_i`' of the hierarchical instance `moduleB_inst`:
  ```
  deleteModulePort moduleA_inst/moduleB_inst data_i
  ```
- The following command deletes bus bits out[7], out[6], out[5], and out[4] from existing top-level bus port out[7:0]:
  ```
  deleteModulePort - out -bus 7:4
  ```

Related Information

- `addModulePort`

deleteNet

```
deleteNet
[-help]
netName
[-bus startId:endId]
[-moduleBased verilogModule]
```

Logically deletes nets from a design. If a net is routed, wire segments associated with the net are also deleted.

If you plan to call `ecoRoute -modifyOnlyLayers bottomLayer:topLayer` later, ensure that the net to be deleted only has wire segments on the specified layer.

Note: To delete wire segments associated with a net without deleting it logically, use the `editDelete` command.

You can use wildcards (`*?`) to specify the nets you want Innovus to delete.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `deleteNet`. For a detailed description of the command and all of its parameters, use the man command: man `deleteNet`
`-bus startId:endId`	Deletes the specified bus bits. The specified bus bit net must be a one pin net, which means that the bus bit net is floating on the other side. The specified range must be adjacent to the most significant bit or the least significant bit of the existing bus. Also, it must have the same ascending or descending order as the existing bus. **Note**: PG bus net is not supported.
`-moduleBased verilogModule`	Deletes the net from the specified verilog module.
netName	Specifies the name of the net to delete.

Example

- The following command deletes one bit h1/net2[3] from existing bus h1/net2[3:0]:
  ```
  deleteNet h1/net2 -bus 3:3
  ```
- The following command deletes two bits h1/net2[3], h1/net2[2] from existing bus h1/net2[3:0]:
  ```
  deleteNet h1/net2 -bus 3:2
  ```
- The following command deletes `net1`:
  ```
  deleteNet net1
  ```
- The following command deletes all nets whose name begin with `net1`:
  ```
  deleteNet net1*
  ```

Related Information

- `addNet`
- `ecoRoute`
- `editDelete`

deleteNotchFill

`deleteNotchFill`

Deletes all gaps, notches, acute angles, and hole geometries that were inserted by the `fillNotch` command. Also deletes geometry with + SHAPE DRCFILL in DEF.

- A gap is a minimum spacing violation.
- A notch is a minimum step violation.
- An acute angle is a 45 degree violation.
- A hole is a minimum enclosed area violation.

Parameters

None

Related Information

- `fillNotch`
- ECO Flows chapter in the *Innovus User Guide*

deleteUninstantiatedModules

```
deleteUninstantiatedModules
[-help]
[-reportOnly]
```

Deletes uninstantiated modules.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command, use the man command: `man deleteUninstantiatedModules`
`-reportOnly`	Reports all uninstantiated vcells instead of deleting them. All uninstantiated vcells will be deleted if `-reportOnly` is not specified. *Data_type*: bool, optional

Related Information

- ECO Flows chapter in the *Innovus User Guide*

detachModulePort

```
detachModulePort
[-help]
hinstName
portName
```

Detaches the net connected to the specified port on the specified hierarchical instance.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `detachModulePort`. For a detailed description of the command and all of its parameters, use the man command: man `detachModulePort`
`hinstName`	Specifies the name of the hierarchical instance. To detach a top-level port, specify "-" as the *`hinstName`*. For example, the following command detaches the port 'a' from the net connected to it: `>detachModulePort - a`
`portName`	Specifies the port on the module.

Example

- The following command detaches port `p1` from `moduleA`:
  ```
  detachModulePort moduleA p1
  ```

Related Information

- `attachModulePort`
- `deleteModulePort`

detachTerm

```
detachTerm
[-help]
[-moduleBased verilogModule]
instName
termName
[netName]
```

Disconnects a terminal from a net.

Note: Detaching a terminal that drives an output terminal of a module produces a Verilog violation at the output terminal if you use `detachTerm`. Instead, use `attachTerm` to attach the terminal to a new net. The `attachTerm` command automatically detaches the terminal from the net connecting to the output terminal, then attaches the terminal to the net you specify.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `detachTerm`. For a detailed description of the command and all of its parameters, use the man command: man `detachTerm`
`instName`	Specifies the instance that contains the terminal you want to detach.
`-moduleBased` *`verilogModule`*	Detaches the terminal from the verilog module.
`netName`	Specifies the net that is already connected to the terminal. If the terminal does not connect to the specified net, or if the net does not exist, the software displays an error message and the command stops. The software uses this parameter for error checking only.
`termName`	Specifies the terminal to disconnect. If the terminal does not exist on the specified instance, the software displays an error message and the command stops.

Example

- The following command disconnects terminal `in1` of instance `i1/i2/i3`:

  ```
  detachTerm i1/i2/i3 in1 i1/i2/net26
  ```

  If instance `i1/i2/i3` does not exist, or terminal `in1` is not a terminal of `i1/i2/i3`, the software displays an error message and the command stops. The net `i1/i2/net26` is specified, so if net `i1/i2/net26` does not exist, or if the terminal is not already connected to net `i1/i2/net26`, the software displays an error message and the command stops.

Related Information

- `attachTerm`

displayBufTree

```
displayBufTree
[-help]
-net netName
[-outfile fileName]
```

Displays a buffer tree associated with the specified net.

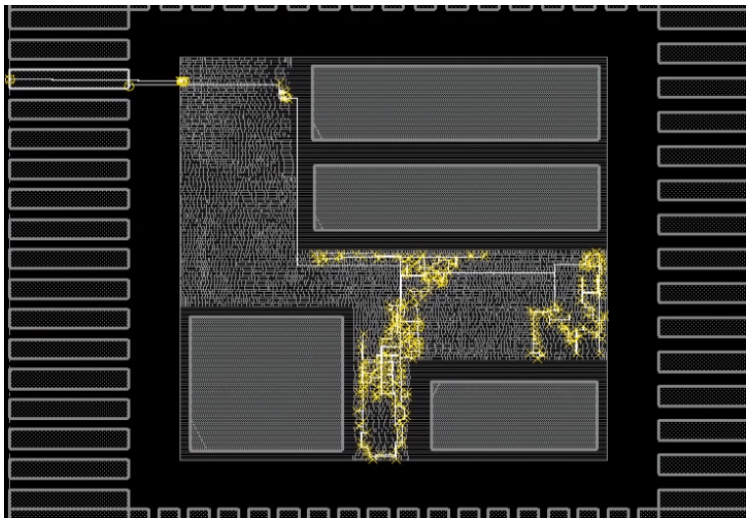
Parameters

`-help`	Outputs a brief description that includes type and default information for each `displayBufTree` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man displayBufTree`.
`-net netName`	Specifies the name of the net for which you want to display the buffer tree.
`-outfile fileName`	Specifies the text file containing buffer locations.

Example

- The following command highlights instances and wires in the buffer tree associated with the net `reset` in the design display area:

```
displayBufTree -net reset
```



```
displayBufTree -outfile reset_buftree
```

```
#########################################
# Buffer tree associated with net reset
# Format: <Buffer_Inst_Name> <x_loc> <y_loc>
#########################################
DTMF_INST/SPI_INST/i_279                          543.180     1091.440
DTMF_INST/ARB_INST/i_320                          851.400     708.400
DTMF_INST/DMA_INST/i_433                          844.800     723.520
DTMF_INST/TDSP_CORE_INST/TDSP_CORE_MACH_INST/i_684  803.220     612.640
DTMF_INST/TDSP_CORE_INST/PROG_BUS_MACH_INST/i_878   803.880     582.400
DTMF_INST/TDSP_CORE_INST/PORT_BUS_MACH_INST/i_1070  768.240     663.040
DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/i_5206        760.320     627.760
DTMF_INST/TDSP_CORE_INST/DECODE_INST/i_5663         782.760     597.520
DTMF_INST/TDSP_CORE_INST/DATA_BUS_MACH_INST/i_6324  817.080     718.480
DTMF_INST/TDSP_DS_CS_INST/i_6352                  781.440     723.520
DTMF_INST/RESULTS_CONV_INST/i_9425                1129.920    743.680
DTMF_INST/DIGIT_REG_INST/i_9464                   1136.520    758.800
IOPADS_INST/Presetip                              0.000       1111.855
abcC_0                                            338.580     1126.720
reset_I_buf                                       341.880     1126.720
C_0__buf                                          335.280     1126.720
~
```

ecoAddRepeater

```
ecoAddRepeater
[-help]
{-net netName | -term term1 term2… }
[-hinstGuide hinstName | -spreadDist distance | -spreadCount number ]
[-name instName | -spreadDist distance | -spreadCount number ]
[-spreadDist distance | -spreadCount number ]
[-newNetName netName | -spreadDist distance | -spreadCount number ]
[-loc {xLoc yLoc} | { xLoc yLoc inv_xLoc inv_yLoc} | -offLoadAtLoc {x1a y1a x1b y1b x2a y2a x2b y2b ....} | { xLoc yLoc inv_xLoc inv_yLoc} |
-offLoadSlack slack | -spreadDist distance | -spreadCount number ]
[-radius µm | -logicalChangeOnly | -spreadDist distance | -spreadCount number ]
[-offLoadAtLoc {x1a y1a x1b y1b x2a y2a x2b y2b ....} | { xLoc yLoc inv_xLoc inv_yLoc} | -offLoadSlack slack | -spreadDist distance | -
spreadCount number ]
[{[-loc {xLoc yLoc} | { xLoc yLoc inv_xLoc inv_yLoc}]
[-relativeDistToSink sinkWeight]
[-offLoadAtLoc {x1a y1a x1b y1b x2a y2a x2b y2b ....} | { xLoc yLoc inv_xLoc inv_yLoc}]} [ -radius µm ] ]
[-loc {xLoc yLoc} | { xLoc yLoc inv_xLoc inv_yLoc} [-bufOrient {R0 | R90 | R180 | R270 | MX | MX90 | MY | MY90}]]
[-net netName [-spreadDist distance]]
[-spreadPrefix prefix]] [-spreadDist distance [-firstSpreadDist distance]]
[-cell cell1 cell2]
[-relativeDistToSink sinkWeight | -offLoadAtLoc {x1a y1a x1b y1b x2a y2a x2b y2b ....} | { xLoc yLoc inv_xLoc inv_yLoc} | -offLoadSlack
slack | -spreadDist distance | -spreadCount number ]
[-loc {xLoc yLoc} | { xLoc yLoc inv_xLoc inv_yLoc} | -noPlace ]
```

Adds either a single buffer or two inverters on a net. This allows you to optionally specify the location and radius, the location relative to the sink pin, and the buffer insertion based on the slack. This command also checks for and observes power domains based on the specified instance name and location. This command cuts wires by default.

This command does not modify `dont_touch` nets if `setEcoMode -honorDontTouch` is set to `true`.

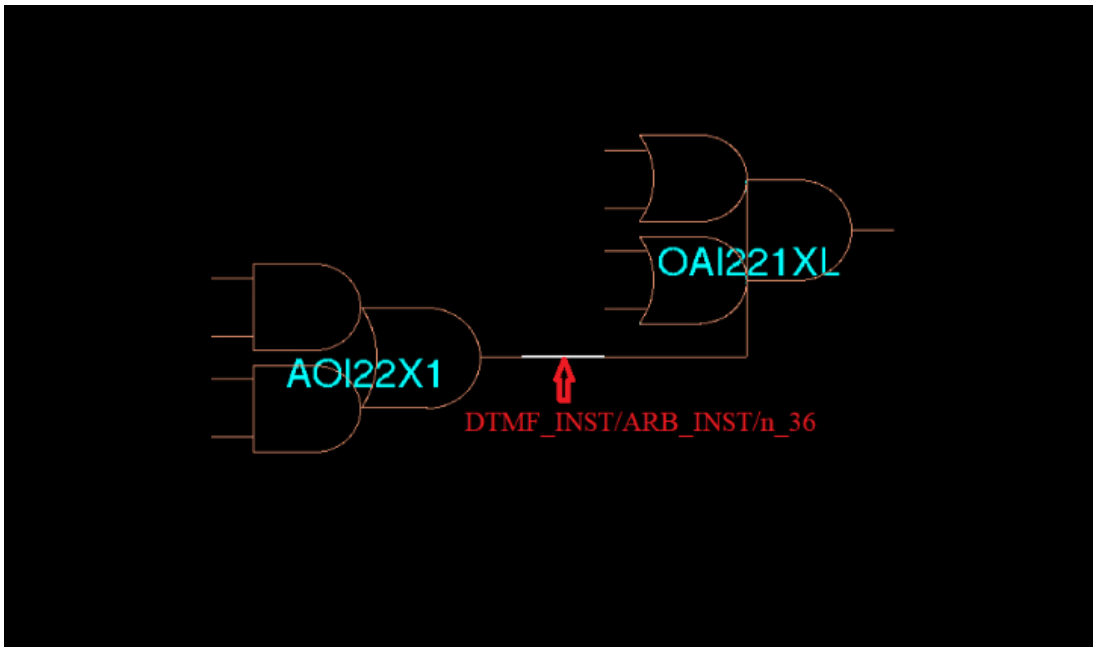
Notes:

- The `ecoAddRepeater` command is simultaneous setup/hold and MMMC aware. To make the command simultaneous setup/hold aware, set the global variable `set_global timing_enable_simultaneous_setup_hold_mode` to `true`. The simultaneous setup/hold mode can only be used for timing analysis. It cannot be used for any physical implementation steps.
  By default, if you specify an inverter as the master cell, `ecoAddRepeater` adds a pair of inverters but if you set `setEcoMode -LEQCheck` as `false`, `ecoAddRepeater` adds a single inverter only.
- `ecoAddRepeater` can return new instance and net name: `newInstName inputNet outputNet`
  Example
  ```
  set result [ecoAddRepeater -net s2_hresp0 -cell inv_32 \
  -loc {1069.018666 128.2435714}]
  set newInstName [lindex $result 0]
  set inputNetName [lindex $result 1]
  set outputNetName [lindex $result 2]
  ```
  If a pair of inverter is added then the return list could be the following, assuming `inv1OutputNet` drives the input pin of `inv2InstName`:
  ```
  {inv1InstName inv1InputNet inv1OutputNet inv2InstName inv2InputNet inv2OutputNet}
  ```
- When `ecoAddRepeater` is used in the post-mask ECO, ensure that it is used with the `-noPlace` option to avoid base layer change. Also, call `ecoPlace -useSpareCells` or `ecoSwapSpareCell` to map the new repeater to the spare cell. If you plan to call `ecoRoute -modifyOnlyLayers bottomLayer:topLayer` later, ensure that the added repeater can be connected through the specified layer. Otherwise, please do not use the `ecoAddRepeater` command. You can also use `loadECO <ecofile> -postMask` to add repeaters in the post-mask ECO.

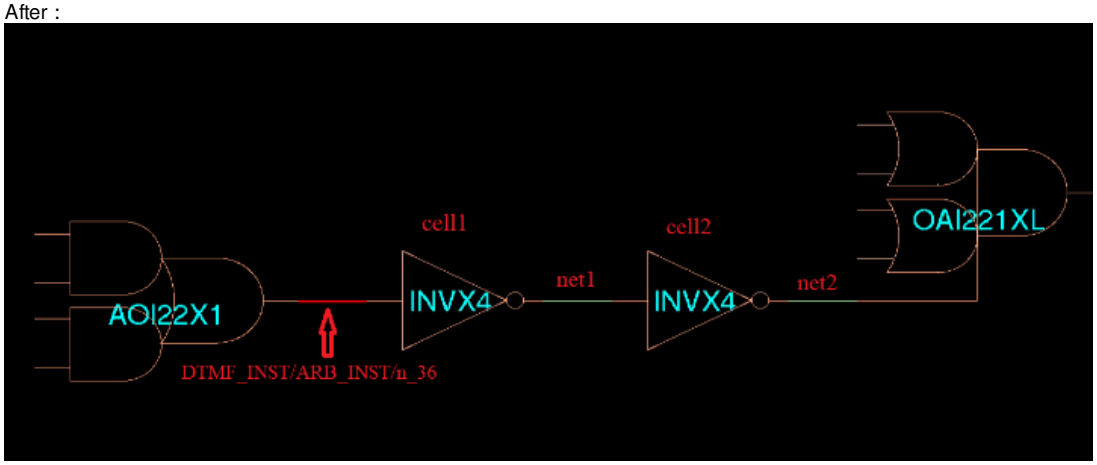
Parameters

`-help`	Outputs a brief description that includes the type and default information for each `ecoAddRepeater`. For a detailed description of the command and all of its parameters, use the man command: man `ecoAddRepeater`

`-bufOrient {R0 | R90 | R180 | R270 | MX | MX90 | MY | MY90}`	Specifies the legalized orientation for the instance. Valid orientation values are: R0, R90, R180, R270, MX, MX90, MY, MY90. **Note**: The `-loc` option is required with `-bufOrient`. For more information on orientation keys, see "Orientation Key" in the Floorplanning the Design chapter of the *Innovus User Guide*.
`-cell` *cell1 cell2 ...*	Specifies the name of the cell(s) which is to be inserted as repeater.
`-firstSpreadDist` *distance*	Specifies the distance (from the driver) after which the first repeater is to be added. With the `-firstSpreadDist` option, you must specify `-spreadDist`. *Minimum*: 2.2250738585072e-308 *Maximum*: 1073741.823
`-hinstGuide` *hinstName*	Indicates the module hierarchy in which the `inst` is inserted by Tempus or a third party tool so that Innovus is able to make the association in case the instance appears in the `-termList` option of an ECO command at a later stage. This option can be only used when `setEcoMode -batchMode true` command is specified. You also need to specify the `-name` option along with it.
`-loc {x y | x1 y1 x2 y2}`	Specifies the x and y coordinates for the location of the buffer or inverter pair to insert. If no location is specified, the tool will automatically determine a location. • If the cell is a buffer and location is specified, the newly created buffer is placed at the specified location (x y). • If multiple buffers are added in the same `ecoAddRepeater` command, the buffers are added in the order of their locations, starting from the buffer farthest from the net's driver. • If the cell is an inverter pair and two locations are specified, the first inverter will be placed at location (x1 y1) and the second inverter will be placed at location (x2 y2). • If the cell is an inverter pair and only one location is specified, both inverters will be placed at the same location (x y).
`-logicalChangeOnly`	Specifies that buffer/inverter addition should be done in the logical-only mode.
`-name` *instName*	Specifies the base name for the repeater to be inserted. When `-cell` is inverter, you can specify two names for inverter pair enclosed in {{ }}.
`-net` *netName*	Specifies the name of the net where the buffer will be inserted.
`-newNetName` *netName*	Specifies the base name for the new net created for the repeater insertion. When `-cell` is inverter, you can specify two names for inverter pair enclosed in {{ }}. Example：`ecoAddRepeater -net DTMF_INST/ARB_INST/n_36 -cell INVX4 -name {{cell1 cell2}} -newNetName {{net1 net2}}` Before：



After :



`-noPlace`	Specifies that the inserted cells should not be placed. Only the logical change in connectivity will be made.
`-offLoadAtLoc {x1a y1a x1b y1b x2a y2a x2b y2b ....}`	Specifies the location(s) at which buffer (or inverter-pair) should be inserted to drive the sinks downstream from the location(s).
`-offLoadSlack` *slack*	Inserts a buffer to offload the paths with less than the specified slack threshold from the driver of a net with multiple cells. This option ignores the *term* option, if it is provided. This option is used to connect the added buffer to drive only noncritical receivers, selected By Slack. This checks the timing graph for noncritical receivers and offloads those from the critical path, and could improve critical path timing by penalizing noncritical path delays. **Note**: • The `-offLoadAtLoc` and `-offLoadSlack` options are mutually exclusive. • The `-loc` and `-offLoadSlack` options are mutually exclusive.

`-radius µm`	Specifies the radius in which the added instances are free to move. If no legal location can be found in the specified radius, the cells would be placed at the specified location resulting in an overlap with other cells. In that case, you should perform legalization. **Note**: When the `-radius` option is used in `ecoAddRepeater`, `{[-loc] [-relativeDistToSink] [-offLoadAtLoc]}` must be specified.
`-relativeDistToSink` *sinkWeight*	Specifies the location of the buffer based on its distance from the sink or the driver pin. The value is a number between 0 and 1. A low value (0.1) places the buffer near the sink; a high value (0.9) places the buffer near the driver. The fraction is based on the length of the wire. This option works: • When one `term` is provided; it does not work if no `term` or multiple terms are specified. • When `-net` with 1-sink net is provided. It does not work if the specified net is not a 1-sink net.
`-spreadCount` *number*	Specifies the number of repeaters to be added for buffering. *Minimum*: 1 *Maximum*: INT_MAX
`-spreadDist` *distance*	Specifies the distance (starting from the driver) after which repeaters are to be added. *Minimum*: 2.2250738585072e-308 *Maximum*: 1073741.823
`-spreadPrefix` *prefix*	Specifies the prefix to be used for new instances and nets added during the spread buffering.
`-term` *term1 term2…*	Specifies a list of terms. The terms specified in the list should be connected to a common net and the tool adds buffer to the common net. If `-term` is not specified, all the terms of the net will be buffered.

Note: You can specify the following in the `setEcoMode` command:

```
setEcoMode -honorFixedStatus false #to modify FIXED instance.

setEcoMode -honorDontUse false #to modify instances which have dontUse attribute on cells.

setEcoMode -honorDontTouch false #to modify instance/nets which are marked dontUse.

setECOMode -honorFixedNetWire false #to insert a cell on wires with the FIXED/COVER status and change the status of these wires from
FIXED/COVER to ROUTED.
```

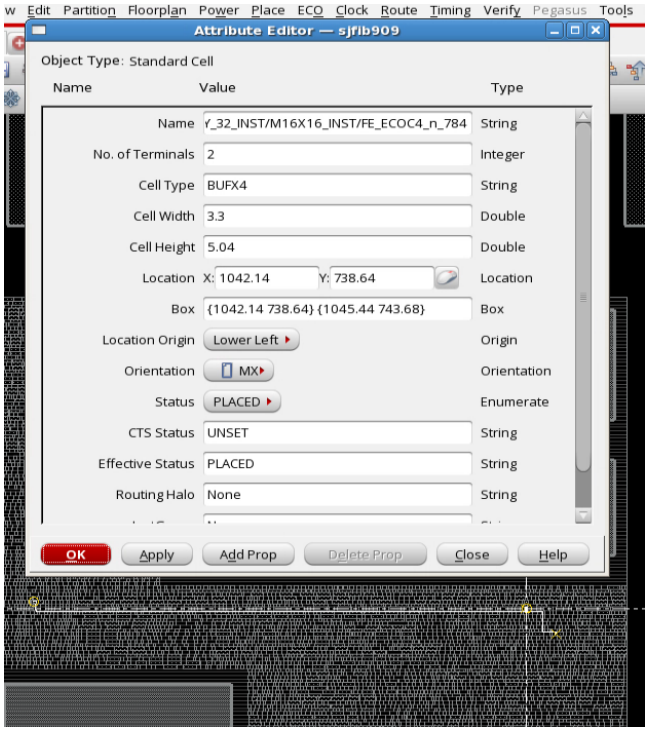
Examples

- The following syntax shows multiple buffer insertion:

  ```
  ecoAddRepeater  {-net|-term}  [-relativeDistToSink 0|1]  -cell {cell1 cell2 cell3..} [-loc <loc1> <loc2> <loc3>..] [-bufOrient
  <orient1> <orient2> <orient3>] [-name <name1> <name2> <name3>..] [-newNetName  <name1> <name2> <name3>...]
  ```

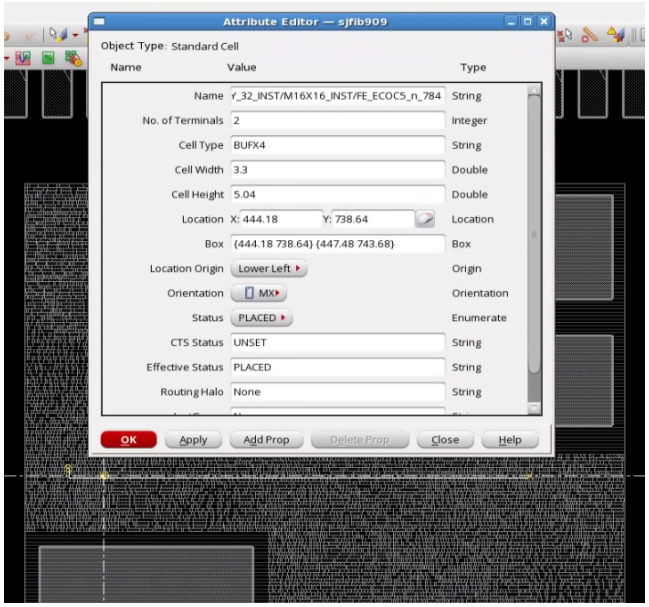
- The following command adds a buffer near the receiver:

  ```
  ecoAddRepeater -net DTMF_INST/TDSP_CORE_INST/MPY_32_INST/M16X16_INST/n_784 -cell BUFX4 -relativeDistToSink 0.1
  ```



- The following command adds a buffer near the driver:

  ```
  ecoAddRepeater -net DTMF_INST/TDSP_CORE_INST/MPY_32_INST/M16X16_INST/n_784 -cell BUFX4 -relativeDistToSink 0.9
  ```



- The following command adds a buffer in a specified location
  ```
  ecoAddRepeater -net wire_I10 -loc {51.5 85.67} -cell BUFFX4
  ```
- The following command adds a buffer by specifying the terminal list:
  ```
  ecoAddRepeater -net n1 -cell INVX4 -term {i1/i2/in1 i1/i2/in2 i1/i2/in3}
  ```
- The following command adds an inverter pair by specifying the slack time:
  ```
  ecoAddRepeater -net wire1_B2 -cell INVX4 -offLoadSlack -0.1
  ```
- The following command adds an inverter pair:
  ```
  ecoAddRepeater -net myNet -cell INVX4 -loc {123 450 145 480} -name {{cell1 cell2}} -newNetName {{net1 net2}}
  ```
- The following command adds inverter pairs and multiple buffers:
  ```
  ecoAddRepeater -cell {inv_1 inv_2} -offLoadAtLoc  {x1a y1a x1b y1b x2a y2a x2b y2b}
  ```
  In the above example, {x1a y1a } {x1b y1b} buffers are for inverter pair 1 (inv_1) and {x2a y2a} {x2b y2b} buffers are for inverter pair2 (inv_2).

Related Information

- `ecoDeleteRepeater`
- `ecoPlace`
- `ecoSwapSpareCell`
- `setEcoMode`
- `timing_enable_simultaneous_setup_hold_mode`
- For more information on orientation keys, see "Orientation Key" in the Floorplanning the Design chapter of the *Innovus User Guide*.

ecoChangeCell

```
ecoChangeCell
[-help]
-cell list_of_cells |-upsize | -downsize
-inst instNames
[-loc {xLoc yLoc} [-orient {R0 | R90 | R180 | R270 | MX | MX90 | MY | MY90}]]
[-pinMap oldpin1 newpin1 oldpin2 newpin2 ...]
```

Upsizes or downsizes the specified instance based on the available footprint. Cell mapping is also possible as long as the output pins are functionally the same. This command also checks for and observes power domains.

This command does not modify `dont_touch` nets if `setEcoMode -honorDontTouch` is set to `true`.

Note: If you use the `ecoChangeCell` command to modify a `fixed' instance, the software issues a warning message.

Instead, you can specify the following using the `setEcoMode` command:

- `setEcoMode -honorFixedStatus false`, to modify FIXED instance.
- `setEcoMode -honorDontUse false` to modify instances which have dontUse attribute on cells.
- `setEcoMode -honorDontTouch false` to modify instance/nets which are marked dontTouch.

Notes:

- The `ecoChangeCell` command is simultaneous setup/hold and MMMC aware. To make the command simultaneous setup/hold aware, set the global variable `set_global timing_enable_simultaneous_setup_hold_mode` to `true`. The simultaneous setup/hold mode can only be used for timing analysis. It cannot be used for any physical implementation steps.
- When resizing instances inside multiple-instantiated blocks, the resize would be applied to all instantiations of the block.
- You can change a buffer to a pair of inverters using the `ecoDeleteRepeater` and `ecoAddRepeater` commands. You can change a buffer to an inverter by first using `setEcoMode -LEQCheck false` and then using `ecoChangeCell`. However, please note that it results in the change in logic.
- The `ecoChangeCell` command cannot be used in the post-mask ECO. Please use l`oadECO <ecofile> -postMask` to change cells in post-mask ECO.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `ecoChangeCell`. For a detailed description of the command and all of its parameters, use the man command: `man ecoChangeCell`
`-orient {R0 | R90 | R180 | R270 | MX | MX90 | MY | MY90}`	Specifies the legalized orientation for the instance. You can specify this parameter with the `-loc` parameter only.
`-inst instNames`	Specifies the names of the instances to be changed.
`-loc {xLoc yLoc}`	Specifies the legalized location where the instance will be placed.
`-upsize`	Upsizes the specified instance with the next available cell in the same footprint unless the specified cell is already of the highest driving strength. Upsizing an instance that drives a large load can improve the driver delay and the transition time at the receivers.
`-downsize`	Downsizes the specified instance with the next available cell in the same footprint unless the specified cell is already of the lowest driving strength. Downsizing an instance on the noncritical path can reduce the loading of its driver on the critical path.
`-cell list_of_cells`	Specifies the name of a specific cell(s) to be substituted in the design. The function of the original cell has to match the function of the new cell for all the outputs used by the old cell. The original master cell and the new master cell must have exactly same number of pins.
`-pinMap oldpin1 newpin1 oldpin2 newpin2`	Specifies the pin mapping for the new cell based on the old cell. This option is required if the new master cell has different pin names than the original cell. The net connected to *oldpin1* of the original cell should be connected to *newpin1* of the new cell, and so forth.

Examples

- Upsizes the specified instance:
  ```
  ecoChangeCell -inst Top_I18 -upsize
  ```

- Downsizes the specified instance:
  ```
  ecoChangeCell -inst Top_I18 -downsize
  ```
- Modifies the specified instance according to the specified cell:
  ```
  ecoChangeCell -inst Top_I18 -cell BUFX20
  ```
- Changes to another cell with a different pin name:
  ```
  ecoChangeCell -inst Top_I27 -cell TXOR2X2 -PINMAP A X B Y Y Z
  ```
- Downsizes the specified instance:
  ```
  ecoChangeCell -inst Top_I18 -downsize
  ```

Related Information

- ecoAddRepeater
- ecoDeleteRepeater
- setEcoMode
- timing_enable_simultaneous_setup_hold_mode

ecoCompareNetlist

```
ecoCompareNetlist
[-help]
-def referenceFileName
[-logical]
[-outfile fileName]
[-outputPhysicalInstInFile physicalInstFile]
[-referenceData {inMemory | external}]
```

Compares a specified netlist with the design in the database for structural or logical equivalence, and creates a file containing the differences found.

`ecoCompareNetlist` supports bus top-level port comparison.

ecoDisableNetRenamingForFlatNetlist

Global variable `ecoDisableNetRenamingForFlatNetlist` is introduced. If it is set to `1`, there would not be any net renaming for flat netlist. Net renaming will take place if net name is hierarchical and the terms it connects to do not fall under the same hierarchy or lies up in the hierarchy. Do `set ecoDisableNetRenamingForFlatNetlist 1` before the `ecoCompareNetlist` command to disable net renaming for flat netlist.

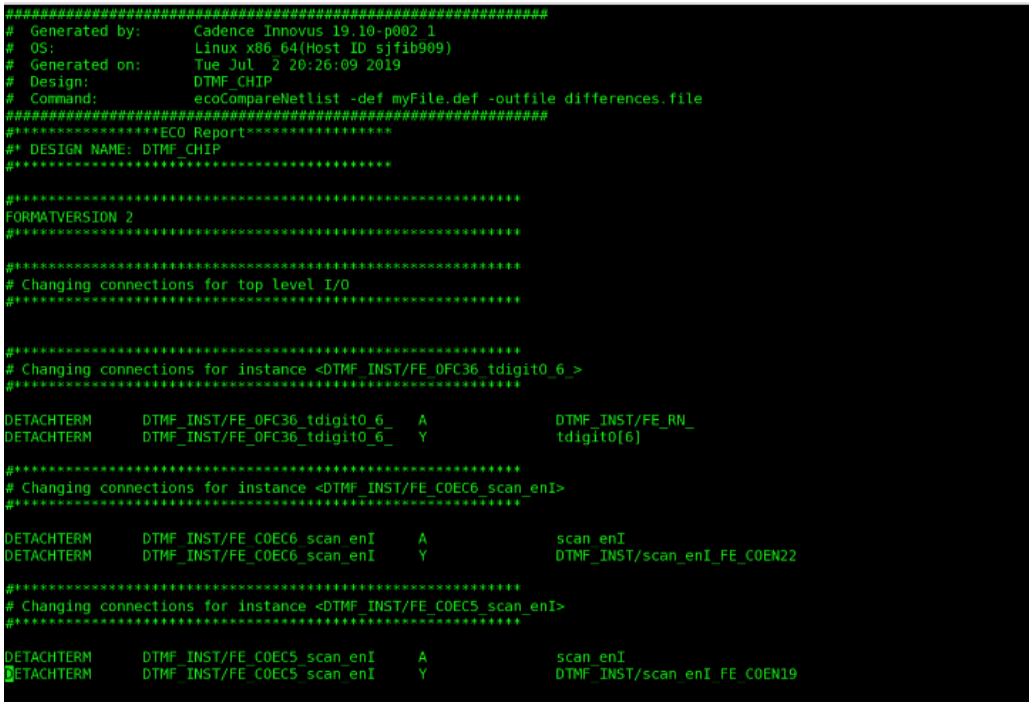
Parameters

`-help`	Prints out the command usage.
`-def referenceFileName`	Specifies the name of the DEF file that Innovus uses to compare with the design.
`-referenceData [inMemory | external]`	Specifies the data from where the post-ECO netlist is to be compared. • `inMemory` compares the post-ECO netlist to the netlist in memory. `inMemory` is the default behavior. • `external` compares the post-ECO netlist to an external netlist.
`-logical`	Compares the netlists for logical equivalence.
`-outfile fileName`	Specifies the file that contains the differences between the specified DEF file and the design in the database. If the directory specified to save the file does not exist, the tool creates the directory. For example, in the following command if `a/` does not exist, the tool will create it: `ecoCompareNetlist -def test.def -outfile ./a/test.rpt`
`-outputPhysicalInstInFile physicalInstFile`	When this option is specified, physical instance eco command is also written to eco file, and the added physical instances names one per line are dumped into the `physicalInstFile` file.

Example

- The following command compares the `myFile.def` file against the design, and writes the results to `differences.file`:

```
ecoCompareNetlist -def myFile.def -outfile differences.file
```



Related Information

- `ecoDisableNetRenamingForFlatNetlist`

ecoDefIn

```
ecoDefIn
[-help]
[-postMask [-suffix suffixName]]
[-useGACells GACoreSite]
[-reportFile fileName]
[-cleanFEData]
[-keepInstLoc]
[-mergeFloatingRouteToPGNet netName]
[-ecoDef {new_DEF_fileName}]
filename
```

Restores physical information from an old design and compares this information with the design in memory. This command uses the specified DEF file in the following ways for pre-mask and post-mask ECO:

- Used in pre-mask mode (without the `-postMask` parameter) this command behaves as follows:
  - New cells and nets
    Cells that exist in memory but not in the DEF file become unplaced.
  - Deleted cells or nets
    Cells and nets that exist in the DEF file but not in memory are discarded.
  - Modified nets
    Nets found both in the DEF file and in memory, but whose connections are different, are marked as "modified" for further processing during `ecoRoute`.
  - Matched cells/nets
    Nets and cells that exist both in the DEF file and in memory, and have the same connections, are placed as described in the DEF file. This includes the soft matching capability used in `defIn`.

Soft matching happens when a DEF net name does not have an equivalent name and another net is found in memory, that has the same connections as described in the DEF file.

The most common case for soft matching is when nets have multiple aliases in a hierarchical design `"net1" = "inst1/net2" = "inst1/inst2/net3"` and so on. Any of these net names can be used in the DEF and can have the same connections.

Without soft matching, net `a', for example, is removed and net `b' is created in it's place, resulting in ripping of the wire.

- In post-mask mode (with the -postMask parameter), the software can only change nets, not cells. This command behaves as follows:
  - New added cells and nets
    Cells that exist in memory but not in the DEF file become unplaced. Later, these cells will be mapped to spare cells when you use `ecoPlace-useSpareCells`.
  - Deleted cells
    Cells and instances that exist in DEF but not in memory are added to a spare cell list in Innovus: They are not discarded. Deleted cells are renamed with their original name and the suffix you specify. The default suffix is `_SPARE`. For example, `cellA` is renamed `cellA_SPARE`. The software retains deleted nets and their routing, which `ecoRoute` reroutes later.
  - Deleted nets
    Nets that exist in DEF but not in memory are kept and processed later when you use the `ecoRoute` command.
  - Modified nets
    Same behavior as in the pre-mask mode.
  - Matched cells and nets
    Same behavior as in the pre-mask mode.
  - Physical cells
    Cells that exist in the DEF file but not in memory, which are marked with `+SOURCE DIST` in the DEF file, are restored.

If you specify `-useGACells`, this implies a post-mask mode. This option behaves as follows:

- For cells that match the specified GA core site:
  - New added GA cell that exists in memory but not in the DEF file are left unplaced.
  - New GA cells that exist in the DEF file but not in memory are deleted.
- For cells that do not match the specified GA core site, the software treats them as standard cells as described in `-postMask` mode.

If you specify a report filename, this command reports the following information:

- New cells and nets found in memory but not in the DEF file
- Deleted cells and nets found in DEF but not in memory
- Nets whose connection does not match
- Cells and nets that match in Innovus and DEF

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `ecoDefIn`. For a detailed description of the command and all of its parameters, use the man command: `man ecoDefIn`
`-cleanFEData`	Specifies that the routed data is DRC clean. ECO routing will skip routing analysis.
`-ecoDef {new_def_fileName}`	Specifies the name of a partial DEF file. The tool compares the original routed DEF file or Innovus database with this file and marks the changed instances, changed location of the existing instances, and changed segments or wires.
fileName	Specifies the name of the routed/original DEF file to load.
`-keepInstLoc`	Applies the placement status and location of the instance in the DEF file to the instance in db. This option is only applicable in the following special situation: When an instance in DEF file and an instance in db have the same name but the master cell of the instance in the DEF and in the db do not match, then the instance in DEF is considered as deleted while the instance in db is added. In this case, if the option is not specified, the general rule for addition or deletion of an instance is applied as described earlier. For the added instance, the placement status is set to unplaced. If the option is specified, the placement status and location of the instance in the DEF file is applied to the instance in db.
`-mergeFloatingRouteToPGNet` *netName*	Merges the routing data to the specified PG net. In this case, the physical-only net will not be created. When this parameter is not specified, if a net is deleted in the new netlist but exists in the DEF file, then the `ecoDefIn` command discards the net in pre-mask ECO and creates a physical-only net in post-mask ECO to preserve the routing data on the net.
`-postMask`	Runs this command in post-route mode.
`-reportFile` *fileName*	Specifies the name for the report generated by the `ecoDefIn` command. *Default:* `ecoDefIn.rpt`
`-suffix` *suffixName*	Appends the specified suffix to cells that appear in the DEF file but have been deleted in the new netlist. *Default:* `_SPARE`
`-useGACells` *GACoreSite*	Specifies the GA Core Site.

Examples

The following example reads a DEF file, `myOld.def`, for post-mask ECO.

```
ecoDefIn -postMask -suffix _SPARE -reportFile ecoDefIn.rpt myOld.def
```

When the software converts a deleted instance to a spare cell, it adds the `_SPARE` suffix to the instance. This command generates a report `ecoDefIn.rpt`.

The following example shows a report file:

```
DELINST DTMF_INST/TDSP_CORE_INST/PROG_BUS_MACH_INST/i_9649 INVX1
(converted to spare cell
DTMF_INST/TDSP_CORE_INST/PROG_BUS_MACH_INST/i_9649_SPARE)

ADDINST DTMF_INST/TDSP_CORE_INST/PROG_BUS_MACH_INST/i_9649 BUFX8
MODIFYNET dtmf_chip_TIELO_Y_r2c1
ADDNET DTMF_INST/UNCONNECTED_000
ADDNET DTMF_INST/UNCONNECTED_001
ADDNET DTMF_INST/TDSP_CORE_INST/UNCONNECTED_000
ADDNET DTMF_INST/TDSP_CORE_INST/UNCONNECTED_009
ADDNET DTMF_INST/TDSP_CORE_INST/UNCONNECTED_010
```

Related Information

- ecoPlace
- ecoRoute

ecoDeleteRepeater

```
ecoDeleteRepeater
[-help][
-logicalChangeOnly]
{ -inst {list_of_instances} | -invPair {{inv1 inv2} {inv3 inv4} ...} }
```

Deletes a buffer or pair of inverters connected back-to-back. This command merges wires after ECO.

This command does not modify `dont_touch` nets if `setEcoMode -honorDontTouch` is set to `true`.

Notes:

- The `ecoDeleteRepeater` command is simultaneous setup/hold and MMMC aware. To make the command simultaneous setup/hold aware, set the global variable `set_global timing_enable_simultaneous_setup_hold_mode` to `true`. The simultaneous setup/hold mode can only be used for timing analysis. It cannot be used for any physical implementation steps.
- The `ecoDeleteRepeater` command cannot be used in the post-mask ECO. Please use `loadECO <ecofile> -postMask` to delete repeaters in the post-mask ECO.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `ecoDeleteRepeater`. For a detailed description of the command and all of its parameters, use the man command: `man ecoDeleteRepeater`
`-inst` *`list_of_instances`*	Specifies a buffer or inverter to delete. If you specify an inverter, the command finds another inverter on the net and deletes the back-to-back pair in order to preserve the logical equivalence. Instances are deleted only if they are tied to each other. To specify a pair of instances, enclose the instances in braces. `-inst` {*`inst1 inst2`*}
`-invPair {`*`inv1 inv2`*`} {`*`inv3 inv4`*`} ...`	Specifies pairs of inverters to be deleted/evaluated for deletion. You can specify a list of inverter pairs to be deleted.
`-logicalChangeOnly`	Specifies that buffer/inverter deletion should be done in the logical-only mode.During buffering, this would be useful when the design is unplaced, the location of new buffer is not available, and all terms of the net-being-buffered are unplaced. If you specify `-logicalChangeOnly` option, the tool will disable placement/legalization of the new buffer and proceed with buffering. During deletion, the tool will add a default resistance to replace the removed buffer/inverter, instead of calculating the accurate resistance based on locations.

Note: You can specify the following using the `setEcoMode` command:

- `setEcoMode -honorDontUse false` to modify instances which have dontUse attribute on cells.
- `setEcoMode -honorDontTouch false` to modify instance/nets which are marked dontTouch.

Examples

- The following command deletes the specified buffer, `A/inst1`:
  `ecoDeleteRepeater -inst A/inst1`
- The following command deletes a back-to-back inverter pair containing `A/inv1`:
  `ecoDeleteRepeater -inst A/inv1`
- The following command deletes instances `DTMF_INST/RESULTS_CONV_INST/FE_PHC845_r1644_2_` and `DTMF_INST/RESULTS_CONV_INST/FE_PHC845_r1643_3_`:
  `ecoDeleteRepeater -inst {DTMF_INST/RESULTS_CONV_INST/FE_PHC845_r1644_2_ DTMF_INST/RESULTS_CONV_INST/FE_PHC845_r1643_3_}`

Related Information

- ecoAddRepeater
- loadECO
- setEcoMode
- timing_enable_simultaneous_setup_hold_mode

ecoDesign

```
  ecoDesign
[-help]
sessionDirectoroy
design
newNetList
[-allow_to_load_binary_timing_file]
[-addHierPortForTieOff]
[-cpfFile newCPFFile | -1801File new1801File]
[-def defFile]
[-ecoMMMC newMMMCFile]
[-ecoSDC newSDCFile]
[-keepInstLoc]
[-lef_files LEFFilesName]
[-noAddFiller]
[-skipCcopt]
[-fillerPrefix prefix | -noDeleteFiller]
[-modifyOnlyLayers MLb:MLt]
[-noEcoPlace]
[-noEcoRoute]
[-noTiming]
[-place_detail_use_GA_filler_groups]
[-postMask]
[-reportFile reportFile]
[-spareCells spareCellName]
[-suffix suffix]
[-tieCell cellNameList]
[-useGACells GACoreSite | -useGAFillerCells list_of_filler_cells ]
```

Takes an Innovus database and a modified netlist as input and performs ECO operations. It restores the design, examines the changes in the new netlist, and automatically implements the required changes with `ecoPlace` and `ecoRoute`.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `ecoDesign`. For a detailed description of the command and all of its parameters, use the man command: man `ecoDesign`
`-1801File` *new1801File*	Specifies the name of the new 1801 file.
`-allow_to_load_binary_timing_file`	Allows the tool to load the binary timing graph or the timing constraint file. If you execute `ecoDesign` based on binary timing graph or timing constraints, the tool will give an error and stop loading the new design. This is because the timing graph or the timing constraints file from the loaded design is a binary file, and if you keep on loading it, it may cause fatal error. In such a case, you need to restore the original design and then save a new design with the no-binary timing file. Then, execute `ecoDesign` based on the new design. If you still want to load the current design, you need to run `ecoDesign -allow_to_load_binary_timing_file 1` to turn on the control.
`-addHierPortForTieOff`	Allows the added tie cells to connect to the tie pins across hierarchical boundaries if other constraints, such as maximum fanout and maximum distance, allow it. This parameter can be used only with `-postMask` and `-tieCell` options.
`-cpfFile` *newCPFFile*	Specifies the name of the CPF file.
`-def` *fileName*	Specifies the DEF file you want the software to load automatically when performing ECO. This parameter automatically assigns the location of the new ECO cells. As a result, you do not need to run `ecoPlace` after performing the ECO.
design	Specifies the top cell name of the design.
`-ecoMMMC` *newMMMCFile*	Specifies the name of the new view definition file.
`-ecoSDC` *newSDCFile*	Specifies the name of the new SDC file.

`-fillerPrefix prefix`	Specifies the filler cell instance prefix. The `ecoDesign` command removes all filler cells before placing the ECOs. If you do not specify a prefix, `deleteFiller` removes filler instances based on the `setFillerMode` command settings. If you have not set `setFillerMode`, then no filler cells are removed.
`-keepInstLoc`	Keeps the instance location of changed cells.
`-lef_files LEFFilesName`	Specifies the LEF files name for loading. Use this option if you want to replace the existing LEF files in the original design with new LEF files. Refer to `restoreDesign -lef_files` for more details and limitations. For example, tech.lef, A.lef are the existing LEFs used in original design. You can change A.lef to A_v2.lef using the following command: `ecoDesign original.enc.dat myDesign myDesign.new.v -lef_files {tech.lef A_v2.lef}`
`-noAddFiller`	Skips filler insertion in the pre-mask eco flow.
`-modifyOnlyLayers MLb:MLt`	Specifies the layers to be modified during the ECO process.
newNetList	Specifies the name of new netlist file.
`-noDeleteFiller`	Skips deleting filler.
`-noEcoPlace`	Prevents placement and routing during the ECO process. `ecoPlace` and/or `ecoRoute` must run manually based on the design's needs.
`-noEcoRoute`	Prevents routing during the ECO process.
`-noTiming`	Ignores timing libraries and MMMC configuration
`-place_detail_use_GA_filler_groups`	Specifies that filler cell instances will only be replaced by the logic instances that belong to the same GA groups. This option can only be used when there is a filler group created by `createGAFillerGroup`. `ecoDesign -place_detail_use_GA_filler_groups` first finds the optimal location for each instance based on its connectivity, then searches for the nearest GA filler cell that is included in the same filler group, places a GA cell at the GA filler location, and deletes the original GA filler. If the GA cell is smaller than the GA filler, the placer creates a new GA filler instance using the list of GA filler cells in the same filler group and places the filler in the gap.
`-postMask`	Specifies that the changes are being made on a post-mask design. `-postMask` is required when option `-suffix`, `-spareCells`, `-modifyOnlyLayers`, `-tieCell`, or `-addHierPortForTieOff` is specified.
`-reportFile reportFile`	Specifies the name of the report file. **Note**: By default, `ecoDesign` creates the default `ecoRestoreDesign.rpt` file. However, this default file will be overwritten by the file name specified using the `-reportFile` option in next eco RUN.
sessionDirectory	Specifies the .dat directory from which to load the files.
`-skipCcopt`	Skips restore ccopt config for `signoffTimeDesign`.
`-spareCells spareCellName`	Specifies the name of the spare cell to use during the ECO process.
`-suffix suffix`	Specifies the suffix to be appended to the spare cells. You can specify this parameter only if you specify `-postMask`. *Default:* `_SPARE`
`-tieCell "cellName1 cellName2"`	Specifies the existing tie cells to be reused to make tie connections for newly created SPARE instance in the design. You can only specify a maximum of two tie-cells, where one cell must be a tie-high driver, and the other a tie-low driver. The two tie-cell names must be surrounded by quotation marks. This parameter can be used only with `-postMask` option.
`-useGACells GA_site`	Specifies the site name of the GA cell. Check the site name defined in the LEF file.
`-useGAFillerCells list_of_filler_cells`	Specifies a list of Gate Array (GA)-style filler cells that the tool can replace with GA cells during post-mask ECO. You can only use this parameter in a post-mask ECO flow. **Note:** You cannot specify the `-useGACells` parameter if you use this parameter.

Examples

- The following command performs a pre-mask ECO:
  ```
  ecoDesign original.enc.dat myDesign myDesign.new.v
  ```
- The following command and options are used to implement a post-mask ECO:
  ```
  ecoDesign -postMask -modifyOnlyLayers 2:3 -spareCells *spare* original.enc.dat myDesign myDesign.new.v
  ```

Related Information

- `restoreDesign`
- ECO Flows

ecoOaDesign

```
ecoOaDesign
[-help]
lib
cell
view
-ecoVerilogFile input_fileName
[-postMask [-suffix suffixName]]
[-useGACells GACoreSite]
[-reportFile output_fileName]
```

Imports an OpenAccess database and an ECO-modified netlist. Restores the physical objects in the database, examines the changes in the new netlist, and implements the required connectivity changes in the logical database without making changes to the physical objects. It preserves the spare cell markings in the old `lib`/`cell`/`view`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `ecoOaDesign` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man ecoOaDesign`.
`cell`	Specifies the OpenAccess cell name of the top cell to import.
`lib`	Specifies the OpenAccess library name.
`-ecoVerilogFile` *`input_fileName`*	Specifies the changed Verilog netlist. **Note:** The original netlist must be written out from the original *`lib cell view`* and it must be hand-modified to incorporate the required ECO changes.
`-reportFile` *`output_fileName`*	Specifies a report file that shows the additions and removals of instances and nets in the new Verilog netlist. *Default:* eco.log
`-postMask`	Specifies that `ecoOaDesign` works in the post-mask mode. If this parameter is not specified, then Innovus considers the mode as pre-mask by default.
`-suffix suffixName`	Specifies the suffix name for the the spare cells. By default, the suffix is _SPARE.
`-useGACells` *`GACoreSite`*	Specifies the GA core site. The `-useGACells` parameter is valid only in the post-mask mode. 1. For cells that match the specified GA core site: ◦ Newly added GA cells that exist in memory but not in the OpenAccess database are left unplaced. ◦ GA cells that exist in the OpenAccess database but not in the memory are deleted in the new database. 2. For cells that do not match the specified GA core site, Innovus treats them as standard cells as described in post-mask mode.
`view`	Specifies the OpenAccess view name to import.

Examples

The following command reads the modified Verilog from the `eco.v` file, complete physical pre-ECO database from the `lib`/`cell`/`view` `designLib`/`TOP`/`final` in the post-mask mode:

```
ecoOaDesign designLib TOP final -ecoVerilogFile eco.v -postMask
```

In post-mask mode, no physical database changes are allowed and the new logical changes coming through `eco.v` are incorporated using spare cells in successive `ecoPlace` and `ecoRoute` commands.

Related Information

- `ecoPlace`
- `ecoRoute`
- For more information, refer to the Innovus-Based ECO Flow section of the Chip Finishing and ECO Flows chapter of *Mixed Signal Interoperability Guide*.

ecoPlace

```
ecoPlace
[-help]
[-fixPlacedInsts {true|false}]
[-place_detail_use_GA_filler_groups {true|false}]
[-reportSpareCells fileName]
[-timing_driven {true|false}]
[-useGACells GACoreSite]
[-useGAFillerCells GAFillerCells]
[-useSpareCells {true|false}]
```

Incrementally places the unplaced standard cells. In a pre-mask flow, this command moves the unplaced standard cells into the core area. During an ECO loop, some new unplaced standard cells may get introduced. If the goal is to move the unplaced standard cells into the core area and place them without disturbing the original placement (pre-mask flow), it is recommended to run `ecoPlace` at this stage. In a post-mask flow, if you specify `-useSpareCells`, the software can map the unplaced cells to the available spare cells. When `-useGAFillerCells` is provided in a post-mask flow, the command can map existing unplaced cells to placed GAFillers in the design.

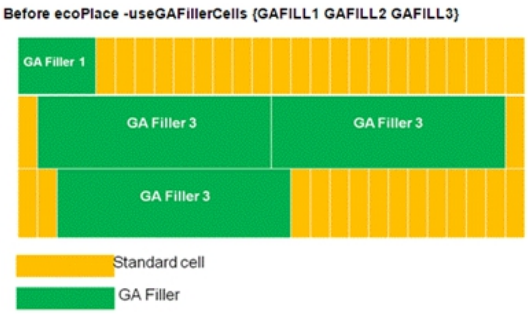
You can use this command after the `ecoDefIn` command when some newly added cells are still unplaced.

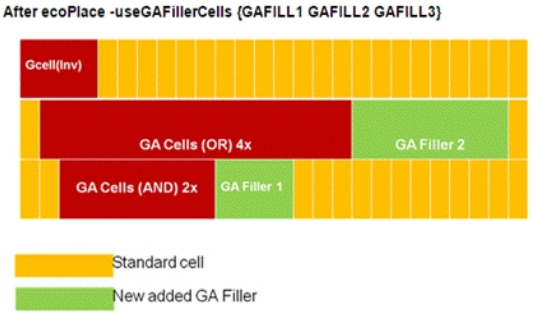
Parameters

`-fixPlacedInsts {true | false}`	
	Specifies that instances with the PLACED status are not moved.
`-help`	Outputs a brief description that includes type and default information for each `ecoPlace` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man ecoPlace`.
`-reportSpareCells` *`fileName`*	Specifies to report the spare gates that are selected to map the unplaced instances. This option works with –`useSpareCells` only.
`-timing_driven {true|false}`	Turns on the slack-driven placement for `ecoPlace`. *Default*: `false`
`-useGACells` *`GACoreSite`*	
	Selects and places the unplaced instances with techSite matching the provided GACoreSite. It is expected that there is free space to place the instances. If not, GA fillers could be deleted before `ecoPlace -useGACells` and re-inserted after the command. You can use this parameter only for post-mask ECO. **Note:** You cannot specify the `-useGAFillerCells` parameter if you use this parameter.
`-useGAFillerCells` *`GAFillerCells`*	
	Specifies a list of Gate Array (GA)-style filler cells that the tool can replace with GA cells during post-mask ECO. You can only use this parameter in a post-mask ECO flow. The new logic instances adding during a post-mask flow must already be present in the netlist. **Note:** You cannot specify the `-useGACells` parameter if you use this parameter.
`-place_detail_use_GA_filler_groups {true|false}`	
	Specifies that filler cell instances will only be replaced by logic instances that belong to the same GA groups. This option can only be used when there is a filler group created by `createGAFillerGroup`. `ecoPlace -place_detail_use_GA_filler_groups` first finds the optimal location for each instance based on its connectivity, then searches for the nearest GA filler cell that is included in the same filler group, places a GA cell at the GA filler location, and deletes the original GA filler. If the GA cell is smaller than the GA filler, the placer creates a new GA filler instance using the list of GA filler cells in the same filler group and places the filler in the gap. If there is only one filler group created, it equals to `ecoPlace - useGAFillerCells`. *Default*: `false`
`-useSpareCells {true | false}`	
	Maps unplaced cells to spare cells during placement. First, checks whether all unplaced cells match spare cells. Next, chooses a spare cell and changes the connections from the unplaced cell to the selected spare cell. Finally, deletes the unplaced cell. You can use this parameter only for post-mask ECO. This option observes the power domain when selecting the spare cell and maps an unplaced cell to the spare cell in the power domain, based on the instance name of the cell. For example, if module A/B is defined for power domain PD2, this option will map an unplaced cell A/B/inst1 to a spare cell in module A/B. *Default*: `false`

Examples

- The following command performs incremental placement:
  ```
  ecoPlace -useSpareCells true
  ```
- The flowing command places the GA-ECO cells on GACORE site.
  ```
  ecoPlace -useGACells GACoreSite
  ```
- The following command places GA-ECO cells on the specified GA-Fillers and fills remaining gaps with GA-Fillers.
  ```
  ecoPlace -useGAFillerCells {GAFiller1 GAFiller2 …}
  ```





- The following commands create GA Filler group that associates GA-Function cells with GA-Fillers. Then, `ecoPlace` will place GA-ECO cells on same group GA-Fillers.
  ```
  createGAFillerGroup -group GROUP1 -ga_cells {GA_Cell1 GA_Cell2… } -ga_fillers {GA_Filler1 GA_Filler2…}
  ```
  ```
  ecoPlace -place_detail_use_GA_filler_groups true
  ```

Related Information

- `createGAFillerGroup`
- `ecoDefIn`
- `ecoOaDesign`

ecoRemoveTiedInputs

```
ecoRemoveTiedInputs
[-help]
[-base_cells cell_list]
[-ignore_base_cells cell_list]
[-ignore_insts inst_list]
[-insts inst_list]
[-relaxed_placement]
[-term term_list]
[-verbose]
[-report]
```

Remaps or restructures gates that have inputs connected to the same net. These gates are remapped into functionally equivalent logic such that none of the new gates has inputs tied together. Usually, such optimization is good in all objectives. For example, a 4-input OR gate with 3 of its inputs tied together (implementing function a+b+b+b) can safely be replaced with a 2-input OR gate (implementing a+b). This move is likely to be better in terms of area, power, and delay. However, in case of complex gates, it may not be possible to restructure a gate in a way that improves delay, but it may be possible to decompose it into smaller discrete gates, none of which has inputs tied together. Area and/or delay may be worse, but in some libraries, it is preferable to not have instances with inputs tied together. The `ecoRemoveTiedInputs` command fixes any gate with two or more inputs connected to the same net regardless of the improvement in delay/area/power. It tries to find improving moves, but it may accept moves that worsen delay/area/power.

The command can also insert buffers in nets tying inputs together. Inserting buffers should be used for the gates that cannot be automatically fixed or that are preserved. You need to specify which buffer cell(s) must be inserted before which input(s). There must be as many buffer cells specified as there are inputs specified. The only exception is specifying a single buffer cell; in that case it is understood that all inputs will be untied by using instances of that same buffer cell.

The command honors `setOptMode` settings for timing update and legalization. It does not call `ecoRoute` if run in the postRoute/ECO stages.

Note: MBFF cells are ignored by default as they always have tied inputs.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `ecoRemoveTiedInputs` parameter. For a detailed description of the command and all of its parameters, use the man command: `man ecoRemoveTiedInputs`
`-base_cells` *`cell_list`*	Specifies the buffer cells to be used for fixing.
`-ignore_base_cells` *`cell_list`*	Specifies to ignore instances of these cells from fixing.
`-ignore_insts` *`inst_list`*	Specifies to ignore these instances from fixing.
`-insts` *`inst_list`*	Specifies to fix only these instances. *Default*: All design minus ignored instances/cells.
`-relaxed_placement`	Accepts moves even if instances cannot be placed. You do not need to set this option if `ecoRemoveTiedInputs` can successfully fix all violations. In some cases, for example if an instance with tied inputs is in a very congested area, the tool may not be able to place the new instances. In such cases, you can use the `-relaxed_placement` option to fix such instances. However, the final placement may not be legal, and the new instances will likely overlap. For this, you can perform incremental placement to legalize the placement.
`-term` *`term_list`*	Specifies the terms to be buffered.
`-verbose`	Specifies to increase the detail of reports and fixing.
`-report`	Reports existing tied inputs, but does not provide fixing details by any method.

Related Information

- `setOptMode`
- `ecoRoute`

ecoSwapSpareCell

```
  ecoSwapSpareCell
[-help]
instName
spareCellInstName
[-keepScan]
[-noScanTrace]
[-preservePin {list_of_spare_cell_pins}]
[-suffix string]
```

Swaps an unplaced or placed cell with a cell in the spare cell list.

You can specify this command only after you have run `specifySpareGate`.

This command can swap cells of the same cell type (same footprint and identical pin name). Cells of different sizes can also be swapped. The spare cell is connected according to the specified global net connections.

If the cell to be swapped is unplaced, it is mapped to the spare cell. *instName* is deleted, and its connection is transferred to the spare cell. If the cell to be swapped is placed, it is swapped with the spare cell and is renamed to *instNameSuffix* if the `-suffix` option is used. If a suffix is not specified, the *instName* cell is renamed to *spareCellInstName*. The *instName* cell's connections are transferred to *spareCellInstName*. The input of *instName* is `tielo`, based on the global connection definition.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `ecoSwapSpareCell` parameter. For a detailed description of the command and all of its parameters, use the man command: `man ecoSwapSpareCell`
`-keepScan`	Keeps the scan connection for both the spare cell and the specified instances. Before you can use this parameter, you must first identify the scan connections by running a scan trace.
instName	Specifies the name of the cell to swap.
`-noScanTrace`	Disables scan tracing while swapping.
`-preservePin {list_of_spare_cell_pins}`	Specifies a list of pins whose connection you want to preserve while swapping spare cells. The same pin on both the spare cell and specified instances is preserved. If both the instance and the spare cell are placed instances, the `-preservePin` parameter is effective for both cells. If the instance is unplaced, `-preservePin` only affects the spare cell because the instance is deleted.
spareCellInstName	Specifies the spare cell in the spare cell list to swap with the placed cell.
`-suffix` *string*	Specifies a string that is appended to the *instName* after swapping.

Examples

The following example swaps a placed cell, `DTMF_INST/TDSP_CORE_INST/PROG_BUS_MACH_INST/i_9649`, with spare cell `spare1`:

```
specifySpareGate -inst *spare*
ecoPlace -useSpareCells
checkPlace

ecoSwapSpareCell DTMF_INST/TDSP_CORE_INST/PROG_BUS_MACH_INST/i_9649 spare1
```

Related Information

- `checkPlace`
- `ecoPlace`
- `specifySpareGate`
- Interactive ECO chapter in the *Innovus User Guide*.
- ECO Flows chapter in the *Innovus User Guide*.

endECO

```
endECO
[-help]
```

Ends logging ECO operations.

Parameters

None

Examples

- The following commands show the usage of `initECO` and `endECO`.

  ```
  initECO FILE1
  ecoAddRepeater -net DTMF_INST/ARB_INST/n_36 -cell INVX4 -name {{cell1 cell2}} -newNetName {{net1 net2}}
  endECO
  vim FILE1
  ```

  All operations performed by ECO Commands are logged in *FILE1*.

```
#                                            #
#  Silicon Perspective, A Cadence Company    #
#  Innovus ECO/IPO Delta file                #
#                                            #
##############################################
# Created on Thu Jul  4 03:15:05 2019


FORMATVERSION 2

START_HECO
USE_UNDERSCORE_FOR_CONFLICT FALSE
SET_DEFAULT_NET_PREFIX FE_RN_
SET_DEFAULT_INST_PREFIX FE_RC_
SET_KEEP_FLOATING_NET FALSE
SET_ASSIGNLESS_PORT_TO_NET TRUE
CURRENT_INST DTMF_INST/ARB_INST
CREATE_INST cell1 INVX4
CONNECT i_21/Y cell1/A
DISCONNECT i_160/C0
CREATE_NET net1

CONNECT_NET net1 cell1/Y
CONNECT cell1/Y i_160/C0
END_HECO # 0
START_HECO
USE_UNDERSCORE_FOR_CONFLICT FALSE
SET_DEFAULT_NET_PREFIX FE_RN_
SET_DEFAULT_INST_PREFIX FE_RC_
SET_KEEP_FLOATING_NET FALSE
SET_ASSIGNLESS_PORT_TO_NET TRUE
CURRENT_INST DTMF_INST/ARB_INST
CREATE_INST cell2 INVX4
CONNECT cell1/Y cell2/A
DISCONNECT i_160/C0
CREATE_NET net2

CONNECT_NET net2 cell2/Y
CONNECT cell2/Y i_160/C0
END_HECO # 0
```

Related Information

- `initECO`

fillNotch

```
fillNotch
[-help]
[-area {x1 y1 x2 y2}]
[-report filename]
[-useExistingDrcResult]
[-useNonDefaultSpacing]
```

Detects and fills notches, gaps, and holes, and corrects acute angle (45 degree) violations.

Creates geometries in `DRCFILL` shapes as special wires attached to each net. Saves the geometries in DEF or GDSII Stream format. In a DEF file, the geometries are saved in the special net section. The software considers the geometries as active design objects.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `fillNotch`. For a detailed description of the command and all of its parameters, use the man command: `man fillNotch`
`-area {x1 y1 x2 y2}`	Specifies the area to check and fill. For example, `fillNotch -area {0 0 100 100}`. *Type:* Real
`-report filename`	Specifies the report file. The file includes the locations, types and number of violations corrected. For example, `fillNotch -report design_fill.rpt`. *Default:* *design*`.fill.rpt` *Type:* String
`-useExistingDrcResult`	Allows fixing the `convexCornerSpacing` issues based on Verify Geometry markers.
`-useNonDefaultSpacing`	Honors nondefault spacing rules when filling notches. This behavior matches the behavior of `verify_drc`; that is, when this parameter is specified, `fillNotch` honors both default and nondefault spacing rules. If this parameter is not specified, `fillNotch` honors default spacing rules but does not honor nondefault spacing rules.

Example

- The following commands check the notch issues and create drc markers with `verify_drc`, and adds drc fill shapes on the drc marker location:

  ```
  verify_drc
  fillNotch -useExistingDrcResult
  ```

Related Information

- `verify_drc`
- ECO Flows appendix to the *Innovus User Guide*

fix_multi_drivers

```
fix_multi_drivers
[-help]
[-exclude_net netName | netNameList]
[-replace_bus_bit]
[-report_only]
```

Fixes multi-drive issues by disconnecting driving ports that do not have a real driver. The fix_multi_drivers command also works for looped multi-driver assign net.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `fix_multi_drivers` parameter.
`-exclude_net` *netName* | *netNameList*	Specifies the list of nets to be excluded.
`-replace_bus_bit`	Replaces all bus ports with wrong directions, if any, by new scalar ports. If not specified, just report without such replacement.
`-report_only`	Reports, but does not fix, ports with wrong directions.

getEcoMode

```
getEcoMode
[-help]
[-addPortAsNeeded]
[-delayCalcEffort]
[-batchMode]
[-honorDontTouch]
[-inheritNetAttr]
[-LEQCheck]
[-prefixName]
[-refinePlace]
[-spreadInverter]
[-updateTiming]
[-honorDontUse]
[-honorFixedNetWire]
[-honorFixedStatus]
[-honorPowerIntent]
[-modifyOnlyLayers]
[-quiet]
```

This command displays the following information about the parameters for the corresponding `setEcoMode` command in the Innovus log file and in the Innovus console: name, possible values, type (Boolean, string, and so on), and source of current setting (default, user).

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `getEcoMode`. For a detailed description of the command and all of its parameters, use the man command: `man getEcoMode`
`parameter_names`	Returns information for the specified parameters. You can specify one or more parameters. **Note:** `-inheritNetAttr` is for Cadence internal use only.
`-quiet`	Returns the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software returns the current settings of all `setEcoMode` parameters in Tcl format.

Examples

- The following command returns the current setting for the `-LEQCheck` parameter:

  ```
  getEcoMode -LEQCheck
  ```

  The software returns the following information:

  ```
  -LEQCheck true # bool, default=true

  true
  ```

- The following command returns the parameters for all the `setEcoMode` command:

  ```
  getEcoMode
  ```

  The software returns the following information:

  ```
  -honorDontTouch true # bool, default=true

  -honorDontUse true # bool, default=true

  -honorFixedStatus true # bool, default=true

  -inheritNetAttr true # bool, default=true

  -LEQCheck true # bool, default=true
  ```

```
-prefixName {ECO} # string, default=ECO

-refinePlace true # bool, default=true

-spreadInverter true # bool, default=true

-updateTiming true # bool, default=true

{honorDontTouch true} {-honorDontUse true} {-honorFixedStatus true} {inheritNetAttr true} {LEQCheck true} {prefixName {ECO}}
{refinePlace true} {spreadInverter true} {updateTiming true}
```

- The following command returns the current setting for the `-spreadInverter` parameter in Tcl list format only:
  ```
  getEcoMode -spreadInverter -quiet
  ```
  The software returns the following information:
  ```
  true
  ```
- The following command returns the current settings for all `setEcoMode` parameters in Tcl list format only:
  ```
  getEcoMode -quiet
  ```
  The software returns the following information:
  ```
  {honorDontTouch true} {-honorDontUse true} {-honorFixedStatus true} {inheritNetAttr true} {LEQCheck true} {prefixName {ECO}}
  {refinePlace true} {spreadInverter true} {updateTiming true}
  ```

Related Information

- `setEcoMode`

group

```
group
[-help]
inst|hInst+
[[-module_name module_name -hinst_base_name hinst_base_name ]]
```

Creates a new module and hinst from a group of the specified hinsts or insts. You can only group hinsts and insts that belong to the same hierarchy (the same hinst parent). Returns the newly created hinst.

Parameters

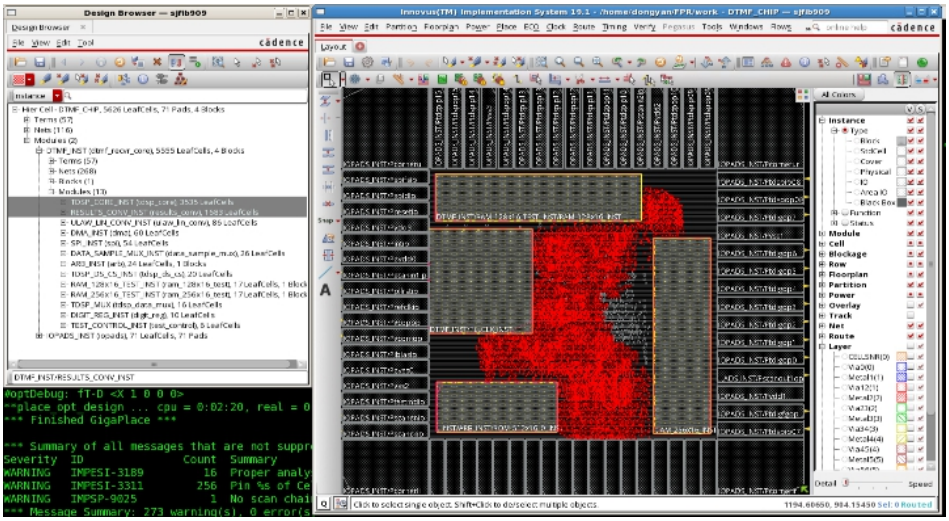
`-help`	Outputs a brief description that includes the type and default information for each `group`. For a detailed description of the command and all of its parameters, use the man command: `man group`
`inst|hInst+`	Specifies the hinst(s) or inst(s), in the same hierarchy, to be grouped.
`-hinst_base_name` *`hinst_base_name`*	Specifies the hinst base name of the new group instantiation.
`-module_name` *`module_name`*	Specifies the module name of the new group.

Example

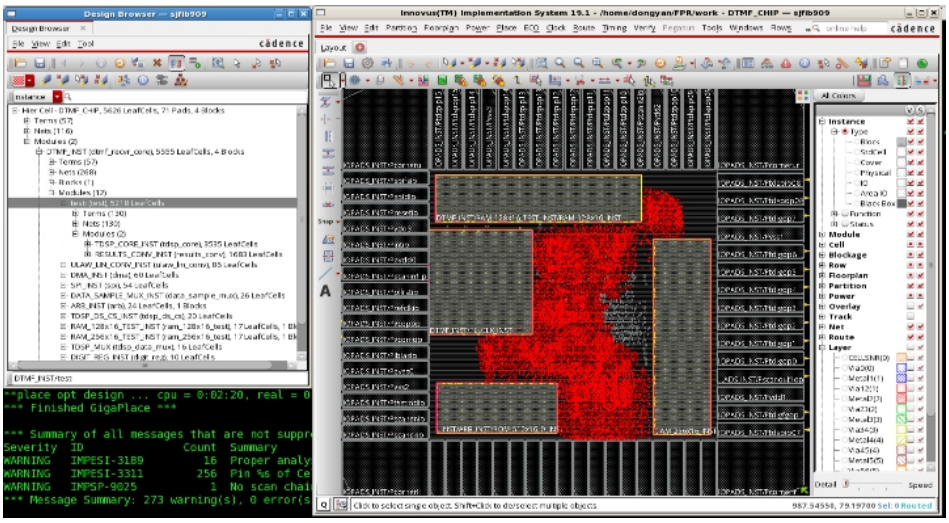
The following command groups two hinsts, `h1` and `h2`, at the top-level into a new module named `test`. The new hinst is named `testi` by default.

```
>group -module_name test -hinst_base_name baseName {DTMF_INST/TDSP_CORE_INST DTMF_INST/RESULTS_CONV_INST}
```

Before:



After:



Related Information

- group_instance_suffix
- ungroup

initECO

```
initECO
[-help]
ecoFileName
```

Logs ECO operations in a log file.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `initECO` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man initECO`.
`ecoFileName`	Specifies the name of the ECO log file.

Examples

- The following commands show the usage of `initECO` and `endECO`..

  ```
  initECO File1
  ....
  <ECO commands>
  ....
  endECO
  ```

  All operations performed by ECO Commands are logged in *`File1`*.

Related Information

- `endECO`
- `loadECO`

loadECO

```
loadECO
[-help]
[-postMask | -useGACells GACoreSiteName]
[-suffix suffix]
[-verbose]
fileName ...
```

Reads a file containing ECO directives and applies the changes to the current netlist.

You can use this command at any time after you import the design.

This command also performs consistency checks while the ECO commands are applied and after the changes are complete. It also checks for and observes power domains. This command produces results compatible with `ecoDefIn`, so you will use `ecoPlace` and `ecoRoute` to complete the ECO process.

The ECO directives do not affect routing, except in the following cases:

- `DELETENET` deletes routing attached to a specified deleted net.
- `DELETEINST` deletes an instance but leaves the wires attached to the instance dangling. You can then repair dangling wires with the router after ECO is complete.

If you want to reroute the entire design, delete routing before running the `loadECO` command.

This command supports ECOs for gate array designs through the `-useGACells` and `-postMask` parameters.

If an error occurs when the `loadECO` command reads the ECO file, the `loadECO` command stops without reading more commands from the file, and writes error messages to the `innovus.log` file.

Note: The first line of the ECO file you read must be the following:

```
FORMATVERSION 2
```

For information on the commands you can include in the ECO command file, see "ECO Directives," in the *Innovus User Guide*.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `loadECO`. For a detailed description of the command and all of its parameters, use the man command: `man loadECO`
`fileName`	Specifies the name of the file that contains the ECO directives. You can specify more than one file. File names must be separated by one space.
`-suffix`	Specifies the suffix this command adds to spare cells created by deleting instances. The default suffix is `SPARE`.
`-useGACells` *`GACoreSite`*	Uses gate array cells when adding or deleting instances. This parameter applies to gate array designs only. If `ADDINST` is specified in the ECO file, the default behavior for `ADDINST` applies. If `DELETEINST` is included in the ECO file, the software deletes an instance if the instance master uses the same site as the specified *`GACoreSite`*, which can be the name of any gate array site defined in the LEF file. If you do not specify this parameter, `DELETEINST` removes the instance from the netlist.
`-postMask`	When adding an instance, the new cell will be unplaced. The unplaced cell will map to the spare cell during placement with the `ecoPlace -useSpareCells` command. When the software deletes an instance, it renames the instance to an instance name at the original level of hierarchy, and appends the specified suffix. If you do not specify the `-suffix` parameter, this parameter appends `SPARE`.
`-verbose`	Provides a report for nets without drivers and the instance input pins with no net connections.

Example

- The following command reads `myfile` and runs the ECO commands it contains:
  ```
  loadECO myfile
  ```
- The `loadECO` command issues a summary report on completion, for example:

```
***************** Load ECO Summary *****************

    Number of New Nets:                          0
```

```
Number of New Instances:                        0
Number of New Ports:                            0
Number of New Assigns:                          0
Number of New Feedthroughs:                     0
Number of Changed Connections:                  0
Number of Changed Instances:                    0
Number of Deleted Nets:                         0
Number of Deleted Instances:                    1
Number of Deleted Ports:                        0
Number of Deleted Assigns:                      0
Number of Deleted Feedthroughs:                 0
```

- The following ECO summary is the output of the loadECO command with -verbose options:

```
***************** Load ECO Summary *****************
Number of New Nets:                               0
Number of New Instances:                          1
Number of New Ports:                              0
Number of New Assigns:                            0
Number of New Feedthroughs:                       0
Number of Changed Connections:                    0
Number of Changed Instances:                      0
Number of Deleted Nets:                           0
Number of Deleted Instances:                      0
Number of Deleted Ports:                          0
Number of Deleted Assigns:                        0
Number of Deleted Feedthroughs:                   0
Nets With No Driver:
        n_23
        u_counter_1/cnt[30]
Number of Nets With No Drivers:                  2
Input Instance Pins With No Net Connection:
        A of inst newInst
        TE of inst u_counter_2/ECO_reg_iflg_reg
Number of Input Instance Pins With No Net Connection:   2
```

Related Information

- endECO
- For an example of an ECO file, see "Example ECO File" in the ECO Directives chapter of the Innovus User Guide.

remove_assigns

```
remove_assigns
[-help]
[-buffer bufName]
[-buffering [-buffer_tie_assigns ][-includeIdealNets ] [-includeClockPath]]
[-honordonttouch]
[-ignorePortConstraints]
[-net netName [-netBuffer netBufName]]
[-prefix bufNamePreFix]
[-report]
```

Removes assign statements from the Verilog as much as possible while keeping the same DEF netlist. This means that the placement and routing remain the same and timing is not disturbed.

The `remove_assigns` command removes all non-top module assign nets without adding buffers.There are cases where you do not want assign nets in your design. However, there are some assign nets in the original net list imported into Innovus, or you have a design with some unwanted assign nets. In such cases, you can use `remove_assigns` to remove all assigns or replace assigns with proper buffers that are automatically selected by the tool.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `remove_assigns`. For a detailed description of the command and all of its parameters, use the man command: `man remove_assigns`
`-buffer bufName`	Specifies the name of the buffer for all assign nets, disables automatic buffer selection.
`-buffer_tie_assigns`	Allows to buffer tie assigns.
`-buffering`	Specifies whether the assign net buffering feature is needed. If only “-buffering” is specified, all assigns, including top and non-top module assigns, will be replaced by buffers. The buffers for each module will be automatically selected to meet all power domain requirements with the minimum area cost.
`-honordonttouch`	Honors dont_touch. This setting prevents assign removal on modules which have preserve/dont_touch settings.
`-ignorePortConstraints`	Allows to ignore all port constraints during assign removal.
`-includeClockPath`	Removes assigns on clock path.
`-includeIdealNets`	Removes assigns on ideal nets.

`-net` *`netName`*	Specifies the name of net (s). Supports removing or buffering assigns of a list of assign nets,
	For example, the following command removes all assigns of assign nets n1, n2 and n3.
	`remove_assigns -net {n1 n2 n3}`
	
	The following command replaces all assigns of assign nets n1, n2 and n3 with buffers.
	`remove_assigns -buffering -net {n1 n2 n3}`
	
	If “-net” is used, “-buffering” must also be used for buffering those assign nets specified in *`netName`*. Otherwise, `remove_assigns` will work same as no options are specified (as default).
	Whenever option “-net” is used, *`netName`* must be provided. Otherwise, a warning message is issued.
	
	You can specify any of the following as the *`netName`*:
	• Single net name
	• Multiple net names separated with space
	• Single wildcard name (all matched assign nets will be collected for buffering)
	• Multiple wildcard names separated with space (all match assign nets will be collected for buffering)
	• Single bus name
	• Multiple bus names
	Note: For each bus, all or portion of bus bits can be assign nets and the others can be non-assign nets. The buffering code replaces only those assign bus bits while leave those non-assign bus bits unchanged.

`-netBuffer` *`netBufName`*	Specifies the name of the buffer for the specific net given in the `-net` option. This option, if specified, should come after the `-net` option.
`-prefix` *`bufNamePreFix`*	Specifies the user-specified prefix for buffer names. If `-prefix` is used, you need to specify a prefix `bufNamePreFix` for buffer names that may be used in this command. The default prefix for added buffers is “`buf_dummy_`” if you do not specify it.
`-report`	Reports list of all buffers and assign nets that were replaced by the buffers. If `-report` is used, a list of all buffers and assign nets that were replaced by the buffers will be reported in log file. If buffers are inserted to the netlist you may need to use Innovus to place and route them.

The `remove_assigns` command:

- Removes assigns that involve only internal "wires".
  For example, in the code below, `assign n2 = n1` is removed.

```
module top();
  wire n1, n2;
  assign n2 = n1;

  BUFX2 u1(.Y(n1));
  BUFX2 u2(.A(n2));
  endmodule
```

becomes

```
module top();
  wire n1;

  BUFX2 u1(.Y(n1));
  BUFX2 u2(.A(n1));
  endmodule
```

- Removes assigns that involve only one port.
  For example, in the code below, `assign p1 = n1` is removed.

```
module hier(p1);
output p1;
wire n1;
assign p1 = n1;
 BUFX2 u1(.Y(n1));
endmodule
```

  becomes

```
module hier(p1);
output p1;
 BUFX2 u1(.Y(p1));
endmodule
```

- Rewires nets with feedthrough assigns that involve at least two ports, at some higher level:

```
module hier2 (
          i,
          o);
    input i;
    output o;
    assign o = i ;
    BUFX2 u1 (.A(i));
endmodule
module hier (
          i,
          o);
    input i;
    output o;

    hier2 u1 (.i(i),
          .o(o));
endmodule
module top ();
    // Internal wires
    wire i;
    wire o;
    BUFX2 u1 (.Y(i));
    hier u2 (.i(i),
          .o(o));
    BUFX2 u3 (.A(o));
endmodule
```

  becomes

```
module hier2 (
          i,
          o);
     input i;
     output o;

     BUFX2 u1 (.A(i));
  endmodule
  module hier (
          i,
          o);
     input i;
     output o;
     hier2 u1 (.i(i));
  endmodule
  module top ();
     // Internal wires
     wire i;

     BUFX2 u1 (.Y(i));
     hier u2 (.i(i));
     BUFX2 u3 (.A(i));
  endmodule
```

setEcoMode

```
setEcoMode
[-help]
[-addPortAsNeeded {true|false}]
[-batchMode {true|false}]
[-reset]
[-honorDontTouch {true|false}]
[-honorDontUse {true|false}]
[-honorFixedNetWire {true|false}]
[-honorFixedStatus {true|false}]
[-honorPowerIntent {true|false}]
[-inheritNetAttr {true|false}]
[-LEQCheck {true|false}]
[-modifyOnlyLayers bottom_layer:top_layer]
[-prefixName prefix]
[-preserveModuleFunction {true|false}]
[-refinePlace {true|false}]
[-spreadInverter {true|false}]e
[-updateTiming {true|false}]
[-delayCalcEffort effort_level]
```

Controls certain behaviors of the `ecoAddRepeater`, `ecoChangeCell`, and `ecoDeleteRepeater` ECO commands.

Use the `getEcoMode` command to return the current settings for the `setEcoMode` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setEcoMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setEcoMode`.
`-addPortAsNeeded {true|false}`	Controls whether timing optimization adds ports to the lower-level hierarchies of the design. It only works during ECO and overwrites the value of `setOptMode -addPortAsNeeded` for the ECO commands, `ecoAddRepeater`, `ecoChangeCell`, and `ecoDeleteRepeater`. *Default*: `true`

`-batchMode {true|false}`	Speeds up the runtime in case the user performs lots of interactive ECO operations. When the user specifies `batchMode true`, various ECO operations are performed in a batch and initialization steps are not performed for every ECO call. In addition, updating timing is performed only once for all the ECO operations. Tool will only perform timing initialization at the first ECO command in the batch mode and timing update at the time of exiting the batchMode. User needs to specify `batchMode false` to exit the `batchMode`.
	The batch mode is activated by `setEcoMode -batchMode true` and exited by `setEcoMode -batchMode false`, and only `ecoAddRepeater` , `ecoDeleteRepeater` and `ecoChangeCell` could be given in between `setEcoMode -batchMode true` and `setEcoMode -batchMode false`:
	`setEcoMode -batchMode true`
	`eco commands (ecoAddRepeater, ecoDeleteRepeater, ecoChangeCell)`
	`setEcoMode -batchMode false`
	If a user activates the batch mode, then the user needs to remember to exit it. If the user does not exit the batch mode, the user might see some unexpected results.
	The `refinePlace` command will not be called in batchMode. It will be called when the tool exits the batchMode and if `refinePlace` is set to `true`.
	All other `setEcoMode` settings should be made prior to activating batch mode. If you would like to change `setEcoMode` settings, you need to exit the batch mode first.
	Note: This option supports all commit ECO commands. Evaluation commands are not supported.

`-delayCalcEffort effort_level`	Specifies the delay calculation effort level { low | medium | high } used in timing update. *Default*: `high`

`-honorDontTouch {true|false}`	Checks for the don't touch attribute on nets and instances. If you specify a terminal list, this parameter checks each terminal for the don't touch attribute. The `ecoAddRepeater` command honors the don't touch nets. The `ecoDeleteRepeater` and `ecoChangeCell` commands honor don't touch instances. *Default:* `true`
`-honorDontUse {true|false}`	Checks for the don't use attribute on cells. The `ecoAddRepeater`, `ecoDeleteRepeater`, and `ecoChangeCell` commands honor don't use cells. *Default:* `true`
`-honorFixedNetWire {true|false}`	Checks for fixed net wires. It restricts the addition or deletion on fixed net so that the fixed status of the wires is not altered. *Default*: `true` When `-honorFixedNetWire` is set to false, `ecoAddRepeater` can insert a cell on the wires with the FIXED/COVER status and change the status of these wires from FIXED/COVER to ROUTED.
`-honorFixedStatus {true|false}`	Does not allow preplaced, fixed instances to be resized. *Default:* `true`

`-honorPowerIntent {true|false}`	Performs MSV checks during ECO. When set to `true`:
	• `ecoChangeCell` does not allow:
	  ◦ Resizing a regular cell with an always-on cell (and vice-versa).
	  ◦ Resizing with cells belonging to different power domains.
	• `ecoAddRepeater` does not allow:
	  ◦ Adding buffer to a power domain it doesn't belong to.
	  ◦ Buffering a cross power domain net if it can cause MSV violations.
	  ◦ Adding a regular buffer to a net that requires an always-on buffer.
	  ◦ Adding an always-on buffer to a net that does not require an always-on buffer.
	• `ecoDeleteRepeater` does not allow deletion of a buffer or an inverter-pair that will cause:
	  ◦ A redundant isolation cell.
	  ◦ A CPF-constrained net to be deleted.
	Note: To prevent buffer insertion failure due to MSV constraints, you need to set `-honorPowerIntent` to `false`.
	Default: `true`

`-inheritNetAttr {true | false}`	Inherits the net attribute. *Default*: `true`
`-LEQCheck {true|false}`	Enables functionality checking when swapping cells. The software can swap cells that are logically equivalent. When `-LEQCheck` is `true`: • `ecoAddRepeater` adds inverter pair if the specified cell is inverter. • `ecoDeleteRepeater` deletes inverter pair if the specified instance is inverter. • `ecoChangeCell` changes cell to another cell type with the same function as long as they have same number of pins. When `-LEQCheck` is set to `false`: • `ecoAddRepeater` can add only one single inverter if the specified cell is inverter. • `ecoDeleteRepeater` can delete only one single inverter if the specified instance is inverter. • `ecoChangeCell` can perform nonequivalent cell change as long as they have same number of pins. *Default:* `true`
`-modifyOnlyLayers` *`bottom_layer:top_layer`*	Modifies routes only on metal layers in this range (both inclusive). *Default*:""

`-prefixName prefix`	Adds a prefix to the inserted cells. *Default:* `ECO` **Note**: If the prefix name for eco cells is changed using the `-prefixName` option, it is not honored when the ECO batch mode is turned on.
`-preserveModuleFunction {true|false}`	Determines whether to preserve logical functions at hierarchical module ports. It only works for manual ECO commands and overwrites the value of `setOptMode -opt_preserve_hpin_function` for the ECO commands, `ecoAddRepeater`, `ecoChangeCell`, and `ecoDeleteRepeater`. *Default:* `false`
`-refinePlace {true|false}`	Legalizes placement whenever `ecoAddRepeater` or `ecoChangeCell` are used. If `false`, you must run `refinePlace` after ECO if you want to legalize placement. **Note:** Specifying `-refinePlace true` can affect performance when large numbers of cells are added or modified. *Default:* `true`
`-reset`	Resets parameters to their default values. The `-reset` parameter must be the first parameter specified. If you specify reset by itself, the software resets all `setEcoMode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported.
`-spreadInverter {true|false}`	Distributes an inverter pair evenly between the driver and the sink. It works when the buffering nets have only one driver and one sink. The design must be routed before you use this parameter. *Default:* `true`
`-updateTiming {true|false}`	Controls whether to `eco*` commands recompute timing. If `true`, the eco* commands recompute timing. If `false`, the commands do not recompute timing, and us can later use `report_timing` to recompute timing. *Default:* `true`

Examples

- The following command does not legalize placement whenever `ecoAddRepeater` or `ecoChangeCell` are used:
  ```
  setEcoMode -refinePlace false
  ```
- The following command resets the `-honorDontTouch` parameter to its default value:
  ```
  setEcoMode -reset -honorDontTouch
  ```
- The following command resets all `setEcoMode` parameters to their default values:
  ```
  setEcoMode -reset
  ```

Related Information

- `ecoAddRepeater`
- `ecoChangeCell`
- `ecoDeleteRepeater`
- `getEcoMode`
- `refinePlace`
- `report_timing`
- `setOptMode`

ungroup

```
ungroup
[-help]
hInst+
[-exclude hinst+]
[-flatten]
[-honor_dont_touch]
[-threshold instance_count]
[-verbose]
[-simple  | -prefix prefix | -escaped ]
[-force | -only_user_hierarchy ]
```

Ungroups the specified hinsts. Ungrouping dissolves the hinsts and moves the contents into its parent directory. By default, an hinst or an inst inside one of the ungrouped hinsts is renamed by concatenating its name to its parent's name with "_". for example, "ungroup h1", would rename h1/i1 to h1_i1. Name collisions are resolved by appending _1, _2, etc. until a unique name is found. If a given hinst cannot be ungrouped, a warning is issued about why it cannot be ungrouped.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each ungroup. For a detailed description of the command and all of its parameters, use the man command: man ungroup
hInst+	Specifies the hinst(s) to be ungrouped.
`-escaped`	Flattened names use escaped hinst names. For example, h1/i1 becomes \h1/i1 in Verilog and h1\/i1 in DEF.
`-exclude` *hinst+*	Excludes specified hinst(s) as ungroup targets.
`-flatten`	Ungroups recursively below the specified hinst(s)
`-force`	Ungroups all hierarchy, mapped or not, and overrides ungroup_allowed if it is set to `false`.
`-honor_dont_touch`	Skips the hinst that is marked as dont touch.
`-only_user_hierarchy`	Ungroups only user hierarchy, synthesized module will not be ungrouped.
`-prefix` *prefix*	Flattened names use a prefix. For example, h1/i1 becomes <prefix>_i1.
`-simple`	Flattened names use the base_name. For example, h1/i1 becomes i1.
`-threshold` *instance_count*	Ungroups only those hinsts that have an instance count less than or equal to the specified integer.
`-verbose`	Prints the names of the hierarchies that are being ungrouped.

Example

- The following command will flatten h1, h2/h3. It will recursively flatten any lower hinsts that have an inst count <= 10 and do not have 'cpu' in their name.
  ```
  >ungroup {h1 h2/h3} -exclude *cpu* -flatten -threshold 10 -prefix test
  ```
- This command ungroups all hinsts matching `h1*` and uses escaped names for the flattened names. This is useful so external SDC files that only refer to inst and pin names will still work but it may cause problems in down-stream tools that do not support escaped names. Note that hpins and hnets of ungrouped hinsts will disappear, so usage of those names will no longer work.
  ```
  >ungroup h1* -escaped
  ```
  An inst named `h1/i1` will become `h1\/i1`. Various commands like `get_cells` will still "soft-match" `h1/i1` to the escaped `h1\/i1` name.

Related Information

- `group_instance_suffix`
- `group`

ECO Global Variables

- bit_blasted_port_style
- ecoDisableNetRenamingForFlatNetlist
- group_instance_suffix
- init_no_new_assigns

bit_blasted_port_style

`bit_blasted_port_style %s_%d`

Type: String

Default: `%s_%d, persistent`

This global variable is used to define the bus bit blasted port name.

Specifies the naming style to be used if mapped ports are bit blasted with the command bitblast_ports. The values of this global need to contain elements %s and %d.

The following rules apply:

- A port name always starts with the base name (represented by %s)
- The suffix is appended to the base name to form port name, according to the format specified in the global value.
- %d represents the bit information.

Example

The default of %s_%d will convert a bus-bit A[0] to A_0.

If you want to keep the original bus-bit name but have it turned into an escaped scalar name you can use:

```
set_db bit_blasted_port_style {%s\[%d\]}
```

This will convert a bus-bit A[0] to an escaped scalar name A\[0\] in Tcl, so the port .name would be “A[0]”, the .escaped_name would be “A\[0\]”, the dual-ported-object name would look like {port:top/A\[0\]} with Tcl adding the {} around it to protect the \ chars, and the Verilog port would look like “\A[0] “.

One advantage of this is that SDC commands and Innovus Tcl commands that use port names with A[0] will still “soft-match” the new A\[0\] name, so they do not have to change. However, using escaped names may require careful Tcl scripting to make sure you do not inadvertently lose the escaped \ chars when you want them.

Related Information

- `bitblast_ports`

ecoDisableNetRenamingForFlatNetlist

`ecoDisableNetRenamingForFlatNetlist`

Type: Integer

Default: `0`

Specifies whether nets would be renamed in ECO report of `ecoCompareNetList`. When set to 1, there will not be any net renaming for flat netlist. The net renaming will take place if net name is hierarchical and the terms it connects to do not fall under the same hierarchy or lies up in the hierarchy. You need to set `ecoDisableNetRenamingForFlatNetlist` to `1` before `ecoCompareNetlist` command is issued to disable net renaming for flat netlists.

Related Information

- `ecoCompareNetlist`

group_instance_suffix

`group_instance_suffix` *`suffix`*

Type: String

Default: `i`

This suffix is added to a new hinst name created by the `group` command. So, `group -name abc` creates a new module named `abc`, and a new `hinst` named `abci`. If you do not want any suffix, you need to set the value to the empty string “”.

Related Information

- `group`

init_no_new_assigns

`init_no_new_assigns`

Type: Boolean

Default: `False`

Turns on an assign-free flow. By default, applications like optimization can add Verilog assigns to remove unnecessary buffers and reduce power. If you have external tools that do not support Verilog assigns, you can set this value to true to prevent applications from adding any new assign statements to the Verilog netlist. If there are existing assigns in the Verilog netlist you can remove them with the `remove_assigns` command.

Related Information

- `remove_assigns`

17

Low Power Commands and Global Variables

- Low Power Commands
- Low Power Global Variables

Low Power Commands

- addPowerSwitch
- check_power_intent
- cloneMSVGate
- commit_power_intent
- compare_power_intent
- cutBoxListFromPowerDomain
- cutPowerDomainByOverlaps
- debugPtnBoundaryPorts
- deletePowerSwitch
- exportPowerSwitch
- free_power_intent
- getCPFUserAttributes
- getInstPowerDomain
- getMsvMode
- getPGPinUseSignalRoute
- getUPFPortAttr
- getValidPowerSwitchLocation
- importPowerSwitch
- mark_physical_power_domains
- modifyPowerDomainAttr
- movePowerSwitch
- read_power_intent
- rechainPowerSwitch
- replacePowerSwitch
- reportIsolation
- reportPowerDomain
- reportPowerSwitch
- reportShifter
- reportVoltage
- restorePowerSwitch
- routePGPinUseSignalRoute
- savePowerSwitch
- setMsvMode
- setPGPinUseSignalRoute
- swapRegularCellWithAlwaysOnCell
- verifyPowerDomain
- verifyPowerSwitch
- write_power_intent

addPowerSwitch

```
addPowerSwitch
[-help]
[-1801PowerSwitchRuleName string]
{-column
-powerDomain
-enablePinIn {listOfPins}
-enableNetIn {listOfNets}
-enablePinOut {listOfPins}
-enableNetOut {netBaseName}
-globalSwitchCellName cell_syntax
-globalBufferCellName cell_syntax
-horizontalPitch float
[-area {x1 y1 x2 y2}]
[-switchModuleInstance]
[-bottomOffset float]
[-leftOffset float]
[-rightOffset value]
[-topOffset value]
[-power {(VDD:VDD1 VDD2...)...}]
[-ground {(GND:GND1 GND2...)...}]
[-incremental]
[-width length_in_um]
[-height length_in_um]
[-orientation orientation_syntax]
[-connectBottomSwitchEnablePins {LtoR | RtoL}
[-parallelEnable]
[-globalPattern pattern_syntax]
[-acknowledgeTreeCell cell_name]
[-acknowledgeTreeHierInstance hier_instance]
[-backToBackChain {LtoR | RtoL}]
[-logical]
[-loopbackAtEnd]
[-maxChainDepth integer]
[-maxDistance distance_in_microns]
[-maxFanout integer]
[-snapToNearest]
[-skipRows]
[-topDown]
[-placeByNetWireIntersect {h_net h_layer v_net v_layer}]
[-placementAdjustX {delta_in_x}]
[-placementAdjustXY {delta_in_x delta_in_y}]
[-placementAdjustY {delta_in_y}]
[-reportFile filename]
[-chainByRow]
[-checkerBoard]
[-noDoubleHeightCheck]
[-tapFromNearestEnableNet]
[-allowPartialStripeRowOverlap]
[-skipX]
[-skipY]
[-honorNonRegularPitchStripe]
[-noFixedStdCellOverlap]
[-placeUnderVerticalNet {v_net v_layer}]
[-noRowVerify]
|-ring
-powerDomain powerDomainName
-enablePinIn {listOfPins}
-enableNetIn {listOfNets}
-globalSwitchCellName cell_sytax
-switchModuleInstance instanceName
-vertex {x1 y1 x2 y1 x2 y2 x3 y2 x3 y3 x1 y3... xn yn}
```

```
[-enableNetOut {listOfNets}]
[-enablePinOut {listOfPins}]
[-power {(VDD:VDD1 VDD2...)...}]
[-ground {(GND:GND1 GND2...)...}]
[-startEnableChainAtCorner integer or corner_syntax
[-firstInstanceName instanceName]
[-distribute]
[-noFiller]
[-incremental]
[-counterclockwise]
[-centerTapPins pinname_list]
[-reportFile filename]
[-specifySideList [0|1 ...]
[-bottomSide [0|1]]
[-leftSide [0|1]]
[-horizontalSide [0|1]]
[-rightSide [0|1]]
[-topSide [0|1]]
[-verticalSide [0|1]
[-bufferCellSideList {cell_syntax ...}]
[-bufferCellNameBottom cell_syntax]}
[-bufferCellNameHorizontal cell_syntax]
[-bufferCellNameLeft cell_syntax]
[-bufferCellNameRight cell_syntax]
[-bufferCellNameTop cell_syntax]
[-bufferCellNameVertical cell_syntax]
[-globalBufferCellName cell_syntax]
[-fillerCellSideList {cell_syntax ...}]
[-fillerCellNameBottom cell_syntax]
[-fillerCellNameHorizontal cell_syntax]
[-fillerCellNameLeft cell_syntax]
[-fillerCellNameRight cell_syntax]
[-fillerCellNameTop cell_syntax]
[-fillerCellNameVertical cell_syntax]
[-globalFillerCellName cell_syntax]
[-switchCellSideList {cell_syntax ...}]
[-switchCellNameBottom cell_syntax]
[-switchCellNameHorizontal cell_syntax]
[-switchCellNameLeft cell_syntax]
[-switchCellNameRight cell_syntax]
[-switchCellNameTop cell_syntax]
[-topSwitchCell float]
[-switchCellNameVertical cell_syntax]
[-switchCellSideList {cell_syntax ...}
[-cornerCellList {listOfCornerCells}]
[-cornerOrientationList {orientation_syntax ...]
[-insideCornerCellList {cellName ...]
[-r0Corner corner_syntax]
[-r0InsideCorner {LT | TR | RB | BL}
[-r0Side {L | T | R | B}]
[-r0SideOrientation orientation_syntax]
[-switchPitch value]
[-switchPitchBottom value]
[-switchPitchHorizontal value]
[-switchPitchLeft value]
[-switchPitchRight value]
[-switchPitchTop value]
[-switchPitchVertical value]
[-switchPitchSideList {value ...}
[-sideOffsetList {value ...}]
[-bottomOffset float]
[-horizontalOffset float]
[-leftOffset float]
```

```
[-rightOffset float]
[-topOffset float]
[-verticalOffset value]
[-sideOffsetList {value ...}]
[-startOffset value]
[-startOffsetBottom value}
[-startOffsetHorizontal value]
[-startOffsetLeft value]
[-startOffsetRight value]
[-startOffsetTop value]
[-startOffsetVertical value]
[-sideStartOffsetList [value...]
[-forceOffset]
[-globalOffset float]
[-endOffset value]
[-endOffsetBottom value]
[-endOffsetHorizontal value]
[-endOffsetLeft value]
[-endOffsetRight value]
[-endOffsetTop value]
[-endOffsetVertical value]
[-sideEndOffsetList {value ...}]
[-sidePatternList {pattern_syntax ...}]
[-bottomPattern pattern_syntax]
[-leftPattern pattern_syntax]
[-horizontalPattern pattern_syntax]
[-rightPattern pattern_syntax]
[-topPattern pattern_syntax]
[-verticalPattern pattern_syntax]
[-continuePattern]
[-sideNumSwitchList {integer ...}
[-bottomNumSwitch integer]
[-horizontalNumSwitch value]
[-leftNumSwitch integer
[-rightNumSwitch integer]
[-topNumSwitch integer]
[-verticalNumSwitch value]
[-sideOrientationList {orientation_syntax ...}]
[-bottomOrientation orientation_syntax]
[-horizontalOrientation orientation_syntax]
[-leftOrientation orientation_syntax]
[-rightOrientation orientation_syntax]
[-topOrientation orientation_syntax]
[-verticalOrientation orientation_syntax]}
[-totalNumberPatterns number_of_patterns]
[-cornerCellListByCorner corner_cells_name_per_corner]
[-numSwitches]
[-noMinGapAdjust]
[-strictNegativeOffset]
[-getSwitchInstances string]
[-powerDomain powerDomainName]
[{-column | ring}]
[-area {x1 y1 x2 y2}]
[-reportFile filename]
[{-column | -ring}
[-bufferDelay {value_in_seconds | value_in_seconds value_in_seconds}]
[-bufferSwitchSameDirection string] [-cellEM string]
[-cellEM value_in_amps]
[-commitPrototype {0|1)]
[-deviceThresholdVolt value_in_volts]
[-idSat value_in_amps]
[-iLeak value_in_amps]
[-loadCapacitance value_in_farads]
```

```
[-maxIRPercent float]
[-maxLeakageCurrent value_in_amps]
[-maxLeakagePercent float]
[-maxRampUpCurrent value_in_amps]
[-maxSwitchIR value_in_volts]
[-noPgDecapEstimate {0|1)]
[-noPgCapacitanceEstimate {0|1)]
[-numberSimultaneousRampUpChain integer]
[-offsetX string] [-orientation string]
[-pgCapacitance value_in_farads]
[-pgCapacitanceFactor value_in_farads]
[-pgInductance value_in_henrys]
[-protoReportFile fileName]
[-prototypeChainDepth {0|1)]
[-prototypeChainDepthGivenRampUpCurrent {0|1)]
[-prototypeDelayGivenRampUpCurrent {0|1)]
[-prototypeIgnoreLeakage {0|1)]
[-prototypeMaxChainDepth {0|1)]
[-prototypeMinChainDepth {0|1)]
[-prototypeNumberSwitches {0|1)]
[-prototypeRampUpTime {0|1)]
[-prototypeSweepChainDepth {min_integer max_integer increment_integer}]
[-prototypeSweepSwitchNumber {min_integer max_integer increment_integer}]
[-rampUpChainDepth integer]
[-rampUpRailVoltagePercent float]
[-rampUpTime min_value_in_seconds max_value_in_seconds]
[-readPowerSwitchCell fileName]
[-rOn {value_in_ohms | value_in_ohms value_in_ohms}]
[-totalLeakagePower value_in_watts]
[-totalPower value_in_watts]
[-voltage value_in_volts] ]
[-noEnableChain]
[-ignoreShifterRule]
[-ignoreSoftBlockage]
[-cpfPowerSwitchRuleName rule_name]
[-cellEnablePin]
[-instancePrefix]
[-simplePrefix]
[-netPrefix]
```

The `addPowerSwitch` command supports five type of cells: switch cells, filler cells, corner cells (for the ring style only), and buffer cells. You can define a cell pattern when you create power switch rings.

The parameters of this command are categorized as below in the Parameters table depending upon their functionality based on how they are to be used:

- Column Type Insertion Parameters
- Power Switch Prototyping Parameters
- Ring Type Insertion Parameters

Column Type Insertion Parameters

Before you insert power switches in a column style, you must assign power switch cells to `CLASS CORE` and the correct `SITE` in the `.lef` file and connect power/ground nets and pins of the power switch cells. The distance between columns of switches (the horizontal pitch value) must be in µm.

The column style is favorable for channelless designs. In this case, all blocks are abutted and there is no space for rings. In ring style, you must have a space greater than the max ring cell height between the power domain and the adjacent power domain. This space is primarily occupied by ring switch cells and power routing.

If you create power switch columns within a specified power domain the distance between the columns of switches is determined by the `-horizontalPitch`. The first (left-most) column of switches is located `-leftOffset` from the power domain's left boundary. The `-enableNetIn`, which might consist of one or more enable nets, drives the `-enablePinIn` of the first switch cell of each column.

In column style, nets are assigned to pins sequentially if more nets than number of pins are specified. For example, if the list consists of three nets, and there are five columns of switches, net `1` drives the pin of the first switch in column `1` and `4`, net `2` connects to column `2` and `5`, and net `3` only drive the `-enablePinIn` of the first switch in column `3`.

Power Switch Prototyping Parameters

These are the parameters, which are used for power switch prototyping.

Ring Type Insertion Parameters

If you insert power switches in ring style, the power switches are added around the selected power domain in the minimum gap available. Unless specified, the power switches are added to the top level hierarchy but the user has a choice to specify to which level in the hierachy should the power switches be added. The command creates a switch ring on the top, bottom, left, and right sides of the specified power domain unless you assign a specific side.

Before you insert power switches in a ring style, you must assign power switch cells to `CLASS RING` in the `.lef` file (SYMMETRY X Y R90, recommended, but not required for switch cells) and connect power/ground nets and pins of the power switch cells. You must have existing enable nets to drive the buffer inside the power switch cells and acknowledge nets exiting the power switch cell. You need these net names when you specify `-enableNetIn` and `-enableNetOut`.

Rectilinear power domains are supported in both ring-style and column-style switch insertion.

Regardless of whether you insert columns or a ring, the software ties the `-enablePinOut` of the first switch cell to the `-enablePinIn` of the next switch cell in the same column or on the same side of a ring. All the enable pin of the switch cells in the same column/side are daisy-chained together. For the last switch of the column, the `-enablePinOut` is tied to `-enableNetOut`.

Note: The parameters which are applicable to both Column Type Insertions and Ring Type Insertions are listed in both sections.

The Syntax Conventions and Examples are listed out before apart from the description the the parameters. For conventions and examples, see:

- Syntax Conventions
- Syntax Examples

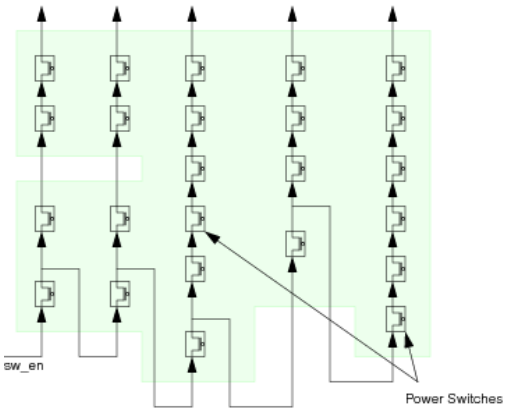
For Power Switch Chaining Unchaining Instances, see the description of the `rechainPowerSwitch` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `addPowerSwitch` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man addPowerSwitch`.
`-getSwitchInstances` *string*	Returns the existing power switch instances previously added by `addPowerSwitch`. Returns the existing power switch instances previously added by `addPowerSwitch`. It returns all the power switches in the current design without any other option, and can be used with `-powerDomain`, `-column`, `-ring` and `-area` options to return specific power switches. By default, the command returns a Tcl list. Use the `-reportFile` option if you want to list the power switch information into a file. Refer to the following example: `addPowerSwitch -getSwitchInstances -powerDomain TDSPCore -area {189.0 31.36 318.0 190.36}`
Column Type Insertion Parameters	
`-1801PowerSwitchRuleName` *string*	Specifies the power switch rule name defined in the 1801 power intent file. By default, `addPowerSwitch` uses the first defined rule.

`-acknowledgeTreeCell` *cell_name*	Specifies the name of cell to use in acknowledge tree synthesis. If you specify this parameter but do not specify `-acknowledgeTreeHierInstance`, the tool builds the acknowledge tree at the top level of the design. This parameter provides control over the time it takes to switch a power domain off and on.
`-acknowledgeTreeHierInstance` *hier_instance*	Inserts an enable net out signal tree of specified cell instances in the specified hierarchical instance in the design. This parameter provides a power domain off and on; and works with option `-acknowledgeTreeCell`.
`-allowPartialStripeRowOverlap`	Allows placement of power switch cells on nearest row location even if crossing stripes do not completely ovelap the power switch cells. Use the option with the option `-placeByNetWireIntersect`.
`-area {`*x1 y1 x2 y2*`}`	Defines an area within a power domain where the tool can insert power switches. The option applies to rectangular area. *x1 y1* is the lower left coordinate of the rectangular area. *x2 y2* is the upper right coordinate corner of the rectangular area. You can specify `-leftOffset`, `-rightOffset`, `-topOffset`, or `-bottomOffset` to offset the switches from the boundaries of the area. **Note:** The power switch instances snap to placement grid so the actual location is slightly different from the specified coordinate.
`-backToBackChain {LtoR | RtoL}`	Specifies the distance between the bottom-most switch in the switch column and the bottom fence boundary of the power domain. You can use this parameter along with `-width`, `-height`, and other offsets to define a unique, specific area within the power domain for placing switches. By default, the switch column can abut the bottom fence boundary. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`
`-bufferSwitchSameDirection` *string*	Used for chaining daisy chain with power switch cell having two enabled pins. The option usage is as follows: • `bufferSwitchSameDirection 0` (Default) Input Enable net `EnIn_1` and `EnIn_2` connect first switches at opposite ends; output `EnOut_1` and `EnOut_2` come out as last switch cells in daisy chain. • `bufferSwitchSameDirection 1` Input Enable net `EnIn_1` and `EnIn_2` connect first switches at the same end; output `EnOut_1` and `EnOut_2` come out as last switch cells in daisy chain.

`-cellEnablePin`	Specify this option to define/provide the pin names of PSO cells used in `addPowerSwitch` when the PSO cells pin names are different than given in the parameters `-enablePinIn` and `-enablePinOut`. This option can be used with ring as well as column insertion.
	The option usage is as follows for defining different cell pin signature:
	`-cellEnablePin {{cellname {enableInputPinList}\ {enableOutputPinList}}}`
	In the following example the I/O pin names of switch cells `Cell65_SW_type1` and `Cell65_SW_type2` are different. You can use the `-cellEnablePin` option to mention the pin name for switch cell `Cell65_SW_type2`.
	`addPowerSwitch -column -powerDomain PDoff \`
	`-enablePinIn { SLEEPIN } \`
	`-enablePinOut { SLEEPOUT } \`
	`-globalSwitchCellName { {Cell65_SW_type1 T1}\`
	`{Cell65_SW_type2 T2} } \`
	`-enableNetIn {Ctrl1_In } \`
	`-enableNetOut { Ctrl1_Out } -globalPattern { T1 T2 }`
	`-cellEnablePin { {Cell65_SW_type2 { SLEEPIN_1 }`
	`{SLEEPOUT_1 } }}`
	In the above example the two pins specified in `-cellEnablePin` are different from those specified in `-enablePinIn { SLEEPIN }` and `-enablePinOut { SLEEPOUT }`

`-chainByRow` *`LtoR RtoL`*	Gives additional control if you need to enable horizontal signal chaining. For example, in rows. By default, signal chaining is enabled from bottom to top. The signal chaining by this option can be: • `LtoR`: Signal chaining is enabled from Left to Right direction • `RtoL`: Signal chaining is enabled from Right to Left direction
`-checkerboard` *`string`*	Places the power switches in a checkerboard pattern within a power domain.
`-connectBottomSwitchEnablePins {`*`LtoR`* `|` *`RtoL`*`}`	Connects the first switch of the first column to the enable net specified. The software connects the first switch in the remaining columns to the output enable net of the first switch in the previous column. The following figure illustrates the `LtoR` option (default) for a rectilinear power domain.  **Note:** This parameter applies to power switch column creation only. *Default:* The enable pin of the first switch of each column is connected directly to the enable net signal specified.

`-cpfPowerSwitchRuleName` *rule_name*	Specify this option when there are multiple power switch rules specified for a power domain in CPF. By default, `addPowerSwitch` uses the first defined rule.
`-enableNetIn {listOfnets}`	Specifies the list of input nets that the software attaches to the enable input pins. The order in which you specify these nets determines the sequence in which nets are assigned to pins. Enable nets and pins are connected automatically if they reside at the same level in the hierarchy. If you have already specified `update_power_switch_rule -enable_condition` or `create_power_domain -shutoff_condition` in your CPF file, you do not need to specify this parameter. `update_power_switch_rule -enable_condition` takes precedence over `create_power_domain -shutoff_condition` for enable net In connection. For automatic enable connection to work, you should also specify `update_power_switch_rule -acknowledge_receiver` info in CPF for `enableNetOut` connection. Otherwise you should provide enable net connections using `-enableNetIn` and `-enableNetOut` options. If you specify `-enableNetIn`, it will override `create_power_domain -shutoff_condition`.
`-enableNetOut {`*netBaseName* `| {listOfNets}}`	Specifies the enable output nets that the software attaches to the enable output pins. The order in which you specify these nets determines the sequence in which nets are assigned to pins. If enable nets and pins reside at the same level in the hierarchy they are connected automatically. The `-enablePinOut` of the last switch of the last column is tied to `-enableNetOut`. If you have already specified `update_power_switch_rule -acknowledge_receiver` in your CPF file, you do not need to specify this parameter. For automatic enable connection to work, you should also specify `update_power_switch_rule -enable_condition` or `create_power_domain -shutoff_condition` in the CPF file. For enableNetIn connection you should provide enable net connections using `-enableNetIn` and `-enableNetOut` options. You must specify *netBaseName* for columns or a list of nets for rings. Automatic `enableNetOut` connection works if there is only one enable net out after insertion.
`-enablePinIn {listOfPins}`	Specifies the list of enable input pins of the buffers inside the switch cell. The order in which you specify these nets determines the sequence in which nets are assigned to pins. If enable nets and pins reside at the same level in the hierarchy they are connected automatically.The software ties the `-enablePinOut` of the first switch cell to the `-enablePinIn` of the next switch cell in the same ring: All the enable pin of the switch cells in the same column are daisy-chained together. The `-enablePinIn` of the first switch of the first column, is tied to `-enableNetIn`.
`-enablePinOut {listOfPins}`	Specifies the list of enable output pin(s) of the buffer(s) inside the switch cell. The order in which you specify these pins is important because nets are assigned to pins sequentially. The software ties the `-enablePinOut` of the first switch cell to the `-enablePinIn` of the next switch cell in the same column: All the enable pin of the switch cells in the same column are daisy-chained together. For the last switch of the column, the `-enablePinOut` is tied to `-enableNetOut`.
`-firstInstanceName` *instanceName*	Specifies the instance name of an existing switch cell in the netlist to use as the first cell in the column or ring. Usually, there is at least one switch instantiated in a module (specified by `-switchModuleInstance`) in the netlist. The software creates a unique instance name for each switch inserted. This parameter is optional. If the netlist does not contain switches, then this parameter is invalid.
`-globalBufferCellName` *cell_syntax*	Specifies the cell name of the buffer cell to insert in the power switch ring surrounding the specified power domain, or in the columns inside the power domain, if there are spaces available.

`-globalFillerCellName` *cell_syntax*	Specifies the cell name of the filler cell to insert in the power switch ring surrounding the specified power domain, or in the columns inside the power domain, if there are spaces available.
`-globalPattern` *pattern_syntax*	Specifies a pattern of cells to repeat either in the switch rings or switch columns. The pattern consists of a group of different types of cells in a specified order.
`-globalSwitchCellName` *cellName*	Specifies the cell name of the switch cell to insert in the power switch ring surrounding the specified power domain, or in the column inside the power domain. This parameter is required.
`-ground` {(*GND:GND1 GND2*...)...}	Specifies a list of ground nets and pin connections. If `-ground` has multiple ground nets, the software uses the last one in the list for tie-hi/lo connections. For example, specify `-ground (VSS:gnd vss!)` to indicate that the `gnd` and `vss!` pins of all the instances in a given power domain are connected to the `VSS` net.
`-height` *length_in_um*	Specifies the height of an area inside the power domain where switches can be placed. You can use this parameter along with `-width`, `-bottomOffset`, and `-leftOffset` to define a unique, specific area in the power domain for placing switches.
`-honorNonRegularPitchStripe`	Honors the option`-skipRows` when the pitch/distance between the power stripe specified in `-placeunderverticalnet` option is not regular. This option is related to column insertion.
`-horizontalPitch` *float*	Specifies the distance between switch columns in µm. This parameter is required. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*.
`-ignoreShifterRule`	Ignores the level shifter rules, which are applied to the power switches in both CPF and UPF.
`-ignoreSoftBlockage`	With this option `addPowerSwitch` can insert power switch cells in areas with soft or partial blockages. By default, `addPowerSwitch` does not add power switch cells in areas with any type of placement blockage: hard, soft, or partial.
`-incremental`	Inserts switches to the power domain without removing the existing switches. *Default:* Running the `addPowerSwitch` command removes the existing placed switches and inserts new switches.
`-instancePrefix` *prefix*	Adds prefix to power switch instances. **Note**: `-instancePrefix` overwrites the prefix specified in CPF by *update_power_switch_rule*.
`-leftOffset` *float*	Specifies the distance between the left-most switch in the switch column and the right fence boundary of the power domain. You can use this parameter along with `-width`, `-height`, and `-bottomOffset` to define a unique, specific area within the power domain for placing switches. By default, the switch column can abut the right fence boundary. **Note:** When you create a power domain, a `minGap` exists for this power domain, and the switches are inserted within this `minGap`. However, if the specified `minGap` is less than the switch dimension, then the tool automatically extends the `minGap` for this power domain. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*, and see commands `-bottomOffset`, `-rightOffset`, `-topOffset`, `-verticalOffset`, and `-horizontalOffset`. *Default:* `0` The switches abut the power domain fence if possible.
`-logical`	Inserts a logical power switch to the partition block so that the software writes out correct PG direction in dumped PG netlist.

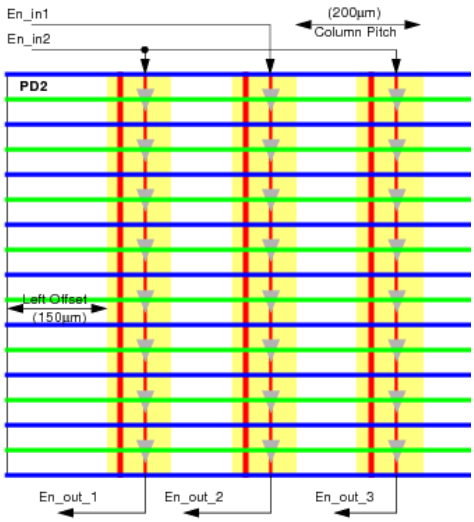
-loopbackAtEnd	Connects the enablePinOut of the last cell in the chain to the enablePinIn of the same cell. This option useful or can be used when a power switch has two enablePin In and Out pins.
-maxChainDepth *integer*	Specifies the maximum number of instances in the enable chain. After the specified maximum, addPowerSwitch starts a new enable chain from the next power switch cell. This feature provides fine control over the time it takes to switch a power domain off and on.
-maxDistance *distance_in_microns*	During power switch insertion, maxDistance is the maximum distance within which the addPowerSwitch looks for the next switch to enable net connections to form an enable signal chain connection.
-maxFanout *integer*	During power switch insertion, maxFanout is the maximum number of PSO cell enable input pins that a PSO cell enable output pin may connect to. This option is to be used with maxDistance.
-noEnableChain	Inserts and places power switches without adding enable signal chaining.
-noFixedStdCellOverlap	Avoids placing PSO instances over fixed physical instances already placed on rows.
-noDoubleHeightCheck	Allows double height power switch insertion even if when SITE name is same for double height as well as single height cells. The recommended flow is to use different SITE names for single and double height cells.
-noRowVerify	Allows you to not verify row coverage at end of a column insertion. By default, the command verifies row coverage.
-offsetX	Specifies the offset number of intersections in X direction. With this, power switch instances can be inserted in checker board style with specified offset value. **Note:** This must used with -placeByNetWireIntersect and -skipX, options. The value should not be more than the value of -skipX.
-orientation *orientation_syntax*	Specifies the orientation of the switch to be placed in the row. The software checks the specified orientation of the switch against the legal orientation of the row in which the switch is to be placed. The software uses the row site information and the row orientation to determine whether the specified orientation is legal. *Default:* Follows the row orientation.
-parallelEnable	Connects a power switch's input pin in parallel style rather than daisy-chaining. You do not have to specify -enablePinOut or -enableNetOut.
-placeByNetWireIntersect {*h_net h_layer v_net v_layer*}	Places switches at the intersection of specified horizontal net and vertical net on the respective LEF layers of a pre-existing power grid. You must specify the horizontal net and layer first, then the vertical net and layer. By default, the switches are placed at the bottom-left intersection of the grid. You can offset the switches from the grid intersection by specifying -placementAdjustX, -placementAdjustXY, or -placementAdjustY in addition to this parameter.
-placementAdjustX {*delta_in_x*}	Specifies the horizontal offset (in micrometers) from the grid intersection specified by -placeByNetWireIntersect. The value can be positive or negative.
-placementAdjustXY {*delta_in_x delta_in_y*}	Specifies the x-y offset (in micrometers) from the grid intersection specified by -placeByNetWireIntersect. The values can be positive or negative.
-placementAdjustY {*delta_in_y*}	Specifies the vertical offset (in micrometers) from the grid intersection specified by -placeByNetWireIntersect. The values can be positive or negative.
-placeUnderVerticalNet {*vert_net vert_layer*}	Places switches under the vertical net of the specified LEF layer of a pre-existing power grid. By default, the switches are placed at the bottom-left intersection of the grid and row but in addition to this option, you can offset the switches from the row by specifying options like -placementAdjustX.

`-power {(VDD:VDD1 VDD2...)...}`	Specifies a list of power nets and pin connections. For example, specify `-power (Vdd1:Vdd Pwr) (Vdd2:VDD vdd!)` to indicate that the `Vdd` and `Pwr` pins of all the instances in a given power domain are connected to the `Vdd1` net, and the `VDD` and `vdd!` pins are connected to the `Vdd2` net. If `-power` has multiple power nets, the software uses the last one in the list for tie-hi/lo connections. For example, if `-power {(vdd1:VDD1)(vdd2:VDD2)}` is specified, then the software uses net `vdd2` for tie-hi connection. **Note:** You must specify both `-ground` and `-power` to ensure that the instance being moved contains the correct `(net:pin)` connections in the new power domain.
`-powerDomain powerDomainName`	Specifies the power domain that shuts off through the inserted power switches. Alternatively, you can use IDs and multipliers, as shown below: `addPowerSwitch -column -powerDomain PD0 \ -globalSwitchCellName {{switchCell_1 A} {switchCell_2 B}} \ -globalPattern {A {B*3}} ..."`
`-reportFile filename`	Writes switch instances and their corresponding input/output enable signal pin/net connections to the specified file name.
-rightOffset	Specifies the distance between the right-most switch in the switch column and the right fence boundary of the power domain. You can use this parameter along with `-width`, `-height`, and `-leftOffset` to define a unique, specific area within the power domain for placing switches. By default, the switch column can abut the right fence boundary. **Note:** When you create a power domain, a `minGap` exists for this power domain, and the switches are inserted within this `minGap`. However, if the specified `minGap` is less than the switch dimension, then the tool automatically extends the `minGap` for this power domain. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`
-skipX	Specifies the number of intersections to skip in X direction. **Note:** This option is to be used with `-placeByNetWireIntersect`.
-skipY	Specifies the number of intersections to skip in Y direction. **Note:** This option is to be used with `-placeByNetWireIntersect`
`-skipRows integer`	Specifies the number of rows to skip when placing a switch in a column. For example, `-skipRows 2` skips two rows before placing the next switch in a column. You can use this parameter with or without the `-checkerboard` parameter.

-simplePrefix	Allows user to create a power switch instance name with simple numbers. By default power switch instance names are created in the format:
	`pso_PDname_SwitchCellName_X_andY_location_SequenceNumber`
	This option can be used with `-instancePrefix prefix`
	Example :
	`addPowerSwitch -column ... \`
	`-instancePrefix strongPsw_ -simplePrefix 1`
	With this command run, the created instance name will be: `strongPsw_1, strongPsw_2 ..`
	Note: `-simplePrefix` overwrites the prefix specified in CPF by `update_power_switch_rule`.

-snapToNearest	Snaps the cell placement to the nearest legal location. This parameter can be used when a hard macro blocks part of a column. The switches that would overlap the hard macro blockage are placed at legal locations nearest to the expected original location. You must specify either `-skipRows` or `-checkerboard` in order to create enough space for the cells to snap. If you specify `-skipRows`, you must set the value greater than zero.
`-spacerCellName` *cell_syntax*	Specifies the spacer cells to place on either side of power switch cells within a power domain to prevent the tool from placing standard cells next to the switch cells.
`-switchModuleInstance` *instanceName*	When this parameter is specified, all the switches are instantiated in the module instance specified. When not specified, the switch instances are inserted at the top level. This parameter is optional.

`-tapFromNearestEnableNet`	Enables `addPowerSwitch` to use the enable net from the nearest, already inserted/placed power switch instance. This option can be used during incremental power switch insertions.
	Example :
	`addPowerSwitch -column \`
	`-powerDomain PD1 -enablePinIn {PinIn} \`
	`-enablePinOut {PinOut} \`
	`-globalSwitchCellName PSO_SWITCH\`
	`-enableNetIn {pwr_on} -enableNetOut {pwr_ack}\`
	`-horizontalPitch 100 \`
	`-leftOffset 5 \`
	`-area { x1 y1 x2 y2 } -incremental \`
	`-tapfromnearesEnableNet`

`-topDown`	Adds switches from the top down rather than from the bottom up. The following figure shows how the software places switches from top to bottom. 
`-topOffset` *value*	Specifies the distance between the top-most switch in the switch column and the top fence boundary of the power domain that shuts off. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`

`-width` *length_in_um*	Specifies the width of an area inside the power domain where switches can be placed. Specifies the distance between the bottom-most switch in the switch column and the bottom fence boundary of the power domain that shuts off. You can use this parameter along with `-bottomOffset`, `-height`, and `-leftOffset` to define a unique, specific area in the power domain for placing switches.
Power Switch Prototyping Parameters	
`-bufferDelay {`*value_in_seconds* `|` `{`*value_in_seconds value_in_seconds*`}}`	Specifies the delay of the enable buffer in switch. If the switch is a mother-daughter type (with two enables), you can specify two values corresponding to the stage-1 and stage-2 enable buffers respectively.
`-cellEM value_in_amps`	Specifies the maximum current that can pass through the switch due to the electromigration limit of the cell.
`-commitPrototype {1|0}`	Adds the number of switches determined by `-prototypeNumberSwitches`. This parameter does not work with any other prototyping commands. *Default*: If you specify `-commitPrototype` but do not specify a value, the default is 1. If you do not specify `-commitPrototype`, the default is 0.
`-deviceThresholdVolt` *value_in_volts*	Specifies the device threshold voltage for ramp up prototyping. If you do not specify this parameter, the tool computes a default value based on the switch characterization parameters.
`-idsat` `{`*value_in_amps* `|` *value_in_amps value_in_amps*`}`	Specifies the switch current when the switch is operated in saturation mode. If the switch is a mother-daughter type (with two enables), then you can specify two values corresponding to the stage-1 and stage-2 enables respectively.
`-ileak` *value_in_amps*	Specifies the switch cell leakage current when the switch is off.
`-loadCapacitance` *value_in_farads*	Specifies the total capacitive load of the domain for ramp-up analysis.
`-maxIRPercent` *float*	Specifies the maximum IR drop allowed inside power domain. *Default*: 2 percent
`-maxLeakageCurrent` *value_in_amps*	Specifies the maximum leakage current constraint of the power domain.
`-maxLeakagePercent` *float*	Specifies the maximum domain leakage constraint as a percentage of the total domain leakage current. *Default*: 50 percent
`-maxRampUpCurrent`*value_in_amps*	Specifies the maximum ramp up current value used in chain depth and switch delay prototyping. Use this parameter with `-prototypeChainDepthGivenRampUpCurrent` or `-prototypeDelayGivenRampUpCurrent`.
-maxSwitchIr*value_in_volts*	Specifies the maximum voltage drop allowed across the switch.

-netPrefix *prefix*	Adds a prefix to power switch enable nets.
	For example:
	`addPowerSwitch -instancePrefix myInst_ -netPrefix myNet_ \`
	`-column -powerDomain MPY -enableNetIn swcontrol_2 \`
	`-enableNetOut swack_2 -globalSwitchCellName CDN_COLUMN_SW \`
	`-enablePinIn A0 -enablePinOut Z0 -leftOffset 10 \`
	`-horizontalPitch 50 -skipRows 2 -reportFile PSWmpy.rpt \`
	`-switchModuleInstance DTMF_INST_4/TDSP_CORE_INST/MPY_32_INST`
	`## From PSWmpy.rpt file`
	`DTMF_INST_4/TDSP_CORE_INST/MPY_32_INST/myInst_psoI_MPY_1_CDN_COLUMN_SW_1415_330`
	`_1 (1415.32 330.87 MX)  # Domain MPY, Column 0, Sequence 1`
	`(A0) :`
	`DTMF_INST_4/TDSP_CORE_INST/MPY_32_INST/myNet_psoPSI_MPY_EnNet__1_1_1415_313_0`
	`(Z0) :`
	`DTMF_INST_4/TDSP_CORE_INST/MPY_32_INST/myNet_psoPSI_MPY_EnNet__1_2_1415_330_0`

`-noPgCapacitanceEstimate {0|1}`	Omits power grid capacitance estimation during ramp up analysis. *Default:* `0`
`-noPgDecapEstimate {0|1}`	Omits decoupling capacitance estimation during ramp up analysis. *Default:* `0`
`-numberSimultaneousRampUpChain integer`	Specifies the number of simultaneous ramp up chains for ramp up analysis.
`-protoReportFile`*fileName*	Specifies the file containing prototyping results.
`-pgCapacitance`*value_in_farads*	Specifies the total power grid capacitance for ramp up analysis.
`-pgCapacitanceFactor`*float*	Specifies the ratio of the power grid capacitance as a factor of the total load capacitance.
`-pgInductance`*value_in_henrys*	Specifies the effective power grid inductance associated with the power grid and package.
`-prototypeChainDepth {0|1}`	Performs prototyping on chain depth given maximum ramp up current. You must specify `-rampUpTime` when you specify this parameter.
`-prototypeChainDepthGivenRampUpCurrent {0|1}`	Performs prototyping on chain depth given maximum ramp up current. You must specify `-maxRampUpCurrent` when you specify this parameter.
`-prototypeDelayGivenRampUpCurrent {0|1}`	Analyzes optimum ramp up time given maximum ramp up current. You must specify `-maxRampUpCurrent` when you use this command.
`-prototypeIgnoreLeakage {0|1}`	Ignores leakage constraints during prototyping.
`-prototypeMaxChainDepth {0|1}`	Performs maximum chain depth prototyping based on ramp up time. You must specify `-rampUpTime` when you specify this parameter.
`-prototypeMinChainDepth {0|1}`	Performs minimum chain depth prototyping based on ramp up time. You must specify `-rampUpTime` when you specify this parameter.
`-prototypeNumberSwitches {0|1}`	Determines the optimum number of switches based on switch characteristics and domain constraints.
`-prototypeRampUpTime {0|1}`	Performs prototyping on ramp up time based on chain depth. You must specify `-rampUpChainDepth` when you specify this parameter.
`-prototypeSweepChainDepth {`*min_integer max_integer increment_integer*`}`	Explores various chain depths from minimum to maximum in specified increments, and reports ramp up metrics for each setting.

`-prototypeSweepSwitchNumber {`*`min_integer max_integer increment_integer`*`}`	Explores the number of switches from minimum to maximum in specified increments, and reports corresponding power domain operation metrics.
`-rampUpChainDepth integer`	Specifies the chain depth used in ramp up time prototyping. Specify this parameter when you specify `-prototypeRampUpTime`.
`-rampUpRailVoltagePercent float`	Specifies the percentage of the full domain voltage upon which the power domain is considered operable when ramping up. *Default*: 90 percent
`-rampUpTime {`*`min_value_in_seconds max_value_in_seconds`*`}`	Specifies the minimum and maximum ramp up time settings for chain depth prototyping. This parameter is required for `-prototypeChainDepth`, `-prototypeMinChainDepth`, and `-prototypeMaxChainDepth`.

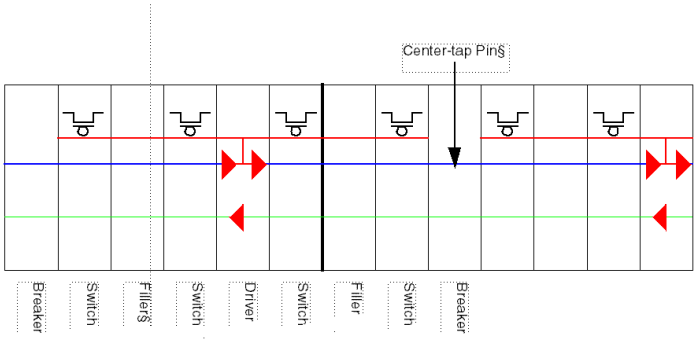
`-readPowerSwitchCell`*`fileName`*	Specifies the file containing the switch characterization data. The file contains a list of characterization data per cell, on one line. If you do not put all of the information for a cell on one line, the tool issues an error message. The format is as follows:
	`CELL cell_name SUPPLY pin_name SWITCHED pin_name RON value_in_ohms IDSAT value_in_mA ILEAK value_in_mA`
	`CELL2...`
	For power switch prototyping, the tool only reads and uses the first line of the file and uses only one cell type.

`-rOn` *`value_in_ohms`*	Specifies the resistance when the switch is operated in linear mode. If the switch is a mother-daughter type (with two enables), you can specify two values corresponding to the stage-1 and stage2 enables respectively.
`-totalLeakagePower`*`value_in_watts`*	Specifies the total leakage power of the power domain before switches are added.
`-totalPower`*`value_in_watts`*	Specifies the total power consumed by the domain. If you do not specify this parameter, report_power computes power consumption.
`-voltage` *`value_in_volts`*	Specifies the primary rail voltage of the power domain.
Ring Type Insertion Parameters	
`-1801PowerSwitchRuleName` *`string`*	Specifies the power switch rule name defined in the 1801 power intent file. By default, *addPowerSwitch* uses the first defined rule.
`-bottomNumSwitch` *`integer`*	Specifies the number of switches to insert on the bottom side(s) of the power domain. **Note:** This parameter applies to power switch ring creation only. Three scenarios can occur: • If the spaces available match the number specified, the software fills these spaces with switches. • If the number exceeds the spaces available, the software issues a warning message. • If the number is less than the spaces available, the tool inserts the switches, then inserts filler cells to fill the space. Also, if you specify `f` instead of an integer, the tool automatically fills the bottom side with switches and filler cells. This parameter is optional. See `-horizontalNumSwitch`, `-verticalNumSwitch`, `-leftNumSwitch`, `-rightNumSwitch`, and `-topNumSwitch`. *Default:* `f`

`-bottomOffset` *float*	Specifies distance from the bottom edge outside of the power domain fence and the bottom side(s) of the power switch ring. See `-horizontalOffset, -verticalOffset, -leftOffset, -rightOffset, and -topOffset`. **Note:** When you create a power domain, a `minGap` exists for this power domain, and the switches are inserted within this `minGap`. However, if the specified `minGap` is less than the switch dimension, then the tool automatically extends the `minGap` for this power domain. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`
`-bottomOrientation` *orientation_syntax*	Specifies the cell orientation for all bottom side(s) of the power switch ring.
`-bottomPattern` *pattern_syntax*	Defines a pattern for the bottom side(s) of the ring only.
`-bottomSide [0 | 1]`	Inserts switches on the bottom side(s) of the power domain. If you do not specify a side, the software creates a full power switch ring around the specified power domain. If you specify a side, the software inserts switches on that side. In this case, the tool sets `-bottomSide` to 1 and sets the other sides to `0`. You can specify multiple sides and the software inserts switches only on the sides you specify. The tool sets the specified sides to `1` and sets the unspecified sides to `0`. *Default:* `1`
`-bufferCellNameBottom` *cell_syntax*	Specifies the buffer cells to use on the bottom side(s) of the switch ring.
`-bufferCellNameHorizontal` *cell_syntax*	Specifies the buffer cells to use on the horizontal side(s) of the switch ring.
`-bufferCellNameLeft` *cell_syntax*	Specifies the buffer cells to use on the left side(s) of the switch ring.
`-bufferCellNameRight` *cell_syntax*	Specifies the buffer cells to use on the right side(s) of the switch ring.
`-bufferCellNameTop` *cell_syntax*	Specifies the buffer cells to use on the top side(s) of the switch ring.
`-bufferCellNameVertical` *cell_syntax*	Specifies the buffer cells to use on the vertical side(s) of the switch ring. This parameter applies to power switch ring creation only.

`-bufferCellSideList {`*cell_syntax* `…}`	Specifies that the buffer cells are inserted on each side of a power domain. The first value of the list indicates the first side related to `Corner 0`, which is the lower left corner of the ring. If the `-counterclockwise` parameter is specified, the first side is considered from from `Corner 0` in the counterclockwise direction. The following example shows the buffer cells specified for an eight-sided power switch ring:
	`-specifySideList {1 0 1 0 1 0 1 0}`
	`-switchCellSideList {cell1 cell1 cell2 cell2 cell3 cell3 cell4 cell4}`
	The `cell1` value is used for the first side. Specify `cell1` again to provide a valid cell type, even when there are no cells on the second side.
	Note: A space must be inserted between the list values. Additionally, only a valid cell name must be specified for each side, even if no cells can be specified for a side. The tools checks whether the number of specified cells is the same as the number specified with `-specifySideList`. In case of any discrepancy, an error message is displayed.

`-cellEnablePin` *string*	Specify this option to define/provide the pin names of PSO cells used in `addPowerSwitch` when the PSO cells pin names are different than given in the parameters `-enablePinIn` and `-enablePinOut`. This option can be used with ring as well as column insertion.
	
	The option usage is as follows for defining different cell pin signature:
	
	`-cellEnablePin {{cellname {enableInputPinList}\ {enableOutputPinList}}}`
	
	In the following example the I/O pin names of switch cells `Cell65_SW_type1` and `Cell65_SW_type2` are different. You can use the `-cellEnablePin` option to mention the pin name for switch cell `Cell65_SW_type2`.
	
	`addPowerSwitch -ring -powerDomain PDoff \`
	`-enablePinIn { SLEEPIN } \`
	`-enablePinOut { SLEEPOUT } \`
	`-globalSwitchCellName { {Cell65_SW_type1 T1}\{`
	`Cell65_SW_type2 T2} } \`
	`-enableNetIn {Ctrl1_In } \`
	`-enableNetOut { Ctrl1_Out } -globalPattern { T1 T2 }-cellEnablePin {`
	`{Cell65_SW_type2 { SLEEPIN_1 } { SLEEPOUT_1 } }}`
	
	In the above example the two pins specified in `-cellEnablePin` are different from those specified in `-enablePinIn { SLEEPIN }` and `-enablePinOut { SLEEPOUT }`

`-centerTapPins` *pinname_list*	Enables the center tap capability, and specifies the pins to use as center taps. You must use breaker cells to terminate the buffer signal propagation on both sides of the center tap buffer. The following figure shows that, depending on the specified pin name list, the software assigns a pin name to the enable net. This feature can be useful for debugging the enable signal. **Note:** This parameter applies to power switch ring creation only. 
`-continuePattern`	Continues a specified pattern from one side of the ring to the next side. You must also specify `-globalPattern` when you use this parameter. *Default:* If you do not use this parameter, the software stops the pattern at the end of a side and restarts the pattern from the beginning on the next side.
`-counterclockwise`	Places the cell pattern in a counterclockwise direction in a switch ring. Switch placement begins at the lower left corner of the ring. *Default:* If you do not specify this parameter, the software places the pattern in a clockwise direction.
`-cornerCellList {`*listOfCornerCells*`}`	Specifies the list of available corner cells that can be used by the tool to fill the corners of the ring. This parameter is optional. If user specifies only one corner cell, that same cell is used on all four corners with the appropriate orientation for a rectangular power domain.

`-cornerCellListByCorner` *`corner_cells_name_per_corner`*	Specifies the corner cells to be used at different corners. This is applicable for ring only. This option is optional.
	In an example of a rectangular power domain with four corners and two different corner cells, we name the cells: `CORNER_cell1` and `CORNER_cell2` using the option `-cornerCelllist` and then we map them to each corner as shown below:
	`addPowerSwitch -powerDomain PSO -ring \`
	`-globalSwitchCellName {{VDDISWITCH } \`
	`-globalFillerCellName {{FILLERCELL } \`
	`-specifySides {1 1 1 1 } \`
	`-globalOffset 4 \`
	`-cornercelllist { CORNER_cell1 CORNER_cell2 } \`
	`-cornerCellListByCorner { CORNER_cell1\ CORNER_cell2 CORNER_cell1 CORNER_cell2 }\`
	`.... \`

`-cornerOrientationList {`*`orientation_syntax`* `…}`	Specifies cell orientations for the tool to use for each corner of switch ring, starting with Corner 0 (the lower left corner). List the orientations in a list, with the first orientation corresponding to the first side, the second orientation to the second side, and so on. If you do not specify this parameter, the software determines the corner cell orientations.
`-cpfPowerSwitchRuleName` *`rule_name`*	Specify this option when there are multiple power switch rules specified for a power domain in CPF. By default, `addPowerSwitch` uses the first defined rule.
`-deviceThresholdVolt` *`value_in_volts`*	Specifies the device threshold voltage for ramp up prototyping.
`-distribute`	Distributes switches of different types as evenly as possible on a ring side. For example, if there are 40 available slots, and there are 20 switches and 20 fillers, the software will alternate switches and fillers in the available slots. Use this option together with `-horizontalNumSwitch`, `-verticalNumSwitch`, `-leftNumSwitch`, `-rightNumSwitch`, `-topNumSwitch`, or `-bottomNumSwitch`.
`-enableNetIn {listOfnets}`	Specifies the list of input nets that the software attaches to the enable input pins. The order in which you specify these nets determines the sequence in which nets are assigned to pins. Enable nets and pins are connected automatically if they reside at the same level in the hierarchy. If you have already specified `update_power_switch_rule -enable_condition` or `create_power_domain -shutoff_condition` in your CPF file, you do not need to specify this parameter. `update_power_switch_rule -enable_condition` takes precedence over `create_power_domain -shutoff_condition` for enable net In connection. For automatic enable connection to work, you should also specify `update_power_switch_rule -acknowledge_receiver` info in CPF for `enableNetOut` connection. Otherwise you should provide enable net connections using `-enableNetIn` and `-enableNetOut` options. If you specify `-enableNetIn`, it will override `create_power_domain -shutoff_condition`.

`-enableNetOut {`*`netBaseName`* `| {listOfNets}}`	Specifies the enable output nets that the software attaches to the enable output pins. The order in which you specify these nets determines the sequence in which nets are assigned to pins. If enable nets and pins reside at the same level in the hierarchy they are connected automatically. For the last switch of the ring, the `-enablePinOut` is tied to `-enableNetOut`. If you have already specified `update_power_switch_rule -acknowledge_receiver` in your CPF file, you do not need to specify this parameter. For automatic enable connection to work, you should also specify `update_power_switch_rule -enable_condition` or `create_power_domain -shutoff_condition` in the CPF file. For `enableNetIn` connection you should provide enable net connections using `-enableNetIn` and `-enableNetOut` options. You must specify *`netBaseName`* for columns or a list of nets for rings. Automatic `enableNetOut` connection works if there is only one enable net out after insertion.
-enablePinIn {listOfPins}	Specifies the list of enable input pins of the buffers inside the switch cell. The order in which you specify these nets determines the sequence in which nets are assigned to pins. If enable nets and pins reside at the same level in the hierarchy they are connected automatically.The software ties the `-enablePinOut` of the first switch cell to the `-enablePinIn` of the next switch cell in the same ring: All the enable pin of the switch cells in the same column are daisy-chained together. For the first switch of the ring, the `-enablePinIn` is tied to `-enableNetIn`.
-enablePinOut {listOfPins}	Specifies the list of enable output pin(s) of the buffer(s) inside the switch cell. The order in which you specify these pins is important because nets are assigned to pins sequentially. The software ties the `-enablePinOut` of the first switch cell to the `-enablePinIn` of the next switch cell in the same column: All the enable pin of the switch cells in the same column are daisy-chained together. For the last switch of the column, the `-enablePinOut` is tied to `-enableNetOut`.
`-endOffset` *`value`*	Places the last switch on every side of the power domain at the specified distance from the end of each side. The side end is not necessarily determined by the power domain boundary. You can use `-leftOffset`, `-rightOffset`, `-topOffset`, `-bottomOffset`, `-horizontalOffset`, and/or `-verticalOffset` to determine the side end relative to the power domain boundary. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default*: `0`
`-endOffsetBottom` *`value`*	Places the last switch on bottom side of the power domain at the specified distance from end of the side. The side end is not necessarily determined by the power domain boundary. You can use `-leftOffset`, `-rightOffset`, and/or `-verticalOffset` to determine the side end relative to the power domain boundary. • If the value is positive, the software places the switch to the right of the end location, or to the left of the end location if `-counterclockwise` is specified. • If the value is negative, the software places the switch to the left of the end location, or to the right of the end location if `-counterclockwise` is specified. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`

`-endOffsetHoriztonal` *value*	Places the last switch on the horizontal side of the power domain at the specified distance from end of the side. The side end is not necessarily determined by the power domain boundary. You can use `-leftOffset`,`-rightOffset`, and/or `-verticalOffset` to determine the side end relative to the power domain boundary. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`
`-endOffsetLeft` *value*	Places the last switch on the left side of the power domain at the specified distance from end of the side. The side end is not necessarily determined by the power domain boundary. You can use `-topOffset -bottomOffset` and/or `-horizontalOffset` to determine the side end location relative to the power domain boundary. • If the value is positive, the software places the switch below the end location, or above the end location if `-counterclockwise` is specified. • If the value is negative, the software places the switch above the end location, or above the end location if `-counterclockwise` is specified. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`
`-endOffsetRight` *value*	Places the last switch on the right side of the power domain at the specified distance from end of the side. The side end is not necessarily determined by the power domain boundary. You can use the `-topOffset`, `-bottomOffset`, or `-horizontalOffset` to determine the side end relative to the power domain boundary. • If the value is positive, the software places the switch above the end location, or below the end location if `-counterclockwise` is specified. • If the value is negative, the software places the switch below the end location, or below the end location if `-counterclockwise` is specified. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`
`-endOffsetTop` *value*	Places the last switch on the top side of the power switch ring at the specified distance from end of the side. The side end is not necessarily determined by the power domain boundary. You can use `-leftOffset`, `-rightOffset` or `-verticalOffset` to determine the side end relative to the power domain boundary. • If the value is positive, the software places the switch to the left of the end location, or to the right of the end location if `-counterclockwise` is specified. • If the value is negative, the software places the switch to the right of the end location, or to the left of the end location if `-counterclockwise` is specified. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`

`-endOffsetVertical` *value*	Places the last switch on the vertical sides of the power switch ring at the specified distance from end of the side. The side end is not necessarily determined by the power domain boundary. You can use `-topOffset`, `-bottomOffset`, and/or `-horizontalOffset` to determine to the side end relative to the power domain boundary. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`
`-fillerCellNameBottom` *cell_syntax*	Specifies the filler cell used for the bottom side(s) of the ring. You can specify a different filler cell for each side of the ring. This parameter is optional. *Default:* The filler cell specified by `-globalFillerCellName`.
`-fillerCellNameHorizontal` *cell_syntax*	Specifies the filler cell used for the horizontal side(s) of the ring. You can specify a different filler cell for each side of the ring. This parameter is optional. *Default:* The filler cell specified by `-globalFillerCellName`.
`-fillerCellNameLeft` *cell_syntax*	Specifies the filler cell used for the left side(s) of the ring. You can specify a different filler cell for each side of the ring. This parameter is optional. *Default:* The filler cell specified by `-globalFillerCellName`.
`-fillerCellNameRight` *cell_syntax*	Specifies the filler cell used for the right side(s) of the ring. You can specify a different filler cell for each side of the ring. This parameter is optional. *Default:* The filler cell specified by `-globalFillerCellName`.
`-fillerCellNameTop` *cell_syntax*	Specifies the filler cell used for the top side(s) of the ring. You can specify a different filler cell for each side of the ring. This parameter is optional. *Default:* The filler cell specified by `-globalFillerCellName`.
`-fillerCellNameVertical` *cell_syntax*	Specifies the filler cell used for the vertical side(s) of the ring. You can specify a different filler cell for each side of the ring. This parameter is optional. *Default:* The filler cell specified by `-globalFillerCellName`.

`-fillerCellSideList {`*cell_syntax* `…}`	Specifies the filler cells to insert on each side of a power domain. The first member of the list indicates the first side relative to Corner 0, which is the lower left corner of the ring. If `-counterclockwise` is specified, the first side is counterclockwise from Corner 0.
	The following example shows the filler cells specified for an eight-sided power switch ring:
	`-specifySideList {1 0 1 0 1 0 1 0}`
	`-fillerCellSideList {cell1 cell1 cell2 cell2 cell3 cell3 cell4\ cell4}`
	`cell1` is used for the first side. You can then specify `cell1` again to provide a valid cell type, even though there can be no cells on the second side.
	Info: You must include a space between each item in the list.
	You *must* specify a valid cell name for each side, even if no cells can be specified for a side.
	The tools checks that the number of cells you specify is the same number you specified with `-specifySideList`, and issues an error message if it finds a discrepancy.

`-fillGapWithFiller`	Forces filling of gap between ring-type power switches using the specified filler cell.

`-firstInstanceName` *instanceName*	Specifies the instance name of an existing switch cell in the netlist to use as the first cell in the column or ring. Usually, there is at least one switch instantiated in a module (specified by `-switchModuleInstance`) in the netlist. The software creates a unique instance name for each switch inserted. This parameter is optional. If the netlist does not contain switches, then this parameter is invalid.
`-forceOffset`	Forces the software to place ring switches at the exact offset you specify for all sides. Sides with unspecified offsets are forced to 0.0 µ. When you specify any other offset without specifying `-forceOffset`, the tool places switches at the smallest distance that is greater than or equal to the specified offset for each side, in order to maintain a contiguous ring. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. **Note:** Using this parameter might result in gaps in the ring.
`-globalBufferCellName` *cell_syntax*	Specifies the cell name of the buffer cell to insert in the power switch ring surrounding the specified power domain, or in the columns inside the power domain, if there are spaces available.
`-globalFillerCellName` *cell_syntax*	Specifies the cell name of the filler cell to insert in the power switch ring surrounding the specified power domain, or in the columns inside the power domain, if there are spaces available.
`-globalOffset` *value*	Offsets all sides of the power switch ring the specified distance from the power domain boundary. Specify the offset in µm. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`
`-globalPattern` *pattern_syntax*	Specifies a pattern of cells to repeat either in the switch rings or switch columns. The pattern consists of a group of different types of cells in a specified order. In the ring style, you can use the `-continuePattern` parameter to continue the pattern around the corner, from one side of the ring to another.
`-globalSwitchCellName` *cellName*	Specifies the cell name of the switch cell to insert in the power switch ring surrounding the specified power domain, or in the column inside the power domain. This parameter is required.
`-ground {(`*GND:GND1 GND2*`...)...}`	Specifies a list of ground nets and pin connections. If `-ground` has multiple ground nets, the software uses the last one in the list for tie-hi/lo connections. For example, specify `-ground (VSS:gnd vss!)` to indicate that the `gnd` and `vss!` pins of all the instances in a given power domain are connected to the `VSS` net.
`-horizontalNumSwitch` *value*	Specifies the number of switches to insert on the horizontal side(s) of the power domain. **Note:** This parameter applies to power switch ring creation only. Three scenarios can occur: • If the spaces available match the number specified, the software fills these spaces with switches. • If the number exceeds the spaces available, the software issues the following error message. • If the number is less than the spaces available, the tool inserts the switches, then inserts filler cells to fill the space. This parameter is optional. See `-leftNumSwitch`, `-rightNumSwitch`, and `-topNumSwitch`.
`-horizontalPattern` *pattern_syntax*	Specifies a pattern of cells to repeat on the horizontal side(s) of the power domain when switches are inserted. The pattern consists of a group of different types of cells in a specified order.

`-horizontalOffset` *value*	Offsets the switch ring from the horizontal edge of the power domain. Specify the offset in µm. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`
`-horizontalOrientation` *orientation_syntax*	Specifies the cell orientation for all horizontal side(s) of the power switch ring.
`-horizontalSide [0|1]`	Inserts switches on all horizontal side(s) of a power domain. You cannot specify this parameter in conjunction with `-verticalSide`, `-bottomSide`, `-topSide`, `-leftSide`, or `-rightSide`. *Default:* `1`
`-ignoreSoftBlockage`	With this option `addPowerSwitch` can insert power switch cells in areas with soft or partial blockages. By default, `addPowerSwitch` does not add power switch cells in areas with any type of placement blockage: hard, soft, or partial.
`-incremental`	Inserts switches to the power domain without removing the existing switches. *Default:* Running the `addPowerSwitch` command removes the existing placed switches and inserts new switches.
`-insideCornerCellList {`*cellName …*`}`	Specifies a list of cells for the tool to choose for the inside corners of the power domain. The following example specifies the corner cells any power switch ring: `-insideCornerCellList {cell1 cell2}` **Note:** You do not need to list a specific cell for a specific corner. The tool selects from the list you provide.
`-instancePrefix` *prefix*	Adds prefix to power switch instances. The option applies to column and ring insertion.
`-leftOffset` *float*	Specifies distance from the right edge outside of the power domain fence and the right side of the power switch ring. See`-horizontalOffset`, `-verticalOffset`, `-rightOffset`, `-bottomOffset`, and `-topOffset`. **Note:** When you create a power domain, a `minGap` exists for this power domain, and the switches are inserted within this `minGap`. However, if the specified `minGap` is less than the switch dimension, then the tool automatically extends the `minGap` for this power domain. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide* and see `-bottomOffset`, `-rightOffset`, `-topOffset`, `-verticalOffset`, and `-horizontalOffset`.. *Default:* `0` The switches abut the power domain fence if possible.
`-leftNumSwitch` *integer*	Specifies the number of switches to insert on left side(s) of the power domain. Three scenarios can occur: • If the spaces available match the number specified, then the software fills these spaces with switches. • If the number exceeds the spaces available, the software issues a warning message. • If the number is less than the spaces available, the tool inserts the switches, then inserts filler cells to fill the space. This parameter is optional. See `-bottomNumSwitch`,`-rightNumSwitch`, and `-topNumSwitch`. *Default:* `f`
`-leftOrientation` *orienation_syntax*	Specifies the cell orientation for all left side(s) of the power switch ring.

`-leftPattern` *pattern_syntax*	Defines a pattern for the left side(s) of the ring only. You can use this parameter with `-bottomPattern`, `-topPattern`, and `-rightPattern`.
`-leftSide [0 | 1]`	Inserts switches on the left side(s) of the power domain. If you do not specify a side, the software creates a full power switch ring around the specified power domain. If you specify a side, the software inserts switches on that side. In this case, the tool sets `-leftSide` to 1 and sets the other sides to `0`. You can specify multiple sides and the software inserts switches only on the sides you specify. The tool sets the specified sides to `1` and sets the unspecified sides to `0`. *Default:* 1
`-noFiller`	Suppresses the addition of filler cells in power switch rings.
`-noMinGapAdjust`	Can be used when the user does not want to extend the minimum power domain gaps to form the power switch ring.
`-noEnableChain`	Inserts and places power switches without adding enable signal chaining.
`-numSwitches`	Defines total number switches to be added around a power domain. This is applicable to ring type insertion only.
`-parallelEnable`	Connects a power switch's input pin in parallel style rather than daisy-chaining. You do not have to specify `-enablePinOut` or `-enableNetOut`.
`-power {(`*VDD:VDD1 VDD2*`...)...}`	Specifies a list of power nets and pin connections. For example, specify `-power (Vdd1:Vdd Pwr) (Vdd2:VDD vdd!)` to indicate that the `Vdd` and `Pwr` pins of all the instances in a given power domain are connected to the `Vdd1` net, and the `VDD` and `vdd!` pins are connected to the `Vdd2` net. If `-power` has multiple power nets, the software uses the last one in the list for tie-hi/lo connections. For example, if `-power {(vdd1:VDD1)(vdd2:VDD2)}` is specified, then the software uses net `vdd2` for tie-hi connection. **Note:** You must specify both `-ground` and `-power` to ensure that the instance being moved contains the correct `(net:pin)` connections in the new power domain.
`-powerDomain` *powerDomainName*	Specifies the power domain that shuts off through the inserted power switches. Alternatively, you can use IDs and multipliers, as shown below: `addPowerSwitch -ring -powerDomain PD0 \ -globalSwitchCellName {{switchCell_1 A} {switchCell_2 B}} \ -globalPattern {A {B*3}} ..."`
`-perSideNumSwitches` *value*	Specifies the number of switches to use on all sides of a power switch ring.
`-reportFile` *filename*	Writes switch instances and their corresponding input/output enable signal pin/net connections to the specified file name.
`-rightOffset`	Specifies distance from the right edge outside of the power domain fence and the right side of the power switch ring. See`-horizontalOffset, -verticalOffset, -leftOffset, -bottomOffset, and -topOffset`. **Note:** When you create a power domain, a `minGap` exists for this power domain, and the switches are inserted within this `minGap`. However, if the specified `minGap` is less than the switch dimension, then the tool automatically extends the `minGap` for this power domain. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`

`-r0Corner` *corner_syntax*	Specifies the side where the corner cell instance can be placed in the `R0` orientation without rotation. • `UL` = Upper left • `LL` = Lower left • `LR` = Lower right • `UR` = Upper right The orientation rotates the corner cell 90 degrees for each corner, proceeding counterclockwise. *Default:* `UL` The corner cell in the upper left position in the ring is placed with orientation R0. Default orientations: • `UL = R0` • `LL = MX` • `LR = R180` • `UR = MY`
`-r0InsideCorner {LT | TR | RB | BL}`	Specifies the inside corner where to place the corner cell instance in the R0 orientation. Use this parameter for rectilinear switch rings containing inside corners. • `LT` = Corner of the left and top sides • `TR` = Corner of the top and right sides • `RB` = Corner of the right and bottom sides • `BL` = Corner of the bottom and left sides *Default:* `LT` The corner cell in the left top position in the ring is placed with the orientation R0. • `LT = R0` • `BL = MX` • `RB = R180` • `TR = MY`

`-r0Side {L | T | R | B}`	Specifies the side where the switch or filler instance can be placed in the `R0` orientation without rotation. The command adjusts the orientation of the remaining sides so that the switches form a continuous ring. The orientation rotates 90 degrees for each side, proceeding counterclockwise. • `L` = Left side • `T` = Top side • `R` = Right side • `B` = Bottom side The orientation rotates 90 degrees for each side, proceeding counterclockwise, unless `-counterClockwise` is specified. *Default:* `T` The switch and filler cells in the top row are placed with orientation `R0`. Default orientations: • `T = R0` • `L = R90` • `B = R180` • `R = R270`
`-r0SideOrientation orientation_syntax`	Enables the software to rotate or mirror the switch instances on the R0 side. The software automatically adjusts the orientation of the other sides of the domain to form a continuous ring.
`-ring`	Specifies ring style power switch insertion. Power switches are placed outside the power domain.
`-rightNumSwitch integer`	Specifies the number of switches to insert on right side(s) of the power domain. Three scenarios can occur: • If the spaces available match the number specified, then the software fills these spaces with switches. • If the number exceeds the spaces available, the software issues a warning message. • If the number is less than the spaces available, the tool inserts the switches, then inserts filler cells to fill the space. Also, if you specify `f` instead of an integer, the tool automatically fills the right side with switches and filler cells. This parameter is optional. See `-bottomNumSwitch`, `-leftNumSwitch`, and `-topNumSwitch`. *Default:* `f`
`-rightOrientation orientation_syntax`	Specifies the cell orientation for all right side(s) of the power switch ring.
`-rightPattern pattern_syntax`	Defines a pattern for the right side(s) of the ring only. You must use this parameter with `-bottomPattern`, `-topPattern`, and `-leftPattern`.

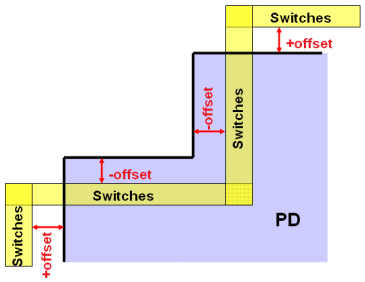
`-rightSide [0 | 1]`	Inserts switches on the right side(s) of the power domain. If you do not specify a side, the software creates a full power switch ring around the specified power domain. If you specify a side, the software inserts switches on that side. In this case, the tool sets `-rightSide` to 1 and sets other sides to `0`. You can specify multiple sides and the software inserts switches only on the sides you specify. The tool sets the specified sides to `1` and sets the unspecified sides to `0`. *Default:* `1`
`-sideEndOffsetList {value ...}`	Places the last switch at the specified distance from the end of the sides. Use this parameter with rectilinear power domains to specify a value for each side. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`

`-sideOffsetList {value …}`	Lists the offset values representing the distance between the power domain fence boundary and each side of the power switch ring. Each value in the list corresponds to a side specified with `-specifySideList`.
	
	The following example show that all sides of an eight-sided power domain can have switches:
	
	`-specifySideList {1 1 1 1 1 1 1 1}`
	
	The following example shows an offset of 8 µm from side1, 8 µm from side2, 8 µm from side 3, 6 µm from side 4, 3 µm from side 5, 3 µm from side 6, 3 µm from side 7, and 6 µm from side 8.
	
	`-sideOffsetList {8 8 8 6 3 3 3 6}`
	
	Note: When you create a power domain, a `minGap` exists for this power domain, and the switches are inserted within this `minGap`. However, if the specified `minGap` is less than the switch dimension, then the tool automatically extends the `minGap` for this power domain.
	
	Info: You must include a space between each item in the list.
	
	You *must* specify a valid cell name for each side, even if no cells can be specified for a side.
	
	The tools checks that the number of cells you specify is the same number you specified with `-specifySideList`, and issues an error message if it finds a discrepancy.
	
	For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*.
	
	Default: `0`
	The switches abut the power domain fence if possible.

`-sideOrientationList {`*orientation_syntax* `…}`	Lists the orientations for cells in each side of the power switch ring. Each orientation in the list corresponds to a side specified
	
	The following example shows which sides of an eight-sided power domain can have switches:
	
	`-specifySideList {1 1 1 1 1 1 1 1 }`
	
	Ring switches can be placed on any side.
	
	The following example shows that cells in side 1 have orientation R90, R180 in side 3, R270 in side5, and R0 in side 7:
	
	`-sideOrientationList {R90 R90 R180 R180 R270 R270 R0 R0}`
	
	Info: You must include a space between each item in the list.
	You *must* specify a valid cell name for each side, even if no cells can be specified for a side.
	
	The tools checks that the number of cells you specify is the same number you specified with `-specifySideList`, and issues an error message if it finds a discrepancy.

`-sidePatternList {`*pattern_syntax* `…}`	Specifies the cell pattern per side of a power switch ring. Each pattern in the list corresponds to a side specified with `-specifySideList`.
	
	The following example shows which sides of an eight-sided power domain can have switches:
	
	`-specifySideList {1 0 1 0 1 0 1 0}`
	
	Ring switches can be placed at sides 1, 3, 5, and 7.
	
	The following example shows that cells in side 1 have orientation R90, R180 in side 3, R270 in side5, and R0 in side 7:
	
	`-sidePatternList {{cell1 cell2 {cell3 * 6}} {cell1 cell2 {cell3 * 6}} {cell2 cell3 {cell4 * 6}} {cell2 cell3 {cell4 * 6}} {cell1 cell2 {cell3 * 6}} {cell1 cell2 {cell3 * 6}} {cell2 cell3 {cell4 * 6}}{cell2 cell3 {cell4 * 6}}`
	
	Info: You must include a space between each item in the list.
	
	You must include a valid pattern for each side, regardless of whether the side has any switches.
	
	The tools checks that the number of cells you specify is the same number you specified with `-specifySideList`, and issues an error message if it finds a discrepancy.

`-startOffset` *value*	Places the first cell on every side of the power domain at the specified distance from the beginning of each side. The beginning of each side is not necessarily determined by the power domain boundary. You can use `-leftOffset`, `-rightOffset`, `-topOffset`, `-bottomOffset`, `-horizontalOffset`, and/or `-verticalOffset` to determine the beginning of each side relative to the power domain boundary. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default*: `0`

`-strictNegativeOffset`	Defines the offset from the domain boundary to the power switch edge facing the corresponding power domain boundary. See diagram for more details. 
`-sideStartOffsetList {`*`value ...`*`}`	Places the first cell at the specified distance from the starting location of the sides. Use this parameter with rectilinear power domains to specify a value for each side. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`

`-simplePrefix`	Allows user to create a power switch instance name with simple numbers. By default power switch instance names are created in the format:
	`pso_`*`PDname_SwitchCellName_X_andY_location_SequenceNumber`*
	This option can be used with `-instancePrefix` *`prefix`*
	Example :
	`addPowerSwitch -column ...\`
	`-instancePrefix strongPsw_ -simplePrefix 1`
	With this command run, the created instance name will be: `strongPsw_1, strongPsw_2 ..`

`-specifySideList {0|1 …}`	Specifies the sides of a power switch ring that can and cannot contain ring switches. In the list, 1 indicates that the side can contain ring switches, and 0 indicates that the side cannot contain ring switches. The list must contain either 0 or 1 for each side of the shape. The first member of the list indicates the first side relative to Corner 0, which is the lower left corner of the ring. If `counterclockwise` is specified, the first side is counterclockwise from Corner 0. The following example shows which sides of an eight-sided power domain can have sw itches: `-specifySideList {1 0 1 0 1 0 1 0}` Ring switches can be inserted at sides 1, 3, 5, and 7. **Note:** You must include a space between each item in the list.
`-startEnableChainAtCorner` *`integer`* or *`corner_syntax`*	Specifies the corner where the switch ring insertion begins. This parameter can accept corner syntax or an integer, which accommodates more than four corners in a rectilinear power domain. The corner numbering starts with corner 0 at the lower left corner, and proceeds clockwise around the switch ring (unless `-counterclockwise` is specified). For example, `-startEnableChainAtCorner 1` starts the enable chain at the first corner found in a clockwise direction from corner 0.

`-startOffsetBottom` *value*	Places the first cell on the bottom side of the power domain at the specified distance from the beginning location of the side.The beginning location of the side is not necessarily determined by the power domain boundary. You can use `-leftOffset`, `-rightOffset` or `-verticalOffset` to determine the beginning location of the side relative to the power domain boundary. • If the value is positive, the software places the cell to the left of the starting location, or to the right of the starting location if `-counterclockwise` is specified. • If the value is negative, the software places the cell to the right of the starting location, or to the left of the starting location if `-counterclockwise` is specified. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`
`-startOffsetHorizontal` *value*	Places the first cell on the horizontal side of the power domain at the specified distance from the beginning location of the sides. The beginning location of the side is not necessarily determined by the power domain boundary. You can use `-leftOffset`, `-rightOffset` or `-verticalOffset` to determine the beginning location of the side relative to the power domain boundary. • If the value is positive, the software places the cell to the left of the starting location, or to the right of the starting location if `-counterclockwise` is specified. • If the value is negative, the software places the cell to the right of the starting location, or to the left of the starting location if `-counterclockwise` is specified. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *Innovus User Guide*. *Default:* `0`
`-startOffsetLeft` *value*	Places the first cell on the left side of the power domain at the specified distance from the beginning location of the side. The beginning location of the side is not necessarily determined by the power domain boundary. You can use `-bottomOffset`, `-topOffset`, or`-horizontalOffset` to determine the beginning location of the side relative to the power domain boundary. • If the value is positive, the software places the cell above the starting location, or below the starting location if `-counterclockwise` is specified. • If the value is negative, the software places the cell below the starting location, or above the starting location if `-counterclockwise` is specified. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`

`-startOffsetRight` *value*	Places the first cell on the right side of the power domain at the specified distance from the beginning location of the side. The beginning location of the side is not necessarily determined by the power domain boundary. You can use `-topOffset`, `-bottomOffset`, or `-horizontalOffset` to determine the beginning location of the side relative to the power domain boundary. • If the value is positive, the software places the first cell below the beginning location of the side, or above the beginning location of the side if `-counterclockwise` is specified. • If the value is negative, the software places the first cell above the beginning location of the side, or below the beginning location of the side if `-counterclockwise` is specified. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`
`-startOffsetTop` *value*	Places the first cell at the specified distance from the starting location of the top offset. Use this parameter with `-leftOffset`, `-rightOffset`, or `-verticalOffset` to determine the beginning location of the side. • If the value is positive, the software places the first cell to the right of the beginning location of the side, or to the left of the beginning location of the side if `-counterclockwise` is specified. • If the value is negative, the software places the first cell to the left of the beginning location of the side, or to the right of the beginning location of the side if `-counterclockwise` is specified. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`
`-startOffsetVertical` *value*	Places the first cell on the vertical sides of the power switch ring at the specified distance from beginning location of the vertical sides. The beginning location of the side is not necessarily determined by the power domain boundary. You can use `-topOffset`, `-bottomOffset`, or `-horizontalOffset` to determine the beginning location of the side relative to the power domain boundary. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`
`-switchCellNameBottom` *cell_syntax*	Specifies a different switch cell for the bottom side(s) of the power switch ring.*Default:* The switch cell specified by `-globalSwitchCellName`.
`-switchCellNameHorizontal` *cell_syntax*	Specifies a different switch cell for the horizontal side(s) of a power switch ring. *Default:* The switch cell specified by `-globalSwitchCellName`.
`-switchCellNameLeft` *cell_syntax*	Specifies a different switch cell for the left side(s) of the power switch ring. *Default:* The switch cell specified by `-globalSwitchCellName`.
`-switchCellNameRight` *cell_syntax*	Specifies a different switch cell for the right side(s) of the power switch ring. *Default:* The switch cell specified by `-globalSwitchCellName`.
`-switchCellNameTop` *cell_syntax*	Specifies a different switch cell for the top side(s) of the power switch ring. *Default:* The switch cell specified by `-globalSwitchCellName`.

`-topSwitchCell` *float*	Specifies a different switch cell for the top side(s) of the power switch ring. *Default:* The switch cell specified by `-globalSwitchCellName`.
`-switchCellNameVertical` *cell_syntax*	Specifies a different switch cell for the vertical side(s) of a power switch ring. *Default:* The switch cell specified by `-globalSwitchCellName`.

`-switchCellSidesList {`*cell_syntax …*`}`	Specifies the switch cells to insert on each side of a power domain. The first member of the list indicates the first side relative to Corner 0, which is the lower left corner of the ring. If `-counterclockwise` is specified, the first side is counterclockwise from Corner 0.
	The following example shows the switch cells specified for an eight-sided power switch ring:
	`-specifySideList {1 0 1 0 1 0 1 0}`
	`-switchCellSideList {cell1 cell1 cell2 cell2 cell3 \ cell3 cell4 cell4}`
	`cell1` is used for the first side. You can then specify `cell1` again to provide a valid cell type, even though there can be no cells on the second side.
	Info: You must include a space between each item in the list.
	You *must* specify a valid cell name for each side, even if no cells can be specified for a side.
	The tools checks that the number of cells you specify is the same number you specified with `-specifySideList`, and issues an error message if it finds a discrepancy.

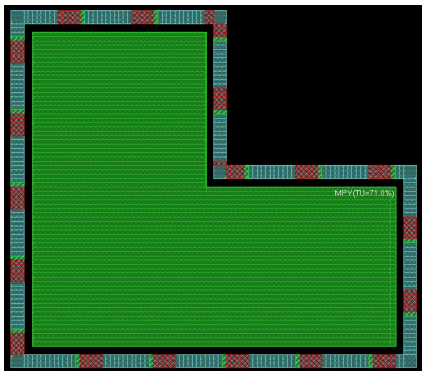
`-switchModuleInstance` *instanceName*	When this parameter is specified, all the switches are instantiated in the module instance specified. When not specified, the switch instances are inserted at the top level. This parameter is optional.
`-switchPitch`	Specifies the global switch pitch value. You cannot use this parameter with the `-switchPitch` parameters (left, right, top, bottom, horizontal, and vertical). For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`
`-switchPitchBottom` *value*	Places a switch the specified distance from the right edge of the previous cell on the bottom side of the power domain. If `-counterclockwise` is specified, the distance is measured from the left edge of the previous cell. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`
`-switchPitchHorizontal` *value*	Places the a switch the specified distance from the left edge of the previous cell on the horizontal side of the power domain. If `-counterclockwise` is specified, the distance is measured from the right edge of the previous cell. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`
`-switchPitchLeft` *value*	Places the a switch the specified distance from the bottom edge of the previous cell on the left side of the power domain. If `-counterclockwise` is specified, the distance is measured from the top edge of the previous cell. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`

`-switchPitchRight` *value*	Places the a switch the specified distance from the top edge of the previous cell on the right side of the power domain. If `-counterclockwise` is specified, the distance is measured from the bottom edge of the previous cell. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`
`-switchPitchTop` *value*	Places the a switch the specified distance from the left edge of the previous cell on the top side of the power domain. If `-counterclockwise` is specified, the distance is measured from the right edge of the previous cell. You can use this parameter with `-switchPitch*`, `-endOffset*`, and `-*Offset` parameters. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`
`-switchPitchSideList {`*value* `...}`	Places the a switch the specified distance from the edge of the previous cell on the sides specified in the list. Use this parameter for rectilinear power domains. You can use this parameter with `-switchPitch*`, `-endOffset*`, and `-*Offset` parameters. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`
`-switchPitchVertical` *value*	Places the a switch the specified distance from the bottom edge of the previous cell on the vertical sides. If `-counterclockwise` is specified, the distance is measured from the top edge of the previous cell. You can use this parameter with `-switchPitch*`, `-endOffset*`, and `-*Offset` parameters. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`
`-topOffset` *float*	Specifies the minimum offset from the top edge outside of the power domain fence boundary where the software can insert switches. See `-bottomOffset`, `-rightOffset`, and `-leftOffset`. **Note:** When you create a power domain, a `minGap` exists for this power domain, and the switches are inserted within this `minGap`. However, if the specified `minGap` is less than the switch dimension, then the tool automatically extends the `minGap` for this power domain. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0` The switches abut the power domain fence if possible. Otherwise, the switches are placed as close as possible to the power domain.

`-topNumSwitch` *integer*	Specifies the number of switches to insert on the top side(s) of the power domain. **Note:** This parameter applies to power switch ring creation only. Three scenarios can occur: • If the spaces available match the number specified, then the software fills these spaces with switches. • If the number exceeds the spaces available, the software issues a warning message. • If the number is less than the spaces available, the tool inserts the switches, then inserts filler cells to fill the space. Also, if you specify `f` instead of an integer, the tool automatically fills the top side with switches and filler cells. This parameter is optional. See `-horizontalNumSwitch`, `-verticalNumSwitch`, `-bottomNumSwitch`,`-leftNumSwitch`, and `-rightNumSwitch`. *Default:* `f`
`-topOrientation` *orientation_syntax*	Specifies the cell orientation for all top side(s) of the power switch ring.
`-topPattern` *pattern_syntax*	Defines a pattern for the top side(s) of the ring only. You must use this parameter with `-horizontalPattern`, `-verticalPattern`, `-bottomPattern`, `-leftPattern`, and `-rightPattern`.
`-topSide [0 | 1]`	Inserts switches on the top side(s) of the power domain. If you do not specify a side, the software creates a full power switch ring around the specified power domain. If you specify a side, the software inserts switches on that side. In this case, the tool sets `-topSide` to 1 and sets the other sides to `0`. You can specify multiple sides and the software inserts switches only on the sides you specify. The tool sets the specified sides to `1` and sets the unspecified sides to `0`. *Default:* `1`
`-totalLeakagePower` *value_in_watts*	Specifies the total leakage power of the power domain before switches are added.

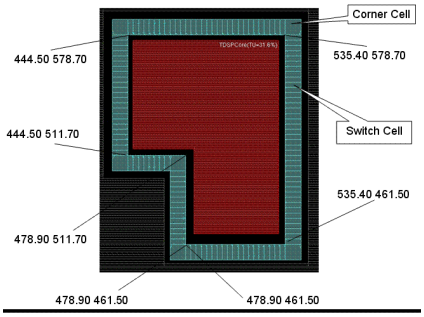
`-totalNumberPatterns` *number_of_patterns*

Adds the defined number of patterns around a power domain in a ring-type insertion. The pattern type for the insertion can be defined by the option `-globalPattern`.



Example:

```
addPowerSwitch -ring -powerDomain MPY -specifySideList {1 1 1\ 1 1 1 } \
-enableNetIn swcontrol_1 -enableNetIn swack_1 \
-globalSwitchCellName {{CDN_RING_SW1 s} {CDN_RING_SW2 b}} \
-globalFillerCellName {{CDN_RING_FILLER f}} \
-cornerCellList CDN_RING_CORNER_UL \
-insideCornerCellList CDN_RING_CORNER_InCell \
-switchModuleInstance DTMF_INST_3 \
-enablePinIn A0 -enablePinOut Z0 -sideOffsetList { 3 3 3 3 3 \ 3 } \
-globalPattern { {s}*5 {b} } \
-distributePattern 1 \
-continuePattern 1 \
-totalNumberPatterns 19 \
-reportFile MPY.rpt
```

`-vertex {x1 y1 x2 y1 x2 y2 x3 y2 x3 y3 x1 y3 ... xn yn}`	Allows you to insert power switches in ring style around the power domain by specifying vertex locations around the power domain. In the ring the cells are towards the outer side of the edge.
	 
	
	`addPowerSwitch \`
	`-ring -vertex \`
	
	`{478.90 461.50 478.90 511.70 444.50 511.70 \ 444.50 578.70 535.40 578.70 535.40 461.50 \ 478.90 461.50} -powerDomain TDSPCore...`

`-verticalNumSwitch value`	Specifies the number of switches to insert on all vertical side(s) of the power domain. Three scenarios can occur: • If the spaces available match the number specified, the software fills these spaces with switches. • If the number exceeds the spaces available, the software issues the following error message. • If the number is less than the spaces available, the tool inserts the switches, then inserts filler cells to fill the space. Also, if you specify `f` instead of an integer, the tool automatically fills the bottom side with switches and filler cells. This parameter is optional. See `-leftNumSwitch`, `-rightNumSwitch`, and `-topNumSwitch`. *Default:* `f`
`-verticalOffset value`	Offsets the switch ring from the vertical edge of the power domain. Specify the offset in µm. For more information, see "Using Pitch Control and Offsets" in the Low Power Design chapter of the *User Guide*. *Default:* `0`
`-verticalOrientation orientation_syntax`	Specifies the cell orientation for all vertical side(s) of the power switch ring.
`-verticalPattern pattern_syntax`	Specifies a pattern of cells to repeat on all vertical side(s) of the power domain. The pattern consists of a group of different types of cells in a specified order.
`-verticalSide [0|1]`	Inserts ring switches on all vertical side(s) of a power domain. *Default:* 1

Related Information

- `deletePowerSwitch`
- `exportPowerSwitch`
- `importPowerSwitch`
- `movePowerSwitch`
- `rechainPowerSwitch`
- `replacePowerSwitch`
- `verifyPowerSwitch`
- Low Power Design chapter in the *Innovus User Guide*

Syntax Conventions

- Cell syntax
  cell_syntax
  cell (single cell)
  {*cell1 cell2 …*} (list of cells)
  {{*cell id*}} (cell with id assigned)
  (cell list with id assigned to each cell)
- Orientation syntax
  orientation_syntax
  R0|R90|R180|R270|MX|MY|MX90|MY90
- Corner syntax (Ring style only)
  corner_syntax
  LT|TR|RB|BL
- Pattern syntax (Ring style only)
  pattern_syntax
  {id1 id2 {id * n}} (Example: n=integer -> repeat id n times)
  - id, id1, id2 are defined in *cell_syntax*
  - Default id is the same as the cell name if *cell_syntax* does not define it.

Syntax Examples

Power switch cell name: CDN_SWT_CELL

Filler cell name: CDN_FLL_CELL

Buffer cell name: CDN_BUF_CELL

Breaker cell name: CDN_BRE_CELL

cell_syntax

cell (single cell): CDN_SWT_CELL

{*cell1 cell2*}: {CDN_SWT_CELL1X CDN_SWT_CELL2X …}

{{*cell id*}}: {{CDN_SWT_CELL S}} <- S can be any alphabetic character from A to Z or a to z, or any combination of alphabetic characters or numerals. The id must always start with an alphabetic character.

Precedence Rules

For the -*direction*Side parameters, the software applies their settings in the following precedence rules:

- -specifySides
- -topSide, -rightSide, -leftSide, -bottomSide
- -horizontalSide, -verticalSide

Higher precedence parameters override lower precedent parameters.

- The following command creates a power switch column within power domain PD:

  ```
  addPowerSwitch -column -powerDomain PD -globalSwitchCellName switchCell -switchModuleInstance switchBlock \
  -enablePinIn A0 -enablePinOut Z0 -enableNetIn {enableNetOn} -enableNetOut {enableNetOut} \
  -leftOffset 30 -bottomOffset 0 -horizontalPitch 240
  ```

  In this example, the software inserts switchCell into power domain PD. This switch cell has only one buffer inside. This example specifies one -enableNetIn (enableNetOn) and one -enableNetOut (enableNetOut). The input pin A0 on the first switch of every column is driven by the net enableNetOn. The enablePinOut (Z0) drives the enablePinIn pin of the next switch cell in the same column. The nets for this connection are created internally and the names are unique. The enablePinOut pin of the last switch for each column is tied to the net enableNetOut. The

first switch cell is placed 30 microns from the left side boundary of the selected power domain. The spacing between the columns is 240 microns.

- The following command creates a power switch ring around power domain `PD`.

```
addPowerSwitch -ring -powerDomain PD -switchModuleInstance switchBlock \
-enablePinIn {A0 A1} -enablePinOut {Z0 Z1} -enableNetIn enIn -enableNetOut enOut \
-globalSwitchCellName switchCell -globalFillerCellName fillerCell -firstInstanceName u1_switchCell \
-leftNumSwitch -rightNumSwitch 28 -topNumSwitch 40 -bottomNumSwitch 50
```

  In in this example, all `switchCell` switches are instantiated in the `switchBlock` module.

  Note: The bottom offset is zero in this example. The first switch cell is placed in the column in or on the bottom-most switch cell row.

  This example inserts the switches as follows:

- Switches are inserted in a full ring pattern (left, top, right, and bottom)
  The switch ring encircles the `PD` power domain fence. `PD` is the power domain that shuts off.
- A specified number of switches are inserted on each side of the ring. The `-numSwitchLeft` parameter has a value of `f` (autofill), so the tool fills the left side with as many switches as possible in the available space, then inserts filler cells.
- If the spaces available match the number of switches specified, then the software fills these spaces with switches. If the number of switches is less than the spaces available, the tool inserts the switches, then inserts filler cells `fillerCell` to fill the space. If the number exceeds the spaces available, then the software issues the following error message:

```
ERROR: number of switches exceeded space availability.
```

  Also in this example, the original netlist already has a switch instantiated inside the module `switchBlock`. The switch is fixed as the first instance in the ring by `-firstInstanceName u1_switchCell`. The tool then generates a unique instance name for each switch.

  You must specify the input and output enable pin names of the switch. You must specify the pins in order because the software attaches them to the enable nets you specify. In this example, there is a loopback type of enable nets, so you must specify one net for input (`-enableNetIn`) and one net for output (`-enableNetOut`). The input net (`enIn`) attaches to the first variable specified in the `-enablePinIn` list (`A0`) of the first switch, and the output net (`enOut`) attaches to the last variable specified in the `-enablePinOut` list (`Z1`) of the next switch inserted.

- The following command presents an example of column power switch insertion:

```
addPowerSwitch -column -powerDomain TDSPCore -switchModuleInstance TDSP_CORE_INST \
-enableNetOut swcontrol_out \
-leftOffset 20 -bottomOffset 0.0 -horizontalPitch 43 -checkerBoard -loopBackAtEnd
```

- The following command presents an example of ring power switch insertion:

```
addPowerSwitch -ring \
-powerDomain TDSPCore \
-switchModuleInstance TDSP_CORE_INST \
-globalSwitchCellName PScell \
-enablePinIn EnableIn \
-enablePinOut EnableOut \
-leftOffset -4.2 \
-topOffset -4.8 \
-rightOffset -4.2 \
-bottomOffset -5.4 \
-cornerCellList CornerCell \
-verticalOrientation R0
```

Related Information

- For more information, see the following topics in the Low Power Design chapter of the *User Guide*.
  - "Overview"
  - "Power Domain Shutdown and Scaling"
  - "Power Shutdown Techniques"
  - "Power Switch Optimization"

check_power_intent

```
check_power_intent
[-help]
-1801
-hierarchical
[-outfile string]
[-verbose]
```

Checks the IEEE1801 power intent coding style for the IEEE1801 partitioning. This command also reports potential IEEE1801 coding style, which is unsuitable for partitioning.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `check_power_intent` parameter. For a detailed description of the command and all its parameters, use the man command: `man check_power_intent`
`-1801`	Specifies that the power intent is IEEE 1801.
`-hierarchical`	Specifies the input power intent style.
`-outfile string`	Specifies the output filename.
`-verbose`	Prints the detailed information related to the `check_power_intent` command.

Related Information

- `commit_power_intent`
- `free_power_intent`
- `read_power_intent`
- `write_power_intent`
- Low Power Design chapter in the *Innovus User Guide*

cloneMSVGate

```
cloneMSVGate
[-help]
[-iso]
[-shifter]
[-instances inst1 inst2 .... ]
[-maxFanout integer]
[-maxTran num]
[-maxCap num]
[-honorPrePlaced]
[-honorDontTouch]
```

Ensures that the cloned isolation or level shifter can drive a group of the receivers and be placed close to this group of receivers. By doing this, the timing QoR can be improved.

Note: The command will not automatically update your CPF/IEEE1801 file to match the new netlist. It is advised that you run Conformal Low Power to check for any issues. When the enable signal for isolation cells is cloned (creating new ports on the boundary of the power domain), the users may need to update `create_isolation_rule` in CPF or `set_isolation` in IEEE1801 by defining the isolation control differently.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `cloneMSVGate` parameter. For a detailed description of the command and all its parameters, use the man command: `man cloneMSVGate`
`-honorDontTouch`	Does not clone don't touch instances or cells. The default is to clone don't touch instances.
`-honorPrePlaced`	Does not clone pre-placed (fixed) instances. The default is to clone fixed instances
`-instances inst1 inst2 ...`	Clones specified instances.
`-iso`	Clones isolation cells.
`-maxFanout integer`	Specifies the maximum fanout count.
`-maxCap num`	An optional check to clone instances with maximum capacitance violation. The default is not to use `-maxCap` violation.
`-maxTran num`	Cloning will be only on instances with maximum transition time violation. The default is `2ns`.
`-shifter`	Clones level shifters.

Related Information

Low Power Design chapter in the *Innovus User Guide*

commit_power_intent

```
commit_power_intent
[-help]
[-clpLog string]
[-eco string]
[-isolation]
[-keepRows]
[-level_shifter]
[-power_domain]
[-power_switch]
[-state_retention]
[-verbose]
```

Commits IEEE1801 power intent specifications for the design for use in verification and implementation of the structure and behavior of the design in context of a given power management architecture. The method supports incremental refinement of power intent specifications required for IP-based design flows.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `commit_power_intent` parameter. For a detailed description of the command and all its parameters, use the man command: `man commit_power_intent`
`-clpLog string`	Specifies the CLP log file for CPF only.
`-eco string`	Specifies the eco CPF file. **Note:** The `-eco` option is available only for the CPF flow.
`-isolation`	Specifies the commit isolation.
`-keepRows`	Specifies the pre-existing rows to be retained. By default, rows are recreated based on the power domain information. The new rows might not match the original rows.
`-level_shifter`	Specifies the commit level shifter.
`-power_domain`	When specified, the command creates the power domains, makes the pg connection and saves the rules such as level shifter, isolation and state retention rule. **Note**: The command does not commit the rules
`-power_switch`	Specifies the commit power switch.
`-state_retention`	Specifies the commit state retention.
`-verbose`	Prints the detailed information related to the `commit_power_intent` command.

Related Information

- `free_power_intent`
- `read_power_intent`
- `write_power_intent`
- Low Power Design chapter in the *Innovus User Guide*

compare_power_intent

```
compare_power_intent
[-help]
[-domain_attr]
[-domain_boundary_port_constr]
[-domain_map_file file_name]
[-file_prefix prefix]
-golden1801 file_name
-hier1801 file_name
[-pin_effective_domain]
[-power_domains list_of_domains]
[-verbose]
```

Compares the golden IEEE1801 and the hierarchical IEEE1801 files generated by the Innovus hierarchical partition flow in terms of the low power database, and reports the mismatched category. If there is no mismatch, the two IEEE1801 files are considered to be equivalent in the Innovus low power flow.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `compare_power_intent` parameter. For a detailed description of the command and all its parameters, use the man command: `man compare_power_intent`
`-domain_attr`	Compares low power attributes and properties of all the attributes, which include instance count, low power cell count, primary supply, available supply, boundary, port low power constraint, and effective domain counter for the leaf instance pin, and port.
`-domain_boundary_port_constr`	Compares the domain boundary port IEEE1801 constraint against the golden IEEE1801.
`-domain_map_file file_name`	Specifies the domain map file saved during partition. The domain map file maps the golden IEEE1801 domain/supply set/low power signal with the hierarchical IEEE1801 domain/supply set/low power signal.
`-file_prefix prefix`	Specifies the file prefix for the report files.
`-golden1801 file_name`	Specifies the golden IEEE1801.
`-hier1801 file_name`	Specifies the hierarchical IEEE1801 to be compared.
`-pin_effective_domain`	Compares only the domain for leaf instance pin and port.
`-power_domains list_of_domains`	Specifies the list of specific power domains to be compared.
`-verbose`	Displays the detailed information related to each category.

Related Information

- `free_power_intent`
- `read_power_intent`
- `write_power_intent`
- Low Power Design chapter in the *Innovus User Guide*

cutBoxListFromPowerDomain

```
cutBoxListFromPowerDomain
[-help]
-powerdomain powerDomainName
-boxList {{box1} ...}
```

Cuts out default power domains from a non-default power domain. This command is mainly used to solve buffering related issues.

Use this command with other commands like `ecoAddRepeater` to insert buffers in the cutout box.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each cutBoxListFromPowerDomain parameter. For a detailed description of the command and all its parameters, use the `man` command: `man cutBoxListFromPowerDomain`
`-powerDomain`	Specifies the Power Domain in which you want to cut out.
`-boxList`	Specifies the list of boxes to be cut.

Examples

```
cutBoxListFromPowerDomain -powerDomain pd -boxList { {list1} {list2}......}
```

The list is numbers in the format: `{llx1 lly1 urx1 ury1}`

```
cutBoxListFromPowerDomain -powerDomain 1.0V_Domain \
-boxList {{10 10 20 20} {35 60 45 70}}
```

This will cut box `{10 10 20 20}` and `{35 60 45 70}` from the domain `1.0V_Domain`, and these two boxes will become the default domain.

Related Information

- `cutPowerDomainByOverlaps`
- Low Power Design chapter in the *Innovus User Guide*

cutPowerDomainByOverlaps

```
cutPowerDomainByOverlaps
powerDomainName
[-help]
```

When you floorplan one power domain (inner domain) completely inside the other power domain (outer domain), use this command to cut the outer domain as a hollow or donut shape.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `cutPowerDomainByOverlaps` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man cutPowerDomainByOverlaps`.
powerDomainName	Specifies the outer power domain where the nested power domain will reside.

Examples

Here is an example of using this command. In this example, the power domains, PD1 and PD2, are defined in the power intent file, and then the bounding box for each domain is specified:

```
modifyPowerDomainAttr PD1 -box llx lly urx usy
modifyPowerDomainAttr PD2 -box llx lly urs usy
....
```

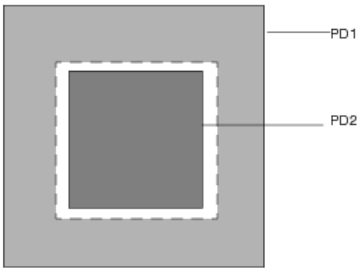
Note: The bounding box for `PD2` must lie completely within the `PD1` bounding box.

Designate `PD1` as the hollow power domain:

```
cutPowerDomainByOverlaps PD1
```

Instances in `PD1` will not be placed within `PD2`.

The following figure shows the resulting power domains.



Related Information

- `cutBoxListFromPowerDomain`
- Low Power Design chapter in the *Innovus User Guide*

debugPtnBoundaryPorts

```
debugPtnBoundaryPorts
-pins
```

The `debugPtnBoundaryPorts` command prints out the boundary pins' debug information during partition in a log which shows how to assign the power domain for the partition pins.

This is useful when debugging CPF partitions as you get the information about how Innovus assigns the power domain for the partition boundary pins during debugging.

Parameters

-pins	Specifies a list of the partition pins. If wildcard * is specified, it means all partition pins.

Example

Specify the partition pins(Hterm) you want to debug:

```
debugPtnBoundaryPorts -pins "I2/d I2/clk"
```

After partition, the command will execute the partition command to print the following DBG info in the log file:
DBG: deriving hterm I2/d power domain
DBG: ….

Related Information

Low Power Design chapter in the *Innovus User Guide*

deletePowerSwitch

```
deletePowerSwitch
[-help]
[-1801PowerSwitchRuleName ruleName]
{{-column | -ring}
-powerDomain powerDomainName}
[-instanceList {instName ...}]
[-isCustomChain]
```

Removes all switches and filler cells that were inserted by the `addPowerSwitch` command, from either columns or rings, or removes specified power switch defined in CPF.

This command retains and unplaces switches added by `addPowerSwitch -switchInstances`, whose cells were already instantiated in the netlist.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `deletePowerSwitch` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man deletePowerSwitch`
`-1801PowerSwitchRuleName ruleName`	Removes power switches based on the specified IEEE 1801 rule.
`-column`	Removes all switches and filler cells that were inserted by the `addPowerSwitch` command, from columns in the specified power domain.
`-instanceList {instName …}`	Specifies the power switch cell instances to delete. The wildcard characters (* or ?) can also be used for specifying the power switch instance name.
`-isCustomChain`	Keeps the rest chain connection for the rest power switches, after deleting the specified power switches, when those power switches are chained in a customized manner.
`-powerDomain powerDomainName`	Specifies the power domain next to which the columns are placed.
`-ring`	Removes all switches and filler cells that were inserted by the `addPowerSwitch` command, from the ring around the specified power domain.

Examples

- The following command removes instances `psoI_TDSP3_instA` and `psoI_TDSP3_InstB` that were added by the `addPowerSwitch` command:
  ```
  deletePowerSwitch -instanceList {psoI_TDSP3_instA psoI_TDSP3_InstB}
  ```
- The following command removes all power switch instances that were inserted by `addPowerSwitch` in columns inside the power domain `PD2`:
  ```
  deletePowerSwitch -column -powerDomain PD2
  ```
- The following command removes all power switch instances that were inserted by `addPowerSwitch` in the ring around `PD1`:
  ```
  deletePowerSwitch -ring -powerDomain PD1
  ```

Related Information

- `addPowerSwitch`
- `exportPowerSwitch`
- `importPowerSwitch`
- `movePowerSwitch`
- `rechainPowerSwitch`
- `replacePowerSwitch`
- `verifyPowerSwitch`
- Low Power Design chapter in the *Innovus User Guide*

exportPowerSwitch

```
exportPowerSwitch
[-help]
[-powerDomain powerDomainName]
[-file fileName]
```

Exports power switch database. The command is needed in cases of defIn-based flows where power switch database required for power switch related operations is not loaded through `defIn`. So, use `exportPowerSwitch` to save power switch database after power switch insertion using `addPowerSwitch` for later use, which can be loaded back using `importPowerSwitch`.

For example, in case power switches are inserted at the Floor Plan stage and the power switch database is exported for further use in the flow, the following command executed after floorplan for power Domain PD1, will save the power switch information into the PD1_powerswitch.db file:

```
exportPowerSwitch -powerDomain PD1 -file PD1_powerswitch.db
```

Post running the above command, if the floorplan DEF is used in the flow from synthesis, the stored power switch information needs to be restored using the `importPowerSwitch` command from the database, which was earlier stored using the `exportPowerSwitch` command after floorplan and before placement.

Note: The enable net IN and OUT connections should remain the same in the netlist in power switch database export/import process and the commands need not be run when `saveDesign` and `defOut` are run because the power switch database is automatically saved and restored.

PSO File Generation and Usage

A PSO file gets generated as an output of the `exportPowerSwitch` command, which can be used in a future session, if needed. One scenario where this file can be needed is when a DEF file containing a power switch chain is not used in the power switch synthesis and hence the incoming netlist to Innovus would not have any power switches. The power switch chain is then created at the `init` stage of Innovus using the `addPowerSwitch` command in such a run. A database saved using `write_db` after running the `addPowerSwitch` command, will have the power switch chain saved in a `<design.pso>` file. When restored, this database can also be used to write out the saved PSO file using the `exportPowerSwitch` command. When the power switches are added using the `addPowerSwitch` command, the `reportPowerSwitch` command can be used to validate their addition.

To avoid regenerating power switches in future implementation runs, save the power switch chain using the `exportPowerSwitch` command along with the floorplan DEF. Ensure that the `-routing` parameter of the `defOut` command is also used along with `-floorplan` and `-noStdCells` to save the power switch network under the NETS section of the DEF.

Refer to the following flows to understand the usage of the PSO file:

Innovus Session 1:

```
<init Design1 - incoming design has no power switches>
addPowerSwitch
exportPowerSwitch test.pso
defOut -floorplan -noStdCells -routing test.def
```

Innovus Session 2:

```
<init Design2 - incoming design has no power switches>
importPowerSwitch -file test.pso
defIn test.def
```

While reading the DEF file, you can identify the power switches using the database commands, such as `get_db` and `dbIsCellPowerSwitch`, even without using the `importPowerSwitch` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `exportPowerSwitch` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man exportPowerSwitch`.
`-powerDomain` *powerDomainName*	Specifies the power domain whose power switch information is to be exported.
`-file` *fileName*	Specifies the name of the export file.

Related Information

- `deletePowerSwitch`
- `importPowerSwitch`
- `movePowerSwitch`
- `rechainPowerSwitch`
- `replacePowerSwitch`
- `verifyPowerSwitch`
- Low Power Design chapter in the *Innovus User Guide*

free_power_intent

```
  free_power_intent
[-help]
[-keep_fence]
```

The command deletes all the power intent data stored by the `commit_power_intent` command in current session and cleans up the design.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `free_power_intent` parameter. For a detailed description of the command and all its parameters, use the man command: `man free_power_intent`
`-keep_fence`	Retains domain groups and their fences at the time of removing the the power intent. Use `read_power_intent` and `commit_power_intent -keepRows` to reload power intent after using this parameter.

Related Information

- `commit_power_intent`
- `read_power_intent`
- `write_power_intent`
- Low Power Design chapter in the *Innovus User Guide*

getCPFUserAttributes

```
getCPFUserAttributes
[-help]
-domain domainName | -net netName
```

Reports the `user_attributes` of the power domain or net specified in the CPF file. If the domain or net has no `user_attributes` in the CPF file, then this command reports an empty string.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `getCPFUserAttributes` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man getCPFUserAttribute`.
`-domain`	Specifies the name of the power domain containing the user_attributes.
`-net`	Specifies the name of the net having the `user_attributes`.

Related Information

- For more information, see the following topics in the Low Power Design chapter of the *User Guide*.
  - "Overview"
  - "Power Domain Shutdown and Scaling"
  - "Support for the Common Power Format"

getInstPowerDomain

```
getInstPowerDomain
instanceName
-help
```

Returns the power domain of the instance.

Parameters

`-help`	Outputs a brief description that includes the type and default information for `getInstPowerDomain`. For a detailed description of the command and all its parameters, use the man command: `man getInstPowerDomain`

Example

The following command returns the power domain of the instance:

```
getInstPowerDomain a/ff1

PD1
```

Related Information

Low Power Design chapter in the *Innovus User Guide*

getMsvMode

```
  getMsvMode
[-help]
[-allowNestedDefaultDomain]
[-allowPowerDomainMinGapZero]
[-checkAllNetsForDomainCrossing]
[-handleBackToBackIsolation]
[-handlePD {true|false}]
[-handlePDComplex {true|false}]
[-honorCpfGlobalConnectionSpecForAoBuffer]
[-honorCpfGlobalConnectionSpecForShifter]
[-honorDotLibRelatedPGPin]
[-honorEffectiveDomainForIsoLsInsertion]
[-markIsoEnablePinAsAlwaysOn]
[-notUseTopFTermDomainForVoltage]
[-nonDefault]
[-shareWellAOBufferingSupport]
[-upf_allow_shifter_voltage_mismatch]
[-upf_insert_on_floating_pins]
[-quiet]
```

Gets the special handling for inter-domain nets and uses effective power domains for ISO/LS insertion. For more details see `setMsvMode`.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each getMvsMode parameter. For a detailed description of the command and all its parameters, use the `man` command: `man` getMvsMode.
parameter_names	Returns information for the specified parameters. You can specify one or more parameters.
`-quiet`	Displays the current settings in Tcl list format only.

Related Information

- `setMsvMode`
- Low Power Design chapter in the *Innovus User Guide*

getPGPinUseSignalRoute

```
getPGPinUseSignalRoute
[-help]
```

Returns all the existing MSV settings that were set by the `setPGPinUseSignalRoute` command in the current database. Use this command to identify the cell PG pins, which are routed by the signal router.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command and all its parameters, use the `man` command: `man getPGPinUseSignalRoute`

Related Information

- `setPGPinUseSignalRoute`
- `routePGPinUseSignalRoute`
- Low Power Design chapter in the *Innovus User Guide*

getUPFPortAttr

```
  getUPFPortAttr
[-help]
hport_name
```

Returns UPF port attributes from UPF or for the specified values. Run this command before design partitioning. Following is the output format of this command:

```
list of {attr_name val}
```

Parameters

`-help`	Prints a brief description that includes the type and default information for each `getUPFPortAttr` parameter. For a detailed description of the command and all its parameters, use the man command: `man getUPFPortAttr`
`hport_name`	Specifies the module term name for which UPF port attributes need to be returned.

Example

The following command returns UPF port attributes for the `iC/in hport`:

```
getUPFPortAttr iC/in
```

Following is the output of the above command:

```
{-driver_supply supplySetName} {attr_name value2} {is_analog FALSE} …
```

Related Information

Low Power Design chapter in the *Innovus User Guide*

getValidPowerSwitchLocation

```
getValidPowerSwitchLocation
-help
-cell cell_name
-loc location
[-orient string]
[-searchArea string]
```

Returns a valid location of the power switch cells according to the specified placement grid, cell name, and nearest location. This valid location of the power switch cells can be used to specify offsets while using the addPowerSwitch command.

Parameters

-help	Prints a brief description that includes the type and default information for each getValidPowerSwitchLocation parameter. For a detailed description of the command and all its parameters, use the man command: man getValidPowerSwitchLocation
-cell *cell_name*	Specifies the name of the power switch cell whose valid location needs to be identified.
-loc *location*	Specifies the nearest location from where the valid location needs to be identified.
-orient *string*	Specifies the orientation. The orientation can be one of the following: r0, r90, r180, r270, mx, mx90, my, and my90.
-searchArea *string*	Specifies the area within which the valid location needs to be identified.

Example

The following command identifies a valid location for the HSWX1 power switch cell:

```
getValidPowerSwitchLocation -cell HSWX1 -orient MX -loc {382 970}
```

Following is the output of the above command:

```
382.0350 970.0750 MX
```

Related Topics

addPowerSwitch

importPowerSwitch

```
importPowerSwitch
[-help]
[-powerDomain powerDomainName]
[-file fileName]
```

Imports power switch databases. The need for this commands arises in case of defIn-based flows where power switch database required for power switch related operations is not loaded through defOut.

So, using `exportPowerSwitch` a user can save power switch database after power switch insertion using `addPowerSwitch` for later use and it can be loaded back using `importPowerSwitch`.

Note: The enable net IN and OUT connections should remain the same in the netlist in power switch database export/import process and the commands need not be run when saveDesign and `defOut` are run because the power switch database is automatically saved and restored.

PSO File Generation and Usage

A PSO file gets generated as an output of the `exportPowerSwitch` command, which can be used in a future session, if needed. One scenario where this file can be needed is when a DEF file containing a power switch chain is not used in the power switch synthesis and hence the incoming netlist to Innovus would not have any power switches. The power switch chain is then created at the `init` stage of Innovus using the `addPowerSwitch` command in such a run. A database saved using `write_db` after running the `addPowerSwitch` command, will have the power switch chain saved in a `<design.pso>` file. When restored, this database can also be used to write out the saved PSO file using the `exportPowerSwitch` command. When the power switches are added using the `addPowerSwitch` command, the `reportPowerSwitch` command can be used to validate their addition.

To avoid regenerating power switches in future implementation runs, save the power switch chain using the `exportPowerSwitch` command along with the floorplan DEF. Ensure that the `-routing` parameter of the `defOut` command is also used along with `-floorplan` and `-noStdCells` to save the power switch network under the NETS section of the DEF.

Refer to the following flows to understand the usage of the PSO file:

Innovus Session 1:

```
<init Design1 - incoming design has no power switches>
addPowerSwitch
exportPowerSwitch test.pso
defOut -floorplan -noStdCells -routing test.def
```

Innovus Session 2:

```
<init Design2 - incoming design has no power switches>
importPowerSwitch -file test.pso
defIn test.def
```

While reading the DEF file, you can identify the power switches using the database commands, such as `get_db` and `dbIsCellPowerSwitch`, even without using the `importPowerSwitch` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each importPowerSwitch parameter. For a detailed description of the command and all its parameters, use the `man` command: `man importPowerSwitch`.
`-powerDomain` *powerDomainName*	Specifies the power domain whose power switch information is to be imported.
`-file` *fileName*	Specifies the name of the import file.

Related Information

- `deletePowerSwitch`
- `exportPowerSwitch`
- `movePowerSwitch`
- `rechainPowerSwitch`
- `replacePowerSwitch`
- `verifyPowerSwitch`
- Low Power Design chapter in the *Innovus User Guide*

mark_physical_power_domains

```
mark_physical_power_domains
[-help]
{-domains physical_domain_list | -reset | -checkOnly}
[-groupPhyLogicalDomains {{phyPd lpd1 lpd2 ...} ...}]
```

Marks the physical power domains that need fences in case the power domains share the same primary supply net and the available supply net setting (if specified). The command also enables the logical domains to exist without any fence.

Note: `mark_physical_power_domains` needs to be called again (after the initial use), whenever the tool reads in an power intent file. Else, the file content is not recognized.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command and all its parameters, use the `man` command: `man mark_physical_power_domains`
`-checkOnly`	Checks the fence sharing against Power/Ground (P/G) nets.
`-domains physical_domain_list`	Marks listed power domains as physical domains (a physical power domain must have a fence). Once a physical domain is marked, the power domain sharing the same P/G nets (not marked in this command) are recognized as logical domain (or fenceless domain). Therefore, this domain can be implemented without a fence. The instances within logical domain are placed within the fence of its corresponding physical domain. This parameter also executes the`-checkOnly` parameter, by default. **Note:** Default power domain is not required to be marked, it is marked by default. This command needs to be called after each run of `commit_power_intent`. Otherwise the physical/logical power domain information will be lost in the database.
`-groupPhyLogicalDomains {{phyPd lpd1 lpd2 ...} ...}`	Groups a list of physical and logical domains for the `mark_physical_power_domains` command.
`-reset`	Resets the status marking settings after running `commit_power_intent`.

Example

The following command marks `PDdefault` as physical domain:

```
mark_physical_power_domains -domains {PDdefault}
```

Related Information

- `commit_power_intent`
- Low Power Design chapter in the *Innovus User Guide*

modifyPowerDomainAttr

```
modifyPowerDomainAttr
[-help]
[-addBlockBox string]
[-core2Left valueInMicron]
[-core2Right valueInMicron]
[-core2Top valueInMicron]
[-core2Bot valueInMicron]
[-defaultTechSite string]
[-disjointHInstBoxList string]
[-firstRowSiteIndex integer]
[-lastRowSiteIndex integer]
[-minGaps {values}|-gapEdges {values}]
powerDomainName
[-rowFlip {first second noflip auto}]
[-rowPatternSite site_name]
[-rowSpaceType {0 1 2}]
[-rowSpacing float]
[-rsExts {values}|-extEdges {values}]
```

Renames a power domain or changes its structure.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `modifyPowerDomainAttr` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man modifyPowerDomainAttr`.
`-addBlockBox string`	Adds a hard block to the disjoint power domain. `-addBlockBox` supports both rectangular and rectilinear macros.
`-core2Left valueInMicron`	Specifies the row margin from left boundary.
`-core2Right valueInMicron`	Specifies the row margin from right boundary.
`-core2Top valueInMicron`	Specifies the row margin from top boundary.
`-core2Bot valueInMicron`	Specifies the row margin from bottom boundary.
`-defaultTechSite string`	Allows selection of one of the tech sites to be not picked by the software, in case there are two smallest tech sites of the same size in a power domain. **Note:** The specified `-defaultTechSite` size value must be same as the current default tech site in the power domain.
`-disjointHInstBoxList`	Specifies a region for an Hinst. It is defined as an Hinst and Region pair.
`-firstRowSiteIndex integer`	Specifies the first row site index value.
`-lastRowSiteIndex integer`	Specifies the last row site index value.
`-extEdges {values}`	Specifies the maximum search distance for power extension connections by domain edge in microns. You can specify the value per edge in clockwise order, starting with the vertical edge at the lower-left corner (smallest Y, and then smallest X).

`-gapEdges {values}`	Specifies the minimum space value to other domain by power domain edge in microns. You can specify the value per edge in clockwise order, starting with the vertical edge at the lower-left corner (smallest Y, and then smallest X).
`-minGaps` *`T B L R`*	Defines the distance, in microns, that must be reserved from the power domain boundary edges for power routing. You can specify four values, one for each edge.
`powerDomainName`	Specifies the current name of the power domain to be modified. This is a positional parameter, and you must specify this value immediately after the command name.

`-rowFlip {first | second | noFlip | auto}`	Flips the orientation of either the first or second row, then follows the flipped and abutted row pattern.
	`first`
	(from bottom to top). MX -> R0 -> MX -> R0 - > …
	`second`
	(from bottom to top). R0 -> MX -> R0 - > MX -> …
	`noFlip`
	(from bottom to top). R0 -> R0 -> R0 - > R0 -> …
	The noFlip option preserves the R0 orientation for all rows (above).
	`auto`
	Detects orientation of row outside of power domain and aligns with power domain bottom. Orientation is applied to the first row at the bottom of power domain.
	So if row on left is R0, the pattern will be (from bottom to top). R0 -> MX -> R0 - > MX -> …
	If row on left is MX, the pattern will be (from bottom to top). MX -> R0 -> MX - > R0 - > …
	The `auto` option resets the software for this automatic behavior if you have previously specified `first`, `second`, or `noFlip`.
	Default: `auto`
	Note: *saveRowImage* resets the option `-rowFlip` to `auto`. In this case, try moving the power domain and check the row changes.

`-rowPatternSite` *`site_name`*	Specifies the site name for the row pattern.
`-rowSpacing` *`value`*	Specifies the row spacing between each row (1) or each pair of rows (2) as specified in `-rowSpaceType`.
`-rowSpaceType {1|2}`	Determines whether the row spacing value applies between each row (1) or each pair of rows (2).
`-rsExts` *`T B L R`*	Specifies the boundary for legal targets to be used by the power planning and routing commands, in conjunction with the power domain boundary. You can specify four values, one for each edge.

Examples

- The following command changes the route search extension values of `PowerDomain1` to 15 µm on each side:
  ```
  modifyPowerDomainAttr PowerDomain1 -rsExts 15 15 15 15
  ```
- The following command defines a disjoint power domain named `PowerDomain2` using the `-disjointHInstBoxList` parameter:
  ```
  modifyPowerDomainAttr PowerDomain2 -disjointHInstBoxList {{disjoint_hinst1 {llx lly urx ury}} {disjoint_hinst2 { llx lly urx ury}}.....}
  ```

Related Information

Low Power Design chapter in the *Innovus User Guide*

movePowerSwitch

```
movePowerSwitch
[-help]
[-area {x1 y1 x2 y2}]
[-columnx string]
[-fixOverlap]
[-honorSoftBlockage]
[-powerDomain string]
[-reportFile string]
[-selected]
[-switchInstances string]
[-xOffset float]
[-yOffset float]
```

The command is used to move power switch instances by offset value in X or Y direction. You can specify multiple power switch instances to be moved in the same offset value.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `movePowerSwitch` parameter. For a detailed description of the command and all its parameters, use the man command: man `movePowerSwitch`
`-area {x1 y1 x2 y2}`	Specifies moving the switches inside the indicated area.
`-columnx string`	Specifies moving columns at the indicated x locations.
`-fixOverlap`	Internally checks any overlap and fixes it with the suggested offset value.
`-honorSoftBlockage`	When used, prevents the `movePowerSwitch` command from moving power switch cells in areas having soft placement blockage.
`-powerDomain string`	Specifies moving the switches found in the indicated power domain.
`-reportFile string`	Specifies the name of the report file that stores information about the moved switches.
`-selected`	Specifies moving the selected switches.
`-switchInstances string`	Specifies moving the switches in the list.
`-xOffset float`	Specifies moving switches by the indicated amount in the x direction.
`-yOffset float`	Specifies moving switches by the indicated amount in the y direction.

Related Information

- `deletePowerSwitch`
- `exportPowerSwitch`
- `importPowerSwitch`
- `rechainPowerSwitch`
- `replacePowerSwitch`
- `verifyPowerSwitch`
- Low Power Design chapter in the *Innovus User Guide*

read_power_intent

```
read_power_intent
[-help]
fileName
{-cpf | -1801 | -msvDB }
```

Reads-in power intent file and do syntax check. The method supports incremental refinement of power intent specifications required for IP-based design flows.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `read_power_intent` parameter. For a detailed description of the command and all its parameters, use the man command: `man read_power_intent`
fileName	Specifies the name of the power intent file to be read.
`-cpf`	Specifies that the CPF format power intent file will be used.
`-1801`	Specifies that the IEEE1801 format power intent file will be used.
`-msvDB`	Specifies that the IEEE1801 data format `(.cpfdb)` power intent file will be used. **Note:** `-msvDB` is for partition flow and must be read for power intent in partition implementation.

Related Information

- `commit_power_intent`
- `free_power_intent`
- `write_power_intent`
- Low Power Design chapter in the *Innovus User Guide*

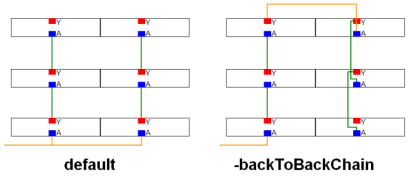
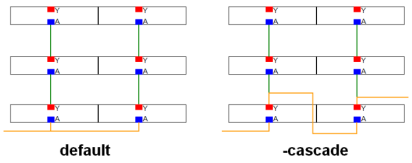
rechainPowerSwitch

```
rechainPowerSwitch
-help
[-backToBackChain]
[-cascade]
[-cellEnablePin]
[-chainByInstances]
[-chainByRow]
[-chainDirectionX LtoR|RtoL]
[-chainDirectionY BtoT|TtoB]
[-chainStyle {min_length_tree single_output_chain}]
[-chainXbeforeY]
[-chainYbeforeX]
[-enableNetIn]
[-enableNetOut]
[-enablePinIn]
[-enablePinOut]
[-maxDistanceX um]
[-maxDistanceY um]
[-mergeDistanceX um]
[-mergeDistanceY um]
[-parallelEnable]
[-reportFile]
[-reverseOrder]
[-selected]
[-switchInstances]
[-unchainByInstances]
```

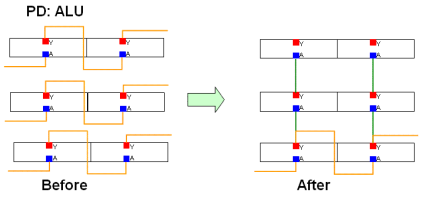
Enables flexible power switch enable chaining and unchaining after power switch insertion.

Note: It is recommended that you use this command for PSO enable chaining.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each rechainPowerSwitch parameter. For a detailed description of the command and all its parameters, use the `man` command: `man` rechainPowerSwitch.
`-backToBackChain`	Connects the `enableNetOut` at the top of a column to the `enableNetIn` at the top of the next column, and connects the `enableNetOut` at the bottom of the column to the `enableNetIn` at the bottom of the next column. The chain proceeds left to right or right to left. By default, the `enableNetIn` is connected to the bottom of each column, and the `enableNetOut` exits from the top of each column, in parallel.  default -backToBackChain This parameter is optional.
`-cascade`	Connects switches in cascade  default -cascade

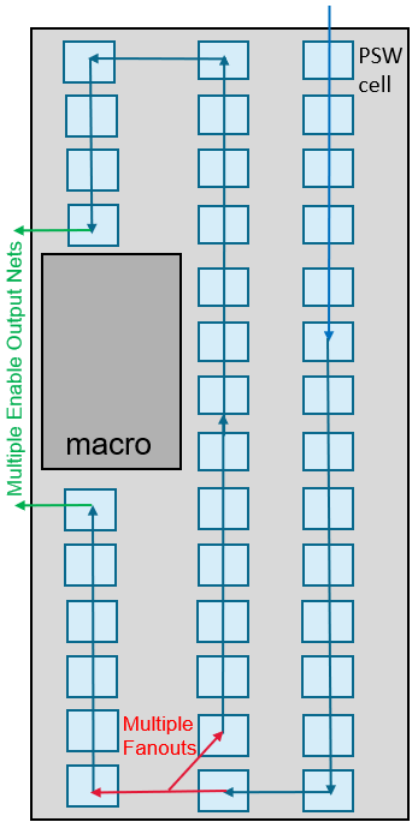
`-cellEnablePin`	Provides the pin names of power switch cells when the power switch cell pin names are different than the specified instance's cell. This option is also useful in cases where all enable pins of instances are not connected by enabling a way to provide pin map.
	Example:
	`set INST [ addPowerSwitch -getSwitchInstances -powerDomain PD1 -column ]`
	
	`rechainPowerSwitch -chainByInstances -chainYbeforeX \`
	`-unchainByInstances -enablePinIn SW_IN -enablePinOut SW_OUT \`
	`-enableNetIn NETin -enableNetOut NETout \`
	`-chainDirectionX RToL -reportFile pd1_chain.rpt \`
	`-backToBackChain \`
	`-cellEnablePin { { SW_withDiff_pinNames {SW_IN_1} {SW_OUT_1} }} \`
	`-switchInstances $INST`

`-chainByInstances`	To chain the instances. This option is used for chaining the instances defined by `-selected` or instances given in `-switchByInstances`.
	 
	
	Select all PSO instances of power domain ALU
	`set ALUinst [addPowerSwitch -column -getSwitchInstances -powerDomain ALU]`
	The below command make the changes in the connection of the switch enable
	`rechainPowerSwitch -unchainByInstances\`
	`-chainByInstances \`
	`-switchInstances $ALUinst -enablePinIn A -enablePinOut Y \`
	`-enableNetIn swcontrol_2 -enableNetOut swack_2 \`
	`-cascade -chainDirectionY BtoT`

`-chainByRow`	Gives additional control if you need to enable horizontal signal chaining. For example, in rows. By default, signal chaining is enabled from bottom to top.
`-chainDirectionX` *`LtoR|RtoL`*	Defines the direction of the chain along X axis.
`-chainDirectionY` *`BtoT|TtoB`*	Defines the direction of the chain along Y axis.

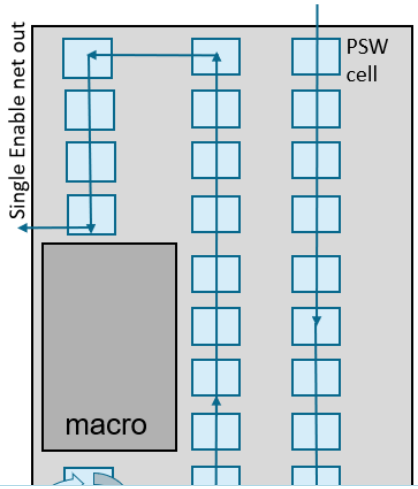
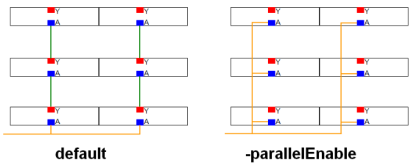
`-chainStyle {min_length_tree single_output_chain}`	Specifies the chain style for enabling power switches and making enable chain connections. The specified chain styles help in optimizing the wire length with longer enable chains, and connecting all the power switches in single pass to avoid blockages. Following are the chain style values specified by this parameter:
	• `min_length_tree`: Connects the enable pin of the switches with a chain, which has multiple branch outputs. The following command presents the usage of this parameter value:
	`rechainPowerSwitch -powerDomain PD1 -selected \`
	`-chainStyle min_length_tree \`

```
-maxDistanceX 40 -maxDistanceY 100 -startLoc "200 6400" \
-enableNetIn pso1 -enableNetOut pso1_out ….
```



- `single_output_chain`: Connects the enable pin of the switches with a chain, which has single branch output. This parameter value can be specified with `-max_distance_x` and `-max_distance_y` parameters. The following command presents the usage of this parameter value:

  ```
  rechainPowerSwitch -powerDomain PD1 -selected \
  -chainStyle single_output_chain \
  -maxDistanceX 40 -maxDistanceY 100 -startLoc "200 6400" \
  -enableNetIn pso1 -enableNetOut pso1_out ….
  ```

	
`-chainXbeforeY`	Defines row chain over column.
`-chainYbeforeX`	Defines column chain over row. This is the default option.
`-enableNetIn {listOfnets}`	Specifies the list of input nets that the software attaches to the enable input pins. The order in which you specify these nets determines the sequence in which nets are assigned to pins. Enable nets and pins are connected automatically if they reside at the same level in the hierarchy.
`-enableNetOut {`*`netBaseName`* `| {listOfNets}}`	Specifies the enable output nets that the software attaches to the enable output pins. The order in which you specify these nets determines the sequence in which nets are assigned to pins. If enable nets and pins reside at the same level in the hierarchy they are connected automatically. For the last switch of the ring, the `-enablePinOut` is tied to `-enableNetOut`.
`-enablePinIn {listOfPins}`	Specifies the list of enable input pins of the buffers inside the switch cell. The order in which you specify these nets determines the sequence in which nets are assigned to pins. If enable nets and pins reside at the same level in the hierarchy they are connected automatically.The software ties the `-enablePinOut` of the first switch cell to the `-enablePinIn` of the next switch cell in the same ring: All the enable pin of the switch cells in the same column are daisy-chained together. For the first switch of the ring, the `-enablePinIn` is tied to `-enableNetIn`.
`-enablePinOut {listOfPins}`	Specifies the list of enable output pin(s) of the buffer(s) inside the switch cell. The order in which you specify these pins is important because nets are assigned to pins sequentially. The software ties the `-enablePinOut` of the first switch cell to the `-enablePinIn` of the next switch cell in the same column: All the enable pin of the switch cells in the same column are daisy-chained together. For the last switch of the column, the `-enablePinOut` is tied to `-enableNetOut`.
`-maxDistanceX` *um*	Maximum X distance a switch can drive.
`-maxDistanceY` *um*	Maximum Y distance a switch can drive.
`-mergeDistanceX` *um*	To consider switches within this in the same column.
`-mergeDistanceY` *um*	To consider switches within this in the same row.
`-parallelEnable`	Connects enable Net In to all switches in parallel type connection. 

`-reportFile`	Writes switch instances and their corresponding input/output enable signal pin/net connections to the specified file name.
`-reverseOrder`	The reverse order of the `-switchInstances`.
`-selected`	Selects switches to be chained and unchained. **Note:** This option must be used with `-chainByInstances` and `-unchainByInstances`.
`-switchInstances`	Specifies the instances to be chained/unchained. When chaining is requested, the sequence of the final chain is taken from the sequence of the instance names given in the provided list.
`-unchainByInstances`	To unchain the instances. This option is used for unchaining the instances defined by `-selected` or instances given in `-switchByInstances`.

Related Information

- `deletePowerSwitch`
- `exportPowerSwitch`
- `importPowerSwitch`
- `movePowerSwitch`
- `replacePowerSwitch`
- `verifyPowerSwitch`
- Low Power Design chapter in the *Innovus User Guide*

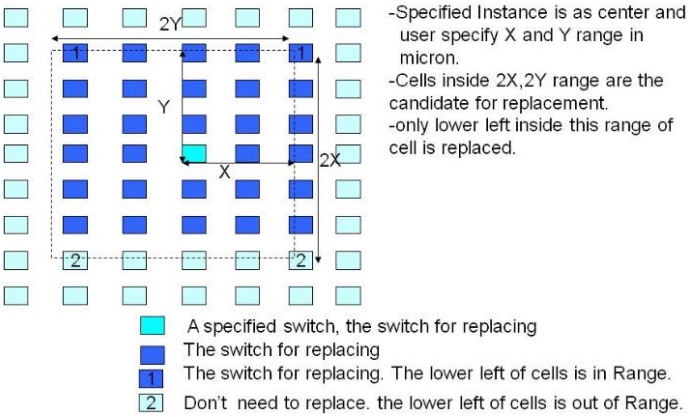
replacePowerSwitch

```
replacePowerSwitch
[-help]
-insts inst_name_list
-cell cell_name
[-xoffset value_in_microns]
[-yoffset value_in_microns]
[-orientation orienation_syntax]
[-cellEnablePin {{cellname {enableInputPinList} {enableOutputPinList}}}]
[-xyRangeFromCenterInst]
```

Replaces the cell of a power switch instance. You can also change the orientation of the instance after cell substitution, and provide a new location for the instance origin relative to the original instance origin.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `replacePowerSwitch` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man` replacePowerSwitch.
`-cell` *`cell_name`*	Specifies the cell to substitute for the instance's cell.
`-cellEnablePin` {{*`cellname`* {*`enableInputPinList`*} {*`enableOutputPinList`*}}}	Provides the pin names of power switch cell specified thru `-cell` option in `replacePowerSwitch` when the power switch cell pin names are different than the specified instance's cell. This option is also useful in cases where all enable pins of instances are not connected by enabling a way to provide pin map.
`-insts` *`inst_name_list`*	Specifies the instance whose cell you want to replace.
`-orientation` *`orientation_syntax`*	Specifies the orientation of the instance after the cell substitution. Choose one of the following values: `R0|R90|R180|R270|MX|MY|MX90|MY90` *Default:* The same orientation as the original instance.
`-xoffset` *`value_in_microns`*	Specifies the instance placement in the x direction after the cell substitution. A positive value moves the instance to the right of the original instance origin. A negative value moves the instance to the left of the original instance origin.

`-xyRangeFromCenterInst`	Selects the power switch instances whose lower left corner is in the range from lower left corner of the specified instance (`-insts`) for replacement with switch cell (`-cell`).  Note: The first instance specified in the `-insts` will be used for the area computation. The rest of the instances in the `-insts` list will be ignored and will not be replaced. For Example: `replacePowerSwitch -insts psoI_TDSP2_1_CDN_COLUMN_SW_669_1622_3 \ -xyRangeFromCenterInst { 60.3 20 } -cell \ CDN_COLUMN_SW`
`-yoffset` *value_in_microns*	Specifies the instance placement in the y direction after the cell substitution. A positive value moves the instance above the original instance origin. A negative value moves the instance above the original instance origin.

Examples

- The following example substitutes cell `CDN_RING_SW1` for the original cell for instance `pso1_TDSP2_1_CDN_RING_SW_636_1309_6`:
  `replacePowerSwtich -insts pso1_TDSP2_1_CDN_RING_SW_636_1309_6 -cell CDN_RING_SW1`
- The following example substitutes cell `CDN_RING_SW1` for the original cell for instance `pso1_TDSP2_1_CDN_RING_SW_636_1315_8` and changes the orientation of the instance to `MY90`:
  `replacePowerSwitch -insts pso1_TDSP2_1_CDN_RING_SW_636_1315_8 -cell CDN_RING_SW1 -orientation MY90`

Related Information

- `deletePowerSwitch`
- `exportPowerSwitch`
- `importPowerSwitch`
- `movePowerSwitch`
- `rechainPowerSwitch`
- `verifyPowerSwitch`
- Low Power Design chapter in the *Innovus User Guide*

reportIsolation

```
reportIsolation
[-help]
[-highlight]
[-outfile fileName]
[-from powerDomain]
[-to powerDomain]
```

Reports or shows the added isolation cells and instances.

Note: Use this command after you add isolation cells.

Parameters

-help	Outputs a brief description that includes the type and default information for each reportIsolation parameter. For a detailed description of the command and all its parameters, use the `man` command: `man` reportIsolation.
`-from`	Reports all the isolation instances connected to the output pins of the power domain specified in `-from` option. *Default:* Reports all isolation cells.
`-highlight`	Highlights isolation instances in the main window.
`-outfile` *fileName*	Writes isolation cell and instance names to a report. *Default:* The software writes the report to a default file: *designName*`.isolation.rpt`
`-to`	Reports all the isolation instances connected to the input pins of the power domain specified in `-to` option. *Default:* Reports all isolation cells.

Examples

The following command highlights isolation cells and writes the cell names to `isolationCellReport`:

```
reportIsolationCell -highlight -outfile isolationCellReport
```

Related Information

Low Power Design chapter in the *Innovus User Guide*

reportPowerDomain

```
reportPowerDomain
[-help]
[-bindLib]
[-file outFile]
[-inst inst_name]
[-isoInst]
[-module {list_of_modules}]
[-net net_name]
[-partitionHports pin | port | hpin+]
[-pgNet]
[-pin]
[-powerDomain powerDomainName [-checkPgScope]]
[-powerDomainBoundaryPort string]
[-powerDomainCoverage pd1 pd2]
[-shifter]
[-supplyCrossing supp1 supp2]
[-verbose]
[-voltage]
```

Reports information about power domains. The report includes the following attributes:

- Power domain name
- Power domain bounding box
- Timing library
- Mingap
- RS extension
- Row type
- Row spacing

The command also reports whether the power domain is always on.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each reportPowerDomain parameter. For a detailed description of the command and all its parameters, use the `man` command: `man` reportPowerDomain.
`-bindLib`	Reports the timing library bound to each instance of a power domain.
`-checkPgScope`	Checks if there are any PG connections that cross UPF-defined scope. All PG connections for UPF scope should be at the boundary PG port. This parameter is used with the `-powerDomain` parameter.
`-file` *outFile*	Specifies the name of the output file. *Default:* *designName*`-powerdomain.rpt`
`-inst` *inst_name*	Reports the power domain-related information for the specified instance: the input, output, and power/ground connections. When `-inst` is used along with the `-verbose` parameter, as `reportPowerDomain -inst -verbose`, the instance PG and tie-high/low pin connections along their its global connection rule are reported.
`-isoInst`	Reports all isolation cells in the power domain.

`-module {list of modules}`	Checks whether the specified instances or hierarchical instances are in the power domain. *Default:* The software does not perform this check.
`-net` *net_name*	Reports power domain-related information for the specified net: • The driver instance of the net and its power domain. • The receiver instances of the net and their power domains.
`-partitionHports` *pin | port | hpin+*	Reports the hierarchical partition (Hinst) boundary port's domain at the time of partitioning.
`-pin` *pin1, pin2 ...*	Specifies the lists the pins and the option is of type string. This option is optional.
`-powerDomain` *powerDomainName*	Specifies the power domain whose attributes you want to report. *Default:* If you do not specify the power domain, this command report data about all power domains.
`-powerDomainBoundaryPort` *string*	Reports if the specified port name of the power domain boundary has an IEEE1801 constraint.
`-powerDomainCoverage` *pd1 pd2*	Reports the coverage between two specified domains.
`-pgNet`	Reports all the power/ground nets in the power domain.
`-shifter`	Reports all shifters in the power domain.
`-supplyCrossing` *supp1 supp2*	Reports if the two specified supplies have the on/off and low/high crossing.
`-verbose`	Prints out the detail information and is optional.
`-voltage`	Reports domain voltage.

Examples

- The following command checks the `I1/PD1` instances to identify if there are any PG connections that do not use the `UPF I1` PG nets. If those are found, an error occurs specifying the instance name and its out-of-scope PG connection.

  ```
  reportPowerDomain -checkPgScope true -powerDomain I1/PD1
  ```

- The following command reports all shifter cells, all isolation cells, and all power/ground nets in power domain `PD1`. The command also checks whether HInst `instA` is contained in `PD1`, and reports all results to `myDesign-powerdomain.rpt`.

  ```
  reportPowerDomain -powerDomain PD1 -file myDesign-powerdomain.rpt -module {instA} -shifter -isoInst -pgNet
  ```

- The following example shows how output file `outfile` contains the binding between the timing libraries and instances in power domain VCORE:

  ```
  reportPowerDomain -bindLib -powerDomain VCORE -file outfile

  Timing library for instance(s):
  ============================
  INST = uLSCVSocDownVCore/uACLKEND/uLVHL (lvl_slow_1v08_1v08)
  INST = uLSCVSocDownVCore/uACLKENI/uLVLHL (lvl_slow_1v08_1v08)
  INST = uLSCVSocDownvCore/uaCLKENP/uLVLHL (lvl_slow_1v08_1v08)
  INST = uLSCVSocDownvCore/uaCLKENRW/uLVLHL (lvl_slow_1v08_1v08)
  INST = uLSCVSocDownvCore/uaBIGENDINIT/uLVHlL (lvl_slow_1v08_1v08)
  INST = uLSCVSocDownvCore/uaCLKIN/uLVHL (lvl_slow_1v08_1v08)
  ```

```
INST = uLSCVSocDownvCore/uCP15SDISABLE/uLVHL (lvl_slow_1v08_1v08)
```

- The following command reports the power domain-related information about instance `PM_INST/state_inst/g45`:

  ```
  reportPowerDomain -inst PM_INST/state_inst/g45
  ```

  Result:

  ```
  Analyzing PM_INST/state_inst/g45 (Cell: AN2D1) power domain:AO
  ------from side ------
  Pin A2 => Net PM_INST/state_inst/n_28
  PM_INST/state_inst/g71/ZN in power domain A0
  Pin A1 => Net PM_INST/state_inst/n_25
  PM_INST/state_inst/g47/Z in power domain A0
  ------to side ------
  Pin Z => Net PM_INST/state_inst/n_36
  PM_INST/state_inst/fsm_reg[1]/qi_reg/D in power domain A0
  ==========PG pins =============
  VDD conneted to Net: VDD
  VSS connected to Net:VSS
  ```

- The following command reports the instance PG and tie-high/low pin connections along their its global connection rule:

  ```
  reportPowerDomain -inst inst2/ls_rule1_0_in1 -verbose
  Analyzing inst2/ls_rule1_0_in1 (Cell: PTLVLANY) power domain: PD2
  ```

  Result:

  ```
  ------- from side --------
    Pin I (related power pin VDDI) => Net inst1_out
      inst1/ff2/Q in power domain PD1
  ------- to side --------
    Pin Z (related power pin VDDO) => Net inst2/n_1
      inst2/ff1/D in power domain PD2
  =============PG pins =================
  VSS connected to Net: VSS
  VDDO connected to Net: VDD2
  VDDI connected to Net: VDD1
  VDD connected to Net: VDD2

  VSS is connected to net 'VSS' by GNC rule: -net VSS -pin VSS -module inst2
  VDDO is connected to net 'VDD2' by GNC rule: -net VDD2 -pin VDDO -singleInst inst2/ls_rule1_0_in1
  VDDI is connected to net 'VDD1' by GNC rule: -net VDD1 -pin VDDI -singleInst inst2/ls_rule1_0_in1
  VDD is connected to net 'VDD2' by GNC rule: -net VDD2 -pin VDD -module inst2
  ```

Related Information

For more information, see the following topics in the Low Power Design chapter of the *User Guide*.

- "Overview"
- "Power Domain Shutdown and Scaling"
- "Power Switch Optimization"

reportPowerSwitch

```
reportPowerSwitch
[-help]
-outFile file_name
```

Reports each power switch instance, the location, and pin/net connections. The report format is as follows:

```
#InstName (x,y)
#   (pin1): net1
#   (pin2): net2
```

Parameters

`-help`	Outputs a brief description that includes the type and default information for each reportPowerSwitch parameter. For a detailed description of the command and all its parameters, use the `man` command: `man reportPowerSwitch.`
`-outFile file_name`	Specifies the name of the report file where the information is written. This is a required parameter.

Related Information

- `deletePowerSwitch`
- `exportPowerSwitch`
- `importPowerSwitch`
- `movePowerSwitch`
- `rechainPowerSwitch`
- `replacePowerSwitch`
- `verifyPowerSwitch`
- Low Power Design chapter in the *Innovus User Guide*

reportShifter

```
reportShifter
[-help]
[-cell cellName]
[-from powerDomainName]
[-to powerDomainName]
[-outfile fileName]
[-highlight]
```

Reports the level shifters that have been read into or defined for the Innovus software.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `reportShifter` parameter. For a detailed description of the command and all its parameters, use the man command: `man reportShifter`.
`-cell`	Reports shifters with the specified cell. *Default:* Reports all level shifters.
`-from powerDomainName`	Reports all the shifter instances connected to the output pins of the power domain specified in the `-from` option. *Default:* Reports all level shifters.
`-highlight`	Highlights placed level shifters.
`-outfile fileName`	Reports all instances in an output file. *Default:* The instances will be reported to a default output file `designName.shifter.rpt`. A warning message is printed to indicate a default output file is created.
`-to powerDomainName`	Reports shifter instances connected to the input pins of the power domain specified in the `-to` option. *Default:* Reports all level shifters.

Related Information

For more information, see the following topics in the Low Power Design chapter of the *User Guide*.

- "Overview"
- "Power Domain Shutdown and Scaling"
- "Power Switch Optimization"

reportVoltage

```
reportVoltage
[-help]
[-early]
[-inst string]
[-late]
[-net string]
[-quiet]
[-term string]
[-view string]
```

This command reports the voltage for the net/pin (term)/inst in the specific view under a specific corner.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `reportVoltage` parameter. For a detailed description of the command and all its parameters, use the man command: `man reportVoltage`
`-early`	Specifies that the voltage is repoted in the early mode.
`-inst string`	Reports the specified instance's power domain, cell name, timing library, view, mode, power net, and voltage information.
`-late`	Specifies that the voltage is reported in the late mode.
`-net string`	Specifies the net voltage (driver's pin voltage).
`-quiet`	Displays only the pin voltage-related information without other additional unnecessary information to avoid huge log size.
`-term string`	Specifies the term voltage.
`-view string`	Specifies the analysis view.

Related Information

For more information, see the Low Power Design chapter of the *User Guide*.

restorePowerSwitch

```
restorePowerSwitch
[-help]
[-file string]
```

Restores the information stored in the power switch databases. You can use this command to load back the power switch information only after completing power switch insertion using the `addPowerSwitch` command and then saving the database information using the `savePowerSwitch` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `restorePowerSwitch` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man restorePowerSwitch`
`-file string`	Specifies the name of the file from which the power switch database information will be restored.

Related Information

- `addPowerSwitch`
- `savePowerSwitch`

routePGPinUseSignalRoute

```
routePGPinUseSignalRoute
[help]
[-all]
[-maxFanout number]
[-nets {listofNets}]
[-nonDefaultRule ruleName]
[-pattern {trunk | steiner}]
[-well]
```

Routes the secondary power/ground pins of always-on cells to the specified power/ground nets in signal route. You can also control the routing pattern and wire by specifying the pattern, the maximum number of pins per cluster, and non-default rules.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `routePGPinUseSignalRoute` parameter. For a detailed description of the command and all its parameters, use the man command: `man routePGPinUseSignalRoute`
`-all`	Routes non-primary power/ground pins of all the cells.
`-nets` *`listOfNets`*	Specifies the power/ground nets to be connected to the secondary power/ground pins. *Default*: If this parameter is not specified, the software will get all the nets from global connections for the cells and pins defined in the command `setPGPinUseSignalRoute`.
`-pattern {trunk | steiner}`	Specifies how the signal router routes the secondary power/ground pins. `Trunk`: When option `maxFanout` value is 1, routes the pins to nearest power grid. When the value is more than 1, routes all pins per cluster to the trunk. `Steiner`: Routes all pins to a trunk. *Default:* `trunk`
`-maxFanout` *`number`*	Specifies the maximum number of pins per cluster in the trunk rotating pattern. *Default:* 5.
`-nonDefaultRule` *`ruleName`*	Specifies the non-default rule the signal router uses to route the secondary power/ground pins. *Default:* Regular signal routing rule is used by default.
`-well`	Includes nwell and pwell pins Type: Boolean, Optional

Related Topics

- `getPGPinUseSignalRoute`
- `setPGPinUseSignalRoute`
- Low Power Design chapter in the *Innovus User Guide*

savePowerSwitch

```
savePowerSwitch
[-help]
[-outFile string]
```

Stores the power switch database information, which can later be restored in the same floorplan through the `restorePowerSwitch` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `savePowerSwitch` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man savePowerSwitch`
`-outFile` *string*	Specifies the output file for storing the power switch database information.

Related Information

- `addPowerSwitch`
- `restorePowerSwitch`

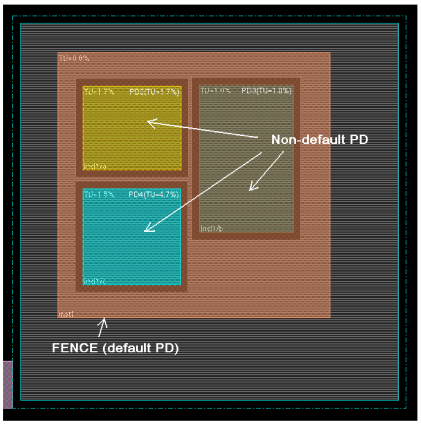
setMsvMode

```
setMsvMode
[-help]
[-allowNestedDefaultDomain {true | false}]
[-allowPowerDomainMinGapZero {true | false}]
[-checkAllNetsForDomainCrossing {true | false}]
[-handleBackToBackIsolation {true | false}]
[-handlePD {true|false}]
[-handlePDComplex {true|false}]
[-honorCpfGlobalConnectionSpecForAoBuffer {true | false}]
[-honorCpfGlobalConnectionSpecForShifter {true | false}]
[-honorDotLibRelatedPGPin {true | false}]
[-honorEffectiveDomainForIsoLsInsertion {true | false}]
[-markIsoEnablePinAsAlwaysOn {true | false}]
[-notUseTopFTermDomainForVoltage {true|false}]
[-reset]
[-shareWellAOBufferingSupport {true|false}]
[-upf_allow_shifter_voltage_mismatch {true|false}]
[-upf_insert_on_floating_pins {true|false}]
```

Sets the special handling for inter-domain nets and to use effective power domains for ISO/LS insertion.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each setMvsMode parameter. For a detailed description of the command and all its parameters, use the `man` command: `man setMvsMode`
`-allowNestedDefaultDomain {true | false}`	

	Allows a non-default power domain member to be logically nested in a default power domain. This default power domain member has fence constraint.
	
	By default, due to grouping, the fence constraint is lost for the non-default power domain member. Set `true` to enable.
	
	 
	
	In the netlist: modules `HInst11` and `HInst12` are under `HInst1`; and `HInst1` is under the top.
	
	You have the following:
	
	`create_power_domain -instances HInst1`
	
	`create_power_domain -default -instances {HInst11 HInst12}...`
	
	That means `HInst11` and `HInst12` are cut from `HInst1` to default power domain. If there is a fence/region constraint for `HInst11` and `HInst12` in default power domain, then please turn this global on. Otherwise, the fence/region for `HInst11` and `HInst12` will be lost.
	
	Default: `true`

`-allowPowerDomainMinGapZero {true | false}`	
	Allows users to set power domain zero `minGap`. *Default*: `true`
`-checkAllNetsForDomainCrossing {true | false}`	
	Checks all the nets for domain crossings. Set `true` to enable. *Default*: `true`
`-handleBackToBackIsolation {true | false}`	
	Supports back-to-back isolation insertion along one cross-domain net based on the CPF ISO rule. Set `true` to enable. *Default*: `false`

`-handlePD {true|false}`	Routes intra-power domain and global nets using the specifications of the power domains. On using this option, earlyGlobalRoute routes intra-power domain nets only within their power domains. Global nets that do not connect with a power domain do not get routed through that power domain. *Default*: `true`
`-handlePDComplex {true|false}`	Routes nets belonging to more than one power domain so that the routing does not violate power domain boundaries, in addition to routing intra-power domain and global nets using the specifications of the power domains. For example, when you specify `-handlePDComplex true`, if a net belongs to power domains PD1 and PD3, but not PD2, earlyGlobalRoute routes the net without going over the PD2 power domain boundary. *Default*: `true`
`-honorCpfGlobalConnectionSpecForAoBuffer {true | false}`	
	Honors connection pecs for AO buffer based on the CPF rules. Set `true` to enable. *Default*: `false`
`-honorCpfGlobalConnectionSpecForShifter {true | false}`	
	Honors connection specs for shifters based on the CPF rules. Set `true` to enable. *Default*: `false`
`-honorDotLibRelatedPGPin {true | false}`	
	Honors library related PG pins. Set `true` to enable. *Default*: `true`
`-honorEffectiveDomainForIsoLsInsertion {true | false}`	
	Honors effective domains for ISO/LS insertions. Set `true` to enable. *Default*: `true`
`-markIsoEnablePinAsAlwaysOn {true | false}`	
	Insert ISO/LS for all the inter-domain nets using effective domain based on the CPF rules. Set `true` to enable. *Default*: `true`
`-notUseTopFTermDomainForVoltage {true|false}`	
	Controls voltage using top fterm domain for IO pin voltage. *Default*: `false`
`-reset`	Resets the MSV mode to the default value. Any values that do not already match the default value and are reset. will be reported.
`-shareWellAOBufferingSupport {true|false}`	
	Shares the isolated Nwell AO buffers. *Default*: `false`
`-upf_allow_shifter_voltage_mismatch {true|false}`	
	When set to false, it prevents insertion of level shifters by allowing voltage tolerance. *Default*: `true`

`-upf_insert_on_floating_pins {true|false}`	
	Allows controlling of low power cell insertion on the floating pins. *Default*: `true`

Examples

- In the following example, allows the non-default power domain members nested in the default power domain member are allowed logically and physically and this default power domain has the fence constraint:
  ```
  setMsvMode -allowNestedDefaultDomain true
  ```
- In the following example, all the nets for the iso/ls insertion are checked:
  ```
  setMsvMode -checkAllNetsForDomainCrossing true
  ```
- In the following example, back-to-back isolation along with one net are inserted:
  ```
  setMsvMode -handleBackToBackIsolation true
  ```
- In the following example, the CPF specification (`create_global_connection -domain`) are used for the ISO/LS PG pin connection:
  ```
  setMsvMode -honorCpfGlobalConnectionSpecForShifter true
  ```
- In the following example, the pin power domain is assigned based on the related power pin specified in `*.lib`:
  ```
  setMsvMode -honorDotLibRelatedPGPin true
  ```
- In the following example, effective domain for the iso/ls insertion is used：
  ```
  setMsvMode -honorEffectiveDomainForIsoLsInsertion true
  ```
- In the following example, the iso enable based on the tag (power modes) is marked:
  ```
  setMsvMode -markIsoEnablePinAsAlwaysOn false
  ```

Related Topics

- `getMsvMode`
- Low Power Design chapter in the *Innovus User Guide*

setPGPinUseSignalRoute

```
setPGPinUseSignalRoute
[-off]
[-all]
cellPins cellName:pinName cellName:pinName2 ...
[-well]
```

Sets the internal bit for a given cell/pin pair to be routed by the signal router. Use this command to enable NanoRoute to route always-on-pins in SRPG (state retention power gating) or always-on buffer cells.

Parameters

`-all`	Routes all backup and non-main_rail primary power/ground pins. Type: Boolean, Optional
`cellPins` *`cellName:pinName`*	• *`cellName`*: Specifies the cell containing the pin for NanoRoute to route. You can use a wildcard to specify all cells of certain type. • *`pinName`*: Specifies the pin name of the secondary PG pin to be routed.
`-off`	Unsets this feature for the specified list of `cellPins` or `cellName:pinName`.
`-well`	Includes nwell and pwell pins. Type: Boolean, Optional

Examples

- The following commands set both the `vddo` and `vsso` pins for the signal router, but NanoRoute will only route `vddo`:

  ```
  setPGPinUseSignalRoute SDRFF:vddo SDRFF:vsso
  routePGPinUseSignalRoute -nets VDD
  ```

- The following command sets NanoRoute to route the `VDD1` pin of all `NAND` cells:

  ```
  setPGPinUseSignalRoute NAND*:VDD1
  ```

- The following command unsets NanoRoute to route the `VDD1` pin of all the `NAND` cells, which were set earlier:

  ```
  setPGPinUseSignalRoute NAND*:VDD1 -off
  ```

Related Topics

- `getPGPinUseSignalRoute`
- `routePGPinUseSignalRoute`
- Low Power Design chapter in the *Innovus User Guide*

swapRegularCellWithAlwaysOnCell

```
swapRegularCellWithAlwaysOnCell
[-help]
[-cellMapFile filename]
[-defaultAlwaysOnCell AoCellNames]
[-nets LPErrorNetList]
[-verbose]
```

Identifies the regular buffer/inverter and swaps it with always-on buffer/inverter. The regular buffer/inverter causes the LP error. Hence, this command fixes the LP error through the regular to always-on buffer/inverter swapping.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `swapRegularCellWithAlwaysOnCell` parameter. For a detailed description of the command and all its parameters, use the man command: `man swapRegularCellWithAlwaysOnCell`
`-cellMapFile` *filename*	Specifies the map file name. The map file provides the equivalent driving strength between the regular buffer/inverter and AO buffer/inverter. This means that the timing will not be degraded with using this parameter, when the mapped always-on cell is swapped in. Following is the map file format: `regularBufCellName AoBufCellName`
`-defaultAlwaysOnCell` *AoCellNames*	Specifies the default always-on buffer/inverter to be used to replace the regular buffer/inverter. When using default always-on buffer/inverter, this parameter just corrects the LP error. Additionally, this parameter may degrade the timing, and you need to run optimization in flow.
`-nets` *LPErrorNetList*	Specifies the list of LP-error nets. By default, the `swapRegularCellWithAlwaysOnCell` command works on all the buffer/inverter trees. When this parameter is used, the command works on the buffer/inverter tree for the specified nets.
`-verbose`	Displays the detailed trace information for the specified nets.

Examples

- The following command swaps the incorrect regular buffer with `AoBuf1`, and the incorrect regular inverter with `AoInverter1`:
  ```
  swapRegularCellWithAlwaysOnCell -defaultAlwaysOnCell "AoBuf1 AoInverter1"
  ```
- The following command swaps the incorrect regular buffer/inverter using the specified mapping cells or the default always-on cells:
  ```
  swapRegularCellWithAlwaysOnCell -cellMapFile mapfile1 -defaultAlwaysOnCell "AoBuf1 AoInverter1"
  ```
- The following command swaps the incorrect regular buffer/inverter using the default always-on cells for the buffer/inverter trees along with the `net1` and `net2` paths:
  ```
  swapRegularCellWithAlwaysOnCell -nets "net1 net2" -defaultAlwaysOnCell "AoBuf1 AoInverter1"
  ```
- The following command swaps the incorrect buffer/inverter along with the `net1` and `net2` paths and displays the detailed trace information:
  ```
  swapRegularCellWithAlwaysOnCell -nets "net1 net2" -defaultAlwaysOnCell "AoBuf1 AoInverter1 -verbose"
  ```

verifyPowerDomain

```
verifyPowerDomain
[-help]
[-allInstInPD]
[-aobBiasConn]
[-aoBufferType]
[-bind]
[-cell_bind {cell ...}]
[-drcFile string]
[-gconn [module_name_list]]
[-isoNetPD [fileName]
[-libBindingVoltage]
[-noFTermIsoC]
[-noFTermShifter]
[-place [-place_rpt report_name]]
[-powerSwitch]
[-pwrWithinMinGap]
[-retention {fileName ...}]
[-xNetPD {fileName}]
```

Verifies whether the power domains are set up correctly in terms of timing library binding and PG connection. This command also checks whether any power domains overlap. The `verifyPowerDomain` command checks that libraries specified for each power domain of each `dc_corner` are complete for each power domain member list. The command also checks if the inserted isolation is consistent with the isolation rule.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `verifyPowerDomain` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man verifyPowerDomain`.
`-allInstInPD`	Verifies all instances are in the power domain.
`-aobBiasConn`	Checks the AOB bias net connection.
`-aoBufferType`	Checks if the AO buffer in each domain has correct type in terms of AO Nwell design and IEEE1801 Nwell specification.
`-bind`	Verifies that all cells are bound to correct timing library cells.
`-cell_bind {cell ...}`	
	Checks if the specified cells are bound in the domain's library set for all active views and domains.
`-drcFile string`	Creates DRC violation markers and save those in the specified file.
`-gconn module_name_list`	

	Verifies that power and ground pins of instances within the power domain are connected to the power and ground nets of that power domain. In addition, this parameter verifies that the tie-high and tie-low connections of an instance are connected to the same power and ground nets of the power domain to which the cell belongs. If you do not specify a list of module names, all modules are verified.
`-isoNetPD`	Verifies if the nets are missing isolation instances according to the power intent file. It also checks if the isolation uses right type of cell specified in the IEEE1801 map_isolation_cell command (if any).
`-libBindingVoltage`	Checks voltage against the binding library.
`-noFTermIsoC`	Ignores the nets connecting to I/O when verifying isolation using `-isoNetPD`.
`-noFTermShifter`	Ignores the nets connecting to I/O when verifying shifter using `-xNetPD`.
`-place`	Verifies if the instances are placed in the boundary of their power domain.
`-place_rpt` *report_name*	
	Specifies a name and location for the report generated when you use this command with the `-place` parameter.
`-powerSwitch`	Verifies whether each row in the power domains contain at least one power switch. The command issues a warning message detailing which rows do not contain at least one switch. It also checks if the power switch uses right type of cell specified in the IEEE1801 map_power_switch_cell command (if any).
`-pwrWithinMinGap`	
	Verifies that the power domain ring is within the distance specified by the `modifyPowerDomainAttr -minGaps` or `createPowerDomain -minGaps` command.
`-retention {`*fileName ...*`}`	
	Specifies a name for storing the verify retention information. It checks if the retention uses the right type of cell specified in the IEEE1801 map_retention_cell command (if any).
`-xNetPD`	Verifies if the nets are missing level shifter according to the power intent file. It also checks if the level shifter uses right type of cell specified in the IEEE1801 map_level_shifter_cell command (if any).

Examples

The following command issues warning messages if instances that are not members of a power domain are placed within that power domain boundary, or if any power domains overlap:

```
verifyPowerDomain -place
```

Related Topics

For more information, see the following topics in the Low Power Design chapter of the *User Guide*:

- Overview
- Power Domain Shutdown and Scaling
- Multiple supply Voltage Flat Flow
- Verify Power domains

verifyPowerSwitch

```
verifyPowerSwitch
[-help]
[-powerDomain powerDomain]
[-checkFloatingInput]
[-checkFloatingOutput]
[-checkFloating]
[-checkOverlap]
[-checkPlace]
[-checkLoop]
[-checkRowCoverage]
[-checkAll]
[-hilite]
[-reportFile]
[-reportLongestChain]
[-startingNet]
```

Verifies power switch placement and enable connections, including the row coverage for column insertion. The command issues a warning message detailing which rows do not contain power switches. The options also enable reporting for the longest signal chain starting from a given enable net. If the start of the enable net is not available, all enable nets of all domains will be used as starting points and all chains will be reported. Both the length of the chain and the switch instances involved in the longest chain will be reported.

The `verifyPowerDomain -powerSwitch` parameter performs the same function.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each verifyPowerSwitch parameter. For a detailed description of the command and all its parameters, use the `man` command: `man verifyPowerSwitch`.
`-checkFloatingInput`	To check floating input enable pins.
`-checkFloatingOutput`	To check floating output enable pins.
`-checkFloating`	To check both floating input and output enable pins.
`-checkOverlap`	To check any overlap among the pins.
`-checkPlace`	To check for unplaced power switches.
`-checkLoop`	To check for accidentally created enable signal loops.
`-checkRowCoverage`	To check if every row is covered by power switch.
`-checkAll`	To verify all the checks.
`-hilite`	To select the violation.
`-startingNet`	To start the net to report longest chain.
`-reportFile`	To report output to a file.
`-reportLongestChain [startingNet` *`enableNet`*`]`	
	Reports the longest chain starting from the given enable net. If `-startingNet` is not given, all enable nets of all domains will be used as the starting point and all chains will be reported. The report will include both: the length of the chain, and the switch instances involved in the longest chain.
`-powerDomain powerDomain`	Verify only the specified power domain.

Examples

The following command verifies the placement status of power switch cells and checks the row coverage by power switch cells for the `powerDomain TDSPCore`:

```
verifyPowerSwitch -powerDomain TDSPCore -checkPlace -checkRowCoverage
Verifying domain: TDSPCore.
Verifying power switch row coverage ...
Verifying power switch placement status ...
Switch verification completed on domain: TDSPCore.
Number of warnings on domain TDSPCore : 0.
Power switch verification completed with 0 warnings..
1
```

Related Topics

- `deletePowerSwitch`
- `exportPowerSwitch`
- `importPowerSwitch`
- `movePowerSwitch`
- `rechainPowerSwitch`
- `replacePowerSwitch`
- Low Power Design chapter in the *Innovus User Guide*

write_power_intent

```
write_power_intent
[-help]
fileName
{-cpf [[-hierInst -topHier] | [-inst]] | -1801 | -incremental_1801}
[-no_extra_connects]
```

Writes the CPF/IEEE1801 file. The command options help to write the file for CPF as well as for the IEEE1801 standard.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `write_power_intent` parameter. For a detailed description of the command and all its parameters, use the man command: `man write_power_intent`
`fileName`	Specifies name of the IEEE1801 input file.
For CPF	
`-cpf`	When specified, Innovus writes out the CPF file.
`-inst`	Specifies the hierarchical instance name. Writes (partitions) the CPF for the hierarchical instance specified in this option. **Note:** Must be used along with `-cpf` option.
`-hierInst`	Writes top-level CPF where the hierarchical instances specified in `-hierInst` option are hard blocks when specified together with `-topHier -cpf`.
`-topHier`	Writes top-level CPF when specified together with `-hierInst` and `-cpf`.
For IEEE1801	
`-1801`	Writes the IEEE1801 file from the low-power database. It filters out the non-existing instances from the original IEEE1801 file and appends to it the following: • PG connection for the newly-inserted always-on buffers • Nwell/Pwell connections for the low-power cells • `-no_isolation`/`shifter` for the isolation/level shifter strategies for the newly-created ports along the power domain boundary
`-incremental_1801`	Specifies the incremental IEEE1801 power intent.
`-no_extra_connects`	If this parameter is specified, `write_power_intent -1801` does not write out any extra `connect_supply_net` (PG connection). For example: `write_power_intent -1801 -no_extra_connects out.upf` **Note:** Extra connects means that the extra `connect_supply_net` commands appended by the tool, which are not included in the original UPF.

Related Topics

- `commit_power_intent`
- `free_power_intent`
- `read_power_intent`
- Low Power Design chapter in the *Innovus User Guide*

Low Power Global Variables

init_power_intent_file

```
init_power_intent_file file_name
```

Type: String

Default: ""

Specifies the power intent file, which supports the UPF 1801 and CPF format.

Related Information

- read_power_intent

18

Machine Learning Commands

- init_ml
- write_ml_design
- write_ml_library
- write_ml_timing_graph
- write_ml_timing_path

init_ml

```
init_ml
[-help]
[-comment string]
[-flow_phase string]
[-name string]
-prefix string
[-scenario string]
```

Initializes the environment to start writing data for JedAI. Run this command before using any of the following commands to ensure that content is properly labeled for big data ingestion and registration:

- `write_ml_design`
- `write_ml_library`
- `write_ml_timing_graph`
- `write_ml_timing_path`

Parameters

`-help`	Prints a brief description that includes the type and default information for each `init_ml` parameter. For a detailed description of the command and all of its parameters, use the man command `man init_ml`.
`-comment` *string*	Specifies a comment. *Data_type*: string, optional
`-flow_phase` *string*	Specifies the flow phase. *Data_type*: string, optional
`-name` *string*	Specifies the design name. *Data_type*: string, optional
`-prefix` *string*	Specifies the prefix for the output files. *Data_type*: string, required
`-scenario` *string*	Specifies the scenario. *Data_type*: string, optional

Example

- The following command the environment to start writing data for JedAI for the `cts` flow phase with the `default` scenario for the `dtmf` design.

  ```
  init_ml -prefix out/ -name dtmf -flow_phase cts -scenario default -comment "This is a demo."
  ```

Related Information

- write_ml_design
- write_ml_library
- write_ml_timing_graph
- write_ml_timing_path

write_ml_design

```
write_ml_design
[-help]
[-no_power]
[-objects object_list]
-prefix string
[-verbose]
[-views view_list]
```

Writes design-related data for JedAI to csv files with specified prefix. It includes data that is design specific and relates to logical, physical, and timing type objects.

Before running `write_ml_design`, the following two commands should be run:

- `report_timing`
- `write_global_slack_report -include_ports`

Parameters

`-help`	Prints a brief description that includes the type and default information for each `write_ml_design` parameter. For a detailed description of the command and all of its parameters, use the man command `man write_ml_design`.
`-no_power`	Specifies that power attributes should not be dumped. *Data_type*: bool, optional
`-objects` *object_list*	

	Specifies the list of db object(s) to be dumped, which could include one or more of the following values:
	`analysis_view arc base_cell base_pin blockage clock congestion constraint_mode`
	`delay_corner design gcell_grid hinst hnet hpin hport inst layer marker net netlist`
	`opcond option p2p partition patch_wire pg_base_pin pg_pin pin pin_shape port`
	`rc_corner route_rule row site skew_group special_net timing_condition track_pattern unit_parasitic via_def`
	Use curly braces to enclose multiple values.
	Data_type: tcl_obj, optional

`-prefix` *string*	Specifies the prefix for the output files. *Data_type*: string, required
`-verbose`	Displays a more detailed message. *Data_type*: bool, optional
`-views` *view_list*	
	Dumps the data for the specified analysis view(s). *Data_type*: tcl_obj, optional

Example

- The following command writes the attributes of all design objects to csv files with the specified prefix:

```
write_ml_design -prefix out/
```

Related Information

- init_ml
- report_timing
- write_global_slack_report
- write_ml_library
- write_ml_timing_graph
- write_ml_timing_path

write_ml_library

```
write_ml_library
[-help]
[-objects object_list]
-prefix string
```

Writes library-related data for JedAI to csv files with the specified prefix. It includes data that is library specific and relates to lib_cells, timing_tables, and power_tables.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `write_ml_library` parameter. For a detailed description of the command and all of its parameters, use the man command `man write_ml_library`.
`-objects object_list`	
	Specifies the list of db object(s) to be dumped, which could include one or more of the following values: `lib_arc lib_cell lib_feature lib_pin library library_set power_table timing_table` Use curly braces to enclose multiple values. *Data_type*: tcl_obj, optional
`-prefix string`	Specifies the prefix for the output files. *Data_type*: string, required

Example

- The following command writes library-related data for JedAI to csv files with the specified prefix:
  ```
  write_ml_library -prefix out/
  ```

Related Information

- init_ml
- write_ml_design
- write_ml_timing_graph
- write_ml_timing_path

write_ml_timing_graph

```
write_ml_timing_graph
[-help]
[-noedge]
-prefix string
[-register_only]
[-views view_list]
```

Writes timing-related data for JedAI to csv files. It includes data representing the complete timing graph for a given set of analysis views.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `write_ml_timing_graph` parameter. For a detailed description of the command and all of its parameters, use the man command `man write_ml_timing_graph`.
`-noedge`	Specifies that edge data should not be dumped. This option can be used to save time. *Data_type*: int, optional
`-prefix string`	Specifies the prefix for the output files. *Data_type*: string, required
`-register_only`	Specifies that only nodes for register should be dumped. *Data_type*: bool, optional
`-views view_list`	Dumps the data for the specified analysis view(s). *Data_type*: tcl_obj, optional

Example

- The following command writes the timing-related data for JedAI to csv files with the specified prefix:

```
write_ml_timing_graph -prefix out/
```

Related Information

- init_ml
- write_ml_design
- write_ml_library
- write_ml_timing_path

write_ml_timing_path

```
write_ml_timing_path
[-help]
[-anchor_file string]
[-each_endpoint]
[-gen_anchor]
[-hpin]
[-io_only]
[-max_paths int]
[-max_slack float]
[-nworst int]
[-path_group string]
[-per_partition]
-prefix string
[-retime {aocv ssta path_slew_propagation aocv_path_slew_propagation}]
[-start_pathid int]
[-verbose]
[-views view_list]
```

Writes timing-related data for JedAI to csv files. It includes data for individual timing paths. Timing path data can be computed in either graph-based analysis (GBA) or path-based analysis (PBA) fashion using the `-retime` option.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `write_ml_timing_path` parameter. For a detailed description of the command and all of its parameters, use the man command `man write_ml_timing_path`.
`-anchor_file string`	Specifies the name of the anchor file to be used as input. *Data_type*: string, optional
`-each_endpoint`	Specifies whether to dump at least one path for each end point. *Data_type*: bool, optional

`-gen_anchor`	Generates the anchor file. *Data_type*: bool, optional
`-hpin`	Dumps hpin in timing points. *Data_type*: bool, optional
`-io_only`	Specifies that only the io path should be dumped. *Data_type*: bool, optional
`-max_paths` *int*	Specifies the max_paths for `report_timing`. *Data_type*: int, optional
`-max_slack` *float*	Specifies the max_slack for `report_timing`. *Data_type*: float, optional
`-nworst` *int*	Specifies the nworst for `report_timing`. *Default*: `100` *Data_type*: int, optional
`-path_group` *string*	Specifies the path_group for `report_timing`. *Data_type*: string, optional
`-per_partition`	Specifies that only paths through top level partitions should be dumped. *Data_type*: bool, optional
`-prefix` *string*	Specifies the prefix for the output files. *Data_type*: string, required
`-retime {aocv ssta path_slew_propagation aocv_path_slew_propagation}`	
	Specifies the method to be used for reanalysis of specified set of paths with `report_timing`. *Data_type*: enum, optional
`-start_pathid` *int*	Specifies the starting path id. *Default*: `1` *Data_type*: int, optional

`-verbose`	Displays a more detailed message. *Data_type*: bool, optional
`-views` *view_list*	Dumps the timing path for the specified analysis view(s). *Data_type*: tcl_obj, optional

Example

- The following command writes out the attributes of timing paths to csv files with the specified prefix:

```
write_ml_timing_path -prefix out/ -max_paths 10000 -nworst 1
```

Related Information

- `init_ml`
- `report_timing`
- `write_ml_design`
- `write_ml_library`
- `write_ml_timing_graph`

19

Metal and Via Fill Commands and Global Variables

- Metal and Via Fill Commands
- Metal and Via Fill Global Variables

Metal and Via Fill Commands

- add_metal_fill_signoff
- add_pg_fill
- add_track_fill
- addViaFill
- deleteMetalFill
- fixOpenFill
- get_metal_fill_signoff_mode
- get_pg_fill_config
- get_track_fill_config
- getMetalFillSpacingTable
- report_metal_fill
- run_pvs_drc_rules
- set_metal_fill_signoff_mode
- set_pg_fill_config
- set_track_fill_config
- setMetalFill
- setMetalFillSpacingTable
- setViaFill
- trimMetalFill
- trimMetalFillNearNet

add_metal_fill_signoff

```
add_metal_fill_signoff
[-help]
[-and_area_size size_value]
[-area {x1 y1 x2 y2}]
[-attach_instance_name attr_num]
[-attach_net_name attr_num]
[-attach_net_prop {{prop_name attr_num} ... }]
[-auto_load_fills]
[-bg]
[-clock]
[-common_blockage {layer_num1 layer_datatype1 ...}]
[-control file_name]
[-delete_area {{layer_list1}{x1 y1 x2 y2}...}]
[-delete_point {{layer_list1}{x1 y1}...}]
[-die_area_as_boundary]
[-dp n]
[-dp_timeout value]
[-excl_area_size size_value]
[-fill | -incremental | -trim | -flatten | -delete_fill | -merge_fill_edits | -view_fills | -delete_flatten_fills]
[-fill_cell cell_name]
[-fill_layer {layers_list}]
[-fill_output_mode {gdsii | oasis}]
[-layer_map_file streamOut_map_file]
[-license_timeout]
[-license_dp_continue]
[-lib_name library_name]
[-lsf]
[-master_lsf]
[-merge list_of_external_stream_files]
[-min_density percent1 layer_list1]
[-net net_names]
[-no_structure_name]
[-no_via_fills]
[-offset {x y}]
[-output_macros]
[-report_file]
[-rule_file rule_file_name]
[-skip_map_check]
[-slack_threshold float]
[-spacing float]
[-spacing_above float]
[-spacing_below float]
[-stripes number]
[-structure_name structureName]
[-tech_lib string]
[-tech_set string]
[-technology string]
[-temp_working_dir work_dir]
[-trim_effort {high | med | low}]
[-trim_layer {layers_list}]
[-trim_mf spacing_file]
[-union_density percent1 layer_list1]
[-uniquify_cell_names]
[-units {100 | 200 | 400 | 800 | 1000 | 2000 | 4000 | 8000 | 10000 | 20000}]
[-window_size float]
[-window_step float]
```

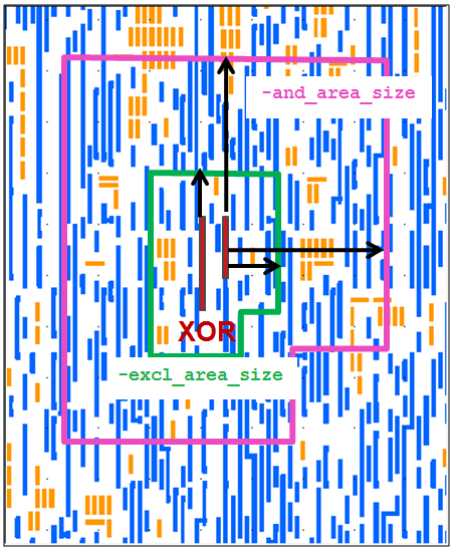
Runs the signoff metal fill flow as per specified primary and sub options.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `add_metal_fill_signoff` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man add_metal_fill_signoff`
`-and_area_size` *`size_value`*	Specifies a value for sizing up the changed area to use shapes within this area. This parameter helps in calculating the density and creating areas to be filled, if they fall below the minimum density. The default value of this parameter is 10. A value lesser than the default value may be used to improve the flow performance, if no accuracy issues occur. A value greater than the default value can be used to fix the accuracy issues. The minimum value of this parameter is `0.001` and maximum value is `1000`.
`-area {x1 y1 x2 y2}`	Specifies a set of coordinates within metal fill can be trimmed.
`-attach_instance_name` *`attr_num`*	Specifies the instance name to attach.
`-attach_net_name` *`attr_num`*	Specifies the net name to attach.
`-attach_net_prop {{`*`prop_name`* `attr_num} ...}`	
	Specifies the net property to attach.
`-auto_load_fills`	Updates the FILL data automatically in the software after when the `add_metal_fill_signoff` command modifies the HMF database. In case the HMF database is not modified, the FILL data will be hidden in the software's design.
`-bg`	Runs metal fill in the background mode without locking up the software license. **Note:** All the metal fill options running in the background must be used with the `-bg` parameter.
`-clock`	Specifies whether to trim metal fill around all clock nets.

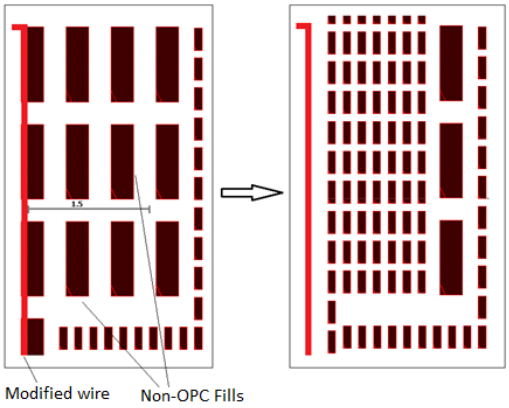
`-common_blockage {`*`layer_num1 layer_datatype1 ...`*`}`	Specifies common blockage fill layers. This allows you to specify the blockage layer only once, instead of multiple individual blockages for the metal fill layers. The parameter value format is `{`*`layer_number layer_datatype`*`}`. You can also specify multiple layers, and each blockage layer should specify a set of two values. The *`layer_number`* must be an integer. Following are the examples of the usage of this parameter:
	`add_metal_fill_signoff -common_blockage {150 28}`
	`add_metal_fill_signoff -common_blockage {150 28 120 55}`

`-control` *`file_name`*	Specifies the control file, which is created automatically when a job is submitted from the PVS GUI. This parameter can be used, if you have run a directory submitted from the GUI. Use this parameter to rerun the PVS job from the command line and get the results, which are exactly the same as the GUI submission. The control file is placed in the run directory with the `pvsdrcctl` name and has only the command entries for certain fields in the GUI. You must not modify the file by adding commands or parameters that are not available in the GUI. If you do that, the commands can be ignored by the PVS rule compiler or the results can be indeterminate.
`-delete_area {{`*`layer_list1`*`} {`*`x1 y1 x2 y2`*`} ...}`	Specifies the x and y coordinates of the area where metal fill is deleted. Following is the usage example of this parameter: `add_metal_fill_signoff -delete_area {{M1 M2 M6} {0 0 50 50}}`
`-delete_fill`	Deletes metal fills in the specified window or coordinates.
`-delete_flatten_fills`	Deletes flattened metal fills without updating the HMF database.
`-delete_point {{`*`layer_list1`*`}{`*`x1 y1`*`}...}`	Specifies the x and y coordinates of the point where metal fill is deleted. Following is the usage example of this parameter: `add_metal_fill_signoff -delete_point {{M2} {134.7 14.21}}`
`-die_area_as_boundary`	Specifies the die area as boundary.

`-dp` *n*	Specifies the number of local cores to use. The default value is fetched from the EDI settings.
`-dp_timeout` *value*	Specifies the time to wait for the slave processes to start.
`-excl_area_size` *size_value*	Specifies the area to be created around the metal fill area to check for the existing metal fill shapes and to avoid creation of metal fill shapes too close to the existing metal fill shapes. The default value of this parameter is `1.6`. A value lesser than the default value may be used to improve the flow performance, if no accuracy issues occur. A value greater than the default value can be used to fix the accuracy issues. The minimum value of this parameter is `0.001` and maximum value is `1000`. The following picture shows the usage of `-and_area_size` and `-excl_area_size` parameters: 
`-fill`	Runs the signoff metal fill flow to generate hierarchical metal fills database.
`-fill_cell` *cell_name*	Specifies metal fill cell name.
`-fill_layer {`*layers_list*`}`	Fills only a set of specified metal and/or via metal layers. Following is the usage example of this parameter: `add_metal_fill_signoff -fill_layer {M1 VIA12 M2}`
`-fill_output_mode {gdsii | oasis}`	Generates metal fill database in the `gdsii` or `oasis` format. The default format is `gdsii`.
`-flatten`	Flattens metal fills database and loads into the software.
`-incremental`	Runs incremental metal fill post ECO.

`-layer_map_file` *streamOut_map_file*	Specifies the `streamOut` metal layer map file.
`-license_timeout`	Specifies the time (in seconds) to wait if the software license is not immediately available while initiating the run. The default value of this parameter is 300 seconds.
`-license_dp_continue`	Initiates the software run using fewer available CPUs based on the number of licenses obtained. Use this parameter in case PVS cannot obtain the required number of licenses in the multiprocessing mode.
`-lib_name library_name`	Specifies the library name.
`-lsf`	Distributes the processing of the software run to different CPUs using LSF. While using this parameter, the `DP_LSF_RESOURCE` environment variable must be set to specify the LSF resource string, as shown below: `setenv DP_LSF_RESOURCE 'resource string for slave processes'` Additionally, the `-temp_working_dir` *work_dir* should be used to specify local working directory for the LSF run.
`-master_lsf`	Submits the master PVS process on LSF. While using this parameter, the `MASTER_LSF_RESOURCE` environment variable should be set to specify LSF resource string, as shown below: `setenv MASTER_LSF_RESOURCE 'resource string for master process'`
`-merge` *list_of_external_stream_files*	
	Merges the external stream files.
`-merge_fill_edits`	Updates metal fill database with the metal fill edits executed in the software.
`-min_density percent1` *layer_list1*	Specifies percentage value of the minimum metal density. The default value of this parameter is `0` for each metal layer. This parameter trims metal fill below when the specified window density percentage could cause any metal density violations. This parameter is controlled by the `-window_size` and `-window_step` values, as shown in the below example: `add_metal_fill_signoff -min_density {20 {M1 M2} 35 M3}`
`-net net_names`	Specifies a list of critical net names or pointers, which are surrounded by the metal fill area.
`-no_structure_name`	Merges the GDSII file without loading the design.
`-no_via_fills`	Specifies not to generate metal via fills.
`-offset {`*x y*`}`	Defines the offset displacement to be applied.
`-output_macros`	Displays the LEF abstract information about macros, such as LEF pin geometries and obstructions.
`-report_file`	Displays the cell name, which is changed to file.
`-rule_file` *rule_file_name*	Specifies metal fill rule file name.
`-skip_map_check`	Ignores the cross-referencing of the `-layer_map_file` parameter and the Innovus layer names.
`-slack_threshold` *float*	Trims any critical net(s) in the timing aware flow with the slack lesser than the specified value (in nanoseconds). Following is the usage example of this parameter: `add_metal_fill_signoff -slack_threshold 0.1`
`-spacing` *float*	Trims metal fill on the specified metal layer around critical net routes. **Note:** The `-spacing` and `-trim_effort` parameters are mutually exclusive.
`-spacing_above` *float*	Trims metal fill on above the specified metal layer around critical net routes.
`-spacing_below` *float*	Trims metal fill on below the specified metal layer around critical net routes.

`-stripes` *number*	Specifies the number of stripes that divide the design. The default is value of this parameter is `1`.
`-structure_name` *structureName*	Specifies the superstructure that consists of all the geometries that the map file writes out.
`-tech_lib` *string*	Specifies the PVS technology library for `-fill` and `-incremental`.
`-tech_set` *string*	Specifies the PVS technology set for `-fill` and `-incremental`.
`-technology` *string*	Specifies the PVS technology to use for identifying the sign-off rules file. **Note:** You can use this parameter with either the `-rule_file` or `-technology` parameter.
`-temp_working_dir` *work_dir*	Specifies the working directory for the background or LSF run.
`-trim`	Trims metal fills in the metal fills database for timing. This parameter is density-related.
`-trim_effort {high | med | low}`	Trims metal fill area with the high, medium, or low effort around the critical net routes. This parameter also helps in determining the blockage size for trimming metal fill. Following are the parameter values: • `high`: Presents area of two times of the layer pitch value. • `med`: : Presents area of 1.5 times of the layer pitch value. • `low`: : Presents the same area of the layer pitch value. Following is the usage example of this parameter: `add_metal_fill_signoff -trim_effort med` Here, if the M2 layer pitch is `0.8 um`, M2 fills will be trimmed around selected nets within the distance of `1.2 um`. **Note:** The `-trim_effort` and `-spacing` parameters are mutually exclusive.
`-trim_layer {`*layers_list*`}`	Trims only the set of specified layers. Following is the usage example of this parameter: `add_metal_fill_signoff -trim_layer {M1 M2}`

`-trim_mf` *spacing_file*	Specifies the spacing file containing information related to trim metal fill in the incremental run. Spacing file contains spacing values between the routed metal, non-OPC and OPC metal fill layers. Metal fills are trimmed around ECO changes according to the specified spacing values, post which incremental run generates fill data in the trimmed area. Following is the format of the spacing file:
	`#layer f-a o-a`
	`M1 0.60 0.16`
	`M2 0.60 0.16`
	`M3 0.60 0.16`
	`M4 0.60 0.16`
	`M5 0.60 0.16`
	`M6 0.60 NA`
	Here:
	• `Mx`: Specifies the LEF layer name.
	• `f-a`: Specifies the metal fill active spacing.
	• `o-a`: Specifies the is OPC fill active spacing.
	• `NA`: Refers to no trimming.
	The following picture shows the usage of `-trim_mf` parameter:
	 
	
	The spacing file for the above example has the following settings for the M2 layer:
	`M2 1.5 0.6`
	In this example, the non-OPC metal fills, which are 1.5 um or closer to the modified wire, are trimmed. Additionally, the metal fill data is regenerated in the trimmed area.

`-union_density {`*percent1 layer_list1* `[{`*percent2 layer_list2*`}]...}`	
	Specifies percentage value of the union metal density. The default value of this parameter is `0` for each metal layer. Union density for a layer is the sum of metal density of the current layer, layer below, and layer above the specified windows. Metal fills are trimmed not to violate the `-union_density` settings for the current metal. Following is the usage example of this parameter: `add_metal_fill_signoff -union_density {20 M1}`
`-uniquify_cell_names`	Creates unique cell names, if there are name collisions while using the `-merge` parameter.
`-units {100 | 200 | 400 | 800 | 1000 | 2000 | 4000 | 8000 | 10000 | 20000}`	
	Specifies the resolution of the input/output file. The default value of this parameter is obtained from software database.
`-view_fills`	Loads the existing HMF database into the software to view the metal fills.

`-window_size float`	Specifies the size of the window, which the metal trim uses to examine the metal density. The default value of this parameter is `100`. Following is the usage example of this parameter: `add_metal_fill_signoff -window_size 50`
`-window_step float`	Specifies the stepping distance between two successive positions of the window for a stepped-window trim density check. The default value of this parameter is `50`. Following is the usage example of this parameter: `add_metal_fill_signoff -window_step 25`

Examples

- The following command runs the background LSF job by setting the resource master and slave LSF resource string before launching the software:
  ```
  setenv MASTER_LSF_RESOURCE'-q lnx64 -W "40:00" -n 4 -P "PVS15.23:PE:test"'
  setenv DP_LSF_RESOURCE'-q lnx64 -W "15:00" -P "PVS15.23:PE:test"'
  ```
- The following commands launch a background LSF job in both the master and slave on LSF:
  ```
  Puts "Launching background lsf job; both master and slave on lsf"
  set Bgdir "./LSF_RUN"
  add_metal_fill_signoff -fill -bg -master_lsf -lsf -temp_working_dir $Bgdir
  ```
  It starts HMF generation in the working directory. Post that, you can choose to continue working in the software or exit from the session. The software license is released, as shown below:

  ```
  ################################################################################
  ###### Running add_metal_fill_signoff
  ################################################################################
  ###### Mode '-fill background mode' on cell 'tdsp'
  ################################################################################

  ######  FILL DB Preparing design data
  Ending "FILL DB Preparing design data" (total cpu=0:00:00.2, real=0:00:00.0, peak res=550.0M, current mem=550.0M)

  ######  FILL DB Submitting -fill background run on LSF
  Job <724739> is submitted to queue <lnx64>.
  Ending "FILL DB Submitting -fill background run on LSF" (total cpu=0:00:00.0, real=0:00:01.0, peak res=550.0M, current mem=532.5M)
  Working directory: "/lan/dfm/pvsqa_test01/PVS/HMF_TESTING/FILL_RUN"

  ################################################################################
  Ending "FILL DB add_metal_fill_signoff" (total cpu=0:00:00.3, real=0:00:01.0, peak res=550.0M, current mem=532.5M)
  ```
- The following command checks the status of the background job:
  ```
  add_metal_fill_signoff -check_bg_run $Bgdir
  ```
  Following is the output of the above command:

```
Run is in progress:
        ################################################################################
        ###### Running add_metal_fill_signoff
        ################################################################################
        BG run information:
         fill_cell:       'tdsp'
         fill_layer:      {M1 M2 M3 M4 M5 M6}
         output_format:   'gdsii'
         LSF job queue:   'lnx64'
         LSF job id:      '724739'

        Checking status of BG run:
         run status:      IN PROGRESS
         operation group:  194/1815
         output files:    NOT READY

Run completed:
        ################################################################################
        ###### Running add_metal_fill_signoff
        ################################################################################
        BG run information:
         fill_cell:       'tdsp'
         fill_layer:      {M1 M2 M3 M4 M5 M6}
         output_format:   'gdsii'
         LSF job queue:   'lnx64'
         LSF job id:      '724739'

        Checking status of BG run:
         run status:      COMPLETED
         operation group:  1815/1815
         output files:    READY
        ################################################################################
        Ending "FILL DB add_metal_fill_signoff" (total cpu=0:00:00.0, real=0:00:01.0, peak res=533.3M, current mem=533.2M)
```

- The following command restores initial software database link and generates HMF:
  ```
  add_metal_fill_signoff -load_fill_db $Bgdir
  ```
- The following command kills the background run that is in progress:
  ```
  add_metal_fill_signoff -kill_bg_run $Bgdir
  ```
  Following is the output of the above command:
  ```
  ################################################################################
  ###### Running add_metal_fill_signoff
  ################################################################################
  Job <724739> is being terminated
  ################################################################################
  Ending "FILL DB add_metal_fill_signoff" (total cpu=0:00:00.0, real=0:00:01.0, peak res=533.6M, current mem=533.6M)
  ```
- The following command presents an LSF example where master is not on LSF:
  ```
  setenv MASTER_LSF_RESOURCE '-q lnx64 -W "40:00" -n 4 -P "PVS:15.23:PE:test"'
  setenv DP_LSF_RESOURCE '-q lnx64 -W "15:00" -P "PVS:15.23:PE:test"'
  add_metal_fill_signoff -fill -bg -lsf -temp_working_dir LSF_RUN1
  ```
  Following is the output of the above command:
  ```
  ################################################################
  Get environment info ...
  ################################################################
  ENV VAR: TCL_LIBRARY = /grid/cic/cm_t3nb_005/PVS/pvs_1611_ehflw/inst_hier/lnx
  EV/share/lib/tcltk/08.64/tcl8.6
  ENV VAR: TCLX_LIBRARY = /grid/cic/cm_t3nb_005/PVS/pvs_1611_ehflw/inst_hier/ln
  DEV/share/lib/tclx/08.40
  ENV VAR: TCLLIBPATH = /grid/cic/cm_t3nb_005/PVS/pvs_1611_ehflw/inst_hier/lnx8
  V/share/lib/itcl/3.4.1 /grid/cic/cm_t3nb_005/PVS/pvs_1611_ehflw/inst_hier/lnx
  EV/share/lib/tclxml/3.2 /grid/cic/cm_t3nb_005/PVS/pvs_1611_ehflw/inst_hier/ln
  DEV/share/lib/tcllib/1.18 /grid/cic/cm_t3nb_005/PVS/pvs_1611_ehflw/inst_hier/
  1/DEV/share/lib/itcl/3.4.1 /grid/cic/cm_t3nb_005/PVS/pvs_1611_ehflw/inst_hier
  -1/DEV/share/lib/tclxml/3.2 /grid/cic/cm_t3nb_005/PVS/pvs_1611_ehflw/inst_hie
  7-1/DEV/share/lib/tcllib/1.18-
  ENV VAR: LSF_ENVDIR = /grid/sfi/farm/CLUSTERS/sjlsf01/7.0.3/conf
  ENV VAR: LSF_SERVERDIR = /grid/sfi/farm/CLUSTERS/sjlsf01/7.0.3/7.0/linux2.6-g
  ENV VAR: DP_LSF_RESOURCE = -q lnx64 -W "15:00" -P "PVS:15.23:PE:test"

  Available Resource: 1 processors
  ```
- The following command checks the run status in the above example:
  ```
  add_metal_fill_signoff -check_bg_run LSF_RUN1
  ```
  Following is the output of the above command:

```
<CMD> add_metal_fill_signoff -check_bg_run LSF_RUN1

#########################################################
###### Running add_metal_fill_signoff
#########################################################

BG run information:
  fill_cell:          'tdsp'
  fill_layer:         {M1 M2 M3 M4 M5 M6}
  output_format:      'gdsii'
  host:               'sj-amisha1'
  PVS pid:            '353'

Checking status of BG run:
  run status:         IN PROGRESS
  operation group:    DATA PREPROCESSING
  output files:       NOT READY
```

- The following command presents an LSF example where master is on LSF:

  ```
  add_metal_fill_signoff -fill -bg -master_lsf -lsf -temp_working_dir LSF_RUN2
  ```

  Following is the output of the above command:

  ```
  ###############################################################
  Get environment info ...
  ###############################################################
  ENV VAR: TCL_LIBRARY = /grid/cic/cm_t3nb_005/PVS/pvs_1611_ehflw/inst_
  EV/share/lib/tcltk/08.64/tcl8.6
  ENV VAR: TCLX_LIBRARY = /grid/cic/cm_t3nb_005/PVS/pvs_1611_ehflw/inst
  DEV/share/lib/tclx/08.40
  ENV VAR: TCLLIBPATH = /grid/cic/cm_t3nb_005/PVS/pvs_1611_ehflw/inst_h
  V/share/lib/itcl/3.4.1 /grid/cic/cm_t3nb_005/PVS/pvs_1611_ehflw/inst_
  EV/share/lib/tclxml/3.2 /grid/cic/cm_t3nb_005/PVS/pvs_1611_ehflw/inst
  DEV/share/lib/tcllib/1.18 /grid/cic/cm_t3nb_005/PVS/pvs_1611_ehflw/in
  1/DEV/share/lib/itcl/3.4.1 /grid/cic/cm_t3nb_005/PVS/pvs_1611_ehflw/i
  -1/DEV/share/lib/tclxml/3.2 /grid/cic/cm_t3nb_005/PVS/pvs_1611_ehflw/
  7-1/DEV/share/lib/tcllib/1.18-
  ENV VAR: LSF_ENVDIR = /grid/sfi/farm/CLUSTERS/sjlsf01/10.1//conf
  ENV VAR: LSF_SERVERDIR = /grid/sfi/hpc/lsf/10.1/linux2.6-glibc2.3-x86
  ENV VAR: DP_LSF_RESOURCE = -q lnx64 -W "15:00" -P "PVS:15.23:PE:test"
  ENV VAR: LSB_JOBID = 800272

  Available Resource: 16 processors
  ```

- The following command checks the run status in the above example:

  ```
  add_metal_fill_signoff -check_bg_run LSF_RUN2
  ```

  Following is the output of the above command:

  ```
  <CMD> add_metal_fill_signoff -check_bg_run LSF_RUN2

  #########################################################
  ###### Running add_metal_fill_signoff
  #########################################################

  BG run information:
    fill_cell:          'tdsp'
    fill_layer:         {M1 M2 M3 M4 M5 M6}
    output_format:      'gdsii'
    LSF job queue:      'lnx64'
    LSF job id:         '800272'

  Checking status of BG run:
    run status:         COMPLETED
    operation group:    75/75
    output files:       READY

  #########################################################
  ```

Related Information

- get_metal_fill_signoff_mode
- set_metal_fill_signoff_mode

add_pg_fill

```
add_pg_fill
[-help]
[-attach_instance_name attr_num]
[-attach_net_name attr_num]
[-attach_net_prop {{prop_name attr_num} ... }]
[-bsub resource_string]
[-core_license license_combination]
[-die_area_as_boundary]
[-dp n]
[-dp_timeout seconds]
[-extra_options options]
[-fix_post_process_drc]
[-format {stream oasis}]
[-incremental]
[-initial_working_dir string]
[-layer_map_file streamOut_map_file]
[-lib_name library_name]
[-license_dp_continue]
[-license_timeout seconds]
[-merge list_of_external_stream_files]
[-nc resource_string]
[-no_structure_name]
[-offset {x y}]
[-output_macros]
[-pg_fill_config_file file_name]
[-pg_hookup_flow {ALL | PG_ONLY | FLOAT_ONLY}]
[-qsub resource_string]
[-report_file]
[-stripes number]
[-structure_name structureName]
[-turbo jobs_num]
[-uniquify_cell_names]
[-units {100 | 200 | 400 | 800 | 1000 | 2000 | 4000 | 8000 | 10000 | 20000}]
[-working_dir working_directory]
[-xor_via_layers]
```

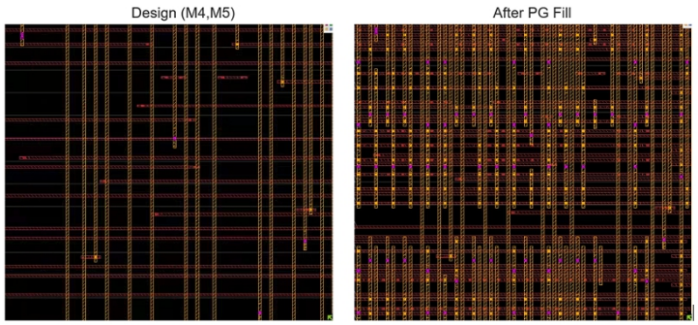
Runs Pegasus PG Fill with default settings and any specified options. This command adds only the floating Track-based Fill (TBF).

Pegasus PG Fill

Pegasus PG Fill is an in-design software used to insert fills connected to Power and/or Ground. PG Fill is generally used for power integrity. These fills are available on the design routing tracks and follow the same DRC rules as design. Pegasus PG Fill:

- Improves IR drop in worst scenarios
- Ensures no negative slack after PG Fill insertion
- Keeps power increase under control

The following example design of the M4 and M5 layers in which fills are inserted post Pegasus PG Fill processing:



Parameters

`-help`	Outputs a brief description that includes the type and default information for each `add_pg_fill` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man add_pg_fill`
`-attach_instance_name` *attr_num*	Specifies the instance name to be attached.
`-attach_net_name` *attr_num*	Specifies the net name to be attached.
`-attach_net_prop {{`*prop_name attr_num*`} ... }`	Specifies the net property to be attached.
`-bsub` *resource_string*	Runs the fill process through BSUB with the specified resource string.
`-core_license` *license_combination*	Sets an exact combination of licenses to be used.
`-die_area_as_boundary`	Writes the die area as a polygon or a boundary. When this parameter is not specified, the `streamOut` command writes the die area as a rectangle or a box.
`-dp` *n*	Runs Pegasus in a multi-processor mode. The *n* variable specified with this parameter presents the number of processors. By default, this value is taken from the Innovus multi-CPU usage.
`-dp_timeout` *seconds*	Specifies the wait time for the DP processes to start.
`-extra_options` *options*	Specifies the additional run options.
`-fix_post_process_drc`	Runs the fill clean-up utility to resolve the DRC issues, which appear post the fill steps.
`-format {stream oasis}`	Specifies the format of the flow.
`-incremental`	Runs Pegasus incremental PG fill. **Note**: This parameter is used with the `-initial_working_dir` and `-working_dir` parameters.
`-initial_working_dir` *string* `-initial_working_dir` *string*	Specifies the working directory with taking information from the previous run.
`-layer_map_file` *streamOut_map_file*	Specifies the name of the `streamOut` layer map file.
`-lib_name` *library_name*	Specifies the library name.
`-license_dp_continue`	Specifies that if the required number of licenses cannot be obtained in the DP processes, the software continues with fewer CPUs based on the number of licenses obtained.
`-license_timeout` *seconds*	Specifies the wait time (in seconds) to run the software, if a license is not immediately available. *Default*: `300`
`-merge` *list_of_external_stream_files*	Specifies the list of external stream files.
`-nc` *resource_string*	Runs the fill process through NC with the specified resource string.
`-no_structure_name`	Merges the GDSII file without loading the software database. **Note:** This parameter is used with the `-merge` parameter.
`-offset {`*x y*`}`	Specifies the location (in microns) of the design in the GDSII output file.
`-output_macros`	Displays the LEF abstract information, such as LEF pin geometries and obstructions for macros.
`-pg_fill_config_file` *file_name*	Specifies the configuration file for the PG fill flow. Any overrides to the default behavior are written in this file.
`-pg_hookup_flow {ALL | PG_ONLY | FLOAT_ONLY}`	Specifies the type of fills to be generated. *Default*: `PG_ONLY`
`-qsub` *resource_string*	Runs the fill process through QSUB with the specified resource string.
`-report_file`	Specifies the cell name to be changed to the file.

`-stripes` *number*	Specifies the number of stripes that divide the design. *Default*: 1
`-structure_name` *structureName*	Specifies the superstructure that consists of all the geometries that the map file writes out.
`-turbo` *jobs_num*	Speeds up the fill process by running the specified number of parallel fill jobs.
`-uniquify_cell_names`	Creates unique cell names if there are name collisions, when the software merges files.
`-units {100 | 200 | 400 | 800 | 1000 | 2000 | 4000 | 8000 | 10000 | 20000}`	
	Specifies the resolution for values in the GDSII/OASIS file.
`-working_dir` *working_directory*	Specifies the working directory.
`-xor_via_layers`	Switches on comparison of via during incremental fill. **Note:** Via-XOR consumes more time on large databases.

add_track_fill

```
add_track_fill
[-help]
[-attach_instance_name attr_num]
[-attach_net_name attr_num]
[-attach_net_prop {{prop_name attr_num} ... }]
[-bsub resource_string]
[-core_license license_combination]
[-die_area_as_boundary]
[-dp n]
[-dp_timeout seconds]
[-extra_options options]
[-fix_post_process_drc]
[-format {stream oasis}]
[-incremental]
[-initial_working_dir string]
[-layer_map_file streamOut_map_file]
[-lib_name library_name]
[-license_dp_continue]
[-license_timeout seconds]
[-merge list_of_external_stream_files]
[-nc resource_string]
[-no_structure_name]
[-offset {x y}]
[-output_macros]
[-qsub resource_string]
[-report_file]
[-stripes number]
[-structure_name structureName]
[-track_fill_config_file file_name]
[-turbo jobs_num]
[-uniquify_cell_names]
[-units {100 | 200 | 400 | 800 | 1000 | 2000 | 4000 | 8000 | 10000 | 20000}]
[-working_dir working_directory]
```

Adds the Track-Based Fill (TBF) flow to the metal fill layers. Pegasus TBF flow is designed to reuse empty tracks for metal filling. It provides a better fill rate than the signoff fill, because the TBF fills are more homogeneous to the design. The TBF solution is timing aware and DRC-neutral. The flow can be run at any stage of the design and does not require any manual input, since all the required information is fetched from the Innovus database.

The TBF flow also allows altering the default TBF behavior through the various configuration parameters. It supports incremental mode to update TBF fills during the ECO cycles. For better performance and scalability, the TBF flow supports partitioning through the `-turbo` parameter, and job distribution through the `-bsub`, `-qsub`, and `-nc` parameters.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `add_tra` For a detailed description of the command and all its parameters, use the `man` command: `man add_track_fill`
`-attach_instance_name attr_num`	Specifies the instance name to be attached.
`-attach_net_name attr_num`	Specifies the net name to be attached.
`-attach_net_prop {{prop_name attr_num} ... }`	Specifies the net property to be attached.
`-bsub resource_string`	Runs the fill process through BSUB with the specified resource string.
`-core_license license_combination`	Sets an exact combination of licenses to be used.
`-die_area_as_boundary`	Writes the die area as a polygon or a boundary. When this parameter is not specified, the
`-dp n`	Runs Pegasus in a multi-processor mode. The `n` variable specified with this parameter pr Innovus multi-CPU usage.
`-dp_timeout seconds`	Specifies the wait time for the DP processes to start.
`-extra_options options`	Specifies the additional run options.

`-fix_post_process_drc`	Runs the fill clean-up utility to resolve the DRC issues, which appear post the fill steps.
`-format {stream oasis}`	Specifies the format of the flow.
`-incremental`	Runs Pegasus incremental track metal fill. **Note**: This parameter is used with the `-initial_working_dir` and `-working_dir` paramete
`-initial_working_dir` *string*`-initial_working_dir` *string*	Specifies the working directory with taking information from the previous run.
`-layer_map_file` *streamOut_map_file*	Specifies the name of the `streamOut` layer map file.
`-lib_name` *library_name*	Specifies the library name.
`-license_dp_continue`	Specifies that if the required number of licenses cannot be obtained in the DP processes, licenses obtained.
`-license_timeout` *seconds*	Specifies the wait time (in seconds) to run the software, if a license is not immediately ava *Default*: `300`
`-merge` *list_of_external_stream_files*	Specifies the list of external stream files.
`-nc` *resource_string*	Runs the fill process through NC with the specified resource string.
`-no_structure_name`	Merges the GDSII file without loading the software database. **Note:** This parameter is used with the `-merge` parameter.
`-offset {`*x y*`}`	Specifies the location (in microns) of the design in the GDSII output file.
`-output_macros`	Displays the LEF abstract information, such as LEF pin geometries and obstructions for m
`-qsub` *resource_string*	Runs the fill process through QSUB with the specified resource string.
`-report_file`	Specifies the cell name to be changed to the file.
`-stripes` *number*	Specifies the number of stripes that divide the design. *Default*: 1
`-structure_name` *structureName*	Specifies the superstructure that consists of all the geometries that the map file writes out.
`-track_fill_config_file` *file_name*	Specifies the configuration file for Track fill flow.
`-turbo` *jobs_num*	Speeds up the fill process by running the specified number of parallel fill jobs.
`-uniquify_cell_names`	Creates unique cell names if there are name collisions, when the software merges files.
`-units {100 | 200 | 400 | 800 | 1000 | 2000 | 4000 | 8000 | 10000 | 20000}`	
	Specifies the resolution for values in the GDSII/OASIS file.
`-working_dir` *working_directory*	Specifies the working directory.

addViaFill

```
addViaFill
[-help]
[-area x1 y1 x2 y2]
[-layer [cutLayerNumber | {list_of_cut_layer_numbers}]]
[-includeVia vias]
[-mode {all | floatingOnly | connectedOnly | {floating connectToPG connectBetweenFill}}]
[-snap]
```

Uses the values specified by the `setMetalFill` command to determine sizes of vias to create. Via fill is inserted to get uniform cut density. Only vias that meet Minimum Area Rule requirements are used. `addViaFill` also uses values from `setViaFill` for the window size, step, and density values.

For information, see the `setMetalFill` and `setViaFill` commands.

Note: Default values from `setMetalFill` for `minWidth` and `minLength` are biased towards the need of `add_metal_fill_signoff` use and should be overridden with values that are more useful for via generation.

Add via fill in the following areas immediately before adding metal fill:

- Where metal fill shapes would intersect with power or ground nets on consecutive layers if no via fill was added.
- Where metal fill shapes on consecutive layers would intersect if no via fill was added.

To reduce the need for via fill, insert multiple-cut vias with the NanoRoute router. For information, see `setNanoRouteMode`.

Also note that this command does not support the 20nm and below node design rules. It is strongly recommended to use the PVS metal fill solution for 20nm and below modes. The same solution should also be used for the sign-off metal fill rule deck of 28nm and above.

Adding Via Fill in Multi-Threading Mode

You can add via fill in multi-threading mode by using the following command before adding the via fill:

- `setMultiCpuUsage`

Via Fill supports the new `setMultiCpuUsage` command use model. For more information, see the following documents:

- Multiple-CPU Processing Commands chapter of the *Innovus Text Command Reference*.
- Accelerating the Design Process by Using Multiple-CPU Processing chapter of the *User Guide*.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `addViaFill` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man addViaFill`.
`-area` *x1 y1 x2 y2*	Specifies a set of coordinates within which to add via fill. For example, specify `-area 0 0 100 100`. *Default:* If you do not specify this parameter, adds via fill to the entire design area.

`-includeVia` *vias*	Specifies a list of vias (VIA in LEF) to be included or via rules (VIARULE GENERATE in LEF) to be used or a combination of the two. Separate the arguments with spaces. You can specify both via names and via rules.
	• The following example shows how to specify a list of via names:
	`-includeVia {VIA12 VIA23}`
	• The following example shows how to specify a list of via rules:
	`-includeVia {via1Array via2Array}`
	• The following example shows how to specify both via names and via rules:
	`-includeVia {VIA12 VIA23 via1Array via2Array}`
	Note: Non-default rule (NDR) vias can also be specified with `-includeVia`.

`-layer {`*cutLayerNumber* | `{`*list_of_cut_layer_numbers*`}}`	Specifies the cut layers for adding via fill. If you specify more than one layer, separate each layer with a space and enclose the list in braces or double quotation marks. To specify *cut12*, *cut23*, and *cut34*, type one of the following commands:
	`addViaFill -layer "VIA12 VIA23 VIA34"`
	`addViaFill -layer {VIA12 VIA23 VIA34}`
	Default: If you do not specify this parameter, adds via fill to all cut layers.

`-mode {all | floatingOnly | connectedOnly | {floating connectToPG connectBetweenFill}}`	
	Specifies the connection mode for via fill. Via fill can be connected to power or ground nets (tied off), or connected between floating metal fill, or unconnected to metal fill (floating). Via fill cannot be connected to signal nets. Specify one or more of the following parameters, as required: • `all`: Enables addition of unconnected and connectToPG via fill. • `floatingOnly`: Adds unconnected via fill only. The option will be made obsolete in the future release. • `floating`: Adds unconnected via fill. It is equivalent to `-mode floatingOnly`, which will be made obsolete in the future release. • `connectToPG`: Adds connected via fill to PG. It is equivalent to `-mode connectOnly`, which will be made obsolete in the future release. • `connectBetweenFill`: Adds connected via fill between floating metal fill. • `connectedOnly`: Adds connected via fill to PG only. The option will be made obsolete in the future release. *Default:* `all`
`-snap`	Snaps edges of via fill shapes and centers of cuts to a user-defined snap grid that is a multiple of the manufacturing grid. When via shapes are snapped to this grid, the edges and centers of the via shapes lie on the grid. If a grid is not specified, the software snaps the shapes to the manufacturing grid. You can define a grid using the `setSnapGrid` command. For more information, see setSnapGrid.

Example

The following set of command inserts unconnected via fill and connected via fill between floating metal fill in layers `VIA1` to `VIA8` and `RV` based on existing `setMetalFill` and `setViaFill` settings:

```
setViaFill -layer {VIA1 VIA2 VIA3 VIA4 VIA5 VIA6 VIA7} /
           -windowSize 100 100 -windowStep 50 50 -minDensity 0.01 /
           -maxDensity 30 -preferredDensity 1

setViaFill -layer {VIA8 RV} -windowSize 100 100 -windowStep 50 50 /
           -minDensity 0.25 -maxDensity 30 -preferredDensity 10

addViaFill -layer {VIA1 VIA2 VIA3 VIA4 VIA5 VIA6 VIA7 VIA8 RV} /
```

```
-mode {floating connectBetweenFill}
```

Related Information

- The following sections in the Optimizing Metal Density chapter of *User Guide*.
  - "Overview"
  - "Adding Via Fill"

deleteMetalFill

```
deleteMetalFill
[-help]
[-area {x1 y1 x2 y2}]
[-layer {layerNumber | {list_of_layer_numbers} } ]
[-mode {all | connectedOnly | floatingOnly} ]
[-shape {[FILLWIRE] [FILLWIREOPC]} ]
```

Removes metal and via fill in order to start over with the metal fill process. If you run this command without specifying parameters, it deletes all the metal and via fill in the design.

Note: Deleting via fill with this command might create open violations.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `deleteMetalFill` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man deleteMetalFill`.
`-area {x1 y1 x2 y2}`	Specifies a set of coordinates within which to delete metal and via fill. For example, `-area {0 0 100 100}`. *Default:* If you do not specify this parameter, deletes fill from the entire design area.

`-layer {layerNumber | {list_of_layer_numbers}}`	Specifies the layers from which to remove metal and via fill. If you specify more than one layer, separate each layer with a space and enclose the list in braces or double quotation marks. This parameter accepts metal layers, cut layers, or a combination.
	• If you specify a cut layer, the command removes vias that contain that cut layer only. For example:
	`deleteMetalFill -layer {VIA2 VIA3}`
	• If you specify a metal layer, metal fill shapes on that layer and vias that contain that layer are removed. For example:
	`deleteMetalFill -layer {metal2 metal3}`
	Default: If you do not specify this parameter, removes metal and via fill from all layers.

`-mode {all | connectedOnly | floatingOnly}`	Specifies whether the command removes only connected fill, only floating fill, or all fill.
	Default: `all`
	Removes all the connected and floating metal and via fill.
	`connectedOnly`
	Removes only connected metal and via fill.
	`floatingOnly`
	Removes only floating metal and via fill.

`-shape {[FILLWIRE] [FILLWIREOPC]}`	Specifies whether the command removes `FILLWIRE` or `FILLWIREOPC` shapes or both kinds of shapes. *Default:* `FILLWIRE FILLWIREOPC`

Examples

- The following command removes via fill from cut layers `VIA12`, `VIA23`, `VIA34`, `VIA45`, `VIA56`:
  ```
  deleteMetalFill -layer {VIA12 VIA23 VIA34 VIA45 VIA56}
  ```
- The following command removes `metal2` floating fill with `FILLWIREOPC` shape and via fill connected to `metal2`:
  ```
  deleteMetalFill -layer metal2 -mode floatingOnly -shape FILLWIREOPC
  ```

Related Information

The Optimizing Metal Density chapter of *User Guide*.

fixOpenFill

```
fixOpenFill
[-help]
[-net list_of_net_names]
[-area x1 y1 x2 y2]
[-remove]
```

Fixes any isolated power/ground (PG) fills. Isolated PG fills are FILL shapes attached to a power-ground (PG) net that are not connected to at least one non-FILL shape.

In designs with floating fill, the trimMetalFill command is used after routing signal nets to trim fill that causes DRC violations. In designs with tied fill, i.e. metal or via fill connected to special nets, `trimMetalFill` causes isolated fills to appear. In such cases, use the `fixOpenFill` command after `trimMetalFill` to either change the isolated fills to floating fills or to remove them from the design.

If you choose to remove the isolated fills with `fixOpenFill -remove`, you can then run incremental `add_metal_fill_signoff` to see if any of those islands can be tied, either to the same or to another PG rail.

If you run the `fixOpenFill` command without specifying any parameters, it changes all isolated fills in the design to floating fills.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `fixOpenFill` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man fixOpenFill`
`-area x1 y1 x2 y2`	Specifies a set of coordinates within which to fix open fill issues. *Default:* If you do not specify this parameter, fixes open fills in the entire design area.
`-net list_of_net_names`	Specifies a list of special net names to fix.
`-remove`	Specifies whether to remove the open fills. If this option is not specified, `fixOpenFill` changes isolated PG fills to floating fills.

Example

The following command removes all isolated PG fills from the design:

```
fixOpenFill -remove
```

Related Information

- The following sections in the Optimizing Metal Density chapter of *User Guide*.
  - "Overview"
  - "Connected and Floating Metal Fill"
  - "Trimming Metal Fill"

get_metal_fill_signoff_mode

```
  get_metal_fill_signoff_mode
[-help]
[-and_area_size]
[-area]
[-attach_instance_name]
[-attach_net_name]
[-attach_net_prop]
[-auto_load_fills]
[-bg]
[-clock]
[-common_blockage]
[-control]
[-delete_area]
[-delete_point]
[-die_area_as_boundary]
[-dp]
[-dp_timeout]
[-excl_area_size]
[-fill_cell]
[-fill_layer]
[-fill_output_mode]
[-layer_map_file]
[-license_timeout]
[-license_dp_continue]
[-lib_name]
[-lsf]
[-master_lsf]
[-merge]
[-min_density]
[-net]
[-no_structure_name]
[-no_via_fills]
[-offset]
[-output_macros]
[-quiet]
[-report_file]
[-reset]
[-rule_file]
[-slack_threshold]
[-spacing]
[-spacing_above]
[-spacing_below]
[-stripes]
[-structure_name]
[-tech_lib]
[-tech_set]
[-technology]
[-temp_working_dir]
[-trim_effort]
[-trim_layer]
[-trim_mf]
[-union_density]
[-uniquify_cell_names]
[-units]
[-window_size]
[-window_step]
```

Returns information about the signoff metal fill flow global variables set by the `set_metal_fill_signoff_mode` command. You can specify one or more parameter. When you invoke this command without any argument, the software prints the current values stored in the database for all the options. If you provide any option, then the software prints the value currently stored in the database.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `get_metal_fill_signoff_mode` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man get_metal_fill_signoff_mode`
`-parameter_names`	Returns information for the specified parameters. You can specify one or more parameters. See `set_metal_fill_signoff_mode` for descriptions of the `get_metal_fill_signoff_mode` parameters.
`-quiet`	Returns the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software returns the current settings of all `set_metal_fill_signoff_mode` parameters in Tcl list format.

Examples

- The following command displays values for all the set parameters:
  ```
  get_metal_fill_signoff_mode
  ```
  Following is the output of the above command:
  ```
  -die_area_as_boundary
  -dp 4
  -fill_cell tdsp
  -fill_output_mode gdsii
  -layer_map_file ./RULE/stream.map
  ```
- The following command displays the value of the `-layer_map_file` parameter:
  ```
  get_metal_fill_signoff_mode -layer_map_file
  ```
  Following is the output of the above command:
  ```
  -layer_map_file ./RULE/stream.map
  ```
- Following is an example of getting values of multiple parameters using the `get_metal_fill_signoff_mode` command:
  get_metal_fill_signoff_mode -layer_map_file -dp -units
  Output:
  -dp 4
  -layer_map_file ./RULE/stream.map
  -units 2000

Related Information

- `add_metal_fill_signoff`
- `set_metal_fill_signoff_mode`

get_pg_fill_config

```
get_pg_fill_config
[-help]
[-attach_instance_name]
[-attach_net_name]
[-attach_net_prop]
[-core_license]
[-die_area_as_boundary]
[-dp]
[-dp_dir]
[-dp_timeout]
[-extra_options]
[-format]
[-layer_map_file]
[-license_dp_continue]
[-license_timeout]
[-merge]
[-no_structure_name]
[-offset]
[-output_macros]
[-pg_fill_config_file]
[-pg_hookup_flow]
[-report_file]
[-stripes]
[-structure_name]
[-uniquify_cell_names]
[-units]
[-working_dir]
```

Prints the parameters of the `add_pg_fill` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `get_pg_fill_config` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man get_pg_fill_config`
`-attach_instance_name`	Prints the instance name to be attached.
`-attach_net_name`	Prints the net name to be attached.
`-attach_net_prop`	Prints the net property to be attached.
`-core_license`	Prints an exact combination of licenses to be used.
`-die_area_as_boundary`	Prints the die area as a polygon or a boundary.
`-dp`	Runs Pegasus in a multi-processor mode. The *n* variable specified with this parameter presents the number of processors. By default, this value is taken from the Innovus multi-CPU usage.
`-dp_dir`	Prints the name of the distributed directory.
`-dp_timeout`	Prints the wait time for the DP processes to start.
`-extra_options`	Prints the additional run options.
`-format`	Prints the format of the flow.
`-layer_map_file`	Prints the name of the `streamOut` layer map file.
`-lib_name`	Prints the library name.
`-license_dp_continue`	Specifies that if the required number of licenses cannot be obtained in the DP processes, the software continues with fewer CPUs based on the number of licenses obtained.
`-license_timeout`	Prints the wait time (in seconds) to run the software, if a license is not immediately available. *Default*: `300`
`-merge`	Prints the list of external stream files.
`-no_structure_name`	Merges the GDSII file without loading the software database. **Note:** This parameter is used with the `-merge` parameter.
`-offset`	Prints the location (in microns) of the design in the GDSII output file.
`-output_macros`	Prints the LEF abstract information, such as LEF pin geometries and obstructions for macros.
`-pg_fill_config_file`	Prints the configuration file for the PG fill flow. Any overrides to the default behavior are written in this file.
`-pg_hookup_flow`	Prints the type of fills to be generated. *Default*: `PG_ONLY`
`-report_file`	Prints the cell name to be changed to the file.
`-stripes`	Prints the number of stripes that divide the design. *Default*: 1
`-structure_name`	Prints the superstructure that consists of all the geometries that the map file writes out.
`-uniquify_cell_names`	Creates unique cell names if there are name collisions, when the software merges files.
`-units`	Prints the resolution for values in the GDSII/OASIS file.
`-working_dir` *working_directory*	Prints the working directory.

get_track_fill_config

```
get_track_fill_config
[-help]
[-attach_instance_name]
[-attach_net_name]
[-attach_net_prop]
[-core_license]
[-die_area_as_boundary]
[-dp]
[-dp_dir]
[-dp_timeout]
[-extra_options]
[-format]
[-layer_map_file]
[-license_dp_continue]
[-license_timeout]
[-merge]
[-no_structure_name]
[-offset]
[-output_macros]
[-report_file]
[-stripes]
[-structure_name]
[-track_fill_config_file file_name]
[-uniquify_cell_names]
[-units]
[-working_dir]
```

Prints the parameters of the `add_track_fill` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `get_track_fill_config` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man get_track_fill_config`
`-attach_instance_name`	Prints the instance name to be attached.
`-attach_net_name`	Prints the net name to be attached.
`-attach_net_prop`	Prints the net property to be attached.
`-core_license`	Prints an exact combination of licenses to be used.
`-die_area_as_boundary`	Prints the die area as a polygon or a boundary.
`-dp`	Runs Pegasus in a multi-processor mode. The *n* variable specified with this parameter presents the number of processors. By default, this value is taken from the Innovus multi-CPU usage.
`-dp_dir`	Prints the name of the distributed directory.
`-dp_timeout`	Prints the wait time for the DP processes to start.
`-extra_options`	Prints the additional run options.
`-format`	Prints the format of the flow.
`-layer_map_file`	Prints the name of the `streamOut` layer map file.
`-lib_name`	Prints the library name.
`-license_dp_continue`	Specifies that if the required number of licenses cannot be obtained in the DP processes, the software continues with fewer CPUs based on the number of licenses obtained.
`-license_timeout`	Prints the wait time (in seconds) to run the software, if a license is not immediately available. *Default*: `300`
`-merge`	Prints the list of external stream files.
`-no_structure_name`	Merges the GDSII file without loading the software database. **Note:** This parameter is used with the `-merge` parameter.
`-offset`	Prints the location (in microns) of the design in the GDSII output file.
`-output_macros`	Prints the LEF abstract information, such as LEF pin geometries and obstructions for macros.
`-report_file`	Prints the cell name to be changed to the file.
`-stripes`	Prints the number of stripes that divide the design. *Default*: 1
`-structure_name`	Prints the superstructure that consists of all the geometries that the map file writes out.
`-track_fill_config_file` *file_name*	Specifies the configuration file for Track fill flow.
`-uniquify_cell_names`	Creates unique cell names if there are name collisions, when the software merges files.
`-units`	Prints the resolution for values in the GDSII/OASIS file.
`-working_dir` *working_directory*	Prints the working directory.

getMetalFillSpacingTable

```
getMetalFillSpacingTable
[-help]
[-layer layer_name_list]
[-fill_to_active]
[-fill_to_fill]
[-fill_to_opc]
[-opc_to_active]
[-opc_to_opc]
```

Prints the spacing table for specified or for all layers. If no option is specified, prints all spacing tables for all layers.

To specify spacing table based on existing metal fill width, use the `setMetalFillSpacingTable` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `getMetalFillSpacingTable` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man getMetalFillSpacingTable`.
`-layer` *`layer_name_list`*	Prints spacing table for specified layers. *Default*: Prints spacing table for all layers
`-fill_to_active`	Prints the fill to active spacing table for specified or all layers.
`-fill_to_fill`	Prints the fill to fill spacing table for specified or all layers.
`-fill_to_opc`	Prints the fill to OPC spacing table for specified or all layers.
`-opc_to_active`	Prints the OPC to active spacing table for specified or all layers.
`-opc_to_opc`	Prints the OPC to OPC spacing table for specified or all layers.

Example

- The following command prints the fill to active spacing table for layer `METAL2`:

  ```
  getMetalFillSpacingTable -layer 2 -fill_to_active
  ```

  The output is as follows:

  ```
  Layer 2,

  fill_to_active {0.08 0.27}{0.14 0.36}{0.32 0.63}
  ```

- The following command prints all spacing tables for all layers:

  ```
  getMetalFillSpacingTable
  ```

  The output is as follows:

  ```
  Layer metal2,

  fill_to_active {0.08 0.27}{0.14 0.36}{0.32 0.63}

  Layer metal3,

  fill_to_fill {0.08 0.2}{0.14 0.3}{0.32 0.6}

  Layer metal3,

  opc_to_active {0.05 0.08}{0.08 0.1}{0.1 0.12}
  ```

Related Information

- The following sections in the Optimizing Metal Density chapter of *User Guide*.
  - "Specifying Metal Fill Spacing Table"
  - "Trimming Metal Fill"

report_metal_fill

```
report_metal_fill
[-help]
[-file file_name]
[-fill_area]
[-fill_length]
[-mesh]
```

Generates a report on metal fill to summarize the number and area of fill shapes on a per-net and per-layer basis. Use the generated report to tune the `add_metal_fill_signoff` flow parameters to achieve the desired distribution across multiple nets and layers as well as to minimize floating fill.

Before generating a metal fill report, you should have:

- Imported the design
- Completed detailed routing
- Added metal fill

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `report_metal_fill` parameter. For a detailed description of the command and all its parameters, use `man report_metal_fill`.
`-file file_name`	Writes the report to a file with the specified name. *Default*: *topCellName*`_metal_fill_report.rpt`
`-fill_area`	Reports area information for metal fill for each layer.
`-fill_length`	Reports length information for metal fill for each layer.
`-mesh`	Reports mesh information for metal fill.

Examples

- The following command writes a metal fill report to the dtmf_metal_fill.rpt file.

  ```
  report_metal_fill -file dtmf_metal_fill.rpt
  ```

  The output is as follows:

  ```
  ############################################################
  #  Generated by:      Cadence Innovus 19.10-p002_1
  #  Design:            DTMF_CHIP
  #  Command:           report_metal_fill -file dtmf_metal_fill.rpt
  ############################################################
  Metal Fill Summary
  <numFillShapes>=<fillArea>squm(<percentTotal>)
  |floating    |total
  ------------------------------------------------------
  M1| 25214= 349610.52 (100.00%)  | 25214= 349610.52
  M2| 27905= 372408.83 (100.00%)  | 27905= 372408.83
  M3| 26106= 348543.27 (100.00%)  | 26106= 348543.27
  M4| 27439= 393704.76 (100.00%)  | 27439= 393704.76
  M5| 24305= 367076.78 (100.00%)  | 24305= 367076.78
  M6| 26208= 421137.91 (100.00%)  | 26208= 421137.91
  ------------------------------------------------------
  sum|157177=2252482.06 (100.00%)  |157177=2252482.06
  ```

- The following command writes out fill length in the metal fill report:

  ```
  report_metal_fill -fill_length -file dtmf_metal_fill_length.rpt
  ```

  The output is as follows:

```
############################################################
# Generated by: Cadence Innovus 19.10-p002_1
# Generated on: Wed May 22 14:56:50 2019
# Design: DTMF_CHIP
# Command: report_metal_fill -fill_length -file dtmf_metal_fill_length.rpt
############################################################
Metal Fill Summary
<numFillShapes>=<fillLength>um(<percentTotal>)
 |floating     |total
-----------------------------------------------------
M1| 25214= 183703.43 (100.00%) | 25214= 183703.43
M2| 27905= 194629.92 (100.00%) | 27905= 194629.92
M3| 26106= 185140.32 (100.00%) | 26106= 185140.32
M4| 27439= 204525.36 (100.00%) | 27439= 204525.36
M5| 24305= 188877.45 (100.00%) | 24305= 188877.45
M6| 26208= 213826.83 (100.00%) | 26208= 213826.83
-----------------------------------------------------
sum|157177=1170703.33 (100.00%) |157177=1170703.33
-----------------------------------------------------
```

Related Information

The Optimizing Metal Density chapter of *User Guide*.

run_pvs_drc_rules

```
run_pvs_drc_rules
[-help]
pvlFile
[-area {x1 y1 x2 y2}]
[-dp N]
[-error integer]
[-extraPvsOptions pvs_option_string]
[-gds_file fileName | -oasis_file fileName]
-mapfile fileName
[-merge list_files]
[-noclean]
[-oa_view view_name]
[-output fileName]
[-repRoutingOnly]
[-report string]
[-stream_out]
[-units integer]
[-version]
[-work_directory working_directory]
```

Calls the Cadence Physical Verification System application to check DRC rules with a sign-off PVS rule deck.

This command is loaded only if you have a PVS installation and the PVS bin directory is in your Linux search path (e.g. the command `pvs` can be run).

It is possible there are different options than described here depending on the version of PVS that you have installed. Use `run_pvs_drc_rules -help` to view the exact command syntax for your current PVS installation.

`run_pvs_drc_rules` will assemble the necessary files, call `pvs`, and then read back the PVS violations so they are visible in the Innovus violation browser. The call to `pvs` requires the appropriate PVS licenses in order to run.

The `run_pvs_drc_rules` command takes as input:

- A GDSII/OASIS layout file (normally created by streamOut command in Innovus)
- A layer map file
- A PVL rule file that contains the DRC rules to check
- The name of the output PVS results database file to be created
- Other optional arguments, such as distributed processing, area to check, number of errors to output, and so on.

Details about the various file formats can be found in the PVS documentation.

This command returns a status of `0 (tcl_OK)` or `1 (tcl_ERROR)`. Error messages or warnings are written to the Innovus log file.

Parameters

`-help`	Prints out the command usage.
`pvlFile`	Specifies the PVS rule file.
`-area {x1 y1 x2 y2}`	
	Defines the area to be checked. The default is the entire chip window.
`-dp N`	Specifies the number of local cores to use. The default is 1.
`-error integer`	Specifies the maximum number of errors to output. The default is 1000000.
`-extraPvsOptions pvs_option_string`	
	Specifies the additional PVS run options.
`-gds_file fileName`	
	Specifies a GDSII file to be read by PVS. If this parameter is specified, then the `-mapfile` parameter is ignored and `streamOut` is not run.

`-mapfile fileName`	Specifies a stream map file. In the usual case, the Innovus map file used by the Innovus `streamOut` command is specified. If this parameter is specified and neither of the parameters `-gds_file` or `-oasis_file` is specified, then the `streamOut` command is automatically run before PVS starts. The `streamOut` command is: `streamOut .pvs_xxx/topcell.gds.gz -mapFile mapfile -libName designLib -outputMacro -mode ALL [-units integer]` Here, *`topcell`* is the topcell name of the design and *`mapfile`* is the file name specified with the `-mapfile` parameter. The optional `-units integer` is taken from the `run_pvs_drc_rules -units` value (if given). If not given, Innovus `streamOut` uses the Innovus DB unit value. Refer to the Innovus streamOut command for more details. For information on the map file format, see "About the GDSII or OASIS Map File" in the Importing and Exporting Designs chapter of the *Innovus User Guide*. **Note:** The `-mapfile` parameter is currently mandatory, even if it is not used due to the presence of `-gds_file` or `-oasis_file`. It will become optional in future release.
`-merge list_files`	Specifies the list of external stream files for `streamOut`.
`-noclean`	Specifies that the working directory, which is created while the `run_pvs_drc_rules` command is running, should not be removed after the run. The working directory name begins with `.pvs`.
`-oa_view view_name`	Specifies the temporary OA view.
`-oasis_file fileName`	
	Specifies an Oasis file to be read by PVS. If this parameter is specified, then the `-mapfile` parameter is ignored and `streamOut` is not run.
`-output fileName`	Specifies the error markers result database in ASCII format. If not specified, *`topCellName`*`.ascii` is used.
`-repRoutingOnly`	Ignores standard cell violations. This parameter is equivalent to setting "`abort_on_layout_error no`" in PVS rule deck. `abort_on_layout_error` checks for problems such as missing cell masters and duplicate cell references. If `-repRoutingOnly` is specified, PVS does not abort even if any of these errors happen. For more information, refer to the *Cadence Physical Verification System Developer Guide*.
`-report fileName`	Specifies the name of the PVS DRC summary report file to be created. When a relative path has been specified, the report file is created relative to the Innovus invocation directory. The default is no report file.
`-stream_out`	Uses the `streamOut` command for GDS in the OA flow.
`-units integer`	Specifies the user-defined DB unit value used for `streamOut` (see `-mapfile` above). This value should be consistent with the DB unit value in PVL rules.
`-version`	Prints the PVS version.
`-work_directory working_directory`	
	Specifies the current working directory.

Examples

- The following command runs sign-off DRC check with the gds file from Innovus using the PVS rule file `28nm.pvl`. The gds file will be streamed out with the specified map file, `gds.map`, and the gds unit will be 1000 as specified:

  ```
  run_pvs_drc_rules 28nm.pvl -mapfile gds.map -units 1000
  ```

- The following command runs sign-off DRC check on routing layers within the specified area {0 0 100 100} using 8 local CPUs. The command will report only up to 1000 DRC issues:

  ```
  run_pvs_drc_rules 28nm.pvl -area {0 0 100 100} -dp 8 -error 1000 -mapfile gds.map
  ```

- The following command runs sign-off check on the existing gds file `ios.gds`, and writes the report file to `pvs.signoff.db`:

  ```
  run_pvs_drc_rules 28nm.pvl -report pvs.signoff.db -gds_file ios.gds
  ```

Related Information

- `streamOut`
- In the *Innovus User Guide*:
  - "About the GDSII or OASIS Map File" section in the Importing and Exporting Designs chapter

set_metal_fill_signoff_mode

```
set_metal_fill_signoff_mode
[-help]
[-and_area_size size_value]
[-area {x1 y1 x2 y2}]
[-attach_instance_name attr_num]
[-attach_net_name attr_num]
[-attach_net_prop {{prop_name attr_num} ...}]
[-auto_load_fills]
[-bg]
[-clock]
[-common_blockage {layer_num1 layer_datatype1 ...}]
[-control file_name]
[-delete_area {{layer_list1} {x1 y1 x2 y2} ...}]
[-delete_point {{layer_list1}{x1 y1}...}]
[-die_area_as_boundary]
[-dp n]
[-dp_timeout value]
[-excl_area_size size_value]
[-fill_cell cell_name]
[-fill_layer {layers_list}]
[-fill_output_mode {gdsii | oasis}]
[-layer_map_file streamOut_map_file]
[-license_timeout]
[-license_dp_continue]
[-lib_name library_name]
[-lsf]
[-master_lsf]
[-merge list_of_external_stream_files]
[-min_density percent1 layer_list1]
[-net net_names]
[-no_structure_name]
[-no_via_fills]
[-offset {x y}]
[-output_macros]
[-report_file]
[-reset {parameter_name | all}]
[-rule_file rule_file_name]
[-slack_threshold float]
[-spacing float]
[-spacing_above float]
[-spacing_below float]
[-stripes number]
[-structure_name structureName]
[-tech_lib string]
[-tech_set string]
[-technology string]
[-temp_working_dir work_dir]
[-trim_effort {high | med | low}]
[-trim_layer {layers_list}]
[-trim_mf spacing_file]
[-union_density {percent1 layer_list1 [{percent2 layer_list2}]...}]
[-uniquify_cell_names]
[-units {100 | 200 |400 |800 |1000 |2000 |4000 |8000 |10000 |20000}]
[-view_fill]
[-window_size float]
[-window_step float]
```

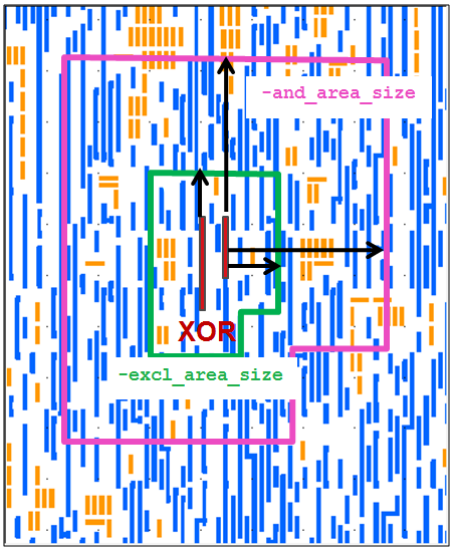
Sets option parameters for the signoff metal fill flow. The options are saved after the `saveDesign` process of the software with the existing design data. To get the list of stored options, use the `get_metal_fill_signoff_mode` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `set_metal_fill_signoff_mode` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man set_metal_fill_signoff_mode`
`-and_area_size` *size_value*	Specifies a value for sizing up the changed area to use shapes within this area. This parameter helps in calculating the density and creating areas to be filled, if they fall below the minimum density. The default value of this parameter is 10. A value lesser than the default value may be used to improve the flow performance, if no accuracy issues occur. A value greater than the default value can be used to fix the accuracy issues. The minimum value of this parameter is `0.001` and maximum value is `1000`.
`-area {x1 y1 x2 y2}`	Specifies a set of coordinates within metal fill can be trimmed.
`-attach_instance_name` *attr_num*	Specifies the instance name to attach.
`-attach_net_name` *attr_num*	Specifies the net name to attach.
`-attach_net_prop {{`*prop_name* `attr_num} ...}`	
	Specifies the net property to attach.
`-auto_load_fills`	Updates the FILL data automatically in the software after when the `add_metal_fill_signoff` command modifies the HMF database. In case the HMF database is not modified, the FILL data will be hidden in the software's design.
`-bg`	Runs metal fill in the background mode without locking up the software license. **Note:** All the metal fill options running in the background must be used with the `-bg` parameter.
`-clock`	Specifies whether to trim metal fill around all clock nets.

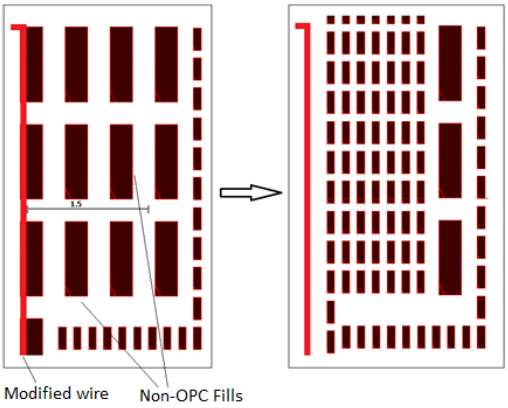
`-common_blockage {`*layer_num1 layer_datatype1* `...}`	Specifies common blockage fill layers. This allows you to specify the blockage layer only once, instead of multiple individual blockages for the metal fill layers. The parameter value format is `{`*layer_number layer_datatype*`}`. You can also specify multiple layers, and each blockage layer should specify a set of two values. The *layer_number* must be an integer. Following are the examples of the usage of this parameter:
	`set_metal_fill_signoff_mode -common_blockage {150 28}`
	`set_metal_fill_signoff_mode -common_blockage {150 28 120 55}`

`-control` *file_name*	Specifies the control file, which is created automatically when a job is submitted from the PVS GUI. This parameter can be used, if you have run a directory submitted from the GUI. Use this parameter to rerun the PVS job from the command line and get the results, which are exactly the same as the GUI submission. The control file is placed in the run directory with the `pvsdrcctl` name and has only the command entries for certain fields in the GUI. You must not modify the file by adding commands or parameters that are not available in the GUI. If you do that, the commands can be ignored by the PVS rule compiler or the results can be indeterminate.
`-delete_area {{`*layer_list1*`} {`*x1 y1 x2 y2*`} ...}`	Specifies the x and y coordinates of the area where metal fill is deleted. Following is the usage example of this parameter: `set_metal_fill_signoff_mode -delete_area {{M1 M2 M6} {0 0 50 50}}`
`-delete_point {{`*layer_list1*`}{`*x1 y1*`}...}`	Specifies the x and y coordinates of the point where metal fill is deleted. Following is the usage example of this parameter: `set_metal_fill_signoff_mode -delete_point {{M2} {134.7 14.21}}`
`-die_area_as_boundary`	Specifies the die area as boundary.

`-dp` *n*	Specifies the number of local cores to use. The default value is fetched from the EDI settings.
`-dp_timeout` *value*	Specifies the time to wait for the slave processes to start.
`-excl_area_size` *size_value*	Specifies the area to be created around the metal fill area to check for the existing metal fill shapes and to avoid creation of metal fill shapes too close to the existing metal fill shapes. The default value of this parameter is `1.6`. A value lesser than the default value may be used to improve the flow performance, if no accuracy issues occur. A value greater than the default value can be used to fix the accuracy issues. The minimum value of this parameter is `0.001` and maximum value is `1000`. The following picture shows the usage of `-and_area_size` and `-excl_area_size` parameters: 
`-fill_cell` *cell_name*	Specifies metal fill cell name.
`-fill_layer {`*layers_list*`}`	Fills only a set of specified metal and/or via metal layers. Following is the usage example of this parameter: `set_metal_fill_signoff_mode -fill_layer {M1 VIA12 M2}`
`-fill_output_mode {gdsii | oasis}`	Generates metal fill database in the `gdsii` or `oasis` format. The default format is `gdsii`.

`-layer_map_file` *streamOut_map_file*	Specifies the `streamOut` metal layer map file.
`-license_timeout`	Specifies the time (in seconds) to wait if the software license is not immediately available while initiating the run. The default value of this parameter is 300 seconds.
`-license_dp_continue`	Initiates the software run using fewer available CPUs based on the number of licenses obtained. Use this parameter in case PVS cannot obtain the required number of licenses in the multiprocessing mode.
`-lib_name library_name`	Specifies the library name.
`-lsf`	Distributes the processing of the software run to different CPUs using LSF. While using this parameter, the `DP_LSF_RESOURCE` environment variable must be set to specify the LSF resource string, as shown below: `setenv DP_LSF_RESOURCE 'resource string for slave processes'` Additionally, the `-temp_working_dir` *work_dir* should be used to specify local working directory for the LSF run.
`-master_lsf`	Submits the master PVS process on LSF. While using this parameter, the `MASTER_LSF_RESOURCE` environment variable should be set to specify LSF resource string, as shown below: `setenv MASTER_LSF_RESOURCE 'resource string for master process'`
`-` `merge` *list_of_external_stream_files*	Merges the external stream files.
`-min_density percent1` *layer_list1*	Specifies percentage value of the minimum metal density. The default value of this parameter is `0` for each metal layer. This parameter trims metal fill below when the specified window density percentage could cause any metal density violations. This parameter is controlled by the `-window_size` and `-window_step` values, as shown in the below example: `set_metal_fill_signoff_mode -min_density {20 {M1 M2} 35 M3}`
`-net net_names`	Specifies a list of critical net names or pointers, which are surrounded by the metal fill area.
`-no_structure_name`	Merges the GDSII file without loading the design.
`-no_via_fills`	Specifies not to generate metal via fills.
`-offset {`*x y*`}`	Defines the offset displacement to be applied.
`-output_macros`	Displays the LEF abstract information about macros, such as LEF pin geometries and obstructions.
`-report_file`	Displays the cell name, which is changed to file.
`-reset {`*parameter_name* `|` *all*`}`	Removes any applied HMF settings.
`-rule_file` *rule_file_name*	Specifies metal fill rule file name.
`-slack_threshold` *float*	Trims any critical net(s) in the timing aware flow with the slack lesser than the specified value (in nanoseconds). Following is the usage example of this parameter: `set_metal_fill_signoff_mode -slack_threshold 0.1`
`-spacing` *float*	Trims metal fill on the specified metal layer around critical net routes. **Note:** The `-spacing` and `-trim_effort` parameters are mutually exclusive.
`-spacing_above` *float*	Trims metal fill on above the specified metal layer around critical net routes.
`-spacing_below` *float*	Trims metal fill on below the specified metal layer around critical net routes.
`-stripes` *number*	Specifies the number of stripes that divide the design. The default is value of this parameter is `1`.

`-structure_name` *structureName*	Specifies the superstructure that consists of all the geometries that the map file writes out.
`-tech_lib` *string*	Specifies the PVS technology library for `-fill` and `-incremental`.
`-tech_set` *string*	Specifies the PVS technology set for `-fill` and `-incremental`.
`-technology` *string*	Specifies the PVS technology to use for identifying the sign-off rules file. **Note:** You can use this parameter with either the `-rule_file` or `-technology` parameter.
`-temp_working_dir` *work_dir*	Specifies the working directory for the background or LSF run.
`-trim_effort {high | med | low}`	Trims metal fill area with the high, medium, or low effort around the critical net routes. This parameter also helps in determining the blockage size for trimming metal fill. Following are the parameter values: • `high`: Presents area of two times of the layer pitch value. • `med`: : Presents area of 1.5 times of the layer pitch value. • `low`: : Presents the same area of the layer pitch value. Following is the usage example of this parameter: `set_metal_fill_signoff_mode -trim_effort med` Here, if the M2 layer pitch is `0.8 um`, M2 fills will be trimmed around selected nets within the distance of `1.2 um`. **Note:** The `-trim_effort` and `-spacing` parameters are mutually exclusive.
`-trim_layer {`*layers_list*`}`	Trims only the set of specified layers. Following is the usage example of this parameter: `set_metal_fill_signoff_mode -trim_layer {M1 M2}`

`-trim_mf` *spacing_file*	Specifies the spacing file containing information related to trim metal fill in the incremental run. Spacing file contains spacing values between the routed metal, non-OPC and OPC metal fill layers. Metal fills are trimmed around ECO changes according to the specified spacing values, post which incremental run generates fill data in the trimmed area. Following is the format of the spacing file:
	`#layer f-a o-a`
	`M1 0.60 0.16`
	`M2 0.60 0.16`
	`M3 0.60 0.16`
	`M4 0.60 0.16`
	`M5 0.60 0.16`
	`M6 0.60 NA`
	Here:
	• `Mx`: Specifies the LEF layer name.
	• `f-a`: Specifies the metal fill active spacing.
	• `o-a`: Specifies the is OPC fill active spacing.
	• `NA`: Refers to no trimming.
	The following picture shows the usage of `-trim_mf` parameter:
	 
	
	The spacing file for the above example has the following settings for the M2 layer:
	`M2 1.5 0.6`
	In this example, the non-OPC metal fills are trimmed in the area of 1.5 times of the modified. Additionally, the metal fill data is regenerated in the trimmed area.

`-union_density` {*percent1 layer_list1* [{*percent2 layer_list2*}]...}	
	Specifies percentage value of the union metal density. The default value of this parameter is `0` for each metal layer. Union density for a layer is the sum of metal density of the current layer, layer below, and layer above the specified windows. Metal fills are trimmed not to violate the `-union_density` settings for the current metal. Following is the usage example of this parameter: `set_metal_fill_signoff_mode -union_density {20 M1}`
`-uniquify_cell_names`	Creates unique cell names, if there are name collisions while using the `-merge` parameter.
`-units {100 | 200 | 400 | 800 | 1000 | 2000 | 4000 | 8000 | 10000 | 20000}`	
	Specifies the resolution of the input/output file. The default value of this parameter is obtained from software database.

`-window_size float`	Specifies the size of the window, which the metal trim uses to examine the metal density. The default value of this parameter is `100`. Following is the usage example of this parameter: `set_metal_fill_signoff_mode -window_size 50`
`-window_step float`	Specifies the stepping distance between two successive positions of the window for a stepped-window trim density check. The default value of this parameter is `50`. Following is the usage example of this parameter: `set_metal_fill_signoff_mode -window_step 25`

Examples

- The following command sets a new option for the signoff metal fill flow:
  `set_metal_fill_signoff_mode -layer_map_file ./RULE/stream.map`
- The following command sets multiple new options for the signoff metal fill flow:
  `set_metal_fill_signoff_mode -layer_map_file ./stream.map -units 2000`
- The following command sets a new option with several parameters for the signoff metal fill flow:
  `set_metal_fill_signoff_mode -union_density {{50 M1} {53 M3}}`
- The following command removes a list of options for the signoff metal fill flow:
  `set_metal_fill_signoff_mode -reset {-layer_map_file}`
- The following command removes all the options for the signoff metal fill flow:
  `set_metal_fill_signoff_mode -reset all`

Related Information

- `add_metal_fill_signoff`
- `get_metal_fill_signoff_mode`

set_pg_fill_config

```
set_pg_fill_config
[-help]
[-attach_instance_name attr_num]
[-attach_net_name attr_num]
[-attach_net_prop {{prop_name attr_num} ... }]
[-core_license license_combination]
[-die_area_as_boundary]
[-dp n]
[-dp_timeout seconds]
[-extra_options options]
[-format {stream oasis}]
[-layer_map_file streamOut_map_file]
[-lib_name library_name]
[-license_dp_continue]
[-license_timeout seconds]
[-merge list_of_external_stream_files]
[-no_structure_name]
[-offset {x y}]
[-output_macros]
[-pg_fill_config_file file_name]
[-pg_hookup_flow {ALL | PG_ONLY | FLOAT_ONLY}]
[-report_file]
[-reset {parameter_name | all}]
[-stripes number]
[-structure_name structureName]
[-uniquify_cell_names]
[-units {100 | 200 | 400 | 800 | 1000 | 2000 | 4000 | 8000 | 10000 | 20000}]
[-working_dir working_directory]
```

Sets the parameters of the `add_pg_fill` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `set_pg_fill_config` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man set_pg_fill_config`
`-attach_instance_name attr_num`	Specifies the instance name to be attached.
`-attach_net_name attr_num`	Specifies the net name to be attached.
`-attach_net_prop {{prop_name attr_num} ... }`	Specifies the net property to be attached.
`-core_license license_combination`	Sets an exact combination of licenses to be used.
`-die_area_as_boundary`	Writes the die area as a polygon or a boundary. When this parameter is not specified, the `streamOut` command writes the die area as a rectangle or a box.
`-dp n`	Runs Pegasus in a multi-processor mode. The `n` variable specified with this parameter presents the number of processors. By default, this value is taken from the Innovus multi-CPU usage.
`-dp_timeout seconds`	Specifies the wait time for the DP processes to start.
`-extra_options options`	Specifies the additional run options.
`-format {stream oasis}`	Specifies the format of the flow.
`-layer_map_file streamOut_map_file`	Specifies the name of the `streamOut` layer map file.
`-lib_name library_name`	Specifies the library name.

`-license_dp_continue`	Specifies that if the required number of licenses cannot be obtained in the DP processes, the software continues with fewer CPUs based on the number of licenses obtained.
`-license_timeout` *seconds*	Specifies the wait time (in seconds) to run the software, if a license is not immediately available. *Default*: `300`
`-merge` *list_of_external_stream_files*	Specifies the list of external stream files.
`-no_structure_name`	Merges the GDSII file without loading the software database. **Note:** This parameter is used with the `-merge` parameter.
`-offset {`*x y*`}`	Specifies the location (in microns) of the design in the GDSII output file.
`-output_macros`	Displays the LEF abstract information, such as LEF pin geometries and obstructions for macros.
`-pg_fill_config_file` *file_name*	Specifies the configuration file for the PG fill flow. Any overrides to the default behavior are written in this file.
`-pg_hookup_flow {ALL | PG_ONLY | FLOAT_ONLY}`	Specifies the type of fills to be generated. *Default*: `PG_ONLY`
`-report_file`	Specifies the cell name to be changed to the file.
`-reset {`*parameter_name* `|` *all*`}`	Resets a specified parameter or all parameters to the default value.
`-stripes` *number*	Specifies the number of stripes that divide the design. *Default*: 1
`-structure_name` *structureName*	Specifies the superstructure that consists of all the geometries that the map file writes out.
`-uniquify_cell_names`	Creates unique cell names if there are name collisions, when the software merges files.
`-units {100 | 200 | 400 | 800 | 1000 | 2000 | 4000 | 8000 | 10000 | 20000}`	
	Specifies the resolution for values in the GDSII/OASIS file.
`-working_dir` *working_directory*	Specifies the working directory.

set_track_fill_config

```
set_track_fill_config
[-help]
[-attach_instance_name attr_num]
[-attach_net_name attr_num]
[-attach_net_prop {{prop_name attr_num} ... }]
[-core_license license_combination]
[-die_area_as_boundary]
[-dp n]
[-dp_timeout seconds]
[-extra_options options]
[-format {stream oasis}]
[-layer_map_file streamOut_map_file]
[-lib_name library_name]
[-license_dp_continue]
[-license_timeout seconds]
[-merge list_of_external_stream_files]
[-no_structure_name]
[-offset {x y}]
[-output_macros]
[-report_file]
[-reset {parameter_name | all}]
[-stripes number]
[-structure_name structureName]
[-track_fill_config_file file_name]
[-uniquify_cell_names]
[-units {100 | 200 | 400 | 800 | 1000 | 2000 | 4000 | 8000 | 10000 | 20000}]
[-working_dir working_directory]
```

Sets the parameters of the add_track_fill command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `set_track_fi` For a detailed description of the command and all its parameters, use the `man` command: `man set_track_fill_config`
`-attach_instance_name` *attr_num*	Specifies the instance name to be attached.
`-attach_net_name` *attr_num*	Specifies the net name to be attached.
`-attach_net_prop {{`*prop_name attr_num*`} ... }`	Specifies the net property to be attached.
`-core_license` *license_combination*	Sets an exact combination of licenses to be used.
`-die_area_as_boundary`	Writes the die area as a polygon or a boundary. When this parameter is not specified, the `strea`
`-dp` *n*	Runs Pegasus in a multi-processor mode. The *n* variable specified with this parameter presents Innovus multi-CPU usage.
`-dp_timeout` *seconds*	Specifies the wait time for the DP processes to start.
`-extra_options` *options*	Specifies the additional run options.
`-format {stream oasis}`	Specifies the format of the flow.
`-layer_map_file` *streamOut_map_file*	Specifies the name of the `streamOut` layer map file.
`-lib_name` *library_name*	Specifies the library name.
`-license_dp_continue`	Specifies that if the required number of licenses cannot be obtained in the DP processes, the s licenses obtained.
`-license_timeout` *seconds*	Specifies the wait time (in seconds) to run the software, if a license is not immediately available *Default*: `300`
`-merge` *list_of_external_stream_files*	Specifies the list of external stream files.
`-no_structure_name`	Merges the GDSII file without loading the software database. **Note:** This parameter is used with the `-merge` parameter.
`-offset {`*x y*`}`	Specifies the location (in microns) of the design in the GDSII output file.
`-output_macros`	Displays the LEF abstract information, such as LEF pin geometries and obstructions for macros
`-report_file`	Specifies the cell name to be changed to the file.
`-reset {`*parameter_name* `|` *all*`}`	Resets a specified parameter or all parameters to the default value.
`-stripes` *number*	Specifies the number of stripes that divide the design. *Default*: 1
`-structure_name` *structureName*	Specifies the superstructure that consists of all the geometries that the map file writes out.
`-track_fill_config_file` *file_name*	Specifies the configuration file for Track fill flow.
`-uniquify_cell_names`	Creates unique cell names if there are name collisions, when the software merges files.
`-units {100 | 200 | 400 | 800 | 1000 | 2000 | 4000 | 8000 | 10000 | 20000}`	
	Specifies the resolution for values in the GDSII/OASIS file.
`-working_dir` *working_directory*	Specifies the working directory.

setMetalFill

```
setMetalFill
[-help]
[-layer {layerNumber} | {list_of_layer_numbers}]
[-activeSpacing spacing]
[-opcActiveSpacing spacing]
[-borderSpacing spacing]
[-decrement value]
[-diagOffset x y]
[-externalDensity percent]
[-gapSpacing spacing]
[-honorLefValue]
[-iterationName name]
[-maxDensity percent]
[-maxLength length]
[-maxUnionDensityToAboveLayer value]
[-maxWidth width]
[-minDensity percent]
[-minLength length]
[-minUnionDensityToAboveLayer value]
[-minWidth width]
[-opc | -regular]
[-preferredDensity percent]
[-windowSize xSize ySize]
[-windowStep xStep yStep]
```

Specifies values for parameters that the `add_metal_fill_signoff`, `trimMetalFill`, and `verifyMetalDensity` commands use for metal fill size, spacing, verification, and metal density. You can also specify some of these parameters in the Layer (Routing) section of the LEF file - they are identified in the "Parameters" section below.

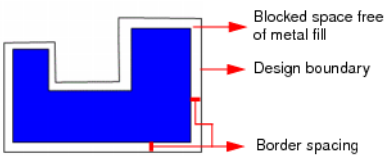
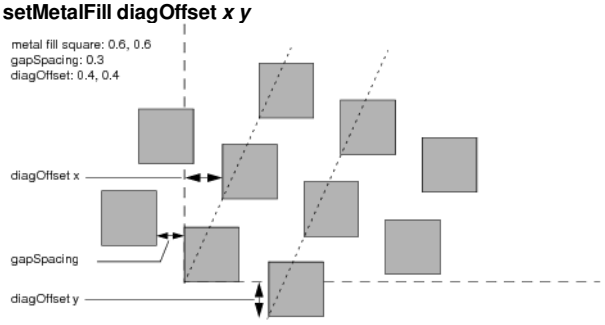
If you want your metal fill settings to be saved in the .init file when saving the design, you must set the `fill_setting_save` global to `1` before saving the design. You can easily reload your settings by restoring the saved design

Note: Check the chip manufacturer's DRC manual specifications for metal fill (sometimes called dummy fill) for the correct values if they are not specified in the LEF file.

- For more information on LEF, see the "LEF Syntax" chapter in the *LEF/DEF Language Reference*.
- For more information on the metal fill commands, see `add_metal_fill_signoff`, `trimMetalFill`, and `verifyMetalDensity`.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `setMetalFill` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man setMetalFill`.
`-activeSpacing spacing`	Specifies the minimum distance in microns between a metal fill shape and any other type of object in the design. For example, `setMetalFill -activeSpacing 0.6`. **Note:** The `setMetalFill -activeSpacing` settings are used for creating regular `FILLWIRE` shapes. For `FILLWIREOPC` shapes, design rules are used. LEF equivalent: `FILLACTIVESPACING` *Default:* `FILLACTIVESPACING` in the LEF file; if not specified, `0.6` for thin layers (less than 0.24), `0.8` for thick layers

`-borderSpacing` *spacing*	Specifies the space in microns along the design border that needs to be kept free of metal fill. This parameter is useful if you want open space next to the place and route border. If you have set border spacing, the `add_metal_fill_signoff` and `verifyMetalDensity` commands consider the density in the specified border space area as 0. 
`-decrement` *value*	Specifies the number of microns the software decreases the size of each successively smaller piece of metal fill or via fill until it reaches the size specified by the `-minWidth` parameter. The software uses the maximum fill size specified unless it is impossible to fit a piece of fill into a particular area. The software then attempts to use successively smaller pieces of fill until the minimum length value is reached. This parameter allows you to control how many different sizes of metal fill or via fill can appear within the design. For example, if the value of `-maxWidth` is `8`, the value of `-minWidth` is `4`, and the value of `-decrement` is `2`, the software can add fill shapes with widths of `8`, `6`, and `4` microns. The software tries to add fill shapes with the width specified by `-maxWidth`. If a shape of that width does not fit in the available space without creating a violation, the software decreases the width by the amount specified by the `-decrement` parameter. It continues to decrease the width until it reaches the width specified by the `-minWidth` parameter. To increase flexibility in adding metal or via fill, decrease the value of this parameter. For example, if the value of `-decrement` is `1`, metal fill with widths of `8`, `7`, `6`, `5`, and `4` microns can be added. *Default:* `WIDTH` value specified in the `Layer` (Routing) statement of the LEF file
`-diagOffset` *x y*	Specifies the offset for staggering metal fill. To enable this parameter, `add_metal_fill_signoff` must be used. *x* Specifies the horizontal offset. *y* Specifies the vertical offset. **setMetalFill diagOffset *x y*** 
`-externalDensity` *percent*	Specifies a percentage value for the external metal density for a layer when `add_metal_fill_signoff` windows lie partially outside of the die area. For each layer, a different external density value can be specified. Metal fill uses the external density value in the estimation of layer density outside of the blocks being developed.

`-gapSpacing` *spacing*	Specifies the minimum distance in microns between one metal fill shape and another metal fill shape. For example, `setMetalFill -gapSpacing 0.4`. *Default:* `0.4` for thin layers (less than 0.24 width/minWidth), `0.8` for thick layers
`-honorLefValue`	Specifies that the LEF values `MINIMUMDENSITY`, `MAXIMUMDENSITY`, `DENSITYCHECKWINDOW`, and `DENSITYCHECKSTEP` should be used to overwrite the `minDensity`, `maxDensity`, `windowSize`, and `windowStep` parameters of `setMetalFill` if the values specified for these parameters are not valid. If the `-layer` option is used with `setMetalFill`, only the specified layers are changed to the LEF values. **Note:** If one of the keywords `-MINIMUMDENSITY`, `MAXIMUMDENSITY`, `DENSITYCHECKWINDOW`, or `DENSITYCHECKSTEP`-is not defined in the LEF, an error message like the following is displayed: `ERROR: DENSITYCHECKWINDOW of MetalX is not defined in the LEF.`

`-iterationName` *name*	Specifies a name for a set of values for `setMetalFill` parameters that the software stores and uses during a later `add_metal_fill_signoff` run. Use `add_metal_fill_signoff` to add the metal fill using the stored set of parameters.
	The window size and step must be the same for all iterations of a specific layer. For example, the following specifications are not allowed because the values are not consistent:
	`setMetalFill -iterationName step1 -layer 1 \`
	`  -minDensity 20 -windowSize 100 100 -windowStep 50 50`
	`setMetalFill -iterationName step1 -layer 1 \`
	`  -minDensity 25 -windowSize 50 50 -windowStep 25 25`
	Important Specifying `setMetalFill -iterationName default` is the same as running `setMetalFill` without specifying the `-iterationName` parameter.
	`setMetalFill -iterationName default` is expected before `verifyMetalDenstiy`. `verifyMetalDenstiy` honors only `default iterationName`. If `default iterationName` is not set, `default` value will be used in `verifyMetalDenstiy`.
	Default: "`default`"

`-layer {`*layerNumber*|`{`*list_of_layer_numbers*`}`	Specifies the layers to which the parameters of the `setMetalFill` command apply. If you specify more than one layer, separate each layer with a space and enclose the list in braces or double quotation marks.
	To specify metal layers Metal1, Metal2, and Metal3, type one of the following commands:
	`setMetalFill -layer "METAL1 METAL2 METAL3"`
	`setMetalFill -layer {METAL1 METAL2 METAL3}`

`-maxDensity` *percent*	Specifies a percentage value for the maximum metal density allowed. LEF equivalent: `MAXIMUMDENSITY` *Default:* `MAXIMUMDENSITY`. If this value is not defined, the default is `80`. **Note:** This parameter only overrides a value in the LEF file if it is more restrictive than the LEF value. For example, if the value specified in the LEF file is 80, and you specify `-maxDensity 70`, metal fill uses 70 percent as the maximum density.
`-maxLength` *length*	Specifies the maximum length of the metal fill shapes, in microns. Metal fill stops at the chip boundary if the specified maximum length would cause it to extend outside the boundary. *Default:* `10.0`
`-maxUnionDensityToAboveLayer` *value*	Specifies the maximum allowed sum union density of the current layer and its top layer window. This avoids the union density violations found while verifying density results of trim metal fill flow using PVS or Pegasus. *Default:* `0`
`-maxWidth` *width*	Specifies the maximum width of metal fill shapes, in microns. Metal fill stops at the chip boundary if the specified maximum length would cause it to extend outside the boundary. *Default:* Maximum routing width defined in the LEF file; if not specified, `2.0` LEF equivalent: `MAXWIDTH` **Note:** This parameter only overrides a value in the LEF file if it is more restrictive than the LEF value.

`-minDensity` *percent*	Specifies a percentage value for the minimum metal density allowed in the design. LEF equivalent: `MINIMUMDENSITY` *Default:* `MINIMUMDENSITY`. If this value is not defined, the default is `20`. **Note:** This parameter only overrides a value in the LEF file if it is more restrictive than the LEF value. For example, if the value specified in the LEF file is 20, and you specify `-minDensity 25`, metal fill uses 25 percent as the minimum density.
`-minLength` *length*	Specifies the minimum length of metal fill shapes, in microns. *Default:* `1.0`
`-minUnionDensityToAboveLayer` *value*	Specifies the minimum allowed sum union density of the current layer and its top layer window. This avoids the union density violations found while verifying density results of trim metal fill flow using PVS or Pegasus. *Default:* `200`
`-minWidth` *width*	Specifies the minimum width of metal fill shapes, in microns. *Default:* `0.4` for thin layers (less than 0.24) and `0.8` for thick layers. **Note:** This parameter only overrides a value in the LEF file if it is more restrictive than the LEF value.
`-opc`	Specifies metal fill that will go through the optical proximity correction (OPC) process. This fill can be smaller than non-OPC metal fill and is governed by the same design rules as regular wires, instead of the rules that govern non-OPC metal fill. • OPC metal fill that is floating (unconnected) is included in the Fills statement of DEF 5.7. • OPC metal fill that is tied off (connected) is included in the Special Wiring statement of DEF 5.7. • The GDSII Stream/OASIS map file identifies the following object names for OPC and non-OPC metal fill: `FILL`, `FILLOPC`, `VIA`, `VIAFILL`. **Note:** `-opc` and `-regular` cannot be specified at the same time.
`-opcActiveSpacing` *spacing*	Specifies the minimum distance in microns between the OPC metal fill shapes (FILLWIREOPC) and main patterns. **Note:** If this parameter is not specified, the minimum DRC spacing is used by default.
`-preferredDensity` *percent*	Specifies a percentage value for the preferred metal density for the layer. When the metal density in a window is less than the minimum metal fill density value, adds metal fill to achieve a density slightly above the preferred density if possible. In addition, metal fill uses the preferred density value in the estimation of `MACRO` density for layers that have most of the `MACRO` covered by `OBS` shapes and for DEF `BLOCKAGES` without both `+ FILLS` and `+ PUSHDOWN`. Metal fill uses the following scheme for the treatment of DEF blockages with `+ FILLS` or `+ PUSHDOWN`: • DEF `BLOCKAGES` with `+ PUSHDOWN` = 100 percent density (the same as real metal segments) • DEF `BLOCKAGES` with `+ FILLS` = 0 percent density Cadence recommends modelling LEF `MACRO`s by using the `DENSITY` statement, with density windows that are smaller than the metal fill windows. For example, if the metal fill windows are 100 x 100, use 25 x 25 or smaller values for the density rectangles. For more information, see the "Adding Metal Fill Over Macros" section in the Optimizing Metal Density chapter of *User Guide*. *Default:* `35`

`-regular`	Specifies that regular metal fill shape is to be used. Use this option to switch to inserting regular metal fill without restarting the session after you have been adding OPC metal fill.
	`setMetalfill -opc`
	`add_metal_fill_signoff`
	`#Inserts opc metal fill`
	`setMetalfill -regular`
	`add_metal_fill_signoff`
	`#Inserts regular metal fill`

`-windowSize` *`xSize ySize`*	Specifies the x and y dimensions in microns of the area `setMetalFill` uses to examine the metal density. The software assesses the metal density in each window, moving the window iteratively through the design. For example, `setMetalFill -windowSize 100 100`. LEF equivalent: `DENSITYCHECKWINDOW` *Default:* `DENSITYCHECKWINDOW`. If this value is not defined, the default is 100 microns in each dimension.
`-windowStep` *`xStep yStep`*	Specifies the x and y step distance in microns the window moves for each iteration of metal density analysis. For example, `setMetalFill -windowStep 50 50`. LEF equivalent: `DENSITYCHECKSTEP` *Default:* `DENSITYCHECKSTEP`. If this value is not defined, the default is 50 microns in each dimension.

Examples

- The following command sets window size and step settings for layer `METAL1`:

  ```
  setMetalFill -layer METAL1 -windowSize 200 200 -windowStep 100 100
  ```

  These parameters are applied when you use the `add_metal_fill_signoff` command.

- The following command specifies an iteration `step1` with various settings for inserting metal fill:

  ```
  setMetalFill -layer {METAL1 METAL2 METAL3 METAL4 METAL5 METAL6 METAL7 METAL8} -iterationName step1 /
               -gapSpacing 0.140 -minWidth 1 -decrement 0.050 /
               -minLength 2 -maxWidth 5 -maxLength 20 -activeSpacing 0.8 /
               -windowSize 50 50 -windowStep 25 25 -minDensity 15 /
               -maxDensity 80 -preferredDensity 35 -diagOffset 1 1
  ```

Related Information

- `fill_setting_save`
- The following sections in the Optimizing Metal Density chapter of *User Guide*.
  - "Overview"
  - "Connected and Floating Metal Fill"
  - "Specifying Metal Fill Parameters"
  - "Timing-Aware Examples"
  - "Specifying the Active Spacing Value"
  - "Adding Via Fill"
- The following section in the Importing and Exporting Designs chapter of *User Guide*.
  - "GDSII Stream or OASIS Map File Format"
- LEF Syntax chapter in the *LEF/DEF Language Reference*
  - "Macro Density"
- DEF Syntax chapter in the *LEF/DEF Language Reference*
  - "Blockages"

setMetalFillSpacingTable

```
setMetalFillSpacingTable
[-help]
[-layer layer_name_list]
[-fill_to_active {{width1 spacing1}{width2 spacing2}...}]
[-fill_to_fill {{width1 spacing1}{width2 spacing2}...}]
[-fill_to_opc {{width1 spacing1}{width2 spacing2}...}]
[-opc_to_active {{width1 spacing1}{width2 spacing2}...}]
[-opc_to_opc {{width1 spacing1}{width2 spacing2}...}]
[-reset]
```

Specifies the spacing table used by `trimMetalFill` based on existing metal fill width. The spacing table includes the following spacing values:

- Fill to active spacing
- Fill to fill spacing
- Fill to OPC spacing
- OPC to OPC spacing
- OPC to active spacing

As `trimMetalFill` honors the spacing table, you can then run `trimMetalFill` to trim metal fill and get expected spacing. When required, you can use the `getMetalFillSpacingTable` command to print the spacing table.

Note: If no spacing table is specified, `trimMetalFill` honors the setMetalFill settings for `-activeSpacing` and `-gapSpacing`. If `setMetalFill` values are also not specified, trimMetalFill uses the default settings.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `setMetalFillSpacingTable` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man setMetalFillSpacingTable`.
`-layer` *layer_name_list*	Specifies the metal layers for the setting. Metal layers can be specified as numbers (`2`) or in the LEF format (`METAL2`) or as `M2`.
`-fill_to_active {{`*width1 spacing1*`}{`*width2 spacing2*`}...}`	Specifies a spacing table for fill to active spacing for trimming metal fill.
`-fill_to_fill {{`*width1 spacing1*`}{`*width2 spacing2*`}...}`	Specifies a spacing table for fill to fill spacing for trimming metal fill.
`-fill_to_opc {{`*width1 spacing1*`}{`*width2 spacing2*`}...}`	Specifies a spacing table for fill to OPC spacing for trimming metal fill.
`-opc_to_active {{`*width1 spacing1*`}{`*width2 spacing2*`}...}`	Specifies a spacing table for OPC to active spacing for trimming metal fill.
`-opc_to_opc {{`*width1 spacing1*`}{`*width2 spacing2*`}...}`	Specifies a spacing table for OPC to OPC spacing for trimming metal fill.
`-reset`	Resets the spacing tables.

Example

The following set of commands specifies the spacing table for fill to active spacing as given in the below table and trims metal fill accordingly:

```
+-------+---------------+-----------------+--------------------+
| Layer | Minwidth (=>) |   Maxwidth (<=) | activeSpacing (>=) |
+-------+---------------+-----------------+--------------------+
|   2   |      0.32     |        ∞        |        0.63        |
+-------+---------------+-----------------+--------------------+
|   2   |      0.14     |    0.32-1MFG    |        0.36        |
```

```
+-------+---------------+-----------------+--------------------+
|   2   |      0.08     |    0.14-1MFG    |        0.27        |
+-------+---------------+-----------------+--------------------+
```

```
setMetalFillSpacingTable -layer {2} -fill_to_active {{0.08 0.27}{0.14 0.36}{0.32 0.63}}
trimMetalFill -layer 2
```

The software checks the existing metal fill with the specified spacing table. If no violations are detected, existing metal fill is retained. If the existing metal fills have activeSpacing violations with other features, `trimMetalFill` uses the fill_to_active spacing table to trim metal fill.

Sample output is as follows:

```
*** START TRIM METALFILL *** (CPU Time: 0:00:00.0 MEM: 558.359M)
 Number of metal fills with spacing or/and short violations:4920
 Total number of deleted metal fills: 4920
 Total number of added metal fills: 4779
 (CPU Time: 0:00:01.2 MEM: 558.359M)
 *** END OF TRIM METALFILL ***
```

Related Information

- `fill_setting_save`
- `trimMetalFill`
- The following sections in the Optimizing Metal Density chapter of *User Guide*.
  - "Specifying Metal Fill Spacing Table"
  - "Trimming Metal Fill"

setViaFill

```
setViaFill
[-help]
[-layer {layerNumber | {list_of_layer_numbers}}]
[-windowSize xSize ySize]
[-windowStep xStep yStep]
[-minDensity percent]
[-maxDensity percent]
[-preferredDensity percent]
```

Specifies window and density values for inserting via fill. Use this command before running `addViaFill`.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `setViaFill` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man setViaFill`.

`-layer layerNumber | {list_of_layer_numbers}`	Specifies the cut layers to which the parameters of the `setViaFill` command apply. If you specify more than one layer, separate each layer with a space and enclose the list in braces or double quotation marks. To specify *cut12*, *cut23*, and *cut34*, type one of the following commands:
	`addViaFill -layer "Via12 Via23 Via34"`
	`addViaFill -layer {Via12 Via23 Via34}`

`-maxDensity percent`	Specifies a percentage value for the maximum cut density. For example, `setViaFill -maxDensity 30`. *Default:* `30`
`-minDensity percent`	Specifies a percentage value for the minimum cut density. For example, `setViaFill -minDensity 0.01`. *Default:* area of two cuts divided by 100 square microns
`-preferredDensity percent`	Specifies a percentage value for the preferred cut density. When the cut density in a window is less than the minimum cut density value, this parameter adds via fill to achieve a density as close as possible to the preferred value. To disable the preferred density feature, specify `0`. For example, `setViaFill -preferredDensity 0.02`. *Default:* area of two cuts in the cut layer
`-windowSize xSize ySize`	Specifies the x and y dimensions in microns of the area `setViaFill` uses to examine the cut density. The software assesses the cut density in each window, moving the window iteratively through the design. For example, `setViaFill windowSize 100 100`. *Default:* 10 microns in each dimension
`-windowStep xStep yStep`	Specifies the x and y step distance in microns the window moves for each iteration of cut density analysis. For example, `setViaFill -windowStep 50 50`. *Default:* 5 microns in each dimension

Example

The following command sets window and density values for inserting via fill in cut layers `VIA1` to `VIA7`:

```
setViaFill -layer {VIA1 VIA2 VIA3 VIA4 VIA5 VIA6 VIA7} -windowSize 100 100 /
-windowStep 50 50 -minDensity 0.01 -MaxDensity 30 /
-preferredDensity 1
```

Related Information

- `fill_setting_save`
- The following sections in the Optimizing Metal Density chapter of *User Guide*.
  - "Overview"
  - "Adding Via Fill"

trimMetalFill

```
trimMetalFill
[-help]
[-allowPolygon]
[-area x1 y1 x2 y2]
[-deleteViol]
[-ignoreSpecialNet]
[-layer {layer_num_list | layer_name_list}]
[-useNonDefaultSpacing]
```

Trims metal fill and via fill (`FILLWIRE` and `FILLWIREOPC` shapes) that causes DRC violations. This command uses the values set by the `setMetalFill` command. Therefore, you must ensure that you have correct `setMetalFill` settings before running `trimMetalFill` to avoid false violations.

Note: Use `setMetalFill` without `-iterationName` or with `-iterationName default` to set the DRC rules used by `trimMetalFill`. `setMetalFill -iterationName name`, where *name* is other than `default`, is used to set construction rules used by `add_metal_fill_signoff`.

Use this command to trim metal fill if the design has floating metal or via fill. By trimming the fill, you reduce the chance of introducing timing problems caused by cross-coupling.

If the design has metal or via fill that is connected to special nets, use `fixOpenFill` after using `trimMetalFill`.

Use the `trimMetalFill` command after routing signal nets. For more information, see Route Commands.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `trimMetalFill` parameter. For a detailed description of the command and all its parameters, use `man trimMetalFill`.
`-allowPolygon`	Allows rectilinear shapes after trimming metal fill (`FILLWIRE` and `FILLWIREOPC` shapes). By default, the Innovus software supports rectangular shapes only.
`-area x1 y1 x2 y2`	Specifies a set of coordinates within which metal fill needs to be trimmed. By limiting the area in which metal fill is trimmed, you can speed up the trimming process. **Note:** This option is recommended for use with only floating metal fill. If you use `trimMetalFill -area` on tied-off fill shapes, the command returns a warning message that some of the shapes may become isolated from the Power/Ground network.
`-deleteViol`	Deletes metal fill shapes that cause DRC violations or shorts, instead of trimming sections of the shapes. After running the `trimMetalFill` command with this parameter specified, the remaining shapes are still rectangles. If using this parameter causes metal density to drop below the target density, re-run the `add_metal_fill_signoff` command. Always specify this parameter in the following two cases: • If the design has minimum step rules, specify this parameter to avoid creating minimum step violations. • If you ran `add_metal_fill_signoff`, specify this parameter to delete entire shapes, instead of trimming sections of the shapes, otherwise the trimmed shapes will not be on-grid.
`-ignoreSpecialNet`	Ignores special nets while checking for DRC violations.
`-layer {layer_num_list | layer_name_list}`	Specifies the metal layers in which metal fill needs to be trimmed. **Note:** This option is recommended for use with only floating metal fill. If you use `trimMetalFill -layer` on tied-off fill shapes, the command returns a warning message that some of the shapes may become isolated from the Power/Ground network.
`-useNonDefaultSpacing`	Uses non-default spacing for trimming metal fill. Some critical nets use a non-default rule (NDR) for routing as they may need to have wider wires or wider spacing than other nets. Use `-useNonDefaultSpacing` to extend NDR control to `trimMetalFill`.

Example

- The following commands trim metal fill after ECO routing.
  ```
  setNanoRouteMode -route_with_eco true
  globalDetailRoute
  trimMetalFill
  ```
- The following command trims metal fill and allows rectilinear shapes:
  ```
  trimMetalFill -allowPolygon
  ```

Related Information

- `add_metal_fill_signoff`
- The following sections in the Optimizing Metal Density chapter of *User Guide*.
  - "Overview"
  - "Connected and Floating Metal Fill"
  - "Trimming Metal Fill"
- DEF Syntax chapter of the *LEF/DEF Language Reference*
  - "Shape"

trimMetalFillNearNet

```
trimMetalFillNearNet
[-help]
[-area {x1 y1 x2 y2}]
[-clock]
[-createFillBlockage blockage_name]
[-layer layer_name_list]
[-minTrimDensity {percent1layer_list1 [{percent2layer_list2}]}]
[-minTrimUnionDensityToAboveLayer {percent1 layer_list1 [{percent2 layer_list2}]..}]
[-net {net_name_list | net_object_pointer_list}]
[-recomputeDensity]
[-remove]
[-slackThreshold slack_in_ns]
[-spacing dist_in_um]
[-spacingAbove dist_in_um]
[-spacingBelow dist_in_um]
```

Trims any metal fill around critical nets. This command enables you to reduce the timing impact on critical nets due to metal fill on the layer above, below and the same layer for the routing of any critical net. Using this command, you can specify the critical net list, desired spacing around critical nets, and metal fill density limit.

The intended usage model is to run timing analysis before metal fill, add metal fill, then run timing analysis after metal fill, then trim the metal fill that caused timing to get worse. See the examples below.

Parameters

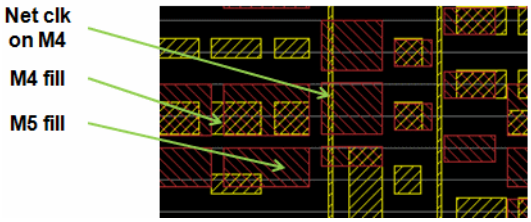
`-help`	Prints a brief description that includes the type and default information for each `trimMetalFillNearNet` parameter. For a detailed description of the command and all its parameters, use `man trimMetalFillNearNet`.
`-area {x1 y1 x2 y2}`	Specifies a set of coordinates within which to trim metal fill. If this parameter is not specified, metal fill is trimmed around critical nets in the entire design area
`-clock`	Specifies whether metal fill is to be trimmed around all clock.
`-createFillBlockage` *blockage_name*	Creates a fill blockage so that any subsequent fill steps do not refill the trimmed area. If you specify a name, the tool uses that name as the fill blockage name. If you do not specify a name, the tool uses the default name *netName_layerName_** for the fill blockage.
`-layer` *layer_name_list*	Trims metal fill only on the specified layers. If this parameter is not specified, metal fill is trimmed on all layers.
`-net {`*net_name_list* | *net_object_pointer_list*`}`	Specifies a list of critical net names or pointers which are surrounded by metal fill.
`-minTrimDensity {`*percent1layer_list1* `[{`*percent2layer_list2*`}]}`	Trims metal fill only until the specified window density percentage is reached to avoid causing any metal density violations. The window sizes are controlled by setMetalFill. verifyMetalDensity is used to compute the metal-density using the same setMetalFill settings. You can specify different window density percentages for different layers. For example, `{{20 1 2} {30 3}}` would set the min-density for metal1, metal2 windows to 20%, and for metal3 to 30%.
`-minTrimUnionDensityToAboveLayer {`*percent1 layer_list1* `[{`*percent2 layer_list2*`}]..}`	Trims metal fill only till meeting the specified percent window density. *Default:* `0`
`-recomputeDensity`	By default, if metal density values are required for -minTrimDensity, and they are available in the database from a previous call to verifyMetalDensity, the existing database values are used. This option forces a call to `verifyMetalDensity` to recompute window densities instead of using the stored values.
`-remove`	Specifies that the metal fill shapes overlapping the critical-net halo area should be removed entirely instead of being trimmed to the halo edge created with different `-spacing*` options. This avoids the possibility of smaller metal fill shapes that might be left over after trimming.
`-slackThreshold` *slack_in_ns*	Trims metal fill around nets with slack less than the specified value. The value specified should be in nanoseconds.
`-spacing` *dist_in_um*	Trims fill on the same layer by specified distance around critical net routes. The distance specified should be in micrometers (`um`). The default is 0 so no trimming is done.
`-spacingAbove` *dist_in_um*	Trims fill on the layer above the current layer by specified distance around critical net routes. The distance specified should be in micrometers (`um`). The default is 0 so no trimming is done.
`-spacingBelow` *dist_in_um*	Trims fill on the layer below the current layer by specified distance around critical net routes. The distance specified should be in micrometers (`um`). The default is 0 so no trimming is done.

Example

- The most common usage model is to run timing analysis before metal fill, add metal fill, run timing analysis after metal fill, and then trim the metal fill that causes timing to get worse. Suppose the WNS before metal fill was -0.2ns, and after metal fill the WNS is -0.4ns. This means the metal fill caused a 0.2ns degradation. One strategy is to trim any net that got pushed about beyond the original WNS of -0.2ns aggressively with a larger trim spacing value because the extra WNS is directly caused by metal-fill, and then trim any negative slack nets above 0.0ns to improve timing margins. The spacing values depend on the technology node and timing criticality. A reasonable starting point would be to choose between 5x to 10x min-spacing for the aggressive trim value. So you might use these commands:

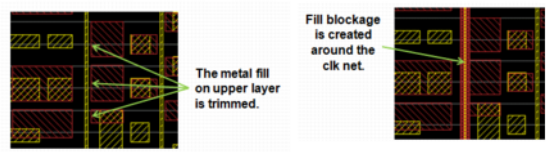
```
#Trim by 1.0um the layer above/below/same for any net with worse slack than the pre-metal fill WNS of -0.2
#Do not trim if the metal density will fall below 20% for metal1, metal2, or 30% for metal3 using the setMetalFill window settings
trimMetalFillNearNet -slackThreshold -0.2 -spacing 1.0 -spacingAbove 1.0 -spacingBelow 1.0 -minTrimDensity {{20 1 2} {30 3}}
#Trim less aggressively using 0.4um spacing for any net with WNS above 0.0 to improve timing margins
trimMetalFillNearNet -slackThreshold 0.0 -spacing 0.4 -spacingAbove 0.4 -spacingBelow 0.4 -minTrimDensity {{20 1 2} {30 3}}
```

- In the following example, there is a critical net overlap with the metal fill on the upper layer, metal5:



The following command trims metal fill on the upper layer, metal5, and creates a fill blockage around the `clk` net.

```
trimMetalFillNearNet -net clk -layer {metal4 metal5} -createFillBlockage -spacingAbove 0.2
```



Related Information

- The following sections in the Optimizing Metal Density chapter of *User Guide*.
  - Overview
  - Trimming Metal Fill
  - Trimming Metal Fill for Timing Closure

Metal and Via Fill Global Variables

- fill_setting_save

fill_setting_save

```
fill_setting_save {0 | 1}
```

Type: Integer

Default: `0`

Specifies whether metal fill settings are to be saved in the .init file when saving the design. Metal fill settings include setMetalFill, setMetalFillSpacingTable, and setViaFill settings. By default, these metal fill settings are lost when you exit Innovus. To enable Innovus to record metal fill settings in the .init file, you must set the `fill_setting_save` to `1` before saving the design. You can easily reload your settings by restoring the saved design:

```
set fill_setting_save 1

setMetalFill iterationName iter1 ...

saveDesign

restoreDesign

#The previous metal fill settings are reloaded.
```

Related Information

- `setMetalFill`
- `setMetalFillSpacingTable`
- `setViaFill`
- `saveDesign`
- `restoreDesign`

20

Mixed Signal Commands

- deleteIntegRouteConstraint
- getIntegRouteConstraint
- pull_block_constraint
- run_vsr
- setIntegRouteConstraint
- writeIntegRouteConstraint

deleteIntegRouteConstraint

```
deleteIntegRouteConstraint
[-help]
{-net net1 net2 ... | -name string | -customConstraints  | -all | -additional }
```

Deletes the routing constraints in the database.

Note: Integration constraints are used to convey the design intent in a mixed signal design. These constraints are fully interoperable between the digital and custom environments.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `deleteIntegRouteConstraint` parameter. For a detailed description of the command and all of its parameters, use the man command `man deleteIntegRouteConstraint`.
`-additional`	Removes the additional constraint file, containing the maximum resistance target values for nets, from memory.
`-all`	Deletes all integration constraints, including constraints that are understood by NanoRoute and used to route nets with simple shield or NDR constraints.
`-customConstraints`	Deletes all diffPair, matchLength, complex shield, and complex netClass constraints.
`-name` *`string`*	Allows the deletion of a specific constraint by name. You can specify multiple constraint names, enclosed in braces. Use the `getIntegRouteConstraint` command to get the name of any constraint in the design.
`-net` *`net1 net2 ...`*	Indicates the nets for which the constraints will be deleted. If the net is part of a netClass, the netClass will be deleted as well.

Example

- The following command deletes all diffPair, matchLength, complex shield, and complex netClass constraints.

  ```
  deleteIntegRouteConstraint -customConstraints
  ```

- The following command deletes the constraints with the specified names:

  ```
  deleteIntegRouteConstraint -name {Constr_2__analogNW__DTMF_INST/analogNW_INST
  Constr_1__analogNW__DTMF_INST/analogNW_INST}
  ```

- The following command deletes all Integration constraints in the design.

  ```
  deleteIntegRouteConstraint -all
  ```

  The tool displays the following warning before deleting the constraints:

  ```
  **WARN: (ENCMSC-270): You are attempting to delete all routing constraints in this
  design. These include many constraints that are intended for use by NR for routing
  nets with simple shield, or NDR constraints.
  ```

Related Information

- `getIntegRouteConstraint`
- `setIntegRouteConstraint`
- `writeIntegRouteConstraint`
- Routing Constraint Interoperability chapter in the *Mixed Signal Interoperability Guide*
- Tools Menu chapter in the *Menu Reference*
  - Mixed Signal -> Integration Constraint Editor

getIntegRouteConstraint

```
getIntegRouteConstraint
[-help]
[-net net_name | -name constraint_name  | -additional]
```

Displays the settings needed for the `setIntegRouteConstraint` command.

If mixed-signal constraints have not yet been defined, the `getIntegRouteConstraint` command does not return anything.

If you run the `getIngegRouteConstraint` command without any option, all nets with all constraint names and the maximum resistance constraint (if already set) are displayed on the console window.

Note: The output is displayed only on the console window and is not written to the log file. You cannot use `>` to write to a file. The only way to send the output to a file is to run the following:

```
redirect {puts [getIntegRouteConstraint xxxx]} > filename
```

Alternatively, use the command `writeIntegRouteConstraint`.

Note: Integration constraints are used to convey the design intent in a mixed-signal design. These constraints are fully interoperable between the digital and custom environments.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `getIntegRouteConstraint` parameter. For a detailed description of the command and all of its parameters, use the man command `man getIntegRouteConstraint`.
`-additional`	Returns the path to the additional constraint file, which contains the maximum resistance target values for nets.
`-name constraint_name`	Returns a specific constraint by name. You can specify multiple constraint names, enclosed in braces.
`-net net_name`	Specifies the net for which the constraints will be returned. If the net is part of a netClass, the constraints for the netClass is returned. You can specify multiple net names, enclosed in braces.

Example

- The following command returns the constraint(s) defined for the net `scan_out_1`:

  ```
  getIntegRouteConstraint -net scan_out_1
  ```

  The output of the command is:

  ```
  {-type shield  -shieldNet { VSS } -shieldType side -shieldTolerance 30.0 -net {
  scan_out_1 }
  }
  ```

- The following command returns details of the constraint named `Constr_13__analogSW__DTMF_INST/analogSW_INST`:

  ```
  getIntegRouteConstraint -name Constr_13__analogSW__DTMF_INST/analogSW_INST
  ```

  The output of the command is:

  ```
  {-type netClass -name Constr_13__analogSW__DTMF_INST/analogSW_INST -shieldNet VSS -
  shieldType side -connectSupply anyPoint -rule NDR8 -net {analogW4I}
  }
  ```

- The following command returns the constraints on the specified nets:
  ```
  getIntegRouteConstraint -net {DTMF_INST/cntrlxo[3] DTMF_INST/cntrlxo[0]}
  ```
  The output of the command is:

  ```
  {-type diffPair -name Constr_2__analogNW__DTMF_INST/analogNW_INST -tolerance 20 -
  rule default -net {DTMF_INST/cntrlxo[3] DTMF_INST/cntrlxo[2]}
  } {-type diffPair -name Constr_1__analogNW__DTMF_INST/analogNW_INST -tolerance 5 -
  rule default -net {DTMF_INST/cntrlxo[0] DTMF_INST/cntrlxo[1]}
  }
  ```

- The following command returns all the constraints on all the nets in the design and the maximum resistance constraint (if already set):
  ```
  getIntegRouteConstraint
  ```

  A sample output of the command is displayed below:

  ```
  {-type diffPair -name Constr_3__analogNL__DTMF_INST/analogNL_INST  -tolerance 20

   -rule default -net {DTMF_INST/digsyncc DTMF_INST/digsyncd}

  } {-type diffPair -name Constr_2__analogNL__DTMF_INST/analogNL_INST  -tolerance

  20 -rule default -net {DTMF_INST/digsynca DTMF_INST/digsyncb}

  } {-type diffPair -name Constr_1__analogNW__DTMF_INST/analogNW_INST  -tolerance

  5 -rule default -net {DTMF_INST/cntrlxo[0] DTMF_INST/cntrlxo[1]}
  ```

```
} {-type diffPair -name Constr_2__analogNW__DTMF_INST/analogNW_INST  -tolerance
20 -rule default -net {DTMF_INST/cntrlxo[3] DTMF_INST/cntrlxo[2]}
} {-type matchLength -name Constr_4__analogNL__DTMF_INST/analogNL_INST  -toleran
ce 20 -matchStyle accordion -rule default -net {DTMF_INST/digsync3 DTMF_INST/dig
sync4}
} {-type matchLength -name Constr_5__analogNL__DTMF_INST/analogNL_INST  -toleran
ce 20 -matchStyle accordion -shieldNet VSS -shieldType unspecified -connectSuppl
y anyPoint -shieldWidth { Metal1 0.56 Metal2 0.56 Metal3 0.56 Metal4 0.56 Metal5
 0.88 Metal6 0.88} -rule default -net {DTMF_INST/digsync2 DTMF_INST/digsync1} ha
sUnknownConstraint
} {-type netClass -name Constr_10__analogSW__DTMF_INST/analogSW_INST  -shieldNet
 VSS -shieldType side -connectSupply anyPoint -rule NDR8 -net {analogW1I} hasUnk
nownConstraint
} {-type netClass -name Constr_11__analogSW__DTMF_INST/analogSW_INST  -shieldNet
 VSS -shieldType side -connectSupply anyPoint -rule NDR8 -net {analogW2I}
} {-type netClass -name Constr_12__analogSW__DTMF_INST/analogSW_INST  -shieldNet
 VSS -shieldType side -connectSupply anyPoint -rule NDR8 -net {analogW3I}
} {-type netClass -name Constr_13__analogSW__DTMF_INST/analogSW_INST  -shieldNet
 VSS -shieldType side -connectSupply anyPoint -rule NDR8 -net {analogW4I}
} {-type shield  -shieldNet { VSS } -shieldType side -net { analogS4I } hasUnkno
wnConstraint
} {-type shield  -shieldNet { VSS } -shieldType side -net { analogS3I } hasUnkno
wnConstraint
} {-type shield  -shieldNet { VSS } -shieldType unspecified -net { analogS2I }
} {-type shield  -shieldNet { VSS } -shieldType side -net { analogS1I }
} {-type shield -rule NDR0.6__analogNE  -shieldNet { AVSS } -shieldType coaxial
-connectSupply anyPoint -shieldWidth { Metal1 0.23 Metal2 0.28 Metal3 0.28 Metal
4 0.28 Metal5 0.28 Metal6 0.44} -tandemWidth { Metal1 1.4 Metal2 1.4 Metal3 1.4
Metal4 1.4 Metal5 1.4 Metal6 1.4} -shieldGap { Metal1 0.23 Metal2 0.28 Metal3 0.
```

```
28 Metal4 0.28 Metal5 0.28 Metal6 0.46} -net { analogE1I } hasUnknownConstraint
} {-type shield  -shieldNet { AVSS } -shieldType coaxial -connectSupply anyPoint
 -shieldWidth { Metal1 0.23 Metal2 0.28 Metal3 0.28 Metal4 0.28 Metal5 0.28 Meta
l6 0.44} -tandemWidth { Metal1 1.4 Metal2 1.4 Metal3 1.4 Metal4 1.4 Metal5 1.4 M
etal6 1.4} -shieldGap { Metal1 0.23 Metal2 0.28 Metal3 0.28 Metal4 0.28 Metal5 0
.28 Metal6 0.46} -net { analogN4I } hasUnknownConstraint
} {-type shield  -shieldNet { AVSS } -shieldType coaxial -connectSupply anyPoint
 -shieldWidth { Metal1 0.23 Metal2 0.28 Metal3 0.28 Metal4 0.28 Metal5 0.28 Meta
l6 0.44} -tandemWidth { Metal2 1.4 Metal3 1.4 Metal4 1.4 Metal5 2.24 Metal6 1.4}
 -shieldGap { Metal1 0.23 Metal2 0.28 Metal3 0.28 Metal4 0.28 Metal5 0.28 Metal6
 0.46} -net { analogN3I } hasUnknownConstraint
} {-type shield  -shieldNet { AVSS } -shieldType coaxial -connectSupply anyPoint
 -shieldWidth { Metal1 0.23 Metal2 0.28 Metal3 0.28 Metal4 0.28 Metal5 0.28 Meta
l6 0.44} -tandemWidth { Metal1 1.4 Metal2 1.4 Metal3 1.4 Metal4 1.4 Metal5 1.4 M
etal6 1.4} -shieldGap { Metal1 0.23 Metal2 0.28 Metal3 0.28 Metal4 0.28 Metal5 0
.28 Metal6 0.46} -net { analogN2I } hasUnknownConstraint
} {-type shield -rule NDR0.6__analogNE  -shieldNet { AVSS } -shieldType coaxial
-connectSupply anyPoint -shieldWidth { Metal1 0.23 Metal2 0.28 Metal3 0.28 Metal
4 0.28 Metal5 0.28 Metal6 0.44} -tandemWidth { Metal1 1.4 Metal2 1.4 Metal3 1.4
Metal4 1.4 Metal5 1.4 Metal6 1.4} -shieldGap { Metal1 0.23 Metal2 0.28 Metal3 0.
28 Metal4 0.28 Metal5 0.28 Metal6 0.46} -net { analogN1I } hasUnknownConstraint
}
```

- The following command returns the path to the additional constraint file, which contains the maximum resistance target values for nets:

  ```
  getIntegRouteConstraint -additional
  ```

  A sample output of the command is displayed below:

  ```
  {-type additional -file 1.txt
  }
  ```

Related Information

- `deleteIntegRouteConstraint`
- `setIntegRouteConstraint`
- `writeIntegRouteConstraint`
- Routing Constraint Interoperability chapter in the *Mixed Signal Interoperability Guide*
- Tools Menu chapter in the *Menu Reference*
    - Mixed Signal -> Integration Constraint Editor

pull_block_constraint

```
pull_block_constraint
[-help]
[-file report_file_name]
[-override]
{-all_blocks | -block list_of_blocks | -inst list_of_instances | -cellview
{list_of_libName_cellName_viewName}}
```

Pulls the routing constraints stored on the interface nets of blocks in a design, to their corresponding top-level nets. The `pull_block_constraint` command uses a two-pass approach to handle conflicting constraints on the same net. In this approach, the `pull_block_constraint` command detects block-level and top-level conflicts and reports a warning.

If the `-override` parameter is not specified (non-override mode) and a block-level conflict is detected, no constraints are pulled.

If the `-override` parameter is specified (override mode), `pull_block_constraint` reports the conflicts but pulls the constraints such that the lower-level net constraints override any constraints that might exist on the top-level nets..

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `pull_block_constraint` parameter. For a detailed description of the command and all of its parameters, use the man command `man pull_block_constraint`
`-cellview {`*`list_of_libName_cellName_viewName`*`}`	
	Specifies the list of library, cell, and view names from which the constraints need to be pulled.
`-file` *`report_file_name`*	Specifies the report file name. Given that the standard out report is in the abbreviated format, more details are written out in this mode indicating: • The lower, and upper level net names, the type of constraints pulled. • All the associated parameters. • Any conflicts that need to be resolved. • Which constraint were written to a particular net. *Default*: `dumpPullInfo.log`
`-override`	When specified, the lower-level net constraints will override any constraints that might exist on the top-level nets. *Default*: `off`
`-all_blocks`	Specifies that constraints with hierarchical scope will be pulled from all blocks in the design.
`-block` *`list_of_blocks`*	Defines the list of block names of the instances in the design for which the constraints with hierarchical scope will be pulled.
`-inst` *`list_of_instances`*	Defines the instances in the design for which the constraints with hierarchical scope will be pulled.

Examples

- Using the following command, the constraints on the pins in the cellview binded to the block macro of all the instantiations of the block list `pll_clock` and `pwr_monitor` will be pulled up to the top-level nets connected to the instance pins.

  ```
  pull_block_constraint -block {pll_clock pwr_monitor}
  ```

- Using the following command, the constraints on the pins in the cellview binded to the macro instance `I1`/`I2` will be pulled up to the nets connected to the pins of the instance.

  ```
  pull_block_constraint -inst {I1/I2}
  ```

Related Information

- `deleteIntegRouteConstraint`
- `writeIntegRouteConstraint`
- Routing Constraint Interoperability chapter in the *Mixed Signal Interoperability Guide*
- Tools Menu chapter in the *Menu Reference*
  - Mixed Signal -> Pull Block Constraint

run_vsr

```
run_vsr
[-help]
[-log_file fileName]
[-no_taper_to_pinwidth]
[-post_load_script scriptname]
[-pre_load_script scriptname]
[-remove_existing_route]
[-route_type_order {bus diffPair match nets shield}]
[-run_VSR_Script scriptname]
[-share_shields]
[-sel_net list_of_selected_nets | -incl_net_type {bus diffPair match nets shield} | -nets_file fileName]
```

Launches the Virtuoso Space-based Router (VSR) from Innovus™ Implementation System. It works on OpenAccess design as well as restored non-OpenAccess designs. It routes the nets specified with the `-sel_net`, `-nets_file`, or `-incl_net_type` option. If the `-incl_net_type` option is used, the nets are routed in the following order: `diffPair`, `match`, `bus`, `nets`, and `shield`. To invoke this wrapper, make sure Virtuoso is available in your path.

Before running the `run_vsr` command, set the following:

```
setOaxMode -compressLevel 1
```

Notes

- Integration constraints are used to convey the design intent in a mixed-signal design. These constraints are fully interoperable between the digital and custom environments.
- If you run `run_vsr` on an abstract inferred from layout, you may see the following WARNING message:

  ```
  "The current database uses quick abstract inference and should be used only for floorplanning and not for routing purposes."
  ```

For more information on using `run_vsr`, see the Routing Constraint Interoperability chapter in the *Mixed Signal Interoperability Guide*.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `run_vsr` parameter. For a detailed description of the command and all of its parameters, use the man command `man run_vsr`
`-incl_net_type {bus diffPair match nets shield}`	
	Specifies the list of valid net types for VSR to route. **Note:** The `-sel_net`, `-nets_file`, and `-incl_net_type` parameters are mutually exclusive. Do not specify these parameters at the same time.
`-log_file` *`fileName`*	Specifies the file name for writing the routing report.
`-nets_file` *`fileName`*	Specifies the name of the file listing the nets to be routed. **Note:** The `-sel_net`, `-nets_file`, and `-incl_net_type` parameters are mutually exclusive. Do not specify these parameters at the same time.
`-no_taper_to_pinwidth`	Does not perform tapering based on the pin width. For instance, if the rule applied on a mixed signal net is NDR1 whose width exceeds the pin width for a particular layer, then VSR automatically tapers the wire to match the pin width. If this option is specified, then tapering is not done.
`-post_load_script` *`scriptname`*	Specifies the name of the script to be executed by VSR after loading the design.
`-pre_load_script` *`scriptname`*	Specifies the name of the script to be executed by VSR before loading the design.
`-remove_existing_route`	Deletes the nets (fixed, normal), including shield, and excluding power nets, which are specified by the -sel_net parameter before routing
`-route_type_order {bus diffPair match nets shield}`	
	Specifies the list of valid constraint types in the preferred routing order.

`-run_VSR_Script` *scriptname*	Specifies a user-defined script to be run by VSR for custom routing. If this option is specified, the router skips all presetting from VSR and routes the design by sourcing the complete custom script as provided.
`-share_shields`	Shares the shielded nets. Using this parameter, you can selectively specify the list of shared shielded nets.
`-sel_net` *list_of_selected_nets*	Specifies the list of nets that have to be routed.

Using Scripts and Environmental Variables with run_vsr

The `run_vsr` command in Innovus supports a limited set of options to route the analog/mixed-signal nets. To have more control of the VSR engine, you can set additional settings through the `.cdsenv` file.

Settings through .cdsenv

Here are two examples of how you can use `.cdsenv` settings to control VSR:

- To force VSR to route on-grid:
  ```
  layout routeUseGrid cyclic "Routing"
  ```

- To force VSR to use the rules from the specified route spec:
  ```
  layout wireConstraintGroup string "LEFDefaultRouteSpec"
  ```

If you set the above settings in the `.cdsenv` file before running the `run_vsr` command, Virtuoso initializes them before calling the VSR engine.

In addition to the settings through `.cdsenv` file, the `run_vsr` command is enhanced to pass additional variables or scripts through `-pre_load_script` and `-post_load_script`.

Preloadscript

Usage of settings through `-pre_load_script` is only for expert VSR users and generally not recommended.

Postloadscript

Example:

- To enable VSR to use the custom vias and not generated or derived vias:
  ```
  setvar db.enable_create_derived_vias false
  ```

- To enable VSR to connect the wires to the center of the pin

  ```
  setvar droute.snap_to_pin_center true
  ```

You can set the above environmental variables in a script and enable them through the following command:

```
run_vsr -sel_net { neta …..} -post_load_script scriptname…
```

For more information on the environmental variables which can be set, refer to the following documentation:

- "Environment Variables" chapter in the *Cadence Space-based Router and Chip Optimizer Command Reference*

Note: Consult Cadence for the best use of these variables to obtain the desired results.

Examples

- The following command routes the bus nets using VSR:

  ```
  run_vsr -sel_net {DATA[0] DATA[1] DATA[2] DATA[3] DATA[4] DATA[5] DATA[6] DATA[7]}
  ```

- The following command routes the given net `lbk_bstart` using mixed-signal constraint, if any constraint has been applied to this net which will be taken from OpenAccess database:

  ```
  run_vsr -sel_net lbk_bstart
  ```

- The following command routes all the nets which have `diffPair`, `matchPair` and then `net` class in the following order of constraint types:

  ```
  run_vsr -incl_net_type {diffPair match nets}
  ```

Related Information

- Routing Constraint Interoperability chapter in the *Mixed Signal Interoperability Guide*
- Tools Menu chapter in the *Menu Reference*
  - Mixed Signal -> Run VSR

setIntegRouteConstraint

```
setIntegRouteConstraint
[-help]
[-accessStyle {none | straight | clustered}]
[-bottomLayer layerNum]
[-busGroupWidth integer]
[-connectSupply {float anyPoint}]
[-cornerStyle {none | crossover | river}]
[-crossTieShieldInterval string]
[-crossTieShieldLayerLimit string]
[-crossTieShieldedLayers string]
[-groupToOutsideSpacing {layer1 spacing1 layer2[:layerN] spacing2...}]
[-hard]
[-hierarchicalScope {local local_below local_above local_above_below}]
[-intergroupGap {layer1 spacing1 layer2[:layerN] spacing2...}]
[-layerGap {layer1 gap1 layer2[:layerN] gap2...}]
[-layerMatch]
[-matchGap {layer1 gap1 layer2[:layerN] gap2...}]
[-matchStyle {accordion | trombone | coil}]
[-maxLength value]
[-maxRes value]
[-minBusBit integer]
[-minLength value]
[-minRes value]
[-name string]
{-net {net1 net2 ...} | -busSinkGroups busSinkGroupNameList  | -file string}
[-noshareshield]
[-rule NDR_name]
[-shieldGap {layer1 gap1 layer2[:layerN] gap2 ...}]
[-shieldNet {net1 net2}]
[-shieldTolerance {net1 value1 net2 value2 ...}]
[-shieldType {unspecified side bottom bottomSide top topSide topBottom coaxial
tandemSplit interleaved crosstie  tandemSplitFull tandemSplitNearBy
tandemSplitSpecial none}]
[-shieldWidth {layer1 width1 layer2[:layerN] width2...}]
[-tandemAggressorAdjacentLayerSeparation layer separation]
[-tandemAggressorSameLayerTrackDistance {layer1 n1 layer2[:layerN] n2...}]
[-tandemCapWidth {layer1 width1 layer2[:layerN] width2...}]
[-tandemNearbyNonAggressorExceptByName {nameString}]
[-tandemNearbyNonAggressorNets {nameString}]
[-tandemNearbyNonAggressorType {typeNameString}]
[-tandemOverhang {layer1 width1 layer2[:layerN] width2...}]
```

```
[-tandemSideOverhang {layer1 width1 layer2[:layerN] width2...}]
[-tandemWidth {layer1 width1 layer2[:layerN] width2...}]
[-tolerance tolerance]
[-topLayer layerNum]
-type {bus diffPair matchLength NDR netClass shield simpleNet additional}
```

Applies specialty routing constraints in the database for nets in the designs. It allows creation of differential pair, match pair, shield nets, and nets with non-default rules.

If required, all routing constraints set by this command can be deleted from the database using the `deleteIntegRouteConstraint` command.

Also see, `getIntegRouteConstraint`.

Notes

- Integration constraints are used to convey the design intent in a mixed signal design. These constraints are fully interoperable between the digital and custom environments.
- If the design has a complex netClass constraint that has net members with different constraints, `setIntegRouteConstraint` will throw the an error. If you still wish to change the integration constraints in the design, check whether the netClass constraints under the *Complex NetClass* tab in the Integration Constraint Editor can be removed. If yes, use the `deleteIntegRouteConstraint` or the GUI to first delete such constraints, and then repeat the `setIntegRouteConstraint` command.
- The rule name `DefaultRule` is obsolete. It still works in this release and is automatically converted to the correct name `default`. Update your scripts to use `default` so that it work correctly in future releases.

Parameters

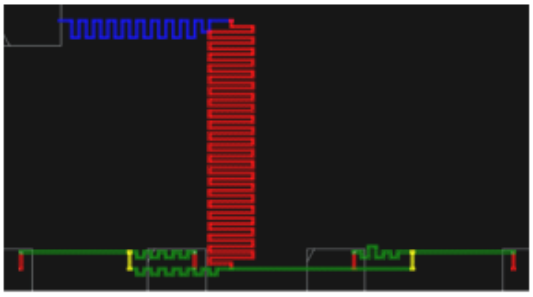
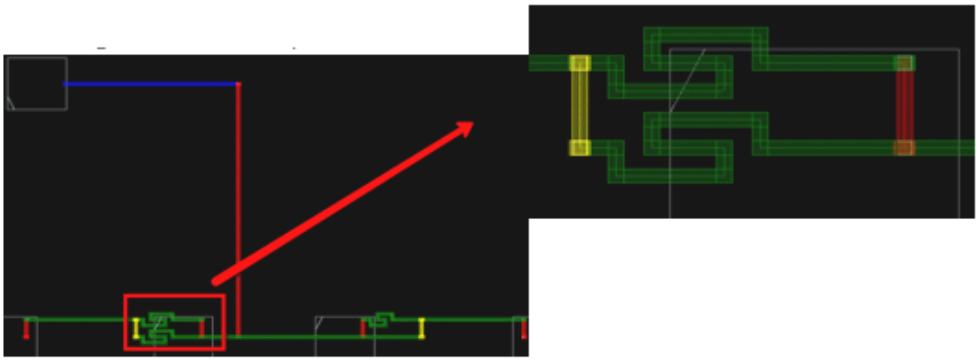
`-help`	Outputs a brief description that includes the type and default information for each `setIntegRouteConstraint` parameter. For a detailed description of the command and all of its parameters, use the man command `man setIntegRouteConstraint`.
`-accessStyle {none | straight | clustered}`	
	Specifies the preferred style for routing the bus after connecting to the bus pins. This parameter is applicable only for `-type bus`. *Default:* `clustered`

`-bottomLayer` *layerNum*	Specifies the bottom routing layer that is to be used.
`-busGroupWidth` *integer*	
	Specifies the width for the bus groups into which the router splits a bus. The width is specified as an integer value. For example, if a bus is 16 bits, you can specify `-busGroupWidth 4` to indicate that the router can route the bus as four bus groups of 4 bits each. *Default*: `1`
`-busSinkGroups` *busSinkGroupNameList*	
	Specifies the list of names of the busSinkGroups. It sets high-frequency/mixed signal constraints for the nets of the busSinkGroup sinks. It supports all constraints of type BUS. These constraints are honored by the NanoRoute high-frequency router.
`-connectSupply {`*float* `anyPoint}`	Indicates how the shield wires will be connected to the supply line. The default option, `anyPoint`, tells the router to connect the wire to supply, anywhere along the length of the shield wire. *Default:* `anyPoint`
`-cornerStyle {none | crossover | river}`	
	Specifies the type of corner style that the differential pair route or bus should follow when changing directions. This parameter is applicable only for `-type bus` and `-type diffPair`. *Default:* `crossover`
`-crossTieShieldInterval` *string*	

	Specifies the number of tracks between the cross-tie wires for opportunistic coaxial shielding. The *string* value should have the following format: `{integer | {layer interval1 layer2[:layerN] interval2 ...}}` The integer value specifies the interval in the form of the number of tracks between cross-ties created on a given layer. For example, a value of `0` means that there are no tracks between cross-ties, which means there is a cross-tie on every track. Similarly, a value of 1 means that there is one track between cross-tie wires, which results in a cross tie on every other track. A value of `2` means two tracks between the cross-tie wires and so on. **Note**: This parameter is applicable only if `-shieldType` is specified as `crosstie`. *Default:* `0`
`-crossTieShieldLayerLimit` *string*	
	Specifies the layer range that can be used to create cross-tie shields. Use the `-crossTieShieldLayerLimit` parameter if you want to use only specific layers to create cross-ties and keep the remaining layers free for other purposes. The *string* value should have the following format: `{layer_number:layer_number}` For example, if the layer range specified is `{METAL2:METAL7}`, cross-tie shields can be created on `METAL2`, `METAL3`, `METAL4`, `METAL5`, `METAL6`, and `METAL7` but not on `METAL8` or the layers above it. **Note**: This parameter is applicable only if `-shieldType` is specified as `crosstie`.
`-crossTieShieldedLayers` *string*	

	Specifies the layer range that will be shield with cross-ties from above and below. Use the `-crossTieShieldedLayers` parameter if you want to minimize congestion by shielding only specific routing layers with cross-ties. The *`string`* value should have the following format: `{layer:layer}` For example, if the layer range specified is `{METAL3:METAL6}`, cross-tie shields will be built for routes on the net on `METAL3`, `METAL4`, `METAL5`, and `METAL6`. **Note**: This parameter is applicable only if `-shieldType` is specified as `crosstie`.
`-file` *`string`*	Specifies the path for an additional constraint file, which contains the maximum resistance target values for nets. ⚠ • `-file` cannot be used with either `-net` or `-busSinkGroups`. • `-file` must be used only with `-type additional`.
`-groupToOutsideSpacing` *`layer1 spacing1 layer2`*`[:`*`layerN`*`]` *`spacing2`*`...`	
	Indicates the spacing per layer (in microns) between the nets in this net group, and the other nets in the design. This parameter is applicable only for bus nets. That is, `-groupToOutsideSpacing` should be specified only for routing constraints with `-type bus`. ⚠ While reading in an OpenAccess database for which constraints have been created elsewhere, Innovus checks for any non-bus net that has constraints with `-groupToOutsideSpacing` specified. Innovus treats such constraints as complex constraints and sets the `isMixedSignal` flag on the corresponding nets.

`-hard`	Indicates that the top or bottom layer routing constraint created should be of hard type. By default, Innovus places the top/bottom routing layer constraints as soft routing constraints in the resulting OpenAccess database.
`-hierarchicalScope {local local_above local_below local_above_below}`	
	Specifies whether the particular constraint is local, and/or can be pulled (above) or pushed (below) across the hierarchy. Setting this option to all possible values enables pull and push. If set to local, it is not saved to the database.
`-intergroupGap {`*`layer1 spacing1 layer2`*`[:`*`layerN`*`] `*`spacing2`*`...}`	
	Specifies the gap to be kept between the split bus groups when `-busGroupWidth` is specified. Default is min spacing.
`-layerGap {`*`layer1 gap1 layer2`*`[:`*`layerN`*`] `*`gap2`*`...}`	
	Indicates the layer names and the gap-spacing values for the nets routed as diffPair. For any layer not listed, the minimum legal spacing for that layer is used. For example: `-layerGap {`*`layer_1 gap_1 layer_2 gap_2`*`...}`
`-layerMatch`	Specifies that each separate layer should be matched for `matchLength` nets. By default, only the total length is matched
`-matchGap {`*`layer1 gap1 layer2`*`[:`*`layerN`*`] `*`gap2`*`...}`	
	Indicates the spacing between two rises of the accordion. This setting is applicable only when `-type` is `matchLength` and `-matchStyle` is `accordion`. You may want to control the spacing between two rises of the accordion if routing resources are restricted.
`-matchStyle {accordion | trombone | coil}`	

	Indicates the routing topology to be used for matching the nets routed as `matchLength`. It can be `accordion`, `trombone`, or `coil`. In `accordion`, pulse structures are used to match length while in `trombone` we use loops. **Matching in accordion style**  **Matching in trombone style**  *Default:* `accordion`
`-maxLength` *value*	Indicates the maximum length the router should route a net. The maximum length is specified as a float value. This parameter is applicable only for `-type matchLength` and `-type simpleNet`.
`-maxRes` *value*	Indicates the maximum resistance to be used for the specific net being routed or for the nets with a `matchLength` constraint if `-type` is `matchLength`. The maximum resistance is specified as a float value. This parameter is applicable only for `-type matchLength`.

`-minBusBit` *integer*	Indicates the minimum bits allowed for the router to split a bus. As channels get narrower, it becomes challenging to accommodate all the bus bits in the same channel. By default, `-minBusBit` value is undefined, not `0`. If you specify an integer value for `-minBusBit`, the router splits a bundle with a minimum of this value. For example, if you set -`minBusBit` to `4`, the router splits the bus such that each part has at least `4` bits.
`-minLength` *value*	Indicates the the minimum length the router should route a net. The minimum length is specified as a float value. This parameter is applicable only for `-type matchLength` and `-type simpleNet`.
`-minRes` *value*	Indicates the minimum resistance to be used for the specific net being routed or for the nets with a `matchLength` constraint if `-type` is `matchLength`. The minimum resistance is specified as a float value. This parameter is applicable only for `-type matchLength`.
`-name` *string*	Specifies the name of the constraint group being created, changing the default name given to the constraint by Innovus.
`-net {`*net1 net2 ...*`}`	
	Specifies the list of net names to be routed. For constraints of type `bus` and `matchLength`, a single net is also supported.
`-noshareshield`	Specifies that there should be individual shields, instead of a common shield, between the bus nets. This option is applicable only for constraints of type `bus`.
`-rule` *NDR_name*	Specifies the name of the routing rule that is used for routing a particular type of net(s). *Default:* `"default"`
`-shieldGap {`*layer1 gap1 layer2*`[:`*layerN*`] gap2 ...}`	
	Specifies the spacing between the shielded wire and the shield net in microns. For any layer not listed, the minimum legal spacing for that layer is used.

`-shieldNet {net1 net2}`	Indicates that the net group should be shielded by the specified nets. Valid net names that you can use with `-shieldNet` are all power and ground net names in the design. You can specify different shield nets for different sides of a wire. The routers then determine which shield net should be used on which side of the wire. **Note:** You can specify a maximum of two net names with this parameter. In addition, you can specify two shield net names only for the following shield types: • `side` • `topSide` • `bottomSide` • `coaxial` • `tandemSplit` For the `top`, `bottom`, or `topBottom` shield types, you can specify only one shield net. *Default*: The first ground net name in the list of ground nets.
`-shieldTolerance {net1 value1 net2 value2 ...}`	

	Indicates the percentage of length of the net that can remain unshielded. You can set the same tolerance value or different tolerance values on a group of nets.
	The examples below depict different use models:
	• In the following example, the same constraint shield tolerance is put on every member of the shield constraint group:
	`setIntegRouteConstraint -type shield -shieldNet {VSS} -shieldType side -shieldTolerance 25.0 -connectSupply anyPoint -net {scan_in_2, abc, xyz}`
	• In the example below, different groups of nets get different shieldTolerance values:
	`setIntegRouteConstraint -type shield -shieldNet {VSS} -shieldType side -shieldTolerance 25.0 -connectSupply anyPoint -net {scan_in_2, abc, xyz} -shieldTolerance {net1 30 net2 30 net3 30}`

- In the following example, the nets `scan_in_2`, `abc` and `xyz` are already nets with the shield constraint, but without tolerance values. The command below adds the tolerance value to the existing shielding constraint of the nets. Note that adding shield tolerance changes the net to a complex shield as opposed to a simple shield:

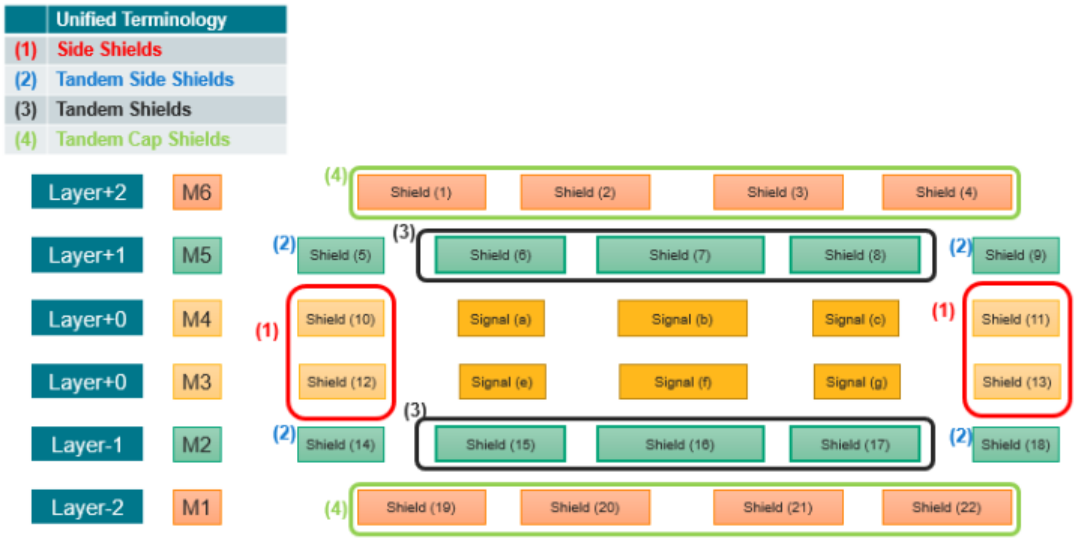
```
setIntegRouteConstraint -type shield -shieldTolerance 25 -net
{scan_in_2 abc xyz}
```

Note:

- The shield tolerance value can be set only on nets that are not intended to be routed by NanoRoute. This means that setting the shield tolerance on any net will make that net a complex shield.
- If you use the `setIntegRouteConstraint` command to set the shield tolerance on a net or a group of nets, those nets automatically get the MS bit in the DB and show up in the GUI as a complex shield.
- Percentage tolerance is considered by VSR as a soft constraint.
- PVS-cv is the only tool in the flow that checks the percentage tolerance against the geometries created by VSR, and flags any such violations.

Default: `20.0`

`-shieldType {unspecified side bottom bottomSide top topSide topBottom coaxial tandemSplit interleaved crosstie  tandemSplitFull tandemSplitNearBy tandemSplitSpecial none}`

	Specifies the shielding type for the net(s) being routed. For complex coaxial shields, shield type can be specified as explained below: • `tandemSplitNearBy` - Specifies that shielding should be added only where required. • `tandemSplitFull` - Specifies that one-layer tandem shielding with overhang should be added. The overhang can be specified using the new `-tandemOverhang` and `-tandemSideOverhang` parameters. • `tandemSplitSpecial` - Specifies that two-layer tandem shielding should be added for complete shielding from other nets. Use the new `-tandemCapWidth` parameter to specify the width of the tandem cap shield (second layer shield).  *Default:* `unspecified` for `diffPair`, `matchLength`, and `bus`; `side` for `NDR`, `simpleNet`, and `netClass`
`-shieldWidth {`*`layer1 width1 layer2`*`[:`*`layerN`*`] `*`width2...`*`}`	
	Indicates the width of the shield wire in microns per layer. This only applies to side shields.
`-tandemAggressorAdjacentLayerSeparation` *`layer separation`*	
	Specifies the minimum distance between the given signal layer and the above/below layers containing the aggressors.

`-tandemAggressorSameLayerTrackDistance {`*`layer1 n1 layer2`*`[:`*`layerN`*`] `*`n2`*`...}`	
	Specifies the minimum pitch tracks (distance) between the given signal net and any aggressors on the same layer.
`-tandemCapWidth {`*`layer1 width1 layer2`*`[:`*`layerN`*`] `*`width2`*`...}`	
	Specifies the wire width (in microns) of the tandem cap shield beyond the edge of the route being shielded. For any layer not listed, the minimum legal width for that layer is used. This parameter is needed only for the `tandemSplitSpecial` shield type.
`-tandemNearbyNonAggressorExceptByName {`*`nameString`*`}`	
	Specifies that only the nets mentioned by name are aggressors; the rest are not considered as aggressors for shielding.
`-tandemNearbyNonAggressorNets {`*`nameString`*`}`	
	Specifies the net names that are not considered as aggressors for shielding.
`-tandemNearbyNonAggressorType {`*`typeNameString`*`}`	
	Specifies those signal types that are not considered as signal types of aggressors. Even if they exist within a certain range, the signal net does not need to be protected by shielding.
`-tandemOverhang {`*`layer1 width1 layer2`*`[:`*`layerN`*`] `*`width2`*`...}`	
	Specifies the overhang value (in microns) for the tandem shield.
`-tandemSideOverhang {`*`layer1 width1 layer2`*`[:`*`layerN`*`] `*`width2`*`...}`	
	Specifies the overhang value (in microns) for the tandem side shield.
`-tandemWidth {`*`layer1 width1 layer2`*`[:`*`layerN`*`] `*`width2`*`...}`	
	Specifies the width (in microns) of the top and bottom shields, beyond the edge of the route being shielded. For any layer not listed, the minimum legal width for that layer is used.
`-tolerance` *`tolerance`*	Indicates the percentage difference in length that is allowed when routing nets as `matchLength` and `diffPair`. *Default:* 20%

`-topLayer layerNum`	Specifies the top routing layer that is to be used for routing.
`-type {bus diffPair matchLength NDR netClass shield simpleNet additional}`	
	Specifies the type of routing constraints to be created: • `bus` - Specifies constraints for bus nets. • `diffPair` - Specifies constraints for differential pair nets. • `matchLength` - Specifies constraints for matched nets. • `NDR` - Specifies constraints for a net with a non-default rule. • `netClass` - Specifies constraints for shielding a group of nets. It can also be used for a group of nets that have the same NDR. • `shield` - Specifies constraints for shielding a single net. • `simpleNet` - Specifies constraints for a simple net. • `additional` - Specifies the maximum resistance target constraints for nets. The NanoRoute High Frequency (NRHF) router routes to match the targets by wire widening. A prerequisite for the maximum resistance flow is that NDRs should be predefined for the used layers. ⚠ `-type additional` can be used only if `-file` is specified.

Examples

- The following command sets the layer 1 gap as 0.4, layer 2 gap as 0.39, and layer 3 gap as 0.38:

  ```
  setIntegRouteConstraint -type diffpair -net {n_7862 n_7875} -layerGap {1 0.4 2 0.39 3 0.38}
  ```

- The following command specifies a shield constraint for the `analogS41` net:

  ```
  setIntegRouteConstraint -type shield -shieldNet VSS -shieldType side -connectSupply anyPoint -rule NDR4 -net analogS4I
  ```

- The following command specifies a shield constraint for the net group comprising nets `analogW1I`, `analogW2I`, `analogW3I`, and `analogW4I`:

  ```
  setIntegRouteConstraint -type netClass -shieldNet VSS -shieldType side -connectSupply anyPoint -net {analogW1I analogW2I analogW3I analogW4I}
  ```

- The following command specifies minimum and maximum length constraints for the `xyz` net:

  ```
  setIntegRouteConstraint -type simpleNet -net xyz -minLength 2.0 -maxLength 3.0
  ```

- The following command specifies that individual shields should be used between the nets in the bus constraint:

  ```
  setIntegRouteConstraint -type bus -nets {nets_1 nets_2 ...} -shieldNet VSS -noshareshield ...
  ```

- The following command specifies maximum resistance constraints for nets as specified in the target file, `1.txt`:

  ```
  setIntegRouteConstraint -type additional -file 1.txt
  ```

Related Information

- `deleteIntegRouteConstraint`
- `getIntegRouteConstraint`
- `writeIntegRouteConstraint`
- Routing Constraint Interoperability chapter in the *Mixed Signal Interoperability Guide*
- Tools Menu chapter in the *Menu Reference*
  - Mixed Signal -> Integration Constraint Editor

writeIntegRouteConstraint

```
writeIntegRouteConstraint
[-help]
-file output_file
[-net net_name | -name string  | -additional]
```

Writes the constraints to an output file in the following order: `diffPair`, `matchLength`, `bus`, `netClass`, `NDR`, `shield`, and `additional`.

Notes

- Integration constraints are used to convey the design intent in a mixed-signal design. These constraints are fully interoperable between the digital and custom environments.
- If the design contains one or more complex netClass constraints with net members that have different constraints, `writeIntegRouteConstraint` reports the following warning:

  ```
  **WARN: (IMPOAX-2124): This design contains unknown complex constraint on
  complexNetClass 'contraint_name'. This is not an interoperable routing constraint
  for Innovus. When sourcing the resulting file from this command in Innovus, these
  complex constraints will be lost. You can see the complex constraints in the design
  in the integration constraint form.
  ```

Parameters

`-help`	Prints a brief description that includes the type and default information for each `writeIntegRouteConstraint` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man writeIntegRouteConstraint`
`-additional`	Writes the path to the additional constraint file, which contains the maximum resistance target values for nets.
`-file output_file`	Specifies the output file name that is to be written out.
`-name` *`string`*	Writes information for the specified constraint by name. Use the `getIntegRouteConstraint` command to get the name of any constraint in the design. You can specify multiple constraint names, enclosed in braces.
`-net` *`net_name`*	Writes the constraint defined on specified net. If the net is part of a netClass, writes constraints for the netClass. You can specify multiple net names, enclosed in braces.

Examples

- The following command writes the constraints in the output file called `myfile.rpt`:
  ```
  writeIntegRouteConstraint -file myfile.rpt
  ```
  The file `myfile.rpt` could have the following sample constraints that are written out:
  - For a `diffPair` constraint
    ```
    setIntegRouteConstraint -type diffPair -name
    Constr_3__analogNL__DTMF_INST/analogNL_INST -tolerance 20 \

    -rule default \
    -net { \
    DTMF_INST/digsyncc \
    DTMF_INST/digsyncd \
    }
    ```
  - For a `matchLength` constraint
    ```
    setIntegRouteConstraint -type matchLength -name
    Constr_4__analogNL__DTMF_INST/analogNL_INST -tolerance 20
    -matchStyle accordion \
    ```

```
-rule default \
-net { \
DTMF_INST/digsync3 \
DTMF_INST/digsync4 \
}
```

- For a `netClass` constraint

```
setIntegRouteConstraint -type netClass -name
Constr_11__analogSW__DTMF_INST/analogSW_INST \
-shieldNet VSS -shieldType side -connectSupply anyPoint \
-rule NDR8 \
-net { \
analogW2I \
}
```

- For a `shield` constraint

```
setIntegRouteConstraint -type shield -shieldNet {VSS} -shieldType side \
-shieldTolerance 30 -net {tdigit[0] tdigit[1] tdigit[2] tdigit[3] tdigit[4]
tdigit[5] \
tdigit[6]tdigit[7]}
```

- For an `additional` constraint

```
setIntegRouteConstraint -type additional -file 1.txt
```

- The following command writes a constraint file called `test.const` for the constraint named `Constr_2__analogNW__DTMF_INST/analogNW_INST`:

```
writeIntegRouteConstraint -file test.const -name
Constr_2__analogNW__DTMF_INST/analogNW_INST
```

The output is as follows:

```
setIntegRouteConstraint -type diffPair -name
Constr_2__analogNW__DTMF_INST/analogNW_INST -tolerance 20 \
-rule default \
-net { \
DTMF_INST/cntrlxo[3] \
DTMF_INST/cntrlxo[2] \
}
```

- The following command writes a constraint file with name `temp.const` for the constraint defined on net `analogW1I`:

```
writeIntegRouteConstraint -file temp.const -net analogW1I
```

The output is as follows:

```
setIntegRouteConstraint -type netClass -name
Constr_10__analogSW__DTMF_INST/analogSW_INST \
-shieldNet VSS -shieldType side -connectSupply anyPoint \
-rule NDR8 \
-net { \
analogW1I \
}
```

- The following command writes a constraint file named `test1.const` for the specified constraint names:

  ```
  writeIntegRouteConstraint -name {Constr_2__analogNW__DTMF_INST/analogNW_INST
  Constr_1__analogNW__DTMF_INST/analogNW_INST} -file test1.const
  ```

  The output is as follows:

  ```
  #Integration constraints ascii out for { ADTMF4 dtmf_chip NoConst_10_18 }

  setIntegRouteConstraint -type diffPair -name
  Constr_2__analogNW__DTMF_INST/analogNW_INST -tolerance 20 \
  -rule default \
  -net { \
  DTMF_INST/cntrlxo[3] \
  DTMF_INST/cntrlxo[2] \
  }
  setIntegRouteConstraint -type diffPair -name
  Constr_1__analogNW__DTMF_INST/analogNW_INST -tolerance 5 \
  -rule default \
  -net { \
  DTMF_INST/cntrlxo[0] \
  DTMF_INST/cntrlxo[1] \
  }
  ```

- The following command writes a constraint file named `test2.const` for the specified net names:

  ```
  writeIntegRouteConstraint -net {DTMF_INST/cntrlxo[3] DTMF_INST/cntrlxo[0]} -file
  test2.const
  ```

  The output is as follows:

  ```
  #Integration constraints ascii out for { ADTMF4 dtmf_chip NoConst_10_18 }

  setIntegRouteConstraint -type diffPair -name
  Constr_2__analogNW__DTMF_INST/analogNW_INST -tolerance 20 \
  -rule default \
  -net { \
  DTMF_INST/cntrlxo[3] \
  DTMF_INST/cntrlxo[2] \
  }
  setIntegRouteConstraint -type diffPair -name
  ```

```
Constr_1__analogNW__DTMF_INST/analogNW_INST -tolerance 5 \
-rule default \
-net { \
DTMF_INST/cntrlxo[0] \
DTMF_INST/cntrlxo[1] \
}
```

Related Information

- deleteIntegRouteConstraint
- getIntegRouteConstraint
- setIntegRouteConstraint
- Routing Constraint Interoperability chapter in the *Mixed Signal Interoperability Guide*
- Tools Menu chapter in the *Menu Reference*
  - Mixed Signal -> Integration Constraint Editor

21

Multiple-CPU Processing Commands

- checkMultiCpuUsage
- getDistributeHost
- getMultiCpuUsage
- monitor_hosts
- report_resource
- setDistributeHost
- setMultiCpuUsage

checkMultiCpuUsage

```
checkMultiCpuUsage
[-help]
```

Checks if distributed processing can work in the software environment. This command also ensures that the working directory is accessible from the master machine as well as the client machines. The `checkMultiCpuUsage` command is specified after configuring the multi-CPU settings (`setDistributeHost` and `setMultiCpuUsage`) to check if all the specified CPUs can be accessed. This command should be invoked before running any commands that use multiple CPUs.

Parameters

`-help`	Outputs the command usage and a brief description about the command.

Examples

- Here are some examples.

- The following example command script checks the distributed computing environment when `set_distributed_hosts -local` is specified:

```
checkMultiCpuUsage

/it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413
/it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413
/it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413
/it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413
Connected to sjfnl624 35779 0 ( PID=13350 )
Connected to sjfnl624 54715 1 ( PID=13356 )
Connected to sjfnl624 40469 3 ( PID=13397 )
Connected to sjfnl624 50442 2 ( PID=13378 )
Task 0 successfully accessed the current working directory.
Task 3 successfully accessed the current working directory.
Task 1 successfully accessed the current working directory.
Task 2 successfully accessed the current working directory.
Task 3: CPU(workload/CPU): 7.84/12 mem(free/total): 94118680k/98813572k.
Task 1: CPU(workload/CPU): 7.84/12 mem(free/total): 94118432k/98813572k.
Task 2: CPU(workload/CPU): 7.84/12 mem(free/total): 94117068k/98813572k.
Task 0: CPU(workload/CPU): 7.84/12 mem(free/total): 94119084k/98813572k.
Test heartbeat for task 0...
The distributed processing environment has been set up correctly.
```

- The following example command script checks the distributed computing environment when when `set_distributed_hosts -lsf -queue lnx64` is specified:

```
checkMultiCpuUsage

Job <5685326> is submitted to queue <lnx64>. The associated lsf task ID is
'0'
Executing
/it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413/.clre_sub_22
662_0
Job <5685328> is submitted to queue <lnx64>. The associated lsf task ID is
'1'
Executing
/it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413/.clre_sub_22
662_1
Job <5685329> is submitted to queue <lnx64>. The associated lsf task ID is
```

```
'2'
Executing
/it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413/.clre_sub_226
62_2
Job <5685330> is submitted to queue <lnx64>. The associated lsf task ID is
'3'
Executing
/it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413/.clre_sub_22
662_3
Connected to sjfnl625 43039 1 ( PID=30833 )
Connected to sjfnl625 34830 2 ( PID=30832 )
Connected to sjfnl625 33887 3 ( PID=30831 )
Connected to sjfib046 60125 0 ( PID=32727 )
Task 1 successfully accessed the current working directory.
Task 3 successfully accessed the current working directory.
Task 2 successfully accessed the current working directory.
Task 0 successfully accessed the current working directory.
Task 3: CPU(workload/CPU): 1.43/12 mem(free/total): 74177208k/82277684k.
Task 2: CPU(workload/CPU): 1.43/12 mem(free/total): 74177208k/82277684k.
Task 1: CPU(workload/CPU): 1.43/12 mem(free/total): 74177208k/82277684k.
Task 0: CPU(workload/CPU): 19.79/20 mem(free/total): 352917068k/396858936k.
Test heartbeat for task 0...
The distributed processing environment has been set up correctly.
```

Related Information

- Accelerating the Design Process By Using Multiple-CPU Processing chapter in the *Innovus User Guide*
  - "Checking the Distributed Computing Environment"
- `getMultiCpuUsage`
- `setMultiCpuUsage`

getDistributeHost

```
getDistributeHost
{-mode | -hosts | -queue | -resource | -custom_script | -env_script | -args | -
single_cpu_lsf_args | -timeout | -shellTimeout | -reportLsfInfo file | -waitForLsfInfo
value}
[-help]
```

Displays settings for the `setDistributeHost` command.

Parameters

`-args`	Displays the additional arguments when the software is running in the Sun Grid Engine (SGE) or Load Sharing Facility (LSF) mode.
`-custom_script`	Displays the name of the custom script when the software is running in `custom` mode.
`-env_script`	Displays the user-defined shell environment script that is run for each of the slave hosts.
`-hosts`	Displays the list of host names when the software is running in `rsh` mode.
`-help`	Outputs the command usage and a brief description about the command parameters.
`-mode`	Displays the distributed processing mode. The possible values are `local`, `sge`, `rsh` , `ssh` , `lsf` and `custom`. *Default*: `local`
`-queue`	Displays the queue name when the software is running in `sge` or `lsf` mode.

`-reportLsfInfo` *file*	Specifies to restore the LSF information to the specified file. This parameter can only be used when `setDistributeHost -reportLsfInfo` is specified in the current session. You must specify the `getDistributeHost -reportLsfInfo` only after the LSF tasks exit to obtain the task CPU time. If you specify this command before the task exits, the command gives a warning message.
`-resource`	Displays the resource string when the software is running in `sge` or `lsf` mode.
`-shellTimeout`	Displays the number of seconds the host machine waits for `rsh/ssh` machines to become available for multiple-CPU processing.
`-single_cpu_lsf_args`	Displays the lsf arguments when the software is running in `lsf` mode for a single CPU application.
`-timeout`	Displays the default timeout for a submitted job that will never start.
`-waitForLsfInfo` *value*	Specifies the timeout period required to wait for task exit and generate the LSF information.

Examples

- The following command displays the current setting for the `-mode` parameter:

  ```
  getDistributeHost -mode
  local
  ```

- The following command saves the LSF information to the `lsf.log` file:

  ```
  getDistributeHost -reportLsfInfo lsf.log
  ```

Related Topics

- Accelerating the Design Process By Using Multiple-CPU Processing chapter in the *Innovus User Guide.*
- "Tools Menu" chapter in the *Innovus Menu Reference*
  - Set Multiple CPU Usage

getMultiCpuUsage

```
getMultiCpuUsage
[-help]
[-autoPageFaultMonitor]
[-cpuPerRemoteHost]
[-keepLicense]
[-localCpu]
[-remoteHost]
[-threadInfo]
[-verbose]
```

Displays the requested number of threads or hosts.

To request threads or hosts, use `setMultiCpuUsage`.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-autoPageFaultMonitor`	Displays the warning rank set for performance issues caused by major page faults
`-cpuPerRemoteHost`	Displays the number of CPUs available on every remote host for Superthreading.
`-keepLicense`	Displays the current setting for the `setMultiCpuUsage -keepLicense` parameter.
`-localCpu`	Displays the number of available local CPUs for multi-threading.
`-remoteHost`	Displays the number of available remote hosts for distributed processing or Superthreading.
`-threadInfo`	Displays the level of informational messages for each thread.
`-verbose`	Displays a message with the current multiple-CPU setting.

Examples

- The following command displays the requested number of threads for multi-threading:

  ```
  getMultiCpuUsage -localCpu
  ```

- The following command displays the requested number of remote hosts for Superthreading or distributed processing:

  ```
  getMultiCpuUsage -remoteHost
  ```

- The following command displays the requested number of CPUs per remote hosts for Superthreading:

  ```
  getMultiCpuUsage -cpuPerRemoteHost
  ```

Related Information

- "Accelerating the Design Process By Using Multiple-CPU Processing" chapter in the *Innovus User Guide.*
  - Running Distributed Processing
- "Options Menu" chapter in the *Innovus Menu Reference*
  - Set Multiple CPU Usage
- `checkMultiCpuUsage`
- `setMultiCpuUsage`

monitor_hosts

```
monitor_hosts
[-help]
[-log string]
[-overwrite]
[-period integer]
```

Monitors hosts and periodically prints the memory or CPU usage, and temporary disk speed of the hosts. This command automatically determines the hosts to be monitored. By default, after every 60 seconds hosts are monitored and the disk information is written to a log file.

This command is helpful to analyze and debug the performance of a session and helps check the machine load, the available memory, and track possible reasons for a slow down or unexpected exit during a session.

The `monitor_hosts` command is supported in STA, DSTA, and DMMMC flows.

The `monitor_hosts` command output is described below:

Status Keys/Tags	Memory (%age of Available Memory on the corresponding host)	CPU Usage	Disk Speed (Mb/sec)
E – Excellent	Memory used less than 75%	CPU Usage less than 85%	Disk speed more than 250
G – Good	Memory used between 75% and 85%	CPU Usage between 85% to 95%	Disk speed between 125 and 250
B – Bad (*)	Memory used between 85% and 99%	CPU Usage between 95% to 105%	Disk speed between 50 and 125

T – Terrible (**)	Memory used above 99%	CPU Usage more than 105%	Disk speed less than 50

The `monitor_hosts` command output will show host/machine specific information and not process specific information. So if multiple processes are running on the same machine, then the command output will show the combined impact of all the processes, on the machine's memory, machine's CPU usage etc.

The "T" or "B" state denotes that a session may be thrashing or may not give the best results with respect to the performance. For better performance, you must re-run this command with better machines.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command and all of its parameters, use the `man` command: `man monitor_hosts`.
`-log` *string*	Specifies the log file name. *Default*: host-monitor.log
`-overwrite`	Overwrites the log file, if it already exists.
`-period` *integer*	Specifies the sampling period after which hosts will be monitored. *Default*: 60

Examples

- A sample monitor log file is shown below:

```
Status keys: E excellent, G good, B bad, T terrible.
Memory values:
total = Real memory of host
progs = Real memory used by processes, including swappable cache
used = Real memory used by processes, excluding swappable cache
Started at: 17/04/25 04:24:22
lgi15(0) TMPDIR is : /tmp/tempus_temp_20836_lgi15_ark_lEAcvw

============ ======== ======================== ========================
Host          CPU      Memory (GB)              TMPDIR (GB)       (Mb/s)
name(id)      util %   total progs used %used   total   avail     rate
```

```
============ ======== ===== ===== ===== ====== ========================
04:24:52
lgi15(0)         6 E     32    24    15    46 E    100.74  94.67 E  262 E
```

- The following example output shows that host “sjfqos011” has very high CPU utilization (123%) because of multiple clients (client 1 and client 3) running on same host. Also, %used more than 99T shows that the memory is ~100% consumed on the “sjfqos011” and “sjfqos010” host machines.

```
Status keys: E excellent, G good, B bad, T terrible.
Memory values:
total = Real memory of host
progs = Real memory used by processes, including swappable cache
used = Real memory used by processes, excluding swappable cache
Started at: 20/07/08 07:10:10
sjfqos009(0) TMPDIR is : /tmp/ssv_tmpdir_25921_eYAlmJ
sjfqos011(1) TMPDIR is : /tmp/ssv_tmpdir_26114_ZuTGDp
sjfqos010(2) TMPDIR is : /tmp/ssv_tmpdir_26124_pjq6Cp
sjfqos011(3) TMPDIR is : /tmp/ssv_tmpdir_26160_vxwbEp
sjfqos012(4) TMPDIR is : /tmp/ssv_tmpdir_26173_FC8xvo

=======================================================
Host CPU Memory (GB) TMPDIR (GB) (Mb/s)
name(id) util % total progs used %used total avail rate
============ ======= === ===== ===== ====== ======= =
07:10:10
sjfqos009(0) 12 E 378 262 128 34 E 274.50 254.26 E N/A
sjfqos011(1) 16 E 378 262 138 36 E 274.50 254.25 E N/A
sjfqos010(2) 19 E 378 262 125 33 E 274.50 254.25 E N/A
sjfqos011(3) 16 E 378 262 138 36 E 274.50 254.25 E N/A
sjfqos012(4) 62 E 378 262 118 31 E 274.50 254.25 E N/A
07:11:10
sjfqos009(0) 12 E  378 262 133 36 E 274.50 254.26 E 127
sjfqos011(1) 123 T 378 262 168 44 E 274.50 245 E    145
sjfqos010(2) 91 G  378 262 138 36 E 274.50 254.25 E 103
sjfqos011(3) 123 T 378 262 143 38 E 274.50 245 E     14
sjfqos012(4) 82 E  378 262 131 35 E 274.50 254.25 E 135
07:12:10
sjfqos009(0) 12 E 378 262 138 36 E  274.50 254.26 E  125
sjfqos011(1) 99 B 378 378 378 100 T 274.50 254.25 E  145
sjfqos010(2) 97 B 378 262 373 99 T  274.50 254.25 E  103
sjfqos011(3) 99 B 378 378 378 100 T 274.50 254.25 E  12
sjfqos012(4) 86 G 378 262 138 36 E  274.50 254.25 E  133
```

Related Commands

- setMultiCpuUsage

report_resource

```
report_resource
[-help]
[-peak]
[-verbose]
[-start string | [-end string [-session_cpu]
[-diff_memory]]]
[-check_MemCpu {true | false} [-period integer]]
```

Displays the peak memory used during a software session. This command is used to measure the peak memory at any given point in the flow and and make the script better for automation flows.

Note: The `report_resource` command is only supported in the 64-bit version of the software and Linux port.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-check_MemCpu {true | false}`	Checks the machine's physical memory and CPU resources. When specified, the software enables memory/CPU checking and reports a warning if the free memory is below 5 percent of the total memory or if the average CPU load large.
`-diff_memory`	Outputs different peak memory and current memory between the step start and step end.
`-end string`	Specifies the end point to measure the peak memory attained during a specific command.
`-peak`	Reports current peak memory till a given point in a flow. This parameter also works with the `-start` and `-end` parameters of the `report_resource` command so that peak memory observed by an individual command can be reported.
`-period integer`	Specifies the period for checking the physical memory and CPU usage. *Default*: 1800s *Min*: 1 *Max*: INT_MAX

`-start` *`string`*	Specifies the start point to measure the peak memory attained during a specific command. **Note:** The `-start` and `-end` parameters support any user-specified string, such as fullFlow, timeDesign, and TD_PR. You must specify the same string for both the `-start` and `-end` parameters to measure the peak memory attained during a specific command.
`-session_cpu`	Reports the current CPU and real time after the reports last step status.
`-verbose`	Gives detailed memory usage information. It displays how much memory is being used at any time and of what form (physical vs. virtual). When specified, the following information is displayed: • `Cpu(s)` is the number of available processors in the machine. • `Load` average is the system load averages for the last 1 minute, 5 minutes, and 15 minutes. • `Mem` and `Swap` are the current memory information of the machine. • `Data Resident Set (DRS)` is the amount of physical memory devoted to other than executable code. "`current mem`" shows this value (Total current DRS) . • `Private Dirty (DRT)` is the memory which must be written to disk before the corresponding physical memory location can be used for some other virtual page. "`peak res`" shows this value (Total peak DRT). This is the minimum number that you must reserve to run the program. • `Virtual Size (VIRT)` is the total amount of virtual memory used by the task. • `Resident Size (RES)` is the non-swapped physical memory a task has used. The number of "Total Peak RES" is the recommended physical memory to reserve.

Examples

- The following command script specifies to display the peak memory attained during the `optDesign` command:

  ```
  report_resource -start "optD"
  optDesign
  report_resource -end "optD"
  ```

  The following message is displayed:

  ```
  Ending "optD" (total cpu=0:15:00, real=0:15:09, peak res=468.8M, current
  mem=396.3M)
  ```

- The following command script specifies to display the peak memory attained during the `timeDesign` command:

  ```
  report_resource -start fullFlow
  report_resource -start TD_PR
  timeDesign -postRoute
  report_resource -end TD_PR
  report_resource -start OD_PR
  optDesign -postRoute
  report_resource -end OD_PR
  report_resource -end fullFlow
  ```

  The following message is displayed:

  ```
  Ending TD_PR (total cpu=0:15:00, real=0:15:09, peak res=468.8M, current
  mem=396.3M)
  Ending OD_PR (total cpu=0:15:00, real=0:15:09, peak res=968.8M, current
  mem=796.3M)
  Ending fullFlow (total cpu=0:35:03, real=0:40:09, peak res=968.8M, current
  mem=796.3M)
  ```

- The following command script specifies to display the peak memory attained during the `optDesign` command:

  ```
  report_resource -peak -start optD1
  report_resource -start optD
  optDesign -postRoute
  ```

```
report_resource -end optD
`Ending "optD" (total cpu=0:00:12.9, real=0:00:48.0, peak res=358.8M, current
mem=300.2M)'
report_resource -peak -end optD1
`Ending "optD1" peak res=358.8M'
```

- The following command specifies to display detailed memory information:

  ```
  report_resource -verbose
  ```

  The following output is displayed:

  ```
  Current (total cpu=0:00:23.5, real=0:10:24, peak res=567.8M, current mem=567.8M)

  Cpu(s) 40, load average (past 1 minute): 7.41, load average (past 5 minutes):
  7.46, load average (past 15 minutes): 7.52
  Mem: 527321900k total, 317217944k used, 210103956k free, 3752k buffers
  Swap: 134217724k total, 0k used, 134217724k free, 183952200k cached

  Memory Detailed Usage :
                   Data Resident Set(DRS)  Private Dirty(DRT)  Virtual Size(VIRT)
  Resident Size(RES)
  Total current :   642.3M                 155.6M              1743.3M
   567.8M
  peak :            642.3M                 155.6M              1743.3M
   567.8M
  ```

setDistributeHost

```
setDistributeHost
[-help]
[-add string]
[-args string]
[-custom]
[-custom_script string]
[-custom_script_list string]
[-local]
[-lsf]
[-nc]
[-noWaitTaskComeUp]
[-queue string]
[-remove string]
[-reportLsfInfo]
[-resource string]
[-rsh]
[-shellTimeout integer]
[-single_cpu_lsf_args string]
[-ssh]
[-timeOut integer]
[-env_script string]
```

Specifies the multiple-CPU processing configuration for distributed processing or Superthreading. The software finds hosts from the last `setDistributeHost` command specified before a distributed processing or Superthreading command starts.

Use `getDistributeHost` to display the current settings for the `setDistributeHost` command.

> This command is required for distributed processing and Superthreading.

Parameters

`-help`	Prints out the command usage.

`-add` *string*	Adds the specified hosts to the `rsh/ssh` configuration. If you specify more than one host, separate each host name with a space.
	Each time you specify a host, the software uses it to run one client process. To run more than one client process on a host, specify it more than once. For example, if you have two machines that each has two CPUs and you want to use both CPUs in both machines, type a command like the following:
	`setDistributeHost -rsh -add {machine1 machine2 \`
	`machine1 machine2}`

`-args` *string*	Specifies any additional arguments you need to provide to the Sun Grid Engine (SGE) or Load Sharing Facility (LSF) job submission command.
`-custom`	Specifies to use the custom job submission script for distributed processing.
`-custom_script` *string*	Specifies a custom script to run distributed processing. You can use this parameter with `-custom` only.
`-custom_script_list` *string*	Specifies to customize the launch of multiple jobs, based on resources required for these jobs. This parameter allows you to specify multiple launch strings in order of priority with a maximum allowed job count for each string. If you request for N number of jobs, the command starts submitting jobs starting from *string1* , until the corresponding count is reached, at which point it starts submitting the next string in the list. Therefore, the command submits the first " *count1* " jobs with *string1* , the next " *count2* " jobs with *string2* and so on. It stops when the total submitted jobs has reached the requested number (N). **Note**: This parameter can only be specified with the `-custom` parameter.

`-env_script` *string*	Specifies a user-defined shell environment script that is run for each of the slave hosts. The parameter allows you to pass these environment settings to the slave hosts by including them in the shell environment script. The license variables used by the software are also automatically passed to the slave hosts to avoid license errors.
`-local`	Runs all distributed processing jobs on the master machine.
`-lsf`	Specifies a Load Sharing Facility configuration. If you do not specify any parameters for `-lsf`, the software uses default settings. **Note:** To use this parameter, LSF must already be set up (typically by specifying the LSF_ENVDIR and LSF_SERVERDIR environment variables) and bsub must be in your search path. Contact your LSF administrator for details.
`-nc`	Uses NC commands (Network Computer from RTDA) for distributed processing.
`-noWaitTaskComeUp`	Does not allow the software to wait for tasks that are delayed or are not likely to occur.
`-queue` *string*	Specifies the queue for the LSF or SGE configuration.
`-remove` *string*	Removes the specified hosts or all hosts from the `rsh/ssh` configuration. If you specify more than one host, separate each host name with a space.

`-reportLsfInfo`	Specifies to report LSF information in a single report. This parameter allows you to collect the system statistics on the status of the distributed jobs. It has the following information: • Host name for master and LSF tasks • Number of CPUs on each machine that are being used by a process • Job ID/PID for each task process • Total memory used on each machine • Total virtual memory used on each machine • Total CPU time used on each machine You must specify this parameter with the `-lsf` parameter. To get the LSF information, specify the `getDistributeHost -reportLsfInfo` command parameter. You must specify the `getDistributeHost -reportLsfInfo` only after the LSF tasks exit to obtain the task CPU time. If you specify this command before the task exits, the command gives a warning message.
`-resource` *string*	Specifies a resource string for the SGE or LSF queue. **Note:** The correct resource string is specific to your installation. Contact your LSF administrator for the appropriate parameters and values.
`-rsh`	Specifies a remote shell configuration. For information, see the `rsh` documentation.
`-shellTimeout` *integer*	Specifies the number of seconds the host machine waits for `rsh/ssh` machines to become available for multiple-CPU processing. *Default:* `5`
`-single_cpu_lsf_args` *string*	Specifies the `bsub` options. For information, see the LSF documentation. **Note:** You must use this parameter to adjust the LSF argument for a single CPU application. Alternatively, use the `-args` parameter to adjust the LSF argument for an application that runs in the Superthreading mode.
`-ssh`	Specifies a secure shell configuration.

`-timeout` *integer*	Specifies the default timeout for a submitted job that will never start. This functionality prevents the master machine from hanging when the clients do not come up for some reason. After timeout, a warning message appears informing you that the distributed CPUs did not come online within the expected time. The master will continue waiting for the clients and it is up to you to continue waiting or kill the run if there are problems with the Distributed Resource Management (DRM) system. A sample warning message is given below: Task id 2 did not come up within the specified timeout period (300 seconds). Use `setDistributeHost -timeout` to change the value. This may be a real error due to incorrect or incomplete platform/machine specification provided via setDistributeHost to your distributed resource management system (LSF/Sun Grid Engine etc.). If running locally (`setDistributeHost -local`), the current machine may be overloaded with too many jobs and swapping (check free memory and CPUs with top). For local runs, consider reducing the number of CPUs you are using via the setMultiCpuUsage command or free up some memory by killing other memory intensive processes. *Default:* 3600 for all the modes (LSF, NC, SGE, RSH, SSH, local and custom **Note**: When the `set_distribute_host` command is not specified, the software sets the default mode as local and the EDP timeout is `3600` seconds.

Examples

rsh

The following command creates an `rsh` configuration and adds hosts `host10`, `host11`, and `host12`:

```
setDistributeHost -rsh -add {host10 host11 host12}
```

The following commands create an `rsh` configuration with `host1` and `host2` as remote hosts and then adds `host3` and `host4` to the configuration:

```
setDistributeHost -rsh -add {host1 host2}
```

```
setDistributeHost -rsh -add {host3 host4}
```

LSF

The following command uses an existing LSF environment, and relies on the general LSF set up by the system administrator. In this example, the name of the queue implies that all the machines have 4 Gb of memory.

```
setDistributeHost -lsf -queue mem4G
```

The following command schedules a LSF job named `bigQueue` to run on a Linux machine for a maximum of 30 minutes. No other jobs can run on the machine while `bigQueue` is running.

```
setDistributeHost -lsf -queue bigQueue -args {-x -W 30} \
   -resource {OSNAME==Linux}
```

The following command schedules a LSF job named `smallQueue` to run on Linux machines with minimum speed of 2000 MHz.

```
setDistributeHost -lsf -queue smallQueue -args {-W 20} \
    -resource {OSNAME==Linux && SPEED>2000}
```

Note: Check with your LSF administrator for the correct values for the resource string, as the parameters and their values differ in each installation. For example, the parameter name for the speed of the machine might be `MHZ` instead of `SPEED` (as in the preceding example), and the units might be specified in gigahertz instead of megahertz.

The following command specifies to report LSF information:

```
setDistributeHost -lsf -args "-q lnx64" -reportLsfInfo
getDistributeHost -reportLsfInfo lsf.log
```

Note: Use the above two commands together to save the LSF information in the specified file.

Local

The following command runs all distributed processing jobs on the machine on which you are running the master version of the software:

```
setDistributeHost -local
```

Custom

The following command runs distributed processing with a custom script:

```
setDistributeHost -custom -custom_script {bsub -q bigQueue -x -R "OS==Linux && memory>20000"}
```

The following command runs distributed processing with a custom script list:

```
setDistributeHost -custom -custom_script_list {{{bsub -P ICD -W 2 -q admin -R "OSNAME==Linux && OSREL==EE50"}1} {{rsh hpc1 -e} 1}}
```

Examples

- The following command allows the client to source the `my_env_file` before start up on the same machine:

  ```
  setDistributeHost -local -env_script my_env_file
  ```

  This command allows the master to pass on the user-defined environment settings.

- The following command allows the client to run using the LSF:

  ```
  setDistributeHost -lsf -noWaitTaskComeUp
  ```

  When the `-noWaitTaskComeUp` parameter is specified, the software will not wait for all the clients to come up when parts of the clients are launching timeout.

Related Topics

- Accelerating the Design Process By Using Multiple-CPU Processing chapter in the *Innovus User Guide.*
- "Tools Menu" chapter in the *Innovus Menu Reference*
  - Set Multiple CPU Usage
- `getDistributeHost`

setMultiCpuUsage

```
setMultiCpuUsage
[-help]
[-acquireLicense <integer>]
[-autoPageFaultMonitor {0 1 2 3}]
[-cpuPerRemoteHost <integer>]
[-keepLicense {true false}]
[-licenseList <string>]
[-localCpu <string>]
[-releaseLicense]
[-remoteHost <integer>]
[-reset]
[-threadInfo {0 1 2}]
[-verbose]
```

Specifies the number of threads to use for multi-threading, or the maximum number of computers to use for distributed processing, or the maximum number of computers and the number of threads to use for Superthreading. Optionally, reports usage information.

This command is required for multi-threading, distributed processing, and Superthreading.

The `setMultiCpuUsage` command enables multi-threaded timing reporting under the following conditions:

- `timing_report_group_based_mode` global variable is set to `true`. If there are multiple paths with the same slack then the selection can change on multiple runs. This may result in some changes in the `report_timing` output.
- `report_analysis_coverage` command is in non-verbose mode.
- `check_timing -include clock_crossing` parameter is specified.
- `-set_table_style -no_frame_fix_width` parameter is specified.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.

`-acquireLicense` *integer*	Acquires licenses to enable the specified number of CPUs. For example, if you specify `-acquireLicense 5`, the software checks out enough licenses to enable 5 CPUs. After the licenses are checked out, they can be used for multi-threading, distributed processing, or super-threading.
`-autoPageFaultMonitor {0 | 1 | 2 | 3}`	Specifies to set a warning rank for performance issues caused by major page faults. The parameter supports the following values: • 0 - specifies to disable the reporting of major page faults. • 1 - specifies to set the warning rank as 1. This means PF/total PF > 0.8, b > 10, and wa > 85% • 2 - specifies to set the warning rank as 2. This means PF/total PF > 0.5, b > 5, and wa > 65% • 3 - specifies to set the warning rank as 3. This means PF/total PF > 0.2, b > 1, and wa > 40% where PF is the number of major page faults from the software, total PF is the total number of major page faults in the machine, b is the number of blocked processors, and wa is CPU waiting time.
`-cpuPerRemoteHost` *integer*	Specifies the number of CPUs used by clients for multi-threading. **Note:** For super threading, you must use this parameter in conjunction with the `-remoteHost` parameter. *Default:* 1
`-keepLicense {true | false}`	Specifies whether to keep the acquired multiple CPU-licenses until the current session ends. Specify this parameter before running any commands that require multiple-CPU applications. To release all multiple-CPU licenses immediately, use the `-releaseLicense` option. *Default:* `true`

`-licenseList {tpsmp}`	Specifies the list or order of licenses the software should use for checking out licenses during multiple-cpu processing. The parameter does not support an empty license list {}. This parameter allows the `tpsmp` option.
`-localCpu` *string*	Specifies the number of CPUs on a local machine. When the `-localcpu` parameter is set to `1` (default), the multi-threaded timing propagation mode is enabled. Also, both CPU multi-threading and distributed process are enabled. The `set_multi_cpu_usage -localCpu` command automatically controls the total number of threads used up to the specified number. The number of specified threads cannot be more than the number of CPUs available in the local computer. Consider a computer having 8 CPUs, then the following command will issue a warning that the number of specified local CPUs is greater than the actual number of CPUs available: `set_multi_cpu_usage -localCpu 10` The local CPU count can be the maximum value that is shown as “processors” on the system under “/proc/cpuinfo”. This value can be obtained by running the “checkSysConf” utility in the "tools/bin" folder. `$> tools/bin/checkSysConf` `============== checkSysConf: Version 3.53 ====================` `OS Version .............: Red Hat Enterprise Linux Workstation release 7.6 (Maipo)` `Kernel Version .........: 3.10.0-957.5.1.el7.x86_64` `Hardware Type ..........: x86_64` `Memory .................: 2063415 Megabyte` `CPU model ..............: Intel(R) Xeon(R) Gold 6254 CPU @ 3.10GHz` **`No. of CPUs ............: 72`** `Total Swap Space........: 131072 Megabyte` `==================================` `$> cat /proc/cpuinfo	grep "processor"	tail -1	awk -F ":" '{print $2 + 1}'` `72` *Default*: 1

`-releaseLicense`	Releases all multiple-CPU license(s) immediately. By default, the software holds multiple-CPU licenses until the end of the current session. To specify that the software release multiple-CPU licenses after every multiple-CPU command runs, use `-keepLicense`.
`-remoteHost` *integer*	Specifies the number of clients. This parameter is required for distributed processing and super threading. *Default:* 0
`-reset`	Resets the specified parameters back to default values.

`-threadInfo {0 | 1 | 2}`	Reports usage information.
	
	Default: 0
	
	Specify one of the following values:
	
	`0`: Does not write messages to the log file.
	
	`1`: Writes the final message to the log file.
	
	For example,
	
	All threaded jobs finished (10 elapsed sec: 0 processor sec (parent), 0 system sec (parent), 0 processor sec (threads), 0 system sec (threads)).
	
	`2`: Writes additional starting/ending information for each thread.
	
	For example,
	
	`Starting threaded job 1...`
	`Starting threaded job 2...`
	`Starting threaded job 3...`
	`Ending threaded job 2 (1 elapsed sec, 0 processor sec, 0 system sec, 1.160M).`
	`Ending threaded job 3 (2 elapsed sec, 0 processor sec, 0 system sec, 0.895M).`
	`Ending threaded job 1 (10 elapsed sec, 0 processor sec, 0 system sec, 1.172M).`
	`All threaded jobs finished (10 elapsed sec: 0 processor sec (parent), 0 system sec (parent), 0 processor sec (threads), 0 system sec (threads)).`

`-verbose`	Displays messages when changing the multiple-CPU settings.

Examples

To run global placement with the maximum number of 4 threads, specify the following commands:

```
setMultiCpuUsage -localCpu 4
place_design
```

To run global placement with the maximum number of available threads, specify the following commands:

```
setMultiCpuUsage -localCpu max
place_design
```

To run detailed routing with the NanoRoute router in Superthreading mode with two machines and six threads, using an LSF queue, specify the following commands:

```
setDistributeHost -lsf -queue myLSFqueue
setMultiCpuUsage -remoteHost 2 -cpuPerRemoteHost 6
detailRoute
```

Related Information

- Accelerating the Design Process by Multiple-CPU Processing chapter in the *Innovus System User Guide*
- Using the NanoRoute Router chapter in the *Innovus System User Guide*
  - Accelerating Routing with Multi-Threading and Superthreading
- "Tools Menu" chapter in the *Innovus System Menu Reference*
  - Set Multiple CPU Usage
- `checkMultiCpuUsage`
- `getMultiCpuUsage`

22

OpenAccess Commands

- add_oa_library_property
- compare_cellview
- copyOaRestoreFiles
- createLib
- dd_get
- get_oa_default_rule_lib
- getOaxMode
- oaIn
- oaOut
- registerTrigger
- report_oa_lib
- save_abstract
- saveOaBlackboxes
- set_cell_binding
- setOaxMode
- unlockOaDesign
- update_oa_lib
- write_lef_library
- write_oa_techfile

add_oa_library_property

```
add_oa_library_property
[-help]
-lib libName
-name prop_name
-type {boolean integer float string}
-value propValue
```

Adds a new property to the specified OpenAccess library. This command cannot be used to change the value of an existing property.

Note: You cannot use the name `techLibName` for the property you are adding because this name is used by the OpenAccess infrastructure to store the property attached to the library.

Parameters

`-help`	Prints a brief description that includes type and default information for each `add_oa_library_property` parameter. For a detailed description of the command and all of its parameters, use the man command: `man add_oa_library_property`.
`-lib libName`	Specifies the name of the OpenAccess library to which the property has to be added.
`-name prop_name`	Specifies the name of the property being created.
`-type {boolean integer float string}`	
	Specifies the type of the property being created.
`-value propValue`	Specifies the value to be set for property being created. The value is interpreted from the input string and converted to the specified type.

Example

In Innovus, the following set of commands creates a library named *myLib*, adds the *prop1* property to it, and saves the design before exiting:

```
createLib myLib -attachTech adiTech

add_oa_lib_property -lib myLib -name prop1 -type string -value value1

saveDesign -cellview {myLib block1 layout}

exit
```

You can now add more properties to the library in Virtuoso. When you restore the design in Innovus, all properties previously added to the library, whether in Innovus or Virtuoso, reflect correctly.

Related Information

- `createLib`

compare_cellview

```
compare_cellview
[-help]
[-ignore {pin_purpose}]
[-log report_file_name]
{ {{-source {from_lib from_cell from_view} -compare {to_lib to_cell to_view}} | {-lib lib -cell cell -view
{sourceView compareView}}} }
```

Compares the boundary and terminal (name, direction, layer, location, size, and port) of the specified cell views. In addition, if both cell views are identified as abstract, the command also compares cell type, symmetry, and obstructions.

Use this command to compare two OpenAccess databases. For instance, you can use this command to compare the lib/cell/view specified using `-source` to the lib/cell/view specified with `-compare`. Typical uses are comparing two versions of a layout cell view or comparing an abstract cell view to a layout cell view.

This command can be used without design import.

Parameters

`-help`	Prints a brief description that includes type and default information for each `compare_cellview` parameter. For a detailed description of the command and all of its parameters, use the man command: `man compare_cellview`.
`-cell` *`cell`*	Specifies the cell for comparing two views.
`-compare {`*`to_lib to_cell to_view`*`}`	Specifies the library/cell/view that is to be compared.
`-ignore {pin_purpose}`	Specifies the list of attributes to be excluded from the cell comparison. As of now, the only attribute that can be ignored is the purpose, which is specified using the value `pin_purpose` with `-ignore`.
`-lib` *`lib`*	Specifies the library for comparing two views.
`-log` *`report_file_name`*	Specifies the report file in which all the comparison detail is to be written.
`-source {`*`from_lib from_cell from_view`*`}`	Specifies the source library, cell, and view.
`-view {`*`sourceView compareView`*`}`	Specifies different views of the same library/cell.

Examples

- The following command compares two views of the same cell that are present in the same library `zambezi45` and writes the report to the `test1a` file:

```
compare_cellview -source {zambezi45 pll_vcodiv layout} -compare {zambezi45 pll_vcodiv abstract} -log test1a
```

A sample output is displayed below:

```
################################################
# #
# Compare_cellview comparison #
# #
################################################

Source Design : 'zambezi45/pll_vcodiv/layout' is identified as layout design
Compare Design : 'zambezi45/pll_vcodiv/abstract' is identified as abstract design

Source Design seems to have been modified after the Compare Design has been created

Compare Celltype

Source Design has celltype 'none' & Compare Design has Celltype 'block'

Compare Cell Boundary

Source Design has different PRBoundary than Compare Design

Compare Terminals

Source Design has Pin 'en' which has Port with shape (36.215 0 36.355 0.14) on layer 'Metal5' and purpose
'pin' is not present in Compare Design
Compare Design has Pin 'en' which has a Port with shape (36.215 0 36.36 0.14) on layer 'Metal5' and purpose
'net' is not present in Source Design
...

Antenna Information

Antenna Information is defined for Pin 'en' in Compare but not defined in Source Design
Antenna Information is defined for Pin 'vcocalen' in Compare but not defined in Source Design
...

Source Design is not equivalent to Compare Design
```

- The following command compares the views `fplan1` and `fplan2` of the library `DesignLib` and the cell `myAlu`:

```
compare_cellview -lib DesignLib -cell myAlu -view {fplan1 fplan2}
```

copyOaRestoreFiles

```
copyOaRestoreFiles
fromLib fromCell fromView
toLib toCell toView
```

Copies Innovus-specific restore files from one OpenAccess database to another. The software copies all of the files (except `master.tag` and `layout.oa`) and renames those with the `cell` to match the `toCell` cell name. This feature supports the ECO flow, where a cellview saved with the `saveDesign` command and set of restore files is edited by an external tool (Virtuoso® Chip Editor), then saved to a new database to avoid overwriting the unedited version. The external tool does not copy the restore files to the new cellview.

If the originating database does not contain restore files (such as `cell.conf`, `cell.mode`, `siFix.options`, and `enc.pref.tcl`, then the software generates an error message such as the following:

```
ERROR: No restore Files found at specified source
/usr4/Tests/CE/OA22/PC44d_test/mydesign/top/layout
```

If the destination OpenAccess database already contains restore files, the software generates a warning message and overwrites the files.

You can use the `copyOaRestoreFiles` command after you start a new Innovus session, and before you use the `restoreDesign -cellview` command.

Parameters

fromLib fromCell fromView	Specifies the location of the files to copy. This is the design from the previous Innovus session.
toLib toCell toView	Specifies where to copy the files. This refers to the new design.

Examples

- The following command copies Innovus-specific restore files from `myDesign/topCell/layout` to `newDesign/newTopCell/newLayout`:

  ```
  copyOaRestoreFiles myDesign topCell layout newDesign newTopCell newLayout
  ```

- The following procedure performs ECOs with the Virtuoso Chip Editor:

  1. From an Innovus session, save the OpenAccess design.

     ```
     saveDesign -cellview {lib cell view}
     ```

  2. Exit the Innovus session.
  3. Open the OpenAccess database in the Virtuoso Chip Editor tool and edit the design. Exit the tool.
     Note: You must use either of the following: Virtuoso Chip Editor, Virtuoso Layout XL , or Virtuoso Layout GXL.
  4. Start an Innovus session.
  5. Copy Innovus-specific files from the old design to the new design.

     ```
     copyOaRestoreFiles fromLib fromCell fromView toLib toCell toView
     ```

  6. Restore the OpenAccess design.

     ```
     restoreDesign -cellview {lib cell view}
     ```

Related Information

- restoreDesign
- saveDesign

createLib

```
createLib
[-help]
libName
[-libPath path]
[-noCompress]
{-attachTech techlib |
-copyTech techlib |
-referenceTech techliblist}
```

Copies or attaches a technology database to a library. If the library exists, this command does not update the library.

Note: The parameters must appear in the order specified by the syntax.

The `createLib` command is part of the OpenAccess design flow. Use this command to create an OpenAccess library at the specified path. `createLib` needs to be called only once before the design is saved. Subsequent `saveDesign` calls to the same library will automatically find out the library path that gets stored in `cds.lib`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `createLib` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man createLib`.
`libName`	Specifies the name of a library to be created. The software creates the library and adds its name to the `cds.lib` file.
`-libPath` *`path`*	Specifies the name of a directory in which to save the library. The path you specify must include the directory that will become the library itself. *Default:* *`./libName`* (Saves the library to the current working directory)
`-noCompress`	Creates a new library without any compression irrespective of the compression settings provided by the `setOaxMode` command.
`-attachTech` *`techlib`*	Attaches the technology database specified by the technology library to the library that is to be created.
`-copyTech` *`techlib`*	Copies the technology database specified by *`techLib`* to the library specified by *`libName`*, and saves the library to the current working directory.
`-referenceTech` *`techliblist`*	Creates a local technology database in the design library that can be modified incrementally.

Examples

- The following command creates the library `myLib` in the current directory, and attaches the technology database in `myRefLib` to `myLib`:

  ```
  createLib myLib -attachTech myRefLib
  ```

  Note: Note: If `myRefLib` is read-only, the `saveDesign -cellview` command generates an error if it needs to update the technology database.
- The following command creates the library `TEST_LIB` at the specified path, and attaches the technology database in `TEST_REF_LIB` to TEST_LIB:

  ```
  createLib TEST_LIB -libPath ./outputs/TEST_LIB -attachTech TEST_REF_LIB
  ```
- The following command generates an error because only one reference library can be specified:

  ```
  createLib myLib -attachTech {myRefLib1 myRefLib2}
  ```
- The following command creates the library `myLib`, and copies the technology database from `myRefLib` to `myLib`. If `myRefLib` is modified, the two technology databases will become out of sync. Additionally, any future write operations from the `saveDesign -cellview` command will modify the technology database copy in `myLib` and also cause the databases to be out of sync.

  ```
  createLib myLib -copyTech myRefLib
  ```
- The following command creates a new library which references existing technology databases (`refLib1` and `refLib2`), but allows local technology updates if required (typically to add new custom vias to the local technology database).

  ```
  createLib myLib -referenceTech {refLib1 refLib2}
  ```

Related Information

- `add_oa_library_property`
- `saveDesign`
- `setOaxMode`

dd_get

```
dd_get
{-all_lib} |{-dd dd_objHandle | -lib libName | -cell {lib cell} | -cellview {lib cell view} }
[-children | -file | -path | -type |-name]}
```

Returns OpenAccess library information.

Parameters

`-all_lib`	Returns a list of design data (`dd`) object handles for the list of libraries present in the library definition file (`cds.lib`).
`-cell {lib cell}`	Specifies the names of the library and cell to be processed.
`-cellview {lib cell view}`	Specifies the names of the library, cell, and view to be processed.
`-children`	Returns the `dd` object handles of the children of the specified `dd` object handle. Children of libraries are cells. Children of cells are views.
`-dd dd_objHandle`	Specifies the `dd` object to be processed.
`-file`	Returns the `dd` object handles of all the files in the specified `dd` object handle.
`-lib libName`	Specifies the name of the library to be processed.
`-name`	Returns the name of the specified `dd` object handle.
`-path`	Returns the absolute path name (`dd` path) of the specified `dd` object handle.
`-type`	Returns the type of the specified `dd` object handle.

Examples

- The following command returns the path of the library/cell/view as shown below:

  ```
  dd_get -cellview {pdkLib cell1 abstract} -path
  ```

  returns:

  ```
  /libs/pkdLib/cell1/abstract
  ```

  This command is equivalent to the following SKILL function:

  ```
  ddGetObjReadPath(ddGetObj("pdkLib" "cell1" "abstract"))
  ```

- The following proc traverses the cds.lib structure (expanding #INCLUDE as required) to get the complete list of OpenAccess libraries and their full paths:

  ```
  # expand #INCLUDE as needed and print all libraries and their full paths
  proc print_all_lib_paths {} {
     puts [format "%16s -> %s" "Library Name" "Library Path"]
     puts "-----------------------------------------------------------------"
     foreach oa_lib [dd_get -all_lib] {
         puts [format "%16s -> %s" [dd_get -dd $oa_lib -name] [dd_get -dd $oa_lib -path]]
     }
  }
  ```

get_oa_default_rule_lib

```
get_oa_default_rule_lib
[-help]
[-rule rule_name]
[-libs lib_list]
[-verbose]
```

Searches through a specified library (or libraries) to check whether a specified LDRS exists or not. You can run this check before reading a library.

Parameters

`-help`	Prints a brief description that includes type and default information for each `get_oa_default_rule_lib` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man get_oa_default_rule_lib`
`-libs lib_list`	Specifies the list of library names to search through. If not specified, all libraries are searched.

`-rule rule_name`	Specifies the name of the rule to find.
	If not specified, the `init_oa_default_rule` variable is used. If the variable has an empty string value, the standard lookup (`LEFDefaultRouteSpec` by type, then by name) is used.
	If the specified *`rule_name`* does not exist, the command displays an error such as:
	`Rule '<rule_name>' was not found in specified libraries.`
	If the specified *`rule_name`* is found, the command returns the name of the library where the specified *`rule_name`* was found. This could be from the `-libs lib_list` or any ITDB that one of those libraries refers to.
	If the specified *`rule_name`* is found but is not a legal LDRS type as determined by the OAX reader code, the command displays an error such as:
	`Constraint group '<rule_name>' was found in library '<lib_name>', but is not a legal LEFDefaultRouteSpec`

`-verbose`	Echoes details of library name where the rule was found (including descending through technology references from libraries specified in the *`lib_list`*.

Examples

- The following example shows how `get_oa_default_rule_lib` can be used to search for a specific library and the graph of techs that it references:

```
set init_oa_ref_lib "tech_lib macro_lib"
set init_oa_default_rule LDRS_65nm_61m
set tech_lib [lindex $init_oa_ref_lib 0]
set lib_found [get_oa_default_rule_lib -libs $tech_lib] # use the init_oa_default_rule to look up the library
if {$lib_found != ""} {
Puts "$init_oa_default_rule was found in $lib_found or in the graph of libraries that $tech_lib references"
} else {
Puts "$init_oa_default_rule was not found in $tech_lib nor in the graph of libraries that $tech_lib
references"
}
```

- The following command shows how the `-verbose` option can be used:

```
get_oa_default_rule_lib -verbose
INFO: Default rule 'LEFDefaultRouteSpec' was found in library 'FEOAreflib8'.
FEOAreflib8
```

Related Information

- `init_oa_default_rule`
- `init_oa_ref_lib`

getOaxMode

```
getOaxMode
[-help]
[-allowAnalysisOnly]
[-allowBitConnection]
[-allowTechUpdate]
[-bindkeyFile]
[-compressLevel]
[-convertTo]
[-cut_pg_rail]
[-cutRows]
[-displayDrfFile]
[-displayDrfInLibrary]
[-drcFillPurpose]
[-encloseQuickAbstractPins]
[-fullLayerList]
[-fullPath]
[-hybridRows]
[-instPlacedIfUnknown]
[-libCreateMode]
[-locking]
[-logicOnlyImport]
[-merge_trim]
[-pinPurpose]
[-pushPinConstraint]
[-quickAbstractForCustomCells]
[-readSystemReservedLayers]
[-saveCdsFixedVias]
[-saveMaskDataLocked]
[-saveNetVoltage]
[-saveRelativePath]
[-saveRestoreFile]
[-silently_ignore_unsupported_vias]
[-textPurpose]
[-tieNet]
[-updateMode]
[-useVirtuosoBindkey]
[-useVirtuosoColor]
[-viewSubType]
[-quiet]
```

Displays information about a specified OpenAccess mode parameter in the log file and at the console. The following information is included:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the software displays information for all of the OpenAccess mode parameters.

Parameter

`-help`	Outputs a brief description that includes type and default information for each `setOaxMode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man getOaxMode`.
`parameter_name`	Displays information for the specified parameters. You can specify one or more parameters. See `setOaxMode` for descriptions of the OpenAccess mode parameters you can specify.
`-quiet`	Displays the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software displays the current settings of all `setOaxMode` parameters in Tcl list format.

Examples

- The following command displays the current setting of the `-fullPath` parameter:

  ```
  getOaxMode -fullPath
  ```

  The software displays the following information:

  ```
  -fullPath false # bool, default=false
  false
  ```

- The following command displays the current settings for all `setOaxMode` parameters:

  ```
  getOaxMode
  ```

- The following command displays the current setting for the `-fullPath` parameter in Tcl list format only:

  ```
  getOaxMode -fullPath -quiet
  ```

  The software displays the following information:

  ```
  false
  ```

- The following command displays the current settings for all `setOaxMode` parameters in Tcl list format only:

  ```
  getOaxMode -quiet
  ```

Related Information

- `setOaxMode`

oaIn

```
oaIn
[-help]
lib
cell
view
[-filter {block_insts blockages boundary fixed_core_insts floorplan pad_insts pin_shapes regions regular_routing
special_routing bump_insts}]
[-net {netnames}]
```

Restores floorplan, placement, and routing data using the OpenAccess format. This command only reads physical information from the OpenAccess database. For information on restoring an OpenAccess database, see restoreDesign.

You can use the `oaIn` command after importing the design.

Notes

- Innovus supports only the following begin/end extension values for DEF `NETS` style wiring segments: 0, width/2 or LEF `LAYER WIREEXTENSION`. If any of the wire extensions has unsupported extension value, then Innovus converts it to a special wire (DEF `SPECIALNETS`). While writing back OpenAccess (`oaOut` or `saveDesign -cellview`), these special wires will again be converted back to DEF `NETS` style wiring.
- `oaIn` issues an error if the `DESIGNRULEWIDTH` attribute is specified in blockage for 20nm and below design nodes. As a lot of spacing rules in advanced nodes no longer just rely on wire width, `DESIGNRULEWIDTH` is not allowed for 20nm and below nodes and a `SPACING` override should be used instead.

Parameters

`-help`	Outputs a brief description that includes type and default information for each parameter of the `oaIn` command. For a detailed description of the command and all of its parameters, use the `man` command: `man oaIn`.
`lib`	Specifies the OpenAccess library name.
`cell`	Specifies the name of the top cell to import.
`view`	Specifies cell view name to import.
`-filter {block_insts blockages boundary fixed_core_insts floorplan pad_insts pin_shapes regions regular_routing special_routing bump_insts}`	

	Lists the OpenAccess cellview information data to be read. When not specified, all information is read from the OpenAccess cellview. When `-filter` is specified, only the object types that are chosen will be read. Additionally, when reading pin_shapes or instances, information in the cellview that refers to terminals or instances that do not exist in memory will be ignored. The legal `-filter` options are: • `block_insts` - Indicates that any CLASS BLOCK instances should be updated. Physical-only block instances may be added to the database if the placement status is `fixed` or `cover`. • `blockages` - Indicates that blockages should be processed. If a blockage is attached to an instance that does not exist in the in-memory database, then it is ignored. • `boundary` - Indicates that the design boundary information, including rows and tracks, should be updated. When `boundary` is specified, the design boundary, rows and tracks are created from the new boundary. • `fixed_core_insts` - Indicates that any CLASS CORE instances that have placement status `fixed` or `cover` should be updated. Physical-only core instances may be added to the database if the placement status is `fixed` or `cover`. • `floorplan` - Is equivalent to specifying "`block_insts blockages boundary fixed_core_insts pad_insts pin_shapes regions special_routing`" • `pad_insts` - Indicates that any CLASS PAD instances should be updated. Physical-only pad instances may be added to the database if the placement status is `fixed` or `cover`. • `pin_shapes` - Indicates that pin shapes should be read. When `pin_shapes` is specified, the existing pins are replaced by the ones being read from the cellview. • `regions` - Indicates regions should be updated. • `regular_routing` - Indicates that nets should be processed and regular routing (and associated shield net wiring) and net constraints updated. • `special_routing` - Indicates that nets should be processed and special routing (except shield nets wiring) should be read. • `bump_insts` - Indicates that bump instances from the cell view need to be read. Use `oaIn -filter` to read floorplanning information from a cellview whose connectivity does not match the current Innovus in-memory connectivity. The`-filter` option allows you to load some data selectively, while not loading others. When `-filter` is used, error checking for database consistency (nets, instances, terminals, etc.) is disabled. While similar to Virtuoso Load Physical View (LPV), the `-filter` option is only a subset and not intended to be a complete replacement.
`-net {`*`netnames`*`}`	
	Specifies the list of nets whose routing (routing and special) and constraints have to be read with the `-filter` option. Shielding wiring/vias are also read in if they are used to shield any of the specified nets. If this option is used, typically `regular_routing` is not specified as part of the `-filter` objects.

Example

- The following command restores data using OpenAccess:
  ```
  oaIn design amba_usb layout
  ```

Related Information

- `oaOut`
- `restoreDesign`
- `saveDesign`
- `setOaxMode`
- *Mixed Signal (MS) Interoperability Guide*
  - Design Data Preparation chapter
  - Schematic-Driven Mixed Signal Design Flow chapter
  - Virtuoso Digital Implementation chapter
  - Chip Finishing and ECO Flows chapter

oaOut

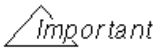
```
oaOut
[-help]
lib
cell
view
[-autoRemaster]
[-copyTechFromReflib]
[-cutRows]
[-early_global_route]
[-leafViewNames {lefView1 lefView2 ...}]
[-noConnectivity [-noStdCells]]
[-noOverwriteAbstract]
[-refLibs {refLib1 refLib2 ...}]
```

Saves floorplan, placement, and routing data in the OpenAccess format.

You can use the `oaOut` command after importing the design.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `oaOut` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man oaOut`.
lib	Specifies the target OpenAccess library name.
cell	Specifies the target cell name.
view	Specifies the target view name.
`-autoRemaster`	Overrides the reference libraries and leaf view names specified binding, and utilizes the cell layout viewing binding stored on the library cell master for each leaf cell (stored in the library cell's layout library and layout view attributes). The binding can be specified using the `set_cell_binding` command.
`-copyTechFromReflib`	Copies the technology file to the current design library. *Default:* If you do not specify this parameter, Innovus attaches the technology to the current design library. **Note:** This parameter applies only when you create libraries.
`-cutRows`	Specifies that rows are cut against placement obstructions and block halos, similar to the behavior of the `defOut` command. *Default:* Does not cut rows.
`-early_global_route`	Writes out a design containing Early Global Route data.
`-leafViewNames {lefView1 lefView2 ...}`	
	Specifies a list of view names to be searched during leaf cell instance binding.

`-noConnectivity`	This option should be used if the OpenAccess layout cellviews that are being used in remastering (-autoRemaster or -refLibs/-leafViewNames) do not have any terminals (as is the case when the layout cellview is created from GDSII/Stream data), in which case it is not possible to maintain netlist connectivity. Writes out a design that contains instances and physical shapes. When specified, the software creates an OpenAccess module domain using the instance hierarchy, but does not attempt to create any nets, pins, or instance pins.  Designs written out using the `-noConnectivity` parameter cannot be reloaded into the Innovus software.
`-noOverwriteAbstract`	Prevents the software from overwriting abstracts of partitions and blackboxes. *Default:* Innovus overwrites abstracts of partitions and blackboxes. Innovus never overwrites abstracts of library data.
`-noStdCells`	Does not save instances of core cells to the OpenAccess database. However, if an instance is either fixed or covered, it is saved to the database. Use this parameter if you just need the preliminary design for DRC checks of the floorplan. In such cases, leaving out the standard cell information makes the file smaller and reduces the DRC run time. **Note**: The `-noStdCells` parameter must be used with the `-noConnectivity` parameter.
`-refLibs {`*`refLib1 refLib2 ...`*`}`	
	Specifies the names of the external libraries that will contain the leaf cell masters, LEF abstract, and technology file. This parameter only accepts library names, not paths. The mapping between the external library name and its path is defined in the `cds.lib` file. Innovus references technology information only from the first library in the list of libraries.

Example

- The following command shows the usage of ITDB style libraries:

```
createLib myDesignLib -referenceTech {techLib}
oaOut -autoRemaster myDesignLib myTopCell layout
```

Related Information

- `createLib`
- `defOut`
- `oaIn`
- `set_cell_binding`
- *Mixed Signal (MS) Interoperability Guide*
  - Design Data Preparation chapter

registerTrigger

```
registerTrigger
[-help]
[{-pre | -post}
commandName
procName]
```

Registers the `-pre` or `-post` callback procedures with the specified commands.

The procedure registered with `-pre` parameter runs before the *`command_name`* parameter is used while the procedure registered with `-post` parameter runs after the *`command_name`* parameter is used.

Note: The arguments passed to *`command_name`* are passed to *`procedure_name`*. If the procedure registered with the `-pre` parameter returns a `TCL_ERROR`, then *`command_name`* is not run.

You can use the `registerTrigger` command after starting the Innovus software.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `registerTrigger` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man registerTrigger`.
`-pre`	Specifies the procedure to be called before the command.
`-post`	Specifies the procedure to be called after the command.
`commandName`	Specifies the command name. The supported command names are `saveDesign`, `restoreDesign`, `oaIn`, and `oaOut`.
`procName`	Specifies the procedure name that is to be registered as callback.

Related Information

- `oaIn`
- `oaOut`
- `restoreDesign`
- `saveDesign`

report_oa_lib

```
report_oa_lib
[-help]
lib_name
[-header]
[-filter {cdsinfo | compression | constraint_groups | property | tech_graph}]
```

Reports information about OpenAccess libraries from within Innovus. You can use this command to query for attributes that are controlled by some of the `setOaxMode` options and the `update_oa_lib` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_oa_lib` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_oa_lib`.
`-filter {cdsinfo | compression | constraint_groups | property | tech_graph}`	
	Specifies the list of information to be included in the report. You can choose from the following filter options: • `cdsinfo` - Reports whether the `cdsinfo.tag` exists in the specified library. • `compression` - Reports the compression level (0 = uncompressed) of the library. • `constraint_groups` - Reports the constraint groups defined in the libraries in the technology graph. Constraint groups that are compatible with the default rule and `NONDEFAULTRULE` requirements will be indicated. • `property` - Reports the property information that was added to the library. The `add_oa_library_property` command is one source of library properties. • `tech_graph` - Reports information about the technology graph from the specified library. If you do not specify a filter option, all the above information is included in the report.
`-header`	Indicates that header information should be displayed. This option is useful if the report is being redirected to a file.
`lib_name`	Specifies the name of the OpenAccess library to be queried.

Example

The following command reports information about the `zambezi45` OpenAccess library along with the header:

```
innovus 18> report_oa_lib zambezi45 -header

;*******************************
; Header Info
;*******************************
; Generated by: Cadence Innovus 18.10-a025_1
; OS: Linux x86_64(Host ID donut)
; Generated on: Mon Feb 12 15:53:07 2018
; Command: report_oa_lib zambezi45


Library Name : zambezi45

cdsinto.tag exists : Yes
```

```
Compression Level : 0

Rapid PDK : No

Library Properties :
#prop name prop type prop value
cdfData AppProp (promptWidth 175 fieldHeight 35 fieldWidth 350 buttonFieldWidth 340 formInitProc nil doneProc nil
parameters nil propList (instDisplayMode "instName" instNameType "schematic" termDisplayMode "none" termSimType
"DC" netNameType "schematic" paramDisplayMode "parameter" paramSimType "DC" paramEvaluate "t nil nil nil nil"
simInfo (nil)))
techLibName StringProp gsclib045

Constraint Groups : 10
Library Name: gsclib045
LEFDefaultRouteSpec (LEFDefaultRouteSpec, default)
validLayers: Metal1 Metal2 Metal3 Metal4 Metal5 Metal6 Metal7 Metal8 Metal9 Metal10 Metal11
validVias:

LEFSpecialRouteSpec
Library Name: gpdk045
LEFDefaultRouteSpec_gpdk045 (LEFDefaultRouteSpec, default)
validLayers: Metal1 Metal2 Metal3 Metal4 Metal5 Metal6 Metal7 Metal8 Metal9 Metal10 Metal11
validVias:

virtuosoDefaultExtractorSetup
virtuosoDefaultSetup
virtuosoDefaultTaper
VLMDefaultSetup
DFM
minPRBoundaryInteriorHaloCG
minProtrusionNumCutCG

Technology Graph:
zambezi45
gsclib045 (attached)
gpdk045 (referenced)
```

Related Information

- setOaxMode
- update_oa_lib
- *Mixed Signal (MS) Interoperability Guide*
  - Technology Data Preparation chapter
  - Static Timing Analysis for Mixed Signal Designs chapter

save_abstract

```
save_abstract
[-help]
[-PGpinLayers layerNameList]
[-cutObs]
[-cutObsMinSpacing]
[-extractBlockObs]
[-extractBlockPGPinLayers layerNameList]
[-lib lib]
[-specifyTopLayer topLayerName]
[-stripePin {top all none}]
[-view view]
```

Generates hierarchical design abstract information for the current block-level design. By default, `save_abstract` creates power and ground pins on the design and creates a single obstruction shape for each routing layer. The output abstract contains the antenna information, if you run antenna verification.

In a bottom-up flow (where the block was not created using the `definePartition` or `partition` command), you normally create a simple OpenAccess abstract that contains:

- one obstruction (`OBS`) shape per routing layer
- signal pins located along the block boundary
- power pins for the top metal layer

You can use the `save_abstract` command after loading a block-level design.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `save_abstract` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man save_abstract`.
`-cutObs`	Creates cut-outs (obstructions) in the blockages for signal pins, power and ground pins, vias, and power and ground stripes. *Default*: Creates a single obstruction shape for each routing layer.
`-cutObsMinSpacing`	Creates cut-outs (obstructions) in the blockages around pins using the minimum width to calculate the spacing.
`-extractBlockObs`	Extracts all obstruction (`OBS`) information from the instantiated `CLASS BLOCK` or `RING` macros and writes out the obstructions to the output OpenAccess file. `OBS` information is extracted for layers that are not already being completely covered under the normal routing layer range and for cut layers between routing layers that are completely covered.
`-extractBlockPGPinLayers layerNameList`	

	Extracts the power and ground pin information from any CLASS BLOCK and CLASS RING macros with pins on layers in the specified layer list and writes out the pin information to the OpenAccess file. You can use metal layer numbers (for example, specify `4` for `metal4)`. Specify a list of layer numbers by leaving a space between each number. For example, to create a list with layers `metal3, metal4,` and `metal5,` specify `-extractBlockPGPinLayers 3 4 5`. *Default*: Does not extract the pin information from any macros.
`-lib`	Specifies the target library name. *Default*: The abstract is written into the library from which the current data was loaded.
`-PGpinLayers` *layerNameList*	
	Specifies the list of layers.
`-specifyTopLayer` *topLayerName*	
	Writes out obstruction information only for layers up to the specified layer. You can use the metal layer number (for example, specify `4` for `metal4`). *Default:* Top routing layer (therefore, writes out obstruction information for all layers)
`-stripePin {top all none}`	Writes out power and ground stripes on the top metal layer as power and ground pins. You can use this parameter in a bottom-up methodology, when you create abstracts for blocks that you will use at the next higher level of a hierarchical design.
`-view` *view*	Specifies the target view name. *Default*: `abstract`

Related Information

- `definePartition`
- `partition`
- `write_lef_abstract`
- *Mixed Signal (MS) Interoperability Guide*
  - Useful Tips chapter

saveOaBlackboxes

```
saveOaBlackboxes
[-help]
-cell list_of_cells
-lib libName
[-view viewName]
```

Saves the specified blackbox abstracts, using the OpenAccess database format. You can use the `saveOaBlackboxes` command at any time after importing the design.

Note that when the design is saved using the `saveDesign` command, the tool automatically saves abstracts for any blackboxes in the design. However, if you are performing pin optimization or manipulating the blockages on a particular macro in the design, you may wish to save the abstract for only that particular macro. The `saveOaBlackboxes` command is useful in such situations.

Parameters

`-help`	Prints a brief description that includes type and default information for each `saveOaBlackboxes` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man saveOaBlackboxes`
`-cell` *`list_of_cells`*	Specifies one or more blackboxes to save. If you specify more than one blackbox, you must enclose them in either double quotation marks or curly braces, and separate them with an empty space.
`-lib` *`libName`*	Specifies the OpenAccess library to use to save the blackbox abstracts. If an OpenAccess library with the same name already exists, the software creates a cell view in that library. If the library does not exist, the software creates the library in the working directory, with the library name and the path name being the same. If a new library is created, the technology file is copied from the technology library to the new library.
`-view` *`viewName`*	Specifies the cell view to save. *Default:* Uses the view name `abstract`

Examples

- The following command saves the blackbox `myBlackbox` in the view `abstract`:
  ```
  saveOaBlackboxes -lib myLib -cell myBlackbox -view abstract
  ```
- The following command saves the blackboxes `myBlackbox1` and `myBlackbox2` in the view `abstract`:
  ```
  saveOaBlackboxes -lib myLib -cell "myBlackbox1 myBlackbox2" -view abstract
  ```
- The following command also saves the blackboxes `myBlackbox1` and `myBlackbox2` in the view `abstract`:
  ```
  saveOaBlackboxes -lib myLib -cell {myBlackbox1 myBlackbox2} -view abstract
  ```

Related Information

- `saveDesign`
- `setOaxMode`

set_cell_binding

```
set_cell_binding
[-help]
[-abstract ]
[-lib list_of_libs]
-cell list_of_cell_patterns
-view list_of_views
```

Changes the layout binding for a cell. This also affects the layout viewing and automatic remastering. When specified with `-abstract` parameter, this instead of affecting the abstract binding replaces the in-memory cell information with the matched cells.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_cell_binding` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_cell_binding`.
`-abstract`	Replaces in-memory cell information for the matched cells.
`-cell` *`list_of_cell_patterns`*	Specifies one or more cell name patterns that are to be replaced.
`-lib` *`list_of_libs`*	Specifies the OpenAccess library that is to be used to look for cell abstracts.
`-view` *`list_of_views`*	Specifies the cellview name.

Example

- The following command looks for the cells in the Innovus database that match the cell pattern and replaces their binding with the specified view found in the library list.

```
set lib_list1 {lib1 lib2}
set_cell_binding -lib $lib_list1 -cell {cell1 cell2 cell3} -view {abc}
```

Related Information

- `oaOut`

setOaxMode

```
setOaxMode
[-help]
[-reset]
[-allowAnalysisOnly {true|false}]
[-allowBitConnection {true | false}]
[-allowTechUpdate {true | false}]
[-bindkeyFile fileName]
[-compressLevel value ]
[-convertTo {polygon|defaultStyle}]
[-cut_pg_rail {true | false}]
[-cutRows {true | false}]
[-displayDrfFile pathName ]
[-displayDrfInLibrary {true|false}]
[-drcFillPurpose {gap_fill|drawing}]
[-encloseQuickAbstractPins {true | false}]
[-fullLayerList {true | false}]
[-fullPath {true | false}]
[-hybridRows {true|false}]
[-instPlacedIfUnknown {true | false}]
[-libCreateMode {none | attach | copy | reference}]
[-locking {true | false | errorIfLocked}]
[-logicOnlyImport {true | false}]
[-merge_trim {true | false}]
[-pinPurpose {true | false}]
[-pushPinConstraint {true | false}]
[-quickAbstractForCustomCells {pcells |stdcells |all | off}]
[-readSystemReservedLayers {all | text | off}]
[-saveCdsFixedVias {new_vias_only | off}]
[-saveMaskDataLocked {true | false}]
[-saveNetVoltage {true | false}]
[-saveRelativePath {true | false}]
[-saveRestoreFile {true | false}]
[-silently_ignore_unsupported_vias {true | false}
[-textPurpose purpose_name]
[-tieNet { tieHighNetName tieLowNetName }]
[-updateMode {true | false | auto}]
[-useVirtuosoBindkey {true | false}]
[-useVirtuosoColor {true | false}]
[-viewSubType {VCE | VXL | none}]
```

Controls aspects of how the Innovus™ Implementation System software reads OpenAccess technology, and creates OpenAccess libraries.

Use the `getOaxMode` command to return the current settings for the `setOaxMode` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setOaxMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setOaxMode`.
`-allowAnalysisOnly {true|false}`	
	Allows analysis only flow. `assembleDesign` is not allowed when `setOaxMode -allowAnalysisOnly` is set to `false` and `setOaxMode -updateMode` is set to `true`, thereby locking the OpenAccess databse. To allow `assembleDesign` to proceed, you must change `setOaxMode -allowAnalysisOnly` to `true`. If you set `-allowAnalysisOnly` to `true` and the Innovus session is in `updateMode` = `true` mode (whether set explicitly or internally on the fly), the `-updateMode` setting is ignored. Note that there may be loss of some semantic information, such as MPPs, while saving the design when `-allowAnalysisOnly` is set to `true`. However, the saved design would be geometrically correct.
`-allowBitConnection {true | false}`	
	Creates databases for instances that only have bit-based terminals, without requiring the corresponding bus terminals. When set to `true`, it saves the connectivity information by bits if the bus connections are not available. When set to false, an error will be issued if the abstract view is missing expected bus terminal information. *Default*: `false`
`-allowTechUpdate {true | false}`	
	Allows the technology database to be updated. The common reason that a technology database would need to be updated is that new vias have been added to the design that are not in a parameterized form. Via masters that are a simple list of rectangles require that a "custom via" be created in the technology database. If the design library had been created with an attached technology, the value should be set to `false`, allowing a reference library technology that is shared by multiple designs to have design-specific vias to be added to it. For design libraries that have been created in reference mode, allowing the technology database to be updated is typically fine but normally it is best to always use paramaterized vias for OpenAccess-based flows. Using `setViaGenMode -symmetrical_via_only true` before any power planning or routing will typically prevent the creation of non-parameterized vias in the flow. When `-allowTechUpdate` is set to `false`, the creation of the OpenAccess database will fail if any "custom vias" would need to be created within the design library. *Default*: `true`
`-bindkeyFile` *fileName*	

	Maps into Innovus-equivalent bindkeys. If the bindkey file is not specified, the Virtuoso default bindkey information is used. *Default*: This is a string value and its default value is " ". For more information, see the "SKILL to TCL Mapping" section in the Importing and Exporting Designs chapter of the *Innovus User Guide*.
`-compressLevel` *value*	Sets the compression level of a library during the library creation process. Cellviews written to a library will follow the compression level of the library and not the current `setOaxMode -compressLevel` setting. Valid values range between 0 to 9. A value of 0 indicates that compression is turned off. *Default*: `1`
`-convertTo {polygon | defaultStyle}`	
	Enables interoperability of the end style of geometries of type path segment between Virtuoso and Innovus. This parameter applies to the diagonal path segments only. If the end-style of a diagonal path segment is `custom`, then the end style is retained as is. If the end style of a diagonal path segment is not `custom`, then if you set the value to `polygon`, Innovus converts the path segment into a geometrically equivalent polygon. If the value specified is `defaultStyle`, then the end style of the path segment is set to `defaultStyle`. In this case, Innovus retains the semantics of the object to be a path segment. **Important**: To ensure that there is no loss of semantic or geometric information, the path segments in Virtuoso -XL and Virtuoso-GXL should use the `custom` end-style on diagonal path segments which can be created using *Create-Shape-Geometric Wire*. *Default*: `polygon`
`-cut_pg_rail {true | false}`	
	Cuts PG rails. *Default*: `true`
`-cutRows {true | false}`	
	Cuts rows against placement obstructions, block halos, or power domain fence minimum gaps. *Default*: `false`
`-displayDrfFile` *pathName*	
	Allows you to specify a specific `display.drf` file that is to be used. It overrides all the searching options and the specified file is used. *Default*: String
`-displayDrfInLibrary {true | false}`	

	Allows the tool to search for the `display.drf` file in the technology library (or complete technology graph in case of ITDB) when set to `true`. If not found, it looks for the file in the current working directory followed by the home directory. When set to `false`, the tool searches for the `display.drf` file first in the current working directory followed by the home directory. *Default*: `false`
`-drcFillPurpose {gap_fill|drawing}`	
	Specifies whether to save DRCFILL shapes to the `gap_fill` or `drawing` purpose. By default, DEF routes with + SHAPE DRCFILL are mapped to the `gapFill` purpose in OpenAccess. However, some OpenAccess tech files do not have this purpose defined so any shapes mapped to the `gapFill` purpose would not be visible in Virtuoso and would not be written out in GDS or OASIS. In such cases, set the `-drcFillPurpose` parameter to `drawing` to save these shapes to the `drawing` purpose. When you do so, these shapes get an additional internal attribute so they can round-trip back to DRCFILL shapes properly. *Default*: `gap_fill`
`-encloseQuickAbstractPins {true | false}`	
	Specifies that shapes enclosing the layout pin shapes should be included as part of the pins during Quick Abstract Inference (QAI) pin modeling. Check the Quick Abstract Inference chapter in the *Mixed Signal (MS) Interoperability Guide* for details. Default: true
`-fullLayerList {true | false}`	
	Imports the complete list of OpenAccess layers, when importing an OpenAccess design. If you specify `false`, only OpenAccess layers defined with `oacMaterial` an type are imported. *Default*: `true`
`-fullPath {true | false}`	
	Creates the `DEFINE` statements in the `lib.def` files using full paths. If you specify `-fullPath false`, the software creates the statements using relative paths. *Default*: `false`
`-hybridRows {true | false}`	
	Reads hybrid rows from the Process Design Kit (PDK). When this parameter is set to `true`, the tool checks whether the sites are abutted horizontally or vertically and calculates the complex site width and height accordingly. *Default*: `false`
`-instPlacedIfUnknown {true | false}`	
	Maps the placement status of instances in the OpenAccess database to the placement status in the Innovus software. When set to `true`, the instances in the OpenAccess database that have an unknown placement status are treated as "placed" in the Innovus software. When set to `false`, instances with an unknown placement status are treated as "unplaced." *Default*: `true`

`-libCreateMode {none | attach | copy | reference}`	
	Specifies the mode in which `saveDesign -cellview` creates a library. `none`: Turns off the ability to have `saveDesign` create a library if it does not already exist. `attach`: Attaches the technology database specified by the technology library to the library that is to be created. `copy`: Copies the technology database specified by `techLib` to the library specified by `libName`, and saves the library to the current working directory. `reference`: Creates a local technology database in the design library that can be modified incrementally. This is the recommended setting to handle the case where the design may have "custom vias" added at some point by power planning/routing. *Default*: `reference`
`-locking {true | false | errorIfLocked}`	
	Specifies whether or not the OpenAccess database will be locked when the design is restored in Innovus. You can specify one of the following options: • `false`: This is the default option. When set to `false`, the tool does not check whether the design cell view is locked before restoring the design. The OpenAccess database can continue to be edited by other processes even while the design is restored in Innovus. **Note**: If the `-updateMode` parameter is set to `true`, the software automatically locks the database, even if the `-locking` parameter is set to `false`. • `true`: When set to `true`, Innovus locks the design cell view during `restoreDesign`, provided that it is not already locked by some other tool. With this setting, other tools cannot edit the OpenAccess database as long as it is locked for the Innovus session but they can still access the database in the read-only mode. To remove the lock, specify the `unlockOaDesign` command or exit the Innovus software. **Note**: If the design cell view is already locked by some other tool before `restoreDesign`, Innovus issues a warning but continues to read in the cell view. • `errorIfLocked`: If you want the tool to be more restrictive, set the `-locking` parameter to `errorIfLocked`. With this option, if the design is already locked while importing or restoring a design, the tool aborts and reports the following error: `**ERROR: (IMPOAX-7995): updateMode/locking is on. Design is already locked by another tool. You are recommended to close this design from another application before proceeding.` *Default*: `false` ⓘ The value of `-locking` cannot be changed while a design is in memory.
`-logicOnlyImport {true | false}`	

	Controls what information the software reads when importing an OpenAccess maskLayout database. When set to `true`, the software only reads the logical (Verilog netlist equivalent) connectivity during the init_design process. When set to `false` (the default value), the software automatically reads physical information (floorplan, placement, routing, etc.) from the database, and updates physical net connectivity for power and ground connections. **Note:** The `-logicOnlyImport` parameter only applies to OpenAccess maskLayout databases. For other types of OpenAccess databases, the software always only reads the logical connectivity. *Default*: `false`
`-merge_trim {true | false}`	
	Merges the trim metal shapes during save when the trim metal gap is less than or equal to the max_gap in microns. *Default*: `true`
`-pinPurpose {true | false}`	
	Writes out pin shapes with the purpose `pin`. When set to `false`, writes out pin shapes with the purpose `drawing`. *Default*: `false`
`-pushPinConstraint {true | false}`	
	Specifies whether or not `saveDesign` or `saveOaBlackboxes` should push an interoperable pin constraint into its corresponding blackbox abstract cellview. **Note:** Interoperable pin constraints refer to pin constraints that are understood by Innovus, Virtuoso, and PVS. *Default*: `false`
`-quickAbstractForCustomCells {pcells | stdcells | all |off}`	
	Trims design shapes of custom cells. By default, this capability is turned on for all types of custom cells. The following settings are allowed for this parameter: • `pcells` - Trims design shapes of only custom parameterized cells (PCells). • `stdcells` - Trims design shapes of only custom standard cells. • `all` - Trims design shapes of all custom cells. • `off` - Does not trim design shapes of any custom cell. **Note**: This feature needs the INVS30 (Innovus Mixed Signal Option) license. If this license is not available, the abstract that gets generated for custom cells will not work properly in Innovus. *Default*: `all`
`-readSystemReservedLayers {all | text | off}`	

	Reads the system-reserved layers from the PDK. The following settings are supported: • `all` - All system-reserved layers are read. • `text` - Only text layer is read. • `off` - No system-reserved layer is read. *Default*: `all`
`-reset`	Resets parameters to their default values. The `-reset` parameter *must* be the first parameter specified. If you specify `-reset` by itself, the software resets all `setOaxMode` parameters to their default values. Any values that do not already match the default value and are reset will be reported. If you specify parameters after `-reset`, the software resets only those parameters to their default values.
`-saveCdsFixedVias {new_vias_only | off}`	
	Controls how fixed/custom vias are created in OpenAccess library when saving a design from Innovus to OpenAccess: • `off` - No cdsFixedVias are created. All fixed/custom vias in the design are saved as oaCustomVia. This is the default option. • `new_vias_only` - Any newly created custom via is saved as a fixed via while any pre-existing custom via in the Incremental Technology Database (ITDB) is reused when saving the design. **Note**: This feature needs the INVS30 (Innovus Mixed Signal Option) license. If this license is not available, the default `off` value will be used and no fixed vias will be created. *Default*: `off`
`-saveMaskDataLocked {true | false}`	
	Sets the shapes with mask coloring (mask values other than 0) to locked in the saved cellview, when set to `true`. By default, `oaOut` and `saveDesign` write out shapes with their mask coloring information in an unlocked state. Some tools treat the unlocked color information as the recommended color. However, for some foundry processes, shapes with unlocked mask coloring information are written to GDSII/Stream as gray (uncolored). In such cases, set the `-saveMaskDataLocked` parameter to `true` to lock the mask coloring of shapes explicitly. **Note**: This option affects only the saving of shapes with non-zero mask coloring values. Review the map file that will be used to create GDSII/Stream output from the OpenAccess cellview to determine if the map file requires shapes to be locked for them to be written out as colored. *Default*: `false`
`-saveNetVoltage {true | false}`	
	Saves the minimum and maximum net voltage across all loaded timing views to every net. When this parameter is set to `true`, `write_db` writes voltage information to the voltage attribute of oaBitNet. This is useful for Virtuoso, and other tools that may read the cellview, to understand the minimum and maximum net voltage without needing to interpret the CPF or .lib file to get that information. Roundtrip of the information into Innovus is not dependent on this setting. *Default*: `false`
`-saveRelativePath {true | false}`	

	Defines the current working directory (`$cwd`) in the configuration file as ".". When set to `true`, a design can be restored, even if the directory path of the design file is changed. *Default*: `true`
`-saveRestoreFile {true | false}`	
	Creates the `view.inn` file when the `saveDesign` command is run with the `-cellview` or `-view` arguments. The `view.inn` file is a file that can be sourced to read the cellview that was saved into a new Innovus session. The file name will be the specified view name with the `.inn` suffix added. When `saveDesign` is used in the *`sessionName`* style (without `-cellview` or `-view` specified), the `inn` file will always be created. In the *`sessionName`* style, the `inn` file will have the name specified by the user even if the view name has the `.inn` omitted (if the `.inn` suffix is not specified, the file and view names will be the same). Set the `-saveRestoreFile` parameter to `true` to create an `inn` file that can be sourced. The use of the `inn` file makes it easier to write scripts that are more generic and work across multiple designs by depending only on the view name and not the design library or cell name. *Default*: `false`
`-silently_ignore_unsupported_vias {true | false}`	
	Specifies whether `restoreDesign` should issue messages about vias that are not supported in Innovus when a Virtuoso design is read in. *Default*: `false`
`-textPurpose` *`purpose_name`*	
	Specifies the purpose name that will be used when text labels are written. By default, any text label created in Innovus (typically via the `add_text` command) is automatically mapped to the `drawing` purpose. Use the `-textPurpose` parameter to map the text labels to a user-defined purpose. **Note**: `setOaxMode -textPurpose` *`purpose_name`* overrides the default `drawing` purpose on output. *Default*: `drawing`
`-tieNet {`*`tieHighNetName tieLowNetName`*`}`	
	Specifies the global tie high and tie low net names to be created in the OpenAccess database. The nets are used to represent the Verilog `1'b1/1'b0` connections to the instance ports. Typically, `tie1` and `tie0` are used as tie high and tie low net names, respectively. If these names exist in the design locally in any module, then the global names should be changed. *Default*: `""`
`-updateMode {true | false | auto}`	
	Specifies whether or not to save or restore an OpenAccess design in update mode. If the `-updateMode` parameter is set to `true`, `restoreDesign -cellview` processes parameterized cells (PCells) and other custom objects correctly in the Innovus database. For information about generating/regenerating the PCell cache, refer to the "Preparing the IP Library" section in the Technology Data Preparation chapter of the *Mixed Signal (MS) Interoperability Guide*. `-updateMode true` also ensures that `saveDesign -cellview` updates the Innovus-modified portion of an

OpenAccess database without making changes to PCells or other custom objects.

⚠ In the `setOaxMode -updateMode true` mode, the tool locks the source cellview for the duration of the Innovus session.

The `updateMode` parameter provides the ability to import an OpenAccess database created from Virtuoso that contains objects which are not understood by Innovus. Complete the digital part in Innovus and update only the objects modified by Innovus in the original OpenAccess database. If your design has any of the following objects, they would only be visible (not editable) in Innovus:

- PCells
- Modgen cells
- Fig-groups
- PCell vias
- RODs (Example, Multi-Part Paths, and so on)
- User defined properties on design objects such as nets, shapes (`lxStickyNet`), instances, abstract views, intermediate hierarchical instances, and so on.

You won't be allowed to edit or modify these objects.

On using this parameter, the property on shapes (for example, `lxStickyNet`) and nets created in VLS-XL / GXL would be kept as-is.

In case of designs where parameterized vias are used, Innovus understands these parameters and evaluates them on-the-fly to create an exact shape and orientation of the cuts.

If the `-updateMode` parameter is set to `auto` and the cellview contains objects that are not supported in Innovus, the software internally enables update mode with a message such as follows:

```
INFO: Objects have been found in the cellview requiring updateMode to be enabled so that
there is no loss of semantic information when the cellview is saved.
```

Note that enabling update mode can affect `saveDesign` performance.

Default: `false`

ⓘ The value of `-updateMode` cannot be changed while a design is in memory. If this parameter has been set to `true` before importing an OpenAccess design into Innovus, you cannot convert `-updateMode` to `false` in any new Innovus session.

ⓘ The on-disk OpenAccess cellview is locked between the restore and save steps when `-updateMode` is set to `true`.

ⓘ In `-updateMode auto`, the lock is applied at the time that `oaIn` or `restoreDesign` starts and released at the end of the command if no objects were encountered that required updateMode to be enabled.

`-useVirtuosoColor {true|false}`

	Allows you to display colors and stipples for the design layers (including full custom layers). Using this parameter, Innovus utilizes the same colors and stipples that you would see if you open the design in Virtuoso. *Default*: `false`
`-useVirtuosoBindkey {true|false}`	
	Allows you to use Virtuoso bindkeys. *Default*: `false`
`-viewSubType {VCE | VXL | none}}`	
	Indicates which Virtuoso application should be used for viewing the layout cellviews in the saved OpenAccess database. When set to `VCE`, cellviews are opened in Virtuoso Chip Editor. When set to `VXL`, cellviews are opened in Virtuoso XL. When set to `none`, it indicates that the OpenAccess database can be edited by any editor within Virtuoso platform based on its license availability. *Default*: `VXL`

Examples

- The following command instructs the Innovus software to create the `DEFINE` statements using full paths:
  ```
  setOaxMode -fullPath true
  ```
- The following command cuts rows against placement obstructions, block halos, or power domain fence minimum gaps:
  ```
  setOaxMode -cutRows true
  ```
- The following command resets the `-fullPath` parameter to its default value:
  ```
  setOaxMode -reset -fullPath
  ```
- The following command resets all `setOaxMode` parameters to their default values:
  ```
  setOaxMode -reset
  ```
- The following command searches for the `mydisplay.drf` file irrespective of the value of the `-displayDrfInLibrary` parameter:
  ```
  setOaxMode -displayDrfFile /servers/oastore/mydisplay.drf
  ```
- The following command sequence enables the `saveDesign` command to write out an `inn` file to enable loading the saved database into a new session without needing to know the design library or cell names:
  ```
  setOaxMode -saveRestoreFile true
  saveDesign -cellview "designLib top placed"
  ```

  In addition to saving the cellview `"designLib top placed"`, the tool creates an `inn` file named `placed.inn` in the current working directory. This file contains the command `restoreDesign -cellview "designLib top placed"`
  To load the design cellview that was saved in a new session, use:
  ```
  innovus -init placed.inn
  ```

Related Information

- `add_text`
- `assembleDesign`
- `getOaxMode`
- `restoreDesign`
- `saveDesign`
- `saveOaBlackboxes`
- `setViaGenMode`
- `oaOut`
- `unlockOaDesign`
- *Mixed Signal (MS) Interoperability Guide*
  - Design Data Preparation
  - Quick Abstract Inference
  - Working with Vias in OpenAccess
  - Using NanoRoute To Route a Design from Virtuoso
  - Chip Finishing and ECO Flows
  - Useful Tips
- Importing and Exporting Designs chapter in the *Innovus User Guide*

unlockOaDesign

```
unlockOaDesign
[-help]
lib
cell
view
```

Unlocks the specified OpenAccess design.

A design cannot be opened if `setOaxMode -locking` is set to `true`, and the design is currently locked. You can use the `unlockOaDesign` command to free the lock so that other tools can access and edit the database. You also can use the command to override the lock, if the design is incorrectly locked.

You can use the `unlockOaDesign` command at any time.

Parameters

cell	Specifies the name of the top cell to unlock.
-help	Outputs a brief description that includes type and default information for each `unlockOaDesign` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man unlockOaDesign`.
lib	Specifies the name of the OpenAccess library in which the design was saved.
view	Specifies the name of the cell view to unlock.

Example

- The following command unlocks the view `layout` in the cell `dtmf_chip` in the library `designLib`:
  ```
  unlockOaDesign designLib dtmf_chip layout
  ```

Related Information

- `setOaxMode`

update_oa_lib

```
update_oa_lib
[-help]
lib_name
[-compress_level value]
[-create_cdsinfo_tag]
[-tech_attach_to_reference]
```

Updates the specified OpenAccess library from within Innovus. You can use this command to:

- Convert the specified library to use a tech reference from the attached tech so that the non-parameterized or custom vias can be written to it. Saving non-parameterized vias to OA require write permission to OA tech so that custom viadef entry can be created. If attached technology is read-only and OpenAccess design has non-parameterized vias, you should run `update_oa_lib` with -tech_attach_to_reference command line option before invoking `saveDesign`. The tool is then able to save the design and does not throw an error because it can now make non-parameterized vias entry to local tech in the converted library.
- Add a cdsinifo.tag to legacy libraries created by earlier versions of Innovus.
- Modify the compression level for the library.

If you run the `update_oa_lib` command on a library that is not in attached mode, a warning is displayed. However, the script continues to run. This is useful if you want to make sure that libraries are writable for new vias and the `saveDesign` command does not fail.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `update_oa_lib` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man update_oa_lib`.
`-compress_level` *value*	Sets the compression level of the specified library. New cellviews written to the library follow the specified compression level. However, existing cellviews in the library are not modified. Use the oazip utility to modify the existing cellviews in the library. Valid compression level values range from 0 to 9. A value of 0 indicates that compression is turned off.
`-create_cdsinfo_tag`	Adds a `cdsinfo.tag` file to the specified library.
lib_name	Specifies the name of the library to be updated.
`-tech_attach_to_reference`	Converts the specified library to use a tech reference from the attached tech so that new custom via definitions can be added to the library.

Examples

- The following command sets the compression level of the `lib1` library to 9. However, it does not change the compression of the existing cellviews in the library:
  ```
  update_oa_lib lib1 -compress_level 9
  ```
  The tool displays warning such as the following:
  ```
  Changed library compressLevel from '0 (uncompressed)' to '9'. Cellviews within the library are not affected
  by changing the library's compressLevel. To update the cellviews, use the oazip utility.
  ```
- The following command turns off the compression for any newly created cellviews in `my_lib` library, which previously is at compression level `1`:
  ```
  update_oa_lib my_lib -compress_level 0
  ```

Related Information

- `setOaxMode`
- *Mixed Signal (MS) Interoperability Guide*
  - Static Timing Analysis for Mixed Signal Designs chapter

write_lef_library

```
write_lef_library
[-help]
[filename]
[-all_auto_generated_via | -macro_only ]
[-lef_layer_order]
[-macro_only [-no_property]]
[-tech_only | -macro_only]
[-tech_only | -macro_list string]
```

Writes the LEF library data (composed of technology data and macros) that is present in the Innovus database.

The `write_lef_library` command facilitates comparisons between LEF-based input and OpenAccess-based input. LEF allows cases where `VIARULE GENERATE` has `WIDTH` specified for either the top or bottom layer. However, such cases are not supported by the OpenAccess tech. To make the comparison between LEF-based and OpenAccess-based input easier, `write_lef_library` dumps `"WIDTH 0 TO 1000 ;"` for any `VIARULE GENERATE` that does not have WIDTH specified in the original LEF.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_lef_library` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_lef_library`.
`fileName`	Specifies the output LEF file name.
`-all_auto_generated_via`	
	Includes all auto generated vias from the `generateVias` command.
`-lef_layer_order`	Writes out only the layers specified in the lefLayerOrder constraint.
`-macro_list` *`string`*	
	Writes the macros specified in the macro list.
`-macro_only`	Writes only the macro data to the LEF file.
`-no_property`	Writes data in the LEF file without any properties. This option can be used only with the `-macro_only` option.
`-tech_only`	Writes only the technology data to the LEF file.

Examples

- The following command writes the technology data into the `tech.lef` file:
  ```
  write_lef_library tech.lef -tech_only
  ```
- You can set the LEF version to 5.7 or 5.8 using `lefDefOutVersion`:
  ```
  set lefDefOutVersion 5.8
  ```

Related Information

- `generateVias`
- `write_oa_techfile`
- *Mixed Signal (MS) Interoperability Guide*
  - Technology Data Preparation chapter
  - Design Data Preparation chapter
  - Working with Vias in OpenAccess chapter

write_oa_techfile

```
write_oa_techfile
[-help]
[fileName]
[-include_rules]
[-process_node coord]
```

Writes out a DFII style ASCII technology file that can be given to the `techLoadDump` executable to create an equivalent OpenAccess technology database.

This command is used to create and verify OpenAccess technology files that match LEF technology files. By default, `write_oa_techfile` writes out only DRC information and no LDRS or VIA information.

The expected flow for new processes is as follows:

1. Read a LEF file into Innovus with `init_design`.
2. Use `write_oa_techfile` to write out the tech (`.tf`) file.
3. Run `techLoadDump` using the `.tf` file.
4. Use `lef2oa -pnrLibDataOnly` to create the LDRS and vias.

For more information, refer to the chapter "Technology File Layer Attributes" of the *Virtuoso Technology Data ASCII Files Reference*.

Parameter

`-help`	Outputs a brief description that includes type and default information for each `write_oa_techfile` parameter. For a detailed description of the command and all of its parameters, use the man command: man `write_oa_techfile`.
`fileName`	Specifies the output ASCII file name.
`-include_rules`	Includes LDRS and VIA information in the output tech file. This option should be used only if the LEF default and NDRs do not utilize fixed vias.
`-process_node` *`coord`*	Specifies a process node value in nm. When specified, `write_oa_techfile` creates a processNode entry in the techfile for processes using the MASK construct. For example, if you specify `-process_node 20` to indicate a 20nm process, the following entry is included in the control() section of the techfile: `processNode(0.020)` If `-process_node` is not specified and the `write_oa_techfile` command detects MASK constructs on any layer, it returns an error.

Examples

- The following set of commands updates an OpenAccess technology file with any missing information from a LEF technology file:

```
#Initialize a design using LEF files
write_lef_library -tech_only lef.lef  #A LEF file in canonical
                                      #sorted order for easy
                                      #comparison with diff
write_oa_techfile lef.tf              #A DFII ASCII tech file
                                      #that can be given to
                                      #techLoadDump executable
#Edit existing OA tech file to add any missing information in
#the lef.tf file
#Initialize a design using the new OA tech. Then:
write_lef_library -tech_only oa.lef #A LEF file in canonical
                                    #order derived from OA tech
#Compare the two LEF files. Any differences need to be fixed
#or explained.
diff lef.lef oa.lef
```

Related Information

- write_lef_library
- *Mixed Signal (MS) Interoperability Guide*
  - Technology Data Preparation chapter
  - Routing Constraint Interoperability chapter

23

Partition Commands

- add_connection_to_neighbor_partition
- addNetToNetGroup
- addPinToPinGroup
- alignPtnClone
- assembleDesign
- assignIoPins
- assignPtnPin
- changeBBoxMasterFromR0
- changeBBoxMasterToR0
- checkHierRoute
- checkPinAssignment
- clonePtnObj
- commit_pushdown_eco
- comparePinAssignStatistics
- connectMacroFeedthrough
- convertBlackBoxToFence
- convertFenceToBlackBox
- create_partition_wrapper
- create_pushdown_eco
- createNetGroup
- createPinBlkg
- createPinGroup
- createPinGuide
- createPtnCut
- createPtnFeedthrough
- definePartition
- deleteAllPtnCuts
- deleteAllPtnFeedthroughs
- deleteNetFromNetGroup
- deleteNetGroup
- deletePartition
- deletePinBlkg
- deletePinFromPinGroup
- deletePinGroup
- deletePinGuide
- deletePtnCut
- editPin
- fix_feedthru_assign
- flattenCoverCell

- flattenPartition
- get_ptn_fplan_mode
- getBlackBoxArea
- getClonePtnOrient
- getPinAssignMode
- getPinConstraint
- getPtnPinStatus
- handlePtnArealo
- hier_clock_route
- hiliteFeedthroughNets
- insertPtnFeedthrough
- legalizePin
- loadBlackBoxNetlist
- loadPtnPin
- merge_hierarchical_def
- partition
- partitionPushSpecialWires
- pinAlignment
- pinAnalysis
- place_cells_at_center_for_feedthrus
- place_partition_clone
- push_ptn_network
- recreatePtnCellBlockage
- report_partitions
- reportPinAssignStatistics
- reportTwoPinChain
- reportUnalignedNets
- resizeBlackBox
- saveBlackBox
- savePartition
- savePtnPin
- selectPtnPinGuide
- set_ptn_fplan_mode
- setClonePtnOrient
- setPinAssignMode
- setPinConstraint
- setPromotedMacroPin
- setPtnPinStatus
- setPtnPinUSE
- showPtnWireX
- specifyBlackBox
- specifyPartition
- undo_uniquify_partition

- uniquify_partition
- unloadPtnPin
- unsetPinConstraint
- unspecifyBlackBox
- write_generalized_feedthru_paths
- changeCurrentDesign
- highlightSyncObj
- createBusNetGroup
- check_bus_guides
- write_net_groups_topo

add_connection_to_neighbor_partition

```
add_connection_to_neighbor_partition
[-help]
[-dictFile file_name]
```

Adds a partition pin on the abutted edge of unique abutting partitions to create a 2-partition pin scenario for target pins. This helps to avoid floating pins on abutted edges. The `add_connection_to_neighbor_partition` command targets the following pins:

- Input partition pins with have tie definition and a GNC/UPF to resolve the tie net to a supply
- Floating output partition pins

Note: The target pins may or may not have any load/driver inside the partition.

This command should be used after the pins have been assigned for the target partition and tie cells have been specified using the `setTieHiLoMode -cell` command.

Parameters

`-dictFile` *`file_name`*	Specifies the name of the file in which the TCL dictionary is written. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `add_connection_to_neighbor_partition` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man add_connection_to_neighbor_partition`

Example

- The following command adds partition pins on abutting partitions and reports the number of new partition pins and tie cells in a table.
  `add_connection_to_neighbor_partition`

```
-------------------------------------------------------------------------------------
|    Type     | Total Pins | Pins placed in abutted edge | New partition pins added |
-------------------------------------------------------------------------------------
|   Tie-low   |          4 |                           4 |                        4 |
|   Tie-hi    |          4 |                           4 |                        4 |
| Floating(I) |         16 |                          16 |                        - |
| Floating(O) |         16 |                          16 |                       16 |
-------------------------------------------------------------------------------------
```

Related Information

- Partitioning the Design chapter in the *Innovus User Guide*

addNetToNetGroup

```
addNetToNetGroup
[-help]
netGroupName
-net {netName | netNameList}
```

Adds a net to specified net group that was created with the `createNetGroup` command. A net can be a single net, bus name, or segment of a bus. You can use this command after defining a net group.

Note: If a net is a part of a net group, the net cannot belong to any other net group.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `addNetToNetGroup` parameter. For a detailed description of the command and all of its parameters, use the man command: `man addNetToNetGroup`
netGroupName	Specifies the name of the created net group. *Data_type*: string, required
`-net {`*netName* `|` *netNameList*`}`	
	Specifies the name of an individual net, bus, or segment of a bus. You can use wildcards (*?) with this parameter. *Data_type*: string, required

Examples

- The following command adds net ENOUT to net group bundle1:
  ```
  addNetToNetGroup bundle1 -net ENOUT
  ```
- The following command adds the entire bus cpua to net group bundle2:
  ```
  addNetToNetGroup bundle2 -net cpua
  ```
- The following command adds only bus segments 2 through 10 to net group bundle1:
  ```
  addNetToNetGroup bundle1 -net cpua[2:10]
  ```

Related Topics

- `createNetGroup`
- `deleteNetGroup`
- `deleteNetFromNetGroup`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Setting Pin Constraints
  - Net Group

addPinToPinGroup

```
 addPinToPinGroup
[-help]
[-cell cellName]
-pinGroup pinGroupName
-pin {pinName | pinNameList}
```

Adds a module port to specified pin group that was created by `createPinGroup` command. You can use this command after defining a net group.

Note: You can use this command for the top-level also.

Parameters

`-cell cellName`	Specifies the name of a module. This parameter is not required for top-level ports. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `addPinToPinGroup` parameter. For a detailed description of the command and all of its parameters, use the man command: `man addPinToPinGroup`
`-pinGroup pinGroupName`	
	Specifies the name of the created pin group. *Data_type*: string, required
`-pin {pinName | pinNameList}`	
	Specifies the name of a module port. You can specify wildcards (*?) with this parameter. *Data_type*: string, required

Example

- The following command adds the module SH17's port RASN to pin group pin17:
  ```
  addPinToPinGroup -cell SH17 -pinGroup pin17 -pin RASN
  ```
- The following command adds the top-level port ADDER to pin group PIN_GRP_TOP
  ```
  addPinToPinGroup -pinGroup PIN_GRP_TOP -pin ADDER
  ```

Related Information

- `createPinGroup`
- `deletePinGroup`
- `deletePinFromPinGroup`
- `Partitioning the Design` chapter in the *Innovus User Guide*
  - Setting Pin Constraints
  - Net Group

alignPtnClone

```
alignPtnClone
[-help]
[ptnName]
[-outFile filename]
[-checkOnly | -snapAllCorners ]
[[-stripeOffset ] | [[-pgHGrid ] [-pgVGrid ] [-pgLayer {layeridList}]
[-updateUserGrid ] [-snapAllCorners ] [-skipFinFETGrid ] [-includeStdCellPinLayers {layeridList}]]  |
[[-symmetryPatternReference string
[-symmetryPatternTarget string [ [-targetOrientation {R0 R90 R180 R270 MX MX90 MY MY90}]
[-targetOrigin {x y}] ]]]] |
[-symmetricOrientation ]]
[-pgLayer {layeridList}
[-pgNet pgNetName | -pgCenterLine float]]
[-pgPitch float [-pgCenterLine float]]
```

Aligns partition clones with the master partition on a power mesh and with routing tracks. You can use this command after specifying a master partition and clone partition(s). This will avoid off-track pins in clones and avoid potential DRC violations due to misalignment of power meshes inside clone.

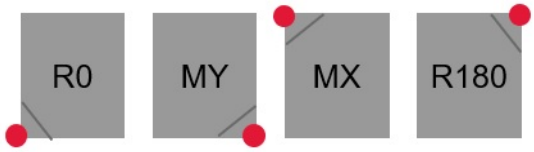
Note: The power mesh must be regular for `alignPtnClone` to work correctly.

The `alignPtnClone` command detects and highlights any gap or overlap of fence created in an abutted floorplan.

Parameters

`-checkOnly`	Specifies that the design should not be modified. When enabled, it only reports messages. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `alignPtnClone` parameter. For a detailed description of the command and all of its parameters, use the man command: `man alignPtnClone`
`-includeStdCellPinLayers {layeridList}`	Specifies the layer(s) of standard cell pins that should be aligned in addition to the user-specified pin layers. *Data_type*: string, optional
`-outFile filename`	Specifies the name of the output file in which to save the master and clone alignment report with details. **Note:** The `alignPtnClone` command supports tracks with multiple definitions. The `-outFile` parameter reports the details of multi-track definitions. *Data_type*: string, optional
`-pgCenterLine float`	Specifies the center-line of the first virtual symmetric pg stripe (in microns). This option can be repeated. *Data_type*: float, optional
`-pgHGrid`	Provide information about horizontal PG grid. *Data_type*: bool, optional
`-pgLayer {layeridList}`	Specifies layer(s) for the pg stripe. This option can be repeated. *Data_type*: string, optional
`-pgNet pgNetName`	Specifies name of the pg stripe net name. This option can be repeated. *Data_type*: string, optional
`-pgPitch float`	Specifies pitch of the pg stripe net (in microns). This option can be repeated. *Data_type*: float, optional
`-pgVGrid`	Provide information about vertical PG grid. *Data_type*: bool, optional
ptnName	Specifies name of the partition. **Note:** If the partition name is not specified, the command is run on all partitions. *Data_type*: string, optional

`-skipFinFETGrid`	Disables alignment with the FinFet grid. *Data_type*: bool, optional
`-snapAllCorners`	Snaps all corners of master and its clone partition to the user grid. **Note:** The `-snapAllCorners` parameter fixes partition overlaps. *Data_type*: bool, optional
`-stripeOffset`	When specified ensure backward compatibility and and enables the use of obsolete `-snap`, `-track`, `-useUserGrid`, `-skip_row`, and `-layer` parameters **Note:** The `alignPtnClone` command supports advanced node fully abutted designs. It moves the clone partition for aligning with the master with all allowed routing tracks by default. It snaps clones such that the pins are never off grid. Consequently, the `-snap`, `-track`, `-useUserGrid`, `-skip_row`, and `-layer` parameters are obsolete. Even though the use of `snap`, `-track`, `-useUserGrid`, `-skip_row`, and `-layer` parameters is supported in this release using the -`stripeOffset` parameter for backward compatibility, the `-stripeOffset` parameter itself is obsolete and will be removed in a future release. To avoid warnings and to ensure compatibility with future releases, remove the obsolete parameters from your script. *Data_type*: bool, optional
`-symmetricOrientation`	Checks the orientation between each pair of partitions and fixes the symmetry by finding the optimum symmetric orientation of clones. **Note:** If Innovus suggests changing the orientation of a partition, it indicates it as (diff) in the report for quick identification. **Note:** The `-symmetricOrientation` parameter can find orientations based upon nested hierarchy. If the `alignPtnClone` command finds no clone connectivity, it can reports the expected orientations and supports nested designs. *Data_type*: bool, optional
`-symmetryPatternReference` *string*	Specifies the names of the partitions that are used as a reference set to maintain a symmetrical pattern for the other set of clone partitions in the group. **Note:** The first partition in the list is considered as the seed partition from which all the relative locations and orientations are calculated for the rest of the partitions in the list. *Data_type*: string, optional
`-symmetryPatternTarget` *string*	Specifies the names of the partitions that are used as a target set to maintain a symmetrical pattern relative to the reference set of partitions in the group. **Note:** • The number of partitions and their order must be the same as that specified with the `-symmetryPatternReference` parameter. • The current location and orientation of the seed partition is used to relatively move the other partitions in the list. The first partition in the list is considered as the seed partition from which all the relative locations and orientations are calculated for the rest of the partitions in the list. • You can specify master partitions in the target partitions set list. However, master partitions can only have R0 orientation. *Data_type*: string, optional
`-targetOrientation {R0 R90 R180 R270 MX MX90 MY MY90}`	Specifies the new orientation of the seed partition of the target partition set (`-symmetryPatternTarget`). The specified orientation overrides the current orientation of the seed partition and enables you to maintain the relative orientation of the different pairs of clones. **Note:** • The seed partition is the first partition specified with the `-symmetryPatternTarget` parameter from which all the relative locations and orientations are calculated for the rest of the partitions in the list. • The orientation of the master partitions cannot be changed. *Data_type*: enum, optional

`-targetOrigin {x y}`	Specifies the new origin of the seed partition of the target partition set (`-symmetryPatternTarget`). The specified location overrides the current location of the target seed partition.  **Note:** • It does not let the target seed partition go outside the core boundary. • The seed partition is the first partition specified with the `-symmetryPatternTarget` parameter from which all the relative locations and orientations are calculated for the rest of the partitions in the list. *Data_type*: point, optional
`-updateUserGrid`	Updates the user grid with the new values. *Data_type*: bool, optional

Example

- The following command aligns the partition clones with the master partition and displays a report:
  `alignPtnClone`

```
================================================================================
                alignPtnClone Summary
================================================================================
Partition                   | Orientation |   Status   | Moved-by(X) | Moved-by(Y)
================================================================================
MASTER_CLONE(I_MC_1)        |             |    Aligned |             |             |
+I_MC_0                     |     MY      |    Aligned |             |             |
MC(I_A/A_MC)                |             |    Aligned |             |             |
+I_B/B_MC                   |     R0      |    Aligned |     -0.2000 |      0.0000 |
K(k4)                       |             |    Aligned |             |             |
+k3                         |     MY      |    Aligned |      0.2000 |     -0.4450 |
+k2                         |     R0      |    Aligned |      0.0000 |      0.0600 |
+k1                         |     R90     | Infeasible |             |             |
b(b1)                       |             |    Aligned |             |             |
+b3                         |     R0      |    Aligned |      0.0000 |      0.1800 |
+b2                         |     R0      |    Aligned |      0.0000 |     -0.3250 |
c(c1)                       |             |    Overlap |             |             |
S(s1)                       |             |    Overlap |             |             |
================================================================================
```

- The following command snaps clones of module `eug` to Master's power grid:
  `alignPtnClone eug`
- The following command aligns with all allowed tracks and automatic detection of pg layers, nets, and pitch of pg grid
  `alignPtnClone eug -pgHGrid -pgVGrid`
- The following command aligns with all tracks and automatic detection of pg nets and pitch of pg grid
  `alignPtnClone eug  -pgHGrid -pgLayer { 2 4 6 } -pgVGrid -pgLayer { 3 5 7}`
- The following command aligns with all tracks and automatic detection of pg pitch of pg grid
  `alignPtnClone eug -pgHGrid -pgLayer { 2 4 6 } -pgNet VSS -pgLayer {4 6} -pgNet VDD -pgVGrid -pgLayer { 3 5 } -pgNet VSS -pgLayer {5 7} -pgNet VDD`
- The following command aligns with all allowed tracks and specified pg grid
  `alignPtnClone eug -pgHGrid -pgLayer { 2 4 6 } -pgNet VSS -pgPitch 1.48 -pgNet VDD -pgPitch 1.28 -pgVGrid -pgLayer { 3 5 7 } -pgNet VSS -pgPitch 1.48 -pgNet VDD -pgPitch 1.28`
- The following command updates the user grid and simulates symmetric PG stripe pattern:
  `alignPtnClone -updateUserGrid -pgVGrid -pgLayer M4 -pgPitch 5.76 -pgCenterLine 0.0 -pgHGrid -pgLayer M5 -pgPitch 46.08 -pgCenterLine 1926.72 -checkOnly`

Related Information

- `checkPinAssignment`
- `partition`
- Partitioning the Design chapter in the *Innovus User Guide*

assembleDesign

```
assembleDesign
[-help]
[-keepPGPinGeometry]
[-keep_block_halo]
[-outFile fileName]
[-skipCells] [-skipComponents] [-skipNets]
[-skipSpecialNets]
{{[-topDir dirname [-mmmcFile mmmcFile] [-cpfFile cpfFile]]
   {[-blockDir dirname [[-fe] | [-fplan]]][-blockData {defFileName netListFileName} | {-fplan|-fe}][-noDefMerge]}}
      | {[-topDesign {topLib topCell topView} [-mmmcFile mmmcFile] [-cpfFile cpfFile]]
        {[-block {blockLib blockCell blockView} | -blockCell blockCell]
   [-allTimingBlocks [-exceptBlocks {list of blocks}] [-reportOnly]]}}}
[-fe | -fplan | -noDefMerge ]
[-keepPinGeometry]
```

Brings back specified block data to the top-level design for chip assembly. The top-level design should instantiate each block as a hard macro whose size matches the design size used for each block when it was implemented. After assembleDesign has completed, the result is a a full-chip design with one or more blocks merged at the location where each block was previously instantiated as a hard macro. You should run this command from the directory with the full chip-level floorplan for the top-down hierarchical flow.

There are two main modes of operation - full and incremental.

- Full mode: When operating in full mode, a session is initiated with the `assembleDesign` command which first loads the top-level design with blocks represented as hard macros then each block is merged within the full-chip context.
- Incremental mode: When operating in incremental mode, a top-level design is loaded prior to calling `assembleDesign` which then merges the contents of one or more blocks.

Notes:

- The `assembleDesign` command supports designs saved in the Innovus database format as well as OpenAccess designs.
- The `assembleDesign` command supports the incremental capability which handles master/clones partitions at different levels of hierarchy and supports nested partitions. The incremental `assembleDesign` capability enables partitions to be brought back in one session, thus, avoiding the need to run multiple sessions for bringing back all the data of a design. You can call the incremental `assembleDesign` for a block multiple times in the same session without having to partition it.
- The `assembleDesign` command supports mixed-origin design. That is, all partitions can have a design origin at the center, lower-left, any location inside the design boundary, or any location outside the design boundary. You can use this command after implementing blocks and top-level designs.
- The `assembleDesign` command preserves Interface Logic Model (ILM) information by automatically bringing back ILMs from block level design to full-chip level design. For DEF data, ILMs can only be brought back if the -blockDir option is specified.
- The `assembleDesign` command supports mixed data types. While specifying partition blocks, you can have different types of data for different partition blocks.
- The `assembleDesign` command supports partitions with flexmodels. It re-assembles partition blocks with flexmodels back to the top-level design.
- The `assembleDesign` command can read design information from the global init_* variables saved in the *.global file.
- The `assembleDesign` command internally calls the `applyGlobalNets` command that applies or restores the global net connectivity rules to the design and creates the necessary connections between instances and global nets. Consequently, the `assembleDesign` command automatically generates a database where the global nets are already connected. This helps to ensure that all instance are correctly tied to PG nets.
- The `assembleDesign` command internally checks color issues and uses the shift-mask functionality to assign correct colors to objects. As a result, at the end of `assembleDesign`, the objects are already given the mask numbers (colorized) and the design has less color violations.
- The `assemble_design` command compares the properties of each block and top and uses a specific resolution for each property.
- The `assembleDesign` command (DEF based) supports multi-threading. It honors the settings of the `setMultiCpuUsage -localCpu` parameter that is used to specify the number of CPUs on the local machine and is required for multi-threading.

  Note: This capability is not supported by the `-fe` based `assembleDesign`.

During `assembleDesign` all rows are cut at the top and only the rows from inside the block are brought at the time of assembly. The `checkPlace` command does not report any false placement violations. Only in cases where partitions are rotated R90 (like vertical rows) are the cells marked as fixed.

A summary report is generated at the end of each `assembleDesign` run which helps in debugging issues. The generated report includes information like the runtime for `assembleDesign`, number of partitions and their instance count.This report is useful in identifying potential issues in the flow.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `assembleDesign` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man assembleDesign`
`-allTimingBlocks`	Enables automatic detection and assembly of blocks with digital content. The `-blockCell` option can be used to manually force blocks to be assembled, and `-ignore_timing_blocks` option can be used to exclude cells from being assembled as it overrides to the default list of blocks to be assembled. *Data_type*: bool, optional
`-block` *blockLib blockCell blockView*	
	Specifies the block-level library, cell, and view names respectively for a design that was saved into an OpenAccess database. This parameter is used when bringing back block data for the partition from an OpenAccess database. **Note:** If you specify more than one block-level design, specify the `-block` option separately for each block. For example: `-block libForOA TDSP_CORE ptnView1 -block libForOA TDSP_ARB ptnView1` *Data_type*: string, optional
`-blockCell` *blockCell*	Specifies the cell name for the block cell. *Data_type*: string, optional
`-blockData` *defFileName netlistName*	
	Specifies the DEF file name and the Verilog netlist file name. Use this parameter if the Verilog netlist file name or the DEF file name is different from the cell or module name. You must provide the full path (absolute or relative) with the file names. This is distinct from using the -blockDir option in that only a DEF file and Verilog netlist are required as opposed to a database saved with the `saveDesign` command. If a design contains multiple instantiations of a block, -blockData only needs to be specified once per master. **Note:** If you specify more than one block-level design, specify the `-blockData` option separately for each block-level DEF file and Verilog netlist file. For example: `-blockData block_a.def block_a.v -blockData block_b.def block_b.v` gzipped files are supported. *Data_type*: string, optional
`-blockDir` *dirname*	Specifies the path to the block-level design(s). This directory should be generated by the `saveDesign -def` command. **Note**: If you specify more than one block-level design, specify the -blockDir option separately for each block-level directory. For example: `-blockDir block_a.enc.dat -blockDir block_b.enc.dat` *Data_type*: string, optional
`-cpfFile` *CPFFile*	Specifies the name of the (Chip Level) CPF file that contains the power intent and timing information. *Data_type*: string, optional
`-exceptBlocks {`*list_of_blocks*`}`	
	Specifies the list of block names to be excluded from being assembled even if they contain digital content. This option only applies to the case where `-allTimingBlocks` is used. If not specified, there is no restriction on the detection and assembly that is automatically determined. *Default*: `""` *Data_type*: string, optional
`-fe`	Specifies that Innovus place-and-route data should be used for `-blockDir` option. **Note**: The `-fe` parameter cannot be used with the `-blockData` parameter because the `-fe` parameter points to data from the Innovus database format whereas the `-blockData` parameter points to data from Verilog netlist and DEF name. *Default*: By default, data from DEF files is used. *Data_type*: bool, optional

`-fplan`	Specifies that only floorplan data should be assembled. When specified it brings back the floorplan changes from a partition block to full-chip level design. It supports un-committed partition(s) and replaces the top level uncommitted partition(s) with updated block floorplan data. **Note:** After `assembleDesign -fplan` command, any signal net at top-level design may be overlapped with other floorplan objects. You may need to reroute or ECO route the design. Additionally, PG net of CHIP, place, and route data may not be brought back. *Data_type*: bool, optional
`-keep_block_halo`	When specified, the `assembleDesign` command preserves the block place and route halo. *Data_type*: bool, optional
`-keepPGPinGeometry`	Retains the partition PG pin geometry. *Data_type*: bool, optional
`-keepPinGeometry`	Specifies that for every partition pin, a DRCFILL metal of the type special route is placed on top of the partition pins. The `-keepPinGeometry` parameter is used for analysis (both RC extraction, and DRC checking). **Note:** If you intend to run incremental routing "flat after `assembleDesign`", then you must either not use this option, or delete the DRCFILL shapes and allow the nanoRoute router to do any repair needed at the pin junctions before incremental routing is done. *Data_type*: bool, optional
`-mmmcFile` *mmmcFile*	Specifies the name of the (Chip Level) MMMC file that is used to supply the MMMC information. Normally, it is a chip level file named *viewDefinition.tcl*. When `assembleDesign` is used to incrementally assemble blocks this option isn't applicable since the timing environment should have already been established when loading the top-level design. *Data_type*: string, optional
`-noDefMerge`	Specifies that the DEF merge capability will be disabled and normal `assembleDesign` will be used. *Data_type*: bool, optional
`-outFile` *fileName*	Specifies name of the output file. *Default*: `./<design>.assembleDesign.rpt` *Data_type*: string, optional
`-reportOnly`	When enabled, the `assembleDesign` command only reports the list of blocks that would be assembled and does not perform actual assembling. **Note:** The `-reportOnly` parameter can only be used in the `assembleDesign` command along with the `-allTimingBlocks` option, to get a list of (and not actually assemble) blocks that would be assembled by a subsequent `assembleDesign -allTimingBlocks` command invocation without the `-reportOnly` option. This list of blocks can be used to include or exclude blocks to be assembled via the `-blockCell` or `-exceptblocks` options. *Data_type*: bool, optional
`-skipCells`	Skips merging standard cells. It does not load the standard cells section from DEF. *Data_type*: bool, optional
`-skipComponents`	Skips merging all components.It does not load the COMPONENT section from DEF. *Data_type*: bool, optional
`-skipNets`	Skips merging signal routes. It does not load the NETS section from DEF. *Data_type*: bool, optional
`-skipSpecialNets`	Skips merging special routes. It does not load the SPECIALNETS section from DEF. *Data_type*: bool, optional
`-topDesign` *topLib topCell topView*	
	Specifies the top-level library, cell, and view names respectively for a design that was saved into an OpenAccess database. This parameter is used when bringing back top-level data from an OpenAccess database. *Data_type*: string, optional

`-topDir` *dirname*	Specifies the path to the top-level design created by `saveDesign -def`. This design directory should contain the Verilog netlist and DEF files. Typically, this directory has the extension .enc.dat *Data_type*: string, optional

Examples

- The following command assembles the design after bringing back information from the top-level cell and two block-level cells. The top-level design data is in the top.enc.dat directory and the block-level data is in the `block_a.enc.dat` and `block_b.enc.dat` directories.
  ```
  assembleDesign -topDir top.enc.dat -blockDir block_a.enc.dat -blockDir block_b.enc.dat
  ```
- The following command assembles a design with the top-level restored from an database which was previously saved with saveDesign -def with block_a instantiated as an abstracted hard macro.
  The contents of block_a are defined by a DEF and Verilog netlist:
  - the DEF file with the name block_a.def
  - the Verilog netlist block_a.v

  ```
  assembleDesign -topDir top.enc.dat -blockData block_a.def block_a.v
  ```
- The following commands load a top-level design then incrementally calls `assembleDesign` to pull in the contents of one hard macro in the design:
  - top.enc.dat is a top-level design saved by saveDesign -def where block_a is represented by an abstract
  - block_a.def and block_a.v represent the physical contents of block_a in DEF and Verilog format
  - `restoreDesign top.enc.dat top`
  - `assembleDesign -blockData block_a.def block_a.v`
- The following command assembles a design that was saved in an OpenAccess database. The design contains two blocks.
  - The library, cell, and view names for the top-level are `libForOA, DTMF,` and `ptnView1` respectively.
  - The library, cell, and view names for the first block are `libForOA, TDSP_CORE,` and `ptnView1` respectively.
  - The library, cell, and view names for the second block are libForOA, TDSP_ARB, and ptnView1 respectively.

  ```
  assembleDesign -topDesign {libForOA DTMF ptnView1} -block {libForOA TDSP_CORE ptnView1} -block {libForOA TDSP_ARB ptnView1}
  ```
- The following command is an example of incremental assemble design capability that can restore master and clones at different levels of hierarchy. Along with assembling nested partitions:
  - Block1 and Block2 are partitions at top level
  - Block1 contains Block3 and Block4
  - Block4 is a clone of Block2

  ```
  restoreDesign topDesign.enc.dat top
  assembleDesign -blockDir Block1.enc.dat -blockDir Block2.enc.dat -blockDir Block3.enc.dat
  ```
- `-mmmcFile` and `-cpfFile` do not work with incremental `assembleDesign` since the timing and power intent is defined during design initialization, however when running assembleDesign to fully initialize the design timing and power intent can be supplied:
  ```
  assembleDesign -topDir top.enc.dat -blockDir block_a.enc.dat -mmmcFile viewDefinition.tcl -cpfFile CPFFile
  ```

Related Information

- `partition`
- `saveDesign`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Flow Methodologies
  - Handling of Blackboxes with Non-R0 Orientation
  - Working with OpenAccess Database

assignIoPins

```
assignIoPins
[-help]
[-align]
[-debugPinAll]
[-debugPinFile fileName]
[-improveSI]
[ {-pin { pinNameList } | -pin_file  fileName  | -exclude_pin_file  fileName } [-moveFixedPin ]  [-ignore_group_pins ]]
[-autoBusGroup]
```

Assigns the top I/O pin locations for a block-level design. Set the maximum or minimum pin layers with the `setPinAssignMode -maxLayer` or `-minLayer` parameters. By default, the `assignIoPins` command sets the depth of all partition pins equal to: MINAREA/pin width. This ensures that the minimum area rule is met. However, if any pin-size-related constraints are specified on these pins, the user-specified pin size will have a higher precedence.

Notes:

- The MINAREA/pin width means that the default pin width will be met MAR (minimum area rule) if an area rule of this layer is defined in the technology LEF. An I/O pin width value is based on the following order:
  1. User-specified pin width constraint.
  2. Minimum area rule of the layer if it is available in the technology file.
  3. Default layer width
- When assigning pins, the `assignIoPins` command does not check for violations between placed pins and shapes of objects (such as macros, standard cells, etc) just inside the partition fence (design boundary). This may result in DRV violations if the fence (design boundary) to core distance is zero.
- The `assignIoPins` command honors preferred routing layer attributes as soft constraints during pin assignment and reports the nets that have these attributes. It also honors the *keep out spacing* constraints on pin groups and net groups.
- In bottom-up flow, the `assignIoPins` command promotes both signal and P/G pins from the list of pins promoted by the `setPromotedMacroPin` command.
- The `assignIoPins` command honors net/pin groups that have all nets from multiple buffer chains added in the same group. Such groups can be generated through bus guide flow, feedthrough insertion flow, or can be user created. It places the pins of such nets in order to achieve better pin alignment and improve the QoR.

You can use this command after running placement and early global routing to legalize the I/O pins. However, placement is not a mandatory step for pin assignment. Placement is recommended for better QOR.

Parameters

`-align`	If you specify this option, each top I/O pin will be aligned with one of the instance pins with which it is connected. The instance pin with which the I/O pin is aligned is the one whose distance from its nearest edge is shortest among all the instance pins connected to the top I/O pin. *Default*: If you do not specify this option, the location of the top pin is based on the average of the distance of all instance pins (that are connected to the top I/O pin) from their respective nearest edges. *Data_type*: bool, optional
`-autoBusGroup`	Specifies that the buses will be grouped automatically. **Note:** Internally the createPinGroup command is applied on buses. It is removed automatically after the command execution. *Data_type*: bool, optional

`-debugPinAll`	Specifies that all pins will be debugged. **Note:** The `-debugPinAll` and `-debugPinFile` parameters can be used to write the following information in the log file: • Pin assign mode information • Connectivity information of each partition pin, including pinGroup/netGroup name and NDR name • Available pin slot information on each edge for each layer and pin slot density • PinGuide box and virtual pinGuide box information, master-clone series, feedthru chain and so on. • Reason for failure on the first failed location • For pins with location constraints, if the pin is not placed at the location constraint then the reason is provided in a tabular form. • For placed partition pins, the pin location, layer, edge, spacing and so on are provided. *Data_type*: bool, optional
`-debugPinFile` *fileName*	Specifies the file containing the pin-list for partition or the net that is to be used for debugging. *Data_type*: string, optional
`-exclude_pin_file` *fileName*	Specifies pin-list to be excluded for partition in a file. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `assignIoPins` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man assignIoPins`
`-ignore_group_pins`	Avoid placing the whole group of pins. With this parameter enabled, the `pinGroup` and `netGroup` pins are ignored during selected pin assignment. *Data_type*: bool, optional
`-improveSI`	Avoid signal integrity issues by blocking tracks while placing pins to avoid overlap of wires. With this parameter specified, the `asignIoPins` command does not allow pins to be placed on the tracks in an area which spans a layer higher or lower (on the preferred routing direction) and also on the tracks on the left and right of the placed pin. *Data_type*: bool, optional
`-moveFixedPin`	If you use this parameter, the specified pins with a *`Fixed`* status can also be moved. However, this parameter must be used with the -pin parameter to specify the list of fixed pins to move while assigning io-pins. **Note:** Pins that are not specified but have a *`Fixed`* status are not moved. *Default*: If you do not specify this parameter, the specified pins with a *`Fixed`* status are *not* moved. *Data_type*: bool, optional
`-pin` *pinNameList*	Specifies that only the specified pins should be assigned. **Note:** To move fixed pins while assigning io pins, you must specify the `-moveFixedPin` parameter. *Default* : If you do not specify this parameter, all pins are assigned. *Data_type*: string, optional

`-pin_file` *fileName*	Specifies the pin list for a partition in a file. In the pin file, the pins and nets are specified in the following format: `Partition:` *PtnName/BlockName* *Pin1* *Pin2* .. *PinN* `Partition:` *PtnName/BlockName* *Pin1* *Pin2* .. *PinN* `Nets:` *netA* *netB* .. *netN* **Note:** While specifying the pin list in the pin_file, it is necessary to give the "Partition" keyword followed by the name of the partition or block. If this is not specified, no operations are performed. You can specify multiple partitions in the same pin_file. The pin_file does not support buses or wild cards for pin names. If a pin specified in the list or a partition is not present in the design, a warning is displayed. *Data_type*: string, optional

Example

- The following command assigns all io pins and can also move pins with fixed-status.
  ```
  assignIoPins -moveFixedPin -pin *
  ```
- The following command accept a file *pinLst.txt* that contains the list of pins and assigns io pins.

  ```
  assignPtnPin -pin_file pinLst.txt
  ```

  Where *pinLst.txt* contains:
  ```
  Partition: top
  out1
  out2
  out3
  Partition: A
  in1
  in2
  in3
  Partition: B
  pina
  pinb
  pinc
  ```

  The `assignIoPins` command selects all Io pins under the section `Partition:top`.

Related Information

- `setPinAssignMode`
- `createPinGroup`
- `createPinGuide`
- `createNetGroup`
- `assignPtnPin`
- `setPromotedMacroPin`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Block Implementation
  - Assigning I/O Pins

assignPtnPin

```
assignPtnPin
[-help]
[-basedOnMasterOnly]
[-blackbox]
[-debugPinAll]
[-debugPinFile fileName]
[-dumpVirtualPinGuide]
[-ignoreAbuttedCheck]
[-improvePinOrder]
[-improveSI]
[-layerPriority {layeridList}]
[-markFixed]
[-printPinMovementStatistics]
[-skipAutoBusBitOrder]
[-skipPlaceExternalUnconnectedPin]
[-unplacedOnly]
[partitionName | {{{-ptn partitionName {-pin {pinName list}}} |
{-pin_file fileName [-exclude_ptn ptnName]} | {-exclude_pin_file ptnName}} [-moveFixedPin ] [ -ignore_group_pins ]}]
[-alignFeedThruPinsOptWithMasterClone ]
[-autoBusGroup ]
[-enforceRoute | -enforceRouteLegal | -enforceFlyline ]
```

Assigns partition pins before the partitions are committed. By default, the assigned pins will be in a Placed status after running this command. You can specify a Fixed status by running this command with the `-markFixed` option. The `assignPtnPin` command does not align pins in the local context thus avoiding mis-aligning in other scenarios. minimizing maximum mis-alignment effects across farsighted net pairs (2-pin nets). For example, in master/clone pin assignment, since the clone pin follows the master's position, the position of the pin on master partition is determined such that pin on clone also gets aligned on clone.

Running the `saveFPlan` command saves the pin assignment in the floorplan to a separate file named `fpFileName.fp.ptnpin`, which is included in main `fpFileName.fp` floorplan file.

By default, the `assignPtnPin` command sets the depth of all partition pins equal to: MINAREA/pin width. This ensures that the minimum area rule is met. However, if any pin-size-related constraints are specified on these pins, the user-specified pin size will have a higher precedence. The `assignPtnPin` command internally calls the `reportUnalignedNets` command to generate the summary report at the command line.

When assigning pins, the `assignPtnPin` command does not check for violations between placed pins and shapes of objects (such as macros, standard cells, etc) just inside the partition fence (design boundary). This may result in DRV violations if the fence (design boundary) to core distance is zero.

You can use this command after specifying the partitions, placing the design, and after running or loaded routing information. If no routing information is available, the Innovus software uses the projected pin assignment. However, placement is not a mandatory step for pin assignment. Placement is recommended for better QOR.

Notes:

- The `assignPtnPin` command honors preferred routing layer attributes as soft constraint during pin assignment and reports the nets that have some attributes assigned to them. It also honors the keep out spacing constraint for a pin group or net group.
- This command only assigns signal pins. Power and ground pins are created during partition. However, this pin optimization honors power stripes and follow pins.
- This command assigns pins for blackboxes as well as regular partitions. If this command is run on a blackbox that has one or more pins with a single LEF port but with multiple shapes within the LEF port, the command will remove these multiple shapes and create a single shape. If this command is run on a blackbox that has one or more pins with multiple LEF ports, the first LEF port will be processed as per the command and the remaining LEF ports will remain unchanged.
- The `assignPtnPin` command performs automatic pin assignment of partitions inside other partitions. It places pins of nested partitions similar to single level partitions.
- In top-down flow, the `assignPtnPin` command only promotes signal pins from the list of pins promoted by the `setPromotedMacroPin` command.
- The `assignPtnPin` command honors net/pin groups that have all nets from multiple buffer chains added in the same group. Such groups can be generated through bus guide flow, feedthrough insertion flow, or can be user created. It places the pins of such nets in order to achieve better pin alignment and improve the QoR.
- The `assignPtnPin` command can place the pins on the partitions that are connected to PG supplies defined in the netlist.

Note: You can use the `check_design -type pin_assign` command to detect the potential problems and correct the design before running `assignPtnPin` command.

Parameters

`-alignFeedThruPinsOptWithMasterClone`	Specifies that the pins should be placed such that they are aligned on different partitions across buffers on master/clone partitions.
	• In symmetric designs, since there is no conflict with master/clone so it aligns the pins of all the path.
	• In non-symmetric designs, it will try to place pins such that master-clone alignment count should not degrade.
	  ◦ It only assigns a feedthru path if there is common alignment zone available between master-clone alignment and feedthru chain alignment.
	  ◦ It assigns pins in common alignment zone for first pins in feedthrough path.
	Data_type: bool, optional

`-autoBusGroup`	Specifies that the buses will be grouped automatically. **Note:** Internally the `createPinGroup` command is applied on buses. It is removed automatically after the command execution. *Data_type*: bool, optional
`-basedOnMasterOnly`	Specifies that for designs that have partition clones, the command should consider only the partition master and use the master assignment for all the clones. *Default*: For designs that have partition clones, this command by default also considers partition clones while assigning pins. *Data_type*: bool, optional
`-blackbox`	Assigns pins of blackboxes. *Data_type*: bool, optional
`-debugPinAll`	Specifies that all pins will be debugged. **Note:** The `-debugPinAll` and `-debugPinFile` parameters can be used to write the following information in the log file: • Pin assign mode information • Connectivity information of each partition pin, including pinGroup/netGroup name and NDR name • Available pin slot information on each edge for each layer and pin slot density • PinGuide box and virtual pinGuide box information, master-clone series, feedthru chain and so on. • Reason for failure on the first failed location • For pins with location constraints, if the pin is not placed at the location constraint then the reason is provided in a tabular form. • For placed partition pins, the pin location, layer, edge, spacing and so on are provided. *Data_type*: bool, optional
`-debugPinFile` *`fileName`*	Specifies the file containing the pin-list for partition or the net that is to be used for debugging. *Data_type*: string, optional
`-dumpVirtualPinGuide`	Writes the virtual pinGuide for netGroup and pinGroup. *Data_type*: bool, optional
`-enforceFlyline`	Specifies that the pin assignment completely follows flylines, without taking into consideration any pin constraints. *Data_type*: bool, optional

`-enforceRoute`	Specifies that the pin assignment completely follows the routing results generated by Early Global Route, without taking into consideration any pin constraints. For example, the following constraints are not considered when the `-enforceRoute` parameter is used: • Constraints related to pin spacing, layer, width, depth, and side • Pin Guide constraints and Pin Group constraints • Constraints specified by the `-pinsOffStrip` parameter of the `assignIoPins` and the `assignPtnPin` commands The `-enforceRoute` parameter can be useful when, for example, you want to compare pin locations based purely on Early Global Route results with pin locations that honor the specified pin constraints. *Data_type*: bool, optional
`-enforceRouteLegal`	Specifies that during pin assignment, the pins are placed close to the routing cross points but at legal locations only. Consequently all the pins are legally placed, even if they may not be on routing cross points always. The `-enforceRouteLegal` parameter ignores the pinGroup, netGroup, and pinGuide constraints. *Data_type*: bool, optional
`-exclude_ptn` *ptnName*	Specifies the list of excluded partition for pin file. *Data_type*: string, optional
`-exclude_pin_file` *fileName*	Specifies pin-list to be excluded for a partition in a file. In the pin file, the pins and nets are specified in the following format: `Partition:` *PtnName/BlockName* *Pin1* *Pin2* .. *PinN* `Partition:` *PtnName/BlockName* *Pin1* *Pin2* .. *PinN* `Nets:` *netA* *netB* .. *netN* *Data_type*: string, optional
`-help`	Outputs a brief description that includes type and default information for each `assignPtnPin` parameter. For a detailed description of the command and all of its parameters, use the man command: `man assignPtnPin`.
`-ignoreAbuttedCheck`	When enabled, the abutted check is ignored during pin assignment. This results in faster pin assignment on abutted designs. *Data_type*: bool, optional
`-ignore_group_pins`	Avoid placing the whole group of pins. With this parameter enabled, the `pinGroup` and `netGroup` pins are ignored during selected pin assignment. **Note:** If all the pins in a group have been individually selected for pin assignment and `-ignore_group_pins` is applied then the pin assignment takes precedence and `-ignore_group_pins` parameter is not honored for this group. Rest of the groups whose pins appear in the selection list continue to be ignored. *Data_type*: bool, optional

`-improvePinOrder`	Reduces flightline crossing during pin alignment. The partition and the blackbox pins are rearranged to be aligned with external connections: • Guided pins are reordered within the pin guide box. • Pins are rearranged at (or close to) their original locations. • Pin ordering is performed independently for each partition edge. The locations of fixed pins, aligned pins or multi-location pins is not changed. Master-clone pins are not reordered. **Note:** `-improvePinOrder` is a post processing step during pin assignment so it can override the pin assignment that is complying with `-maxPinMovementForAlign`. This option should not be used with `-maxPinMovementForAlign`, if you strictly want to control the maximum pin movement for alignment. *Data_type*: bool, optional
`-improveSI`	Avoid signal integrity issues by blocking tracks while placing pins to avoid overlap of wires. With this parameter specified, the `assignPtnPin` command does not allow pins to be placed on the tracks in an area which spans a layer higher or lower (on the preferred routing direction) and also on the tracks on the left and right of the placed pin. *Data_type*: bool, optional
`-layerPriority {`*`layeridList`*`}`	Specifies the metal layer(s) that should be used with priority during pin assignment such that there is an increase in alignment in the prioritized layers. With this parameter specified, there is no degradation in the total unaligned nets of the partition/design. There is an increase in the unaligned number in the prioritized layers and a decrease in the over all pin count in the non-prioritized layers. **Note**: The list of layers to be given priority must be a sunset of the layers selected with `setPinAssignMode -maxLayer -minLayer` parameters. **Note**: In case of conflict in the layer priority between the `assignPtnPi`n command and the `createPinGuide` command on some edges, the `createPinGuide` command priority takes precedence. For example, in the following command: `assignPtnPin -layerPriority {7 6 2 3}` • Most aligned pins are in layers 7 and 3 or 6 and 2 (horizontal / vertical) • Most unaligned pins are also in 7,6,2,3 • Maximum pins are in these layers *Data_type*: string, optional
`-markFixed`	Specifies that all pins will have a *Fixed* status. *Data_type*: bool, optional
`-moveFixedPin`	If you use this parameter, the specified pins with a *Fixed* status can also be moved. **Note:** Pins that are not specified but have a *Fixed* status are not moved. *Default*: The specified pins with a *Fixed* status are not moved. *Data_type*: bool, optional
`-ptn` *`partitionName`*	Specifies the partition names for pin assignment. *Default*: Pins are assigned for all partitions. **Note:** Pin assignment for a selected partition by specifying the *`partitionName`* is obsolete and will be removed in the next major release of the software. The obsolete parameter still works in this release, but to avoid warnings, and to ensure compatibility with future releases, update your scripts to use `-ptn` *`partitionName`* `-pin` *`pinList`* `...` *Data_type*: string, optional

`-pin_file` *fileName*	Specifies the pin list for a partition in a file. In the pin file, the pins and nets are specified in the following format: `Partition:` *PtnName/BlockName* *Pin1* *Pin2* .. *PinN* `Partition:` *PtnName/BlockName* *Pin1* *Pin2* .. *PinN* `Nets:` *netA* *netB* .. *netN* **Note:** While specifying the pin list in the pin_file, it is necessary to give the “Partition” keyword followed by the name of the partition or block. If this is not specified, no operations are performed. You can specify multiple partitions in the same pin_file. The `pin_file` does not support buses or wild cards for pin names. If a pin specified in the list or a partition is not present in the design, a warning is displayed. *Data_type*: string, optional
`-pin pinList ...`	Specifies the list of pins to be assigned in the respective partitions. *Default*: Pins are assigned for all partitions. *Data_type*: string, optional

`-printPinMovementStatistics`	Prints a summary of the deviation (in micrometers) of the final pin locations from the point of intersection of the net and the partition edge. The report on the screen contains the following statistics:
	• Maximum displacement on the X axis. This is the difference in the x-coordinates of the point of intersection of the net and the partition edge, and the point at which the pin is finally assigned.
	• Maximum displacement on the Y axis.
	• Maximum X+Y displacement. The X+Y displacement of a pin is the sum of the displacement of a pin on the X axis and the displacement of the pin in the Y axis. For example, if a pin has moved 10 micrometers on the X axis and 5 micrometers on the Y axis, the X+Y displacement for that pin is 15. Thus, if there are three pins and their X+Y displacements are 5, 23, and 30 respectively, the maximum X+Y displacement is reported as 30.
	• Mean of the X+Y displacement.
	The report appended to the Innovus log file contains the following additional details:
	• Names of the pins that have moved the maximum distances.
	• The coordinates of the point of intersection of the net and the partition edge, and the point at which the pin is finally assigned.
	Here is a sample report displayed on screen.
	`Partition: [tdsp_core]`
	`maximum X = 120.1200 um (Pin: t_data_out[6])`
	`maximum Y = 58.8000 um (Pin: rom_data_in[4])`
	`maximum X+Y = 154.5600 um (Pin: t_data_out[6])`
	`moved from (706.2000, 1025.0800) -> (586.0800, 990.6400)`
	`mean X+Y = 12.5600 um`
	`total pins = 114`
	`pins moved = 75`
	The log file has additional information such as:
	`Pin: int: X Movement: 3.3000 um ; Y Movement: 0.0000. From (701.2500, 1247.6800) -> (697.9500, 1247.6800)`
	Data_type: bool, optional

`-skipAutoBusBitOrder`	Skips automatically placing the bus pins together. *Data_type*: bool, optional
`-skipPlaceExternalUnconnectedPin`	Skips placing partition pins that are not connected outside the partition. *Data_type*: bool, optional
`-unplacedOnly`	Assigns only unplaced pins. *Data_type*: bool, optional

Example

- The following command assigns partition pins of all partitions
  ```
  assignPtnPin
  ```
- The following command assigns partition pins, including those which have fixed status of all partitions and mark them as fixed after assigning them
  ```
  assignPtnPin -markFixed -moveFixedPin
  ```
- The following command accepts a file *`pinLst.txt`* that contains the list of pins to be placed.

  ```
  assignPtnPin -pin_file pinLst.txt
  ```

  Where *`pinLst.txt`* contains:
  ```
  Partition: top
  out1
  out2
  out3
  Partition: A
  in1
  in2
  in3
  Partition: B
  pina
  pinb
  pinc
  ```

  The `assignPtnPin` command, considers all pins of all partitions in the file, except the pins under the section Partition:top.
- The following command ignores the `partition B` mentioned in the pin file pin.list.
  ```
  assignPtnPin -pin_file pinLst.txt -exclude_ptn {B}
  ```

Related Information

- `reportUnalignedNets`
- `setPinAssignMode`
- `getPinAssignMode`
- `createPinGuide`
- `loadPtnPin`
- `unloadPtnPin`
- `savePtnPin`
- Partitioning the Design chapter in the *Innovus User Guide*

changeBBoxMasterFromR0

```
changeBBoxMasterFromR0
[-help]
black_box_cell
{R90 R180 R270 MX MX90 MY MY90}
new_x_origin
new_y_origin
```

Converts the orientation of the master instance blackboxes from R0 to the specified orientation.

Parameters

black_box_cell	Specifies the name of the balckbox cell. *Data_type*: string, required
-help	Outputs a brief description that includes the type and default information for each `changeBBoxMasterFromRO` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man changeBBoxMasterFromRO`
new_x_origin	Specifies the new origin x-coordinate. *Data_type*: float, required
new_y_origin	Specifies the new origin y-coordinate. *Data_type*: float, required
{R90 R180 R270 MX MX90 MY MY90}	
	Specifies the new orientation value of the cell. The orientation value can be R0, R90, R180, R270, MX, MX90, MY, or MY90. *Data_type*: enum, required

Examples

- The following commands converts the orientation of the master blackboxes CORE_CELL120 to R90.
  `changeBBoxMasterFromR0 CORE_CELL120` {R90}

Related Information

- `assignPtnPin`
- partition
- `assembleDesign`
- `flattenPartition`
- `convertBlackBoxToFence`
- `editPin`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Handling of Blackboxes with Non-R0 Orientation

changeBBoxMasterToR0

```
changeBBoxMasterToR0
[-help]
[-checkOnly]
[cellName | {cellNameList}]
```

Converts the orientation of the master instance blackboxes to R0. Partitioning-related commands such as `assignPtnPin`, `partition`, `assembleDesign`, `flattenPartition`, `convertBlackBoxToFence`, and `editPin` work only with those blackboxes whose master instances have an R0 orientation. If the design has master blackboxes whose orientation is non-R0, you can use the `changeBBoxMasterToR0` command to change the orientation of these master blackboxes to R0.

Note: Orientations of corresponding clone instances are also relatively transformed.

Parameters

`-checkOnly`	Specifies that the command will not actually convert the orientation of any master blackbox; it only displays the number of master blackboxes whose orientation would have been changed had the command been run without the `-checkOnly` parameter. *Default:* The orientation of master blackboxes in the design are converted to R0. *Data_type*: bool, optional
cellName | {*cellNameList*}	
	Specifies that cell, or the list of blackbox cells, whose orientation will be converted to R0. *Default:* The orientation of all master blackboxes is converted to R0. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `changeBBoxMasterToR0` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man changeBBoxMasterToR0`

Examples

- The following commands converts the orientation of all master blackboxes that have a non-R0 orientation to R0.
  ```
  changeBBoxMasterToR0
  ```
- The following commands converts the orientation of the master blackboxes CORE_CELL120 and CORE_CELL121 to R0.
  ```
  changeBBoxMasterToR0 {CORE_CELL120 CORE_CELL121}
  ```
- The following command uses the `-checkOnly` parameter. The command, therefore, does not actually convert the orientation of any master blackboxes; it only returns the number of blackboxes whose orientation would have been converted to R0 had the command been run without the `-checkOnly` parameter.
  ```
  changeBBoxMasterToR0 -checkOnly
  ```

Related Information

- `assignPtnPin`
- `partition`
- `assembleDesign`
- `flattenPartition`
- `convertBlackBoxToFence`
- `editPin`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Handling of Blackboxes with Non-R0 Orientation

checkHierRoute

```
checkHierRoute
[-help]
[-ptn {ptnName | ptnList}]
[-outFile outputFileName]
```

Checks and reports hierarchy violations for nets in the specified partitions.

The following conditions for hierarchy violations are checked:

- Intra-partition nets: All routes of this net should be within the partition boundary.
- Pure-Top nets: All routes of this net should be outside the partition boundary or touch the partition boundary from *outside*.
- Inter-partition nets: At least one route of this net must cross the partition boundary at partition pin. No route should cross the partition boundary at any other point.

Note: You can use this command to check for hierarchy violations before you push down the signal routes with the `partition -pushRoute` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `checkHierRoute` parameter. For a detailed description of the command and all of its parameters, use the man command: `man checkHierRoute`.
`-ptn {ptnName | ptnList}`	Report the nets that violate the hierarchy with respect to the specified partition(s). *Default:* The command checks the hierarchy violations for nets on all partitions. *Data_type*: string, optional
`-outFile outputFileName`	Prints the details of net-wise violations in the specified file. *Default:* By default, the name of the file is `hier.rpt`. *Data_type*: string, optional

Examples of hierarchy violation conditions:

The following are examples of hierarchy violation conditions:

- If wires of a inter-partition net, are crossing the partition boundary at a place other than the corresponding pin shape.
- If wires of a inter-partition net, are crossing the partition boundary at multiple places.
- If wires of a pure top net, are crossing over a partition.
- If wires of a intra partition net are going outside the boundary of its partition.

Example

The following command checks the violations for nets on all the partitions in the design. A report of the violations is printed in the `hier.rpt` file.

```
checkHierRoute
```

The output of this command on the display terminal is as follows:

```
------------------------------------------------------------
Summary of hierarchy violating nets with respect to each partition:
Partition: p1: 3 nets (0 intra, 2 inter, 1 top, 0 global)
Partition: p2: 1 nets (0 intra, 1 inter, 0 top, 0 global)
Partition: p3: 2 nets (1 intra, 0 inter, 1 top, 0 global)
Details of hierarchy violating nets reported in: hier.rpt
------------------------------------------------------------
```

The report printed in the `hier.rpt` file is as follows:

```
------------------------------------------------------------

Added the new header as follows:
############################################################
```

```
# Generated by: Cadence Innovus
# OS: Linux i686(Host ID icdlnx08s)
# Generated on:      Mon Oct 11 15:23:22 2010
#  Design: dtmf_chip
# Command: checkHierRoute
############################################################
Partition: p1 . List of hierarchy violating nets against this partition:
Hierarchy violating net   Category   Hierarchy violating wire box(llx lly urx ury)
--------------------------------------------------------------------------------
inter_ok_n1                Inter-Partition "No wire crossing partition pin"
inter_hierarchy_violating_n Inter-Partition (2189.41 2549.82 2447.09 2550.10)
top_hierarchy_violating_n2  Top (1192.15 2310.14 1192.43 2695.14)

Partition: p2 . List of hierarchy violating nets against this partition:
Hierarchy violating net   Category   Hierarchy violating wire box(llx lly urx ury)
--------------------------------------------------------------------------------
inter_ok_n1             Inter-Partition         "No wire crossing partition pin"
--------------------------------------------------------------------------------
```

Related Information

- partition
- Partitioning the Design chapter in the *Innovus User Guide*
  - Pushing Down Signal Routes

checkPinAssignment

```
checkPinAssignment
[-help]
[-outFile fileName]
[-reportSameNetPinAsOverlap]
[topCellName | -ptn partitionName list ]
[-ptn partitionName list | {-pin_file fileName [-exclude_ptn fileName]} | -exclude_pin_file fileName]
[ -report_violating_pin ]
[-ignore {bus_guide net_group pin_abutment pin_depth pin_group pin_guide pin_layer pin_min_area pin_on_fence pin_on_track pin_spacing
pin_spacing_constraint pin_spacing_routeBlk pin_width clones logical_pin pin_color unplaced} ]
[ [-preCheck [-skipCheck {overlap layer spacingConstraint fixedIoPins multiTermNet masterClone netGroup connection relativeOrientRelation
gridTrackAlignment symmetryOffset masterCloneSize alignZone}]
[-excludeNetFile fileName] ] |
{ [-ptn partitionName list] [-pin {pinName list}] [-pin_file fileName] [-exclude_pin_file fileName]
[-ignore {bus_guide net_group pin_abutment pin_depth pin_group pin_guide pin_layer pin_min_area pin_on_fence pin_on_track pin_spacing
pin_spacing_constraint pin_spacing_routeBlk pin_width clones logical_pin pin_color unplaced}]
[-report_violating_pin ] [-exclude_ptn fileName]}]
```

Checks the generated partition pins and I/O pins for the following:

- Pins are assigned to the proper metal layer depending on the side of the partition
- Pins are assigned within the range of layers reserved for the partition
- Any particular location on a routing layer has only one pin assigned
- Pins are assigned on the routing track
- Pins have a valid spacing from:
  - routing blockages
  - power pins and ground pins
- Pins have the correct size, status, and type
- Pin assignment follows the minimum area rule
- If the `-minArea` parameter is specified, the command checks for and reports the pins that are:
  - floating from inside (that is, the pins that are connected only to the top level)
  - floating from both inside the partition and at the top level.
- Pins follow the *`keep out spacing`* constraints for a pin group or net group and reports any violations in pin spacing.
- Checks and reports pin violations on clones in a master clone design.

The summary report generated by the command also includes information about feedthrough pins. These reports also provide a summary for all pins of the partition, blackbox, and the top module. However, summary report for pins is only displayed when there are multiple objects. It does not provide such a summed up pin summary report when we use the `checkPinAssignment` command for individual objects. For example, when you use the `-ptn` option, the summary report for all pins is not displayed.

Note:

- The `checkPinAssignment` command honors preferred routing layer attributes as soft constraint during pin assignment and reports violations in pin layers as an error.
- You can use this command after running early global routing and then partitioning with pin assignment.
- If this command is run on a blackbox that has one or more pins with a single LEF port but with multiple shapes within the LEF port, the command will check violations only with respect to the first shape and will ignore all other shapes. If this command is run on a blackbox that has one or more pins with multiple LEF ports, the command will check violations only with respect to the first LEF port and will ignore all other LEF ports.
- The `checkPinAssignment` command supports nested partitions and reports the pin status for pins of nested partitions.
- Each time the `checkPinAssignment` command is run, it automatically refreshes the violations as properties of pins. It clears all violation properties on all pins and populates fresh violation properties on each pin based on the latest `checkPinAssignment` results. These properties are persistent in save/restore DB. You can query results of the pin violations report generated by the `checkPinAssignment` command to automate the flow. For example:
  - To query for all partition pins with depth violation, you can use the following query:
    ```
    get_db [get_db partitions .master -if {.obj_type == hinst}] .local_hpins -if {.vio_depth == 1}
    ```
  - To query for all pins (partition pins, bbox pins and IO pins) with depth violation, you can use the following query:
    ```
    puts [concat \
    ```

```
[get_db ports -if {.vio_depth == 1}] \
[get_db [get_db partitions .master -if {.obj_type == inst}] .pins -if {.vio_depth == 1}] \
[get_db [get_db partitions .master -if {.obj_type == hinst}] .local_hpins -if {.vio_depth == 1}] \
]
```

- To query for all pins in a master partition having pin depth violations, you can use the following query:
  `get_db [get_db partitions .master -if {.obj_type == hinst && .name == "S"}] .local_hpins -if {.vio_depth == 1}`

Parameters

`-help`	Outputs a brief description that includes type and default information for each `checkPinAssignment` parameter. For a detailed description of the command and all of its parameters, use the man command: `man checkPinAssignment`
`-excludeNetFile` *filename*	Specifies the file containing the list of nets that are excluded while checking the pin assignment. *Data_type*: string, optional
`-exclude_ptn` *partitionName*	Specifies the list of excluded partition for pin file. *Data_type*: string, optional
`-exclude_pin_file` *fileName*	Specifies pin-list to be excluded for partition in a file. *Data_type*: string, optional

`-ignore`	Ignores the mentioned list of categories from checking and reporting.
	• `bus_guide`: When specified, the command ignores the violations of bus guide constraints.
	• `net_group`: When specified, the command ignores the ordering and exclusion checks on nets.
	• `pin_abutment`:When specified, the command ignores pin positions for abutted partitions.
	• `pin_depth`: When specified, the command ignores violations of pin depth constraints.
	• `pin_group`: When specified, the command ignores ordering and exclusion checks on pins.
	• `pin_guide`: When specified, the command ignores violations of pin guide constraints.
	• `pin_layer`: When specified, the command ignores violations of pin layer constraints.
	• `pin_min_area`:When specified, the command ignores the pins that are:
	  ◦ floating from inside the partition
	  ◦ floating from both inside the partition and at the top level
	• `pin_on_fence`: When specified, the command ignores whether the pins are on fence or not.
	• `pin_on_track`: When specified, the command ignores pins that are placed on layer tracks.
	• `pin_spacing`: When specified, the command ignores violations of pin spacing constraints.
	• `pin_width`: When specified, the command ignores violations of pin width constraints.
	• `clones`: When specified, the command ignores violations relating to clone pins.
	• `logical_pin`: When specified, the command ignores violations relating to logical pins.
	• `pin_color`: When specified, the command ignores violations relating to pin color.
	• `pin_spacing_constraint`: When specified, the command ignores violations relating to pin spacing constraints.
	• `pin_spacing_routeBlk`: When specified, the command ignores violations relating to pin spacing around routing blockages.
	• `unplaced`: When specified, the command ignores violations relating to unplaced pins.
	Data_type: enum_list, optional

`-outFile` *fileName*	Specifies that in addition to displaying the report on screen, the command should also print the output in the specified file. *Default*: `<design_name>.checkPin.rpt` *Data_type*: string, optional

`-pin {`*`pinName	pinNameList`*`}`	Specifies that the command should be run on the specified pin(s) only. *Data_type*: string, optional

`-pin_file` *`fileName`*	Specifies the pin list for a partition in a file.
	In the pin file, the pins and nets are specified in the following format:
	`Partition:` *`PtnName/BlockName`*
	`Pin1`
	`Pin2`
	`..`
	`PinN`
	`Partition:` *`PtnName/BlockName`*
	`Pin1`
	`Pin2`
	`..`
	`PinN`
	`Nets:`
	`netA`
	`netB`
	`..`
	`netN`
	Note: While specifying the pin list in the pin_file, it is necessary to give the “Partition” keyword followed by the name of the partition or block. If this is not specified, no operations are performed. You can specify multiple partitions in the same pin_file.
	The `pin_file` does not support buses or wild cards for pin names. If a pin specified in the list or a partition is not present in the design, a warning is displayed.
	Data_type: string, optional

`-preCheck`	Checks for the feasibility of pin assignment mainly to catch the reason for unplaced pins in fully abutted design. **Note**: No other `checkPinAssignment` command parameter can be used if `-preCheck` is being used. *Data_type*: bool, optional
`-ptn` *`partitionName`*	Specifies the partition for which the pins will be checked. This parameter is required if the partition is not the top-level module. *Data_type*: string, optional
`-reportSameNetPinAsOverlap`	Reports the pins (connected to same net) placed at the same location and the same layer as overlap violation. *Data_type*: bool, optional
`-report_violating_pin`	Reports all the pins with violations. *Data_type*: bool, optional
`-skipCheck {overlap layer spacingConstraint fixedIoPins multiTermNet masterClone netGroup connection relativeOrientRelation gridTrackAlignment symmetryOffset masterCloneSize alignZone}`	

	Skips checking the category from the list of type of checks. This parameter is used with the `-preCheck` parameter. • `overlap`: When specified, the command skips checking for any partition fences which are overlapping. • `layer`: When specified, the command skips checking for allowed layers on abutted edges of partition. • `spacingConstraint`: When specified, the command skips checking for spacing on abutted edges. • `fixedIoPins`: When specified, the command skips checking if the block pin is placed on an abutted edge with its connected partition. • `multiTermNet`: When specified, the command skips checking for multi partition pin nets in abutted design. • `masterClone`: When specified, the command skips checking for master and clone partitions. • `netGroup`: When specified, the command skips checking the net group ordering check in master and clone. • `connection`: When specified, the command skips checking if symmetric abutted designs have each master-clone pair connected to same pin. • `relativeOrientRelation`: When specified, the command skips checking the clone inside clone orientation. The clone inside a clone should have a relative orientation with its parent which is checked with master/clone inside master. • `gridTrackAlignment`: When specified, the command skips checking for routing track offset in Master-Clone design to avoid off grid pins. • `symmetryOffset`: When specified, the command skips checking if the clone to clone offset (relative distance on abutted edges) is same as in the master to master/clone connection pair. This is required so that the pins in each clone-clone and master-master/clone pair is the same. • `masterCloneSize`: When specified, the command skips checking the master-clone size. This is required to check the number of poly edges of clones with master and check each edge size for any mismatch. • `alignZone`: When specified, the command skips checking for the common alignment zone between master-clone and their connected partitions. *Data_type*: enum_list, optional
topCellName	Specifies the top cell name. *Data_type*: string, optional

Example

- The following command checks violations on pins of partition `tdsp_core` and reports them to file `reportViolation.txt` :

  ```
  checkPinAssignment -ptn tdsp_core -outFile reportViolation.txt
  ```

  The output will look like:

  ```
  Start Checking pins of partition: [tdsp_core]
  Summary report for partition: [tdsp_core]
  Total Pins : 134
  Legally Assigned Pins : 110
  Illegally Assigned Pins : 0
  Unplaced Pins : 0
  Constant/Spl Net Pins : 24
  Internal Pins : 0
  Legally Assigned Feedthru Pins : 0
  Illegally Assigned Feedthru Pins : 0
  End of Summary report
  End Checking pins of partition: [tdsp_core]
  ```

- The following command accepts a file that contains the list of pins for partition A and partition B and checks any violations in pin assignment.

  ```
  checkPinAssignment -ptn {A B} -pin_file pinLst.txt
  ```

Related Information

- `assignIoPins`
- `assignPtnPin`
- `legalizePin`
- `setAttribute`
- `alignPtnClone`
- `pinAlignment`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Assigning Pins
  - Assigning I/O Pins
  - Validating Pin Placement

clonePtnObj

```
clonePtnObj
[-help]
[-checkOnly]
[-objs {all busGuides}]
[-redo]
[-refHinsts list_of_hinst_names]
[-sync]
[-targetHinsts list_of_hinst_names]
[-undo]
```

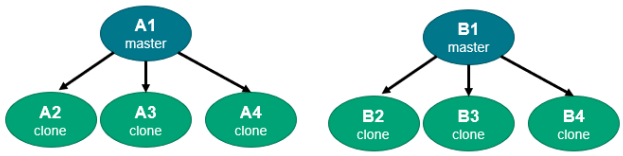
Syncs up the changes made to the specified objects in the master or clone partitions in the design with other hierarchical instances (master and clones) of the same module.

Parameters

`-checkOnly`	When specified, the tool check if the master and clone partitions are out of sync or not. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `clonePtnObj` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man clonePtnObj`
`-objs {all busGuides}`	When specified, the tool does the sync-up of the specified objects. *Data_type*: enum, optional
`-redo`	When specified, the changes made to the object are done again . The master and clone partitions are brought to the next committed sync state. **Note**: Currently this parameter only supports the `busGuides` object type (-objs). **Note**: This parameter is exclusive with the `-refHinsts` and –`targetHinsts` parameter options. *Data_type*: bool, optional
`-refHinsts list_of_hinst_names`	Specifies the name of the master or clone hierarchical instance *from which* the obj data is to be copied. *Default*: All master partitions in the design. *Data_type*: string, optional
`-sync`	When specified, the objects in all master and clone partitions are synced up by looking at the changes in shape in any hierarchical instance of the master and clone partitions. **Note**: Currently this parameter only supports the `busGuides` object type (-objs). **Note**: This parameter is exclusive with the `-refHinsts` and –`targetHinsts` parameter options. *Data_type*: bool, optional

`-targetHinsts` *list_of_hinst_names*	Specifies the name of the master or clone hierarchical instance *to which* the obj data is to be copied. • When a master partition is specified with the `-targetHinsts` parameter, specifying the `-refHinsts` becomes mandatory as a user provided hinst is considered to sync up objects. • When clones partitions are specified with the `-targetHinsts` parameter, specifying the `-refHinsts` is optional as master hierarchical instances are automatically considered (default) to sync-up objects *Default*: All clone partitions in the design. *Data_type*: string, optional
`-undo`	When specified, the changes made to the object are reverted. The master and clone partitions are brought to the last committed sync state. **Note**: Currently this parameter only supports the `busGuides` object type (-objs). **Note**: This parameter is exclusive with the `-refHinsts` and –`targetHinsts` parameter options. *Data_type*: bool, optional

Example




```
Hinst: A1 (master), A2 (clone), A3(clone), A4 (clone)
Hinst: B1 (master), B2 (clone), B3(clone), B4 (clone)
```

- The following commands syncs all clones to their respective master.
  ```
  clonePtnObj
  ```
- The following command syncs the hinsts A2 and A3 (clones) to the master of module A (A1).
  ```
  clonePtnObj -targetHinsts {A2 A3}
  ```
- The following command syncs the hinsts A1 (master) and A3 (clone) to the hinst A2 (clone)
  ```
  clonePtnObj -targetHinsts {A1 A3} -refHinsts {A2}
  ```
- The following command syncs the hinsts A1 (master), A3, B2, B3 (clones) to hinsts A2 (clone) and the master of module B (B1)
  ```
  clonePtnObj -targetHinsts {A1 A3 B2 B3} -refHinsts {A2}
  ```
- The following command checks and reports if the hinsts A1(master) and A3, B2, B3 (clones) are synced to hinst A2 (clone) and the master of module B (B1)
  ```
  clonePtnObj -targetHinsts {A1 A3 B2 B3} -refHinsts {A2} -checkOnly
  ```
- The following command checks and reports if the hinsts A1 (master) and A3, B2, B3 (clones) are synced to hinst A2 (clone) and the master of module B (B1) for bus guides only.
  ```
  clonePtnObj -targetHinsts {A1 A3 B2 B3} -refHinsts {A2} -checkOnly -objs {busGuides}
  ```
- The following command syncs the hinsts A1 (master) and A3, B2, B3 (clones) to hinst A2 (clone) and the master of module B (B1) for bus guides only.
  ```
  clonePtnObj -targetHinsts {A1 A3 B2 B3} -refHinsts {A2} -objs {busGuides}
  ```

Related Information

- `place_partition_clone`
- Partitioning the Design chapter in the *Innovus User Guide*

commit_pushdown_eco

```
commit_pushdown_eco
[-help]
-eco_object_dir dirName
-post_eco_dir dirName
-pre_eco_dir dirName
[-ptns ptnNameList]
```

Commits ECO for a partition block by applying ECO and generating a database for the block. It applies the previously generated ECO objects, to each partition, in parallel, using EDP (parallel/distributed) framework.

Note: The `commit_pushdown_eco` command also commits the FlexHtree amendment into the partition block. Clock amendments can be restored by using the `create_ccopt_clock_tree_spec` command.

Parameters

`-eco_object_dir dirName`	Specifies the parent directory which contains all partition sub-directories in which ECO object files of partitions are kept. **Note:** The eco object file names are pre-determined to be: *eco_logical.txt* *eco_physical.txt* *eco_net.def.gz* *eco_pin.txt* *Data_type:* string, required
`-help`	Outputs a brief description that includes type and default information for each `commit_pushdown_eco` parameter. For a detailed description of the command and all of its parameters, use the man command: `man commit_pushdown_eco`.
`-post_eco_dir dirName`	Specifies the parent directory in which, after applying the ECOs, the partition databases are to be saved. *Default:* *./post_eco_dir* *Data_type:* string, required
`-pre_eco_dir dirName`	Specifies the latest version of original design, to which ECO objects need to be applied. It contains sub-directories to represent latest version of each partition database. *Data_type:* string, required
`-ptns ptnNameList`	Specifies the list of partition names for which ECO objects are created. *Data_type:* string, optional

Related Information

- `create_pushdown_eco`
- `push_ptn_network`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Partition Pushdown Replay Flow

comparePinAssignStatistics

```
comparePinAssignStatistics
[-help]
[-ignoreMinWireLength minWireLength]
[-outFile pinQoRCompareFileName]
-reference referenceFileName
-target targetFileName
```

Generates a pin QoR comparison file based on a reference and a target pin QoR file.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `comparePinAssignStatistic` parameter. For a detailed description of the command and all of its parameters, use the man command: `man comparePinAssignStatistic`.
`-ignoreMinWireLength minWireLength`	Specifies the minimum wire length. If reference wire length at a partition pin is less than the *`minWireLength`*, then it is not reported in the output file. *Default*: 60 *Data_type*: string, optional
`-outFile pinQoRCompareFileName`	Specifies an output file for the pin QoR comparison. *Default*: `./qor_compare.txt` *Data_type*: string, optional
`-reference referenceFileName`	Specifies the reference file generated using the `reportPinAssignStatistics` command. For example, the following command generates a reference pin QoR file named `pin_qor_reference.txt`. `reportPinAssignStatistics -outFile pin_qor_reference.txt` *Data_type*: string, required
`-target targetFileName`	Specifies the target file generated using the `reportPinAssignStatistics` command. For example, the following command generates a target pin QoR file named `pin_qor_target.txt`. `reportPinAssignStatistics -outFile pin_qor_target.txt` *Data_type*: string, required

Example

The following command generates a pin QoR comparison file, `pin_qor_compare.txt`:

```
reportPinAssignStatistics -outFile pin_qor_reference.txt
reportPinAssignStatistics -outFile pin_qor_target.txt
comparePinAssignStatistics -reference pin_qor_reference.txt -target pin_qor_target.txt -outFile pin_qor_compare.txt
```

Related Information

- `reportPinAssignStatistics`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Pin QoR Metrics and Comparison

connectMacroFeedthrough

```
connectMacroFeedthrough
[-help]
[-selectNet string | -excludeNet string | -selectMarkedNet]
[-selectInst string | -excludeInst string]
[-ecoFile string | -checkOnly]
[-portMap string]
[-verbose]
[-maxSearchDistance float]
[-searchAllSides]
[-abutment float]
[-abutmentFile string]
[-floatingPortList string]
```

Connects predefined feedthrough pins in custom macros to overlapping nets that have wires crossing over the custom macros, based on the routing information. The feedthrough connectivity details are defined in a mapping file, which is specified through the `-portmap filename` parameter. You can include or exclude specific nets or instances. This command deletes routing only for the modified nets. You can run this command after placing the design and running Early Global Route.

Parameters

`-abutment float`	Specifies the distance that pins on the same track and layer must have from one another so as to be considered as abutted. If two or more pins are considered as abutted, any feedthrough path that passes through one of the pins will pass through the other pin(s) as well. *Default:* If this parameter is not specified, the default value is `1` micron. *Data_type*: float, optional

`-abutmentFile string`	Reports, in the file specified, the list of pins that are considered as abutted. In the file, each feedthrough path is traced from the first input pin to the last input pin. Each path is preceded by the word `Begin` on a new line and followed by the word `End` on a new line.
	Here is the extract from an example file:
	`Begin`
	`Inst [Inst1], Term In [A1] Term Out [X1]`
	`Inst [Inst2], Term In [A2] Term Out [X2]`
	`End`
	In the previous example, the out term `X1` of instance `Inst1` is abutted with the in term `A2` of instance `Inst2`.
	Data_type: string, optional

`-checkOnly`	Specifies that only a report of the messages should be displayed but no changes should be made to the netlist. *Data_type*: bool, optional

`-ecoFile string`	Specifies the name of the ECO file that will contain a report of the changes. You can use this file while performing ECO changes.
	Here is a sample report generated through this parameter:
	`FORMATVERSION 2`
	`ATTACHTERM eco_instance1 inputbuf2[2] testnet`
	`ADDNET FE_FTM_1_testnet`
	`ATTACHTERM eco_instance1 outputbuf2[2] FE_FTM_1_testnet ATTACHTERM i1 A FE_FTM_1_testnet ATTACHTERM eco_instance1 rstb FE_FTM_1_testnet`
	Data_type: string, optional

`-excludeInst string`	Prevents feedthrough buffers from being added for the specified instance. The names of the instances are specified in a file, specified by `fileName`. In the file, you can specify wildcards (*) for the instance names to exclude. *Default:* All instances of the cell types specified in the portmap file are considered. *Data_type*: string, optional

`-excludeNet` *string*	Prevents feedthrough buffers from being added to the specified net. The names of the nets are specified in a file, specified by *fileName*. In the file, you can specify wildcards (*) for the net names to exclude. *Default:* All nets or bus names are considered. *Data_type*: string, optional
`-floatingPortList` *string*	Specifies that the list of unused ports should be saved in the specified file. The file lists the instance name and the pin name for each unused port. After running the `connectMacroFeedthrough` command, the unused ports should be manually tied. You can use this parameter to save the list of the unused ports to a file. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `connectMacroFeedthrough` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man connectMacroFeedthrough`
`-maxSearchDistance` *float*	Specifies the distance in microns from the wire crossing till which the command should search to find a connected feedthrough. **Note:** Specifying a large value for this parameter might have an adverse impact on the Quality of Results (QoR). *Data_type*: float, optional

`-portMap` *string*	Specifies the name of the file that contains the feedthrough connectivity definitions.
	Default: If you do not specify a file name, a file with the name `portmap` in the current directory is used.by default.
	Note: A file with the feethrough connectivity definitions must be available.
	The syntax of the file is as follows:
	`MACRO MacroName`
	`Macro definition section`
	`END MACRO`
	The definition of the macro is provided in the *`Macro definition`* section, which can contain one or more feedthrough sections. The name of the feedthrough section is optional.
	The syntax of the Feedthrough section is as follows:
	`Feedthrough [FeedthroughName]`
	`Pin Section`
	`END FEEDTHROUGH`
	Each Feedthrough section contains one section for the INPUT PIN and one section for the output pin.
	Note: Multi-fanout feedthrough sections are not supported.
	The syntax of the pin section is as follows:
	`PIN PinName`
	`DIRECTION {in | out };`
	`SIDE {left | right | top | bottom};`
	`END PIN`
	If the pin direction is not specified, it is read from the LEF file. If you specify a value for pin direction, ensure that it is consistent with the description in the LEF file.
	Note: The pin direction `inout` is not supported.
	Note: All the predefined macro feedthrough pins should be floating pins.
	Here is an example of a mapping file:
	`MACRO RAMXXX`
	`FEEDTHROUGH feedthrough1`
	`PIN feedthrough1_in`
	`END PIN`
	`PIN feedthrough1_out`
	`END PIN`
	`END FEEDTHROUGH`
	`FEEDTHROUGH feedthrough2`
	`PIN feedthrough2_in`
	`END PIN`
	`PIN feedthrough2_out`
	`END PIN`
	`END FEEDTHROUGH`
	`END MACRO`
	Data_type: string, optional

`-searchAllSides`	Specifies that if the closest available feedthrough pin on the same side cannot be connected, the command should search for the feedthrough pins on other sides as well. When you specify this parameter, it is recommended to limit the distance of the search using the `-maxSearchDistance` option to get good QoR. *Default*: The command searches for a feedthrough whose in and out pins lie *on the same sides* of the macro on which the wires enter and exit the macro. *Data_type*: bool, optional
`-selectInst` *string*	Specifies that the feedthrough buffers should be added only for the specified instances. *Data_type*: string, optional

`-selectMarkedNet`	Specifies that feedthrough buffers should be added only to the nets that are currently selected. *Default:* All nets.are considered. *Data_type*: bool, optional
`-selectNet` *string*	Specifies that the feedthrough buffers should be added only to the specified nets. *Data_type*: string, optional
`-verbose`	Provides additional messages, for example net-specific messages. *Data_type*: bool, optional

Example

The following command connects predefined feedthrough buffers in custom macros for nets that are listed in the file named `NetsToIncludeFile`. The command searches for a distance of 10 microns. The feedthrough connectivity is described in a file named `portmapFileForNets`.

```
connectMacroFeedthrough portMap portmapFileForNets -selectNet NetsToIncludeFile
-maxSearchDistance 10
```

Related Information

- `insertPtnFeedthrough`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Utilizing Pre-defined Feedthrough Pins in Custom Macros

convertBlackBoxToFence

```
convertBlackBoxToFence
[-help]
{-cell cellName | -inst instName}
```

Converts a blackbox to a fence. Once the black box netlist is well defined, use this command to convert the black box back to a regular partition.

Parameters

`-cell cellName`	Specifies the name of a module or submodule that should be converted to a fence. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `convertBlackBoxToFence` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man convertBlackBoxToFence`
`-inst instName`	Specifies the name of the blackbox instance that should be converted to a fence. *Data_type*: string, optional

Example

The following command converts a blackbox, hinst5, to a fence:

```
convertBlackBoxToFence -inst hinst5
```

Related Information

- `convertFenceToBlackBox`
- `getBlackBoxArea`
- `loadBlackBoxNetlist`
- `setImportMode`
- `specifyBlackBox`
- `saveDesign`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Defining Blackboxes

convertFenceToBlackBox

```
convertFenceToBlackBox
[-help]
{-cell cellName | -hinst hinstName}
[-noRebuildTiming]
[-size { x y }]
```

Converts a fence to a blackbox. Use this command before committing partition.

Parameters

`-cell cellName`	Specifies the name of a module or submodule that should be converted to a black box. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `convertFenceToBlackBox` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man convertFenceToBlackBox`
`-hinst hinstName`	Specifies the name of the hierarchical instance that should be converted to a black box. *Data_type*: string, optional
`-noRebuildTiming`	Skips rebuilding the timing graph and re-initializing the top cell information. *Data_type*: bool, optional
`-size {x y}`	Specifies the width and height of the blackbox. *Default:* If you do not specify this parameter, the current fence size is taken as the initial size of the blackbox. *Data_type*: point, optional

Example

The following command converts a fence, cp1A to a blackbox with 12,44 dimensions.

```
convertFenceToBlackBox -cell cp1A -size 12 44
```

Related Information

- `convertBlackBoxToFence`
- `getBlackBoxArea`
- `loadBlackBoxNetlist`
- `setImportMode`
- `specifyBlackBox`
- `saveDesign`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Defining Blackboxes

create_partition_wrapper

```
create_partition_wrapper
[-help]
-partitions partition_names
[-pushdown]
[-wrapper_name wrapper_suffix_string]
```

Creates a wrapper partition that embeds the specified partition cell to preserve its verilog module definition in the netlist. The wrapper partition can be used for making netlist changes without changing the netlist of the embedded (wrapped) cell definition.

The wrapper partition has the same partition definition attributes as the specified partition. The timing constraints (through SDC) and low power information (through UPF) of the wrapper partition is updated with the correct hierarchical pin information. The pin constraints applied on pins of the original partition are mapped to the new wrapper pins.

Note: After the wrapper partition is created using the `create_partition_wrapper` command:

- the original partition ceases to exist (`deletePartition`).
- the fence attributes and pin constraints of the original partition are removed.
- the module pins of the original partition as preserved such that they cannot be altered later.

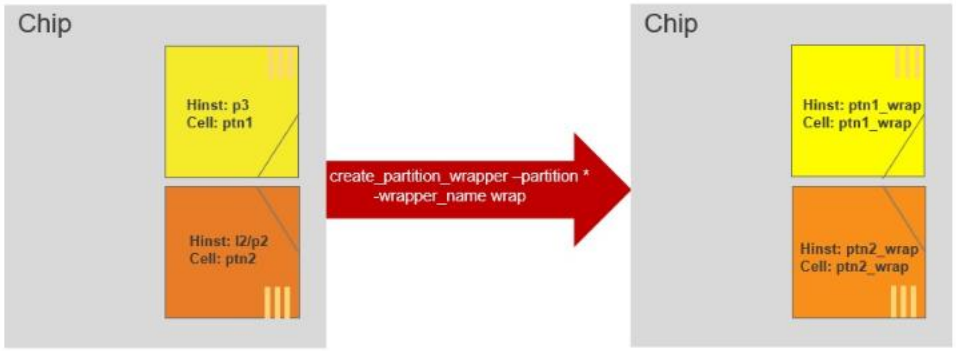
Parameters

`-help`	Outputs a brief description that includes the type and default information for each `create_partition_wrapper` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man create_partition_wrapper`
`-partitions partition_names`	Specifies the name of one or more partitions on which the wrapper is created. **Note:** The partition_wrapper will have the same: • Number, name and locations (If already assigned) of pins • Area, shape, location, orientation *Data_type*: string, optional
`-pushdown`	When enabled, it keeps the partition hinst intact for the wrapper partition. *Data_type*: bool, optional
`-wrapper_name wrapper_suffix_string`	Specifies the suffix that is added to a partition name for creating a partition wrapper. *Default*: <`ptnName`>`_wrapper` *Data_type*: string, optional

Example

The following command creates a wrapper partition with suffix wrap for all the partitions in the design:

```
create_partition_wrapper -partition * -wrapper_name wrap
```



Related Information

- deletePartition
- Partitioning the Design chapter in the *Innovus User Guide*

create_pushdown_eco

```
create_pushdown_eco
[-help]
[-gen_script_only]
[-ptn_net_dir ptnNetDirName]
[-ptns ptnNameList]
{-original_dir originalDirName | -original_tag tagName }
{-pushdown_dir pushdownDirName | -pushdown_tag tagName }
[-eco_object_dir outputDirName | -eco_tag tagName ]
```

Creates logical and physical ECO files that are used to push down ECO objects by comparing two partition databases. It generates logical and physical ECO objects for each partition, in parallel, using EDP (parallel/distributed) framework. The regular/special wire ECO is generated for those nets which are specified via `-ptn_net_dir`.

Note: A FlexHtree amendment file will also be created besides the logical ECO and DEF file.

Parameters

`-eco_object_dir` *outputDirName*	Specifies the directory in which eco objects for each partition will be created. The eco objects are saved in sub-directories, one for each partition. *Default*: `./eco_object_dir` *Data_type*: string, optional
`-eco_tag` *tagName*	Specifies the tag name where ECO file will be saved. *Default*: `eco_object_dir` *Data_type*: string, optional
`-gen_script_only`	Generates a TCL command file but does not execute it. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes type and default information for each `create_pushdown_eco` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_pushdown_eco`.
`-original_dir` *originalDirName*	Specifies the directory in which the original partition sub-directories are present. *Data_type*: string, optional
`-original_tag` *tagName*	Specifies the tag name where the original partition pnr model is present. *Data_type*: string, optional
`-ptn_net_dir` *ptnNetDirName*	Specifies the directory in which net file for each partition is present. The net file contains the nets for which regular/special wire ECO is generated. **Note**: Specifying `-ptn_net_dir` iis not mandatory for the `create_pushdown_eco` command. If the `-ptn_net_dir` parameter is not specified, then logically modified nets are selected for DEF ECO generation. Otherwise, a union of `-ptn_net_dir` and logically-modified-nets is selected for DEF ECO generation. *Data_type*: string, optional
`-ptns` *ptnNameList*	Specifies the list of partition names for which ECO objects are to be created. *Data_type*: string, optional
`-pushdown_dir` *pushdownDirName*	Specifies the directory in which pushdown (Example, clock tree constructed) version of partition sub-directories are present. These partitions will be compared with respective original partitions, and ECO objects will be generated for each of them. *Data_type*: string, required
`-pushdown_tag` *tagName*	Specifies the tag name of the pnr model that contains the pushed down data (clock or feedthrough). *Data_type*: string, optional

Example

- The following is an example dbTCL script for generating the original database of each partition , generating the pushdown database, and then generating partition pushdown ECO:

```
# load full-chip database;
partition -pushDownSpecialNetAsObs $snets
savePartition -dir $originalDir -def
flattenPartition
# Create push down (say, htree/push_ptn_network) database
push_ptn_network -nets $clk_nets -output_nets_for_wire_distribution nets.txt …
partition -pushDownSpecialNetAsObs $snets -inputNetsForWireDistribution nets.txt
savePartition -dir $htreeBlocksDir -def
# Create ECO by comparing original and pushdown databases
create_pushdown_eco \
-original_dir $originalDir \
 -pushdown_dir $htreeBlocksDir \
 -ptns [concat [dbget top.ptns.master.cell.name] [dbget top.name]] \
-eco_object_dir $ecoObjectDir \
-gen_script_only
```

  The above script generates ECO objects, for each partition, in $ecoObjectDir. ECO is generated only for the partitions specified via –ptns option.
- The following command generates logical and physical ECO objects based on the specified tag names of the pnr model:

```
create_pushdown_eco -original_tag original -pushdown_tag block_with_ft  -eco_tag eco_data
```

Related Information

- commit_pushdown_eco
- push_ptn_network
- Partitioning the Design chapter in the *Innovus User Guide*
  - Partition Pushdown Replay Flow

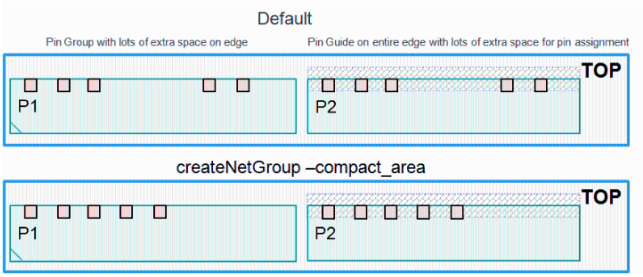
createNetGroup

```
createNetGroup
[-help]
[-mixed_signal]
[-spacing  minSpaceInTracks]
[ {[-optimizeOrder ]} | -spreadPin | -compact_area ]
[-excludePin {allLayer | sameLayer | allLayerInGuidedArea} [-keep_out_spacing integer]]
{ [netGroupName [-net netNameList]]}
```

Creates an empty net group. A net group can contain net names, bus names, or segments of a bus. To add nets into the net group, use the `addNetToNetGroup` command. You can use this command after importing the design.

Note: If a net is part of a net group, and you add the same net to another net group, the Innovus software will display a warning, but the net will be added to the second net group too. However, when pin assignment is performed, the net is considered part only of the *first* net group to which it was added – and *not* part of any other net group to which it was subsequently added.

Parameters

`-compact_area`	Specifies that pins in the group are assigned tightly together. By default, pins which are part of a group and associated to a guide can be spread inside the guide based on the area available and alignment to targets. The `compact_area` option, when used, ignores the alignment and available area and forces the pins to the minimum area on guide honoring the minimum spacing constraint and overlooking any alignment to targets.  *Data_type*: bool, optional
`-excludePin {allLayer | sameLayer | allLayerInGuidedArea}`	
	Specifies that pins in the group exclude foreign pins on all layers, same layer, or on all layers in the guided area. • `allLayer`: Specifies that pins in the group exclude foreign pins between grouped pins on all layers. • `sameLayer`: Specifies that pins in the group exclude foreign pins grouped pins on all same layer. • `allLayerInGuidedArea`: Specifies that pins in the group exclude foreign pins in the guide area defined for the group, on all layers. **Note:** If guide is not defined for a group down the flow, `allLayerInGuidedArea` option would be treated like `-excludePin {allLayer}` for the group without guides behavior. *Data_type*: enum, optional
`-help`	Outputs a brief description that includes type and default information for each `createNetGroup` parameter. For a detailed description of the command and all of its parameters, use the man command: `man createNetGroup`.
`-keep_out_spacing integer`	Specifies the minimum spacing between the pins placed outside of the group, in number of tracks or pitch. *Minimum:* 0 *Maximum:* 65535 *Data_type*: int, optional
`-mixed_signal`	Specifies to identify a mixed signal group. *Data_type*: bool, optional

`-net netNameList`	Specifies the name of the net(s). You can use wildcards (`*?`) with this parameter. *Data_type*: string, optional
`netGroupName`	Specifies the name of the created net group name. *Data_type*: string, optional
`-optimizeOrder`	Specifies that pins in the group are reordered to optimize wire length. If this option is not selected, the pin order is exactly as specified in the pin group. *Data_type*: bool, optional
`-spacing minSpaceInTracks`	Specifies the minimum spacing between the pins of the net group. *Default:* The default value is two tracks. *Data_type*: int, optional
`-spreadPin`	Specifies that pins in the group must be distributed evenly. *Data_type*: bool, optional

Example

- The following command creates a net group `bundle1`:
  ```
  createNetGroup bundle1
  ```
- The following command creates a net group `bundle1` and allows re-ordering of the nets definition inside the group:
  ```
  createNetGroup bundle1 -optimizeOrder
  ```

Related Information

- `addNetToNetGroup`
- `deleteNetGroup`
- `deleteNetFromNetGroup`
- `reportNetGroup`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Setting Pin Constraints
  - Net Group

createPinBlkg

```
createPinBlkg
[-help] [-cell cell_name] [-layer {layer_id_list}]
{ -area {x1 y1 x2 y2} | [-edge edge_number [[-offset_start distance] [-offset_end distance]]] |
[-abutted_master_clone_channel_access_only [-feedthru_blockage_file file_name]] }
[ -name blockage_name [ -includePinPatterns {pin_name_pattern_list} | -excludePinPatterns {pin_name_pattern_list} ]]
```

Creates a pin blockage for a module. You can run this command on the top-level module also. You can use this command during partition floorplanning.

Parameters

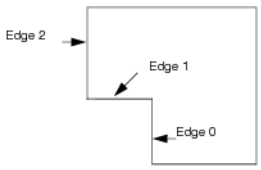
`-abutted_master_clone_channel_access_only`	Creates pin blockages on abutted edges of master-clones that abut to its parent or other master-clones. *Data_type*: bool, optional
`-area x1 y1 x2 y2`	Specifies the lower left x coordinate, lower left y coordinate, upper right x coordinate, and upper right y coordinate respectively of the blockage area. **Note:** The values of *x1, y1, x2, and y2* are relative to the top-level coordinates. *Data_type*: box, optional
`-cell cell_name`	Specifies the module for which the pin blockage will be created. • If `-cell` option is not used, then the blockage applies to top-level pins only. • If `-cell` option is used, then only, the Partition/Blackbox pins will be considered. *Data_type*: string, optional
`-edge edge_number`	Specifies the integer value where edge numbering starts from lower-left corner of a partition clock-wise. Edge `0` is the edge that has the smallest y value. *Data_type*: int, optional
`-excludePinPatterns {pin_name_pattern_list}`	Specifies the pin-list on which the pin-blockage is not applicable. Pins which are specified in this list can be placed in this area legally. **Note**: The `-excludePinPatterns` and `-includePinPatterns` options are mutually exclusive and cannot be used in a single command. Both these options require the use of the `-name` option to specify the name of the partition pin blockage. *Data_type*: string, optional
`-feedthru_blockage_file file_name`	Outputs a file with coordinates of blocked edges for pin assignment transformed into feedthru blockage_file format. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `createPinBlkg` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man createPinBlkg`
`-includePinPatterns {pin_name_pattern_list}`	Specifies the pin-list on which the pin-blockage is applicable. All other pins can be placed in this area legally. **Note**: The `-excludePinPatterns` and `-includePinPatterns` options are mutually exclusive and cannot be used in a single command. Both these options require the use of the `-name` option to specify the name of the partition pin blockage. *Data_type*: string, optional
`-layer {layer_id_list}`	Specifies the metal layer(s) for which pins are not to be created. **Note:** Layers can be specified using the LEF layer names or layer ID numbers. *Default:* If no layer number is specified, all metal layers are blocked. *Data_type*: string, optional

`-name` *`blockage_name`*	Specifies the name of the partition pin blockage. *Default:* By default, the name is `defPtnPinBlkName`. *Data_type*: string, optional
`-offset_end` *`distance`*	Specifies the distance, in microns, from the ending corner of the edge. *Minimum Value:* 0 *Maximum Value:* 65535 *Data_type*: float, optional
`-offset_start` *`distance`*	Specifies the distance, in microns, from the starting corner of the edge. *Minimum Value:* 0 *Maximum Value:* 65535 *Data_type*: float, optional

Examples

- The following command creates a pin blockage named `pinBlkg1` on the edge `0` of layers `Metal5` and `Metal7` for the module `PtnBlock1`.

```
createPinBlkg -edge 0 -name pinBlkg1 -cell PtnBlock1 -layer {Metal5 Metal7}
```



- The following command creates a pin blockage named `pinBlkg2` on layers Metal5 and Metal7 for the module `PtnBlock2`. The blockage area is bounded by the lower-left coordinates `100`, `200` and the upper-right coordinates `300`, `400`.

```
createPinBlkg -area 100 200 300 400 -name pinBlkg2 -cell PtnBlock2 -layer {Metal5 Metal7}
```

- The following command creates a pin blockage named pinBlkg3 that only applies to partition pins that match pattern1 and pattern2. All other partition pins can be placed in this area legally.

```
createPinBlkg -includePinPatterns {*pattern1* *pattern2*} -cell PtnBlock3 -area $area -name pinBlkg3
```

- The following command creates a pin blockage named pinBlkg4 that only applies to all partition pins except those that match pattern1 and pattern2. Only pins that match pattern1 and pattern2 can be placed at this edge legally.

```
createPinBlkg -excludePinPatterns {*pattern1* *pattern2*} -cell PtnBlock4 -edge 0 -name pinBlkg4
```

Related Information

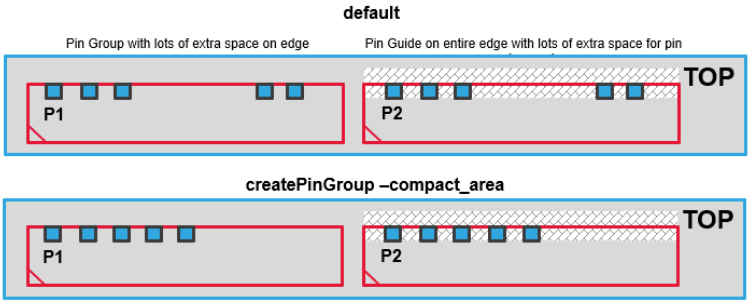
- `deletePinBlkg`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Setting Pin Constraints
  - Pin Blockage

createPinGroup

```
createPinGroup
[-help]
[-cell string]
[-excludePin {allLayer | sameLayer | allLayerInGuidedArea}]
[-keep_out_spacing integer]
[-spacing integer]
[-width width]
[{[-optimizeOrder ]} | -spreadPin | -compact_area ]
[{pinGroupName [-pin {pinNameList}]} ]
```

Creates a pin group for a partition or for the top-level. Pin groups in conjunction with pin guides are used to guide pin placement when the `assignIoPins` or `assignPtnPin` command is called. A pin group is a first class data object–delete with the `deletePinGroup` command.

Parameters

`-cell string`	Specifies the name of the module or submodule to which the pin group applies. This parameter is not required if the pin group being created is for the top level. *Data_type*: string, optional
`-compact_area`	Specifies that pins in the group are assigned tightly together. By default, pins which are part of a group and associated to a guide can be spread inside the guide based on the area available and alignment to targets. The `compact_area` option, when used, ignores the alignment and available area and forces the pins to the minimum area on guide honoring the minimum spacing constraint and overlooking any alignment to targets.  *Data_type*: bool, optional
`-excludePin {allLayer | sameLayer | allLayerInGuidedArea }`	
	Specifies that pins in the group exclude foreign pins on all layers, same layer, or on all layers in the guided area. • `allLayer`: Specifies that pins in the group exclude foreign pins between grouped pins on all layers. • `sameLayer`: Specifies that pins in the group exclude foreign pins between grouped pins on the same layer. • `allLayerInGuidedArea`: Specifies that pins in the group exclude foreign pins in the guide area defined for the group, on all layers. **Note**: If a guide is not defined for a pin group, `-excludePin allLayerInGuidedArea` option is treated like `-excludePin {allLayer}`. *Data_type*: enum, optional
`-help`	Outputs a brief description that includes type and default information for each `createPinGroup` parameter. For a detailed description of the command and all of its parameters, use the man command: `man createPinGroup`
`-keep_out_spacing integer`	
	Specifies the minimum spacing between the pins named in the pin group and any other pins, in number of tracks. *Minimum*: 0 *Maximum*: 65535 *Data_type*: int, optional

`-optimizeOrder`	Specifies that pins in the group are to be reordered to optimize wire length during pin placement. Otherwise the pin order is exactly as specified in the pin group. *Data_type*: bool, optional
`-pin {pinNameList}`	Specifies the pins to be added to the pin group. Additional pins can be added subsequently with the `addPinToPinGroup` command. *Data_type*: string, optional
pinGroupName	Specifies the name of the pin group. *Data_type*: string, required
`-spacing integer`	Specifies the minimum spacing between the pins of the pin group, in number of tracks. *Default*: Two tracks. *Data_type*: int, optional
`-spreadPin`	Specifies that pins in the group are distributed evenly. *Data_type*: bool, optional
`-width width`	Specifies the pin width, in microns. *Data_type*: float, optional

Example

- The following command creates a pin group PG_A22 for module SH17 and adds the pins pinA22_1 and pinA22_2 to it.
  ```
  createPinGroup PG_A22 -cell SH17 -pin {pinA22_1 pinA22_2}
  ```
- The following command creates a pin group PG_A for module SH17 and adds the pins pinA22_1 and pinA22_2 to it. No other pins are to be placed between pinA22_1 and pinA22_2 in any layer.
  The pins may be reordered to optimize wire length.
  ```
  createPinGroup PG_A22 -cell SH17 -pin {pinA22_1 pinA22_2} -excludePin allLayer -optimizeOrder
  ```
- The following command deletes the pin group PinGroup17 of module SH17:
  ```
  deletePinGroup -cell SH17 -pinGroup PinGroup17
  ```

Related Information

- `assignIoPins`
- `assignPtnPin`
- `addPinToPinGroup`
- `deletePinGroup`
- `createPinGuide`
- `setPinAssignMode`
- `deletePinFromPinGroup`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Setting Pin Constraints
  - Pin Group

createPinGuide

```
createPinGuide
[-help]
{[ -area {x1 y1 x2 y2} | [-edge integer [[-offset_start float] [-offset_end float]]][ -layer {layeridList} [-layerPriority ]]}
 [-pin string | -pinGroup string | -net string | -netGroup string | -name string ]
 [-edge integer | -net string | -netGroup string]
 [-cell string | -net string | -netGroup string]
```

Creates a pin guide object. The pin guide restricts the pins associated with the pin guide within the specified area. You can run this command on the top-level module also. You can assign the same pin group or net group to multiple pin guides. Pin guides are generated for buses, nets, net groups, pins, and pin groups. For buses and nets that do not physically connect to the partition, pin guides will not be created. You can use this command during partition floorplanning.

Parameters

`-area x1 y1 x2 y2`	Specifies the coordinates of the pin guide area. • x1: Specifies the lower left x coordinate of the pin guide area. • y1: Specifies the lower left y coordinate of the pin guide area. • x2: Specifies the upper right x coordinate of the pin guide area. • y2: Specifies the upper right y coordinate of the pin guide area. *Data_type*: rect, required
`-cell string`	Specifies the name of the cell. This parameter is not required if you are using the command for a top-level module. *Data_type*: string, optional
`-edge integer`	Specifies the integer value where edge numbering starts from lower-left corner of a partition clock-wise. Edge 0 is the edge that has the smallest y value. *Data_type*: int, optional
`-help`	Outputs a brief description that includes type and default information for each `createPinGuide` parameter. For a detailed description of the command and all of its parameters, use the man command: `man createPinGuide`.
`-layer {layerIdList}`	Specifies the metal layer(s) for the pin guide. *Default*: You must specify the correct pin layer. If you do not specify the pin layer, Innovus will display an error instead of choosing the preferred layer to assign pins like the old behavior. **Note**: Pins will not be assigned on non-preferred layers. However, pins can be assigned on Metal1 layer if the `ptnAssignPinsOnLayerM1` global variable is set to 1. **Note**: Layers can be specified using the LEF layer names or layer ID numbers. *Data_type*: string, optional
`-layerPriority`	Assign pins based on the order of -layer specified by the user. *Data_type*: bool, optional

`-name string`	Specifies the name of the partition pin guide object. If you specify this parameter, the command will automatically detect whether the object is a pin, pin group, net, net group, or bus. In case there are different objects with the same name, the command selects the object with the following precedence: • If a cell name is specified, the command selects the object with the following precedence: a. pin b. pin group c. net d. net group or bus • If a cell name is not specified, the command selects the object with the following precedence: a. net b. net group or bus For example, if the value of objectName is P1, and there is a pin group named P1 as well as a bus named P1, the command will select the pin group named P1. *Data_type*: string, optional
`-net string`	For nets, specifies the name of the net. For buses, specifies the name of the bus or the bus radix. *Data_type*: string, optional
`-netGroup string`	Specifies the name of the net group. *Data_type*: string, optional
`-offset_end float`	Specifies the distance, in microns, from the ending corner of the edge. *Minimum Value*: 0 *Maximum Value*: 65535 *Data_type*: float, optional
`-offset_start float`	Specifies the distance, in microns, from the starting corner of the edge. *Minimum Value*: 0 *Maximum Value*: 65535 *Data_type*: float, optional
`-pin string`	Specifies the name of the pin. *Data_type*: string, optional
`-pinGroup string`	Specifies the name of the pin group. *Default*: By default the name is defRGuideName. *Data_type*: string, optional

Example

- The following command creates a pin guide object for bus addr:
  ```
  createPinGuide -area 2353.8000 7123.000 2653.8000 8298.2000 -name addr
  ```

Related Information

- `deletePinGuide`
- `selectPtnPinGuide`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Setting Pin Constraints
  - Pin Guides

createPtnCut

```
createPtnCut
[-help]
[-ptn string]
x1 y1 x2 y2
```

Creates a partition rectangular cut from a partition. This object can be created after specifying a module guide to be a partition. Rectangular cuts can only occur on the edge of the partition guide. You can use this command after specifying partition.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `createPtnCut` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man createPtnCut`
x1 y1 x2 y2	Specifies the coordinates of the cut area (*x1 y1 x2 y2*). *Data_type*: rect, required
`-ptn` *string*	Specifies a partition to create a partition cut. If this option is not selected, the command loops through all the partitions an attempts to find the overlapped partition, then creates a partition cut for that partition. *Data_type*: string, optional

Example

The following command creates a partition cut area to modify a rectangular partition:

```
createPtnCut 2353.8000 7123.000 2653.8000 8298.2000
```

Related Information

- Partitioning the Design chapter in the *Innovus User Guide*

createPtnFeedthrough

```
createPtnFeedthrough
[-help]
feedBox
[layerId]
[-name string]
```

Creates a partition routing feedthrough object. The object area keeps pins from being created on the specified metal layer. A dump file with routing obstruction is created for the top level partition (done when saving the partition). You can use this command during partition floorplanning.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `createPtnFeedthrough` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man createPtnFeedthrough`
`feedBox`	Specifies the coordinates *(x1 y1 x2 y2)* of the feedthrough area. *Data_type*: rect, optional
`layerId`	Specifies the metal layer where pins are not to be created. **Note:** If *`layerId`* is not specified then by default feed-through is created on all the partition layers. *Data_type*: int, optional
`-name` *`string`*	Specifies the name of the routing feedthrough. *Data_type*: string, optional

Example

The following command creates a feedthrough object to block pins being generated on layer *Metal5*:

```
createPtnFeedthrough 2353.8000 7123.000 2653.8000 8298.2000 5
```

Related Information

- `insertPtnFeedthrough`
- `addRepeaterByRule`
- Inserting Feedthroughs in the *Innovus User Guide*

definePartition

```
 definePartition
[-help]
[-coreSpacing {left right top bottom}]
[-minPitchBottom x]
[-minPitchLeft x]
[-minPitchRight x]
[-minPitchTop x]
[-pinLayerBottom {list_of_layers}]
[-pinLayerLeft {list_of_layers}]
[-pinLayerRight {llist_of_layers}]
[-pinLayerTop {llist_of_layers}]
[-placementHalo {left right top bottom }]
[-railwidth railWidth]
[-reservedLayer {list_of_layers}]
[-routingHalo floatValue]
[-routingHaloBottomLayer integerValue]
[-routingHaloTopLayer integerValue]
[-stdCellHeight x]
{-hinst string | -proto {master_hinst_name_list} | -cell cellName}
[-checkCloneAlignment ]
[-hinst hInstName [-copyFrom refPtnName]]
[-cell cellName [-copyFrom refPtnName]]
```

Defines a new partition corresponding to a hierarchical instance. The `definePartition` command supports nested partitions and can be used to specify partitions inside a partition. While specifying partitions, you can specify the definition in any order. You can use this command after importing the design.

Note: The `definePartition` command does not check for fence constraint on clones and does not align them to the partition with respect to the master. If the clones do not have fence constraint, `definePartition` makes fences for clone modules.

Note: For designs that have backside metal layers, both the backside and frontside layers are accepted. However, the automatic pin assignment flow is not impacted with the support for backside metal layers. It assigns pins on front layers only.

Parameters

`-cell` *cellName*	Specifies the name of the cell for defining a new partition at an instance that has R0 orientation. **Note**: For black-boxes, an instance with non-R0 orientation is chosen if an instance with R0 orientation is not found. *Data_type*: string, optional
`-checkCloneAlignment`	When specified, the `definePartition` command only checks the alignment of the clone with the master partition and reports a mismatch. **Note**: The `definePartition` command does not move the clones. *Data_type*: bool, optional
`-copyFrom` *refPtnName*	Specifies the name of the reference partition from which to copy their properties. While copying the constraints, the shape, orientation, and location are considered: • **Orientation**: The orientation of the partition being defined is copied from the reference partition (*`refPtnName`*). If the reference partition has clones, then the orientation of the master is considered. • **Shape**: The shape of the partition being defined is copied from the reference partition (*`refPtnName`*). • **Location**: The fence location of the partition hinst being defined takes priority. In case the target partition hinst does not have a fence constraint (location), the location information is copied from the reference partition master hinst. This will result in overlapping fences. **Note:** If the `-copyFrom` parameter is being used with the -cell parameter that has multiple Hinsts, then the location of the hinst that has a fence (location) constraint is considered as the master of the partition being defined. *Data_type*: string, optional
`-coreSpacing {left right top bottom}`	

	Specifies the space, in micrometers, between the module boundary and core design area of the partition module on the left, right, top, and bottom sides respectively. The partition pins will be located at the partition instance boundary and the core area. **Note**: These core-to-edge values are always synchronized between the partition and the corresponding power domain as follows: If the specified instance was also earlier defined as a power domain, these values for the partition, by default, will have the same value as the values for the power domain. If you specify different values for the partition, the values for the power domain are updated so that they are the same as the values defined for the partition. Similarly, if a power domain is created (or an existing power domain modified) on the specified instance after the partition is defined, the values set earlier for the partition are updated so that they are the same as the values defined for the power domain. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `definePartition` parameter. For a detailed description of the command and all of its parameters, use the man command: `man definePartition`
`-hinst` *`hInstName`*	Specifies the name of the hierarchical instance. **Note**: None of its ancestors or descendants in the netlist tree can be a partition. *Data_type*: string, optional
`-minPitchBottom` *`x`*	Specifies the pin pitch dimension for the bottom partition side. The default is 2, which places one pin for every two metal tracks. The actual pin pitch depends on the pin's metal layer. The Partition Pin Guide floorplan object will supersede this entry and use the Min. Space entry in the object's Attribute Editor. *Data_type*: int, optional
`-minPitchLeft` *`x`*	Specifies the pin pitch dimension for the left partition side. The default is 2, which places one pin for every two metal tracks. The actual pin pitch depends on the pin's metal layer. The Partition Pin Guide floorplan object will supersede this entry and use the Min. Space entry in the object's Attribute Editor. *Data_type*: int, optional
`-minPitchRight` *`x`*	Specifies the pin pitch dimension for the right partition side. The default is 2, which places one pin for every two metal tracks. The actual pin pitch depends on the pin's metal layer. The Partition Pin Guide floorplan object will supersede this entry and use the Min. Space entry in the object's Attribute Editor. *Data_type*: int, optional
`-minPitchTop` *`x`*	Specifies the pin pitch dimension for the top partition side. The default is 2, which places one pin for every two metal tracks. The actual pin pitch depends on the pin's metal layer. The Partition Pin Guide floorplan object will supersede this entry and use the Min. Space entry in the object's Attribute Editor. *Data_type*: int, optional
`-pinLayerBottom {`*`list_of_layers`*`}`	Specifies the metal layers to use for the partition. The metal layers correspond to the sides of the partition, where the vertical metal layers (for example, Metal2, Metal4, and so on) are for the Top and Bottom sides, and horizontal metal layers (for example, Metal3, Metal5, and so on) are for the Left and Right sides of the partition. Specify this value in integers, for example `-pinLayerBottom {2 4}` **Note**: Only layers in their preferred routing direction are accepted. **Note**: If these option is not supplied, all allowed layers in preferred routing direction will be assumed for that side. **Note:** This pin layer option does not support backside metal layers, it only supports frontside layers. *Default*: All allowed layers in preferred direction. *Data_type*: string, optional

`-pinLayerLeft {list_of_layers}`	Specifies the metal layers to use for the partition. The metal layers correspond to the sides of the partition, where the vertical metal layers (for example, Metal2, Metal4, and so on) are for the Top and Bottom sides, and horizontal metal layers (for example, Metal3, Metal5, and so on) are for the Left and Right sides of the partition. Specify this value in integers, for example `-pinLayerLeft {3 5}` **Note**: Only layers in their preferred routing direction are accepted. **Note**: If these option is not supplied, all allowed layers in preferred routing direction will be assumed for that side. **Note:** This pin layer option does not support backside metal layers, it only supports frontside layers. *Default*: All allowed layers in preferred direction. *Data_type*: string, optional
`-pinLayerRight {list_of_layers}`	Specifies the metal layers to use for the partition. The metal layers correspond to the sides of the partition, where the vertical metal layers (for example, Metal2, Metal4, and so on) are for the Top and Bottom sides, and horizontal metal layers (for example, Metal3, Metal5, and so on) are for the Left and Right sides of the partition. Specify this value in integers, for example `-pinLayerRight {3 5}` **Note**: Only layers in their preferred routing direction are accepted. **Note**: If these option is not supplied, all allowed layers in preferred routing direction will be assumed for that side. **Note:** This pin layer option does not support backside metal layers, it only supports frontside layers. *Default*: All allowed layers in preferred direction. *Data_type*: string, optional
`-pinLayerTop {list_of_layers}`	Specifies the metal layers to use for the partition. The metal layers correspond to the sides of the partition, where the vertical metal layers (for example, Metal2, Metal4, and so on) are for the Top and Bottom sides, and horizontal metal layers (for example, Metal3, Metal5, and so on) are for the Left and Right sides of the partition. Specify this value in integers, for example `-pinLayerTop {2 4}` **Note**: Only layers in their preferred routing direction are accepted. **Note**: If these option is not supplied, all allowed layers in preferred routing direction will be assumed for that side. **Note:** This pin layer option does not support backside metal layers, it only supports frontside layers. *Default*: All allowed layers in preferred direction. *Data_type*: string, optional
`-placementHalo {left right top bottom}`	
	Specifies extra spacing, in micrometers (around the left, bottom, right and top sides respectively) of the partition that should not be used for placement. The placement program will not place standard cells in the area for partition placement. This spacing is called placement halo. At the top-level design, this information is saved as part of the partition section in a floorplan file. This information is also saved in a partition floorplan file when saving partitions. By default, the value is 0 for all the sides. If the hierarchical instance was earlier defined as a power domain, then this field corresponds to the minimum gap parameter of the power domain. **Note**: The minimum gap value of the power domain and the placement halo values of the partition are always synchronized as follows: If the specified instance was also earlier defined as a power domain, the placement halo values for the partition, by default, will have the same value as the minimum gap values for the power domain. If you specify a different value for the partition, the value for the power domain is updated so that it is the same as the value defined for the partition. Similarly, if a power domain is created (or an existing power domain modified) on the specified instance after the partition is defined, the value set earlier for the partition is updated so that it is the same as the value defined for the power domain. **Note**: `placementHalo` created by `definePartition` cannot be honored by `place_design` before partitions have not been committed. *Data_type*: string, optional
`-proto {master_hinst_name_list}`	Specifies the master hierarchical instances list for the prototyping flow. It defines new partitions for the specified hierarchical instances with guide constraints. It is used before `proto_design`, which will convert the physical constraint from guide to fence finally. *Data_type*: string, optional

`-railwidth railWidth`	Specifies the standard cell power rail width, in micrometers. If partition pins are not allowed above power or ground rail width, specify the dimension of the power rail width. If a value of 0.0 is entered (the default), the width from the Technology file will be used. *Data_type*: float, optional
`-reservedLayer {list_of_layers}`	
	Specifies the metal layers that are used for routing in the partition and generating partition pins. Any metal layers that are not specified, usually the top-most metal layers, are allowed to route over the partition. A normal six-metal layer specification is Metal1, Metal2, Metal3, Metal4 and Metal5 specified, and Metal6 unspecified. When saving the partition, the LEF generated for this partition will have routing blockages on their layers so that the top-level router is aware of which metal layers are being used in the partition. **Note:** For designs that have backside metal layers, both the backside and frontside layers are accepted. *Data_type*: string, optional
`-routingHalo floatValue`	
	Specifies routing spacing, in micrometers, around the sides of the partition. A routing halo is honored only by the signal router. The signal router treats routing on the specific routing layers in the routing halo area as very high cost routing. However, perpendicular routing or straight connections to pins are acceptable. The special router does not honor routing halos. To block special routing, use routing blockages instead. You can specify a positive or a negative value for the routing halo. A positive value means that the halo will be outside the partition. A negative value means that the halo will be applicable during block implementation, inside of the partition. The same value is used for all sides of the partition. **Note**: For instance blocks - blackboxes, hard macros or block-level designs - specify the routing halo through the *Floorplan - Edit Floorplan - Edit Halos* form in the GUI or through the `addRoutingHalo` command. Also, for bottom- up hierarchical flow, specify routing halo for a block at the top-level design or at the block-level design through the *Floorplan - Edit Floorplan - Edit Halos* form in the GUI or through the `addRoutingHalo` command. *Data_type*: float, optional
`-routingHaloTopLayer integerValue`	
	Specifies the top partition layer for which routing halo will be created. *Default*: By default, the top layer reserved for the partition is used. *Data_type*: int, optional
`-routingHaloBottomLayer integerValue`	
	Specifies the bottom partition layer for which routing halo will be created. *Default*: By default, the bottom layer reserved for the partition is used. *Data_type*: int, optional
`-stdCellHeight x`	Specifies the standard cell height for the partition. *Default*: The default value is derived from the default technology site and row height. *Data_type*: float, optional

Example

- The following example defines a new partition:

```
definePartition -hinst ctr_inst
-coreSpacing 0.56 0.56 0.0 0.0
-railWidth 0.0
-minPitchLeft 2 -minPitchRight 2
-minPitchTop 2 -minPitchBottom 2
-reservedLayer {1 2 3 4}
-pinLayerTop {2 4} -pinLayerLeft {3} -pinLayerBottom {2 4} -pinLayerRight {3}
-placementHalo 10.0 0.0 0.0 0.0 -routingHalo 10.0
-routingHaloTopLayer 7 -routingHaloBottomLayer 1
```

- The following commands define a nest partition where PTN1 is a parent partition and PTN2 is a child:

```
definePartition -hinst PTN1
definePartition -hinst PTN2
```

or

```
definePartition -hinst PTN2
definePartition -hinst PTN1
```

Related Information

- `partition`
- `deletePartition`
- `savePartition`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Defining Partitions
  - Working with Nested Partitions

deleteAllPtnCuts

```
deleteAllPtnCuts
[-help]
```

Deletes all the partition cuts in the floorplan. You can use this command after specifying a partition.

Note: Command `deleteAllPtnCuts` is now obsolete and will be removed in the next major release of the software. This command has been replaced by the `deletePtnCut` command.

Parameters

`-help`	Outputs a brief description of the `deleteAllPtnCuts` command. For a detailed description of the command, use the `man` command: `man deleteAllPtnCuts`

Related Information

- `partition`
- `deletePtnCut`
- `deletePartition`
- Partitioning the Design chapter in the *Innovus User Guide*

deleteAllPtnFeedthroughs

```
deleteAllPtnFeedthroughs
[-help]
```

Deletes all the partition feedthrough objects in the floorplan. You can use this command during partition floorplanning.

Parameters

`-help`	Outputs a brief description of the `deleteAllPtnFeedthroughs` command. For a detailed description of the command, use the `man` command: `man deleteAllPtnFeedthroughs`

Example

The following command removes all feedthrough objects from the floorplan:

```
deleteAllPtnFeedthroughs
```

Related Information

- Partitioning the Design chapter in the *Innovus User Guide*
- Inserting Feedthroughs

deleteNetFromNetGroup

```
deleteNetFromNetGroup
[-help]
netGroupName
-net {netName | netNameList}
```

Removes a net from a net group. You can use this command after importing the design.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `deleteNetFromNetGroup` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man deleteNetFromNetGroup`
`netGroupName`	Specifies the name of the net group. You can use wildcards (`*?`) with this parameter. *Data_type*: string, required
`-net {`*`netName`* `|` *`netNameList`*`}`	Specifies the name of the net(s). You can use wildcards (`*?`) with this parameter. *Data_type*: string, required

Example

The following command removes net `NET1` from net group `NG22`:

```
deleteNetFromNetGroup NG22 NET1
```

Related Information

- `addNetToNetGroup`
- `createNetGroup`
- `deleteNetGroup`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Setting Pin Constraints
  - Net Group

deleteNetGroup

```
deleteNetGroup
[-help]
netGroupName
```

Removes a net group that was created earlier.

Note: When you delete a net group, any bus guide associated with the net group also gets deleted.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `deleteNetGroup` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man deleteNetGroup`
`netGroupName`	Specifies the name of the net group. You can use wildcards (`*?`) with this parameter. *Data_type*: string, required

Example

The following command removes netgroup `NG_22`.

```
deleteNetGroup NG_22
```

Related Information

- `addNetToNetGroup`
- `createNetGroup`
- `deleteNetFromNetGroup`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Setting Pin Constraints
  - Net Group

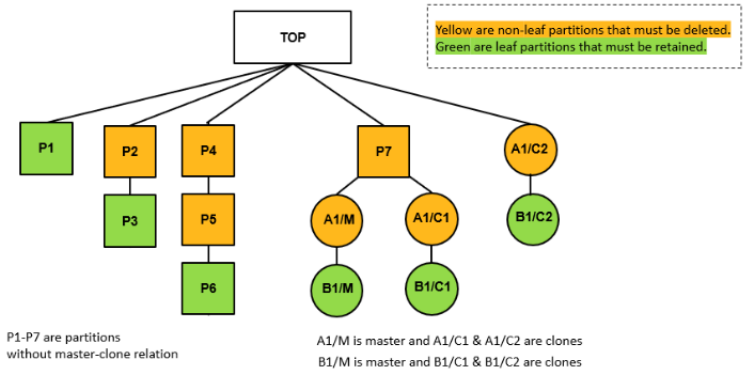
deletePartition

```
deletePartition
[-help]
{ partition_name | -all | -nested | -non_leaf }
[-undo_tcl_file file_name [-non_leaf ]]
```

Deletes the specified partition.

Note: The Hinst retains the fence property.

Parameters

`-all`	Deletes all partitions. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `deletePartition` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man deletePartition.`
`-nested`	Deletes all nested partitions in the design. *Data_type*: bool, optional
`-non_leaf`	Deletes all non-leaf partitions in the design. **Note:** It saves the information of the deleted partitions in a tcl file, so that the same can be used to restore the deleted partitions. For example, in the diagram below, all the non-leaf partitions (marked in yellow) must be deleted and dumped in a tcl file for restoration purpose later on.  *Data_type*: bool, optional
partition_name	Specifies the name of the partition to delete. *Data_type*: string, optional
`-undo_tcl_file` *file_name*	
	Specifies the file that can be sourced to undo deletion of partitions. With this parameter, the partition definition of the partition being deleted is written to the specified file. Later, if required, this file can be sourced to bring back the partition definition. *Data_type*: string, optional

Example

- The following command deletes the partition `p1`.
  ```
  deletePartition p1
  ```
- The following command writes the partition definition of partition p1 in file ptn.tcl and then deletes it.
  ```
  deletePartition p1 -undo_tcl_file ptn.tcl
  ```

Related Information

- `partition`
- `definePartition`
- Partitioning the Design chapter in the *Innovus User Guide*

deletePinBlkg

```
deletePinBlkg
[-help]
{-name pinBlkgName | -all}
```

Deletes a pin blockage or all pin blockages.

Parameters

`-all`	Specifies that all pin blockages will be deleted. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `deletePinBlkg` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man deletePinBlkg`
`-name pinBlkgName`	Specifies the name of the pin blockage. You can specify wildcards (?*) with this parameter. *Data_type*: string, optional

Example

The following command deletes the pin blockage `PtnPinBlkg1`:

```
deletePinBlkg -name PtnPinBlkg1
```

Related Information

- `createPinBlkg`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Setting Pin Constraints
  - Pin Blockage

deletePinFromPinGroup

```
deletePinFromPinGroup
[-help]
[-cell cellName]
-pinGroup pinGroupName
-pin {pinName | pinNameList}
```

Removes a port from a specified pin group.

Parameters

`-cell cellName`	Specifies the name of the module. The module name is not required if the command is run on a top-level module. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `deletePinFromPinGroup` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man deletePinFromPinGroup`
`-pinGroup pinGroupName`	Specifies the name of the pin group that was created. *Data_type*: string, required
`-pin {pinName | pinNameList}`	Specifies the name(s) of the port(s) of the module. *Data_type*: string, required

Example

The following command removes ports `RASN1` and `RASN2` of module `SH17` from pin group `pin17`:

```
deletePinFromPinGroup -cell SH17 -pinGroup pin17 -pin {RASN1 RASN2}
```

Related Information

- `addPinToPinGroup`
- `createPinGroup`
- `deletePinGroup`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Setting Pin Constraints
  - Pin Group

deletePinGroup

```
deletePinGroup
[-help]
[-cell cellName]
-pinGroup pinGroupName
```

Deletes a pin group or all pin groups that were created earlier.

Parameters

`-cell cellName`	Specifies the name of the module. The module name is not required if the command is run on a top-level module. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `deletePinGroup` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man deletePinGroup`
`-pinGroup pinGroupName`	Specifies the name of the pin group that was created. Wildcards accepted. *Data_type*: string, required

Example

- The following command deletes the pin group `PinGroup17` of module `SH17`:
  ```
  deletePinGroup -cell SH17 -pinGroup PinGroup17
  ```
- The follow command deletes all the pin groups of module SH17:
  ```
  deletePinGroup -cell SH17 -pinGroup *
  ```

Related Information

- `createPinGroup`
- `addPinToPinGroup`
- `deletePinFromPinGroup`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Setting Pin Constraints
  - Pin Group

deletePinGuide

```
deletePinGuide
[-help]
{ -pin pin_name | -pinGroup pin_group_name | -net net_name | -netGroup net_group_name | -name object_name | -all }
[-cell ptn_name | -net net_name | -netGroup net_group_name]
```

Deletes a pin guide object in the floorplan. You can use this command during partition floorplanning.

Parameters

`-all`	Specifies that all pin guides will be deleted. *Data_type*: bool, optional
`-cell` *ptn_name*	Specifies the name of the module. The module name is not required if the command is run on a top-level module. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `deletePinGuide` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man deletePinGuide`
`-name` *object_name*	Specifies the name of the partition pin guide object. If you specify this parameter, the command will automatically detect whether the object is a pin, pin group, net, net group, or bus. In case there are different objects with the same name, the command selects the object with the following precedence: • If a cell name is specified, the command selects the object with the following precedence: 1. pin 2. pin group 3. net 4. net group or bus • If a cell name is not specified, the command selects the object with the following precedence: 1. net 2. net group or bus For example, if the value of *objectName* is `P1`, and there is a pin group named `P1` as well as a bus named `P1`, the command will select the pin group named `P1`. *Data_type*: string, optional
`-net` *net_name*	For nets, specifies the name of the net. For buses, specifies the name of the bus or the bus radix. *Data_type*: string, optional
`-netGroup` *net_group_name*	Specifies the name of the net group. *Data_type*: string, optional
`-pin` *pin_name*	Specifies the name of the pin. *Data_type*: string, optional
`-pinGroup` *pin_group_name*	Specifies the name of the pin group. *Default:* By default the name is `defRGuideName`. *Data_type*: string, optional

Example

The following command deletes a pin guide object named `group_1`:

```
deletePinGuide -name group_1
```

Related Information

- `createPinGuide`
- `selectPtnPinGuide`
- Partitioning the Design chapter in the *Innovus User Guide*

deletePtnCut

```
deletePtnCut
[-help]
{-ptn {ptnName | ptnNameList} | -all}
```

Deletes all the partition cuts.

Parameters

`-all`	Deletes all the partition cuts. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes type and default information for each `deletePtnCut` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deletePtnCut`
`-ptn {ptnName | ptnNameList}`	Specifies the name of the partition, or the list of partitions for which partition cuts are deleted. *Data_type*: string, optional

Example

The following command deletes partition cuts for partition `sheet17`:

```
deletePtnCut -ptn sheet17
```

Related Information

- `partition`
- `deleteAllPtnCuts`
- `deletePartition`
- Partitioning the Design chapter in the *Innovus User Guide*

editPin

```
editPin
[-help]
[-cell cellName | {-hinst hinstName [-refMaster]}]
[-end {x y}]
[-fixedPin]
[-global_location]
[-include_rectilinear_edge]
[-masterCloneAware]
[-offsetEnd value]
[-offsetStart value]
-pin {pinName | pinNameList}
[-pinDepth pinDepthValue]
[-pinWidth pinWidthValue]
[-quiet]
[-skipWrappingPins]
[-snap {TRACK | USERGRID | MGRID}]
[-spreadDirection {clockwise | counterclockwise | both}]
[-start {x y}]
[-use { SIGNAL | CLOCK | ANALOG }]
[{-layer {layerId | layerIdListName} |
 {[-layerH layerId] [-layerV layerId]}}
 [-layer_priority]
 [-assign {x y} |
  -spreadType {START | CENTER | SIDE | EDGE | RANGE} |
  -pattern {fill_track | fill_layer | fill_optimised | fill_diagonal | fill_sinusoidal | fill_checkerboard}]]
[-side sideName | -edge edgeNumber]
[-fixOverlap {0|1} [-honorConstraint {0|1}]]
[-pattern {fill_track | fill_layer | fill_optimised | fill_diagonal | fill_sinusoidal | fill_checkerboard} [-reverse_alternate]]
[-spacing spacingValue [-unit {MICRON | TRACK}]]
```

Modifies properties of pins, such as pin spreading, pin location, pin width and depth, spacing, snap-to location, and status. The `editPin` command provides the equivalent capabilities of the *Pin Editor* form.

Note: The `editPin` command does not support wildcards.

The `editPin` command honors the minimum area rule as follows:

- If neither the `-pinWidth` nor the `-pinDepth` parameters have been specified, the `editPin` command takes the minimum pin width and the minimum area from the LEF file and calculates the pin depth accordingly.
- If only the `-pinWidth` parameter has been specified, the `editPin` command takes the minimum area from the LEF file and calculates the pin depth accordingly.
- If only the `-pinDepth` parameter has been specified, the `editPin` command takes the minimum area from the LEF file and calculates the pin width accordingly.

If both the `-pinWidth` and the `-pinDepth` parameters are specified, the `editPin` command takes these specified values, without enforcing the minimum area rule.

The `editPin` command is clone context aware. The command:

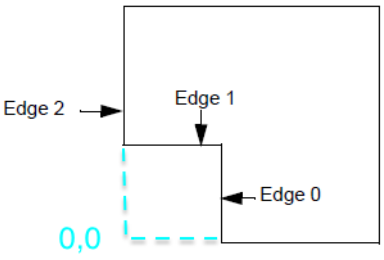
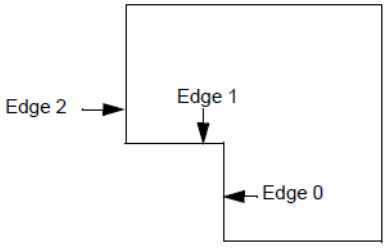
- Looks at routing blockages on clone and honors them while keeping pins of master with `-cell <>`.
- Looks at abutted macro blockages and honors them while keeping pins of master with `-cell <>`.
- Honors route blockage on master.

Note: The value of the parameters of the `editPin` command is not stored across multiple runs of the command – you need to specify the required value with every run of the command.
For example, if you do not specify the `-pinWidth` parameter, the default pin width of the specified layer is used.

Note: The `editPin` command checks and corrects DRC violations and overlapping while placing internal pins. It supports pin placement on Metal 1 and accepts all layers for pin placement by default.

Note: The `editPin` command is used for manual pin assignment in nested partitions. It places pins of nested partitions similar to single level partitions.

Parameters

`-assign {x y}`	Preassigns a pin at the specified location. You can use this parameter with other parameters such as `-snap`, `-fixOverlap` **Note**: -assign {*x y*} considers the local coordinates from the bounding box of the fence.  *Data_type*: point, optional
`-cell cellName`	Specifies the name of the module. If you do not specify a cell name, the command runs on the top-level cell. *Data_type*: string, optional
`-edge edgeNumber`	For rectilinear partitions, specifies the edge along which the pins will be placed. The edge number is an integer. The edg Edge 0 is the edge that has the smallest y value.  To spread pins on multiple rectilinear edges that are on the same side, use the `-side` parameter. For rectangular partitions, you can specify a side with the -side parameter. *Data_type*: int, optional
`-end {x y}`	For pin spreading along a range, specifies the coordinates where pins should end. This parameter is used with the `-spr` *Data_type*: point, optional
`-fixedPin`	A value of 1 sets the pin status to Fixed. *Default*: The pin status is placed. *Data_type*: bool, optional
`-fixOverlap {0 | 1}`	With this parameter on, the `editPin` command tries to resolve pin-overlap with other/nearby objects. The pins are moved *Default*: 1. Resolve pin-overlap is on. **Note**: This parameter applies only when pins are snapped to layer track. *Data_type*: bool, optional

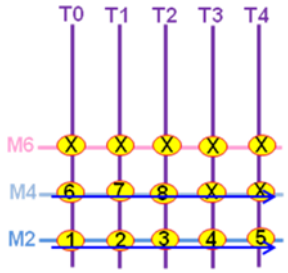
`-global_location`	Specifies that the global location should be considered by the command.
	Note: By default all locations that the `editPin` command considers are coordinates with respect to lower-left corner of pa accept coordinates with respect to the lower-left corner of the design.
	For example, in the following command, the pin will be placed 3u from the lower-left corner of the partition.
	`editPin -cell c -pin cIn -assign 0 3 -layer 2 -side left`
	When you use the `-global_location` parameter, the pin will be placed 3u from the lower-left corner of the design.
	`editPin -cell c -pin cIn -assign 0 3 -layer 2 -side left -global_location`
	Data_type: bool, optional

`-help`	Outputs a brief description that includes type and default information for each `editPin` parameter. For a detailed description of the command and all of its parameters, use the man command: `man editPin`
`-hinst` *hinstName*	Specifies the hinst name, including black box or committed partition and top. *Data_type*: string, optional
`-honorConstraint {0|1}`	Specifies whether user-defined pin constraints should be honored. The following constraints are honored when this parameter is set to on: • pin spacing • pin size • pin-to-corner distance • partition-allowed pin layers. *Default*: 0 (user-defined pin constraints are not honored) *Data_type*: bool, optional
`-include_rectilinear_edge`	Specifies that all the edges coming in the solution space should be included. *Data_type*: bool, optional
`-layer {`*layerId* `|` *layerName*`}`	Specifies the layer on which the pins will be assigned. Layers can be specified using the LEF layer names or layer ID nu **Note**: This parameter is optional but only when `-spreadType` is not specified. **Note:** For designs that have backside metal layers, both the backside and frontside layers are accepted. *Data_type*: bool, optional
`-layerH` *layerId*	Specifies the preferred horizontal layer on which the pins will be assigned. **Note:** For designs that have backside metal layers, both the backside and frontside layers are accepted. *Data_type*: string, optional
`-layerV` *layerId*	Specifies the preferred vertical layer on which the pins will be assigned. **Note:** For designs that have backside metal layers, both the backside and frontside layers are accepted. *Data_type*: string, optional
`-layer_priority`	Specifies that the input layer is given based on its priority. *Data_type*: bool, optional
`-masterCloneAware`	Enables master-clone context awareness for feasibility checking. *Data_type*: bool, optional
`-offsetEnd` *value*	Sets the offset from the end of the edge (in microns) to be considered as the endpoint for placing the pin. **Note**: The `-offsetStart` and `-offsetEnd` options can only be used with the `-edge` option. *Data_type*: float, optional
`-offsetStart` *value*	Sets the offset from the start of the edge (in microns) to be considered as the start point for placing the pin. **Note**: The `-offsetStart` and `-offsetEnd` parameters can only be used with the `-edge` parameter. *Data_type*: float, optional

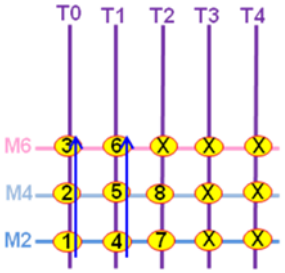
`-pattern {fill_track | fill_layer | fill_optimised | fill_diagonal | fill_sinusoidal | fill_checkerboard}`

Specifies the multi-layer-spread-pattern must be followed by the set of selected pins.

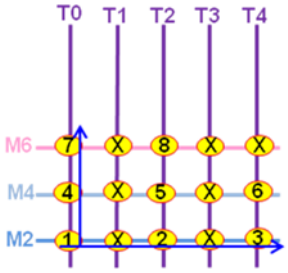
fill_track



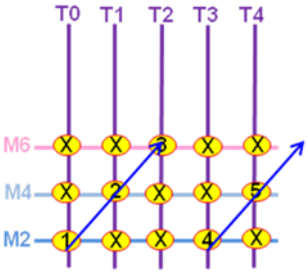
fill_layer



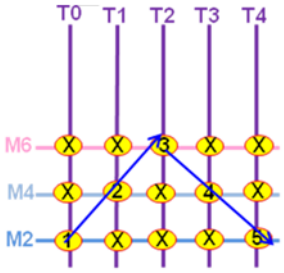
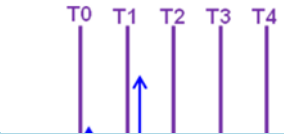
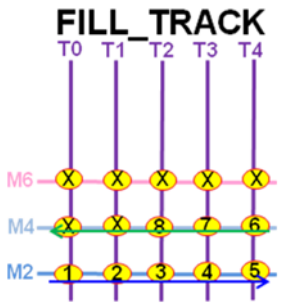
fill_optimised



fill_diagonal



fill_sinusoidal

	 fill_checkerboard 
`-pin {pinName | pinNameList}`	Specifies the pins, or the list of pins of the specified module on which the command will run. **Note**: The naming convention to identify individual ports is based on the order in which the ports are stored inside the da port1, port2, respectively. For example: VDD.port0, VDD.port1, VDD.port2 *Data_type*: string, required
`-pinDepth pinDepthValue`	Sets the depth of the pin in microns. *Default*: If you do not specify this parameter, the default depth of the specified layer is used. *Data_type:* float, optional
`-pinWidth pinWidthValue`	Sets the width of the pin in microns. *Default*: If you do not specify this parameter, the default width of the specified layer is used. *Data_type:* float, optional
`-quiet`	Specifies that the messages should be suppressed. *Data_type*: bool, optional
`-refMaster`	Specifies that the clone's edge, spreadDirection, and offset are with respect to the master. *Data_type*: bool, optional
`-reverse_alternate`	Specifies that the reverse of the multi-layer-spread-pattern must be followed by the set of selected pins. For example, `fill_track -reverse_alternate`  *Data_type*: bool, optional
`-side sideName`	For rectangular partitions, specifies the side on which the pins will be placed. The values for sideName can be: TOP, BC **Note**: The value INSIDE can only be specified with the `-assign` parameter. For rectilinear partitions, you can also specify an edge with the `-edge` parameter. **Note:** The `setPinAssignMode -pinEditInBatch` parameter does not support the internal pins (where the side is INSIDE). *Data_type*: string, optional

`-skipWrappingPins`	Skips pin wrapping on the adjacent edge or the same edge from the other side. *Data_type*: bool, optional
`-snap {TRACK | USERGRID | MGRID}`	
	Specifies how the pins will be snapped. • `TRACK`: Snaps pins to layer tracks. • `USERGRID`: Snaps pins to the user-defined grid. • `MGRID`: Snaps pins to the manufacturing grid. *Default*: `TRACK` *Data_type*: enum, optional
`-spacing` *spacingValue*	Displays the edge-track-to-edge-track spacing between pins for multi-pin editing. The spacing is either in micrometers o placed on every track with no empty tracks between them. Default: The spacing is set automatically. *Data_type*: string, optional
`-spreadDirection {clockwise | counterclockwise | both}`	
	Specifies the direction in which the pins are spread. You can specify: • `clockwise`: Sets the pins in the clockwise direction. • `counterclockwise`: Sets the pins in the counter clockwise direction. • `both`: Sets the pins by looking for the closest legal location to the desired location in both directions. *Default*: clockwise *Data_type*: enum, optional
`-spreadType {START | CENTER | SIDE | EDGE | RANGE}`	
	Specifies how to spread pins along a block's edge. • `START`: Uses the coordinates of the first pin specified with the `-pin` parameter as the starting point (anchor) for spreadir parameter. • `CENTER`: If a single pin is specified with the `-pin` parameter, the pin will be placed at the center of the specified edge/sic points such that the center pin(s) of the group are placed at the center of the specified edge or side. • `SIDE`: If you specify a side with the `-side` parameter and specify the `-spreadType SIDE` parameter, the pins are spread and from bottom to top. • `EDGE`: If you specify an edge with the `-edge` parameter and specify the `-spreadType EDGE` parameter, the pins are sprea points. • `RANGE`: Spreads pins evenly along the block edge between points you specify with `-start {x y}` and `-end {x y}` para Editor determines the appropriate spacing. **Note:** For rectilinear partitions, if the start and/or end points are not on the specified side, the start and/or end points are s *Data_type*: enum, optional
`-start {x y}`	For pin spreading along a range, specifies the coordinates where pins should start. This parameter is used with the `-spr` *Data_type*: point, optional
`-unit {MICRON | TRACK}`	Specifies the unit in microns or in tracks. This parameter applies only to the unit of the -spacing parameter. *Default*: `Micron` *Data_type*: enum, optional
`-use {SIGNAL | CLOCK | ANALOG}`	
	Specifies the +USE property of the partition pin. *Data_type*: stringl, optional

Example

The following command modifies the pins as specified here:

- the command runs on the module DTMF_CORE_INST1
- the command runs on all pins whose name starts with DTMF_CORE_INST1_OUTPIN
- the pins are spread on the edge number 2
- the USE property is set to CLOCK
- the pins are spread between the coordinates (17 23) and (117 123)
- the pins are assigned on the layer Metal3
- the pin status is set to Fixed
- the pins are snapped to layer tracks
- the unit is in tracks
- the spacing between pins for multi-editing is set to 2 tracks
- the pin depth is set to 10 microns.

  Note: The `-unit` parameter is set to TRACK but that does not affect the unit of pin depth.
- the pins are moved to the nearest acceptable location as the `-fixOverlap` parameter is set to 1. However, the `-overlapConstraints` parameter is not specified.

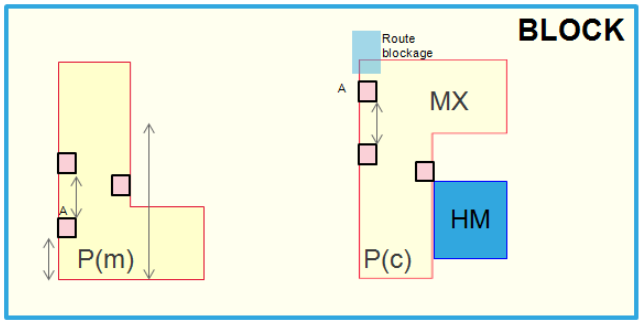
```
editPin
-cell DTMF_CORE_INST1
-pin {DTMF_CORE_INST1_OUTPIN}
-spreadType RANGE
-edge 2
-layer Metal3
-use CLOCK
-fixOverlap 1
-snap TRACK
-pinDepth 10
-start {17 23}
-end {117 23}
-spacing 2
-unit TRACK
-fixedPin 1
```

- The `editPin` command looks at routing blockages on clone and honors them. In the figure below, the following command would not start pins on master from Y=0 as there is a route blockage on clone at desired location.

  ```
  editPin -hinst p(m) -spreadType start -start 0 0 -edge 0 -layer 2 -pin A*
  ```

  Also, the following command would not start pins on master from Y=10 as there is a macro with obstruction on clone at desired location.

  ```
  editPin -hinst p(m) -assign 10 5 -edge 2 -layer 2 -pin B -spreadDirection {both}
  ```



- The following commands are used to edit pins in the batch mode:

  ```
  setPinAssignMode -pinEditInBatch true
  ```

```
editPin -hinst p(m) -spreadType start -start 0 0 -edge 0 -layer 2 -pin A*
...
...
editPin -hinst p(m) -assign 10 5 -edge 2 -layer 2 -pin B -spreadDirection {both}
setPinAssignMode -pinEditInBatch false
```

Related Information

- `setPinAssignMode`
- `legalizePin`
- `assignPtnPin`
- `assignIoPins`
- "Editing Pins" in the *Innovus User Guide*.
- "Assigning Pins" in the *Innovus User Guide*.
- "Pin Editor" in the *Innovus Menu Reference*.

fix_feedthru_assign

```
fix_feedthru_assign
[-help]
-buffer string
[-delete_feed]
```

Fixes the assign statements that are associated with partitions by replacing them with the specified buffer. This helps to minimize the number of buffers that are added to the design when assign statements are deleted.

The `fix_feedthru_assign` command enables selective replacement of assign statements with real buffers.

Parameters

`-buffer` *`string`*	Specifies the buffer to be used for replacing assign statements. *Data_type*: string, required
`-delete_feed`	Removes input-output(s) assigns from within the partition module. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `fix_feedthru_assign` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man fix_feedthru_assign`

Example

The following command replaces the assign statements associated with partitions with the specified buffer.

```
fix_feedthru_assign -buffer abc
```

Related Information

- `remove_assigns`
- `insertPtnFeedthrough`
- Partitioning the Design chapter in the *Innovus User Guide*

flattenCoverCell

```
flattenCoverCell
[-help]
[-inst {instanceName | instanceNameList}]
```

Dissolves and flattens any instance whose master has class `COVER`, but no subclass `BUMP`. You can use this command after loading a DEF file (after the defIn command).

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `flattenCoverCell` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man flattenCoverCell`
`-inst {instanceName | instanceNameList}`	
	Specifies the instance, or a list of instances, to be flattened. If you are specifying a list, separate the instance names with a space. *Default*: If you do not specify this parameter, all instances whose masters have class `COVER` are flattened. *Data_type*: string, optional

Examples

- The following command flattens the cover cells named `coverCell1` and `coverCell2`
  ```
  flattenCoverCell -inst {coverCell1 coverCell2}
  ```
- The following command flattens *all* cover cells
  ```
  flattenCoverCell
  ```

Related Information

- `defIn`
- Partitioning the Design chapter in the *Innovus User Guide*

flattenPartition

```
flattenPartition
[-help]
[partition_name]
[-bringBackRow]
[-keepPinGeometry]
[-noUninheritPhysical]
```

Uncommits (flattens) the partition back to normal module status. This command should also be used to unassign blackbox pins *after* the blackbox is committed. To unassign blackbox pins *before* the blackbox is committed, use the `setPtnPinStatus` command. This command unassigns partition pins as well. To bring back row information for a partition, use the `-bringBackRow` parameter. If the partition rows are non-integer-multiple-height rows or are not aligned with the top-level rows, the `flattenPartition` command will automatically create a power domain associated with the partition. The power domain will have the following characteristics:

- The timing library will the same as that for the top level.
- If a global net connection is defined in the partition block, the power domain will have the same global net connection. Otherwise the global net connection for the power domain will be set to the top-level net connection.
- The minimum gap values will be the same as the placement halo values for the partition.

The recommended use model for bringing back non-integer-multiple height rows or unaligned rows is as follows.

Top-down flow:

1. Create a power domain at full chip-level design.
2. Specify the same hierarchical instance as the partition

Bottom-up flow:

1. Create a power domain at top-level design.
2. Assign the instance block as a member of the power domain created in step 1.

The `flattenPartition` command supports partitions with FlexModels. It restores back FlexModel information if the committed partition has a FlexModel netlist.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `flattenPartition` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man flattenPartition`
`-bringBackRow`	Specifies that partition row information should be brought back for cases where: • the partition is not a power domain and row information in partition is different from top-level design, *and* • after uncommiting the partition, the rows of the partition space are not aligned with the top-level rows When you use this parameter, a power domain is created for the partition. *Default:* If you do not specify this parameter, the row information is *not* brought back in such cases. *Data_type*: bool, optional
`-keepPinGeometry`	Specifies that for every partition pin, a DRCFILL metal of the type special route is placed on top of the partition pins. *Data_type*: bool, optional
`-noUninheritPhysical`	Specifies that physical only cell instances are removed during `flattenPartition`. *Data_type*: bool, optional
partitionName	Specifies the name(s) of the partition(s). You can specify any number of partitions. *Data_type*: string, optional

Example

The following command uncommits the partition status of partitions `sheet17` and `sheet25`:

```
flattenPartition sheet17 sheet25
```

Related Information

- `setPtnPinStatus`
- Partitioning the Design chapter in the *Innovus User Guide*
- Floorplanning the Design chapter in the *Innovus User Guide*

get_ptn_fplan_mode

```
get_ptn_fplan_mode
[-help]
[-export]
[-import]
[-quiet]
[-nonDefault]
```

Retrieves information about the option values set using the `set_ptn_fplan_mode` command.

Parameters

`-export`	Retrieves the value of data that `saveParititon -fplan` will write. An empty list means default objects are written. *Default:* `special_route` (default data + special route is always saved) *Data_type:* enums={macro special_route pin_constraint}_list
`-help`	Outputs a brief description that includes the type and default information for each `get_ptn_fplan_mode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_ptn_fplan_mode`
`-import`	Retrieves the value of data that `assembleDesign -fplan` will bring back. An empty list means default objects are brought back. *Default:* By default, `-import` brings only defaults objects *Data_type:* enums={macro special_route pin_constraint}_list
`-nonDefault`	Displays the current non-default settings.
`-quiet`	Displays the current settings for the specified parameters in Tcl list format only.

Example

The following command reports the value for the `-export` parameter set using the `set_ptn_fplan_mode` command.

```
get_ptn_fplan_mode -export
```

Related Information

- `set_ptn_fplan_mode`
- Partitioning the Design chapter in the *Innovus User Guide*

getBlackBoxArea

```
getBlackBoxArea
[-help]
-cell cellName
[-stdCellArea]
[-macroArea]
[-cellUtil]
```

Retrieves the standard cell area, macro area, and cell utilization value for the specified blackbox. If only the blackbox cell name is specified, the command reports the total blackbox area.

Note: You can use this command to report the black box area only if you had specified the blackbox using the `specifyBlackBox` command with the `-gateArea` parameter.

Parameters

`-cell cellName`	Specifies the name of the blackbox cell. *Data_type*: string, required
`-cellUtil`	Reports the standard cell utilization value for the blackbox. The standard cell utilization is calculated as follows: `standard cell area / (blackbox area - total macro area)` This parameter cannot be used if the blackbox was specified based on the total area or the gate count. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `getBlackBoxArea` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man getBlackBoxArea`
`-macroArea`	Reports the area of the macros included in the blackbox area. This parameter cannot be used if the blackbox was specified based on the total area or the gate count. *Data_type*: bool, optional
`-stdCellArea`	Reports the area of the standard cell included in the blackbox area. This parameter cannot be used if the blackbox was specified based on the total area or the gate count. *Data_type*: bool, optional

Examples

- The following command retrieves the total blackbox area for the blackbox `bbox_inst1`:
  ```
  getBlackBoxArea -cell bbox_inst1
  ```
- The following command retrieves the standard cell area, macro area, and cell utilization value for the blackbox `bbox_inst2`:
  ```
  getBlackBoxArea -cell bbox_inst2 -stdCellArea -macroArea -cellUtil
  ```

Related Information

- `specifyBlackBox`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Defining Blackboxes

getClonePtnOrient

```
getClonePtnOrient
[-help]
hinst
```

Retrieves the orientation information of the specified partition clone. You can use this command after using the `setClonePtnOrient` command.

Parameters

`-help`	Outputs a brief description of the `getClonePtnOrient` command. For a detailed description of the command, use the `man` command: `man getClonePtnOrient`
hinst	Specifies the name of the partition clone whose orientation information has to be retrieved. This can be an hierarchical instance name of a partition or an instance name of a black box. **Note:** The hinst argument should be registered as a hinst and not as a string to support DPO usage. *Data_type*: (hinst), required

Example

The following command retrieves the the orientation information for the partition clone `TDSP_ARB_C1`:

```
getClonePtnOrient TDSP_ARB_C1
```

Related Topics

- `setClonePtnOrient`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Changing Partition Clone Orientation

getPinAssignMode

```
getPinAssignMode
[-help]
[-advance_node_rule_support]
[-allow_unconnected_in_abutted_edge]
[-allowNonNdrNetPinsOnNdrTracks]
[-block_boundary_macro_distance]
[-force_abutment_with_fixed]
[-insidePinSearchDistance]
[-max_distance_pairing]
[-maxChannelWidthAsAbutted]
[-pinEditInBatch]
[-pinOffStripe]
[-pinToStripeDistance]
[-pinToViaDistanceNonPrefDirection]
[-pinToViaDistancePrefDirection]
[-promotedMacroMaxLayer]
[-promotedMacroMinLayer]
[-restrict_boundary_macro_distance]
[-strict_abutment]
[-quiet]
```

Returns information about the specified `setPinAssignMode` parameter in the Innovus log file and the console. The following information is returned:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `getPinAssignMode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man getPinAssignMode`
`-parameter_names`	Displays information for the specified parameters. You can specify one or more parameters. See `setPinAssignMode` for descriptions of the parameters you can specify.
`-quiet`	Displays the current settings for the specified parameters in Tcl list format only.

Examples

- The following command returns the current settings for the `setPinAssignMode` parameters:

  ```
  getPinAssignMode
  ```

  The software returns the following information:

  ```
  -advanced_node_rule_support true # bool, default=true
  -allow_unconnected_in_abutted_edge true # bool, default=true
  -allowNonNdrNetPinsOnNdrTracks true # bool, default=true
  -block_boundary_macro_distance 3 # float, default=3, min=0, max=100
  -force_abutment_with_fixed true # bool, default=true
  -max_distance_pairing 50 # float, default=50, min=0, max=10000
  -maxChannelWidthAsAbutted 3 # float, default=3, min=0, max=400
  -pinEditInBatch false # bool, default=false
  -pinOffStripe none # enums={below all none}, default=none
  -pinToStripeDistance {} # string, default=""
  -pinToViaDistanceNonPrefDirection {} # string, default=""
  -pinToViaDistancePrefDirection {} # string, default=""
  -promotedMacroMaxLayer 31 # string, default=31
  -promotedMacroMinLayer 31 # string, default=31
  -restrict_boundary_macro_distance 30 # float, default=30, min=0, max=100
  -strict_abutment true # bool, default=true
  ```

Related Information

- `assignPtnPin`
- `assignIoPins`
- `checkPinAssignment`
- `createPinGuide`
- `legalizePin`
- `reportUnalignedNets`
- `setPinAssignMode`
- Partitioning the Design chapter in the *Innovus User Guide*

getPinConstraint

```
getPinConstraint
[-help]
[-dictFile string]
[[-global | -cell cell_name]
[-global | -pin pin_name_list]]
```

Retrieves information about the constraints set for a partition pin or an I/O pin using the `setPinConstraint` command. It displays the information in a tabular format output report that includes constraints from other sources (top/bottom preferred layer and NDR-rule from `setAttibute`, reserved layer for partition, pin/net-group /guide constraints presence)) and segregates between user-defined, default, and lef values.

Parameters

`-cell` *cell_name*	Specifies the name of the module. The module name is not required if the command is run on a top-level module. You can use *Data_type*: string, optional
`-dictFile` *string*	Writes the output of the command in the specified text file (in dictionary format). *Data_type*: string, optional
`-global`	Specifies to get global constraints. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `getPinConstraint` parameter. For a detailed use the `man` command: `man getPinConstraint`
`-pin` *pin_name_list*	Specifies the name of the partition pin. You can use wildcards (`*?`) with this parameter. *Data_type*: string, optional

Example

- The following command prints all the constraints set for the specified module.

```
getPinConstraint -cell dtmf_chip
```

The following result is displayed on the screen:

```
Default Global Values:
----------------------
Minimum pin-to-pin track spacing : 2
Pin to corner distance : 5

Template from tech LEF
------------------------------
| Layer | width | depth |
------------------------------
| 2 | 0.2800 | 0.2800 |
| 3 | 0.2800 | 0.2800 |
| 4 | 0.2800 | 0.2800 |
| 5 | 0.2800 | 0.2800 |
| 6 | 0.4400 | 0.4400 |
------------------------------

Global Constraint:
-------------------

Constraint on partition dtmf_chip :
-----------------------
Partition reserved layers { 1 2 3 4 5 6 7 }
--------------------------
| Side | allowed layer |
--------------------------
| top | 2 4 6 |
| left | 3 5 |
| bottom | 2 4 6 |
| right | 3 5 |
--------------------------
```

Related Information

- setPinConstraint
- unsetPinConstraint
- Partitioning the Design chapter in the *Innovus User Guide*
  - Setting Pin Constraints

getPtnPinStatus

```
getPtnPinStatus
[-help]
[-cell cellName]
[-pin {pinName | pinNameList}]
```

Retrieves the pin status that was defined with the `setPtnPinStatus` command.

Parameters

`-cell cellName`	Specifies the name of the partition. You can use wildcards (`*?`) with this parameter. *Data_type*: string, optional
`-help`	Outputs a brief description that includes type and default information for each `getPtnPinStatus` parameter. For a detailed description of the command and all of its parameters, use the man command: `man getPtnPinStatus`.
`-pin pinName | pinNameList`	
	Specifies the name of the pin(s) of the partition. You can use wildcards (`*?`) with this parameter. *Default:* If the pin name is not specified, the command is run on all pins. *Data_type*: string, optional

Example

The following command tells the status of pin `DATA` of partition `EUG`:

```
getPtnPinStatus -pin DATA -cell EUG
```

Related Information

- `setPtnPinStatus`
- Partitioning the Design chapter in the *Innovus User Guide*

handlePtnAreaIo

```
handlePtnAreaIo
[-insertBuffer bufferName | -noInsertBuffer]
[-top extension]
[-bottom extension]
[-left extension]
[-right extension]
[-selectedCell]
[-pinOnBoundary]
```

Specifies the name of the buffer to be used for feedthrough insertion in a flip chip design. Use this command before partitioning the flip chip design.

Parameters

`-bottom extension`	Specifies how far, in microns, to expand the Area I/O cell box in the bottom direction when determining the size of the placement and routing obstructions to be pushed down. *Data_type*: int, optional
`-insertBuffer bufferName`	
	Specifies either the name of the buffer, or that there are no buffers to be inserted. *Default*: `-noInsertBuffer` *Data_type*: string, optional
`-left extension`	Specifies how far, in microns, to expand the Area I/O cell box in the left direction when determining the size of the placement and routing obstructions to be pushed down. *Data_type*: int, optional
`-noInsertBuffer`	Specifies that no buffers are to be inserted. *Data_type*: bool, optional
`-pinOnBoundary`	Specifies the internal pin created to be placed at the intersection of the hole punch (placement obstruction) and the space reserved for the partition. If it is not specified, the default is that the internal pin is placed directly on top of the area I/O cell. *Data_type*: bool, optional
`-right extension`	Specifies how far, in microns, to expand the Area I/O cell box in the right direction when determining the size of the placement and routing obstructions to be pushed down. *Data_type*: int, optional
`-selectedCell`	Specifies hole punches for all currently selected cells that overlap partitions if they are Area I/O cells. If you do not specify this parameter, the command specifies hole punches for all Area I/O cells that overlap partitions. *Data_type*: bool, optional
`-top extension`	Specifies how far, in microns, to expand the Area I/O cell box in the top direction when determining the size of the placement and routing obstructions to be pushed down. *Data_type*: int, optional

Related Information

- Partitioning the Design chapter in the *Innovus User Guide*

hier_clock_route

```
hier_clock_route
[-help]
-dir ptnParentDirName
-ptn_net_dir ptnNetDirName
[-ptns ptnNameList]
```

Runs `globalDetailRoute` on the nets specified in the net file (*<ptnName>*`.net` file) provided for each of the specified partitions, in parallel, using EDP (parallel/distributed) framework.

Note:

Before running the `hier_clock_route` command if you have a requirement to set any custom preferences or callbacks (For example,VIA preferences, routing layers preference etc), or perform any other operation, a file with name *`hier_clock_route_global_detail_route.tcl`* must be created in current run directory.

The *`hier_clock_route_global_detail_route.tcl`* file must have a procedure defined like below:

```
proc ::hier_clock_route_pre_global_detail_route {ptnName} {
Puts "called ::hier_clock_route_pre_global_detail_route $ptnName"
<body: operations/settings to be written in tcl format>
. . . .
}

proc ::hier_clock_route_post_global_detail_route {ptnName} {
Puts "called ::hier_clock_route_post_global_detail_route $ptnName"
<body: operations/settings to be written in tcl format>
. . . .
}
```

For example,

```
proc ::hier_clock_route_pre_global_detail_route {ptnName} {
Puts "called ::hier_clock_route_pre_global_detail_route $ptnName"
set bot_layer 6
set top_layer 9
setDesignMode -bottomRoutingLayer $bot_layer
setDesignMode -topRoutingLayer $top_layer
setGenerateViaMode -reset
setGenerateViaMode -ndr_only true
}

proc ::hier_clock_route_post_global_detail_route {ptnName} {
Puts "called ::hier_clock_route_post_global_detail_route $ptnName"
setGenerateViaMode -reset
}
```

Parameters

`-dir` *`ptnParentDirName`*	Specifies the directory in which partition databases are present. *Data_type*: string, required
`-help`	Outputs a brief description that includes the type and default information for each `hier_clock_route` parameter. For a detailed description of the command and all of its parameters, use the man command: `man hier_clock_route`
`-ptn_net_dir` *`ptnNetDirName`*	Specifies the directory in which net file for each partition is present. The net file contains the nets to be routed. *Data_type*: string, required
`-ptns` *`ptnNameList`*	Specifies the list of partition names. *Data_type*: string, optional

Example

The following command routes the nets specified in the net files (./nets/tdsp_core.txt and ./nets/controller.txt), within the ./nets partition net directory, provided for each of the specified partitions (tdsp_core and controller)

```
hier_clock_route -dir ./ptns -ptn_net_dir ./nets -ptns {tdsp_core controller}
```

where,

- ./ptns directory contains partitions, tdsp_core and controller.
- ./nets/tdsp_core.txt contains net names: N1, N2, N3
- ./nets/controller.txt contains net names: N4, N5

The result of this command is equivalent to the following tcl code:

```
 foreach ptnName $ptnNameList {
       cd $ptnParentDir/$ptnName
       restoreDesign . $ptnName
       # select nets specified in $ptnNetDir/$ptnName.net
       # set routeGuide, if found in $ptnNetDir/$ptnName.rgf
       # source user-defined-procs for global detail route
       if {[file exists ${current_run_dir}/hier_clock_route_global_detail_route.tcl]} {
            source ${current_run_dir}/hier_clock_route_global_detail_route.tcl
       }
       # invoke call back : pre global detail route
       if {[info procs ::hier_clock_route_pre_global_detail_route] != {}} {
           ::hier_clock_route_pre_global_detail_route $ptnName
       }

       # run globalNanoRoute on the selected nets
       setNanoRouteMode -route_selected_net_only true
       globalDetailRoute
       setNanoRouteMode -route_selected_net_only false

       # invoke call back : post global detail route
       if {[info procs ::hier_clock_route_post_global_detail_route] != {}} {
           ::hier_clock_route_post_global_detail_route $ptnName
       }

       # write def file
       defOut -routing $ptnName.def.gz
       }
```

Related Information

- globalDetailRoute
- Partitioning the Design chapter in the *Innovus User Guide*

hiliteFeedthroughNets

```
hiliteFeedthroughNets
[-help]
fileName
[netNameList ]
```

Highlights all nets for which feedthrough buffers were inserted with the `insertPtnFeedthrough` command. The highlighted feedthrough path consists of the nets, the terms that the nets connect to, and the instances that contain those terms.
Note: For the `hiliteFeedthroughNets` to work, the `insertPtnFeedthrough` command must be run with the `-netMapping` parameter. The net mapping file generated with the `insertPtnFeedthrough -netMapping` parameter is used by the `hiliteFeedthroughNets` command to highlight the feedthrough nets.

To dehighlight the feedthrough nets, run the `dehighlight` command.

Note: It displays a warning message when used for a design with nested partitions and generates results based on single level partitions.

Parameters

fileName	Specifies the net mapping file that was generated with the `insertPtnFeedthrough -netMapping` parameter. *Data_type*: string, required
`-help`	Outputs a brief description that includes the type and default information for each `hiliteFeedthroughNets` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man hiliteFeedthroughNets`
netNameList	Specifies the name of the net(s) whose feedthrough path should be highlighted. *Default:* The feedthrough paths of all nets specified in the net mapping file are highlighted. *Data_type*: string, optional

Examples

- The following command highlights all feedthrough nets specified in the net mapping file `nmpFile1`.
  ```
  hiliteFeedthroughNets nmpFile1
  ```
- The following command highlights all feedthrough nets that are specified in the net mapping file nmpFile1 whose name begins with `feedthruNet_core`.
  ```
  hiliteFeedthroughNets nmpFile1 feedthruNet_core*
  ```

Related Information

- `insertPtnFeedthrough`
- `dehighlight`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Inserting Feedthroughs
  - Highlighting the Nets for which Feedthrough Buffers Have been Inserted

insertPtnFeedthrough

```
insertPtnFeedthrough
[-help]
[-allLowercase]
[-blockageFile fileName]
[-blockedEdgesFile fileName]
[-instPrefix instancePrefix]
[-netPrefix netPrefix]
[-repeatedSymmetricAbuttedFPlan]
[-reuseBuffer {true false 0 1}]
[-saveTopoFile fileName]
[-useShortName]
[-verbose]
[[[-selectNet fileName | -selectMarkedNet | -autoSelectNetsByCongestion | -autoSelectNetsBySlack |
[ [-excludeNet fileName] ]] ] | -topoFile topologyFileName ]
[ -topoFile topologyFileName | -routeBased | -routeBasedStrict | [-excludePtnList {ptnName | ptnNameList} ] ]
[ -topoFile topologyFileName | -routeBased | -routeBasedStrict ]
[ -routeBased [-autoSelectNetsBySlack ] [-autoSelectNetsByCongestion ]]
[-noBuffer | [[-bufCell {cellName | cellNameList}] [-doubleBuffer ]]]
[[-checkOnly ] | [[-ecoFile fileName] [-netMapping fileName]] ] [-topoFile topologyFileName ] [-noBuffer ]
```

Inserts feedthrough buffers into the partitions, changing the original netlist, to avoid routing a net over a block area. ECO placement runs automatically with this command. The buffer supplied during design import is taken as default. For Alternately, you can specify a buffer with the `-bufCell` parameter or specify the `-noBuffer` parameter to generate an ASSIGN statement in the netlist. You can use this command after the floorplanning steps for a partitioned design.

The `insertPtnFeedthrough` command can detect if the design has power domains. This way, appropriate buffers which are inserted into power domains will automatically be derived based on power domain library binding. An error message is reported if no regular buffer is found for an *Always On* power domain in the feedthrough path. Both placement and route based feedthrough insertions can support *Always On* and switchable power domains.

You can save the feedthrough buffer topology tree information in a file by using the `-saveTopoFile` parameter. You can later use the `-topoFile` parameter to replicate feedthrough buffer insertions for an ECO netlist by specifying the file that has the topology tree information. You can also manually create a topology file to guide feedthrough insertion for specific nets.

Notes:

- The `insertPtnFeedthrough` command by default derives the feedthrough topology using a shortest-path algorithm, which is based on the placement results.
- The `insertPtnFeedthrough` command by default considers a design to be channel-less, having no channel available for routing. Therefore all nets are candidates for feedthrough insertion.
  The default behavior assumes a true channel-less design - that is, the possibility of routing through channels is considered minimal. It does not consider routing congestion, routing blockages, route guides, pin guides, and bus guides. If these considerations are required, use the `-routeBased` parameter to derive the feedthrough topology based on the routing results.
- Feedthroughs are inserted in nets that connect to floating ports. Floating ports are hierarchical ports that are not connected to any logic inside the module.
- The `insertPtnFeedthrough` command removes nets that are inserted with feedthrough buffers from any net groups to which they belong. After running this command you should, therefore, update the net groups that contain feedthrough nets.
- If a pin guide exists for a net for which feedthrough buffers are inserted, the pin guide information is updated.
- The `insertPtnFeedthrough` command will handle designs with master/clone partitions automatically. However, it may create infeasible pin solution for channel-less designs with fully abutted partitions.
- The `insertPtnFeedthrough` command supports nested partitions. For inserting feedthrough buffers into nested partitions, the `insertPtnFeedthrough` command, first, automatically deletes the parent partitions and keeps only the bottom-most partition. It then does a single-level feedthrough insertion and then automatically restores the parent partitions and brings the nested partition definition back as it is. In nested designs, the `insertPtnFeedthrough` command only puts buffers and ports in the bottom most level of partitions.
- The `insertPtnFeedthrough` command prints a summary of all messages issued for nets in the log file.
- A feedthrough always uses a low power compatible buffer/AO-buffer. If it cannot use a low power complaint cell, then the default buffer (specified with the `-bufCell` parameter) is used, and the buffer name is prefixed with “MSV_INVALID”.

Parameters

`-allLowercase`	Specifies all lowercase characters for the inserted feedthrough buffer and feedthrough net names. *Data_type*: bool, optional
`-autoSelectNetsByCongestion`	Specifies that the critical nets in congested channels will be identified automatically. These critical nets are routed flat so that later they can be feedthrough over partition(s) to reduce congestion, while other remaining nets are routed by honoring fences. Once the routing is done, feedthroughs are inserted based on the routing result. *Data_type*: bool, optional
`-autoSelectNetsBySlack`	Specifies that the critical nets in the design will be identified automatically. These critical nets are routed flat so that later on they can be feedthrough over partition(s) to improve timing while other remaining nets are routed by honoring fences. Once the routing is done, feedthroughs are inserted based on the routing result. *Data_type*: bool, optional
`-blockageFile` *fileName*	Specifies a file that contains the coordinates of line segments that are considered as blockages for placement based feedthrough insertion. **Note:** The blockages must be defined as lines that do not overlap with partitions or macros. These blockage lines are supported only with automatic feedthrough insertion or placement based feedthrough insertion. The following is the format of the blockage file: *x1 y1 x2 y2* *x1 y1 x2 y2* *...* *xn yn xm ym* The blockage lines are specified as coordinates of a box, however, the boxes should be of zero width or height. *Data_type*: string, optional
`-blockedEdgesFile` *fileName*	Specifies a file that contains the names of the partitions followed by the edge numbers for the partition which are considered as blocked for the purpose of `insertPtnFeedthrough` command. *Data_type*: string, optional
`-bufCell {`*cellName* `|` *cellNameList*`}`	
	Specifies the name of the feedthrough buffer cell type. **Note:** You can specify multiple buffer cells with MSV designs. The first buffer will be used for the default power domain. *Data_type*: string, optional
`-checkOnly`	Specifies that the feedthrough buffers are *not* inserted. This parameter is useful when you want to know how the `insertPtnFeedthrough` command will affect the design. A summary information will be printed. You can specify this parameter along with the `-saveTopoFile` parameter. This will save the feedthrough topology information. *Data_type*: bool, optional
`-doubleBuffer`	Adds a buffer close to the feedthrough input pin and adds a buffer close to the feedthrough output pin. *Data_type*: bool, optional
`-ecoFile` *fileName*	Specifies the file for logging ECO directives. This parameter cannot be used with the `-checkOnly` parameter. *Data_type*: string, optional
`-excludeNet` *fileName*	Specifies a file that contains net names to be excluded from being a feedthrough net for a partition. *Data_type*: string, optional
`-excludePtnList {`*partitionName* `|` *partitionList*`}`	
	Specifies that feedthrough buffers should not be added to the specified partitions. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `insertPtnFeedthrough` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man insertPtnFeedthrough`
`-instPrefix` *instancePrefix*	Specifies a prefix for the newly created instances. The prefix that you specify will be used instead of the default prefix. *Data_type*: string, optional

`-netMapping` *fileName*	Generates a file that maps the original net name to the new net name created for the feedthrough buffers. **Note:** You must specify this parameter if you later want to run the `hiliteFeedthroughNets` command to highlight the feedthrough paths of the nets. *Data_type*: string, optional
`-netPrefix` *netPrefix*	Specifies a prefix for the newly created nets. The prefix that you specify will be used instead of the default prefix. *Data_type*: string, optional
`-noBuffer`	Generates an `ASSIGN` statement in the netlist instead of inserting a buffer. *Data_type*: bool, optional
`-repeatedSymmetricAbuttedFPlan`	This option is relevant to master clone designs. It is to be used for the floorplans which have repeated components arranged in a symmetric fashion with a data flow going from the top to each component independently. **Note**: If the design has nested partitions, it will delete all the partitions which are parents. It will hold more partitions within them, perform its operation and restore the parent partitions once finished. *Data_type*: bool, optional
`-reuseBuffer {true | false | 0 | 1}`	
	Reuses inserted buffers across multiple instantiation of a module within one run. This option should only be used for designs that have master and clone partitions. *Default*: `true` *Data_type*: enum, optional
`-routeBased`	Derives the feedthrough topology based on the routing results. *Data_type*: bool, optional
`-routeBasedStrict`	Derives the feedthrough topology based on the routing results and follows each criss-cross. In comparison to the `-routeBased` parameter, the `-routeBasedStrict` option inserts more number of buffers depending on the routing cross-overs and also supports special routes. It can insert buffers on intra-partition net, if the routing crosses outside the boundary. However, such a net needs to be handled via selected feedthrough insertion and not default selection. The `-routeBasedStrict` parameter places the pins at each of the routing cross-point. **Note:** You can use the `showPtnWireX` command to get `[design_name].nets_not_crossing_mc.txt` for a selection of nets on which to use this option. *Data_type*: bool, optional
`-saveTopoFile` *fileName*	Saves a file with feedthrough buffer topology tree information from an existing floorplan. *Data_type*: string, optional
`-selectMarkedNet`	Specifies that feedthrough buffers should be inserted on the nets that are currently selected in the GUI. *Data_type*: bool, optional
`-selectNet` *fileName*	Specifies a file that contains net names to be feedthrough nets for a partition and inserted feedthrough buffers. *Data_type*: bool, optional
`-topoFile` *topologyFileName*	Specifies the file that has the topology tree information. This information is used to create feedthrough buffers for the netlist. If you use the `-topoFile` parameter, only those nets that are specified in the topology file are considered for feedthrough buffer insertion. If a net does not exist in the design, it should not be in the topology file. For example, if ECO changes remove a net, that net should be removed from the topology file. **Note:** Hierarchical bus names are not supported in the Innovus software. Therefore, you cannot use hierarchical bus names in the topology file. **Note:** The names of the feedthrough nets, buffers, and ports created might change in the different runs of the command. Only the feedthrough path (the partitions through which the net will pass) will always remain the same across the different runs of the command. *Data_type*: string, optional
`-useShortName`	Specifies the abbreviation of inserted feedthrough net names so that the net names do not extend too long for multiple runs. *Data_type*: bool, optional

`-verbose`	Specifies that net-specific information is included in the logs. *Default:* Only the summary information is included in the logs. *Data_type*: bool, optional

Examples

- The following command generates a file, `K_ABC.topo`, with the feedthrough buffer topology tree information:
  ```
  insertPtnFeedthrough -saveTopofile K_ABC.topo -netMapping abcfile
  ```

  The following is the generated topology file:

  ```
  ##############################################################
  #  Generated by:      Cadence Innovus
  #  OS:                Linux x86_64(Host ID nofcnl040)
  #  Generated on:      Mon Jan 21 12:34:59 2015
  #  Design:            test
  #  Command:           insertPtnFeedthrough -saveTopoFile K_ABC.topo ...
  ##############################################################
  version 1.0;
  net K_ABC_NET
  hinst-hinst p4 p8;
  hinst-hinst p4 p7;
  end net
  ```
- The following command specifies that topology tree information from the `topology_info` file should be used to generate feedthrough buffer insertions:
  ```
  insertPtnFeedthrough -topofile topology_info
  ```

Related Information

- `hiliteFeedthroughNets`
- `showPtnWireX`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Inserting Feedthroughs

legalizePin

```
legalizePin
[-help]
[-auto_pairing]
[-ignore {pin_spacing pin_spacing_constraint pin_spacing_routeBlk pin_width pin_depth pin_layer pin_min_area pin_on_track pin_abutment
pin_non_nbr_abutment clones pin_color}
[-internalPin]
[-keepLayer]
[-keepOrder]
[-moveFixedPin]
[-pin {pinName | pinNameList}]
[-ptn ptnName]
[-snapToBoundary]
[-verbose]
[-ptn ptnName | {-pin_file fileName [-exclude_ptn ptnName]}]
[-pin_file fileName  | -exclude_pin_file fileName]
```

Moves a pin from its existing location. It moves the pins in the following cases:

- The pin overlaps with any other object or pin. The objects considered are:The pin violates pin-to-pin spacing constraints
    - stripes
    - vias
    - feedthrough buffers
    - routing blockages
    - pin blockages
- The pin is on a non-preferred layer, a non-reserved layer, or the Metal1 layer
- The pin is not on the routing track
- The pin does not honor corner mask settings

The pin is assigned to the nearest legal location.

Note: If this command is run on a blackbox that has one or more pins with a single LEF port but with multiple shapes within the LEF port, the command will remove these multiple shapes and create a single shape. If this command is run on a blackbox that has one or more pins with multiple LEF ports, the first LEF port will be processed as per the command and the remaining LEF ports will remain unchanged.

The `legalizePin` command supports nested partitions and corrects the pin position of illegal pins across hierarchies. It gives a warning if the first level pin is placed in second level partition or vice versa and legalizes them to the nearest track. It also supports pin overlapping across N levels for overlapping partition boundaries.

The `legalizePin` command honors net weights and accordingly legalizes pins. For example, if there are 2 or 3 pins that can be put into a slot, the `legalizePin` command automatically looks at the net weight of these pins and assigns the slot according the net weight priority.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `legalizePin` parameter. For a detailed description of the command and all of its parameters, use the man command: `man legalizePin`
`-auto_pairing`	Automatically calculates the channel width or distance between pins in a design which will be considered for pairing. *Data_type*: bool, optional
`-exclude_ptn ptnName`	Specifies the list of excluded partition for pin file. *Data_type*: string, optional
`-exclude_pin_file fileName`	
	Specifies pin-list to be excluded for partition in a file. *Data_type*: string, optional
`-ignore {pin_spacing pin_spacing_constraint pin_spacing_routeBlk pin_width pin_depth pin_layer pin_min_area pin_on_track pin_abutment pin_non_nbr_abutment clones pin_color}`	

	Legalizes by taking the category(ies) of ignore types into account. *Data_type*: (enum)+, optional
`-internalPin`	Legalizes internal pins by moving them to the closest track on the preferred routing layer. *Data_type*: bool, optional
`-keepLayer`	Maintains the pin layer while legalizing the pins. If the pin is on a layer that is not on the allowed partition layer list then an error is reported and the pin is not legalized. *Data_type*: bool, optional
`-keepOrder`	Maintains the pin order while legalizing pins irrespective of the layers. Initially, the pins are legalized within the range of original pin locations. However, if the legalizePin `-keepOrder` command is unable to find a legal location, `legalizePin` will try to find a valid location that is outside the original range and give warning message. If `legalizePin` fails to find a valid location, it reports an error. **Note**: The option is only applicable for pins on the fence. It does not apply to internal pins. *Data_type*: bool, optional
`-moveFixedPin`	If you use this parameter, the specified pins with a Fixed status can also be moved. **Note**: Pins that are not specified but have a Fixed status are not moved. *Default*: If you do not specify this option, the specified pins with a Fixed status are not moved. *Data_type*: bool, optional
`-ptn` *ptnName*	Specifies the partition for which the pins will be checked. This parameter is required if the partition is not the top-level module. **Note**: The `legalizePin` command can also resolve I/O pins from overlapping power stripes. In this case, ptnName specifies the block-level design name. *Data_type*: string, optional
`-pin {`*pinName* `|` *pinNameList*`}`	
	Specifies that the command should be run on the specified pin(s) only. *Data_type*: string, optional

`-pin_file` *fileName*	Specifies the pin list for a partition in a file. In the pin file, the pins and nets are specified in the following format:
	`Partition: PtnName/BlockName`
	Pin1
	Pin2
	`..`
	`Partition: PtnName/BlockName`
	Pin1
	Pin2
	`..`
	`Nets:`
	netA
	netB
	`..`
	Note: While specifying the pin list in the pin_file, it is necessary to give the “Partition” keyword followed by the name of the partition or block. If this is not specified, no operations are performed. You can specify multiple partitions in the same pin_file.
	The `pin_file` does not support buses or wild cards for pin names. If a pin specified in the list or a partition is not present in the design, a warning is displayed.
	Data_type: string, optional

`-snapToBoundary`	Snaps internal partition pins to the boundary and legalizes them. **Note**: The `-snapToBoundary` parameter does not work with `-internalPin` as both work on internal pins and their results are different. *Data_type*: bool, optional
`-verbose`	Reports all the existing pin violations. *Data_type*: bool, optional

Example

The following command legalizes pins in same the metal as they were present and moves fixed pins if they are placed illegally:

```
legalizePin -keepLayer -moveFixedPin
```

The following command accepts a file that contains the list of pins for partition A and partition B and legalizes them.

```
legalizePin -ptn {A B} -pin_file pinLst.txt
```

where, the *pinList.txt*

```
Partition: top
out1
out2
out3
Partition: A
in1
in2
in3
Partition: B
pina
pinb
pinc
```

Related Information

- checkPinAssignment
- assignPtnPin
- assignIoPins
- setPtnPinStatus
- Partitioning the Design chapter in the *Innovus User Guide*
  - Validating Pin Placement

loadBlackBoxNetlist

```
loadBlackBoxNetlist
[-help]
netlist
```

Loads a netlist for a black box. You can use this command to incrementally load an updated netlist for a blackbox. You can use this command before converting a blackbox to a fence.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `loadBlackBoxNetList` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man loadBlackBoxNetList`
`netlist`	Specifies the netlist name. The netlist can contain more than one black box netlist, but it should not contain any non-blackbox netlist. *Data_type*: string, required

Example

The following command loads the netlist `netlist_v1`.

```
loadBlackBoxNetlist netlist_v1
```

Related Information

- `convertBlackBoxToFence`
- `convertFenceToBlackBox`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Specifying Partitions and Blackboxes
  - Blackbox Flow

loadPtnPin

```
loadPtnPin
[-help]
[-report string]
[-snapModifiedToBoundary]
{-ptnName string | -all}
{-file string | -def string}
```

Loads either a partition floorplan file (from a saved partition design session) or a DEF file, and then displays the preassigned I/O pins for the partition. This command must be run for each partition in the design. You can use this command on a partition design after loading the top level floorplan and after running/loading placement and early global route data. Use `unloadPtnPin` to remove the preassigned I/O pins.

Note: The `loadPtnPin` command can load partition files that are saved using the `saveFPlan` command.

Parameters

`-all`	Specifies that pin positions for all the partitions should be loaded. *Data_type*: bool, optional
`-def string`	Specifies the full pathname of the partition def file. **Note:** You can specify a DEF file in the gzip format (with the .gz extension) also. *Data_type*: string, optional
`-file string`	Specifies the full path name of the partition floorplan file. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `loadPtnPin` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man loadPtnPin`
`-ptnName string`	Specifies the name of the partition. *Data_type*: string, optional
`-report string`	Specifies the optional report filename that contains any pins deleted or newly added. *Data_type*: string, optional
`-snapModifiedToBoundary`	Snaps pins to the modified partition boundary. The pin is unplaced if it cannot be snapped to edge at same location in same direction. If the existing pin is off-grid, the `loadPtnPin` command does not snap it to the track and you need to run the `legalizePin` command to snap the pin to the track. *Data_type*: bool, optional

Example

The following command loads pin information from file `EUG.fp.ptn` for partition `EUG`

```
loadPtnPin -ptnName EUG -file EUG.fp.ptn
```

Related Information

- `unloadPtnPin`
- `saveFPlan`
- `savePtnPin`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Assigning Partition and Blackbox Pins
  - ECO Pin Assignment

merge_hierarchical_def

```
merge_hierarchical_def
[-help]
defFiles <...>
[-add_blockage]
[-delete_blockages_over_partition_macro]
[-incremental_ilm_def]
[-ignore_drcfill]
[-rdl_def string]
[-rdl_orientation orientation]  [-rdl_placement {x y}]
[-secondary {{suffix def_file_list}...}]
[-skipNets]
[-skip_filler]
[-skip_pg]
[-skip_routing {[pg|signal] [regular|special] [high_layer_routing]}]
[-to_layer routing_layer_name]
[-top_scope]
[-topcell_orientation orientation]
[-topcell_placement {x y}]
[-transform_file string]
[-use_top_def_die_area]
```

Merges DEF data from various blocks to a single chip database. The `merge_hierarchical_def` command provides the following features:

- Takes the TOP DEF file and a set of block DEF files and merges them into a single chip database.
- The Verilog files for each of the blocks, and TOP, need to be loaded up front .
- The block and TOP Verilog files (or their DEF files) can be specified in any order. However, if there are empty Verilog module stubs of blocks in some of the parent Verilog files, ensure that the parent Verilog files are specified prior to the child Verilog files.
- There is no need to specify the LEF files for the blocks.
- During the course of this command, TOP rows will be cut around the block boundary, and all block rows will be brought back as is.
- Partition definitions are created for each of the assembled block as per the DIE AREA statement in the block DEF.
- The wire at the block boundaries will be stitched with the electrically connected net's wires at the TOP.
- All orientations of blocks are supported for assembly. Repeated blocks are also supported. Non-R0 master is also supported.

Parameters

`-add_blockage`	Specifies that the netlist is a reduced one and converts the undefined instances into blockages. *Data_type*: bool, optional
defFiles <...>	Specifies DEF files to be merged. *Data_type*: list, required
`-delete_blockages_over_partition_macro`	
	Deletes top design placement and routing blockages overlapping partitions. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `merge_hierarchical_def` parameter. For a detailed description of the command and all of its parameters, use the man command: `man merge_hierarchical_def`
`-ignore_drcfill`	Ignores the special routing with DRCFILL shape attribute. *Data_type*: bool, optional

`-incremental_ilm_def`	Incrementally flattens the ILM def. *Data_type*: bool, optional
`-rdl_def` *string*	Specifies RDL def file. *Data_type*: string, optional
`-rdl_orientation` *orientation*	Specifies orientation for RDL def. *Data_type*: enum, optional
`-rdl_placement {`*x y*`}`	Specifies offset of RDL def. *Data_type*: point, optional
`-secondary {{`*suffix def_file_list*`}...}`	
	Specifies the secondary DEF file(s) to load. *Data_type*: string, optional
`-skip_filler`	Skips merging filler and decap cells. *Data_type*: bool, optional
`-skip_pg`	Skips merging routing on pg nets. *Data_type*: bool, optional
`-skipNets`	Skips merging signal nets section. *Data_type*: bool, optional
`-skip_routing {[pg|signal] [regular|special] [high_layer_routing]}...`	
	Skips merging routing based on the settings specified. This setting can be any combination of the three optional keywords from the following set: • `[pg|signal]`: Specifies the type of the net on which the routing to be skipped. pg – pg net, signal – signal net, both if not specified. • `[regular|special]`: Specifies the type of routing to be skipped. regular – regular routing, special – special routing, both if not specified. • `[high_layer_routing]`: Specifies that only the routing on high layers to be skipped, where the layers are calculated automatically. If not specified, the routing on all layers are skipped. The layers can be specified with `-to_layer` option. In that case, the routing on the layers from top to the specified layer are skipped. **Note:** You must specify the `-to_layer` parameter explicitly if `high_layer_routing` is specified for `-skip_routing`. Otherwise, `high_layer_routing` has no effect. *Data_type*: string, optional
`-to_layer` *routing_layer_name*	Skips the routing of layers from top to the specified layer (inclusive) when the `high_layer_routing` option is specified for the `-skip_routing` parameter. *Data_type*: (layer), optional
`-top_scope`	Merges routing only on top scope nets. *Data_type*: bool, optional
`-topcell_orientation` *orientation*	

	Specifies orientation for topcell. *Data_type*: enum, optional
`-topcell_placement {x y}`	Specifies placement for topcell. *Data_type*: point, optional
`-transform_file string`	Specifies the name of the transform file that contains the final x and y offsets, orientation in the top-level of each LEF block that got replaced by DEF, and the block's origin. *Data_type*: string, optional
`-use_top_def_die_area`	Enables the use of TOP DEF DIE AREA. *Data_type*: bool, optional

Example

- The following command merges DEF data from `top.def.gz` and `blockA.def.gz` to a single chip database.

```
merge_hierarchical_def top.def.gz blockA.def.gz
```

Related Information

- Partitioning the Design chapter in the *Innovus User Guide*

partition

```
partition
[-help]
[partitionName {list}]
[-addPinForPGWireLayersInDistance {{layer_name1 distance1} {layer_name2 distance2}}]
[-inputNetsForWireDistribution filename]
[-noTopPushDown]
[-noViaCutSpace]
[-pgPushDownHonorUPF]
[-pushDownHonorInsidePinForSpecialNet]
[-pushDownNonPGSpecialNet]
[-pushDownSelectedPGWiresAsRouteBlockage]
[-pushDownSpecialNetAsObs {SNet list}]
[-skipRouteBlockagesOutsidePartition]
[-tcf]
[-pinCutSpace]
[-noInheritPhysical]
[-pushRoute]
[-pushdownAllBumps | -pushDownSelectedPGBump]
```

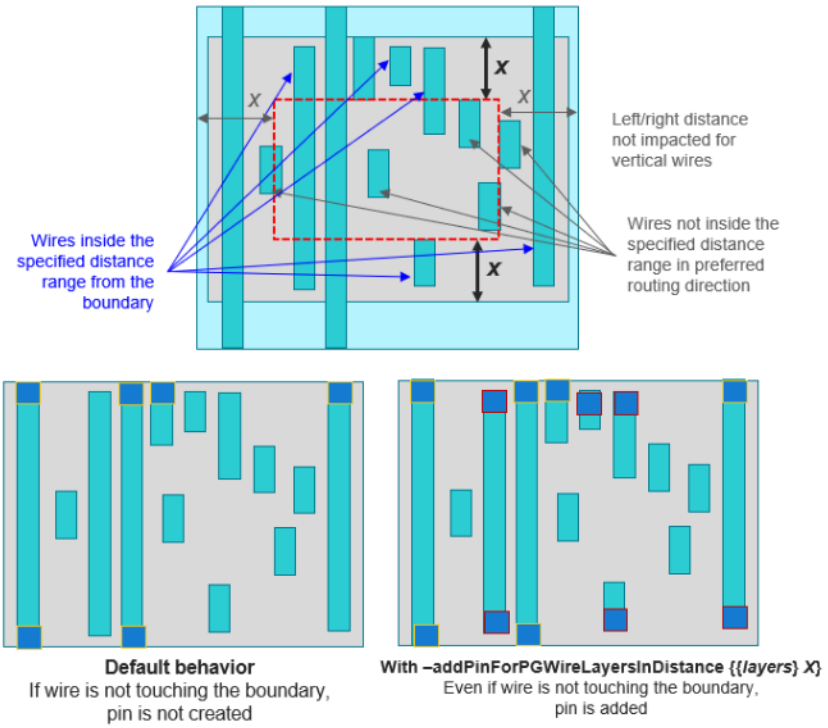
Converts the specified module fences to be partitions, pushes down the physical cells or power routing information to the partition level design(s), and duplicates strips in the partition that overlap with the partition boundary at their original widths. You can use this command after specifying the partition(s).

Notes:

- While bringing back mis-aligned rows to top-level design, the `partition` command automatically detects mis-aligned rows and pushes down the row information as it is to the top-level design.
- To assign signal pins for the partitions, use the `assignPtnPin` command.
- This command should be used in combination with the `assignPtnPin` command. Cadence recommends not mixing these version 4.2 pin commands with previous (version 4.1 or earlier) pin commands in the design flow.
- While handling metal fills, the `partition` command removes the metal-fill shapes that cross the partition boundary. It pushes down only those fill-shapes that are inside the partition boundary and are on the metal-layers owned by the partition.
- In top-down flow, the partition command only promotes P/G pins from the list of pins promoted by the `setPromotedMacroPin` command.
- The `partition` command supports partitions with flexmodels and structured data paths.
- The `partition` command pushes down the route blockage layer even when the layer type is master-slice. It creates routing blockages for master-slice layers, in the "OBS" section of LEF (from macro/IP/PADs), that are overlapping partition fences. With this feature:
  - The shape of the routing blockage is the same as the overlapping shape of the layer, in the "OBS" section.
  - The routing blockage is given a “pushdown” property, so that it is not brought back in `assembleDesign`.
  - The union of overlapping shapes of layers in “OBS” section is taken for master and clones.
- The `partition` command automatically saves all the bus guide constraints with associated net groups. This enables bus routing at block level as per chip level constraints.
- The `partition` command automatically saves all the cell padding information to each partition block data. This information is saved in partitioned block .fp file.
- The `partition` command inherits all the attributes assigned to a net for block implementation flow. Preserving these attributes allows the Early Global Router and the NanoRoute Router to route the nets following specific requirements.
- The `partition` command now preserves all the pin shapes of nets.

Parameters

`-addPinForPGWireLayersInDistance {{layer_name1 distance1} {layer_name2 distance2}}`

	Specifies the layers and distance to consider for creating pins on PG stripes that lie in the specified distance range from the partition boundary. **Note:** Some designs shorten the PG stripes to avoid stripes of different net/voltages in different abutted partition abutting together and creating a short. In such wire shapes, the `partition` command does not create any pins by default since the PG stripes do not touch the partition boundary. However, you can use the `-addPinForPGWireLayersInDistance` parameter to create PG pins during block implementation for wire shapes that lie in the specified distance range from the partition boundary. The `-addPinForPGWireLayersInDistance` parameter identifies the PG wires for the specified layers in the given proximity (in preferred direction of route) of the partition boundary to create pin shapes on them.  For example, the following command creates pins on PG stripes on layers M10-M13 that are within a distance of 5 um from the partition boundary. `partition -addPinForPGLayersInDistance {{M10 5} {M11 5} {M12 5}}` *Data_type:* string, optional
`-help`	Outputs a brief description that includes type and default information for each `partition` parameter. For a detailed description of the command and all of its parameters, use the `man` command: man partition.
`-inputNetsForWireDistribution` *filename*	
	Specifies the file containing the net names to use for wire distribution. *Data_type:* string, optional
`-noInheritPhysical`	Specifies that physical only (filler) cells are not pushed down into the partition; they remain at the top level. *Data_type:* bool, optional
`-noTopPushDown`	Specifies that top-level power routing is not pushed down from top-level design to partition blocks on layers not reserved for partition. Nothing is pushed down on layers reserved for top design. *Data_type:* bool, optional

`-noViaCutSpace`	Specifies that there are no cuts in the obstructions around vias. You can use this parameter when you want to ensure that the top-level routing can connect the vias without using the space between the vias and the corresponding cover blockages. *Data_type:* bool, optional
partitionName ...	Specifies the partition names to be committed. *Data_type:* string, optional
`-pgPushDownHonorUPF`	When enabled, the UPF information is pushed down to partitions. *Data_type:* bool, optional
`-pinCutSpace`	Specifies that there are cuts in the obstructions around pins. *Default:* By default, there are no cuts in the obstructions around pins so that the top-level routing can connect the pins without using the space between the pins and the corresponding cover blockages. *Data_type:* bool, optional
`-pushRoute`	Pushes down signal routes when the design is partitioned. **Note:** All the wires of a hierarchy-violating net are discarded, in case even a single wire of that net has a hierarchy violation. *Data_type:* bool, optional
`-pushdownAllBumps`	Specifies that all the PG bumps should be pushed down into the partition. *Data_type:* bool, optional
`-pushDownHonorInsidePinForSpecialNet`	

Preserves wire segments till partition pins at the top level while wire segments beyond partition pins are still pushdown as it is. With this option enabled, Innovus automatically detects the list of nets with inside pin having a fixed status over the partition and all top- level nets crossing the partition boundary. The pure top level nets along with wire segments of nets connected to partition pins are replaced by route blockages in block data and the actual wire segments are retained at the top level.

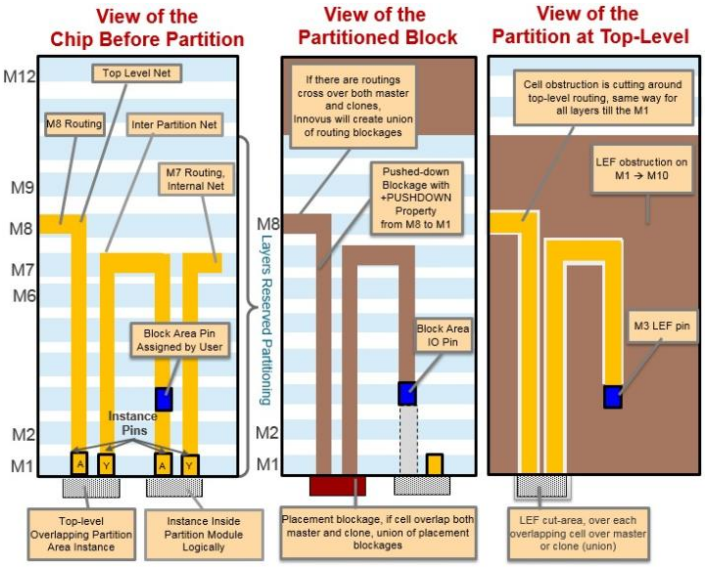
At the top-level:

- All wires till the partition pin (with fixed status) are kept. For pure top nets, all wires are kept.
- LEF obstructions are cut out for preserved wires segments.
- LEF shape against overlapping top instances are cut out.

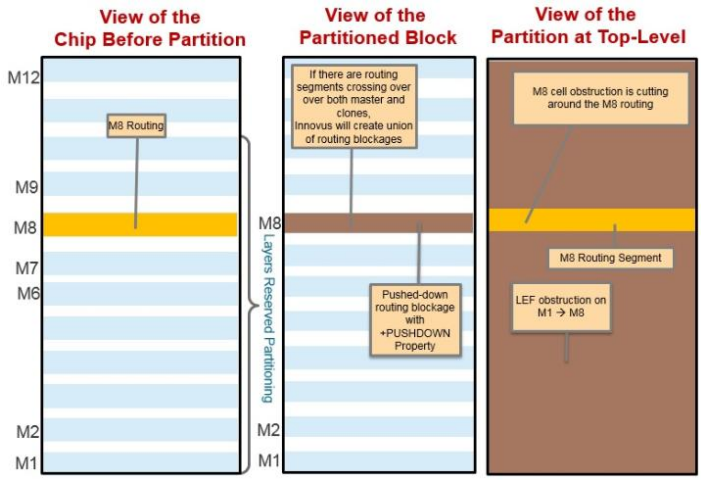
At the block level:

- Routing blockages till the partition pin for wires that were retained at top level are created.
- All wires segments after the partition pin are retained.
- Placement blockages are created for top instances overlapping partition fence area.

Note: Before using this option, ensure that partition area pins – of the corresponding net – must be FIXED.



Data_type: bool, optional

`-pushDownNonPGSpecialNet`	Specifies that the non PG special nets should also be pushed down into the partition. *Data_type:* bool, optional
`-pushDownSelectedPGBump`	Specifies that only the selected PG bumps should be pushed down into the partition. The bumps which are not selected come as route blockage. *Data_type:* bool, optional
`-pushDownSelectedPGWiresAsRouteBlockage`	
	Pushes down selected PG routes as route blockages into the partition. This parameter enables you to select the wire segments that you want to keep at the top-level design. These non-pushed down wires must be special PG Routes.  *Data_type:* bool, optional
`-pushDownSpecialNetAsObs {`*`SNet list`*`}`	
	Specifies a list of special nets, which will be pushed down as routing blockages. It supports all types of special nets including power/ground nets. You can use wildcards while specifying the net name. *Data_type:* string, optional
`-skipRouteBlockagesOutsidePartition`	Skips pushing of objects which are close to partition fence. *Data_type:* bool, optional
`-tcf`	Enables tcf writing for the block. **Note:** A TCF file is used to store power related information of the nets in the design by keeping their toggle count related data. With the `-tcf` parameter enabled, the `partition` command generates tcf related data of partition blocks. This data can be later saved using the `savePartition` command into their respective directories. *Data_type:* bool, optional

Example

- The following command converts fences to partition data and pushes down route crossing over fence `EUG` inside the partition boundary:
  ```
  partition EUG -pushRoute
  ```
- PG is Metal7, Metal8
  Partition is defined with reserved layer Metal6

  The following command copies PG from top to block for all the PG layers:
  ```
  Partition ptnName
  ```

  The following command cuts PG from top to block for PG layers reserved for partitions. For layers above the reserved layer, PG is not pushed down to ptn.
  ```
  partition ptnName -noTopPushDown
  ```
  If reserved layer is Metal7 then Metal7 PG is cut at top and pushed down and Metal8 PG will not be pushed down.

Related Information

- `assembleDesign`
- `assignPtnPin`
- `flattenPartition`
- `setPromotedMacroPin`
- `savePartition`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Pushing Down Signal Routes

partitionPushSpecialWires

```
partitionPushSpecialWires
[-help]
[-checkOnly]
[-nets list_of_net_names]
```

Creates Verilog ports (connections) in partitions for nets whose special wires overlap a partition, so that the special wire of the net gets pushed down when the `partition` command is used.

Note: For pushdown of special wires in master and clone partitions, it checks for the symmetry of the special wire shapes. For symmetrical shapes only, the `partitionPushSpecialWires` command creates one port in the Verilog module and establishes a logical connection between the new port and the net of special wire over all master and clone hinsts (reuse of logical port in master and clones).

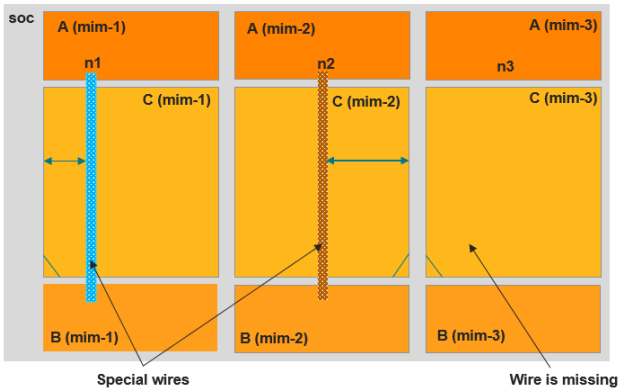
Note: If the original net was connected to a partition port of type inout, then an inout port is created inside the overlapping partition else only an input port is created.

Parameters

`-checkOnly`	When specified, the tool checks if the selected nets special-wires are symmetric over all hints of overlapping master clone cells or not. In master and clone partitions, it checks for the symmetry of nets and reports asymmetrical nets. When the `-checkOnly` parameter is not specified, the tool check the nets specified and creates logical ports only on valid nets. Data_type: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `partitionPushSpecialWires` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man partitionPushSpecialWires`
`-nets list_of_net_names`	Specifies the list of nets (Verilog nets) which have special wires to be pushed into blocks. *Data_type*: string, required

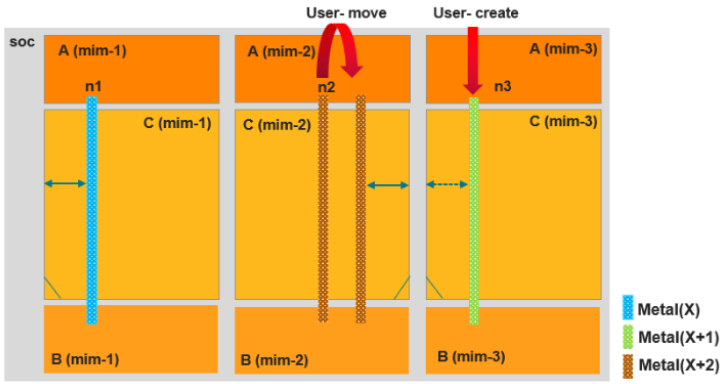
Example

- The following command checks the special wires of specified Verilog nets n1 and creates ports on valid nets.
  ```
  partitionPushSpecialWires -nets {n1}
  ```
- The following command checks if the special-wires of the specified nets are symmetric over all hints of overlapping master clone cells or not.
  ```
  partitionPushSpecialWires -nets {n1 n2} -checkOnly
  ```



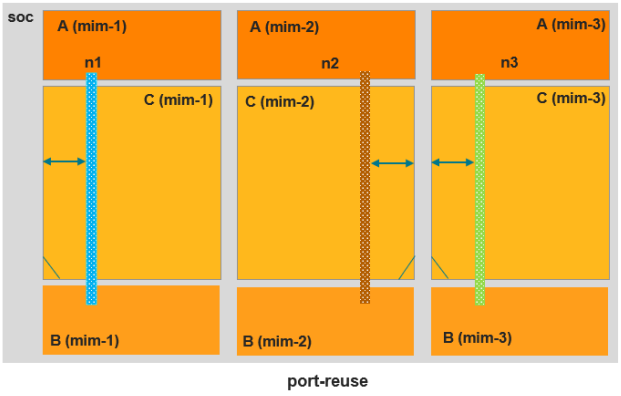

```
Report: C (mim-1) net n1 box missing in C mim-2
Report: C (mim-1) net n1 box missing in C mim-3
Report: C (mim-2) net n2 box missing in C mim-1
Report: C (mim-2) net n2 box missing in C mim-3
```

For pushdown, you must fix the wire asymmetry.



After the symmetry is fixed, the following command pushes down symmetrical nets:

```
partitionPushSpecialWires -nets {n1 n2}
```




```
Report: user Net n1, n2 and derived nets n3 pushdown, with one logical port
```

Related Information

- Partitioning the Design chapter in the *Innovus User Guide*

pinAlignment

```
pinAlignment
[-help]
[-ignore_ref_pin_shape]
[-markFixed]
[-moveFixedPin]
[-keepLayer | -newLayer layer]
[-legalizePin ]
[-noSnap]
[-pinNames pinList]
[-ptnInst ptnName]
[-refObj refObjName]
[-refType {partition_pin hinst}]
[-verbose]
```

Aligns pins between blocks on their facing edges. By default, this command snaps pins to the routing grid. You can use this command after partitioning floorplanning.

Notes:

- Specified pins will not be aligned if they do not have the reference connections on the reference block, or if the reference pins are not assigned prior to running this command.
- If this command is run on a blackbox that has one or more pins with a single LEF port but with multiple shapes within the LEF port, the command will remove these multiple shapes and create a single shape. If this command is run on a blackbox that has one or more pins with multiple LEF ports, the first LEF port will be processed as per the command and the remaining LEF ports will remain unchanged.
- This command supports Multi Level Hierarchical Flow.

The pin alignment priority is as follows:

- If you specify the `-refObj` and the `-ptnInst` parameters but not the `-pinNames` parameter, the `pinAlignment` command aligns *all* pins of specified instances that are connected to the referenced object.
- If you specify the `-refObj` parameter but not the `-ptnInst` and the `-pinNames` parameter, the `pinAlignment` command aligns *all* pins of all partitions that are connected to the referenced object.
- If you specify the `-refObj`, `-ptnInst`, and the `-pinNames` parameters, the `pinAlignment` command tries to align *all* possible pins that can be aligned.
- If you do not specify the `-refObj` parameter, the following order of priority will be used for aligning partition pins that have multiple connections:
    - Hard macro pins
    - I/O pads or pins
    - Partition pins
    - Standard cell pins

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `pinAlignment` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man pinAlignment`
`-ignore_ref_pin_shape`	Specifies that the aligned pin will ignore matching of shape with the reference pin. *Data_type*: bool, optional
`-keepLayer |`	Specifies whether to keep the current pin layer. *Default*: If you do not specify this option, the new pin layer will be the same as the referenced layer to reduce wire routing. *Data_type*: bool, optional
`-legalizePin`	Specifies that the `legalizePin` command should run automatically after the `pinAlignment` command. *Data_type*: bool, optional
`-markFixed`	Specifies that the pins that are moved by this command will have a *Fixed* status. *Default*: By default, the pins moved by this command have a *Placed* status. *Data_type*: bool, optional
`-moveFixedPin`	Specifies that the pins with a *Fixed* status can also be moved. *Data_type*: bool, optional
`-newLayer` *layer*	Specifies a new pin layer. **Note:** Layers can be specified using the LEF layer names or layer ID numbers. *Data_type*: string, optional
`-noSnap`	Disables pins from being snapped to the routing grid. *Data_type*: bool, optional
`-pinNames` *pinList*	Specifies the pin(s) to be aligned between the facing edges of the target block and reference block. You can use wildcards (`*?`) with this parameter. If you specify more than one pin, you must enclose them in curly braces. *Data_type*: string, optional
`-ptnInst` *ptnName*	Specifies the target instance where its pins will be aligned to the pins of the specified instance. *Data_type*: string, optional
`-refObj` *refObjectName*	Specifies the reference object where the specified pins of the target block will be aligned to the pins of the reference object. The reference object can be a standard cell, an I/O pin, an I/O pad, a macro, or all I/O pins in the design by specifying the top cell as the reference object. Now the `pinAlignment` command can also take uncommitted ptn as the reference object. **Note:** You can also use wildcard (* or ?) for specifying the reference object names. *Data_type*: string, optional
`-refType {partition_pin hinst}`	Specifies the type of reference objects to consider. You can specify partition pins or hinsts. *Data_type*: enum, optional
`-verbose`	Generates intermediate messages. *Data_type*: bool, optional

Examples

- The following command aligns `A0` and `A1` pins of `p1` to the reference pins of `blockA`:

  ```
  pinAlignment -refObj blockA -ptnInst p1 -pinNames {A0 A1}
  ```

- Assume the following netlist:

  ```
  module testcase (in1, in2, in3);
  input in1, in2, in3;
  ```

```
wire net;
a i_a(.in1(in1), .in2(in2), .in3(in3), .out1(net));
BUFX1 i0(.A(net));
endmodule

module a(in1, in2, in3, out1);
input in1, in2, in3;
output out1;
endmodule
```

The following command aligns pins `in1`, `in2`, and `in3` on partition `i_a`:

```
pinAlignment -refObj testcase -ptnInst i_a
```

```
  Moving pin [in1] of partition [a] connected to pin [in1] of top cell [testcase] to ALIGNED location ( 0.000 21.000 Metal3).
  Moving pin [in2] of partition [a] connected to pin [in2] of top cell [testcase] to ALIGNED location ( 0.000 38.920 Metal5).
  Moving pin [in3] of partition [a] connected to pin [in3] of top cell [testcase] to ALIGNED location ( 0.000 57.960 Metal3).
  Aligned 3 pin(s) of the partition [a] successfully.
```

Note: Only 3 of the 4 partition pins are moved because pin `out1` is connected to a nearby instance rather than a top-level IO pin.

Related Information

- `assignIoPins`
- `assignPtnPin`
- `legalizePin`
- `checkPinAssignment`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Validating Pin Placement Results
  - Aligning Partition Pins

pinAnalysis

```
pinAnalysis
[-help]
[-checkLegality]
[-outFile fileName]
[-noHtml]
```

Reports certain Quality of Results (QoR) metrics for pin assignment. The `pinAnalysis` command deletes the existing routes, reroutes the design ensuring that the routes pass through partition pins, and reports pin assignment QoR metrics. You can run this command after assigning partition pins.

Notes:

- The `pinAnalysis` command creates new routes. The original routes are not retained.
- The `pinAnalysis` command reroutes the design using `setRouteMode -earlyGlobalRoutePartitionHonorPin`. This command thus takes at least as much time as running Early Global Route.
- In addition to displaying the report on screen, the command also generates the report in an HTML file named `pinAnalysis.html`.
- This command is not supported by Multi Level Hierarchical Flow. It displays a warning message when used for a design with nested partitions and generates results based on single level partitions.

Parameters

`-checkLegality`	Checks the legality of pin assignment (similar to the `checkPinAssignment` command) and prints a summary report of the results. The legality details for every partition are available through hyperlinks that are provided in the HTML report file. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `pinAnalysis` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man pinAnalysis`
`-noHtml`	Specifies that HTML report file should not be created. *Data_type*: bool, optional
`-outFile` *fileName*	In addition to displaying the report on screen, prints the report to the specified file. *Data_type*: string, optional

Example

The following command checks the legality of pin assignment, displays the QoR report for pin assignment on the screen, and also prints the QoR report to a file named `pinAnalysisMetricReport`

```
pinAnalysis -checkLegality -outFile pinAnalysisMetricReport
```

Related Information

- `checkPinAssignment`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Validating Pin Placement Results
  - Reporting QoR of Pin Assignment

place_cells_at_center_for_feedthrus

```
place_cells_at_center_for_feedthrus
[-help]
[-insts list_of_selected_insts]
```

Places all the instances corresponding to a given partition at its center to facilitate feed-through insertion in the flow during the floorplanning stage. It quickly changes the design status to “placed” by placing all unplaced cells in the middle of the partition, so that feedthrough can be run.

Note: It places all the cells on the same location (overlapping).

Note: It can handle rectilinear partition fences and nested designs.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `place_cells_at_center_for_feedthrus` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man place_cells_at_center_for_feedthrus`
`-insts list_of_selected_insts`	Specifies the name of the instances that will be placed at the center of the partition. You can use wildcards (*?) with this parameter. For example, the following command places all unplaced cells that have ECO in their name to the center of the partition: `place_cells_at_center_for_feedthrus -insts *ECO*` *Data_type*: string, optional

Example

The following command changes the design status to “placed” by placing all unplaced cells in the middle of the partition, so that feedthrough can be run.

```
place_cells_at_center_for_feedthrus
```

Related Information

- `insertPtnFeedthrough`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Inserting Feedthroughs

place_partition_clone

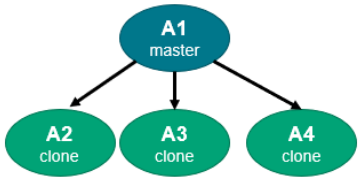
```
place_partition_clone
[-help]
-hinst partition_clone_hinst_name
-location {x y}
[-orient {R0 R180 MX MY}]
[-skip_sync]
[-sync_objs {placeBlockage routeBlockage pinBlockage macros fences}]
```

Places the clones, set their orientation and syncs up the data with other hierarchical instances (master and clones) of the same module.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `place_partition_clone` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man place_partition_clone`
`-hinst partition_clone_hinst_name`	Specifies the name of partition clone hierarchical instance. *Data_type*: string, required
`-location {x y}`	Specifies location of the clone partition. *Data_type*: point, required
`-orient {R0 R180 MX MY}`	Specifies orientation of the clone partition. The orientation can be one of the following: `R0, R180, MX, and MY` *Data_type*: enum, optional
`-skip_sync`	When specified, the tool skips the sync-up of reference hinst (`-hinst`) objects with other hinsts of the same module. *Data_type*: bool, optional
`-sync_objs {placeBlockage routeBlockage pinBlockage macros fences}`	When specified, the tool does the sync-up of the specified objects from reference hinst with other hinsts of same module. The objects that can be synced up include placeBlockage, routeBlockage, pinBlockage, macros, and fences. *Data_type*: string, optional

Example




```
Hinst: A1 (master), A2 (clone), A3(clone), A4 (clone)
```

- The following commands places the clone A2 at location 2,5. If partition block has FP/def/route objs/data present, the command will sync the data from A2 to all other hinsts (A1 (master), A3 , A4 (clones)).
  ```
  place_partition_clones -hinst A2 -location {2 5}
  ```
- The following commands places the clone A3 at location 12,15 and changes the orientation to R180 (if not already R180). If partition block has FP/def/route objs/data present, the command will sync the data from A3 to all other hinsts (A1 (master), A2 , A4 (clones))
  ```
  place_partition_clones -hinst A3 -location {12 15} -orient R180
  ```
- The following commands places the master A1 at location 22,25. If partition block has FP/def/route objs/data present, the command will sync the data from A1 to all other hinsts (A2, A3, A4 (clones)).

```
place_partition_clones -hinst A1 -location {22 25}
```

- The following commands places the master A1 at location 22,25. If partition block has FP/def/route objs/data present, the command will not sync any data to other hinsts.

  ```
  place_partition_clones -hinst A1 -location {22 25} -skip_sync
  ```

- The following commands places the clone A3 at location 12,15 and changes the orientation to R180 (if not already R180). If partition block has FP/def/route objs/data present, the command will not sync any data to other hinsts.

  ```
  place_partition_clones -hinst A3 -location {12 15} -orient R180 -skip_sync
  ```

- The following commands places the clone A3 at location 12,15 and changes the orientation to R180 (if not already R180). If partition block has FP/def/route objs/data present, the command will sync the bus-guide data from A3 to all other hinsts (A1 (master), A2 , A4 (clones)).

  ```
  place_partition_clones -hinst A3 -location {12 15} -orient R180 -sync_obj {bus_guide}
  ```

Related Information

- clonePtnObj
- Partitioning the Design chapter in the *Innovus User Guide*

push_ptn_network

```
push_ptn_network
[-help]
[-exclude_ptn_list ptnNameList]
[-filter string]
[-inst_prefix instPrefixName]
[-logical_only]
[-net_prefix netPrefixName]
[-output_logical_only_nets string]
[-output_nets_for_wire_distribution string]
[-ptn_net_dir fileName]
{-nets netNameList | -file fileName | -buf_only | -insts instNameList }
[-check_only [-output fileName]]
```

Pushes down a network (std cells and connected wires) logically and physically into overlapping partitions. The `push_ptn_network` command:

- Logically pushes down a set of nets into overlapping partitions by doing all the necessary logical port creations.
- Physically pushes down the special wires of the nets into overlapping partitions and does all the necessary partition pin creations. The pushed wires look exactly the same as the original wires.
- Creates Verilog assign statements wherever required to ensure that the pushed logic is logically equivalent to the original logic.

Notes:

- The `push_ptn_network` command does not support designs with master and clone partitions. In such scenarios, only masters will be seen and the overlapping wires and logic will be pushed down into masters only.
- The `push_ptn_network` command takes care of new nets connected at top level nets automatically post committing partitions.
- The `push_ptn_network` command supports partially routed nets. It does not require the wires of a net to be DRC/LVS clean. Wires need not connect to the instance-pin of a driver or loads. Even if a wire is in the 5u radius of a source-instance-pin or a sink-instance-pin, the tool considers the net for pushdown.

Parameters

`-buf_only`	Specifies that only all buffers, and their connected nets, will be considered for pushed down. *Data_type*: bool, required
`-check_only`	Informs which nets are going to be pushed down. *Data_type*: bool, optional
`-exclude_ptn_list` *ptnNameList*	Specifies that the instances/nets within the specified partition(s) will not be pushed down. *Data_type*: string, optional
`-file` *fileName*	Specifies the name of the file containing the list of net names to be considered for push down. *Data_type*: string, required
`-filter` *string*	Specifies that only instances/nets whose name matches this specified regular expression will be pushed down. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `push_ptn_network` parameter. For a detailed description of the command and all of its parameters, use the man command: `man push_ptn_network`
`-inst_prefix` *instPrefixName*	Specifies a custom prefix for instances that are pushed down. *Data_type*: string, optional
`-insts` *instNameList*	Specifies that all instances, and their connected nets, will be considered for pushed down. **Note:** The `push_ptn_network` command accepts instances names with backspace '\' in them. *Data_type*: string, required
`-logical_only`	Logically pushes down network into overlapping partitions. *Data_type*: bool, optional
`-net_prefix` *netPrefixName*	Specifies a custom prefix for nets that are pushed down. *Data_type*: string, optional
`-nets` *netNameList*	Specifies that list of net names, and their connected instances, will be considered for pushed down. **Note:** The `push_ptn_network` command accepts nets names with backspace '\' in them. *Data_type*: string, required
`-output` *fileName*	Specifies a file name in which net names, to be pushed down, will be mentioned. This parameter is used with `-check_only`. *Data_type*: string, optional
`-output_logical_only_nets` *string*	
	Specifies a file name in which net names are mentioned. This parameter is used with `-logical_only`. *Data_type*: string, optional
`-output_nets_for_wire_distribution` *string*	
	Outputs the user-specified nets so that they can be used later during the `partition` command. *Data_type*: string, optional
`-ptn_net_dir` *fileName*	Specifies the directory that consists of the names of the pushed nets corresponding to each partition. *Data_type*: string, optional

Example

- The following command consider nets n1, n2 and n3 for push down.
  ```
  push_ptn_network -nets {n1 n2 n3}
  ```
- The following command generates a file, push_down_nets.txt, containing the names of nets that would have been considered for push down.
  ```
  push_ptn_network -nets [dbGet -e top.nets.name] -check_only -output push_down_nets.txt
  ```
- The following command generates a file, push_down_nets.txt, containing the names of nets (connected to the specified instances)that would be considered for push down.
  ```
  push_ptn_network -insts [dbGet -e top.insts.name] -check_only -output push_down_nets.txt
  ```
- The following command considers all buffers for push down.
  ```
  push_ptn_network -buf_only
  ```
- The following command considers all buffers and their connected nets for push down which have their hierarchical name (TCL) regexp matching to KK.*.
  ```
  push_ptn_network -buf_only -filter KK.*
  ```
- The following command considers all buffers and their connected nets for push down. The pushed down buffers will have their base names starting with II_
  ```
  push_ptn_network -buf_only -inst_prefix II
  ```
- `The following command` considers all buffers and their connected nets for push down. The pushed down buffers will have their base names starting with II_, and the pushed down nets will have their base names starting with NN_.`push_ptn_network -buf_only -inst_prefix II -net_prefix NN`

Related Information

- `create_pushdown_eco`
- `commit_pushdown_eco`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Partition Pushdown Replay Flow

recreatePtnCellBlockage

```
recreatePtnCellBlockage
[-help]
[ptnName]
[-noPinCutSpace | -hasPinCutSpace]
[-noViaCutSpace | -hasViaCutSpace]
```

Recreates cell blockage for a specified partition, or all partitions. This reflects incremental changes, such as pin movement. You can use this command after editing the pin in a partition with the Pin Editor, or using the `editPin` command, to move pins in the partition.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `recreatePtnCellBlockage` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man recreatePtnCellBlockage`
ptnName	Specifies the partition to recreate the cell blockage. *Default*: All partitions *Data_type*: string, optional
`-noPinCutSpace`	Specifies that no cut space will be created for pins in the partition cell blockage. **Note:** Blockages are always cut for power pins. *Data_type*: bool, optional
`-hasPinCutSpace`	Specifies that cut space will be created for pins in the partition cell blockage. **Note:** This is the default. *Data_type*: bool, optional
`-noViaCutSpace`	Specifies that no cut space will be created for vias in the partition cell blockage. *Data_type*: bool, optional
`-hasViaCutSpace`	Specifies that cut space will be created for vias in the partition cell blockage. **Note:** This is the default. *Data_type*: bool, optional

Example

- The following command recreates pin blockages for partition `EUG`:

  ```
  recreatePtnCellBlockage EUG
  ```

Related Information

- `editPin`
- Partitioning the Design chapter in the *Innovus User Guide*

report_partitions

```
report_partitions
[-help]
[topCell]
```

Reports the partitions and clones in a correct tree like format.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `report_partitions` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_partitions`
topCell	Specifies the top cell to print its cell partition tree. *Data_type*: string, optional

Example

The following command prints the partitions and clones in the design in a tree like format.

```
report_partitions
```

```
Format: partitionName, MasterOrClone, Orient, PartitionOrBlackBox (CommittedOrUncommited, Level, BottomUpLevel, In/Active): InstOrHInstName

- top (design name)
- top_partition(0x7fec92643638), Single, R0, Partition (Uncommitted,1,2,Active): T1
- sh_act_3(0x7fec92643310), Single, R0, Partition (Uncommitted,2,0,Active): T1/S2
- sh_act_2(0x7fec92642fe8), Single, R0, Partition (Uncommitted,2,0,Active): T1/S3
- sh_act_1(0x7fec92642cc0), Single, R0, Partition (Uncommitted,2,0,Active): T1/S4
- sh_act(0x7fec92642998), Single, R0, Partition (Uncommitted,2,0,Active): T1/S1
- dpu(0x7fec94c40bc0), Clone, R0, Partition (Uncommitted,2,1,Active): T1/P2
- scl(0x7fec94c405c0), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P2/c15
- scl(0x7fec94c40560), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P2/c14
- scl(0x7fec94c40500), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P2/c13
- scl(0x7fec94c404a0), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P2/c12
- scl(0x7fec94c40440), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P2/c11
- scl(0x7fec94c403e0), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P2/c10
- scl(0x7fec94c40380), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P2/c9
- scl(0x7fec94c40320), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P2/c8
- scl(0x7fec94c402c0), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P2/c7
- scl(0x7fec94c40260), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P2/c6
- scl(0x7fec94c40200), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P2/c5
- scl(0x7fec94c401a0), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P2/c4
- scl(0x7fec94c40140), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P2/c3
- scl(0x7fec94c400e0), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P2/c2
- scl(0x7fec94c40080), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P2/c1
- scl(0x7fec94c40020), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P2/m1
- dpu(0x7fec92642670), Master, R0, Partition (Uncommitted,2,1,Active): T1/P1
- scl(0x7fec94c40b60), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P1/c15
- scl(0x7fec94c40b00), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P1/c14
- scl(0x7fec94c40aa0), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P1/c13
- scl(0x7fec94c40a40), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P1/c12
- scl(0x7fec94c409e0), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P1/c11
- scl(0x7fec94c40980), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P1/c10
- scl(0x7fec94c40920), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P1/c9
- scl(0x7fec94c408c0), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P1/c8
- scl(0x7fec94c40860), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P1/c7
- scl(0x7fec94c40800), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P1/c6
- scl(0x7fec94c407a0), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P1/c5
- scl(0x7fec94c40740), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P1/c4
- scl(0x7fec94c406e0), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P1/c3
- scl(0x7fec94c40680), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P1/c2
- scl(0x7fec94c40620), Clone, R0, Partition (Uncommitted,3,0,Active): T1/P1/c1
- scl(0x7fec92642348), Master, R0, Partition (Uncommitted,3,0,Active): T1/P1/m1
```

Related Information

- Partitioning the Design chapter in the *Innovus User Guide*

reportPinAssignStatistics

```
reportPinAssignStatistics
[-help]
[-maxFanOut fanOutCount]
[-maxRegisterFanOut registerFanOutCount]
[-outFile pinQoRFileName]
[-hinst hinstNameList | -ptn ptnNameList]
```

Generates the pin QoR data for a specified partition(s).

Note:

- If a net is not routed, then, manhattan distance between source-sink pairs is be used to replace corresponding wire-length distance.
- For floating pins (from outside) of a partition,the pin is always assumed to be within the timing cone box.
- Pin QoR statistics is not generated for blackbox partitions

Parameters

`-help`	Outputs a brief description that includes type and default information for each `reportPinAssignStatistic` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reportPinAssignStatistic`.
`-hinst hinstNameList`	Generates the pin QoR metrics for all pins of the specified hierarchical instance name or list of instance names. *Default*: All partition and clone hierarchical instances. *Data_type*: string, optional
`-maxFanOut fanOutCount`	Specifies the maximum fanout count. The QoR report is not generated for a pin if the number of instances in its timing cone fanout exceeds this maximum fanout count, *Default*: 1000 *Data_type*: string, optional
`-maxRegisterFanOut registerFanOutCount`	
	Specifies the maximum register fanout count. The QoR report is not generated for a pin if the number of registers in its timing cone fanout exceeds this maximum fanout count. *Default*: 50 *Data_type*: string, optional
`-outFile pinQoRFileName`	Specifies an output report file for the pin QoR data. *Default*: `./pin_qor.txt` *Data_type*: string, optional
`-ptn ptnNameList`	Generates the pin QoR metrics for all pins of the partition or list of partitions. *Default*: All partition and clones. *Data_type*: string, optional

Example

- The following command creates a `pin_qor.txt` report file:
  ```
  reportPinAssignStatistics
  ```
- The following command generates a reference pin QoR file named `pin_qor_reference.txt`.
  ```
  reportPinAssignStatistics -outFile pin_qor_reference.txt
  ```
- The following command generates a target pin QoR file named `pin_qor_target.txt`.
  ```
  reportPinAssignStatistics -outFile pin_qor_target.txt
  ```

Related Information

- `comparePinAssignStatistics`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Pin QoR Metrics and Comparison

reportTwoPinChain

```
reportTwoPinChain
[-help]
[-reset  |
[[-output fileName] [-through string]
[-noOverwrite ]
[-showAll ]
[-strict ]
[-tolerance value]]]
```

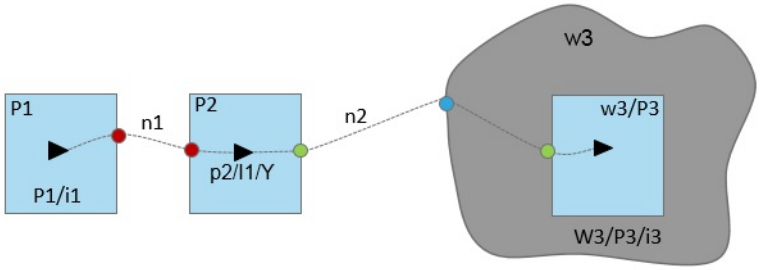
Reports two pin chains for nets, instterms, and hinstterms. This command can traverse through buffers, inverters, and assign-statements and report the complete two pin chain. You can specify any element of the chain to get the same results. The `reportTwoPinChain` command is used after pin assignment to analyse results and debug issues especially in cases where the expectation to align all partition pins across a two-pin chain is not met.

Parameter

`-help`	Outputs a brief description that includes type and default information for each `reportTwoPinChain` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reportTwoPinChain`
`-noOverwrite`	When specified the report continues to display the previous 2-pin-chains also. *Default*: false *Data_type*: bool, optional
`-output fileName`	Specifies the file in which the 2-pin nets are traced and dumped. *Default*: stdout *Data_type*: string, optional
`-reset`	Removes GUI markers and GUI objects, created during previous invocations. No two pin chain reporting. *Default*: false *Data_type*: bool, optional
`-showAll`	Displays the intermediate instances of the chain also. *Default*: false *Data_type*: bool, optional
`-strict`	Strictly displays only pure two pin chains even at module port connections. *Default*: false *Data_type*: bool, optional
`-through string`	Specifies the list of names of net, instterms, hinstterms for which 2-pin chains are traced and dumped. *Data_type*: string, optional
`-tolerance value`	Specifies the tolerance range. This specified difference in ordinates treated as still aligned. *Default*: 1.0 microns *Data_type*: float, optional

Example

- In the following diagram P1, P2, W3/P3 are partitions and W3 is a non-partition verilog module.
  It shows a pure two pin connection where red and green pins aligned together in one chain.



  The following command reports this two pin chain passing through p2/I1/Y:
  ```
  reportTwoPinChain -through p2/I1/Y
  ```

- The following command reports the two pin chains passing through `DTMF_INST/TDSP_CORE_INST/port_pad_data_out[13]`:
  ```
  reportTwoPinChain -through DTMF_INST/TDSP_CORE_INST/port_pad_data_out[13]
  ```

  ```
  Report:
  2-pin-chain through: DTMF_INST/TDSP_CORE_INST/port_pad_data_out[13] ...
  chain terminator: other pin of buffer is multi-fanout
  net: DTMF_INST/TDSP_CORE_INST/port_data_in[13]
  buffer-term                    :output :placed      : DTMF_INST/TDSP_CORE_INST/TDSP_CORE_GLUE_INST/i_9686/Y
  module-port                    :output :unplaced    : DTMF_INST/TDSP_CORE_INST/TDSP_CORE_GLUE_INST/port_data_in[13]
  module-port                    :input  :unplaced    : DTMF_INST/TDSP_CORE_INST/PORT_BUS_MACH_INST/data_in[13]
  buffer-term                    :input  :placed      : DTMF_INST/TDSP_CORE_INST/PORT_BUS_MACH_INST/FE_RC_7_0/A
  net: DTMF_INST/FE_FEEDX_NET_D__tdsp_core__0_tdsp_portO_13_
  buffer-term                    :output :placed      : DTMF_INST/TDSP_CORE_INST/PORT_BUS_MACH_INST/FE_RC_7_0/Y
  module-port                    :output :unplaced    : DTMF_INST/TDSP_CORE_INST/PORT_BUS_MACH_INST/pad_data_out[13]
  partition-pin                  :output :placed      : DTMF_INST/TDSP_CORE_INST/port_pad_data_out[13]
  partition-pin                  :input  :placed      : DTMF_INST/RESULTS_CONV_INST/FE_FEEDX_NET_D__tdsp_core__0_tdsp_portO_13_
  buffer-term                    :input  :placed      : DTMF_INST/RESULTS_CONV_INST/FE_OFC28_FE_FEEDX_NET_D__tdsp_core__0_tdsp_portO_13_/A
  net: tdsp_portO[13]
  buffer-term                    :output :placed      : DTMF_INST/RESULTS_CONV_INST/FE_OFC28_FE_FEEDX_NET_D__tdsp_core__0_tdsp_portO_13_/Y
  partition-pin                  :output :placed      : DTMF_INST/RESULTS_CONV_INST/FE_FEEDX_P_tdsp_portO_13_
  module-port                    :output :unplaced    : DTMF_INST/port_pad_data_out[13]
  module-port                    :input  :unplaced    : IOPADS_INST/tdsp_portO[13]
  buffer-term                    :input  :placed      : IOPADS_INST/Ptdspop13/I
  net: port_pad_data_out[13]
  buffer-term                    :output :placed      : IOPADS_INST/Ptdspop13/PAD
  module-port                    :output :unplaced    : IOPADS_INST/tdsp_port_out[13]
  chip-io                        :output :fixed       : port_pad_data_out[13]
  chain terminator: chip-io
  ```

- The following command reports the two pin chains passing through `F2/D2/A/out` and writes it to `report.txt` file:
  ```
  reportTwoPinChain -through F2/D2/A/out -output report.txt
  ```

Related Information

- `assignPtnPin`
- Partitioning the Design chapter in the *Innovus User Guide*

reportUnalignedNets

```
reportUnalignedNets
[-help]
[-alignDistance distance]
[-alignFeedThruPins]
[-considerMacroPinWithinDistance distance]
[-ignoreLength length]
[-noHighlight]
[-reportAllWithinChannelWidth width]
[-rptFile filename]
[-statistics]
[-steinerBox]
[-verbose]
[[ -ptnToPtn {aligned unaligned layerMismatch samePtn nonNbr multiFanout all unplaced none} ]
[ -topToPtn {aligned unaligned layerMismatch samePtn nonNbr multiFanout all unplaced none} ]]
[-pin_file filename | -exclude_pin_file filename]
```

Reports the names of the nets where the pins are not aligned. It reports the nets in the following cases:

- nets that have two pins on two different partitions and/or clones and are not connected to top level hard-macro, standard cell, or I/O.
- nets that have two pins that are connected to the same partition.
- nets that have one pin connected to top level hard-macro, standard cell, or I/O and the other pin connected to a partition or a clone.

Notes:

- The nets reported by this command are also highlighted in the artwork window.
- The `assignPtnPin` command internally calls the `reportUnalignedNets` command to generate the summary report at the command line.
- The `reportUnalignedNets` command measures the average misalignment with standard deviation for multi-partition nets.
- The `reportUnalignedNets` command detects and reports pins that are placed on non-facing edges.

Parameters

`-alignDistance` *distance*	Specifies the distance, in microns, up to which the pins are assumed to be aligned. With this parameter specified, the `reportUnalignedNets` command assumes that the pins are aligned within the specified distance even though they are not physically aligned. *Data_type*: float, optional
`-alignFeedThruPins`	Reports the deviation of partition pins from chain's source-target box. *Data_type*: bool, optional
`-considerMacroPinWithinDistance` *distance*	Reports nets connected to macro-pin which are within specified distance from nearest partition boundary. *Default*: 3 microns *Data_type*: float, optional

`-exclude_pin_file` *filename*	Specifies the pin list to be excluded for a partition in a file. In the pin file, the pins and nets are specified in the following format: `Partition:` *PtnName/BlockName* *Pin1* *Pin2* .. *PinN* `Partition:` *PtnName/BlockName* *Pin1* *Pin2* .. *PinN* `Nets:` *netA* *netB* .. *netN* **Note:** While specifying the pin list in the pin_file, it is necessary to give the “Partition” keyword followed by the name of the partition or block. If this is not specified, no operations are performed. You can specify multiple partitions in the same pin_file. The pin_file does not support buses or wild cards for pin names. If a pin specified in the list or a partition is not present in the design, a warning is displayed. *Data_type*: string, optional
`-help`	Outputs a brief description that includes type and default information for each `reportUnalignedNets` command parameter. For a detailed description of the command and all of its parameters, use the man command: `man reportUnalignedNets`
`-ignoreLength` *length*	Specifies the length, in microns. It ignores the nets from checking unalignment whose length is greater the specified length value. *Default*: 5000 These ignored nets will be reported in the ignored category in the output report file. *Data_type*: float, optional
`-steinerBox`	Report the deviation of partition pins from the connectivity box of the partition. *Data_type*: bool, optional
`-noHighlight`	Disables the highlight capability. *Data_type*: bool, optional
`-rptFile` *fileName*	Specifies an output report file. **Note**: Default name for report file is `<designName>_unaligned.rpt` *Data_type*: string, optional
`-ptnToPtn {aligned | unaligned | layermismatch | sameptn |nonnbr | multifanout | all | unplaced | none}`	

	Reports nets that have two pins on two different partitions and/or clones and are not connected to top level hard-macro, standard cell, or I/O. • `aligned`: Reports nets that have aligned pins. • `all`: Reports all categories of unaligned pins. • `layermismatch`: Reports nets that have pins at an aligned location but different layers. • `multiFanout`: Reports nets that have multiple fanouts • `none`: Disables reporting of nets. • `nonnbr`: Reports nets that have two pins on two different partitions that are not adjacent. • `sameptn`: Reports nets that have two pins that are connected to the same partition. • `unaligned`: Reports nets that have unaligned pins. • `unplaced`: Reports nets that have unplaced pins. *Data_type*: (enum)+, optional
`-pin_file` *filename*	Specifies the pin list for a partition in a file. In the pin file, the pins and nets are specified in the following format: `Partition:` *PtnName/BlockName* *Pin1* *Pin2* .. *PinN* `Partition:` *PtnName/BlockName* *Pin1* *Pin2* .. *PinN* `Nets:` *netA* *netB* .. *netN* **Note:** While specifying the pin list in the pin_file, it is necessary to give the “Partition” keyword followed by the name of the partition or block. If this is not specified, no operations are performed. You can specify multiple partitions in the same pin_file. The pin_file does not support buses or wild cards for pin names. If a pin specified in the list or a partition is not present in the design, a warning is displayed. *Data_type*: string, optional
`-reportAllWithinChannelWidth` *width*	Reports all the unaligned nets within the specified channel width even if the `-ignoreLength` parameter has been specified. *Data_type*: float, optional
`-statistics`	Reports average misalignment with standard deviation and layer difference count for aligned, layer mismatched, and unaligned ptnToPtn 2-pin nets of partitions. It is also used for reporting pin QOR. *Data_type*: bool, optional
`-topToPtn {aligned | unaligned | layermismatch | sameptn |nonnbr | multifanout | all | unplaced | none}`	

	Reports nets that have one pin connected to top level hard-macro, standard cell, or I/O and the other pin connected to a partition or a clone. • `aligned`: Reports nets that have aligned pins. • `all`: Reports all categories of unaligned pins. • `layerMismatch`: Reports nets that have pins at an aligned location but different layers. • `multiFanout`: Reports nets that have multiple fanouts • `none`: Disables reporting of nets. • `nonNbr`: Reports nets that have two pins on two different partitions that are not adjacent. • `samePtn`: Reports nets that have two pins that are connected to the same partition. • `unaligned`: Reports nets that have unaligned pins. • `unplaced`: Reports nets that have unplaced pins. *Data_type*: (enum)+, optional
`-verbose`	Reports all categories in log file. By default, this parameter is not enabled and only the nets that have unaligned pins are reported in the log file. **Note**: There will be no change in the report file with use of verbose parameter. *Data_type*: bool, optional

The following are some of the attributes specified in the output report file to report the names of nets where the pins are not aligned:

- `layer mismatch`: Specifies nets that have pins at an aligned location but different layers.
- `non-neighbor`: Specifies nets that have two pins on two different partitions that are not adjacent.
- `ptnToPtn`: Specifies nets that have two pins on two different partitions and/or clones.
- `topToPtn`: Specifies nets that have one pin connected to top level hard-macro, standard cell, or I/O and the other pin connected to a partition or a clone.
- `unAligned`: Specifies nets that have unaligned pins.

Example

The following command reports all nets which are placed in a channel equal to or narrower than 7.5 microns irrespective of the `-ignoreLength` parameter.

```
reportUnalignedNets -ptnToPtn unaligned -rptFile a -ignoreLength 10.0 -reportAllWithinChannelWidth 7.5
```

The following command creates an a.out report file:

```
reportUnalignedNets rptFile a.out -noHighlight -ptnToPtn all -topToPtn all
```

A summary of the reported unaligned nets displays on the Innovus console and in the Innovus log file.
The following is a sample of the resulting a.out report file.

```
############################################################
#  Generated by:      Cadence Innovus
#  OS:                     Linux x86_64(Host ID nofclo110)
#  Generated on:      Wed Feb 27 16:50:02 2013
#  Design:                NESTED_TOP
#  Command:           reportUnalignedNets -ptnToPtn all -topToPtn all -rptFi...
############################################################
====================
Net Specific report:
====================
pin_i_TOP_A0__A1__A2forPorts_BB_[3] | nested-2-pin | aligned 3/4
pin_i_A1__A2forPorts_BB_[3] (I_K2/I_K2) | pin_i_A0__A1__A2forPorts_BB_[3] (I_K2) | ptnToPtn | aligned
pin_i_A2forPorts_BB_[3] (I_K2/I_K2/I_K2) | pin_i_A1__A2forPorts_BB_[3] (I_K2/I_K2) | ptnToPtn | aligned
input_bus[3] (I_K2/I_K2/I_K2/I_A2_K4/BBOX) | pin_i_A2forPorts_BB_[3] (I_K2/I_K2/I_K2) | ptnToPtn | aligned
pin_i_A0__A1__A2forPorts_BB_[3] (I_K2) | pin_i_TOP_A0__A1__A2forPorts_BB_[3] (NESTED_TOP) | topToPtn | unaligned
pin_i_TOP_A0__A1__A2forPorts_BB_[2] | nested-2-pin | aligned 1/4
pin_i_A2forPorts_BB_[2] (I_K2/I_K2/I_K2) | pin_i_A1__A2forPorts_BB_[2] (I_K2/I_K2) | ptnToPtn | aligned
pin_i_A0__A1__A2forPorts_BB_[2] (I_K2) | pin_i_TOP_A0__A1__A2forPorts_BB_[2] (NESTED_TOP) | topToPtn | unaligned
pin_i_A1__A2forPorts_BB_[2] (I_K2/I_K2) | pin_i_A0__A1__A2forPorts_BB_[2] (I_K2) | ptnToPtn | unaligned
input_bus[2] (I_K2/I_K2/I_K2/I_A2_K4/BBOX) | pin_i_A2forPorts_BB_[2] (I_K2/I_K2/I_K2) | ptnToPtn | unaligned
...
...
```

```
I_K3/net15_OP_B1_0_And_IP_B1_2_SC[3] | 2-pin | aligned 0/1
pin_o_B1_0_III_SC_[0] (I_K3/I_K4) | pin_i_B1_2_SC_[0] (I_K3/I_K2) | ptnToPtn | nonNbr
I_K3/net16_OP_B1_0_And_IP_B1_2_SC | 2-pin | aligned 0/1
pin_o_B1_0_III_SC_4 (I_K3/I_K4) | pin_i_B1_2_SC_4 (I_K3/I_K2) | ptnToPtn | nonNbr

==================
Net Summary report:
==================
There are 54 ptnToPtn 2-pin net(s)
There are 0 topToPtn 2-pin net(s)
There are 200 nested ptnToPtn 2-pin net(s) with multiple partition pins between source and sink
There are 34 nested topToPtn 2-pin net(s) with multiple partition pins between source and sink
================================================================================
Summary of pin pairs on above nets
================================================================================+=====================
         |aligned|unaligned|layerMismatch|samePtn|nonNeighbor|multiFanout|unplaced|ignored|Total
=============================================================================================================
topToPtn |    11 |      22 |           1 |     0 |         0 |         0 |      0 |     0 |  34
ptnToPtn |   590 |     220 |          16 |     0 |        58 |         0 |      0 |     0 | 884|
=============================================================================================================
Total    |   601 |     242 |          17 |     0 |        58 |         0 |      0 |     0 | 918
=============================================================================================================
=============================================================================================================
                 Level based partition net summary
=============================================================================================================================================
                        |aligned|unaligned|layerMismatch|samePtn|nonNeighbor|multiFanout|unplaced|ignored|Total|
=============================================================================================================================================
NESTED_TOP (level-0)    |   121 |      30 |           1 |     0 |         0 |         0 |      0 |     0 | 152 |
I_K2 (level-1)          |   161 |      18 |           7 |     0 |         0 |         0 |      0 |     0 | 186 |
I_K3 (level-1)          |    52 |      42 |           0 |     0 |        58 |         0 |      0 |     0 | 152 |
I_K2/I_K2 (level-2)     |   184 |      35 |           1 |     0 |         0 |         0 |      0 |     0 | 220 |
I_K3/I_K2 (level-2)     |    10 |       6 |           0 |     0 |         0 |         0 |      0 |     0 |  16 |
I_K3/I_K3 (level-2)     |    16 |      16 |           0 |     0 |         0 |         0 |      0 |     0 |  32 |
I_K3/I_K4 (level-2)     |    12 |      32 |           4 |     0 |         0 |         0 |      0 |     0 |  48 |
I_K2/I_K2/I_K2 (level-3)|    45 |      63 |           4 |     0 |         0 |         0 |      0 |     0 | 112 |
=============================================================================================================================================
```

Related Information

- alignPtnClone
- assignPtnPin
- Partitioning the Design chapter in the *Innovus User Guide*

resizeBlackBox

```
resizeBlackBox
blackBoxName
[-help]
[-width width]
[-height height]
[-aspectRatio ratio]
```

Resizes a blackbox.

Parameters

`-aspectRatio` *ratio*	Specifies the new aspect ratio, which is height divided by width, with the blackbox area constant. *Data_type*: float, optional
blackBoxName	Specifies the name of the blackbox. *Data_type*: string, required
`-height` *height*	Specifies the new height in user units. *Data_type*: float, optional
`-help`	Outputs a brief description that includes the type and default information for each `resizeBlackBox` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man resizeBlackBox`
`-width` *width*	Specifies the new width in user units. *Data_type*: float, optional

Examples

- The following command resizes the height of `blackbox1` to 600.5 µm. The width remains the same dimension.
  ```
  resizeBlackBox blackbox1 -height 600.5
  ```
- The following command resizes the height of `blackbox2` to be twice as long as its width. The area remains the same.
  ```
  resizeBlackBox blackbox2 -aspectRatio 2
  ```

Related Information

- `specifyBlackBox`
- Partitioning the Design chapter in the *Innovus User Guide*

saveBlackBox

```
saveBlackBox
[-help]
{instName | -all}
[-5.6]
[-lefFile string]
```

Creates a blackbox LEF file containing the pin definitions and obstructions.

Note: The `savePartition` command also automatically saves this file, among others.

Parameters

`-5.6`	Specifies the LEF file format version. *Data_type*: bool, optional
`-all`	Specifies all blackbox instances. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `saveBlackBox` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man saveBlackBox`
instName	Specifies the name of the blackbox instance. You can use wildcards (`*?`) with this parameter. *Data_type*: string, optional
`-lefFile` *string*	Specifies the LEF filename. If you do not specify a name, the default is *cellTypeName*`.lef`. *Data_type*: string, optional

Example

The following command creates a LEF file containing the pin definitions and obstructions for all blackbox instances that start with the letter `A` to a LEF file named `BBox.lef`:

```
saveBlackBox A* -lefFile BBox.lef
```

Related Information

- `specifyBlackBox`
- `savePartition`
- `getBlackBoxArea`
- `unspecifyBlackBox`
- Partitioning the Design chapter in the *Innovus User Guide*

savePartition

```
savePartition
[-help]
[ partitionName  { list }]
[-fplan]
[-noStdCells]
[-noTrialRoute]
[-verilog]
[ [-dir  dirName ]
[ {[-noFPlan | {[-noNetlist] [-savePlacement]}]
[-def [-savePlacement]]
[-defNoCutRow]
[-ptnSite]
[-module_model_tag tag_name]
[-topLevelNoPtnModule]} |
{[-ptnLib  ptnLibName ][-ptnView  ptnViewName ][-topView  topViewName ]}]]
```

Saves the partition information to the current directory or a specified directory. This command is run after running the `partition` command to generate timing budget files and stamp model files.

Notes:

- The `savePartition` command does not write GNC rules in the partition floorplan file for multiple-supply voltage (MSV) designs.The Common Power Format (CPF) file contains this information.
- The `savePartition` command supports partitions with FlexModels. If there are FlexModels at the top-level, then the saved top-level netlist contains FlexModels. If there are no FlexModels outside partitions, then the top-level netlist will be the original full netlist. Similarly, if a partition has FlexModels, then its partition netlist will contain flexmodes.
- The `savePartition` command saves a file with the hierarchy information into the top-level design directory for a FlexModel design with nested partitions. This hierarchy information is used by the `replace_proto_model` command. When the `replace_proto_model` command replaces the FlexModel netlist back to the original full netlist after the `savePartition` command, the hierarchy information for the nested partitions is automatically available.
- The `savePartition` command saves the design information to the global init_* variables in the *.global file.
- The `savePartition` command supports multi-threading. This reduces the run time for huge designs by saving individual blocks in parallel.
- The `savePartition` command saves mmmc data without timing information for partitions for which timing budgets are not derived.
- The `savePartition` command automatically saves all the bus guide constraints with associated net groups. This enables bus routing at block level as per chip level constraints.
- The `savePartition` command automatically saves all the cell padding information to each partition block data. This information is saved in partitioned block .fp file.
- The `savePartition` command inherits all the attributes assigned to a net for block implementation flow. Preserving these attributes allows the Early Global Router and the NanoRoute Router to route the nets following specific requirements.
- The `savePartition` command writes out an additional DEF file for each block with its scan information in the `<partition_name>.scandef.gz` format.
- The `savePartition` command saves an additional file in each parent directory in the <design_name>.pg.gz format to support for PG-file in hierarchical flow. The `assembleDesign` command loads this file when it is run with the `-fe` mode.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `savePartition` parameter. For a detailed description of the command and all of its parameters, use the man command: `man savePartition`
`-def`	Writes out the DEF format files for the design. *Data_type*: bool, optional
`-defNoCutRow`	Specifies that, when row information is written to the DEF file, the rows are not cut against placement obstructions, block halos, or power domain fence minimum gaps. *Data_type*: bool, optional

`-dir dirName`	The directory name that will contain the partition information/files. *Data_type*: string, optional
`-fplan`	Writes out incremental floorplan by pushing down floorplan changes from a full chip level design to a partition block. This option allows you to run `savePartition` for a uncommitted partition without running the `partition` command. For committed partition `savePartition -fplan` reports an error. The `savePartition -fplan -dir` command honors blackboxes and saves limited floorplan information for blackboxes in the previously saved data created by the `savePartition -dir` command.. **Note:** After `savePartition -fplan` command, a non-committed partition still remains uncommitted. *Data_type*: bool, optional
`-module_model_tag tag_name`	Specifies the module model tag to be used for saving the partition. This creates pnr and lef module models for each partition block. For top-level design, this also creates a pnr module model. **Note:** This parameter is part of the Integrated Hierarchical Database (iHDB) flow. Before using this parameter, you must set the default central directory name (`set_module_model -default_dir default_model_data_directory`) where the module models data will be saved/loaded. **Note:** These block-level and top-level pnr models should be used for implementing the block-level and top-level designs, respectively. For example, if design dtmf has the ptn_wrapper partition and the default global data directory is /myproject/DATA. With `savePartition -module_model_tag init`, a pnr model will be created for top-level design in the following directory: `myproject/DATA/dtmf/init/pnr`. For partition ptn_wrapper, a pnr and lef models will be created in the `/myproject/DATA/ptn_wrapper/init/pnr` and `/myproject/DATA/ptn_wrapper/init/lef` directories. *Data_type*: string, optional
`-noFPlan`	Saves partitions without generating a floorplan file for the partitioned design. *Data_type*: bool, optional
`-noNetlist`	Saves partitions without generating a netlist file for the partitioned design. *Data_type*: bool, optional
`-noStdCells`	When you specify this option, standard cells are not written in the DEF file. *Data_type*: bool, optional
`-noTrialRoute`	When you specify this option, routed wires are not written in the DEF file. *Data_type*: bool, optional
partitionName list	Specifies the names of each partition to be saved. *Data_type*: string, optional
`-ptnLib ptnLibName`	Specifies an OpenAccess database library directory where the top-level design and the block-level design for all the partitions will be saved. This parameter is used when saving the partition into an OpenAccess database. **Note:** For OpenAccess data, you can save the specified partitions using the `partitionName` parameter. However, if the specified partition is a top-level design then all committed partitions must be saved just like no partition names are specified to avoid unbound references in OpenAccess. *Data_type*: string, optional
`-ptnSite`	Outputs sites for partitions in the LEF file. *Data_type*: bool, optional

`-ptnView` *ptnViewName*	Specifies the view name for the top and the partition views. This parameter is used when saving the partition into an OpenAccess database. *Default:* The default value of the view is layout **Note:** • If `-ptnView` is specified, but not `-ptnLib`, then the current design lib is used. You do not have to use the `savePartition` command to create a special library to hold the partition data. You can create the partition data into the same design library that holds the full-flat version of the design. In this case, cellviews will be overwritten if they already exist. • If `-ptnView` is specified, but not `-topView`, then all cell views created with the `savePartition` command will have the `ptnViewName` appended to the top cell's cellview. In order to prevent overwriting the top cell's cellview if the name already exists, use the `-topView` option to specify a different view name than the one used for the rest of the partitions being created by the `savePartition` command. • If `-dir` is specified, but `-ptnView` is not, then the `savePartition` command would be in the non-OA mode. If `-dir` and `-ptnView` are specified then the `dir` is created, but instead of putting design data into it, only the `cds.lib` is added to the directories created. *Data_type*: string, optional
`-savePlacement`	When used with the `-def` parameter, specifies that the unplaced standard cells should also be saved to the DEF file. This is equivalent to the `-unplaced` parameter of the `defOut` command. When used with the `-pdef` parameter, specifies that the placed standard cells should also be saved to the PDEF file. *Data_type*: bool, optional
`-topLevelNoPtnModule`	Suppresses stub verilog modules for partitions in the top-level partition netlist. *Data_type*: bool, optional
`-topView` *topViewName*	Specifies view name for top design. *Data_type*: string, optional
`-verilog`	Writes out an ASCII Verilog file to the saved partition directory. The netlist is always stored as a binary file, so this Verilog file is not used by `restoreDesign`. It is sometimes useful to save the Verilog in addition to the other saved files to make it possible to send the Verilog directly to other tools, or to open it in an editor for debugging. Otherwise, you need to load the design into Innovus, and call `saveNetlist` to see the Verilog file. *Data_type*: bool, optional

Example

- The following command saves the partition information/files in a directory `try3` hierarchically for each of the three partitions:

  ```
  savePartition -dir try3 TOPCHIP_SP sheet17 sheet25 sheet7
  ```

- The following command saves the partition information/files in the OpenAccess database format.
  The information for the top and the block level designs (all blocks) will be written in the `libForOA` directory view with the name `ptnView1`.
  It saves partitions without generating a floorplan file or a net list for the partitioned design. It also does not write the standard cells or trialRoute wires in the DEF file.

  ```
  savePartition -ptnLib libForOA -ptnView ptnView1 -noFPlan -noNetlist -noStdCells -noTrialRoute
  ```

Related Information

- `partition`
- `deletePartition`
- `definePartition`
- `restoreDesign`
- `assembleDesign`
- `set_module_model`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Saving Partitions

savePtnPin

```
savePtnPin
[-help]
{-ptn partitionName | -all | -design | -selected}
fileName
```

Saves the pin assignment information for one or more partitions. You can subsequently load the saved pin assignment information with the `loadPtnPin` command. The `savePtnPin` command also saves the pin assignment information for blackboxes.

Parameters

`-all`	Specifies that pin assignment information will be saved for all partitions, including the top-level design. *Default*: By default, the `savePtnPin` command saves the pin assignment information for all partitions. *Data_type*: bool, optional
`-design`	Specifies that pin assignment information will be saved only for the top-level design. *Data_type*: bool, optional
`fileName`	Specifies the name of the file in which the partition pin assignment information will be saved. *Data_type*: string, required
`-help`	Outputs a brief description that includes the type and default information for each `savePtnPin` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man savePtnPin`
`-ptn` *`partitionName`*	Specifies the name of the partition for which the pin assignment information is to be saved. *Data_type*: string, optional
`-selected`	Specifies that the pin assignment information will be saved for all the selected pins of the partitions, including the top-level design. *Data_type*: bool, optional

Example

The following command saves the pin assignment information for the partition named `tdsp_block` into a file named `tdsp_block_pin_info`.

```
savePtnPin -ptn tdsp_block tdsp_block_pin_info
```

Related Information

- `assignPtnPin`
- `loadPtnPin`
- Partitioning the Design chapter in the *Innovus User Guide*
- Saving the Partition Pins

selectPtnPinGuide

```
selectPtnPinGuide
[-help]
x1 y1 x2 y2
pinGuideName
metalLayer
minSpace
[pinGroupCell]
```

Selects a partition pin guide and highlights it in the design display window.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `selectPtnPinGuide` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man selectPtnPinGuide`
`x1 y1 x2 y2`	Specifies the coordinates of the pin guide (x1 y1 x2 y2). *Data_type*: box, required
`metalLayer`	Specifies the layer on which the pin guide resides. *Data_type*: string, required
`minSpace`	Specifies the minimum pin spacing for the pin guide. *Data_type*: int, required
`pinGroupCell`	Specifies the cell name if the pin guide is associated with a cell pin group. *Data_type*: string, optional
`pinGuideName`	Specifies the name of the pin guide. *Data_type*: string, required

Related Information

- `createPinGuide`
- `deletePinGuide`
- Partitioning the Design chapter in the *Innovus User Guide*

set_ptn_fplan_mode

```
set_ptn_fplan_mode
[-help]
[-reset]
[-export {macro | special_route | pin_constraint}}]
[-import {macro | special_route | pin_constraint}}]
```

Sets what floorplan objects will be written-out and/or read back in. The `set_ptn_fplan_mode` command allows you to specify which objects should be written out during `savePartition -fplan` or read back in during `assembleDesign -fplan`.

Parameters

`-export {macro | special_route | pin_constraint}`	
	Specifies the objects to be exported. This is used by `savePartition -fplan` command to save the data to the fp file. Using the `-export` option, you can choose from saving `macro` and/or not saving `special_route` data. *Default:* `special_route` (default data + special route is always saved) *Data_type:* enums={macro special_route pin_constraint}_list
`-help`	Outputs a brief description that includes the type and default information for each `set_ptn_fplan_mode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_ptn_fplan_mode`
`-import {macro | special_route | pin_constraint}`	
	Specifies the objects to be imported. This is used by `assembleDesign -fplan` command to load the data from the fp file. Using the `-import` option, you can choose from bringing `macro` and/or `special_route` data. *Default:* By default, `-import` brings only defaults objects *Data_type:* enums={macro special_route pin_constraint}_list
`-reset`	Restores all parameters modified using `set_ptn_fplan_mode` to their default settings. The `-reset` parameter must be the first parameter specified. If you specify -reset by itself, the software resets all `set_ptn_fplan_mode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported.

Example

- The following command will save only macro information.
  ```
  set_ptn_fplan_mode -export {macro}
  ```
- The following command will reset the value to its default value.
  ```
  set_ptn_fplan_mode -export -reset
  ```

Related Information

- `assembleDesign`
- `get_ptn_fplan_mode`
- `savePartition`
- Partitioning the Design chapter in the *Innovus User Guide*

setClonePtnOrient

```
setClonePtnOrient
[-help]
name
{R0 R90 R180 R270 MX MX90 MY MY90}
```

Changes the orientation of the specified partition clone.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setClonePtnOrient` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setClonePtnOrient`.
name	Specifies the name of the partition clone whose orientation has to be changed. *Data_type*: (hinst), required
`{R0 R90 R180 R270 MX MX90 MY MY90}`	Specifies the orientation. The orientation can be one of the following: • `R0` • `MY` • `MX` • `R180` • `MX90` • `R90` • `R270` • `MY90` *Data_type*: enum, required

Example

The following command sets the orientation of the partition clone name `TDSP_ARB_C1 to R270:`

```
setClonePtnOrient TDSP_ARB_C1 R270
```

Related Information

- `getClonePtnOrient`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Changing Partition Clone Orientation

setPinAssignMode

```
setPinAssignMode
[-help]
[-reset]
[-advanced_node_rule_support {true | false}]
[-allow_unconnected_in_abutted_edge {true | false}]
[-allowNonNdrNetPinsOnNdrTracks {true | false}]
[-block_boundary_macro_distance distance]
[-force_abutment_with_fixed {true | false}]
[-insidePinSearchDistance {nrTracks}]
[-max_distance_pairing distance]
[-maxChannelWidthAsAbutted channelWidthInMicrons]
[-pinEditInBatch {true | false}]
[-pinOffStripe {below | all | none}]
[-pinToStripeDistance {listOfLayerDistanceValuePair}]
[-pinToViaDistanceNonPrefDirection {listOfLayerDistanceValuePair}]
[-pinToViaDistancePrefDirection {listOfLayerDistanceValuePair}]
[-promotedMacroMaxLayer layerName]
[-promotedMacroMinLayer layerName]
[-restrict_boundary_macro_distance distance ]
[-strict_abutment {true | false}]
```

Sets the global parameters of the partition or block pin assignment feature. Parameters specified with the `setPinAssignMode` command are then used whenever you run the pin related commands. Use the `getPinAssignMode` command to return the current settings for the `setPinAssignMode` command. You can use this command before assigning pins.

Note: You can use the `-topRoutingLayer` and `-bottomRoutingLayer` parameters of the `setDesignMode` to specify the maximum and minimum layers, respectively, for routing, partition, or block pin assignment.

Note: To control the pin assignment layers, you can use the `setPinConstraint` command. The `setPinConstraint` command sets the constraints for partition pins or I/O pins. The constraints can be applied on three different levels: global pin constraints, cell specific pin constraints, and pin specific constraints. If no pin specific constraints exist, then the cell level constraints are applied. If cell specific pin constraints are not defined, then global constraints are applied.

Parameters

`-advanced_node_rule_support {true | false}`	
	A `true` value specifies that 20nm (and below) as well as DPT LEF rules will be respected (if present in LEF technology file) for determining and resolving any DRC violations, during all pin assignment related commands. *Default*: `true` *Data_type*: bool
`-allow_unconnected_in_abutted_edge {true | false}`	
	A `true` value specifies that unconnected partition pins can be placed in abutted edges. *Default*: `true` *Data_type*: bool
`-allowNonNdrNetPinsOnNdrTracks {true | false}`	
	A `true` value allows general pins to be placed on NDR tracks (if available). Alternatively, if the value is set to false, the NDR-track rule is followed strictly and the net pins are placed on matching NDR tracks only. *Default*: `true` *Data_type*: bool
`-block_boundary_macro_distance distance`	

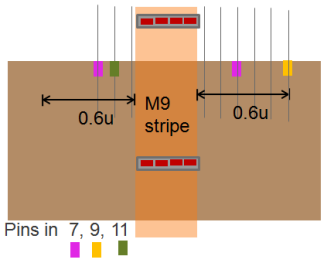
	Blocks automatic pin assignment from putting pins, on the part of the partition boundary facing the block, where the channel width between the partition boundary and the block is less than the value specified. For channel width below the specified value, it blocks pins on the boundary facing this channel. *Minimum*: 0 *Maximum*: 100 *Default*: 3 *Data_type*: float
`-help`	Outputs a brief description that includes the type and default information for each `setPinAssignMode` parameter. For a detailed description of the command and all of its parameters, use the man command: man setPinAssignMode
`-force_abutment_with_fixed {true|false}`	
	Specifies that for the connected pin of fixed abutted pins, ignore soft constraints check. Some partitions in abutted design have preassigned pins with 1-track spacing and marked as fixed. As the design is abutted, there is no other legal location for such pins. Also, the automatic pin assignment has default spacing of two tracks. As a result, all pins connecting to these reference pins with one spacing are not assigned and lot of pins are left unassigned. When the `-force_abutment_with_fixed` is enabled, track spacing violations are ignored. As a result, assigns the corresponding pins opposite to the prefixed pins while maintaining abutment of pins. *Default*: true *Data_type*: bool
`-insidePinSearchDistance {`*`nrTracks`*`}`	Controls the search distance for legalizing the internal pins. *Minimum*: 1 *Maximum*: 100 *Default*: 10 *Data_type*: int
`-max_distance_pairing` *`distance`*	Ignores moving corresponding connected pins of 2-pin-connection nets where distance (in microns) between the pins is equal or greater than distance specified. You can avoid moving the corresponding pin for any channel width (routing bends are acceptable in channels) by using the distance value for the `-max_distance_paring` parameter as 0. *Minimum*: 0 *Maximum*: 10000 *Default*: 50 *Data_type*: float
`-maxChannelWidthAsAbutted` *`channelWidthInMicrons`*	
	Specifies that all channels with width less than or equal to the specified width will be considered as abutted and abutment rules will be applied on them. **Notes:** • The `assignPtnPin`, `assignIoPins`, and the `legalizePin` commands do not place non-neighbor pins in the thin channels considered abutted. • The `checkPinAssignment` command reports an abutment violation in case a non-neighboring pin is placed in the thin channel (abutted). The `-preCheck` option of the `checkPinAssignment` command that checks for the feasibility of pin assignment, also treats these thin channels as abutted. *Minimum*: 0 *Maximum*: 400 *Default*: 3 *Data_type*: float

`-pinEditInBatch {true | false}`	Enables pin-editing in batch mode. **Note**: Once the interactive pin assignment is complete, it is important to set the mode to false. This can be done using the `setPinAssignMode -pinEditInBatch false` command. *Default*: `false` *Data_type*: bool
`-pinOffStripe {below | all | none}`	Prevents the generation of pins on metal layers with respect to the power and ground stripe. • `below`: Prevents the generation of pins on metal layers below the power and ground stripes. • `all`: Prevents the generation of pins on all metal layers, above and below the power and ground stripes. • `none`: Allows the generation of overlapping pins on all metal layers. *Default*: none *Data_type*: enums={below all none}
`-pinToStripeDistance {listOfLayerDistanceValuePair}`	
	Specifies the distance (in microns) from pin to stripe. It is layer specific. *Default*: "" *Data_type*: string
`-pinToViaDistanceNonPrefDirection {listOfLayerDistanceValuePair}`	
	Specifies the distance (in microns) from pin to via-stack. It is layer specific. *Default*: "" *Data_type*: string
`-pinToViaDistancePrefDirection {listOfLayerDistanceValuePair}`	
	Specifies the distance (in microns) from pin to via-stack from the partition fence boundary. It is layer specific. *Default*: "" *Data_type*: string
`-promotedMacroMaxLayer layerName`	Specifies the maximum metal layer name for promoting macro pins. *Default*: 31 *Data_type*: string
`-promotedMacroMinLayer layerName`	Specifies the minimum metal layer name for promoting macro pins. *Default*: 31 *Data_type*: string
`-reset`	Restores all parameters modified using `setPinAssignMode` to their default settings. The `-reset` parameter must be the first parameter specified. If you specify -reset by itself, the software resets all `setPinAssignMode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported.
`-restrict_boundary_macro_distance distance`	
	Blocks automatic pin assignment, from putting pins on the part of the partition boundary facing the block, in the places where the channel width between the partition boundary and the block is less than the specified value. For the channel width below the specified value, allowed layers and the channel length/width will be considered to allow a suitable number of pins on the boundary facing this channel. *Minimum*: 0 *Maximum*: 100 *Default*: 30 *Data_type*: float

`-strict_abutment {true | false}`	Controls abutted check for non-neighbour 2-pin and multi-partition pins. *Default*: true *Data_type*: bool

Examples

- The following command prevents the generation of pins below power and ground strips:
  ```
  setPinAssignMode -pinOffStripe below
  ```
- The following command resets all setPinAssignMode parameters to their default values:
  ```
  setPinAssignMode -reset
  ```
- The following command will leave .6u of space between Metal9 stripe and Metal9 pins, and leave 1u of space between Metal11 stripe and Metal11 signal pin
  ```
  setPinAssignMode -pinToStripeDistance {{9 .6} {11 1}}
  ```

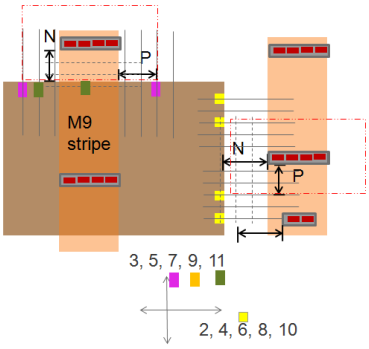


- For preferred direction pins, the following commands will:
  - Leave .1u of space between Metal2, Metal4, Metal6 stack via to pins in same layer, and leave .3u of space between Metal9 stacked via and corresponding signal pin, on left/right edges
  - Leave .1u of space between Metal3, Metal5, Metal7 stack via to pins in same layer, and leave .3u of space between M9 stacked via and corresponding signal pin, on top/bottom edges

  ```
  setPinAssignMode -pinToViaDistancePrefDirection {{2 .1} {3 .1} {4 .1} {5 .1} {6 .1} {7 .1} {8 .3} {9 .3}}
  ```
- For any pins in non preferred direction, the following commands will:
  - Leave .2u of space between Metal2, Metal4, Metal6 stack via to pins in same layer, and leave .4u of space between M9 stacked via and corresponding signal pin, on top/bottom edges
  - Leave .2u of space between Metal3, Metal5, Metal7 stack via to pins in same layer, and leave .4u of space between M9 stacked via and corresponding signal pin, on left/right edges

  ```
  -pinToViaDistanceNonPrefDirection {{2 .2} {3 .2} {4 .2} {5 .2} {6 .2} {7 .2} {8 .4} {9 .4}}
  ```

  Note: In the following picture, N stands for non-preferred and P stands for preferred.



Related Information

- `assignPtnPin`
- `assignIoPins`
- `checkPinAssignment`
- `createPinGuide`
- `getPinAssignMode`
- `legalizePin`
- `reportUnalignedNets`
- `setPromotedMacroPin`
- Partitioning the Design chapter in the *Innovus User Guide*

setPinConstraint

```
setPinConstraint
[-help]
{[-global | -cell cell_name | -matchEarlyGlobalRoute ]
 [-global | -pin pin_name_list| -area {x1 y1 x2 y2} | -side side_name | -matchEarlyGlobalRoute ]
 [-global | -layer {layer_id_or_name_list} | -loc {x y z}]
 [-global | -loc {x y z} | -edge edge_numbers | -corner corner_number]
 [[-pin pin_name_list [-loc {x y z}] [-edge edge_nmbers]] |
 [[-loc {x y z}] [-edge edge_numbers]] |
 [-corner_to_pin_distance pin_distance [-corner corner_number]]]
 [{-pin pin_name_list | -pinTemplate {layer_id_or_name_list}}
 [-use_min_width_as_depth | -depth depth] [-width width]]
 [[-layer {layer_id_or_name_list} [-side side_names]] |
[-pinTemplate {layer_id_or_name_list} [-use_min_width_as_depth ]]]
[-spacing spacing [-area {x1 y1 x2 y2}] [-side side_names]]}
[-depth depth [-special_wire ]]
```

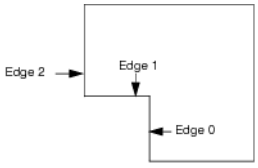
Sets the constraints for partition pins or an I/O pins. You can use this command before assigning pins. The constraints can be applied on three different levels:

- Global pin constraint
- Cell specific pin constraint
- Pin specific constraint

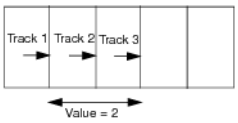
If no pin specific constraints exist, then the cell level constraints are applied. If cell specific pin constraints are not defined, then global constraints are applied.

Note: To retrieve the pin constraints specified using the `setPinConstraint` command, use the `getPinConstraint` command. To remove the constraints for a partition pin, use the `unsetPinConstraint` command.

Parameters

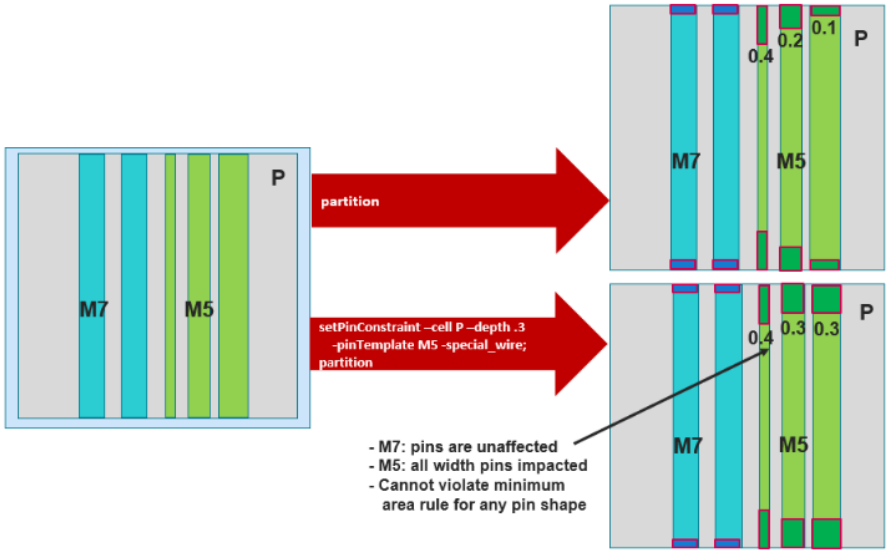
`-area {x1 y1 x2 y2}`	Specifies an area overlapping with partition where spacing constraint. *Data_type*: rect, optional
`-cell cell_name`	Specifies the name of the module. The module name is not required if the command is run on a top-level module. You can use wildcards (*?) with this parameter. *Data_type*: string, optional
`-corner corner_number`	Specifies the corner of the partition block. *Data_type*: int, optional
`-corner_to_pin_distance pin_distance`	
	Specifies the corner to pin distance in tracks. *Data_type*: int, optional
`-depth depth`	Specifies the pin depth in microns. *Data_type*: float, optional
`-edge edge_numbers`	Specifies the integer value where edge numbering starts from lower-left corner of a partition clock-wise. Edge `0` is the edge that has the smallest y value.  You can set multiple sides in the `-edge` option. This allows you to set multiple sides in a rectilinear design. *Data_type*: string, optional

`-global`	Specifies that the global constraints for spacing, width, and depth to be honored. **Note:** If no pin specific constraints exist, then the cell level constraints are applied. If cell specific pin constraints are not defined, then global constraints are applied. For example, the following command specifies that the global constraints are to be honored. `setPinConstraint -global -spacing 2 -width 2 -pinTemplate { 2 3 4 }` *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `setPinConstraint` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setPinConstraint`
`-layer {layer_id_or_name_list}`	
	Specifies the metal layers for the partition pin. If you specify multiple layers, separate the layer names with a space. In this case, the command will constrain the pin to one of the specified layers. **Notes:** • This parameter does not restrict the pin to a location on the layer. If you want to restrict a pin on a particular location on a particular layer, use the `-loc` parameter instead. • The `-layer` parameter cannot be used with the `-loc` parameter. • The `-layer` parameter cannot be used with the `-spacing` parameter. • Layers can be specified using the LEF layer names or layer ID numbers. *Data_type*: string, optional
`-loc {x y z}`	Specifies the x and y locations and z layer ID number of the partition pin. The x and locations are calculated with respect to the distance in microns from the lower left corner of the module. The layer number specified should be a preferred layer. Otherwise, the command will move the pin to a preferred layer. For example, `-loc {10 12 7}` specifies that the pin should be constrained to a location 10 microns on the x axis and 12 microns on the y axis from the lower left corner of the partition. on layer `Metal7`. **Notes:** • If the pin location as per the specified x and y locations is not on a track, the pin is snapped to the nearest track on the specified layer. • If you specify this parameter, the assigned pins have a *Placed* status. • The `-loc` parameter cannot be used with the `-layer` parameter. *Data_type*: string, optional
`-matchEarlyGlobalRoute`	When specified, it changes the pin constraints to make them compliant with `earlyGlobalRoute` and does the following for every partition: • Allows all layers from `setDesignMode -topRoutingLayer` to `-bottomRoutingLayer`. • Sets pin spacing as 1 **Note:** The changes in pin constraints impact the output of the pin assignment commands - `assignPtnPin`, `assignIoPins`, and `editPin`. *Data_type*: bool, optional
`-pin pin_name_list`	Specifies the name of the partition pin. You can use wildcards (`*?`) with this parameter. *Data_type*: string, optional
`-pinTemplate {layer_id_or_name_list}`	
	Specifies the metal layers for the width, depth, and spacing constraints at global level and cell level *Data_type*: string, optional
`-side side_name`	Specifies name of the side. Valid values are { N S E W } or { L B T R } or all. *Data_type*: string, optional

`-spacing spacing`	Specifies the minimum spacing between pins in pin pitch value, that is, the distance between tracks. For example, if you specify a value of `2`, the minimum pin spacing will be equal to the distance between track 1 and track 3.  *Data_type*: int, optional
`-special_wire`	Specifies the depth constraint to be set and honored for special wire pins. When `-special_wire` parameter is specified with the `-depth`, `-cell` and `-pinTemplate` parameters, it enables the depth on desired layers of specified cells to be honored for special wire pins by the `partition` command only. *Data_type*: bool, optional
`-use_min_width_as_depth`	Minimum width should be used as the pin depth. *Data_type*: bool, optional
`-width width`	Specifies the pin width in microns. *Data_type*: float, optional

Example

- The following command specifies a width of 10 micrometers for all partition pins in the partition `sheet14`. These pins will be placed either on layer 2 or on layer 4.
  ```
  setPinConstraint -cell sheet14 -pin * -layer {Metal2 Metal4}
  ```
- The following commands set the pin width and depth of layer `Metal2` for partition cell `ALU` to 0.4 and 0.6 respectively.
  ```
  setPinConstraint -cell ALU -layer Metal2 -width 0.4
  setPinConstraint -cell ALU -layer Metal2 -depth 0.6
  ```
- The following commands set the pin width of pin pGroup1 to 0.3 and pin depth of pin pGroup1 to default.
  ```
  setPinConstraint -cell ALU -pin pGroup1 -width 0.3
  setPinConstraint -cell ALU -pin pGroup1 -depth default
  ```
  With this example, all the pins of pGroup1 will have the width 0.3 and the default depth.
- The following commands set the minimum pin spacing for top and bottom edge of partition cell ALU to 1 track.
  ```
  setPinConstraint -cell ALU -edge T -spacing 1
  setPinConstraint -cell ALU -edge B -spacing 1
  ```
- The following command sets minimum pin spacing for all edges of partition cell TDSP to 3 tracks
  ```
  setPinConstraint -cell TDSP -all -spacing 3
  ```
- The following command sets the sides, T and B, of layer 3 and 5 of partition cell ptn4
  ```
  setPinConstraint -cell ptn4 -layer {3 5} -side {T B}
  ```
- The following command sets the edges, 2 and 0, of partition cell ptn4
  ```
  setPinConstraint -cell ptn4 -pin * -edge {2 0}
  ```
- The following commands sets the pin depth of layer `M5` for partition cell `P` to 0.3 using the `-special_wire` parameter so that it is honored only by the `partition` command.
  ```
  setPinConstraint -cell P -depth .3 -pinTemplate M5 -special_wire;
  partition
  ```



Related Information

- `getPinConstraint`
- `unsetPinConstraint`
- Setting Pin Constraints in the *Innovus User Guide*

setPromotedMacroPin

```
setPromotedMacroPin
[-help]
{-reset  | -reportOnlyFile fileName.tcl |
 {[-insts {instName | instNameList}]
  [-pins {pinName | pinNameList}]
  [-layers {layerId | layerIdList}]
  [-override ]
  [-abuttedToBoundaryOnly ]}}
```

Selects or marks signal and/or PG pin shapes to be promoted later on. You can use this command before assigning pins.

Note: The `setPromotedMacroPin` command should be called after specifying the maximum and minimum metal layer names for promoting macro pins using the `setPinAssignMode -promotedMacroMaxLayer maxLayer and -promotedMacroMinLayer minLayer` parameters, respectively.

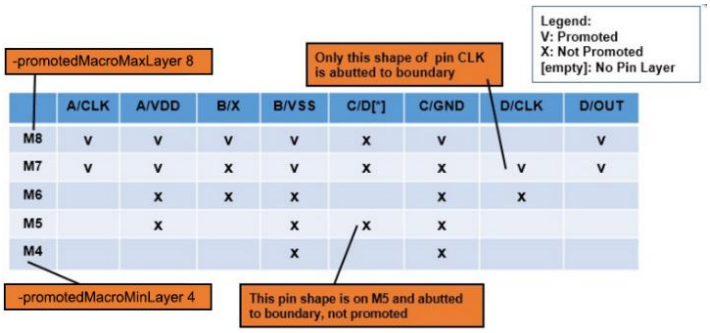
Parameters

`-abuttedToBoundaryOnly`	Specifies that all macro pin shapes that are abutted to the partition boundary will be selected for promotion. The `-pins` parameter can also be used to select abutted pins . **Note:** You can use the `-maxChannelWidthAsAbutted` parameter of the `setPinAssignMode` command to enable thin channels in the design to be considered as abutted partitions in the pin assignment flow. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `setPromotedMacroPin` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setPromotedMacroPin`
`-insts {instName | instNameList}`	
	Specifies the name of instances. You can also use wildcard (* or ?) for specifying the instance names. This parameter is optional. If `-insts` is not specified, all BLOCK, PAD, and AREAIO macros are selected. This is similar to specifying `-insts *` *Data_type*: string, optional
`-layers {layerId | layerIdList}`	
	Specifies the metal layer(s) of the macro pins. This parameter is optional. If `-layers` is not specified, all pin layers between `setPinAssignMode -promotedMinLayer` and `setPinAssignMode -promotedMaxLayer` range are selected. *Data_type*: string, optional
`-override`	Specifies to override specific option(s) where previous settings of these options will be removed and be replaced by new specified value(s). For example, specifying `setPromotedMacroPin -layers {} -override` means no layer is promoted from now on. **Note:** All `setPromotedMacroPin` commands are incremental with no conflict resolution, unless the `-override` option is used. *Data_type*: bool, optional
`-pins {pinName | pinNameList}`	Specifies the name of macro pins. You can also use wildcard (* or ?) for specifying the pin names. This parameter is optional. If `-insts` is not specified, all signal and P/G pins are selected. This is similar to specifying `-pins *` *Data_type*: string, optional
`-reportOnlyFile fileName.tcl`	Writes out all `setPromotedMacroPin` settings in memory into an output TCL file. *Data_type*: string, optional
`-reset`	Resets all or specified promoted pin selections. *Data_type*: bool, optional

Examples

- The following command promotes the top most layer pins of selected macros

```
setPinAssignMode -promotedMacroMinLayer 4 -promotedMacroMaxLayer 8
setPromotedMacroPin -reset
setPromotedMacroPin -insts {A B} -pins * -layers {7 8}
setPromotedMacroPin -insts B -pins X -layers 8 -override
setPromotedMacroPin -insts C -layers 5 -abuttedToBoundaryOnly
setPromotedMacroPin -insts C -layers {8}
setPromotedMacroPin -insts C -pins D[*] -layers {} -override
setPromotedMacroPin -insts D -pins OUT
setPromotedMacroPin -insts D -pins CLK -abuttedToBoundaryOnly
assignIoPins
```



	A/CLK	A/VDD	B/X	B/VSS	C/D[*]	C/GND	D/CLK	D/OUT
M8	v	v	v	v	x	v		v
M7	v	v	x	v	x	x	v	v
M6		x	x	x		x	x	
M5		x		x	x	x		
M4				x		x		

Related Information

- `setPinAssignMode`
- `assignIoPins`
- Partitioning the Design chapter in the *Innovus User Guide*

setPtnPinStatus

```
setPtnPinStatus
[-help]
[-selectedPinStatus {fixed | placed | cover}]
[-pin {pinName | pinNameList}]
[-status {fixed | placed | unplaced | cover}]
{[-pin {pinName | pinNameList} -status{fixed | placed | unplaced | cover} [-cell partitionName]] }
```

Sets the pin status for a pin of a partition as fixed, placed, cover, or unplaced. The `setPtnPinStatus` command operates, like a lot of other pin commands, on the top cell in addition to on partitions. If the top cell name is specified, or if `setPtnPinStatus` receives a "*" as its first argument, the IO pin status is modified. You can use this command any time after assigning pins.

Note:

- Use this command to unassign blackbox pins before the blackbox is committed. To unassign blackbox pins after the blackbox is committed, use the `flattenPartition` command instead.
- If this command is run on a blackbox that has one or more pins with a single LEF port but with multiple shapes within the LEF port, the command will remove these multiple shapes and create a single shape. If this command is run on a blackbox that has one or more pins with multiple LEF ports, the first LEF port will be processed as per the command and the remaining LEF ports will remain unchanged.

Parameters

`-cell` *partitionName*	Specifies the name of the partition. You can use wildcards (*?) with this parameter. *Data_type*: string, optional
`-help`	Outputs a brief description that includes type and default information for each `setPtnPinStatus` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setPtnPinStatus`.
`-pin` *pinName* | *pinNameList*	
	Specifies the name of the pin(s) of the partition. You can use wildcards (*?) with this parameter. *Data_type*: string, optional
`-selectedPinStatus {fixed | placed | cover}`	
	Selects pins with only specified status. This options enables the `setPtnPinStatus` command to change the status of selected pins based on their present status and not just the names. • `fixed`: Selects pins with fixed status. • `placed`: Selects pins with placed status. • `cover`: Selects pins with cover status. *Data_type*: enum, optional
`-status {fixed | placed | unplaced | cover}`	
	Specifies the status of the pin. • `Fixed`: Fixed pin status specifies the pin's location, orientation, and that it's location cannot be moved by automatic tools, but can be moved by interactive commands. **Note**: Fixed pins can be moved by automatic pin assignment commands by using the `-moveFixed` parameter. • `Placed`: Placed pin status specifies the pin's location, orientation, and that it's location is fixed, but can be moved during automatic layout. • `Unplaced`: Unplaced pin status specifies that the pin does not have a location. • `cover`: Cover pin status specifies the pin's location, orientation, and that it is a part of the cover macro. A cover pin cannot be moved by automatic tools or by interactive commands. *Data_type*: enum, optional

Example

The following command marks all partition pins to fixed:

```
setPtnPinStatus -cell * -pin * -status fixed
```

Related Information

- `getPtnPinStatus`
- `flattenPartition`
- Partitioning the Design chapter in the *Innovus User Guide*

setPtnPinUSE

```
setPtnPinUSE
[-help]
ptnName
pinName
pinUse
```

Allows you to change the `USE` property of a partition pin.

Note: A partition pin can also be an I/O pin if the partition is a top-level cell.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `setPtnPinUSE` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setPtnPinUse`
ptnName	Specifies the name of the partition, blackbox or top-level cell name. *Data_type*: string, required
pinName	Specifies the pin name. *Data_type*: string, required
pinUse	The choices represent `USE` properties, SIGNAL | CLOCK | ANALOG, that you can change with this command. *Data_type*: string, required

Example

- The following command sets the `USE` property of pin `A` in partition `PTN1` to `CLOCK`:

  ```
  setPtnPinUSE PTN1 A CLOCK
  ```

- The following command sets the USE property for pin scan_clk to CLOCK:

  Original DEF:

  ```
  - scan_clk + NET scan_clk + DIRECTION INPUT + USE SIGNAL ;
  ```

  Run `setPtnPinUse` to change USE to CLOCK:

  ```
  setPtnPinUSE [dbGet top.name] scan_clk CLOCK
  ```

  DEF after change:

  ```
  - scan_clk + NET scan_clk + DIRECTION INPUT + USE CLOCK ;
  ```

Related Information

- Partitioning the Design chapter in the *Innovus User Guide*

showPtnWireX

```
showPtnWireX
[-help]
[ptnName]
[-outfile fileName]
[-excludeNet exNetFileName]
[-excludeClock]
[-excludeTri]
[-excludeHighFan nrFan]
[-allNets]
[-delta]
```

Displays and writes a file of wires that physically cross over partitions. You can use this command after routing the design.

Note: The `showPtnWireX` command supports designs with nested partitions.

Parameters

`-allNets`	Reports nets that cross partitions that also connect to the ports of partitions. If this parameter is not specified, only nets that cross the partitions are reported. *Data_type*: bool, optional
`-delta`	Reports the total segment length of the wires crossing the block in the X and Y directions. *Data_type*: bool, optional
`-excludeClock`	Excludes clock nets from the wire crossing information. *Data_type*: bool, optional
`-excludeHighFan` *nrFan*	Excludes nets with fanouts over nrFan from the display. *Data_type*: int, optional
`-excludeNet` *exNetFileName*	Specifies the filename that contains nets to be excluded from the display. *Data_type*: string, optional
`-excludeTri`	Excludes all tristate nets from the display. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `showPtnWireX` parameter. For a detailed description of the command and all of its parameters, use the man command: `man showPtnWireX`
`-outfile` *fileName*	Specifies the name of the output file of wire crossing information. *Default*: If you do not specify this parameter, the detailed information is output to the ptnName.wireCrossing file, where ptnName is the name of the partition. For master clone designs, three output files are created. These files can be used by the `insertPtnFeedthrough` command for path based feedthrough insertions in master and clone designs. For example, when you specify, `ShowPtnWireX -outFile abc` The following files are generated: • abc.mc_connected_nets.txt: For all nets connected to master/clone partitions. • abc.mc_crossing_nets.txt: For all nets with wires over master/clone partitions (but not in the above file) and not intra partition nets. • abc.non_mc.txt: For all remaining nets which are not in above files and are not intra partition nets. **Note**: The default output file, abc.wireCrossing, is now only valid for non master/clone designs and has no significance for master and clone designs. **Note**: It is not recommended to use the `insertPtnFeedthrough -routeBased` option for the nets in abc.mc_connected_nets.txt and abc.mc_crossing_nets.txt. *Data_type*: string, optional
ptnName	Specifies the name of the partition to show wire crossing. If you choose this option with other options, it must be the first specified when running this command. *Default*: If you do not specify this parameter, it will show all partition wire crossings. *Data_type*: string, optional

Example

The following command displays the routes crossing over partition, sheet17, and outputs the detail information into file, sheet17.crossing:

```
showPtnWireX sheet17 -outfile sheet17.crossing
```

The wire crossing report section lists the nets, their wire lengths, in micrometers, and the shape of the wire in relation to the partition.
For example, the following report segment is for a partition module named `sheet17`:

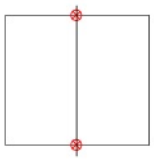
```
#########################################################
# Nets that cross partition module sheet17
# Box (335 335) (833 567)
```

```
# Format: Net <netName> <wireLength> <shape>
##########################################################

Net A 65 I
Net B 80 L
Net C 1050 T
Net D 132 X
...
```

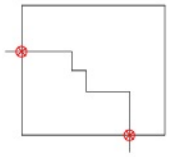
The first net in the report, A, has a wire length of 65 micrometers in an `I' shape, which indicates that the net crosses the partition on opposite sides, as follows:

```
Net A 65 I
```



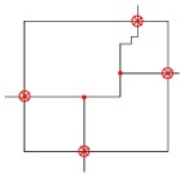
The second net in the report, B, has a wire length of 80 micrometers in an `L' shape, which indicates that the net crosses the partition on adjacent sides, as follows:

```
Net B 80 L
```



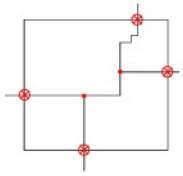
The third net in the report, C, has a wire length of 105 micrometers in an `T' shape, which indicates that the net crosses the partition on three sides, as follows:

```
Net C 105 T
```



The fourth net in the report, D, has a wire length of 132.30 micrometers in an `X' shape, which indicates that the net crosses the partition on all four sides, as follows:

```
Net D 132 X
```



Related Information

- `insertPtnFeedthrough`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Generating the Wire Crossing Report

specifyBlackBox

```
specifyBlackBox
[-help]
[-coreSpacing { left right top bottom }]
[-minPitchLeft integer ]
[-minPitchRight integer]
[-minPitchTop integer]
[-minPitchBottom integer]
[-pinLayerLeft { list_of_layers }]
[-pinLayerRight {l ist_of_layers }]
[-pinLayerTop { list_of_layers }]
[-pinLayerBottom { list_of_layers }]
[-reservedLayer { list_of_layers }]
[-placementHalo { left right top bottom }]
[-routingHalo float]
[-routingHaloTopLayer integer ]
[-routingHaloBottomLayer integer ]
[-noRebuildTiming]
[ -cell string| -masterInst string]
{-size { x y } | -area float | -gateCount string
   | {-gateArea float [-cellUtil float ]
[ -includeMacroArea {0 | 1}
{-macroArea float | -macroList { macroName [ count ]}}]}}
[{-area float | -gateCount string }
[-minWidth float | -minHeight float
| -fixedWidth float | -fixedHeight float ]]
[-minWidth float | -minHeight float
| -fixedWidth float | -fixedHeight float |-aspectRatioRange { min max }]
[-aspectRatio float |-aspectRatioRange { min max }]
```

Converts a hard macro into a blackbox. It is also used for specifying an instance to be the master of multiplying instantiated blackboxes. You can use this command after importing the design.

Note: You can convert a hard macro into a blackbox; however, Cadence does not recommend this because it will not update the blockage definitions when you change the blackbox size.

Note: For designs that have backside metal layers, both the backside and frontside layers are accepted. However, the automatic pin assignment flow is not impacted with the support for backside metal layers. It assigns pins on front layers only.

Parameters

`-area float`	Specifies the total area in square micrometers. If you specify the block size area, the default for the created block aspect ratio is 1. *Data_type*: float, optional
`-aspectRatio float`	Specifies the aspect ratio value. This value combined with the area information determines the width and height. *Data_type*: float, optional
`-aspectRatioRange {min max}`	Sets the minimum and maximum values for the aspect ratio range. This value combined with the specified related area information enables the blackbox to be reshaped while maintaining the current area. *Data_type*: point, optional
`-cell string`	Specifies the name of the cell. *Data_type*: string, optional
`-cellUtil float`	Specifies the cell utilization. This value is used to derive the area of the blackbox, which is calculated using the following formula: `Blackbox area = (standard cell area / cell utilization) + macro area` *Default*: The default value of cell utilization is `1` (100%). *Data_type*: float, optional

`-coreSpacing {left right top bottom}`	Specifies the space, in micrometers, between the module boundary and core design area of the module on the left, right, top, and bottom sides respectively. The partition pins will be located at the partition instance boundary and the core area. *Data_type*: string, optional
`-fixedHeight float`	Specifies the fixed height of the module. *Data_type*: float, optional
`-fixedWidth float`	Specifies the fixed width of the module. *Data_type*: float, optional
`-gateArea float`	Specifies the gate area. If the value of the `-includeMacroArea` parameter is `0`, the gate area is equal to the standard cell area. If the value of the `-includeMacroArea` parameter is `1`, the gate area also includes the macro area. *Data_type*: float, optional
`-gateCount string`	Specifies a gate count value. The total cell area is equal to the specified gate count multiplied by the specified area per gate. • *`count`:* Specifies the gate count value. • *`areaPerGate`:* Specifies an area per gate value in square micrometers. To get the cell area, this value is multiplied by the specified gate count. • *`utilization Value`:* Specifies the cell utilization. This value derives the area that is reserved for routing. The final block size is equal to the total cell area plus the routing area. *Default*: The default value is 0.7 (70%) *Data_type*: string, optional
`-help`	Outputs a brief description that includes type and default information for each `specifyBlackBox` parameter. For a detailed description of the command and all of its parameters, use the man command: `man specifyBlackBox`
`-includeMacroArea {0 | 1}`	A value of `0` specifies that the gate area does not include the macro area specified through the `-macroArea` or the `-macroList` parameter. A value of `1` specifies that the gate area includes the macro area specified through the `-macroArea` or the `-macroList` parameters. *Default:* The default value is `0`. *Data_type*: enum, optional

`-macroArea float`	Specifies the total area of the macro(s) in square microns.
	• If the value of the `-includeMacroArea` parameter is set to 0:
	`Total blackbox area = (gate area / cell utilization value) + macro area`
	• If the value of the `-includeMacroArea` parameter is set to 1:
	`Total blackbox area = ((gate area - macro area) / cell utilization value) + macro area`
	Data_type: float, optional

`-macroList {macroName [count ]}`	Specifies the name of the hard macro whose area will be included while calculating the total gate area. The `macroName` argument specifies the name of the macro, and the `count` argument specifies the number of times the macro is referenced. Thus, by specifying a value for count, you avoid repeating the macro names for macros that are referenced multiple times.
	• If the value of the `-includeMacroArea` parameter is set to 0:
	`Total blackbox area = (gate area / cell utilization value) + macro area`
	• If the value of the `-includeMacroArea` parameter is set to 1:
	`Total blackbox area = ((gate area - macro area) / cell utilization value) + macro area`
	Default: The default value of *`count`* is `1`.
	Data_type: string, optional

`-masterInst string`	Specifies the name of the master blackbox. *Data_type*: string, optional
`-minHeight float`	Specifies the minimum height of the specified module. *Data_type*: float, optional

`-minPitchLeft` *integer* `-minPitchRight` *integer* `-minPitchTop` *integer* `-minPitchBottom` *integer*	Specifies the pin pitch dimension for the *Left*, *Right*, *Top*, and *Bottom* blackbox sides. The default is 2, which places one pin for every two metal tracks. The actual pin pitch depends on the pin's metal layer. *Data_type*: int, optional
`-minWidth` *float*	Specifies the minimum height of the specified module. *Data_type*: float, optional
`-pinLayerLeft {`*list_of_layers*`}` `-pinLayerRight {`*list_of_layers*`} -pinLayerTop {`*list_of_layers*`} -pinLayerBottom {`*list_of_layers*`}`	
	Specifies the metal layers to use for the blackbox. The metal layers correspond to the sides of the blackbox, where the vertical metal layers (for example, *Metal2, Metal4*, and so on) are for the Top and Bottom sides, and horizontal metal layers (for example, `Metal3, Metal5`, and so on) are for the Left and Right sides of the blackbox. Specify this value in integers, for example `-pinLayerTop {2 4}`. *Data_type*: string, optional
`-placementHalo {left right top bottom}`	Specifies extra spacing, in micrometers around the sides (left, bottom, right and top respectively) of the blackbox that should not be used for placement. At the top-level design, this information is saved as part of the partition section in a floorplan file. This information is also saved in a partition floorplan file when saving partitions. By default, the value is 0 for all the sides. *Data_type*: string, optional
`-reservedLayer {`*list_of_layers*`}`	Specifies the metal layers that are used for routing in the partition and generating partition pins. Any metal layers that are not specified, usually the top-most metal layers, are allowed to route over the partition. A normal six-metal layer specification is Metal1, Metal2, Metal3, Metal4 and Metal5 specified, and Metal6 unspecified. When saving the partition, the LEF generated for this partition will have routing blockages on their layers so that the top-level router is aware of which metal layers are being used in the partition. *Data_type*: string, optional
`-routingHalo` *float*	Specifies routing spacing, in micrometers, around the sides of the blackbox. You can specify a positive or a negative value for the routing halo. A positive value means that the halo will be outside the blackbox. A negative value means that the halo will be applicable during block implementation, inside of the blackbox. The same value is used for all sides of the blackbox. *Data_type*: float, optional
`-routingHaloBottomLayer` *integer*	Specifies the bottom partition layer for which routing halo will be created. *Default*: By default, the bottom layer reserved for the partition is used. *Data_type*: int, optional
`-routingHaloTopLayer` *integer*	Specifies the top partition layer for which routing halo will be created. *Default*: By default, the top layer reserved for the partition is used. *Data_type*: int, optional
`-size x y`	Specifies fixed width and height of a specified module, in micrometers. *Data_type*: point, optional
`-noRebuildTiming`	Skips re-building the time graph and re-initializing the top cell information in order to improve the run time of the blackbox specification for designs with a large number of blackboxes. **Note:** Since the timing graph only needs to build for the last blackbox, you must specify the final `specifyBlackBox` command without this parameter. For example, if your design has three blackboxes, the first two `specifyBlackBox` commands should be specified with the `-noRebuildTiming` parameter while the last `specifyBlackBox` command should be specified without this parameter. *Data_type*: bool, optional

Examples

- The following command converts the hard macro `hn246` into a black box. The gate area is specified as 100000 square microns and the specified macro area is 220 square microns. The cell utilization value is not specified, and so the default value of 1 is used. The `-includeMacroArea` parameter is not specified and so the default value of 0 is used.
  When the `-includeMacroArea` parameter is 0, the blackbox area is calculated using the formula:
  ```
  Total blackbox area = (gate area / cell utilization value) + macro area
  ```
  The total black box area in this case is, therefore, (100000 / 1) + 220 that is 100220 square microns.
  ```
  specifyBlackBox -cell hn246 -gateArea 100000 -macroArea 220
  ```
- The following command is similar to the command in the previous example, except that the value of `-includeMacroArea parameter` is set to 1 and the value of the `-cellUtil` parameter is set to `.7`
  When the `-includeMacroArea` parameter is set to 1, the blackbox area is calculated using the formula
  ```
  Total blackbox area = ((gate area - macro area)/cell utilization value) + macro area
  ```
  The total blackbox area in this case is, therefore, ((100000 -220) /.7 + 220) that is 142762.86 square microns.
  ```
  specifyBlackBox -cell hn246 -gateArea 100000 -includeMacroArea 1 -macroArea 220 -cellUtil 0.7
  ```
- The following command converts the macro `hn246` into a blackbox and specifies that the areas of macros `macro_2A` and `macro_2B` should be added while calculating the blackbox area. The macro named `macro_2A` is included four times and the macro named `macro_2B` is included three times.
  ```
  specifyBlackBox -cell hn246 -gateArea 100000 -macroList macro_2A 4 macro_2B 3
  ```

Related Information

- `getBlackBoxArea`
- `unspecifyBlackBox`
- `saveBlackBox`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Defining Blackboxes

specifyPartition

```
specifyPartition
[-help]
partitionSpecFileName
[-noEqualizePtnHInst]
```

Loads the partition specification file. The partition specification file is created from the *Specify Partition* form. You can use this command after importing the design.

Note: You can create partitions inside power domains as well.

Note: Command `specifyPartition` is now obsolete and will be removed in the next major release of the software. This command has been replaced by the `definePartition` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `specifyPartition` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man specifyPartition`
`-noEqualizePtnHInst`	Disables the snapping capability of the clone partitions to the power grid, such that clones do not necessarily have the same row structure and pattern as their master. *Data_type*: bool, optional
partitionSpecFileName	Specifies the file containing the partition specification. *Data_type*: string, required

Example

The following command loads the partition specification file `app1.ptn` and updates the Floorplan view:

```
specifyPartition app1_p.ptn
```

Related Information

- `definePartition`
- `partition`
- `deletePartition`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Defining Partitions

undo_uniquify_partition

```
undo_uniquify_partition
[-help]
-hinst hierarchical_instance_name
[-preserve_routes]
```

Brings back the recursion of cells in a netlist and restores the master and clone association.

Note: The `uniquify_partition` command is used to remove the master and clone association by uniquifying the netlist.

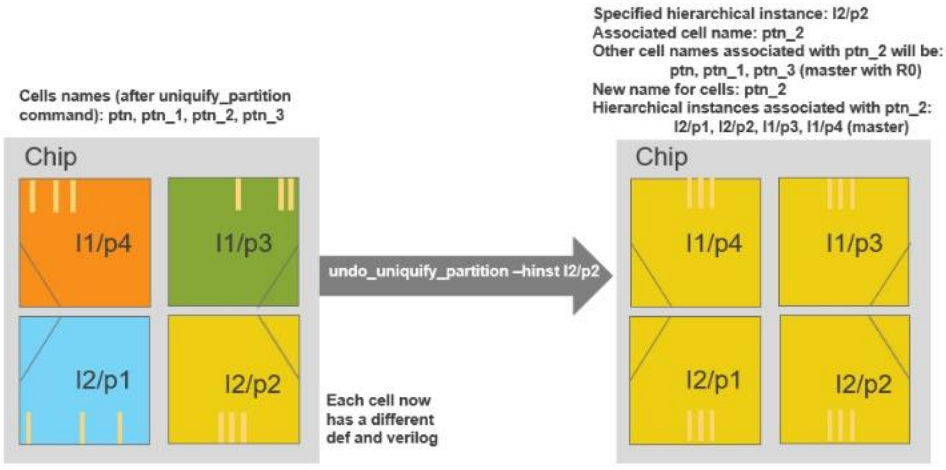
Parameters

`-help`	Outputs a brief description that includes the type and default information for each `undo_uniquify_partition` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man undo_uniquify_partition`
`-hinst hierarchical_instance_name`	Specifies name of the hierarchical instance that is used to bring back the recursion of cells in a netlist and restore the master and clone association. Based of the selected hierarchical instance, its cell (module) is identified. All other occurrences of the identified cell with the same naming format (`cellname_<number>`) are then renamed to the identified cell name. That means all instances will have the same cell name and definition again, thereby restoring the master and clone relationship. **Note:** • You can specify any hierarchical instance with the `undo_uniquify_partition` command. • Any hierarchical instance with R0 orientation can become a master. • The name of the cells will change to the name of the cell of the specified hierarchical instance. • The netlist and def of the cell of the hierarchical instance name specified with the `undo_uniquify_partition` command will be copied in all hierarchical instances. *Data_type*: string, optional
`-preserve_routes`	Preserves the routes during `undo_uniquify_partition`. *Data_type*: bool, optional

Example

The following command brings back the master and clone association for the cell name associated with `I2/p2`:

```
undo_uniquify_partition -hinst I2/p2
```



Related Information

- uniquify_partition
- Partitioning the Design chapter in the *Innovus User Guide*

uniquify_partition

```
uniquify_partition
[-help]
-hinst hierarchical_instance_name
```

Removes the master and clone association by uniquifying the cells (module) in the netlist.

Note: You can use the `undo_uniquify_partition` command to remove the uniquification and bring back the master and clone association.

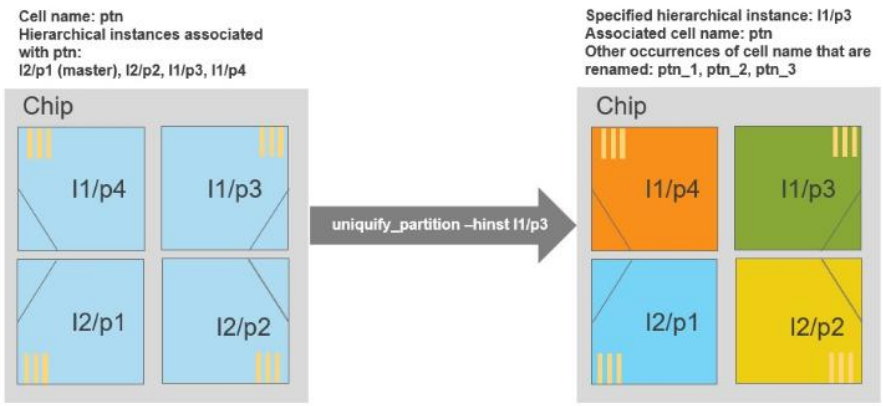
Parameters

`-help`	Outputs a brief description that includes the type and default information for each `uniquify_partition` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man uniquify_partition`
`-hinst hierarchical_instance_name`	Specifies the name of the hierarchical instance to uniquify. Based of the specified hierarchical instance, its cell (module) is identified. All other occurrences of the identified cell are renamed by adding the `_<number>` suffix (for example `cellname_<number>`) in order to uniquify the cell. **Note:** • You can specify any hierarchical instance with the `uniquify_partition` command. It does not necessarily have to be a master. • The `uniquify_partition` command preserves the orientation of the hierarchical instances. • The `uniquify_partition` command preserves the pin location of the hierarchical instances. • A new cell name is created for each hierarchical instance. *Data_type*: string, required

Example

The following command identifies the cell associated with `I1/p3` and then renames all other occurrences of the cell in order to remove the master and clone association.

```
uniquify_partition -hinst I1/p3
```



Related Information

- `undo_uniquify_partition`
- Partitioning the Design chapter in the *Innovus User Guide*

unloadPtnPin

```
unloadPtnPin
[-help]
-ptnName partitionName
{-file floorplanFileName }
```

Removes the loading of partition pins by the `loadPtnPin` command. This is used to remove the preassignment of pins that were previously reassigned by the loading.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `unloadPtnPin` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man unloadPtnPin`
`-file floorplanFileName`	Specifies the full pathname of the partition floorplan file. *Data_type*: string, optional
`-ptnName partitionName`	Specifies the name of the partition. *Data_type*: string, required

Example

The following command unloads pin positions for partition `EUG` from file `EUG.fp.ptn`:

```
unloadPtnPin -ptnName EUG -file EUG.fp.ptn
```

Related Information

- `loadPtnPin`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Assigning Partition and Blackbox Pins
  - ECO Pin Assignment

unsetPinConstraint

```
unsetPinConstraint
[-help]
{[-global | -cell cell_name | -all_user ]
[-global | -pin pin_name_list | -area {x1 y1 x2 y2} | -all_area | -side side_names]
[-global | -layer ] [-global | -loc | -edge | -corner corner_number]
[[-pin pin_name_list [-loc ] [-edge ]] |
[-corner_to_pin_distance [-corner corner_number]]]
[{-pin pin_name_list | -pinTemplate {layer_id_or_name_list}} -width ]
[{-pin pin_name_list | -pinTemplate {layer_id_or_name_list}} -depth ]
[[-layer [-side side_names]] | -pinTemplate {layer_id_or_name_list}]
[-spacing [-area {x1 y1 x2 y2}]
[-all_area ] [-side side_names]] }
[-depth [-special_wire ]] [-all_user ]
```

Removes the constraints for a partition pin. You can use this command after setting the pin's constraints with the `setPinConstraint` command.

Parameters

`-all_area`	Specifies that spacing constraints on all areas should be removed. *Data_type*: bool, optional
`-all_user`	Specifies that all user specified constraints should be removed from all levels (`-global`, `-cell`, and `-pin` level). *Data_type*: bool, optional
`-area {x1 y1 x2 y2}`	Specifies that the area overlapping with partition for which the spacing constraints have been specified should be removed. *Data_type*: box, optional
`-cell cell_name`	Specifies the name of the cell (partition). The cell name is not required if the command is run on a top-level cell. *Data_type*: string, optional
`-corner corner_number`	Specifies the corner of the partition block. *Data_type*: int, optional
`-corner_to_pin_distance`	
	Specifies that the corner to pin distance constraints should be removed. *Data_type*: bool, optional
`-depth`	Specifies the pin depth related constraints should be removed. *Data_type*: bool, optional
`-edge`	Specifies that edge-related constraints should be removed. *Data_type*: bool, optional
`-global`	Specifies to reset global constraints. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `unsetPinConstraint` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man unsetPinConstraint`
`-layer`	Specifies that the layer-related constraints should be removed. *Data_type*: bool, optional
`-loc`	Specifies that the location-related constraints should be removed. *Data_type*: bool, optional
`-pin pin_name_list`	Specifies the name of the partition pin. You can use wildcards (`*?`) with this parameter. *Data_type*: string, optional
`-side side_name`	Specifies name of the side. Valid values are { N S E W } or { L B T R } or all. *Data_type*: string, optional
`-spacing`	Specifies that the spacing-related constraints should be removed. *Data_type*: bool, optional
`-special_wire`	Specifies that the depth constraint that is honored for special wire pins by the `partition` command should be removed. *Data_type*: bool, optional
`-pinTemplate { layer_id_or_name_list }`	
	Specifies the metal layers for the width and depth constraints at global level and cell level. *Data_type*: string, optional
`-width width`	Specifies that the pin width related constraints should be removed. *Data_type*: bool, optional

Example

- The following command removes the spacing- and layer-related constraints for all partition pins in the partition `sheet14`.
  ```
  unsetPinConstraint sheet14 -pin * -layer -spacing
  ```
- The following command removes all constraints on globals.
  ```
  unsetPinConstraint -global
  ```
- The following command removes all constraints on all cells.
  ```
  unsetPinConstraint -cell *
  ```
- The following command removes all constraints on all pins.
  ```
  unsetPinConstraint -cell * -pin *
  ```
- The following command removes all constraints on cells, pins, and globals.
  ```
  unsetPinConstraint -all_user
  ```

Related Information

- `setPinConstraint`
- `getPinConstraint`
- Partitioning the Design chapter in the *Innovus User Guide*
  - Setting Pin Constraints

unspecifyBlackBox

```
unspecifyBlackBox
[-help]
{-cell cellName | -all}
[-keepPtn]
```

Unspecifies or deletes a blackbox and converts it to a module if the blackbox was originally defined as a module, or a block if the blackbox has a cdump or LEF macro definition. You can use this command after importing the design.

Note: You cannot delete a blackbox of an undefined module or cell.

Parameters

`-all`	Selects all blackboxes for deletion. *Data_type*: bool, optional
`-cell cellname`	Selects an individual blackbox for deletion. You can use wildcards (`*?`) with this parameter. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `unspecifyBlackBox` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man unspecifyBlackBox`
`-keepPtn`	Preserves the partition information. You should only use this option if the blackbox is placed inside the core area and is defined as a module in the netlist. *Data_type*: bool, optional

Example

- The following command deletes all blackboxes that begin with the letter A-some of which are inside the core area, and some which are outside the core area and converts them to either a fence or `module`.

  ```
  unspecifyBlackBox -cell A*
  ```

Related Information

- `specifyBlackBox`
- Partitioning the Design chapter in the *Innovus User Guide*

write_generalized_feedthru_paths

```
write_generalized_feedthru_paths
[-help]
[-dont_generalize_reverse_paths]
[-dont_generalize_sub_paths]
[-topological_out_file file_name]
-use_topological_file file_name
```

Automatically derives feedthrough topological paths from the user specified paths to cover all the chain connectivity probabilities in a master and clone hinst scenario. It reads an input topological file, in version 0.5 and creates the version 2.0 topological file.

Note: For the given set of master-clone hinst connectivity, based on floorplan, multiple user defined paths may be required for other same set hinst master and clones. The `write_generalized_feedthru_paths` command can automatically choose correct path for individual master and clone hinst, based on matching connectivity and floorplan, from the set of paths provided by the user.

Parameters

`-dont_generalize_reverse_paths`	
	When enabled, the generalization algorithm does not generalize reverse chain paths. *Data_type*: bool, optional
`-dont_generalize_sub_paths`	
	When enabled, the generalization algorithm does not generalize sub-paths. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `write_generalized_feedthru_paths` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_generalized_feedthru_paths`
`-topological_out_file file_name`	
	Specifies the name of the output topo 2.0 file that contains all paths for all master and clone chain. **Note:** This file should be used as an input file for the `insertPtnFeedthrough` command. *Default:* path.topo *Data_type*: string, optional
`-use_topological_file file_name`	
	Specifies the user specified feedthrough path guidance file. The input topological file with new version 0.5 format contains a path for one master and clone chain to derive path of all master and clone chains. **Note:** You can use the *Path Guidance GUI* form (*Partition - Feedthough Ports - Path Guidance)* to generate the topological file with new version 0.5 format. *Data_type*: string, required

Example

- The following command reads the topology file “a.topo” in version 0.5 and creates the version 2.0 topological file “path.topo”:

  ```
  write_generalized_feedthru_paths  -use_topological_file a.topo
  ```

- The following command reads the topological file “a.topo” in version 0.5 and creates the version 2.0 topological file “new.topo”:

  ```
  write_generalized_feedthru_paths  -use_topological_file a.topo -topological_out_file new.topo
  ```

Related Information

- `insertPtnFeedthrough`
- Partitioning the Design chapter in the *Innovus User Guide*

changeCurrentDesign

```
changeCurrentDesign
[-help]
[ptnName]
```

Changes the partition view. It helps you to switch between different block cells and get back to the top cell.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `changeCurrentDesign` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man changeCurrentDesign`
ptnName	Specifies name of the partition to set as top cell. **Note:** If no partition name is specified, then you are taken to the top cell of the current design. *Data_type*: string, optional

Examples

- The following commands changes sets `tdsp_top` as the top cell.
  ```
  changeCurrentDesign tdsp_top
  ```

Related Information

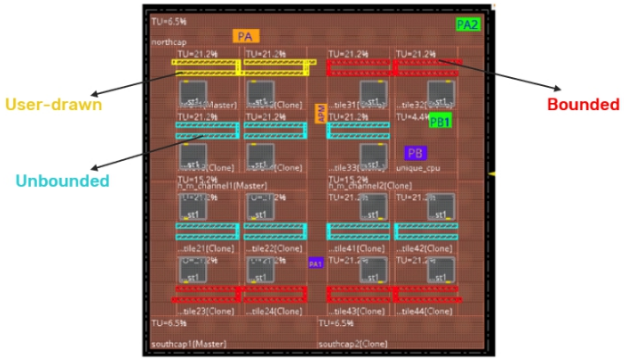
- Change Partition View chapter in the *Innovus Menu Reference*
- Partitioning the Design chapter in the *Innovus User Guide*

highlightSyncObj

```
highlightSyncObj
[-help]
[-obj {all busGuides}]
```

Adds highlight color on sync objects during master-clone-aware bus guide operations. The colors used for highlighting are based on different color schema categories – user-drawn, bounded, or unbounded. These categories are derived from the last sync command (`clonePtnObj -sync` or `setFPlanMode -autoSyncMasterClone busGuides`) used. This command makes it easy to distinguish between different bus guide objects in the design.

- User-drawn bus guides are highlighted in yellow.
- Bounded bus guides are highlighted in red. Nets in bounded bus guides can be determined during sync operation. If either (or both) the source or driver partition is a master-clone, the tool can determine the equivalent net during sync (in master clone chains).
  For example: Consider A1-B1-C1 and A2-B2-C2 chains where A/B/C1 A/B/C2 are master clones. A1/2 pin k is connected to net n1 (in A1) and n2 (in A2). As we can easily determine n1 and n2 in the chains, this is a case of bounded bus guides.
- Unbounded bus guides are highlighted in blue. Nets in unbounded bus guides cannot be determined during sync operation. If both source or driver partition are unique or (special case master and clone but their equivalent pin is floating), the tool cannot determine the net during sync (in master and clone chains).
  For example: Consider UA1-B1-UC1 and UA2-B2-UC2 chains where B1 and B2 are master clones. UA1/UC1/UA2/UC2 are all unique but UA1/UA2 pin may be different. Here since we cannot determine equivalent nets of chains, this is a case of unbounded bus guides.



Parameters

`-help`	Outputs a brief description that includes the type and default information for each `highlightSyncObj` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man highlightSyncObj`
`-obj {all busGuides}`	Specifies the object to be highlighted. *Data_type*: enum_list, optional

Example

- The following command highlights busguides based on different color schema categories:
  `highlightSyncObj -obj busGuides`

Related Information

- Partitioning the Design chapter in the *Innovus User Guide*

createBusNetGroup

```
createBusNetGroup
[-help]
[-considerClone]
[-createBusSinkGroup]
[-dontCommit]
[-file string]
[-ignoreBusSize integer]
[-ignoreConnectivity]
[-ignoreMultiPartitionPinNet]
[-netGroupOptionsList string]
[-overwrite]
[-prefix string]
[-reverseOrder]
[-useShortName]
[-bus string | -all ]
[-forTopOnly | -forPartitionOnly ]
[-includeMacro | -forPartitionOnly ]
[-checkOnly | {[-bus string] [-all ]} ]
```

Creates a net group for buses.

Parameters

`-all`	Creates a net group connected to the design partition for all the buses. **Note:** This parameter is mutually exclusive with the `-bus` parameter. *Data_type*: bool, optional
`-bus string`	Creates a net group connected to different partitions of the specified bus only. **Note:** This parameter is mutually exclusive with the `-all` parameter. *Data_type*: string, optional
`-checkOnly`	Checks and reports all the buses present in the design. *Data_type*: bool, optional
`-considerClone`	Creates a net group for buses connected to clones. *Data_type*: bool, optional
`-createBusSinkGroup`	Creates BusSinkGroup instead of a net group. *Data_type*: bool, optional
`-dontCommit`	Creates nets groups and stores the info in a file instead of creating in the DB. *Data_type*: bool, optional
`-file string`	Specifies the name of the file in which the defined groups are stored. *Data_type*: string, optional
`-forPartitionOnly`	Specifies that the net groups should be connected to partitions only. *Data_type*: bool, optional
`-forTopOnly`	Specifies that the net groups should be connected to the Top only. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `createBusNetGroup` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man createBusNetGroup`
`-ignoreBusSize integer`	Specifies that the buses smaller than the specified size should be ignored. *Data_type*: int, optional

`-ignoreConnectivity`	Specifies that the partition connectivity should be ignored and the net groups should be created for the whole bus. *Data_type*: bool, optional
`-ignoreMultiPartitionPinNet`	Specifies that the buses connected to multiple partitions/clones should be ignored. *Data_type*: bool, optional
`-includeMacro`	Specifies that bus net group being created should have a connection with macros. *Data_type*: bool, optional
`-netGroupOptionsList` *string*	Specifies attributes (like optimizeOrder, spread etc.) that can be added to the bus net group. *Data_type*: string, optional
`-overwrite`	Overwrites the already created net group be deleting the current net group element from an earlier association. *Data_type*: bool, optional
`-prefix` *string*	Specifies a prefix for the bus net group. *Data_type*: string, optional
`-reverseOrder`	When specified, the bus order of the buses in the net group is reversed. *Data_type*: bool, optional
`-useShortName`	When specified, a short name is applied to the bus net group. *Data_type*: bool, optional

Related Information

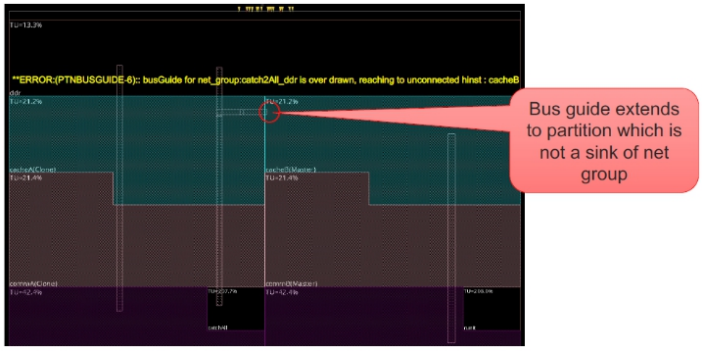
- Partitioning the Design chapter in the *Innovus User Guide*

check_bus_guides

```
check_bus_guides
[-help]
[-net_groups list_of_net_groups]
[-outFile file_name]
```

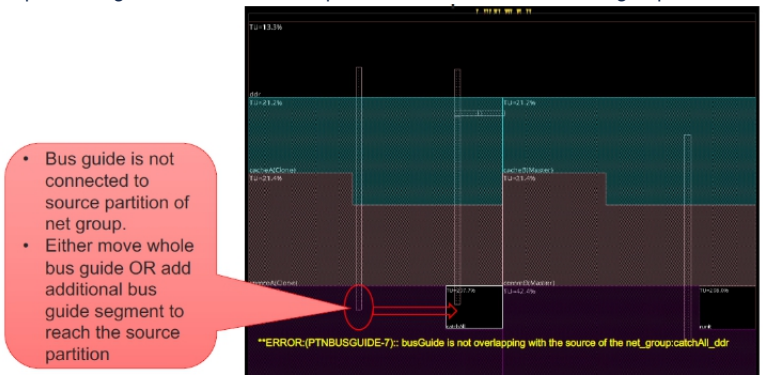
Checks bus guides for violations during master-clone-aware net group implementation flow. It checks for the consistency of net groups and bus guides by performing the following:

- Bus-guide physical checks:
  - Reports over-drawn bus guides.



  - Reports under-drawn bus guides (bus guide does not reach the sink of net group).

  - Reports bus guide that do not overlap/connect with the source of net group.



- Checks the net-group consistency and reports non-homogeneous nets in a net group.

Note: The `check_bus_guides` command does not support nested designs yet.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `check_bus_guides` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man check_bus_guides`
`-net_groups` *`list_of_net_groups`*	Specifies the net groups whose bus guides are to be checked. *Default* : *. Example: # -net_groups {ng1 ng2 ng3-*} (string, optional) *Data_type*: string, optional
`-outFile` *`file_name`*	Specifies the name of the file where the bus guide violation log is stored. *Default:* `checkBg.rpt` *Data_type*: string, optional

Example

- The following command checks the bus guides for violations.
  ```
  check_bus_guides
  ```
- The following command checks the bus guides of net groups ng1, ng2, and ng3 for violations.
  ```
  check_bus_guides -net_groups {ng1 ng2 ng3}
  ```

Related Information

- Partitioning the Design chapter in the *Innovus User Guide*

write_net_groups_topo

```
write_net_groups_topo
[-help]
[-net_groups list_of_net_groups]
[-outFile file_name]
```

Creates a topological file (used by the `insertPtnFeedthrough` command) for nets of the net group associated with a bus guide. The topology of nets for feedthrough is derived from the shape of the bus guide. All the partitions that the bus guide shape overlaps with, become a node in the topological path.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `write_net_groups_topo` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_net_groups_topo`
`-net_groups` *list_of_net_groups*	Specifies the net groups to be added in topo file. *Default: *.* For example, the following command specifies that the ng1, ng2, and ng3 netgroups must be added to the topo file. `write_net_groups_topo -net_groups {ng1 ng2 ng3-*}` *Data_type*: string, optional
`-outFile` *file_name*	Writes the bus guide violation log to the specified file. *Default:* `bus_guided.topo.v2` *Data_type*: string, optional

Example

- The following command writes the net group information to a topology file.
  ```
  write_net_groups_topo
  ```

Related Information

- Partitioning the Design chapter in the *Innovus User Guide*

24

Placement Commands and Global Variables

- Placement Commands
- Placement Global Variables

Placement Commands

- add_decomp_filler
- add_gate_array_filler
- add_tap_walls
- addEndCap
- addFiller
- addFillerGap
- addMimCap
- addSpareInstance
- addWellTap
- change_IB_cell_site_name
- check_base_layer_density
- check_library
- checkFiller
- checkPlace
- clearScanDisplay
- clearSpareCellDisplay
- clonePlace
- congRepair
- create_inst_space_group
- createGAFillerGroup
- createPipelineBusGuide
- createPipelineNetGroup
- createSpareModule
- delete_cell_stack_area

- delete_cell_stack_group
- delete_cell_virtual_align
- delete_inst_space_group
- delete_pg_keepout
- deleteAllCellPad
- deleteAllScanCells
- deleteCellEdgeSpacing
- deleteCellEdgeType
- deleteCellPad
- deleteFiller
- deleteGAFillerGroup
- deleteInstPad
- deleteMimCap
- deletePipelineNetGroup
- deleteScanCell
- deleteScanChain
- deleteScanChainPartition
- deleteSpareModule
- displayScanChain
- displaySpareCell
- generate_vertical_cell_edge_constraint
- get_well_tap_mode
- getDensityMapMode
- getEndCapMode
- getFillerMode
- getPinDensityMapMode
- getPlaceMode
- getScanReorderMode
- modifyPipelineNetGroup
- pack_align_macros
- place_connected
- place_design
- place_opt_design
- placeInstance
- placeJtag
- placePad

- placePipeline
- placeSpareModule
- power_integrity_place
- queryDensityInBox
- queryPinDensity
- refinePlace
- report_cell_edge_spacing
- report_cell_edge_type
- report_cell_stack_area
- report_cell_stack_group
- report_cell_virtual_align
- report_inst_space_group
- report_pg_keepout
- report_scan_chain
- reportCellPad
- reportCongestion
- reportDensityMap
- reportInstPad
- reportJtagInst
- reportPinDensityMap
- reportPipeline
- reportScanCell
- reportScanChainPartition
- reset_spare_insts
- scanReorder
- scanTrace
- set_well_tap_mode
- setDensityMapMode
- setEndCapMode
- setFillerMode
- setPinDensityMapMode
- setPlaceMode
- setScanReorderMode
- specify_cell_stack_area
- specify_cell_stack_group
- specify_cell_virtual_align

- specify_pg_keepout
- specifyCellEdgeSpacing
- specifyCellEdgeType
- specifyCellPad
- specifyInstPad
- specifyJtag
- specifyLockupElement
- specifyScanCell
- specifyScanChain
- specifyScanChainPartition
- specifySelectiveBlkgGate
- specifySpareGate
- swap_well_taps
- traceJtag
- unplaceJtag
- unspecifyJtag
- unspecifySelectiveBlkgGate

add_decomp_filler

```
add_decomp_filler
[-help]
-bottom_edge_endcap {cell_name}
-core_column_cell {cell_name}
-core_row_cell {cell_name_list}
[-offsetx offset_in_microns]
[-offsety offset_in_microns]
[-power_domain power_domain_name]
[-prefix prefix_name]
-stepx step_in_microns
-stepy step_in_microns
-top_edge_endcap {cell_name}
-vertical_edge_endcap {cell_name}
```

Inserts breaker cells to divide the chip into several small parts. This reduces the run time of coloring while manufacturing the chip.

This command is run after `floorPlan` and before `addEndCap`.

Parameters

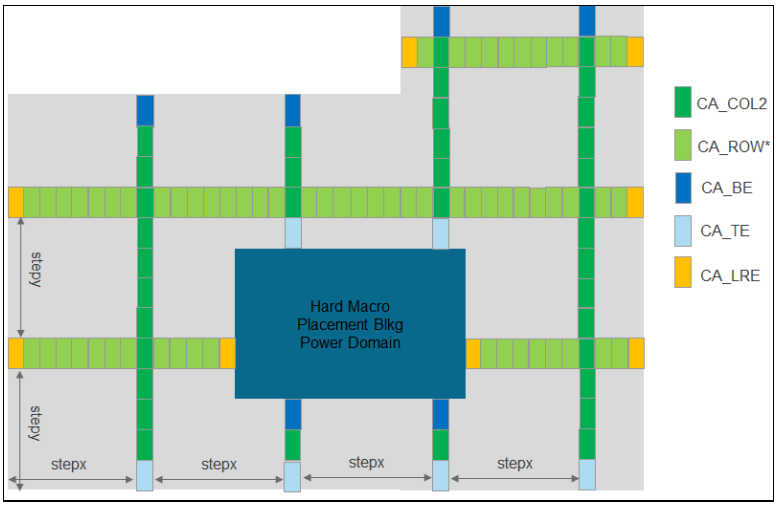
`-bottom_edge_endcap {cell_name}`	Specifies the CA-break end cap cell for rows, to make bottom_edge of endcap face core.
`-core_column_cell {cell_name}`	Specifies the CA-break filler cell for inserting into columns.
`-core_row_cell {cell_name_list}`	Specifies a list of CA-break filler cells inserted horizontally in the core rows.
`-help`	Outputs a brief description that includes type and default information for each `add_decomp_filler` parameter. For a detailed description of the command and all of its parameters, use the man command: `man add_decomp_filler`.
`-offsetx offset_in_microns`	Specifies the offset added to the first step in the horizontal direction.
`-offsety offset_in_microns`	Specifies the offset added to the first step in the vertical direction.
`-power_domain power_domain_name`	Specifies the power domain for which CA-break filler cells are to be inserted. **Note**: If the `-power_domain` option is used, the offset value is applied to the specified power domain. However, if the -power domain is not specified, the offset value is applied to all available power domains. To use different offset value for a specific power domain, you first need to delete the CA fillers from the power domain if it is already filled and then reinsert CA filler to the power domain with the target offset value.
`-prefix prefix_name`	Specifies the prefix for decomp cells. If this option is specified, the user-specified prefix is applied to all inserted decomp fillers, else the default prefix name is used.
`-stepx step_in_microns`	Specifies the decomposition step in horizontal direction.
`-stepy step_in_microns`	Specifies the decomposition step in vertical direction.
`-top_edge_endcap {cell_name}`	Specifies CA-break end cap cell for rows, to make top_edge of endcap face core.
`-vertical_edge_endcap {cell_name}`	Specifies CA-break end cap cell inserted at the left and right edges as end caps.

Example

The code below shows the usage of the `add_decomp_filler` command.

```
add_decomp_filler
-core_row_cell {CA_ROW2 CA_ROW3} -core_column_cell {CA_COL2}
-top_edge_endcap CA_TE -bottom_edge_endcap CA_BE
-vertical_edge_endcap CA_LRE
-stepx 500 -stepy 500
```

Below is the graphical representation of the above example:



Related Information

- addEndCap
- floorPlan

add_gate_array_filler

```
add_gate_array_filler
[-help]
[-allow_shift_distance x y]
[-area x1 y1 x2 y2]
[-cell_orientation {R0|MX|MY|R180}]
[-offsetx offset_in_microns]
[-offsety offset_in_microns]
[-powerDomain powerDomainName]
[-prefix prefixName]
{-cell cellName -stepx step_in_microns -stepy step_in_microns}
```

Inserts GA filler cells with regular pattern during the pre-place stage.

The GA filler cells created by this command are marked as physical-only cells. This means that the `isPhysOnly` attribute of these GA filler cells is set to `true`. This is a prerequisite for the `ecoPlace -useGAFillerCells` command to map the unplaced ECO cells to the added GA filler cells, in case of a GA filler-based post-mask ECO.

Parameters

`-allow_shift_distance x y`	Specifies the distance between the new and the original position in the x and y directions. If you do not specify `-allow_shift_distance`, the GA filler will be removed if it is added at original location with a violation. If `-allow_shift_distance` is specified, the GA can be shifted to avoid violation. If a GA filler cannot be added at the original location because of the violation, it will be moved to the left side, right side, upside, or downside of the original location to ensure that there is no violation. The next filler keeps the step value with the original position of the moved filler.
`-area x1 y1 x2 y2`	Specifies the area in which GA fillers are to be inserted. If no area is specified, GA fillers will be inserted for the complete core area.
`-cell cellName`	Specifies the name of the filler cell to be used.
`-cell_orientation {R0|MX|MY|R180}`	Specifies all legal orientations of GFILL instances.
`-offsetx offset_in_microns`	Specifies the offset, in microns, from the origin in the horizontal direction. If the `-area` parameter is specified, the origin is (x1, y1) of the box defined by `-area`. If `-area` is not specified the origin is (0,0).
`-offsety offset_in_microns`	Specifies the offset, in microns, from the origin in the vertical direction. If the `-area` parameter is specified, the origin is (x1, y1) of the box defined by `-area`. If `-area` is not specified the origin is (0,0).
`-powerDomain powerDomainName`	Specifies the power domain for which fillers are to be inserted.
`-prefix prefixName`	Specifies the prefix for the inserted GA filler cells.
`-stepx step_in_microns`	Specifies the distance between GA fillers in the horizontal direction.
`-stepy step_in_microns`	Specifies the distance between GA fillers in the vertical direction

Examples

- The following command adds GA fillers of type `GAFILL4` with spacing in horizontal and vertical direction of 20 microns. The first GAFILL4 will be placed at (10, 0).

  ```
  add_gate_array_filler -cell GAFILL4 -stepx 20 -stepy 20 -offsetx 10
  ```

- The following command adds GA fillers of type `GAFILL4` with regular spacing 20 and start location (0, 0).

  ```
  add_gate_array_filler -cell GAFILL4 -stepx 20 -stepy 20
  ```

- The following command adds GA fillers of type `GAFILL4` with vertical and horizontal spacing of 10 microns in the area defined by coordinates 100 100 500 500.

  ```
  add_gate_array_filler -area 100 100 500 500 -cell GAFILL4 -stepx 10 -stepy 10
  ```

Related Information

- ecoPlace

add_tap_walls

```
add_tap_walls
[-help]
[-bottom]
-cell cellName
[-layer layer_name]
[-left]
[-orientation {R0 MX MY R180}]
[-pitch distance]
[-prefix prefix]
[-range lx ux|ly uy]
[-right]
[-top]
[-y_flip_side {left right both}]
```

Adds tap walls to the assigned left, right, top, and/or bottom side of a block.

Parameters

`-bottom`	Specifies the add tap-wall cell on the bottom side.
`-cell cellName`	Specifies the name of the cell to insert.
`-help`	Outputs a brief description that includes type and default information for each `add_tap_walls` parameter. For a detailed description of the command and all of its parameters, use the man command: `man add_tap_walls`.
`-layer layer_name`	Specifies the layer name to check rule.
`-left`	Specifies the add tap-wall cell on the left side.
`-orientation {R0 MX MY R180}`	Specifies orientation of the added tap-wall cells.
`-pitch distance`	Specifies the distance from the center of one tap-wall cell to the center of the following wall-tap cell.
`-prefix prefix`	Specifies the prefix of tap-wall cells to inserted.
`-range lx ux|ly uy`	Specifies the {lx, ux} or {ly uy} as the start or end point at the user given side.
`-right`	Specifies the add tap-wall cell on the right side.
`-top`	Specifies the add tap-wall cell on the top side.
`-y_flip_side {left right both}`	Specifies the orientation of the added tap-wall cells. Defines whether flipping tap walls located at certain side in the Y direction if site symmetry allows it. By default, both left and right side tap walls are R0 or MX. If -y_flip_side is set to left, the left side tap walls would be MY or R180. If it is set to both, both left and right side tap walls would be MY or R180.

addEndCap

```
addEndCap
[-help]
[-area llx lly urx ury]
[-coreBoundaryOnly]
[-powerDomain powerDomainName]
[-prefix prefixName]
```

Adds physical-only end cap cells at the ends of the site rows. A single-height cap cell is required at the end of each row. For deep N-well technology, the command can be used to insert row-caps to isolate p-well from top and bottom. Use `setEndCapMode` to specify the row-cap cells. For deep N-well technology, `addEndCap` also gets the end cap list from LEF if the end cap cell is specified with the following MACRO CLASS, but user definition has higher priority than LEF specification.

```
[CLASS ENDCAP {
| TOPEDGE | BOTTOMEDGE | LEFTEDGE | RIGHTEDGE
| LEFTTOPEDGE | RIGHTTOPEDGE
| LEFTBOTTOMEDGE | RIGHTBOTTOMEDGE
| LEFTTOPCORNER | RIGHTTOPCORNER
| LEFTBOTTOMCORNER | RIGHTBOTTOMCORNER}
} ]
```

Note: `addEndCap` is always called after floorplan is done and before `addWellTap`.

Note: End cap insertion honors core boundary, power domain boundary, placement blockage, fixed macro, and fixed macro halo

To delete the end cap cells, use the `deleteFiller` command.

Parameters

`-area llx lly urx ury`	Specifies the coordinates of the bounding box, in microns. By default, the core box that encloses all core rows is used.
`-coreBoundaryOnly`	Specifies that end-cap cells are placed only within the core boundary. This ensures that end-cap cells do not get placed at locations where site rows are obstructed by blockages.
`-help`	Outputs a brief description that includes type and default information for each `addEndCap` parameter. For a detailed description of the command and all of its parameters, use the man command: `man addEndCap`.
`-powerDomain powerDomainName`	Specifies the power domain in which the end-cap cells are inserted. This identifies the area where to place the end-cap cells and connect to the correct power and ground nets.
`-prefix prefixName`	Specifies the prefix for the placed instances. The generated name for the instance uses this prefix as `prefix_uniqueString`. *Prefix Default:* `ENDCAP` ⓘ Do not use an underscore at the end of the prefix name.

Note: If the given leftEdge cell is the same as the rightEdge cell, and the cell and site symmetries allow it, the postCap instances will be flipped in the Y direction.

Examples

- The following command preplaces cell `abc` at the beginning of each site row of the design, and cell `xyz` at the end of each site row of the design in `R0` orientation, and adds the prefix `PwrCap` for the placed instances.

  ```
  setEndCapMode -leftEdge abc -rightEdge xyz; addEndCap -prefix PwrCap
  ```

- The following command deletes all physical-only cells with the prefix `PwrCap`. As a result, all cells created in the previous example are deleted from the design.

  ```
  deleteFiller -prefix PwrCap
  ```

- The following command preplaces cell `abc` at the beginning and end of each site row of the design and, if site symmetry allows, flips the post-cap cell in Y direction. Thus, for N-oriented rows, placed pre-cap cell is in N orientation, and post-cap cell is in FN orientation.

  ```
  setEndCapMode -leftEdge abc -rightEdge abc; addEndCap -prefix PwrCap
  ```

- The following command, in which the pre-cap cell is the same as the post-cap cell, places pre-cap cells in FN orientation and post-cap cells in N orientation in N-oriented rows, if site symmetry allows:

  ```
  setEndCapMode -leftEdge abc -rightEdge abc -flipY; addEndCap -prefix PwrCap
  ```

- The following command flips both the pre-cap and post-cap cell in Y direction. This preplaces both cells `abc` and `xyz` in FN orientation in N-oriented rows:

  ```
  setEndCapMode -leftEdge abc -rightEdge xyz -flipY; addEndCap -prefix PwrCap
  ```

Related Information

- Placing the Design chapter in the *Innovus User Guide*
  - "Adding End-Cap Cells"
- `addWellTap`
- `deleteFiller`
- `setEndCapMode`

addFiller

```
addFiller
[-area  x1   y1   x2   y2 ]
[-cell { filler_cell_list }]
[-createRows {true | false}]
[-diffCellViol {true|false}]
[-check_signal_drc {true | false}]
[-ecoMode {true | false}]
[-fitGap]
[-fixDRC]
[-fix_horizontal_max_length_violation]
[-fix_vertical_max_length_violation]
[-honorPrerouteAsObs {true | false}]
[-markFixed]
[-merge {true | false}]
[-minHole {true | false}]
[-powerDomain powerDomainName ]
[-prefix prefixName]
[-prevent_vertical_stack_max_length_violation]
[-util targetUtilization ]
[-viaEnclosure {true | false}]
```

Inserts filler cell instances in the gaps between standard cell instances. Filler cells provide continuity for the VDD/VSS power rail, as well as for n-wells. In most cases, use this command after placement and before detailed routing. If the design is routed, this command also does DRC checks of the filler cells added to the wires in the design. It does not check adjacent cells. This process ensures that this command is fast enough to be used many times in the design flow.

Note: Before using `addFiller`, run the `globalNetConnect` command to provide global-net-connection rules for supply pins of the added fillers. Without these rules, the built-in design-rule checks of `addFiller` will not be accurate.

To resolve adjacent cell DRC violations created by the insertion of filler cells, run the `verify_drc` command to mark these violations. Then, use the `addFiller` command again with the `-fixDRC` parameter to replace the filler cells that caused the violations.

Note: This command resolves only filler-related violations; it cannot resolve net-based violations. To repair net-based violations, reroute the violated nets.

The `addFiller` command recognizes up to eight implant layers and attempts to honor implant layer width and spacing rules within each implant layer. If it does not find the correct filler cell to add to an implant layer, it tries to add "no-implant" filler cells. If it cannot add the "no-implant" cells, it inserts a filler cell that is large enough that it does not violate the minimum width rule of the relevant implant layer.

Use the `deleteFiller` command to remove the added filler cell instances.

Note: This command supports multithreading.

Some `addFiller` parameters can also be specified with `setFillerMode`. However, the parameters you specify with `addFiller` apply to the current `addFiller` run only, and supersede the parameters set by `setFillerMode`. For more information, see `setFillerMode` .

Parameters

`-area` *x1y1x2y2*	Adds filler cells to the specified area. If the area is not defined properly, does not add filler cells. Normally, designs are placed before filler cells are added. However, If you use this parameter and the design is not placed, you get a warning, and the command adds fillers in the specified area. **Note**: You can specify this parameter with `addFiller` only, and not with `setFillerMode`. • Specifies the area's lower left point on the x axis. • Specifies the area's lower left point on the y axis. • Specifies the area's upper right point on the x axis. • Specifies the area's upper right point on the y axis.
`-cell` *filler_cell_list*	Specifies the list of filler cells to add. Separate filler cell names with spaces and enclose the list with quotation marks (") or curly braces ({}). **Note**: Cells specified by this parameter override the list of cells provided by the `-core` parameter of `setFillerMode`.
`-createRows {true | false}`	Controls whether `addFiller` creates rows in the floorplan if a cell's site does not have rows. Specify false for this parameter to prevent `addFiller` from creating the rows. *Default*: `true`
`-diffCellViol {true|false}`	Checks violations between different cell types. *Default*: `false` **Note**: Using this feature will increase the runtime of `addFiller`.
`-check_signal_drc {true | false}`	Controls postroute DRC violation checking against existing net routing, without considering adjacent cell DRC violations. This parameter does not disable regular placement checks. *Default*: `true` In 16.10, addFiller has been enhanced with DRC. This flow would fill gaps as much as possible even with DRC, and will give the violation report and marker for easy debug. Also, the `-add_fillers_with_drc` option has been added to the `setFillerMode` command to call second addFiller without DRC checking. If filler cells are added after routing, the software checks for DRC violations between regular nets and the filler cells. When the inserted filler violates the existing routing, addFiller tries to use other filler cells to replace the violating one. If it fails to fill the gap without violation, it will insert a violated filler in the second pass and give the violation report and marker for easy debug when `-add_fillers_with_drc` is turned on.

`-ecoMode {true | false}`	Controls whether the software resolves overlaps of standard cells with fillers by removing the overlapping fillers, and inserting new fillers in any existing or newly created gaps due to partial overlaps in eco flow. If this parameter is used after routing, the command also resolves DRC violations of filler geometries of all fillers with regular wires. *Default*: `false`
`-fitGap`	Fills a gap between cells by adding a combination of cells, instead of by adding the single largest cell that fits, if doing so avoids leaving an unfilled single-width gap.
`-fixDRC`	Corrects DRC violations reported by `verify_drc` between filler cells and adjacent standard cells. **Note**: The `addFiller` command resolves only filler-related violations--it cannot resolve net-based violations. To repair net-based violations, reroute the violated nets. Use this parameter in subsequent runs of the `addFiller` command, after adding filler cells and checking for violations with `verify_drc`. The command replaces filler cells that cause DRC violations with fillers that do not cause violations. If it cannot replace a violating filler cell without causing another violation, it leaves a gap. **Note**: You can specify this parameter with `addFiller` only, and not with `setFillerMode`.
`-fix_horizontal_max_length_violation`	Fixes the horizontal max-length violation.
`-fix_vertical_max_length_violation`	Fixes the vertical stack max-length violation.
`-honorPrerouteAsObs {true | false}`	Instructs `addFiller` to honor the setting of `setPlaceMode -place_detail_preroute_as_obs`. That is, when this parameter is `true`, `addFiller` treats preroutes on metal layers where filler cells have geometries as obstructions, and ignores preroutes on other layers. In most cases, this means that `addFiller` treats preroutes on metal1 as obstructions and ignores preroutes on other metal layers. For example, if `setPlaceMode -place_detail_preroute_as_obs` is set to metal1 and metal2, and all the filler cell geometries are on metal1, `addFiller` treats metal1 preroutes as obstructions but not metal2 preroutes. *Default*: `false` `addFiller` ignores `setPlaceMode -place_detail_preroute_as_obs` and inserts fillers in all legal sites.
`-markFixed`	Places newly added filler cells in FIXED status in the Innovus database. **Note**: You can specify this parameter with `addFiller` only, and not with `setFillerMode`.

`-merge {true | false}`	When checking for spacing violations, merges same-net geometries to determine width-dependent spacing. If the LEF file contains width-dependent rules, this parameter causes `addFiller` to have the following behavior: • If you specify true, the command merges overlapping same-net objects to create a large rectangle. • If you specify false, the command uses the shape of the object from the database. *Default*: `false`
`-minHole {true | false}`	Calls `verify_drc` to check for minimum hole violations before adding filler cells and repairs the violations when `addFiller` runs. *Default*: `false`
`-powerDomain` *powerDomainName*	Specifies the power domain in which to add filler cells. *Default*: If this parameter is not specified, and a list of filler cells is specified with the `-cell` parameter, the `addFiller` command tries to add fillers to all power domains. **Note**: You can specify this parameter with `addFiller` only, and not with `setFillerMode`.
`-prefix` *prefixName*	Specifies the prefix for the placed instances. *Default*: `FILLER`
`-prevent_vertical_stack_max_length_violation`	Adds a repair endcap cell to cut the vertical stack of endcap cells to prevent its vertical max-length violation.
`-util` *targetUtilization*	Specifies the percent of free space remaining after placement to fill with given filler cell instances. Use `-util` to add gate-array core filler cells to use later in the design flow, along with gate-array core cells, for metal-only mask changes. **Note**: You can specify this parameter with `addFiller` only, and not with `setFillerMode`. *Range*: More than `0.0` (`0%`) and equal to or less than `1.0` (`100%`) *Default*: `1.0`
`[-viaEnclosure {true | false}]`	Calls `verify_drc` to check for via enclosure violations before adding filler cells and repairs the violations when `addFiller` runs. *Default*: `false`

Examples

- The following command inserts filler cell instances for cells `FILL64`, `FILL16`, `FILL4`, `FILL2`, and `FILL1`:

  ```
  addFiller -cell FILL64 FILL16 FILL4 FILL2 FILL1 -prefix FILLER
  ```

- The following commands show the use of the `-util` parameter to set the utilization of `GACORE` filler cells to 50 percent:

  ```
  # Place the design
  place_design
  ```

```
# Now fill with GACORE cells up to 50% of remaining space
# area utilization
addFiller -cell {gcore16 gcore8 gcore4} -prefix GACORE -util 0.5
# Now fill with the standard filler cells
addFiller -cell {fill8 fill4 fill2 fill} -prefix FILLER
# Continue with routing
```

- The following command enable postRoute DRC checking for min hole violations and via enclosure violations:
  ```
  addFiller -cell FILL16 FILL8 FILL4 FILL2 -check_signal_drc true -minHole true -viaEnclosure true
  ```

Related Information

- deleteFiller
- globalNetConnect
- setFillerMode
- setPlaceMode

addFillerGap

```
addFillerGap
minGap
[-effort {none | low | medium | high}]
[-radius micron]
```

Moves placed standard cell instances to create gaps so that filler cells can be added. Tries to move cells to create a gap of the specified size. If enough space is not available, moves cells until they touch each other. If a gap already exists, and it is larger than the specified size, the command does not move cells to make it smaller.

By default, this command deletes all routing, because the routing is no longer valid due to the movement of the cells. To override this behavior, specify the following command:

```
setPlaceMode -place_detail_preserve_routing true
```

This command has the following limitations:

- It does not moved `FIXED` cells.
- It does not guarantee to that all gaps will be larger than the specified minimum gap size.

Use this command after placement and before routing.

Parameters

minGap	Specifies the minimum-size gap in which a filler cell can be added. Unit is micron.
`-effort {none | low| medium | high}`	Specifies the effort the command needs to spend for honoring the given gap constraint. • `none` effort specifies that no standard cell is moved for honoring the giving gap constraint. • `low` effort specifies that only single height standard cells can be moved within the same row. • `medium` effort specifies that only single height standard cells can be moved across the rows. • `high` effort specifies that the standard cells can be moved across rows, and double height standard cells can be moved within the same row. *Default*: `low`
`-radius` *micron*	Specifies the searching radius when solving the minimum filler gap. Default: 16

Example

The following command moves standard cells within a row to create gaps of at least two microns between the cells:

```
addFillerGap 0.168
```

Related Topics

- `setPlaceMode`

addMimCap

```
addMimCap
[-help]
-cell cellName
[-orient orientation]
[-prefix prefixName]
[{-stepX step_in_microns -stepY step_in_microns}
[-area {x1 y1 x2 y2}]][{-area {x1 y1 x2 y2} | -loc {x y}}] [-loc {x y} | -offset {x y}]
```

Adds Metal-Insulator-Metal (MIM) cap cell(s) into the design. This command is used before placing the standard cells.

Parameters

`-area x1 y1 x2 y2`	Specifies the area in which MIM caps are to be inserted. If no area is specifies, MIM caps will be inserted in the complete core area. This option must be used in conjunction with -`stepX` and `-step`Y and is mutually exclusive with `-loc`.
`-cell` *cellName*	Specifies the name of the MIM cap cell.
`-help`	Outputs a brief description that includes type and default information for each `addMimCap` parameter. For a detailed description of the command and all of its parameters, use the man command: `man addMimCap`.
`-loc {x y}`	Inserts MIM cap cell at the specified location.
`-offset {x y}`	Specifies the offset in microns from the origin, in horizontal and vertical directions. If the `-area` parameter is specified, the origin is (x1, y1) of the box defined by `-area`. If `-area` is not specified, the origin is (0,0). This option must be used in conjunction with -`stepX` and `-step`Y and is mutually exclusive with `-loc`.
`-orient` *orientation*	Specifies the orientation for the inserted MIM cap cell(s). *Default*: `R0`
`-prefix` *prefixName*	Specifies the prefix for the inserted MIM cap cells.
`-stepX` *step_in_microns*	Specifies the distance between MIM cap cells in the horizontal direction. This option must be used together with `-stepY` and is mutually exclusive with `-loc`.
`-stepY` *step_in_microns*	Specifies the distance between MIM cap cells in the vertical direction. This option must be used together with `-stepX` and is mutually exclusive with `-loc`.

Examples

- The following command inserts the MIM cap cells of type `mim_cell1` to a specific area, with a custom offset, orientation, and step:
  ```
  addMimCap -cell mim_cell1 -orient R0 -stepX 34.56 -stepY 34.56 -offset { 9.317 8.336 } -area 20 20 100 100
  ```

Related Topics

- `deleteMimCap`

addSpareInstance

```
addSpareInstance
[-help]
[-async netName:terminalList [netName:terminalList … ]
[-clock netName]
-file fileName
[-fix_placed_insts {TRUE|FALSE}]
[-hier moduleName | -powerDomain domainName [-submodule submoduleName]]
[-prefix prefix]
[-tie {0 | 1}]
```

Specifies a file that lists spare cells to add to the netlist. You can use this command before or after standard cell placement.

- If you use this command before placement, the `place_design` command distributes and places the added instances without including them in spare modules. It adds the instances to the top level of the design by default.
- If you use this command after placement, the software places the added instances without creating spare modules or disturbing the existing placement. The distribution of the cells is dependent upon the number of instantiations of each cell type.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `addSpareInstance` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man addSpareInstance`.
`-async` *`netName:terminalList`*	Specifies which net(s) to tie off the async pin(s). Default: async pins will be tied to the logical zero net.
`-clock` *`netName`*	Specifies the clock net that is tied to the clock pins.
`-file` *`fileName`*	Specifies a text file that lists the spare cells, and the number of each, to add to the netlist. The file has the following format: "*`cellName number`*" Begin each line of a comment with a number sign (`#`).
`-fix_placed_insts {TRUE|FALSE}`	Specifies not to move the existing placed instances to find location for newly added spare instances.
`-hier` *`moduleName`*	Specifies the name of the module to which spares will be added.
`-powerDomain` *`domainName`*	Specifies the name of the power domain to which spares will be added.
`-prefix` *`prefix`*	Specifies a prefix for the names of the newly added cells. *Default:* `FE_spare`
`-submodule` *`submoduleName`*	Specifies the name of the module created for spares.
`-tie {0 | 1}`	Specifies the logical net to which the cells are connected. *Default:* `1`

Example

The following command adds instances of the cells specified in `spare_file` and ties them to logical net `0`. It ties the clock pins to `sysclk`:

```
addSpareInstance -file spare_file -tie 0 -clock sysclk
```

`spare_file` contains the following:

```
DFFNSRX2T1T10   1
INVX4T1T10      2
NOR2X2T1T10     2
```

The following command adds instances of the cells specified in spare.txt.It ties the async pin RN to net VDD:

```
addSpareInstance -file spare.txt -async VDD:RN
```

`spare.txt` contains the following:

```
SDFFRHQXL 5
```

addWellTap

```
addWellTap
[-help]
[-area x1 y1 x2 y2]
[-avoidAbutment]
[-cell cellName]
[-cellInterval microns]
[-channel_offset offset_value]
[-checkerBoard]
[-fixedGap]
[-inRowOffset microns]
[{-check_channel | -vertical_boundary_spacing spacing_with_well_tap | -incremental list_of_cells}]
[-powerDomain powerDomainName]
[-prefix prefixName]
[-siteOffset number_of_sites]
[-skipRow number]
[-startRowNum number]
[{{-termination_cell termination_cell_list | {-top_termination_cell cell_list -bottom_termination_cell cell_list}} -
column_cell tap_column_cell_list} [-block_boundary_only {true | false}]]
[{-wellCutCell list_of_well_cut_cells}
[-fixCutCell {true | false}]]
[-pitch microns [-pitchOffset microns]]
[{-insert_cells list_of_cells | -incremental list_of_cells}]
```

Adds physical-only well-tap cells. Well-tap cells are physical-only filler cells that are required by some technology libraries to limit resistance between power or ground connections and wells of the substrate. Well-tap cells are placed in a preplaced status.

- To delete well-tap cells, use the `deleteFiller` command.
- To verify well-tap placement, generate markers for violations caused by well-tap cells, and create a violation report, use the `verifyWellTap` command.

`addWellTap` treats the endcap cell as a tap function cell when `set_well_tap_mode -tap_function_cells` is set.

Parameters

`-area x1y1x2y2`	Specifies the coordinates of the bounding box, in microns. By default, the core box that encloses all core rows is used. • *x1:* Specifies the area's lower left point on the x axis. • *y1:* Specifies the area's lower left point on the y axis. • *x2:* Specifies the area's upper right point on the x axis. • *y2:* Specifies the area's upper right point on the y axis.
`-avoidAbutment`	Specifies whether the inserted taps should avoid horizontal and vertical abutment.
`-block_boundary_only {true | false}`	Specifies whether to insert termination tap cells only for well tap columns on the block boundary. Default is false, termination cells are inserted on both core boundary row and block boundary row. **Note:** This parameter works only when inserting termination cells.
`-bottom_termination_cell cell_list`	Specifies the list of termination cells to be used at the bottom.

`-cell` *cellName*	Specifies the name of the cell in the technology library to use as a well-tap. The cell should be of class type CORE.
`-cellInterval` *microns*	Specifies the maximum distance from the center of one well-tap cell to the center of the following well-tap cell in the same row. If well-tap cells cannot be placed at this location due to a placement blockage, the software searches for a legal location: first, towards the previous well-tap cell, and then farther away.
`-channel_offset` *offset_value*	Specifies the offset value between taps and blockage in channel. *Minimum*: 0 *Maximum*: 1.79769313486232e+308
`-check_channel`	Inserts well taps inside channel.
`-checkerBoard`	Places the well-tap cells in a checkered pattern, that is, with every other row offset by one-half gap. In rows where a one-half offset gap is not available in the row above or below, the tap-cell is inserted at one-half gap in the row in question.Thus, for a simple floorplan, the first and last rows will have well-taps at one-half gap only, while all other rows will have the full gap. When a floorplan has hard blocks or placement blockages, the rows above and below the block (or blockage) will also have well-taps at one-half gap. **Note:** When this parameter is specified, `-skipRow` is ignored.
`-column_cell tap_column_cell_list`	Specifies the list of well tap columns that need to be terminated. This parameter must be specified for termination tap cell insertion.
`-fixCutCell`	Fixes all the instances which belong to the `wellCutCell` list.
`-fixedGap`	Ensures that the specified gap is always maintained. That is, the software does not search for a legal location if it cannot place well-tap cells at the specified `-cellInterval`. Instead, it goes on to place well-tap cells at the next position. You can modify the gap by using the `-pitch` and `-pitchOffset` parameters.
`-help`	Outputs a brief description that includes type and default information for each `addWellTap` parameter. For a detailed description of the command and all of its parameters, use the man command: `man addWellTap`.
`-incremental` *list_of_cells*	Checks whether any instances of the cells specified in the *list_of_cells* are within 0.45 times the `-cellInterval` from the next well-tap cell in the same row. If so, does not add well-tap cells at that location.
`-inRowOffset` *microns*	Specifies the offset, in microns, of the first well-tap cell from the start of the site row. *Default*: `0` (first cell on beginning of row is the starting point).
`-insert_cells` *list_of_cells*	Inserts mixed well-taps, and supports the LUP layer and boundary layer rules.
`-pitch` *microns*	Used with the `-fixedGap` parameter to give the software added flexibility to place well-tap cells if it is unable to place a cell at the specified location. If you specify this parameter, the software first tries to fit the well-tap cells in the locations specified by `-cellInterval`, or forced by `-fixedGap`; if it cannot do so, it adds well tap cells at the closest location allowed by this parameter.

`-pitchOffset` *microns*	Used with the `-fixedGap` and `-pitch` parameters to give the software greater flexibility. If you specify this parameter, the software first tries to fit the well-tap cells in the locations specified by `-cellInterval`, or forced by `-fixedGap`; if it cannot do so, it adds well tap cells at the closest location allowed by this parameter.
`-powerDomain` *powerDomainName*	Specifies the power domain in which the well-tap cells are inserted. This is the area where the well-tap cells are placed and connected to the correct power and ground nets.
`-prefix` *prefixName*	Specifies the prefix for the placed well-tap cells. The software adds an underscore and a unique string to the prefix, in the following format: `prefix_`*`uniqueString`*. *Prefix Default:* `WELLTAP` ⓘ Do not include an underscore when you specify a prefix.
`-siteOffset` *number_of_sites*	Specifies an offset value, in number of sites, so that the vertical taps are aware of.
`-skipRow` *number*	Specifies the number of rows to skip between two rows of an identical pattern. **Note:** If you specify `-checkerBoard`, this parameter is ignored. *Default*: `0` (No rows are skipped, so well-tap cells are added in the same pattern to every row)
`-startRowNum` *number*	Specifies the row number starting from the from bottom or left side of the design on which the well-tap cells must be placed. *Default*: 1 (first row from the bottom is the starting point)
`-termination_cell` *termination_cell_list*	Specifies the list of cells that can be used as termination taps. `addWellTap` selects the best fit termination cells from this list and inserts them at the top and bottom of well tap column segment. The existing non-termination well taps are deleted as needed. If the well tap columns have already been terminated with a valid termination cell, the existing termination cell are kept in the incremental run and no changes are made. **Note:** It is recommended that well tap columns are added before termination cell insertion. **Note:** This parameter must be specified for termination tap cell insertion.
`-top_termination_cell` *cell_list*	Specifies the list of termination cells to be used at the top.
`-vertical_boundary_spacing` *spacing_with_well_tap*	Specifies vertical boundary spacing with well tap. `addWellTap` will recognize all vertical boundaries and keep center of tap cells to these boundaries equal to or less than the specified distance when inserting. *Minimum*: -1 *Maximum*: 1.79769313486232e+308
`-wellCutCell` list_of_well_cut_cells	Specifies the list of cells with the well cuts. The `addWellTap -incremental` flow treats cells specified in this list as placement blockage because they cut the row and fix the latch up violations within them.

Examples

- The following command places instances of well-tap cell `xyz` on every row in the design. Instances in the same row are at least 22.50 µm but no more than 50 µm apart. Each instance name is prefixed with `welltap_`.

  ```
  addWellTap -cell xyz -prefix welltap -cellInterval 50
  ```

- The following commands place instances of well-tap cell `xyz` in a checkered pattern. The first instance in the odd row numbers (`1`, `3`, `5`, and so on) is placed at `0` (zero) offset. The first instance in even row numbers (2, 4, 6, and so on) is placed at 50 µm offset. Instances in the same row are at least 45 µm but no more than 100 µm apart. Instances on the odd rows are named `wtap_odd_1`, `wtap_odd_2`, and so on. Instances on the even rows are named `wtap_even_1`, `wtap_even_2`, and so on.

  ```
  addWellTap -cell xyz -prefix wtap_odd -cellInterval 100 -skipRow 1
  addWellTap -cell xyz -prefix wtap_even -cellInterval 100 -skipRow 1 \
      -startRowNum 2 -inRowOffset 50
  ```

- The following command places instances of well-tap cell `WTAP` on every row in the design.Before placing the instances, the software checks for cells named `WTAP` and `PSO1`, and if it finds any within the minimum gap (that is, 21.68 µm from the nearest well-tap cell in that row) it does not place any instances in that area.

  ```
  addWellTap -cell WTAP -incremental "WTAP PSO1"
  ```

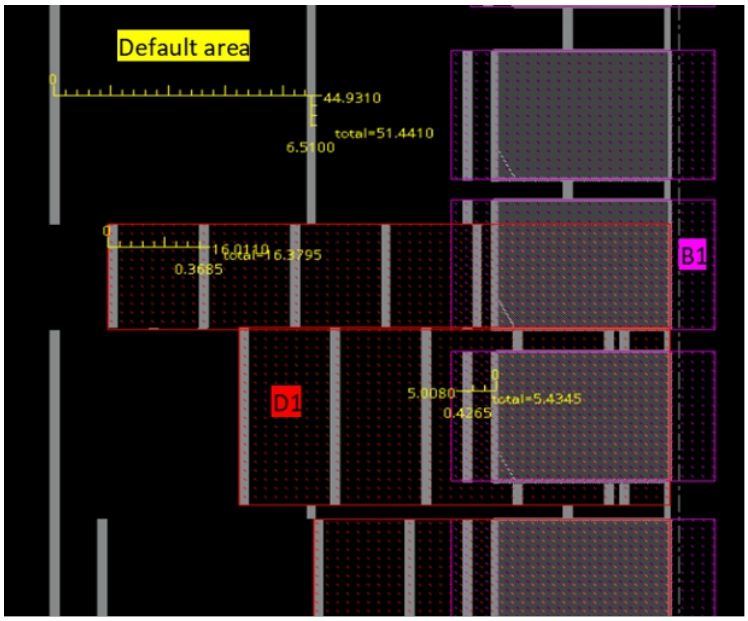
- In the following commands, cell TAP is specified as general well tap and TAP_TERMINATION is specified as termination well tap. TAP cells and tap columns are inserted first, followed by TAP_TERMINATION cells as termination cells at the top and bottom of each TAP column.

  ```
  addWellTap -cell TAP -cellInterval 28


  addWellTap -termination_cell TAP_TERMINATION -column_cell TAP
  ```

- The following command places well tap cell `xyz` on both boundary layer region and dense layer region with different rules.

  ```
  addWellTap -insert_cells { { xyz rule 22.5 boundary_layer B1 boundary_rule 5 } { xyz dense_layer D1 dense_rule 8
  }}
  ```



Related Information

- `deleteFiller`
- `verifyWellTap`
- Low Power Design chapter in the *Innovus User Guide*
  - Substrate Biasing
- Placing the Design chapter in the *Innovus User Guide*
  - Adding Well-Tap Cells

change_IB_cell_site_name

```
change_IB_cell_site_name
[-help]
[-NF site_name]
[-NNF site_name]
[-PF site_name]
[-PPF site_name]
[-new site_name]
[-original site_name]
[-vertical]
```

Change sites for the cells whose site definitions do not match the site definitions used by Innovus. This is applicable only for specific nodes.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `change_IB_cell_site_name` parameter. For a detailed description of the command and all of its parameters, use the man command: `man change_IB_cell_site_name`
`-NF site_name`	Specifies the site name of the N-filler cell.
`-NNF site_name`	Specifies site name of NN-filler cell.
`-PF site_name`	Specifies the site name of the P-filler cell.
`-PPF site_name`	Specifies site name of PP-filler cell.
`-new site_name`	Specifies the new site name of the IB cell
`-original site_name`	Specifies the original site name of the IB cell.
`-vertical`	Is IB transform.

check_base_layer_density

```
  check_base_layer_density
[-help]
[-layer layerName]
[-site_parameter sitePara]
```

Checks the CPODE density by the defined pattern.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `check_base_layer_density` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man check_base_layer_density`.
`-layer layerName`	Specifies the layer name for the base layer density check.
`-site_parameter sitePara`	Specifies the site parameter of the CPODE layer. The site parameters of the CPODE layer include: • `shape_num`: Number of the CPODE shape • `site_num`: Number of the site in row. • `step`: Number of stepping The tool checks the CPODE number of consecutive `site_num` sites by every `step` steps. The CPODE number should be less than the defined `shape_num`. Otherwise, the tool will give a violation.

Example

- In the following example, the command checks the CPODE in the number of consecutive 3 sites by every 1 step. The CPODE number should less than 4.

```
check_base_layer_density -site_parameter  {4 4 1}  -layer  CPODE

Number of CPODE:3

Number of site in row:3

Number of stepping:1
```

- In the following example, the command checks the CPODE number of consecutive 2 sites by every 1 step. The CPODE number should less than 4.

```
check_base_layer_density -site_parameter  {5 4 1} -layer  CPODE

Number of CPODE:5

Number of site in row:4

Number of stepping:1
```

check_library

```
  check_library
[-help]
[-place [-cell cell_name|-all_lib_cell][-legal_frequency ] [-file fileName] ]
```

Checks the problematic cells that cause issues such as bad geometries for DRC and bad pin access for routing. You can specify a library cell or a partition cell. It also performs checks for the multi-height cell legal frequency. The report is dumped into an output file.

Parameters

`-all_lib_cell`	Specifies to check all library cells.
`-cell cell_name`	Specifies the cell to check.
`-file fileName`	Specifies the name of the output file.
`-help`	Outputs a brief description that includes type and default information for each `check_library` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man check_library`.
`-legal_frequency`	Specifies to check multi-height cell legal frequency
`-place`	Specifies to do library checking for placement.

Example

- The following command checks the `buf01` cell:

```
check_library -place -cell buf01

Estimated cell power/ground rail width = 0.021 um

*info: running library checker ...

*info: total 1 cell checked.

*reporting cell (buf01) ...

reporting cell drc : "legal"

reporting cell pin access : "legal"

reporting cell symmetry : cell (buf01) has lef symmetry X Y

reporting cell pin alignment : "legal"

reporting cell legal frequency ...
cell (buf01) legalization frequency: one legal location out of every 1.25 sites
```

checkFiller

```
checkFiller
[-help]
[-area llx lly urx ury]
[-file fileName]
[-horizontal_max_length]
[-powerDomain power_domain_name]
[-vertical_stack_max_length]
[-reportGap microns | -adjacentFiller1]
```

Checks for locations that are missing filler cells, after adding the cells with the addFiller command. Optionally, generates a report with details on sites that are missing filler cells. It also does the static check, and reports the CPODE Group Spacing violation if the distance of the two CPODE groups is less than $value. The distance of the two groups should be greater than or equal to $value.

If you run this command without any parameters, it reports the total number of sites in the core area that are missing filler cells.

Parameters

`-adjacentFiller1`	Checks for locations that are filled with adjacent one-site-width fillers. With this parameter, `checkFiller` flags any one-site-width filler placed next to each other as a violation. To enable this parameter, you must first specify a one-site-width filler by using the following command: `setFillerMode -core fillerCellList`
`-area {llx lly urx ury}`	Specifies a bounding box for the area in which to check for missing filler cells. *Default:* Entire core area
`-file fileName`	Writes the output of the `checkFiller` command to the specified file.
`-horizontal_max_length`	Checks if the horizontal stack length violates max limit.
`-help`	Outputs a brief description that includes type and default information for each `checkFiller` parameter. For a detailed description of the command and all of its parameters, use the man command: `man checkFiller`.
`-powerDomain power_domain_name`	Checks for missing filler cells in the specified power domain. The software checks for missing filler cells in all power domains if you do not specify this parameter.
`-reportGap microns`	Checks only for gaps of the specified size--ignores any other gap, whether larger or smaller.
`-vertical_stack_max_length`	Checks if the vertical stack length violates the max limit.

Examples

The following command checks the entire core area after running `addFiller` and reports the number of sites that are missing filler cells:

```
checkFiller
```

The software reports the following:

```
*INFO: Total number of gaps found: 85
```

The following command checks within the specified bounding box at coordinates {0 0 100 100} and reports the number of sites that have a gap of exactly 0.14 microns:

```
checkFiller -reportGap 0.14 -area {0 0 100 100}
```

The software reports the following:

```
*INFO: Total number of 0.140 microns width gap found: 3
```

Related Information

- addFiller
- setFillerMode

checkPlace

```
checkPlace
[violationReportFileName]
[-clearMarker]
[-honorPrerouteForFiller]
[-ignoreFillerInUtil]
[-ignoreOutOfCore]
[-inst list_of_instances]
[-ioPinBlockage]
[-macroBlockage]
[-noCheckPinAccess]
[-noHardFence]
[-noHalo]
[-noPreplaced]
[-selectedOnly]
```

Checks `FIXED` and `PLACED` cells for violations, adds violation markers to the design display area, and generates a violation report. Use the Violation Browser to see the violation markers.

This command considers preplaced instances and macros as `FIXED`.

Note: The utilization value reported by `checkPlace` and the density value reported by `timeDesign` might differ because `checkPlace` considers padding when calculating utilization but `timeDesign` does not consider padding when calculating density.

You can delete padding by running the `deleteAllCellPad` command before running `checkPlace`.

Note: If `setPlaceMode -place_detail_legalization_inst_gap` is defined, `checkPlace` will report the gaps that violate the specified spacing.

The `checkPlace` command checks for the following violations:

- Implant max cover length. If this violation is detected, GUI adds markers for every instance. As per this violation rule the implant shapes cannot be fully covered horizontally by neighbor shapes on both top and bottom opposite sides. If checkPlace detects the violation, GUI adds markers for every instance
- Region and fence violations (Instances placed outside their region or fence)
- Out of core area violations (Instances placed partly or wholly outside the core area)
- Not placed on placement grid (Standard cells not on the placement grid, or blocks not on the manufacturing grid). If the design is using the FinFET technology, checkPlace will also check if standard cells and blocks are not aligned on the FinFET grid and report violations.
- Row orientation violations (Instances whose orientation is illegal for their row; may short power to ground)
- Tech site violations (Instances placed on tech sites not matching their cells and instances that do not have rows for their tech sites)
- Overlapping with other instance (Placed instances that overlap with other instances)
- Pin access violations (Instances with at least one signal pin that does not have routing access due to pre-wires)
- Orientation violations (Instances whose orientation is not legal based on its cell symmetry rules)
- Routing/placement blockage violations in the core area (Instances placed in an obstruction created by pre-wires or routing blockage)
  The command does not check fixed macros.
- Site orientation violations (Instances whose orientation does not match the tech site orientation)
- Not of fence violations (Instances placed inside a fence to which it does not belong)

- Instance padding violations if `setPlaceMode -place_detail_honor_inst_pad` is set to `true`.
- Standard cells' power-pin connection

In addition to checking for violations, the command reports the following:

- Number of placed instances in the design
- Number of unplaced macros in the design
- Placement density, in terms of used sites over available sites
- Default power domain density

Parameters

`-noCheckPinAccess`	Specifies whether to check for pin access violations.
`-clearMarker`	Removes the violation markers from a previous `checkPlace` run, without rerunning `checkPlace`.
`-honorPrerouteForFiller`	Checks for placement blockage violations on physical instances placed under preroutes or specified by `setPlaceMode -place_detail_preroute_as_obs`. By default, `checkPlace` does not treat filler cells placed under preroutes as violations, even though they are placement/routing blockages; however, it does treat standard cells placed under preroutes as violations.
`-ignoreFillerInUtil`	Ignores filler cells in calculating utilization.
`-ignoreOutOfCore`	Skips instances outside the core design area.
`-inst` *`list_of_instances`*	Specifies the list of instances to check.
`-ioPinBlockage`	Instructs legalization routines to treat top-level I/O pins as fixed metal obstructions while placing standard cell instances at the block level. There is a minor run-time penalty with this parameter, so it is not turned on by default.
`-macroBlockage`	Checks for overlap between hard macro and macro-only blockage and reports macro blockage violation. By default, macro-only blockage is ignored for both hard macro and standard cell. With this setting, it is treated as a placement obstruction for macros.
`-noHalo`	Skips block halos. Uses the place-and-route boundary for the placement overlap check.
`-noHardFence`	Ignores fence and region placement constraints.
`-noPreplaced`	Does not check fixed instances. When this parameter is specified, `checkPlace` treats fixed instances as placement blockages.
`-selectedOnly`	Specifies to check only the selected instances legality.
`violationReportFileName`	Specifies the report file. If you do not specify this parameter, the command displays a summary report. The report file is more detailed than the summary report, and lists every instance with a violation.It contains details on the violations along with a Tcl procedure to select instances based on the violation type.

Note: If a design is loaded where a macro is fixed in an orientation that is not supported according to its LEF, running `checkPlace` changes the orientation and the position of the macro, even if the `-noPreplaced` option is specified. Here is an example:

Cell positions before checkPlace:

```
FPFEL0_SLM/fpicl/fpiclmsc/ThermRemote_FP_1: {544.32 648}
FPFEL0_SLM/fpicl/fpiclmsc/ThermRemote_FP_2: {544.32 669.6}

< CMD> checkPlace
**WARN: (ENCDB-1266): Both given orientation and original orientation not legal,
FPFEL0_SLM/fpicl/fpiclmsc/ThermRemote_FP_1 changes to orientation R0
**WARN: (ENCDB-1266): Both given orientation and original orientation not legal,
FPFEL0_SLM/fpicl/fpiclmsc/ThermRemote_FP_2 changes to orientation R0
Design has 3 cells with cell padding or block halo.
Begin checking placement ... (start mem=690.9M, init mem=714.5M)
*info: Placed = 1098
*info: Unplaced = 0
Placement Density:0.00%(0/407468)
Finished checkPlace (cpu: total=0:00:07.6, vio checks=0:00:00.0; mem=696.2M)
```

Cell positions after checkPlace:

```
FPFEL0_SLM/fpicl/fpiclmsc/ThermRemote_FP_1: {544.446 648}
FPFEL0_SLM/fpicl/fpiclmsc/ThermRemote_FP_2: {544.446 669.6}
```

If few instances are getting overlapped, the report will clearly specifies the violation type and list all the instances. Here is an example:

```
#########################################################

## Total instances with placement violations: 5

#########################################################

## Insts placed overlapping with other insts (5) ##

#########################################################

### 4 (Preplaced) Overlapping with other instance>

set pchkOverlapFixedList [list \

 {proc0/rf0/u0/u0/rfd_reg_22_27/DFF} \

 {proc0/rf0/u0/u0/rfd_reg_37_26/DFF} \

 {proc0/rf0/u0/u0/rfd_reg_37_27/DFF} \

 {proc0/rf0/u0/u0/rfd_reg_38_27/DFF} \

 ]
```

Related Information

- timeDesign
- deleteAllCellPad
- setPlaceMode
- verify_drc
- Tools Menu chapter in the *Innovus Menu Reference*
    - Violation Browser

clearScanDisplay

```
clearScanDisplay
[-help]
[chain_name]
```

Removes scan chains from the display.

Parameters

chain_name	Specifies the scan chain to remove from the display. *Default:* Removes all scan chains from the display.

Related Information

- Placing the Design chapter in the *Innovus User Guide*
  - Optimizing and Reordering Scan Chains

clearSpareCellDisplay

```
clearSpareCellDisplay
```

Removes spare cell instances from the display.

Parameters

None

clonePlace

```
  clonePlace
[-help]
[-clone_list cloneinstance(s)orientation(s)]
[-master_inst instance name]
```

Places all clone instances with relative location and relative orientation according to the master partition instances. Adjusts instance orientation based on the clone orientation. Use this command after running `place_design`.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `clonePlace`. For a detailed description of the command and all of its parameters, use the man command: `man clonePlace`
`-clone_list` *`cloneinstance(s)orientation`*	Specifies clone instances with relative location.
`-master_inst` *`instance name`*	Specifies the master partition instance.

Related Information

- `place_design`

congRepair

```
congRepair
-help
-area llx lly urx ury
```

Relieves the congestion of the design. This command performs incremental placement based on early global route congestion results and improves the congestion hot spots by spreading out the cells. It is mostly used after `place_design`, but sometimes it can be used after pre CTS optimization as well. Since this command tends to move many cells around, it should not be run after post CTS optimization.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `congRepair` . For a detailed description of the command and all of its parameters, use the man command: `man congRepair`
`-area` *`llx lly urx ury`*	Specifies the area in which `congRepair` will run. • *`llx:`* Specifies the area's lower left x coordinate. • *`lly:`* Specifies the area's lower left y coordinate. • *`urx:`* Specifies the area's upper right x coordinate. • *`ury:`* Specifies the area's upper right y coordinate. **Note**: If you do not specify this option, `congRepair` will run on the complete design.

Related Information

- `place_design`

create_inst_space_group

```
create_inst_space_group
[-help]
groupName
-inst listOfInstances
{[{[ -spacing_y Distance] [-spacing_x Distance]} [-checking_box {all|cross_only}]] | [-spacing_radial Distance]}
```

Creates space group for instances with a specific vertical or horizontal-distance constraint.

The `create_inst_space_group` information can be saved in DB.

Parameters

`-checking_box {all cross_only}`	Checks all area or the intersection area from `-spacing_x` and `-spacing_y`. • `cross_only`: Checks the intersection area of -`spacing_x` and -`spacing_y`. If any instance violates both -`spacing_x` and -`spacing_y`, it is marked as a violation. • `all`: Checks all area covered by `spacing_x` and `spacing_y`. If any instance violates -`spacing_x` or -`spacing_y`, it is marked as a violation. The constrained box is ended by core/blockage/power domain/hard macro/row boundary. *Default*: `cross_only`
groupName	Specifies the name of the group.
`-help`	Outputs a brief description that includes type and default information for each `create_inst_space_group` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_inst_space_group`.
`-inst` *listOfInstances*	Specifies all instances that belong to the same space group by name.
`-spacing_radial` *Distance*	Specifies the radial distance, in microns.
`-spacing_x` *Distance*	Specifies the distance, in microns, between each specific instance in the X direction.
`-spacing_y` *Distance*	Specifies the distance, in microns, between each specific instance in the Y direction.

Related Information

- `delete_inst_space_group`
- `report_inst_space_group`

createGAFillerGroup

```
createGAFillerGroup
[-help]
-ga_cells gaCellList
-ga_fillers gaFillerCellList
-group groupName
```

It creates gate array filler groups for the post-mask `ecoPlace` flow. When a filler group is created, `ecoPlace -place_detail_use_GA_filler_groups true` can only place the GA cells at the GA filler location that is included in the same filler group.

This command can be used multiple times to define several gate array groups for `ecoPlace`. If one filler group name is defined many times, the last one will be kept.

Parameters

`-ga_cells gaCellList`	Specifies the list of GA cells. The * wildcard is supported to specify the cell list.
`-ga_fillers gaFillerCellList`	Specifies the list of GA filler cells. **Note**: One GA cell or GA filler cell cannot be defined in two groups, otherwise it will give error.
`-group groupName`	Specifies the group name.

Example

The following commands show the flow for GA Filler groups:

```
innovus> createGAFillerGroup -ga_cells <GA_cell_list> -ga_fillers <GA_filler_list> -group <group>
innovus> ecoPlace -place_detail_use_GA_filler_groups true
```

Related Information

- `deleteGAFillerGroup`
- `ecoPlace`

createPipelineBusGuide

```
createPipelineBusGuide
[-help]
[-netGroup netGroupNames]
[-widthFactor value]
[-layer id1:id2]
[-report fileName]
[-spacing value {-report filename}]
```

Generates bus-guide for the pipeline netgroup. The bus-guide is used to guide the router for expected path. It can be used in the analysis flow or after the pipeline flops are inserted in the netlist.

Note: The bus-guide width is scalable by the width factor. Default width factor is 1.0. The bus width equals to the (width factor) x (the largest pitch of given routing layers) x (the number of bus bits) + 1 extra pitch. The extra pitch is added to account for the possibility that bus-guides do not line up exactly with the routing grid.

Parameters

`-netGroup netGroupNames`	Specifies the netgroup name(s) that the bus-guide is created for. Default: `""`
`-widthFactor value`	Specifies the width factor of bus-guide. Default: `1.0`
`-layer id1:id2`	Specifies the layer(s) to use for the generated bus-guide. It can be a single layer or layer range. Default: `all`
`-report fileName`	Specifies the analysis report file name. Default: `<filename>.rpt`
`-spacing float`	Specifies the maximum space between flops. Default: `""`

Example

- The following command writes out a report file 'busguide.rpt' which tells the total estimated length of bus (in microns), the number of stages that will be needed (based on the given spacing of 200 um), and location of each stage of PFFs for all net groups on all routing layers.
  ```
  createPipelineBusGuide -spacing 200 -report busguide.rpt
  ```
- The following command create bus-guide for net group 'NG1' on routing layers 2, 3, and 4. The bus-guide width is 2 times the default width.
  ```
  createPipelineBusGuide -netGroup NG1 -widthFactor 2.0 -layer 2:4
  ```

Related Information

- createPipelineNetGroup
- deletePipelineNetGroup
- modifyPipelineNetGroup

createPipelineNetGroup

```
createPipelineNetGroup
[-help]
netGroupName
[-endReserve reserveDistanceInMicrons]
[-endTerm termList]
[-inst instNameListOrRegularExp]
[-net listOfNetnamesOrRegularExp]
[-startReserve reserveDistanceInMicrons]
[-startTerm termList]
```

Creates Net Groups for Pipeline Flip-Flop (PFF) Placement and Reporting. This command can be used for analysis before the flops exist in the netlist; or it can be used post flop insertion to add a new net group.

Parameters

`-endReserve reserveDistanceInMicrons`	Specifies a distance value from the end points on block boundary to the internal connection inside the block, which is accounted for in each stage distance value. The unit is in micron.
`-endTerm termList`	Specifies the end points of net groups.
`netGroupName`	Specifies the name of the net group.
`-inst instNameListOrRegularExp`	Creates net group of all nets connected to the given instances. All the instances must be flip-flops or latches. All the nets attached to the D and Q pins are put into the PFF net group.
`-net listOfNetNamesOrRegularExp`	Creates net group of all given nets.
`-startReserve reserveDistanceInMicrons`	Specifies a distance budget from internal connections inside block to the start point of the PFF net. This value is added to the first stage distance when calculating equidistant spacing for each stage. The unit is in micron.
`-startTerm termList`	Creates net group of all data nets between the given end points. The PFF-chain is traced from the startTerm, through D and Q terminals of PFF(s) until an endTerm is matched.

Example

- The following command creates a net group named `netGroup1' for Pipeline Flip-Flop (PFF) of all data nets between `blockA/DATA_OUT*` and `blockB/DATA_IN*`:

  ```
  createPipelineNetGroup netGroup1 -startTerm blockA/DATA_OUT* -endTerm blockB/DATA_IN*
  ```

- The following command creates a net group named `netGroup2` for all nets connected with all instance names that begin with `PIPELINE_INST/in`:

  ```
  createPipelineNetGroup NetGroup2 -inst PIPELINE_INST/in*
  ```

- The following command creates a net group named `netGroup3` for all net names that begin with `SUM_P4`.

  ```
  createPipelineNetGroup NetGroup3 -net SUM_P4*
  ```

Related Information

- deletePipelineNetGroup
- modifyPipelineNetGroup

createSpareModule

```
createSpareModule
[-help]
-moduleName moduleName
-cell {cellName [number] [cellName [number] ] …}
[-clock netName]
[-reset netName:pinName [pinName] …]
[-tie {tieCellName [tieCellname] } ]
[-tieLo {pinList | *} ]
[-useCellAsPrefix]
```

Creates a spare module made up of the specified cells.

Parameters

`-cell cellName {[number] [cellName [number] ]}`	Specifies the names of the cells, and the number of each, to add to the spare module. If you do not specify the number of cells to add, the `setTieHiLoMode -maxFanOut` parameter controls the number of cells.
`-clock netName`	Specifies a clock net to connect to the clock pins of the instances in the spare module.
`-help`	Outputs a brief description that includes type and default information for each `createSpareModule` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man createSpareModule`.
`-moduleName moduleName`	Specifies a name for the spare module.
`-reset netName:pinName [pinName]`	Specifies a net and the pins to connect to the reset pins of sequential spare cells in the spare module. If you do not specify this parameter, these reset pins are not connected.
`-tie tieCellName [tieCellName]`	Specifies tie-high and tie-low cells to add to the spare module. The software recognizes whether the pins are high or low.
`-tieLo {pinList | *}`	Specifies the pins to connect to the specified tie-low cell. Pins not specified here will be connected to `1'b1` or the tie-high cell. If no tie-low cells are specified with the `-tie` parameter, it connects the pins to `1'b0`. Use the following format for the pin list: • *pinList*: {*cellName*:*pin*} … • `*` : Specify the pin name and it could be *.
`-useCellAsPrefix`	Adds the master cell name as a prefix to the name of gates that are instantiated inside the module. The software adds `spr_gate_n` as a prefix by default; when you specify `-useCellAsPrefix`, it prepends the master cell name to the default prefix.

Examples

- The following command creates spare module `mod1` with cells `cellA`, `cellB`, and `tiehi4`:

  ```
  createSpareModule -cell {cellA cellB} -tie tiehi4 -moduleName mod1
  ```

- The following command creates spare module `mod1` with the following gates: `and_spr_gate_1`, `and_spr_gate_2`, `nand_spr_gate_1`, `nor_spr_gate_1`:

  ```
  createSpareModule -cell {and 2 nand nor} -moduleName mod1 -useCellAsPrefix
  ```

Related Information

- `deleteSpareModule`
- `placeSpareModule`

delete_cell_stack_area

```
delete_cell_stack_area
[-help]
[-cell cell_name]
[-group group_name]
```

Deletes the cell stack area definition.

Parameters

`-cell cell_name`	Specifies the cell(s) for which the cell stack area definition needs to be deleted.
`-group group_name`	Specifies the group(s) for which the cell stack area definition needs to be deleted.
`-help`	Outputs a brief description that includes type and default information for each `delete_cell_stack_area` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_cell_stack_area`

Related Information

- `report_cell_stack_area`
- `specify_cell_stack_area`

delete_cell_stack_group

```
delete_cell_stack_group
[-help]
[-name group_name]
```

Deletes cell stack group definition.

Parameters

`-name group_name`	Specifies the group(s) for which the cell stack group definition is to be deleted.
`-help`	Outputs a brief description that includes type and default information for each `delete_cell_stack_group` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_cell_stack_group`

Related Information

- `report_cell_stack_group`
- `specify_cell_stack_group`

delete_cell_virtual_align

```
delete_cell_virtual_align
[-help]
[-cell cells]
```

Deletes the cells that need to be virtually aligned.

Parameters

-cell *cells*	Specifies the lib cells that require virtual alignment.
-help	Outputs a brief description that includes type and default information for each `delete_cell_virtual_align` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_cell_virtual_align`

Related Information

- `report_cell_virtual_align`
- `specify_cell_virtual_align`

delete_inst_space_group

```
delete_inst_space_group
[-help]
groupName
```

Deletes instance space group by a specific group name.

Parameters

groupName	Specifies the name of the group.
-help	Outputs a brief description that includes type and default information for each `delete_inst_space_group` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_inst_space_group`.

Related Topics

- `create_inst_space_group`
- `report_inst_space_group`

delete_pg_keepout

```
delete_pg_keepout
[-help]
{-cell lib_cell_name | -inst instance_name | -all }
[-layer layer_name | -layerId layer_id]
```

Deletes PG keep out boxes in design for user defined objects.

Parameters

`-all`	Removes all PG keepout boxes in design.
`-cell` *`lib_cell_name`*	Specifies the lib cell to remove the PG keepout box applied to the specified cell.
`-help`	Outputs a brief description that includes type and default information for each `delete_pg_keepout` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_pg_keepout`
`-inst` *`instance_name`*	Specifies the instance to remove the PG keepout box applied to the specified instance.
`-layer` *`layer_name`*	Specifies the names of the layers to delete the PG keepout box applied on the specified metal layers.
`-layerId` *`layer_id`*	Specifies the layer IDs to delete the PG keepout box applied on the specified metal layer.

Related Information

- `report_pg_keepout`
- `specify_pg_keepout`

deleteAllCellPad

`deleteAllCellPad`

Removes placement padding specified by `specifyCellPad` for all cells. If you run this command before running `checkPlace`, `checkPlace` does not consider the cell padding when calculating effective utilization.

Parameters

None

Examples

The following set of commands specifies the cell padding for `HDSDFRSPQ1` and `HDSDFRSPQ2`, and then deletes all padding by using `deleteAllCellPad`:

```
specifyCellPad HDSDFRSPQ1 2
specifyCellPad HDSDFRSPQ2 2
deleteAllCellPad
```

Related Information

- `checkPlace`
- `specifyCellPad`
- Placing the Design chapter in the *Innovus User Guide*
    - Adding Padding

deleteAllScanCells

```
deleteAllScanCells
[-help]
```

Unassigns all assigned scan cells. This command can be called after importing the design if there are scan cell specifications in library; or else it can be used after the `specifyScanCell` command.

Parameters

None

Example

The following set of commands deletes all scan cells after loading design:

```
restoreDesign init.enc.dat top
deleteAllScanCells
```

Related Information

- Placing the Design chapter in the *Innovus User Guide*
    - Optimizing and Reordering Scan Chains

deleteCellEdgeSpacing

```
deleteCellEdgeSpacing
[-help]
edgeType1
edgeType2
```

Deletes the spacing rule between two edge types. The edge types specified must be predefined.

Note: This command only deletes the cell edge constraint specified by Tcl and does not delete the edge spacing defined in LEF.

Parameters

edgeType1	Specifies edge type1 for edge spacing.
edgeType1	Specifies edge type2 for edge spacing.
-help	Outputs a brief description that includes type and default information for each `deleteCellEdgeSpacing` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deleteCellEdgeSpacing`.

Related Information

- `report_cell_edge_spacing`
- `specifyCellEdgeSpacing`

deleteCellEdgeType

```
deleteCellEdgeType
[-help]
-cell cellName
-edgeType edgeType
```

Deletes the cell edge type constraint of the specified cell.

Parameters

`-cell cellName`	Specifies cell name to remove constraint.
`-edgeType edgeType`	Specifies the name of edge type to be removed.
`-help`	Outputs a brief description that includes type and default information for each `deleteCellEdgeType` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deleteCellEdgeType`.

Related Information

- `report_cell_edge_type`
- `specifyCellEdgeType`

deleteCellPad

```
deleteCellPad
[-help]
cellNames
```

Deletes the padding for a cell or list of cells. If a regular expression or a list of cells is provided, this command returns the number of cells for which the padding was deleted.

Parameters

cellNames	Specifies the cell name for which padding has to be deleted. The cell name can be a regular expression, or a list of cell names or expressions.
-help	Outputs a brief description that includes type and default information for each `deleteCellPad` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man deleteCellPad`.

Examples

- The following command deletes padding for the cells with prefix `FF_SCAN`:
  ```
  deleteCellPad FF_SCAN*
  ```
- The following command deletes padding for all cells that have names starting with `BUF`, `INV`, and `DLY`. If a matched cell name does not have padding information, it is ignored without any message.
  ```
  deleteCellPad {BUF* INV* DLY*}
  ```
- The following command deletes padding for cell `FF_SCAN1` and cell `FF_SCAN2`:
  ```
  deleteCellPad {FF_SCAN1 FF_SCAN2}
  ```

Related Information

- `reportCellPad`
- `specifyCellPad`

deleteFiller

```
deleteFiller
[-cell fillerCellList]
[-area {llx lly urx ury}]
[-inst instanceName]
[-prefix prefix]
[-keepFixed]
```

Removes physical cell instances, such as filler cells, end-cap cells, and well-tap cells, from standard cell rows. This command can be called if there are filler cells, end-cap cells, or well-tap cells added in the design.

⚠ Physical cells should not be removed with the `<Delete>` key. If you delete a cell using the `<Delete>` key, the cell will become unplaced. The cell will not be displayed in the layout, but it will still be part of the design. You should remove a physical cell using the `deleteFiller` command.

ⓘ Some `deleteFiller` parameters can also be specified with `setFillerMode`. However, the parameters you specify with `deleteFiller` apply to the current `deleteFiller` run only, and supersede parameters set by `setFillerMode`. For more information, see `setFillerMode`.

Parameters

`-area {llx lly urx ury}`	Specifies the bounding box within which the filler cells are deleted. The argument type is a box, in microns. **Note:** Only those filler cells that are completely contained within the area will be deleted. the command does not delete filler cells that cross the area boundary.
`-cell` *`fillerCellList`*	Specifies a cell name or list of cells to delete. Separate cell names with spaces. **Note:** Cells specified by this parameter override the list of cells provided by the `-core` parameter of `setFillerMode`.
`-keepFixed`	Controls the command to keep the End-cap and Well-tap cells that are marked FIXED. By default, those cells will be deleted if this option is not specified. Deletes End-cap and Well-tap cells by default, though they are marked as `FIXED`. *Default:* `true`
`-prefix` *`prefix`*	Specifies the prefix of the cell instances to delete. For example, to delete all filler cell instances with the prefix `FILL`, type the following command: `deleteFiller -prefix FILL` The software will remove all physical instances whose name starts with `FILL`, including instances named `FILL_1`, `FILLTEMP_1`, and so on. Default: `FILLER`
`-inst` *`instanceName`*	Specifies the instance of the cell to delete. You can use wildcards (`*?`) with this parameter. **Note:** You can specify this parameter with this command or with `addFiller`, and not with `setFillerMode`.

Examples

- The following command deletes all physical-only cells from the design:
  ```
  deleteFiller -cell xyz
  ```
- The following command deletes only the `xyz` cells whose instance names have a `wtap_even` prefix. If well-tap cells in even rows have a `wtap_even` prefix and well-tap cells in odd rows have a different prefix, the cells in rows `2`, `4`, `6`, and so on are deleted.
  ```
  deleteFiller -cell xyz -prefix wtap_even
  ```
- The following command deletes all added cell instances with the prefix instance name `FILLER`:
  ```
  deleteFiller -prefix FILLER
  ```
- The following command deletes all instances of filler cell `FILL2`, end-cap `ecp17`, and well-tap `wtp5`:
  ```
  deleteFiller -cell FILL2 ecp17 wtp5
  ```
- The following command deletes all instances of given the spacer cells from the design:
  ```
  deleteFiller -cell FILL64 FILL32 Fill8 FILL16 FILL2 FILL1
  ```
- The following command deletes all well-tap and end-cap cell instances from the design:
  ```
  deleteFiller -cell WELLTAP PRECAP POSTCAP
  ```
- The following command deletes all physical cells with prefix `WELLTAP` in the specified bounding box of {0 0 300 300} except those cells with `FIXED` status:
  ```
  deleteFiller -prefix WELLTAP -area {0 0 300 300} -keepFixed
  ```
- The following command deletes all physical cells with prefix WELLTAP in the specified bounding box of `{0 0 300 300}`, except those cells with `FIXED` status:
  ```
  deleteFiller -prefix WELLTAP -area {0 0 300 300} -keepFixed
  ```

Related Information

- `addFiller`
- `setFillerMode`

deleteGAFillerGroup

```
deleteGAFillerGroup
[-help]
groupName
```

Deletes the gate array filler groups created by `createGAFillerGroup`.

Parameters

groupName	Specifies the name of the group to be deleted. The * wildcard is supported.

Related Information

- `createGAFillerGroup`
- `ecoPlace`

deleteInstPad

```
deleteInstPad
[ instName | -all]
-suppress
```

Deletes the padding for the specified instances, or all instance-based padding, in the design.

Parameters

instName	Deletes the padding on the specified instance.
-all	Deletes all instance-based padding.
-suppress	Suppresses verbose messages. The option is of type boolean and is optional.

Examples

- The following command deletes padding for all instances in the design:
  ```
  deleteInstPad -all
  ```

  It outputs following:

  ```
  ...

  Delete Padding for Instant [U27312].

  Delete Padding for Instant [U27313].

  Delete Padding for Instant [U27314].

  Delete Padding for Instant [U27315].

  16432
  ```

- The following command deletes padding for one instances named `U27316`:
  ```
  deleteInstPad U27316
  ```

  It outputs following:

  ```
  Delete Padding for Instant [U27316].
  ```

Note: `deleteInstPad` deletes padding for either all instances or single instance. Wildcard is not supported yet.

Related Information

- `reportInstPad`
- `specifyInstPad`
- Placing the Design chapter in the *Innovus User Guide*

- Adding Padding

deleteMimCap

```
deleteMimCap
[-help]
[{[-cell cellName] [-prefix prefixName]} [-area llx lly urx ury]]
```

Deletes the mim caps from the design based on cell type or prefix. This command can be used after `addMimCap`.

Parameters

`-area llx lly urx ury`	Deletes only the Mim caps that are in the specified box. The `-cell` or `-prefix` options are required when this option is specified.
`-cell cellType`	Deletes the mim caps by cell type.
`-help`	Outputs a brief description that includes type and default information for each `deleteMimCap` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deleteMimCap`.
`-prefix prefixName`	Deletes the mim caps by prefix.

Examples

- The following command deletes the mim cap cell `mim_cell1`:

  ```
  deleteMimCap -cell mim_cell1
  ```

Related Information

- `addMimCap`

deletePipelineNetGroup

```
deletePipelineNetGroup
[-help]
netGroupName
```

Deletes pipeline net groups including the subgroups created inside the net groups.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `deletePipelineNetGroup` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man deletePipelineNetGroup`.
`netGroupName`	Specifies the name of the net group.

Example

- The following command deletes a net group named `` `netGroup1' ``:

  ```
  deletePipelineNetGroup netGroup1
  ```

Related Information

- `createPipelineNetGroup`
- `modifyPipelineNetGroup`
- `reportPipeline`

deleteScanCell

```
deleteScanCell
[-help]
[cellName]
```

Removes the scan cell specification from a library cell. This command can be called after importing design if there are scan cell specifications in library; or else it can be used after `specifyScanCell`.

Parameters

cellName	Specifies the cell from which to remove the scan specification.

Example

The following command removes the scan specification from the cell named `FF_SCAN`:

```
deleteScanCell FF_SCAN
```

Related Information

- `reportScanCell`
- `specifyScanCell`
- Placing the Design chapter in the *Innovus User Guide*
  - Optimizing and Reordering Scan Chains

deleteScanChain

```
deleteScanChain
[-help]
[chainName]
```

Removes a scan chain specification from the design. Use this command if there is scan chain specified or traced.

When you are debugging a new scan chain specification, it is useful to delete all scan chains and start over.

Parameters

chainName	Specifies the name of the scan chain to remove from the design. If you do not specify a scan chain name, the command deletes all scan chains.

Example

The following commands load the scan information from a DEF file, remove the scan chain ABC, and output the scan information to a DEF file:

```
defIn -scanChain
deleteScanChain ABC
defOut -scanChain
```

Related Information

- defIn
- defOut
- displayScanChain
- specifyScanChain
- report_scan_chain
- Placing the Design chapter in the *Innovus User Guide*
  - Optimizing and Reordering Scan Chains

deleteScanChainPartition

```
deleteScanChainPartition
[-help]
{-all | -partition partitionName1 partitionName2 ...}
```

Deletes scan chain partitions. The partitions are used to group scan chains so elements in compatible scan chains can be swapped. The scan chains are then reordered on a per-chain basis.

Specify scan chain partitions with the following command: `specifyScanChainPartition`.

Parameters

`-all`	Deletes all partitions from compatible groups.
`-partition partitionName1 partitionName2 …`	Specifies the partitions to delete.

Example

The following command deletes partition `clk1Domain`:

```
deleteScanChainPartition clk1Domain
```

Related Topics

- `reportScanChainPartition`
- `specifyScanChainPartition`
- Placing the Design chapter in the *Innovus User Guide*
    - Optimizing and Reordering Scan Chains

deleteSpareModule

```
deleteSpareModule hInst Name
```

Deletes the instantiation of a created spare module and all the spare instances under the hinst.

Parameters

hInstName	Specifies the instantiation to delete.

Example

The following code shows the creation of the spare module SPGM, placement of the module and its instances in the design, and deletion of the hInst of "SPGM_spr_1":

```
#create a spare module
createSpareModule -moduleName SPGM -cell {DFFNSRX2 INVX4 \ NOR2X2} -clock sysclk

#instantiation the spare module and place them in the design
placeSpareModule -moduleName SPGM -channel -minWidth 4.7 -maxWidth 10 \ -minLength 11 -stepx 10 -stepy 10 -util 0.6

#delete one instantiation of the spare module
deleteSpareModule top/SPGM_spr_1
```

Related Information

- createSpareModule
- placeSpareModule
- Placing the Design chapter in the *Innovus User Guide*
  - Placing Spare Cells and Spare Modules

displayScanChain

```
displayScanChain
[-help]
[chain_name]
```

Displays a scan chain. You must load the scan information, place the design, and view the design in the Placement view. Use this command after scan chains are traced.

Parameters

chain_name	Specifies the scan chain. *Default:* If you do not specify this parameter, the software displays all scan chains.

Examples

- The following command displays all scan chains in the design:
  ```
  displayScanChain
  ```
- The following command displays the scan chain whose name is `chain0`.
  ```
  displayScanChain chain0
  ```

Related Information

- `defIn`
- `defOut`
- `specifyScanChain`
- `report_scan_chain`
- Placing the Design chapter in the *Innovus User Guide*
  - Optimizing and Reordering Scan Chains

displaySpareCell

`displaySpareCell`

Displays the results of placing the spare instances. You must specify the spare cell types or spare instances before placing the design.

Parameters

None

Examples

The following set of commands displays all spare instances defined by the `specifySpareGate` command:

```
specifySpareGate -cell ADDFXLT1T10
place_design
displaySpareCell
```

Related Information

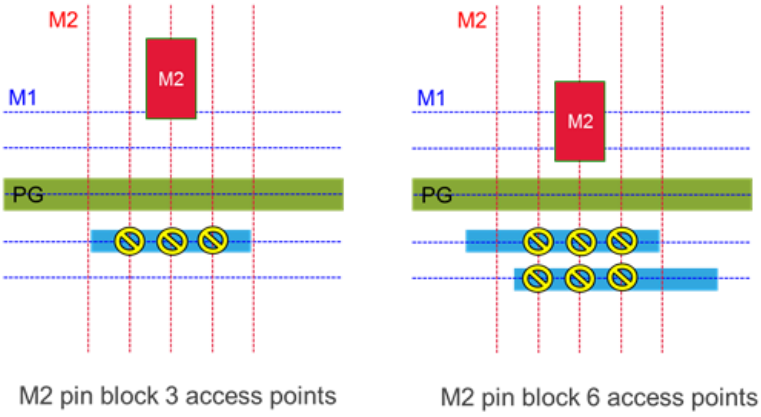
- `place_design`
- `specifySpareGate`

generate_vertical_cell_edge_constraint

```
generate_vertical_cell_edge_constraint
[-help]
{-file tclFile | -updateDB }
[[-check_m2_only ] | [-check_m2_to_m1_only ]]
```

Generates the bottom and top cell edge constraints for 10nm libraries to avoid vertical cell abutment that may cause pin access issue on M1 and M2. The vertical M2 fat PIN or OBS has impact on the abutted cell when it is near to the top and bottom boundaries. It can cause DRC violations between M2 PIN/OBS, block some of the M1 pin access points in the abutted cell, and make router hard to access those M1 PINs.

The following graphic show an example of how fat M2 Pin/OBS may block 3 or 6 access points.



The `generate_vertical_cell_edge_constraint` command generates vertical cell edge constraints based on the cell libraries and LEF rules. Placer is aware of these constraints and avoids the bad placement patterns.

Parameters

`-check_m2_only`	Checks M2 to M2 pins only in abutted 2 cells when generating cell edge constraints.
`-check_m2_to_m1_only`	Checks M2 to M1 pins only in abutted 2 cells when generating cell edge constraints.
`-help`	Outputs a brief description that includes type and default information for each `generate_vertical_cell_edge_constraint` parameter. For a detailed description of the command and all of its parameters, use the man command: `man generate_vertical_cell_edge_constraint`.
`-file tclFile`	Specifies file name to output vertical tcl constraint.
`-updateDB`	Updates the constraints to DB.

Note:

- You must specify `-file` or `-updateDB`.
- The `-file` option is for the debug mode, and you need to source the tcl file to let placer know about the constraints.
- The `-updated` option is for the standard flow usage. It reads the constraints automatically.

get_well_tap_mode

```
 get_well_tap_mode
[-help]
[-avoidAbutment]
[-block_boundary_only]
[-bottom_tap_cell]
[-bottom_termination_cell]
[-cell]
[-channel_offset]
[-check_channel]
[-column_cell]
[-inRowOffset]
[-insert_cells]
[-rule]
[-siteOffset]
[-tap_function_cells]
[-tap_termination_alignment]
[-termination_align]
[-termination_cell]
[-top_tap_cell]
[-top_termination_cell]
[-vertical_boundary_spacing]
[-well_cut_cell]
[-quiet]
[-nonDefault]
```

Returns the information about `set_well_tap_mode` parameters in the Innovus log file and in the Innovus console. Returns the following information:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the software displays values for all of the `set_well_tap_mode` mode parameters.

Related Information

- `set_well_tap_mode`

getDensityMapMode

```
getDensityMapMode
[-gridInMicron]
[-gridInRow]
[-ignoreBlock]
[-ignoreFiller]
[-quiet]
[-threshold]
```

Returns the information about `setDensityMapMode` parameters in the Innovus log file and in the Innovus console. Returns the following information.

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Source of current value (default, user)

If you do not specify a parameter, the software displays values for all of the `setDensityMapMode` mode parameters.

Related Information

- `setDensityMapMode`
- Placing the Design chapter in *Innovus User Guide*.

getEndCapMode

```
getEndCapMode
[-help]
[-avoidTwoSitesCellAbut]
[-bottomEdge]
[-boundary_tap]
[-boundary_tap_swap_flow]
[-cells]
[-create_rows]
[-DoubleHeightIncornerProtrusion]
[-disable_check_zone_at_boundary]
[-flipY]
[-incrementalLeftEdge]
[-incrementalRightEdge]
[-leftBottomCorner]
[-leftBottomCornerEven]
[-leftBottomCornerNeighbor]
[-leftBottomCornerOdd]
[-leftBottomEdge]
[-leftBottomEdgeEven]
[-leftBottomEdgeNeighbor]
[-leftBottomEdgeOdd]
[-leftCornerBottomBorder]
[-leftCornerTopBorder]
[-leftEdge]
[-leftEdgeBottomBorder]
[-leftEdgeEven]
[-leftEdgeOdd]
[-leftEdgeTopBorder]
[-leftTopCorner]
[-leftTopCornerEven]
[-leftTopCornerNeighbor]
[-leftTopCornerOdd]
[-leftTopEdge]
[-leftTopEdgeEven]
[-leftTopEdgeNeighbor]
[-leftTopEdgeOdd]
[-min_horizontal_channel_width]
[-min_jog_height]
[-min_jog_width]
[-min_vertical_channel_width]
[-min_horizontal_channel_width]
[-min_jog_height]
[-min_jog_width]
[-min_vertical_channel_width]
[-prefix]
[-reset]
[-rightBottomCorner]
[-rightBottomCornerEven]
[-rightBottomCornerNeighbor]
[-rightBottomCornerOdd]
[-rightBottomEdge]
[-rightBottomEdgeEven]
[-rightBottomEdgeNeighbor]
[-rightBottomEdgeOdd]
[-rightCornerBottomBorder]
```

```
[-rightCornerTopBorder]
[-rightEdge]
[-rightEdgeBottomBorder]
[-rightEdgeEven]
[-rightEdgeOdd]
[-rightEdgeTopBorder]
[-rightTopCorner]
[-rightTopCornerEven]
[-rightTopCornerNeighbor]
[-rightTopCornerOdd]
[-rightTopEdge]
[-rightTopEdgeEven]
[-rightTopEdgeNeighbor]
[-rightTopEdgeOdd]
[-topBottomEdge]
[-topEdge]
[-useEvenOddSite]
[-wall_keepout_from_vertical_boundary]
[-wall_offset]
[-wall_pitch]
[-wall_to_incorner_spacing]
```

Returns the information about `setEndCapMode` parameters in the Innovus log file and in the Innovus console. Returns the following information:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Source of current value (default, user)

If you do not specify a parameter, the software displays values for all of the `setEndCapMode` mode parameters.

Related Information

- `addEndCap`
- `setEndCapMode`
- `verifyEndCap`

getFillerMode

```
getFillerMode
[add_fillers_with_drc true]
[avoid_abutment_patterns]
[-check_signal_drc]
[-core]
[-corePrefix]
[-createRows]
[-diffCellViol]
[-distribute_implant_evenly]
[-ecoMode]
[-fitGap]
[-honorPrerouteAsObs]
[-horizontal_exception_cell]
[-horizontal_max_length]
[-horizontal_repair_cell]
[-keepFixed]
[-merge]
[-minHole]
[-preserveUserOrder]
[-scheme]
[-swap_cell]
[-vertical_stack_exception_cell]
[-vertical_stack_left_edge_exception_cell]
[-vertical_stack_max_length]
[-vertical_stack_repair_cell]
[-vertical_stack_repair_edge]
[-vertical_stack_right_edge_exception_cell]
[-viaEnclosure]
[-y_flip_type]
[-quiet]
```

Returns the information about `setFillerMode` parameters in the Innovus log file and in the Innovus console. Returns the following information.

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the software displays values for all of the `setFillerMode` mode parameters.

Parameters

parameter_names	Returns information for the specified parameters. You can specify one or more parameters. See `setFillerMode` for parameter descriptions.
`-quiet`	Returns the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software returns the current settings of all `setFillerMode` parameters in Tcl list format.

Examples

- The following command returns the current setting of the `setFillerMode -core` in the Tcl list format:

  ```
  getFillerMode -core -quiet
  ```

  The software returns the following information:

  ```
  {}
  ```

- The following command returns the current settings for the `-core` and `-corePrefixFILLER`:

  ```
  getFillerMode -core -corePrefix
  ```

  The software returns the following information:

  ```
  -core {} # string, default=""

  -corePrefix FILLER # string, default="",user setting

  {core {}} {corePrefix FILLER}
  ```

- The following command returns the current settings for `setFillerMode-core` and `-corePrefix` in Tcl list format:

  ```
  getFillerMode -core -corePrefix -quiet
  ```

  The software returns the following information:

  ```
  {core {}} {corePrefix FILLER}
  ```

- The following command returns the current settings for all `setFillerMode` parameters in Tcl list format:

  ```
  getFillerMode -quiet
  ```

  The software returns the following information:

  ```
  {add_fillers_with_drc true} {avoid_abutment_patterns {}} {check_signal_drc true} {core {}} {corePrefix {}}
  {createRows true} {diffCellViol false} {distribute_implant_evenly false} {ecoMode false} {fitGap true}
  {honorPrerouteAsObs false} {horizontal_exception_cell {}} {horizontal_max_length 0} {horizontal_repair_cell {}}
  {keepFixed false} {preserveUserOrder false} {scheme locationFirst} {swap_cell {}} {vertical_stack_exception_cell
  {}} {vertical_stack_left_edge_exception_cell {}} {vertical_stack_max_length 0} {vertical_stack_repair_cell {}}
  {vertical_stack_repair_edge ""} {vertical_stack_right_edge_exception_cell {}} {y_flip_type {}}
  ```

Related Information

- addFiller
- `deleteFiller`
- `setFillerMode`

getPinDensityMapMode

```
getPinDensityMapMode
[-gridInMicron]
[-gridInRow]
[-threshold]
[-quiet]
```

Returns the information about `setPinDensityMapMode` parameters in the Innovus log file and in the Innovus console. Returns the following information:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Source of current value (default, user)

If you do not specify a parameter, the software displays values for all of the `setPinDensityMapMode` mode parameters.

Related Information

- reportPinDensityMap
- `setPinDensityMapMode`

getPlaceMode

```
getPlaceMode
[-place_design_floorplan_mode]
[-place_design_integrity_ir_fix_effort]
[-place_design_refine_macro]
[-place_design_refine_place]
[-place_detail_activity_power_driven]
[-place_detail_allow_border_pin_abut]
[-place_detail_allow_single_height_row_symmetry_x]
[-place_detail_check_cut_spacing]
[-place_detail_check_inst_space_group]
[-place_detail_check_route]
[-place_detail_color_aware_legal]
[-place_detail_context_aware_legal]
[-place_detail_eco_max_distance]
[-place_detail_eco_priority_insts]
[-place_detail_fixed_shifter]
[-place_detail_honor_inst_pad]
[-place_detail_io_pin_blockage]
[-place_detail_iraware_max_drive_strength]
[-place_detail_irdrop_aware_timing_effort]
[-place_detail_irdrop_region_number]
[-place_detail_irdrop_aware_effort]
[-place_detail_legalization_inst_gap]
[-place_detail_max_shifter_column_depth]
[-place_detail_max_shifter_depth]
[-place_detail_max_shifter_row_depth]
[-place_detail_no_filler_without_implant]
[-place_detail_pad_fixed_insts]
[-place_detail_preroute_as_obs]
[-place_detail_preserve_routing]
[-place_detail_remove_affected_routing]
[-place_detail_swap_eeq_cells]
[-place_detail_use_check_drc]
[-place_detail_use_diffusion_transition_fill]
[-place_detail_use_GA_filler_groups]
[-place_detail_use_no_diffusion_one_site_filler]
[-place_detail_wire_length_opt_effort]
[-place_global_activity_power_driven]
[-place_global_activity_power_driven_effort]
[-place_global_align_macro]
[-place_global_allow_3d_stack]
[-place_global_auto_blockage_in_channel]
[-place_global_clock_gate_aware]
[-place_global_clock_power_driven]
[-place_global_clock_power_driven_effort]
[-place_global_cong_effort]
[-place_global_cpg_effort]
[-place_global_cpg_file]
[-place_global_enable_advanced_pipeline]
[-place_global_enable_distributed_place]
[-place_global_group_flop_to_macro]
[-place_global_group_flop_to_macro_level]
[-place_global_group_flop_to_macro_list]
[-place_global_ignore_scan]
[-place_global_ignore_spare]
```

```
[-place_global_max_density]
[-place_global_module_aware_spare]
[-place_global_module_padding]
[-place_global_place_io_pins]
[-place_global_reorder_scan]
[-place_global_soft_guide_strength]
[-place_global_timing_effort]
[-place_global_uniform_density]
[-place_hard_fence]
[-place_hierarchical_flow]
[-place_opt_post_place_tcl]
[-place_opt_run_global_place]
[-place_spare_update_timing_graph]
[-quiet]
```

Displays the information about `setPlaceMode` parameters in the Innovus log file and in the Innovus console. Displays the following information:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the Innovus software displays values for all of the `setPlaceMode` mode parameters.

Parameters

parameter_names	Returns information for the specified parameters. You can specify one or more parameters. See `setPlaceMode` for description of parameters.
`-quiet`	Returns the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software returns the current settings of all `setPlaceMode` parameters in Tcl list format.

Examples

- The following command returns the current setting for the `-place_global_place_io_pins` parameter in Tcl list format:

  ```
  getPlaceMode -place_global_place_io_pins -quiet
  ```

  The software returns the following information:

  ```
  false
  ```

- The following command returns the current settings for the `-place_global_place_io_pins` and -place_global_solver_effort parameters:
  getPlaceMode -place_global_place_io_pins -place_global_solver_effort

  The software returns the following information:

  ```
  -place_global_place_io_pins           # enums={false true}, default=false
  -place_global_solver_effort high        # enums={medium high}, default=high
  ```

```
{-place_global_place_io_pins false} {place_global_solver_effort high}
```

- The following command returns the current settings for all `setPlaceMode` parameters in Tcl list format:

  ```
  getPlaceMode -quiet
  ```

  The software returns the following information:

  {place_design_floorplan_mode false} {place_design_refine_place true} {place_detail_allow_border_pin_abut false} {place_detail_check_cut_spacing false} {place_detail_check_route false} {place_detail_color_aware_legal false} {place_detail_context_aware_legal all} {place_detail_eco_max_distance 0} {place_detail_eco_priority_insts placed} {place_detail_fixed_shifter false} {place_detail_hard_fence true} {place_detail_honor_inst_pad false} {place_detail_io_pin_blockage false} {place_detail_legalization_inst_gap 0} {place_detail_max_shifter_column_depth 9999} {place_detail_max_shifter_depth 9999} {place_detail_max_shifter_row_depth 9999} {place_detail_no_filler_without_implant false} {place_detail_pad_fixed_insts false} {place_detail_preroute_as_obs {}} {place_detail_preserve_routing false} {place_detail_remove_affected_routing false} {place_detail_swap_eeq_cells false} {place_detail_use_check_drc false} {place_detail_use_GA_filler_groups false} {place_detail_use_no_diffusion_one_site_filler false} {place_detail_wire_length_opt_effort medium} {place_global_auto_blockage_in_channel none} {place_global_clock_gate_aware true} {place_global_clock_power_driven false} {place_global_cong_effort auto} {place_global_enable_distributed_place false} {place_global_group_flop_to_macro false} {place_global_group_flop_to_macro_level 1} {place_global_group_flop_to_macro_list {}} {place_global_ignore_scan true} {place_global_ignore_spare false} {place_global_max_density -1} {place_global_module_aware_spare false} {place_global_module_padding {}} {place_global_place_io_pins false} {place_global_reorder_scan true} {place_global_soft_guide_strength low} {place_global_solver_effort high} {place_global_timing_effort medium} {place_global_uniform_density false} {place_opt_post_place_tcl {}}

Related Information

- `checkPlace`
- `finishFloorplan`
- place_design
- `place_opt_design`
- `placeSpareModule`
- `refinePlace`
- `setOptMode`
- `setPlaceMode`
- `specifyInstPad`
- `specifyScanCell`
- `specifySpareGate`
- Using the Mixed Placer chapter of the *Innovus User Guide*.

getScanReorderMode

```
getScanReorderMode
[-addScanPortPrefix]
[-allowSwapping]
[-clkAware]
[-compLogic {true | false}]
[-enable_for_partition]
[-keepPDPorts]
[-keepPort]
[-preferH]
[-preferV]
[-scanEffort]
[-skipMode]
[-quiet]
```

Returns the information about `setScanReorderMode` parameters in the Innovus log file and in the Innovus console. Returns the following information:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Source of current value (default, user)

If you do not specify a parameter, the Innovus software returns values for all of the `setScanReorderMode` mode parameters.

Parameters

parameter_names	Returns information for the specified parameter. You can specify one or more parameters. See `setScanReorderMode` parameter descriptions.
`-quiet`	Returns the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software returns the current settings of all `setScanReorderMode` parameters in Tcl list format.

Examples

- The following command returns the current setting for the `-keepPDPorts` parameter in Tcl list format:

  ```
  getScanReorderMode -keepPDPorts -quiet
  ```

  The software returns the following information:

  ```
  false
  ```

- The following command returns the current settings for the `-keepPDPorts` and `-preferH` parameters:

  ```
  getScanReorderMode -keepPDPorts -preferH
  ```

  The software returns the following information:

  ```
  -keepPDPorts false # bool, default=false
  ```

```
-preferH false # bool, default=false

{keepPDPorts false} {preferH false}
```

- The following command returns the current settings for the `-keepPDPorts` and `-preferH` parameters in Tcl list format:

  ```
  getScanReorderMode -keepPDPorts -preferH -quiet
  ```

  The software returns the following information:

  ```
  {keepPDPorts false} {preferH false}
  ```

- The following command returns the current settings for all `setScanReorderMode` parameters in Tcl list format:

  ```
  getScanReorderMode -quiet
  ```

  The software returns the following information:

  ```
  {allowSwapping false} {clkAware false} {keepPDPorts true} {keepPort (false)} {preferH false} {preferV false}
  {scanEffort medium} {skipMode skipNone}
  ```

Related Information

- `setScanReorderMode`
- `scanReorder`
- Placing the Design chapter in the *Innovus User Guide*
  - Optimizing and Reordering Scan Chains

modifyPipelineNetGroup

```
modifyPipelineNetGroup
netGroupName
[{-inst instNameListOrRegularExp |
      -startTerm termList -endTerm termList |
      -net listOfNetnamesOrRegularExp }]
[-startReserve distInMicrons] [-endReserve distInMicrons]
```

Modifies the existing net group attributes or converts an existing net group added during floorplanning to be of type pipeline. This command can be used if there are existing pipe line net groups in the design.

Parameters

`-endReserve` *ReserveDistanceInMicrons*	Specifies a distance value from the end points on block boundary to the internal connection inside the block that needs to be added to the last stage distance, to achieve equidistant spacing. The unit is in micron.
`-inst` *instNameListOrRegularExp*	Modifies net group to add all nets connected to the given instances. The instances must all be flip-flops or latches. All the nets attached to the D and Q pins are put into the PFF net group.
`-net` *listOfNetNamesOrRegularExp*	Creates net group of all given nets.
`-startReserve` *ReserveDistanceInMicrons*	Specifies a distance budget from internal connections inside block to the start point of the PFF net. This value is added to the first stage distance when calculating equidistant spacing for each stage. The unit is in micron.
`-startTerm` *termList*	Modifies net group to add all data nets between the given end points. The PFF-chain is traced from the startTerm, through D and Q terminals of PFF(s) until an endTerm is matched.
`-endTerm` *termList*	Modifies net group to add all data nets between the given end points. The PFF-chain is traced from the startTerm, through D and Q terminals of PFF(s) until an endTerm is matched.
`netGroupName`	Specifies the name of net group.

Example

The following command modifies the existing net group named `netGroup1` between `blockA/DATA_OUT*` and `blockB/DATA_IN*` with a budget distance 10 microns from internal connections inside the blockA to the start point of `DATA_OUT*` :

```
modifyPipelineNetGroup netGroup1 -startTerm blockA/DATA_OUT* -endTerm blockB/DATA_IN* -startReserve 10
```

Related Information

- `createPipelineNetGroup`
- `deletePipelineNetGroup`
- `reportPipeline`

pack_align_macros

```
pack_align_macros
[-help]
```

Pushes macros (or pack of macros) to core corners and marks them fixed.

The `pack_align_macros` command executes the following two steps:

- Aligns macros of the same size (initially placed together) on the entire design.
- Pushes macros (or pack of macros) to core corners and marks them fixed.

The `pack_align_macros` command honors the following rules of `set_macro_place_constraint`.

- Min space to macro
- Min space to core
- Forbidden space to macro
- Forbidden space to core
- Avoid abut macro edge with pins
- Macro corner keepout

Figure 1: Macros before running `pack_align_macros`

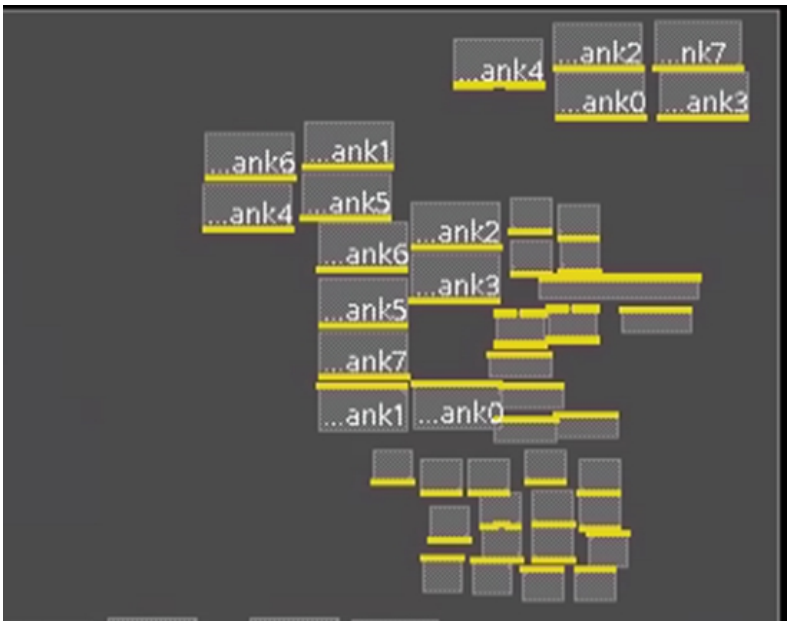
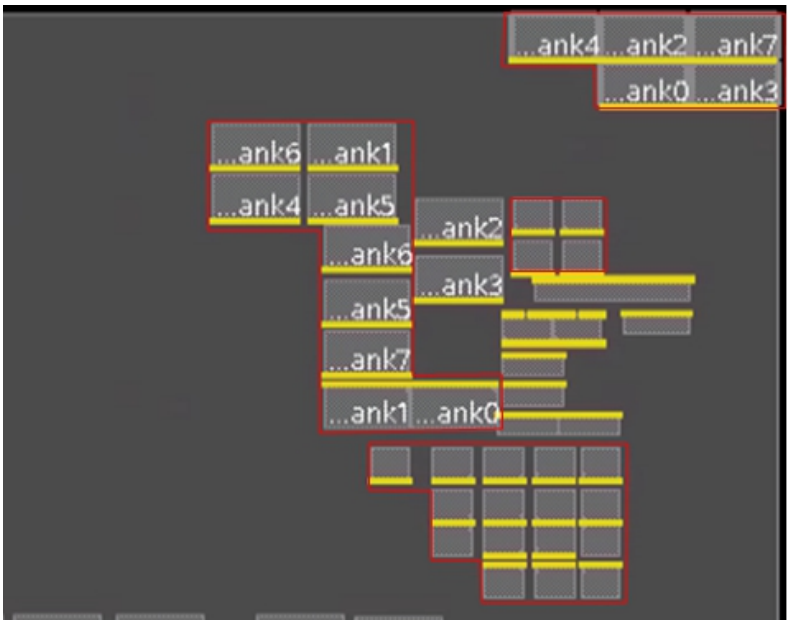
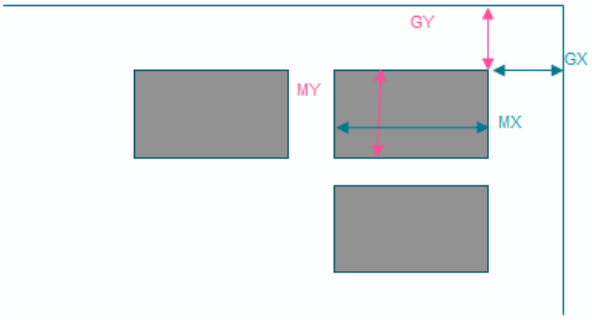


Figure 2: Macros aligned after running `pack_align_macros`



The Figure 1 and Figure 2 show a design before and after running the `pack_align_macros`, respectively. The Figure 2 shows that the push till the core corner takes place when the gap between the core corner and the corner of the nearest macro satisfies the following condition:
GY <= MY And GX <= MX



Parameters

None

place_connected

```
place_connected
[-help]
-attractor macro/IO list
[-fixed]
[-move_fixed]
[-placed]
{-level integer  | -sequential all_connected | direct_connected | -stop_points instance_name | -before_points
instance_name | -instance_pin instance/pin}
[{ -stop_points instance_name | -key_cell cell_name }]
[{ -instance_pin instance/pin | -key_cell cell_name }]
[{ -attractor_pin pin_list } [-instance_pin instance/pin ]]
```

Places the specified standard cells close to the specified attractor with legal location. The attractor can be IOs, hard macros, fixed standard cell. The recommended flow is to run this command before `place_design`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each place_connected parameter.
`-attractor` *`macro/IO list`*	Specifies target hard macros, IOs, or fixed standard cell for attracting standard cells. Wildcards are supported. Tool processes attractors in list one by one. If there are instances shared by different attractors, the locations of shared instances are determined by the later one. In that case, the placement result may vary with different order of objects in list.
`-attractor_pin` *`pin_list`*	Places only standard cells connected with specified pins of attractors close to attractor. Wildcards are supported. Without this setting, standard cells connected with all pins of attractors are pulled.
`-before_points` *`instance_name`*	The attractor stops attracting before these instances.
`-fixed`	Sets the placement status as fixed.
`-instance_pin` *`instance/pin`*	Pulls all instances, instance pins, and IO pins specified in the list close to the attractor. This option requires only full instance pin name. Wildcards are not supported. Option `-attractor_pin` needs to be used with this option.
`-key_cell` *`cell_name`*	Places specified type of cells close to attractors, as well as other logics between the key cell and attractor. Wildcards are supported. Without this setting, all standard cells connected with attractors are pulled.
`-level` *`integer`*	Pulls all instances within certain logic level of connections to the attractor close to attractors. This option is mutually exclusive with -sequential, and user must specify one of them.
`-move_fixed`	Specifies that instances with the fixed status can also be moved close to the attractor.
`-placed`	Specifies instances being placed is in “placed” status. By default, the pulled cells are marked as “softfixed”.
`-sequential` *`all_connected`* | *`direct_connected`*	Pulls sequential cells connected with the attractor close to the attractor. `all_connected`: attracts all sequential cells from/to attractor; combinational logics between the sequential instance and attractor are attracted as well. `direct_connected`: attracts sequential cells directly connected with attractor.
`-stop_points` *`instance_name`*	Pulls all instances connected between the attractor and the specified stop point close to attractors. This option requires full instance name or instance pin name. Wildcards are supported.

Examples

- The following command places all instances directly connected with all pins of `RAM1` close to `RAM1`.

  ```
  place_connected -attractor RAM1 -level 1
  ```

- The following command pulls all sequential cells from `RAM1` and all combinational cells between `RAM1` and the sequential cells close to `RAM1`.

  ```
  place_connected -attractor RAM1 -sequential all_connected
  ```

- Only the sequential cells from “IN*” pins of `RAM1` and their combinational logics are caught to be placed nearby `RAM1`.

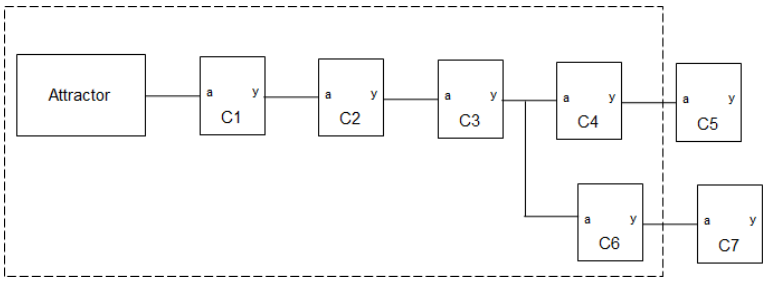
  ```
  place_connected -attractor RAM1 -sequential all_connected -attractor_pin IN*
  ```

- The following command picks out instances that are `"BUF1X"` cell type and directly connected with `RAM1`, and places them close to `RAM1` with `"placed"` status.

  ```
  place_connected -attractor RAM1 -level1 -key_cell BUF1X -placed
  ```

- The following command places instances, C1, C2, C3, C4, C6, close to the attractor RAM1.

  ```
  place_connected -attractor RAM1 -before_points {C5 C7}
  ```



Related Information

- `place_design`

place_design

```
place_design
[-help]
[-concurrent_macros]
[-incremental]
[-noPrePlaceOpt]
[-sdp]
```

Places standard cells based on the global settings for placement, RC extraction, timing analysis, and early global routing. It also relieves the congestion and reorders the scan cells. Also performs the following operations:

- Pre-placement optimization
- Scan tracing and scan reordering
- Placement optimization during the placement process
- Incremental placement on an already placed design

By default, `place_design` performs timing-driven placement with scan reordering, except during prototyping, when it runs in non-timing-driven mode. To change this behavior, run `setPlaceMode` with the following parameters prior to running `place_design`:

- `-place_global_reorder_scan false`
- `-place_global_timing_effort false`

Or, in prototyping mode:

- `-place_design_floorplan_mode`

If there are no timing constraints, `place_design` assumes non-timing-driven mode.

Note: This command supports multiple-CPU processing. For information, see "Multiple-CPU Processing Commands" and "Accelerating the Design Process by Using Multiple-CPU Processing".

Notes:

- The `get_metric` command can be used with the `place_design` command to retrieve metrics.
- The `place_design` command:
- Supports multithreading
- Honors the `set_proto_mode -place_effort` setting
- Honors instance padding constraint as a hard rule if `setPlaceMode -place_detail_honor_inst_pad` is set to `true`.
- Honors settings for the following commands with the exceptions noted in the table below:

Command	Exceptions/Notes
`setAnalysisMode`	• Does not honor `-checkType hold`. If you specify `-checkType` hold, issues a warning and assumes `-checkType setup` mode. • Does not honor `-clockPropagation sdcControl`. If you specify `-clockPropagation sdcControl`, assumes `-clockPropagation forcedIdeal`. • Does not honor `-clkSrcPath true`. If you specify `-clkSrcPath true`, assumes `-clkSrcPath false`.
`setExtractRCMode`	Does not honor `-detail`. If you specify `-detail`, issues a warning and assumes `-default` mode.

setOptMode	All parameters honored
setPlaceMode	If you specify -place_global_clock_gate_aware, you must specify a clock specification file before running place_design.
setScanReorderMode	You must specify scan chain information or import a scan DEF file or the software issues a warning message and skips scan reordering.
setRouteMode	All parameters honored

Parameters

-help	Outputs a brief description that includes type and default information for each place_design parameter. For a detailed description of the command and all of its parameters, use the man command: man place_design.

-concurrent_macros	When this option is enabled, place_design will do concurrent timing driven placement of macros and standard cells. By default, the place_design -concurrent_macros command does not call the refine_macro_place command to legalize the macros. It is recommended to check the "brut" mixed placer result before invoking the refine_macro_place command. When the place_design -concurrent_macros command is run, it sets the value of setPlaceMode -place_opt_run_global_place to seed.
	Flow usage is as follows:
	`place_design -concurrent_macros`
	`saveDesign macroPlace.enc`
	`…`
	`refine_macro_place`
	`saveDesign macroRefine.enc`
	`place_opt_design`
	⚠ **Note:** When the -place_global_place_io_pins parameter of the setPlaceMode command is set to true, the place_design -concurrent_macros command places the placed and unplaced I/O pins in the mixed placer flow and optimize their locations to reduce wire length and congestion.
	This support is a limited-access feature in this release. It is enabled by a variable specified using the setLimitedAccessFeature command. To use this feature, contact your Cadence representative to explain your usage requirements, and make sure this feature meets your needs before deploying it widely.

-incremental	Runs placement incrementally. This parameter works only when the design is already placed. **Note:** Using this parameter defaults to -noPrePlaceOpt.
-noPrePlaceOpt	Disables pre-place optimization during the placement run. By default, the command removes all cells with the specified buffer footprint (same as with the deleteBufferTree command), except when using the -incremental parameter.
-sdp	Enables SDP-aware (Structured Data Path) placement.

Command Order

Use the `place_design` command after performing general floorplanning, including macro placement, regioning (if applicable), and power planning on the design. The design could be in the unplaced state, or in a state that the existing placement is to be replaced.

- To run timing-driven placement (the default), make sure the timing constraints are loaded in the design.
- To run scan reorder (the default), specify the scan chain or import the scan DEF data before running this command.

Examples

- The following commands perform non-timing-driven placement:

  ```
  setPlaceMode -place_global_timing_effort false
  place_design
  ```

- The following commands disable scan chain reordering:

  ```
  setPlaceMode -place_global_reorder_scan false
  place_design
  ```

- The following command disables the pre-placed buffer tree removal:

  ```
  place_design -noPrePlaceOpt
  ```

- After the design is placed, the following command performs incremental placement, and disables the pre-placed buffer tree removal:

  ```
  place_design -incremental
  ```

Related Information

- `get_metric`
- `place_connected`
- `refine_macro_place`
- `saveDesign`
- `setAnalysisMode`
- `setExtractRCMode`
- `setOptMode`
- `setPlaceMode`
- `setScanReorderMode`
- `setRouteMode`
- Placing the Design chapter in the *Innovus User Guide*
- Accelerating the Design Process by Using Multiple-CPU Processing chapter in *Innovus User Guide*
- Multiple-CPU Processing Commands chapter of the Innovus System Text Command Reference
- *Multiple CPU Processing* form in the Tools Menu chapter of *Innovus Menu Reference*

place_opt_design

```
place_opt_design
[-help]
[-expanded_views]
[-incremental]
[-incremental_timing]
[-num_paths number_of_paths]
[-out_dir outputDirectory]
[-predict_prects]
[-prefix outputFileName]
[-timing_debug_report]
```

Executes pre-CTS flow with both placement and pre-CTS optimization.

Earlier, users had to perform the following two steps to do placement and preCTS optimization:

1. `place_design`
2. `optDesign -preCTS`

With `place_opt_design`, users need to run only a single command to do placement and preCTS optimization.

The `place_opt_design` command provides better integration between placement and optimization to achieve faster runtime and better PPA.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `place_opt_design` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man place_opt_design`.
`-expanded_views`	Prints timing information for each active view at the end of the command restoring all pruned views if absent.

`-incremental`	Specifies `place_opt_design` to be run in the incremental mode. In the incremental mode `place_opt_design` incrementally improves the existing placement based on the critical timing paths and congestion. The command then re-optimizes the netlist based on the new placement. The optimization in incremental `place_opt_design` does DRV fixing, WNS, and TNS optimization. The input to `place_opt_design -incremental` can be either a partial placement or full placement with placement violations, and the output is a legal placement as `place_detail` is called at the end. This can be used if you want to further improve the congestion and timing QoR or if there is a minor floorplan change and this by default, congestion-driven, and timing-driven. It also honors density screens and floorplan constraints. If you run `place_opt_design` and notice some areas of congestion, you can introduce density screens and run `place_opt_design -incremental`. As `place_opt_design -incremental` honors density screens, it would spread out some of the cells, reducing congestion in that area. In contrast, `place_detail` would not do this as it only works on the hard placement violations such as cell overlaps, while density screens is a "soft" constraint. There is no scan-reordering done in `place_opt_design -incremental`. **Note**: Early clock flow mode `place_opt_design -incremental` reuses the existing early clock flow clock tree without any logical or physical modification and makes adjustments only to useful skew using virtual delays. It is also essential to note that the CCOpt clock tree spec must not be deleted between the original and incremental `place_opt_design` steps, just like it must not be done between `place_opt_design` and `ccopt_design`, otherwise any useful skew will be lost.
`-incremental_timing`	In the `incremental_timing` mode, `place_opt_design` does incremental DRV and timing optimization. In this mode, `place_opt_design` incrementally optimizes the critical timing path. Also, the optimization in incremental timing `place_opt_design` does DRV fixing, WNS, and TNS optimization.
`-num_paths` *`number_of_paths`*	Specifies the number of paths reported in the timing report. By default, all timing paths are reported.
`-out_dir` *`outputDirectory`*	Specifies the output directory in which the timing/drv reports will be saved.
`-predict_prects`	Performs optimization specific to physical synthesis exchange.
`-prefix` *`outputFileName`*	Specifies the file name prefix for the timing/drv reports that are generated at the end of `place_opt_design`.
`-timing_debug_report`	When specified, generates a machine-readable timing debug report after completing placement and preCTS optimization. This report is used for debugging timing results using the timing debug feature in Innovus. **Note**: You can specify the number of paths to be reported using the `-num_paths` parameter.

Related Information

- `optDesign`
- `place_design`
- `setOptMode`
- `setPlaceMode`
- Optimizing Timing chapter in the *Innovus User Guide*
- Placing the Design chapter in the *Innovus User Guide*

placeInstance

```
placeInstance
[-help]
instance_name
location
[orientation]
[{-fixed | -placed | -softFixed}]
```

Places a leaf instance in the core box. Instance is snapped to the site whose lower left corner is nearest to the location specified. If the location specified is outside of the core box, then the instance is snapped to site whose lower left corner is nearest to the location specified. Instance is not checked for legality–it can overlap other boxes, either placed or fixed, and it can be placed within placement blockages. If no orientation is specified, or if an illegal orientation is specified, `placeInstance` will orient the instance with the default orientation of the site in which the instance is being placed. You can check the legality of the placed instance by the `checkPlace` command.

Note: If you place a master instance blackbox to non-R0 orientation, the new non-R0 orientation will be converted to R0 orientation.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `placeInstance` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man placeInstance`.
`instance_name`	Specifies the leaf instance name.
`location`	Specifies x and y coordinates of new origin. • *`x`*: Specifies the lower-left x coordinate of the instance on the x axis. • *`y`*: Specifies the lower-left y coordinate of the instance on the y axis.
`orientation`	Specifies the orientation of the instance. Specify one of the following values: `R0, R90, R180, R270, MX, MX90, MY,` or `MY90`. *`Default`*: `R0`
`-fixed | -placed | -softFixed`	Specifies that the cell being placed is in the `fixed`, `placed`, or `softFixed` status. *`Default`*: `-fixed`

Example

- The following command places the instance `SH22/I40` with mirrored through Y axis orientation at location `1500.3, 980.3` in the floorplan. The instance SH22/I40 is fixed by default and the placement program will not move this instance later:
  ```
  placeInstance SH22/I40 1500.3 980.3 MY
  ```
- The following command places the instance SH22/I51 with default `R0` orientation at location 398.7, 1480.2 in the floorplan. The instance SH22/I52 has 'placed' status and it can be moved during optimization, clock synthesis, or refinePlace:
  ```
  placeInstance SH22/I51 398.7 1480.2 -placed
  ```

Related Information

- `checkPlace`
- Using the Mixed Placer chapter of the *Innovus User Guide*
- ECO Flows chapter of the *Innovus User Guide*
- Floorplanning the Design chapter of the *Innovus User Guide*
  - Orientation Key
- Partitioning the Design chapter of the *Innovus User Guide*
  - Handling of Blackboxes with Non-R0 Orientation

placeJtag

```
placeJtag
[-help]
-nrRow nrRow
[-nrRowTop nrRowTop]
[-nrRowBottom nrRowBottom]
[-nrRowLeft nrRowLeft]
[-nrRowRight nrRowRight]
[-hardMacro {true | false}]
[-areaIo {true | false}]
[-ioNetWeight netWtValue]
[-blockNetWeight netWtValue]
[-contour {true | false}]
[-ignoreScan {true | false}]
[-orientTop orientation]
[-orientBottom orientation]
[-orientLeft orientation]
[-orientRight orientation]
```

Places JTAG cells. Use this command for JTAG and other cells where placement is required to be close to the perimeter I/Os.

Note: The command does not honor MSMV design.

Parameters

`-areaIo {true | false}`	Considers the area around I/O cells as a legal placement location. When specifying this parameter, the `-ioNetWeight` parameter is applied to the nets that are connected to the Area I/O cells. Default: `false`
`-blockNetWeight netWtValue`	Specifies the weight for nets connecting to blocks. This allows `placeJtag` to place more critical block nets in a smaller area. Default: `20` Maximum value*:* `512`
`-contour {true | false}`	Creates an outline along the boundary of the chip, with its width based on the number of standard cell rows, as specified by the `-nrRow` parameter(s). You can specify different values for `-nrRowTop`, `-nrRowBottom`, `-nrRowLeft`, and `-nrRowRight`. Default: `false`
`-help`	Outputs a brief description that includes type and default information for each `placeJtag` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man placeJtag`.
`-hardMacro {true | false}`	Includes the boundaries of the hard macros in the JTAG placement. Default: `false`

`-ignoreScan {true | false}`	Ignores all scan-in pins, scan enable pins, clock pins, and asynchronous control pins. Default: `false` ⚠ `placeJtag` relies on the timing library to provide information, such as what pins are scan-in, scan enable, clk, and async control. If the timing library does not have the scan-in and scan enable information, `placeJtag` is not able to ignore them. In this case, before you run `placeJtag`, you must specify those terms using the `specifyScanCell` command.
`-ioNetWeight` *`netWtValue`*	Specifies the weight for nets connecting to I/Os. This allows `placeJtag` to place more critical I/O nets in a smaller area. Default: `100` Maximum: `512` `-ioNetWeight` applies to the nets that are connected to any I/O cell, whether it is a standard peripheral I/O cell or an Area I/O cell. This behavior is different from `-blockNetWeight`, which specifies the weight of the nets connected to blocks separately from the I/O nets.
`-nrRow nrRow`	Specifies the number of standard cell rows to place the JTAG instances.
`-nrRowBottom` *`nrRowBottom`*	Specifies the number of standard cell rows at the bottom of the core area to place JTAG logic.
`-nrRowLeft` *`nrRowLeft`*	Specifies the number of standard cell row heights at the left of the core area to place JTAG logic.
`-nrRowRight` *`nrRowRight`*	Specifies the number of standard cell row heights at the right of the core area to place JTAG logic.
`-nrRowTop` *`nrRowTop`*	Specifies the number of standard cell rows at the top of the core area to place JTAG logic.
`-orientBottom` *`orientation`*	Considers the specified orientation at each side first and then looks for other legal orientations. Specify one of the following values: `R0 R90 R180 R270 MX MX90 MY MY90`.
`-orientLeft` *`orientation`*	Considers the specified orientation at each side first and then looks for other legal orientations. Specify one of the following values: `R0 R90 R180 R270 MX MX90 MY MY90`.
`-orientRight` *`orientation`*	Considers the specified orientation at each side first and then looks for other legal orientations. Specify one of the following values: `R0 R90 R180 R270 MX MX90 MY MY90`.
`-orientTop` *`orientation`*	Considers the specified orientation at each side first and then looks for other legal orientations. Specify one of the following values: `R0 R90 R180 R270 MX MX90 MY MY90`.

Examples

The following command places the JTAG instances within four standard cell rows at the top and bottom of the chip and within four times the standard cell row height at the sides of the chip:

```
placeJtag -nrRow 4
```

The following command places the JTAG instances within ten standard cell rows at the top and bottom of the chip and within ten times the standard cell row height at the sides of the chip. It increases the I/O net weight to `256`.

```
placeJtag -nrRow 10 -ioNetWeight 256
```

Related Information

- reportJtagInst
- specifyScanCell
- specifyJtag
- traceJtag
- unplaceJtag
- Placing the Design chapter in the *Innovus User Guide*
  - Specifying and Placing JTAG and Other Cells Close to the I/Os

placePad

```
placePad
[-help]
pad_name
[orientation]
location
```

Places an I/O pad instance. This command can be used before the `place_design` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `placePad` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man placePad`.
location	Coordinates of new origin. The option is a point and a must.
orientation	Specifies the orientation of the pad cell. Specify one of the following values: `R0`, `R90`, `R180`, `R270`, `MX`, `MX90`, `MY`, and `MY90`. *Default:* `R0`
pad_name	Specifies the pad instance name.

Example

The following command places the I/O pad instance with mirrored axis orientation, rotated 90 degrees counterclockwise:

```
placePad SH14/I412 498.3 5000.8 MX90
```

Related Information

- `place_design`

placePipeline

```
placePipeline
[-netGroup {netGroupName}]
[-region {true | false }]
[-status {fixed | placed}
[-timingBased]
[-utilization utilization_value]
```

Places the registers of the specified pipeline flip-flops (PFF) net group(s) so that the distances between the stages of each group are roughly equal. This command can be used if the pipeline net groups are specified in the design.

Parameters

`-netGroup {netGroupName}`	Specifies the PFF net group, or a list of net groups, to place. By default, all PFF net groups in design are considered.
`-region {true | false}`	Creates region constraints for flops in each stage of pipeline, to allow adjustments at a later stage. *Default:* `false`
`-status {fixed | placed}`	Places the PFFs with fixed placement status, or placed status. *Default:* `placed`
`-timingBased`	Automatically finds reserve lengths. The option is of type boolean and is optional.
`-utilization utilization_value`	Specifies the expected utilization of regions for flops. *Minimum:* `0` *Maximum:* `1` **Note**: The `-region` option is required with the `-utilization`.

Example

- The following command places the registers of all pipeline flip-flops (PFF) net groups at equal distance:
  ```
  placePipeline
  ```
- The following command places the registers of pipeline flip-flops (PFF) net group named `netGroup1` at equal distance and marked the registers status `FIXED`:
  ```
  placePipeline -netGroup netGroup1 -status fixed
  ```

Related Information

- `createPipelineNetGroup`
- `deletePipelineNetGroup`
- `modifyPipelineNetGroup`
- `reportPipeline`

placeSpareModule

```
placeSpareModule
-moduleName moduleName
{-numModules integer | -stepx stepDistance -stepy stepDistance}
[-area llx lly urx ury]
[-channel
    {-maxWidth maxWidth}
    {-minWidth minWidth}
    {-minLen minLength}]
[-offsetx offsetDistance]
[-offsety offsetDistance]
[-powerDomain powerDomainName]
[-prefix prefix]
[-util utilizationFactor]
```

Instantiates a created spare module, and places spare modules across the design. You can run this command before or after placement. However, if you run the command before placement it will only add the cells and mark all the added spare cells as unplaced. If you run it after placement, it calls `ecoPlace`.

Parameters

`-area llx lly urx ury`	Specifies box coordinates (*`llx, lly, urx, ury`*) in micron.
`-channel`	Places spare modules in thin channels between placement blockages.
`-maxWidth maxWidth`	Specifies the maximum channel width, in microns. The *`maxWidth`* determines whether the placeable area between two macro blocks is considered a channel.
`-minLen minLength`	Specifies the minimum channel length, in microns. The *`minLength`* must be greater than the specified *`maxWidth`*.
`-minWidth minWidth`	Specifies the minimum channel width, in microns. The *`minWidth`* should be greater than the width of the largest-space cell provided.
`-moduleName moduleName`	Specifies the name of the spare module. This module must have been created using the `createSpareModule` command.
`-numModules integer`	Specifies the total number of spare modules to add. The software calculates how far apart to place the modules.
`-offsetx offsetDistance`	Specifies the offset along the x axis, in microns, from the left core boundary.
`-offsety offsetDistance`	Specifies the offset along the y axis, in microns, from the bottom core boundary.
`-powerDomain powerDomainName`	Specifies the power domain in which to place the spare modules.
`-prefix prefix`	Specifies a prefix to add to the automatically generated prefix for spare modules. The software already uses the prefix spr_*n* for spare modules. If you specify `-prefix spareMod`, the instances of this module are named `spareMod_spr_1, spareMod_spr_2, spareMod_spr_3`, and so on.
`-stepx stepDistance`	Specifies the distance, in microns, between the instantiated spare modules along the x axis. The *`stepDistance`* must be greater than the spare module bounding box.
`-stepy stepDistance`	Specifies the distance, in microns, between the instantiated spare modules along the y axis. The *`stepDistance`* must be greater than the spare module bounding box.
`-util utilizationFactor`	Specifies the utilization factor of the spare modules. *Value Range:* Greater than 0.0; less than 1.0. If the value of util is low, the placeable region will be larger. There would be more space for placing the spare cells and they may not be tightly bound. Similarly, if the util value is high, the placeable region will be small and spares will be tightly bound. **Note**: If you specify a high util value, the placeable region will be small and as a result the tool might not be able to accommodate all spare modules in the small region. In such cases, the tool will give the following message: `*INFO: Some spare modules might not contain all the provided spare cells.`

Note: The `placeSpareModule` command does not honor `-util` when `-numModules` switch is used.

Command Order

Run this command after running `createSpareModule`.

Examples

The following command instantiates module `mod1` and places the instantiations at intervals of 50 microns along the x and y axis. It places the instantiations at an offset of 5 microns from the left core boundary and the bottom core boundary:

```
placeSpareModule -moduleName mod1 -stepx 50 -stepy 50 -offsetx 5 -offsety 5
```

The following commands create spare module SPGM with cells DFFNSRX2, INVX4, and NOR2X2 using clock net sysclk; then instantiates module SPGMand places the instantiations between channels that are between 4.7 um and 10.0 um wide and at least 11.0 um long. The step size is 10 um in the x direction and 10 um in the y direction and the maximum utilization rate is 60 percent:

```
createSpareModule -moduleName SPGM -cell {DFFNSRX2 INVX4 \
  NOR2X2} -clock sysclk
placeSpareModule -moduleName SPGM -channel -minWidth 4.7 -maxWidth 10 \
  -minLength 11 -stepx 10 -stepy 10 -util 0.6
```

Related Information

- `createSpareModule`
- `deleteSpareModule`
- `ecoPlace`
- Placing the Design chapter in the *Innovus User Guide*
  - Placing Spare Cells and Spare Modules

power_integrity_place

```
power_integrity_place
[-help]
```

Calls Voltus to run irdrop analysis including power analysis, rail analysis, and debug IRdrop. The aggressors come from the Voltus feedback. Tool adds padding on the aggressors to push cell away to fix IRdrop. On the basis of the effort level set through this option, tool selects the IRdrop fixing strength flow.

The flow involves incremental placement and refine placement.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `power_integrity_place` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man power_integrity_place`.

queryDensityInBox

```
queryDensityInBox
query_box
```

Outputs placement density for a specific area in the design. Considers placement blockages and honors the metal layers specified by the `setPlaceMode -place_detail_preroute_as_obs`. The argument type is box in units of microns. This command is optimized for small area interactive queries.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `queryDensityInBox` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man queryDensityInBox`
query_box	Specifies box coordinates (*x1, y1, x2, y2*) in micron. • *x1: Specifies the area's lower left point on the x axis.* • *y1: Specifies the area's lower left point on the y axis.* • *x2: Specifies the area's upper right point on the x axis.* • *y2: Specifies the area's upper right point on the y axis.*

Example

The following command reports the placement density for the specific area 2835.342 3381.807 4807.673 4530.904:

```
queryDensityInBox 2835.342 3381.807 4807.673 4530.904
```

The output of this command is:

```
StdInstArea/freeSpace = 27.0%(197105.76/728824.32)
macroInstArea/totArea = 67.7%(1539687.84/2272665.60)
powerMetalArea/totArea = 0.0%(0.00/2272665.60)
PlacementObsArea/totArea = 0.0%(0.00/2272665.60)
Utilization in area (2835342 3381807) (4807673 4530904) = 27.04%
```

Related Information

- `setPlaceMode`

queryPinDensity

```
queryPinDensity
[-help]
[-area <x1 y1 x2 y2>]
```

Outputs pin density for the design. The argument type is box in units of microns.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `queryPinDensity` parameter. For a detailed description of the command and all of its parameters, use the man command: `man queryPinDensity`
`-area`	Species the box coordinates in microns. • *x1*: Specifies the area's lower left point on the x axis. • *y1*: Specifies the area's lower left point on the y axis. • *x2*: Specifies the area's upper right point on the x axis. • *y2*: Specifies the area's upper right point on the y axis.

Examples

The following command reports the pin density for the specific area 43.24 23.505 139.626 113.411:

```
queryPinDensity -area 43.24 23.505 139.626 113.411
```

```
calculate placement pin density in box (43.20, 16.00, 140.00, 120.00) Pin Density = 0.3223.
= total # of pins 507 / total area 1573.
```

refinePlace

```
refinePlace
[-help]
[-area {lx ly ux uy}]
[-eco {true|false}]
[-inst list_of_instances]
[-wire_length_reclaim {true|false}]
```

Corrects flawed cell locations and reports corrections or instance overlap problems. You do not need a fully placed design to run `refinePlace`. The `refinePlace` command only makes changes if the placement is not legal, such as for instance overlap, incorrect orientation, constraint violations, and placement/routing blockage violations. You can use the `checkPlace` command to highlight placement violations in the design. The `refinePlace` command is called automatically at the end of each major super command such as `place_opt_design`, `optDesign`, and `ccopt_design` to ensure that the final placement is violation-free. If any instances are moved manually, `refinePlace` can be run to legalize any cell placements that may have resulted in violations due to this manual movement.

To place unplaced instances, use the `ecoPlace` command.

`refinePlace` only operates on the cells which have the 'placed' placement status.

Note: You can also use `setPlaceMode` to specify parameters for this command. If you use `setPlaceMode`, the parameters apply every time you run `refinePlace` in the current session, unless they are overwritten by the `refinePlace` settings.

Note: `refinePlace` and other commands using `refinePlace` API honor instance padding constraint as a hard rule if `setPlaceMode -place_detail_honor_inst_pad` is set to `true`.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `refinePlace` parameter. For a detailed description of the command and all of its parameters, use the man command: `man refinePlace`
`-area {llx lly urx ury}`	Specifies `refinePlace` area. • *`llx`*: Specifies the area's lower left x coordinate. • *`lly`*: Specifies the area's lower left y coordinate. • *`urx`*: Specifies the area's upper right x coordinate. • *`ury`*: Specifies the area's upper right y coordinate. **Note**: The coordinates for the `-area` option should be provided in brackets, else the tool will report an error.
`-eco {true|false}`	Activates ECO mode for the post-route design. *Default*: `false` **Note**: This option is controlled by `setPlaceMode -place_detail_eco_max_distance`.
`-inst list_of_instances`	Legalizes the list of instances provided by the user. All other instances are treated as fixed. **Note**: This option is controlled by `setPlaceMode -place_detail_eco_max_distance`.
`-wire_length_reclaim {true|false}`	Activates the wire length reclaim mode.

Command Order

Use this command after finding an error with the `checkPlace` command.

Example

- The following command refines the placement in the specified area:

```
refinePlace -area {7.925 29.762 11.702 27.531}
*** Starting refinePlace (0:04:48 mem=991.9M) ***
Move report: Detail placement moves 0 insts, mean move: 0.00 um, max move: 0.00 um
        Runtime: CPU: 0:00:00.0 REAL: 0:00:00.0 MEM: 991.9MB
Summary Report:
Instances move: 0 (out of 6 movable)
Mean displacement: 0.00 um
Max displacement: 0.00 um
Runtime: CPU: 0:00:00.0 REAL: 0:00:00.0 MEM: 991.9MB
*** Finished refinePlace (0:04:48 mem=991.9M) ***
```

Related Information

- checkPlace
- ecoPlace
- setPlaceMode

report_cell_edge_spacing

```
report_cell_edge_spacing
[-help]
[-file fileName]
```

Reports all the cell edge spacing rules in both terminal and log file.

Parameters

`-file fileName`	Specifies the file name to dump cell edge spacing rules.
`-help`	Outputs a brief description that includes the type and default information for each `report_cell_edge_spacing` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_cell_edge_spacing`

Related Information

- `specifyCellEdgeSpacing`
- `deleteCellEdgeSpacing`

report_cell_edge_type

```
report_cell_edge_type
[-help]
[-cell cellName]
[-file fileName]
```

Reports the edge type of the specified cell.

Parameters

`-cell` *cellName*	Specifies the cell name for which edge type is to be reported. If the cell name is not specified, all the cell edge types are reported in the edge_type.rpt file report file.
`-file` *fileName*	Specifies the report file name. If no file name is specified, the edge_type.rpt file is used to generate the report.
`-help`	Outputs a brief description that includes type and default information for each `report_cell_edge_type` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_cell_edge_type`.

Related Information

- `specifyCellEdgeType`
- `deleteCellEdgeType`

report_cell_stack_area

```
report_cell_stack_area
[-help]
[-cell cell_name]
[-file file_name]
[-group group_name]
```

Reports the cell stack area definition for cells.

Parameters

`-cell cell_name`	Specifies the cell(s) for which the cell stack area definition needs to be reported.
`-file file_name`	Specifies the file name in which the reported information is saved. By default, all cell stack groups are reported to the `cell_stack_area.rpt` file.
`-group group_name`	Specifies the group(s) for which the cell stack area definition needs to be reported.
`-help`	Outputs a brief description that includes type and default information for each `report_cell_stack_area` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_cell_stack_area`

Related Information

- `specify_cell_stack_area`
- `delete_cell_stack_area`

report_cell_stack_group

```
report_cell_stack_group
[-help]
[-file <string>]
[-name <string>]
```

Reports the cell stack group definition.

Parameters

`-file` *`file_name`*	Specifies the file name in which the reported information is saved. By default, all cell stack groups are reported to the `cell_stack_group.rpt` file.
`-help`	Outputs a brief description that includes type and default information for each `report_cell_stack_group` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_cell_stack_group`
`-name` *`group_name`*	Specifies the group(s) that needs to be reported.

Related Information

- `specify_cell_stack_group`
- `delete_cell_stack_group`

report_cell_virtual_align

```
report_cell_virtual_align
[-help]
[-cell cells]
```

Reports the cells that need to be virtually aligned.

Parameters

`-cell cells`	Specifies the lib cells that require virtual alignment.
-help	Outputs a brief description that includes type and default information for each `report_cell_virtual_align` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_cell_virtual_align`

Related Information

- `specify_cell_virtual_align`
- `delete_cell_virtual_align`

report_inst_space_group

```
report_inst_space_group
[-help]
[-file fileName]
[-group groupName]
```

Reports all instance space groups into the inst_space_group report file by default.

Parameters

`-file fileName`	Specifies the file name to dump space group
`-group groupName`	Specifies the name of the group to be reported.
`-help`	Outputs a brief description that includes type and default information for each `report_inst_space_group` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_inst_space_group`.

Related Information

- `create_inst_space_group`
- `delete_inst_space_group`

report_pg_keepout

```
report_pg_keepout
[-help]
[-file file_name]
{-cell lib_cell_name | -inst instance_name}
```

Reports all PG keepout boxes in the design if no option is specified.

Parameters

-cell lib_cell_name	Reports PG keepout boxes applied to the specified lib cell only.
`-file file_name`	Specifies the file name where all the details are dumped.
-help	Outputs a brief description that includes type and default information for each `report_pg_keepout` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_pg_keepout`
-inst instance_name	Reports PG keepout boxes applied to the specified instance only.

Related Information

- `specify_pg_keepout`
- `delete_pg_keepout`

report_scan_chain

```
report_scan_chain
[-help]
[-chain chainName]
[-out_file fileName]
[-verbose {true|false}]
```

Prints the scan chain information.

Parameters

`-chain` *chainName*	Specifies the scan chain name.
`-help`	Outputs a brief description that includes type and default information for each `report_scan_chain` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_scan_chain`.
`-out_file` *fileName*	Specifies the output file name of scan chain.
`-verbose {true|false}`	Prints out more message

Related Information

- `specifyScanChain`
- `deleteScanChain`

reportCellPad

```
reportCellPad
[-help]
[-cell cellName]
[-file fileName]
```

Prints the number of cells, which contain user-specified padding, to the log file. Optionally, you can obtain a complete list of cells with their padding information in a file using the `-file` parameter. The format of this file is same as used in the `specifyCellPad` command, that is,

cellName padding (in grid units)

Parameters

`-cell cellName`	Specifies the cell name to report padding.
`-file fileName`	Specifies the generated file name to dump cell padding.
`-help`	Outputs a brief description that includes type and default information for each `reportCellPad` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man reportCellPad`.

Examples

The following steps show the usage of `reportCellPad` command:

1. Specify the following padding using the `specifyCellPad` command:

   ```
   specifyCellPad FF_SCAN 2
   specifyCellPad DFFSHQX1 5
   specifyCellPad DFFSRX4 4
   ```

2. Run the `reportCellPad` command. After running this command, you get the value `3` and the following message is printed to the log file:

   ```
   3 cells have padding.
   ```

3. To print the above-mentioned list of cells in the file `myfile`, specify the following command:

   ```
   reportCellPad -file myfile
   ```

4. After running this command, the following contents of `myfile` are displayed:

   ```
   FF_SCAN 2
   DFFSHQX1 5
   DFFSRX4 4
   # 3 cells have padding.
   ```

Related Information

- specifyCellPad
- deleteCellPad

reportCongestion

```
reportCongestion
[-help]
[-3d]
[-hotSpot]
[-includeBlockage]
[-num_hotspot number]
[-overflow]
```

Reports the average congestion and the local hotspot score. Route needs be called before using this command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `reportCongestion` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man reportCongestion`.
`-3d`	Honors 3d congestion map. When specified, it enables the layer-based congestion reporting.
`-hotSpot`	Reports the local hotspot score. It reports the maximum and total hotspot area. Hotspot score refers to the contiguous area of gcells (there is some normalization done internally) which have routing overflow. Typically, a max hotspot score < 100 indicates that the design is routable. • Max hotspot – It is the largest contiguous area of gcells with overflow • Total hotspot – It is the total of all hotspots Greater the number of adjacent gcells with overflow, bigger will be the hotspot. **Note:** You can use this parameter to view the congestion hotspot regions seen by the router after global route.
`-includeBlockage`	Includes blockage area in the hotspot report.
`-num_hotspot number`	Specifies the number of hotspots to be reported.
`-overflow`	Reports horizontal and vertical overflow.

Examples

- The following command reports the congestion:

  ```
  reportCongestion -overflow
  ```

  Output:

  ```
  Usage: (10.0%H 8.1%V) = (2.425e+05um 3.107e+05um) = (72854 61452)
  Overflow: 210 = 7 (0.01% H) + 202 (0.33% V)

  Congestion distribution:

  Remain cntH cntV
  ------------------------------------
  -5: 0 0.00% 1 0.00%
  -4: 0 0.00% 7 0.01%
  -3: 0 0.00% 18 0.03%
  ```

```
-2: 1 0.00% 41 0.07%
-1: 6 0.01% 97 0.16%
--------------------------------------
0: 125 0.21% 256 0.42%
1: 568 0.95% 685 1.13%
2: 285 0.47% 7995 13.14%
3: 889 1.48% 6145 10.10%
4: 7457 12.41% 917 1.51%
5: 50749 84.47% 44681 73.44%
```

- The following command reports the local hotspot score:

  ```
  reportCongestion -hotSpot
  ```

  Output:

  ```
  [hotspot] +-----+---------------------------------------+---------------+
  [hotspot] | Top |            hotspot bbox               | hotspot score |
  [hotspot] +-----+---------------------------------------+---------------+
  [hotspot] |  1  |       32.16 262.56 216.48 400.80      |    54.89      |
  [hotspot] +-----+---------------------------------------+---------------+
  [hotspot] |  2  |       1.44 446.88 124.32 585.12       |    46.89      |
  [hotspot] +-----+---------------------------------------+---------------+
  [hotspot] |  3  |      554.40 170.40 638.88 277.92      |    30.89      |
  [hotspot] +-----+---------------------------------------+---------------+
  [hotspot] |  4  |      477.60 708.00 585.12 800.16      |    25.22      |
  [hotspot] +-----+---------------------------------------+---------------+
  [hotspot] |  5  |     62.88 108.96 185.76 170.40        |    21.22      |
  [hotspot] +-----+---------------------------------------+---------------+
  ```

- The following command reports the local hotspot score:

  ```
  reportCongestion -hotSpot -3d
  ```

  ```
  [hotspot] +------------+---------------+---------------+---------------------------------------+
  [hotspot] |    layer   |  max hotspot  | total hotspot |            hotspot bbox               |
  [hotspot] +------------+---------------+---------------+---------------------------------------+
  [hotspot] |    M1(H)   |          0.00 |          0.00 |   (none)                              |
  [hotspot] |    M2(V)   |         19.78 |        109.56 |   554.40   600.48   615.84   692.64 |
  [hotspot] |    M3(H)   |        174.33 |        693.67 |    47.52   201.12   293.28   723.36 |
  [hotspot] |    M4(V)   |          0.89 |          6.67 |   247.20   354.72   277.92   385.44 |
  [hotspot] |    M5(H)   |          4.89 |         26.22 |   108.96   170.40   139.68   201.12 |
  [hotspot] +------------+---------------+---------------+---------------------------------------+
  ```

```
[hotspot] |   worst    | (M3)   174.33 | (M3)   693.67 |                                      |
[hotspot] +------------+---------------+---------------+--------------------------------------+
[hotspot] | all layers |        168.89 |        654.89 |                                      |
[hotspot] +------------+---------------+---------------+--------------------------------------+
Local HotSpot Analysis (3d): normalized congestion max/total hotspot area = 168.89/654.89 (area is in unit of 4
std-cell row bins)
```

reportDensityMap

```
reportDensityMap
[-help]
[-gridInMicron microns] [-gridInRow numberRows]
[-ignoreBlock {true|false}]
[-ignoreFiller {true|false}]
[-threshold density]
```

Generates a map that uses colors to represent placement density and generates a placement density report. If you run this command without parameters, it generates a map with a threshold of 0.75 and measures the grid in row height.

Note: You can also use `setDensityMapMode` to set the threshold, and specify whether the map displays bins in microns or rows.

Use this command any time after placement.

Use the Display Density Map form to select colors and step-size for the density levels in the map.

Parameters

`-gridInMicron` *microns*	Specifies the horizontal or vertical dimension of a bin in the map, measured in microns. *Default:* `50` *Upper limit:* `1000`
`-gridInRow` *numberRows*	Specifies the horizontal or vertical dimension of a grid in the map, measured in row height. If you specify a value less than 1, the software uses 1. *Default:* `10` *Upper limit:* `100`
`-help`	Outputs a brief description that includes type and default information for each `reportDensityMap` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man reportDensityMap`.
`-ignoreBlock {true|false}`	Skips block cells instances from both supply and demand calculation. *Default*: `false`
`-ignoreFiller {true|false}`	Ignores the density of fillers in the post-filler density heat map to show the density of standard cells . *Default*: `false`
`-threshold` *density*	Specifies a threshold density value for the grid. Any grid under the threshold value is not reported. The value must be positive. *Default:* `0.75` Upper limit: `2.0`

Example

The following are output from command `reportDensityMap`. It generates a map with default threshold of 0.75 and measured in row height:

```
powerDomain TOP: bins with density > 0.75 = 0.519 % ( 6 / 1156 )
bin size: 59 sites by 10 row(s). total 1156 bins ( 34 by 34 )
density range #bins %
0.75 - 0.8 5 0.4325
0.8 - 0.85 1 0.08651
total 6 0.519
```

Related Information

- setDensityMapMode
- Place Menu chapter of the *Innovus Menu Reference*
    - Display Density Map

reportInstPad

```
reportInstPad {instName | -all}
```

Reports the padding for a specific instance, or all instance padding in the design. This command can be used if there is instance padding specified.

Parameters

`-all`	Reports all instance padding in the design.
instName	Reports the padding for the specified instance.

Example

The following command reports the padding for a specified instance named U19567:

```
reportInstPad U19567
```

It outputs following:

```
Padding for Instant [U19567] is 2.
```

Related Information

- `deleteInstPad`
- `specifyInstPad`
- Placing the Design chapter in the *Innovus User Guide*
  - Adding Padding

reportJtagInst

```
reportJtagInst
[-outfile]
```

Displays JTAG information specified by the `specifyJtag` command or creates an output file that contains the information.

Parameters

`-outfile`	Specifies the name of the file. If you do not specify this parameter, the command prints the information to the screen without saving it in a file.

Related Information

- `specifyJtag`
- Placing the Design chapter in the *Innovus User Guide*
  - Specifying and Placing JTAG and Other Cells Close to the I/Os

reportPinDensityMap

```
reportPinDensityMap
[-help]
[-displayStep step_value]
[-gridInMicron micron]
[-gridInRow numberRow]
[-threshold density]
```

Generates a map that uses colors to represent pin density and generates a pin density report. Use this command any time after placement.

You can also use `setPinDensityMapMode` to set the threshold, and specify whether the map displays bins in microns or rows.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `reportPinDensityMap` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reportPinDensityMap`
`-displayStep step_value`	Specifies the range step for displaying the pin density map. *Default*: `0.05`
`-gridInMicron micron`	Specifies the horizontal or vertical dimension of a bin in the map in microns. *Default:* `50` *Minimum*: `0.000000` *Maximum:* `1000.000000`
`-gridInRow numberRow`	Specifies the value (in row-height) of side of square grid for pin density calculations. *Default:* `10` *Minimum:* `1` *Maximum:* `100`
`-threshold density`	Specifies a threshold density value over which the densities must be reported. Any grid under the threshold value is not reported. The value must be positive. *Default*: `0.5` *Minimum*: `0.000000` *Maximum*: `2.000000`

Examples

The following command reports the pin density map:

```
reportPinDensityMap
*** Starting report pin density ***
default core: bins with pin density > 0.5 = 0 % (0 / 1)
bin size: 90 sites by 10 row(s). total 1 bins ( 1 by 1 )
pin density range   #bins     %
total                 0       0
```

Related Information

- setPinDensityMapMode

reportPipeline

```
reportPipeline
[-help]
[-netGroup netGroupNames]
[{-spacing value  | -timingBased  | -psPMBased } -file fileName]
```

The command can be used for analysis before the flops exist in the netlist; or it can be used post flop insertion to either report expected average wire-length between stages of a pipeline net group, or to report the achieved average distance between the placed stages of the pipeline. When used for analysis, the command reports expected number of PFF stages for each net group based on the user-specified value. Optionally the command generates bus-guides for reported route-path.

Parameters

`-file fileName`	Specified the output for additional information from the command. The output includes expected placement location of PFF at each stage in analysis flow.
`-netGroup netGroupNames`	Restricts the report to the net group name specified. If no net group name is specified, all pipeline net groups are reported.
`-spacing value | -timingBased | -psPMBased`	Given the target approximate wire-length spacing between stages, `-spacing` suggests the number of flop stages to insert. `-timingBased` is used to automatically estimate the number of pipeline FFs needed, based on timing delay on routing layers and the clock cycle requirement. It is mutually exclusive to option `-spacing` because the spacing can be decided automatically. `-psPMBased` is used to automatically find maximum spacing using pspm model.

Example

- The following command reports expected number of PFF stages for net group "netGroup1" if the distance between each stage is around 300 microns:
  ```
  reportPipeline -netGroup netGroup1 -spacing 300
  ```
- The following command reports the expected number of PFF stages for net group `netGroup1` if the distance between each stage is around 300 microns. Additional information of expected placement location of each stage are written in the file `PFF.rpt`:
  ```
  reportPipeline -netGroup netGroup1 -spacing 300 -file PFF.rpt
  ```

Related Information

- createPipelineNetGroup
- deletePipelineNetGroup

reportScanCell

`reportScanCell`

Displays a list of the scan cells that are read from the timing library or by the `specifyScanCell` command. This command can be called in any stage of flow after importing design if there are scan cell specifications in library; or else it can be used after the `specifyScanCell` command.

Parameters

None

Example

Following is an example of the output from `reportScanCell`:

```
ScanFF SEDFFXL
  FT SI is test_scan_in
  FT SE is Scan Enable Clock
  FT QN is test_scan_out_inverted
  FT Q is test_scan_out
ScanFF SEDFFX4
  FT SI is test_scan_in
  FT SE is Scan Enable Clock
  FT QN is test_scan_out_inverted
  FT Q is test_scan_out
...
```

Related Information

- `deleteAllScanCells`
- `specifyScanCell`
- Placing the Design chapter in the *Innovus User Guide*
  - Optimizing and Reordering Scan Chains

reportScanChainPartition

```
reportScanChainPartition
[-help]
```

Generates a report that lists compatible scan chains in scan chain partitions.

Specify scan chain partitions with the following command: `specifyScanChainPartition`.

Parameters

None

Example

To see a list of the compatible scan chain groups in a design with one partition, `clk1Domain`, that has two compatible scan chains, type the following command:

```
reportScanChainPartition
```

The command outputs the following messages:

```
Info: Scan Chain Partition Group set to:

      Partition group: clk1Domain

                chain: chain1

                chain: chain2
```

Related Information

- `deleteScanChainPartition`
- `specifyScanChainPartition`
- Placing the Design chapter in the *Innovus User Guide*
    - Optimizing and Reordering Scan Chains

reset_spare_insts

```
reset_spare_insts
[-help]
{-cell cellName | -inst instName | -hinst hinstName}
```

Asserts the specified instance is not a spare gate. This command can be used after `specifySpareGate`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `reset_spare_insts` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reset_spare_insts`.
`-cell cellName`	Specifies the name of the spare cell to be reset. Wildcards are supported.
`-hinst hinstName`	Specifies the name of the spare hierarchical instance to be reset. Wildcards are supported.
`-inst instName`	Specifies the name of the spare instance to be reset. Wildcards are supported.

Example

- In the following example, “AND2X1” cell is defined as a spare cell in the design. The example shows the query result of the spare cell of AND2X1:

```
innovus 36> dbGet [dbGet [dbGet top.insts.cell.name AND2X1 -p2 ].isSpareGate 1 -p ].name
spr_51/spr_gate0 spr_48/spr_gate0 spr_47/spr_gate0 spr_46/spr_gate0 spr_45/spr_gate0 spr_44/spr_gate0
spr_43/spr_gate0 spr_42/spr_gate0 spr_41/spr_gate0 spr_40/spr_gate0 spr_39/spr_gate0
```

- The following example resets the spare cell instance, `spr_39`, with the `-inst` option. As a result, the spare cell is removed from spare module list and is not a spare gate any more:

```
innovus 32> dbGet [dbGetInstByName spr_39/spr_gate0].isSpareGate
1
innovus 33> reset_spare_insts -inst spr_39/spr_gate0
Resetting spare instance [spr_39/spr_gate0].
Reset 1 spare instances.
innovus 34> dbGet [dbGetInstByName spr_39/spr_gate0].isSpareGate
0
```

- The following commands reset the spare cell hierarchical instance, `spr_40`, with the `-hinst` option, removes it from the spare module list, and it is not a spare gate any more.

```
innovus 29> dbGet [dbGetInstByName spr_40/spr_gate0].isSpareGate
1
innovus 30> reset_spare_insts -hinst spr_40
Resetting spare mode HInst [spr_40].
innovus 31> dbGet [dbGetInstByName spr_40/spr_gate0].isSpareGate
0
```

- The following commands reset the spare cell type, `AND2X1`, using the `-cell` option. All the instances of the `AND2X1` cell type are removed from spare module list and it is not a spare gate any more.

```
innovus 37> reset_spare_insts -cell AND2X1
Resetting all spare instances of cell [AND2X1].
innovus 38> dbGet [dbGet [dbGet top.insts.cell.name AND2X1 -p2 ].isSpareGate 1 -p ].name
0x0
```

Related Information

- addSpareInstance
- specifySpareGate

scanReorder

```
scanReorder
[-help]
[-addScanPortPrefix prefix]
[-clkAware {true|false}]
[-preferH {true|false}]
[-preferV {true|false}]
[-keepPDPorts {true|false} ]
[-scanEffort {low|medium|high|auto} ]
[-skipMode {skipNone|skipBuffer}]
```

Reorders the scan cells after running placement. Controls the skipping and removal of buffers and inverters in the scan chain.

Set global parameters for this command with the `setScanReorderMode` command.

This command automatically detects multiple power domains and minimizes the number of scan chains that pass through them. It respects rectilinear and rectangular power domain boundaries. If a scan chain crosses power domains that operate at the same voltage, the software does not insert level shifters. If the scan chain crosses power domains operating at different voltages, the software inserts the appropriate level shifters.

If scan reorder fails, use the `scanTrace -verbose` command to see where the scan chain stops.

Note: Parameters set by this command override the settings of the `setScanReorderMode` command.

Note: This command supports multithreading.

Parameters

`-addScanPortPrefix` *prefix*	Specifies the prefix to be added to scan hports. *Default:* `""`
`-clkAware {true | false}`	Specifies whether scan reordering is clock tree aware. • When `true`, scan reordering is clock tree aware, and is clock tree driven. • When `false`, scan reordering is not clock tree aware, and is wirelength driven. *Default:* `false` (The software is not clock tree aware) This parameter can also have the value `0, 1` or any floating point number between `0` and `1`. If this parameter is not specified, or if `0` is specified for this parameter, this indicates: Do not consider clock tree at all. If the value is close to `1`, this indicates: Give greater consideration to clock tree.
`-keepPDPorts {true | false}`	Maintains or corrects the hierarchical ports without deleting or inserting shifters, and reorders the remaining scan chain. To turn this option off, use `scanReorder -keepPDports false` or `setScanReorderMode -keepPDPorts false`. *Default:* `true` **Note:** This parameter works in the MSV flow only.

`-preferH {true | false}`	Specifies that the preferred direction for scan reordering connections is horizontal. Reorders the scan chain to snake a sideways "S" configuration if the block is short and wide or the design has vertical routing congestion. *Default:* `false` **Note:** If both `-preferV` and `-preferH` are set to `true`, the software does not honor the setting and treats them both as if they were `false`.
`-preferV {true | false}`	Specifies that the preferred direction for scan reordering connections is vertical. Reorders the scan chain to snake a vertical "S" configuration if the block is long and narrow or the design has horizontal routing congestion. *Default:* `false` **Note:** If both `-preferV` and `-preferH` are set to `true`, the software does not honor the setting and treats them both as if they were `false`.
`-scanEffort {low|medium|high|auto}`	Specifies the effort level for reordering scan chains. *Default:* `-medium` Specify one of the following values: • `auto`: Runs internal reorder effort, and it may tune at different stages. • `high`: Runs more iterations of scan reorder to achieve better net length. Using this parameter increases the run time. • `low`: Runs fewer iterations of scan reorder to minimize net length quickly. • `medium`: Runs a normal number of iterations.
`-skipMode {skipNone | skipBuffer}`	Specifies how scan reordering handles buffers and inverters in the scan chain. *Default:* `skipNone` Specify one of the following: • `skipNone`: Specifies that buffers and inverters remain after the scan chain reorder. • `skipBuffer`: Ignores buffers in the scan chain. If a buffer is not connected elsewhere in the design, `scanReorder` removes the buffer.

Command Order

Use this command after running placement with the ignore scan connection parameter (`setPlaceMode -place_global_ignore_scan true`), and after specifying the scan chain or loading scan chain information in the DEF or TDF files.

Example

The following command reconnects the scan chain so that buffers are skipped:

```
scanReorder -skipMode skipBuffer
```

Related Information

- `scanTrace`
- `setScanReorderMode`
- Placing the Design chapter in the *Innovus User Guide*
    - Optimizing and Reordering Scan Chains

scanTrace

```
scanTrace
[-help]
[-compLogic | -noCompLogic]
[-lockup | -noLockup]
[-verbose]
```

Records the total number of elements in the scan chain in the log file, and records the starting and ending scan points if the `-verbose` option is used. The `scanReorder` command automatically calls `scanTrace` internally if you have not previously run `scanTrace`.

Note: If scan chains are specified by reading in a DEF file, the software does a native scan trace. The scan DEF file is stored in the database and, when the `scanReorder` command runs, the software matches the scans and honors the ordered sections.

Parameters

`-compLogic`	Traces through multiple input gate on the scan path.
`-noCompLogic`	Does not trace through multiple input gate on the scan path.
`-lockup`	Specifies that the tracing detects lockup latches automatically. **Note:** If the lockup elements are flops (non scan flip-flops), the lockup elements must be identified using the `specifyLockupElement` command prior to running `scanTrace` in its default mode.
`-noLockup`	Specifies that the tracing will not detect lockup elements. This is the default.
`-verbose`	Prints the information stored in the logv file, i.e starting and ending scan points, and the cell instance names that `scanTrace` determines are part of the scan chain. If scan chain tracing fails, use this parameter to see where the scan chain tracing stops.

Command Order

Use this command after specifying scan chains using the `specifyScanChain` command.

Examples

The following set of commands traces all scan chains defined by `specifyScanChian` command:

```
specifyScanChain chain0 -start BG_scan_in_1 -stop BG_scan_out_1
specifyScanChain chain1 -start BG_scan_in_2 -stop BG_scan_out_2
specifyScanChain chain2 -start BG_scan_in_3 -stop BG_scan_out_3
scanTrace
```

Related Information

- `scanReorder`
- `specifyLockupElement`
- `specifyScanChain`
- Placing the Design chapter in the *Innovus User Guide*
  - Optimizing and Reordering Scan Chains

set_well_tap_mode

```
set_well_tap_mode
[-help]
[-reset]
[-avoid_vertical_well_abutment {none|ntap|ptap|all_tap}]
[-avoidAbutment {true|false}]
[-block_boundary_only {true | false}]]
[-bottom_tap_cell cellName]
[-bottom_termination_cell cellList]
[-cell cellName]
[-channel_offset offset_value]
[-check_channel {true|false}]
[-column_cell cellList]
[-create_rows {true|false}]
[-disable_check_zone_at_boundary {none|vdd|vss|both}]
[-inRowOffset offset_value]
[-insert_cells cellName rule dense_layer dense_rule boundary_layer boundary_rule]
[-rule microns]
[-siteOffset number_of_sites]
[-tap_function_cells cell_list]
[-tap_termination_alignment {true|false}]
[-termination_align {left|right|center}]
[-termination_cell cellList]
[-top_tap_cell cellName]
[-top_termination_cell cellList]
[-vertical_boundary_spacing spacing_with_well_tap]
[-well_cut_cell cellList]
```

Controls the behavior of `addWellTap` command. The mode setting is persistent and saved along with the database by the `saveDesign` command. Use `get_well_tap_mode` to return the current settings for `set_well_tap_mode` command.

Note: Parameters specified by the `addWellTap` command supersede those specified by the `set_well_tap_mode` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_well_tap_mode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_well_tap_mode`.
`-avoid_vertical_well_abutment {none|ntap|ptap|all_tap}`	Specifies which type of well-tap cells to be forbidden from the vertical abutment. *Default*: `none`
`-avoidAbutment {true|false}`	Specifies whether the inserted taps should avoid horizontal and vertical abutment. *Default*: `false`
`-block_boundary_only {true|false}`	Specifies whether to insert termination tap cells only for well tap columns on the block boundary. Default is false, termination cells are inserted on both core boundary row and block boundary row. **Note**: This parameter works only when inserting termination cells.
`-bottom_tap_cell cellName`	Specifies well taps inserted into bottom rows during enclosed end cap insertion. **Note**: This parameter must be specified when '`setEndCapMode -boundary_tap true`'.

`-bottom_termination_cell` *cellList*	Specifies the list of bottom termination cells to be used. *Default*: `""`
`-cell`	Specifies the name of the cell in the technology library to use as a well-tap. The cell should be of class type CORE. *Default*: `""`
`-channel_offset` *offset_value*	Specifies the offset value between taps and blockage in channel. *Default*: `0` *Minimum*: `0` *Maximum*: `1.79769313486232e+308`
`-check_channel {true|false}`	Specifies whether the blockage channel will be checked.
`-column_cell` *cellist*	Specifies the list of well tap columns that need to be terminated. This parameter must be specified for termination tap cell insertion.
`-create_rows {true|false}`	Creates rows for welltap cell techSites which do not have rows in the floorplan. *Default*: `false`
`-disable_check_zone_at_boundary {none|vdd|vss|both}`	Disables the check zone at boundary. *Default*: `none`
`-inRowOffset` *offset_value*	Specifies the distance between the first welltap in a row and the start of the row. *Default*: 0 (first cell in beginning of row is the starting point) *Minimum*: `0` *Maximum*: `1.79769313486232e+308`
`-reset`	Resets all or specified options to the default value. Any values that do not already match the default value and are reset will be reported.

`-insert_cells` *`cellName rule dense_layer dense__rule boundary_layer boundary_rule`*	Inserts well-taps based on cell, layer, and interval. • *`cellName`*: Specifies the cell name. • *`rule`*: Cell with this rule would be added as a general tap cell for all placeable area. The center to center spacing of tap cells would be `≤ (2*$rule)` without `-checkerBoard` and `≤ (4*$rule)` with -`checkerBoard`. **Note**: Center of the tap cell is used to check the spacing. • *`dense_layername`*: Specifies the dense layer name. • *`dense_rule`*: Cell with dense layer would be added to the given layer regions only. Cell with dense layer must follow the dense rule. The center to center spacing of dense tap cells would be `≤ (2*$dense_rule)` without `-checkerBoard` and `≤ (4 *$dense_rule)` with `-checkerBoard`. • *`boundary_layername`*: Specifies the boundary layer name. • *`boundary_rule`*: A cell would be added for boundaries inside the given layer regions. The cell must follow the boundary rule. The spacing between the boundary to the center of boundary tap cells would be ≤ $ boundary rule. Boundary layer regions which cover partial sites should be extended. Boundary taps can be located in or out of boundary layer regions. Note: • You can define multiple cells for -insert_cells by separating each cell by { }. • There should be at lest one rule, dense layer or boundary layer, defined for each cell. • The order of tap cell insertion is: a. Dense tap b. Boundary tap c. General tap • If the area is covered by both dense layer region and boundary layer region, both layer rules must be satisfied. • If `-insert_cells` is defined multiple times, the last setting is accepted by the tool. • If `-insert_cells` is defined, you do not need define `-rule` while using the `verifyWellTap` command.
`-rule`	Specifies the rule to be honored when adding well taps.
`-siteOffset` *`number_of_sites`*	Specifies an offset value, in number of sites, so that the vertical taps are aware of.

`-tap_function_cells` *cell_list*	Defines the endcap cells which have tap functions. Use of tap function cells reduces the number of tap cells to be inserted. The defined endcap cells are treated as tap function cells during the tap cell insertion. The tool will consider the tap distance between welltap cell and the defined endcap cell with tap function. To insert tap cells considering the tap function not only on the checkboard but also on the termination tap cell insertion flow, `addWellTap` treats the endcap cell as a tap function cell when `set_well_tap_mode -tap_function_cells` is set. The `verifyWellTap` command honors the `set_well_tap_mode -tap_function_cells` settings to check the tap distance. *Default*: `""`
`-tap_termination_alignment {true|false}`	Specifies whether to align well-taps and termination cells. *Default*: false
`-termination_align {left|right|center}`	Specifies the termination cell alignment. *Default*: `left`
`-termination_cell` *cellList*	Specifies the list of cells that can be used as termination taps. The command `addWellTap` selects the best fit termination cells from this list and inserts them at the top and bottom of well tap column segment. The existing non-termination well taps are deleted as needed. If the well tap columns have already been terminated with a valid termination cell, the existing termination cell are kept in the incremental run and no changes are made. **Note:** It is recommended that well tap columns are added before termination cell insertion. **Note:** This parameter must be specified for termination tap cell insertion.
`-top_tap_cell` *cellName*	Specifies well taps inserted into top rows during enclosed end cap insertion. Note: This parameter must be specified when `setEndCapMode -boundary_tap` is set to `true`.
`-top_termination_cell` *cellList*	Specifies the list of top termination cells to be used. *Default*: `""`
`-vertical_boundary_spacing` *spacing_with_well_tap*	Specifies the spacing between center of tap cell and vertical boundary *Default*: `-1` *Minimum*: `-1` *Maximum*: `1.79769313486232e+308`
`-well_cut_cell` *cellist*	Specifies the list of cells with the well cuts. `addWellTap` considers the specified cells cutting the row.

Related Information

- `addWellTap`
- `saveDesign`
- `setEndCapMode`
- `verifyWellTap`

setDensityMapMode

```
setDensityMapMode
[-help]
[-reset]
[-gridInMicron microns] [-gridInRow numberRows]
[-ignoreBlock {true|false}]
[-ignoreFiller {true|false}]
[-threshold density]
```

Sets global parameters for the `reportDensityMap` command. Parameters that you specify with `setDensityMapMode` are then used when you run `reportDensityMap`.

Note: Parameters specified by the `reportDensityMap` command supersede those specified by the `setDensityMapMode` command.

See `getDensityMapMode` to return the current settings for `setDensityMapMode`.

Parameters

`-gridInMicron` *microns*	Specifies the horizontal or vertical dimension of a grid in the map, measured in microns. *Default:* `50`
`-gridInRow` *numberRows*	Specifies the horizontal or vertical dimension of a grid in the map, measured in number of rows. If you specify a value less than `1`, the software uses 1. *Default:* `10`
`-help`	Outputs a brief description that includes type and default information for each `setDensityMapMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setDensityMapMode`.
`-ignoreBlock {true|false}`	Skips block cells instances from both supply and demand calculation. *Default*: `false`
`-ignoreFiller {true|false}`	Ignores the density of fillers in the post-filler density heat map to show the density of standard cells . *Default*: `false`
`-reset`	Resets parameters to their default values. The `-reset` parameter *must* be the first parameter specified. If you specify `-reset` by itself, the software resets all `setDensityMapMode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported.
`-threshold` *density*	Specifies a threshold density value for the grid. Any grid under the threshold value is not reported. The value must be positive. Cadence recommends an upper limit of `1.20`. *Default:* `0.75`

Examples

- The following command resets the `-gridInRow` parameter to its default value:
  ```
  setDensityMapMode -reset -gridInRow
  ```
- The following command resets all `setDensityMapMode` parameters to their default values:

```
setDensityMapMode -reset
```

- The following command sets density threshold to 0.25 for `reportDensityMap`:

```
setDensityMapMode -threshold 0.25
```

Related Information

- `getDensityMapMode`
- `reportDensityMap`

setEndCapMode

```
setEndCapMode
[-help]
[-avoidTwoSitesCellAbut {true|false}]
[-bottomEdge ListOfCellNames]
[-boundary_tap {true|false}]
[-boundary_tap_swap_flow {true|false}]
[-cells ListOfCellNames]
[-create_rows {true|false}]
[-DoubleHeightIncornerProtrusion {true|false}]
[-flipY {true|false}]
[-incrementalLeftEdge ListOfCellNames]
[-incrementalRightEdge ListOfCellNames]
[-leftBottomCorner ListOfCellNames]
[-leftBottomCornerEven cellName]
[-leftBottomCornerNeighbor cellName]
[-leftBottomCornerOdd cellName]
[-leftBottomEdge cellName]
[-leftBottomEdgeEven cellName]
[-leftBottomEdgeNeighbor site1_cell site2_cell...]
[-leftBottomEdgeOdd cellName]
[-leftCornerBottomBorder ListOfCellNames]
[-leftCornerTopBorder ListOfCellNames]
[-leftEdge cellName]
[-leftEdgeBottomBorder cellName]
[-leftEdgeEven cellName]
[-leftEdgeOdd cellName]
[-leftEdgeTopBorder cellName]
[-leftTopCorner ListOfCellNames]
[-leftTopCornerEven cellName]
[-leftTopCornerNeighbor cellName]
[-leftTopCornerOdd cellName]
[-leftTopEdge cellName]
[-leftTopEdgeEven cellName]
[-leftTopEdgeNeighbor site1_cell site2_cell...]
[-leftTopEdgeOdd cellName]
[-min_horizontal_channel_width channelWidth]
[-min_jog_height jogHeight]
[-min_jog_width jogWidth]
[-min_vertical_channel_width channelWidth]
[-prefix endcaprefix]
[-reset]
[-rightBottomCorner ListOfCellNames]
[-rightBottomCornerEven cellName]
[-rightBottomCornerNeighbor cellName]
[-rightBottomCornerOdd cellName]
[-rightBottomEdge cellName]
[-rightBottomEdgeEven cellName]
[-rightBottomEdgeNeighbor site1_cell site2_cell...]
[-rightBottomEdgeOdd cellName]
[-rightCornerBottomBorder ListOfCellNames]
[-rightCornerTopBorder ListOfCellNames]
[-rightEdge cellName]
[-rightEdgeBottomBorder cellName]
[-rightEdgeEven cellName]
[-rightEdgeOdd cellName]
```

```
[-rightEdgeTopBorder cellName]
[-rightTopCorner ListOfCellNames]
[-rightTopCornerEven cellName]
[-rightTopCornerNeighbor cellName]
[-rightTopCornerOdd cellName]
[-rightTopEdge cellName]
[-rightTopEdgeEven cellName]
[-rightTopEdgeNeighbor site1_cell site2_cell...]
[-rightTopEdgeOdd cellName]
[-topBottomEdge ListOfCellNames]
[-topEdge ListOfCellNames]
[-useEvenOddSite {none|even|odd}]
[-wall_keepout_from_vertical_boundary macro]
[-wall_offset macro]
[-wall_pitch macro]
[-wall_to_convex_corner_spacing macro]
```

Controls the behavior of `addEndCap` command. Specifies a list of cells that `addEndCap` command uses for end-cap or triple-well cap insertion.

Note: The options with *EVEN/*ODD are only for 10nm designs that need to fit even and odd sites. These options should not be used with the options that do not have *EVEN/*ODD.
For example, the `-rightEdge` option should not be used with the`-rightEdgeEven` or `-rightEdgeOdd` option.

Example for 10nm design if there is EVEN/ODD sites requirement:

```
setEndCapMode \
-rightEdgeOdd cellName \
-rightEdgeEven cellName \
-rightBottomCornerEven cellName \
-rightBottomCornerOdd cellName \
-rightTopCornerEven cellName \
-rightTopCornerOdd cellName \
-rightBottomEdgeEven cellName \
-rightBottomEdgeOdd cellName \
-rightTopEdgeEven cellName \
-rightTopEdgeOdd cellName \
-topEdge {ListOfCellNames} \
-bottomEdge {ListOfCellNames}
```

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setEndCapMode` parameter. For a detailed description of the command and all of its parameters, use the man command: man `setEndCapMode`.
`-avoidTwoSitesCellAbut {true|false}`	Avoids two sites boundary cell abut to incorner cell in N10. *Default*: `false`
`-bottomEdge ListOfCellNames`	Specifies the list of cells that have n-well cap on their bottom-edge when in R0 orientation. Thus, these cells are used to cap a top-core row, or row at the bottom of a hard macro.

`-boundary_tap`	Specifies the boundary tap and is applicable to `getEndCapMode`. The type is boolean. *Default*: `false`
`-boundary_tap_swap_flow {true|false}`	Specifies the boundary tap swap flow. *Default*: `True`
`-cells ListOfCellNames`	Specifies end cap cell candidates for N10 that are defined in LEF.
`-create_rows {true|false}`	Creates rows for endtap cell techSites which do not have rows in the floorplan. *Default*: `false`
`-DoubleHeightIncornerProtrusion {true|false}`	Allows the double height in-corner cell to protrude into core area. *Default*: `false`
`-flipY {true|false}`	Flips the orientation of the end cap instances in Y direction if the site symmetry allows it. *Default*: `false`
`-incrementalLeftEdge` *`ListOfCellNames`*	Specifies the list of additional cells which should be treated as end cap by `verifyEndCap` when checking the right edge of the core rows or rows ending at left edge of a hard macro. *Default*=""
`-incrementalRightEdge` *`ListOfCellNames`*	Specifies the list of additional cells which should be treated as end cap by `verifyEndCap` when checking the left edge of the core rows or rows starting at right edge of a hard macro. *Default*=""
`-leftBottomCorner` *`ListOfCellNames`*	Specifies a list of cells that have n-well cap on its left-bottom corner when in R0 orientation. These cells are added on inner corners of core, or concave corner of a rectilinear hard macro.
`-leftBottomCornerEven cellName`	Specifies end cap with n-well at the left-bottom corner, even sites. *Default*: `""`
`-leftBottomCornerNeighbor` *`cellName`*	Specifies the cell to be abutted with the left side of leftBottomCorner in R0 orientation. *Default*: `""`
`-leftBottomCornerOdd cellName`	Specifies end cap with n-well at the left-bottom corner, odd sites. *Default*: `" "`
`-leftBottomEdge` *`cellName`*	Specifies the one cell that has n-well cap on its left-bottom edge when in R0 orientation. Thus, this cell, in R0 orientation, can be used to cap a row ending at left-bottom convex corner of a hard macro, or rectilinear core; and if allowed to flip in Y, it can be used on MY rows that end in left-top convex corner. With both X and Y symmetry, it can cap all convex corners, though only R0 rows at bottom, and MY rows for top corners.

`-leftBottomEdgeEven` *cellName*	Specifies end cap with n-well at the left-bottom edge, even sites. *Default*: " "
`-leftBottomEdgeNeighbor` *site1_cell site2_cell...*	Specifies the list of cells to be abutted with the right edge of leftBottomEdge in R0 orientation. *Default*: " "
`-leftBottomEdgeOdd` *cellName*	Specifies end cap with n-well at the left-bottom edge, odd sites. *Default*: " "
`-leftCornerBottomBorder` *ListOfCellNames*	Specifies the inner left corner bottom border cell. *Default*: " "
`-leftCornerTopBorder` *ListOfCellNames*	Specifies the inner left corner top border cell. *Default*: " "
`-leftEdge` *cellName*	Specifies the one cell that has n-well cap on its left edge when in R0 orientation. Thus, this cell is used to cap right edge of the core rows, or rows ending at left edge of a hard macro. This would be same as a post-cap, in a two well process.
`-leftEdgeBottomBorder` *cellName*	Specifies end cap with n-well at the left edge, poly at the bottom boundary. *Default*: " "
`-leftEdgeEven` *cellName*	Specifies end cap with n-well at the left edge, even sites. *Default*: " "
`-leftEdgeOdd cellName`	Specifies end cap with n-well at the left edge, odd sites. *Default*: " "
`-leftEdgeTopBorder` *cellName*	Specifies end cap with n-well at the left edge, poly at the top boundary. *Default*: " "
`-leftTopCorner` *ListOfCellNames*	Specifies a list of cells that have n-well cap on its left top corner when in R0 orientation. Thus, this cell is added on inner corners of core, or concave corner of a rectilinear hard macro.
`-leftTopCornerEven` *cellName*	Specifies end cap with n-well at the left-top corner, even sites. *Default*: " "
`-leftTopCornerNeighbor cellName`	Specifies the cell to be abutted with the left side of leftTopCorner in R0 orientation. *Default*: " "
`-leftTopCornerOdd` *cellName*	Specifies end cap with n-well at the left-top corner, odd sites. *Default*: " "
`-leftTopEdge` *cellName*	Specifies the cell that has n-well cap on its top-left edge when in R0 orientation. Thus, in R0 orient, it is the flipped-in-Y version of left-bottom edge cell.
`-leftTopEdgeEven` *cellName*	Specifies end cap with n-well at the left-top edge, even sites. *Default*: " "

`-leftTopEdgeNeighbor` *site1_cell site2_cell...*	Specifies the list of cells to be abutted with the right edge of leftTopEdge in R0 orientation. *Default*: `" "`
`-leftTopEdgeOdd` *cellName*	Specifies end cap with n-well at the left-top edge, odd sites. *Default*: `" "`
`-min_horizontal_channel_width` *channelWidth*	If horizontal channel width is smaller than specified value, addEndCap will cover this channel with hard blockage.
`-min_jog_height` *jogHeight*	If jog height is smaller than specified value, addEndCap will cover this jog with hard blockage.
`-min_jog_width` *jogWidth*	If jog width is smaller than specified value, addEndCap will cover this jog with hard blockage.
`-min_vertical_channel_width` *channelWidth*	If vertical channel width is smaller than specified value, addEndCap will cover this channel with hard blockage.
`-prefix` *endcap_prefix*	Adds prefix of EndCap cells.
`-reset`	Resets all or specified options to the default value. Any values that do not already match the default value and are reset will be reported.
`-rightBottomCorner` *ListOfCellNames*	Specifies a list of cells that have n-well cap on its right bottom corner when in R0 orientation. Thus, this cell is added on inner corners of core, or concave corner of a rectilinear hard macro.
`-rightBottomCornerEven` *cellName*	Specifies end cap with n-well at the right-bottom corner, even sites. *Default*: `" "`
`-rightBottomCornerNeighbor cellName`	The cell to be abutted with the right side of rightBottomCorner in R0 orientation. *Default*: `" "`
`-rightBottomCornerOdd` *cellName*	Specifies end cap with n-well at the right-bottom corner, odd sites. *Default*: `" "`
`-rightBottomEdge` *cellName*	Specifies the cell that has n-well cap on its right bottom edge when in R0 orientation. Thus, these cells are the flipped-in-X version of leftBottomEdge cell. Usuallym one of these 2 is sufficient for a library.
`-rightBottomEdgeEven` *cellName*	Specifies end cap with n-well at the right-bottom edge, even sites. *Default*: `" "`
`-rightBottomEdgeNeighbor` *site1_cell site2_cell...*	Specifies the list of cells to be abutted with the left edge of rightBottomEdge in R0 orientation. *Default*: `" "`
`-rightBottomEdgeOdd` *cellName*	Specifies end cap with n-well at the right-bottom edge, odd sites. *Default*: `" "`

`-rightCornerBottomBorder` *ListOfCellNames*	Specifies the inner right corner bottom border cell. *Default*: " "
`-rightCornerTopBorder` *ListOfCellNames*	Specifies the inner right corner bottom border cell. *Default*: " "
`-rightEdge` *cellName*	Specifies the cell that has n-well cap on its right edge when in R0 orientation. Thus, these cells are used to cap the left edge of the core-rows in R0. In 2 well process, this would be the pre-cap cell.
`-rightEdgeBottomBorder` *cellName*	Specifies end cap with n-well at the right edge, poly at the bottom boundary. *Default*: " "
`-rightEdgeEven` *cellName*	Specifies end cap with n-well at the right edge, even sites. *Default*: " "
`-rightEdgeOdd` *cellName*	Specifies end cap with n-well at the right edge, odd sites. *Default*: " "
`-rightEdgeTopBorder` *cellName*	Specifies end cap with n-well at the right edge, poly at the top boundary. *Default*: " "
`-rightTopCorner` *ListOfCellNames*	Specifies a list of cells that have n-well cap on their right top corner when in R0 orientation. Thus, this cell, in R0, is added on right corner of core, or concave corner of a rectilinear hard macro.
`-rightTopCornerEven` *cellName*	Specifies end cap with n-well at the right-top corner, even sites. *Default*: " "
`-rightTopCornerNeighbor` *cellName*	Specifies the cell to be abutted with the right side of rightTopCorner in R0 orientation. *Default*: " "
`-rightTopCornerOdd` *cellName*	Specifies end cap with n-well at the right-top corner, odd sites. *Default*: " "
`-rightTopEdge` *cellName*	Specifies the cell that has n-well cap on its right top edge when in R0 orientation. Thus, this cell is the R180 version of leftBottomEdge.
`-rightTopEdgeEven` *cellName*	Specifies end cap with n-well at the right-top edge, even sites. *Default*: " "
`-rightTopEdgeNeighbor` *site1_cell site2_cell...*	Specifies the list of cells to be abutted with the left edge of rightTopEdge in R0 orientation. *Default*: " "
`-rightTopEdgeOdd` *cellName*	Specifies end cap with n-well at the right-top edge, odd sites. *Default*: " "

`-topBottomEdge` *`ListOfCellNames`*	Specifies the cell that has an n-well cap on its top edge as well as bottom edge when in R0 orientation.
`-topEdge` *`ListOfCellNames`*	Specifies the list of cells that have n-well cap on their top edge when in R0 orientation. Thus, these cells are used to cap the bottom-core row, or row at top of a hard macro.
`-useEvenOddSite {none|even|odd}`	Specifies to use even or odd poly sites. • `none`: EVEN/ODD cells are not considered even if they are specified • `even`: Need to specify EVEN boundary cell, suc as `-leftEdgeEven` • `odd`: Need to specify ODD boundary cell, such as -l`eftEdgeOdd` *Default*: `none`
`-wall_keepout_from_vertical_boundary` *`macro`*	Specifies the wall keep-out spacing from left/right boundary cell. *Default*: 0 *Minimum*: 0 *Maximum*: 2147483647
`-wall_offset` *`macro`*	Specifies the x coordinate of the first wall column. *Default*: 0 *Minimum*: 0 *Maximum*: 2147483647
`-wall_pitch` *`macro`*	Specifies the wall pitch. *Default*: 0 *Minimum*: 0 *Maximum*: 2147483647
`-wall_to_convex_corner_spacing` *`macro`*	Constraints the spacing between wall and convex corner. *Default*: 0 *Minimum*: 0 *Maximum*: 2147483647

Example

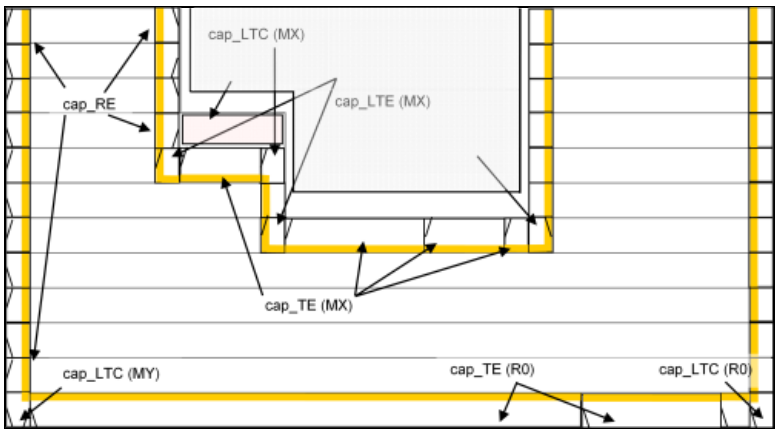
- The following command controls that the cell ABC being placed at the end of each row and cell XYZ being placed at beginning of each row by `addEndCap` command:

  ```
  setEndCapMode -leftEdge ABC -rightEdge XYZ
  ```

- An example of a library with 4 types of cells, `RightEdge RE`, `TopEdge TE`, `LeftTopEdge LTE`, and `LeftTopCorner LTC`, would have the following placement of end caps:
  - Pre-caps for start of rows would be `cap_RE` in `R0 (N)` and `MX (FS)` orient.
  - Post-caps for end of rows would be `cap_RE` in `MY (FN)` and `R180 (S)` orient.
  - Caps for bottom of core, and in core-row on top of macro blocks would be `cap_TE` in `R0 (N)` orient (i.e. core has to start from `R0` row; else the bottom `FS` row would be wasted).
  - Caps for top of core, and at bottom of macro blocks would be `cap_TE` in `MX (FS)` orient (that is, if block gets placed on a `FS` row, and row below it is `N` row, then the `N` row has to be blocked so caps can be placed below it on `FS` orient row).

- A `cap_LTE` cell can be used at 4 corners in `R0` (left-bottom), `MY` (right-bottom), `R180` (right-top), and `MX` (left-top) orient.
- Similarly, a `cap_LTC` cell is used for inner edges of a rectilinear block macro in its 4 orients, just as the corner cell.

All the cells need to have `SYMMETRY X Y` defined in LEF.



Related Information

- `addEndCap`
- `getEndCapMode`
- `setPlaceMode`
- `verifyEndCap`

setFillerMode

```
setFillerMode
[-help]
[-reset]
[-add_fillers_with_drc {true|false}]
[-avoid_abutment_patterns { 1:1 1:2 ...}]
[-check_signal_drc {true | false}]
[-core {{list_of_cells1} {list_of_cells2} ...}]
[-corePrefix prefix]
[-createRows {true | false}]
[-keepFixed {true | false}]
[-diffCellViol {true | false}]
[-distribute_implant_evenly {true|false}]
[-ecoMode {true | false}]
[-fitGap {true | false}]
[-honorPrerouteAsObs {true | false}]
[-horizontal_exception_cell {fillerCell ...}]
[-horizontal_max_length instance_length]
[-horizontal_repair_cell {fillCell ...}]
[-preserveUserOrder {true | false}]
[-scheme {locationFirst | cellFirst}]
[-swap_cell {{cell swap pair1} {cell swap pair2} ...}]
[-vertical_stack_exception_cell ListOfCellNames}]
[-vertical_stack_left_edge_exception_cell ListOfCellNames]
[-vertical_stack_max_length instance_length]
[-vertical_stack_repair_cell {fillerCell ...}]
[-vertical_stack_repair_edge {left|right}]
[-vertical_stack_right_edge_exception_cell <ListOfCellNames>]
[-y_flip_type { integer : { first | last } ... }]
```

Controls certain aspects of how the software adds filler cells. The mode setting is persistent and saved along with the database by the `saveDesign` command.

Use `getFillerMode` to return the current settings for the `setFillerMode` command.

The `setFillerMode` parameters affect the behavior of the following commands:

- `addFiller`
- `deleteFiller`

  Note: Parameters specified by the `addFiller` or `deleteFiller` command supersede those specified by the `setFillerMode` command.

Parameters

`-add_fillers_with_drc {true|false}`	Calls the second addFiller without DRC checking. *Default:* `true` **Note**: When `-add_fillers_with_drc` is `true`, only DRC checking is switched off with routing in the second pass `addFiller` for the opt stage.
`-avoid_abutment_patterns { 1:1 1:2 ...}]`	Specifies illegal abutment patterns.

`-check_signal_drc {true | false}`	When `addFiller` is called after routing, instructs the software to run DRC checks of filler pins with signal net wires. To disable the DRC checks, set this parameter to `false`. *Default:* `true`
`-core {{list_of_cells1} {list_of_cells2} ...}`	Specifies the filler cells to add with `addFiller`. Enclose the filler cell names in quotation marks (") or curly braces ({ }). You can specify multiple cell lists at one time, using curly braces ({ }) to separate each one. *Default:* `""` (empty string)
`-corePrefix` *prefix*	Specifies the prefix for the added filler cells. *Default:* `""` (empty string)
`-createRows {true | false}`	Instructs `addFiller` to create rows in the floorplan. *Default:* `true`. If a cell's site does not have rows in the floorplan, `addFiller` creates the rows.
`-keepFixed {true | false}`	Keeps the `FIXED` filler cells. *Default:* `false`
`-diffCellViol {true|false}`	Checks violations between different cell types. Default: `false` **Note:** Using this feature will increase the runtime of `addFiller`.
`-distribute_implant_evenly {true|false}`	Distributes implant evenly for fillers. *Default*: `false`
`-ecoMode {true | false}`	Resolves overlaps of standard cells with fillers by removing the overlapping fillers and inserting new fillers in existing or newly created gaps due to partial overlaps. If you use this parameter after routing, the command also resolves DRC violations of filler geometries of all fillers with regular wires. *Default:* `false`
`-fitGap {true | false}`	Fills a gap between cells by adding a combination of cells, instead of by adding the single largest cell that fits, if doing so avoids leaving an unfilled single-width gap. *Default:* `true`
`-help`	Outputs a brief description that includes type and default information for each `setFillerMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setFillerMode`.

`-honorPrerouteAsObs {true | false}`	Instructs `addFiller` to honor the setting of `setPlaceMode -place_detail_preroute_as_obs`. By default, `addFiller` does not consider any preroutes to be obstructions. When this parameter is `true`, `addFiller` treats preroutes on metal layers where filler cells have geometries as obstructions, and ignores preroutes on other layers. In most cases, this means that `addFiller` treats preroutes on *metal1* as obstructions and ignores preroutes on other metal layers. For example, if `setPlaceMode -place_detail_preroute_as_obs` is set to *metal1* and *metal2*, and all the filler cell geometries are on *metal1*, `addFiller` treats *metal1* preroutes as obstructions but does not treat *metal2* preroutes as obstructions. *Default:* `false`. `addFiller` ignores `setPlaceMode -place_detail_preroute_as_obs` and inserts fillers in all legal sites.
`-horizontal_exception_cell {fillerCell ...}`	Specifies the list of exceptional cells for horizontal stack checking. *Default*: `""`
`-horizontal_max_length` *instance_length*	Specifies the instance max length (in micron) aligned in the horizontal direction. *Default*: `100` *Min*: `0` *Max*: `50000`
`-horizontal_repair_cell {fillCell ...}`	Specifies the list of filler cells to fix the horizontal max length violation. *Default*: `""`
`-preserveUserOrder {true | false}`	Specifies whether using the user defined filler cells order for filler insertion. Default: `false` Filler cells in each core list are sorted from largest to smallest. **Note**: The `-fitGap` and `-preserveUserOrder` options are mutually exclusive.
`-scheme {locationFirst | cellFirst}`	Specifies the filler insertion scheme. Default: `locationFirst` `locationFirst` fills each given location with the best filler before proceeding to next location. It always starts with the first empty site. `cellFirst` inserts filler cells based on the user-specified list. The tool tries to insert the first filler from the user list in all possible locations before moving to the second cell in the list.
`-swap_cell {{cell swap pair1} {cell swap pair2} ...}`	Swaps the specified cell pairs. *Default*: `""`
`-vertical_stack_exception_cell` *ListOfCellNames*	Specifies the list of exceptional cells for vertical stack checking. *Default*: `""`
`-vertical_stack_left_edge_exception_cell` *ListOfCellNames*	Specifies a list of cells in which only the left edge will not be checked for vertical stack violation. *Default*: `""`

`-vertical_stack_max_length` *instance_length*	Specifies the instance max length (in micron) aligned in the vertical direction. *Default*: `50` *Min*: `0` *Max*: `20000`
`-vertical_stack_repair_cell {fillerCell ...}`	Specifies the list of filler cells to fix the vertical max length violation. *Default*: `""`
`-vertical_stack_repair_edge {left|right}`	Specifies the edge to use the vertical repair cell. *Default*: `left`
`-vertical_stack_right_edge_exception_cell` *ListOfCellNames*	Specifies a list of cells in which only the right edge will not be checked for vertical stack violation. *Default*: `""`
`-reset`	Resets parameters to their default values. The `-reset` parameter *must* be the first parameter specified. If you specify `-reset` by itself, the software resets all `setFillerMode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported.
`-y_flip_type { integer : { first | last } ... }`	Defines the first or the last cell with certain cell size should be y-flipped. *Default*: `""`

Examples

- The following command assigns a prefix of `ABC` to filler instances of cells `FILL64` and `FILL8` when `addFiller` inserts the instances:
  ```
  setFillerMode -corePrefix ABC -core "FILL64 FILL8"
  ```
- The following command resets all `setFillerMode` parameters to their default values:
  ```
  setFillerMode -reset
  ```
- The following command make addFiller insert filler cells based on the user specified cell order `{FILLER8 FILLER16 FILLER4}`.
  ```
  setFillerMode -core {FILLER8 FILLER16 FILLER4} -preserveUserOrder true
  ```

Related Information

- `addFiller`
- `deleteFiller`
- `getFillerMode`
- `setPlaceMode`

setPinDensityMapMode

```
setPinDensityMapMode
[-help]
[-reset]
[-displayStep step_value]
[-gridInMicron micron]
[-gridInRow numberRow]
[-threshold density]
```

Sets global parameters for the `reportPinDensityMap` command. Parameters that you specify with `setPinDensityMapMode` are then used when you run `reportPinDensityMap`.

⚠ Parameters specified by the `reportPinDensityMap` command supersede the parameters specified by the `setPinDensityMapMode` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setPinDensityMapMode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setPinDensityMapMode`
`-reset`	Resets all or specified options to default value. Any values that do not already match the default value and are reset will be reported.
`-displayStep step_value`	Specifies the range step for displaying the pin density map calculations. *Default:* `.05`
`-gridInMicron micron`	Specifies the value (in micron) of side of the square grid for pin density calculations. *Default:* `50` *Minimum:* `0.000000` *Maximum:* `1000.000000`
`-gridInRow numberRow`	Specifies the value (in row-height) of side of the square grid for pin density calculations. *Default:* `10` *Minimum:* `1` *Maximum:* `100`
`-threshold density`	Specifies the threshold value over which the densities must be reported. *Default*: `0.5`

Related Information

- `getPinDensityMapMode`
- `reportPinDensityMap`

setPlaceMode

```
setPlaceMode
[-help]
[-reset]
[-place_design_floorplan_mode {true|false}]
[-place_design_integrity_ir_fix_effort {low|medium|high}]
[-place_design_refine_macro {true|false}]
[-place_design_refine_place {true|false}]
[-place_detail_activity_power_driven {true|false}]
[-place_detail_allow_border_pin_abut {true | false}]
[-place_detail_allow_single_height_row_symmetry_x siteName]
[-place_detail_check_cut_spacing {true|false}]
[-place_detail_check_inst_space_group {true|false}]
[-place_detail_check_route {true | false}]
[-place_detail_color_aware_legal {true|false}]
[-place_detail_context_aware_legal {none|all|optional|required|user}]
[-place_detail_eco_max_distance maxDistance]
[-place_detail_eco_priority_insts {placed|fixed|eco}]
[-place_detail_fixed_shifter {true|false}]
[-place_detail_honor_inst_pad {true|false}]
[-place_detail_io_pin_blockage {true|false}]
[-place_detail_iraware_max_drive_strength maxDriveStrength]
[-place_detail_irdrop_aware_effort {none|low|medium|high}]
[-place_detail_irdrop_aware_timing_effort {none|standard|high}]
[-place_detail_irdrop_region_number valueOfDebugIrdrop]
[-place_detail_legalization_inst_gap numberOfSites]
[-place_detail_max_shifter_column_depth value]
[-place_detail_max_shifter_depth value]
[-place_detail_max_shifter_row_depth value]
[-place_detail_no_filler_without_implant {true|false}]
[-place_detail_pad_fixed_insts {true|false}]
[-place_detail_pad_physical_cells {true|false}]
[-place_detail_preroute_as_obs {layerNum ...}]
[-place_detail_preserve_routing {true|false}]
[-place_detail_remove_affected_routing {true|false}]
[-place_detail_swap_eeq_cells {true|false}]
[-place_detail_use_check_drc {true|false}]
[-place_detail_use_diffusion_transition_fill {true|false}]
[-place_detail_use_GA_filler_groups {true|false}]
[-place_detail_use_no_diffusion_one_site_filler {true|false}]
[-place_detail_wire_length_opt_effort {none|medium|high}]
[-place_global_activity_power_driven {false|true}]
[-place_global_activity_power_driven_effort {none|standard|high}]
[-place_global_align_macro {true|false}]
[-place_global_allow_3d_stack {true|false}]
[-place_global_auto_blockage_in_channel {none | soft | partial}]
[-place_global_clock_gate_aware {true|false}]
[-place_global_clock_power_driven {true|false}]
[-place_global_clock_power_driven_effort {low|standard|high}]
[-place_global_cong_effort {low|medium|high|auto}]
[-place_global_cpg_effort {low|medium|high}]
[-place_global_cpg_file macro_loc_ori_file_name]
[-place_global_enable_advanced_pipeline {true|false}]
[-place_global_enable_distributed_place {true|false}]
[-place_global_ignore_scan {true|1|false|0|auto}]
[-place_global_ignore_spare {true|false}]
```

```
[-place_global_max_density value]
[-place_global_module_aware_spare {true|false}]
[-place_global_module_padding module factor]
[-place_global_place_io_pins {true|false}]
[-place_global_reorder_scan {true|false}]
[-place_global_soft_guide_strength {low|medium|high}]
[-place_global_timing_effort {medium|high}]
[-place_global_uniform_density {false|true}]
[-place_hard_fence {true|false}]
[-place_hierarchical_flow {true|false}]
[-place_opt_post_place_tcl tcl_file]
[-place_opt_run_global_place {none|seed|full}]
[-place_spare_update_timing_graph {true|false}]
```

Controls certain aspects of how the software places cells.

Use the `getPlaceMode` command to return the current settings for the `setPlaceMode` command.

The `setPlaceMode` parameters affect the behavior of the following commands:

- `place_design`
- `refinePlace`

  Note: Parameters specified by the `place_design` or `refinePlace` commands supersede those specified by the `setPlaceMode` command.

Parameters

`-place_design_floorplan_mode {true | false}`	Runs placement in floorplan mode. This mode is used for prototyping and runs quickly to gauge the feasibility of the netlist, but might not place design components in legal locations. **Note:** When `-place_design_floorplan_mode` is set to `true`, it disables congestion-driven placement. Option `-place_global_cong_effort` is set to `low`. *Default:* `false`
`-place_design_integrity_ir_fix_effort {low|medium|high}`	Sets the power integrity placement IR fixing effort. *Default:* `low`
`-place_design_refine_macro {true|false}`	Specifies whether to run `refine_macro_place` after `place_design -concurrent_macros`. When `place_design -concurrent_macros` is run, it sets the value of `setPlaceMode -place_opt_run_global_place` to `seed`. *Default:* `false`
`-place_design_refine_place {true | false}`	Specifies whether to run detailed placement (`refinePlace`) to legalize placed standard cells. If `setDesignMode -powerEffort` is set to `high`, `-place_design_refine_place` automatically set to `true`. *Default:* `true`

`-place_detail_activity_power_driven {true|false}`	When `true`, it turns on the activity-driven tweakage, and flow change depends on the effort. *Default:* `false`
`-place_detail_allow_border_pin_abut {true | false}`	Some libraries have cells with pin geometry that is not set back by half-routing pitch from the normal PR-boundary of that cell. In cases where the pin geometries touch the cell PR_boundary, placement of an instance of this cell can cause a DRC violation with a normal pin geometry of the other cells. To avoid these DRCs, placer will add one Site padding to the cell-side during pre-processing, thus satisfying the min-spacing rules. The `-place_detail_allow_border_pin_abut` option disables this pre-emptive DRC avoidance behavior, and no auto-padding is done when the option is set to true. *Default*: `false`
`-place_detail_allow_single_height_row_symmetry_x` *`siteName`*	Specifies a single height row tech-site, which allows symmetry X during detail place. *Default:* `""`
`-place_detail_check_cut_spacing {true | false}`	Enables the via-to-via spacing check during `refinePlace` and `checkPlace`. *Default*: `false` This option is turned on by default for 16nm and below process designs.
`-place_detail_check_inst_space_group {true|false}`	Specifies whether the distance between instances that belong to the same space group would be honored during placement. *Default:* `false`
`-place_detail_check_route {true | false}`	Considers pre-routed `FIXED` signal wires and avoids creating DRC violations between `FIXED` signal wires and instance pins. When this parameter is not specified, `refinePlace` ignores `FIXED` signal wires. *Default:* `false` **Note:** Using this parameter might result in longer run time. If you specify `refinePlace -place_detail_io_pin_blockage`, you do not need to specify `-place_detail_check_route` for signal pins because `refinePlace` will treat the signal pins similar to supply pins for blockages. However, signal wires still need this parameter.

`-place_detail_color_aware_legal {true | false}`	Enables placer to honor and fix double pattern constraint violations between adjacent cells. *Default*: `false` **Note**: This option does not affect cell to pre-route color DRC checks.
`-place_detail_context_aware_legal {none|all|optional|required|user|ignore_soft}`	Instructs placer to support cell edge to cell edge spacing constraints as defined in the LEF files. The `optional`, `required`, `user`, `ignore_soft` argument values can be used separately or together, but they are exclusive to `all` and `none`. *Default*: `all`
`-place_detail_eco_max_distance` *maxDistance*	Specifies max distance, in microns, in the horizontal or vertical direction for the refinePlace ECO mode. For example, if the original location of the instance is at (x0 y0) and the specified eco max distance is M, the search window will be (x0-M, yo-M) (x0+M y0+M). The maximum movement should be less than 2M. *Default:* `0` *Minimum*: 0.000000 *Maximum*: 9999.000000 If `-place_detail_eco_max_distance` is not defined by the user, and the user does `refinePlace -eco`, the value of `-place_detail_eco_max_distance` is internally set to 10 Standard Cell height units. **Note**: This option will impact both `refinePlace -eco` and `refinePlace -inst`.
`-place_detail_eco_priority_insts {placed|fixed|eco}`	Specifies the instance priority for `refinePlace` ECO mode. • `placed`: Treats current instances as placeable but with higher priority. • `fixed`: Treats placed instances as fixed. • `eco`: Treats ECO instances with higher priority. *Default:* `placed`
`-place_detail_fixed_shifter {true | false}`	Specifies whether to set the shifters to `FIXED` status after placing them. When set to `true`, the level shifters are `FIXED` after placement. However, the placement for some of the level shifters can be skipped due to the following reasons: • The shifter has no connections outside its power domains. • The shifter has no valid location. *Default:* `false`

`-place_detail_honor_inst_pad {true|false}`	Honors padding from `specifyInstPad` in detail placement. *Default:* `false` When this option is set to `true`, instance padding is also considered in `refinePlace` and `checkPlace`.
`-place_detail_io_pin_blockage {true|false}`	Treats ioPins from top-level as pre-routes during DRV checks in legalization. *Default:* `false`
`-place_detail_iraware_max_drive_strength` *maxDriveStrength*	Specifies the current density value for the IR critical instances above which the instances gets padded during `refinePlace`. This means that the IR critical instances with the current density value more than the specified value gets padded. **Note**: There may be larger cell movement with this option, which can degrade the timing. So, it is preferred to be used at the pre-CTS stage in the flow. The current density values for the IR-critical instances are dumped in the Voltus Reports. *Default*: `0` *Minimum*: `0` *Maximum*: `2147483647`
`-place_detail_irdrop_aware_effort {none|low|medium|high}`	Used to enable the IR-aware feature in refinePlace. You can use the following settings to specify IR-drop optimization effort level to be used during `refinePlace`. You can specify the following optimization effort levels: • `none`: Disables the feature, default value. • `low`: Moves the IR-violating instances to nearby rows (in the vertical direction). The aggressor has the same priority as the other instances during legalization. • `medium`: : Moves the IR violating instances in both horizontal and vertical directions. The aggressor has higher priority than the other instances during legalization. • `high`: Moves the aggressor instances to nearby rows. The aggressor has higher priority than the other instances during legalization and considers the IR-drop violation as a hard rule during legalization. This option is not recommended in this version. *Default*: `none`

`-place_detail_irdrop_aware_timing_effort {none|standard|high}`	Specifies the timing-effort level to be used during IR-aware `refinePlace`. You can specify the following timing effort levels: • `none`: Disables the feature, refineplace is not timing aware. Tool uses the default value. • `medium`: `refineplace` is timing aware and minimizes the movement of the timing sensitive instances during `refineplace`. • `high`: `refineplace` is timing aware and does not move the timing sensitive instances during `refineplace`. *Default*: `high`
`-place_detail_irdrop_region_number` *valueOfDebugIrdrop*	Specifies the total number of regions in which the tool identifies the IR-violating instances. The tool dumps the IR-violating instances inside these regions in the reports. *Default:* `100`
`-place_detail_legalization_inst_gap` *numberOfSites*	Specifies the minimum sites gap between instances. *Default:* `0`
`-place_detail_max_shifter_column_depth` *microns*	Specifies the maximum distance from a vertical side that a shifter can be placed. *Default:* `999` (no effect)
`-place_detail_max_shifter_depth` *microns*	Specifies the maximum distance from a horizontal or vertical side that a shifter can be placed. *Default:* `999` (no effect)
`-place_detail_max_shifter_row_depth` *microns*	Specifies the maximum distance from a horizontal side that a shifter can be placed. The distance is measured from the power domain boundary, not from the edge of the placeable areas and accounts for gaps. *Default:* `999` (no effect)
`-place_detail_no_filler_without_implant {true|false}`	Specifies that there is no cell without the implant obs. For designs in which all standard cells and fillers have implant obs, you can specify this option to preserve enough spacing during refinePlace for later filler insertion. *Default:* `false`
`-place_detail_pad_fixed_insts {true | false}`	Specifies whether the `refinePlace` command honors cell padding on `FIXED` instances. *Default:* `false`
`-place_detail_pad_physical_cells {true|false}`	Honors cell-padding for physical instances in `refinePlace` and `addFiller`. *Default:* `false`

`-place_detail_preroute_as_obs {`*`layerNum ...`*`}`	Specifies layers on which preroute are taken as OBS. You can change this behavior by using the following command before running placement: `setPlaceMode -place_detail_preroute_as_obs` *Default:* `""` **Note**: This option is not recommended for Advanced nodes of 20nm and below.
`-place_detail_preserve_routing {true | false}`	Preserves all routed wires. This option is applicable to all the calls in `refinePlace`, either stand-alone, or internal to place_opt_design. *Default:* `true`. `refinePlace` deletes all routed wires.
`-place_detail_remove_affected_routing {true | false}`	Specifies whether `refinePlace` removes wires connected to moved cells only or to all cells. *Default:* `false`. Removes all wires, whether connected cells were moved or not.
`-place_detail_swap_eeq_cells {true | false}`	Determines whether master cells can be replaced by EEQ cells during detailed placement to improve routability. After detailed placement, the software reports the number of replacements. EEQ cells are defined in the LEF file. *Default:* `false`
`-place_detail_use_check_drc {true|false}`	Enables the FGC engine inside placement for DRC checking. This DRC engine is the same as one used by NanoRoute and `verify_drc`. *Default*: `false`
`-place_detail_use_diffusion_transition_fill {true|false}`	Specifies that the design has fillers with jogged diffusion obs . For designs where fillers with jogged diffusion obs are to be inserted between cells with varying diffusion widths, please specify this option. *Default*: `false`
`-place_detail_use_GA_filler_groups {true|false}`	Specifies that filler cell insts will only be replaced by logic insts that belong to the same GA groups. *Default*: `false`
`-place_detail_use_no_diffusion_one_site_filler {true|false}`	Specifies that all one-site fillers do not have diffusion obs. For designs in which all one-site fillers without diffusion obs, you can use this option to preserve enough spacing during refinePlace for later filler insertion. This option can override the setting of `-place_detail_legalization_inst_gap` to `0`. *Default*: `false`

`-place_detail_wire_length_opt_effort {none|medium|high}`	Optimizes wire length by swapping cells. Optimization can reduce the total wire length up to two percent without degrading placement. In high-effort mode, it might add up to ten percent to the run time. *Default*: `medium`
`-place_global_activity_power_driven {false|true}`	Identifies and constrains power-critical nets to reduce switching power. *Default*: `false`
`-place_global_activity_power_driven_effort {none|standard|high}`	Specifies `place_global_activity_power_driven_effort` mode. *Default*: `standard`
`-place_global_align_macro {true|false}`	Enables mixed-placer to align nearby same-size macros during the global placement. *Default*: `true`
`-place_global_allow_3d_stack {true|false}`	Enables the 3D placement. *Default*: `false`
`-place_global_auto_blockage_in_channel {none | soft| partial}`	Creates placement blockages in narrow channels between macros before Global Placement • `soft`: Adds physical soft blockages by calling the `finishFloorplan` command. • `partial`: Creates a partial blockage whose density will be determined automatically. • `none`: Prevents the creation of blockage automatically by placer. *Default*: `none` **Note:** The "soft" option only prevents cells being placed inside the channels during global placement. refinePlace called by optimization can still place the cells inside the channels. **Note:** The threshold to activate the soft blockage insertion when the option is set to "soft" is when the channels between macros are less than 50 um in width.
`-place_global_clock_gate_aware {true | false}`	Specifies that placement is aware of clock gate cells in the design. *Default*: `true`. **Note:** `-place_global_clock_gate_aware` now finds better placement for clock gating elements towards the center of gravity for fanout.
`-place_global_clock_power_driven {true|false}`	Improves clock wirelength and subsequently clock power by adding more weight to clock nets between clock gates and their respective Flops. *Default*: `true`

`-place_global_clock_power_driven_effort {low|standard|high}`	Specifies the place_global_clock_power_driven_effort mode. *Default:* `low`
`-place_global_cong_effort {low | medium | high |auto}`	Specifies the effort level for relieving congestion. *Default:* `auto` Specify one of the following values: • `auto`: Automatically determines whether the design is congested and performs extra congestion driven effort for highly congested designs. • `high`: Runs more iterations of placement in an effort to achieve better congestion results. This parameter increases the run time. • `low`: Runs fewer iterations of placement to arrive quickly at a legal placement. This parameter might decrease placement quality. • `medium`: Runs placement on designs at a normal effort level.
`-place_global_cpg_effort {low|medium|high}`	Specifies the effort level of pushing instances to the guided locations during the CPG flow. With higher level, placement will increase the force to push instances to the guided locations, but might degrade wire length and timing QOR **Note**: This option need to be specified together with `-place_global_cpg_file`. *Default*: `low`
`-place_global_cpg_file` *cpgFileName*	Specifies the CPG guide file. When this option is specified, the CPG flow is turned ON automatically. *Default*: "" **Note**: To run the CPG flow, you must save a guide file from the old version DB and read it during the mixed placement flow on the new version netlist.
`-place_global_enable_advanced_pipeline {true|false}`	Enables advanced pipeline placement. *Default*: `false`
`-place_global_enable_distributed_place {true|false}`	Enables distributed placement platform. *Default*: `false` **Note**: This option is part of a limited-access feature in this release. It is enabled by a variable specified using the `setLimitedAccessFeature` command. To use this feature, contact your Cadence representative to explain your usage requirements, and make sure this feature meets your needs before deploying it widely.

`-place_global_ignore_scan{true|1|false|0|auto}`	Disregards scan connections while placing the instances that belong to scan chains. Before you place the design, you must already have specified the scan cells with the `specifyScanCell` command, or the scan cell information must be in the timing library. *Default*: `true` **Note:** Scan groups are sometimes called scan partitions.
`-place_global_ignore_spare {true | false}`	Disregards spare cell connections while placing spare cells. Before you place the design, you must already have specified the spare cells with the `specifySpareGate` command, or the spare cell information must be in the timing library. *Default*: `false`. `place_design` ignores spare cell connections. **Note:** The placer disconnects high-fanout (nets with more than 75 terminals) from spare cells, even when this parameter is set to `false`.
`-place_global_max_density` *value*	Controls the maximum density of local bins during global placement. Sets the maximum density of core area so that the placement engine satisfies the maximum density constraint while working on timing, wire length, and congestion. Use a value between 0 (zero percent) and 1 (100 percent). For example, a value of 0.5 indicates that the maximum density in the entire design will be 50 percent or less. *Default*: `-1.000` **Note**: It is recommended that the `place_global_max_density` value should be greater than standard cell utilization of design. **Note**: Using this option might degrade congestion and timing QOR of Global placement.
`-place_global_module_aware_spare {true | false}`	Places spare cells in the netlist within a hierarchical module, even if no region or fence constraints are specified. • When this parameter is `true`, spare cells in a logical hierarchy are placed within the bounds of the hierarchy. • When this parameter is `false`, spare cells are spread out within the placement area, and are not bound to a hierarchical module. *Default:* `false`

`-place_global_module_padding` *`module factor`*	Specifies a module in which to add padding (placement clearance) and a factor to use to calculate the padding dimension. Adding padding reduces placement density and localized congestion (hotspots) by spreading out cell instances in the specified modules. Module padding provides guidance for global placement only, and is ignored during placement legalization (`refinePlace`). *Default:* `""` (empty string) • `module`: Specifies a hierarchical instance to which the factor applies. You can use "prefix matching" when you specify the instance name. For example, you can use the following command to specify the module padding for all u_ram_block_{2,4,6} module. `setPlaceMode -place_global_module_padding {u_ram_block_ 2.0}` • `factor`: Specifies a factor to use to calculate the padding dimension. The placer multiplies the instance area of all cell instances under the specified module by the factor. For example, a factor of 2 means that the placer "sees" each cell as twice its actual size. In most cases, a factor of 1.2 (which increases the area by 20 percent) is good enough. The placer ignores factors that are less than 1.0. *Default*: `1`
`-place_global_place_io_pins {true | false}`	Moves placed and unplaced I/O pins, based on the placement of connected instances in an attempt to find a better I/O pin placement than the one specified in the I/O pin placement or floorplan file. At the end of global placement, the I/O pin location is legalized. When the value of this parameter is `false`, I/O pins are ignored during global placement and no legalization is done at the end. *Default:* `false` ⚠ If `place_design` does not place the standard cells, for example if all instances are fixed or if there is no placeable area, then `place_design` also skips I/O pin assignment.
`-place_global_reorder_scan {true | false}`	Performs scan chain reordering after placement. If you specify `false` for this parameter, the software does not perform scan chain reordering after placement. *Default*: `true`

`-place_global_soft_guide_strength {low | medium | high}`	Instructs placement engine to place those modules which have been specified as softguide in the floorplan file closer. The degree of closeness depends on the strength given. In case, there are fixed/preplaced instances, the module placed gets affected by those fixed instances and therefore placed close to them. *Default:* `low`
`-place_global_timing_effort {medium|high}`	Specifies the level of effort for timing driven global placer. **Note**: `setPlaceMode -place_global_timing_effort high` is not recommended for congested designs. *Default*: `medium` ⚠ This option is part of a limited-access feature in this release. It is enabled by a variable specified using the `setLimitedAccessFeature` command. To use this feature, contact your Cadence representative to explain your usage requirements, and make sure this feature meets your needs before deploying it widely.
`-place_global_uniform_density {true | false}`	Enable even cell distribution for designs with less than 70% utilization. *Default*: `false`
`-place_hard_fence {true|false}]`	Specifies whether `refinePlace` and `checkPlace` honor or ignore the fence and region placement constraints while legalizing placed instances. *Default:* `true` (`refinePlace` and `checkPlace` honor the constraints) **Note**: In optimization, the fence and region constraints are disabled by default in non-MSV designs for better timing enclosure, which is controlled by `setOptMode -opt_honor_fences` with the `false` default value. For the fence and region constraints to be honored by optimization commands, both `setOptMode -opt_honor_fences` and `setPlaceMode -place_hard_fence` must be set to `true`.
`-place_hierarchical_flow {true|false}`	Enables hierarchical pin assignment flow. *Default*: `false`
`-place_opt_post_place_tcl` *`tcl_file`*	Sources the tcl script after placement and before running optimization inside `place_opt_design`. *Default:* `""`

`-place_opt_run_global_place {none|seed|full}`	Changes the Global Placement behavior inside `place_opt_design`. • `none`: Skips place_design and runs only preCTS Optimization inside place_opt_design. • `seed`: Uses seed placement from Concurrent Macro Placement or other sources and runs incremental standard cell placement, then runs preCTS optimization. • `full`: Runs full global placement and preCTS optimization. *Default:* `full` When `place_design -concurrent_macros` is run, it sets the value of `setPlaceMode -place_opt_run_global_place` to `seed`.
`-place_spare_update_timing_graph {true|false}`	Updates the timing graph during `placeSpareModule`. *Default:* `true`
`-reset`	Resets parameters to their default values. The `-reset` parameter *must* be the first parameter specified. If you specify `-reset` by itself, the software resets all `setPlaceMode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported.

Examples

- The following commands add padding to the cells in module `ABC` so that the placer sees them as 1.5 times (50 percent larger) than their actual area:

```
setPlaceMode -place_global_module_padding ABC 1.5
place_design
```

- The following command resets all `setPlaceMode` parameters to their default values:

```
setPlaceMode -reset
```

- The following command will enable `place_design` to improve congestion:

```
setPlaceMode -place_global_cong_effort high -place_global_max_density .8
```

Related Information

- checkPlace
- debug_irdrop
- finishFloorplan
- getPlaceMode
- place_design
- place_opt_design
- placeSpareModule
- refinePlace
- refine_macro_place
- setOptMode
- specifyInstPad
- specifyScanCell
- specifySpareGate
- Using the Mixed Placer chapter of the *Innovus User Guide*.

setScanReorderMode

```
setScanReorderMode
[-help]
[-reset]
[-addScanPortPrefix prefix]
[-clkAware {true | false}]
[-compLogic {true | false}]
[-enable_for_partition {true|false}]
[-keepPDPorts {true | false}]
[-keepPort fileName]
[-preferH {true | false}]
[-preferV {true | false}]
[-scanEffort {low | medium | high | auto}]
[-skipMode {skipNone | skipBuffer | skipFloatingBuffer}]
[-swapEffort {low|medium|high}]
```

Controls certain aspects of how the software reorders scan chains.

Use the `getScanReorderMode` command to return the current settings for the `setScanReorderMode` command.

Note: Parameters specified by the `scanReorder` command supersede those specified by the `setScanReorderMode` command.

Parameters

`-addScanPortPrefix` *prefix*	Specifies the prefix to be added to scan hports. *Default:* `""`
`-clkAware {true | false}`	Specifies whether scan reordering is clock tree aware. • When `true`, scan reordering is clock tree aware, and is clock tree driven. • When `false`, scan reordering is not clock tree aware, and is wirelength driven. *Default:* `false` (The software is not clock tree aware.)
`-compLogic {true | false}`	If set to `true`, enables `-scanTrace` trace through multiple input logic gates (whose input/output pin is not defined in the input scan DEF) in the scan path. *Default*: `false` **Note:** When `-compLogic` is set to true, there can be multiple valid scan paths. In that case, `scanTrace` will select one of the valid paths. However, it is recommended that you specify the relevant gate's input/output pin to avoid trace through alternative paths.
`-enable_for_partition {true|false}`	Enables `scanReorder` for a design with partition. This option works with the `-keepPort` option. *Default*: `false`
`-help`	Outputs a brief description that includes type and default information for each `setScanReorderMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setScanReorderMode`.

`-keepPDPorts {true | false}`	Maintains or corrects the hierarchical ports without deleting or inserting shifters, and reorders the remaining scan chain. *Default:* `true` **Note:** This parameter works in the MSV flow only.
`-keepPort` *fileName*	Specifies a file that lists the names of hierarchical instances whose ports cannot be changed, removed, duplicated, or reordered during scan reordering. *Default:* `""`
`-preferH {true | false}`	Specifies that the preferred direction for scan reordering connections is horizontal. Reorders the scan chain to snake a sideways "S" configuration if the block is short and wide or the design has vertical routing congestion. *Default:* `false` **Note:** If both `-preferV` and `-preferH` are set to `true`, the software does not honor the setting and treats them both as if they were `false`.
`-preferV {true | false}`	Specifies that the preferred direction for scan reordering connections is vertical. Reorders the scan chain to snake a vertical "S" configuration if the block is long and narrow or the design has horizontal routing congestion. *Default:* `false` **Note:** If both `-preferV` and `-preferH` are set to `true`, the software does not honor the setting and treats them both as if they were `false`.
`-reset`	Resets parameters to their default values. The `-reset` parameter *must* be the first parameter specified. If you specify `-reset` by itself, the software resets all `setScanReorderMode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported.
`-scanEffort {low | medium | high | auto}`	Specifies the effort level for reordering scan chains. *Default:* `auto` Specify one of the following values: • `auto`: Runs internal reorder effort, and it may tune at different stages. • `high`: Runs more iterations of scan reorder to achieve better net length. Using this parameter increases the run time. • `low`: Runs fewer iterations of scan reorder to minimize net length quickly. • `medium`: Runs a normal number of iterations.

`-skipMode {skipNone | skipBuffer | skipFloatingBuffer}`	Specifies how scan reordering handles buffers and inverters in the scan chain. *Default:* `skipNone` Specify one of the following: • `skipNone`: Specifies that buffers and inverters remain after the scan chain reorder. • `skipBuffer`: Ignores buffers in the scan chain. If a buffer is not connected elsewhere in the design, `scanReorder` removes the buffer. • `skipFloatingBuffer`: Skips only buffers in the floating section and keeps the buffers that belong to an ordered section. **Note**: During scan reordering, the order of scan elements in the ordered section must be kept as importing scan specification. However, the order of the floating elements can be changed.
`-swapEffort {low|medium|high}`	Specifies the effort level for scanReorder different chains swapping. *Default:* `high` Specify one of the following values: • `high` • `low` • `medium`

Examples

- The following command resets the `-keepPDPorts` parameter to its default value:
  ```
  setScanReorderMode -reset -keepPDPorts
  ```
- The following command resets all `setScanReorderMode` parameters to their default values:
  ```
  setScanReorderMode -reset
  ```

Related Information

- `getScanReorderMode`
- `scanReorder`
- Placing the Design chapter in the *Innovus User Guide*
  - Optimizing and Reordering Scan Chains

specify_cell_stack_area

```
specify_cell_stack_area
[-help]
-cell cell_name
-group group_name
-rect {x1 y1 x2 y2}
```

Defines the cell area that needs to be involved while calculating stack rule of the corresponding group.

Parameters

`-cell` *`cell_name`*	Specifies the cell name that has the stack constraint.
`-help`	Outputs a brief description that includes type and default information for each `specify_cell_stack_area` parameter. For a detailed description of the command and all of its parameters, use the man command: man `specify_cell_stack_area`
`-group` *`group_name`*	Specifies the cell group name with the stack constraint. **Note**: The group name must be pre-defined in the LEF file or by the `specify_cell_stack_group` command
`-rect {x1 y1 x2 y2}`	Specifies the constraint rectangular area with respect to the cell origin. If the whole cell is constrained, the rectangular coordinates match the cell boundary. If the rectilinear area in cell is constrained, divide the rectilinear area into multiple rectangular area then define each rectangular area in separate command lines.

Related Information

- `delete_cell_stack_area`
- `delete_cell_stack_group`
- `report_cell_stack_area`
- `report_cell_stack_group`
- `specify_cell_stack_group`

specify_cell_stack_group

```
specify_cell_stack_group
[-help]
-name group_name
{-max_horizontal_length max_continuous_length }
```

Defines the stack rule of a cell group.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `specify_cell_stack_group` parameter. For a detailed description of the command and all of its parameters, use the man command: `man specify_cell_stack_group`
`-max_horizontal_length` *`max_continuous_length`*	Specifies the maximum continuous length in horizontal direction.
`-name` *`group_name`*	Specifies the cell group name with the stack constraint.

Related Information

- `delete_cell_stack_area`
- `delete_cell_stack_group`
- `specify_cell_stack_area`
- `report_cell_stack_group`
- `specify_cell_stack_area`

specify_cell_virtual_align

```
specify_cell_virtual_align
[-help]
[-cell cells]
[-mask <id>]
[-pin_x_loc <microns>]
```

Aligns a filler cell that does not have any pin or obs geometries to align, but the cell still needs to be aligned. For example, a GA-Filler cell after the post-mask ECO becomes a logic cell and needs to be correctly aligned for the router to eco-route it.

The `specify_cell_virtual_align` command can be used to provide mask for the first track inside the cell, or 'offset' to the first track (or both). You need to specify `-mask` or `-pin_x_loc`.

The command auto-detects the layer for vertical alignment. If as per the placement initialization there is no need for vertical alignment (site-width and vertical-track-pitch are same, pin-mask are not fixed), the virtual alignment is turned off. this means that the `specify_cell_virtual_align` command will not help.

Parameters

`-cell` *`cells`*	Specifies the lib-cells with virtual alignment requirement.
-help	Outputs a brief description that includes type and default information for each `specify_cell_virtual_align` parameter. For a detailed description of the command and all of its parameters, use the man command: `man specify_cell_virtual_align`
`-mask` *`virtual_mask_value`*	Specifies the virtual mask value for the first track in the cell. If you specify `-mask` only, the offset of the first track in the core is used as the offset.
`-pin_x_loc` *`location_in_microns`*	Specifies the location of the virtual geometry from the cell boundary. If you specify offset only, the track can be of any mask-color, as long as offset matches. Default: *First track location*

Related Information

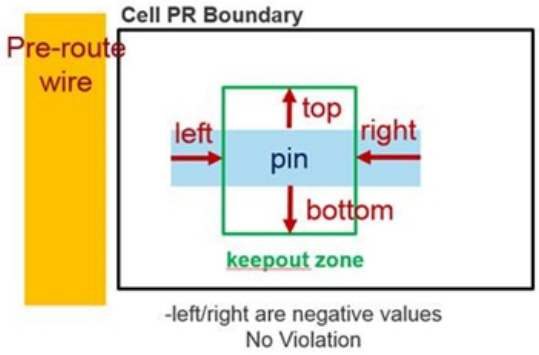
- `delete_cell_virtual_align`
- `report_cell_virtual_align`

specify_pg_keepout

specify_pg_keepout
```
[-help]
[-bottom value]
[-left value]
[-pin <string>]
[-right value]
[-top value]
{-cell lib_cell_name | -inst instance_name} {-layer layer_name | -layerId layer_id}
```

Defines the PG keepout box for a cell or a pin.

PG keepouts can be applied on the cell or cell pin. Keepout values can be positive or negative and are always from the edge for both pin and cell.



Parameters

`-bottom value`	Specifies the value to expand the keepout box on the bottom side.
`-cell lib_cell_name`	Specifies the lib cell that needs to be constrained by the PG keepout box.
`-help`	Outputs a brief description that includes type and default information for each specify_pg_keepout parameter. For a detailed description of the command and all of its parameters, use the man command: `man specify_pg_keepout`
`-inst instance_name`	Specifies the instance that needs to be constrained by the PG keepout box.
`-layer layer_name`	Specifies the layer on which the PG keepout box is applied. The layer is the LEF/OA name of routing layers.
`-layerId layer_id`	Specifies the layer Ids on which the PG keepout box is applied. The layer Id is the layer number of routing layers.
`-left value`	Specifies the value to expand the keepout box on the left side.
`-pin pin_name`	Specifies the pin that needs to be constrained by the PG keepout box.
`-right value`	Specifies the value to expand the keepout box on the right side.
`-top value`	Specifies the value to expand the keepout box at the top.

Example

The following command keeps `mycustomcell`/`the_m4_pin` away from M5 power stripes:

```
specify_pg_keepout -cell mycustomcell -pin the_m4_pin -layer M5
```

Related Information

- `delete_pg_keepout`
- `report_pg_keepout`

specifyCellEdgeSpacing

```
specifyCellEdgeSpacing
[-help]
{<edgeType1> <edgeType2> <spacing>
[-exceptAbutted | -forbiddenSpacing][-exceptNonFillerInBetween]} | [-reset]
```

Creates the cell edge spacing rule as in `CELLEDGESPACINGTABLE` in LEF.

⚠ The tool by default supports the latest Tcl settings. If a user tries to change the edge spacing, the tool will generate a warning message that resetting edge spacing would overwrite the old value.

Parameters

-exceptAbutted	Specifies that two abutted macros of the given edge types are also allowed.
edgeType1	Specifies edge `edgeType1` for edge spacing.
edgeType2	Specifies edge `edgeType2` for edge spacing.
`-exceptNonFillerInBetween`	Specifies that the edge spacing is only for adjacent instances. If two MACROs (cells) have a edge-spacing rule, say 0.8 micron, then normally the rule applies regardless of presence of another macro (cell) in the gap between. When the `-exceptNonFillerInBetween` option is used, the rule is exempted when a non-filler cell is present between the two constrained cell edges.
`-forbiddenSpacing`	Forbids *edgeType1* and *edgeType2* with the space.
`-help`	Outputs a brief description that includes type and default information for each `specifyCellEdgeSpacing` parameter. For a detailed description of the command and all of its parameters, use the man command: `man specifyCellEdgeSpacing`
`-reset`	Resets the cell edge spacing to LEF.
spacing	Specifies edge spacing between *edgeType1* and *edgeType2*. If there is no spacing defined between two edge types, it indicates that there is no spacing constraint (or 0.0 µm) between them.

Example

The following command creates `0.18 µm` spacing between `grp1` and `grp2`:

```
specifyCellEdgeSpacing grp1 grp2 0.18
```

Related Information

- deleteCellEdgeSpacing
- report_cell_edge_spacing
- specifyCellEdgeType

specifyCellEdgeType

```
specifyCellEdgeType
[-help]
{-left edgeType | -right edgeType | -top edgeType | -bottom edgeType | -reset }
[{-left edgeType | -right edgeType} [-cellRow rowNumber]]
[{-left edgeType | -right edgeType} [-half {top | bottom}]]
[{-top edgeType | -bottom edgeType} [-range {x y}]]
[{-cell cellName} [-reset]]
```

Specifies edge type for cell edges as `EDGETYPE` in macro LEF. It allows to specify one edge at a time for a particular cell.

To ensure the correct edge type specification, edge types asserted by `specifyCellEdgeType` command should be pre-defined by the `specifyCellEdgeSpacing` command.

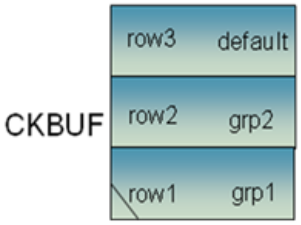
For example:

```
specifyCellEdgeSpacing grp1 grp2 0.18
specifyCellEdgeType -cell BUF1X -left grp1
```

If the edge type is specified without corresponding spacing rule, tool gives a warning and ignores the edge type specification.

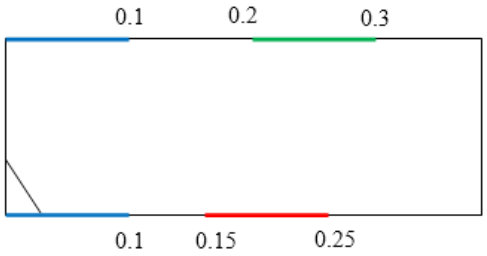
Parameters

`-bottom` *edgeType*	Specifies an edge type on the bottom side.
`-cell` *cellName*	Specifies cell to which to add edge constraint.

`-cellRow` *rowNumber*	Specifies the row number for horizontal edge type.
	You can define different edge types for each row of multiple height cells using `-cellRow`.
	For example, there is triple height cell CKBUF.
	 
	The following commands define different edge types for different rows of the CKBUF cell.
	`specifyCellEdgeType -cell CKBUF -right grp1 -cellRow 1`
	`specifyCellEdgeType -cell CKBUF -right grp2 -cellRow 2`
	In the above example, grp1 is applied to the first row (bottom row) of the right edge, grp2 is applied to the second row of right edge, and no specific edge type is defined for the third row of right edge and is of DEFAULT type.
	The legal value of `-cellRow` is from 1 to n, where n is the number of cell height of the cell.

`-half {top bottom}`	Defines the edge types corresponding to the top or bottom half of the cell. The following example associates the edge type 'grp1' with the bottom row, top half of the multi-height cell SDFF8: `specifyCellEdgeType -cell SDFF8 -right grp1 -cellRow 1 -half top`

`-help`	Outputs a brief description that includes type and default information for each `specifyCellEdgeType` parameter. For a detailed description of the command and all of its parameters, use the man command: `man specifyCellEdgeType`
`-left` *edgeType*	Defines the edge types corresponding to the top or bottom half of the cell. The following example associates the edge type 'grp1' with the bottom row, top half of the multi-height cell SDFF8. `specifyCellEdgeType -cell SDFF8 -right grp1 -cellRow 1 -half top`

`-range {`*x y*`}`	Specifies the range for the vertical edge type.
	The range for the vertical edge type can also be negative.
	The following example defines a partial range of an edge that the type should be labeled.This could only be defined on TOP or BOTTOM edges.
	`specifyCellEdgeType -cell A -top GROUP1 -range 0 0.1`
	`specifyCellEdgeType -cell A -bottom GROUP1 -range 0 0.1`
	`specifyCellEdgeType -cell A -top GROUP2 -range 0.2 0.3`
	`specifyCellEdgeType -cell A -top GROUP3 -range 0.15 0.25`
	 
	Blue line indicates GROUP1 edge
	Green line indicates GROUP2 edge
	Red line indicates GROUP3 edge
	Example of
	MACRO A
	…
	PROPERTY LEF58_EDGETYPE “
	EDGETYPE TOP GROUP1 RANGE 0 0.1 ;
	EDGETYPE BOTTOM GROUP1 RANGE 0 0.1 ;
	EDGETYPE TOP GROUP2 RANGE 0.2 0.3 ;
	EDGETYPE BOTTOM GROUP3 RANGE 0.15 0.25 ; “ ;
	…
	END A

`-reset`	Resets the specified cell constraint to the LEF status.
`-right` *edgeType*	Specifies an edge type on the right side.
`-top` *edgeType*	Specifies an edge type on the top side.

Example

The following commands specify one edge at a time for the `BUF1X` cell

- `specifyCellEdgeType -cell BUF1X -left grp1`
- `specifyCellEdgeType -cell BUF1X -right grp1 -cellRow 1`
- `specifyCellEdgeType -cell BUF1X -right grp1 -cellRow 2`

Related Information

- deleteCellEdgeSpacing
- `report_cell_edge_spacing`
- `specifyCellEdgeSpacing`

specifyCellPad

```
specifyCellPad
leaf_cellName
{padding | [[-right padding] [-left padding] [-top padding] [-bottom padding]]}
```

Specifies leaf cells to which padding (placement clearance) is to be added in the left, right, top, or bottom directions and a factor to use to calculate the padding dimension. The placer adds the padding on the sides of the specified cells during placement. The padding is retained during optimization and CTS.

If there are congestion problems, delete the padding by using the `deleteAllCellPad` command.

Parameters

`-bottom bottomPadding`	Specifies a factor to calculate the padding dimension on the bottom side. *Minimum*: `0` *Max*: `500`
leaf_cellName	Specifies the cell name(s) of a cell type to which the factor applies. You can use wildcards (`*`) with this parameter.
`-left leftPadding`	Specifies a factor to calculate the padding dimension on the left side.
padding	Specifies a factor to use to calculate the padding dimension. The placer multiplies the placement SITE by the factor. It reads the placement SITE value from the technology file. The factor should be integer number only. The unit of padding on left/right side is the number of sites. The unit of padding on top/bottom side is the number of rows
`-right rightPadding`	Specifies a factor to calculate the padding dimension on the right side.
`-top topPadding`	Specifies a factor to calculate the padding dimension on the top side. *Minimum*: `0` *Max*: `500`

Example

The following command adds padding that is twice the placement SITE to leaf cells named `FJK2`:

```
specifyCellPad FJK2 2
```

Related Information

- `deleteAllCellPad`
- Placing the Design chapter in the *Innovus User Guide*
  - Adding Padding

specifyInstPad

```
specifyInstPad
instName
{padding | [[-right padding] [-left padding] [-top padding] [-bottom padding]]}
```

Specifies a value (number of sites) for padding for an instance. Padding expands the area of an instance, so that the software leaves extra space around the instance during global placement (`place_design`), which helps reduce congestion in both the horizontal and vertical directions. In detail placement, user should specify `setPlaceMode -place_detail_honor_inst_pad` to `true` to honor left, right, top, and bottom instance padding.

Adding instance padding reduces horizontal and vertical congestion, but also decreases utilization.

Note: To remove instance padding, use the `deleteInstPad` command.

You can use the `reportInstPad` to report the padding for a specific instance, or all instance padding in the design.

Parameters

`-bottom` *padding*	Specifies the amount of padding at the bottom, in terms of number of rows.
instName	Specifies the instance to pad. You can use wildcards (`*`) when you specify the instance.
`-left` *padding*	Specifies the amount of padding on the left side, in terms of number of sites occupied by the instance.
padding	Specifies the amount of padding to add, in terms of number of sites occupied by the instance (sites are defined in the LEF file). For example, if an instance occupies eight sites, specifying 8 expands the area horizontally of the instance by 100 percent, and specifying 12 expands the area horizontally of the instance by 150 percent. The number you specify must be a whole number that is equal to or greater than zero.
`-right` *padding*	Specifies the amount of padding on the right side, in terms of number of sites occupied by the instance.
`-top` *padding*	Specifies the amount of padding at the top, in terms of number of rows.

Examples

- The following command adds padding of 2 sites around instance named U27316:

  ```
  specifyInstPad U27316 2
  ```

- The following command adds padding of 3 sites for all instances with prefix U2:

  ```
  specifyInstPad U2* 3
  ```

Related Information

- `deleteInstPad`
- `reportInstPad`
- `setPlaceMode`
- Placing the Design chapter in the *Innovus User Guide*
  - Adding Padding

- LEF Syntax chapter in the Innovus LEF/DEF Language Reference
    - Site

specifyJtag

```
specifyJtag
{-help}
{-cell leafCellName
    | -inst instName
    | -instGroup
    | -hinst hInstName
    | -group {hInstName | cellName}
    | -groupWArea {hInstName | cellName width height} }
```

Specifies instances to place with the `placeJtag` command. This command should be used before the `placeJtag` command.

Parameters

`-cell leafCellName`	Marks all instances of the specified cell type as JTAG instances.
`-group hInstName | cellName`	Marks all hierarchical instances or intermediate instances as JTAG instances.
`-groupWArea hInstName | cellName width height`	Marks all instances inside the hierarchical instance or intermediate cells as JTAG instances, and specifies a rectangular width and height where all instances or cells are placed within that rectangular region.
`-help`	Outputs a brief description that includes type and default information for each `specifyJtag` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man specifyJtag`.
`-hinst hInstName`	Marks a hierarchical instance as a JTAG instance.
`-inst instName`	Marks an instance as a JTAG instance. You can use a wildcard (`*`) character for this parameter.
`-instGroup`	Marks the instGroup name to be made JTAG instance

Command Order

Use this command before running the `placeJtag` command.

Example

- The following command specifies that the cell `jtag_FlipFlop` will be placed when using the `placeJtag` command:
  ```
  specifyJtag -cell jtag_FlipFlop
  ```
- The following command specifies that all instance names that begin with `instG` will be placed when using the `placeJtag` command:
  ```
  specifyJtag -inst instG*
  ```

Related Information

- `placeJtag`
- `reportJtagInst`
- `traceJtag`
- `unplaceJtag`
- Placing the Design chapter in the *Innovus User Guide*
  - Specifying and Placing JTAG and Other Cells Close to the I/Os

specifyLockupElement

```
specifyLockupElement
{-cell leafCellName | -inst instName}
-in ftname
-out ftname
```

Specifies a cell or instance as a lockup element. Lockup elements are present in scan chains where the chain crosses between two different clock domains or clock phases. Use this command after running placement.

Parameters

`-cell leafCellName`	Specifies the cell to be a lockup element.
`-in ftname`	Specifies the name of the scan input.
`-inst instName`	Specifies the instance to be a lockup element.
`-out ftname`	Specifies the name of the scan output.

Example

The following command specifies that cell `TLATX1` is a lockup element with a scan input pin `D` and an output pin `Q`:

```
specifyLockupElement -cell TLATX1 -in D -out Q
```

Related Information

- Placing the Design chapter in the *Innovus User Guide*
  - Optimizing and Reordering Scan Chains

specifyScanCell

```
specifyScanCell
[-help]
[cellName]
[-in ftname]
[-out ftname]
[-scanClock ftname]
[-scanEnable ftname]
```

Specifies scan cells that are not listed in the timing library.

Parameters

cellName	Specifies name of the scan cell.
-in *ftname*	Specifies the name of the scan input.
-out *ftname*	Specifies the name of the scan output.
-scanClock *ftname*	Specifies the name of the scan clock.
-scanEnable *ftname*	Specifies name of the scan enable f-term or pin names.

Example

The following command specifies cell `SDFFQ` to be a scan cell:

```
specifyScanCell SDFFQ -in SI -out Q
```

Related Information

- `deleteAllScanCells`
- `deleteScanCell`
- `reportScanCell`
- Placing the Design chapter in the *Innovus User Guide*
    - Optimizing and Reordering Scan Chains

specifyScanChain

```
specifyScanChain
[-help]
[scanChainName]
[-start {ftname | instPinName}]
[-stop {ftname | instPinName}]
```

Specifies a scan chain or group in a design. The actual tracing of the scan chain is performed by the `scanTrace` or `scanReorder` command. If scan chain specification fails, use the `scanTrace -verbose` command to see where the scan chain stops.

Note: Scan groups are sometimes called scan partitions.

> This command is not necessary for scan chains that are already specified in DEF or TDF files.

Parameters

scanChainName	Specifies a scan chain or scan group in a design.
`-start {`*ftname* `|` *instPinName*`}`	Specifies the starting scan pin name for the scan chain or scan group. Specifies the design input pin name. Specifies the instance output pin name.
`-stop {`*ftname* `|` *instPinName*`}`	Specifies the stopping scan pin name for the scan chain or scan group. Specifies the design output pin name. Specifies the instance input pin name.

Example

The following command sets scan chain `test_si` with a starting I/O pin name and a stopping pin name:

```
specifyScanChain test_si -start test23 -stop test_so
```

Related Information

- `deleteScanChain`
- `displayScanChain`
- report_scan_chain
- scanTrace
- `scanReorder`
- Placing the Design chapter in the *Innovus User Guide*
  - Optimizing and Reordering Scan Chains

specifyScanChainPartition

```
specifyScanChainPartition
-partition partitionName
{-all | chainName1 chainName2 ...}
```

Defines a group (a partition) of compatible scan chains for the scan reordering flow. The software can swap elements in scan chains in the partition. Swapping elements helps reduce the total scan wire length in the partition and improves local congestion in the design.

To enable swapping, type the following command:

```
setScanReorderMode -allowSwapping true
```

Parameters

`-all`	All scan chains in the specified partition are compatible.
chainName1 chainName2 …	Only the specified scan chains in the specified partition are compatible.
`-partition` *partitionName*	Specifies a name for the partition and a list of the scan chains that belong to the partition. The scan flops in these chains are swappable.

Example

The following commands define two scan chains using `specifyScanChain` command, and define one partition using `specifyScanChainPartition`. The `setScanReorderMode -allowSwapping` parameter allows the software to swap elements between compatible chains.

```
specifyScanChain chain1 -start SI1 -stop SO1
specifyScanChain chain2 -start SI2 -stop SO2
specifyScanChainPartition -partition clk1Domain chain1 chain2
setScanReorderMode -allowSwapping true
scanReorder
```

Related Information

- `deleteScanChainPartition`
- `getScanReorderMode`
- `reportScanChainPartition`
- `setScanReorderMode`
- `specifyScanChain`
- Placing the Design chapter in the *Innovus User Guide*
  - Optimizing and Reordering Scan Chains

specifySelectiveBlkgGate

```
specifySelectiveBlkgGate
[-help]
[-cell cellName]
[-inst instName]
```

Specifies cells and instances that are allowed to be placed inside the soft blockage during legalization.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `specifySelectiveBlkGate` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man specifySelectiveBlkGate`.
`-cell cellName`	Specifies the cell type name as legal for soft blockage. You can use wildcards (`*` or `?`) with this parameter.
`-inst instName`	Specifies the instance name as legal for soft blockage. You can use wildcards (`*` or `?`) with this parameter.

Examples

The following command specifies that cell `NR2RLX1` and all instance names that begin with `MEMCTLU` are allowed to be moved into soft blockage when legalization. `specifySelectiveBlkgGate -cell NR2RLX1 -inst MEMCTLU*`

Related Information

- `unspecifySelectiveBlkgGate`

specifySpareGate

```
specifySpareGate
{-help}
{-cell leafCellName
   |-inst instanceName
   | -hinst {hierarchicalInstanceName}}
```

Specifies spare gates to place. To localize spare gate placement, guide the module containing the spare gate during the floorplanning session. This command should be used before placing the design.

Parameters

`-cell leafCellName`	Specifies the cell type names of the spare gate cells. You can use this parameter to specify leaf cells (standard cells)--you cannot use it to specify spare modules. You can use wildcards with this parameter.
`-help`	Outputs a brief description that includes type and default information for each `specifySpareGate` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man specifySpareGate`.
`-hinst {hierarchicalInstanceName}`	Specifies a hierarchical instance that contains spare gate cells. A hierarchical instance acts as one unit (a submodule). The units are evenly spaced and are close together when they are distributed. You can use wildcards when specifying the hierarchical instance names.
`-inst instanceName`	Specifies the leaf instance names of the spare gates. You can use wildcards when specifying instance names.

Examples

- The following command specifies all cell types `YFD2S_SPG` to be spare gates:
  ```
  specifySpareGate -cell YFD2S_SPG
  ```
- The following command specifies instance `SH16/I1022` to be a spare gate:
  ```
  specifySpareGate -inst SH16/I1022
  ```
- The following command specifies that all instances in `SH16` to be spare gates:
  ```
  specifySpareGate -inst SH16/*
  ```
- The following command treats all instances with the cell type `NAND2X1` as spare cells. If the design uses some of these cells as spares and some as not spare, use the `-inst` parameter instead.
  ```
  specifySpareGate -cell NAND2X1
  ```
- The following example specifies multiple spare gates instances for the ECO placement:
  `specifySpareGate -inst spare_lv_more_* specifySpareGate -inst *SPAREcells_lv_*`
  Note: The `specifySpareGate` does not accept the "list" option for defining multiple spare gates.

Related Information

- Placing the Design chapter in the *Innovus User Guide*
  - Placing Spare Cells and Spare Modules

swap_well_taps

```
swap_well_taps
[-help]
[-both_violation cellName]
-cells cellName
[-check_only]
[-default_cell defaultCellName]
-diffusion_forbidden_spacing float
[-left_violation cellName]
[-right_violation cellName]
[-skip_cells cellNameList]
[-swap_report reportName]
[-violation_report reportName]
```

Swaps to custom tap cell to avoid OD spacing rule.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `swap_well_taps` parameter. For a detailed description of the command and all of its parameters, use the man command: `man swap_well_taps`.
`-both_violation cellName`	Specifies well taps to swap if only violating rules on both sides.
`-cells cellNameList`	Specifies well taps to swap if there is violation of rules.
`-check_only`	Checks violations only without swapping tap cells.
`-default_cell defaultCellName`	Swaps to default well tap first. Default well tap must be one of *cells*.
`-diffusion_forbidden_spacing float`	Specifies forbidden spacing rule.
`-left_violation cellName`	Specifies well taps to swap if only violating rules on the left side.
`-right_violation cellName`	Specifies well taps to swap if only violating rules on the right side.
`-skip_cells cellNameList`	Specifies the cell that would skip the rule check.
`-swap_report reportName`	Reports the swapped instances.
`-violation_report reportName`	Reports the violated instances.

Related Information

- addWellTap
- set_well_tap_mode

traceJtag

```
traceJtag
-infile jtagPinFile
-outfile jtagInstanceFile
```

Traces JTAG logic between the I/O pins and the pins of given leaf instances. The input file contains a list of pins of instances that are the endpoints of JTAG logic. The command writes out a Tcl file that lists all the JTAG logic traced. You can then source the file to add this information to the Innovus database.

Parameters

`-infile jtagPinFile`	Specifies a file that lists the endpoints of JTAG logic. In the file, list each leaf instance or list the cell and its port.
	• To list the leaf instances, use the `instpin` statement, as follows:
	`instpin leafInstanceNamepinName`
	• If all instances of a cell are used only in JTAG logic, you can list that cell and its port by using the `cellport` statement, as follows:
	`cellport cellTypeNameportName`
	To avoid tracing non-JTAG cells, the cells you specify in a `cellport` statement must be JTAG-only cells. The cell type cannot be an intermediate cell. It must have at least one physical leaf instance.

`-outfile jtagInstanceFile`	Specifies the JTAG logic file in Tcl format.

The following commands trace the JTAG logic based on the input file `jtag.leaf`, write the output file `jtag.out`, and add the logic specified in the file to the Innovus database:Example

```
traceJtag -infile jtag.leaf -outfile jtag.out
source jtag.out
```

Following is an example of an output file:

```
specifyJtag -inst regA
specifyJtag -inst regB
specifyJtag -inst regC
specifyJtag -inst regD
specifyJtag -inst regE
specifyJtag -inst regF
specifyJtag -inst regG
specifyJtag -inst regH
```

Related Information

- specifyJtag
- placeJtag
- reportJtagInst
- unplaceJtag

- Placing the Design chapter in the *Innovus User Guide*
  - Specifying and Placing JTAG and Other Cells Close to the I/Os

unplaceJtag

`unplaceJtag`

Changes the placement status of JTAG inst from FIXED to PLACED. This command can be used after the `placeJtag` command.

Parameters

None

Examples

The following set of commands places all Jtag instances of cell INVX1 as FIXED, and then using `unplaceJtag` to change their status to PLACED:

```
specifyJtag -cell INVX1
placeJtag
unplaceJtag
```

Related Information

- `placeJtag`
- `specifyJtag`
- `reportJtagInst`
- `specifyJtag`
- `traceJtag`
- Placing the Design chapter in the *Innovus User Guide*
    - Specifying and Placing JTAG and Other Cells Close to the I/Os

unspecifyJtag

```
unspecifyJtag
{-cell leaf_cellName
 | -inst instanceName
 | -hinst hierarchicalInstanceName
 | -group hierarchicalInstanceName | cellName}
```

Unspecifies instances that were specified by the `specifyJtag` command. This command can be used if there is instance specified by the `specifyJtag` command.

Parameters

`-cell leaf_cellName`	Unmarks all instances of a specified cell type as JTAG instances.
`-group hierarchicalInstanceName | cellName`	Unmarks all hierarchical instances or intermediate instances as JTAG instances.
`-hinst hierarchicalInstanceName`	Unspecifies a hierarchical JTAG instance.
`-inst instanceName`	Unspecifies a JTAG instance. You can use wildcards (`*` or `?`) with this parameter.

Example

- The following command unspecifies cell `jtag_FlipFlop` that was specified by the `specifyJtag` command:
  ```
  unspecifyJtag -cell jtag_FlipFlop
  ```
- The following command unspecifies all instance names that begin with `instG` that were specified by the `specifyJtag` command:
  ```
  unspecifyJtag -inst instG*
  ```

Related Information

- `reportJtagInst`
- `specifyJtag`
- `traceJtag`
- `unplaceJtag`
- Placing the Design chapter in the *Innovus User Guide*

unspecifySelectiveBlkgGate

```
unspecifySelectiveBlkgGate
[-help]
[-cell cellName]
[-inst instName]
```

Specifies cells and instances that are forbidden to be placed inside the soft blockage during legalization.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `unspecifySelectiveBlkgGate` parameter. For a detailed description of the command and all of its parameters, use the man command: `man unspecifySelectiveBlkgGate`.
`-cell cellName`	Specifies the cell type name as illegal for soft blockage. You can use wildcards (`*` or `?`) with this parameter.
`-inst instName`	Specifies the instance name as illegal for soft blockage. You can use wildcards (`*` or `?`) with this parameter.

Examples

The following command specifies that cell `NR2RLX1` and all instance names that begin with `MEMCTLU` cannot be moved into soft blockage during legalization.

```
unspecifySelectiveBlkgGate -cell NR2RLX1 -inst MEMCTLU*
```

Related Information

- `specifySelectiveBlkgGate`

Placement Global Variables

- spgM3StripePushDown
- spgM3StripeShrink

spgM3StripePushDown

`spgM3StripePushDown`

Specifies the width threshold in site units for virtual push down of M3 stripe to M2 for M1 pin-access.

If the default value is set to `-1`, it indicates that no M3 stripe will be pushed down. If the default value is set to `0`, it indicates that all M3 stripes will be pushed down. If the default value is larger than `0`, then all m3 stripes having width greater than the specified value will be pushed down.

spgM3StripeShrink

`spgM3StripeShrink`

Specifies the shrink effort of the virtual M2 stripe by given total site units. This global should work with `spgM3StripePushDown`.

If the default value is set to `0`, it indicates that the entire pushed down M3 stripe is used to evaluate M1 pin access. If the default value is set to more than `0`, it indicates that M3 stripe will shrink by the specified value before it is pushed down to M2. The left side will shrink first. For example, if the specified value is set to `1`, the M3 stripe will shrink by 1 site on left, and if the specified value is set to `2`, 1 site on left will shrink, along with 1 site on right.

25

Power Analysis Commands

- getPGNetResis
- getSpecialNetResis

getPGNetResis

```
getPGNetResis
[-help]
[-net {netName +}]
[-outfile fileName]
[-temperature temperature]
[-view viewName]
[-worst | -p2p <x1 y1 layerName1 x2 y2 layerName2]
```

Computes the resistance of the power or ground net.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `getPGNetResis` parameter. For a detailed description of the command and all its parameters, use the man command: man `getPGNetResis`.
`-net {netName +}`	Specifies the name of the net for which resistance is to be computed. The command accepts multiple names. **Note:** The total resistance is the algebraic sum of the resistance of each individual wire and via, rather than the effective resistance of the entire resistor network.

`-outfile` *`fileName`*	Creates a report containing the nets' resistance values. The power analysis software adds the extension .res to the *fileName* you specify.
`-temperature` *`temperature`*	Specifies the temperature to be used during temperature-dependent resistance extraction. The temperature-dependent resistance coefficient is taken from the capacitance table.
`-view` *`viewName`*	Specifies the view to analyze.
`-worst`	For cases where a special net , that connects two or more instances by their non-PG pins, reports information on the worst (largest) effective resistance based on pair of pins, and the path that has the worst resistance.
`-p2p <x1 y1 layerName1 x2 y2 layerName2`	Computes the effective resistance between two instance pins. The point/layer specification is used to find the corresponding pin for calculation. • If you do not specify the option `-p2p`, the total resistance is the algebraic sum of the resistance of each individual wire and via. • If you specify the option `-p2p`, the total resistance is the effective resistance of the entire resistor network. Use the option `-p2p` with caution as it will take a while to get an effective R from large PG nets.

Example

- The following command displays a report within the Innovus console:
  ```
  pgNetResis vdd!
  ```
- The report is generated with the format shown in the following example:

```
Net Name: vdd!

total resistance: 5.843993e+01 Ohm

1 resistance: 3.530900e+01 Ohm

2 resistance: 0.000000e+00 Ohm

3 resistance: 0.000000e+00 Ohm

4 resistance: 0.000000e+00 Ohm

5 resistance: 2.459333e+00 Ohm

6 resistance: 2.324933e+00 Ohm

V12 resistance: 3.360000e+00 Ohm

V23 resistance: 3.360000e+00 Ohm

V34 resistance: 3.360000e+00 Ohm

V45 resistance: 5.600000e+00 Ohm

V56 resistance: 2.666667e+00 Ohm

resistance of selected wires/vias: 1.666667e-01 Ohm
```

getSpecialNetResis

```
getSpecialNetResis
[-help]
[-net {netName +}]
[-outfile fileName]
[-temperature temperature]
[-view viewName]
[-worst | -p2p <x1 y1 layerName1 x2 y2 layerName2]
```

Reports resistance values for specified nets. Use this command after power planning.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `getSpecialNetResis` parameter. For a detailed description of the command and all its parameters, use the man command: man `getSpecialNetResis`.
`-net {netName +}`	Specifies the names of regular or power/ground nets that appear in the `SPECIALNETS` section of the DEF file.
`-outfile` *`fileName`*	Creates a report containing the resistance values of nets. The power analysis software adds the extension `.res` to the *`fileName`* you specify.
`-temperature` *`temperature`*	Specifies the temperature to be used during temperature-dependent resistance extraction. The temperature-dependent resistance coefficient is taken from the capacitance table.
`-worst`	For cases where a special net , that connects two or more instances by their non-PG pins, reports information on the worst (largest) effective resistance based on pair of pins, and the path that has the worst resistance.
`-p2p <`*`x1 y1 layer1 x2 y2 layer2`*	Calculate resistance based on point-to-point coordinate and layer between two port layer shapes of instance pins. The pair of the pins are suggested both as PG pins or non-PG pins.

`-view` *viewName*	Specifies the view to analyze.

Examples

- The following command constructing power graph for P/G net `VDD_LOW`:

```
getSpecialNetResis -p2p 1095 2960 M5 3316 5705 M6 VDD_LOW

Inst1 is block0

Inst2 is WIR@IO@0154@0263

##### Start Calculating Resistance between each Dc Source and Current Sink #####

Width-based sheet resistance table (silicon width) is used ...

Resistance: 6.9025 Ohm : From block0/AVD (1094.94 2959.89 METAL5) To
WIR@IO@0154@0263/VDD (3298 5702 METAL6)

Resistance: 6.9014 Ohm : From WIR@IO@0154@0263/VDD (3316 5704.5 METAL6) To
block0/AVD (1082.44 2959.39 METAL5)

Effective resistance on net VDD_LOW between block0/AVD and WIR@IO@0154@0263/VDD =
6.9025 Ohms
```

- The following command to construct power graph for net `SPNS_RESDEL3V`:

```
getSpecialNetResis -worst SPNS_RESDEL3V

##### Start Calculating Resistance between each Dc Source and Current Sink #####

Resistance: 6.971 Ohm : From buff_RESETZ0/POCREL3V (506.12 -2755.78 METAL2) To
buff_RESETZ0/RESDEL3V (508.54 -2756.2 METAL2)

Resistance: 381.44 Ohm : From buff_RESETZ0/POCREL3V (506.12 -2755.78 METAL2) To
ctop0/vreg0/REGRES3V (-1094.98 -2191.92 METAL2)
```

```
Resistance: 29.175 Ohm : From buff_RESETZ0/POCREL3V (506.12 -2755.78 METAL2) To
ctop0/nf_RESETZ0/RESDEL3V (531.27 -2673.66 METAL2)

Resistance: 8.1427 Ohm : From buff_RESETZ0/POCREL3V (506.12 -2755.92 METAL3) To
buff_RESETZ0/RESDEL3V (508.54 -2756.2 METAL2)

Resistance: 382.62 Ohm : From buff_RESETZ0/POCREL3V (506.12 -2755.92 METAL3) To
ctop0/vreg0/REGRES3V (-1094.98 -2191.92 METAL2)

Resistance: 30.346 Ohm : From buff_RESETZ0/POCREL3V (506.12 -2755.92 METAL3) To
ctop0/nf_RESETZ0/RESDEL3V (531.27 -2673.66 METAL2)

…

…

Worst effective resistance on net SPNS_RESDEL3V between buff_RESETZ0/RESDEL3V and
ctop0/vreg0/REGRES3V = 388.32 Ohms
```

26

Power Calculation Commands

- analyze_thermal
- create_thermal_model
- dump_unannotated_nets
- extract_metal_density
- encrypt_thermal_file
- get_default_switching_activity
- get_dynamic_power_simulation
- get_power_analysis_mode
- map_activity_file
- merge_vtm
- propagate_activity
- query_power_data
- read_activity_file
- report_instance_power
- report_power
- reset_power_activity
- restore_power_database
- set_default_switching_activity
- set_dynamic_power_simulation
- set_inst_temperature_file
- set_metal_density_options
- set_power
- set_power_analysis_mode
- set_power_calc_temperature
- set_power_include_file
- set_power_output_dir
- set_switching_activity
- set_thermal_analysis_mode

- set_twf_attribute
- set_virtual_clock_network_parameters
- write_power_constraints
- write_tcf

analyze_thermal

```
analyze_thermal
-help
```

Specifies to invoke Celsius Thermal Solver to run die-only thermal analysis. The command generates the die temperature map file that is used by Voltus to run temperature-aware power/IR/EM analysis. Before specifying this command, use the `set_thermal_analysis_mode` command to set the parameters for performing chip-centric thermal analysis.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.

Examples

- The following command specifies to run die-only thermal analysis:
  ```
  analyze_thermal
  ```

Related Topics

- set_thermal_analysis_mode

create_thermal_model

```
create_thermal_model
[-help]
[-leakage_temperature_scale_table {$temp1 $scalefactor1 $temp2 $scalefactor2 $temp3 $scalefactor3 .....}]
[-leakage_temperature_scale_table_file filename]
[-metal_density_file filename]
[-power_db path_to_power_database]
[-vtm_SI_unit {true | false}]
[-vtm_conductivity_inputs filename]
[-vtm_densitytable_tile {Xint Yint}]
[-vtm_file filename]
[-vtm_format {simple | stack} ]
[-vtm_format_version {new | default} ]
[-vtm_header_include_file filename]
[-vtm_leakage_temp {temp1 temp2 temp3} ]
[-vtm_output_dir directory]
[-vtm_powertable_tile {Xint Yint} ]
[-vtm_tile {Xint Yint}]
[-vtm_tile_size {$X_um $Y_um}]
```

Sets the parameters for extracting the metal density information. This command must be specified before the `extract_metal_density` command.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-leakage_temperature_scale_table {$temp1 $scalefactor1 $temp2 $scalefactor2 $temp3 $scalefactor3 .....}`	
	Specifies the global scale factor for a temperature value to enable scaling of the leakage power values coming from the input dotlib file. This parameter allows you to scale the power values only at certain specific temperatures. This parameter allows you to generate a multi-temperature leakage power map table from a single input dotlib file in the power map generation flow. `-leakage_temperature_scale_table` and `-leakage_temperature_scale_table_file` are mutually exclusive.
`-leakage_temperature_scale_table_file filename`	

Specifies a file containing the user-defined temperature values and the corresponding scale factors for each library, cell, or instance. The format of the file is:

```
#Type Name temp1 temp2 temp3 temp4
Lib    library_name   scalefactor@temp1   scalefactor@temp2  scalefactor@temp3   scalefactor@temp4
Cell   cell_name      scalefactor@temp1   scalefactor@temp2  scalefactor@temp3   scalefactor@temp4
inst   instance_name  scalefactor@temp1   scalefactor@temp2  scalefactor@temp3   scalefactor@temp4
```

This parameter allows you to generate a multi-temperature leakage power map table from a single input dotlib file in the power map generation flow. `-leakage_temperature_scale_table` and `-leakage_temperature_scale_table_file` are mutually exclusive.

File Format Example for the Type Lib:

```
#Type Name         25 50 125 150
Lib   fast         1 2 10 100
Lib   bufao        1 2 10 100
Lib   header       1 2 10 100
Lib   pso_ring_wc  1 2 10 100
```

`-metal_density_file filename`

Specifies the VTM metal density file generated by the `extract_metal_density` command.

`-power_db path_to_power_database`

Specifies the path to the power database with cell boundary box, location, and rotation information. This is generated by specifying the `set_power_analysis_mode -save_bbx true` parameter.

`-vtm_SI_unit {true | false}`

Specifies the unit definition for power and length. The possible arguments are:

- `true`: unit will be in m and W.
- `false`: unit will be um and mW.

Default is `true` with unit m(meter) for length and W(Watt) for power.

`-vtm_conductivity_inputs filename`

This parameter is same as the `set_metal_density_options -thermal_conductivity_file` parameter. When you want to overwrite the thermal conductivity defined in the metal density file, you can provide a new thermal conductivity file using the `create_thermal_model -vtm_conductivity_inputs` command.

`-vtm_densitytable_tile {Xint Yint}`

Specifies the number of tiles in the X and Y directions for the metal density tables only.

`-vtm_file filename`

Specifies the name of the output Voltus thermal model file.

`-vtm_format {simple | stack}`

	Generates a tile-based power map file in the simple or stack format to perform accurate thermal analysis. The possible arguments are: • `simple`: generates only power tables without metal density tables. • `stack`: generates both power and metal density tables.
`-vtm_format_version {new | default}`	
	Determines whether to generate the default or the new power map file format. This parameter allows you to have different tile number between the power table and the metal density table. When set to `new`, the power map file format with non-uniform tile numbers for the power (`-vtm_powertable_tile`) and metal density tables (`-vtm_densitytable_tile`) can be generated.
`-vtm_header_include_file` *filename*	
	Allows you to define extra header information like file location.
`-vtm_leakage_temp {`*temp1 temp2 temp3*`}`	
	Specifies the leakage temperature values used for trilib (single voltage but different temperatures dot libs) in the power calculation flow to output user-defined temperature points' leakage power tables.
`-vtm_output_dir` *directory*	
	Specifies the directory path to the Voltus thermal model output folder.
`-vtm_powertable_tile {`*Xint Yint*`}`	
	Specifies the number of tiles in the X and Y directions for the power tables only.
`-vtm_tile {`*Xint Yint*`}`	
	Specifies the number of tiles in the X and Y directions for the generated Voltus thermal model file. The default is 10x10.
`-vtm_tile_size {`*$X_um $Y_um*`}`	
	Specifies the unit tile size in the X and Y directions for the generated thermal model file.

Examples

- The following command is an example of the trilib Tcl command, that is, uses the temperature in `vtm_leakage_temp` to calculate the leakage:

```
create_thermal_model \
-power_db powerdb \
-metal_density_file vtm_MD_db/my_chip.rpt \
-vtm_output_dir ./vtm_output \
-vtm_leakage_temp {20 50 80}
```

- The following command is an example of the mix-usage Tcl command, that is, some instances have trilib data while other instances only have a single library:

```
create_thermal_model \
-power_db powerdb \
-metal_density_file vtm_MD_db/my_chip.rpt \
-vtm_output_dir ./vtm_output \
-vtm_leakage_temp {20 50 80} \
-leakage_temperature_scale_table {0 1 20 2 50 3 100 10}
```

Related Topics

- `extract_metal_density`

dump_unannotated_nets

```
dump_unannotated_nets
[-help]
[-file filename]
-type annotationType
```

Shows the percentage of nets annotated by a specific type and also prints the names of all nets that are not annotated by that type.

Annotation information is stored on a per-file basis for activity, VCD, TCF, and user commands to provide detailed annotation reports when requested.

`dump_unannotated_nets` can be invoked multiple times with different report types (`-type`). You must specify this command before `report_power`.

Parameters

`-file filename`	Specifies the text file name that includes the annotation report information. If no file is specified, the report will be written to the default output file `annotation_type.log`, where `type` is taken from the `-type` argument specified.
`-help`	Outputs the command usage and a brief description about the command parameters.

`-type` *annotationType*	The type can be any of the following: • `all` Shows annotation reports for all types. • `activity` Shows annotation data for all types of activity specifications including TCF, VCD, SDC, TWF, and user commands that set activity. • `cap` Shows annotations from the SPEF files. • `fsdb` Shows annotations from the FSDB files. • `saif` Shows annotations from the SAIF files. • `tcf` Shows annotations from the TCF files. • `timing` Shows annotations from the TWF files. This type is applicable only to the dynamic power analysis flow. • `vcd` Shows annotation from the VCD files.

Example

- The following command dumps the nets in the netlist that are missing in the vcd file:
  ```
  dump_unannotated_nets -type vcd -file vcd.log
  ```
  The net names are written in the file `vcd`.log.
- The following command dumps the nets in the netlist which do not have an activity annotated from any source (VCD/TCF/SDC/TWF/user defined) and writes the net names in the file `unannotatednets.log`:
  ```
  dump_unannotated_nets -type all -file unannotatednets.log
  ```

Related Information

- `read_activity_file`
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

extract_metal_density

```
extract_metal_density
-help
```

Specifies to extract the metal density information. The command generates the Voltus Thermal Model (VTM) for Celsius only if you do not want to include power information. Before specifying this command, use the `set_metal_density_options` command to set the parameters for extracting the metal density information.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.

Examples

- The following command specifies to run extract metal density:
  ```
  extract_metal_density
  ```

Related Topics

- `set_metal_density_options`

encrypt_thermal_file

```
encrypt_thermal_file
-input ASCII.txt
-output $encrypted_conductivity.txt
```

Specifies to encrypt the file containing the thermal conductivity value for layers, and to protect foundry's thermal conductivity parameters used for power map generation. This encrypted file is then specified with the `report_power -thermal_conductivity_inputs` parameter for generating a tile-based power map file.

Parameters

`-input ASCII.txt`	

	Specifies the name of the ASCII thermal file that needs to be encrypted. This file contains the thermal conductivity parameters used for power map generation.
	The `-input` parameter supports both layer type and layer name based thermal conductivity file.
	The following is a snippet of the input thermal conductivity file content (layer type based file):
	`Metal 430`
	`Silicon 250`
	`Dielectric 0.6`

`-output $encrypted_conductivity.txt`	

	Specifies the name of the encrypted output thermal file.
	The following is a snippet of the encrypted thermal conductivity file content:
	`Metal $encryption_value`
	`Silicon $encryption_value`
	`Dielectric $ecncryption_value`

Examples

- The following command encrypts the specified input file (`format_1.txt`) to generate the encrypted output file (`format_1_encr.txt`):

  ```
  encrypt_thermal_file -input encrypt/format_1.txt -output encrypt/format_1_encr.txt
  ```

  The encrypted file (`format_1_encr.txt`) is then specified for generating a tile-based power map file, as shown below:

  ```
  report_power \
      -outfile                    static.rpt \
      -thermal_power_map_tile     {5 5}\
      -thermal_power_map_format   stack \
      -thermal_power_map_file     powermap_format1_encr_in.txt \
      -thermal_conductivity_inputs ./encrypt/format_1_encr.txt
  ```

Related Information

- `report_power`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*

get_default_switching_activity

```
get_default_switching_activity
[-help]
[-clip_activity_to_domain_freq]
[-clock_gates_enable]
[-clock_gates_output]
[-clock_gates_output_ratio]
[-comb_clockgate_ratio]
[-duty]
[-global_activity]
[-icg_ratio]
[-input_activity]
[-macro_activity]
[-period]
[-seq_activity]
```

Displays the following information about a `set_default_switching_activity` parameter in the system log file and in the system console:

- Parameter name
- Current value

If you do not specify a parameter, the software displays information for all of the `set_default_switching_activity` parameters. The `get_default_switching_activity` command prints only the user-specified value for the parameters. If a parameter has not been set using the `set_default_switching_activity` command, the `get_default_switching_activity` command does not output any information in the console.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_default_switching_activity` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_default_switching_activity`.
`parameter_names`	Displays information for the specified parameters. You can specify one or more parameters. See `set_default_switching_activity` for description of the parameters you can specify.

Examples

The following command displays the current setting of the `-period` parameter:

```
get_default_switching_activity -period
```

The software displays the following information:

```
-period {20}
```

get_dynamic_power_simulation

```
get_dynamic_power_simulation
[-help]
[-activity_pattern]
[-period]
[-resolution]
```

Displays the following information about a `set_dynamic_power_simulation` parameter in the system log file and in the system console:

- Parameter name
- Current value

If you do not specify a parameter, the software displays information for all of the `set_dynamic_power_simulation` parameters. The `get_dynamic_power_simulation` command prints only the user-specified value for the parameters. If a parameter has not been set using the `set_dynamic_power_simulation` command, the `get_dynamic_power_simulation` command does not output any information in the console.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_dynamic_power_simulation` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_dynamic_power_simulation.`
`parameter_names`	Displays information for the specified parameters. You can specify one or more parameters. See `set_dynamic_power_simulation` for description of the parameters you can specify.

Examples

The following command displays the current setting of the `-period` parameter:

```
get_dynamic_power_simulation -period
```

The software displays the following information:

```
-period {20}
```

Related Information

- `set_power_analysis_mode`
- `set_dynamic_power_simulation`
- `set_default_switching_activity`
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

get_power_analysis_mode

```
get_power_analysis_mode
[-help]
[-quiet]
```

Use the `get_power_analysis_mode` command to display the current settings for the `set_power_analysis_mode` command.

If you do not specify a parameter, the software returns information for all of the `set_power_analysis_mode` parameters.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `get_power_analysis_mode` parameter. For a detailed description of the command and all of its parameters, use the man command `man get_power_analysis_mode`.
`-quiet`	Returns the current settings of all `set_power_analysis_mode` parameters in Tcl list format.

Example

- The following command returns the current settings for all `set_power_analysis_mode` parameters:

  ```
  get_power_analysis_mode \
  -honor_negative_energy true # bool, default=true \
  -ignore_control_signals true # bool, default=true \
  -leakage_scale_factor_for_temp 1 # float, default=1 \
  -off_pg_nets {} # string, default="" \
  -x_transition_factor 0.5 # float, default=0.5 \
  -z_transition_factor 0.25 # float, default=0.25
  ```

- The following command returns the current settings for all `set_power_analysis_mode` parameters in Tcl list format only:

  ```
  get_power_analysis_mode -quiet
  ```

  The software returns the following information:

  ```
  {honor_negative_energy true} {ignore_control_signals true} {leakage_scale_factor_for_temp 1}
  {off_pg_nets {}} {x_transition_factor 0.5} {z_transition_factor 0.25}
  ```

Related Information

- `set_power_analysis_mode`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

map_activity_file

```
map_activity_file
[-help]
[-golden {rtl | gate}]
[-gate_block block_name]
[-reset]
-rtl2gate mapping_file
[-rtl_block block_name]
[-two_column_mapping_file mapping_file]
```

Specifies instance name mapping between RTL netlist and GATE level netlist. Power analysis uses this instance name mapping to annotate the output nets of this instance from the pin-based RTL VCD or TCF. The mapping file used for instance name mapping is generated by Conformal.

You must specify this command before `report_power`.

Parameters

`-gate_block` *`block_name`*	
	Specifies the GATE activity block name to map the block-level RTL mapping files to the top-level RTL activity files. The GATE block name is appended before the GATE level entries in the mapping file to match the entries in the top-level RTL activity file.
`-golden {rtl | gate}`	
	Specifies whether RTL or Gate is Golden. The one not specified as golden will be considered as Revised. Currently the Conformal MatchPoint file does not specify this. *Default*: `rtl`
`-help`	Outputs a brief description that includes the type and default information for each `map_activity_file` parameter. For a detailed description of the command and all of its parameters, use the man command `man map_activity_file`.

`-reset`	Specifies to reset and ignore all the previous mapping files specified using the `map_activity_file` command. *Default* : All the previous mapping files specified using the `map_activity_file` command are loaded and used.
`-rtl2gate` *`mapping_file`*	
	Specifies instance name mapping between RTL netlist and GATE level netlist. RTL level VCD or TCF are obtained from simulation on RTL Verilog netlist. Power analysis uses this instance name mapping to annotate the output nets of this instance from the pin based RTL VCD or TCF. The mapping file used for instance name mapping is generated by Conformal.
`-rtl_block` *`block_name`*	
	Specifies the RTL activity block name to map the block-level RTL mapping files to the top-level RTL activity files. The RTL block name is appended before the RTL level entries in the mapping file to match the entries in the top-level RTL activity file.
`-two_column_mapping_file` *`mapping_file`*	

	Specifies a file that contains the instance name mapping between the RTL netlist and GATE level netlist.
	This parameter can be used to specify a mapping file instead of the LEC generated mapping file (`-rtl2gate` *`mapping_file`*) to map points from RTL to Gate netlist. The format of the two column mapping file is:
	`#Gate name =>  # RTL name`
	`gate_inst_name1  => rtl_inst_name1`
	`...`
	Here, the first column corresponds to the gate name and the second column corresponds to the RTL name.

Example

- The following command specifies a mapping file rtl_map:
  ```
  map_activity_file -rtl2gate rtl_map
  ```

- The following command performs instance name mapping between RTL netlist and GATE level netlist, wherein the GATE level netlist is Golden and the RTL netlist is Revised; it specifies the mapping file MapFile.alt

```
map_activity_file -rtl2gate MapFile.alt -golden gate
```

- The following commands appends the RTL block name and GATE block name before the RTL and GATE level entries in the mapping file to match the entries in the top-level RTL activity file:

```
map_activity_file -rtl2gate LEC.top \
        -rtl_block  "core/block_top" \
        -gate_block "core/block_top"


map_activity_file -rtl2gate LEC.tile.top \
          -rtl_block  "core/tile_instance\[0\]/tile_inst" \
          -gate_block "core/tile_instance_0_tile_inst"
```

- The following commands specify to ignore the previous mapping file (`old.map`), and use the modified mapping file (`updated.map`):

```
map_activity_file -rtl2gate old.map
map_activity_file -reset
map_activity_file -rtl2gate updated.map
```

An example of the mapping file output from Conformal is described as follows:

```
Mapped points: SYSTEM class
1-th mapped points:
 (G) + 1 PI /x_ick
 (R) + 595 PI /x_ick
2-th mapped points:
 (G) + 2 PI /x_jreset_cp_p
 (R) + 594 PI /x_jreset_cp_p
3-th mapped points:
 (G) + 3 PI /x_mreset_cp_p
 (R) + 593 PI /x_mreset_cp_p
........
60-th mapped points:
 (G) + 4582 DFF /cpexec0/cpddecls0/gi_opls/q_reg_reg[35]
 (R) + 3856 DFF /cpexec0/cpddecls0/gi_opls/q_reg_reg_35_

61-th mapped points:
 (G) + 4583 DFF /cpexec0/cpddecls0/gi_opls/q_reg_reg[34]
 (R) + 3855 DFF /cpexec0/cpddecls0/gi_opls/q_reg_reg_34_

62-th mapped points:
 (G) + 4633 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg[21]
 (R) + 3805 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg_21_

63-th mapped points:
 (G) + 4634 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg[20]
 (R) + 3804 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg_20_

64-th mapped points:
 (G) + 4635 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg[19]
```

```
 (R) + 3803 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg_19_

65-th mapped points:
 (G) + 4636 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg[18]
 (R) + 3802 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg_18_
```

where, `G` refers to the Golden netlist name and `R` refers to the Revised netlist name.

By default, the tool assumes that the Golden netlist name refers to the RTL net name and Revised netlist refers to the GATE level net name.

Related Information

- `report_power`
- `read_activity_file`
- "RTL Activity File Flow in Static Power Calculation" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

merge_vtm

```
merge_vtm
[-help]
-config filename
[-rundir directory]
[-merged_file filename]
[-format {simple | metal | stack}]
```

Specifies to import the top-level and other block-level Voltus thermal models (VTMs) having the specified design configuration to generate a merged uniform single VTM for Celsius usage.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-config filename`	

	Specifies the name of the design configuration file.
	The format of the file is:
	`$instance_name $Design_name $X(um),$Y(um) $Cell_orientation $VTM_file format`
	The file includes:
	• instance name
	• design name
	• X and Y locations of the instance
	• LEF/DEF style orientation for the instance
	• path to the thermal map file
	• format of the thermal map file
	Example:
	`octa octa 0,0 N vtm_output_top/top.vtm.txt both`
	`inst_8 super_filter 62.97,47.035 N vtm_output_block/block.vtm.txt both`
	`inst_1 super_filter 1157.09,765.605 FE vtm_output_block/block.vtm.txt both`
	`inst_2 super_filter 777.085,765.605 E vtm_output_block/block.vtm.txt both`
	`inst_3 super_filter 397.085,765.605 W vtm_output_block/block.vtm.txt both`
	`inst_4 super_filter 17.085,765.605 FW vtm_output_block/block.vtm.txt both`
	`inst_5 super_filter 1127.97,47.035 S vtm_output_block/block.vtm.txt both`
	`inst_6 super_filter 772.97,47.035 FS vtm_output_block/block.vtm.txt both`
	`inst_7 super_filter 417.97,47.035 FN vtm_output_block/block.vtm.txt both`

`-format {simple | metal | stack}`	

	Specifies the format of the thermal map file. The possible arguments are: • `simple`: contains only power tables. • `metal`: contains only metal density tables • `stack`: contains both power and metal density tables.
`-merged_file` *`filename`*	
	Specifies the name of the merged VTM.
`-rundir` *`directory`*	
	Specifies the output folder for the merged thermal model.

Examples

- The following command specifies to generate a merged VTM:
  ```
  merge_vtm -config /Run_Merge_VTM/design_both.cfg -rundir merged_vtm_out_both -merged_file
  merge_vtm.txt
  ```
- The following command specifies to generate a merged VTM containing only power tables:
  ```
  merge_vtm -config /Run_Merge_VTM/design_both.cfg -rundir merged_vtm_out_simple -merged_file
  merge_vtm.txt -format simple
  ```
- The following command specifies to generate a merged VTM containing only metal density tables:
  ```
  merge_vtm -config /Run_Merge_VTM/design_both.cfg -rundir merged_vtm_out_metal -merged_file
  merge_vtm.txt -format metal
  ```
- The following command specifies to generate a merged VTM containing both power and metal density tables:
  ```
  merge_vtm -config /Run_Merge_VTM/design_both.cfg -rundir merged_vtm_both_both -merged_file
  merge_vtm.txt -format stack
  ```

Related Topics

- `set_metal_density_options`

propagate_activity

```
propagate_activity

[-set_net_freq {true | false}]
```

Propagates the activity file in the database after it is read in using the `read_activity_file` command. This command propagates the activity for all primary inputs, nets, and other devices in the design whose activity has not been previously defined through user attributes.

`propagate_activity` can be used to propagate activities in the design and check for activity annotation before running power calculation. This is not a mandatory step as execution of `report_power` will also do the same and then do power calculation based on the propagated activities.

Parameters

`-set_net_freq {true | false}`	
	Specifies that activity from the generated TCF file needs to be used to capture net frequency for signal EM analysis. When this parameter is set, you do not have to read the TCF file to set net frequency.

Example

- The following commands read and propagate the activity file in the database:

  ```
  read_activity_file
  propagate_activity
  ```

Related Information

- `read_activity_file`
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

query_power_data

```
query_power_data
[-help]
[<in_file <file1[file2] ...>>]
[-average_current]
[-clock_average]
[-dump_pwl]
[-frame <string>]
[-instance_list <inst_list>]
[-instance_list_file filename]
[-list]
[-output <string>]
[-peak_current]
[-peak_time]
[-pin]
[-time_steps]
[-stdout]
[-total_current <input_current_file> ]
[-output_tap_name <tap_name>]
```

The `query_power_data` command allows you to query the binary current files (.ptiavg/.ptipeak) that Power Calculation/Rail Analysis creates. It reads the tap current data in the current files and reports various information for debugging.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`<in_file <file1[file2] ...>>`	
	Specifies the names of the VDD and VSS instance voltage pti files.
`-average_current`	
	Specifies to write the average current of all instances.
`-clock_average`	
	Specifies to write the clock based current report.
`-dump_pwl`	
	Specifies to write the tap current in the PWL format. You can use this parameter to view the contents of the binary data file.
`-frame <string>`	

	Specifies intervals for the clock_average report.
`-instance_list <inst_list>`	
	Specify the list of instances to be dumped in the ASCII format, one instance per line. If not specified, it prints all instances. The `-instance_list` parameter is specified with the `-dump_pwl`, `-peak_current`, `-peak_time`, and`-average_current` parameters.
`-instance_list_file filename`	
	Specifies a file with the list of instances to be dumped in the ASCII format.
`-list`	
	Specifies to write the tap names from the input file.
`-output <string>`	
	Specifies the name of the output file.
`-output_tap_name <tap_name>`	
	Specifies the name of the output tap in the new current file, which is created using the `-total_current` parameter.
`-peak_current`	
	Specifies to write the peak current of all instances
`-peak_time`	
	Specifies to write the peak current time of all instances.
`-pin`	
	Specifies to print the pin names with the instance names. The `-pin` parameter is specified with the `-dump_pwl` and `-list` parameters.
`-stdout`	
	Specifies to print the report to standard output.
`-time_steps`	
	Specifies to summarize the minimum current, maximum current, average current, and total current for each interval in the data file. You can use this data to estimate which interval in a transient response might contain the worst-case data.
`-total_current <input_current_file> <output_total_current_file>`	

	Specifies to create a new current (.ptiavg) file containing the total current for all taps of a net. This parameter can be used if you have a very large current file generated by the Power Calculation engine and want to see only a total current waveform in the SimVision waveform viewer. *input_current_file* is the current file that will be read in (output from the Power Calculation engine). Use the following syntax to create a total current file: `query_power_data -total_current <input_current_file> -output <output_total_current_file>` `-output` is used to specify a user-defined output current filename, and is not a mandatory parameter. If not specified, the output file will be generated automatically with the name of the input file suffixed with `.ptiavg`.

Example

- The following command creates a total current waveform file `VDD` from the input file `dynamic_VDD.ptiavg`:
  ```
  query_power_data -output_tap_name vdd_tap -total_current dynamic_VDD.ptiavg -output VDD
  ```

Related Information

- `report_power`
- `set_power_analysis_mode`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

read_activity_file

```
read_activity_file name
[-end { time1 time2 ...timen } ]
[-format { VCD | TCF | SAIF | FSDB | PHY | SHM}]
[-hier_separator separator ]
[-reset]
[-start { time1 time2 ...timen } ]
[-set_net_freq {true | false}]
[-block block_name ]
[-scope scope_name ]
[-scale_duration scalefactor ]
[-start_time_shift value ]
[-rtl {true|false}]
[-name_mapping_rule file]
[-weight value]
[-zero_delay {true|false}]
[-cell cell_name ]
```

Specifies the name and type of activity file to be used as input. This command must be used when specifying activity files for static or dynamic power calculation.

Parameters

`-block block_name`	
	Specifies the name of the FSDB/SAIF/TCF/VCD block to map a sub-block activity file with a top level Verilog file. Provides the ability to support multiple activity files based upon block instance names. The block argument is added to the FSDB/SAIF/TCF/VCD name before comparing it to the Verilog file.
`-cell cell_name`	

	Specifies the master cell name for the activity file. This parameter allows you to support master-cell-based activity specification. That is, you need to specify only the master cell name for the activity file, instead of specifying all the instances that were instantiated from a master cell.
	Example
	Let us consider a design that has a cell named "super_filer" with four block instances "super_filer1", "super_filer2", "super_filer3",and "super_filer1":
	Instance-Based Use Model:
	`read_activity_file -format VCD -block super_filer1`
	`read_activity_file -format VCD -block super_filer2`
	`read_activity_file -format VCD -block super_filer3`
	`read_activity_file -format VCD -block super_filer4`
	Master-Cell-Based Use Model:
	`read_activity_file -format VCD -cell super_filer`

`-end {`*`time1 time2 ...timen`*`}`	

	When you use the vector-driven method of performing dynamic instance-based power-consumption calculation, you can specify the part of the activity file from which you want power analysis to use for the power calculation. This option specifies the end time of this range and is used in conjunction with the `-start` option. Specifies the end time as time-value pairs to report the average power across non-overlapping multiple windows specified in the activity file for a block, level of hierarchy, or other part of the design that you want to analyze. Units are in seconds (`s`), milliseconds (`ms`), microseconds (`us`), nanoseconds (`ns`), or picoseconds (`ps`). *Default*: The time unit is defined in the first `.lib` file read during design import. You can specify the `$worst_power_window_end` variable with the `-end` parameter to take the worst case end time from the last vector profiler run. Not required for TCF and SAIF. **Note**: You can read multiple windows in a single VCD file by specifying multiple pairs of the start time and end time for the non-overlapping multiple windows specified in the activity file. The use model for reading multiple windows is: `read_activity_file -format VCD design.vcd -start {start1 start2 start3} -end {end1 end2 end3}` The *start1/2/3* and *end1/2/3* values should be in the ascending order. Multiple current waveforms representing multiple VCD windows are merged into a single continuous current waveform. To capture the waveform for signals at the edge of user windows, a buffer time is added between the windows being stitched using the `set_power_analysis_mode -settling_buffer` parameter. This buffer time ensures that the stitched waveform is contiguous. The generated waveform hence has the duration of user windows and the intervening settling buffers. The default settling buffer period is 400ps.
`-format {VCD | TCF | SAIF | FSDB | PHY | SHM}`	
	Specifies the net activity of the design using an activity file in the VCD, TCF, FSDB, SAIF, PHY, or SHM format. The arguments specified with the `-format` parameter are case-sensitive. • The VCD file format uses the toggle count information as the basis for the power consumption calculation. These files can be in compressed (zipped) format using the .gz extension. • The TCF file format contains data on the switching activity of the nets in the design. The switching activity includes the toggle count information and the probability of the net or pin being in the logic 1 state. These files can be in compressed (zipped) format using the .gz extension. Supports extended TCF which contains transition density for X and Z as shown in the following extended TCF example:

"out" :"0.0 0" "0.005000 2" "0.0 0"

The fields are defined as follows:

Field 1: net or pin name
Field 2: duty and transition density for transition of 0-1
Field 3: duty and transition density for transition of X(0-X-1)
Field 4: duty and transition density for transition of Z(0-Z-1)

- The SAIF file format contains aggregate switching activity information on the nets and/or ports in the design. It does not contain time-based information. This file format is used for quick average power analysis of the design.
- The FSDB file format specifies the net activity of the design using an activity file in FSDB format. Innovus supports the FSDB version up to 6.0.
  Use Model

  ```
  read_activity_file -format FSDB fsdbfile
  report_power
  ```

  (or)

  ```
  report_vector_profile
  ```

  Example

  ```
  read_activity_file -format FSDB test.fsdb -scope top
  ```

- The PHY (Palladium database format generated in the Palladium environment) format contains the simulation activity data generated during emulator-based simulation. The PHY format can be used in all vector-based power analysis flows (vector profiler, static power analysis and dynamic vector-based power analysis).
  Use Model

  ```
  read_activity_file-format PHY <path to PHY database directory>
  ```

  Example

  ```
  read_activity_file -format PHY ./phy2/trace.phy -scope test
  ```

  Notes:

  - The xeDebug software must be present in the `PATH` environment. Add xeDebug in the path using the following command:
    ```
    set path = (<install_dir>/bin $path)
    ```
    The xeDebug software version should be the same as the one using which the PHY database is generated.
  - The PHY database generated from the following UXE/VXE version is supported:

    ```
    UXE18.6.0 onwards
    VXE18.6.0 onwards
    ```

  - When reading a PHY format activity file, you require a DPA

(DPA_ToggleCountFormat) license.

- For information on error messages displayed while reading the PHY database, refer to the `xe.msg` file in the PHY database directory.
- Make sure that the PHY database is not locked before invoking the Voltus software.
- Make sure that the xeDebug software is not already running on the machine.
- Ensure that the following directories are present in the top-level PHY database directory: `*.phy`, `PDB`, `cellList`, `QTDB`, `dbFiles`, and `.design`.
- It is recommended that the write permission is assigned on the entire PHY database. In case the write permission cannot be granted, you must set the `SWFV_FREEZE_PHY` environment variable to `1`.
- If you want to run parallel Voltus executables on the same top-level PHY directory with multiple `*.phy` databases, it is suggested to create local copies of the multiple top-level directories, with links to the original PHY database. This will allow local copies of the PHY database to acquire separate locks.

- The Simulation History Manager (SHM) database format is a record of the data signal changes that occur during design simulation. The SHM database consists of a directory, typically with the "`.shm`" suffix. The directory contains two disk files, namely `*.trn` and `*.dsn`. For more information about the SHM format, refer to the *SimVision Analysis Environment* manual available on the Cadence Online Support (COS) website.
  Example
  ```
  read_activity_file -format SHM  /../shm/ADDER.trn -scope
  adder_e/u1 -start 0ns -end 60ns
  ```

`-hier_separator` *separator*

Specifies the separator character in the hierarchical net names, bus names, and pin names for the block or other part of the design that you want to analyze. It must be the same as the separator character in the design netlist.

Default: slash (/)

`-name_mapping_rule` *file*

	Specifies to enable rule-based mapping to map RTL vectors to gate-level netlist. This parameter helps in resolving name mismatch between RTL and gate level netlists.
	The name mapping rule file format is:
	`Name_Mapping_Rule_file`
	`# RTL  # Gate`
	`<rtlstring> <gatestring>`
	where, each line is a rule. The string in the first column(RTL) is what we want to replace with the string in the second column(Gate). The rules in every line are added onto the previous rules.
	For example, to replace all the square `[]` brackets with under score _ for RTL-to-Gate matching, the rule file will be:
	`#RTL  #GATE`
	`[  _`
	`]  _`
	Note: `-name_mapping_rule` is not supported when the `set_power_analysis_mode -comprehensive_automapping` parameter is set to `true`.

`-reset`	If `-reset` is specified, all previous activity files specified using the `read_activity_file` command will be reset and ignored. *Default*: All the previous activity files specified using the `read_activity_file` command are loaded and used (They are additive).
`-rtl {true|false}`	
	Specifies to process the vector as an RTL vector and perform state propagation. The propagation behavior is global and the full design is propagated. RTL vectors are zero-delay vectors so delay annotation is always done for signals from this vector file. *Default*: `false`
`-scale_duration` *scalefactor*	
	Scales the FSDB/SAIF/TCF/VCD duration value by the specified scale factor. For example, an FSDB scale of 2 will double the FSDB duration value. **Note:** The `-start` and `-end` parameters of the `read_activity_file` command are post scaled.
`-scope` *scope_name*	

Specifies the FSDB/SAIF/TCF/VCD scope, that is, the name of the module within the activity file associated with the block or other part of the design that you want to analyze.

The scope argument is removed from the FSDB/SAIF/TCF/VCD name before comparing it to the Verilog file.

⚠ If the scope value has backslash '\' or square brackets '[]', you must specify the scope within curly brackets {{…..}} to ensure that '\' or '[]' are correctly processed by the Tcl Interpreter. In the following example, the scope value is specified within curly brackets as it contains '\' and '[]':

```
read_activity_file -scope
{{reg_based_buffer.reg_inst\/sv_xpol_trans_q_reg[0][14][7]
[sv_re][4]}}
```

If '\' and '[]' are not there, curly brackets are not required.

Example

The following command specifies the u1 module within the VCD file as the module associated with the part of the design being analyzed, and specifies the `dut_5buf_full_chip` block instance as the block to map:

```
read_activity_file \
-format  VCD \
-scope  adder/u1 \
-start  {0ps 2000ps} \
-end    {1100ps 3100ps} \
-block  ../vcd/dut_5buf_full_chip
```

`-set_net_freq {true | false}`

This option should be set when an activity from a TCF or VCD file needs to be used to capture net frequency for signal EM analysis (`verifyACLimit`). In addition, you must use the `verifyACLimit -use_db_freq` option to read the toggle rates from the activity file.

Default: `false`

Example

The following example reads a TCF file which is used to capture net frequency for signal EM analysis:

```
read_activity_file -format TCF -set_net_freq true design.tcf
verifyACLimit -use_db_freq -report signal_em.rpt
```

`-start { time1 time2 ... timen }`

	When you use the vector-driven method of performing dynamic instance-based power-consumption calculation, you can specify the part of the activity file from which you want power analysis to use for the power calculation. This option specifies the start time of this range and is used in conjunction with the `-end` option. Specifies the start time as time-value pairs to report the average power across non-overlapping multiple windows specified in the VCD file for a block, level of hierarchy, or other part of the design that you want to analyze. Units are in seconds (`s`), milliseconds (`ms`), microseconds (`us`), nanoseconds (`ns`), or picoseconds (`ps`). *Default*: The time unit is defined in the first `.lib` file read during design import. You can specify the `$worst_power_window_start` variable with the `-start` parameter to take the worst case start time from the last vector profiler run. Not required for TCF and SAIF. **Note**: You can read multiple windows in a single VCD file by specifying multiple pairs of the start time and end time for the non-overlapping multiple windows specified in the activity file. The use model for reading multiple windows is: `read_activity_file -format VCD design.vcd -start {start1 start2 start3} -end {end1 end2 end3}` The *start1/2/3* and *end1/2/3* values should be in the ascending order. Multiple current waveforms representing multiple VCD windows are merged into a single continuous current waveform. To capture the waveform for signals at the edge of user windows, a buffer time is added between the windows being stitched using the `set_power_analysis_mode -settling_buffer` parameter. This buffer time ensures that the stitched waveform is contiguous. The generated waveform hence has the duration of user windows and the intervening settling buffers. The default settling buffer period is 400ps.
`-start_time_shift` *value*	
	Specifies to shift the start time of the specified activity files. When you specify the `-scale_duration` parameter with the `-start_time_shift` parameter, it scales the start time shift along with the start/end time.
name	The name of the activity file.
`-weight` *value*	

	Specifies the weight number of the TCF file. `1.0` is the default value. The value should be a floating number that is smaller than `1.0`. This parameter allows you to merge multiple TCF files at different function modes for static power estimation of a design.
	In the following example, TCF *file1*, *file2*, and *file3* has the weight numbers 0.3, 0.2, and 0.4, respectively.
	`read_activity_file -format TCF file1 -weight 0.3`
	`read_activity_file -format TCF file2 -weight 0.2`
	`read_activity_file -format TCF file3 -weight 0.4`
	Here, *file1*, *file2*, and *file3* will be merged together to represent the toggle and duty for the nets/pins in the design. The relative weight of 0.3/0.9, 0.2/0.9, and 0.4/0.9 would be used for static power calculation. In this example, 0.9 is the sum of weights.

`-zero_delay {true|false}`	
	Specifies to process the vector as a zero-delay vector and perform delay annotation. *Default* : `false`

Activity Precedence

The following commands can define activity:

```
read_activity_file file
     [-format { VCD | TCF | SAIF ]}
     [-hier_separator separator ]
     [-start time ]
     [-end time ]
     [-reset]
     [-scope scope_name ]
     [-block block_name ]

set_default_switching_activity
    [-input_activity factor ]
    [-seq_activity factor ]
    [-period value ]
    [-duty value ]
    [-global_activity factor ]
    [-hier hierarchy _name]
    [-clock_gates_output activity_factor ]

set_switching_activity
    [-reset]
    [-clock clock_name]
    [-period value ]
    [-unclocked]
```

```
[-activity factor | -density transition_density ]
[-net net_name | -port port_name | -pin pin_name ]
[-duty value ]
[-inst]
```

The precedence of activity is as follows:

1. User-Defined: Activities applied to specific nets/pins/ports
2. Activity File: Activity specification through a VCD/FSDB/SAIF/TCF format
3. Clock Gates Output Activity: Activity specification to the output of all clock gates
4. SDC/TWF constants: Set constants to a list of pins or ports for use by the timing engine (timing analysis)
5. Hierarchical Global Activity: Activity specification for a specific hierarchy
6. Global Activity: Activity specification for all instances that are part of the data network.
7. Sequential Element Activity: Activity specification applied to the output of all sequential elements.
8. Primary Input Activity: Activity specification for all primary inputs

If you specify multiple activity data for the same items, the power engine will use the one with the higher precedence.

Examples

- The following command reads the compressed ap_wait_test_pll1_mod.vcd.gz VCD file that contains the toggle count information for the power consumption calculations, specifies the crm_ap module within the VCD file as the module associated with the part of the design being analyzed, specifies the slash character (/) as the hierarchical separator for hierarchical net names, bus names, and pin names for the part of the design being analyzed, and specifies the start (180002ns) and end (189802ns) times from the VCD file for the part of the design being analyzed:

```
read_activity_file -format VCD \
-scope testbench/top/ap/mcu_platform/crm_ap \
-hier_separator \
-start 180002ns \
-end 189802ns \
ap_wait_test_pll1_mod.vcd.gz
```

- The following command reads the compressed A.vcd.gz VCD file that contains the toggle count information for the power consumption calculations, specifies A1 as the block to map a sub-block VCD file with a block instance at the top-level, and specifies the start (180002ns) and end (189802ns) times from the VCD file for the part of the design being analyzed:

```
read_activity_file -format VCD \
-block ap/A1 \
-start 180002ns \
-end 189802ns \
A.vcd.gz
```

- The next example specifies a VCD activity file called dmac_mac.vcd for the scope top/dma_dut.

```
read_activity_file -format VCD -start 10ns -end 20ns \
-scope top/dma_dut dmac_mac.vcd
```

- The following command specifies multiple pairs of the start time and end time for the multiple windows in the VCD file：

```
read_activity_file \
-format  VCD \
-scope  adder/u1 \
-start  {0ps 2000ps} \
-end    {1100ps 3100ps} \
-block  ../vcd/dut_5buf_full_chip.vcd
```

- The following command specifies to read the RTL-level VCD activity file:

```
read_activity_file -format VCD -scope scope_top -start 300ns -end 700ns /DesignData/VCD/rtl.vcd -block
inst_C1 -rtl true
set_power_analysis_mode -method vector_profile -vector_profile_mode transient -worst_window_type full
-power_grid_library /DesignData/PGV/library_pv.cl
```

Related Information

- set_default_switching_activity
- set_switching_activity
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

report_instance_power

```
report_instance_power
[-help] instance [-file filename]
```

Generates a text based report for the specified instance. This command enables you to determine how the different components of static power are calculated. You can use the report to debug static power numbers after performing power analysis (`report_power`).

This command is available through both the static and dynamic power engines. This command is supported in the dynamic power engine by including this command in the power include file through the command `set_power_include_file`.

Note: For dynamic power engine, the `report_instance_power` command does not generate text reports incrementally. If the dynamic run has been performed and you need the debug information for a static power number, you should re-run power analysis.

Parameters

`-file` *filename*	Writes out the static power analysis report into the file that you specify.
`-help`	Outputs the command usage and a brief description about the command parameters.
instance	Specifies the instances to include in the instance power report.

Example

- The following command specifies an instance power report file `debug.rep` for the instance `top/i_1`:

  ```
  report_instance_power -file debug.rep top/i_1
  ```

Related Information

- `report_power`
- `set_power_analysis_mode`
- "Static Power, IRdrop and EM Analysis" in the *Voltus Stylus Common UI User Guide*
  - "Debugging Instance Power"

report_power

```
report_power
[-cap]
[-cell {cell_list}]
[-cell_type {all | {macro io combinational sequential clock_combinational clock_sequential}}]
[-clock_domain {clock_domain_list}]
[-clock_network {all | {clock_list}}]
[-count_seq_elements_in_clock_network]
[-die_instance_name dieinstname]
[-exclude_cells {cell_list}]
[-exclude_cells_file filename ]
[-exclude_instances {instance_list}]
[-exclude_instances_file filename ]
[-instances {instance_list}]
[-hierarchy {all | hierarchy_level}]
[-hierarchical_instances {hierarchy_inst_list}]
[-leakage]
[-net [-nworst number_of_nets]]
[-no_wrap]
[-outfile filename]
[-pg_net {all | pg_net_name_list}]
[-power_domain {all | {power_domain_list}}]
[-sort {internal | switching | leakage | total}]
[-threshold value]
[-view view_name]
[-threshold_voltage_group { all | group_name}]
[-clock_gating_efficiency]
[-register_gating_efficiency]
[-cluster_gating_efficiency]
[-thermal_leakage_temp {temp_list}]
[-thermal_power_map_file file_name]
[-thermal_power_map_tile {Xint Yint}]
[-thermal_power_map_format {simple | stack }]
[-pg_pin]
[-thermal_conductivity_inputs file_name]
[-output directory]
[-o directory]
[-report_prefix prefix]
[-toggle_rate]
[-format { simple | detailed }]
[-comb_seq_power]
[-group_type user_defined_group_name]
[-distribute]
[-print_memory_power]
[-cap_unit unit]
[-cell_list_file filename]
[-float_precision value]
[-inst_list_file filename]
[-power_db_directory directory]
[-power_unit unit ]
```

```
[-time_unit unit]
[-power_density_tiles value]
[-power_density_tiles_row_col {value1 value2}]
[-power_density_tiles_size {value1 value2}]
[-compress compression_ratio]
[-stat]
[-time_based_report]
[-thermal_power_map_header_include_file filename]
[-thermal_leakage_temperature_scale_table_file filename]
[-thermal_leakage_temperature_scale_table {$temp1 $scalefactor1 $temp2 $scalefactor2 $temp3 $scalefactor3
.....}]
[-thermal_material_file filename]
[-thermal_power_map_SI_unit {true | false}]
[-thermal_power_map_powertable_tile {Xint Yint}]
[-thermal_power_map_densitytable_tile {Xint Yint}]
[-thermal_power_map_format_version {new|default}]
[-thermal_power_map_bottom_layer layer_name]
[-thermal_power_map_top_layer layer_name]
[-use_geometry_cell_size {true | false}]
```

Reports global power, specific instance power, clock network power, clock domain power, and net switching power. The command also reports the power for power domains and specific power nets. The power unit used throughout the reports is Milliwatt.

This command starts the execution of power analysis (both static and dynamic). The `report_power` command has a character limit of 1015.

> The parameters of the `report_power` command, except `-cap`, `-clock_gating_efficiency`, `-register_gating_efficiency`, and `-cluster_gating_efficiency`, are not supported in the dynamic-only mode (`set_power_analysis_mode -disable_static true`). These parameters are only supported in the static mode. However, the `-power_domain` parameter is not supported even if the static power mode is enabled.
> The `set_power_include_file` command can be used to pass the `report_power` command options in the dynamic analysis mode.

The `report_power` command allows generation of multiple power reports with a single `report_power` command. It can be used to generate all the static reports during dynamic analysis. `report_power` eliminates the need to run incremental `report_power` in static power analysis using a Verilog netlist. By default, a detailed power summary report, and a short summary will be printed at the beginning of all reports. To enable this flow, specify the following parameters:

- `-output/o`
- `-report_prefix`

The following `report_power` command parameters are supported by the multi-report flow, both in static and dynamic power analysis:

- `-cell`
- `-clock_gating_efficiency`
- `-register_gating_efficiency`

- `-cluster_gating_efficiency`
- `-net`
- `-instances`
- `-pg_net`
- `-clock_network`
- `-hierarchy`

The following parameters are not supported in the multi-report flow:

- `-cell_type`
- `-power_domain`
- `-clock_domain`
- `-clock_gating_efficiency` and `-clock_network` in the same run

⚠ In the static power analysis flow, you can run the `report_power` command after the `restore_power_database` command without loading the design. The following command is an example of the use model:

```
restore_power_database -file power.db
report_power -output same -report_prefix report -clock_network {all} -instances {*}
```

The following parameters of the `report_power` command are supported without design loading:

- `-cell`
- `-cell_type`
- `-instances`
- `-clock_network`
- `-clock_domain`
- `-pg_net`
- `-pg_net -format detailed`
- `-hierarchy`
- `-hierarchical_instances`

Parameters

`-cap`	Dumps out the list of net, pin, and total capacitances in three different columns.
`-cap_unit` *unit*	
	Sets the capacitance unit to be used for power report generation. The default unit is pF (picoFarads).
`-cell` **{***cell_list***}**	
	Specifies the cells to be included in the power report. Accepts wildcards (*)
`-cell_list_file` *filename*	
	Specifies a file containing a list of cells that are to be included in the power reports.
`-cell_type {all | {macro io combinational sequential clock_combinational clock_sequential}}`	
	Specifies the cell types to be included in the power report. *Default*: `all` When you specify the cell types, `-cell_type combinational` or `sequential`, the power report does not show the combinational clock or sequential clock cells.
`-clock_domain` ***{****clock_domain_list****}***	
	Generates detailed instance power reports for the specified clock domains. Wildcards are not accepted. **Usage** `report_power -clock_domain {clock_domain_list} -outfile filename`
`-clock_gating_efficiency`	

	Generates a report that includes the Clock Gating Efficiency (CGE) for all clock gating instances, as well as different hierarchies in the design. Average CGE for different hierarchies is also reported.
	Clock Gating Efficiency is calculated as:
	`CGE = (toggles at clock gate output / toggles at clock gate input)`
	`Average CGE = (avg of all CGEs above)`
	When a CGE report is generated using the command `report_power -clock_gating_efficiency -outfile cge.rpt`, the following information will be displayed:
	• **Clock-gating Efficiency Report** - for each clock domain, it includes the toggle rate, number of registers, number of clock gates, average clock toggle at registers, average toggle savings at registers, and average toggle savings histogram.
	• **Hierarchical View of Average Toggle Savings** - number of clock gates and average toggle savings for each hierarchical module in the design.

`-clock_network` {`all` | **{***clock_list***}**}	

	Reports the power consumption of the clock networks, including the power for the generated clocks. Use `clock_list` to report the power consumed by specific clocks. *Default*: `all`
`-cluster_gating_efficiency`	
	Generates a cluster efficiency report in the RGE report. This report shows the CGE/RGE metrics for the registers at the fan-out of each clock gate instance in the design.
`-comb_seq_power`	
	Generates a report that shows the ratio of combinational versus sequential power for all modules of a given hierarchy. This parameter must be specified with the `-instances` parameter.
`-compress` *`compression_ratio`*	
	Specifies to compress the power analysis report files in the GNU Zip format. The compressed files will have the suffix `.gz`. *`compression_ratio`* is from 0-9. `0` indicates that the report files are not to be compressed. The compression levels vary from `1` to `9`. `1` provides the fastest compression speed but at a lower ratio, so the file size will be larger. `9` provides the highest compression ratio but at a lower speed. *Default*: 0
`-count_seq_elements_in_clock_network`	
	Includes the leaf flip-flop power in the clock network power report. *Default*: The leaf flip-flop power will not be included in the clock network power report.
`-die_instance_name` *`dieinstname`*	
	Specifies the die instance name for multi-die analysis.
`-distribute`	Enables standalone distributed power analysis. This parameter is used for running dynamic power analysis in the multi-CPU mode without running rail analysis.
`-exclude_cells {`*`cell_list`*`}`	
	Specifies a list of cells that should be excluded from power analysis.
`-exclude_cells_file` *`filename`*	

Specifies a file containing a list of cells (one per line) which should be excluded from power analysis.
`-exclude_instances {`*`instance_list`*`}`
Specifies a list of instances that should be excluded from power analysis.
`-exclude_instances_file` *`filename`*
Specifies a file containing a list of instances (one per line) which should be excluded from power analysis.
`-float_precision` *`value`*
Sets the precision value to be used for power report generation. The default value is 4.
`-format { simple | detailed }`
Specifies the type of report summary required for static power reporting. The possible arguments of this parameter are: • `simple` - reports the amount of power consumed by all power nets in a condensed format, breaking down power based on the switching, internal, and leakage components. • `detailed` - reports the amount of power consumed by all power nets in a detailed format. The detailed format includes additional information, such as output pin capacitance (`Cpin Sum`), output net capacitance (`Cnet Sum`), Total Load Capacitance (`Cpin Sum` + `Cnet Sum = Total Load_Cap`), maximum frequency at the input and output pins (`Max_Clk_Freq`), maximum activity factor at the input and output pins (`Max_Activity`), cell name of the instance (`Cell Name`), clock or data cell (`C/D`), and library used in power analysis for the specific cell (`Lib`). The libraries are numbered in the header section. The name of the generated report file is `report.*.detailed.rpt`. Currently, the `-format detailed` parameter is only available when it is used with the `-pg_net` parameter.
`-hierarchical_instances {`*`hierarchy_inst_list`*`}`
Specifies to report all the power content for the specified hierarchical instances/ blocks. It reports the `Total Power` (a breakup of power components) and `Group` (a breakup of power by cell type) of the specified block at each hierarchy. To generate a complete hierarchical instance power report, it is recommended to specify `-hierarchical_instances` with `-report_prefix`. The report is written in the specified output directory. If you specify `-outfile` instead of `-report_prefix`, an instance file is output without hierarchy-level details. `-report_prefix` generates the following 3 reports: • `<`*`prefix`*`>.rpt` - reports power details for all instances in the design. • `<`*`prefix`*`>.hierpwr.rpt` - reports hierarchical power, that is, power summary, internal, switching, leakage, and total power for all hierarchical instances. • `<`*`prefix`*`>.hierinstpwr.rpt` - reports power details for the requested block instances. **Note**: `-hierarchical_instances` must be specified with `-hierarchy` to include hierarchy-level power report. When `-hierarchical_instances` is used without `-hierarchy`, only top-level summary of the power consumption is written.

`-hierarchy {all | hierarchy_level}`	
	Specifies the hierarchy level that is included in the power report. You can specify the *hierarchy_level* as a number that corresponds to a specific hierarchy level in the design. A *hierarchy_level* of 0 specifies the top level. *Default*: `all`
`-instances {instance_list}`	
	Specifies the instances to include in the power report. The power report provides the amount of power consumed by a specified list of hierarchical or leaf-level instances. *instance_list* accepts wildcards (*) for leaf-level instances only. An instance with 0 power will not be listed in the power report.
`-inst_list_file filename`	
	Specifies a file containing a list of instances that are to be included in the power reports.
`-leakage`	Reports leakage power of the design.
`-net`	Reports the net switching power, load capacitance, toggle rate, and switching values for each net in the design.
`-no_wrap`	Displays the `report_power -cell_type all` output on a single line in the output report file.
`-nworst number_of_nets`	
	Limits the reporting to the specified number of nets having the highest switching power. **Note:** The `-nworst` parameter must be specified with the `-net` parameter.
`-output/-o directory`	
	Specifies the name of the directory where the power analysis reports are saved. The default is the current working directory. This parameter eliminates the need to specify `set_power_output_dir`. However, if both `report_power -output` and `set_power_output_dir` are specified, `-output` has higher priority.

`-outfile filename`

Specifies the name of the report file to which the power consumption information is written. Hierarchical directory names can be included.

You can convert a power report file to the gzip file format by specifying `-outfile filename.gz`.

Note: If you do not specify this parameter, the software directs the power report to the command line, as well as the log file, and no power report is written to a separate file.

`-pg_net {all | pg_net_name_list}`

When set to `all`, the amount of power consumed by all power nets is reported. If a list of nets is specified, the power consumption of each instance that is connected to the specified power net is separately reported.

Note: When using this parameter, the static power calculation engine always performs propagation and power calculation.

Default: `all`

`-pg_pin`

Reports power consumed by all power pins in a design.

`-power_db_directory directory`

Specifies the location of the power database that the Power Calculation engine uses for generating standalone power reports.

`-power_domain {all | {power_domain_list}}`

When set to `all`, the power consumed by all power domains is reported. If a list of power domains is specified, the reporting will be limited to those power domains. A summary of power break-down for each domain, before listing power for each instance in the domain, is also generated.

`-power_density_tiles value`

Specifies the number of tiles used for power density calculation.

Default: `100`

`-power_density_tiles_row_col {value1 value2}`

Defines the number of tiles by rows * columns. *value1* is the number of tile rows and *value2* is the number of tile columns.

The following example specifies to perform power density aware vector profiling using 2 tile rows and 2 column rows:

```
set_power_analysis_mode -method vector_profile -vector_profile_mode power_density -
worst_step_size 10ns
report_power  -power_density_tiles_row_col  {2 2} -time_based_report -outfile tiling1.rpt
```

The following is the vector profile report for this command:

```
Tile 0_0_0 5.06 5.33 14.95 14.555
Tile 1_0_1 14.95 5.33 24.84 14.555
Tile 2_1_0 5.06 14.555 14.95 23.78
Tile 3_1_1 14.95 14.555 24.84 23.78
...
...
```

`-power_density_tiles_size {`*value1 value2*`}`

Defines the absolute tile size. *value1* is the horizontal width of a tile (in micron) and *value2* is the vertical height of a tile (in micron).

`-power_unit` *unit*

Sets the power unit to be used for power report generation. The default unit is mW (miliAmps).

`-print_memory_power`

Specifies to report power values for the memory and macro cells under separate groups (`Memory` and `Macro`).

The following is a snapshot of the power report showing separate groups for memory and macro cells:

Group	Internal Power	Switching Power	Leakage Power	Total Power	Percentage (%)
Sequential	0.4031	0.1283	0.006583	0.538	18.38
Memory	0.1624	0.004345	0.011	0.1778	6.074
IO	0	0	0	0	0
Physical-Only	0	0	2.91e-06	2.91e-06	9.945e-05
Combinational	0.8711	1.05	0.01579	1.937	66.2
Clock (Combinational)	0.002302	0.01018	7.654e-05	0.01255	0.429
Clock (Sequential)	0.01177	0.008387	0.000347	0.0205	0.7005
Macro (non-mem)	0.2317	0.007259	0.001234	0.2402	8.208
Total	1.682	1.209	0.03503	2.926	100

`-group_type` *user_defined_group_name*

Specifies to report power (internal/switching/leakage/total) for a user-defined collection of cells that are stated under a power group in the power summary report. The following Tempus commands can be used to create power groups:

- `define_property` - define a property.
- `set_property` - assign the defined property (group type) and its value (power group) to a list of cells.

The use model of the `-group_type` parameter is:

```
report_power -group_type user_defined_group_type
```

Here, *`user_defined_group_type`* is the user-defined power group type applied to a list of cells in a design. When the `-group_type` parameter is specified, the command reports power for the power groups associated with the specified group type in the power summary report.

Example:

The following command defines the `poly_bias` property (group type) on library cells:

```
define_property poly_bias -object_type lib_cell -type string
```

The following commands apply the property `poly_bias` with the group name `P0` and `P10` to the collection of cells:

```
set_property [get_lib_cells -of_objects [get_cells -hier * -filter {ref_name=~*PB0*}]] poly_bias
P0
set_property [get_lib_cells -of_objects [get_cells -hier * -filter {ref_name=~*PB10*}]]
poly_bias  P10
```

The following command reports power for the user-defined power groups that are associated with the property `poly_bias`:

```
report_power -outfile basic_group_type.rpt -group_type poly_bias
```

The following is a snippet of the power report that shows the user-defined power groups `P0` and `P10` that are associated with the property `poly_bias`, in addition to the existing groups (sequential, macro, IO, and so on):

```
User Group                          Internal   Switching     Leakage       Total  Percentage
                                    Power      Power         Power         Power  (%)
-------------------------------------------------------------------------------------------
P0                                   0.01498   0.0002017     0.01765     0.03283       15.63%
P10                                   0.1268     0.04026     0.01011      0.1772       84.37%
-------------------------------------------------------------------------------------------
Total                                 0.1418     0.04047     0.02776        0.21          100%
-------------------------------------------------------------------------------------------
```

```
Group                          Internal   Switching     Leakage       Total  Percentage
                               Power      Power         Power         Power  (%)
-------------------------------------------------------------------------------------
Sequential                      0.02958   0.0003768     0.01458     0.04453       16.62
Macro                                 0           0           0           0           0
IO                                    0           0           0           0           0
Combinational                0.008217     0.002633     0.01025      0.0211       7.877
Clock (Combinational)           0.1239      0.04421    0.007419      0.1755       65.51
Clock (Sequential)              0.01773    0.008783   0.0002459     0.02676       9.988
-------------------------------------------------------------------------------------
Total                            0.1794       0.056     0.03249      0.2679         100
-------------------------------------------------------------------------------------
```

`-register_gating_efficiency`

Generates a report that includes the Register Gating Efficiency (RGE) and Data Aware Gating Efficiency (DAGE) for all sequential cell instances, as well as different hierarchies in the design. Average RGE and DAGE for different hierarchies is reported. It also displays a section on ICG cluster information, that is, information about registers at the fan-out of each ICG instance along with their RGE metrics.

Register Gating Efficiency is calculated as:

```
RGE = 1 - (toggles at register clock pin/root clock toggles)
```

When an RGE report is generated using the `report_power -register_gating_efficiency -outfile rge.rpt` command, the following information will be included:

- **Register-gating Efficiency Report** - for each clock domain, it includes the toggle rate, number of registers, number of clock gates, average clock toggle at registers, average toggle savings at registers, and average toggle savings histogram.
- **Register Gating Opportunities Report** - this report is sorted based on the Q/CLK toggle ratio. When a register's Q/CLK ratio is greater than .25 (25%), Q toggles every other clock cycle. However, when it is less than 25%, there is a gating opportunity to reduce power.

Data Aware Gating Efficiency is calculated as:

```
DAGE = 1 - ((toggles at clock pin - 2*toggles at Q pin)/(root clock toggles))
```

`-report_prefix` *`prefix`*

Specifies a prefix for the report in which the top-level summary of the power consumption information is written. All reports will be named *`prefix`*`.*.rpt`. The detailed summary will be named `prefix.rpt`.

`-sort` {`internal | switching | leakage | total`}

Sorts the instance-based power consumption data by `internal`, `switching`, `leakage`, or `total` power.

Default: `total`

`-stat`

Reports a set of statistics on the net status after propagation, for example, duty, transition density, or activity. This parameter is useful in viewing and debugging user-defined activity factors after the power run is complete.

Example:

```
report_power -time_based_report -distribute -outfile power.rpt -stat
-------------------------------------------------------------------------------------------
-- Net status after propagation --
-------------------------------------------------------------------------------------------
Net          Transition Density        Duty           Activity        Ref. Freq
-------------------------------------------------------------------------------------------
VDDA1             0              1               0             1.25e+08
VSSA1             0              0               0             1.25e+08
...
```

`-thermal_conductivity_inputs` *`file_name`*

Specifies a file containing the thermal conductivity value for layers. You can use this parameter to include the thermal conductivity value of the following layers in the power map file:

- Silicon
- Metal
- Dielectric
- Micro Bump (optional for 3DIC design)
- Fill material in micro bump layer (optional for 3DIC design)

The thermal conductivity file format is:
Section 1 (layer type based thermal conductivity value)

```
#default
$layer_type1 $thermal_conductivity_parameter1
$layer_type2 $thermal_conductivity_parameter2
...
```

Section 2 (layer name based thermal conductivity value)

```
#perlayer
$layer1_name $thermal_conductivity_parameter1 $dielectric_thermal_conductivity_1
$layer2_name $thermal_conductivity_parameter2 $dielectric_thermal_conductivity_2
...
```

Here,

- You can either specify both Section 1 (layer type based) and Section 2 (layer name based), or specify one of the two sections. If both sections are specified, then layer name based section takes precedence over layer type based section.
- The layer name based section has thermal conductivity value and its dielectric layer's thermal conductivity value for each metal/via layer.
- If section 2 has no parameters defined, the software will use the default parameters defined in the section 1.

Example:
###default

```
Metal      430
Silicon    250
Dielectric 0.6
```

###Perlayer

```
Metal1    430    0.4
Via1      410    0.5
Metal2    390    0.5
Via2      410    0.5
```

...

For information on the thermal conductivity input file and power map file, refer to the "Static Power, IRdrop and EM Analysis" chapter in the *Voltus User Guide*.

`-thermal_leakage_temp {temp_list}`

Defines the temperature to calculate leakage for the power map file.

`-thermal_leakage_temperature_scale_table {$temp1 $scalefactor1 $temp2 $scalefactor2 $temp3 $scalefactor3 .....}`

Specifies the global scale factor for a temperature value to enable scaling of the leakage power values coming from the input dotlib file. This parameter allows you to scale the power values only at certain specific temperatures.

This parameter allows you to generate a multi-temperature leakage power map table from a single input dotlib file in the power map generation flow. `-thermal_leakage_temperature_scale_table` and `-thermal_leakage_temperature_scale_table_file` are mutually exclusive.

`-thermal_leakage_temperature_scale_table_file filename`

Specifies a file containing the user-defined temperature values and the corresponding scale factors for each library, cell, or instance. The format of the file is:

```
#Type Name temp1 temp2 temp3 temp4
Lib    library_name   scalefactor@temp1   scalefactor@temp2  scalefactor@temp3
scalefactor@temp4
Cell   cell_name      scalefactor@temp1   scalefactor@temp2  scalefactor@temp3
scalefactor@temp4
inst   instance_name  scalefactor@temp1   scalefactor@temp2  scalefactor@temp3
scalefactor@temp4
```

This parameter allows you to generate a multi-temperature leakage power map table from a single input dotlib file in the power map generation flow. `-thermal_leakage_temperature_scale_table` and `-thermal_leakage_temperature_scale_table_file` are mutually exclusive.

File Format Example for the Type Lib:

```
#Type Name         25 50 125 150
Lib   fast         1 2 10 100
Lib   bufao        1 2 10 100
Lib   header       1 2 10 100
Lib   pso_ring_wc  1 2 10 100
```

`-thermal_material_file filename`

Specifies the name of a file containing the user-specified substrate layer thickness. The layer thickness value for the power map generation flow can either be a user-specified value or derived from the technology file. If both the values are available, the user-specified value takes precedence.

`-thermal_power_map_bottom_layer layer_name`

Writes zero metal density tables for layers lower than the user-specified layer in the power map file. This parameter prevents duplication of metal density tables when merging the top and block thermal power map files.

`-thermal_power_map_densitytable_tile {Xint Yint}`

Specifies the number of tiles in the X and Y directions for the metal density table.

`-thermal_power_map_file file_name`

Generates the power map file with the specified name in the output directory.
`-thermal_power_map_format {simple | stack }`
Generates a tile-based power map file in the simple or stack format to perform accurate thermal analysis. The stack format must be used to support multiple dies in a package for thermal analysis.
`-thermal_power_map_format_version {new|default}`
Determines whether to generate the default or the new power map file format. When set to `new`, the power map file format with non-uniform tile numbers for the power (`-thermal_power_map_powertable_tile`) and metal density tables (`-thermal_power_map_densitytable_tile`) can be generated. The following are the features of the new report format: • version number added to the header "Version : 2020.0" • nx/ny under the "DIE_AREA" section has been removed • XCOR/YCOR for each layer has been removed • nx/ny has been added for the POWER and METAL_DENSITY sections
`-thermal_power_map_header_include_file` *`filename`*
Specifies a file that contains comments or additional information to be included in the header section of the power map file.
`-thermal_power_map_powertable_tile {`*`Xint Yint`*`}`
Specifies the number of tiles in the X and Y directions for the power table.
`-thermal_power_map_top_layer` *`layer_name`*
Writes zero metal density tables for layers higher than the user-specified layer in the power map file. This parameter prevents duplication of metal density tables when merging the top and block thermal power map files.
`-thermal_power_map_SI_unit {true | false}`
Specifies to change chip thermal power map units to SI in the thermal power map file. This parameter was introduced to support the thermal power map file format used by foundry. When this parameter is set to `true`, the values for all the power map parameters (length, power, and thermal conductivity) will be reported in the SI units. • **Length Unit:** default is `um` and SI is `m` • **Power Unit**: default is `mW` and SI is `W` • **Metal/Silicon Thermal Conductivity Unit**: default is `mW/(um * C)` and SI is `W/(m * C)` *Default* : `true`
`-thermal_power_map_tile {`*`Xint Yint`*`}`
Defines the tile number in the X and Y direction. By default 10*10 will be used.

`-threshold` *value*

Filters out the instances whose total power dissipation is more than the specified value. The power unit used is Milliwatt.

Default: `0`

`-threshold_voltage_group { all | group_name}`

Reports leakage by voltage threshold group in the power report. This parameter supports the argument `all` or any other string, such as hvt or lvt.

Default: `all`

`-time_based_report`

Writes out a detailed VCD profiling report into the file that you specify.

`-time_unit` *unit*

Sets the time unit to be used for power report generation. The default unit is ns (nanoseconds).

`-toggle_rate`

Specifies to print instance-based activity to the outfile. The toggle rate for each instance is calculated based on the clock domain to which it belongs. If multiple clocks are reaching the instance, then the fastest clock is used as a reference.

`-use_geometry_cell_size {true | false}`

Specifies whether to use the actual geometry or placement bounding box for macro PGV cell size during generation of tile-based power map file. This parameter is used when a macro's power is distributed across multiple tiles. When this parameter is set to `true`, the geometry-based bounding box will be used for power map generation and power will be distributed evenly across tiles.

Default : `false`

`-view` *view_name*

Specifies the active timing view that was created using the multi-mode multi-corner (MMMC) `set_analysis_view -setup|-hold` command. Power analysis is performed for the specified view. The command prints the `MMMC` view name in the output file even if you do not specify the `-view` parameter.

If `report_power -view` *view_name* is set, it will override the view set in the `set_power_analysis_mode -view` command. However, if you specify the `set_analysis_view -dynamic` and `-leakage` parameters, then the view set using these `set_analysis_view` command parameters has higher priority over the `report_power -view` command.

Default : Uses current view in Innovus.

Examples

- The following command reports the amount of power consumed by the clock nets clk and test_clk (including leaf flip-flop power) and writes the output information to a power report file named clk_pwr.rep:

```
report_power -clock_network{clk test_clk} \
-count_seq_elements_in_clock_network -outfile clk_pwr.rep
```

- The following command reports the amount of power consumed down to hierarchy level 3 from the top and writes the output information to a power report file named hier.rep:

```
report_power -hierarchy 3 -outfile hier.rep
```

- The following command reports the power consumption and writes the power report file pg_net.rep in the compressed gzip format:

```
report_power -outfile pg_net.rep.gz
```

- The following command reports the amount of power consumed by leaf-level instances that match ECO* and writes the output information to a power report file named inst.rep:

```
report_power -instances ECO* -outfile inst.rep
```

- The following command reports the amount of power consumed by all cells that match buf*6 and writes the output information to a power report file named cell.rep:

```
report_power -cell buf*6 -outfile cell.rep
```

- The following command reports the amount of power consumed by all I/O and sequential cells and writes the output information to a power report file named cell_type.rep:

```
report_power -cell_type {io sequential} -outfile cell_type.rep
```

- The following command sorts the instance-based power consumption data by leakage power and writes the output information to a power report file named sort.rep:

```
report_power -sort leakage -outfile sort.rep
```

- The following command reports the total power of all instances whose total power dissipation is less than 1 milliwatt and writes the output information to a power report file named threshold.rep:

```
report_power -threshold 1 -outfile threshold.rep
```

- The following command reports the net switching power, load capacitance, toggle rate, and switching values for each net in the design, reports the top 100 nets with the highest net switching power, and writes the output information to a power report file named net.rep:

```
report_power -net -nworst 100 -outfile net.rep
```

- The following command reports the amount of power consumed by each power domain and writes the output information to a power report file named pd_all.rep:

```
report_power -power_domain all -outfile pd_all.rep
```

- The following command reports the amount of power consumed by each power domain PD2 and writes the output information to a power report file named pd.rep:

```
report_power -power_domain PD2 -outfile pd.rep
```

- The following command reports the amount of power consumed by view view_max-hp-max-lp1 and writes the output information to a power report file named view.rep:

```
report_power -view view_max-hp-max-lp1 -outfile view.rep
```

- The following command reports the amount of power consumed by each instance connected to power nets vdd and vdd1 and writes the output information to a power report file named pg_net.rep:
  ```
  report_power -pg_net {vdd vddl} -outfile pg_net.rep
  ```
- The following command reports the leakage for the threshold voltage group HVT:
  ```
  report_power -leakage -threshold_voltage_group { HVT }
  ```
- The following command writes out a detailed vector profiling report for the combinational and macro cell types in the `vectorprofile.rpt` file:
  ```
  report_power -time_based_report -outfile vectorprofile.rpt -cell_type {combinational macro}
  ```
- The following command is an example of generating multiple reports at the same time:
  ```
  report_power -output static_power -report_prefix design -net {name1} -instance {inst_name}
  ```
  The following reports will be generated:
  ```
  static_power/design.rpt
  static_power/design.netpwr.rpt
  static_power/design.instpwr.rpt
  ```
- The following command specifies to report power for combinational cells on the clock network.
  ```
  report_power -cell_type clock_combinational -output static_power -report_prefix power_clk
  ```
- The following command generates a power map file in the simple format for the specified tile:
  ```
  report_power -outfile powermeter.txt -thermal_power_map_file powermap.txt -thermal_power_map_tile { 4 6 } -thermal_power_map_format simple
  ```
- The following command specifies to report the power content for the `inst1` hierarchical instance in the `report.hierinstpwr.rpt` report:
  ```
  report_power -output static -report_prefix report -clock_network {all} -instances * -no_wrap -hierarchy {all} -hierarchical_instances inst1 -cell_type {all}
  ```
- Example of the output of the power report.
  ```
  *------------------------------------------------------------------------------
  *        -  - Version  64-bit
  *       Date & Time:    2008-Jan-27 15:49:51 (2008-Jan-27 23:49:51 GMT)
  *------------------------------------------------------------------------------

  *       Design: dtmf_chip
  *
  *       Liberty Libraries used:
  *               /sev//DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_rvt_ss_1p08v_125c.lib
  *               /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_rvt_ss_0p8v_1p08v_125c.lib
  *               /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_rvt_ss_0p8v_125c.lib
  *               /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_lvt_ss_1p08v_125c.lib
  *               /sevDTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_lvt_ss_0p8v_1p08v_125c.lib
  *               /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_lvt_ss_0p8v_125c.lib
  *               /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_hvt_ss_1p08v_125c.lib
  *               /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_hvt_ss_0p8v_1p08v_125c.lib
  ```

```
*               /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_hvt_ss_0p8v_125c.lib
*    /sev/DTMF_CHIP/TIMING/max/scmetro_cmos10lp_rvt_ss_1p08v_125c.lib
*    /sev/DTMF_CHIP/TIMING/max/scmetro_cmos10lp_rvt_ss_0p8v_125c.lib
*    /sev/DTMF_CHIP/TIMING/max/scmetro_cmos10lp_lvt_ss_1p08v_125c.lib
*    /sev/DTMF_CHIP/TIMING/max/scmetro_cmos10lp_lvt_ss_0p8v_125c.lib
*    /sev/DTMF_CHIP/TIMING/max/scmetro_cmos10lp_hvt_ss_1p08v_125c.lib
*     /sev/DTMF_CHIP/TIMING/max/scmetro_cmos10lp_hvt_ss_0p8v_125c.lib
*     /sev/TIMING/max/rom_512x16A_ss_1v08_125c_syn.lib
*     /sev/DTMF_CHIP/TIMING/max/pllclk_slow.lib
*               /sev/DTMF_CHIP/TIMING/max/iometroil_cmos10lp_hvt_ss_1p08v_2p3v_2p5v_125c.lib
*               /sev/DTMF_CHIP/TIMING/max/iometroil_cmos10lp_hvt_ss_0p8v_2p3v_2p5v_125c.lib
*               /sev/DTMF_CHIP/TIMING/min/scmetropmk_cmos10lp_rvt_ff_1p32v_m40c.lib
*               /sev/DTMF_CHIP/TIMING/min/scmetropmk_cmos10lp_rvt_ff_1p1v_1p32v_m40c.lib
*               /sev/DTMF_CHIP/TIMING/min/scmetropmk_cmos10lp_lvt_ff_1p32v_m40c.lib
*               /sev/DTMF_CHIP/TIMING/min/scmetropmk_cmos10lp_lvt_ff_1p1v_1p32v_m40c.lib
*               /sev/DTMF_CHIP/TIMING/min/scmetropmk_cmos10lp_hvt_ff_1p32v_m40c.lib
*               /sev/DTMF_CHIP/TIMING/min/scmetropmk_cmos10lp_hvt_ff_1p1v_1p32v_m40c.lib
*

/sev/DTMF_CHIP/TIMING/min/scmetro_cmos10lp_rvt_ff_1p32v_m40c.lib
*
/sev/DTMF_CHIP/TIMING/min/scmetro_cmos10lp_lvt_ff_1p32v_m40c.lib
*
/sev/DTMF_CHIP/TIMING/min/scmetro_cmos10lp_hvt_ff_1p32v_m40c.lib
*
/sev/DTMF_CHIP/TIMING/min/rom_512x16A_ff_1v32_m40c_syn.lib
*
/sev/DTMF_CHIP/TIMING/min/pllclk_fast.lib
*               /sev/DTMF_CHIP/TIMING/min/iometroil_cmos10lp_hvt_ff_1p32v_2p7v_2p5v_m40c.lib
*               /sev/DTMF_CHIP/TIMING/min/iometroil_cmos10lp_hvt_ff_1p1v_2p7v_2p5v_m40c.lib
*
*       Power Domain used:
*               Rail:     VDD_AO        Voltage:       1.32
*               Rail: VDD_TDSPCore      Voltage:       1.32
*               Rail: VDD_TDSPCore_R    Voltage:       1.32
*
*       Parasitic Files used:
* /sev/DTMF_CHIP/SPEF/dtmf_chip.decoup.spef
*
*       DEF Files used:
* /sev/DTMF_CHIP/DEF/dtmf_chip.def
```

```
*
*       Power Units = 1mW
*       Time Units = 1e-09 secs
*       report_power
--------------------------------------------------------------------------------
  Total Power
--------------------------------------------------------------------------------
Total Internal Power:      0.01943(62.6774%)
Total Switching Power:     0.01057(34.0968%)
Total Leakage Power:       0.001(3.2258%)
Total Power:               0.031
--------------------------------------------------------------------------------
Group      Internal      Switching     Leakage       Total  Percentage
Power      Power         Power         Power  (%)
--------------------------------------------------------------------------------
Sequential    0.0006738   0.0002215   4.045e-05   0.0009358     3.019
Macro         0.0004916   1.515e-05   0.0002609   0.0007677     2.477
IO            0.01743     0.009085    0.0003172   0.02683      86.56
Combinational 0.0008324   0.001247    0.0003815   0.002461      7.941
--------------------------------------------------------------------------------
Total 0.01943     0.01057       0.001       0.031         100
--------------------------------------------------------------------------------
Rail                   Voltage   Internal   Switching   Leakage Total  Percentage
                      Power     Power      Power        Power  (%)
--------------------------------------------------------------------------------
VDD_AO                  1.32  0.0008238   0.0003404   0.0002694    0.001434       4.625
VDD_TDSPCore           1.32   0.001126    0.001022   0.0003931    0.002541       8.197
VDD_TDSPCore_R         1.32  5.833e-06   3.253e-07   8.371e-07   6.996e-06     0.02257


Clock                    Internal   Switching     Leakage       Total  Percentage
                            Power      Power         Power         Power  (%)
--------------------------------------------------------------------------------

m_digit_clk               0           0           0           0           0
m_ram_clk                 0           0           0           0           0
m_dsram_clk               0           0           0           0           0
m_spi_clk           1.018e-06    1.09e-06   2.572e-08   2.134e-06    0.006884
m_rcc_clk           1.282e-05   1.513e-05   2.672e-07   2.821e-05     0.09103
m_clk               5.039e-05   7.278e-05   7.658e-07   0.0001239      0.3998
refclk              1.11e-05   3.017e-05   2.488e-07   4.152e-05       0.134
--------------------------------------------------------------------------------
```

```
Total               7.533e-05   0.0001192   1.307e-06   0.0001958      0.6317
--------------------------------------------------------------------------------

*       Power Distribution Summary:
*  Highest Average Power:     IOPADS_INST/Prefclkip (PBCSCT2B):     0.003727
*  Highest Leakage Power: DTMF_INST/ROM_512x16_0_INST (rom_512x16A): 8.899e-05
*               Total Cap:      2.46221e-10 F
*               Total instances in design:  8525
*               Total instances in design with no power:    24
*                 Total instances in design with no activty:    24
*               Total Fillers and Decap:     8
--------------------------------------------------------------------------------
```

- The following command creates a leakage report (see example report below):

```
report_power -leakage
*--------------------------------------------------------------------------------
*    Tempus Timing Signoff Solution 08.10-b050_1 (64bit) 08/21/2008 19:23 (Linux 2.6)
*
*
*       Date & Time:    2008-Aug-24 21:34:17 (2008-Aug-25 04:34:17 GMT)
*
*--------------------------------------------------------------------------------
*
*       Design: dma_mac
*
*       Liberty Libraries used:
*               DATA/timing_libs//tcbn65lp_LVL_c060217wc0d90d9.lib
*               DATA/timing_libs//tcbn65lpcgwc.lib
*               DATA/timing_libs//tcbn65lplvtwc.lib
*               DATA/timing_libs//tcbn65lpwc.lib
*               DATA/timing_libs//tsdn65lpa1024x32m8f_100a_wc.lib
*               DATA/timing_libs//tsdn65lpa128x16m8f_100a_wc.lib
*               DATA/timing_libs//tcbn65lpcgwc0d72.lib
*               DATA/timing_libs//tcbn65lplvtwc0d72.lib
*               DATA/timing_libs//tcbn65lpwc0d72.lib
*               DATA/timing_libs//tsdn65lpa1024x32m8f_0p84v_wc.lib
*               DATA/timing_libs//tsdn65lpa128x16m8f_0p84v_wc.lib
*
*       Power Domain used:
*               Rail:      VDDlu        Voltage:       0.84
*               Rail:       VDDm        Voltage:       1.08
*               Rail:        VDD        Voltage:       1.08
```

```
*
*       Power Units = 1mW
*
*       Time Units = 1e-09 secs
*
*       report_power -outfile Leakage.rep -leakage
*
--------------------------------------------------------------------------------
Total Power
--------------------------------------------------------------------------------
Total Leakage Power:       0.1879
--------------------------------------------------------------------------------
Group                            Leakage       Percentage
                                 Power         (%)
--------------------------------------------------------------------------------
Sequential                         0.0127       6.756
Macro                                0.1556        82.8
IO                                      0           0
Combinational                      0.01962       10.44
--------------------------------------------------------------------------------
Total                              0.1879         100
--------------------------------------------------------------------------------
Rail                  Voltage   Leakage       Percentage
                                Power         (%)
--------------------------------------------------------------------------------
VDDau                       0          0           0
VDDlu                    0.84    0.06477       34.47
VDDm                     1.08   0.003587       1.909
VDD                      1.08     0.1195       63.62

Clock                            Leakage       Percentage
                                 Power         (%)
--------------------------------------------------------------------------------
phy_rxclk_2                             0           0
phy_txclk_2                     0.0004786      0.2547
phy_rxclk_1                             0           0
phy_txclk_1                      3.39e-05     0.01804
clk                              0.001723      0.9171
--------------------------------------------------------------------------------
Total                            0.002236        1.19
--------------------------------------------------------------------------------
```

```
--------------------------------------------------------------------------------

*       Power Distribution Summary:
*               Highest Average Power: ethernet_mac_2/tx_fifo_module_from_dma/fifo1/dual_port_
   ram_4_tx_fifo/ram2P1024x32/ram (TSDN65LPA1024X32M8F):      0.02424
*               Highest Leakage Power: ethernet_mac_2/tx_fifo_module_from_dma/fifo1/dual_port_
   ram_4_tx_fifo/ram2P1024x32/ram (TSDN65LPA1024X32M8F):      0.02424
*               Total Cap:      1.01625e-10 F
*               Total instances in design: 29235
*               Total instances in design with no power:  3596
*          Total instances in design with no activty:   115
*               Total Fillers and Decap:     0
--------------------------------------------------------------------------------

Total leakage power = 0.187903 mW
  Cell usage statistics:
  Library tsdn65lpa128x16m8f_1v08, 1 cells ( 0.003900%) , 0.01016 mW (    5.407063%  )
  Library tsdn65lpa1024x32m8f_1v08, 6 cells ( 0.023402%), 0.145427 mW (    77.394568%  )
  Library tcbn65lp_LVL_c060217wc0d90d9 , 120 cells ( 0.468037%), 0.000175605 mW    ( 0.093455%  )

  Library unknown , 3881 cells ( 15.137096%) , 0.00630072 mW ( 3.353178%  )
  Library tcbn65lpcgwc , 118 cells ( 0.460236%) , 0.000625092 mW ( 0.332667%  )
  Library tcbn65lplvtwc , 3762 cells ( 14.672959%) , 0.0110529 mW ( 5.882233%  )
  Library tcbn65lpwc , 17751 cells ( 69.234370%) , 0.0141619 mW ( 7.536835%  )
```

Related Information

- set_power_analysis_mode
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

reset_power_activity

```
reset_power_activity
```

Resets all activity in the database. You can specify this command to reset all activity and power information in a given session.

> You can reset activity from an activity file using `read_activity_file -reset`, reset activity for nets, pins, and ports using `set_switching_activity -reset`, and reset the default switching activities using `set_default_switching_activity -reset`.

Parameters

None

Related Information

- `read_activity_file`
- `set_default_switching_activity`
- `set_switching_activity`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

restore_power_database

```
restore_power_database
[-help]
-file {file1 file2 file3 ...}
[-hierarchy {prefix1 prefix2 prefix3 ...}]
[-hierarchy_separator "/"]
```

Restores any `power.db` file in the Voltus/Innovus session. This command can be used after restoration of a design to view the static power results or incremental report generation without having to do power analysis again.

Note: You will be able to restore the power database that you created previously, only if you created the power database using the `set_power_analysis_mode -create_binary_db true` command. Also, the command loads power databases created in the 9.1 release only, and not in the older releases.

- The command can map a hierarchical instance to a flat netlist.
- It can restore multiple power dbs for blocks and top level, that are generated separately.
- When the information for a net or an instance is present in multiple power dbs, the last one will overwrite.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `restore_power_database` parameter. For a detailed description of the command and all of its parameters, use the man command `man restore_power_database`.
`-file {file1 file2 file3 ...}`	
	Specifies the name of the power database file.
`-hierarchy {prefix1 prefix2 prefix3 ...}`	
	Specifies to map each instance in the file with a prefixN/instance (for example, prefixA/inst) in the flat netlist, where '/' is the hierarchy separator. This is an optional parameter. *Default*: `all`
`-hierarchy_separator "/"`	

	Specifies the separator character in the hierarchical net names, bus names, and pin names for the block or other part of the design that you want to analyze. It must be the same as the separator character in the design netlist. This is an optional parameter. *Default*: slash (/)

Use Model

```
restoreDesign
restore_power_database -file power.db
report_power # for text based power reports
```

Example

For each instance in file 'file2' a prefix "A" is added to instance name such as instC, A/instC. No prefix is added to 'file1' since prefix1 is none for file1.

```
restore_power_database -file {file1 file2} -hierarchy {none A}
```

Related Information

- `report_power`
- `set_power_analysis_mode`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_default_switching_activity

```
set_default_switching_activity
[-black_box_density value]
[-black_box_duty value]
[-duty value]
[-global_activity factor]
[-hier hierarchy_name]
[-input_activity factor]
[-period value]
[-seq_activity factor]
[-reset]
[-reset_type { global_activity | seq_activity ... }]
[-clock_gates_enable {activity_factor}]
[-icg_ratio num]
[-comb_clockgate_ratio num]
[-clock_gates_output {activity_factor}]
[-clock_gates_output_ratio num]
[-clip_activity_to_domain_freq {true | false}]
[-name activity_name]
[-macro_activity factor]
```

Specifies the switching activity for all primary inputs, nets, and other devices in the design whose activity has not been previously defined through user attributes, the toggle count format (TCF) file, the value change dump (VCD) file, or the tracing of the clock network.

You can use this command to specify user-defined activities at the design level.

Parameters

`-black_box_density value`	
	Specifies to set the default transition density on all the output pins of all blackboxes in the design. The `-black_box_density` and `-black_box_duty` parameters must be used together. If one or both of these parameters are not specified, then the software will use the average transition density from all the input pins of the blackboxes.
`-black_box_duty value`	

	Specifies to set the default duty on all the output pins of all blackboxes in the design. The `-black_box_density` and `-black_box_duty` parameters must be used together. If one or both of these parameters are not specified, then the software will use the average transition density from all the input pins of the blackboxes.
`-clip_activity_to_domain_freq {true | false}`	
	Specifies to perform domain-based activity clipping. This parameter clips data paths to 1x frequency whereas the clock networks are still at 2x frequency. This parameter allows you to have more control to determine the rate of clipping with respect to frequency.
`-clock_gates_enable {`*`activity_factor`*`}`	
	Specifies the activity at the enable pin(s) of the clock gating cells.
`-clock_gates_output {`*`activity_factor`*`}`	
	Specifies the activity at the output pin(s) of the clock gating cells. This works for both ICGs as well as combinational clock gating cells. **Example**: `set_default_switching_activity -global_activity 0.02 -clock_gates_output 2.0` (clock is transparent) transition_density of clock gate output is calculated by: `(1/clock_domain_frequency) * clockgateOutputsActivity = 1/(0.431*e-9) * activity_factor` In this case, transition_density = `clock_domain_frequency * clockgateOutputsActivity` = 1e9/0.431 * 2 = 4,640,371,200
`-clock_gates_output_ratio` *`num`*	

	Specifies to control transition density at the output pin(s) of the integrated and combinational clock gating cells. This parameter is equivalent to specifying the parameters `-icg_ratio` and `-comb_clockgate_ratio` together with the same value.
	
	For example, the `set_default_switching_activity -clock_gates_output_ratio 0.6` is equivalent to `set_default_switching_activity -icg_ratio 0.6 -comb_clockgate_ratio 0.6`.
	
	Note: This parameter has a cascading effect when there are multiple levels of ICG cells on the clock path. For example, when `-clock_gates_output_ratio` is set to `0.5`, the first level ICG cell will have 0.5 ratio of the root clock's toggle density, the second level ICG cell will have 0.5*0.5=`0.25` ratio of the root clock's toggle density, and the third level ICG cell will have 0.5*0.5*0.5=`0.125` ratio of the root clock's toggle density, and so on.
	
	Note: For dynamic power analysis, fastest clock on the path is used for calculating output toggle density. However, for static power analysis, weighted averaging of input toggle densities is used.
	
	Example for static power analysis:
	`set_default_switching_activity -global_activity 0.02 -clock_gates_output_ratio 1.0`
	
	Transition density for either a combinational or an integrated clock gating cell with 'N' inputs is calculated by:
	`output_pin_transition_density = (summation) weighted_input_transition_density = clock_gates_output_ratio * [(duty_input_pin1 * Td_input_pin1) + (duty_input_pin2 * Td_input_pin2) + ... + (duty_input_pinN * Td_input_pinN)]`
	
	Given a 2:1 clock Mux and assuming duty of 0.5 for all input pins:
	Td is the toggle density for each input pin of the instance `(Td_pinSe1 , Td_pinI0 , Td_pinI1).`
	
	`output_pin_transition_density = 1.0 * [0.5 * Td_pinSe1 + 0.5 * Td_pinI0 + 0.5 * Td_pinI1]`

`-comb_clockgate_ratio` *num*	

	Specifies to set the propagation ratio for combinational clock gate cell outputs. When specified, this parameter sets the output activity of any instance that is identified as a combinational clock gating cell to *num* times the fastest transition density arriving on the instance's clock inputs. This parameter cannot be specified with the `-hier` parameter. **Notes:** • This parameter does not apply to combinational clockgates with a constant data input, that is, a select pin tied low. • For dynamic power analysis, fastest clock on the path is used for calculating output toggle density. However, for static power analysis, weighted averaging of input toggle densities is used. For a detailed static power analysis example, refer to '`-clock_gates_output_ratio`', limiting calculations only to combinational clock gates.
`-duty` *value*	Specifies the duty cycle, which is the probability that the signal is a logical 1. **Note:** You must specify the `-period` parameter when assigning a duty. *Default*: `0.5` (equivalent to a 50 percent duty cycle)
`-global_activity` *factor*	
	Specifies the average number of times that all unset nodes switch in a clock cycle. If not specified the fastest domain frequency will be used unless one turns off clock domains. **Note**: Due to activity precedence, the `-global_activity` and `-seq_activity` parameters of the `set_default_switching_activity` command are mutually exclusive. If both the parameters are specified, `-global_activity` takes precedence.
`-hier` *hierarchy_name*	
	A hierarchical name that the global activity is being assigned to. If not provided, the whole design is assumed.
`-icg_ratio` *num*	
	Specifies to set the propagation ratio for integrated clock gate (ICG) cell outputs. When specified, this parameter sets the output pin of all identified ICG instances to *num* times the fastest transition density arriving on clock inputs of the instance. This parameter cannot be specified with the `-hier` parameter.
`-input_activity` *factor*	
	Specifies the average number of times that a primary input switches in a clock cycle. It can be any positive number. *Default*: `0.2` (Switches once every five clock cycles.)

`-macro_activity` *factor*	
	Specifies the activity factor for macros in a design. The allowed values for this parameter are between `0` to `1`. If you set the macro activity to `.30`, 30% of macros in the design would be selected for switching in a given cycle during dynamic simulation. When `-macro_activity` is specified for the static power analysis, all output pins of the macros are assigned the specified activity. *Default*: `0.2`
`-name` *activity_name*	

	Allows you to set the activity level for the specified jitter activity name. You can define a low/high activity level for jitter analysis.
	Examples:
	`set_default_switching_activity -name low -clock_gates_output 0 -global_activity 0.01`
	`set_default_switching_activity -name high -clock_gates_output 2 -global_activity 0.2`

`-period` *value*	Specifies the default operating period (1/frequency). It is the period that the activity is referenced to. If not specified, the software uses the *dominant* clock in the design. The specified period will only be applied to nets that do not have reference clock (ref_clock) associated with the net. If no clock is specified, the software generates an error and prompts you to enter one. Units in seconds (`s`), milliseconds (`ms`), microseconds (`us`), nanoseconds (`ns`), or picoseconds (`ps`). **Note:** If you enter the period as a negative number, the software reverts back to using the default frequency. *Default*: The time unit is defined in the first `.lib` file read during design import.
`-seq_activity` *factor*	
	Specifies the activity of outputs of the sequential logic. This value will be used at the output ports of flops for propagating activity. If the sequential activity is not specified, propagation will be done for the full design starting from the input ports.
`-reset`	Resets all specified options back to default values.
`-reset_type { global_activity | seq_activity ... }`	
	Specifies to selectively reset the global specifications. **Note:** You must use the `-reset_type` parameter in conjunction with the `-reset` parameter.

Activity Precedence

See "Activity Precedence" in `read_activity_file`.

Examples

- The following command specifies that the primary inputs in the design will switch once every ten clock cycles, with a clock period of 7520 ns and a duty cycle of 50 percent:
  ```
  set_default_switching_activity -input_activity 0.10 -period 7520ns -duty .5
  ```
- The following command specifies that all unswitched nodes in the design will switch once every ten clock cycles, with a clock period of 7520 ns and a duty cycle of 50 percent:
  ```
  set_default_switching_activity -global_activity 0.10 -period 7520ns -duty .5
  ```
- The next example defines the switching activity, sequential activity, and the period:
  ```
  set_default_switching_activity -input_activity 0.2 -period 10 \
      -seq_activity 0.1
  ```
- The next example sets the default global activity to 0.5 for the hierarchy dma_dut:
  ```
  set_default_switching_activity -global_activity 0.5 -hier dma_dut
  ```
- The following command resets the sequential and global activity specification:
  ```
  set_default_switching_activity -reset -reset_type
  {seq_activity,global_activity}
  ```

Related Information

- `read_activity_file`
- `set_switching_activity`
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_dynamic_power_simulation

```
set_dynamic_power_simulation
[-help]
[-period value]
[-resolution value]
[-reset]
[-activity_pattern {time1 activity_name1 time2 activity_name2 .... }]
```

Specifies the period and resolution for a dynamic power simulation. This command is used to specify the simulation period in the dynamic vectorbased/vectorless and power-up flows.

Generally, the number of simulation steps (simulation period divided by resolution) should be selected such that the number is around 1000. Around 20 clock cycles of the dominant clock specified as period and 20/50ps specified as resolution will capture all switching events in the design. This can be fine-tuned to balance the run time and accuracy. For example, if the dominant clock frequency is 1GHz, and the clock period is 1ns, the recommended simulation period is 20ns and resolution is 20ps, as shown in the following example:

```
set_dynamic_power_simulation -period 20ns -resolution 20ps
```

In this case, the number of simulation steps is 1000.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_dynamic_power_simulation` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_dynamic_power_simulation`.
`-activity_pattern {time1 activity_name1 time2 activity_name2 .... }`	

	Divides the total simulation time based on the windows given in the `activity_pattern` parameter. For a particular window, it uses the defined activity specified with the `set_default_switching_activity` command. When the user-defined low cycle and high cycle time is specified for the input/sequential activity, all the clocks will be in the low or high cycle until completely past the cycle transition time. Following is the use model of this parameter: • `set_default_switching_activity` **`-name low -input_activity 0.05 -seq_activity 0.1`** `-clock_gates_output 0.2` • `set_default_switching_activity` **`-name high -input_activity 0.1 -seq_activity 0.2`** `-clock_gates_output 2.0` • `set_dynamic_power_simulation -period 15ns -resolution 50ps` **`-activity_pattern {0 high 5ns low 10ns high}`** Here, high activity is used for 0ns to 5ns, low activity is used for 5ns to 10ns, and high activity is used for 10ns to 15ns.
`-period` *value*	Specifies the simulation period that dynamic power analysis will use. The supported time units are `s`, `ms`, `us`, `ns`, and `ps`. If no explicit time unit is specified, the default `ns` (nanosecond) will be applied.
`-reset`	Resets all specified options back to default values.
`-resolution` *value*	
	Specifies the transient time step size or resolution. The supported time units are `s`, `ms`, `us`, `ns`, and `ps`. If no explicit time unit is specified, the default `ps` (picosecond) will be applied.

Examples

- The following command sets up the period and resolution for a dynamic simulation:
  `set_dynamic_power_simulation -period 5ns -resolution 100ps`

Related Information

- `set_power_analysis_mode`
- `set_default_switching_activity`
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_inst_temperature_file

```
set_inst_temperature_file
file
[-reset]
```

Specifies the instance temperature file to support thermal-aware leakage. You can use this command to do thermal-aware power calculation. Based on the temperature specified in the file, the software will calculate temperature and leakage power.

In the presence of a trilib set, the internal and leakage power will be interpolated as the per the given temperature specification.

Parameters

file	Specifies the name of the instance temperature file..
-reset	Resets the temperature file.

Related Information

- report_power
- set_power_analysis_mode
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_metal_density_options

```
set_metal_density_options
[-metal_density_include_file filename]
-power_grid_library pgv_path
[-reuse_md_db_path MD_db_path]
-tech_file tech_file_path
-thermal_conductivity_file thermal_file
[-user_defined_metal_density_file user_defined_md_file}]
[-vtm_densitytable_tile {Xint Yint} ]
[-vtm_densitytable_tile_size {Xsize Ysize}]
[-vtm_header_include_file user_defined_header_content_file]
-vtm_path output_path
```

Sets the parameters for extracting the metal density information. This command must be specified before the `extract_metal_density` command.

Parameters

`-metal_density_include_file` *filename*
Specifies the file for listing the additional variables that can be included when extracting metal density information. For example, you can specify "`setvar report_with_lef_layer_names true`" to output the LEF layer names.
`-power_grid_library` *pgv_path*
Specifies the path to the power cell libraries (`.cl`).
`-reuse_md_db_path` *MD_db_path*
Specifies to reuse the generated metal density database for `extract_metal_density`. When specified, the PGV/tech file specified with `set_metal_density_options` will be ignored.
`-tech_file` *tech_file_path*
Specifies the path to the extraction technology file to be used for top-level power-grid extraction.
`-thermal_conductivity_file` *thermal_file*

Specifies a file containing the thermal conductivity value for layers. You can use this parameter to include the thermal conductivity value of the following layers in the power map file:

- Silicon
- Metal
- Dielectric
- Micro Bump (optional for 3DIC design)
- Fill material in micro bump layer (optional for 3DIC design)

The thermal conductivity file format is:
Section 1 (layer type based thermal conductivity value)

```
#default
$layer_type1 $thermal_conductivity_parameter1
$layer_type2 $thermal_conductivity_parameter2
...
```

Section 2 (layer name based thermal conductivity value)

```
#perlayer
$layer1_name $thermal_conductivity_parameter1 $dielectric_thermal_conductivity_1
$layer2_name $thermal_conductivity_parameter2 $dielectric_thermal_conductivity_2
...
```

Here,

- You can either specify both Section 1 (layer type based) and Section 2 (layer name based), or specify one of the two sections. If both sections are specified, then layer name based section takes precedence over layer type based section.
- The layer name based section has thermal conductivity value and its dielectric layer's thermal conductivity value for each metal/via layer.
- If section 2 has no parameters defined, the software will use the default parameters defined in the section 1.

Example:
###default

```
Metal      430
Silicon    250
Dielectric 0.6
```

###Perlayer

```
Metal1    430    0.4
Via1      410    0.5
Metal2    390    0.5
Via2      410    0.5
...
```

`-user_defined_metal_density_file` *user_defined_md_file*}

Allows you to define the metal density in the user-defined metal density flow.

The format of the user-defined metal density file is:

```
CELL <cell_name>
CELL_AREA [ xmin ymin xmax ymax | x1 y1 x2 y2 x3 y3 x4 y4 ... x1 y1]     # Must specify if cell PGV
is not imported
LAYER <layer_name(s)>                          # Layer to be defined metal density; multiple layers
are available (LEF layer name based)
ALL <metal_density_value>                      # To define metal density for whole cell area as
'ALL' only 'ALL' can be used now
END_LAYER
END_CELL
```

Example:

```
CELL Macro1
CELL_AREA 0 0 32 16
LAYER Metal1 Metal2 Metal3
ALL 0.2
END_LAYER
LAYER Via1 Via2
ALL 0.05
END_LAYER
END_CELL

CELL Macro2
CELL_AREA 0 0 32 16
LAYER Metal1 Metal2 Metal3
ALL 0.2
END_LAYER
LAYER Via1 Via2
ALL 0.05
END_LAYER
END_CELL
```

`-vtm_densitytable_tile {Xint Yint}`

Specifies the number of tiles in the X and Y directions for the metal density table.

`-vtm_densitytable_tile_size {Xint Yint}`

Specifies the size of tiles instead of tile number in the X and Y directions for the metal density table.

`-vtm_header_include_file user_defined_header_content_file`

Allows you to define extra header information like file location.

`-vtm_path output_path`

Specifies the directory path to the metal density file output folder.

Examples

- The following command specifies to set the metal density parameters:

```
set_metal_density_options \
-power_grid_library "$DesignDir/PGV/tech_pgv/techonly.cl" \
-tech_file $DesignDir/QRC/qrcTechFile \
-vtm_path ./vtm_MD_db \
-thermal_conductivity_file $DesignDir/MISC/thermal_conduct.txt \
-vtm_densitytable_tile {10 10} \
-vtm_header_include_file $DesignDir/MISC/user_header.txt
```

Related Topics

- extract_metal_density

set_power

```
set_power
[value]
[-cap value]
[-fanout_limit value]
[-reset]
[-leakage value]
[-pg_pin {pg_pin}]
[-pg_net railname]
[-pwl]
[-sticky]
[-dynamic_async_mode { mode_name }]
[-dynamic_switch_pattern pattern]
[-dynamic_switch_event { switching_events }]
[-scale_factor value]
[-include_ddv_modes {pattern_list}]
[-internal]
[-switching value]
[-clock {all|clock_name}]
[-exclude_clock]
[-exclude_ddv_modes {pattern_list}]
[-cell cell_name value]
[-instance inst_list value]
[-pin pin_name pin_power]
[-force]
[-custom_macro_pwl filename]
[-repeat]
[-ascii_power_file filename]
[-trigger_time_adjustment time]
[-no_propagation]
[-static_mode { custom_label_name | mode_name }]
[-slew value]
[-signal_net netname]
[-default_dynamic_pgv_mode { mode_name }]
```

Specifies the amount of power for all or part of the design. This command is used to define user-specified power on cells and instances. You must specify this command before `report_power`.

The `set_power` command can accept a cell/instance list using the `get_cells`/`get_lib_cells` commands. You can use the `get*` commands to create a collection of cells/instances whose name matches the supplied pattern list. Therefore, providing you with the flexibility in providing lists through use of regular filter patterns. This feature is supported in both static and dynamic analysis engines.

Parameters

```
-ascii_power_file filename
```

	Specifies the name of an instance list file. This file contains the list of instances in the following format: *instance_name power_value <pin_name>* Here, • *instance_name* is the instance name to which power is to be distributed. • *power_value* is the power value for the instance in watts. • *pin_name* is an optional argument to specify the pin name. When specified, allows you to assign power to the specific pin of the instance. • The three fields should be in one line, and each instance should be in a new line.
`-cap` *value*	
	Specifies the capacitance value to all instances with nets more than or equal to the fanout limit (`-fanout_limit`).
`-cell` *cell_name value*	
	Specifies the cell name. If the cell or instance name is not specified, the power will be distributed to all rails. If you want to specify power for a group of cells/instances, use the `get_cells`/`get_lib_cells` commands. The `-cell` parameter can accept these commands to create a collection of cells/instances whose name matches the supplied pattern list. For example, `-cell[get_lib_cells NAND*] 2mw`
`-clock {all|`*clock_name*`}`	
	Specifies the power value for clock networks. You can specify the power value for a specific clock network or the clock network power value for the whole design. **Note:** This parameter cannot be used with the `-cell`, `-instance` and `-pg_net` parameters.
`-custom_macro_pwl` *filename*	
	Specifies to use a custom trigger file to apply instance or cell PWL waveforms. Given one master cell PWL waveform, the software can automatically generate scaled PWL waveforms of the other companion cells defined in the trigger file proportional to the cell internal power. **Note**: This parameter must be used with `set_power -dynamic_switch_pattern` in the vectorless flow. For more information on the trigger file format, see Trigger File Format.
`-default_dynamic_pgv_mode {` *mode_name* `}`	

	Specifies the current characterization mode to be used during dynamic power and jitter analyses. This parameter can be specified when using a PGV with multiple functional modes. When this parameter is specified, the Power Calculation engine uses this preferred mode in scheduling the switching events for the macro blocks, and saves these events in the trigger file. The use model of this parameter is: `set_power -cell {} | -instance {} -default_dynamic_pgv_mode { mode_name }` This parameter is mutually exclusive to the `-dynamic_switch_pattern` parameter. If a cell or instance has both the `-dynamic_switch_pattern` and `-default_dynamic_pgv_mode` parameters defined in the same Tcl script, `-dynamic_switch_pattern` takes higher priority.
`-dynamic_async_mode { mode_name }`	

	Specifies to trigger asynchronous macros based on the length of the PWL waveform to create a single continuous waveform. This parameter allows you to specify the asynchronous mode for an instance or a cell. The start and end of the PWL saved in the PGV must have similar current value in order to avoid sudden jump in the current.
	Use model:
	`set_power -dynamic_async_mode {mode} [-instance {name} | -cell {name}]`
	Example:
	`set_power -cell pll -dynamic_asynchronous_mode {READ_C}`
	Here, power analysis will trigger the `READ_C` mode and automatically splice the waveform based on its length.

`-dynamic_switch_event { switching_events }`	
	Specifies the switching events for cells, instances, and pins along with their rise and fall time. `switching_events` is a string of toggle time followed by “r” or “f”, where “r” is for rise toggle and “f” is for fall toggle. The `-dynamic_switch_event` parameter should not be specified with the `-repeat` parameter.
`-dynamic_switch_pattern pattern`	

	Specifies stimulus pattern on the pins of an instance and cell. Specify this parameter only in the dynamic mode for the type `instance` and `cell`.
	
	The *`pattern`* is applied to all pins (input and output) on the instance/cell. It is a string.
	
	`pattern` specifies the switching pattern. If there is more than one clock domain with timing windows on a pin, only the windows on the fastest clock will be used, and the pattern will be applied to that clock period. It is a string of `0`, `1`, `r`, and `f` values. `0` for cycles not to switch and `1` for cycles to switch. `r` is for rise only and `f` for fall only.
	
	The pattern string should include enough digits to cover all of the clock cycles in the simulation period. If there are not enough digits, the pattern will be padded with 0 values at the end. If there are too many, the pattern will be truncated to the number of applicable cycles. Transitions will be scheduled in the center of each applicable timing window.
	
	Notes
	• `0` and `1` is supported only in the probability-based vectorless flow. It is not supported in the state-propagation-based vectorless flow and the vector-based flow.
	• In case of macros, you can specify the mode names, that is, which mode switches in which clock cycle `-dynamic_switch_pattern { mode1 - mode2 - }`.
	
	You can specify "`-`" to indicate that instances will not switch in a particular clock cycle. If "`-`" is the first value in the string of values specified within `{}`, you must insert a space before "`-`", as shown below:
	• no switching: `-dynamic_switch_pattern { - - }`
	• no switching in the 1st clock period: `-dynamic_switch_pattern { - r }`
	
	Examples:
	
	In the following example, instance will not switch in the 4th clock cycle:
	`set_power -dynamic_switch_pattern { f r f - r }`
	
	In the following example, the mode `write` will switch in the 1st clock cycle, the mode `read` will switch in the 2nd and 4th clock cycle, and none of the modes will switch in the 3rd clock cycle:
	`set_power -dynamic_switch_pattern { write read - read }`
	
	Note: Wildcards are supported only for `-dynamic_switch_pattern` with `-pin`. When `-dynamic_switch_pattern` is applied for `-instance` or `-cell` without the `-pin` parameter, wildcards are not supported.

`-exclude_clock`	Excludes clock networks when specifying a scale factor or power specification for the whole design. This parameter cannot be used with the `-cell, -instance` and `-pg_net` parameters.
`-exclude_ddv_modes {`*`pattern_list`*`}`	

	Excludes the current characterization (or Dynamic Detailed View) modes that match with the specified patterns. This parameter allows you to control the modes to be switched, instead of switching all the current characterization modes defined in the power-grid view. **Use Model** `set_power -instance|-cell` *`name`* `-exclude_ddv_modes {`*`pattern_list`*`}` Here: • *`pattern_list`* is a list of mode names, separated by space or comma • the instance, cell, and pattern names can contain wildcard characters
`-fanout_limit` *`value`*	
	Specifies the fanout limit for nets. If an instance has nets more than or equal to the fanout limit, its switching/internal/leakage power is taken to be zero. When this parameter is specified, the user-defined power value is applied to all the driver instances that have nets more than or equal to the fanout limit.
`-force`	Specifies to apply the `set_power` command even if there are Verilog mismatches. For dynamic or DEF-based static analysis, this parameter allows you to apply `set_power` to the DEF instances that do not exist in Verilog.
`-help`	Outputs the command usage.
`-include_ddv_modes {`*`pattern_list`*`}`	
	Includes only the current characterization (or DDV) modes that match with the specified patterns. This parameter allows you to control the modes to be switched, instead of switching all the current characterization modes defined in the power-grid view. **Use Model** `set_power -instance|-cell` *`name`* `-include_ddv_modes {`*`pattern_list`*`}` Here: • *`pattern_list`* is a list of mode names, separated by space or comma • the instance, cell, and pattern names can contain wildcard characters
`-internal`	Specifies the target internal power of the design, cell, or instance. To specify internal power for cells and instances, you can use the `-internal` parameter in conjunction with the `-cell` or `-instance` parameter.
`-instance` *`inst_listvalue`*	
	Specifies the instance name. If the cell or instance name is not specified, the power will be distributed to all rails.

`-leakage`	Specifies the target leakage power of the design, cell, instance, or power net. To specify leakage power for cells and instances you can use `-leakage` option in conjunction with `-cell` or `-instance` option. To specify leakage power of a certain power net in the design you can use `-pg_net` option along with `-leakage` option.
`-no_propagation`	
	Specifies that the user-defined switch pattern is applied only after state-propagation is complete. The use model of the command parameter is: `set_power -dynamic_switch_pattern {pattern} -no_propagation` The `-no_propagation` parameter must be specified with the `-dynamic_switch_pattern` parameter. When the `-no_propagation` parameter is specified, the dynamic power engine performs state-propagation first, and then overrides the single instance switching event by the user-defined dynamic switch pattern without any further propagation.
`-pg_net railname`	
	Optional. The specified power or pwl for the cell, instance or design is applied to the specified rail. When not specified, the power or pwl is applied to all the rails associated with the instance. This option is generally used for MSMV cells. When this option is not specified the behavior is different for pwl and power options. When not specified and power option is used, the power is applied for the whole design, cell, or instance. When not specified and pwl option is used, the pwl is applied to every rail of the cell or instance.
`-pg_pin {pg_pin}`	
	Specifies the switch pattern for a specific PG pin of an instance. This parameter must be specified with the `-dynamic_switch_pattern` parameter.
`-pin pin_name pin_power`	

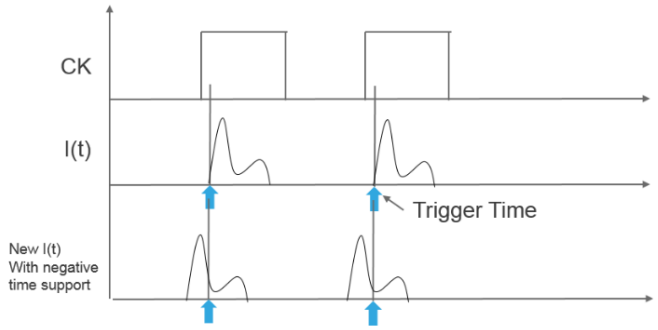
	Specifies pin-wise power numbers for a specific instance. This parameter allows you to control the power assigned to power ground pins of an instance. Following is the use model of this parameter:
	`set_power -instance <inst_name> -pin <VDDA> <pin_power>`
	`set_power -instance <inst_name> -pin <VDDB> <pin_power>`

`-pwl`	Specifies that power argument (`power | pwl`) is a waveform rather than a power number.
`-repeat`	Repeats dynamic switch pattern of all the clock cycles in the simulation period.
`-reset`	This parameter causes all previous `set_power` commands to be ignored. Note: You can selectively reset the instance/cell specifications. In addition, you can also selectively reset the cell and instance specifications of a list of objects through the `get_cells` and `get_lib_cells` commands.

`-scale_factor` *value*	

	Scales the power value by the specified scale factor. This parameter allows you to set a user-defined scale factor in the trigger file to scale the dynamic current of the PGV.
	To modify the scale factor, use the `-reset` parameter with the `-scale_factor` parameter.
	Example:
	`set_power -leakage -reset  -scale_factor 5 -instance A`
	You can use the following power scaling commands in the power-driven optimization flow when running Voltus within the Innovus platform:
	• `set_power -scale_factor <value> -leakage` => scale the leakage power value by the specified scale factor
	• `set_power -scale_factor <value> -internal` => scale the internal power value by the specified scale factor
	• `set_power -scale_factor <value> -switching` => scale the switching power value by the specified scale factor
	• `set_power -scale_factor <value>` => scale the leakage, internal, and switching power values by the specified scale factor
	When specifying these commands in the optimization flow, you must not specify any other parameter of the `set_power` command.
	Optimization Flow Example:
	`setOptMode -opt_power_effort high`
	`setOptMode -opt_leakage_to_dynamic_ratio 0.5`
	`set_power -scale_factor 15 -internal`
	`set_power -scale_factor 20 -switching`
	`report_power`

`-signal_net` *netname*	
	Specifies the signal net name for power assignment. When this parameter is specified, the driver instances connected to the signal net will be assigned the user-defined power value.
`-slew` *value*	
	Specifies the slew value to all instances with nets more than or equal to the fanout limit (`-fanout_limit`).
`-static_mode { custom_label_name | mode_name }`	

	Specifies the mode name or the custom label name associated with a cell or instance for computing instance current from PGV with static multi-voltage multi-frequency (MvMf) modes. This current is used to compute the instance's internal power, and its switching and leakage power components would be set to zero. The use model of this parameter is given below: `set_power -instance -static_mode READ` If you specify custom label of a static MvMf mode, currents would be taken from it directly. Alternatively, if only mode name is specified for a cell/instance, the software snaps to the nearest custom label defined within this mode, based on the instance's voltage and frequency to get the currents.
`-sticky`	Applies the simulation based PWL waveform as is, to the specified cell/instance for dynamic power analysis. Use this option only if you have specified `-pwl`.
`-switching` *value*	
	Specifies the target switching power of the design, cell, or instance. To specify switching power for cells and instances, you can use the `-switching` parameter in conjunction with the `-cell` or `-instance` parameter.
`-trigger_time_adjustment` *time*	
	Specifies to adjust the trigger time of the current waveforms for the various functional modes (read/write/idle) of the custom macros. This parameter allows you to start the waveform before the trigger time (negative time support), as shown below:  The specified time will be applied to the computed trigger time for each mode in the trigger files generated by power analysis. For example, if the trigger time (from timing window/vector) is computed as 1ns, 2ns, and 3ns, and if you specify `set_power -trigger_time_adjustment -100ps`, the software will adjust the trigger time to .9ns, 1.9ns, and 2.9ns.

value	Specifies power in milliwatts during static power calculation or piecewise linear (`-pwl`) current waveform, during dynamic power calculation for cell or instance. The command accepts a unit specification (e.g. 10w).
	The format of the PWL waveform is:
	`{time1 current1 time2 current2 ...time_n current_n}`
	Time is in ns and current in mA.
	Example:
	`set_power -pwl -pg_net {} -cell BUFX2MTL {0 0 10 0 10.1 5 10.5 0}`
	Here, at `0ns` its value is `0mA`, at `10ns` its value is `0mA`, at `10.1ns` its value is `5mA`, and so on.

Examples

- The following command sets power for vdd rail to 10 milliwatts:
  ```
  set_power -pg_net vdd 10mw
  ```
- The following command sets a power value of .005 milliwatts on power net vdd for instance clkin_neg_L14_I1:
  ```
  set_power -pg_net vdd -instance Top/A/clkin_neg_L14_I1 .005
  ```
- The following command sets a power value of .005 milliwatts for any cell named INVX2 in the design:
  ```
  set_power -cell INVX2 .005mw
  ```
- The following command defines a piecewise linear waveform of {0 0 10 0 10.1 5 10.5 0} for the cell of type BUFX2MTL for all power rails.
  ```
  set_power -pwl -pg_net {} -cell BUFX2MTL {0 0 10 0 10.1 5 10.5 0}
  ```
- The following command specifies the leakage power for the cell DFFRX1.
  ```
  set_power -leakage -cell DFFRX1 10mw
  ```
- The following commands specify the target leakage power for power nets VDD1 and VDD2.
  ```
  set_power -leakage -pg_net VDD1 100mw
  set_power -leakage -pg_net VDD2 200mw
  ```
- The following command specifies the PWL description of current for the instance SEHD130_1024X32X1CM8. The -sticky option tells the software to use the specified PWL simulation based waveforms as is.
  ```
  set_power -pwl -instance SEHD130_1024X32X1CM8 -sticky {0 0 1 1 2 2 7 0}
  ```
- The following command scales the total power of the instance i_1 by a scale factor of 2:
  ```
  set_power -scale_factor 2 -instance i_1
  ```
- The following command sets the total power of the design to 20mw while keeping the clock network power the same:
  ```
  set_power 20mw -exclude_clock {all}
  ```
- The following command assigns a switching power of 3mw to all instances of the cell AND1:
  ```
  set_power -cell AND1 -switching 3mw
  ```
- The following command assigns 2mw power to all cells that match the specified pattern:

```
set_power -cell[get_lib_cells NAND*] 2mw
```

- The following command specifies that mode1 switches in the first cycle, mode2 switches in the third cycle and no switching in others:
  ```
  set_power -dynamic_switch_pattern { mode1 0 mode2 0 }
  ```
- The following command resets power of all cells that match the specified pattern:
  ```
  set_power -reset -cell[get_cells A/*]
  ```
- The following command is an example of cell-based switching:
  ```
  set_power -cell HS65_LS_* -dynamic_switch_pattern {1101}
  ```
- The following command specifies that the switching pattern value `1010` will be applied to all the instance names starting with `m_inst`:
  ```
  set_power -instance m_inst* -dynamic_switch_pattern {1010}
  ```
- The following command does not schedule DDV mode `scan*`:
  ```
  set_power -cell { * } -exclude_ddv_modes { scan* }
  ```
- The following command does not schedule DDV mode `scan*` for instance "`A1`":
  ```
  set_power -instance { A1 } -exclude_ddv_modes { scan* }
  ```
- The following command only schedules DDV mode names with the `*read*` and `*write*` pattern:
  ```
  set_power -cell { * } -include_ddv_modes { *read*, *write* }
  ```
- The following commands specify the toggle time for the pin names Q7 and Q6:
  ```
  set_power -dynamic_switch_event { 1.1ns r 5.5ns f } -instance clk_b_i2 -pin Q7
  set_power -dynamic_switch_event { 2.2ns r 6.6ns f } -instance clk_b_i2 -pin Q6
  ```

Trigger File Format

For specifying a custom trigger file, you must use the `-custom_macro_pwl` parameter. The format of the trigger file is:

Trigger File Format	Description
#Cell names	
`MASTER CELL <cellname1>`	Name of the reference cell.
`COMPANION_CELLS <cellname1 cellname2 … >`	Name of the cells that will use scaled PWL from the master cell.
#Mode and conditional details for trigger	
`MODE_NAME <mode1>` `CONDITIONAL_INPUT <conditional input statement for cellname1>`	Mode name and conditional input statement for the cell. Conditional input of current for the given mode.
`CONDITIONAL_PIN <pin_name> (rise|fall|both)`	Conditional pin statement for the cell. Signal pin for which the tool should rise or fall.

CONDITIONAL_STIMULUS_FILE <usim1.txt>	Simulation vector detail in the given mode.
USER_PWL_FILE <pwl1.txt>	User-specified PWL file that contains information about the PWL waveforms (power/ground currents). For an example of a PWL file, see PWL Format Example.
END_MODE	End mode section
#End cell section	
END	End cell section

Example of a Custom Macro Power Trigger File

```
MASTER_CELL         SRAM_A1
COMPANION_CELLS     { SRAM_A2, SRAM_A3, SRAM_A4, SRAM_A5 }
      MODE_NAME WRITE
               CONDITIONAL_INPUT = ( !SD & !SLP & !CEB & !WEB )
               CONDITIONAL_PIN = CLK { RISE }
               USER_PWL_FILE write_rise.txt
       END_MODE
      MODE_NAME READ
               CONDITIONAL_INPUT = ( !SD & !SLP & !CEB & WEB )
               CONDITIONAL_PIN = CLK { RISE }
               USER_PWL_FILE read_rise.txt
       END_MODE
      ...
END
```

Example of “USER_PWL_FILE” (Multiple regions)

```
UNIT CURRENT mA
UNIT TIME ns
       NET VSS
               REGION { 0 0 0.1 -1 0.2 -2 0.3 -3 0.4 -10 0.5 -50 0.6 -40 0.7 -30 0.8 -20 1 0 2 0 3 0 3.9 0
4 -0.1 4.5 -0.2 5 -4 5.5 -3 6 -2 7 0 }
       NET VDD
               REGION { 0 0 0.1 1 0.2 2 0.3 3 0.4 10 0.5 50 0.6 40 0.7 30 0.8 20 1 0 2 0 3 0 3.9 0 4 0.1
4.5 0.2 5 4 5.5 3 6 2 7 0 }
       NET VDDM
               REGION { 0 0 0.1 1 0.2 2 0.3 3 0.4 10 0.5 50 0.6 40 0.7 30 0.8 20 1 0 2 0 3 0 3.9 0 4 0.1
4.5 0.2 5 4 5.5 3 6 2 7 0 }
```

Related Information

- set_power_analysis_mode
- report_power
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_power_analysis_mode

```
set_power_analysis_mode
[-reset]
[-add_simulation {true | false}]
[-adjust_input_activity_in_iterations {true | false}]
[-adjust_macro_activity_in_iterations {true | false}]
[-annotation_detail_report {true | false}]
[-auto_twf_delay_annotation {true | false}]
[-average_rise_fall_cap {true | false}]
[-binary_db_name filename ]
[-block_independent_peakpower {true | false} ]
[-boundary_gate_leakage_file filenname]
[-boundary_gate_leakage_report {true | false}]
[-boundary_leakage_cell_property_file filenname]
[-boundary_leakage_edge_annotation_file filenname]
[-boundary_leakage_multi_pgpin_support {true | false}]
[-boundary_leakage_pessimism_removal {true | false}]
[-boundary_leakage_pxe_support {true | false}]
[-bulk_pins {bulk_pin_list}]
[-capacity {low|medium|high}]
[-case_insensitive_mapping {true | false}]
[-clock_source_as_clock {true | false}]
[-comprehensive_automapping {true | false}]
[-constant_override {true | false}]
[-corner {min|max}]
[-create_binary_db {true | false}]
[-create_driver_db  {true | false}]
[-create_gui_db {true | false}]
[-current_generation_method { avg | peak }]
[-decap_cell_list cell_list ]
[-default_frequency value]
[-default_slew value]
[-default_supply_voltage value]
[-detailed_view_current_only {true | false} ]
[-disable_clock_gate_clipping {true | false} ]
[-disable_leakage_scaling {true | false}]
[-disable_static {true | false}]
[-distribute_switching_power {true | false}]
[-distributed_combine_report_format {none | detailed | reduced}]
[-distributed_setup file]
[-domain_based_clipping  {true | false} ]
[-dynamic_glitch_filter   value]
[-dynamic_scale_clock_by_frequency { freq_A factor_A freq_B factor_B freq_C factor_C ... }]
[-dynamic_scale_clock_by_name { clk_A factor_A clk_B factor_B clk_C factor_C ... }]
[-dynamic_vectorless_ranking_methods {load | clock | vector_activity}]
[-enable_auto_queue {true | false}]
[-enable_auto_mapping {true | false}]
[-enable_disk_mapping {true | false}]
[-enable_duty_prop_with_global {true | false}]
[-enable_dynamic_current_slew_load_interpolation {true | false}]
```

```
[-enable_dynamic_scaling {true | false}]
[-enable_flop_state_propagation {true | false}]
[-enable_generated_clock {true | false}]
[-enable_input_net_power {true | false}]
[-enable_interactive_reports {true | false}]
[-enable_mt_in_vectorbasedflow  {true | false} ]
[-enable_mt_reports {true | false}]
[-enable_mt_state_propagation {true | false} ]
[-enable_pba_for_tempus_pi {true | false}]
[-enable_power_target_flow {true | false}]
[-enable_rtl_vectorbased_dynamic_analysis {true | false}]
[-enable_scan_report {true | false}]
[-enable_single_cycle_scheduling {true | false}]
[-enable_slew_based_ccs_pin_cap {true | false}]
[-enable_state_propagation {true | false}]
[-enable_tempus_pi {true | false}]
[-enable_xp {true | false}]
[-enhanced_blackbox_avg {true | false}]
[-enhanced_blackbox_max {true | false}]
[-equivalent_annotation {true | false}]
[-event_based_leakage_power {true | false}]
[-external_load_config_file filename]
[-extraction_tech_file filename]
[-extractor_include filename]
[-fanout_limit value ]
[-flatten_xpgv_block_instances  filename]
[-force_library_merging {true | false}]
[-from_x_transition_factor value]
[-from_z_transition_factor value]
[-generate_activity_mapping_report {true | false}]
[-generate_current_for_rail railnames ]
[-generate_flop_ranking_data directoryname]
[-generate_leakage_power_map_based_on_calculated_leakage {true | false}]
[-generate_static_report_from_state_propagation {true | false}]
[-handle_glitch {true|false}]
[-handle_transport_glitch {true | false}]
[-handle_tri_state {false|true} ]
[-hier_delimiter character]
[-honor_combinational_logic_on_clock_net {true | false}]
[-honor_negative_energy {true | false}]
[-honor_net_activity {true | false}]
[-honor_non_mission_leakage {true | false} ]
[-hybrid_analysis {true | false}]
[-ignore_control_signals {true | false}]
[-ignore_data_phase_for_clk {true | false}]
[-ignore_end_toggles_in_profile {true | false}]
[-ignore_glitches_at_same_time_stamp {true | false}]
[-ignore_inout_pin_cap {true | false}]
[-ignore_macro_leakage_scale_for_temp {true | false}]
[-include_seq_clockpin_power {true | false}]
[-include_timing_in_current_file {true | false}]
[-ir_derated_timing_view view_name]
```

```
[-keep_clock_gate_ratio_in_iterations {true | false}]
[-leakage_scale_factor_for_temp scale ]
[-library_preference {voltage | ecsm_ccsp}]
[-macro_pgv_leakage {true | false}]
[-macro_toggle_pin_percentage percentage ]
[-mbff_toggle_behavior { simultaneous | independent | sbff }]
[-mbff_toggle_pin_percentage percentage ]
[-memory_current_scaling_method {1 | 2 | 3}]
[-merge_switched_net_currents {true | false}]
[-method { static | dynamic_vectorless | dynamic_vectorbased | dynamic_mixed_mode | event_based |
vector_profile}]
[-min_leaf_count value]
[-multi_scenario_simulation {true | false}]
[-off_pg_nets net _list]
[-output_current_data_prefix prefix ]
[-partition_count value]
[-partition_twf {true | false}]
[-pin_based_twf {true | false}]
[-power_grid_library { library _list } ]
[-power_include_initial_x_transitions {true | false}]
[-power_match_state_for_logic_x value ]
[-precision value]
[-pre_simulation_empty_period period]
[-pre_simulation_period period]
[-pre_simulation_power_exclude_period period]
[-quit_on_activity_coverage_threshold threshold_value ]
[-read_rcdb {true | false}]
[-relax_arc_match {true | false}]
[-report_black_boxes {true | false}]
[-report_clock_instance_switching_info {true | false}]
[-report_idle_instances {true | false}]
[-report_instance_switching_info {all | output_logic | none} ]
[-report_instance_switching_list filename]
[-report_library_usage {true | false}]
[-report_missing_bulk_connectivity {true | false}]
[-report_missing_input {true | false}]
[-report_missing_nets { true|false }]
[-report_scan_chain_stats {true | false}]
[-report_stat {true | false}]
[-report_time_display_fraction_digits value]
[-report_twf_attributes {detailed | summary}]
[-reuse_flop_ranking_data directoryname]
[-reuse_flop_ranking_data_hier blockname]
[-save_bbox {true | false}]
[-scale_to_sdc_clock_frequency {true | false}]
[-scan_chain_activity {design_name  chain_name  activity_number} | {vectorless  activity_number}]
[-scan_chain_name {design_name  chain_name  pattern}]
[-scan_control_file filename]
[-scan_mbff_chain_type type]
[-set_power_event_based {true | false}]
[-settling_buffer value]
[-smart_window {true | false}]
```

```
[-split_bus_power {true | false}]
[-start_time_alignment {true |false}]
[-state_dependent_leakage {true | false}]
[-stateprop_ignore_unannot_pins {true | false}]
[-static_multi_mode_scenario_file  filename]
[-static_netlist {verilog | def}]
[-switching_power_on_rise_only {true | false}]
[-thermal_input_file file]
[-thermal_leakage_temperature_scale_table_file filename]
[-to_x_transition_factor value]
[-to_z_transition_factor value]
[-transition_factor_based_duty {true | false}]
[-transition_time_method {min | avg | max}]
[-twf_delay_annotation {min | avg | max}]
[-twf_load_cap {min | avg | max}]
[-unified_power_switch_flow {true | false}]
[-use_cell_leakage_power_density {true | false}]
[-use_fastest_clock_for_dynamic_scheduling {true | false}]
[-use_lef_for_missing_cells {true | false}]
[-use_only_ddv_current {true | false}]
[-use_physical_partition {true | false}]
[-use_zero_delay_vector_file {true | false}]
[-vector_profile_mode {activity | event_based | power_density | transient}]
[-worst_case_vector_activity  {true | false}]
[-worst_step_size value]
[-worst_window_count value]
[-worst_window_coverage { top | tile }]
[-worst_window_reports {full | worst | both}]
[-worst_window_size value]
[-worst_window_type { power | activity | delta_power | delta_activity | ir | critical_inst_power } ]
[-write_boundary_leakage_edge_annotation {true | false}]
[-write_default_uti {true | false}]
[-write_dynamic_currents {true | false}]
[-write_profiling_db {true | false}]
[-write_simulation_db {true | false}]
[-write_static_currents {true | false}]
[-x_count_transition_using_3_states {true | false}]
[-x_transition_factor value ]
[-z_transition_factor value ]
[-zero_delay_vector_toggle_shift value]
```

Sets the parameters for doing the power analysis.

Parameters

```
-add_simulation {true | false}
```

	Default: `false` Specifies to add simulation if there are no simulation vectors or if all primary sources are not annotated by vectors. This parameter can be used to ensure complete design coverage. Event simulation is done for primary sources like primary input, flops, and macros. **Usage Guidelines**: This parameter is only applicable to the event-based power analysis flow (`set_power_analysis_mode -method event_based`).
`-adjust_input_activity_in_iterations {true | false}`	
	Default: `true` Specifies whether to keep the input activity fixed or automatically adjust the activity value for each iteration of state propagation. **Usage Guidelines**: This parameter is only applicable to the power target flow (`set_power_analysis_mode -enable_power_target_flow true`).
`-adjust_macro_activity_in_iterations {true | false}`	
	Default: `true` Specifies whether to keep the macro activity fixed or automatically adjust the activity value for each iteration of state propagation. **Usage Guidelines**: This parameter is only applicable to the power target flow (`set_power_analysis_mode -enable_power_target_flow true`).
`-annotation_detail_report {true | false}`	
	Specifies to generate a detailed annotation report per instance in the Vector Profiling flow (`set_power_analysis_mode -method vector_profile`). When specified, it creates the `AnnotationDetail.rpt` file in the user-specified output directory. *Default* : `false`
`-auto_twf_delay_annotation {true | false}`	
	Default: `false` Specifies to automatically detect the zero-delay vectors and perform delay annotation. As a result, you do not need to specify additional parameters (`set_power_analysis_mode -use_zero_delay_vector_file` and `read_activity_file -zero_delay`) to indicate the vectors that have delay annotation. **Usage Guidelines**: This parameter is only applicable to the event-based power analysis flow (`set_power_analysis_mode -method` **`event_based`**).

`-average_rise_fall_cap {true | false}`

Default: `false`

When set to `true`, the software uses the average of rise and fall capacitance from the liberty file. This parameter can be specified for all library formats.

Usage Guidelines:

This parameter is not applicable to the `-static_netlist def` flow.

`-binary_db_name` *`filename`*

Default: The default file name is `power.db`

Specifies the name of the binary power database file (`filename.db`) used for analyzing power plots. This database contains instances, locations, and power. The generated power database allows you to perform GUI interactive debugging using the *Power & Rail Plots* and *Power Debug* forms.

You can specify this parameter in scenarios where more than one database is generated and you need to track which one was generated in which mode when restoring the database, or if your design has several blocks and analysis is done on one block all the time and all the databases are named `power.db`.

Usage Guidelines:

You must specify this parameter with the `-create_binary_db true` parameter. This parameter is applicable to both the static and dynamic power calculation flows.

`-block_independent_peakpower {true | false}`

Default: `false`

Specifies to identify the worst power interval of each block, and run power analysis of the full design using the worst interval of each block. This parameter allows you to perform power and current analysis for design blocks that have independent peak power intervals.

When set to `false`, power analysis is performed using the common worst interval found using simulation of all vectors together.

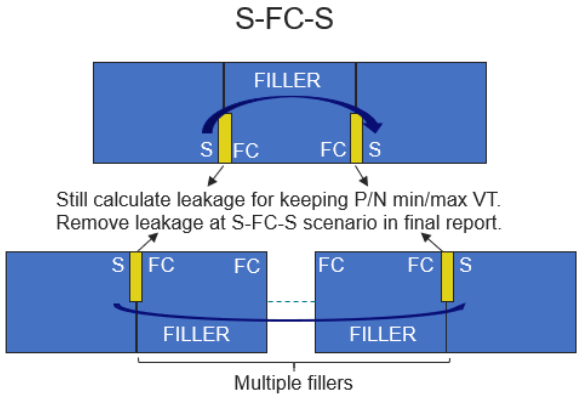
`-boundary_gate_leakage_file` *`filenname`*

Specifies a list of Liberty side files required for boundary gate leakage (context-aware leakage) calculation. This file defines the boundary current for different threshold voltage (Vt) layers at different operating conditions. The file includes:

- multiple leakage tables corresponding to different operating conditions.
- leakage probability factor for different abutments (when abutting instances are on the same Vt layer or when they are on different Vt layers).

When `-boundary_gate_leakage_file` is specified, the `power.boundarygateleakage.rpt` file is dumped by default. This file contains the design-level summary for the three combinations of Vt select fail ratio (0, 1 and the one specified with HighVT_select_fail_ratio).

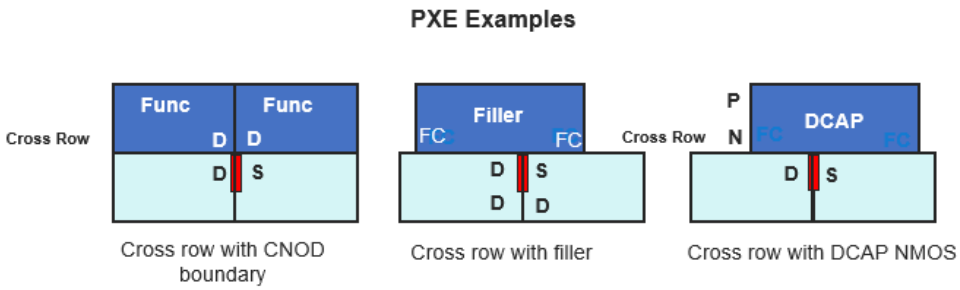
`-boundary_gate_leakage_report {true | false}`

	Default: `false` Generates a verbose report for every abutted instance-edge for boundary leakage calculation. This detailed report is useful in debugging and understanding the leakage power calculation for an instance. The generated report name is `boundarygateleakage.detailed.rpt`.
`-boundary_leakage_cell_property_file` *filenname*	
	Specifies the list of edge information files containing cell properties.
`-boundary_leakage_edge_annotation_file` *filenname*	
	Specifies the edge annotation file with updated Vt layers at a given edge location to be used during boundary leakage calculation. This parameter is used to provide custom Vt layers for an abutted edge at a given location to be used during power computation. **Note**: When running the edge annotation feature, the user-defined VT will have precedence over the one from the cell properties file.
`-boundary_leakage_multi_pgpin_support {true | false}`	
	Default: `false` Enables support for cells with multiple power/ground pins.
`-boundary_leakage_pessimism_removal {true | false}`	
	Default: `true` Enables pessimism removal for the Source to FCs to Source abutment type. This abutment type normally has 0 CNOD leakage. Therefore, the non-zero leakage computed at the starting S-FC and terminating FC-S boundaries needs to be removed for accurate estimation. This parameter is specified to remove the non-zero leakage or pessimism. Revised CNOD leakage = CNOD Leakage – Leakage (S-FC-S). 
`-boundary_leakage_pxe_support {true | false}`	

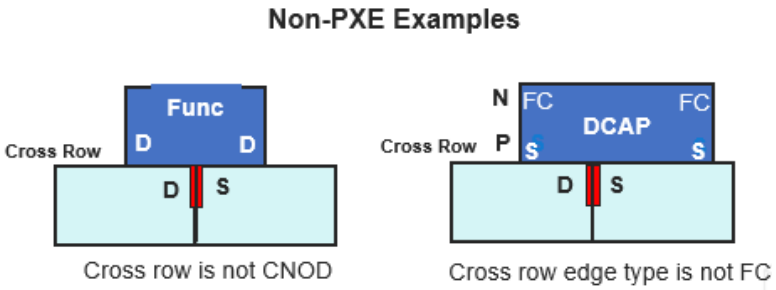
Default: `false`

Enables the PXE feature. PXE is used for calculation of CNOD leakage by cross row environment. The T structure or inverted T structure with both edges of the type FC is PXE.

The following diagram illustrates examples of PXE:



Every scenario other than the ones mentioned in PXE examples is non-PXE. The following diagram illustrates examples of non-PXE:



- If this parameter is set to `false`, Voltus does not consider the relationship by cross row environment.
- If this parameter is set to `true`, Voltus will find out the PXE or non-PXE structure and look up different leakage tables for obtaining the leakage value.

`-bulk_pins {`*`bulk_pin_list`*`}`

	Default: None Defines power and ground bulk LEF pins for the design. Use this parameter when library cells have bulk pins defined in LEF but the liberty file does not contain power associated with these pins. If a liberty file does not have body bias definition, power analysis does not distribute power to the body bias domain. **Usage Guidelines**: This parameter is only applicable to the static power calculation flow. It does not support the `-static_netlist def` and dynamic power calculation flows.
`-capacity {low|medium|high}`	
	Enables segmented power processing. When this parameter is specified, the software performs concurrent processing of vector simulation annotation, power calculation, and report construction. This parameter enables performance optimization with some trade-off on memory consumption and runtime. The arguments of the `-capacity` parameter are: • `low` - reduces memory for a flat run. • `medium` - further reduces memory with moderate increase in runtime (1.2X). • `high` - maximum memory reduction at about 2X runtime. The `-capacity` parameter is used for reducing memory when processing long simulation vectors. In case of the FSDB and SHM format activity file, the runtime can increase up to 2X.
`-case_insensitive_mapping {true | false}`	
	Default: `false` Specifies whether instance name mapping between the RTL netlist and GATE level netlist in the mapping file is case-sensitive or case-insensitive. When set to `true`, name mappings are case-insensitive.
`-clock_source_as_clock {true | false}`	
	Default: `false` Specifies to use the correct clock frequency for activity calculation. If you stop the arrival of the clock signal by setting `set_case_analysis` to 0, the transition density at the output net of the flop is calculated using the topological source clock frequency instead of the default frequency. When this option is set to `false`, then the Verilog flow will stop clock propagation for any disable clock path that is set with `set_case_analysis`. **Usage Guidelines**: This parameter is not applicable to the `-static_netlist def` flow.
`-comprehensive_automapping {true | false}`	

	Default: `false` Specifies to perform all-inclusive instance name mapping between the RTL netlist and GATE level netlist. When set to `true`, the software honors all possible naming rules (includes handling of characters like '[' , ']', '.', '/', '\\', '_', ':' and handling of strings like "MBFF", "_reg", "_MB_" etc.) for better annotation coverage. This feature is applicable to both file-based mapping (`map_activity_file -rtl2gate | -two_column_mapping_file`) and automatic mapping (`set_power_analysis_mode -enable_auto_mapping`). When `-comprehensive_automapping` is specified with either `-rtl2gate` or `-two_column_mapping_file`, the RTL mapping entries are given priority over the automatic rule-based mapping entries and are used to annotate the design objects.
`-constant_override {true | false}`	
	Default: `false` Specifies to give higher precedence to propagated constants defined using the `set_case_analysis` command. It allows propagated constants to override global activity.
`-corner {min|max}`	
	Default: `max` Defines the library corner for the non-MMMC setup (`read_lib -min | -max | -typ`). • `min` corner means the power calculator will use min timing libraries for power calculation. • `max` corner means the power calculator will use max timing libraries for power calculation. **Usage Guidelines**: This parameter is applicable to both the static and dynamic power calculation flows.
`-create_binary_db {true | false}`	
	Default: `false` A value of `true` creates a binary database of power results. This database contains instances, locations, and power. The generated power database allows you to perform GUI interactive debugging using the *Power & Rail Plots* and *Power Debug* forms. **Usage Guidelines**: This parameter is applicable to both the static and dynamic power calculation flows.
`-create_driver_db {true | false}`	

	Default: `false` Specifies to generate all the flop-based drivers/receivers list. This parameter is required to run the noise margin calculation flow, and differential voltage calculation in the rush-current analysis flow. The driver/receiver pairs are generated only for the state-elements (flops) as noise margins will be calculated only for the driver-receiver pairs where flop is a receiver. Noise margins are calculated with worst voltages seen on the driver-receiver pins. The driver/receiver list is saved in the `power2rail_config.json` file located in the power output directory. **Usage Guidelines**: This parameter is applicable only to the dynamic power calculation flow.
`-create_gui_db {true | false}`	
	Default: `false` Specifies to create a binary database of power results in the XP mode. When set to `true`, the tool creates sliced power database file for each rail partition and saves it to the `gui.db` sub-directory under the power output directory (example: `power_dir/part_*/gui.db/rail_part_*/power.x.db` (x = 1, 2, ..., 14). The generated power database allows you to perform GUI interactive debugging using the *Power & Rail Plots* and *Power Debug* forms. **Usage Guidelines**: This parameter is applicable to both the static and dynamic power calculation flows.
`-current_generation_method { avg | peak }`	

	Default: `avg` Specifies the method of instance current discretization (or continuous current waveform generation) based on the user-specified time step size (`set_dynamic_power_simulation -resolution <value in ps>`). The current waveform generated for an instance depends on the time step size/resolution for dynamic power simulation. The recommended value for `-resolution` is 20ps. The two possible arguments of this parameter are: • `peak` - Specifies to use the peak current value of each time step. The output current filename is *filename*`.ptipeak`. It is recommended to use the `peak` method when using the Composite Current Source Power (CCSP) Liberty files. This is because the CCSP library provides real current waveforms from simulation that is used directly by the software. • `avg` - Specifies to use the average current value of each time step. The output current filename is *filename*`.ptiavg`. It is recommended to use the `avg` method when using the non-CCSP (NLPM) library. In this case, the Voltus power engine heuristically constructs a triangular waveform from the look-up table. **Usage Guidelines**: • This parameter is applicable only to the dynamic power calculation flow. • It is used only for instance current waveform generation, and does not apply to total current waveform generation. When generating total current (composite) waveform, the peak current values of all instances are summed up for each step. • For very small step size, the average and peak methods will converge.
`-default_frequency` *value*	
	Default: `100MHz` Specifies to set the default frequency in the MHz unit for net not annotated by TWF. You can specify a numerical value without unit.
`-default_slew` *value*	
	Default: For static analysis CTE flow, the default slew = 0. For TWF flow, the default slew = 0.1 * default frequency in MHz. Specifies to set the default slew in the ns unit for net not annotated by TWF. You can specify a numerical value without unit. **Usage Guidelines**: This option depends on `-default_frequency` for the TWF flow. If -default_frequency is not specified, then it will take frequency of the shortest clock period.
`-default_supply_voltage` *value*	
	Default: `1.0` Specifies to set a default voltage for power nets in a scenario where the power engine cannot determine the voltage of these nets.

`-detailed_view_current_only {true | false}`

Default: `true`

Controls instance power reporting for cells that have power information available in PGV. When set to `true`, specifies to skip writing of the switching and leakage current and use only the internal current from DDV PGV. When set to `false`, specifies to report the switching and leakage current values in addition to the internal current from DDV PGV. If a cell is defined in both PGV and dotlib, leakage is computed from dotlib. It is recommended to set this parameter to `true` because that would prevent over-estimation of the instance current.

`-disable_clock_gate_clipping {true | false}`

Default: `true`

Specifies to enable clock gate output clipping so that the output transition density of an ICG cell does not exceed the input clock pin transition density.

`-disable_leakage_scaling {true | false}`

Default: `false`

Specifies to exclude leakage power during scaling factor computation when target power is specified using the `set_power` command. When `set_power_analysis_mode -disable_leakage_scaling` is set to `true`, the leakage power does not get scaled, and only the internal and switching components are scaled to meet the user-specified total power.

This parameter is relevant only for the `set_power` commands for which total power is specified, and has no effect for the `set_power` command specified with the `-internal`, `-switching`, or `-leakage` parameters.

`-disable_static {true | false}`

Default: `false`

When set to true in the dynamic vector-based or dynamic vectorless flows, the command performs only dynamic analysis and turns-off static power calculation.

Note: When you perform static power calculation in the dynamic power analysis flow, you can use either the `-disable_static` or `-write_static_currents` parameter to enable static power calculation and write static current file. If only `-disable_static` is set to `false` and `-write_static_currents` is not set, the tool automatically sets `-write_static_currents` to `true`. Similarly, if `-write_static_currents` is set to `true`, and `-disable_static` is not set, the tool automatically sets `-disable_static` to `false`.

`-distribute_switching_power {true | false}`

Default: `false`

Specifies to equally distribute the switching power among all drivers if a net has more than one driver. When set to `false`, stores all switching power on one of the net drivers.

`-distributed_combine_report_format {none | detailed | reduced}`

	Specifies to generate consolidated XP power reports. This parameter allows you to combine power reports that are stored across multiple partition directories to give a single unified view of power data. The possible arguments of this parameter are:
	• `none` - generates distributed reports.
	• `reduced` - generates consolidated reports for the following:
	  ◦ power.missingdata.* (static and dynamic flows)
	  ◦ power.stateprop.* (dynamic flow)
	  ◦ power.stats.setpower (dynamic flow)
	• `detailed` - generates consolidated reports for the following:
	  ◦ power.rpt (static and dynamic flows)
	  ◦ power.instpwr.rpt (static flow)
	  ◦ power.lib.rpt (static flow)
	  ◦ power.pgpinpwr.rpt (static flow)
	  ◦ power.pgnet.detailed.rpt
	  ◦ power.missingdata.* (static and dynamic flows)
	  ◦ power.pgnet.detailed.rpt (static and dynamic flows)
	  ◦ power.stateprop.* (dynamic flow)
	  ◦ power.stats.setpower (dynamic flow)
	  ◦ pg_trigger.txt (dynamic flow)
	  ◦ trigger.txt (dynamic flow)
	  ◦ *.totalcurrent (dynamic flow)
	  ◦ voltus_power.cellstat (static and dynamic flows)
	  ◦ voltus_power.ptifiles (static and dynamic flows)

`-distributed_setup` *file*	

	Default: `""` Specifies a file containing the customized setup details for running power analysis in the distributed mode. The specified file has 4 columns: `INST/CELL <name> block <power.inc>` where, • `INST/CELL`: Keyword indicating cell or instance to be run in the distributed mode. Cell can be useful for repeated DEF blocks. • *`name`*: Name of the hierarchical block or instance. • `block`: Keyword `block` implies that only the lower level DEF will be included • *`power.inc`*: name of the power include file. This column is optional. The unspecified instance/cell will be run with the top level as a single group. Each distributed block/instance should have corresponding TWF and SPEF to improve runtime. **Note**: Each block will be run independently from other blocks, thus the external load or cell seen at the pin of the block can be different from flat analysis. If the partition boundary analysis is important, distributed processing is not recommended. **Usage Guidelines**: • This parameter is not applicable to the `-static_netlist verilog` flow.
`-decap_cell_list` *`cell_list`*	
	Default: `""` Specifies the physical only cells, such as decap cells, to include in the power report. **Usage Guidelines**: • This parameter is applicable only to the static power calculation flow. • This parameter is not applicable to the `-static_netlist def` flow.
`-domain_based_clipping {true | false}`	
	Determines the frequency to be used to clip propagated activity that is too high. When set to `false`, the frequency of the fastest clock in the design is used. If you set this parameter to `true`, the fastest clock domain on the net is used to clip propagated activity. *Default*: `false`
`-dynamic_glitch_filter` *`value`*	

	Sets the absolute threshold time (in ns) for filtering glitches in net toggles. When you specify a time value for this parameter, glitches in net toggles that are equal to or smaller in width than the specified time value are removed. *Default*: -1 **Usage Guidelines**: • This parameter is applicable only to the state-propagation-based vectorless flow (`set_power_analysis_mode -method dynamic_vectorless -enable_state_propagation true`).
`-dynamic_scale_clock_by_frequency { freq_A factor_A freq_B factor_B freq_C factor_C ... }`	
	Specifies the scale factors for the clock frequency values. This parameter allows you to perform frequency scaling for clocks running at relatively low frequencies, thereby ensuring higher toggle coverage for limited simulation period. The parameter is applicable only to the dynamic vectorless flow. The following units are supported for frequency: Hz, KHz, MHz, and GHz.
`-dynamic_scale_clock_by_name { clk_A factor_A clk_B factor_B clk_C factor_C ... }`	
	Specifies the scale factors for the low frequency clocks. This parameter allows you to perform frequency scaling for clocks running at relatively low frequencies, thereby ensuring higher toggle coverage for limited simulation period. The parameter is applicable only to the dynamic vectorless flow. The following units are supported for frequency: Hz, KHz, MHz, and GHz.
`-dynamic_vectorless_ranking_methods {load | clock | vector_activity}`	
	Default: "" Specifies to change the ranking method from the default to the following: • `load` - Prioritizes scheduling cells with higher fanout • `clock` – Prioritizes scheduling cells associated with faster clocks • `vector_activity` - Prioritizes scheduling cells with higher activity **Usage Guidelines**: • This parameter is applicable only to the state-propagation-based vectorless flow (`set_power_analysis_mode -method dynamic_vectorless -enable_state_propagation true`). • This parameter is not applicable to the `-static_netlist verilog` flow.
`-enable_auto_queue {true | false}`	
	Default: `false` Specifies to automatically determine the queue for power jobs. This parameter is used in the Voltus-XP distributed power flow with hierarchical DEFs. In the XP flow, partitions are divided into multiple queues depending on the number of CPUs (For example, Default queue: 1 CPU, Small Queue: 2 CPUs, Medium; 4 CPUs, Large : 8 CPUs, and so on.). By default, all power partitions run on the same queue. When the parameter is set to `true`, the blocks can run on different queues depending on the size (number of instances) of the blocks. The `-enable_auto_queue` parameter will be beneficial when there are uneven power block partitions. For example, if there are some blocks with 20-30M instances and there are some with 3-5M instances. Using this parameter, bigger blocks will be automatically assigned to a bigger queue, resulting in better runtime.

`-enable_auto_mapping {true | false}`	
	Default: `false` Specifies to automatically perform instance name mapping between the RTL netlist and GATE level netlist. This parameter allows you to map the RTL vector to the Gate level netlist without a mapping file. You can use this parameter with the `read_activity_file -name_mapping_rule <filename>` command parameter to specify a file containing customized or design-specific rules. **Usage Guidelines**: This parameter is applicable only to the vector-based static/dynamic power calculation flow (`set_power_analysis_mode -method static | dynamic_vectorbased | dynamic_mixed_mode`).
`-enable_disk_mapping {true | false}`	
	Default: `false` Specifies to enable disk caching of the toggles. Therefore, the toggles are stored in the disk instead of keeping them in memory. This has made large vectors or long simulation duration runs possible even with lesser available memory. **Usage Guidelines**: This parameter is only applicable to the event-based power analysis flow (`set_power_analysis_mode -method event_based`) in the XP mode.
`-enable_duty_prop_with_global {true | false}`	
	Default: `false` Specifies to propagate duty cycle with the global switching activity setting for all data nets in the dynamic vectorless analysis flow. **Usage Guidelines**: This parameter is applicable to the static power calculation and probability-based dynamic vectorless flows. It is not supported in the state-propagation-based vectorless flow (`set_power_analysis_mode -enable_state_propagation true`).
`-enable_dynamic_current_slew_load_interpolation {true | false}`	
	Default: `false` Specifies to use the charge-based slew load interpolation. When this parameter is set to `true`, Power Analysis takes dynamic currents from the 4 nearest slew/load values and performs either a 4-way (lower slew, lower load, upper slew, upper load) or a 2-way (lower slew, upper slew OR lower load, upper load) interpolation to calculate the dynamic current waveform. The parameter allows you to derive a more realistic dynamic current waveform based on the exact slew load values from the .lib dynamic current table.
`-enable_dynamic_scaling {true | false}`	

	Default: `false` Enables `set_power` scaling for dynamic power analysis. The power and scaling specifications are used to scale dynamic current on both the power and ground currents of the specified instance/cell. This parameter honors the user-defined average (static) power for leaf-level instances and hierarchies (instances/cells/modules) along with power net specification in the dynamic current scaling flows such that the average of the dynamic current is equal to the assigned average power. When you specify this parameter, the average of the dynamic current of these leaf-level instances will match the scaled static power numbers. **Usage Guidelines**: • This parameter is applicable to the probability-based vectorless flow. It is not supported in the state-propagation-based vectorless flow (`set_power_analysis_mode -enable_state_propagation true`). • This parameter is not applicable to the `-static_netlist verilog` flow.
`-enable_flop_state_propagation {true | false}`	
	Default: `false` Enables accurate clock state propagation for sequential cells. When this parameter is set to `true`, flops propagate in a manner that toggles on the output pins are in sync with the clock. This parameter helps to align the output pins' toggles with the clock toggles.
`-enable_generated_clock {true | false}`	
	Default: `true` Specifies to get generated clock frequency during activity propagation for static power analysis. **Usage Guidelines**: • This parameter is applicable only to the static power calculation flow. • This parameter is not applicable to the `-static_netlist def` flow.
`-enable_input_net_power {true | false}`	
	Default: `false` Specifies to calculate the switching power of input nets. **Usage Guidelines**: This parameter is applicable only to the static power calculation flow.
`-enable_interactive_reports {true | false}`	

	Default: `false` Specifies to generate multiple dynamic reports based on the power analysis run done for a netlist. When this parameter is set to `true`, you can generate various types of power reports interactively using the `report_power` command without the need to perform power calculation on the same netlist again. The `-enable_interactive_reports` parameter is useful when you need to generate reports for the unaltered netlist within the same power analysis session. **Usage Guidelines**: This parameter is applicable only to the event-based power analysis flow with area-based partitioning (`-use_physical_partition`) in the XP mode.
`-enable_mt_in_vectorbasedflow {true | false}`	
	Default: `false` Specifies to enable multi-threading for the VCD/FSDB based dynamic vector-based flows. To enable multi-threading in the vector-based flows, set this parameter to `true`. **Usage Guidelines**: • This parameter is applicable only to the dynamic vector-based flow. • This parameter is not applicable to the `-static_netlist verilog` flow.
`-enable_mt_reports {true | false}`	
	Default: `false` Specifies to enable multi-threading for generation of power calculation reports. When set to `true`, it improves the memory and runtime for power reporting in large designs with more than 100 million instances.
`-enable_mt_state_propagation {true | false}`	
	Default: `true` Specifies to enable multi-threading in state propagation to help improve the runtime for large simulations. For example, when the parameter was specified for a design with 300 million instances and simulation duration of 120 microseconds, the runtime has improved from 21 hours to 1.5 hours. **Usage Guidelines**: This parameter is only applicable to the event-based power analysis flow (`set_power_analysis_mode -method event_based`).
`-enable_pba_for_tempus_pi {true | false}`	

	Default: `false` Specifies to enable the Path-Based Analysis (PBA) for slack calculation in the Tempus PI flow. By default, the Graph-Based Analysis (GBA) is used. **Usage Guidelines**: This parameter is only applicable to the state-propagation-based vectorless flow (`set_power_analysis_mode -method dynamic_vectorless -enable_state_propagation true`).
`-enable_power_target_flow {true | false}`	
	Default: `false` Specifies to enable the power target flow. When set to `true`, the tool allows you to specify power targets for full-chip power analysis. You can use the `set_power` command to specify the power targets for full-chip, physical block, logical block, and power/ground net. When these targets are specified, Voltus will automatically scale the toggle rates of clock gate ratio, input, macro, and sequential activities to meet the specified power targets. **Usage Guidelines**: This parameter is only applicable to the state-propagation-based vectorless flow (`set_power_analysis_mode -method dynamic_vectorless -enable_state_propagation true`) in the XP mode.
`-enable_rtl_vectorbased_dynamic_analysis {true | false}`	
	Specifies to take an RTL or partial VCD/FSDB file as input and use that for dynamic vector-based flow. This parameter allows you to estimate current even for instances which are missing from the activity file. You must use this parameter in conjunction with the `-method dynamic_vectorbased` parameter. Default is `false`.
`-enable_scan_report {true | false}`	
	Default: `false` Specifies to generate all the scan mode analysis reports. The following reports will be generated: • `voltus_power.scanff.duplicated.rpt` - flip-flops that are specified in multiple chains • `voltus_power.scanff.missing.rpt` - flip-flops that are missing in the scan control file but exist in the design • `voltus_power.scanff.nonassigned.rpt` - flip-flops that do not exist in the design and are dropped from the scan control file • `voltus_power.scanff.nonscan`.rpt - assigned flip-flops that have no scan-out pin • `voltus_power.scanff.nonswitched.rpt` - non-switching flip-flops in the scan control file • `voltus_power.scanff.rpt` - detailed scan chain report containing global statistics for all scan chains and statistics for each scan chain in the design • `voltus_power.scanff.stateconflict.rpt` - flip-flops that have conflicting state assignment

`-enable_single_cycle_scheduling {true | false}`	
	Default: `false` Schedules macro instances in every dominant clock cycle in the dynamic vectorless flow. This parameter can be used achieve faster toggle coverage. After the scheduling of every instance and attaining the maximum toggle coverage, the scheduler cycles back to the same switch source element and then schedules it again with the opposite polarity. This parameter helps in achieving the same toggle coverage within half the simulation period compared to dual cycle scheduling. When the parameter is set to `false`, the macros are scheduled every two cycles wherein the adjacent cycle is used for reversing the transition polarity.
`-enable_slew_based_ccs_pin_cap {true | false}`	
	Default: `false` Specifies to use the Composite Current Source (CCS) capacitance value based on the slew given in the input pin section of the library cell in the Liberty file. If it is set to `true`, the tool honors the CCS pin capacitance; if it is set to `false`, it takes `max_capacitance` from .lib. This parameter can be used when timing windows and/or a Tempus timing session is not available.
`-enable_state_propagation {true | false}`	
	Specifies to enable the vectorless state-propagation-based dynamic analysis flow. Default is `false`. For more information, refer to the **State-Propagation-Based Vectorless Methodology** section in the "*Dynamic Power and IRDrop Analysis*" chapter of the *Voltus User Guide*.
`-enable_tempus_pi {true | false}`	
	Default: `false` Specifies to enable the Tempus Power Integrity (Tempus PI) analysis flow. When this parameter is set to `true`, data path scheduling is timing-aware, that is, scheduling is based on timing slacks. For more information, refer to the *Tempus Power Integrity Analysis* chapter in *Voltus User Guide*. **Usage Guidelines**: This parameter is only applicable to the state-propagation-based vectorless flow (`set_power_analysis_mode -method dynamic_vectorless -enable_state_propagation true`).
`-enable_xp {true | false}`	
	Default: `false` Specifies to run standalone report generation in the XP mode.
`-enhanced_blackbox_avg {true | false}`	
	Specifies to use the average toggle rate of related inputs for enhanced blackbox propagation. *Default* : `false`
`-enhanced_blackbox_max {true | false}`	

	Specifies to use the maximum toggle rate of related inputs for enhanced blackbox propagation. *Default* : `false`
`-equivalent_annotation {true | false}`	
	Default: `false` Specifies to enable equivalent annotation for Q or Qbar pins when only one of them is annotated.
`-event_based_leakage_power {true | false}`	
	Default: `false` Specifies to calculate state-dependent leakage power. When this parameter is set to `true`, it calculates leakage power and current based on the state of an instance at each time point. For example, at time 0ns, instance instA is in state !I ZN (0 on input, 1 on output), and at time 71ns, the instance is in state I !ZN (1 on input, 0 on output). The leakage power is calculated based on the different states of instA at different time points.
`-external_load_config_file` *`filename`*	

	Specifies to honor the user-specified load capacitance values for power calculation. This parameter allows you to specify an external load capacitance file when you have a timing window file without the `EXTERNAL_LOAD` constructs.
	File Format:
	The format of the user-specified input file is:
	`INST|CELL` *`blockName external_load_flle`*
	Here, the format of *`external_load_flle`* is:
	`NET|PORT` *`name load_value`*
	Examples:
	Configuration file:
	`INST TOP top_wcard.load`
	External load file (`top_wcard.load`):
	`UNIT pF`
	`NET result 0.02`
	`NET overflow 0.02`

`-extraction_tech_file` *`filename`*	
	Specifies the extraction technology file to be used for top-level power-grid extraction.
`-extractor_include` *`filename`*	
	Specifies the extractor (ZX) commands and variables in the form of an include file. This parameter is applicable only for the standalone report generation flow in the XP mode (`set_power_analysis_mode -enable_xp true`).
`-fanout_limit` *`value`*	

	Specifies the fanout limit for nets. If an instance has nets more than or equal to the fanout limit, its switching/internal/leakage power is taken to be zero.
`-flatten_xpgv_block_instances` *filename*	

	Specifies a file that includes a list of block instances that use DEF instead of xPGV for power analysis, that is, specify the instances that are not to be used as xPGV. This parameter allows you to specify a few instances of the same block that will use DEF, and the others that will use xPGV at the top-level. If you do not specify this parameter, and if both DEF and xPGV are given for a block, xPGV takes precedence over DEF.
	File Format:
	<*block_name*> <*block_instance_name*|All|None>
	In this file, you can specify if a block is using a DEF by specifying the block instance name, or with the `All` or `None` keywords. `All` indicates that all instances are using DEF, and `None` indicates that none of the instances are using DEF.
	Example:
	For a design with three blocks, if you want to use the DEF and xPGV as following:
	`Block_1 - > A1 (xPGV) & A2 (DEF)`
	`Block_2 - > B1 & B2 (xPGV for both instances)`
	`Block_3 - > C1 & C2 (DEF for both instances)`
	The specified file will contain the following information:
	`Block_1 A2`
	`Block_2 None`
	`Block_3 All`

`-force_library_merging {true | false}`	
	Specifies to force merge PGVs even if there are some minor conflicts in the merging libraries. The default value of this parameter is `false`. This parameter is applicable only for the standalone report generation flow in the XP mode (`set_power_analysis_mode -enable_xp true`). *Default* : `false`
`-from_x_transition_factor` *value*	
	Specifies how transitions from X to 0/1 are counted. The default value is `0.5`.
`-from_z_transition_factor` *value*	
	Specifies how transitions from Z to 0/1 are counted. The default value is `0.25`.
`-generate_activity_mapping_report {true | false}`	

	Default: `false` Specifies to generate the `activity_mapping.rpt` report in the vector-based dynamic power flow. This report gives the list of signal name mappings between the Golden (RTL netlist - input name in the vector file) and Revised (Gate netlist - annotated design object name) netlists that were used in the flow. The report displays the list of annotated and unannotated objects and helps to debug the mapping file for unannotated design objects. `activity_mapping.rpt` supports activity mapping done through the following methods: • User-specified mapping file (`map_activity_file -two_column_mapping_file`) • Conformal-generated RTL mapping file (`map_activity_file -rtl2gate`) • Voltus auto-generated mapping file (`set_power_analysis_mode -enable_auto_mapping`). **Usage Guidelines**: This parameter is only applicable to the vector-based dynamic flow (`set_power_analysis_mode -method dynamic_vectorbased`).
`-generate_current_for_rail` *railnames*	
	Generates current files for the specified rail. By default, the command generates current files for all power rails.
`-generate_flop_ranking_data` *directoryname*	
	Specifies the name of the directory where the flop ranking data is saved. Using this parameter, you can save the flop ranking data that can be reused across different versions of the design database for the same block or transferred from the top-level to the block-level run. This parameter is used with the `-reuse_flop_ranking_data` parameter.
`-generate_leakage_power_map_based_on_calculated_leakage {true | false}`	
	Default: `false` Specifies to scale the instance leakage power by directly multiplying the user-specified scaling factor with the final calculated leakage power value, instead of scaling the leakage power value from the input dotlib. If this parameter is not specified, the user-specified scaling factor will be applied on the leakage power values from the input dotlib.
`-generate_static_report_from_state_propagation {true | false}`	

	Default: `false` Specifies to directly generate the static power analysis result based on the RTL-VCD input and state-propagation. When this parameter is specified, the RTL vector power analysis flow does not require the generation of a TCF/FSDB file. **Usage Guidelines**: • This parameter is applicable only to the dynamic propagation-based flow (vector-based/vectorless/RTL/Mixed-mode). • This parameter is not applicable to the `-static_netlist verilog` flow.
`-handle_glitch {true | false}`	
	Default: `false` A transition is defined as a glitch when the time difference between two sequential toggles are less than half of the rise transition time + fall transition time in the vector-based static power calculation. Both static and dynamic current are derated for glitch transitions. The same derate factor is used for both power and current. If set to `true`, glitch power will be identified and calculated according to this definition and it will be reported in a separate column. Switching or internal power calculation does not include power due to glitches. If set to `false`, glitch power will not be identified, and the respective power will be calculated as normal signal switches. Hence, the total power will be a little higher.
`-handle_transport_glitch {true | false}`	
	Default: `false` Transport glitch is referred to as functional toggles for a net in a single clock period before it reaches a stable value. You can use this parameter to report the internal and switching power values for transport glitch. The transport glitch power is reported in the following reports: • Average Power Summary Report (`power.rpt.avgpwr`) - reported in the Total, Group, Clock, and Rail power sections. • Instance Power Report (`power.rpt.instpwr`) - reported for all instances. • Hierarchy Power Report (`power.rpt.hierpwr`) - reported for all hierarchies. This parameter is automatically set to `true` with `-handle_glitch true`. That is, reports both inertial and transport glitch power. When set to `false`, only the inertial glitch power is reported.
`-handle_tri_state {false|true}`	
	When set to `true`, all tristate device enable pin values will be taken into account when determining the propagation of the activity through tristate gates. *Default* : `false`
`-hier_delimiter` *character*	

	Specifies the hierarchical delimiter in the DEF file. The default hierarchical delimiter is a forward slash (/) but can be changed by setting the `-hier_delimiter` parameter. You can use this parameter to override the existing hierarchical delimiter.
`-honor_combinational_logic_on_clock_net {true | false}`	
	Specifies the behavior of combination logic on clock net. • `true` (default) - specifies to honor activity propagation through combinational logic. • `false` - specifies that the combination logic will be treated like a combinational clockgate.
`-honor_negative_energy {true | false}`	
	A value of `true` specifies that Voltus will keep negative internal energy numbers from the `.lib` internal power table. When set to false, negative power is treated as 0. *Default* : `true`
`-honor_net_activity {true | false}`	
	Specifies to use activity from the `Net` section of the Switching Activity Interchange Format (SAIF) file for power calculation and optimization, and to ignore the pin-based state-dependent path delay (SDPD) activity. The default value of this parameter is `true`. When this parameter is set to `false`, the power calculation engine can use both the net-based and pin-based activity. In this case, precedence will be given to activity from net or pin that appears later in the SAIF file.
`-honor_non_mission_leakage {true | false}`	

	Supports modeling of well-to sub-leakage information when the circuit power supply is off for the level shifter cells. This is referred to as the non-mission mode leakage power calculation. The leakage power for a level shifter cell depends on the state of each PG pin. The Liberty file contains sections for PG pin conditions of each mode/state and non-mission mode leakage power for each PG pin of that mode.
	When this parameter is set to `true`, Voltus identifies the pg_pin mode and assigns non-mission mode leakage power associated with the mode.
	The use model of the non-mission mode leakage power calculation feature is:
	`set_power_analysis_mode -honor_non_mission_leakage true`
	`set_power_analysis_mode -off_pg_nets {VDD VDDO}`
	Example:
	Let us consider a level shifter cell with the PG mode "`retained_VDDO_off`" and the following `pg_pin_condition`:
	`pg_setting_value (retained_VDDO_off) {`
	`pg_pin_condition : "!VBB*VDD*VDDI ";`
	`pg_pin_active_state (VBB, V1);`
	`pg_pin_active_state (VDD, V2);`
	`pg_pin_active_state (VDDI, V2);`
	When you specify a list of PG nets to be turned off along with the `-honor_non_mission_leakage` parameter, all cells/instances that have the PG state "`retained_VDDO_off`" associated with the specified PG pin condition, will use the corresponding non-mission leakage power.

`-hybrid_analysis {true | false}`	
	Default: `false` Specifies to perform both vector-based and vectorless power analysis in a single session.
`-ignore_control_signals {true | false}`	
	Specifies that control signals will be ignored when propagating activity. It is recommended to set this parameter to `true`. When set to `true`, the control pins (reset, preset) will not impact the sequential instance's output pin activity during activity propagation. *Default* : `true`
`-ignore_data_phase_for_clk {true | false}`	
	Default: `false` Specifies to ignore data frequency in the clock path for the clock instances in the design. When this parameter is set to `true`, the software propagates only the fastest clock pin frequency instead of the data pin frequency for activity propagation through clock network. In the default mode `(false)`, the software picks the fastest frequency between the data phase and clock phase on that pin.
`-ignore_glitches_at_same_time_stamp {true | false}`	
	Specifies to ignore glitches at the same transient time in the vector-based static and dynamic power analysis flow. When `-ignore_glitch_at_same_time_stamp` is set to `true`, only the last signal value change is honored, and all the preceding value changes at the same time stamp are ignored. The last signal value will overwrite the previous value for the same signal at the same time stamp, so the last value will represent the state of the signal at that time stamp. *Default* : `true`
`-ignore_inout_pin_cap {true | false}`	
	Default: `false` When set to `true`, ignores the bidirectional pin capacitances (`direction : inout`) defined for I/O cells in the .lib file, when calculating switching power and internal power; this also includes the default capacitance value for `inout` pins.
`-ignore_macro_leakage_scale_for_temp {true | false}`	
	Default: `false` The `-leakage_scale_factor_for_temp` parameter is used to perform linear scaling of the leakage power for temperature in both the standard cell and macro libraries. The `-ignore_macro_leakage_scale_for_temp` parameter allows you to selectively apply the scale factor only for standard cells, and not use the scale factor for macro cells.
`-ignore_end_toggles_in_profile {true | false}`	

	By default, the software profiles the toggles in the user-selected vector time window by steps including the toggles at the start/end boundary of the window. In each step, the toggles on the start boundary edge of the step is counted in the current step; the toggles on the end boundary edge is counted in the next step. When this parameter is set to `false`, the toggles on the end boundary edge is also counted in the last step, as there is no next step to it. This may lead to double count of the power profile for the last step, especially for zero-delay vector profiling. The default of this parameter is `false`. When this parameter is set to `true`, the software ignores the toggle on the end boundary of the last step.
`-include_seq_clockpin_power {true | false}`	
	Reports the clock pin power of flip-flops as part of the clock network power. *Default* : `false`
`-include_timing_in_current_file {true | false}`	
	Default: `false` Specifies to write timing bits for non-switching instances in the current files during the state-propagation-based flow. When specified, rail analysis will report effective instance voltage (EIV) for both switching and non-switching instances. If the parameter is set to `false`, rail analysis will report EIV for the switching instances only. **Usage Guidelines**: This parameter is only applicable to the state-propagation-based vectorless flow (`set_power_analysis_mode -method dynamic_vectorless -enable_state_propagation true`).
`-ir_derated_timing_view` *view_name*	
	Specifies the name of the timing view used to calculate timing slack and to rank the flops in the Tempus PI flow. This parameter must be specified if there are multiple views. The `-ir_derated_timing_view` parameter defines the timing view used to detect the IR-sensitive paths using a lower voltage than the actual analysis. This lower voltage simulates the IR-drop for each instance in the design. The software uses this view to prioritize the end-points that need to be prioritized for switching.
`-keep_clock_gate_ratio_in_iterations {true | false}`	
	Default: `false` Specifies whether to keep the clock gate output (CGO) fixed or automatically adjust the value for each iteration of state propagation. **Usage Guidelines**: This parameter is only applicable to the power target flow (`set_power_analysis_mode -enable_power_target_flow true`).
`-leakage_scale_factor_for_temp` *scale*	

	Performs a linear scaling of the leakage power for temperature in all libraries. This replaces the k-factor for temperature in the .lib file. For process and voltage, the scaling will continue to be based on the corresponding k-factors in the .lib file. A value of 0.8 scales the leakage power in the library to 80 percent. A value of 1.2 scales the leakage power in the library to 120 percent. *Default* : `1`
`-library_preference {voltage | ecsm_ccsp}`	
	Instances of the same cell can be attached to different power domains in a design. If you have specified different PVT libraries, then the software will by default bind each instance to the closest library definition. You can use this parameter to select either the closest Effective Current Source Model (ECSM)/Composite Current Source-power (CCSP), or the closest voltage library for the instance: • `voltage`: when specified, enables library binding to the closest voltage library. • `ecsm_ccsp`: when specified, enables library binding to the closest ECSM/CCSP library. The default is `voltage`.
`-macro_pgv_leakage {true | false}`	
	Default: `true` Controls leakage power reporting for cells that have power information available in both dotlib and PGV. When the parameter is set to `false`, the leakage current from Liberty is used and when the parameter is set to `true`, the leakage current from PGV is used.
`-macro_toggle_pin_percentage` *percentage*	
	Default: `50%` Specifies the percentage of output data pins (ignoring the control pins) to be scheduled for a macro cell. This parameter is applicable to the state-propagation based vectorless flow. The default macro pin toggle percentage is 50%.
`-mbff_toggle_behavior { simultaneous | independent | sbff }`	
	Specifies the toggle behavior of multi-bit flip-flops (MBFF). The possible arguments are: • `simultaneous`: The multiple bits of the MBFF will toggle or not toggle at the same time. The MBFF will be considered as one flip-flop. • `independent`: The multiple bits of the MBFF will toggle independently. The MBFF will be considered as one flip-flop. • `sbff`: The MBFF will be considered as multiple Single-Bit Flip-Flops (SBFFs). Each bit will be considered as one flip-flop, and will toggle independently. The default value is `independent`.
`-mbff_toggle_pin_percentage` *percentage*	

	Specifies the percentage of Multi Bit Flip Flop (MBFF) pins to be scheduled. This parameter is applicable to the state-propagation based vectorless flow. This parameter will only work when `-mbff_toggle_behavior` is set to `independent`.
`-memory_current_scaling_method {1 | 2 | 3}`	
	Specifies to enable scaling of the NLPM-based currents by a user-specified scale factor for macro cells that are switching. The applicable arguments are `1`, `2`, and `3`, which means enable a scaling factor of `0.5`, `0.25`, and `0.1` respectively.
`-merge_switched_net_currents { true | false}`	
	Specifies to merge switched net currents into always-on currents. For designs with a large number of switched nets, this parameter helps improve performance by merging switched net instance currents with always-on instance currents. However, when this parameter is set to `true`, you will not be able to use the generated current files to run power-up analysis. *Default* : `false`
`-method { static | dynamic_vectorless | dynamic_vectorbased | dynamic_mixed_mode | event_based | vector_profile}`	

	Specifies the type of analysis to be performed. You can perform two types of power analysis, namely static and dynamic.
	• `static` - specifies to perform static power analysis to calculate average power.
	Dynamic power analysis can be a vector-based or vectorless approach.
	• `dynamic_vectorbased` - uses the full VCD output of a logic simulator to identify the instances that are switching and when they switch.
	• `dynamic_vectorless` - uses the timing arrival window information from a static timing analysis tool to determine when instances switch. A timing arrival window describes when a signal can change within a clock cycle. An additional algorithm then determines the instances that switch.
	• `dynamic_mixed_mode` - specifies to combine the vector-based and vectorless (mixed) dynamic analysis. This flow can be used in the following cases:
	◦ **case 1**: vectors are not available for every block of a full-chip design.
	In this case, the software uses the vectorless methodology for blocks with incomplete vectors
	◦ **case 2**: blocks with vectors having unannotated combinational/ sequential logic.
	In this case, the software performs state propagation for the missing combinational/ sequential logic. It uses the vectorless methodology for a logic cone that does not have the primary inputs or flops annotated from the vector, and then performs state propagation
	To enable the dynamic mixed mode analysis flow, you must specify the following commands:
	`set_default_switching_activity -clock_gates_output 2.0 -global_activity <activity>`
	`set_power_analysis_mode -method dynamic_mixed_mode`
	`read_activity_file -scope <> -block <>` -> (for available blocks)
	• `event_based -` specifies to enable event-based power analysis. This method supports multiple power analysis flows simultaneously. When specified, the tool performs accurate state-based power estimation for all events of a scenario in a design. Event-Based Power Analysis is applicable to all

event-based vectors, such as VCD, FSDB, SHM, and PHY.
You can specify this method to generate and analyze multiple types of power analysis flow reports in a single run. The following table shows the additional options required for the different flows:

Power Analysis Flows	Additional Options for `-method event_based`
Static	`no additional option required`
Average Power & Current	`-write_static_currents`
Dynamic Vectorbased	`-write_dynamic_currents`
Mixed Mode	`-add_simulation`
Dynamic Vectorless	
Profiling	`-worst_step_size`

- `vector_profile` - specifies to enable vector profiling. This is used to identify windows with maximum activity and power which can then be used to drive dynamic vector-based power analysis.

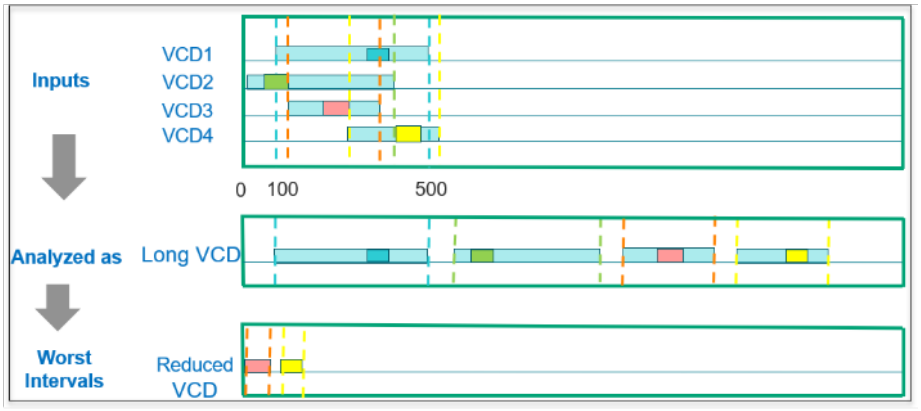
Default : `static`

`-min_leaf_count` *value*

Specifies the minimum number of leaf instances required for writing the hierarchy data in the profiling database. It allows you to restrict the number of hierarchies for which waveforms are reported in the profiling database when using both the `-write_profiling_db` and `report_power -hierarchy` parameters.

The default value of this parameter is `0`. Any hierarchy whose leaf instance count in its complete sub-tree is less than the `-min_leaf_count` value will be skipped from getting reported in the profiling database. This parameter can be used to skip very small hierarchies and as a result save on the database writing time and disk space.

`-multi_scenario_simulation {true | false}`

	Specifies to create a long vector by combining multiple vectors and to identify the worst power intervals across the combined vector. The long vector includes all scenarios that are used sequentially and are non-overlapping.  This parameter allows you to perform power and current analysis using worst peak power interval across multiple simulation vectors. *Default* : `false`
`-off_pg_nets` *net_list*	
	The name of the power rails that will be turned off. The net name can contain instance hierarchy if appropriate. This data will be taken into account when figuring out the list of power gates which are switched off. **Note:** A net is turned off if all connected power gates are turned off by control signals. When a net is off, all instances connected to that net will contribute zero power.
`-output_current_data_prefix` *prefix*	
	Specifies a prefix to the static or dynamic current files generated by the software. When power analysis is done in the multi-CPU mode, the output current data directory name and the underlying current files will also use the same prefix by default.
`-partition_count` *value*	
	Default: `0` Specifies the number of physical partitions or jobs for a distributed run in the area-based partitioning flow. **Usage Guidelines**: • This parameter is only applicable to the event-based power analysis flow (`set_power_analysis_mode -method event_based`) in the XP mode. • This parameter must be specified with the `-use_physical_partition` parameter.
`-partition_twf {true | false}`	

	Specifies to read the input TWF data (flat or hierarchical TWF files), and generate block-level TWFs. The default is `false`. This parameter must be specified when the block/partition level TWF files are not available.
`-pin_based_twf {true | false}`	
	Specifies whether to generate pin-based TWF or net-based TWF. The default value is `false`. • When set to `false`, a net-based TWF file is generated. Net-based TWF file is smaller in size and therefore results in faster processing. • When set to `true`, a pin-based TWF file is generated. Pin-based TWF file is bigger in size and requires longer processing time, however, is more accurate because the slews numbers are pin-based. **Note:** This parameter must be used with the `read_sdc` command.
`-power_grid_library` **{** *`library_list`* **}**	
	Specifies the name of the Cadence power cell libraries (`.cl`). You can also specify a list of defined power-grid libraries having decap cells tagged in them. **Note**: When specifying a list of PGVs to power analysis using the `set_power_analysis_mode` command, you must specify the technology library as the first PGV library in the list, followed by the standard and macro PGV libraries. The selection between the standard and macro PGV library is order based. Therefore, if multiple power-grid libraries are specified for a cell, the power engine maps the cell with the first library after the technology library.
`-power_include_initial_x_transitions {true | false}`	
	Controls how the initial X state toggle should be counted. The initial X state means the state at time `0`. The default value is `true`. This means that the software counts the power caused by the `X->0` or `X->1` toggle. When set to `false`, the software does not count the power caused by the initial X state toggle.
`-power_match_state_for_logic_x` *`value`*	
	Controls how the logic X will be evaluated in the boolean function of the '`when`' state of a power table. The default value is `0`. The parameter uses the following arguments: • `0` - regards X as 0 • `1` - regards X as 1 • `x/X` - regards X as neither `0` nor `1`, that is, whenever there is any X logic in it, the boolean function will be evaluated false.
`-precision` *`value`*	
	Specifies precision of decimal range [1-8] to ensure consistent decimal places are displayed for each power component. The default value is `8`. This parameter is used for precision handling in `report_instance_power` and `report_power`.

`-pre_simulation_empty_period` *period*	
	Specifies to add an empty pre-simulation period before simulation during IR drop analysis. Power analysis is not done for the pre-simulation period and flat dynamic currents are created.
`-pre_simulation_period` *period*	
	Specifies to set a pre-simulation period for IR drop analysis. Pre-simulation is used for generating power analysis reports and dynamic currents.
`-pre_simulation_power_exclude_period` *period*	
	Specifies to exclude power analysis during the pre-simulation period used for IR drop analysis. Pre-simulation is used only to create dynamic currents and not used for power reports.
`-quit_on_activity_coverage_threshold` *threshold_value*	
	Specifies the activity (VCD/FSDB/TCF) coverage threshold value. This parameter allows you to control the acceptable activity coverage that you are expecting and enforce the software to quit when it does not meet the criteria. When you specify the threshold value, the software exits if the real activity coverage is below the threshold.
`-read_rcdb {true | false}`	
	Specifies to write a SPEF file containing only the total C for signal nets and pass it to the Dynamic Power engine. Dynamic power analysis requires only the total C for signal nets. Therefore, instead of writing a large SPEF file (with distributed RC) from the RCDB, this parameter specifies to create and pass a SPEF file with only total C for all the signal nets. This significantly reduces the parasitic annotation time in the Dynamic Power engine. *Default* : `false`
`-relax_arc_match {true | false}`	
	Specifies to relax arc match when the exact arc is not found in a .lib. When set to `true`, the internal energy is calculated as an average of multiple arcs with relaxed arc matching. If this parameter is set to `false`, relaxed arc search is not done and 0 internal energy is returned. *Default* : `true`
`-report_black_boxes {true | false}`	
	Specify this parameter to report cells that are used as black boxes. A black box is a cell which is either missing from the liberty file or has no functionality defined in the liberty file. *Default* : `false` This is an optional parameter.
`-report_clock_instance_switching_info {true | false}`	
	Specifies to generate a report for clock instances that switched during the state-propagation based vectorless flow. The generated report contains the toggle time, instance name, input pin event, and output pin event for each switching event of the clock instance.

`-report_idle_instances {true | false}`	

	Specifies to generate a report (`idleinstance.rpt`) that lists all the idle or non-switching instances.
	
	The default value is `false`.
	
	The following is a snippet of the `idleinstance.rpt` report:
	`DFF22/DFF F`
	`DFF42/DFF F`
	`...`
	
	Here, the first column is the instance name and the second column is the type of instance (`F` indicates flop and `C` indicates combinational).

`-report_instance_switching_info {all | output_logic | none}`	

	Specifies to generate a report for combinational instance switched during state-propagation. In the output file, it contains the toggle time, instance name, input pin event, and output pin event for each switching event of the combination gate. The possible arguments of this parameter are:
	• `all` - Specifies to report instance toggle information for both input and output pins. When this parameter is set to `all`, the switching of both the input and output pins of all the instances are reported in the `voltus_power.stateprop.switchinst` file and the switching of flops are reported in the `voltus_power.stateprop.switchsrc` file.
	• `output_logic` - Specifies to report instance toggle information for the output pins only. When this parameter is set to `output_logic`, the switching of the output pins of all the instances are reported in the `voltus_power.stateprop.switchinst` file and the switching of flops are reported in the `voltus_power.stateprop.switchsrc` file.
	• `none` - Disables generation of instance toggle report. This is the default option. When this parameter is set to `none`, only switching of flops are reported in the `voltus_power.stateprop.switchsrc` file. The file does not report switching of instances.
	The default value is `none`. This parameter is used for generating switching details of state-propagation and should be specified only for debugging purpose.
	The following is a snippet of the `voltus_power.stateprop.switchinst` report:
	`*-----------------------------------------------`
	`*Switch Time  Instance Name  Pin Name  Logic`
	`*-----------------------------------------------`
	`*`
	`  0.097  B11/BUFF  I  rise`
	`  0.097  DFF11/DFF  Q  rise`
	...
	Here,
	• `Switch Time`: switching time of instances that can be either combinational or sequential
	• `Instance Name`: instance name
	• `Pin Name`: input/output pin of the instance
	• `Logic`: present logic of the pin that can be either "rise" or "fall"

`-report_instance_switching_list` *filename*	

Specifies to generate a report (`voltus_power.stateprop.switchlist`) containing all the switching times for the specified instances when the `-enable_state_propagation` parameter is set to `true`.

The format of the specified file is:

```
instname1
instname2
...
```

The following is the format of the `voltus_power.stateprop.switchlist` report:

```
*-------------------------------------------------------------------------------------
*Instance Name   Pin Name   Pin Direction   Frequency   Switch Time
*-------------------------------------------------------------------------------------
*
```

Here,

- `Instance Name`: instance name that can be either combinational or sequential
- `Pin Name`: name of the input/output pin of the instance
- `Pin Direction`: indicates whether the pin is an INPUT (I) or OUTPUT (O) pin
- `Frequency`: clock frequency
- `Switch Time`: gives the switching pattern for the instance, that is, <*time1*> <*logic*> <*time2*> <*logic*>. The logic of the pin can be "`r`" or "`f`". `r` indicates rise and `f` indicates fall.

The following is a snippet of the `voltus_power.stateprop.switchlist` report:

```
*-------------------------------------------------------------------------------------
*Instance Name   Pin Name   Pin Direction   Frequency   Switch Time
*-------------------------------------------------------------------------------------
*
   DFF11/DFF        QN        O              100       0.106 f 10.107 r
   DFF11/DFF        Q         O              100       0.097 r 10.099 f
   DFF11/DFF        CP        I              100       0.059 r 5.056 f 10.059 r 15.056 ...
...
```

`-report_library_usage {true | false}`

Default: `false`

Specifies to report library usage statistics in a text report. The report contains information such as, cell name, library format used (CCSP/ECSM/NLPM) for each cell, library name, and so on. This parameter allows you to perform library integrity checking for debugging purpose.

The following is the use model of the `-report_library_usage` parameter:

```
set_power_analysis_mode -method static | dynamic_vectorless | dynamic_vectorbased \
                        ...
                        -report_library_usage true
report_power -output <power_dir> -report_prefix <prefix>
```

The report is saved at `<power_dir>/<prefix>.lib.rpt`.

File Format:

```
Instance Cell  Net      Pin    Operational  Library   Library     Missing       Library
Name     Name  Name     Name    Voltage     Voltage   Format      Tables          Name
```

Examples:

```
Instance Cell  Net      Pin    Operational  Library   Library     Missing       Library
Name     Name  Name     Name    Voltage     Voltage   Format      Tables          Name
INV_178  INV   VDD,VDDA VDDG,VDD  0.9        1.0       NLDM      leakage,timing    A
INV_179  INV   VDDC     VDDG      0.75       0.75      CCSP,NLDM   leakage         B
```

Usage Guidelines:
This parameter must be specified with the `report_power -report_prefix` parameter.

`-report_missing_bulk_connectivity {true | false}`

Specifies to report missing bulk pins in the missing input report (`-report_missing_input`).

`-report_missing_input {true | false}`

Specifies to report missing netlist information in input files, such as library, LEF/DEF, PGV, SPEF, TWF, and missing logical connectivity for cell instance (PGNET).

The name of the report file is: `<prefix>_missingdata.<spef|lefdef|lib|pgv|twf|pgnet>`

Following is the use model:

```
set_power_analysis_mode -report_missing_input true
report_power -output <dirname> -report_prefix <prefix>
```

`-report_missing_nets { true|false }`

Controls reporting of missing nets in both static and dynamic power analysis.

This parameter controls reporting for TWF and activity files in static analysis, and controls reporting for TWF/SDC/SPEF and activity files in dynamic analysis.

Default : `false`

`-report_scan_chain_stats {true | false}`

Specifies to generate a report file called `voltus_power.scanchain.stat` during scan mode analysis that contains scan chain statistics.

The default value is `false`.

The report has 2 parts:

- **Global Statistics**: this section has the cumulative statistics of all scan chains in the design
- **Chain Statistics**: this section has the statistics for each scan chain in the design

The Global and Chain Statistics sections have the following information:

- `Total`: total instances specified in the control file or in a scan chain
- `AssignedFF`: flip-flops defined in the scan control file which exist in the design
- `SwitchedFF`: flip-flops that have switching event at the output
- `Non-ScanFF`: flip-flops that have no scan-out pin
- `DuplicatedFF`: flip-flops that are specified in multiple chains
- `StateConflictFF`: flip-flops that have conflicting state assignment

`-report_stat {true | false}`

Reports a set of statistics on the instance power, instance power density, clock power, or transition density. This parameter is useful in viewing and debugging user defined activity factors.

Default : `false`

`-report_time_display_fraction_digits` *value*

Sets the number of fractional digits or decimal places displayed for the instance switching time values in the `voltus_power.stateprop.switchinst` file. This parameter displays fractional digits that show the exact time difference if there are two events (rise/fall) of an instance switching at the same time.

Default : `-1`

`-report_twf_attributes {detailed | summary}`

Controls the generation of the TWF attribute report based on the TWF attributes that are specified using the `set_twf_attribute` command.

The possible arguments are:

- `summary` - generates a summarized static timing analysis report "`voltus_power.twf_attribute.summary`" that displays the number of user-defined TWF attributes (delay, slew, and slack values for both the rise and fall transitions) and the number of annotated TWF attributes for dynamic power analysis. The following is a snippet of the report:

```
STATISTICS:
                    User Defined            Annotated               Annotated/Total(%)
shift_arrival_time   5                       4                       80
rise_slew            2                       2                       100
fall_slew            2                       2                       100
rise_delay           5                       4                       80
fall_delay           5                       4                       80
rise_slack           2                       2                       100
fall_slack           2                       2                       100
```

- `detailed` - generates a detailed static timing analysis report "`voltus_power.twf_attribute.detailed`" that includes the summary report, and a detailed report of the rise and fall transitions for the TWF attributes of each annotated instance/pin. It also includes the shift arrival time report.

  This argument also allows you to generate a report `voltus_power.twf.rpt` containing the TWF statistics to highlight clock-frequency and delay used for the static and dynamic power calculation.

  The TWF statistics report is generated only if:

  - the `read_twf` command is specified before `set_power_analysis_mode`
  - the `set_power_analysis_mode -method static -static_netlist def` or `set_power_analysis_mode -method dynamic_vectorless | dynamic_vectorbased` command is specified.

  The format of the `voltus_power.twf.rpt` report is:

```
# Voltus TWF Statistics
# Units: Freq (Hz), Delay (second), Slew (second)
# TWF File List:
#     TWF_file1
#     TWF_fileN

USER DEFINED TWF ATTRIBUTE STATISTICS
<Clock Freq> <Min Rise_Delay> <Rise Delay> <Max Rise Delay> <Min Fall Delay> <Fall
Delay> <Max Fall Delay> <Net>
```

  The `USER DEFINED TWF ATTRIBUTE STATISTICS` section is included in the report only if the TWF attributes are defined using the `set_twf_attribute` command.

Example:

#check all instances of inst_C1 get shifted by 3ns

```
set_twf_attribute -pin inst_C1 -shift_arrival_time 3
```

#check all instances of inst_D1 get shifted by 3ns

```
set_twf_attribute -net inst_D1 -shift_arrival_time 3

set_power_analysis_mode -method dynamic_vectorless -create_binary_db false -disable_static
false -enable_state_propagation true -current_generation_method peak -power_grid_library
/../library_pv.cl -report_twf_attributes summary
```

`-reset`	Resets all or specified parameters to their default values. When you specify only the `-reset` parameter, the software resets all parameters for the `set_power_analysis_mode` command. When resetting specific parameters, the `-reset` parameter must be the first followed by the specific parameter. For example, you can specify the following command to reset only the `-method` parameter while keeping all other parameters' values unchanged: `set_power_analysis_mode -reset -method` **Note:** When resetting a specific parameter, you must not specify any arguments for that parameter.
`-reuse_flop_ranking_data` *`directoryname`*	
	Specifies the name of the directory where the flop ranking data to be reused is placed.
`-reuse_flop_ranking_data_hier` *`blockname`*	
	Allows you to choose the block/hierarchy whose ranking data is to be used for the block-level analysis. This parameter is used when the flop ranking data is transferred from a top-level analysis where blocks are flattened to another block-level analysis. The `-reuse_flop_ranking_data_hier` parameter enables you to specify the required block if the top-level contains multiple instantiations of the same DEF block.
`-save_bbox {true | false}`	
	Default: `false` Specifies to save the boundary box of each instance, its placement, and rotation at the top level. When set to `true`, generates the power database with each instance boundary box, placement, and rotation information at the top level. **Usage Guidelines**: This parameter must be specified when generating the Voltus thermal model.
`-scale_to_sdc_clock_frequency {true | false}`	

	Specifies to enable the VCD clock frequency to be scaled up to the SDC clock frequency. This allows you to match the VCD clock to the SDC clock to run analysis. By default, the clock frequency is scaled based on the activity file (Default: `false`). When you set the `-scale_to_sdc_clock_frequency` parameter to `true`, the power report and current waveforms apply the after-scaled clock frequency. The default behavior for clock frequency scaling in the RTL and Gate level VCD flows is different, as described below: • **RTL VCD** - By default, the clock frequency is scaled to the SDC clock frequency (`-scale_to_sdc_clock_frequency true`) when the RTL VCD flow is enabled (`set_power_analysis_mode -enable_rtl_vectorbased_dynamic_analysis true`). You can disable clock frequency scaling by setting the `set_power_analysis_mode -scale_to_sdc_clock_frequency` parameter to `false`. • **Gate VCD** - By default, scaling of the clock frequency is disabled (`-scale_to_sdc_clock_frequency false`) . You can enable the VCD clock frequency to be scaled up to the SDC clock frequency by setting the `set_power_analysis_mode -scale_to_sdc_clock_frequency` parameter to `true`. This works only when `set_power_analysis_mode -use_zero_delay_vector_file` is set to `true`, which means that the gate VCD is a zero-delay gate VCD.
`-scan_chain_activity {design_name chain_name activity_number} | {vectorless activity_number}`	

	Specifies to either provide vectorless activity for all scan chains or provide a specific activity for a specific chain. This parameter allows you to specify vectorless or user-defined activity to scan chains.
	The following are the two use models:
	• Generate the scan control file for a specific chain:
	`set_power_analysis_mode -scan_chain_activity {design_name chain_name activity_number}`
	• Generate the scan control file for all scan chains:
	`set_power_analysis_mode -scan_chain_activity {vectorless activity_number}`
	The `-scan_chain_activity` parameter is supported only in the XP mode.

`-scan_chain_name {`*`design_name chain_name pattern`*`}`	
	Specifies to automatically generate a scan-chain control file from a scan-chain based DEF file for the shift mode scan-chain analysis. This parameter allows you to create a control file for a large-sized DEF containing long chains or for multiple-DEF files. The generated scan-chain control file is saved in the power output directory. The use model of the parameter is: • `-scan_chain_name {all pattern}` - generate scan control file for all scan chains. **Example**: `-scan_chain_name {all 11010}` • `-scan_chain_name {design_name chain_name pattern}` - generate scan control file for a specific chain. **Example**: `-scan_chain_name {counter AutoChain_1_seg1_clk_rising 10010}` Using this parameter, you can generate the control file for both flat and hierarchical DEFs.

`-scan_control_file` *filename*	
	Specifies the scan control file name required to run the Scan Mode Analysis flow. The scan control file contains details about the scan analysis mode used for testing. For more information, refer to Scan Control File Format.
`-scan_mbff_chain_type` *type*	
	Specifies the type of multi-bit scan flip-flop. • `liberty` - specifies to use the syntax in Liberty to decide the multi-bit flip-flop type. • `serial` - specifies to pick the first flop as the starting flop, and treat the scan chain as serial, regardless of .lib. • `parallel` - specifies to treat the scan chain as parallel, regardless of .lib.
`-set_power_event_based {true | false}`	

	Default: `false`
	Specifies the instance-based power constraints that are to be taken into consideration for the power analysis flow. The parameter controls whether the power constraints are applied to the dynamic or static flow. The power constraints are specified using the `set_power` command.
	The default value of `false` specifies to apply the power constraints in the static flow. When set to `true`, the constraints are applied to the dynamic flow.
	Example:
	The following command specifies to honor the instance-based power constraints in the dynamic power analysis flow:
	`set_power_analysis_mode -method event_based -set_power_event_based true`
	`set_power -ascii_power_file instPower.txt`
	`report_power -outfile power.rpt`

`-settling_buffer` *value*	

Specifies the settling buffer time (in ps unit) between multiple windows of a VCD file. This parameter is used when you specify multiple pairs of the start time and end time for the non-overlapping multiple windows specified in an activity file (using the `read_activity_file` command).

Multiple current waveforms representing multiple VCD windows are merged into a single continuous current waveform. To capture the waveform for signals at the edge of user windows, a buffer time is added between the windows being stitched. This buffer time ensures that the stitched waveform is contiguous. The generated waveform hence has the duration of user windows and the intervening settling buffers. The default settling buffer period is 400ps.

You must specify the buffer value without unit.

The following is an example of this feature:

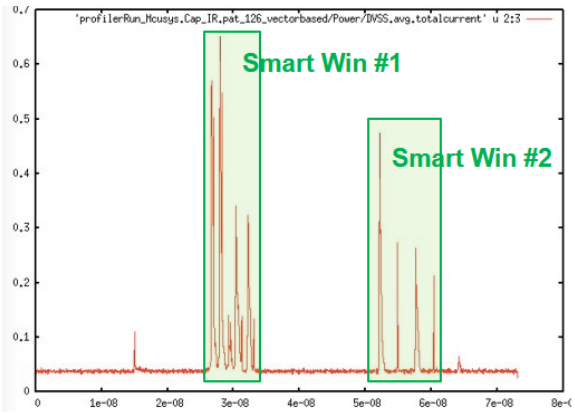
```
set_power_analysis_mode -verbosity true \
    -settling_buffer 200 -report_missing_nets true

read_activity_file \
    -format  VCD -scope  adder/u1 \
    -start  {0ps 2000ps} -end    {1100ps 3100ps} \
    -block  ../vcd/dut_5buf_full_chip.vcd
```

`-smart_window {true | false}`

Default: `false`

Specifies to enable the smart window feature to extract peak power scenarios from a vector-based activity file. A smart window is a variable size window to capture maximum regions of high power dissipation. An activity file can have multiple variable-sized regions that are ranked in the descending order of power metric.



The use model of the parameter is:

```
set_power_analysis_mode -smart_window true
set_power_analysis_mode -worst_step_size value
set_power_analysis_mode -worst_window_type [ power | activity
| delta_power | delta_activity | ir | critical_inst_power ]
```

`-split_bus_power {true | false}`

A value of `false` applies the internal power number to each individual bit.

A value of `true` divides the internal power number by bus width, before applying it to the bus bit.

Default : `false`

`-state_dependent_leakage {true | false}`

When set to `false`, power analysis performs state independent leakage power calculation.

Default : `true`

`-stateprop_ignore_unannot_pins {true | false}`

Default: `false`

Specifies to ignore unannotated pins in the functional logic of output pins for the RTL propagation flow. This parameter is set to `true` if there are unannotated input pins for instances and such pins are used in the functional logic of output pins that are propagated. For example output pin Q is propagated with functional logic A & B where A is unannotated and might cause wrong propagation for the Q pin.

`-static_multi_mode_scenario_file` *filename*

Specifies a file with information on instance scaling scenario for the static multi-mode IR analysis flow. When this parameter is specified, Power Analysis creates the multi-mode static current file that is used during rail analysis. The format of the input file is:

```
INST_NAME       PIN_NAME        SCALE_FACTOR_1       …..   SCALE_FACTOR_N
```

Here, the first column indicates the macro instance name, the second column indicates the pin name, followed by multiple current scaling factors. Each column of the scaling factor is referred to as a scenario. You must specify positive values only for scale factors.

The following is a snippet from the specified input file:

```
top/macroA     VDD     1     0.5     0.5     1
top/macroA     VDDM    0.5   0.5     0.5     1
top/macroB     VDD     0.5   1       0.5     0.5
```

If scale factors are not specified for certain scenarios, the tool will automatically use the instance's last stored scale factor value to complete current file generation.

In the following example, scale factor value `0.45` will be used in the 3rd and 4th scenario for instanceC, and scale factor value `0.35` will be used in the 4th scenario for instanceD:

```
instanceA VDD 0.45 0.55 0.65 0.31
instanceB VDD 0.31 0.5 0.67 0.21
instanceC VDD 0.25 0.45
instanceD VDD 0.31 0.25 0.35
```

`-static_netlist {verilog | def}`

	Specifies to perform static power analysis using a Verilog or DEF only neltist. By default, the static power analysis is performed using the Verilog netlist as the primary netlist. Using this parameter, you can specify DEF when a DEF netlist with logical connectivity is available, and use it as primary and not consider the Verilog netlist. This parameter is also enabled when `set_rail_analysis_mode -report_power_in_parallel` is `true` for static power computation. *Default* : `verilog`
`-start_time_alignment {true |false}`	
	Specifies to align all the signals of the activity file at time zero. When set to `true`, the simulation period is `max(endtime - starttime)`. When it is `false`, the simulation period is `max(endtime)- min(startime)`. *Default* : `true` **Note:** This parameter is not supported in the vector profiling flow. In the vector profiling flow, the start time will always be the `min(starttime)` and the end time will always be `max(endtime)`.
`-switching_power_on_rise_only {true | false}`	
	Default: `false` Specifies to perform switching power analysis only for rise toggles.
`-thermal_input_file` *file*	

Specifies to read a power map file to perform thermal analysis. Thermal input file contains die area divided into tiles, and temperature value for each tile. This temperature of each tile is used to calculate power of all the instances that fall in that tile's bounding box.

The format of the thermal input file is:

```
DIE_TEMPERATURE_MAP
DIE_AREA   xmin(um) ymin(um) xmax(um) ymax(um) nx(int) ny(int)

NUMBER_OF_LAYERS    nLayers(int)
LAYER    layer_name(string)
TEMPERATURE
XCOOR    x0(0)    x0(1)    x0(2)... ...x0(nx-1)
YCOOR    y0(0)    y0(1)    y0(2)... ...y0(ny-1)

T0(0,0)    T0(1,0)    ...    T0(nx-1,0)
T0(0,1)    T0(1,1)    ...    T0(nx-1,1)

:
T0(0,ny-1)    T0(1,ny-1)         T0(nx-1,ny-1)
END_TEMPERATURE
END_LAYER
END_DIE_TEMPERATURE_MAP
```

where,

- `DIE_AREA` specifies the coordinates of the four points of die outline as well as the tile number in X and Y direction.
- `NUMBER_OF_LAYERS` is the number of layers in each die instance
- `LAYER` specifies the LEF layer name
- XCOOR and YCOOR are the coordinates of each tile
- `T0(0,0)` …. is the temperature value assigned to a tile at a specific coordinate

`-thermal_leakage_temperature_scale_table_file` *filename*

Specifies a file containing the user-defined temperature values and temperature scale factors for each instance. This parameter allows you to use the current file (.ptiavg) with re-adjusted leakage power values to perform thermal-aware IR drop analysis.

When this parameter is specified, the generated power will include the scaled leakage power values. The `-thermal_leakage_temperature_scale_table_file` parameter must be specified with the `-thermal_input_file` parameter to import the temperature map file generated by Celsius.

The following is the format of the temperature-aware leakage scaling table:

```
#Type    Instance_Name        temp1             temp2             temp3             temp4
(Type: Lib/Cell/inst)

inst    inst_name1    scalefactor@temp1   scalefactor@temp2  scalefactor@temp3
scalefactor@temp4
inst    inst_name2    scalefactor@temp1   scalefactor@temp2  scalefactor@temp3
scalefactor@temp4
```

File Format Example:

```
#Type     Name        25    50    125    150

inst    ring/b213    1     2     5      10
inst    ring/b212    1     2     5      10
inst    ring/b211    1     2     5      10
inst    ring/b210    1     2     5      10
```

`-to_x_transition_factor` *value*

Specifies how 0/1 to X transitions are counted. The default value is `0.5`.

`-to_z_transition_factor` *value*

Specifies how 0/1 to Z transitions are counted. The default value is `0.25`.

`-transition_factor_based_duty {true | false}`

Default : `false`

Specifies that duty for X and Z toggles is calculated using the X and Z transition factor of toggle specified using the following parameters:

- `-from_x_transition_factor`
- `-from_z_transition_factor`
- `-to_x_transition_factor`
- `-to_z_transition_factor`

The default value of `false` specifies to use the duty value given with the `set_default_switching_activity`/`set_switching_activity` `-duty` parameter.

`-transition_time_method {min | avg | max}`

	Default: The default value is: • `min` for static power analysis using the Verilog netlist (`-static_netlist verilog`). • `avg` for static power analysis using the DEF netlist (`-static_netlist def`) and dynamic power analysis. Specifies the minimum, maximum, or average transition time method that will be used with the integrated timer or the external TWF.
`-twf_delay_annotation {min | avg | max}`	
	Allows you to choose between min, max, or avg arrival times from the timing window file (TWF). The arrival time is annotated to the vectors processed from the RTL VCD/FSDB Zero-Delay flows. The default value is `avg`.
`-twf_load_cap {min | avg | max}`	
	Allows you to select minimum, maximum, or average value of TWF external load or capacitance for power calculation. *Default* : `max`
`-unified_power_switch_flow {true | false}`	
	Default: `false` Specifies to enable the unified analysis of the always-on and switch nets. When set to `true`, the power-up and always-on analyses are performed in a single flow, wherein the switch nets and always-on nets are simulated together resulting in improved performance and accuracy.
`-use_cell_leakage_power_density {true | false}`	
	Specifies to use library leakage density times area (`default_leakage_power_density`), if defined, instead of the library's default cell leakage power (`default_cell_leakage_power`). This is used when leakage power for cells are missing or not defined. The default value of this parameter is `true`.
`-use_fastest_clock_for_dynamic_scheduling {true | false}`	
	Specifies to use only the respective fastest clock associated with each net or pin to schedule events for a given simulation period. The default value of this parameter is `false`.
`-use_lef_for_missing_cells {true | false}`	
	Allows a combination of LEF and PGV in the dynamic analysis flow. The default value of this parameter is `false`. If there are cells that are common to LEF and PGV, the PGV cell will get the precedence.
`-use_only_ddv_current {true | false}`	

	Default: `true` Controls the use of static current of a detailed dynamic power-grid view (DDV PGV) cell for static power calculation. The parameter is set to `true` by default, that is, the DDV PGV static current is not used in the static power calculation flow. If you want to use the DDV PGV static current for static power calculation, set this parameter to `false`.
`-use_physical_partition {true | false}`	
	Default: `false` Enables tile or area-based partitioning of a design for distributed processing. When this parameter is specified, the top-level design is divided into physical tiles, which are then grouped to ensure that each group or partition has the same instance count. Each partition is considered and processed as a separate job. **Usage Guidelines**: • This parameter is only applicable to the event-based power analysis flow (`set_power_analysis_mode -method event_based`) in the XP mode. • This parameter must be specified with the `-partition_count` parameter.
`-use_zero_delay_vector_file {true | false}`	
	Enables zero delay mode in vector-based dynamic analysis to avoid pessimism in current estimation. When set to `true`, the software reads the cycle accurate (zero delay) VCD/FSDB file and adds the timing/delay information from the Timing Analyzer or external TWF file to the VCD/FSDB file. Default: `false`
`-vector_profile_mode {activity | event_based | power_density | transient}`	

	Default: `event_based` Specifies the method used to perform vector profiling. The possible arguments are: • `activity` - performs activity profiling. This parameter generates only the worst activity report, and does not generate the average power report. The name of the file is as specified using the `report_power -outfile` parameter. • `event_based` - generates the average power report. This is the average power of all events in the user-specified window. This report includes a summary based on the power components, cell type, power rail, and clock network. The worst window report is captured in a file specified with the `report_power -outfile` parameter, and the report is suffixed by '`.avgpower`'. When `-outfile` is not specified, the `vectorprofile.report.avgpower` report file is created. • `power_density` - performs area or power density profiling. For this method, power is reported for each tile of the design, where design can be considered to be physically divided into equal area tiles. By default, a design is divided in 10x10 tiles. • `transient` - generates the event-based average power report and current files for the worst power window. **Usage Guidelines**: This parameter must be specified with the `-method vector_profile` parameter.
`-worst_case_vector_activity {true | false}`	
	Default: `false` Specifies to use the worst activity value when multiple vectors are specified in the static power calculation flow.
`-worst_step_size` *value*	
	Default: Total simulation duration/100 Specifies the step size or resolution for vector profiling. Activity/ Power profiling is carried out for every step size that you specify. When this parameter is used with `-worst_window_size`, the specified step size is the resolution at which window sliding is performed. The supported time units are s, ms, us, ns, and ps. If no explicit time unit is specified, the default ns will be applied.
`-worst_window_count` *value*	
	Default: 1 (implements single worst window) Specifies the number of worst windows from the last vector profile run to be used for dynamic power analysis. It provides n top windows with maximum activity/power/IR. This parameter is used when you need to combine dynamic power analysis and vector profiling for multiple worst-case windows in a single run.
`-worst_window_coverage { top | tile }`	

Default: `top`

Specifies how the worst window type is applied for multi-window analysis in the vector profiling flow. The following are the possible options of this parameter:

- `top`: worst window selection for the full design
- `tile`: interval selection as per the worst tile power

Use Model:

```
set_power_analysis_mode -method event_based -worst_window_coverage tile
report_power -outfile power.rpt -power_density_tiles 100
```

This flow should be run with `-method event_based`.

When the `-worst_window_coverage tile` parameter is specified, you should include any one of the following parameters to specify the total number of tiles:

- `report_power -power_density_tile_size " 15 15 "`
- `report_power -power_density_tiles_row_col " 2 2 "`
- `report_power -power_density_tiles 100`

The software generates the `power.rpt.density` report that has the power density tiles profile report with an additional section "Intervals covering unique tiles having worst power density".

`-worst_window_reports {full | worst | both}`

Specifies to generate static power analysis reports for the specified worst window type. The possible arguments are:

- `full`: creates the static power report for the full simulation window specified with the `-start`/`-end` parameters of the `read_activity_file` command.
- `worst`: creates the static power report for the worst window only.
- `both`: creates the static power report for both the full simulation and worst windows.

The default value is `full`.

The worst window report is created with '**.1**' added in the file name, as shown below:

- Static reports for full simulation: `power.rpt.avgpwr`, `power.rpt.instpwr`
- Static reports for worst window: `power.rpt.1.avgpwr`, `power.rpt.1.instpwr`

`-worst_window_size` *value*

Default: None

Specifies the duration of dynamic analysis. This parameter is used for sliding window vector profiling that involves sliding the profiling windows by using a very small resolution value to get the window with the worst current profile. `-worst_window_size` must be specified with `-worst_step_size`.

The supported time units are s, ms, us, ns, and ps. If no explicit time unit is specified, the default ns will be applied.

`-worst_window_type { power | activity | delta_power | delta_activity | ir | critical_inst_power }`

	Default: `power` Specifies the window type to be used for multi-window analysis. The possible arguments are: • `power` - worst instance power • `activity` - worst instance activity • `delta_power` - worst dp/dt (rate of change of power) • `delta_activity` - worst da/dt (rate of change of activity) • `ir` - worst IR • `critical_inst_power` - worst power for critical/IR-sensitive instance (worst timing and IR)
`-write_boundary_leakage_edge_annotation {true | false}`	
	Default: `false` Writes an edge annotation summary file for boundary leakage calculation. This parameter is used to report location-based Vt layer information on all the abutted edges in the design. The generated report name is `boundarygateleakage.edgeannotation.rpt`.
`-write_default_uti {true | false}`	
	Default: `true` For instances that are not hooked to any power/ground rail, the current for these instances are reflected in the default static current files: • `static_default_ground_rail.ptiavg` • `static_default_power_rail.ptiavg`
`-write_dynamic_currents {true | false}`	

	Default: `false`
	Specifies to create dynamic current files either for full simulation duration or for user-specified duration.
	Usage Guidelines:
	This parameter is only applicable to the event-based power analysis flow (`set_power_analysis_mode -method event_based`).
	Example:
	`set_power_analysis_mode -method event_based -write_dynamic_currents true -enable_state_propagation true`
	`read_activity_file sim.vcd -format VCD`
	`report_power -outfile power.rpt`

`-write_profiling_db {true | false}`	

	Default: `false` Writes out profiling databases which can later be viewed as histograms using the SimVision interface. You can specify this parameter during both the activity and power vector profiling. The profiling databases support instance level and power/ground net histograms. It creates an output file name with the `*.trn` extension.
`-write_simulation_db {true | false}`	
	Default: `false` Specifies to save functional waveforms for each instance when parsing a vector-based activity file. When set to `true`, creates SimVision waveform database (SHM database) for toggle simulation used during power analysis, which can later be viewed as histograms using the SimVision interface. The simulation database (shmdb.trn) is generated in the power analysis output directory.
`-write_static_currents {true | false}`	
	Default: `false` A value of `true` tells Voltus to generate the current data files per net. **Note:** When you perform static power calculation in the dynamic power analysis flow, you can use either the `-disable_static` or `-write_static_currents` parameter to enable static power calculation and write static current file. If only `-disable_static` is set to `false` and `-write_static_currents` is not set, the tool automatically sets `-write_static_currents` to `true`. Similarly, if `-write_static_currents` is set to `true`, and `-disable_static` is not set, the tool automatically sets `-disable_static` to `false`.
`-x_count_transition_using_3_states {true | false}`	
	Default: `false` Specifies to calculate the toggle rate using 3 states, pre-previous, previous and current states.
`-x_transition_factor` *value*	
	When using VCD/FSDB, one can specify how transitions to and from X are counted. These transitions are defined as shown in Table 5-1. Transitions to and from the X state will be treated as full transitions multiplied by the specified factor. **Note:** 0 -> X -> 1 or 1 -> X -> 0 count as full transitions and they are not controlled by the parameter. *Default* : `0.5`
`-z_transition_factor` *value*	
	When using VCD/FSDB, one can specify how transitions to and from Z are counted. These transitions are defined as shown in Table 5-1. Transitions to and from the Z state will be treated as full transitions multiplied by the specified factor. **Note:** 0 -> XZ-> 1 or 1 -> Z-> 0 count as full transitions and they are not controlled by the `parameter`. *Default* : `0.25`
`-zero_delay_vector_toggle_shift` *value*	

	Specifies to shift the current waveform in the zero-delay VCD flow. This parameter is useful in removing the idle period for the current waveform result with a long idle period (small flat current) before the first high current peak because of delay annotation. The parameter allows to offset the idle period from the delay values. The use model is given below: `set_power_analysis_mode -zero_delay_vector_toggle_shift -3.0ns` The default value is 0ns. The supported time units are s, ms, us, ns, and ps. The default unit is ns. You can specify a negative or a positive value. A positive value means right shift in the timeline or add more delay to the original arrival time, and a negative value means left shift in the timeline or trim the unwanted idle period. The `-zero_delay_vector_toggle_shift` parameter enables you to make a more realistic current waveform that can be easily aligned with the other current waveforms. **Note**: This parameter is similar to the `set_twf_attribute -shift_arrival_time` parameter, with the only difference that `-zero_delay_vector_toggle_shift` supports both positive and negative values while `-shift_arrival_time` supports only a positive value.

Table 5-1 **Transitions from/to X and Z values and**

Transition	Definition	Default
`to/from X`	`[ 0 or 1 or Z] <-> X`	`0.5`
`to/from Z`	`[ 0 or 1 ] <-> Z`	`0.25`

Scan Control File Format

The use model for the Scan Mode Analysis flow is:

`set_power_analysis_mode -method dynamic_vectorbased -scan_control_file` *`filename`*

The format of the scan control file is:

```
MODE = [ SHIFT | VECTORLESS]
VECTORLESS_GLOBAL_ACTIVITY = <factor> [Between 0 - 1]
<chain_name> <flop_inst> <initial_state> <master_cell_name> <scale> <mode> <scan_in_pin> <scan_out_pin>
<scan_en_pin>
```

Note that <`chain_name`> is optional, <`initial_state`> is optional in vector-less mode, and <`master_cell_name`>, <`scale`>, <`mode`>, <`scan_in_pin`>, <`scan_out_pin`>, <`scan_en_pin`> are optional and only needed for the first internal scan flop inside one macro.

Where,

- `Mode` specifies the mode used for scan design analysis. The two modes are: `SHIFT` and `VECTORLESS`.
- `VECTORLESS_GLOBAL_ACTIVITY` specifies the activity specification used in the `VECTORLESS` mode.
- `chain_name` is the name of the scan chain.
- `initial_state` is the initial state of all scan flops.
- `master_cell_name` is the name of the macro.
- `scale` is the scaling factor applied to the current.
- `mode` is mode name of the scan mode in the PGV view.
- `scan_in_pin` is the macro's scan_in pin name.
- `scan_out_pin` is the macro's scan_out pin name.
- `scan_en_pin` is the macro's scan_enable pin name.

Examples

- The following command uses the negative internal power from the .lib files during analysis, specifies that the transitions that are to and from the Z state will be multiplied by.2, and specifies that the transitions that are to and from the X state will be multiplied by .45:

```
set_power_analysis_mode -honor_negative_energy true \
-z_transition_factor .2 \
-x_transition_factor .45
```

- The following command resets the -honor_negative_energy parameter to its default value:

```
set_power_analysis_mode -reset -honor_negative_energy
```

- The following command resets all set_power_analysis_mode parameters to their default values:

```
set_power_analysis_mode -reset
```

- The following command ignores leakage power, and scales only the switching and internal power by the specified scale factor:

```
set_power -instance top.block2 2mW
set_power_analysis_mode -disable_leakage_scaling true
```

- The following command sets up dynamic power analysis.

```
set_power_analysis_mode \
    -method dynamic_vectorless \
    -corner max \
    -create_binary_db true \
    -write_static_currents true \
    -honor_negative_energy true \
    -ignore_control_signals false \
    -power_grid_library { fast_allcells.cl }
```

- The following command adds a prefix SM_ to the current data directory and the underlying current files:

```
set_power_analysis_mode -ouput_current_data_prefix SM_
```

The following current files are generated:

```
SM_dynamic_VDD.ptiavg
./SM_PTIData/SM_dynamic_1_VDD.ptiavg
./SM_PTIData/SM_dynamic_2_VDD.ptiavg
```

- The following command is another example of dynamic power analysis.

```
set_power_analysis_mode \
    -method dynamic_vectorless \
    -off_pg_nets VDDlu \
    -create_binary_db true \
    -write_static_currents true \
    -honor_negative_energy true \
    -ignore_control_signals false \
    -power_grid_library {accurate_stdcells.cl TSDN65LPA1024X32M8F.cl
```

- The following command is an example of the state-propagation-based dynamic vectorless flow, and also specifies to generate a report (`voltus_power.stateprop.switchlist`) containing all the switching times for the instances specified in the `inst_list` file:

```
set_power_analysis_mode -method dynamic_vectorless -enable_state_propagation true -
report_instance_switching_list inst_list
```

- The following is an example of running power analysis in the distributed mode:

```
set_distribute_host -rsh -add "m1 m2"

##The following blue-highlighted parameters specify the number of remote hosts and number of CPUs per
remote host
set_multi_cpu_usage -remoteHost 8 -cpuPerRemoteHost 2 -localCpu 16

##The following blue-highlighted parameter specifies that all the blocks included in the dp_setup.txt file will
be run in parallel
set_power_analysis_mode -method dynamic_vectorless -distributed_setup dp_setup.txt
```

The following is the content of the `dp_setup.txt` file:

```
INST top.block1 block
INST top.block4 block
CELL block2 block
```

In the above example, 4 power distributed jobs are created for: top.block1, top.block4, hierarchical cell block2, and any other hierarchical instances not specified in `dp_setup.txt` that are run with the top-level.

- The following is an example of performing standalone power report generation in the XP mode:

```
set_power_analysis_mode -method dynamic_vectorless \
-enable_state_propagation true \
-power_grid_library {  /pgv_dir/tech_pgv/techonly.cl  /pgv_dir/stdcell_pgv/stdcells.cl
/pgv_dir/macro_pgv/macros_pll.cl }\ -extraction_tech_file /qrcTechFile -enable_xp true
...
```

```
report_power
```

- The following command specifies to enable area-based partitioning, and distribute the event-based analysis into four jobs:
  ```
  set_power_analysis_mode -method event_based -enable_xp true -use_physical_partition true -
  partition_count 4
  ```
- The following command specifies to enable interactive generation of power reports:
  ```
  set_power_analysis_mode -method event_based -use_physical_partition true -partition_count 2 -
  enable_interactive_reports true -enable_xp true
  ```
- The following command specifies to enable disk caching of toggles:
  ```
  set_power_analysis_mode -method event_based -enable_disk_mapping true -enable_xp true
  ```

Related Information

- `get_power_analysis_mode`
- `report_power`
- RTL-to-Gate Rule Mapping
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_power_calc_temperature

```
set_power_calc_temperature
temperature
```

Sets the temperature for static power calculation.

In the presence of a trilib set, the internal and leakage power will be interpolated as the per the given temperature specification.

Parameters

temperature	Specifies the temperature for static power calculation. **Note:** The static power calculation software gets the temperature value of each instance from the following sources (in order of precedence): • Instance temperature file • `set_power_calc_temperature` command • The current operating condition temperature

Related Information

- `set_power_analysis_mode`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_power_include_file

```
set_power_include_file
file
[-reset]
```

Specifies a PowerMeter include file. You can use the `set_power_include_file` command to specify certain commands of the dynamic engine which do not have an Voltus equivalent. For more information on the PowerMeter commands, refer to the "Advanced PowerMeter Options" chapter of the *Voltus Text Command Reference*.

Note: The `set_power_include_file` command is used to pass PowerMeter and `report_power` command options only in the dynamic mode (vectorbased and vectorless). If `set_power_analysis_mode` is set to `static`, the `set_power_include_file` command is ignored.

Parameters

file	Name of the include file. This is used when translating PowerMeter commands to Voltus IC Power Integrity Solution commands. This file will include all PowerMeter commands that can not be translated to Voltus commands. In addition, you can also include the `report_power` command options to generate different power reports.
`-reset`	Resets all options back to default values.

Examples

- The following command specifies a power include file of power.inc:
  ```
  set_power_include_file power.inc
  ```
- The following example shows the contents of the power.inc file.
  # power.inc file must use the pre-defined design and power objects as "design" # and "ChipPwr" respectively. The power output object is not supported.
  ```
  design calCellsToIgnore BUFX1
  ChipPwr calRailVoltage VDD 1.2

  report_power -net -outfile Net.rep
  report_power -cell {INV*} -outfile Invpower.rep
  ```

Related Information

- `set_power_analysis_mode`
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_power_output_dir

```
set_power_output_dir dir
[-reset]
```

Specifies the name of the directory that power information will be output to.

Parameters

dir	Specifies the name of the directory in which power analysis output information is written.
-reset	Resets all options back to default values.

Examples

- The following command specifies the output directory called `static_power_AvallOn` in which the power information is written.
  ```
  set_power_output_dir static_power_AVallOn
  ```

Related Information

- `set_power_analysis_mode`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_switching_activity

```
set_switching_activity
[-reset]
[-activity factor | -density transition_density ]
[-clock clock_name]
[-duty value]
[-inst instance_name]
[-net net_name | -port port_name | -pin pin_name | -input_port port_name |-output_port port_name | -
bidir_port port_name]
[-period value]
[-unclocked]
[-icg_ratio num]
[-comb_clockgate_ratio num]
[-hier hierarchy_name]
[-scale_factor value]
[-cell cell_name]
[-force]
[-quiet]
```

Specifies activity for nets, pins, and ports. This command is used to specify user-defined activities at specific pins/nets/ports. You must specify this command before `report_power`.

The `set_switching_activity` command can accept a net/pin/port list using the `get_pins`/`get_ports`/`get_nets` commands. You can use the `get*` commands to create a collection of nets/pins/ports whose name matches the supplied pattern list. Therefore, providing you with the flexibility in providing lists through use of regular filter patterns. This feature is supported in both static and dynamic analysis engines.

Parameters

`-activity` *factor*	
	Specifies the activity for a net, pin, or top-level port. The *activity_factor* specifies the number of times the net, pin, or port switches in a clock cycle. Enter any positive number. If if the activity is set to 0.5, the activity happens once for every two clock cycles. If the activity is set to 2.0, the activity happens two times per clock cycle. There is no default value.
`-bidir_port` *port_name*	
	Specifies the name of the bidirectional port. The port names are in the following format: bidirectional*-port*. This parameter supports wildcards (*)

`-cell` *`cell_name`*	
	Specifies the activity on all output pins for all instances of the specified cell.
`-clock` *`clock_name`*	
	Assigns global activity factor to all nets in domain of *`clock_name`* if and only if *`clock_name`* is the fastest clock in the domain set. To use this option, `-clock` and `-activity` must be used without any other options.
`-comb_clockgate_ratio` *`num`*	
	Specifies to set the propagation ratio for combinational clock gate cell outputs. When specified, this parameter sets the output activity of any instance that is identified as a combinational clock gating cell to *`num`* times the fastest transition density arriving on the instance's clock inputs. You can specify this ratio for a net, pin, port, or clock. This parameter cannot be specified with the `-hier` parameter.
`-density` *`transition_density`*	
	Specifies the transition density for a specific net, pin, or top-level port. Transition density specifies the number of transitions per second. it can be any positive number. There is no default value. **Note:** If you specify the `-density` parameter in conjunction with the `-period` parameter, the software issues a warning and the `-period` parameter is ignored.
`-duty` *`value`*	
	Specifies the duty cycle, which is the probability that the signal is a logical 1. The `-period` parameter must be specified to add a duty. If not specified, the `set_switcing_activity` command picks this value from the external TWF or SDC through the Common Timing Engine (CTE). *Default*: Uses the duty specified by the `set_default_switching_activity` command if the `-duty` parameter is not specified.

`-force`	
	Specifies to apply user-defined activities for specific nets, pins, and ports, without having to load a verilog netlist using the `set_top_module` command. When the `set_top_module` command is not specified, the `set_switching_activity` command does not support the `-cell` parameter, and the net/pin/port list using the `get_pins/get_ports/get_nets` commands.
`-hier` *`hierarchy_name`*	
	A hierarchical name that an activity is being assigned to.
`-icg_ratio` *`num`*	
	Specifies to set the propagation ratio for integrated clock gate (ICG) cell outputs. When specified, this parameter sets the output pin of all identified ICG instances to *`num`* times the fastest transition density arriving on clock inputs of the instance. You can specify this ratio for a net, pin, port, or clock. This parameter cannot be specified with the `-hier` parameter.
`-input_port` *`port_name`*	
	Specifies the name of the input port. The port names are in the following format: *`input-port`*. This parameter supports wildcards (`*`)
`-inst` *`instance_name`*	
	Specifies the activity on a specific instance.
`-net` *`net_name`*	
	Specifies the name of the net. This parameter supports wildcards (`*`).
`-output_port` *`port_name`*	
	Specifies the name of the output port. The port names are in the following format: *`output-port`*. This parameter supports wildcards (`*`)
`-period` *`value`*	

	Specifies the period that the activity is referenced to. If not specified, a default frequency is used. If a negative number is specified, the static power calculation engine reverts back to the default frequency. Units in seconds (`s`), milliseconds (`ms`), microseconds (`us`), nanoseconds (`ns`), or picoseconds (`ps`). **Note:** If you specify the `-period` parameter in conjunction with the `-density` parameter, the software issues a warning and the `-period` parameter is ignored. *Default*: The time unit is defined in the first `.lib` file read during design import. If the time unit is not defined, the period specified in the `set_default_switching_activity` command is used.
`-pin` *pin_name*	
	Specifies the name of a pin. The pin names are in the following format: *instance-name + default-hierarchy-separator + pin*. This parameter supports wildcards (`*`)
`-port` *port_name*	
	Specifies the name of a top-level port. The port names are in the following format: *top-level-port*. This parameter supports wildcards (`*`)
`-quiet`	Suppresses all error and warning messages generated when the `set_switching_activity` command is run.
`-reset`	Resets all specified options back to default values. The reset option should be the first option specified, otherwise all options set prior to this will be reset. **Note:** You can selectively reset the pin/port/net specifications. In addition, you can also selectively reset the pin/port/net specifications of a list of objects through the `get_pins`/`get_ports`/`get_nets` commands.
`-scale_factor` *value*	

	Specifies to scale frequencies of the specified clock domains. This parameter allows you to scale transition densities of all instances associated with the clock domain by the specified scale factor to compute internal and switching power. This parameter must be specified with the `-clock` parameter. The `-scale_factor` parameter is only applicable to static power and dynamic vectorless power analysis.
`-unclocked`	Applies *`activity`* (with reference *`period`*) to all nets that have no clock domain (`-unclocked`). To use this option, `-unclocked` must be used with `-activity` and `-period` with no other options.

Activity Precedence

See "Activity Precedence".

Examples

- The following command specifies that net `n189` will switch once every ten clock cycles, with a clock period of `7520ns` and a duty cycle of 50 percent:
  ```
  set_switching_activity -net n189 -activity 0.10 -period 7520ns -duty .5
  ```
- The following command sets the transition density of port `out_cpu_top` to `20000` transitions per second:
  ```
  set_switching_activity -port out_cpu_top -density 20000
  ```
- The following command applies a transition density of `1e+06` to all input ports that match the specified pattern:
  ```
  set_switching_activity -port[get_ports "mode*" -filter {port_direction==in}] -density 1e+06
  ```
- The following command resets the previously defined activity on all nets that match the specified pattern:
  ```
  set_switching_activity -reset -net[get_nets n23*]
  ```
- The following command specifies the clock gate ratio for the clock domain `sys_clk`:
  ```
  set_switching_activity -clock sys_clk -icg_ratio 0.8 -comb_clockgate_ratio 0.5
  ```
- The following command specifies to set the default switching activity for all other clock domains of the design:
  ```
  set_default_switching_activity -icg_ratio 1.0 -comb_clockgate_ratio 1.0
  ```

Related Information

- `read_activity_file`
- `set_default_switching_activity`
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_thermal_analysis_mode

```
set_thermal_analysis_mode
[-help]
[-boundary_condition_file filename]
[-power_map_file filename]
[-temperature_map_file filename]
[-tool_path path]
```

Sets the parameters for performing chip-centric thermal analysis after generation of the Voltus thermal model (VTM). This command must be specified before the `analyze_thermal` command.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-boundary_condition_file filename`	

	Specifies the name of the boundary condition file. The format of the boundary condition file is:
	`#Length Unit : um`
	`#Temperature Unit: C`
	`die_thickness=100`
	`front Tfix=25`
	`back Tfix=25`
	`top Tfix=50`
	`bottom Tfix=50`
	`left Tfix=25`
	`right Tfix=25`

`-power_map_file filename`	
	Specifies the name of the generated power map file.
`-temperature_map_file filename`	
	Specifies the name of the output temperature map file. Each analysis run would create a directory named “ThermalDir_<number>” automatically. Soft links to the power mapping file and boundary condition file would be created inside the directory. The analysis’s final result is a temperature map named as “`<DieName>_TemperatureMap.dat`”, and is saved in this directory.
`-tool_path path`	
	Specifies the full path to the bin folder of the thermal analysis tool (Celsius).

Examples

- The following command specifies to set thermal analysis parameters:

```
set_thermal_analysis_mode -tool_path $CELSIUSTOOL \
-boundary_condition_file $bcDir/bc_100.txt \
-temperature_map_file temperatureMap_100 \
-power_map_file vtm_output/top.vtm.txt
```

Related Topics

- analyze_thermal

set_twf_attribute

```
set_twf_attribute
[-help]
[-clock clock_name]
[-clock_edge {rise | fall}]
[-clock_freq value]
[-fall_delay {min:max or single_value}]
[-fall_slack {min:max or single_value}]
[-fall_slew {min:max or single_value}]
[-net net_name]
[-pin pin_name]
[-reset]
[-rise_delay {min:max or single_value}]
[-rise_slack {min:max or single_value}]
[-rise_slew {min:max or single_value}]
[-type {data | clock}]
[-shift_arrival_time   time]
[-clear_twf_attr]
```

Specifies static timing analysis information. The command allows you to override timing parameters, such as slew and clock, on desired pins/nets from an existing timing windows file (TWF).

Parameters

`-clear_twf_attr`	
	Specifies to reset the TWF file values for a specific pin or net. The use model of this parameter is: `set_twf_attribute -clear_twf_attr -pin/-net <name>`
`-clock clock_name`	
	Specifies the clock name. This indicates that the nets and pins that are timing constrained have arrival times (timing windows) caused by this clock.
`-clock_edge {rise | fall}`	
	Specifies the capturing clock edge direction: rise or fall.
`-clock_freq value`	
	Specifies the clock frequency. You must specify the `-clock` parameter along with the `-clock_freq` parameter to assign the clock for which the frequency needs to be updated.
`-fall_delay {min:max` or `single_value}`	

	Specifies the fall delay for the pin. This parameter is not supported in the static power analysis flow.
`-fall_slack` *{min:max* or *single_value}*	
	Specifies the fall slack for the pin.
`-fall_slew` *{min:max* or *single_value}*	
	Specifies the fall slew for the pin.
`-net` *net_name*	
	Specifies the net name. The `-net` and `-pin` parameters are mutually exclusive.
-pin pin_name	
	Specifies the pin name. The `-net` and `-pin` parameters are mutually exclusive.
`-reset`	Resets all specified options back to default values.
`-rise_delay` *{min:max* or *single_value}*	
	Specifies the rise delay for the pin. This parameter is not supported in the static power analysis flow.
`-rise_slack` *{min:max* or *single_value}*	
	Specifies the rise slack for the pin.
`-rise_slew` *{min:max* or *single_value}*	
	Specifies the rise slew for the pin.
`-shift_arrival_time` *time*	

	Specifies to filter the idle period of the current waveform in the dynamic power analysis flow. This parameter is useful in removing the idle period of the current waveform result with long idle period (small current) before the 1st high current peak because of TWF delay annotation. The parameter allows to offset idle period from the TWF delay values.
	The use model is given below:
	`set_twf_attribute -shift_arrival_time 3.0ns`
	The default value is 0ns. The supported time units are s, ms, us, ns, and ps. The default unit is ns. You must specify a positive value only. A positive value means shift right in the time line or add more delay to the original arrival time.
	You can specify the `-clock` parameter to limit the changes to a clock domain. In addition, you can further add the `-pin/-net` parameter to limit the changes to a block. The following are examples of shifting different delays to different clock domains because different clocks may have different clock insertion delays at the top level:
	`set_twf_attribute -shift_arrival_time 1.2ns -clock clk_a`
	`set_twf_attribute -shift_arrival_time 2.3ns -clock clk_b`
	If a clock domain is not specified, the delay will be applied to all the specified nets/pins.
	Note: The final rise/fall delays for the annotated nets/pins after the shift has been applied are reported under the "Shift Arrival Time Report" section of the detailed TWF Attributes report. This report is generated using the `set_power_analysis_mode -report_twf_attributes detailed` parameter.
	The following example illustrates a sample command and report snippet:
	Command:
	`set_twf_attribute -shift_arrival_time 3.0ns`
	`set_power_analysis_mode -method dynamic_vectorless ... -report_twf_attributes detailed`
	Report:
	`Clock domain annotated`
	`- CK1_1`
	`- CK1_2`
	`- CK2_1`
	`- CK2_2`
	`Rise Delay Fall Delay Net Name`
	`1e-09 1e-09 CK1_1`
	`1e-09 1e-09 CK1_2`
	`1e-09 1e-09 CK2_1`
	`...`

`-type {data | clock}`	
	Specifies whether the net or pin is part of the clock or data network.

Examples

- The following command specifies to use the following TWF attributes for the net `netB1`:

  ```
  set_twf_attribute -net netB1 -rise_delay 2.1978:2.1978 -rise_slew 0.05:0.05 -rise_slack 0.2741:3.6961
  -fall_delay 2.1819:2.1819 -fall_slew 0.05:0.05 -fall_slack 0.237:3.6948 -clock CK1_1 -clock_freq 100 -
  type data
  ```

- The following commands reset the TWF values for the "`DFF11/DFF/Q`" pin to default, and then sets the rise/fall delay:

  ```
  set_twf_attribute -clear_twf_attr -pin "DFF11/DFF/Q"
  set_twf_attribute -pin "DFF11/DFF/Q" -rise_delay 0.35 -fall_delay 0.45 -clock CLK1
  ```

Related Information

- `set_power_analysis_mode`
- `read_twf`
- `report_power`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_virtual_clock_network_parameters

```
set_virtual_clock_network_parameters
[-help]
[-cell cell_name]
[-clock list_of_clocks]
[-library library_name]
[-max_fanout value]
[-wire_load_model wireload_model]
[-reset]
```

Specifies to estimate power for a virtual clock tree. When the command is specified, the software reports the power of the clock network based on the buffers inserted in the clock tree. It also reports the number of buffers used for implementing the clock tree and the depth of the clock tree.

You can use this command for fast power estimation of clock network power of a pre-CTS netlist.

This command should be invoked before running the `report_power` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `set_virtual_clock_network_parameters` parameter. For a detailed description of the command and all of its parameters, use the man command `man set_virtual_clock_network_parameters`.
`-cell cell_name`	
	Specifies the name of the buffer or inverter cell that has to be used to build the clock tree.
`-clock list_of_clocks`	
	Specifies the list of clocks for which the clock tree needs to be implemented. The default is for all the clocks.
`-library library_name`	
	Specifies the name of the library which contains the cell.
`-max_fanout value`	
	Specifies the maximum fanout that the buffer cell can drive.

`-wire_load_model` *wireload_model*	
	Specifies the wire load model used to compute the wire load model for the nets in the clock network.
`-reset`	Resets all specified parameters back to default values.

Example

The following command specifies to use the cell `BUFX12`, with a maximum fanout value of `2`, to build a virtual clock tree:

```
set_virtual_clock_network_parameters -cell { BUFX12 } -library typical -max_fanout 2 -wire_load_model B0X0
```

Related Information

- `set_power_analysis_mode`
- `report_power`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

write_power_constraints

```
write_power_constraints
[-help]
[-outfile filename]
```

Writes out all the power constraints such as the power analysis mode, activity file specifications, simulation period specifications, and so on, into a file. You can use this command at any time during the session to dump out the power commands/ options specified till that point.

You can source this file into other Voltus/Tempus/Innovus sessions in order to use the same settings.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `write_power_constraints` parameter. For a detailed description of the command and all of its parameters, use the man command `man write_power_constraints`.
`-outfile` *`filename`*	Specifies the output file into which the power constraints are to be written.

Example

```
write_power_constraints -outfile test1.tcl
```

Related Information

- `set_power_analysis_mode`
- `report_power`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

write_tcf

```
write_tcf file
[-block block_name]
[-exclude_hier_insts list_of_instances]
[-primary_input]
[-seq_output]
[-pin]
[-macro_output]
```

Writes out the propagated switching activity information to a toggle count format (TCF) file. The TCF file contains switching activity information, toggle count information, and the probability of the net or pin being in the logic1 state.

Parameters

`-block block_name`	
	Specifies to generate the block-level TCF from the top-level design. When specified, it writes out the block-level duty and activity in the generated TCF file. The use model of this parameter is: `write_tcf -pin -block blockname TCF_filename`
`-exclude_hier_insts list_of_instances`	
	Excludes the specified hierarchical modules or instances when writing out a pin-based TCF. This parameter allows you to ignore the activity generated for the excluded instances. The use model of this parameter is: `write_tcf -pin -exclude_hier_insts {list_of_instances}`
`-macro_output`	Specifies to generate a TCF file that includes all the activities for the macro outputs in the design.
`-pin`	Specifies to write out pin-based TCF.
`-primary_input`	Generates a TCF file that includes only the primary inputs in the design.
`-seq_output`	Generates a TCF file that includes all the activities for the sequential outputs in the design.
file	Specifies the name of the output TCF file.

Example

- The following command writes the propagated switching activity information to TCF file mmm.tcf:
  ```
  write_tcf mmm.tcf
  ```
- The following command writes a pin-based TCF for the whole design:
  ```
  write_tcf -pin pin_design.tcf
  ```
- The following command writes out a TCF which only includes activity information for the primary inputs and sequential outputs in the design:
  ```
  write_tcf -primary_input -seq_output Power.tcf
  ```
- The following command writes out a pin-based TCF that excludes the specified hierarchical instances:
  ```
  write_tcf -pin -exclude_hier_insts {DTMF_INST/CORE_INST DTMF_INST/RAM_128x16 DTMF_INST/RESULTS_INST }
  ```

Related Information

- set_power_analysis_mode
- report_power
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

27

Power-Grid Library Commands

- check_pg_library
- generate_pg_library
- generate_tech_pg_lib
- generate_xpgv
- merge_pg_library
- rename_pg_library
- set_advanced_pg_library_mode
- set_pg_library_mode
- trim_pg_library
- validate_pg_library

check_pg_library

```
check_pg_library
[-help]
power_library_paths
[-cell_list_file file]
[-check_compatibility {PGV1 PGV2 ...} | -lef_consistency]
[-check_parameters]
[-integrity_check]
[-output file]
[-list]
[-report]
[-rule_file file]
[-skip_signal_net_check {true | false}]
[-skip_layer_list_file filename]
[-summary]
[-tech_layers]
[-total_currents]
[-report_detail_multiple_voltage_cap]
[-command_dump]
[-metal_density_map]
```

Specifies to perform a comprehensive check of power-grid views (PGV) to be used in rail analysis. These checks include:

- Verify that all the PGV views exist: Early/IR/EM
- Report the dynamic PWL and static current
- Perform a compatibility check on version, layermap and resistivity parameters
- Check any missing information, such as temperature, qrcTechFile location, and so on.

Parameters

`-help`	Outputs the command usage.
power_library_path	

	Default: None **(Mandatory)** Specifies the absolute path to the power library (`.cl`) file. You can run multiple power library files (.cl) at the same time.
`-cell_list_file` *file*	

	Default: None
	Specifies a file containing the list of cells for which the LEF consistency checks will be done. This parameter allows you to perform the consistency checks only for the user-defined cells.
	The format of the input file is:
	`CELL1`
	`CELL2`
	`CELL3`

`-check_compatibility {`*PGV1 PGV2 ...*`}`	
	Default: None Checks for compatibility between the specified cell libraries. It checks whether the following parameters are the same for the libraries that are to be merged: • Time Units • Capacitance Units • Layout Scale Factor • Manufacturing Grid If one or more of these parameters are not the same, a warning message appears stating that the libraries are not compatible and cannot be merged. It also specifies the parameters that are different. Based on the above electrical/physical parameter check, a compatibility report (compatibility.report) with pass/fail is generated. Fail implies that the library is not compatible and cannot be used in Voltus rail analysis.
`-check_parameters`	
	Default: None Checks for the following electrical parameters of the specified cell libraries: • `CINT_MIN and CINT_MAX:` Intrinsic device capacitance • `CG_MIN and CG_MAX:` Grid capacitance • `RON_MIN and RON_MAX:` On-resistance for powergates • `IDSAT_MIN and IDSAT_MAX:` Saturation current for powergates • `ILEAK_MIN and ILEAK_MAX:` Leakage current for powergates • `IAVG_MIN and IAVG_MAX:` Static current • `IPEAK_MIN and IPEAK_MAX:` Dynamic peak current • `ESR_MIN and ESR_MAX`: Equivalent series resistance For powergates, `-check_parameters` will check the Vds-Ids table for the correct trend. It checks whether: • for a given `Vgs`, `Ids` increases with `Vds` • for a given `Vds`, `Ids` increases with `Vgs` where; `Vgs` is voltage across gate and source `Vds` is voltage across drain and source `Ids` is current from drain to source (of a transistor)
`-command_dump`	

	Default: None Specifies to print the settings used during PGV characterization to a text file called `command_list.txt`. This parameter allows you to determine the user-specified or default settings used during PGV generation for the specified power library (.cl).
`-integrity_check`	
	Specifies to load all the views from a power-grid library to ensure that the library contains the required data. This parameter checks if there are any missing files or views, and that the PGV is usable in rail analysis.
`-lef_consistency`	
	Default: None Checks for LEF consistency of the cells in the specified cell libraries. It reads the LEF and PGV files, and compares the given LEF file with the LEF file used in PGV generation.

`-list`	**Default**: None
	Specifies to list the PGVs and the cells in each PGV. The parameter allows you to determine the name of the PGV library that contains a specific cell and determine whether a cell is there in multiple libraries.
	The format of the output file is:
	`PGV Library: <Number of PGV libraries>`
	`<Lib_1> <TOOL VERSION> <PGV VERSION> <GENERATE_DATE> <Path>`
	`<Lib_2> <TOOL VERSION> <PGV VERSION> <GENERATE_DATE> <Path>`
	`...`
	`Duplicated Cells: <Number of cells in multiple libraries>`
	`<CELLNAME> <Lib_1 Lib_2 Lib_3>`
	`Cell List: <Number of cells in a library>`
	`<CELLNAME> <LIB#>`

`-metal_density_map`	
	Default: None Generates the metal density map report file (`<pgvname>.metal_density`) in the PGV output directory. This file contains information about the macro size, number of tiles in the x and y direction, metal layer ID, layer name, and metal density values for each tile. **Usage Guidelines**: This parameter is used with the `-calculate_metal_layer_density` parameter of the `set_pg_library_mode` command.
`-output` *dir_name*	
	Default: The default is the current working directory. Specifies the name of the output directory in which the power-grid library report is created.
`-report`	**Default**: None Specifies to generate the power-grid library report. The report name is `<pgvname>.report`.
`-report_detail_multiple_voltage_cap`	
	Default: None Specifies to generate a voltage-dependent capacitance table that contains the detailed capacitance values of the given power-grid library. The capacitance table file is called `<pgvname>.cap_table`. **Usage Guidelines**: This parameter is used with the `-enable_multi_voltage_cap_generation` parameter of the `set_pg_library_mode` command.
`-rule_file` *file*	

	Default: None
	Specifies the rule file name used for parameter checking. This file contains the list of electrical parameters to be checked.
	File Format:
	`CINT_MIN <min cap value>`
	`CINT_MAX <max cap value>`
	`RON_MIN <value>`
	`RON_MAX <value>`
	`IDSAT_MIN <value>`
	`IDSAT_MAX <value>`
	`ILEAK_MIN <value>`
	`ILEAK_MAX <value>`
	`IPEAK_MIN <value>`
	`IPEAK_MAX <value>`
	`IAVG_MIN <value>`
	`IAVG_MAX <value>`
	`ESR_MIN <value>`
	`ESR_MAX <value>`
	Usage Guidelines:
	This parameter must be specified with the `-check_parameters` parameter.

`-skip_layer_list_file filename`	
	Default: None Specifies the name of a file containing a list of layers to ignore while checking PGV and LEF compatibility. Its format is one layer per line. **Usage Guidelines**: This parameter must be specified with the `-check_compatibility` parameter.
`-skip_signal_net_check {true | false}`	
	Default: `false` Specifies to skip LEF consistency checks for signal nets. This forces skipping of the signal pin checks when performing a comprehensive check of power-grid views to be used in rail analysis. The `-skip_signal_net_check` parameter can be used if you want to check LEF consistency of only the power and ground pins.
`-summary`	
	Default: None Specifies to generate a detailed report of the library contents for the specified cells that can be used for debugging purpose.
`-tech_layers`	
	Default: None Specifies to report technology layer information and layer mapping.
`-total_currents`	
	Default: None Specifies to report the static current for each power/ground net, and dynamic PWL for each mode. For each mode, it reports the peak and average current, and the number of points for power supply nets. When specified, the total current is reported in `<lib_name>.total_current` for each library.

Examples

- The following command creates a report for the library std_cells.cl:
  ```
  check_pg_library std_cells_fast/std_cells_fast.cl -report
  ```
- The following command checks for LEF consistency:
  ```
  read_lib -lef ddk.0.lef
  read_lib -lef ddk.1.lef
  check_pg_library -lef_consistency \
  ```

```
"/../library1.cl\
/../library2.cl"
```

- The following command displays the list of layers (defined in the technology file) on stdout:
  ```
  check_pg_library -tech_layers ./check_param/powergate/stdcells.cl
  ```
- The following command checks for compatibility between the specified cell libraries and generates a compatibility report (compatibility.report) in the output directory:
  ```
  check_pg_library -output out -check_compatibility  {/../lib1.cl /../lib2.cl}
  ```
- The following command specifies to list the cells corresponding to the specified libraries, `pgv1.cl` and `pgv2.cl`:
  ```
  check_pg_library -list {/../pgv1.cl  /../pgv2.cl}
  ```
- The following command checks for electrical parameters of the stdcells.cl library:
  ```
  check_pg_library -output OUT -check_parameters ./stdcells.cl
  ```
  The following is a sample output of this command:

```
           PARAM        VALUE            RANGE         PASS/FAIL

Cell: Cell1
   Net VDDG
        Device Cap    4.50e-14    (1.00e-18,1.00e-11)    PASS

   Net VSS
        Device Cap    2.61e-14    (1.00e-18,1.00e-11)    PASS

   Net VDD
        Device Cap    2.13e-14    (1.00e-18,1.00e-11)    PASS

   Power Gate SLEEP
             Ron      1.84e+01    (2.00e+01,1.00e+03)    FAIL
             Ileak    7.01e-08    (1.00e-15,1.00e-06)    PASS
             Idsat    3.36e-02    (1.00e-09,1.00e-02)    FAIL

   Trend Checks:
       Ron Trend   : PASS
       Idsat Trend  : PASS
       Ileak Trend : PASS
```

- The following command reports the static current for each power/ground net:
  ```
  check_pg_library -total_currents /../pg.cl
  ```
  The following is a sample output of this command:

```
Net VSS
Static current = -0.00021181

Net VDD
Static current = 0.000211809

MODE1
Trigger Condition: ( !CEN & !WEN )
Net VDD
    Peak = 0.0138308
    Avg = 0.000276053
    Number of points 190

Net VSS
    Peak = -0.0166879
    Avg = -0.000300184
    Number of points 190

MODE2
...
```

- The following command generates a capacitance table (`powergate_stdcells.cap_table`) that contains the detailed capacitance values of the powergate_stdcells.cl library:
  ```
  check_pg_library -report_detail_multiple_voltage_cap ../powergate_stdcells.cl
  ```
  The following is a snippet of the capacitance table file (`powergate_stdcells.cap_table`):

```
========================================================================
CellName    NetName      Voltage       Capacitance
------------------------------------------------------------------------
```

```
powercell    vccm        0.1000        4.586e-14
powercell    vccm        0.2825        4.102e-14
...
```

- The following command specifies to output the settings used during generation of `macros_pll.cl` to a text file called `command_list.txt`:
  ```
  check_pg_library -command_dump macro_pgv/macros_pll.cl
  ```

  The following is a sample snippet of the `command_list.txt` file:
  ```
  temperature 25
  spice_subckts /../spice.sp
  spice_models /../spectre_load.sp
  extraction_tech_file /../tech.tch
  gds_files /../file1.gds
  stop@via VIA_1
  spice_subckts_xyscaling 1000
  use_powergate_data_from_lib 0
  default_frequency 2e+08
  decap_frequency 1e+08
  ```
- The following command generates the power-grid library report:
  ```
  check_pg_library -output report -report /../pg.cl
  ```

  The following is a sample output of this command:
  ```
  ================================================================================

  Library path name:        /it/../library
  Library name:            pg
  Hierarchical library:    NO
  User login:              user
  Date:                    2015-Sep-03 03:58:45 (2015-Sep-03 10:58:45 GMT)
  ToolIdentifier:          VOLTUS
  Application version:     15.20-d127_1
  Number of cells in library:    1

  ================================================================================
  Cell                PowerNets       Capacitance      PowerGrid Views
  ------------------------------------------------------------------------------------------------------------------------|

  cell1

                      VSS(0.0000)     5.3e-12          EARLY(255 taps) EM(8827 taps) IR(8827 taps)
                      VDD(0.9000)     6.8e-12          EARLY(193 taps) EM(7534 taps) IR(7534 taps)

   Cell Type = MACRO                                    Cell_Status: PASS

  INFO: Dynamic Tap Current views were created for this cell. MODE1 has Conditional input = "( !CEN & !WEN ) " and Conditional pin = "CLK
  RISE".
  INFO: Dynamic Tap Current views were created for this cell. MODE2 has Conditional input = "( CEN & WEN ) " and Conditional pin = "CLK
  RISE".

  ------------------------------------------------------------------------------------------------------------------------|
  ```

Related Information

- `validate_pg_library`
- "Power-Grid Library Generation" in the *Voltus IC Power Integrity Solution User Guide*

generate_pg_library

```
generate_pg_library
[-help]
[-output dir_name]
[-library_prefix prefix]
```

Generates the output for power-grid library generation.

Parameters

`-help`
Outputs the command usage.
`-library_prefix prefix`
Specifies a prefix for the power-grid library. When this parameter is specified, the default PGV for each cell type is as follows: • techonly - `prefix_techonly.cl (default: techonly.cl)` • stdcells - `prefix_stdcells.cl (default: stdcells.cl)` • macros - `prefix_cellname.cl (default: macros_cellname.cl)` You can generate macro PGVs without the "`macro_`" string in the name and without any prefix by specifying an empty string for `-library_prefix`, as shown below: `generate_pg_library -output output_folder -library_prefix ""`
`-output dir_name`
Specifies the name of the output directory in which power-grid libraries are created. The default is the current working directory. **Note**: If you specify both `-output output_dir` and `-library_prefix prefix` parameters, the command generates the PGV in the *output_dir* directory with PGV name `prefix_<name>.cl`.

Examples

- The following command generates a power-grid library in the *temp* folder:
  ```
  generate_pg_library -output temp/
  ```
- The following command specifies to generate a technology library with the default PGV name `techonly` and prefix `ab`:
  ```
  set_pg_library_mode -celltype techonly ...
  generate_pg_library -output ./ -library_prefix ab
  ```
  The generated library name will be `ab_techonly.cl`
- The following command specifies to generate a macro library with the default PGV name `<cell_name>` and prefix `ab`:
  ```
  set_pg_library_mode -celltype macros ...
  generate_pg_library -output ./ -library_prefix ab
  ```
  The generated library name will be `ab_<cell_name>.cl`

Related Information

- `set_pg_library_mode`
- `set_advanced_pg_library_mode`
- `generate_xpgv`
- "Power-Grid Library Generation" in the *Voltus IC Power Integrity Solution User Guide*

generate_tech_pg_lib

```
generate_tech_pg_lib
[-help]
[-decap_cells cell_list]
[-default_power_voltage value]
[-esd_cells cell_list]
[-filler_cells cell_list]
[-keep {true | false}]
[-lef_layermap filename]
[-output_dir dir_name]
[-power_gate_file filename]
```

-tech_file *filename*
[-tech_only {true | false}]

Generates the technology power-grid library on the fly. The generated tech-PGV will be deleted after the session is over. If this command is specified, the Power Analysis and Rail Analysis engines will automatically use the on the fly generated tech-PGV. The purpose of generating the tech-only PGV on the fly is that you do not need to keep a track of the tech-only PGVs. Thus, it ensures that the tech-only PGV is generated from the correct Quantus technology file.

The tech-only PGV library will be generated under the working directory and will be called by Power/Rail Analysis on the fly, as shown in the following log file snippet:

```
Begin Loading PGV Libraries for Power Calculation
.//pgv/techonly.cl
Ended Loading PGV Libraries for Power Calculation: (cpu:0:00:00,real=0:00:00,mem(process/total/peak)=275.25MB/2585.14MB/1194.52MB)

Begin Processing Cell Libraries for Rail Analysis

Merging libraries:
.//pgv/techonly.cl
../../LIBS/voltus_pgv/macro_pgv/macros_sram.cl
../../LIBS/voltus_pgv/stdcells_pgv/stdcells.cl

Output library: ./work/lib_merged.cl
```

Parameters

`-help`
Outputs the command usage.
`-decap_cells cell_list`
Specifies the names of cells that have explicit decoupling capacitance for use in decap optimization. These cells are nearly equivalent to feedthrough cells, except that there is typically an active device between the power and ground nets to provide extra decoupling capacitance. Power-grid view library generation uses this list of cells to ensure that a capacitance value is associated with each cell. If no capacitance is associated with a cell, a warning message will be issued.
`-default_power_voltage value`
Specifies voltage for the power pins which are present in LEF. The default value is 1.2.
`-esd_cells cell_list`
Specifies a list of Electrostatic Discharge (ESD) cells. You can use this parameter to characterize ESD cells.
`-filler_cells cell_list`
List the names of the filler or feedthrough cells that have no GDS data. These cells can be swapped out for decaps during decap optimization.
`-keep {true | false}`
Specifies that the generated library should be retained after the session completes. The default value is `false`.
`-lef_layermap filename`

Specifies a file that provides the mapping of layers between the LEF and technology file. This parameter is optional. Library Generation automatically generates the layermap file using the technology LEF (containing all layer and via definitions) and Quantus Tech (extraction library) files, therefore, a LEF layermap file is not a mandatory input. The automatically generated layermap file is located in the PGV temp directory.

The following is the format and sample content of the auto-generated file (`lefdefLayerMap_AutoGenerated.txt`):

```
Layer Type   tech layername   lefdef    lefdef layername
via              VIA_9        lefdef      VIA9
metal            METAL_1      lefdef      METAL1
```

You can specify this parameter if you want to provide your own layermap file.

`-output_dir` *`dir_name`*

Specifies the name of the output directory in which tech-only power-grid library is created. The default is the current working directory.

`-power_gate_file` *`filename`*

Specifies a file with the information that technology library generation needs to characterize a powergate cell. Specifies the cell name, unswitched power pin, switched power pin, on resistance in ohms, saturation current in milliamps, and leakage current in milliamps. The format of the file is:

`cell supply switched Ron Idsat Ileak`

The following is an example of multiple footer powergate:

```
FOOTBUFVHSV16 VSSG VSS 63 3.167 0.000128252
FOOTBUFVHSV32 VSSG VSS 63 3.167 0.000128252
FOOTBUFVHSV64 VSSG VSS 63 3.167 0.000128252
FOOTBUFVHSV8 VSSG VSS 63 3.167 0.000128252
```

`-tech_file` *`filename`*

Mandatory parameter. Specifies the name of the technology file that will be used for extraction.

`-tech_only {true | false}`

Specifies to generate the technology PGV without any cells. The default value is `false`.

Examples

- The following command generates a tech-only power-grid library:

```
generate_tech_pg_lib -tech_file $tech_path/qrcTechFile -filler_cells *FILL* -decap_cells *DCAP* -default_power_voltage 0.9
```

Related Information

- `set_pg_library_mode`
- `set_advanced_pg_library_mode`
- `generate_pg_library`
- "Power-Grid Library Generation" in the *Voltus IC Power Integrity Solution User Guide*

generate_xpgv

```
generate_xpgv
-cell cell_name
[-inst_list inst1 inst2 .. instn ]
[-libgen_include file_name]
[-output_dir directory_name]
[-rail_include file_name]
[-reuse_block_state_directory directory_name]
-state_directory directory_name
[-mvmf_config config_file]
```

Creates power-grid view (xPGV) models for large digital blocks.The block xPGV can be generated either after the signoff dynamic IR drop analysis or after the dynamic Early Rail Analysis (ERA) of the block. In both cases, the xPGV is generated using the R-C grid of the block and currents captured at the via layer during the analysis. These block-level xPGVs can then be instantiated during the next level of the design or the top-level full-chip analysis. The next level rail analysis will run faster with the block-level xPGVs because these runs use the power-grid that is modeled inside the xPGV.

Parameters

`-help`	Outputs the command usage.
`-cell` *`cell_name`*	
	Specifies the block name for which xPGV model is generated.
`-inst_list` *`inst1`* `inst2 ..` *`instn`*	
	Specifies the block instances for which xPGV model is generated.
`-libgen_include` *`file_name`*	
	Specifies a file that includes additional library generation options that will be run for xPGV model generation.
`-mvmf_config` *`config_file`*	
	Specifies to generate a single xPGV model with different functional modes. Using this parameter, you can specify a file that includes the different modes and the mapping state directory for each mode. When `-mvmf_config` is specified, the block-level currents are saved for multiple voltages and frequency (MvMf) per each functional mode in a single PGV. During xPGV instantiation, the different modes are automatically selected based on the power domain to which each mode is connected. The format of the *`config_file`* file is: *<modeName1> <stateDir_1>* *<modeName2> <stateDir_2>* … *<modeNameN> <stateDir_N>* Before generating MvMf-xPGV, you need to perform multiple rail analysis runs for different corners to generate different state directories (stateDir_*) to be included in the configuration file.
`-output_dir` *`directory_name`*	
	Specifies the name of the output directory in which the power-grid library report is created. The default is the current working directory.
`-rail_include` *`file_name`*	
	Specifies a file that includes additional rail analysis options that will be run for xPGV model generation.
`-reuse_block_state_directory` *`directory_name`*	

	Specifies an existing rail analysis state directory to skip block extraction. The software assumes that the design data (DEFs and GDS) and power-grid libraries are not changed, and will generate xPGV using the extraction data inside the specified state directory. If the state directory does not contain the necessary design and power-grid database files, it will issue an error message and provide information on the missing files. The `-reuse_block_state_directory` parameter is used if the entire design that is analyzed in the rail run should be used to create the xPGV. However, this parameter must not be used in case you require to generate xPGVs for each instance (sub-blocks) of a block. That is, for generating xPGVs for the sub-blocks B and C of the block A (rail analysis was performed on block A), you need to re-extract the block-level grids of B and C as part of xPGV generation for accurate current calculation. In this case, `-reuse_block_state_directory` must not be specified. For net based analysis, you should specify net-based state directory, as shown below: • `generate_xpgv -reuse_block_state_directory VDD_25C_avg_1` For domain based analysis, you should specify the domain's state directory, as shown below: • `generate_xpgv -reuse_block_state_directory domain_25C_dynamic_1` A new state directory will be generated for the new analysis with soft links to the old state directory files that were reused.
`-state_directory directory_name`	
	Specifies the path to the rail analysis state directory.

Examples

- The following command creates an xPGV library `hier_block.cl`:
  ```
  generate_xpgv -state_directory dynamic_rail/core_125C_dynamic_1/ -cell hier_block -output_dir ../xpgv/
  ```
- The following command creates an xPGV library `super_filter_A` with multiple functional modes specified in the `mode.config` file:
  ```
  generate_xpgv -cell super_filter_A -mvmf_config mode.config -output_dir ./mode_mvmf -rail_include rail.inc
  ```
  The following is a snippet from the mode.config file:
  ```
  label1 dynamic_rail/core_25C_dynamic_1
  lable2 dynamic_rail/core_25C_dynamic_1
  ```

Related Information

- `set_pg_library_mode`
- `set_advanced_pg_library_mode`
- `generate_pg_library`
- "Power-Grid Library Generation" in the *Voltus IC Power Integrity Solution User Guide*

merge_pg_library

```
merge_pg_library
[-help ]
[-force {true | false}]
-library_list_file file_name
[-library_prefix prefix]
[-merge_single_voltage_pgvs {true |false}]
[-output directory_name]
[-remove_tech {true | false}]
[-validate_rail_merge {true | false}]
```

Specifies to merge power-grid libraries. Using this command, you can merge multiple same-type power-grid libraries into one consolidated power-grid library.

Parameters

`-help`	Outputs the command usage.
`-force {true | false}`	
	Specifies to force merge PGVs with different resistivity and layers. It uses the technology information of the first library. If multiple tech-only PGVs a the first tech-only PGV definition and the first cell definition. If you are merging a tech-only PGV with other PGVs, it will keep the tech-only technol cell definition. The default value of the parameter is `false`.
`-library_list_file file_name`	
	Specifies the file containing the list of libraries.
`-library_prefix prefix`	
	Specifies a prefix for the generated power-grid library.
`-merge_single_voltage_pgvs {true |false}`	
	Specifies to merge all the single voltage PGVs into a multi-voltage PGV for better handling of the library characterization flow. This parameter is u you have generated multiple PGVs for different voltages, instead of generating a single multi-voltage PGV that has voltage-dependent capacitanc The default value of the parameter is `false`.
`-output directory_name`	
	Specifies the name of the output directory in which power-grid libraries are created. The default is the current working directory. **Note**: If you specify both `-output output_dir` and `-library_prefix prefix` parameters, the command generates the PGV in the *output_dir* directo `prefix.cl`.
`-remove_tech {true | false}`	
	Specifies to remove technology dependency from the specified or merged library. The newly created library will not contain the technology informa The default value of the parameter is `false`.
`-validate_rail_merge {true | false}`	
	Specifies to validate the merged PGV in the same way as done when merging PGV during rail analysis. When this parameter is specified, the log and info messages if the tool finds any issues during PGV merging. The parameter allows you to check the PGV merging issues as reported durin having to run the full design. `-validate_rail_merge` is used only for validation purpose, and the merged PGV is not to be used for analysis.

Examples

- The following command merges the power-grid libraries specified in the `merge_lib.list` file into one consolidated power-grid library:
  ```
  merge_pg_library -library_list_file "merge_lib.list" -output ./ -library_prefix "new_merged"
  ```
  The content of `merge_lib.list` file is:
  library1.cl
  library2.cl
  library3.cl

Related Information

- `set_pg_library_mode`
- `set_advanced_pg_library_mode`
- `generate_pg_library`
- "Power-Grid Library Generation" in the *Voltus IC Power Integrity Solution User Guide*

rename_pg_library

```
rename_pg_library
[-help]
[-cell_list_map_file filename]
[-merge_library_list_file filename]
[-output directory_name]
[-output_library new_library_name]
```

Specifies to copy and rename the specified power-grid view library. The command also allows you to rename the cells of the specified input library.

Parameters

`-help`	Outputs the command usage.
`-cell_list_map_file filename`	
	Specifies a file containing the list of cells to be renamed. The format of the specified file is: *<old cell name1> <new cell name1>* *<old cell name2> <new cell name2>* ...
`-merge_library_list_file filename`	
	Specifies a file containing the input library to be renamed. The format of the specified file is: *<path to the input library>*
`-output directory_name`	
	Specifies the name of the output directory in which the new power-grid library is created. The default is the current working directory.
`-output_library new_library_name`	
	Specifies the new name of the specified input library.

Examples

- The following command specifies to rename the input library to `new_lib`:

  ```
  rename_pg_library -merge_library_list_file new.list -cell_list_map_file cell.list -output_library new_lib -output .
  ```

Related Information

- `set_pg_library_mode`
- `set_advanced_pg_library_mode`
- "Power-Grid Library Generation" in the *Voltus IC Power Integrity Solution User Guide*

set_advanced_pg_library_mode

```
set_advanced_pg_library_mode
[-abort_on_extraction_errors {true|false}]
[-add_port_labels file_name]
[-assume_foreigns {true | false}]
[-assume_foreigns_mode [1 | 0]]
[-cell_accura_data_file file]
[-cell_per_distributed_host value]
[-cell_pinnet_map_file file]
[-circuit_include_file thunder.inc]
[-cluster_via_rule {{via_layer1 number_of_equidistant_vias}...}]
[-cluster_via_size value]
[-common_supply_pins {net_name+}]
[-create_static_view_from_dynamic_view {true | false}]
[-damping_decap_cell_list {cell1 cell2 ..}]
[-damping_decap_frequency value]
[-decap_frequency value]
[-default_frequency value]
[-default_power_voltage value]
[-delete_ddv_fsdb_files {true|false} ]
[-disable_powergate_rampup_simulation {true | false}]
[-distribute_current_to_switch_net {true | false}]
[-enable_ac_simulation_for_IO_characterization {true | false}]
[-enable_spectre_netlist_flow  {true | false} ]
[-enable_subconductor_layers {true | false}]
[-enable_via_based_current_distribution {true | false}]
[-esd_cells {cell_list}]
[-esd_cells_list_file filename]
[-esd_device_list {device1 device2 .... deviceN}]
[-esd_parameters_file filename]
[-esd_pin_list {pin1 pin2 .... pinN}]
[-exclude_tap_region_file filename]
[-extraction_command_file file]
[-followpins_tap_layer {lowest_lef_pin_layer | highest_lef_pin_layer | all_lef_pin_layers}]
[-followpins_interface_node_layer {lowest_lef_pin_layer | highest_lef_pin_layer | all_lef_pin_layers}]
[-generate_bulk_pin_cap_separately {true | false}]
[-generate_graybox_data {true | false} ]
[-generate_itfnode_x_direction {true | false}]
[-generate_itfnode_y_direction {true | false}]
[-generate_itfnodes_at_via_layer {true | false}]
[-ignore_pg_nets {{cellname  netname}+}]
[-import_xdspf_list_file filename]
[-input_port_value_list {pin1 value1 pin2 value2 ....pinN valueN}]
[-interface_node_location_file filename]
[-lef_layer_ignore_list {lef_layername +}]
[-lef_layer_ignore_list_file filename]
[-lef_pin_short {true | false}]
[-lef_pin_short_cell_file filename]
[-lef_pin_short_cell_list {cell1 cell2 ...}]
[-libgen_command_file file]
[-macro_parasitic_file filename]
[-marker_layermap filename]
[-mcpu_rerun_count count]
[-pgdb_layermap filename]
[-pgdb_list_file filename]
[-powergate_characterization_voltages { val1 val2 val3 .... }]
[-powergate_multi_enable_characterization_method {enable_all | enable_single} ]
[-process_bulk_diffusion_ports {true|false}]
[-remove_emview_dangling_resistor {true|false}]
[-retain_generated_pgv_on_error {true | false}]
[-ron_measure_threshold value]
[-schematic {true|false} ]
[-skip_switch_net_extraction {true | false}]
[-source_location_file {filename}]
[-spectre_path value]
[-stdcell_characterization_voltage_value  {val1  val2  val3 ...}]
```

```
[-strict_input_check {true|false}]
[-tap_node_distance value]
[-techgen_dir directory]
[-thunder_command_file file]
[-trim_metal_layer_map file]
[-use_embedded_spectre {true | false}]
[-use_peak_current_distribution {true | false}]
[-verbosity {true | false}]
[-via_based_current_config_file filename]
[-well_cap_file filename]
[-xdspf_layermap filename]
[-xtc_command_file filename]
[-xtc_include_file_for_qdv filename]
```

Specifies the advanced power-grid library generation features. If the `set_advanced_pg_library_mode` command is specified twice, the last specified command takes precedence.

Parameters

`-abort_on_extraction_errors {true | false}`	
	Specifies whether the software should exit with an error message or continue when it encounters extraction errors. *Default*: `true`
`-add_port_labels file_name`	
	Specifies the name of a file containing details about the label of a switched rail. Using this parameter, you can add a label for the switched rail manually in the specified port file, if the switched rail is not labeled. The format of the port file is given below: MACRO *macro_name* PIN *pin_name*
`-assume_foreigns {true | false}`	
	Enables the Library Characterization engine to process LEF files in which FOREIGN statements are not defined as part of the macro definition. When generating detailed power-grid views, the engine assumes that FOREIGN statements are defined as part of the macro definition in the LEF file. When this option is set to `true`, the engine processes the LEF file by using the name defined by the MACRO statement rather than requiring a FOREIGN statement. The `-assume_foreigns` parameter ensures that the FOREIGN statement is placed at 0,0. Therefore, if the MACRO statement has an offset, you must provide a FOREIGN statement that contains an offset. This option works for offsets that are 0,0 or the inverse of the origin. *Default*: false
`-assume_foreigns_mode {1 | 0}`	
	Determines how the Library Characterization engine processes cells for which FOREIGN statements are assumed. If set to 0, the engine assumes that the cell for which FOREIGN statements are assumed is placed at (0,0). When you set it to 1, the engine assumes that the FOREIGN cell is placed at the inverse of the macro origin. *Default*: 1
`-cell_accura_data_file file`	
	Specifies the name of a file containing data about the electrical characteristics of cells. Voltus library simulator requires this data to generate currents during power-grid view generation.
`-cell_per_distributed_host value`	
	Specifies the number of cells to be submitted per host for distributed PGV generation. *Default*: 50
`-cell_pinnet_map_file file`	
	Specifies the mapping between the cell LEF pin names and the internal names of the net to which those pins are attached. It is used when multiple pins are attached to a single net.

`-circuit_include_file simulator.inc`

Specifies the name of the input file for Voltus library simulator in the Library Characterization command file.

`-cluster_via_rule {{via_layer1 number_of_equidistant_vias}...}`

Controls the number of vias to cluster on a layer basis. The VIA clustering rule specified using this parameter will override the default clustering rule for PGV generation. You can use this parameter when you need clustering at VIA1, or different clustering for different layers. The default cluster size for PGV generation is `64` for all via layers except VIA1.

Example:

The following command will cluster 100 equidistant VIA1 cuts, 200 equidistant VIA2 cuts, and 300 equidistant VIA7 cuts:

```
set_advanced_pg_library_mode -cluster_via_rule {{VIA1 100} {VIA2 200} {VIA7 300}}
```

`-cluster_via_size value`

Specifies that power-grid extraction for library cell needs to perform 64 (8x8) via clustering inside PGV for all via layers above VIA1 between Metal1 and Metal2.

The following command specifies to perform 16 (4x4) via clustering:

```
set_advanced_pg_library_mode -cluster_via_size 16
```

Default: 64

`-common_supply_pins{net_name+}`

Identifies the generic supply net or nets which could act as either a power or ground supply net. No wildcards are allowed for the specification of net names.

Generic supply nets are used for pads or bump cells that could be connected to either a power or ground net. They will be connected to the correct power-grid based on the DEF or GDS file.

`-create_static_view_from_dynamic_view {true | false}`

Specifies to generate static power-grid view by averaging of dynamic currents in the multi-mode characterization flow. When this parameter is specified, the software performs averaging of the dynamic currents of all the modes and saves it as static current in the power-grid view.

Default: `false`

`-damping_decap_cell_list {cell_list}`

Specifies a list of damping decap cells.

`-damping_decap_frequency value`

Specifies the frequency of operation of the damping decap cells. The default value is 200MHz.

Default: `200MHz`

`-decap_frequency value`

Specifies the decoupling capacitance frequency used during PGV generation. Intrinsic capacitance of cells has a significant dependency on frequency used during PGV generation. PGVs characterized with higher frequency may lead to pessimism during dynamic analysis due to less impact of intrinsic capacitance of the cells characterized with higher frequency. Using this parameter, you can specify the dominant frequency to be used for PGV generation.

The default value for decoupling capacitance frequency is 1GHz.

`-default_frequency value`

Specifies the default frequency of signal nets in hertz (Hz); this is used to calculate the current and capacitance values during the creation of detailed power-grid views.

For example, the following command specifies that the default switching frequency is 100 MHz:

- `set_advanced_pg_library_mode -default_frequency 100e+06`

`-default_power_voltage value`

Specifies the default voltage for power nets. This parameter assigns a default value to power nets for which voltage has not been defined using the `-power_pins` parameter.

`-delete_ddv_fsdb_files {true | false}`

	Specifies to delete the intermediate FSDB files generated in the Spice deck (user-specified) method of current characterization. This parameter allows you to retain or delete the huge FSDB files.
`-disable_powergate_rampup_simulation {true | false}`	
	Specifies whether to run power gate ramp-up simulation during power-grid view generation. When this parameter is set to `true`, the Ids-Vds table will not be present in PGV, therefore, this PGV cannot be used for the power-up or rush current analysis flow. This parameter can be specified to reduce the run time when you require only the RON/Idsat/Ileak and Cap values in PGV. `disable_powergate_rampup_simulation` is used when power gate parameters are specified using the `set_pg_library_mode -powergate_parameters_file` parameter. This parameter can only be used for the standard cell PGV generation. *Default*: `false`
`-distribute_current_to_switch_net {true | false}`	
	Distributes the PWL waveform, which is specified to the always-on power net, to the taps of both the always-on power net and the related switched power nets in the fine-grain memory characterization flow. By default, the current waveform is distributed only to the taps of the always-on power net when you specify the PWL to the always-on power net.
`-enable_ac_simulation_for_IO_characterization {true | false}`	
	Specifies to perform an AC analysis based coupled capacitance calculation for IO cells. When set to `false`, the software performs DC analysis based coupled capacitance calculation for IO cells. The AC analysis based coupled capacitance calculation is slower than the DC analysis based coupled capacitance calculation. *Default*: `false`
`-enable_spectre_netlist_flow {true | false}`	
	Specifies to use the Spectre simulator to read the Spice/Spectre netlist for the standard cells characterization flow. This option should be used when the netlist is in Spectre format, or when the netlist is encrypted by Spectre. *Default*: `false`
`-enable_subconductor_layers {true | false}`	
	Allows to control the generation of the sub-conductor layers in the PGV. When this parameter is set to `false`, the sub-conductor layers are not generated in the PGV. *Default*: `true`
`-enable_via_based_current_distribution {true | false}`	
	Default: `false` Specifies to enable location-based via current modelling during GDS-based macro power-grid view generation. When set to `true`, defines the current values only on the user-defined via location. This parameter allows you to define average and peak current values for specific-location vias. The `-enable_via_based_current_distribution` parameter must be specified with the `-via_based_current_config_file` parameter. If the parameter is set to `false`, current is uniformly distributed across all vias in the macro PGV.
`-esd_cells {`*`cell_list`*`}`	
	Specifies a list of Electrostatic Discharge (ESD) cells. You can use this parameter to characterize ESD cells. When you specify this parameter, the list of ESD cells are later used in the `check_esd` command while loading the power-grid view library. As a result, you do not need to specify the ESD cell names with the `-esd_cell_list` parameter of the `check_esd` command.
`-esd_cells_list_file` *`filename`*	

Specifies a file containing a list of ESD cells that are later used in the ESD analysis flow while loading the power-grid view library.

The `-esd_cells_list_file` parameter is the same as the existing `-esd_cells` parameter of the `set_advanced_pg_library_mode` command, except that the `-esd_cells_list_file` parameter allows you to specify a list of ESD cells in a file while the `-esd_cells` parameter allows you to specify the ESD cells in the command line itself.

The following is an example of this parameter:

```
set_advanced_pg_library_mode -esd_cells_list_file {./Esd_Cell_list.txt}
```

The following is the ESD cell file format:

```
CELL_1
CELL_2
...
CELL_n
```

`-esd_device_list {`*device1 device2 .... deviceN*`}`

Specifies the list of ESD devices. The ESD device information is used for accurate ESD analysis as it takes into account the internal grid from the ports to the ESD devices. The followings inputs are required for ESD tagging during macro PGV generation:

- an xDSPF file
- ESD device list

`-esd_parameters_file` *filename*

Specifies resistance information for the following types of ESD cells during ESD cell characterization:

- Diodes
- RC Clamp

When this parameter is specified, the software uses these electrical parameters from the generated PGV for ESD analysis.

The format of the input file is:

```
Cell <cell name>  <Ron> <Roff> <power_pin> <ground_pin> <voltage>
Type <esd cell type> <Ron> <Roff> <power_pin> <ground_pin> <voltage>
```

Here,

- `Cell` and `Type` are keywords
- *<cell name>* specifies the ESD cell name
- *<esd cell type>* specifies the ESD cell type, such as DIODE and CLAMP
- `RON` is the resistance of the complete cell when it is switched on.
- `ROFF` is the resistance of the complete cell when it is switched off.
- *power_pin* is the name of the power pin connected to the power net
- *ground_pin* is the name of the ground pin connected to the ground net
- *voltage* is the threshold value between the power and ground pin.
- When you specify the cell name, the RON and ROFF would be applied only for that cell, and when you specify the cell type, the RON and ROFF would be applied for all the ESD cells of the particular type.

The following is a snippet of the input file:

```
Cell HEADER_SWITCH 10 5 pwr_pin gnd_pin 0.4
```

`-esd_pin_list {`*pin1 pin2 ....pinN*`}`

Specifies the list of ESD pins. This parameter tags the user-specified pin as the ESD pin before running the `analyze_esd_network` command, and ensures that the software traces only this ESD power pin to bump and extracts resistance. The ESD cell pins that are tagged using this parameter are listed in the `.report` file.

`-exclude_tap_region_file` *filename*

Specifies to exclude certain tap points during macro PGV generation. This parameter helps to avoid false IR-drop violations inside macros caused by inadvertently assigning evenly-distributed tap current to certain undesired tie high/low paths or shielding paths at certain via layers. The Spice netlist and models are not mandatory parameters for excluding the tap points in the macro PGV generation flow.

The format of the region file is:

```
REGION X1 Y1 X2 Y2
REGION X3 Y3 X4 Y4
```

where, `REGION` keyword specifies that the coordinates following it is a tap current region. The unit for coordinates is micron.

`-extraction_command_file` *file*

Specifies the file name of an extraction command file.

`-followpins_tap_layer {lowest_lef_pin_layer | highest_lef_pin_layer | all_lef_pin_layers}`

Specifies the LEF metal layer used to generate current taps. This parameter has two arguments:

- `lowest_lef_pin_layer`: The tool generates current taps only on the lowest LEF metal layer. For example, if both METAL1 and METAL2 metal layers exist for the power/ground pin in the cell LEF, the tool uses METAL1 layer as the tap layer.
- `highest_lef_pin_layer`: The tool generates current taps only on the highest LEF metal layer. For example, if both METAL1 and METAL2 metal layers exist for the power/ground pin in the cell LEF, the tool uses METAL2 layer as the tap layer.
- `all_lef_pin_layers`: The tool generates current taps on all LEF metal layers. For example, if both METAL1 and METAL2 metal layers exist for the power/ground pin in the cell LEF, the tool uses both METAL1 and METAL2 layers as the tap layer. This is the default value.

`-followpins_interface_node_layer {lowest_lef_pin_layer | highest_lef_pin_layer | all_lef_pin_layers}`

Specifies the LEF metal layer used to generate interface nodes. This parameter has three arguments:

- `lowest_lef_pin_layer`: The tool generates interface nodes only on the lowest LEF metal layer. For example, if both METAL1 and METAL2 metal layers exist for the power/ground pin in the cell LEF, the tool will only generate the interface nodes on the METAL1 layer.
- `highest_lef_pin_layer`: The tool generates interface nodes only on the highest LEF metal layer. For example, if both METAL1 and METAL2 metal layers exist for the power/ground pin in the cell LEF, the tool will only generate the interface nodes on the METAL2 layer.
- `all_lef_pin_layers`: The tool generates interface nodes on all LEF metal layers. For example, if both METAL1 and METAL2 metal layers exist for the power/ground pin in the cell LEF, the tool generates the interface nodes on both METAL1 and METAL2 layers. This is the default value.

`-generate_bulk_pin_cap_separately {true | false}`

Specifies to generate separate capacitance values for body bias pins in the library report. This parameter allows you to check the bulk pin capacitance based on the Spice simulation. When set to `false`, the software would sum up the capacitance values for the body bias pins and power/ground pins.

Default: `false`

The following is a snippet of the library report:

```
cell_A
            VPP(0.9350)     2.3e-16         EARLY(1 taps) EM(1 taps) IR(1 taps)
            VBB(0.0000)     2.3e-16         EARLY(1 taps) EM(1 taps) IR(1 taps)
            VDD(0.9350)     8.1e-15         EARLY(1 taps) EM(1 taps) IR(1 taps)
            VSS(0.0000)     8.1e-15         EARLY(1 taps) EM(1 taps) IR(1 taps)
Cell Type = STDCELL                    Cell_Status: PASS
```

Here, the blue-highlighted text shows the capacitance values for the `VPP` and `VBB` body bias pins.

`-generate_graybox_data {true | false}`

Controls the generation of gray box data and storage of this data in the cell library. It is recommended to set this parameter to `false` to reduce the size of the PGV significantly (from 10MB to 1MB) without loss of accuracy.

Default: `true`

`-generate_itfnode_x_direction {true | false}`

When the LEF port geometry is bigger than the tap node distance (`set_advanced_pg_library_mode -tap_node_distance`), multiple interface nodes are generated. You can specify this parameter to generate the interface nodes along the X direction.

When set to `false`, interface nodes are not generated along the specified direction.

Default: `true`

`-generate_itfnode_y_direction {true | false}`

When the LEF port geometry is bigger than the tap node distance (`set_advanced_pg_library_mode -tap_node_distance`), multiple interface nodes are generated. You can specify this parameter to generate the interface nodes along the Y direction.

When set to `false`, interface nodes are not generated along the specified direction.

Default: `true`

`-generate_itfnodes_at_via_layer {true | false}`

Specifies to generate interface nodes at via locations in scenarios where the tap current source is a VIA layer.

`-ignore_pg_nets {{cellname netname}+}`

Skips the specified power or ground nets in PGV generation.

`-import_xdspf_list_file filename`

Specifies the name of the xDSPF (third-party netlist) parasitic file that is imported by the Library Characterization engine to create PGDB. You must also specify the mapping file using the `-xdspf_layermap` parameter.

When you specify this parameter, the following parameters are not supported:

- `-xtc_command_file`
- `set_pg_library_mode -spice_subckts`
- `set_pg_library_mode -gds_files`
- `set_pg_library_mode -gds_layermap`
- `-techgen_dir`

`-input_port_value_list {pin1 value1 pin2 value2 ....pinN valueN}`

Specifies the voltage conditions of the signal inputs during power-grid library characterization. A typical usage model of this parameter include setting the SET or RESET signal port to be the same voltage as supply or ground (tie high/low) to obtain the correct model of operation.

The following example illustrates the use model:

```
set_advanced_pg_library_mode -input_port_value_list { RESETVDD 3.3 }
```

`-interface_node_location_file filename`

Specifies a file containing the location of interface nodes where tap currents are to be generated. The specified file includes the cell name, net name, layer name, and location of the nodes. This parameter allows to generate current taps at user-specified locations during PGV generation.

The format of the file is:

```
CELL <cell_name>
NET <net_name>
LAYER <layer_name> (x1,y1),(x2,y2)
```

Example:

```
CELL cellA
NET VDD
LAYER M0 (0,0.15),(0.585,0.177),(0.2,0.15),(0.3,0.177)

NET VSS
LAYER M0 (0.485,0.019)

CELL cellB
NET VDD
LAYER M0 (0.49,0.008),(0.24,0.008)
```

`-lef_pin_short {true | false}`

Enables shorting of the unconnected LEF pins and ports to establish electrical connectivity. When set to true, shorts the interface nodes for all the cells that are being processed.

Default: false

`-lef_layer_ignore_list {lef_layername+}`

Specifies the names of the LEF layers that power-grid library generation is to ignore when it creates the cell library.

`-lef_layer_ignore_list_file filename`

Specifies the name of a file containing a list of the LEF layers that power-grid library generation is to ignore when it creates the cell library. Each line in the file must consist of a single LEF layer name.

The format of the file is:

```
layer1
layer2
...
layern
```

`-lef_pin_short_cell_file filename`

Specifies the name of a file containing a list of cells and the net name, metal layer name, and resistance value for each layer. It is used to short the LEF pins and ports by the user-specified resistance value. When this parameter is specified, only the given list of nets are shorted with the user-defined resistance values. If a resistance value is not specified for a layer, the default resistance value of 0.001 is used.

The format of the file is:

```
CELL <cell_name>
<net_name> <layer_name> <resistance_value>
```

Example:

```
CELL bump_70N_M4
VDD Metal4 12
CELL bump_70N_M5
VDD Metal4 15
VDD Metal5 12
```

`-lef_pin_short_cell_list {cell1 cell2 ...}`

Specifies a list of cells for which the the LEF pins and ports are shorted by the default resistance value of 0.001ohm. When this parameter is specified, all the nets of the given cells are shorted using the resistance value of 0.001ohm.

`-libgen_command_file file`

Specifies the file name of a power-grid library generation command file.

`-macro_parasitic_file filename`

Specifies the name of the SPEF file that provides signal net parasitics while creating a detailed dynamic view in Library Characterization. The SPEF should be extracted in the decouple_rc mode.

Note: A SPEF file is required only when you specify your own netlist in the Netlist-based flow. The SPEF file is a requirement when the netlist does not have signal caps. You should not specify the signal parasitics using the -ultasim_include_file option.

`-marker_layermap filename`

Specifies the GDS layer mapping file containing the list of marker GDS layers that are used for layer operations during XTC processing. Marker layers are layers which do not have corresponding tech layers.

This parameter is used to support extraction of Metal-Insulator-Metal Capacitor (MIMCAP) geometries during power-grid library characterization. The following is the format of the mapping file:

Layer_Type	marker_layername	GDS_keyword	gds_layer_number
metal	CTM_O	gds	77 1
metal	CBM_O	gds	88 1

`-mcpu_rerun_count count`

Specifies to rerun the cells that failed due to machine issues during PGV library generation. This parameter allows you to specify the number of times the failed cells can be resubmitted in the distributed processing mode.

Default: 0

`-pgdb_list_file filename`

Specifies the name of the PGDB header file containing the list of cells for which Library Characterization should generate power-grid views. The syntax of the file is:

CELL *cell_name*

PGDB *pgdb_directory*

The PGDB directory contains the PGDB for all the power and ground nets of the cell. The PGDB directory has cellname_Netname.* files.

When you specify this parameter, the following parameters are not supported:

- `-xtc_command_file`
- `set_pg_library_mode -spice_subckts`
- `set_pg_library_mode -gds_files`
- `set_pg_library_mode -gds_layermap`
- `-techgen_dir`

`-pgdb_layermap` *`filename`*

Specifies the name of the file containing the mapping between the layer name used in the technology file and the layer name used in the PGDB.

`-powergate_characterization_voltages { val1 val2 val3 .... }`

Supports characterization of power gates at multiple voltages. You can specify multiple values of voltages so that the library characterization engine will characterize the power gates at each of the voltages. The software will then be able to get the characterized cell from the cell library for a particular voltage.

`-powergate_multi_enable_characterization_method {enable_all | enable_single}`

Specifies the Ron/Idsat characterization method for power gate cells with multi-enable pins. This parameter allows you to control the generation of RON and Id-Vd curve for each enable pin. The possible characterization methods are:

- `enable_all`: All the enable pins are switched ON for Ron/Idsat characterization. The generated PGV model will contain a single Ron resistor with the total equivalent resistance value. The cell library report displays the equivalent Ron value of all the enable pins. This is the default method.
- `enable_single`: Only one enable pin is switched ON at a time for Ron/Idsat characterization, and the other pins are switched OFF. The generated PGV model will contain total number of Ron resistors that is equal to the number of enable pins. The cell library report displays Ron values separately for each enable pin.

This parameter is supported only for standard cells when the power gate parameters are specified using the `-powergate_parameters_file` parameter. For multi-enable power gate cells, Ileak/Roff are always calculated by keeping all the enable pins OFF.

Default: `enable_all`

Example:

```
set_pg_library_mode ... -powergate_parameters_file ./pg1.txt
set_advanced_pg_library_mode -powergate_multi_enable_characterization_method enable_all
```

`-process_bulk_diffusion_ports {true | false}`

Specifies to create the `DIFF` layer ports of body bias pins in PGV.

By default, library generation ignores ports on the diffusion layer. In order to process body bias pin's ports on the diffusion layer for electrical analysis on body bias network, set this parameter to `true` during library generation.

`-remove_emview_dangling_resistor {true | false}`

Specifies to restore or remove dangling resistors from power-grid views. It is recommended to minimize dangling resistors for optimal analysis of IR drops and accurate Blech length calculation at the top level in EM analysis.

Default: `false`

`-retain_generated_pgv_on_error {true | false}`

Specifies to retain an incomplete cell library created during PGV generation even if the tool exits with a fatal error. When set to `false`, the tool deletes the incomplete cell library. This parameter is recommended for use during initial library creation and debugging.

The default value is `false`.

`-ron_measure_threshold` *`value`*

	Specifies the switched pin voltage during Ron simulation. The specified value will be multiplied by the VDD voltage (<value>*vdd_vol) and applied to the switched pin during Ron simulation. The default value is `0.99`. You can specify a value between 0 and 1.
`-schematic {true | false}`	
	Specifies whether the subcircuit netlist is a schematic netlist.
`-skip_switch_net_extraction {true | false}`	
	Specifies to skip extraction of switched nets inside the GDS file for a fine-grain memory cell. The software will add the capacitance from the switched net to the always-on net. The software will thus consider the cell as a regular memory without switched/power-gated nets. When this parameter is specified, you do not need to specify the port file (`set_advanced_pg_library_mode -add_port_labels`) as the switched nets are not traced. The final PGV will be created with the always-on and VSS nets. The power-gate library report file will contain information about the missing switched net, and the capacitance and current that was contributed by the switched net.
`-spectre_path` *value*	
	Specifies an external Spectre path for PGV generation. This parameter allows you to use a different Spectre version than the one embedded with the software.
`-source_location_file` *filename*	
	Specifies the name of the file containing the locations of the voltage sources to be connected to the pad cells. For more information on the source location file, see Source Location File Format.
`-stdcell_characterization_voltage_value {`*val1 val2 val3* `...}`	

	Specifies capacitance characterization voltages when generating standard cell PGVs containing voltage-dependent capacitance table for the specified cell. This parameter allows you to compute capacitance based on the user-defined voltage values. The `-stdcell_characterization_voltage_value` parameter must be specified with the `set_pg_library_mode -enable_multi_voltage_cap_generation true` parameter.
	If this parameter is not specified with `-enable_multi_voltage_cap_generation`, capacitance is calculated based on the software-derived voltage range.
	When the `-stdcell_characterization_voltage_value` parameter is specified, the library generation engine prints the capacitance values and corresponding voltages in the report file, and the rail analysis engine prints the capacitance used for each instance.
	The following example illustrates the use model of this parameter:
	`set_pg_library_mode -ground_pins VSS -power_pins {VDD 0.9 TVDD 0.9 } -celltype stdcells \`
	`...`
	`-enable_multi_voltage_cap_generation true`
	`set_advanced_pg_library_mode` **`-stdcell_characterization_voltage_value`** `{1.0 1.2 1.5 2.0 3.3}`

`-strict_input_check {true | false}`	
	Specifies that the software should not create any view for the cell if the EM view is not generated. When set to `true`, the software displays an error message if the Spice netlist or GDS is not specified, and gives details about the cell name and the reason for not generating the view. *Default*: false
`-tap_node_distance` *value*	
	Specifies to set the maximum distance between interface nodes when they get generated in power-grid library generation. The default is `50` microns. Library generation creates interface nodes on port geometries in LEF and Extraction, uses them for top level connectivity tracing and also to connect the cell/block onto top level routing. The interface node generation is purely based on port geometries (tap points are placed one per port). For ports greater than `-tap_node_distance`, they are placed at `-tap_node_distance` apart. If the port pin geometry in LEF is bigger than the setting of the `-tap_node_distance` parameter, at least two nodes will get generated per port geometry.
`-techgen_dir` *directory*	
	Specifies the techgen directory.
`-thunder_command_file` *file*	
	Specifies the file name of a thunder command file.
`-trim_metal_layer_map` *file*	

Specifies the name of the file containing the mapping between a metal layer and its corresponding trim layer. When this parameter is specified, the trim layer shapes are automatically removed from the routing layer shapes to correctly extract all the nets in the generated PGV.

File Format:

metal_layer_name trim_layer_name

```
M0 CM0
M1 CM1
M2 CM2
```

`-use_embedded_spectre {true | false}`

When set to `false`, allows you to specify a different Spectre version than the one embedded with the software. Library Generation will automatically pick Spectre from users' path.

Default : `true`

`-use_peak_current_distribution {true | false}`

Default: `false`

Specifies to use the maximum currents of devices in the Spice netlist for current distribution in macros. When set to `true`, enables peak current distribution for faster PGV generation. The default (`false`) behavior is to enable activity propagation based current distribution.

`-verbosity {true | false}`

Controls the amount of information displayed in the console and written to the log files during power-grid library generation.

Default: false

You can use this parameter for detailed troubleshooting and controlling the verbosity level in the log file.

`-via_based_current_config_file filename`

Specifies the name of the configuration file containing tap current details for location-based via current generation. If `-enable_via_based_current_distribution` is set to `true`, this parameter must be specified.

Configuration File Format:

```
#-----------------------------------------------------------------------
CURRENT_UNIT <default_value>
INPUT_SLEW <default_value>
OUTPUT_SLEW <default_value>
MAX_DISTANCE <Threshold distance for snapping via tap if the exact xy location is not found as specified>
DELAY <default_value>
PERIOD <default_value>
INTRINSIC_CAP { power_net cap ground_net cap}
#-----------------------------------------------------------------------
ID     PGPIN_NAME     X     Y     AVG_CURRENT     PEAK_CURRENT
#-----------------------------------------------------------------------
```

Configuration File Example

```
#-----------------------------------------------------------------------
CURRENT_UNIT mA
INPUT_SLEW 20ps
OUTPUT_SLEW 10ps
MAX_DISTANCE 2.5
DELAY 5ps
INTRINSIC_CAP { VDD 40nF VSS 5nF }
#-----------------------------------------------------------------------
ID      PGPIN_NAME     X          Y     AVG_CURRENT     PEAK_CURRENT
#-----------------------------------------------------------------------
PVDD_1   VDD       16.3090   7.6470     50.0373       100.037
PVDD_2   VDD       16.6900   7.6470     50.0373       100.037
PVDD_3   VDD       17.0700   7.6470     50.0373       100.037
...
```

When this feature is enabled, the tool by default generates a report "`dropped_via_location_taps.txt`" in the output directory to give a list of via IDs that were dropped because they were not within the snapping threshold limit. By default, the snapping threshold is zero. It is controlled by "`MAX DISTANCE`" in the configuration file.

`-well_cap_file filename`

	Specifies the name of a file containing a list of cells and the well capacitance value for each.
`-xdspf_layermap` *filename*	
	Specifies the name of the xDSPF mapping file containing layer type and layer stack information from bottom most layer to top most. You must define layers in the stack from top most layers at the top of the stack to bottom most layers at the bottom of the stack. With reference to this layermap stack, library generation will create the PGDB for the layers as defined in the xDSPF layermap file.
`-xtc_command_file` *filename*	
	Specifies the name of the XTC command file.
`-xtc_include_file_for_qdv` *filename*	
	Specifies the file name of an XTC include file used during characterization. For example, you can include XTC commands to ignore GDS cells, to blackbox lower-level hierarchy cells, or to perform some basic GDS layer operations.

Examples

- The following command creates a power-grid library:
  ```
  set_advanced_pg_library_mode \
  -libgen_command_file library_inc.cmd \
  -thunder_command_file simulator.cmd
  generate_pg_library -output output_dir
  ```
- In the following example, power-grid library generation ignores the LEF *metal1* and *metal3* layers when it creates the cell library:
  ```
  set_advanced_pg_library_mode -lef_layer_ignore_list {metal1 metal3}
  ```
- The following line indicates that a file named *nolef.txt* contains a list of the LEF layer names that power-grid library generation should ignore:
  ```
  set_advanced_pg_library_mode -lef_layer_ignore_list_file {nolef.txt}
  ```
- The following commands tags the `VDD` and `TVDD` pins as the ESD pins:
  ```
  set_advanced_pg_library_mode -esd_cells {IOESD} -esd_pin_list {VDD TVDD}
  ```
- The following command specifies the ESD device `dev1` and the xDSPF file `cell1.xdspf` to be used for ESD tagging:
  ```
  set_advanced_pg_library_mode -import_xdspf_list_file cell1.xdspf -esd_cells {IOESD} -esd_device_list {dev1}
  ```
- The following example specifies that FOREIGN statements are to be assumed for cells with no FOREIGN statements and that the FOREIGN cells should be placed at the inverse of the origin:
  ```
  -assume_foreigns true
  -assume_foreigns_mode 1
  ```
- The following example specifies that FOREIGN statements are to be assumed for cells with no FOREIGN statements, and the FOREIGN cells should be placed at (0,0):
  ```
  -assume_foreigns true
  -assume_foreigns_mode 0
  ```

Source Location File Format

The voltage source location file format is as follows:

```
<MIN_SPACING spacing_value>
<DENSITY density_value>
CELL cell_name
<[PORT|DETAILED] {>
NET net_name
<[PORT|DETAILED] {>
lef_layer_name <PORT|DETAILED> x y [x y]
lef_layer_name <PORT|DETAILED> x y [x y]
<}>
<}>
This source location file syntax includes the following parameters:
```

- NET net_name
  These specifications apply to the voltage source locations that follow until Library Generation encounters a new CELL or NET line. You can specify voltage sources as single points or as rectangles. If you specify them as rectangles, Library Generation generates a grid of voltage sources inside these rectangles.
- PORT | DETAILED
  The source location file can specify voltage sources to be used with port or detailed views.

lef_layer_name <PORT|DETAILED> x y [x y]
Lines beginning with a LEF layer name specify the actual location of the voltage sources.

You can specify this location as a single point by providing one set of values for x and y. These x and y coordinates are specified in microns and use the same coordinate system as that used for LEF ports in the LEF file.

Alternatively, you can specify the voltage source locations as a rectangle by giving two sets of x and y locations. In this case, Library Generation attempts to generate a 10-by-10 grid of voltage sources inside this area. From a rectangle specification, it does not create voltage sources closer than 1 micron apart, so if the rectangle is less than about 10 microns on a side, it generates fewer voltage sources.

In the source location file, the location of the voltage sources is specified by using a LEF layer name and location in LEF coordinate space. However, because of the way LibGen uses this data internally, no LEF geometries are actually required to exist at the location where the voltage sources are to be placed. Library Generation uses this data internally with the layout of the block (that is, the GDS data) during detailed power-grid view generation, so it only requires that there be layout (GDS) geometries at the location of the voltage sources. Similarly, the LEF layer name used in the source location file does not actually have to be included in any of the input LEF files; it must be defined only in the layer map used by Library Generation.

This is an example of the contents of the voltage source location file:

```
CELL BUMP
NET vcc
METAL1 11.0 16.0
METAL1 0.0 15.0 22.1 17.5
```

Here is an example file using MIN_SPACING and DENSITY to create a 10 x 10 grid of voltage sources on the VSS net in the detailed view of the NAND1 cell:

```
DENSITY 10
MIN_SPACING 1
CELL BUMP
DETAILED
NET VSS
```

Related Information

- set_pg_library_mode
- generate_pg_library
- "Power-Grid Library Generation" in the *Voltus IC Power Integrity Solution User Guide*

set_pg_library_mode

```
set_pg_library_mode
[-help]
[-add_ploc_cells cell1 cell2 ...]
[-bulk_ground_pins { ground_pin_list }]
[-bulk_power_pins { pin1 voltage1 ... pinN voltageN }]
[-calculate_metal_layer_density {true | false}]
[-cell_decap_file filename ]
[-cell_list_file   filename ]
[-cell_ploc_details filename]
-celltype { techonly | stdcells | macros }
[-current_distribution {propagation | dynamic_simulation trigger_file | current_region region_file}]
[-custom_label label]
[-decap_cells cell_name_list ]
[-default_area_cap value ]
[-default_power_voltage value ]
[-design_qrc_layermap file ]
[-enable_device_size_current_distribution {true | false}]
[-enable_distributed_processing  {true|false} ]
[-enable_multi_voltage_cap_generation {true | false}]
[-exclude_cell_list_file filename]
-extraction_tech_file file
[-filler_cells cell_name_list ]
[-fine_grain_memory_cell {true | false}]
[-gds_files file_list ]
[-gds_layermap file ]
[-generate_port_using_gds mbb_layer_name ]
[-ground_pins { ground_pin_list }]
[-ignore_gds_cells {gds_cell_name_list}]
[-internal_ground_pins ground_pin_list]
[-internal_power_pins power_pin_list]
[-layer_tap_node_distance { {LayerName1 tap_node_distance} {LayerName2 tap_node_distance} ... }]
[-layers_to_be_ignored_during_tap_creation {layername+}]
[-layers_to_have_ports_created {layername+}]
[-lef_layermap file ]
[-liberty_files file_list]
[-macros_config_file   filename]
[-max_metal_density_tile_dimension {maxlength maxwidth}]
[-non_zero_ground_pins {pin1 voltage1 ... pinN voltageN}]
[-pin_shape_current_distribution filename]
[-power_pins { pin1 voltage1 ... pinN voltageN } ]
[-powergate_finegrain_simulation {true|false} ]
[-powergate_parameters   {{ cell supply switched Ron Idsat Ileak Control_Pin_Logic} + } ]
[-powergate_parameters_file  filename]
[-spice_corners corner_list]
[-spice_models file_list]
[-spice_netlist_list_file filename]
[-spice_subckts file_list ]
[-spice_subckts_xyscaling value]
[-stop@via layername ]
[-tap_node_distance_file filename]
[-tech_gds_layermap filename]
[-temperature temp_value_in_degree_celcius ]
[-use_lefonly_for_detailedview_generation {true | false} ]
[-use_powergate_data_from_lib {true|false}]
```

Specifies what kind of power-grid library will be created. This command is used to define the type of input and configuration for generating power-grid libraries. You must specify this command before running the `generate_pg_library` command.

Parameters

`-help`	
	Outputs the command usage.

`-add_ploc_cells` *cell1 cell2 ...*

Specifies the name of the low-dropout (LDO) cell for which a voltage source is to be added. You can specify a list of cells. This parameter is applicable to the LDO cells only, and must be used when creating voltage sources for these cells. When this parameter is specified, all the output pins will be made voltage sources using the highest voltage value of the cell.

The `-add_ploc_cells` parameter is used with the `-cell_ploc_details` parameter.

`-bulk_ground_pins { ground_pin_list }`

Specifies the names of nets that connect transistor bulk terminals to the ground net. This information is used to prevent any current from being assigned to these nets. If a net is connected to any transistor terminals other than a bulk terminal, you must specify that net with the `-ground_pins` parameter.

`-bulk_power_pins { pin1 voltage1 ... pinN voltageN }`

Specifies the names of nets that connect transistor bulk terminals to the power net. This information is used to prevent any current from being assigned to these nets. If a net is connected to any transistor terminals other than a bulk terminal, you must specify that net with the `-power_pins` parameter.

`-calculate_metal_layer_density {true | false}`

Enables metal layer density calculation for macros. The default value is `false`.

When this parameter is set to `true`, Library Generation calculates and stores the area density information for each metal layer in macros for accurate thermal analysis. Metal density for macros is calculated by dividing the macro cell into tiles, and computing the area of polygons in each tile. The calculated metal density values are used in the Power Analysis flow to generate the metal power density map. This power map file is later used as input by Celsius Thermal Solver, the thermal analysis tool for packages and boards.

This parameter is applicable only for macro PGV generation (`-celltype macros`).

`-cell_decap_file` *filename*

Specifies the name of a file containing a list of cells and the decoupling capacitance value for each. You can also specify the series resistance value along with the decoupling capacitance value to accurately model the cell decoupling capacitance. The cell decap file format allows you to specify capacitance for multiple voltage values to support multi-voltage capacitance simulation for standard cell and macro PGV generation.

File Format:

```
UNIT <X>
CELL cell_name
PIN pin_name capacitance resistance

or
UNIT <X>
CELL cell_name
VOLTAGE value1
PIN pin_name capacitance resistance
...
....

VOLTAGE value2
PIN pin_name capacitance resistance
...
...
```

Unit is optional, default is `1e-15`. The *value* for each cell will be multiplied by the unit, except if engineering notation is provided.

Examples:

```
UNIT 1
CELL cell1

VOLTAGE 1.0
PIN VDD  1e-18
PIN VSS  1e-18

VOLTAGE 0.9
PIN VDD  1e-18
PIN VSS  1e-18
```

`-cell_list_file` *filename*

Specifies a file containing a list of cells that will be processed.

The `-cell_list_file` parameter is required when `set_pg_library_mode -celltype macros` is selected. The program spawns the library generation job in serial mode for each macro in the cell list file. A separate library for each macro is generated. This facilitates debugging when library generation of any macro fails. It also helps in improving the overall performance. The following is a sample cell list file:

```
CELL1
CELL2
CELL3
```

`-celltype {techonly | stdcells | macros}`

Specifies the type of library creation.

- `techonly`
  Creates a technology Library that contains the tech file, area capacitance specification, decap/filler/powergate cells, and Tech view for all the cells. A technology library is a requirement for running rail analysis.
- `stdcells`
  Generates a power-grid library for all the standard cells using the Spice netlist. The stdcells power-grid library does not contain the tech file. Power-grid is connected at the LEF pin, and all the capacitance are derived from simulation. The power-grid library contains three kind of power-grid views (PGVs), Early/IR/EM, and the views are selected based on the accuracy mode of rail analysis.
- `macros`
  Generates a power-grid library for memory, I/Os and macros using the Spice and GDS netlist. GDS is mandatory for the cell type macros. The `macros` power-grid library does not contain the tech file. Power-grid is connected down to contact/specified via, and all the capacitance are derived from simulation. The power-grid library contains three kind of PGVs, Early/IR/EM, and the views are selected based on the accuracy mode of rail analysis.

`-cell_ploc_details` *`filename`*

Specifies a file that contains all the information required to add a voltage source at the desired location for the power pad and LDO cells. This file includes the pad cell name, pin name, layer name, and domain name.

The format of the file is:

```
cell_name      pin_name    layer_name  domain_name
PADCELL           VDD       Metal3       VDD
```

- *`pin_name`* is the power/ground name for pad cells, and is the output pin name for LDO cells. The pin name could be represented by using the wildcard character "*" to indicate that voltage sources are to be added to all pins of a power pad/LDO cell.
- *`domain_name`* is the pin name for pad cells. For LDO cells, it is the power or ground name whose voltage needs to be used.

Example:

```
set_pg_library_mode \
    -ground_pins                {VSS 0 ExtVSS 0} \
    -power_pins                 {VDD 1.2 ExtVDD 1.2} \
    -celltype                   techonly \
    ...
    -add_ploc_cells             {CELL1 CELL2 CELL3} \
    -cell_ploc_details          /VoltageSource/config.txt \
```

The following is a snippet of the config.txt file:

```
CELL1 VDD Metal3 VDD
CELL1 VSS Metal2 VSS
CELL1 Z Metal2 VDD
CELL2 Z Metal3 VSS
CELL3 * VDD
```

`-current_distribution {propagation | dynamic_simulation` *`trigger_file`* `| current_region` *`region_file`*`}`

Specifies the method of current distribution inside the cell.

- **Propagation** uses Voltus library simulator to determine the propagation activity to distribute the current.
- **Dynamic simulation** uses a trigger file to run detailed analysis to determine current distribution.
- **Current region** specifies the amount of current to distribute within a cell region. For more information on the format, see Current Region File.

`-custom_label` *`label`*

Saves the specified label in the PGV for easy identification. This label can be used for revision control, label for Spice netlist/model, and internal keywords. It will be stored as “Custom Label” in VERSION.TXT in the .cl

`-decap_cells` *cell_name_list*

Specifies the names of cells that have explicit decoupling capacitance for use in decap optimization. These cells are nearly equivalent to feedthrough cells, except that there is typically an active device between the power and ground nets to provide extra decoupling capacitance. Power-grid view library generation uses this list of cells to ensure that a capacitance value is associated with each cell. If no capacitance is associated with a cell, a warning message will be issued.

This parameter supports wildcards (*).

`-default_area_cap` *value*

Specifies the default amount of area based decoupling capacitance in a cell. The default area-cap is 0.0 (unit is fF per micron-square). You must provide a value for area-based decoupling capacitance.

`-default_power_voltage` *value*

Specifies voltage for the power pins which are present in LEF but not specified using the `-power_pins` parameter. As a result, the `-power_pins` parameter of the command is optional. The default voltage value specified using `-default_power_voltage`is added to all power pins of the cell.

If both the parameters `-power_pins` and `-default_power_voltage` are not specified, the default voltage of 1.2 will be used.

`-design_qrc_layermap` *file*

Default: None

Specifies a file that provides the mapping of the layer names in the LEF design file to the layer names in the Quantus technology file.
File Format:

```
lefdef_layername    technology_layername
M1                   METAL_1
VIA1                 VIA_1

M2                   METAL_2
VIA2                 VIA_2
...
```

`-enable_device_size_current_distribution {true | false}`

Specifies to compute the tap current distribution for a macro based on device sizes (Width, Length), without performing any simulation. This parameter is used for macro PGV generation in a scenario when the Spice netlist is available but the Spice models are not available.

Default: `false`

`-enable_distributed_processing {true|false}`

Specifies to enable parallel processing for the Library Generation flow.

`-enable_multi_voltage_cap_generation {true | false}`

Specifies to compute capacitance values over a range of input voltages, including the nominal voltage. It generates 10 values from 0.1 to 1.3 times the vdd/vss voltage. The default value is `false`.

When set to `true`, it generates a PGV that contains the voltage-dependent capacitance table for the specified cell. This parameter allows you to use the accurate cell capacitance at different supply voltages in power-up analysis and in different voltage domains in dynamic analysis.

`-exclude_cell_list_file` *filename*

Specifies a file containing a list of cells (one per line) which should be excluded from power-grid library generation.

`-extraction_tech_file` *file*

Specifies the name of the technology file that will be used for extraction.

`-filler_cells` *cell_name_list*

List the names of the filler or feedthrough cells that have no GDS data. These cells can be swapped out for decaps during decap optimization.

This parameter supports wildcards (*).

`-fine_grain_memory_cell {true | false}`

Specifies to tag the cell as the fine-grain cell in the PGV library when characterizing the cell, and ensures that the software automatically uses this information to identify the fine-grain memory cells in the rail analysis flow. The default value is `false`.

This parameter is used with the `-internal_power_pins` and `-internal_ground_pins` parameters to specify the list of switch power/ground pins. It is recommended to use these parameters when characterizing fine-grain cells to avoid any potential cell type identification conflict in the rail analysis flow.

`-gds_files` *file_list*

Specifies the names of the GDSII files used during power-grid library generation.

`-gds_layermap` *file*

Specifies a file that provides the mapping of layers between the LEF and GDS files. The following is a sample GDS layermap file:

```
type    name    gds    #
via     VIA9    gds    59
```

`-generate_port_using_gds` *mbb_layer_name*

Identifies the layer name that library generation uses to compute the placement bounding box for GDSII cells that are placed by the DEF file.

`-ground_pins { ground_pin_list }`

Specifies the names of the nets considered ground nets for all cells for which the ground nets are not specified.

`-ignore_gds_cells {`*gds_cell_name_list*`}`

Specifies to ignore the specified cells during macro PGV generation. When this parameter is specified, the Library Generation engine does not extract data for these cells. This parameter supports wildcards.

Example:

The following command does not extract data for the `cell3` cell:

```
set_pg_library_mode -ignore_gds_cells {cell3}
```

`-internal_power_pins` *power_pin_list*

Specifies the list of switch power pins.

`-internal_ground_pins` *ground_pin_list*

Specifies the list of switch ground pins.

`-layer_tap_node_distance { {`*LayerName1 tap_node_distance*`} {`*LayerName2 tap_node_distance*`} ... }`

Default: None

Specifies tap node distances for certain layers. Using this parameter, you can customize the tap node distance for each layer during power-grid library generation.

The tap node distance is in micron. If the tap node distance is specified for a single layer, double brackets {{}} are still required.

`-layers_to_be_ignored_during_tap_creation {`*layername*`+}`

Specifies the names of the layers that library generation is to ignore when it creates the current taps. This parameter can be used to suppress the generation of any nodes or taps on layers that should not be extracted. Some technologies include layers that are only used to influence the extraction of other layers.

`-layers_to_have_ports_created {`*layername*`+}`

Specifies the names of the layers for which ports are to be created. This parameter can be used to create ports for the poly and diffusion layers in the technology file. By default ports are only created on metal layers.

`-lef_layermap` *file*

Specifies a file that provides the mapping of layers between the LEF and technology file. This parameter is optional. Library Generation automatically generates the layermap file using the technology LEF (containing all layer and via definitions) and Quantus Tech (extraction library) files, therefore, a LEF layermap file is not a mandatory input. The automatically generated layermap file is located in the PGV temp directory.

The following is the format and sample content of the auto-generated file (`lefdefLayerMap_AutoGenerated.txt`):

```
Layer Type   tech layername   lefdef    lefdef layername
via              VIA_9        lefdef      VIA9
metal            METAL_1      lefdef      METAL1
```

You can specify this parameter if you want to provide your own layermap file.

`-liberty_files` *file_list*

Specifies to directly read decoupling capacitance and resistance from the specified Liberty file for standard cell and macro PGV generation.

The following is an example of the Liberty construct required for capacitance/resistance:

```
intrinsic_parasitic () {
  when : "I";

  intrinsic_capacitance (VDD) {
    value : 0.000248992;
  }

  intrinsic_resistance (VSS) {
    related_output : ZN;
    value : 1.89195;
  }

  intrinsic_capacitance (VSS) {
    value : 0.000639008;
  }
}
```

If there are multiple `when` conditions, the average intrinsic resistance/capacitance value is taken.

`-macros_config_file` *filename*

Specifies to provide configuration details for each macro cell per Library Generation run. This is an additional input for `macros` cell type wherein you can provide macro specific Spice and GDS. The format of the file is:

```
CELL <cell1>
GDS_FILE <gds_file1>
SUBCKT_FILE <subckt_file1>
DECAP_FILE <decap_file1>
TRIGGER_FILE <trigger_file1>
POWER_GATE_PARAMS supply <supplyName> switched <switchedName>
CELL <cell2>
GDS_FILE <gds_file1>
SUBCKT_FILE <subckt_file1>
```

`CELL` is the keyword to specify a new cell that is to be characterized. This is a mandatory keyword. This keyword is followed by optional keywords, like `GDS_FILE` and `SUBCKT_FILE`, that are specific to the cell.

`-max_metal_density_tile_dimension {`*maxlength maxwidth*`}`

Specifies the tile dimensions, wherein *`maxlength`* denotes maximum length and *`maxwidth`* denotes maximum width in µm. The default value for length or width is 50µm. The number of tiles in the X and Y direction is determined using the length and width dimension.

This parameter should only be used when the `-calculate_metal_layer_density` parameter is set to `true`.

`-non_zero_ground_pins {`*pin1 voltage1 ...pinN voltageN*`}`

Default: None

Specifies the name and voltage of the nets considered ground nets for all cells for which the elevated ground nets need to be specified. This parameter allows you to provide a list of non-zero ground be it positive or negative for the ground nets. If the same name has been specified for both the parameters `-non_zero_ground_pins` and `-ground_pins`, then `-non_zero_ground_pins` will be given priority.

This parameter is applicable only for standard cell and macro PGV generation (`-celltype stdcells | macros`).

Example:

```
set_pg_library_mode -non_zero_ground_pins {VSS -1.05 VSSA 0.08 VSSA1 1.09}
```

`-pin_shape_current_distribution` *filename*

Default: None

Specifies a current distribution file that allows you to state how current should be distributed across multiple pin shapes of power and ground pins. Multiple pin shapes exist for VDD and VSS pins, and current drawn through each pin shape is non-uniform depending upon the output bit that is switching. This parameter allows you to define current distribution for multi-height multi-bit flip-flop (MBFF) characterization. The parameter allows you to accurately model the current through each pin shape.

File Format

```
CELL <cell name>
PGPIN VDD:(<LEF_layer_name>,<LEF_coordinates>):(<LEF_layer_name>,<LEF_coordinates>)
PGPIN VSS:(<LEF_layer_name>,<LEF_coordinates>):(<LEF_layer_name>,<LEF_coordinates>)
PIN <signal_pin_name> <current_ratio_for_VDD> <current_ratio_for_VSS>
```

Here,

- *<LEF_layer_name>,<LEF_coordinates>* specifies to assign the current ratio to the pin shape occupying the coordinates.
- *LEF_layer_name* the layer name to which the current taps are to be created

Example

```
CELL NAND3X8
PGPIN VDD:(Metal1,1,1):(Metal1,4,1)
PGPIN VSS:(Metal1,2,2):(Metal2,5,2):(Metal2,8,2)
PIN Q1 0.00:1.00 0.00:0.00:1.00
PIN Q2 0.90:0.10 0.95:0.05:0.00
```

`-spice_netlist_list_file` *filename*

Specifies a file containing all the spice netlists to be used for library generation, the format is one netlist per line. These spice netlist files must contain the subcircuit definitions for the cells being processed by power-grid library generation.

The following example specifies a file called *spice_netlist_list_file* that contains the list of spice netlist files:

```
set_pg_library_mode -spice_netlist_list_file spice_netlist_list_file
```

`-power_pins { pin1 voltage1 ... pinN voltageN }`

Specifies the name and voltage of the nets considered power nets for all cells for which the power nets are not specified. If both the parameters `-power_pins` and `-default_power_voltage` are not specified, the default voltage of 1.2 will be used.

`-powergate_finegrain_simulation [true|false}`

Specifies to enable finegrain powergate simulation.

`-powergate_parameters {{ cell supply switched Ron Idsat Ileak Control_Pin_Logic} + }`

Specifies the information that technology library generation needs to characterize a powergate cell. It includes the cell name, unswitched power pin, switched power pin, on resistance in ohms, saturation current in milliamps, leakage current in milliamps, and control pin logic.

If ron, idsat, or ileak values are specified, they will be used to override the data extracted by Voltus library simulator. `Control_Pin_Logic` can have the following values

- `enableHigh` - control pin(s) of the control logic will be connected to the power net to turn on the power gate instances.
- `enableLow` - control pin(s) of the control logic will be connected to the ground net to turn on the power gate instances.

The control pin logic allows you to specify whether the enable pin of the specified power gate is active high or active low, thus ensuring that the appropriate voltages are applied to the enable pins.

This parameter also allows you to specify multiple functional modes to enable different combinations of control pins in each mode. For multiple functional modes, the tool calculates RON for each mode separately and generates a single PGV for the specified modes.

Format for Multiple Functional Modes:

```
set_pg_library_mode -powergate_parameters { {cell_name mode_name1 {supply switch Ron Idsat Ileak Control_Pin_Logic} mode_name2
{supply switch Ron Idsat Ileak Control_Pin_Logic}... mode_nameN {supply switch Ron Idsat Ileak Control_Pin_Logic} }
```

Examples:

- For control pin logic:
  ```
  -powergate_parameters {  {HEADER_SWITCH TVDD VDD enableHigh} }
  ```
- For Single Powergate:
  ```
  -powergate_parameters {{HDRSID0 TVDD VDD}}
  ```
- For 2 Header Powergate:
  ```
  -powergate_parameters {{HEAD16_A12TH_C35 VDDG VDD} {HEAD8_A12TH_C35 VDDG VDD}}
  ```
- For Multiple Footer Powergate
  ```
  -powergate_parameters {{FOOTBUFVHSV16 VSSG VSS} {FOOTBUFVHSV32 VSSG VSS} {FOOTBUFVHSV64 VSSG VSS} {FOOTBUFVHSV8 VSSG VSS}}
  ```
- For Multiple Functional Modes:
  ```
  -powergate_parameters { {HEADER_SWITCH S1 {TVDD VDD 750 .5 4.0e-8V} S3 {TVDD1 VDD 350 .3 5.0e-8V} S2 {TVDD VDD 450 .5 4.0e-8V} }
  {RING_SWITCH TVDD VDD 750 .5 4.0e-8} }
  ```

`-powergate_parameters_file` *filename*

Specifies a file that contains details of power switches that have multiple control pins, input pins, and power nets. The control pins, input pins, and power nets can have either sweep voltages or user-defined voltages.

This parameter also allows you to specify multiple functional modes to enable different combinations of control pins in each mode. For multiple functional modes, the tool calculates RON for each mode separately and generates a single PGV for the specified modes.

File Format:

```
CELL <cell_name>
PGATE_PAIR  <AON_PIN> <SW_PIN> <CTRL_PIN> <Polarity|VoltageValues>
PGATE_PAIR  <AON_PIN> <SW_PIN> <CTRL_PIN> <Polarity|VoltageValues>
…
PWR_PIN <net_name> {voltage_values}
PWR_PIN <net_name> {voltage_values}
…
INPUT_PIN <pin_name> <Polarity|VoltageValues>
INPUT_PIN <pin_name> <Polarity|VoltageValues>
…

PGATE_OFF_PIN <CTRL_PIN> <Polarity|VoltageValues>
PGATE_OFF_PIN <CTRL_PIN> <Polarity|VoltageValues>
PGATE_OFF_PIN <CTRL_PIN> <Polarity|VoltageValues>
…

CHARACTERIZATION_VOLTAGES { val1 val2 val3 .... }
END
```

File Format for Multiple Functional Modes:

```
CELL <cell_name>
MODE_NAME mode_name1
PGATE_PAIR <AON_PIN> <SW_PIN> <CTRL_PIN> <Polarity|VoltageValues>
PWR_PIN <net_name> {voltage_values}
INPUT_PIN <pin_name> <Polarity|VoltageValues>
```

```
PGATE_OFF_PIN <CTRL_PIN> <Polarity|VoltageValues>
CHARACTERIZATION_VOLTAGES <VolageValues>
END_MODE

MODE_NAME mode_name2
PGATE_PAIR  <AON_PIN> <SW_PIN> <CTRL_PIN> <Polarity|VoltageValues>
PWR_PIN <net_name> {voltage_values}
INPUT_PIN <pin_name> <Polarity|VoltageValues>
PGATE_OFF_PIN <CTRL_PIN> <Polarity|VoltageValues>
CHARACTERIZATION_VOLTAGES <VolageValues>
END_MODE
...

END
```

- PGATE_PAIR and CHARACTERIZATION_VOLTAGES are mandatory keywords.
- PGATE_PAIR is the keyword to specify the always-on power pin, switched power pin, and control pin names, and the voltage values at which these pins are simulated. If you specify voltage values, then the software uses these user-defined values during power gate characterization. Alternatively, if you specify polarity (high/low), then the software uses the sweep voltage values (CHARACTERIZATION_VOLTAGES) during power gate characterization. If you do not specify both polarity or voltage values, then the software by default uses the sweep voltage values (CHARACTERIZATION_VOLTAGES) during power gate characterization.
- CHARACTERIZATION_VOLTAGES is the keyword to specify sweep voltage values. The number of voltage values specified with the PGATE_PAIR keyword must match with the number of voltage values specified with the CHARACTERIZATION_VOLTAGES keyword.
- PWR_PIN is the keyword to specify additional power pins of the cell other than the always-on and switched power pins.
- INPUT_PIN is the keyword to specify additional signal pins of the cell other than the control pins. If you specify voltage values, then the software uses these user-defined values during power gate characterization. Alternatively, if you specify polarity (high/low), then the software uses the sweep voltage values (CHARACTERIZATION_VOLTAGES) during power gate characterization. If you do not specify both polarity or voltage values, then the software by default uses the sweep voltage values (CHARACTERIZATION_VOLTAGES) during power gate characterization.
- PGATE_OFF_PIN is the keyword to specify the control pin names and the voltage values at which the off-resistance is computed. If you specify voltage values, then the software uses these user-defined values during power gate characterization. Alternatively, if you specify polarity (high/low), then the software uses the sweep voltage values (CHARACTERIZATION_VOLTAGES) during power gate characterization. If you do not specify both polarity or voltage values, then the software by default uses the sweep voltage values (CHARACTERIZATION_VOLTAGES) during power gate characterization.

Examples:

Example 1: User-specified voltage values for control pins, input pins, and power nets

```
CELL cellA
PGATE_PAIR AONVDD SWVDD SLEEPN {0.600 0.660 0.700 0.750 0.800 }
PWR_PIN VDD_PS {0.7 0.8 0.9 1.0 1.1 }
INPUT_PIN PS_DISABLE {0.600 0.660 0.700 0.750 0.800 }
CHARACTERIZATION_VOLTAGES {0.600 0.660 0.700 0.750 0.800 }
END
```

Example 2: User-specified polarity

```
CELL cellb
PGATE_PAIR TVDD VDD SLEEPN
PWR_PIN VDD_SIDE
INPUT_PIN S1 HIGH
CHARACTERIZATION_VOLTAGES {0.600 0.660 0.700 0.750 0.800 }
END
```

Example 3: powergate_param_file.txt

```
CELL HEADER_SWITCH
MODE_NAME S1
PGATE_PAIR TVDD VDD NSLEEPIN LOW
CHARACTERIZATION_VOLTAGES {0.9 1.0 1.1}
END_MODE

MODE_NAME S2
PGATE_PAIR TVDD VDD NSLEEPIN HIGH
CHARACTERIZATION_VOLTAGES {0.9}
END_MODE

MODE_NAME S3
PGATE_PAIR TVDD1 VDD NSLEEPIN LOW
```

```
CHARACTERIZATION_VOLTAGES {0.9 1.1}
END_MODE

END
```

`-spice_corners` *corner_list*

Specifies a list of SPICE corners. This is an optional parameter. If you specify SPICE corner(s), the number of corner items should be equal to the number of SPICE model files specified in `-spice_models` *file_list*. One Spice model can map to many Spice corners but not vice versa.

Examples:

- if there are three model files but only two corners, a blank corner definition is required to match the model-corner pair, as shown below:

  ```
  -spice_models { model1.scs model2.scs model3.scs}
  -spice_corners {{corner1} {} {corner2}}
  ```

- if a SPICE model1 has three corners and model2 has two corners, you should specify:

  ```
  -spice_models { spice_1.model spice_2.model }
  -spice_corners { {s11 s12 s13} {s21 s22} }
  ```

  Based on the above parameter description, the following information gets updated in the library generation file:

  ```
  spice_model_file spice_1.model {s11 s12 s13}
  spice_mode_file spice_2.model {s21 s22}
  ```

`-spice_models` *file_list*

Specifies the names of the SPICE model files used by the SPICE netlist file (`-spice_subckts`).

Some Spice process models use `wnom/Wn` and `lnom/Ln` instead of `w` and `l` (device model parameters names for length and width) for MOS models. Voltus recognizes and supports these alternate parameter names.

`-spice_subckts` *file_list*

Specifies the names of the SPICE netlist files containing the subcircuits corresponding to the cells being processed during power-grid library generation. These subcircuit netlists will be used to calculate pin capacitances using Voltus library simulator.

`-spice_subckts_xyscaling` *value*

Specifies the current region scale factor for the Netlist-based flow . The current region scale factor is the factor by which the netlist coordinates are scaled before finding the closest node to attach the tap currents.

The netlist-based flow fails when there is a mismatch in the units of the PGDB coordinates (in nm) and the coordinates in the SPICE netlist causing improper current tap creation. To match these coordinates, you need to specify the current region scale factor. The default scale factor is `1000` considering that the SPICE netlist coordinate unit is `um`.

`-stop@via` *layername*

Stops the extraction of the power-grid network inside a cell at the specified via layer.

`-tap_node_distance_file` *filename*

Specifies a file containing the distance between tap nodes for the given list of cells. The specified file includes the cell name and tap node distance in micron. The default unit for tap node distance is `micron`, and the default tap node distance is `50 microns`. This parameter is applicable for technology and standard cell PGV generation.

This parameter allows you to customize the tap node distance for each cell during power-grid library generation.

The format of the file is:
cell_name tap_node_distance

Example:

```
OA_cellA 1
OA_cellC 1
```

`-tech_gds_layermap` *filename*

Specifies a file that provides the mapping of the layer names in the Quantus technology file to the layer names in the GDS file.

File Format:

```
technology_layername   use      gds_number  data_type
METAL_1                text       131          0
VIA_1                   x          51          0

METAL_2                text       132          0
VIA_2                   x          52          0
...
```

If `-design_qrc_layermap` is specified along with `-tech_gds_layermap`, then the format of `-tech_gds_layermap` is treated as:

```
lefdef_layername   use      gds_number  data_type
      M1           text       131          0
```

`-temperature` *temp_value_in_degree_celcius*

Specifies the temperature in degree celcius used in library characterization for thermal aware EM/IR analysis.

Default: 25

`-use_lefonly_for_detailedview_generation {true | false}`

Specifies to generate a detailed PGV with the bump LEF without loading a GDS file. You can specify this parameter to perform the RC extraction of a hybrid bump or via cell while skipping the extraction of its top and bottom routing metal layers. When set to `true`, the top and bottom metal layers are preserved as geometries in the library and promoted to the top level, whereas the via layer shapes are dropped and their resistance is used directly from the PGV.

Default: `false`

`-use_powergate_data_from_lib {true|false}`

Specifies to read powergate parameters ron, saturation current, and leakage current from the Liberty files specified using the existing `-liberty_files` parameter. This parameter allows you to obtain the powergate parameters from a Liberty file when a Spice netlist is not available, or when the Liberty file contains the powergate parameters.

Examples

- The following command sets the power library mode:

```
set_pg_library_mode
-extraction_tech_file RCgen.tch
-lef_layermap lefdef.map
-celltype techonly
-power_pins { VDD 1.08 VDDO 1.08 VDDG 1.08 }
-ground_pins { VSS GND VSSG }
-cell_list_file built_cells_list
-temperature -40
-spice_models models/v1d1_usage.l
-spice_corners { ss_lib }
-spice_subckts { typical_max_m40.spice }
-powergate_parameters { { cell1 VDDG VDD } { cell2 VDDG VDD } }
-current_distribution propagation
```

- The following command generates a PGV that contains the voltage dependent capacitance table for the specified cells:

```
set_pg_library_mode -celltype stdcells -cell_list_file cellfile -extraction_tech_file ./../qrcTechFile -lef_layermap
./../lefdef.layermap -temperature 125 -power_pins { vcc 1.26 vccpg 1.26 vccr 1.26 } -ground_pins { vss } -powergate_parameters
{{powercell_pg1 vcc vccpg} {powercell_pg1 vcc vccr} {powercell_pg2 vcc vccpg} {powercell_pg2 vcc vccr}} -spice_corners { tt } -
spice_models spice_model/libspice -spice_subckts { powercell_pg.net powercell_pg2.net } -enable_multi_voltage_cap_generation true
```

The PGV report includes the following information:

```
INFO: PGV contains voltage dependent cap table for this cell
```

Current Region File Format

The following shows the current_region_file format:

```
 UNIT CURRENT default_unit_in_amps
UNIT CAPACITANCE default_unit_in_farads
CELL cell_name
<PROP [FLAT | HIER]>
NET net_name
LAYER layer_name
[ REGION x1 y1 x2 y2 current_value <cap_value> | REGION current_value <cap_value> ]
```

UNIT	Defines the default scale to be used with the current and capacitance values read from the current region file. These scales apply to any value that is not followed by an explicit unit specification. Valid current values are: A (Amps), mA (miliAmps), uA (microAmps), and nA (nanoAmps). Valid capacitance values are: F(Farads), mF (miliFarads), uF (microfarads), nF (nanofarads), pF (picoFarads), and fF (femtoFarads).
CELL *cell_name*	This command specifies the name of the cell. All the following NET, LAYER, and REGION commands will apply to this CELL until another CELL specification is encountered. Wildcards can be used in the cell_name.
<PROP [FLAT | HIER]>	If the keyword FLAT is applied, LibGen will not perform hierarchical tracing for the given net in this cell. If the keyword HIER is presented, LibGen will perform hierarchical tracing for the given net in this cell. This provides the ability for LibGen to automatically specify a current value to all instances of a cell in the design hierarchy. The assumption is that all of these instances of the cell will consume the same power. The default is HIER.
NET *net_name*	This command specifies that all the LAYER and REGION commands following it applies to the net_name. This stays in effect until another NET or CELL specification is encountered. Wildcards can be used in the net_name.
LAYER *layer_name*	This command specifies that all the REGION commands following it apply to the layer_name. This stays in effect until another LAYER, NET, or CELL command is encountered. The layer_name must be a library layer name. If the layer is not found, LibGen will issue an error message and exit. You must provide a valid layer name for LibGen to run. If you specified a valid layer, but LibGen does not find any via in the assigned current region, LibGen will issue a warning message, but the process will still continue and exit successfully. In this case, you may get a 0 current value for the specified cell.
REGION *x1 y1 x2 y2 current_value cap_value*	This command specifies that the current value and optionally the capacitance value to be associated with the rectangular region specified by the bounding box (*x1 y1 x2 y2*). Current taps will be created at the locations of the vias or contacts specified by the most recent LAYER command. These current taps will be attached to the layer that is below the specified via or contact layer. Coordinate values are assumed to be in GDSII coordinate units and will be interpreted according to the scale and precision settings contained in the GDSII file (or spice netlist with device XY location) that contains the cell. The value of the current drawn by each current tap will be 1/n of the value specified for the current region, where n is the number of taps within the current region. The second variation of the REGION command without the bounding box is valid only if there is only one current region. In this case LibGen can automatically uses the bounding box of the cell for the region.

Example of Current Region File

```
UNIT CURRENT uA
UNIT CAPACITANCE fF
CELL an4_80
NET vcc
LAYER CONT
REGION 0 14000 28900 17000 3.0 5.0
NET gnd
LAYER CONT
REGION 1.5e-2mA
```

Related Information

- set_advanced_pg_library_mode
- generate_pg_library
- "Power-Grid Library Generation" in the *Voltus IC Power Integrity Solution User Guide*

trim_pg_library

```
trim_pg_library
[-help]
[-delete_cells cell1 cell2 ...]
[-delete_cells_file file_name]
-library_list_file file_name
[-output directory_name]
-output_library library_name
```

Specifies to remove a list of cells from the specified PGV. This command allows you to trim the size of a cell library by removing unwanted cells, which can reduce the overall time required for extraction.

Parameters

-help	Outputs the command usage.
-delete_cells *cell1 cell2 ...*	
	Default: None Specifies the name of a cell or a list of cells to be removed from the power-grid library.
-delete_cells_file *file_name*	
	Default: None Specifies the name of a file containing the list of cells (one per line) to be removed from the power-grid library. **File Format**: cell1 cell2 cell3
-library_list_file *file_name*	
	Default: None (**Mandatory**) Specifies the name of a file containing the library or libraries (one per line) that are to be trimmed. If multiple libraries are given, Library Generation merges the multiple same-type power-grid libraries into one consolidated power-grid library. **File Format**: PGV1 PGV2 PGV3
-output *directory_name*	
	Default: current working directory Specifies the name of the output directory in which the trimmed power-grid library will be created.
-output_library *library_name*	
	Default: None (**Mandatory**) Specifies the name of the trimmed power-grid library created after removing the specified cells.

Examples

- The following command removes the cell `AND2X2` from the power-grid libraries specified in the `library_list.txt` file and generates one consolidated power-grid library `trimlibrary`:

```
trim_pg_library -library_list_file library_list.txt -output_library trimlibrary -delete_cells AND2X2
```

Related Information

- `merge_pg_library`
- "Power-Grid Library Generation" in the *Voltus User Guide*

validate_pg_library

```
validate_pg_library
[-help]
[-activity value]
-cell cell_name
[-clock {clk1 clk2 ...}]
[-disable_rail_analysis {true | false}]
[-dynamic_analysis_duration value]
[-enable_xp {true | false}]
-extraction_tech_file tech_file_path
[-frequency value]
[-input_pin_set_case_analysis_low {pin1 pin2 ...}]
[-input_pin_set_case_analysis_high {pin1 pin2 ...}]
[-liberty liberty_file ]
[-max_ir_drop value]
[-net_capacitance value]
[-output output_dir]
[-pin_capacitance value]
-power_grid_library_path pgv_path
[-power_grid_library_view view_name]
[-power_pad_size value]
[-rail_analysis_type {static | dynamic}]
[-sdc_file filename]
[-set_power_switching_pattern trigger_file]
[-set_total_power value]
[-step_size value]
[-supply_voltage value]
-tech_lef_file tech_lef_file_path
[-transition_time time]
[-power_include_file filename]
[-rail_include_file_begin filename]
[-rail_include_file_end filename]
[-rail_trigger_switching_pattern pattern]
[-temperature value]
[-twf_path twf_location]
[-user_defined_ploc_file filename]
[-pgv_mode_dynamic_switch_pattern pattern]
```

Specifies to validate a macro PGV. The command takes a macro PGV and Liberty file as input, and performs IR Drop based validation by running static and dynamic power and rail analysis. In this flow, Library Generation will read the input PGV and generate a dummy DEF file that has an instance of the macro (one-cell design). It will then run static/dynamic power and rail analysis on this design. You can analyze the power and rail results to check for hotspots, connectivity, and current distribution inside the PGV.

Parameters

`-help`	
	Outputs the command usage.
`-activity value`	
	Specifies the activity value for input pins. If not specified, a default activity of 0.2 is used. The activity specifies the number of times an input pin switches
`-cell cell_name`	
	Specifies the name of the cell to be validated.
`-clock {clk1 clk2 ...}`	
	Specifies the list of clock pins.
`-disable_rail_analysis {true | false}`	

Specifies to disable rail analysis during macro PGV validation. The following is the behavior of `-disable_rail_analysis true` with the `-enable_xp` para

- if `-enable_xp` is set to `true`, the tool will run only power analysis in the XP mode
- if `-enable_xp` is set to `false`, the tool will enable only power analysis in the non-XP or normal mode

Default: `false`

`-dynamic_analysis_duration value`

Specifies the simulation period that dynamic power analysis will use. It is the duration for which a window is selected for calculation of IR drop during pc default value is 10ns.

`-enable_xp {true | false}`

Specifies to validate the macro PGV in the extensively parallel (XP) mode.

Default: `false`

`-extraction_tech_file tech_file_path`

Specifies the path of the extraction technology file.

`-frequency value`

Specifies the clock frequency. The default value is 100Mhz.

`-input_pin_set_case_analysis_low {pin1 pin2 ...}`

Specifies the list of pins to be kept at a logical low. The `set_case_analysis` command in the SDC file will have the same effect as the set pin to constant described here.

`-input_pin_set_case_analysis_high {pin1 pin2 ...}`

Specifies the list of pins to be kept at a logical high. The `set_case_analysis` command in the SDC file will have the same effect as the set pin to constan described here.

`-liberty liberty_file`

Specifies the name of the Liberty file for power calculation.

This parameter is optional for the static power analysis flow, and is mandatory for the dynamic power analysis flow. In the static flow, you need to specify `set_total_power`. If both the parameters are specified, the `-set_total_power` parameter will be honored.

`-max_ir_drop value`

Specifies the maximum IR drop allowed. Default is 5 (in percentage).

`-net_capacitance value`

Specifies the value of the net capacitance. The default value is 1pF.

`-output output_dir`

Specifies the name of the output directory. This directory saves the power and rail reports, and the design output (such as .lef, .def, and so on).

`-pgv_mode_dynamic_switch_pattern pattern`

Specifies the switching patterns for the functional modes in the dynamic power calculation flow. The use model of this parameter is same as the `set_pov dynamic_switch_pattern` parameter.

In the following example, the mode `write` will switch in the 1st clock cycle, the mode `read` will switch in the 2nd and 4th clock cycles, and none of the mo 3rd clock cycle:

```
validate_pg_library -pgv_mode_dynamic_switch_pattern { write read - read }
```

`-pin_capacitance value`

Specifies the value of the pin capacitance. The default value is 1pF.

`-power_grid_library_path pgv_path`

Specifies the path of the power-grid library file.

`-power_grid_library_view` *view_name*

Specifies the power-grid view name.

`-power_include_file` *filename*

Specifies a power analysis include file. This file specifies certain commands of the dynamic engine that do not have a Voltus equivalent. This parameter the dynamic power analysis flow.

`-power_pad_size` *value*

Specifies the distance between two adjacent DEF voltage sources. The default value is `50` (in Micron).

`-rail_analysis_type {static | dynamic}`

Specifies whether static or dynamic analysis will be done. The default value is `static`.

`-rail_include_file_begin` *filename*

Provides a file that includes additional rail options that will be run prior to the `analyze` command. This can be used for those legacy rail analysis comma Voltus equivalent.

`-rail_include_file_end` *filename*

Provides a file that includes additional rail options that will be run after the `analyze` command. This can be used for those legacy rail analysis command Voltus equivalent.

`-rail_trigger_switching_pattern` *pattern*

Default: None

Specifies a dynamic trigger file. When a Macro EM view with current signature is used in a design, it needs to be determined when this block/cell is trigg trigger file is used to determine this. The use model of this parameter is same as the `set_rail_analysis_mode -dynamic_trigger_file` parameter.

`-sdc_file` *filename*

Specifies the name of the timing constraints file.

`-set_power_switching_pattern` *trigger_file*

Default: None

Specifies the stimulus pattern on the pins of an instance and cell. Specify this parameter only in the dynamic mode for the type instance and cell. The us parameter is same as the `set_power -dynamic_switch_pattern` parameter.

`-set_total_power` *value*

Specifies the total power for the design in Watts. This power is distributed based on cell area.

`-step_size` *value*

Specifies the simulation step size, in picoseconds. The default value is 20ps. This parameter is applicable only to the dynamic power analysis flow.

`-supply_voltage` *value*

Specifies the voltage of each power supply pin. The default value is 1V.

`-tech_lef_file` *tech_lef_file_path*

Specifies the path of the technology LEF file.

`-temperature` *value*

Sets the temperature for static power calculation. The default is 125 degrees.

`-transition_time` *time*

Specifies the transition time for specific pins or instances in the design. This parameter is applicable only to the dynamic power analysis flow.

`-twf_path` *twf_location*

Specifies the location of the Timing Window File (TWF) for noise and power calculation.

`-user_defined_ploc_file` *filename*

Specifies the name of a file containing net names and corresponding user-defined ploc or power pad files (.pploc). The .pploc file includes power pin or locations for power and ground nets. The parameter gives you the flexibility to include either user-specified or tool-generated voltage sources.

The following is a sample file with the list of nets and power pad files:

```
----ploc.list----
#<NET_NAME> <Pointer to ploc file>

VDD   input/ploc/VDD.pploc
VDDA  input/ploc/VDDA.pploc
VDDF  input/ploc/VDDF.pploc
VSS   input/ploc/VSS.pploc
```

When specified, the voltage source location files (.pploc) in the given *ploc.list* file will be used for library generation. If the pploc file is missing for a few file, then library generation will automatically generate `.pploc` for those nets.

Examples

- The following command specifies to validate the cell `cell1` by running static power and rail analysis:

```
validate_pg_library -cell cell1 -liberty 125c.lib -clock {CLKW CLKR} -power_grid_library_path macros_cell.cl -extraction_tech_file caps.tch -tech_lef_file tech.lef -output validate_pg_output_static -rail_analysis_type static
```

- The following command specifies to validate the cell `cell1` by running dynamic power and rail analysis

```
validate_pg_library -cell cell1 -liberty 125c.lib -clock {CLKW CLKR} -power_grid_library_path macros_cell.cl -temperature 122 -extraction_tech_file caps.tch -tech_lef_file tech.lef -output validate_pg_output_dynamic -rail_analysis_type dynamic
```

Related Information

- `check_pg_library`
- "Power-Grid Library Generation" in the *Voltus IC Power Integrity Solution User Guide*

28

Power Planning Commands

- add_target_pg
- addRing
- addSpecialRoute
- addSplitPowerVia
- addStripe
- colorizePowerMesh
- editPowerVia
- get_add_target_pg_mode
- get_reinforce_pg_mode
- getAddRingMode
- getAddStripeMode
- getViaGenMode
- loadSpecialRoute
- reinforce_pg
- route_pg
- saveSpecialRoute
- set_add_target_pg_mode
- set_reinforce_pg_mode
- setAddRingMode
- setAddStripeMode
- setViaGenMode
- trim_pg

add_target_pg

```
   add_target_pg
[-help]
[-area {x1 y1 x2 y2}{x1 y1 x2 y2 x3 y3 x4 y4 ...}]
[-force]
[-instance {list_of_instances}]
-nets {list_of_nets}
-pin_layer layer
[-stack_layer_config {stack_layer_pattern}]
[-stack_layer_config_file filename]
-target_layer {layer1 layer2}
[-uda subclass_string]
```

Creates a PG path for a certain cell or instance. There are several cases where the IR drop hotspots occur only on certain cells, like high driving strength clock buffer cell, and there are some cells wherein secondary PG pins need to be connected with power mesh. So, instead of increasing the density of regular PG mesh, the `add_target_pg` command allocates a dedicated PG path from the top layer low resistance PG stripe to these cells and solves most of the IR-drop hotspot issues. As a result, more resources can be preserved for routing.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `add_target_pg` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man add_target_pg`
`-area {x1 y1 x2 y2}{x1 y1 x2 y2 x3 y3 x4 y4 ...}`	
	Signifies that an instance inside the specified area can be connected.
`-force`	Keeps the added PG even when the connection has a DRC, and adds a DRC marker. When this parameter is not set, the newly added PG will remain DRC clean. In such a case, if the PG pin cannot connect to the target layer with DRC clean, the added PG is deleted and leaves the pin open.
`-instance {list_of_instances}`	Specifies the instances to be connected. Note that only those instances are connected, which belong to the cell defined by the `set_add_target_pg_mode -pins` parameter.
`-nets {list_of_nets}`	Specifies the power/ground nets to be connected.
`-pin_layer layer`	Specifies the layer name of the cell pin to be connected.
`-stack_layer_config {stack_layer_pattern}`	
	Defines the pattern of the stripes on intermediate layers between the `pin` layer and target layer. The value of this parameter is in the `{layer1:n1:s1:w1:p1:g1 layer2:n2:s2:w2:p2;g2 …}` format. Here: • `layer1`, `layer2`, … specifies the layer names. • `n1`, `n2`, … specifies the stapling count in a set. • `w1`, `w2`, … specifies the stapling width. • `s1`, `s2`, … specifies the spacing inside a stapling set. • `p1`, `p2`, … specifies the pitch (set-to-set distance) between the stapling sets. • `g1`, `g2`, … specifies the grid or half grid. Following is an example of the `-stack_layer_config` parameter value: `-stack_layer_config {M3:2:0.12:0.04:0.36 M4:1:0.08:0.08:0.9}`

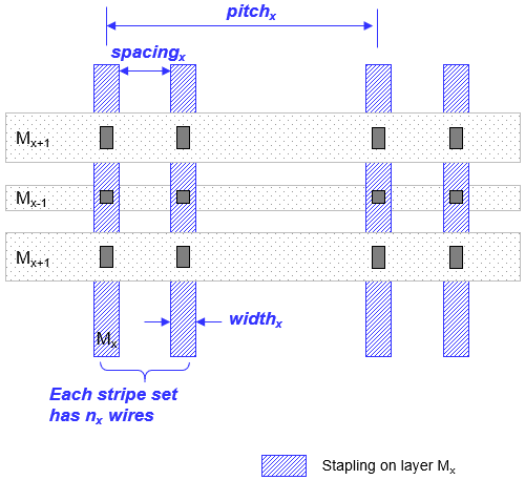
`-stack_layer_config_file filename`	Specifies the stack layer configuration file defining the pattern of the stripes on intermediate layer between the pin and target layers. Stack layer denotes the intermediate layer(s) between the cell pin and target stripe. Following is the syntax of the stack layer configuration file:
	`STACKLAYER COUNT SPACING WIDTH PITCH SNAP`
	`…`
	`<M_x> <n_x> <spacing_x> <width_x> <pitch_x> <Grid/Half_Grid/->`
	`…`
	`END STACKLAYER`
	Here:
	• `STACKLAYER`: Specifies the LEF name of the layer.
	• `COUNT`: Specifies the number of stripes on the layer. The default `COUNT` value is `1`.
	• `WIDTH`: Specifies the stripe width in microns. The default `WIDTH` value is the minimum width in LEF.
	• `SPACING`: Specifies the stripe spacing in microns. The default `SPACING` value is the minimum spacing in LEF.
	• `PITCH`: Specifies the set-to-set distance in a stripe. It indicates the range that one stripe set can cover. The default `PITCH` value is `MAX`, which means one set.
	• `SNAP`: Controls the wires' snap to grid or half grid. The default value of `SNAP` is `Grid`.
	`COUNT`, `WIDTH`, `SPACING`, or `SNAP` can be omitted in the configuration file by specifying a

hyphen (-). For example, `2` stripes on `M4` with minimum width and spacing can be described as:

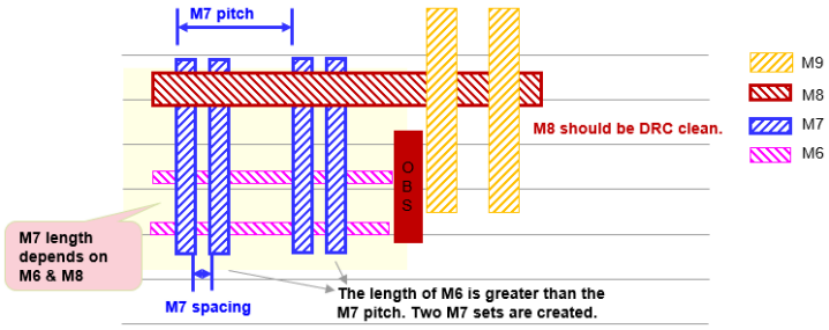
```
M4 2 - -
```

If the entire line of a layer is omitted, it means that the layer will have one stripe with minimum width and spacing.

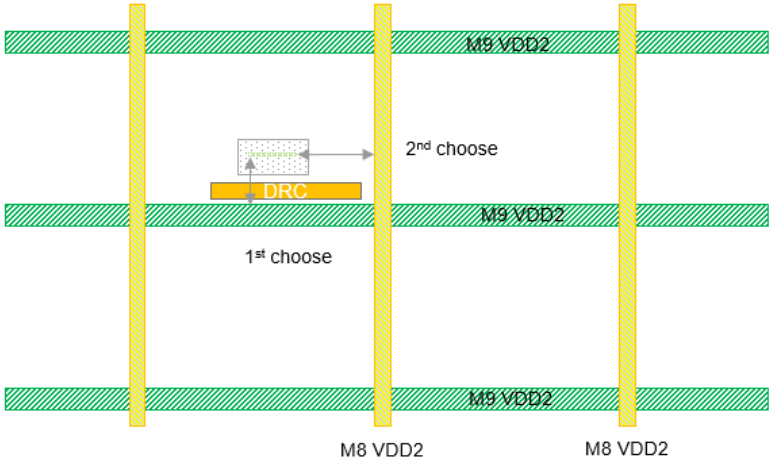
Following is an example of a pattern defined in the stack layer configuration file:



The generation of each layer stapling depends only on its lower and upper stapling. The added PG can be shifted to find a DRC clean location. If a DRC clean location is not found, the added PG of this pin is removed and kept open. The length of the `Mx` layer depends on location of the `Mx-1` and `Mx+1` layers, as shown in the figure below. The number of wires created on the `Mx-1` and `Mx+1` layers needs to fulfill the defined count number criteria.



Note: This parameter is exclusive to the `-stack_layer_config` parameter.

`-target_layer {layer1 layer2}`	Defines one or two destination power/ground stripes by specifying the target layer names. In case two target layers are specified, those should be adjacent. In such a case, the software connects to the closet wires of these two adjacent target layers with DRC clean. The directions of these two target layers are orthogonal. The configuration file is created based on the higher of these two layers. The lower target layer uses its own configuration file and the configuration files of the layers between the cell pin.
	For example, the `stack.cfg` configuration file for a stack is created based on the target layer `M9`. Its target layer `M8` uses the configuration files of the layers from `M3` to `M7`, as shown below:
	`add_target_pg -nets {VDD2} -target_layer {M9 M8} -pin_layer M2 -stack_layer_config stack.cfg`
	Following are the contents of the `stack.cfg` configuration file:
	`STACKLAYER COUNT SPACING WIDTH PITCH`
	`M3 2 0.12 0.04 0.36`
	`M4 2 0.12 0.04 0.9`
	`M5 - - 0.04 0.36`
	`M6 2 0.12 0.04 0.9`
	`M7 2 0.2 0.2 1.2`
	`M8 - 0.2 0.2 1.2`
	`END_STACK_LAYER_CONFIG`
	Following is the output of the above command:
	 
	As shown in the above output, since the first layer is not DRC clean, the software searches for the second closest target stripe to connect.

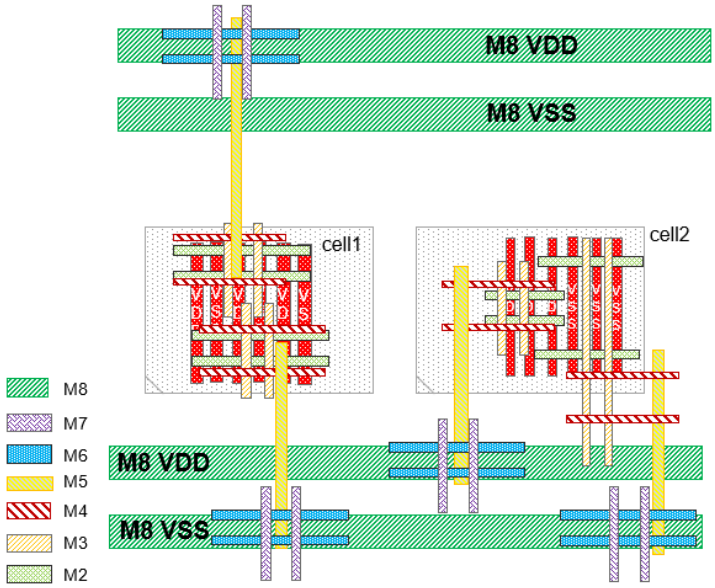
`-uda subclass_string`	Indicates the name string of the user-defined attribute subcalss of wires to route.

Examples

- The following command connects different cells and pins specifying the `-target_layer` as `M8` and `-pin_layer` as `M1`:
  `set_add_target_pg_mode -pins {cell1:VDD cell1:VSS cell2:VDD cell2:VSS} -share_resource false`
  `add_target_pg -nets {VDD VSS} -target_layer M8 -pin_layer M1 -stack_layer_config_file target_pg.config`
  Following are the `target_pg.config` file contents:

```
STACKLAYER    COUNT         SPACING           WIDTH
M2            2             0.06              0.04
M3            2             0.12              -
M4            2             0.12              0.04
M5            -             -                 0.04
M6            2             0.12              0.04
M7            2             0.2               0.2
END STACKLAYER
```

Following is the output of this example:



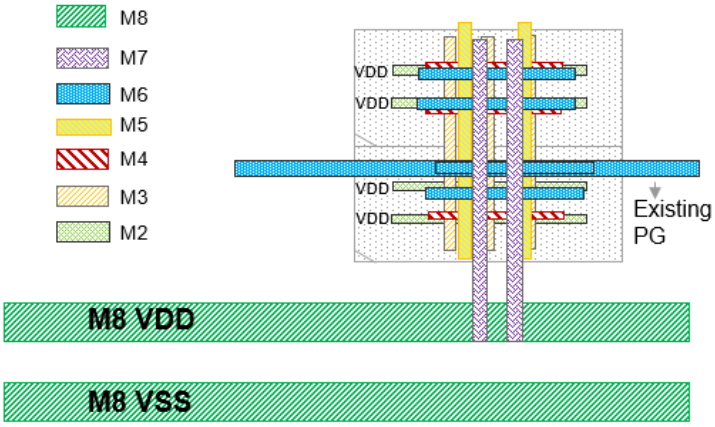
- The following set of commands connect different cells and pins specifying the `-target_layer` as `M8` and `-pin_layer` as `M2`:

```
set_add_target_pg_mode -pins {cell:VDD} -share_resource true
add_target_pg -nets {VDD} -target_layer M8 -pin_layer M2 -stack_layer_config_file  target_pg.config
```

Following are the `target_pg.config` file contents:

```
STACKLAYER    COUNT         SPACING       WIDTH         PITCH
M3            3             0.12          -             -
M4            2             0.12          0.04          0.5
M5            2             0.12          0.04          -
M6            2             -             0.08          0.5
M7            2             0.2           0.2           -
END STACKLAYER
```

Following is the output of this example:



Related Information

- get_add_target_pg_mode
- set_add_target_pg_mode

addRing

```
  addRing
[-help]
[-around {each_block each_reef power_domain default_power_domain selected cluster shared_cluster user_defined}]
[-center {0 | 1}]
[-exclude_selected {0 | 1}]
[-extend_corner {tl tr bl br lt lb rt rb}]
[-follow {core io}]
[-jog_distance real_value]
{-nets name-or-value}
[-offset_adjustment {automatic fixed}][-rectangle {0 | 1}]
[-skip_side {top bottom left right}]
[-snap_wire_center_to_grid {None Grid Half_Grid Either}]
[-threshold {value | auto}]
[-type {core_rings block_rings}]
[-uda subclass_string]
[-use_interleaving_wire_group {0 | 1}]
[-use_wire_group {0 | 1}]
[-use_wire_group_bits value]
[-use_wire_group_reinforcement {0 | 1}]
[-use_wire_group_reinforcement_group_via {0 | 1}]
[-use_wire_group_reinforcement_spacing_width {spacing width}]
[-user_defined_region {x1 y1 x2 y2 ...}]
{-layer {layer | {top top_layer bottom bottom_layer left left_layer right right_layer}} } {-width {value | {top top_width bottom
bottom_width left left_width right right_width}} } {-spacing {value | {top top_spacing bottom bottom_spacing left left_spacing right
right_spacing}} }
[-offset {value | {top top_offset bottom bottom_offset left left_offset right right_offset}}]
```

Creates rings for specified nets around the core boundary or selected blocks and groups of core rows. Use this command after creating an initial floorplan.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `addRing` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man addRing`
`-around`	Specifies the objects around which to create rings. This parameter is used in conjunction with the `-type` parameter. Specify one of the following: • `each_block|each_reef|power_domain`: Creates rings around each power domain or around each fence that surrounds a soft block in the design. This option can only be used if you also specify `-type block_rings`. • `default_power_domain`: Generates ring around a default power domain. • `selected|cluster|shared_cluster`: Creates rings around all selected blocks with shared edges residing in the middle of the common areas between adjacent blocks. This option can only be used if you also specify `-type block_rings`. • `user_defined`: Creates rings around user-defined blocks.
`-center {0 | 1}`	Specifies whether to center the core rings between the I/O pads and core boundaries. If you do not specify this parameter with a value of 1, you must specify the parameter `-offset {value | {top top_offset bottom bottom_offset left left_offset right right_offset}`. *Default:* `0`

`-exclude_selected {0 | 1}`	Specifies whether the core ring should be created so that selected objects (blocks and power domains) are not included within its boundary. *Default:* `0`
`-extend_corner {[tl] [tr] [bl] [bt] [lt] [lb] [rt] [rb]}`	
	Extends the specified segment of the ring to the specified direction. Following are the parameter values: • `bl`: Extends the bottom segment of the ring to the left until encountering a target. • `br`: Extends the bottom segment of the ring to the right until encountering a target. • `lb`: Extends the left segment of the ring to the bottom until encountering a target. • `lt`: Extends the left segment of the ring to the top until encountering a target. • `rb`: Extends the right segment of the ring to the bottom until encountering a target. • `rt`: Extends the right segment of the ring to the top until encountering a target. • `tl`: Extends the top segment of the ring to the left until encountering a target. • `tr`: Extends the top segment of the ring to the right until encountering a target.
`-follow {core | io}`	Specifies whether core rings are placed along the core boundary or the I/O boundary. This parameter is used only with the `-type core_rings` parameter. *Default:* `core`
`-jog_distance value`	Specifies the least amount of jog allowed (to follow the contour of the referenced object) before a jog is removed. This value is in micrometers. If the ring would need to jog a distance equal to or less than this value in order to follow the contour of the object, the ring will not jog. *Default:* If you do not specify this parameter, the jog distance value is automatically calculated based on the worst spacing in the technology file.
`-layer {layer | {top top_layer bottom bottom_layer left left_layer right right_layer}}`	

	Can now be used to replace old options: `-layer_top / - layer_bottom / - layer_left / -layer_right` `-layer metal7` means all four sides of the ring will use layer metal7. **Example:** • `-layer metal7` : all four sides of the ring will use layer metal7. • `-layer {top metal7 bottom metal7 left metal8 right metal8}`: top and bottom ring wires will be generated on metal7, left and right ring wires will be on metal8. The following types of formulas are supported: 1. Lefname 2. Metal layer number **Example:** If Metal2 is named by 'MET2' in LEF file, then you can use 1. MET2 2. 2 to identify metal2.
`-nets` *list_of_nets*	Specifies the names of the nets for which power rings are to be created. You must enclose the list of net names within curly braces.
`-offset_adjustment` *automatic* | *fixed*	
	Specifies whether the offset is automatically adjusted when you change the width or spacing values. *Default:* `automatic`
`-offset {value | {top top_offset bottom bottom_offset left left_offset right right_offset}}`	
	Can now be used to replace old options: `-offset_top / -offset_bottom / -offset_left / -offset_right` `-offset 5` means all four sides of the ring will use 5um offset. Example: • `-offset 5`: 5 means all four sides of the ring will use 5um offset. (existing) • `-offset {top 5 bottom 6 left 7 right 8}`: top ring wires will use 5um offset, bottom ring wires will use 6 um offset, left ring wires will use 7 um offset, right ring wires will use 8 um offset.
`-rectangle {0 | 1}`	If set to `1`, creates rectangular rings only. *Default:* If you do not specify this parameter or if you specify it with a value of `0`, the creation of rectilinear rings is permitted.
`-skip_side`	Can now be used to replace old options: `-top / -bottom / -left / -right` `-skip_side {left}` has is the same as `-left 0` in the old behavior.
`-snap_wire_center_to_grid {`*None* | *Grid* | *Half_Grid* | *Either*`}`	

	Controls the snapping of the center of the rings. If you do not specify this parameter, rings are not snapped to the routing grid. Specify one of the following values: • `None`: Does no snapping of the wires. • `Grid`: Snaps the centers of the wires to the routing grid of the same layer as the wire. • `Half_Grid`: Snaps the centers of the wires to the half-routing grid. The half-routing grid is a virtual grid with the tracks at the center of every two adjacent tracks of the routing grid. • `Either`: Snaps the centers of the wires to the closer of the routing grid or half-grid track.
`-spacing` *spacing*	Can now be used to replace old options: `-spacing _top / -spacing _bottom / -spacing _left / -spacing _right` `-spacing 1` means all four sides of the ring will use 1um for the spacing. Example: • `-spacing 1` : all four sides of the ring will use 1um for the spacing. (existing) • `-spacing {top 1 bottom 1 left 2 right 2}` : top ring wires will use 1um for the spacing, bottom ring wires will use 1 um for the spacing, left ring wires will use 2 um for the spacing, right ring wires will use 2 um for the spacing.
`-threshold {`*value* `|` *auto*`}`	• `value:` Specifies the least amount of spacing allowed between ring segments of adjacent blocks before the rings are merged. If the spacing is less than this value, the common sides of the rings are merged. Units in micrometers. • `auto:` Specifies that the Innovus will automatically determine where to merge the wires being created with the prerouted wires. *Default:* 10 µm (if you do not specify this parameter, rings farther than 10 µm from each other are not merged).
`-type` *core_rings* `|` *block_rings*	
	Specifies the ring type to create. *Default:* If you do not specify this parameter, core rings are created. Specify one of the following values: Creates core rings that follow the contour of the core boundary or the I/O boundary. If you specify `core_rings`, then you must specify either the `-follow` parameter or the `-around user_defined` parameter. *Default:* The ring is offset from the core, based on offset values that you specify in the `-offset` parameters. `block_rings` Creates block rings that follow the contour of the specified objects. If you specify `block_rings`, then you must specify the `-around` argument with one of its required options.
`-uda` *subclass_string*	Displays the user defined attribute attached to each created ring segment.
`-user_defined_region {`*x1 y1 x2 y2 …*`}`	

	Specifies the coordinates for a rectilinear region if you use the `-around user_defined` parameter. You must specify at least four sets of coordinates and the coordinate list must be enclosed within curly braces. You must also specify the coordinates in a linear sequence. For example, you cannot specify the coordinates in a sequence such as bottom left, top right, bottom right, top left.
`-use_interleaving_wire_group {0 | 1}`	
	If set to 1, connects multiple wires from the same net together, even if separated by wires from a different net. You can only specify this parameter if you also specify `-use_wire_group 1`. *Default:* If you do not specify this parameter or if you specify it with a value of `0`, interleaving wire groups are not used.
`-use_wire_group {0 | 1}`	If set to `1`, connects multiple wires from the same net together. **Note:** For stripes to connect to rings using wire groups, you must specify `-useWireGroup` in both the `addRing` and `addStripe` commands. The following figure shows the difference in the connections. *Default:* If you do not specify this parameter or if you specify it with a value of `0`, wire groups are not used.
`-use_wire_group_bits` *integer*	
	Specifies how many nets are to be included in the wire group. You can only specify this parameter if you also specify `-use_wire_group 1`. *Default:* The same number of nets as specified in the `-nets` parameter.
`-use_wire_group_reinforcement {0 | 1}`	
	Specifies reinforcement stripes. The option is boolean and optional. *Default:* 0
`-use_wire_group_reinforcement_group_via {0 | 1}`	
	Creates group vias for wire group corners along with reinforcement stripes placed in the horizontal and vertical directions for the same wires. You can use this parameter to improve the yield and reliability issues associated with vias and reinforcement stripes by ensuring that if one of the vias or reinforcement stripes produces a void, the contact will be maintained through one of the adjoining vias or reinforcement stripes. *Default:* `0`
`-use_wire_group_reinforcement_spacing_width` *spacing width*	
	Creates group vias for wire group corners along with reinforcement stripes placed in the spacing and width for the same wires. You can use this parameter to improve the yield and reliability issues associated with vias and reinforcement stripes by ensuring that if one of the vias or reinforcement stripes produces a void, the contact will be maintained through one of the adjoining vias or reinforcement stripes.

`-width width`	Can now be used to replace old options: `-width_top / -width_bottom / -width_left / -width_right` `-width 4` means that four sides of the ring will use 4um metal width. Example: • `-width 4` : all four sides of the ring will use 4um metal width. (existing) • `-width {top 4 bottom 4 left 6 right 6}` : top ring wires will use 4um metal width, bottom ring wires will use 4um metal width, left ring wires will use 6um metal width, right ring wires will use 6um metal width.

Examples

- The following command adds a core ring to the `vdd` and `gnd` nets on layers `METAL5` and `METAL6`:
  ```
  addRing -nets {vdd gnd} -type core_rings -center 1 -layer {top METAL6 bottom METAL6 right METAL5 left METAL5} -width 10 -spacing 2
  ```
  This ring is centered between the I/O pad area and the core boundary, has a width of 10 µm, and is located at least 2 µm away from any other ring.
- A user may use `addRing -nets {VSS VDD} -layer {top metal5 bottom metal5 left metal6 right metal6} …` instead of `addRing -nets {VSS VDD} -layer {top {metal5 metal5} bottom {metal5 metal5} left {metal6 metal6} right {metal6 metal6}} …`
- Per Old behavior:
  ```
  addRing -nets {VSS VDD} -width_bottom 5 -width_top 5 -width_right 5 -width_left 5 -layer_top Metal5 -layer_bottom Metal5 -layer_left Metal6 -layer_right Metal6 -offset_top 2 -offset_bottom 2 -offset_left 2 -offset_right 2 -spacing_top 1 -spacing_bottom 1 -spacing_left 1 -spacing_right 1
  ```
- Per New behavior:
  ```
  addRing -nets {VSS VDD} -width 5 -layer {top Metal5 left Metal6 bottom Metal5 right metal6} -offset 2 -spacing 1
  ```

Related Information

- `getAddRingMode`
- `setAddRingMode`
- Power Planning and Routing chapter in the *Innovus User Guide*

addSpecialRoute

```
addSpecialRoute
[-help]
filename
```

Incrementally adds special route information from a file. This includes only special route wires, vias, and metal fill. If the specified filename is compressed (with the `.gz` extension), the file is read in directly. Use this command after you have imported a design.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `addSpecialRoute` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man addSpecialRoute`.
`filename`	Specifies the name of the file that contains special route information, to be used as input for the design. Only `.spr` files are supported.

Related Information

- `loadSpecialRoute`
- `saveSpecialRoute`
- To check how undo and redo works with this wire editing command, see `undo` and `redo` in the *Text Command Reference*.
- To review global controls that affect wire editing, see `setEditMode` and `getEditMode` in the *Text Command Reference*.
- Power Planning and Routing chapter in the *Innovus User Guide*.
- For more help on editing wires, see the Editing Wires chapter of *User Guide*.

addSplitPowerVia

```
addSplitPowerVia
[-help]
[-delete | -wires_only]
[-edgeLength {layer1 value1 layer2 value2 ...}]
[-forceCount]
[-localCheckerBoard]
[-master cell_name]
[-maxViaSize {x y}]
[-maxWireLength {layer1 value1 layer2 value2 ...}]
[-maxWireSpacing {layer1 value1 layer2 value2 ...}]
[-mergeCount {x y}]
[-mergeDistance {x y}]
[-minEnclosure]
[-minViaSize {x y}]
[-minWireLength {layer1 value1 layer2 value2 ...}]
[-minWireSpacing {layer1 value1 layer2 value2 ...}]
[-pinLayer {layer1 layer2 ...}]
[-prerouteAlign {none samelayer alllayer}]
[-snap_wire_center_to_grid {layer1 {none|grid|half_grid|either} layer2 {none|grid|half_grid|either}} ...]
[-stackedCheckerBoard]
[-uda <userDefinedAttribute>]
[-wireCount {layer1 value1|auto layer2 value2|auto … }]
[-wireOffset {layer1 value1 layer2 value2 ...}]
[-wireWidth {layer1 value1 layer2 value2 ...}]
```

Adds or deletes Jenga vias and does not modify vias. It passes on all Jenga parameters to separate options.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `addSplitPowerVia` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man addSplitPowerVia`.
`-delete`	Deletes the via for selected wires/objects. This is a boolean value and is optional.
`-edgeLength {layer1 value1 layer2 value2 ...}`	
	Sets extension length of the wires for each layer in um. The value is a string and is optional.
`-forceCount`	Forces the tool to generate the given wirecount even if the wires extend beyond the overlap area. This is a boolean value and is optional.
`-localCheckerBoard`	Allows local checkerboard cut generation instead of matching the global checkerboard pattern.
`-master cell_name`	Passes the selected wires through the specified cells.
`-maxViaSize {x y}`	Displays the maximum via size. This value is in points and is optional.
`-maxWireLength {layer1 value1 layer2 value2 ...}`	
	Sets the maximum length of the wires for each layer in um. This value is a string and is optional.
`-maxWireSpacing {layer1 value1 layer2 value2 ...}`	
	Sets the maximum wire spacing of wires for each layer in um. The value is in strings and is optional.
`-mergeCount {x y}`	Merges the neighboring wire count. The value is an integer and is optional.
`-mergeDistance {x y}`	
	Merges the distance in x and y direction. The value is in points and is optional.
`-minEnclosure`	Creates the shortest wires that meet DRC rules rather than the default wire length of the entire overlap area. This is a boolean value and is optional.
`-minViaSize {x y}`	Sets the minimum via size. The value is in point and is optional.
`-minWireSpacing {layer1 value1 layer2 value2 ...}`	
	Sets the minimum wire spacing of wires for each layer in um. The value is a string and is optional.
`-minWireLength {layer1 value1 layer2 value2 ...}`	
	Sets the minimum length of the wires for each layer in um.The value is a string and is optional.

`-snap_wire_center_to_grid {layer1 {none|grid|half_grid|either} layer2 {none|grid|half_grid|either}}`	
	Specifies the wire center alignment style for middle layer jenga wires. If you do not specify this parameter, stripes are not snapped to the routing grid. Specify one of the following options: • `none`: Does no snapping of the wires. • `grid`: Snaps the center of the wire to the routing grid of the same layer as the wire. • `half_grid`: Snaps the center of the wire to the half-routing grid. The half-routing grid is a virtual grid with the tracks at the center of every two adjacent tracks of the routing grid. • `either`: Snaps the center of the wire to the closer of the routing grid or half-grid track.
`-wireCount {layer1 value1|auto layer2 value2|auto … }`	
	Sets the count of wires for each layer in integer or 'auto'. If the value of a layer is not set, the tool takes '2' as default. If `-wireCount {Mx auto}` is specified, the wire count on Mx will be max wire count calculated by intersection area, wire width and wire spacing. The 'auto' setting is mostly used for adding split power vias between paralleled wires. The value is a string and is optional.
`-wireOffset {layer1 value1 layer2 value2 ...}`	
	Sets the wire offset from the intersection edge of the selected wires in floating value. It also defines the left edge for vertical and bottom edges for horizontal middle layer split wires. The value is a string and is optional.
`-wireWidth {layer1 value1 layer2 value2 ...}`	
	Sets the width of the wires for each layer in um. The value is a string and is optional.
`-pinLayer {layer1 layer2...}`	
	Lists pin layers in a macros used to create via with crossover wire.
`-prerouteAlign {none|samelayer|alllayer}`	
	Defines the alignment of via wires with preexisting wires on the same net. The value is numerical and is optional.
`-stackedCheckerBoard`	Creates stacked checker board vias between intermediate layers.
`-uda`	Adds the user-defined attribute to the to-be-built split vias and wires.
`-wires_only`	Generates only those wires, which are without violations. By default, the command generates both wires and vias. Prior to implementing this option, if any via was not dropped because of violation, the tool was not generating any wires.

Examples

Note: The command will add split power via after the user selects the wires which need to add the split power via (else, no split power via will be added).

- The command will add split power via and merge wires which are within 1.2 for both x and y direction
  ```
  addSplitPowerVia -mergeDistance 1.2
  ```
- The command will add split power via and merge wires which are within 1.2 for x direction and 1.5 for y direction.
  ```
  addSplitPowerVia -mergeDistance { 1.2 1.5 }
  ```
- The command will add split power via and merge wires of count 2 in both x and y direction.
  ```
  addSplitPowerVia -mergeCount 2
  ```

- The command will add split power via and merge wires of count 2 in x direction and count of 4 in y direction.
  ```
  addSplitPowerVia -mergeCount { 2 4}
  ```
- The command will add split power via and create wire lengths to a max of 1.3 in both x and y direction for all intermediate layers within a crossover b/w wire and a wire or b/w wire and a pin.
  ```
  addSplitPowerVia -maxWireLength 1.3
  ```

Related Information

- `editPowerVia`
- Power Planning and Routing chapter in the *Innovus User Guide.*

addStripe

```
addStripe
[-help] [-all_blocks {0 | 1}] [-area {{x1 y1 x2 y2}{x1 y1 x2 y2 x3 y3 x4 y4 ...} ...}]
[-area_blockage {{x1 y1 x2 y2}{x1 y1 x2 y2 x3 y3 x4 y4 ...} ...}]
[-between_bumps {0 | 1}] [-block_ring_bottom_layer_limit layer]
[-block_ring_top_layer_limit layer] [-create_pins {0 | 1}]
[-direction {horizontal | vertical}]
[-extend_to {design_boundary | first_padring | last_padring | all_domains}]
[-insts instance_name]
[-layer layer]
[-master master_cell]
[-max_same_layer_jog_length real_value]
[-merge_stripes_value {auto | value}]
[-narrow_channel {0 | 1}]
[-nets {list_of_nets}]
[-number_of_sets integer_value]
[-over_bumps {0 | 1}]
[-over_physical_pins {0 | 1}]
[-over_pins {0 | 1}] [-over_power_domain {0 | 1}]
[-padcore_ring_bottom_layer_limit layer]
[-padcore_ring_top_layer_limit layer] [-pin_layer layer]
[-pin_offset real_value]
[-pin_width {min_value max_value}]
[-power_domains domain_name]
[-report_cut_stripe filename]
[-same_layer_target_only {0 | 1}]
[-set_to_set_distance real_value]
[-snap_wire_center_to_grid {None |Grid | Mask1_Grid | Mask2_Grid | Half_Grid | Either} [-allow_snapping_override_custom_spacing {0 | 1}]]
[-spacing name_or_value] [-start real_value]
[-stapling {length | auto {offset layer1_pitch:n | reference_layer1 [reference_layer2]}}]
[-start_from {left | right | bottom | top}]
[-start_offset real_value] [-stop real_value]
[-stop_offset real_value] [-switch_layer_over_obs {0 | 1}]
[-uda subclass_string] [-use_interleaving_wire_group {0 | 1}]
[-use_wire_group {-1 | 0 | 1}]
[-use_wire_group_bits integer_value]
[-via_rows integer]
[-via_columns integer]
[-width name_or_value]
```

Creates power stripes within the specified area. If the router encounters an obstruction, the stripe connects to the last stripe on the same net. Otherwise, the stripes stop at the core row boundary. Use this command after creating an initial floorplan.

Adding Stripes in Multi-Threading Mode

You can generate stripes in multi-threading mode by using the following command before `addStripe`:
`setMultiCpuUsage`

When the specified CPU number is bigger than the estimated CPU number, `addStripe` uses the estimated CPU number for multi-threading. Otherwise, it uses the specified CPU number.

Estimated CPU number = (free memory – 512 MB)/(added memory*2)

If any of the following options are specified, `addStripe` will internally disable the multi-threading mode:

- `{-over_bumps {0 | 1} | -between_bumps {0 | 1}} | -over_pins {0 | 1} -pin_layer layer {-master master_cells | -all_blocks}}`
- UDM global variable `pwStripeRouteOverRowsOnly` is equal to 1

`addStripe` supports the new `setMultiCpuUsage` command use model.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `addStripe` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man addStripe`
`-all_blocks {0 | 1}`	Specifies that stripes should be generated over all blocks. *Default:* `0`
`-allow_snapping_override_custom_spacing {0 | 1}`	When set to `1`, it allows snapping of stripes overriding the specified spacing to the closest track. If the closest track has conflicts, the stripe is removed. When set to `0`, it snaps the stripes in the first set to the inward direction, and the other stripes to the closest track. This parameter keeps the actual spacing greater than or equal to the specified spacing. *Default:* `0`
`-area {{x1 y1 x2 y2}{x1 y1 x2 y2 x3 y3 x4 y4 ...} ...}`	
	Specifies the coordinates of the design area in which the stripes must be created. The `-x_left_offset` and `-x_right_offset` values are measured from this boundary area for vertical stripes, and the `-y_top_offset` and `-y_bottom_offset` values are measured from this boundary area for horizontal stripes. **Note:** • If a core ring exists in the design, the stripes automatically connect to the core ring if you do not use the `-area` parameter. You can also create stripes in an area defined by a polygon (rectilinear). For example: `addStripe -area {100 100 100 200 150 200 150 150 200 150 200 100}` *Default:* If you do not specify this parameter, stripes are created for the entire core box. • `-area` and `-extend_to design_boundary` are mutually exclusive. The following error is issued on them being used together: *You cannot specify stripes over design boundary and set an area constraint. Instead, specify area to cover design boundary.* • `-area` and `-extend_to first_padring/ last_padring` are mutually exclusive. The following error is issued on them being used together: *You cannot specify stripes over padring and set an area constraint. Instead, specify area to cover padring.* • When this parameter is used with `setAddStripeMode -domain_offset_from_core true`, the `-start_offset` value for specified area is measured from core on using `addStripe -area`.
`-area_blockage {{x1 y1 x2 y2}{x1 y1 x2 y2 x3 y3 x4 y4 ...} ...}`	

	Specifies the blockage coordinates to restrict the addition of any stripes to this region.
	This parameter supports rectangular and rectilinear area_blockages. Their spacing is 0 during stripe generation.
	Examples
	• Rectangular area_blockage:
	`-area_blockage {100 100 200 200}`
	• Rectilinear area_blockage:
	`-area_blockage {100 100 100 200 150 200 150 150 200 150 200 100}`
	You can add multiple blockages at the same time.
	Rectangular areas or rectilinear area or their mixture can be specified. For multiple areas, each of them must be inside brace.
	Examples
	• multiple rectangles
	`-area_blockage {{x1 y1 x2 y2} {a1 b1 a2 b2} ...}`
	• multiple rectilinear area
	`-area_blockage {{x1 y1 x2 y2 ... } {a1 b1 a2 b2 a3 b3 ...} ...}`
	• mixture of them
	`-area_blockage {{x1 y1 x2 y2} {a1 b1 a2 b2} ... {x1 y1 x2 y2 x3 y3 ...} {a1 b1 a2 b2 a3 b3 ...} ...}`
	The specified areas that are not included inside brace will be ignored. For example
	`-area_blockage {{x1 y1 x2 y2} a1 b1 a2 b2 a3 b3 ... {c1 d1 c2 d2 c3 d3 ...}}`
	`{x1 y1 x2 y2}` and `{c1 d1 c2 d2 c3 d3 ...}` will be used, and `a1 b1 a2 b2 a3 b3 ...` are ignored

`-between_bumps {0 | 1}`	Specifies that stripes should be generated between bumps. *Default*: `0`
`-block_ring_bottom_layer_limit` *layer_name*	
	Specifies the lowest layer that stripes can switch to when encountering a block ring. Stripes that encounter a block ring can only switch to this layer or higher.
`-block_ring_top_layer_limit` *layer_name*	
	Specifies the highest layer that stripes can switch to when encountering a block ring. Stripes that encounter a block ring can only switch to this layer or lower.
`-create_pins {0 | 1}`	If set to `1`, IO pins are created for each stripe. Any overlap with existing IO pins at the design boundary is flagged as a warning after the stripes are created. If set to `0`, pins are not created. By default, `addStripe` would create pins for any stripe touching the design boundary. When `-extend_to design_boundary` is used the option `-create_pins` is 1 by default.
`-direction {horizontal | vertical}`	
	Sets the stripe direction. *Default:* If you do not specify this parameter, the new generated stripe follows the preferred direction defined in the `tlef` file.

`-extend_to`	The boundary for the stripes. Specify one of the following values:
	`all_domains`
	Stripes are created over all power domains for each power and ground net within each power domain. Stripes start and stop at each power domain boundary or ring around the power domain, and ignore the values in the `-nets` parameter.
	`first_padring`
	Stripes are created within the area enclosed by the pad ring closest to the core area.
	`last_padring`
	Stripes are created within the area enclosed by the pad ring closest to the design boundary.
	`design_boundary`
	Stripes are created within the design boundary

`-insts` *instance_name*	Specifies the instance name for generating power stripes.
`-layer` *layer*	Specifies the name of the layer in which to create stripes. Stripes can be created on only one layer at a time. The following types of formulas are supported: 1. Lefname 2. Metal layer number **Example:** If Metal2 is named by 'MET2' in LEF file, then you can use 1. MET2 2. 2 to identify metal2.
`-master` *master_cells*	Specifies the instances of the standard cell (or block) masters over which stripes should be generated. The option supports regular expression: `?` matches any single characters and `*` matches any sequence of characters.
`-max_same_layer_jog_length` *value*	
	The maximum length, in micrometers, that a stripe can jog on the same layer before switching to an adjacent layer. For example, if you specify a value of 2 micrometers, and an obstruction causes a stripe on layer METAL4 to jog for 3 micrometers, the jog will occur on layer METAL3 or METAL5. However, if an obstruction causes the stripe to jog for only 1 micrometer, the jog occurs on layer METAL4. *Default:* `2` (If you do not specify this parameter, the stripe automatically switches to an adjacent layer if the jog length is two times the width of the stripe.)
`-merge_stripes_value` *value*	
	Merges a stripe with a nearby block ring if the threshold spacing between the stripe and ring is smaller than the *value* (in micrometers) specified with this parameter. The stripe jogs to connect to the nearby ring only on the stripe layer. **Note:** To prevent stripes from merging with block rings, set the value of this parameter to `– 1`. *Default:* 10 (If you do not specify this parameter, stripes do not jog to merge with rings that are more than 10 µm away.) To merge the stripe on all layers, set the following variable of the `setAddStripeMode` command: `-merge_with_all_layers 1`
`-narrow_channel {0 | 1}`	Generates stripes over narrow channel area. Narrow channel usually exists between two blocks/power domains, or between core boundary and blocks/power domains. If there are part of rows without a certain net and layer of power mesh crossing over, the area of neighboring part of rows is defined as a narrow channel for that net and layer. When adding overall stripes,

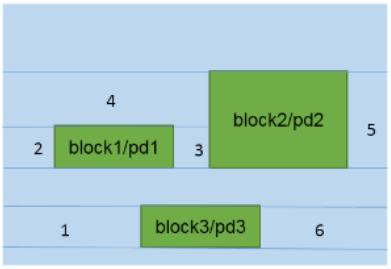
some narrow channels might be missed because of start offset and set to set distance. In this circumstance, adding stripes in narrow channel can help to supply power to stdcells in that channel. The recommended flow is to add overall stripes first and then to add stripes in narrow channel, if needed.

> ⚠ When the `-narrow_channel` parameter is used with `setAddStripeMode -extend_to_closest_target stripe`, the narrow channel stripes are extended to connect the closest followpin and stripes.

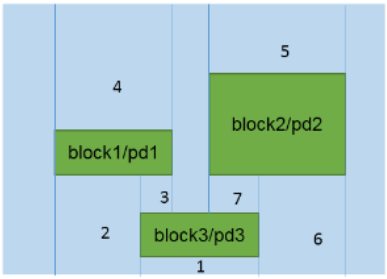
Default : `0`

Examples of narrow channels:

Narrow potential channels for vertical stripes

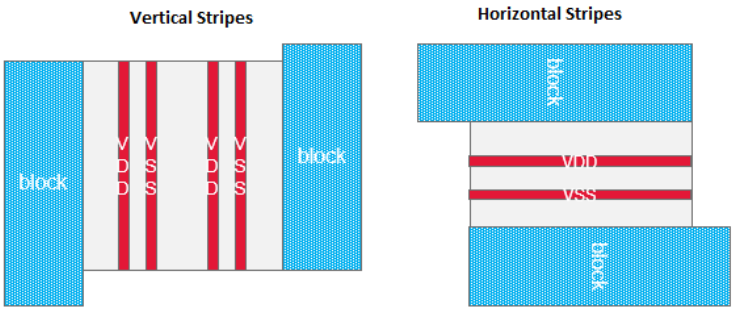


Narrow potential channels for horizontal stripes



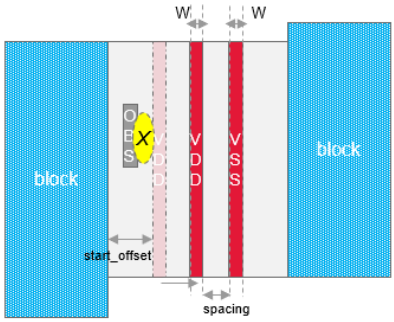
Note: This parameter can be used with the `-power_domains` and `-over_power_domain` parameters.

The `-narrow_channel` parameter generates stripes with uniform channel width in both the vertical and horizontal directions, if the `-number_of_sets` parameter is set to an integer value greater than `1`. The added wires are created at the channel evenly, as shown below:



If the `-start_offset` parameter is used for specifying the starting offset value before the above setting of the `-narrow_channel` parameter, and the specified offset location has a DRC, the added wire shifts to the left or up to the closed DRC clean location, as shown below:

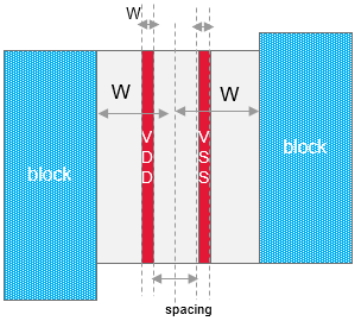
```
addStripe -width $W -spacing $spacing -number_of_sets N -narrow_channel 1 -nets
{$net1 $net2 ...} ...
```



If the `-start_offset` parameter is not used before the above setting of the `-narrow_channel` parameter, the wires are added to the center of the channel. With this setting:

- When the `-number_of_sets` parameter is set to `1`, this center has the same distance from the channel boundaries, as shown below:

  ```
  addStripe -width $W -spacing $spacing -number_of_sets 1 -narrow_channel 1 -nets
  {$net1 $net2 ...} ...
  ```

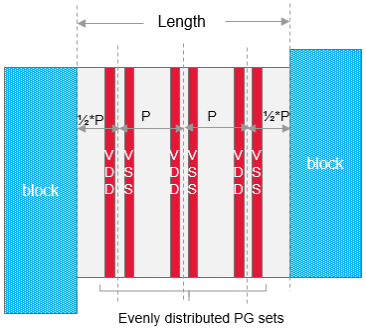


- When the `-number_of_sets` parameter is set to an integer value (`N`) greater than `1`, `addStripe` calculates the pitch to evenly distribute the PG in the channel.

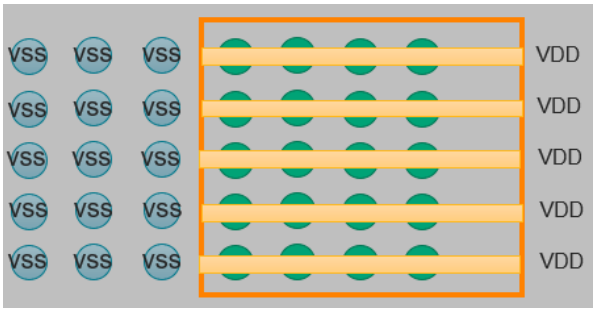
  ```
  Pitch: P = Length / N
  ```

  Additionally, the two PG stripes set at the edge of the channel are at the same distance from the middle of the PG set and the channel boundary is at `P/2`, as shown below:

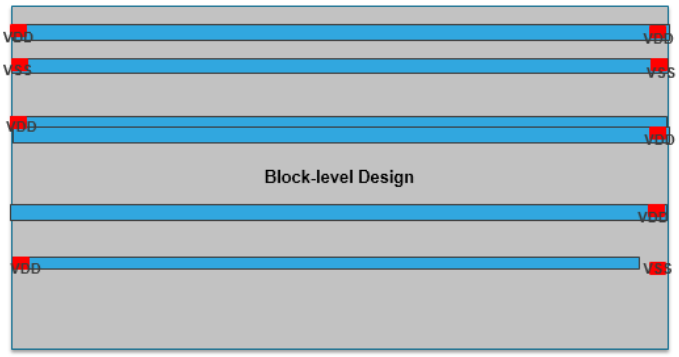
  ```
  addStripe -width $W -spacing $spacing -number_of_sets $N -narrow_channel 1 -nets
  {$net1 $net2 ...} ...
  ```



`-nets {list_of_nets}`	Specifies the names of the nets for which stripes are to be created. The number of net names determines the number of stripes within each set of stripes. The first net in the name list is created first and corresponds to the left or bottom stripe of the set. Separate multiple net names with a space and enclose the list in curly braces, for example, `-nets {net1 net2 net3 net3}`. Braces are not required for one name.
`-number_of_sets integer`	Specifies the number of stripe sets to be created. You must specify either this parameter or the `-set_to_set_distance` parameter. *Default:* If you do not specify this parameter, the number of sets is derived from the `-set_to_set_distance` parameter.

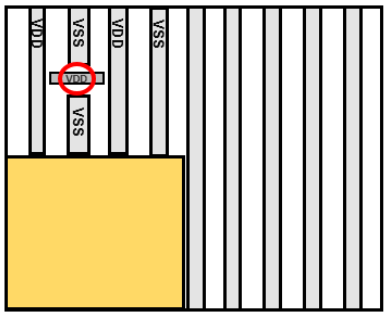
`-over_bumps {0 | 1}`	Specifies that stripes should be generated over bumps. When used with `-area`, it extends stripes to the boundary of the specified area.
	Run `setAddStripeMode -extend_to_closest_target area_boundary` before generating stripes over bumps using the `addStripe -over_bumps` parameter. See the example below:
	`setAddStripeMode -extend_to_closest_target area_boundary`
	`addStripe -net VDD -direction horizontal -area {x1 y1 x2 y2} -over bumps 1 …`
	Following is the output of the above commands:
	 
	
	Default: `0`

`-over_physical_pins {0 | 1}`	

	Adds stripes over the physical pins to detect the physical PG terminals and add stripes to continue them. Following is the use model of this option:
	`addStripe -over_physical_pins 1 -nets {net_names} -width {pin_width|width_value} -direction {vertical|horizontal} [-pin_layer Mx] [-pin_offset offset_value]`
	While using this option:
	◦ The tool generates one stripe on one physical pin separately, except for the following special cases:
	▪ When two same net pins (left and right) are totally aligned. In this case, one stripe between two pins is generated.
	▪ When two different net pins are partially or totally aligned. In this case, one stripe on left/bottom pin is generated and break at right/up pin with DRC clean distance.
	◦ `-width pin_width` makes stripe width same as that of related physical P/G pin.
	◦ When `offset_value` in `-pin_offset offset_value` is greater than zero, stripes shift up for left/right pins and right for up/down pins.
	◦ If both `-over_physical_pins` and `-over_pin` are specified, `-over_physical_pins` is ignored.
	In the following example, stripes are added over the physical pins in the horizontal direction for the `vdd` and vss nets:
	`addStripe -over_physical_pins 1 -nets {VDD VSS} -width 10 -direction horizontal -pin_layer metal11`
	Following is the block-level design of the output:
	 

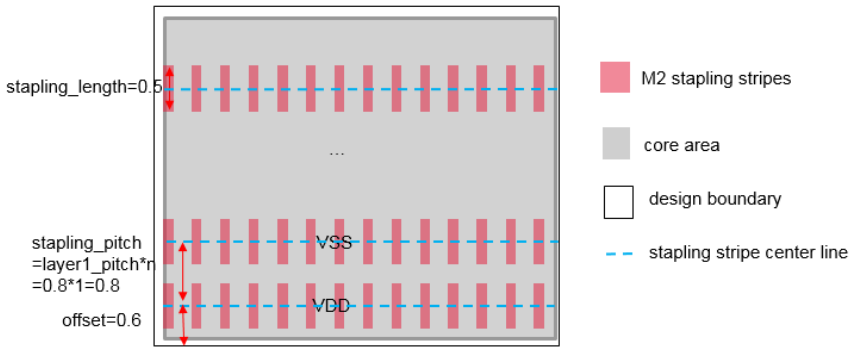
`-over_pins {0 | 1}`	Specifies that stripes should be generated over power/ground pins in a standard cell or block. *Default*: `0` **Note:** With this option, user could define value "pin_width" in option "-width", then tool will generate stripe according to pin width automatically.
`-over_power_domain {0 | 1}`	

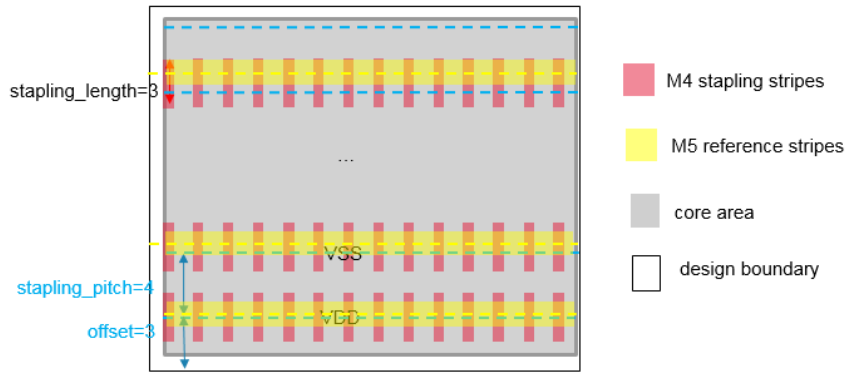
	If set to `1`, adds stripes only within power domain boundaries or the fence around a soft block. For power domains, these stripes start and stop at the power ring that surrounds the selected power domain boundary. For fences, these stripes start and stop at the fence. If both a power domain and a fence are selected, the power domain takes precedence.If you do not specify this parameter or if you specify it with a value of `0`, stripes are prevented from extending to the power domain boundary. **Note:** • When this option is specified, you can use `-start_from {bottom | top | left | right} and -start_offset` parameters to set boundaries, but you cannot use the `-start` and `-stop` parameters to specify an absolute location. • In addition, the software issues a warning message if the specified power and ground nets do not match the power domain's power or ground nets. • When `addStripe -over_power_domain 1` is used with `setAddStripeMode -domain_offset_from_core true`, the `-start_offset` value for the selected power domain/block is measured from core on using `addStripe -over_power_domain 1`. *Default:* `0`
`-padcore_ring_bottom_layer_limit` *layer_name*	
	Specifies the lowest layer that stripes can switch to when encountering a pad or core ring. Stripes that encounter a pad or core ring can only switch to this layer or higher.
`-padcore_ring_top_layer_limit` *layer_name*	
	Specifies the highest layer that stripes can switch to when encountering a pad or core ring. Stripes that encounter a pad or core ring can only switch to this layer or lower.
`-pin_layer` *layer*	Specifies the layer on which the stripe is generated. **Note:** The layer you specify with this parameter is the same as the layer you specify with the `-layer` parameter. *Default*: `TOP`
`-pin_offset`	Offsets the stripe a specified distance from the pin edge to make well tap cell connections. The offset is from the left edge of the stripe to the left edge of the same net pin. This can be a negative number and is in user units.
`-pin_width {`*min_value max_value*`}`	
	Specifies the range of minimum and maximum pin width for `-over_pins` or `-over_physical_pins`.
`-power_domains` *domain_name*	
	Specifies the power domain name for generating power stripes. **Note:** This parameter cannot be used with the `-narrow_channel` parameter. Use the `-area` parameter to generate `-narrow_channel` stripes over certain areas.
`-report_cut_stripe` *filename*	

	Saves the information related to a cut stripe in a DRC file. This helps in identifying the power stripe that has been removed/cut from the design to avoid any DRC violation following user settings.
	You can use the following command to load the generated DRC file into the design and view the cut stripe mark in the GUI:
	`loadDrc *.drc`
	The following example shows how the information related to a cut stripe is stored in the a.drc file:
	`addStripe -report_cut_stripe a.drc`
	The following figure depicts that the place highlighted in red circle will be marked, because here the VSS stripe is cut to avoid short violation to VDD segment:
	 

`-same_layer_target_only`	Specifies connection to overlapping targets on the same layer only.
`-set_to_set_distance` *real*	
	Specifies the distance (pitch) from the reference stripe of one set to the reference stripe of the next set. The value of this parameter must be a positive number specified in micrometers. You must specify either this parameter or the `-number_of_sets` parameter. *Default:* If you do not specify this parameter, the distance between sets is derived from the `-number_of_sets` parameter.
`-snap_wire_center_to_grid {None | Grid | Mask1_Grid | Mask2_Grid | Half_Grid | Either}`	
	Controls the snapping of the center of the stripes. If you do not specify this parameter, stripes are not snapped to the routing grid. **Note:** When tech LEF specifies `RIGHTWAYONGRIDONLY` on a specific layer, stripes are automatically snapped to the routing grid. Specify one of the following options: • `None`: Does no snapping of the wires. • `Grid`: Snaps the center of the wire to the routing grid of the same layer as the wire. • `Mask1_Grid`: Snaps the center of the wire to the routing track of `Mask1`. • `Mask2_Grid`: Snaps the center of the wire to the routing track of `Mask2` . • `Half_Grid`: Snaps the center of the wire to the half-routing grid. The half-routing grid is a virtual grid with the tracks at the center of every two adjacent tracks of the routing grid. • `Either`: Snaps the center of the wire to the closer of the routing grid or half-grid track. *Default:* None
`-spacing` *spacing*	Specifies the width of the gap between stripes in each set. You can specify different spacing between each pair of stripes. For *n* number of stripes, you can specify *n* –1 spacing values. If fewer values are specified, the last-specified spacing is used for the remaining unmatched stripe pairs.
`-stapling {length | auto {offset layer1_pitch:n | reference_layer1 [reference_layer2]}}`	

Turns on the stapling stripe generation feature and controls the stapling pattern. In the syntax:

- `length`: Specifies the length of stapling stripes.
- `auto`: When specified, stapling length is calculated by the tool automatically, with the minimum length that can be DRC clean.
- `offset`: Specifies the distance from design boundary to the center of the first stapling stripe by default. Refer to the following example:
  Example 1
  ```
  addStripe -stapling {0.5 0.6 0.8:1} -nets {VDD VSS} -layer metal2 -width 0.1 -
  direction vertical -set_to_set_distance 0.5
  ```
  

- `pitch:n`: Specifies the center to center distance between two stapling stripes in end direction (refer to **Example 1** above). It is defined by `n` times of the track pitch on `layer1`, where stripes are needed to be connected by stapling stripes.
- `reference_layer1`: When the stripes on `layer1` to be connected already exist, you can directly specify the reference layer 1 name. It specifies that the stapling stripe center is on the center line of the reference stripes (offset value is the center offset of first bottom/left stripe; the `pitch:n` is the set-to-set distance of stripes on `layer1`).
- `reference_layer2`: When the stripes on `layer2` to be connected already exist, you can specify the reference layer 2 name, so that the stapling stripes are generated referring to both `layer1` and `layer2` and connect them. When `reference_layer2` is specified, the real offset and stapling pitch will be the average value from `layer1` and `layer2` settings. It aims to find the center of two layers of wires to be connected by stapling stripes. Refer to the example below:
  Example 2
  ```
  set metal3_track_offset 3
  set metal3_pitch 0.8
  addStripe -stapling '3 $metal3_track_offset $metal3_pitch:5 metal5' -nets {VDD
  VSS} -layer metal4 -width 0.5 -direction vertical -set_to_set_distance 2
  ```
  

You can use `-stapling` in the following ways:

- `-stapling {length|auto offset pitch:n}`
- `-stapling {length|auto reference_layer}`

	• `-stapling {length|auto offset pitch:n reference_layer2}` • `-stapling {length|auto reference_layer1 reference_layer2}`
`-start` *value*	The start coordinate for adding stripes.
`-start_from {bottom | top | left | right}`	
	For horizontal stripes: • `bottom` indicates that stripes should be generated from bottom to top, taking the bottom offset into account. • `top` indicates that stripes should be generated from top to bottom, taking the top offset into account. *Default*: `bottom` For vertical stripes: • `left` indicates that stripes should be generated from left to right, taking the left offset into account. • `right` indicates that stripes should be generated from right to left, taking the right offset into account. *Default*: `left` **Note:** Offset values set with this parameter are considered "hard": The power planning software adheres strictly to the values. However, the opposite values are considered "soft": Such values could be greater than the hard values, but will never be less than the hard values. For example, if you set a `right` offset value of 4 µm, the `left` value will be at least 4 µm, but could be greater.
`-start_offset` *real_value*	
	Specifies the starting offset value off of the appropriate boundary of the stripe generation area to the nearest edge of the first stripe. For example, for vertical stripes starting from the left, offset is measured from the left side of the stripe generation area to the left edge of the first stripe. This is an optional field. **Note:** The parameters `-start` and `-start_offset` are exclusive, and cannot be used together.
`-stop`	The stop coordinate for the region for adding the stripe.
`-stop_offset` *real_value*	
	Specifies the stopping offset value off of the appropriate boundary of the stripe generation area. For example, for vertical stripes starting from the left, the offset is measured from the right side of the stripe generation area. This is an optional field. **Note:** The parameters `-stop` and `-stop_offset` are exclusive, and cannot be used together.
`-switch_layer_over_obs {0 | 1}`	
	Specifies if the switch layer is placed over obstructions. It controls whether a stripe switches layers, is routed over blocks, and then continues on the original layer. Using this parameter eliminates the need for a power routing mesh layer, and avoids maximum via stack rule violations. *Default*: 0 *Type: Boolean, optional*
`-uda` *subclass_string*	Displays the user defined attribute attached to each created stripe segment.
`-use_interleaving_wire_group {0 | 1}`	

	If set to 1, connects multiple wires from the same net together, even if separated by wires from a different net. You can only specify this parameter if you also specify `-use_wire_group 1`. If you do not specify this parameter or if you specify it with a value of `0`, interleaving wire groups are not used. *Default:* `0`
`-use_wire_group {-1 | 0 | 1}`	
	If set to `1`, connects wires from the same net together. If you do not specify this parameter or if you specify it with a value of `0`, wire groups are not used. If set to `-1` and multiple same net are specified in -nets, the wire group property will not be updated or created. **Note:** For stripes to connect to rings using wire groups, you must specify the `-useWireGroup` parameter in both the `addRing` and `addStripe` commands. *Default:* `0`
`-use_wire_group_bits` *integer*	
	Specifies how many nets are to be included in the wire group. You can only specify this parameter if you also specify `-use_wire_group 1`. *Default:* The same number of nets as specified in the `-nets` parameter.
`-via_rows` *integer*	
`-via_columns` *integer*	
	Specifies the number of cuts to make in the power vias (for both stack and non-stack). Default: `0`
`-width` *width*	
	Specifies the width of the stripes. You can specify a different width for each net within a set. If the number of widths specified does not match the number of net names, the last-specified width is used for the rest of unmatched nets.

Example

The following command adds stripes in the vertical direction for the `vdd` and `gnd` nets:

```
addStripe -direction vertical -nets {vdd gnd} -width 10 -spacing 1 -layer METAL6 -start_offset 50  -set_to_set_distance  50
```

Related Information

- `getAddStripeMode`
- `setAddStripeMode`
- Power Planning and Routing chapter in the *Innovus User Guide*.

colorizePowerMesh

```
colorizePowerMesh
[-help] [-reverse_with_nondefault_width {0 | 1}]
```

Use this command to colorize (assign mask information) power structures, including power wires and power via metals, in multi-pattern layers. This command also fixes color violations of power metals including via metals. It is recommended to use this command once all the power meshes are built. This command honors the FIXEDMASKS keyword in the LEF of the cell, in the case of colored LEF.

Note:

1. This command colors only the PG wires and PG via enclosure. but not the PG via cut. Also, it does not color pre-routed signal nets. All signal nets are colored while using the `place_opt_design` or `routeDesign` command.
2. The command does not fix via cut colors. To fix cut color or trim cut with violation, use the `fixVia` command. This command requires correct node setting in `setDesignMode -process/setNanoRouteMode -route_process_node`.
3. The wide NDR wires will be assigned with a different color than the track color through this command.

Parameter

`-help`	Prints a brief description that includes the type and default information for each `colorizePowerMesh` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man colorizePowerMesh`
`-reverse_with_nondefault_width {0 | 1}`	
	Reverses the 'on track' wire color to the different 'from track' color, when the wire width is not the default width.

Related Information

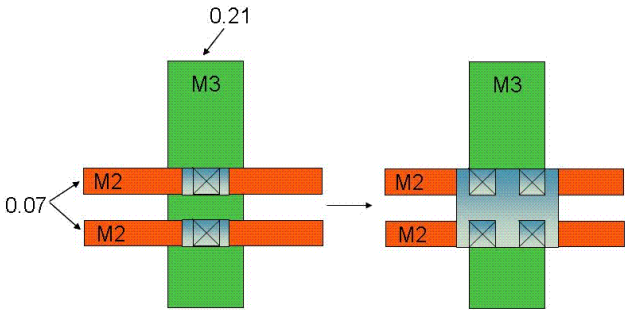
- Power Planning and Routing chapter in the *Innovus User Guide.*
- `fixVia`
- `setDesignMode`
- `setNanoRouteMode`

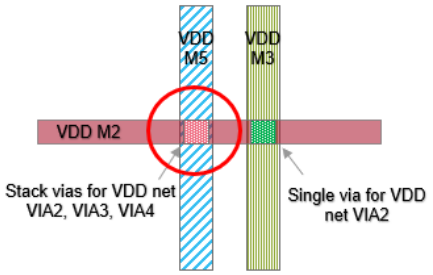
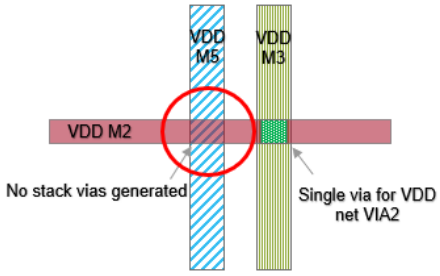
editPowerVia

```
 editPowerVia
[-help]
{-add_vias {0 | 1}
    | -modify_vias {0 | 1}
    | -delete_vias {0 | 1}}
[-area {{x1 y1 x2 y2} {x3 y3 x4 y4} ...}]
[-between_selected_wires {0 | 1}
    | -selected_blocks {0 | 1}
    | -selected_wires {0 | 1}
    | -selected_vias {0 | 1}]
[-bottom_layer layername]
[-create_via_on_merged_target_on_layer layerName]
[-create_via_on_merged_target_within micron]
[-create_via_on_signal_pins {0 | 1}]
[-exclude_stack_vias {0 | 1}]
[-followpin_via_stapling {cutclass1 weight1 cutclass2 weight2}]
[-nets net_list]
[-orthogonal_only {0 | 1}]
[-split_long_via {threshold step offset length}]
[-split_vias {0 | 1}
[-same_sized_stack_vias {0 | 1}
[-skip_via_on_wire_shape {[Blockring] [Stripe] [Followpin]
    [Corewire] [Blockwire] [Iowire] [Padring] [Ring] [Fillwire] [Noshape]}
[-skip_via_on_pin {[Pad] [Block] [Cover] [Standardcell] [Physicalpin]}]
[-skip_via_on_wire_status {[routed] [fixed] [cover] [shield]}]
[-top_layer layername]
[-uda]
[{-via_scale_height integer -via_scale_width integer
    | -via_rows integer -via_columns integer
    | -via_height real_number -via_width real_number}]
[-via_using_exact_crossover_size {0 | 1}]
```

Modifies or deletes existing power vias or adds new power vias to the design.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `editPowerVia` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man editPowerVia`
`-add_vias {0 | 1}`	When set to `1`, adds power vias to the design. *Default:* `0`
`-area {{x1 y1 x2 y2} {x3 y3 x4 y4} ...}`	
	Adds, modifies, or deletes power vias within the area regions enclosed by the specified coordinates. *Note:* If you need to add power vias within the specified area only, run `setViaGenMode -area_only 1` before `editPowerVia`.
`-between_selected_wires {0 | 1}`	
	When set to `1`, adds, modifies, or deletes power vias between selected wires only. You can select the wires interactively in the design display area or by using the `select_obj` command. *Default:* `0`
`-bottom_layer layername`	Specifies the lowest layer to which vias can connect. *Default:* The lowest metal layer in the design.
`-create_via_on_merged_target_within micron`	
	Helps `viaGen` merge the targets within the halo distance on the specified layer.
`-create_via_on_merged_target_on_layer layerName`	
	Helps `viaGen` add a virtual target internally, ignore the original targets, and drop vias to the virtual target instead. 
`-create_via_on_signal_pins {0 | 1}`	

	When set to `1`, allows generation of vias from special wire to signal pins. `editPowerVia` drops vias between special route and PG/signal pins, including the flipchip route. *Default*: `0` **Note:** This parameter can be used with the `-area` parameter to specify the area for adding vias. It mainly occurs at the tie-high and tie-low cell pins.
`-delete_vias {0 | 1}`	Deletes power vias from your design. *Default:* `0`
`-exclude_stack_vias {0 | 1}`	Avoids generation of stack vias between two non-adjacent layers to handle the intersection cases of cross multiple layers. See the examples below to understand the behavior of this parameter with different values: • `editPowerVia -exclude_stack_vias 0 -add_vias 1 ...`  • `editPowerVia -exclude_stack_vias 1 -add_vias 1 ...`  *Default:* `0`
`-followpin_via_stapling {cutclass1 weight1 cutclass2 weight2}`	
	Defines two cut classes to be tried in priority in the followpin via stapling to maximize the effective number of cuts. The weight value is for via insertion ratio calculation.
`-modify_vias`	Modifies vias in the design.

`-nets net_list`	Specifies the list of nets on which to add, delete, or edit vias.
`-orthogonal_only {0 | 1}`	When set to `1`, creates vias *only* for orthogonal wires and pin intersections. *Default*: If you do not specify this parameter or if you specify it with a value of `1`, the software creates vias for orthogonal wires and pin intersections.
`-same_sized_stack_vias {0 | 1}`	When set to `1`, the software only generates same-sized stacked vias. If set to `0`, the software trims the vias as needed to meet the DRC rules. *Default:* `0`
`-selected_blocks {0 | 1}`	Specifies whether vias in the selected blocks only should be looked up.
`-selected_vias {0 | 1}`	When set to `1`, modifies or deletes the selected vias only. You can select the wires interactively in the design display area or by using the `select_obj` command.
`-selected_wires {0 | 1}`	When set to `1`, drops vias between selected wires and all same net objects across the selected wires. You can select the wires interactively in the design display area or by using the `select_obj` command.
`-skip_via_on_pin {[Pad] [Block] [Cover] [Standardcell] [Physicalpin]}`	
	Prevents vias from being generated on the specified types of pins. If you specify `-skip_via_on_pin {}`, the power planning software makes connections to all five categories of pins. *Default:* If you do not specify this parameter, vias are generated for block pins, pad pins, and cover macro pins, but not for standard cell pins.
`-skip_via_on_wire_shape {[Blockring] [Stripe] [Followpin] [Corewire] [Blockwire] [Iowire] [Padring] [Ring] [Fillwire] [Noshape]}`	
	Prevents vias from being generated for the specified wire shapes. *Default:* If you do not specify this parameter, vias are generated for all wire shapes.
`-skip_via_on_wire_status {[routed] [fixed] [cover] [shield]}`	
	Prevents vias between wires with routed, cover, fixed, or shield status and other power wires. *Default:* If you do not specify this parameter, vias are generated for any wire status.
`-split_long_via {threshold step offset length}`	
	Splits vias longer than the specified length (`threshold`, in micrometers) into smaller vias with specified center-to-center spacing (`step`, in micrometers), left/bottom end offset (`offset`, in micrometers) and vertical/horizontal length (`length`, in micrometers) values . This is independent of the option `-split_via`. *Default*: If you do not specify this parameter, the software does not split vias. **Note:** If the `offset` is given as negative, the vias will be placed symmetrically on both ends. The default value of `offset` is `-1`.

`-split_vias {0 | 1}`	When set to `1`, two or more partial vias can be created at the crossover between a stripe and any target (pin or wire) if an obstruction (macro obstruction; wire or pin on a different net) prevents the creation of a full-size via. This can be useful, for example, if several metal blockages partially obstruct the area where a full-size via would normally be placed. This parameter permits the creation of partial vias that would not violate the process rules, in the open areas. A value of 1 allows multiple partial vias to be created. A value of 0 prevents multiple partial vias from being created. *Default:* `0`
`-top_layer` *layername*	Specifies the highest layer to which vias can connect. *Default:* The highest metal layer in the design.
`-uda`	Indicates the user defined attribute subclass for wires to route. This option is of type string and is optional.
`-via_height` *real_number*	Specifies the absolute height of the added or modified power vias. You can use this parameter only if you choose the `-add_vias` or `-modify_vias` parameter. Default: `0.0`
`-via_width` *real_number*	Specifies the absolute width of the added or modified power vias. You can use this parameter only if you choose the `-add_vias` or `-modify_vias` parameter.
`-via_rows` *integer*	Specifies the number of cuts to make in the power vias (for both stack and non-stack). You can use this parameter only if you choose the `-add_vias` or `-modify_vias` parameter. Default: `0`
`-via_columns` *integer*	Specifies the number of cuts to make in the power vias (for both stack and non-stack). You can use this parameter only if you choose the `-add_vias` or `-modify_vias` parameter. Default: `0`
`-via_scale_height` *integer*	Specifies the percentage to scale the power via height. You can use this parameter only if you choose the `-modify_vias` or `-add_vias` parameter.
`-via_scale_width` *integer*	Specifies the percentage to scale the power via width. You can use this parameter only if you choose the `-modify_vias` or `-add_vias` parameter.
`-via_using_exact_crossover_size {0| 1}`	When set to `1`, generates partial vias of the exact crossover size. A value of 1 allows partial vias to be generated. A value of 0 prevents partial vias, and the software always generates full-size vias. *Default:* `1`

Examples

- The following command creates a power via between selected wires with a cut pattern of 3 by 3:
  ```
  editPowerVia -add_vias 1 -between_selected_wires 1 -via_rows 3 -via_columns 3
  ```
- The below command deletes all power vias in the area defined by {0 0 500 600}:
  ```
  editPowerVia -delete_vias 1 -area {0 0 500 600}
  ```
- The following command adds power vias on the VDD net, in the area defined by {0 0 100 100}:
  ```
  editPowerVia -add_vias 1 -net VDD -area {0 0 100 100}
  ```

Related Information

- Power Planning and Routing chapter in the *Innovus User Guide.*
- `setViaGenMode`
- `setPreference` (see `SelectByArea` in Preference Variables section)
- `editSelect`
- `addSplitPowerVia`
- `verifyPowerVia`
- `select_obj`

get_add_target_pg_mode

```
  get_add_target_pg_mode
[-help]
[-allow_weak_connect]
[-max_extension_distance]
[-pins]
[-pins_group_distance]
[-respect_routes]
[-share_resource]
[-quiet]
[-nonDefault]
```

Returns the information for the specified parameter. Note that you can specify one or more parameter. When you invoke this command without any argument, the software prints the current values stored in the database for all the options. If you provide an option, then the software prints the value currently stored in the database.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `get_add_target_pg_mode` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man get_add_target_pg_mode`
`-parameter_names`	Returns the information about the specified parameters. You can specify one or more parameters. See `set_add_target_pg_mode` for the description of the `get_add_target_pg_mode` parameters.
`-quiet`	Returns the current settings for the specified parameters only in the Tcl list format. If you specify `-quiet` without any parameters, the software returns the current settings of all the `set_add_target_pg_mode` parameters in Tcl list format.

Related Information

- `add_target_pg`
- `set_add_target_pg_mode`

get_reinforce_pg_mode

```
  get_reinforce_pg_mode
[-help]
[-auto_ir_fix_effort]
[-critical_path_slack]
[-irdrop_hierarchical_block]
[-irdrop_hierarchical_top_rail_analysis_directory]
[-nonDefault]
[-respect_defined_nets]
[-respect_routes]
[-respect_stdcell_geometry]
[-timing_aware_effort]
[-quiet]
```

Returns the information for the specified parameter. Note that you can specify one or more parameter. When you invoke this command without any argument, the software prints the current values stored in the database for all the options. If you provide an option, then the software prints the value currently stored in the database.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `get_reinforce_pg_mode` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man get_reinforce_pg_mode`
`-parameter_names`	Returns the information for the specified parameters. You can specify one or more parameters. See `set_reinforce_pg_mode` for descriptions of the `get_reinforce_pg_mode` parameters you can specify.
`-quiet`	Returns the current settings for the specified parameters only in the Tcl list format. If you specify `-quiet` without any parameters, the software returns the current settings of all `set_reinforce_pg_mode` parameters in Tcl list format.

Related Information

- `reinforce_pg`
- `set_reinforce_pg_mode`
- Power Planning and Routing chapter in the *Innovus User Guide.*

getAddRingMode

```
getAddRingMode
[-help]
[-avoid_short]
[-break_core_ring_io_list]
[-continue_on_no_selection]
[-detailed_log]
[-extend_blockring_search_distance]
[-extend_corering_search_distance]
[-extend_merge_with_prewires]
[-extend_over_row]
[-extend_search_nets]
[-extend_stripe_search_distance]
[-gap_width_without_io]
[-ignore_rows]
[-max_via_size]
[-nonDefault]
[-orthogonal_only]
[-ring_target]
[-skip_crossing_trunks]
[-skip_via_on_pin]
[-skip_via_on_wire_shape]
[-spacing_from_block]
[-split_long_via]
[-stacked_via_bottom_layer]
[-stacked_via_top_layer]
[-via_using_exact_crossover_size]
[-wire_center_offset]
[-quiet]
```

Returns information for the specified parameter. The user can specify one or more parameter. When you invoke these commands without any argument, Innovus prints the current values stored in the database for all the options. If you provide any option, then Innovus would print the value currently stored in the database.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `getAddRingMode` parameter. For a detailed description of the command and all its parameters, use the man command: `man getAddRingMode`.
`-parameter_names`	Displays information for the specified parameters. You can specify one or more parameters. See `setAddRingMode` for descriptions of the parameters you can specify.
`-quiet`	Displays the current settings for the specified parameters in Tcl list format only.

Example

The following command returns information for the specified parameter:

```
getAddRingMode

-ring_target default (default: default)

-extend_over_row 0 (default: 0)

-skip_crossing_trunks none (default: none)

-avoid_short 0 (default: 0)

-extend_merge_with_prewires 1 (default: 1)

-extend_corering_search_distance 0.00 (default: 0.0)
```

`-extend_blockring_search_distance` *`0.00`* `(default: 0.0)`

`-extend_stripe_search_distance` *`0.00`* `(default: 0.0)`

`-ignore_rows` *`0`* `(default: 0)`

`-avoid_short` *`0`* `(default: 0)`

Related Information

- `addRing`
- `setAddRingMode`
- Power Planning and Routing chapter in the *Innovus User Guide*

getAddStripeMode

```
getAddStripeMode
[-help]
[-allow_jog]
[-allow_nonpreferred_dir]
[-area]
[-break_at]
[-color_balance]
[-continue_on_no_selection]
[-detailed_log]
[-domain_offset_from_core]
[-extend_to_closest_target]
[-extend_to_first_ring]
[-ignore_block_check]
[-ignore_blockring_when_breaking]
[-ignore_DRC]
[-ignore_nondefault_domains]
[-inside_cell_allow_shift]
[-inside_cell_only]
[-keep_pitch_after_snap]
[-max_extension_distance]
[-max_via_size]
[-merge_with_all_layers]
[-mesh_via]
[-offset_from_core]
[-optimize_stripe_for_routing_track]
[-orthogonal_offset]
[-orthogonal_only]
[-over_row_extension]
[-partial_set_thru_domain]
[-recommend_width]
[-remove_floating_stapling]
[-remove_floating_stripe_over_block]
[-remove_stripe_under_ring]
[-respect_routes]
[-route_over_rows_only]
[-rows_without_stripes_only]
[-same_sized_stack_vias]
[-skip_via_on_pin]
[-skip_via_on_wire_shape]
[-spacing_from_block]
[-spacing_type]
[-split_long_via]
[-split_vias]
[-split_wire_spacing]
[-split_wire_weight]
[-split_wire_width]
[-stacked_via_bottom_layer]
[-stacked_via_top_layer]
[-stapling_extend_to_minimum_spacing]
[-stapling_nets_style]
[-stapling_shift]
[-stop_at_closest_target]
[-stop_at_last_wire_for_area]
[-stripe_min_length]
[-stripe_min_width]
[-switch_cellname]
[-switch_layer_overlap_length]
[-trim_antenna_back_to_shape]
[-trim_antenna_max_distance]
[-trim_stripe]
[-use_exact_spacing]
[-use_point2point_router]
[-use_stripe_width]
[-via_using_exact_crossover_size]
```

```
[-quiet]
[-nonDefault]
```

Returns the current settings of the `setAddStripeMode` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `getAddStripeMode` parameter. For a detailed description of the command and all its parameters, use the man command: `man getAddStripeMode`
`-parameter_names`	Returns information for the specified parameters. You can specify one or more parameters. See `setAddStripeMode` for descriptions of the `setAddStripeMode` parameters you can specify.
`-quiet`	Returns the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software returns the current settings of all `setAddStripeMode` parameters in Tcl list format.

Examples

- The following command returns the current settings of `setAddStripeMode -offset_from_core` in the Tcl list format:
  ```
  setAddStripeMode -offset_from_core -quiet
  ```
  The software returns the following information:
  ```
  true
  ```
- The following command returns the current settings for `setAddStripeMode -offset_from_core`:
  ```
  setAddStripeMode -offset_from_core
  ```
  The software returns the following information:
  ```
  -offset_from_core true # bool, default=true
  ```

Related Information

- `addStripe`
- `setAddStripeMode`
- Power Planning and Routing chapter in the *Innovus User Guide*.

getViaGenMode

```
getViaGenMode
[-help]
[-accuracy_effort]
[-add_pin_to_pin_via]
[-align_merged_stack_via_metals]
[-allow_via_expansion]
[-allow_wire_shape_change]
[-area_only]
[-create_double_row_cut_via]
[-create_max_row_cut_via]
[-cutclass_preference]
[-disable_via_merging]
[-extend_out_wire_end]
[-full_cut_via_only]
[-genvia_naming_prefix]
[-hookup_contact_max]
[-hookup_contact_pg_track]
[-hookup_rail_pair]
[-hookup_via_distance]
[-hookup_via_fixed_grid]
[-hookup_via_min_distance]
[-hookup_via_style]
[-hookup_via_viarule]
[-hookup_virtual_trim_grid]
[-ignore_design_boundary]
[-ignore_DRC]
[-ignore_viarule_enclosure]
[-inherit_wire_status]
[-invoke_verifyGeometry]
[-keep_existing_via]
[-keep_fixed_via]
[-mincut_preference]
[-optimize_cross_via]
[-optimize_via_on_routing_track]
[-parameterized_via_only]
[-partial_overlap_threshold]
[-preferred_vias_only]
[-reference_boundary]
[-respect_signal_routes]
[-respect_stdcell_geometry]
[-respect_wire_ndr]
[-set_via_expansion_dir]
[-snap_via_center_to_grid]
[-split_long_via_global_grid]
[-symmetrical_via_only]
[-use_track_offset]
[-use_trim_metal_enclosure]
[-viarule_preference]
[-quiet]
[-nonDefault]
```

Returns the current settings of the `setViaGenMode` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `getViaGenMode` parameter. For a detailed description of the command and all its parameters, use the man command: `man getViaGenMode`
`-parameter_names`	Returns information for the specified parameters. You can specify one or more parameters. See `setViaGenMode` for descriptions of the parameters you can specify.
`-quiet`	Displays the current settings for the specified parameters in Tcl list format only. If you specify the `-quiet` parameter without any the other parameter(s), the software returns the current settings of all the `setViaGenMode` parameters in Tcl list format.

Examples

The following command returns the current settings of the `setViaGenMode -genvia_naming_prefix` parameter:

```
getViaGenMode -genvia_naming_prefix
```

The software returns the following information:

```
-genvia_naming_prefix {} # string, default=""
```

Related Information

- Power Planning and Routing chapter in the *Innovus User Guide*.
- `setViaGenMode`

loadSpecialRoute

```
loadSpecialRoute
[-help]
filename
```

Loads the spr file generated through the `saveSpecialRoute` command. It also clears all special routes and loads the specified spr file. If the spr files are loaded incrementally, the `addSpecialRoute` command must be used. If the specified filename is compressed (with the .gz extension), the file is read in directly.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `loadSpecialRoute` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man loadSpecialRoute`
`filename`	Specifies the name of the file that contains special route information, to be used as input for the design.

Related Information

- `addSpecialRoute`
- `saveSpecialRoute`
- Power Planning and Routing chapter in the *Innovus User Guide*.

reinforce_pg

```
  reinforce_pg
[-help]
[-area {x1 y1 x2 y2}]
[-area_blockage {{x1 y1 x2 y2} {x1 y1 x2 y2 x3 y3 x4 y4 ...} ...}]
[-pattern_file file_name]
[-snap_wire_center_to_grid {default | Grid}]
{-pattern_map {{irdrop_value1 pattern_name1} {irdrop_value2 pattern_name2}..} | -write_pattern_template file_name | -boxes_file
file_name | -auto_ir_fix {preroute | postroute}}
[[-tile_row_column {x y}  | -tile_size {x y} | -number_region value] | -write_pattern_template file_name]
[-rail_analysis_directory {directory} | -write_pattern_template file_name]
```

Reinforces PG stripes between the existing original PG stripes in Innovus to fix the IR drop violations reported from Voltus.

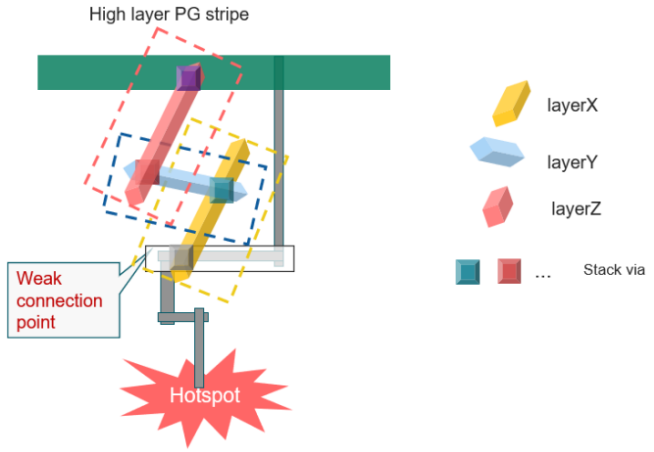
Parameters

`-help`	Outputs a brief description that includes the type and default information for each `reinforce_pg` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man reinforce_pg`
`-area {x1 y1 x2 y2}`	Specifies the coordinates of the area in which the IR drop violation boxes from Voltus can be fixed by the `reinforce_pg` command.

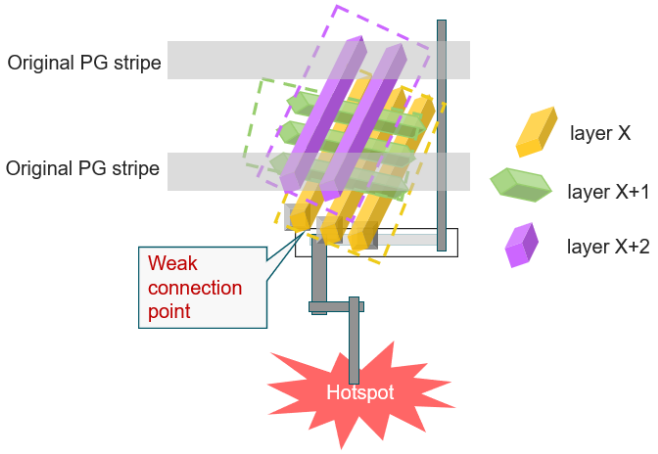
`-area_blockage {{x1 y1 x2 y2} {x1 y1 x2 y2 x3 y3 x4 y4 ...} ...}`	Specifies the blockage coordinates of one or more area regions to restrict the addition of any stripes. This parameter supports both the rectangular and rectilinear area regions along with their combinations. Their spacing is `0` during the stripe generation. Following are the possible parameter usage scenarios:
	• Single Rectangular Area:
	`-area_blockage {x1 y1 x2 y2}`
	For example:
	`-area_blockage {100 100 200 200}`
	• Single Rectilinear Area:
	`-area_blockage {x1 y1 x2 y2 x3 y3 x4 y4 x5 y5 x6 y6}`
	For example:
	`-area_blockage {100 100 100 200 150 200 150 150 200 150 200 100}`
	• Multiple Rectangular Areas:
	`-area_blockage {{x1 y1 x2 y2} {a1 b1 a2 b2} ...}`
	• Multiple Rectilinear Areas:
	`-area_blockage {{x1 y1 x2 y2 ... } {a1 b1 a2 b2 a3 b3 ...} ...}`
	• Combination of Rectangular and Rectilinear Areas:
	`-area_blockage {{x1 y1 x2 y2} {a1 b1 a2 b2} ... {x1 y1 x2 y2 x3 y3 ...} {a1 b1 a2 b2 a3 b3 ...} ...}`
	While blocking multiple areas using this parameter, each of them must be inside the braces. The areas that are not included inside the braces will be ignored. For example, `-area_blockage {{x1 y1 x2 y2} a1 b1 a2 b2 a3 b3 ... {c1 d1 c2 d2 c3 d3 ...}}`, `{x1 y1 x2 y2}`, and `{c1 d1 c2 d2 c3 d3 ...}` will be accepted by the parameter, and `a1 b1 a2 b2 a3 b3 ...` will be ignored.

`-auto_ir_fix {preroute | postroute}`	Fixes IR drop with an automated flow. No pattern is needed with the automated flow provided by this parameter. This parameter extracts violation tiles based on RLRP, and automatically fixes the IR drop based on the location provided by Voltus at each layer. When this parameter is used, Voltus rail analysis is performed. ⚠ The tile size and location is provided by Voltus. Use one of the following values with this parameter:

- `preroute`: Fixes IR drop using minimum resources. Use this flow before signal routing to ensure a good stripe connection. When this parameter value is specified, the software adds several metal layers between the weak point and high PG stripe. The location and layer are automatically controlled by the software analyzer. The following figure presents an example of using `-auto_ir_fix preroute`:



- `postroute`: Fixes IR drop with all the remaining available metal resources. Use this flow after routing so that all the available resources can be used. When this parameter value is specified, the software adds all the rest metal resources at every metal layer according to the information received from Voltus. By this, every metal layer makes connection with the adjacent layer. The layer and tile location are automatically controlled by the software analyzer. By default, `set_reinforce_pg_mode -respect_routes` is set to `all` when `reinforce_pg -auto_ir_fix postroute` is specified, which you can change manually. The following figure presents an example of using `-auto_ir_fix postroute`:



⚠ When the `-auto_ir_fix` parameter is used with pattern flow, the pattern is ignored.

Default: `none`

`-boxes_file` *file_name*	Specifies the file for including the list of boxes and corresponding pattern names. The format of the file is mentioned below:
	`Pattern1 x01 y01 x02 y02`
	`Pattern2 x11 y11 x12 y12`
	……

`-number_region` *value*	Specifies the number of regions to be analyzed by the software. The integer value of this parameter ranges from 1 to 65536. *Default:* `100`
`-pattern_map {{`*irdrop_value1 pattern_name1*`} {`*irdrop_value2 pattern_name2*`}..}`	
	Specifies the pattern for IR drop fixing through the IR drop value and pattern name defined by the `-pattern_file` parameter. If `irdrop_value1` is lesser than `irdrop_value2`, the `reinforce_pg` command uses `pattern_name1` to fix the IR drop between `irdrop_value1` and `irdrop_value2`. And `pattern_name2` is used to fix the IR drop bigger than or equal to `irdrop_value2`. It is recommended to specify the `irdrop_value` in the ascending order.
`-rail_analysis_directory {`*directory*`}`	Allows the `reinforce_pg` command to read the rail analysis information available in the specified directory.
`-snap_wire_center_to_grid {default | Grid}`	Specifies the snap wire center for routing grid. Use this parameter to control the added stripe for turning on or off the grid for the `reinforce_pg` command. Following are the parameter values: • `default`: Snaps the added wires to gird or half grid, according to the wire width. • `grid`: Snaps the the added wires only to the grid. *Default:* `default`
`-tile_row_column {`*x y*`}`	Specifies the number of tiles for power analysis in the x and y directions. This parameter enables tile-based analysis of the hot-spot regions to divide the design based on user-defined tile count and reports IR drop violations on each tile.
`-tile_size {`*x y*`}`	Specifies the size of tiles for power analysis in the x and y directions. This parameter enables tile-based analysis of the hot-spot regions to divide the design based on user-defined tile size and reports IR drop violations on each tile.
`-write_pattern_template` *file_name*	Specifies the template of the pattern file. This parameter cannot be used with other parameters. Refer to the following example to understand the usage of this parameter: `reinforce_pg -write_pattern_template pattern.cfg`

`-pattern_file` *file_name*	Defines the pattern for the pattern template of reinforce stripes. The `reinforce_pg` command uses different patterns for fixing the corresponding IR drop value sections.
	The following is the file syntax of the pattern template for single metal layer:
	`NAME pattern_name`
	`NETS {list_of_nets}`
	`LAYER metal_layer`
	`MAXPITCH real_value`
	`MINDISTANCE real_value`
	`WIDTH real_value`
	`VIACONTROL {metal_of_viaHookUpBot metal_of_viaHookUpTop}`
	`SPACING real_value ##optional`
	`UDA subclass_string ##optional`
	Note: You can define the density of the added PG using the `MAXPITCH` and `MINDISTANCE` parameters.
	The following is the description of the pattern template parameters for a single metal layer:
	◦ `NAME`: Specifies the name of the pattern. The `pattern_name` used at `pattern_map` should be the

same as the name defined in the pattern file.

- NETS: Specifies the names of the nets for which the stripes are required to be generated.
- LAYER: Specifies the name of the layer in which the stripes are required to be generated.
- WIDTH: Specifies the width of the stripes.
- VIACONTROL: Specifies the metal layers to which the generated PG wires will be connected.
- SPACING: Specifies the spacing between the stripes/nets defined for the NETS attribute. If SPACING is not defined, the Multi nets defined through NETS will use the minimum DRC free distance.
- UDA: Displays the user-defined attribute attached to every generated stripe segment.
- MAXPITCH: Specifies the maximum distance between the generated PG stripes. The distance between the added PG wires and original PG stripes cannot be more than MAXPITCH.
- MINDISTANCE: Specifies the minimum distance between the generated PG stripes. The distance between added PG wires and original PG stripes should be more than MINDISTANCE.

Note: You can define the density of the added PG using the MAXPITCH and MINDISTANCE parameters. If the distance between the two PG wires for any net is smaller than MINDISTANCE after adding the PG, the added PG wires will be deleted.

The following is the file syntax of pattern template for two metal layers:

```
NAME  pattern_name
NETS    {list_of_nets}
LAYER  metal_layer1 metal_layer2
MAXPITCH         real_value1   [real_value2]
MINDISTANCE         real_value1   [real_value2]
WIDTH  real_value1   [real_value2]
VIACONTROL      {metal_of_viaHookUpBot metal_of_viaHookUpTop} [{metal_of_viaHookUpBot
metal_of_viaHookUpTop}]
SPACING  real_value                    ##optional
UDA  subclass_string                   ##optional
```

Note: If both the MAXPITCH and MINDISTANCE parameters define the same value and the two added metal layers are in the same direction, then metal_layer2 should align with metal_layer1.

The following is the description of the pattern template parameters for two metal layers:

- NETS: Specifies the net names of the two metal layers to be created.
  Note: The list of nets is not based on the metal layers. Every metal layer needs to generate all the nets defined.
- LAYER: Specifies the name of the layers in which stripes are to be created. Two metal layers are required in this scenario.
- WIDTH, VIACONTROL, SPACING, MAXPITCH, and MINDISTANCE: These parameters can be defined twice for the corresponding metal layer. If only one value is specified for these parameters, two metal layers will use the same value.

You can use an optional keyword, OVERVIAMETAL *metal_layer*, which makes the reinforce_pg command create PG over the via metal of same PG net. Else than the OVERVIAMETAL keyword, the *metal_layer* parameter is also optional. If *metal_layer* is defined with the OVERVIAMETAL keyword, only the defined metal layers follow the OVERVIAMETAL rule. Else, all the metal layers defined at the LAYER need to follow the OVERVIAMETAL rule.

The following is the syntax of the OVERVIAMETAL keyword:

```
OVERVIAMETAL metal_layer*
```

The following is the file syntax of the special pattern template for multiple layers:

```
NAME pattern_name
TYPE SMART
NETS {list_of_nets}
LAYERS layer1:layer2                                          ##optional
PITCHFACTOR integer_value                                     ##optional
VIACONTROL {metal_of_viaHookUpBot metal_of_viaHookUpTop}      ##optional
```

	`UDA subclass_string ##optional` Following are the pattern template attributes: • `NAME`: Specifies the name of pattern. The `NAME` attribute used with `pattern_map` should be same as the name defined in the pattern file. • `TYPE`: Specifies the type of the PG pattern. If `TYPE` is specifies as `SMART`, `reinforce_pg` runs PG pattern by smart PG flow. Otherwise, it runs PG pattern with the normal pattern flow. • `NETS`: Specifies the names of the nets for which stripes are to be created. • `LAYERS`: Specifies the name of the layer in which stripes need to be created. This is an optional attribute and presents smart PG pattern support layer range. Following are various attribute values: ◦ `layer1:layer2`: Signifies all the layers from `layer1` to `layer2`. ◦ `-`: Presents the default top or bottom routing layer. ◦ `-:layer2`: Signifies the default bottom routing layers to `layer2`. ◦ `layer1:-`: Specifies that `layer1` will be default for top routing layers. ◦ `default`: Includes all the routing layers. • `UDA`: Displays the user-defined attribute attached to each created stripe segment. This is an optional attribute. *Default*: `rpg_$pattern1` • `PITCHFACTOR`: Specifies the average pitch number between two added stripes. This is an optional attribute. *Default*: `1` • `VIACONTROL`: Specifies the metal layers to which the generated PG of bottom layer and top layer will be connected. *Default*: The adjacent lower layer of defined `layer1` and the adjacent upper layer of defined `layer2` of `LAYERS`.

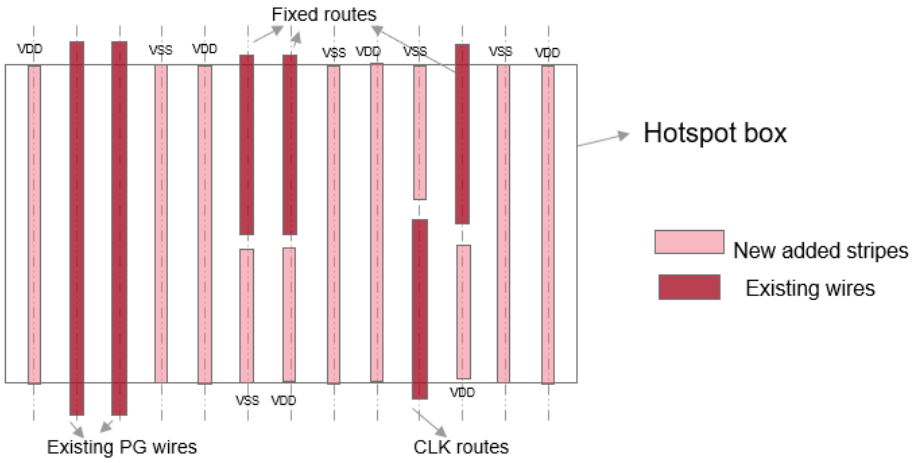
Examples

- The following command defines pattern for the pattern template of reinforce stripes with the following `pattern.tcl` file values:

```
NAME   smart1
TYPE   SMART
NETS   VDD VSS:

reinforce_pg -pattern_file pattern.tcl -pattern_map {0.04 smart1}
```

The following is the output of the above command:

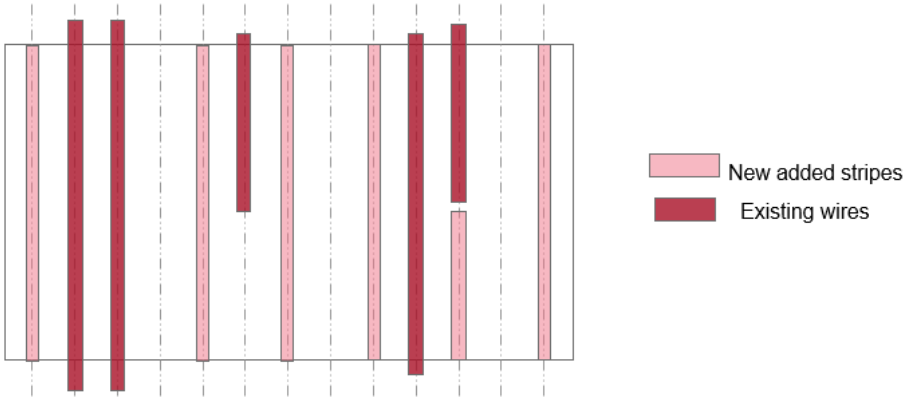


- The following command defines pattern for the pattern template of reinforce stripes with the following `pattern.tcl` file values:

```
NAME    smart1
TYPE    SMART
NETS    VDD
LAYERS    M5
PITCHFACTOR    2
VIACONTROL    M2 M6
UDA    PGSmart_1

reinforce_pg -pattern_file pattern.tcl -pattern_map {0.04 smart1}
```

The following is the output of the above command:

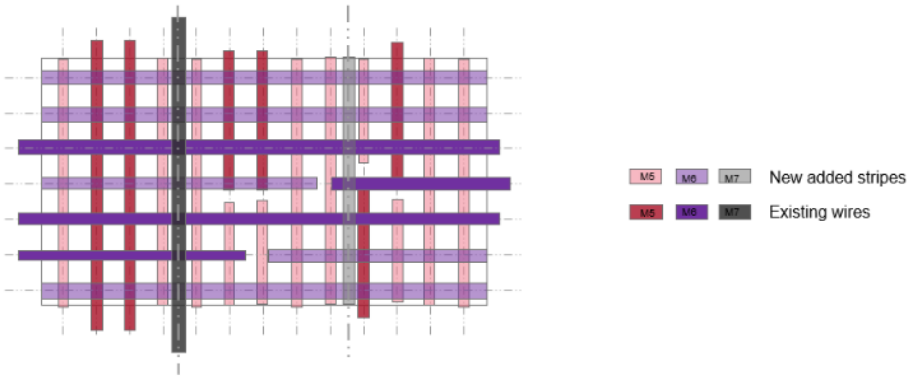


- The following command defines pattern for the pattern template of reinforce stripes on multiple layers with the following `pattern.tcl` file values:

```
NAME    smart1
TYPE    SMART
NETS    VDD VSS
LAYERS    M5:M7
VIACONTROL    M4 M10
UDA    PGSmart_1

reinforce_pg -pattern_file pattern.tcl -pattern_map {0.04 smart1}
```

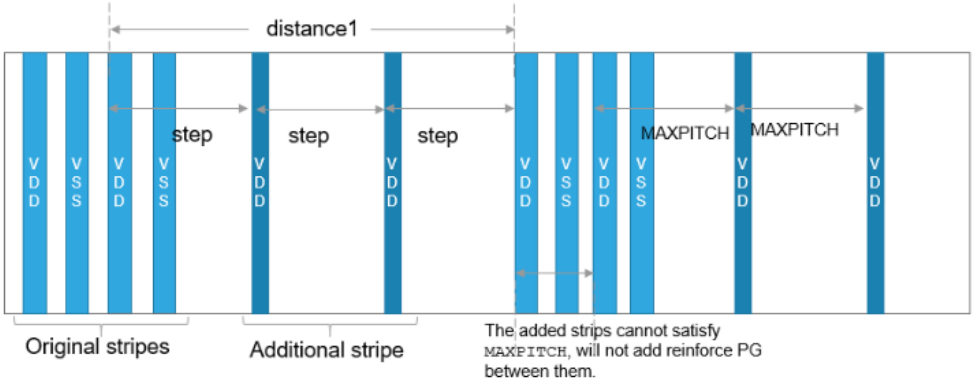
The following is the output of the above command:



- The following command presents an example of defining a pattern file parameters and calculating the number of additional stripes to be added between two adjacent stripes of the same net:

  ```
  reinforce_pg -pattern_file pattern.tcl -pattern_map {0.03 PATTERN1}
  ```

  The following is the output of the above command:

  The following are the contents of the defined `pattern.tcl` file:

  ```
  NAME  PATTERN1

  NETS   VDD

  LAYER  M5

  MAXPITCH    15

  MINDISTANCE      5

  WIDTH  0.3

  VIACONTROL  {M4 M6}

  UDA  PGFIX_1
  ```

  Additional stripes can only be added when the distance between the two adjacent stripes of the same net is more than `MAXPITCH` and `2*MINDISTANCE`, as shown below:

  ```
  Distance1 > MAXPITCH  &&  Distance1 > 2*MINDISTANCE
  ```

  To calculate the number of additional stripes to be added between two adjacent stripes of the same net:

  ```
  N = ceil (distance1 / MAXPITCH)
  ```

  The `N-1` stripes will be evenly distributed between two adjacent stripes with:

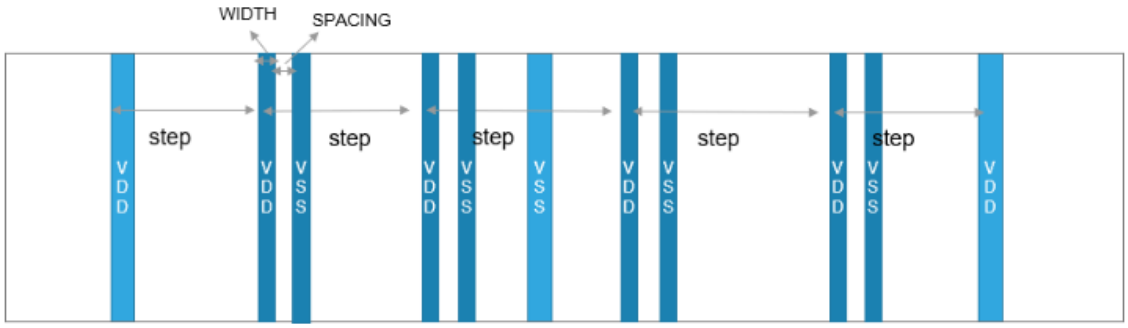
  ```
  step = distance1/N
  ```

  Note: When some new stripes added between two adjacent stripes are shifted and the distance between them is more than the `Maxpitch`, the tool adds extra stripe to satisfy the `Maxpitch` defined.

- The following command defines a metal layer with two nets:

  ```
  reinforce_pg -pattern_file pattern.tcl -pattern_map {0.06 pattern2}
  ```

The following is the output of the above command:



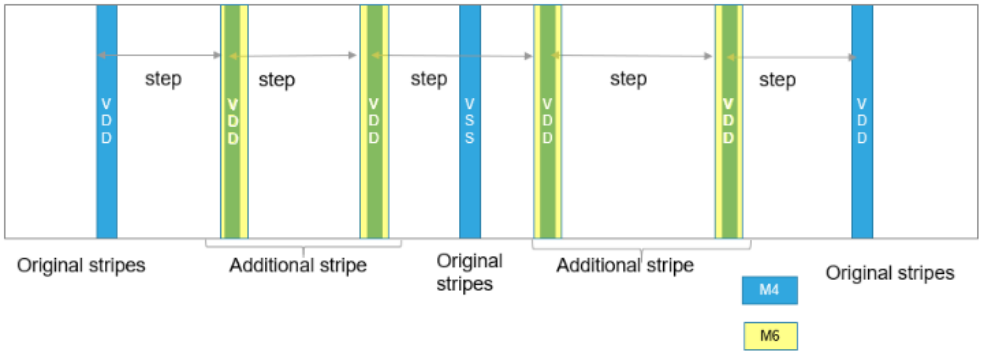
The following are the contents of the defined `pattern.tcl` file:

```
NAME        pattern2
NETS         VDD VSS
LAYER                M5
MAXPITCH          10
MINDISTANCE    3
WIDTH         0.3
SPACING         0.3
VIACONTROL     {M2 M8}
UDA         PGFIX_2
```

- The following command defines two metal layers in the same direction and the `MAXPITCH` and `MINDISTANCE` parameters with one value. Here, the `M6` metal layer should align with `M4`. The number and location of the additional stripes will depend on the first layer defined:

  ```
  reinforce_pg -pattern_file pattern.tcl -pattern_map {0.06 pattern3}
  ```

  The following is the output of the above command:



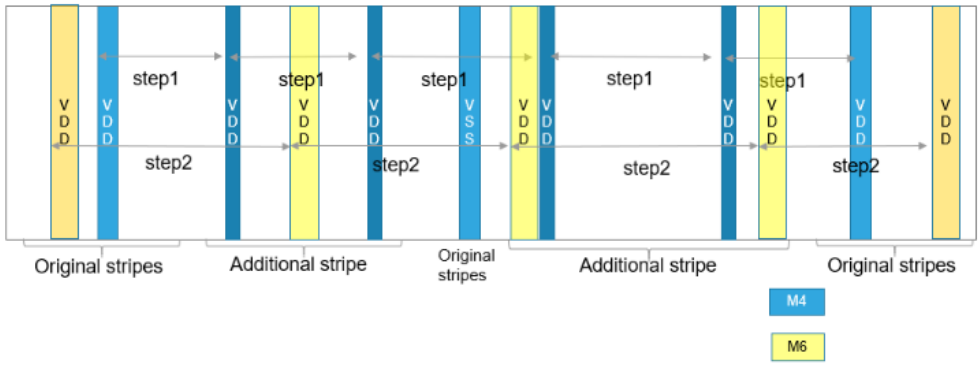
  The following are the contents of the defined `pattern.tcl` file:

  ```
  NAME  pattern3
  NETS   VDD
  LAYER  M4 M6
  MAXPITCH       20
  MINDISTANCE    5
  WIDTH  0.3  0.5
  VIACONTROL {M3 M5}  {M5 M7}
  UDA  PGFIX_3
  ```

- The following command defines two metal layers with same direction and and `MAXPITCH` and `MINDISTANCE` parameters with two values. The additional stripes of two metal layers will also be added respectively:

  ```
  reinforce_pg -pattern_file pattern.tcl -pattern_map {0.06 pattern4}
  ```

  The following is the output of the above command:



The following are the contents of the defined `pattern.tcl` file:

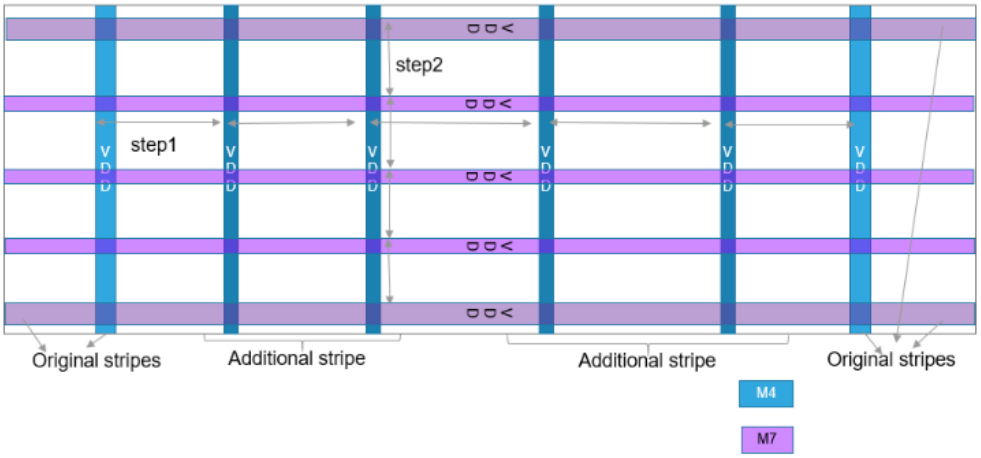
```
NAME  pattern4
NETS   VDD
LAYER  M4 M6
MAXPITCH       20 30
MINDISTANCE    5   8
WIDTH  0.3  0.5
VIACONTROL {M3 M5}  {M5 M7}
UDA  PGFIX_3
```

The `M4` and `M6` metal layers calculate this step respectively, where `M4` uses `step1` and `M6` uses `step2` to generate PG.

- The following command defines two metal layers with different directions and `MAXPITCH` and `MINDISTANCE` parameters with two values. The additional stripes of two metal layers will also be added respectively:

  ```
  reinforce_pg -pattern_file pattern.tcl -pattern_map {0.06 pattern5}
  ```

  The following is the output of the above command:



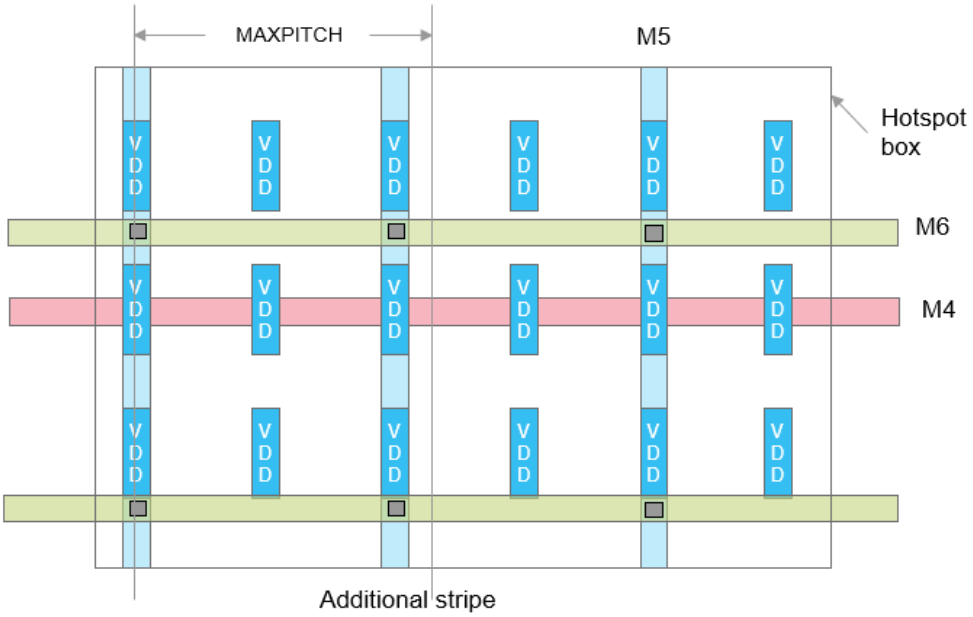
The following are the contents of the defined `pattern.tcl` file:

```
NAME  pattern5
NETS   VDD
LAYER  M4 M7
MAXPITCH       20 30
MINDISTANCE    5   8
WIDTH  0.3  0.5
VIACONTROL {M1 M5}  {M6 M10}
UDA  PGFIX_3
```

- The following command defines a metal layer with the `OVERVIAMETAL` keyword:

  ```
  reinforce_pg -pattern_file pattern.tcl -pattern_map {0.06 pattern6}
  ```

  The following is the output of the above command:



The following are the contents of the defined `pattern.tcl` file:

```
NAME pattern6
OVERVIAMETAL
NETS VDD
LAYER M5
MAXPITCH    15
MINDISTANCE      5
WIDTH 0.3
VIACONTROL  {M4 M6}
UDA PGFIX_1
```

Related Information

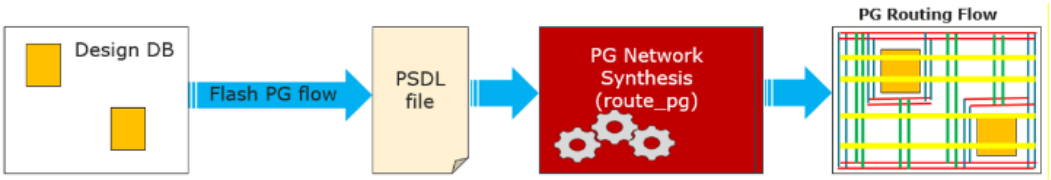
- `get_reinforce_pg_mode`
- `set_reinforce_pg_mode`
- Power Planning and Routing chapter in the *Innovus User Guide*.

route_pg

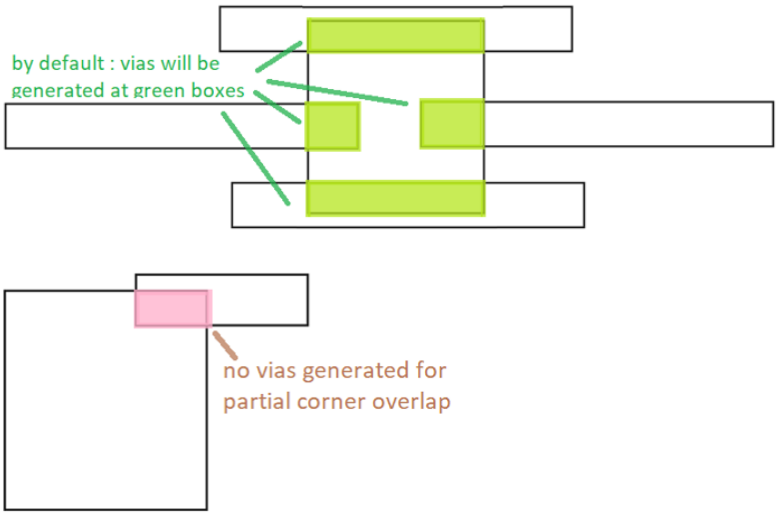
```
  route_pg
[-help]
[-check_psdl_only]
[-delete_floating]
[-fast]
[-psdl_file file_name]
[-skip_block_pin]
[-skip_partial_overlap_via]
[-skip_via]
[-write_psdl output_file]
```

Uses the Innovus database, builds the region with the help of PSDL, where the PG structure is to be created, and parses it. This command also uses all the syntax in PSDL to build the PG structure.

The following figure presents the process of developing a PG routing flow using the `route_pg` command:



Parameters

`-help`	Outputs a brief description that includes the type and default information for each `route_pg` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man route_pg`
`-check_psdl_only`	Performs sanity check only on the PSDL input file.
`-delete_floating`	Removes floating wires.
`-fast`	Generates PG routing flow in the Fast PG mode.
`-psdl_file file_name`	Specifies the PSDL input file.
`-skip_block_pin`	Skips the formation of connection to the block pins.
`-skip_partial_overlap_via`	Skips building of via on partial overlap intersection. See the example below: 

`-skip_via`	Skips building via object.
`-write_psdl` *output_file*	Writes a PSDL file to ensure that the input numerical/string Tcl variable is correctly calculated and applied.

Examples

- The following command checks the `-psdl_file` syntax only (without running the `route_pg` command):
  `route_pg -fast -psdl_file` *filename* `-check_psdl_only`
- The following command removes a floating wire:
  `route_pg -fast -psdl_file` *psdl_file* `-delete_floating`
- The following command does not generate via while running the `route_pg` command:
  `route_pg -fast -psdl_file` *psdl_file* `-skip_via`
- The following command skips partial generated vias:
  `route_pg -fast -psdl_file` *psdl_file* `-skip_partial_overlap_via`
- The following command generates a PG structure in the Fast PG mode and writes a PSDL file during its run:
  `route_pg -fast -psdl_file` *file_name* `-write_psdl` *output_file*
- The above command generates a PG structure in the Complete PG mode and writes a PSDL file during its run:
  `route_pg -psdl_file` *file_name* `-write_psdl` *output_file*
- The above command checks the PSDL syntax and writes a PSDL file without generating the PG structure:
  `route_pg -psdl_file` *file_name* `-check_psdl_only -write_psdl` *output_file*

Related Information

The following sections in Innovus User Guide:

- PG Network Synthesis
- PG Structure Description Language

saveSpecialRoute

```
saveSpecialRoute
[-help]
filename
-pre10
```

Saves special route information in a file. If the specified filename is compressed (with the `.gz` extension), the file is gzipped.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `saveSpecialRoute` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man saveSpecialRoute`
`filename`	Specifies the name of the output file into which special route information to include metal fill is saved. The file is saved in the `.spr` format.
`-pre10`	Saves the routing file with an older version.

Related Information

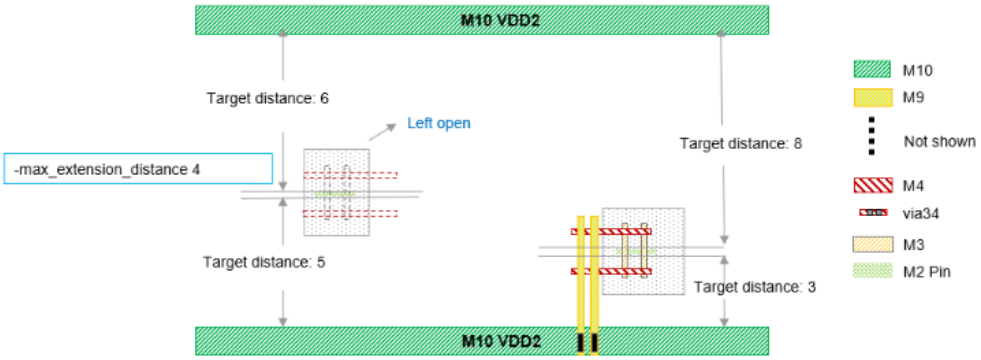
- `addSpecialRoute`
- `loadSpecialRoute`
- Power Planning and Routing and Optimizing Metal Density chapters in the *Innovus User Guide*

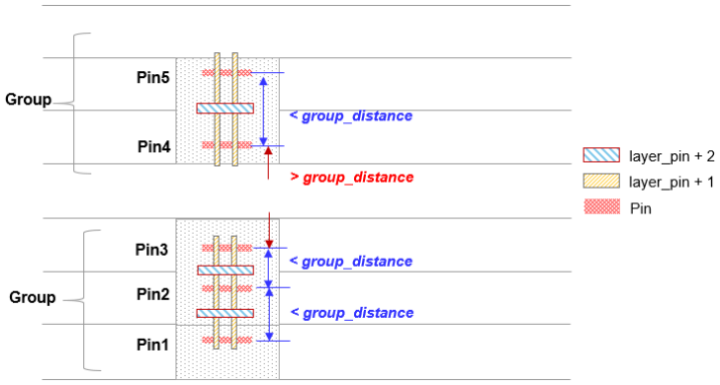
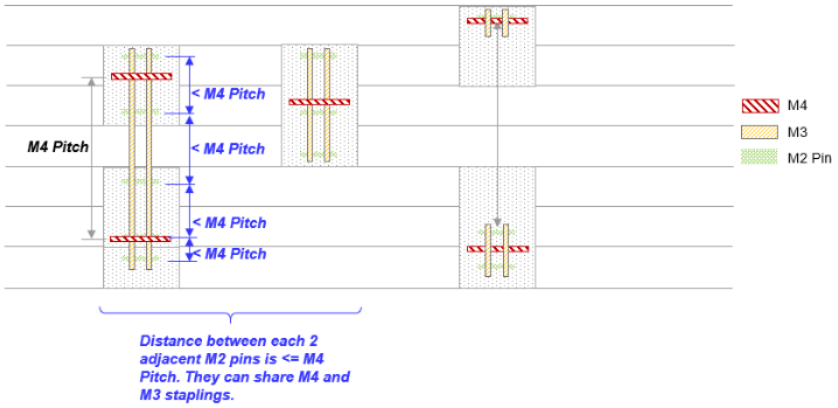
set_add_target_pg_mode

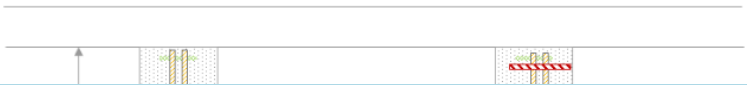
```
  set_add_target_pg_mode
[-help]
[-allow_weak_connect {true | false}]
[-max_extension_distance value]
[-pins {cell1:pin1 cell1:pin2 cell2:pin1 ...}]
[-pins_group_distance value]
[-reset]
[-respect_routes {none | fixed | fixed_and_clock | all}]
[-share_resource {true | false}]
```

Sets global variables for the `add_target_pg` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `set_add_target_pg_mode` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man set_add_target_pg_mode`
`-allow_weak_connect {true | false}`	When this parameter is set to `false`, the number of wires created on every stack layer need to satisfy the defined count with DRC clean. Else, the PG pin is left open. When set to `true`, the count lesser than defined can also be accepted while creating the total count causing DRC. *Default*: `false`
`-max_extension_distance value`	Specifies the maximum distance that the target PG pin can extend to the target layer. If the distance between the target PG pin and the candidate target layer is lesser than or equal to the specified distance, the candidate target is treated as the real target. If the target PG wire cannot be found within `-max_extension_distance`, the instance is left open. If this parameter is not used, the software connects the stripe layer to the closest target PG wire with DRC clean, by default. See the figure below for understanding:  *Default*: `2147483647`
`-pins {cell1:pin1 cell1:pin2 cell2:pin1 ...}`	

	Specifies the cell pins to be connected. *Default*: ""
`-pins_group_distance value`	Specifies the side-to-side distance of the pins to be treated as a group while connecting the target PG pins. The pins belonging to the same group share the same target PG stripes connection. When this parameter is set to its default value, `auto`, the `add_target_pg` command uses the default behavior of target PG generation to group the pins. If this parameter is not specified, the software uses the pitch of metal layer above the pins layer with same direction (`layer_pin+2`) as the group distance. If the pitch of metal layer above the pins layer with the same direction (`layer_pin+2`) is not defined, the pins will be connected separately. If the distance between two pins is lesser than `group_distance`, they will be treated as one group. See the figure below for understanding:  When `pins_group_distance` is not defined, the software uses the pitch of the metal layer above the pins layer with the same direction (`layer_pin+2`) as the group distance. See the figure below for understanding:  The following figure presents how the target PG pin's horizontal offset affects the stapling sharing:

	
`-reset`	Resets all or specified parameters to their default values.
`-respect_routes {none | fixed | fixed_and_clock | all}`	
	Specifies the route types to respect for DRC check during target PG generation. This parameter always honor all covers and PGs, but does not honor the trail routes, Following are the parameter values: • `none`: Specifies that no route types are respected. • `fixed`: Respects fixed regular wires. • `fixed_and_clock`: Respects fixed and routed clocks. • `all`: Respects fixed and routed clocks along with the routed signals. For more information on the support of the `set_add_target_pg_mode -respect_route` parameter values with various statuses of net, see Support of set_add_target_pg_mode -respect_route Parameter Values. *Default*: `fixed_and_clock`
`-share_resource {true | false}`	When set to `true`, the newly added PGs of the intermediate stacked layer can connect to the existing PG wire of the same layers and are preferred to be shifted to the track of the existing PG to save the track resource. If there are DRCs between the added PG and the existing PG after shifting, the added PG will follow the behavior explained below: • If the added wire is on different net from the existing PG, it will be shifted again to avoid DRC. • If the added wire is on the same net of the existing PG, it will be merged with the existing wire. When set to `false`, the stacked layer cannot connect to the existing intermediate layers between the cell PG pin and target PG layer. Additionally, the newly added PG will be shifted if its original location causes DRC. *Default*: `true`

Support of set_add_target_pg_mode -respect_route Parameter Values

The following table presents the support of the `set_add_target_pg_mode -respect_route` parameter values with various statuses of net:

Target PG Honoring a Wire			-respect_route Values			
Net Use	**Status**		**none**	**fixed**	**fixed_and_clock**	**all**
P/G			Yes	Yes	Yes	Yes
Non-P/G	**Cover**		Yes	Yes	Yes	Yes
	Fixed		No	Yes	Yes	Yes
	Routed	**Clock**	No	No	Yes	Yes
		Signal	No	No	No	Yes
	Unknown		No	No	No	No

Related Information

- `add_target_pg`
- `get_add_target_pg_mode`

set_reinforce_pg_mode

```
  set_reinforce_pg_mode
[-help]
[-auto_ir_fix_effort {low | medium | high}]
[-critical_path_slack {slack}]
[-irdrop_hierarchical_block {block1 block2 ... | *}]
[-irdrop_hierarchical_top_rail_analysis_directory {directory_name}]
[-reset]
[-respect_defined_nets {file_name}]
[-respect_routes {none | fixed | fixed_and_clock | all}]
[-respect_stdcell_geometry {none | sequential | all}]
[-timing_aware_effort {none | low | medium | high}]
```

Sets global variables for the `reinforce_pg` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `set_reinforce_pg_mode` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man set_reinforce_pg_mode`
`-auto_ir_fix_effort {low | medium | high}`	
	Specifies the IR fixing effort level during the stripe generation. Use this parameter to control the IR fixing effort. The number of fixed tiles is changed according to the effort settings. The higher the effort value, the larger number of tiles are passed back from Voltus to RPG for polygon insertion, which results in insertion of more polygons. ⚠ This parameter works only with the `reinforce_pg -auto_ir_fix` parameter. *Default*: `low`
`-critical_path_slack {slack}`	
	Specifies a slack threshold value (in nanoseconds) for critical nets during timing-driven PG stripe reinforcement. It means that `reinforce_pg` honors all the violation paths with slack value lesser than the specified `-critical_path_slack` value, since all such associated nets are considered critical. *Default:* `0`
`-irdrop_hierarchical_block {block1 block2 ... | *}`	
	Reports the IR drop violations on the specified list of hierarchical block(s) to fix IR drop. The names of the specified hierarchical blocks should be instantiations of the same master block, which is the loaded block. If you specify the `*` value of this parameter, all the instantiations of the loaded block are considered.
`-irdrop_hierarchical_top_rail_analysis_directory {directory_name}`	
	Reads the rail analysis information of full chip available in the specified directory for the hierarchical flow blocks. It collects the IR drop information of the specified block and uses it to fix the IR drop.
`-reset`	Resets all or specified parameters to their default values.
`-respect_defined_nets {file_name}`	
	When this parameter is used, `reinforce_pg` respects the user-defined nets listed in the specified file, which are critical in stripe generation. This helps in reserving space around the specified nets, and the newly added PG wires are DRC clean with these nets. This parameter is independent of the `-respect_routes` parameter. However, when it is used together with the `-respect_routes` parameter, `reinforce_pg` respects all of them. *Default*: ""

`-respect_routes {none | fixed | fixed_and_clock | all}`

Specifies the route types to honor check of pre-existing routes during stripe generation. This parameter always honor all covers and PGs, but does not honor the trail routes, Following are the parameter values:

- `none`: Specifies that no route types are respected.
- `fixed`: Respects fixed regular wires.
- `fixed_and_clock`: Respects fixed and routed clocks.
- `all`: Respects fixed and routed clocks along with the routed signals.

For more information on the support of the `set_reinforce_pg_mode -respect_route` parameter values with various statuses of net, see Support of set_reinforce_pg_mode -respect_route Parameter Values.

Default: `fixed_and_clock`

`-respect_stdcell_geometry {none | sequential | all}`

Specifies all types of geometries (including `pin` and `obs`) to be respected in via generation. This parameter checks the standard cell types to decide weather the DRC violation needs to be checked on its geometries (including `pin` and `obs`) in via generation.

Default: `all`

`-timing_aware_effort {none | low | medium | high}`

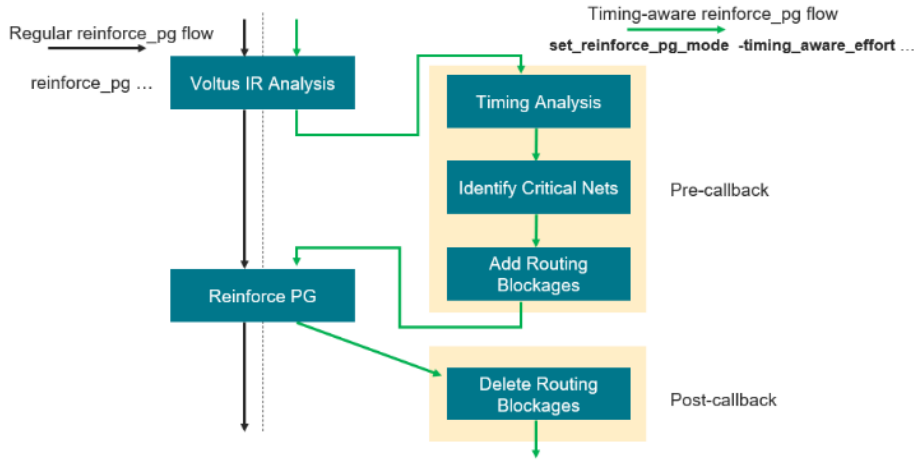
Specifies the timing aware effort level on critical paths of `reinforce_pg`. The `reinforce_pg` command performs timing analysis, when the `-timing_aware_effort` parameter is set to `low`, `medium`, or `high`. Use the `set_reinforce_pg_mode -critical_path_slack` parameter to determine the slack range.

When the `set_reinforce_pg_mode -critical_path_slack` parameter is not defined, following will be the behavior of the non-default values of `-timing_aware_effort`:

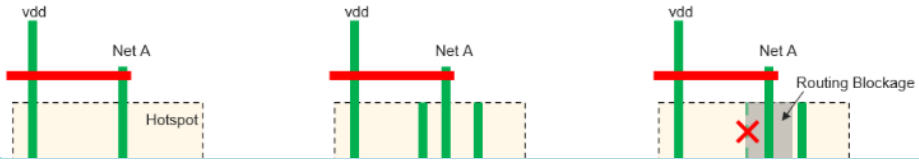
- `low`: Timing clock protection is performed, which uses slack value and reserve space size.
- `medium`: Performs clock protection, with `max_slack` as 10+ ps (since 10ps is used to cover the borderline paths), `max_path` as `number_of_flops`, and `nworst` as 1. You can set the `-critical_path_slack` parameter to overwrite this value, `max_path` to cover all the end points and single worst path per end point.
- `high`: Performs clock protection, with `max_slack` as 10+ ps (since 10ps is used to cover the borderline paths), `max_path` as `10 * number_of_flops`, and `nworst` as `10,2 * blockage_size`. You can set the `-critical_path_slack` parameter to overwrite this value, `max_path` to cover all the end points and single worst path per end point, with covering 10 worst paths per end point.

⚠ The `-critical_path_slack` parameter can work only when the `-timing_aware_effort` parameter is set to `low`, `medium`, or `high`.

See the figure below for understanding the timing aware reinforce PG flow:



The following figure presents reserving of spaces around critical net using with and without the `-timing_aware_effort` parameter:



Support of set_reinforce_pg_mode -respect_route Parameter Values

The following table presents the support of the `set_reinforce_pg_mode -respect_route` parameter values with various statuses of net:

Target PG Honoring a Wire			-respect_route Values			
Net Use	**Status**		**none**	**fixed**	**fixed_and_clock**	**all**
P/G			Yes	Yes	Yes	Yes
Non-P/G	**Cover**		Yes	Yes	Yes	Yes
	Fixed		No	Yes	Yes	Yes
	Routed	**Clock**	No	No	Yes	Yes
		Signal	No	No	No	Yes
	Unknown		No	No	No	No

Related Information

- `reinforce_pg`
- `get_reinforce_pg_mode`
- Power Planning and Routing chapter in the *Innovus User Guide.*

setAddRingMode

```
 setAddRingMode
[-help]
[-avoid_short {true | false}]
[-break_core_ring_io_list io_list]
[-continue_on_no_selection {true | false}]
[-detailed_log {true | false}]
[-extend_blockring_search_distance distance ]
[-extend_corering_search_distance float ]
[-extend_merge_with_prewires {true | false}]
[-extend_over_row {true | false}]
[-extend_search_nets {* | {net_names}}]
[-extend_stripe_search_distance float ]
[-gap_width_without_io width]
[-ignore_rows {true | false}]
[-max_via_size { shape width% height% target_penetration% }]
[-orthogonal_only {true | false}]
[-reset]
[-ring_target {core_ring | default | first_ring | pad_ring | stripe}]
[-skip_crossing_trunks {horizontal | vertical | none}]
[-skip_via_on_pin {[Pad] [Block] [Cover] [Standardcell] [Physicalpin]}]
[-skip_via_on_wire_shape {blockring | stripe | followpin | corewire | blockwire | iowire | padring | ring | fillwire | noshape}]
[-spacing_from_block value ]
[-split_long_via {threshold_value step_value offset_value height_value}]
[-stacked_via_bottom_layer layername ]
[-stacked_via_top_layer layername ]
[-via_using_exact_crossover_size {true | false}]
[-wire_center_offset {true | false}]
```

Sets global variables for block rings and core rings when you use the `addRing` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `setAddRingMode` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man setAddRingMode`
`-avoid_short {true | false}`	
	Specifies whether to prevent a ring or a ring segment from being created if it would cause a short violation to occur. Specify `1` to prevent the creation of rings that would cause a short violation. Specify `0` to enable the creation of rings that cause a short violation. *Default:* `false`
`-break_core_ring_io_list io_list`	
	Breaks or excludes core rings for specific instances.
`-continue_on_no_selection {true | false}`	
	If the design has no power domain, or no fences are selected, `addRing` ignores the option `-around power_domain` and generates core rings. Default: false
`-detailed_log {true|false}`	
	Setting this option will make `addRing`, `addStripe` output an extensive log or a detailed log. *Default:* false
`-extend_blockring_search_distance` *distance*	

	Extends the block ring search a specified distance beyond the initial block ring target to enable the connection of multiple block ring targets. For example, if VDD consists of three bits (segments) running parallel to one another, each having a width of two microns and a spacing between them of one micron, using this variable with a specified distance of eight microns will extend the block ring search a distance of eight microns to enable a connection to the multiple block ring targets. The *distance* is specified as the measured distance from the first to last bits. Units in microns.
	Note: You must specify this variable before using the addRing command. When using the command, you must include the following parameters:
	`-use_wire_group_bits 1`
	`-use_wire_group 1`

`-extend_corering_search_distance` *distance*	
	Specifies the wire group search distance for connecting all bits of the wire group stripes. *Default*: 0.0
`-extend_merge_with_prewires {true | false}`	
	Automatically merges newly-created ring wires with same net pre-routed wires. *Default*: `true`
`-extend_over_row {true | false}`	
	If 1 is specified, the cut row is not considered a blockage by the addRing command, which generates the ring over the cut row. If `0` is specified, the cut row is considered a blockage and the ring is not generated over the row. Rings are created based on the parameters that you specify in the addRings command. *Default: false*
`-extend_search_nets {* | {net_names}}`	
	Specifies the net names to append to the wire group search distance properties.
`-extend_stripe_search_distance` *distance*	
	Specifies the wire group search distance for connecting all bits of the wire group core rings. *Default*: `0.0`

`-gap_width_without_io  width`	
	Specifies the max missing IO Pads area width between IO Pads. *Default:* `0`
`-ignore_rows {true | false}`	
	Specifies whether the power planning software ignores all rows (cut rows and native rows) when rings are created. Specify `1` to create rings that ignore rows. Specify `0` to create rings that do not ignore rows. *Default:* `false`
`-max_via_size {`*`shape width% height% target_penetration%`*`}`	
	The maximum size of a crossover via, specified as a percentage of a full-size via. You must specify three values for each shape that you specify. The type values are: • Pad • Ring • Stripe • Blockring • Blockpin • Cover • Noshape The three values you must specify for each shape are: • The maximum width for the crossover via, as a percentage of a full-size via • The maximum height for the crossover via, as a percentage of a full-size via • The target penetration, which specifies a smaller size for vias connecting to targets of the specified shape.
`-orthogonal_only {true | false}`	
	Creates vias *only* for orthogonal wires and pin intersections. *Default*: If you do not specify this parameter or if you specify it with a value `true`, the software only creates vias for orthogonal wires and pin intersections.
`-reset`	Resets all parameters to their default values. Any values that do not already match the default value and are reset. will be reported.
`-ring_target {core_ring | default | first_ring | pad_ring | stripe}`	
`core_ring`	Ring wires can consider core rings as targets, even when core rings are inside the I/O pad area. Only rings are considered targets.
`pad_ring`	Ring wires can be extended to pad rings.
`first_ring`	Ring wires consider either the first core ring or the pad ring as a target.

`stripe`	Both rings and stripes are considered targets.
	Note: You can specify that rings are extended to both pad rings and to stripes. To do this you must specify the ring_target parameter twice, as show in the following example:
	`-ring_target stripe`
	`-ring_target pad_ring`

`-skip_crossing_trunks {horizontal | vertical | none}`	
	Removes the horizontal or vertical portion of rings (shared or inside edges) when used in conjunction with the addRing -block_rings -around shared_cluster command.
`-skip_via_on_pin {[Pad] [Block] [Cover] [Standardcell] [Physicalpin]}`	

	Prevents vias from being generated on the specified types of pins. If you specify `-skip_via_on_pin {}`, the power planning software makes connections to all five categories of pins. *Default:* If you do not specify this parameter, vias are generated for block pins, pad pins, and cover macro pins, but are not generated for standard cell pins.
`-skip_via_on_wire_shape {blockring | stripe | followpin | corewire | blockwire | iowire | padring | ring | fillwire | noshape}`	
	Prevents vias from being generated on the specified wire shapes. If you specify `-skip_via_on_wire_shape {}`, the power planning software makes connections to all five categories of wire shapes. *Default:* If you do not specify this parameter, vias can be generated on all wire shapes *except* no-shape wires.
`-spacing_from_block` *value*	
	To be used together with `-break_core_ring_io_list` *io list* to specify the spacing from block.
`-split_long_via {`*threshold_value step_value offset_value height_value*`}`	
	Splits vias longer than the specified length (`threshold_value`, in micrometers) into smaller vias with specified center-to-center spacing (`step_value`, in micrometers), left/bottom end offset (`offset_value`, in micrometers) and vertical/horizontal height (`height_value`, in micrometers) values . This is independent of the option `-split_via`. *Default:* If you do not specify this parameter, the software does not split vias. **Note:** If the offset_value is given as negative, the vias will be placed symmetrically on both ends. The default value of `offset_value` is `-1`.
`-stacked_via_bottom_layer` *layername*	
	Specifies the lowest layer in which vias can be stacked. *Default:* The lowest metal layer in the design.
`-stacked_via_top_layer` *layername*	
	Specifies the highest layer in which vias can be stacked. *Default:* The highest metal layer in the design.
`-wire_center_offset{true | false}`	
	Used for calculating the offset. When set to 1 `addRing` will use wire centre. When set to 0, `addRing` will use wire boundary. *Default*: false
`-via_using_exact_crossover_size {true | false}`	
	Specifies whether to generate partial vias of the exact crossover size. A value of `1` allows partial vias to be generated. A value of `0` prevents partial vias, and the software always generates full-size vias. *Default:* `true`

Example

The following command specifies that rings can use stripes as targets and that the rings generated around clusters of block rings with shared edges do not have horizontal wires between the blocks.

```
setAddRingMode -ring_target stripe -skip_crossing_trunks horizontal
```

Related Information

- `addRing`
- `getAddRingMode`
- Power Planning and Routing chapter in the *Innovus User Guide*

setAddStripeMode

```
setAddStripeMode
[-help]
[-reset]
[-allow_jog {none|{[padcore_ring ][block_ring ]}}]
[-allow_nonpreferred_dir {none | [blockring] [corering] [padring] [stripe]}]
[-area {lx ly ux uy | x1 y1 x2 y2 x3 y3 ...}]
[-break_at {none|{[block_ring] [selected_block] [unassigned_bump][overlap_ringpin] [outer_ring][outside_ringmacro]
[blocks_without_same_net]}}]
[-color_balance {same_color|alternate_color|none}]
[-continue_on_no_selection {true | false}]
[-detailed_log {true | false}]
[-domain_offset_from_core {true | false}]
[-extend_to_closest_target {ring|stripe|same_dir_stripe|area_boundary|none}]
[-extend_to_first_ring {true | false}]
[-ignore_block_check {true | false}]
[-ignore_blockring_when_breaking {true | false}]
[-ignore_DRC {true | false}]
[-ignore_nondefault_domains {true | false}]
[-inside_cell_allow_shift {true | false}]
[-inside_cell_only {true | false}]
[-keep_pitch_after_snap {true | false}]
[-max_extension_distance distance]
[-max_via_size {shape width% height% target_penetration%}]
[-merge_with_all_layers {true | false}]
[-mesh_via {true | false}]
[-offset_from_core {true | false}]
[-optimize_stripe_for_routing_track {shift|none}]
[-orthogonal_offset {[{edge1 orthogonal_offset1} {edge2 edge3 orthogonal_offset2} {edge4 edge5 … orthogonal_offset3 } …]|[all
orthogonal_offset]}]
[-orthogonal_only {true | false}]
[-over_row_extension {true | false}]
[-partial_set_thru_domain {true | false}]
[-recommend_width file_name]
[-remove_floating_stapling {true | false}]
[-remove_floating_stripe_over_block {true | false}]
[-remove_stripe_under_ring {true | false}]
[-respect_routes {none | fixed | fixed_and_clock | all}]
[-route_over_rows_only {true | false}]
[-rows_without_stripes_only {true | false}]
[-same_sized_stack_vias {true | false}]
[-skip_via_on_pin {[Pad] [Block] [Cover] [Standardcell] [Physicalpin]}]
[-skip_via_on_wire_shape {blockring | stripe | followpin | corewire | blockwire | iowire | padring | ring | fillwire | noshape}
[-spacing_type {edge_to_edge|center_to_center}]
[-spacing_from_block floating_point_value]
[-split_long_via {threshold_value step_value offset_value height_value}]
[-split_vias {true | false}]
[-split_wire_spacing floating_point_value]
[-split_wire_weight integer_0_to_10]
[-split_wire_width floating_point_value]
[-stacked_via_bottom_layer layerName]
[-stacked_via_top_layer layerName]
[-stapling_extend_to_minimum_spacing {true | false}]
[-stapling_nets_style {end_to_end | side_to_side | side_to_side_full_nets}]
[-stapling_shift {true | false}]
[-stop_at_closest_target {none|block_ring|core_ring|stripe}]
[-stop_at_last_wire_for_area {true | false}]
[-stripe_min_length {floating_point_value | stripe_width}]
[-stripe_min_width floating_point_value]
[-switch_cellname cellname]
[-switch_layer_overlap_length floating_point_value]
[-trim_antenna_back_to_shape {none | block_ring | core_ring | pad_ring | stripe}]
[-trim_antenna_max_distance distance]
[-trim_stripe {none|design_boundary|core_boundary}]
[-use_exact_spacing {true | false}]
```

```
[-use_point2point_router {true | false}]
[-use_stripe_width {true | false}]
[-via_using_exact_crossover_size {true | false}]
```

Sets global variables for power stripes.

Parameters

`-help`	
	Outputs a brief description that includes the type and default information for each `setAddStripeMode` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man setAddStripeMode`
`-allow_jog {none|{[padcore_ring ][block_ring ]}}`	
	Specifies the type that stripe can jog to connect. If set to `padcore_ring` and/or `block_ring`, the stripes are allowed to be on the same/different metal layer to connect to `padcore_ring`/`block_ring`. If set to `none`, the stripe is not allowed to jog on any layer. A straight/direct connection (no jog) with switch layer is allowed to make the connection. *Default:* `padcore_ring block_ring`
`-allow_nonpreferred_dir {none | blockring| corering | padring | stripe}`	
	Specifies whether to allow non-preferred direction to jog to target. When set to `none`, the non-preferred direction jogging is disabled. **Note:** 1. '0' or 'false' is not a legal value in this option. 2. There are five available option values in this option. 3. The value 'none' is exclusive to other option values. 4. The default value is `padring`.
`-area { lx ly ux uy | x1 y1 x2 y2 x3 y3 ...}`	

	Specifies the coordinates of the rectangular and/or rectilinear areas in which the stripes to be created. It appends the specified area to previous ones.
	Default:""
	Note:
	• If `setAddStripeMode -area` conflicts with `addStripe -area`, `addStripe -area` being on higher priority, `setAddStripeMode -area` is ignored.
	• If stripes are only to be created in the area in current command, reset the previous areas by one of the following three ways:
	`setAddStripeMode -reset -area`
	`setAddStripeMode -area {}`
	`setAddStripeMode -area ""`
	• When this parameter is used with `setAddStripeMode -domain_offset_from_core true`, the `-start_offset` value for specified area is measured from core on using `setAddStripeMode -area`.

`-break_at {none|{[block_ring] [selected_block] [unassigned_bump][overlap_ringpin] [outer_ring][outside_ringmacro] [blocks_without_same_net]}}`	

Specifies the break point of stripes.

Default: `none`

If any of the options `block_ring`, `selected_block`, `unassigned_bump`, `overlap_ringpin`, `outer_ring` or `outside_ringmacro` is specified, the stripe breaks when it encounters the specified structure that are not in the same wire group.

`block_ring`
Stripes break and connect to the block rings and ring pins if they are encountered.

Note: This parameter applies only to block rings that surround a block, and does not apply to block rings that surround rows or standard cells. If the block
is a ring macro, the stripe will break at the ring pin inside the block ring.

`selected_block`
Stripes stop at the block boundary when the block does not contain a blockage.

Stripes stop at the block boundary when the block contains a blockage and spacing from the block boundary to the edge of the blockage is greater than the minimum spacing value of that layer.

Stripes stop at the minimum spacing away from the blockage when the spacing from the block boundary to the edge of the blockage is less than the minimum spacing of that layer.

Note: If the selected block has block ring, stripes break and connect to the block ring.

`unassigned_bumps`
Stripes break at unassigned bumps. It prevents the unassigned bumps from being ignored and shorted with the power and ground nets by the stripes created in the same metal layer.

`overlap_ringpin`
Prevents stripes from overlapping with same direction ring pins of a different net.

Note: This option is only valid if the ring pin is more than half the length of the macro in the direction of the ring pin.

`outer_ring`
When a stripe encounters nested block rings or ring pins that are not in the same wire group, the stripe breaks at the outer ring. If not specified, the stripe breaks at the innermost ring.

Note: This parameter works only in conjunction with the following values of `setAddStripeMode -break_at`:

- `block_ring`
- `selected_block`

`outside_ringmacro`
Stripes break at a small distance outside the block if the selected block has ring pins.

Note: This parameter is available only if you also specify `setAddStripeMode -break_at selected_block`

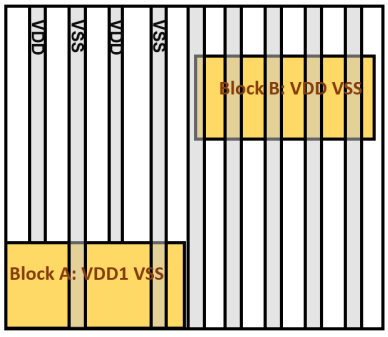
`blocks_without_same_net`
Breaks stripes at blocks, where no pin has same net as the nets specified in the `addStripe` command.

Example

Block A has power supply VDD1 and VSS, Block B has power supply VDD and VSS, commands are as follows:

```
setAddStripeMode -break_at blocks_without_same_net
addStripe -nets {VDD VSS}…
```

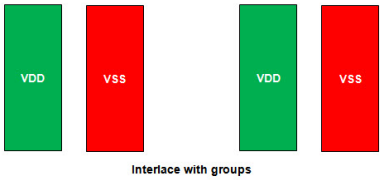
In the above commands, VDD breaks at Block A, VSS goes through Block A, VDD and VSS will go through Block B. Refer to the screen below:



`-color_balance {same_color | alternate_color | none}`

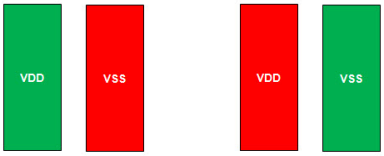
Specifies addition of VDD/VSS stripes with balanced color.

`same_color`
Corresponds to interlace with groups.



Interlace with groups

`alternate_color`
Corresponds to interlace within the group.



Interlace within groups

To achieve color balancing, the color of stripes occupying even number tracks is picked alternatively.

Note: The stripes location will always be either on full grid/track or half grid/track:

- If the stripes occupy odd number of tracks, they must be on full grid.
- If the stripes occupy even number of tracks, they must be on half grid.

To achieve color balance, the stripes color will be as mentioned below:

- If the stripes occupy odd number of tracks, the color is opposite to the center track color.
- If the stripes occupy even number of tracks, the color can be any of red or green and will keep alternating.

Only two nets are supported.

The option must be specified along with `-recommend_width`.

`-continue_on_no_selection {true | false}`

If the design has no power domain, or no fences are selected, `addStripe` ignores the option `-over_power_domain`, and generates stripes extending to core ring.

Default: `false`

`-detailed_log {true | false}`

Setting this parameter will make the `addStripe` output an extensive log or a detailed log.

Default: `false`

`-domain_offset_from_core {true | false}`

Creates stripes over a power domain to align with the global stripes in your design by enabling you to use the offset that was used for the global stripes as the offset for the power domain stripes. This parameter uses the offset from the core, which is the default for global stripes.

Default: `false`

`-extend_to_closest_target {none | stripe | ring | same_dir_stripe | area_boundary}`

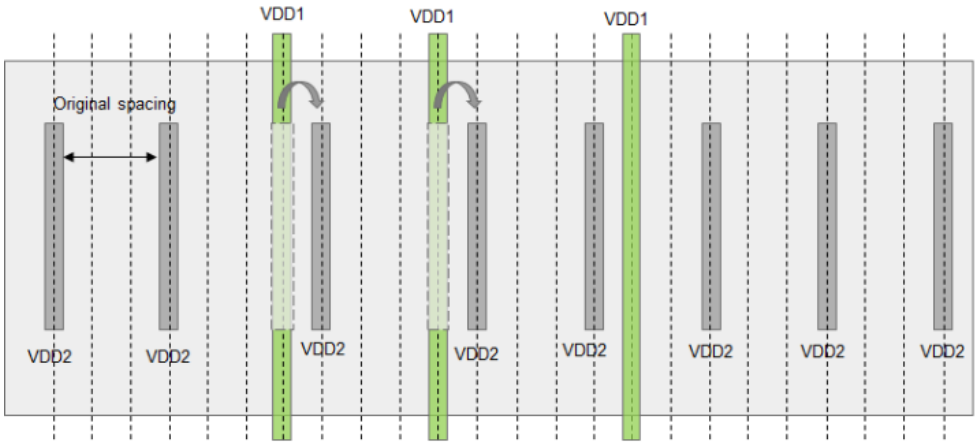
	Extends the stripe to the specified target. This parameter can work in conjunction with: `setAddStripeMode -max_extension_distance` Following is the description of the parameter values: • `none`: Restricts the stripe from extending to any target. • `stripe`: Allows the stripe to extend to the nearby stripe. • `ring`: Allows the stripe to extend to the nearby ring. • `same_dir_stripe`: Allows the stripe to extend to the stripe in the same direction. • `area_boundary`: Allows the stripe to extend to certain area boundary. *Default*: `none`
`-extend_to_first_ring {true | false}`	
	If set to `true`, stripe antennas are extended to the closest ring. If set to `false`, stripes stop at the boundary specified by the `-extend_to` parameter of the command. *Default*: `false`
`-ignore_block_check {true | false}`	
	If set to `false`, when a stripe encounters a ring, the software does not check whether that ring is surrounding a block. If set to `true`, when a stripe encounters a ring, the software automatically checks whether a block is enclosed within the ring. **Note:** This parameter only works in conjunction with the `-break_at` parameter of the `addStripe` command. *Default:* `false`
`-ignore_blockring_when_breaking {true | false}`	
	If set to `true`, breaks the stripe outside the block if the stripe cannot go over the block. If set to `false`, breaks the stripe at the block ring if it cannot go over the block. Default: `false`
`-ignore_nondefault_domains {true | false}`	
	Creates global stripes over domains without breaking. If you do not want the stripes to break at the non-default domains, set this variable to `true` before using the `addStripe` command. *Default*: `false`
`-ignore_DRC {true | false}`	
	Setting this parameter creates stripes ignoring DRCs but honoring user defined values. To be used for prototyping only. Default: `false`
`-inside_cell_allow_shift {true | false}`	

Allows to shift the newly created pillars when meeting DRC, if there is already an existing `special_wire` on the given track. This parameter works only when the `-inside_cell_only` parameter is set to `true`.

When the `-inside_cell_allow_shift` parameter is set to `false`, the pillar is removed when meeting violation. When set to `true`, the newly created pillar is shifted to right among the three tracks when meeting violation. The next pillar's position depends on the final position of the current pillar.

The distance between these pillars should satisfy the user specified spacing. The spacing between the two pillars should not be smaller than the user-specified spacing.

The following example shows that using the `-inside_cell_allow_shift` parameter, the newly created pillars of `VDD2` are shifted right to avoid DRC with `VDD1`.



Default: `false`

`-inside_cell_only {true | false}`

Control whether the stripe will be generated inside selected or specified instances. If no instance is selected or without the `addStripe -master`, no stripe will be generated.

Default: `false`

Note: When `setAddStripeMode -inside_cell_only true` is used with `setAddStripeMode -domain_offset_from_core true`, the `-start_offset` value for the value specified std-cell area is measured from core on using `setAddStripeMode -inside_cell_only true`.

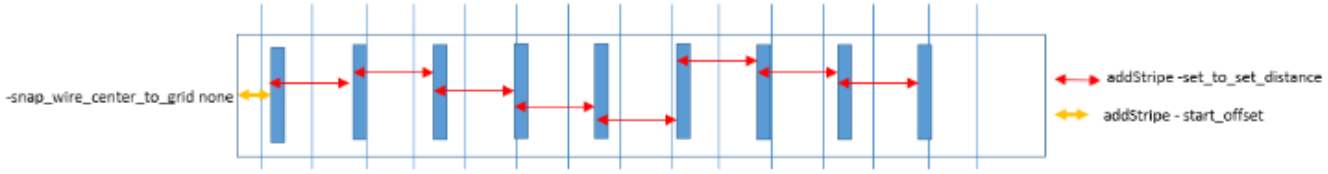
`-keep_pitch_after_snap {true | false}`

When set to `true`, the distance between two PG sets for snapping is set to greater than or equal to the pitch value specified through the `addStripe -set_to_set_distance` parameter. When set to `false`, the newly created PG sets are snapped from the current location to the nearest location valid for the `-keep_pitch_after_snap` parameter. The average pitch of the newly generated stripe is approximately equal to the specified `addStripe -set_to_set_distance` parameter value.

This parameter works with the `addStripe -snap_wire_center_to_grid` parameter. The behavior of the `-keep_pitch_after_snap` parameter under different scenarios is explained below.

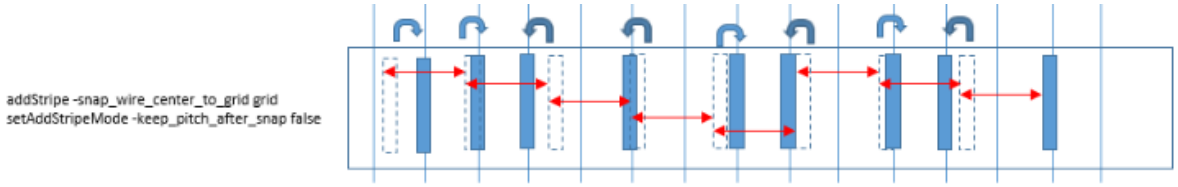
Scenario 1:

When the `addStripe -snap_wire_center_to_grid` parameter is set to `none`, the `setAddStripeMode -keep_pitch_after_snap` parameter does not work. See the example below for understanding:



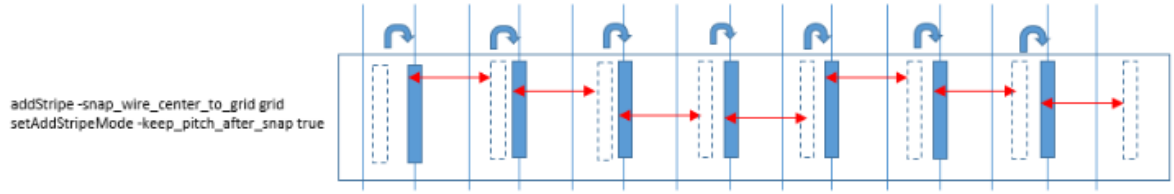
Scenario 2:

When the `addStripe -snap_wire_center_to_grid` parameter is set to `grid`, and the `setAddStripeMode -keep_pitch_after_snap` parameter is set to `false`, all the locations for snapping are decided based on the start location. Post then, the stripes are snapped to the nearest track. The average distance between of newly generated stripes is approximately equal to the specified pitch (`addStripe -set_to_set_distance` parameter value). See the example below for understanding:



Scenario 3:

When the `addStripe -snap_wire_center_to_grid` parameter is set to `grid`, and the `setAddStripeMode -keep_pitch_after_snap` parameter is set to `true`, the location for snapping is decided based on the previous location. Post then, the stripes are snapped to the next track. See the example below for understanding:



Note:

- The first stripe of the design always snaps to the next track to meet the `addStripe -start_offset` value. To start the first stripe snapping at the nearest track, set the `addStripe -allow_snapping_override_custom_spacing` parameter to `1`.
- By default, the `setAddStripeMode -keep_pitch_after_snap` parameter is turned on only at the `N10` nodes.

Default: `false`

`-max_extension_distance` *distance*

Sets the maximum extension distance between stripes to nearby stripes in the same layer and the same direction.

Default: `2.14748e+09`

`-max_via_size {shape width% height% target_penetration%}`

	The maximum size of a crossover via, specified as a percentage of a full-size via. You must specify three values for each shape that you specify. The shape values are: • Pad via size between stripe and pad pin/ring • Ring via size between stripe and core ring • Stripe via size between stripes • Blockring via size between stripe and block ring • Blockpin via size between stripe and block pin, including ring pin • Cover via size between stripe and the macro which has CLASS COVER defined in LEF • Selected via size between stripe and selected objects • No shape via size between stripe and wires with shape None • Standardcell via size between stripe and standard cell pin The three values you must specify for each shape are: • The maximum width for the crossover via, as a percentage of a full-size via • The maximum height for the crossover via, as a percentage of a full-size via • The target penetration, which specifies a smaller size for vias connecting to targets of the specified shape *Default*: ""
`-merge_with_all_layers {true | false}`	
	Merges a stripe to a block ring or pin on all layers. This parameter is similar to the `-merge_stripes_value` parameter of the `addStripe` command, except that it merges a stripe to all layers instead of only the stripe layer. *Default*: `false`
`-mesh_via {true | false}`	

Enables to use mesh via pillars to reduce memory.

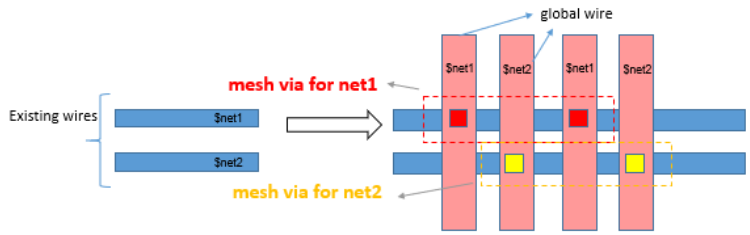
Default: `false`

Example 1:

The following commands use mesh via pillars with the help of the `-mesh_via` parameter:

```
setAddStripeMode -mesh_via 1
addStripe -nets {$net1 $net2 $net1 $net2} ...
```

Following is the output:

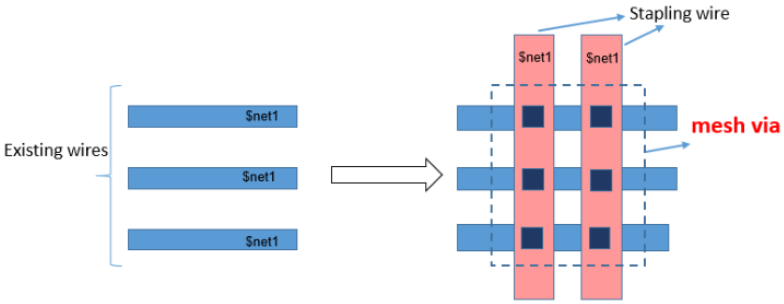


Example 2:

The following commands generate mesh via automatically in one stapling stripe:

```
setAddStripeMode -mesh_via 1
addStripe -stapling{ …} -nets {$net1 …$net1…} …
```

Following is the output:



`-offset_from_core {true | false}`

If set to `true`, the first or last stripe is offset from the core area of the design. If set to `false`, the first or last stripe is offset from core ring, pad ring, depending on the parameters specified with the option.

Note: This parameter does not affect the offset if you specify the `addStripe` command with any of the following parameters:

- `-over_power_domain 1`
- `-extend_to all_domains`
- `-area`

Default: `true`

`-optimize_stripe_for_routing_track {shift | none}`

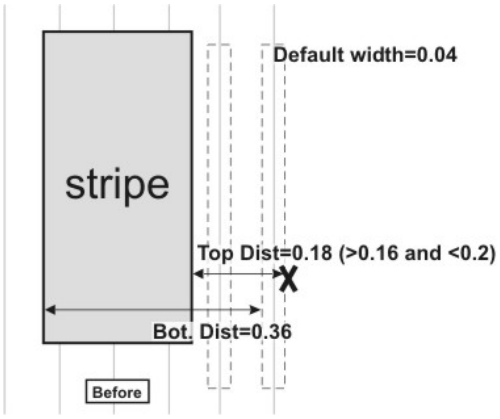
Specifies the location setting to move stripe center line to avoid the forbidden spacing. If both sides cannot be fixed at the same time, one side is always fixed.

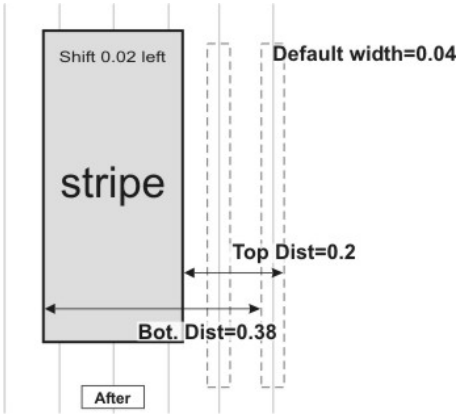
When set to `shift`, addStripe automatically shifts stripes to preserve routing resource.

Default: `none`

Example
For the rule below,

```
FORBIDDENSPACING 0.16 0.2 WIDTH 0.05 PRL 0;
```





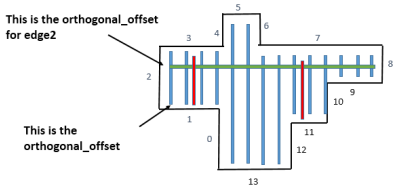
After shifting the stripe 0.02 left, Top Dist. and Bot. Dist. are >=0.2, and one track is saved.

`-over_row_extension {true | false}`

Extends the stripes to cover the followpin at the row end.

Default: `false`

`-orthogonal_offset {[{edge1 orthogonal_offset1} {edge2 edge3 orthogonal_offset2} {edge4 edge5 … orthogonal_offset3 } …]|[all orthogonal_offset]}`

	Defines `orthogonal_offset` of stripes for every edge of block level designs, blocks or standard cells for specifying the offsets for edges orthogonally. Edge number starts from the left bottom vertical edge. When this option is set for the second time, all the `orthogonal_offset` values are reset firstly.
	Default: `{none}`
	Example
	`setAddStripeMode -orthogonal_offset {{1 orthogonal_offset1} {2 orthogonal_offset2} {3 orthogonal_offset3} {5 orthogonal_offset5} {7 orthogonal_offset7} {8 orthogonal_offset8} {9 orthogonal_offset9} {11 orthogonal_offset11} {13 orthogonal_offset13}}`
	`addStripe -layer green …`
	`addStripe -layer blue …`
	`setAddStripeMode -orthogonal_offset {{1 orthogonal_offset1} {3 orthogonal_offset3} {7 orthogonal_offset7} {11 orthogonal_offset11}}`
	`addStripe -layer red …`
	 
	See the Examples section below for more examples on this parameter.

`-orthogonal_only {true | false}`	
	Creates vias *only* for orthogonal wires and pin intersections. *Default*: If you do not specify this parameter or if you specify it with a value of `1`, the software creates vias for orthogonal wires and pin intersections.
`-partial_set_thru_domain {true | false}`	
	Specifies whether to go over the power domain if domain contains the specified net. *Default:* `false`
`-recommend_width` *`file_name`*	
	The `file_name` specifies a list of recommended width values for the stripes. **Note:** This option must be specified along with `-color_balance`.
`-remove_floating_stapling {true | false}`	
	Removes those floating stapling stripes that are not connected to the power stripe/follow pin, when the stapling stripes are generated using a reference layer. This parameter does not remove floating stapling stripes when they are added through the user-specified offset and pitch. *Default:* `true`
`-remove_floating_stripe_over_block {true | false}`	
	If set to `true`, stripe fragments that start and end inside the same block or ring macro are removed. If set to `false`, stripe fragments are not removed. *Default:* `true`
`-reset`	Resets all parameters to their default values. Any values that do not already match the default value and are reset, will be reported.
`-remove_stripe_under_ring {true | false}`	
	Removes stripe segments, which have overlapped area and are in the same direction as the existing ring. The stripe can be in a different layer to the existing ring. Default: `false`
`-respect_routes {none | fixed | fixed_and_clock | all}`	

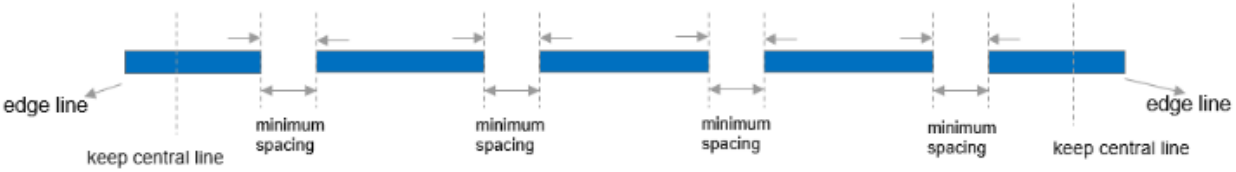
	Specifies the route types to ignore DRC check during stripe generation. This parameter always honor all covers and PGs, but does not honor the trail routes, Following are the parameter values: • `none`: Specifies that no route types are respected. • `fixed`: Respects fixed regular wires. • `fixed_and_clock`: Respects fixed and routed clocks. • `all`: Respects fixed and routed clocks along with the routed signals. For more information on the support of the `setAddStripeMode -respect_route` parameter values with various statuses of net, see Support of setAddStripeMode -respect_route Parameter Values. *Default*: `fixed_and_clock`
`-route_over_rows_only {true | false}`	
	If set to `true`, route stripes only over standard cells rows. If set to `false`, stripes are not limited to being routed over rows within this boundary. *Default:* `false`
`-rows_without_stripes_only {true | false}`	
	If set to `true`, global stripes are generated as specified by other parameters. Then stripes are also generated over row clusters that do not already contain stripes. If set to `false`, stripes are generated as specified by other parameters and not over row clusters without stripes. *Default:* `false`
`-same_sized_stack_vias {true | false}`	
	If set to `true`, the software always generates same-sized stacked vias. If set to `false`, the software trims the vias as needed to meet the DRC rules. *Default:* `false`
`-skip_via_on_pin {[Pad] [Block] [Cover] [Standardcell] [Physicalpin]}`	
	Prevents vias from being generated on the specified types of pins. If you specify `-skip_via_on_pin {}`, the power planning software makes connections to all five categories of pins. If you do not specify this parameter, vias are generated for block pins, pad pins, and cover macro pins, but are not generated for standard cell pins. *Default*: `Standardcell`
`-skip_via_on_wire_shape {blockring | stripe | followpin | corewire | blockwire | iowire | padring | ring | fillwire | noshape}`	
	Prevents vias from being generated on the specified wire shapes. If you specify `-skip_via_on_wire_shape {}`, the power planning software makes connections to all five categories of wire shapes. *Default:* If you do not specify this parameter, vias can be generated on all wire shapes *except* no-shape wires.
`-spacing_from_block` *float*	
	If the value is less than 0, over-the-block stripes end the specified distance from the boundary. If the value is greater than 0, global stripes break at the specified user units from the obstructed blocks. If the value is equal to 0, the stripe offset is 0. *Default*: 0
`-spacing_type {edge_to_edge|center_to_center}`	
	Specifies the type of spacing for `addStripe -spacing`. *Default:* `edge_to_edge`
`-split_long_via {`*threshold_value step_value offset_value height_value*`}`	

	Splits vias longer than the specified length (`threshold_value`, in micrometers) into smaller vias with specified center-to-center spacing (`step_value`, in micrometers), left/bottom end offset (`offset_value`, in micrometers) and vertical/horizontal height (`height_value`, in micrometers) values . This is independent of the `-split_vias` parameter. *Default:* If you do not specify this parameter, the software does not split vias. **Note:** If the offset_value is given as negative, the vias will be placed symmetrically on both ends. The default value of `offset_value` is `-1`.
`-split_vias {true | false}`	
	Specifies whether two or more partial vias can be created at the crossover between a stripe and any target (pin or wire) if an obstruction (macro obstruction; wire or pin on a different net) prevents the creation of a full-size via. This can be useful, for example, if several metal blockages partially obstruct the area where a full-size via would normally be placed. This parameter permits the creation of partial vias that would not violate the process rules, in the open areas. A value of `1` allows multiple partial vias to be created. A value of `0` prevents multiple partial vias from being created. *Default:* false
`-split_wire_spacing` *`floating_point_value`*	
	Specifies the spacing between each split wire. *Default*: If you omit this parameter, or set its value to `0`, the software uses the spacing rule in the LEF file.
`-split_wire_weight` *`integer_0_to_10`*	
	When split wires from a stripe on one layer attempt to make a target connection on another layer but are obstructed by I/O wires, some I/O wires might be deleted so that approximately *n* of 10 split stripe wires complete the target connection. **Note:** When you specify this option, the software deletes the fewest I/O wires to accommodate approximately *n* of 10 split stripe wires. *Default*: If you omit this parameter, or set its value to `0`, no I/O wires are deleted when the above connections are made.
`-split_wire_width` *`floating_point_value`*	
	Specifies the width of each split wire. *Default*: If you omit this parameter, or set its value to `0`, the software uses the maximum width value in the LEF file.
`-stacked_via_bottom_layer` *`layername`*	
	Specifies the lowest layer in which vias can be stacked. *Default:* The lowest metal layer in the design.
`-stacked_via_top_layer` *`layername`*	
	Specifies the highest layer in which vias can be stacked. *Default:* The highest metal layer in the design.
`-stapling_extend_to_minimum_spacing {true | false}`	

Automatically extends the newly created adjacent stapling to satisfy the end-to-end minimum spacing requirement, which will also be free from DRCs. This parameter works only with the stapling wires.

When set to to `false`, the length of the newly added stripe follows the length defined through the `addStripe -stapling` parameter. The following figure presents the placing of stripes when the parameter is set to `false`:

When set to `true`, the newly added stripes are extended automatically. The wire on the edge with only one end will have end-to-end spacing with the newly generated stripes. Only the end-to-end side is extended and the edge line end is not extended. The extended stripe still remains within the die area. The following figure presents the placing of the extended stripes when the parameter is set to `true`:



Default: `false`

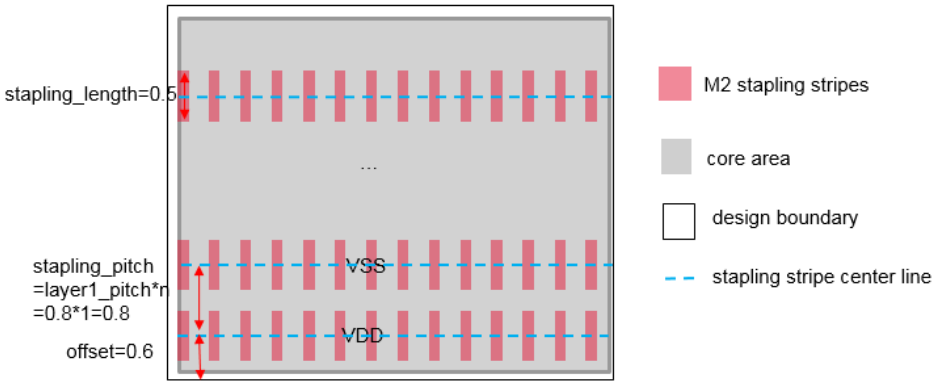
`-stapling_nets_style {end_to_end | side_to_side | side_to_side_full_nets}`

Specifies the stapling style, if multiple nets are used. It controls the stapling stripe arrangement style specified in `addStripe -nets`. The default value of this parameter is `end_to_end`. Please note that:

- When `end_to_end` (default value) is specified, nets are arranged in end direction. Refer to the following examples:

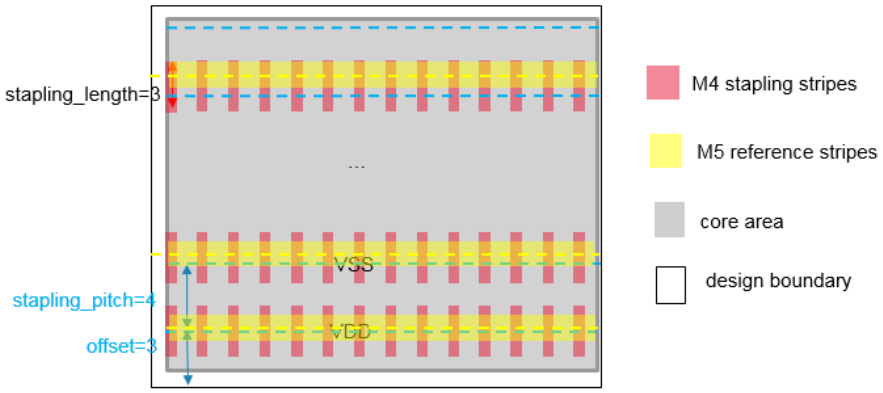
  Example 1

  ```
  addStripe -stapling {0.5 0.6 0.8:1} -nets {VDD VSS} -layer metal2 -width 0.1 -direction vertical -set_to_set_distance 0.5
  ```

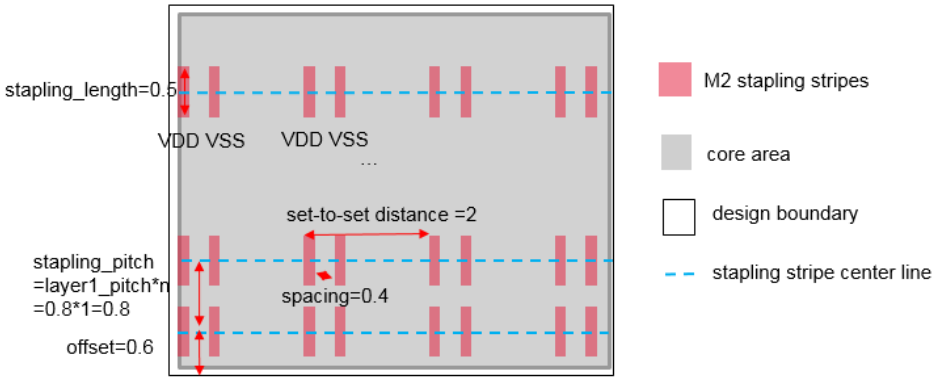
  

  Example 2

  ```
  set metal3_track_offset 3
  set metal3_pitch 0.8
  addStripe -stapling '3 $metal3_track_offset $metal3_pitch:5 metal5' -nets {VDD VSS} -layer metal4 -width 0.5 -direction
  vertical -set_to_set_distance 2
  ```



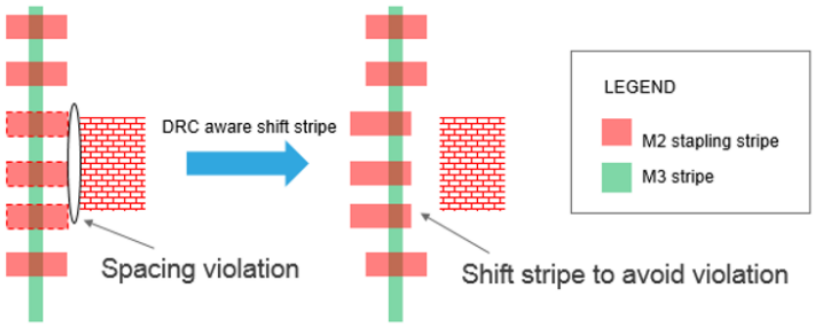
- When `side_to_side` is specified, nets are arranged in the side direction. Refer to the following example:
  Example 3
  ```
  addStripe -stapling {0.5 0.6 0.8:1} -nets {VDD VSS} -spacing 0.4 -layer metal2 -width 0.1 -direction vertical -
  set_to_set_distance 2
  ```



- When `side_to_side_full_nets` is specified, nets are arranged in the side direction. Besides the `side_to_side` behavior, it creates other net stapling according to the reference nets. The `side_to_side_full_nets` value can be generated only over the existing reference stripe (with `-stapling {length refer_layer}`). Refer to the following example:
  Example 4
  ```
  -stapling_nets_style side_to_side_full_nets #Besides side_to_side behavior, creating other net stapling according to the
  reference nets
  ```

`-stapling_shift {true | false}`

Shifts the stapling stripes automatically to avoid the DRC. See the example below:



Default: `true`

`-stop_at_closest_target {none | block_ring | core_ring | stripe}`

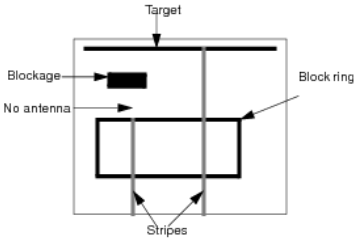
	Stops the stripe at closest target while drawing the stripe on the master cell. *Default*: `none`
`-stop_at_last_wire_for_area {true | false}`	
	If set to `false`, the stripe antennas are trimmed back to the closest wire or pin of the same net. If set to `true`, stripe antennas are not trimmed. **Note:** This option is only available if you specify the `-area` parameter. *Default:* `false`
`-stripe_min_length {floating_point_value | stripe_width}`	
	Specifies the minimum stripe length. `stripe_width` is a fixed string. The fixed string here refers to the width of present wire. *Default*: `stripe_width`
`-stripe_min_width floating_point_value`	
	Specifies the minimum width of the remaining stripe after being trimmed. *Default*: `0.0`
`-switch_cellname {cellAcellB ...}`	
	Treats the specified switch cell as a ring. Pins in the switch cell can then be used as targets for stripes. If the switch cell is in the core area, the software treats it as a block ring. If the switch cell is outside of the floorplan row, the cell is treated as a core ring, and used as the stripe boundary. You can specify one or more switch cell names. To turn off this functionality, specify one of the following: • `-switch_cellname ""` • `-switch_cellname none` *Default*: `none`
`-switch_layer_overlap_length floating_point_value`	
	Controls the overlap distance used for the via between a stripe layer and the layer being switched to. **Note:** The overlap applies *only* to vias created for straight (non-jogging) connections. *Default*: If you omit this parameter, or set its value to `0`, the power planning software creates a square via.
`-trim_antenna_back_to_shape {none | block_ring | core_ring | pad_ring | stripe}`	

Prevents stripes from creating antennas when a wire of the specified shape is the last target before a blockage that prevents connection to another target. Specify one of the following shapes:

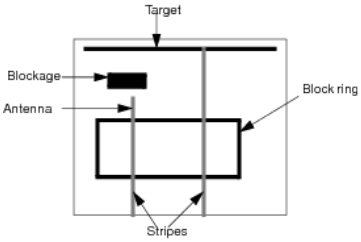
- `block_ring`
- `core_ring`
- `pad_ring`
- `stripe`

Default: none

The following figure illustrates the software behavior if you specify this parameter with a value of `block_ring`.



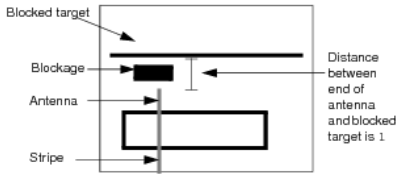
Default: Stripes extend to the blockage, leaving an antenna between the last connection and the blockage. The following figure illustrates the software behavior if you do not specify this parameter.



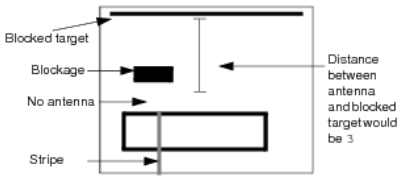
`-trim_antenna_max_distance` *distance*

Use this option to specify a distance, in micrometers, between the end of the antenna and a blocked target. This value controls whether an antenna is kept or trimmed for the selected shape. Cadence recommends that you specify a value equivalent to the distance between the core area and the pad ring or core ring.

- If the blocked target is within the specified distance, the antenna is kept, and you can use the SRoute software to complete the connection. For example, if you specify a distance of `2` micrometers and the distance between the end of the antenna and the blocked target is `1` micrometer, the antenna is not trimmed.



- If the blocked target is beyond the specified distance, the antenna is trimmed back to the last connection. For example, if you specify a distance of `2` micrometers and the distance between the antenna and the blocked target would be `3` micrometers, the antenna is trimmed back.



Default: `0.0`

`-trim_stripe {none | design_boundary | core_boundary}`

Specifies if shape is trimmed by the specified shape.

Default: `None`

`-use_exact_spacing {true | false}`

If set to `true`, the power stripe breaks using the exact spacing rule from the obstructing blockage, pin, or wire to avoid a design rule violation. If set to `false`, the power stripe breaks at 1.5 times of the maximum range rule spacing of all layers.

Default:

- `true`: For advance/N7 nodes
- `false`: For non-advance/non-N7 nodes

`-use_stripe_width {true | false}`

If set to `true`, stripes that jog to connect to a ring always use the stripe width for the jog. If set to `false`, the jog is either the width of the stripe or the width of the ring segment, which ever is smaller.

Note: This parameter only works in conjunction with the following parameter of the `addStripe command`:

`-jog_to_connect` 1

Default: `true`

`-use_point2point_router {true | false}`

If set to `false`, the point-to-point router engine is turned off. If set to `true`, the `addStripe` command automatically calls the internal point-to-point router engine to route the difficult power stripe connections (such as connecting power stripe antennas that were not connected by the `addStripe` command).

Default: `false`

`-via_using_exact_crossover_size {true | false}`

	Specifies whether to generate partial vias of the exact crossover size. A value of `1` allows partial vias to be generated. A value of `0` prevents partial vias, and the software always generates full-size vias. *Default:* `true`

Support of setAddStripeMode -respect_route Parameter Values

The following table presents the support of the `setAddStripeMode -respect_route` parameter values with various statuses of net:

Target PG Honoring a Wire			-respect_route Values			
Net Use	**Status**		**none**	**fixed**	**fixed_and_clock**	**all**
P/G			Yes	Yes	Yes	Yes
Non-P/G	**Cover**		Yes	Yes	Yes	Yes
	Fixed		No	Yes	Yes	Yes
	Routed	**Clock**	No	No	Yes	Yes
		Signal	No	No	No	Yes
	Unknown		No	No	No	No

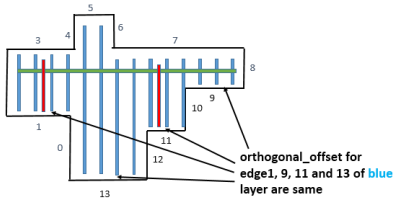
Examples

The following command specifies that stripes that jog to connect to a target always use the stripe width.

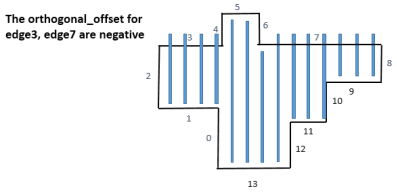
```
setAddStripeMode -use_stripe_width 1
```

The following examples include commands for specifying that stripes are generated in the block with a specified orthogonal offset:

```
setAddStripeMode -orthogonal_offset {{1 9 11 13 orthogonal_offset1} {2 orthogonal_offset2} {3 orthogonal_offset3} {5 orthogonal_offset5}
{7 orthogonal_offset7} {8 orthogonal_offset8}}
addStripe -layer green …
addStripe -layer blue …
setAddStripeMode -orthogonal_offset {{1 orthogonal_offset1} {3 orthogonal_offset3} {7 orthogonal_offset7} {11 orthogonal_offset11}}
addStripe -layer red …
```




```
setAddStripeMode -orthogonal_offset {{3 7 orthogonal_offset3} {1 5 9 11 13 orthogonal1}}
addStripe -layer blue …
```



Related Information

- `addStripe`
- `getAddStripeMode`
- Power Planning and Routing chapter in the *Innovus User Guide*.

setViaGenMode

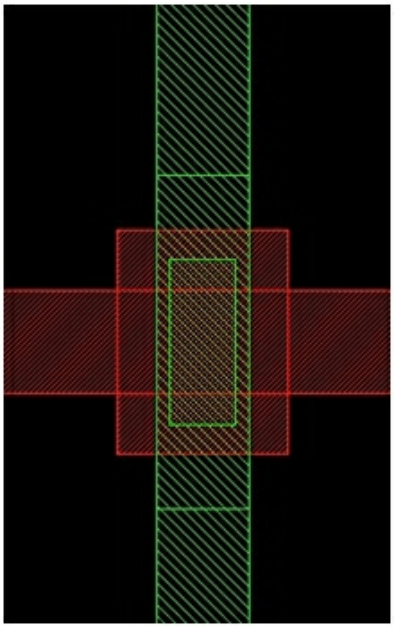
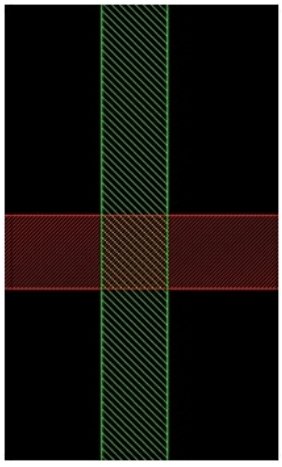
```
setViaGenMode
[-help]
[-reset]
[-accuracy_effort {low | medium | high}]
[-add_pin_to_pin_via {true | false}]
[-align_merged_stack_via_metals {true | false}]
[-allow_via_expansion {true | false}]
[-allow_wire_shape_change {true | false}]
[-area_only {true | false}]
[-create_double_row_cut_via {0 | 1 | 2}]
[-create_max_row_cut_via {true | false}]
[-cutclass_preference {default | {[square] [bar] [large]} | cutclass_name_list} | file_name]
[-disable_via_merging {true | false}]
[-extend_out_wire_end {true | false}]
[-full_cut_via_only {true | false}]
[-genvia_naming_prefix viaNamePrefix]
[-hookup_contact_max {distance1_max distance2_max}]
[-hookup_contact_pg_track {start_value pitch_value}]
[-hookup_rail_pair {0 | 1}]
[-hookup_via_distance {distance_value1 [distance_value2 [boundary_spacing]]}]
[-hookup_via_fixed_grid {true | false}]
[-hookup_via_min_distance distance]
[-hookup_via_style {none | loose | compact | moderate}]
[-hookup_via_viarule {filler {default |{list_of_via rule/cell_names }} ...}]
[-hookup_virtual_trim_grid {default |{-layer -mask1 {-pitch -core_offset...}}}]
[-ignore_design_boundary {true | false}]
[-ignore_DRC {true | false}]
[-ignore_viarule_enclosure {true | false}]
[-inherit_wire_status {true | false}]
[-invoke_verifyGeometry {true | false}]
[-keep_existing_via {0 | 1 | 2}]
[-keep_fixed_via {true | false}]
[-mincut_preference value]
[-optimize_cross_via {true | false}]
[-optimize_via_on_routing_track {true | false}]
[-parameterized_via_only {true | false | auto}]
[-partial_overlap_threshold float]
[-preferred_vias_only {use_lef | open | keep}]
[-reference_boundary {design|core}]
[-respect_signal_routes {0 | 1 | 2}]
[-respect_stdcell_geometry {true | false}]
[-respect_wire_ndr {true | false}]
[-set_via_expansion_dir {auto | horizontal | vertical}]
[-snap_via_center_to_grid {{ layer1 {none | grid | half_grid | either } [hard]} layer2 {{none | grid | half_grid | either } [hard]} … }]
[-split_long_via_global_grid {x_offset x_pitch x_length_threshold x_length_multiplier y_offset y_pitch y_length_threshold
y_length_multiplier}]
[-symmetrical_via_only {true | false | auto}]
[-use_track_offset {true | false}]
[-use_trim_metal_enclosure {true | false}]
[-viarule_preference {default | predefined | generated | list of via rule/cell names | file_name}]
```

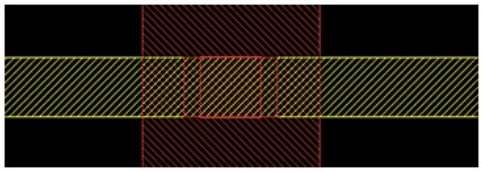
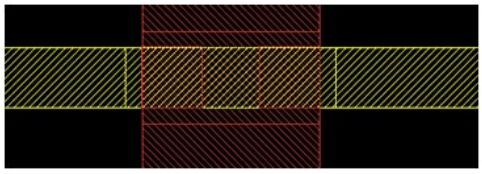
Sets global variables for vias that connect rings and stripes when you use the `addRing`, `addStripe`, `editPowerVia`, `sroute`, `addSplitPowerVia` or `editAddRoute` commands.

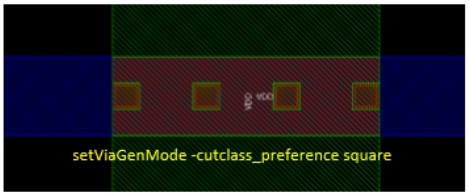
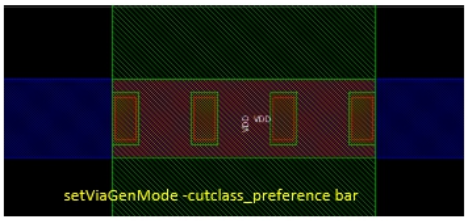
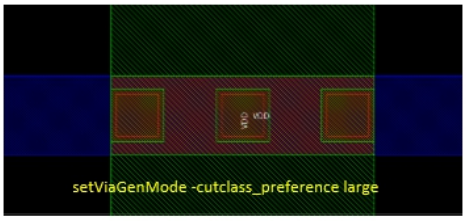
Parameters

`-help`	Outputs a brief description that includes the type and default information for each `setViaGenMode` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man setViaGenMode`
`-accuracy_effort {low | medium | high}`	

	Controls the balance between accuracy and runtime during via generation. • When set to `high`, viagen checks full DRC rules and creates DRC clean vias. • When set to `medium`, viagen runs DRC check in the moderate mode. • When set to `low`, viagen runs in the quick DRC check mode. With this value specified, some available vias may be missed, since the conservative rules are checked. The vias generated through this parameter are DRC clean. This parameter generates vias with reduced runtime in the special scenarios where complex DRC rules are triggered. These complex rules include (but are not limited to) `same metal aligned cuts spacing`. *Default:* `high`
`-add_pin_to_pin_via {true | false}`	
	Inserts vias to connect pins between different cover macros. *Default*: `false`
`-align_merged_stack_via_metals {true | false}`	
	Aligns merged stack vias. *Default:* `false`
`-allow_via_expansion {true | false}`	
	Allows via to expand out of intersection area to meet LEF rules. *Default:* `true`
`-allow_wire_shape_change {true | false}`	

	Allows the via to change the metal shape of existing wires. *Default:* true **Note:** For stack vias, the option is only valid for top and bottom metal layers. **Example:** • `-allow_wire_shape_change true`  • `-allow_wire_shape_change false` 
`-area_only {true | false}`	
	Adds vias only within the specified areas. *Default:* `false`
`-create_double_row_cut_via {0 | 1 | 2}`	

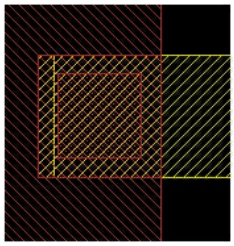
	Try to expand vias to have one additional cut than original when set the option to `1` or `2`. That is, expand original vias of `1xn` or `nx1` cuts to vias of `1x(n+1)` or `(n+1)x1` cuts. • 1: create vias with one additional cut where wires on both top and down layers are enough and need no shape change. • 2: create vias with one additional cut where wires on one of the layers are enough. *Default*: `0` **Example:** • `create_double_row_cut 0`  • `create_double_row_cut 1` 
`-create_max_row_cut_via {true | false}`	
	Try to expand vias to have one additional row or column of cuts than original when the option is set to true. It works similar to `-create_double_row_cut_via` except that there is no restriction on one row or one column via. *Default*: `false`
`-cutclass_preference {default | {[square] [bar] [large]} | cutclass_name_list | file_name}`	

	Specifies the preference for the via cut class while inserting special vias. You can specify one of the following values: • `default`: Prefers to use the first viacell in tech LEF without drc violation or the first generate viarule to generate maximum cut area. For example, `cutWidth * cutHeight * cutNumber`. • `square`: Prefers to use the SINGLE cut vias. In the LEF definition, the cut width and length are usually the same and the smallest in that layer. For example, 0.032 by 0.032. • `bar`: Prefers to use the DOUBLE cut vias. In the LEF definition, the cut width is the usually the smallest. For example, 0.032 by 0.08. It mostly equals to 2 SINGLE cut. • `large`: Perfers to use the LARGE cut vias. In the LEF definition, the neither the cut width not the length is the smallest. For example, 0.08 by 0.08, equals to 4 SINGLE cut. The following figure shows the difference between the square, bar, and large values:    **Note:** 1. Parameter setting does not support incremental specification. The last cutClass preference setting is used. 2. When you specify `{[square] [bar] [large]}`, there should be 2 or 3 cutClass defined in the LEF file. If Innovus detects that there are more than 3 or less than 2 cutClasses, the setting is ignored. 3. In the specified file, different arguments can be separate by space or line break.
`-disable_via_merging {true | false}`	
	Disables via merging in the layers next to the followpin layer within the range of two when set the option to true and there is followpin layer in the design. Triming will be the only way to solve drc issues. For example, if the followpin layer is in 1, disable via merging in 1/2/3; if the followpin layer is in 2, disable via merging in 1/2/3/4. This function helps if the merging of via consums additional wire track unnecessarily. *Default:* • `false` in normal scenarios • `true` for advance nodes (with designs of 12 nm or below)

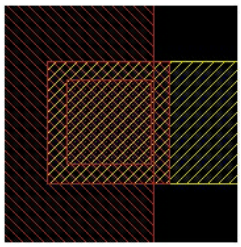
`-extend_out_wire_end {true | false}`

Allows via extending outside of existing wire ends at T-type of intersection. When set to false, via extending outside of the existing wire end are not allowed and may shift inside.

- `-extend_out_wire_end 1`



- `-extend_out_wire_end 0`



Default: `true`

`-full_cut_via_only {true | false}`

Allows the user to create only full cut vias.

Example
The following via is generated because of the second left cut will have drc violations when the option is set to false.

- `-full_cut_via_only 0`

When set the option to true, only full cut vias will be generated:

- `-full_cut_via_only 1`

&

Default: `false`

`-genvia_naming_prefix` *`viaNamePrefix`*

Sets the specified prefix name as the prefix for all generated vias created by the Innovus software.

This helps user to select and edit the vias that need further modification.

Default: `none`

`-hookup_contact_max {`*`distance1_max distance2_max`*`}`

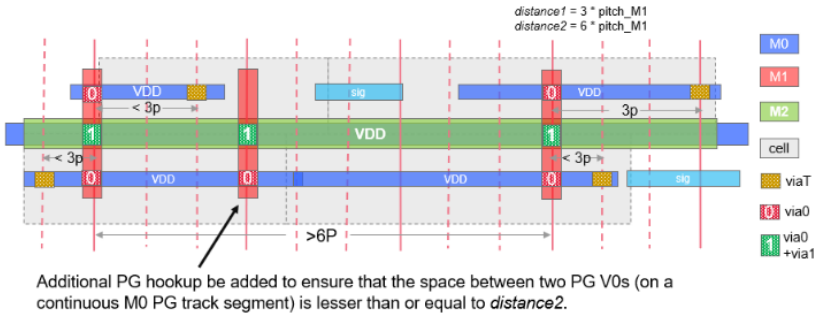
Sets the maximum distance to create hookup via from contact, and the maximum distance between two hookup via on a continuous `M0` PG pin. Following are the parameter values:

- `distance1_max`: Specifies the distance required by every viaT for at least one hookup via drop. For deciding the location of hookup via for `distance1_max`, every viaT needs at least one hookup via drop within `distance1`.
- `distance2_max`: Specifies the maximum distance between any two PG hookup vias on a continuous `M0` PG pin. An additional hookup via must be dropped from the previous hookup via on a continuous `M0` PG segment with `distance2_max`. Also, an additional hookup via must be dropped within the maximum `distance2` from the endpoint of a continuous `M0` PG pin.

Default: `0 0`

The following scenarios depict the usage of this parameter:

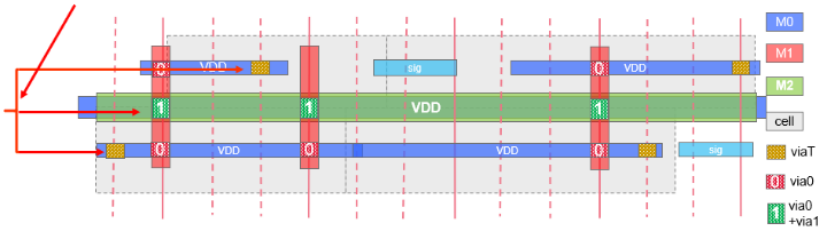
- Deciding the location of hookup via by viaT:



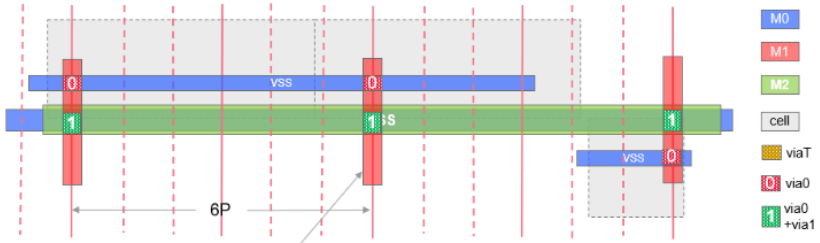
Additional PG hookup be added to ensure that the space between two PG V0s (on a continuous M0 PG track segment) is lesser than or equal to *distance2*.

- Referencing of the location of viaT by the tool to add hookup vias:

The location of hookup via is decided by viaT. The tool references only the viaT, which aligns with the cuts defined by via cells.



- At least one hookup via needs to be available for each continuous `M0` PG segment, including PG segment at both sides of the row edge and the PG pin. The continuous `M0` PG segment without viaT also needs to follow the decision rule of `distance2`:



Add an additional hookup via to meet 6P max distance

- Four individual via cells are defined in the LEF file and VIAMAP. Of these, one is 3-cut, two are 2-cut, and the remaining one is 1-cut. The further breakup of these via cells is mentioned below:
  - 3-cut via cells: Include three `v0` cuts, three `m0` metals, and one `m1` metals
  - 2-cut via cells: Include two `v0` cuts, two `m0` metals, and one `m1` metals
  - 1-cut via cell: Include one `v0` cuts, one `m0` metals, and one `m1` metals

`-hookup_contact_pg_track {start_value pitch_value}`

Sets start point and the pitch of PG track used by hookup contact. Following are the parameter values:

- `start_value`: Specifies the location of the first PG track.
- `pitch_value`: Specifies the distance (steps) between two adjacent PG tracks. It needs to be an integer multiple of the `m1` pitch.

Note: The PG track needs to be aligned with the `m1` track.

Default: `0 0`

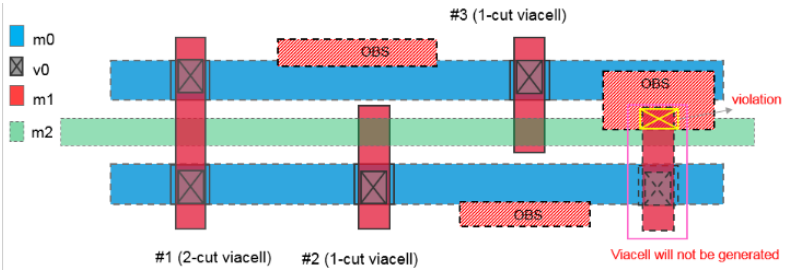
`-hookup_rail_pair {0 | 1}`

Adds hookup via over the pairing `M0` followpins that are distributed on both sides of a row with specially defined viacells at LEF file. Note that while using this parameter, the PG pins need to be on track, and the width of the `M0` followpin needs to be uniform.

This parameter works only with setting the `loose` value of the `-hookup_via_style` parameter. Three viacells are defined in the LEF file. One of those is 2-cut, and the remaining two are 1-cut. Refer to their details below:

- 2-cut viacell = Two `v0` cuts + Two `m0` metals + One `m1` metals
- 1-cut viacell = One `v0` cuts + One `m0` metals + One m1 metals

For the 1-cut viacell, `m1` needs to overlap completely with `M2`. If this is not possible, the viacell will not be generated. Refer to the example below:



Default: `0`

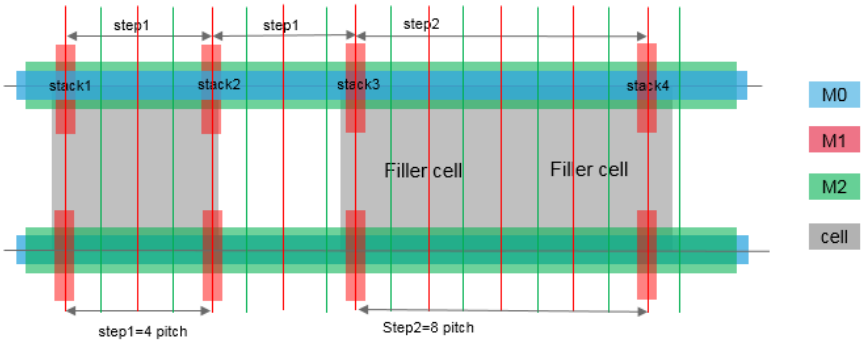
`-hookup_via_distance {distance_value1 [distance_value2 [boundary_spacing]]}`

Enables control to put the first pg via for each followpin. You can use this parameter in the following ways:

```
-hookup_via_distance {distance_value1}
-hookup_via_distance {distance_value1 distance_value2}
-hookup_via_distance {distance_value1 distance_value2 boundary_spacing}
```

Following are the parameter's various possible values:

- *`distance_value1`*: Specifies the distance (step) between via stacks and is rounded down to `M1 pitch*n`. Default value is `M1 pitch*4`.
- *`distance_value2`*: Specifies the distance between via stacks on filler cell. This value can work with all hookup styles. Default value is `M1 pitch*4`. It indicates the via distance when two neighboring via stacks are over filler cells. Refer to the figure below:



- *`boundary_spacing`*: Specifies the distance between the first via and core boundary. It also refers the minimum distance between the last via and core boundary.

`-hookup_via_fixed_grid {true | false}`

When set to `false` (default value), the added hookup via can be shifted to avoid DRC. When set as `true`, the hookup via can only be added at the fixed grid.

The fixed grid works with `-hookup_via_style`. For the `loose` and `moderate` styles, the hookup via will be removed, if it has DRC, when created. For the `compact` style, the hookup via will be kept, whether or not it has DRC.

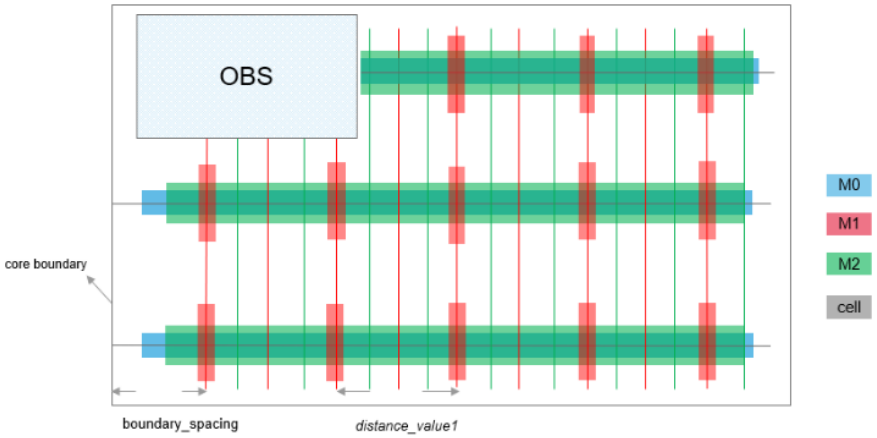
The fixed grid is controlled by the following parameter:

`-hookup_via_distance {distance_value1 [distance_value2 [boundary_spacing]]}`

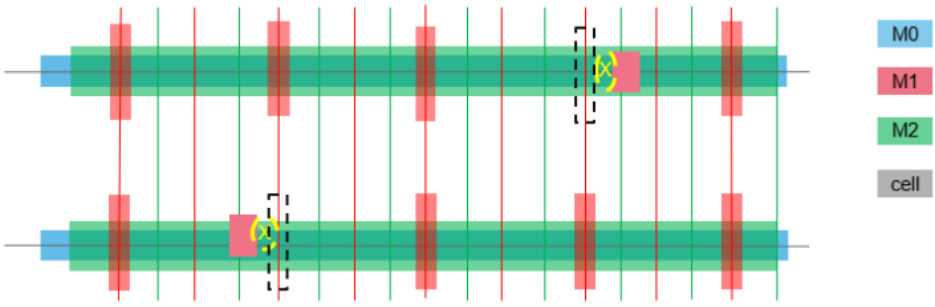
Here:

- `distance_value1` is the pitch of grid.
- `boundary_spacing` is the distance between the first grid and core boundary.
- The fixed grid does not get influenced by the shape of the design or macros.
- `distance_value2` needs to be set same as `distance_value1`.

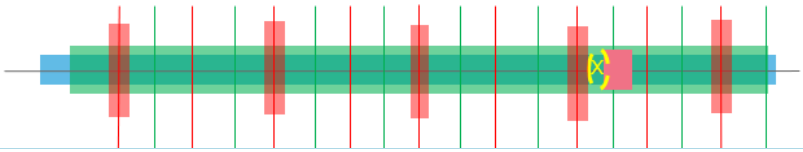
Below is an example of formation fixed grid using `boundary_spacing` and `distance_value1`. Note that all the vias of different lines need to be aligned.



Below is an example of DRC control for the `loose` and `moderate` types of hookup via styles. Here, the hookup vias are removed, if DRC exists.



Below is an example of DRC control for the `compact` type of hookup via style. Here, the hookup via will be kept, irrespective of the existence of DRC.



`-hookup_via_min_distance` *`distance`*

Specifies the minimum constrained distance between two PG vias. This parameter works with the distance between:

1. Two newly added PG vias
2. Existing PG vias and newly added PG vias on the followpin

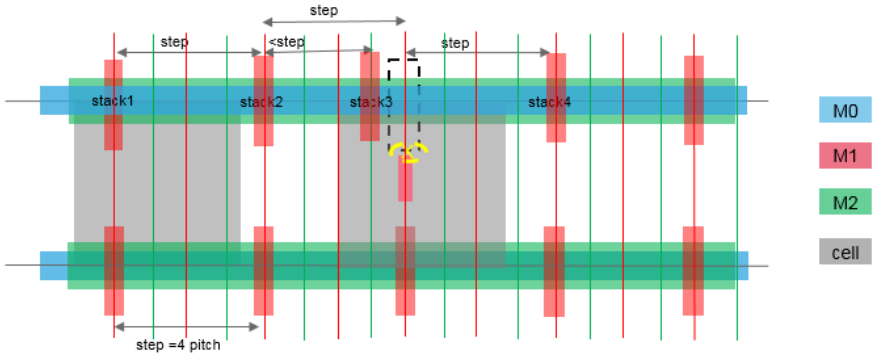
Default: `min`

`-hookup_via_style {none | loose | compact | moderate}`

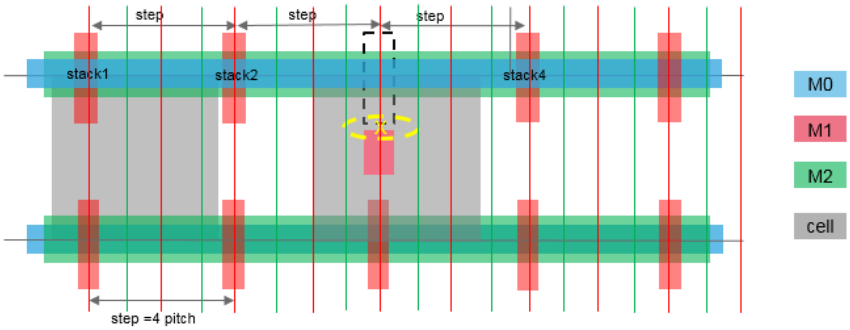
Sets hookup via style. The hookup via works between two parallel metal layers (like `M0` and `M2`, or `M1` and `M3`). Assuming it for `M0` and `M2`, viagen adds stack vias between `M0` and `M2` according to the hookup via settings.

The associated functionality of various options is provided below:

- `none`: No via hookup style is set. This is the default value.
- `loose`: The tool inserts vias between paralleled wires with the specified step in `-hookup_via_distance`. The center of the left-most via needs to be on the track, which is inside the intersection. The via also needs to be inside the intersection. If a via stack (like `stack3`) cannot be dropped at a certain step position (dot line area in the figure below) because of the DRC violation, it is moved to a closer or farther position with the previous stack (`stack2`) to ensure DRC clean. Additionally, the next via (`stack4`) needs to keep the step value with the original position of moved stack (dot line area in the figure below).

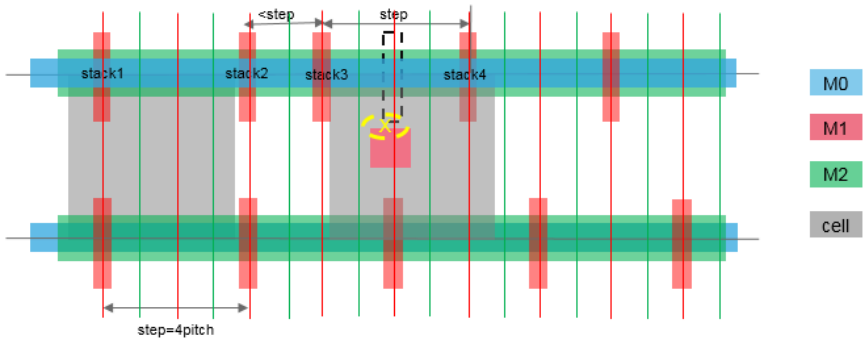


If DRC clean does not occurs, the via stack is removed, refer to the figure below:

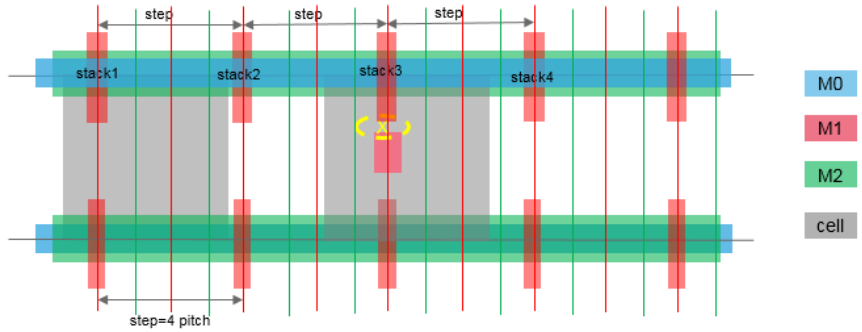


Note: The hookup style is `loose` by default, if the via map table does not exist.

- `compact`: If a via stack (like `stack3`) cannot be dropped at a certain step position (dot line area in the figure below) because of DRC violation, it is moved to a closer position with the previous stack (`stack2`) to ensure DRC clean. And the next via (`stack4`) needs to keep the step value with the said via (`stack3`). Refer to the figure below:

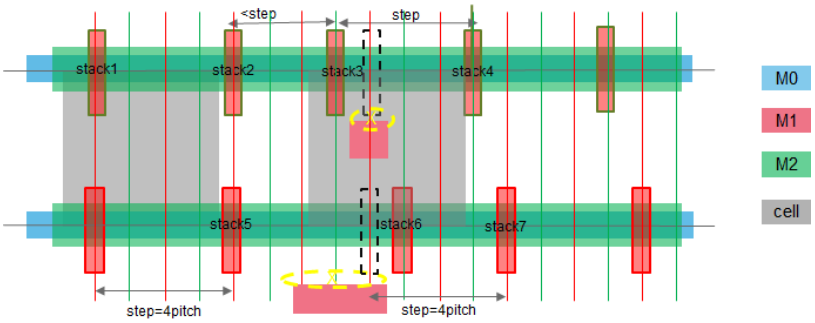


If DRC clean does not occurs by moving the stack, insert the via stack at the original position with DRC violation marker. Refer to the figure below:

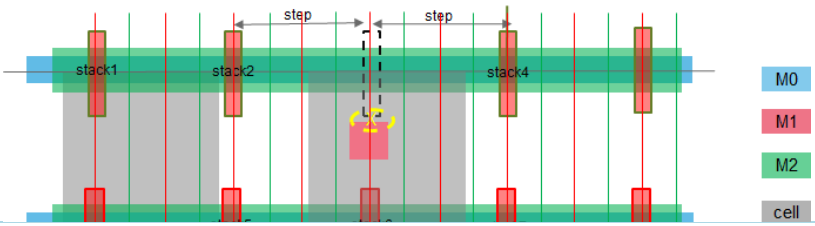


Note: The hookup style is `compact` by default, if the via map table exists.

- `moderate`: If a via stack (like `stack3`) cannot be dropped at a certain step position (dotted line area in the figure below) due to the DRC violation, it is moved to a closer or farther position with the previous stack (`stack2`) to ensure DRC clean. If the via stack (`stack3`) is dropped to a closer position with the previous stack (`stack2`), the next via (`stack4`) needs to keep the step value with the said stack (`stack3`). If the via stack (`stack6`) is dropped to a farther position with the previous stack (`stack5`), the next via (`stack7`) needs to keep the step value with the original position of the moved stack.



If DRC clean is not occurred, `viaGen` checks the next tracks followed to insert the via stack at the nearest DRC clean track.



`-hookup_via_viarule {filler {default |{list_of_via rule/cell_names }} ...}`

Specifies the preferred viarule for the designated cell area during the hookup via generation. The *default* value of the parameter follows the rule of the `-viarule_preference` parameter to generate a hookup via. The *list_of_via rule/cell_names* value uses the list of via rules or cells as the preferred usage for filler cell during the hookup via generation. The `-hookup_via_viarule` parameter can also work with the `-preferred_vias_only` parameter to use different types of viarule orders for filler cell to be DRC clean.

`-hookup_virtual_trim_grid {default |{-layer -mask1 {-pitch -core_offset...}}}`

Creates a virtual trim grid for PG hookup via generation. The created virtual trim grid should align with the via metal of the newly created hookup via. When this parameter is specified, the existing trim grid is ignored.

To honor the existing trim grid, either remove this parameter or define the existing trim grid within this parameter. The `-core_offset` parameter is calculated from the bottom core boundary to the center of the first grid.

Note: The width value for this parameter is taken from the LEF file.

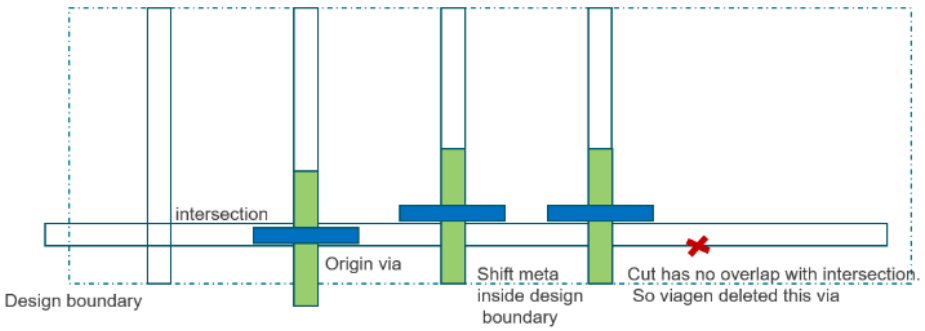
Default: `""`

`-ignore_design_boundary {true | false}`

Specifies whether a via can be extended out of the design boundary (floorplan area).

- When set to `true`, viagen generates vias and ignores the design boundary.
- When set to `false`, viagen tries to shift vias first inside the boundary area. If it does not overlap with the intersection point after shifting, viagen deletes it. If it overlaps with the intersection point after shifting with DRC clean, viagen keeps it.

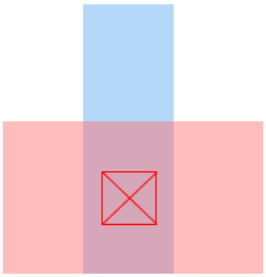
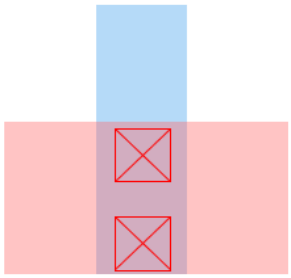
See the figure below for understanding the usage of this parameter:

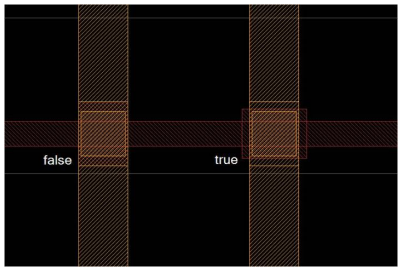
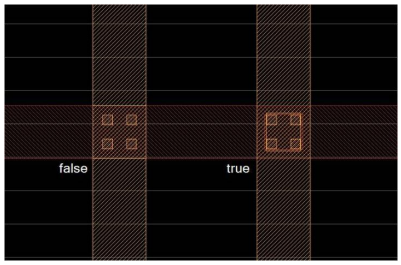


Default: `false`

`-ignore_DRC {true | false}`

	Setting this option ignores DRC but honors user-defined values when creating vias. This speeds up via creation during power-planning but normally causes DRC violations. It should only be used during early prototyping to experiment with different power-planning choices that will be updated later to be DRC clean. **Note:** This parameter ignores only the DRC violations from third party objects, and the software will still check the violation inside via. *Default:* `false`
`-ignore_viarule_enclosure {true | false}`	
	When set to `true`, tool will ignore enclosure value in VIARULE GENERATE and use the optimized enclosure from cut layer rules, to get the maximum number of cuts, minimum via metal shape. When set to `false`, the enclosure value in VIARULE GENERATE must be satisfied. When this VIARULE fails DRC check, tool will take next VIARULE or viacell from LEF file. *Default*: `true`
`-inherit_wire_status {true | false}`	
	Inherits the wire status. *Type: Boolean* *Default:* `false`
`-invoke_verifyGeometry {true | false}`	
	Enables or disables the `verify_drc` command called by the via generation software. • When set to `true`, the via generation software calls the `verify_drc` command to detect any `minStep` violations, then calls the `fixVia -minStep` command to fix them. This may impact the run-time performance. • When set to `false`, the via generation software will not call the `verify_drc` command. This may leave `minStep` violations in the design. **Note:** When both the `-invoke_verifyGeometry` and `-optimize_cross_via` parameters are set to `true`, and the process node is above `10N`, DRC violations can be reported. *Default*: `false`
`-keep_existing_via {0 | 1 | 2}`	
	If a via exists in an intersection and does not cover the whole intersection, a new via is tried to be dropped in the same intersection. Following are the parameter values: • `0`: Prevents keeping an existing via, while dropping a new via. • `1`: Keeps an existing via, while dropping a new via. • `2`: Keeps an existing via and does not drop a new via at the same intersection, even when the existing via does not cover the whole intersection area. *Default*: `0` (`1` for 10 nm designs)
`-keep_fixed_via {true | false}`	
	Controls whether to preserve the fixed vias or not when `viaGen` is called for via generation. *Default:* `true`
`-mincut_preference` *value*	

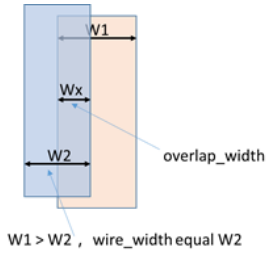
	Specifies the minimum number of cuts during via generation at an intersection. For example: • If this parameter is set to `1`, viagen generates at least one cut at an intersection, as shown below:  • If this parameter is set to `2`, viagen generates at least two cuts at an intersection, as shown below:  *Default*: `1`
`-optimize_cross_via {true | false}`	

	Generates vias with the minimum enclosure value on each layer. The different results with false or true values are shown below:   *Default*: `false` **Note:** The default value is `true` for advanced node (10 NM and 7 NM) designs.
`-optimize_via_on_routing_track {true | false}`	
	Free up routing tracks by shift or modify the vias minimally in intermediate layers and not applied to layers where the wires or pins exist. This option only supports generated vias and not predefined vias. The default is `false`. **Note:** Free up routing tracks against forbidden spacing rule for cross via and save routing tracks against end of line spacing rule for non cross via. `setViaGenMode -optimize_cross_via true` will try to generate cross via.
`-reset`	Resets all or specified parameters to their default values. Any values that do not already match the default value and are reset. will be reported.
`-parameterized_via_only {true | false | auto}`	
	Generates parameterized vias only. When set to: • `true`, ViaGen generates parameterized vias only, which means the DEF syntax of generated vias is with VIARULE. It helps in OpenAcceess designs. • `false`, ViaGen generates both parameterized and fixed vias. • `auto`, it is `true` for OpenAccess design, `false` for non-OpenAccess design. *Default:* • `false`, when the `setOaxMode -saveCdsFixedVias` parameter is enabled in the OpenAccess design. • `auto`, for all other cases.
`-partial_overlap_threshold` *float*	

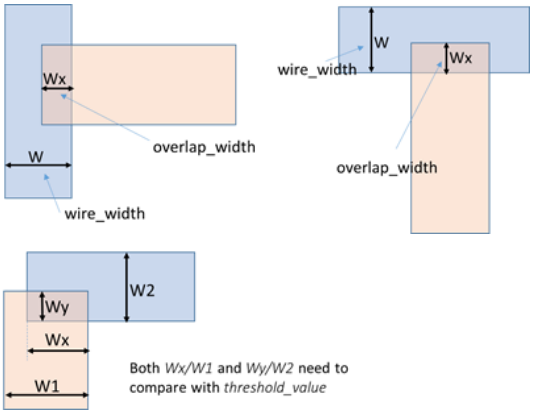
Specifies ratio threshold of via creation on partial overlap wires. The parameter value may range from `0` to `1`. The parameter value `1` specifies that no via is dropped on partial overlap intersection. The tool drops via only when `overlap_width`/`wire_width` is greater than or equal to the `threshold_value`.

Default: `0`

The following figure depicts parallel overlapping wires:



The following figures depict vertical cross wires:



`-preferred_vias_only {use_lef | open | keep}`

Generates only the vias set in `-viarule_preference`.

Default: `use_lef`

Note: This option must be used with the `-viarule_preference` option.

When the value of this parameter is set to:

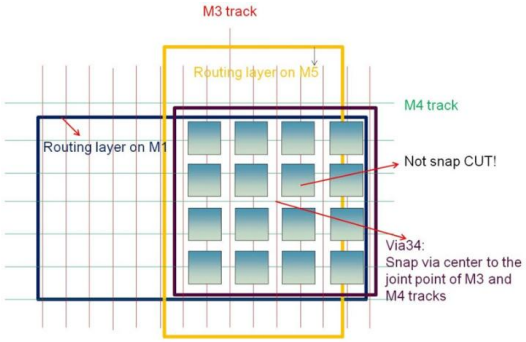
- `use_lef`: viarule or via cell specified in `-viarule_preference` is firstly considered. If none of them fits, viagen chooses the first DRC clean predefined viarule in the LEF file or auto viarule that can bring maximum cut area.
  Note that:
  - Cut area = single cut area * cut number
  - Auto viarule includes generate viarule and virtual viarule. Virtual viarule means viarule generated by viagen based on the cut layer rule.
- `open`: Only predefined viarule or via cell specified in `-viarule_preference` is considered. The first one with DRC clean is kept. If all of them have DRC issues, the whole stack is left open.
- `keep`: Only predefined viarule or via cell specified in `-viarule_preference` is considered. The first one with DRC clean is kept. If all of them have DRC issues, keep the first one which is DRC clean with surrounding. If all of them have DRC issues with surrounding, the whole stack is left open.

For more information on predefined viarule and generate viarule, see the *Via Rule* and *Generate Via Rule* sections respectively of the LEF Syntax chapter of the *LEF/DEF Language Reference* document.

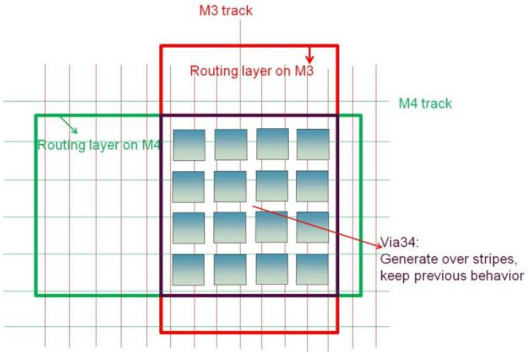
`-reference_boundary {design | core}`

	Specifies the offset reference when specifying offset values in the `-split_long_via_global_grid` parameter. Following are the parameter values: • `design`: ViaGen uses design boundary as reference. • `core`: ViaGen uses core boundary as reference. *Default:* `-reference_boundary design`
`-respect_signal_routes {0 | 1 | 2}`	
	Controls whether the via generation software honors pre-existing signal routes. If set to `1`, instructs the via generation software to read pre-existing signal routes and avoid creating DRC violations for the routes. If set to 2, instructs the via generation software to read pre-existing signal routes and avoid creating DRC violations for the routes. And improve the runtime of VIA12 process. **Note:** Using this setting can affect software performance significantly. If set to `0`, instructs the via generation software to ignore any pre-existing signal routes. **Note:** DRC violations can occur if this setting is used. *Default*: `0`
`-respect_stdcell_geometry {true | false}`	
	Respects geometry inside standard cell patterns. *Default:* `false`
`-respect_wire_ndr {true | false}`	
	Controls spacing between hookup via and signal routes. This allows viagen to support all NDR rules. • When set to `false`, viagen honors a set of NDR rules, which include PG wire NDR, PG via NDR, pin wire NDR, pin via NDR, and signal via NDR. • When set to `true`, the signal wire NDR is also honored, including those listed above with the `false` value selection of this parameter. This means that setting this parameter to `true` honors all the NDR rules. *Default:* `false`
`-set_via_expansion_dir {auto | horizontal | vertical}`	
	Sets via expansion directions. *Default:* `auto`
`-snap_via_center_to_grid {{ layer1 {none | grid | half_grid | either } [hard]} layer2 {{none | grid | half_grid | either } [hard]} … }`	

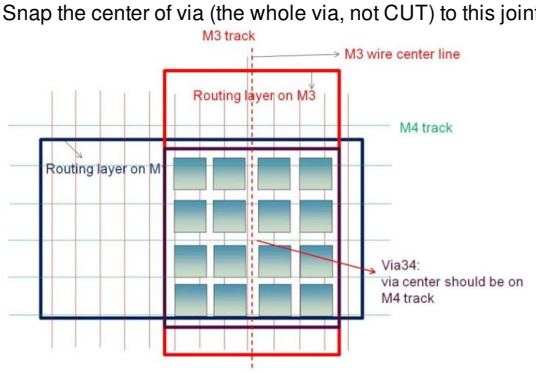
	Specifies the via center alignment style per routing layer. When this parameter is set as `hard`, the added via center will be snapped to metal track, if the added via will not be extended out of the wire side. If snap via to track will be extended out of the wire side, viagen will keep via in the stripe center to save the routing resource.
	`none`
	Keeps previous behavior, only snaps to wire center line.
	`grid`
	Snaps via to the full grid layer track.
	`half_grid`
	Ignores the full grid. Only snaps to the `half_grid` layer track.
	`either`
	Can snap to both full and half grid layer track.
	Default: `none`
	Examples
	• **Snap the vias to each adjacent up and down layers, except Metal1 and Metal2**
	◦ NO stripes on up and down layers
	◦ Snap the center of via (the whole via, not CUT) to the closest track intersection



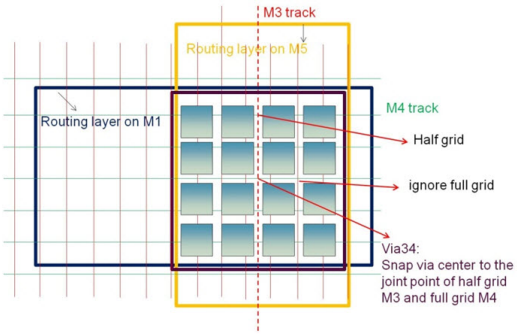
- **Stripes on both up and down layers**
  - Generate via on the stripes intersection area (the same as previous behavior, NO change)



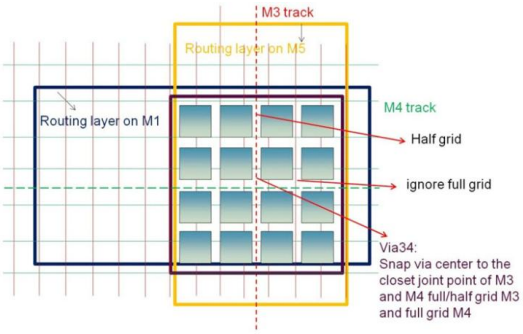
- **Stripe on adjacent layer 3 and NO stripe on the other neighboring routing layer**
  - Find the joint point between stripe center and the closest track
  - Snap the center of via (the whole via, not CUT) to this joint point



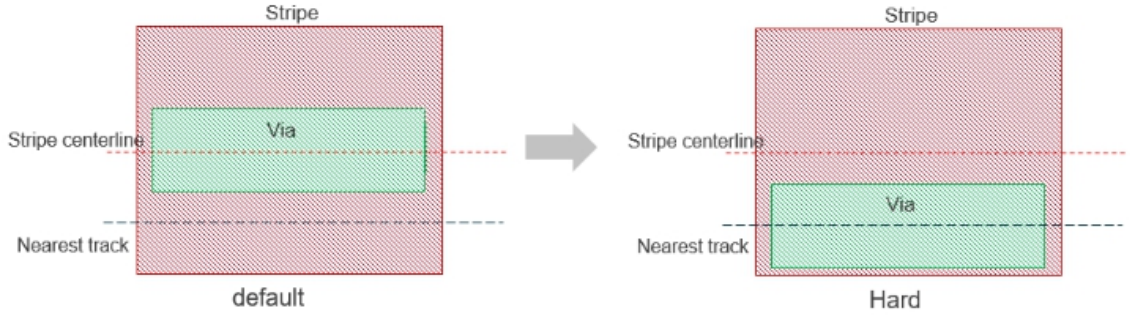
- **Snap via to the full grid of 4) and half grid of 3**
  - `setViaGenMode -snap_via_to_grid {3 half_grid 4 grid}`
  - Snap via to 3 half grid and 4 full grid



- **Snap via to full or half grid of 3 and 4**
  - `setViaGenMode -snap_via_to_grid {3 either 4 either}`
  - Tool could snap to closest full or half grid automatically



- **Snap via to track if the via is not extended out of wire side, when set as hard**



- **Keep via in stripe center if the snap via to track is extended out of the wire side, when set as hard**

`-split_long_via_global_grid {x_offset x_pitch x_length_threshold x_length_multiplier y_offset y_pitch y_length_threshold y_length_multiplier}`

	Provides options to control global grids so that vias on the grids are in line on x and/or y directions. • `x_offset, y_offset`: specifies in microns the offset from design boundry • `x_pitch, y_pitch`: specifies in microns the centre to centre distance in split vias • `x_length_threshold, y_length_threshold`: specifies vias longer or equal to threshold are being split • `x_length_multiplier, y_length_multiplier`: specifies unit via length multiplier *Default*: `default`
`-symmetrical_via_only {true | false | auto}`	
	By default the metal-width of a generated-via matches the width of the intersection of the metal above and below the via. If the width is an odd number of manufacturing grids (mgrids) wide, the resulting via cannot be represented by a parameterized via (LEF/DEF VIA with VIARULE and parameters or OpenAccess standardVia) because they require a symmetrical enclosure values on both sides of the cuts. In that case, a fixed-name via (LEF/DEF VIA with RECT statements or OpenAccess customVia) must be created to have the same width as the intersection area. If this option is true, the via metal is enlarged one mgrid in the odd-mgrid directions order to make the via symmetrical so it can be represented as a parameterized via. This does not cause any DRC violations in general because the via metal should still be covered by the metal above/below the via and avoid any min-step violations. Fixed-name vias can cause problems in hierarchical flows as the fixed-names from different blocks may collide. Parameterized via-names can be freely changed by applications and avoid this problem. If the design has odd-mgrid width wires or pins, this option is normally a good choice. If this parameter is selected as `auto`, it is `true` for OpenAccess design, `false` for non-OpenAccess design. *Default:* `auto`
`-use_track_offset {true | false}`	
	Uses track offset. *Default:* `false`
`-use_trim_metal_enclosure {true | false}`	
	Forces the tool to use enclosures with the trimmed layer. *Default:* `false`
`-viarule_preference {default | predefined | generated | list_of_via rule/cell_names | file_name}`	

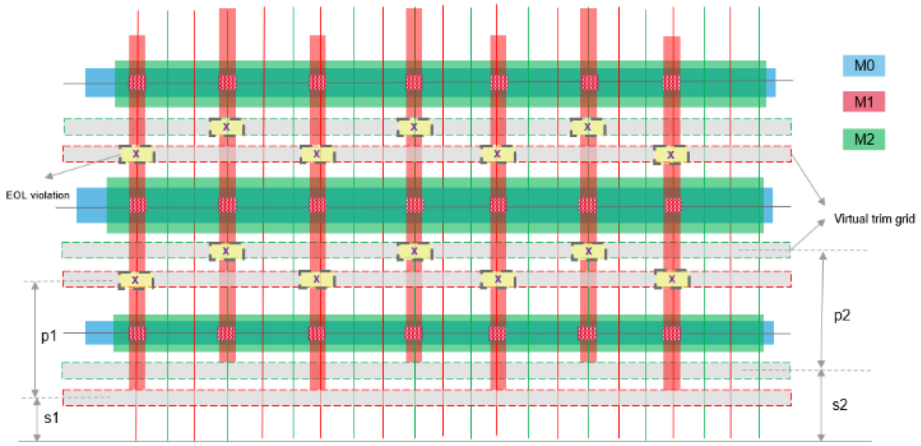
	Sets the preferred via-rule usage. Use this parameter before running the `fcroute`, `sroute`, or power planning commands.
	• default
	Use the first DRC clean predefined viarule or auto rule that can bring maximum cut area.
	• `predefined`
	Uses the predefined via rule defined in the LEF technology file as the preferred usage.
	• `generated`
	Uses the generated via rule defined in the LEF technology file as the preferred usage.
	• `list of via rule/cell names`
	Uses the list of via rules/cells as the preferred usage.
	• `file name`
	Uses via rules/cells which are contained in the specific file.
	This parameter works in the incremental mode. See the example below:
	`setViaGenMode -viarule_preference {V1 V2}`
	`setViaGenMode -viarule_preference {V3}`
	`getViaGenMode -viarule_preference`
	`V1 V2 V3`
	Default: `default`

Example

The following command creates a virtual trim grid for PG hookup via generation:

```
setViaGenMode -hookup_virtual_trim_grid {-layer M1 -mask1 {-pitch p1 -core_offset s1 } -mask2 { -pitch p2 -core_offset s2 }}
```

Following is the output of the above command:



Related Information

- editPowerVia
- Power Planning and Routing chapter in the *Innovus User Guide*.

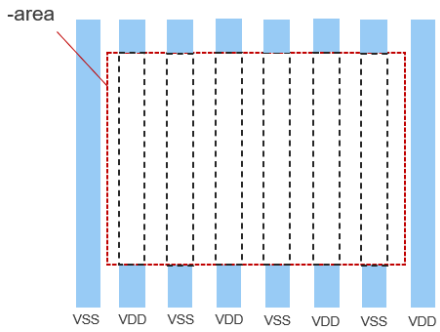
trim_pg

```
trim_pg
[-help]
[-area {x1, y1, x2, y2} | -candidate_file file_name]
-net {list_of_nets} {{-layer {layer_name1 [layer_name2]} -pattern {str1 [str2 … strN]} -type {stripe | via}} | -candidate_file file_name}
[-exclude_local_grid {{lx1 ly1 ux1 uy1} {lx2 ly2 ux2 uy2}...} | -candidate_file file_name]
[-exclude {{lx1 ly1 ux1 uy1} {lx2 ly2 ux2 uy2}...} | -candidate_file file_name]
[-keep_dangling_via]
[-threshold threshold_value | -candidate_file file_name]
```

This command offers a semi-automated mechanism that helps in trimming the redundant PG stripes and vias using an existing power grid pattern based on IR drop analysis reports. The command eliminates the need to manually adjust the power structure to reduce the power grid pessimism for area, timing, or power improvement in a design.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `trim_pg` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man trim_pg`
`-area {x1 y1 x2 y2}`	Defines the area in which stripes and vias are to be trimmed. The stripes overlapped with the box defined by `-area` are trimmed. *Default*: entire core area
`-candidate_file file_name`	Specifies the candidate filename.
`-exclude {{lx1 ly1 ux1 uy1} {lx2 ly2 ux2 uy2}...}`	
	Excludes the box areas for the stripes and vias in which the part of stripes need not to be trimmed. This parameter is used along with the `-area` parameter to specify the exclusion areas. If a stripe partially overlaps with the exclusion area, the overlapping segment of the stripe is not trimmed. The exclusion box can partially overlap the `-area` box.
`-exclude_local_grid {{lx1 ly1 ux1 uy1} {lx2 ly2 ux2 uy2}...}`	
	Excludes the areas in which the whole stripes are skipped for trimming. If a stripe is completely inside one of the `-exclude_local_grid box` (can touch the boundary), it is excluded from the trim candidates and is not counted when applying the trim pattern. It is useful when trimming is done on a large area but there are local stripes in this area, which could affect the global trim pattern. The exclusion box can partially overlap the `-area` box.
`-keep_dangling_via`	Keeps the dangling vias while trimming stripes. By default, dangling vias are removed.
`-layer {layer_name1 [layer_name2]}`	
	Specifies the metal layer names for stripe and via trimming. While trimming a stripe, specify *`layer_name1`* on which the stripes are to be trimmed. While trimming via, specify *`layer_name2`* between which the stack vias are to be trimmed. The first layer is used as the reference while applying the via trim pattern.

`-net {list_of_nets}`	Specifies the list of power or ground nets to be trimmed. The following figure presents area trimmed by the `-net` parameter: 
`-pattern {str1 [str2 … strN]}`	
	Specifies the trimming pattern. While trimming a stripe, specify *str1*. The string is a sequence of “0” and “1” to indicate whether to trim or keep a stripe. While trimming a via, *str1* and multiple strings (if needed) define a multidimensional trim pattern. Each string is a sequence of “0” and “1” to indicate whether to trim or keep a stack via along the corresponding stripe on the reference layer. The reference layer is the first layer in the `-layer` parameter.
`-threshold threshold_value`	Specifies the threshold value for stripe merging. The stripes with center-line spacing smaller than or equal to the threshold value are considered for trimming. *Default*: `0`
`-type {stripe | via}`	Specifies the type of trimming, which is either on the stripe of a layer or the stack via between two layers.

Examples

- The following command trims 50% of the `VDD` stripes on a `M6` metal layer inside a `{0 0 500 500}` box:
  ```
  trim_pg -net VDD -type stripe -layer M6 -area {0 0 500 500} -pattern 10
  ```
- The following command trims a via (stack) along with half of the `M9 VDD` stripes, and one-third of its stack vias between the `M9` and `M6` metal layers:
  ```
  trim_pg -net VDD -type via -layer {M9 M6} -area {0 0 500 500} -pattern {110 1}
  ```

Related Information

For more details and examples, see the "Trimming Redundant PG Stripes and Vias" section of the Power Planning and Routing chapter in the *Innovus User Guide.*

29

Power Route Commands

- fixVia
- getSrouteMode
- reportPowerRoute
- setSrouteMode
- sroute

fixVia

```
fixVia
[-help]
{-short | -minCut | -minStep}
[-area]
```

This command fixes the following types of violations in special net vias: short, mincut, and minstep. If the via cannot be replaced, the violation marker is not removed. Use this command after running the `verify_drc` command. It only fix violations indicated by markers from `verify_drc`.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `fixVia` parameter. For a detailed description of the command and all its parameters, use the man command: `man fixVia`
`-area {x1 y1 x2 y2}`	
	Specifies the rectangular area. Thus, the command applies to the violations within the area only. The area option should be compatible with all existing options and supports all violations that the command can support at the moment. Specifying the area improves the run-time for large design with significant number of violations. `x1`, `y1` are the coordinates for lower left corner of the area. `x2`, `y2` are the coordinates for the upper right corner of the area. By default, this option is off and with no default value for any of its argument.
`-short`	Fixes vias which have a short violation.
`-minCut`	Replaces power vias flagged by a violation marker due to a violation of the LEF `MINIMUMCUT` rule. The via is replaced with a via that contains the minimum number of cuts based on the LEF rule, and the violation marker is removed.
`-minStep`	Replaces power vias flagged by a violation marker due to a violation of the LEF `MINIMUMSTEP` rule. The via is replaced with a via that contains the minimum number of steps based on the LEF rule, and the violation marker is removed.

Examples

The following command replaces power vias for the specified rectangular area:

```
fixVia -area {131.877 153.962 161.278 176.684} -minCut
```

Related Information

Power Planning and Routing chapter in the *Innovus User Guide.*

getSrouteMode

```
getSrouteMode
[-help]
[-allowWrongWayRoute]
[-avoidOverCoreRowLayer]
[-blockPinConnectRingPinCorners]
[-blockPinRouteWithPinWidth]
[-connectBrokenCorePin]
[-corePinIgnoreObs]
[-corePinJoinLimit]
[-corePinLength]
[-corePinLengthAsInstance]
[-corePinMaxViaScale]
[-corePinReferToFollowPin]
[-corePinReferenceMacro]
[-corePinSiteRailWidth]
[-corePinSnapTo]
[-corePinStopRoute]
[-extendNearestTarget]
[-jogThresholdRatio]
[-layerNormalCost]
[-layerWrongWayCost]
[-nonDefault]
[-padPinMinViaSize]
[-padPinSplit]
[-padRingLefConvention]
[-secondaryPinMaxGap]
[-secondaryPinRailWidth]
[-signalPinAsPG]
[-splitLongVia]
[-srpgAonCellPin]
[-targetNumber]
[-targetSearchDistance]
[-timeLimit]
[-treatEndcapAsCore]
[-treatWelltapAsEndcap]
[-viaConnectToShape]
[-viaThruToClosestRing]
[-quiet]
```

For descriptions of the above options see `setSrouteMode`.

Related Information

- `sroute`
- `setSrouteMode`
- Power Planning and Routing chapter in the *Innovus User Guide*

reportPowerRoute

```
reportPowerRoute
[-help]
[-followpin_cut_ratio {cutClass1 weight1 cutClass2 weight2}]
[-net string]
```

Reports the cut ratio when two types of cuts are inserted between two parallel followpin rails with `editPowerVia -followpin_via_stapling` to maximize the effective number of cuts. The ratio is calculated by dividing the sum of weighted cut count of each cut class by theoretical cut count, if the smallest via is inserted along with the power rail with minimum spacing.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `reportPowerRoute` parameter. For a detailed description of the command and all its parameters, use the man command: `man reportPowerRoute`
`-followpin_cut_ratio {`*`cutClass1 weight1 cutClass2 weight2`*`}`	

	Specifies the names and weights of the two types of the cut classes.
	
	Note: The `cutClass` must be defined in the header lef file before loading the design. Otherwise, it will print error message(IMPPP-4411). Additionally, the format of the `cutClass` to be defined in the lef file should be:
	
	`PROPERTYDEFINITIONS`
	` €¦  €¦`
	`LAYER LEF58_CUTCLASS STRING ;`
	` €¦  €¦`
	`END PROPERTYDEFINITIONS`
	`LAYER VIA0`
	`TYPE CUT ;`
	`PROPERTY LEF58_CUTCLASS "`
	`CUTCLASS VSINGLECUT WIDTH 0.020 LENGTH 0.020 CUTS 1;`
	`CUTCLASS VDOUBLECUT WIDTH 0.020 LENGTH 0.050 CUTS 2;`
	`" ;`
	` €¦  €¦`
	`END VIA0`

`-net` *`string`*	Specifies the followpin net names to be reported. If this parameter is not specified, the report is calculated for all the followpin nets.

Example

The following command specifies the weights for the `VDOUBLE` and `VSINGLE` cut classes and calculates the reports for the `VSS` followpin net:

```
reportPowerRoute -followpin_cut_ratio {VDOUBLE 1.8 VSINGLE 1} -net VSS
```

Related Information

Power Planning and Routing chapter in the *Innovus User Guide.*

setSrouteMode

```
setSrouteMode
[-help]
[-allowWrongWayRoute {true | false}]
[-avoidOverCoreRowLayer layerName]
[-blockPinConnectRingPinCorners {true | false}]
[-blockPinRouteWithPinWidth {true | false}]
[-connectBrokenCorePin {true | false}]
[-corePinIgnoreObs {none | placement_blockage | overlap_obs | block_halo}]
[-corePinJoinLimit value]
[-corePinLength value]
[-corePinMaxViaScale widthPct heightPct]
[-corePinSiteRailWidth {{site1 layer11 Width11 layer12 Width12 …} {site2 layer21
Width21 layer22 Width22 …} …}]
[-corePinSnapTo {M1_pin | opt_routing_track | grid | half_grid}]
[-corePinStopRoute {RowEnd | CellPinEnd}]
[-corePinReferenceMacro {macro1 macro2 ...}]
[-corePinReferToFollowPin {true | false}]
[-extendNearestTarget {true | false}]
[-jogThresholdRatio value]
[-layerNormalCost layer1 cost1 layer2 cost2 ...]
[-layerWrongWayCost layer1 cost1 layer2 cost2 ...]
[-padPinMinViaSize viaSizePercent]
[-padRingLefConvention {true | false}]
[-secondaryPinMaxGap value]
[-secondaryPinRailWidth value]
[-signalPinAsPG {true | false}]
[-splitLongVia {threshold_value step_value offset_value height_value}]
[-targetNumber value]
[-targetSearchDistance dist]
[-timeLimit value]
[-treatEndcapAsCore {true | false}]
[-treatWelltapAsEndcap cell_name_list]
[-viaConnectToShape {padring | ring | stripe | blockring | blockpin | coverpin |
noshape | blockwire | corewire | followpin | iowire}]
[-viaThruToClosestRing {true | false}]
[-corePinLengthAsInstance {true | false}]
[-padPinSplit {width spacing | width1 spacing1 width2 spacing2 width3 spacing3 …}]
[-srpgAonCellPin {cellName:pinName}]
[-reset]
```

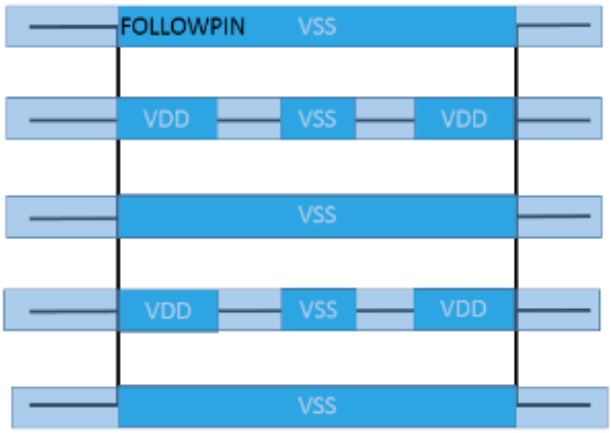
Comprises some of the `sroute -extraConfig` options. It is recommended to use `setSrouteMode` for

Compliment some of the `sroute` checking options. It is recommended to use `setSrouteMode` for `sroute`.

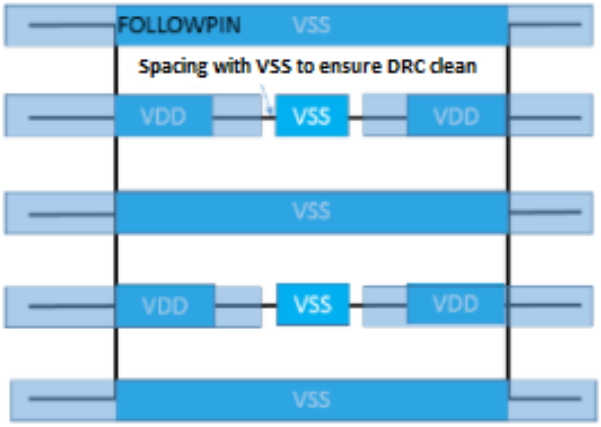
Parameters

`-help`	
	Outputs a brief description that includes the type and default information for each setSrouteMode parameter. For a detailed description of the command and all its parameters, use the `man` command: `man setSrouteMode`
`-allowWrongWayRoute {true | false}`	
	Allow wrong way routing. *Default:* `false` **Note:** If the metal1 preferred routing direction is vertical and standard cells have their VDD and VSS pins on layer metal1, sroute will route metal1 wires on horizontal direction to continue extending the followpin connections to core rings by this option.
`-avoidOverCoreRowLayer` *layerName*	
	Specifies the number of layers to avoid routing of block pin and pad pin over the core row area. *Default*: `max layer of standard cell pin`
`-blockPinConnectRingPinCorners {true|false}`	
	Specifies whether to extend the ring pins at corners. *Default:* `false`
`-blockPinRouteWithPinWidth {true|false}`	
	Specifies that the block wire used for a connection is to have the same width as the pin from which it connects. *Default:* `false`
`-corePinJoinLimit` *value*	

	Joins follow pin rail gaps smaller than this variable, bigger gaps will remain open. The option is of type *float*. *Default:* `0`
`-corePinLength` *value*	
	Connects core pins of specified length. The option is of type *float*. *Default:* `0`
`-corePinLengthAsInstance {true | false}`	
	Connects core pins whose length equals or is greater than the length of instance. *Default:* `false`
`-corePinMaxViaScale` *widthPct heightPct*	
	Controls the length and width of vias at the crossover of core pin rails and stripes. Specify this value as percentages of the width of the via at the top of the stack. Smaller vias leave room for more routing tracks on layers between the vias. *Default:* If you do not specify this parameter, the software uses a value of 100, which specifies a full-size via.
`-connectBrokenCorePin {true | false}`	
	When set to true, `sroute` checks the signal pin, second power pin, and obs, and then breaks the followpin to avoid violations. When set to false, `sroute` does not check internal geometries and does not break the followpin. Note that for ABUTMENT standardcell pins, followpins will go through the pin; for FEEDTHRU standard cell pins, followpins will break at the at the pin edge on the standard cell boundary. Refer to LEF/DEF Language Reference for more details. See the examples below for understanding the usage of this parameter: **For ABUTMENT Standard Cell Pins:** `setSrouteMode -connectBrokenCorePin false`

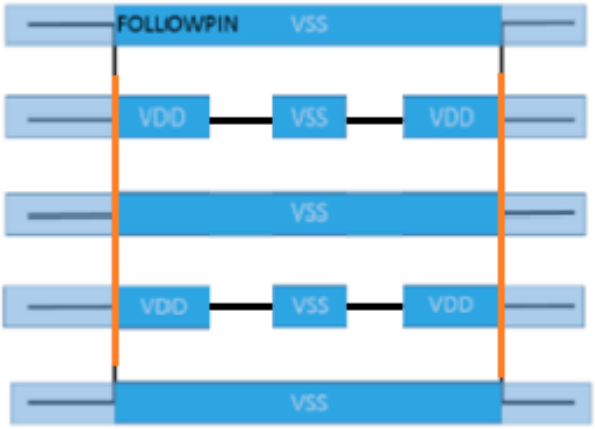



```
setSrouteMode -connectBrokenCorePin true
```

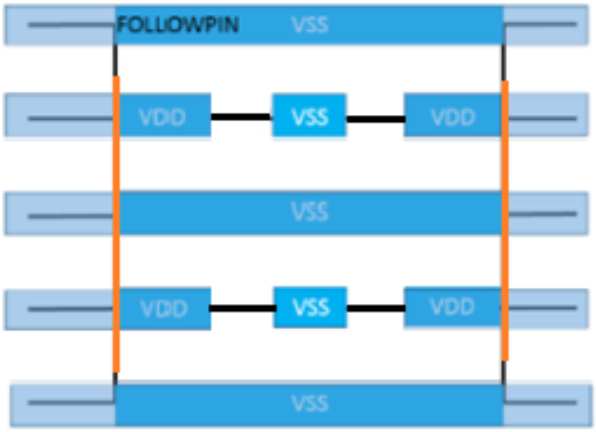


For FEEDTHRU Standard Cell Pins:

```
setSrouteMode -connectBrokenCorePin false
```




```
setSrouteMode -connectBrokenCorePin true
```



Default: `false`

`-corePinIgnoreObs {none | placement_blockage | overlap_obs | block_halo}`

Ignores the specified blockage while connecting the follow pin.

`-corePinReferenceMacro {`*`macro1 macro2 ...`*`}`

Uses specified standard cell master's cell name as references for followpin creation. If you specify multiple macros for a row height, the software picks the first macro as reference. This parameter is of *`string`* type.

Default: ""

`-corePinSiteRailWidth {{`*`site1 layer11 Width11 layer12 Width12 …`*`} {`*`site2 layer21 Width21 layer22 Width22 …`*`} …}`

Defines the followpin layer width for the specified site rail for followpin generation. The followpin width defined by this parameter has higher priority than the width of the same layer defined in the macro LEF, but lower than that defined by the `sroute -corePinWidth` parameter.

Following are the values used with this parameter:

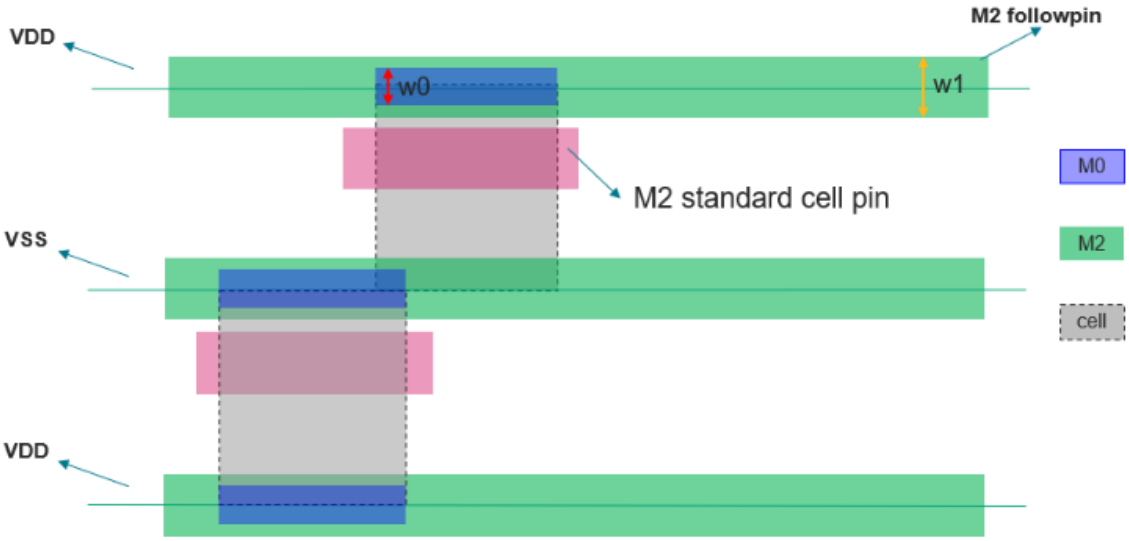
- `site`: Specifies the site name defined in the LEF file.
- `layer`: Specifies the name of the layer defined in the LEF file.
- `width`: Specifies the followpin width on the top and bottom sides of the defined site.

`-corePinSnapTo {M1_pin | opt_routing_track | grid | half_grid}`	
	Snaps metal2 followpins in the location to save routing tracks in a preventive manner. When set to: • `M1_pin`: Generated metal2 followpins are snapped to metal1 stdcell pin center. • `opt_routing_track`: Generated metal2 followpins are snapped to track or half track to save routing tracks. • `grid`: Generated metal2 followpins are snapped to the nearest routing track. • `half_grid`: Generated metal2 followpins are snapped to the nearest half routing track. *Default*: `M1_pin` **Note:** This option can only work when the option `-corePinReferToFollowPin` is set to `true`.
`-corePinStopRoute {RowEnd | CellPinEnd}`	
	Allows the command to extend the follow pin wire. Following are the parameter values: • `RowEnd`: Extends the follow pin wire to the row-end. • `CellPinEnd`: Extends the follow pin wire to the cell-end. *Default:* `RowEnd`
`-corePinReferToFollowPin {true | false}`	

If set to true, `sroute` generates followpins based on the standard followpins, whether or not there are standard cell pins on the defined metal layer. Standard followpins refer to the followpins on the first routing layer. This parameter can be used with any set of layers with same direction. Following is a usage example of this parameter:

```
setSrouteMode -corePinReferToFollowPin 1 …
sroute -corePinLayer M2 -corePinWidth w1 … #w0 is the standard
followpin's width
```

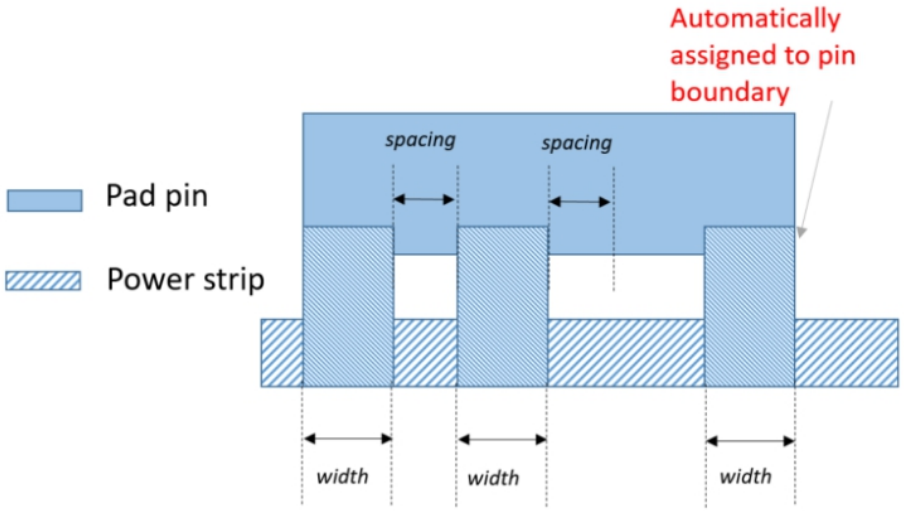
Following is the output of the above command:



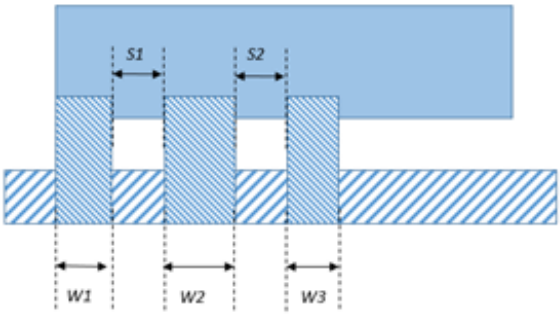
Default: `false`

`-extendNearestTarget {true | false}`

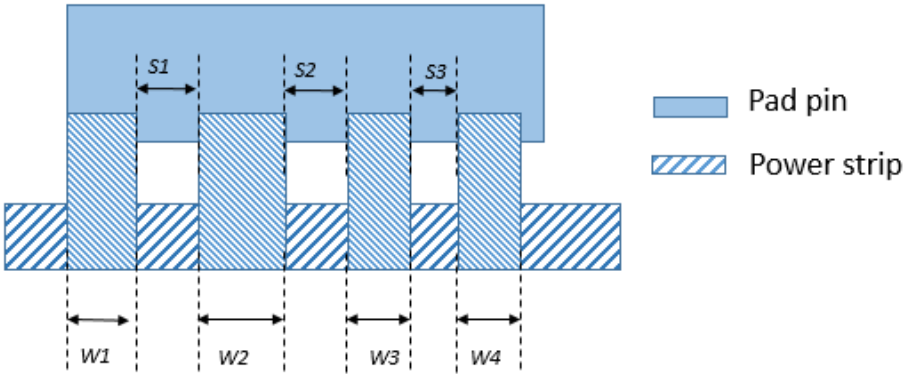
	Extends the nearest target so that it can be connected. The option is of type *bool*. *Default*: `false`
`-jogThresholdRatio` *value*	
	Defines the distance ratio between a jogging target and a straight target. The option is of type *float*. *Default:* 10
`-layerNormalCost` *layer1 cost1 layer2 cost2 ...*	
	Specifies layer cost of preferred direction. The option is of type: string. *Default*: ""
`-layerWrongWayCost` *layer1 cost1 layer2 cost2 ...*	
	Specifies layer cost of non-preferred direction. *Default:* ""
`-padPinMinViaSize viaSizePercent`	
	Permits the software to reduce the size of vias at pad pins when changing layers, if there is not enough room for a standard size via. *Default:* If you do not specify this parameter, the software uses a value of `20`, which permits vias that are 20% of the standard via size.
`-padPinSplit {`*width spacing* `|` *width1 spacing1 width2 spacing2 width3 spacing3 …*`}`	
	Splits the width and spacing to connect wide pad pins. When you use `{`*width spacing*`}`, `sroute` repeats it to connect the wide pad pins. It stops when the rest distance cannot put a full width wire. When you use {*width1 spacing1 width2 spacing2 width3 spacing3 …*}, `sroute` sets the connection for wide pad pins only once. It stops when the rest distance cannot put a full width wire. Refer to the command usage in various cases and corresponding outputs below: `setSrouteMode -padPinSplit {width spacing}`




```
setSrouteMode -padPinSplit {W1 S1 W2 S2 W3 S3}
```




```
setSrouteMode -padPinSplit {W1 S1 W2 S2 W3 S3 W4 S4 W5 S5 W6 S6}
```



Default: `""`

`-padRingLefConvention {true | false}`	
	Routes the pad rings based on the information specified in the LEF file. If set to 0, Innovus chooses the pad pin connection style (ABUTMENT or FEEDTHRU) when routing pad rings, disregarding what is specified in the LEF file. The option is of type *Boolean*. *Default:* `true`
`-reset`	
	Resets all or specified options to default value. Any values that do not already match the default value and are reset. will be reported.
`-secondaryPinMaxGap` *value*	
	Specifies variable value for gaps bigger than this value cause level shifter rail to break. The option type is *float*. *Default:* `0`
`-secondaryPinRailWidth` *value*	
	Specifies width of followpin rail. The option type is *float*. *Default:* `0`
`-signalPinAsPG {true | false}`	
	Specifies signal pins as PG pins in pad ring creation. When this parameter is set to `true`, the `sroute -nets` parameter supports wildcard characters (like * and ?). *Default*: `false`
`-splitLongVia {`*threshold_value step_value offset_value height_value*`}`	

	Splits vias longer than the specified length (`threshold_value`, in micrometers) into smaller vias with specified center-to-center spacing (`step_value`, in micrometers), left/bottom end offset (`offset_value`, in micrometers) and vertical/horizontal height (`height_value`, in micrometers) values . This is independent of the option `-split_via`. *Default:* If you do not specify this parameter, the software does not split vias. **Note:** If the offset_value is given as negative, the vias will be placed symmetrically on both ends. The default value of `offset_value` is `-1`.
`-srpgAonCellPin {`*`cellName: pinName`*`}`	
	Lists the cell/pin information NOT extracted from CPF. *Default:* `""`
`-targetNumber` *`value`*	
	Specify number of target to connect. The option type is *int*. *Default:* `0` (minimum). Maximum is `100000`.
`-targetSearchDistance` *`dist`*	
	Specifies target search distance. The option type is *float*. *Default*: 0
`-timeLimit` *`value`*	
	Specify time limit (in seconds) that the router is allowed in routing each port. The option type is *float*. *Default:* `0`
`-treatEndcapAsCore {true | false}`	
	Specifies to treat Endcap macro as core macro. The option type is *bool*. *Default*: false
`-treatWelltapAsEndcap` *`cell_name_list`*	

	Enables `sroute` to treat the `Welltap` cells as the `Endcap` cells. Additionally, the `ENDCAP` class cells are ignored by default, when generating the `followpin` wires. *Default*: ""
`-viaConnectToShape {padring | ring | stripe | blockring | blockpin | coverpin | noshape | blockwire | corewire | followpin | iowire}`	
	Specifies which shapes vias can connect to. The option type is *enum.* *Default:* `padring ring stripe blockring blockpin coverpin noshape blockwire corewire followpin iowire`
`-viaThruToClosestRing {true | false}`	
	Specifies that a via should connect only to the closest ring layer when multiple rings overlap. *Default:* If you do not specify this parameter, vias connect to all overlapping rings.

Examples

The following command defines followpin layer width for generating followpins:

```
setSrouteMode -corePinSiteRailWidth {{site1 layer1 w1} {site2 layer1 w2}}
```

Following is the output of the above command:

Related Information

- `sroute`
- `getSrouteMode`
- Power Planning and Routing chapter in the *Innovus User Guide*

sroute

```
  sroute
[-help]
[-allowJogging {0 | 1}]
[-allowLayerChange {0 | 1}]
[-area {x1 y1 x2 y2}]
[-blockPin {useLef all onBoundary leftBoundary rightBoundary topBoundary bottomBoundary
abutPins}]
[-blockPinLayerRange {minLayerName maxLayerName}]
[-blockPinTarget {nearestTarget | boundaryWithPin | farthestPadRing} | {[blockring]
[padring] [ring] [stripe] [ringpin] [blockpin]}]
[-blockPinWidthRange {min max}]
[-connect {[blockPin] [corePin] [padPin] [padRing] [floatingStripe]
[secondaryPowerPin]}]
[-connectAlignedBlockAndPadPin {blockPinAsTarget | padPinAsTarget}]
[-connectInsideArea]
[-corePinCheckStdcellGeoms]
[-corePinLayer layerNumList]
[-corePinTarget {{[blockring] [ring] [stripe] [padring] [ringpin] [blockpin]}|{
firstAfterRowEnd | boundaryWithPin | farthestPadRing | none }}]
[-corePinWidth real]
[-crossoverViaLayerRange {bot top}]
[-deleteExistingRoutes]
[-detailed_log]
[-floatingStripeTarget {[blockring] [ring] [stripe] [padring] [ringpin] [blockpin]
[followpin]}]
[-inst {names_of_blocks}]
[-layerChangeRange {bottomLayerName topLayerName}]
[-nets {names}]
[-noBlockPinOneAmongOverlappedPins]
[-padPinLayerRange {minLayerName maxLayerName}]
[-padPinWidth real]
[-padPinPortConnect {[onePort | allPort ][ oneGeom |allGeom] | preferLayer}
[-padRingWidth real]
[-padRingLayer {layerNumList}]
[-powerDomains [powerDomainName1 powerDomainName2 …]]
[-secondaryPinNet {list_of_nets}]
[-secondaryPinRailVerticalStripeGrid {topLayerNum width pitch}]
[-stripeLayerRange {minLayerName maxLayerName}]
[-targetObjListFile filename]
[-targetViaLayerRange {bot top}]
```

```
[-uda subclass_string]
[-padCellSkipRoutingOnPadSide {top bottom left right}]
[-padPinTarget {nearestTarget | {[blockring] [ring] [stripe] [ringpin] [blockpin]
[followpin]}}]
[-secondaryPinRailLayer]
```

Routes power structures. Use this command after creating power rings and power stripes. Depending on the parameters you use, it can do any or all of the following:

- Create pad rings
- Connect pins on specified nets on the blocks
- Connect pads to nearby rings or stripes
- Extend unconnected stripes to make a connection
- Route standard cell pins

Note: To undo the `sroute` command, use the `defOut` command to save the design database before you issue the `sroute` command. Then you can use the `defIn` command to restore the design to its state before running `sroute`.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `sroute` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man sroute`
`-allowJogging {0 | 1)`	Specifies that jogs are allowed during routing to avoid DRC violations. **Note:** You can use `-allowJogging 1` and `-connect padRing` together to enable jog connection in pad ring connection.

`-allowLayerChange {0 | 1}`	Allows connections to targets on different layers. Default: **Note:** This parameter replaces the `-noLayerChangeRoute` parameters. If the 1 preferred routing direction is vertical and standard cells have their VDD and VSS pins on layer 1, sroute will change routing layers to continue extending the followpin connections to core rings by this option
`-area {`*`x1 y1 x2 y2`*`}`	Specifies the coordinates for the area in which power is to be routed. *Default:* If you do not specify this parameter, the entire design is routed. If you specify this parameter, existing routes are always preserved, even if you also specify the `-deleteExistingRoutes` parameter.
`-blockPin {useLef all onBoundary leftBoundary rightBoundary topBoundary bottomBoundary abutPins}`	

	Specifies which block pins to connect. **Note:** This parameter replaces the `-blockPin {useLefConvention | allPins | boundaryPins}`, `-blockPinBoundaryOnly`, and `-blockSides` parameters. Use one of the following options: • `abutPins`: Pins are abutted. • `all`: All pins on all ports are connected, regardless of how pins are specified in the LEF file. • `bottomBoundary`: Only pins on the bottom boundary are connected. • `onBoundary`: Connects *only* pins that abut block boundaries. • `topBoundary`: Only pins on the top boundary are connected. • `useLef`: Pins are connected exactly as specified in the LEF file. For example, if the LEF file has multiple ports with one geometry connecting to each port, then one pin per port is connected.
`-blockPinLayerRange {minLayerName maxLayerName}`	Specifies the layers on which the tool can make connections, between the bottom-most and top-most layers. A value of `1` indicates the lowest metal later, `2` indicates the next layer up, and so forth. • `maxLayerName`: Specifies the top-most block pin layer to connect to. • `minLayerName`: Specifies the bottom-most block pin layer to connect to.
`-blockPinTarget {nearestTarget | boundaryWithPin | farthestPadRing} | {[blockring] [padring] [ring] [stripe] [ringpin] [blockpin]}}`	

	Extends the block pin to targets of specified type. Multiple target types are allowed. The block pin will be open if no specified target is found: • `nearestTarget`: Extends the block pin to the nearest legal target. • `boundaryWithPin`: Extends the block pin to the design boundary and creates a new power pin along the design boundary. Any overlaps with existing I/O pins at the design boundary are flagged as violations after the extension. • `farthestPadRing` : Extends the block pin to the last pad ring that already exists on top of the I/O pad cells. • `blockring padring ring stripe ringpin blockpin` **Note:** • Specify either {`nearestTarget`}, `{boundaryWithPin}`, `{farthestPadRing}`, or a value from the following for which multiple keywords are allowed: {`blockring padring ring stripe ringpin blockpin`} • Shielding wire is not regarded as the target of the block pin.
`-blockPinWidthRange {min max}`	Specifies the allowable pin widths, between the maximum and minimum widths given. Specifies the maximum block pin width to connect to. *Default:* If you do not specify this parameter, the software uses a value of `100`. Specifies the minimum block pin width to connect to. The default value is `0.0`
`-connect {[blockPin] [corePin] [padPin] [padRing] [floatingStripe] [secondaryPowerPin] }`	

	Connects the specified objects to rings and stripes. If `-connect` is not provided, sroute connects `padPin`, `padRing`, `corePin`, `blockPin` and `floatingStripe` by default. If `-connect {}` is provided, sroute does nothing and exits. While connecting to the `padRing`, the connecting result may appear unexpected if the pads do not abut to the boundary. This is because in such cases, the direction calculation is inaccurate. Creating IO rows in consistency with the pad ring direction can guide the connecting direction. To connect `secondaryPowerPin`, you must explicitly use the combination of `-connect {secondaryPowerPin}` and `-secondaryPinNet {netname}`. **Note:** You can use `-allowJogging 1` and `-connect padRing` together to enable jog connection in pad ring connection.
`-connectAlignedBlockAndPadPin {blockPinAsTarget | padPinAsTarget}`	
	Specifies whether block or pad pins can be connection targets for aligned pad or block pins. **Note**: This parameter replaces -blockPinToAlignedPadPin and -padPinToAlignedBlockPin • `blockPinAsTarget`: Specifies that a block pin can be considered a connection target for a pad pin if the block pin is aligned with the pad pin. • `padPinAsTarget`: Specifies that a pad pin can be considered a connection target for a block pin if the pad pin is aligned with the block pin.

`-connectInsideArea`	Specifies that connections from all sources within the specified area can connect only to targets that are also inside that area. **Note:** This parameter is only available if you also specify the `-area` parameter. *Default:* If you do not specify this option the software makes connections from all sources within the specified area to targets both inside and outside the specified area.
`-corePinCheckStdcellGeoms`	Checks the standard cell geometries in the design for any spacing violations between the power rail to power stripe vias and cell blockages to repair any DRC violations. When detected, Innovus repairs these violations by trimming the via arrays as needed. Use this parameter after routing the power structures. *Default*: off
`-corePinLayer {`*`layerNumList`*`}`	Connects power pins only on the specified layers. A value of `1` indicates the lowest metal later, `2` indicates the next layer up, and so forth. If you specify more than one layer, you must enclose the list in curly braces. *Default:* If you do not specify this parameter, only pins on 1 or 2 will be connected. **Note:** Recommend to use LEF layer name.
`-corePinTarget {{[blockring] [ring] [stripe] [padring] [ringpin] [blockpin]}|{ firstAfterRowEnd | boundaryWithPin | farthestPadRing | none }}`	

	Specifies the extension target for followpins. • `firstAfterRowEnd`: Extends the standard cell pin to the first ring or stripe outside of the row. • `boundaryWithPin`: Extends the standard cell pin to the design boundary and creates a new power pin along the design boundary. Any overlaps with existing I/O pins at the design boundary are flagged as violations after the extension. • `farthestPadRing`: Extends the standard cell pin to the last pad ring that already exists on top of the I/O pad cells. • `none`: Unconnected standard cell pins are not extended, and connections may be left open. • `blockring ring stripe padring ringpin blockpin` **Note:** Followpin wire can be targets for stripe and secondary pin connection, but not for followpin connection. *Default:* `firstAfterRowEnd`.
`-corePinWidth real`	If `sroute -corePinWidth` is run prior to placement, then the standard cell rails are created with the specified width. If it is run after the placement, *sroute* will only route pins that have the specified width.
`-crossoverViaLayerRange {`*`bot top`*`}`	Specifies the highest and lowest layer that can be used for via stacking at the crossover point between power structures. **Note:** This parameter does not apply to T-pattern connections. *Default:* If you do not specify this parameter, all layers are used.

`-deleteExistingRoutes`	Specifies that the software remove existing connections when you use the `sroute` command multiple times. *Default:* The software maintains existing connections each time you issue the `sroute` command. **Note:** If you specify the `-area` parameter, existing routes are always preserved, even if you also specify the `-deleteExistingRoutes` parameter.
`-detailed_log`	Prints the detailed log file.
`-floatingStripeTarget {[blockring] [ring] [stripe] [padring] [ringpin] [blockpin] [followpin]}`	
	Specifies the target shape for floating stripes. *Default:* `{blockring ring stripe padring ringpin blockpin followpin}`
`-inst {`*`names_of_blocks`*`}`	Specifies the blocks to connect. **Note:** This parameter replaces the `-blocks` parameter.
`-layerChangeRange {bottomLayerName topLayerName}`	Allows routing between the specified bottom-most and top-most layer, inclusive. *Default*: If you do not specify this parameter, the tool can use any layer. **Note**: This parameter replaces -layerChangeBotLayer and `-layerChangeTopLayer`.
`-nets {names}`	Specifies the names of the nets to connect. If you connect to more than one net, you must enclose the net names in curly braces and separate the names with a space. This parameter supports wildcard characters (like * and ?), when the `setSrouteMode -signalPinAsPG` parameter is set to `true`. *Default:* If you do not specify this parameter, the software connects all power and ground nets in the design.

`-noBlockPinOneAmongOverlappedPins`	Specifies that all pins (including those that overlap) should be routed. **Note:** This parameter is effective *only* if `sroute -blockPin allPins` is specified. *Default*: If you omit this parameter, if multiple pins overlap either fully or partially, only one will be routed.
`-padCellSkipRoutingOnPadSide {top bottom left right}`	Specifies the pad pins of the pads should not be connected. • `bottom`: Specifies the pad pins on the bottom side of the pads should not be connected.. • `left`: Specifies the pad pins on the left side of the pads should not be connected. • `right`: Specifies that power pads along the right side of the chip should not be connected. • `top`: Specifies that power pads along the top side of the chip should not be connected.
`-padPinLayerRange {`*`minLayerName maxLayerName`*`}`	Specifies that routing is allowed between the bottom-most and top-most pad pin layers, inclusive. This parameter replaces `-padPinMaxLayer` and `-padPinMinLayer`. • `maxLayerName`: Specifies the top-most pad pin layer allowed for connection. • `minLayerName`: Specifies the bottom-most pad pin layer allowed for connection.
`-padPinPortConnect {[onePort | allPort ][ oneGeom |allGeom] | preferLayer}`	

	Specifies connections to pad pin ports. • `onePort`: Routes to only one port of a pad pin if multiple ports are defined in the LEF file. • `allPort`: Routes to all ports. • `oneGeom`: Routes to one geometry per port only. • `allGeom`: Routes to all geometries of the same port for each pad pin. • `preferLayer`: Routes to all geometries on the same port, but in the preferred routing direction only.
`-padPinTarget {nearestTarget | {[blockring] [ring] [stripe] [ringpin] [blockpin] [followpin]}}`	
	Specifies the target type.Extends the pad pin to targets of specified type. Multiple target types are allowed. The pad pin will be open if no specified target found. • `nearestTarget`: Extends the pad pin to the nearest legal target. • `blockring ring stripe ringpin blockpin`: *Default*: `nearestTarget`
`-padPinWidth` *real*	Routes only pad pins that have the specified width. If no width is specified, the software automatically calculates the width of the pad pins to use. It is usually not necessary to specify a pin width.
`-padRingLayer` *layerNumber*	Specifies which routing layer to use. A value of `1` indicates the lowest metal later, `2` indicates the next layer up, and so forth. The default is all layers. **Note:** Recommend to use LEF layer name.
`-padRingWidth` *real*	Specifies the width of pins to connect with. By default, the software automatically calculates this width.
`-powerDomains [`*powerDomainName1 powerDomainName2 …*`]`	

	Restricts `sroute` activity to the specified power domains. *Default*: All power domains **Note:** The `-deleteExistingRoutes` parameter will ignore the `-powerDomains` parameter while removing existing connections. To remove special nets in a power domain, run `editDelete` manually before `sroute`.
`-secondaryPinNet {`*`list_of_nets`*`}`	Connects voltage level shifter pins on the specified nets to the closest segment of the ring around the power domain. This option is available only if a power domain is defined. **Note:** This parameter replaces `-shifterPinConnection`.
`-secondaryPinRailLayer` *`layerName`*	Specifies connections to secondary pin rail layers. This is optional.
`-secondaryPinRailVerticalStripeGrid {topLayerNum width pitch}`	
	Specifies connections to secondary power pins. **Note:** This parameter replaces `-lsStripeLayer`, `-lsStripeWidth`, and `-lsStripePitch`. • `topLayerNum`: Specifies the top-most stripe layer on which the tool can route secondary power pins. • `width`: Specifies the width (in user units) of stripes that connect to secondary power pins. • `pitch`: Specifies the pitch (in user units) of stripes that connect to secondary power pins.
`-stripeLayerRange {`*`minLayerName maxLayerName`*`}`	

	Enables you to control the floating stripes (to be connected by `sroute`) by specifying a layer range helpful in case of advanced node designs. You need to specify the following values with this parameter: • `maxLayerName`: Specifies the top-most floating stripe layer allowed for connection. • `minLayerName`: Specifies the bottom-most floating stripe layer allowed for connection.
`-targetObjListFile` *`filename`*	Specifies a `.obj` file that contains a list of objects that the sroute software is to either consider as a target or remove from the list of available targets. **Note:** Cadence recommends that you use the Target List Editing page of the SRoute form to create this file. For more information, see "SRoute" in the Route Menu chapter of the *Innovus Menu Reference*.
`-targetViaLayerRange {`*`bot top`*`}`	Specifies the highest and lowest layer that can be used for via stacking at a target. *Default:* If you do not specify this parameter, all layers are used.
`-uda` *`subclass_string`*	Displays the user defined attribute attached to followpin, blockwire, core wires, iowires and padring.

Extra Configuration File Options for sroute

You should only use the extra configuration file if you are familiar with its use.

The following variables can be used in the extra configuration file:

- `srouteAlignViaOnStripe {true | false}`
- `srouteAutoRingPadBlockpinRoute {true | false}`
  Generates rings between pad pins and block pins. When this option is set to true, the Innovus software automatically generates the rings between pad pins and block pins to

be used as targets for pad pins and block pins.

The default is `false`.

- `srouteAutoRingLayer layer_number`
  Specifies the layer of the rings.
- `srouteAutoRingOffsetFromBlockPin db_unit`
  Specifies the clearance between the rings and the block pins.
- `srouteConnectAreaIO {true | false}`
- `srouteFollowPinOnShapePin {true | false}`
  The default is `false`.

Specifies whether the primary pins are O-shaped.

- `srouteForbidStackVia {true | false}`

  Note: This variable is obsolete and has been replaced by `srouteForbidStackVia`.
- `srouteMinRailLength minLength`
  Limits the creation of followpin rails to a length at or above the specified value. SRoute will not create followpin rails with a length less than the specified value. You can use this variable to restrict the creation of followpin rails where row spacing is limited. Units in database units.
- `srouteMinRailLength minLength`
  Limits the creation of followpin rails to a length at or above the specified value. SRoute will not create followpin rails with a length less than the specified value. You can use this variable to restrict the creation of followpin rails where row spacing is limited. Units in database units.
- `srouteRoutePowerBarPort {true | false}`
  The default is `true`.

When set to `false`, SRoute does not attempt to route outside the block after the power bars are routed.

- srouteAlignViaOnStripe {true | false}
  The default is `false`.

Controls the alignment of stacked cross vias between followpin rails and stripes. When a followpin rail passes through a pair of stripes, the power routing software generates stacked vias to connect the followpin rail to the stripe of the same net. The stacked via is created between the stripes and spans from the followpin rail layer to the layer just below the stripe layer. The via between the stripe layer and the layer below is placed at the intersection of the followpin rail and the stripe of the same net. The vias are connected with a wire from the layer just below the stripe layer.

The `srouteAlignViaOnStripe` configuration file option is enabled when set to `true` and only when the following conditions are met:

- When you specify two nets with the sroute command. For example:
  `sroute -nets {VDD GND}`
- When stripes are of the same layer and of the same width
- When the distance between the stripes is less than their width
- When there are only two stripe wires within the distance of the width (not split)

  Note: This variable replaces `srouteAlignViaOnStripe`.

- `srouteSecondaryPinIgnoreStandardCell {true | false}`
  In the previous release, if the followpins were on the `1` or the `2` layer, the command always treated the standard cells in the path of the followpin wires as blockages.

In this release, you can set the option to `true` to specify that the standard cells in the path of the followpin wires should not be considered as blockages even if the followpins are on the `1` or the `2` layer. This way, the secondary pin routing can route through these standard cells.

By default, the value is `false`. This means that the standard cells in the path of the followpin wires are treated as blockages if the followpins are on the `1` or the `2` layer.

- `srouteShifterPinObjListFile file_name`
  Specifies a file containing a list of secondary pins for sroute to ignore when routing. Use this variable when you have already routed certain shifter pins, if the shifter pins have different alignment, or if you want to use a different router (such as NanoRoute) to connect the pins, for example.

The file format is as follows:

```
delete pin pin_name macro macro_name layer layer_name
```

In the file, you can repeat this line for any pins as you want.

Currently, `layer` is ignored.

- `srouteForbidStackVia {true | false}`
  The default is `false`.

Note: This variable replaces `srouteForbidStack`.

- `srouteViaOnPadCorePin {true | false}`
  The default is `false`.

Earlier pad core pins were only considered for cross via generation. With the option `srouteViaOnPadCorePin`, pad core pins are considered as routing targets.

- `srouteTandemPadBlockpinRoute [1|2|0]`
  Controls the tandem route and routes block pin and pad pin in both preferred and non-preferred layers. Where,
  - 0 stands for do not do tandem route. This is the default.
  - 1 stands for preferred layer only.
  - 2 stands for all layers.
- `srouteTandemRouteReliefViaInterval dbunit`
  Creates stack via over tandem wires at specified intervals.

Related Information

- `getSrouteMode`
- `setSrouteMode`
- Power Planning and Routing chapter in the *Innovus User Guide*

30

Prototyping Commands

- FlexModel Commands
- SoC Architecture Info (SAI) Commands

FlexModel Commands

- assemble_proto_model
- create_buffer_psPM_table
- create_flexfiller_route_blockage
- create_proto_model
- create_ps_per_micron_model
- get_proto_design_mode
- get_proto_mode
- get_proto_model
- hilite_proto_model
- identify_proto_model
- proto_design
- replace_proto_model
- report_proto_model
- set_proto_design_mode
- set_proto_mode
- set_proto_model
- set_proto_model_physical_constraint

- set_proto_timing_settings

assemble_proto_model

```
assemble_proto_model
-topdir {topDesignDir [protoModelDirName]}
[-blockdir {blockDesignDir [protoModeldirName]}]
-mmmc_file mmmcfileName
[-cpf_file cpfFileName]
[-help]
```

Generate full chip FlexModel netlist by assembling FlexModel partition netlists back to the top-level design for hierarchical FlexModel generation. It automatically creates a full chip FlexModel directory (proto_model) by merging back FlexModel information of all specified partition blocks and assembling back into the top-level design.

The purpose for this command is to get a FlexModel netlist DB in chip-level by assembling the block and top data. This command may be used to help memory usage by not loading a full chip netlist and can be run from where you will run top-down hierarchical flow. This flow should be used if there are partition block netlists.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `assemble_proto_model` parameter. For detailed description of the command and all its parameters, use the man command: `man assemble_proto_model` .
`-blockdir {blockDesignDir [protoModeldirName]}`	

	Specifies the path to block-level design. This directory should contain full netlist and FlexModel netlist of the block-level design and the corresponding proto model directory. If you specify more than one block-level design, then you must specify the `-blockdir` option separately for each block-level design. *protoModelDirName:* Optional. Specifies the directory name where FlexModel models can be found. The default directory name is retrieved from the setting of `set_proto_mode -create_dir` parameter that is specified before running `assemble_proto_model`. If `-create_dir` information is not available then the default directory name will be `proto_model` at the current run directory. It is recommended to specify *`protoModelDirName`* especially for -blockdir parameter. *Data_type*: string, optional
`-cpf_file`	Specifies chip level CPF file. This option should be specified for a design with power domain. *Data_type*: string, optional
`-mmmc_file`	Specifies chip level MMMC file. *Data_type*: string, optional
`protoModelDirName`	Optional. Specifies the directory name where FlexModel models can be found. The default directory name is retrieved from the setting of `set_proto_mode -create_dir` parameter that is specified before running `assemble_proto_model`. If `-create_dir` information is not available then the default directory name will be `proto_model` at the current run directory. It is recommended to specify *`protoModelDirName`* especially for -blockdir parameter. *Data_type*: string, optional
`-topdir {topDesignDir [`*`protoModelDirName`*`]}`	

	Specifies the path to the top-level design directory. This directory should contain top-level design netlist. If the design has Flexmodels then the top-level FlexModel netlist should also be found in this directory with the corresponding proto model directory. *protoModelDirName:* Optional. Specifies the directory name where FlexModel models can be found. The default directory name is retrieved from the setting of `set_proto_mode -create_dir` parameter that is specified before running `assemble_proto_model`. If `-create_dir` information is not available then the default directory name will be `proto_model` at the current run directory. It is recommended to specify *`protoModelDirName`* especially for -blockdir parameter. *Data_type*: string, optional

Examples

- The following command merges all models of partition block designs arb and tdsp_core to the top-level design DTMF_CHIP that does not have any FlexModels.
```
assemble_proto_model -topdir {PTN_dir/DTMF_CHIP} \
-blockdir { PTN_dir/arb/arb_modelgen.enc.dat PTN_dir/arb/proto_model } \
-blockdir { PTN_dir/tdsp_core/tdsp_core_modelgen.enc.dat
PTN_dir/tdsp_core/proto_model} \
-mmmc_file viewDefinition.tcl
```
- The following command merges model of partition block ptn_wrapper to the top-level design dtmf_recvr_core that has FlexModel(s)
```
assemble_proto_model -topdir {PTN/dtmf_recvr_core/modelgen.enc.dat proto_model} \
-blockdir { PTN/ptn_wrapper/modelgen.enc.dat PTN/ptn_wrapper/proto_model } \
-mmmc_file DBS/dtmf_recvr_core.enc.dat/viewDefinition.tcl
```

Related Information

- `create_proto_model`
- Prototyping Methodologies chapter in the *Innovus User Guide*

create_buffer_psPM_table

```
create_buffer_psPM_table
[-help]
[-buffers string]
[-file string]
[-inverters string]
[-layer string]
-max_target_slew float
-min_target_slew float
[-rpt_file string]
[-slew_interval float]
```

Automatically calculates how much distance a specified list of buffers or inverters can drive across a set of routing layers and generates a rule file for the repeaters

For each repeater and routing layer a two pin net is created with given repeater as source and sink. Based on timing, the distance between source and sink is automatically adjusted until slew at the sink is close to the same as the input slew at the source. The command outputs a repeater rule file with one entry for each specified repeater with the distance value that gives the least total delay among all slews and layers. This repeater rule file can be used as an input to the repeater-by-rule commands: `addRepeaterByRule`

Here is a sample rule file:

```
SetBufferDrivingStrength bufferName maxDistanceInMicrons maxCapLoad
SetBufferDrivingStrength ...
SetDefaultDrivingStrength maxDistanceInMicrons maxCapLoad
SetDistLastRepToSink maxDistanceInMicrons
```

There is more information on repeater rule syntax here: `addRepeaterByRule`

Parameters

`-help`	Prints a brief description that includes the type and default information for each `create_buffer_psPM_table` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_buffer_psPM_table`

`-buffers` *string*	List of input buffers. Specified buffer should have only one input and one output. *Data_type*: string, optional
`-file` *string*	Specifies the `addRepeaterByRule` rule file name. *Default*: `<designName>_repeater.rule`. *Data_type*: string, optional
`-inverters` *string*	List of input inverters. Specified inverter should have only one input and one output. *Data_type*: string, optional
`-layer` *string*	Specifies the layer name. *Default*: All available routing layers. *Data_type*: string, optional
`-max_target_slew` *float*	Maximum target slew in picoseconds. *Data_type*: float, required
`-min_target_slew` *float*	Minimum target slew in picoseconds. *Data_type*: float, required
`-rpt_file` *string*	Report file name. *Default*: `<designName>_buffer_psPM.rpt`. *Data_type*: string, optional
`-slew_interval` *float*	Calculates the step and slew range interval. Specified value should be within [0.01 1.0]. The step is calculated using the following: `step = ($max_target_slew - $min_target_slew) * $slew_interval` *Default*: 0.1 *Data_type*: float, optional

Examples

- The following command reports distances and pico-second delay values per micron for specified buffers `buf2` and `oabuf1` with given minimum and maximum slews:

  ```
  create_buffer_psPM_table -buffers {buf2 oabuf1} -max_target_slew 100 -min_target_slew 70
  ```

- The following command reports distance and pico-second delay values per micron for specified buffers `TBUFX1`, `TBUFX3`, and `TBUFX20` with given minimum, maximum slews, and slew interval:
  ```
  create_buffer_psPM_table -buffers {TBUFX1 TBUFX3 TBUFX20} -min_target_slew 90 -max_target_slew 120 -slew_interval 0.8
  ```
- The following command reports distances and pico-second delay values per micron for specified inverter CLKINVX8 with given minimum, maximum slews, and slew interval:
  ```
  create_buffer_psPM_table -inverters CLKINVX8 -min_target_slew 70.0 -max_target_slew 80.0 -slew_interval 1.0
  ```

Related Information

- `addRepeaterByRule`
- Prototyping Methodologies chapter in the *Innovus User Guide*

create_flexfiller_route_blockage

```
create_flexfiller_route_blockage
[-help]
```

Generates default congestion model to estimate routing blockages over FlexFillers based on user-specified percentage value for characterizing. This command can handle both module-based and instance group-based FlexModels. Routing blockage value of an instance group-based FlexModels will be derived based on its parent module routing blockage value. You should run this command when models are already created.

Note: This command should be used for Active-logic Reduction technique (ART) based model generation flow. For non ART based flow, congestion model can be created during `create_proto_model` command.

Note: The FlexFillers generation capability supports hybrid rows and generates FlexFillers that can fit in the floorplan rows and can be placed legally in the floorplan.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_flexfiller_route_blockage` parameter. For a detailed description of the command and all its parameters, use the man command: `man create_flexfiller_route_blockage`.

Examples

- The following example indicates that 10% of created models will be used for routing blockage characterization:

  ```
  create_flexfiller_route_blockage
  ```

- The following example indicates that 80% of created flexmodels will be used for routing blockage characterization and `tdsp_core` and `alu` modules should be part of these 80% models:

  ```
  set_proto_mode -include_model_route_blockage {tdsp_core alu}
  set_proto_mode -characterize_percent_rt_blockage 0.8
  create_flexfiller_route_blockage
  ```

- ```
  Begin pinDensity
  # model_name pin_density flag
  # If flag is 1, use place_design and earlyGlobalRoute result for pin density.
  # If flat is 0, tool auto calculate it based on the pin area and total instance area.
  ```
  ```

```
MOUT_DT_CTL 1.952371 1
ZRAS_RZ 2.361781 0
End pinDensity
```

Related Information

- `create_proto_model`
- Prototyping Methodologies chapter in the *Innovus User Guide*

create_proto_model

```
create_proto_model
[-help]
[-keep_inst_file string]
[ [-out_dir string] [-module_model_tag string] ]
```

Creates prototyping models for the modules or instance groups that are marked as FlexModels on a disk. By default, ART based FlexModels are generated.

Note: To improve the usability, the `create_proto_model` command checks at the beginning if the current loaded design has any marking FlexModel. If the design has marked some modules as FlexModels, then the `create_proto_model` creates prototyping models for the modules or instance groups that are marked as FlexModels. If no FlexModel is marked, then it stops and displays a warning that no FlexModel will be created. Alternatively, if the design has existing FlexModels, which means the netlist has been reduced, then the `create_proto_model` command cannot be run and you need to switch to full netlist to re-generate it.

To generate prototyping models in a specific directory, type the following command:

```
set_proto_mode -create_dir directory_name
```

The default directory name is `proto_model`.

Note: This command supports multi-CPU settings and distributed processing.

```
setMultiCpuUsage -remoteHost 2 -cpuPerRemoteHost 1
setDistributeHost -local
create_proto_model -out_dir
```

Note: This command can handle only upto 1000 FlexModels in a design.

Note: Scan chains are supported during FlexModel creation.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `create_proto_model` parameter. For detailed description of the command and all its parameters, use the `man` command: `man create_proto_model`.

`-keep_inst_file` *string*	Specifies the name of the file that contains the list of instances to be kept specifically for FlexModel generation. **Note**: One instance name must be specified per line. *Data_type*: string, optional
`-module_model_tag` *string*	Specifies the module model tag to be used for ART based FlexModel generation in the iHDB directory structure. This module model tag is used in the Integrated Hierarchical Database (iHDB) flow. **Note:** This parameter is part of the iHDB flow. Before using this parameter, you must set the default central directory name (`set_module_model -default_dir` *default_model_data_directory*) where the module models data will be saved/loaded. For example: If there is a design `ptn_wrapper` and the default central directory is `/project/DATA` on specifying `create_proto_model -module_model_tag proto_model` a flexModel will be created in the following directory: `/project/DATA/ptn_wrapper/proto_model/pnr` **Note:** The structure followed here is: *<module_model_data_dir>/<design_name>/<tag_name>/<type>* *Data_type*: string, optional
`-out_dir` *string*	Specifies a directory name where ART based FlexModel design will be saved. *Default:* `DBS/model_gen.enc` *Data_type*: string, optional

Examples

- The following command creates FlexModels for all the identified modules in the `proto_dir` directory:

  ```
  set_proto_mode -identify_min_inst 2000 -identify_max_inst 5000
  identify_flexmodel
  set_proto_mode -create_dir proto_dir
  create_proto_model -out_dir DBS/model_design.enc
  ```

- The following command generates models using ART technique and save committed FlexModel design into DBS/model_design.enc.dat directory

  ```
  create_proto_model -out_dir DBS/model_design.enc
  ```

Related Information

- `set_proto_mode`
- `setMultiCpuUsage`
- `setDistributeHost`
- Prototyping Methodologies chapter in the *Innovus User Guide*

create_ps_per_micron_model

```
create_ps_per_micron_model
[-help]
[-overwrite]
```

Derives psPM.model as a separate command. This command is used for active logic flat top model generation. For non-ART model generation, psPM.model can still be generated during `create_proto_model` step. It supports MMMC where it can automatically generate a psPM model file for each delay corner. with this, you can do corner dependent psPM budgeting and generated hold budgeting that is accurate where correct corner is used for hold instead of using one corner for both hold and setup.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_ps_per_micron_model` parameter. For a detailed description of the command and all its parameters, use the man command: `man create_ps_per_micron_model`.
`-overwrite`	Overwrites an existing psPM.model. *Data_type*: bool, optional

Examples

The following commands generate psPM.model before running the `create_proto_model` command:

```
restoreDesign DBS/init_design.enc.dat dtmf_recvr_core

create_ps_per_micron_model
identify_proto_model
create_proto_model -flat_top DBS/model_gen.enc
```

Related Information

- `create_proto_model`
- `restoreDesign`
- `identify_proto_model`
- Prototyping Methodologies chapter in the *Innovus User Guide*

get_proto_design_mode

```
get_proto_design_mode
[-help]
[-effort]
[-cover_fixed_macros]
[-flexmodel_constraint_type]
[-keep_guide]
[-place_macro]
[-remove_overlap]
[-quiet]
[-nonDefault]
```

Returns information about a specified `set_proto_design_mode` parameter in the Innovus log file and the console. The following information is returned:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the Innovus software returns values for all of the `set_proto_design_mode` parameters.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `get_proto_design_mode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man` get_proto_design_mode
`parameter_names`	Returns information for the specified parameters. You can specify one or more parameters. See `set_proto_design_mode` for description of parameters.

Examples

- The following command returns the current setting of all proto_design modes:
  `get_proto_design_mode`

Related Information

- `proto_design`
- `set_proto_design_mode`
- Prototyping Methodologies chapter in the *Innovus User Guide*

get_proto_mode

```
get_proto_mode
[-help]
[-allow_model_with_io]
[-allow_powerdomain_in_flexmodel]
[-create_dir]
[-create_high_fanout_psPM]
[-create_lib]
[-create_metal_fill_NDR]
[-create_metal_fill_nominal]
[-create_multi_corner_psPM]
[-create_NDR_psPM_model]
[-create_no_flex_filler]
[-create_partition_as_flexmodel]
[-create_pipeline_flop]
[-create_powerdomain_psPM]
[-create_timing_budget]
[-identify_algorithm]
[-identify_estimated_flexmodel_number]
[-identify_exclude_module]
[-identify_exclude_module_and_parent]
[-identify_exclude_module_tree]
[-identify_honor_objects_hierarchy]
[-identify_max_inst]
[-identify_min_inst]
[-keep_inst_file_only]
[-keep_instance_defined_in_sdc]
[-keep_slack_improve_NDR]
[-max_report_NDR_net]
[-route_net_NDR]
[-timing_net_delay_model]
[-timing_ps_per_micron]
[-verbose]
[-quiet]
[-nonDefault]
```

Returns information about a specified `set_proto_mode` parameter in the Innovus log file and the console. The following information is returned:

- Parameter name
- Current value
- Type (Boolean, string, and so on)

- Whether the current value was set by user

If you do not specify a parameter, the Innovus software returns values for all of the `set_proto_mode` parameters.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `get_proto_mode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man get_proto_mode`
`parameter_names`	Returns information for the specified parameters. You can specify one or more parameters. See `set_proto_mode` for description of parameters.

Examples

- The following command retrieves the list of modules that have been excluded from being marked as FlexModels:

  ```
  get_proto_mode -identify_exclude_module
  ```

- The following command retrieves the name of FlexModel directory and estimated delay per micron for each layer:

  ```
  get_proto_model -create_dir -timing_ps_per_micron
  ```

Related Information

- `set_proto_mode`
- `identify_proto_model`
- `set_proto_model`
- `report_proto_model`
- Prototyping Methodologies chapter in the *Innovus User Guide*

get_proto_model

```
get_proto_model
[-help]
[-committed]
[-create_optimize_effort]
[-exclude string]
[-file]
[-flip_table]
[-include_default]
[-tcl]
{-all | -model <string> | {[-type_match {flex_module flex_instgroup}] [-source_match {user auto}] }}
[-name | -type  | -create_total_area  | -create_gate_area  | -create_gate_count  | -create_area_per_gate  | -create_extra_macro | -planDesign_target_util | -flexfiller_route_blockage ]
```

Retrieves all the information that you specify on a model in a design. You can retrieve information on all the models or based on your search criteria and also on a specific module name. This commands returns the following information about `set_proto_model` parameters in the Innovus console:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the Innovus software returns values for all of the `set_proto_model` parameters.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `get_proto_model` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_proto_model`
`-all`	Reports the attributes of all modules. *Data_type*: bool, optional

`-committed`	Reports attributes of committed models only. *Data_type*: bool, optional
`-create_area_per_gate`	Reports the specified area per gate value. *Data_type*: bool, optional
`-create_total_area`	Reports the total area including the standard cell area and the macro area. *Data_type*: bool, optional
`-create_extra_macro`	Reports the specified additional macros. *Data_type*: bool, optional
`-create_gate_area`	Reports the specified gate area value. *Data_type*: bool, optional
`-create_gate_count`	Reports the specified gate count value. *Data_type*: bool, optional
`-create_optimize_effort`	Reports the specified optimization setting. *Data_type*: bool, optional
`-exclude` *string*	Excludes specified option. *Data_type*: string, optional
`-file`	Allows you to output results to a file. *Data_type*: string, optional
`-flexfiller_route_blockage`	Reports percentages of flexFiller route blockage. *Data_type*: bool, optional
`-flip_table`	Flips orientation of the table. *Data_type*: bool, optional
`-include_default`	Includes public options with default parameters in parentheses (for non-tcl output). *Data_type*: bool, optional

`-model` *string*	Reports the attributes of specified models. *Data_type*: string, optional
`-name`	Reports the module name. **Note:** Does not support wildcards. *Data_type*: bool, optional
`-planDesign_target_util` *number*	Specifies the target utilization value to derive initial module size of a model. *Data_type*: bool, optional
`-source_match {user auto}`	Specifies source type for reporting. Valid values are `user` and `auto`. *Data_type*: enum_list, optional
`-tcl`	Reports the model as Tcl list. *Default*: table print *Data_type*: bool, optional
`-type`	Reports the module type. *Data_type*: bool, optional
`-type_match {flex_module flex_instgroup}`	Specifies the type match for reporting. *Default*: flex_module *Data_type*: enum_list, optional

Examples

- The following `get_proto_model` command retrieves the type, source, total area, and the target utilization of the `alu` and `crossbar` modules.

```
set_proto_model -model alu  -type flex_module  -create_total_area   1000  -
planDesign_target_util 0.75
set_proto_model -model crossbar -type full -create_total_area   3000
get_proto_model -all
Model     type          source     create_total_area    planDesign_target_util
alu       flex_module    user             1000                    0.75
crossbar  full           user             3000
```

- The following command returns the values of all models in the Tcl list format.

```
get_proto_model -all  -tcl
{{name alu}  {type flex_module}  {source user}  {create_total_area 1000}
{planDesign_target_util 0.75}}
{{name crossbar}  {type full}  {source user}  {create_total_area 3000}}
```

- The following command only returns flex module's information in Tcl list format

```
get_proto_model -type_match {flex_module}  -tcl
{{name alu}  {type flex_module}  {source user}  {create_total_area 1000}
{planDesign_target_util 0.75}}
```

Related Information

- `set_proto_model`
- Prototyping Methodologies chapter in the *Innovus User Guide*

hilite_proto_model

```
hilite_proto_model
[-help]
[-keep_existing_hilite]
[-type {flex_module flex_instgroup partition power_domain}]
[-reset]
[-level integer] | {[-min_level integer][-max_level integer]}
```

Highlights objects, such as, FlexModels, partitions, and/or power domains. This command automatically resets the list of used colors for each partition, so that the same color(s) can be re-used in different partitions.

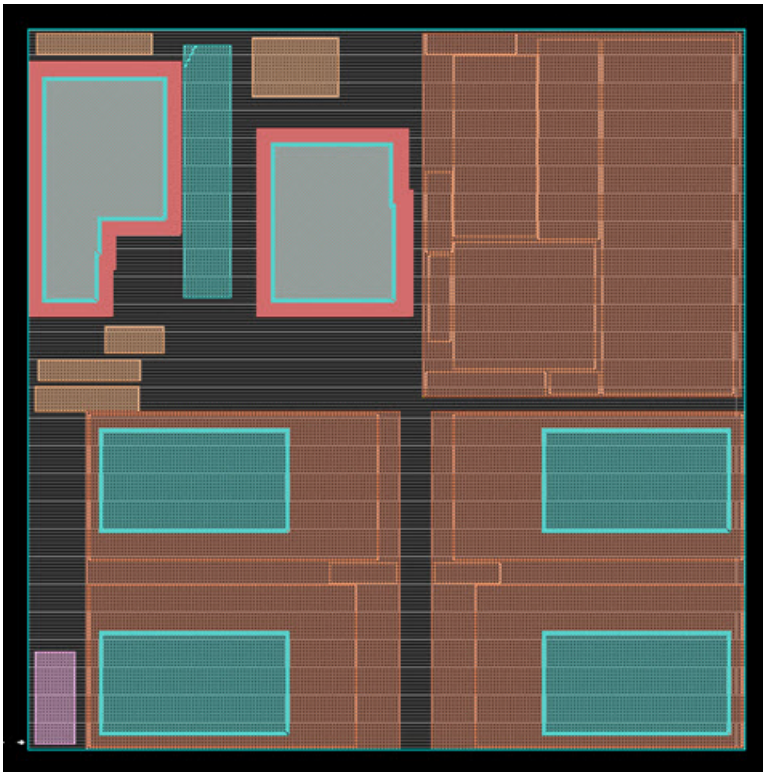
Parameters

`-help`	Prints out the command usage.
`-keep_existing_hilite`	Highlights new objects in addition to the existing highlighted objects. *Data_type*: bool, optional
`-level integer`	Specifies the level to highlight. *Data_type*: int, optional
`-max_level integer`	Highlights objects from level 0 to the specified level. *Data_type*: int, optional
`-min_level integer`	Minimum level to highlight. *Data_type*: int, optional
`-reset`	Resets all the previously highlighted models. *Data_type*: bool, optional
`-type {flex_module flex_instgroup partition power_domain}`	
	Allows you to specify which model type(s) to be highlighted. *Default*: `flex_module flex_instgroup` *Data_type*: enum_list, optional

Examples

The example below has the level range from 0 to 1. So we have `min_level` 0 and `max_level` 1. Default `hilite_proto_model` works levels from min to max.

Before `hilite_proto_model`

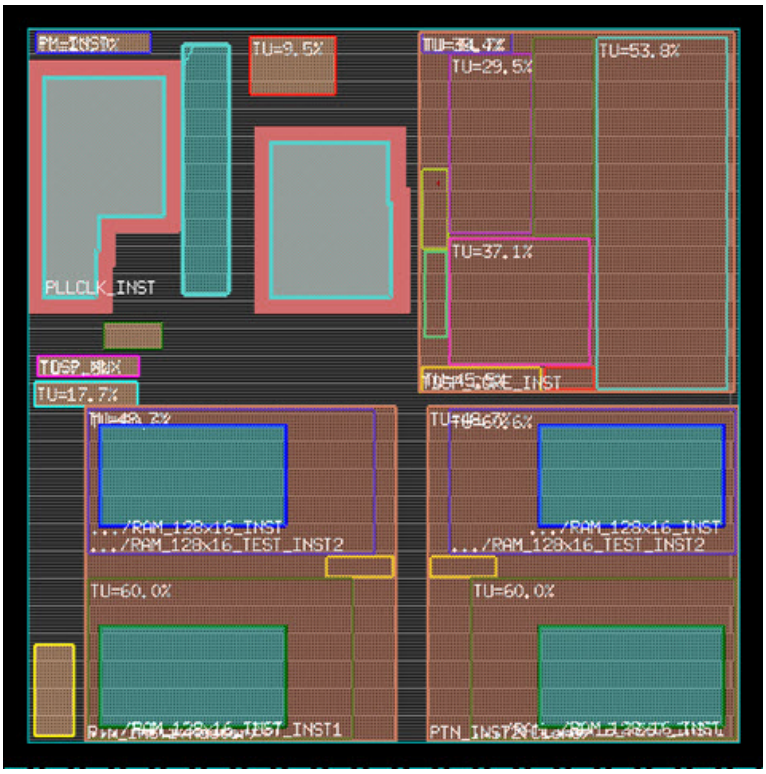


```
hilite_proto_model or <CMD>hilite_proto_model -min_level 0 -max_level 1
```

or

```
hilite_proto_model -min_level 0 #
```

These three usages have the same result on this design.



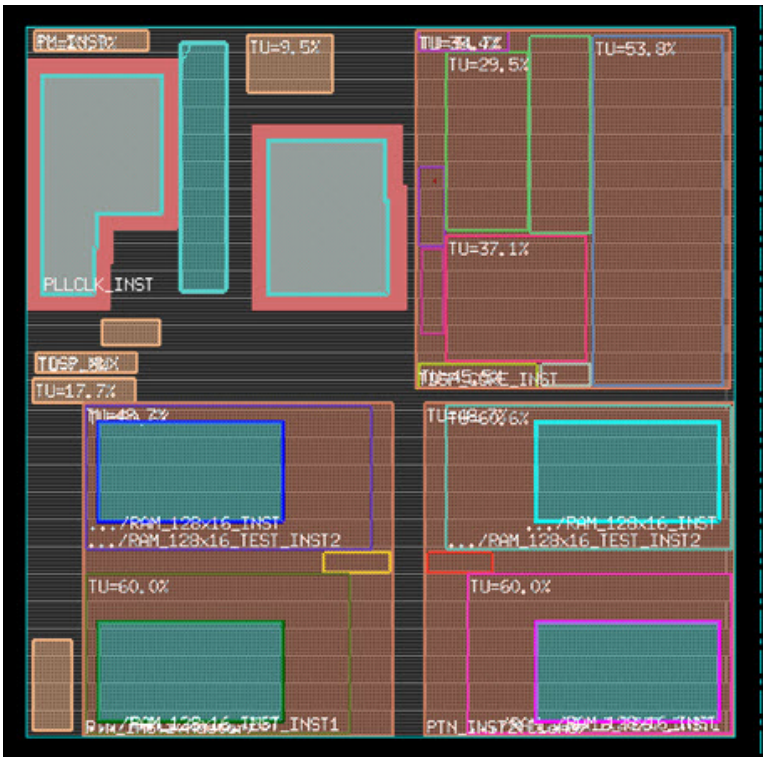

```
hilite_proto_model -level 1 #
```

Only display FlexModels on the specified level.

Or

```
hilite_proto_model -min_level 1 #
```

Display FlexModels upper or equal with level 1.



Related Information

- Prototyping Methodologies chapter in the *Innovus User Guide*

identify_proto_model

```
identify_proto_model
[-help]
[-skip_report]
```

Identifies the modules or instance groups of a design that will become FlexModels. This command supports either module-based or instance group-based FlexModels. If a design netlist has no logical hierarchy, the command will automatically create instance groups based on placement clustering algorithm and identify them as FlexModels.

The `identify_proto_model` command uses the parameters of `set_proto_mode` that have `-identify` as the prefix.

```
set_proto_mode -identify_min_inst number
set_proto_mode -identify_max_inst number
set_proto_mode -identify_exclude_module moduleName
set_proto_mode -identify_exclude_module_tree moduleTree
```

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `identify_proto_model` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man identify_proto_model`.
`-skip_report`	Skips reporting the list of identified flexModels at the end of the command. *Data_type*: bool, optional

Examples

- Consider the following design hierarchy that has 130000 instances at the top level. `MOD1` has 60000 instances, `MOD2` has 40000 instances and `MOD3` has 30000 instances.

```
TOP (130000)
          MOD1 (60000)
                MOD1a (20000)
                MOD1b (10000)
                MOD1c (30000)
          MOD2 (40000)
                MOD2a (20000)
                MOD2b (20000)
          MOD3 (30000)
                MOD3a (20000)
```

- The following commands identify the modules with minimum 20000 instances and maximum 50000 instances, and also excludes the module `MOD3` from being selected as flex_module.

```
set_proto_mode -identify_min_inst 20000 -identify_max_inst 50000
set_proto_mode -identify_exclude_module {MOD3}
identify_proto_model
get_proto_model -type flex_module -tcl -name
MOD1c MOD2 MOD3a
```

  In this example, MOD1a, MOD1c, MOD2, and MOD3a are being marked as flexModels. Here, MOD1 does not get marked as a flexModel because it is more than the maximum instance threshold. MOD1b is less than minimum instance threshold so it is also not marked as a flexModel. MOD3 is excluded because we want to exclude it as mentioned above. However its sub-module MOD3a is marked as a flexModel because it satisfies the minimum instance threshold.

Related Information

- `set_proto_mode`
- Prototyping Methodologies chapter in the *Innovus User Guide*

proto_design

```
proto_design
[-help]
[-constraints name]
[-summary file_name]
```

Generates an initial floorplan for prototyping that can be used as a start point for making the final floorplan. It is a super command that places flexModels, power domains, and/or user-specified module seeds.

Note: The `proto_design` command supports multiple levels of hierarchy, such as nested partitions and nested power domains. To support this feature, you can use the `set_proto_mode -proto_design_level multi_level` option. You also have a choice to run proto_design to place FlexModels and/or macro blocks for each level of hierarchy for better placement QOR.

By default, the `proto_design` command makes settings internally so that FlexModel floorplans are available even though any user-defined settings are not made. However, if you make any changes, then the `proto_design` command settings will be overwritten. This feature helps to improve the usability.

Note: The `proto_design` command honors the fence spacing rules.

Parameters

`-constraints` *name*	Specifies the constraint file name (string). Uses the constraints specified in the constraint file for fence creation and module guide generation. A constraint file is a text file that can contain a list of seeds used when generating fences and module guides, and basic relative, spacing and orientation constraints to follow during macro placement. *Data_type*: string, optional
`-help`	Outputs a brief description that includes type and default information for each `proto_design` parameter. For a detailed description of the command and all of its parameters, use the man command: `man proto_design`
`-summary` *file_name*	Specifies the name of the file in which the summary output report with some checkPoints like violation summary, TU of modules, master/clones shape are written. *Data_type*: string, optional

Examples

The following example invokes `proto_design` with the specified constraint file `proto_user_constraint.txt` file with all the default settings:

```
set_proto_design_mode -timing_aware true
proto_design -constraints proto_user_constraint.txt
```

Related Information

- `get_proto_design_mode`
- `set_proto_design_mode`
- Prototyping Methodologies chapter in the *Innovus User Guide*

replace_proto_model

```
replace_proto_model
[-help]
[-out_dir string]
[-ptn_dir string]
[-module_model_tag string -full_module_model_tag string]
```

Converts flexmodel netlist back to the original full netlist with changes at the partition block level whenever feedthrough insertion step is needed in the flow. You can use this command after using the `savePartition` command. This command saves a directory under each partition directory with the default name *<designName>_proto_full.enc.dat* and you need to use this DB for block implementation.

To use this command in a FlexModel based partition flow:

1. Import a full chip FlexModel netlist design.
2. Floorplan the design.
3. Run the following commands to analyze the floorplan.
    a. `place_design`
    b. `earlyGlobalRoute`
    c. `timeDesign -proto`
    d. `load_timing_debug_report -proto`
4. Do feedthrough insertion .
5. Do pin assignment
6. Optionally optimize the design.
7. Budget the timing for blocks using the `deriveTimingBudget` command.
8. Partition the design using the `partition` command.
9. Save the partition using `savePartition -dir $ptnDir` (for placement and pin locations copy)
10. `replace_proto_model -ptn_dir $ptnDir -outdir $ptnDir`

Note: The `replace_proto_model` command only supports the netlist changes caused by feedthrough insertion, so `replace_proto_model` will have issues if there is some netlist change issued by other commands such as `createLogicHierarchy` before calling `replace_proto_model`. This is a limitation for the `replace_proto_model` command.

Parameters

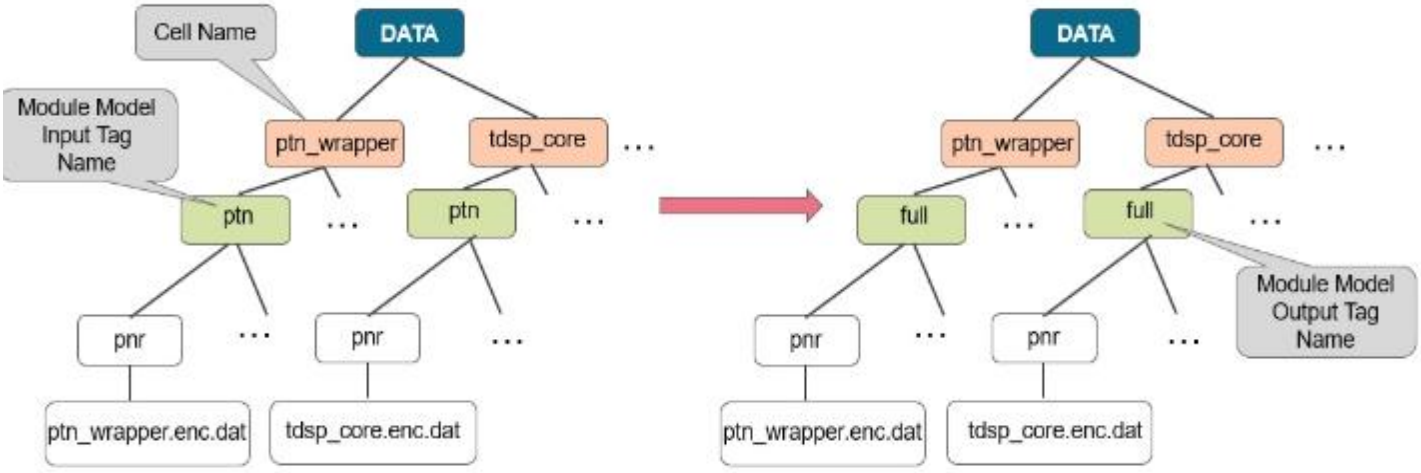
`-help`	Outputs a brief description that includes type and default information for each `replace_proto_model` parameter. For a detailed description of the command and all of its parameters, use the man command: `man replace_proto_model`.
`-full_module_model_tag` *string*	Specifies the module model tag for the original full netlist that is used in the iHDB directory structure for saving the full hierarchical DB (*output*) after eco. **Note:** The `-full_module_model_tag` and `-module_model_tag` parameters are part of the Integrated Hierarchical Database (iHDB) flow and are used by the `replace_proto_model` command to convert the FlexModel netlist back to the original full netlist in the iHDB directory structure. Before using these parameters, you must set the default central directory name (`set_module_model -default_dir` *default_model_data_directory*) where the module models data will be saved/loaded. *Data_type*: string, optional
`-module_model_tag` *string*	Specifies the module model tag for the hierarchical DB (*input*). This module model tag is used in the Integrated Hierarchical Database flow. **Note:** The `-full_module_model_tag` and `-module_model_tag` parameters are part of the iHDB flow and are used by the `replace_proto_model` command to convert the FlexModel netlist back to the original full netlist in the iHDB directory structure. Before using these parameters, you must set the default central directory name (`set_module_model -default_dir` *default_model_data_directory*) where the module models data will be saved/loaded. *Data_type*: string, optional

`-out_dir` *string*	Specifies the directory to save post ECO full top/block netlist partition data. *Data_type*: string, optional
`-ptn_dir` *string*	Specifies the partition directory for original full netlist. *Data_type*: string, optional

Example

The following command will read the FlexModel data with tag name “ptn” as input, and dump the original full netlist with tag name “full” in the iHDB directory structure.

```
replace_proto_mode  -module_model_tag ptn -full_module_model_tag full
```



Related Information

- `savePartition`
- `set_proto_model`
- Prototyping Methodologies chapter in the *Innovus User Guide*

report_proto_model

```
report_proto_model
[-help]
[-constraint]
[-file string]
[-min_inst integer | -identified | -created]
[-min_inst integer]
[-instance_ratio float | -min_inst integer]
```

Reports statistics information for models.

Note: The `report_proto_model` command takes the top-level cells in account while reporting the netlist reduction ratio at the end.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `report_proto_model` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_proto_model`
`-constraint`	Report physical constraint information. *Data_type*: bool, optional
`-created`	Reports the statistics, including reduction ratio of the models that have been created. *Data_type*: bool, optional
`-file` *string*	Specifies output report file name. *Data_type*: string, optional
`-identified`	Reports the statistics of the modules/instgroups that have been identified as models. *Data_type*: bool, optional
`-instance_ratio` *float*	Specifies ratio value where `report_proto_model` only reports flexModels that have instance ratio greater than or equal to this specified value. *Min:* 0 *Max:* 1.79769313486232e+308 *Data_type*: float, optional
`-min_inst` *integer*	Specifies the threshold of an instance count that is used to filter modules. *Data_type*: int, optional

Examples

- The following command reports modules or instance groups have been identified as FlexModels

```
report_proto_model -identified
```

```
[DEV]innovus 13> report_proto_model -identified
Starting "report_proto_model" on Tue Mar 31 14:34:26 2015...
-------------------------------------------------------------------------------------------------------------
                    Model Instance Summary
-------------------------------------------------------------------------------------------------------------
Mod  Mod     Source  #Insts
Num  Type            Stdcell  Cell/GroupName        HinstName
-------------------------------------------------------------------------------------------------------------
  1  FlexMod  auto        106 CPF_ISO_MOD_1         CPF_ISO_HIER_INST_107
  2  FlexMod  auto         33 dma                   DMA_INST
  3  FlexMod  user         52 power_manager         PM_INST
  4  FlexMod  user         33 ram_128x16_test       PTN_INST1/RAM_128x16_TEST_INST1
  5  FlexMod  user         33 ram_128x16_test       PTN_INST1/RAM_128x16_TEST_INST2
  6  FlexMod  user         33 ram_128x16_test       PTN_INST2/RAM_128x16_TEST_INST1
  7  FlexMod  user         33 ram_128x16_test       PTN_INST2/RAM_128x16_TEST_INST2
  8  FlexMod  user         33 tdsp_data_mux         PTN_INST1/TDSP_MUX
  9  FlexMod  user         33 tdsp_data_mux         PTN_INST2/TDSP_MUX
 10  FlexMod  user         33 tdsp_data_mux         TDSP_MUX
 11  FlexMod  user         34 spi                   SPI_INST
 12  FlexMod  auto        560 alu_32                TDSP_CORE_INST/ALU_32_INST
 13  FlexMod  auto         59 CPF_LS_MOD_4889       TDSP_CORE_INST/CPF_LS_HIER_INST_61
 14  FlexMod  auto         49 data_bus_mach         TDSP_CORE_INST/DATA_BUS_MACH_INST
 15  FlexMod  auto        107 decode_i              TDSP_CORE_INST/DECODE_INST
 16  FlexMod  auto        828 execute_i             TDSP_CORE_INST/EXECUTE_INST
 17  FlexMod  user         37 mult_32               TDSP_CORE_INST/MPY_32_INST
 18  FlexMod  user         44 port_bus_mach         TDSP_CORE_INST/PORT_BUS_MACH_INST
 19  FlexMod  user         42 prog_bus_mach         TDSP_CORE_INST/PROG_BUS_MACH_INST
 20  FlexMod  auto        650 tdsp_core_glue        TDSP_CORE_INST/TDSP_CORE_GLUE_INST
 21  FlexMod  auto         76 ulaw_lin_conv         ULAW_LIN_CONV_INST
-------------------------------------------------------------------------------------------------------------
            2908 out of 3332 (87.3% of chip's stdcells are in models)
Ending "report_proto_model" (total cpu=0:00:00.0, real=0:00:00.0, peak res=349.9M, current mem=400.7M)
```

- The following command reports models in the design with hierarchical modules that have at least 30 instances:

```
report_proto_model -min_inst 30
```

```
[DEV]innovus 15> report_proto_model -min_inst 30
Starting "report_proto_model" on Tue Mar 31 14:36:28 2015...
          #STDCELLS          #MACROS
    Total Parent Children    Parent Children   HinstName                                CellName
-------------------------------------------------------------------------------------------------------
M     106    106        0         0        0  CPF_ISO_HIER_INST_107                    CPF_ISO_MOD_1
M      76     76        0         0        0  ULAW_LIN_CONV_INST                       ulaw_lin_conv
M      33     33        0         0        0  TDSP_MUX                                 tdsp_data_mux
      2640    211     2429         0        0  TDSP_CORE_INST                           tdsp_core
M      59     59        0         0        0  TDSP_CORE_INST/CPF_LS_HIER_INST_61       CPF_LS_MOD_4889
       33     23       10         0        0  TDSP_CORE_INST/TDSP_CORE_MACH_INST       tdsp_core_mach
M     650    427      223         0        0  TDSP_CORE_INST/TDSP_CORE_GLUE_INST       tdsp_core_glue
M      42     23       19         0        0  TDSP_CORE_INST/PROG_BUS_MACH_INST        prog_bus_mach
M      44     24       20         0        0  TDSP_CORE_INST/PORT_BUS_MACH_INST        port_bus_mach
M      37      0       37         0        0  TDSP_CORE_INST/MPY_32_INST               mult_32
M     828    673      155         0        0  TDSP_CORE_INST/EXECUTE_INST              execute_i
M     107     66       41         0        0  TDSP_CORE_INST/DECODE_INST               decode_i
M      49     26       23         0        0  TDSP_CORE_INST/DATA_BUS_MACH_INST        data_bus_mach
M     560    277      283         0        0  TDSP_CORE_INST/ALU_32_INST               alu_32
M      34     29        5         0        0  SPI_INST                                 spi
      124     27       97         0        2  PTN_INST2                                ptn_wrapper
M      33     33        0         0        0  PTN_INST2/TDSP_MUX                       tdsp_data_mux
M      32      0       32         1        0  PTN_INST2/RAM_128x16_TEST_INST2          ram_128x16_test
M      32      0       32         1        0  PTN_INST2/RAM_128x16_TEST_INST1          ram_128x16_test
      124     27       97         0        2  PTN_INST1                                ptn_wrapper
M      33     33        0         0        0  PTN_INST1/TDSP_MUX                       tdsp_data_mux
M      32      0       32         1        0  PTN_INST1/RAM_128x16_TEST_INST2          ram_128x16_test
M      32      0       32         1        0  PTN_INST1/RAM_128x16_TEST_INST1          ram_128x16_test
M      52     43        9         0        0  PM_INST                                  power_manager
M      33     32        1         0        0  DMA_INST                                 dma
-------------------------------------------------------------------------------------------------------
      3325      3     3322         3        4  dtmf_recvr_core                          dtmf_recvr_core
-------------------------------------------------------------------------------------------------------
     2904                           4          MODEL TOTALS     dtmf_recvr_core
Total Num of Models = 16
Ending "report_proto_model" (total cpu=0:00:00.2, real=0:00:00.0, peak res=349.9M, current mem=400.8M)
^
```

- The following command reports all models after the FlexModels have been committed:

  `report_proto_model -created`

```
=============================================================================================================
                                        commit_proto_model Report
=============================================================================================================
Mod      #Instances        #Instances                #
Num   Stdcell   Filler   MODEL    Actual    Ratio   Macro   CellName/GroupName   HinstName
=============================================================================================================
  1       107        0     107       107    1.000       0   CPF_ISO_MOD_1        CPF_ISO_HIER_INST_107
  2        59        0      59        59    1.000       0   CPF_LS_MOD_4889      TDSP_CORE_INST/CPF_LS_HIER_INST_61
  3       555        0     555       555    1.000       0   alu_32               TDSP_CORE_INST/ALU_32_INST
  4        30        0      30        41    0.732       0   data_bus_mach        TDSP_CORE_INST/DATA_BUS_MACH_INST
  5        77        1      78       137    0.569       0   decode_i             TDSP_CORE_INST/DECODE_INST
  6        23        1      24        67    0.358       0   dma                  DMA_INST
  7       472        8     480       838    0.573       0   execute_i            TDSP_CORE_INST/EXECUTE_INST
  8         0       37      37         0      N/A       0   mult_32              TDSP_CORE_INST/MPY_32_INST
  9        26        0      26        33    0.788       0   port_bus_mach        TDSP_CORE_INST/PORT_BUS_MACH_INST
 10        50        0      50        69    0.725       0   power_manager        PM_INST
 11        25        0      25        31    0.806       0   prog_bus_mach        TDSP_CORE_INST/PROG_BUS_MACH_INST
 12        32        0      32        33    0.970       1   ram_128x16_test      PTN_INST1/RAM_128x16_TEST_INST1
 13        32        0      32        33    0.970       1   ram_128x16_test      PTN_INST1/RAM_128x16_TEST_INST2
 14        32        0      32        33    0.970       1   ram_128x16_test      PTN_INST2/RAM_128x16_TEST_INST1
 15        32        0      32        33    0.970       1   ram_128x16_test      PTN_INST2/RAM_128x16_TEST_INST2
 16        29        3      32        62    0.516       0   spi                  SPI_INST
 17       749        0     749       749    1.000       0   tdsp_core_glue       TDSP_CORE_INST/TDSP_CORE_GLUE_INST
 18        20        0      20        20    1.000       0   tdsp_data_mux        PTN_INST1/TDSP_MUX
 19        20        0      20        20    1.000       0   tdsp_data_mux        PTN_INST2/TDSP_MUX
 20        20        0      20        20    1.000       0   tdsp_data_mux        TDSP_MUX
 21        91        0      91        91    1.000       0   ulaw_lin_conv        ULAW_LIN_CONV_INST
=============================================================================================================
        2481       50    2531      3031    0.835       4   SUBTOTAL
                                     320                0   top-level stdcells
=============================================================================================================
        2801       50    2851      3351    0.851       4   TOTAL
=============================================================================================================
Ending "report_proto_model" (total cpu=0:00:00.2, real=0:00:02.0, peak res=256.0M, current mem=542.6M)
```

- If `-constraint` option is specified, the report has the has *PhysicalConstraint* column

  `report_proto_model -created`

```
================================================================================
                          create_proto_model Report
================================================================================
Mod      #Instances        #Instances                    #
Num   Stdcell   Filler   MODEL   Actual     Ratio   Macro   NrRef   CellName/GroupName
================================================================================
  1         0       37      37        0     0.000       0       1   mult_32
  2        26        0      26       33     0.788       0       1   port_bus_mach
  3        50        0      50       69     0.725       0       1   power_manager
  4        25        0      25       31     0.806       0       1   prog_bus_mach
  5        32        0      32       33     0.970       1       4   ram_128x16_test
  6        29        3      32       62     0.516       0       1   spi
  7        20        0      20       20     1.000       0       3   tdsp_data_mux
================================================================================
         318       40     358      387     0.925       4
================================================================================
```

```
report_proto_model -created -constraint
```

```
================================================================================================================
                                        create_proto_model Report
================================================================================================================
Mod      #Instances        #Instances                    #
Num   Stdcell   Filler   MODEL   Actual     Ratio   Macro   NrRef   CellName/GroupName   PhysicalConstraint
================================================================================================================
  1         0       37      37        0     0.000       0       1   mult_32              region {370.4 235.03 451.4 453.91}
  2        26        0      26       33     0.788       0       1   port_bus_mach        region {338.0 235.03 368.6 248.71}
  3        50        0      50       69     0.725       0       1   power_manager        region {21.405 445.36 92.405 457.33}
  4        25        0      25       31     0.806       0       1   prog_bus_mach        region {263.6 267.52 277.4 320.53}
  5        32        0      32       33     0.970       1       4   ram_128x16_test      fence {53.8 17.86 218.2 117.04}
  6        29        3      32       62     0.516       0       1   spi                  guide {20.6 19.57 44.642 76.0}
  7        20        0      20       20     1.000       0       3   tdsp_data_mux        region {202.8 118.75 243.6 130.72}
================================================================================================================
         318       40     358      387     0.925       4
================================================================================================================
```

Related Information

- set_proto_model
- Prototyping Methodologies chapter in the *Innovus User Guide*

set_proto_design_mode

```
set_proto_design_mode
[-help]
[-reset]
[-cover_fixed_macros {true | false}]
[-effort {low | high}]
[-flexmodel_constraint_type {guide |region |fence}]
[-keep_guide {true  | false}]
[-place_macro {true | false}]
[-remove_overlap {true | false}]
```

Controls certain aspects of how the `proto_design` command generates an initial floorplan. Use the `get_proto_design_mode` command to return the current settings for the `set_proto_design_mode` command.

Parameters

`-cover_fixed_macros {true|false}`	When enabled it forces flexModel to cover fixed macros that belong to it when using the `proto_design` command. *Default*: `false` *Data_type*: bool
`-effort {low | high}`	Specifies the effort level used by the `proto_design` command. *Default*: `low` *Data_type*: bool
`-flexmodel_constraint_type {guide | region | fence}`	
	Specifies the type of constraint the FlexModel is converted to after `proto_design`. *Default*: `region` *Data_type*: enums={guide region fence}
`-help`	Outputs a brief description that includes type and default information for each `set_proto_design_mode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_proto_design_mode`

`-keep_guide {true|false}`	Retains module guides after the floorplan has been created. *Default*: `true` *Data_type*: bool
`-place_macro {true|false}`	Specifies if macro placement is done when using the `proto_design` command. *Default*: `true` *Data_type*: bool
`-remove_overlap {true|false}`	Specifies if overlap should be removed when using the `proto_design` command. *Default*: `true` *Data_type*: bool
`-reset`	Restores all parameters modified using `set_proto_design_mode` to their default settings. The `-reset` parameter must be the first parameter specified. If you specify `-reset` by itself, the software resets all `set_proto_design_mode` parameters to their default values. If you specify parameters after -reset, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported.

Examples

The following command runs the prototyping module/macro placer with congestion aware setting:

```
set_proto_design_mode -effort low
proto_design
```

Related Information

- `get_proto_design_mode`
- `proto_design`
- Prototyping Methodologies chapter in the *Innovus User Guide*

set_proto_mode

```
set_proto_mode
[-reset]
[-allow_model_with_io {true|false}]
[-allow_powerdomain_in_flexmodel {true|false}]
[-create_dir directoryName]
[-create_high_fanout_psPM {true|false}]
[-create_lib libraryName]
[-create_metal_fill_NDR double]
[-create_metal_fill_nominal double]
[-create_multi_corner_psPM {true|false}]
[-create_NDR_psPM_model {auto|on|off}]
[-create_partition_as_flexmodel {true|false}]
[-create_pipeline_flop {*AB*}]
[-create_powerdomain_psPM {true|false}]
[-create_timing_budget {true|false}]
[-create_no_flex_filler {true|false}]
[-identify_algorithm {auto|module_based|instgroup_based}]
[-identify_estimated_flexmodel_number number]
[-identify_exclude_module {module1 module2 ...}]
[-identify_exclude_module_and_parent {module1 module2 ...}]
[-identify_exclude_module_tree {module1 module2 ...}]
[-identify_honor_objects_hierarchy {object_list}]
[-identify_max_inst number]
[-identify_min_inst number]
[-keep_inst_file_only {true|false}]
[-keep_instance_defined_in_sdc {true|false}]
[-keep_slack_improve_NDR {true|false}]
[-max_report_NDR_net double]
[-route_net_NDR ruleName]
[-timing_net_delay_model {best_layer_no_detour|best_layer_blockage_aware|use_actual_wire}]
[-timing_ps_per_micron {M number1[:M number2] number3 ...}]
[-verbose]
```

Sets the mode for all the prototyping model commands as well as other commands, such as `place_design` and `timeDesign`. The parameters in this command having prefix as `identify` are used by the `identify_flexmodel` command and the ones with `create` prefix are used with the `create_proto_model` command.

Use the `get_proto_mode` command to return the current settings for the `set_proto_mode` command.

Parameters

`-allow_model_with_io {true|false}`	Allows modules with I/O to be selected as FlexModel. *Default*: `false` *Data_type*: bool
`-allow_powerdomain_in_flexmodel {true|false}`	Allows modules that contains power domain(s) to be selected as FlexModel. *Default*: `false` *Data_type*: bool
`-create_dir` *directoryName*	Specifies the directory to save created models. *Default*: proto_model *Data_type*: string
`-create_high_fanout_psPM {true|false}`	Creates high fanout psPM models. *Default*: `true` *Data_type*: bool
`-create_lib` *libraryName*	Specifies the OpenAccess library to save created models. *Default*: `proto_model_lib` *Data_type*: string
`-create_metal_fill_NDR` *double*	Creates metal fill that is used by `create_ps_per_micron_model` for creating psPM model for nets with non default rules. *Default*: 0 *Data_type*: float
`-create_metal_fill_nominal` *double*	Creates metal fill that is used by create_ps_per_micron_model for creating psPM model for normal nets. *Default*: 0.5 *Data_type*: float

`-create_multi_corner_psPM {true|false}`	Create multi corner psPM models. *Default*: `false` *Data_type*: bool
`-create_NDR_psPM_model {auto|on|off}`	Specifies whether `create_ps_per_micron_model` should generate psPM model for non default rule or not. *Default*: `auto` *Data_type*: enums={auto on off}
`-create_no_flex_filler {true|false}`	Specifies whether to create proto model without adding flex filler. *Default*: `false` *Data_type*: bool
`-create_partition_as_flexmodel {true|false}`	Creates partition specified as flexmodel automatically. *Default*: `false` *Data_type*: bool
`-create_pipeline_flop {*AB*}`	Specifies a list of pipeline flop instance patterns that are used in the design. The default value is an empty list. *Default*: `" "` *Data_type*: string
`-create_powerdomain_psPM {true|false}`	Creates power domain psPM models. *Default*: `false` *Data_type*: bool

`-create_timing_budget {true|false}`	When enabled, it uses Active-Logic Reduction Technology (ART) in timing budget mode during flexModel generation. The FlexModel creation uses the same settings as the `deriveTimingBudget` command to preserve logic paths so that they yield good timing budgets later on in the flow. **Note:** This option should only be specified if design has partitions defined. *Default*: `false` *Data_type*: bool
`-identify_algorithm {auto|module_based|instgroup_based}`	
	Identifies the proto algorithm. Specifying `instgroup_based` enables the instance group based `identify_proto_model`. *Default*: `auto` *Data_type*: enums={auto module_based instgroup_based},
`-identify_estimated_flexmodel_number` *number*	Specifies the estimated flexModel number to identify the design when using the instance group based `identify_proto_model`. *Default*: `20` *Data_type*: int
`-identify_exclude_module {`*module1 module2 ...*`}`	Excludes the specified module(s) but not its sub-modules for being marked as models. *Default*: `""` *Data_type*: string
`-identify_exclude_module_and_parent {`*module1 module2 ...*`}`	Excludes the specified module(s) and its parents for being marked as models. *Default*: `""` *Data_type*: string

`-identify_exclude_module_tree {`*`module1 module2 ...`*`}`	Excludes the specified module(s) and its sub-modules for being marked as flexModels. *Default*: `""` *Data_type*: string
`-identify_honor_objects_hierarchy {`*`object_list`*`}`	Specifies the hierarchy of the list of objects that the instance group based `identify_proto_model` will honor. *Default*: `""` *Data_type*: string
`-identify_max_inst` *number*	Specifies the maximum instance count threshold of a module to be marked as model. The default value is eight percent of the total instance count and goes no lower than 300000 instances. *Default*: -1 *Data_type*: int
`-identify_min_inst` *number*	Specifies the minimum instance count threshold of a module that is to be marked as a model. *Default*: `1000` *Data_type*: int
`-keep_inst_file_only {true|false}`	Specifies that the instance list for flexmodels should only be kept in the `keep_inst_file`. *Default*: false *Data_type*: bool
`-keep_instance_defined_in_sdc {true|false}`	Specifies that the flexmodel will keep the instance defined in the sdc file. *Default*: false *Data_type*: bool

`-keep_slack_improve_NDR {true|false}`	Specifies that report_NDR_WNS_gain will keep top critical nets as non default rule nets. *Default*: false *Data_type*: bool
`-max_report_NDR_net` *double*	Specifies that create_ps_per_micron_model creates psPM model for nets with non default rules. *Default*: 0.05 *Data_type*: float
`-route_net_NDR` *ruleName*	Specifies that create_ps_per_micron_model creates psPM model for nets with non default rules. *Default*: "" *Data_type*: string
`-timing_net_delay_model {best_layer_no_detour|best_layer_blockage_aware|use_actual_wire}]`	
	Allows you to the set mode of calculating wire delay. Note: Using the `best_layer_blockage_aware` option, Innovus uses an obstacle aware approach for estimating the channel-routing-wire-length between two timing points. The `timeDesign -proto` command is aware of routing blockages and other partition fences when the `best_layer_blockage_aware` option is specified. *Default:* `best_layer_no_detour` *Data_type*: enums={best_layer_no_detour best_layer_blockage_aware use_actual_wire}
`-timing_ps_per_micron {`*M<num1>[:M<num2>] psPerMicron ...}, {MetalLayerName1[:MetalLayerName2] psPerMicron ...} {1[:2] psPerMicron ...}, or {view {M<num1>[:M<num2>] psPerMicron ...} ...}, {view {MetalLayerName1[:MetalLayerName2] psPerMicron ...} ...} {view {1[:2] psPerMicron ...} ...}*	

	Specifies the estimated delay per micron for each routing layer (DB or LEF layer names). You can also specify layer pair value for each view. *Data_type*: " " *Data_type*: string
`-reset`	Restores all parameters modified using `set_proto_mode` to their default settings. The `-reset` parameter must be the first parameter specified. If you specify `-reset` by itself, the software resets all `set_proto_mode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported.
`-verbose`	Outputs detailed information into the log file. *Data_type*: false *Data_type*: bool

Examples

- The following command sets the prototyping mode to exclude the module `MOD1` and `MOD2` but not their children for being marked as flexModels:
  ```
  set_proto_mode -identify_exclude_module {MOD1 MOD2}
  ```
- The following command sets the prototyping model to exclude the modules `MOD3` and `MOD4` and their children for being marked as flexModels:
  ```
  set_proto_mode -identify_exclude_module_tree {MOD3 MOD4}
  ```
- The following command specifies the the estimated delay per micron for the routing layer `Metal1` as `0.50`, Metal`2` to Metal`5` as `0.25`, and Metal`6` to Metal`7` as 0.14:
  ```
  set_proto_mode -timing_ps_per_micron {Metal1 0.50 Metal2:Metal5 0.25 Metal6:Metal7 0.14}
  ```
- The following command specifies the estimated delay per micron values that will be used for all

timing views:

```
set_proto_mode -timing_ps_per_micron {M1 0.90 Metal2:Metal4 0.50 Metal5:Metal6 0.25}
```

- The following command specifies the estimated layer pair value for specific views:

```
set_proto_mode -timing_ps_per_micron {setup_func {M1 0.90 Metal2:Metal4 0.50
Metal5:Metal6 0.25} hold_func {Metal1 0.60 Metal2:Metal4 0.30 Metal5:Metal6 0.15}}
```

- The specified percentage of flexModels will be used to characterized for flexFiller routing blockage is 30%.

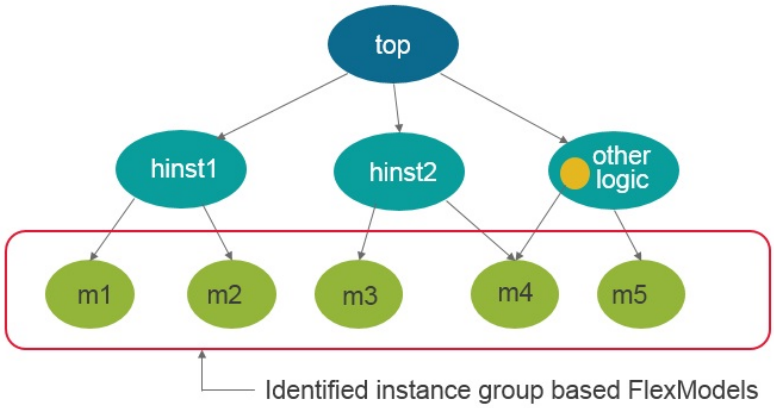
```
set_proto_mode -create_characterize_percent_rt_blkg 0.3
```

- The following command specifies that instance group based `identify_proto_model` will keep user-specified FlexModels Metal1 and Metal2 and identify all others as instance group based FlexModels.

```
set_proto_model -model Metal1 -type flex_module
set_proto_model -model Metal2 -type flex_module
set_proto_mode -identify_algorithm instgroup_based
identify_proto_model
report_proto_model -identified
```

- The following command ensures that the hierarchy under hinst1 is maintained and modules under hinst1 are not be mixed with other hierarchical modules.

```
set_proto_mode -identify_algorithm instgroup_based
set_proto_mode -identify_honor_objects_hierarchy {hinst1}
identify_proto_model
```



Related Information

- `get_proto_mode`
- `identify_proto_model`
- `set_proto_model`
- `report_proto_model`
- Prototyping Methodologies chapter in the *Innovus User Guide*

set_proto_model

```
set_proto_model
[-help]
-model name_list
[-type {full | flex_module | flex_instgroup}]
[-create_total_area number | -create_gate_area number |
{-create_gate_count number -create_area_per_gate number}]
[-create_extra_macro {macro_name number
[macro_name number] ...}
[-planDesign_target_util number]
[-reset]
```

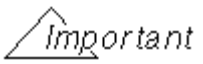
The `set_proto_model` command specifies the model type of a specific module.

Note: Wildcards are not supported by this command.

Parameters

`-create_area_per_gate number`	Specifies the area per gate. *Data_type*: float, optional
`-create_extra_macro {macro_namenumber[macro_namenumber] ...}`	Specifies the additional macro(s) of a module that you want to add in the original netlist. If you wish to add extra macros, this option changes the netlist. For example, if your original netlist has macro `A` and you specify an additional macro named macro `B`, then the flexModel with have both macro `A` and macro `B`. *Data_type*: string, optional
`-create_gate_area number`	Specifies the total area of gates. This value does not include macro area. Its unit is square microns. The total area (equivalent to `-create_total_area`) is calculated as the sum of specified total gate area and macro area. *Data_type*: float, optional

`-create_gate_count` *number*	Specifies the number of gates. If you specify this parameter, you also need to specify area per gate value.Total gate area is equal to the product of gate count and area per gate. The calculated total gate area does not include the macro area. So, the total area (equivalent to `-create_total_area`) is equal to the sum of total gate area and the macro area. *Data_type*: int, optional
`-create_total_area` *number*	Specifies the module area of a model that includes the standard cell area and the macro area. **Note**: A warning message is issued when macro area is greater than the area specified using the `-create_total_area` parameter. **Note:** For `-create_area_per_gate number`, `-create_total_area number`, `-create_gate_area number` and `-create_gate_count number`; you can define the area instead of the value calculated by the tool (for times when you wish to specify larger area for the module to achieve better results). *Data_type*: float, optional
`-help`	Outputs a brief description that includes type and default information for each `set_proto_model` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_proto_model`.
`-model` *name_list*	Specifies a name for the model type. *Data_type*: string, required

`-planDesign_target_util` *number*	Specifies the target utilization value to derive initial module size of a model. This value is used by the `proto_design` command to derive the module size of a specified flexModel if its module is not placed yet. *Default*: current design utilization **Note:** You can reduce the target utilization value by 10 to 20 percent to account for 10 to 20 percent increase in instance count during optimization.  If the post-target utilization is 0.75, then Cadence recommends you to set the following: `set_proto_mode -model` *name_list* `-planDesign_target_util 0.55` *Data_type*: float, optional
`-reset`	Resets all the attributes of a module. *Data_type*: bool, optional
`-type {full | flex_module | flex_instgroup}`	
	Specifies the model type as full or a flex module or a flexInstgroup. You use this parameter when you want to convert a flex_module back to the original netlist or converse. *Data_type*: enum, optional

Examples

- The following command specifies the number of gates as 10000 and area per gate as 0.65 for the tdsp_core model:

  ```
  set_proto_model -model tdsp_core -create_gate_count 10000 -create_area_per_gate 0.65
  -type flex_module
  ```

- The following command specifies model alu as flex_module with total area `1000` and target utilization `0.75`:

  ```
  set_proto_model -model alu -type flex_module  -create_total_area   1000 -
  planDesign_target_util 0.75
  ```

- The following command specifies model crossbar as flex_module with total area `3000`:

  ```
  set_proto_model -model crossbar  -type flex_module -create_total_area   3000
  ```

- The following command specifies the extra macro `{pllclk 2 ram128x164}` of the `chico` model:

  ```
  set_proto_model -model chico -create_extra_macro {pllclk 2 ram128x164}
  ```

- The following command resets all the attributes of the `alu` model:

  ```
  set_proto_model -model alu -reset
  ```

Related Information

- `get_proto_model`
- `set_proto_mode`
- `assemble_proto_model`
- `create_proto_model`
- `identify_proto_model`
- Prototyping Methodologies chapter in the *Innovus User Guide*

set_proto_model_physical_constraint

```
set_proto_model_physical_constraint
[-help]
-type {guide | fence | region | soft_guide | none}
[-model string]
```

Sets FlexModels to specific physical constraints to guide placement. By default, all the FlexModels will be set to the specified type.

Parameters

`-help`	Prints out the command usage.
`-model` *string*	Specifies a list of flexmodels. *Data_type*: string, optional
`-type {guide | fence | region | soft_guide | none}`	
	Specifies the constraint type. You can select one of the following options: • `guide` • `fence` • `region` • `soft_guide` • `none` *Data_type*: enum, required

Examples

The following commands change physical constraint of all FlexModels to guide before running detailed placement:

```
set_proto_model_physical_constraint -type guide
set_proto_mode -effort high
place_opt_design
earlyGlobalRoute
timeDesign -proto
```

```
load_timing_debug_report -proto
```

Related Information

- set_proto_model
- Prototyping Methodologies chapter in the *Innovus User Guide*

set_proto_timing_settings

```
  set_proto_timing_settings
[-help]
[-reset]
```

Sets or resets the prototyping settings in the flow. The following variables are set by the command:

- `setInputTransitionDelay 80.0psps`
  (*Default:* `250.0ps`)
- `setDefaultNetLoad 0.0pf`
  (*Default:* `0.1pf`)
- `setDefaultNetDelay 0ns`
  (*Default:* `2.5ns`)
- `setUseDefaultDelayLimit 0`
  (*Default:* `300`)
- `setDelayCalMode -enable_high_fanout true`
  (*Default:* `false`)

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_proto-timing_settings` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_proto-timing_settings`.
`-reset`	Resets all prototyping settings. The settings are enabled by default. It resets the variables mentioned above and rebuilds the timing graph. It also resets the following setting: `setDelayCalMode -ignoreNetLoad false` The above is set to true by `timeDesign -proto`, and must be reset. *Data_type*: bool, optional

Related Information

- `set_proto_model`
- Prototyping Methodologies chapter in the *Innovus User Guide*

SoC Architecture Info (SAI) Commands

- add_sai_boundary_flops
- end_sai
- read_sai
- report_sai_constraint

add_sai_boundary_flops

```
add_sai_boundary_flops
[-help]
[-clock string]
[-delete_flops]
-hinsts string
-ports string
[-ref_flop string [-clock_input_pin string]
[-data_input_pin string]
[-data_output_pin string]]
```

Allows insertion of boundary flops for empty modules/blocks. The command options specify the clock net name, the module name and the module pin name.

Parameters

`-clock` *string*	Specifies clock net name. *Data_type*: string, optional
`-clock_input_pin` *string*	Specifies the cell CLK port name. *Data_type*: string, optional
`-data_input_pin` *string*	Specifies the cell D port name. *Data_type*: string, optional
`-data_output_pin` *string*	Specifies the cell Q port name. *Data_type*: string, optional
`-delete_flops`	Removes all the flops that have been added. *Data_type*: string, optional
`-help`	Outputs a brief description that includes type and default information for each `add_sai_boundary_flops` parameter.
`-hinsts` *string*	Specifies the module name. **Note:** While specifying multiple hinsts and multiple ports, ensure that the number of specified hinsts is the same as the number of specified ports. *Data_type*: string, optional
`-ports` *string*	Specifies the module pin name. *Data_type*: string, optional
`-ref_flop` *string*	Specifies the flop cell name to be inserted along the boundary. *Data_type*: string, optional

Related Information

- `read_sai`
- `end_sai`
- Prototyping Methodologies chapter in the *Innovus User Guide*
- Syntax and Scripts chapter in the *Innovus User Guide*
    - Supported SAI Commands

end_sai

```
end_sai
[-help]
```

Disables the SoC Architecture Information (SAI) mode.

Parameters

`-help`	Prints the command usage.

Related Information

- `read_sai`
- Prototyping Methodologies chapter in the *Innovus User Guide*
- Syntax and Scripts chapter in the *Innovus User Guide*
  - Supported SAI Commands

read_sai

```
read_sai
[-help]
[name]
[-partial]
[-reduce_by_flexfiller]
[-rule_only]
```

Enables the SoC Architecture Information (SAI) mode that allows the use of SAI commands interactively at the command line.

Parameters

`-help`	Prints out the command usage.
name	Specifies the SAI file name. *Data_type*: string, optional
`-partial`	When specified, it enables SAI to support designs loaded using the `restoreDesign` command. Use this option to enable partial netlist support in the SAI mode. *Data_type*: bool, optional
`-reduce_by_flexfiller`	When specified, it adds flexfillers in hinsts added by the `add_module` and `create_module` commands. Use this option to reduce the size of the generated SAI netlist using the FlexFiller technology that is same as the FlexModel methodology. This helps to address the capability and run-time limitations of a growing design size (10 or 100 of million instances). *Data_type*: bool, optional
`-rule_only`	When specified, the rules are imported only in the SAI file. *Data_type*: bool, optional

Example

The following command reads in the *example2.0.sai* file and reduces the SAI netlist while maintaining the module areas:

```
read_sai example2.0.sai -reduce_by_flexfiller
```

Related Information

- end_sai
- Prototyping Methodologies chapter in the *Innovus User Guide*
- Syntax and Scripts chapter in the *Innovus User Guide*
    - Supported SAI Commands

report_sai_constraint

```
report_sai_constraint
[-help]
[-area_weight float]
[-spacing_weight float]
[-order_weight float]
[-congestion_weight float]
[-rule {all | rule_name | rule_name_list}]
```

You can use the `report_sai_constraint` command to validate a floorplan result and to check whether the specified SAI constraints are met. This enables front-end designers to easily evaluate a floorplan provided by their physical designers against the specified floorplan rules without requiring them to have much knowledge about using the Innovus System. The `report_sai_constraint` command reports:

- The general floorplan violations such as macro-to-macro overlap, fence-to-fence overlap, macro overlaps with fence, and/or if macros are placed outside their parent module boundary.
- SAI 2.0 constraint violations
- The Floorplan Quality Index that enables you to estimate whether a floorplan is ready to be used. Floorplan Quality Index is similar to the total negative slack (TNS) concept of timing analysis
- Support area, spacing, net weight, order, and congestion quality indexes; and their index weights.

Besides generating a report, it also creates a violation marker if violations are detected. These violation markers will be displayed in the Innovus artwork window and can be queried from the Innovus violation browser.

Based on the reported floorplan quality index, you can decide whether this floorplan is good to go or not. If the floorplan quality index does not meet your requirements, you can further:

- Adjust the wire length, congestion, and/or overlap cost factors and rerun the `proto_design` command to produce new floorplan result(s).
- Improve the current floorplan by manual adjustment.

Note: Use this command in the SAI interactive mode or write in the SAI file.

Parameters

`-area_weight` *float*	Specifies the weight of the area index. *Default*: 1.0 *Data_type*: float, optional
`-congestion_weight` *float*	Specifies the weight of the congestion index. *Default*: 1.0 *Data_type*: float, optional
`-help`	Outputs a brief description that includes type and default information for each `report_sai_constraint` parameter.
`-order_weight` *float*	Specifies the weight of the order index. *Default*: 1.0 *Data_type*: float, optional
`-rule {all |` *rule_name* `|` *rule_name_list*`}`	
	Checks and reports the violation(s) of the specified rule(s). *Default*: all (All SAI rules of the current design) *Data_type*: string, optional
`-spacing_weight` *float*	Specifies the weight of the area index. *Default*: 1.0 *Data_type*: float, optional

Related Information

- `read_sai`
- `end_sai`
- Prototyping Methodologies chapter in the *Innovus User Guide*
- Syntax and Scripts chapter in the *Innovus User Guide*
    - Supported SAI Commands

31

Rail Analysis Commands

- analyze_esd_network
- analyze_esd_voltage
- analyze_joule_heat
- analyze_package
- analyze_rail
- analyze_resistance
- analyze_self_heat
- analyze_signal_resistance
- calculate_didt
- calculate_differential_voltage
- calculate_noise_margin
- convert_gds_to_def
- create_current_region
- create_die_model
- create_hier_view
- create_power_pads
- create_what_if_shape
- debug_irdrop
- extract_package
- generate_esd_rlrp_report
- generate_power_up_report
- get_ir_insight
- map_dies
- map_die_package
- read_power_rail_results
- report_power_rail_results
- scale_what_if_capacitance
- scale_what_if_current
- scale_what_if_resistance
- set_advanced_package_options
- set_advanced_rail_options
- set_die_model
- set_dynamic_rail_simulation
- set_multi_die_analysis_mode
- set_net_group
- set_offchip_package_trace
- set_package

- set_pg_nets
- set_power_data
- set_power_pads
- set_rail_analysis_domain
- set_rail_analysis_mode
- set_self_heat_analysis_mode
- set_voltage_regulator_module
- view_dynamic_movie
- view_dynamic_waveform
- view_package_results

analyze_esd_network

```
analyze_esd_network
[-help]
<run_name>
[-clamp_lowest_layer_taps {true | false}]
[-clamp_pin_short_file pin_shorting_file]
-config_file rule_filename
[-output directory]
[-remove_layers_below layer_name]
[-ta | -task_assistant]
-type {net | domain}
[-use_power_pad {true | false}]
```

Performs different types of effective resistance checks, such as bump to clamp, bump to bump, and clamp to clamp, and current density analysis during an ESD zap event. This command checks whether every power or ground bump has an ESD device, and the device is placed in such a way that it does not violate the effective resistance or current density limit.

The objective of ESD Checker is to easily identify the violations caused by high effective resistance or the current density above the specified threshold value.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`<run_name>`	Mandatory parameter. Specifies the domain or net name. The result directory is prefixed with `<run_name>`.
`-clamp_lowest_layer_taps {true | false}`	
	Specifies to keep only the lowest layer of clamp taps for effective resistance analysis. The default value is `false`. • When set to `true`, the ESD pin nodes on the lowest layer are shorted, and effective resistance checks are performed to the shorted node on the lowest layer. • When set to `false`, the ESD pin nodes on all layers are shorted, and effective resistance checks are performed to the shorted pin nodes of clamps on different layers.
`-clamp_pin_short_file pin_shorting_file`	

	Specifies the list of pin layers to be shorted for bump-to-clamp and clamp-to-clamp resistance checks. If multiple layers for the same pin are specified, then these layers are shorted with each other.
	The format of the file is:
	`#Cell specification`
	`CELL <cell name> <pin> <layer>`
	`#Instance specification`
	`INST <instance name> <pin> <layer>`
	In this file, multiple layers for the same cell or instance can be specified by adding multiple rows for the same cell or instance name corresponding to each layer.
	If the `-clamp_pin_short_file` parameter is not specified, the software shorts all pin nodes of the lowest ESD cell pin layer, and performs effective resistance checks to the shorted node on the lowest layer.
	Note: If the `-clamp_pin_short_file` parameter is used in conjunction with `-clamp_lowest_layer_taps`, the `-clamp_pin_short_file` parameter will override the settings of `-clamp_lowest_layer_taps`.

`-config_file` *rule_filename*	
	Mandatory parameter. Specifies the name of the rule file that includes all the ESD checks to be performed as a set of rules. It also specifies the effective resistance threshold limit. For more information, refer to the **ESD Rule Specification** section in the "ESD Analysis" chapter of *Voltus User Guide*.
`-output` *directory*	
	Specifies the name of the output directory in which the ESD analysis results are saved.
`-remove_layers_below` *layer_name*	
	Specifies the layer name below which all layers are dropped from analysis. This parameter can be used in scenarios where the ESD cells connect to the PG grid at high layers, and therefore the impact of the lower PG layers is very small on effective resistance calculation and current density checks.
`-ta | -task_assistant`	
	Opens ESD Analysis Task Assistant, a task-based quick help functionality for the ESD analysis flow. The information in the Task Assistant is organized by topics relevant to the task you are performing. It answers frequently asked questions, such as format of the rule file, use model of the command, or supported ESD checks.
`-type {net | domain}`	
	Mandatory parameter. Specifies whether to run rail analysis on a domain or a net.
`-use_power_pad {true | false}`	
	Specifies that the x,y locations of the bump cells are to be obtained from the power pad file. This parameter can be used to support designs for which bump cells are not visible. The default value is `false`.

Examples

- The following commands specify to run ESD analysis on the domain `top`:

```
set_rail_analysis_mode -method static -accuracy hd -power_grid_library ./techonly.cl -ict_em_models ./Data/ESD_em.ict
analyze_esd_network top -config_file ./esd_new_rule.txt -type domain -output ESD_analysis -use_power_pad true
```

Related Information

- `set_rail_analysis_mode`
- `analyze_esd_voltage`
- "ESD Analysis" in the *Voltus IC Power Integrity Solution User Guide*

analyze_esd_voltage

```
analyze_esd_voltage
[-help]
-current_distribution_layer layer
-driver_receiver_file filename
-esd_pin_location_file filename
[-maximum_violations value]
-net NetName
[-report filename]
[-threshold value]
-total_current value
[-detailed_report {true | false}]
```

Performs IR Drop analysis for every driver/receiver pair and reports voltage drop above the given limit. Use this command to provide the current injection points (sinks), the current distribution layer (source), and the total ESD current to be distributed. The command generates a report containing all the violations where potential difference across driver/receiver is greater than the given threshold.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-current_distribution_layer` *layer*	
	Specifies to distribute current to all nodes of the given layer. This is the LEF/DEF layer name. This is a required parameter.
`-detailed_report {true | false}`	
	Specifies to report the differential voltage for each zap point above the specified threshold. It gives the x,y location of the zap point and the zap point name. Using this parameter, you can fix all the differential voltage violations in a single run. This parameter will generate the report file with the same name as specified using the `-report` parameter and with the `_detail.rpt` suffix. *Default* : `false` The following is an example of the parameter: `analyze_esd_voltage -net VSS -driver_receiver_file ./driverdb_railVSS -esd_pin_location_file VSS.pp -current_distribution_layer METAL5 -total_current 8 -report FULL_new_gnd_esd_cdm.rpt -detailed_report true -threshold 2`
`-driver_receiver_file` *filename*	
	Specifies the name of the driver/receiver file of the net. This file is created by the `-create_driver_db` parameter of the `set_power_analysis_mode`. This is a required parameter. The file has multiple columns, wherein the first column is the receiver, and the remaining columns are drivers to the receiver instance. The following is an example of the file content: **`padring/I_RCV_7`** `io_toplevel/padring/I_DECAP io_toplevel/padring/` In this example: `padring/I_RCV_7` is the receiver instance. `io_toplevel/padring/I_DECAP` and `io_toplevel/padring/` are driver instances.
`-esd_pin_location_file` *filename*	

	Specifies the name of the ESD pin location file. This is a required parameter.
	The format of the file is:
	`#Pin_name x coord y coord layer`
	`gnd_pin_1 2284.6 -2898.05 m4`
	`...`
	`...`

`-maximum_violations` *value*	

	Specifies the number of max worst violations to be reported. This is an optional parameter. When this value is specified, only the specified number of top violations will be reported. If a value is not specified, voltage drop for all driver/receiver pairs will be reported.
`-net` *`NetName`*	
	Specifies the name of the net to be analyzed. This is a required parameter.
`-report` *`filename`*	
	Specifies the name of the output report file in which details of the driver-receiver violation pair is written. This is an optional parameter. The report is filtered in descending order of differential voltage. The output file has four columns: • First column - `Differential voltage(V)` • Second column - `Driver name (node location(um) )` • Third column - `Receiver name (node location(um))` • Fourth column - `ESD pin name (location(nm))` The following is the sample ESD file format: **`14.1324 pmc/a_ip_pmc_chassis_c55fg ( 2050.875 -2760.085 ap ) io_toplevel/padring/I_ESDCLAMP_39 ( 1428.200 2884.250 m4 ) gnd_pin_22 ( 1458.200 2898.050 m4 )`**
`-threshold` *`value`*	
	Specifies the minimum allowed driver/receiver voltage for filtering the report. This is an optional parameter. When a threshold value is specified, driver/receiver pairs with differential voltage greater than the specified threshold value will be reported. If a threshold value is not specified, voltage drop for all driver/receiver pairs will be reported.
`-total_current` *`value`*	
	Specifies the total ESD current to be distributed. The default current unit is amperes. This is a required parameter.

Example

- The following example performs static IR Drop analysis to check the voltage drop between all driver and receiver pairs:

```
analyze_esd_voltage -net gnd -driver_receiver_file ./POWER/driverdb_railgnd  -esd_pin_location_file ./epinfile -
current_distribution_layer m1 -total_current 8 -report FULL_new_gnd_esd_cdm.rpt
```

Related Information

- `set_rail_analysis_mode`
- `analyze_esd_network`
- "ESD Analysis" in the *Voltus IC Power Integrity Solution User Guide*

analyze_joule_heat

```
analyze_joule_heat
[-help]
[-detail_delta_temperature_file filename]
[-detail_delta_temperature_region {x1 y1 x2 y2}]
[-domain rail_domain_name]
[-net pg_net_name]
[-report_conn_pin_wire filename]
[-scale_rms_limit value]
[-tile_delta_temperature_file filename]
[-tiles {n_tile_in_x m_tile_in_y}]
[-output_directory dir_name]
```

Specifies to perform joule heat analysis. This command will use the dynamic RMS current to compute delta-T RMS on the power and signal wires for all segments in the design. The `analyze_joule_heat` command is similar to the `analyze_self_heat` command, except that the `analyze_self_heat` command computes both delta-T FEOL (FEOL self heat) and delta-T RMS (BEOL self heat) whereas the `analyze_joule_heat` command computes only delta-T RMS.

Joule heat analysis (`analyze_joule_heat`) should be performed for the entire design, that is, you must use domain analysis with all PG nets (`-domain ALL`). The `-net <pgnet>` parameter should be used only for preliminary checking.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `analyze_joule_heat` parameter. For a detailed description of the command and all of its parameters, use the man command `man analyze_joule_heat`.
`-detail_delta_temperature_file filename`	
	Specifies the name of the generated BEOL detailed delta-temperature file.
`-detail_delta_temperature_region {x1 y1 x2 y2}`	
	Specifies the region coordinates for generating detailed delta temperature file.
`-domain rail_domain_name`	
	Specifies the domain name of the rail.
`-net pg_net_name`	
	Specifies a net for joule heat analysis.
`-output_directory dir_name`	
	Specifies the directory name for storing the joule heat analysis results. When you specify this parameter, the following result files are saved in the specified directory: • Rail analysis output directory • Detail delta temperature file If `-output_directory` is not specified, by default the joule heat analysis results are stored in the current working directory.
`-report_conn_pin_wire filename`	
	Reports results on wires connecting to cell pins.
`-scale_rms_limit value`	
	Specifies the scale factor for the root-mean-square (rms) limit relaxation factor during Electromigration current limit analysis.
`-tile_delta_temperature_file filename`	
	Specifies the name of the generated BEOL tile-based delta-temperature file.

`-tiles {n_tile_in_x m_tile_in_y}`	
	Specifies the number of tiles in the x and y directions in the tile-based delta temperature file.

Examples

- The following command is an example of `analyze_joule_heat` command:

```
analyze_joule_heat -domain ALL -tile_delta_temperature_file joule_ttm.txt -tiles {10 10} -detail_delta_temperature_file
joule_heat_detail.rpt -report_conn_pin_wire joule_heat.rpt
```

Related Information

- `set_rail_analysis_mode`
- `analyze_self_heat`
- "Self-Heating Effect Analysis" in the *Voltus IC Power Integrity Solution User Guide*

analyze_package

```
analyze_package
[-help] <domain>
[-computer_resources string]
[-output <directoryname>]
-pkg_bga_name <bga_circuitname>
-pkg_die_name <die_circuitname>
-pkg_pin_current_file <filename>
-result_name <filename>
[-spd_file filename]
-type <package_analysis_type>
-workspace <workspace_name>
```

Specifies how the package analysis will be performed, and to run package analysis.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
domain	Specifies the rail analysis domain name.
`-computer_resources` *string*	
	Specifies the computer resources for the package analysis run. This parameter allows you to customize the host distribution/CPU usage for package analysis. The default setting is "`-smt 0 -local -cn localhost -cpus 8`".
`-output <`*directoryname*`>`	
	Specifies the output directory name of the package analysis results. By default, the package model will be created in the specified workspace.
`-pkg_bga_name <`*bga_circuitname*`>`	
	Defines the ball grid array (BGA) circuit name in the package model.
`-pkg_die_name <`*die_circuitname*`>`	
	Defines the die circuit name in the package model.
`-pkg_pin_current_file <`*filename*`>`	
	Specifies the name of the generated pin current file from rail analysis that contains the package pin names and the current values. It is automatically generated after the package model is created.
`-result_name <`*filename*`>`	
	Specifies the name of the .xml result file.
`-spd_file` *filename*	
	Specifies an existing Sigrity .spd file for package analysis. This parameter allows you to specify a .spd file instead of the workspace name (package model database) of the package analysis (PowerDC) tool.
`-type <`*package_analysis_type*`>`	
	Specifies the type of package analysis. Currently, the software supports only DC package analysis.
`-workspace <`*workspace_name*`>`	
	Specifies the workspace name for package analysis (powerDC). The file name extension is `.pdcx`.

Example

- The following example specifies the package analysis details and starts package analysis:

```
analyze_package ALL \
-type dc \
-workspace /../sigrity/rak_pdc_irdrop/test.pdcx \
-pkg_bga_name FPBGA144_Signal02 \
-pkg_die_name CD1_Signal01 \
-output rak_pdc_irdrop_voltus_cpf \
-result_name rak_pdc_irdrop_voltus \
-pkg_pin_current_file /../spa_cpf/ALL_125C_avg_1/Reports/super_filter_pkg_iv.rpt
```

Related Information

- set_rail_analysis_mode
- view_package_results
- "Package Analysis" in the *Voltus IC Power Integrity Solution User Guide*

analyze_rail

```
analyze_rail
[-help]
[-type {domain | net}]
[-output directory_name]
name
```

Runs rail analysis on a net or domain. This command must be specified after defining the rail analysis mode properties using the `set_rail_analysis_mode` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `analyze_rail` parameter. For a detailed description of the command and all of its parameters, use the man command `man analyze_rail`.
name	Specifies the name of the domain or net that you want to analyze.
`-output` *directory_name*	
	Specifies the name of the directory in which to store the output directories created during the analysis. *Default*: Current working directory.
`-type {domain | net}`	
	Specifies whether to run rail analysis on a domain or a net. A domain is a list of power or ground nets to analyze together. See `set_power_pads` for more details. You can specify the variable "ALL" to analyze all domains.

Examples

- The following command performs an analysis for the `vdd` net and places the results directories in the `analysis_out` directory:
  ```
  analyze_rail -type net -output analysis_out vdd
  ```
- The next command performs rail analysis on TDSP power domain and writes the results in current working directory.
  ```
  analyze_rail -type domain -output ./ TDSP
  ```
- The following command performs rail analysis on all power domains and writes the results in the current working directory:
  ```
  analyze_rail -type domain ALL -output ./
  ```

Related Information

- `set_rail_analysis_mode`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

analyze_resistance

```
analyze_resistance
[-help]
[-cell master_cell_name]
[-die_mode {3dic}]
[-force_reextraction {true | false}]
[-instance_list {instance1 pin1 instance2 pin2...}]
[-instance_list_file filename]
[-instance_pair_list {instance1 instance2 pin1  pin2 ...}]
[-instance_pair_list_file filename]
-net netName | -domain domainName
[-node_list {node1 node2.....  METAL_LAYER LABEL}]
[-node_list_file filename]
[-node_pair_list {node1X node1Y METAL_LAYER1 node2X node2Y METAL_LAYER2...LABEL}]
[-node_pair_list_file filename]
[-output filename]
[-output_dir directory_name]
[-region {x1 y1 x2 y2 layer number_of_nodes region_name}]
[-report_limit value]
[-threshold value]
[-print_cell_name]
[-domain_threshold value]
[-net_threshold value]
[-exclude_list_file filename]
```

Performs effective resistance analysis. Effective resistance computes effective resistance between two nodes on the power-grid, or from a node to all the voltage sources in the design, or from an instance to all the voltage sources in the design.

You can run this command without performing power calculation and IRdrop analysis. The flow requires you to load the design, set power nets, set power pads, and run resistance analysis. This command enables the tool to consider the entire power structure, including embedded ring structure within IO cells if any.

Note: The `analyze_resistance` full chip analysis supports running the `set_rail_analysis_mode -enable_rlrp_analysis` command to perform least resistance path analysis (RLRP) in conjunction with effective resistance analysis (REFF) in the DP mode. However, RLRP and REFF analysis should be done separately in the XP mode. When performing the point-to-point effective resistance analysis (`analyze_resistance`) for the power/signal nets in the XP mode, disable the RLRP analysis (`set_rail_analysis_mode -enable_rlrp_analysis false`).

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `analyze_resistance` parameter. For a detailed description of the command and all of its parameters, use the man command `man analyze_resistance`.
`-cell` *master_cell_name*	
	Reports effective resistance for all instances of the specified master cell name. The `-cell` parameter improves the usability of effective resistance ca specify only the cell name of the instances you want to analyze. You can specify multiple cells for performing effective resistance analysis of multipl
`-die_mode {3dic}`	
	Specifies to perform multi-die effective resistance analysis.
`-domain` *domainName*	
	Specifies the domain name that is to be analyzed.
`-domain_threshold` *value*	
	Specifies the minimum allowed effective resistance (effective resistance sum of each net) to instance. For specific instance list analysis, the instanc will be reported as “PASS”, while the instances above the threshold value will be reported as “FAIL”. For full chip analysis, the instances below the t reported in the domain report. The default value is 0.
`-exclude_list_file` *filename*	

Specifies the name of a file containing a list of cells or instances that will not be processed during the resistance analysis flow. The specified cells/ir report generation.
The use model of this parameter is:

```
analyze_resistance -exclude_list_file <file_name>
```

File Format:

```
<CELL | INST> <name>
```

Example:

```
CELL BUMP_MASTER
CELL ESD_MASTER
INST INSTA
```

`-force_reextraction {true | false}`

Specifies to force a re-extract of resistances.

`-instance_list {instance1 pin1 instance2 pin2...}`

Specifies the list of instances for which effective resistance is to be calculated. *pin* is an optional argument. By default, the tool will use the instance being analyzed. The *pin* argument can be used in special cases when two different pins of an instance are connected to the same net.

Note: The `-instance_list` and `-instance_list_file` parameters are mutually exclusive to the `-instance_pair_list` and `-instance_pair_list_fil` that you can specify `-instance_list` and `-instance_list_file` together but you cannot specify `-instance_list` and `-instance_list_pair` together.

`-instance_list_file filename`

Specifies the name of the instance list file. This file contains the list of instances in the same format as mentioned for the `-instance _list` paramete specify a large number of instances. `-instance _list_file` and `-instance _list` are mutually exclusive.

Note: The `-instance_list` and `-instance_list_file` parameters are mutually exclusive to the `-instance_pair_list` and `-instance_pair_list_fil` that you can specify `-instance_list` and `-instance_list_file` together but you cannot specify `-instance_list` and `-instance_list_pair` together.

`-instance_pair_list {instance1 instance2 pin1 pin2 ... }`

Specifies the instance pair list to enable effective resistance calculation between two instances in the design. *pin1* and *pin2* are pins of *instance1*

Note: The `-instance_list` and `-instance_list_file` parameters are mutually exclusive to the `-instance_pair_list` and `-instance_pair_list_fil` that you can specify `-instance_list` and `-instance_list_file` together but you cannot specify `-instance_list` and `-instance_list_pair` together.

`-instance_pair_list_file filename`

Specifies the name of the instance pair list file. This file contains the instance pair list in the same format as mentioned for the `-instance _pair_list` you to specify a large number of instance pairs. `-instance_pair_list_file` and `-instance_pair_list` are mutually exclusive.

Note: The `-instance_list` and `-instance_list_file` parameters are mutually exclusive to the `-instance_pair_list` and `-instance_pair_list_fil` that you can specify `-instance_list` and `-instance_list_file` together but you cannot specify `-instance_list` and `-instance_list_pair` together.

`-net netName`

Specifies the net name that is to be analyzed.

`-net_threshold value`

The effective resistance requirements for different power nets vary depending on the design. For example, an array or periphery supply to an SRAM lower effective resistance threshold compared to the core supplies. This parameter allows you to specify different threshold values for different nets.

The use model of this parameter is given below:

```
set_rail_analysis_domain -name ALL -pwrnets {vdd vddar vdda vdd1 vdd2} -gndnets vss
analyze_resistance -threshold 50 -net_threshold {{vdd 50} {vddar 20} {vdda 35} {vss 40}} -domain ALL -output_dir Reffective
```

In this example, the threshold value for different nets are:

vdd -> 50
vddar →20
vdda -> 35
vss -> 40
vdd1 -> 50
vdd2 -> 50

Here, for all the remaining nets for which a value is not specified using the `-net_threshold` parameter, the value specified with the `-threshold` parar

The different threshold values will have an impact on the list of instances reported in the `<state_dir>/Reports/<net>/<net>.effr` report.

`-node_list {node1 node2..... METAL_LAYER LABEL}`

Specifies the list of user-defined nodes for effective resistance calculation between the specified user-defined nodes and all voltage sources. You ca on the region on a given layer. `LABEL` is any name chosen by you to identify node entries in the list. This is optional but helps during debugging.

Note: The `-node_list` and `-node_list_file` parameters are mutually exclusive to the `-node_pair_list` and `-node_pair_list_file` parameters. This `-node_list` and `-node_list_file` together but you cannot specify `-node_list` and `-node_pair_list` together.

`-node_list_file filename`

Specifies the name of the node list file. This file contains the list of user-defined nodes in the same format as mentioned for the `-node_list` paramete specify a large number of nodes.

You can specify multiple nets for node list based resistance analysis. The use model for multiple nets is:
`analyze_resistance -domain <domain name> -node_list_file <filename>`

Here,

- the `-domain` parameter will specify the domain name, that is, the domain with nets that is required to be analyzed.
- the `-node_list_file` parameter will include the multiple net names

The format for specifying multiple nets is:

```
NET <netname1>
node1X node1Y METAL_LAYER [LABEL]
node2X node2Y METAL_LAYER [LABEL]

NET <netname2>
node1X node1Y METAL_LAYER [LABEL]
node2X node2Y METAL_LAYER [LABEL]
```

Note: The `-node_list` and `-node_list_file` parameters are mutually exclusive to the `-node_pair_list` and `-node_pair_list_file` parameters. This `-node_list` and `-node_list_file` together but you cannot specify `-node_list` and `-node_pair_list` together.

`-node_pair_list {node1X node1Y METAL_LAYER1 node2X node2Y METAL_LAYER2... LABEL }`

Specifies the node pair list for effective resistance calculation between two nodes on the power-grid. `You can add a list of nodes based on the LABEL` is any name chosen by you to identify node entries in the list. This is optional but helps during debugging.

Note: The `-node_list` and `-node_list_file` parameters are mutually exclusive to the `-node_pair_list` and `-node_pair_list_file` parameters. This `-node_list` and `-node_list_file` together but you cannot specify `-node_list` and `-node_pair_list` together.

`-node_pair_list_file filename`

Specifies the name of the node pair list file. This file contains the node pair list in the same format as mentioned for the `-node_pair_list` parameter. a large number of node pairs.

You can specify multiple nets for node pair list based resistance analysis. The use model for multiple nets is:
`analyze_resistance -domain <domain name> -node_pair_list_file <filename>`

Here,

- the `-domain` parameter will specify the domain name, that is, the domain with nets that is required to be analyzed.
- the `-node_pair_list_file` parameter will include the multiple net names

The format for specifying multiple nets is:

```
NET <netname1>
node1X node1Y METAL_LAYER1 node2X node2Y METAL_LAYER2 [LABEL]
node3X node3Y METAL_LAYER3 node4X node4Y METAL_LAYER4 [LABEL]

NET <netname2>
node1X node1Y METAL_LAYER1 node2X node2Y METAL_LAYER2 [LABEL]
node3X node3Y METAL_LAYER1 node4X node4Y METAL_LAYER4 [LABEL]
```

Note: The `-node_list` and `-node_list_file` parameters are mutually exclusive to the `-node_pair_list` and `-node_pair_list_file` parameters. This `-node_list` and `-node_list_file` together but you cannot specify `-node_list` and `-node_pair_list` together.

`-output filename`

Specifies the name of the output report file that contains the report header and the sorted list for resistance values.

Note: This parameter should not be specified in the XP flow as the XP reports are consolidated under the state directory by default.

`-output_dir` *directory_name*	
	Specifies the name of the directory in which the output report file is saved.
`-print_cell_name`	
	Specifies to print cell names in the effective resistance report. This parameter allows you to sort resistance values by cell names.
`-region {`*x1 y1 x2 y2 layer number_of_nodes region_name*`}`	
	Analyzes resistance in the specified region. You can draw a box on a specific layer, and specify the number of nodes to be selected (maximum limit parameter allows you to compute effective resistance on a given layer across the chip and determine how the resistance is changing for the design can assign a name for the region.
`-report_limit` *value*	
	Specifies the number of instances printed in the report to save disk space.
`-threshold value`	
	Specifies the minimum allowed effective resistance to display node/node-pair/instance/instance-pair that have a resistance value above this thresh threshold as a non zero value, then instances with resistance value above that threshold will be tagged as FAIL and the rest will be tagged as PASS

Examples

- The following command performs effective resistance calculation for the list of nodes and instances:
  ```
  analyze_resistance -net VSS -node_list { 231 1061.6 M1 } -instance_list_file instance.list
  ```
- The following command specifies a list of user-defined nodes and a node list file for effective resistance calculation:
  ```
  analyze_resistance -net VDD -node_list {{ 213.556 133.469 M1  I123 }} -node_list_file /../node_list.txt.1 -output reff_rpt
  ```
- The following is an example of specifying a list of instances for effective resistance calculation:
  ```
  analyze_resistance -instance_list {{INV11 vdd} {INV21 vdd}} -net VDD
  ```
- The following is an example of specifying a list of user-defined nodes for effective resistance calculation:
  ```
  analyze_resistance -net VDD -node_list {{800 11.88 M4} {999.768 11.88 M4} {1000.296 11.880 M4}} -output out.txt
  ```
- The following is an example of specifying node pair list for effective resistance calculation:
  ```
  analyze_resistance -net VDD -node_pair_list {195.728 133.369 M1 213.556 133.469 M1} -output out_nodelist.txt
  ```
- The following is an example of specifying instance pair list for effective resistance calculation:
  ```
  analyze_resistance -net GND -instance_pair_list {inst1 inst2 GND GND} -output outfile -threshold 100
  ```
- The following command performs effective resistance calculation for full chip analysis:
  ```
  analyze_resistance -threshold 13.0  -domain_threshold 25.0 -domain ALL
  ```

  Sample Report:

  ```
  NET: VSS VDD
  # Threshold:   25.000
  # Total Effective resistance for instances sorted from high to low
  # PASS/FAIL         REFF  INSTANCE                 PIN...

    FAIL            25.664  clk__L1_I1       VDD(51.52%)      VSS(48.48%)
    FAIL             25.35  clk__L2_I1       VDD(52.16%)      VSS(47.84%)
    FAIL            25.037  clk__L3_I1       VDD(51.56%)      VSS(48.44%)
  ```

- The following command performs effective resistance calculation for instance list analysis:
  ```
  analyze_resistance -domain ALL -threshold 13.0 -domain_threshold 25.0 -instance_list_file inst.list
  ```

  Sample Report:

  ```
  NET: VSS VDD
  # Threshold:   25.000
  # Total Effective resistance for instances sorted from high to low

  # PASS/FAIL         REFF  INSTANCE             PIN...
    FAIL            25.664  clk__L1_I1       VDD(51.52%)      VSS(48.48%)
    FAIL            25.35   clk__L2_I1       VDD(52.16%)      VSS(47.84%)
    FAIL            25.037  clk__L3_I1       VDD(51.56%)      VSS(48.44%)
    PASS            24.724  clk__L4_I1       VDD(52.21%)      VSS(47.79%)
    PASS            23.047  inst_pll         VDD(50.55%)      VSS(49.45%)
  ```

- The following command reports reff for all instances of the cell type CKBD8:
  ```
  analyze_resistance -net VDD -cell CKBD8
  ```
  The following is a sample reff report generated using the `-cell` parameter:

  ```
  ****************

  # Layer Representation: LEF
  # Coordinate Unit: um
  # Resistance Unit: Ohm

  NET: VDD
  # Threshold:    0.000
  # Effective resistance for instances sorted from low to high
  # PASS/FAIL       REFF        INSTANCE          PIN
    -             3.5752    add_94_37_I2/g695    VDD
    -             3.7521    add_94_37_I2/g694    VDD
  ```

Related Information

- `set_rail_analysis_mode`
- `analyze_signal_resistance`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*

analyze_self_heat

```
analyze_self_heat
[-help]
[-domain rail_domain_name]
[-output_directory   dir_name]
```

Runs Self-Heating Effect (SHE) analysis on a single die or multiple dies. This command must be specified after defining the SHE properties using the `set_self_heat_analysis_mode` command.

Self-heat analysis (`analyze_self_heat`) should be performed for the entire design, that is, you must use domain analysis with all PG nets (`-domain ALL`).

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `analyze_self_heat` parameter. For a detailed description of the command and all of its parameters, use the man command `man analyze_self_heat`.
`-domain rail_domain_name`	
	Specifies the domain name of the rail.
`-output_directory   dir_name`	
	Specifies the directory name for storing the self-heat analysis results. When you specify this parameter, the following result files are saved in the specified directory: • Rail analysis output directory • Detail delta temperature file • Instance delta temperature file • Tile delta temperature file If `-output_directory` is not specified, by default the self-heat analysis results are stored in the current working directory.

Examples

- The following command is an example of the `analyze_self_heat` command in the multi-die analysis flow:

```
set_self_heat_analysis_mode \
-die_instance_name die0 \
-alpha_parameters {{M1 0.74 0.85} {M2 0.62 0.85} {M3 0.53 0.85} ....} \
-beta_parameters {0.00423 -0.00045 0.81534}\
-instance_power_file ./dynamic_power.rpt \
-cell_thermal_resistance_file TRF.txt \
-instance_delta_temperature_file sh_itm.txt \
-tile_delta_temperature_file sh_ttm.txt \
-tiles {10 10} \
-detail_delta_temperature_file power_detail_die0.rpt

set_self_heat_analysis_mode \
-die_instance_name die1 \
-alpha_parameters {{M1 0.74 0.85} {M2 0.62 0.85} {M3 0.53 0.85} ...} \
-beta_parameters {0.00423 -0.00045 0.81534}\
-instance_power_file ./dynamic_power.rpt \
-cell_thermal_resistance_file TRF.txt \
-instance_delta_temperature_file sh_itm.txt \
-tile_delta_temperature_file sh_ttm.txt \
-tiles {10 10} \
-detail_delta_temperature_file power_detail_die1.rpt

set_self_heat_analysis_mode \
-die_instance_name die2 \
-alpha_parameters {{M1 0.74 0.85} {M2 0.62 0.85} {M3 0.53 0.85} ...} \
-beta_parameters {0.00423 -0.00045 0.81534}\
-instance_power_file ./dynamic_power.rpt \
-cell_thermal_resistance_file TRF.txt \
-instance_delta_temperature_file sh_itm.txt \
-tile_delta_temperature_file sh_ttm.txt \
-tiles {10 10} \
-detail_delta_temperature_file power_detail_die2.rpt

analyze_self_heat -domain core
```

Related Information

- `set_rail_analysis_mode`
- `analyze_joule_heat`
- "Self-Heating Effect Analysis" in the *Voltus IC Power Integrity Solution User Guide*

analyze_signal_resistance

```
analyze_signal_resistance

[-help]
[-instance_pair_list_file filename]
-net net_name
[-output_dir directory]
[-report filename]
[-threshold value]
```

Computes effective resistance for signal nets from a given instance1/pin1 to instance2/pin2 accounting all applicable manufacturing effects. You can use the effective resistance output file to determine whether the critical routes are within the specified resistance limits.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-instance_pair_list_file filename`	
	Specifies the file name containing the signal net instance-pin pairs for which effective resistance needs to be computed. The format of the file is: `INST1 PIN1 INST2 PIN2`
`-net net_name`	
	Specifies the name of the signal net.
`-output_dir directory`	
	Specifies the name of the output directory.
`-report filename`	
	Specifies the name of the report file in which details of effective resistance between instance-pin pairs are written.
`-threshold value`	
	Specifies the minimum allowed effective resistance between instance-pin pairs.

Examples

- The following command computes effective resistance for the signal net `n_13190`:
  ```
  analyze_signal_resistance -net n_13190
  ```
- The following command computes effective resistance for the signal net `n_13190`, and writes the effective resistance details in the report `n_13190.rpt`:
  ```
  analyze_signal_resistance -net n_13190 -report n_13190.rpt
  ```
- The following command computes effective resistance for signal net instance-pin pairs specified in the file `instance.list`:
  ```
  analyze_signal_resistance -net n_0 -instance_pair_list_file instance.list
  ```

Related Information

- `set_rail_analysis_mode`
- `analyze_resistance`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*

calculate_didt

```
calculate_didt
[-help]
-net_name net_name
-nworst value
-state_directory dir
[-window_steps value | -window_size value ]
```

Generates the on-die di/dt information from an existing dynamic rail analysis run and saves it in a text file. This command allows you to determine the worst N di/dt events for each bump and for the sum of all bumps (vc_sum).

Parameters

`-help`	Prints a brief description that includes the type and default information for each `calculate_didt` parameter. For a detailed description of the command and all of its parameters, use the man command: `man calculate_didt`
`-net_name net_name`	
	Specifies a net for which the di/dt report is to be generated.
`-nworst value`	
	Specifies the number of worst windows with the largest di/dt values to be reported per bump.
`-state_directory dir`	
	Specifies the name of the state directory where the information on the bump-based di/dt information from the previous run resides.
`-window_steps value`	
	Specifies the transient time step size or resolution of the sliding window. The default value is `5`.
`-window_size value`	
	Specifies the duration of the sliding window in seconds. **Note**: The `-window_size` and `-window_steps` parameters are mutually exclusive. When both the parameters are specified, the `-window_size` parameter takes precedence. If you do not specify either of the two parameters, the results will be based on a default of 5 resolution steps.

Examples

- The following command generates the on-die di/dt information:
  ```
  calculate_didt -nworst 3 -window_steps 10 -net_name VDD10 -state_directory ./dynamic_rail/core_125C_dynamic_1
  ```

Related Topics

- "Static Power, IRdrop and EM Analysis" in the *Voltus User Guide*

calculate_differential_voltage

```
calculate_differential_voltage
[-help]
-diffv_limit value
-power_directory directory_name
-state_directory directory_name
```

Determines the differential voltage between a receiver/driver pair during ramp-up analysis. This command can be used to mitigate the risk of high short-circuit current for a driver-receiver instance pair due to slow/fast (differential) ramp-up voltage at the output of the power-switch cell connected to the driver/receiver cell.

The `calculate_differential_voltage` command reports the differential potential for those receiver/driver pairs that are greater than the the specified differential voltage threshold, and the time interval at which the differential voltage remains greater than the threshold. You must specify the `set_power_analysis_mode -create_driver_db true` parameter before specifying this command.

The following is the format of the output report:

```
# <driverName>   <receiverName>   <peak_diffv>  <minT>   <maxT>
```

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-diffv_limit value`	
	Specifies to report differential voltage for the receiver/driver pairs that exceed the specified threshold limit.
`-power_directory directory_name`	
	Specifies the path to the power directory to access the driver-receiver pairs.
`-state_directory directory_name`	
	Specifies the path to the state directory generated by rail analysis to access the EIV or timing database.

Examples

- The following command lists all the driver-receiver pairs that exceed the differential voltage limit of 0.001:

  ```
  calculate_differential_voltage -diffv_limit 0.001 -state_directory dynamic_rail/core_105C_dynamic_1 -power_directory dynamic_power
  ```

  The following is a snippet of the generated report:

  ```
  # <driverName>          <receiverName>        <peak_diffv>     <minT>     <maxT>
  column/MSV_I2         column/MULT8_reg_1_4     0.7382          5e-11    2.75e-09
  column/HIER_INST      column/MULT8_reg_1_4     0.7382          5e-11    2.75e-09
  ...
  ```

Related Information

- `set_rail_analysis_mode`
- `calculate_noise_margin`
- "Static Power, IRdrop and EM Analysis" and "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

calculate_noise_margin

```
calculate_noise_margin
[-help]
-noise_limit_file input_file
-power_directory dir
-state_directory dir
```

Specifies to calculate noise margins with worst voltages on the driver-receiver pins. To run this command, you need to specify the default or cell-specific noise limit, signal noise, guardband, rail directory, and power directory. Before running the `calculate_noise_margin` command, you need to specify the `set_power_analysis_mode -create_driver_db true` parameter to generate the driver-receiver database.

The use model of the command is:

```
set_power_analysis_mode ... -create_driver_db true
calculate_noise_margin -noise_limit_file input_file -state_directory dir -power_directory file
```

The command generates the noise margin output file in the state directory. The format of the noise margin output file (*powernetname_groundnetname*`.noise_margin`) is:

Margin_High(V)	Margin_Low(V)	NMH(V)	NMI(V)	Voh(V)	Vol(V)	Vih(V)	Vil(V)	Signal_Noise(V)	Guardband(V)	Dr Inst/pin	Rx Inst/pin
NMH - signal noise - guardband	NMI - signal noise - guardband	Voh - Vih	Vil - Vol	VDD_driver - VSS_receiver	VSS_driver - VSS_receiver	(1-noise_limit) * (VDD_receiver - VSS_receiver) noise_limit is the noise limit value specified in the noise limit file	noise_limit* (VDD_receiver - VSS_receiver) noise_limit is the noise limit value specified in the noise limit file	Signal noise value specified in the noise limit file	Guardband value specified in the noise limit file	Driver pin name	Receiver pin name

Parameters

`-help`	Prints a brief description that includes the type and default information for each `calculate_noise_margin` parameter. For a detailed description of the command and all of its parameters, use the man command: `man calculate_noise_margin`
`-noise_limit_file` *input_file*	

	Specifies the name of the input noise limit file that contains the default or cell-specific noise limit, signal noise, and guardband. The format of the noise limit file is:
	`default_noise_limit <value>`
	`default_signal_noise <value>`
	`default_guard_band <value>`
	`<cell_name> <noise_limit> <signal_noise> <guard_band>`
	The following is a snippet of the noise limit file:
	`default_noise_limit 0.46`
	`default_signal_noise 0.1`
	`default_guard_band 0.05`

`-power_directory` *dir*	

	Specifies the path to the power directory to access driver-receiver pairs.
`-state_directory` *dir*	
	Specifies the path to the state directory generated by rail analysis to access EIV database.

Examples

- The following command calculates noise margin:
  ```
  calculate_noise_margin -noise_limit_file ./noiselimit -state_directory dynamic_rail/core_105C_dynamic_1 -power_directory dynamic_power
  ```

Related Information

- `set_rail_analysis_mode`
- `calculate_differential_voltage`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*

convert_gds_to_def

```
convert_gds_to_def
[-help]
[-black_box_cell_file filename]
-gds_file filename
-gds_layermap filename
[-inst_name_file filename]
-net_location_file filename
[-output_components {ALL | NONE | CONN}]
-output_def_file filename
[-output_lef_macros filename]
-power_grid_library libraryname
-top_cell cellname
[-use_gds_label {ALL | TOP | <tech_layer_name> | <cell_name>:<tech_layer_name> | <cell_name>:ALL}]
[-verilog_file filename]
[-spef_file filename]
[-esd_marker_layer_file filename]
[-esd_cell_bbox_file filename]
[-inst_name_prefix prefix]
[-ignore_gds_inst_name]
[-print_complete_short_path]
[-extract_signal_net_logic_info_only]
[-zero_pin_size]
```

A GDS2DEF utility that converts a given GDSII file to a DEF file for power/rail signoff analysis. The utility can convert both cell-level GDS and full-chip level GDS to DEF. The generated DEF contains all the geometries for the P/G routing, the specified P/G pin information, component and logical connectivity information.

Parameters

`-help`	Outputs the command usage.
`-black_box_cell_file filename`	

	Specifies the name of a file containing a list of black box cells that the utility is to ignore. You can specify both exact string matches and wildcard searches in the specified file.
	For exact string matches, each cell name must reside on a separate line in the file, as shown below:
	`CellA`
	`CellB`
	The wildcard search feature allows you to search cell names that contain wildcard characters, such as square brackets and underscore. When you want to perform wildcard search, type the keyword `wildcard` first, followed by the cell name. For example, if you want to ignore cell names which contains CLK, add one line `wildcard .*CLK.*`.

`-esd_marker_layer_file filename`	

	Specifies a file that includes a list of ESD cell/instance names and their corresponding GDS marker layers that represent the bounding box of the ESD cells. The format of the file is:
	`<cell_name> <marker_layer> [top_layer] [bottom_layer]`
	Here,
	• `cell_name` is the ESD cell/instance name
	• `marker_layer` could contain multiple GDS layers, separated by ";"
	• The optional `top_layer` and `bottom_layer` specifies the top and bottom technology layers of the ESD cell pin shape. You can use these optional arguments to specify the top and bottom technology layers of the ESD cell pin shape. This provides you with the flexibility to specify the lowest layers to be used for creating pins in the LEF. In the following example, only the shapes between M3 and M1 layers are ESD cell pin shapes:
	Marker file example:
	`ESD_diode 168;255:3 M3 M1`
	`ESD_clamp 122 M3 M1`

`-esd_cell_bbox_file` *filename*

Specifies a file that includes the bounding box for each ESD cell. The format of the file is:
`<cell_name> <x1 y1 x2 y2> [top_layer] [bottom_layer]`

Here,

- `cell_name` is the ESD cell/instance name
- `x1 y1 x2 y2` are the bounding box coordinates
- The optional `top_layer` and `bottom_layer` specifies the top and bottom technology layers of the ESD cell pin shape. You can use these optional arguments to specify the top and bottom technology layers of the ESD cell pin shape. This provides you with the flexibility to specify the lowest layers to be used for creating pins in the LEF. In the following example, only the shapes between AP and M11 layers are ESD cell pin shapes:

Bbox file example:

```
ESD_diode_0 1281296 860923 1296972 895445 AP M11
ESD_diode_1 1281296 898651 1296972 933173 AP M11
```

`-extract_signal_net_logic_info_only`

Specifies to print only the logical connectivity for the signal nets in the `NETS` section of the output DEF file. If this parameter is not specified, the software prints both the logical connectivity and routing shapes for the signal nets in the `SPECIALNETS` section of the DEF file.

`-gds_file` *filename*

Mandatory parameter. Specifies the name of the input GDS file. The top cell name in the GDSII file is the design name in the generated DEF.

`-gds_layermap` *filename*

Mandatory parameter. The layer mapping file between GDS and tech layers. All the layers defined in a given mapping file is extracted. The layers not defined in the mapping file are ignored.

The format of the layer mapping file is:

`<layer_type> <tech layer name> <gds> <gds layer#> [gds layer data type] [PORT] [TEXT_ONLY] <mask keyword> [dummy]`

The format of the layer mapping file to identify the die area in the output DEF is:

`<layer> <prBoundary> <gds> <gds layer#> [gds layer data type]`

Format Description:

`<layer_type>` specifies the type of library layer. The various layer types are: `via`, `metal`, and `layer`.

`<tech layer name>` specifies the name of the library layer to map to the GDSII layer number. The library layers are technology layers which can be dumped using the `Techgen -techinfo` command.

`<gds>` specifies the keyword for the GDS layer mapping.

`<gds layer#>` specifies the layer number of the GDSII layer.

`[gds layer data type]` specifies the optional number of the subclass for the GDSII layer. If you do not specify a datatype, the software uses all datatypes for that GDSII layer. If GDS datatype is not defined, the software considers it as a regular layout routing layer where all the shapes will be extracted.

The GDS layer may have additional information/attribute assigned using the following keywords:

- `PORT`: Converts text labels in this layer to ports.
- `TEXT_ONLY`: Includes only the text in the named layer and ignores all geometries.

`<mask keyword>` keyword used to define the back-end metal layers with color masks (MASK A and Mask B).

`[dummy]` indicates that the layer is a metal fill layer. The `convert_gds_to_def` utility identifies these metal fill shapes and writes them to the `FILLS` section in the output DEF.

`<prBoundary>` specifies that the shapes on the `prBoundary` (Place-and-Route Boundaries) layer are used to determine the die area in the output DEF. This keyword can be used if you want to have the required die area in the output DEF file. If the `prBoundary` keyword is not specified, the die area is determined by the routing shapes specified in the GDS file, cell size, and the pin shapes of all the layers.

Example1:

```
metal METAL1 gds 15
via VIA1 gds 16
```

Example2:

The following is a sample layermap file with the mask keywords (`mask_a` and `mask_b`):

```
metal METAL1 gds 31 250
metal METAL1 gds 31 255 mask_a
metal METAL1 gds 31 256 mask_b
```

Here, the mask and main layers are defined using the same GDS layer number 31, but uses a different GDS data type (250, 255, and 256).

Example3:

The following is a sample layermap file with the `dummy` keyword:

```
metal  METAL5 gds 35 367 dummy
```

Here, the layer with the GDS layer ID `35` and data type `367` contains the metal fill shapes only.

Example4:
The following is a sample layermap file with the `prBoundary` keyword:

```
layer   prBoundary      gds     62 21
```

`-ignore_gds_inst_name`

Specifies to ignore the instance names from the specified GDS file, and the software will then automatically assign instance names to all the instances in the output DEF and Verilog files. This parameter can be used if instance names in the GDS file cannot be used, primarily in cases, such as duplicate instance names.

`-inst_name_file filename`

	Specifies the file name containing instance names for the generated DEF in the following format: `<x coord> <y coord> <instance name>` The x y coordinates are in the unit of micron. The priority of getting an instance name is: 1. instance name file 2. GDS label 3. internally assigned name The `-inst_name_file` parameter allows to provide user-specified instance names. The `convert_gds_to_def` utility reads the GDS label to decide the instance name. If there are no labels, the software internally assigns instance names.
`-inst_name_prefix` *prefix*	
	Specifies a prefix for all the instance names in the output DEF and Verilog files.
`-net_location_file` *filename*	

	Mandatory parameter. Specifies the name of the net location file containing a list of pin locations for nets that needed to be considered. All the geometries of the P/G nets as well as the instance pins connected to the specified P/G nets are included in the `SPECIALNETS` section of DEF. The instances connected to the P/G nets are included in the `COMPONENTS` section of DEF.
	The format of the net location file is:
	<net name> <x loc of pin> <y loc of pin> <layer> <POWER|GROUND|SIGNAL> [*pin name*]
	Example:
	`VDD 60 73 METAL3 POWER`
	`VSS 75 40 METAL2 GROUND`
	Here,
	• The x/y location in the file has the unit of micron.
	• The layer name is the tech layer name.
	• The type of a given net can only be POWER, GROUND, or SIGNAL. If the net type is marked as `SIGNAL`, the net will be traced from the GDS file and included in the `SPECIALNETS` section with "`USE SIGNAL`" in the output DEF file. In addition, the signal pin connectivity is specified for the net.
	• The pin name field is optional. If there is no pin name field, the pin name will be same as the net name.
	If you do not provide pin locations, the utility uses the GDS text labels in either the top-level cell or cells throughout the hierarchy to get the location. In such a case, the format of net location file is:
	<net name> <POWER|GROUND|SIGNAL>
	Example:
	`VDD POWER`
	`VSS GROUND`

`-output_components {ALL | NONE | CONN}`	
	Specifies the components to be printed in the DEF file `COMPONENTS` section during GDS to DEF conversion. The default is `ALL`. • `ALL`: Specifies to output all the instances in the DEF Components section. • `NONE`: Specifies not to print instances. • `CONN`: Specifies to output instances with logical connectivity only.
`-output_def_file` *filename*	
	Mandatory parameter. Specifies the name of the output DEF file. If the given DEF name is specified with "`.gz`", the software will gzip the DEF file to reduce the file size.
`-output_lef_macros` *filename*	
	Specifies the name of the file containing a list of macros for which LEF files are to be generated. One LEF file is generated for each macro. The generated LEF pin shapes are all shapes that connect to top-level nets on all metal/via layer shapes. The cell boundary of the LEF macro is derived by the bounding box of all shapes within the GDS cell.
`-power_grid_library` *libraryname*	
	Mandatory parameter. Specifies the name of the merged power-grid library (.cl). You can specify only a single .cl library.

`-print_complete_short_path`

Specifies to print the complete shorting path when there is a short between two nets. When this parameter is specified, the entire shorting path is assigned to one of the shorted nets and the path is printed in the generated DEF file. The shapes of the shorting path will be assigned to the net that is specified first in the net location file. For example, if a tracing point of VDD is shorted with a tracing point of VSS, the software honors the net that is defined first in the net location file.

If this parameter is not specified, the software assigns the shapes of the path to either of the shorted nets by default.

`-spef_file` *filename*

Specifies the name of the SPEF file that can be used to include the signal nets in the generated DEF file. The input SPEF file is required to have layer annotation in the `RES` statements to indicate the pin node layers.

For the signal nets, both the `*P` and `*I` nodes are used as the starting tracing points, and the SPEF file pins `(*P)` are honored as the top-level pins and are reported in the `PINS` section in the output DEF file. For the PG nets, the `*P` nodes and the user-defined locations in the net location file are used as the starting tracing points. The SPEF file connections (`*I` statements) are honored when generating the DEF logic connectivity for both the signal and PG nets. That is, the `convert_gds_to_def` utility relies on the SPEF file for logic connection if a SPEF file is specified. The instance and pin names are from the SPEF `*I` statements. The output DEF file includes all the signal net pins from `*P` in the `PINS` section, the signal routing shapes in the `NETS` section, and the PG net routing shapes in the `SPECIALNETS` section.

In addition to the SPEF file, you need to specify the following parameters for including the signal nets in the DEF file:

- `-gds_file`
- `-net_location_file`
- `-power_grid_library`
- `-gds_layermap`
- `-output_def_file`
- `-top_cell`

Notes:

- The `-net_location_file` parameter is used to identify the power and ground nets, and therefore the remaining nets are considered as signal nets.
- The `-power_grid_library` specifies the name of the power-grid library that includes the list of standard cells or macros that should be consistent with the SPEF file. That is, the cell of the instances in the SPEF file should be defined in the PGV.

`-top_cell` *cellname*

Mandatory parameter. Specifies the top cell name in the GDSII file.

`-use_gds_label {ALL | TOP |` *<tech_layer_name>* `|` *<cell_name>*`:`*<tech_layer_name>* `|` *<cell_name>*`:ALL>}`

Specifies the criteria for selecting text labels.

- `ALL`: specifies to honor all the text labels from GDS.
- `TOP`: specifies to honor just the top-level text labels.
- *<tech_layer_name>*: specifies to honor all the text labels on the specified layer.
- *<cellname>*`:`*<tech_layer_name>*: specifies to honor all the text labels of the specified GDS cell on the specified layer.
- *<cellname>*`:ALL`: specifies to honor all the text labels of the specified GDS cell.

The following example specifies to use the text labels of the cell "hierblock" on layer METAL10:

```
convert_gds_to_def -use_gds_label hierblock:METAL10
```

`-verilog_file` *filename*

Specifies the name of the output verilog file. This parameter allows you to use the GUI to display the rail analysis results of the GDS-only flow.

`-zero_pin_size`

Specifies to set the pin shape to size zero "(0 0)(0 0)" in the output DEF file. You can use this parameter when the port shape is outside the drawn shape (metal geometry) at places where the port label is defined near the edge of the geometry.

Examples

- The following command converts the specified GDS file `new.gds` to a DEF file `top.def`.

  ```
  convert_gds_to_def -net_location_file net.loc -gds_file new.gds -power_grid_library ../pv_library.cl -gds_layermap lmap -top_cell cella
  -output_def_file top.def
  ```

- The following command converts the specified GDS file `chip.gds` to a DEF file `my.def`:

  ```
  convert_gds_to_def -gds_file chip.gds.gz -power_grid_library  {pgv/techonly/techonly.cl   pgv/full.cl} \
                     -net_location_file netlocfile -gds_layermap  layermap -output_def_file my.def -top_cell chip -spef_file chip.spef.gz
  ```

  Here, the SPEF file `chip.spef` is specified to include the signal nets in the output DEF file.

- The following command specifies a marker layer file `esdcells` to enable ESD analysis using `convert_gds_to_def`:

  ```
  convert_gds_to_def -gds_file gw16_500_usr_test.gds  -power_grid_library { /../TECH/techonly.cl }  -net_location_file my.nlf -
  gds_layermap  gds_layer.map \
      -output_def_file my.def  -top_cell gw16_500_usr -use_gds_label ALL -esd_marker_layer_file esdcells
  ```

  `convert_gds_to_def` utility generates a LEF file for each ESD cell specified in the `esdcells` file to enable PGV generation. The output DEF includes the user-defined nets and the ESD instances based on the specified marker layer file.

Related Information

- `set_rail_analysis_mode`
- "GDS2DEF Utility" in the *Voltus IC Power Integrity Solution User Guide*

create_current_region

```
create_current_region
[-help]
-reset | {-region x1 y1 x2 y2
[-current value | -current_pwl_file filename]
[-die_instance_name dieinstname]
[-file filename]
[-layer layername]
[-intrinsic_cap value]
[-loading_cap value]
[-series_resistance value]
[-net net_name]
[-polygon {x1 y1 x2 y2 x3 y3 ..x1 y1}]
[-pwrnet net_name]
[-gndnet net_name]
}
```

Specifies static or dynamic current regions on global power-grids over the placed macro to accurately capture high and low current consuming regions during rail analysis.

You can use this command prior to rail analysis to specify current regions or power distribution over the top of power-grid on a user-defined layer. These current regions are generally specified for a placed macro for which power-grid view library is not available, and power/current distribution inside this macro could possibly impact IRdrop in neighboring regions. These current regions can also be defined over top-level power-grid for an unplaced macro during early stages of power-planning.

Parameters

`-current value`	
	In the static mode (`set_rail_analysis_mode -method static`), the parameter specifies a single current value in mAmp. In the dynamic mode (`set_rail_analysis_mode -method dynamic`) the parameter specifies a PWL current waveform in time (in ns) and current (in mA) format. For example, 0ns 0mA 0.5ns 1mA 1ns 2mA 3ns 1mA 4ns 0.
`-current_pwl_file filename`	

	Specifies a PWL current file. The file has two columns, time (in ns) and current value (in mA), as shown below:
	`time1 value1`
	`time2 value2`
	The `-current_pwl_file` and `-current` parameters are mutually exclusive.

`-die_instance_name dieinstname`	
	Specifies the name of the die to which you need to apply a current region. This parameter is only applicable to the multi-die analysis flow (`set_rail_analysis_mode -die_mode multi-die`). The parameter allows you to accurately assign current to power and ground nets in specific geometric regions for individual dies during rail analysis in 3D-IC designs.
`-file filename`	

	Specifies the name of a file containing the list of current regions for each die. `-file` can be used only with the `-die_instance_name` parameter. The other parameters of the `create_current_region` command can be included within the specified file.
	Example:
	`create_current_region -file create_current_region_die0.tcl -die_instance_name Die0`
	`create_current_region -file create_current_region_die1.tcl -die_instance_name Die1`
	`###File content in create_current_region_die0.tcl`
	`-net VDD -current 50mA -layer M2 -region "10 10 100 100"`
	`-net VDD -current 50mA -layer M2 -region "100 100 200 200"`
	`-net VSS -current 100mA -layer M1 -region "10 10 200 200"`
	`###File content in create_current_region_die1.tcl`
	`-net VDD -current 150mA -layer M2 -region "10 10 50 50"`
	`-net VDD -current 50mA -layer M2 -region "100 100 200 200"`
	`-net VSS -current 200mA -layer M1 -region "10 10 200 200"`

`-help`	Outputs a brief description that includes the type and default information for each `create_current_region` parameter. For a detailed description of the command and all of its parameters, use the man command `man create_current_region`.
`-gndnet net_name`	
	Specifies a ground net name on which the specified current is to be applied.
`-intrinsic_cap value`	
	Specifies the cell intrinsic capacitance in pico farads (pf). This is required parameter when set in the dynamic mode (`set_rail_analysis_mode -method dynamic`). This parameter is disabled in static mode.
`-layer layername`	
	Specifies a layer name on which the scaling will be performed. Multiple commands can be issued to scale resistance on multiple layers. This is a required parameter.
`-loading_cap value`	
	Specifies the cell loading capacitance in pico farads (pf). This is required parameter when set in the dynamic mode (`set_rail_analysis_mode -method dynamic`). This parameter is disabled in static mode.
`-net net_name`	
	Specifies a net name on which the specified current is to be applied.
`-polygon {x1 y1 x2 y2 x3 y3 ..x1 y1}`	
	Specifies a rectilinear current region where the current will be distributed within. The rectilinear region enables you to specify multiple x,y points to add current regions in the areas that are not rectangular in shape. When specified, the last xy coordinate location will be the same as the first xy location. The xy coordinates will be in the counterclockwise sequence.
`-pwrnet net_name`	
	Specifies a power net name on which the specified current is to be applied.
`-region x1 y1 x2 y2`	
	Specifies the region on the layer across which the specified current and capacitance (in dynamic mode) is distributed by the rail analysis engine. The units of x and y should be in microns. This is a required parameter.
`-reset`	Resets all the previous `create_current_region` command settings.
`-series_resistance value`	
	Specifies the series resistance (ESR) value attached to each tap point in the current region definition.

Examples

- The following command specifies static current region:
  ```
  create_current_region -current 2mA -layer METAL_1 -region "100 100 200 200"
  ```
- The following command specifies dynamic current region:
  ```
  create_current_region -current {0ns 0mA 0.5ns 1mA 1ns 2mA 3ns 1mA 4ns 0} -layer METAL_1 -region "200 200 400 400" -intrinsic_cap 40 -loading_cap 20
  ```
- The following command resets all `create_current_region` parameters to their default values:
  ```
  create_current_region -reset
  ```
- The following command specifies rectilinear current regions for die instance `Die1`:
  ```
  create_current_region -net VDD -current 50mA -layer Metal5 -polygon "10 10 100 10 100 100 10 100 10 10 " -die_instance_name Die1
  create_current_region -net VDD -current 50mA -layer Metal5 -polygon "100 100 200 100 200 200 100 200 100 100" -die_instance_name Die1
  ```

Related Information

- `scale_what_if_capacitance`
- `scale_what_if_resistance`
- `scale_what_if_current`
- "What-If Rail Analysis" in the *Voltus IC Power Integrity Solution User Guide*

create_die_model

```
create_die_model
[-accuracy {xd | hd }]
[-output_directory directory_name]
-state_directory directory_name
[-rail_analysis_domain domain_name | -net net_name]
[-repeat time]
[-probing_node_file file_name]
[-use_sigrity_repeat]
[-max_ports_per_net integer]
[-enable_reference_node]
[-enable_package_pin_map { true|false }]
[-disable_isrc_node_zero { true|false }]
[-disable_global_node_zero { true|false }]
[-multi_die {true | false}]
[-port_grouping_by_ploc_files { true|false }]
[-port_grouping_file filename]
[-temp_directory_name directory]
[-effective_rc]
[-probing_region_file filename]
[-effective_rc_deck]
[-port_grouping_tiles_file  filename]
[-enable_xp {true | false}]
[-ignore_port_grouping_conflict]
[-generate_sipi_port_definition_file {true | false}]
[-grid_reduction_cut_layer layer_name]
```

Specifies to generate a reduced n-port die model in the form of equivalent die parasitic and current profile after rail analysis to perform system-level analysis with extracted package and board models. Resonance and electro-magnetic interference like system-level analyses can be performed using this die-model in package designing software like Allegro Package Design and Sigrity.

ⓘ This command requires a VTS-XL license.

Parameters

`-accuracy {xd | hd }`	
	Specifies the accuracy mode for die-model generation. *Default* : `hd` • `xd`: xd or accelerated definition accuracy mode is used for faster die model generation with slight reduction in RC accuracy. • `hd`: hd or high definition accuracy mode is used for most accurate die model generation.
`-disable_isrc_node_zero { true|false }`	

	Specifies to disable node zero for current sources (isrc) in a generated die-model. The isrc node zero is used to connect decoupling capacitance and the current source in the generated die-model. This parameter is added to generate a die model without isrc node zero. By default, a virtual ground is used to connect decoupling caps and the current source in the generated die-model. When set to `true`, generates die model without the virtual ground and uses the ground network for connecting decoupling caps and current sources. It also creates the <BLOCK>_nportModel.sp.isrc_balanced file in the die-model directory. This file has information on the current sources and their port connections. The .subckt file generated for Spice simulation points to the .isrc_balanced file. *Default*: `false`
`-disable_global_node_zero { true|false }`	
	Specifies to disable the global node zero in a generated die model. • When set to `true`, generates die models without the virtual ground and uses the ground network for connecting decoupling caps and current sources. • When set to `false`, a virtual ground is used to connect decoupling caps and the current source in the generated die-model. *Default*: `true`
`-enable_package_pin_map { true|false }`	

	Specifies to support the 6-column style MCP format to map package pins to circuit ports. The following are the old and new MCP file formats:
	
	The original 5-column format is:
	<vsrc_name> <Ckt_port> <Net_name> <X coord> <Y coord>
	
	The new 6-column format is:
	<package_pin> <Ckt_port> <Net_name> <X coord> <Y coord> <vsrc_name>
	
	Example:
	
	Original format:
	`chip1/bmp_100_6_12 D1_bmp_100_6_12 VDDE 350.000 147.000`
	`chip2/bmp_122_3_15 D2_bmp_122_3_15 VDD2 321.000 983.800`
	
	New format:
	`bmp_100_6_12 D1_bmp_100_6_12 VDDE 350.000 147.000 chip1/bmp_100_6_12`
	`bmp_122_3_15 D2_bmp_122_3_15 VDD2 321.000 983.800 chip2/bmp_122_3_15`

`-effective_rc`	Generates an additional report file (.txt) containing the effective resistance and capacitance information. The parameter is used for generation of a table for effective resistance and capacitance across multiple operating frequencies.
	The following is a sample report format:
	`# Net:VDD_AO, Net:VSS`
	`# frequency  # rdie  # cdie`
	`1.0000000e+07  4.73148443  2.47034747e-11`
	`1.2589254e+07  4.90209173  2.46916698e-11`
	`....`
	`....`
	`# Net:VDD_external, Net:VSS`
	`# frequency  # rdie  # cdie`
	`1.0000000e+07  -183.231085  6.65466673e-12`
	`1.2589254e+07  -164.007129  6.55112569e-12`
	`....`
	`....`

`-effective_rc_deck`	

	Specifies to include the Rdie/Cdie (effective resistance/capacitance) calculation for the die-model frequency domain simulation setup in the Spice (.sp) format.
	The following is a snippet of the .sp file with the frequency-domain setup for Rdie/Cdie calculation:
	`**********`
	`* Frequency domain analysis to capture input impedance.`
	`.AC DEC 10 10Meg 5G`
	`.PRINT AC RDIE=PAR('VR(VSS_in,VDD_AO_in)') CDIE=PAR('1/(6.283185*freq*(VI(VDD_AO_in)-VI(VSS_in)))')`
	`**********`

`-enable_reference_node`	
	Specifies to replace the global node zero with the reference port in the generated die model.
`-enable_xp {true | false}`	
	Specifies to generate the die model in the Extensively Parallel (XP) mode in which processing is distributed over a large number of machines. *Default* : `false`
`-generate_sipi_port_definition_file {true | false}`	
	Generates a ports definition file during die-model creation. When set to `true`, the port definitions are output to the `port_defintion.txt` file, which is later used by Sigrity SystemPI for connecting the die model to the package design. *Default* : `false`
`-grid_reduction_cut_layer` *layer_name*	
	Specifies a cut via layer below which all the layers will be reduced. This feature minimizes data and enables efficient die model generation.
`-ignore_port_grouping_conflict`	

	Specifies to not generate an error message in case there is a conflict between the user-defined port grouping and the default port groups (voltage sources that are grouped by mapping to the same package terminal). For example, vsrcA and vsrcB are both mapped to the package terminal PKG_T, but in the user-specified grouping file, vsrcA and vsrcB belong to two groups. When this parameter is specified, the tool will not generate an error message if there is a conflict, but instead will group the voltage sources according to the user-specified port grouping file. If this parameter is not specified and there is a conflict in port grouping, an error will be generated.
`-max_ports_per_net` *integer*	
	Specifies the maximum number of ports allowed for each net in a generated die model. If you do not specify this parameter, the software by default enables port grouping with number of ports set to `200`. If the number specified through this parameter is `0`, port grouping will be disabled. The maximum number of ports per net should be `0` or a positive value. A negative value will be treated as 0. **Note:** The `-max_ports_per_net` parameter is only meaningful for N-port die model creation. The 2-port die model creation always groups all ports together.
`-multi_die {true | false}`	
	Specifies to generate a complete die model for a TSV/SiP design. *Default*: `false`
`-output_directory` *directory_name*	
	Specifies the directory in which you save the die-model files.
`-port_grouping_by_ploc_files {true | false}`	
	Specifies to read the PLOC group information from the user-defined PLOC files. You can specify the port information in the PLOC file, as shown below: `Vsource_5_VDD 21985.00 23785.00 AP VDD p18` # with net and group `Vsource_7_VDD 22385.00 23785.00 AP VDD p18 C=1e-09 L=1e-11` # with capacitance and inductance When `-port_grouping_by_ploc_files` is set to `true`, the die model flow will read the user-specified port grouping information and generate a reduced die model. This parameter is mutually exclusive with the `-port_grouping_file` parameter.
`-port_grouping_file` *filename*	

	Specifies a text file containing the user-defined pin grouping details on a per power/ground net basis. When this parameter is specified, a reduced die model is generated that honors the user-specified port grouping. This die model has a reduced RC netlist and `.isrc` when compared with the original un-grouped die model. The degree of reduction will depend on how many pins are being grouped.
	The format of the pin grouping file is as follows:
	`NET <netname>`
	`<grouped pin name> <vsrc names which have to be grouped>`
	`<grouped pin name> <vsrc names which have to be grouped>`
	`NET <netname>`
	`<grouped pin name> <vsrc names to be grouped>`
	`<grouped pin name> <vsrc names to be grouped>`
	`END`
	where,
	• `NET` is a keyword followed by the power or ground net name
	• *`grouped pin name`* is the user defined port name
	• *`vsrc names`* are the actual design voltage sources that need to be grouped

`-port_grouping_tiles_file` *`filename`*	

	Specifies to group power ground ports by a user-defined tile array. All the ports in a specific tile are grouped together.
	
	The use model of the parameter is:
	`create_die_model -port_grouping_tiles_file tiles.txt`
	
	The following is the format of the port grouping tiles format is:
	`NET <net>`
	`BBOX <x1 y1 x2 y2>`
	`ROWS <num rows>`
	`COLUMNS <num columns>`
	
	Here,
	• the keywords are highlighted in blue.
	• *<net>* is the power supply net name
	• *<x1 y1 x2 y2>* are the coordinates of the bounding box lower and upper corners. This is optional. If the bounding box is not specified, the default is the chip bounding box.
	• *<num rows>* is the number of rows in the tile array
	• *<num columns>* is the number of columns in the tile array
	This parameter allows you to partition the voltage sources of the die model in to several port groups. If the same grouping pattern is used for the voltage sources of the die model and the package pins, it will help in matching the die ports and package terminals, and connecting the die model to the package netlist.
	
	The grouping information is available in the MCP header of the generated die model file, as shown below:
	`* [REM]`
	`* [Power Nets]`
	`* vsrc_821 VDD2_GROUP_BUMP_tile01 VDD2 6316.25 20`
	`* vsrc_820 VDD2_GROUP_BUMP_tile01 VDD2 623.75 20`
	`* vsrc_822 VDD2_GROUP_BUMP_tile00 VDD2 5000 40`
	`* vsrc_823 VDD2_GROUP_BUMP_tile00 VDD2 5002 40`
	……
	`*[Ground Net]`
	`* vsrc_100 VSS_GROUP_BUMP_tile01 VSS 6300 25`
	`* vsrc_101 VSS_GROUP_BUMP_tile01 VSS 6250 25`
	`* vsrc_102 VSS_GROUP_BUMP_tile00 VSS 4000 40`
	`* vsrc_103 VSS_GROUP_BUMP_tile00 VSS 4002 40`
	Here
	• the first column is the voltage source name.
	• the second column is the port name under which the voltage sources are grouped. It also displays the tile number to check a specific tile's power and group port during simulation.
	• the third column is the net name.
	• the fourth and fifth columns indicate the X and Y coordinates, respectively. The unit of x/y coordinate location is micrometer (µm).

`-probing_node_file` *file_name*	

	Specifies the probe node file containing the probe point on a die. The file format for specifying probing nodes is as follows:
	`<user defined unique probe name> <location = x> <location = y> <layer> <Power/ground net name>`
	You can also specify the package node voltages and resistor currents that are to be probed in the following format:
	`<probe name> pkg ir <pkg circuit node name>`
	`<probe name> pkg rc <pkg circuit resistor name>`
	Example
	`probe_1 18 16 METAL_1 VDD`
	`# pkg node voltage`
	`probe_pkg_1 pkg ir GND.in0`
	`probe_pkg_6 pkg ir VDD.in2`
	`# pkg resistor current`
	`probe_pkg_cur_1 pkg rc R_1_1`
	`probe_pkg_cur_2 pkg rc R_5_2`
	Note: The units of x and y should be in microns.

`-probing_region_file` *filename*	
	Specifies a probe node region file to define a probe region and use all points in the region as probe contact points on the die. The file format for specifying a probing region is as follows: *region_label x1 y1 x2 y2 layer_name net_name*
`-rail_analysis_domain` *domain_name* | `-net` *net_name*	
	Specifies the domain name or net name for which the die-model is to be generated. When the domain name is specified, coupled die-model is generated and it includes all power and ground nets associated with the specified domain. When net name is specified, decoupled die-model is generated for the specified net. Before specifying this command parameter, the power net name must be specified using the `set_pg_nets` command. This is a required option and one of them must be specified.
`-repeat` *time*	Specifies the repeat time in ns. When this parameter is specified without the `-use_sigrity_repeat` parameter, the generated die model has the dynamic current profile of the die repeated to the specified time. By default, the die model consists of the default simulation period saved in dynamic current files during IR drop anlaysis. When this parameter is specified along with the `-use_sigrity_repeat` parameter, the software forces to write out repeating waveforms in the Sigrity specific format. If the `-use_sigrity_repeat` parameter is specified, the value specified through the `-repeat` parameter indicates the starting point of the repeating waveform.
`-state_directory` *directory_name*	

	Specifies the top-level state directory that is generated by the static or dynamic IR drop analysis. This state directory contains analysis of all nets associated with the power domain; For example, `core_25C_dynamic_1`. This is a required parameter.
`-temp_directory_name` *directory*	
	Specifies a directory to save temporary files for die-model generation.
`-use_sigrity_repeat`	When this parameter is specified along with the `-repeat` parameter of the `create_die_model` command, the software forces to write out repeating waveforms in the Sigrity specific format. If the `-use_sigrity_repeat` parameter is specified, the value specified through the `-repeat` parameter indicates the starting point of the repeating waveform. If `-use_sigrity_repeat` is specified without specifying `-repeat`, time `0` is used for the starting point. The default unit for the value is "ns". **Note:** When the `-use_sigrity_repeat` parameter is specified, the software will write the stimulus current in the specific Sigrity format, which cannot be used by other Spice like simulators.

Examples

- The following command creates an `n-port` die model having the dynamic current profile of the die repeated upto `50ns`, using the state directory `allDomains_25C_dynamic_1` and saves the generated n-port die-model file in the `n_port_diemodel` output directory:

```
create_die_model
-state_directory irdrop_allOn/allDomains_25C_dynamic_1
-rail_analysis_domain allDomains -output_directory n_port_diemodel -repeat 50
```

- The following example shows the use model of the `-effective_rc` parameter to create a report file containing the effective resistance and capacitance information:

```
create_die_model
  -effective_rc -state_directory staticRailResults/ALL_125C_avg_1/
  -output_directory static_die_model -rail_analysis_domain ALL
```

Related Information

- `set_package`
- "Package Analysis" in the *Voltus IC Power Integrity Solution User Guide*

create_hier_view

```
create_hier_view
[-help]
-library_name name
[-output_dir dir]
[-block_lef lef_file |-block_gds gds_file]
[-block_lef lef_file | -gds_port_file gds_port_file]
[-libgen_command_file file]
[-max_viacluster_mode {true | false}]
[-extraction_mode {fast | accurate}]
[-cell_name name]
[-def list_of_DEF_files]
[-oaRef OA_reference_libs]
[-powergrid_view_libraries primitive_cells.cl]
[-ground_nets {pin1 pin2 ...pinN}]
[-power_nets {pin1 value1 pin2 value2 ...pinN valueN}]
[-extraction_work_directory path_to_extraction_work_directory]
[-compress_powergrid_database {true | false}]
[-parasitic_extractor_command_file filename]
[-cluster_via1_ports {true | false}]
[-cluster_via_rule { {via_layer1 number_of_equidistant_vias}...}]
[-cluster_via_size value]
[-ignore_fillers {true | false}]
[-ignore_decaps {true | false}]
[-ignore_shorts {true | false}]
[-force_library_merging {true | false}]
```

Creates a hierarchical power-grid view. The hierarchical PGV generation of a block is part of the top-level Hierarchical Rail Analysis.

Parameters

`-block_lef lef_file`	
	Specifies the name of the block LEF file. The `lef_file` must have all power pin port geometries defined for the hierarchical block.
`-block_gds gds_file`	
	In absence of block LEF, you can specify block GDS and port label file to generate hierarchical power-grid view.
`-cell_name name`	Specifies the block cell name as specified in the block DEF file.
`-cluster_via1_ports {true | false}`	
	Designs with follow-pin routing on M1 and M2 layers have standard cells library LEF with M1, M2, and VIA1 ports. This parameter specifies to cluster VIA1 ports for such cells in order to improve overall extraction and rail analysis performance. *Default*: `true` This parameter is set to `true` by default to improve VIA1 port handling in designs with M1 and M2 follow-pin routing. This parameter gives significantly better performance by clustering equidistant VIA1 ports defined in cell LEF without significant loss of accuracy. However, if you specify a clustering rule, the user specification takes precedence over automated VIA1 clustering.
`-cluster_via_rule { {via_layer1 number_of_equidistant_vias}...}`	
	Controls the number of vias to cluster on a layer basis. The VIA clustering rule specified using this parameter will override the default clustering rule for a given accuracy mode. The following command will cluster 100 equidistant VIA1 cuts, 200 equidistant VIA2 cuts, and 300 equidistant VIA7 cuts: `create_hier_view -cluster_via_rule { {VIA1 100}} {VIA2 200} {VIA7 300}}` The rest of the VIAs will be clustered using the default clustering rule depending upon the rail analysis accuracy mode.
`-cluster_via_size value`	

	Specifies the size of the via array to cluster all via layers except VIA1 (layer between Metal1 and Metal2). The default value is `625` (25x25 via array), except when running static analysis using HD mode. The default for static HD mode is `16` (4x4), primarily to preserve more accuracy for EM analysis. *Default*: `625`
`-compress_powergrid_database {true | false}`	
	Specifies to compress the output files generated by extraction and rail analysis to save disk space usage. This parameter reduces disk space requirement from the extraction and rail analysis stages at approximately 5-10% performance penalty. *Default*: `false`
`-def` *list_of_DEF_files*	
	Specifies a list of block DEF files for hierarchical analysis.
`-extraction_mode {fast | accurate}`	
	Determines what power-grid views will be created, dependent on whether fast or accurate extraction is to be done. By default, hierarchical PGV generates both detailed and reduced views.
`-extraction_work_directory` *path_to_extraction_work_directory*	
	Specifies to re-use the extracted power-grid database to generate HPGV. This parameter is only supported to generate hierarchical extraction view of a block in the design. The specified extraction work directory must contain the extracted data for all the nets for which the hierarchical extraction view needs to be generated. This parameter is mutually exclusive with the `-state_directories` and `-def` parameters of the `create_hier_view` command.
`-force_library_merging {true | false}`	
	Specifies to force merge PGVs with different resistivity and layers. It uses the technology information of the first library. If multiple tech-only PGVs are specified, it will take the first tech-only PGV definition and the first cell definition. If you are merging a tech-only PGV with other PGVs, it will keep the tech-only technology definition and the first cell definition.
`-gds_port_file` *gds_port_file*	
	In absence of block LEF, you can specify block GDS and port label file to generate hierarchical power-grid view.
`-ground_nets {`*pin1pin2...pinN*`}`	
	Specifies a list of ground nets of the block.
`-help`	Outputs a brief description that includes the type and default information for each `create_hier_view` parameter. For a detailed description of the command and all of its parameters, use the man command `man create_hier_view`.
`-ignore_decaps {true | false}`	
	Ignores decap cells during rail analysis. It ignores cells that are tagged as DECAP cells during library characterization or set as decap cells using the `set_rail_analysis_mode -decap_cell_list` command parameter. If decaps are used to preserve connectivity on the follow-pin routing, you must set this parameter to `false`. Ignoring decaps during dynamic analysis is not recommended. *Default*: `true` for static, `false` for dynamic
`-ignore_fillers {true | false}`	
	Ignores filler cells during rail analysis if the list of filler cells are defined in PGV generation, or specified as fillers using the `set_rail_analysis_mode -filler_cell_list` command parameter. During rail analysis, ignore library cells that are tagged as FILLER cells and have no capacitance associated with the interface nodes. If fillers are used to preserve connectivity on the follow-pin routing, you must set this parameter to `false`. *Default*: `true`
`-ignore_shorts {true | false}`	

	Ignores shorts that are found during extraction and continues hierarchical PGV generation. However, `-ignore_shorts` cannot be applied if the design contains GDS. *Default*: `false`
`-libgen_command_file` *file*	
	Specifies the file name of a power-grid library generation (libgen) command file. When specified, it will be included in the auto-generated `libgen.cmd` file at the end as `include <file>`. This parameter allows you to provide a file that includes specific library generation options.
`-library_name` *name*	
	Specifies the name of the library that the block power-grid view will be placed in. A `.cl` extension will automatically be added to the name.
`-max_viacluster_mode {true | false}`	
	Specifies the extractor to cluster vias automatically to reduce the RC elements by clustering vias into one node, therefore reducing the IR drop analysis runtime in dynamic mode. It is based on the notion that current evenly distributes through all vias of an array via. *Default*: `true`
`-oaRef` *OA_Reference_libs*	
	Reads OpenAccess reference libraries.
`-output_dir` *dir*	Specifies the output directory that the results will be save in.
`-parasitic_extractor_command_file` *filename*	
	Specifies the name of the file input to the extractor invoked by the Library Generation engine. This file contains information about the cells in the design and is used for resistance and capacitance extraction. For variables and functions that can be placed in this command file, see *Extractor Options in Voltus Text Command Reference*. In general, this parameter is used for debugging purposes.
`-powergrid_view_libraries` *primitive_cells.cl*	
	Specifies a list of primitive cell libraries for hierarchical PGV generation.
`-power_nets {`*pin1value1pin2value2 ...pinN valueN*`}`	
	Specifies a list of power nets and the nominal voltage of those nets.

Examples

- The following command generates a hierarchical power-grid view for the hierarchical block using LEF and extraction directories for VDD and VSS nets:

  ```
  create_hier_view -block_lef blockA.lef -extraction_work_directory work -library_name blockA  -powergrid_view_libraries
  ./pgv/stdcells.cl  -power_nets {vdd 1.2} -ground_nets {gnd} -cell_name blockA
  ```

  The design.lef must have all power pin port geometries defined for the hierarchical block. The generated hierarchical PGV library is blockA.cl. The library is in binary format and it stores the block's connectivity to the top-level based on the provided LEF data. It stores the extracted power-grid network and current distribution from the provided directories.

- The following command specifies to force merge PGVs with different resistivity and layers:

  ```
  create_hier_view  \
    -force_library_merging true \
    -library_name lib1  \
    -view hier_extraction_view \
    -powergrid_view_libraries {../accurate_stdcells_A.cl ../accurate_stdcells_A.cl ... ../accurate_stdcells_AN.cl } \
    -def {../design.def.gz} \
    -cell_name  CELLA \
    -power_nets {VDD 1.32} \
    -ground_nets {VSS} \
    -block_lef {./design.lef}  \
    -extraction_mode accurate \
  generate_pg_library -output pgv
  ```

Related Information

- set_rail_analysis_mode
- set_dynamic_rail_simulation
- analyze_rail
- "Hierarchical Rail Analysis" in the *Voltus IC Power Integrity Solution User Guide*

create_power_pads

```
create_power_pads
[-append_to_vsrc_file]
[-auto_fetch]
[-cell {cell_list} ]
[-cell_pin {{cellname1 pinname1}...{cellnamen pinnamen}+} ]
[-clear]
[-display]
[-format {padcell xy tsv}]
[-honor_pin_connection]
[-instance {inst_list} ]
[-instance_pin {{instname1 pinname1}...{instnamen pinnamen}+} ]
[-layer {layername}]
[-net netname]
[-package_resistance value ]
[-package_capacitance value ]
[-package_inductance value ]
[-region {x1 y1 x2 y2}]
[-region_pitch {xpitch ypitch}]
[-snap_distance { true|false }]
[-vsrc_file vsrc_filename]
[[-add | -delete] -loc {x y}]
[-list]
```

Returns a list of DC sources from your design. DC sources are identified in the LEF file as CLASS PAD POWER, or in the DEF file as PIN with +USE POWER/GROUND. If a single port shape is connected to N number of wires, N number of DC sources are reported for that port shape.

You can use one of the following three methods to fetch voltage sources into the database:

- fetch all voltage sources
- fetch the voltage sources within the specified region of the specified layer controlled by the specified xPitch/yPitch
  If the voltage sources with the specified xPitch/yPitch are not on the stripe, you can use the `-snap_distance` parameter to snap the voltage sources to the stripe. As a result, the voltage sources are generated only on the stripes and the xPitch/yPitch is honored. Use this feature to generate mesh type voltage sources on any process layer.
- fetch voltage source for the specified cell/cell pin/instance/instance pin

You can edit the fetched voltage sources before saving them to a voltage source file. To save the fetched list to a file, use the `-vsrc_file` parameter. By default, voltage sources will not be displayed on GUI after they are saved to file.

> The `create_power_pads` command supports hierarchical IR drop analysis for a TSV design. When you specify the `create_power_pads` command to fetch the voltage sources from the LEF file, the command not only searches for the P/G pin of the pad connected to the specified P/G net, but also fetches for the embedded bump TSV property definition for the P/G pin connected to the P/G net. The port that contains the point defined in the embedded bump property is considered as the embedded bump.

Parameters

`-add`	Adds the location of a new voltage source to memory.
`-append_to_vsrc_file`	Appends the fetched voltage sources to the existing voltage source file. You can use this parameter to fetch voltage sources in the incremental mode.
`-auto_fetch`	Specifies to fetch all voltage sources.
`-cell {cell_list}`	
	Specifies the cells from which voltage sources are determined.
`-cell_pin {{cellname1pinname1}...{cellnamenpinnamen}+}`	

	Specifies the cells and their pins from which voltage sources are determined.
`-clear`	Clears the display of the DC sources that were specified in the *Edit Pad Location* form.
`-delete`	Deletes the location of a voltage source from memory.
`-display`	Loads and displays the pad location data from the specified file.
`-format {padcell | xy | tsv}`	
	The format of the saved pad location file. The choices are Pad File (VoltageStorm), XY File (xy coordinates), and TSV. You can create and save the voltage sources location file with a unique name for the voltage sources (both VDD and VSS). *Default*: `xy`
`-help`	Outputs the command usage and a brief description about the command parameters.
`-honor_pin_connection`	
	Honors pin wire connection when fetching voltage source location from a power pad. When you specify this parameter, only pins with wire connection are output as voltage sources. You have the choice to specify whether real wire connection is to be honored or not. Typically, during the sign-off stage, power pads are already fully routed so the unconnected power pads should not be counted in as voltage sources. You can specify this using the -honor_pin_connection parameter. Alternatively, in the early stage, power pads may not have been fully routed so you do not need to set this parameter. **Note:** For a port with shapes at multiple layers, all layer connected shapes are fetched as voltage sources.
`-instance {inst_list}`	
	Specifies the instances from which voltage sources are determined.
`-instance_pin {{instnamelpinname1}...{instnamenpinnamen}+}`	
	Specifies the instances and their pins from which voltage sources are determined.
`-layer` *layerName*	
	Specifies the process layer in LEF on which the voltage sources are to be generated. When the `-layer` parameter is specified in conjunction with the `-region {x1 y1 x2 y2}` and `-region_pitch {xpitch ypitch}` parameters, the layer set here is the layer for which voltage sources are to be fetched. If the `-layer` parameter is not used together with the `-region {x1 y1 x2 y2}` and `-region_pitch {xpitch ypitch}` parameters, the layer set here is for the lowest layer to snap (the default value is `M5`).
`-list`	Loads and displays all voltage sources from memory.
`-loc {x y}`	Specifies the location at which the specified voltage source is to be added or deleted.
`-net` *netname*	Specifies the name of a power net for which to list DC sources. *Default:* If you do not specify a net name, the power analyzer automatically lists DC sources for all nets.
`-package_resistance` *value*	
	Specifies the resistance value of the package resistance-inductance-capacitance (RLC) model in Ohm.
`-package_capacitance` *value*	
	Specifies the capacitance value of the package resistance-inductance-capacitance (RLC) model in Farad (F).
`-package_inductance` *value*	
	Specifies the inductance value of the package resistance-inductance-capacitance (RLC) model in Henry (H).
`-region {x1 y1 x2 y2}`	
	Generates the voltage sources in the specified region.
`-region_pitch {xpitch ypitch}`	
	Specifies the region pitch in the x and y direction for the voltage sources.
`-snap_distance` *value*	

	Specifies the snapping distance constraint from IO/pad pins. If this parameter is not specified with a value, no snapping will be done. *Default:* 0 Specifies whether to snap DC sources from I/O pad pins to some location on the power structure. When set to true, the software performs autofetch with snapping. For I/O pad pins that are connected to core ring, the software selects the highest layer and center of the pin as the DC source location. When I/O pad pins are not connected to core ring, the software snaps the location of the I/O pad to somewhere on the core ring. The DC source location should be within the distance of snapping distance constraint from I/O pad pins, and the layer should not be lower than the lowest layer snap constraint.
`-vsrc_file` *vsrc_filename*	
	Specifies the name of the voltage source file that is required to save the auto fetched result.

Examples

- The following command fetches all voltage sources of the net `VDD`:
  ```
  create_power_pads -net VDD -auto_fetch
  ```
- The following command loads and displays all voltage sources from memory:
  ```
  create_power_pads -list
  ```
- The following command specifies the location at which the specified voltage source is to be added:
  ```
  create_power_pads -add -net VDD -layer METTOP -loc {500 500}
  ```
- The following command specifies the location from which the specified voltage source is to be deleted:
  ```
  create_power_pads -delete -loc {500 500} -layer METTOP -net VDD
  ```
- The following command fetches the voltage sources within the specified region of the layer `METAL6` controlled by the specified xpitch/ypitch, and saves the sources to the file `report2` in the xy format:
  ```
  create_power_pads -net VDD -region 100 100 3000 3000 -region_pitch 555 444 -layer METAL6 -format xy -snap_distance 100 -vsrc_file report2.pp
  ```
  > ⚠ In the first example, the output was not saved but it remains in the memory till you specify the second example. Before loading and saving the output of the second example, the first example output in the memory is saved to an auto generated dc source file incrementally, like pp.src, pp1.src, pp2.src, and so on.
- The following command fetches voltage source for the cell BUMPCELL in the xy format:
  ```
  create_power_pads -net VDD -cell BUMPCELL -format xy
  ```
- The following command clears the voltage sources displayed in the GUI and the fetched voltage sources in the database:
  ```
  create_power_pads -clear
  ```
- The following command displays the fetched and saved voltage sources for the net `VDD` in the file `report2` :
  ```
  create_power_pads -net VDD -auto_fetch -vsrc_file report2.pp -display
  ```
- The following command displays voltage sources in the GUI by loading the saved voltage source file `report1`:
  ```
  create_power_pads -vsrc_file report1.pp -display
  ```
- The following command saves the fetched voltage sources to the voltage source file `report1.pp`:
  ```
  create_power_pads -vsrc_file report1.pp
  ```

Related Information

- `set_rail_analysis_mode`
- `analyze_rail`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

create_what_if_shape

```
create_what_if_shape
[-help]
[-area {llx lly urx ury}]
[-display]
[-direction {hor | ver}]
[-offset {X Y}]
[-add -method {auto | exact}]
[-layer layerName]
[-nets {net1 net2 net3…}]
[-spacing value]
[-objects {selected | all} ]
[-add  | -select | -deselect |
 -delete | -list  | -save filename | -load filename]
[-pitch pitch]
[-add -width width]
[-type {wire | via} ]
[-format {txt def}]
[-save filename -append ]
[-remove_existing_wires]
```

Specifies to add, select, deselect, delete, list, and display the what if wires/vias to a design.

Parameters

`-help`	
	Outputs the command usage.
`-add`	
	Specifies to add what-if wires/vias.
`-append`	
	Specifies to append all what-if wires/vias to the existing file specified with the `-save filename` parameter.
`-area {llx lly urx ury}`	
	Specifies the rectangular region in which the what if wires/vias are to be generated.
`-delete`	
	Specifies to delete all or selected what-if shapes by constraint `-net/-layer/-type`. You can use this parameter in conjunction with the `-objects` parameter.
`-deselect`	
	Specifies to deselect all what-if shapes by constraints `-net/-layer/-type`. Use this parameter in conjunction with `-area` to deselect what-if shapes overlapping with the specified area.
`-direction {hor | ver}`	
	Controls the direction of the wires added.
`-display`	
	Invokes the GUI to show what-if shapes and zooms to the selected shapes. It works only in the `-win` mode.
`-format {txt def}`	
	Specifies the output file format. The `def` format specifies to create incremental DEF to load into GUI using `read_def -top`. The default output format is `txt`.
`-layer layerName`	
	Specifies the LEF layer name for wires, or layer pair for via to be added.

`-list`

Specifies to list all or selected what-if shapes by constraint `-net/-layer/-type` . You can use this parameter in conjunction with the `-objects` parameter.

`-load` *filename*

Specifies to load the saved what-if shape file.

`-method {auto | exact}`

Specifies the method used to add vias:

- `auto`: When area and layer pair are specified, the tool automatically adds vias on all intersecting points on the specified layer pairs in the given area.
- `exact`: When an area and layer pair are specified, the tool adds a via of exact size of the specified area between the specified layer pair.

`-nets` *{net1 net2 net3…}*

Specifies the name of the nets for which what-if wires/vias is to be created. The what-if engine will create a shape based on the net pattern.

`-objects {selected | all}`

Specify `-objects` in conjunction with the `-delete` or `-list` parameters to delete or list the selected what-if shapes. You can also delete or list all what-if shapes. The default value is `all`.

For example, you can use the following command to delete the selected objects:

```
create_what_if_shape -delete -objects selected
```

`-offset {X Y}`

Specifies to create metal/via from the area + offset. This parameter is useful when you run multiple what-if shape creation commands and want to keep the same area but have different offset.

`-pitch` *pitch*

Specifies the distance between the centre line of two adjacent what-if wires (pitch) to be drawn in the specified `-area` in the preferred direction of the specified `-layer`. The direction of created wires can also be controlled using the `-direction` parameter.

If this parameter is not provided, a single rectangular shape of size {llx,lly,urx,ury} is created.

`-remove_existing_wires`

Specifies to remove a wire and connected vias from the specified layer and area before adding what-if shapes.

`-save` *filename*

Specifies to save all the in-memory what-if wires/vias to the specified file. The format of the file is:

```
type wire net netName area llx lly urx ury layer layer1
type via net netName area llx lly urx ury layer layer1 layer2
```

Following is an example of the file containing one wire and one via shape:

```
type wire net VDD area 191.000000 763.000000 207.000000 772.000000 layer METTOP
type via net VDD area 513.000000 497.700000 517.000000 499.100000 layer METG1 ME
```

`-select`

Specifies to select all what-if shapes by constrains `-net/-layer/-type`. Use this parameter in conjunction with `-area` to select what-if shapes overlapping with the specified area.

`-spacing` *value*

Specify the spacing between each net. Use this parameter to control the spacing between each net. The `-pitch` value specified should be greater than number of nets * (width+spacing).

`-type {wire | via}`

Specifies what kind of shape is to be created.

`-width` *value1 value2 ..valuen*

Specifies the width of what-if wires. You can use the `-width` parameter to specify a separate width value for each net.

Examples

- The following command adds a what-if wire for the net `VDD` and on the LEF layer `METTOP`:

```
create_what_if_shape -type wire -net VDD -area {391.000000 663.000000 407.000000 672.000000} -layer METTOP -add
create_what_if_shape -type wire -net VDD -area {191.000000 663.000000 207.000000 772.000000} -layer METTOP -add -pitch 9 -width 0.4
```

- The following command creates a what-if via for the net `VDD` and on the LEF layers `METG1 MET5`:

```
create_what_if_shape -type via -net VDD -area {513.000000 497.700000 517.000000 499.100000} -layer {METG1 MET5} -add -method exact
create_what_if_shape -type via -area {0 0 1000 1000} -method auto -layer {METTOP METG1} -add -net VDD
```

- The following command specifies to list selected what-if shapes:

```
create_what_if_shape -list -objects selected
```

- The following command specifies to select and highlight all wires in the win mode:

```
create_what_if_shape -select
```

- The following command specifies to select and highlight all what-if vias using area:

```
create_what_if_shape -type via -area {500 490 530 500} -select -net VDD
```

- The following command specifies to save what-if shapes into the file `whatif.shapes`:

```
create_what_if_shape -save whatif.shapes
```

- The following command specifies to delete all what-if shapes in memory:

```
create_what_if_shape -delete -objects all
```

- In the following example, for nets "`VDD VDD1 VDD2 VSS`", three spacing is possible (between VDD and VDD1, between VDD1 and VDD2, and between VDD2 and VSS), that is, spacing is "0.5 0.6 0.7" and width of what-if wires for all nets is "0.5":

```
create_what_if_shape -net {VDD VDD1 VDD2 VSS} -pitch 4 -width 0.5 -spacing {0.5 0.6 0.7 -area {0 0 12 12} -type wire -add -layer M10 -direction hor
```

- The following command will remove all wires on `M7` in the `0,0` to `100,100` region as well as any vias connected to the deleted wires:

```
create_what_if_shape -remove_existing_wires -layer {M7} -area { 0 0 100 100 }
```

Related Information

- `scale_what_if_capacitance`
- `create_current_region`
- `scale_what_if_current`
- `scale_what_if_resistance`
- "What-If Rail Analysis" in the *Voltus IC Power Integrity Solution User Guide*

debug_irdrop

```
debug_irdrop
[-help]
[-nworst_instances value]
[-output_directory dir]
-state_directory dir
[-net netname | -domain domainname]
[-region {x1 y1 x2 y2 }| -nregion value]
[-eco_report]
[-net_threshold value | -domain_threshold value]
[-summary_only]
[-tile_row_col {X Y}]
[-tile_size {X Y}]
[-non_violating_instslimit value]
[-enable_xp {true | false}]
```

Specifies to automatically identify the region with worst IRdrops and EM violations, and provide necessary information to determine and fix the root cause. The reports are consolidated for each region to find root-cause. Root cause analysis is done by identifying top 10 instances in each hotspot and report the following:

- Hotspot Region Coordinate
- Timing Library Analysis
- Power-Grid View Analysis
- Power-Grid Integrity Analysis
- IR-Drop Composition
- Root Cause Analysis

You must specify this command after running the `analyze_rail` command.

⚠ The `debug_irdrop` command requires a VTS-XL license.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `debug_irdrop` parameter. For a detailed description of the command and all of its parameters, use the man command man `debug_irdrop`.
`-domain domainname`	
	Specifies the name of the power domain.
`-domain_threshold value`	
	Specifies the minimum allowed voltage for the domain being analyzed. The value will be in percentage. This parameter behavior is same as the `set_rail_analysis_domain -threshold` parameter.
`-eco_report`	
	Specifies to generate a file with IR drop information that can be used by Tempus ECO to fix IR voltage drop for all instances in a design. The report (`eco.ird`) is generated in the specified output directory (`debug_irdrop -output_directory`).
`-enable_xp {true | false}`	
	Specifies to identify the worst IR drop and EM violations regions in the Extensively Parallel (XP) mode wherein processing is distributed over a large number of machines. *Default* : `false`
`-net netname`	
	Specifies the name of the power or ground net that you want to analyze.

`-net_threshold` *value*	
	Specifies the minimum allowed voltage for the power net being analyzed. The value must be an absolute number. This parameter behavior is same as the `set_pg_nets -threshold` parameter.
`-non_violating_instslimit` *value*	
	Reports non-violating instances within hotspots (violating tiles) that are close to the threshold value. This parameter allows you to define a limit (sub-threshold) that can be used to report the non-violating instances that are close to the threshold. For example, if the `-net_threshold` or `-domain_threshold` is `60mV`, and `-non_violating_instslimit` is `3mV` close to the threshold, the `debug_irdrop` command will report all the violating instances above the domain/net threshold and also the non-violating instances which are in the range of 60mV-57mV. This information about the non-violating instances can be used for IR improvement in the Timing-aware IR Drop Fixing flow.
`-nregion` *value*	
	Specifies the number of regions to be analyzed. The default value is `3`.
`-nworst_instances` *value*	
	Specifies the number of worst instances to be reported. The default value is 5.
`-output_directory` *dir*	
	Specifies the output directory in which the Irdrop and EM violation results will be saved in.
`-region {`*x1 y1 x2 y2* `}`	
	Specifies the region containing the worst instances. The default unit of *x1 y1 x2 y2* is in database units.
`-summary_only`	
	Specifies to generate a violation summary report that displays a summary of the IR drop violation distribution for all the cells. In addition, the report also displays the summary of the IR drop violation distribution for the standards cells. If this parameter is specified, the detailed IR report is not generated.
`-state_directory` *dir*	
	Specifies the top-level state directory that is generated by the dynamic IR drop analysis. This state directory contains analysis of all nets associated with the power domain. This is the output directory in which the results of the `analyze_rail` command is stored (for example, core_25C_dynamic_1).
`-tile_row_col {X Y}`	
	Specify the number of tiles in the x and y direction. The `-tile_row_col` and `-tile_size` parameters enable tile-based analysis of the hotspot regions. Tile-based analysis divides the design into user-defined tile size, and reports IR drop violations on each tile. The default value is `100um^2 (10x10 tiles)`.
`-tile_size {X Y}`	
	Specify the size of tiles in the x and y direction. The `-tile_row_col` and `-tile_size` parameters enable tile-based analysis of the hotspot regions. Tile-based analysis divides the design into user-defined tile size, and reports IR drop violations on each tile.

```
Hot-Spot Debugger Report

* Top Worst Instances
ID       | Instance Name
---------------------------------
       1| inst_I2
       2| inst_I3
...

#Power-Grid Integrity Analysis
Instance | LEF Pin:PG Net            | Floating PG Pins
-----------------------------------------------------------------
       1| VDD:VDD ; VSS:VSS         | None
       2| VDD:VDD ; VSS:VSS         | None
...

* Number of Weakly Connected Wire Segments
         VDD |         VSS
-------------------------
           0 |           0

#Power-Grid View Library Status
Instance| Status | Power Pins(V)| Intrinsic Cap (fF)    | View Used  | #Current Taps | PGV Library Path
-----------------------------------------------------------------------
      1| PASS   | VDD:1.1      | VDD:13.480 VSS:13.480 | Early      | VDD:1 VSS:1 | /data/cl/stdcells.cl

...

#IR-Drop Composition
Instance| Total IR-drop (V)        | IR-drop on Power Net (V)| Ground-bounce on ground net (V)
-----------------------------------------------------------------------
      1| 0.072 (TW) 0.072 (Worst) | 0.031 (TW) 0.031 (Worst)| 0.043 (TW) 0.043 (Worst)
...

* Layer Based IR-Drop for Net VDD
Instance| IR Across Package(mV) | IR Across Power-Gate(mV)| Layer Based IR(mV)
-----------------------------------------------------------------------
       |                       |                         |          M5L       0.683
       |                       |                         |        C4L5L       0.477
 1     |  0.035                |  0.000                  |          M4L       0.394
-----------------------------------------------------------------------
2 ...
-----------------------------------------------------------------------

* Layer Based IR-Drop for Net VSS
...
-----------------------------------------------------------------------

#Root Cause Analysis Report

* Net: VDD
Instance  | IR-Drop(V) | RLRP(Ohm) | Loading Cap(fF)| Current Density (mA/um2)| Transition Density| Frequency(Mhz) | Slew
-------------------------------------------------------------------------------------
        1| 0.031*     | 66.5*     | 64.371*        | 0.022*                  | 615384576*        | 307*           | 0.100*
...
-------------------------------------------------------------------------------------
Design Max| 0.058      | 159       | 140332.844     | 0.061                   | 615384640         | 307            | 0.984
Design Avg| 0.003      | 43.7      | 2.585          | 0.002                   | 93809752          | 300            | 0.076
Design Min| 0.000      | 0.0988    | 0.000          | 0.000                   | 0                 | 25             | 0.002

* Net: VSS
Instance  | IR-Drop(V) | RLRP(Ohm) | Loading Cap(fF)| Current Density (mA/um2)| Transition Density| Frequency(Mhz) | Slew
...
```

Examples

- The following command generates IR debug report for the `vss` net in the specified output directory:
  ```
  debug_irdrop -state_directory dynamic_rail/core_25C_dynamic_1/VSS -net VSS -output_directory IR_DEBUG_VSS -nworst_instances 10
  ```
- The following command generates a violation summary report that displays a summary of the IR drop violation distribution:
  ```
  debug_irdrop -summary_only
  ```

  The following is a sample summary report:

  ```
  Summary For All Cells:

  Worst IR drop: 0.123326V

  Specified IR drop threshold: 0.09V

  Worst IR drop violation: 0.033326V

  Total IR drop violation: 29.9416V

  IR drop violation distribution

  Violation Range   Violation Count  Accumulated Violation count  Total Violation(V)  Accumulated Total Violation(V)

  120mV ~ 125mV:       3                       3                       0.094411               0.094411

  115mV ~ 120mV:       32                      35                      0.858654               0.953065

  110mV ~ 115mV:       72                      107                     1.57548                2.52854

  105mV ~ 110mV:       150                     257                     2.57834                5.10688

  100mV ~ 105mV:       586                     843                     6.95461                12.0615

  95mV ~ 100mV:        1518                    2361                    11.115                 23.1765

  90mV ~ 95mV:         3013                    5374                    6.76508                29.9416


  Summary For StdCells:

  Worst IR drop: 0.123326V

  Specified IR drop threshold: 0.09V

  Worst IR drop violation: 0.033326V

  Total IR drop violation: 29.9416V

  IR drop violation distribution

  Violation Range   Violation Count  Accumulated Violation count  Total Violation(V)  Accumulated Total Violation(V)

  120mV ~ 125mV:       3                       3                       0.094411               0.094411

  115mV ~ 120mV:       32                      35                      0.858654               0.953065

  110mV ~ 115mV:       72                      107                     1.57548                2.52854

  105mV ~ 110mV:       150                     257                     2.57834                5.10688

  100mV ~ 105mV:       586                     843                     6.95461                12.0615

  95mV ~ 100mV:        1518                    2361                    11.115                 23.1765

  90mV ~ 95mV:         3013                    5374                    6.76508                29.941
  ```

Related Information

- `set_rail_analysis_mode`
- "Static Power, IRdrop and EM Analysis" in the *Voltus Stylus Common UI User Guide*

extract_package

```
extract_package
[-help] <domain>
[-output <directoryname>]
-pkg_bga_name <bga_circuitname>
-pkg_die_name <die_circuitname>
[-reference_net <netname>]
[-spd_file filename]
-workspace <workspace_name>
```

Specifies to start package model extraction.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
domain	Specifies the rail analysis domain name.
`-output <directoryname>`	
	By default, the package model will be created in the workspace. However, if you specify an output directory, then a copy of the result will be made in the new directory.
`-pkg_bga_name <bga_circuitname>`	
	Defines the ball grid array (BGA) circuit name in the package model. It is the PCB component in the package model, which eventually connects to the PCB model.
`-pkg_die_name <die_circuitname>`	
	Defines the die circuit name in the package model. When specified, the software automatically creates a mapping between the die name in the package extraction tool and the die name in the software.
`-reference_net <netname>`	
	Specifies the ground net that is selected as reference net for pin-based model generation. This is the ground net that is being referred to by the power nets.
`-spd_file filename`	
	Specifies an existing Sigrity .spd file for package extraction. This parameter allows you to specify a .spd file instead of the workspace name (package model database) of the package extraction tool (XtractIM).
`-workspace <workspace_name>`	
	Specifies the workspace name (package model database) of the package extraction tool (XtractIM). The file name extension is `.ximx`.

Example

- The following example specifies the package models details and starts extraction:

```
extract_package ALL \
  -workspace ../rak_xim_pkg_model/C1_Voltus_4L.ximx \
  -pkg_bga_name FPBGA144 \
  -pkg_die_name CD1 \
  -output rak_xim_pkg_model_voltus_cpf \
  -reference_net VSS
```

Related Information

- `set_advanced_package_options`
- "Package Analysis" in the *Voltus IC Power Integrity Solution User Guide*

generate_esd_rlrp_report

```
generate_esd_rlrp_report
[-help]
-net netname
[-instance_pair_list {{inst1 inst2 [pin1 pin2]}+} | -instance_pair_list_file filename]
[-number_threads value]
-state_directory directory
```

Reports the least-resistance path (RLRP) for the bump-to-clamp rule type in the ESD flow. This command helps to make debugging faster when a fail status is given for a bump-to-clamp pair. Using this command, you can:

- view the RLRP path for the selected bump-to-clamp pair in the GUI
- generate a point-to-point RLRP path report that displays the resistance between each node (resistor segment) pair along the RLRP path

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-net` *netname*	
	Specifies the name of the net to be analyzed.
`-number_threads` *value*	
	Specifies the maximum number of threads to use when reporting the least-resistance path (RLRP) for the bump-to-clamp rule type in the ESD flow. Generally, using more threads decreases the run time. There is no absolute limit to the number of threads you can use, however, it is recommended that you do not specify more threads than the number of CPUs available. The default value is `1`, and the maximum value that can be specified is the total number of available CPUs.
`-instance_pair_list {{inst1 inst2 [pin1 pin2]}+}`	
	Specifies the instance pair list to enable effective resistance calculation between two instances in the design. *pin* is an optional argument. By default, the tool will use the instance pin associated with the net being analyzed. The *pin* argument can be used in special cases when two different pins of an instance are connected to the same net. *pin1* and *pin2* are pins of *inst2* respectively. If a pin name is not given, all pins associated to the specified instance are used.
`-instance_pair_list_file` *filename*	
	Specifies the name of the instance pair list file. This file contains the instance pair list in the same format as mentioned for the `-instance_pair_list` parameter. The file allows you to specify a large number of instance pairs. `-instance_pair_list_file` and `-instance_pair_list` are mutually exclusive.
`-state_directory` *directory*	
	Specifies the name of the state directory in which the ESD analysis results are saved.

Examples

- The following commands specify to generate point-to-point RLRP path report for net `VDD_AO`:

```
generate_esd_rlrp_report \
-state_directory /rak_for_esd/ESD/ALL_125C_esd_1 \
-net VDD_AO \
-instance_pair_list { {VDD_AO_vsrc1 u0} {VDD_AO_vsrc2 u0 NA VDD} }
```

generate_power_up_report

```
generate_power_up_report
[-help]
[-switch_net net_name]
-output_directory directory_name
-state_directory directory_name
```

Specifies to generate power-up statistics report to view the ramp up/inrush current, worst power switch voltage, and additional instance voltage data (average, median, max, and standard deviation voltage data versus time) for the powering-up rail. The following files are generated in the GIF and text formats:

- average voltage of instances (.avgV)
- max voltage of instances (.maxV)
- min voltage of instances (.minV)
- median of instance voltages (.medianV)
- standard deviation of instance voltages (.deltaV)
- total inrush current (.inrushI)
- average voltage of power gates (.avgV)
- max voltage of power gates (.maxV)
- min voltage of power gates (.minV)
- median of power gate voltages (.medianV)
- standard deviation of power gate voltages (.deltaV)

Parameters

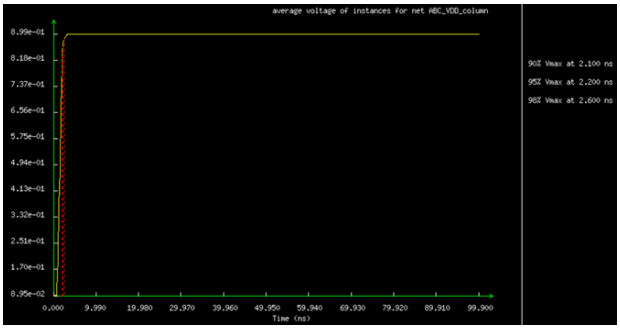
`-switch_net net_name`	
	Specifies the name of the powering up rail for which you want to generate detailed power-up statistics.
`-output_directory directory_name`	
	Specifies the output directory in which the power-up statistics results will be saved in.
`-state_directory directory_name`	
	Specifies the top-level state directory that is generated by IR drop analysis. This state directory contains analysis of all nets associated with the power domain; for example, core_25C_dynamic_1.

Examples

- The following command generates power-up statistics for the `ABC_VDD_column` net:
  ```
  generate_power_up_report -switch_net ABC_VDD_column -state_directory core_25C_dynamic_1 -output_directory rpt
  ```

  The following is a snippet of the text and GIF formats with information on average voltage of instances:

Text Format (`ABC_VDD_column.avgV`)	**GIF Format (`ABC_VDD_column.avgV`)**

`#List average voltage of instances for net ABC_VDD_column`
`# 90% of nominal voltage at 2.1 ns`
`# 95% of nominal voltage at 2.2 ns`
`# 98% of nominal voltage at 2.6 ns`
 	
`#XLABEL# Time (ns)`
`#YLABEL# Voltage (V)`
`0 0.0894674`
`0.1 0.0894675`
`0.2 0.0894676`
`...`
	

Related Information

- `set_rail_analysis_mode`
- "Power Gate Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

get_ir_insight

```
get_ir_insight
[-help]
-domain domainname
-domain_threshold value
[-eco_list list_of_instances ]
[-eco_mode {violation | aggressor | both}]
[-eco_report ]
-enable_xp {true | false}
[-lut_path directoryname]
[-nregion value]
[-nworst_instances value]
[-nworst_sensitivity value]
-output_directory directoryname
[-predict_eco]
[-predict_output filename]
[-reset_predict]
-state_directory directoryname
[-watch_box filename]
[-watch_inst filename]
```

The `get_ir_insight` command allows you do the following:

- IR drop distribution and summary reporting by the specified IR threshold - Worst Instance Voltage Drop (WIV), Total Instance Voltage Drop (TIV), and Number of Violation Instance Voltage Drop (NVIV)
- Aggressor IR impact analysis and reporting by the specified violation and aggressor limits
- Support various Tempus IR-ECO flows (violation or aggressor based flow with aggressor IR impact analysis and ECO IR estimation)
- IR drop prediction/estimation on input ECO changes
- IR violation reporting by regions

You must specify this command after running the `analyze_rail` command.

⚠ The `get_ir_insight` command requires the `invs_vi` (Innovus Voltus InsightAI Option) license.

Parameters

`-help`
Outputs a brief description that includes the type and default information for each `get_ir_insight` parameter. For a detailed description of the command and all of its parameters, use the man command man `get_ir_insight`.
`-domain domainname`
Specifies the name of the power domain.
`-domain_threshold value`
Specifies the minimum allowed voltage for the domain being analyzed. The value will be in percentage. This parameter behavior is same as the `set_rail_analysis_domain -threshold` parameter.
`-eco_list list_of_instances`
Specifies the list of instances which were changed for `-predict_eco`. The format of the file is: *<original x>,<original y>,<original orientation>,<original cell>,<x>,<y>,<orientation>,<cell>,<instance>*
`-eco_mode {violation | aggressor | both}`

Default: `violation`

Specifies the mode to be used for generating the ECO report with the `-eco_report` option. The possible options are:

- `violation`: Generates a list of violations in the report.
- `aggressor`: Generates a list of aggressors in the report.
- `both`: Generates a report that includes both the violations and aggressors.

`-eco_report`

Generates a file (`eco.ird`) that is used by the Tempus-ECO flow to fix IR drop violations in a design. The `eco.ird` file is located at: *output_directory/eco.ird*. The `-lut_path`, parameter must be specified with the `-eco_report` parameter. When the `-eco_report` is specified, it is recommended to also specify the `-eco_mode` parameter to support the required ECO flow.

`-enable_xp {true | false}`

Default : `false`

Specifies to identify the worst IR drop and EM violations regions in the Extensively Parallel (XP) mode wherein processing is distributed over a large number of machines.

`-lut_path` *`directoryname`*

Specifies the rail state directory containing the sensitivity database. You can enable sensitivity database generation by specifying `set_rail_analysis_mode -create_sensitivity_db true`. The `-lut_path` parameter must be specified with the `-predict_eco` and the `-eco_mode {aggressor | both}` parameters.

`-nregion` *`value`*

Default: 3

Specifies the number of regions to be analyzed.

`-nworst_instances` *`value`*

Default: 5

Specifies the number of worst instances to be reported.

`-nworst_sensitivity` *`value`*

Default: 50

Specifies the number of aggressors that should be reported for each violation.

`-output_directory` *`directoryname`*

Specifies the output directory in which the insight (IR drop and EM violation) results will be saved in.

`-predict_eco`

Specifies to generate the IR prediction for the ECO changes specified with the `-eco_list` parameter. Based on the list of instances specified with `-eco_list`, the tool performs IR prediction. You can also choose to use the `-watch_box` and `-watch_inst` parameters to specify either the area or instance you intend to observe and predict. You can specify multiple ECO change lists, and all the accumulated ECO changes are saved until you reset them using the `-reset_predict` parameter. This option is associated with `-predict_output`, `-eco_list`, `-watch_box`, `-watch_inst`, `-reset_predict`, and `-lut_path` parameters.

The format of prediction output is:

```
<instance_name1> <predicted_eiv_value1>
<instance_name2> <predicted_eiv_value2>
```

`-predict_output` *`filename`*

Specifies the output file where the prediction results for the `-predict_eco` parameter are saved.

The format of prediction output is:

```
instance_name1 predicted_eiv_value1
instance_name2 predicted_eiv_value2
```

`-reset_predict`

Specifies to reset all the previous ECO changes. If this parameter is not specified, the previous changes in `-eco_list` will be retained for the `-predict_eco` runs.
`-state_directory` *directoryname*
Specifies the top-level state directory that is generated by the dynamic IR drop analysis. This state directory contains analysis of all nets associated with the power domain. This is the output directory in which the results of the `analyze_rail` command is stored (for example, core_25C_dynamic_1).
`-watch_box` *filename*

Specifies a file containing the list of regions, wherein only the instances inside these regions will be reported for prediction. You can write multiple regions in the file. This parameter is used with the `-predict_eco` parameter.
The format of the file is:
`{{x1,y1}{x2,y2}}`
`{{x3,y3}{x4,y4}}`

`-watch_inst` *filename*

Specifies a file containing the list of instances to be reported for prediction. You can write multiple instances in the file. This parameter is used with the `-predict_eco` parameter.
The format of the file is:
`instance_1`
`Instance_2`

Examples

- The following command specifies to perform IR prediction/estimation:

```
get_ir_insight -predict_eco \
-state_directory Rail/latest \
-lut_path Rail/latest  \
-eco_list eco_five \
-predict_output predict.out \
-enable_xp true\
-domain PD \
-domain_threshold 0.1 \
-watch_box region_watch\
-watch_inst instance_watch\
-output_directory predictIR
```

  The generated report will be saved at: *predict.out*

  The following is the sample report:

```
CTS_buf_01236 0.828523
CTS_buf_01248 0.829177
CTS_buf_02745 0.817085
```

- Specifies to generate the eco.ird file for Tempus IR-ECO consumption:

```
get_ir_insight -eco_report \
-state_directory Rail/latest  \
-eco_mode both \
-enable_xp true\
-domain PD \
-domain_threshold 0.1 \
-lut_path Rail/latest  \
-output_directory debugIR
```

  The generated report will be saved at: *debugIR/eco.ird*

  The following is the sample report:

```
PGNET START
  NET   VDD  0.900
  NET   VSS  0.000
PGNET END

REGION START
  INST ua/FE_n10786  1
    IRDROP VDD:VSS  8.000e-11  0.1307  0.000e+00  7.029e-04  3.315e-03
  INST ua/FE_n6442  1
    IRDROP VDD:VSS  8.000e-11  0.1390  0.000e+00  7.452e-04  3.666e-03
```

Sample Summary Report

```
Found 8826 instances with IR-drop violations in design based on eiv_method: Worst and eiv_eval_window: elapse.

Summary For All Cells :
Worst IR drop : 0.12337V
Specified IR drop threshold : 0.09V
Worst IR drop violation : 0.033327V
Total IR drop violation : 48.94V

IR drop violation distribution

Violation Range   Violation Count  Accumulated Violation count  Total Violation(V)  Accumulated Total Violation(V)

120mV ~ 125mV:          6                       6                   0.186857                0.186857

115mV ~ 120mV:          34                      40                  0.918714                1.10557

110mV ~ 115mV:          83                      123                 1.81664                 2.92221

105mV ~ 110mV:          194                     317                 3.32192                 6.24413

100mV ~ 105mV:          904                     1221                10.5601                 16.8043

95mV ~ 100mV:          2940                     4161                21.3391                 38.1433
```

```
90mV ~ 95mV:          4665                    8826                10.7967                 48.94
```

Related Topics

- *"Voltus InsightAI"* in *Innovus User Guide*

map_dies

```
map_dies
[-help]
-3dic_design_stack_file filename
[-component_config_file filename]
[-die_pad_file_pair {{<die_name1> <pad file1>} {<die_name1> <pad file2>}...}]
[-map_search_distance distance]
[-mode sanity_check]
[-output directory]
[-stacked_die_mapping filename]
```

A utility to generate the inter die data mapping file based on the design stack configuration file and die ploc files. This mapping file is required if there is a direct connection between dies. The generated mapping file defines the connection between dies (between interposer and logic die). This mapping file is then specified with the `set_multi_die_analysis_mode -stacked_die_mapping` command for multi-die analysis.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-3dic_design_stack_file filename`	
	Specifies the name of the XML file containing the design stack information. This file is from the Integrity™ 3D-IC product. It includes each die's tier level (Z direction), die instance name, size (post shrink size if there is a scale factor), flip, rotation (counter, clockwise), placement location, and contact layer.
`-component_config_file filename`	

	Specifies the name of the design configuration file. This file is used to define the power pad location files, and the power and ground net names.
	
	File Format:
	`COMPONENT IDIE`
	`Power_pad { {ldie_VDD.ploc VDD_AO} {ldie_VSS.ploc VSS} }`
	`[Power_net { VDD_AO}]`
	`[Ground_net {VSS}]`
	`END`
	`COMPONENT LDIE`
	`Power_pad { {ldie_ALL.ploc all} }`
	`[Power_net {VDD1}]`
	`[Ground_net {VSS2}]`
	`END`

`-die_pad_file_pair {{<die_name1> <pad file1>} {<die_name1> <pad file2>}...}`	
	Specifies the mapping between die instance names and their corresponding power pad file for the interposer die and logic dies.
`-map_search_distance distance`	
	Specifies the legal mapping distance range between the interposer and legal dies. The unit for distance is micrometer (um).
`-mode sanity_check`	
	Specifies to perform sanity checks for the generated mapping file.
`-output directory`	

	Specifies the output directory containing the mapping results. The output folder contains the generated mapping file (`die_die.map`).
	The format of the 6-column mapping file is:
	`IDIE_INST_NAME(idie_inst_name) Inet_name Ibump_name`
	`LDIE_INST_NAME(ldie_inst_name) Lnet_name Lbump_name`
	Here,
	• The first 3 columns are for the interposer die instance name, net name, and bump name(vsrc name), and the last 3 columns are for the logic die instance name, net name, and bump name.
	• The die instance names are written in upper case that is followed by the die instance names specified in the rail TCL for each die instance name.
	• The file is split in two sections, POWER and GROUND.

`-stacked_die_mapping` *filename*	

	Specifies the mapping file used to define the connection between dies (between interposer and logic die). This file provides the information on bump-to-bump mapping (one bump from interposer die and the other bump from logic die) to provide logical connectivity.
	The format of the 6-column mapping file is:
	`IDIE VDD_AO BUMP_PWR_1 LDIE1 VDD_AO LDIE_PWR_BUMP7`
	`IDIE VDD_AO BUMP_PWR_2 LDIE1 VDD_AO LDIE_PWR_BUMP8`
	`...`
	Here, the first 3 columns are for the interposer die instance name, net name, and bump name, and the last 3 columns are for the logic die instance name, net name, and bump name.

Output

The following is the list of reports that are generated for sanity checking:

- `die_map.summary` - This report has 2 sections:
  - Interface mapping summary - Lists all the die_inst1/net to die_inst2/net mapping pairs, and their matched bump mappings in the overlap region.
  - Die connect-terminal summary - Reports matched and total terminals for each die instance/net/layer group.
- `map_diagnosis.log` - Reports a list of unmapped bumps/terminals for a die-to-die pair list and the reasons for the unmapped bumps (For example, unknown die, unknown net, undefined terminal, power to ground short, and floating terminal).
- `terminal_map.rpt` - Lists mapping pairs (`die1:net1:bump1 :: die2:net2:bump2`) that pass the mapping and alignment checks, therefore are considered as legal mapping pairs.
- `terminal_offset.rpt` - Reports all mapping pairs where offset is greater than the specified `map_search_distance`.

Examples

- The following command generates a mapping file that includes the connection between dies (between interposer die INTERPOSER and logic dies DIE0, DIE1, DIE2, DIE3):

```
map_dies \
-output results \
-3dic_config_file ../data/design_stack.xml \
-die_pad_file_pair { {INTERPOSER ../data/Interposer_all.ploc} \
                     {DIE0 ../data/Die0_all.ploc} \
                     {DIE1 ../data/Die1_all.ploc} \
                     {DIE2 ../data/Die2_all.ploc} \
                     {DIE3 ../data/Die3_all.ploc} }
```

- The following command performs the sanity check for the generated mapping file:

```
map_dies \
-mode sanity_check \
-output Results \
-3dic_design_stack_file ./data/design_stack.xml \
-3dic_config_file ./data/design_stack.xml \
-component_config_file ./data/Components.config \
-stacked_die_mapping ./data/die2die_bump.map \
-map_search_distance 5 \
```

Related Topics

- "Package Analysis" in the *Voltus User Guide*

map_die_package

```
map_die_package
[-help]
[-output directory]
-package_model_file filename
{-die_mcp_header filename | -net_pad_file_pair {{<net1> <pad file1>} {<net2> <pad file2>}...}}
```

The command launches the *MCP Editor* GUI to make manual connections between package and die pins. It also performs the following functions:

- maps the voltage source files with the package model to check and correct mapping information before rail analysis.
- generates a mapping file that can be used in previous versions of the software in case there is no mapping between die and package pins.

The command has two types of inputs:

- Voltage source files in the xy format and a package model file with the MCP header.
- A circuit file with the MCP header and a package model file with the MCP header.

The output will be the same for both types of inputs:

- A ckt mapping file containing only the MCP header, with the name `pkg_model_base_name + "_map"+suffix`.
- The file named '`un_mapped_bumps.txt`' will contain all the bumps which cannot be mapped.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-die_mcp_header` *filename*	
	Specifies a circuit file with the MCP header containing the die abstract.
`-net_pad_file_pair {{<net1> <pad file1>} {<net2> <pad file2>}...}`	
	Specifies the voltage source files mapping to each net.
`-output` *directory*	
	Specifies the output directory containing the mapping results.
`-package_model_file` *filename*	
	Specifies the name of the package model file.

Examples

- The following command specifies a circuit file with MCP header and a package model file with the MCP header:
  ```
  map_die_package -die_mcp_header super_filter_mcp_ploc.ckt -package_model_file C1_Voltus_4L_C1_Voltus_4L_Final_PinBaseSPICE.ckt -output out_ploc
  ```
- The following command specifies voltage source files in the XY format and a package model file with the MCP header:
  ```
  map_die_package -output ./out_pad -net_pad_file_pair {{VDD VDD_AO_srcs.pp} {VDD VDD_external_srcs.pp} {VSS VSS_srcs.pp}} -package_model_file C1_Voltus_4L_C1_Voltus_4L_Final_PinBaseSPICE.ckt
  ```

Related Information

- `set_rail_analysis_mode`
- "Package Analysis" in the *Voltus IC Power Integrity Solution User Guide*

read_power_rail_results

```
read_power_rail_results
[-help]
[-detail_delta_temperature_file file]
[-effective_resistance_file file]
[-instance_delta_temperature_file file]
[-power_db file ]
[-rail_directory dir]
[-tile_delta_temperature_file file]
[-cell_library { library1.cl}]
[-reset]
[-custom_value_inst_file_1 file]
[-nets net_names]
[-instance_voltage_method { best | worst | avg | worstavg }]
[-instance_voltage_window { timing | whole }]
[-instances instancenames]
[-enable_plots plot_names]
[-sem_db file]
[-short_file file]
[-esd_directory dir]
[-thermal_file filename.txt]
[-esd_rj dir]
[-die_instance_name dieinstname]
[-data_type {rail | all}]
[-physical_db dir_name]
[-profiling_power_file file]
[-top_cell_name topcell_name]
[-tile_thermal_config_file filename]
[-power_directory power_output_directory]
```

Loading power/rail analysis results into GUI.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `read_power_rail_results` parameter. For a detailed description of the command and all of its parameters, use the man command: `man read_power_rail_results`.
`-cell_library { library1.cl}`	
	Specifies the single cell library that is required to load the state directory. The specified cell library must contain the technology layer information that is required to load the state directory. Therefore, you do not need to load all cell libraries used during analysis.
`-custom_value_inst_file_1 file`	

	Specifies the name of the custom value instance file (CVIF). This file can be used to display a user-specified instance voltage file in GUI.
	The format of the CVIF file is:
	`value1  instancename1`
	`value2  instancename2`
	`....`
	where, *value1* and *value2* are voltage values mapping to the specified instance.
	You can use the `cvil` argument of the `set_power_rail_display -plot` command parameter to view the EIV map in GUI.

`-data_type {rail | all}`	

	Controls the type of data to be loaded in the GUI. The possible arguments of this parameter are: • `rail` - Loads only the rail analysis results. • `all` - Loads both the power and rail analysis results. This is the default value.
`-detail_delta_temperature_file` *file*	
	Specifies the name of the detailed wire-based delta temperature file from self-heating analysis.
`-die_instance_name` *dieinstname*	
	Specifies the name of the die whose results you need to view. This parameter is only applicable to the multi-die analysis flow (`set_rail_analysis_mode -die_mode multi-die`). The `-die_instance_name` parameter must be specified with the `-rail_directory` parameter (multi-die rail directory).
`-effective_resistance_file` *file*	
	Specifies the result file from effective resistance analysis (only instance-based result is supported). When the rail analysis plot type `effr` is selected and a result file is specified, the layout shows effective resistance analysis result for each instance. **Note**: Currently, this feature only supports loading one net at a time.
`-enable_plots` *plot_names*	
	Specifies the list of power or rail plots names that are to be loaded.
`-esd_directory` *dir*	
	Specifies the ESD analysis directory.
`-esd_rj` *dir*	
	Specifies the name of the directory where the current density/EM results generated by a previous ESD analysis run reside. This parameter allows you to selectively load the required EM result directory if there are multiple ESD EM result directories under the `ESD` directory. You must either load the rail directory (`-rail_directory`) before running this command, or specify the rail directory with the `-esd_rj` parameter.
`-instances` *instancenames*	
	Specifies a list of instances to load in the GUI. This parameter loads only those partitions that include the specified instances. The `-instances` parameter allows you to analyze parts of the designs, thereby reducing the loading time and memory footprint. The `-instances` parameter is applicable to the Voltus-XP flow only.
`-instance_delta_temperature_file` *file*	
	Specifies the name of the instance-based delta temperature file from self-heating analysis. This is the device instance temperature.
`-instance_voltage_method { best | worst | avg | worstavg }`	
	Loads the instance voltage data for the specified EIV computing method. If this parameter is not specified (default), the instance voltage results for all the four methods are loaded.
`-instance_voltage_window { timing | whole }`	
	Loads the instance voltage data for the selected EIV window. If this parameter is not specified (default), the instance voltage results for both the timing and switching windows, and the entire rail simulation are loaded.
`-nets` *net_names*	
	Loads only the specified nets in the domain-based state directory to the GUI. If this parameter is not specified with the `-rail_directory` parameter, all the nets will be loaded.
`-physical_db` *dir_name*	
	Specifies the name of the hierarchical design directory to be loaded to the GUI along with the rail analysis results.
`-power_db` *file*	
	Specifies the power database file. This file was created by using the `-create_binary_db` option of the `set_power_analysis_mode` command.
`-power_directory` *power_output_directory*	

	Specifies to load the power analysis results and display the power plots without loading the rail analysis results.
`-profiling_power_file` *file*	
	Specifies to display the tile-based power density results generated from the vector profiling flow.
`-reset`	
	Frees the loaded analysis state directory or power database from the memory.
`-rail_directory` *dir*	
	Specifies the rail analysis directory. You can load all the domain results by selecting the automatically generated analysis directory under the output directory (`<domain>_<temp>_<dynamic/avg>_#`), or load individual nets by going one level further and selecting the net directory.
`-sem_db` *file*	
	Specifies to read the signal electromigration data from the specified database file generated by the `verifyACLimit -report_db` command parameter.
`-short_file` *file*	
	Specifies the path to the short violations report (`voltus_short.report`) . This file is generated using the `set_rail_analysis_mode -report_shorts true` command parameter, and is located in the work directory.
`-thermal_file` *filename*`.txt`	

	Specifies to load the tile-based temperature map file generated by Sigrity Celsius to be used for both power calculation and IR drop analysis.
	The following is an example of reading the thermal map file:
	`read_power_rail_results -rail_directory output/static_rail/core_25C_avg_1/ -thermal_file output/power_map_by_layer.txt`
	`set_power_rail_display -plot thermal`

`-tile_delta_temperature_file` *file*	
	Specifies the name of the tile-based delta temperature file of self-heating analysis. This is the worst instance temperature among all wire segments in a tile. This is also layer-based. The FEOL or cell layer is used to show the device instance temperature.
`-tile_thermal_config_file` *filename*	

	Specifies to load the configuration file that contains the mapping between the hierarchical levels with corresponding layer-based temperature map files generated by Celsius (system-level thermal analysis tool). When specified, you can view tile-based temperature distribution for each layer of the chip.
	The following is the format and example of the configuration file:
	<hierarchy> <block name> <X_tile_no,Y_tile_no> <LEF/DEF Flip/Rotation> <temperature map>
	`TOP DIE 0,0 N /thermal_data/DIE_TemperatureMap`
	`TOP/IP1 IP1 25,50 N /thermal_data/IP1_TemperatureMap`
	`TOP/IP1/B1 B1 50,125 N /thermal_data/B1_TemperatureMap`
	`TOP/IP1/B2 B2 200,125 S /thermal_data/B2_TemperatureMap`
	This is a mandatory parameter for the `tile_thermal` plot when loading hierarchical temperature maps.

`-top_cell_name` *topcell_name*	
	Specifies the name of the top module or top cell name to be loaded. This parameter must be specified with the `-physical_db` parameter.

Examples

- The following command specifies the appropriate files needed for viewing the instance voltage and power data:

```
read_power_rail_results\
   -rail_directory analysis/domain_25C_avg_1 \
   -power_db powermeter.db \
```

- The following command specifies to load the VDD and VSS nets for the worst and best EIV methods during the entire simulation:

```
read_power_rail_results -rail_directory dynamic_rail/core_25C_dynamic_1 \
    -nets { VDD VSS } -instance_voltage_window { whole } -instance_voltage_method { worst best }
```

- The following command specifies to load only the rail directory:

```
read_power_rail_results -rail_directory dynamic_rail/core_25C_dynamic_1 -data_type rail
```

- The following command specifies to load the hierarchical database and the rail analysis results to the GUI:

```
read_power_rail_results -rail_directory ./dynamic_rail/latest/ -physical_db /dma/Dynamic/saveDB -top_cell_name dma_mac
```

- The following command specifies to display the tile-based power density results in the GUI:

```
read_power_rail_results -profiling_power_file ./vectorProfileResults/profile.rpt.density
```

- The following command specifies to load the power analysis results and display the power plots directly by running the power analysis without loading the rail analysis results:

```
read_power_rail_results -power_directory ./dynamic_power
```

Related Information

- report_power_rail_results
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*

report_power_rail_results

```
report_power_rail_results
[-help]
[-append]
[-enable_filter_summary]
[-filename file ]
[-filter_max max_value]
[-filter_min min_value]
[-ignore_limit_bound]
[-layers {All, layer_names } ]
[-limit N ]
[-nets {all | all_pwr | all_gnd | net_names}]
[-plot plot_type]
[-range_max max_value]
[-range_min min_value]
[-region x1 y1 x2 y2]
[-rlrp_inst {instance_name}]
```

Generates RLRP, region-based, and rail/power/capacitance reports.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_power_rail_results` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_power_rail_results`.
`-append`	Appends instance reports to the current report file.
`-enable_filter_summary`	
	Displays a summary of the filtered data in the histogram given in the *Summary* tab of the Result Browser form.
`-filename file`	
	Specifies the name of the text based violation report file to be created. The default file name is: • for region-based report: `region_x1_y1_x2_y2.rpt` • for RLRP report: `rlrp_instname.rpt` • for plot-based report: `power_rail_plot.rpt`
`-filter_max max_value`	
	Specifies the maximum value to include in the report. Use this option to further filter out the data within the range specified. This is used to obtain Optional, but if specified, the `-plot` option is required, and requires both `-filter_min` and `-filter_max` to be specified.
`-filter_min max_value`	
	Specifies the minimum value to include in the report. Use this option to further filter out the data within the range specified. This is used to obtain s Optional, but if specified, the `-plot` option is required, and requires both `-filter_min` and `-filter_max` to be specified.
`-ignore_limit_bound`	
	Specifies to ignore the maximum upper limit of `10000`, and instead use the higher value given with the `-limit` parameter for specifying the numbe in the power/rail report.
`-layers {all, layer_names }`	
	Specifies the layers to be reported. You can use `all` for reporting all layers, or list the specific layers that are to be reported. If this parameter is sp `-plot` parameter.
`-limit N`	

	Specifies the number of data points printed in the report. If this parameter is specified, you must specify the `-plot` parameter. *Default* : first `1000` data points
`-nets {all | all_pwr | all_gnd | net_names}`	

	Specifies the nets to be used for analysis reporting. You can use `all` for reporting all nets, `all_pwr` for reporting all power nets, `all_gnd` for reporti specific nets that are to be reported. If this parameter is specified, you must specify the `-plot` parameter.
	This parameter also allows you to specify the switched and always-on nets to be used for analysis reporting.
	The following is the use model to generate a report for the specified switched net:
	`report_power_rail_results -plot ir -nets {switchedNetName}`
	The following is the use model to generate a report for the specified always-on net, including its switched nets:
	`report_power_rail_results -plot ir -nets {alwaysOnNetName} -all`
	You must specify `-all` to include all switched nets associated with the specified always-on net.

`-plot plot_type`	
	Plots the specified plot type. See Plot Types for a description of each of the plot types.
`-range_max max_value`	
	Specifies the maximum value to report. Optional, but if specified, the `- plot` option is required, and requires both `-range_min` and `- range_max` to b
`-range_min min_value`	
	Specifies the minimum value to report. Optional, but if specified, the `- plot` option is required, and requires both `-range_min` and `- range_max` to be
`-region x1 y1 x2 y2`	
	Measures tap current and capacitance information in the specified region defined by x1, y1, and x2 y2 coordinates, and prints results in the log fil
`-rlrp_inst instance_name`	
	Reports the RLRP path for the specified instance. The default file name of the RLRP path report is `<rlrp_instname.rpt>`. You can modify the file parameter.

Plot Types

Plot Type	Description
`ac`	Available/feasible capacitance (Farad) Displays the feasible decoupling capacitance that can be added in the area. The feasible capacitance in the area is calculated based on the filler cells placement. The program matches the placed filler cells with the decap cells available in the power-grid library and computes the feasible capacitance. The capacitance of the decap cell is stored in the power-grid library.
`activity`	Displays the worst activity for the output and inout pins of every instance.
`cap`	Grid capacitance (Farad) Plots the capacitance of the extracted power-grid network.
`dd`	Decoupling capacitance (decap) density (Farad/um ^{2}) Plots decap density of the design. The decap density is the cell internal decoupling capacitance in the unit area (µm ^{2}).
`dr`	Decoupling capacitance required Reports the decoupling capacitance required to fix dynamic IR-drop violations. This option suggests how much decap needs to be added in a given area to fix the IR-drop violations. The voltage threshold used to determine the IR-drop violations for this analysis is the value specified by the `-threshold` option of the `set_pg_nets` command.

`ivdn`	Instance Voltage Drop (Volts) – net-based Plots the net-based instance voltage drop.
`ivdn`	Instance Voltage Drop (Volts) – domain-based Plots the domain-based instance voltage drop. For domain based, at least one power and one ground net must be selected.
`fd`	Filler cell density (1/μm^2) Displays the filler cell density per unit area (μm^2). The plot can be used to analyze the feasibility of adding decap cells by replacing the filler cells and not changing the placement of the logic cells.
`freq`	Frequency domain (Hz) Displays the associated clock frequency of all instances in the design. The instances associated with multiple clock domains are displayed using the fastest clock frequency. This plot can be used to analyze and debug the power distribution in the design.
`ip`	Instance total power (mW) The total power consumed by each instance is displayed. The total power is the sum of the internal, switching and leakage power of the instance. The power data is read from the power database and can be specified using the `-power_db` option or it is queried directly from memory, if power calculation is run during the session.
`ip_i`	Instance internal power (mW) The power consumed by charging and discharging of the internal interconnect and device capacitances of the instance.
`ip_l`	Instance leakage power (mW) The power consumed by devices when not switching.
`ip_s`	Instance switching power (mW) The power consumed by charging and discharging of the interconnect or output load.
`ipd_i`	Specifies internal power density in mW/um^2.
`ipd_l`	Specifies leakage power density in mW/um^2.
`ipd_s`	Specifies switching power density in mW/um^2.
`ipd_t`	Specifies total power density in in mW/um^2.
`ip_tr`	Displays toggle rates for all instances. The toggle rate for an instance is calculated as the average of toggle rates of all outputs of the instance. This plot allows you to determine the design area that toggles more than the others.
`ir`	IRdrop or voltage drop (Volts) Displays the IRdrop across each extracted power-grid segment in the design.
`jrms`	Specifies current density as j/jmax based on the Irms current. This plot is only supported for dynamic analysis. A jrms gif file will be saved in the state directory that shows where the Irms based EM violation occurred. This plot type allows you to verify the analysis results of "jrms" using the GUI.
`load`	Loading capacitance (Farad) Displays the output loading capacitance driven by the instance. This plot can be used to analyze and debug the regions with high switching power.
`pi`	Power-switch (gate) current violation: I/Idsat Displays the ratio of current through the power-switch and the saturation current (Idsat) of the power-switch. This plot can be used to analyze and debug whether the current through power-switches exceeds the saturation current characterized for the power-switch devices. The saturation current and on-resistance of the power-switches are characterized and stored inside the power-grid library of the power-switch cell. A ratio of greater than 1 means that the current requirement of the power-gated block can not be met with placed power-switch instance. Either the power-switch needs to be made larger or power-switch instances need to be added.

`pv`	Voltage drop across power-switch (Volt) Displays the IRdrop across power-switch instances. This plot can be used to analyze and debug the IRdrop inside the power-gated block, e.g. if the IRdrop across power-switches is already high, the IRdrop inside the power-gated block will be much higher. In this case, the power-switch placement will have to be refined in order to resolve the IRdrop problem.
`rc`	Resistor current (Amp) Displays the current across power-grid resistor segments. This plot can be used to analyze and debug how the current flows from the voltage sources into the rest of the design, e.g. high current should flow from pads to the top-level routing level layers and then to the lower level metal layers. This plot can also be used to debug current density violations (`rj`).
`rlrp`	The total resistance of the instance along the least resistive path. Displays the total resistance of the instance along the least resistive path, meaning the resistance of the instance along the path from where it gets the most amount of current. This plot can be used to analyze and debug the power-grid integrity early in the design stage. An instance with high resistance is likely to have high static or dynamic IRdrop depending on the vector that is simulated later in the design cycle. This plot can be also used during decap optimization, since regions with high dynamic IRdrop and high resistance may not be improved upon by adding decaps, because the effectiveness of decoupling capacitance depends on the resistance of the power-grid network. You can also highlight the least resistive path of the instance using the GUI and selecting the instance with left mouse button. The *RLRP Path* checkbox should be enabled.
`res`	Displays the total resistance of the least resistive path from each node to voltage source. This plot can be used to view high resistance regions of the power-grid. It is useful during IRdrop and decap analysis to identify weak power-grid connectivity.
`rj`	Resistor current density, J/Jmax. Displays the current density (current per unit area) violation on each power-grid segment using the ratio of calculated current density and maximum allowed current density. A ratio of greater than 1 means that the current density limit of the segment is violated. The current density limits are supplied by the foundry and can be read during analysis using the `set_rail_analysis_mode -em_models` command.
rs	Specifies the plot type for resistor sensitivity.
`slack`	Instance slack (Seconds) Displays worst instance slack. The slack data is queried from either the timing database or if the power database is specified using `-power_db` option, then the worst slack data is read from the power database. When using an external TWF, the slack data is read from the TWF.
`tc`	Tap current (Amp) Displays the current consumed by the instance devices or current taps. During rail analysis, it is based on the power consumption of the instance and is distributed inside the cell. The current distribution inside the cell is based on the extracted device netlist stored inside the power-grid library of the cell. The current distribution is generally based on the device width, length, and spice models. This plot can be used to analyze and debug IRdrop inside the macro.
`td`	Transition density (Hz) Displays the worst transition density (activity * frequency) of the instance. This plot can be used to analyze and debug activity distribution and high power density regions.
`tpd`	Displays power density map with user-specified grid size.
`unc`	Unconnected segments Displays power-grid segments that are disconnected from the voltage source (power pads). This plot can be used to debug unusually high IRdrop in the region.
`vc`	Voltage source current (Amp) Displays current through the voltage sources (power pads). This plot can be used to analyze the current carrying capacity of the power-pad.
`vu`	Voltage drop across package (Volt) Displays voltage drop across package. If analysis is run with package model attached to the power pads, this plot displays the IRdrop due to package RLCK elements.

Examples

- The following command will report the worst 100 IR segments on Metal 1 of the VDD net:
  `report_power_rail_results -plot ir -layers M1 -limit 100 -nets VDD`
- The following command will report the IR drop of the VDD_ring switched net:
  `report_power_rail_results -plot ir -net {VDD_ring}`
- The following command will report the IR drop of the VDD_AO always-on net, including its switched nets:
  `report_power_rail_results -plot ir -net {VDD_AO} -all`
- The following command filters the data such that it does not display the data points below 200 in the Result Browser histogram:
  `report_power_rail_results -plot ivdd -filter_min 200 -enable_filter_summary`

Related Information

- `read_power_rail_results`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*

scale_what_if_capacitance

```
scale_what_if_capacitance
[-help]
-reset | {{-global | -region x1 y1 x2 y2 | -cell cell_name | -instance inst_name | -file filename}
[-instrinsic_cap value]
[-loading_cap value]
[-grid_cap value]
[-add]
[-layer name]
[-net net_name]
}
```

Scales the intrinsic cell capacitance value to determine how much additional capacitance can be added in a region to reduce dynamic IR drop.

Parameters

`-add`	
	Specifies to add capacitance (farads) on the power-grid on a specific layer and region. The parameter works in conjunction with the existing parameters, such as `-intrinsic_cap`, `-grid_cap`, and `-loading_cap`. When this parameter is specified, the value provided with the `-intrinsic_cap`, `-grid_cap`, and `-loading_cap` parameters will be treated as absolute value instead of a scale factor value, which is the default behavior.
`-cell cell_name`	
	Specifies to scale the capacitance value for the specified cell.
`-file filename`	

	Specifies a file containing the list of instances and cells that use the same scaling factor. The format of the file is:
	`CELL cellname`
	`INSTANCE instancename`

`-global | -region x1 y1 x2 y2`	
	You must specify either `-global` or `-region x1 y1 x2 y2`. The `-global` option scales the specified capacitances for the entire design. The `-region x1 y1 x2 y2` scales the specified capacitances in the region. The default unit of `x1 y1 x2 y2` is in database units
`-grid_cap value`	

	Scales power-grid capacitance of the design to the specified scale factor. The default scale value is 1. ⚠ The `-grid_cap` parameter can be specified either with the `-global` or the `-region` parameter. This parameter is not supported with the `-cell` and `-instance` parameters.
`-help`	Outputs a brief description that includes the type and default information for each `scale_what_if_capacitance` parameter. For a detailed description of the command and all of its parameters, use the man command: `man scale_what_if_capacitance`.
`-instance` *`inst_name`*	
	Specifies to scale the capacitance value for the specified instance.
`-intrinsic_cap` *`value`*	
	Scales instance internal gate capacitances to the specified scale factor. The default scale value is 1.
`-layer` *`layername`* `|` `all`	
	Specifies the LEF layer on which capacitance scaling is performed. Multiple commands can be issued to scale capacitance on multiple layers. You can specify the `all` argument to scale all the layers.
`-loading_cap` *`value`*	
	Scales loading capacitance of the instance to the specified scale factor. The default scale value is 1.
`-net` *`net_name`*	
	Specifies to scale the capacitance value for the specified net.
`-reset`	Resets all the previous `scale_what_if_capacitance` command settings.

Examples

- **Cell-based capacitance scaling** - The following command scales intrinsic capacitance on all instances of the cell *cellA* by 1e-10 (<*intrinsic_cap*> * 1e-10):
  ```
  scale_what_if_capacitance -cell cellA  -intrinsic_cap 1e-10
  ```
- **Pin-based capacitance scaling** - The following command scales intrinsic and loading capacitance on all instances of the cell *cellA* connected to the pin *vdd* by 1e-10 (<*intrinsic_cap*> * 1e-10 and <*loading_cap*> *1e-10):
  ```
  scale_what_if_capacitance -cell cellA -net vdd -intrinsic_cap 1e-10  -add -loading_cap 1e-10
  ```
- **Region-based capacitance scaling** - The following command scales capacitance in the specified region by 1e-10 (<*intrinsic_cap*> * 1e-10, <*loading_cap*> *1e-10, and <*grid_cap*> * 1e-10):
  ```
  scale_what_if_capacitance -region {896.877  937.661  1276.9365  1615.358} -intrinsic_cap 1e-10  -loading_cap 1e-10  -grid_cap 1e-10
  ```
- The following command scales grid capacitance in the specified layer by 1e-10:
  ```
  scale_what_if_capacitance -net VDD -region { 0 0 100 100 } -layer M2 -grid_cap 1e-10
  ```

Related Information

- `create_current_region`
- `scale_what_if_resistance`
- `scale_what_if_current`
- "What-If Rail Analysis" in the *Voltus IC Power Integrity Solution User Guide*

scale_what_if_current

```
scale_what_if_current
[-help]
[-hierarchy hier_name | -region x1 y1 x2 y2 | -global | -inst_list filename]
[-scale_clock_network]
[-scale value | -current value]
[-net net_name]
[-reset]
```

Scales current to do what-if analysis for a hierarchical partition and assess its effect on IR drop.

Parameters

`-current value`	
	Specifies current value in Amperes. This is used as target peak current for the given hierarchy and is then scaled appropriately. You cannot specify this option with `-region x1 y1 x2 y2` or `-global` options.
`-global`	Specifies current scaling to be applied to entire design.
`-help`	Outputs a brief description that includes the type and default information for each `scale_what_if_current` parameter. For a detailed description of the command and all of its parameters, use the man command `man scale_what_if_current`.
`-hierarchy hier_name`	
	Specifies the name of the hierarchical block. For example, `-hierarchy top/blockA`. To specify global scaling for the design use "`/`". For example, `-hierarchy /` `-hierarchy` is a required parameter. This parameter allows instance-based current scaling by the specified scale factor. The parameter can be used to scale down peak current values. The use model of instance current scaling is: `scale_what_if_current -hierarchy <instance_name> -scale <value>`
`-net net_name`	
	Scales current of the specified net.
`-inst_list filename`	

	Specifies to scale current for the specified nets. This parameter allows you to specify a file containing the list of instances and their scale factor.
	Use Model
	`scale_what_if_current -inst_list filename -scale val`
	The format of `filename` is:
	`instanceName scaleFactor`
	`instanceName scaleFactor`
	Here,
	• `val` means the global scale factor. This value will be used if the local or instance-specific scale factor is not specified.
	• `scaleFactor` means the local scale factor. If both global and local scale factors are specified, the local scale factor takes precedence.
	Default scale factor value is `1`.

`-region x1 y1 x2 y2`	
	Specifies a region where current scaling is to be applied. The default unit of `x` and `y` coordinates is in microns (um).
`-reset`	Resets all the previous `scale_what_if_current` command settings.
`-scale value`	

	Specifies any value greater than `0`. For example, a value of `0.5` will scale the current in half. This is a required parameter.
`-scale_clock_network`	
	When specified, scales current for all instances within the given hierarchy. When not specified, the software does not scale the instances that belong to the clock network. It scales current for the rest of the instances within the given hierarchy to the specified value. You cannot specify this option with `-region x1 y1 x2 y2` or `-global` options. This is an optional parameter.

Examples

- The following command scales current for the hierarchical block `BLOCK_C`:
  ```
  scale_what_if_current -hierarchy top/BLOCK_C -current 0.5
  ```

Related Information

- `scale_what_if_capacitance`
- `scale_what_if_resistance`
- `create_current_region`
- "What-If Rail Analysis" in the *Voltus IC Power Integrity Solution User Guide*

scale_what_if_resistance

```
scale_what_if_resistance
[-help]
-reset | {-net netname
{-global | -region x1 y1 x2 y2 | -instance instancename | -cell cellname | -file filename}
[-scale value | -scale_ron {R_ON_scale_factor | R_OFF}]
[-auto_scale_adjacent_via_layers {true | false}]
[-layer {layername | all} ]
}
```

Scales resistance globally or in a region to determine how much resistance can be modified in a region to reduce the entire IR drop. In designs with power switches, the scaling of resistance will be applied to always-on net as well as its switched nets in a given region. Currently, it is not possible to apply different scaling factors for always-on and switched nets.

Parameters

`-auto_scale_adjacent_via_layers {true|false}`	
	Specifies to use auto via scaling for what-if resistance analysis. When scaling resistance on a specific metal layer, via resistance on the adjacent via layers are automatically scaled with the same scaling factor. The default value is `true`. Use `-auto_scale_adjacent_via_layers false` to keep via resistances un-scaled.
`-global | -region x1 y1 x2 y2 | -instance instancename | -cell cellname | -file filename`	

	You must specify either `-global`, `-region`, `-cell`, `-instance`, or `-file`.
	• When the `-global` parameter is specified with the `-scale` parameter, it scales resistance by the specified value globally for a given layer. If the `-global` parameter is specified with the `-scale_ron` parameter, it scales the resistance value of all power switch cells globally for a given layer. This is the default.
	• When the `-region` parameter is specified with the `-scale` parameter, it scales the grid resistance in the specified region. If the `region` parameter is specified with the `-scale_ron` parameter, it scales the resistance of all power switch cells in the specified region. The default unit of x1 y1 x2 y2 is in database units.
	• The `-instance instancename` or `-cell cellname` parameter specifies to scale the internal resistors from the PGV for the specified instance or cell.
	• The `-file filename` parameter specifies a file containing the list of instances and cells that use the same scaling factor. The format of the file is:
	`CELL cellname`
	`INSTANCE instancename`
	This is a required parameter. Default is `-global`.
	The precedence for resistance scaling is as follows:
	1. Instance
	2. Cell
	3. Region
	4. Global

`-help`	Outputs a brief description that includes the type and default information for each `scale_what_if_resistance` parameter. For a detailed description of the command and all of its parameters, use the man command `man scale_what_if_resistance`.
`-layer layername | all`	
	Specifies the layer name (LEF layer or technology layer) on which the scaling is performed. Multiple commands can be issued to scale resistance on multiple layers. You can specify the `all` argument to scale all the layers. When the `all` argument is used, the `-auto_scale_adjacent_via_layer` parameter is disabled. This is a required parameter.
`-net netname`	

	Scales resistance of the specified net in the domain which is being analyzed. This is a required parameter.
`-reset`	Resets all the previous `scale_what_if_resistance` command settings.
`-scale` *value*	
	Specifies any value greater than 0. For example, a value of 0.5 will scale the resistance in half. This is a required parameter.
`-scale_ron {`*R_ON_scale_factor* `| R_OFF}`	
	Specifies the value used to scale the RON of the power-gate instance/cell. The default value is `1`. You can also specify the "`R_OFF`"(case insensitive) keyword to apply the off state power-gate resistance for the specified instance/cell.

Related Information

- `scale_what_if_capacitance`
- `create_current_region`
- `scale_what_if_current`
- "What-If Rail Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_advanced_package_options

```
set_advanced_package_options
[-help]
-reset  |
{[-tool_path <binary_path>] [-net_mapping_file <filename>]}
```

Specifies the path to the bin folder of package extraction and analysis tools. Also, a net mapping file needs to be specified. The mapping file is a simple two column format containing the die net name and the corresponding package net names. After running this command, use the `extract_package` command to start package model extraction.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-tool_path <binary_path>`	
	Specifies the full path to the bin folder of the package extraction and analysis tools (XtractIM and PowerDC).
`-net_mapping_file <filename>`	
	Specifies the name of the two column mapping file that contains the mapping between the net names of package with the net names of die.
`-reset`	
	Resets all specified options back to default values.

Example

- The following example specifies the path to the bin folder of package extraction and analysis tools:

```
set_advanced_package_options \
   -tool_path /home/sigrity/tools/bin/ \
   -net_mapping_file net_mapping.txt
```

Related Information

- `extract_package`
- "Package Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_advanced_rail_options

```
set_advanced_rail_options
[-help]
[-reset] | [[-voltus_rail_include_file_begin file_name1]
[-voltus_rail_include_file_end file_name2] ]
```

Provides the ability to include a file that provides a list of additional rail options that should be used for rail analysis. If you do not specify values to the set_advanced_rail_options command parameters, the parameter values are set to the default value, that is, empty file name.

For more information on the `vstorm2` commands, refer to the "Rail Analysis Options" section in the *Voltus IC Power Integrity Solution Text Command Reference.*

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-reset`	Resets parameters to their default values. This means that an extra include file is not required.
`-voltus_rail_include_file_begin file_name`	
	Provides the ability to provide a file that includes additional rail options that will be run prior to the analyze command. This can be used for those legacy rail analysis commands that do not have a Voltus equivalent.
`-voltus_rail_include_file_end file_name`	
	Provides the ability to provide a file that includes additional rail options that will be run after the analyze command. This can be used for those legacy rail analysis commands that do not have a Voltus equivalent.

Examples

- The following command needs to be set to override the default precision digits:
  ```
  set_advanced_rail_options -voltus_rail_include_file_begin vstorm.file


  <vstorm.file>
  setvar precision_digits 10
  ```
  With this variable, you will get IR with 10 precision digits (10 includes all digits before and after the decimal point).

Related Information

- `set_rail_analysis_mode`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_die_model

```
set_die_model
[-help]
[-subckt subcircuitname]
{-reset  | [[-die_instance_name dieinstname [-spice model_file [-mapping mapping_file]]] ]
}
```

Specifies to import a die model on die Interposer or on package netlist.

This command gives information about the die model that will be used in the rail analysis of another die, wherein the two die are connected through die Interposer die or package netlist. For example, if you are performing IR drop analysis for a die (**die2**) on an Interposer/package design that is connected to another die (**die1**), and if the two die share the power/ground network, it is imperative to import the die model of **die1** during IR drop analysis of **die2**.

You can import multiple die models by specifying multiple `set_die_model` commands.

Parameters

`-die_instance_name` *dieinstname*	
	Mandatory parameter. Specifies the die instance name for die model. This is the die model of the die that needs to be imported.
`-help`	Outputs a brief description that includes the type and default information for each `set_die_model` parameter. For a detailed description of the command and all of its parameters, use the man command `man set_die_model`.
`-mapping` *mapping_file*	

	Mandatory parameter. Specifies the name of the mapping file that contains the port connection mapping between the die model and voltage source of the chip to be analyzed.
	The two mapping file formats are:
	• **Mapping when die model on LEF/DEF-based Interposer Die**
	`#port_of_die_model  cell  die_name/VSRC_pad_name`
	`VDD_Bump_VDD_9_0  cell  Interposer/Bump_DIE0_VDD_0`
	`VDD_Bump_VDD_9_0  cell  Interposer/Bump_DIE0_VDD_1`
	...
	• **Mapping when die model on Package Design**
	`#port_of_die_model  pkg  port_of_pkg_netlist`
	`VDD_Bump_VDD_9_0  pkg  VDD_3`
	`VDD_Bump_VDD_9_0  pkg  VDD_4`

`-reset`	Resets all the previous `set_die_model` command settings to the default settings.
`-spice` *model_file*	
	Mandatory parameter. Specifies a file containing a SPICE subcircuit describing the die model.
`-subckt` *subcircuitname*	
	Defines the top-level subcircuit in package files. When this parameter is not specified, the first circuit in the Spice file is used as the default top circuit.

Examples

- The following command imports two die models `Die1` and `Die2`:

```
set_die_model -die_instance_name Die1 -spice die1_model.ckt -mapping die1_port.txt
set_die_model -die_instance_name Die2 -spice die2_model.ckt -mapping die2_port.txt
```

Related Information

- set_package
- create_die_model
- "Package Analysis" in the *Voltus Stylus Common UI User Guide*

set_dynamic_rail_simulation

```
set_dynamic_rail_simulation
[-help]
[-start value]
[-stop value]
[-resolution value]
[-reset]
[-auto_repeat]
```

An optional command to set the parameters for dynamic rail analysis. By default, dynamic rail analysis uses the start time, stop time, and resolution from current data file. To analyze fewer clock cycles, you can choose different start and stop times using the `-start` and `-stop` options. It is recommended that at least one or two full cycles are simulated to fully charge and discharge the on-chip capacitance.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `set_dynamic_rail_simulation` parameter. For a detailed description of the command and all of its parameters, use the man command `man set_dynamic_rail_simulation`.
`-auto_repeat`	

	Specifies to repeat the dynamic current waveform till the `-stop` time specified. If `-stop` is not specified, all the waveforms will repeat till the longest waveform duration. The `-auto_repeat` parameter can be used when multiple dynamic current files have different simulation periods or when IR drop analysis needs to be run for longer duration.
	The following command sets the dynamic simulation time from 20ns to 100ns in which the current data will be repeated until 100ns is reached:
	`set_dynamic_rail_simulation -start 20ns -auto_repeat -stop 100ns`
	The `-repeat` parameter of the `set_power_data` command has higher priority than `set_dynamic_rail_simulation -auto_repeat`, as shown in the following example:
	`set_power_data VDD1.ptiavg -repeat 50`
	`set_power_data VDD2.ptiavg`
	`set_dynamic_rail_simulation -auto_repeat -stop 100`
	In this example, VDD1 waveform will repeat till 50ns, while VDD2 waveform will repeat till 100ns.

`-reset`	Resets all the previous `set_dynamic_rail_simulation` command settings to the default settings.
`-resolution value`	
	By default rail analysis picks up the resolution (or step size) saved in the dynamic current files. To achieve better rail analysis performance, you can increase the step size using this option. However, using larger than two times the step size of current data file is not recommended as it can impact accuracy of dynamic IR drop analysis. You can specify the units with the value. If you do not specify a unit, the software uses the default unit. *Default*: Picoseconds (ps).
`-start value`	
	Defines the start time to be used for dynamic rail analysis. This option is used with the `-stop` option and enables you to perform a more detailed analysis in specific time ranges. You can specify the units with the value. If you do not specify a unit, the software uses the default unit. *Default*: The default unit is nanoseconds (ns). The default value is the start time from current data file. The start time in the current data file is usually 0ns.
`-stop value`	
	Defines the stop time to be used for dynamic rail analysis. This option is used with the `-start` option and enables you to perform a more detailed analysis in specific time ranges. You can specify the units with the value. If you do not specify a unit, the software uses the default unit. *Default*: The default unit is nanoseconds (ns). The default value is the end of the current data. If multiple current files are specified and they all have different simulation periods, the largest simulation period is used to decide the stop time.

Examples

- The following command sets the dynamic simulation time from 2ns to 12ns with a resolution of 100ps:
  ```
  set_dynamic_rail_simulation -start 2ns -stop 12ns -resolution 100ps
  ```

Related Information

- `set_rail_analysis_mode`
- `set_power_data`
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_multi_die_analysis_mode

```
set_multi_die_analysis_mode
[-help]
-name analysis_name
-die_list {die_instance_name1 die_instance_name2}
[-report_rlrp_path_instance_list filename]
[-stacked_die_mapping Innovus_map_file]
```

Specifies how the multi-die analysis will be performed.

Parameters

`-die_list` *die_instance_name1 die_instance_name2*	
	Specifies the die instance names.
`-help`	Outputs a brief description that includes the type and default information for each `set_multi_die_analysis_mode` parameter. For a detailed description of the command and all of its parameters, use the man command `man set_multi_die_analysis_mode`.
`-name` *analysis_name*	
	Specifies the name of the analysis.
`-report_rlrp_path_instance_list` *filename*	

	Specifies a file containing a list of die instances (one per line) for which the RLRP path should be reported. This parameter is used to generate the l die.
	The format of the specified input file is:
	<die_name> <net_name>
	The `-report_rlrp_path_instance_list` parameter generates a report `mdie_inst_rlrp_path.rpt` at the following location: *<output_dir>/<state_di*
	The format of the report is:
	`Die Name: <die_name>`
	`Net Name: <net_name>`
	`Pin Name: <pin_name>`
	`Instance Name: <inst_name>`
	`Die Layer/Via Resistance(cumulative) Ohm Length Node1#(X,Y) Node2#(X,Y) Net`
	Example of an Output File:
	`Die Name: LDIE1`
	`Net Name: VDD_AO`
	`Pin Name: TVDD`
	`Instance Name: ring/PSO_RING_78`
	`Die Layer/Via Resistance(cumulative) Ohm Length Node1#(X,Y) Node2#(X,Y) Net`
	`LDIE1 METAL_1 0.1238(3.4182) 2.210 169.28500,233.27500 167.07500,233.27500 VDD_AO`
	`LDIE1 METAL_1 0.1302(2.7137) 2.035 169.28500,221.89000 169.28500,223.92500 VDD_AO`
	`LDIE1 VIA_1 0.0269(2.5835) 0.140 169.28500,221.89000 169.28500,221.89000 VDD_AO`
	`LDIE1 METAL_2 0.0070(2.5565) 0.145 169.28500,221.74500 169.28500,221.89000 VDD_AO`
	`LDIE1 VIA_2 0.0875(2.5496) 0.140 169.28500,221.74500 169.28500,221.74500 VDD_AO`
	`LDIE1 VIA_3 0.0875(2.4621) 0.140 169.28500,221.74500 169.28500,221.74500 VDD_AO`
	...

`-stacked_die_mapping` *Innovus_map_file*	
	Specifies the name of the inter die data mapping file from Innovus. The mapping file is a virtual mapping that provides the connections between any communication directly between the die (master/slave configuration).

Examples

- The following command specifies an analysis name called `tsv`, two die instances called `mother_die` and `daughter_die`,and a stacked mapping file `pwr.mapping`:

```
set_multi_die_analysis_mode -name tsv -die_list {mother_die daughter_die}
-stacked_die_mapping ../data/pwr.mapping
```

Related Information

- `set_rail_analysis_mode`
- `create_die_model`
- "Through-Silicon Via and System-in-Package" in the *Voltus IC Power Integrity Solution User Guide*

set_net_group

```
set_net_group
[-help]
-reset | {-name group_name
-type [power | ground]
-nets {net_list}
}
```

Specifies the list of either power or ground nets that are isolated but need to be analyzed together as a single net. The results are generated for the new grouped net. The new grouped net name must be one of the net names in the group. To analyze the grouped net, you must also specify power pads for the grouped net name.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `set_net_group` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_net_group`.
`-name group_name`	
	Name of the net group. **Note:** The name of the net group must be one of the net names in the group. Using a net name that is not in the net group will trigger error when displaying results in the flow.
`-nets {net_list}`	
	Specifies a list of power or ground nets.
`-reset`	Resets all previously created net name(s) with the new grouped net name(s).
`-type [power | ground]`	
	Specifies whether it is a power or ground net.

Examples

- The following example groups the power nets that will be analyzed together as a group called VDD.

```
set_net_group -name VDD -type power -nets { VDD VDD1 VDD2 VDD3 }
set_power_pads -net VDD -format xy -file VDD.pp
analyze_rail -type net VDD
```

Related Information

- `set_rail_analysis_mode`
- `set_power_pads`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_offchip_package_trace

```
set_offchip_package_trace
[-help]
-mapping mapping_file
-name switched_name
-ploc_file_list list_of_bump_location_files
[-reset]
-spice spice_model_name
[-subckt subcircuitname]
```

Specifies to include RDL0 (off-chip package trace) resistance effects during the static and dynamic rail analysis. This flow allows you to analyze switched net bumps that are marked as RDL0 bumps. When this command is specified, the software automatically generates switched net bump current and bump voltage waveforms in the `voltus_rail_pad.tran.ptiavg` file created in the new switch net directory located under the rail state directory.

You can specify multiple `set_offchip_package_trace` commands to define connectivity of different bump nets.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `set_offchip_package_trace` parameter. For a detailed description of the command and all of its parameters, use the man command `man set_offchip_package_trace`.
`-mapping mapping_file`	
	Specifies the name of the mapping file that contains the mapping between the package port and switched net bump names. The format of the file is: `<spice_circuit_port_name> pin <vsrcName_in_switch_net's_ploc>`
`-name trace_model_name`	
	Specifies the trace model name or switched net bump of RDL0.
`-ploc_file_list list_of_bump_location_files`	

	Specifies the name of a file containing switched net names and associated ploc/power pad files. The format of the file is:
	`SWITCHED_NET1 plocfile1`
	`SWITCHED_NET2 plocfile2`
	`…..`
	The format of plocfile (power pad) is:
	`<voltage_source_name> <x> <y> <metal_layer>`
	The format of this file is same as specified for the `set_power_pads -format xy` format.

`-reset`	Resets all the previous `set_offchip_package_trace` command settings to the default settings.
`-spice model_file`	
	Specifies a file containing a SPICE subcircuit (RDL0 netlist or full-chip package netlist) for off-chip package trace.
`-subckt subcircuitname`	
	Defines the top-level subcircuit for off-chip package tracing.

Examples

- The following command specifies to analyze the switched net `VDDau`:

```
set_offchip_package_trace -name VDDau  -ploc_file_list  ploc_file_list.txt -spice pkg.ckt -mapping pkg_mapping.file
```

Related Information

- `set_package`
- `create_die_model`
- "Package Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_package

```
set_package
[-help]
-reset | {-spice model_file
[-mapping mapping_file]
[-offset {x y}]
[-die_instance_name dieinstname]
[-rotate rotation_angle_in_degrees]
[-flip {true | false}]
[-subckt subcircuitname]
[-scale scaling_factor_number]
[-mapping_type { distributed | lumped }]
}
```

Specifies the package model information to be used in power rail analysis.

Parameters

`-die_instance_name` *`dieinstname`*	
	Specifies the die instance name for multi-die analysis. This option is only available in multi-die analysis (`set_rail_analysis_mode -die_mode multi-die`).
`-flip {true | false}`	
	Specifies to flip the die along the Y axis. *Default*: `false`
`-help`	Outputs a brief description that includes the type and default information for each `set_package` parameter. For a detailed description of the command and all of its parameters, use the man command `man set_package`.
`-mapping` *`mapping_file`*	
	Specifies the name of the package terminal mapping file, which contains the mapping of the terminals of the SPICE subcircuit in the package model file (Specified by the `-spice` *`model_file`* parameter) to the locations at the grid or top-level pins. The mapping file formats supported by the software are: • Model Connection Protocol (MCP) • Voltus-specific Format For more information on the mapping file formats, refer to the *Package Model and Mapping File* section in the "File Formats" chapter of *Voltus User Guide*.
`-mapping_type { distributed | lumped }`	

	Specifies the type of mapping to be used in the Voltus-specific terminal mapping file that will be added in the rail analysis flow. The possible arguments are:
	• `distributed` – A distributed mapping file lists out the mapping for each bump on the die to its corresponding package port. The mapping file is specified with the `set_package -mapping` parameter.
	• `lumped` - A lumped mapping file allows for auto-grouping of the die bumps to the lumped package ports. When `lumped` is specified, the software implements a lumped package by internally creating a lumped package model and its mapping file.
	
	The format of the lumped mapping file is:
	`<terminal_name>  <die | vsrc>  <net_name>`
	`vddcore_in  die  vddcore`
	`vddcore_out  vsrc  vddmpu`
	
	In this format:
	◦ the software connects "`<terminal_name> die <net_name>`" to the bumps belonging to `<net_name>` on the die
	◦ the software connects "`<terminal_name> vsrc <net_name>`" to the voltage source of `<net_name>`
	◦ The number of lines in the lumped mapping file are few and are equal to the number of terminals on the package model
	Default: `distributed`

`-offset {x y}`	
	Specifies to adjust and shift the X Y coordinates in the MCP header for each pin. When performing package analysis, the MCP is used to map the terminals of the SPICE subcircuit in the package model file to the locations at the grid or top-level pins in DEF. The die origin specified in MCP must be same as specified in DEF. If the die origin is not the same, you can use the `-offset` parameter to modify the XY location specified in the MCP file.
`-reset`	Resets all the previous `set_package` command settings to the default settings.
`-rotate` *rotation_angle_in_degrees*	
	Specifies the rotation angle of the die. The possible values are: `0/45/90/135/180/225/270/315`. Rotation is always in the counter clockwise direction from the original point. Default value is `0`. This information is required to do coordinate conversion for xy based pin mapping.
`-scale` *scaling_factor_number*	
	Specifies to scale the package model.
`-spice` *model_file*	
	Specifies a file containing a SPICE subcircuit describing the package model.
`-subckt` *subcircuitname*	
	Defines the top level subcircuit in package files.

Related Information

- `create_die_model`
- "Package Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_pg_nets

```
set_pg_nets
[-help]
-net net_name
-voltage value
[-threshold value]
-tolerance value
-force
[-die_instance_name dieinstname]
[-pkg_net_name package_net_name]
[-lib_binding_voltage value]
[-vu_limit value]
```

Specifies the power grid nets. Using this command, you can specify power/ground net voltages in the static and dynamic power analysis flow, and the rail analysis flow.

For the power analysis flow, you must specify the `set_pg_nets` command before `report_power`.

Note: When specifying a ground net with a non-zero nominal operating voltage value, a threshold greater than the ground value must be specified in order for the net to be treated as the ground net.

Parameters

`-die_instance_name dieinstname`	
	Specifies the die instance name for multi-die analysis. This option is only available in multi-die analysis (`set_rail_analysis_mode -die_mode mult`
`-force`	Skips the validity check on power nets.
`-help`	Outputs a brief description that includes the type and default information for each `set_pg_nets` parameter. For a detailed description of the command and all of its parameters, use the man command `man set_pg_nets`.
`-lib_binding_voltage value`	
	Specifies the voltage to be used for library binding in power analysis. This parameter binds the instances to the library that is closest to the user-sp voltage.
`-net net_name`	
	Specifies the name of the power or ground net that you want to analyze.
`-pkg_net_name package_net_name`	
	Specifies the net name on the package side. By default, the package net name is same as the net name.
`-threshold value`	
	Specifies the minimum allowed voltage on the power net or the maximum rise on the ground net, depending on the setting of the `-voltage` parame parameter is used to verify that the power grid passes the limit check and to set the filters for an IR plot.
`-tolerance value`	
	Specifies the tolerance range of the voltage value. Specify a value from 0 to 1 (not including 1). A value of 0.2 results in a range of "voltage +20%" *Default*: 0.3.
`-voltage value`	
	Specifies the voltage on the power rail to be analyzed.
`-vu_limit value`	

	Specifies the voltage drop limit for a net. This parameter allows you to specify the voltage threshold value for each net separately. The specified tl the rail reports, histograms, and plots. If the threshold value is not specified for some of the nets being analyzed, the default range will be applied to *Default Range*: 0mV to max VU

Examples

- The following command specifies a `VDD_TDSPCore` value of 0.9 volts with a threshold of 0.89V and a tolerance of 0.3:

  ```
  set_pg_nets -net VDD_TDSPCore -voltage 0.9 -threshold 0.890 -tolerance 0.3
  ```

- The following command specifies to use the library binding voltage for binding the libraries to the instances instead of "`-voltage`":

  ```
  set_pg_nets -net vdd -voltage 0.90 -lib_binding_voltage 0.75
  ```

Related Information

- `set_rail_analysis_mode`
- `set_rail_analysis_domain`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_power_data

```
set_power_data
[-help]
-reset | {-format {current | ascii | area | ascii_current}
[-instance instance_name]
[-offset offset_value]
[-power value]
[-scale factor]
[-repeat time]
[-bias_voltage value]
[-die_instance_name dieinstname]
[-power_output_directory dir]
[-repeat_window {start end}]
[-strip_hier_instance instance]
[filename]
}
```

Specifies the static and dynamic power or current files for IRdrop and EM analysis. The power data for the design can be generated using the `report_power` command. This command also has ability to read external ASCII static power files to do static rail analysis. For dynamic analysis, you must use the `report_power` command to generate dynamic current files.

This command should be specified prior to the `analyze_rail` command. You must specify the `set_power_data` command after the `report_power` command in the flow script if power data is not available.

Parameters

`-bias_voltage value`	
	Specifies the value of the bias voltage. This parameter must be set if -format is set to ascii or area.
`-die_instance_name dieinstname`	
	Specifies the die instance name for multi-die analysis. This option is only available in multi-die analysis (`set_rail_analysis_mode -die_mode m`
filename	Specifies the name of the power data files. The file is optional if -format area is specified.
`-format {current | ascii | area | ascii_current}`	
	Specifies the format of the power data. The power data format can be `current, ascii`, or `area`. The parameter `-bias_voltage` must be set if `area` or `ascii` is specified. • When using the current format option, specify the current file (*.ptiavg) that is generated by the power analysis software within Voltus . For ex `set_power_data -format current static_VDD.ptiavg`

	• When using the `ascii` format option, specify the ascii instance based power file, generated by PowerMeter or CPE software within Voltus. F
	`set_power_data -format ascii -bias_voltage 1.08 powermeter.pwr`
	Note: The command accepts an ASCII power file in three-column format consisting of *instance/cell name*, *power* in W,and *power pin name*. are treated as comment lines.
	The format of the three-column input file is as follows:
	`<instance/cell name> <power in Watts> <power pin name>`
	Example:
	`top/inst1 0.001 vdd`
	`top/inst2 0.001 vdd1`
	`top/inst3 0.003 vdd3`
	where vdd, vdd1, and vdd3 are power pin names of the instance
	ⓘ Although `-bias_voltage` is a required argument with `-format ascii`, when 3-column power file format is specified, the program will ignore rely on associated net voltages to determine current for the pins.

	• When using the `area` format option, specify the total power of the chip in Watts. This power is distributed based on cell area. Optionally, you for macro cells using a file, for example, `set_power_data -format area -bias_voltage 1.08 -power 10 cell_power.file` where, `-bias_voltage` is the voltage at which power was characterized; If the analysis voltage is different, this power will be scaled accordingly. `cell_power.file` is in two column format with *`cell_name`power_in_watts*.
	• When using the `ascii_current` format option, specify an ASCII instance current file for static analysis. The format of the three-column input fi *`<instance/cell name> <current in Ampere> <power pin name>`*
`-help`	Outputs a brief description that includes the type and default information for each `set_power_data` parameter. For a detailed description of the command and all of its parameters, use the man command `man set_power_data`.
`-instance` *`instance_name`*	
	Specifies the hierarchical instance name for which the current files are included. This option is useful when power is calculated for hierarchical partitions of the design, enabling you to read power for the hierarchical partitions For example, `set_power_data -format current -instance xy {dynamic_VDD.ptiavg dynamic_VSS.ptiavg}`
`-offset` *`offset_value`*	
	Specifies the start time offset for current waveforms saved inside the current files. The default unit is in seconds; However, you can also specify the unit with the offset time value. This option is useful in the hierarchical flow to align current waveforms of various partitions. It can be also used to shift power-up rush current w UltraSim software. For example, `set_power_data -format current -offset 1ns -instance xy {dynamic_VDD.ptiavg dynamic_VSS.ptiavg}`
`-power` *`value`*	
	Specifies the total power of the chip in Watts and is used with -format area option.
`-power_output_directory` *`dir`*	
	Specifies the name of the directory that contains the current files required to run power analysis in parallel with rail analysis.

`-repeat time`	
	Repeats the dynamic current waveform for the specified time. You must ensure that `set_dynamic_rail_simulation -stop` is also specified and duration. This option is only available for current format files during dynamic analysis. The repeat option can be used when multiple current files periods or when IR drop analysis needs to be run for longer duration. See example below. *Default units*: ns.
`-repeat_window {start end}`	
	Specifies to extract the given window from the current waveform and repeat it. You can pick a window such that the current at the `start_time` ar parameter allows you to identify and repeat windows where the start and end time have similar values to ensure that the repeated waveform is no high current values. The unit for the start and end time is ns. The following command specifies that the pattern between {20ns 80ns} will be repeated until simulation stops: `set_power_data -repeat_window {20ns 80ns}`
`-reset`	Overwrites previously specified current data.
`-scale factor`	
	Specifies a scale factor to apply to the current data. You can use this parameter to scale currents to a more optimistic or pessimistic power-consumption estimate. This parameter is only for `-format current` and `-format ascii`.
`-strip_hier_instance instance`	
	Specifies to use power generated from the top-level rail analysis for block-level rail analysis. This parameter allows you to specify the top-level for which the current files are to be included. When the parameter is specified, the tool removes the prefix from the top-level and reads the top-le during the block-level analysis. The parameter is useful when top-level power data is already available, therefore, you do not need to re-run blo

Examples

- The following command specifies a format of type current and a scale of 1 for the three given files.:
  ```
  set_power_data -format current -scale 1 {static_VDD_TDSPCore.ptiavg \ static_VDD_TDSPCore_R.ptiavg static_VSS.ptiavg}
  ```
- The next example specifies two current data files. The first file specifies vdd_vdd.ptipeak. The second one specifies dynamic_VDD.ptiavg and this current data will be repeated every 100ns, which is the stop time specified for the dynamic analysis.
  ```
  set_power_data -format current vdd_vdd.ptipeak
  set_power_data -format current -repeat=100ns dynamic_VDD.ptiavg
  set_dynamic_rail_simulation -stop 100ns
  analyze_rail ...
  ```

Related Information

- `set_rail_analysis_mode`
- `set_dynamic_rail_simulation`
- `analyze_rail`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_power_pads

```
set_power_pads
-reset |{
[-auto_voltage_source_creation {true|false}]
[-def_shape_only {true | false}]
[-die_instance_name dieinstname]
[-enable_switchnet_ploc_generation {true | false}]
[-format {defpin | padcell | xy | boundary | xyiv}]
[-layer vialayername]
-net net_name
[-number_of_voltage_source value]
[-file file]
[-region {x1 y1 x2 y2}]
[-short_pin_nodes {true | false}]
[-tile {x y}]
}
```

Specifies power pad information.

You can specify multiple power pad formats (`xy` coordinates, pad cell, or power pin) for the same power net, in both, net-based and domain-based rail analysis.

Note: All voltage source names in the power pad location file must be unique for all nets in the domain, e.g. if voltage source name vsrc1 is used in pad location file of net VDD as well as VSS in domain based analysis, program will issue error and exit. In addition, the names of voltage sources in the power pad location file must be different from the names in the DEF pin section.

Parameters

`-auto_voltage_source_creation {true|false}`	
	Specifies to automatically create voltage sources either for all nets (`-net ALL`) or for a specific net (`-net net_name`). The voltage sources are created on the top metal of all via cuts located either on the user-specified via layer or the design top via layer (default). Default: `false`.
`-def_shape_only {true | false}`	
	Specifies to generate voltage sources only on the DEF routing shapes. This parameter controls the generation of voltage sources on the cell instance shapes. When this parameter is set to `false`, voltage sources are created for both the DEF routing shapes and shapes inside cell instances. This parameter is specified with the `-auto_voltage_source_creation` parameter. Default: `true`
`-die_instance_name dieinstname`	
	Specifies the die instance name for multi-die analysis. This option is only available in multi-die analysis (`set_rail_analysis_mode -die_mode multi-die`).
`-enable_switchnet_ploc_generation {true | false}`	
	Specifies to generate voltage sources on all switched nets. This parameter is specified with the `-auto_voltage_source_creation` parameter. Default: `false`
`-file file`	Specifies the name of the file containing the voltage source information. Not required for `-format defpin` option.
`-format {defpin | padcell | xy | boundary | xyiv}`	
	Specifies how the voltage source is determined. Required for `xy`, `padcell`, or `boundary`.
	`defpin` - Specifies that voltage sources will be determined by power pins.
	`padcell` - Specifies that voltage sources will be determined by pad cells.

	`xy` - Specifies that voltage sources will be specified by x y coordinates. You can also specify a PWL function on the X,Y location for a voltage source. When specified, Rail Analysis will read the power pad file and use the PWL waveform for the voltage source.
	Example:
	`set_power_pads -format xy -file <xy_plocFile>`
	
	Add the PWL option in the xy file:
	`*vsrc_name x y layer_name [pwl=]`
	`vsrc3 8.39 370.35 Metal5 pwl=(t1,v1,t2,v2,t3,v3…)`

	`boundary` - Specifies that voltage source locations will be used for instances in the design.
	`xyiv -` Specifies whether the voltage sources specified in the xy power pad location file are current source or reference voltage source. The format is the same as the xy file, except the additional first and last columns to indicate the current/voltage source and current/voltage value.
`-layer` *`vialayername`*	
	Specifies the via layer on which the voltage sources are to be generated. If this parameter is not specified, the voltage sources will be generated on the top via layer of the design. This parameter is specified with the `-auto_voltage_source_creation` parameter.
`-net` *`net_name`*	Specifies the name of the power net.
`-number_of_voltage_source` *`value`*	
	Specifies the number of voltage sources to be generated. You can adjust either `-tile` or `-number_of_voltage_source` to reduce the number of voltage sources created. This parameter is specified with the `-auto_voltage_source_creation` parameter. Default: `1 million`.
`-region {`*`x1 y1 x2 y2`*`}`	
	Generates the voltage sources in the specified region. If this parameter is not specified, the software uses the entire design/layout size as the region. This parameter is specified with the `-auto_voltage_source_creation` parameter.
`-reset`	Resets all the previous `set_power_pads` commands
`-short_pin_nodes {true | false}`	
	Specifies to short all interface nodes on a pad cell pin to a single node and create voltage source for that node. ***Default***: `false` **Example**: `set_power_pads -net vdd -format padcell -short_pin_nodes -file pad_cell.file`
`-tile {`*`x y`*`}`	
	Defines the number of tiles in the X and Y direction for automatic creation of voltage sources. When `-tile` is specified, the area (`-region` or default design area) will be tiled vertically and horizontally per the given tile size (x*y). For each tile, one voltage source will be created on the top metal of the biggest via array. If `-tile` is not specified, the software creates voltage sources on the top metal of all via cuts located either on the user-specified via layer or the top via layer of the design. You can adjust either `-tile` or `-number_of_voltage_source` to reduce the number of voltage sources created. The `-tile` parameter is specified with the `-auto_voltage_source_creation` parameter.

Examples

- The following command specifies the power pad information for the `VDD_TDSPCore_R` net in the file `dtmf_chip.VDD_TDSPCore_R.vsr`c. The voltages sources are specified by x y coordinates.
  `set_power_pads -net VDD_TDSPCore_R -format xy -file dtmf_chip.VDD_TDSPCore_R.vsrc`
- The following command specifies `xy`, `padcell`, and `defpin` formats for a single net (`VDD`) as follows:

```
set_power_pads -net VDD -format xy -file VDD.pp
set_power_pads -net VDD -format padcell -file VDD.padcell
set_power_pads -net VDD -format defpin
```

- The following command specifies power pad information using a combination of user-specified and automatic voltage source creation methods:
  # Specific setting for net VSS using automatic voltage source creation
  ```
  set_power_pads -net VSS -auto_voltage_source_creation true -layer VIA_6
  ```

  # Global setting for all nets using automatic voltage source creation
  ```
  set_power_pads -auto_voltage_source_creation true -net ALL -layer VIA_5
  ```

  # User-specified power pad file for net `VDD_AO`, therefore automatic voltage source creation feature will not be enabled for this net
  ```
  set_power_pads -net VDD_AO -format padcell -file padcell.file -short_pin_nodes true
  ```

Example of power pad location file with XY coordinates for net VDD

```
*-------------------------------------------------------------------------------------------------------------------------------------------------------
----
* XY power pad location file
* vsrc name X(um) Y(um) LEF/Tech Layer <R=value ohm> <C=value F> <L=value H>
* Package R,L and C parameters are optional
*-------------------------------------------------------------------------------------------------------------------------------------------------------
-----
VDD100 5.000 793.500 M7
VDD101 1187.835 293.500 M7
VDD102 695.000 5.000 M6
VDD103 1187.835 493.500 M7
VDD104 595.000 1293.800 M6
VDD105 595.000 5.000 M6
```

Example of pad cell file for VDD net,

```
*-------------------------------------------------------------------------------------------------------------------------------------------------------
----
* Pad cell file for Voltus
* cellname <r=value> <l=value> <c=value>
* Package parameters are optional
*-------------------------------------------------------------------------------------------------------------------------------------------------------
-----
PVDD1 r=2.5e-3 l=2.5e-11 c=120e-12
PVDD2 r=2.5e-3 l=2.5e-11 c=120e-12
```

Related Information

- `set_rail_analysis_mode`
- `set_net_group`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_rail_analysis_domain

```
set_rail_analysis_domain
[-help]
-name domain_name
-pwrnets power_net_list
-gndnets ground_net_list
[-threshold value]
```

Specifies a rail analysis power domain.

Note: For a power-gated domain, you should only specify the "always on" net. The switched net should not be specified as part of power domain definition, because it is traced by the program automatically.

Parameters

`-gndnets ground_net_list`	
	Specifies a list of ground nets that are in the domain.
`-help`	Outputs a brief description that includes the type and default information for each `set_rail_analysis_domain` parameter. For a detailed description of the command and all of its parameters, use the man command `man set_rail_analysis_domain`.
`-name domain_name`	
	Specifies the name of the power domain.
`-pwrnets power_net_list`	
	Specifies a list of power nets that are in the domain.
`-threshold value`	
	Specifies the minimum allowed voltage on the power net or the maximum rise on the ground net, depending on the setting of the`-voltage` parameter. The `-threshold`parameter is used to verify that the power grid passes the limit check and to set the filters for an IR plot. **Note:** This parameter does not support domains with MSMV power nets. If you have specified both net (`set_pg_nets -threshold`) and domain-based (`set_rail_analysis_domain -threshold`) thresholds, the net-based threshold will be honored. The valid value for threshold is between 0 and 0.5 (50%). For example, `set_rail_analysis_domain -threshold 0.1` indicates that 0.1 is 10% IRdrop. If a power net in the domain has nominal voltage of 1v, 10% of threshold will translate to 100mV of maximum effective IRdrop between the power and ground net.

Examples

- The following command sets the power domain `TDSP` to include `VDD_TDSPCore_R` and `VSS` nets:
  ```
  set_rail_analysis_domain -name TDSP -pwrnets VDD_TDSPCore_R -gndnets VSS
  ```

Related Information

- `set_rail_analysis_mode`
- `set_pg_nets`
- `analyze_rail`
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_rail_analysis_mode

```
set_rail_analysis_mode
-method {static | dynamic | era_static | era_dynamic}
-accuracy {xd | hd }
-power_grid_library dir_list
[-analysis_view view]
[-avg_em_analysis {true | false}]
[-off_rails net_name_list]
[-em_models file]
[-em_temperature string ]
[-default_package_resistor value]
[-default_package_inductor value]
[-default_package_capacitor value]
[-vsrc_search_distance value]
[-report_msmv_format {true | false}]
[-temp_directory_name directory]
[-gds_purpose {metalFill | flipChip | fullChip}]
[-gds_file file [-gds_offset {x y}]]
[-gds_top_cell cell_name ]
[-decap_cell_list { cell1 cell2 ... celln }]
[-filler_cell_list { cell1 cell2 ... celln }]
[-suppress_message {message_id + } ]
[-disable_analysis_types {list of analysis types}]
[-gds_map file]
[-enable_sensitivity_analysis {true | false}]
[-cell_ignore_file filename]
[-save_voltage_waveforms {true | false}]
[-read_thermal_map thermal_map_file]
[-temperature value]
[-extractor_include filename]
[-record_results_start_time time]
[-lef_map lefMappingFile]
[-lifetime value]
[-die_mode {single|multi-die|3dic} -die_instance_name dieinstname [-design designname]]
[-dynamic_trigger_file filename]
[-block_powerup_rail netname | -powering_up_rails net_name1... netname_n]
[-process_techgen_em_rules {true | false}]
[-powerup_sequence_file filename]
[-enable_vsrc_in_gif {true | false}]
[-work_directory_name directory]
[-powerup_fast_mode {true | false}]
[-ignore_shorts {true | false}]
[-disable_parallel_extraction]
[-enable_distributed_processing_in_solver {true | false}]
[-extraction_tech_file filename]
[-generate_hier_bbv {true | false}]
[-check_vsrc_placement_on_switched_net {true | false}]
[-process_bulk_pins_for_body_bias {true | false}]
[-cluster_via_rule { {via_layer1number_of_equidistant_vias}… }]
[-cluster_via1_ports {true | false}]
[-ignore_fillers {true | false}]
[-ignore_decaps {true | false}]
[-limit_number_of_steps {true | false}]
[-check_thermal_aware_em {true | false}]
[-compress_powergrid_database {true | false}]
[-cluster_via_size value]
[-report_power_in_parallel {true | false}]
[-use_early_view_list filename]
[-use_ir_view_list filename]
[-use_em_view_list filename]
[-enable_manufacturing_effects {true|false}]
[-enable_rlrp_analysis {true|false}]
[-enable_voltage_across_vias {true|false}]
[-eiv_eval_layers {layerlist}]
[-eiv_eval_window {switching | timing | both | elapse}]
[-eiv_method {worst best avg worstavg bestavg}]
[-reuse_state_directory dir_name]
```

```
[-snap_layer_for_current_taps file_name]
[-enable_scheduler {true | false}]
[-era_current_distribution_factor_for_placed value]
[-era_current_region_file filename]
[-era_current_distribution_layer layer_name]
[-era_current_distribution { unplaced | placed | all | none }]
[-era_insert_virtual_via_on_layers  value ]
[-era_lef_layermap filename]
[-era_power_gate_file filename]
[-era_insert_virtual_followpins { standard | extended | none }]
[-era_skip_virtual_via_by_type {whatif | def | all | none}]
[-era_skip_virtual_via_on_layers {{ layer1layer2 } { layer3layer4 } ... }]
[-import_what_if_shapes {true | false}]
[-what_if_shapes_file filename]
[-check_current_balanced_power_grid_em {true | false}]
[-em_threshold value]
[-gif_resolution {low | medium | high}]
[-era_insert_virtual_followpin_for_io {true | false}]
[-report_voltage_drop {true | false}]
[-eiv_report{auto | netonly | all}]
[-eiv_thresholdvalue]
[-eiv_max_instancesvalue]
[-prechain_powerup_sequence_filefilename]
[-rdl_def def_file]
[-rdl_placement{X Y}]
[-rdl_orientation{ N|S|W|E|FN|FS|FE|FW }]
[-topcell_placement{X Y}]
[-topcell_orientation{ N|S|W|E|FN|FS|FE|FW }]
[-watch_location_waveform { {layerNamexCoordyCoord}+ }]
[-enable_rc_analysis{true | false}]
[-em_peak_analysis {true | false}]
[-em_limit_scale_factor {{avg value} {rms value} {peak value}}]
[-em_rms_delta_t temp]
[-ict_em_models file]
[-hpgv_block_lefs list_of_files]
[-hpgv_generate_view {ir | em | all}]
[-era_current_distribution_unplaced_area {instance | diearea}]
[-era_current_distribution_nets {net1net2 ...}]
[-era_check_wires_for_generated_current_regions {true | false}]
[-era_techlib_generation {true | false}]
[-force_library_merging {true | false}]
[-unconnected_die_pkg_pins {ignore | error | edit}]
[-mcp_model_mapping {{<mcp model name1> <die DEF name1>} {<mcp model name2> <die DEF name2>} ...}]
[-eiv_pin_based_report {true | false}]
[-gif_zoom_area { x1y1x2y2 }]
[-gif_zoom_topcell_diearea {true | false}]
[-generate_instance_ir_report file]
[-generate_instance_pin_ir_report file]
[-finegrain_powergate_ron {min avg max}]
[-finegrain_powergate_ron_list filename]
[-ignore_incomplete_net {true | false}]
[-generate_combined_ivd_gif {true | false}]
[-gif_iv_thresholdvalue]
[-verify_logical_connectivity{true | false}]
[-enable_ccs_analysis{true | false}]
[-rlrp_threshold value]
[-rlrp_percentage_threshold value]
[-rlrp_pin_based_report {true | false}]
[-probe_waveform_list filename ]
[-probe_pin_voltage_list filename]
[-eiv_eval_nodes{tap | port}]
[-hier_delimiter character]
[-force_extraction {true | false}]
[-skip_extraction {true | false}]
[-report_power_options option_names]
[-package_trace_connectivity{true | false}]
[-reff_pin_report_method { best | worst } ]
[-rlrp_pin_report_method { best | worst | eiv_best | eiv_worst} ]
[-reff_pin_report_layer { top | bottom | all }]
```

```
[-rlrp_pin_report_layer { top | bottom | all }]
[-probing_node_file file_name]
[-enlarge_vsrc_in_vuvc {true | false}]
[-gif_new_color_scale {true | false}]
[-scale_initial_condition_current filename]
[-probe_instance_tap_waveforms {true | false}]
[-rlrp_eval_nodes {port | tap}]
[-report_shorts{true | false}]
[-extract_subconductor_layers {true | false}]
[-eiv_average_per_window_list filename]
[-em_ignore_pgv_resistors {true | false}]
[-eiv_eval_gnd_window {true | false}]
[-honor_negative_static_current {true | false}]
[-accumulate_overlapping_ddv_pwl_waveforms {true | false}]
[-report_layers_above_pin_for_instance_ir {true | false}]
[-switchbit_file_for_eiv_calculationfilename]
[-rlrp_detail_report{true | false}]
[-era_instance_dynamic_current_filefilename]
[-eiv_print_time{true | false}]
[-eiv_histogram_max value]
[-eiv_histogram_min value]
[-eiv_histogram_number_of_bucketvalue]
[-reff_detail_report {true|false}]
[-record_inst_peak_current {true | false}]
[-pre_simulation_period value]
[-pre_simulation_resolution value]
[-fine_pre_simulation_period value]
[-enable_instance_powergate_report {true | false}]
[-disable_em_split_ac_dc_rules {true | false}]
[-lowest_layer_for_em_check layer_name]
[-ignore_nets_without_vsrc {true | false}]
[-eiv_pin_location{true | false}]
[-verbosity{true | false}]
[-enable_reff_analysis {true | false}]
[-rdl_def_list {{rdl_filename1x1y1orient1} {rdl_filename2x2y2orient2} ...}]
[-remove_duplicate_inst {true | false}]
[-generate_instance_effr_report file]
[-generate_package_pin_current_voltage {true | false}]
[-reff_domain_report_format {1 | 2 | 3 | 4}]
[-em_report_line_threshold value]
[-powering_down_rails net_name1... netname_n]
[-flatten_xpgv_block_instancesfilename ]
[-xpgv_config_file filename]
[-def_based_hierarchical_reports {true | false}]
[-enable_2d_partition_extraction{true | false}]
[-print_gif_range_percentage{true | false}]
[-generate_multi_voltage_library {true | false}]
[-feol_scale_factor non_negative_integer_number]
[-enable_seb {true | false}]
[-env_temperature temperature]
[-seb_lifetime value]
[-seb_table table_filename]
[-seb_temperature temperature]
[-use_rms_delta_t {true | false}]
[-enable_xp {true | false}]
[-xp_cpu_per_job_power value]
[-xp_cpu_per_job_simulation value]
[-xp_host_allocation_method {on_demand | at_startup}]
[-xp_purge {full}]
[-xp_resume {true | false}]
[-xp_reuse_extraction_directory directory_name]
[-xp_simulation_cpu_timeout value]
[-xp_simulation_min_cpu value]
[-enable_package_battery_current_probing{true | false}]
[-probe_package_interface_ports{true | false}]
[-static_multi_mode_analysis {true | false}]
[-em_temperature_layer_list {{<LEF_layer_name> <em_temperature_in_C>}+}]
[-em_rms_delta_T_layer_list {{<LEF_layer_name> <delta_T_in_C>}+}]
```

```
[-report_instances_missing_current_data{true | false}]
[-common_res_inst_pair_file_namefile]
[-promote_pin_shapes_only {filler | decap | both} ]
[-unconnectedcell_ignore_filefilename]
[-ignore_duplicate_vsrc {true | false}]
[-write_demand_current_region_filefilename]
[-shorting_resistancevalue]
[-probe_instance_pin_waveforms{true | false}}
[-reset]
[-tsv_subckt_modelfile_list filename]
[-em_cdf_percentage value]
[-ircx_models {RC.ircx EMIR.ircx}|{RC.ircx}]
[-enable_multi_die_connectivity_check {true | false}]
[-ignore_from_reporting filename]
[-enable_xpgv_scaling { true | false }]
[-static_trigger_file filename]
[-rlrp_cell_report_layer filename]
[-peak_em_analysis {true | false}]
[-rms_em_analysis {true | false}]
[-reff_eval_nodes {port | tap}]
[-reff_report_all {true | false}]
[-set_analyze_bbox {xmin ymin xmax ymax}]
[-override_set_power_data {true | false}]
[-unified_power_switch_flow {true | false}]
```

Specifies how the rail analysis will be performed.

Parameters

`-accumulate_overlapping_ddv_pwl_waveforms {true | false}`	
	Specifies to accumulate the current values of Dynamic Detailed View (DDV) PWL waveforms from different trigger events when the PW are overlapping with each other. If set to `true`, the current values of the overlapping PWL waveforms are added together for rail analysis. The default value is `false`.
`-accuracy {xd | hd }`	
	Specifies the accuracy mode of analysis. *Default* : `hd`
	• `xd`: xd or accelerated definition accuracy mode is used for early implementation stage IR/EM analysis.
	• `hd`: hd or high definition accuracy mode is used for final verification stage IR/EM analysis.
`-analysis_view view`	
	Specifies analysis view created in a CPF (Common Power Format) file. Optional.
`-avg_em_analysis {true | false}`	
	Specifies to calculate the average EM in the dynamic rail analysis run. This parameter generates the `<net_name>.rj.avg.rpt` report. *Default* : `false`
`-block_powerup_rail netname`	
	Specifies a net for block power-up analysis. You can specify only one net for block power-up analysis at a time.
`-cell_ignore_file filename`	
	If specified, excludes all references to the named cells in the specified file, from the design database.
`-check_current_balanced_power_grid_em {true | false}`	

	Specifies to check the power-grid EM rules (power-rail terminal-via EM rule). This parameter allows EM comparison with/without this E is only applicable in TSMC N20 process nodes where the EM limit may be relaxed if certain constraints are met. Refer to TSMC DRM f
`-check_thermal_aware_em {true | false}`	
	Specifies to use the thermal map for EM checks. The thermal map file is specified using the `-read_thermal_map` parameter. The Rail Ar temperature specified in the thermal map file for EM analysis. EM limit has a strong temperature dependency. For a chip that has substa variation across the chip, the Rail Analysis engine can use the temperature specified in the thermal map file to perform accurate EM an
`-check_vsrc_placement_on_switched_net {true | false}`	
	Specifies to check voltage source placement on switched nets. When this parameter is set to `true`, the software checks whether the specified voltage source location file places any voltage sources o the power-gated domain, and disables voltage source placement on the switched net. It also detects shorts and generates an error mes
`-cluster_via1_ports {true | false}`	
	Designs with follow-pin routing on M1 and M2 layers have standard cells library LEF with M1, M2, and VIA1 ports. This parameter spec ports for such cells in order to improve overall extraction and rail analysis performance. *Default* : `true` This parameter is set to `true` by default to improve VIA1 port handling in designs with M1 and M2 follow-pin routing. This parameter giv performance by clustering equidistant VIA1 ports defined in cell LEF without significant loss of accuracy. However, if you specify a clus specification takes precedence over automated VIA1 clustering.
`-cluster_via_rule { {`*`via_layer1 number_of_equidistant_vias`*`}...}`	
	Controls the number of vias to cluster on a layer basis. The VIA clustering rule specified using this parameter will override the default cl given accuracy mode. The following command will cluster 100 equidistant VIA1 cuts, 200 equidistant VIA2 cuts, and 300 equidistant VIA7 cuts: `set_rail_analysis_mode -cluster_via_rule { {VIA1 100} {VIA2 200} {VIA7 300}}` The rest of the VIAs will be clustered using the default clustering rule depending upon the rail analysis accuracy mode.
`-cluster_via_size` *`value`*	
	Specifies the size of the via array to cluster all via layers except VIA1 (layer between Metal1 and Metal2). The default value is `625` (25x25 via array), except when running static analysis using HD mode. The default for static HD mode is `16` (4x preserve more accuracy for EM analysis. *Default* : `625`
`-common_res_inst_pair_file_name` *`file`*	

	Specifies to calculate common resistance along the least resistance path (RLRP) between a pair of cells having the same voltage sour allows you to check RLRP along cell instance pairs from multiple voltage source locations.
	The format of the specified file is:
	`inst_name1 inst_name2`
	`inst_name3 inst_name4`
	`...`
	The `-common_res_inst_pair_file_name` parameter generates a report `<`*`net_name`*`>.common_res` in the following location:
	`<`*`output_dir`*`>/<`*`state_dir`*`>/Reports/<`*`net_name`*`>/`
	The format of the report is:
	`inst_name1 inst_name2 common_res1 pad1`
	`inst_name3 inst_name4 common_res2 pad2`
	`...`
	The following is a snippet from the report:
	`inst_name1 inst_name2` 13.4315 PAD_VDD2

`-compress_powergrid_database {true | false}`	
	Specifies to compress the output files generated by extraction and rail analysis to save disk space usage. This parameter reduces disk from the extraction and rail analysis stages at approximately 5-10% performance penalty. *Default* : `false`

`-decap_cell_list { cell1 cell2 ... celln }`

Specifies a list of decap cells. This option is used when the decap cells are not tagged in the power-grid library. A cell characterized as
power-grid view generation with LibGen does not need to be specified.

For dynamic only.

`-def_based_hierarchical_reports {true | false}`

Specifies to generate separate rail reports for each DEF instance. The net-based effective instance voltage, RLRP, and effective resista
generated hierarchically.

Reports will be generated for every net in each DEF hierarchy using the following directory structure:
`<output_dir>/<state_dir>/Reports/HIER/hier_<HIER_ID>/<domain_name | net_name>.<report_type>`

In the `Reports` folder, the Rail Analysis engine will generate a hierarchical map (`def_hierarchy.map` file) during DEF parsing to associa
with each DEF instance, as shown below:

```
#hier_id   hier_name   #def_name

   0        chip        chipCore
```

Default : `false`

`-default_package_inductor value`

Specifies the default inductance value of the package resistance-inductance-capacitance (RLC) model to use when you specify voltag

If the power-grid views contain area-based voltage source locations, the following equation is used to define the inductance applied to

```
Li = L * number_of_sources
```

where:

`Li` is the inductance on an individual voltage source.

`L` is the default inductance set by `-default_package_inductor`.

`Number_of_sources` is the number of voltage sources in the power-grid view.

The default inductance value is specified in `henrys`.

`-default_package_resistor value`

Specifies the default resistance value of the package resistance-inductance-capacitance (RLC) model to use when you specify voltage

If the power-grid views contain area-based voltage source locations, the following equation is used to define the resistance applied to t

```
Ri = R * number_of_sources
```

where:

`Ri` is the resistance on an individual voltage source.

`R` is the default resistance set by `-default_package_resistor`.

`Number_of_sources` is the number of voltage sources in the power-grid view.

The default resistance value is specified in `ohms`.

`-default_package_capacitor value`

Specifies the default capacitance value of the package resistance-inductance-capacitance (RLC) model to use when you specify voltag

If the power-grid views contain area-based voltage source locations, the following equation is used to define the capacitance applied to

```
Ci = C/number_of_sources
```

where:

`Ci` is the effective capacitance on an individual voltage source.

`C` is the default capacitance set by `-default_package_capacitor`.

`Number_of_sources` is the number of voltage sources in the power-grid view.

The default capacitance value is specified in `farads`.

`-design designname`

	Specifies the DEF or GDS design name for multi-die analysis. This option is only available in multi-die analysis (`-die_mode multi-die`)
`-die_mode {single|multi-die|3dic}`	
	Specifies whether single or multi-die analysis will be performed. The `3dic` option is used for performing multi-die rlrp analysis. *Default* : `single`
`-die_instance_name` *dieinstname*	
	Specifies the die instance name for multi-die analysis. This option is only available in multi-die analysis (`-die_mode multi-die`).
`-disable_analysis_types` { *list of analysis types* }	
	Disables specific analysis types during rail analysis. For example, `set_rail_analysis_mode -disable_analysis_types { rlrp vu pi pv }` disables `rlrp`, `vu`, `pi`, and `pv` analysis types du By default, the disable list is empty. The following analysis types can be disabled: `ir tc iv vc dd pi pv javg rj jrms rc node_effr`
`-disable_em_split_ac_dc_rules {true | false}`	
	Specifies to use the same AC/DC rules for the EM analysis of the signal and PG nets. When this parameter is set to `true`, the software handling of the AC/DC rules. By default (when set to `false`), the software uses separate EM rules for the signal and PG nets.
`-disable_parallel_extraction`	
	Specifies to use a single processor for extracting resistance or capacitance. If this parameter is not specified, the software uses parallel extraction.
`-dynamic_trigger_file` *filename*	
	Specifies a dynamic trigger file. When a Macro EM view with current signature is used in a design, it needs to be determine when this b The dynamic trigger file is used to determine this. At the trigger time, the dynamic current waveforms of the EM view block will be applie **Note**: If you specify trigger for cells or instances using this parameter, the tool would give higher precedence to this user-defined trigger generated trigger files in the power analysis directory (`power_output_dir/trigger*.txt`).
`-eiv_average_per_window_list` *filename*	

	Specifies to generate a report containing different sets of Effective Instance Voltage (EIV) for each timing/switching window for a list of
	The format of the specified file is:
	`INST xxx`
	`CELL xxx`
	The report contains a set of average EIVs for each instance based on the timing/switching window in the current files. The report file na `name.win_avg.rpt`.
	Example:
	`set_rail_analysis_mode -eiv_average_per_window_list ./clock_network.insts`
	File format:
	`INST clk__L2_I1`
	`INST clk__L11_I2`
	`INST clk__L13_I4`
	`INST clk__L13_I3`
	`INST clk__L13_I2`
	Output EIV report format:
	`#INST_NAME START_TIME:END_TIME EIV_AVG`
	`clk__L15_I4 1.32e-09:1.78e-09 1.06354 6.38e-09:6.8e-09 1.06217`
	`clk__L15_I3 1.32e-09:1.68e-09 1.06273 6.38e-09:6.72e-09 1.06854`
	`...`

`-eiv_eval_gnd_window {true | false}`	
	Specifies that the ground net switching window should also be considered for the EIV computation. The switching windows from the pc will be merged into a single switching window.
`-eiv_eval_layers {`*layerlist*`}`	

	Specifies to report the EIV of nodes only on the specified layers. When this parameter is not specified, the software reports the EIV valu across all layers. The parameter allows you to analyze the EIV values for the user-defined layers. **Example**: `set_rail_analysis_mode -eiv_eval_layers B_M0 -eiv_eval_nodes port` This command reports only the EIV values of nodes on the `B_M0` layer.
`-eiv_eval_nodes {tap | port}`	
	Specifies to report EIV based on the instance nodes (tap or port). By default, the software uses the tap nodes for EIV reporting.
`-eiv_eval_window {switching | timing | both | elapse}`	
	Controls the window where EIV is evaluated. EIV is obtained by processing the voltage waveforms and finding the worse-case effectiv power and ground pins during a specific window. There are four possible windows to evaluate EIV: • `switching` – any switching activity (current is non-leakage) reported in power analysis • `timing` – timing window information from TWF or embedded timing engine • `both (Default)` – both switching or timing window • `elapse` – entire rail simulation not filtered to a specific window **Note**: Although power analysis uses timing window information to generate switching activities, it is possible that switching window wil timing window if the instance is scheduled to switch at the very beginning or the very end of the timing window. Current waveform may than the time of switching.
`-eiv_histogram_max` *value*	
	Specifies the maximum voltage value of the user-defined voltage range for EIV histogram generation.
`-eiv_histogram_min` *value*	
	Specifies the minimum voltage value of the user-defined voltage range for EIV histogram generation.
`-eiv_histogram_number_of_bucket` *value*	
	Specifies the number of buckets/ranges to be used for EIV histogram generation. The default number of buckets is 8. The data points a based on the range specified.
`-eiv_max_instances` *value*	
	Specifies the maximum number of instances to be included in the .iv files.
`-eiv_method {worst best avg worstavg bestavg}`	
	Allows different methods of computing EIV during rail analysis. Using this parameter, you can write out the best, worst, or average effec between power and ground nets. The `worstavg` EIV method computes the average instance voltage during each evaluation window defined by the `-eiv_eval_window` pa the worst average instance voltage out of all the windows. Similarly, the `bestavg` EIV method computes the average instance voltage d window defined by the `-eiv_eval_window` parameter, and reports the best average instance voltage out of all the windows. If `-eiv_meth` and `-eiv_eval_window elapse` (i.e. 1 evaluation window) are specified, then this method will report average voltage across all time step *Default*: `worst`
`-eiv_pin_based_report {true | false}`	

	Specifies to report an individual pin's instance voltage for each instance. When enabled, the format of the instance voltage file (.iv file) v instance pin per line with pin name added to the last column. If an instance has multiple pins connected to the same net, it will have mu file.
	The format of the report is:
	<Instance Name> <Instance Voltage> <Cell Name> <Pin Names>
	The following is a snippet of the report from the .iv file:
	`NET`
	`INSTANCESUPPLY 18118 VDD 1.08 POWER`
	`BEGIN`
	`- inst1  1.0798 CKND12 VDD`
	`- inst2  1.0799 INVD6LVT VDD`
	`DOMAIN`
	`INSTANCESUPPLY 18118 VDD 1.08 DIFFERENTIATE`
	`- clk__L13_I3  1.0767 CKND12 VDD VSS`
	`- clk__L13_I4  1.0767 CKND12 VDD VSS`

`-eiv_pin_location {true | false}`	
	Specifies to report an individual pin's location for each instance. This parameter works only with the `-eiv_pin_based_report` parameter format of the instance voltage file (.iv file) will be changed to one instance pin per line with the pin location added to the last column. Th works for `ivdn` (net-based instance voltage drop). It is not supported for `ivdd` (domain-based instance voltage drop). That is, the pin loca only in the net-based iv report file (for example, VDD.iv file) and not in the domain-based iv report file (for example, VDD-VSS_div.iv). The default value of the `-eiv_pin_location` parameter is `false`. The format of the report is: *<Instance_Name> <PWR_IV> <Cell_Name> <Pin_Names> <X Y Coordinates>* The following is a snippet of the report from the .iv file: `- INST1 1.13831 MUX2_F8_75LL VDD met1` **`<6152370, 5708640>`** `- INST1 1.13831 ENOR4_F4_75LL VDD met1` **`<6153420, 5714520>`** `- INST2 1.13831 BUF_F1_1SR_75LL VDD met1` **`<6147330, 5708640>`**
`-eiv_print_time {true | false}`	
	Prints the time stamp for the worst voltage in the switching window of the instance, and for the whole (elapse) window. The time stamp the detailed EIV report. The default value of this parameter is `false`. Using the time stamp information, you can determine the current signature related to the worst EIV for easy debugging. The default unit
`-eiv_report {auto | netonly | all}`	
	Specifies the type of EIV report to be generated. The possible options are: • `auto` - generates net-based .iv report for net analysis, and domain-based .iv report for domain analysis. • `netonly` - generates net-based .iv report. • `all` - generates both net and domain .iv reports, if available.
`-eiv_threshold` *value*	
	Specifies the EIV drop threshold in mV for reporting. Only voltage drop above the threshold will be saved into the report.
`-em_cdf_percentage` *value*	
	Specifies to calculate the Cumulative Distribution Factor (CDF) derating factor based on the EM rules specified in the ICT EM file, given `ict_em_models` parameter. This parameter is used to calculate the fail rate and reliability of a design. The `-em_cdf_percentage` paramet the ICT EM file with the `cdf_percentage` keyword. The value of the CDF percentage should be between 0 and 100. The default value is `1e-9`. The use model of the new parameter is: `set_rail_analysis_mode -em_cdf_percentage 5` When the CDF related rule exits in the ICT EM file, the software uses the default value `1e-09` for `-em_cdf_percentage` if it is not specifie
`-em_ignore_pgv_resistors {true | false}`	
	Specifies to filter out all the EM violations of resistors inside a macro (resistors coming from PGV).

`-em_limit_scale_factor {{avg value} {rms value} {peak value}}`	
	Scales the calculated EM limits by the respective factor for avg, rms or peak current limit specified by `{avg value} {rms value} {peak` compares the current through the net against the scaled EM limit to report violations in power EM.
`-em_models file`	Specifies the name of a file that includes the electromigration (em) models. The `-process_techgen_em_rules` and `-em_models` parameters are mutually exclusive. In the static mode, if `-em_models` is not specified, `process_techgen_em_rules` is not set to `true`, current density (RJ) analysis will be disabled due to lack of EM models.
`-em_peak_analysis {true | false}`	
	Specifies to perform peak EM analysis. When this parameter is specified with `-process_techgen_em_rules true` or `-em_models`, both Rl be reported in the same run.
`-em_report_line_threshold value`	
	Controls the size of the EM (`rj.avg.rpt`) text report by limiting the number of lines written to the report file. This feature is useful in limiti you specify a very low EM threshold. **Example**: The following command reports only the top 18 lines: `set_rail_analysis_mode -em_report_line_threshold 18` The truncated EM report file has the following note: # NOTE: Aborting EM Report printing as total number of lines has exceeded the Line Threshold "em_report_line_threshold = 18"
`-em_rms_delta_t temp`	
	Specifies the delta temperature for the EM RMS current limit analysis. Default is 5C.
`-em_rms_delta_T_layer_list {{<LEF_layer_name> <delta_T_in_C>}+}`	
	Specifies the delta temperature for specific LEF layers during power EM RMS current limit analysis.
`-em_temperature <string>`	Sets a different temperature for EM analysis. As a result, you can use a different but higher t temperature for conservative EM analysis.
`-em_temperature_layer_list {{<LEF_layer_name> <em_temperature_in_C>}+}`	
	Sets a different temperature for specific LEF layers during power EM AVG current limit analysis.
`-em_threshold value`	
	Specifies to set EM analysis threshold value for the resistors being reported. Using this parameter, you can control the reporting thresh By default, rail analysis generates EM report for any resistor having current above `0.9` of its limit. Changing the value to `0` will show eve limit to the failed resistors only.
`-enable_2d_partition_extraction {true | false}`	
	Specifies to enable the upgraded extraction algorithm that allows 2D partitioning. The 2D extraction algorithm uses an updated capacit qrcTechFile that provides better accuracy for capacitance calculation. The resistance accuracy level of the 2D extraction algorithm is th extraction algorithm. It is recommended to use 2D partitioning as it improves the parallelism of the extraction jobs for large designs. The parameter is `false`. When the `-enable_2d_partition_extraction` parameter is set to `true`, you must provide the complete qrcTechFile using the `set_rail_extraction_tech_file` parameter. This parameter does not support the 3DIC, inductance extraction, and full-chip GDS flows.
`-enable_ccs_analysis {true | false}`	
	Specifies to enable constant current source analysis. The default value is `false`.
`-enable_distributed_processing_in_solver {true | false}`	
	Specifies to enable distributed processing for Matrix Solver. When set to `true`, Matrix Solver will be able to use multiple hosts across th power-grid. *Default* : `true` You can specify this parameter when peak memory of the analysis exceeds machine memory in the local multi-threaded mode. In the d memory on the local machine will be reduced as child processes will be executed on network hosts. However, the master thread on the continue to consume majority of peak memory even in the distributed mode.

`-enable_instance_powergate_report {true | false}`

Specifies to generate a report (`powergate.inst.rpt`) that summarizes information about each power gate instance.

The default is `false`.

The format of the report is:

```
<inst_name> <cell_name> <Inst X> <Inst Y> <V_Ref> <Ion(total)> <PI(Ipeak/Iavg)> PV(IR)>
```

Here,

- <`inst_name`> is the power gate instance name
- <`cell_name`> is the name of the power gate cell
- <`Inst X`> is the x coordinate of the power gate instance
- <`Inst Y`> is the y coordinate of the power gate instance
- <`V_Ref`> is the rail voltage
- <`Ion(total)`> is the total current for enabled powergate instance
- <`PI(Ipeak/Iavg)`> is the worst Idsat ratio out of all the Ron values
- <`PV(IR)`> is the voltage across the power switch using the combination for the worst V(AON Pin)-V(SW Pin)

`-enable_manufacturing_effects {true | false}`

Specifies to honor manufacturing effects during static or dynamic analysis.

Default : `false`

The manufacturing effects are enabled by default in the hd accuracy mode. That is, if you specify `set_rail_analysis_mode -accuracy` `enable_manufacturing_effects` parameter is set to `true` by default.

`-enable_multi_die_connectivity_check {true | false}`

Specifies to check if voltage sources with `type=nonvsrc` have resistance path to any voltage source with `type=vsrc` inside each die. Thi you to determine if the non-real voltage source (type=`nonvsrc`) is floating or does not have any resistance path to an ideal voltage sourc ignore floating voltage sources during rail analysis.

The "`vsrc`" and "`nonvsrc`" types are given in the .ploc file specified with the `set_power_pads` command. When the `-enable_multi_die_connectivity_check` parameter is specified, the log file displays the list of disconnected terminals for each net. This used in both the single-die and multiple-die analysis flows.

`-enable_package_battery_current_probing {true | false}`

Specifies to include the battery current waveform data in the `tran_<net>_pad.ptiavg` file. This file includes the voltage of all battery side ground_pin mapped pins.

Default : `false`

`-enable_rc_analysis {true | false}`

Enables the Resistor Current (RC) analysis. This parameter supports designs without EM rules/analysis to generate the RC plot for IR

`-enable_reff_analysis {true | false}`

Specifies to run full-chip effective resistance analysis as part of the static or dynamic IR drop analysis. Effective resistance analysis will domain specified with `analyze_rail`. This parameter allows you to run both resistance and IR drop analyses in a single session.

The use model of this parameter is:

```
set_rail_analysis_mode -enable_reff_analysis  true
analyze_rail -output <dir_name> -type domain <domain_name>
```

Default : `false`

`-enable_rlrp_analysis {true|false}`

Specifies to enable least resistance path analysis for all the instances in the design. The RLRP path can be displayed in the GUI using `set_power_rail_display -enable_rlrp` command.

`-enable_scheduler {true | false}`

	Controls the advanced scheduler that improves data handling and multi-CPU scalability of rail analysis . When set to `true`, the schedul improve performance and memory consumption, especially when running on large number of processors. The default value is `true`.
`-enable_seb {true | false}`	
	Specifies to enable the Statistical ElectroMigration Budgeting (SEB) analysis. *Default* : `false`
`-enable_sensitivity_analysis {true | false}`	
	Only available in the static mode. When set to `true`, the software performs sensitivity analysis on all power-grid segments and highlights those which can be optimized to IRdrop profile of the design. The power-grid segments can be optimized using the `scale_what_if_resistance` command. *Default* : `false`
`-enable_voltage_across_vias {true|false}`	
	Specifies to report voltage across vias.
`-enable_vsrc_in_gif {true|false}`	
	Specifies to enable or disable the display of voltage sources in the generated IR drop plot GIFs. When set to `false`, disables voltage sources (power pads) from the *Layout* tab during IR drop analysis. This parameter allows you to co voltage sources in the generated IR drop plot GIFs so that the plot is easily interpreted. *Default* : `true`
`-enable_xp {true | false}`	
	Specifies to run rail analysis in the Extensively Parallel (XP) mode. When this parameter is set to `true`, the rail analysis engine distributes processing over a large number of machines in every stage of th extraction, reporting, GUI, and simulation) to reduce the processing time. *Default* : `false`
`-enable_xpgv_scaling {true | false}`	
	Specifies to enable xPGV current scaling that is calculated as a ratio of voltages. If PGV voltage is V1 and analysis voltage is V2, then would be V2/V1. This factor is applied to each power net. In case there are multiple power nets and a single ground net, the tool will us factor from the power rails and apply it to the ground net. This parameter is used for scaling of currents stored in xPGV. The parameter allows you to use the same xPGV in different designs, irre at which the xPGV is generated. `-enable_xpgv_scaling` and `-static_trigger_file` are mutually exclusive. *Default* : `false`
`-enlarge_vsrc_in_vuvc {true | false}`	
	Specifies to increase the size of voltage sources represented for the voltage source current (vc) and voltage drop across package (vu) p visibility.
`-env_temperature <`*`temperature in °C`*`>`	
	Specifies the ambient temperature used to calculate S_{DC} for SEB analysis. *Default* : `25`
`-era_check_wires_for_generated_current_regions {true | false}`	
	Specifies to generate unplaced current regions only in areas where actual net wires are present. The default value is `false`. The curren generated from the power domain information present in the design.
`-era_current_distribution_factor_for_placed` *`value`*	

	When you use `-era_current_distribution` and set to distribute current for placed blocks/macros, the ratio of power allocated to each p calculated based on its total area. Using this parameter, you can control the current distribution factors for the placed blocks/macros, he for area-based power calculation. For example, if you specify `0.5`, the tool will assume all placed blocks/macros to be 50% of its actual current accordingly. *Default* : `1`
`-era_current_region_file` *filename*	

	Specifies a file that includes a list of regions and the amount of current to be distributed within them for the power-grid. Default units in f specify rectilinear current regions in the file.
	#Format:
	`LABEL` *name* `NET` *netName* `AREA` *x1 y1 x2 y2* `LAYER` *layername* `<CURRENT` *value* `| PWL (`*t1 i1 t2 i2 …*`)> INTRINSIC_CAP <`*value*`> LO` `[ADD_OVERLAP | SUBTRACT_OVERLAP]`
	#Unit:
	current mA, cap pf, time ns, coordinate um
	• `ADD_OVERLAP`: Honors all overlapping regions, and adds current in the overlapped regions.
	• `SUBTRACT_OVERLAP`: Specifies that if two XY regions overlap, the first specified region will be honored, and the subsequent region wil overlapped area to apply the region current to the remainder area.
	Static Example :
	`label test1 net VDD area 100 200 400 500 layer M1 current 10`
	Dynamic Example :
	`label test1 net VDD area 100 200 400 500 layer M1 pwl (0ns 0mA 1ns 0mA 1.9ns 0mA 2ns 10mA) intrinsic_cap 10 loading_ca`

`-era_current_distribution_layer` *layer_name*	
	Specifies the layer name for distributing unplaced current in the early rail analysis mode.
`-era_current_distribution { unplaced | placed | all | none }`	
	This parameter controls the behavior of era current distribution. `set_power_data` area based power, or `current_region_file` need to be current distribution to work. Placed instanced without uti or ascii based power can also be considered for era current distribution. • Unplaced: Enable current distribution only for unplaced instances. If `set_power_data` area based power is specified, `-era_current_` will be required. • Placed: Enable current distribution for placed instances without any power specified. • All: Both unplaced and placed instances without power specified; will have ERA current. • None: Disable ERA current distribution.
`-era_current_distribution_nets {`*net1 net2* `...}`	
	Specifies to distribute current for unplaced instances by total die area on specified nets only. The default behavior is to distribute curren parameter will be required when `-era_current_distribution_unplaced_area diearea` is used. For single net analysis, you can either specify a single net (for example, `-era_current_distribution_nets {VDD_AO}`) or perform the ar parameter.
`-era_current_distribution_unplaced_area {instance | diearea}`	
	Specifies to distribute current for unplaced instances by total die area or actual instance area. The default value is `instance`. When `diearea` is specified, the entire unplaced die area is considered for computation of current distribution. The die area considered e current regions, placed instances, and placement blockages.
`-era_insert_virtual_followpins { standard | extended | none }`	
	Specifies to generate virtual followpins. The `extended` followpins will create followpins that extend from one stripe to another. The `stan` extend to previous stripe but does not reach the next stripe. *Default*: `none`
`-era_insert_virtual_followpin_for_io {true | false}`	

	By default, ERA virtual follow pin insertion only works in the core area. When this parameter is set to `true`, ERA follow pin routing is ext as well.
`-era_insert_virtual_via_on_layers` *value*	
	Enables you to control on which layers virtual via can be inserted in early rail analysis. Using this parameter, virtual via can only be ins pair of layer names provided in the design. Alternatively, you can provide wildcard, '*', as the second layer name and virtual via will be from the first layer to any layers in the design. For example, `-era_insert_virtual_via_on_layers { {M1 *} }` will allow virtual via to be inserted from M1 to any layers in the design. This parameter does not have a default value but ERA will insert virtual via for all layers if -era_skip_virtual_via parameters are not pro
`-era_instance_dynamic_current_file` *filename*	

	Specifies an instance/cell based PWL file in the dynamic ERA flow (`set_rail_analysis_mode -era_dynamic`). This feature allows you ERA without having to specify the dynamic .ptiavg/.ptiavk current files (`set_power_data`).
	The specified PWL should cover the entire simulation period. If the simulation period is longer than the specified PWL, the software wil the remaining period (`set_dynamic_rail_simulation -stop <time>`). You can also use the `set_dynamic_rail_simulation -auto_repea` the dynamic current waveform till the `-stop` time specified, instead of assuming 0 current for the remaining period.
	The format of the file is:
	`INSTANCE | CELL <NAME> PWL { <time> <current> } <PG_PIN> [ADD_OVERLAP | SUBTRACT_OVERLAP]`
	• Default time unit = `ns`.
	• Default current unit = `mA`
	• `<PG_PIN>` is optional. If it is not specified, the software mirrors the current on VSS pins.
	• `ADD_OVERLAP` keyword indicates to honor all overlapping instance currents, that is, instance to instance overlap and instance to regio
	• `SUBTRACT_OVERLAP` keyword indicates to honor instance current, and subtract its area from the overlapped X-Y region to apply the X- remainder area. `SUBTRACT_OVERLAP` is default. This keyword is not applicable for instance to instance overlap.
	The following is a snippet of the specified file:
	`INSTANCE top/instA PWL { 0ns 0mA 0.1ns 1mA 0.2ns 0.5mA } VDD`
	`INSTANCE top/instA PWL { 0ns 0mA 0.1ns -1mA 0.2ns -0.5mA } VSS`
	`CELL macro_A { 0 0 0.1ns 1 0.2 0.5 }`
	`...`
	If you have specified both the `-era_instance_dynamic_current_file` and `-era_current_region_file` parameters, and there are overla the specified current region, the `SUBTRACT_OVERLAP` behavior is implemented by default.

`-era_lef_layermap` *filename*	

	If a technology library or an extraction techfile is not provided, ERA can automatically create a technology library and the layermap nee library. If the automatic layer map generation fails, you can use this parameter to manually specify a layer map file.
	The format of the layer map file is:
	File Format:
	`Layer Type tech_layername lefdef lefdef_layername`
	`metal METAL_1 lefdef M1`
	`via VIA_1 lefdef VIA1`
	`metal METAL_2 lefdef M2`
	`via VIA_2 lefdef VIA2`
	`metal METAL_3 lefdef M3`

`-era_power_gate_file` *filename*	

	If a power gate PGV is not provided, this parameter specifies the name of the power gate (power switch) file that will be used to perform state analysis.
	File format:
	`CELL <CELLNAME> SUPPLY <SUPPLY_PIN> SWITCHED <SWITCHNET_PIN> RON <VALUE> IDSAT <VALUE> ILEAK <VALUE>`
	`CELL HDRSID0 SUPPLY TVDD SWITCHED VDD RON 500 IDSAT 1 ILEAK 0.001`

`-era_skip_virtual_via_by_type {whatif | def | all | none}`	

	Specifies to skip a given via type. By default, ERA generates all virtual via layer types. • `whatif` vias are virtual vias that have connectivity to user-defined what if shapes. • `def` vias are virtual vias between two metal shapes defined in DEF. • `all` will skip all virtual via generation. • `none` will insert both what-if and DEF vias. It will insert vias on all layers, unless the layers are controlled by other parameters. *Default*: `all`
`-era_skip_virtual_via_on_layers {{ layer1 layer2 } { layer3 layer4 } ... }`	
	Skips via insertion between stripes and non-stripes, on the specified LEF layer pairs.
	{ layer1 layer2 } { layer3 layer4 } ...: Specifies the LEF layer names.
`-era_techlib_generation {true | false}`	
	Specifies to skip technology PGV generation in the early rail analysis mode. This parameter skips technology PGV generation even if t `extraction_tech_file` parameter is specified. The default value is `true`.
`-extractor_include filename`	
	Specifies extractor (ZX) commands and variables in the form of an include file.
`-extract_subconductor_layers {true | false}`	
	Specifies to perform extraction of the sub-conductor layers. The default is `false`.
`-extraction_tech_file filename`	
	Specifies the extraction technology file to be used for top-level power-grid extraction. If you do not specify this parameter, the software v technology file stored inside the power-grid view library.
`-feol_scale_factor non_negative_integer_number`	
	Specifies to scale the front-end-of-line (FEOL) self-heat value for all instances in the self-heating analysis flow. The delta T FEOL will b the static power, thermal resistance, and FEOL scale factor.
`-filler_cell_list { cell1 cell2 ... celln }`	
	Specifies a list of filler cells. This option is used when the filler cells are not tagged in the power-grid library.
`-finegrain_powergate_ron {min avg max}`	
	Specifies whether the minimum, maximum, or average Ron will be used for a powergate cell in finegrain memory characterization with The default value is `max`.
`-finegrain_powergate_ron_list filename`	

	Specifies the name of a file containing the powergate cell names and their corresponding Ron values (minimum, maximum, or average finegrain memory characterization with multiple Ron values. The format of the file is:
	`CELL cell1 min/max/avg`
	`CELL cell2 min/max/avg`
	`INST inst1 min/max/avg`
	Note: The `-finegrain_powergate_ron` and `-finegrain_powergate_ron_list` parameters allow you to specify multiple Ron values in a Ron value can change with respect to different modes of simulation.

`-fine_pre_simulation_period value`	
	Specifies the fine-grain pre-simulation period that rail analysis will use. The supported time units are s, ms, us, ns, and ps. If no explicit the default ns (nanosecond) will be applied. The time step in the fine-grain pre-simulation period will be the same as the simulation per Pre-simulation allows you to determine the current profile and analyze IRdrop of the circuit when power supply is in steady state. **Note**: When both the coarse-grain (`-pre_simulation_period`) and fine-grain pre-simulation periods are specified, the fine-grain pre-sim after the coarse-grain pre-simulation period. The simulation phase starts after the fine-grain pre-simulation period. Both fine-grain pre-si grain pre-simulation can be specified just by themselves without requiring the other one to be specified. In the case where fine-grain pr specified, the simulation will start right after the coarse-grain period.
`-flatten_xpgv_block_instances filename`	

	Specifies a file that includes a list of block instances that use DEF instead of xPGV for rail analysis, that is, specify the instances that ar xPGV. This parameter allows you to specify a few instances of the same block that will use DEF, and the others that will use xPGV at th not specify this parameter, and if both DEF and xPGV are given for a block, xPGV takes precedence over DEF.
	Instance List File Format:
	`<block_name> <block_instance_name|All|None>`
	In this file, you can specify if a block is using a DEF by specifying the block instance name, or with the `All` or `None` keywords. `All` indica are using DEF, and `None` indicates that none of the instances are using DEF.
	Example:
	`CORE /top/x1/x12/instA`
	For a design with three blocks, if you want to use the DEF and xPGV as following:
	Block_1 - > A1 (xPGV) & A2 (DEF)
	Block_2 - > B1 & B2 (xPGV for both instances)
	Block_3 - > C1 & C2 (DEF for both instances)
	The specified file will contain the following information:
	`Block_1 A2`
	`Block_2 None`
	`Block_3 All`

`-force_extraction {true | false}`	
	Specifies whether the existing work directory extracted data should be used, or the design needs to be re-extracted during rail analysis *Default* : `false`
`-force_library_merging {true | false}`	
	Specifies to force merge PGVs with different resistivity and layers during rail analysis, wherein the first library definition will be used. *Default* : `false`
`-gds_file` *file*	Specifies a GDS file. Partial (RDL GDS in flip-chip) or full-chip GDS can be specified. You can view extracted parasitic network for GDS GDS layout it needs to be imported as an OA database.

`-gds_map` *file*	Specifies the GDS layermap for the GDS file specified using `-gds_file` parameter. The GDS layermap file format is as follows:
	`#layer gds layer_name gds_layer_number`
	`<gds_data_type><[port|text]>`
	`layer gds via8 85`
	`layer gds metal9 74`
	`layer gds metal9 126 1 text`

`-gds_offset` { *x y* }	
	Specifies the offset value used to adjust and shift the GDS of a Redistribution Layer (RDL) cell in the mixed DEF+GDS flow. This parameter is available only when the `-gds_file` parameter is specified.
`-gds_purpose {metalFill | flipChip | fullChip}`	
	Specifies the data type of the GDS provided. • `metalFill`: The GDS data will be used as fill metals for manufacturing effects calculation but the physical shapes will not be extracte • `flipChip`: The GDS data contains only the RDL layers, and the extracted physical shapes will be merged with the DEF netlist. • `fullChip`: The GDS data contains physical shapes for the entire design.
`-gds_top_cell` *cell_name*	
	Specifies the name of the GDS top cell.
`-generate_hier_bbv {true | false}`	

Specifies to control the generation of block boundary voltage (BBV) for all hierarchical block instances that use xPGVs. The xPGVs for generated using the xPGV flow (Refer to "Hierarchical Power Integrity Analysis with Extreme Modeling" in the *Voltus User Guide*). The parameter is `false`. The BBV file contains realistic static or dynamic voltages for all nodes in the xPGVs starting where the top-level po the xPGVs. Since the BBV file is generated in context of full-chip analysis, running block-level analysis using this file gives insight to IF lower layers not included in the xPGV. This top-down flow can be used to identify and fix IR drop problems at the partition level without expensive fullchip analysis.

`generate_hier_bbv` is a global parameter so all instances of xPGV will be generated. The output is written to the state directory for each analyzed as '`instance_name.pp`'. The file contains '`putvsrc`' commands. You can output on a per-instance basis using the '`write block` command in an include file, as shown below:

```
set_advanced_rail_options -voltus_rail_include_file_begin voltus_begin.inc
```

voltus_begin.inc:

```
write block_boundary_voltage instance_name output_file_name
```

For dynamic analysis, add `-watch`:

```
write block_boundary_voltage instance_name output_file_name -watch
```

The dynamic flow is recommended since transient information is not saved by default.

To use the output BBV file at the block level, return to the block-level execution script and change `set_power_pads -format` to `boundary` file:

```
set_power_pads -net net_name -format boundary -file bbv_file
```

Example of a BBV file for dynamic analysis:

```
* Boundary conditions for instance sample_instance
# putvsrc vsrc_# IR/Bounce x y PWLFILE pwl_file_for_node_location
putvsrc K4 Vbc_VSS_1 1.210359e-01 247.296 179.712 PWLFILE sample_instance.pp.ptiavg 1
putvsrc K3 Vbc_VSS_2 1.210360e-01 247.296 179.712 PWLFILE sample_instance.pp.ptiavg 2
putvsrc K4 Vbc_VSS_3 1.210097e-01 247.296 175.104 PWLFILE sample_instance.pp.ptiavg 3
…
```

`-generate_instance_effr_report file`

Generates a layer-based instance effective resistance report for specified cells/instances. The specified file contains the list of cells or i Reff report is to be generated. The format of the input file is:

```
CELL cell1
CELL cell2
INST inst1
```

The output file will be stored under each net's report folder `<net>.instance_effr.rpt`. For example, `Reports/VDD/VDD.instance_effr.r`

For each cell or instance specified in the input file, the report has information such as layer-based effective resistance ranges, worst no statistics, and interface node resistances. This feature is applicable for both net-based and domain-based analysis.

`-generate_instance_ir_report file`

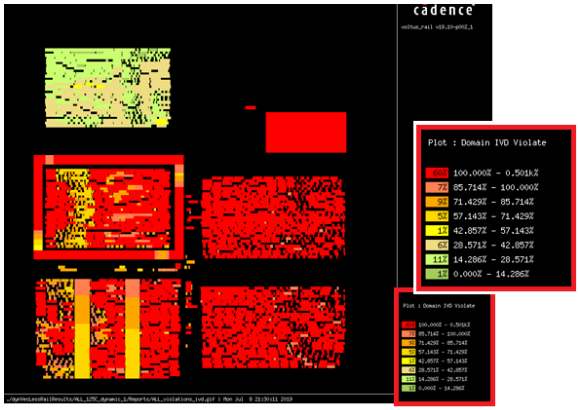
Specifies to generate a layer-based IR drop report for a set of cells/instances. The *`file`* contains the list of cells or instances for which t be generated. The format of the file:

```
CELL cell1
CELL cell2
INST inst1
```

The output file will be stored under each net's report folder `<net>.instance_ir.rpt`. This feature is applicable for both net-based and d

`-generate_instance_pin_ir_report file`

	Specifies to generate a detailed report containing boundary node IR information for a set of cells/instances.
	The file contains the list of cells or instances for which the IR drop report is to be generated. The format of the file is:
	`INST <instance_name>`
	`CELL <cell_name>`
	The instance pin IR report will list all the pins' node information (layer, location) and the worst IR drop value.
	The output file will be stored under each net's report folder `NET/<net>.instance_pin_ir.rpt`. The following is a snippet of the IR drop re
	`instance: <NAME>`
	`cell: <CELL>`
	`pin name: <PIN>`
	`# pin nodes: <total count>`
	`Min/Avg/Max: <min/avg/max interface node IR>`
	`For each of the interface node:`
	`IR X Y Layer`
	`......`
	The interface node output is sorted in the descending order of IR drop (that is, worst IR drop first).

`-generate_combined_ivd_gif {true | false}`	
	Specifies to generate a combined instance voltage drop GIF file for all the nets specified in the domain. This GIF allows you to see the instance voltage plot. The GIF file displays color scaling based on voltage drop as percent of different domain thresholds (print threshol domains), as shown below:  The file location of this GIF file is `<rail_dir>/Reports/ALL_violations_ivd.gif`.
`-generate_multi_voltage_library {true | false}`	
	When this parameter is specified, all the PGVs for each cell will be considered, and voltage-dependent capacitance information will be analysis, similar to the multi-voltage PGVs. The `-generate_multi_voltage_library` parameter is used in a scenario when you have generated multiple PGVs for different voltages a single multi-voltage PGV that has voltage-dependent capacitance. The default value of the parameter is `false`.
`-generate_package_pin_current_voltage {true | false}`	

	Specifies to generate a file containing transient current and voltage information of the package pins for the package analysis flow. This dynamic rail analysis. The default value is `false`. This parameter is used in both the Sigrity Package Analysis (SPA) and non-SPA package analysis flows: • SPA flow - used in Sigrity tools to do chip-aware package analysis. This parameter is enabled by default (set to `true`) in the SPA flo • Non-SPA flow - allows you to check the current on package pins. The format of the generated report in the static analysis mode (3 column format) is: *pin_name1 current1 voltage1* *pin_name2 current2 voltage2* ... The format of the generated report in the dynamic analysis mode is: .SUBCKT *Subcircuit_Name <list of pins for power and ground nets>* *MCP header with package pin name IA1 A1 0 PWL t1 i1 t2 i2 t3 i3 t4 i4 t5 i5 +t6 i6 … .ENDS *Subcircuit_Name* Here, • `IA1` (first item) is the current source element name • `A1` (second item) and `0` (third item) are the power and ground nodes • `PWL` keyword is followed by the time and current values
`-gif_iv_threshold` *value*	
	Specifies the instance voltage threshold for the combined instance voltage drop GIF for all the nets specified in the domain. The thresh voltage drop value. Any instance voltage drop value greater than the threshold will be put into the red bucket range in GIF, and the rest linearly distributed down to the minimum instance voltage drop.
`-gif_new_color_scale {true | false}`	
	Specifies to change the color scheme of the GIF file generated in the state directory during the static and dynamic rail analysis flow to n scale.
`-gif_resolution {low | medium | high}`	
	Controls the resolution of the generated GIFs after rail analysis. This option can have the following values: `low`, `medium`, and `high`. The d
`-gif_zoom_area { x1 y1 x2 y2 }`	
	Specifies to set the GIF generation bounding box to the given area (in microns).
`-gif_zoom_topcell_diearea {true | false}`	
	Specifies to set the GIF generation bounding box to the topcell DIEAREA listed in the DEF file, instead of the total design size includin used, this parameter has no effect. This parameter will change the bounding box of all the GIFs generated in rail analysis. **Note:** The `-gif_zoom_area` and `-gif_zoom_topcell_diearea` parameters are mutually exclusive. If both the parameters are specified, th parameter takes precedence.
`-hier_delimiter` *character*	
	Specifies the hierarchical delimiter in the DEF file. The default hierarchical delimiter is a forward slash (/) but can be changed by settin parameter. You can use this parameter to override the existing hierarchical delimiter.
`-honor_negative_static_current {true | false}`	
	Specifies to honor negative current value in the current file (`.ptiavg`) during static rail analysis. The default is `false`.
`-hpgv_block_lefs` *list_of_files*	
	Specifies the block LEF file for all hierarchical DEF blocks in the design. This will trigger automatic hierarchical PGV generation for the software will error out if any of the LEF files are missing for a DEF hierarchy.
`-hpgv_generate_view {ir | em | all}`	

	Specifies the view type to generate for the hierarchical PGV. The allowed values are "ir", "em" and "all". "all" means both ir and em vie you choose to only generate ir or em view, disk consumption can be reduced since only one view will be generated. *Default*: all
-ict_em_models *file*	
	Specifies the ICT-EM file which only contains the EM rules and excludes the RC extraction data in ICT. The software will use the EM ru ICT-EM file to calculate EM limits for power net EM analysis even if there are embedded EM rules in the Quantus tech file.
-import_what_if_shapes {true | false}	
	Specifies to import the what-if shapes created by the add_what_if_shapes command for rail analysis. **Note**: If both the -import_what_if_shapes and -what_if_shapes_file parameters are specified, union of two sets of what-if shapes is c
-ignore_decaps {true | false}	
	Ignores decap cells during rail analysis. It ignores cells that are tagged as DECAP cells during library characterization or set as decap c set_rail_analysis_mode -decap_cell_list command parameter. If decaps are used to preserve connectivity on the follow-pin routing parameter to false. Ignoring decaps during dynamic analysis is not recommended. *Default*: true for static, false for dynamic
-ignore_duplicate_vsrc {true | false}	
	Specifies to check for multiple voltage sources with the same name in the power pad (.ploc) file. When set to true, the duplicate voltag When this parameter is set to false and duplicate voltage sources are present in the .ploc file, an error will be generated. If there are duplicate voltage sources, rail analysis will report an error message like the following: `** ERROR: (VOLTUS_RAIL-3647): Failed to run rail analysis due to redefined voltage source name in power pin location f /vsrc_same_name/PADs/VDD_AO.pp.` *Default*: true
-ignore_fillers {true | false}	
	Ignores filler cells during rail analysis if the list of filler cells are defined in PGV generation, or specified as fillers using the set_rail_an filler_cell_list command parameter. During rail analysis, ignore library cells that are tagged as FILLER cells and have no capacita the interface nodes. If fillers are used to preserve connectivity on the follow-pin routing, you must set this parameter to false. *Default*: true
-ignore_from_reporting *filename*	

	Specifies the name of a file containing a list of cells and instances that are not to be included in the EIV report.
	A sample file is given below:
	`INST AO2/MULT16_reg_2_6`
	`CELL BUFX*`

-ignore_incomplete_net {true | false}	
	Specifies to skip power and ground nets that are not defined in the design, or have been defined but with no instance connection.
-ignore_nets_without_vsrc {true | false}	
	Specifies if Rail Analysis should continue or exit with an error message if there are nets with no voltage sources. The possible argumer • true: prints an error message only but will allow the Rail Analysis run to continue. • false: exits immediately and returns to the Tcl prompt. If all the VDD/VSS nets have no voltage sources attached, the Rail Analysis run will error out and exit even if -ignore_nets_without_v *Default*: false
-ignore_shorts {true | false}	
	This parameter allows you to ignore shorts that are found during rail extraction and continue rail analysis. However, -ignore_shorts ca design contains GDS.

`-ircx_models {RC.ircx EMIR.ircx}|{RC.ircx}`	
	Specifies an encrypted or unencrypted RC/EM technology (.ircx) file. The foundry-specific IRCX file, which contains the process inform EM modeling, can be used when the ICT-EM file is not available for electromigration analysis. You can either specify two IRCX files, one file containing the RC process data (`RC.ircx`) and the other file containing EM rules (`EM.ircx` that includes both the RC and EM sections (`RC.ircx`). A single IRCX file is used for the mature nodes, and separate RC and EM IRCX f advanced process nodes.
`-lef_map` *`lefMappingFile`*	
	Specifies the LEF layermap file, which contains the mapping of the LEF layer and the technology layer. The LEF layermap file format is as follows: `layer map lef_layer_name tech_layer_name`
`-lifetime` *`value`*	
	Specifies the lifetime value in hours. *Default* : 87,600 hours (= 10 years)
`-limit_number_of_steps {true | false}`	
	Specifies to limit the number of steps (simulation period, and resolution or step size) during dynamic IRdrop analysis or power-up analy The default value of this parameter is `true`. If the number of steps during dynamic IRdrop analysis or power-up analysis exceeds `1000`, the tool gives an error message and exits ar ensures that you are aware that there could be a problem in how the analysis is set up. If the performance risks are understood, you car `false` and re-run the analysis. The step-size and simulation period for the analysis is derived from current files generated by power analysis. You can explicitly set the simulation period during power analysis using the `set_dynamic_power_simulation` command. During power-up analysis, the simulation is derived based on the turn-on time of the last power-switch in the power-up domain. If this time is too large, you may want to check if t power-gates are correctly captured in the timing window file. Optionally, you can reduce the number of steps by explicitly specifying the analysis using the `set_dynamic_rail_simulation` command.
`-lowest_layer_for_em_check` *`layer_name`*	
	Specifies the layers that should not be displayed in the EM plots (such as, rj, jrms, and javg) and reports. *`<layer_name>`* is the layer below which the EM plots and reports will not be generated. The default value is `false`.
`-mcp_model_mapping {{`*`<mcp model name1> <die DEF name1>`*`} {`*`<mcp model name2> <die DEF name2>`*`} ...}`	
	Specifies to set the mapping between the package model and die name.
`-method {static | dynamic | era_static | era_dynamic}`	
	Specifies whether static or dynamic rail analysis, or static or dynamic early rail analysis will be performed. *Default* : `static`
`-off_rails` *`net_name_list`*	
	Specifies a list of nets to exclude from analysis. Used in designs with power switches.
`-override_set_power_data {true | false}`	
	Default: `false` Specifies to override the previous settings of the `set_power_data` command and use the last specified setting of `set_power_data` when t design object using the same current file. This feature is convenient when many `set_power_data` commands have been set and you wa them with a new scaling factor or other options. **Example**: In the following example, the first command will be discarded and the second command will be honored: • `set_power_data -format current -instance XYZ dynamic_VDD.ptiavg` • `set_power_data -format current -instance XYZ dynamic_VDD.ptiavg -scale 2.0`
`-package_trace_connectivity {true | false}`	

	When `-package_trace_connectivity` is set to `true`, connectivity tracing is enabled from the die pins to the board pins of all the element file. When this parameter is set to `false`, connectivity tracing is disabled. *Default* : `false`
`-peak_em_analysis {true | false}`	
	Specifies to calculate the peak EM in the dynamic rail analysis run. This parameter generates the `<net_name>.rj.peak.rpt` report. *Default* : `false`
`-power_grid_library` *dir_list*	
	Specifies the name of the power-grid view library directories.
`-powerup_fast_mode {true | false}`	
	Specifies to enable fast mode simulation for native power-up analysis. When the fast mode is enabled, the software uses lumped paras instances connected to the powering up net. The fast mode native power-up analysis allows you to quickly estimate turn-on time and rush current injected in the global power-grid a is powering up. The fast-mode power-up analysis gives faster turn-around time and similar accuracy as accurate power-up analysis. *Default* : `true`
`-powerup_sequence_file` *filename*	

	Specifies the name of the file containing the firing time for each power gate instance. The power-up sequence file format is as follows:
	`<power gate instance name> <pin name> <always- on net> <switched net> <rise_time> <rise_slew> <fall_time> <fall_slew> <`
	`<control pin voltage>` is optional.
	A sample power-up sequence file is given below:
	`inst_name E VCC_NORMAL VCC_SWITCHED 5.50E-09 2.90E-10 5.50E-09 2.90E-10 0.84`
	`inst_name E VCC_NORMAL VCC_SWITCHED 5.93E-09 7.23E-10 5.93E-09 7.23E-10 0.84`

`-powering_up_rails` *net_name1* ... *netname_n*	
	Specifies a list of powering up nets.
`-powering_down_rails` *net_name1... netname_n*	
	Specifies a list of powering down nets.
`-pre_simulation_period` *value*	
	Specifies the pre-simulation period that rail analysis will use. The supported time units are s, ms, us, ns, and ps. If no explicit time unit is ns (nanosecond) will be applied. The pre-simulation period include the coarse-grain and fine-grain pre-simulation periods. If `-fine_pre_simulation_period` is not speci pre-simulation period will do coarse pre-simulation.
`-pre_simulation_resolution` *value*	
	Specifies the transient time step size or resolution. The supported time units are s, ms, us, ns, and ps. If no explicit time unit is specified (picosecond) will be applied.
`-prechain_powerup_sequence_file` *filename*	

	Specifies a power-up sequence file that contains the pre-chain start/end and mother (data) chain start/end timing information. In the pov specify this pre-chain sequence file instead of a complete power-up sequence file to modify the ramp-up timing of different power switcl constraint in the specified file. The timing information for the other power-gate instances is calculated using the pre-chain sequence file The format of the file is:
	`Prechain_start <pggate_instance_name> <pin_name> <d_time> <d_delay> <d_slew>`
	`Prechain_end <pggate_instance_name>`
	`Datachain_start <pgagate_instance_name> <pin_name><m_time> <m_delay> <m_slew>`
	`Datachain_end pggate_instance_name`

`-print_gif_range_percentage {true | false}`	

Specifies to display the plot title and voltage drop percentage (`Drop%`) for the following GIF files:

- **IR** (File Location: `<rail_dir>/Reports/<power_net>/`)

  `ir_linear.gif`

  `ir_limit.gif`

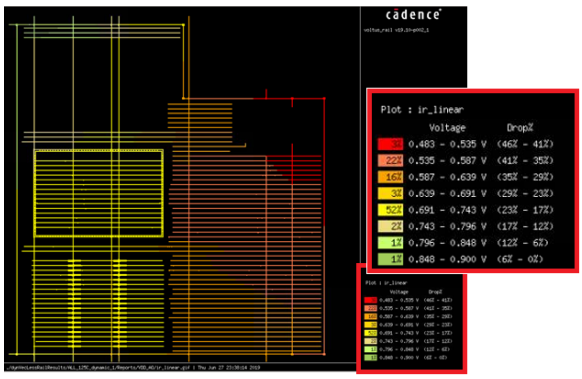
- **EIV** (File Location: `<rail_dir>/Reports/`)

  `<pg_pair>.<eiv_method>_eiv.gif`

  `<pg_pair>.<eiv_method>_limit_eiv.gif`

`IR Drop%` is calculated by 1-(voltage/ideal_voltage), and `EIV Drop%` is calculated by 1-(effective_voltage/ideal_effective_voltage)

The following image shows the voltage drop percentage for the `ir_linear` plot:



Default : `false`

`-probe_instance_pin_waveforms {true | false}`

Specifies to probe current and voltage waveforms for all the pins belonging to probing instances. When `-probe_instance_pin_waveforr` writes all the voltages and currents of the pins of probing instances to the waveform files.

The use model of this parameter is:

```
set_rail_analysis_mode -method dynamic -probe_instance_pin_waveforms true
```

The watched instance has the pin names in the waveform file, as shown below:

Voltage waveform name: `<instance_name>:ir`
Current waveform name: `TC_<instance_name>:ir`

Default : `false`

`-probe_instance_tap_waveforms {true | false}`

Specifies to probe current and voltage waveforms for all tap nodes belonging to probing instances. When `-probe_instance_tap_wavefo` writes all the tap node voltages and currents of the instances from `-probe_waveform_list` to the waveform files. The default value of the `false`.

The use model of this parameter is:

```
set_rail_analysis_mode -method dynamic -probe_instance_tap_waveforms true -probe_waveform_list instance_list_file
```

The instance's tap nodes are named based on instance name, and its node ID and location in the waveform file, as shown below:

Voltage waveform name: `<instance_name>:N_...:X_..:Y_:ir`
Current waveform name: `TC_<instance_name>:N_...:X_..:Y_...`

`-probe_package_interface_ports {true | false}`

Specifies to probe voltages for all the ports of the top-level subcircuit and the interfaces within the package. When this parameter is spe engine retrieves the package interface ports from the package model and writes them into a probing node file to watch their voltage wa

Default : `false`

`-probe_pin_voltage_list filename`

	Writes the voltage of the interface nodes for the specified instance/cell to a block boundary voltage file. The software generates a block with file name based on the block instance name, `<instance_name>.pp` for every instance/cell in the list.
`-probe_waveform_list` *filename*	

	Specifies the name of a file containing the instance and cell name(s) for which you want to capture voltage and current waveforms. This to control whether to probe voltage or current waveforms or both.
	You can specify the optional parameter `PROBE <VOLTAGE|CURRENT|ALL>` in the file. Default is `ALL`. When `VOLTAGE` or `CURRENT` is specified, current waveforms will be probed.
	The format of the file is:
	`CELL/INST <name>`
	Following is an example of the file:
	`INST I1_1`
	`INST I2_1`

`-probing_node_file` *file_name*	

	Specifies the probe node file containing the probe point on a die. The file format for specifying probing nodes is as follows:
	`<user defined unique probe name> <location = x> <location = y> <layer> <Power/ground net name>`
	You can also specify the package node voltages and resistor currents that are to be probed in the following format:
	`<probe name> pkg ir <pkg circuit node name>`
	`<probe name> pkg rc <pkg circuit resistor name>`
	Example
	`probe_1 18 16 METAL_1 VDD`
	`# pkg node voltage`
	`probe_pkg_1 pkg ir GND.in0`
	`probe_pkg_6 pkg ir VDD.in2`
	`# pkg resistor current`
	`probe_pkg_cur_1 pkg rc R_1_1`
	`probe_pkg_cur_2 pkg rc R_5_2`
	Note: The units of x and y should be in microns.

`-process_techgen_em_rules {true | false}`	
	Processes the EM rules defined in the extraction technology file. When `-process_techgen_em_rules` is specified with `-extraction_tec` technology file will be processed. If `-process_techgen_em_rules` is specified without `-extraction_tech_file`, the .cl model will be proc *Default* : `false` The `-process_techgen_em_rules` and `-em_models` parameters are mutually exclusive. In the static mode, if `-em_models` is not specified, `process_techgen_em_rules` is not set to `true`, current density (RJ) analysis will be disabled due to lack of EM models.
`-process_bulk_pins_for_body_bias {true | false}`	
	When set to `true`, rail analysis will process current sinks on the cell's bulk pin connections. This parameter is used during body bias an bias net is connected to bulk pins of the cell. The bulk pins for a cell is defined in the cell library using the `set_pg_library_mode -bulk_power_pins -bulk_ground_pins` command p If bulk pins are defined using `set_pg_library_mode -power_pins -ground_pins`, then the `-process_bulk_pins_for_body_bias` paramet *Default* : `false`
`-promote_pin_shapes_only {filler | decap | both}`	
	Specifies to promote either only filler shapes, only decap shapes, or both types of pin shapes during rail analysis. When this parameter interface nodes for the filler and decap cells are ignored. The `-promote_pin_shapes_only` parameter helps to reduce the resistor and no designs without affecting connectivity. This parameter must be specified with `set_rail_analysis_mode -enable_2d_partition_extraction true` (enabled by default for the X
`-rdl_def` *def_file*	

	Specifies to instantiate the RDL instance/DEF at the top level and merge it with the DEF of the top level block being analyzed.
	The following is the usage model:
	`set_rail_analysis_mode -rdl_def <def_file> -rdl_placement { X Y }`
	For GUI display, the same parameters also work for `read_def`:
	`read_def -rdl_def <def_file> -rdl_placement { X Y }`

`-rdl_def_list {{rdl_filename1 x1 y1 orient1} {rdl_filename2 x2 y2 orient2} ...}`	
	Specifies to import the list of the RDL DEF files for the top-level rail analysis. This parameter allows to import multiple RDL DEFs with t rotation information in the top level. Example: `-rdl_def_list { {RDL1.def 0 0 N} {RDL2.def 100 200 N}}`
`-rdl_placement {X Y}`	
	Specifies to place the RDL at a given X,Y location (in micron).
`-rdl_orientation { N|S|W|E|FN|FS|FE|FW }`	
	Specifies the orientation of the RDL DEF file.
`-read_thermal_map thermal_map_file`	
	Specifies to process the thermal map file to calculate resistance based on uniform temperature. Resistor scaling is supported for the tile-based power map file. Using the tile-base thermal map from Sigrity PowerDC, the software will value based on the temperature of the resistor's physical location, which provides more accurate IR and EM results with thermal impac *Default* : `None` For more information on the thermal map file format, refer to the Thermal Map File section in the "File Formats" chapter of the *Voltus Us*
`-record_inst_peak_current {true | false}`	
	Specifies to save the instance peak current during rail analysis in the rail state directory. This parameter allows you to check and debug abnormal instance peak current, which may result in high IR drop. The default value of this parameter is `false`.
`-record_results_start_time time`	
	Records rail analysis results based on the specified start time. The default unit of this parameter is `ns`. The software will use the specified time for all types of analysis, for example, instance voltage calculation, dynamic waveforms, and po
`-reff_detail_report {true|false}`	
	Specifies to include the following additional columns for instance pins in the effective resistance (`effr.rpt`) report: • `X` and `Y`: location (x/y coordinates) • `LAYER`: layer name • `PIN_NODES(FLOAT/TOTAL)`: number of floating nodes per total number of nodes of the instance By default, these columns are not included in `effr.rpt`.
`-reff_domain_report_format {1 | 2 | 3 | 4}`	

Controls the format of the effective resistance analysis domain report (`domain_effr.rpt`). This parameter allows you to specify the level displayed in the domain-based analysis report.

The following table gives the report format for the four format values:

Format Value	Format
1 (1 is the default format)	# PASS/FAIL REFF INSTANCE PIN(Pin REFF Percentage) **Example**: FAIL 10.0 instanceName VDD1(50.00%) VDD2(30%) VSS1(10.00%) VSS2(10%)
2	# PASS/FAIL REFF INSTANCE POWER-PIN GROUND-PIN (Pin REFF Percentage) **Example**: FAIL 6.0 instanceName VDD1(83.33%) VSS1(16.67%)
3	# PASS/FAIL REFF INSTANCE PIN (Pin Absolute REFF Value, Pin REFF Percentage) **Example**: FAIL 10.0 instanceName VDD1(5.0, 50.00%) VDD2(3.0, 30%) VSS1(1.0, 10.00%) VSS2(1.0 10%)
4	# PASS/FAIL REFF INSTANCE PWRNET PIN Layer X Y % GNDNET PIN Layer X Y % **Example**: FAIL 22.709 instanceName VDD pinName layerName 571.340 473.060 value(37.49%) VSS pinName Layername 571.340 471.800 value(62.51%)

`-reff_eval_nodes {port | tap}`

Default: `port`

Reports effective resistance based on the port (pin) or tap nodes. By default, the software uses the port nodes for REFF reporting.

`-reff_pin_report_layer { top | bottom | all }`

Specifies which layer of the pin shapes are to be reported in the EFFR report. This parameter allows you specify which cell boundary p for the resistance report.

The possible arguments of this parameter are:

- `top`: Specifies to report only the pin shapes located on the highest pin layer.
- `bottom`: Specifies to report only the pin shapes located on the lowest pin layer.
- `all`: Specifies to report the pin shapes located on every layer.

The default value is `all`.

`-reff_pin_report_method { best | worst }`

Specifies whether to report the best or worst effective resistance value of an instance pin with multiple nodes.

The possible arguments of these parameters are:

- `best`: Reports the smallest resistance value among the nodes of instance pin shapes for REFF.
- `worst`: Reports the largest resistance value among the nodes of instance pin shapes for REFF.

The default value is `worst`.

`-reff_report_all {true | false}`

	Default: `false` Reports both fail and pass instance pins in the REFF report. By default, only the "FAIL" instance pins are reported.
`-remove_duplicate_inst {true | false}`	
	Specifies to remove duplicate cell instances during the DEF parsing stage of the rail analysis run. These duplicate cell instance names DEF file, and are also printed in a warning message so that you are not required to debug these disconnected instances. When this pa the duplicate cell instances within the same DEF file will not be reported as disconnected instances in the *Physically Disconnected Ins* power-grid integrity report. The default value is `false`.
`-report_instances_missing_current_data {true | false}`	
	Specifies to generate a report that includes a list of all instances with missing current data. This parameter is supported in both the net-l based analyses. When this parameter is specified, the `<net_name>.nocurrentdata.inst.rpt` file is created in the `<state directory>/<net_name>/` direc file includes the annotation statistics for instances with current data available on a per-net basis, as shown below: `#Instances missing current data: #count/#total_instances (%#percent value)` *Default* : `false`
`-report_layer_based_ir_average {true | false}`	
	Specifies to add the average IR drop value to the layer-based IR drop report and log file.
`-report_layers_above_pin_for_instance_ir {true | false}`	
	Specifies to report nodes above the pin layer. With this parameter set to `true`, the report will also contain layers above the pin nodes to layer. The reporting will be based on the largest rectangular bounding box of the instance.
`-report_power_options option_names`	
	This parameter is used when `-report_power_in_parallel` is set to `true`. When the `-report_power_in_parallel` parameter is specified, you will not run the `report_power` command, therefore the reporting para specified. Using the `-report_power_options` parameter, you can specify the required `report_power` command parameters that are to be with rail analysis.
`-report_msmv_format {true | false}`	
	When set to `true`, the software generates an instance voltage file for each MSMV power domain in the design, e.g. VDD_VSS.iv and V VDD and VDDm are MSMV power nets with common VSS net. *Default* : `true`
`-report_power_in_parallel {true | false}`	
	Enables power analysis in parallel with rail analysis to improve the overall turnaround time. For information on the parallel flow setup, refer to the "**Run Dynamic Power Analysis and Rail Analysis in Parallel**" section in the *Dy IRDrop Analysis chapter* of the *Voltus User Guide*.
`-report_shorts {true | false}`	

Specifies to check for power and ground shorts between two geometries belonging to different nets. The parameter performs power and for the following:

- PG and PG nets
- PG and signal nets
- All extracted nets

When the parameter is set to `true`, if shorts are detected, the file "`voltus_short.report`" will be generated in the extraction work directo report file is:

LayerName X-coord Y-coord NetName1 NetName2

If one of the net is not an extracted net (that is, signal net or non-extracted PG net), it will be marked as N/A in the report.

The following is the use model of the parameter:

```
set_rail_analysis_mode -report_shorts true
```

The following is a snippet from `voltus_short.report`:

```
# Layer   X          Y       NET         NET

   AP  98.500    211.000   VSS         N/A
   AP  98.500    219.550   VSS         N/A
   AP  98.500    301.000   VDD25_LVTL N/A
   AP  98.500    309.550   VDD25_LVTL N/A
   M8  13007.800 2267.200  VSS         VDD
...
```

`-report_voltage_drop {true | false}`

When set to `true`, instead of reporting the actual (effective) instance voltage, all the values will be reported in terms of voltage drop/bou full pwr-gnd is 1V, pwr is 0.9 and gnd is 0.1, the IV file will report 0.8 (0.9-0.1) if this parameter is set to `false`, or 0.2 (0.1drop+ 0.1bounc set to `true`.

`-reset`

Specifies to reset all the `set_rail_analysis_mode` command parameters to their default values.

`-reuse_state_directory` *dir_name*

Specifies an existing state directory. The software assumes that the design data (DEFs and GDS) and power-grid libraries are not char analysis of the design using power-grid database inside the specified state directory. If the state directory does not contain the necessa files, it will issue an error message and provide information on the missing files.
For net based analysis, you should specify net-based state directory, as shown below:

- `set_rail_analysis_mode -reuse_state_directory` *VDD_25C_avg_1*

For domain based analysis, you should specify the domain's state directory, as shown below:

- `set_rail_analysis_mode -reuse_state_directory` *domain_25C_dynamic_1*

A new state directory will be generated for the new analysis with soft links to the old state directory files that were reused.

`-rlrp_cell_report_layer` *filename*

Specifies the cell names and layer of the pin shapes that are to be reported in the RLRP report. This parameter allows you to perform c reporting for the instance RLRP report.

The format of the specified input file is:

```
#cell_name #netname #pinlayer
```

`netname` is either a netname or "`ALL`". If you specify `ALL` for a net name, all the nets in the analysis domain will be considered for RLRP r

Example of an Input File

```
PDVDD_18_18_NT_DR_V ALL {M9,M1}
PDVDD_18_18_NT_DR_V DVDD_D1 {M9,M1}
PDVDD_18_18_NT_DR_V DVDD_D1 {M9,M1} VSS_D1 {M2}
```

`-rlrp_detail_report {true | false}`

	Specifies to trace and report the total power gate resistance along the RLRP path for each instance. The RLRP report (`<netName>.rlrp` following additional columns for each instance: • `RLRP_TOTAL` - Total RLRP resistance including the power gate path (from the always-on and switched nets) • `NET_RES` - Total RLRP resistance excluding the power gate path • `PGATE_RES` - RLRP resistance for the power gate path • `PGATE_RON` - Number of power gate RON resistors The report allows you to trace the Ron resistors in each power gate instance and generate the RLRP report based on power gate instar The default value of this parameter is `false`.
`-rlrp_eval_nodes {port | tap}`	
	Reports RLRP based on the port (pin) or tap nodes. By default, the software uses the port nodes for RLRP reporting.
`-rlrp_percentage_threshold` *value*	
	Specifies to output top x% of the instances with large RLRP.
`-rlrp_pin_based_report {true | false}`	
	Specifies to generate a pin-based RLRP report. When specified, a new column "*Pin*" is included in the RLRP report.
`-rlrp_pin_report_layer { top | bottom | all }`	
	Specifies which layer of the pin shapes are to be reported in the RLRP report. This parameter allows you specify which cell boundary p for the resistance report. The possible arguments of this parameter are: • `top`: Specifies to report only the pin shapes located on the highest pin layer. • `bottom`: Specifies to report only the pin shapes located on the lowest pin layer. • `all`: Specifies to report the pin shapes with best or worst resistance value among all layers. The default value is `all`.
`-rlrp_pin_report_method { best | worst | eiv_best | eiv_worst}`	
	Specifies whether to report the best or worst effective resistance value of an instance pin with multiple nodes. The possible arguments of these parameters are: • `best`: Reports the smallest resistance value among the nodes of instance pin shapes for RLRP. • `worst`: Reports the largest resistance value among the nodes of instance pin shapes for RLRP. • `eiv_worst`: Reports the RLRP of the node that has the worst EIV value among the tap nodes of instance pin shapes. • `eiv_best`: Reports the RLRP of the node that has the best EIV value among the tap nodes of instance pin shapes. The default value is `worst`.
`-rlrp_threshold` *value*	
	Specifies to ignore instances with RLRP less than the specified threshold value.
`-rms_em_analysis {true | false}`	
	Specifies to calculate the RMS EM in the dynamic rail analysis run. When not specified, it will be enabled automatically if the EM mode You can set this parameter to `false` to skip the RMS EM analysis even if the EM models are provided. This parameter generates the `<n` report. *Default* : `false`
`-save_voltage_waveforms {true | false}`	
	When set to `true`, instance voltage waveforms are saved in the state directory (.ptiavg). This waveform file can be loaded in the `view_dy` command to probe voltage waveform for desired instances. This waveform file can be fed to Tempus to perform SPICE critical path ana waveforms. It can be used in Substrate Noise Analysis as well.
`-scale_initial_condition_current` *filename*	

	Specifies a file containing the current scale factor for the initial condition of the specified nets. The parameter allows you to increase/de current at time 0 to minimize oscillation in the package and to shorten the settling time.
	The format of the file is:
	`default <default scale factor for all nets>`
	`<net name> <scale factor for this net>`
	`...`
	The use model of this parameter is:
	`set_rail_analysis_mode -method dynamic -scale_initial_condition_current scale_factor.file`
	The content of *scale_factor.file* is:
	`default 1.0`
	`VDD 0.8`
	`VSS 0.7`

`-seb_lifetime value`	
	Specifies the life time of product in hours. *Default* : `43800` hours (that is, 5 years)
`-seb_table table_filename`	
	Specifies the file name of the Failures in Time (FIT) table. This file is provided by the foundry. The file has the following columns: • Layer • MTF • Sigma • Activate energy • Current density exponent
`-seb_temperature temperature`	
	Specifies the temperature used to update the median time to failure (MTF) with respect to the ambient temperature. • If this temperature is specified, it would be used globally for all the resistors. • If this temperature is not specified, the seb_temperature may take the following possible values depending on the specific circumsta ◦ If detailed delta temperature file is not used for SEB, the `seb_temperature` always takes the same value as that used for EM analy ◦ If detailed delta temperature file is used for SEB, the seb_temperature is calculated by the following equation: `seb_temperature = env_temperature + delta_T` Where delta_T is the maximum temperature rise, including joule heat and coupling heat from FEOL, in the resistor's bounding bo
`-set_analyze_bbox {xmin ymin xmax ymax}`	
	Specifies the X, Y, min and max coordinates of the bounding box in which the instances can be analyzed. The values of the coordinate microns (um).
`-shorting_resistance value`	
	Specifies the minimum shorting resistance value (in ohm) assigned to resistors. When specified, those resistors with resistance equal c specified value are merged to reduce the size of the R network. *Default* : `0`
`-skip_extraction {true | false}`	
	Specifies to reuse a work directory. It is recommended to use this parameter when you want to reuse the previous extraction work direc net rail analysis flow. *Default* : `false`
`-snap_layer_for_current_taps` *file_name*	

Specifies the layer file name for adaptive current modeling. It snaps current taps from lower via layers to higher via layers for better perf is supported for EM view of both macro PGV and hierarchical PGV. The current taps from lower layers will be processed based on the redistributed equally across all the vias within the tile at the higher via layers. The default tile size is `5umx5um (25 um2)` for macro PGV, a `um2`) for hierarchical PGV.

For example, when running full chip analysis, you already have detailed analysis on the sub-blocks or hierarchical blocks so you might the analysis by merging the current taps from lower layer to spread it across higher via layers. The current drawn from within the PGV w same so the accuracy at the top level will not be affected.

The format of the file is:

#Instance Based
```
INST <inst_name> <via_layer_name> <tile_size>
```

#Cell Based
```
CELL <cell_name> <via_layer_name> <tile_size>
```

`via_layer_name` is the layer name used in the LEF file.
`tile_size` is optional.

`-static_multi_mode_analysis {true | false}`

Specifies to analyze the worst-case static IR drop across multiple functional modes of operation for macros. When this parameter is spe will use the multi-mode static current file (`set_power_analysis_mode -static_multimode_scenario_file`) to iterate over multiple scenar instances, and output the worst-case IR and EM report. The generated report identifies the worst Differential Instance Voltage (DIV) and produces worst DIV across all instances.

Default : `false`

This parameter generates the following report in the Reports folder:

`State_dir/Reports/powerNetName_gndNetName``.worst_scenario_div.rpt` - this file reports each instance's worst scenario. It lists the ins DIV in the ascending order to determine the worst among all worst instance DIVs.

The following is a snippet from the worst scenario report:

```
#INST_NAME      SCENARIO_ID   DIV
block19/ICG4/ICG    12     -0.107583
block19/ICG3/ICG    12     -0.107557
...
```

Here, `SCENARIO_ID` is the worst time in the current file.

`-static_trigger_file` *`filename`*

Specifies a file containing user-defined xPGV current scaling information on a per PG net basis for static analysis. This parameter is us currents stored in xPGV.

You must provide scaling for all PG nets associated with the xPGV in the trigger file. The parameter allows you to use the same xPGV i irrespective of the voltage at which the xPGV is generated.

The format of the user-specified trigger file for a net is:
```
<INSTANCE|CELL>   inst_name/cell_name  time  scale_factor   mode  ref   net
```

Instance-based scale factors get higher precedence than the cell- based scale factors.

Example of a Trigger File:
```
INSTANCE    inst_C1    0   3   IPPGV::DEFAULT::super_filter   0   VDDD1
INSTANCE    inst_C1    0   2   IPPGV::DEFAULT::super_filter   0   VDDC1
INSTANCE    inst_C1    0   3   IPPGV::DEFAULT::super_filter   0   VSSA1
```

`-enable_xpgv_scaling` and `-static_trigger_file` are mutually exclusive.

`-suppress_message` **{** *`message_id`* **+ }**

The messages with the specified message ids will not be output.

Default : No messages are suppressed.

`-switchbit_file_for_eiv_calculation` *`filename`*

	Specifies the file that includes the cell or instance based switch-bit information. This parameter allows you to override the switch-bit in a switch-bit for a specific cell or instance.
	Note: The switch-bit information is defined for memory cells to distinguish between the pre-charge and functional mode (read/write) ope charge, the memory cells may have large IRdrops due to large demand current in a short period of time. However, these cells can tolera because word-lines are shut-off. To accurately measure the effect of IRdrop on the operation of a memory, the instance voltage should the functional mode. The switch-bit signal is used by the software to calculate the instance voltage when memory is in the functional mo
	The format of the specified file is:
	`CELL | INST <cell_name | instance_name>`
	`<MODE_NAME> <Tmin1> <Tmax1> <Tmin2> <Tmax2>`
	where,
	• *cell_name*/*instance_name* must have a PGV with the dynamic current waveform.
	• `MODE_NAME` must match one of the mode names in the PGV for the cell.
	• `Tmin` and `Tmax` is the window when the switch-bit is ON, and when EIV will be calculated. The default time unit is ns.
	The Rail Analysis engine reads this switch-bit information, and reports the elapsed and worst instance voltage drop (IVD) when the swi Analysis will generate two IVD values for a given instance in the EIV report:
	• Worst IVD across simulation (`ELAPSE_EIV`)
	• Worst IVD in switching window (`WIN_EIV`)

`-temp_directory_name` *directory*	
	Specifies a directory to use for temporary files. You can also use the global environment variable `TMPDIR` to specify an alternative location for the temporary data. If both are specified, using the `-temp_directory_name` parameter will take precedence. Following is an example for setting the TMPDIR environment variable: `setenv TMPDIR ./temp_directory`
`-temperature` *value*	
	Specifies the temperature in degree celcius during rail analysis. The default temperature is 25 degree celsius. This is an optional parameter.
`-topcell_placement {X Y}`	

	Specifies to place the top DEF block at a given X,Y location (in micron) with respect to virtual top (0,0).
	The following is the usage model:
	`set_rail_analysis_mode -topcell_placement { X Y } -topcell_orientation { N|S|W|E|FN|FS|FE|FW }`
	For GUI display, the same parameters also work for `read_def`:
	`read_def -topcell_placement { X Y } -topcell_orientation { N|S|W|E|FN|FS|FE|FW }`

`-topcell_orientation { N|S|W|E|FN|FS|FE|FW }`	
	Specifies the orientation of the top DEF block.
`-tsv_subckt_modelfile_list` *filename*	

	Specifies the name of the external Through Silicon Via (TSV) subckt model files to be used for 3D-IC extraction. You can specify a list space. Each file contains the subckt model information for TSV. The specified file has the Spice format.
	A sample file is shown below.
	`.subckt TSV top bottom`
	`R1 top n2 0.03075`
	`R2 n2 bottom 0.03075`
	`C1 n2 0 28f`
	`.ends`

`-unconnectedcell_ignore_file` *filename*	

	Specifies the name of a file containing the list of cells and instances to be ignored while reporting connectivity issues in the .rpt files, log parameter allows you to filter out the cells/instances that are known to have connectivity related issues and are not to be fixed. The format of the file is: `CELL/INST  Cell Name/Instance Name  Pin Name` Here: • `CELL` and `INST` are keywords • Pin names are not mandatory. If pin names are not given, the corresponding cell/instance is ignored. **Example**: `CELL AND` => ignore all AND cell(s) `INST INST1` => ignore instance INST1 `CELL BLOCK2*` => Ignore all blocks with the name starting with BLOCK2 `CELL *` => ignore all cells `INST *` => ignore all instances `CELL AN1 VDD*` => ignore AN1 cells's VDD* pins `INST INST2 {VDD1 VDD2}` => ignore multiple pins of instance INST2 `INST INS* VDD` => ignore VDD pin of all instances with the name starting with INS
`-unconnected_die_pkg_pins {ignore | error | edit}`	
	Specifies whether or not to ignore unconnected pins between die and package (if any), and consequently if the simulation should be pa continued. This option enables the manual connection between package and die through the *MCP Editor* utility if any disconnects are p The default value is `ignore`.
`-unified_power_switch_flow {true | false}`	
	Specifies to enable the unified analysis of the always-on and switch nets. When set to `true`, the power-up and always-on analyses are flow, wherein the switch nets and always-on nets are simulated together resulting in improved performance and accuracy. **Use Model**: • Always-on Flow: `set_rail_analysis_mode -unified_power_switch_flow true` • Power-up Flow: `set_rail_analysis_mode -unified_power_switch_flow true` and `-powering_up_rails <powerup_rail_list>`
`-use_early_view_list` *filename*	
	Specify cells in the file for which you want to use Early power-grid views. The file format accepts cell name in each line. You can specif cell names.
`-use_em_view_list` *filename*	
	Specify cells in the file for which you want to use EM power-grid views. The file format accepts cell name in each line. You can specify names.
`-use_ir_view_list` *filename*	
	Specify cells in the file for which you want to use IR power-grid views. The file format accepts cell name in each line. You can specify w names.
`-use_rms_delta_t {true | false}`	
	When this parameter is set to `true`, the `delta_T` used to calculate `seb_temperature` would be based on the RMS delta T (joule heat only If this parameter is set to `false`, the `delta_T` used to calculate `seb_temperature` would be based on the sum of self-heat delta T and RM *Default* : `false`
`-verbosity {true | false}`	
	Specifies to get side logs for extraction and rail analysis. The default value of this parameter is `false`.
`-verify_logical_connectivity {true | false}`	
	Specifies to report the list power and ground pins of each cell instance that do not have logical connectivity. The list of pins are reported report.
`-vsrc_search_distance` *value*	

	Specifies the search distance in microns for adding voltage sources at power pin locations. *Default* : `50`
`-watch_location_waveform { {`*layerName xCoord yCoord*`}+ }`	
	Specifies to perform node based voltage probing for rail analysis. You can use this parameter to observe voltage waveform at a given l x,y coordinates and the metal layer. The unit of x,y coordinates is um, and the layer name should be a LEF/DEF layer.
`-what_if_shapes_file` *filename*	
	Specifies to take the what-if shape specification from a file. This file is the file created with the `-write` parameter of the `add_what_if_sha` **Note**: If both the `-import_what_if_shapes` and `-what_if_shapes_file` parameters are specified, union of two sets of what-if shapes is c
`-work_directory_name` *directory*	
	Specifies the work directory location for extraction data. When you specify this parameter, the software generates extraction temporary directory.
`-write_demand_current_region_file` *filename*	

	Specifies a file that contains regions for custom demand current generation. This parameter allows to compute the total current within e sum up all tap currents that are outside the specified regions as a global port.
	The format of the region file is:
	`<REGION_PORT_LABEL> <XL YL XR YR> <NET>`
	here,
	• `REGION_PORT_LABEL` specifies a name for the region.
	• `XL YL XR YR` specifies the enclosing region coordinates in microns.
	• `NET` specifies the power/ground net name for analysis.
	Example file:
	`A 0 0 100 100 VDD_AO`
	When specified, one demand current waveform "`tran_<netname>_region.ptiavg`" will be generated per net in the rail output directory a
	• For DP: `./Reports/Waveforms/tran_<netname>_region.ptiavg`
	• For XP: `./reports/rail/waveforms/tran_<netname>_region.ptiavg`

`-xpgv_config_file` *filename*	

	Specifies block details, such as, cell name for which xPGV is generated, via layer at which current is saved, block LEF file names, and for which xPGV is generated. This parameter is specified during block-level analysis.
	Configuration File Format:
	`CELL <`*block_name*`>`
	`CUT_LAYER <`*cut_via_layer_name*`>`
	`Lef <`*tech_lef_name block_lef_name*`>`
	`END`
	Note: This parameter is not supported with the `analyze_resistance` command.

`-xp_cpu_per_job_power` *value*	
	Specifies the number of CPUs per power analysis job in the XP mode (`-enable_xp`). The specified value must be greater than `0`. This value should not exceed the value specified with `set_multi_cpu_usage -cpuPerRemot` is not specified, the value will be determined automatically based on the number of host resources.
`-xp_cpu_per_job_simulation` *value*	
	Specifies the number of CPUs per simulation job in the XP mode (`-enable_xp`). The specified value must be greater than `0`. This value should not exceed the value specified with `set_multi_cpu_usage -cpuPerRemot` is not specified, the value will be determined automatically based on the number of host resources.
`-xp_host_allocation_method {on_demand | at_startup}`	

	Specifies the method for remote host allocation during distributed processing. The possible arguments are: • `at_startup` – Specifies to wait for all available remote hosts before starting rail analysis. This parameter allocates all available remo `set_multi_cpu_usage` at startup. • `on_demand` - Allocates the available remote hosts based on demand. *Default* : `on_demand`
`-xp_purge {full}`	
	Specifies to clean up disk after run completion to delete temporary directories created for each analysis step (data generated in the volt this parameter is specified, the reuse parameters (`-reuse_state_directory and -xp_reuse_extraction_directory`) will not work for th in the future runs. This parameter is applicable only in the XP mode (`-enable_xp`).
`-xp_resume {true | false}`	
	Specifies to resume from the failed step, and skip the previously successful run steps. This parameter is applicable only in the XP mod *Default* : `false`
`-xp_reuse_extraction_directory` *directory_name*	
	Specifies the path to the state directory that contains the extraction power-grid database. This parameter allows you to skip extraction a data from a previous run in the XP mode.
`-xp_simulation_cpu_timeout` *value*	
	Specifies the time limit to either acquire the number of CPUs requested at the start of the run or acquire the number of CPUs specified v `xp_simulation_min_cpu` parameter in the XP mode. When this parameter is specified, the tool will wait for the given time limit to acquire simulation before exiting. The unit for timeout is seconds. If a timeout value is not specified, the tool will wait for 7200 seconds (two hours) to acquire the CPUs. *Default* : `7200`
`-xp_simulation_min_cpu` *value*	
	Specifies the minimum number of CPUs required for the rail simulation stage in the XP mode. The specified value must be greater than When this parameter is specified, the tool waits to acquire the specified number of CPUs before performing simulation. This parameter want to perform simulation with the minimum number of CPUs required to proceed with simulation instead of waiting for all the CPUs n If this parameter is not specified, the tool performs simulation with the available number of CPUs.

Examples

- The following command sets the rail analysis mode to static and xd accuracy, along with some other parameters:

```
set_rail_analysis_mode \
-method static \
-accuracy xd \
-power_grid_library fast_library/fast_allcells.cl \
-vsrc_search_distance 50
```

- The following command is an example of setting up rail analysis in the XP mode:

```
#resume previously failed XP run
set_rail_analysis_mode -enable_xp true -xp_resume true

set_rail_analysis_mode -method dynamic -accuracy hd \
...
-enable_xp true \
-xp_host_allocation_method at_startup \
-xp_reuse_extraction_directory dynamic_rail/core_125C_dynamic_1 \
-xp_cpu_per_job_power 2 \
-xp_cpu_per_job_simulation 8 \
-xp_purge full
```

- The following is an example of net-based early rail static analysis on an unplaced or partial placed design where virtual via/followpin would be created.

The power would be distributed to placed instances on the basis of area. For unplaced instances, current region would be created on the M1 layer.

```
set_pg_nets -net vdd -voltage 0.998 -threshold 0.99301
set_power_pads -net vdd -format xy -file vdd.pp
set_power_data -format area -bias_voltage 0.998 -power 2.15
set_rail_analysis_mode -method era_static -accuracy xd -extraction_tech_file ./qrcTechFile -era_current_distribution_layer M1 -
era_current_distribution unplaced
analyze_rail -type net vdd
```

- The following is an example of domain-based early rail static analysis with virtual via/followpin. Here, user-specified instance current files are used.

```
set_pg_nets -net VSS -voltage 0 -threshold 0.18
set_pg_nets -net VDDm -voltage 1 -threshold 0.856
set_pg_nets -net VDD -voltage 0.84 -threshold 0.756
set_power_pads -net VDD -format xy -file vdd.pp
set_power_pads -net VDDm -format xy -file vddm.pp
set_power_pads -net VSS -format xy -file vss.pp
set_power_data -format current {instance_current_files/static_VSS.ptiavg instance_current_files/static_VDD.ptiavg
instance_current_files/static_VDDm.ptiavg  instance_current_files /static_VDDlu.ptiavg instance_current_files/static_VDDau.ptiavg}
set_rail_analysis_mode -method era_static -accuracy xd -extraction_tech_file typical.tch -era_power_gate_file pwrgates_port.txt
set_rail_analysis_domain -name PD -pwrnets {VDD VDDm} -gndnets VSS
analyze_rail -type domain PD
```

- The following is an example of net-based early rail analysis with user specified ASCII instance and current region file. Here, all virtual vias and followpin creation have been skipped.

```
set_pg_nets -net vss -voltage 0.0 -threshold 0.01
set_power_pads -net vss -format xy -file vss.pp
set_power_data -format ascii -bias_voltage 1 -power 1.5 instance_power_file.txt
set_power_data -bias_voltage 1 -power 1.5

set_rail_analysis_mode -method era_static -accuracy xd -extraction_tech_file ./qrcTechFile -era_current_region_file
./cur_region_file  -era_insert_virtual_followpins none -era_skip_virtual_via_on_layers MI M2

analyze_rail -type net vdd
```

- The following is an example of static early rail analysis with current distribution for unplaced instances by total die area:

```
set_rail_analysis_mode -power_grid_library fast_library/fast_allcells.cl -extraction_tech_file typical.tch -accuracy hd -ignore_shorts
true  -method era_static -enable_rlrp_analysis true -vsrc_search_distance 50  -era_current_distribution unplaced -
era_current_distribution_layer Metal1 -era_current_distribution_unplaced_area diearea -era_techlib_generation false -
era_check_wires_for_generated_current_regions true
```

- Let us consider a design with cell type A that has the M1 and M2 PG pin shapes, and cell type B that has the M3 and M4 PG pin shapes.
  The following command calculates the reff values of the interface nodes on the M2 pin shapes for the cell type A and on the M4 pin shapes for the cell type B, and reports the worst (biggest) value among them. Similarly, it calculates the rlrp values of the interface nodes on the M1 pin shapes for the cell type A and on the M3 pin shapes for the cell type B, and reports the best (smallest) value among them.

```
set_rail_analysis_mode \
      -reff_pin_report_layer top \
      -reff_pin_report_method worst \
      -enable_rlrp_analysis true \
      -rlrp_pin_report_layer bottom \
      -rlrp_pin_report_method best \
…

analyze_resistance
```

- The following command reads the switch-bit information in the specified memory cells:

```
set_rail_analysis_mode \
  -power_grid_library "${Libgen_Golden_Pgv}" \
  -accuracy hd \
  -ignore_shorts true \
  -method dynamic \
  -enable_rlrp_analysis true \
  -eiv_eval_window switching  \
  -limit_number_of_steps false \
  -watch_location_waveform u0 \
  -switchbit_file_for_eiv_calculation  sw.list
```

The following is a snippet of the sw.list file:

```
CELL MEM128Kx32
READ 0.1ns 0.13ns
WRITE 0.1ns 0.15ns
CELL MEM64Kx32
READWRITE  0.1ns 0.2ns 0.3ns 0.4ns
```

The following is a snippet of the report:

```
INST_NAME WIN_EIV PWR_WIN_IV GND_WIN_IV WIN_TIME ELAPSE_EIV PWR_ELAPSE_IV GND_ELAPSE_IV ELAPSE_TIME CELL_NAME
*"NA" means the instance is disconnected from the net or the instance does not have timing/switching window
BEGIN
- external/MULT8_reg_3_6 0.75841 0.85220 0.09379 48.470 0.75841 0.85220 0.09379 48.470 SDFFQXL
```

- The following command specifies the instance-based PWL current file in the dynamic ERA flow:

  ```
  set_rail_analysis_mode -method era_dynamic -accuracy hd -vsrc_search_distance 500 -era_instance_dynamic_current_file
  ./inst_cell_pwl_file -power_grid_library ./../techonly.cl
  ```

  The following is a snippet of the PWL file:

  ```
  instance ram/ram3 net VDD pwl (0ns 0mA 0.1ns 1mA 0.2nA 0.5mA) intrinsic_cap 10pf loading_cap 60pf
  instance ram/ram2 net VDD pwl (0ns 0mA 0.1ns 1mA 0.2nA 0.5mA) intrinsic_cap 10pf loading_cap 60pf
  ...
  ```

- The following command specifies different temperature values to the layers M1, M2, M3, VIA1, VIA2, and VIA3 for average current limit analysis:

  ```
  set_rail_analysis_mode
  ...
     -em_temperature 110 \ #temperature for the remaining layers
     -em_temperature_layer_list {M1 125 M2 115 M3 105 VIA1 115 VIA2 105 VIA3 120} #layer-based temperature
  ```

  The log file will display the following info message:

  ```
  Layer specific EM temperature will be used for AVG check for layer M1(125C), M2(115C), M3(105C), VIA1(115C), VIA2(105C), VIA3(120C).
  ```

  The header of the power EM report displays the layer-based temperature, as shown below:

  ```
  # Temperature: 110.0 (C)
  # Jmax_factor:
  # ……
  #   M2  0.981000 @ 115.0 (C)
  #  ……
  ```

- The following command specifies to perform unified power switch flow:

  ```
  set_rail_analysis_mode -method dynamic -accuracy hd \
  ....
  -powering_up_rails "VDD_sw1 VDD_sw2" \
  -enable_xp true \
  -unified_power_switch_flow true
  ```

Related Information

- analyze_rail
- "Static Power, IRdrop and EM Analysis" in the *Voltus IC Power Integrity Solution User Guide*
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

set_self_heat_analysis_mode

```
set_self_heat_analysis_mode
[-help]
[-alpha_parameters {{<layer_name> < α_overlapping > < α_connecting >}+}]
[-beta_parameters {C1 C2 C3}]
[-cell_thermal_resistance_file filename]
[-detail_delta_temperature_file filename]
[-detail_delta_temperature_region {x1 y1 x2 y2}]
[-die_instance_input_signal_em filename]
[-die_instance_name dieinstname]
[-instance_delta_temperature_file filename]
[-instance_power_file filename]
[-report_conn_pin_wire filename]
[-scale_rms_limit value]
[-tile_delta_temperature_file filename]
[-tiles {n_tile_in_x m_tile_in_y}]
```

Specifies the number of tiles, layer-based α coefficients, β coefficients, and list of cells with cell thermal resistances. This command will use the static power data to compute delta T FEOL for all instances in the design. This command:

- computes the RMS currents on signal wires
- computes the RMS currents on power network wires
- generates Detail Delta Temperature File (ddt), Instance Delta Temperature File (idt), and Tile Delta Temperature File (tdt)

The command supports both single-die and multi-die flows.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `set_self_heat_analysis_mode` parameter. For a detailed description of the command and all of its parameters, use the man command `man set_self_heat_analysis_mode`.
`-alpha_parameters {{<layer_name> < αoverlapping > < αconnecting >}+}`	
	Specifies the alpha parameters for all layers. It is the metal layer effect.
`-beta_parameters {C1 C2 C3}`	
	Specifies the thermal coupling effect coefficient Value.
`-cell_thermal_resistance_file filename`	
	Specifies the name of the cell's equivalent thermal resistance file. The specified file contains the list of cells and their corresponding thermal resistance value. The unit of thermal resistance is in °C/W (temperature rise per unit rate of heat dissipation). The format of the file is: `<cell_name> <thermal_resistance_in_C/W>` To get instance deltaT, use half of the instance power multiplied by the cell thermal resistance.
`-detail_delta_temperature_file filename`	
	Specifies the name of the generated BEOL detailed delta-temperature file.
`-detail_delta_temperature_region {x1 y1 x2 y2}`	
	Specifies the region coordinates for generating detailed delta temperature file.
`-die_instance_input_signal_em filename`	
	Specifies the setup Tcl script for each die instance.
`-die_instance_name dieinstname`	
	Specifies the die instance name for multi-die analysis. This option is only available in multi-die analysis (`set_rail_analysis_mode -die_mode multi-die`).

`-instance_delta_temperature_file` *filename*	
	Specifies the name of the generated FEOL instance-based delta-temperature file.
`-instance_power_file` *filename*	
	Specifies a file for instances and their corresponding static power. It is a two column format of <*instance_name*> and <*total_power*>.
`-report_conn_pin_wire` *filename*	
	Reports results on wires connecting to cell pins.
`-scale_rms_limit` *value*	
	Specifies the scale factor for the root-mean-square (rms) limit relaxation factor during Electromigration current limit analysis.
`-tile_delta_temperature_file` *filename*	
	Specifies the name of the generated FEOL/BEOL tile-based delta-temperature file.
`-tiles` *{n_tile_in_x m_tile_in_y}*	
	Specifies the number of tiles in the x and y directions in the tile-based delta temperature file.

Examples

- The following command is an example of `set_self_heat_analysis_mode` command:

```
set_self_heat_analysis_mode \
-die_instance_name die0 \
-alpha_parameters {{M1 0.74 0.85} {M2 0.62 0.85} {M3 0.53 0.85} {M4 0.42 0.85} {M5 0.32 0.85} {M6 0.26 0.85} {M7 0.26 0.85} {M8 0.26
0.85} {M9 0.26 0.85} {M10 0.26 0.85} {M11 0.26 0.85} {M12 0.26 0.85}} \
-beta_parameters {0.00423 -0.00045 0.81534}\
-instance_power_file ./dynamic_power.rpt \
-cell_thermal_resistance_file TRF.txt \
-instance_delta_temperature_file sh_itm.txt \
-tile_delta_temperature_file sh_ttm.txt \
-tiles {10 10} \
-detail_delta_temperature_file power_detail_die0.rpt
```

```
set_self_heat_analysis_mode \
-die_instance_name die1 \
-alpha_parameters {{M1 0.74 0.85} {M2 0.62 0.85} {M3 0.53 0.85} {M4 0.42 0.85} {M5 0.32 0.85} {M6 0.26 0.85} {M7 0.26 0.85} {M8 0.26
0.85} {M9 0.26 0.85} {M10 0.26 0.85} {M11 0.26 0.85} {M12 0.26 0.85}} \
-beta_parameters {0.00423 -0.00045 0.81534}\
-instance_power_file ./dynamic_power.rpt \
-cell_thermal_resistance_file TRF.txt \
-instance_delta_temperature_file sh_itm.txt \
-tile_delta_temperature_file sh_ttm.txt \
-tiles {10 10} \
-detail_delta_temperature_file power_detail_die1.rpt
```

```
set_self_heat_analysis_mode \
-die_instance_name die2 \
-alpha_parameters {{M1 0.74 0.85} {M2 0.62 0.85} {M3 0.53 0.85} {M4 0.42 0.85} {M5 0.32 0.85} {M6 0.26 0.85} {M7 0.26 0.85} {M8 0.26
0.85} {M9 0.26 0.85} {M10 0.26 0.85} {M11 0.26 0.85} {M12 0.26 0.85}} \
-beta_parameters {0.00423 -0.00045 0.81534}\
-instance_power_file ./dynamic_power.rpt \
-cell_thermal_resistance_file TRF.txt \
-instance_delta_temperature_file sh_itm.txt \
-tile_delta_temperature_file sh_ttm.txt \
-tiles {10 10} \
-detail_delta_temperature_file power_detail_die2.rpt
```

Related Topics

- "Self-Heating Effect Analysis" in the *Voltus User Guide*

set_voltage_regulator_module

```
set_voltage_regulator_module
-component_list {VRM_DEF_component_list}
-ground_pin_mapping {{spice_pin_name[DEF_net_name]}+}
-input_pwr_pin_mapping {{spice_pin_name[DEF_net_name]}+}
-name vrm_name
-netlist vrm_spice_netlist_file_name
-output_pwr_pin_mapping {{spice_pin_name[DEF_net_name]}+}
-reset
-subckt_name vrm_subckt_name
```

Analyzes voltage regulator effects. Use this command to enable the Vreg flow.

You must specify this command before running the `analyze_rail` command.

Parameters

`-component_list {VRM_DEF_component_list}`
Specifies the names of the DEF instances which comprise the VRM. The component name must be the instance name used for the specified VRM. If this parameter is not specified, it is assumed that VRM is represented as a single MACRO on the die.
`-ground_pin_mapping {{spice_pin_name[DEF_net_name]}+}`
Specifies the mapping between the ground pin name in the SPICE netlist and the corresponding DEF net name for the same power pin.
`-input_pwr_pin_mapping {{spice_pin_name[DEF_net_name]}+}`
Specifies the mapping between the input power pin name in the SPICE netlist and the corresponding DEF net name for the same power pin.
`-name vrm_name`
Specifies the VRM name assigned to a voltage regulator. You need to specify this parameter for each voltage regulator in the design.
`-netlist vrm_spice_netlist_file_name`
Specifies the VRM netlist file in the SPICE format that is encapsulated in a single sub-circuit. This should also include SPICE models and testbench for each voltage regulator.
`-output_pwr_pin_mapping {{spice_pin_name[DEF_net_name]}+}`
Specifies the mapping between the output power pin name in the SPICE netlist and the corresponding DEF net name for the same power pin.
`-reset`
Resets all specified parameters back to default values.
`-subckt_name vrm_subckt_name`
Specifies the subcircuit that corresponds to the VRM. This parameter is specified when the specified SPICE netlist has multiple subcircuit definitions. If this optional parameter is not specified, the first subcircuit found in the netlist file will be used.

Examples

- The following command performs Vreg analysis for the VRM `vrm1`:

  ```
  set_voltage_regulator_module -name vrm1 -netlist ./TEST.spi -input_pwr_pin_mapping {{VCC VCC}} -output_pwr_pin_mapping {{VDD VDD}} -
  ground_pin_mapping {{ VSS VSS }} -subckt_name LDO_MODEL -component_list { vmesh_reg_1 }
  ```

Related Information

- `set_rail_analysis_mode`
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

view_dynamic_movie

```
view_dynamic_movie
[-help]
[-type [ir|tc] ]
[-movies_directory dir]
```

Provides the ability to view movies created during dynamic analysis. Movies are sequences of analysis plots displayed over a specific simulation time.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `view_dynamic_movie` parameter. For a detailed description of the command and all of its parameters, use the man command `man view_dynamic_movie`.
`-movies_directory dir`	
	Specifies the movie directory that was generated during dynamic rail analysis (that is, `state_directory/movie_analysis`)
`-type [ir | tc]`	
	Specifies the movie type. `ir`: IRdrop `tc`: Tap current

Examples

- The below command views the dynamic movie of type `ir` in the specified state directory:

```
view_dynamic_movie -type ir -movies_directory \ dynamic_rail/TDSP_25C_dynamic_1/VDD_TDSPCore_R/movie_analysis
```

Related Information

- `set_rail_analysis_mode`
- `view_dynamic_waveform`
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

view_dynamic_waveform

```
view_dynamic_waveform
[-help]
[-type [current | voltage | profile]]
[-waveform_files {file1 file2 ... fileN}]
[-instance_name instance_name]
[-composite_waveform_type [hierarchy | clock | clock_with_seq
    | total_current]
[-composite_waveform_name [hierarchy_name | clock_name]]
[-power_db [powermeter.db]]
[-effective_voltage_waveform]
[-free_data]
[-state_directory directory_name]
[-hier_block_file filename]
[-clear_hier]
```

Provides the ability to view dynamic current or voltage waveforms generated during analysis.

Parameters

`-clear_hier`	Specifies to clear the hierarchical block data that was loaded using the `-hier_block_file` parameter.
`-composite_waveform_name [hierarchy_name | clock_name]`	
	Only available for `-type current`. If `hierarchy_name` is specified, plots total current waveform for the specified hierarchy. If `clock_name` is specified, plots current waveform for the selected or all clocks. If `-composite_waveform_type clock_with_seq` is specified, plots total current waveform for all sequential cells. If `-composite_waveform_type total_current` option is specified, plots total current waveform for all instances in the current file.
`-composite_waveform_type [hierarchy | clock | clock_with_seq | total_current]`	
	You can specify to view a waveform for a specific hierarchy, a clock, clock with sequential cells, or total current.
`-effective_voltage_waveform`	
	Plots the effective voltage waveform (i.e. VDD - VSS). You must specify the voltage for two nets to generate the effective waveform.
`-free_data`	Frees the memory but leaves the menu entries.
`-help`	Outputs a brief description that includes the type and default information for each `view_dynamic_waveform` parameter. For a detailed description of the command and all of its parameters, use the man command `man view_dynamic_waveform`.
`-hier_block_file filename`	

	Specifies to analyze the multiple (top/block-level) current (.ptiavg) files of a hierarchical design in the SimVision waveform viewer interface. This parameter can be used to easily find an instance of a specific block when there are multiple blocks with the same instance name.
	`-hier_block_file` specifies a text file that contains the list of blocks. For example, if the block name `BLOCK_A` is specified in the text file, you can match the instance named `BLOCK_A/Instance_A` and `Instance_A` in the `BLOCK_A`'s .ptiavg file.
	When specifying multiple block names in the text file, one block name should be given per line.
	The following is a snippet from the `block_hier` file:
	`u_top/vcore3`
	`u_top/vcore2`
	`u_top/vcore1`

`-instance_name instance_name`	

	Plot the waveform for the specified instance.
`-power_db [`*`powermeter`*`.db]`	
	Specifies the power database. Must be specified if `-composite_waveform_type` is `clock` or `clock_with_seq`.
`-state_directory` *`directory_name`*	
	Specifies to load the state directory where the current or voltage waveform files have been saved. The current and voltage waveforms are saved in the state directory generated during rail analysis. If you specify this state directory when viewing dynamic waveforms, the generated waveform file is automatically populated in the GUI.
`-type [current | voltage | profile]`	
	Specifies to display either current (power) or voltage waveform. The `profile` option displays the profiling db and the SimVision interface to view the histograms.
`-waveform_files {`*`file1 file2 ... fileN`*`}`	
	Specifies one or more current or voltage waveform files generated with a `.ptiavg` extension.

Examples

- The following command views dynamic composite total current waveforms from the file vddm_vddm.ptipeak:

```
view_dynamic_waveform -type current -composite_waveform_type total_current \
    -waveform_files vddm_vddm.ptipeak
```

- The following command views the dynamic current waveform for instance inst1:

```
view_dynamic_waveform \
    -type current \
    -waveform_files dynamic_vss.ptiavg \
    -instance_name inst1
```

- The following command is for viewing the total current waveform:

```
view_dynamic_waveform \
    -type current \
    -waveform_files dynamic_vss.ptiavg \
    -composite_waveform_type total_current
```

- The following command plots a clock domain composite waveform:

```
view_dynamic_waveform \
    -type current \
    -waveform_files dynamic_vss.ptiavg \
    -power_db power.db \
    -composite_waveform_type clock \
    -composite_waveform_name clk
```

- The following command plots a hierarchical composite waveform:

```
view_dynamic_waveform \
    -type current \
    -waveform_files dynamic_vss.ptiavg \
    -composite_waveform_type hierarchy \
    -composite_waveform_name ethernet_mac_2
```

Related Information

- `set_rail_analysis_mode`
- `view_dynamic_movie`
- "Dynamic Power and IRDrop Analysis" in the *Voltus IC Power Integrity Solution User Guide*

view_package_results

```
view_package_results
[-help]
[-result_file <filename>]
-type <package_analysis_type>
[-workspace <workspacename>]
```

Specifies to view the package IR drop analysis results in the PowerDC GUI which will be invoked from within Voltus/Innovus.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-result_file <filename>`	
	Specifies the name of the .xml result file.
`-type <package_analysis_type>`	
	Specifies the type of package analysis. Currently, the software supports only the `dc` package analysis type.
`-workspace <workspace_name>`	
	Specifies the workspace name for package analysis (powerDC). The file name extension is `.pdcx`.

Example

- The following example specifies to view the package analysis results

  ```
  view_package_results -type dc
  ```

Related Information

- `set_advanced_package_options`
- `extract_package`
- "Package Analysis" in the *Voltus IC Power Integrity Solution User Guide*

32

RC Extraction Commands and Global Variable

- RC Extraction Commands
- RC Extraction Global Variable

RC Extraction Commands

- create_parasitic_coordinate_transform
- extractRC
- generateCapTbl
- generateRCFactor
- getExtractRCMode
- rcOut
- read_parasitics
- report_net_parasitics
- report_rcdb
- report_unit_parasitics
- reset_parasitics
- restoreRC
- saveRC
- setExtractRCMode
- setShrinkFactor
- spefIn
- wireload
- write_extraction_spec

create_parasitic_coordinate_transform

```
create_parasitic_coordinate_transform
[-help]
{-inst inst_name
[-x_block_origin value]
[-x_offset value]
[-y_block_origin value]
[-y_offset value]
[-orient {R0 R90 R180 R270 MY MX90 MX MY90}]
[-relative_to_parent]} | {-reset}
```

Determines accurate location information of nodes during SPEF reading.

In case of hierarchical designs, there can be multiple blocks in a design that are implemented separately. During Top level STA analysis, SPEFs of these blocks are stitched together with top for flat STA analysis. This works well if pin location of blocks is not required. In case of AOCV analysis, when spatial analysis is enabled, accurate location of all nodes (pin/net) in the design is required for calculating accurate derates in the design. This makes it necessary to determine accurate nodes location during SPEF reading. This can be achieved by transforming SPEF coordinates as per their physical locations.

Note:

- This command works with all possible SPEF and RCDB input combinations with `read_parasitics` command, which means for only RCDB(s) as input, mix of SPEF(s) and RCDB(s) as input and only SPEF(s) as input.
- If this command is not specified, the `spefIn` command tries to automatically find the orientation and origin of the hierarchical SPEF blocks and applies those identified transforms to the parasitic co-ordinates.
- The `create_parasitic_coordinate_transform` command settings remain in effect for all the subsequent `spefIn` and `read_parasitics` until they are reset using the `-reset` option.
- Whenever the block's origin (in block SPEF/DEF) is non-zero, the x and y coordinates need to be specified for applying the transform. The last two parameters, `-x_block_origin` and `-y_block_origin` in the following command use-model are only required to be specified for the case of non-zero block origin.

  ```
  create_parasitic_coordinate_transform -inst inst_name
  -x_offset x-offset of block origin in top-level DEF/SPEF
  -y_offset y-offset of block origin in top-level DEF/SPEF
  -orient orientation
  -x_block_origin x-coord of non-zero block origin
  -y_block_origin y-coord of non-zero block origin
  ```

  For example:

  - **For Non-Zero Origin Block**: Specify the block origin as shown below:
    ```
    create_parasitic_coordinate_transform -inst eco_inst1
    -x_offset 421.6
    -y_offset 485
    -orient R180
    -x_block_origin 3.4
    ```

```
-y_block_origin 9.7
```

- **For zero origin block**: No need to specify the block origin as shown below:

  ```
  create_parasitic_coordinate_transform
  -inst controller
  -x_offset 16.8
  -y_offset 205
  -orient R0
  ```

  or

  ```
  create_parasitic_coordinate_transform
  -inst controller
  -x_offset 16.8
  -y_offset 205
  -orient R0
  -x_block_origin 0
  -y_block_origin 0
  ```

  Both the above commands are equivalent.

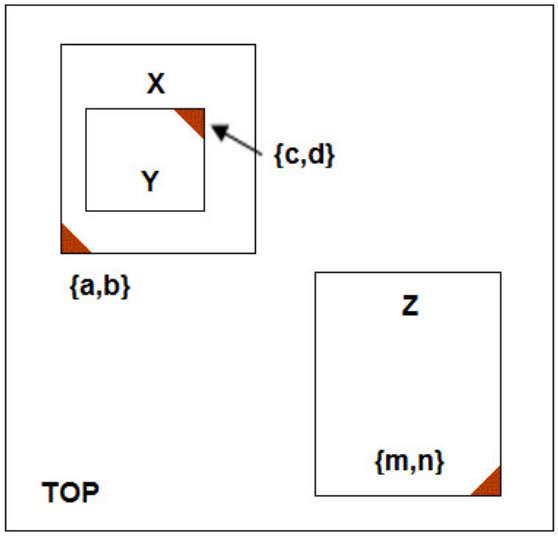
- The impact of this command is valid till first `spefIn` command is read (in case of MMMC, till all corners SPEFs have been read). If you want to read SPEF again, this command needs to be applied once again. These transformations will be applied during `spefIn` command.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command and all of its parameters, use the `man` command: `man create_parasitic_coordinate_transform`
`-inst` *`inst_name`*	Indicates the hierarchical name of the instance for which SPEF is being read.
`-orient {R0 R90 R180 R270 MY MX90 MX MY90}`	Indicates orientation change of instances. The valid values are `R0`, `R90`, `R180`, `R270`, `MX`,`MX90`, `MY`, and `MY90`.
`-relative_to_parent`	When this parameter is specified, the software considers block offsets for the specified instance relative to the immediate parent. By default, the software considers the offset value relative to the top hinst.
`-reset`	Resets SPEF transform values.
`-x_block_origin` *`value`*	Specifies that in case of non-origin block, offset x-coordinate in microns
`-x_offset` *`value`*	Indicates the actual x-coordinate as exists in the physical floor plan, corresponding to the specified instance.
`-y_block_origin` *`value`*	Specifies that in case of non-origin block, offset y-coordinate in microns.
`-y_offset` *`value`*	Indicates the actual y-coordinate as exists in the physical floor plan, corresponding to specified instance.

Examples

Consider the following hierarchical design where X and Z are two blocks inside TOP, and Y is another hierarchical block in X:



Here, X,Y, and Z have the following physical locations in the floor plan:

X - (a,b)
Y- (c,d)
Z- (m,n)

You can apply transformations to all blocks with respect to TOP, by using the following commands:

```
create_parasitic_coordinate_transform -inst X -x_offset a -y_offset b

create_parasitic_coordinate_transform -inst Z -x_offset m -y_offset n -orient MY

create_parasitic_coordinate_transform -inst X/Y -x_offset c -y_offset d -orient R180
```

These transformations will be applied when spefIn command is issued.

Related Commands

- set_analysis_view
- redirect
- read_parasitics
- spefIn

extractRC

```
extractRC
[-help]
```

Extracts resistance and capacitance for the interconnects and stores the results in an RC database. Optionally, an ASCII report file can be generated. Use this command after performing early global routing or final routing, and after using the `setExtractRCMode` command.

Parameter

`-help`	Outputs a brief description of the `extractRC` command. For a detailed description of the command, use the `man` command: `man extractRC`

Example
The following command extracts interconnect resistance and capacitance:

```
extractRC
```

RC database is created.

The `extractRC` command can be followed by the `rcOut` command to extract the parasitics from the RC database, and to generate an ASCII report containing the database information.

Related Information

- `setExtractRCMode`
- The following topic in the *Innovus User Guide*
  - Performing Extraction in Innovus

generateCapTbl

```
generateCapTbl
[-help]
[-cap totalCapFactor]
[-encrypt]
-output fileName
[-res resistanceFactor]
[-xcap crossCouplingFactor]
[-lef fileName [-shrinkFactor value]]
{-ict fileName | -incaptable fileName}
```

Reads in the interconnect technology (ICT) process file and generates a Capacitance table file. The Capacitance table file contains a header having a summary of process information and manufacturing effects followed by tables containing coefficients used during RC extraction.

The capacitance table is used both for preRoute extraction (above 32nm) and postRoute extraction `-effortLevel low`.

Note: For TQuantus, IQuantus, preRoute extraction (32nm and below), and Standalone Quantus, a Quantus Techfile is needed instead of a Capacitance table.

The `generateCapTbl` command automatically omits the creation of coefficients for the poly layers, local interconnect layers, and mimcap layers. The `generateCapTbl` command recognizes poly layers by the `gate_forming_layer true` statement.

Note: The command accepts ICT files that contain the `gate_forming_layer true` construct in multiple `conductor` sections to allow handling of diffusion models as a conductor. In this case, the Innovus captable generation process ignores all but the last `gate_forming_layer true` construct. The last one is considered as `poly` and is used as the ground plate level for the metal layers above.

The command does not support `grown_from_conformal` (non-planar stack) construct defined in the ICT file.

You can use the multi-CPU processing commands to generate a capacitance table in the parallel mode. However, this functionality is only available when invoking `generateCapTbl` from the software prompt, and not from the standalone utility `generateCapTbl`.

To generate a capacitance table file in the parallel mode, you must first set up the multiple-CPU processing mode using the multiple-CPU processing commands.

See the following multiple-CPU processing commands for more information:

- `setDistributeHost`
- `setMultiCpuUsage`

For more information, see Distributed Processing in Extraction in the *Innovus User Guide*.

Examples

- To run the `generateCapTbl` command in the parallel mode on different hosts, specify the following commands:

  ```
  setDistributeHost -rsh -add { host1 host2 host3 }
  setMultiCpuUsage -remoteHost 3
  generateCapTbl -ict sample.ict -output sample.capTbl
  ```

- To run the `generateCapTbl` command with three threads, specify the following commands:

  ```
  setDistributeHost -local
  setMultiCpuUsage -localCpu 3
  generateCapTbl -ict sample.ict -output sample.capTbl
  ```

Parameters

`-help`	Outputs a brief description that includes type and default information for each `generateCaptbl` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man generateCaptbl`
`-cap` *totalCapFactor*	Specifies the capacitance scale factor. Use this parameter with the -incaptable parameter to generate a capacitance table with capacitance values scaled by the specified scale factor.
`-encrypt`	Encrypts the generated capacitance table to protect the process information in the capacitance table header. When you use the `create_rc_corner -cap_table` command to read the encrypted capacitance table, the software writes the version number that was used to generate the file, and the name of the process to the log file.
`-ict` *fileName*	Specifies the name and path to the ICT (interconnect technology) process file. To support multiple process corners, a capacitance table should be created for each corner from the matching corner ICT file.
`-incaptable` *fileName*	Specifies the name of an existing capacitance table in ASCII format. You use this parameter along with the `-encrypt` parameter to create an encrypted version of an existing capacitance file. **Note**: The `-ict` and `-incaptable` parameters are mutualy exclusive. One of the two parameters must be specified.
`-lef` *fileName*	Specifies the name and the full path to the process LEF file for optimal accuracy. With the LEF file, this parameter generates a capacitance table by using the actual wire widths as specified in the non-default rules defined in the LEF. Without the LEF, the software uses generic widths and spacings, which might increase the need for interpolation during extraction.
`-output` *fileName*	Specifies the output filename that contains the capacitance table. This is a required parameter.

`-res` *resistanceFactor*	Specifies the resistance scale factor. Use this parameter with the -incaptable parameter to generate a capacitance table with resistance values scaled by the specified scale factor.
`-shrinkFactor` *value*	Specifies the value by which the measurements in the optional LEF file have to be shrunk for the technology used with this design. Use this parameter only with the `-lef` parameter and when the LEF technology does not match the target technology in the ICT file. Innovus scales the dimensions when reading the LEF file using the shrinkFactor specified. The ICT file should always refer to the target technology (min width and min spacing). **Note**: When working on a design that needs to be scaled down in Innovus, you must also set the shrinkfactor in *Design Import*. This tells the extractor to scale down all geometries before processing. When the ICT file contains a `layout_scale` in the process declaration part, you do not need to specify the shrink factor. In this case, the embedded shrink factor will also be written in the captable header. Also, there is then no need to set the shrink factor in the *Design Import* form. The embedded `layout_scale` is used to support half nodes such as 40nm technology. *Default:* 1.0
`-xcap` *crossCouplingFactor*	Specifies the cross-coupling capacitance scale factor. Use this parameter with the `-incaptable` parameter to generate a capacitance table with cross-coupling capacitance values scaled by the specified scale factor.

Example

The following command reads in the ICT file `test.ict`, and generates a `new.capTbl` output file:

```
generateCapTbl -ict test.ict -output new.CapTbl
```

Related Information

- `create_rc_corner`
- `extractRC`
- `setExtractRCMode`
- `setDistributeHost`
- `setMultiCpuUsage`
- The following topic in the *Innovus User Guide*
  - Generating a Capacitance Table

generateRCFactor

```
generateRCFactor
[-help]
[-outputFile file_name]
[-preroute {true | false}]
[-postroute {low | medium}]
[-reference {low | medium | high | signoff | externalSpef}]
[-spefMapFile file_name]
[-use_ostrich_filters file_name]
```

Generates the resistance and capacitance scale factors for optimal RC correlation by comparing the SPEF files generated by native extraction either with the user-specified reference RC data or with the automatic running of extraction in the higher accuracy mode. After generating the scale factors, the command prints a summary of the scale factors to a file. You can use this file to set the actual scale factors.

This command allows you to generate scale factors for one engine or multiple engines, depending on the parameters you specify. You can run preRoute (`-preroute`), or postRoute (`-postroute`) or both types of extraction to generate scale factors. For postRoute extraction, you can run extraction with effortLevel low or with effortLevel low and medium. The reference for comparison can be either an external SPEF or a SPEF generated from the postRoute extraction engine with the specified effortLevel.

When using an external SPEF as reference, use the `-spefMapFile` parameter to specify the name of a file containing multiple SPEF files for use in an MMMC setup.

Note: A Standalone Quantus extraction licence is required to run sign-off extraction in this command. Specify the Quantus path in the path setup. If you do not have a Standalone Quantus extraction license, you can use the `-spefMapFile` parameter to specify the signoff SPEF file(s). In addition, a proper setup of Quantus technology file(s) is required when specifying `-reference medium, high` or `signoff`.

Parameters

`-help`	Outputs a brief description that includes type and default information for the `generateRCFactor` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man generateRCFactor`
`-outputFile file_name`	
	Specifies the name of the file in which the command will save the new RC factor setting. You can use this file to set the actual scale factors.
`-preroute {true | false}`	
	Generates the scale factor for preroute extraction. When set to `true`, generates optimal scale factors for both signal nets and clock nets for preroute extraction. *Default*: `true`
`-postroute {low | medium}`	

	Specifies whether scale factors are to be generated for postRoute extraction. It also specifies whether scale factors are to be generated for engine with effortLevel low only, or for both engines with effortLevel low and medium. If the `-postroute` parameter is not specified, scale factors are not generated for postRoute extraction. When you use the `-postroute low` parameter, the software generates different scale factors for signal nets and clock nets.
`-reference {low | medium | high | signoff | externalSpef}`	
	Specifies the postRoute engine or external SPEF that you want to use as reference for comparison. *Default*: `signoff` If you specify `externalSpef`, you should use the `-spefMapFile` parameter to specify the third party SPEF file(s) to be used for comparison.
`-spefMapFile` *`file_name`*	

	Specifies the SPEF map file to be used for comparison when using MMMC designs. When you use this parameter, the design must be placed and fully routed, and the referenced SPEF file must be the one generated from the same routed design.
	The map file has a list of RC corners and the corresponding SPEF file for each RC corner, as shown in the following example:
	`rcmin design_rcmin.spef`
	`rcmax design_rcmax.spef`

`-use_ostrich_filters file_name`	

	Specifies the filename containing user-specified ostrich filter values for Tcap, res, and Xcap. These user-specified values are applied during the Ostrich run.
	In case this parameter is not specified by the user, default ostrich filter values will be used.
	The format of the file to be specified by the users for ostrich filters is:
	`-tcap {-min_data <value> -max_data <value> -min_fanout`
	`<value> -max_fanout <value> -max_perc_dev <value>}`
	`-res {-min_data <value> -max_data <value> -min_fanout <value>`
	`-max_fanout <value> -max_perc_dev <value>}`
	`-xcap {-min_data <value> -max_data <value> -min_fanout`
	`<value> -max_fanout <value> -max_perc_dev <value>}`

Examples

- The following command generates the resistance and capacitance scale factors for the postRoute engine with effortLevel low in an MMMC design by comparing the parasitics with the SPEF file from signoff extraction:

  ```
  generateRCFactor -preroute false -postroute low -reference signoff -outputFile low_signoff.tcl
  ```

  Following are some sample sections in the log file for the above command:

  ```
  ======================================================
  Summary of scale factor generation
  ======================================================
  RC-Corner Name: rc_typical
  Extraction Engine "Postroute", Effort Level "low"
  Cap scale factor : 0.973457
  Res scale factor : 1.015816
  XCap scale factor : 0.91458
  Clock Cap scale factor : 1.031517
  Clock Res scale factor : 1.000277
  RC-Corner Name: rc_best
  Extraction Engine "Postroute", Effort Level "low"
  Cap scale factor : 1.087774
  Res scale factor : 0.843309
  XCap scale factor : 1.137418
  Clock Cap scale factor : 1.11112
  Clock Res scale factor : 0.827263
  RC-Corner Name: rc_worst
  Extraction Engine "Postroute", Effort Level "low"
  Cap scale factor : 0.92273
  Res scale factor : 1.186163
  XCap scale factor : 0.809296
  Clock Cap scale factor : 0.994961
  Clock Res scale factor : 1.170787
  Finished correlation ......................
  ```

- The following command generates scale factors for preroute and postroute -effortLevel low engines in an MMMC design by comparing the parasitics with an external supplied list of SPEF files:

  ```
  generateRCFactor -preroute true -postroute low -reference externalSpef -spefMapFile
  preRoute_mapFile
  ```

  In this example, the SPEF files specified in the file referenced in the `-spefMapFile` parameter must be generated using a postroute extraction tool on the same routed design.

Related Information

- `extractRC`
- `setExtractRCMode`
- The following topic in the *Innovus User Guide*
  - Setting the Scale Factors

getExtractRCMode

```
getExtractRCMode
[-help]
[-assumeMetFill]
[-capFilterMode]
[-cerebrus_license_only]
[-compressOptMemRCDB]
[-coupled]
[-coupling_c_th]
[-defViaCap]
[-effortLevel]
[-engine]
[-extraCmdFile]
[-extractionFillStreamMapFile]
[-hardBlockObs]
[-incremental]
[-layerIndependent]
[-lefTechFileMap]
[-localCpu]
[-pvs_fill]
[-qrcCmdFile]
[-qrcCmdType]
[-qrcOutputMode]
[-qrcRunMode]
[-relative_c_th]
[-signoff_stream_layer_map]
[-total_c_th]
[-tQuantusModelFile]
[-turboReduce]
[-tsvSubcktFile]
[-useQrcOAInterface]
[-useShieldingInDetailMode]
[-viaCap]
[-writeDefOptionsForSignoffExtract]
[-quiet]
```

Returns information about the specified `setExtractRCMode` parameter in the Innovus log file and the console. The following information is returned:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the software displays information for all the `setExtractRCMode` parameters.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `getExtractRCMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man getExtractRCMode`
`parameter_names`	Displays information for the specified parameters. You can specify one or more parameters. See `setExtractRCMode` for descriptions of the extraction setup parameters you can specify.
`-quiet`	Displays the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software displays the current settings of all `setExtractRCMode` parameters in Tcl list format.

Examples

- The following command displays the current setting for the `-viaCap` parameter:
  ```
  getExtractRCMode -viaCap
  ```
  The software displays the following information:
  ```
  -viaCap true # bool, default=true
  true
  ```
- The following command displays the current settings for the `-viaCap` and `-engine` parameters:
  ```
  getExtractRCMode -viaCap -engine
  ```
  The software displays the following information:
  ```
  -viaCap true # bool, default=true
  -engine preRoute # enums={preRoute postRoute}, default=preRoute, user setting
  {viaCap true} {engine preRoute}
  ```
- The following command displays the current setting for the `-viaCap` parameter in Tcl list format only:
  ```
  getExtractRCMode -viaCap -quiet
  ```
  The software displays the following information:
  ```
  true
  ```
- The following command displays the current settings for the `-viaCap` and `-engine` parameters in Tcl list format only:
  ```
  getExtractRCMode -viaCap -engine -quiet
  ```
  The software displays the following information:
  ```
  {viaCap true} {engine preRoute}
  ```
- The following command displays the current settings for all `setExtractRCMode` parameters in Tcl list format only:
  ```
  getExtractRCMode -quiet
  ```

Related Commands

- extractRC
- setExtractRCMode

rcOut

```
rcOut
[-help]
[-excludeIlm]
[-noRes]
[-useEEQCellWithLibertyInfo]
{{{-spef fileName}[-excNetFile fileName | -net fileName | -netName list_of_nets]}}
[-view viewName | -rc_corner rcCornerName]
[-cUnit {pF fF}]
```

Reads the parasitic database and outputs the parasitics in the Standard Parasitic Exchange Format (SPEF).

Parameters

`-help`	Outputs a brief description that includes type and default information for each `rcOut` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man rcOut`
`-cUnit` *unit*	Specifies the unit of capacitance as `fF` or `pF` in the output SPEF file. *Default:* `pF`

`-excludeIlm`	Writes out only the top-level SPEF while excluding the ILM parasitics.
	Note: When this parameter is specified, the top cell-name is suffixed in the SPEF output filename.
	For example, if `test.spef` is the name of the SPEF file and `top_cell_name` is the cell-name, then when you run below command:
	`rcOut -spef test.spef -excludeIlm`
	The output SPEF filename will be:
	`test.spef.top_cell_name`

`-excNetFile` *fileName*	Specifies the name of the file that contains nets to be excluded from the output file.
`-net` *fileName*	Specifies the name of the input file that contains the net names to be extracted. *Default:* Extracts all net names
`-netName` *list_of_nets*	Specifies the names of the nets to be written in the SPEF file. *Default*: Writes all nets

`-noRes`	Generates a SPEF file without the resistance information. The SPEF file generated by this parameter can only be used to improve the performance of third-party simulator tools in early stages of the design. **Caution**: Most tools expect the resistance information to be present in the SPEF file for performing extraction. Therefore, SPEF files with no resistance information should not be used with Ostrich for RC correlation.
`-rc_corner` *rcCornerName*	Specifies the RC corner that is used to generate the parasitic output file in the multi-corner flow.
`-spef` *fileName*	Specifies the name of the SPEF output file. If you add `.gz` as a suffix, the `rcOut` command automatically compresses the generated file.
`-useEEQCellWithLibertyInfo`	Writes EEQ master cell in case the EEQ cells are present.
`-view` *viewName*	Specifies that the RC corner associated with the specified view is used to generate the output SPEF file in the multi-corner flow.

Examples

- Use any of the commands listed below to read the parasitic database created during RC extraction and write parasitic information for two nets for `corner1` in SPEF format to the specified file:

```
rcOut -spef spefFile -rc_corner corner1 -netName [ list net1 net2 ]
```
OR
```
rcOut -spef spefFile -rc_corner corner1 -netName {net1 net2}
```
OR
```
rcOut -spef spefFile -rc_corner corner1 -netName "net1 net2"
```

Related Information

- `spefIn`
- The following topics in the *Innovus User Guide*
  - Performing Extraction in Innovus
  - Correlating Native Extraction With Sign-Off Extraction

read_parasitics

```
read_parasitics
[-help]
[-force]
[-rc_corner_map rc_corner1:spef_field1 rc_corner2:spef_field2 ...]
[-starN]
{[{{[-rc_corner <string>]}}]}
[-via_variation_file viaVariationFile]
```

Reads the SPEF and RCDB-based RC parasitics information and information from the RCDBs already created in Innovus. For hierarchical designs, use this command to read top-level as well as block-level parasitic information.

Note: The following are important aspects to be considered for using this command:

- The RCDB parasitic file must have the `.rcdb.d` extension.
- The RCDB-based parasitic data that is provided as input to this command must be the one that was saved in the same major release of the software. The command will not read the RCDB data saved in a previous release. For example, if you provide RCDB saved in the software's 10.1.x release as input to the `read_parasitics` command in the software's 11 release, then it cannot be read.
- Only the RCDB parasitic data on systems that are same as those used for saving the data can be read. For example, if the data is saved on Linux in 64-bit mode, you can only restore it on Linux 64-bit mode.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `read_parasitics` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_parasitics`
`-force`	Allows reading of parasitic data even when block level RCDB/ SPEF is missing for a corner. **Note:** This parameter is only applicable for hierarchical designs. It is required that the top cell should have parasitic data specified for all the active RC corners.
`-rc_corner` *list of RCcorner specific RCDBs/SPEFs for hierarchical design | RCcorner specific RCDB/SPEF*	
	Specifies the RC corner name along with a corner-specific RCDB or SPEF for both top and block levels. For hierarchical designs, this parameter lists the corner-specific SPEFs/RCDBs for both top and block levels. **Note:** A corner-specific RCDB parasitic file can only be provided as input to the RC corner for which it was generated. It cannot be provided as input to other RC corners.
`-rc_corner_map` *rc_corner1:spef_field1 rc_corner2:spef_field2 ...*	
	Provides a mapping of RC corners to the SPEF fields of multi-value SPEF files.
`-starN`	Specifies whether or not to read the *N statements in the SPEF file. The *N statements define the location of the RC nodes. When this parameter is specified, the *N statements are read, otherwise the *N statements are ignored.
`-via_variation_file` *viaVariationFile*	

	Specifies the name of the file for modeling via variation. The contents of the file include all the via layer names with the corresponding MEAN_SHIFT and STDDEV so that all the VIA resistance in SPEF can be mapped to their own variation multiplier. This option is used to specify the via_variation file only when the design has a single corner.
	For example,
	`read_parasitics`
	`-rc_corner rcCornerName \`
	`-via_variation_file` *viaVariationFileName*

Examples

- The example below is for hierarchical design with Top (T), Block 1(B1) and Block 2(B2), and two active RC corners, C1 and C2.
  - The following command specifies the RC corner names along with corner-specific RCDB and SPEF for both top and block levels:

    ```
    read_parasitics \
    -rc_corner C1 Top_C1.spef Block1_C1.spef Block2_C1.spef.gz \
    ```

```
-rc_corner C2 Top_C2.rcdb.d Block1_C2.spef Block2_C2.rcdb.d
```

Related Commands

- `extractRC`
- `setExtractRCMode`
- RC Extraction chapter in the *Innovus User Guide*

report_net_parasitics

```
report_net_parasitics
net_name
[-help]
-rc_corner rc_corner_name
[-file_name output_file_name]
```

Reports the RC parasitics of the specified net.

The `report_net_parasitics` command reports a list of attackers in the order of worst coupling capacitance by default. The software determines and reports the probable top N worst attackers for a victim. The command output will show worst coupling capacitance, in addition to RC parasitics.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_net_parasitics` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_net_parasitics`
`-file_name` *output_file_name*	Specifies the name of the output file.
`-` `rc_corner` *rc_corner_name*	Specifies the RC corner name for which the RC parasitics are to be reported.
net_name	Specifies the name of the net for which the RC parasitics are to be reported.

Example

- The following command reports the RC parasitics for net, NET1 for RC corner, `rc_best`:

```
report_net_parasitics -rc_corner rc_best NET1

Net: NET1
RC Corner: rc_best
Total Cap = 0.473082 ff
Total Res = 2.104041 ohms
Total XCap = 0.334035 ff
Cap Distribution:

Total Ground Cap = 0.139046 ff
XCap With Nets:

NET3 = 0.059964 ff
NET4 = 0.040277 ff
NET5 = 0.233794 ff
```

Related Commands

- extractRC
- setExtractRCMode

report_rcdb

```
report_rcdb
[-help]
rcdb_dir_name
```

Displays the contents of the RCDB being read.

Note:

1. This command is supported only for reading flat single RCDB. Multiple RCDB's cannot be read simultaneously using this command.
2. The RCDB being read must be generated on the same platform and architecture on which it is being read. For example, if you have generated the RCDB on Linux platform, then the `report_rcdb` command cannot report its content on the IBM AIX platform. Similarly, an RCDB generated on a 64-bit system cannot be reported on a 32-bit system.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_rcdb` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_rcdb`
`rcdb_dir_name`	Specifies the location of the RCDB directory.

Example

- The following command reports the contents of the RCDB being read in the `test.rcdb.d` directory:

```
report_rcdb test.rcdb.d
Opening parasitic data file 'test.rcdb.d/header.da' for reading.
RCDB VERSION           : 11.1
OS-bit                 : 64
OS-endianness          : little-endian
RCDB Compression       : on
RCDB Source            : Innovus
DESIGN                 : test
RC-Corners:            : typical
Coupled                : yes
Node-loc               : no
Statistical            : no
```

Related Commands

- extractRC
- setExtractRCMode

report_unit_parasitics

```
report_unit_parasitics
[-help]
-layer metal_layer_name
-rc_corner rc_corner_name
[-space wire_space_in_um]
[-via_res via_resistance]
[-width wire_width_in_um]
```

Reports unit parasitics for individual metal layers and vias. The report prints the per unit resistance and capacitance for the specified layer. The resistance values are reported in ohms per micron for min width and min spacing of the layer. The capacitance values are reported in femtofarad (fF) per micron for min width and min spacing of the layer.

Via parasitics are printed considering the layer mentioned in the command as the lower layer of the via. The default via specified in the LEF file will be considered in this case.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_unit_parasitics` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_unit_parasitics`
`-layer` *`metal_layer_name`*	Specifies the name of the metal layer for which the unit parasitics are to be reported.
`-rc_corner` *`rc_corner_name`*	Specifies the name of the RC corner for which the unit parasitics are to be reported.
`-space` *`wire_space_in_um`*	Specifies the wiring space in microns. *Default*: minimum spacing
`-via_res`	When this parameter is specified, the resistances of all vias, whose upper layer is specified using the `-layer` parameter, are reported.
`-width` *`wire_width_in_um`*	Specifies the wiring width in microns. *Default*: minimum width

Examples

- The following command reports the unit parasitics for metal layer, `Metal5`, for the specified RC corner, `rc_worst`:

  ```
  report_unit_parasitics -layer Metal5 -rc_corner rc_worst
  ```

  The software gives the following information:

  ```
  Unit parasitics for layer Metal5 (in fF,ohm)
      RC-Corner = rc_worst
        Cap = 0.213204
        Res = 0.900873
  ```

```
Parasitics for default Via "VIA5_0_VH"
      Via Cap = 0.017901
      Via Res = 8.859576
```

- The following commands reports the unit parasitics for `Metal5` with spacing two times the minimum spacing:

  ```
  report_unit_parasitics -layer Metal5 -rc_corner rc_worst -space [expr 2 * [dbGet [dbGet
  head.layers.extName Metal5 -p].minSpacing]]
  ```

  The software gives the following information:

  ```
  Unit parasitics for layer Metal5 (in fF,ohm)

      RC-Corner = rc_worst

          Cap = 0.188657
          Res = 0.900873

  Parasitics for default Via "VIA5_0_VH"

          Via Cap = 0.017901
          Via Res = 8.859576
  ```

- The following commands reports the unit parasitics for Metal5 with width two times the minimum width:

  ```
  report_unit_parasitics -layer Metal5 -rc_corner rc_worst -width [expr 2 * [dbGet [dbGet
  head.layers.extName Metal5 -p].minWidth]]
  ```

  The software gives the following information:

  ```
  Unit parasitics for layer Metal5 (in fF,ohm)

      RC-Corner = rc_worst
      Cap = 0.198657
      Res = 0.555333

  Parasitics for default Via "VIA5_0_VH"

      Via Cap = 0.017901
      Via Res = 8.859576
  ```

- The following commands reports the unit parasitics for vias with top layer of Metal5:

  ```
  report_unit_parasitics -layer Metal5 -rc_corner rc_worst -via_res
  ```

  The software gives the following information:

  ```
  Unit parasitics for layer Metal5 (in fF,ohm)

      RC-Corner = rc_worst
      Cap = 0.213204
      Res = 0.900873

  Parasitics for default Via "VIA5_0_VH"

      Via Cap = 0.017901
      Via Res = 8.859576
      Other Via res
      VIA45_1C 8.859576
      VIA45_1C_H 8.859576
      VIA45_1C_V 8.859576
      VIA45_1ST_N 8.859576
  ```

```
VIA45_1ST_S 8.859576
VIA45_2C_E 4.429788
VIA45_2C_W 4.429788
VIA45_2C_N 4.429788
VIA45_2C_S 4.429788
VIA45_2ST_N 4.429788
VIA45_2ST_S 4.429788
VIA45_4C 2.214894
```

Related Commands

- extractRC
- setExtractRCMode

reset_parasitics

```
reset_parasitics
[-help]
```

Deletes the parasitic database without resetting the extraction modes specified using the `setExtractRCMode` command.

Parameter

`-help`	Outputs a brief description of the `reset_parasitics` parameter.

Related Commands

- `extractRC`
- `setExtractRCMode`

restoreRC

```
restoreRC
[-help]
dirName
```

Restores the RC extraction data that was previously saved either with the `saveDesign name -rc` or the `saveRC` command.

Note: The systems you use to save and restore RC extraction data must match (32-bit or 64-bit data models).

Parameters

`-help`	Outputs a brief description that includes type and default information for the `restoreRC` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man restoreRC`
dirName	Specifies the location of the RCDB directory. The syntax of the directory name is `*.rcdb.d`. For example, you can restore the parasitic data in `test.rcdb.d`.

Related Commands

- `saveRC`
- `saveDesign`

saveRC

```
saveRC
[-help]
dirName
```

Saves the RC extraction data in the specified directory. Use the `restoreRC` command to restore the parasitic data. Use this command after running either `extractRC` or `spefIn` to ensure that the parasitic data is available in Innovus database.

Note: You can restore the parasitic data on systems that are same as those used for saving the data. For example, if the data was saved on Linux in 32-bit mode, you can restore it on Linux 32-bit mode only.

Parameters

`-help`	Outputs a brief description that includes type and default information for the `saveRC` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man saveRC`
`dirName`	Specifies the location of the RCDB directory. The directory name should follow the `*.rcdb.d` format.

Related Commands

- `restoreRC`
- `extractRC`
- `spefIn`

setExtractRCMode

```
setExtractRCMode
[-help]
[-reset]
[-assumeMetFill scalevalue]
[-capFilterMode {relOnly | relAndCoup | relOrCoup}]
[-cerebrus_license_only {true | false}]
[-compressOptMemRCDB {true | false}]
[-coupled {true | false}]
[-coupling_c_th value_in_fF]
[-defViaCap {true | false}]
[-effortLevel {low | medium | high | signoff}]
[-engine {preRoute | postRoute}]
[-extraCmdFile fileName]
[-extractionFillStreamMapFile fileName]
[-hardBlockObs {true | false}]
[-incremental {true | false}]
[-layerIndependent {0 | 1}]
[-lefTechFileMap fileName]
[-localCpu number_of_cpu]
[-pvs_fill {true | false}]
[-qrcCmdFile fileName]
[-qrcCmdType {auto | partial | custom}]
[-qrcOutputMode {rcdb | spef}]
[-qrcRunMode {concurrent | sequential}]
[-relative_c_th value]
[-signoff_stream_layer_map fileName]
[-total_c_th value_in_fF]
[-tQuantusModelFile fileName]
[-tsvSubcktFile file_name]
[-turboReduce {true | false | auto}]
[-useQrcOAInterface {true | false}]
[-useShieldingInDetailMode {true | false}]
[-viaCap {true | false}]
[-writeDefOptionsForSignoffExtract list_of_def_options]
```

Sets the native RC extraction mode. Use this command before using the `extractRC` command. You can use the `setExtractRCMode` command multiple times during an Innovus session to specify additional parameters or change the previously defined values for the extraction mode. You can perform RC extraction during preRoute and postRoute stages, and select an extraction engine based on the expected run time and accuracy level. Each extraction engine supports a specific set of `setExtractRCMode` command parameters. The following table lists the engine support for each parameter.

Note: In previous releases, the `setExtractRCMode` command accepted incomplete parameter names. Now, only complete parameter names are recognized by the software. If you enter an incomplete parameter name, the software will return an error that the given parameter is not a legal argument. To avoid this error, update your scripts to use the legal (documented) parameter syntax.

setExtractRCMode Parameters	PreRoute Extraction Engine	PostRoute Extraction Engine low	PostRoute Extraction Engine Medium & high	PostRoute Extraction Engine signoff
-assumeMetalFill	X	X		
-capFilterMode		X	X	X
-compressOptMemRCDB		X	X	
-coupling_c_th (-coupled true only)		X	X	X
-coupled		X	X	X
-defViaCap	X			
-effortLevel		X	X	X
-extraCmdFile			X	
-extractionFillStreamMapFile				X
-hardBlockObs		X		
-pvs_fill				X
-qrcCmdFile				X
-qrcCmdType				X
-relative_c_th		X	X	X
-reset	X	X	X	X
-signoff_stream_layer_map				X
-specialNet	X	X		
-total_c_th		X	X	X
-turboReduce {true | false}			X	X
-useShieldingInDetailMode		X		
-viaCap		X		

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setExtractRCMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setExtractRCMode`

`-assumeMetFill` *scalevalue*	Takes into account the metal fill in the design. For postRoute extraction, only extraction engine with `-effortLevel low` supports this parameter. In postRoute extraction with `-effortLevel low`, *scalevalue* is not required and will be ignored. Detail extraction uses the `FILLACTIVESPACING` values from LEF to account for metal fill in the design. If the `FILLACTIVESPACING` values are not specified in the LEF file, the software uses the default value, which is `0.6um`. In preRoute extraction, when calculating the capacitance of a wire segment in layer *N*, this parameter always assumes layer *N-1* and layer *N+1* to be filled. Specify one of the following values: • `0`: Assume no metal fill in layer *N*. ▪ When you do not specify this parameter, the software assumes no metal fill in layer *N*. This is equivalent to setting the *scalevalue* parameter to `0`. • `1`: Assume full metal fill at minimum spacing distance. ▪ Where there is a minimum of signal-to-signal spacing, the cross-coupling capacitance will be extracted and reported. ▪ Where there is no wire within the minimum spacing, a capacitance to ground will mimic minimum-spaced metal fill in that location. • x: Capacitance value is calculated once with full metal fill and once without metal fill. The user-specified scale value (decimal value between 0 and 1) determines the interpolation point between the two calculated values, for example, `0.5` *Default:* 0
`-capFilterMode { relOnly | relAndCoup | relOrCoup }`	

	Specifies the capacitance filtering mode. The filtering modes are conditions which define how to use the `-coupling_c_th` and `-relative_c_th` values, or their combination. Based on the filtering mode specified, the software determines the nets in the design, which will have their coupling capacitance lumped to the ground. **Note**: TQuantus only supports the `relandCoup` mode of this parameter. It does not support the `relOnly` and `relOrCoup` modes as they are for old nodes only. **Note:** The behavior of the `-total_c_th` filter is independent of the `-capFilterMode` parameter setting. Nets that have a total capacitance value lower than the value specified with the `-total_c_th` parameter will have their coupling capacitances grounded regardless of the `-capFilterMode` setting. You can use one of the following modes: • `relOnly`: Grounds the coupling capacitance of nets based on the threshold value set by the `-relative_c_th` parameter. • `relAndCoup`: Grounds the coupling capacitance of nets only when it is lower than the threshold value specified with the `-relative_c_th` parameter as well as the `-coupling_c_th` parameter. In this mode, you enforce maximum restriction on grounding the coupling capacitances. • `relOrCoup`: Grounds the coupling capacitance of nets if it is lower than the threshold value specified with one of the `-relative_c_th` or `-coupling_c_th` parameters. *Default*: For TQuantus, IQuantus, and Standalone Quantus extraction, the default value is `relAndCoup`. For postRoute extraction with `-effortLevel low`, the default value is dependent on the process node specified with the `setDesignMode -process` command. If the process node is 130nm or below, the default value is `relAndCoup` and if the process node is above 130nm, the default value is `relOnly`.
`-cerebrus_license_only {true | false}`	This parameter is applicable to the Cerebrus-Quantus flow. When set to `true`, only the Cerebrus license will be considered and Innovus will not fall back to use Quantus license when Cerebrus license(s) is/are not available. By default, the option is set to `false`, which means the regular Quantus license will be considered when Cerebrus license(s) is/are not available. *Default*: `false`
`-compressOptMemRCDB {true | false }`	

	Generates compressed RCDB data. *Default*: `false` **Note:** This functionality reduces the disk usage but leads to marginal increase in run time.
`-coupled {true | false}`	Separately outputs grounded and coupling capacitance. The `-coupled` false value is typically used for static timing analysis. Signal Integrity analysis requires the coupled mode. *Default:* `true` (for postRoute extraction) **Note**: The TQuantus extraction engine supports this parameter.
`-coupling_c_th` *value_in_fF*	Specifies the threshold value that determines when the extractor lumps a net's coupling capacitance to ground. This parameter is applicable only when the `-coupled` parameter is set to `true`. The software decouples the coupling capacitance of nets when the total coupling capacitance between the pair of nets is lower than the threshold specified with this parameter. The actual decoupling also depends on the setting of the `-capFilterMode` parameter. *Default:* `0.2fF`. If the `setDesignMode -process` *process_node* command is used, the software assigns the recommended default value for this parameter based on the process node specified. **Note**: See the `setDesignMode` command for the list of default values that are applied based on the process node setting. **Note**: The TQuantus extraction engine supports this parameter.
`-defViaCap {true | false}`	Takes into account the via capacitance in the preRoute RC extraction results. Specify a value of `true`, if you are experiencing correlation issues caused by deviations in clock latency when running preRoute RC extraction. *Default*: `false` **Note**: For 20nm and below nodes, the default value of this parameter is set to `true`.
`-effortLevel { low | medium | high | signoff }`	

	Specifies the postRoute engine variant that you want to use for performing RC extraction. *Default*: `low` **Note**: TQuantus (`-effortLevel medium`) is the default extraction engine in the postRoute flow for 65 nm and below designs. However, TQuantus and IQuantus require a Quantus Techfile. Therefore, Native Detailed (`-effortLevel low`) engine remains the default engine if no Quantus Techfile has been defined. This parameter supports the following values: • `low`: Invokes the native detailed extraction engine. This is the same as specifying the -engine postRoute setting. • `medium`: Invokes the TQuantus extraction engine. This engine supports distributed processing. TQuantus engine is recommended for process nodes < 65nm. For this effort level, specify the extra command file using the `-extraCmdFile` parameter. • `high`: Invokes the Integrated-Quantus (IQuantus) extraction engine. IQuantus provides superior accuracy compared to TQuantus. IQuantus is recommended for extraction after ECO. In addition, IQuantus supports distributed processing. For this effort level, specify the extra command file using the `-extraCmdFile` parameter. **Note**: IQuantus requires a Quantus-XL license. • `signoff`: Invokes the Standalone Quantus extraction engine. This engine choice provides the highest accuracy. The engine has several runModes, thereby, providing maximum flexibility. For this effort level, specify the extra command file using the `-qrcCmdFile` parameter.
`-engine { preRoute | postRoute }`	Specifies the extraction engine to use. • `preRoute`: Uses preRoute engine. RC extraction is done by the fast density measurements of the surrounding wires; coupling is not reported. Use this mode to perform extraction on pre-routed designs after clock tree synthesis. • `postRoute`: Uses postRoute engine. RC extraction is done by the detailed measurement of the distance to the surrounding wires; coupling is reported. The `-effortLevel` parameter further specifies which postRoute engine is used for balancing performance versus accuracy needs. *Default:* `preRoute`
`-extraCmdFile` *fileName*	

Specifies the Quantus extra command file for TQuantus/IQuantus extraction. The extra command file supports the following variables:

- `log_file -dump_options {`**`true`** `| false}`
- `extraction_setup -copy_port_to_obs {`**`true`**`|false}`
- `extraction_setup -remove_net_pin_overlap {true |`**`false`**`}`
- `metal_fill -type {grounded |` **`floating`** `| virtual}`
- `output_setup -compress_temporary_files {true |` **`false`**`}`

Notes:

- TQuantus only supports advanced virtual metal fill in `metal_fill` in the extra command file. It does not support the grounded and floating options.
- When the `metal_fill` type is virtual, provide the absolute paths to the vmf rule file and the metal schema file. For example,

```
metal_fill \
     -type virtual \
     -enable_advanced_virtual_fill true \
     -vmf_metal_scheme_file
     .../saveTestcase_vmf/missing_vmf_files/abc.vmf_metal_scheme
     \
     -vmf_rule_file
     .../saveTestcase_vmf/missing_vmf_files/abc.vmf_rule
```

- The `-extraCmdFile` parameter becomes “non-portable” if the vmf rule file and/or schema file have been specified. This means that the two file paths will not be automatically modified or ported. You can modify the file paths, if required. To make the extra command file portable, ensure that the vmf rule file and schema files are available at the locations specified in the extra command file.
- The **bold** text in the above example indicates that these values are used by default when the commands are not explicitly specified in the extra command file. For example, IQuantus extracts the “**`floating`**” metal fills by default when either the `-extraCmdFile` option is not specified or the `metal_fill` variable is not specified in the extra command file.
- This parameter is only valid if `-effortLevel medium` or `high` is specified.
- This parameter is not available on the IBM platform.

Default: `" "`

`-extractionFillStreamMapFile` *`fileName`*	This parameter is specified only when `-pvs_fill` parameter is set to `true`. When the `-pvs_fill` parameter is set to `true`, a Quantus stream/oasis map Tcl command is required to map the PVS layer/dtype data to the corresponding Quantus layers.
	If the `pvs_fill.data` is generated based on stream (GDS), the map file should be in the following format:
	`#layer gds_layer gds_dtype`
	`<layer_name> <layer_number> <data_type>`
	…
	If the `pvs_fill.data` is generated based on OASIS, the map file should be in the following format:
	`#layer oasis_layer oasis_dtype`
	`<layer_name> <layer_number> <data_type>`
	…
	Where,
	• `layer_name` is the layer name in the design
	• `layer_number` is the layer ID in the GDS/OASIS file
	• `data_type` is the datatype specified in the GDS/OASIS file
	For example,
	`M1 31 0`
	`M1 31 1`
	`M2 32 0`
	`M2 32 1`
	Note: This parameter only affects signoff Quantus extraction.
	Default: `" "`

`-hardBlockObs {true | false}`	Makes cell geometries on metal layers from obstruction, and power and ground pin shapes visible to the extractor for hard blocks only. The software recognizes the following three types of hard blocks: • Hard macro (`CLASS BLOCK` in LEF file) • Blackbox (`CLASS BLACKBOX` in LEF file) • Partition (Committed during the Innovus session) Specify a value of `true` when routing goes over the listed hard blocks. A value of `true` removes the optimism for such routes, and provides better correlation with sign-off extraction without significantly impacting runtime. **Note**: This parameter is not supported when the extraction engine is TQuantus. *Default:* `false`
`-incremental {true | false}`	Uses incremental IQuantus extraction mode. If the incremental mode is set after performing initial extraction on full design, extraction checks for design changes, and if the amount of changes are below threshold, extraction is rerun in incremental mode on the modified areas only. Incremental mode can provide significant performance improvement during signal integrity optimization phase. *Default:* `true`
`-layerIndependent {0 | 1}`	
	By default, this parameter is set to `1`. In this mode, layer-independent extraction is enabled. This mode of extraction is recommended for designs with layer stacks that have significant variations in RC characteristics. This mode uses advanced techniques to avoid timing jumps due to differences in routing-layer assignments between preRoute and postRoute stages. When set to `0`, layer-independent extraction is disabled. **Note**: PreRoute extraction scale-factors should be computed without setting this mode. *Default*: `1`

`-lefTechFileMap` *`fileName`*	Specifies the location of the optional layer map file for the IQuantus and Standalone Quantus engines.
	
	Note: For IQuantus, layer mapping between the LEF or DEF file and the Quantus technology file is done automatically. This parameter is only necessary if this mechanism fails. If necessary, you can use a copy of the automatically generated layer map file `extLogDir/IQRC_*/<design>.layermap.log` and modify it for your own use.
	
	Layer map file is required for use with Standalone Quantus.
	
	For TSV designs, you are required to provide the name of the layer map file for IQuantus and Standalone Quantus.
	
	Note: If used, the layer map file must be in the Common Command Language (CCL) format.
	
	The following is an example of the layer map file format:
	
	`extraction_setup \`
	`-technology_layer_map \`
	`# <LEF/DEF Name> #<Tech File Name>`
	`M1 METAL_1 \`
	`VIA1 VIA_1 \`
	`M2 METAL_2 \`
	
	Note: For cell-level IQuantus layer map, the DEF/LEF layer names are case sensitive, but ICT layer names are case-insensitive.
	
	Note: TQuantus is not impacted by this parameter because it always automatically generates the layer map.
	
	Default: `" "`

`-localCpu` *`number_of_cpus`*	Specifies the number of CPUs to be used for IQuantus extraction. **Note**: • This option is only applicable for IQuantus extraction engine. • When specified, it overrides the `setMultiCpuUsage -localCpu` option for IQuantus. *Default*: As specified in the `setMultiCpuUsage -localCpu` setting.
`-pvs_fill`	When set to `true`, the PVS fill data attached to the Innovus database is sent to Quantus so that the effects of the fill data are extracted. This parameter only affects signoff Quantus extraction. It has no effect on tQuantus or iQuantus extraction done inside Innovus. This parameter requires that either the `-extractionFillStreamMapFile` parameter or the `-signoff_stream_layer_map` parameter is specified. *Default*: `false`

`-qrcCmdFile` *fileName*	Specifies the user created Quantus command file to be used for running Standalone Quantus. This command file should be in the new CCL syntax. **Note:** This parameter is only valid if `-effortLevel signoff` and `-qrcCmdType` partial or custom is selected. *Default*: `" "`
`-qrcCmdType {auto | partial | custom}`	
	Specifies whether to derive the settings for running Quantus from the Innovus settings, or from user-created command file using the `-qrcCmdFile` parameter. The `-qrcCmdType` parameter is only valid if `-effortLevel signoff` is selected. This parameter supports the following values: • `auto`: Derives the Quantus settings automatically from the Innovus parameter settings. **Note**: In this mode, the `-lefTechFileMap` parameter is required to be specified. • `partial`: Appends user commands from the qrcCmdFile to the settings derived from the Innovus parameter settings. You must provide the partial mode command file in CCL syntax. Use the `partial` command file only when you add extraction directives for those options that cannot be set in Innovus. Following are some important restrictions: ◦ Do not specify the following options in the partial mode command file: ▪ Threshold setting ▪ Coupled/decoupled mode ▪ Quantus Techfile (s) ▪ Operating Temperature ◦ Do not specify the "`-directory_name`" and "`-temporary_directory_name`" in the `output_setup` section of the command file ◦ Do not specify output file using `output_setup -file_name` command in the partial command file **Note**: The multi-temperature multi-corner extraction (MTMC) feature is enabled in the partial mode only if process technology is not specified in the user command file. In MTMC extraction, for RC corners with same Quantus technology file but different temperatures, resistance is extracted for multiple corners while capacitance is extracted only for a single corner.

	Note: There is no checking for syntax or content of this file. This file is appended as is to the automatically created Quantus command file. • `custom`: Gets the complete Quantus settings from the user-created command file. Except for input and output, no settings from Innovus parameters are used. You must provide the complete Quantus command file in CCL syntax. Following are some important restrictions for using custom command file: ◦ Do not specify an input DEF/OA DB in the custom mode command file ◦ Do not specify the "`-directory_name`" and "`-temporary_directory_name`" and "`-file_name`" in the `output_setup` section of the command file. ◦ Specify the output SPEF file with the following convention for the name:`<design.spef>`. In the MMMC mode, the output SPEF files will be named: `<design>_<corner>_<Temperature>.spef`. Here "design" refers to the top cell name. **Note:** Do not prefix or postfix the output file name. ◦ Specify the Quantus Techfile(s) to be used. In the MMMC mode, the corner names should match the names used in the `create_rc_corner` command ◦ Provide Quantus Techfile layermapping information in the custom command file ◦ If GDS is provided for cell contents, provide a Quantus Techfile to GDS layer number matching **Note**: The MTMC extraction feature is not enabled in the custom mode of Quantus extraction. **Note:** There is no checking for syntax or content of this command file prior to running the extraction. The command file will be used as is. If the output file is not specified correctly, the resulting parasitic file will not be read into the Innovus RC database. *Default*: `auto`
`-qrcOutputMode {rcdb | spef}`	
	Sets the Quantus output write mode to RCDB or SPEF. **Note:** This parameter is only valid if `-effortLevel` is `signoff`. *Default:* `spef`
`-qrcRunMode {concurrent | sequential}`	

	Specifies the Quantus run mode in MMMC designs. Note*:* For MMMC designs, in Quantus partial command type, do not specify the output SPEF file name and RC corner name in the command file to run Quantus in concurrent or sequential mode. For MMMC designs, in Quantus custom command type, do not specify the output SPEF file name and RC corner name together in the command file. Quantus sequential mode is not supported in custom command type. This parameter supports the following values: • `Concurrent`: Runs Quantus for all active corners concurrently. • `Sequential`: Runs Quantus for all active corners sequentially. *Default*: `concurrent`
`-relative_c_th` *value*	Sets a ratio threshold value that determines when the extractor lumps a net's coupling capacitance to ground. This parameter is applicable only when the `-coupled` parameter is set to `true`. If the total coupling capacitance between a pair of nets is less than the percentage (specified with this parameter) of the total capacitance of the net with the smaller total capacitance in the pair, the coupling capacitance between these two nets will be considered for grounding. The actual decoupling also depends on the setting of the `-capFilterMode` parameter. The range of this value is 0 to 1. If the `setDesignMode -process` *process_node* command is used, the software assigns the recommended default value for this parameter based on the process node specified. **Note**: See the `setDesignMode` command for the list of default values that are applied based on the process node settings. **Note**: The tQuantus extraction engine supports this parameter. *Default:* 0.03
`-reset`	Resets the extraction status of the loaded design and the `setExtractRCMode` parameters to their default values. The `-reset` parameter *must* be the first parameter specified. If you specify `-reset` by itself, the software resets all `setExtractRCMode` parameters to their default values. If you specify parameters after `-reset` , the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported.

`-signoff_stream_layer_map` *fileName*	This parameter is specified only when the `-pvs_fill` parameter is set to `true`. When the `-pvs_fill` parameter is set to `true`, a Quantus stream/oasis map Tcl command is required to map the PVS layer/dtype data to the corresponding Quantus layers.
	If the `pvs_fill.data` is generated based on stream (GDS/OASIS), the map file should be in the following format:
	`#layer gds_layer gds_dtype use-type color-ID`
	`<layer_name> <layer_number> <data_type> <use-type> <color-ID>`
	…
	Where,
	• `layer_name` is the LEF layer name in the design
	• `layer_number` is the layer ID in the GDS/OASIS file
	• `data_type` is the datatype of the layer in the GDS/OASIS file
	• `use_type` is the keyword for identifying the use of the data. The valid values are:
	  ◦ `gray`: library data in the GDS/OASIS file
	  ◦ `fill`: regular metal fill data in the GDS/OASIS file
	  ◦ `active`: active metal fill data in the GDS/OASIS file
	• `color-ID` is the color ID. Use “`1`” or “`2`” if the layer is colored, or use hyphen “`-`” if the layer is not colored
	For example,
	`M1  31  0 gray -`
	`M1  31  1 fill -`
	`M2  32  0 gray -`
	`M2  32  1 fill -`
	Note: This parameter only affects signoff Quantus extraction.
	Default: `""`

`-total_c_th` *value_in_fF*	Specifies the threshold value (femtoFarads) that determines when the extractor lumps a net's coupling capacitance to ground. This parameter is applicable only when the `-coupled` parameter is set to `true`. The software grounds the coupling capacitances for the nets which have a total capacitance value less than the value specified with this parameter. If the `setDesignMode -process` *process_node* command is used, the software assigns the recommended default value for this parameter based on the process node specified. **Note**: See the `setDesignMode` command for the list of default values that are applied based on the process node settings. **Note**: The TQuantus extraction engine supports this parameter. *Default:* `5 fF`
`-tQuantusModelFile` *fileName*	
	Specifies the name of the tQuantus model file to be used for postRoute effort-level medium extraction. You can specify the model file as follows: `setExtractRCMode -engine postRoute -effortLevel medium -tQuantusModelFile tQuantus_model.bin` For details of the use model, see Creating RC Model Data in TQuantus Model File in the *Innovus User Guide*. *Default*: `""` **Note**: If this file is not specified, it will be generated automatically by the software.

`-tsvSubcktFile` *file_name*	Specifies the name of the Through Silicon Via (TSV) subckt file to be used. The file contains the .subckt model information for TSV. The file has a Spice format. A sample file is shown below.
	`.subckt TSV top bottom`
	`R1 top n2 0.03075`
	`R2 n2 bottom 0.03075`
	`C1 n2 0 28f`
	`.ends`
	Where,
	`top` and `bottom` refer to the terminal nodes on the top and bottom layers of the TSV
	`n2` is the internal node between the top and bottom layers
	`R1 top n2 0.03075` means that a resistor with a resistance of `0.03075 Ohms` is added between the `top` and `n2` nodes
	`R2 n2 bottom 0.03075` means that a resistor with a resistance of `0.03075 Ohms` is added between the `bottom` and `n2` nodes
	`C1 n2 0 28f` means that a ground capacitance of `28 fF` is added on the `n2` node
	Note: The IQuantus engine might not keep the subckt connectivity as it is. The optimization of RES network will remove node n2 (if there is no other connections) and keep a single resistor between the top and bottom nodes with an accumulated resistance. For example, in the above example, the accumulated resistance will be `0.0615 Ohms`. Capacitance will also be moved to a “higher layered node”. In the above example, this layer is `top`.
	Default: `""` (empty string)

`-turboReduce {true|false|auto}`	

	Controls the turbo reduction feature for tQuantus/IQuantus, and Standalone Quantus extraction. It has no effect on tQuantus. This is a cell-level reduction that provides a complimentary reduction capability and only affects the coupling capacitors. It has no effect on lumped capacitor extraction. When set to `auto`, the software automatically turns `on` turbo reduction and proceeds when the required Quantus license is available. However, if the required Quantus license is not installed in the license server, the software automatically turns off Turbo reduction and proceeds with a warning message. If the license is installed in the license server but temporarily not available, then two scenarios are possible. First, if the license wait mode is not turned on, then Quantus will look for the license right away and error out because the license is not available. Second, if the license wait mode is turned on, then IQuantus will wait and check for the license periodically until timeout. If the license is available before the timeout, turbo reduction will be performed but if the license is not available before timeout, IQuantus will error out. When set to `true`, the turbo reduction engine is `on`. However, if the required Quantus license is not installed in the license server, then Standalone Quantus and IQuantus will error out with a message and the run will stop. It is because setting the option to true means the user really wants to perform turbo reduction. When set to `false`, the turbo reduction engine is switched `off`. For details, see the table titled, Behavior of Turbo Reduction, below. However, note that the licensing rules given in this table are not applicable to tQuantus because tQuantus does not require any Quantus license. *Default:* `auto`
`-useQrcOAInterface {true | false}`	
	When set to `true`, specifies that OpenAccess (OA) interface be used for invoking Quantus extraction instead of using the DEF-based flow. *Default*: `false`
`-useShieldingInDetailMode {true | false}`	

	Assumes shielding for any wire segment of a clock net that does not have any shielding yet, and the space between the wire segment and its neighbor is greater than or equal to the following: `2s+w` where, *s* = minimum spacing or non-default spacing *w* = minimum width or non-default width This parameter is available in `postRoute -effortLevel low` mode only. *Default:* `false`
`-viaCap {true | false}`	Extracts capacitance values considering part of the via geometries that do not overlap with the wire geometries. Via geometries that do not overlap the wire geometries are often found if you are designing using newer design technologies such as 90nm or when using multiple cut via. This parameter is only available in `postRoute -effortLevel low` mode. *Default:* `true`
`-writeDefOptionsForSignoffExtract` *list_of_def_options*	Specifies the list of `defOut` options for signoff extraction. For signoff extraction, `setExtractRCMode -engine postRoute -effortLevel signoff`, the software internally calls the `defOut` command. However, in some specific scenarios, you need to provide additional options for the `defOut` command. Use this parameter to provide the list of `defOut` options to be included. When this parameter is specified, `extractRC` will include these options while calling the `defOut` command. *Default:* `""`

Behavior of Turbo Reduction

	Availability of Quantus QRC AA License		
Option Selected (setExtractRCMode –turboReduce true|false|auto)	Quantus QRC AA Installed and Available (IQRC gets the license right away even in the wait mode because sufficient licenses are available in the license server)	Quantus QRC AA Installed but not Available (If license wait mode is not turned on, then Quantus looks for the license . If the wait mode is turned on and if the license is temporarily not available, then IQRC waits and checks for the license periodically until timeout.)	Quantus QRC AA Not Installed on License Server
auto	perform turbo reduction	1. error out if wait mode is not on or if the specified wait time is exceeded 2. perform turbo reduction if wait mode is on and the license is available within the specified time.	print warning and do not perform turbo reduction
true	perform turbo reduction	error out	error out because the user explicitly wants to use turbo reduction
false	do not perform turbo reduction	do not perform turbo reduction	do not perform turbo reduction

Examples

- The following command selects the fastest postRoute extraction engine:

```
setExtractRCMode -engine postRoute -effortLevel low
extractRC
rcOut -rc_corner corner1 -spef fe_extended.spef
```

- The following is a typical command sequence for the metal fill parameter:

```
setExtractRCMode -engine postRoute -effortLevel low
-assumeMetFill 1
extractRC
```

- The following command resets the `-relative_c_th` parameter to its default value:

```
setExtractRCMode -reset -relative_c_th
```

- The following command selects the Standalone Quantus for signoff grade extraction and uses a user supplied command file:

```
setExtractRCMode -engine postRoute -effortLevel signoff -qrcCmdFile
custom_qrc.cmd -qrcCmdType custom
extractRC
rcout -rc_corner corner1 -spef signoff.spef
```

Related Information

- `extractRC`
- `setDesignMode`
- `getExtractRCMode`
- The following topic in the *Innovus User Guide*
  - Performing Extraction in Innovus

setShrinkFactor

```
setShrinkFactor
[-help]
shrinkFactor
```

Sets the value by which the software shrinks the design before extraction. The shrink factor is used only for extraction calculations. It does not change the actual design.

Note: You do not need to specify the shrink factor if the capacitance table file in use contains the shrink factor specification in its header.

Parameters

`-help`	Outputs a brief description that includes type and default information for the `setShrinkFactor` parameter. For a detailed description of the command and its parameter, use the `man` command: `man setShrinkFactor`
`shrinkFactor`	Specifies the value by which the software shrinks a design. *Default:* `1.0`

Example

The following command sets the shrink factor to 0.9:

```
setShrinkFactor 0.9
```

Setting the shrink factor to 0.9 reduces the wire lengths, widths, and spacings in the design by 0.9 times the original values.

Related Commands

- `extractRC`
- `setDesignMode`
- `setExtractRCMode`

spefIn

```
spefIn
[-help]
fileNameList
[-extended]
[-idx]
[-scaleNets netFileName]
[-scaleRC]
[-starN | -noStarN]
[{-rc_corner list_of_rc_corners [-spef_field {1 2 3 … N}]}]
[-via_variation_file viaVariationFileName]
```

Loads resistors and capacitors for the interconnects in SPEF into the Innovus database to calculate delays or build a timing graph. Use this command to read in a SPEF file generated by a third-party extraction tool.

Note: You can use this command to load a SPEF file, which is compressed (`.gzip`) format.

Note: This command supports multi-threaded SPEF reading. The number of CPUs can be specified using the standard use model of Innovus.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `spefIn` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man spefIn`
`-extended`	Enables the reading of "`$llx=<value1> $lly=<value2> $urx=<value3> $ury=<value4> $LAYER=<number>`" to set the layer number for nodes of a resistance from Quantus extended SPEF file. It also reads in extended `*LAYER_MAP section`, if it is present. This parameter can also read in "`$l=<length> $w=<width> $si_w=<swidth> $lvl=<number>`" to set the width, silicon width, or layer for nodes of a resistance from `StarRC` extended SPEF file.
`-idx`	Enables Inter-Die Extraction (IDX) SPEF reading. IDX SPEF models the Inter-Die capacitive impact in the form of coupling and ground capacitances. Individual die and IDX SPEFs need to be extracted by Quantus and in the extended SPEF (xSPEF) format. The `-extended` parameter is also required to be specified to read the xSPEF format. The software also supports multiple IDX SPEF reading. Flat die-level SPEFs need to be specified along with IDX SPEFs. With the `-idx` parameter, die-level and IDX SPEFs can be specified in any order.
fileNameList	Specifies a SPEF file or list of SPEF files as input for flat chip or hierarchical SPEF stitching, respectively.
`-rc_corner` *list_of_rc_corners*	Annotates the parasitics to the predefined RC corners for multi-corner analysis. You can specify a list of RC corners to be annotated. If you want to use SPEF information when calculating delays in multi-mode multi-corner analysis (MMMC) mode, you must annotate the parasitics for all RC corners in the design.

A few examples are provided below to explain different scenarios of specifying rc corners are their corresponding corner indices and SPEF files.

- `spefIn -rc_corner {rcCorner1 rcCorner2 rcCorner3 rcCorner4 rcCorner5} -spef_field {1 2 3 4 5} rc-spef.gz`

  This means that SPEF file can contain parasitics from N RC corners and names of RC corners in the RC corner list map to the index fields specified using the `-spef_field` option as shown above. Assuming that the SPEF file, `rc-spef.gz` has RC fields from the specified five RC corners, the above command would be interpreted as follows:
  1st field in SPEF maps to RC corner, `rcCorner1`
  2nd field in SPEF maps to RC corner, `rcCorner2`
  3rd field in SPEF maps to RC corner, `rcCorner3`
  4th field in SPEF maps to RC corner, `rcCorner4`, and
  5th field in SPEF maps to RC corner, `rcCorner5`

- `spefIn -rc_corner {rc_max rc_min} -spef_field {2 5} rc-spef.gz`

  Assuming `rc-spef.gz` in above example has RC fields from "N" RC corners, the above command would read SPEF data for two RC corners only and would be interpreted as follows:
  2nd field in SPEF maps to RC corner, rc_max
  5th field in SPEF maps to RC corner, `rc_min`

- `spefIn -rc_corner {rcCorner1 rcCorner2 rcCorner3} -spef_field {1 1 3} rc-spef.gz`

  Assuming `rc-spef.gz` in the above example has RC fields from N RC corners, the above command would read SPEF data for 3 RC corners only and would be interpreted as follows:

  1st field in SPEF maps to RC corner, `rcCorner1`
  1st field in SPEF also maps to RC corner, `rcCorner2`
  3rd field in SPEF maps to RC corner, `rcCorner3`

- `spefIn -rc_corner {rcCorner1 rcCorner2 rcCorner3 rcCorner4 rcCorner5} -spef_field {1 2 3 4 5}`
  `{top.spef.gz block1.spef.gz block2.spef.gz}`

  In the above example, hierarchical design SPEF files are read in. It reads top level multi-field SPEF file followed by the multi-field SPEF files from `block1` and `block2`.

Note: SPEF reading will not be triggered until SPEF files for all active RC corners are specified.

`-scaleNets` *`netFileName`*

	Enables the scaling of resistances and capacitances for user-specified nets as per the specified scale factors. The list of nets is specified in a file. The scale factors are specified using the `create_rc_corner` and `update_rc_corner` commands. **Note**: This parameter can only be used if the `-scaleRC` parameter is specified. Otherwise, it will be ignored. For example: `spefIn -scaleRC -scaleNets netFileName -rc_corner r1 detail.spef`
`-scaleRC`	Scales the SPEF read in for the specified corner as per the `postRoute_cap`, `postRoute_xcap`, and `postRoute_res` scale factors defined using the `create_rc_corner` and `update_rc_corner` commands. When this parameter is specified, the scale factors for extraction effort-level `high` are used for RC scaling. However, if they are not available, the priority shifts to the scale factors for effort-level `medium` and `low`, in that order, if specified. **Note**: Scale factors of `postRoute_clkcap, postRoute_clkxcap,` and `postRoute_clkres` are not applied even when the net is a clock attribute.
`-spef_field {1 2 3 … N}`	Specifies the list of RC corner indices in SPEF to be loaded. This parameter provides the flexibility to perform order-independent and field-independent mapping to RC corners. For example, SPEF may contain data for 10 corner fields but you can choose to read data only for 6 corners. You can also choose to use data from one particular field to map to one or more RC corners. A few examples to illustrate different scenarios are provided in the description of the `-rc_corner` parameter above. **Note:** This parameter is relevant only when the `-rc_corner` parameter is specified. The `-spef_field` is an optional parameter, and if not specified, the software assumes the default fields of {1 2 3 … N} where N is equal to the number of RC corners provided in the `-rc_corner` list.
`-starN |-noStarN`	Specifies whether or not to read the *N statements in the SPEF file. The *N statements define the location of the RC nodes. The `-starN` parameter reads the *N statements and the `-noStarN` parameter ignores the *N statements. *Default:* `-StarN` (By default, the software reads the *N statements in the SPEF file.)
`-via_variation_file viaVariationFileName`	

	Specifies the name of the file for modeling via variation. The contents of the file include all the via layer names with corresponding MEAN_SHIFT and STDDEV so that all the VIA resistance in SPEF can be mapped to their own variation multiplier. This option is used to specify the via_variation file only when the design has a single corner.
	
	For example,
	
	`spefIn`
	
	`-rc_corner rcCornerName \`
	
	`-via_variation_file viaVariationFileName`

A sample via_variation file is provided below:

```
// corner_name  : rcMax

// wildcard to cover default
object_type     : design
delay_type      : via

layer_name      : *

area            : 0.20 0.40 0.80

// no nominal for wildcard default
table_min_sigma : 0.80 0.90 0.95

table_max_sigma : 1.60 1.30 1.15

table_meanshift : 0.05 0.03 0.02

table_stdev     : 0.10 0.05 0.02

// per via layer data
object_type     : design
delay_type      : via
layer_name      : VIA1

Area            : 0.0004 0.0008 0.0016

table_min_sigma : 0.80 0.90 0.95

table_max_sigma : 1.60 1.30 1.15

table_meanshift : 0.015 0.015 0.012

table_stdev     : 0.214 0.150 0.117

object_type     : design

delay_type      : via

layer_name      : VIA2

area            : 0.0004 0.0008 0.0016

table_min_sigma : 0.80 0.90 0.95

table_max_sigma : 1.60 1.30 1.15

table_meanshift : 0.015 0.012 0.014

…..

……

table_stdev : 0.223 0.183 0.140
```

Note: Each line in the file lists the basic components of the statistical variation approach.

The values in the "area" line list all possible areas. In the above example, the area line lists three cases that can happen in the design. It can list different number of cases, for example one or four. The subsequent lines list the corresponding values matching to the areas defined in the area line.

Examples

- The following command loads the SPEF file `TOPCHIP_SP.spef` for a flat chip methodology for RC corner, `corner1`:

  ```
  spefIn -rc_corner corner1 TOPCHIP_SP.spef
  ```

- The following command loads multiple SPEF files for hierarchical SPEF stitching:

  ```
  spefIn -rc_corner corner1 { PTN/results_conv/results_conv.spef tdsp_core/tdsp_core/tdsp_core.spef
  PTN/TOPCHIP_SP/TOPCHIP_SP.spef}
  ```

- The following commands first setup the appropriate scale factors for the corners `RCmax` and `RCmin` and then scale the SPEF read for the defined corners `RCmax` and `RCmin`:

  ```
  === MMMC setup ====

  create_rc_corner -name RCmax -postRoute_cap 1.4 -postRoute_xcap 1.4 -postRoute_res 1.4
  create_rc_corner -name RCmin -postRoute_cap 0.8 -postRoute_xcap 0.8 -postRoute_res 0.8
  ...

  === run script===
  ...

  spefIn -rc_corner RCmax typSPEF.spef -scaleRC
  spefIn -rc_corner RCmin typSPEF.spef -scaleRC
  ```

Related Commands

- rcOut
- create_rc_corner
- update_rc_corner

wireload

```
wireload
[-help]
[-outfile fileNamePrefix]
[-scale factor]
[-percent samplingRate]
[-cell cellName | -inst instanceName [-name modelName]]
[-instanceBased]
[-cellLimit moduleSize]
[-custom]
[-noSmooth]
[-netCoverRatio]
[-logarithmic]
[-peakCap]
[-view viewName]
```

Generates the wireload models and outputs the models in hierarchical and flat formats. Use this command after extracting the RC data in your design.

The wireload models in the hierarchical format are generated for cells and instances based on external nets (hierarchical view). The hierarchical format considers only nets that crosses the boundaries of child modules within the module, while the flat format considers all nets within the module and its child modules.The wireload models in the flat format are generated for cells and instances based on internal nets (flat view).

Two wireload model formats are available: standard and custom.

Parameters

`-help`	Outputs a brief description of the `wireload` command. For a detailed description of the command, use the `man` command: `man wireload`
`-cell` *`cellName`*	Specifies the cell name for generating a wireload model.
`-cellLimit` *`moduleSize`*	Specifies the module size limit of cells or instances for generating a wireload model. Cells or instances that contain fewer than the *`moduleSize`* cells or instances will not have wireload models generated. *Default:* `100`
`-custom`	Generates the custom wireload model format, which is also known as the long format. The default is the standard wireload model format, which is also known as the short format.
`-inst` *`instanceName`*	Specifies the name of the hierarchical instance in a path for which to generate a wireload model. The hierarchical instance can be at any point in the path or at the beginning or end of the path.
`-instanceBased`	Generates wireload models for each instance in the design. The default is to generate wireload models for each cell type used in the design.
`-logarithmic`	Uses logarithmic average of the fanout value to calculate the wireload model.
`-name` *`modelName`*	Renames the cell name or instance name in the wireload model file.
`-netCoverRatio`	Specifies the percentage of nets to be considered for generating wireload model.
`-noSmooth`	Suppresses the smoothing of the wireload models.
`-outfile` *`fileNamePrefix`*	Specifies the filenames for the generated wireload models. The software generates four files for the hierarchical and flat models, and one command file each to load the model files back into the synthesis tool.
`-peakCap`	Uses peak fanout value to calculate the wireload model.
`-percent` *`samplingRate`*	Specifies the sample rate of the number of nets for calculation of wireload models. The values are between `0.0` and `1.0` (all sampled). *Default:* `1.0`
`-scale` *`factor`*	Specifies the scaling value used to multiply the results. *Default:* `1.0`.
`-view` *`viewName`*	Generates the wireload models for the specified analysis view. *Default*: Uses the default analysis view.

Examples

- The following command generates instance-based wireload models and outputs two model files, `design_name.wl.flat` and `design_name.wl.hier` and two command files for loading:

  ```
  wireload -instanceBased
  ```

- The following command uses logarithmic average of the fanout value to calculate the wireload model and outputs two model files, `design_name.wl.flat` and `design_name.wl.hier`:

  ```
  wireload -celldtmf_chip -logarithmic-outfile dtmf.wl
  wireLoad Option : -cell dtmf_chip -logarithmic -outfile dtmf.wl design/module characterization
  flags:
  ...
  ...
  Generation hierarchical Wireload Model File dtmf.wl.hier ...
  Generation flat Wireload Model File dtmf.wl.flat ...
  ```

Related Commands

- `extractRC`
- `setExtractRCMode`

write_extraction_spec

```
 write_extraction_spec
[-help]
[-no_design_data]
[-out_dir {dir_name}]
[-rc_corners {list_of_rc_corners}]
```

Creates the Quantus command file or the Common Command Language (CCL) file in the current working directory and saves the design data for running standalone Quantus. The command may be specified with a directory path where technology library file, LEF list file (for DEF-based flow), and DEF or OA design will be stored. However, if no directory path is specified, then a directory will be created in the current directory. The name of the directory will be in the format: `tmp_qrc_pid_time`. You can also specify whether you want to save the design data or not by using the `-no_design_data` parameter. If it is specified, then no design data, such as DEF or OA design will be saved.

In addition, to generate a command file for the OA-based flow, ensure that the `setExtractRCMode -useQrcOAInterface` parameter is set to `true`.

Note: Run this command after loading the design, setting up the MMMC environment, and specifying the CCL layer map file, if required.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_extraction_spec` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_extraction_spec`
`-no_design_data`	When this parameter is specified, no design data, such as DEF or OA design will be saved. By default, the design data will be saved. If this option is specified, then follow the steps listed below to run Quantus. 1. For DEF-based flow, provide the DEF file while invoking standalone Quantus. 2. For OA-based flow, the command file that is generated will assume an OA design in the following directory: `<lib name>/<top level cell name>/for_qrc` The value of `<lib name>` and `<top level cell name>` will be written into the command file that is generated. If you want to specify a different OA design path, modify the command file. **Note**: It is recommended that you use this parameter if you already have the DEF or OA design saved.
`-out_dir {dir_name}`	Specifies the location of the output directory in which the technology library file and associated data, and design data will be saved. If no path is specified, then a directory, `tmp_qrc_pid_time` will be created in the current directory by default.
`-rc_corners {list_of_rc_corners}`	Specifies the list of RC corners for which the Quantus command file is created. By default, the command outputs all active RC corners. If this parameter is specified, then only the specified corners are included in the Quantus command file. For example, Consider two RC corners, `RC1` and `RC2`, of which only `RC1` is active. • If this parameter is not specified, then the command will generate a CCL file with one active corner, `RC1`. • If this parameter is specified with a non-active corner, `RC2`, then the command will create a CCL file for the specified corner. `write_extraction_spec -rc_corners RC2` *Default*: `all active corners`

Related Information

- `extractRC`
- `setExtractRCMode`
- The following topic in the *Innovus User Guide*
  - Standalone Quantus for Signoff Extraction

RC Extraction Global Variable

- extract_shrink_factor

extract_shrink_factor

`extract_shrink_factor` *`shrink_factor_value`*

Type: Double

Default: `1.0`

Specifies the shrink factor by which to shrink the design before extraction. The shrink factor is used only for extraction calculations. It does not change the actual design.

This global variable must only be used when restoring design.

Note: This global is not recommended for newer technologies where shrink factor is defined in the Quantus techfiles or capacitance tables.

To set this global, use the `set` command.

Example

- The following global variable sets the shrink factor to `0.9`:

  ```
  set extract_shrink_factor 0.9
  ```

  Setting the shrink factor to `0.9` means the value of wire lengths, widths, and spacings in the design will shrink to 90% of their original values.

33

Route Commands and Global Variables

- Route Commands
- Route Global Variables

Route Commands

- add_tracks
- addChannelDensityControl
- check_tracks
- checkTQuantusModelFile
- colorize_cell_library
- createShield
- createTQuantusModelFile
- deleteShield
- detailRoute
- dumpCongestArea
- dumpNanoCongestArea
- dumpNetsInCongestedArea
- earlyGlobalRoute
- ecoRoute
- fix_drc_with_patch
- getAttribute
- getNanoRouteMode
- getRouteMode
- globalDetailRoute
- globalRoute
- report_route
- report_tracks
- reportCongestArea
- reportRoute
- reportShield
- reportWire
- route_fix_signoff_drc
- routeDesign
- saveRouteGuide
- setAttribute
- setNanoRouteMode
- setRouteMode

- writeHif
- route_fix_ir

add_tracks

```
add_tracks
[-help]
[-honor_pitch]
[-keep_default_width]
[-mask_pattern {{layer1  mask [mask …]} [layer2 ...] }]
[-mode {replace append}]
[-offsets {layer1 horiz|vert [die_box] offset} [{layer2 …}] ...}]
[-pitch_pattern {layer1 offset offset pitch pitch [pitch pitch …]} [{layer2 …}]...}]
[-pitches {layer1 horiz|vert pitch} [{layer2 horiz|vert pitch}] ...}]
[-rect_only_pattern {{layer1 [pattern ...]} {layer2 [pattern ...]} .. }]
[-route_rule route_rule_name]
[-snap_ml_track_to_cell_pins]
[-width_pitch_pattern {{layer1 offset offset width width pitch pitch [width width pitch pitch]...} [{layer2 …}]...}]
```

Deletes existing routing tracks and generates new routing tracks based on the technology data or user specific values. If no parameters are given to the command, it examines the LEF layer offset and pitch values, but may adjust them based on the LEF width and spacing rules, via-rules, or the width and height of the standard cells to create tracks optimized for routing. If you want to override the automatically generated pitch and offset values, you should use the parameters below:

- Use `-offsets` and `-pitches` to create uniform track patterns.
- Use `-pitch_pattern` or `-width_pitch_pattern` for non-uniform track patterns.

The Innovus software generates tracks automatically. You do not have to run this command unless you want to override the defaults or if you have imported a DEF file using the `defIn` command and you wanted to overwrite the track definitions in the DEF file. To overwrite the track definitions, run this command after running the `defIn` command. Various warning messages are generated if any values given will likely give poor routing results (For example, A pitch less than minimum wire-to-via spacing, or a bottom routing layer tracks that do not align to the standard-cell heights.)

- Non-preferred routing tracks of each layer are aligned with the preferred routing tracks of the next lower layer.
- Non-preferred routing tracks of the first layer are aligned with the tracks of the next upper layer.

If the LEF FIRSTLASTPITCH property (or OA equivalent) is defined for a layer, then non-uniform tracks will be created with the first and last pitch of the tracks set to align to the standard cell rows. This makes the tracks aligned with the standard-cell power rails having a wider-pitch on both sides to allow a wider power-route with min-spacing to the next routing track.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `add_tracks` parameter. For a detailed description of the command and all of its parameters, use the man command: `man add_tracks`.
`-honor_pitch`	Honors the pitch specified in the LEF file. By default, the `add_tracks` command adjust the pitch to what it considers best. The `-honor_pitch` parameter forces `add_tracks` to use the layer pitch value specified in the LEF or OA technology file. If the layer pitch specified in the LEF file is smaller than the minimum line-to-via pitch, the pitch is reset to the minimum line-to-via pitch and a warning message is displayed in the log file. **Note:** The `-honor_pitch` parameter cannot be used with any of the options setting pitch values directly (`-pitches, -pitch_pattern, -width_pitch_pattern`). *Data_type*: bool, optional
`-keep_default_width`	Keeps user specified default width between routing tracks. **Note:** This enables the interoperability of width spacing patterns with Virtuoso. *Data_type*: bool, optional
`-mask_pattern {{layer1 [mask ...]} {layer2 [mask ...]} .. }`	

	Specifies the mask pattern for the specified layers that have multiple masks. The mask values are applied to the preferred direction tracks, and then repeat. There is no color for non-preferred direction tracks. If `-pitch_pattern`, or `-width_pitch_pattern` is given, the number of elements in `-mask_pattern` must match for each layer or an error occurs. • *layer*: You can use LEF layer names or built-in routing-layer names (Metal1 for the first routing layer, Metal2 for the second layer) for the *layer* name. • *mask*: Specify 1, 2, or 3. The layer must be defined to use the specified number of masks. For example, `-mask_pattern {metal1 1 2 3 1 2 metal2 1 2 1 2 1}` *Data_type*: string, optional
`-mode {replace append}`	Enables incremental replace/append operations for tracks and regenerates tracks for specified layers only. In the incremental mode, you can replace or append any specific tracks one-by-one. You do not need to specify a long list of track definitions anymore. • `replace`: Enables the `add_tracks` command to remove all the tracks in the specified layers and then only generate/add specified tracks from the command line. • `append`: When specified, the `add_tracks` command does not remove any tracks. It will append specified tracks into existing tracks. *Data_type*: enum, optional
`-offsets {`*layer1* `horiz|vert [die_box]` *offset*`} [{`*layer2* `…}] ...}`	
	Specifies the horizontal offset or vertical offset for the specified layers. • *layer*: You can use LEF layer names or built-in routing-layer names (Metal1 for the first routing layer, Metal2 for the second layer) for the *layer* name. • *offset*: Specifies the offsets (in microns) for the specified layer. **Note:** The value of *offset* is relative to the lower-left corner of the core_box that is aligned to the standard cell rows, and can be a negative number. If die_box is given, the *offset* is relative to the lower-left corner of the die_box rather than the core_box. **Note:** die_box should not be used for the first two routing layers that need to align with the standard cells pins, but it can be used to align some of the upper routing layer tracks with a power-route grid structure. For example, `-offsets {metal1 horiz 0.1 metal2 vert 0.0 metal4 vert die_box 0.3}` *Data_type*: string, optional
`-pitch_pattern {`*layer1* `offset` *offset* `pitch` *pitch* `[pitch` *pitch* `…]} [{`*layer2* `…}] ...}`	
	Specifies an offset, and pitch pattern for the tracks in the preferred routing direction when non-uniform spacing is needed. These values override any `-pitches` or `-offsets` values for this layer in the preferred direction. • *layer*: You can use LEF layer names or built-in routing-layer names (Metal1 for the first routing layer, Metal2 for the second layer) for the *layer* name. • *offset*: Specifies the offsets (in microns) for the specified layer. • *pitch*: Specifies the pitch value. You can repeat the last pitch value using `repeat num`. It is best to add {} to make it clear which pitch value is repeated. For example, `-pitch_pattern {metal1 offset 0.2 pitch 0.2 {pitch 0.1 repeat 9} pitch 0.2 metal2 offset 0.1 pitch 0.2 {pitch 0.1 repeat 8}` } **Note**: {} are ignored but allowed to make it easier to read the command. For example, in the following command the first track is placed at 44.00, the next 0.55 away and then 66.00 away and then repeat 0.55.and 0.66. `add_tracks -pitch_pattern { metal15 offset 44.00 pitch 0.55 pitch 66.00}` *Data_type*: string, optional
`-pitches {`*layer1* `horiz|vert` *pitch*`} [{`*layer2* `horiz|vert` *pitch*`}] ...}`	

	Specifies the horizontal pitch or vertical pitch for the specified layers. This is only used for layers that have uniform track spacing. • *layer*: You can use LEF layer names or built-in routing-layer names (Metal1 for the first routing layer, Metal2 for the second layer) for the *layer* name. • *pitch*: Specifies the pitch value (in microns). **Note**: {} are ignored but allowed to make it easier to read the command. For example, `-pitches {metal1 vert 0.1 metal2 horiz 0.1 metal4 horiz 0.3}` *Data_type*: string, optional
`-rect_only_pattern {{layer1 [pattern ...]} {layer2 [pattern ...]} .. }`	
	Enables you to specify which subset of tracks would only allow rectangular shapes (rect_only). **Note:** In the GUI, 'rect_only' is added behind the width to indicate that the track has the “RECTONLY PARTIALTRACKS” LEF property assigned to it. The presence of the *rect_only* tag indicates that the track has the 'rectangular shape only' restriction and should be honored in routing. • *layer*: The layer must be defined to use the rect_only routing pattern. • *pattern*: Use 0 or 1 to specify the rect_only routing pattern. ◦ 0: Indicates no rect_only in generated tracks. ◦ 1: Indicates rect_only in generated tracks. For example, `add_tracks -rect_only_pattern {m2 0 0 1 1 1}` *Data_type*: string, optional

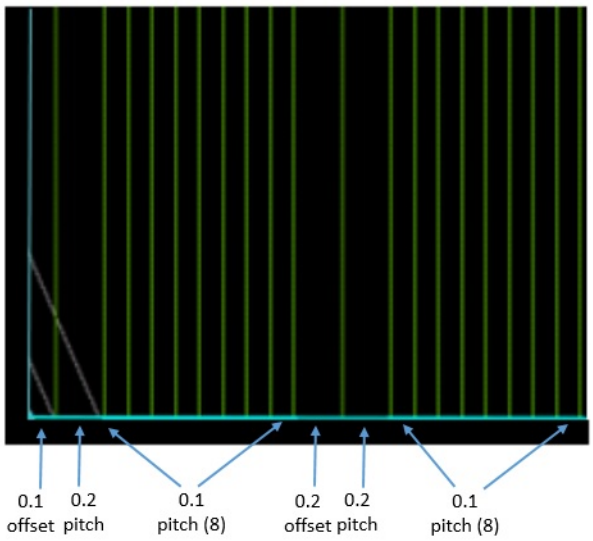
`-route_rule route_rule_name`	Specifies the name of a non-default route rule. When specified, the existing tracks are not deleted. Instead, an additional set of tracks are created for use by this *route_rule_name* on top of the existing tracks.
	Note: When the `-route_rule` parameter is used, you must specify everything for this specific *route_rule_name* in one command, irrespective of what `-mode` option you specify.
	Example:
	In the following command, M6 and M8 NDR tracks are added in one command. These should ***NOT*** be done in the two commands.
	`set addTrack_pitch_pattern_2w "{\`
	`{m6 offset 0.490 width 0.076 pitch 0.076 width 0.076 pitch 0.076 width 0.076 pitch 0.076 width 0.076 pitch 0.076 width 0.076 pitch 0.304} \`
	`{m8 offset 0.490 width 0.076 pitch 0.076 width 0.076 pitch 0.076 width 0.076 pitch 0.076 width 0.076 pitch 0.076 width 0.076 pitch 0.304}}"`
	`add_tracks -width_pitch_pattern $addTrack_pitch_pattern_2w -mode append -route_rule {TrunkNDR_2W}`
	Note: To later modify/append NDR tracks, you must delete all tracks (including default and NDR tracks) using the `deleteTrack` command and then regenerate all of them.
	Data_type: string, optional

`-snap_m1_track_to_cell_pins`	Extracts pins to generate tracks for the first routing layer. When specified, it scans all standard cells in the library and tries to extract all possible rectangle pins shapes and mask numbers. Tracks are created to hit the center of these pins. All tracks are assigned a mask based on the pin mask value, if specified. This is currently only intended for some 10nm technologies. *Data_type*: bool, optional
`-width_pitch_pattern { {layer1 offset offset width width pitch pitch [width width pitch pitch]...} [{layer2 …}] ...}`	

	Specifies track width patterns for the preferred direction. Use { } to specify track width pattern for more than one layer.
	Specifies the width and pitch pattern for the tracks in the preferred routing direction, for technologies that need a width value attached to each track. These values override any `-pitches` or `-offsets` values for this layer in the preferred direction. If `-pitch_pattern` is given for any of the layers, `-width_pitch_pattern` will override the `-pitch_pattern` values.
	• *`layer`*: You can use LEF layer names or built-in routing-layer names (Metal1 for the first routing layer, Metal2 for the second layer) for the *`layer`* name.
	• *`width`*: Specifies the width value.
	• *`pitch`*: Specifies the pitch value (in microns).
	Note: You can repeat the last width and pitch value using `repeat num`. It is best to add {} to make it clear which value are repeated.
	For example,
	`-width_pitch_pattern`
	`{metal1 offset 0.2 width 0.2 pitch 0.2 {width 0.1 pitch 0.1 repeat 8} width 0.1 pitch 0.2`
	`metal2 offset 0.1 width 0.2 pitch 0.2 {width 0.1 pitch 0.1 repeat 8} width 0.1 pitch 0.2 }`
	Data_type: string, optional

Examples

- The following command deletes any existing tracks, and generates new routing tracks based on the LEF or OA layer offset and pitch settings, and the standard-cell pins.
  ```
  add_tracks
  ```
- The following command generates uniform vertical tracks on metal1 with a horizontal offset of 0.1 from the core_box, a horizontal pitch of 0.2, and horizontal tracks on metal2 with a vertical offset of 0.2 from the core_box, and a vertical pitch of 0.3. Other layers will use the LEF or OA layer settings.
  ```
  add_tracks -offsets {metal1 horiz 0.1 Metal2 vert 0.2} -pitches {metal1 horiz 0.2 Metal2 vert 0.3}
  ```
- The following command generates a non-uniform pattern of tracks for the preferred routing direction, using the values given. Extra {} are added to make it more readable.
  ```
  add_tracks -pitch_pattern {metal1 offset 0.1 pitch 0.2 {pitch 0.1 repeat 8} metal2 offset 0.2 pitch 0.2 {pitch 0.1 repeat 8} }
  ```



- The following command generates a non-uniform pattern of tracks with width values for the preferred routing direction, using the values given settings.
  ```
  add_tracks -width_pitch_pattern \
  {{metal1 offset 0.1 width 0.2 pitch 0.2 {width 0.1 pitch 0.1 repeat 8} width 0.1 pitch 0.2} \
  {metal2 offset 0.2 width pitch 0.2 {pitch 0.1 repeat 8}} }
  ```

Pitch Rule Examples

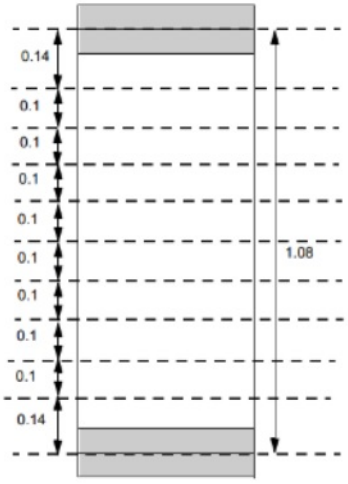
Example 1

The following example shows how routing_tracks are created for a layer with FIRSTLASTPITCH.
Suppose the LEF has this statement for the Metal1 layer.

```
PROPERTY LEF58_PITCH STRING "
    PITCH 0.1 FIRSTLASTPITCH 0.14 ; ";
```

Then the horizontal metal1 tracks will be created as shown below by default, with no extra parameters passed to add_tracks.

Example 2



In this example, the cell height is 1.08. The dotted line represents the routing tracks. The first and last tracks are 0.14 from the cell row boundary while the intermediate tracks are 0.1 apart making 2 * 0.14 + 8 * 0.1 = 1.08. The routing track definition would be repeated based on the cell rows.

Related Information

- report_tracks
- check_tracks
- Using the NanoRoute Router chapter in the *Innovus User Guide*
  - Adding Tracks
  - Running the NanoRoute Router with Innovus Text Commands

addChannelDensityControl

```
addChannelDensityControl
[-help]
llx lly urx ury layer ratio
```

Controls the routing density for a specified area on a layer by setting a maximum density percentage that early Global Route cannot exceed. Use the `addChannelDensityControl` command after performing placement.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `addChannelDensityControl` parameter. For a detailed description of the command and all of its parameters, use the man command `man addChannelDensityControl`.
`layer`	Specifies the layer on which to specify the routing density control. **Note:** Layers can be specified using the layer names or layer ID numbers. *Data_type*: string, required
`llx`	Specifies the lower left x coordinate of the density area. *Data_type*: float, required
`lly`	Specifies the lower left y coordinate of the density area. *Data_type*: float, required
`ratio`	Specifies the maximum routing density allowed for the area, as a percentage. *Min*: 0 *Max:* 100 *Value:* `0` (zero percent) to `1` (100 percent) *Data_type*: float, required
`urx`	Specifies the upper right x coordinate of the density area. *Data_type*: float, required
`ury`	Specifies the upper right y coordinate of the density area. *Data_type*: float, required

Examples

The following command creates a routing density control area on layer Metal3 with a maximum density of 50 percent:

```
addChannelDensityControl 10 10 500 30 Metal3 0.50
```

Related Information

- `setRouteMode`
- `getRouteMode`
- Using Early Global Route for Congestion and Timing Analysis chapter in the *Innovus User Guide*

check_tracks

```
check_tracks
[-help]
[-direction {horiz | vert | H | V}]
[-grid spacing]
[-layer layer_name]
[-mask mask_name]
[-offset string]
[-origin {design | core | D | C}]
```

Checks a predefined set of tracks to verify their routing track offset, direction, and grid. It reports the following conditions in the Innovus log file:

1. Tracks whose `STEP` is smaller than the minimum pitch for the designated layer.
2. The percentage of on-track pins and pins that are not accessible (for regular routing).
3. Fully blocked pins and potential off-grid track pins (for special routing).

Before running this command, complete the following steps:

- Load a LEF file with the library and technology information.
- Place the design.

Parameters

`-direction {horiz | vert | H | V}`	Specifies the direction of the tracks to check. *Data_type*: string, optional
`-grid spacing`	Specifies the track spacing. *Data_type*: string, optional
`-help`	Outputs a brief description that includes type and default information for each `check_tracks` parameter. For a detailed description of the command and all of its parameters, use the man command: `man check_tracks`
`-layer layer_name`	Specifies the track layer. *Data_type*: string, optional
`-mask mask_name`	Specifies the mask name. *Data_type*: string, optional
`-offset string`	Specifies the track offset from the starting point. *Data_type*: string, optional
`-origin {design | core | D | C}`	Specifies the track starting point. *Data_type*: string, optional

Examples

- The following command checks the tracks in the horizontal direction on layer M3 with a grid spacing of 3.
  ```
  check_tracks -direction H -layer M3 -grid 3
  ```

  The following output displays on screen:

  ```
  Total 35 tracks meet specification: layer=Metal3 dir=horiz offset=0 grid=3
  Total 2622 tracks not hit under specified command line options
  ```

Related Information

- `add_tracks`
- `report_tracks`
- Using the NanoRoute Router chapter in the *Innovus User Guide*
  - Resolving Open Nets
  - Diagnosing Problems Using the check_tracks Command

checkTQuantusModelFile

```
checkTQuantusModelFile
[-help]
```

Checks the TQuantus model file created using the `createTQuantusModelFile` command. It ensures that the TQuantus model file is consistent with the following input files inside the Innovus database:

- QRC techfiles
- Pitch information from the technology LEF file
- RC corners

The following commands invoke this command automatically to check the consistency of the TQuantus model file if TQuantus is chosen as the extraction engine.

- `routeDesign -trackOpt`
- `optDesign -postRoute`
- `timeDesign`
- `extractRC`

Note: The software issues warnings if inconsistencies are detected between the TQuantus file and the RC corner information, QRC techfile, or pitch information in the LEF technology file. If such warnings are issued, the TQuantus model file will have to be re-generated manually.

Parameter

`-help`	Outputs a brief description of the `checkTQuantusModelFile` parameter. For a detailed description of the command, use the man command: `man checkTQuantusModelFile`.

Example

- The following commands create the RC model data and store it in the `tQuantus_model.bin` file and then check the TQuantus model file:

  ```
  createTQuantusModelFile -file tQuantus_model.bin
  checkTQuantusModelFile
  ```

Related Topic

- `createTQuantusModelFile`
- `routeDesign`
- `optDesign`
- `timeDesign`
- `extractRC`
- Using the NanoRoute Router chapter in the *Innovus User Guide*
  - Creating RC Model Data in TQuantus Model File

colorize_cell_library

```
colorize_cell_library
[-help]
[-cell cell_name]
[ [-reset ] | [-enforce ] ]
```

Colors each standard cell in the library, using the pre-color method.

Parameters

`-cell` *cell_name*	Specifies the name of the cell to colorize. The `colorize_cell_library` command only targets the specified cell, irrespective of the cell type. *Data_type*: (libCell/ptnCell)+, optional
`-help`	Outputs a brief description that includes the type and default information for each `colorize_cell_library` parameter. For a detailed description of the command and all of its parameters, use the man command `man colorize_cell_library`.
`-enforce`	Specifies that re-colorization will be enforced. It colors the cell even if it is already colored. *Data_type*: string, optional
`-reset`	Resets the library cell color data. It cleans the color of every standard cell, except the LEF-color. *Data_type*: bool, optional

Example

- The following command resets the library cell color data.
  ```
  colorize_cell_library -reset
  ```

Related Information

- `setNanoRouteMode`
- Using the NanoRoute Router chapter in the *Innovus User Guide*

createShield

```
createShield
[-help]
[-selected]
[-crosstie_only]
[-include_fixed]
```

Creates shield wires for the NanoRoute router. After creating the shields, the router generates a report that includes shielding coverage statistics.

Parameters

`-crosstie_only`	Adds crosstie for coaxial shielding only. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes the type and default information for each `createShield` parameter. For a detailed description of the command and all of its parameters, use the man command `man createShield`
`-include_fixed`	Includes fixed wires for shielding. The wire will not be changed but the electrical characteristics can be altered by adding shielding so that this option is required to include the fixed wire for shielding. *Data_type*: bool, optional
`-selected`	Creates shield wires for the selected nets. If you do not specify this option, all the clock nets will be shielding. *Data_type*: bool, optional

Example

- The following command creates shield wires for selected nets:
  ```
  createShield -selected
  ```

Related Information

- `deleteShield`
- `reportShield`
- Using the NanoRoute Router chapter in the *Innovus User Guide*
  - Performing Shielded Routing

createTQuantusModelFile

```
createTQuantusModelFile
[-help]
-file rc_model_file_name
[-reset]
```

Creates the RC model data and stores it in a TQuantus model file. This data is used by the RC extractor. This command eliminates the need for the RC extractor to rebuild the RC models every time it runs. It can just use the model data created using this command. Before creating the TQuantus model file, ensure the following:

- QRC techfiles are available
- Pitch information from the technology LEF file is available
- All RC corners are set up

The consistency of the created TQuantus model file is checked using the `checkTQuantusModelFile` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `createTQuantusModelFile` parameter. For a detailed description of the command and all of its parameters, use the man command `man createTQuantusModelFile`.
`-file rc_model_file_name`	Writes the created RC model data into a file with the specified file name. *Data_type*: string, optional
`-reset`	Resets the RC model file. *Data_type*: bool, optional

Example

- The following command creates the RC model data and stores it in the `tQuantus_model.bin` file:
  ```
  createTQuantusModelFile -file tQuantus_model.bin
  ```

Related Topic

- `checkTQuantusModelFile`
- `routeDesign`
- `optDesign`
- `timeDesign`
- `extractRC`
- Using the NanoRoute Router chapter in the *Innovus User Guide*
  - Creating RC Model Data in TQuantus Model File

deleteShield

```
deleteShield
[-help]
{-nets list_of_signal_nets | -selected}
[-routes_only {0 | 1}]
```

Deletes physical shielding wires from the specified or selected signal nets and removes `SHIELDNET` attributes from the nets. Use the `deleteShield` command after setting the `SHIELDNET` attribute on a net. To set the `SHIELDNET` attribute, use the setAttribute command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `deleteShield` parameter. For a detailed description of the command and all of its parameters, use the man command `man deleteShield`
`-nets list_of_signal_nets`	Specifies a list of signal nets with one or more `SHIELDNET` attributes. You can use the asterisk (`*`) wildcard in the net names. *Data_type*: string, optional
`-routes_only {0 | 1}`	Specifies that only shield wires and vias will be deleted. *Data_type*: enum, optional
`-selected`	Lets you select nets. *Data_type*: bool, optional

Examples

- The following command deletes `SHIELDNET` attributes and physical shielding wires from net `clk1`:
  ```
  deleteShield -nets clk1
  ```
- The following command deletes `SHIELDNET` attributes and physical shielding wires from nets `c*`:
  ```
  deleteShield -nets c*
  ```
- The following command deletes `SHIELDNET` attributes and physical shielding wires from selected nets:
  ```
  deleteShield -selected
  ```

Related Information

- `createShield`
- `reportShield`
- Using the NanoRoute Router chapter in the *Innovus User Guide*
  - Performing Shielded Routing

detailRoute

```
detailRoute
[-help]
[-select]
[area]
```

Uses the NanoRoute router to perform detailed routing on the entire design, an area of the design specified by the bounding box, or on selected nets. If you do not specify parameters, the router routes the entire design. If you specify both the bounding box and selected nets, the router routes only selected nets within the bounding box. You can perform detailed routing on specified area only if you are rerouting the design. If you have not already run detailed routing on the entire design, you cannot run detailed routing on a specified area.

Note: This command supports multiple-CPU processing.

Use this command after completing the following steps:

1. Placing the design.
2. Routing special nets.
3. Setting net attributes with `setAttribute` and NanoRoute run-time options with `setNanoRouteMode`.
   Note: This step is optional. Complete this step only if you do not want to use the current values for attributes or default values for options.
4. Running global routing with `globalRoute`.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `detailRoute` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man detailRoute`
area	Specifies the x and y coordinates of the bounding box area. *Data_type*: rect, optional
`-select`	Routes selected nets. To select a net, use the `selectNet` command or click the net in the design display window. **Tip**: You can also specify `setNanoRouteMode -route_selected_net_only true` to route the selected nets. *Data_type*: bool, optional

Example

The following command selects `net1` and runs detailed routing on it:

```
selectNet net1
detailRoute -select
```

Related Information

- `globalRoute`
- `globalDetailRoute`
- `routeDesign`
- `setAttribute`
- `setNanoRouteMode`
- Using the NanoRoute Router chapter in the *Innovus User Guide*
  - Detailed Routing

dumpCongestArea

```
dumpCongestArea
[-help]
file_name
[-all]
```

Writes the routing congestion information to an ASCII output file. The output file reports the coordinates of overflow cells, vertical tracks, and horizontal tracks. You can use the `dumpCongestArea` command after running Early Global Route.

Parameters

`-all`	Writes the congestion information for all gcells in the design to the specified output file, regardless of the congestion level. If the `-all` parameter is not specified, this command writes the congestion information for only the most congested gcells in the design to the specified output file. *Data_type*: bool, optional
file_name	Specifies the name of the output file to which the congestion information is written. *Data_type*: string, required
`-help`	Outputs a brief description that includes the type and default information for each `dumpCongestArea` parameter. For a detailed description of the command and all of its parameters, use the man command `man dumpCongestArea`.

Example

- The following command writes the routing congestion information for all gcells to the `congest.txt` file:

  ```
  dumpCongestArea -all congest.txt
  ```

Related Information

- `dumpNanoCongestArea`
- `dumpNetsInCongestedArea`
- `reportCongestArea`
- Using Early Global Route for Congestion and Timing Analysis chapter in the *Innovus User Guide*

dumpNanoCongestArea

```
dumpNanoCongestArea
[-help]
file_name
[-all]
```

Writes the NanoRoute routing congestion information to an ASCII output file. The output file reports the total number of overflowed tracks, and the coordinates of overflow cells, vertical tracks, and horizontal tracks. The following example shows the information for a gcell in the ASCII output file:

```
(1192900, 974400) (1209700, 991200) V: -1/9/24 H: 0/16/39
```

The gcell has one vertical (`V`) track overflow (`-1`) from a layer. However, if all tracks are counted, it has nine remaining vertical tracks among 24 total tracks.

For horizontal (`H`) tracks, there was no overflow from any layer (`0`), and it has 16 remaining tracks among 39.

You can use the `dumpNanoCongestArea` command after running the NanoRoute `globalRoute` command.

Parameters

`-all`	Writes the congestion information for all gcells in the design to the specified output file.
`file_name`	Writes the congestion information for only the most congested gcells in the design to the specified output file.
`-help`	Outputs a brief description that includes the type and default information for each `dumpNanoCongestArea` parameter. For a detailed description of the command and all of its parameters, use the man command `man dumpNanoCongestArea`.

Related Information

- `dumpCongestArea`
- `dumpNetsInCongestedArea`
- `earlyGlobalRoute`
- Using the NanoRoute Router chapter in the *Innovus User Guide*

dumpNetsInCongestedArea

```
dumpNetsInCongestedArea
[-help]
file_name
```

Outputs the names of nets in the congested area of the design into the specified file, based on `earlyGlobalRoute` results. You can use the `dumpNetsInCongestedArea` command after running Early Global Route.

Parameters

file_name	Specifies the name of the file in which to output the net names. *Data_type*: string, required
-help	Outputs a brief description that includes the type and default information for each `dumpNetsInCongestedArea` parameter. For a detailed description of the command and all of its parameters, use the man command `man dumpNetsInCongestedArea`.

Related Information

- `dumpCongestArea`
- `dumpNanoCongestArea`
- `earlyGlobalRoute`
- Using Early Global Route for Congestion and Timing Analysis chapter in the *Innovus User Guide*

earlyGlobalRoute

```
earlyGlobalRoute
[-help]
```

Early Global Route (`earlyGlobalRoute`) is a quick global routing for estimating routing-related congestion and parasitic (resistance and capacitance) values. The Early Global Route results are also used for pin assignment when you commit partitions in hierarchical designs. Early Global Route does not guarantee DRC-clean routing results. Do not perform signal integrity analysis on a design that has been routed using Early Global Route, because these routes are only used to estimate parasitic values for timing analysis. Route designs with the NanoRoute router if you want to perform signal integrity analysis.

Wire overlap can occur if a gcell is only partially covered by a routing blockage or pre-routed wires. Overlapping of non-default rule wires can also occur during track assignment; however, this information is modeled when the router estimates congestion. You can use the `earlyGlobalRoute` command after running placement. If the design uses additional placement features, such as clock tree synthesis and scan groups, use the command after running these placement features. If you have run IPO commands on your design, you must run the `earlyGlobalRoute` command to update interconnections for the design.

Notes:

- You can set the global parameters for the Early Global Route program by using the `setRouteMode` command. Parameters that you specify with `setRouteMode` are then used automatically whenever you run Early Global Route, either explicitly through the `earlyGlobalRoute` command or internally during in-place optimization, partitioning, or placement.
- The `assignPtnPin` command reports the number of guide pins from route guides and fixed pins. Since `earlyGlobalRoute` chooses guide points provided by `assignPtnPin` to lay wires on partition boundary, the `earlyGlobalRoute` report clearly differentiates between fixed pins and the pins which are in the pin guide.
- The Early Global Router supports nested partitions. The partitions specified with the `setRouteMode -earlyGlobalRoutePartitionHonorFence` and the `setRouteMode -earlyGlobalRoutePartitionHonorPin` parameters are routed in the style that honors fence and pin constraints. All other partitions are still routed flat.
- The Early Global Router supports get_metric APIs and enables you to query existing routing metrics at command line. To enable this capability, `routeDesign` just calls the 'get_metrics' API after generating the routing metrics.
- If bus guides have been created in the GUI, the `earlyGlobalRoute` command honors the bus guides automatically.
- The `earlyGlobalRoute` command honors multiple-CPU setting for track assignment inside `earlyGlobalRoute` by default.
- The Early Global Router supports blackboxes by default. It internally calls the `assignPtnPin` command and places and routes all unplaced blackbox pins.
- The Early Global Router supports uses two-row-height gcells with congestion modelling improvement for the technology nodes whose row-height is smaller than 7 track. It uses 2x2 gcell size as default for small default gcell size (<=7). By default, it checks for the number of tracks rather than checking the technology of the design to grow gcell size.
  Note: This enhancement is designed only for N7 designs as these designs have less than 7 tracks. If an N7 design has more than 7 tracks then these setting will not be applied in order to ensure a better congestion accuracy.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `earlyGlobalRoute` parameter. For a detailed description of the command and all of its parameters, use the man command: `man earlyGlobalRoute`.

Example

- The following command honors partition fences with single entry and does not allow crossing other partitions, when `earlyGlobalRoute` is run:

```
setRouteMode -earlyGlobalRoutePartitionHonorFence {.}
earlyGlobalRoute
```

- The following command does quick global routing for estimating routing-related congestion and parasitic values.

```
earlyGlobalRoute
```

```
#% Begin earlyGlobalRoute (date=02/10 16:11:30, mem=1043.0M)
[NR-eGR] Started Early Global Route kernel ( Curr Mem: 1187.03 MB )
[NR-eGR] Num Prerouted Nets = 0 Num Prerouted Wires = 0
[NR-eGR] Read 7482 nets ( ignored 0 )
[NR-eGR] Layer group 1: route 7425 net(s) in layer range [2, 6]
[NR-eGR] Early Global Route overflow of layer group 1: 0.01% H + 0.00% V. EstWL: 5.111114e+05um
[NR-eGR] Overflow after Early Global Route 0.03% H + 0.00% V
[NR-eGR] Length (um) Vias
[NR-eGR] ------------------------------------
[NR-eGR] Metal1 (1H) 0 25155
[NR-eGR] Metal2 (2V) 134127 35802
[NR-eGR] Metal3 (3H) 189844 5296
[NR-eGR] Metal4 (4V) 95825 1646
[NR-eGR] Metal5 (5H) 77137 416
[NR-eGR] Metal6 (6V) 29560 0
[NR-eGR] ------------------------------------
[NR-eGR] Total 526492 68315
[NR-eGR] ------------------------------------------------------------------------------
[NR-eGR] Total half perimeter of net bounding box: 471333um
[NR-eGR] Total length: 526492um, number of vias: 68315
[NR-eGR] ------------------------------------------------------------------------------
[NR-eGR] Total eGR-routed clock nets wire length: 0um, number of vias: 0
[NR-eGR] ------------------------------------------------------------------------------
[NR-eGR] Finished Early Global Route kernel ( CPU: 0.47 sec, Real: 0.72 sec, Curr Mem: 1225.82 MB )
#% End earlyGlobalRoute (date=02/10 16:11:31, total cpu=0:00:00.5, real=0:00:01.0, peak res=1073.6M, current mem=1058.7M)
```

Related Information

- `setRouteMode`
- `getRouteMode`
- Using Early Global Route for Congestion and Timing Analysis chapter in the *Innovus User Guide*

ecoRoute

```
ecoRoute
[-help]
[-ignore_route string]
[-handlePartition ]
[-fix_drc [-cut_color_flip {0 1}]
[-trim_layer_patch ] [-layer_range bottomLayer:topLayer] [area]]
[-fix_filler_drc_with_patch_only [-passive_fill ] [-layer_range bottomLayer:topLayer] [area]]
[-fix_filler_drc_with_patch_only | -fix_drc ]
[-target | -fix_drc | -prototype ]
[-modifyOnlyLayers bottomLayer:topLayer | -fix_drc ]
```

This command is part of an ECO flow process and is based on the NanoRoute® router. Based on information from the `ecoDefIn` command, this command preprocesses the design for the NanoRoute router and launches ECO routing within the router. This command retrieves the router's mode settings and returns them to their original state at the end of the ECO routing operation.

- When you specify two layers to modify, the first number representing the bottom layer should be smaller than the second number representing top layer (for example, `-modifyOnlyLayers 2:3`), `ecoRoute` checks for pin accessibility for new or modified nets and drops `VIA12,` as necessary.

  Note: The `via12` and `via34` will be used, but tool will not change the metal shapes on M1 and M4.
- When a net is modified, some parts of the wire must be reassigned to the new nets. Initially, a whole wire between two cells is assigned to `netA`. When a buffer is inserted on `netA`, `ecoRoute` cuts the wire segment and assigns the left side of the wire to `netA` and the remaining segment on the right to `netB`.
- You must first run `ecoDefIn` and `ecoPlace` before you use this command.
- You can run ECO routing in multiple-CPU processing mode.

Parameters

area	Fixes DRC violations is the specified route area. You can specify the area using the *x1 y1 x2 y2* coordinate values. **Note**: The parameter must be used with the `-fix_drc` parameter. *Data_type*: rect, optional
`-cut_color_flip {0 1}`	
	Fixes cut color DRC violations by flipping. When enabled (1), the ecoRoute command fixes the cut same-mask spacing violations by flipping the color of the cut. **Note**: The parameter must be used with the `-fix_drc` parameter.
`-fix_drc`	Fixes violations on existing DRC markers in the design. When enabled, it allows NanoRoute to fix only the DRC's on existing DRC markers in the design. If violations increase then the ecoRoute command reverts to the original snap shot of the design. **Note:** If open nets are found in the design, NanoRoute will error out. **Note:** This parameter does not support the `-modifyOnlyLayers` arguments. *Data_type*: bool, optional
`-fix_filler_drc_with_patch_only`	Fixes filler related drc with metal patch only. • If only the filler-related metal patch drc exists after filler insertion, it can be fixed with the `-fix_filler_drc_with_patch_only` parameter. • The recommendation flow for chip-finish is: 1. `ecoRoute`. For changed cell eco 2. `addFiller`. Inserts full fillers, even if decap and filler insertion is performed in single or multiple steps. 3. `ecoRoute -fix_filler_drc_with_patch_only` **Note:** Some decap cells might be replaced with normal filler cells due to handling of the vertical stack rule during `addFiller`. As a result DRC violations relating to patch metal may still be left after full filler insertion if router is called after only decap cells are inserted. Hence, Cadence recommends you to follow the above mentioned chip-finish flow. *Data_type*: bool, optional

`-help`	Outputs a brief description that includes the type and default information for each `ecoRoute` parameter. For a detailed description of the command and all of its parameters, use the man command: `man ecoRoute`
`-handlePartition`	Turns on handling partition flag. For the nets in hierarchical partition design, the ECO routing performed is partition aware with this option. • Pure top level nets should be routed only in top area. • Inter-partition nets should be routed based on partition pin locations, and there are no multiple entries to a connected partition, the specific partition pin is the only entry for the partition connection. • Intra-partition nets should be routed inside corresponding partition fence. *Data_type*: bool, optional

`-ignore_route` *string*	When specified, the selected net will be routed with highest priority by ignoring existing routes in post-route mode.
	• `all`: Ignores all other nets in the design that are not selected, including clock, secondary PG, and signal nets.
	• `signal`: Ignores only other signal nets that are not selected, but honors clock and secondary PG nets.
	For example, the following commands select and route a list of nets by ignoring only other signal nets, and considers secondary PG and clock nets during routing. After that, a second `ecoRoute` is run to fix DRC violations:
	`selectNet {net1 net2 net3}`
	`setNanoRouteMode -route_selected_net_only true`
	`ecoRoute -ignore_route signal`
	`setNanoRouteMode -reset -route_selected_net_only`
	`verify_drc -limit 0`
	`ecoRoute`
	Default: " "
	Type: string, optional

`-layer_range` *bottomLayer:topLayer*	Enables the NanoRoute router to fix violations on existing DRC markers in the specified layer range (*bottomLayer:topLayer*). *Type:* string, optional
`-modifyOnlyLayers` *bottomLayer:topLayer*	
	Enables the NanoRoute router to remove segments of net routing in the specified layer range, from `bottomLayer` to topLayer, inclusively. You must include a colon (`:`) between the `bottomLayer` and `topLayer` values. This option also restricts eco routing to carry out only on eco nets. *Default:* ECO routing can occur on any layer and any nets. **Note:** You might want to use `-modifyOnlyLayers` to specify all layers in order for `ecoRoute` to avoid touching non-eco nets. In some cases, approach could be faster and result in fewer DRCs. This command is similar to `-bottomRoutingLayer` and `-topRoutingLayer` parameters of the `setDesignMode` command and the `-route_with_eco` parameter of the `setNanoRouteMode` command. For more information on the `ecoRoute` command, see the ECO Flows chapter of the *Innovus User Guide*. **Note:** The NanoRoute router does not support the `-modifyOnlyLayer` option for 10nm and below nodes and displays an error when used. **Note:** The `-modifyOnlyLayers` parameter cannot be specified with the `-fix_drc` parameter. **Note:** The recommended methods to know the layers and vias that were touched in the freeze metal routing with `-modifyOnlyLayers` *bottomLayer:topLayer* are: • Use the `defComp -metalShapeDiff` command to compare the def file before and after using the `ecoRoute` command. • Use `Pegasus` to compare the gds file. *Data_type*: string, optional
`-passive_fill`	When specified, it does passive fill after filler fixing. This helps to save extra import/export time. *Data_type*: bool, optional

`-prototype`	Enables prototyping while doing ECO routing. When specified, it enables NanoRoute to work on the nets selected to route and stop early before DRC cleanup to provide a quick snapshot for the designer. This mode is intended to help the designer know if the routing is possible and if the ECO can improve timing. **Note:** In this mode the routes are not DRC clean
`-target`	Enables NanoRoute to work on only eco nets identified by NanoRoute to route. In this mode, the router ignores the already existing DRC violation markers that are not on the ECO nets or are on nets selected to route. *Data_type*: bool, optional
`-trim_layer_patch`	Enables the ecoRoute command to only add trim and patch metals in a given area. Using the `-trim_layer_patch` parameter, you can fix drc violations flagged in sign-off/verify_drc and only add trim and patch metals where minimum spacing criteria is not met. *Data_type*: bool, optional

Examples

The following example allows the NanoRoute router to perform ECO routing only on or between layers 2 and 3, VIA12 will be dropped if necessary :

```
ecoRoute -modifyOnlyLayers 2:3
```

If DRCs exist after you run the `ecoRoute` command, this could be because `ecoRoute` cannot touch non-eco nets. You can reroute the DRC area as shown below:

```
setNanoRouteMode -quiet -route_with_eco true
ecoRoute -modifyOnlyLayers 1:3
setNanoRouteMode -quiet -route_selected_net_only false
globalDetailRoute 100.0 1200.0 350.0 600.0
```

Related Topic

- `ecoDefIn`
- `ecoPlace`
- `setNanoRouteMode`
- `globalDetailRoute`
- ECO Flows chapter in the *Innovus User Guide*
- Accelerating the Design Process By Using Multiple-CPU Processing chapter in the *Innovus User Guide*
- Tools Menu chapter in the *Innovus Menu Reference*
  - Multiple CPU Processing

fix_drc_with_patch

```
fix_drc_with_patch
[-help]
[area]
[-layer_range bottomLayer:topLayer]
```

Fixes violations by adding trim/patch in the whole design which includes fixed cell objects, preRouted nets, and special nets on a preRoute db. You can use this command after floorplanning the design or after PG routing or on preRouted routing database, however, you must use it before running `routeDesign`.

Note: Cadence recommends you to use the `fix_drc_with_patch` command to check the sign-off on preRoute db. It should not be used for flow continuation.

Parameters

area	Enables the tool to trim/add patches and fix DRC violations in the specified route area. You can specify the area using the *x1 y1 x2 y2* coordinate values. *Data_type*: box, optional
-help	Outputs a brief description that includes the type and default information for each `fix_drc_with_patch` parameter. For a detailed description of the command and all of its parameters, use the man command: `man fix_drc_with_patch`
-layer_range *bottomLayer:topLayer*	Enables the tool to trim/add patches and fix DRC violations within the specified layer range (*bottomLayer:topLayer*). *Type:* string, optional

Examples

The following command adds patches and fixes DRC violations within the 2:4 layer range:

```
fix_drc_with_patch -layer_range 2:4
```

Related Topic

- ecoRoute
- `routeDesign`
- Using the NanoRoute Router chapter in the *Innovus User Guide*

getAttribute

```
getAttribute
[-help]
-net netName
[-quiet]
```

Displays the current net attribute settings in the Innovus console and in the Innovus log file. At the end of the display, the NanoRoute router lists each attribute, enclosing it in braces (`{}`) so you can use it in a Tcl script with variables. See `setAttribute` for descriptions of NanoRoute attributes.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `getAttribute` parameter. For a detailed description of the command and all of its parameters, use the man command `man getAttribute`.
`-net netName`	Specifies the net for which to display attributes. *Data_type*: string, required
`-quiet`	Displays the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software displays the current settings of all `setAttribute` parameters in Tcl list format. *Data_type*: bool, optional

Example

The following command displays the attributes for `net1`:

```
getAttribute -net net1
```

The Innovus software returns the following:

```
#NET net1 attributes:
# weight = 2 (default)
# non_default_rule = default (default)
# shield_net = none (default)
# pattern = steiner (default)
# top_preferred_routing_layer = 15 (default)
# bottom_preferred_routing_layer = 0 (default)
# preferred_extra_space = 0 (default)
# avoid_detour = false (default)
# skip_routing = false (default)
# skip_antenna_fix = false (default)
# si_post_route_fix = false (default)
{weight 2} {non_default_rule default} {shield_net none} {pattern steiner} {top_preferred_routing_layer 15} {bottom_preferred_routing_layer
0} {preferred_extra_space 0} {avoid_detour false} {skip_routing false} {skip_antenna_fix false} {si_post_route_fix false}
```

Related Information

- `setAttribute`
- Using the NanoRoute Router chapter in the *Innovus User Guide*

getNanoRouteMode

```
getNanoRouteMode
[-help]
[-extract_keep_fill_wires]
[-route_adjust_auto_via_weight]
[-route_allow_inst_overlaps]
[-route_ignore_follow_pin_shapes]
[-route_process_node]
[-route_rc_extraction_corner]
[-route_skip_analog]
[-route_via_weight]
[-route_detail_add_passive_fill_only_on_layers]
[-route_detail_allow_passive_fill_only_in_layers]
[-route_detail_antenna_eco_list_file]
[-route_detail_auto_stop]
[-route_detail_check_mar_on_cell_pin]
[-route_detail_end_iteration]
[-route_detail_fix_antenna]
[-route_detail_fix_antenna_on_secondary_pg_nets]
[-route_detail_fix_antenna_with_gate_array_filler_mode]
[-route_detail_merge_abutting_cut]
[-route_detail_min_length_for_spread_wire]
[-route_detail_min_length_for_widen_wire]
[-route_detail_min_slack_for_opt_wire]
[-route_detail_no_taper_in_layers]
[-route_detail_no_taper_on_output_pin]
[-route_detail_on_grid_only]
[-route_detail_post_route_litho_repair]
[-route_detail_post_route_spread_wire]
[-route_detail_post_route_swap_via]
[-route_detail_postroute_via_priority]
[-route_detail_post_route_via_pillar_effort]
[-route_detail_post_route_wire_widen]
[-route_detail_post_route_wire_widen_rule]
[-route_detail_search_and_repair]
[-route_detail_signoff_effort]
[-route_detail_stub_routing_in_first_layer]
[-route_detail_use_multi_cut_via_effort]
[-route_number_fail_limit]
[-route_number_thread]
[-route_number_warning_limit]
[-route_third_party_data]
[-route_high_freq_constraint_groups]
[-route_high_freq_match_report_file]
[-route_high_freq_num_reserved_layers]
[-route_high_freq_remove_floating_shield]
[-route_high_freq_search_repair]
[-route_high_freq_shield_trim_length]
[-route_interposer_allow_diagonal_trunk]
[-route_interposer_interlayer_shielding_layers]
[-route_interposer_interlayer_shielding_nets]
[-route_interposer_interlayer_shielding_offsets]
[-route_interposer_interlayer_shielding_widths]
[-route_interposer_same_layer_shielding_net]
[-route_interposer_same_layer_shielding_width_spacing]
[-route_interposer_trunk_routing_layers]
[-route_interposer_trunk_routing_width_spacing]
[-route_add_antenna_inst_prefix]
[-route_allow_pin_as_feedthru]
[-route_antenna_cell_name]
[-route_concurrent_minimize_via_count_effort]
[-route_connect_to_bumps]
[-route_fix_clock_nets]
[-route_route_clock_nets_first]
[-route_eco_ignore_existing_route]
[-route_enable_route_rule_si_limit_length]
[-route_enforce_route_rule_on_special_net_wire]
```

```
[-route_extra_via_enclosure]
[-route_honor_exclusive_region]
[-route_honor_power_domain]
[-route_ignore_antenna_top_cell_pin]
[-route_antenna_diode_insertion]
[-route_diode_insertion_for_clock_nets]
[-route_shield_tap_cell_insertion]
[-route_relaxed_route_rule_spacing_to_power_ground_nets]
[-route_reserve_space_for_multi_cut]
[-route_reverse_direction]
[-route_selected_net_only]
[-route_shield_crosstie_offset]
[-route_shield_length_threshold]
[-route_shield_report_skip_status]
[-route_shield_tap_cell_name]
[-route_strictly_honor_1d_routing]
[-route_strict_honor_route_rule]
[-route_stripe_layer_range]
[-route_tieoff_to_shapes]
[-route_trim_pull_back_distance_from_boundary]
[-route_trunk_with_cluster_target_size]
[-route_use_auto_via]
[-route_with_eco]
[-route_with_litho_driven]
[-route_with_si_driven]
[-route_with_timing_driven]
[-route_with_trim_metal]
[-route_with_via_in_pin]
[-route_with_via_only_for_block_cell_pin]
[-route_with_via_only_for_stdcell_pin]
[-quiet]
[-nonDefault]
```

Returns information about the specified `setNanoRouteMode` parameter in the Innovus log file and the console. The following information is returned:

- Option name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the software displays information for all of the `setNanoRouteMode` mode parameters.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `getNanoRouteMode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man getNanoRouteMode`.
`parameter_names`	Displays information for the specified parameters. You can specify one or more parameters. See `setNanoRouteMode` for descriptions of the parameters you can specify.
`-nonDefault`	Displays the current settings which are not default settings.
`-quiet`	Displays the current settings for the specified parameters in TCL list format only. If you specify `-quiet` without any parameters, the software displays the current settings of all `setNanoRouteMode` parameters in TCL list format.

Examples

- The following command displays the current setting of the `-route_detail_fix_antenna` parameter:

  ```
  getNanoRouteMode -route_detail_fix_antenna
  ```

  The software displays the following information:

  ```
  -route_detail_fix_antenna true # bool, default=true
  true
  ```

- The following command displays the current setting for the `-route_detail_fix_antenna` parameter in Tcl list format only:

```
getNanoRouteMode -quiet -route_detail_fix_antenna
```

The software displays the following information:

```
true
```

- The following command displays the current settings for all `setNanoRouteMode` parameters:

  ```
  getNanoRouteMode
  ```

- The following command displays the current settings for all `setNanoRouteMode` parameters in Tcl list format only:

  ```
  getNanoRouteMode -quiet
  ```

Related Information

- `routeDesign`
- `setNanoRouteMode`
- Using the NanoRoute Router chapter in the *Innovus User Guide*

getRouteMode

```
getRouteMode
[-help]
[-earlyGlobalEffortLevel]
[-earlyGlobalHonorMsvRouteConstraint]
[-earlyGlobalNumTracksPerClockWire]
[-earlyGlobalReverseDirection]
[-earlyGlobalRouteBumpNets]
[-earlyGlobalRoutePartitionAllowFeedthru]
[-earlyGlobalRoutePartitionHonorFence]
[-earlyGlobalRoutePartitionHonorPin]
[-earlyGlobalRoutePartitionPinGuide]
[-earlyGlobalRouteSecondPG]
[-earlyGlobalRouteSelectedNetsOnly]
[-earlyGlobalRouteStripeLayerRange]
[-earlyGlobalSecondPGMaxFanout]
[-quiet]
[-nonDefault]
```

Returns information about the specified Early Global Route mode parameter in the Innovus log file and the console. The following information is returned:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the software displays information for all of the Early Global Route mode parameters.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `getRouteMode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man getRouteMode`
`-nonDefault`	Displays the current settings that are not default settings.
`parameter_names`	Displays information for the specified parameters. You can specify one or more parameters. See `setRouteMode` for descriptions of the Early Global Route parameters you can specify.
`-quiet`	Displays the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software displays the current settings of all `setRouteMode` parameters in Tcl list format.

Examples

- The following command displays the current setting for the `-earlyGlobalNumTracksPerClockWire` parameter:

  ```
  getRouteMode -earlyGlobalNumTracksPerClockWire
  ```

  The software displays the following information:

  ```
  -earlyGlobalNumTracksPerClockWire 0 # int, default=0, min=0, max=8
  0
  ```

- The following command displays the current setting for the `-earlyGlobalRoutePartitionPinGuide` parameter:

  ```
  getRouteMode -earlyGlobalRoutePartitionPinGuide
  ```

  The software displays the following information:

  ```
  -earlyGlobalHoutePartitionPinGuide true # bool, default=true
   {earlyGlobalRoutePartitionPinGuide true}
  ```

Related Information

- `setRouteMode`
- `earlyGlobalRoute`
- Using Early Global Route for Congestion and Timing Analysis chapter in the *Innovus User Guide*

globalDetailRoute

```
globalDetailRoute
[-help]
[-select]
[area]
```

Uses the NanoRoute router to perform both global and detailed routing with one command. When you load a partially routed database, the router completes the routes.

- To perform global routing only, use `globalRoute`.
- To perform detailed routing only, use `detailRoute`.

This command supports multiple-CPU processing.

Use this command after completing the following steps:

1. Placing the design.
2. Routing special nets.
3. Setting net attributes with `setAttribute` command, and NanoRoute run-time options with `setNanoRouteMode` command.

Note: To use NanoRoute current values for attributes, and default values for options, this step is optional.

Parameters

area	Specifies the coordinates (*x1 y1 x2 y2*) of the bounding box area for detailed routing. The entire design is globally routed. *Data_type*: rect, optional
-help	Outputs a brief description that includes the type and default information for each `globalDetailRoute` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man globalDetailRoute`
-select	Routes selected nets. To select a net, use the `selectNet` command or click the net in the design display window. **Tip**: You can also specify `setNanoRouteMode -route_selected_net_only true` to route the selected nets. *Data_type*: bool, optional

Note: If you have not already run detailed routing on the entire design, you cannot run detailed routing on a specified area.

Example

The following commands run global and detailed routing, save the design as `droute`, and generate a routed DEF file named `droute.def`.

```
globalDetailRoute
saveDesign droute
defOut -floorplan -placement -routing droute.def
```

Related Information

- `detailRoute`
- `globalRoute`
- `routeDesign`
- `setAttribute`
- `setNanoRouteMode`
- Using the NanoRoute Router chapter in the *Innovus User Guide*
  - Routing Phases

globalRoute

```
globalRoute
[-help]
```

Plans the interconnect by breaking the routing portion of the design into rectangles called global routing cells (gcells) and assigning the signal nets to the gcells.

Note: If you are running ECO routing, use `globalDetailRoute`. Your results are not predictable with other routing commands.

Use this command after completing the following steps:

1. Placing the design.
2. Setting net attributes with `setAttribute`, and NanoRoute run-time options with `setNanoRouteMode`. To use current values for attributes and default values for options, this step is optional.

Parameters

`-help`	Outputs a brief description of the `globalRoute` command. For a detailed description of the command, use the man command: `man globalRoute`

Example

The following command globally routes the design and saves the design as `droute`.

```
globalRoute
saveDesign droute
```

Related Information

- `detailRoute`
- `globalDetailRoute`
- `routeDesign`
- `setAttribute`
- `setNanoRouteMode`
- Using the NanoRoute Router chapter in the *Innovus User Guide*
  - Global Routing

report_route

```
report_route
[-help]
[-clock]
[-em]
[-fanout integer]
[-fill]
[-inst string]
[-multi_cut]
[-ndr]
[-net net_name]
[-net_length length]
[-patch]
[-pg_net string]
[-secondary_pg_nets {cellName1:pinName1}]
[-selected]
[-shield]
[-summary]
[-via_pillar]
[-track_utilization [-area {{lx1 ly1 ux1 uy1} {lx2 ly2 ux2 uy2}...} | -view_window ]
  [-layer string] [-include_regular_routes ]]
```

Report all the routing related information of the design. The `report_route` command is used for reporting:

- Congestion
- Single-cut or multi-cut table for each layer.
- Via Usage summary
- DFM via reporting
- Shield coverage
- EM coverage
- Wire length
- Wire density
- DRC violations
- Statistics for selected nets
- Track utilization

Parameter

`-area {{lx1 ly1 ux1 uy1} {lx2 ly2 ux2 uy2}...}`	
	Specifies the coordinates of one or more area regions for reporting track utilization. For example, `report_route -track_utilization -area {299.9015 301.332 371.8345 372.4}` *Data_type*: box_list, optional
`-clock`	Reports statistics of clock nets. *Data_type*: bool, optional
`-em`	Reports EM statistics of nets. *Data_type*: bool, optional
`-fanout integer`	Specifies the minimum fanout for limit. *Data_type*: int, optional
`-fill`	Specifies the patch wire and active fill information. *Data_type*: bool, optional

`-help`	Outputs a brief description that includes type and default information for each `report_route` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_route`
`-include_regular_routes`	Reports track utilization for tracks blocked by regular wiring. **Note:** By default, the `report_route` command does not include regular wiring when computing track utilization. *Data_type*: bool, optional
`-inst` *string*	Specifies the instance name. *Data_type*: string, optional
`-layer` *layer* | *layer:layer*	Specifies the layer or layer range for which track utilization report is generated. For example: `report_route -track_utilization -layer 1:4` *Data_type*: string, optional
`-multi_cut`	Prints single-cut and multi-cut table for each layers. If you specify the DFM via list with `setNanoRouteMode -dbViaWeight` then this *Data_type*: bool, optional
`-ndr`	Reports NDR statistics of nets. *Data_type*: bool, optional
`-net` *net_name*	Specifies the name of the net for which the report is generated. *Data_type*: string, optional
`-net_length length`	Specifies the minimum wire length of the net for reporting the net statistics. All nets whose length is more than the specified value *Data_type*: float, optional
`-patch`	Adds the patch length information to the report. *Data_type*: bool, optional
`-pg_net` *string*	Specifies the PG net name. *Data_type*: string, optional
`-secondary_pg_nets {`*cellName1*`:`*pinName1*`}`	
	Specifies the secondary pg pin information. It reports the routing length of the secondary pg pin from the grid tap point to pin. You report all the secondary pg pins and sort the results in descending order. Use model: `report_route -secondary_pg_nets {ENDCAP:BIASNW}` *Data_type*: string, optional
`-selected`	Reports statistics for selected nets only. *Data_type*: bool, optional
`-shield`	Reports shield coverage for nets routed by the NanoRoute router. *Data_type*: bool, optional
`-summary`	Specifies that just the summary table should be displayed in the report. The summary report contains: • Number of objects, such as cells, nets, and pins • Number and percentage of nets with one, two, three, four, or more terminals. • Names and numbers of primitives used. • Net length and connection length statistics • Total wire length and number of vias, per layer. *Data_type*: bool, optional
`-track_utilization`	Reports the track utilization. *Data_type*: bool, optional

`-via_pillar`	Reports stacking via statistics. *Data_type*: bool, optional
`-view_window`	Report area in ui_view_box. *Data_type*: bool, optional

Example Report

- The following command reports the track utilization in the specified area for layers Metal1-Metal4 and include regular routes.

```
report_route -track_utilization -area {299.9015 301.332 371.8345 372.4} -layer 1:4 -include_regular_routes

#Calculating Track Utilization in region ( 299.90150 301.33200 371.83450 372.40000 ) for layer range: 1-4 [Metal1-Metal4] ...
#
#+--------+------------------+--------------+-----------+-------------+-------------+-----------+--------+----------------+
#| | BLOCKED BY | |
#+--------+------------------+--------------+-----------+-------------+-------------+-----------+--------+----------------+
#| Layer | # of Tracks | TrackLength | PG Mesh % | Blkg & IO % | BlockCell % | StdCell % | Nets % | Avail Tracks % |
#+--------+------------------+--------------+-----------+-------------+-------------+-----------+--------+----------------+
#| Metal1 | 127 | 9135.55 | 0.00 | 0.00 | 0.00 | 15.91 | 0.00 | 84.09 |
#| Metal2 | 109 | 7746.47 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 100.00 |
#| Metal3 | 127 | 9135.55 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 100.00 |
#| Metal4 | 109 | 7746.47 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 100.00 |
#+--------+------------------+--------------+-----------+-------------+-------------+-----------+--------+----------------+
#| TOTAL | 472 | 33764.04 | 0.00 | 0.00 | 0.00 | 4.30 | 0.00 | 95.70 |
#+--------+------------------+--------------+-----------+-------------+-------------+-----------+--------+----------------+
#Calculating Track Utilization: cpu:00:00:00, real:00:00:00, mem:771.6 MB, peak:1016.7 MB
```

- The following command will report for a summary.

```
report_route -summary

############################################################
Statistics for design dtmf_chip :

+-------------------+----------+
| Design Metric | Count |
+-------------------+----------+
| Instances | 7923 |
| Nets | 7483 |
| Tri-State Nets | 0 |
| Degen nets | 1 |
| Pins | 25442 |
| I/O Terminals | 57 |
+-------------------+----------+


Net Pin Statistics :

+------------------+----------------+---------------+
| Number of | Number of | Percentage |
| Connected Pins | Nets | of Nets |
+------------------+----------------+---------------+
| 1 | 1 | 0.013 |
| 2 | 3862 | 51.610 |
| 3 | 1894 | 25.311 |
| 4 | 716 | 9.568 |
| 5 | 267 | 1.924 |
| 6 | 144 | 1.924 |
| 7 | 102 | 1.363 |
| 8 | 64 | 0.855 |
| 9 | 44 | 0.588 |
| >=10 | 389 | 5.198 |
+------------------+----------------+---------------+
```

```
Wire Length Statistics :

+----------------+-----------------+
| Layer Name | Wire Length |
+----------------+-----------------+
| Metal1 | 48450.08 um |
| Metal2 | 155839.58 um |
| Metal3 | 162909.59 um |
| Metal4 | 103520.34 um |
| Metal5 | 68618.63 um |
| Metal6 | 11536.34 um |
+----------------+-----------------+
| Total | 550874.560 um |
+----------------+-----------------+


Via Count Statistics :

+----------------+---------+
| Via-cut Name | Count |
+----------------+---------+
| - | 0 |
| Metal1(Via12) | 25657 |
| Metal2(Via23) | 17674 |
| Metal3(Via34) | 3852 |
| Metal4(Via45) | 1460 |
| Metal5(Via56) | 408 |
+----------------+---------+
| Total | 49051 |
+----------------+---------+
*** Net length and connection length statistics (cell dtmf_chip) ***

Total net length = 483697.285 (248977.085 234720.200)

Avg net length = 64.639 (sigma = 105.239)
Sqrt of avg square net length = 123.506

Avg connection length = 26.932 (sigma = 47.753)
Sqrt of avg square connection length = 54.824

Net and connection length distribution:

+------------------------+------------------+--------------------+
| | Number of Nets | Number of Nets |
| Length Range | with | with |
| | Wire Length | Fly-line Length |
+------------------------+------------------+--------------------+
| 0.00 um - 13.20 um | 1633 | 2847 |
| 13.20 um - 26.40 um | 1924 | 2391 |
| 26.40 um - 39.60 um | 1082 | 974 |
| 39.60 um - 52.80 um | 594 | 444 |
| 52.80 um - 66.00 um | 391 | 223 |
| 66.00 um - 79.20 um | 295 | 144 |
| 79.20 um - 92.40 um | 251 | 98 |
| 92.40 um - 105.60 um | 192 | 62 |
| 105.60 um - 118.80 um | 150 | 46 |
| 118.80 um - 132.00 um | 100 | 24 |
| 132.00 um - 145.20 um | 91 | 19 |
| 145.20 um - 158.40 um | 68 | 12 |
| 158.40 um - 171.60 um | 68 | 15 |
| 171.60 um - 184.80 um | 46 | 16 |
| 184.80 um - 198.00 um | 38 | 8 |
| 198.00 um - 211.20 um | 38 | 14 |
| 211.20 um - 224.40 um | 32 | 7 |
| 224.40 um - 237.60 um | 30 | 3 |
| 237.60 um - 250.80 um | 37 | 9 |
| 250.80 um - 264.00 um | 27 | 6 |
| 264.00 um - 277.20 um | 31 | 10 |
| 277.20 um - 290.40 um | 17 | 6 |
| 290.40 um - 303.60 um | 27 | 5 |
| 303.60 um - 316.80 um | 20 | 4 |
```

```
| 316.80 um - 330.00 um | 27 | 5 |
| 330.00 um - 343.20 um | 24 | 5 |
| 343.20 um - 356.40 um | 19 | 2 |
| 356.40 um - 369.60 um | 27 | 6 |
| 369.60 um - 382.80 um | 25 | 1 |
| 382.80 um - 396.00 um | 21 | 4 |
| 396.00 um - 409.20 um | 22 | 6 |
| 409.20 um - 422.40 um | 12 | 3 |
| 422.40 um - 435.60 um | 8 | 7 |
| 435.60 um - 448.80 um | 9 | 6 |
| 448.80 um - 462.00 um | 8 | 3 |
| 462.00 um - 475.20 um | 7 | 3 |
| 475.20 um - 488.40 um | 2 | 2 |
| 488.40 um - 501.60 um | 5 | 1 |
| 501.60 um - 514.80 um | 3 | 2 |
| 514.80 um - 528.00 um | 7 | 2 |
| 528.00 um - 541.20 um | 5 | 2 |
| 541.20 um - 554.40 um | 4 | 2 |
| 554.40 um - 567.60 um | 4 | 2 |
| 567.60 um - 580.80 um | 5 | 3 |
| 580.80 um - 594.00 um | 4 | 2 |
| 594.00 um - 607.20 um | 3 | 2 |
| 607.20 um - 620.40 um | 4 | 1 |
| 620.40 um - 633.60 um | 2 | 0 |
| 633.60 um - 646.80 um | 1 | 1 |
| 660.00 um - 673.20 um | 3 | 2 |
| 673.20 um - 686.40 um | 2 | 1 |
| 686.40 um - 699.60 um | 2 | 1 |
| 699.60 um - 712.80 um | 3 | 2 |
| 712.80 um - 726.00 um | 4 | 3 |
| 726.00 um - 739.20 um | 2 | 1 |
| 739.20 um - 752.40 um | 1 | 1 |
| 752.40 um - 765.60 um | 1 | 1 |
| 765.60 um - 778.80 um | 3 | 1 |
| 778.80 um - 792.00 um | 1 | 1 |
| 792.00 um - 805.20 um | 1 | 0 |
| 805.20 um - 818.40 um | 2 | 0 |
| 831.60 um - 844.80 um | 2 | 1 |
| 844.80 um - 858.00 um | 1 | 0 |
| 884.40 um - 897.60 um | 1 | 1 |
| 910.80 um - 924.00 um | 2 | 2 |
| 937.20 um - 950.40 um | 1 | 1 |
| 990.00 um - 1003.20 um | 2 | 0 |
| 1003.20 um - 1016.40 um | 1 | 0 |
| 1016.40 um - 1029.60 um | 3 | 1 |
| 1029.60 um - 1042.80 um | 1 | 1 |
| 1042.80 um - 1056.00 um | 1 | 1 |
| 1056.00 um - 1069.20 um | 1 | 0 |
| 1069.20 um - 1082.40 um | 1 | 1 |
| 1254.00 um - 1267.20 um | 1 | 0 |
+-------------------------+------------------+--------------------+


Total number of long connections = 0
Io: nrCon=0
```

- The following command will print single-cut and multi-cut table for each layer and provide dfm via ratio.

```
report_route -multi_cut
```

```
+--------------------+--------------------+---------------+----------------+-----------+
| Via-Cut Layer Name |    single-cut      |   multi-cut   |      DFM       |  Total    |
+--------------------+--------------------+---------------+----------------+-----------+
|       VIA12        |                  0 |             0 |              0 |         0 |
|       VIA23        |    27975 (100.0%)  |     0 (0.0%)  | 18499 (66.1%)  |    27975  |
|       VIA34        |     27407 (73.7%)  | 9788 (26.3%)  | 29894 (80.4%)  |    37195  |
|       VIA45        |     23998 (62.1%)  |14625 (37.9%)  | 30590 (79.2%)  |    38623  |
|       VIA56        |     18885 (51.6%)  |17717 (48.4%)  | 20227 (55.3%)  |    36602  |
|       VIA67        |     37190 (69.0%)  |16738 (31.0%)  | 17670 (32.8%)  |    53928  |
|       VIA78        |     34278 (70.5%)  |14323 (29.5%)  | 15770 (32.4%)  |    48601  |
|       VIA89        |     19440 (56.8%)  |14806 (43.2%)  | 15319 (44.7%)  |    34246  |
```

```
|       VIA910       |     27544 (90.6%)  |  2859 (9.4%)  |  3838 (12.6%)  |    30403  |
|      VIA1011       |      2207 (46.1%)  | 2585 (53.9%)  |  2848 (59.4%)  |     4792  |
|      VIA1112       |      2299 (52.6%)  | 2071 (47.4%)  |  2089 (47.8%)  |     4370  |
|      VIA1213       |       442 (30.9%)  |  990 (69.1%)  |   990 (69.1%)  |     1432  |
|      VIA1314       |       193 (20.4%)  |  751 (79.6%)  |   751 (79.6%)  |      944  |
+--------------------+--------------------+---------------+----------------+-----------+
|       Total        |    221858 (69.5%)  |97253 (30.5%)  |158485 (49.7%)  |   319111  |
```

Related Information

- Using the NanoRoute Router chapter in the *Innovus User Guide*

report_tracks

```
report_tracks
[-help]
[-prefer_only]
```

Reports track definitions for each layer. The output is in the DEF TRACKS format. This command can be used after the floorplan is created or after using the `add_tracks` command to see exactly which tracks were created.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `report_tracks` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_tracks`.
`-prefer_only`	Reports track definitions only for the preferred direction of each layer. *Data_type*: bool, optional

Example

- The following command reports the track definitions for each layer:
  report_tracks

```
TRACKS Y 560 DO 2657 STEP 1120 LAYER Metal6 ;
TRACKS X 660 DO 1503 STEP 1980 LAYER Metal6 ;
TRACKS X 660 DO 2255 STEP 1320 LAYER Metal5 ;
TRACKS Y 560 DO 2657 STEP 1120 LAYER Metal5 ;
TRACKS Y 560 DO 2657 STEP 1120 LAYER Metal4 ;
TRACKS X 660 DO 2255 STEP 1320 LAYER Metal4 ;
TRACKS X 660 DO 2255 STEP 1320 LAYER Metal3 ;
TRACKS Y 560 DO 2657 STEP 1120 LAYER Metal3 ;
TRACKS Y 560 DO 2657 STEP 1120 LAYER Metal2 ;
TRACKS X 660 DO 2255 STEP 1320 LAYER Metal2 ;
TRACKS X 660 DO 2255 STEP 1320 LAYER Metal1 ;
TRACKS Y 560 DO 2657 STEP 1120 LAYER Metal1 ;
```

- The following command reports the track definitions only for the preferred direction of each layer:
  `report_tracks -prefer_only`

```
TRACKS X 660 DO 1503 STEP 1980 LAYER Metal6 ;
TRACKS Y 560 DO 2657 STEP 1120 LAYER Metal5 ;
TRACKS X 660 DO 2255 STEP 1320 LAYER Metal4 ;
TRACKS Y 560 DO 2657 STEP 1120 LAYER Metal3 ;
TRACKS X 660 DO 2255 STEP 1320 LAYER Metal2 ;
TRACKS Y 560 DO 2657 STEP 1120 LAYER Metal1 ;
```

Related Information

- `add_tracks`
- `check_tracks`
- Using the NanoRoute Router chapter in the *Innovus User Guide*
  - Adding Tracks
  - Running the NanoRoute Router with Innovus Text Commands

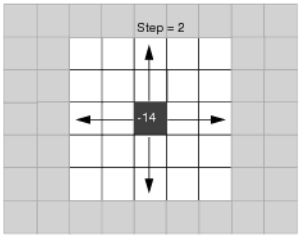
reportCongestArea

```
reportCongestArea
[-help]
[-num [all | numOfHotSpot ]]
[-cutOffRatio cutOffRatioOfPeakHotSpot ]
[-cutOffValue cutOffCongestNum ]
[-mode modeName]
[-step num ]
[-cutoffArea cutOffGcellNum ]
[-outfile fileName ]
```

Reports the congestion hot spots in the design.

The Innovus software uses the following algorithm to define the boundaries of hot spots:

1. Finds the most over-congested gcell (that is, the gcell with the largest overflow value) in the design. A gcell has overflow if its demand exceeds its supply. Typically, the supply is the number of unobstructed tracks crossing the gcell, and the demand is the number of wires assigned to it.
2. Starting from the over-congested gcell, checks a specified number of gcells (using -step) in one direction for congestion. If the gcells have an overflow value that is less than the specified congestion threshold (using `-cutOffRatio` or `-cutOffValue`), the software determines this to be the boundary for that side of the hot spot, and begins checking in a new direction.



3. After the software establishes the boundaries for the first hot spot, it finds the next most over-congested gcell that is not part of the already defined hot spot and begins defining the boundaries for it. You can use the `reportCongestArea` command after running Early Global Route.

Parameters

`-cutoffArea` *`cutOffGcellNum`*	Filters out any hot spots that have an area (specified in gcells) that is less than *`cutOffGcellNum`*. *Default:* `0` (Reports all hot spots in the design) *Data_type*: int, optional
`-cutOffRatio` *`cutOffRatioOfPeakHotSpot`*	Specifies the minimum congestion threshold value a gcell must have to be considered part of the hot spot, as a ratio of the peak overflow value. The software checks the congestion for the gcell by dividing the overflow value of the most over-congested gcell (that is, the peak overflow value) by the overflow value of the gcell. If the resulting congestion value is less than the specified *`cutOffRatioOfPeakHotSpot`*, the software determines this to be the boundary for that side of the hot spot, and begins checking in a new direction. *Min*: `0` *Max*: `1` *Default:* `0.01` *Data_type*: float, optional
`-cutOffValue` *`cutOffCongestNum`*	Specifies the minimum overflow value a gcell must have to be considered part of the hot spot. *Default:* `-2` *Data_type*: int, optional
`-mode` *`modeName`*	Specifies the mode, average or max, on which congestion hot spots in the design are reported. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `reportCongestArea` parameter. For a detailed description of the command and all of its parameters, use the man command `man reportCongestArea`.
`-num [all | numOfHotSpot]`	
	Specifies how many hot spots to report. *Default:* `10` • all: Reports all hot spots in the design. • *`numOfHotSpot`*: Reports up to the specified number of hot spots in the design. *Data_type*: string, optional
`-outfile` *`fileName`*	Outputs the hot spot report to the specified file. *Data_type*: string, optional
`-step` *`num`*	Specifies the number of gcells to check for congestion. *Default:* `3` *Data_type*: string, optional

Examples

- The following command outputs a report describing up to10 hot spots in the design that are found by checking three gcells at a time, using an absolute congestion threshold of `-2` and a relative threshold value of `0.01`:

  ```
  reportCongestArea -step 3 -cutoffRatio 0.010000 -cutoffValue -2 -num 10 -mode average
  ```

  The resulting report contains the following hot spot information for vertical and horizontal congestion:

  - The largest overflow value for the hot spot (`#Peak`)
  - The location of the gcell that contains the largest overflow value (`Location`)
  - The boundary coordinates for the hot spot (`Boundary Box`)
  - The area of the hot spot, in gcells (`Area`)
  - The following example shows the congestion hot spot report generated by the previous command:

    ```
    # Vertical Congestion Area Report
    #Peak   Location(X,Y)      Boundary Box                Area(#Gcell)
    -36      (1791.7,  376.4)    (1778,  332) (1792,  786)    (6x123)
    -23      ( 607.2,  284.1)    ( 580,  251) ( 607,  306)    (12x15)
    -20      (1791.7, 2236.1)    (1778, 2214) (1792, 2247)    (6x9)
    -18      ( 607.2,  125.5)    ( 593,   92) ( 607,  148)    (6x15)
    -15      ( 637.1,  380.1)    ( 630,  347) ( 720,  435)    (39x24)
    -14      (1791.7,  859.8)    (1778,  860) (1792,  982)    (6x33)

    # Horizontal Congestion Area Report
    #Peak   Location(X,Y)      Boundary Box                Area(#Gcell)
    -92      (1777.9, 114.4)    (1750, 4) (1854, 114)         (45x30)
    -60      ( 623.3, 114.4)    ( 472, 4) ( 816, 125)         (150x33)
    ```

- The following line from the example shows that the largest overflow value for one hot spot is `-36`, the gcell that contains the overflow value is located at `1791.7 376.4`, the coordinates for the hot spot boundary are (`1778, 332`) (`1792, 786`), and the hot spot is 6 gcells by 123 gcells in size:

  ```
  -36      (1791.7,  376.4)    (1778,  332) (1792,  786)    (6x123)
  ```

Related Information

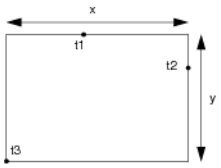
- `dumpCongestArea`
- Using the NanoRoute Router chapter in the *Innovus User Guide*

reportRoute

```
reportRoute
[-help]
[-clock]
[-em]
[-ndr]
[-selected_net_only]
[-via_pillar]
```

Reports routing statistics for signal nets, including the following information:

- Number of objects, such as cells, nets, and pins
- Number and percentage of nets with one, two, three, four, or more terminals
- Names and numbers of primitives used
- EM statistics of nets.
- NDR statistics of nets.
- Net length and connection length statistics
  These values are computed using the net bounding box half-perimeter.
  The following figure shows a net bounding box for a net that must be connected to three terminals, t1, t2, and t3. The net bounding box is the perimeter of the smallest rectangle that can be formed that has sides passing through the three terminals. The half-perimeter is the length of `x` plus the length of y.



- Total wire length and number of vias, per layer
  These are the actual wire length and number of vias on each layer. The actual wire length might be different from the net bounding box half-perimeter if the net detours.
  The layer statistics are in the last paragraph of the report.

Note: Use the `reportWire` command to generate a more detailed wire statistics report. Use the `reportSpecialRoute` command to generate a report for special nets.

You can use the `reportRoute` command after routing with any of the routing commands in the Innovus System software.

Parameter

`-clock`	Reports statistics of clock nets. *Data_type*: bool, optional
`-em`	Reports EM statistics of nets. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes type and default information for each `reportRoute` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reportRoute`
`-ndr`	Reports NDR statistics of nets. *Data_type*: bool, optional
`-selected_net_only`	Reports statistics for selected nets only. *Data_type*: bool, optional
`-via_pillar`	Reports stacking via statistics. *Data_type*: bool, optional

Example Report

In the example report the terms `long connection`, `nrCon`, `and total length` are used.

- `long net` is a net that is longer than the last net listed in the report. In the report that follows, it is any net longer than `1.82e+06`.
- `nrCon` is the total number of I/O connections in the nets with long connections.
- `total length` is the total wire length.

```
*** Statistics for net list dtmf_chip ***
Number of cells = 7923
Number of nets = 7483
Number of tri-nets = 0
Number of degen nets = 1
Number of pins = 25442
Number of i/os = 57

Number of nets with 1 terms = 1 (0.0%)
Number of nets with 2 terms = 3862 (51.6%)
Number of nets with 3 terms = 1894 (25.3%)
Number of nets with 4 terms = 716 (9.6%)
Number of nets with 5 terms = 267 (3.6%)
Number of nets with 6 terms = 144 (1.9%)
Number of nets with 7 terms = 102 (1.4%)
Number of nets with 8 terms = 64 (0.9%)
Number of nets with 9 terms = 44 (0.6%)
Number of nets with >=10 terms = 389 (5.2%)

*** 99 Primitives used:
Primitive ram_128x16A (1 insts)
Primitive pllclk (1 insts)
Primitive rom_512x16A (1 insts)
Primitive ram_256x16A (1 insts)
Primitive PVSS1DGZ (5 insts)
Primitive PVDD1DGZ (5 insts)
Primitive PFEED5 (54 insts)
...
...
..
************
*** Net length and connection length statistics (cell dtmf_chip) ***

Total net length = 4.713e+05 (2.410e+05 2.303e+05)

Avg net length = 6.299e+01 (sigma = 1.035e+02)
Sqrt of avg square net length = 1.211e+02

Avg connection length = 2.624e+01 (sigma = 4.632e+01)
Sqrt of avg square connection length = 5.323e+01

Net and connection length distribution:

[length range ] #net #connection
[0.00e+00 1.32e+01]: 1811 3050
[1.32e+01 2.64e+01]: 1860 2276
[2.64e+01 3.96e+01]: 1033 925
[3.96e+01 5.28e+01]: 584 433
[5.28e+01 6.60e+01]: 367 218
[6.60e+01 7.92e+01]: 301 143
...
...

Total number of long connections = 0
Io: nrCon=0

Total length: 1.841e+03um, number of vias: 16
Metal1(H) length: 0.000e+00um, number of vias: 2
Metal2(V) length: 1.198e+01um, number of vias: 3
Metal3(H) length: 1.016e+02um, number of vias: 3
Metal4(V) length: 2.274e+02um, number of vias: 3
Metal5(H) length: 7.927e+02um, number of vias: 5
Metal6(V) length: 7.075e+02um
```

Related Information

- routeDesign
- Using the NanoRoute Router chapter in the *Innovus User Guide*

reportShield

```
reportShield
[-help]
[-crosstie]
[-include_layer]
[-out_file fileName]
[-selected]
[-verbose]
```

Reports shield coverage for nets routed by the NanoRoute router.

Parameters

`-crosstie`	Reports the shield coverage of all shielded nets with crosstie attributes. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes type and default information for each `reportShield` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man reportShield`.
`-include_layer`	Reports the shielding ratio per layer per net in addition to the ratio for the whole net. *Data_type*: bool, optional
`-out_file fileName`	Specifies the report file name. *Default:* `designName_shield.rpt` *Data_type*: string, optional
`-selected`	Reports statistics for selected nets only. *Data_type*: bool, optional
`-verbose`	Includes a list of nets that are skipped, which includes the following nets: • `FIXED` nets • `SPECIAL` nets • Nets with the `skip_routing` attribute • Nets with opens *Data_type*: bool, optional

Examples

The following command reports for selected nets only but breaks them down by segment. The summary report will be printed in log, and detailed report will be in *myDesign_shield.rpt* file.

```
reportShield -crosstie -selected -verbose -out_file myDesign_shield.rpt
In myDesign_shield.rpt file:
NET in:
+-----------------------------+------------+------------+----------+------------+-----------+--------+--------------+
| Segment                     | Layer name | Expected # | Actual # | Percentage | Pre-route | Shield | Power:Ground |
+-----------------------------+------------+------------+----------+------------+-----------+--------+--------------+
| 0.1600 4.8960 2.9120 4.8960 | metal1     | 42         | 0        | 0%         | 0         | 0      | 0:0          |
| 0.1600 4.8960 2.9120 4.8960 | metal3     | 28         | 11       | 39%        | 0         | 11     | 0:11         |
+-----------------------------+------------+------------+----------+------------+-----------+--------+--------------+
| Total                       |            | 70         | 11       | 15%        | 0         | 11     | 0:11         |
+-----------------------------+------------+------------+----------+------------+-----------+--------+--------------+

In the log file:
#Crosstie Table:
#+------------+------------+----------+------------+-----------+--------+--------------+
#| Layer name | Expected # | Actual # | Percentage | Pre-route | Shield | Power:Ground |
#+------------+------------+----------+------------+-----------+--------+--------------+
#| metal1     | 42         | 0        | 0%         | 0         | 0      | 0:0          |
#| metal3     | 28         | 11       | 39%        | 0         | 11     | 0:11         |
#+------------+------------+----------+------------+-----------+--------+--------------+
#| Total      | 70         | 11       | 15%        | 0         | 11     | 0:11         |
```

```
#+------------+------------+----------+------------+-----------+--------+--------------+
```

The following command prints a detailed report in *myDesign_shield_2.rpt* file without breaking down by segment. This command is called without the `-verbose` option.

```
reportShield -crosstie -selected -out_file myDesign_shield_2.rpt

In file myDesign_shield_2.rpt:
NET in:
+-------------+------------+------------+-------------+------------+------------+-----------------------+
| Layer name  | Expected # | Actual #   | Percentage  | Pre-route  | Shield     | Power:Ground          |
+-------------+------------+------------+-------------+------------+------------+-----------------------+
| metal1      | 42         | 0          | 0%          | 0          | 0          | 0:0                   |
| metal3      | 28         | 11         | 39%         | 0          | 11         | 0:11                  |
+-------------+------------+------------+-------------+------------+------------+-----------------------+
| Total       | 70         | 11         | 15%         | 0          | 11         | 0:11                  |
+-------------+------------+------------+-------------+------------+------------+-----------------------+

In Log file:
#Crosstie Table:
#+------------+------------+----------+------------+-----------+--------+--------------+
#| Layer name | Expected # | Actual # | Percentage | Pre-route | Shield | Power:Ground |
#+------------+------------+----------+------------+-----------+--------+--------------+
#| metal1     | 42         | 0        | 0%         | 0         | 0      | 0:0          |
#| metal3     | 28         | 11       | 39%        | 0         | 11     | 0:11         |
#+------------+------------+----------+------------+-----------+--------+--------------+
#| Total      | 70         | 11       | 15%        | 0         | 11     | 0:11         |
#+------------+------------+----------+------------+-----------+--------+--------------+
```

Related Information

- `createShield`
- `deleteShield`
- Using the NanoRoute Router chapter in the *Innovus User Guide*
  - Performing Shielded Routing

reportWire

```
reportWire
[-help]
[fileName]
[threshold]
[-detail]
[-maxFanout integer]
[-minWireLength float]
[-sort {ratio wire_length half_perimeter}]
[-summary]
```

Creates a report file that contains wire statistics for signal nets with a real wire length or minimum wire length that is greater than the specified threshold ratio. The wire statistics for signal nets can also be based on the fanout or wire length of the net.

Ratio (*r*) is calculated as *r* = (*total net routing length*) / (*half-perimeter of net bounding box*).

The wire information reported includes the net name, the wire ratio, the real and minimum wire lengths, the number of vias, and the number of fanins and fanouts on the net. Use the `reportWire` command after routing with any of the routing commands in the Innovus software.

Note: Use the `reportRoute` command to generate a layer-by-layer wire statistics report. Use the `reportSpecialRoute` command to generate a report for special nets.

Parameters

`-detail`	Includes detailed information in the report. This includes the wire length and number of vias for each layer, and a summary of the total wire length and via number for the design. *Data_type*: bool, optional
fileName	Creates a report file using the specified name. *Default:* Creates a report file named <`topcell`>`.wirerpt` *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `reportWire` parameter. For a detailed description of the command and all of its parameters, use the man command `man reportWire`.
`-maxFanout` *fanout*	Specifies the maximum number of net's fanout. *Data_type*: int, optional
`-minWireLength` *length*	Specifies the minimum wire length of the net. *Data_type*: float, optional
`-sort {ratio wire_length half_perimeter}`	Specifies the order in which the nets are sorted in the output report. The nets can be sorted as per the ratio, wire length, or half perimeter length. *Data_type*: enum, optional
`-summary`	Specifies that just the summary table should be displayed in the report. *Data_type*: bool, optional
threshold	Reports wire statistics for wires with lengths greater than the specified threshold ratio. If you do not specify a threshold ratio, the `reportWire` command calculates wire information for wires with a ratio greater than `1.0`. *Default:* `1.0` **Note:** There are many situations in which the threshold ratio can be less than `1.0`. To ensure that the software creates a report on all nets, specify a ratio value of `0`. *Data_type*: float, optional

Examples

- The following command writes the wire statistics for all nets with wire lengths greater than the 1.0 threshold ratio:
  ```
  reportWire 1.0
  ```

The following example shows a section of the wire information reported:

```
ratio      wire_length  half_perimeter  vias fan_in fan_out net_name
                            length
------------------------------------------------------------------------
1.08 Total   177.090     164.16 Total     4     1       1     dOut2[1]
1.12 Total    79.185      70.94 Total     3     1       1     dOut2[0]
1.04 Total   179.960     173.46 Total     7     1       4     dOut2[1]
1.00 Total   178.680     178.02 Total     8     1       4     dOut2[0]
1.02 Total   203.420     199.76 Total    10     1       4     dOut2[1]
1.25 Total   222.420     178.30 Total    12     1       4     dOut2[0]
1.18 Total   185.645     157.32 Total    11     1       4     s2[1]
1.09 Total   456.740     155.76 Total    10     1       4     s2[0]
...
```

- The following command writes the detailed wire statistics for all nets with wire lengths greater than the 1.0 ratio to the `detail.wirerpt` file. This report includes the wire length and number of vias for each layer, and a summary of the total wire length and via number for the design:

  ```
  reportWire -detail detail.wirerpt 1.0
  ```

  The following example shows a section of the wire information and the summary from the end of the report:

  ```
  ratio      wire_length      half_perimeter  vias fan_in fan_out net_name
                                length
  ------------------------------------------------------------------------
  1.08 Total   177.090         164.16 Total     4     1       1     dOut2[1]
       Metal1         7.650               Via01      0
       Metal2       103.440               Via12      2
       Metal3        66.000               Via23      2
       Metal4         0.000               Via34      0
       Metal5         0.000               Via45      0
       Metal6         0.000               Via56      0
  1.12 Total    79.185          70.94 Total     3     1       1     dOut2[0]
       Metal1         8.245               Via01      0
       Metal2        61.040               Via12      1
       Metal3         9.900               Via23      2
       Metal4         0.000               Via34      0
       Metal5         0.000               Via45      0
  ...
  ### Summary Report ###
  Total Metal1 Wire Length = 33.99
  Total Metal2 Wire Length = 1869.13
  Total Metal3 Wire Length = 1885.62
  Total Metal4 Wire Length = 67.76
  Total Metal5 Wire Length = 0.00
  Total Metal6 Wire Length = 0.00
  ---------------------------------
  Total Wire Length = 3856.51
  Total Via01 Number = 0
  Total Via12 Number = 272
  Total Via23 Number = 242
  Total Via34 Number = 3
  Total Via45 Number = 0
  Total Via56 Number = 0
  ---------------------------------
  ```

Related Information

- `reportRoute`
- `reportSpecialRoute`
- Using the NanoRoute Router chapter in the *Innovus User Guide*

route_fix_signoff_drc

```
route_fix_signoff_drc
[-help]
[-fix_rule ruleName | -exclude_rule ruleName]
```

Fixes sign-off DRC. It fixes third party DRC violations which cannot be detect by the Innovus verification tool (like verify_drc). The third party markers need to be converted with the `loadViolationReport` command. The supported drc types include, spacing, mincut, colorLoop, area, minhole, maxstackedvia, enclosure, width, and minstep.

Parameters

`-exclude_rule` *ruleName*	Specifies the sign_off tool rule type to exclude. *Data_type*: string, optional
`-fix_rule` *ruleName*	Specifies the sign_off tool rule type to fix. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `route_fix_signoff_drc` parameter. For a detailed description of the command and all of its parameters, use the man command `man route_fix_signoff_drc .`

Related Information

- `loadViolationReport`
- Using the NanoRoute Router chapter in the *Innovus User Guide*

routeDesign

```
routeDesign
[-help]
[-backside]
[-bump]
[-highFrequency]
[-selected]
[-viaPillarOpt]
[[[-clockEco ] [-globalDetail ]
  [-placementCheck | -noPlacementCheck ]
  [[ -trackOpt ] [-idealClock ]]
  [-viaOpt ] [-wireOpt ]] | [-passiveFill ]]
```

Runs routing or postroute via or wire optimization using the NanoRoute router. If you specify this command without any arguments, it runs global and detailed routing.

The `routeDesign` command has the following characteristics:

- It supports multiple-CPU processing.
- It checks for conflicts in option settings and issues warning messages when it finds problems. For example, trying to fix postroute lithography problems and optimize vias concurrently can cause conflicts.
  If `routeDesign` detects requests for both types of operation, it issues a warning, turns off via optimization, and proceeds with fixing lithography problems.
- It honors all `setNanoRouteMode` settings that do not have conflicts
- `routeDesign` is a super command that handles the setting of NanoRoute variables so that it is run in timing-driven mode then sets them back postRoute. It runs SMART routing by default; that is, it runs in both timing- and signal integrity-driven mode by default.
  Note: The other routing commands are not timing- or signal-integrity driven by default, but you can use the following `setNanoRouteMode` parameters to turn on timing- and signal-integrity-driven routing for those commands:
  To turn on timing-driven routing, run the following command before running `routeDesign`:
  ```
  setNanoRouteMode -routeWithTimingDriven true
  ```
  To turn on SI-driven routing, run the following command before running `routeDesign`:
  ```
  setNanoRouteMode -routeWithSIDriven true
  ```
  If you run either (or both) of these commands, the software stores the mode settings in the .mode file. If you restore the design and run `routeDesign` again, it honors the these settings. For example, if you type the following commands, the next time you restore the design and run `routeDesign`, it routes the design in timing-driven mode:
  ```
  setNanoRouteMode -routeWithTimingDriven true
  routeDesign -globalDetail
  saveDesign
  ```
- It updates the timing file automatically after all the nets are detail routed in timing-driven mode. Updating the file ensures that critical nets are not touched during postroute multiple-cut via swap, wire spreading and wire widening. When determining whether all the nets are detail routed, the router skips pins without physical ports.
- It runs a placement check prior to routing and displays a warning message if the placement is not clean.
- To turn off the placement check, specify the following parameter: `-noPlacementCheck`
- It changes the status of clock nets from FIXED to ROUTED so it can modify them during routing and routes them before routing other nets. Once the status of the clock nets is set to ROUTED, it does not change it back to FIXED.
  To keep clock nets' status FIXED, run the following command before running `routeDesign`:
  ```
  setNanoRouteMode -routeDesignFixClockNets true
  ```
  To stop the router from routing clock nets first, run the following command before running `routeDesign`:
  ```
  SetNanoRouteMode -routeDesignRouteClockNetsFirst false
  ```

Note: You can use the `check_design -type route` command to detect the potential problems and correct the design before running the `routeDesign` command

- For issues related to unplaced instances that clearly affect routing, the `check_design` command displays error messages and returns a status to discontinue the flow. You must resolve the issue before continuing to the route phase.
- For issues related to adherence to the track rules, the `check_design` command checks the design. It currently makes a basic check that the structures needed for routing adhere to the tracking rules before routing begins.
- For issues related to unplaced top level pins that clearly affect routing, the `check_design` command displays error messages and returns a status to

discontinue the flow. You must resolve the issue before continuing to the route phase.

- For issues related to adherence to the third party routing rules, the check_design command checks the design for metal violations. If the design is taped out or known to be clean, the presence of violations represents a discrepancy between the design and the tech.lef. It normally indicates that the tech may be of the wrong version or may need to be updated to the technology that the design was taped out using. These issues must be understood in detail and waved individually and on purpose by changing the tech.lef before continuing to the route phase.
- For issues related to overlapping cells that clearly affect routing, the `check_design` command displays error messages and returns a status to discontinue the flow. You must resolve the issue before continuing to the route phase.
- For issues related pin spacing violations, the `check_design` command checks if the pin has a spacing violation with the same cell or a different metal cell. It checks macro pin related spacing violations and runs the `verify_drc` command to catch spacing issues that may block the pin access.You must resolve the issue before continuing to the route phase.
- For issues related to routing congestion, the `check_design` command checks the NanoRoute congestion table and reports the congestion in the NanoRoute log format (with the information that the source is NanoRoute Global). Alternatively, it reports the Early Global Route congestion in the NanoRoute format. By default, the `check_design` command stops if the Worst Over Congested Gcell is above 0.75%. Even though these issues can be ignored, it is recommended that you understand them in detail and wave them individually before continuing to the route phase.
- For issues related to a blockage between the pin and the attribute constraint layer at the pin location, the `check_design` command displays error messages and returns a status to discontinue the flow. You must resolve the issue before continuing to the route phase.
- For issues related to pre-existing violations in ECO routing, the `check_design` command runs the `verify_drc` command and then checks the results for routing wire DRC and ignores non-routing wire DRC. If any metal violation is found, the `check_design` command displays error messages and returns a status to discontinue the flow. You must resolve the issue before continuing with ECO routing.
- For issues related to adherence to the third party routing rules, the check_design command checks the design for metal violations. If the design is taped out or known to be clean, the presence of violations represents a discrepancy between the design and the tech.lef. It normally indicates that the tech may be of the wrong version or may need to be updated to the technology that the design was taped out using. These issues must be understood in detail and waved individually and on purpose by changing the tech.lef before continuing to the route phase.
- For issues related to unplaced top level pins that clearly affect routing, the `check_design` command displays error messages and returns a status to discontinue the flow. You must resolve the issue before continuing to the route phase.
- For issues related to pin access for non-default rules, the `check_design` command checks each NDR nets output pin for two-cut APA accessibility. For macros, it checks if the pin width is less than or equal to the NDR. The `check_design` command displays error messages if the NDR is hard or if the no taper property is set to true. You must resolve the issue before continuing to the route phase.

Parameters

`-backside`	Routes back side layers only. The NanoRoute router supports back side routing for the layers defined as "BACKSIDE" in LEF and cell library pins. When the `-backside` parameter is enabled, the router connects the pins in the back side layers only. **Note:** Back side routing layer has the “BACKSIDE” LEF property assigned to it. The back side rule is used to specify that the routing layer is used on the underside of the die. *Data_type*: bool, optional
`-bump`	Enables interposer routing to connect the macro bumps with bus routing patterns. **Note:** Before enabling this functionality, you must specify the routing settings using the `-route_interposer*` parameters of the `setNanoRouteMode` command. *Data_type*: bool, optional
`-help`	Outputs a brief description that includes type and default information for each `routeDesign` parameter. For a detailed description of the command and all of its parameters, use the man command: `man routeDesign`.
`-highFrequency`	Runs high frequency routing. It routes high frequency digital signals to achieve critical performance. *Data_type*: bool, optional
`-clockEco`	Runs eco routing only on clock nets. *Data_type*: bool, optional

`-globalDetail`	Runs timing-driven and SI-driven global and detailed routing. `-globalDetail` is the default value for this command. When specified, it unfixes the clock nets and then routes all nets, including clocks that still need to be routed. It does not ECO route the clock nets prior to other signals so clock nets are not given priority as they are with `routeDesign`. **Note**: Running the `routeDesign` command without the `-globalDetail` parameter, unfixes the clock nets and will ECO route the clock nets first before routing the remaining signal nets. *Data_type*: bool, optional
`-idealClock`	Performs timing optimization using ideal clocks instead of propagated clocks. Using ideal clocks enables you to either optimize timing without building a clock tree first, or to ignore the existing clock tree, to close ideal timing. If this parameter is not specified, the software assumes that the clock tree has been built, and optimizes timing based on propagated clocks. **Note**: The `-trackOpt` parameter must be specified while using this parameter. **Note**: When this parameter is specified, all propagated clock mode settings are reset; the `setAnalysisMode -clockPropagation` parameter is set to `forcedIdeal`. *Data_type*: bool, optional
`-noPlacementCheck`	Turns off placement checking. By default, `routeDesign` runs a placement check before routing and displays a warning message if the placement is not clean. For information on the types of checks and the violation report, see `checkPlace`. *Data_type*: bool, optional
`-passiveFill`	Performs passive fill. *Data_type*: bool, optional
`-placementCheck`	Checks for placement violations before routing. If the placement is not clean, `routeDesign` display a warning message. To bypass this check use `routeDesign -noPlacementCheck`. *Data_type*: bool, optional
`-selected`	Routes selected nets only. *Data_type*: bool, optional
`-trackOpt`	Performs timing optimization after track assignment. Specify this option to ensure that TQuantus model file is used during `timeDesign` and `optDesign -postRoute` stages. **Note**: If this parameter is specified and the TQuantus model file is missing, the TQuantus model file will be generated and set automatically. If the timing library does not exist, timing optimization will be skipped and continue routing. For details of the TQuantus model file usage, see the "Creating RC Model Data in TQuantus Model File" section in the *Innovus User Guide*. *Data_type*: bool, optional
`-viaOpt`	Optimizes vias after routing. **Warning**: Exercise caution with the use of this parameter, as the router might create violations during postroute via optimization. It runs a search and repair step after optimization if violations occur. *Data_type*: bool, optional
`-viaPillarOpt`	Performs opportunistic via pillar insertion. **Note**: After the design is routed or post-route optimized, the NanoRoute router looks for available tracks around the instance pin to either add a new via pillar or upgrade the original smaller via pillar to the one with more fingers. You can perform opportunistic via pillar insertion by using the `routeDesign -viaPillarOpt` command. After opportunistic via pillar insertion, there is no need for `ecoRoute`; opportunistic via pillar insertion is DRC clean by construction. *Data_type*: bool, optional
`-wireOpt`	Optimizes wires after routing. This parameter, when specified, widens and shrinks wires after detail routing has been performed. The settings specified using the following `setNanoRouteMode` options impact the behavior of this parameter: • `-droutePostRouteWidenWire` • `-droutePostRouteSpreadWire` • `-droutePostRouteWidenWireRule` *Data_type*: bool, optional

Examples

Routing

- To run timing- and SI-driven global and detailed routing, type the following command:
  ```
  routeDesign
  ```
- To run timing- and SI-driven global and detailed routing, type the following commands:
  ```
  routeDesign -globalDetail
  ```
- To run non-timing-driven global and detailed routing, type the following commands:
  ```
  setNanoRouteMode -routeWithTimingDriven false
  routeDesign -globalDetail
  ```

Postroute Yield Optimization

- To run postroute via reduction, type the following commands:
  ```
  setNanoRouteMode -drouteMinSlackForWireOptimization slack
  setNanoRouteMode -droutePostRouteMinimizeViaCount true
  routeDesign -viaOpt
  ```
  Note : When you run these commands, the software optimize and reduce the via count. Use these commands only if no concurrent via optimization was done.
- To run postroute single-cut via to multiple-cut via swap, type the following commands:
  ```
  setNanoRouteMode -drouteMinSlackForWireOptimization slack
  setNanoRouteMode -droutePostRouteSwapVia multiCut
  routeDesign -viaOpt
  ```
- To run non-timing-driven postroute single-cut via to multiple-cut via swap, type the following commands:
  ```
  setNanoRouteMode -routeWithTimingDriven false
  setNanoRouteMode -droutePostRouteSwapVia multiCut
  routeDesign -viaOpt
  ```
- To run postroute multiple-cut via to single-cut via swap, type the following commands:
  ```
  setNanoRouteMode -drouteMinSlackForWireOptimization slack
  setNanoRouteMode -droutePostRouteSwapVia singleCut
  routeDesign -viaOpt
  ```
- To run non-timing driven postroute multiple-cut via to single-cut via swap, type the following commands:
  ```
  setNanoRouteMode -routeWithTimingDriven false
  setNanoRouteMode -droutePostRouteSwapVia singleCut
  routeDesign -viaOpt
  ```
- To run postroute wire widening, type the following commands:
  ```
  setNanoRouteMode -drouteMinSlackForWireOptimization slack
  setNanoRouteMode -droutePostRouteWidenWireRule ruleName
  setNanoRouteMode -droutePostRouteWidenWire widen
  routeDesign -wireOpt
  ```
- To run non-timing driven postroute wire widening, type the following commands:
  ```
  setNanoRouteMode -droutePostRouteWidenWireRule ruleName
  setNanoRouteMode -routeWithTimingDriven false
  setNanoRouteMode -droutePostRouteWidenWire widen
  routeDesign -wireOpt
  ```
- To run postroute wire unwidening, type the following commands:
  ```
  setNanoRouteMode -drouteMinSlackForWireOptimization slack
  setNanoRouteMode -droutePostRouteWidenWireRule ruleName
  setNanoRouteMode -droutePostRouteWidenWire shrink
  routeDesign -wireOpt
  ```
- To run non-timing-driven postroute wire unwidening, type the following commands:

```
setNanoRouteMode -droutePostRouteWidenWireRule ruleName
setNanoRouteMode -routeWithTimingDriven false
setNanoRouteMode -droutePostRouteWidenWire shrink
routeDesign -wireOpt
```

- To run postroute wire spreading, type the following commands:
  ```
  setNanoRouteMode -drouteMinSlackForWireOptimization slack
  setNanoRouteMode -droutePostRouteSpreadWire true
  routeDesign -wireOpt
  ```
- To run non-timing-driven postroute wire spreading, type the following commands:
  ```
  setNanoRouteMode -routeWithTimingDriven false
  setNanoRouteMode -droutePostRouteSpreadWire true
  routeDesign -wireOpt
  ```

Related Information

- `detailRoute`
- `globalDetailRoute`
- `globalRoute`
- `setNanoRouteMode`
- Using the NanoRoute Router chapter in the *Innovus User Guide*
  - Using the routeDesign Supercommand
  - Postroute Via Optimization
- Accelerating the Design Process By Using Multiple-CPU Processing chapter in the *Innovus User Guide*

saveRouteGuide

```
saveRouteGuide
[-help]
[-rguide routeGuideFileName]
[-selNetFile selNetFileName]
```

Saves the routing information to a route guide format file. Routing guide files help control the routing topology by creating regions in which Early Global Route can create wires. You can use the `saveRouteGuide` command after running Early Global Route or after restoring routing data.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `saveRouteGuide` parameter. For a detailed description of the command and all of its parameters, use the man command `man saveRouteGuide`.
`-rguide` *routeGuideFileName*	Specifies the route guide file to which to save the routing information. *Data_type*: string, optional
`-selNetFile` *selNetFileName*	Specifies a file of hierarchical net names for which to save route information. Routing information is saved only for these nets. *Data_type*: string, optional

Examples

- The following command writes the route information in route guide format to the `cts_3.guide` file:
  ```
  saveRouteGuide -rguide cts_3.guide
  ```

Related Information

- `earlyGlobalRoute`
- Using Early Global Route for Congestion and Timing Analysis chapter in the *Innovus User Guide*

setAttribute

```
setAttribute
[-help]
[-max_fanout integer]
[-reset]
[-stripe_layer_range minLayer:maxLayer]
{-net netName | -cell_pin cell_name:pin_name}
[-sub_net sub_net_name [-comp_pin comp:pin_name list][-inst_pin instance:pin_name list]]
[-weight integer]
[-non_default_rule rule_name ]
[-non_default_rule_effort {auto|soft|hard|no_taper}]
[-shield_net special_net_name ]
[-shield_side {one_side | two_sides}]
[[[-em_ndr_rule ruleName] [-em_ndr_dist double]] | [-em_width_rule layer width:dist pairs]]
[-ndr_si_length_limit layer:distance list]
[-coaxial_shielded_layers layer_number:layer_number]
[-coaxial_shield_limit layer_number:layer_number]
[-coaxial_shield_interval tracks_number | metal_layer_name:tracks_number …]
[-pattern {steiner|trunk}]
[-top_preferred_routing_layer layer_number]
[-bottom_preferred_routing_layer layer_number]
[-top_preferred_shielding_layer layer_number]
[-bottom_preferred_shielding_layer layer_number]
[-preferred_routing_layer_effort {low|medium|high|hard}]
[-preferred_extra_space integer]
[-avoid_detour {true|false}]
[-skip_routing {true|false}]
[-skip_antenna_fix {true|false}]
[-multi_cut_via_effort {low|medium|high}]
[-si_post_route_fix {true|false}]
[-min_stack_layer layer_number]
[-stack_distance distance]
[-mask mask_number]
[-layer_mask bottomLayerNum:topLayerNum]
[-one_side_spacing bottomLayerNum:topLayerNum]
```

Attaches attributes to nets and subnets. Attaching the attributes allows the NanoRoute routing commands (`globalRoute`, `detailRoute`, `globalDetailRoute`, `routeDesign`, and `setNanoRouteMode`), and the `earlyGlobalRoute` command to route the nets following specific requirements. Attributes are persistent; that is, throughout the routing process, from global routing to optimization, the routers honor the attributes. The attributes are saved in the Innovus System database. If you save the database and exit, the attributes remain attached to the nets when you re import the database.

Note: Attributes are not saved in the DEF file.

To see the current attribute settings, use the `getAttribute` command. If you attempt to run the `setAttribute` command on any net that is part of a diffPair, matchLength, or complex shield constraint group, or a simple or complex netClass, the command reports an error stating that the command cannot be run on nets with integration constraints.
Tip: To revert to the default value of an attribute, specify default for the attribute.

Parameters

`-avoid_detour {true | false}`	Avoids detours of roughly more than a few gcell grids on the specified nets. This attribute affects global routing only. **Note:** Cadence recommends that you use caution with this attribute, as it adds congestion to the design. *Data_type*: string, optional
`-bottom_preferred_routing_layer layer_number`	

	Specifies the preferred lowest routing layer. This attribute is a soft limit; that is, NanoRoute might use a layer below the specified layer if necessary to complete routing. *Range:* 0-15 **Note:** Layers can only be specified using their layer ID numbers. Use of LEF layer names is not supported by this parameter. **Note:** If the top routing layer is equal to or greater than 4 and no bottom preferred routing layer is specified for clock nets, the router automatically sets the bottom preferred routing layer for clock nets to 3, to prevent the clock nets from blocking other nets. *Data_type*: string, optional
`-bottom_preferred_shielding_layer` *layer_number*	
	Specifies the preferred lowest shielding layer. **Note:** Layers can only be specified using their layer ID numbers. Use of LEF layer names is not supported by this parameter. *Data_type*: string, optional
`-cell_pin` *cell_name*:*pin_name*	Specifies attributes for all the nets connected to pins of instances having the same master cells. You can use : or / as a delimiter when you specify the cell name and the corresponding pin name. **Note:** If the specified `setAttribute` commands conflict, then the last setting takes precedence over all the previous `setAttribute` settings. *Data_type*: string, optional
`-coaxial_shield_interval` *tracks_number* | *metal_layer_name*:*tracks_number …*	
	Specifies the interval of crossties by routing track. *Range:* 0-63 *Default*: 0. Every track could be used by crosstie. For example, when `-coaxial_shield_interval 2` is specified, two tracks between two crossties will be empty. *Data_type*: string, optional
`-coaxial_shield_limit` *layer_number:layer_number*	
	Specifies the bottom and top routing layer limits for coaxial shields. *Data_type*: string, optional
`-coaxial_shielded_layers` *layer_number*:*layer_number*	
	Specifies the bottom and top routing layer for coaxial shields. *Data_type*: string, optional
`-comp_pin` *comp*:*pin_name list*	Specifies attributes for all nets and sub-nets connected to pins of instances having the same component. You can specify a list of pins. You can use **:** or / as a delimiter when you specify the component names and the corresponding pin names. *Data_type*: string, optional
`-em_ndr_dist` *double*	Specifies the distance from source to sink where it is needed to be applied with the EM ndr rule *Default*: 0 *Data_type*: string, optional
`-em_ndr_rule` *ruleName*	Specifies the non-default rule name which is used for EM issue fixing. *Default*: " " empty *Data_type*: string, optional

`-em_width_rule` *`layer width:dist pairs`*	Specifies the table of layer width:distance pairs. The layer width pair denotes the width for any layer to be changed from default width for EM fixing in distance X from driver. For example, `Metal1 1.0:0.5, Metal2 2.0:1.0` **Note**: The `-em_width_rule` parameter must be used with the `-em_ndr_dist` parameter. It can not be used with the `-em_ndr_rule` parameter as only one parameter can specify the non default width to apply. *Data_type*: string, optional
`-help`	Outputs a brief description that includes the type and default information for each `setAttribute` parameter.For a detailed description of the command and all of its parameters, use the man command: `man setAttribute`.
`-inst_pin` *`instance:pin_name list`*	Specifies attributes for all nets and sub-nets connected to pins of the same instance. You can specify a list of pins. You can use **:** or / as a delimiter when you specify the pin names and the corresponding instance names. *Data_type*: string, optional
`-layer_mask` *`bottomLayerNum:topLayerNum`*	
	Specifies the layer range to which the mask constraint should be applied. For example, if you specify 3:4, it means that the constraint should be applied on layers, Metal3 and Metal4. *Data_type*: string, optional
`-mask` *`mask_number`*	Masks constraint setting for the net. Legal values are from 0 to the mask number for the layers. A value of 0 indicates that there is no mask constraint on the net. A non-zero value indicates that all wiring on the net should be created on the specified mask. Layers that do not support the specified value will be treated as unconstrained. *Data_type*: string, optional
`-max_fanout` *`integer`*	Specifies the maximum fanout value for nets and subnets. Specifying `setAttribute -net` *`netname`* `-max_fanout` *`value`* provides per net based max_fanout constraint/control for a net. ◦ If `-max_fanout value` of a net is 0, and the `-pattern` of the net is `trunk`, the NanoRoute router honors the value of `getNanoRouteMode -routeTrunkWithClusterTargetSize` as the max fanout. ◦ If `-max_fanout value` of a net is larger than 0, and the `-pattern` of the net is `trunk`, the `-max_fanuout` value overrides the value of `getNanoRouteMode -routeTrunkWithClusterTargetSize` for the net. *Default*: 0 *Data_type*: string, optional
`-min_stack_layer` *`layer_number`*	Specifies the minimum stacking layer. It indicates that stacked vias should be added to the specified layer on output pin only. If a pin already has pre-defined custom stacked vias, and the uppermost layer of the vias is still below the specified layer, additional stack vias should be added to the specified layer. Then, regular routing would start from the specified layer of the stacked vias. *Data_type*: string, optional
`-multi_cut_via_effort {low|medium|high}`	
	Specifies the multi-cut via effort. The effort level can be low, medium, or high. *Data_type*: string, optional
`-ndr_si_length_limit {`*`layer:distance`* `list}`	

	Specifies the table of parallel run length on each layer. For example, Metal1 1.0, Metal2 2.0. It relaxes a hard NDR into a soft NDR whenever the parallel run length between the NDR wire and an other wire is smaller than the specified value
	For example,
	`setNanoRouteMode -route_strict_honor_route_rule true`
	`setAttribute -net {xgxs_tx_rdy_lane[1]} -ndr_si_length_limit {metal3:0.054}`
	Where, xgxs_tx_rdy_lane[1] is the net name, metal3:0.054 is the prl limit 0.054 in Metal3 layer.
	This setting is for, if parallel run length is less than distance(0.054) in the layer(Metal3) for the specified net, no NDR spacing is needed for this net (wire segment).
	Data_type: string, optional

`-net` *netName*	Specifies the net for which to set attributes. Do not enter more than one net name. • To specify more than one net, use `*` and `?` wildcard characters. • To specify nondefault rule nets, use `@RULE`. • To specify clock nets, use `@CLOCK`. (To select clock nets use `selectNet -allDefClock`.) • To specify prerouted nets, use `@PREROUTED`. **Note**: Remove an attribute from a net by specifying none. For example, to remove the - shield_net attribute from net_1, specify the following command: `setAttribute -net net_1 -shield_net none` *Data_type*: string, optional
`-non_default_rule` *rule_name*	Identifies the nondefault routing rule to use with the specified net. Use this attribute for critical nets, such as clock nets, that might need to have wider wires or wider spacing than other nets. The NanoRoute router treats nondefault rule spacing as a soft attribute; that is, when routing resources are available, it honors the nondefault rule. If the area is too congested, and resources are not available, the router might not honor nondefault spacing rules. You can override this behavior by specifying `setNanoRouteMode -route_strict_honor_route_rule true`. When you specify this parameter, the router uses the nondefault spacing rules. You define nondefault routing rules in the NONDEFAULTRULE statement of the LEF file. *Data_type*: string, optional
`-non_default_rule_effort {auto|soft|hard|no_taper}`	
	Specifies the non-default rule effort to use with the specified net. You can set any one of the options: • `auto`: follows the HARDSPACING for the NDR in the LEF file. • `hard`: makes the NDR hard even if HARDSPACING is not set in the LEF file. • `soft`: makes the NDR soft even if HARDSPACING is set in the LEF file. • `no_taper`: follows the HARDSPACING for the NDR in the LEF file and does not allow tapering at all pins. *Default*: `auto` *Data_type*: string, optional
`-one_side_spacing` *bottomLayerNum:topLayerNum*	
	Specifies the layer range that the `one_side_spacing` constraint should be applied on. This constraint means that spacing is reserved on any one side. For example, 3:4 means that the constraint should be applied on layers, Metal3 and Metal4. *Data_type*: string, optional
`-pattern {steiner | trunk}`	Specifies the routing pattern. Select one of the following: • `steiner`: Routes all pins with a single steiner tree. • `trunk`: Routes pins to the nearest special net of the same name. Creates multiple steiner trees and connects them to the trunk. • *Data_type*: string, optional

`-preferred_extra_space` *integer*	Specifies the preferred extra spacing for nets. Gives additional pitch spacing to the specified net. Use this attribute to give critical nets extra space to reduce coupling. Specify `-preferred_extra_space 1` to give a net 1 extra pitch spacing, compared to other nets. This parameter creates a soft rule and, if the design is congested, might not give the extra space to the net. *Range:* 0-3 **Note:** If you specify this attribute, Early Global Route also gives additional space to the specified nets. *Data_type*: string, optional
`-preferred_routing_layer_effort {low | medium | high | hard}`	
	Determines how flexible the NanoRoute router is when setting layer limits for routing specified nets. Use this attribute with the `-top_preferred_routing_layer` and `-bottom_preferred_routing_layer` attributes. Specify medium, high, or hard to make NanoRoute less flexible (that is, to make NanoRoute obey the preferred routing layer range more strictly). When set to hard, NanoRoute prevents the nets from going to the non-preferred routing layers by setting large costs. *Default:* low (more flexible) *Data_type*: string, optional

`-reset`	Currently you cannot reset all the attributes of a net. You can reset each attribute of a net by using the `-reset` parameter with the attribute or you can use the “default” keyword for the value of each attribute.
	For example:
	• The following command sets the weight attribute of the net (scan_in_1) to 3 and the mask attribute to 2:
	`setAttribute -net scan_in_1 -weight 3 -mask 2`
	• The following command resets the weight attribute of the net:
	`setAttribute -net scan_in_1 -reset -weight -mask 2`
	• The following command resets the mask attribute of the net:
	`setAttribute -net scan_in_1 -weight 3 -mask default`
	Data_type: string, optional

`-shield_net` *special_net_name*	Specifies a special net to use as a shield for a critical or high-speed net. Typically, you route shielded nets before routing other nets. You can specify one or two shield nets, however, you cannot control which side is used by the special net that is routed on the same layer as the signal routing. The shield terminates near the pin. Shielding information is saved in the DEF file. **Note:** The router does not support shielding nets that are defined in the SPECIALNET section of the DEF file. This limitation extends to nets that are defined partly in the SPECIALNET section and partly in the regular NET section of the file, such as some clock nets. If you set the shielding attribute on these nets, the router automatically sets the - skip_routing attribute on them as well. As a result, the nets are not modified by the router and, if they are not completely routed already, the router does not try to finish the connections. The router does not delete the shielding wires. *Data_type*: string, optional
`-shield_side {one_side | two_sides}`	
	Specifies the number of sides to route shield. *Default*: `two_sides` *Data_type*: string, optional
`-si_post_route_fix {true | false}`	
	Repairs crosstalk violations on specified nets. *Data_type*: string, optional
`-skip_antenna_fix {true | false}`	Prevents the router from repairing violations on specified nets with process antenna violations. This attribute is useful on nets that will have antenna violations repaired at the next level of hierarchy. *Data_type*: string, optional

`-skip_routing {true | false}`	Prevents the NanoRoute router from routing or rerouting unrouted or partially routed nets. When you specify `-skip_routing` for a net, the router treats the net as if you had marked it + FIXED in the DEF file and it becomes an obstruction for other nets. If the net is partially routed, the router does not complete the routing. **Note**: The value you specify for this attribute also applies to Early Global Route. **Note:** To skip routing on prerouted nets, specify `setAttribute` *Data_type*: string, optional
`-stack_distance` *distance*	Specifies the stacking distance in microns. *Default*: `0` *Data_type*: string, optional
`-stripe_layer_range` *minLayer*`:`*maxLayer*	Specifies the layer range that accommodates different StripeLayerRanges for different nets and subnets. You need to specify the following values with this parameter: • *maxLayer*: Specifies the top-most floating stripe layer allowed for connection. • *minLayer*: Specifies the bottom-most floating stripe layer allowed for connection For example, 3:4 means that the constraint should be applied on M3 and M4. *Data_type*: string, optional

`-sub_net` *sub_net_name*	Specifies the subnet for which to set attributes. This parameter is used with the `-net` parameter.
	For example:
	`setAttribute -net` *netName* `-sub_net` *sub_net_name*
	Note: Cadence recommends you to use the `dbSet` command for changing the specified attribute value for a database object. The `dbSet` command takes an object or object list as a starting point, then uses a period to traverse to other related objects, or access attributes. Additional periods can be used to continue traversing the database schema.
	For example, to specify the top preferred routing layer and bottom preferred routing layers attributes for a subnet, you can use the following `dbSet` commands:
	`dbSet [dbGet [dbGet head.libcells.name $cell_name -p].pgTerms.name $pg_pin_name -p].topPreferredLayer [dbGet head.layers.num 3 -p]`
	`dbSet [dbGet [dbGet head.libcells.name $cell_name -p].pgTerms.name $pg_pin_name -p].bottomPreferredLayer [dbGet head.layers.num 2 -p]`
	instead of using the `setAttribute -sub_net` command.
	`setAttribute -net VDD1 -sub_net VDD1_2 -inst_pin iA/FEAOB*:TVDD -top_preferred_routing_layer 3 -bottom_preferred_routing_layer 2`
	Data_type: string, optional

`-top_preferred_routing_layer` *layer_number*	
	Specifies the preferred highest layer for routing specified signal nets. This attribute is a soft limit; that is, NanoRoute might use a layer above the specified layer if necessary to complete routing. Use this attribute to improve timing by restricting the routing layers available for critical nets, while simultaneously routing other nets on all routing layers. *Range:* 0-15 **Note:** Layers can only be specified using their layer ID numbers. Use of LEF layer names is not supported by this parameter. *Data_type*: string, optional
`-top_preferred_shielding_layer` *layer_number*	
	Specifies the preferred highest shielding layer. **Note:** Layers can only be specified using their layer ID numbers. Use of LEF layer names is not supported by this parameter. *Data_type*: string, optional

`-weight integer`	Specifies a relative weight for the nets. This value is honored by placement and not by the router. *Default:* 2 *Data_type*: string, optional

Examples

- The following command sets the net weight for `net1` to `4`:
  ```
  setAttribute -net net1 -weight 4
  ```
  Setting the weight to `4` forces the router to route `net1` before it routes nets whose net weight is less than 4.
- The following command sets the net weight for all nets whose net name starts with `net1` to `4`:
  ```
  setAttribute -net net1* -weight 4
  ```
- The following command limits routing for `net1` to layers Metal4 and Metal5 if possible:
  ```
  setAttribute -net net1 -top_preferred_routing_layer Metal5\
  -bottom_preferred_routing_layer Metal4
  ```
- The following command adds two additional grid units around `net1`. The additional grid units do not have any routing.
  ```
  setAttribute -net net1 -preferred_extra_space 2
  ```
- The following command uses the nondefault rule `wide_wire1` for routing `net1`:
  ```
  setAttribute -net net1 -non_default_rule wide_wire1
  ```
- The following command uses the default rule for routing `net1`.
  ```
  setAttribute -net net1 -non_default_rule default
  ```
- The following commands shield `net1` with both power and ground wires, and shield `net2` with a ground wire:
  ```
  setAttribute -net net1 -shield_net {vdd vss}
  setAttribute -net net2 -shield_net vss
  globalDetailRoute
  ```
- The following commands show how to shield two nets (do not shield more than one net with the same command):
  ```
  setAttribute -net net1 -shield_net abc_gnd
  setAttribute -net net2 -shield_net abc_gnd
  ```

Related Information

- `earlyGlobalRoute`
- `getAttribute`
- `globalRoute`
- `globalDetailRoute`
- `routeDesign`
- `setNanoRouteMode`
- Using the NanoRoute Router chapter in the *Innovus User Guide*

setNanoRouteMode

```
setNanoRouteMode
[-help]
[-extract_keep_fill_wires {true|false}]
[-reset]
[-route_adjust_auto_via_weight {true|false}]
[-route_allow_inst_overlaps {true|false}]
[-route_ignore_follow_pin_shapes {true|false}]
[-route_process_node value]
[-route_rc_extraction_corner value]
[-route_skip_analog {true|false}]
[-route_via_weight{via_name via_weight ...}]
[-route_detail_add_passive_fill_only_on_layers layers]
[-route_detail_allow_passive_fill_only_in_layers layers]
[-route_detail_antenna_eco_list_file file_name]
[-route_detail_auto_stop {true|false}]
[-route_detail_check_mar_on_cell_pin {true|false}]
[-route_detail_end_iteration pass_number]
[-route_detail_fix_antenna {true|false}]
[-route_detail_fix_antenna_on_secondary_pg_nets {true|false}]
[-route_detail_fix_antenna_with_gate_array_filler_mode {true|false}]
[-route_detail_merge_abutting_cut {true|false}]
[-route_detail_min_length_for_spread_wire {layer_1 length_1 ... layer_n length_n}]
[-route_detail_min_length_for_widen_wire value]
[-route_detail_min_slack_for_opt_wire value]
[-route_detail_no_taper_in_layers "bottom_layer_number:top_layer_number"]
[-route_detail_no_taper_on_output_pin {true|false|auto}]
[-route_detail_on_grid_only none | false | true | all | {[wire [ml:mh]] | [via [ml:mh]]}]
[-route_detail_post_route_litho_repair {true|false}]
[-route_detail_post_route_spread_wire {auto| true | false | 0 | 0.25 | 0.5 | 1 }]
[-route_detail_post_route_swap_via {true | multiCut | false | none}]
[-route_detail_postroute_via_priority {auto | allNets | criticalNetsfirst | nonCriticalNetOnly}]
[-route_detail_post_route_via_pillar_effort {none | low | medium | high}]
[-route_detail_post_route_wire_widen {widen | shrink | none}]
[-route_detail_post_route_wire_widen_rule rule_name]
[-route_detail_search_and_repair {true|false}]
[-route_detail_signoff_effort {high | medium | low | auto | n}]
[-route_detail_stub_routing_in_first_layer {true|false}]
[-route_detail_use_multi_cut_via_effort {low | medium | high}]
[-route_number_fail_limit integer]
[-route_number_thread number_processors]
[-route_number_warning_limit integer]
[-route_third_party_data {true|false}]
[-route_high_freq_constraint_groups {order net match bus pair shield}]
[-route_high_freq_match_report_file file_name]
[-route_high_freq_num_reserved_layers value]
[-route_high_freq_remove_floating_shield {true|false}]
[-route_high_freq_search_repair {auto | false | true | only}]
[-route_high_freq_shield_trim_length value]
[-route_interposer_allow_diagonal_trunk {auto | true | false}]
[-route_interposer_interlayer_shielding_layers value]
[-route_interposer_interlayer_shielding_nets value]
[-route_interposer_interlayer_shielding_offsets value]
[-route_interposer_interlayer_shielding_widths value]
[-route_interposer_same_layer_shielding_net net_name]
[-route_interposer_same_layer_shielding_width_spacing {width width}]
[-route_interposer_trunk_routing_layers value]
[-route_interposer_trunk_routing_width_spacing {width width}]
[-route_add_antenna_inst_prefix value]
[-route_allow_pin_as_feedthru {true|TRUE|false|FALSE|none|NONE|output|input|inout, bottomLayerNum:topLayerNum}]
[-route_antenna_cell_name {"cell_name" | "list_of_cell_names"}]
[-route_concurrent_minimize_via_count_effort value]
[-route_connect_to_bumps {true | false}]
[-route_fix_clock_nets {true|false}]
[-route_route_clock_nets_first {true|false}]
[-route_eco_ignore_existing_route value]
[-route_enable_route_rule_si_limit_length value]
```

```
[-route_enforce_route_rule_on_special_net_wire {false | true | special_net_name_list}]
[-route_extra_via_enclosure distance]
[-route_honor_exclusive_region {true|false}]
[-route_honor_power_domain {true|false}]
[-route_ignore_antenna_top_cell_pin {true|false}]
[-route_antenna_diode_insertion {true|false}]
[-route_diode_insertion_for_clock_nets {true|false}]
[-route_shield_tap_cell_insertion {true|false}]
[-route_relaxed_route_rule_spacing_to_power_ground_nets {layer:spacing layer:spacing}]
[-route_reserve_space_for_multi_cut {true|false}]
[-route_reverse_direction {(lx ly ux uy bottom_layer:top_layer)}]
[-route_selected_net_only {true|false}]
[-route_shield_crosstie_offset {layerName:numTrack1 layerName2:numTrack2...}]
[-route_shield_length_threshold value]
[-route_shield_report_skip_status {true|false}]
[-route_shield_tap_cell_name string]
[-route_strictly_honor_1d_routing value]
[-route_strict_honor_route_rule {true | false | bottomLayerNum:topLayerNum | wire bottomLayerNum:topLayerNum}]
[-route_stripe_layer_range "bottomLayerNum:topLayerNum"]
[-route_tieoff_to_shapes "auto | stripe | ring | followpin | powergroundpin"]
[-route_trim_pull_back_distance_from_boundary {layer:value ...}]
[-route_trunk_with_cluster_target_size integer]
[-route_use_auto_via value]
[-route_with_eco {true|false}]
[-route_with_litho_driven {true|false}]
[-route_with_si_driven {true|false}]
[-route_with_timing_driven {true|false}]
[-route_with_trim_metal value]
[-route_with_via_in_pin {true| false | bottomLayerNum:topLayerNum}]
[-route_with_via_only_for_block_cell_pin value]
[-route_with_via_only_for_stdcell_pin {true| false | bottomLayerNum:topLayerNum}]
```

Controls certain aspects of how the NanoRoute router routes the design. Use the `getNanoRouteMode` command to return the current settings for the `setNanoRouteMode` command. The `setNanoRouteMode` parameters affect the behavior of the following commands:

- `detailRoute`
- `globalRoute`
- `globalDetailRoute`
- `routeDesign`

Notes:

- The Nanoroute Router honors litho routing halos (`addRoutingHalo -lithoHalo`). It only allows planar connections to macro pins inside the litho routing halos. It does not allow any other wire or via insertion above and below the litho halo.
- The NanoRoute Router enables off grid shielding when the default track is not available in order to optimize the routing resources.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setNanoRouteMode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setNanoRouteMode`.
`-extract_keep_fill_wires {true|false}`	When enabled, the NanoRoute Router imports metal fill wires so that RC extraction can calculate them. *Default:* `false` *Data_type:* bool

`-reset`	Restores all parameters modified using `setNanoRouteMode` to their default settings. The `-reset` parameter must be the first parameter specified. If you specify -reset by itself, the software resets all `setNanoRouteMode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported.
`-route_add_antenna_inst_prefix` *value*	Specifies the prefix that is added in the diode inst names. These diodes are added to fix antennas during routing. *Default*: " " *Data_type*: string
`-route_adjust_auto_via_weight {true|false}`	Allows auto tuning of internal via weight. It adjusts auto via weight for via swapping. • When set to `true`, it enables NanoRoute to auto tune internal via weight for double cut via ratio improvement. • When set to `false`, it disables NanoRoute to auto tune internal via weight and the via weight specified using the `-route_via_weight` parameter is honored for via swapping. **Note**: NanoRoute does not swap the existing dcut via's to DFM via's. *Default:* `true` *Data_type:* bool
`-route_allow_inst_overlaps {true|false}`	Specifies that instance overlap check will be skipped when this parameter is set to true. *Default:* `true` *Data_type:* bool
`-route_allow_pin_as_feedthru {true|TRUE|false|FALSE|none|NONE|output|input|inout,` *bottomLayerNum*`:`*topLayerNum*`}`	
	Specifies that pins are allowed as feedthrough within the specified layer range. • `true`: Specifies that input/output/inout pins can be used as feedthrough. • `false`: Specifies that output/inout pins can be used as feedthrough. • `none`: Specifies that no pin is allowed as feedthrough. • `output`: Specifies that output pins can be used as feedthrough. • `input`: Specifies that input pins can be used as feedthrough. • `inout`: Specifies that all inout pins can be used as feedthrough. • `bottomLayerNum:topLayerNum`: Specifies the layer range that allows pins as feedthroughs. For example, the following command allows feedthrough for M1 and M2 pins only. `setNanoRouteMode -route_allow_pin_as_feedthrough "true, 1:2"` *Default:* `true` *Data_type*: enum
`-route_antenna_cell_name {"`*cell_name*`" | "`*list_of_cell_names*`"}`	Specifies antenna diode cells to use during postroute optimization. The cells must have the same LEF `SITE` definition as the standard cells. Use this parameter with `-route_antenna_diode_insertion`. *Default:* `""` (empty string) If you do not specify a cell, the router uses a core macro of type ANTENNACELL specified in the LEF MACRO statement. *Data_type*: string

`-route_antenna_diode_insertion {true|false}`	Inserts and places antenna diode cells if there are available placement locations. By default, the NanoRoute router repairs antenna violations by changing layers (also called antenna stapling or layer hopping), but it can also repair antenna violations by inserting diode cells as close as possible to input gates to discharge current. The router inserts antenna diode cells even if filler cells are already placed. It can swap filler cells with antenna diode cells and fill the gap automatically if an antenna diode cell is not the same size as the filler cell it replaced. To specify the diode cells, use `-route_antenna_cell_name`. *Default:* `false` *Data_type*: bool
`-route_concurrent_minimize_via_count_effort value`	Specifies the effort level for reducing the total number of vias during routing. Use this parameter together with the `-route_detail_use_multi_cut_via_effort` parameter to control the effort toward reducing the total number of vias and the effort toward increasing the ratio of double-cut vias to single-cut vias concurrently with routing. If you specify `-route_detail_use_multi_cut_via_effort medium` or `high`, the software sets the value of `-route_concurrent_minimize_via_count_effort` to medium unless you specify `-route_concurrent_minimize_via_count_effort` yourself. To realize further savings, set `-route_concurrent_minimize_via_count_effort` to high. This could further reduce the total via count up to 10 percent. **Note:** If you specify `setDesignMode -flowEffort express`, the software sets the value of `-route_concurrent_minimize_via_count_effort` to `high`. **Note:** Results might vary with your design. Specify one of the following parameters: • `low`: Reduces the total via count using low effort. • `medium`: Reduces total via count by 15 percent, with negligible runtime overhead. This is the default setting. • `high`: Reduces the total via count by 20 percent to 25 percent, with marginal runtime overhead. *Default:* `medium` *Data_type*: string
`-route_connect_to_bumps {true | false}`	Enables bump to bump routing. *Default*: false *Data_type*: bool
`-route_detail_add_passive_fill_only_on_layers layers`	Specifies the layers where passive fill will be added. *Default:* `""`(empty string) *Data_type:* string
`-route_detail_allow_passive_fill_only_in_layers layers`	When specified, allows passive fill only in layers. *Default:* false *Data_type:* string

`-route_detail_antenna_eco_list_file` *file_name*	Specifies the file generated during process antenna repair that contains the list of diodes that were inserted. If you insert diodes during more than one detailed routing pass, the NanoRoute router creates a separate file for each pass and gives it a unique name. For example, if you use the default filename, the router creates the following files: • After the first detailed routing pass: `nano_ant_diode.list` • After the second detailed routing pass: `nano_ant_diode.list1` • After the third detailed routing pass: `nano_ant_diode.list2` *Default:* `""`(empty string) *Data_type:* string
`-route_detail_auto_stop {true | false}`	Controls whether NanoRoute continues routing if there are many violations. In highly congested designs, NanoRoute stops if the number of violations is too high. Specify false to continue routing until there is no improvement. **Note:** `-route_detail_auto_stop` is `false` by default for 20nm designs and below. **Note:** You should only override this setting when you are sure that NanoRoute will be able to resolve the violations. There are several reasons due to which NanoRoute may find the design unrouteable and stop: • Congestion: Review the congestion map to identify hot spots. If the majority of the violations are centered on these hot spots then reduce the congestion in this area by reducing the placement density using partial placement blockages or by modifying the floorplan. • Pin Access Issue: If majority of violations are on Metal1 and Metal2, it can indicate pin access problems on standard cell pins. Review the violations at these pins and determine why they are occurring. For example, check if the blockage is too close to the pin, pin is off the routing grid, or a proper via is not defined which can be dropped to the pin. • Pins under Stripes: If the violations are occurring when trying to route to standard cells placed under stripes, then use the `setPlaceMode -place_detail_preroute_as_obs` command to treat these stripes as placement blockages. *Default:* `true` *Data_type:* bool
`-route_detail_check_mar_on_cell_pin {true|false}`	Enables mar checking on used cell pins. *Default:* `false` *Data_type:* bool
`-route_detail_end_iteration` *pass_number*	Specifies the last pass in a detailed routing step. To run postroute optimization and repair remaining violations, including process antenna violations, set this parameter to its default value using the following command: `setNanoRouteMode -reset -route_detail_end_iteration` When set to its default value, the router dynamically decides when to stop. However, when you specify `-route_detail_end_iteration 0`, the end iteration will be at 0. **Note:** Specifying a higher value than 20 for `-route_detail_end_iteration` generally does not improve results. *Default:* `0` *Data_type:* int

`-route_detail_fix_antenna {true|false}`	Repairs process antenna violations by jumping layers (antennastapling). The router repairs process antenna violations if it can do so without creating DRC violations. The router might need to make several passes before repairing all antenna violations. Specify `-route_detail_fix_antenna true` to fix antenna violation on clock net during ccopt and routeDesign (clock eco). **Note**: You must set this option to true explicitly. The default value is true if you have antenna information defined in the LEF, but the router fixes antenna violations only on data nets during `routeDesign`, `globalDetailRoute`, or `detailRoute`. *Default:* `true` *Data_type:* bool
`-route_detail_fix_antenna_on_secondary_pg_nets {true|false}`	Fixes the antenna violation on secondary pg nets. *Default*: `false` *Data_type*: bool
`-route_detail_fix_antenna_with_gate_array_filler_mode {true|false}`	When enabled, the NanoRoute routers detects the GA filler flow and uses the GA filler diode cells for antenna violation fixing. *Default*: `false` *Data_type*: bool
`-route_detail_merge_abutting_cut {true|false}`	Allows you to enable or disable the capability to merge cell cut shapes. When enabled, it merges two abutting cell cut shapes into a new one. *Default:* `true` *Data_type:* bool
`-route_detail_min_length_for_spread_wire {`*`layer_1 length_1 ... layer_n length_n`*`}`	

	Specifies the minimum length in microns for a wire segment of a net before the necessity or a jog to avoid OPC issues. Spreading wires further apart (creating jogs in the wires) helps avoid design rule violations, signal integrity problems, and yield loss caused by shorts in the wires.
	This parameter supports the following values:
	• `layer`: Specifies the metal layer. The supported metal layer name is Mx, they are Metal1, Metal2, Metal3, etc.
	• `length`: Specifies the minimum length, in microns. The length value can be a floating number, default, or *skip*.
	  ◦ The default value is 0, it means wire spreading will determine the threshold automatically.
	  ◦ The *skip* value means there is no wire spreading on this layer.
	Note: You should specify the value for each metal layer. To get the number of metal layers used in the design, use db command (dbGet head.layers.name).
	Note: Turn wire spreading on or off with `-route_detail_post_route_spread_wire`
	For example, when you specify the following command, no wire spreading is done on layer Metal1. For layer Metal2, the wire spreading value is set to 0 signifying that wire spreading will determine the threshold automatically. Similarly, for layers Metal3, Metal4, Metal5, and Metal6 the wire spreading length is set to 2.0.
	`setNanoRouteMode -route_detail_min_length_for_spread_wire {Metal1 skip Metal2 0 Metal3 2.0 Metal4 2.0 Metal5 2.0 Metal6 2.0 }`
	Default: 0
	Data_type: string

`-route_detail_min_length_for_widen_wire` *value*	Specifies the minimum length in microns for a wire segment of a net before the segment is a candidate for widening. Wires are widened by an amount defined by a nondefault rule. They are not widened unless they meeting the following criteria: • Timing is not adversely affected • Routing resources are available • No overflow is created • No design rule or process antenna violations are created Widening wires helps avoid design rule violations, signal integrity problems, and yield loss. **Note**: To enable this parameter, you must also include a wide wire nondefault rule in the LEF file and specify the `-route_detail_post_route_wire_widen_rule true` parameter. *Default*: `1` *Data_type:* float
`-route_detail_min_slack_for_opt_wire` *value*	Specifies the minimum slack on pins associated with nets that are candidates for wire spreading or wire widening. Using this parameter reduces the timing impact of wire spreading, wire widening, and via swapping, and helps reduce congestion and the yield loss due to shorts and opens. **Note**: To enable this parameter, you must also specify `-route_with_timing_driven true`. Wire spreading and wire widening use the minimum slack value differently. While wire spreading only works for timing critical nets where the worst setup slack is less than the minimum slack value, wire widening only works for non-timing critical nets where the worst setup slack is greater than the minimum slack value. You can use the `-route_detail_min_slack_for_opt_wire` parameter to reduce the timing closure effort during post-route ECO. NanoRoute will replace via to multi-cut via on non-critical nets, if a violation does not occur. *Default*: `1` *Data_type:* float
`-route_detail_no_taper_in_layers` "*bottom_layer_number*`:`*top_layer_number*"	Prohibits the detailed router from tapering in the specified layer range, creating non-default-width wires in that range. *bottom_layer_number*`:`*top_layer_number* signifies the layer range format. *Default:* "" (empty string) *Data_type:* string
`-route_detail_no_taper_on_output_pin {true|false|auto}`	Prohibits the detailed router from tapering at standard cells, macro cells, and block output pins. **Note**: The default value of `-route_detail_no_taper_on_output_pin` is `auto` for 32nm and below nodes. *Default:* `false` *Data_type:* ternary
`-route_detail_on_grid_only none | false | true | all | {[wire [m1:mh]] | [via [m1:mh]]}`	

	Controls the routing of the objects, wires, and vias in relation to the user-defined routing tracks. • On grid wire means that the center of the wire is on the preferred routing track. • On grid via means that the center of the via is on the intersection of preferred routing tracks. **Note**: A preferred routing track is the track which has the same direction as that of the metal routing direction defined in the LAYER section in LEF. • When there are off-grid objects on fully-routed data, NanoRoute skips those nets and issues a warning message. • When there are fixed off-grid objects, NanoRoute skips those nets and issues a warning message. • When there are non-fixed off-grid objects, NanoRoute deletes those nets and issues a warning message. Specify one of the following arguments: • `all`: Specifies that all routing objects (vias and wires) have to be on the preferred and non-preferred routing tracks. It forces all routing objects to be on grid. • `none`: Specifies that the routing objects (vias and wires) can be off-grid. It does not force routing objects to be on grid . They can go off-grid depending on other restrictions. • `via [ml:mh]`: Specifies that the center of the vias must be at the intersection of preferred routing tracks. If `ml` and `mh` are specified, all the vias between `ml` and `mh` must be at the intersection of preferred routing tracks. For example, via [1:3] means all via12 must be at the intersection of preferred routing tracks of layer Metal1 and Metal2 and all via23 must be at the intersection of preferred routing tracks of layer Metal2 and Metal3. • `wire [ml:mh]`: Specifies that the center of the wire must be on preferred routing tracks. When a wire makes a turn on the non-preferred direction, the center of the wire might not fall on any routing tracks. If `ml` and `mh` are specified, all the center of the wires in between those metal layers `ml` and `mh` must be on the preferred routing tracks. For example, wire [1:3] means the center of the Metal1, Metal2, and Metal3 wires must be on the preferred routing tracks of Metal1, Metal2, and Metal3. • `false`: For backward compatibility purpose, the `-route_detail_on_grid_only` parameter also takes the value of `false`. This is equivalent to `'none'`. • `true`: Specifies that all wires have to be on the preferred and non-preferred routing tracks, but vias can be off the preferred and non-preferred routing tracks. **Note**: Block pins can be accessed by planar off-track wires. *Default:* `none` *Data_type:* string
`-route_detail_post_route_litho_repair {true|false}`	Corrects bridging, necking, and line-end shortening problems caused by lithography. Takes the severity of the problems into account and fixes the high-severity problems first. Prior to running the router with this parameter, import a hotspot interface format (HIF) file with `loadViolationReport`. *Default:* `false` *Data_type:* bool
`-route_detail_post_route_spread_wire {auto| true | false | 0 | 0.25 | 0.5 | 1 }`	

	Spreads routing, post route, by the percentage of the track. The router moves the wires without moving the vias. Specifying false for this parameter signifies no spreading, while specifying true signifies 0.5 / 1 track spreading by default for all nodes. The use of `true` and `false` is not needed as a value greater than `0` and less than equal to `1` (0<x=<1) activates the spreading. Specifying this parameter will not increase the overall routing run-time much but helps avoid yield loss caused by shorts and improve the timing as well. Specify one of the following: • `auto`: Spreading is on if the timing effort is set to high (`setDesignMode -flowEffort express`). Otherwise it is off. • `true`: 0.5 / 1 track spreading. Set `-route_detail_post_route_spread_wire` to `true` if you want to spread. • `false`: no spreading • `0.25`: 0.25 / 0.5 / 1 track spreading • `0.5`: 0.5 / 1 track spreading • `1`: 1 track spreading **Note:** To turn off this feature, set `setNanoRouteMode -route_detail_post_route_spread_wire` to `false` explicitly. To call wire spreading explicitly, set `-route_detail_post_route_spread_wire` to true and call `routeDesign -wireOpt`. Post route wire spreading automatically avoids fractional spreading for DPT designs. If `-route_detail_post_route_spread_wire` is enabled in 20nm designs and below, only 1 track wire spreading is done on DPT layers. *Default:* `auto` *Data_type:* string
`-route_detail_post_route_swap_via {true | multicut | false | none}`	Swaps single-cut vias for multiple-cut vias or reverses swapping on critical nets in a fully routed design, so that multiple-cut vias are swapped for single-cut vias on those nets. It swaps single-cut vias to multi-cut vias including vias of non-default nets. However, while swapping multi-cut vias to single cut vias, NanoRoute does not swap the vias of non-default rule nets. In timing-driven routing, does not swap vias if doing so would negatively impact timing. A net with a pin on a critical path is considered a critical net if the pin slack, plus the margin defined by the `-route_detail_min_slack_for_opt_wire` parameter, is less than or equal to 0. Use this parameter with the `routeDesign` command. You do not have to specify a value for this parameter when you run the following command, because the software does so automatically: `routeDesign -viaOpt` **Note:** This parameter is not enabled concurrently with postroute wire widening or wire spreading. In addition, the software does not perform via swapping if `-route_detail_post_route_litho_repair true` is specified. Specify one of the following: • `true`: Swaps all nets. The nets with attributes will get double cut and DFM precedence for others. Its weight can be specified with `-route_via_weight`. • `multicut`: All nets will be swapped with dcut precedence. • `none`: Does not swap vias. • `false`: Only nets with attributes are swapped. *Default:* `false` *Data_type:* string
`-route_detail_postroute_via_priority {auto | allNets | criticalNetsfirst | nonCriticalNetOnly}`	

	Sets the net priority for postRoute via swapping based on timing. • `auto`: Specifies the current behavior of the software, which is to detect the node and timing files to determine whether to do `criticalNetsFirst` or `nonCriticalNetOnly`. This is the default behavior. • `allNets`: Swaps single-cut vias for multiple-cut vias on all nets. • `criticalNetsFirst`: Specifies that swapping should be performed first on timing critical nets and then on other nets. This option is recommended for process nodes below 32nm. • `nonCriticalNetOnly`: Specifies that swapping should not be performed on timing critical nets. This option is recommended for 32 nm and above process nodes. *Default*: `auto` *Data_type:* string
`-route_detail_post_route_via_pillar_effort {none | low | medium | high}`	Specifies the effort levels for adding via pillars. Specify one of the following: • `none`: Adds via pillars only during routing that are required for EM constraints. • `low`: Adds via pillars during timing-driven global routing where they help in timing and do no impact congestion • `medium`: Adds via pillars more aggressively during routing. • `high`: Adds via pillars throughout routing to maximize via pillar impact on timing and area. *Default*: `low` *Data_type:* string
`-route_detail_post_route_wire_widen {widen | shrink | none}`	Applies the rule specified by the `-route_detail_post_route_wire_widen_rule` parameter to widen or shrink wires. Use this parameter with the `routeDesign` command. **Note:** This parameter is not enabled concurrently with postroute via swapping. **Note:** Wire widening is not performed on layer Metal1. Specify one of the following: • `widen`: During timing-driven routing, widens only non-critical nets; otherwise widens as many nets as possible that are covered by the non-default rule for widening wires. • `shrink`: During timing-driven routing, shrinks only critical nets; otherwise shrinks as many nets as possible that are covered by the non-default rule for widening wires. • `none`: Turns off this parameter, so the router does not widen or shrink wires. *Default:* `none` *Data_type:* string
`-route_detail_post_route_wire_widen_rule` *rule_name*	Specifies the non-default rule for widening wires. The rule is specified in the LEF file. Use this parameter with the `routeDesign` command. *Default:* "" (empty string) *Data_type:* string
`-route_detail_search_and_repair {true|false}`	Runs a search-and-repair step after the initial detailed routing (end iteration 20). **Tip:** Specify `false` for this parameter and run detailed routing to determine quickly if the design is routable. *Default:* `true` *Data_type:* bool

`-route_detail_signoff_effort {high | medium | low | auto | n}`	Specifies when to skip fixing violations related to fixed objects. The default value is `high`. In this mode, the software will ignore violations related to fixed objects after 20 detail route iterations and will continue with post-route operations. • When set to `medium`, the software will ignore any violations related to fixed objects after detail route iteration number 15. • When set to `low`, the software will ignore any violations related to fixed objects after detail route iteration number 5 and it will also skip post-route operations. • When set to `auto`, the software will automatically detect when to ignore violations related to fixed objects and it will also skip post-route operations. You can also specify the number of iterations or "n". When the iteration number is specified, the software will ignore violations related to fixed objects after the specified number of detail route iteration and will also skip post-route operations. **Note**: n must be between 3 and 20. *Default*: `high` *Data_type:* string
`-route_detail_stub_routing_in_first_layer {true|false}`	Specifies that minimum routing should be done on the first routing layer defined in the LEF. *Default:* false *Data_type:* bool
`-route_detail_use_multi_cut_via_effort {low | medium | high}`	Specifies the effort level toward increasing the ratio of double-cut vias to single-cut vias concurrently with routing. Increasing the effort level increases the double-cut via ratio and decreases the total number of vias in the design, which means that it also decreases the number of single-cut vias. The lower the number of single-cut vias, the better the yield will be. After routing with this parameter specified, you can add more double-cut vias or larger overhang vias by using the `routeDesign` command. Specify one of the following: • `low`: Specifies normal routing, so the router uses single-cut vias only. • `medium`: Balances the need for a high double-cut viaration with run time and congestion. Specifying this parameter increases the run time somewhat compared with the low parameter. The router inserts some double-cut vias, although not as many as if the high parameter were specified. • `high`: Specifies the highest yield possible for vias, as the router puts its best effort toward achieving the highest possible double-cut via ratio at the expense of run time and congestion. *Default:* `low` *Data_type:* string
`-route_diode_insertion_for_clock_nets {true|false}`	Inserts diodes to repair process antenna violations on clock nets that are in the regular net section of the DEF file. *Default:* `false` *Data_type*: bool

`-route_eco_ignore_existing_route` *value*	When specified, the selected net will be routed with highest priority by ignoring existing routes in post-route mode.
	• `all`: Ignores all other nets in the design that are not selected, including clock, secondary PG, and signal nets.
	• `signal`: Ignores only other signal nets that are not selected, but honors clock and secondary PG nets.
	For example, the following commands select and route a list of nets by ignoring only other signal nets, and considers secondary PG and clock nets during routing. After that, a second `ecoRoute` is run to fix DRC violations:
	`selectNet {net1 net2 net3}`
	`setNanoRouteMode -route_selected_net_only true`
	`setNanoRouteMode -route_eco_ignore_existing_route signal`
	`ecoRoute`
	`setNanoRouteMode -reset -route_selected_net_only`
	`setNanoRouteMode -reset -route_eco_ignore_existing_route`
	`verify_drc -limit 0`
	`ecoRoute`
	Default: " "
	Type: string

`-route_enable_route_rule_si_limit_length` *value*	Use MAR wire length as NDR PRL requirement for metal spacing check. *Default:* `false` *Data_type*: string
`-route_enforce_route_rule_on_special_net_wire {false | true |` *special_net_name_list*`}`	
	Specifies that non-default rule will be enforced on all special wire segments of specified nets. Specify one of the following parameters: • *`false`*: When specified, the non-default rule is not enforced on any wire segment of any special net. It is the default value. • *`true`*: When specified, the non-default rule is enforced on all wire segments of all special nets (with non default rules attached). • *`specified_special_net_list`*: When specified, the non-default rule is enforced on all wire segments of those specified special nets. *Default:* `false` *Data_type*: string
`-route_extra_via_enclosure` *distance*	Specifies an extra via enclosure to use when connecting to block pins and special wires. To center vias for via access, specify a value that is equal to one-half the width of the largest pin or wire. An extra enclosure of this size forces NanoRoute to connect tie-high and tie-low nets to the center of power routes. *Default:* `0` (microns) *Data_type*: float

`-route_fix_clock_nets {true|false}`	Specifies whether to maintain the current status of clock nets when the `routeDesign` command runs or change the status to ROUTED. • If you specify true for this parameter, the `routeDesign` command does not change the status of clock nets. • If you do not specify this parameter (or you specify false for this parameter), the `routeDesign` command changes the status of clock nets changes to ROUTED. **Note**: The settings of this parameter apply to the `routeDesign` command only. They do not impact the `globalRoute`, `detailRoute`, or `globalDetailRoute` commands. *Default:* false *Data_type*: bool
`-route_high_freq_constraint_groups {order net match bus pair shield}`	
	Routes only the specified constraint groups. The constraint groups can be - `net match bus pair shield`. **Note:** You can add "order" in front to control the route order. If order is present, the NanoRoute high frequency router will honor the order, if not, the router will determine the order. *Default: " "* *Data_type*: string
`-route_high_freq_match_report_file` *file_name*	Specifies the name of the match report file. *Default*: " " *Data_type*: string
`-route_high_freq_num_reserved_layers` *value*	Specifies the number of metal layers to reserve for high frequency routing above the standard cell area so that routing has more resources to access standard cell pins. *Default*: 1 *Data_type*: int
`-route_high_freq_remove_floating_shield {true|false}`	Removes floating shield segments for high frequency nets. *Default*: `false` *Data_type*: bool
`-route_high_freq_search_repair {auto | false | true | only}`	Runs search and repair to remove violations. *Default*: `auto` *Data_type*: string
`-route_high_freq_shield_trim_length` *value*	Specifies the shielding wire length. All shielding wires longer than the specified value are preserved by NanoRoute for high frequency nets. *Default*: 0 *Data_type*: float
`-route_honor_exclusive_region {true|false}`	When set to true, it enables the NanoRoute router to honor exclusive region routing. *Default:* `true`

`-route_honor_power_domain {true|false}`	Routes nets without crossing power domains. • If a net connects pins that all belong to one power domain, the software tries to route the net within that power domain. If completing the routing is not possible without breaking the rules, due to congestion for example, completes the routing and issues a warning message. • If a net connects pins that belong to different power domains, the software tries to route the net within the involved power domains, without crossing over into other power domains that do not include any of the pins in the net. If completing the routing is not possible without breaking this rule, completes the routing and issues a warning message. **Note:** You must specify a power domain before using this parameter. *Default:* `false` *Data_type*: bool
`-route_ignore_antenna_top_cell_pin {true|false}`	If true, ignores any process antenna data from external I/O pins (e.g. DEF PIN ANTENNA* data), so only the routing and antenna data from cells inside the design are used to find and repair local antenna violations. This is most commonly used with methodologies that insert diodes at the boundary pins (either inside the block or at the next level up), or in bottom-up flows where the block is routed, updated antenna data is created with the `verify_antenna` command, and then passed up to the top-level in an updated block abstract for the top-level to resolve any violations with added diodes or layer-hops. If false, the external IO antenna data is used to find and repair violations. This option will add protection diodes if needed to any external output/inout IOs that have antenna data that requires protection (e.g. have gate area), and will include the effects of external antenna data when finding and fixing local antenna violations. This is used when the user has “antenna budgets” added to the external IOs that estimate the worst-case top-level routing and gate-areas that needs protection inside this block. Currently these types of antenna budgets can only be added with defIn or oaIn. *Default:* `true` *Data_type*: bool
`-route_ignore_follow_pin_shapes {true|false}`	Ignores follow pin via shapes. *Default:* `false` *Data_type:* bool
`-route_interposer_allow_diagonal_trunk {auto | true | false}`	Allows you to control the trunk patterns in interposer routing. • `auto`: Allows the routing to decide the diagonal or straight trunk patterns based on the routing resources. • `true`: Enables diagonal trunk in interposer routing. • `false`: Disables diagonal trunk in interposer routing. *Default*: " " *Data_type*: string
`-route_interposer_interlayer_shielding_layers "`*`layer_name direction layer_name1:layer_name2 direction …`*`"`	

	Specifies the signal layers that require shielding along with the direction - top, bottom, both. By default, all routing layers (used by current routing command) are shielded from bottom. For example, the following command specifies that the M2/M3/M4 signal layers will be shielded from the top, M7 from the bottom, and M9 will be shielded from both sides: `setNanoRouteMode -route_interposer_interlayer_shielding_layers "M2:M4 top M7 bottom M9 both"` *Default*: " " *Data_type*: string
`-route_interposer_interlayer_shielding_nets "layer_name1:layer_nameN net_name …"`	
	Specifies the inter-layer shielding nets. For example, the following command specifies that the M3/M4/M5/M6 shielding net name is VSS and the M8/M9/M10 shielding net name is VDD: `setNanoRouteMode -route_interposer_interlayer_shielding_nets "M3:M6 VSS M8:M10 VDD"` *Default*: " " *Data_type*: string
`-route_interposer_interlayer_shielding_offsets "layer_name offset layer_name1:layer_name2 offset …"`	
	Specifies the center-to-center offset values between the shielding and signal wires. By default, all routing layers (used by current routing command) have an offset value of 0. For example, the following command specifies that the offset value is 1.1 um between the M1 shielding wire and signal wire and the offset value is 2.2 um between M2:M3 shielding wire and signal wire: `setNanoRouteMode -route_interposer_interlayer_shielding_offsets "M1 1.1 M2:M3 2.2"` *Default*: " " *Data_type*: string
`-route_interposer_interlayer_shielding_widths "layer_name width layer_name1:layer_name2 width …"`	
	Specifies the inter-layer shielding widths. By default, all routing layers (used by current routing command) are shielded by the default width wires. For example, the following command specifies that the shielding M1 wires width is 1.1 um, shielding M2/M3 wires width is 2.2 um: `setNanoRouteMode -route_interposer_interlayer_shielding_widths "M1 1.1 M2:M3 2.2"` *Default*: " " *Data_type*: string

`-route_interposer_same_layer_shielding_net net_name`	Specifies the same-layer shielding nets.
	For example, the following commands specifies that the M1/M3/M5 shielding net name is VSS:
	`setNanoRouteMode -route_interposer_trunk_routing_layers {M1 M3 M5}`
	`setNanoRouteMode -route_interposer_same_layer_shielding_net {VSS}`
	Default: " "
	Data_type: string

`-route_interposer_same_layer_shielding_width_spacing {width width}`	

	Specifies the same-layer shielding widths. For example, the following commands specify that the shielding width space between M1/M3/M5 is 0.6 and 1.2: `setNanoRouteMode -route_interposer_same_layer_shielding_width_spacing {0.6 1.2}` *Default*: " " *Data_type*: string
`-route_interposer_trunk_routing_layers` *value*	Specifies the trunk routing layers. For examples, the following command sets layers M1, M3, and M5 as the interposer trunk routing layers. `setNanoRouteMode -route_interposer_trunk_routing_layers {M1 M3 M5}` *Default*: " " *Data_type*: string
`-route_interposer_trunk_routing_width_spacing {`*width width*`}`	Specifies the trunk routing width spacing. For example, `setNanoRouteMode -route_interposer_trunk_routing_width_spacing {1.0 1.0}` *Default*: " " *Data_type*: string
`-route_number_fail_limit` *integer*	Specifies the maximum number of command fails allowed. In interactive mode, if a command fails you can continue with the next command. But in batch mode, if one command fails, the script stops at the failed command, however you can increase the fail limit to let it continue. *Default:* 1 *Data_type:* int
`-route_number_thread` *number_processors*	**Note:** This parameter is included for backward compatibility. Cadence recommends that you use the `setMultiCpuUsage` command to specify the number of threads in multiple-CPU processing. Specifies the maximum number of threads to use in multi-threading. Generally, using more threads decreases the run time. Specify this parameter before running any routing commands. There is no absolute limit to the number of threads the router can use, however, Cadence recommends that you do not specify more threads than the number of CPUs available. The router does not necessarily use the number of threads you specify – it determines the optimal number up to the value you specify. A small design generally uses fewer threads than a large design. *Default:* `1` *Data_type:* int **Note:** To enable a value greater than `1` for this parameter, you must have licenses available for the additional threads.
`-route_number_warning_limit` *integer*	Specifies the maximum number of warning messages for any one command reported to the log file. After reaching this number, the router continues processing, but does not report warnings for this command. *Default:* `20` *Data_type:* int

`-route_process_node` *value*	Specifies the process node. This parameter is an umbrella option to set the process related settings for routing. It supports the following values: • `S2`: Sets the Samsung 2nm process related settings. • `N2`: Sets the TSMC 2nm process related settings. • `N2X`: Sets the TSMC N2X process related settings. • `S3`: Sets the Samsung 3nm process related settings. • `N3`: Sets the TSMC 3nm process related settings. • `N3E`: Sets the TSMC N3E process related settings. • `S4` : Sets the Samsung 4nm process related settings. • `I5` : Sets the INTEL 5nm process related settings. • `N6`: Sets the TSMC 6nm process related settings • `C7`: Sets the SMIC 7nm process related settings. • `G7`: Sets the GlobalFoundries 7nm process related settings. • `I7` : Sets the INTEL 7nm process related settings. • `N7`: Sets the TSMC 7nm process related settings. • `S7`: Sets the Samsung 7nm process related settings. • `I10`: Sets the ICF 10nm process related settings. • `N10`: Sets the TSMC 10nm process related settings. • `SU10`: Sets the Samsung 10LPU process related settings. • `S11`: Sets the Samsung 11nm process related settings. • `G12`: Sets the GlobalFoundries 12nm process related settings. • `N12`: Sets the TSMC 12nm process related settings. • `G14`: Sets the GlobalFoundries 14nm process related settings. • `N22`: Sets the TSMC 22nm process related settings. **Note**: Some advance process nodes may have minimum width pins smaller than the default width defined in the LEF. For such cases, the NanoRoute Router automatically tapers from the default width to the min width pin. *Default:* " " *Data_type:* string
`-route_rc_extraction_corner` *value*	Specifies the RC extraction corner to use in routing. This helps in better resistance calculation. *Default:* " " *Data_type:* string
`-route_relaxed_route_rule_spacing_to_power_ground_nets {`*layer*`:`*spacing layer*`:`*spacing*`}`	

	Relaxes the spacing requirement from NDR spacing for the layers. *Layer_spacing* signifies a distance in microns (per layer) of how close any net with NDR can get to to the power/ground stripes.
	For example,
	`setNanoRouteMode -route_strict_honor_route_rule true`
	`setNanoRouteMode -route_relaxed_route_rule_spacing_to_power_ground_nets {metal1:0.04 metal2:0.04 metal3:0.04 metal4:0.04 metal5:0.04}`
	It only works for PG usage net and relaxes the NDR requirement to the specified spacing. The specified spacing should be greater than the `min_spacing` and less than the `hard_ndr_spacing`.
	Default: `none`
	Data_type: string

`-route_reserve_space_for_multi_cut {true|false}`	Reserves space to insert multicut vias in postroute stage. This option has to be set before routing. After routing with this parameter specified, you can add double-cut vias or larger overhang vias by using the `routeDesign -viaOpt` command. **Note:** `-route_reserve_space_for_multi_cut` is `true` by default for 32nm designs and below. *Default*: `false` *Data_type*: bool
`-route_reverse_direction {(lx ly ux uy bottom_layer:top_layer)}`	Reverses the preferred routing direction in the specified area or layer. You can specify one or more areas. **Note**: *bottom_layer* and *top_layer* can be either layer names or layer routing IDs. **Note**: Changing the routing direction can conflict with the power planning and pin access for macros and standard cells. It can also add congestion that may impact the runtime and result in violations. Cadence recommends that you use caution when using this parameter. *Default*: `" "` *Data_type*: string
`-route_route_clock_nets_first {true|false}`	Detects whether clock nets require ECO routing after the `routeDesign` super command checks whether the existing clock nets were fully or partially routed with the NanoRoute router and reroutes them before routing the remaining nets. If the existing clock nets were not routed with NanoRoute, `routeDesign` skips routing on those nets. *Default:* `true` *Data_type*: bool
`-route_selected_net_only {true|false}`	Specifies whether NanoRoute routes all nets at once or routes selected nets only. To route critical nets as short as possible, select the critical nets and set this parameter to `true`. NanoRoute does the following: • Removes incomplete nets unless they are marked `+ FIXED` or have a `-skip_routing` net attribute. • Routes the remaining selected nets. **Note:** To select a net, use the selectNet command or click the net in the design display window. *Default*: `false` *Data_type*: bool

`-route_shield_crosstie_offset {`*`layerName`*`:`*`numTrack1 layerName2`*`:`*`numTrack2`*`...}`	Specifies the offset in terms of number of tracks for adding crossties. It enables you to control the starting track for adding crossties. Default is 0 offset for all layers. *Default:* " " *Data_type*: string
`-route_shield_length_threshold` *value*	Specifies the minimum route segment length below which shielding is not required. *Default*: `-1` *Data_type*: float
`-route_shield_report_skip_status {true|false}`	Generates a report file containing a list of nets that have been skipped for shielding. *Default*: `false` *Data_type*: bool
`-route_shield_tap_cell_insertion {true|false}`	Specifies the tap cell names for shielding. *Default:* " " *Data_type*: string
`-route_shield_tap_cell_name` *string*	Specifies the tap cell names for shielding. *Default:* " " *Data_type*: string
`-route_skip_analog {true|false}`	Skips routing nets or pins marked `+ USE ANALOG` in the DEF file or where `signalType` modified to `analog` in the OA DB. Specify this parameter for NanoRoute to skip the routing of nets with Analog Signal Type. *Default:* `false` *Data_type:* bool
`-route_strictly_honor_1d_routing {true | false |` *bottomLayerNum*`:`*topLayerNum*`}`	When specified, the NanoRoute router strictly honors 1d routing. This enables you to turn off jog routing on a per layer basis. *Default*: `false` *Data_type*: string

`-route_strict_honor_route_rule {true | false | bottomLayerNum:topLayerNum | wire bottomLayerNum:topLayerNum}`	Uses nondefault spacing rules for nondefault wires. Note: The router tapers wires and uses default vias even when this parameter is set to `true`. You define nondefault routing rules in the `NONDEFAULTRULE` statement of the LEF file. Specify one of the following arguments: • `false`: Specifies that the nondefault rule (NDR) spacing is a soft rule. In congested areas NanoRoute may need to use the extra NDR spacing to meet the minimum foundry rule set. • `true`: Specifies that the nondefault rule spacing is a hard rule. NanoRoute will attempt to honor the requested NDR spacing irrespective of resource availability. • *bottomLayerNum*:*topLayerNum*: Specifies that the nondefault rule spacing is a hard rule for all the layers specified in the layer range. • `wire` *bottomLayerNum*:*topLayerNum*: Specifies that the nondefault rule spacing is a hard rule for the wire but not the via metal to enable better DFM via optimization. For example, • `setNanoRouteMode -route_strict_honor_route_rulee 3:6` Specifies that the NDR spacing will be used in layers Metal3, Metal4, Metal5, and Metal6 for wire and via metal. • `setNanoRouteMode -route_strict_honor_route_rule "wire 3:6"` Specifies that the NDR spacing will be used in Metal3, Metal4, Metal5, and Metal6 for wire and not via metal. **Note:** Setting the rule to `false` will attempt to honor the NDR rule and allow short hot spots without detouring. If you set the rule to `true` (hard rule) we recommend you work more on minimizing the congested areas to preserve the NDR topology. *Default:* `false` *Data_type*: string
`-route_stripe_layer_range "bottomLayerNum:topLayerNum"`	Specifies the range of routing layers for stripes. *Default:* `""` (empty string) *Data_type*: string
`-route_third_party_data {true|false}`	When enabled, NanoRoute supports third-party data as input. It accepts non-DEF connectivity models, multiple NDRs per net, and other third-party structures in order to enable ECO routing on non-native data. **Note:** Since the third-party data uses DEF data types for different purposes and is formed for a completely different use model, it is important to translate it into Cadence types. With proper setup Innovus accepts third-party data for ECO Routing. *Default:* `false` *Data_type:* bool

`-route_tieoff_to_shapes "auto | stripe | ring | followpin | powergroundpin"`	Specifies the target to special wire shapes or target instance pin for tie net connection. Specify one of the following arguments:
	• `auto`: Specify `auto` to target special wire shapes, including stripe, ring, and standard cell followpin.
	• `stripe`: Specify `stripe` to target stripes only. Stripes are the special wiring used for creating the power mesh.
	• `ring`: Specify `ring` to target only rings. The power mesh along the macro boundary forms a ring. If it is in a macro placed in the design, only exposes power ring-pin shapes. When power stripes are added across the macro, they connect to each side of the ring-pin but do not go inside the ring.
	• `followpin`: Specify `followpin` to target standard cell followpin only. Followpin is the special wiring used for connecting standard cells to power mesh.
	• `powergroundpin`: Specify `powergroundpin` to target the power and ground pin of the instances only.
	• `auto powergroundpin`: Specifies the target to special wire shapes or target instance pin for tie net.
	Note: The target shapes can be specified together. The value `auto` can only be specified with `powergroundpin`, since the other values stripe, ring, and followpin are included in auto.
	For example:
	• The following command signifies that it can tie to followpin or stripe.
	`-route_tieoff_to_shapes "followpin stripe"`
	• The following command signifies that it can tie to followpin or power ground pin
	`-route_tieoff_to_shapes "followpin powergroundpin"`
	• The following command signifies that it can tie to stripe or power ground pin
	`-route_tieoff_to_shapes "stripe powergroundpin"`
	Note: Using "`auto stripe`" is not recommended as `stripe` is included by auto.
	Default: `auto`
	Data_type: string

`-route_trim_pull_back_distance_from_boundary {layer:value ...}`	Specifies the boundary pullback value per layer for trim metal layers. The pullback value signifies how far the trim metal router brings signals and fill metal to a boundary (for example, block edge) in the absence of any obstructions. *Default*: " " *Data_type*: string
`-route_trunk_with_cluster_target_size integer`	Controls the routing pattern of nets with the trunk pattern attribute. By default, the router connects each pin in nets on which you have applied `setAttribute -pattern trunk` directly to the trunk of the special net; use this parameter to change the routing pattern for those nets: • Specify 0 to route nets with the trunk pattern specified with a `Steiner` tree pattern instead of a trunk pattern. • Specify an integer greater than 1 to route the nets with a fishbone pattern. In a fishbone pattern, several pins in a column are clustered together before connecting to the special net. The value of this parameter represents the approximate number of pins in each cluster. *Default*: 1 *Data_type*: int

`-route_use_auto_via` *value*	When specified, it allows the use of internal generated vias. In new advance nodes, it enables you to specify some custom via's names from the LEF for the router to use. This helps to provide better pin access.
	The following is the syntax of the supported values:
	`{true | false| auto | {-rule ruleName1 viaName viaName} {-rule ruleName2 viaName viaName} {-prefix_name viaPrefix }}`
	Specify one of the following arguments:
	• `true`: Specify true to turn on auto via generation.
	• `false`: Specify false to turn off auto via generation.
	• `auto`: Specify auto to enable via generation based on process nodes and allow Innovus to decide to turn on or off.
	• `-rule` *`ruleName1 viaName viaName`*: Specify -rule to enable Innovus to use the provided via list for the specified rule.
	• `-prefix_name`: Specify -prefix_name to add a prefix to the auto generated via names. By default, NR_ prefix is used by the tool.
	For example,
	• The following command signifies that M0 pins use VIA0_1 and VIA0_2 for default routing.
	`setNanoRouteMode -route_use_auto_via "-rule default VIA0_1 VIA0_2"`
	• The following command signifies that for nets with HV_NDR rule, use the specified list via's only for those layers. For other layers Innovus can choose other via's defined in the NDR rule section or it can auto generate. Only 5 via's per layer per rule can be defined:
	`setNanoRouteMode -route_use_auto_via {-rule HV_NDR HV_NDR_VIA4_2x1 HV_NDR_VIA5_2x1 HV_NDR_VIA6_2x1HV_NDR_VIA7_2x1}`
	Note: If you have two NDR rules for which you want to specify a list of via's then it should be specified as one string.
	`setNanoRouteMode -route_use_auto_via { {-rule ruleName viaName viaName} {-rule ruleName2 viaName2 } }`
	`routeDesign/globalDetailRoute`
	Note: The NanoRoute router treats VIA and USEVIA definitions as soft constraints when the `-routeUseAutoVia` parameter is enabled (default is enabled). The vias specified with VIA or USEVIA LEF rules are considered as target vias along with other auto-generated vias. To specify a hard constraint for the vias to use, use the USEVIACUTCLASS rule. Alternatively, specify the following:
	`setNanoRouteMode -route_use_auto_via -rule ndr_rule_name viaName`
	• The following command signifies that the NanoRoute router will use blockA as the prefix for auto generated vias names instead of NR_.
	`setNanoRouteMode -route_use_auto_via "-prefix_name blockA"`
	Default: `auto`
	Data_type: string

`-route_via_weight{`*`via_name via_weight ...`*`}`	Specifies the priority or the order of vias for post route via swap to use. It sets the via weight for the specified via. The bigger the number, the higher priority the via is given. A negative value means the via will not be used.
	You can use wildcards for the *via_name*. For example:
	`setNanoRouteMode -route_via_weight {*FBD* 8, *FBS* 7, *PBD* 6, *PBS* 5, *2cut_p1* 4, *2cut_p2* 3, *2cut_p3* 2, *FAT* 1}`
	Note: Via name can be a predefined class of vias such as @1cut and @2cut. Here, @1cut stands for single cut vias, @2cut stands for double cut vias.
	In case of duplicated value setting of via, the latest value will be used. For example:
	`setNanoRouteMode -route_via_weight {@1cut 2 @2cut 4 VIA12_1cut 1}`
	Here, VIA12_1cut was first set weight to 2, and then set to 1, the final weight of VIA12_1cut is 1.
	When you specify this option to do post route via swap, it's important to set the related options to improve the DFM and double cut via ratio.
	• Specify `-route_reserve_space_for_multi_cut true` before detail route to reserve space for inserting multicut vias in postroute via swap stage.
	• Specify `-route_with_timing_driven false` and `-route_detail_post_route_swap_via multiCut` to swap the single cut via to DFM and double cut via as much as possible.
	Note: If you have two NDR rules for which you want to specify a list of via's then it should be specified as one string.
	`setNanoRouteMode -route_use_auto_via { {-rule rule_name via_name via_name} {-ruleruleName2 viaName2 } }`
	`routeDesign/globalDetailRoute`
	Note: The NanoRoute router treats VIA and USEVIA definitions as soft constraints when the `-routeUseAutoVia` parameter is enabled (default is enabled). The vias specified with VIA or USEVIA LEF rules are considered as target vias along with other auto-generated vias. To specify a hard constraint for the vias to use, use the USEVIACUTCLASS rule. Alternatively, specify the following:
	`setNanoRouteMode -route_use_auto_via -rule ndr_rule_name viaName`
	The following command signifies that the NanoRoute router will use blockA as the prefix for auto generated vias names instead of NR_.
	`setNanoRouteMode -route_use_auto_via "-prefix_name blockA"`
	Default: `auto`
	Data_type: string
	Range: `-1 to 256`
	Default: `""`(empty string)
	Data_type: string

`-route_with_eco {true|false}`	Routes in engineering change order (ECO) mode if `globalDetailRoute` is specified. During ECO mode, NanoRoute completes partial routes with added logic, while maintaining the existing wire segments as much as possible. **Note:** The Innovus software also has an `ecoRoute` command. *Default*: false *Data_type*: bool
`-route_with_litho_driven {true|false}`	Avoids lithography problems during routing by avoiding certain routing patterns that might lead to the creation of lithography hotspots. *Default*: false *Data_type*: bool

`-route_with_si_driven {true|false}`	Prevents or reduces crosstalk. Works in conjunction with timing-driven routing. When timing-driven routing is specified, uses SMART routing to identify victim nets and minimize crosstalk by wire spacing, layer hopping, net ordering, and minimizing the use of long parallel wires. When timing-driven routing is not specified, uses an older signal integrity engine to minimize crosstalk by preventing or reducing the use of long parallel wires. Specify `-route_with_timing_driven` and `-route_with_si_driven` to run SMART routing. *Default*: false *Data_type*: bool
`-route_with_timing_driven {true|false}`	Minimizes timing violations by analyzing the timing slack for each path, the drive strengths of each cell in the library, and the maximum capacitance and maximum transition limits. During timing-driven routing, NanoRoute routes multi-pin nets to the most critical sink first, performs wire optimization by reducing resistance and coupling, and continually adjusts detouring. When this parameter is set to true, the router automatically generates a timing information file named .timingfile.tif in the working directory. It uses this file when it runs timing driven routing. **Note**: Specify the following commands and to skip the CTE call and timing file generation step and use the specified file: `writeDesignTiming` *file_name* `setNanoRouteMode -route_with_timing_driven true` Specify `-route_with_timing_driven` and `-route_with_si_driven` to run SMART routing. You can use the `-route_with_timing_driven` parameter to reduce the timing closure effort during post-route ECO. NanoRoute will replace via to multi-cut via on non-critical nets, if a violation does not occur. *Default*: false *Data_type*: bool

`-route_with_trim_metal` *value*	Specifies the preferred trim metal grid for NanoRoute Router to use. You can specify the value in the following format:
	
	`{-layer layerNumber -mask1 {-pitch value -core_offset value} -mask1 { -pitch value ... } -mask2 { -pitch value ... } -mask2 {-pitch value ... }}`
	
	For example, the following command specifies the preferred trim metal grid on layer 2.
	
	`SetNanoRouteMode -route_with_trim_metal {-layer 2 -mask1 {-pitch 0.300 -core_offset 0.025 } \`
	`-mask1 {-pitch 0.300 -core_offset 0.195 } \`
	`-mask2 {-pitch 0.300 -core_offset 0.065 } \`
	`-mask2 {-pitch 0.300 -core_offset 0.235 }}`
	
	Note: The width value is taken from the LEF.
	
	Default: " "
	
	Data_type: string

`-route_with_via_in_pin {true| false | bottomLayerNum:topLayerNum}`	Encloses via geometries completely inside standard cell pins. Specify true for this parameter to avoid MINSTEP violations for standard cell pin access. Default: false
	Note: To avoid violations, specify the same value for this parameter throughout the entire design flow.
	• `true`: Enclose vias inside standard cell pins on all layers.
	• `false`: The router is not required to enclose vias inside standard cell pins.
	• `bottomLayerNum:topLayerNum`: Enclose vias inside standard cell pins on layers in the specified layer range.
	  ◦ `1:1`: Specifies that Metal1 pins must enclose vias.
	  ◦ `2:5`: Specifies that pins on Metal2, Metal3, Metal4, and Metal5 must enclose vias. Pins on other layers are not required to enclose vias.
	Default: false
	Data_type: string

`-route_with_via_only_for_block_cell_pin value`	Encloses via in pin shape for macro cell pins. *Default*: false *Data_type*: string

`-route_with_via_only_for_stdcell_pin {true| false | bottomLayerNum:topLayerNum}`	Allows standard cell pins to be accessed by vias only. Prohibits planar access. Default: false
	• `true`: Always use vias for standard cell pin access on all layers.
	• `false`: Use vias or planar access for standard cell pins on all layers.
	• *`bottomLayerNum:topLayerNum`*: Always use vias for pin access in the specified layer range.
	  ◦ `1:1`: Always use vias for access to Metal1 standard cell pins. Metal2 and above can use vias or planar access.
	  ◦ `2:5`: Always use vias for access to standard cell pins on Metal2, Metal3, Metal4, and Metal5. Metal1 and Metal6 (and above) can use vias or planar access.
	Note: Set this option before any routing or verification because it affects the pin access preparation steps. In addition, after you set this option, the access remains for same the entire Innovus session. Do not change the setting, even for future routing sessions that load saved data from the database or a saved DEF file.
	For example, if a design with pins on Metal1 and Metal2 is routed with this option set to 1:1, the router uses vias for access to all standard cell pins on Metal1, and vias or planar access to standard cell pins on Metal2. If you save the design after routing, then import the design in a different Innovus session and route it again with this option set to true (that is, forcing the router to use vias to access all standard cell pins on all layers), you might have planar connections (from the previous routing session) to pins on Metal2. All the planar connections to those pins become violations, because they are not allowed when this option is set to true.
	Default: false
	Data_type: string

Examples

- The following commands show the steps in basic routing strategy with the NanoRoute router.
  The router globally routes the design:
  ```
  globalRoute
  ```
- The router does the initial detailed routing (iteration 0 does not include a search-and-repair step), and saves the design as `droute0`:
  ```
  setNanoRouteMode -route_detail_end_iteration 0
  detailRoute
  saveDesign droute0
  ```
- The router does the first search-and-repair iteration, and saves the design for analysis:
  ```
  setNanoRouteMode -route_detail_end_iteration 1
  detailRoute
  saveDesign droute1
  ```
- The router does the second to nineteenth search-and-repair iterations, and saves the design for analysis. The switch box grows larger as the iteration number increases.
  ```
  setNanoRouteMode -route_detail_end_iteration 19
  detailRoute
  saveDesign droute19
  ```
- The router runs postroute optimization (-`route_detail_end_iteration default`) and additional search-and-repair operations, and saves the design as `droute`:
  ```
  setNanoRouteMode -route_detail_end_iteration default
  detailRoute
  saveDesign droute
  ```
- The following commands route selected nets, then route the remaining nets in the design:
  ```
  setNanoRouteMode -route_selected_net_only true
  globalDetailRoute
  setNanoRouteMode -route_selected_net_only false
  globalDetailRoute
  ```
- The following commands route a placed design in timing-driven mode and output a routed DEF file:
  ```
  restoreDesign
  setNanoRouteMode -route_with_timing_driven true
  saveDesign init
  globalDetailRoute
  saveDesign droute
  defOut -floorplan -routing droute.def
  ```
- The following command resets the `-route_selected_net_only` parameter to its default value:
  ```
  setNanoRouteMode -reset -route_selected_net_only
  ```
- The following command resets all `setNanoRouteMode` parameters to their default values:
  ```
  setNanoRouteMode -reset
  ```
- The following command sets the weight of via12 o 1 and via23 to 2:
  ```
  setNanoRouteMode -route_via_weight {via12 1 via23 2}
  ```
- The following command resets the weight of via12 to 3:
  ```
  setNanoRouteMode -route_via_weight {via12 3 via34 2}
  ```

Related Information

- `detailRoute`
- `getNanoRouteMode`
- `globalRoute`
- `globalDetailRoute`
- `routeDesign`
- `setAttribute`
- Using the NanoRoute Router chapter in the *Innovus User Guide*

setRouteMode

```
setRouteMode
[-help]
[-reset]
[-earlyGlobalEffortLevel {standard | medium | low }]
[-earlyGlobalHonorMsvRouteConstraint {true|false}]
[-earlyGlobalNumTracksPerClockWire value]
[-earlyGlobalReverseDirection "( x1 y1 x2 y2 ) Metal2:Metal2 ( x3 y3 x4 y4 ) Metal3:Metal3 ... "]
[-earlyGlobalRouteBumpNets {true|false}]
[-earlyGlobalRoutePartitionAllowFeedthru list_of_ptn_cell_names]
[-earlyGlobalRoutePartitionHonorFence list_of_ptn_cell_names]
[-earlyGlobalRoutePartitionHonorPin list_of_ptn_cell_names]
[-earlyGlobalRoutePartitionPinGuide {true|false}]
[-earlyGlobalRouteSecondPG {true|false}]
[-earlyGlobalRouteSelectedNetsOnly {true|false}]
[-earlyGlobalRouteStripeLayerRange {layerIdx1:layerIdx2}]
[-earlyGlobalSecondPGMaxFanout integer]
```

Sets global parameters for the Early Global Route or `earlyGlobalRoute`. The parameters that you specify with `setRouteMode` are then used automatically whenever you run the Early Global Router through the `earlyGlobalRoute` command or internally during in-place optimization, partitioning, or placement.

The `setRouteMode` command parameter settings affect the results of the Early Global Router, which is run in all the steps during the preRoute flow. Consequently, with changes in these settings, the results of `place_opt_design`, `ccopt_design`, and postCTS will change.

Note: If not required, it is best not to change the tool technology settings.

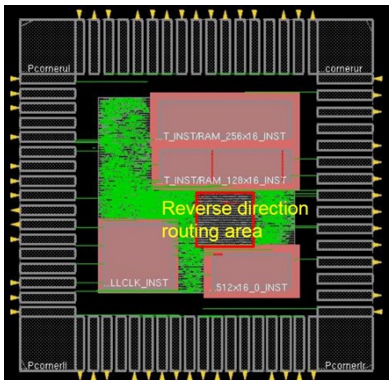
Use the `getRouteMode` command to return the current settings for the `setRouteMode` command.

Early Global Route supports nested partitions. The partitions specified with the `-earlyGlobalRoutePartitionHonorFence` and the `-earlyGlobalRoutePartitionHonorPin` parameters are routed in the style that honors fence and pin constraints. All other partitions are routed

Parameters

`-earlyGlobalEffortLevel {standard| medium | low }`	
	Specifies the congestion effort level. **Note:** The `low` effort level value will reduce the runtime while `standard` effort level will increase the congestion accuracy • `standard`: Congestion is more accurate and runtime is larger. • `medium`: Runtime will be much smaller and congestion report will have less accuracy. • low: Runtime is faster than `medium` but less accurate. *Default*: `standard` **Note:** While the `setRouteMode -earlyGlobalEffortLevel` parameter is mainly used for speed (low and medium will make it faster but less accurate), the `setDesignMode -congEffort` parameter is used to increase optimization effort to resolve congestion. Even though these parameters are not related directly, changing the Early Global Route effort level (decreasing it) will change the results for the required congestion effort. Hence, if you set the `setDesignMode -congEffort` as high, it is recommended that you keep the `setRouteMode -earlyGlobalEffortLevel` parameter at standard (default) level. *Data_type*: enum, optional
`-help`	Outputs a brief description that includes type and default information for each `setRouteMode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setRouteMode`.
`-earlyGlobalHonorMsvRouteConstraint {true | false}`	

	Honors MSV domain constraints. This option is used to decide whether Early Global Route should honor power domain settings. By default, Early Global Route ignores the power domain settings and routes as flat design. When enabled, Early Global Route honors power domain settings and route wires inside preferred power domain as much as possible. *Default* : false *Data_type*: bool, optional
`-earlyGlobalNumTracksPerClockWire value`	Specifies the special number of tracks per clock wires. *Max*: `8` *Min*: 0 *Default*: `0` *Data_type*: int, optional
`-reset`	Restores all parameters modified using `setRouteMode` to their default settings. The `-reset` parameter must be the first parameter specified. If you specify -reset by itself, the software resets all `setRouteMode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported.
`-earlyGlobalReverseDirection "( x1 y1 x2 y2 ) Metal2:Metal2 ( x3 y3 x4 y4 ) Metal3:Metal3 ... "`	

	Reverses the routing direction in the given region on the specified layer-range. It specifies an area in which the Early Global Router routes wires in the non-preferred direction.
	Note: You can define the reverse direction routing area by *"(x1 y1 x2 y2)"* and routing layer by *"Metal2:Metal2"*. In release 16.1, the Early Global Router only supports a single layer with this option. Consequently, *"Metal2:Metal3"* is not supported.
	The `-earlyGlobalReverseDirection` parameter only has impact on the congestion value calculation. By routing on reverse direction, you can reduce the routing capacity on the specified layer and add extract resource on the above and below layers. Innovus does not display the routing wires in reverse direction routing region.
	For example, the following command reverses the routing direction in the specified region on layer *Metal3*.
	`setRouteMode -earlyGlobalReverseDirection "(757.996 542.697 995.323 772.5685) Metal3:Metal3"`
	 
	The congestion value without reverse direction routing:
	`Overflow after earlyGlobalRoute 0.02% H + 0.00% V`
	The congestion value with reverse direction routing:
	`earlyGlobalRoute overflow: 0.93% H + 0.10% V`
	Default: ""
	Data_type: string, optional

`-earlyGlobalRouteBumpNets {true|false}`	
	Specifies that the Early Global Router will honor a net with a connecting bump and enable bump net routing. By default, Early Global Route ignores bump nets as they are marked as special in the database. When enabled, the Early Global Route honors bump nets and routes them. *Default*: false *Data_type*: bool, optional
`-earlyGlobalRoutePartitionAllowFeedthru` *list_of_ptn_cell_names*	
	Specifies the partition cell names for which the Early Global Router honors partition fences. *Default* : "" *Data_type*: string, optional
`-earlyGlobalRoutePartitionHonorFence` *list_of_ptn_cell_names*	

	Honors partition fences with a single-entry constraint and also honors fixed pins. Under the single-entry constraint once a hierarchical route enters a partition fence it does not cross the boundary again. The `-earlyGlobalRoutePartitionHonorFence` parameter routes a net belonging to more than one partition so that the routing does not violate partition boundaries, in addition to routing intra-partition and global nets using the specifications of the partitions. For example, if a net belongs to partitions P1 and P3, but not P2, the net is routed without going over the P2 partition boundary. **Notes:** • For flat partitioned designs, you can specify `-earlyGlobalRoutePartitionHonorFence` to simulate channel-based routing. For channelless designs, you must perform feedthrough insertion before using this parameter. • Supports designs with up to 180 partitions. • Early Global Route supports nested partitions. The partitions specified with the `-earlyGlobalRoutePartitionHonorFence` and the `-earlyGlobalRoutePartitionHonorPin` parameters are routed in the style that honors fence and pin constraints. All other partitions are routed flat. *Default* : "" *Data_type*: string, optional
`-earlyGlobalRoutePartitionHonorPin` *list_of_ptn_cell_names*	
	Honors partition fences with single-entry constraint and pre-assigned pins (pins marked `FIXED`) and assigned pins (pins marked `PLACED`). *Default*: "" *Data_type*: string, optional
`-earlyGlobalRoutePartitionPinGuide {true | false}`	
	Honors partition pin guides. It uses the pin guide statements in the floorplan file to guide the routing through partition pin points. **Note:** Fixed pins and pin guides are only honored when `-earlyGlobalRoutePartitionPinGuide` is `true`. *Default*: true *Data_type*: bool, optional
`-earlyGlobalRouteSecondPG {true | false}`	
	Specifies whether the secondary PG pin should be routed or not. When enabled, it routes the pin specified by the `setPGPinUseSignalRoute` command. *Default*: `false` *Data_type*: bool, optional
`-earlyGlobalRouteSelectedNetsOnly {true | false}`	
	Specifies that only the nets that are selected in the database are to be routed. *Default*: `false` *Data_type*: bool, optional
`-earlyGlobalRouteStripeLayerRange` { *layerIdx1*:*layerIdx2* }	
	Specifies the stripe layer range that is routed to the secondary PG pin. *Default*: `" "` *Data_type*: string, optional
`-earlyGlobalSecondPGMaxFanout` *integer*	

	Specifies the maximum fanout value during secondary PG pin routing. *Max*: `INT_MAX` *Min*: `1` *Default*: `2147483647` *Data_type*: int, optional

Example

- The following command honors partition fences with single entry and does not allow crossing other partitions, when `earlyGlobalRoute` is run:
  ```
  setRouteMode -earlyGlobalRoutePartitionHonorFence {.}
  earlyGlobalRoute
  ```
- The following command honors partition fences with single entry and also honors placed pins, when `earlyGlobalRoute` is run:
  ```
  setRouteMode -earlyGlobalRoutePartitionHonorPin {.}
  earlyGlobalRoute
  ```
- The following command resets all `setRouteMode` parameters to their default values:
  ```
  setRouteMode -reset
  ```

Related Information

- `getRouteMode`
- `earlyGlobalRoute`
- Using Early Global Route for Congestion and Timing Analysis chapter in the *Innovus User Guide*

writeHif

```
writeHif
[-help]
-file filename
```

Writes out a HIF file after the NanoRoute router fixes lithography problems. The file reports the areas where the router made changes. Use this command with `loadViolationReport` for incremental lithography checking and fixing.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `writeHif` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man writeHif`.
`-file filename`	Specifies the HIF file to write out. *Data_type*: string, required

Example

The following example writes the HIF file `result.hif` marking the areas in which the NanoRoute router has made changes to fix lithography problems:

```
writeHif -file result.hif
```

Related Information

- `loadViolationReport`
- Using Early Global Route for Congestion and Timing Analysis chapter in the *Innovus User Guide*
- Using the NanoRoute Router chapter in the *Innovus User Guide*

route_fix_ir

```
route_fix_ir
[-help]
[-concurrent_pg_and_signal {0 1}]
-ground_net string
[-hotspot_boxes {{lx1 ly1 ux1 uy1} {lx2 ly2 ux2 uy2}...}]
[-irdrop_threshold float]
-power_net string
[-rail_path string]
-top_layer string
```

Fixes IR Drop (voltage drop) violations with the NanoRoute engine. It creates additional PG connection paths from top layer PG stripes down to low layer PG rail shapes based on rail analysis results. This method also makes it possible to concurrently route signal nets and IR drop fixing PG nets. It has congestion driven and timing aware capabilities.

You can use the `route_fix_ir` command after postroute.

Parameters

`-concurrent_pg_and_signal {0 1}`	When set to 1, the NanoRoute router routes the signal nets and pg nets for IR drop fixing together. When set to 0, the NanoRoute router routes the PG net wires only and does not touch the signal nets. *Default:* 0 *Data_type*: enum, optional
`-ground_net string`	Specifies the ground net name for IR drop fixing. *Data_type*: string, required
`-help`	Outputs a brief description that includes the type and default information for each `route_fix_ir` parameter. For a detailed description of the command and all of its parameters, use the man command `man route_fix_ir`.
`-hotspot_boxes {{lx1 ly1 ux1 uy1} {lx2 ly2 ux2 uy2}...}`	
	Specifies coordinates of the hotspot boxes. **Note**: When this option is specified, the `-rail_path` option is ignored. *Data_type*: box_list, optional
`-irdrop_threshold float`	Specifies the minimum allowed voltage IR drop for the domain being analyzed. **Note**: This option works with the `-rail_path` option. *Data_type*: float, optional
`-power_net string`	Specifies the power net name for IR drop fixing. *Data_type*: string, required
`-rail_path string`	Specifies the path of the rail analysis results. The NanoRoute router then calls the debug IR drop functionality to create hotspot files. **Note**: If the rail analysis path directory is not specified, the NanoRoute router automatically calls the power analysis, rail analysis, and debug IR drop functionalities to create hotspot files. **Note**: This option works with the `-irdrop_threshold` option. *Data_type*: string, optional
`-top_layer string`	Specifies the top stripe layer to connect the PG nets for IR drop fixing. *Data_type*: string, required

Examples

- The following command uses the rail analysis results for IR drop fixing.
  ```
  route_fix_ir  -rail_path rail_static/ALL_85C_avg_1 -top_layer M15 -irdrop_threshold 0.08 -power_net VDD -ground_net VSS
  ```
- The following command specifies the hotspot boxes.
  ```
  route_fix_ir -hotspot_boxes {{box1} {box2} …} -top_layer M15 -power_net VDD -ground_net VSS
  ```

Related Information

- analyze_rail
- debug_irdrop
- report_power
- Using the NanoRoute Router chapter in the *Innovus User Guide*

Route Global Variables

- trPrintIgnoredPadNets

trPrintIgnoredPadNets

```
trPrintIgnoredPadNets value
```

Type: Integer

Default: 0

Sets a limit to the number of nets to be printed during earlyGlobalRoute. This limit is used to warn about the nets that have connectivity from a pad term to a `fterm` without geometry. Without a geometry on the terminal these nets can not be routed. A pad term is a pin of a pad logically connected to a top level module port (fterm).

Type 'set trPrintIgnoredPadNets ' prior to earlyGlobalRoute to have it report each net up to limit value.

The default value of this integer variable is 0

Related Commands

- `earlyGlobalRoute`

34

Signal ElectroMigration Commands

- get_signal_em_analysis_mode
- set_signal_em_analysis_mode

get_signal_em_analysis_mode

```
get_signal_em_analysis_mode
[-help]
[-Ipeak_Td_method]
[-avgRecovery]
[-default_freq_for_unconstrained_nets]
[-delta_T]
[-effort_level]
[-em_temperature]
[-em_res_width]
[-error]
[-forceHoldView]
[-lifetime]
[-method]
[-minPeakDutyRatio]
[-minPeakFreq]
[-net_file]
[-toggle]
[-useQrcTech]
[-use_db_freq]
[-view ]
[-net | -selected ]
[-report  [-detailed]]
[-report_db]
[-set_current_file]
[-skip_net]
[-skip_net_file]
[-ict_em_models]
[-current_scale_factor]
[-em_limit_scale_factor]
[-em_threshold]
[-current_scale_table]
[-em_limit_scale_table]
[-top_scope_ignore_block_internal_nets_on_boundary_path]
[-extraction_tech_file]
[-handle_pin_obs_via]
```

Displays the following information about a `set_signal_em_analysis_mode` parameter in the system log file and in the system console:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the software displays information for all of the `set_signal_em_analysis_mode` parameters.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_signal_em_analysis_mode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_signal_em_analysis_mode`.
`parameter_names`	Displays information for the specified parameters. You can specify one or more parameters. See `set_signal_em_analysis_mode` for description of the parameters you can specify.
`-quiet`	Displays the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software displays the current settings of all `set_signal_em_analysis_mode` parameters in the Tcl list format.

Examples

The following command displays the current setting of the `-use_db_freq` parameter:

```
get_signal_em_analysis_mode -use_db_freq
```

The software displays the following information:

```
[DEV]innovus 8> get_signal_em_analysis_mode -use_db_freq
-use_db_freq false                      # bool, default=false
false
```

Related Information

- `set_signal_em_analysis_mode`
- "Signal ElectroMigration" in the *Voltus IC Power Integrity Solution User Guide*

set_signal_em_analysis_mode

```
set_signal_em_analysis_mode
[-help]
[-Ipeak_Td_method {effective_width_from_integration effective_half_peak_width max_equivalent_dc_peak sum_half_peak_width}]
[-avgRecovery em_recover]
[-default_freq_for_unconstrained_nets freq_in_Hz]
[-delta_T value]
[-effort_level {low | medium | high}]
[-em_temperature value]
[-em_res_width {drawn silicon}]
[-error value]
[-forceHoldView]
[-lifetime value]
[-method {rms peak avg}]
[-minPeakDutyRatio value]
[-minPeakFreq value]
[-net_file list_file]
[-toggle value]
[-useQrcTech]
[-view viewName]
[-net netNames | -selected ]
[-report filename [-detailed]]
[-report_db]
[-set_current_file filename]
[-skip_net net_name_list]
[-skip_net_file list_file]
[-ict_em_models file]
[-current_scale_factor {{avg value} {rms value} {peak value}}]
[-em_limit_scale_factor {{avg value} {rms value} {peak value}}]
[-em_threshold <value>]
[-current_scale_table current_scale_table_file]
[-em_limit_scale_table em_limit_scale_table_file]
[-extraction_tech_file filename]
[-handle_pin_obs_via]
[-read_detail_delta_temperature_file filename]
[-reset]
[-check_thermal_aware_em {true | false}]
[-enable_seb {true | false}]
[-env_temperature temperature]
[-seb_lifetime value]
[-seb_table table_filename]
[-seb_temperature temperature]
[-use_rms_delta_t {true | false}]
[-skip_category_mode {0 | 1 | 2}]
[-em_temperature_layer_list {{<LEF_layer_name> <em_temperature_in_C>}+}]
[-delta_T_layer_list {{<LEF_layer_name> <delta_T_in_C>}+}]
[-em_cdf_percentage value]
[-additional_individual_violation_report]
[-net_freq_file filename]
[-ircx_models {RC.ircx EMIR.ircx}|{RC.ircx}]
[-simulation_net list_of_nets]
[-simulation_net_file filename]
```

Sets global parameters for signal electromigration. Use this command to specify the required `verify_AC_limit` mode settings before running any commands that perform signal electromigration. The parameters specified with the `set_signal_em_analysis_mode` command are then used automatically whenever signal electromigration takes place.

Use the `get_signal_em_analysis_mode` command to return the current settings for the `set_signal_em_analysis_mode` command.

Note: Use the `-method` parameter to specify the waveform calculation method to be used (`rms`, `peak`, or `avg`).

Parameters

`-help`

Outputs a brief description that includes type and default information for each `set_signal_em_analysis_mode` parameter.

For a detailed description of the command and all of its parameters, use the `man` command:
`man set_signal_em_analysis_mode`

`-additional_individual_violation_report`

Specifies to generate separate violation reports for each current waveform calculation method (`rms`, `peak`, or `avg`). If this parameter is not speci violation report file is generated using the `-method {RMS AVG peak}` parameter. This parameter gives you the flexibility of generating either a s report or individual reports for each analysis method.

Example:

```
set_signal_em_analysis_mode \
-method {rms peak avg} \
-report context1.rpt \
...
-additional_individual_violation_report
```

This command will generate 3 different reports, namely `context1.rpt_avg`, `context1.rpt_peak`, and `context1.rpt_rms`. If there are no EM viol method, the tool will not generate the corresponding separate report. In this example, if there are violations for `avg` and `rms` but none for `peak`, t only two reports, `context1.rpt_avg` and `context1.rpt_rms`.

`-avgRecovery` *`em_recover`*

Specifies the recovery factor used for calculating I_{avg} current density with recovery. By default, the recovery factor (per layer) is read from the C you specify `-avgRecovery em_recover`, it overrides the Quantus tech `em_recover` factor for all layers used in I_{avg} limits. The minimum `-avgRec` and the maximum is `1`.

Note: If you do not specify `-avgRecovery` and the `em_recover` factor is not specified in the Quantus techfile as well, the tool gives an error that computed.

`-check_thermal_aware_em {true | false}`

Specifies to use the thermal map for EM checks. The thermal map file is specified using the `-read_thermal_map` parameter. The Rail Analysis temperature specified in the thermal map file for EM analysis. EM limit has a strong temperature dependency. For a chip that has substantial t variation across the chip, the Rail Analysis engine can use the temperature specified in the thermal map file to perform accurate EM analysis.

Default : `false`

`-current_scale_factor {{avg` *`value`*`} {rms` *`value`*`} {peak` *`value`*`}}`

Scales the signal EM current by the respective factor for avg, rms, or peak current specified by `{avg` *`value`*`} {rms` *`value`*`} {peak` *`value`*`}`.

The value specified for this parameter must be > 0 and <= 10.

`-current_scale_table` *`current_scale_table_file`*

Specifies the name of the scale table file used for layer-based scaling of the signal EM current. The scale table contains the scale factor for av current for each layer.

This parameter is mutually exclusive with `-current_scale_factor`. The `-current_scale_table` parameter takes precedence over `-current_sc`

The format of the table is given below:

```
layer         avg             rms           peak

layer1    scalefactor1    scalefactor2    scalefactor3
layer2    scalefactor1    scalefactor2    scalefactor3
```

Here,

- layer name is the LEF layer name that could be a metal/via layer.
- avg, rms, peak correspond to the respective scale factor for avg, rms, or peak current. The scale factor value must be > 0 and <= 10.

`-default_freq_for_unconstrained_nets` *`freq_in_Hz`*

Specifies the frequency for EM calculation when a net has no defined frequency or has a defined frequency of 0 Hz in the design.

Default: `1MHz`

`-delta_T` *`temp`*

	Specifies the maximum temperature rise permitted in units of Celsius. This value is normally used in the Quantus tech file for the RMS current example: `em_jmax_ac_rms EQU sqrt(12.66 * delta_T * (w - 0.003)^2 * (w - 0.003 + 0.264) / (w - 0.003 + 0.0443))` *Default*: `5`
`-delta_T_layer_list {{<LEF_layer_name> <delta_T_in_C>}+}`	
	Specifies the delta temperature for specific LEF layers during signal EM RMS current limit analysis.
`-detailed`	Generates a detailed report containing information for all signal nets, including those without violations.
`-effort_level {low | medium | high}`	

	Specifies the method used for current calculation of nets in the Signal ElectroMigration analysis flow. The two methods are:
	• **non-simulation mode** - use the capacitance ratio to distribute the current down the stream from the driver to the receiver.
	• **simulation mode** - simulates resistance and capacitance to get the true current on each segment. This mode is more accurate but requires
	This parameter has the following three arguments:
	• `low`: enables the non-simulation mode for all nets where the current distribution is based on the capacitance ratio.
	• `medium`: enables the simulation mode for the nets with physical loops, and the non-simulation mode for the non-loop nets.
	• `high`: enables the simulation mode for all nets.
	The detailed report (`set_signal_em_analysis_mode -report <filename> -detailed`) shows "`N/A`" in the `Cap` column for nets in the simulation
	The default value for `-effort_level` is `low`. The `medium` and `high` levels require a VTS-AA license.
	Note: For the simulation mode with effort level `medium` or `high`:
	• If you use `extract_rc`, run `set_extract_rc_mode -qrcCmdType partial -qrcCmdFile qrc.user.cmd`, where `qrc.user.cmd` must include the commands:
	`extraction_setup -analysis em -cluster_vias_in_em true`
	`output_db -type spef -subtype "EXTENDED"`
	Refer to the example below:
	`output_db \`
	` -type spef -busbit_delimiter "[]" -escape_special_character false \`
	` -hierarchy_delimiter "/" -output_incomplete_nets true \`
	` -output_unrouted_nets true -pin_delimiter ":" -subtype "EXTENDED" \`
	` -disable_subnodes false -add_explicit_vias true \`
	` -include_parasitic_res_width true -include_parasitic_res_width_drawn true`
	• If you run standalone Quantus to generate the required SPEF file in the extended format separately, then instead of `extract_rc`, use the fo for signal EM analysis:
	`read_spef -starN -extended spef_file`

`-em_cdf_percentage value`	
	Specify the Cumulative Distribution Factor (CDF) derating factor for the EM rules specified in the ICT file or the technology file. This paramete calculate the fail rate and reliability of a design. The value of the CDF percentage should be between 0 and 100. If the `-em_cdf_percentage` parameter is not specified, and the CDF related rule exists in the technology file, the software will error out.
`-em_limit_scale_factor {{avg value} {rms value} {peak value}}`	
	Scales the calculated signal EM limits by the respective factor for avg, rms or peak current limit specified by `{avg value} {rms value} {peak` software compares the current through the net against the scaled EM limit to report violations in signal EM. The value specified for this parameter must be > 0 and <= 10.
`-em_limit_scale_table em_limit_scale_table_file`	

	Specifies the name of the scale table file used for layer-based scaling of the signal EM limit. The scale table contains the scale factor for avg, current for each layer.
	This parameter is mutually exclusive with `-em_limit_scale_factor`. The `-em_limit_scale_table` parameter takes precedence over `-em_limit`
	The format of the table is given below:
	`layer avg rms peak`
	`layer1 scalefactor1 scalefactor2 scalefactor3`
	`layer2 scalefactor1 scalefactor2 scalefactor3`
	Here,
	• layer name is the LEF layer name that could be a metal/via layer.
	• avg, rms, peak correspond to the respective scale factor for avg, rms, or peak current. The scale factor value must be > 0 and <= 10.

`-em_res_width {drawn silicon}`	
	Specifies width used to get EM limit. The legal values are: • `drawn` • `silicon` *Default*: `drawn`
`-em_temperature value`	
	Specifies the temperature expected for the device and is used to look up the temperature scaling factor. By default, no scaling is done. The `va` units of degrees Celsius.
`-em_temperature_layer_list {{<LEF_layer_name> <em_temperature_in_C>}+}`	
	Sets a different temperature for specific LEF layers during signal EM AVG current limit analysis.
`-em_threshold value`	
	Specifies to report violations in signal EM above the threshold value. The value is the ratio of signal current (avg/peak/rms) to the respective E `1.0`.
`-enable_seb {true | false}`	
	Specifies to enable the Statistical ElectroMigration Budgeting (SEB) analysis. *Default* : `false`
`-env_temperature <temperature in °C>`	
	Specifies the ambient temperature used to calculate S_{DC} for SEB analysis. *Default* : `25`
`-error value`	
	Controls the number of errors to be included in the signal EM violation report file. If you do not specify this parameter, the top 10000 signal net be included in the violation report. This parameter can be used to control the size of the violation report file. *Default:* `10000`
`-extraction_tech_file filename`	
	Specifies an extraction technology file for the Signal ElectroMigration analysis flow. If this parameter is not specified, then by default the extra file will be taken from the analysis view (current RC corner). This parameter must be used when the technology file is not specified through the analysis view (`create_rc_corner -qx_tech_file`). If you s `create_rc_corner -qx_tech_file` and `set_signal_em_analysis_mode -extraction_tech_file`, then the technology file specified with the `-extraction_tech_file` parameter will take precedence. In addition, if a technology file is not specified using either of these two command par software displays a warning message for the missing technology file, and will not read the EM models specified through the `-ict_em_models` p
`-forceHoldView`	
	Forces `set_signal_em_analysis_mode` to use the hold view for current calculation even during setup fixing.

`-handle_pin_obs_via`

Specifies to consider the vias defined in the `OBS` (obstructions/blockages) section of LEF in the Signal ElectroMigration flow. By default, the sc consider via in obstruction.

This parameter does not honor the via obstructions with "`SPACING 0`".

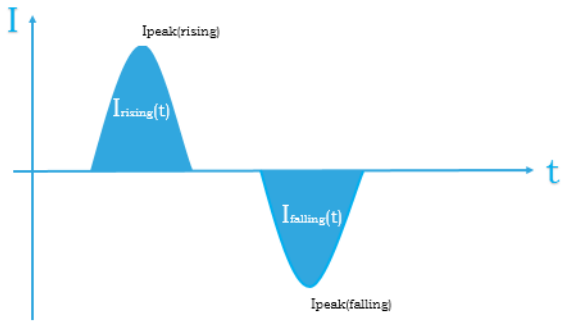
`-ict_em_models` *`file`*

Specifies the ICT-EM file which only contains the EM rules and excludes the RC extraction data in ICT. The software will use the EM rules in EM file to calculate EM limits for signal net EM analysis even if there are embedded EM rules in the Quantus tech file.

`-Ipeak_Td_method {effective_width_from_integration effective_half_peak_width max_equivalent_dc_peak sum_half_peak_width}`

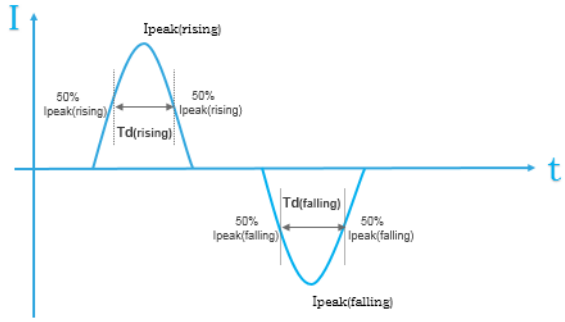
Specifies the Td calculation method for Ipeak check. The legal values are depicted below.

`effective_width_from_integration`



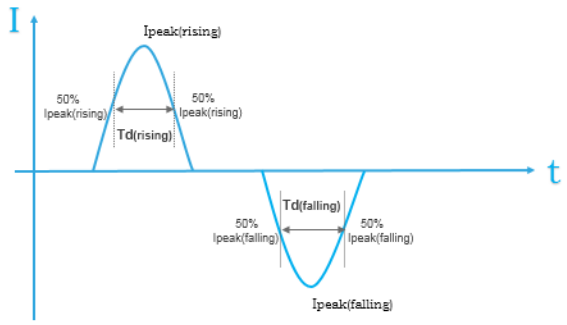
- $Td = \frac{\int_0^{Tsw} |I(t)|dt}{Ipeak_measured}$
  - Tsw = Period of one switching cycle (rising + falling)
  - Ipeak_measured = abs.max(Ipeak(rising), Ipeak(ralling))

`effective_half_peak_width`



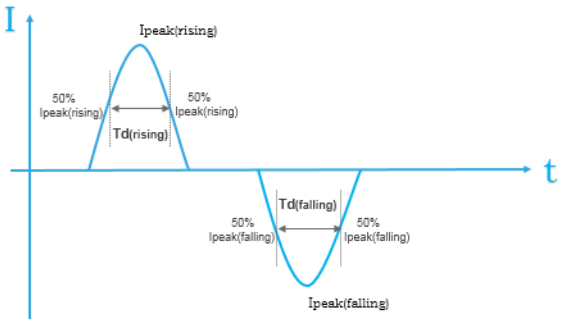
- When Ipeak(rising) >= Ipeak(falling)
  $Td = Td(rising) + (\frac{Ipeak(falling)}{Ipeak(rising)})^2 * Td(falling)$
- When Ipeak(rising) < Ipeak(falling)
  $Td = Td(falling) + (\frac{Ipeak(rising)}{Ipeak(falling)})^2 * Td(rising)$

max_equivalent_dc_peak



- $R(rising) = Td(rising)/(T/2)$
- $R(falling) = Td(falling)/(T/2)$
- Choose $max(Ipeak_{(rising)}*sqrt(R_{(rising)}), Ipeak_{(falling)}*sqrt(R_{(falling)}))$ to compare against Ipeak_dc_limit
- $Ipeak_ac_limit = Ipeak_dc_limit/sqrt(R)$
- When $Ipeak_{(rising)}*sqrt(R_{(rising)}) >= Ipeak_{(falling)}*sqrt(R_{(falling)})$
  $Td = 2*Td(rising)$
- When $Ipeak_{(rising)}*sqrt(R_{(rising)}) < Ipeak_{(falling)}*sqrt(R_{(falling)})$
  $Td = 2*Td(falling)$

sum_half_peak_width



- $Td = Td_{(rising)} + Td_{(falling)}$

Default: `effective_width_from_integration`

`-ircx_models {RC.ircx EMIR.ircx}|{RC.ircx}`	
	Specifies an encrypted or unencrypted RC/EM technology (.ircx) file. The foundry-specific IRCX file, which contains the process information f modeling, can be used when the ICT-EM file is not available for electromigration analysis. You can either specify two IRCX files, one file containing the RC process data (`RC.ircx`) and the other file containing EM rules (`EM.ircx`), or a includes both the RC and EM sections (`RC.ircx`). A single IRCX file is used for the mature nodes, and separate RC and EM IRCX files are us process nodes.
`-lifetime` *value*	
	Specifies the total hours of operation expected for the device, and is used to look up the lifetime scaling factor. *value* is used to find the scalin interpolating from a piece-wise-linear table in the LEF (see `ACCURRENTDENSITY HOURSPWL` table) or the Quantus tech file (`em_jmax_ac model` wit table). If there is no scaling factor table, no scaling is done. *value* is a float, in units of hours.
`-method {rms peak avg}`	
	Specifies the current waveform calculation method. You can specify one or more of the following options: • `avg`: Checks I_{avg} limits and reports any average current limit violations in the design. • `peak`: Checks I_{peak} limits and reports any peak current limit violations in the design. • `rms`: Checks I_{rms} limits and reports any RMS current limit violations in the design. *Default*: `rms` **Notes** • If you specify `peak` or `avg` but AAE is not enabled, the tool displays an error message. • When you run set_signal_em_analysis_mode -method `rms`, the command uses AC limit rules in LEF even if the EM qrcTechfile is provide `set_signal_em_analysis_mode -method rms -useQrcTech` to check rms EM limits from qrcTechfile.
`-minPeakDutyRatio` *duty_ratio*	
	Sets the minimum duty ratio value for I_{peak} calculation. If the calculated duty ratio of a net is lower than the specified value, the specified valu calculate I_{peak}. The minimum value for *duty_ratio* is `0.0` and the maximum is `1.0`. Default: `0.05` The following determines the priority and precedence of the `-minPeakDutyRatio` parameter: • When there are explicit duty ratio (r) dependencies on EM rules, the user-specified `minPeakDutyRatio` value will overwrite the software-cal (r), if the calculated duty ratio is smaller than the user-specified `minPeakDutyRatio` value. The `minPeakDutyRatio` value will then be applied selection to find the corresponding EM rule with duty ratio (r) dependency for EM limit calculation. • When there are no explicit duty ratio (r) dependencies on EM rules, the user-specified `minPeakDutyRatio` value will overwrite the tool-calcu if the calculated duty ratio is smaller than the user-specified `minPeakDutyRatio` value, and the software will calculate the EM limit based on `minPeakDutyRatio`. If you do not specify a value to `minPeakDutyRatio`, the default value of 0.05 will be applied.
`-minPeakFreq` *freq_in_Hz*	
	Ignores I_{peak} current limit calculation for signal nets that have an effective frequency below the specified value (in Hertz). *Default*: `1e+06 Hz (1 MHz)`
`-net {`*netNames*`}| -selected`	

	Specifies whether to check named or selected nets. If you name only one net, you do not need the braces around the net name. If you specify you must separate the net names with a space. For example, `set_signal_em_analysis_mode -nets {clk0 clk1}`.
	The `-selected` parameter analyzes the net specified with the `selectNet {netName} | [command]` command, and/or by selecting a net in the ma The following are examples of this parameter:
	Example 1:
	`selectNet {netA}`
	`set_signal_em_analysis_mode -selected`
	Example 2:
	`selectNet [get_ccopt_clock_tree_nets]`
	`set_signal_em_analysis_mode -selected`
	Default: If you do not specify `-nets` or `-selected`, `set_signal_em_analysis_mode` verifies all nets.

`-net_file list_file`	

	Specifies the net from file to check the signalEM analysis. The format is one net name per line, with comments using # as the first character. F
	`#list of nets to skip`
	`net_a`
	`net_b`
	`net_c`

`-net_freq_file filename`	

	Specifies a file containing the user-specified frequency for a list of nets. This parameter allows you to overwrite the frequency for specific nets set by `-toggle` needs to be modified.
	The format of the file is:
	`<net name>  <freq in Hz>`
	Example:
	`IN_1    1000000000`
	`n_I2_1  2700000000`
	`OUT_1   1200000000`

`-read_detail_delta_temperature_file filename`	
	Specifies to read the detailed delta temperature file generated by the `analyze_self_heat` command (self-heat analysis).
`-report filename`	
	Specifies the report file for the violation data. The report file includes name of the net, I_{rms}, interpolated `ACCURRENTDENSITY` limit (I_{limit}), layer, wi location, rise and fall time, effective frequency for the net (F_{eff}), Cnet and Vdd for each net.
`-reset`	Resets all the specified parameters back to the default values.
`-report_db`	
	Specifies to write the signal EM data into a database file (*`filename.db`*) after performing signal EM analysis. This file contains the information loading and displaying signal EM data in GUI. *`filename`* is the name specified using the `-report` parameter.
`-seb_lifetime value`	
	Specifies the life time of product in hours. *Default* : `43800` hours (that is, 5 years)
`-seb_table table_filename`	
	Specifies the file name of the Failures in Time (FIT) table. This file is provided by the foundry. The file has the following columns: • Layer • MTF • Sigma • Activate energy • Current density exponent

`-seb_temperature` *temperature*	

	Specifies the temperature used to update the median time to failure (MTF) with respect to the ambient temperature.
	• If this temperature is specified, it would be used globally for all the resistors.
	• If this temperature is not specified, the seb_temperature may take the following possible values depending on the specific circumstances:
	  ◦ If detailed delta temperature file is not used for SEB, the `seb_temperature` always takes the same value as that used for EM analysis.
	  ◦ If detailed delta temperature file is used for SEB, the seb_temperature is calculated by the following equation:
	  `seb_temperature = env_temperature + delta_T`
	  Where delta_T is the maximum temperature rise, including joule heat and coupling heat from FEOL, in the resistor's bounding box with

`-set_current_file` *filename*	

	Specifies the current of a macro's output pins for EM calculation, in case the macro does not have the respective timing library. When the mac current is set by the current file, the value calculated (if any) from AAE is overwritten.
	The format of the current file is:
	`{CELL` *cell_name*`|INSTANCE` *instance_name*`} PIN` *pin_name* `RMS` *rms_current* `PEAK` *peak_current* `AVG` *avg_current* `Td <`*value*`> NetFreq <v`
	The unit of `Td` is Nanosecond and `NetFreq` is GHz.
	Following are few examples of the current file format:
	`CELL cell_macro_A PIN Q1 RMS 0.01 PEAK 0.1 AVG 0.01`
	`CELL cell_macro_B PIN Q1 RMS 0.01 PEAK 0.1 AVG 0.01`
	`INSTANCE inst_macro_1 PIN Q2 RMS 0.01 PEAK 0.1 AVG 0.01`
	`INSTANCE inst_macro_2 PIN Q2 RMS 0.01 PEAK 0.1 AVG 0.01`

`-simulation_net` *list_of_nets*	
	Forces the specified nets to use the simulation mode.
`-simulation_net_file` *filename*	
	Forces the nets in the specified file to use the simulation mode.
`-skip_category_mode {0 | 1 | 2}`	

Specifies to skip nets that belong to categories mapping to the specified mode. This parameter allows you to specify three modes, namely 0, 1

The following table lists the common categories for all modes, and the categories specific to each mode:

Common Categories	0 Skip Category Mode	1 Skip Category Mode	2 Skip Category Mode
• Non top scope nets • Internal nets on boundary path • P/G nets • Constant nets • User-defined skipped nets • Nets below effective frequency threshold • Physically broken nets • Nets without loading capacitance annotation • Nets with SPEF back-annotation failures during simulation • Nets failed in analysis	• Slave nets by Verilog Assign • Nets (non-constant) without frequency annotation • Nets without physical shape for output pin • Nets without wire shape connection to output pin • Nets with zero pin current	• Equivalent nets by Verilog Assign • Nets without driver • Nets without receiver	• Nets without pins • Equivalent nets by Verilog Assign • Nets without driver • Nets without receiver

For example, if you set `set_signal_em_analysis_mode -skip_category_mode to 1`, the report file lists all categories listed in the *Common Ca* *Skip Category Mode* columns.

Following is a snippet of the report file for mode **1**:

```
Total Nets in Design:               33451
Total Selected Nets in Design:          4
Total Analyzed Nets:                    0
Total Skipped Nets:                     4
  Non top scope nets:                                          0
  Internal nets on boundary path:                              0
  Equivalent nets by Verilog Assign:                           0
  P/G nets:                                                    0
  Constant nets:                                               2
  User-defined skipped nets:                                   0
  Nets below effective frequency threshold:                    0
  Nets without driver:                                         1
  Nets without receiver:                                       1
  Physically broken nets:                                      0
  Nets without loading capacitance annotation:                 0
  Nets with SPEF back-annotation failures during simulation:   0
  Nets failed in analysis:                                     0
```

`-skip_net` *`net_name_list`*

Specifies the net to be skipped in the signalEM analysis.

Refer to the example below:

```
{net_a net_b net_c}
```

`-skip_net_file` *`list_file`*

Specifies the net file to be skipped in the signalEM analysis.

The format is one net name per line, with comments using # as the first char. For example:

```
#list of nets to skip
net_a
net_b
net_c
```

`-toggle` *`value`*

Specifies the switching factor for signal nets. A value of `1.0` means that if a flip-flop is clocked by a 20 Mhz clock, it changes state on 100% of Therefore, the signal has 20 million transitions, which is half the number of transitions of the input clock. As a result, the final switching freque net is 10 Mhz.

Default: `1.0`
Value Range: `0.0` to `1.0`

`-useQrcTech`	Forces I_{rms} checks to use the Quantus tech file instead of the technology LEF file. If either I_{peak} or I_{avg} is also checked, all checks will use th including I_{rms}. This parameter is enabled by default. **Note:** The `-method avg` and `-method peak` analysis will only support AC current density rules from Quantus tech file. If either `avg` or `peak` is als all checks will use the Quantus tech file equations, including RMS.
`-use_rms_delta_t {true | false}`	
	When this parameter is set to `true`, the `delta_T` used to calculate `seb_temperature` would be based on the RMS delta T (joule heat only). If this parameter is set to `false`, the `delta_T` used to calculate `seb_temperature` would be based on the sum of self-heat delta T and RMS delta *Default* : `false`
`-view viewName`	
	Specifies the name of the view.

Example

- The following command specifies to calculate current waveform using the rms, avg, and peak methods, and save violations in a report named `sigem.rpt`

```
set_signal_em_analysis_mode \

    -method {rms avg peak} \
    -detailed true \
    -report sigem.rpt \
    -avgRecovery 0  \
    -useQrcTech true \
    -ict_em_models /../ictfile.ict \
    -em_threshold 0 \

verify_AC_limit
```

- The following is an example of the `-extraction_tech_file` parameter:

```
set_signal_em_analysis_mode -method {avg rms peak} -detailed \
-ict_em_models ../../EMdata.ict \
-extraction_tech_file qrc_file-em_temperature 105 \
-avgRecovery 0.5 -report AC_signalEM.tcf.rpt -error 100000000
```

- The following command specifies different temperature values to the layers M1, M2, M3, VIA1, VIA2, and VIA3 for average current limit analysis:

```
set_signal_em_analysis_mode
...
   -em_temperature 110 \ #temperature for the remaining layers
   -em_temperature_layer_list {M1 125 M2 115 M3 105 VIA1 115 VIA2 105 VIA3 120} #layer-based temperature
```

  The log file will display the following info message:

```
Layer specific EM temperature will be used for AVG check for layer M1(125C), M2(115C), M3(105C), VIA1(115C), VIA2(105C), VIA3(120C).
```

Related Information

- `get_signal_em_analysis_mode`
- "Signal ElectroMigration" in the *Voltus IC Power Integrity Solution User Guide*

35

Signal Integrity Commands

- check_noise
- create_spice_deck
- define_bundled_bus
- fixACLimitViolation
- get_glitch_threshold
- getSIMode
- readTransitionFile
- report_bundled_bus
- report_double_clocking
- report_instance_cdb
- report_noise
- report_noise_lib_pin
- report_noise_propagation
- report_voltage_scaling
- report_voltage_swing
- reset_glitch_derate
- reset_noise_lib_pin
- set_annotated_glitch
- set_cdb_binding
- set_glitch_derate
- set_glitch_threshold
- set_model_priority
- set_mutex_condition
- set_noise_lib_pin
- set_quiet_attacker
- setOutboundReport
- setSIMode
- update_glitch
- analyze_ir_paths
- define_glitch_holding_resistance

check_noise

```
check_noise
[-help]
[-all]
[-check_only {receiver_no_propagation_model | receiver_no_dc_tolerence |
driver_no_current_source_model | driver_no_model}]
[-instance_list string]
[-instance_only {true | false}]
[-pin_list string]
[-type {receiver_model | driver_model}]
[-unique {true | false}]
[-verbose]
[-view string]
[ [ > | >> ] filename ]
```

Performs several consistency and completeness checks on noise models specified for a design. These checks help to ensure that the right set of models are available for the SI analysis engine to perform crosstalk noise analysis.

Note: You need to set `setDelayCalMode -siAware` to `true` before running the `check_noise` command.

Parameters

`-help`	Prints out the command usage.
`-all`	Displays all the debug information.
`-check_only {receiver_no_propagation_model | receiver_no_dc_tolerence | driver_no_current_source_model | driver_no_model}`	Checks the warning name. The following warnings are checked: • `receiver_no_propagation_model`: Shows how many design pins are missing the receiver model for propagation. • `receiver_no_dc_tolerence`: Shows how many pins have no DC VH/VL tolerance specified. • `driver_no_current_source_model`: Shows how many design pins are missing the current source model. • `driver_no_model`: Specifies how many design pins are missing the driver model.
`-instance_list string`	Specifies the instance name.
`-instance_only {true | false}`	Prints each instance only once.
`-pin_list string`	Specifies the instance pin names.
`-type {receiver_model | driver_model}`	Checks the presence of a valid model type - `receiver_model` or `driver_model`. If the specified model is missing, a warning will be issued. The valid receiver models are the models having vivo tables, such as, CDB, CCSN, ECSMN. The valid driver models are CDB, CCS, ECSM, NLDM, and UDN.
`-unique {true | false}`	Reports unique cells or pins for each power domain.
`-verbose`	Generates verbose details.
`-view string`	Specifies the view name.
`>filename`	Redirects output to a file.
`>>filename`	Appends to an output file.

Examples

- The following command generates the output as shown below:

```
check_noise -type receiver_model

   **WARN: (IMPESI-3125): AAE_WARN: The analysis view specified to check_noise is not found or specified. Checking view
   default_emulate_view_setup.
   --------------------------------------------------------------------------------
```

```
CHECK NOISE SUMMARY
------------------------------------------------------------------------------
Warning Number of Warning
------------------------------------------------------------------------------
receiver_no_propagation_model 17931
receiver_no_dc_tolerence 17931

------------------------------------------------------------------------------
# CHECK NOISE WARNING DESCRIPTIONS:

# receiver_no_propagation_model: The receiver model for glitch
# propagation or ROP is not available, only RIP analysis will be performed.

# receiver_no_dc_tolerence: The input pin do not have vh/vl dc
# tolerence specified in the noise file. Default tolerences will be used.

------------------------------------------------------------------------------
```

Related Information

- setDelayCalMode
- getDelayCalMode

create_spice_deck

```
create_spice_deck
[-help]
[-add_side_load_measurement {true | false}]
[-add_voltage_for_missing_sensitization]
[-bbox_cells string]
[-depth integer]
[-enable_arrival_measurements {true | false}]
[-enable_fast_simulation {true | false}]
[-enable_spectre_x {true | false}]
[-exclude_side_load_cells string]
[-exclude_side_load_cells_for_missing_subckt {true | false}]
[-force_delete_output_dir]
[-file_prefix string]
[-format string]
[-from string]
[-from_fall string]
[-from_rise string]
[-glitch_simulation_mode {input_output | output}]
[-ground string]
[-include_cell_variation {true | false}]
[-lsf_job_string string]
[-measure_point {all | path | stage}]
[-merge_spice_deck {true | false}]
[-mc_simulation_init_args string]
[-mc_simulation_tran_args string]
[-model_file string]
[-no_hard_stop_at_to_pins {true | false}]
[-outdir string]
[-power string]
[-receiver_pin string]
[-report_noise string]
[-report_timing string]
[-run_simulation]
[-segment {data_path | launch_path | launch_clock_path | capture_path all}]
[-segment_end string]
[-segment_start string]
[-side_path_level integer]
[-spectre string]
[-spectre_analysis_options <string>]
[-spectre_cmd_options string]
[-spectre_lang_escape_chars string]
[-spice_include string]
[-stimuli_file string]
[-subckt_file string]
[-supply_voltage_file string]
[-sweep_range float]
[-sweep_steps integer]
[-sweep_victim {true | false}]
[-temperature float]
[-through string]
[-to string]
[-use_control_param {true | false}]
[-use_distributed_load_for_side_receivers {true | false}]
[-user_defined_measure_statements {append | overwrite}]
[-user_defined_measure_statements_file string]
[-waveform_aware_pulse_width_checks {true | false}]
[-view string]
[-xtalk {delay | vl_glitch | vh_glitch | vlu_glitch | vho_glitch}]
[[ > | >> ]]
```

Generates the SPICE trace for a path.

While generating a Spice Deck, the `create_spice_deck` command looks for library cells required to run Spice and adds all cells as ".include" in the generated Spice netlist.

The output SPICE deck includes:

- Active and passive devices such as field-effect transistors (FETs), capacitors, and resistors.
- Initial conditions and voltage sources
- Actual names of the gates, nets, and cell instances, which helps the simulator interpret this information effectively. For example, consider the following excerpt, which shows an entry for a gate named `U0` in the SPICE deck:

```
*Gate U0
*--------------------
*Cell ports in order : x a VSS VDD
*--------------------
XU0 U0/x U0/a 0 U0/VDD inv_ahvt_64
*--------------------
```

Note: In order to output a Spice deck that generates the same report output as the `report_timing` command, you must run the `create_spice_deck` command with the `-report_timing`, `-segment_start`, and `-segment_end` parameters.

Parameters

`-help`	Prints out the command usage.
`-add_side_load_measurement {true | false}`	Adds the side load measurement in clock mesh Spice deck. By default, Innovus only measures the latencies, delays, and slews of arcs that are part of clock mesh specified using the `-from`/`-to` parameters. When set to `true`, the software additionally measures the latencies and slews of the side recievers at each stage. *Default*: `false`
`-add_voltage_for_missing_sensitization`	Adds the voltage for missing sensitization.
`-bbox_cells string`	Specifies the list of cells to be considered as black boxes for SPICE simulations. This parameter enables you to continue with the simulations even if subckts are missing for one or more cells. All the instantiations of these cells will be replaced by pin caps to replace the input pins connectivity of black-boxes, and the software computed PWL waveforms applied at the output pins of black boxes. The SPICE delays/slews reported for these blackboxes will be the software computed values.
`-depth integer`	Creates a clock mesh Spice deck of the specified stage depth. Innovus only traces the clock mesh up to the user-specified depth, starting from the specified `-from` pin. If both the `-depth` and `-to` parameters are specified, then Innovus generates the deck till the depth which is minimum of the `-to` pin depth and the user-specified depth.
`-enable_arrival_measurements {true | false}`	Allows adding arrival measurements.
`-enable_fast_simulation {true | false}`	When set to `true`, the software enables the fast simulation mode. In this mode, additional clock cycles are not required to initialize sequential cells in the Spice deck - this speeds up Spice simulations. All the sequential cells are initialized using the initial conditions and only a single clock edge is applied as input. By default, this parameter is set to `false`. In this mode, two clock cycles are applied as input to the clock network, where the first triggering edge is used for initialization of sequential cells, and the second edge is used for delay measurement.
`-enable_spectre_x {true | false}`	Enables the spectreX license checkout when launching Spectre.
`-exclude_side_load_cells string`	Specifies a list of cells for which the pin capacitance will be used for side loads instead of instantiating them.
`-exclude_side_load_cells_for_missing_subckt {true | false}`	When set to `true`, the software will add side load pin capacitance for missing Spice subckt cell instead of instantiating it.

`-file_prefix` *string*	Adds the specified prefix to the generated Spice deck files. If this parameter is not used, one of the following default names of the generated Spice decks will be used: • `path_<index>_setup.sp: for setup paths` • `path_<index>_hold.sp: for hold paths` • `path_<index>_vl_glitch.sp: for VL glitch` • `path_<index>_vh_glitch.sp: for VH glitch` The `<index>` in the above file names is controlled based on the number of decks that are generated using the `create_spice_deck` command. A user-specified string (specified using this parameter) is used as a prefix to the above mentioned file names.
`-force_delete_output_dir`	Removes the directory specified by the user using the `-outdir` parameter.
`-format` *string*	Allows you to make changes to the run path simulation internal reporting format. The format columns should be compatible with the `report_timing_format` settings.
`-from` *string*	Creates the Spice deck with mesh net starting from the specified pin.
`-from_fall` *string*	Creates the Spice deck with mesh net starting from the falling transition of the specified from pin.
`-from_rise` *string*	Creates the Spice deck with mesh net starting from the rising transition of the specified from pin.
`-generate_output_only {true | false}`	Generates the report output only, without triggering any simulations. *Default*: `false`
`-glitch_simulation_mode {input_output | output}`	Specifies the glitch simulation mode. When set to `input_output`, the software uses both the RIP and ROP for Spice calculations. When `output` is specified, the Spice ROP will be computed using the RIP.
`-ground` *string*	Specifies a list of ground nodes.
`-include_cell_variation {true | false}`	Writes out the Spice deck required to run simulations. When set to `true`, the software runs Monte-Carlo simulations from within the Tempus software. *Default*: `false`
`-lsf_job_string` *string*	Specifies a handle to the farm job for launching the simulation.
`-mc_simulation_init_args` *string*	Passes necessary Monte Carlo parameters that are required to run MC simulations. The Monte Carlo analysis refers to “statistics blocks”, where statistical distributions and correlations of netlist parameters are specified. This parameter initializes these parameters under the statistics block. **Note**: This parameter can be used when `-include_cell_variation` is set to `true`.
`-mc_simulation_tran_args` *string*	Initializes full transient time of simulation and transient time step.
`-measure_point {all | path | stage}`	Specifies whether to add all, path, or stage measurements. *Default*: `all`
`-merge_spice_deck {true | false}`	Allows you to enable merged Spice deck.
`-model_file` *string*	Specifies the path of SPICE model files, which contains spice model information. This information is necessary to get accurate Spice information. If sub-circuit and model files are not provided and cdBs are available, spice information is dumped using cdB.

`-no_hard_stop_at_to_pins {true | false}`	When set to `false` (default mode), the Spice deck is generated only for traced paths starting from the `-from` pin and ending at the `-to` pin. It does not traverse through the fanout branches that do not lead to the `-to` specified pin. When set to `true`, traversal occurs through the whole fanout cone of the `-from` pin till it hits a clock pin of a flop, or any stop point specified with the `-to` parameter, or passes through a depth specified with the `-depth` parameter.
`-outdir` *string*	Specifies the name of the output directory that will contain the generated delay and/or glitch spice decks. If simulation is also performed using the `-run_simulation` parameter, the Spice simulations output will also be saved in this directory.
`-power` *string*	Specifies the list of global names for the power nets. If the `-power` and `-ground` parameters are not specified, the software obtains this information from the power/ground supplies set via other ways in the design, such as, from the libraries/cdb, and so on. If the information is not available, the software uses the default list of power node names (VDD and VCC) and ground node names (GND and VSS).
`-receiver_pin` *string*	Specifies the list of receiver pins for which glitch Spice decks need to be generated. When used with the `-run_simulation` parameter, the software writes out a comparison report comparing various glitch parameters between Innovus and Spice, such as RIP, input glitch area, and ROP (only if ROP node name is available in the noise models) for the given receiver pins.
`-report_noise` *string*	Specifies the `report_noise` command parameters as a list of arguments within brackets. When specified, the Spice deck will be generated for all the receiver pins that are reported by the `report_noise` command for the specified options. When specified with the `-run_simulation` parameter, the software writes out a comparison report comparing various glitch parameters between Innovus and Spice, such as, RIP, input glitch area, and ROP (only if ROP node name is available in the noise models).
`-report_timing` *string*	Specifies the `report_timing` command options as a list of arguments within curly brackets. You can use this option to define the paths for which to generate a Spice trace. To select the paths for generating the Spice trace, specify the `report_timing` command options, as shown: `create_spice_deck -report_timing {-clock_from CLK -clock_to CPU_CLK -nworst 2}` When the `-report_timing` option is not used, the software generates a Spice trace for the worst path identified by the `report_timing` command. If the `-path_type {full_clock}` is used, then the clock path will also be processed. See the `report_timing` command for a complete list of options that can be used as arguments with the `-report_timing` option. **Note:** The `end` and `summary` values of the `-path_type` option of `report_timing` are not supported.
`-run_simulation`	Enables path simulation in addition to generating Spice Deck.
`-segment {data_path | launch_path launch_clock_path | capture_path all}`	Creates the Spice deck of a segment out of the path spice deck. The following command/global variable settings have the following impact on the `-segment` parameter options: • `-path_type full_clock` is defined under the create_spice_deck-report_timing parameter, then the -segment default value is `launch_path`. • `-path_type full_clock` is not specified, then default value is `data_path`. • `timing_disable_retime_clock_path_slew_propagation` global variable is set to `false`, the default value is `launch_path` (irrespective of `-path_type` settings). • `timing_disable_retime_clock_path_slew_propagation` global variable is set to `true` (default), then `launch_clock_path` and `capture_path` options are supported only when the `-path_type full_clock` option of the `create_spice_deck -report_timing` parameter is used.
`-segment_end` *string*	Specifies the name of the segment end term.

`-segment_start` *string*	Specifies the name of the segment start term.
`-side_path_level` *integer*	Specifies the number of levels of side path stages to be included in the Spice trace. The maximum number of levels that can be specified is 4. *Default*: `1`
`-spectre` *string*	Specifies the location of the Spectre version that should be used for simulation. If the path is not specified with this option, the software will check if you have the Spectre location defined in your path.
`-spectre_analysis_options <string>`	Adds Spectre analysis options to the run. When specified, the software provides instructions to Spectre, e.g., run a session with extra accuracy.
`-spectre_cmd_options` *string*	Specifies the command line options for Spectre.
`-spectre_lang_escape_chars` *string*	Specifies a list of escape characters in Spectre.
`-spice_include` *string*	Specifies the user-defined lines that should be written to the Spice deck. For example, `create_spice_deck -spice_include {.include extra_cmds.sp}` where `extra_cmds.sp` includes additional SPICE commands that are required for the run and are not included in the SPICE model file loaded with the `-models` parameter.

`-stimuli_file` *string*	Allows you to specify the file path containing stimuli to be used for the specified library cells. The cells' sensitization coming from the user-specified stimuli file takes precedence over the software inferred sensitization.
	Syntax for delay Spice decks is given below:
	`lib_cell` *string* `<from | from_rise | from_fall>` *string* `<to | to_rise | to_fall>` *string* `input_vector` *string* `num_clk_pulse` *integer*
	Syntax for glitch Spice decks is given below:
	`lib_cell` *string* `glitch_type <VL | VH> output_pin` *string* `input_vector` *string* `num_clk_pulse` *integer* `# For victim driver sensitization`
	`lib_cell` *cell_name* `glitch_type <VL | VH> input_pin` *string* `input_vector` *string* `num_clk_pulse` *integer* `# For victim receiver sensitization`
	The `num_clock_pulse` *integer* holds significance in case of sequential arcs, and specifies where to align the data pin transition with respect to the clock transition.
	For example, a value of '1' (default setting) specifies that the data pin transition needs to be aligned before the last clock transition where delay measurement is intended. This value can be used in case of regular flip-flops.
	The value '2' specifies that data pin transition needs to be aligned with respect to the second-last triggering clock edge. This value can be used for cascaded 2-bit flip flops.
	Additionally for sequential arcs, the first pin in the user-specified input vector needs to be the data pin, the signal of which will be latched in the sequential cell.
	Here are some examples of stimuli file:
	• The following example shows the stimuli file containing sensitizations for CP(rise)->Q(fall) and CP(rise)->Q(rise) timing arcs of library cell FF1:
	`lib_cell FF1 from_rise CP to_fall Q input_vector !D&CDN&!SE&!SI num_clk_pulse 2`
	`lib_cell FF1 from_rise CP to_rise Q input_vector D&CDN&!SE&!SI num_clk_pulse 2`
	• The following example shows the stimuli file containing sensitizations for the victim driver cell AOI for both VL and VH glitches:
	`lib_cell AOI glitch_type VL output_pin Z input_vector !A&!B&!C`
	`lib_cell AOI glitch_type VH output_pin Z input_vector A&!B&C`

`-subckt_file` *string*	Specifies the path of Spice sub-circuit file which contains the Spice information. This information is necessary to get accurate Spice information. If subcircuit and model files are not provided and cdBs are available, then Spice information in dumped using cdB.
`-sweep_range` *float*	Specifies the range to sweep the victim.

`-sweep_steps` *integer*	Specifies the number of steps by which the victim is swept.
`-sweep_victim {true | false}`	Enables victim sweep.
`-supply_voltage_file` *string*	Specifies a user-defined power/ground voltage file.
`-temperature` *float*	Specifies a user-defined temperature to be used in SPICE simulations. *Unit*: Celsius
`-through` *string*	Creates the Spice deck with mesh net going by `-through` points.
`-to` *string*	Creates the Spice deck with mesh net ending at the specified to pins.
`-use_control_param {true | false}`	Allows you to control the power/ground voltage levels in the created Spice deck through user-specified parameters. When set to `true`, an additional file with the same name as the deck, and extension .control is created that contains the parameterized design voltage levels of the PG pins of all the cells instantiated in the deck. Your can modify the *.control file as per the requirement without modifying the original deck.
`-use_distributed_load_for_side_receivers {true | false}`	Allows you to control whether to use the receiver output load as distributed or lumped load. *Default*: `false`
`-user_defined_measure_statements {append | overwrite}`	Specifies whether to overwrite the default measurement statements in the Spice deck by user-defined measurements specified using the `-user_defined_measure_statements_file` parameter, or append it with default measure statements. *Default*: `append`
`-user_defined_measure_statements_file` *string*	Allows you to add the delay/slew measurement statements for any given segement of the path for which Spice deck is generated. The format of the file is given below: `<MEASURE_TAG_NAME TRIGGER_PIN_NAME TARGET_PIN_NAME>` Multiple such lines with unique tags can be specified in the file. The measure statement is written in the Spice deck for delay measurement between `TRIGGER_PIN` and `TARGET_PIN`, and slew measurement at `TARGET_PIN`.
`-view` *string*	Specifies the view name - if a glitch Spice deck is created for a view other than the default view.
`-waveform_aware_pulse_width_checks {true | false}`	Generates the Spice deck for waveform-aware pulse width checks.
`-xtalk {delay | vl_glitch | vh_glitch | vlu_glitch | vho_glitch}`	Generates the SI Spice deck for the stage specified using the `report_timing` parameter. • A stage can start with victim driver input to victim receiver input. • A stage can start with victim driver input to victim receiver output. The stage will be specified in the `report_timing` command using the `-point_to_point` parameter. You can specify one of the following options: • `delay`: Allows to dump the SI delay Spice deck. • `vl_glitch | vh_glitch`: Specifies the type of glitch (VL or VH) the Spice deck will be used for. This option must be used for glitch Spice deck generation. • `vlu_glitch` | `vho_glitch`: Supports simulation of vlu/vho glitches.

Examples

- The following set of commands generate a SPICE deck for a path and output it in the `path1_spice` directory:

```
set rpt_tim_opt [ list -clock_from CLK -clock_to CPU_CLK -nworst 2 ]
create_spice_deck -outdir path1_spice -report_timing $rpt_tim_opt
-subckt_file  /path/subckt.sp  -model_file /path/models.sp
```

- The following command creates a SPICE deck and simulates it using the Spectre simulator:

```
create_spice_deck -run_simulation -spectre /tools/spectre
-subckt_file  /path/subckt.sp  -model_file /path/models.sp
```

- The following command creates a SPICE deck in `mydir` and keeps the instances up to three levels for each side path attached with the main path:

```
create_spice_deck -side_path_level 3 -outdir mydir
-subckt_file  /path/subckt.sp  -model_file /path/models.sp
```

- The following command creates a SPICE deck for the best path:

```
create_spice_deck -report_timing { -early }
-subckt_file  /path/subckt.sp  -model_file /path/models.sp
```

- The following command creates a Crosstalk Spice Deck for the specified Stage:

```
create_spice_deck -xtalk delay -model_file /path/models.sp -subckt_file /path/subckt.sp -report_timing {-point_to_point -from
<victim_driver_input> -to <victim_receiver_driver_output>} -output spice_output
```

- The following command dumps the VL spice deck `report_timing` command options so that the transition at receiver pin is failing:

```
create_spice_deck -xtalk glitch -report_timing < -from driver/input -to_fall receiver/input -point_to_point > -outdir <output dir> -
model_file <model file> -subckt_file <subckt file>
```

- The following command dumps the VH spice deck `report_timing` command options so that the transition at receiver pin is rising:

```
create_spice_deck -xtalk glitch -report_timing < -from driver/input -to_rise receiver/input -point_to_point > -outdir <output dir> -
model_file <model file> -subckt_file <subckt file>
```

- The following command run simulations and then generates a report:

```
create_spice_deck -report_timing "-net -from clk -to ca53/as_12/ff/D -path_type full_clock" -run_simulation -include_cell_variation
true > spice.socv.rpt
```

- The following command generates a report (if simulations have already been run):

```
create_spice_deck -generate_output_only true -report_timing "-net -from clk -to ca53/as_12/ff/D -path_type full_clock" -run_simulation
-include_cell_variation_true -format {arrival spice_arrival spice_arrival_detail arrival_error_detail hpin} > spice.socv.rpt
```

- The following section in SPICE deck specifies global supplies and transient analysis window time.

```
*---------------------
*Command and Options
*---------------------
.GLOBAL vdd vbbnw
V0Xvdd  vdd 0 1.0
V0Xvbbnw vbbnw 0 1.0
.GLOBAL vss vbbpw
V0Xvss vss 0 0
V0Xvbbpw vbbpw 0 0
.TEMP -40.0000
.TRAN 1p 10256p
*---------------------
```

- The following section shows an example of a net and its instance connections.

```
*-------------------------
*Net Instances
*-------------------------
*Net module1/inst_1/net
*-------------------------
Xmodule1/inst_1/net module1/inst_1/q
+ module1/inst_2/a wc:module1/inst_1/q
*-------------------------
```

- The following section shows an example of a gate instance and its port connections.

```
*----------------------------------------
*Gate Instances
*----------------------------------------
*Gate module1/inst_2
*----------------------------------------
*Cell ports in order : a y vss vdd vbbpw vbbnw
*----------------------------------------
Xmodule1/inst_2/a module1/inst_2/y
+ 0 module1/inst_2/vdd 0 module1/inst_2/vbbnw
+ inv1
*----------------------------------------
```

- The following section shows the measurement statements for delay and slew.

```
*---------------------
*Measurement statements
*---------------------
.MEASURE TRAN Stage_0001_module1/inst_1/clk_DELAY_
module1/inst_1/q                                  ---------clk to q delay
+ TRIG V(module1/inst_1/clk) VAL=0.5 TD=167.9p RISE=1       of module1/inst_1
+ TARG V(module1/inst_1/q) VAL=0.5 TD=167.9p RISE=1
.MEASURE TRAN Stage_0001_module1/inst_1/q_SLEW
+ TRIG V(module1/inst_1/q) VAL=0.30 TD=167.9p RISE=1
+ TARG V(module1/inst_1/q) VAL=0.70 TD=167.9p RISE=1  ---------Slew at
.MEASURE TRAN Stage_0002_module1/inst_1/q_DELAY_              module1/inst_1/q pin
module1/inst_2/a
+ TRIG V(module1/inst_1/q) VAL=0.5 TD=167.9p RISE=1   ---------Interconnect Delay
+ TARG V(module1/inst_2/a) VAL=0.5 TD=167.9p RISE=1          and Input Slew
.MEASURE TRAN Stage_0002_module1/inst_2/a_SLEW
+ TRIG V(module1/inst_2/a) VAL=0.30 TD=167.9p RISE=1  ---------Interconnect Delay
+ TARG V(module1/inst_2/a) VAL=0.70 TD=167.9p RISE=1         and Input Slew
```

Note: The `DELAY` and `SLEW` keywords in the `MEASURE` statements indicate that these are the measurement statements for delay and slew respectively.

Also, Stage numbers (Stage_0001 and Stage_0002) in the `MEASURE` statement indicate the respective stages; first and second stage, in the timing path, which makes it easier for correlation with the `report_timing` report.

- The following shows an example for generating a VL glitch SPICE deck:

```
create_spice_deck -xtalk vl_glitch -receiver_pin x1/x2/I -outdir SPICE/net1_vl -model_file ./all_cells.model -subckt_file
./all_cells.sp
```

- The following shows an example for generating a VH glitch SPICE deck:

```
create_spice_deck -xtalk vh_glitch -receiver_pin x1/x2/I -outdir SPICE/net1_vh -model_file ./all_cells.model -subckt_file
./all_cells.sp
```

- The following command sets the simulation time stop criteria to be 13.40 ns and step time to be 1ps:

```
create_spice_deck -mc_simulation_init_args "stop=1.34e-08 step=1p"
```

- The following command `writes` out statistics block in a deck to run Monte Carlo simulations:

```
create_spice_deck -mc_simulation_init_args "numruns=3000 seed=1234 distribute=fork numprocesses=8 variations=mismatch
scalarfile=MCDataSetLoad.txt dut=[%s]"
simulator lang = spectre

mc1 montecarlo numruns=3000 seed=1234 distribute=fork numprocesses=8 variations=mismatch scalarfile=MCDataSetLoad.txt dut=[
XStage2.XCTS_ccl_a_inv_01730 XStage3.XCTS_ccl_inv_01726 XStage4.XCTS_ccl_inv_01718 XStage5.XCTS_ccl_a_inv_01712 \
XStage6.XCTS_cpc_sk_inv_03405 XStage7.XCTS_cpc_sk_inv_03406 XStage8.XCTS_cdb_inv_03303 XStage9.XCTS_cdb_inv_02837 \
XStage10.XCTS_ccl_inv_01669 XStage11.XCTS_ccl_a_inv_01540 XStage12.XCTS_cpc_sk_inv_03860 XStage13.XCTS_cpc_sk_inv_03859 \
XStage14.Xugenq_uaesflushu1_aes_flush_u1_reg XStage15.XPLACEOPTFE_RC_149_0 XStage16.XPLACEOPTFE_RC_150_0 \
XStage17.XU22704 XStage18.XU22707 XStage19.XU22857 XStage20.XPLACEOPTFE_OFC2135_n30081 XStage21.XU23049 \
XStage22.XU140015 XStage23.XU132065 XStage24.XU132066 XStage25.XU132710 XStage26.XU132716 XStage27.XU133277 \
XStage28.XU133278 XStage29.XU133279 XStage30.XU133285 ]

{
tran1 tran stop=2.88312e-08 step=1p
}
simulator lang = spice
```

Related Commands

- report_timing
- report_timing_format
- report_noise

define_bundled_bus

```
define_bundled_bus
[-help]
[-add_members string]
[-delete_members string]
[-name string]
```

Allows you to control user-defined bus groups.

The software automatically identifies bus nets. However, in some cases the software may not be able to mark bus bits properly (or treat marked bits as bus). For improved bus identification, the `define_bundled_bus` command allows you to control the nets that you want to add (or remove) from the list of recognized bus bits.

Parameters

`-help`	Prints out the command usage.
`-add_members string`	Specifies the name of nets to be added to the group.
`-delete_members string`	Specifies the name of nets to be deleted from the group.
`-name string`	Specifies the name of the group.

Examples

- The following example command will create a bus named bus1 with 3 net members x[1] and x[2]:

```
define_bundled_bus -name bus1 -add_members {x[1] x[2]}
report_bundled_bus -type user_defined
############################################################

Summary:
Total number of user defined bus groups reported = 1
Total number of tool inferred bus groups reported = 0

------------------------------------------------------------

Group Name (user_defined) : bus1

Constituents:
- x[1]
- x[2]
```

- The following example command will add one more member x[3] and delete member x[2] to the bus 'bus1':

```
define_bundled_bus -name bus1 -add_members x[3] -delete_members x[2]

Added [1] net(s) to group 'bus1'.

Deleted [1] net(s) from user-defined group 'bus1'.
```

Related Commands

- report_bundled_bus

fixACLimitViolation

```
fixACLimitViolation
[-help]
[-excludeIO {true | false}]
[-excludeNetFile filename]
[-fixNetCategory {dataOnly | clockOnly | clockAndData}]
[-fixingMethod {route_rule | down_size | buffer}]
[-postCTS]
[-selectNetFile filename]
[-useReportFile filename]
[-useSegmentBasedNDR {true | false}]
```

Repairs violations associated with AC current density, wire-self-heat, Joules heat, or electromigration, mostly in a postRoute stage. Fixing methods are applying routing rules, downsizing drivers or adding a buffer to split up the net load on the violated nets. Use this command in conjunction with the `verifyACLimit` command. The `verifyACLimit` command can mark the violating net for the `fixACLimitViolation` command to repair, and it can generate the report file that `fixACLimitViolation` can read.

The `fixACLimitViolation` command only applies a single fixing method to fix EM violations. You can select the fixing method by specifying the `-fixingMethod` parameter. If the method is not specified, the command will fix EM violations by setting route rules on violating routes. For example, the following usage will apply route rules for both clock and data nets while excluding nets with I/O pins:

```
fixACLimitViolation -fixNetCategory clockAndData -excludeIO true
```

Before running `fixACLimitViolation`, you can generate an incoming EM violation report using the `verifyACLimit` command. This operation will ensure that the appropriate data for `fixACLimitViolation` is included in the violation report. After that, this report should be provided as an input to `fixACLimitViolation` using the `-useReportFile` parameter. The following is an example of the sequence of operations:

```
verifyACLimit -report beforeFix.rpt -view hold_view

fixACLimitViolation -useReportFile beforeFix.rpt

verifyACLimit -report afterFix.rpt -view hold_view
```

By default, `verifyACLimit` will calculate EM currents using the hold view. This means that `verifyACLimit` will internally invoke a hold timing update. If it is desired to maintain the current setup timing graph in memory for use after EM fixing, the `-forceHoldView` parameter should be provided to `verifyACLimit` so that the hold timing update doesn't nullify the setup timing graph. Below is an example of this scenario:

```
timeDesign -postRoute
verifyACLimit -report beforeFix.rpt -forceHoldView
fixACLimitViolation -useReportFile beforeFix.rpt
verifyACLimit -report afterFix.rpt -forceHoldView
```

Note: It is important to set the right switching activity factor on the nets before using the `verifyACLimit` command. Refer to the `set_power_analysis_mode`, `set_default_switching_activity` and `set_switching_activity` commands for further details on the default behaviors of the software and how to modify them. The example below shows how to set the global switching activity for verifying and fixing AC limit violations.

```
set_default_switching_activity -global_activity 0.2
propagate_activity -set_net_freq true
verifyACLimit -use_db_freq -report em.report
fixACLimitViolation -useReportFile em.report
```

Alternatively, `verifyACLimit -toggle` parameter can be used for more pessimistic analysis. In this case, modifying `set_default_switching_activity` is not needed. The following is an example of forcing every signal net to toggle at 50% of the clock rate:

```
verifyACLimit -toggle 0.5 -report em.report
fixACLimitViolation -useReportFile em.report
```

See `verifyACLimit` for further command parameters and usage.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `fixACLimitViolation` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man fixACLimitViolation`
`-excludeIO {true | false}`	If set to `true`, excludes nets connected to I/O pins during violation fixing. *Default:* `false`
`-excludeNetFile` *filename*	Specifies a file that contains the names of nets to exclude from AC limit violation repair.
`-fixNetCategory { dataOnly | clockOnly | clockAndData}`	Specifies fixing net types. For `dataOnly`, fixes data nets only. If `clockOnly` is specified, fixes clock nets. If `clockAndData` is specified, fixes clock and data nets. **Note**: For clock nets, only route rules will be used for fixing. Downsizing or buffering will not be used. *Default:* `dataOnly`
`-fixingMethod {route_rule | down_size | buffer}`	Specifies the fixing method for correcting EM violations. You can specify any of the following: • `route_rule`: routing rules will be applied for fixing violations • `down_size`: driver cell downsizing will be applied for the violating nets • `buffer`: buffering will be applied to the violating nets. *Default*: `route_rule`
`-postCTS`	If specified, enables fix for the clock nets, for postCTS DBs. This parameter should be used only with `-fixNetCategory clockOnly`. *Default*: `omitted`
`-selectNetFile` *fileName*	Specifies a file that contains the names of nets for which to perform AC limit violation repair.
`-useReportFile` *filename*	Reads the `verifyACLimit` report, and uses the violation information from the report file to set up a proper routing rules. This is recommended for the best fixing performance.
`-useSegmentBasedNDR {true | false}`	Enables/disables segment-based route rules. If set to `true`, generates and applies segment-based route rules for fixing violations. This approach enables you to specify the desired width and distance for each layer of the violated net. *Default*: `true`

Examples

- The following command performs buffering only, for the nets specified in the `netFile.txt` only:
  ```
  fixACLimitViolation -fixingMethod buffer -selectNetFile "netFile.txt"
  ```
- To use downsizing, excluding nets with I/O pins:
  ```
  fixACLimitViolation -fixingMethod down_size -excludeIO true
  ```
- To fix both clock and data nets with route rules:
  ```
  fixACLimitViolation -fixNetCategory clockAndData
  ```
- The following example shows how to feed `verifyACLimit` report through the `-useReportFile` parameter.
  ```
  fixACLimitViolation -useReportFile em.report
  ```

Related Commands

- `set_power_analysis_mode`
- `set_default_switching_activity`
- `set_switching_activity`
- `timeDesign`
- `verifyACLimit`

Related Topics

- The following chapters in the *Innovus User Guide*:
  - Identifying and Viewing Violations
  - "Optimizing Signal EM Violations at PostRoute Stage" in the Optimizing Timing chapter

get_glitch_threshold

```
get_glitch_threshold
[parameter_names]
```

Displays information for the specified `set_glitch_threshold` parameters for the current Tempus session. If you do not specify a parameter, the software displays information for all `set_glitch_threshold` parameters.

Parameters

parameter_names	Displays information for the specified parameter. You can specify one or more parameters. See `set_glitch_threshold` for descriptions of the timing analysis mode parameters you can specify.

Examples

- The following command shows the failure threshold for ROP for single stage cells:
  ```
  get_glitch_threshold -failure_point last
  ```
  Here's the command output:
  ```
  value=0.4 -threshold_type reporting -glitch_type both -pin_type all -failure_point last -failure_dependency none
  ```
- The following command shows the failure threshold for the ROP for the internal net of multi-stage cells:
  ```
  get_glitch_threshold -failure_point internal
  ```
  Here's the command output:
  ```
  value=0.2 -threshold_type reporting -glitch_type both -pin_type all -failure_point internal -failure_dependency none
  ```
- The following command shows the failure threshold for the RIP for the input net of cells without CCSN/ECSMN/cdB
  ```
  get_glitch_threshold -failure_point input
  ```
  Here's the command output:
  ```
  value=0.5 -threshold_type reporting -glitch_type both -pin_type all -failure_point input -failure_dependency none
  ```

Related Commands

- `set_glitch_threshold`

getSIMode

```
getSIMode
[-help]
[-accumulated_small_attacker_factor]
[-accumulated_small_attacker_mode]
[-accumulated_small_attacker_threshold]
[-analysisType]
[-attacker_alignment]
[-clock_delta_delay_threshold]
[-clocks]
[-delta_delay_annotation_mode]
[-delta_delay_threshold]
[-enable_double_clocking_check]
[-enable_glitch_propagation]
[-enable_glitch_propagation_spice_deck]
[-enable_glitch_report]
[-enable_logical_correlation]
[-hold_slack]
[-individual_attacker_clock_threshold]
[-individual_attacker_simulation_filtering]
[-individual_attacker_threshold]
[-initial_si_iteration_tw]
[-input_glitch_full_propagation_threshold]
[-input_glitch_full_propagation_vh_threshold]
[-input_glitch_full_propagation_vl_threshold]
[-input_glitch_threshold]
[-input_glitch_vh_threshold]
[-input_glitch_vl_threshold]
[-nonDefault]
[-nonlinear_attacker_slew]
[-num_si_iteration]
[-pessimistic_mode]
[-receiver_clk_peak_limit]
[-receiver_latch_peak_limit]
[-receiver_peak_limit]
[-report_si_slew_max_transition]
[-secondary_attacker_decoupling_factor]
[-separate_delta_delay_on_data]
[-setup_slack]
[-si_reselection]
[-si_reselection_delay_threshold]
[-skip_tw]
[-switch_prob]
[-unconstrained_net_use_inf_tw]
[-use_infinite_TW]
[-quiet]
```

Displays the following information about the specified `setSIMode` parameters in the Innovus log file and the Innovus console:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify any parameters, the software displays information for all of the parameters.

Parameters

parameter_names	Displays information for the specified parameters. You can specify one or more parameters. See `setSIMode` for descriptions of the parameters you can specify.

`-quiet`	Displays the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software displays the current settings of all the `setSIMode` parameters in Tcl list format.

Examples

- The following command displays the current setting of the `-analyzeNoiseThreshold`, and `-deltaDelayThreshold` parameters:

  ```
  getSIMode -analyzeNoiseThreshold -deltaDelayThreshold
  ```

  The software displays the following information:

  ```
  -analyzeNoiseThreshold 20 # int, default=20
  -deltaDelayThreshold -1 # float, default=-1
  {analyzeNoiseThreshold 20} {deltaDelayThreshold -1}
  ```

- The following command displays the current settings for all `setSIMode` parameters in Tcl list format only:

  ```
  getSIMode -quiet
  ```

  The software displays the following information:

  ```
  {acceptableWNS same} {analyzeNoiseThreshold 20} {deltaDelayThreshold -1} {insCeltICPostTcl {}} {insCeltICPreTcl {}} {maxNrIter 2}
  {noiseTwfMode {}} {targetPathGroups {reg2reg in2out in2reg reg2out}} {usePrevCeltICRunDir {}{vaDecoupleThreshold 10} {vaThreshold 15}
  {vaMode cap}}
  ```

Related Commands

- `setSIMode`
- `reportDelayCalculation`
- `report_timing`
- `report_noise`
- `update_glitch`

readTransitionFile

```
readTransitionFile
-file fileName
[-view viewName]
```

Reads external transition files that are used to perform maximum transition violation reporting and fixing.

The contents of a transition file must be in the following format:

```
setTranTime [-add] [-maxR slewValue | -maxF slewValue] {pinName}
```

or

```
setTranTime [-add] [-minR slewValue | -minF slewValue] {pinName}
```

Where:

- `-add`: Specifies that the transition value is incremental.
- `-maxR | -maxF`: Specifies the maximum rise or fall transition.
- `-minR | -minF`: Specifies the minimum rise or fall transition.

  Note: The minimum transition values will be ignored while performing transition violation fixing.
- *`pinName`*: Specifies the pin name for which the transition value is specified. To specify multiple pin names, you need to use separate `setTranTime` statements as given below:

  ```
  setTranTime -add -maxR 3.5 inst/A
  setTranTime -add -maxR 3.5 pinB
  ```

Note: The `readTransitionFile` command uses the time unit that is written in the slew file. If time unit is specified as 1e-12, it reads the values in picoseconds. For time unit specified as 1e-09, the command reads the values in nanoseconds. For example,

```
# Time Unit : 1e-09 seconds
setTranTime -minR 0.80050 {AAA/BBB/CCC/A}
setTranTime -minF 0.580300 {AAA/BBB/DDD/A}
```

When a transition file is read and third-party slew values are annotated, Innovus uses the following command to report transition violations with the annotated slews:

```
timeDesign {-postRoute | -signoff | -reportOnly} -useTransitionFiles
```

Parameters

`-file` *`fileName`*	Specifies the name of the file that contains information about the transition time.
`-view` *`viewName`*	Specifies the view name for which the transition file will be read in case of MMMC designs. You need to ensure that you specify the transition files for all setup views to perform transition violation fixing. **Note**: This parameter is required for MMMC designs only.

Example

- The following command reads an external transition file called `sampleFile` for non-MMMC designs:

  ```
  readTransitionFile -file sampleFile
  ```
- The following commands read view specific external transition files for a MMMC design:

  ```
  readTransitionFile -view view1 -file sampleFile1
  readTransitionFile -view view2 -file sampleFile2
  ```

report_bundled_bus

```
report_bundled_bus
[-help]
[-name string]
[-nets string]
[-type {user_defined | tool_inferred}]
[[ > | >> ]]
```

Generates a report for bundled buses.

Parameters

-help	Prints out the command usage.
-name *string*	Specifies the name of the user-defined bus group(s) to be reported.
-nets *string*	Specifies the name of the net(s) to be reported.
-type {user_defined | tool_inferred}	Reports the type of bundled bus(es) - user-defined or tool-inferred - to include in the report. By default, the `report_bundled_bus` command generates reports for both user-defined and software-inferred type of buses.
> *filename*	Redirects the output to the specified file.
>> *filename*	Appends to the specified output file.

Examples

- The following command shows the user-defined bus groups:

```
report_bundled_bus

Summary:
Total number of user defined bus groups reported = 3
Total number of tool inferred bus groups reported = 41

-----------------------------------------------------
Group Name (user_defined) : grp1

Constituents:
- EJ_PartNumber[15]
- EJ_PartNumber[14]
- EJ_PartNumber[13]
- EJ_PartNumber[12]
- EJ_PartNumber[11]
- EJ_PartNumber[10]
- EJ_PartNumber[9]
- EJ_PartNumber[8]
- EJ_PartNumber[7]
- EJ_PartNumber[6]
- EJ_PartNumber[5]
- EJ_PartNumber[4]
- EJ_PartNumber[3]
- EJ_PartNumber[2]
- EJ_PartNumber[1]
- EJ_PartNumber[0]
-----------------------------------------------------
Group Name (user_defined) : grp2

Constituents:
- EJ_ManufID[10]
- EJ_ManufID[9]
- EJ_ManufID[8]
```

Related Commands

- define_bundled_bus

report_double_clocking

```
report_double_clocking
[-help]
[-file string]
[-max_delta_delay float]
[-nworst integer]
[-reverse_slope_limit float]
[-reverse_voltage_change_threshold float]
[-view string]
```

Reports noise impact on clock nets by generating information on the double clocking, transition type, and reverse slope.

The report output displays violations in sorted order - based on the absolute value of the slope, a negative value indicates a reverse slope.

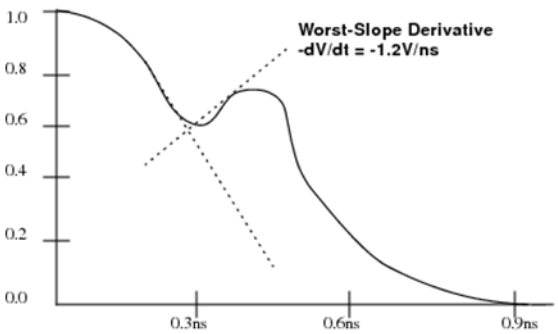
Note: The `report_double_clocking` command requires that clocks have been defined and applied prior to timing analysis with SI (`update_timing` or `report_timing`). The `report_double_clocking` command does not work with the glitch-only flow (`update_glitch`).

Double clocking occurs when the noise glitch attacks a transitioning signal. The multiple transition or double clocking check supplements the transition time check performed in static timing analysis. Transition time is defined as the duration of time that a signal takes to make the transition between two threshold points. However, transition time does not account for the waveform shape between the voltage thresholds.

For special signals such as clocks it is important to analyze the waveform shape between the voltage threshold in case the signal transitions slowly or in the opposite direction (even for a short duration of time) causing multiple clock edges. This command enables you to report the worst-case slope (dV/dtfor rising signal and -dV/dt for falling signal) of a signal transitioning between the voltage thresholds. The slope derivative is measured in units of volts per nanosecond (V/ns). This value can be positive if the signal does not change direction, 0 if the signal is flat, and negative if the signal changes direction even for a short duration of time.

The following figure shows two falling signal transitions:

Figure: Negative Worst-Slope Derivative



In the above figure, the falling signal transitions have identical transition times. However, the worst-case slope derivatives for the two falling waveforms are different. If a clock signal exhibits a negative slope derivative, there is a possibility of a double clocking functional failure. A higher reverse slope results in a higher possibility of a double clocking functional failure.

The double clocking report contains the following sections and their corresponding fields:

- **Double Clocking**
  - Rise Slope (V/ns): Specifies the worst reverse slope of the rising clock signal as a measure of volt per nano-second.
  - Fall Slope (V/ns): Specifies the worst reverse slope for the falling clock signal as a measure of volt per nano-second.
  - VictimNet: Specifies the name of the victim net.
- **Instance Pin**: Displays the name of the instance pin.
- **Transition Type:** Indicates the type of transition the net is undergoing – R for rising, F for falling.
- **Reverse Slope:** Indicates the slope that is opposite to the transition type and the value reported is the worst reverse slope.

Parameters

`-help`	Prints out the command usage.
`-file <string>`	Specifies the output filename. *Default*: {$parm_filename_prefix}_clock.text
`-max_delta_delay float`	Reports clock nets, for which the delay change caused due to noise, exceeds the specified number. *Default*: 0
`-max_slew float`	Reports clock nets for which the transition time, including SI, exceeds the specified number. *Default*: 0
`-nworst integer`	Specifies the number of clock nets to report in each category. *Default*: 1
`-reverse_slope_limit float`	Reports the clock nets for which the slope is below the specified number. *Default*: 0
`-reverse_voltage_change_threshold float`	Controls the reverse slope failure criteria by allowing you to supply a minimum voltage change as a percentage of VDD. The reverse slope must be sustained in order for it to be considered as failing. *Default*:0.1
`-view string`	Specifies the view name for which the report to be generated.

Examples

- The following command reports clock nets that have a reverse slope less than -1:
  ```
  report_double_clocking -reverse_slope_limit -1
  ```
- The following command reports 100 clock net violations of each category:
  ```
  report_double_clocking -nworst 100
  ```
- The following command shows the double clocking report for `default_emulate_view` view:

  ```
  report_double_clocking -view default_emulate_view

  ********************************************************************************
  Double Clocking Report For View: default_emulate_view
  Generated: Thu Mar 12 08:43:01 2015
  ********************************************************************************
  Instance Pin                                    Transition Type Reverse Slope

  DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/acc_reg_13/CK    R               -0.32465
  DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/acc_reg_13/CK    R               -0.32465
  ```

Related Commands

- report_timing
- report_noise

report_instance_cdb

```
report_instance_cdb
[-help]
[-analysis_type {early | late}]
[{instance_list or collection}]
[-output_file filename]
[-quiet]
[-view viewname]
```

Allows you to report the cdB associated with all the instances or a subset of instances in a design.

Note: This command must be issued after `report_timing`, `update_timing`, or `update_glitch` commands.

Parameters

`-help`	Outputs a brief description that includes type and default information for each`report_instance_cdb` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_insance_cdb`
`-analysis_type { early | late }`	Specify the analysis type for the query. *Default*: `late`
instance_list or collection	
	Specify the instance for which the cdB association is desired. By default, the cdB association for all the instances is reported.
`-output_file` *filename*	Specify the output filename for the instance report to go to. By default, the report output is displayed on the screen.
`-quiet`	Suppresses warning messages for cells missing in CDB files.
`-view {`*viewnames*`}`	
	Specifies the view names. By default, values for all the views are reported.

Examples

- The following command reports the cdB instance connection for all the instances in the design and write it to the inst_cdb.rpt file:

```
> report_instance_cdb -output_file inst_cdb.rpt
```

- The following command reports the cdB instance connection for the u5 instance to the screen:

```
> report_instance_cdb u5

VIEW: default_emulate_view -- ANALYSIS TYPE: late
u5 : inv_t1 : test1.cdb.txt
```

- The following command reports the cdB instance connection for the early analysis type for the u5 and u6 instances

```
> report_instance_cdb -analysis_type early {u5 u6}

VIEW: default_emulate_view -- ANALYSIS TYPE: early
u5 : inv_t1 : test1.cdb.txt
u6 : inv_t2 : test2.cdb.txt
```

Related Commands

- `report_timing`
- `update_glitch`

report_noise

```
report_noise
[-help]
[-bumpy_waveform]
[-cell_type {bbox | clock | data | latch}]
[-delay {min | max}]
[-double_clocking]
[-exclude_failures]
[-expand_virtual_attacker_bus]
[-failure]
[-gui_victim_nets_file]
[-format string]
[-histogram]
[-level {vl | vh | vlu | vho}]
[-max_threshold float]
[-merge_dir string]
[-nets string]
[-output_format string]
[-output_file string]
[-overshoot_undershoot]
[-pins string]
[-precision integer]
[-quiet_nets]
[-remove_status_row <string>]
[-report_virtual_attacker_constituents]
[-reverse_voltage_change_threshold float]
[-slack float]
[-skip_nets_or_pins_in_header]
[-sort_by string]
[-style {default | extended}]
[-threshold float]
[-txtfile filename[.gz]]
[-view]
```

Generates text-based noise reports.

Note: You need to ensure that the `setSIMode -enable_glitch_report` (to generate glitch report) or `-enable_delay_report` (to generate detailed delay report) parameters are set to `true`, before the `report_timing`, `update_timing`, or `update_glitch` commands are run.

Parameters

`-help`	Prints out the command usage.
`-bumpy_waveform`	Flags nets with bumpy waveforms at the receiver. **Note**: This parameter will always show maximum delay bumpy report (even if the `-delay min` option is specified).
`-cell_type {bbox | clock | data | latch}`	Generates a glitch report for the user-specified cell-type. You can specify one of the following options: • `bbox`: Generates a report for only the black-box receivers • `clock`: Generates a report for only the clock pin receivers • `data`: Generates a report for only the data pin receivers • `latch`: Generates a report for only the latch pin receivers
`-delay {min | max}`	Reports delay uncertainty instead of noise values. The only legal values are `min` or `max` for minimum or maximum path delay analysis.
`-double_clocking`	Enables double clocking reporting.
`-exclude_failures`	Excludes all failing nets from the glitch report.
`-expand_virtual_attacker_bus`	Reports virtual attacker components and expands bus bits in the noise reports. This parameter works only when the bus virtual attacker feature is enabled using the `setSIMode -enable_virtual_attacker_constituent_report true` setting.

`-failure`	Reports receiver peak failures and DC tolerance failure nets. Use this option only for glitch noise failure reports.
	The `report_noise`, without the `-failure` option, will use 0.10 of vdd as default.
	When you specify `-failure` option you need to use `-threshold` to lowest order of noise that is saved, if not specified which is 0.05 of Vdd.
	Option: Directory Name:
	`-sort_by noise glitch`
	`-sort_by rcvr_peak glitch`

`-format` *string*	Specifies the output glitch report format fields. The valid fields are: arc, attacker_edge, attacker_logical_correlation_phase, attacker_net, attacker_offset, attacker_peak, attacker_slew, attacker_source, attacker_status, attacker_vdd, attacker_xcap, cell_type, delay, glitch_slack, incr_delay, level, peak, receiver_cell, receiver_input_failure_threshold, receiver_input_pin, receiver_output_failure_threshold, receiver_output_peak, slack, slack_status, stage_info, thresh_source, verbose_logical_correlation_report, victim_edge, victim_library, victim_logical_correlation_phase, victim_net, victim_slew, victim_total_area, victim_total_cap, victim_total_cross_cap, victim_vdd, victim_width Each field corresponds to the following columns of the glitch report: • arc: Arc • attacker_edge: Edge • attacker_net: Net • attacker_offset: Offset (ps) • attacker_peak: Peak (mV) • attacker_slew: Slew (ps) • attacker_source: Constituents: Source • attacker_status: Statue • attacker_vdd Vdd (V) • attacker_xcap: Xcap (fF) • glitch_slack: Slack • level: Level • peak: Peak (mV) • receiver_cell: Receiver cell • receiver_input_failure_threshold: Receiver Input Threshold • receiver_input_pin: ReceiverNet • receiver_output_failure_threshold: Failure threshold • receiver_output_peak: Receiver output peak Value • stage_info • thresh_source: ReceiverThresholdType (Global | User | Library) • victim_library: Library • victim_net: VictimNet • victim_total_area: TotalArea • victim_total_cap: TotalCap (fF) • victim_vdd: Vdd (V) • victim_width: Width (ps) The following options are not available with the single line format: • delay • incr_delay • slack • thresh_source • victim_edge • victim_slew • victim_total_cross_cap To choose whether the report format should be on a single line or multiple lines, you can use the `-output_format` parameter.

`-gui_victim_nets_file`	Generates the GUI format glitch reports (`aae or aae_<viewname>` files under the reporting directory) and incremental delay reports (`aae_<max/min>_incDelay or aae_<viewname>_<max/min>_incDelay` files under the reporting directory) to be used in the Display Noise Net form.
`-histogram`	Generates the noise histogram.
`-level {vl | vh | vlu | vho}`	Specifies the level for which a report needs to be generated when single-level report is desired. You can specify one of these options - `vl`, `vh`, `vlu`, and `vho`.
`-max_threshold` *float*	Specifies the maximum peak noise or receiver peak thresholds to report on.
`-merge_dir` *string*	Takes a directory name and generates a software compatible database.
`-nets` *string*	Reports the specified nets in the noise report.
`-output_file` *string*	Specifies the name of the output file name prefix. The name of the output file is generated by combining the prefix and the specified view name. If you do not specify this option: • The software automatically names the text report as 'glitch' for noise glitch reports and 'delay' for noise delay reports. • The software dumps the generated report to the console window for better usability. **Note**: If the text report file with the `.gz` extension is used, the output file is gzipped to generate a compressed report.
`-pins` *string*	Specifies the name of the pins to be reported.
`-precision` *integer*	Specifies the precision value up to which the glitch data is to be printed. The allowed values are 0, 1, 2 and 3.
`-output_format` *string*	Specifies if the output format should be on a single line or on multiple lines. You can specify one of the following options: • `single_line` • `multiple_line` If the `single_line` option is specified and the attacker info is chosen as a field, then all the attacker related format options will be printed together. The `-output_format` parameter only works with the `-format` option and does not affect the default reporting output format. *Default*: `multiple_line`
`-overshoot_undershoot`	Reports the overshoot (VHO) and undershoot (VLU) glitches. The `setSIMode -enable_overshoot_undershoot_glitch` must be set to `true` before the analysis is performed, in order to generate the overshoot-undershoot glitch report. *Default*: If this option is not used during glitch reporting, then VL and VH type of glitches will be reported.
`-quiet_nets`	Generates a report of all the quiet attackers.
`-remove_status_row <string>`	Shows only the active attackers and filters out Sb, UF, and LC from the glitch report.
`-report_virtual_attacker_constituents`	Specifies the maximum slack value for a bumpy net.
`-reverse_voltage_change_threshold <float>`	Reports the clock nets with reverse swing greater than the user-specified threshold value (ratio of VDD). By default, the threshold value is set to 0.1.
`-slack` *float*	Generates a report for bumpy nets based on slack.
`-skip_nets_or_pins_in_header`	Generates the glitch report without a list of nets or pins in the report header.

`-sort_by` *string*	Requests the sorting of entries. You can sort by: • `noise` • `receiver_peak` • `slack` • `cell_type` The software determines the default HTML output directories based on the options you choose. Depending on the sorting option you select, the software sorts the report fields in the following order: **-sort_by Option Level 1 Level 2 Level 3 Level 4** rcvr_peak receiver peak noise netname level noise noise - netname level
	For example, if you use the `-sort_by noise` option, the first level of sorting will be based on noise, followed by receiver peak, netname, and level. **Note**: The report displays the `vl` noise level before the `vh` noise level. *Default*: `receiver_peak`
`-style {default | extended}`	Generates reports in one of the styles - `extended` or `default`. • `extended`: Generates extended reports with victim and aggressor timing windows, dropped aggressor slews, and reasons for dropped aggressors. Additionally, the report also shows driver pins and driver library pins. • `default`: Generates the default report, which is similar to the text report generated using the `-text` parameter.
`-threshold` *float*	Specifies the minimum peak noise, absolute delay uncertainty, relative delay uncertainty, and receiver peak thresholds to report on. *Default*: • 0.10 of vdd for nets - when the glitch report is generated using the `report_noise -sort_by noise` option. • 10 (ps) for nets - when a detailed delay report is generated using the `report_noise -delay` parameter). **Note**: • The `-threshold` parameter accepts percentage values. • All `-sort_by` values that represent voltages will be in mV.
`-txtfile` *filename[.gz]*	Specifies the name of the text report file. If the specified file name is compressed, with the .gz extension, the file is gzipped. If you do not specify the `-txtfile` option: • The software automatically names the text report as ‘glitch’ for noise glitch reports and ‘delay’ for noise delay reports. • The software dumps the generated report to the console window for better usability. If the `-txtfile` parameter is not used, then Innovus will output the SI report to the screen. When the `-txtfile` parameter is specified, then Innovus uses the user-specified name. In case of MMMC, Innovus uses the <txtfile Name>_<view Name> naming convention.
`-view`	Specifies the view name.

Examples

- The following command reports noise for the view named `default_emulate_view`:

```
report_noise -delay max -threshold 1600 -txtfile delay_max_format.rpt -format {attacker_slew  attacker_xcap  attacker_status
attacker_edge   attacker_net stage_info slack  victim_slew  victim_total_cross_cap victim_net }

*****************************************************
Noise Delay Report (for view default_emulate_view)
*****************************************************

Report Options:
-----------------------------------------------------
-threshold 1600 (ps)
-delay max
[-style default]
[-sortBy IncrDelay]
-----------------------------------------------------
-----------------------------------------------------
Slack(ps)   Slew(ps)   Total    Xcap(fF)    VictimNet
-2.973      0.635      277.574  seg3/u6/A   {seg3/n5}

Stage information:
            Cell       Pin
Driver:     INV        seg3/u5/Y
Receiver:   INV        seg3/u6/A

Constituents:
Slew(ps)   Xcap(fF)   Status   Edge   Net
331.500    138.787    INF      F      seg1/n5
358.667    138.759    SY       F      seg2/n5
-----------------------------------------------------
Slack(ps)  Slew(ps)   Total Xcap(fF)  VictimNet
14.906     0.597      278.115         seg1/u6/A {seg1/n5}

Stage information:
           Cell       Pin
Driver :   INV        seg1/u5/Y
Receiver : INV        seg1/u6/A

Constituents:
Slew(ps)   Xcap(fF)   Status   Edge   Net
279.500    139.280    INF      F      w3
359.333    138.807    INF      F      seg3/n5
-----------------------------------------------------
Slack(ps) Slew(ps)    Total Xcap(fF)  VictimNet
15.737    0.587       270.604         seg1/u12/A {seg1/n11}

Stage information:
           Cell    Pin
Driver :   INV     seg1/u11/Y
Receiver : INV     seg1/u12/A

Constituents:
Slew(ps)   Xcap(fF)   Status   Edge   Net
276.500    135.654    INF      F      w5
337.167    134.517    INF      F      seg3/n11
```

- The following command reports noise for the specified reporting fields:

```
report_noise -delay max -format {delay incr_delay victim_slew victim_total_cap victim_total_cross_cap victim_vdd victim_net
attacker_offset attacker_slew attacker_xcap attacker_vdd attacker_edge attacker_status attacker_net} -txtfile custom.rpt
--------------------------------
Delay(NomMax/Max) IncrDelay(ps) Slew(ps) TotalCap(fF) Total Xcap(fF) Vdd(V) VictimNet


-43.400/2418.200  2461.600      12.372   639.088      599.796        1.62   i1/A {n1}


Type  IncrDelay(ps) Delay(NomMax/Max)  VictimInst
```

```
R     3253.300      5457.100/8710.400  drv/Y {INV:A}

Constituents:
Offset(ps)  Slew(ps)  Xcap(fF)  Vdd(V)  Edge  Status  Net
-245.000    30.000    52.145    1.62    F     att1
-199.000    120.000   26.751    0.00    F     INF     ~n1
------------------------------
```

- The following command shows glitch values greater than 0.20 VDD as:

```
report_noise -threshold 0.20
Output report file name prefix is (null)
Info: Processing report for view default_emulate_view ...
############################################################
# Command: report_noise -threshold 0.20
# View: default_emulate_view
############################################################

Report Options:
--------------------------------------------------------------------------------
-type glitch
-threshold 20% (of vdd)
```

- To generate a glitch report, you can use the following command:
  ```
  report_noise -txtfile glitch.txt
  ```
- To generate a glitch report sorted by input noise, you can use the following command:
  ```
  report_noise -txtfile sort_by_glitch_input.txt -sort_by noise
  ```
- To generate a glitch failure report, you can use the following command:
  ```
  report_noise -txtfile glitch_failure.txt -failure
  ```
- To generate a glitch report for VL noises sorted by input noise, you can use the following command:
  ```
  report_noise -sort_by noise -level vl
  ```
- To generate a glitch report for only latch receiver pin cell types, you can use the following command:
  ```
  report_noise -txtfile latch_glitch.txt -cell_type latch
  ```

Related Commands

- report_timing
- report_timing_format
- check_noise

report_noise_lib_pin

```
report_noise_lib_pin
[-help]
[> filename]
```

Generates a report providing information on all the user-specified noise mappings in the design.

Parameters

`-help`	Prints out the command usage.
`> filename`	Writes the report to the specified file name. If the file already exists, the software will overwrite it.

Examples

- The following command generates the output report 'noise_mapping.rpt' which contains all the noise mappings present in the design:
  ```
  report_noise_lib_pin > noise_mapping.rpt
  ```

  noise_mapping.rpt

  ```
  ---------------------------------------------------
  from_pin                    to_pin
  ---------------------------------------------------
  library1/inverter/I         library2/macro/A
  library1/buffer/I           library3/memory/ADD[0]
  ---------------------------------------------------
  ```

Related Commands

- set_noise_lib_pin
- reset_noise_lib_pin

report_noise_propagation

```
report_noise_propagation
[-help]
[-level {VL | VH | worst | all}]
[-max_paths integer]
[-max_slack float]
[-nets string]
[-nworst integer]
[-pins string]
[-source_depth integer]
[-views string]
[ [ > | >> ] ]
```

Prints text-based noise propagation reports that show glitch-source tracing for all the user-specified or all-violating receiver pins. The report allows you to view full paths showing the glitch propagating from the source to the violating receiver pin. This helps to easily identify and fix glitch violations.

Note: You need to set `setSIMode -enable_glitch_report` parameter to `true` to enable glitch reporting.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command and all of its parameters, use the `man` command: `man report_noise_propagation`
`-level {VL | VH | worst | all}`	Reports paths for specified glitch level. You can specify one of these options - VL, VH, worst, or all.
`-max_paths integer`	Reports the specified number of paths.
`-max_slack float`	Reports paths with glitch slack equal to or lesser than the specified value.
`-nets string`	Reports paths with endpoint being all the input pins connected to the specified net.
`-nworst integer`	Specifies the maximum number of sources per endpoint for which paths need to be reported.
`-pins string`	Reports the paths for the specified endpoint pins.
`-source_depth integer`	Specifies the maximum number of sources to be traced in any given branch in the fanin cone of violation.
`-views string`	Reports the paths for the specified view.
`> filename`	Redirects output to the specified file.
`>> filename`	Appends output to the specified output file.

report_voltage_scaling

```
report_voltage_scaling
[-help]
[objectListOrCollection]
[-analysis_type {early | late}]
[-view string]
[-output_file string]
```

Reports instances whose timing data is interpolated due to voltage mismatch. When a cell is operating in the design at a voltage different from voltages at which it is characterized in the timing libraries, delay calculation (DC) interpolates/derives the timing data at the operating voltage.

Parameters

`-analysis_type {early | late}`	Specifies whether early or late views have to be checked for voltage scaling. If unspecified, only 'late' views are checked.
`-output_file string`	Specifies the name of the output file to which the report will be written. When the specified file name contains a directory path, the directory hierarchy is automatically created, if not present. If this option is not specified, the report is written to standard output (screen).
objectListOrCollection	Specifies the name(s) or a collection of leaf-level instance(s) for which voltage scaling needs to be checked and reported. If not specified, all instances of the design are checked.
`-view string`	Specify the name(s) of view(s) to be checked for voltage scaling. If unspecified, all active analysis views are checked.

Examples

The following example shows a sample `report_voltage_scaling` report:

```
tempus 2 > report_voltage_scaling i_6 -analysis_type late -view func_worst_cmax

VIEW: func_worst_cmax -- ANALYSIS TYPE: late
<pin-name> : <cell-name> : <actual-voltage> : <low-lib-name> : <high-lib-name>
i_6/Y      : BUFX5MTH    : 0.820            : elc_08v        : elc_09v
```

The report is generated with the `func_worst_cmax` view. You can use the `-view` parameter to report for other views.

You can run the `report_analysis_views` command to list views.

report_voltage_swing

```
report_voltage_swing
[-help]
-high_threshold float
-low_threshold float
[-output_file string]
[-required_width {all | clock}]
[-view string]
```

Reports the voltage swings that are shorter than the user-specified thresholds for both clock and data signals.

Parameters

`-help`	Outputs a brief description of the `report_voltage_swing` command. For a detailed description of the command, use the man command: `man report_voltage_swing`
`-high_threshold float`	Specifies the high threshold percentage of the pin supply voltage.
`-low_threshold float`	Specifies the low threshold percentage of the pin supply voltage.
`-output_file string`	Specifies the name of the output file.
`-required_width {all | clock}`	Reports the required high and low pulse widths corresponding to the user-specified high and low thresholds. You can specify one of the following options: • `clock`: Reports the required widths for all clock pins in the design. • `all`: Reports the required pulse widths for all input pins in the design. *Default*: `all`
`-view string`	Specifies the name of the view being defined. **Note**: If this option is not specified, it will dump report for all views.

Examples

An output file in the text format containing violating pin information is presented below:

```
View Name: default_setup_view1
Hpin     Type_of_Voilation   Vdd    Vss    Slews(Rise/fall)  Margin (threshold)
X/buf/A (upper/lower)       0.9v   0.0V   210ps/240ps       745mv (810mv)
```

reset_glitch_derate

```
reset_glitch_derate
[-help]
[-derate_height]
[-derate_width]
[-glitch_type string]
[-instance_pin string]
[-offset]
[-pin string]
[-view string]
```

Removes the derating factors for glitch waveforms that were previously set using the `set_glitch_derate` command.

Parameters

`-help`	Prints out the command usage.
`-derate_height`	Resets the derate height.
`-derate_width`	Resets the derate width.
`-glitch_type string`	Specifies the glitch type vl, vh, or both.
`-instance_pin string`	Specifies the instance pin names on which derate factors are to be reset.
`-offset`	Resets the derate offset.
`-pin string`	Specifies the library pin names on which derate factors are to be reset.
`-view string`	Specifies the view name.

Related Commands

- `set_glitch_derate`

reset_noise_lib_pin

```
reset_noise_lib_pin
[-help]
[-all]
[-to string]
```

Allows you to reset any noise property mapping, specified previously using the `set_noise_lib_pin` command.

Parameters

`-help`	Prints out the command usage.
`-all`	Resets all the noise mappings in the design.
`-to string`	Specifies the library pin name for which the user-specified noise mapping (if any) needs to be reset. **Note**: The `-to` parameter accepts library cell pin names (that can be obtained using the `get_lib_pins` command). This parameters does not honour collections or instance pin names.

Examples

- The following command resets any noise mapping previously applied to the library pin library1/macro_cell/A :
  ```
  reset_noise_lib_pin -to library1/macro_cell/A
  ```
- The following command resets all the existing noise mappings in the design:
  ```
  reset_noise_lib_pin -all
  ```

Related Commands

- `set_noise_lib_pin`
- `report_noise_lib_pin`

set_annotated_glitch

```
set_annotated_glitch
[-help]
[-port string]
[-vh_glitch string]
[-vho_glitch string]
[-view string]
[-vl_glitch string]
[-vlu_glitch string]
```

Allows the ability to annotate glitches at any specified pin/port. This command setting will override the VH/VL glitch with the specified value on the given point and use that glitch value to propagate downstream.

If the defined annotated glitch waveforms do not start and end at 0, then the software will not consider those waveforms and a warning will be issued. In the following example both VL and VH waves should start and end with 0.0 values. Since the VH wave is not ending at 0, this waveform will be ignored.
`set_annotated_glitch -vl_glitch {0 0 100e-12 0.01 200e-12 0.04 300e-12 0.03 600e-12 0} -port dff1/CK -vh_glitch {0 0 100e-12 0.1 200e-12 0.4 300e-12 0.3 600e-12 0.1} .`

The software does not save (and report) waveforms below 5% of VDD. Consider for example, the annotated glitch defined on a pin as:
`set_annotated_glitch -port dff1/CK -vh_glitch {0 0 100e-12 0.01 200e-12 0.04 300e-12 0.03 600e-12 0}`

In this case, the glitch peak is 0.04 (40mV), which is below 5% of VDD (1080mV) - equal to 54mV. Since this annotated glitch waveform peak is very low, the `report_noise` command will not report it. However, if the failure criteria is lower than 5%, then the software will save and report small glitch violations.

Parameters

`-help`	Prints out the command usage.
`-port string`	Specifies port or pin names on which you need to apply glitches. This parameter works only when `setSIMode -enable_glitch_propagation true` is set in the `report_timing` flow. In the `update_glitch` flow, the `-enable_glitch_propagation` parameter is already set to `true`.
`-vh_glitch string`	Specifies a list of VH glitch wave points. You must specify at least 3 time value pairs.
`-vho_glitch string`	Specifies a list of VHO glitch wave points.
`-view string`	Specifies a list of view names.
`-vl_glitch string`	Specifies a list of VL glitch wave points. You must specify at least 3 time value pairs.
`-vlu_glitch string`	Specifies a list of VLU glitch wave points.

Examples

- The following example annotates the specified VL glitch wave points:
  ```
  set_annotated_glitch -vl_glitch {0 0.0  0.2e-9 0.540 0.405e-9 0} -vh_glitch {0.1e-9  0  0.205e-9 0.50 0.405e-9 0.200  0.6e-9 0.0} -port {A}
  ```
- The following command overrides the glitch value at RIP with the annotated glitch, because glitch is annotated at the receiver input:
  ```
  set_annotated_glitch -vl_glitch {0 0 100e-12 0.1 200e-12 0.4 300e-12 0.3 600e-12 0} -port seg2/u9/D
  update_glitch
  report_noise -nets seg2/n8

  ------------------------------------------------------------------------
  Peak(mV) Level TotalCap(fF) TotalArea Width(ps) Vdd(mV) Library VictimNet
  399.924  VL    144.11       110.00    550.104   1080    cell_w  seg2/u9/D {seg2/n8}

  Receiver Input Threshold:
  Value    (Threshold)  ReceiverNet Receiver failure_type
  399.924  (431.244)    seg2/u9/D   (DFF)    [bbox]

  Constituents:
  Source Peak(mV) Offset(ps) Slew(ps) Xcap(fF) Vdd(mV) Edge Status Net
  Prp:            399.924    --       --       --      --   --     seg2/u8/Y
  ```

```
-------------------------------------------------------------------------
```

- In the following example, the impact of aggressors coupled with the victim stage (along with the annotated glitch) is also considered because annotation is done at the driver output:

```
set_annotated_glitch -vl_glitch {0 0 100e-12 0.1 200e-12 0.4 300e-12 0.3 600e-12 0} -port seg2/u8/Y
update_glitch
report_noise -nets seg2/n8

-------------------------------------------------------------------------------
Peak(mV) Level TotalCap(fF) TotalArea Width(ps) Vdd(mV) Library  VictimNet
548.208  VL    144.11       159.30    581.172   1080    cell_w   seg2/u9/D {seg2/n8}

Receiver Input Threshold:
Value (Threshold) ReceiverNet Receiver failure_type
548.208 (432.648) seg2/u9/D   (DFF)    [bbox]

Constituents:
Source Peak(mV) Offset(ps) Slew(ps) Xcap(fF) Vdd(mV) Edge  Status  Net
Cpl:            80.148     371.000  340.000  68.138  1080  R ACT   seg3/n8
Cpl:            75.159     381.000  348.500  62.007  1080  R ACT   seg1/n8
Prp:            392.904    --       --       --      --    --      seg2/u8/Y
-------------------------------------------------------------------------------
```

Related Commands

- report_noise
- update_glitch

set_cdb_binding

```
set_cdb_binding
[-help]
-from <string>
-to <string>
```

Provides an overriding mechanism that allows overriding a UDN to Liberty mapping at the Tempus command file level.

This command helps to correct any mismapped cases by enabling the software to select a user-specified cdB to Liberty mapping.

Parameters

`-help`	Prints out the command usage.
`-from <string>`	Specifies the CDB file name and cell name.
`-to <string>`	Specifies the library file name.

Examples

When loading multiple cdB's with the same PVT, but different secondary voltages, you can use the `set_cdb_binding` command to ensure that they get bound correctly, as shown below:

```
set_cdb_binding -from pvt1_1.cdb -to pvt1_1.lib
set_cdb_binding -from pvt1_2.cdb -to pvt1_2.lib
```

set_glitch_derate

```
set_glitch_derate
[-help]
[-derate_height float]
[-derate_width float]
[-glitch_type string]
[-instance_pin string]
[-offset float]
[-pin string]
[-view string]
```

Sets the derating factors for glitch waveforms and determines the effect of on-chip variation on noise.

Parameters

`-help`	Prints out the command usage.
`-derate_height float`	Specifies the derate height value. This parameter scales all the glitch waveforms on the specified pins. A value of 1.0 indicates that no glitch waveform derating will take place. A value greater than 1.0, such as 1.1, indicates a 10 percent increase in the magnitude of the complete glitch waveform, which results in an increase in propagated noise and noise failure. A value less than 1.0 reduces the magnitude of the complete glitch waveform.
`-derate_width float`	Specifies the derate width value.
`-glitch_type string`	Specifies the glitch type - vl, vh, or all.
`-instance_pin string`	Specifies the instance pin names on which derate is to be applied.
`-offset float`	Specifies the derate offset value.
`-pin string`	Specifies the library pin names on which derate value is to be applied.
`-view string`	Specifies the name of the view.

Related Commands

- reset_glitch_derate
- update_glitch

set_glitch_threshold

```
set_glitch_threshold
[-help]
[-cell <string>]
[-failure_point {none | input | internal | last}]
[-glitch_type {both | vl | vh | vlu | vho | vl_vh | vlu_vho}]
[-library_pins <string>]
[-nets string]
[-pins string]
[-pin_type {clock | data | latch | all}]
[-threshold_type {failure | reporting | propagation}]
-value float
[-views string]
```

Allows for additional control of failure criteria for SI glitch analysis.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command and all of its parameters, use the `man` command: `man set_glitch_threshold`.
`-cell <string>`	Specifies list of library cell names.
`-failure_point {none | input | internal | last}`	Specifies where the failure or reporting criteria applies. ◦ `none`: Criteria does not apply. ◦ `input`: Threshold applies when the failure point is at the input pin of the receiver cell. ◦ `internal`: Threshold is applied when the failure point is internal to the cell. This occurs for multi-stage cells. The failure point will be the output of the first stage inside the cell. ◦ `last`: Threshold is applied when the failure point is the output of the cell. This occurs for single-stage cells, such as, inverters. *Default*: `internal` and `last`
`-glitch_type {both | vl | vh | vlu | vho | vl_vh | vlu_vho}`	Specifies the glitch type the command applies to. • `both`: Refers to `vl` and `vh` type of glitches. • `vl_vh`: Includes VL and VH noise. • `vlu_vho`: Includes VLU and VHO noise. *Default*: `both`
`-library_pins string`	Specifies a list of library pin names.
`-nets string`	Specifies the list of nets the command applies to. Default is not net-specific.
`-pins string`	Specifies the list of pins or primary ports. Default is not pin-specific.
`-pin_type {clock |data | latch |all}`	Specifies the pin type the command applies to. *Default*: all

`-threshold_type {failure | reporting | propagation}`	Specifies whether the command options apply to failure, reporting, or propagation criteria. In case of VL/VH glitch analysis, it is mandatory to specify the `-threshold_type` parameter. This is because when the computed glitch (ROP) at any given stage exceeds the failure threshold (when glitch propagation is enabled), then it is not propagated and is flagged as failure in the noise report. With the `-threshold_type failure` setting, the propagation will stop as per the new threshold, and the failure flagging will change accordingly. However, with the `-threshold_type reporting` setting, the propagation threshold remains unchanged, but the failure reporting is updated. When `-threshold_type propagation` is used, the software allows for failing nets with ROP values less than the propagation threshold to propagate through single stage cells. In case of VLU/VHO glitch analysis, it is optional to use the `-threshold_type` parameter because the above flow is not applicable. *Default*: `failure`
`-value float`	Specifies the value for the threshold as a ratio of the local VDD. *Default*: 0.1
`-views string`	Specifies a list of view names.

Examples

Here are some examples.

- The following command moves the failure point from ROP to RIP for `inst1/a1` and uses a RIP failure of *0.2*ReceiverVDD*:
  ```
  set_glitch_threshold -failure_point input -pin inst1/a1 -threshold_type failure -value 0.2
  ```
- The following command sets clock failure criteria to 0.1*VDD, latch failure criteria to 0.2*VDD and all other nets to 0.3*VDD (set before update_glitch):
  ```
  set_glitch_threshold -threshold_type failure -pin_type all -failure_point last -value 0.3
  set_glitch_threshold -threshold_type failure -pin_type clock -failure_point last -value 0.2
  set_glitch_threshold -threshold_type failure -pin_type latch -failure_point last -value 0.1
  ```
- The following command sets the failure threshold at the output of the 1st channel connect inside all the instances of macro1 to be 0.2 (set before update_glitch):
  ```
  set_glitch_threshold -threshold_type failure -cell {macro1} -failure_point internal -value 0.2
  ```
- The following command changes the reporting criteria for latches to be 0.15*VDD (set after `update_glitch`):
  ```
  set_glitch_threshold -threshold_type reporting -pin_type latch -value 0.15
  ```

  If you need to set all the data failure criteria to be 0.3, then you need to make the following settings:
  ```
  set_glitch_threshold -failure_point internal \
  -threshold_type failure -value 0.3 \
  -pin_type data

  set_glitch_threshold -failure_point last \
  -threshold_type failure -value 0.3 \
  -pin_type data
  ```

Related Commands

- `get_glitch_threshold`
- `update_glitch`

set_model_priority

```
set_model_priority
[-help]
[-objects string]
[-priority_list string]
```

Allows you to define the priority order of models.

Parameters

`-help`	Prints out the command usage
`-objects string`	Specifies a list of library cells or library pins for which priority is specified.
`-priority_list string`	Specifies a list of pin model priority. Default: `{ecsm ccs nldm cdb udn}`

Examples

To force CCSN to be used, you can set `timing_library_read_ccs_noise_data` global variable to `true` before reading the timing libraries and then use the following command before using the `update_glitch` command:

```
set_model_priority -priority_list ccs
```

Related Commands

- `update_glitch`

set_mutex_condition

```
set_mutex_condition
[-help]
[netNames]
```

Allows the software to use one of the specified attackers - each attacker is then evaluated and the worst one (for each victim net) is selected.

This command applies a mutex group to a set of attackers (for a victim) to reduce pessimism. A mutex group is a group of nets of which only one can transition in a given cycle.

Parameters

`-help`	Prints out the command usage.
netNames	Specifies the net names of the mutex group.

Examples

- The following example shows that only one of the specified nets, between abc[0-3], can switch at any given point of time:
  ```
  set_mutex_condition {abc[0] abc[1] abc[2] abc[3]}
  ```

  Each of the specified attackers will be evaluated and the worst one will be used.

set_noise_lib_pin

```
set_noise_lib_pin
[-help]
-from string
-to string
```

Allows you to map noise properties of an instance pin to that of different library cell pin.

Parameters

`-help`	Prints out the command usage.
`-from string`	Specifies the library pin name whose definition will be mapped from the given pin name.
`-to string`	Specifies the library pin name whose definition will be mapped to the given pin name. When multiple `set_noise_lib_pin` commands are run on the same `-to` pin, the software will honor the first command only. **Note**: The `-from` and `-to` parameters also accept library cell pin names (that can be retrieved using the `get_lib_pins` command). These parameters do not honor collections or instance pin names.

Examples

- The input pin A of the `macro_cell` cell inherits the noise characteristics of the input pin I of the library cell, `ccs_noise_lib/inv`. This information is used to calculate the noise immunity of the pin, `library1/macro_cell/A`.
  ```
  > set_noise_lib_pin -to library1/macro_cell/A -from ccs_noise_lib/inv/I
  ```
- The following syntax specifies the library pin name whose definition will be mapped from the given pin name to the given pin name:
  ```
  > set_noise_lib_pin -from [from_lib/from_cell/from_pin] -to [to_lib/to_cell/to_pin]
  ```
- The following syntax specifies that a bidi pin is mapped to two from pins; that is, – 1 input pin and 1 output pin.
  ```
  > set_noise_lib_pin -to library1/macro_cell/PAD -from "ccs_noise_lib/inv/I ccs_noise_lib/inv/Z"
  ```

Related Commands

- `report_noise_lib_pin`
- `reset_noise_lib_pin`
- `report_noise`

set_quiet_attacker

```
set_quiet_attacker
[-help]
[-analysis_type {early | late}]
[-attacker string]
[-transition {rise | fall}]
[-victim string]
[-view string]
```

Specifies one or more coupled nets as non-attackers (silent) for the delay and glitch analysis of a given victim net.

You can run the `update_glitch` or `update_timing` command and view the results in the `report_noise` or `report_timing` command output.

Parameters

`-help`	Prints out the command usage.
`-analysis_type {early | late}`	Specifies the analysis type for which the attackers will be assumed quiet. If no value is specified, attackers are skipped for both `early/late` analysis.
`-attacker string`	Specifies the attackers which should be made quiet for given victim net(s). If no attacker is specified, all the aggressors of given victim(s) will be ignored. **Note**: If no victim is specified, the specified attackers are made quiet for all victim nets in the design.
`-transition {rise | fall}`	Specifies the victim transition for which attackers will be assumed quiet. If no value is specified, attackers are skipped for both `rise` and `fall` transitions of the victim net.
`-victim string`	Specifies a list of victim nets for which attackers should be made quiet. Specific attackers can be skipped using the `-attacker` option of this command. If the `-attacker` option is not specified, all the attackers of the given nets are made silent and SI analysis will not be performed on these nets. **Note**: If this option is not specified, the attackers specified in the `-attacker` option are made quiet for all the nets in the design.
`-view string`	Specifies the view for which attackers should be made quiet. If no view is specified, attackers are assumed quiet in all views

Examples

Consider a design having an attacker `agg` to victim nets `vic1` and `vic2`. The following command sets the attacker `agg` as quiet for both the victim nets `vic1` and `vic2`.

```
set_quiet_attacker -attacker {agg}
```

- The following command sets the attackers A1 and A2 as quiet when V1 is rising during late analysis.

  ```
  set_quiet_attacker -victim V1 -attacker A1 A2 -transition rise -analysis_type late
  ```

- The following command sets the attacker A1 quiet for victim V1 in the analysis view VIEW1. In this case, A1 may be a valid attacker during SI analysis for victim V1 in views other than VIEW1.

  ```
  set_quiet_attacker -victim V1 -attacker A1 -view {VIEW1}
  ```

Consider a design having victim `vic` with attackers `agg1` and `agg2`:

- The following command sets both the attackers `agg1` and `agg2` as quiet for the victim net `vic`.

  ```
  set_quiet_attacker -victim {vic}
  ```

- The following command sets the attacker `agg1` as quiet for the given victim net `vic`

  ```
  set_quiet_attacker -attacker {agg1} -victim {vic}
  ```

Related Commands

- `report_noise`
- `report_timing`

setOutboundReport

```
setOutboundReport
[-help]
-delay {early | late | both}
[-filename_prefix string]
[-report_constant_nets {true | false}]
[-type {lower | upper | both}]
```

Controls what extrapolation information is to be saved in the log file(s). The slew extrapolation information can be stored into separate files.

When this command is not specified, slew extrapolation will be reported for both minimum and maximum analysis, if outbound reporting has been enabled by specifying the `setDelayCalMode -reportOutBound true` command.

Parameters

`-help`	Prints out the command usage
`-delay {early | late | both}`	Control whether extrapolation information is to be shown for min or max (or both) analyses. *Default*: `both`
`-filename_prefix string`	Specify the prefix to be used in the name(s) of the report file(s). *Default*: `design-name`
`-report_constant_nets {true | false}`	Controls whether extrapolation information is to be shown for constant nets or not.
`-type {lower | upper | both}`	Controls whether extrapolation information is to be shown for lower or upper (or both) outbound slews.

Examples

The following command will will dump slew extrapolation information for maximum analysis into the myreport.dc.outbound.slew.late.<view-name>.txt file:

```
setOutboundReport -delay late -filename_prefix myreport
```

Related Commands

- `setDelayCalMode`

setSIMode

```
setSIMode
[-help]
[-reset]
[-accumulated_small_attacker_factor double]
[-accumulated_small_attacker_mode {cap | current | zero_mean}]
[-accumulated_small_attacker_threshold double]
[-attacker_alignment {path | path_overlap | timing_aware_edge}]
[-clock_delta_delay_threshold double]
[-clocks {asynchronous|synchronous}]
[-constrained_input_threshold_failure_point {input | both}]
[-delta_delay_annotation_mode {lumpedOnNet | arc}]
[-delta_delay_threshold double]
[-enable_bus_attacker_correlation {true | false}]
[-enable_delay_report {true | false}]
[-enable_drv_with_delta_slew {true | false}]
[-enable_dynamic_receiver_peak_limits {true | false}]
[-enable_double_clocking_check {true | false}]
[-enable_glitch_propagation  {true | false}]
[-enable_glitch_propagation_spice_deck {true | false}]
[-enable_glitch_report {true | false}]
[-enable_logical_correlation {true | false}]
[-enable_overshoot_undershoot_glitch {true | false}]
[-enable_two_stage_driver_weakening {true | false}]
[-enable_virtual_attacker_constituent_report {true | false}]
[-glitch_accuracy_level <int>]
[-glitch_analysis_type {source | full_propagation}]
[-hold_slack double]
[-individual_attacker_clock_threshold float]
[-individual_attacker_simulation_filtering {true | false}]
[-individual_attacker_threshold double]
[-input_glitch_full_propagation_threshold double]
[-input_glitch_full_propagation_vh_threshold double]
[-input_glitch_full_propagation_vl_threshold double]
[-input_glitch_threshold double]
[-input_glitch_vh_threshold double]
[-input_glitch_vl_threshold double]
[-nonlinear_attacker_slew {true | false}]
[-num_si_iteration int]
[-pessimistic_mode {advanced | advanced_node | reduced_pessimism | increased_pessimism | high_pessimism}]
[-receiver_clk_peak_limit double]
[-receiver_latch_peak_limit double]
[-receiver_peak_limit double]
[-report_max_virtual_attacker_constituents int]
[-report_si_slew_max_transition {true | false}]
[-secondary_attacker_decoupling_factor double]
[-separate_delta_delay_on_data {true | false}]
[-setup_slack double]
[-si_reselection {delta_delay | slack}]
[-si_reselection_delay_threshold double]
[-skip_noise_model_check pins]
[-skip_tw nets]
[-switch_prob double]
[-unconstrained_net_use_inf_tw {true | false}]
[-use_infinite_TW {true | false}]
```

Sets signal integrity configuration parameters used for crosstalk analysis and repair.

Parameters

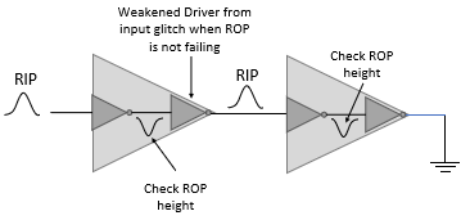
`-help`	Outputs a brief description that includes type and default information for each `setSI` parameters, use the man command: `man setSIMode`
`-reset`	Resets all or specified options to their default value. **Note**: Any values that do not already match the default value and are reset will be re

`-accumulated_small_attacker_factor` *double*	Specifies the multiplication factor that controls the effect of the cap mode accumulat attacker that represents the sum of all the weak attackers on a specified net. To redu value to anything between 0 and 1. A value of 0 means that the cap mode accumula *Default*: `1` **Note**: This option does not work with the current mode accumulated small attacker,
`-accumulated_small_attacker_mode {cap | current | zero_mean}`	Specifies whether to use the current-matching based method for accumulated small `cap`: Uses the accumulated small attacker method, which matches the total coupling virtualized. This is the default mode for all process nodes above 45nm. `current`: Uses the current-matching based method for accumulated small attackers. Piece Wise Linear (PWL) waveforms. The PWL waveforms are generated in a way induced by all accumulated small attacker components. `zero_mean`: A new accumulation method where series of small attackers, that is, atta transition in opposing directions and thus cancel one another in aggregate. *Default*: `current` - for 45nm and below process nodes.
`-accumulated_small_attacker_threshold` *double*	Changes the accumulated small attacker glitch threshold. If the virtual attacker prod coupling capacitors of the accumulated small attacker are split into two ground capa value is specified as a ratio of VDD. To disable the accumulated small attacker, set the value greater than or equal to 1. *Default*: `10.01` By default, this option is disabled.
`-attacker_alignment {path | path_overlap | timing_aware_edge}`	Specifies the attacker alignment. `path`: The victim worst arrival edge is used while determining the attackers. Crosstal victim for max/min analysis. `path_overlap`: The full timing window of victim is considered while determining the a anywhere in timing window, which may experience maximum impact. `timing_aware_edge`: Realistic analysis similar to path-based analysis is performed. I timing window edge aligns with the victim. However, to ensure GBA bounds PBA a timing properties in GBA analysis. *Default*: `path`
`-clock_delta_delay_threshold` *double*	Specifies the value above which the clock delta delay should be calculated. *Default*: `-1` The `-clock_delta_delay_threshold` parameter provides a way to specify different d
`-clocks {asynchronous | synchronous}`	Treats the clocks either synchronous or asynchronous to each other. *Default*: `synchronous`
`-constrained_input_threshold_failure_point {input | both}`	Allows you to control the glitch failure criteria when a user specifies an object-speci When set to `input`, the software skips ROP analysis and performs RIP-based failure threshold is set using the `set_glitch_threshold -failure_point input -net` (or `-pi` When set to `both`, the software performs both RIP and ROP failure checks, whereve flagged as a RIP violation. If RIP criteria passes and receiver noise models exist, th for failure determination. For the following cases where the object-specific RIP thresholds are not specified, t • For receiver pins with noise models present, ROP-based failure check is perform • For receiver pins with noise models missing, the RIP-based failure criteria is che *Default*: `input`

`-delta_delay_annotation_mode {lumpedOnNet | arc}]`	Calculates the delay for each arc (cell arc and net arc) in a path. By default, AAE ca view the SI delay separately, it would show annotated to each arc. Alternatively, you can choose to annotate the SI delay solely to the net. In this case, solely to the net. `lumpedOnNet`: Lumps the delta delay all onto the net if it is broken out. `arc`: Separates the delta delay onto each cell arc and net arc as calculated *Default*: `arc`
`-delta_delay_threshold <double>`	Specifies a threshold value for a design below which the crosstalk impact will be re impact on a stage if the impact is less than the specified threshold. The threshold va *Default*: -1 By default, the software uses the `-delta_delay_threshold` settings. If you set the `-c` nets. When a threshold value using the -`delta_delay_threshold` parameter is specified (a used), then Tempus will use the `-delta_delay_threshold` value for both data and cl If you define values for both `-delta_delay_threshold` and `-clock_delta_delay_thr` data and clock, respectively.
`-enable_bus_attacker_correlation {true | false}`	Enables the software to treat bits of bus as correlated attackers. When set to `true`, the bits of bus type nets are grouped together based on their cum The bus bits that act as attackers to a specific victim are treated as a group. All the a attackers form one group and bits of the same bus form multiple other groups. In cas combined effect of a full bus can be used to decide whether an attacker is significan *Default*: `false`
`-enable_double_clocking_check {true | false}`	Enables double clocking check in AAE. This option enables double clocking analysis and `report_double_clocking` comma *Default*: `false`

`-enable_delay_report {true | false}`	Enables SI delay reports.
	For generating SI delay report, use the following:
	`setSIMode -enable_delay_report true;`
	`report_noise -delay max -txtfile max_delay.txt;`
	`report_noise -delay min -txtfile min_delay.txt`
	To run `report_noise -delay`, you need to set this parameter prior to `update_timing`.
	Default: `false`

`-enable_drv_with_delta_slew {true | false}`	When set to `true`, the `max_transition` violations are reported based on the SI slews the `signoffOptDesign` command and should be applied prior running the `signoffTi` *Default*: `false`
`-enable_dynamic_receiver_peak_limits {true | false}`	Enables automatic computation of ROP threshold value of two-stage cells, using the When set to `true`, the software computes the noise threshold of the last stage from tl *Default*: `false`
`-enable_logical_correlation {true | false}`	Enables the AAE-SI logical correlation. Logical correlation refers to correlation of lo simple case would be input and output nets of an invertor never switch in the same *Default*: `false` **Note**: You can use this parameter for Glitch Analysis also.

`-enable_two_stage_driver_weakening {true | false}`	Enables the two-stage driver weakening mode. A non-failing significant glitch at the input of a two-stage driver can propagate throug crosstalk-induced glitch to amplify at the next stage receiver (though it may not be si enabled, the software considers weakened holding resistance for the two-stage driv The following diagram illustrates two-stage driver weakening:  The dual stage driver weakening honors the glitch propagation threshold criteria. **Note**: This feature is supported with CCS-N models only. *Default*: `true`
`-enable_virtual_attacker_constituent_report {true | false}`	Enables virtual attacker constituent reporting. The virtual attacker constituent reporting feature is useful for identifying significant b This feature is enabled when the `setSIMode -enable_bus_attacker_correlation` pa Default: false
`-enable_glitch_propagation {true | false}`	Enables glitch propagation through single stage cells and driver weakening through software does not delete the glitch waveform saved on instance input pins. The wav receiver pin. *Default*: `false`
`-enable_glitch_propagation_spice_deck {true | false}`	
	Enables the driver input glitch waveform to be printed in the SPICE deck such that S is set to `true`, the software does not delete the glitch waveform saved on instance in the next stage receiver pin. *Default*: `false`

`-enable_glitch_report {true | false}`	Enables the storage of glitch information for further debug using the `report_noise` c
	When set to `false`, the `timeDesign`/`optDesign` command will still report glitches in th will not be available for use.
	When set to `true`, the `report_noise` command may be used and glitch information w
	For generating a `report_noise` glitch report, you can use the following commands:
	`setSIMode -enable_glitch_report true;`
	`report_noise -txtfile glitch.txt`
	Default: `false`
	Note: For SI glitch fixing, you must set the `-enable_glitch_report` parameter to `tru` fixing.

`-enable_overshoot_undershoot_glitch {true | false}`	Enables overshoot/undershoot analysis and reporting. The life span of transistors can be described by the overshoot and undershoot value overshoot/undershoot, then it may suffer a reduced life span. *Default*: `false`
`-glitch_accuracy_level <int>`	Specifies the accuracy levels for glitch analysis. *Default*: 0

`-glitch_analysis_type {source | full_propagation}`	Specifies the glitch analysis type - `source` or `full_propagation`. *Default*: `source`
`-hold_slack` *double*	Specifies the slack reselection threshold for hold. *Default*:`-1e-12` seconds
`-individual_attacker_clock_threshold` *float*	Sets the glitch tolerance for individual attacker nets on victim clock nets. The softwa victim. If the peak of this noise exceeds the value of `individual_attacker_clock_th` constituents section of the report. Otherwise, this aggressor becomes eligible for the `accumulated_small_attacker_threshold`, the aggressor nets in the virtual attacker a `individual_attacker_clock_threshold` is not specified, then clock nets will use the **Note**: The minimum allowed individual attacker for clock nets to be kept is 0.1% of v 0.001. *Default*: `1.5%` or `0.015`
`-individual_attacker_threshold` *double*	Sets the glitch tolerance for individual attacker nets on the victim net. The software e the peak of this noise exceeds the value of `individual_attacker_threshold`, the att of the report. Otherwise, this aggressor becomes eligible for the virtual attacker. If th `accumulated_small_attacker_threshold`, the aggressor nets in the virtual attacker a **Note**: Minimum allowed individual attacker to be kept is `0.1%` of vdd, which is equiva *Default*: 1.5% or 0.015
`-input_glitch_full_propagation_threshold` *double*	Specifies the receiver input glitch threshold beyond which Tempus will not propaga flagged.
`-input_glitch_full_propagation_vh_threshold` *double*	Specifies the receiver VH input glitch threshold beyond which Tempus will not prop flagged. This parameter takes precedence over `-input_glitch_full_propagation_threshol`
`-input_glitch_full_propagation_vl_threshold` *double*	Specifies the receiver VL input glitch threshold beyond which Tempus will not propa flagged. This option takes precedence over `-input_glitch_full_propagation_threshold`.
	Note: These parameters can be used only when full propagation mode is enabled u
`-individual_attacker_simulation_filtering {true | false}`	Uses electrical filtering to reduce pessimism due to small attackers being preserved value. *Default*: `true` **Note**: This option can only be used when the accumulated small attacker is turned o
`-input_glitch_threshold` *double*	Specifies the input glitch failure threshold value during AAE analysis. *Default*: `0.4`
`-input_glitch_vh_threshold` *double*	Specifies the input glitch `vh` failure threshold value during AAE analysis. *Default*: `0.4`
`-input_glitch_vl_threshold` *double*	Specifies the input glitch `vl` failure threshold value during AAE analysis. *Default*: `0.4`
`-nonlinear_attacker_slew {true | false}`	Uses a non-linear attacker. If set to `false`, it uses a linear attacker. *Default*: `true`
`-num_si_iteration` *value*	Specifies the maximum number of timing window iterations that should be performe to the timing windows, which can potentially change the victim/attacker net combina where the delay changes are negligible. *Default*: `2` **Note**: The minimum allowed timing window iterations is `2` for performing SI analysis

`-pessimistic_mode {advanced | advanced_node | reduced_pessimism | increased_pessimism | high_pessimism}`	Specifies the pessimistic mode setting for adjusting delay calculation results. You c • `advanced`: This is the default mode. This option may give good results at 28nm, b • `advanced_node`: This option works for newer technology nodes, such as 7nm onw tool by adjusting pin capacitances, which may help with slew alignment, whilst m • `reduced_pessimism`: In cases where Innovus is too pessimistic when correlating t a better match. This mode may be useful at technology nodes, such as 16nm. • `high_pessimism`: In cases where Innovus is too optimistic when correlating to ext better match. This mode is similar to `advanced_node`, but without the pin capacita • `increased_pessimism`: In cases where Innovus is too optimistic when correlating for a better match. This is an older option, `high_pessimism` or `advanced_node` opti **Note**: This parameter impacts delay calculation only when the `setDesignMode -pes` `high_pessimism`, and `increased_pessimism` options should give results that are on th nodes.
`-receiver_clk_peak_limit double`	Defines the glitch check limit for clock inputs in receiver peak mode. This parameter *Default* : `0.05` (5 percent of VDD)
`-receiver_latch_peak_limit double`	Defines the glitch check limit for latch inputs in receiver peak mode. This parameter *Default* : `0.1` (`10` percent of VDD)
`-receiver_peak_limit double`	Defines the receiver peak/glitch peak limit at all nodes. The value specified for this p *Default*: `0.15`
`-report_max_virtual_attacker_constituents int`	Specifies the maximum number of virtual attacker constituents to report. This feature works only when bus virtual attacker constituent reporting is enabled us setting. *Default*: 5
`-report_si_slew_max_transition {true | false}`	Enables reporting of SI slews for DRV maximum transition reporting only. The delay commands, such as `reportTranViolation`, `report_constraint`, `reportDelayCalcul` `timeDesign` under the ./timingReports/<design name>.tran file only and the number `timeDesign` summary report. **Note**: The `setSIMode -report_si_slew_max_transition true` does not enable fixin `timeDesign` DRV reporting. To enable SI slew fixing, use `setOptMode -opt_post_ro` *Default*: `false`
`-secondary_attacker_decoupling_factor <double>`	Specifies decoupling factor for secondary aggressors. *Default*: 1
`-separate_delta_delay_on_data {true | false}`	Separates the delta delay on data (delta delay is always separated for clock). Works with `-delta_delay_annotation_mode` to determine where the separated delay shows in th Using this parameter, you can increase runtime/memory. *Default*: `false`
`-setup_slack <double>`	Specifies the slack reselection threshold for setup. *Default*:`-1e-12` seconds
`-si_reselection {delta_delay | slack}`	Specifies the SI reselection criteria during timing window iterations. `delta_delay`: Reselects a net for calculation if the delta delay on the net is greater th `slack`: Re-selects nets that have failing slacks during timing window iterations. *Default*: `slack`

`-si_reselection_delay_threshold` *double*	Allows you to change the delay reselection criteria. You can specify `-si_reselection_delay_threshold` with the `-delta_delay` parameter only. *Default*: `1e-11` Units in seconds
`-skip_noise_model_check` *pins*	Specifies a list of pins for which noise model check will not take place. Default= " "
`-skip_tw <nets>`	Specifies the list of nets for which timing window is to be skipped. The timing windo When specified, only the last setting is retained. The `setSiMode -skip_tw` command *Default*: `" "`
`-switch_prob <double>`	Sets the attacker switching probability. If you specify a value less than `0.0` or more t of `0.3`. *Default*: `0.3`
`-use_infinite_TW {true | false}`	Uses infinite timing windows in a single SI run. *Default*: `false`
`-unconstrained_net_use_inf_tw {true | false}`	Specifies that unconstrained nets will have an infinite timing window. To turn this of *Default*: `true`

Examples

- The following command runs AAE keeping all individual attackers larger than `0.01*`VDD and not using any accumulated small attackers:
  ```
  setSIMode -individual_attacker_threshold 0.01 -accumulated_small_attacker_threshold 10.01
  ```
- The following command sets the glitch peak limit at `0.5*vdd` at data nodes, `0.1*vdd` at clock nodes, and `0.2*vdd` at latch nodes.
  ```
  setSIMode -receiver_peak_limit  0.5 -receiver_latch_peak_limit 0.2 -receiver_clk_peak_limit 0.1
  ```
- To keep all attackers less than the `setSIMode -individual_attacker_threshold` whose accumulated glitch bump exceeds 1% of VDD, you can use the following command
  ```
  setSIMode -accumulated_small_attacker_threshold 0.01
  ```

Related Information

- getSIMode
- report_timing
- report_noise
- update_glitch
- reportDelayCalculation

update_glitch

```
update_glitch
[-help]
[-effort {medium | high}]
```

Allows you to run glitch analysis without running SI delay analysis. This command provides the ability to run glitch analysis on all the nets in a design or on a subset of nets.

Using the `update_glitch` command without `report_timing` or `update_timing` will only run glitch analysis.

Note: It is not recommended to run the `update_glitch` command after the `report_timing` or `update_timing` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `update_glitch` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man update_glitch`.
`-effort {medium | high}`	Specifies the effort level for update glitch. When set to `high`, glitch analysis uses more accurate modeling of glitch for small attackers, thereby reducing pessimism. **Note**: The run time in `high` mode may be slightly higher, and is largely dependent on the distribution of coupling. *Default*: `medium`

Examples

- The following command will run glitch analysis without SI delay:
  ```
  > update_glitch
  ```

Related Commands

- `report_noise`
- `report_timing`
- `setSIMode`
- `getSIMode`

analyze_ir_paths

```
analyze_ir_paths
[-help]
-clock_period <float>
[-hierarchical_activity_file <string>]
[-max_slack <float>]
[-pre_simulation_period <float>]
-seq_activity <float>
-simulation_period <float>
[-user_defined_critical_paths <string>]
```

Generates a physical-aware multi-cycle switching activity file containing toggle information of voltage sensitive paths as well as timing critical paths in the design for Tempus Power Integrity analysis. This command generates ‘swsrc.txt’ file in the run directory, which then is fed back to the software using the `read_activity_file` command.

Parameters

`-help`	Prints out the command usage.
`-clock_period <float>`	Specifies the period of dominant clock of the design in library time units. Tempus PI refers to this specified period for multi-cycle activity generation.

`-hierarchical_activity_file <string>`	Provides a mechanism for different sequential activity for several partitions in the design. The following is the required format:
	`<hierarchical instance 1> <activity>`
	`<hierarchical instance 2> <activity>`
	…

`-max_slack <float>`	Specifies the maximum slack limit in library time units, above which the timing paths are not considered for activity generation.
`-pre_simulation_period <float>`	Specifies the pre-simulation duration that will be used by Tempus PI for activity generation. This parameter is useful when package model information is read-in using `set_package` command, as it allows the package oscillations to settle down. **Note**: The value specified must match to value specified under `set_rail_analysis_mode -record_results_start_time` setting.
`-seq_activity <float>`	Specifies the activity of sequential endpoints in the design. This value will be used to control the number of IR sensitive paths, along with their fanout cone, to toggle in each cycle.
`-simulation_period <float>`	Specifies the simulation period in library time units that Tempus PI dynamic power analysis will use. The larger the simulation period, the more will be the number of paths covered in the analysis. The following formula can be used to set the simulation period: *sim_cycles = (2/seq_act)*N* Where, • *seq_act* is the input sequential activity (e.g., 0.05, 0.1 etc.) • *N* is the number of nworst paths to be covered per endpoint • *sim_period* is equal to the sim_cycles*<time period of dominant clock> It is recommended to have N>=1, that is, at least one path per endpoint should be scheduled in the analysis.
`-user_defined_critical_paths <string>`	Specifies a collection of timing paths that are to be prioritized for toggling while generating Tempus PI activity. This feature is useful when designers have prior information on certain timing paths that may switch together and can potentially cause IR-induced STA violations.

Examples

- The following command generates the activity file for design with dominant clock of 2GHz, with input sequential activity being 0.1 and simulation duration being 20ns:

```
analyze_ir_paths -seq_activity 0.1 -simulation_period 20 -clock_period 0.5
```

- The following example specifies different sequential activity for different hierarchical partitions, with 0.2 sequential activity for unspecified partitions and top:

```
analyze_ir_paths -seq_activity 0.2 -simulation_period 30 -clock_period 0.5 -hierarchical_activity_file hact.txt
Contents of 'hact.txt':
hierA/inst1 0.25
hierB/inst2 0.15
```

- The following command specifies the pre-simulation duration, particularly useful for package oscillations to settle when package models have been read-in:

```
analyze_ir_paths -seq_activity 0.1 -simulation_period 30 -clock_period 0.5 -pre_simulation_period 10
```

define_glitch_holding_resistance

```
define_glitch_holding_resistance
[-help]
[-glitch_type {both | vl | vh | vlu | vho | vl_vh | vlu_vho}]
[-instance_pin string]
[-lib_pin string]
-value double
[-views string]
```

Allows you to specify the holding resistance.

You can use this command to define a net-specific holding resistance instead of using the default holding resistance of 1000 ohms.

This command can be specified more than once per net. If there is a duplicate specification, then the last setting is used.

Parameters

`-help`	Prints out the command usage.
`-glitch_type {both | vl | vh | vlu | vho | vl_vh | vlu_vho}`	Specifies the glitch type for resistance Hold application. The option `both` applies the value to VL and VH.
`-instance_pin string`	Specifies a list of instance pin names.
`-lib_pin string`	Specifies the instance/pin name of the driver that the holding resistance should be applied to.
`-value double`	Specifies the value in ohms of the holding resistance to be used by glitch analysis.
`-views string`	Specifies a list of view names. If this parameter is not specified, then by default all the views are applied.

Examples

- The following command sets the holding resistance on the Y pin of inst1 to 233ohms for VL:
  ```
  define_glitch_holding_resitance -glitch_type vl -pins {inst1/Y} 233
  ```
- The following command sets the holding resistance on the Y pin of inst2 to 380ohms for both VL & VH:
  ```
  define_glitch_holding_resitance -glitch_type both -pins {inst1/Y} 233
  ```

36

Structured Data Path Commands

- addSdpGroupMember
- addSdpObject
- checkSdpGroup
- createSdpGroup
- deleteSdpObject
- detachSdpGroup
- flipSdpObject
- getSdpGroupAttribute
- getSdpMode
- moveSdpObject
- placeSdpGroup
- readSdpFile
- setSdpGroupAttribute
- setSdpMode
- setSdpObjectStatus
- setSdpTopGroup
- writeSdpFile

addSdpGroupMember

```
addSdpGroupMember
[-help]
-group group_name
-object object_name_list
[-orientation orientStr]
[-before refObject | -after refObject]
```

Adds an object or member to an existing structured data path (SDP) group. The added object can be an instance or another SDP group.

The `addSdpGroupMember` command enables you to create a structured data path using TCL commands. You do not need to learn SDP relative placement format for creating a data path.

Before adding a member to an SDP group, you should have:

- Imported the design
- Created an SDP group. For more information, see `createSdpGroup`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `addSdpGroupMember` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man addSdpGroupMember`.
`-after` *`refObject`*	Adds the SDP group or object after the specified reference object.
`-before` *`refObject`*	Adds the SDP group or object before the specified reference object.
`-group` *`group_name`*	Specifies the name of the SDP group to which you are adding the member.
`-object` *`object_name_list`*	Specifies the name of the object(s) to be added to the SDP group.
`-orientation` *`orientStr`*	Specifies the orientation to be set for the object. You can specify orientation as one of the following: `R0`, `R90`, `R180`, `R270`, `MX`, `MY`, `MX90`, `MY90` **Note:** If you do not specify this parameter, the object has `R0` orientation (no rotation) by default.

Examples

- The following example adds the `DTMF_INST/PLLCLK_INST` instance to the SDP group `sdpGrp1`. The instance is rotated 90 degrees counter-clockwise.
  ```
  addSdpGroupMember -group sdpGrp1 -object DTMF_INST/PLLCLK_INST -orientation R90
  ```
- The following examples adds two objects, `DTMF_INST/tdsp_core_inst/inst1` and `DTMF_INST/tdsp_core_inst/inst2`, to the `sdpGrp1` group.
  ```
  addSdpGroupMember -group sdpGrp1 -object {DTMF_INST/tdsp_core_inst/inst1
  DTMF_INST/tdsp_core_inst/inst2}
  ```

Related Information

- `addSdpObject`
- `createSdpGroup`
- `detachSdpGroup`
- `sdp` object in *Innovus Database Object Information*
- Using Structured Data Paths chapter of the *User Guide*
- Floorplan Menu chapter of the *Menu Reference*
    - *Structured Data Path*

addSdpObject

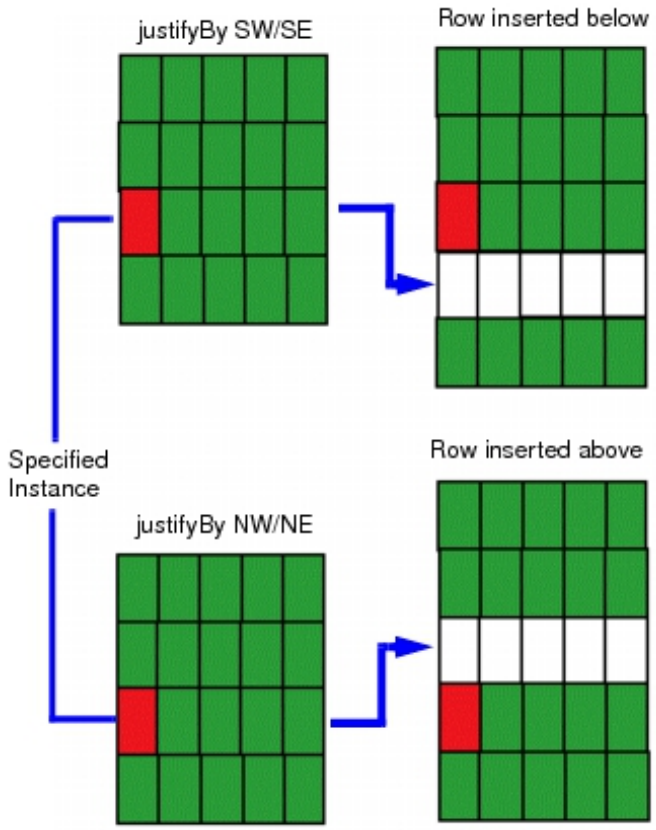
```
addSdpObject
[-help]
{-obstruct group_name_list | {-emptyRow instance_name -numRow value}}
```

Creates a placement blockage on top of the specified SDP group OR inserts one or more empty rows across the whole die above/below the specified instance.

Use the `addSdpObject` command to prevent cells from being placed inside the SDP group area. You can also use this command to create empty rows for clock or IPO buffers that will be inserted later on in the flow.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `addSdpObject` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man addSdpObject`.
`-emptyRow` *`instance_name`*	Inserts empty row(s) above/below the specified instance based on the `justifyBy` constraint of the group to which the instance belongs. The empty row(s) are inserted across all SDP groups. The number of rows to be inserted is specified by the `-numRow` parameter. If `-numRow` is not specified, one empty row is inserted by default. **Note:** The core boundary will be extended to accommodate the newly added rows if the current core box is not big enough.



`-numRow` *value*	Indicates the number of empty rows to be inserted.
`-obstruct` *group_name_list*	Creates a placement blockage over the specified SDP group(s). SDP Placement Blockage (`_SDP_OBSTRUCT_`*group_name*)

Examples

- The following example creates a placement blockage over the SDP group `sdpGrp1`.

  ```
  addSdpObject -obstruct sdpGrp1
  ```

- The following example inserts two empty rows below the `DTMF_INST/ROM_512x16_0_INST` instance.

  ```
  addSdpObject -emptyRow DTMF_INST/ROM_512x16_0_INST -numRow 2
  ```

Related Information

- `createSdpGroup`
- `deleteSdpObject`
- `flipSdpObject`
- `moveSdpObject`
- `setSdpObjectStatus`
- `sdp` object in *Innovus Database Object Information*
- Using Structured Data Paths chapter of the *User Guide*
- Floorplan Menu chapter of the *Menu Reference*
  - *Structured Data Path*

checkSdpGroup

```
checkSdpGroup
[-help]
[-file fileName]
[-sdpGroups {sdp_name...}]
```

Checks current SDP placement against the SDP constraints, such as order, alignment, and orientation, which may have been originally specified via an SDP relative placement file or a set of TCL commands and reports violations. The `checkSdpGroup` command also creates markers in the GUI for any SDP violation it detects. The violation markers can be viewed in the Violation Browser.

Whenever you perform any manual editing and/or optimization step, you can use `checkSdpGroup` to check for any resulting SDP violations. This ensures that current placement is still valid as per the

SDP constraints you had originally specified.

Before checking SDP placement, you should have:

- Imported the design
- Created and placed SDP groups

Parameters

`-help`	Prints a brief description that includes type and default information for each `checkSdpGroup` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man checkSdpGroup`.
`-file` *`fileName`*	Generates a detailed report for the following checking categories: • SDP group/instance orientation • SDP group/instance justifyBy constraint • Alignment by pin name constraint • SDP group/instance flip constraint • Skip space constraint • SDP group/instance overlapping • Inside parent box violation For each of the above checking category, the report includes the following information: • Total number of violations • List of SDP group/instance names that have that violation *Default*: *`designName`*`.checkSdp.rpt`
`-sdpGroups {`*`sdp_name...`*`}`	Checks only the specified sdps. *Data_type*: string, optional

Example

- The following example checks SDP placement and generates the default report:

  ```
  checkSdpGroup
  ```

  Sample output:

  ```
  ############################################################
  # Generated By: Cadence Innovus 18.10-d183_1
  # Generated on: Fri Dec 15 08:53:35 2017
  # HostName: noi-leenap
  # Design: DTMF_CHIP
  # Command: checkSdpGroup
  ############################################################


  ############################################################
  Orientation and flip checking are only done for the object
  which constaint 'orient/flip' is specified in the sdp file.
  Due to the instance placement needs to match the row orientation,
  the instance actual orientation might not be same as the value
  that specified in the file. The orientation/flip checking will
  report this kind of mismatch
  ############################################################

  ERROR: check justifyBy fail, sdp group
  DTMF_INST/TDSP_CORE_INST/MPY_32_INST/M16X16_INST/i_361
  ERROR: check justifyBy fail, sdp group
  DTMF_INST/TDSP_CORE_INST/MPY_32_INST/M16X16_INST/i_375
  ERROR: check skipSpace fail, sdp group DPNULL

  Orientation violation: 0
  JustifyBy violation: 2
  AlignByPinName violation: 0
  SkipSpace violation: 1
  Overlap violation: 0
  InsideParentBox violation: 0
  ------------------------------------------------
  Total violation: 3
  ```

- The following example checks SDP placement and generates a detailed report named `checkSdp1.rpt`.

  ```
  checkSdpGroup -file checkSdp1.rpt
  ```

Related Information

- `checkPlace`
- `createSdpGroup`
- `getSdpGroupAttribute`
- `placeSdpGroup`
- `setSdpGroupAttribute`
- `setSdpMode`
- `sdp` object in *Innovus Database Object Information*
- Using Structured Data Paths chapter of the *User Guide*
  - *Checking SDP Placement*
- Floorplan Menu chapter of the *Menu Reference*
  - *Structured Data Path*

createSdpGroup

```
createSdpGroup
[-help]
[-justifyBy {SW|SE|NW|NE|N|W|S|E|MID}]
-name sdpGroupName
[-type {row|col|space}]
{{-skip_space value} | -skip_space_by_site_width value siteName | -skip_space_by_micron
value}
| {{{-inst  WCardName... [-keepOrder ]} | -selected }
  [[-spread_group value ] | [-spread_group_by_site_width value siteName] | [-
spread_group_by_micron value]
| [[-numRow value] [-numCol  value]]]}
```

Creates an SDP group. The added object can be an instance or another SDP group.

The `createSdpGroup` command enables you to create a structured data path using TCL commands. You do not need to learn SDP relative placement syntax for creating a data path.

Before creating an SDP group, you should have:

- Imported the design

Parameters

`-help`	Outputs a brief description that includes type and default information for each `createSdpGroup` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man createSdpGroup`.
`-inst WCardName...`	Specifies one or more instance names. Use this parameter if you want to create the SDP group from the specified instances. You can additionally specify `-keepOrder` if you want the tool to honor the order in which SDP instance are specified in the list. **Note:** Wildcards can be used to specify the instance names.
`-justifyBy {SW|SE|NW|NE|N|W|S|E|MID}`	
	Specifies the anchor point that will be used to align the SDP group you are creating. *Default*: `SW`

`-keepOrder`	Honors the order of instances in the SDP instance list that you specify as input with the `-inst` parameter.
`-name` *sdpGroupName*	Specifies the name of the SDP group you are creating.
`-numCol` *value*	Indicates the number of columns to be created.
`-numRow` *value*	Indicates the number of rows to be created. **Note:** If you specify both `-numRow` and `-numCol`, use the `-type` parameter to specify the parent structure type (`row` or `col`). If you do not specify type, the default parent structure will be column.
`-selected`	Indicates that you want to create the SDP group from the current set of selected instances. If the selected cells are already placed and you specify neither the `-numRow` or `-numCol` options nor the `-type` option, the command will automatically generate one column with rows where the SDP structure will be based on the physical location of the selected cells. The tool will print the following message to indicate that the SDP structure created will be derived from the physical location of the selected cells: `Command is running with physical mode extraction where SDP structure will be derived from the existing physical cell placement.` If the selected cells are not yet placed or if some of the selected cells are placed while others are not, the tool prints a message indicating that SDP physical mode extraction cannot be activated and SDP structure cannot be derived from the existing physical placement because the selected instance is not placed.

`-skip_space` *value*	Specifies a space value to be skipped. By default, if `-skip_space` is defined in a column, its value represents the number of skipped rows. If `-skip_space` is defined in a row, its value is for column skipping and represents the number of M2 tracks (pitch of first vertical layer). Skip space value can also be specified as number of widths of a specified row site by using the `-skip_space_by_site_width` *value siteName* parameter. However, row site width should be specified only for an SDP row, and not for an SDP column. Alternatively, you can specify the `-skip_space_by_micron` parameter to specify the distance to be skipped in microns (`um`). **Note:** Use the `-skip_space` parameter with the `-type space` option to add row/column spaces.
`-skip_space_by_micron` *value*	Specifies the space value to be skipped in microns (`um`). If the specified distance value is not equal to the exact number of first vertical metal pitches or number of rows, `createSdpGroup` rounds off the value. **Note**: This option cannot be used with either `-skip_space` or `-skip_space_by_site_width`.
`-skip_space_by_site_width` *value siteName*	Specifies the number of row/column space to be skipped and the name of the tech site. Uses the width of the specified site as the skip unit in the horizontal direction. **Note**: This option cannot be used with either `-skip_space` or `-skip_space_by_micron`.
`-spread_group` *value*	Inserts space between group of instances. If `-spread_group` is specified in a column, spacing will be the number of added rows. If `-spread_group` is specified in a row, spacing will be the number of placement grids added between the columns in that row.
`-spread_group_by_micron` *value*	Specifies the space to be inserted between groups of instances in microns (`um`). **Note**: This option cannot be used with either`-spread_group` or `-spread_group_by_site_width`.

`-spread_group_by_site_width` *value siteName*	Specifies the number of row/column space to be inserted and the name of the tech site. Uses the width of the specified site as the spread unit in the horizontal direction. **Note**: This option cannot be used with either `-spread_group` or `-spread_group_by_micron`.
`-type {row|col|space}`	Indicates whether the new group is a row, column, or an empty (skipped) space. *Default*: `col`

Examples

- The following example creates an SDP group named `TestGrp` of type `space` with two spaces:
  ```
  createSdpGroup -name TestGrp -type space -skip_space 2
  ```
- The following example creates an SDP group named `sdpGrp1` of type `row` with four columns from the selected instances. The group is aligned in the South East direction:
  ```
  createSdpGroup -name sdpGrp1 -type row -justifyBy SE -numCol 4 -selected
  ```
- The following example creates a row data path that has three columns, each with two rows, for all the instances that have name matched the pattern `DTMF_INST/TDSP_CORE_INST*`:
  ```
  createSdpGroup -name sdp_group1 -type row -inst DTMF_INST/TDSP_CORE_INST* -numRow 2
  -numCol 3
  ```
- The following command automatically generates one column with rows where the SDP structure will be based on the physical location of the selected cells, provided that the selected cells are already placed:
  ```
  createSdpGroup -selected -name group2
  ```

Related Information

- `getSdpGroupAttribute`
- `getSdpMode`
- `placeSdpGroup`
- `setSdpGroupAttribute`
- `setSdpMode`
- `sdp` object in *Innovus Database Object Information*
- Using Structured Data Paths chapter of the *User Guide*
- Floorplan Menu chapter of the *Menu Reference*
    - *Structured Data Path*

deleteSdpObject

```
deleteSdpObject
[-help]
{-object object_name_list [-keepCellPlacement ] | -obstruct group_name_list | -emptyRow
instance_name [-numRow value]}
```

Deletes one of the following:

- SDP group(s) or element(s)
- A placement blockage associated with the specified SDP group(s)
- Empty row(s) adjacent to the specified instance

Use the `deleteSdpObject` command to modify structure of existing data path(s) easily and to remove existing SDP groups or elements.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `deleteSdpObject` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man deleteSdpObject`.
`-emptyRow` *`instance_name`*	Deletes the empty row(s) adjacent to the specified instance based on the `justifyBy` constraint of the group to which the instance belongs. If the group is justified by `SW`/`SE`, the empty rows below the specified instance are deleted. If the group is justified by `NW`/`NE`, the empty rows above the specified instance are deleted. **Note:** If the rows adjacent to the specified instances are not empty, an error message is returned.
`-keepCellPlacement`	
	Preserves the cell placement of remaining SDP members when an SDP object is deleted. If this parameter is not specified, `deleteSdpObject` unplaces all SDP members while deleting the specified SDPs. This option can be used only with `-object`.
`-numRow` *`value`*	Indicates the number of empty rows to be deleted. **Note:** If this parameter is not specified, one empty row is deleted by default.
`-object` *`object_name_list`*	Deletes the specified SDP group or instance. If you want to delete more than one object, enclose the object names in {}.
`-obstruct` *`group_name_list`*	Deletes the placement blockage over the specified SDP group(s).

Examples

- The following command deletes the SDP group `TestGrp`.

  ```
  deleteSdpObject -object TestGrp
  ```

- The following command deletes a placement blockage over the SDP group `sdpGrp1`.

  ```
  deleteSdpObject -obstruct sdpGrp1
  ```

- The following command deletes two empty rows below the `DTMF_INST/ ROM_512x16_0_INST`

instance where the group to which `ROM_512x16_0_INST` belongs has `justifyBy` constraint set as `SW`.

```
deleteSdpObject -emptyRow DTMF_INST/ROM_512x16_0_INST -numRow 2
```

Related Information

- `addSdpObject`
- `createSdpGroup`
- `flipSdpObject`
- `moveSdpObject`
- `setSdpObjectStatus`
- `sdp` object in *Innovus Database Object Information*
- Using Structured Data Paths chapter of the *User Guide*
- Floorplan Menu chapter of the *Menu Reference*
  - *Structured Data Path*

detachSdpGroup

```
detachSdpGroup
[-help]
-name {objNames}
```

Detaches the specified Structured Data Path (SDP) object from the parent group.

Parameters

`-help`	Prints a brief description that includes type and default information for each `detachSdpGroup` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man detachSdpGroup`
`-name {objNames}`	Specifies the name of the SDP object to be detached from the parent group.

Related Information

- `checkSdpGroup`
- `createSdpGroup`
- `getSdpGroupAttribute`
- `placeSdpGroup`
- `setSdpGroupAttribute`
- `sdp` object in *Innovus Database Object Information*
- Using Structured Data Paths chapter of the *User Guide*
- Floorplan Menu chapter of the *Menu Reference*
    - *Structured Data Path*

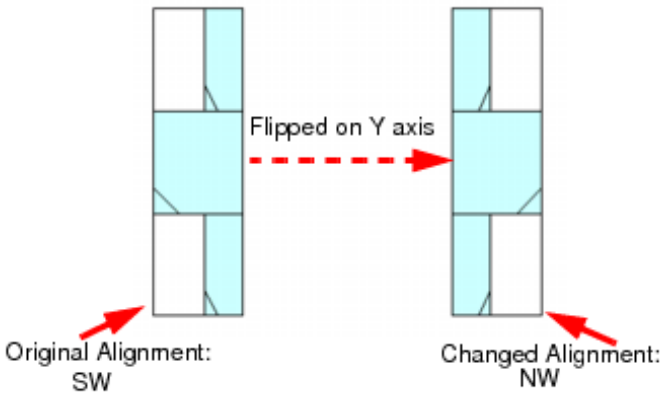
flipSdpObject

```
flipSdpObject
[-help]
-object object_name_list
[-x | -y ]
```

Flips an SDP group on X or Y axis along with its elements. The alignment information (`justifyBy`) is adjusted accordingly.

Use the `flipSdpObject` command when flipping can reduce wire length. In the traditional SDP flow, this command can be used while defining SDP constraints or while analyzing SDP alignment.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `flipSdpObject` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man flipSdpObject`.
`-object` *`object_name_list`*	Specifies the names of the SDP groups to be flipped.
`-x | -y`	Specifies the axis on which the specified group (along with its elements) should be flipped. *Default*: `-y` 

Examples

- The following command flips the SDP group `sdpGrp1` on Y axis.

  ```
  flipSdpObject -object sdpGrp1
  ```

Related Information

- `addSdpObject`
- `checkSdpGroup`
- `createSdpGroup`
- `deleteSdpObject`
- `moveSdpObject`
- `setSdpObjectStatus`
- `sdp` object in *Innovus Database Object Information*
- Using Structured Data Paths chapter of the *User Guide*
- Floorplan Menu chapter of the *Menu Reference*
  - *Structured Data Path*

getSdpGroupAttribute

```
getSdpGroupAttribute
[-help]
[-flip]
[-justifyBy]
-name sdp_name
[-orient]
[-origin]
[-padding]
[-skipSpace]
[-type]
```

Displays the current values of various attributes of the SDP group specified with `-name`. If you do not specify any parameter except `-name`, `getSdpGroupAttribute` returns information on all of the `setSdpGroupAttribute` parameters for the SDP group specified with `-name`.

Parameters

`-help`	Prints a brief description that includes type and default information for each `getSdpGroupAttribute` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man getSdpGroupAttribute`.
`-flip`	Returns the flip information of the specified SDP group.
`-justifyBy`	Returns the anchor point used to align the specified SDP group.
`-name sdp_name`	Specifies the name of the SDP for you which you want to retrieve settings. This parameter is mandatory.
`-orient`	Returns the orientation of the specified SDP group.
`-origin`	Returns the location of the top SDP group.
`-padding`	Returns the padding of the top sdp object.
`-skipSpace`	Returns the spacing pattern of the specified SDP group.
`-type`	Returns the type of the specified SDP group.

Example

- The following command returns information on all of the `setSdpGroupAttribute` parameters for the SDP group `group2`:

```
innovus 16> getSdpGroupAttribute -name group2
{flip NONE} {justifyBy SW} {orient R0} {origin (700.26 649.04)} {skipSpace {0 0 0 0
0 0 }} {type column}
```

Related Information

- `createSdpGroup`
- `placeSdpGroup`
- `setSdpGroupAttribute`
- `sdp` object in *Innovus Database Object Information*
- Using Structured Data Paths chapter of the *User Guide*
- Floorplan Menu chapter of the *Menu Reference*
    - *Structured Data Path*

getSdpMode

```
getSdpMode
[-help]
[-clock_location]
[-disable_extended_core]
[-honor_dont_use]
[-honor_orient]
[-legalization]
[-legalization_effort]
[-max_move_distance]
[-num_column]
[-place_report]
[-pre_fixed_cells_blockage_direction]
[-quiet]
[-nonDefault]
```

Displays the current values of various SDP-related sticky options. Use this command when you want to retrieve current values of all or specified `setSdpMode` settings. If you do not specify any parameter, `getSdpMode` returns information on all of the `setSdpMode` parameters.

Parameters

`-help`	Prints a brief description that includes type and default information for each `getSdpMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man getSdpMode`.
`parameter_names`	Displays information for the specified parameters. You can specify one or more parameters. See `setSdpMode` for descriptions of the parameters you can specify.
`-quiet`	Displays the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software displays the current settings of all `setSdpMode` parameters in Tcl list format.
`-nonDefault`	Displays the current settings that are not default settings.

Example

- The following command displays the current setting of the `-clock_location` parameter:

  ```
  getSdpMode -clock_location
  ```

  The software displays the following information:

  ```
  -clock_location center # string, default=center
  center
  ```

- The following command displays the current settings for all `setSdpMode` parameters in Tcl list format only:

  ```
  getSdpMode -quiet
  ```

  The software returns the following information:

  ```
  {clock_location center} {disable_extended_core true} {honor_dont_use false}
  {honor_orient false} {legalization NONE} {legalization_effort low}
  {max_move_distance 0} {num_column 2} {place_report {}}
  {pre_fixed_cells_blockage_direction NONE}
  ```

Related Information

- `getSdpGroupAttribute`
- `placeSdpGroup`
- `place_design`
- `proto_design`
- `setSdpGroupAttribute`
- `setSdpMode`
- `sdp` object in *Innovus Database Object Information*
- Using Structured Data Paths chapter of the *User Guide*
- Floorplan Menu chapter of the *Menu Reference*
  - *Structured Data Path*

moveSdpObject

```
moveSdpObject
[-help]
moveObject {objNames}
{-before refObject | -after refObject}
```

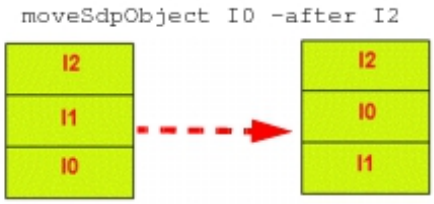
Moves specified SDP objects before or after the specified reference object. Use this command to:

- Move a subgroup or member to a different position within the current SDP group
- Move a subgroup or member to a different group to improve congestion and/or timing.
- Move multiple instances, SDP groups, or a mix of instances and SDP groups to the specified location.

In the traditional SDP flow, this command can be used while defining SDP constraints or while analyzing SDP alignment.

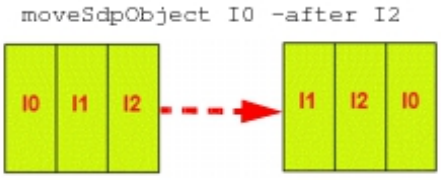
Parameters

`-help`	Prints a brief description that includes the type and default information for each `moveSdpObject` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man moveSdpObject`.
`-after` *`refObject`*	Indicates that the specified SDP objects should be moved after the reference object: • For column with `justifyBy SW/SE`, `after` means the specified SDP objects are placed above the reference object. Column with `justifyBy SW/SE` • For column with `justifyBy NW/NE`, `after` means the specified SDP objects are placed below the reference object.



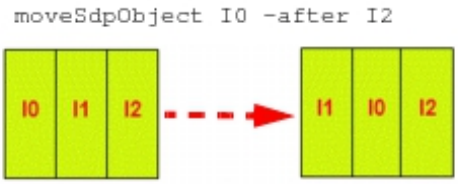
Column with `justifyBy NW/NE`

- For row with justifyBy SW/NW, `after` means that the specified SDP objects are placed on the right side of the reference object.



Row with `justifyBy SW/NW`

- For row with justifyBy SE/NE, `after` means that the specified SDP objects are placed on the left side of the reference object.

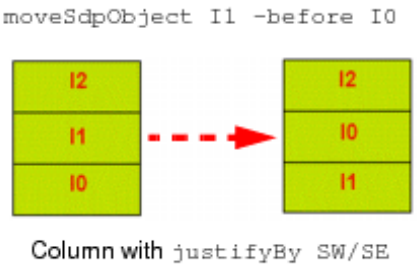


Row with `justifyBy SE/NE`

`-before` *`refObject`*

Indicates that the specified SDP objects should be moved before the reference object:

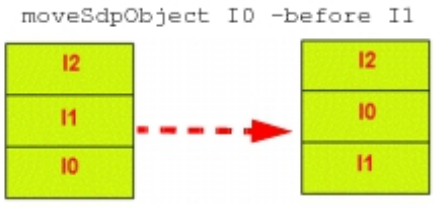
- For column with `justifyBy SW/SE`, `before` means the specified SDP objects are placed below the reference object.



Column with `justifyBy SW/SE`

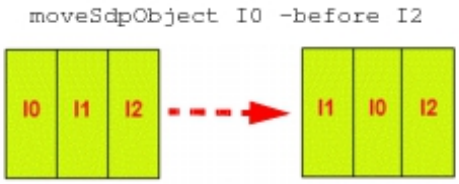
- For column with `justifyBy NW/NE`, `before` means the specified SDP objects are

placed above the reference object.



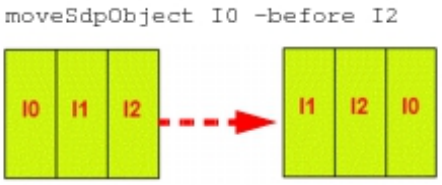
Column with `justifyBy NW/NE`

- For row with justifyBy SW/NW, `before` means that the specified SDP objects are placed on the left side of the reference object.



Row with `justifyBy SW/NW`

- For row with justifyBy SE/NE, `before` means that the specified SDP objects are placed on the right side of the reference object.



Row with `justifyBy SE/NE`

moveObject {objNames}	Specifies the names of the SDP objects to be moved. The objects can be SDP groups, instances, or a mix of both. If multiple SDP objects are specified, the option works as follows: • If both the objects being moved and the reference object are instances, the instances being moved will become members of the SDP group to which the reference instance belongs. • If both the objects being moved and the reference object are SDP groups, the SDP groups being moved will become members of the SDP group to which the reference SDP group belongs. • If the objects being moved are SDP groups and the reference object is an instance, the SDP groups being moved will become members of the SDP group to which the reference instance belongs. • If the objects being moved are instances and the reference object is an SDP group, the instances being moved will become members of the SDP group to which the reference SDP group belongs. • If the objects being moved are a mix of instances and SDP groups and the reference object is an instance, the instances and SDP groups being moved will become members of the SDP group to which the reference instance belongs and the result is based on the input sequence. • If the objects being moved are a mix of instances and SDP groups and the reference object is an SDP group, the instances and SDP groups being moved will become members of the SDP group to which the reference SDP group belongs and the result is based on the input sequence.

Examples

- The following command moves the SDP group `sdpGrp1`, which is an SDP group of type `row` with `justifyBy SE`, to the right side of the group `sdpGrp3`:

  ```
  moveSdpObject sdpGrp1 -before sdpGrp3
  ```

- The following command moves the SDP group `SDP_col145`, which is an SDP group of `type column` with `justifyBy NE`, to the position below `SDP_col170`.

  ```
  moveSdpObject SDP_col145 -after SDP_col170
  ```

- Consider the SDP group structure given below:

```
column group1 {
   justifyBy SW
   row selInstGrp_9 {
      justifyBy SW
      inst DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/ar1_reg_1
      inst DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/ar1_reg_13 orient MX
      inst DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/ar1_reg_12
      inst DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/ar1_reg_3
      inst DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/ar0_reg_12 orient R0
      inst DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/ar0_reg_13 orient MX
      inst DTMF_INST/TDSP_CORE_INST/PORT_BUS_MACH_INST/data_out_reg_8
      inst DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/ar1_reg_14
   }
   .....
}
```

- The following command moves the specified instances before `selInstGrp_9`:

  ```
  moveSdpObject {DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/acc_reg_30
  DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/acc_reg_32} -before selInstGrp_9
  ```

  The resulting structure is as follows:

  ```
  column group1 {
     justifyBy SW
     inst DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/acc_reg_30
     inst DTME_INST/TDSP_CORE_INST/EXECUTE_INST/acc_reg_32
     row selInstGrp_9 {
        justifyBy SW
        inst DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/ar1_reg_1
        inst DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/ar1_reg_13 orient MX
        inst DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/ar1_reg_12
        inst DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/ar1_reg_3
        inst DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/ar0_reg_12 orient R0
        inst DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/ar0_reg_13 orient MX
        inst DTMF_INST/TDSP_CORE_INST/PORT_BUS_MACH_INST/data_out_reg_8
        inst DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/ar1_reg_14
     }
     .....
  }
  ```

- The following command moves the specified instances after `selInstGrp_9`:

  ```
  moveSdpObject {DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/acc_reg_30
  DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/acc_reg_32} -after selInstGrp_9
  ```

  The resulting structure is as follows:

  ```
  column group1 {
  ```

```
    justifyBy SW
    row selInstGrp_9 {
       justifyBy SW
       inst DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/ar1_reg_1
       inst DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/ar1_reg_13 orient MX
       inst DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/ar1_reg_12
       inst DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/ar1_reg_3
       inst DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/ar0_reg_12 orient R0
       inst DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/ar0_reg_13 orient MX
       inst DTMF_INST/TDSP_CORE_INST/PORT_BUS_MACH_INST/data_out_reg_8
       inst DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/ar1_reg_14
    }
    inst DTMF_INST/TDSP_CORE_INST/EXECUTE_INST/acc_reg_30
    inst DTME_INST/TDSP_CORE_INST/EXECUTE_INST/acc_reg_32
    .....
}
```

Related Information

- addSdpObject
- createSdpGroup
- deleteSdpObject
- flipSdpObject
- setSdpObjectStatus
- sdp object in *Innovus Database Object Information*
- Using Structured Data Paths chapter of the *User Guide*
- Floorplan Menu chapter of the *Menu Reference*
    - *Structured Data Path*

placeSdpGroup

```
placeSdpGroup
[-help]
[-create_inst_group]
[-deleteOverlap {cell...} {sdp name...}]
[-sdpGroups {name...}]
[-noResolveOverlap | [-keepSdpArea]]
```

Places elements or instances of all defined SDP groups in row(s) and column(s) based on user-defined relative placement information. The `placeSdpGroup` command:

- Adjusts the core boundary automatically if the existing core is not big enough for placing all specified SDPs
- Resolves automatically the overlaps on the row/column cell list specified in `setSdpMode`.
- Honors the `alignByPinName` SDP constraint for column groups
- Marks all placed SDP instances to fixed status
- Places the SDP boundary outside the die area on the right hand side if origin information is not defined.

Before placing SDP groups, you should have:

- Imported the design
- Defined SDPs using TCL commands or modified SDPs, as required

Parameters

`-help`	Outputs a brief description of the `placeSdpGroup` command. For a detailed description of the command, use the `man` command: `man placeSdpGroup`.
`-create_inst_group`	Converts failed top sdp groups to instance groups. In the 2G flow, the tool supports the conversion of failed top SDP groups to instGroups by using this parameter. This option works only on top SDPs. This means the tool will convert the top SDPs to instGroups, irrespective of whether it was the top SDPs or child SDPs that failed. The instances in the instance groups are soft_fixed.
`-deleteOverlap {cell...} {sdp name...}`	Deletes specified physical cells that overlap with all or specified SDP groups. **Note:** If you do not specify *sdp name ...*, the specified physical cells are removed for all SDPs.
`-keepSdpArea`	Keeps the original SDP structure when resolving overlaps with other instances. Use this option if you want to resolve the cell overlap by moving the SDP structure instead of changing it.
`-noResolveOverlap`	Ignores overlaps with column/row cells that are specified in `setSdpMode`. Use this option to take a look at how the groups will be placed before resolving overlaps.
`-sdpGroups {name...}`	Places only the specified SDP groups.

Examples

- The following command places all defined SDP groups without removing overlaps:
  ```
  placeSdpGroup -noResolveOverlap
  ```

Related Information

- `checkSdpGroup`
- `createSdpGroup`
- `getSdpGroupAttribute`
- `getSdpMode`
- `setSdpGroupAttribute`
- `setSdpMode`
- `sdp` object in *Innovus Database Object Information*
- Using Structured Data Paths chapter of the *User Guide*
- Floorplan Menu chapter of the *Menu Reference*
  - *Structured Data Path*

readSdpFile

```
readSdpFile
[-help]
-file file_name
[-hierPath path]
[-leftoverGroup]
[-origin llx lly]
[-skipPlacement]
```

Reads in a relative placement file (in .sdp format) and then calls `placeSdpGroup` to place all the structured data path (SDP) elements defined in the file. If `-skipPlacement` is specified, placement is skipped.

If the SDP file refers to an instance that is not present in the netlist or design, the software issues a warning but continues to load the rest of the SDP file.

Before using `readSdpFile`, you should have:

- Imported the design

Parameters

`-help`	Outputs a brief description that includes type and default information for each `readSdpFile` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man readSdpFile`.
`-file` *`file_name`*	Specifies the name of the input file containing SDP definitions. The SDP file format supports wildcards (`*` and `?`) for instance names. It also supports numeric bus bit range as part of an instance name.
`-hierPath` *`path`*	Specifies the hierarchical path name of the SDP elements specified in the SDP file. This path name is appended to all SDP instances defined in the file.
`-leftoverGroup`	
	Stores all remaining instances in the current hierarchical path that do not belong to any SDP group into a new SDP group called *`SDPname`*`_LEFTOVER_RPLGRP`. An error is issued if a logical hierarchy has more than one SDP group.
`-origin` *`llx lly`*	Defines the SDP location by specifying the SDP block origin. The origin information is applied to all SDPs specified in the SDP file.
`-skipPlacement`	
	Reads in the SDP group definition but does not place the group.

Examples

- The following command reads the `dtmf_chip.sdp` file and places all defined SDP elements. It also appends the specified hierarchical path to all elements.

  ```
  readSdpFile -file dtmf_chip.sdp -hierPath DTMF_CHIP/TDSP_CORE_INST
  ```

Related Information

- `placeSdpGroup`
- `writeSdpFile`
- `sdp` object in *Innovus Database Object Information*

- Using Structured Data Paths chapter of the *User Guide*
- Floorplan Menu chapter of the *Menu Reference*
    - *Structured Data Path*

setSdpGroupAttribute

```
setSdpGroupAttribute
[-help]
[-flip {X Y XY NONE}]
[-justifyBy {SW SE NW NE MID}]
-name sdp_name
[-orient {R0 MX MY R180 NONE}]
[-origin {x y}]
[-padding {x y}]
[-skipSpace spacing_pattern]
```

Sets or modifies the specified attributes of an existing SDP group. This command can be used in anywhere in the flow after an SDP group is created.

Parameters

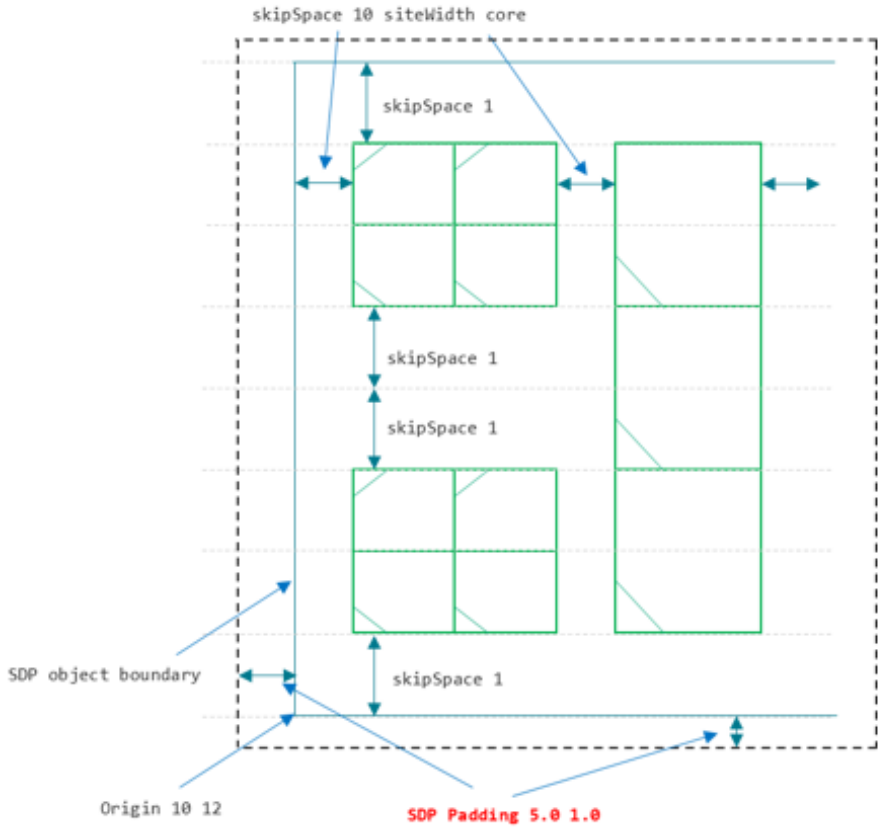
`-help`	Prints a brief description that includes type and default information for each `setSdpGroupAttribute` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setSdpGroupAttribute`.
`-flip {X Y XY NONE}`	Specifies that the SDP group should be flipped on the X or Y axis along with its elements. The alignment information is adjusted accordingly.
`-justifyBy {SW SE NW NE MID}`	Specifies the anchor point that will be used to align the SDP group.
`-name sdp_name`	Specifies the name of the SDP object.
`-orient {R0 MX MY R180 NONE}`	Specifies the orientation of the SDP object.
`-origin {x y}`	Specifies the location of the top SDP group. It can be set only to the top group.

`-padding {x y}`

Specifies the edge extend value of the SDP bounding box in horizontal and vertical direction for the specified top SDP group. The tool uses the sum of the actual SDP bounding box and the extended value to place the SDP groups during global placement. This way, less SDP groups overlap because the tool can enlarge the SDP bounding box by using padding during global placement.

For example, the following command specifies a padding of `{5.0 1.0}` for the SDP top group named `top_group1`:

```
setSdpGroupAttribute -name top_group1 -padding {5.0 1.0}
```



Default: `{0 0}`

Note: The padding is an attribute of each top SDP group. If the specified SDP group name is not a top SDP group, the tool will give an error and return.

`-skipSpace` *spacing_pattern*	Specifies the spacing pattern of the SDP group. The value for row skipping represents the number of skipped rows. The value for column skipping represents the number of the first vertical layer pitch. If the SDP group for which attributes are being set is of type `space`, the tool will accept a value for `-skipSpace`.

Examples

- Consider the SDP group, `sub_group1`, with origin at {383.46 628.88}. The following command will update the origin of the SDP group:
  ```
  setSdpGroupAttribute -name sub_group1 -origin {500 500}
  ```
  If you now check, the origin will have been updated without the need for running the `placeSdpGroup` command:
  ```
  > puts "[dbGet top.fplan.topSdps.pt]"
  {500.28 497.84}
  ```

Related Information

- `createSdpGroup`
- `getSdpGroupAttribute`
- `getSdpMode`
- `placeSdpGroup`
- `setSdpMode`
- `sdp` object in *Innovus Database Object Information*
- Using Structured Data Paths chapter of the *User Guide*
- Floorplan Menu chapter of the *Menu Reference*
  - *Structured Data Path*

setSdpMode

```
setSdpMode
[-help]
[-clock_location {bottom|center}]
[-disable_extended_core {true|false}]
[-honor_dont_use {true|false}]
[-honor_orient {true|false}]
[-legalization {NONE|SW|SE|NW|NE|AUTO}]
[-legalization_effort {low|medium|high}]
[-max_move_distance value]
[-num_column numberOfCol]
[-place_report file_name]
[-pre_fixed_cells_blockage_direction {NONE|X|Y}]
[-reset]
```

Sets SDP-related sticky options before using commands like `placeSdpGroup` and `place_design`. You can use the `setSdpMode` command to disable extension of core boundary before running the `placeSdpGroup` command. You can also use this command to generate a detailed SDP placement report.

Before setting SDP options with `setSdpMode`, you should have:

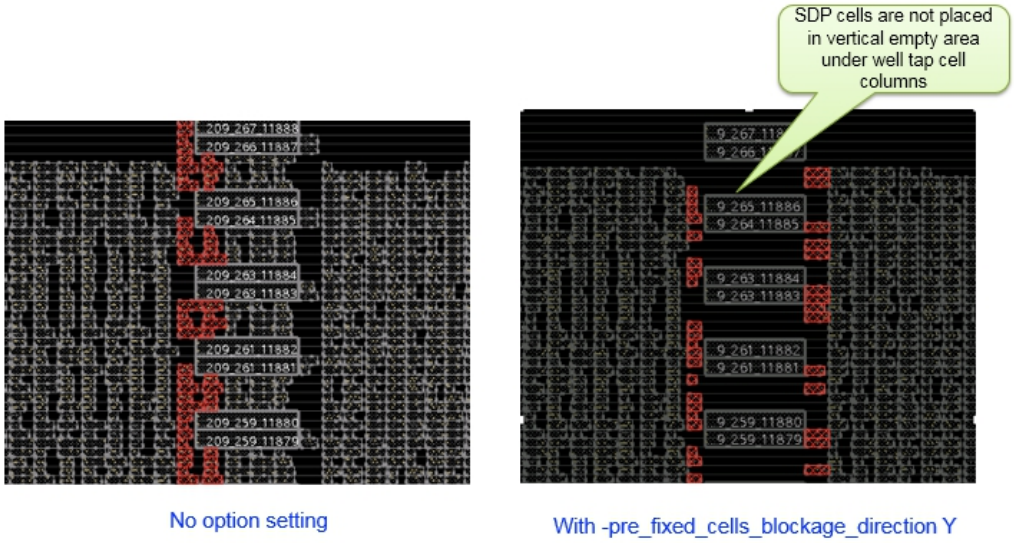
- Imported the design

Parameters

`-help`	Prints a brief description that includes type and default information for each `setSdpMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setSdpMode`.
`-clock_location {bottom | center}`	Specifies where the Integrated Clock Gating (ICG) cell will be placed in an SDP register column. *Default*: `center`

`-disable_extended_core {true | false}`	Disables extension of the core boundary. If you set `-disable_extended_core` to `true` before running the `placeSdpGroup` command, the software does not adjust the core boundary to accommodate all SDP placements. Instead, the software places the SDP elements outside the core boundary if they do not fit within the boundary. *Default*: `true`
`-honor_dont_use {true|false}`	Specifies whether the `dont_use` attribute on base cells should be honored by the `placeSdpGroup` command during EEQ swapping in SDP legalization. By default, `placeSdpGroup` takes all cells into account. If `-honor_dont_use` is set to `true`, `placeSdpGroup` ignores cells with the `dont_use` attribute. *Default*: `false`
`-honor_orient {true|false}`	Honors user-specified orientation during legalization. User-specified orientation can be set using the `setSdpGroupAttribute` command. You can use the `getSdpGroupAttribute` command to check the current orientation setting. Note that the SDP writer writes out orientation information only if its value is defined by the user through the SDP file or TCL command. *Default*: `false`
`-legalization {NONE|SW|SE|NW|NE|AUTO}`	
	Specifies the corner of the SDP box that will be kept fixed while legalizing an SDP instance. *Default*: `NONE`
`-legalization_effort {low|medium|high}`	

	Specifies the legalization effort in placing SDP groups. When you run `placeSdpGroup`, the SDP groups are legalized on the basis of their order, which is determined by their size. Bigger SDP groups are placed first, and multiple iterations are used to place and align the remaining SDP groups. You can also control the legalization effort applied by using the `-legalization_effort` parameter. The higher the setting of this parameter, the more the number of iterations attempted to try and place the SDP groups legally. *Default*: `low`
`-max_move_distance` *`value`*	Specifies how far an SDP can be moved from its original clustering location to resolve overlaps. By default, this value is half the perimeter of the core box. The Manhattan distance between the new and original locations of the SDP must be less than the value of the `-max_move_distance` option. The `-max_move_distance` option is honored by the `refinePlace` command. SDP columns start at the center-of-mass of the instances and then try to find a legal location without overlaps. *Default*: 0
`-num_column` *`numberOfCol`*	Specifies the number of flip flop columns for each SDP group. *Default*: `2`
`-place_report` *`file_name`*	Generates a detailed SDP placement report when `placeSdpGroup` is run. *Default*: If `setSdpMode -place_report` has not been specified before running `placeSdpGroup`, this report is not generated. **Note:** To disable the `-place_report` option once it has been specified, specify *`file_name`* as a null string (`""`). The detailed SDP placement report will not be generated subsequently.
`-pre_fixed_cells_blockage_direction {NONE|X|Y}`	

	Creates temporary placement blockage internally over pre-fixed cells (that is, well tap or power switch cells) in `X` or `Y` direction to prevent SDP cells to be placed in the horizontal or vertical empty spaces when resolving overlapping. The illustration below shows the difference between SDP placement without this setting and with `-pre_fixed_cells_blockage_direction` set to `Y`.  *Default:* `NONE`
`-reset`	Resets all or specified options to default value

Examples

- The following command disables extension of the core boundary if the boundary cannot accommodate all SDP placements:

  ```
  setSdpMode -disable_extended_core true
  ```

- The following commands place elements or instances of all defined SDP groups and generate a detailed SDP placement report named `dtmf_chip.sdp.rpt`:

  ```
  setSdpMode -place_report dtmf_chip.sdp.rpt
  placeSdpGroup
  ```

- The following commands control SDP movement from its original cluster location to resolve overlaps:

  ```
  setSdpMode -max_move_distance 30
  place_design -sdp
  ```

Related Information

- `getSdpGroupAttribute`
- `getSdpMode`
- `placeSdpGroup`
- `place_design`
- `proto_design`
- `setSdpGroupAttribute`
- `sdp` object in *Innovus Database Object Information*
- Using Structured Data Paths chapter of the *User Guide*
- Floorplan Menu chapter of the *Menu Reference*
  - *Structured Data Path*

setSdpObjectStatus

```
setSdpObjectStatus
[-help]
-status {unplaced|placed|fixed|cover}
[-name object_name_list | -all]
```

Sets the placement status of all or specified SDP group/instances. With `setSdpObjectStatus`, you can make the placer move or prevent it from moving to adjust placement of SDP elements. For example, if you specify status as `unplaced`, the command unplaces the data path and moves its boundary outside the die area on the right hand side.

Use this command after placement to set SDP status to `fixed` so other applications will preserve SDP placement (don't touch). Use the `setSdpObjectStatus` command before running optimization to set SDP placement status to `placed` so `optDesign` can further resize instances of SDP.

Before setting object status with `setSdpObjectStatus`, you should have:

- Imported the design
- Defined the SDP group or instance

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setSdpObjectStatus` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setSdpObjectStatus`.
`-all`	Changes the placement status of all members of all SDP groups. **Note:** If neither `-all` nor `-name` is specified, `setSdpObjectStatus` changes placement status of all SDPs.
`-name` *`object_name_list`*	Specifies the names of SDP groups or instances whose placement status needs to change.
`-status {unplaced|placed|fixed|cover}`	Specifies the new placement status.

Examples

The following command changes the placement status of all SDP groups/instances to `placed`:

```
setSdpObjectStatus -status placed
```

Related Information

- `addSdpObject`
- `createSdpGroup`
- `deleteSdpObject`
- `moveSdpObject`
- `flipSdpObject`
- `sdp` object in *Innovus Database Object Information*
- Using Structured Data Paths chapter of the *User Guide*
- Floorplan Menu chapter of the *Menu Reference*
    - *Structured Data Path*

setSdpTopGroup

```
setSdpTopGroup
[-help]
-group group_name
[-force]
{-location llx lly | -hierPath path}
```

Specifies the location or the hierarchical path of a data path. If you know where SDP will be placed in the design, use the`-location` option to specify SDP location. The advantage of using `setSdpTopGroup` is that you do not need to specify the hierarchical path for each member of an SDP group.

Before setting `setSdpTopGroup -hierPath`, you should have:

- Imported the design
- Created the SDP group using `createSdpGroup`

Before setting `setSdpTopGroup -location`, you should have:

- Imported the design
- Created the SDP group using `createSdpGroup`
- Added members to the SDP group using `addSdpGroupMember`

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setSdpTopGroup` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setSdpTopGroup`.
`-group` *`group_name`*	Sets the specified SDP group as the SDP top group.
`-force`	Indicates that the specified location should be set even if it is outside the existing core boundary. `-force` automatically adjusts the core boundary to accommodate the specified location.
`-location` *`llx lly`*	Specifies the location of the SDP top group.
`-hierPath` *`path`*	Specifies the hierarchical path of the SDP top group.

Examples

- The following command specifies the location of the SDP group `tdsp_sdpGrp`:
  ```
  setSdpTopGroup -group tdsp_sdpGrp -location 848.82 586.75
  ```
- The following command sets the hierarchical path for the SDP group `column_GRP5` as `/DTMF_INST/EXECUTE_INST`. As a result, the specified instance names of all members of this SDP will have this hierarchical path.
  ```
  setSdpTopGroup -group column_GRP5 -hierPath /DTMF_INST/EXECUTE_INST
  ```

Related Information

- `addSdpGroupMember`
- `createSdpGroup`
- `getSdpGroupAttribute`
- `setSdpGroupAttribute`
- Using Structured Data Paths chapter of the *Innovus User Guide*
- `sdp` object in *Innovus Database Object Information*
- Using Structured Data Paths chapter of the *User Guide*
- Floorplan Menu chapter of the *Menu Reference*
    - *Structured Data Path*

writeSdpFile

```
writeSdpFile
[-help]
-file file_name
[-template | -topGroup group_name]
[-template | -command]
[-template  | [-updateRelativePlaceConstraint | -updateOrientation]]
```

Saves the current SDP relative placement information into a file. By default, the output is in `.sdp` format. `writeSdpFile` enables you to save SDP relative information so it can be reloaded in another Innovus session.

The `writeSdpFile` command can be run any time after SDPs have been defined.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `writeSdpFile` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man writeSdpFile`.
`-command`	Specifies that the relative placement information should be saved in TCL commands format instead of `.sdp` format.
`-file` *file_name*	Specifies the name of the output file containing SDP relative information.
`-template`	Specifies that the file should be saved as a template in `.sdp` format. **Note:** This parameter cannot be used with `-command` or `-topGroup`.
`-topGroup` *group_name*	Indicates that the relative placement information of only the specified SDP group should be written.
`-updateOrientation`	Updates the orientation of SDP instances before writing them to the SDP file.
`-updateRelativePlaceConstraint`	Updates relative placement constraints of SDPs before writing out the SDP file. Use this parameter to write out the updated SDP/TCL file after any change in SDP relative placement due to `placeDesign` or manual changes. The `-updateRelativePlaceConstraint` option can be used with the `-command` and `-topGroup` options as well.

Examples

- The following command writes the current SDP information in SDP format in a file named `test.sdp`:

  ```
  writeSdpFile -file test
  ```

- The following command writes the SDP information for the `sdpGrpA` group in the SDP TCL command format to the output file `output.sdp`.

```
writeSdpFile -file output.sdp -command -topGroup sdpGrpA
```

Related Information

- `placeSdpGroup`
- `readSdpFile`
- `sdp` object in *Innovus Database Object Information*
- Using Structured Data Paths chapter of the *User Guide*
- Floorplan Menu chapter of the *Menu Reference*
    - *Structured Data Path*

37

Timing Analysis Commands

- all_analysis_views
- all_constraint_modes
- all_delay_corners
- all_hold_analysis_views
- all_library_sets
- all_op_conds
- all_rc_corners
- all_setup_analysis_views
- calNegSlack
- check_instance_library_in_views
- check_ldb_version
- check_timing
- checkTimingLibrary
- create_analysis_view
- create_constraint_mode
- create_delay_corner
- create_library_set
- create_op_cond
- create_property_alias
- create_rc_corner
- createUserDisableForCombLoopBreak
- display_timing_map
- get_analysis_view

- get_capacitance_unit
- get_clock_network_objects
- get_constant_for_timing
- get_constraint_mode
- get_delay_corner
- get_equivalent_cells
- get_interactive_constraint_modes
- get_lib_cell_leakage_power
- get_library_set
- get_op_cond
- get_pba_mode
- get_propagated_clock
- get_rc_corner
- get_resistance_unit
- get_socv_rc_variation_factor
- get_socv_reporting_nsigma_multiplier
- get_time_unit
- getAnalysisMode
- getTimeLibFile
- read_path_descriptions
- read_sdf
- read_spdf
- read_twf
- report_analysis_coverage
- report_analysis_summary
- report_analysis_views
- report_annotated_assertions
- report_annotated_check

- report_annotated_delay
- report_annotated_parasitics
- report_annotations
- report_aocv_derate
- report_case_analysis
- report_cell_instance_timing
- report_clock_gating_check
- report_clock_propagation
- report_clock_timing
- report_clocks
- report_constraint
- report_cppr
- report_design
- report_fanin
- report_fanout
- report_inactive_arcs
- report_instance_library
- report_lib_arcs
- report_min_pulse_width
- report_mode
- report_net
- report_path_exceptions
- report_path_groups
- report_ports
- report_socv_interconnect_variation
- report_socv_library
- report_statistical_timing_derate_factors
- report_timing

- report_timing_derate
- report_wire_load
- reportTimingLib
- reset_instance_library
- reset_path_adjust_group
- reset_sdf_assertions
- reset_timing_derate
- reset_wire_load_mode
- reset_wire_load_model
- reset_wire_load_selection_group
- set_analysis_view
- set_aocv_thresholds
- set_default_view
- set_instance_library
- set_interactive_constraint_modes
- set_io_thresholds
- set_normalized_driver_waveform_lib
- set_path_adjust
- set_path_adjust_group
- set_pba_mode
- set_socv_constraint_config
- set_socv_rc_variation_factor
- set_socv_reporting_nsigma_multiplier
- set_table_style
- set_timing_derate
- set_wire_load_mode
- set_wire_load_model
- set_wire_load_selection_group

- setAnalysisMode
- signoffTimeDesign
- timeDesign
- update_analysis_view
- update_constraint_mode
- update_delay_corner
- update_io_latency
- update_library_set
- update_rc_corner
- write_global_slack_report
- write_ldb
- write_path_descriptions
- write_sdf
- write_timing_windows
- writeAnnotatedTransition
- writeDesignTiming
- writeTimingCon

Path Exception Priorities

The following are the path exception priorities if a path in the design matches more than one path exception:

1. `set_false_path`
2. `set_min_delay`, `set_max_delay`
3. `set_multicycle_path`

If there is more than one exception of a given type, for example the `set_multicycle_path` command, the path exception that is more specific has higher priority. A path exception has higher priority if it specifies more of the `-from`/`-through`/`-to` parameters. For example, the `-from -to` options would have priority over the `-from` option.

If the path has the same number of reference points:

- `-from` option has priority over the `-to` option
- `-to` option has priority over the `-through` option
- `-from` option has priority over the `-to` option

Note: To check for ignored path exceptions, use the `report_path_exceptions -ignored` command.

The following table shows the priorities for path exceptions applied to the same path.

Table 38-1 **Path Exception Priorities**

Priority	Path Exception
1. (Highest)	`set_false_path`
2.	`set_max_delay -from` *`pin_list`*
3.	`set_max_delay -to` *`pin_list`*
4.	`set_max_delay -through` *`pin_list`*
5.	`set_max_delay -from` *`clkwave_name`*
6.	`set_max_delay -to` *`clkwave_name`*
7.	`set_max_delay` (The most constraining adjustment has the higher priority over less constraining adjustments.)
8.	`set_multicycle_path -from` *`pin_list`*
9.	`set_multicycle_path -to` *`pin_list`*
10.	`set_multicycle_path -through` *`pin_list`*
11.	`set_multicycle_path -from` *`clkwave_name`*
12.	`set_multicycle_path -to` *`clkwave_name`*
13. (Lowest)	`set_multicycle_path` (The most constraining adjustment has the higher priority over less constraining adjustments.)

Examples of Path Exception Priorities

The following are examples of the path exception priorities for a path starting at `A`, going through `B`, and ending with `C`. The pairs of examples are ordered such that the highest priority constraint in one pair has precedence over the highest priority constraint in the next pair.

- In the following pair of constraints, the false path overrides the cycle addition for the path from `A` through `B` to `C`. For all other paths from `A`, the cycle addition applies.

  ```
  set_false_path -from A -through B -to C set_multicycle_path -from A 2
  ```

- In the following pair, the first constraint is applied (two cycles added) because the `-from` option has priority over the `-to` option:

  ```
  set_multicycle_path -from A 3 set_multicycle_path -to C 2
  ```

- In the following pair, the first constraint is applied (two cycles added) because it specifies a `-through` pin as well as a `-to` pin:

  ```
  set_multicycle_path -through A -to B 3 set_multicycle_path -to B 2
  ```

- In the following pair, the first constraint is applied (two cycles added) because the `-from` option has precedence over the `-to` option:

  ```
  set_multicycle_path -from CLKA 3 set_multicycle_path -to CLKB 2
  ```

- In the following pair, the second constraint is applied because the second constraint overwrites the first one:

  ```
  set_multicycle_path -from A 2 set_multicycle_path -from A 3
  ```

Bidirectional Pin Defaults

Bidirectional pins have both an input pin and an output pin type. This section explains the default pin type that the system uses for bidirectional pins.

Defaults for Pin (or Port) Constraints

The following defines the pin and port types that are used in the next table.

Table 38-2 **Bidirectional Pin (or Port) Type Definitions**
Bidirectional Pin (or Port) Type Definitions

Pin Type	Type 1	Type 2
Blackbox pins	out	in

Instance pins	out	in
Netlist (primary) ports	in	out
Hierarchical ports	in	out

Table 38-3 **Bidirectional Pin Type Defaults**

Pin or Port Constraint	Type	Hierarchical Port
set_input_delay	1	ignored
set_output_delay	2	ignored
set_clock_latency	1	ignored
create_generated_clock	1	used
set_input_transition	1	ignored
set_driving_cell	1	ignored
set_drive	1	ignored

Defaults for Port Constraints

The following table defines the port types that are used in the next table.

Table 10-4 **Bidirectional Port Type Definitions**

Pin Type	Type 1	Type 2
Netlist (primary) ports	in	out
Hierarchical ports	in	out

Table 10-5 **Bidirectional Port Type Defaults**

Port Constraint	Type	Hierarchical Port
set_fanout_load	2	ignored
set_load	2	ignored
set_max_transition	2	ignored

all_analysis_views

```
all_analysis_views
[-help]
[-type string]
```

Returns a Tcl list of analysis views in the design, according to `type`.

Use this command after creating analysis views using `create_analysis_view`.

Parameters

`-help`	Prints out the command usage.
`-type string`	Specifies the type of analysis views: `setup`, `hold`, `dynamic`, `leakage`, `active`, or `all`.

Examples

- The following commands create analysis views `missionSlow` and `missionFast`:

```
create_analysis_view \
  -name missionSlow \
  -constraint_mode missionSetup \
  -delay_corner dcWCCOM
create_analysis_view \
  -name missionFast \
  -constraint_mode missionHold \
  -delay_corner dcBCCOM
```

- The following command returns a list of the defined analysis views in the design:

```
all_analysis_views
```

The software returns the following results:

```
{missionSlow missionFast}
```

Related Information

- create_analysis_view
- set_analysis_view

all_constraint_modes

```
all_constraint_modes [-active | -active_setup | -active_hold]
```

Returns a Tcl list of all defined constraint modes in the design.

Use this command after creating multiple constraint modes (`create_constraint_mode`).

Parameters

`-active`	Returns a list of the defined constraint modes associated with all active analysis views.
`-active_setup`	Returns a list of the defined constraint modes associated with active setup analysis views only.
`-active_hold`	Returns a list of the defined constraint modes associated with active hold analysis views only.

Example

- The following commands create constraint modes `missionSetup` and `missionHold`:

  ```
  create_constraint_mode -name missionSetup
  -sdc_files [list io.sdc mission1-clks.sdc mission1-except.sdc]

  create_constraint_mode -name missionHold
  -sdc_files [list io.sdc mission1-clks.sdc mission1-except.sdc dont_use.sdc]
  ```

- The following command returns a list all the defined constraints modes in the design:

  ```
  all_constraint_modes
  ```

  The software returns the following results:

  ```
  {missionSetup missionHold}
  ```

- The following command returns a list of all the defined constraint modes associated with the active setup analysis views in the design:

  ```
  all_constraint_modes -active_setup
  ```

Related Information

- create_constraint_mode

all_delay_corners

```
all_delay_corners
[-help]
[-active]
```

Returns a Tcl list of all defined delay calculation corner objects in the design.

Use this command after creating delay calculation corner objects with create_delay_corner.

Parameters

`-help`	Prints out the command usage.
`-active`	Returns a list of the defined delay corners associated with all the active analysis views.

Examples

- The following commands create the delay calculation corners `dcWCCOM` and `dcBCCOM`:

```
create_delay_corner -name dcWCCOM \
  -library_set IsCOM-1V \
  -opcond_library stdcell_1V \
  -opcond WCCOM \
  -rc_corner rc-cworst
create_delay_corner -name dcBCCOM \
  -library_set IsCOM-1V \
  -opcond_library stdcell_1V \
  -opcond BCCOM \
  -rc_corner rc-cbest
```

- The following command returns a list of all the delay calculation corners in the design:

```
all_delay_corners
```

  The software returns the following information:

```
{dcWCCOM dcBCCOM}
```

Related Commands

- create_delay_corner

all_hold_analysis_views

```
all_hold_analysis_views
```

Returns a Tcl list of all active hold analysis views in the design.

Use this command after setting active hold analysis views for the design (set_analysis_view).

Parameters

None

Example

- The following command defines the active setup and hold analysis views for the design:
  ```
  set_analysis_view -setup {missionWCCOM mission2WCCOM} -hold {missionBCCOM testBCCOM}
  ```
- The following command returns a list of the active hold analysis views in the design:
  ```
  all_hold_analysis_views
  ```

  The software returns the following results:
  ```
  {missionBCCOM testBCCOM}
  ```

Related Commands

- set_analysis_view
- create_analysis_view

all_library_sets

`all_library_sets`

Returns a Tcl list of all currently defined library sets in the design.

Use this command after creating multiple library sets (create_library_set).

Parameters

None

Examples

- The following commands create the library sets `IsCOM-1V` and `IsCOM-2V`:

  ```
  create_library_set -name IsCOM-1V -timing [list stdcell_F_1V.lib ram_F.lib pad.lib]

  create_library_set -name IsCOM-2V -timing [list stdcell_F_2V.lib ram_F.lib pad.lib]
  ```

- The following command returns a list of all defined library sets in the design:

  ```
  all_library_sets
  ```

  The software returns the following results:

  ```
  {IsCOM-1V IsCOM-2V}
  ```

Related Information

- create_library_set

all_op_conds

`all_op_conds`

Returns a Tcl list of all operating conditions defined for the design.

Use this command after creating multiple operating conditions (`create_op_cond`).

Parameters

None

Examples

- The following commands create operating conditions called `myPVT1` and `myPVT2` for libraries `IsCOM-1V.lib` and `IsCOM-2V.lib`:

```
create_op_cond -name PVT1
   -library_file IsCOM-1V.lib
   -P 1.0
   -V 1.2
   -T 120

create_op_cond -name PVT2
   -library_file IsCOM-2V.lib
   -P 1.0
   -V 1.2
   -T 20
```

- The following command creates a collection called `myOpConds` that contains the operating conditions `myPVT1` and `myPVT2`:

```
set myOpConds [all_op_conds]
```

Related Information

- `create_op_cond`

all_rc_corners

```
all_rc_corners
[-help]
[-active]
```

Returns a Tcl list of all currently defined RC corner objects in the design.

Use this command after creating multiple RC corner objects (create_rc_corner).

Parameters

`-help`	Prints out the command usage.
`-active`	Returns a list of the defined RC corners associated with all active analysis views.

Examples

- The following commands create the RC corners `rc-cbest`, `rc-cworst`, and `rc-typical`:

  ```
  create_rc_corner -name rc-cbest -cap_table myTech_bc.CapTbl -T 50
  create_rc_corner -name rc-cworst -cap_table myTech_wc.CapTbl
  create_rc_corner -name rc-typical -cap_table myTech_typ.CapTbl
  ```

  The following command returns a list of all defined RC corners in the design:

  ```
  all_rc_corners
  ```

  The software returns the following results:

  ```
  rc-cbest rc-cworst rc-typical
  ```

- The following command reads a spef file for each defined RC corner in the design:

  ```
  set corners [all_rc_corners]
  foreach corner $corners { read_spef -rc_corner $corner $corner.spef.gz }
  ```

Related Information

- create_rc_corner

all_setup_analysis_views

```
all_setup_analysis_views
```

Returns a Tcl list of all active setup analysis views in the design.

Use this command after setting active setup analysis views for the design (set_analysis_view).

Parameters

None

Example

- The following command defines the active setup and hold analysis views for the design:
  ```
  set_analysis_view -setup {missionWCCOM mission2WCCOM} -hold {missionBCCOM testBCCOM}
  ```
- The following command returns a list of the active setup analysis views in the design:
  ```
  all_setup_analysis_views
  ```

  The software returns the following results:
  ```
  {missionWCCOM mission2WCCOM}
  ```

Related Information

- set_analysis_view

calNegSlack

`calNegSlack`

Reports the worst slack, total negative slack, and the number of violating paths.

Example

The `calNegSlack` command reports the total negative slack, worst negative slack, and the number of violating paths. You can also find this report in the log file.

calNegSlack

Total negative slacks(TNS)= xxx

Worst negative slacks(WNS)= xxx

Number of violating paths= xxx

check_instance_library_in_views

```
check_instance_library_in_views
```

Checks for missing libraries for instances in the active view.

This command checks whether the library for the cell associated with the instance is present in the associated power domain library set of the instance in the active views.

Parameters

None

Examples

- The following command displays a report showing missing libraries:

  ```
  check_instance_library_in_views
  ```

  The software displays the following information:

  ```
  Checking the Library binding for instance in active view 'vA'
  Pass. All instances have library definition in view 'vA'
  Checking the Library binding for instance in active view 'vB'
  No library found for instance 'U1', cell 'AND2X1', in powerdomain 'PD_BAD', of view
  'vB'
  Fail. Missing Library for some instance found in view 'vB'
  Checking the Library binding for instance in active view 'vAB'
  Pass. All instances have library definition in view 'vAB'
  ```

check_ldb_version

```
check_ldb_version
-ldb string
[-help]
```

Reports the LDB version, size, and the source of the software that is used to generate the library database (LDB) file. The LDB is Cadence binary library format.

Parameters

`-help`	Prints out the command usage.
`-ldb string`	Specifies the LDB (library database) file whose version is to be printed.

Examples

The following example displays the missing features/recommendations for the library database cell_w.db:

```
> check_ldb_version -ldb cell_w.ldb
```

The output is as shown below:

```
LDB file compiled with tool version: v15.20 ((64bit) mm/dd/yyyy 08:40 (Linux 2.6.18-371.el5))
File Size is xxxxx
Source Tool: Innovus
```

check_timing

```
check_timing
[-type type_list ]
[-verbose]
[-check_only warning_list]
[-clock_crossing_output_format {summary | csv}]
[-include_warning warning_list ]
[-exclude_warning warning_list ]
[-view analysis_view_name ]
[ port_or_pin_list ]
[-tcl_list]
[{> | >> } file_name [.gz]]
```

Performs a variety of consistency and completeness checks on the timing constraints specified for a design.

The checks include arrival time and external delay (or required time) for each clock in a multiple clock system. In addition, clock connectivity and data connectivity are checked to make sure the clock or data is propagated as expected. Clock gating points are also reported.

Use this command on generic or mapped netlists. The `check_timing` command considers constants applied to a pin while reporting warnings.

> Use the `check_timing` command before using any commands for timing analysis. Warnings are displayed when problems occur. Refer to check_timing warnings. For best results, rerun the `check_timing` command after resolving each warning. Redirecting the output to a file is the quickest run method.

Parameters

`-check_only` *warning_list*	Allows you to specify warning types. For example, the following command lists the specified warnings only: `check_timing -check_only {uncons_endpoint clock_crossing}` This list can also include warnings that are not listed by default. Valid entries in warning_list are: clock_clipping_freq, clock_clipping_gate, clock_crossing, clock_expected, clock_gating_controlling_edge_unknown, clock_gating_no_data, clock_missing_at_sequential_output, clock_not_propagated, clocks_masked_by_another_clock, const_collision, data_check_multiple_reference_signal, data_check_no_reference_signal, ideal_clock_waveform, invalid_pll_configuration, loop, master_clk_edge_not_reaching, missing_clock_groups, missing_drive, multiple_clock, no_drive, no_gen_clock_source, no_input_delay, partial_input_delay, pulse_non-pulse_clock_merge, signal_level, uncons_endpoint, uncons_endpoints_due_to_case_analysis
`-clock_crossing_output_format {summary | csv}`	Specifies the format of the clock crossing report to be generated - as a summary or in csv format.

{> | >>} *file_name*	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *filename* parameter must be the last argument in the list. The *filename* and -tcl_list parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the .gz extension to the file name. *Default:* Report is displayed on standard output without being saved.
-exclude_warning *warning_list*	
	Excludes the specified warning types when generating the report. Use this parameter to exclude any of the warnings included in the report by default. By default, the software includes the following warnings in the report: clock_expected, loop, ideal_clock_waveform, master_clk_edge_not_reaching, partial_input_delay, no_input_delay, no_drive, no_gen_clock_source, uncons_endpoint,uncons_endpoints_due_to_case_analysis. The complete list of warnings and their descriptions is given in check_timing warnings.
-include_warning *warning_list*	

	Includes the specified warning types when generating the report. Use this parameter to add any of the warnings not included in the report by default. By default, the software reports all warnings except the following: `clock_clipping_gate`, `const_collision`, `clock_crossing`, `clock_not_propagated`, `missing_drive`, `multiple_clock`, `data_check_multiple_reference_signal`, `data_check_no_reference_signal`, `clock_gating_no_data`, `clock_gating_controlling_edge_unknown`, `clock_missing_at_sequential_output`, `signal_level`, `clocks_masked_by_another_clock`. The complete list of warnings and their descriptions is given in check_timing warnings.
port_or_pin_list	Lists pins or ports. For example: `check_timing -verbose [get_ports in*]`
`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *`filename`* parameters are mutually exclusive; you cannot specify them together.
`-type` *type_list*	Limits the checks to the specified type. Valid types are: `clocks`, `endpoints`, `inputs`, `nets`, `loops`, `clock_clipping`, `clock_gating_inferred`, and `constant_collision`. *Default*: All types. Only check violations will be reported. To report all the checks you can use the `timing_check_timing_report_all_checks` global variable. When set to `true`, the software reports all the checks performed by the `check_timing` command, even if there are no violations. By default, this global variable is set to `false`.

	`clocks`: Reports the following clock warnings: • clock_crossing • clock_expected • clock_not_propagated • ideal_clock_waveform • master_clk_edge_not_reaching • no_gen_clock_source
	`endpoints`: For output ports, warns if no output delay constraint is applied to the port or if a specific (min/max rise/fall) output delay is missing. It also warns if an output delay constraint is applied with no clock information. For all endpoints (output ports and register data pins), it warns if any incoming signal is unconstrained. Also warns if a clock arrives where clock is not expected, or if multiple signals arrive at an endpoint.
	`inputs`: Reports the following input delay, arrival time assertion, and drive assertion warnings: • partial_input_delay • missing_drive • no_input_delay • no_drive
	`nets`:
	`loops`: Warns when a combinational loop is detected. Reports the loop and the arc used to break the loop. The `-pin` option is ignored in this case. Also reports loops caused by generated clocks.

	`clock_clipping`: Warns when there is a problem with clock gating that can result in clock clipping. Reports the following warnings: • clock_clipping_freq • clock_clipping_gate
	`clock_gating_inferred`: Warns when inferred clock gating checks are not analyzed by the software, even if a clock signal reaches and is used downstream the clock tree. Reports the following warnings. • clock_gating_controlling_edge_unknown • clock_gating_no_data
	`constant_collision`: Warns when there is a constant collision on a net connected to the pin or if there is a contradiction on a pin. Constant collision occurs when a net is driven simultaneously by conflicting values and a constant contradiction occurs when a constant constraint on a pin conflicts with the driven value.
`-verbose`	Gives detailed information about the warnings. When specified, warnings are grouped and sorted by warning type. *Default:* Gives a summary of how many warnings of each type exists.
`-view`	Reports warning(s) that belong to the specified analysis view.

Command Order

Use this command after setting all the constraints but before optimizing your design or generating any timing reports.

Warning Messages

The following table shows the `check_timing` warning messages.

check_timing Warning Messages

Warning	Description
`clock_clipping_gate`	**Warning**: Clock clipping possible due to wrong gate type or wrong trigger for data input.
`clock_clipping_freq`	**Warning**: Clock clipping possible due to incompatible clock signal and data signal frequencies.

`clock_crossing`	**Warning**: Clock domains interact. **Comments:** Checks clock interactions between multiple clock domains in the design. When you specify this parameter, the Timing Check Crossing Clocks report shows the interaction of clocks in the design and indicates whether the paths between the clocks are `False` or `Partial False`. If a clock launches one or more paths, which are captured by other clocks, the report lists all pairs of crossing clocks. • This includes inferred clock gating with different reference and signal clock. • If a path has been disabled using the `set_disable_timing` command, crossing clocks on that path are not reported. If all the paths between two clocks are false paths or they are exclusive/asynchronous clocks, the path is marked as `False`. The false path can be defined on clocks, for example, `set_false_path -clock_to CLK` or on pins or ports, such as, `-through buf1/Y`. If only part of paths are set as false paths or exclusive/asynchronous clocks, the path is marked as `Partial False`. • If a false path is defined only for one check type (say, setup), the crossing clocks are marked as `Partial False` if the simultaneous setup hold mode is set to `on`. The crossing clocks are marked `False` if the analysis type is setup and the simultaneous setup hold mode is set to `off`. • If only one edge (rise/fall) of a particular clock pair has false paths (say, using `-clock_fall`), then the path is marked as a `Partial False`.

`clock_expected`	**Warning:** Clock not found where clock is expected. **Comments:** Indicates that no clock signal is defined at the clock pin. See create_clock.
`clock_gating_controlling_edge_unknown`	**Warning:** Unknown controlling edge to perform clock gating check. **Comments:** Reports library cells with complex functions (MUX or AOI) where controlling edge of signal to perform clock gating check cannot be determined automatically. You need to specify correct edge by `-high`/`-low` option using the `set_clock_gating_check` command to resolve such warning. **Note:** The software does not report this warning when either: • `set_disable_clock_gatin g_check` command is applied on such pins, or • global settings to disable clock gating checks are on.

`clock_gating_no_data`	**Warning:** No data signal reaching clock gating enable pin **Comments:** Reports clock gating enable pins where no valid data signal arrives. You need to do either of the following: • specify valid data signal to reach enable pin of clock gating cells, or • remove any path exceptions or disabled checks that blocks valid data signal to reach enable pin **Note:** The software does not report this warning when either • `set_disable_clock_gatin g_check` is applied on such pins, or • global settings to disable clock gating checks are on.
`clocks_masked_by_another_clock`	**Warning:** Clock(s) in fanin cone are dropped due to another clock **Comments:** Reported timing pins on which creation of a regular or a generated clock causes clock(s) in fanins of such pins have been blocked or masked from propagating further. However, for pins having only master clock waveform on the fanin, this warning is not reported. You can use the detailed mode (`-verbose` option) to report names of all the incoming clock waveforms that have been blocked.

`clock_missing_at_sequential_output`	**Warning:** Found latency phase at sequential output on clock tree instead of clock phase **Comments:** Reports output pins of sequential elements that lie on source clock tree for the generated clocks created downstream and do not have any valid clock phase. This indicates possible missing clock related checks (e.g., clock gating) on downstream clock tree. You can add clocks on this pin or downstream clock tree only if no valid clock exists until downstream generated clock creation point.
`clock_not_propagated`	**Warning:** Clock not propagated **Comments:** Reports ideal clocks reaching the reference pin of a timing check.
`const_collision`	**Warning:** Constant collision **Comments:** See set_case_analysis.
`data_check_multiple_reference_signal`	**Warning:** Multiple clocked signals are reaching data check reference pin **Comments:** Reports on reference pins for non sequential timing check (data check) where more than one reference data arrives.
`data_check_no_reference_signal`	**Warning:** No clocked signal is reaching data check reference pin **Comments:** Reports reference pins for non-sequential timing check (data check) where no valid reference signal (data phase) arrives. You can do one of the following to resolve such warnings: • add valid reference signals to reach at such reference pins • remove path exceptions or disabled arcs that are blocking such reference signals

`ideal_clock_waveform`	**Warning:** Clock waveform is ideal. **Comments:** Reports clocks that are ideal and are not propagated. This warning type is reported by default (unless `-exclude_warning` or `-check_only` parameter is specified). You can use the set_propagated_clock command to avoid this warning.
`invalid_pll_configuration`	**Warning:** No valid path exists from output pin to feedback pin of PLL. **Comments**: Shows PLL (phase locked loop) check warnings. These checks allow you to identify and fix issues in the PLL flow.
`latch_fanout_clock_connectivity`	**Warning**: Latch and its fanout latch work on same clock. **Comments**: Checks for a latch and its fanout latch(es) that works on the same clock.
`loop`	**Warning:** Timing loop found in the design. **Comments:** Indicates that there are timing loops in the design. It also includes loops caused by generated clocks.
master_clk_edge_not_reaching	**Warning:** Master clock edge does not reach the generated clock target. **Comments:** This warning means that rise/fall latency edge is not reaching to create generated clock root from the master clock. You can verify if the path that is seeing this warning is actually broken, or edge is somewhere disabled earlier on this path. There is a global `timing_enable_genclk_edge_based_source_latency` that controls how the software chooses generated clock source latency paths. Details of this global can be found at Tempus text command reference.

`missing_clock_groups`	**Warning**: Set clock group related checking. **Comments:** Reports any potential issues with the defined clock grouping data.
`partial_input_delay`	**Warning:** Indicates that a rise or fall input delay assertion is not applied.
`missing_drive`	**Warning:** Missing specific (rise/fall) drive constraint **Comments:** Either a rise or fall drive constraint is applied and the other one is missing.
`multiple_clock`	**Warning:** Multiple clocks are reaching clock reference pin. **Comments:** Reports clock pins of sequential or clock gating cells where more than one clock waveform arrives. This warning is for information only and the software performs timing analysis with respect to all such clocks.
`no_input_delay`	**Warning:** No input delay assertion with or without `-clock` option specified. **Comments:** See set_input_delay.
`no_drive`	**Warning:** No drive constraint **Comments:** See set_driving_cell and set_drive.
`no_gen_clock_source`	**Warning:** No clock source found for generated clock. **Comments:** Indicates that the source pin of the clock generated by the create_clock command is not driven by a clock source.
`pulse_non-pulse_clock_merge`	**Warning**: Merge of pulse and non-pulse clock paths. **Comments**: Reports points where a pulse clock and a non-pulse clock merges.

`signal_level`	**Warning:** Driver pin voltage value do not match with sink pin voltage level **Comments:** Reports net driver/receiver pin pairs for which the pin voltage does not match.
`uncons_endpoint`	**Warning:** No constrained signal reaching pin. **Comments:** For output ports: Primary output port has no implicit timing check. Use set_output_delay, set_max_delay, or set_min_delay to create a valid constraint. For register data pins: There is no valid clock reaching the CK pin of the register data pin. Check for missing create_clock for the clock network. The endpoints are reported as unconstrained in both in early and late timing. This does not cover all the cases and scenarios because of which endpoint can become unconstrained. To cover all possible cases, you can use the `report_analysis_coverage` command.
`uncons_endpoints_due_to_case_analysis`	**Warning:** Unconstrained signal arriving at end point due to case analysis **Comments:** Reports unconstrained endpoints due to case analysis. To enable this warning, you must set the `timing_report_check_timing_unconstrained_endpoints_due_to_constants` global variable to `true`.

Examples

- Consider the following example:

```
in---------|>-----------|>---------out

buf1 buf2
```

The Timing Check Crossing Clocks reports False or Partial False in some of the scenarios given below:

a) `set_input_delay` with respect to CLK1 on in and set_output_delay with respect to CLK2 on out:

From Clock Crossing Clocks

CLK1 CLK2

b) `set_input_delay` with respect to CLK1 on in, set_output_delay with respect to CLK2 on out, and set_false_path -to CLK2 (clock-based false path):

From Clock Crossing Clocks

CLK1 CLK2 (False)

c) If the set_false_path becomes set_false_path -rise_to CLK2, the following is reported:

```
From Clock          Crossing Clocks
```

CLK1 CLK2 (Partial False)

The following command displays the default summary report:

```
tempus> check_timing

------------------------------------------------------------------------------
                              TIMING CHECK SUMMARY
------------------------------------------------------------------------------------------
---------
Warning                       Warning Description                              Number
of

Warnings
------------------------------------------------------------------------------------------
---------
ideal_clock_waveform          Clock waveform is ideal
```

```
     1
clock_expected                Clock not found where clock is expected
     3
master_clk_edge_not_reaching  Master clock edge does not reach the generated clock
target  2
no_drive                      No drive assertion
      8
no_input_delay                No input delay assertion with respect to clock
      4
uncons_endpoint               Unconstrained signal arriving at end point
      5
--------------------------------------------------------------------------------
---------
```

- The following command displays a specified type of warning using the `-type` option if a timing loop is found in the design:

```
check_timing -type loops

-------------------------------------------------------
                  TIMING CHECK SUMMARY
-------------------------------------------------------
Warning      Warning Description              Number of
                                              Warnings
-------------------------------------------------------
loop         Timing loop found in the design  0
-------------------------------------------------------
```

- The following command provides more information about problems and prints a summary using the `-verbose` option:

```
check_timing -verbose

-----------------------------------------------------------------------------
                           TIMING CHECK SUMMARY
-----------------------------------------------------------------------------
-----------
Warning                       Warning Description
Number of
```

```
Warnings
-------------------------------------------------------------------------------
-------------
ideal_clock_waveform          Clock waveform is ideal
    1
clock_expected                Clock not found where clock is expected
    3
master_clk_edge_not_reaching  Master clock edge does not reach the generated clock
target 2
no_drive                      No drive assertion
     8
no_input_delay                No input delay assertion with respect to clock
     4
uncons_endpoint               Unconstrained signal arriving at end point
     5
-------------------------------------------------------------------------------
---------
----------------------------------------
TIMING CHECK DETAIL
----------------------------------------
Pin         Warning
-----------------------------------------------------------------
u99/CK      Clock not found where clock is expected
u5/CK       Clock not found where clock is expected
u8/CK       Clock not found where clock is expected
u8/Q        Master clock edge does not reach the target for generated clock GCLK4
CLK1        No drive assertion
in1         No drive assertion
in1         No input delay assertion with respect to clock
out1        Unconstrained signal arriving at end point
-----------------------------------------------------------------
-------------------------
TIMING CHECK IDEAL CLOCKS
-------------------------
Clock  Waveform
------------------------
CLK1
------------------------
```

- When `timing_report_check_timing_unconstrained_endpoints_due_to_constants` global variable is enabled, the case analysis warnings are reported as shown below:

```
> check_timing
--------------------------------------------------------------------------------
                                TIMING CHECK SUMMARY
--------------------------------------------------------------------------------
Warning              Warning Description                                Number of
                                                                        Warnings
--------------------------------------------------------------------------------
clock_expected       Clock not found where clock is expected              4202
no_drive             No drive assertion                                   241
no_input_delay       No input delay assertion with respect to clock       160
uncons_endpoint      Unconstrained signal arriving at end point           145259
uncons_endpoints_due_to_case_analysis
                  >>>>Unconstrained signal arriving at end point due to case analysis
                                                                        5969437
--------------------------------------------------------------------------------
```

Related Information

- report_timing
- set_input_delay
- create_clock
- set_case_analysis
- set_output_delay
- set_max_delay
- set_min_delay
- set_drive
- set_driving_cell
- set_false_path
- set_clock_gating_check
- set_disable_clock_gating_check

checkTimingLibrary

```
checkTimingLibrary
[-outfile string]
[-cellName string]
[-library string]
[-checkPower]
[-reportMissingPowerOnly]
```

Checks the contents of the timing library and reports any inconsistencies in a log file.

You can use the `checkTimingLibrary` command after importing the design and timing libraries.

Parameters

`-cellName` *string*	Checks if the timing is defined for all cells with the specified name, in all timing libraries. **Note:** In general, the `-cellName` parameter is optional. However, if you specify the `-library` parameter, then you *must* specify `-cellName`.
`-checkPower`	Checks if power constructs are defined for the cells, such as internal_power, leakage_power, or ecsm_power. **Note:** In general, the `-checkPower` parameter is optional. However, if you specify `-reportMissingPowerOnly`, then you *must* specify `-checkPower`. If you use the `-library` parameter, you do not have to specify the `-checkPower` parameter, because when a library-specific cell is given as input, the software automatically checks for power constructs. *Default:* Only checks if timing is defined for the cells.

`-library` *string*	Specifies the timing library in which to check a particular cell. You must specify the `-cellName` parameter with `-library`, to specify the particular library cell to check. **Note:** If you use the `-library` parameter, you do not have to specify the `-checkPower` parameter, because when a library-specific cell is given as input, the software automatically checks for power constructs.
`-outfile` *string*	Writes the report to the specified file.
`-reportMissingPowerOnly`	Reports only the cells that do not have internal_power (at pin level), or leakage_power defined. You must specify the `-checkPower` parameter with the `-reportMissingPowerOnly` parameter.

Examples

- The following command checks if timing is defined for all cells that have been read into the design, and writes a report to the log file:

  ```
  checkTimingLibrary
  ```

- The following command checks if timing is defined for all cells that have been read into the design, and writes a report to a file named `CheckTiming1`:

  ```
  checkTimingLibrary -outfile CheckTiming1
  ```

- The following command checks if timing and power constructs are defined for all cells that have been read into the design, and writes the report to the log file:

  ```
  checkTimingLibrary -checkPower
  ```

- The following command checks if timing and power constructs are defined for all cell that have been read into the design, and writes the report to a file named `CheckTiming2`:

  ```
  checkTimingLibrary -outfile CheckTiming2 -checkPower
  ```

- The following command checks if timing is defined for all cells with the name `A01` in all timing libraries, and writes the report to a file named `CheckTiming3`:

  ```
  checkTimingLibrary -outfile CheckTiming3 -cellName A01
  ```

- The following command checks if timing and power constructs are defined for all cells with the name `A01`, in all timing libraries, and writes the report to a file named `AllA01Check`:

  ```
  checkTimingLibrary -outfile AllA01Check -cellName A01 -checkPower
  ```

- The following command checks if timing and power constructs are defined for the cell with the name `A01` in the timing library `minFile1.lib`, and writes the report to a file named `A01Check`:

  ```
  checkTimingLibrary -outfile A01Check -cellName A01 -library minFile1.lib
  ```

  Note: When you use the `-library` parameter, you do not have to specify the `-checkPower` parameter, because when a library-specific cell is given as input, the software automatically checks for power constructs.

- The following command checks if the internal_power and/or leakage_power constructs are defined for all cells that have been read into the design, and reports only those cells that do not have the constructs defined. The software writes the report to a file named `MissingPower`:

  ```
  checkTimingLibrary -outfile MissingPower -checkPower -reportMissingPowerOnly
  ```

create_analysis_view

```
create_analysis_view
-name viewName
-constraint_mode modeName
-delay_corner dcCornerObj
```

Creates an analysis view object that associates a delay calculation corner with a constraint mode. An analysis view object provides all of the information necessary to control a single multi-mode multi-corner analysis.

After creating analysis views, use the set_analysis_view command to specify which views to use for setup and hold optimization or timing analysis.

Use this command after creating constraint modes (create_constraint_mode) and delay calculation corners (create_delay_corner).

Parameters

`-name` *viewName*	Specifies the name of the analysis view being defined.
`-constraint_mode` *modeName*	Specifies the name of the constraint mode to associate with this analysis view. A constraint mode groups a set of constraints to be used with any given analysis. To create the constraint mode, use the create_constraint_mode command.
`-delay_corner` *dcCornerObj*	Specifies the name of the delay calculation corner object to associate with this analysis view. A delay calculation corner object contains all of the information needed to control delay calculation for a specific analysis view. To create a delay calculation corner, use the create_delay_corner command.

Examples

- The following command creates an analysis view called `missionSlow` using the constraint mode `missionSetup` and the delay calculation corner `dcWCCOM`:

```
create_analysis_view
```

```
-name missionSlow
-constraint_mode missionSetup
-delay_corner dcWCCOM
```

- The following command creates an analysis view called `missionFast` using the constraint mode `missionHold` and the delay calculation corner `dcBCCOM`:

```
create_analysis_view
-name missionFast
-constraint_mode missionHold
-delay_corner dcBCCOM
```

Related Information

- `set_analysis_view`
- `create_constraint_mode`
- `create_delay_corner`

create_constraint_mode

```
create_constraint_mode
-name modeName
-sdc_files {file1.sdc file2.sdc ...}
[-ilm_sdc_files {file1.sdc file2.sdc ...}]
[-tcl_vars {{var_name1 value1} {var_name2 value2} {var_name3 value3} ...}]
```

Associates a list of SDC constraint files with a specified constraint mode name, for multi-mode multi-corner analysis. This constraint mode name can be referred to later when creating analysis views. A constraint mode defines one of possibly many different functional, test, or Dynamic Voltage and Frequency Scaling (DVFS) modes of a design. SDC files can be shared by many different constraint modes, and the same constraint mode can be associated with multiple analysis views.

SDC files typically contain timing analysis information, such as the clock specifications, case analysis constraints, I/O timings, and path exceptions that make each mode unique. If the ILM is being used, the timing system utilizes the files specified with `-ilm_sdc_files` during the flatten ILM mode (`flattenIlm`); otherwise, the system uses the files specified with `-sdc_files`.

Parameters

`-ilm_sdc_files {file1.sdc file2.sdc ...}`	Specifies a Tcl list of ILM SDC files to be included in the constraint mode.
`-name modeName`	Specifies the name of the constraint mode to be created.
`-sdc_files {file1.sdc file2.sdc ...}`	Specifies a Tcl list of SDC files to be included in the constraint mode.
`-tcl_vars {{var_name1 value1} {var_name2 value2} {var_name3 value3} ...}`	
	Specifies a list of tcl variables and values to be used in the constraint mode.

Examples

- The following command groups the SDC files `io.sdc`, `mission1-clks.sdc`, and `mission1-except.sdc` to create a mode object named `missionSetup`:

```
create_constraint_mode -name missionSetup

-sdc_files [list io.sdc mission1-clks.sdc mission1-except.sdc]
```

- The following command groups the SDC files `io.sdc`, `mission1-clks.sdc`, `mission1-except.sdc`, and `dont_use.sdc` to create a mode object named `missionHold`:

```
create_constraint_mode -name missionHold

-sdc_files [list io.sdc mission1-clks.sdc mission1-except.sdc      dont_use.sdc]
```

- The following command groups the SDC files `test-io.sdc`, `test-clks.sdc`, and `test-except.sdc` to create a mode object named `testHold`:

```
create_constraint_mode -name testHold

-sdc_files [list test-io.sdc test-clks.sdc test-except.sdc]
```

Related Information

- `update_constraint_mode`
- `create_analysis_view`

create_delay_corner

```
create_delay_corner
[-help]
[-early_estimated_worst_irdrop_factor <percent>]
[-late_estimated_worst_irdrop_factor <percent>]
-name delayCornerName
[-si_enabled true | false]
{ { [-library_set libSetObj] | [-late_library_set libSetObj -early_library_set libSetObj ]
}
[-opcond_library libName ]
[-opcond opcondName]
[-pg_net_voltages voltage_pairs]
[-rc_corner rcCornerObj | [-late_rc_corner rcCornerObj -early_rc_corner rcCornerObj]]
[-irdrop_file list_of_files | -irdrop_data files_or_directories]
[-late_opcond_library libName]
[-early_estimated_worst_irDrop_factor percent]
[-early_opcond_library libName]
[-late_opcond opCondName]
[-early_opcond opCondName]
[-late_estimated_worst_irDrop_factor percent]
[[-late_irdrop_file list_of_files -early_irdrop_file libSetObj] | [-
late_irdrop_data files_or_directories -early_irdrop_data files_or_directories]]
[-temp_file tempName | [-late_temp_file tempName -early_temp_file tempName]] }
```

Creates a named delay calculation corner object that can be referenced later when creating an analysis view. A delay calculation corner provides all of the information necessary to control delay calculation for a specific view. Each corner contains information on the libraries to use, the operating conditions with which the libraries should be accessed, and the RC extraction parameters to use for calculating parasitic data. Delay corner objects can be shared by multiple top-level analysis views.

The `-early_*` and `-late_*` parameters should not be used to represent best and worst case corners. Instead use separate delay corners to define Best-Case and Worst-Case differences. Use the -early_* and -late_* parameters within a single delay calculation corner to control on-chip variation. In this latter situation different conditions or libraries will be used for the launch and capture paths of individual timing paths.

Note: A single delay calculation corner object specifies the delay calculation rules for the entire

design. If a design includes power domains, the delay calculation corner can contain domain-specific subsections that specify the required operating condition information, and any necessary timing library rebinding for the power domain. Use the `update_delay_corner` command to add a power domain definition to a delay calculation corner.

Parameters

`-help`	Prints out the command usage.
`-early_estimated_worst_irDrop_factor` *`per cent`*	Specifies the early voltage derating factor. When this parameter is specified, the software performs delay calculations using the voltage = nomVolt * (1 – value). *Default*: 0.0
`-early_estimated_worst_irdrop_factor` *`percent`*	Specifies the early voltage derating factor.
`-early_irdrop_data` *`files_or_directories`*	Specifies the IR drop files or directories to apply when calculating the early arrival times at a single delay corner.
`-early_irdrop_file` *`list_of_files`*	Specifies the list of IR drop files to apply when calculating early arrival times at a single delay corner. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis. **Note**: This parameter is obsolete and will be removed in a future release. To ensure compatibility with future releases, you should update your scripts.
`-early_library_set` *`libSetObj`*	Specifies the library set to associate with this delay corner object for calculating early arrival times at a single delay corner. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis.

`-early_opcond` *opcondName*	Specifies the operating condition to use for calculating early arrival times at a single delay corner. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis. *Default:* Each library in the early library set uses its own default operating condition.
`-early_opcond_library` *libName*	Specifies the internal library name for the library in which the early operating condition is defined. This is *not* the library file name. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis. *Default:* The software searches the early library set for the specified operating condition (`-early_opcond`), starting with the master library.
`-early_rc_corner` *rcCornerObj*	Specifies the RC corner object to associate with the early corner object.
`-early_temp_file` *tempName*	Specifies the name of the temperature file for temperature aware delay calculation for early corner.
`-irdrop_data` *files_or_directories*	Specifies the IR drop files or directories to apply to both the early and late delay calculation for the specified delay corner object.
`-irdrop_file` *list_of_files*	Specifies the list of IR drop files to apply to both early and late delay calculation for this delay corner object. This parameter is used primarily to configure single-corner or Best-Case Worst-Case (BC-WC) analysis modes. **Note**: This parameter is obsolete and will be removed in a future release. To ensure compatibility with future releases, you should update your scripts.

`-late_estimated_worst_irDrop_factor` *percent*	Specifies the late voltage derating factor. When this parameter is specified, the software performs delay calculations using the voltage = nomVolt * (1 – value). *Default*: 0.0
`-late_irdrop_data` *files_or_directories*	Specifies the IR drop files or directories to apply when calculating the late arrival times at a single delay corner.
`-late_irdrop_file` *list_of_files*	Specifies the list of IR drop files to apply when calculating late arrival times at a single delay corner. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis. **Note**: This parameter is obsolete and will be removed in a future release. To ensure compatibility with future releases, you should update your scripts.
`-late_library_set` *libSetObj*	Specifies the library set to associate with this delay corner object for calculating late arrival times at a single delay corner. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis.
`-late_opcond` *opcondName*	Specifies the operating condition to use for calculating late arrival times at a single delay corner. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis. *Default:* Each library in the late library set uses its own default operating condition.

`-late_opcond_library` *libName*	Specifies the internal library name for the library in which the late operating condition is defined. This is *not* the library file name. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis. *Default:* The software searches the late library set for the specified operating condition (`-late_opcond`), starting with the master library.
`-late_rc_corner` *rcCornerObj*	Specifies the RC corner object to associate with the late corner object.
`-late_temp_file` *tempName*	Specifies the name of the temperature file for temperature aware delay calculation for late corner.

`-library_set` *libSetObj(s)*	Specifies the library set(s) to associate with this delay corner object. To configure an ECSM-based multi-library interpolation , you should specify a Tcl list of library_set, such as:
	`create_delay_corner -name dc_P1_T1`
	`-library_set {p1_v1_t1`
	`p1_v2_t1`
	`p1_v3_t1}`

`-name` *delayCornerName*	Specifies the name for the delay corner object being created.
`-opcond` *opcondName*	Specifies the operating condition to use for setup and hold analysis. This parameter is used primarily to configure single-corner or Best-Case Worst-Case (BC-WC) analysis modes. *Default:* Each library in the library set uses its own default operating condition.

`-opcond_library` *libName*	Specifies the internal library name for the library in which the operating condition is defined. This is *not* the library file name. This parameter is used primarily to configure single-corner or Best-Case Worst-Case (BC-WC) analysis modes. *Default:* The software searches the library set for the specified operating condition (`-opcond`), starting with the master library.
`-pg_net_voltages` *voltage_pairs*	Specifies the power/ground net voltages to be used for both early/late timing analysis.
`-rc_corner` *rcCornerObj*	Specifies the RC corner object to associate with this delay corner object. This parameter is used primarily to configure OCV and Best-Case Worst-Case (BC-WC) analysis modes.
`-si_enabled {true | false}`	Controls signal integrity (SI) analysis. When set to `true`, the analysis of all MMMC views will include SI analysis by default. You can use this parameter to further determine which analysis views receive SI analysis and which do not. When this parameter is set to `false` on a delay corner, the software will disable SI analysis for all the views that reference that corner. By default (or when this parameter is set to `true`), SI analysis is computed for all the views related to that corner - whenever SI is globally enabled. (The global control of signal integrity (SI) analysis is determined by the `setDelayCalMode -SIAware` setting.)
`-temp_file` *tempName*	Specifies the name of the temperature file for temperature aware delay calculation for single corner.

Examples

- The following command creates a delay calculation corner called `dcWCCOM`. This corner uses the libraries from `IsCOM-1V`, sets the operating condition to `WCCOM`, as defined in the `stdcell_1V` timing library, and uses the `rc-cworst` RCcorner:

```
create_delay_corner -name dcWCCOM
-library_set IsCOM-1V
-opcond_library stdcell_1V
-opcond WCCOM
-rc_corner rc-cworst
```

- The following command creates a delay calculation corner called `dcBCCOM`.This corner uses the libraries from `IsCOM-1V`, sets the operating condition to `BCCOM`, as defined in the `stdcell_1V` timing library, and uses the `rc-cbest` RC corner:

```
create_delay_corner -name dcBCCOM
-library_set IsCOM-1V
-opcond_library stdcell_1V
-opcond BCCOM
-rc_corner rc-cbest
```

- The following commands create best-case (`dcBBCOM-OCV`) and worst-case (`dcWCCOM-OCV`) delay calculation corners. The OCV variance in each corner is due to different early and late libraries and operating conditions within each delay corner.

```
create_delay_corner -name dcBCCOM-OCV
-late_library_set BCCOM-SLOW
-early_library_set BCCOM-FAST
-late_opcond_library BCCOM-SLOW
-early_opcond_library BCCOM-FAST
-late_opcond SLOW
-early_opcond FAST


create_delay_corner -name dcWCCOM-OCV
-late_library_set WCCOM-SLOW
-early_library_set WCCOM-FAST
-late_opcond_library WCCOM-SLOW
-early_opcond_library WCCOM-FAST
```

```
-late_opcond SLOW
-early_opcond FAST
```

Related Information

- update_delay_corner
- get_delay_corner

create_library_set

```
create_library_set
[-help]
[-aocv string]
[-library_side_file string]
-name libSetName
[-si string]
[-socv string]
[-timing string]
```

Associates a TCL list of timing and cdB/UDN libraries with a specified library set name. Library sets allow a group of library files to be treated as a single entity so that higher-level descriptions (delay calculation corners) can simply refer to the library configuration by name. The same library set can be referenced multiple times by different delay calculation corners.

Timing libraries can be in text format or Cadence binary library format (LDB). The LDB (library database) files generated from other tools, like Voltus, or Genus are read in to the software with an error message (TECHLIB-1249). The LDB files are not forward compatible, that is, a LDB generated from 15.2 version cannot be used with 15.1 or prior versions of the software.

Note: The `create_library_set` command determines the priority of ECSM vs CCS data (if both are present). The software first checks for ECSM construct, if it is missing then CCS construct is loaded.

> The order in which you define timing libraries is important. The software considers the first library you specify in the list as the master library, with each successive library having a lower priority.

Parameters

`-help`	Prints out the command usage.
`-aocv string`	Specifies the AOCV libraries to include in the library set.
`-library_side_file string`	Specifies the library side file.

`-name` *libSetName*	Specifies the name for the library set being created. This name is used to associate the library list with a specific delay calculation corner.
`-si` *string*	Specifies cdB libraries and/or user-defined noise (UDN) models to include in the library set. The cdB and/or UDN libraries are required for performing signal integrity analysis. **Note**: You cannot use this parameter for CCS-N or ECSM-N models, that is, for cdBs only.
`-socv` *string*	Specifies the SOCV libraries to include in the library set.
`-timing` *string*	Specifies the timing libraries to include in the library set. **Note:** Libraries intended for use in an ECSM-based multi-library interpolation must be grouped within an additional level of Tcl list structure to identify the intended usage to the library subsystem. Specify three timing libraries that are characterized at different voltages (typically the lower and upper voltage bounds, and a third intermediate point). All voltages required by the software will be interpolated from this set of libraries. The libraries in the set can differ only in the characterization voltage; they must be identical to each other in all other aspects. However, if you want to perform multi-temperature based library interpolation, you can use two library files that are characterized at different temperatures for each voltage value. See the examples for creating an ECSM-based multi-library set and multi-temperature based library interpolation in the Examples section.

The ECSM and CCS libraries can be very large and loading them may take significant time. You can convert the ASCII NLDM (Liberty .lib), ECSM, and CCS libraries to a binary version using the `write_ldb` command. For conversion of libraries to a binary version, refer to `write_ldb`.

Instead of the original ASCII libraries, you can specify these binary libraries in the configuration file:

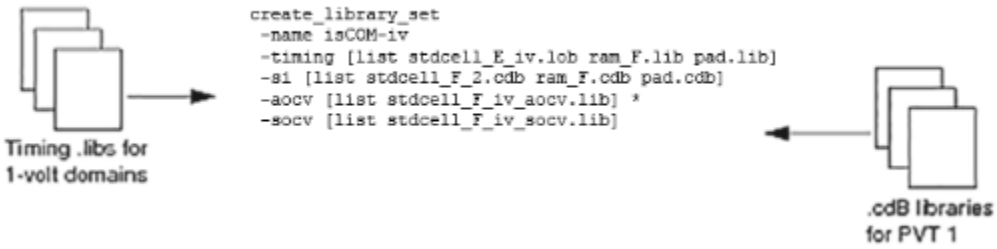
- **Original MMMC setup file**:

```
create_library_set -name libName -
timing "AAA.lib BBB.ecsm CCC.ccs"
```

- **New MMMC setup file**:

```
create_library_set -name libName -
timing "AAA.ldb BBB.ldb CCC.ldb"
```

The library sets can be created only after the top module is set. The following figure shows the creation of a library set that associates timing libraries and cdB libraries with a nominal voltage of 1 volt with the library name isCOM-1V.



Note: Either AOCV or SOCV libraries can be specified at a time. The AOCV and SOCV files can also be added to the library set by either `-aocv [list aocv1.lib aocv2.lib]` or `-socv [list socv1.lib socv2.lib]` in the `create_library_set` settings.

Examples

- The following command creates a library set that associates timing libraries and cdB libraries with a nominal voltage of 1 volt with the library name `IsCOM-1V`:

```
create_library_set
-name IsCOM-1V -timing [list stdcell_F_1V.lib ram_F.lib pad.lib]
-si [list stdcell_F_2.cdb ram_F.cdb pad.cdb]
```

- The following command creates a library set that can be used to specify the 2-volt version of a core cell library, which could be associated later with a 2-volt MSMV power domain specified in the design:

```
create_library_set
-name IsCOM-2V
-timing [list stdcell_F_2V.lib]
```

Related Information

- `get_library_set`
- `update_library_set`

create_op_cond

```
create_op_cond
-name virtualOpcondName
-library_file libraryFileName
-P processValue
-V voltageValue
-T temperatureValue
```

Creates a set of virtual operating conditions in the specified library without actually modifying the library. These virtual operating conditions can then be referenced by a delay calculation corner (`create_delay_corner`) as if they actually exist in the library.

Parameters

`-library_file` *libraryFileName*	Specifies the library for which to create the virtual operating conditions.
`-name` *virtualOpcondName*	Specifies the name for the operating condition being created.
`-P` *processValue*	Specifies the process value for the operating condition.
`-T` *temperatureValue*	Specifies the temperature value for the operating condition.
`-V` *voltageValue*	Specifies the voltage value for the operating condition.

Examples

- The following command creates a virtual operating condition called `PVT1` for the library `IsCOM-1V`.lib:

```
create_op_cond -name PVT1
   -library_file IsCOM-1V.lib
   -P 1.0
   -V 1.2
   -T 120
```

- The following command creates a virtual operating condition called `PVT2` for the library `IsCOM-2V.lib`:

```
create_op_cond -name PVT2
   -library_file IsCOM-2V.lib
   -P 1.0
   -V 1.2
   -T 20
```

Related Information

- `get_op_cond`
- `create_delay_corner`

create_property_alias

```
create_property_alias
[-help]
-map <alias_propery_name_pairs>
-obj_type {pin port cell net clock lib_cell lib_pin design lib timing_path timing_point
timing_arc path_group lib_timing_arc si_victim si_attacker lib_pg_pin pg_pin pg_net}
```

Creates a property alias for the selected objects.

Parameters

`-help`	Prints out the command usage.
`-map <alias_propery_name_pairs>`	Specifies pair list of alias and property to be associate with that alias, such as, {alias1 prop1} or {{ialias1 prop1}{alias2 prop2}...}.
`-obj_type {pin port cell net clock lib_cell lib_pin design lib timing_path timing_point timing_arc path_group lib_timing_arc si_victim si_attacker lib_pg_pin pg_pin pg_net}`	Specifies the object type for which a property alias needs to be created.

create_rc_corner

```
create_rc_corner
-help
[-T temperatureValue]
[-cap_table capTableFile]
-name rcCornerName
[-postRoute_cap {capFactor1 capFactor2 capFactor3}]
[-postRoute_clkcap {capFactor1 capFactor2 capFactor3}]
[-postRoute_clkres {resFactor1 resFactor2 resFactor3}]
[-postRoute_res {resFactor1 resFactor2 resFactor3}]
[-postRoute_xcap {xcapFactor1 xcapFactor2 xcapFactor3}]
[-preRoute_cap capFactor]
[-preRoute_clkcap capFactor]
[-preRoute_clkres resFactor]
[-preRoute_res resFactor]
[-qx_tech_file fileName]
[-via_variation_file viaVariationFile]
```

Creates a named RC corner object that can be referenced later when creating a delay calculation corner object. An RC corner object provides the software with all of the information necessary to properly extract, annotate, and use the RCs for delay calculation.

Note: You must use the RC corner scaling attributes when running the software in multi-mode multi-corner analysis mode. Use the `setExtractRCMode` command to set the extraction mode for all RC corner objects in the design.

Note: In multi-mode multi-corner mode, only the default RC corners created by the software inherit the derating factor values defined in the configuration file.

You can set different postroute scale factors for the postroute engine variants by specifying them as duplets and triplets. For example,

resFactor1 resFactor2 resFactor3

Where:

- Single value: If you specify one value, the scale factor applies to effort level `low`. Scale factor value of `1` is used for `medium` and `high` by default.
- Duplet: If you specify two values, the first value is used for effort level `low` and the second value is used for `medium`. Scale factor value of `1` is used for `high` by default.
- Triplet: If you specify three values, the first value is used for effort level `low`, the second value is used for `medium`, and the third value is used for `high`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each create_rc_corner parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_rc_corner`.
`-cap_table` *capTableFile*	Specifies the capacitance table to use for extraction when using this RC corner. *Default:* The Innovus extraction system uses internal rules for extracting resistances and capacitances.
`-preRoute_cap` *capFactor*	Specifies the capacitance scale factor for RC Extraction in preRoute mode. *Default:* `1.0`
`-preRoute_clkcap` *capFactor*	Specifies the scale factor for capacitance of clock nets in preRoute mode. *Default:* `0` When this parameter is not specified, the value specified with the `-postRoute_cap` parameter is used for preRoute extraction. When specified, this parameter overrides the value specified with the `-postRoute_cap` parameter for clock nets.
`-preRoute_clkres` *resFactor*	Specifies the scale factor for resistance of clock nets in prerRoute mode. *Default:* `0` When this parameter is not specified, the value specified with the `-postRoute_res` parameter is used for preRoute extraction. When specified, this parameter overrides the value specified with the `-postRoute_res` parameter for clock nets.

`-preRoute_res` *resFactor*	Specifies the resistance scale factor for RC extraction in preRoute mode.
`-postRoute_cap` {*capFactor1 capFactor2 capFactor3*}	Specifies the capacitance scale factor for RC extraction in postRoute mode. *Default:* `1.0`
`-postRoute_clkcap` {*capFactor1 capFactor2 capFactor3*}	Specifies the scale factor for capacitance of clock nets in the postRoute extraction mode. *Default:* `0` When this parameter is not specified, the value specified with the `-postRoute_cap` parameter is used for postRoute extraction. When specified, this parameter overrides the value specified with the `-postRoute_cap` parameter.
`-postRoute_clkres` {*resFactor1 resFactor2 resFactor3*}	Specifies the scale factor for resistance of clock nets in the postroute extraction mode. *Default:* `0` When this parameter is not specified, the value specified with the `-postRoute_res` parameter is used for postRoute extraction. When specified, this parameter overrides the value specified with the `-postRoute_res` parameter for clock nets.
`-postRoute_res` {*resFactor1 resFactor2 resFactor3*}	Specifies the resistance scale factor for RC extraction in post-route mode.
`-postRoute_xcap` {*xcapFactor1 xcapFactor2 xcapFactor3*}	Specifies the coupling capacitance scale factor for RC extraction in postroute mode. *Default:* `1.0`
`-name` *rcCornerName*	Specifies the name for the RC corner object being created. You can choose any name for an RC corner object, but it is recommended that you use a name that can be easily recognized as referring to a specific RC corner (for example, rc-min, rc-max, c-min, c-max, and so on).

`-qx_tech_file` *fileName*	Specifies the name of the Quantus technology (`.tch`) file used by TQuantus/IQuantus or Quantus sign-off RC extraction. The technology file is generated by TechGen, a built-in functionality of Quantus. For most technologies you can get this file from technology vendors. For technologies that are not available through the technology vendors, you enter the fabrication process information into an ASCII-format interconnect technology (ICT) input file. The ICT file is then used as input to IceCaps, which in turn generates the technology file. For more information on creating the ICT files, see the chapter, "Creating the ICT File", in the *Innovus User Guide*.
`-T` *rcTemperatureValue*	Specifies the temperature, in units of Celsius, to use to derate resistance values. Use this parameter when you want to override the default temperature specified in the capacitance table or the Quantus technology file. **Note:** The software does not require that the temperature specified by the PVT operating condition at the delay calculation corner level be the same as the RC nominal or user-specified temperature value. *Default:* The Innovus extraction system uses the default temperature of 25 degrees Celsius, unless it is overridden by the capacitance table or the Quantus technology file.

`-via_variation_file` *viaVariationFile*	Specifies a file for modeling via variation. The VIA variation file lists up all the VIA layer names with the corresponding multiplier. Each RC corner can have its own VIA variation file.
	For example:
	`create_rc_corner`
	`-rc_corner rcCornerName1 \`
	`-via_variation_file viaVariationFileName1`
	`create_rc_corner`
	`-rc_corner rcCornerName2 \`
	`-via_variation_file viaVariationFileName2`

Examples

- The following command creates an RC corner called `rc-cbest` that uses the capacitance table `myTech_bc.CapTbl`, and derates the resistance values based on the temperature of 50 Celsius:
  `create_rc_corner -name rc-cbest -cap_table myTech.4_bc.CapTbl -T 50`

Related Commands

- `update_rc_corner`

createUserDisableForCombLoopBreak

```
createUserDisableForCombLoopBreak
-input_file fileName
-outfile fileName
```

Generates SDC file containing constraints for combinational loop breaking. The command selects the disable arc (s) with flag set to 1 from the report file, and generates the equivalent SDC commands in an output file.

Note: You need to set `set_interactive_constraint_modes` command before sourcing the SDC file.

Parameters

`-input_file` *fileName*	Specifies the disabled time arcs report that you generate. To generate the report, use the following command: `report_disbale_timing -nosplit > pt_disable_arcs.rpt`
`-outfile` *fileName*	Specifies the name of the output SDC file.

Command Order

Use this command after generating the disable arcs report.

Example

The following command generates an `encCombLoopBreak.sdc` file when the input is a `pt_disable_arcs.rpt` file:

```
createUserDisableForCombLoopBreak -input_file pt_disable_arcs.rpt
           -outfile encCombLoopBreak.sdc
```

The contents of `pt_disable_arcs.rpt` file are as follows:

Cell or Port	From To Sense Flag Reason
----------	
uchip_core/u_s5_power	oLpcSd_4_ orig_oSmAcpiSd_4_ * 1

The contents of output file `encCombLoopBreak.sdc` are as follows:

```
:
set_disable_timing -from {oLpcSd_4_} -to {orig_oSmAcpiSd_4_}
                [get_cells {uchip_core/u_s5_power}]:
```

display_timing_map

```
display_timing_map
[-help]
{[-step float  -threshold float ] -check_type {setup | hold}] | -clear}
```

Sets color-coded slack threshold values for worst timing paths.

Parameters

`-help`	Prints a brief description that includes type and default information for each `display_timing_map` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man display_timing_map`
`-check_type {setup | hold}`	Specifies the check type for timing paths. The possible arguments are `setup` and `hold`.
`-clear`	Clears the timing slack threshold values.
`-step`	Sets the step value.
`-threshold`	Sets color-coded slack threshold values for worst timing paths. You can specify a range of threshold values. When specified, the instances as seen on the Timing Debug form are color-coded according to the worst timing slack threshold value. For information on timing map, see "Display Timing Map" in the *Innovus Menu Reference Guide* .

Examples

- The following example sets a threshold value of -1, for setup timing check:
  ```
  display_timing_map -step 0.2 -threshold -1 -check_type setup
  ```

get_analysis_view

```
get_analysis_view
viewName
{-constraint_mode | -delay_corner}
```

Returns the constraint mode or delay calculation corner associated with the specified analysis view.

Use this command after creating at least one analysis view (create_analysis_view).

Parameters

`-constraint_mode`	Returns the constraint mode associated with the specified analysis view.
`-delay_corner`	Returns the delay calculation corner associated with the specified analysis view.
viewName	Specifies the name of the analysis view to query.

Examples

- The following command creates the analysis view `missionSlow`:

  ```
  create_analysis_view
  -name missionSlow
  -constraint_mode missionSetup
  -delay_corner dcWCCOM
  ```

- The following command returns the delay calculation corner associated with the `missionSlow` analysis view:

  ```
  get_analysis_view missionSlow -delay_corner
  ```

  The software returns the following result:

  ```
  dcBCCOM
  ```

Related Information

- setAnalysisMode

get_capacitance_unit

`get_capacitance_unit`

Returns the session capacitance unit in picofarads.

Example

The following command returns the session capacitance unit:

get_capacitance_unit

1.000000pf

Related Information

- `setLibraryUnit`

get_clock_network_objects

```
get_clock_network_objects
[-help]
[-clocks clock_list]
[-type {cell | pin}]
```

Returns the cells or pins that exist in the clock network of the specified clocks.

Parameters

`-help`	Prints out the command usage.
`-clocks` *`clock_list`*	Returns the cells or pins present in the clock network of the specified clock(s).
`-type {cell | pin}`	Returns the specified cells or pins in the clock network.

Examples

- The following command returns the pins present in the clock network of all the clocks:
  ```
  get_clock_network_objects
  ```
- The following command returns the pins that exist in the clock network of CLK and CLK2 clocks:
  ```
  get_clock_network_objects -clocks {CLK CLK2}
  ```
- The following command returns the cells that exist in the clock network of CLK clock.
  ```
  get_clock_network_objects -clocks {CLK} -type cell
  ```

get_constant_for_timing

```
get_constant_for_timing
[-bidi_input | -bidi_output]
pin_name
```

Queries the design database for the state of the specified pin or the state propagated through the combinational logic cone to that pin. The returned value is `0` for logic 0, `1` for logic 1, `X` for undefined, and `Z` for tristate.

Note: Constants on hierarchical ports are not computed.

The `get_constant_for_timing` command returns the state for one pin only. If you want to know the state on all ports that have a constant set, use the `-type constant` option with the report_ports command. For example:
`report_ports -type constant`. Use the `-tcl_list` option to process the list in Tcl.

See report_net examples for an example of the `-tcl_list` output.

Parameters

`-bidi_input | -bidi_output`	Returns the constant that is set on the input or output part of bidirectional pins. *Default*: Returns the constant on the output part.
pin_name	Specifies the name of the single pin to query for pin state. The pin must be an instance pin or primary I/O port.

Examples

- The following command sets the value of the specified pin(s) to either a 1 or 0 for use by the timing engine: `set_case_analysis 0 J_block/zbuf0/A`
- The following command queries the design database for the state of the specified pin or the state

propagated through the combinational logic cone to that pin: `get_constant_for_timing J_block/zbuf0/A`

Related Information

- `report_ports`
- `set_case_analysis`
- `report_case_analysis`

get_constraint_mode

```
get_constraint_mode
modeName
{-sdc_files | -ilm_sdc_files | -tcl_vars}
```

Returns a Tcl list of the regular SDC constraint files, or the ILM SDC constraint files associated with the specified constraint mode.

Use this command after creating constraint modes (create_constraint_mode).

Parameters

`-ilm_sdc_files`	Returns the ILM SDC files associated with the specified constraint mode.
modeName	Specifies the name of the constraint mode to query.
`-sdc_files`	Returns SDC files associated with the specified constraint mode.
`-tcl_vars`	Returns a list of tcl variables associated with the specified constraint mode.

Example

- The following command creates the constraint mode `missionSetup`:

  ```
  create_constraint_mode -name missionSetup


  -sdc_files [list io.sdc mission1-clks.sdc mission1-except.sdc]
  ```

- The following command returns the SDC constraint files associated with `missionSetup`:

  ```
  get_constraint_mode missionSetup -sdc_files
  ```

  The software returns the following Tcl list:

  ```
  {io.sdc mission1-clks.sdc mission1-except.sdc}
  ```

Related Information

- create_constraint_mode
- update_constraint_mode

get_delay_corner

```
get_delay_corner
[-help]
delayCornerName
[-early_irdrop_data]
[-early_irdrop_file]
[-early_library_set]
[-early_opcond]
[-early_opcond_library]
[-early_rc_corner]
[-early_temp_file]
[-irdrop_data]
[-irdrop_file]
[-late_irdrop_data]
[-late_irdrop_file]
[-late_library_set]
[-late_opcond]
[-late_opcond_library]
[-late_rc_corner]
[-late_temp_file]
[-library_set]
[-opcond]
[-opcond_library]
[-power_domain powerDomainName]
[-power_domain_list]
[-rc_corner]
[-si_enabled]
[-supply_set supplySetName]
[-supply_set_list]
[-temp_file]
```

Returns attribute information for the specified delay calculation corner object, or for one of its power domains.

Use this command after creating at least one delay calculation corner object (`create_delay_corner`). The delay calculation corner must include a power domain definition in order to query power domain information (`update_delay_corner`).

Parameters

`-help`	Prints a brief description that includes type and default information for each `get_delay_corner` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_delay_corner`.
`parameter_names`	See `create_delay_corner` for descriptions of the parameters that you can specify.

Examples

- The following command creates the delay calculation corner `dc1`:

```
create_delay_corner -name dc1
-library_set libs-2volt
-opcond_library delayvolt_2V
-opcond slow_2V
-rc_corner rc-cworst
```

- The following command returns the operating condition for the delay calculation corner `dc1`:

```
get_delay_corner dc1 -opcond
```

  The software returns the following results:

```
slow_2V
```

- The following commands add definitions for power domains `domain-3V` and `domain- 5V` to the delay calculation corner `dc1`:

```
update_delay_corner -name dc1
-power_domain domain-3V
-library_set libs-3volt
-opcond_library delayvolt_3V
-opcond slow_3V
update_delay_corner -name dc1
-power_domain domain-5V
-library_set libs-5volt
-opcond_library delayvolt_5V
```

```
-opcond slow_5V
```

- The following command returns a list of all the power domains associated with the delay calculation corner `dc1`:

  ```
  get_delay_corner dc1 -power_domain_list
  ```

  The software returns the following results:

  ```
  {domain-3V domain-5V}
  ```

- The following command returns the name of the library set associated with the power domain `domain-5V`:

  ```
  get_delay_corner dc1 -power_domain domain-5V -library_set
  ```

  The software returns the following results:

  ```
  libs-5volt
  ```

Related Information

- `create_delay_corner`
- `update_delay_corner`

get_equivalent_cells

```
get_equivalent_cells
[-help]
-cell string
[ [ { { -delay_corner string [ -early_only | -late_only ] } | -max |-min } [ -power_domain
string ] ] | -library string ]
```

Returns equivalent cells from minimum/maximum libraries or from libraries of specific power domains.

Parameters

`-cell` *string*	Specifies the cell name for which equivalent cells will be fetched.
`-delay_corner` *string*	Specifies delay corner for which equivalent cells from libraries will be fetched.
`-early_only`	Fetches cells only for early libraries of the specified delay corner.
`-late_only`	Fetches cells only for late libraries of the specified delay corner.
`-library` *string*	Specifies the library for which cells will be fetched.
`-max`	Fetches equivalent cells for max libraries only.
`-min`	Fetches equivalent cells for min libraries only.
`-power_domain` *string*	Fetches cells for libraries of a specific power domain.

Examples

- The following command will return all the cells equivalent to BUF_X4 (buffers) present in the power domain PD1 in the max libraries:

  ```
  > get_equivalent_cells -cell BUF_X4 -max -power_domain PD1
  ```

  Output:

  ```
  CORELIB/BUF_X1 CORELIB/BUF_X16 CORELIB/BUF_X2 CORELIB/BUF_X32 CORELIB/BUF_X4
  ```

- The following command will return all the AND cells of different drives present in the library USERLIB:

  ```
  > get_equivalent_cells -cell AND_X8 -library USERLIB
  ```

get_interactive_constraint_modes

```
get_interactive_constraint_modes
[-help]
```

Returns a list of the multi-mode multi-corner constraint modes that are in interactive constraint entry mode.

When the software is in interactive constraint entry mode (`set_interactive_constraint_modes`), any timing constraints you specify take effect immediately on all active analysis views that are associated with these constraint modes.

Note: Interactive constraint mode only works when the software is in multi-mode multi-corner timing analysis mode.

Related Information

- `set_interactive_constraint_modes`

get_lib_cell_leakage_power

```
get_lib_cell_leakage_power
[-help]
-cell string
[-file string]
[-stateleakagegrp]
[ {-view string | -delay_corner string}
[-power_domain string]]
[ -early  | -late  ]
```

Returns the default leakage power of a cell.

Parameters

`-help`	Prints out the command usage
`-cell` *string*	Specifies the cell Name for which Leakage Power need to be fetched (string, required)
`-delay_corner` *string*	Selects cell from specified delay corner
`-early`	Selects cell from early library of the specified delay corner. If this option is not specified by default late will be used. (bool, optional)
`-file` *string*	Specifies a.Name of the output file. (string, optional)
`-late`	Selects cell from late library of the specified delay corner. If this option is not specified by default late will be used. (bool, optional)
`-power_domain` *string*	Selects cell from the library of the specified power domain (string, optional)
`-stateleakagegrp`	Reports state dependent leakage power information.
-view *string*	Selects the cell from the specified view.

Examples

- The following example reports state independent cell leakage power along with return value:
  ```
  get_lib_cell_leakage_power -cell xyz
  ```

- The following command shows state dependent groups:

  ```
  get_lib_cell_leakage_power -cell xyz -stateleakagegrp
  ```

Related Information

- report_power

get_library_set

```
get_library_set
libSetName
{-timing  | -si | -aocv | -socv}
```

Returns a Tcl list of the timing or cdB libraries for the specified library set.

Use this command after creating at least one library set (`create_library_set`).

Parameters

`-aocv`	Returns AOCV library names for the specified library set.
libSetName	Specifies the name of the library set.
`-si` *libSetName*	Returns the cdb library names for the specified library set.
`-socv`	Returns the SOCV library names for the specified library set.
`-timing`	Returns a list of the timing libraries for the specified library set. This list contains the names of files from where the libraries were sourced from.

Examples

- The following commands create the library sets `IsCOM-1V`:

  ```
  create_library_set -name IsCOM-1V -timing [list stdcell_F_1V.lib ram_F.lib pad.lib]
   -si [list stdcell_F_2.cdb ram_F.cdb pad.cdb}]
  ```

- The following command returns a TCL list of the timing libraries for library set `IsCOM-1V`:

  ```
  get_library_set -timing IsCOM-1V
  ```

  The software returns the following results:

```
{stdcell_F_1V.lib ram_F.lib pad.lib}
```

Related Information

- create_library_set

get_op_cond

```
get_op_cond
virtualOpcondName
{-library_file | -P | -V | -T}
```

Returns the process, voltage, or temperature value for the specified operating condition.

Use this command after creating at least one operating condition (`create_op_cond`).

Parameters

`-library_file`	Specifies the library file in which the operating condition is defined.
`-P`	Returns the process value for the operating condition.
`-T`	Returns the temperature value for the operating condition.
`-V`	Returns the voltage value for the operating condition.
`virtualOpcondName`	Specifies the name of the operating condition to query

Examples

- The following command creates an operating condition called `myPVT2` for the library `IsCOM-2V.lib`:

  ```
  create_op_cond -name myPVT2
     -library_file IsCOM-2V.lib
     -P 1.0
     -V 1.2
     -T 20
  ```

- The following command creates a collection called myPVT2-temp that contains the temperature

value of the `myPVT2` operating condition:

```
set myPVT2-temp [get_op_cond myPVT2 -T]
```

Related Information

- `create_op_cond`

get_pba_mode

```
get_pba_mode
[-help]
[-epba_bounded_mode]
[-quiet]
[-nonDefault]
```

Displays information for the IPBA mode set using the `set_pba_mode` command.

Parameters

`-help`	Prints out the command usage.
`-epba_bounded_mode`	Enables/disables the IPBA mode with exhaustive PBA analysis (enabled using the `report_timing -retime_mode exhaustive` command).
`-quiet`	Displays the current settings in Tcl list format only.
`-nonDefault`	Displays the current settings which are not default settings.

Related Commands

- `set_pba_mode`
- `report_timing`

get_propagated_clock

```
get_propagated_clock
[-clock clock_list] [-pin pin_list]
[> filename]
```

Returns the clock propagation mode, either `ideal` or `propagated`, for the specified clocks or pins.

Parameters

`> filename`	Specifies the name of a file in which to save the report. You can add the `.gz` extension to the file name to generate a compressed report. **Note:** The file name must be the last argument in the list.
`-clock clock_list`	Specifies one or more clock waveforms for which you want to retrieve the clock propagation mode information.
`-pin pin_list`	Specifies one or more pins or ports for which you want to retrieve the clock propagation mode information. You cannot specify hierarchical pins.

Example

- The following command gets the clock propagation mode information for all clock waveforms:
  ```
  get_propagated_clock -clock {*}
  ```

Related Information

- `set_propagated_clock`
- `reset_propagated_clock`

get_rc_corner

```
get_rc_corner
rcCornerName
-rcCornerAttribute
```

Returns attribute information for the specified RC corner object.

Use this command after creating at least one RC corner object (create_rc_corner).

Parameters

-rcCornerAttribute	Returns the setting for the specified RC corner attribute. You can query one of the following attributes: `-cap_table`, `-T`, `-preRoute_res`, `-postRoute_res`, `-preRoute_cap`, `-postRoute_cap`, `-postRoute_xcap`, `-qx_tech_file`. For attribute information, see the create_rc_corner command.
rcCornerName	Specifies the name of the RC corner to query.

Examples

- The following commands create the RC corners `rc-cbest` and `rc-cworst`:

  ```
  create_rc_corner -name rc-cbest -cap_table myTech_bc.CapTbl -T 50
  create_rc_corner -name rc-cworst -cap_table myTech_wc.CapTbl
  ```

- The following command returns the capacitance table specified for `rc-cworst`:

  ```
  get_rc_corner rc-cworst -cap_table
  ```

  The software returns the following results:

  ```
  myTech_wc.CapTbl
  ```

Related Information

- create_rc_corner
- update_rc_corner

get_resistance_unit

```
get_resistance_unit
```

Returns the resistance unit for the current session.

Example

The following command returns the resistance unit for the current session:

```
get_resistance_unit
```

1.000000 kohm

get_socv_rc_variation_factor

```
get_socv_rc_variation_factor
[-help]
-view viewName
{-early | -late }
```

Displays the variation factor for interconnect delays.

Parameters

`-help`	Prints out the command usage.
`-early`	Displays the variation factor for early interconnect delays.
`-late`	Displays the variation factor for late interconnect delays.
`-view viewName`	Displays the SOCV interconnect variation factor for specified view.

Related Commands

- set_socv_rc_variation_factor

get_socv_reporting_nsigma_multiplier

```
get_socv_reporting_nsigma_multiplier
[-help]
-view viewName
{[-setup | -hold ]}
```

Allows you to query sigma multiplier setting for the specified view in setup or hold mode.

Parameters

`-help`	Prints out the command usage.
`-hold`	Returns the sigma multiplier for hold mode.
`-setup`	Returns the sigma multiplier for setup mode.
`-view viewName`	Returns the sigma multiplier value for the specified view. **Note**: Applies to active views only.

Examples

- The following command displays the sigma multiplier setting for the specified view:

```
innovus > set_socv_reporting_nsigma_multiplier -hold 2.5 -view
default_emulate_view_hold
innovus > get_socv_reporting_nsigma_multiplier -view default_emulate_view_hold -hold
2.500000
```

Related Commands

- set_socv_reporting_nsigma_multiplier

get_time_unit

```
get_time_unit
```

Returns the current time unit for the session.

Example

- The following command returns the time unit for the session:
  get_time_unit

  1.000000ns

Related Information

- setLibraryUnit

getAnalysisMode

```
getAnalysisMode
[-analysisType]
[-aocv]
[-asyncChecks]
[-caseAnalysis]
[-checkType]
[-clkNetsMarking]
[-clkSrcPath]
[-clockGatingCheck]
[-clockPropagation]
[-cppr]
[-enableMultipleDriveNet]
[-honorActiveLogicView]
[-honorClockDomains]
[-log]
[-multi_input_switching_mode]
[-propSlew]
[-nonDefault]
[-quiet]
[-sequentialConstProp]
[-skew]
[-socv]
[-timeBorrowing]
[-timingEngine]
[-timingSelfLoopsNoSkew]
[-usefulSkew]
[-useOutputPinCap]
[-warn]
```

Returns the following information about a specified timing analysis mode parameter in the System log file and in the System console:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by the user

If you do not specify a parameter, the software returns information for all timing analysis mode parameters.

Parameters

parameter_names	Returns information for the specified parameters. You can specify one or more parameters. See `setAnalysisMode` for descriptions of the timing analysis mode parameters you can specify.
`-quiet`	Returns the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software returns the current settings of all `setAnalysisMode` parameters in Tcl list format.

Examples

- The following command returns the current setting of the `-checkType` parameter:

  ```
  getAnalysisMode -checkType

  The software returns the following information:

  -checkType setup # enums={setup hold}, default=setup

  setup
  ```

- The following command returns the current setting of the `-checkType` parameter in Tcl list format:

  ```
  getAnalysisMode -checkType -quiet
  ```

  The software returns the following information:

  ```
  setup
  ```

- The following command returns the current settings for the `-checkType`, `-skew`, and `-analysisType` parameters:

  ```
  getAnalysisMode -checkType -skew -analysisType

  The software returns the following information:
  ```

```
-checkType setup # enums={setup hold}, default=setup

-skew true # bool, default=true

-analysisType bcwc # enums={single bcwc onChipVariation}, default=bcwc

{checkType setup} {skew true} {analysisType bcwc}
```

- The following command returns the current settings for the `-checkType`, `-skew`, and `-analysisType` parameters in Tcl list format:

  ```
  getAnalysisMode -checkType -skew -analysisType -quiet
  ```

  The software returns the following information:

  ```
  {checkType setup} {skew true} {analysisType bcwc}
  ```

Related Information

- `setAnalysisMode`

getTimeLibFile

```
getTimeLibFile
library_name
```

Reports the library files that match the specified library name.

Parameters

library_name	Specifies the library name for which to check.

read_path_descriptions

```
read_path_descriptions
[-help]
filename
[-hpin]
[-path_type {end summary full full_clock end_slack_only summary_slack_only}]
[-retime {aocv ssta path_slew_propagation aocv_path_slew_propagation
waveform_propagation}]
```

Reads the file created by the `write_path_descriptions` command. This command reads the complete description of the paths specified in the file in another Tempus session.

The paths description when read back can be saved as a collection that can be used as a regular path collection for any reporting command or queried using the `get_property` command.

Parameters

`-help`	Prints out the command usage.
`filename`	Specifies the name of the input file.
`-hpin`	Prints all the hierarchical crossings of an arc under the "hpin" column in the `report_timing` table.
`-path_type {end | summary | full | full_clock | end_slack_only | summary_slack_only}`	Allows you to choose the format of the report based on the path type.
`-retime {aocv | ssta | path_slew_propagation | aocv_path_slew_propagation | waveform_propagation}`	Reanalyzes the specified set of paths using the specified analysis method.

Examples

- The following example command reads out the pathfile1.txt. The file contains details of the collection of timing paths with hierarchical pins (`-hpin`) format based on timing and exact path availability in the session in which the path description is read:

  ```
  read_path_descriptions pathfile1.txt -hpin
  ```

Related Information

`write_path_descriptions`

read_sdf

```
read_sdf
[-help]
sdf_file_name
[-continue_on_error]
[-ilm_filter]
[-increment]
[-overwrite_incremental_delay]
[-path instanceName]
[-persistent]
[-print_summary]
[-scale float]
[-strict_cond_matching]
[-view viewName]
[-sdf_field {min|typ|max} | [-early_sdf_field {min|typ|max}] [-late_sdf_field
{min|typ|max}]]
```

Reads an OVI Standard Delay Format (SDF) file and annotates user-specified delay information from the SDF file into the timing system. You can use the `read_sdf` command to supply pre-calculated delays and timing check values from an external delay calculator, or from a previous Innovus session.

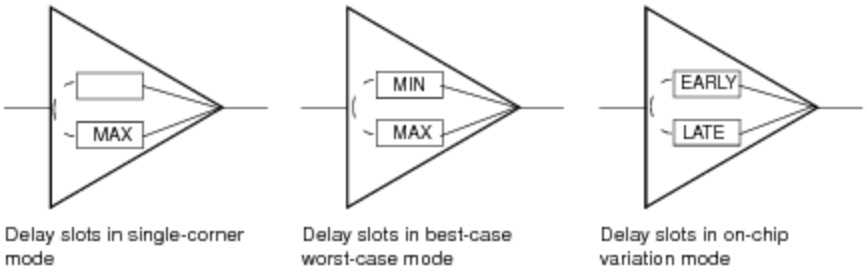
The `read_sdf` command can read version 2.1 and 3.0 of the SDF files, and can read both plain-text and GNU zipped (`.gz`) format files. The command and the Innovus software do not support the Timing Environment section of the SDF specification.

The SDF file can provide one, two, or three different values for each timing arc delay or timing check value. These values are passed in the min, typ, or max SDF delay fields, represented as min:typ:max.

Innovus provides multiple delay place holders to store the calculated or annotated delay information:

- In single-corner (`setAnalysisMode -analysisType single`) analysis mode, only max delays are available.
- In best-case worst-case (`setAnalysisMode -analysisType bcwc`) mode, a min delay slot is provided for storing min delays used for hold analysis. A max delay slot stores delays used for setup analysis.
- In on-chip variation (`setAnalysisMode -analysisType onChipVariation`) mode, the early delay slots store delays used for calculated early arrival times. The late delay slot stores delays used for calculated late arrival times.
- In multi-corner mode, an early and a late delay slot are provided for each multi-mode multi-corner analysis view.

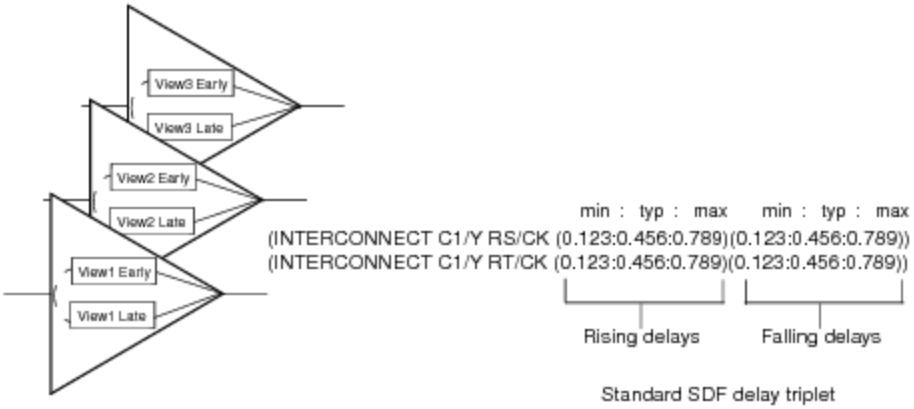
The `read_sdf` command provides parameters for controlling how the SDF min:typ:max triplet values are mapped into the Innovus software's delay slots. The correct mapping depends on how the delay information in the SDF file was created, and what type of analysis you are performing.



Delay slots in single-corner mode

Delay slots in best-case worst-case mode

Delay slots in on-chip variation mode



Delay slots in multi-corner mode

Standard SDF delay triplet

The timing system provides special handling for annotating incremental delays. These delays typically represent plus or minus offsets due to crosstalk effects. Incremental delay annotation to the Innovus software can be done by:

- Importing an incremental SDF file (SDF has an internal `INCREMENT` keyword) using the `read_sdf` command.
- Tcl sourcing a file of `set_annotated_delay -net -delta_only` constraints.

The incremental delay annotations can be viewed in the `report_timing` report by adding the `incr_delay` option to the `report_timing -format` list, or to the `report_timing_format` global variable.

Parameters

`-help`	Prints out the command usage.

`-continue_on_error`	Continues to read the file if an error occurs. By default, when an error occurs, the `read_sdf` command stops reading the file. Use this parameter to attempt to continue reading the file, if possible. **Note**: If you continue some annotations may be missing due to the errors. You can use the `report_annotated_parasitics` command to find the missing annotations. If the SDF file is corrupted, you can obtain a file.
`-early_sdf_field {min | typ | max}`	Annotates the early delay from the specified SDF field. In single-corner mode, all setup and hold analysis use maximum delays. The software ignores any minimum or early delay information. In best-case worst-case mode, use this parameter to specify which SDF field to use for hold analysis. In on-chip variation mode, the software uses the early delay for calculation of fast paths.
`-ilm_filter`	Ignores ILM-related errors. By default, the `read_sdf` command does not ignore errors reported due to missing arcs in core ILM.
`-increment`	Increments the total delay without affecting the delta delay. By default, when the SDF file contains the INCREMENT delay type, the `read_sdf` command will perform an adjustment on the delta delay slot which models SI cross-talk push-out/pull-in effects. When this option is used, the INCREMENT delays will be applied against the static base delay values instead.

`-late_sdf_field {min | typ | max}`	Annotates the late delay from the specified SDF field. In single-corner mode, use this parameter to specify which of the SDF triplet values to annotate for both setup and hold checks. In best-case worst-case mode, use this parameter to specify which SDF field to use for setup analysis. In on-chip variation mode, the software uses the late delay for the calculation of slow paths.
`-overwrite_incremental_delay`	When annotating an incremental SDF file, replaces any pre-existing delta delays with the new SDF values. The total delay will be equivalent to the original base delay, plus the new delta delay. If the delays in the incremental SDF file represent crosstalk delay push-outs from a signal integrity analysis, use the `-overwrite_incremental_delay` parameter to ensure that any stale data from a previous analysis is removed. By default, when an incremental SDF file (SDF has an internal `INCREMENT` keyword) is annotated, the delays are added to the base delay, and also stored separately as delta delays. If an additional incremental SDF file is annotated, the software adds the existing base and delta delays with the new values.
`-path` *instanceName*	Specifies the instance in the design to which the specified SDF file should be applied.
`-persistent`	Keeps the delays from certain SDF files persistently in use during various steps of the timing optimization . Use this parameter to support pre-timed structures, such as clock meshes.
`-print_summary`	Outputs the summary information of annotated delays.

`-scale` *float*	Overrides the automatic scaling of SDF delays to the library units by the value given. To disable all scaling, use a value of 1.0.
`-sdf_field {min | typ | max}`	Annotates the delay from the specified SDF slot. In single-corner and best-case worst-case modes, the software uses this value for both setup and hold analysis. In on-chip variation mode, the software uses the same value from the specified SDF slot for both the early and late delays of the specified arc.
sdf_filename	Specifies the name of the SDF file that has the back annotation data.
`-strict_cond_matching`	Annotates only the conditions that are an exact syntax-to-syntax match between the SDF file and the `.lib` file. *Default:* Uses looser conditional matching rules between the `.lib` and SDF arc specifications.
`-view` *viewName*	Annotates the specified analysis view(s) with the delays from the SDF file. You can use this parameter only when you are in multi-mode multi-corner analysis mode.

Examples

- The following command annotates delay information from the SDF min field to the min/early slots, and maps the SDF max field data to the max/late delay slots:

  ```
  read_sdf sdf_filename
  ```

  Given the following SDF, the default annotation would be:

  (INTERCONNECT C1/Y RS/CK (0.123:0.456:0.789) (0.123:0.456:0.789))

  - In single-corner analysis mode, a delay of 0.789 ns is annotated to the max delay slot.
  - In best-case worst-case mode, a delay of 0.123 ns is annotated to the min delay slot, and a value of 0.789 ns is placed in the max delay slot.

    - In on-chip variation mode, a delay of 0.123 is used for early delays, and a value of 0.789 ns is used for late delays.
- The following command annotates the SDF typ field to both the early and late delay slots:
  `read_sdf -sdf_field typ sdf_filename`
- The following command annotates the SDF typ field to the early delay slot, and the SDF max field to the late delay slot:
  `read_sdf -early_sdf_field typ -late_sdf_field max sdf_filename`
- The following command annotates the SDF typ field to the late delay slot. The early delay slot is annotated with the default min SDF field:
  `read_sdf -late_sdf_field typ sdf_filename`

Related Information

- write_sdf
- set_annotated_delay
- report_timing
- report_annotated_parasitics
- setAnalysisMode

read_spdf

```
read_spdf
{-help]
SPDF_file_name
```

Reads the statistical parameter distribution format (SPDF) file. An SPDF file contains statistics of variation of process parameters. The SPDF file is required for statistical static timing analysis (SSTA).

Parameters

`-help`	Prints out the command usage.
`SPDF_file_name`	Specifies the name of a SPDF file - that contains statistics of variation of process parameters.

read_twf

```
read_twf
[-help]
[filenames]
[-cell string]
[-prefix string]
[-quiet]
[-reset]
[-scope string]
[-skipconst]
[-skiptw]
[-strip_prefix string]
[-verbose]
[-view string]
```

Reads the external timing window file (TWF) for noise (Signal Integrity) and power calculation (Common Power Engine - CPE). This file is used to obtain clocks, constants, external load, slews, timing windows, and slack information. The external TWF specification is optional. In the absence of a TWF, this information is automatically derived using the Common Timing Engine (CTE), if timing constraints are read.

Note: If the `VOLTAGE_THRESHOLD` construct is missing from the TWF file header, the values 10 and 90 are automatically assigned as the minimum and maximum threshold values, respectively. For example:

```
VOLTAGE_THRESHOLD (10 90)
```

Parameters

`-help`	Prints out the command usage.
`-cell string`	Specifies the master cell for the timing window file.
filenames	Specifies the names of the timing window files.
`-prefix string`	Adds a prefix to the net or pin name.
`-quiet`	Skips information and warning messages.
`-reset`	Clears the previously read TWF files.

`-scope` *string*	Specifies the scope for the given hierarchical timing window file.
`-skipconst`	Skips constant processing.
`-skiptw`	Skips timing windows processing.
`-strip_prefix`	Removes hierarchical path name.
`-verbose`	Prints the command progress details.
`-view` *string*	Specifies the view for the timing window file.

Examples

- The following command reads the timing window file TW.in for the view 'view01', while skipping the constant processing from the timing window file:

  ```
  read_twf TW.in -skipconst -view view01
  ```

- The following command clears the previously read TWF file:

  ```
  read_twf -reset
  ```

Related Commands

- `write_timing_windows`

report_analysis_coverage

```
report_analysis_coverage
[-help]
[pin_port_list]
[-check_type check_type_list]
[-exclude_untested reason_list]
[-include_dyn_checks]
[-include_parallel_check_arcs]
[-include_when_cond]
[-max_paths{type integer}]
[-nworst <integer>]
[-retime {aocv | ssta | path_slew_propagation | aocv_path_slew_propagation}]
[-sort {pin | refpin | checktype | slack | reason}]
[-tcl_list]
[-unsorted]
[-verbose check_status_list]
[-view view_name]
[> <filename[.gz]>]
[>> <filename[.gz]>]
```

Provides information about the timing checks in the design. Valid check types are clock_gating_hold, clock_gating_pulse_width, clock_gating_setup, clock_period, clock_separation, external_delay, hold, no_change_hold, no_change_setup, path_delay, pulse_width, recovery, removal, setup, skew, and time_borrow. Valid status types are: met, untested, and violated.

For each type of timing check, the command reports the number of checks that meet constraints, violated constraints, or that are untested in the `SUMMARY` section. The `DETAILS` section includes information about the signal pin, reference pin, check type, slack, and the reason for being untested.

A `met` check shows as a positive value in the `Slack` column. A `violated` check shows as a negative value in the `Slack` column. An `untested` check shows as `UNTESTED` in the `Slack` column and the reason is given in the `Reason` column. The `Reason` column is blank when a `Slack` number is given because the check has been tested.

Note: If there are no timing checks in the design of the type specified by `-check_type`, the report shows "No timing checks found."

Use `check_timing` to look for missing constraints.

Parameters

{> | >>} *file_name*	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *filename* parameter must be the last argument in the list. The *filename* and -tcl_list parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the .gz extension to the file name. *Default:* Report is displayed on standard output without being saved.

`-check_type` *check_type_list*	Limits the report to the type of check(s) in the *check_type_list*. *Default*: All (all types of check(s) are reported) Valid check types are: • `setup`: Reports paths with setup timing checks. • `hold`: Reports paths with hold timing checks. • `pulse_width`: Reports paths with pulse width timing checks. • `clock_period`: Reports paths with clock period timing checks. • `clock_gating_setup`: Reports paths with clock gating setup timing checks. • `clock_gating_hold`: Reports paths with clock gating hold timing checks. • `clock_gating_pulse_width`: Reports paths with clock gating pulse width checks. • `recovery`: Reports paths with recovery timing checks. • `removal`: Reports paths with removal timing checks. • `clock_separation`: Reports paths with clock separation timing checks. • `skew`: Reports paths with skew checks. • `data_setup`: Reports paths with data to data setup checks. • `data_hold`: Reports paths with data to data hold checks.

`-exclude_untested` *reason_list*	Excludes from the report endpoints that are unconstrained due to a specific reason. Valid reasons are: • `const`: Constant reaching signal/reference pin of any timing check. • `false_path`: Path disabled due to `set_false_path`. • `no_endpoint_clock`: Missing clock at the reference pin of the timing check. • `no_startpoint_clock`: Missing clock at the signal pin of the timing check. This may happen if the clock signal does not reach the launching register. • `unknown`: None of the known reasons are identified for an untested path. • `user_disable`: Checks arc disable due to `set_disable_timing`.
`-include_dyn_checks`	Reports the dynamic checks (data_to_data, path_delay) on pins that are constrained by set_data_check, set_max_delay, or set_min_delay constraints. By default, these checks will not be reported and the command only reports the pins constrained by all library checks, set_output_delay or inferred clock gating checks. The dynamic checks, such as set_data_check or set_max_delay are not reported by default, when `-include_dyn_checks` parameter is not specified.
`-include_parallel_check_arcs`	Reports all the parallel check arcs that have been merged.
`-include_when_cond`	Reports the when condition of check arcs.
`-max_paths{type` *integer}*	Enumerates the specified number of worst paths in the design, regardless of the endpoint.

`-nworst` *integer*	Enumerates the specified number of paths for each endpoint.
`-retime {aocv | ssta | path_slew_propagation | aocv_path_slew_propagation}`	Reanalyzes the set of paths using the specified method.
pin_port_list	Limits the report to timing checks on specified pins or ports.

`-sort {pin | refpin | checktype | slack | reason}`	Sorts the checks by the specified method while reporting detailed information using the `-verbose` option. The `-sort` option does not change the order of the columns. The criteria by which sorting is attempted for the `slack`, `pin`, and `checktype` arguments is as follows: • When sorting by slack, if the slack values are the same, the software then attempts to sort the checks by pin name. If the pin names are the same, the software attempts to sort by timing check type. When you sort by `slack`, the slack value is reported in order of least negative to most negative (worst slack last). • When sorting by pin name, if the pin names are the same, the software then attempts to sort the checks by slack value. If the slack values are the same, the software attempts to sort by timing check type. • When sorting by timing check type, if the check types are the same, the software then attempts to sort by slack value. If the slack values are the same, the software attempts to sort by pin name. *Default*: By default, checks are sorted by slack value (`slack`). Improve usability of the report for debugging purposes by using the -verbose and -sort options as shown in these examples: `report_analysis_coverage -verbose untested -sort reason report_analysis_coverage -verbose violated -sort slack`

`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *`filename`* parameters are mutually exclusive; you cannot specify them together.
`-unsorted`	Reports unsorted slacks with untested end points at the end of the report.
`-verbose` *`check_status_list`*	Limits the detailed information to only the checks with the status specified. Valid status types are: `met`, `violated`, and `untested`. Reduce the number of lines in the `DETAILS` section and improve usability of the report by using the `-verbose` option to filter out the `met` checks as shown in this example: `report_analysis_coverage -verbose {violated untested}` **Note**: The `-verbose` parameter supports multi-threading.
`-view` *`view_name`*	Generates a timing check report for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner analysis mode.

Examples

- The following command filters from the report all endpoints that are unconstrained because of the reason `const`:

```
> report_analysis_coverage -exclude_untested const -verbose untested
-------------------------------------------------------------
TIMING CHECK COVERAGE SUMMARY
-------------------------------------------------------------
Check Type             No. of     Met      Violated    Untested
                       Checks
-------------------------------------------------------------
Clock Gating Hold      1          0 (0%)    1 (100%)   0 (0%)
Clock Gating Setup     1          1 (100%)  0 (0%)     0 (0%)
ClockPeriod            28         24 (85%)  0 (0%)     4 (14%)
ExternalDelay (Early)  1          0 (0%)    0 (0%)     1 (100%)
ExternalDelay (Late)   1          0 (0%)    0 (0%)     1 (100%)
Hold                   13         8 (61%)   1 (7%)     4 (30%)
PulseWidth             28         24 (85%)  0 (0%)     4 (14%)
Setup                  13         8 (61%)  1 (7%)      4 (30%)
-------------------------------------------------------------

TIMING CHECK COVERAGE DETAILS
-----------------------------------------------------------
Pin     Reference Pin      Check Type      Slack            Reason
-----------------------------------------------------------
out1       -               ExternalDelay(Early)  UNTESTED    No data signal
out1       -               ExternalDelay(Late)   UNTESTED    No data signal
u99/CK ^   u99/CK ^        ClockPeriod           UNTESTED    No endpoint clock
u99/CK ^   u99/CK v        PulseWidth            UNTESTED    No endpoint clock
u99/CK v   u99/CK ^        PulseWidth            UNTESTED    No endpoint clock
u99/CK v   u99/CK v        ClockPeriod           UNTESTED    No endpoint clock
u99/D      u99/CK ^        Hold                  UNTESTED    No endpoint clock
u99/D      u99/CK ^        Setup                 UNTESTED    No endpoint clock
u8/CK ^    u8/CK ^         ClockPeriod           UNTESTED    No endpoint clock
u8/CK ^    u8/CK v         PulseWidth            UNTESTED    No endpoint clock
u8/CK v    u8/CK ^         PulseWidth            UNTESTED    No endpoint clock
u8/CK v    u8/CK v         ClockPeriod           UNTESTED    No endpoint clock
seg1/u9/D  seg1/u9/CK ^    Hold                  UNTESTED    No data signal
seg1/u9/D  seg1/u9/CK ^    Setup                 UNTESTED    No data signal
seg2/u9/D  seg2/u9/CK ^    Hold                  UNTESTED    No data signal
seg2/u9/D  seg2/u9/CK ^    Setup                 UNTESTED    No data signal
-------------------------------------------------------------------------------
```

When the software is in multi-mode multi-corner timing analysis mode, You can specify the -view

parameter to generate a timing check report for the specified analysis view only:

```
> report_analysis_coverage -view view1
-----------------------------------------------------
TIMING CHECK COVERAGE SUMMARY (Analysis View: view1)
-----------------------------------------------------------
Check Type          No. of     Met       Violated      Untested
                    Checks
-----------------------------------------------------------------

Clock Gating Hold     1         0 (0%)     1 (100%)     0 (0%)
Clock Gating Setup    1         1 (100%)   0 (0%)       0 (0%)
ClockPeriod           28        24 (85%)   0 (0%)       4 (14%)
ExternalDelay (Early) 1         0 (0%)     0 (0%)       1 (100%)
ExternalDelay (Late)  1         0 (0%)     0 (0%)       1 (100%)
Hold                  14        8 (57%)    1 (7%)       5 (35%)
PulseWidth            28       24 (85%)    0 (0%)       4 (14%)
Setup                 14        8 (57%)    1 (7%)       5 (35%)
-----------------------------------------------------------------
```

Related Information

- check_timing
- report_timing

report_analysis_summary

```
report_analysis_summary
[-help]
[-csv string]
[-merged_views]
[-retime {path_slew_propagation | aocv_path_slew_propagation}]
[-retime_mode {exhaustive}]
[-view viewName]
[> <filename[.gz]>]
[>> <filename[.gz]>]
[-early | -late ]
[-expanded_summary_clocks | -merged_groups]
```

Generates timing slack summary reports for specified analysis view(s). This command categorizes the slack summary in terms of groups - In2reg, Reg2reg, Reg2Out, and In2Out. A column "ALL" is displayed that regroups in2reg, in2out, reg2reg, and reg2out details.

By default, the slack is categorized in terms of worst negative slack (WNS), total negative slack (TNS), and the total number of violations in group-based mode.

The `report_analysis_summary` command is supported in path-based analysis (PBA) mode.

Parameters

`-help`	Prints out the command usage.
{> | >>} *filename*	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. **Note:** To write a compressed report, add the .gzextension to the file name. *Default:* Report is displayed on standard output without being saved.
`-csv` *string*	Specifies the name of the CSV output file.
`-early`	Generates report for hold analysis mode for early paths.
`-expanded_summary_clocks`	Generates report in terms of each possible clock domain pair separately.
`-late`	Generates report for setup analysis mode for late paths.
`-merged_groups`	Merges all the clock groups for a consolidated timing summary.
`-merged_views`	Generates merged reports across all the views.
`-retime {path_slew_propagation | aocv_path_slew_propagation}`	Reanalyzes the defined set of paths using the specified method. By default, the timing report shows a maximum of 2 million paths for retime PBA analysis.
`-retime_mode {exhaustive}`	Reanalyzes the specified set of paths using the specified retime mode.
`-view` *viewName*	Generates report for the specified analysis views only. By default, all the active views are analyzed.

Example

- The `report_analysis_summary` command shows the following report output:

```
REPORT ANALYSIS SUMMARY (View:default_emulate_view) (EL:Setup)
============================================================

REPORT ANALYSIS SUMMARY (View:default_emulate_view) (EL:Hold)
============================================================

GroupType:ClockGate
============================================================

All               IN2REG          IN2OUT          REG2REG          REG2OUT
-------------------------------------------------------------------------------
------
WNS:TNS:#VIO    WNS:TNS:#VIO    WNS:TNS:#VIO   WNS:TNS:#VIO    WNS:TNS:#VIO
FromClk->ToClk
-------------------------------------------------------------------------------
-------
-5.614:-5.614:1  0.000:0.000:0  0.000:0.000:0  -5.614:-5.614:1 0.000:0.000:0  *->*
-------------------------------------------------------------------------------
-------

GroupType:Standard
===============================================================================
=======
All              IN2REG           IN2OUT          REG2REG          REG2OUT
-------------------------------------------------------------------------------
---
WNS:TNS:#VIO     WNS:TNS:#VIO    WNS:TNS:#VIO   WNS:TNS:#VIO    WNS:TNS:#VIO
FromClk->ToClk
-------------------------------------------------------------------------------
---
-8.897:-8.897:1  0.000:0.000:0   0.000:0.000:0  -8.897:-8.897:1 0.000:0.000:0  *-
>CLK_W_1
-------------------------------------------------------------------------------
-------
-0.571:-0.571:1  -0.571:-0.571:1 0.000:0.000:0   0.000:0.000:0  0.000:0.000:0  *-
>CLK_W_2
-------------------------------------------------------------------------------
-------
```

Related Information

- set_default_view
- set_analysis_view

report_analysis_views

```
report_analysis_views
-help
[-type {all | setup | hold | active}]
```

Generates a hierarchical report of the current multi-mode multi-corner configuration. By default, the `report_analysis_views` command generates a report of all defined views in the design. Use the `-type` parameter to generate a report on a specific type of analysis view.

Use this command after creating analysis views (`create_analysis_view`), and setting active setup and hold views (`set_analysis_view`) for the design.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_analysis_views` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_analysis_views`.
`-type {all | setup | hold | active}`	Specifies the type of report to generate. *Default:* `all` • active: Generates a report of all active setup and hold views in the design. • all: Generates a report of all defined views in the design, including those that are currently inactive. • hold: Generates a report of all active hold views in the design. • setup: Generates a report of all active setup views in the design.

Examples

Examples

- The following command generates a hierarchical report of the active setup analysis views in the design:

  ```
  report_analysis_views -type setup
  ```

  The software reports the following results:

  ```
  Multi-Mode Analysis View Report
  ___________________________________

  + SETUP Views
  |
  + Analysis View: default_emulate_view
  |
  + Delay Calc Corner: default_emulate_delay_corner
  | |
  | |
  | |
  | + rc_corner: default_emulate_rc_corner
  | | |
  | | + preRoute_res: 1
  | | |
  | | + postRoute_res: 1
  | | |
  | | + preRoute_cap: 1
  | | |
  | | + postRoute_cap: 1
  | | |
  | | + postRoute_xcap: 1
  | | |
  | | + preRoute_clkcap: 0
  | | |
  | | + postRoute_clkcap: 0 0 0
  | | |
  | | + preRoute_clkres: 0
  | | |
  | | + postRoute_clkres: 0 0 0
  | |
  | + late_library_set: default_emulate_libset_max
  | | |
  | | + timing: /icd/ssv_pe_t2b/timing/pawang/unit_test_cases/unit_2/data/cell_w.lib
  | | |
  ```

```
| | + si: /icd/ssv_pe_t2b/timing/pawang/unit_test_cases/unit_2/data/celtic-NDC.cdB
| |
| + early_library_set: default_emulate_libset_min
| | |
| | + timing: /icd/ssv_pe_t2b/timing/pawang/unit_test_cases/unit_2/data/cell_w.lib
| | |
| | + si: /icd/ssv_pe_t2b/timing/pawang/unit_test_cases/unit_2/data/celtic-NDC.cdB
| |
| + si_enabled: true
|
+ Constraint Mode: default_emulate_constraint_mode
| |
| + SDC Constraint Files: /dev/null
```

Related Information

- create_analysis_view
- set_analysis_view

report_annotated_assertions

```
report_annotated_assertions
-type {load | transition}
[-list_annotated]
[-list_non_annotated]
[-view viewName]
[-max_lines number]
[-min | -max | -min_max]
object_list
[{> | >>} file_name]
```

Reports information for annotated assertions, set using the `set_load` or `set_annotated_transition` command, according to the specified assertion type. You can select load or transition reporting.

Note: The `report_annotated_assertions` command is now obsolete. This command will continue to be supported in the current release, but will be removed in the next major release of the software.

Parameters

`{> | >>} file_name`	Writes the report to the specified file name. When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified filename. If the file already exists, the software concatenates the report to the end of the file.
`-list_annotated`	Lists the detailed information for the specified assertion types.
`-list_non_annotated`	Lists information for non-annotated assertions.

`-min	-max	-min_max`	Specifies to report minimum, maximum, or both values of the specified assertion type. When input assertions are applied using both -min and -max parameters, you can specify the -min_max parameter to report both the assertion values. In this case, you do not need to specify this command twice, that is, separately with -min and -max parameters. *Default*: max
`-max_lines` *number*	Controls the number of output lines reported by `-list_annotated` or `-list_non_annotated` parameter. By default, the specified net, pin, or port names are reported.
object_list	Specifies the list of nets, pins, or ports to be reported.
`-type {load	transition}`	Specifies the assertion type, load or transition, to be reported.
`-view` *viewName*	Reports the annotations for the specified analysis view. In multi-mode multi-corner timing analysis mode, annotations are reported for the specified mode or corner.

Examples

- The following command reports all the load annotated assertions:
  ```
  report_annotated_assertions -type load -list_annotated -min_max
  ```

Related Information

- `set_load`
- `set_annotated_transition`

report_annotated_check

```
report_annotated_check
[-check_type check_type_list]
[-max_line number]
[instance_list]
[-list_annotated]
[-list_non_annotated]
[-view viewName]
[> | >> filename[.gz] | -tcl_list]
```

Reports coverage of annotated timing checks.

Parameters

`-check_type` *check_type_list*	Allows you to specify the check type. The valid check types are `setup`, `hold`, `pulse_width`, `clock_period`, `clock_gating_setup`, `clock_gating_hold`, `clock_gating_pulse_width`, `data_setup`, `data_hold`, `recovery`, `removal`, `clock_separation`, `no_change_setup`, `no_change_hold`, and `skew`.

`> filename`	When specified with `>`, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with `>>`, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *`filename`* parameter must be the last argument in the list. The *`filename`* and `-tcl_list` parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.
`instance_list`	Specifies instance names for which annotations of check arcs are to be reported.
`-list_annotated`	Lists the detailed annotation information for the following timing checks: • setup • hold • recovery • removal • pulse-width • nochange-setup • clock-period • skew **Note**: If the software is in multi-mode multi-corner timing analysis mode, you need to specify the `-view` parameter.

`-list_non_annotated`	Lists the timing checks that do not have annotated delay information. The software checks for the following timing checks: • setup • hold • recovery • removal • pulse-width • nochange-setup • clock-period • skew **Note**: If the software is in multi-mode multi-corner timing analysis mode, you need to specify the `-view` parameter.
`-max_line` *number*	Controls the number of output lines reported by the `-list_annotated` or `list_non_annotated` parameter.
`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *`filename`* parameters are mutually exclusive; you cannot specify them together.
`-view` *viewName*	Reports coverage of annotated timing checks for the specified analysis view. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode. *Default:* Reports annotated timing checks for the default analysis view.

Examples

- The `report_annotated_check` command shows the following report output in simultaneous setup and hold analysis mode:

```
tempus> report_annotated_check

------------------------------------------------------------------

| Check arc type         | Total | Annotated | Not       |Percentage|
|                        |       |           | Annotated |Arcs      |
|                        |       |           |           |Annotated |
|------------------------+-------+-----------+-----------+----------|
| Setup arcs             | 1642  | 2         | 1640      | 0.121803 |
| Hold arcs              | 1642  | 1         | 1641      | 0.060901 |
| Recovery arcs          | 368   | 0         | 368       | 0.000000 |
| Removal arcs           | 367   | 1         | 366       | 0.272480 |
| Min pulse width arcs   | 1403  | 0         | 1403      | 0.000000 |
| Period arcs            | 6     | 0         | 6         | 0.000000 |
| Clock gating hold arcs | 2     | 0         | 2         | 0.000000 |
+------------------------+-------+-----------+-----------+----------+
| All arcs               | 5430  | 4         | 5426      | 0.073665 |
+------------------------------------------------------------------+
```

- The following command lists down all the annotated checks in the `f_slow` view:

```
tempus> report_annotated_check -list_annotated -view f_slow

-----------------------------------------------------
Analysis View: f_slow

+-------------------------------------------------------------------------------------------
---------------+
| Setup Checks |                                  |                          |
|------------------------+-----------------------+--------------------------+-----------
---------------|
| SourceSignal         | RefSignal            | Delays           | Type
 |Cond      |View   |
|--------------------------------+--------------------------------+------------------+---
----------------
| DTMF_IN/TDSP_CR/PROG| DTMF_IN/TDSP_CR/PROG| 0.500:0.500:0.500|
setup_rising_clk_rise|RN_EQ_1_AN|f_slow |
| BUS_MC_IN/dt_reg_2  | BUS_MC_IN/dt_reg_2  |                  |                          |
              |
| D ^                 | CK ^                |                  |                          |
```

```
      |       |
| DTMF_IN/TDSP_CR/PROG| DTMF_IN/TDSP_CR/PROG|
0.500:0.500:0.500|setup_falling_clk_rise|RN_EQ_1_AN|f_slow |
| BUS_MC_IN/dt_reg_2  | BUS_MC_IN/dt_reg_2  |                 |                       |
      |       |
| D v                 | CK ^                |                 |                       |
      |       |
+-------------------------------------------------------------------------------------------
---------------

+-------------------------------------------------------------------+
| Check arc type        | Total | Annotated | Not       | Percentage|
|                       |       |           | Annotated | Arcs      |
|                       |       |           |           | Annotated |
|-----------------------+-------+-----------+-----------+-----------|
| Setup arcs            | 1642  | 2         | 1640      | 0.121803  |
| Recovery arcs         | 368   | 0         | 368       | 0.000000  |
| Min pulse width arcs | 1403  | 0         | 1403      | 0.000000  |
| Period arcs           | 6     | 0         | 6         | 0.000000  |
+-----------------------+-------+-----------+-----------+-----------+
| All arcs              | 3419  | 2         | 3417      | 0.058497  |
+-------------------------------------------------------------------+
```

- The following command lists down all the annotated checks in the f_slow view:

  ```
  tempus > report_annotated_check -list_annotated -view f_slow
  ```

```
-------------------------------------------------
Analysis View: f_fast
+-------------------------------------------------------------------------------------------
---------------+
| Hold Checks |                                  |                         |
|-------------------------+-----------------------+-------------------------+-----------
---------------|
| SourceSignal        | RefSignal           | Delays           | Type
 |Cond      |View   |
|-----------------------------+-------------------------------+------------------+---
----------------
| DTMF_INST/TEST_INST | DTMF_INST/TEST_INST | 0.400:0.400:0.400| hold_rising_clk_rise |
      | f_fast|
| RAM_INST/A[5] ^     | RAM_INST/CLK ^      |                 |                       |
      |       |
| DTMF_INST/TEST_INST | DTMF_INST/TEST_INST | 0.400:0.400:0.400  hold_falling_clk_rise|
      | f_fast|
| RAM_INST/A[5] v     | RAM_INST/CLK ^      |                 |                       |
```

```
      |       |
+-------------------------------------------------------------------------------------------
--------------+

+-------------------------------------------------------------------------------------------
--------------+
| Removal Checks |                                  |                          |
|------------------------+------------------------+--------------------------+-----------
--------------|
| SourceSignal         | RefSignal            | Delays          | Type
 |Cond      |View    |
|------------------------------+------------------------------+------------------+---
--------------|

| DTMF_IN/SPI_IN/dflag| DTMF_IN/SPI_IN/dflag|0.300:0.300:0.300|removal_rising_clk_rise|
     | f_fast |
| N ^                 | K ^                  |                 |                          |
     |        |
+-------------------------------------------------------------------------------------------
--------------+
+-------------------------------------------------------------------+

| Check arc type         | Total | Annotated | Not       | Percentage|
|                        |       |           | Annotated | Arcs      |
|                        |       |           |           | Annotated |
|----------------------+-------+-----------+-----------+-------------|
| Hold arcs              | 1642  | 1         | 1641      | 0.060901  |
| Removal arcs           | 367   | 1         | 366       | 0.272480  |
| Clock gating hold arcs | 2     | 0         | 2         | 0.000000  |
+------------------------+-------+-----------+-----------+-----------+
| All arcs               | 2011  | 2         | 2009      | 0.099453 |
+-------------------------------------------------------------------+
```

Related Information

- set_annotated_check
- reset_annotated_check

report_annotated_delay

```
report_annotated_delay
[-help]
[-ignore_tied_low_high_net_arcs]
[-list_annotated]
[-list_non_annotated]
[-view viewName]
[-max_line number]
[{> | >>} filename[.gz] | -tcl_list]
[pin_port_list]
```

Reports coverage of annotations on a design. These annotations include SDF delay arc annotations. Besides SDF annotations, the command also reports annotations that are set using the `set_annotated_delay` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_annotated_delay` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_annotated_delay`.

`>	>> filename`	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *filename* parameter must be the last argument in the list. The *filename* and `-tcl_list` parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.
`-ignore_tied_low_high_net_arcs`	Ignores timing arcs of power and ground nets.
`-list_annotated`	Lists detailed annotation information for the following annotated arcs: • annotated internal net arcs • annotated input port arcs • annotated output port arcs • annotated cell arcs
`-list_non_annotated`	Lists the arc information for the non-annotated arcs corresponding to the following types of arcs: • non-annotated internal net arcs • non-annotated input port arcs • non-annotated output port arcs • non-annotated cell arcs
`-max_line` *number*	Controls the number of output lines reported by `-list_annotated` or `-list_non_annotated` parameter.

`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *`filename`* parameters are mutually exclusive; you cannot specify them together.
`-view` *`viewName`*	Generates the report for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.

Examples

If your SDF has only cell arcs, and net delays are included in the cell delays, you can report the status of annotation for cell arcs, internal net arcs, net arcs from primary inputs, and net arcs to primary outputs, and all timing arcs separately. The following report is generated for `report_annotated_delay` for delay arcs types:

```
report_annotated_delay
------------------------------------------------------------
Delay Arc Type     Total    Annotated    Not Annotated    Percentage Annotated
------------------------------------------------------------------------------
Cell arcs          128      0            128              0.000000
Input net arcs     14       0            14               0.000000
Internal net arcs  107      0            107              0.000000
Output net arcs    7        0            7                0.000000
------------------------------------------------------------------
All timing arcs    256      0            256              0.000000
------------------------------------------------------------------
```

Related Information

- `set_annotated_delay`
- `report_net`

report_annotated_parasitics

```
report_annotated_parasitics
[-help]
[-net_file string]
[-nets string]
[-view {viewName}]
[> {write_fileName}]
[>> {append_fileName}]
[{[-list_annotated]
[-list_broken_net]
[-list_extra_pin_net]
[-list_float_net]
[-list_nodriver_net]
[-list_noload_net]
[-list_not_annotated]
[-list_real_net]
[-list_supply_net]
[-list_zero_cap_net]
[-max_missing {max_missing_int}]}]
```

Reports the back-annotated parasitics of the design. By default only summary report is generated. The software groups all the nets by net-types and annotation-status (annotated or not-annotated).

Parameters

`-help`	Outputs a brief description that includes type and default information for the `report_annotated_parasitics` parameters. For a detailed description of the command and all of its parameters, use the `man` command: `man report_annotated_parasitics`.
`>> {append_fileName}`	Appends the report to the specified file name.
`-list_annotated`	Generates a list of annotated nets.
`-list_broken_net`	Reports broken nets in the design.

`-list_extra_pin_net`	Generates a list of nets that have extra pins in the SPEF but not in the Verilog.
`-list_float_net`	Generates a list of floating nets.
`-list_nodriver_net`	Lists all the terminal nets that have no driver.
`-list_noload_net`	Lists all the terminal nets that have no load.
`-list_not_annotated`	Generates a list of not annotated nets.
`-list_real_net`	Lists all real nets.
`-list_supply_net`	Lists all supply nets.
`-list_zero_cap_net`	Generates a list of zero capacitance nets.
`-max_missing {max_missing_int}`	Limits the number of missing annotations to the specified number of annotations. *Default*: 1000
`-net_file` *string*	Accepts a file name, which contains a list of net names, as input. When this parameter is specified, the parasitic annotation summary is provided only for the listed nets. The net names in the file can be specified as one net per line.
`-nets` *string*	Specifies the list of net names for which annotation summary is required.
`-view` *viewname*	Reports the back-annotated parasitics for the specified analysis view. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode. *Default*: Reports against the default analysis view
`> {write_fileName}`	Writes the report to the specified file name.

Examples

- The following command generates a list of annotated and not-annotated nets:

```
report_annotated_parasitics -list_annotated -list_not_annotated -list_real_net -
list_float_net -list_nodriver_net \ + -list_noload_net -list_supply_net -max_missing
```

```
1000

# list syntax: type: net_name [nrCap Cap nrXCap XCap nrRes Res]
# unit: pF, Ohm
# List the first 82 of total 82 real nets:
Annotated Real Net: [seg3/w7] [ 10 0.0690723 25 0.0512442 9 219.251 ]
Annotated Real Net: [seg3/w5] [ 14 0.0373353 24 0.0265601 13 161.843 ]
:
:
Annotated Real Net: [CLK3] [ 2 0.00046867 0 0 1 18.872 ]
Annotated Real Net (broken): [CLK4] [ 1 0.0002379 0 0 2 0.0002 ]
# List the first 20 of total 20 not annotated real nets:
Not Annotated Real Net: [seg3/wout]
Not Annotated Real Net: [CLK44]
:
:
Not Annotated Real Net: [out1]
Not Annotated Real Net: [in1]
# No annotated supply net.
# List the first 4 of total 4 not annotated supply nets:
Not Annotated Supply Net: [gnd]
Not Annotated Supply Net: [GND]
Not Annotated Supply Net: [VDD]
Not Annotated Supply Net: [vdd]
# No annotated floating net.
# No not annotated floating net.
# No annotated no driver net.
# ist the first 9 of total 9 not annotated no driver dangling nets:
Not Annotated Dangling Net (no driver): [seg3/n8]
Not Annotated Dangling Net (no driver): [seg3/n111111]
:
:
Not Annotated Dangling Net (no driver): [in]
Not Annotated Dangling Net (no driver): [out5]
# List the first 8 of total 8 annotated no load dangling nets:
Annotated Dangling Net (no load): [seg3/n14] [ 1 0.00520893 0 0 0 0 ]
Annotated Dangling Net (no load): [seg2/n14] [ 1 0.00636874 0 0 0 0 ]
:
:
Annotated Dangling Net (no load): [n6] [ 2 0.1086 0 0 1 0.0001 ]
Annotated Dangling Net (no load): [n5] [ 2 0.1065 0 0 1 0.0001 ]
# List the first 16 of total 16 not annotated no load dangling nets:
Not Annotated Dangling Net (no load): [seg3/u14:QN]
Not Annotated Dangling Net (no load): [seg3/u9:QN]
```

```
:
:
Not Annotated Dangling Net (no load): [u2:QN]
Not Annotated Dangling Net (no load): [in4]
# Summary of Annotated Parasitics:
+-------------------------------------------------------------------+
| Net Type             | Count  | Annotated (%) | Not Annotated (%) |
+----------------------+--------+---------------+-------------------+
| total                | 135    | 90 66.7%      | 45 33.3%          |
+----------------------+--------+---------------+-------------------+
| no drive (*)         | 9      | 0 0.0%        | 9 100.0%          |
| 1-term:no load       | 24     | 8 33.3%       | 16 66.7%          |
| real net (complete)  | 81     | 61 75.3%      | 20 24.7%          |
| real net (broken)    | 21     | 21 100.0%     | 0 0.0%            |
+-------------------------------------------------------------------+
Note for Net Types marked "(*)": Such nets are never timed, but reported here for
informational purpose.

+-------------------------------------------------+
| Annotated  | Res (KOhm) | Cap (pF) | XCap (pF) |
+------------+------------+----------+-----------+
| Count      | 982        | 1011     | 1152      |
| Value      | 20.0869    | 7.7289   | 5.4598    |
+-------------------------------------------------+
```

Related Commands

- read_sdf
- spefin

report_annotations

```
report_annotations
[-missing_resistances]
[-missing_capacitances]
[-missing_rc]
[-missing_delays]
[-missing_spf]
[-max_missing integer ]
[-tcl_list]
[-ignore_floating_nets]
[-ignore_tied_low_high_nets]
[-ignore_tied_low_high_net_arcs]
[-list_annotated]
[-list_non_annotated]
[-view viewName ]
[> filename ]
```

Reports the coverage of the annotations on a design. These annotations include SDF delay arc annotations, SPF annotations, wire resistances, and wire capacitances.

Use the `-missing_*` options to get a detailed report of all delay arcs and wires that are missing annotations. You can use the information in a Tcl program.

Note: The `report_annotations` command reports only the first 1000 nets, which is the default value of the `-max_missing` option. Set this option to a higher value if needed, for example: `report_annotations -max_missing 1000000`.

Command `report_annotations` is obsolete. Use `report_annotated_parasitics` for RC annotation report and `report_annotated_delay` for delay arc annotation instead.

Parameters

`> filename`	Writes the report to the specified file name. If the file already exists, the software overwrites it. The *filename* parameter must be the last argument in the list. The *filename* and `-tcl_list` parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.
`-ignore_floating_nets`	
	Ignores nets without drivers, power, and ground nets.
`-ignore_tied_low_high_net_arcs`	Ignores timing arcs of power and ground nets.
`-ignore_tied_low_high_nets`	Ignores nets connected to `tied_low` and `tied_high` cells.
`-list_annotated`	Lists detailed annotation information for the following annotated arcs: • annotated internal net arcs • annotated input port arcs • annotated output port arcs • annotated cell arcs

`-list_non_annotated`	Lists the arc information for the non-annotated arcs corresponding to the following types of arcs: • annotated internal net arcs • annotated input port arcs • annotated output port arcs • annotated cell arcs
`-max_missing` *integer*	Limits the number of missing annotations to the number specified. *Default*: 1000
`-missing_capacitances`	Reports the nets that are missing capacitances. *Default*: Only a summary of the coverage is reported.
`-missing_delays`	Reports the arcs that are missing SDF annotations. *Default*: Only a summary of the coverage is reported.
`-missing_rc`	Reports the nets that are missing resistances and capacitances. *Default*: Only a summary of the coverage is reported.
`-missing_resistances`	Reports the nets that are missing resistances. *Default*: Only a summary of the coverage is reported.
`-missing_spf`	Reports the nets that are missing SPF data. *Default*: Only a summary of the coverage is reported.

`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *`filename`* parameters are mutually exclusive; you cannot specify them together. See report_net for an example of the `-tcl_list` output.
`-view` *viewName*	Reports the coverage of annotations for the specified analysis view. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode. *Default:* Reports against the default analysis view.

Examples

- The following command reports the nets that are missing capacitances: report_annotations -missing_capacitances
  Default: Only a summary of the coverage is reported.
- The following commands show the difference between the summary and the `-missing_spf` reports:

report_annotations Summary Report

```
+-------------------------------------------------------------------------------+
| Total |  Total C  | Percentage |  Total R  | Percentage | Total SPF | Percentage |
|  Nets | Annotated|     C      | Annotated |     R      | Annotated |    SPF     |
|       |          | Annotated  |           | Annotated  |           | Annotated  |
|-------+----------+------------+-----------+------------+-----------+------------|
|    21 |        0 |   0.000000 |         0 |   0.000000 |        15 |   0.714286 |
+-------------------------------------------------------------------------------+
+---------------------------------+
| Total |   Total   | Percentage |
|  Arcs |   Arcs    |    Arcs    |
|       | Annotated |  Annotated |
|-------+-----------+------------|
|   272 |         0 |   0.000000 |
+---------------------------------+
```

report_annotations -missing_spf Report

	Min			Typ			Max		
Net Name	R	C	SPF	R	C	SPF	R	C	SPF
bar/n019	-	-	N	-	-	N	-	-	N
bar/n018	-	-	N	-	-	N	-	-	N
bar/n017	-	-	N	-	-	N	-	-	N

- If your SDF has only cell arcs, and net delays are included in the cell delays, you can report the status of annotation for cell arcs, internal net arcs, net arcs from primary inputs, and net arcs to primary outputs, and all timing arcs separately as shown below:

report_annotations Report for Delay Arc Types

Delay Arc Type	Total	Annotated	Not Annotated	Percentage Annotated
Cell arcs	551319	353103	198216	64.046951
Input net arcs	18	7	11	38.888889
Internal net arcs	501156	304341	196815	60.727798
Output net arcs	597	456	141	76.381912
All timing arcs	1053090	657907	395183	62.473957

Related Information

- report_net

report_aocv_derate

```
report_aocv_derate
[-help]
[-check_global_derating]
[-clock]
[-data]
-file <string>
[-list_cell_not_annotated]
[-mesh]
[-voltage <float>]
[-design | -cell <string> ]
[{-delay_corner <string> [-power_domain <string>] } | -max | -min ]
[-early | -late]
[-rise | -fall]
```

Reports the AOCV derating tables from AOCV libraries. The command generates the output report in AOCV library format.

Parameters

`-help`	Prints out command usage.
`-cell` *string*	Reports the table for the specified library cell name. You can specify a regular expression corresponding to library cell names.
`-check_global_derating`	Reports the list of cells for which derating table or global derating table was not specified.
`-clock`	Reports derate tables read for clock paths.
`-delay_corner` *string*	Reports the derating tables from the specified delay corner. For a MMMC design, if a delay corner specific table is read using the `create_library_set -aocv` command, that derating table is reported. Otherwise the default table is reported. For a non-MMMC design, no details are reported. The `-delay_corner` and `-min`/`-max` parameters are mutually exclusive.
`-data`	Reports derate tables read for data paths.
`-design`	Reports the default cell or net derating tables.
`-early` | `-late`	Reports the early/late derating table.
`-file` *string*	Reports all the derating tables. This parameter is mandatory.
`-list_cell_not_annotated`	Reports the list of cells for which derating table is not specified. This parameter reports an error when specified with `-voltage`, `-design`, or `-cell` parameters.
`-max`	Reports derate tables from max AOCV libraries.
`-mesh`	Reports derate tables read for mesh.
`-min`	Reports derate tables from min AOCV libraries.

`-power_domain` *string*	Reports derate tables from the specified power domain. This option should always be specified with delay corner.
`-rise` | `-fall`	Reports the rise/fall table for specified transition.
`-voltage` *float*	Reports derating tables for the specified voltage value.

Examples

Here are some examples.

- The following command reports derating tables for the specified cells:

```
report_aocv_derate -cell "BUF" -delay_corner default_emulate_delay_corner -file
report.lib

#content of file "report.lib" :

Cell BUF
LATE-DATA-RISE
Stage 1.000000 2.000000 3.000000 4.000000 5.000000 8.000000 9.000000 18.000000
19.000000
1.180000 1.160000 1.140000 1.120000 1.100000 1.080000 1.060000 1.040000 1.020000

Cell BUF
LATE-DATA-FALL
Stage 1.000000 2.000000 3.000000 4.000000 5.000000 8.000000 9.000000 18.000000
19.000000
1.180000 1.160000 1.140000 1.120000 1.100000 1.080000 1.060000 1.040000 1.020000

Cell BUF
LATE-CLOCK-RISE
Stage 1.000000 2.000000 3.000000 4.000000 5.000000 8.000000 9.000000 18.000000
19.000000
1.180000 1.160000 1.140000 1.120000 1.100000 1.080000 1.060000 1.040000 1.020000

Cell BUF
LATE-CLOCK-FALL
Stage 1.000000 2.000000 3.000000 4.000000 5.000000 8.000000 9.000000 18.000000
19.000000
1.180000 1.160000 1.140000 1.120000 1.100000 1.080000 1.060000 1.040000 1.020000
```

- The following command reports derating tables for the cells:

```
report_aocv_derate -clock -file aocv_clock_derate.rpt

Cell AN2D1
EARLY-CLOCK-RISE
Stage 1.000000 2.000000 3.000000 4.000000 5.000000 6.000000 7.000000 8.000000
9.000000 10.000000
0.70000 0.850000 0.880000 0.890000 0.900000 0.910000 0.920000 0.9200000 0.930000
0.935000

Cell AN2D1
EARLY-CLOCK-FALL
Stage 1.000000 2.000000 3.000000 4.000000 5.000000 6.000000 7.000000 8.000000
9.000000 10.000000
0.760090 0.830010 0.860060 0.880300 0.890200 0.901000 0.902000 0.910000 0.920000
0.9202594
Cell BUFF
EARLY-CLOCK-RISE
Stage 1.000000 2.000000 3.000000 4.000000 5.000000 6.000000 7.000000 8.000000
9.000000 10.000000
0.842000 0.888258 0.908854 0.921954 0.929953 0.935137 0.940946 0.944305 0.942031
0.959500

Cell BUFF
EARLY-CLOCK-FALL
Stage 1.000000 2.000000 3.000000 4.000000 5.000000 6.000000 7.000000 8.000000
9.000000 10.000000
0.739532 0.810305 0.840208 0.861346 0.880605 0.899462 0.903028 0.900408 0.912154
0.916708
* The following command reports derating tables for the cells in delay corner in mesh
report_aocv_derate -mesh -delay_corner dc_125c_rcworst -file
aocv_clock_delay_corner_dc_125c_rcworst.rpt
Cell AN2D1
EARLY-MESH-RISE

Cell AN2D1
EARLY-MESH-FALL
Stage 1.000000 2.000000 3.000000 4.000000 5.000000 6.000000 7.000000 8.000000
9.000000 10.000000
0.739532 0.810305 0.840208 0.861346 0.880605 0.899462 0.903028 0.900408 0.912154
0.916708

Cell AN2D1
LATE-MESH-RISE
Stage 1.000000 2.000000 3.000000 4.000000 5.000000 6.000000 7.000000 8.000000
9.000000 10.000000
```

```
0.842000 0.888258 0.908854 0.921954 0.929953 0.935137 0.940946 0.944305 0.942031
0.959500

Cell AN2D1
LATE-MESH-FALL
Stage 1.000000 2.000000 3.000000 4.000000 5.000000 6.000000 7.000000 8.000000
9.000000 10.000000
0.842000 0.888258 0.908854 0.921954 0.929953 0.935137 0.940946 0.944305 0.942031
0.959500
```

report_case_analysis

```
report_case_analysis
[-all]
[-nosplit]
[-dir]
[-view view_name]
[-propagated]
[-verbose]
[pins_and_ports_list]
[{> | >>} filename | -tcl_list]
```

Generates a User Case Analysis table, which reports ports and pin that have a set_case_analysis constraint. These ports and pins are constant only for timing analysis. You can optionally generate a Logic Constant Value table, which reports ports and pins whose constant status is part of the design. Information from this table includes ports or pins with `set_logic` constraints (`set_logic_zero` and `set_logic_one`), netlist explicit constants, and output pins with constant output functions.

Parameters

`{> | >>} filename`	When specified with `>`, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with `>>`, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *`filename`* parameter must be the last argument in the list. The *`filename`* and `-tcl_list` parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.

`-all`	Specifies that the Logic Constant Value table report pins or ports with `set_logic` constants, constants in the netlist, and output pins with a constant function. These output pins are from library cells or netlist modules.
`-nosplit`	Specifies that reports with long names will not be split into multiple lines. **Note:** The `-nosplit` parameter works if `set_table_style -no_frame_fix_width` command is specified.
`-dir`	Specifies that the report include the direction of the pin or port, which is reported as input, output, or inout (internal pins may not have constant constraints).
`pins_and_ports_list`	Generates a report only for the pins and ports in the specified list. *Default:* Generates a report for all pins and ports in the design.
`-propagated`	Generates a report that includes all constant pins, including pins with propagated constants. When specified, the report column header for constant values changes to `Constant value`. *Default:* Generates a report that includes only pins with a `set_case_analysis` constraint. (The column header for the constant values is labeled as `User case analysis value`.)
`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *`filename`* parameters are mutually exclusive; you cannot specify them together.

`-verbose`	Generates a report in which each pin is listed on a separate line with a cause. If the pin is a propagated constant, a table is included that reports a chain of constants from an original cause to the reported pin. *Default:* All pins are reported in a table.
`-view` *view_name*	Generates a User Case Analysis table for the specified analysis view only. To specify this parameter, the software must be in multi-mode multi-corner (MMMC) analysis mode. *Default:* When in MMMC analysis mode, generates a table for all active analysis views

Examples

- The following command uses the `-dir` option, which shows that 1 is on the input part of the `timing_const_on_Z/Z` pin:

  ```
  report_case_analysis -dir
  ```

  The following report is generated:

  ```
  ------------------------------------------------
  Pin name        Dir               User case
                                   Analysis value
  ------------------------------------------------
  gscanmode       input             0
  gscanenable     input             0
  ------------------------------------------------
  ```

- The following command generates a report that includes all constant pins, including pins with propagated constants. When specified, the report column header for constant values changes to `Constant value`.

  ```
  report_case_analysis -propagated
  ```

  The following report is generated:

  ```
  ----------------------------------------
  Pin name                  Constant value
  ```

```
-----------------------------------------
clockandlatch/U8/TE         0
clockandlatch_dup1/U8/TE    1
clockandlatch_dup2/U8/TE    1
clockandlatch_dup3/U8/TE    0
-----------------------------------------
```

- The following command generates a report in which each pin is listed on a separate line. The report includes all constant pins, including pins with propagated constants. The pin `clockandlatch_dup3/U8/TE` is reported because of a `set_case_analysis 0` assertion on pin `gscanenable`.

  ```
  report_case_analysis -propagated -verbose clockandlatch_dup3/U8/TE
  ```

  The following report is generated:

  ```
  -----------------------------------------------
  Pin name                       Constant value
  -----------------------------------------------
  gscanenable                   0
  FE_OFC306_gscanenable/I       0
  FE_OFC306_gscanenable/ZN      1
  FE_OFC307_gscanenable/I       1
  FE_OFC307_gscanenable/ZN      0
  FE_OFC315_gscanenable/I       0
  FE_OFC315_gscanenable/Z       0
  coreclockandlatch_dup3/U8/TE  0
  -----------------------------------------------
  ```

- The following report shows that there are no `set_case_analysis` constraints but there are logical constraints:

  ```
  report_case_analysis -propagated -verbose core_q_reg_1_/SI
  ```

  ```
  -------------------------------------------
  Pin name              Logic constant value
  -------------------------------------------
  ram256x10/TEST1       0
  ram1024x32/TEST1      0
  core_q_reg_2_/SI      1
  core_q_reg_11_/SI     0
  core_q_reg_1_/SI      1
  -------------------------------------------
  ```

Related Commands

- set_case_analysis

report_cell_instance_timing

```
report_cell_instance_timing
[-help]
list_of_cell_instances
[-early | -late]
[-clock_timing]
[-tcl_list]
[-view view_name]
[{> | >>} filename]
```

Reports the following information about the timing context for all the pin and delay arcs of the specified instances:

- Arrival time
- Required time
- Slack
- Delays

Note: Internal pins (pins inside the instance boundary) are also reported.

This command uses the results of earlier timing analysis runs; therefore, you must run the `report_timing` command before using `report_cell_instance_timing`.

To analyze the setup (maximum) paths, you must use the `setAnalysisMode -checkType setup` command and run the `report_timing` command before using the `-late` parameter. Similarly, to analyze the hold (minimum) paths, you must use the `setAnalysisMode -checkType hold` command and run the `report_timing` command before using the `-early` parameter.

The `report_cell_instance_timing` timing report contains the following tables:

- The first table in the report contains information about the pins of the instance.
  The propagated slew is the slew that is propagated to the pin, based on the slew propagation mode.
- The second table contains information about the arcs through the instance.
  The slew out is the slew calculated at the arc output from the arc input slew, using the timing model.
- The third table, generated for sequential devices, contains information about the timing checks, if present, on the cell.
  Only checks that are from the library are shown in this table. Automatic checks, such as clock gating setup and hold, are not reported.

The *Phase* column of each table can contain the following symbols:

- `D` indicates it is a data signal in the clock domain.
- `C` indicates it is a clock signal in the clock domain.
- `P` indicates it is a positive phase.
- `N` indicates it is a negative phase.
- `L` indicates it is a latency phase.
- Asterisk (`*`) indicates there is a timing exception associated with the signal.

  Note: Currently, it is not possible to expand the asterisk to determine which exception is associated with the reported timing phase.

When the software is in multi-mode multi-corner timing analysis mode, the timing context information is reported for each active analysis view, with the view name listed in the *Phase* column.

Parameters

`-help`	Prints out the command usage.
`{> | >>} filename`	When specified with `>`, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with `>>`, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *`filename`* parameter must be the last argument in the list. The *`filename`* and `-tcl_list` parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.
`-clock_timing`	Specifies to report slack for clock style checks.

`-early	-late`	Reports the information for early (maximum) path (data hold/clock setup) or late (minimum) path (data setup/clock hold). You must use the `setAnalysisMode -checkType hold` command and run the `report_timing` command before using the `-early` parameter. Similarly, you must use the `setAnalysisMode-checkType setup` command and run the `report_timing` command before using the `-late` parameter. *Default*: If you do not specify any of the parameters, the software uses the `-late` parameter.
list_of_cell_instances	Specifies a flat (non-hierarchical) cell instance or list of cell instances. Use curly braces (`{}`) to enclose the list and separate the instance names with white space.
`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *`filename`* parameters are mutually exclusive; you cannot specify them together.
`-view` *view_name*	Specifies the analysis view in which to query the object attribute.

Examples

- The following example reports information about the timing context of instance `seg1/u9`:

  Example: report_cell_instance_timing seg1/u9

```
------------------------------------------------------------------------
                  Instance seg3/u14 of DFF
------------------------------------------------------------------------
Pin   Dir   Propagated  Arrival  Required  Slack  Phase
            Slew
------------------------------------------------------------------------
```

```
CK ^ IN    0.062        0.621                      CLK_W_3(C)(P) *
CK v IN    0.065        1.503                      CLK_W_3(C)(P) *
CK ^ IN    0.062        0.621                      CLK_W_3(D)(P) *
CK v IN    0.065                                   CLK_W_3(D)(P) *
CK ^ IN    0.062                                   CLK_W_3(D)(N) *
CK v IN    0.065        1.503                      CLK_W_3(D)(N) *
D ^  IN    1.077        2.062   2.434     0.372    CLK_W_3(D)(P)
D v  IN    0.003        1.815   2.465     0.650    CLK_W_3(D)(P)
Q ^  OUT   0.044        0.788                      CLK_W_3(D)(P)
Q v  OUT   0.040        0.748                      CLK_W_3(D)(P)
QN ^ OUT   0.058        0.816                      CLK_W_3(D)(P)
QN v OUT   0.041        0.857                      CLK_W_3(D)(P)
-------------------------------------------------------------------
-------------------------------------------------------------------
               Instance seg3/u14 of DFF
-------------------------------------------------------------------
Arc         Slew
-------------------------------------------------------------------
From  To   In     Out    Load   Delay  Phase
-------------------------------------------------------------------
CK ^  QN ^ 0.062  0.058  0.000  0.196  CLK_W_3(C)(P) *
CK ^  QN v 0.062  0.041  0.000  0.236  CLK_W_3(C)(P) *
CK ^  QN ^ 0.062  0.058  0.000  0.196  CLK_W_3(D)(P) *
CK ^  QN v 0.062  0.041  0.000  0.236  CLK_W_3(D)(P) *
CK ^  QN ^ 0.062         0.000         CLK_W_3(D)(N) *
CK ^  QN v 0.062         0.000         CLK_W_3(D)(N) *
CK ^  Q ^  0.062  0.044  0.000  0.167  CLK_W_3(C)(P) *
CK ^  Q v  0.062  0.040  0.000  0.128  CLK_W_3(C)(P) *
CK ^  Q ^  0.062  0.044  0.000  0.167  CLK_W_3(D)(P) *
CK ^  Q v  0.062  0.040  0.000  0.128  CLK_W_3(D)(P) *
CK ^  Q ^  0.062         0.000         CLK_W_3(D)(N) *
CK ^  Q v  0.062         0.000         CLK_W_3(D)(N) *
-------------------------------------------------------------------
-------------------------------------------------------------------
                          Instance seg3/u14 of DFF
-------------------------------------------------------------------
            Pins                                Phase
-------------------------------------------------------------------
Check  Sig  Ref   Slack   Delay   Adjust  Sig            Ref
-------------------------------------------------------------------
HOLD   D ^  CK ^  1.570   -0.129  0.000   CLK_W_3(D)(P)  CLK_W_3(C)(P) *
HOLD   D v  CK ^  1.222   -0.027  0.000   CLK_W_3(D)(P)  CLK_W_3(C)(P) *
SETUP  D ^  CK ^  0.372   0.186   -2.000  CLK_W_3(D)(P)  CLK_W_3(C)(P) *
SETUP  D v  CK ^  0.650   0.155   -2.000  CLK_W_3(D)(P)  CLK_W_3(C)(P) *
```

```
-------------------------------------------------------------------------
```

- If a constant is asserted or propagated to the instance pin, it is reported in the *Phase* column of the pin table, as shown in the following example:

Example: report_cell_instance_timing p5042A

```
-------------------------------------------------------------------
Instance p5042A of MUX2HVTD1
-------------------------------------------------------------------------
Pin   Dir  Propagated   Arrival  Required  Slack    Phase
Slew
-------------------------------------------------------------------
I0 ^ IN    0.032        0.127    8.583    8.455     EJ_TCK(D)(P)
I0 v IN    0.021        0.127    8.568    8.441     EJ_TCK(D)(P)
I1 ^ IN    0.060        0.295                       EJ_TCK(D)(P)
I1 v IN    0.035        0.260                       EJ_TCK(D)(P)
S    IN                                            Constant 0
Z ^  OUT   0.020        0.168    8.624    8.455     EJ_TCK(D)(P)
Z v  OUT   0.021        0.170    8.612    8.441     EJ_TCK(D)(P)
-------------------------------------------------------------------------

-------------------------------------------------------------------
Instance p5042A of MUX2HVTD1
-------------------------------------------------------------------
Arc         Slew
-------------------------------------------------------------------------
From  To    In      Out    Load    Delay    Phase
-------------------------------------------------------------------------
I0 ^  Z ^   0.032   0.020  0.002   0.041    EJ_TCK(D)(P)
I0 v  Z v   0.021   0.021  0.002   0.044    EJ_TCK(D)(P)
I1 ^  Z ^   0.060   0.002                   EJ_TCK(D)(P)
I1 v  Z v   0.035   0.002                   EJ_TCK(D)(P)
-------------------------------------------------------------------------
```

- When the software is in multi-mode multi-corner analysis mode, the timing context information is reported for each active analysis view. The view name is reported in the *Phase* column, as shown in the following example:

```
report_cell_instance_timing FE_OFC304_EJ_TDO

-------------------------------------------------------------------------------------------
---------
                          Instance FE_OFC304_EJ_TDO of BUFFHVTD8
-------------------------------------------------------------------------------------------
---------
```

```
Pin     Dir     Propagated     Arrival         Required     Slack        Phase
                  Slew
-------------------------------------------------------------------------------
---------
I ^      IN      0.052         8.812           16.260    7.448     EJ_TCK(D)(N)
(setup_view1)*
I v      IN      0.033         8.799           16.319    7.520     EJ_TCK(D)(N)
(setup_view1)*
Z ^      OUT     0.338         8.919           16.367    7.448     EJ_TCK(D)(N)
(setup_view1)*
Z v      OUT     0.167         8.852           16.372    7.520     EJ_TCK(D)(N)
(setup_view1)*
-------------------------------------------------------------------------------
---------
```

Related Information

- report_net
- report_timing

report_clock_gating_check

```
report_clock_gating_check
[object_list]
[-view view_name]
[{> | >>} file_name]
```

Reports information of all or specific clock gating checks of the current design. If you specify a list of pins, cells, or clocks, the clock gating check information is reported for the specified objects only. If you use the `report_clock_gating_check` command without an object list, all clock gating check information is reported.

The `Type` column of the generated report (see examples below) can contain one of the following abbreviations:

- `I` indicates that the clock gating check is inferred.
- `L` indicates that the clock gating check is from the library (integrated clock gating check).

Set the `report_precision` global variable using the `set_global` command to specify the accuracy of the result in terms of number of significant digits.

Parameters

object_list	Specifies a list of pins, instances, or clocks. A collection of these can also be specified. • Pin: Any pin of a clock gating cell can be specified and the check information will be reported for that pin. • Instance: Any instance name of a clock gating cell can be specified and the check information will be reported for that instance. • Clock: Only a clock at a given clock (reference) pin of a cell can be specified for reporting the check information. Check information about the clocks at enable (signal) pin cannot be reported. If the *object_list* parameter is not specified, all the clock gating check points (inferred and library) of the current design are reported.
-view *view_name*	Specifies a view for which clock gating check is to be reported. The default is worst of all the views.
{> | >>} *filename*	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file.

Examples

The following command returns a clock gating check report for the current design:

```
report_clock_gating_check

The clock gating check report returned by this command is as follows:

+--------------------------------------------------------------------------+
```

```
|                              Clock Gating Check Report                  |
|-------------------------------------------------------------------------|
| Instance | Enable | Clock | Type | Level | Setup | Setup | Hold | Hold |
|          |        |       |      |       | Rise  | Fall  | Rise | Fall |
|----------+--------+-------+------+-------+-------+-------+------+----|
|    i1    |    A   |   B   |   I  |   L   | 0.000 | 0.000 |   -  |  -   |
|    i2    |    B   |   A   |   I  |   H   | 0.000 | 0.000 |   -  |  -   |
|   icg    |  CEN   |  CLK  |   L  |   H   | 0.149 | 0.181 |   -  |  -   |
+-------------------------------------------------------------------------+
```

Related Information

- report_precision
- set_global

report_clock_propagation

```
report_clock_propagation
[-help]
-clock clock_list
[-max_paths number of paths]
-to pin_port_list
[-verbose]
[-view viewName]
[> filename[.gz]]
[>> filename[.gz]]
```

This command provides the ability to determine clocks that do not propagate to a particular node in the network, and report the causes for such occurrences.

Parameters

`-help`	Prints out the command usage
`-clock clock_list`	List or collection of clocks that are to be checked.
`-max_paths number of paths`	Reports number of maximum clock paths per clock.
`-to pin_port_list`	Specifies the pin/port list.
`-verbose`	Generates a verbose report for clock propagation stop reason.
`-view viewName`	Generates a report for the specified analysis view.
`> filename[.gz] | >> filename[.gz]`	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. **Note**: To write a compressed report, add the .gz extension to the file name.

Examples

- The following command shows that clock propagation stops due to an inactive timing arc from b12/A to b12/Y

```
set_disable_timing -from A -to Y b12
report_clock_propagation -to u99/CK -clock CLK_W_4

Clock: CLK_W_4
Point: u99/CK
Reason 1: The clock 'CLK_W_4' propagation stops due to the inactive timing arc:
(b12/A->b12/Y)

The partial clock path:

-------------------------------------------------
Source    Sink      ArcType         ArcSense
-------------------------------------------------
CLK4     b11/A   (net)           (net)
b11/A    b11/Y   combinational   positive_unate
b11/Y    b12/A   (net)           (net)
b12/A    b12/Y   combinational   positive_unate
-------------------------------------------------
```

- The following command shows that clock propagation stops due to no clock source found:

```
report_clock_propagation -to launchff/Q -clock [get_clocks gen_clk]
report_clock_propagation -to seg3/u15/Q -clock CLK_W_5_gen


Clock: CLK_W_5_gen
Point: seg3/u15/Q
Reason 1: The generated clock ‘CLK_W_5_gen could not be created on its root pin
‘seg3/u15/Q’ because no clock source found for it
```

- The following command shows that clock propagation stops at the register clock pin as all the arcs from the clock pin of the register have been disabled:

```
set_disable_timing -from A -to Y seg3/u17_a
report_clock_propagation -to seg3/u14/CK -clock CLK_W_3

Clock: CLK_W_3
Point: seg3/u14/CK
Reason 1: The clock 'CLK_W_3' propagation stops due to the inactive timing arc:
(seg3/u17_a/A->seg3/u17_a/Y)
```

```
The partial clock path:
-------------------------------------------------------------
Source          Sink          ArcType         ArcSense
-------------------------------------------------------------
CLK3           seg3/u16/A     (net)           (net)
seg3/u16/A     seg3/u16/Y     combinational   positive_unate
seg3/u16/Y     seg3/u17/A     (net)           (net)
seg3/u17/A     seg3/u17/Y     combinational   positive_unate
seg3/u17/Y     seg3/u17_a/A   (net)           (net)
seg3/u17_a/A   seg3/u17_a/Y   combinational   positive_unate
-------------------------------------------------------------
```

Related Information

- set_disable_timing

report_clock_timing

```
report_clock_timing
[-absolute_compare]
[-cppr_relative]
[-latency_greater_than]
-type [{skew interclock_skew jitter summary latency cppr_stage_count}

      [ [-launch | -capture]
        [-rise | -fall]
        | [-histogram [-histogram_range interval size]]
        [-logic_level [-source {clock_root | generated_clock}]]
        ]
      ]
[-early | -late]
[-clock clock_list]
[-from_clock from_clock_list]
[-to_clock to_clock_list]
[-from from_list]
[-to to_list]
[-nworst worst_entries]
[-greater_than lower_limit]
[-view view_name]
[-verbose]
[-format column_list]
[-tcl_list]
[{> | >>} filename]
```

Generates a clock analysis report for the current design. The software generates a separate report for each specified clock or pair of clocks. You specify the clocks using the `-clock` parameter, and clock pairs using the `-from_clock` and `-to_clock` parameters. You can use the `-nworst` and `-greater_than` parameters with the `-from_clock` and `-to_clock` parameters to generate reports with skew to the specified sink pins. You can use the `-early` and `-late` parameters to specify the type of skew to be reported.

Note: To remove common path pessimism from the reported skew, use the `setAnalysisMode -cppr both` command before using the `report_clock_timing` command. The report contains a separate column for common path adjustment values.

You can use the `report_clock_timing` command to perform MMMC based SSTA analysis. In MMMC mode, the command reports the latency and clock skew for all individual views. You can use the `report_clock_timing -view` parameter to report clock timing for a user-specified view.

Parameters

{> | >>} *filename*	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *filename* parameter must be the last argument in the list. The *filename* and -tcl_list parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the .gz extension to the file name. *Default:* Report is displayed on standard output without being saved.
-absolute_compare	Reports skews with values compared in absolute form.
-clock *clock_list*	Specifies the list of clocks to be reported.
-cppr_relative	Reports the latency in skew reports from the CPPR common pin. When specified, the reports are printed with latency measured from branch point to the register's CK pin. The CPPR value is no longer required in reports.
-early	Uses hold skew to generate the report. Hold skew is the difference between minimum latency at the start point flip-flop and the maximum latency at the end point flip-flop. Default: Uses the setup skew.

`-format` *column_list*	Specifies which columns to display for path reports, and the order in which they should appear. For example: `-format {instance arc pin cell slew load}` The `-format` parameter can be used only for latency or skew reports (`-type skew`, `-type interclock_skew`, or `-type latency`), and if the `-verbose` parameter is specified.
`-from` *from_list*	Specifies the list of sequential device clock pins or ports to consider as from-pins in the generated report. If any pin in the *from_list* is not a sequential device clock pin, it is replaced with all the sequential device clock pins in the transitive fanout of the named pin. If there are no sequential pins in the transitive fanout of the pin, the software drops the pin from the list and generates a warning message.
`-from_clock` *from_clock_list*	Specifies the list of from-clocks to be used in generating the current interclock skew report. You can use this option only in an interclock skew report. *Default:* Report is generated considering all clocks as from-clocks.
`-greater_than` *lower_limit*	Reports skews with values greater than the lower_limit value.

`-histogram`	Generates a textual histogram of the latency report. The `-histogram` parameter can be used only when `-type latency` is specified. When specified in conjunction with the `-clock` parameter, the software limits the latency reporting to the specified clocks only. When specified with the `-greater_than` parameter, the software limits the latency reporting to clocks with a latency value that is greater than the specified value. **Note:** The `-histogram` parameter cannot be specified in conjunction with the following parameters: `-launch`,`-capture`, `-from`, `-to`, `-from_clock`, `-to_clock`, `-verbose`, `-nworst`.
`-histogram_range` *`interval_size`*	Generates the histogram report at the specified intervals. The interval size specified must be a positive integer or decimal values. For example, if the maximum latency value is 437, and you specify `-histogram_range 50`, the histogram would be generated with intervals of: 0-50, 50-100, 100-150, ... 400-450. **Note:** The unit of *`interval_size`* will be the same as that used for generating the timing report. This parameter can be used only when the `-histogram` parameter is specified.
`-late`	Uses setup skew to generate the report. Setup skew is the difference between the maximum latency at the start point flip-flop and minimum latency at the end point flip-flop.
`-latency_greater_than <`*`lower_limit`*`>`	Reports are filtered based on the specified -type option. When specified with: • `-type skew` or `interclock_skew`: Reports are filtered for latency greater than the provided value or depth. • `-type cppr_stage_count`: Reports are filtered for depth greater than the provided value.

`-launch	-capture`	When specified with `-type latency`, generates a report of either the launch latency values (`-launch`), or the capture latency values (`-capture`). **Note:** You can specify these parameters only with `-type latency`. *Default*: `-launch`
`-logic_level`	Reports the levels (the stage count of gates) between the clock root, or generated clock root, and the capturing register. The `-logic_level` parameter can be used only when `-type latency` is specified. The `-logic_level` parameter cannot be specified in conjunction with the following parameters: `-launch`,`-capture`, `-from`, `-to`, `-from_clock`, `-to_clock`, `-verbose`, `-nworst`.
`-nworst` *worst_entries*	Specifies the number of entries to be reported per clock domain. *Default:* `1`
`-rise	-fall`	Specifies active transition at sequential devices clock pins. The `-rise` and `-fall` parameters are mutually exclusive and cannot be specified together. **Note:** You can specify these parameters only with the `-type latency` parameter.

`-source {clock_root | generated_clock}`	Reports the latency from either the master clock, or the generated clock that was derived from it. The `-source` parameter can be used only when `-logic_level` is specified. The `-source` parameter cannot be specified in conjunction with the following parameters: `-launch`,`-capture`, `-from`, `-to`, `-from_clock`, `-to_clock`, `-verbose`, `-nworst`. *Default:* Reports the latency from the master clock (`-source clock_root`)
`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *`filename`* parameters are mutually exclusive; you cannot specify them together.
`-to` *`to_list`*	Specifies the list or collection of sequential device clock pins or ports to be considered for reporting. If any pin in the *`to_list`* is not a sequential device clock pin, it is replaced with all the sequential device clock pins in the transitive fanout of the named pin. If there are no sequential pins in the transitive fanout of the pin, the software drops the pin from the list and generates a warning message.
`-to_clock` *`to_clock_list`*	Specifies the list of to-clocks to be used in generating the current interclock skew report. You can use this option only in an interclock skew report. *Default:* Report is generated considering all clocks as to-clocks.
`-type {skew interclock_skew jitter summary latency cppr_stage_count}`	Specifies the type of report to generate. The `-type` options are described below: • `skew`: Reports the skew for all clocks, or for clocks that you specify using the -clock

parameter.

- `interclock_skew`: Reports the skew between all clocks in the design, or between clocks that you specify using the `-from_clock` and `-to_clock` parameters.
- `jitter`: Reports the difference in late and early arrival times of a clock phase at a clock end point. When specified, the software generates a clock jitter report for one clock end point, for each clock in the design.
  You can generate a jitter report for all clocks, or for clocks specified with the `-clock` parameter. You can limit the report to a specific clock end point using the `-to` parameter.
  Note: The `report_clock_timing -type jitter` option can be used only when the software is in On-Chip Variation (OCV) timing analysis mode. Jitter analysis does not remove any common path pessimism.
- `summary`: Generates a summary report for each analyzed clock that includes the following information:
  - When setup skew is used (`-late`):
    - maximum launch latency
    - minimum capture latency
    - maximum setup skew
  - When hold skew is used (`-early`):
    - minimum launch latency
    - maximum capture latency
    - maximum hold skew
- `latency`: Reports the network and source latency values for all the clock pins in the

	design, or for clock pins that you specify using the `-clock` parameter.
	• `cppr_stage_count`: The generated report is arranged in terms of difference of stages from common pin to CK pin. The reports are generated on the basis of logical CPPR points derived between each pair of phases reaching the launch/capture CK pins. Only same clock pairs (similar to type skew) are picked. For example,
	`Branch Point Difference Depth Clock Pin`
	`----------------------------------------`
	`-`
	`inv2/Y`
	`5 r ff2/CK`

`-verbose`	Generates a report that contains the full clock path from the clock source to the sink pin (the clock pin of the flop or latch). You can specify the `-verbose` parameter only for skew or latency reports (`-type skew`, `-type interclock_skew`, or `-type latency`). When this parameter is specified in SOCV mode, the intermediate arrival times of pins are shown as "-" in the report output.
`-view` *view_name*	Generates a report for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.

Examples

- The following command reports worst case skew for all clocks in a design:

```
report_clock_timing -type skew -nworst 4

  Clock: CLK_W_4
  Skew    Latency   Clock Pin
  ---------------------------
```

```
        0.150 r   u99/CK
0.000   0.150 r   u55/CK

Clock: CLK_W_3
Skew   Latency   Clock Pin
-----------------------------
        0.682 r seg3/u9/CK
0.061   0.621 r seg3/u14/CK

Clock: CLK_W_2
Skew    Latency Clock Pin
-----------------------------
        0.682 r seg2/u9/CK
0.061   0.621 r seg2/u14/CK

Clock: CLK_W_1
Skew    Latency Clock Pin
------------------------------
        0.682 r seg1/u9/CK
0.061   0.621 r seg1/u14/CK
```

Here 'r' indicates the transition (r - rise, f - fall) at the reported register clock pin.

- The following command reports interclock skew with -from_clock, -to_clock, -from, and -to parameters defined:

```
report_clock_timing -type interclock_skew -nworst 10 -from CLK4 -to u5/CK -from_clock
CLK_W_4 -to_clock CLK_W_4_gen2

  Clocks: CLK_W_4 -> CLK_W_4_gen2
  Skew       Latency Clock Pin
  -----------------------------
                     CLK4
             0.180 r u99/CK
                     u2/Q
                     u8/Q
  -0.205    0.386 ri u5/CK
```

Here 'r' indicates the transition (r - rise, f - fall) at the reported register clock pin. The 'i' refers to the 'inverted' polarity of the clock signal at the reported register clock pin.

- The following command reports skew for all clocks in the design including common path pessimism removal:

```
set_analysis_mode -cppr both
report_clock_timing -type skew -nworst 10

  Clock: CLK_W_4
```

```
Skew    Latency    CPPR    Clock Pin
-----------------------------------------
        0.180              r u99/CK
0.000   0.120       -0.060 r u55/CK

Clock: CLK_W_3
Skew     Latency  CPPR    Clock Pin
-----------------------------------------
         0.818            r seg3/u9/CK
0.139    0.496 -0.183     r seg3/u14/CK

Clock: CLK_W_2
Skew    Latency  CPPR     Clock Pin
-----------------------------------------
        0.818             r seg2/u9/CK
0.139   0.496    -0.183   r seg2/u14/CK

Clock: CLK_W_1
Skew     Latency   CPPR     Clock Pin
-----------------------------------------
         0.818              r seg1/u9/CK
0.139    0.496    -0.183    r seg1/u14/CK
```

- The following command generates a summary report for the clock:

```
report_clock_timing -type summary -nworst 10 -clock CLK_W_1

Clock: CLK_W_1
                  Skew    Latency    Clock Pin
-----------------------------------------------
Maximum launch latency:   0.818 r    seg1/u9/CK
Minimum capture latency:  0.000 r    u2/CK
Maximum skew:             0.818 r    seg1/u9/CK
                  0.322   0.496 r    seg1/u14/CK
-----------------------------------------------
```

- The following command reports the Network and source latency values for the clock CLK_W_1:

```
report_clock_timing -type latency -nworst 10 -clock CLK_W_1

Clock: CLK_W_1
---Latency---
Source  Network  Total   Clock Pin
-----------------------------------------
0.000   0.818    0.818   r seg1/u9/CK
0.000   0.000    0.000   r u2/CK
-----------------------------------------
```

- The following command reports the Network and source latency values for the clock GCLK1, as well as the full clock paths:

```
report_clock_timing -type latency -verbose -clock CLK_W_1
```

```
  Clock: CLK_W_1
  ---Latency---
  Source Network Total Clock Pin
  ----------------------------------------
  0.000 0.818 0.818 r seg1/u9/CK

  Clock latency Path
  Clock Rise Edge 0.000
  = Beginpoint Arrival Time 0.000
  Other End Path:
  ----------------------------------------
  Instance       Arc      Cell     Delay   Clock
  Latency
  ------------------------------------------------
  -            CLK1 ^       -      -       0.000
  seg1/u16     A ^ -> Y ^  CLKBUF   0.111  0.111
  seg1/u17     A ^ -> Y ^  CLKBUF   0.126  0.236
  seg1/u17_a   A ^ -> Y ^  CLKBUF   0.107  0.343
  seg1/and1    B ^ -> Y ^  AND      0.071  0.818
  seg1/u9      CK ^        DFF      0.000  0.818
  ------------------------------------------------
```

- When the software is in multi-mode multi-corner timing analysis mode, the software generates a report for every active analysis view in the design:

```
report_clock_timing -type latency
```

```
  Clock: EJ_TCK
  Analysis View: hold_view1
  ---Latency---
  Source Network Total Clock Pin
  ----------------------------------------
  0.000 0.000 0.000 r cregister_q_reg_3_/CP

  Clock: EJ_TCK
  Analysis View: hold_view2
  ---Latency---
  Source Network Total Clock Pin
  ----------------------------------------
  0.000 0.000 0.000 r cregister_q_reg_3_/CP
```

```
  Clock: EJ_TCK
  Analysis View: setup_view1
  ---Latency---
  Source Network Total Clock Pin
  ----------------------------------------
  0.000 0.000 0.000 r cregister_q_reg_3_/CP
```

- Specify the `-view` parameter to generate a report only for a specific analysis view:

```
report_clock_timing -view setup_view1 -type latency
```

```
  Clock: EJ_TCK
  Analysis View: setup_view1
  ---Latency---
  Source    Network    Total    Clock       Pin
  ----------------------------------------
  0.000     0.000      0.000    r    cregister_q_reg_3_/CP

  Clock: SI_ClkIn
  Analysis View: setup_view1
  ---Latency---
  Source   Network   Total    Clock        Pin
  ----------------------------------------
  0.000    0.000     0.000     r     cregister_q_reg_13_/CP
```

- The following command generates a textual histogram of the latency report:

```
report_clock_timing -type latency -histogram
```

```
[Delay Chart: CLK4(CLK_W_4_gen2)]
************************************
Min Delay : 0.778
Max Delay : 0.778
Count : 1
************************************
----------------------------------------
Delay    Range
Min      Max      Count   1
----------------------------------------
0.000  100.000 1          |o
```

- The following command generates the histogram report at intervals of `0.5`:

```
report_clock_timing -type latency -histogram -histogram_range 0.5
```

```
[Delay Chart: CLK4(CLK_W_4_gen2)]
*************************************
Min Delay : 0.778
Max Delay : 0.778
Count : 1
*************************************
-------------------------------------
Delay Range
Min    Max    Count   1
-------------------------------------
0.500  1.000  1       |o
0.000  0.500  0
```

- The following command generates a latency report that includes the stage gate counts between the clock root or generated clock root, and the capturing register:

```
report_clock_timing -type latency -logic_level
```

```
************************************************
Start point:CLK2(CLK_W_4_gen1(CLK_W_3))
-----------------------------------------------
Delay   Count   End Point
1.018   8       seg2/u9/CK r
1.018   8       seg2/u3/CK r
0.945   7       seg2/u14/CK r

************************************************
Start point:CLK3(CLK_W_4_gen2(CLK_W_4))
-----------------------------------------------
Delay   Count  End Point
1.345   8      seg3/u6/CK r
1.345   8      seg3/u1/CK r
1.218   7      seg3/u12/CK r
```

- The following command generates a latency report starting from the generated clock, and includes the stage gate counts between the generated clock and the capturing register:

```
report_clock_timing -type latency -logic_level -source generated_clock
```

```
************************************************
Start point:CLK2(CLK_W_4_gen1(CLK_W_3))
-----------------------------------------------
Delay   Count   End Point
0.818   8       seg2/u9/CK r
0.818   8       seg2/u3/CK r
0.745   7       seg2/u14/CK r
```

```
  ************************************************
  Start point:CLK3(CLK_W_4_gen2(CLK_W_4))
  -----------------------------------------
  Delay   Count   End Point
  0.845   8       seg3/u6/CK r
  0.845   8       seg3/u1/CK r
  0.718   7       seg3/u12/CK r
```

- The following command generates a latency report starting from the generated clock CLK_W_4_gen2, and includes the stage gate counts between the generated clock and the capturing register:

```
report_clock_timing -type latency -logic_level -source generated_clock -clock
CLK_W_4_gen2
```

```
  ************************************************

  Start point:CLK3(CLK_W_4_gen2(CLK_W_4))
  -----------------------------------------
  Delay   Count    End Point
  0.845   8        seg3/u6/CK r
  0.845   8        seg3/u1/CK r
  0.718   7        seg3/u12/CK r
```

- The following command uses the -format parameter to customize the generated latency report:

```
report_clock_timing -type latency -verbose -format {instance arc pin cell slew load
user_derate}
```

Clock: CLK_W_4

```
---Latency---
Source     Network      Total      Clock Pin
-------------------------------------------------------------------------------
0.000      0.152         0.152      r u55/CK

Clock latency Path
Clock Rise Edge 0.000
= Beginpoint Arrival Time 0.000
Other End Path:
-----------------------------------------------------------------
Instance    Arc           Pin     Cell   Slew    Load     User
                                                        Derate
-----------------------------------------------------------------
-           CLK4 ^        CLK4    -      0.003   0.025    -
b11         A ^ -> Y ^    Y       BUF    0.131   0.011    1.000
```

```
b12          A ^ -> Y ^     Y        BUF     0.117   0.015    1.000
u55          CK ^           CK       DFF     0.117   0.015    1.000
-------------------------------------------------------------
```

- The following command reports the worst case jitter for all clocks in the design:

```
report_clock_timing -type jitter -nworst 2

****************************************
 Report: clock timing
 -type jitter
 -setup
 -nworst 2
 CRPR analysis: off
****************************************
 Clock: CLK1
 Jitter   Latency(Late)   Latency(Early)   Clock Pin
-----------------------------------------------------
 0.186    0.350 r         0.164 r          i_17/CK
 0.118    0.227 r         0.109 r          i_15/CK

 Clock: CLK0

 Jitter Latency(Late)  Latency(Early)    Clock Pin
-----------------------------------------------------------
 0.387  0.508     r    0.120      r      i_5/CK
 0.178  0.336     r    0.159      r      i_7/CK
```

- The following command reports the worst case jitter for the clock CLK1:

```
report_clock_timing -type jitter -clock CLK1 -nworst 2

****************************************
 Report: clock timing
 -type jitter
 -setup
 -nworst 2
 CRPR analysis: off
****************************************
```

```
 Clock: CLK1

 Jitter   Latency(Late) Latency(Early) Clock Pin
-----------------------------------------------------
 0.186    0.350    r    0.164    r     i_17/CK
 0.118    0.227    r    0.109    r     i_15/CK
```

- The following command reports the worst case jitter for the specified clock end point:

```
report_clock_timing -type jitter -to i_15/CK

****************************************
 Report: clock timing
 -type jitter
 -setup
 -nworst 2
 CRPR analysis: off
****************************************
 Clock: CLK1

 Jitter   Latency(Late)  Latency(Early)   Clock Pin
-------------------------------------------------------------
 0.118    0.227      r   0.109      r     i_15/CK

---Latency---
Source     Network      Total      Clock Pin
-------------------------------------------------------------------------------
0.000      0.152         0.152      r u55/CK

Clock latency Path
Clock Rise Edge 0.000
= Beginpoint Arrival Time 0.000
Other End Path:
-----------------------------------------------------------------
Instance    Arc           Pin     Cell   Slew    Load     User
                                                        Derate
-----------------------------------------------------------------
-           CLK4 ^        CLK4    -      0.003   0.025    -
b11         A ^ -> Y ^    Y       BUF    0.131   0.011    1.000
b12         A ^ -> Y ^    Y       BUF    0.117   0.015    1.000
u55         CK ^          CK      DFF    0.117   0.015    1.000
-----------------------------------------------------------------
```

Related Information

- setAnalysisMode
- report_timing

report_clocks

```
report_clocks
[-description]
[-arrival_points]
[-phase_shift_table]
[-total_shift_table]
[-uncertainty_table]
[-adjustment_table]
[-delay_adjustment_table]
[-source_insertion]
[-insertion]
[-hierarchy]
[-groups]
[-clocks clk_signame | clk_signame_list]
[-view view_name]
[{> | >>} filename | -tcl_list]
```

Reports information on clocks created with the `create_clock` and `create_generated_clock` command. The report includes information about all clock waveforms, clock arrival points, and clock uncertainties.

Use the `-description`, `-arrival_points`, `-uncertainty_table`, and the `*_table` parameters to specify different parts of the report. If one of these options is supplied, only the section that the option corresponds to is reported. If other options are supplied, those sections are also reported.

Based on the current setup or hold analysis mode, the following parameters report only one of the late or early tables:

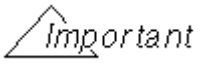
- `-phase_shift_table`
- `-total_shift_table`
- `-uncertainty_table`
- `-adjustment_table`
- `-delay_adjustment_table`

However, in simultaneous (setup and hold) mode both the tables are reported.

The `report_clocks` command only reports constraints with respect to clock waveforms. Use the `report_ports -type` command for constraints applied to clock pins.

Parameters

{> | >>} *filename*	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *filename* parameter must be the last argument in the list. The *filename* and -tcl_list parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the .gz extension to the file name. *Default:* Report is displayed on standard output without being saved.
-adjustment_table	Reports two tables that detail the hold and setup cycle adjustments between clock waveforms as set by the set_multicycle_path command. **Note:** If specific ideal (or generated) clocks are requested with the -clocks *clock_name_list* option, then only the relationships between those clock signals are reported.
-arrival_points	Reports only the clock arrival points. A clock arrival point is the pin where the create_clock or create_generated_clock command has been asserted. **Note:** This parameter is obsolete and will be removed in a future release.

`-clocks` *`clk_signame`* | *`clk_signame_list`*	Requests information on a specific ideal (or generated) clock. If a list is given, only information about those clocks is reported. You can use wildcards in the names. *Default*: All clocks, except generated clocks, are reported. Use the `-generated` option to report generated clocks.  This parameter is obsolete and will be removed in a future release. Cadence recommends that you specify clock names without the `-clocks`parameter.
`-delay_adjustment_table`	Reports two tables that detail the early and late path delay adjustments between clock waveforms as set by the `set_max_delay` and `set_min_delay` commands. **Note:** If specific ideal (or generated) clocks are requested with the `-clocks` *`clock_name_list`* option, then only the relationships between those clock signals are reported.
`-description`	Reports only the description of the specified clock waveform(s).
`-generated`	Includes information on generated clocks in the report, such as source and master clock of the generated clock, and other information specified in the `create_generated_clock` constraints. The report uses the internal clock names created by the system, if you did not specify new clock names while creating these clocks. ⓘ This parameter is obsolete and will be removed in a future release. The software now reports generated clocks by default.
`-groups`	Reports the existing clock groups.

`-hierarchy`	Reports the inter-clock relationship in tree style format.
`-insertion`	Reports the network insertion delays specified on clock waveforms. The values in the table are in `min:typ:max` format.
`-phase_shift_table`	Reports the table that details the phase shift between clock waveforms. Based upon the current analysis mode, the table shows the phase shifts either in late or early mode. In case of simultaneous setup hold mode, both the tables are reported for phase shift in late and early mode. **Note:** If specific ideal (or generated) clocks are requested with the `-clocks` *`clock_name_list`* option, then only the relationships between those clock signals are reported.
`-source_insertion`	Reports the source insertion delays specified on clock waveforms.
`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *`filename`* parameters are mutually exclusive; you cannot specify them together.
`-total_shift_table`	Based on the current analysis mode, reports one of the tables (late or early mode) that details the total relationship between clock waveforms. In case of simultaneous setup hold mode both the tables are reported. **Note:** If specific ideal (or generated) clocks are requested with the `-clocks` *`clock_name_list`* option, then only the relationships between those clock signals are reported.

`-uncertainty_table`	Based on the current analysis mode, reports one of the tables (late mode or early mode) that detail the uncertainty between clock waveforms as specified using the `set_clock_uncertainty` command. In case of simultaneous setup hold mode both the tables are reported. The tables report only those clock waveforms for which uncertainty has been defined using the `set_clock_uncertainty` command. **Note:** If specific ideal (or generated) clocks are requested with the `-clocks` *`clock_name_list`* option, then only the relationships between those clock signals are reported.
`-view` *`view_name`*	Generates a clock report for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.

Examples

Here are some examples.

- `report_clocks`

```
------------------------------------------------------------
Clock Descriptions
------------------------------------------------------------
Attributes
------------------------------------------------------------------
Clock Name  Source  Period  Lead   Trail  Generated Propagated
------------------------------------------------------------------
CLK1        CLK1    20.000  0.000  10.000  n        y
CLK2        CLK1    10.000  0.000   5.000  n        y
------------------------------------------------------------
Generated-Clock Descriptions
```

```
--------------------------------------------------------------------------------
---------
Name        Generated   Master      Master-clock Invert Freq.      Duty-Cycle Edges
Edge-Shift
            Source(pin) Source(pin)                 Multiplier
--------------------------------------------------------------------------------
---------
GCLK        u8/Q        CK1         CLK1         n      1/2         -          -
 -
--------------------------------------------------------------------------------
--------
```

- The following command displays only the uncertainty information:

```
set_clock_uncertainty 5 CLK_W_1
set_clock_uncertainty 4 -from CLK_W_1 -to CLK_W_4_gen1
set_clock_uncertainty 6 -from CLK_W_2 -to CLK_W_4_gen2
report_clocks -uncertainty_table

-------------------------------------------------------
Clock To Clock Uncertainty (Late)
-------------------------------------------------------
From    To           L->L    L->T    T->L     T->T
-------------------------------------------------------
*       CLK_W_1      5.000  5.000   5.000     5.000
CLK_W_1 CLK_W_4_gen1 4.000  4.000   4.000     4.000
CLK_W_2 CLK_W_4_gen2 6.000  6.000   6.000     6.000
-------------------------------------------------------
Clock To Clock Uncertainty (Early)
-------------------------------------------------------------
From     To            L->L   L->T   T->L   T->T
-------------------------------------------------------------
*        CLK_W_1      -5.000 -5.000 -5.000 -5.000
CLK_W_1  CLK_W_4_gen1 -4.000 -4.000 -4.000 -4.000
CLK_W_2  CLK_W_4_gen2 -6.000 -6.000 -6.000 -6.000
```

```
-------------------------------------------------------
```

- Consider the following clock definition:
  create_clock -name "CLK_W_1" -add -period 60 -waveform {0 30}
  report_clocks CLK_W_1 -phase_shift_table

```
-----------------------------------------------------
Clock To Clock Phase Shift (Late)
-----------------------------------------------------
From    To      L->L    L->T  T->L   T->T
-----------------------------------------------------
CLK_W_1 CLK_W_1 60.000  0.000 60.000 60.000
-----------------------------------------------------
-----------------------------------------------------
Clock To Clock Phase Shift (Early)
-----------------------------------------------------
From    To      L->L   L->T   T->L  T->T
-----------------------------------------------------
CLK_W_1 CLK_W_1 0.000 -60.000 0.000 0.000
-----------------------------------------------------
```

- You can use the following command to add uncertainty:
  `set_clock_uncertainty 2 CLK_W_1`

  The phase shift and total shift table values will be different. The following command displays the changed shift values as given below:
  `report_clocks -total_shift_table CLK_W_1`

```
-----------------------------------------------------
Total Clock To Clock Phase Shift Required (Late)
-----------------------------------------------------
From    To      L->L    L->T  T->L   T->T
-----------------------------------------------------
CLK_W_1 CLK_W_1 58.000 -2.000 58.000 58.000
-----------------------------------------------------
-----------------------------------------------------
Total Clock To Clock Phase Shift Required (Early)
-----------------------------------------------------
From    To      L->L   L->T   T->L  T->T
-----------------------------------------------------
CLK_W_1 CLK_W_1 2.000 -58.000 2.000 2.000
-----------------------------------------------------
```

You can see that the uncertainty is subtracted from the phase shift.
Consider the following example path:

```
Path 1: MET Setup Check with Pin u2/CK
Endpoint: u2/D (v) checked with leading edge of 'CLK_W_1'
Beginpoint: CLK2 (v) triggered by trailing edge of 'CLK_W_2'
Path Groups: {CLK_W_1}
Other End Arrival Time 0.000
- Setup 0.188
+ Phase Shift 15.000
- Uncertainty 2.000
= Required Time 12.812
- Arrival Time 12.463
= Slack Time 0.349
Clock Fall Edge 12.500
= Beginpoint Arrival Time 12.500
-------------------------------------
Load  Slew    Delay   Arrival   Cell  Arc          Pin
                      Time
-----------------------------------------------------
0.034  0.003  12.500                  CLK2 v       CLK2
0.003  0.003  -0.037  12.463   BUF    A v -> Y v   u10999/Y
0.003  0.003  0.000   12.463   DFF    D v          u2/D
-----------------------------------------------------
```

Here the begin point CLK_W_2 to end point CLK_W_1 shows a phase shift value of 15.00. This means that if a path starts from the falling edge of CLK_W_2 , and ends at the rising edge of CLK_W_1, the phase shift is 15.00. In other words, when the timing (setup time) is reported, the required time at the end point begins with 15.00 (in addition to adjustments like setup time, time borrowing, uncertainty, etc). However, the arrival time of this path starts at 12.500.

- When the software is in multi-mode multi-corner timing analysis mode, the software reports the clock information for each active analysis view:

```
report_clocks -total_shift_table -description

------------------------------------------------------------
Clock Descriptions
------------------------------------------------------------
Attributes
-------------------------------------------------------------------------
Clock Name Source  View       Period  Lead   Trail Generated Propagated
-------------------------------------------------------------------------
EJ_TCK     EJ_TCK  hold_view2 17.400  0.000  8.700  n         n
```

```
EJ_TCK     EJ_TCK  hold_view1 17.400  0.000  8.700  n         n
EJ_TCK     EJ_TCK  setup_view117.400  0.000  8.700  n         n
------------------------------------------------------------------------
-------------------------------------------------------------
Total Clock To Clock Phase Shift Required (Late)
-------------------------------------------------------------
From      To        View        L->L   L->T   T->L    T->T
-------------------------------------------------------------
EJ_TCK    EJ_TCK    hold_view2  17.400  0.000 17.400  17.400
EJ_TCK    EJ_TCK    hold_view1  17.400  0.000 17.400  17.400
EJ_TCK    EJ_TCK    setup_view2 17.400  0.000 17.400  17.400
------------------------------------------------------------------------
```

- Specify the `-view` parameter to generate clock information for a specific analysis view only:

```
report_clocks -total_shift_table -description -view setup_view1 > clk_sv1

-------------------------------------------------------------
Clock Descriptions
-------------------------------------------------------------
Attributes
-------------------------------------------------------------
Clock Name      Source  View         Period  Lead  Trail  Generated Propagated
-------------------------------------------------------------------------------
EJ_TCK          EJ_TCK  setup_view1  17.400  0.000 8.700  n         n
virt_in_EJ_TCK  -       setup_view1  17.400  0.000 8.700  n         n
-------------------------------------------------------------------------------
-----------------------------------------------------------------
Total Clock To Clock Phase Shift Required (Late)
-----------------------------------------------------------------
From            To      View        L->L     L->T   T->L     T->T
-----------------------------------------------------------------
EJ_TCK           EJ_TCK setup_view1 17.400   0.000  17.400  17.400
virt_in_EJ_TCK   EJ_TCK setup_view1 17.400   0.000  17.400  17.400
-----------------------------------------------------------------
```

- The `report_clocks` command output displays a column showing active or inactive clocks in the design:

```
-------------------------------------------------------------
Clock Descriptions
-------------------------------------------------------------
Attributes
-------------------------------------------------------------
Clock Name   Source   Period    Lead    Trail   Generated   Propagated   Active
-------------------------------------------------------------------------------
```

```
CLK_W_1       CLK1     20.000    0.000   10.000   n          y            y
CLK_W_2       CLK2     25.000    0.000   12.500   n          y            y
CLK_W_4_gen2  u8/Q     20.000    0.000   10.000   y          y            y
-------------------------------------------------------------------------------
```

Related Commands

- create_clock
- create_generated_clock
- set_clock_uncertainty

report_constraint

```
report_constraint
[-all_violators]
[-verbose]
[-late]
[-early]
[-check_type { pulse_width | clock_period | recovery | removal | clock_gating_setup |
clock_gating_hold | skew | pulse_clock_max_width |
pulse_clock_min_width | electromigration | rail_swing}]
[-clock]
[-connection_class]
[-data]
[-drv_fields {pin pin_type sequential direction ref_cell partition max_frequency_clock
max_clock_frequency max_transition min_transition max_capacitance min_capacitance
max_fanout min_fanout transition_actual capacitance_actual fanout_actual transition_si
transition_slack capacitance_slack fanout_slack max_slack min_slack user_constraint
lib_constraint drv_source driver_pin driver_ref_cell driver_transition
driver_max_transition driver_min_transition driver_max_capacitance driver_min_capacitance
net_fanout driver_max_fanout receiver_pin receiver_ref_cell receiver_transition
receiver_max_transition net net_type total_capacitance net_resistance view edge
transition_nsigma}]
[-drv_output_format {txt|csv}]
[-drv_violation_type { max_capacitance | max_transition | max_fanout | min_capacitance |
min_transition | min_fanout | pulse_clock_max_transition | pulse_clock_min_transition}]
[-max_pins_per_drv <integer>]
[-retime {aocv | path_slew_propagation | aocv_path_slew_propagation}]
[-retime_mode {path | exhaustive}]
[-view viewName ]
[-worst_drv_per_net]
[pin_port_list]
[{> | >>} filename[.gz]]
```

Reports constraint information of current design. The constraints information includes information such as whether or not constraint is violated, by how much amount, and the worst violator object.

The following timing checks are reported when you use the `report_constraint` command:

- All setup and hold checks
- DRV
- Clock and asynchronous checks including pulse_width, min_period, recovery, removal, clock_gating_setup, clock_gating_hold, skew

The `report_constraint` command does not honor the `report_timing_format` global variable by default. To honor this global you need to set the `timing_report_constraint_use_report_timing_format` global variable to `true`.

The `report_constraint` command supports DRV constraints (`set_min_fanout`, `set_max_fanout`, `set_min_transition`, `set_max_transition`, `set_min_capacitance`, and `set_max_capacitance`) applied on library pins.

The `report_constraint` command supports multi-threaded reporting. You can make the following settings to enable multi-threaded reporting:

```
setMultiCpuUsage -localCpu number_of_CPU
set_table_style -no_frame_fix_width
```

By default, the `report_constraint` command reports only the mean slews. You can enable DRV checking to use mean as well as the n-sigma slew values (in SOCV mode), with the following setting:

```
set timing_report_max_transition_check_using_nsigma_slew true
```

Parameters

`{> | >>}` *`filename`*	When specified with `>`, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with `>>`, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *`filename`* parameter must be the last argument in the list. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.

`-all_violators`	Reports negative slack values for timing checks. The supported check types are: • setup • hold • clock_gating_setup • clock_gating_hold • recovery • removal • clock_period • pulse_width • max_capacitance • max_transition • max_fanout • min_capacitance • min_transition • min_fanout • skew • pulse_clock_max_width • pulse_clock_min_width • pulse_clock_max_transition • pulse_clock_min_transition You can use the `-all_violators` parameter for reporting specific check types to generate report for that particular check type only. For example, you can use `-all_violators` with `-late` to report setup checks and path delays from set_max_delay.
`-check_type`	Reports setup or hold checks. The supported check types are: • clock_gating_setup: Reports only the paths that end at the clock_gating_setup timing check. • clock_gating_hold: Reports only the paths that end at the

clock_gating_hold timing check.

- `electromigration`: Reports signal EM analysis related violations. When high density current passes through a thin metal wire, the high energy electrons exert forces on atoms causing them to migrate. Electromigration (EM) can drastically reduce the lifetime of a chip by increasing the resistivity of a metal wire or creating a short circuit between adjacent lines. When this option is specified, the software reports standard cell level electromigration violations.
- recovery: Reports paths that end at the recovery timing check.
- removal: Reports paths that end at the removal timing check.
  Note: The check types - recovery, removal, clock_gating_setup, and clock_gating_hold are also supported in AOCV mode.
- pulse_width: Reports only the paths that end at the pulse_width timing check.
  If you do not specify the `-all_violators` parameter, then the `pulse_width` option reports only one entry of the most critical pulse_width check. The DRV checks also report only the most critical entry.

  To report all the violated pulse_width checks, you need to specify `report_constraint -check_type pulse_width -all_violators` command (which is consistent with other check types of the `report_constraint` command).

  To report both violating and met entries, you can use the `report_min_pulse_width` command.
- clock_period: Reports only the paths that end at the clock_period timing check.
- skew: Reports skew checks.
  The `report_constraint` command performs skew checks by default. Skew checks are implemented as setup checks that are reported in the late analysis or simultaneous mode. These checks are not reported in the early mode. Skew checks are performed assuming the signal and reference clocks have the same frequency. If the frequencies of the signal and reference clocks are different, a warning message is displayed.
- pulse_clock_max_width: Reports the maximum pulse clock width checks

	set using the set using `set_pulse_clock_max_width` command. • pulse_clock_min_width: Reports the minimum pulse clock width checks. • rail_swing: Reports details related to the rail swing checks. For rail swing check settings, you can use the following global variables: ○ `timing_rail_swing_checks_low_voltage_threshold` ○ `timing_rail_swing_checks_high_voltage_threshold`
`-clock`	Specifies that the DRV checks will be restricted to the clock network. If any pin is part of both the clock and data network, it will be reported as a clock.
`-connection_class`	Reports violations if the connection class attributes of the driving pin and receiving pin of a net are different.
`-data`	Specifies that the DRV check will be restricted to the data network. If any pin is part of both the clock and data network, it will be reported as a clock network pin. By default, the DRV checks are applied to both clock and data networks.
`-drv_fields {field list}`	Allows only the specified fields to be included in the DRV report - in the same order as defined.
`-drv_output_format {txt | csv}`	Specifies that DRV output format – txt or CSV. The default value is “txt”.

`-drv_violation_type`	Reports DRV violations. The supported DRV check types are: • max_capacitance: Reports pins having the worst `max_capacitance` constraint. If you specify the `-all_violators` parameter, you can see all the pins whose actual capacitance is greater than the `max_capacitance` value specified in library and timing constraints file. • min_capacitance: Reports pins having the worst `min_capacitance` constraint. If you specify the `-all_violators` parameter, you can see all the pins whose actual capacitance is less than the `min_capacitance` value specified in the library and timing constraints file. • max_transition: Reports pins having the worst `max_transition` constraint. If you specify the `-all_violators` parameter, you can see all the pins whose actual transition is greater than the `max_transition` value specified in the library and timing constraint file. **Note:** These checks are not reported in hold analysis mode. The `set_si_mode -enable_drv_with_delta_slew true` setting is required for max_transition DRV violations with SI slew. • min_transition: Reports pins having the worst `min_transition` constraint. If you specify the `-all_violators` parameter, you can see all the pins whose actual transition is less than the `min_transition` value specified in the library and timing constraints file. • max_fanout: Reports pins having the worst `max_fanout` constraint. If you specify the `-all_violators` parameter, you can see all the pins whose actual fanout is greater than the `max_fanout` value specified in the library and timing constraints file. • min_fanout: Reports pins having the worst `min_fanout` constraint. If you specify the `-all_violators` parameter, you can see all the pins whose actual fanout is less than the `min_fanout` value specified in the library and timing constraints file. • pulse_clock_max_transition: Reports maximum pulse clock transition checks. • pulse_clock_min_transition: Reports minimum pulse clock transition checks.

`-early`	Reports all of the hold checks, including regular, clock_gating_hold, and data-to-data hold checks. Also reports path delays from set_min_delay. **Note:** This parameter does not report removal checks.
`-late`	Reports all of the setup checks, including regular, clock_gating_setup, and data-to-data setup checks. Also reports path delays from set_max_delay. **Note:** This parameter does not report recovery checks.
`-max_pins_per_drv`	Reports the specified number of top worst violations per DRV category, such as, max_transition, max_capacitance. If this parameter is not specified, then by default 1000 entries per DRV category will be reported.
pin_port_list	Controls the output of DRV constraints. When specified, all DRV violations are reported for the specified pins or ports only.
`-retime {aocv | path_slew_propagation | aocv_path_slew_propagation}`	
	Reanalyzes the specified set of paths using the specified method. **Note**: The parameter supports setup/hold check types only. `aocv`: Re-evaluates the given set of graph-based analysis (GBA) paths by reverting back graph-based AOCV derate factors and applying path- based AOCV derates instead of less pessimistic derates. In this case the slew (and respective delays) values are worst that are propagated during graph-based analysis. `aocv_path_slew_propagation`: Re-evaluates the given set of graph-based analysis (GBA) paths by using path-based AOCV derates and delays using actual path- based propagated slew. `path_slew_propagation`: Re-evaluates the given set of graph-based analysis (GBA) paths by recalculating the delay values based upon actual propagated slew across the path. In this case the AOCV derates are the ones that are reported during graph-based analysis.

`-retime_mode {path | exhaustive}`	Allows the `report_constraint` command to be run in exhaustive path-based analysis (PBA) mode. In this mode, the software will exhaustively examine nworst paths of the selected endpoints and then select the true worst path to be reported - based on retimed slacks of all the examined paths. To enable exhaustive mode, the use model is given below: `report_constraint -all_violators -retime path_slew_propagation -retime_mode exhaustive` When you use this parameter, reporting can be run time intensive as the software needs to do an exhaustive search for true worst paths. Therefore, it is recommended to use this option in the later phases of sign-off stage when there are very few violations left for analysis. *Default*: `path`
`-verbose`	Generates a report that displays the full data path with accompanying required time and slack calculation.
`-view viewName`	Generates a report containing endpoint and other constraint information for the specified analysis view. You can only specify this parameter when the software is in multi-mode multi-corner analysis mode. *Default:* Generates a report containing the worst endpoint and constraint information based on all of the analysis views.
`-worst_drv_per_net`	Reports only the worst DRV violation per net.

Examples

- Consider a design having 3 sequential endpoints:
  - setup_pin: Having a setup check with slack equal to -9.000
  - recovery_pin: Having a recovery check with slack equal to -8.000
  - both_setup_recovery_pin: Having both setup check (slack = -6.000) and recovery check (slack = -7.000)
    In this case, the report_constraint -late -all_violators command will display the

following output:

```
max_delay/setup
-------------------------------------------------------
End Point                  Slack          Cause
-------------------------------------------------------
setup_pin f                -9.000         VIOLATED
both_setup_recovery_pin    -6.000         VIOLATED
-------------------------------------------------------
```

- The `report_constraint -check_type recovery -all_violators` command will display the following output:

```
Check type : recovery
-------------------------------------------------------------
End Point                     Slack          Cause
-------------------------------------------------------------
recovery_pin r                -8.000         VIOLATED
both_setup_recovery_pin       -7.000         VIOLATED
-------------------------------------------------------------
```

- The `report_constraint -late -check_type recovery -all_violators` command displays the following output:

```
max_delay/setup
-------------------------------------------------------------------------------
End Point                   Slack           Cause
-------------------------------------------------------------------------------
setup_pin f                 -9.000          VIOLATED
recovery_pin r              -8.000          VIOLATED
both_setup_recovery_pin     -7.000          VIOLATED
-------------------------------------------------------------
```

- The `report_constraint` command displays the following output:

```
max_delay/setup
-------------------------------------------------------------
End Point             Slack                 Cause
-------------------------------------------------------------
out2 r              -4111.834               VIOLATED
```

```
------------------------------------------------------------
Check type : clock_period
---------------------------
No paths found
Check type : pulse_width
------------------------------------------------------------------
                          SEQUENTIAL CLOCK PULSE WIDTH
------------------------------------------------------------------
Pin             Required         Actual Pulse         Slack
                Pulse Width      Width
------------------------------------------------------------------
dff2/CK (low)   0.260            -670.800            -671.060
------------------------------------------------------------------
Check type : max_transition
---------------------------
Pin : sub1/dff1/D
max_transition : 4.500
Transition Time: 5114.060
--------------------------------------------------------------------------------
slack: -5109.560 (VIOLATED)
Check type : max_capacitance
---------------------------
Pin : buf133/Y
max_capacitance: 0.000
-Capacitance : 0.423
----------------------------
slack : -0.423 (VIOLATED)
Check type : max_fanout
---------------------------
Pin : specialClkGate/o
max_fanout : 0.557
fanout : 1.000
---------------------------
slack : -0.443 ( VIOLATED )
```

- The `report_constraint -all_violators` command displays the following output:

```
max_delay/setup

-----------------------------------------------------------
End Point                Slack               Cause
-----------------------------------------------------------
out2 r                   -4111.834           VIOLATED
dff3/D r                 -4110.260           VIOLATED
lat2/D r                 -3774.174           VIOLATED
dff2/D r                 -3770.285           VIOLATED
out r                    -3161.115           VIOLATED
and1/B r                 -2464.880           VIOLATED
dff1/D r                 -2098.868           VIOLATED
sub2/bufH1/dff1/D r      -2098.821           VIOLATED
sub2/bufH2/dff1/D r      -2028.843           VIOLATED
sub1/dff1/D r            -2028.843           VIOLATED
dff5/D r                 -1279.155           VIOLATED
-----------------------------------------------------------
Check type : clock_period
---------------------------
No paths found
Check type : skew
---------------------------
No paths found
Check type : pulse_width
-------------------------------------------------------------------
                            SEQUENTIAL CLOCK PULSE WIDTH
-------------------------------------------------------------------
Pin                        Required      Actual Pulse     Slack
                           Pulse Width   Width
-------------------------------------------------------------------
dff2/CK (low)               0.260       -670.800        -671.060
lat2/GN (low)               0.201       -670.800        -671.001
sub2/bufH1/dff1/CK (high)   0.167       -670.800        -670.968
dff1/CK (high)              0.167       -670.800        -670.968
```

```
sub2/bufH2/dff1/CK (high)   0.167       -670.800        -670.968
sub1/dff1/CK (high)         0.167       -670.800        -670.968
dff11/CK (high)             0.167       -670.800        -670.968
-------------------------------------------------------------------
Check type : max_transition
-------------------------------------------------------------------
Pin Name                Required       Actual         Slack
-------------------------------------------------------------------
and_parallel_1/A        4.500          5114.060      -5109.560
buf4/A                  4.500          5114.060      -5109.560
sub1/dff1/D             4.500          5114.060      -5109.560
Check type : max_capacitance
-----------------------------------------------------------------
Pin Name                 Required        Actual      Slack
-----------------------------------------------------------------
buf133/Y                 0.000          0.423       -0.423
buf11/Y                  0.000          0.105       -0.105
buf5/Y                   0.000          0.105       -0.105
dff11/Q                  0.000          0.105       -0.105
Check type : max_fanout
-----------------------------------------------------------------
Pin Name                 Required        Actual      Slack
-----------------------------------------------------------------
specialClkGate/o         0.557           1.000       -0.443
```

- The `report_constraint -all_violators -drv_violation_type min_capacitance` command reports the following:

```
-----------------------------------------------
Pin Name    Required     Actual       Slack
-----------------------------------------------
DIN        0.0400        0.0013       -0.0387
F2/Q       0.0400        0.0013       -0.0387
CLK1       0.0400        0.0051       -0.0349
```

- The `report_constraint -drv_violation_type min_capacitance` command outputs the following:

```
Check type : min_capacitance
--------------------------
Pin : F2/Q
Capacitance: 0.0013
- min_capacitance: 0.0400
---------------------------------------------------
slack: -0.0387 ( VIOLATED )
```

- The `report_constraint -all_violators -drv_violation_type min_transition` command outputs the following:

```
-----------------------------------------------
Pin Name    Required      Actual      Slack
-----------------------------------------------
I0/A       0.0300        0.0001       -0.0299
I1/A       0.0300        0.0001       -0.0299
CLK0       0.0300        0.0001       -0.0299
CLK1       0.0300        0.0001       -0.0299
DIN        0.0300        0.0001       -0.0299
I4/S0      0.0300        0.0001       -0.0299
F1/DATA    0.0300        0.0001       -0.0299
SEL        0.0300        0.0001       -0.0299
```

- The `report_constraint -drv_violation_type min_transition` command outputs the following:

```
Check type : min_transition
--------------------------
Pin : F1/DATA


Transition Time:0.0001
- min_transition :0.0300
---------------------------------------------------
slack : -0.0299 ( VIOLATED )
```

- The `report_constraint -all_violators -drv_violation_type min_fanout` command outputs the following:

```
---------------------------------------------------
Pin Name    Required       Actual        Slack
```

```
-----------------------------------------------------
F5/Q       2.0000         0.0000         -2.0000
CLK0       2.0000         1.0000         -1.0000
F3/Q       2.0000         1.0000         -1.0000
F2/Q       2.0000         1.0000         -1.0000
F1/Q       2.0000         1.0000         -1.0000
SEL        2.0000         1.0000         -1.0000
CLK1       2.0000         1.0000         -1.0000
```

- The `report_constraint -drv_violation_type min_fanout` command outputs the following:

```
Check type : min_fanout
---------------------------
Pin : F5/Q
Fanout: 0.0000
- min_fanout: 2.0000
-----------------------------------------------------
slack : -2.0000 ( VIOLATED )
```

- The `report_constraints -check_type clock_period` command reports the following:

```
Check type : clock_period
-----------------------------------------------------------
End Point                Slack      Cause
-----------------------------------------------------------
ffo_ram/mm0/mm1/CLKB f   -0.027     VIOLATED
-----------------------------------------------------------
```

Related Information

- set_min_fanout
- set_max_fanout
- set_min_transition
- set_max_transition
- set_min_capacitance
- set_max_capacitance
- report_timing_format
- report_timing

report_cppr

```
report_cppr
[-help]
[-check_type {setup | hold | clock_gating_setup | clock_gating_hold |
clock_gating_pulse_width | data_setup | data_hold | recovery | removal | pulse_width |
clock_period | clock_separation | skew | no_change_setup | no_change_hold | time_borrow}]
-from pin_or_port
-to pin_or_port
[-from_clock clkname]
[-to_clock clkname]
[-early | -late]
[-view view_name]
[> filename]
```

Reports the clock reconvergence pessimism value at the common branching point of the early and late paths in the clock network of the specified data path.

The `report_cppr` command reports separate values for both opening and closing edges for paths with latches as destinations. If both source and destination flops are triggered by multiple clocks, then one report is generated for each separate clock.

When the software is in multi-mode multi-corner analysis mode, `report_cppr` reports the results for each active analysis view.

To use the `report_cppr` command, you must first set the following global variables:

- `timing_cppr_transition_sense`
- `timing_cppr_threshold_ps`

Parameters

`-help`	Prints out the command usage.

`-check_type {setup | hold | clock_gating_setup | clock_gating_hold | clock_gating_pulse_width | data_setup | data_hold | recovery | removal | pulse_width | clock_period | clock_separation | skew | no_change_setup | no_change_hold | time_borrow}`	Reports the CPPR for the paths that end at the specified timing check. The following check types are reported when the software is in setup analysis mode: `setup`, `pulse_width`, `clock_period`, `clock_gating_setup`, `clock_gating_pulse_width`, `data_setup`, `recovery`, skew, `clock_separation`, `time_borrow`, and `no_change_setup`. The following check types are reported when the software is in hold analysis mode: `hold`, `clock_gating_hold`, `data_hold`, `removal`, and `no_change_hold`. If the software is in simultaneous setup and hold analysis mode, all of the check types listed above are reported.
`> filename`	Writes the report to the specified file name. If the file already exists, the software overwrites it. The *`filename`* parameter must be the last argument in the list. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.
`-early`	Reports the hold analysis clock reconvergence pessimism value. You can specify this parameter when the software is in hold analysis mode, or in simultaneous setup and hold analysis mode (`set_global timing_enable_simultaneous_setup_hold_mode true`).
`-from` *`pin_or_port`*	Reports the clock reconvergence pessimism value for paths starting from the specified clock pin of the source flop, or from the specified port.

`-from_clock` *clkname*	Reports the clock reconvergence pessimism value for paths with the specified triggering clock. Use this parameter when there is more than one clock triggering the source flop.
`-late`	Reports the setup analysis clock reconvergence pessimism value. You can specify this parameter when the software is in setup analysis mode, or in simultaneous setup and hold analysis mode (`set_global timing_enable_simultaneous_setup_hold_mode true`).
`-to` *pin_or_port*	Reports the clock reconvergence pessimism value for paths ending at the specified clock pin of the destination flop, or at the specified port.
`-to_clock` *clkname*	Reports the clock reconvergence pessimism value for paths with the specified capturing clock. Use this parameter when there is more than one clock capturing the destination flop.
`-view` *view_name*	Reports the clock reconvergence pessimism values for paths in the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.

Examples

Here are some examples.

- When the software is in multi-mode multi-corner timing analysis mode, you can specify the `-view` parameter to report the CPPR values for a specific analysis view:

```
report_cppr -from CLK3 -to seg3/u14/CK


Start Clock Pin: CLK3 (Triggered by rising edge of clock CLK_W_3)
End Clock Pin: seg3/u14/CK (Triggered by rising edge of clock CLK_W_3)
Analysis Type: Setup
Analysis View: setup_view1
```

```
Common Branch Point : seg3/u17_c/Y
Launching Edge Type At Common Point : Rising
Capturing Edge Type At Common Point : Rising
Pessimism Threshold Value : 0.020
-------------------------------------
Arrival   Late    Early   Pessimism
Times
-------------------------------------
Rise      0.549   0.366   0.183
Fall      0.400   0.267   0.133
-------------------------------------
Edge Selection Mode : Normal
Edge Selection : Match, Using Rise
CPPR Adjustment : 0.183
```

- The following report shows sigma and mean values:

```
report_cppr -from FF01/CP -to FF02/CP


Start Clock Pin: FF01/CP (Triggered by rising edge of clock CK)
End Clock Pin: FF02/CP (Triggered by rising edge of clock CK)
Analysis Type: Setup
Analysis View: view1
Common Branch Point : CK
Launching Edge Type At Common Point : Rising
Capturing Edge Type At Common Point : Rising
Pessimism Threshold Value : 0.020
+-------------------------------------------------------------------------------+
| Arrival | Late  | Late  | Early | Early | Pessimism | Pessimism | Pessimism |
| Times   | Mean  | Sigma | Mean  | Sigma | Mean      | Sigma     |           |
|---------+-------+-------+-------+-------+-----------+-----------+-----------|
| Rise    | 0.000 | 0.000 | 0.000 | 0.000 | 0.000     | 0.000     | 0.000     |
| Fall    | 0.000 | 0.000 | 0.000 | 0.000 | 0.000     | 0.000     | 0.000     |
+-------------------------------------------------------------------------------+
Edge Selection Mode : Normal
Edge Selection : Match, Using Rise
CPPR Adjustment : 0.000 (+3.00S) | 0.000 | 0.00
```

Related Information

- report_timing

report_design

```
report_design
[-view viewName]
[{> | >>} filename | -tcl_list]
[-early | -late]
```

Reports the minimum and/or maximum operating conditions and active design rules for the current design.

When `setAnalysisMode -analysisType single` option is specified, the `report_design` command reports only the maximum operating conditions.

When `setAnalysisMode -analysisType bcWc` option is specified, the `report_design` command reports the minimum operating conditions, if `setAnalysisMode -checkType hold` option is specified.

Both minimum and maximum operating conditions are available for on-chip variation analysis mode.

When simultaneous setup hold analysis mode is set to `on`, the minimum or maximum conditions can be reported using the `-early`/`-late` parameters.

Parameters

{> | >>} *filename*	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *filename* parameter must be the last argument in the list. The *filename* and -tcl_list parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the .gz extension to the file name. *Default:* Report is displayed on standard output without being saved.
-early | -late	The -early parameter reports minimum operating condition in bcwc or onChipVariation analysis mode. The -late parameter reports maximum operating condition. The -late and -early parameters are mutually exclusive.
-tcl_list	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The -tcl_list and *filename* parameters are mutually exclusive; you cannot specify them together.
-view *viewName*	Reports view specific operating conditions.

Examples

- The following is a sample report generated using the `report_design` command:
  `report_design`

Design Property	Value
Design Name	dsp_core
Operating Condition Name	slow
Process	1.000
Voltage	1.620
Temperature	125.000
Tree Type	balanced

Design Rule	Value
Max Transition	NA
Min Transition	NA
Max Capacitance	NA
Min Capacitance	NA
Max Fanout	24.000

- The following sample report shows both the minimum and maximum operating condition details in simultaneous mode:
  `report_design`

Design Property	Value	
Design Name	dsp_test	
	MAX	**MIN**
Operating Condition Name	tdsp_core/%NOM_PVT	tdsp_core/%NOM_PVT

Process	1.000	1.000
Voltage	1.800	1.800
Temperature	25.000	25.000
Tree Type	worst_case	worst_case

Design Rule	Value
Max Transition	NA
Min Transition	NA
Max Capacitance	NA
Min Capacitance	NA
Max Fanout	NA
Min Fanout	NA

Related Commands

- `report_timing`
- `setAnalysisMode`

report_fanin

```
report_fanin
-to pin_port_net_list
[-trace_through {case_disable | user_disable | all}]
[-pin_levels numOfPinLevels]
[-view viewName]
[-nosplit]
[{> | >> } filename]
```

Allows a cone traversal that is not tied to the timing graph, that is, not blocked by `set_disable_timing` and case analysis. By default, the trace matches the timing view and the reports generated using the `all_fanin` and `all_fanout` commands. The reports indicate whether disabled or case analysis arcs were traversed.

Parameters

`> | >>` *filename*	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file.
`-nosplit`	Specifies that reports with long names will not be split into multiple lines. **Note**: You should use the `-nosplit` parameter after specifying the `set_table_style -no_frame_fix_width` parameter.
`-pin_levels` *numOfPinLevels*	Specifies the desired depth of the output.
`-trace_through {case_disable | user_disable | all}`	Allows traversal through disabled arcs. You can specify one of the following options: • `case_disable`: Arcs which are disabled because of set_case_analysis are traversed but because of set_disable_timing are not traversed • `user_disable`: Arcs disabled because of set_disable_timing are traversed but because of set_case_analysis are not traversed • `all`: Trace through all arcs
`-to` *pin_port_net_list*	Specifies a list or collection of pins, ports, or nets. A mixed list or heterogeneous collection is also allowed.
`-view` *viewName*	Specifies disabled arcs in the specified view.

Examples

- The report_fanin -to h0/A command shows the following output:

```
Fanin Network for Pin: h0/A
-------------------------------------------------------------------------------
Source Pin    Sink Pin     Cell Type    Timing Sense   Arc          Case
                                                       Disabled     Analysis
-------------------------------------------------------------------------------
 u1/Y          h0/A         (net)         -               -         -
 -----------------------------------------------------------------------------
 u1/A          u1/Y          BUFX2        positive_unate   -        -
 -----------------------------------------------------------------------------
 in            u1/A          (net)         -               -        -
 -----------------------------------------------------------------------------
```

Related Information

- set_disable_timing
- set_case_analysis
- all_fanin
- all_fanout

report_fanout

```
report_fanout
{-from pin_port_net_list | -clock_tree}
[-trace_through {case_disable | user_disable | all}]
[-pin_levels numOfPinLevels]
[-view viewName]
[-nosplit]
[{> | >> } filename]
```

Allows a cone traversal that is not tied to the timing graph, that is, not blocked by `set_disable_timing` and case analysis. By default, the trace matches the timing view and the reports generated using the `all_fanin` and `all_fanout` commands. The reports indicate whether disabled or case analysis arcs were traversed.

Parameters

`> | >> filename`	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file.
`-clock_tree`	Reports the fanout of the source pin, one after the other for all the clocks in the design.
`-nosplit`	Specifies that reports with long names will not be split into multiple lines. **Note**: You should use the `-nosplit` parameter after specifying the `set_table_style -no_frame_fix_width` parameter.
`-pin_levels numOfPinLevels`	Specifies the desired depth of the output.

`-trace_through {case_disable | user_disable | all}`	Allows traversal through disabled arcs. You can specify one of the following options: • `case_disable`: Arcs which are disabled because of set_case_analysis are traversed but because of set_disable_timing are not traversed • `user_disable`: Arcs disabled because of set_disable_timing are traversed but because of set_case_analysis are not traversed • `all`: Trace through all arcs
`-from` *`pin_port_net_list`*	Specifies a list or collection of pins, ports, or nets. A mixed list or heterogeneous collection is also allowed.
`-view` *`viewName`*	Specifies disabled arcs in the specified view.

Examples

- Suppose arc seg1/u1 is disabled due to following constraint:
  `set_disable_timing -from A -to Y seg1/u1`
  `report_fanout -from IN1` (command will show the following output):

```
Fanout Network for Port: IN1
-----------------------------------------------------------------------
Source Pin  Sink Pin    Cell Type   Timing Sense   Arc       Case
                                                   Disabled  Analysis
-----------------------------------------------------------------------
IN1         seg1/u1/A   (net)       -              -         -
-----------------------------------------------------------------------
```

- The following command will show the output below:
  `report_fanout -from IN1 -trace_through user_disable`

```
Fanout Network for Port: IN1
-----------------------------------------------------------------------
Source Pin    Sink Pin   Cell Type  Timing Sense    Arc       Case
                                                    Disabled  Analysis
-----------------------------------------------------------------------
IN1           seg1/u1/A  (net)      -               -         -
-----------------------------------------------------------------------
seg1/u1/A     seg1/u1/Y  CLKBUF     positive_unate  Y         -
-----------------------------------------------------------------------
seg1/u1/Y     seg1/u2/A  (net)      -               -         -
-----------------------------------------------------------------------
```

Related Commands

- set_disable_timing
- set_case_analysis
- all_fanin
- all_fanout

report_inactive_arcs

```
report_inactive_arcs
[-help]
[instance_or_port_list]
[-exclude_type {const snipped disable library_disable global_disable conditional_disable
Library_Mode}]
[-include_net_arcs]
[-include_when_cond]
[-type {const snipped disable library_disable global_disable conditional_disable
Library_Mode power_mode_disable}]
[-view view_name]
[> <filename[.gz]>]
[>> <filename[.gz]>]
[-delay_arcs_only | -check_arcs_only]
```

Reports information about disabled timing and timing check arcs in the design. Reports all arcs disabled due to user-specified exceptions such as `set_disable_timing` or `set_case_analysis`, as well as information about arcs disabled by constant propagation during timing analysis, snipped loop arcs, and arcs disabled in the timing library.

The report output prints only the 'from' and 'to' information of arcs, and not the full instance name for each arc.

Parameters

`-help`	Prints out the command usage.
instance_or_port_list	Reports disabled timing arcs for the specified list of instances or ports. If you specify a list of instances, the software reports all disabled arcs of the specified cells. If you specify a list of ports, the software reports all disabled arcs directly connected to the ports. *Default:* Reports all disabled net and cell arcs.
`-check_arcs_only`	Lists only disabled check arcs. The `check_arcs` are disabled by using the `set_disable_timing` command. *Default*: Both disabled timing delay arcs and check arcs are listed.

`-delay_arcs_only`	Lists only disabled timing delay arcs.
`-exclude_type {const snipped disable library_disable global_disable conditional_disable Library_Mode}`	Excludes the disabled arcs that belong to the specified category of disabled arcs. The valid options are: const, snipped, disable, library_disable, global_disable, conditional_disable, and, Library_Mode.
`-include_net_arcs`	Includes disabled net arcs in the report.
`-include_when_cond`	Shows the when_cond of arcs in the output report.
`-type {const snipped disable library_disable global_disable conditional_disable Library_Mode power_mode_disable}`	Lists only the disabled arcs that belong to the specified category of disabled arcs. The `-type` options restrict the report to only those arcs which are disabled due to a particular timing analysis step, such as constant propagation, specified disables, or loop arc disables. The valid type options are: • `const`: Propagated constant -- arcs disabled due to a constant value. The constant value can apply to one of the arc nets/pins, to side inputs which disable a condition for the arc, to nets that disable an active mode for the arc, or disable a default conditional arc. • `snipped`: Loop snipped arcs -- arcs automatically disabled by the software as part of a feedback loop. • `disable`: Defined by using the `set_disable_timing` command. • `library_disable`: Delay arc disabled due to library disables. These disables may be caused by the following conditions: ◦ Disables specified using the `set_disable_timing` command. ◦ The arc is an asynchronous arc and the `timing_enable_preset_clear_arcs` global

	variable is set to `false`. • `global_disable`: Lists the arcs that are disabled through user-specified global variable settings. • `conditional_disable`: Arcs disabled if a `when` condition is specified. There is no constant on sink or source pin and arc is disabled due to the `when` condition. • `Library_Mode`: Displays library mode of disabled arcs. • `power_mode_disable`: Reports the arcs for which power mode is disabled.
`-view` *`view_name`*	Generates the report for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.
`{> | >>}` *`filename`*	When specified with `>`, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with `>>`, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *`filename`* parameter must be the last argument in the list. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.

Examples

- The following command reports all disabled arcs in the design:

  ```
  report_inactive_arcs > Report
  ```

  The following example shows the report generated:

  ```
  ------------------------------------------------
  Flags :   const                propagated constant
            snipped              loop snipped arcs
            disable              set_disable_timing
            disable_clock_gating set_disable_clock_gating
            library_disable      set_disable_cell_timing
            Missing_Phase        No Arc Phase Data
  From         To           DisableType ArcType              Reason
  ------------------------------------------------------------------------
  u8/CK ^      u8/D ^       const       hold_rising_clk_rise  u8/D = 1
  u8/CK ^      u8/D v       const       hold_falling_clk_rise u8/D = 1
  seg3/u12/A ^ seg3/u12/Y v disable     combinational         User Disable
  ------------------------------------------------------------------------
  ```

- When the software is in multi-mode multi-corner analysis mode, the report generated lists the disabled arcs by active analysis view, as shown in the following example:

  ```
  ------------------------------------------------
  Flags : const propagated constant
  snipped loop snipped arcs
  disable set_disable_timing
  disable_clock_gating set_disable_clock_gating
  library_disable set_disable_cell_timing
  Missing_Phase No Arc Phase Data
  From        To                 DisableType ArcType  Reason         View
  ------------------------------------------------------------------------------

  gscanmode ^ FE_O305_gmode/I ^  const       net_arc  gscanmode = 0  setup_view2
  gscanmode v FE_O305_gmode/I v  const       net_arc  gscanmode = 0  setup_view2
  gscanmode ^ FE_O305_gmode/I ^  const       net_arc  gscanmode = 0  hold_view1
  gscanmode v FE_O305_gmode/I v  const       net_arc  gscanmode = 0  hold_view1
  gscanmode ^ FE_O305_gnmode/I ^ const       net_arc  gscanmode = 0  hold_view2
  ------------------------------------------------------------------------------
  ```

Related Information

- set_disable_timing
- set_case_analysis

report_instance_library

```
report_instance_library
-file fileName
[-instance instanceName]
[-view viewName | -delay_corner delayCornerName]
[-power_domain domainName]
[-early | -late]
```

Reports library related information for an instance(s). The report prints the following information about an instance:

- Instance name
- Cell type
- View name or delay corner
- Power domain name
- Library/Libset (early/late)
- Library file (early/late)
- Operating conditions
- Binding Status : Prints out the status for the matched voltage library available. If the user setting is `True`, then the binding status is not displayed.

Parameters

`-delay_corner` *delayCornerName*	Specifies the delay corner from which an instance library is to be selected. If no delay corner is specified, then all the instance libraries will be printed.
`-early` | `-late`	Specifies the library set (early/late) from which a library instance is to be printed. If either of these options are not specified then both the early and late library sets are used.

`-file` *fileName*	Specifies the name of the output file. If no file is specified, the report will be printed on standard output file.
`-instance` *instanceName*	Specifies the instance(s) for which the report is to be printed. If no instance is provided the report will print information of all the instances. The value of instance can be a list of regular expressions.
`-power_domain` *domainName*	Specifies the power domain from which instance(s) are to be selected. This option should be specified with the `-delay_corner` or `-view` options.
`-view` *viewName*	Specifies the view from which instance library is to be selected.

Examples

- The `report_instance_library` command displays the following information about instance `ff2` :

```
Instance              : ff2
Analysis View         : view1
Power Domain          : Default
Library/Libset(early) : locv/libCom
Library File (early)  : locv/locv2.lib
Library/Libset(late)  : locv/libCom
Library File (late)   : locv/locv2.lib
Op Cond               : P->1.000000, V->3.300000, T->25.000000 (locv/%NOM_PVT)
Cell Type             : gateCell
```

- The following command displays the following information about instance AO1/g1080:

```
report_instance_library -instance AO1/g1080

Instance : AO1/g1080
Cell : INVXL
Analysis View : AV_bc_on
Power Domain : Default
Library/Libset : slow/INV.lib
Library File : INV.lib
User Setting : True
Op Cond : P->1.000000, V->1.500000, T->0.000000 (fast)
Cell Type : gateCell
```

Related Commands

- `check_instance_library_in_views`

report_lib_arcs

```
report_lib_arcs
[-help]
-arc <string>
[-fall]
[-nldm]
[-quiet]
[-rise]
[-lvf  [-early]]
[-lvf  [-late]]
[-moments  [-mean_shift]]
[-moments  [-std_dev]]
[-moments  [-skewness]]
[-input_net_transition <float> | -constrained_pin_transition <float>]
[-input_net_transition <float> | -related_pin_transition <float>]
[-total_output_net_capacitance <float> | -constrained_pin_transition <float>]
[-total_output_net_capacitance <float> | -related_pin_transition <float>]
[-constrained_pin_transition <float> | -delay ] [-constrained_pin_transition <float> | -
transition ]
[-related_pin_transition <float> | -delay ]
[-related_pin_transition <float> | -transition ]
```

Reports the library arc data. The command also returns the data in a Tcl object, which can be used in scripts.

This command uses a collection of arcs as input, which can be queried using the `get_lib_arcs` command.

The units for the input and output data can be obtained using the `get_time_unit` and `get_capacitance_unit` commands.

Parameters

`-help`	Prints out the command usage.
`-arc <string>`	Specifies the arc(s) for which the report is to be printed.

`-constrained_pin_transition <float>`	Specifies the constrained pin slew (valid for constraint arcs) at which the parameters need to be reported. The slew units must be specified using the `get_time_unit` command.
`-delay`	Specifies if delay is to be reported.
`-early`	Specifies if early LVF data is to be reported. This parameter can be used only in conjunction with the `-lvf`.
`-fall`	Specifies if details of fall tables are to be reported.
`-input_net_transition <float>`	Specifies the slew at which the parameters need to be reported. The slew units must be specified using the `get_time_unit` command.
`-late`	Specifies if late LVF data is to be reported. This parameter can be used only in conjunction with the `-lvf`.
`-lvf`	Specifies if LVF data is to be reported.
`-mean_shift`	Specifies if the mean shift data is to be reported. This parameter can be used only in conjunction with the `-moments`.
`-moments`	Specifies if moments data is to be reported.
`-nldm`	Specifies if NLDM data is to be reported.
`-quiet`	Disables output to be displayed on the screen.
`-related_pin_transition <float>`	Specifies the related pin slew (valid for constraint arcs) at which the parameters need to be reported. The slew units must be specified using the `get_time_unit` command.
`-rise`	Specifies if details of rise tables are to be reported.

`-skewness`	Specifies if skewness data is to be reported. This parameter can be used only in conjunction with the `-moments`.
`-std_dev`	Specifies if standard deviation data is to be reported. This parameter can be used only in conjunction with the `-moments`.
`-total_output_net_capacitance <float>`	Specifies the load at which the parameters need to be reported. The load must be specified using the `get_capacitance_unit` command.
`-transition`	Specifies if the output slew is to be reported.

Examples

- The following command reports the library arc data containing mean shift rise tables:

```
report_lib_arcs -arc [get_lib_arcs -from cell_w/DFF/CK -to cell_w/DFF/Q] -
input_net_transition 0.08 /
       -total_output_net_capacitance 0.055 -mean_shift -rise -delay -moments -nldm

Units of Report:- Time: '1000ps', Capacitance: '1000fF'.
input_net_transition:0.080000, total_output_net_capacitance:0.055000
-------------------------------------------------------------------
| Index | Timing Arc | Value(s) |
| ------------------------------------------------------------------------------|
|       | Cell       | Source/Sink Pin | Timing Sense | Timing Type | cell_rise |
-------------------------------------------------------------------------------
| 1     | cell_w/DFF | CK -> Q         | non_unate    | rising_edge | 0.258444  |
-------------------------------------------------------------------------------
{{cell_rise 0.258444279432} {ocv_mean_shift_cell_rise NA}}
```

- The following command reports the library arc data containing fall tables for moments data:

```
report_lib_arcs -arc [get_lib_arcs -from cell_w/BUF/A -to cell_w/BUF/Y] /
```

```
    -input_net_transition 0.055 -total_output_net_capacitance 0.15 -fall -delay -
moments -nldm

Units of Report:- Time: '1000ps', Capacitance: '1000fF'.
input_net_transition:0.055000, total_output_net_capacitance:0.150000
-------------------------------------------------------------------------------
| Index | Timing Arc                                                  | Value(s)   |
| ------------------------------------------------------------------------------|
|       | Cell       | Source/Sink Pin | Timing Sense   | Timing Type  | cell_fall  |
-------------------------------------------------------------------------------
| 1     | cell_w/BUF | A -> Y          | positive_unate | combinational| 0.129744   |
-------------------------------------------------------------------------------
-------------------
{{cell_fall 0.129} {ocv_std_dev_cell_fall NA} {ocv_mean_shift_cell_fall NA}
{ocv_skewness_cell_fall NA}}
```

- The following command reports the library arc data containing rise tables data with output slew:

```
report_lib_arcs -arc [get_lib_arcs -from cell_w/DFF/CK -to cell_w/DFF/Q] -
input_net_transition 0.08 /
     -total_output_net_capacitance 0.055 -mean_shift -rise -delay -moments -nldm -
transition

Units of Report:- Time: '1000ps', Capacitance: '1000fF'.
input_net_transition:0.080000, total_output_net_capacitance:0.055000
-------------------------------------------------------------------------------
--------------
| Index | Timing Arc
| Value(s)  |
| ------------------------------------------------------------------------------
--------------
|       | Cell       | Source/Sink Pin | Timing Sense | Timing Type | rise_transition
| cell_rise |
-------------------------------------------------------------------------------
--------------
| 1     | cell_w/DFF | CK -> Q         | non_unate    | rising_edge | 0.21692
 | 0.258444  |
-------------------------------------------------------------------------------
--------------
{{cell_rise 0.258} {rise_transition 0.216} {ocv_mean_shift_cell_rise NA}
{ocv_mean_shift_rise_transition NA}}
```

This command also returns the data in a Tcl object that can be parsed and used in scripts. A 3-Dimensional Tcl List will return the following information:

{{TableName Value} {TableName Value}} {{TableName Value} {TableName Value}}

Here, the first list (left column) indicates the Table-Value pair in one arc. The second list indicates the various tables present per arc. And the third list (last column) indicates the list of total arcs in the collection.

- The following command reports the slew and capacitance load:

```
set items [report_lib_arcs -arc [ get_lib_arcs -from sc_cmoslp/AO22X/clk -to
sc_cmoslp/AO22X /out1] /
     -input_net_transition 0.003 -total_output_net_capacitance 0.021]


Units of Report:- Time: '1000ps', Capacitance: '1000fF'.
input_net_transition:0.003, total_output_net_capacitance:0.021
-------------------------------------------------------------------------------------
-----------------------------------------------------
| Index | Timing Arc
                                        | Value(s) |
|------------------------------------------------------------------------------------
-----------------------------------------------------
|       | Cell            | Source/Sink Pin | Timing Sense   | Timing Type   |
rise_transition | fall_transition | cell_rise | cell_fall |
-------------------------------------------------------------------------------------
-----------------------------------------------------
| 1     | sc_cmoslp/AO22X | clk -> out1     | positive_unate | combinational |
0.20**          | 0.30**          | 1.00**    | 1.20**    |
-------------------------------------------------------------------------------------
-----------------------------------------------------
| 2     | sc_cmoslp/AO22X | clk -> out1     | negative_unate | combinational |
0.20**          | 0.30**          | 1.00**    | 1.20**    |
-------------------------------------------------------------------------------------
-----------------------------------------------------

** --> Scalar Value{{cell_rise 0.999} {rise_transition 0.199} {cell_fall 1.199}
{fall_transition 0.299}} {{cell_rise 0.999} {rise_transition 0.199} {cell_fall 1.199}
{fall_transition 0.299}}

tempus 5 > puts $items
{{cell_rise 0.999} {rise_transition 0.199} {cell_fall 1.199} {fall_transition 0.299}}
{{cell_rise 0.999} {rise_transition 0.199} {cell_fall 1.199} {fall_transition 0.299}}
```

Related Commands

- get_lib_arcs
- get_time_unit
- get_capacitance_unit

report_min_pulse_width

```
report_min_pulse_width
[-verbose]
[-clock <object_list>]
[-path_type {summary | short | full_clock}]
[-pins object_list]
[-net]
[-violation_only]
[-view view_name]
[-waveform_aware_type {regular | waveform_aware | both}]
[ { > | >> } filename | -tcl_list]
```

Reports a set of minimum pulse width violations on pins and ports on the clock network. The report shows the required pulse width, the actual pulse width, and by how much the constraint is violated or met. This is reported separately for sequential elements and instances on the clock tree.

You can customize pulse width reporting format by using the `report_timing_format` global variable.

The report header shows clock edge information to make it more intuitive for clock style checks.

Parameters

`{> | >>} filename`	When specified with `>`, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with `>>`, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *`filename`* parameter must be the last argument in the list. The *`filename`* and `-tcl_list` parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.

`-clock <object_list>`	Specifies a set of clocks to for which to report the minimum pulse width.
`-net`	Reports the net arcs. This parameter can be specified with the `-path_type full_clock` option only.
`-path_type {summary | short | full_clock}`	Specifies the type of report to generate. *Default:* `summary` • `summary`: Generates a summary report for each pulse width check that shows the required width, the actual width, and the resulting slack. • `short`: Generates a path report for each pulse width check that lists the latency of each edge of the pulse as a single lumped value, instead of reporting a detailed path report. • `full_clock`: Generates a path report for each pulse width check that shows the detailed pin-to-pin path for each edge of the pulse. **Note:** The `-path_type full_clock` parameter may cause an increased runtime. In that case you can use the `-path_type short` parameter.
`-pins` *object_list*	Specifies a set of ports and pins on the clock network. If this list is not specified, then the report contains all the pins and ports in the current design.
`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *`filename`* parameters are mutually exclusive; you cannot specify them together.

`-verbose`	Provides detailed information about minimum pulse width calculations. **Note:** This parameter is obsolete. The parameter still works in this release, but to ensure compatibility with future releases, use the `-path_type` parameter instead.
`-view` *view_name*	Generates the report for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.
`-violation_only`	Reports only the negative slack minimum pulse width violations.
`-waveform_aware_type {regular | waveform_aware | both}`	Reports the waveform-aware pulse width checks. You can specify the following options: • `waveform_aware`: Reports only the waveform-aware pulse width checks. • `both`: Reports waveform-aware as well as regular pulse width checks. • `regular`: Reports regular pulse width checks. This is the default setting.

Examples

- The `report_min_pulse_width` will generate a summary report as shown below:

```
report_min_pulse_width

--------------------------------------------------
SEQUENTIAL CLOCK PULSE WIDTH
--------------------------------------------------
Pin                Required     Actual Pulse   Slack
                   Pulse Width  Width
--------------------------------------------------
seg3/u14/CK (high) 0.169        0.860         0.691
seg3/u3/CK (high)  0.169        0.875         0.706
seg3/u9/CK (high)  0.169        0.875         0.706
seg3/u3/CK (low)   0.195        1.083         0.888
seg3/u9/CK (low)   0.195        1.083         0.888
--------------------------------------------------
```

- A short report can be generated at any clock pin having pulse width check, as shown below. The pulse width is shown as sequential clock pulse width in case it is reported on sequential pins:

```
report_min_pulse_width -path_type short

SEQUENTIAL CLOCK PULSE WIDTH
----------------------------
Path 1: MET PulseWidth Check with Pin seg3/u14/CK
Ending Clock Edge: seg3/u14/CK (v) checked with trailing edge of 'CLK_W_3'
Beginning Clock Edge: seg3/u14/CK (^) triggered by leading edge of 'CLK_W_3'
Other End Arrival Time 1.503
- PulseWidth 0.169
+ Phase Shift 0.000
= Required Time 1.334
- Arrival Time 0.621
= Slack Time 0.714
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.621
= Beginpoint Arrival Time 0.621
----------------------------------------
Pin         Cell     Delay  Arrival  Edge
                            Time
----------------------------------------
seg3/u14/CK DFF              0.621   ^
----------------------------------------
Path 2: MET PulseWidth Check with Pin seg3/u3/CK
Ending Clock Edge: seg3/u3/CK (v) checked with trailing edge of 'CLK_W_3'
```

```
Beginning Clock Edge: seg3/u3/CK (^) triggered by leading edge of 'CLK_W_3'
Other End Arrival Time 1.578
- PulseWidth 0.169
+ Phase Shift 0.000
= Required Time 1.409
- Arrival Time 0.682
= Slack Time 0.727
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.682
= Beginpoint Arrival Time 0.682
----------------------------------------
Pin        Cell    Delay  Arrival   Edge
                          Time
----------------------------------------
seg3/u3/CK DFF             0.682     ^
----------------------------------------
```

Related Commands

- report_analysis_coverage
- report_timing
- set_min_pulse_width
- reset_min_pulse_width

report_mode

```
report_mode
instance_list
[-view viewName]
```

Reports the status of library modes associated with a specified instance. The report includes the status of each mode and its condition, for each mode group defined in the timing library.

Parameters

instance_list	Generates a report for the specified instances.
-view viewName	Reports the status of library modes in the specified active analysis view. Default: Reports the status of library modes in all active analysis views.

Examples

- The following command activates the `tst_mode_disable` library mode for instances of `i_ram`:
  ```
  set_mode tst_mode_disable [get_cells i_ram]
  ```

- The following command sets the active setup and hold analysis views:
  set_analysis_views -setup {v1 v2} -hold {v3}

- The following command reports the status of all modes associated with m4k_top:
  ```
  report_mode m4k_top

  ------------------------------------------------
  Mode Group tets_mode of m4k_top
  ------------------------------------------------
  Mode Name         Status    Condition     View Name
  ------------------------------------------------
  test_mode_enable  ACTIVE    TST_BYPASS    View1
  test_mode_disable INACTIVE  !(TST_BYPASS) View2
  ------------------------------------------------
  ```

- The following command reports the status of all modes in the analysis view `View1`:
  ```
  report_mode m4k_top -view View1

  ------------------------------------------
  Mode Group tets_mode of m4k_top
  ------------------------------------------
  Mode Name        Status  Condition  View Name
  ------------------------------------------
  test_mode_enable ACTIVE  TST_BYPASS View1
  ------------------------------------------
  ```

Related Information

- set_analysis_view
- set_mode

report_net

```
report_net
[-min_fanout int]
[-max_fanout int]
{-net list_of_net_name_or_id | -pin list_of_pin_name_or_id}
[-tcl_list]
[-hier]
[[{> | >>} filename | -output filename]
 | -tcl_list]
```

Reports the net information on the current module. The information includes the net name, the number of source pins, sink pins and bidirectional pins on the net, wire capacitance, and total capacitance.

Electrical information such as pin capacitance, and wire capacitance with which the net is associated are also included.

Use the `-pin` option to report the net that is connected to the specified pin (or port). Both `-net` and `-pin` cannot be specified at the same time. The net (or pin) names can be hierarchical names that are unique relative to the top cell.

This command only searches for the defined nets and does not modify any object in database.

Parameters

{> | >>} *filename*	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *filename* parameter must be the last argument in the list. The *filename*, -output, and -tcl_list parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the .gz extension to the file name. *Default:* Report is displayed on standard output without being saved.

`-hier`	Reports all of the nets in the design hierarchy. Without this argument, only the nets in the current level of hierarchy are reported.
`-max_fanout` *`int`*	Reports the net whose number of fanouts is less than or equal to the maximum number specified by *`integer`*.
`-min_fanout` *`int`*	Reports the net whose number of fanouts is greater than or equal to the minimum number specified by *`integer`*.
`-net` *`list_of_net_name_or_id`*	Reports the set of nets on the list of net names. This argument cannot be specified at the same time as `-pin`. If neither `-net` or `-pin` is specified, all nets are listed by default. **Note:** This argument also reports the capacitance of the net and the input pins.
`-output` *`filename`*	Dumps the report to the specified file instead of to the standard output. The *`filename`* argument must be the last argument in the list. This parameter cannot be used simultaneously with the `-tcl_list` option.
`-pin` *`list_of_pin_name_or_id`*	Reports the set of nets to which the pins on the list *`list_of_pin_name_or_id`* are connected. This argument cannot be specified at the same time as `-net`.
`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for extracting information from the report in a Tcl program. The `-tcl_list`, *`filename`* and `-output` parameters are mutually exclusive; you cannot specify them together.

Examples

- Control the `report_net` table fields, such as setting the minimum and maximum column width using the `set_table_style` command. The table names are `report_net_source_table`, `report_net_sink_table`, `report_net_summary_table`, `report_net`, and `report_net_header`.
- For example, the following command sets an indentation of six columns for the table and disables printing of the first column:

```
set_table_style -name report_net_source_table -indent 6 -max_widths {0,}
report_net
```

- The format of the `report_net` report is the same for all the examples, as shown in the first example. The following command reports information about net named `in`, as shown in the following example:

```
report_net -net n5

Net Name : n5
Number of Sources : 1
Number of Sinks : 1
Number of Bidis : 0
Source of parasitics : Wireload model(.lib None Generated)
Net Capacitance : 0.000
Total Capacitance : 0.011
------------------------------------------------------------------------
Source
------------------------------------------------------------------------
Pin R Driver Load Slew
------------------------------------------------------------------------
Name  Dir  Cell  Cap   Model   Model  Ctotal  Rise   Fall  Phase
------------------------------------------------------------------------
u5/Q  OUT  DFF   0.000 Unknown CTOTAL 1.000   0.003  0.003 CLK_W_4_gen2(D)(N)
u5/Q  OUT  DFF   0.000 Unknown CTOTAL 1.000   0.003  0.003 CLK_W_4_gen1_inv(D)(P)
------------------------------------------------------------------------
------------------------------------------------------------------------
Sinks for pin u5/Q
------------------------------------------------------------------------
Pin R Delay Slew
------------------------------------------------------------------------
Name   Dir  Cell  Cap    Parameters Rise   Fall   Rise   Fall   Phase
------------------------------------------------------------------------
u7/A  IN   BUF    0.011  -          0.000  0.000  0.003  0.003   CLK_W_4_gen2(D)(N)
```

```
u7/A  IN   BUF    0.011  -          0.000  0.000  0.003  0.003   CLK_W_4_gen1_inv(D)
(P)
-------------------------------------------------------------------------------
```

- The following command reports the net to which the pin interesting_pin is connected:
  ```
  report_net -pin interesting_pin
  ```
- The following command reports all the nets in the current module:
  ```
  report_net -net *
  ```
- The following command reports all nets with a fanout greater than 16:
  ```
  report_net -net * -min_fanout 16
  ```

Related Information

- report_path_exceptions
- report_timing

report_path_exceptions

```
report_path_exceptions
[-early | -late]
[-ignored]
[-view view_name]
[{> | >>} filename | -tcl_list]
```

Generates a report about path exceptions specified using the `set_false_path`, `set_min_delay`, `set_max_delay`, and `set_multicycle_path` commands.

By default, the `report_path_exceptions` command only reports activated path exceptions. To report only the path exceptions that have been ignored by timing analysis, specify the `-ignored` parameter.

If more than one path exception is applied on a timing path, multiple path exceptions that match a given path are prioritized. The adjustment on the path honors the path exception with the highest priority.

When the software is in multi-mode multi-corner timing analysis mode, the report generates path exception information for every active analysis view.

The `report_path_exceptions` command output also includes 2 columns that show the file name and line number. The report output shows the exact file name and line number of the applied/ignored constraints. The compressed constraints will be decompressed first and then shown as individual constraints as written in the SDC file.

Parameters

{> | >>} *filename*	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *filename* parameter must be the last argument in the list. The *filename* and -tcl_list parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the .gz extension to the file name. *Default:* Report is displayed on standard output without being saved.
-early | -late	Generates the path exception report for either early paths or late paths. *Default:* Generates a report based on the current timing analysis mode.

-ignored	Reports only the path exceptions that have been ignored by timing analysis. This parameter can be used for debugging constraints that may be ineffective or incorrectly entered. A path exception may be ineffective if it is covered by, or overridden by, another path exception. For example:
	
	`set_false_path -to OUT`
	`set_multicycle_path -to OUT 2`
	
	In the above scenario, the set_multicycle_path command will be ignored as false paths have higher priority over cycle addition on the same timing paths.

`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *`filename`* parameters are mutually exclusive; you cannot specify them together.
`-view` *`view_name`*	Generates a path exception report for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.

Examples

- The following commands shows that a false path has priority over cycle addition on the same path:

```
set_false_path -from seg1/u3/q -to seg1/u9/d
set_multicycle_path -from seg1/u3/Q -to seg1/u9/D 2

> report_path_exception -ignored
-----------------------------------------------------
From       To          Early           Late
-----------------------------------------------------
seg1/u3/Q  seg1/u9/D   false ignored   false ignored
seg1/u3/Q  seg1/u9/D   -               add 1 ignored
seg1/u3/Q  seg1/u9/D   add 0 ignored   -
-----------------------------------------------------
```

  The following command shows how edges are handled:

```
set_false_path -rise_from {IN1} -through {OUT1}
set_false_path -rise_from {IN1 IN2} -fall_to {OUT1}
set_false_path -rise_from {IN1}

tempus> report_path_exceptions -ignored
-------------------------------------------------------------
From        Through    To      Early            Late
-------------------------------------------------------------
^ IN1 IN2   -          v OUT1  false ignored   false ignored
^ IN1       OUT1       -       false ignored   false ignored
-------------------------------------------------------------
```

```
> report_path_exceptions -early
-----------------------------------
From       Through    To  Early
-----------------------------------
^ IN1      -          -   false
-----------------------------------

tempus> report_path_exceptions -late
-----------------------------------
From   Through  To  Late
------------------------------
^ IN1  -       -    false
------------------------------
```

When the software is in multi-mode multi-corner timing analysis mode, the software generates path exception information for every active analysis view:

```
set_analysis_view -setup {setup1 setup2} -hold {hold1 hold2}

> report_path_exceptions
----------------------------------------
From              Early  Late     View
                                  Name
----------------------------------------
test_mode reset  false           setup2
test_mode reset  false           setup1
test_mode reset         false    hold1
test_mode reset         false    hold2
----------------------------------------
```

Specify the -view parameter to generate a report only for a specific active analysis view:

```
> report_path_exceptions -view setup1
-------------------------------------------
From       Early    Late     View
                             Name
-------------------------------------
test_mode reset    false     setup1
-------------------------------------
```

Related Information

- report_timing

report_path_groups

```
report_path_groups
[-name group_name]
[{> | >>} filename | -tcl_list]
```

Lists the names of all path groups with the corresponding paths.

Parameters

`{> | >>} filename`	When specified with `>`, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with `>>`, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *`filename`* parameter must be the last argument in the list. The *`filename`* and `-tcl_list` parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.
`-name group_name`	Limits the report to the specified path group. *Default*: All path groups are reported.
`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *`filename`* parameters are mutually exclusive; you cannot specify them together.

Example

The following command shows a sample report:

```
> report_path_groups
---------------------------------
From    To         Group Name
---------------------------------
CLK1    seg1/u3/D  path1
---------------------------------
```

Related Information

- set_clock_groups
- reset_clock_groups
- reset_path_group
- get_path_groups
- group_path

report_ports

```
report_ports
[-type {[input] | [source_insertion] | [insertion] | [clock_root]
| [uncertainty] | [arrival] | [required] | [external] | [clk_arrival]
| [port_cap] | [fanout_load] | [fanout_load_limit] | [drive_resistance]
| [drive_cell] | [slew_time] | [slew_limit] | [constant] | external_detail
| drive_resistance_detail}]
[-include_pins]
[-pins port_name_list ]
[-view viewName ]
[> filename | -tcl_list]
```

Reports timing constraints on ports. By default, the command reports all timing constraints on all ports of the current module.

The direction (*DIR*) column of the generated report can contain one of the following abbreviations:

- `IN` indicates that the pin is an input pin.
- `OUT` indicates that the pin is an output pin.
- `INTL` indicates that the pin is an internal pin (a pin inside the instance boundary).

The *Clock Name* column of the report can contain the following symbols:

- `D` indicates it is a data signal in the clock domain.
- `C` indicates it is a clock signal in the clock domain.
- `P` indicates it is a positive phase.
- `N` indicates it is a negative phase.
- Asterisk (`*`) indicates there is a timing exception associated with the signal.

  Note: Currently, it is not possible to expand the asterisk to determine which exception is associated with the reported timing phase.

Parameters

`> filename`	Writes the report to the specified file name. If the file already exists, the software overwrites it. The *filename* parameter must be the last argument in the list. The *filename* and `-tcl_list` parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.
`-include_pins`	Reports timing constraints on instance pins, as well as on ports. *Default:* Reports timing constraints on ports only.
`-pins port_name_list`	Specifies the pins or ports to be reported. Pins can be specified by name, or by object ID in a Tcl list.
`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *filename* parameters are mutually exclusive; you cannot specify them together.
`-type {assertion_type | assertion_type_list}`	Specifies the constraints to be reported. *Default*: All constraints are reported. The following are valid assertion_type values: • `arrival`: Reports arrival time constraints as set by `set_input_delay` command. • `clk_arrival`: Reports clock arrival time constraints as set by the `set_clock_latency` command. • `clock_root`: Reports the clock root as set by the `create_clock` command.

- `constant`: Reports constant constraints as set by the `set_case_analysis` command.
- `drive_cell`: Reports drive cell constraints as set by the `set_driving_cell` command.
- `drive_resistance`: Reports drive resistance constraints as set by the `set_drive` command.
- `drive_resistance_detail`: Reports detail about the input ports with drive resistance constraints.
- `external`: Reports external delay constraints as set by the `set_output_delay` command.
- `external_detail`: Reports detail about the output ports with external delay constraints.
- `fanout_load`: Reports fanout load constraints as set by the `set_fanout_load` command.
- `fanout_load_limit`: Reports fanout load limit constraints as set by the `set_max_fanout` command.
- `input`: Reports the input delay constraints as set by the `set_input_delay` command.
- `insertion`: Reports the network insertion delay as set by the `set_clock_latency` command.
- `port_cap`: Reports port capacitance constraints as set by the `set_load` command.
- `required`: Reports required time constraints as set by the `set_output_delay` command.
- `slew_limit`: Reports slew time limit constraints as set by the `set_max_transition` command.
- `slew_time`: Reports slew time constraints as set by the `set_input_transition` command.
- `source_insertion`: Reports the source insertion

	delay as set by the `set_clock_latency -source` command. • `uncertainty`: Reports the clock uncertainty as set by the `set_clock_uncertainty` command.
`-view` *viewName*	Specifies the view name.

Examples

- The following command shows a port report, as shown in the following example:

```
report_ports

------------------------------------------------------------------------
Early Late
------------------------------------------------------------------------
Pin     Dir     Assertion          Clock Name    Rise     Fall    Rise   Fall
Name
------------------------------------------------------------------------
CLK1    IN      clock_root          CK2(C)(P)
CLK2    IN      clock_root          CLK2(C)(P)
CLK3    IN      clock_root          CLK3(C)(P)
IN1     IN      drive_resistance    *(D)(P)       2.000   2.000   2.000   2.000
IN2     IN      drive_resistance    *(D)(P)       2.000   2.000   2.000   2.000
IN3     IN      drive_cell          *(D)(P)       INV     INV     INV     INV
OUT1    OUT     external            GCLK1(C)(P)   1.000   1.000   1.000   1.000
OUT2    OUT
OUT3    OUT derived_clock GCLK4(C)(P)
------------------------------------------------------------------------------
Pin   Dir    Assertion               Value
Name
----------------------------------------------------------
CLK3   IN   slew_limit              ( 0.900 : )
CLK3   IN   port_cap_limit          ( 0.020 : 0.900 )
CLK3   IN   fanout_load_limit       5.000
CLK2   IN   slew_limit              ( 0.900 : )
CLK2   IN   port_cap_limit          ( 0.020 : 0.900 )
CLK2   IN   fanout_load_limit       5.000
CLK1   IN   slew_limit              ( 0.900 : )
CLK1   IN   port_cap_limit          ( 0.020 : 0.900 )
```

```
CLK1    IN    fanout_load_limit       5.000
IN3     IN    slew_limit              ( 0.900 : )
IN3     IN    port_cap                ( 1.500 : : 2.600 )
IN3     IN    port_wire_cap           ( 1.200 : : 3.000 )
IN3     IN    port_cap_limit          ( 0.020 : 0.900 )
IN3     IN    fanout_load_limit       5.000

IN2     IN    slew_limit              ( 0.900 : )
IN2     IN    port_cap                ( 1.500 : : 2.600 )
IN2     IN    port_wire_cap           ( 1.200 : : 3.000 )
IN2     IN    port_cap_limit          ( 0.020 : 0.900 )
IN2     IN    fanout_load_limit       5.000
IN1     IN    slew_limit              ( 0.900 : )
IN1     IN    port_cap                ( 1.500 : : 2.600 )
IN1     IN    port_wire_cap           ( 1.200 : : 3.000 )
IN1     IN    port_cap_limit          ( 0.020 : 0.900 )
IN1     IN    fanout_load_limit       5.000
OUT3    OUT   slew_limit              ( 0.900 : )
OUT3    OUT   port_cap_limit          ( 0.020 : 0.900 )
OUT2    OUT   slew_limit              ( 0.900 : )
OUT2    OUT   port_cap_limit          ( 0.020 : 0.900 )
OUT1    OUT   slew_limit              ( 0.900 : )
OUT1    OUT   port_cap_limit          ( 0.020 : 0.900 )
```

- The following command shows `source_insertion` and `uncertainty` assertion type options as shown in the following example:

```
report_ports -type clock_root source_insertion input -pin clk in


> report_ports -type {uncertainty source_insertion} -pin CLK1
-------------------------------------------------------------------------
Early Late
-------------------------------------------------------------------------
Pin       Dir   Assertion       Clock  Rise     Fall     Rise   Fall
Name                            Name
-------------------------------------------------------------------------
CLK1     IN  source_insertion *(C)(P)  -0.500  -0.500   -0.500 -0.500
-------------------------------------------------------------------------

Pin     Dir     Assertion    Value
Name
```

```
-------------------------------------------------------------------
CLK1   IN       uncertainty  0.400(T) 0.400(T) 0.400(T) 0.400(T)
-------------------------------------------------------------------
```

- The following command shows `clock_root` and `uncertainty` assertion type options as shown in the following example:

```
> report_ports -type {input uncertainty clock_root}
-------------------------------------------------------------------------
Early Late
-------------------------------------------------------------------------
Pin      Dir   Assertion     Clock Name     Rise   Fall   Rise   Fall
Name
-------------------------------------------------------------------------
CLK1     IN    clock_root    CLK_W_1(C)(P)
in1      IN    input         @(D)(P)        1.000  1.000  1.000  1.000
-------------------------------------------------------------------------

Pin    Dir    Assertion     Value
Name
-----------------------------------------------------------------
CLK1   IN     uncertainty   0.400(T) 0.400(T) 0.400(T) 0.400(T)
-----------------------------------------------------------------
```

Related Information

- `report_clocks`
- `report_net`
- `report_timing`

report_socv_interconnect_variation

```
report_socv_interconnect_variation
[-help]
[-delay_corner string]
[-power_domain string]
```

Reports the RC variation multiplier data provided in the SOCV files.

Parameters

`-help`	Prints out the command usage.
`-delay_corner` *string*	Specifies the name of the delay corner.
`-power_domain` *string*	Specifies the name of the power domain.

report_socv_library

```
report_socv_library
[-help]
[-categories string]
[-coverage_verbose]
[-list_not_annotated]
[ -libset string | { -delay_corner string [ -power_domain string] }| { -view string [-
power_domain string]}] [ -libs string | -lib_cellsstring]
[-coverage]
```

Reports the coverage of SOCV library(s) loaded in the design.

Parameters

`-help`	Prints out the command usage.
`-categories string`	Allows you to control the types of table data to be used to check delay arcs, transition arcs, and check arcs. *Default*: all
`-coverage`	Reports coverage library data.
`-coverage_verbose`	Generates detailed report table for each cell.
`-delay_cornerstring`	Checks all the libraries defined in the specified library set.
`-lib_cells string`	Specifies library cells. The allowed format is \u201clibrary/cell\u201d name or pattern or collection. This is typically used for debugging purpose.
`-libs string`	Specifies the name of the library. If -library_set, or -delay_corner parameter is not specified then all libraries matching the library name across the MMMC configuration will get checked.
`-libset string`	Checks all the libraries defined in the specified library set.
`-list_not_annotated`	Control the types of table data to be used to check delay arcs, transition arcs, and check arcs. *Default*: all

`-power_domain` *string*	Checks and reports the specified power domain.
`-view` *string*	Checks all the delay corners associated with a view.

report_statistical_timing_derate_factors

```
report_statistical_timing_derate_factors
-type {arrival_time_based
      | slack_time_based}
-ssta_view ssta_view_name
-sta_view sta_view_name
-path_group groupname_list
-derate_cell
-derate_net
-derate_cell_check
-early | -late]
[-path_based_ssta]
[-clock_factor | -data_factor]
[-max_paths integer]
[-max_slack float]
[-max_points integer]
```

Generates design specific OCV factor. The scaling factor (hold or setup) is computed either by using the arrival time or slack time approach.

Parameters

`-clock_factor	-data_factor`	Specifies if the user-defined factor value is for clock or data. If you specify the `-clock_factor` parameter, then the `-data_factor` parameter is calculated by the software, and vice versa. These parameters are applicable to type `slack_time_based`. **Note:** The `-clock_factor` parameter and the `-data_factor` parameter are mutually exclusive. You cannot specify them together.
`-derate_cell`	Applies user-specified scaling factors on cell delays.
`-derate_cell_check`	Applies user-specified scaling factors on timing checks.
`-derate_net`	Applies user-specified scaling factors on net delays.

`-late	-early`	Specifies early or late mode analysis. Default is `late`.
`-max_paths`	Reports the specified number of worst paths in the design, regardless of the endpoint.
`-max_points` *integer*	Reports the worst path that is found to each endpoint up to the number specified by the `-max_points` option.
`-max_slack` *float*	Reports only those paths with slack equal to or less than the specified float value.
`-path_based_ssta`	Enables generation of the statistical OCV factor using path based SSTA analysis.
`-path_group` *groupname_list*	Considers only the path contained in the groups in the specified *groupname_list* for computing the statistical timing derate factor.
`-ssta_view` *ssta_view_name*	Specifies view for SSTA run.
`-sta_view` *sta_view_name*	Specifies view for STA run.
`-type arrival_time_based	slack_time_based`	
	Specifies if the OCV factor needs to be generated using the arrival time or slack time approach. In the arrival time approach, the SSTA arrival time values are used to compute the statistical OCV factor for clock and data. In the slack time approach, the STA and SSTA slacks are used to generate the statistical OCV factor for either clock or data depending on whether you specify the factor for data or clock. *Default:* `arrival_time_based` You can use the `-type arrival_time_based` parameter with the following parameters only: • `-late` • `-early` • `-path_based_ssta`

	You can use the `-type slack_time_based` parameter with the following parameters only: • `-ssta_view` • `-sta_view` • `-late` • `-early` • `-clock_factor` • `-data_factor` • `-derate_cell` • `-derate_net` • `-derate_cell_check` • `-max_paths` • `-max_slack`

Examples

- The following example computes the factors for setup analysis using the arrival time based approach:
  ```
  report_statistical_timing_derate_factors -type arrival_time_based -late
  ```
- The following example computes the data factor for setup analysis using the slack time based approach applying a user defined clock factor of 0.95:
  ```
  report_statistical_timing_derate_factors -type slack_time_based -ssta_view viewname -sta_view viewname -clock_factor 0.95
  ```
- The following example computes the clock factor for hold analysis using the slack time based approach applying a user defined data factor of 1.05:
  ```
  report_statistical_timing_derate_factors -type slack_time_based -ssta_view viewname -sta_view viewname -data_factor 1.05 -early
  ```

report_timing

```
report_timing
[-help]
[-analysis_summary_csv_extended_file <string>]
[-analysis_summary_csv_file <string>]
[-analysis_summary_file <string>]
[-check_clocks]
[-check_type {setup hold clock_gating_setup clock_gating_hold clock_gating_pulse_width
data_setup data_hold recovery removal pulse_width clock_period clock_separation skew
no_change_setup no_change_hold}]
[-debug {unconstrained | time_borrow | cppr_point}]
[-delay_limit float]
[-derate_summary]
[-gui]
[-format column_list]
[-hpin]
[-max_slack float]
[-min_slack float]
[-net]
[-output_format {text | csv | gtd | binary}]
[-path_exceptions {applied ignored all}]
[-path_group groupname_list]
[-path_type {end summary full full_clock end_slack_only summary_slack_only}]
[-retime {aocv path_slew_propagation aocv_path_slew_propagation}]
[-retime_delaycal_pins <pin_list>]
[-retime_format {manual | retime_compare | retime_replace}]
[-retime_mode {path exhaustive}]
[-skip_io_paths]
[-unique_pins]
[-view viewName]
[-worst_rc_corner]
[> <filename[.gz]>]
[>> <filename[.gz]>]
[-late | -early]
[-rise | -fall]
[-begin_end_pair | [[ -max_paths <integer>]
[-nworst <integer>]]]
[-unconstrained | -point_to_point]
[-collection | -machine_readable | -tcl_list]
[-not_through object_list | -not_rise_through object_list | -not_fall_through object_list]
[[-from pin_list | -from_rise pin_list | -from_fall pin_list]
[-clock_from clk_signame_list [-edge_from {lead trail}]]
```

```
[-through pin_list | -through_rise pin_list | -through_fall pin_list]
[-to pin_list | -to_rise pin_list | -to_fall pin_list]
[-clock_to clk_signame_list [-edge_to {lead trail}]]]
```

Generates a timing report that provides information about the various paths in the design.

The reports typically contain data on the delay through the entire path. The start node and the end node of each path is identified.

The timing report contains the following information:

- The slack times of the arriving signal at the end node
- The start node
- The associated transitions
- The signal required times and the actual signal arrival times
- Summary of propagated vs ideal status of launching and capturing clocks
- Any phase shifts applied when evaluating timing checks
- Any CPPR values applied to timing check evaluation

The `report_timing` command output accomadates both the instance pin names and net names in the same column. Since both instance pin names and net names can be long, to save screen space alternating instance pin and net names (depending on whether the row displays gate delay or net delay) are reported in a single column.

When the `timing_report_timing_header_detail_info` timing global variable is set to `extended`, the software generates timing reports with additional header information, that includes the following details:

- The analysis mode - best case worst case or on chip variation
- The active views (displayed only in MMMC mode)
- The clock reconvergence pessimism (CPPR)
- The latch slack mode
- The specified clock propagation value
- The delay calculation
- The specified delay calculation engine

When `timing_report_group_based_mode` global variable is set to `true`, paths are grouped by clock domain. In this mode, the `report_timing` command will have the following four types of groups:

- Clock gating group: This is an internal group. This group reports the clock gating check type paths.
- Asynchronous group: This is also an internal group and reports the asynchronous check type paths.
- Groups for all the clocks in the design: These are also internal groups, classified on the basis of each reference clock. Paths belonging to each reference clock are reported separately under each clock name's group. These groups do not honor the clock gating/asynchronous check types. Suppose there are three clocks C1, C2, and C3 in the design, then three groups will be reported for each of these clocks. The endpoint sharing the same reference clock will be part of the same group. In case the endpoint belongs to multiple groups then it will be reported separately in each group.
- Default group: This is an internal group - for combinational max delay or min delay paths.
- User-Defined Groups: These are user-defined path groups. These are created by users with single or multiple constraints. These groups honor all the check types. These groups are now reported by default.

By default, the `timing_report_group_based_mode` global variable is set to `false`.

Note: The global does not honor user-defined path groups so the paths will still be grouped by different clocks.

In this mode, the following commands will have the same behavior and can be used interchangeably:

`report_timing -path_group` *`Clock_Name`*

`report_timing -clock_to` *`Clock_Name`*

The `report_timing` command supports multi-threaded reporting. You can make the following settings to enable multi-threaded reporting:

`setMultiCpuUsage -localCpu` *`number_of_CPU`*

`set_table_style -no_frame_fix_width`

Note: When `timing_allow_input_delay_on_clock_source` global variable is enabled and the `set_input_delay` command is applied to a clock input, the specified delay value will be considered as the source clock latency for clock paths and as regular data path arrival time for clock source paths.

This page provides the following `report_timing` details:

- Parameters
- Use of Informational Flags
- Reporting Timing Checks Between Various Types of Clocks

- Retime Analysis Report Header Description
- SPD (Sequential Path Delay) Examples
- Sample Reports
- Related Information

Parameters

`{> | >>} filename`	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *filename* parameter must be the last argument in the list. The *filename* and -tcl_list parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the .gz extension to the file name. *Default:* Report is displayed on standard output without being saved. **Note:** The last argument specified in the `report_timing` command is not considered a filename.
`-analysis_summary_csv_extended_file <string>`	Generates the CSV file to be used for merging.
`-analysis_summary_csv_file <string>`	Generates the analysis summary CSV file.
`-analysis_summary_file <string>`	Generates the analysis summary file.

`-begin_end_pair`	Reports all violating paths between unique source and target register pairs. To report other paths, specify this parameter with the -max_slack parameter. Paths are not sorted by slack; instead, paths with the same source pins are reported adjacent to each other. When specified with the `-from`, `-through`, or `-to` parameters, the software only reports the unique register pair paths that match the path description. **Note:** You cannot specify this parameter with the `-max_paths`, `-nworst`, `-unique_pins`, or `-min_slack` parameters. The `report_timing -begin_end_pair` parameter will always report the paths grouped by path_groups irrespective of `timing_report_group_based_mode` global variable setting.
`-check_clocks`	Generates reports based on timing paths on the clock network instead of the standard timing to data endpoints. This report includes clock paths that end at the reference end of a check or a clock gating end point. *Default*: Data paths and all the other clock paths (for example, a clock path ending at a D pin of a register) are reported.
`-check_type` *check_type*	Reports only the paths that end at the specified timing check. The following check types can be reported when the software is in setup analysis mode: setup, skew, pulse_width, clock_period, clock_gating_setup, clock_gating_pulse_width, data_setup, recovery, clock_separation, and no_change_setup. The following check types can be reported when the software is in hold analysis mode: hold, clock_gating_hold, data_hold, removal, and no_change_hold. If the software is in simultaneous setup and hold

	analysis mode, all of the check types listed above can be reported. By default, the following checks are reported: • setup • hold • clock_gating_setup • clock_gating_hold • data_setup • data_hold • recovery • removal • no_change_setup • no_change_hold You can use the `-check_type` parameter to specify the following clock style checks: • pulse_width • clock_gating_pulse_width • clock_period • Skew • clock_separation Do not use this parameter with the `-unconstrained`, `-early`, or `-late` parameters. **Note:** The `report_timing` command does not perform skew checks by default.
`-clock_from` *clk_signame_list*	Generates reports based on source clock waveform(s). Reports only those paths whose source clocks are the clock signals in *clk_signame_list*.

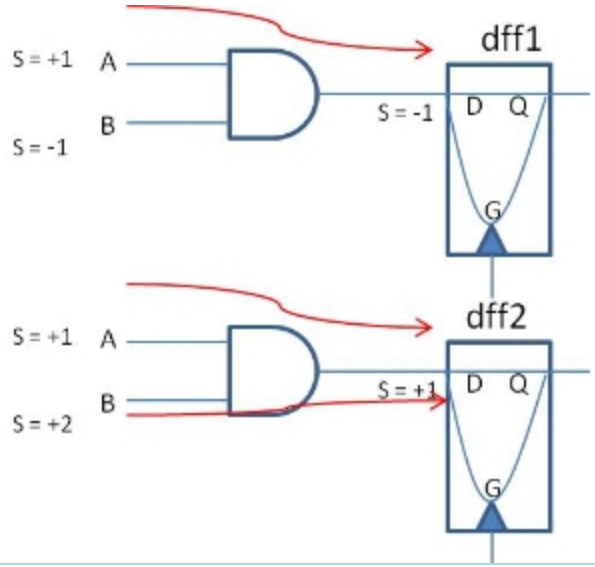
`-clock_to` *clk_signame_list*	Generates reports based on target clock waveform(s). Reports only those paths whose target clocks are the clock signals in *clk_signame_list*.
`-collection`	Returns a collection of timing paths. This is useful for performing Tcl queries on selected timing reports. To generate a collection of paths analyzed by Path Based Analysis (PBA), you can use the `-retime` parameter. **Note** : This parameter is not supported in DMMMC mode.
`-debug {unconstrained | time_borrow | cppr_point}`	You can specify the following option for debugging: • `unconstrained`: Determines the reasons for a path to be unconstrained. The report output will list all the applicable arcs/constraints that could not constrain the paths, along with the reasons. • `time_borrow`: Reports time borrowing calculations. Time borrowing is the amount of time borrowed by a previous logic. • `cppr_point`: Reports the CPPR point information for the reported paths.
`-delay_limit` *float*	Specifies the path delay limit for unconstrained paths (`-unconstrained` option). For early paths (-early option), reports only those paths with path delay less than the delay limit. For late paths (-late option) reports only those paths with path delay more than the delay limit. **Note:** The `-delay_limit` option only can be used in conjunction with the `-unconstrained` option.
`-derate_summary`	Generates an AOCV derating summary table for the launch and capture paths, in addition to the timing report.

`-early	-late`	Generates the timing report for early paths (hold checks) or late paths (setup checks). *Default*: `-late` **Note:** If you specify a check, setup or hold, using the set_analysis_mode command, the report_timing command reports that check by default.
`-edge_from {lead	trail}`	Generates reports based on source clock edge, either leading or trailing. **Note:** The `-edge_from` parameter only can be used in conjunction with the -clock_from parameter. *Default*: Generates reports based on both source clock edges
`-edge_to {lead	trail}`	Generates reports based on the target clock edge, either leading or trailing. **Note:** The `-edge_to` parameter only can be used in conjunction with the -clock_to parameter. *Default*: Generates reports based on both target clock edges
`-format` *column_list*	Formats the report according to the *column_list*. The *column_list* specifies which columns to display in the timing report and the order in which they appear. You can use the `-format` parameter to customize the reports to your needs by requesting the exact fields in which you have an interest. The valid format columns are: adjustment, abs_delay_err, abs_slew_err, annotation, aocv_adj_stages, aocv_derate, aocv_weight, arc, arrival, arrival_mean, arrival_sigma, cell, delay, delay_mean, delay_sigma, direction, edge, fanin, fanout, flags, hpin, incr_delay, instance, instance_location, latch_window, load, module, net, pct_delay_err, pct_slew_err, phase, phys_info, pin, pin_load, pin_location, power_domain, required, retime_delay, retime_delay_mean, retime_delay_sigma, retime_slew,

	retime_slew_mean, retime_slew_sigma, stage_count, slew, slew_mean, slew_sigma, socv_derate, spice_arrival, spice_delay, spice_slew, stolen, timing_point, total_derate, user_derate, voltage, when_cond, wire_load, wlmodel For more details on the 'flags' column, refer to the section on Use of Informational Flags. For example: `-format {hpin cell delay required arrival required edge}` See Table for a list of valid options. The default net format (with -net option) for the full path is: {hpin edge net cell delay arrival required}. If the `-unconstrained` option is specified, the required column is not displayed. The adjustment column is automatically added to the timing report. To disable this feature you can set the `report_timing_default_adjustment_column` global variable to `false`. You can also use a combination of the -format and -tcl_list parameters to integrate the timing reports into your Tcl scripts. You can use the `-from` parameter to limit the number of paths reported, and to find specific paths in the design. The `-format` option cannot be used with the `-path_type end` or `-path_type summary` options. You can change the re-timing columns by using the `timing_report_retime_formatting_mode` global variable.
`-gui`	Displays the timing report in a GUI window.

{-from | -from_rise | -from_fall} *pin_list*	Reports paths starting from the object(s) specified in the *pin_list*. The supported objects are pins and ports. Using -from_rise (or -from_fall) specifies that the rising (or falling) edge of the signals on the pins/ports in the *pin_list* are the start of the paths. You can use this option with the -through and -to parameters for specifying particular paths in the design. **Note** : These parameters do not allow clock objects. You can use the -clock_from and -clock_to parameters to reference clock waveforms.
-hpin	Prints all hierarchical crossings of an arc under hpin column in report_timing table.
-machine_readable	Generates detailed timing report in machine-readable format. This report is used for debugging timing results using the timing debug feature.
-max_paths *integer*	Reports the specified number of worst paths in the design, with maximum of nworst (-nworst) paths to any single endpoint (default 1). In case of group based mode (when timing_report_group_based_mode is set to true), the -max_paths parameter specifies the number of worst paths reported per clock group. *Default*: 1

`-max_slack` *float*	Reports only those paths with slack less than the value of the *float*. The `-max_slack` option limits the report to paths that fall into the specified range. A positive slack value indicates that timing was met. A negative value for slack indicates a timing violation. The `-max_slack` option cannot be used with the -unconstrained option. When the `-retime_mode exhaustive` option is specified, the `-max_slack` is set to `0` by default.
`-min_slack` *float*	Reports only those paths whose slack is greater than the value of float. The `-min_slack` option cannot be used with the `-unconstrained` option. You can generate a timing report showing paths with slack greater than the specified slack by using the `-max_paths` option. For example, to report all paths with slack greater than 2ns, you can issue the following command: `min_slack 2.0 -max_paths 1000` In this case the maximum number of paths reported will be 1000. When the `-min_slack` parameter is specified, the total numbers of paths processed will be as determined by the `max_path`s/`nworst` settings and only those paths satisfying the `-min_slack` criteria will be reported. The software will not process paths beyond the `max_paths`/`nworst` limit to find other paths satisfying the `-min_slack` limit. In the below diagram, if `-min_slack` is 0 and `-max_paths 4 -nworst 2` is specified, then only 3 paths will be reported; path with slack value of -1 will be ignored. Here, even if there is another path with slack that is greater than or equal to 2, then it will not be reported because worst max_paths have been processed.

	
`-net`	Adds a row for the net arc. This parameter also separates the cell delay from the wire delay. *Default*: The net arc is not shown, and the net delay is added to the following delay. **Note**: The report_timing -net command displays the net delays separate from the cell delays. However, the net delay is sometimes shown as 0.0.To get the net delays to show up, increase the report precision using the following global: `set_global report_precision 5`

{-not_through | -not_rise_through | -not_fall_through} *object_list*	-not_through: Reports paths that do not traverse through the specified nets, ports, or pins of a cell. By default the parameter will be ignored when an object has already been specified. In this case the parameter shows the following behavior: • If a collection of all the input ports has been specified (using the all_inputs command), then the report will show paths to the inout part, and paths starting from the internal part of inout will be excluded. • If a collection of all output ports has been specified (using the all_outputs command), then the report will show paths starting from the internal part of inout, and paths ending at inout part will be excluded. • If any other collection (not specified using the all_inputs and all_outputs commands) is specified, then paths to both the parts will not be reported.
	-not_rise_through: Excludes paths with rise transition.
	-not_fall_through: Excludes paths with fall transition.

`-nworst` *integer*	Specifies the maximum number of paths which can be reported for a particular endpoint (the total number of paths reported is specified by the `-max_paths` parameter). In case of group based mode (when `timing_report_group_based_mode` global variable is set to `true`), the `-nworst` parameter specifies the maximum number of paths for a particular endpoint per clock group. You can use the `-nworst` option to report all the checks at an endpoint or use the `-check_type` option to report a specific check. *Default*: 1
`-output_format {text | csv | gtd | binary}`	Generates a detailed timing report in the specified format - text, CSV, GTD, or binary.

`-path_exceptions {applied | ignored | all}`	Includes information of path exceptions applied and considered for the reported path. If multiple from, through, or to pins are specfied, the exception report for that path will show only those pins that are part of the current path. **Note:** You cannot use this parameter with the following report_timing parameters: • `-point_to_point` • `-collection` • `-path_type end | end_slack_only | summary | summary_slack_only` • `-tcl_list` In addition, you cannot use this parameter with block-based SSTA. **Note**: If a path is already unconstrained, then all further path exceptions applied on that particular path will be reported as ignored exceptions. The `-path_exception` parameter also reports -start/-end options with multi-cycle paths. Without start or end specification, by default, the setup check is relative to the end clock and the hold check is relative to the start clock. The `report_timing -path_exception all` command will report all the non-default -start/-end specifications.
	applied: Reports the honored exception on the reported path.
	ignored: Reports the ignored exceptions on the reported path.
	all: Reports both applied and ignored set of exceptions on the reported. **Note:** Analyzing all the exceptions for a path will cause increase in runtime.

`-path_group` *groupname_list*	Reports only paths contained in the groups specified in *groupname_list*. Report paths belonging to the default path group by using -path_group default. The paths that do not belong to any user-specified path group belong to the default path group named default. In group-based mode, the internal groups - clock gating, clock names, asynchronous - and user defined path groups can be selected using the `-path_group` parameter.
`-path_type {end | summary | full | full_clock | end_slack_only | summary_slack_only}`	The `path_type` option lets you choose the format of the report by path type. The default format, if the `-path_type` option is not specified is `full`. You can choose one of the following options: • `end`: Generates an end point report for each path consisting of an endpoint, cause, slack, arrival time, required time, and phase. This option generates a very fast report. The -format option does not have any impact on this report format. • `summary`: Generates a summary report for each path consisting of a start point, endpoint, cause, slack, arrival time, required time, and phase. The `-format` option does not have any impact on this report format. • `full`: Generates a report that displays the full path with accompanying required time and slack calculation. This is the default path type. Control the report format using the `-format` option. • `full_clock`: If the path reported ends at a timing check, this option also reports the full clock path (Other End Path) in addition to the full data path (Timing Path). • `end_slack_only`: Generates an end point report, similar to `-path_type end` report, for each path

	consisting of an endpoint, slack and cause (as Violated/Met). This option generates a very fast report. The `-format` option does not have any impact on this report format. **Note**: When the `-path_type end_slack_only` option is used, the path collections will not be reported. • `summary_slack_only`: Generates a summary report, similar to `-path_type summary`, for each path consisting of a start point, endpoint, slack and cause (as Violated/Met). The `-format` parameter does not have any impact on this report format. In MMMC mode, when you specify the `report_timing -path_type` parameter (with `end`, `end_slack_only`, `summary`, or `summary_slack_only` options), the output report displays a view column for all the reported paths. **Note**: The paths contained in a collection are not reported with upgraded path type.
`-point_to_point`	Traces the worst delay path between a pair of from-to pins. Paths reported with this parameter only include the cumulative delay of the path. You must specify either the -from or -to parameter, to define the begin and end points of the path. If you do not specify a `-from` pin, the standard definition of the begin point of a path applies. If you specify a `-from` pin, it is treated as the begin point of the path. If you specify a -from pin that is not a valid begin point, the arrival time of the first pin starts from 0. If you do not specify a `-to` pin, the standard definition of the end point applies. If you specify a `-to` pin, it is treated as the end point, and only those paths ending in -to pins are reported. If you specify

	more than one -to pin, the worst path to each pin-to-pin is reported. When from-to pins are specified, the software will trace across the trigger arcs and report a path, if there is a clock reaching the CK pin of the trigger arc. However, if there is a begin or end point present between the `-from` and `-to` pins, the software will not trace beyond it. If there is no clock reaching the CK pin of the trigger arc, the software cannot trace across the trigger arc, and no path is reported. Additionally, clock path delay is included only when the clock is in propagated mode. The `-point_to_point` parameter can be used with the `-max_paths` and `-nworst` parameters. **Note:** The `-point_to_point` and `-through` parameters are mutually exclusive; you cannot specify them together. **Note**: The `-point_to_point` parameter supports a single `-from` and `-to` pin; a list of pins is not supported. See Sample Reports.
`-retime_delaycal_pins <pin_list>`	Reports the PBA delay calculation details for timing arcs that fanin to the user-specified to-pin list. You can specify multiple pins in the pin list. The delay calculation report will be printed at the end of each timing path for the arcs, which are retimed in that path. The PBA delay calculation report format is similar to the `report_delay_calculation` output in GBA mode, however, the results may be vary because this output is based on path-based analysis. When a cell output pin list is provided, the generated report is for the cell arcs; the cell input pin list generates the net arc report. This parameter is supported for PBA reporting only.

`-retime_format {manual | retime_compare | retime_replace}`	Allows you to set the format of timing reports for retime related fields. You can specify one of the following options: • `manual`: Does not change the `report_timing` format. • `retime_compare`: If GBA columns are present in the report format, then the corresponding retime options are added. If PBA columns are already present, then there is no change in the format. • `retime_replace`: If GBA columns are present in the report format, then the GBA columns are replaced by the corresponding PBA columns. If PBA columns are already in the report, then there is no change in the format. **Note**: The timing report format is also controlled using the `timing_report_retime_formatting_mode` global variable. The `report_timing` command setting has precedence over this global variable.

`-retime_mode {path | exhaustive}`	Reanalyzes the specified set of paths using the specified method. You can use one of the following options: • `path`: Reanalyzes the specified set of paths. By default, the software uses `-retime_mode path` setting, when the `-retime path_slew_propagation` option is specified. • `exhaustive`: Enables exhaustive path-based analysis mode. In this mode, the software will exhaustively examine nworst paths of the selected endpoints and then select the true worst path to be reported - based on retimed slacks of all the examined paths. When you use this option, reporting can be run time intensive as the software needs to do an exhaustive search for true worst paths. Therefore, it is recommended to use this option in the later phases of the sign-off stage when there are very few violations left for analysis. *Default*: `path`
`-retime {aocv | |path_slew_propagation | aocv_path_slew_propagation}`	Reanalyzes the specified set of paths using path based analysis (1 path per endpoint). **Note:** This feature is not meant to be run on all the paths of a design.

	aocv: Computes AOCV stage count based on specific paths and applies the AOCV derate values to GBA (graph-based analysis) delays. The `-retime aocv` option can be used with the following parameters only: • `-clock_from` • `-edge_from` • `-clock_to` • `-edge_to` • `-rise/-fall` • `-early/-late` • `-check_type` • `-max_paths` • `-begin_end_pair` • `-from/-from_rise/-from_fall` • `-through/-through_rise/-through_fall` • `-to/-to_rise/-to_fall`

	• `-path_group` • `-path_exceptions` • `-net` • `-path_type` • `-max_slack` • `-min_slack` • `-view` • `-format` *`column_list`* • `-derate_summary` • `-machine_readable` • `-nworst`
	`path_slew_propagation`: Re-evaluates the given set of graph-based analysis (GBA) paths by recalculating the delay values based upon actual propagated slew across the path. In this case the AOCV derates are the ones that are reported during graph-based analysis. The default timing analysis uses worst-case slew propagation to calculate the delays at each stage. As a result, timing analysis results can be pessimistic. **Note:** This feature should be run only on paths that are not meeting the slack margin criteria, to help identify actual failing paths from those that are simply failing due to the conservative delay calculation. Signal integrity incremental delays can be recomputed for the entire path. You can use the following command to perform path based signal integrity analysis: `report_timing -retime path_slew_propagation` In the report_timing command output, the Retime

	Incr Delay column reports recomputed signal integrity delays. To view this column, specify the `report_timing -format retime_incr_delay` option or use `set_global report_timing_format` timing global variable. The `-retime` option also supports retiming a collection of paths. This option does not retiming an already retimed path collection. This feature works in graph-based analysis (GBA) mode for path collections. You can use the following use model to retime collections: `report_timing $collection -retime path_slew_propagation` A path collection stores context dependent data, that is, a delay property in GBA will reflect delay data of the timing path report generated in GBA mode. The same property in PBA will reflect the re-time delay data of the timing path report in PBA mode. The delay column in timing reports is also context dependent - under the `timing_report_retime_formatting_mode retime_replace` global variable. **Note**: For memory optimization purposes, both the delays will not be stored in the same path collection. You can use this setting for comparison purposes.
	`aocv_path_slew_propagation`: Re-evaluates the given set of graph-based analysis (GBA) paths by using path-based AOCV derates and delays using actual path- based propagated slew. This option is a combination of `-aocv` and `-path_slew_propagation`. For retime report header details, see Retime Analysis Report Header Description.
`-skip_io_paths`	Specifies whether the input/output paths are to be ignored.

`-rise	-fall`	Reports the path with the specified edge on the endpoint. If an endpoint is specified using -to_rise (or -to_fall) option, the -rise (or -fall) option is ignored and paths with edge specified by `-to_rise` (or `-to_fall`) are reported. The `-rise` option applies to end point only.
`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The -tcl_list and *filename* parameters are mutually exclusive; you cannot specify them together.
`{-through	-through_rise	-through_fall}` *pin_list*	Reports paths that pass through pin(s) or net(s) specified by the *pin_list*. Any number of -through pins or nets can be specified. Using -through_rise (or `-through_fall`) specifies that the paths go through the rising (or falling) edge of the signals on the pins/nets in *pin_list*. The *pin_list* is a logical OR function. The resulting path may pass through any of the pins or nets in the -through *pin_list*. To force the report to pass through multiple pins or nets, separate -through statements are needed. You can use these parameters with the `-net` option.
`-to	-to_rise	-to_fall` *pin_list*	Reports paths leading to the pin(s) specified by the *pin_list*. Pins in the *pin_list* can be either pins on the design boundary (ports) or pins on an instance. Only one list of -to pins can be specified per report. Using -to_rise (or -to_fall) specifies that the rising (or falling) edge of the signals on the pins in *pin_list* are at the end of the paths. The `-to_rise` option applies to end point as well as intermediate pins.

`-unconstrained`	Reports only the unconstrained paths (paths with no slack). Each signal arriving at the path end node which does not have a matching required time, results in an unconstrained path. The `report_timing` command without the `-unconstrained` parameter reports only constrained paths. If no constrained path is found, there may be unconstrained paths or the path may not exist. **Note**: The `-min_slack` and `-max_slack` cannot be specified with the `-unconstrained` parameter. See Sample Reports for the conditions under which a path is reported as constrained or unconstrained.
`-unique_pins`	Reports paths through unique set of pins. When this parameter is specified, in group-based mode for a given begin-end point pair, the software will consider all different launch clock paths for reporting unique paths. If for a given begin-end pair and a given capture clock, there is a possibility of different launch clock paths, then each such path will be reported. **Note**: Using this option may significantly affect runtime and memory usage if the number of paths to be reported is high.
`-view {`*`viewName`*`}`	Generates report for the specified analysis view. You must be in multi-mode multi-corner analysis mode in order to use this parameter. To create a view, you use the `create_analysis_view` command. To set an active view for analysis, you use the `set_analysis_view` command. *Default:* Reports the worst path to each endpoint across all analysis views. The report_timing command to performs block based analysis in MMMC mode. By default, the command reports the worst end-point(s) across all views. You can use the -view parameter to report for individual views.

-worst_rc_corner	Identifies worst interconnect parameter combination that leads to minimum slack (desired) and generates a report for the corner.

Use of Informational Flags

Informational tags are used providing additional details. The following informational tags are supported:

Flags	Description
(i)	Net is ideal
(P)	Instance is preserved / don't_touch
(p)	Instance is preserved / don't_touch but may be resized

The output format uses the flag keys on the timing report and adds a description at the end of the report for reference. A sample output is as shown below:

```
#-------------------------------------------------------------------------
# Timing Point  Delay (ns) Latch  Window   Arrival   Arc   Flags   Adjustment
# (ns)
#-------------------------------------------------------------------------
f0/CP           -          -    1.000000            CP       -       -
f0/Q            0.254100   -    1.254100            CP->Q    -       -
b32/Z           0.598446   -    1.852546            I->Z     -       -
...
b4/Z            0.086940   -    2.502181            I->Z     -       -
f1/D            0.000000   -    2.502181            D        <<<     -
#-------------------------------------------------------------------------

(P) : Instance is preserved
(p) : Instance is preserved but may be resized
(i) : Net is ideal.
(b) : Timing paths are broken.
```

Report Timing- Column List Options

Option	Description
adjustment	Reports generated clock adjustment values. When reporting the detailed path for a generated clock, there are circumstances where the arrival time needs to be adjusted when a `create_generated_clock` assertion is encountered along the path. The adjustment argument can be used to provide a clearer indication of why a "jump" in the arrival time is seen in the path report. When the `create_generated_clock -edge_shift` parameter is specified, the adjustment column will show the amount of shift specified.
abs_delay_err	Reports the absolute delay difference of Tempus from Spice.
abs_slew_err	Reports the absolute slew difference of Tempus from Spice.
annotation	Reports RC parasitic annotations for net arcs, and delay annotations for either gate or net arcs. The following annotations are reported: • SDF: SDF back-annotation. The annotation status reflects the gate arc, not the interconnect arc. • SPEF: This entry appears if parasitic extraction has been done or annotated using SPEF. • LumpedRC: Lumped RC annotation • DlyAssert: Assertion set with set_annotated_delay command • WLM: Wire load model back-annotation • <none>: All others
aocv_adj_stages	Shows up "Aocv Adjust Stages" column. This column reports the AOCV adjustment stages that indicate the total effective stage count.

aocv_derate	Reports the graph-based AOCV derating factor used for cells/nets in AOCV mode. In retime mode, this column will report graph-based derating for cells/nets before the AOCV branch point and path-based derating.
aocv_weight	Reports AOCV weight values for the path elements.
arc	Reports the arc as described by the from pin, from pin edge, to pin, and to pin edge. For example, the arc from the rising edge of pin A to the falling edge of pin Z is reported as `A -> Z`.
arrival	Reports the arrival time on the pin.
arrival_mean	Reports mean of the arrival time on a pin.
arrival_sigma	Reports standard deviation of the arrival time on a pin.
cell	Reports the cell name of the given pin's instance.
delay	Reports the arc delay. If the stolen slack at the arc output pin (output of transparent latches) is not zero, the stolen column is also displayed along with this column.
delay_mean	Reports the mean of each arc's delay.
delay_sigma	Reports the standard deviation of each arc's delay. **Note**: In the absence of correlated and uncorrelated components of variation, the individual delay sigma values will not be summed up.
direction	Reports the pin direction (IN, OUT).
edge	Reports the edge of a pin (^=R, v=F).
fanin	Reports the number of source nodes of a net connected to a timing pin.
fanout	Reports the number of sinks of a net connected to a timing pin.

flags	Indicates the size_only, dont_use, dont_touch, and ideal attributes. When the 'flags' option is specified, a capacitance group entry (if it exists) will be displayed under the "Flags" column. This value indicates the input capacitance group being used for delay calculations – C1C2 (2-piece model) or C1CN (n-piece model).
hpin	Reports the hierarchical name for a given pin. **Note**: The hpin, pin, and timing_point will print the same values in the report output.
incr_delay	Reports the incremental delay of a pertaining arc. The software reads the incremental delay from an incremental SDF file using the `read_sdf` command.
instance	Reports the hierarchical name of a given pin's instance.
instance_location	Reports the location (x,y) of an instance. If a design is not placed, this column will be blank. This option displays the `instance` column automatically.
latch_window	Reports the available window for latch to sample the data.
load	Reports the total capacitance load on a given pin.
module	Reports the module name of the corresponding block hierarchy. This hierarchical module name will be reported only when the `-hpin` parameter of the `report_timing` command is specified.
net	Reports the hierarchical name of a net connected to a given pin.
pct_delay_err	Reports the percent delay difference of Tempus from Spice.
pct_slew_err	Reports the percent slew difference of Tempus from Spice.

phase	Reports the phase name of a pin.
phys_info	Shows minimum/maximum assigned layer, and non-default rule name information.
pin	Reports the reference name for the given pin.
pin_load	Reports the pin load data. The pin load refers to the sum of pin capacitance of all the pins connected to the current net.
pin_location	Reports the location (x,y) of a pin. If a design is not placed, this column will be blank.
power_domain	Reports power domain details of the specified path instance.
required	Reports the required time on a pin.
retime_delay	Reports the recalculated arc delay based on the path being reanalyzed with the specified retime method.
retime_delay_mean	Reports the mean of the recalculated arc delay on the path being reanalyzed with the specified retime method.
retime_delay_sigma	Reports the standard deviation of each parameter of an arc's retimed delay.
retime_slew	Reports the recalculated slew based on the path being reanalyzed with the specified retime method.
retime_slew_mean	Reports the mean of each parameter of an arc sink pin's retimed slew.
retime_slew_sigma	Reports the standard deviation of each parameter of an arc sink pin's retimed slew.
stage_count	Displays the aggregate stage count of a specific instance in graph-based analysis (GBA) mode, and the actual stage count in path-based analysis (PBA) mode.
slew	Reports the propagated slew at the given pin.

slew_mean	Reports mean of each parameter of an arc sink pin's slew.
slew_sigma	Reports the standard deviation of each parameter of an arc sink pin's slew.
socv_derate	Reports the SOCV or the spatial derating factor applied on the cells or nets in SOCV analysis mode.
spice_arrival	Reports the Spice arrival time at the timing point.
spice_delay	Reports the Spice delay of a timing arc.
spice_slew	Reports the Spice slew of a timing point.
stolen	Reports the slack stolen (or the time given to the previous stage) at the given pin. If the slack is not zero and the `delay` column is specified, then this column is displayed by default (but only visible on the output of transparent latches).
timing_point	Reports the hierarchical names for the given points.
total_derate	Reports the total derating factor applied on the corresponding cell or nets that reflect the combined impact of all the derates (OCV/AOCV/spatial) applied on each stage of the timing path.
user_derate	Reports the derating scale factors set with the `set_timing_derate` command. **Note:** The timing system cannot provide an accurate derating factor for the lumped gate and wire delay when the `set_timing_derate` factors for `-cell_delay` and `-net_delay` are not equivalent. In this case, an asterisk (*) is reported in the SSI Derate column.
voltage	Returns the voltage that is used during delay calculations.
when_cond	Reports the "when" condition of the arc specified in the library.

wire_load	Reports the wire load.
wlmodel	Reports the kind of parasitic used by a pin or net, such as SPEF. If the wire load model is derived from a Liberty file, the information is reported in the following format: `Wireload model (.lib model_name library_name)`

Reporting Timing Checks Between Various Types of Clocks

The following table summarizes the timing checks and the min/max delay checks that are performed based on the types of launching and capturing clocks:

- NZ (Non Zero Period Clock - Regular Clock)
- Z (Zero Period Clock - Regular clock whose period is equal to zero)
- @ (Asynchronous Clock - Clock which triggers the default input delay and the input delay without a clock)
- CPD (Combinational Path Delay)

Launching Clock	Capturing Clock	Timing Checks	Path Delay Checks
@	@	NO	CPD
Z	@	NO	CPD
NZ	@	NO	CPD
@	Z	YES	SPD
@	NZ	YES	SPD
NZ	Z	NO	SPD
Z	NZ	YES	SPD
Z	Z	YES	SPD
NZ	NZ	YES	SPD

Retime Analysis Report Header Description

AOCV(default)	Indicates that the selected retime analysis includes recalculation of path depths and AOCV derates. The AOCV retime is enabled with the `-retime aocv` or `aocv_path_slew_propagation` parameter. The AOCV stage counting mode (set using the `timing_aocv_analysis_mode` global variable) is indicated in the parenthesis. The stage counting modes are given below: • `launch_capture(default)` `(Default)` • `combine_launch_capture` `(Combine)` • `separate_clock_data` `(Separate)` • `clock_only` `(Clock)`
Corr	Indicates that the current SOCV accuracy mode considers correlation of variation between stages. Starting with SOCV accuracy mode `medium` (specified using `setDelayCalMode -socv_accuracy_mode` parameter), it is assumed that some portion of the variation is not completely uncorrelated random variation, so the software tracks a correlated and uncorrelated component for each of the applicable moments.
Data Clock	Indicates the portion of the path that has been retimed. By default, only the data path is retimed. When the `timing_disable_retime_clock_path_slew_propagation` global variable is set to `false`, then the clock path will also be retimed, and “Clock” will be added to the retime status.
GP	Indicates that graph-based PBA mode settings are enabled.

GS	Indicates that GBA skewness mode settings are enabled.
Path-Slew	Indicates the selected retime analysis is set to use path correct slew rates to reduce the pessimism from slew merging. This can be controlled with the `report_timing -retime` parameter. The path slews are recalculated in `path_slew_propagation` and `aocv_path_slew_propagation` modes.
RC	Indicates that RC variation is enabled with the `timing_socv_rc_variation_mode` setting, and an SOCV format RC variation side file library is available.
SI	Indicates that SI analysis was enabled during the retime analysis. The path-based analysis will recompute SI effects using path-correct victim timing windows.
SOCV(Ultra)	Indicates that SOCV analysis is enabled and has been incorporated into the retiming. The SOCV analysis mode (specified using the `setDelayCalMode -socv_accuracy_mode` parameter) is shown in parenthesis. The accuracy modes are mapped as follows: • `low` `<not shown>` • `medium` `Medium` • `high` `High` • `ultra` `Ultra`

Spatial (CS)	Indicates that spatial derating side files have been loaded and an explicit spatial derate factor will be calculated. This is not shown for AOCV 2D derate tables, where the spatial and depth derates are combined. The GBA spatial derate mode (specified with the `timing_spatial_derate_distance_mode` setting) is shown in parenthesis with the possible values below: • `chip_size (CS)` • `bounding_box (BB)` The GBA spatial derate mode does not affect the path-based analysis bounding box calculation.
WP	Indicates that waveform propagation is enabled (`setDelayCalMode -equivalent_waveform_model propagation` setting). The path-based retime will also use this delay calculation accuracy setting when recomputing delays and transitions for the path.

SPD (Sequential Path Delay) Examples

- The following command reports paths leading to the pin(s) specified by the *pin_list* :

```
report_timing -to u55/D


Path 1: MET Setup Check with Pin u55/CK
Endpoint: u55/D (^) checked with leading edge of 'CLK_W_4'
Beginpoint: u55/QN (^) triggered by leading edge of 'CLK_W_4'
Path Groups: {CLK_W_4}
Other End Arrival Time 0.152
- Setup 0.306
+ Phase Shift 10.000
= Required Time 9.845
- Arrival Time 1.902
= Slack Time 7.943
Clock Rise Edge 0.000
```

```
+ Clock Network Latency (Prop) 0.152
= Beginpoint Arrival Time 0.152
-------------------------------------------------------------------------
Pin      Arc            Cell      Delay     Arrival    Required    Instance
                                              Time      Time
-------------------------------------------------------------------------
            -
u55/CK    CK ^           -       - 0.151    8.094                  u55
u55/QN    CK ^ -> QN ^   DFF      1.750     1.901       9.844      u55
u55/D ->  D ^            DFF      0.001     1.902       9.845      u55
-------------------------------------------------------------------------
```

- The following command shows a report similar to the first with the addition of net arc information.

```
report_timing -to u55/D -net

Path 1: MET Setup Check with Pin u55/CK
Endpoint: u55/D (^) checked with leading edge of 'CLK_W_4'
Beginpoint: u55/QN (^) triggered by leading edge of 'CLK_W_4'
Path Groups: {CLK_W_4}
Other End Arrival Time 0.152
- Setup 0.306
+ Phase Shift 10.000
= Required Time 9.845
- Arrival Time 1.902
= Slack Time 7.943
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.152
= Beginpoint Arrival Time 0.152
-------------------------------------------------------------------------
Pin        Arc           Cell    Delay    Arrival    Required    Instance
Time Time
-------------------------------------------------------------------------
u55/CK     CK ^           -      -        0.151     8.094       u55
u55/QN     CK ^ -> QN ^   DFF     1.750    1.901     9.844       u55
u55/D ->   -              DFF     0.001    1.902     9.845       u55
-------------------------------------------------------------------------
```

- The `report_timing` command also accepts `-collection` as an argument. For example,

```
report_timing [report_timing -collection]
```

```
set rpt [report_timing -collection]
report_timing $rpt
```

- The following command displays the worst late path in the design. The format of the report is similar to the first example:
  ```
  report_timing
  ```
- The following command displays the worst late path to each violating endpoint that has a slack less than -1.0:
  ```
  report_timing -max_slack -1.0
  ```
- The following command displays all the late paths that end at port `out[2]` and that have negative slack up to a maximum of 1000 worst paths. If there are more than 1000 paths, only the 1000 worst paths are reported:
  ```
  report_timing -to out[2] -max_paths 1000 -max_slack 0.0
  ```
- The following command reports the worst late path that starts at `in[0]` and ends at `out[1]`:
  ```
  report_timing -from in[0] -to out[1]
  ```
- The following command displays the three worst paths that start at `in[1]` and end at `out[3]`. With reconvergent fanout, more than one path may exist:
  ```
  report_timing -from in[1] -to out[3] -max_paths 3
  ```
- The following command displays the ten worst paths. Only the paths between the specified pins are enumerated.With reconvergent fanout, more than one path may exist: `report_timing -from i102/Z -to i123/A -max_paths 10`
- The following command enumerates the worst ten paths through the given two pins, starting at the beginning points in the design and ending at the endpoints, similar to using the `-from` option: `report_timing -through i102/Z -through i123/A -max_paths 10`
- The following command reports the ten worst paths through blocks A and C, or blocks B and C. The path only has to satisfy one element in a through list, but all through lists must be satisfied. The example can be thought of as ((A or B) and C):
  ```
  report_timing -through {A/*B/*} -through {C/*} -max_paths 10
  ```
- The following command forces a path through pin A and pin B. Use the syntax shown in the example. Do not use `-through {A B}`:
  ```
  report_timing -through A -through B
  ```
- The following command displays the clock path through the clock root to Launch flop: `report_timing -through aBC_bs/bs_mex1/r2_00_q1_reg_10_/CK -to .... -path_type`

```
full_clock
```

- You will not see the path for the launch clock if you specify the following command because the report_timing command starts reporting from the CK pin:

```
report_timing -from aBC_bs/bs_mex1/r2_00_q1_reg_10_/CK
```

- The first command displays the path leading to the `CK` pin.
- The following command shows the `-unconstrained` option:

```
report_timing -unconstrained

Path 1:Endpoint: OUT4 (^) (unconstrained output)
Beginpoint: u55/Q (^) triggered by leading edge of 'CLK_W_4'
Arrival Time 0.365
+ Clock Network Latency (Prop) 0.150
= Beginpoint Arrival Time 0.150
-------------------------------------
Load Slew Delay Arrival Cell Arc Pin
Time
-------------------------------------
0.013 0.116 - 0.150 - CK ^ u55/CK
0.011 0.072 0.215 0.365 DFF CK ^ -> Q ^ u55/Q
0.011 0.131 0.000 0.365 - OUT4 ^ OUT4 ->
-------------------------------------
```

- The following command shows the `-unconstrained -path_type end` report:

```
report_timing -nworst 5 -unconstrained -path_type end -format {pin arrival phase}

-------------------------------------------------------------------
Path No. Pin           Cause               Arrival  Phase
-------------------------------------------------------------------
1        seg3/u9/CK ^  Unconstrained Path  0.682    CLK_W_3(D)(P) *
2        seg3/u9/CK ^  Unconstrained Path  0.682    CLK_W_3(D)(P) *
3        seg3/u3/CK ^  Unconstrained Path  0.682    CLK_W_3(D)(P) *
4        seg3/u3/CK ^  Unconstrained Path  0.682    CLK_W_3(D)(P) *
5        seg2/u9/CK ^  Unconstrained Path  0.682    CLK_W_2(D)(P) *
-------------------------------------------------------------------
```

- The following command sets a false path from u99/Q to u55/D:

```
set_false_path -from u99/Q -to u55/D
```

```
set_false_path -through u4/Y
report_timing -from u99/Q -to u55/D -unconstrained -path_exceptions all

Path 1:Endpoint: u55/D (^) (unconstrained output)
Beginpoint: u99/Q (v) triggered by leading edge of 'CLK_W_4'
Arrival Time 0.827
+ Clock Network Latency (Prop) 0.150
= Beginpoint Arrival Time 0.150
------------------------------------------------------------
Load    Slew    Delay   Arrival   Cell   Arc   Pin
                        Time
------------------------------------------------------------
0.013   0.116   -        0.150    -     CK ^        u99/CK
0.027   0.086   0.195    0.345    DFF   CK ^ -> Q v u99/Q ->
1.006   1.140   0.482    0.827    INV   A v -> Y ^  u4/Y
1.006   1.140   0.000    0.827    DFF   D ^         u55/D ->
------------------------------------------------------------
Applied exceptions:
---------------------------------
Through Late
---------------------------------
u4/Y false
---------------------------------
Ignored exceptions:
---------------------------------
From To Late
---------------------------------
u99/Q u55/D false
---------------------------------
```

- For hold paths, if the path has a setup multi-cycle path then those exceptions will also be reported because hold slacks get impacted by inferred path exceptions. For example:

```
set_multicycle_path 2 -setup -from IN1

Path 1: VIOLATED Hold Check with Pin seg1/u3/CK
```

```
Endpoint: seg1/u3/D (v) checked with leading edge of 'CLK_W_1'
Beginpoint: IN1 (v) triggered by leading edge of '@'
Path Groups: {CLK_W_1}
Other End Arrival Time 0.682
+ Hold -0.034
+ Phase Shift 0.000
- Cycle Adjustment -20.000
= Required Time 20.648
Arrival Time 0.117
Slack Time -20.531
Clock Rise Edge 0.000
+ Input Delay 0.000
+ Drive Adjustment -0.014
= Beginpoint Arrival Time -0.014
-------------------------------------------------------------
Load    Slew    Delay   Arrival   Cell   Arc           Pin
                        Time
-------------------------------------------------------------
0.013   0.029   -      -0.014    -       IN1 v         IN1
0.013   0.057   0.054  0.040     CLKBUF  A v -> Y v    seg1/u1/Y
0.003   0.050   0.077  0.117     CLKBUF  A v -> Y v    seg1/u2/Y
0.003   0.050   0.000  0.117     DFF     D v           seg1/u3/D
-------------------------------------------------------------
Applied exceptions:
---------------------------------
From Early Late
---------------------------------
IN1 - cycles 2
---------------------------------
```

- The following command displays the timing report for analysis view `setup_view1`:

```
report_timing -view setup_view1

Path 1: MET Late External Delay Assertion
Endpoint: EJ_TDOzstate (^) checked with leading edge of 'virt_out_EJ_TCK'
```

```
Beginpoint: core_ejt_ejt_tap_ejt_tck__EJ_TDOzstate_q_reg_0_/Q (^) triggered by
trailing edge of 'EJ_TCK'
Path Groups: {virt_out_EJ_TCK}
Analysis View: setup_view1
Other End Arrival Time 0.000
+ Source Insertion Delay 0.300
- External Delay 1.250
+ Phase Shift 17.400
+ CPPR Adjustment 0.000
= Required Time 16.450
- Arrival Time 9.318
= Slack Time 7.132
Clock Fall Edge 8.700
+ Clock Network Latency (Ideal) 0.000
= Beginpoint Arrival Time 8.700
------------------------------------------------------------------
Load   Slew   Delay   Arrival   Cell       Arc             Pin
                       Time
------------------------------------------------------------------
0.010  0.000          8.700                CP ^            core_ejt_ejt
0.118  0.145  0.127   8.828      DFSVTD4   CP ^ -> Q ^     core_reg_0_/Q
0.525  0.438  0.411   9.238      BUFVTD6   I ^ -> Z ^      FE_OFC260_/Z
0.525  0.474  0.080   9.318                EJ_TDOzstate ^  EJ_TDOzstate
------------------------------------------------------------------
```

Sample Reports

- `report_timing -to u55/D -from u99/Q -point_to_point` #when -from is intermediate begin point:

```
Path 1:Endpoint: u55/D (^) (unconstrained output)
Beginpoint: u99/Q (v) triggered by leading edge of 'CLK_W_4'
Arrival Time 0.482
------------------------------------------------------------
```

```
Load   Slew    Delay   Arrival   Cell   Arc        Pin
                       Time
-------------------------------------------------------------
0.027  0.086           0.000             Q v        u99/Q ->
1.006  1.140  0.482    0.482       INV   A v -> Y ^ u4/Y
1.006  1.140  0.000    0.482       DFF   D ^        u55/D ->
-------------------------------------------------------------
```

- The following command recalculates the specified critical paths using the AOCV factor and generates the following timing report:

```
report_timing -from in3 -to Top_I9/D -retime aocv -path_type full_clock


Path 1: MET Setup Check with Pin Top_I9/CK
Endpoint: Top_I9/D (v) checked with leading edge of 'clk2'
Beginpoint: in3 (^) triggered by leading edge of '@'
Path Groups: {clk2}
Retime Analysis { AOCV(Default) }
Other End Arrival Time 0.000
- Setup 0.248
+ Phase Shift 1.000
= Required Time 0.752
- Arrival Time 0.061
= Slack Time 0.691
Clock Rise Edge 0.000
+ Input Delay 0.000
= Beginpoint Arrival Time 0.000
Timing Path:
-----------------------------------------------------------
Load   Slew    Delay   Aocv    Cell   Arc    Pin
                       Derate
-----------------------------------------------------------
0.011  0.004                           in3 ^       in3 ->
0.011  0.004  0.001   1.173   INVX1               Top_I8/A
0.007  0.076  0.060   1.175   INVX1  A ^ -> Y v   Top_I8/Y
0.007  0.076  0.000   1.173   DFFX1               Top_I9/D ->
-----------------------------------------------------------
```

```
Clock Rise Edge 0.000
= Beginpoint Arrival Time 0.000
Other End Path:
-----------------------------------------------------
Load    Slew    Delay  Aocv  Cell   Arc     Pin
                       Derate
-----------------------------------------------------
0.012   0.000                       clk2 ^   clk2
0.012   0.000  0.000   0.825 DFFX1          Top_I9/CK
-----------------------------------------------------
```

- The following command recalculates the specified critical paths using the AOCV factor and generates an AOCV derating summary table for the launch and capture paths, in addition to the timing report:

```
report_timing -from in3 -to Top_I9/D -retime aocv -path_type full_clock -
derate_summary

Path 1: MET Setup Check with Pin Top_I9/CK
Endpoint: Top_I9/D (v) checked with leading edge of 'clk2'
Beginpoint: in3 (^) triggered by leading edge of '@'
Path Groups: {clk2}
Retime Analysis { AOCV(Default) }
Other End Arrival Time 0.000
- Setup 0.248
+ Phase Shift 1.000
= Required Time 0.752
- Arrival Time 0.061
= Slack Time 0.691
Clock Rise Edge 0.000
+ Input Delay 0.000
= Beginpoint Arrival Time 0.000
Timing Path:
-----------------------------------------------------------
Load    Slew   Delay  Aocv   Cell   Arc         Pin
Derate
-----------------------------------------------------------
0.011   0.004                       in3 ^      in3 ->
0.011   0.004  0.001  1.173  INVX1             Top_I8/A
0.007   0.076  0.060  1.175  INVX1  A ^ -> Y v Top_I8/Y
0.007   0.076  0.000  1.173  DFFX1             Top_I9/D ->
-----------------------------------------------------------
```

```
Clock Rise Edge 0.000
= Beginpoint Arrival Time 0.000
Other End Path:
-------------------------------------------------
Load   Slew    Delay  Aocv   Cell   Arc      Pin
                      Derate
-------------------------------------------------
0.012  0.000                         clk2 ^  clk2
0.012  0.000  0.000   0.825  DFFX1          Top_I9/CK
-------------------------------------------------
DUAL LOCV Derate Summary
Clock Branch Point None
Distance 89.944
```

- The following command reports pin names and net names in the same column alternatively:

```
report_timing -format { load slew delay arrival cell pin_load fanout timing_point} -
net

Path 1: MET Setup Check with Pin u55/CK
Endpoint: u55/D (^) checked with leading edge of 'CLK_W_4'
Beginpoint: u99/Q (v) triggered by leading edge of 'CLK_W_4'
Path Groups: {CLK_W_4}
Other End Arrival Time 0.150
- Setup 0.176
+ Phase Shift 10.000
= Required Time 9.974
- Arrival Time 0.827
= Slack Time 9.148
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.150
= Beginpoint Arrival Time 0.150
-------------------------------------------------------------------
Load    Slew    Delay  Arrival  Cell    Pin Load Fanout Timing Point
                      Time
-------------------------------------------------------------------
        0.116          0.150                          u99/CK
        0.086  0.195   0.345    DFF                    u99/Q
0.027                           (net)   0.027    1      n3
        0.086  0.000   0.345    INV                    u4/A
        1.140  0.482   0.827    INV                    u4/Y
1.006                           (net)   1.006    2      n4
        1.140  0.000   0.827    DFF                    u55/D ->
-------------------------------------------------------------------
```

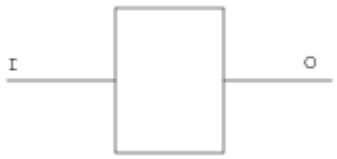
- The following command reports power domain details of path instances:

```
report_timing -format { load slew delay arrival cell arc hpin power_domain}

Path 1: MET Setup Check with Pin u55/CK
Endpoint: u55/D (^) checked with leading edge of 'CLK_W_4'
Beginpoint: u99/Q (v) triggered by leading edge of 'CLK_W_4'
Path Groups: {CLK_W_4}
Other End Arrival Time 0.150
- Setup 0.176
+ Phase Shift 10.000
= Required Time 9.974
- Arrival Time 0.827
= Slack Time 9.148
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.150
= Beginpoint Arrival Time 0.150
-------------------------------------------------------------------
Load   Slew   Delay  Arrival   Cell   Arc        Pin       Power
                     Time                                 Domain
-------------------------------------------------------------------
0.013  0.116         0.150            CK ^        u99/CK
0.027  0.086  0.195  0.345      DFF   CK ^ -> Q v u99/Q     PD1
1.006  1.140  0.482  0.827      INV   A v -> Y ^  u4/Y      PD2
1.006  1.140  0.000  0.827      DFF   D ^         u55/D -> PD1
-------------------------------------------------------------------
```

- The following examples refer to the figure shown below:

Conditions for Unconstrained and Constrained Paths



- Shows the path from `I` to `O` is reported as an unconstrained path under the following conditions (constraints on `I` and `O`):
- No constraints on either `I` or `O`
- `set_input_delay -clock ck1 1.0 I`
  No `set_output_delay` constraint on `O`
- `set_input_delay 1.0 I`

```
set_output_delay 1.1 O
```

- Shows the path from `I` to `O` is reported as a constrained path under the following conditions (constraints on `I` and `O`):

- ```
  set_input_delay  -clock ck1 1.0 I
  set_output_delay  -clock ck1 1.0 O
  ```

- ```
  set_input_delay  -clock ck1 1.0 I
  set_output_delay  -clock ck2 1.0 O
  ```

- ```
  set_output_delay  -clock ck1 1.0 O
  ```
  No `set_input_delay` constraint on I. It uses default delay of zero.
  Default: `-late`

- The following command reports AOCV weight details of path elements:
  ```
  report_timing -from in3 -to Top_I9/D -format {instance cell arc delay
       aocv_derate stage_count arrival aocv_weight} -path_type full_clock

  Path 1: MET Setup Check with Pin Top_I9/CK
  Endpoint: Top_I9/D (v) checked with leading edge of 'clk2'
  Beginpoint: in3 (^) triggered by leading edge of '@'
  Path Groups: {clk2}
  Other End Arrival Time 0.000
  - Setup 0.248
  + Phase Shift 1.000
  = Required Time 0.752
  - Arrival Time 0.061
  = Slack Time 0.691
  Clock Rise Edge 0.000
  + Input Delay 0.000
  = Beginpoint Arrival Time 0.000
  Timing Path:
  ---------------------------------------------------------------------
  Instance     Cell      Arc     Delay    Aocv    Aocv    Arrival    Aocv
                                         Derate  Stage   Time       Weight
                                                 Count
  ```
  ```

```
-------------------------------------------------------------------------
                      in3 ^                                 0.000
Top_I8       INVX1    A ^ -> Y v  0.061  1.175  1.000      0.061      1
Top_I9       DFFX1    D v         0.000  1.173  2.000      0.061      1
-------------------------------------------------------------------------
Clock Rise Edge 0.000
= Beginpoint Arrival Time 0.000
Other End Path:
-------------------------------------------------------------------------
Instance   Cell      Arc       Delay     Aocv    Aocv   Arrival    Aocv
                                       Derate  Stage  Time       Weight
                                               Count
-------------------------------------------------------------------------
                    clk2 ^                              0.000
Top_I9    DFFX1     CK ^      0.000     0.825   1.000  0.000      1
-------------------------------------------------------------------------
```

- The following command reports paths through unique set of pins:

```
report_timing -nworst 2 -unique_pins

Path 1: MET Setup Check with Pin seg3/u14/CK
Endpoint: seg3/u14/D (^) checked with leading edge of 'CLK_W_3'
Beginpoint: seg3/u9/Q (^) triggered by leading edge of 'CLK_W_3'
Path Groups: {CLK_W_3}
Other End Arrival Time 0.621
- Setup 0.186
+ Phase Shift 2.000
= Required Time 2.434
- Arrival Time 2.062
= Slack Time 0.372
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.682
= Beginpoint Arrival Time 0.682
-------------------------------------------------------------
Load   Slew    Delay  Arrival   Cell   Arc          Pin
```

```
                      Time
-------------------------------------------------------------
0.008  0.049  0.682                    CK ^         seg3/u9/CK
0.011  0.075  0.185   0.867     DFF    CK ^ -> Q ^  seg3/u9/Q
0.027  0.077  0.073   0.940     BUF    A ^ -> Y ^   seg3/u10/Y
1.027  0.644  0.317   1.257     INV    A ^ -> Y v   seg3/u11/Y
1.011  1.087  0.402   1.659     INV    A v -> Y ^   seg3/u12/Y
0.003  1.077  0.403   2.062     BUF    A ^ -> Y ^   seg3/u13/Y
0.003  1.077  0.000   2.062     DFF    D ^          seg3/u14/D
-------------------------------------------------------------
Path 2: MET Setup Check with Pin u2/CK
Endpoint: u2/D (v) checked with leading edge of 'CLK_W_1'
Beginpoint: CLK2 (v) triggered by trailing edge of 'CLK_W_2'
Path Groups: {CLK_W_1}
Other End Arrival Time 0.000
- Setup 0.188
+ Phase Shift 15.000
= Required Time 14.812
- Arrival Time 12.463
= Slack Time 2.349
Clock Fall Edge 12.500
= Beginpoint Arrival Time 12.500
-------------------------------------------------------------
Load    Slew   Delay    Arrival    Cell    Arc    Pin
                        Time
-------------------------------------------------------------
0.034  0.003   12.500                    CLK2 v      CLK2
0.003  0.003   -0.037   12.463     BUF    A v -> Y v  u10999/Y
0.003  0.003   0.000    12.463     DFF    D v         u2/D
-------------------------------------------------------------
```

- The following command reports all the violating paths (or having slack greater than the value specified with ‘-max_slack’) between unique source and target register pairs:

```
report_timing -to u55/D -begin_end_pair -max_slack 10
```

```
Path 1: MET Setup Check with Pin u55/CK
Endpoint: u55/D (^) checked with leading edge of 'CLK_W_4'
Beginpoint: u99/Q (v) triggered by leading edge of 'CLK_W_4'
Path Groups: {CLK_W_4}
Other End Arrival Time 0.150
- Setup 0.176
+ Phase Shift 10.000
= Required Time 9.974
- Arrival Time 0.827
= Slack Time 9.148
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.150
= Beginpoint Arrival Time 0.150
-------------------------------------------------------------
Load    Slew    Delay    Arrival    Cell    Arc    Pin
                         Time
-------------------------------------------------------------
0.013   0.116   0.150                      CK ^         u99/CK
0.027   0.086   0.195    0.345      DFF    CK ^ -> Q v  u99/Q
1.006   1.140   0.482    0.827      INV    A v -> Y ^   u4/Y
1.006   1.140   0.000    0.827      DFF    D ^          u55/D ->
-------------------------------------------------------------
```

- The following command generates a group based report in non-group mode:

```
report_timing -begin_end_pair -max_slack 10 -path_type end_slack_only


Path Groups: max_delay/setup {CLK_W_1}
-----------------------------------------------
End Point Slack Cause
-----------------------------------------------
u2/D f 2.349 MET
-----------------------------------------------

Path Groups: max_delay/setup {CLK_W_3}
-----------------------------------------------
End Point Slack Cause
-----------------------------------------------
```

```
seg3/u14/D r 0.372 MET
seg3/u3/D f 2.397 MET
-----------------------------------------

53. report_timing - -path_type full_clock -net -debug time_borrow
Path 1: MET Latch Borrowed Time Check with Pin latch2/GN
Endpoint: latch2/D (^) checked with trailing edge of 'CK1'
Beginpoint: ff1/Q (v) triggered by leading edge of 'CK1'
Path Groups: {CK1}
Other End Arrival Time 2.558
+ Borrow Edge Adjust 0.043
+ Phase Shift 0.000
+ CPPR Adjustment 0.000
= Required Time 2.601
- Arrival Time 0.544
= Slack Time 2.057
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.313
= Beginpoint Arrival Time 0.313
-------------------------------------------------------------------------------
Load   Slew    Delay   Incr   Arc  Fanout Cell     Arc          User    Arrival
Instance
Delay          Annotation                Derate   Time
-------------------------------------------------------------------------------
0.005  0.055                   WLM  2                CK ^                0.313   ff1
0.004  0.042   0.123   0.000  WLM  1      DFFHQX1   CK ^ -> Q v  1.000  0.436   ff1
0.004  0.044   0.035   0.000  WLM  1      INVX1     A v -> Y ^   1.000  0.471   invd4
0.004  0.026   0.019   0.000  WLM  1      INVX1     A ^ -> Y v   1.000  0.491   invd5
0.010  0.082   0.053   0.000  WLM  1      INVX1     A v -> Y ^   1.000  0.544   invd6
0.010  0.082   0.000   0.000  WLM          TLATNRX4 D ^          1.000  0.544
latch2
-------------------------------------------------------------------------------

Time Borrow Calculations
-------------------------

Latch Early Close Edge 4.482
- Library Setup 0.057
- Latch Early Open Edge 2.558
= Max Allowed Borrow Time 1.866
- Time Borrowed 0.043
= Available Time Borrow 1.824
```

- The following command generates the PBA delay calculation report for the net arc terminating at ff2/D pin.

```
report_timing -to ff2/D -retime path_slew_propagation -retime_delaycal_pins ff2/D

------------------------------------
report_delay_calculation PBA Report
------------------------------------
From pin : inv_inst/ZN
Voltage : 0.765 V
Cell : INV
Library : ccs_timing_si
To pin : ff2/D
Voltage : 0.765 V
Cell : DFF
Library : ccs_timing_si
Base or SI: SI delay
Delay type: net

Rise Rise Fall Fall Slew
Thresholds: Delay Slew Delay Slew Derate
From pin : 50 30->70 50 70->30 0.5
To pin : 50 30->70 50 70->30 0.5
-------------------------------------------------------------
From pin waveforms
-------------------------------------------------------------
Rise Waveform : { 0 0 6.88001e-12 -0.000345826 1.376e-11 -0.000759363 2.064e-11 -
0.00122762 2.752e-11 -0.00172836 3.44001e-11 -0.00223374 4.81601e-11 -0.00319517
6.19201e-11 -0.00401515 7.56801e-11 -0.00442153 1.032e-10 -0.00183165 1.1696e-10
0.0012542 1.2384e-10 0.00306022 1.3072e-10 0.00495619 1.4448e-10 0.00885701 1.5824e-
10 0.0128341 1.8576e-10 0.0208058 2.408e-10 0.0366083 3.50881e-10 0.067488 5.71041e-
10 0.126359 6.26081e-10 0.140509 6.53601e-10 0.147502 7.08641e-10 0.161279 7.36161e-
10 0.164503 7.49921e-10 0.164456 7.63681e-10 0.164333 7.77441e-10 0.164129 8.04961e-
10 0.161906 8.32481e-10 0.158058 8.60001e-10 0.154065 8.87521e-10 0.149911 9.42561e-
10 0.141834 9.97601e-10 0.134669 1.05264e-09 0.129404 1.10768e-09 0.126477 1.12144e-
09 0.123602 1.12832e-09 0.119572 1.1352e-09 0.114331 1.14208e-09 0.108821 1.14896e-09
0.10317 1.16272e-09 0.0926769 1.17648e-09 0.0833663 1.204e-09 0.0657246 1.21776e-09
0.0578875 1.23152e-09 0.0515706 1.24528e-09 0.0458775 1.25904e-09 0.0406301 1.28656e-
09 0.0323652 1.31408e-09 0.0270132 1.36912e-09 0.0280973 1.39664e-09 0.0336656
1.42416e-09 0.0379847 1.43792e-09 0.0378836 1.45168e-09 0.0365497 1.4792e-09
0.0332861 1.49296e-09 0.0314441 1.52048e-09 0.0278608 1.548e-09 0.0244482 1.60304e-09
0.0191201 1.65808e-09 0.0162272 1.71312e-09 0.0171329 1.76816e-09 0.0205911 1.8232e-
09 0.0256714 1.87824e-09 0.0351823 1.90576e-09 0.0413911 1.91952e-09 0.0445492
1.9264e-09 0.0461416 1.94016e-09 0.048697 1.9436e-09 0.046998 1.94704e-09 0.0443201
1.95392e-09 0.038436 1.95736e-09 0.0352619 1.96424e-09 0.0289844 1.97112e-09 0.022866
```

```
1.98488e-09 0.0125791 1.99864e-09 0.00580478 2.0124e-09 0.000710309 2.02616e-09 -
0.00178897 2.03992e-09 -0.00114673 2.05368e-09 0.000872076 2.06744e-09 0.00382227
2.0812e-09 0.00744581 2.10872e-09 0.0151135 2.13624e-09 0.0227224 2.16376e-09
0.0302613 2.2188e-09 0.0451577 2.32888e-09 0.0742571 2.54904e-09 0.129727 2.60408e-09
0.143045 2.6316e-09 0.143044 2.64536e-09 0.140082 2.65912e-09 0.137093 2.67288e-09
0.134064 2.7004e-09 0.12951 2.72792e-09 0.126729 2.78296e-09 0.1221 2.81048e-09
0.120269 2.86553e-09 0.118138 2.92057e-09 0.118162 3.03065e-09 0.12258 3.14073e-09
0.13482 3.25081e-09 0.155185 3.30585e-09 0.166687 3.36089e-09 0.178314 3.41593e-09
0.190705 3.47097e-09 0.203779 3.58105e-09 0.229323 3.63609e-09 0.241791 3.74617e-09
0.266165 3.96633e-09 0.312679 4.18649e-09 0.35642 4.29657e-09 0.377296 4.40665e-09
0.397506 4.51673e-09 0.417121 4.62681e-09 0.436119 4.84697e-09 0.472392 5.06713e-09
0.506472 5.28729e-09 0.538488 5.50745e-09 0.568534 5.72761e-09 0.596685 5.94777e-09
0.62296 6.16793e-09 0.647336 6.38809e-09 0.669575 6.60825e-09 0.689497 6.82841e-09
0.706756 7.04857e-09 0.721042 7.26873e-09 0.732528 7.48889e-09 0.741269 7.70905e-09
0.747704 7.92921e-09 0.752418 }
-------------------------------------------------------------
To pin waveforms
-------------------------------------------------------------
Rise Waveform : { 0 0 6.88001e-12 -0.00031215 1.376e-11 -0.000717461 2.064e-11 -
0.00118119 2.752e-11 -0.00168008 3.44001e-11 -0.00218582 4.81601e-11 -0.00315118
6.19201e-11 -0.0039801 7.56801e-11 -0.00441444 1.032e-10 -0.00195223 1.1696e-10
0.00108838 1.2384e-10 0.00288516 1.3072e-10 0.00477386 1.4448e-10 0.00866932 1.5824e-
10 0.0126446 1.8576e-10 0.0206168 2.408e-10 0.0364222 3.50881e-10 0.0673075 5.71041e-
10 0.12619 6.26081e-10 0.140342 6.53601e-10 0.147336 7.08641e-10 0.161117 7.36161e-10
0.164325 7.49921e-10 0.16428 7.63681e-10 0.164156 7.77441e-10 0.163952 8.04961e-10
0.161721 8.32481e-10 0.157871 8.60001e-10 0.153877 8.87521e-10 0.149721 9.42561e-10
0.141646 9.97601e-10 0.134476 1.05264e-09 0.129217 1.10768e-09 0.126291 1.12144e-09
0.123396 1.12832e-09 0.11934 1.1352e-09 0.114094 1.14208e-09 0.108576 1.14896e-09
0.102955 1.16272e-09 0.0924557 1.17648e-09 0.0831177 1.204e-09 0.0655087 1.21776e-09
0.05769 1.23152e-09 0.0513455 1.24528e-09 0.0456527 1.25904e-09 0.0404119 1.28656e-09
0.0321485 1.31408e-09 0.0268007 1.36912e-09 0.0278778 1.39664e-09 0.0334662 1.42416e-
09 0.0377678 1.43792e-09 0.0376836 1.45168e-09 0.0363449 1.4792e-09 0.0330793
1.49296e-09 0.0312404 1.52048e-09 0.027657 1.548e-09 0.0242445 1.60304e-09 0.0189107
1.65808e-09 0.0160238 1.71312e-09 0.0169355 1.76816e-09 0.020394 1.8232e-09 0.0254886
1.87824e-09 0.034983 1.90576e-09 0.0412062 1.91952e-09 0.0443699 1.9264e-09 0.0459651
1.94016e-09 0.0485097 1.9436e-09 0.0467553 1.94704e-09 0.0440755 1.95392e-09
0.0381743 1.95736e-09 0.0350146 1.96424e-09 0.0287352 1.97112e-09 0.0226158 1.98488e-
09 0.0123552 1.99864e-09 0.00553888 2.0124e-09 0.000528693 2.02616e-09 -0.00200176
2.03992e-09 -0.00134081 2.05368e-09 0.000687182 2.06744e-09 0.00364017 2.0812e-09
0.00726146 2.10872e-09 0.0149298 2.13624e-09 0.0225402 2.16376e-09 0.0300808 2.2188e-
09 0.0449804 2.32888e-09 0.0740856 2.54904e-09 0.129567 2.60408e-09 0.142888 2.6316e-
09 0.142859 2.64536e-09 0.139899 2.65912e-09 0.136908 2.67288e-09 0.133879 2.7004e-09
0.129349 2.72792e-09 0.126549 2.78296e-09 0.121913 2.81048e-09 0.120103 2.86553e-09
0.117961 2.92057e-09 0.117997 3.03065e-09 0.122409 3.14073e-09 0.134658 3.25081e-09
```

```
0.155035 3.30585e-09 0.166534 3.36089e-09 0.178163 3.41593e-09 0.190548 3.47097e-09
0.203625 3.58105e-09 0.229173 3.63609e-09 0.241644 3.74617e-09 0.266022 3.96633e-09
0.312545 4.18649e-09 0.356294 4.29657e-09 0.377174 4.40665e-09 0.397388 4.51673e-09
0.417006 4.62681e-09 0.436007 4.84697e-09 0.472288 5.06713e-09 0.506374 5.28729e-09
0.538396 5.50745e-09 0.568448 5.72761e-09 0.596605 5.94777e-09 0.622885 6.16793e-09
0.647267 6.38809e-09 0.669513 6.60825e-09 0.689441 6.82841e-09 0.70671 7.04857e-09
0.721003 7.26873e-09 0.732498 7.48889e-09 0.741247 7.70905e-09 0.747687 7.92921e-09
0.752406 }
-------------------------------------------------------------


-------------------------------------------------------------
RC Summary for net vic_net
-------------------------------------------------------------
Number of capacitance : 3
Net capacitance : 0.0568429 pF
Total rise capacitance: 0.0572236 pF
Total fall capacitance: 0.0572328 pF
Number of resistance : 5
Total resistance : 15.6659 Ohm
-------------------------------------------------------------
SI information for rise
-------------------------------------------------------------------------------------
Victim clock : myclk
-------------------------------------------------------------------------------------
Attacker Status Direction Slew Coupling cap Fanout Voltage Bump Timing window Clock
-------------------------------------------------------------------------------------
att1 SY Fall 0.404581 ns 0.012500 pF 1 0.765 v 0.135482 v 1.123200 ns 1.123300 ns
myclk
att2 SY Fall 0.205181 ns 0.010500 pF 1 0.765 v 0.125344 v 1.390900 ns 1.419600 ns
myclk
-------------------------------------------------------------------------------------
Model types
-------------------------------------------------------------------------------------
Net Term Cell Model
-------------------------------------------------------------------------------------
vic_net inv_inst/ZN INV CCS
vic_net ff2/D DFF CCS
att1 ZN INV CCS
att2 Q DFF CCS
-------------------------------------------------------------------------------------
Underated Delays Rise
-------------------------------------------------------------
Incr Net delay : 2.586069 ns
Net delay : 2.586721 ns
```

```
------------------------------------------------------------
Transitions Rise
------------------------------------------------------------
Input transition : 3.370655 ns
Output transition : 3.370683 ns
------------------------------------------------------------
Abbreviative attacker status and filtered reason:
------------------------------------------------------------
SY : Synchronous
INF: Attacker with infinite timing window
PE : Filtered due to physical exclusion
UF : Filtered by user
CN : Filtered because it is a constant net
SB : Filtered due to small bump
TA : Filtered because of timing window does not overlap
LC : Filtered due to logical correlation
ACT: Attacker is Active.
------------------------------------------------------------
```

Scenarios for Group Path Reporting for User-Defined Groups

- A user-defined group is created with the same name as any of the defined reference clocks and with the same constraint as for the "to" clock. For example, `group_path -name CLK -to CLK`.

  In this case only one group named `CLK` will be displayed in the timing report. This will be the same default group with reference clock `CLK`. Note that default groups - clock gating and asynchronous checks - will not be honored for this group.

- A user-defined group is created with the same name as any of the reference clocks but with different constraint as specified for the "to" clock. E.g., `group_path -name CLK - from pin1 -to CLK`.

  In this case the two groups named `CLK` will be displayed in the timing report. One group will be the same as the default group with reference clock `CLK` (which will not honor clock gating or asynchronous checks) and the other will be the user-defined path group with name `CLK`, which will honor all the checks.

Related Information

- report_net
- report_ports
- set_table_style

report_timing_derate

```
report_timing_derate
[-delay_corner delayCornerName]
[-include_inherited]
[object_list]
[{> | >>} filename[.gz]]
```

Reports delay scaling or derating factors for early and late clock and data paths in the current design.

By default, the `report_timing_derate` output shows pulse_width_check and clock_period_check details for instance objects (see `set_timing_derate`).

If you specify this command without any parameters, reports are generated for global derating factors for each instance, net, or cell for which derating is asserted.

The following reports are also generated:

- In best case worst case, maximum and minimum corner
- Early values from minimum corner and late values from maximum corner in on-chip variation mode
- In MMMC mode, specified corners, or for all corners when corner is not specified
- Power domains (if present in the design)

Parameters

`> | >> filename`	When specified with `>`, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with `>>`, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. **Note:** To write a compressed report, add the `.gz` extension to the file name.
`-delay_corner delayCornerName`	Reports derating factors for the specified delay calculation corner only. *Default:* Reports derating factors for all delay calculation corners.
`-include_inherited`	Reports the object names from which the derating factor values are inherited. When specified, the report displays the name of the cell for each instance from which derate values are inherited.
object_list	Specifies the list of cells, nets, or library cells.

Examples

- The following commands display global derating factors:

```
set_timing_derate 1.2
set_timing_derate -clock -early 1.1
set_timing_derate -data -late 1.2

report_timing_derate
                   --------Clock-----------      ----------Data-----------------
                   Early              Late        Early             Late
                   ------------------------------------------------------------
Cell Delay         1.100000           1.200000    1.200000          1.200000
Net Delay Static   1.100000           1.200000    1.200000          1.200000
Net Delay Dynamic  1.100000           1.200000    1.200000          1.200000
Cell Check         --                 --          --                --
Retain delay       --                 --          --                --
```

- The following commands display instance derating factors:

```
set_timing_derate -cell_delay -clock -late 1.3 [get_cells {u55}]
set_timing_derate -cell_check -late 1.4 [get_cells {u55}]

report_timing_derate

---- Report Timing Derate for DC Corner: default_emulate_delay_corner

-------------Clock----------------- ---------------Data-----------------
                      Rise          Fall           Rise         Fall
                    Early Late     Early Late     Early Late   Early Late
-------------------------------------------------------------------------

Cell (leaf): u55
Cell Delay          -     1.300    --    1.300    --    --     --   --
Cell Check          --    1.400    --    1.400    --    1.400  --   1.400
```

- The following commands display hierarchical derating factors:

```
set_timing_derate 1.2
set_timing_derate 1.1 -late h4
set_timing_derate 1.2 -early h4/h2
set_timing_derate -cell_check 1.2 h4
```

```
report_timing_derate -include_inherited

                 ---------------Clock-------------  -----------Data---------
                 Early                Late           Early             Late
                 ------------------------------------------------------------
Cell Delay       1.200000          1.200000       1.200000          1.200000
Net Delay Static 1.200000          1.200000       1.200000          1.200000
Net Delay Dynamic 1.200000         1.200000       1.200000          1.200000
Cell Check       --                --             --                --
Retain delay     --                --             --                --

Cell (hier): h4
Cell Delay       1.200000          1.100000       1.200000          1.100000
inherited        top               --             top               --
Net Delay Static 1.200000          1.100000       1.200000          1.100000
inherited        top               --             top               --
Net Delay Dynamic 1.200000         1.100000       1.200000          1.100000
inherited        top               --             top               --
Cell Check       1.200000          1.200000       1.200000          1.200000
inherited        --                --             --                --

Cell (hier): h4/h2
Cell Delay       1.200000          1.100000       1.200000          1.100000
inherited        --                h4             --                h4
Net Delay Static 1.200000          1.100000       1.200000          1.100000
inherited        --                h4             --                h4
Net Delay Dynamic 1.200000         1.100000       1.200000          1.100000
inherited        --                h4             --                h4
```

- The following commands display best case worst case derating factors:

```
set_timing_derate -cell_delay -clock -late 1.2 [get_cells {f2}] -min
set_timing_derate -cell_delay -late 1.3 [get_cells {f2}] -max

report_timing_derate

------------------------Report Timing Derate for MIN corner----------------------
                   ---------------Clock-------------  -----------Data---------
                   Early                Late           Early             Late
                   ------------------------------------------------------------
 Cell Delay        --                 --              --                --
 Net Delay Static  --                 --              --                --
 Net Delay Dynamic --                 --              --                --
 Cell Check        --                 --              --                --
 Retain delay      --                 --              --                --

 Cell (leaf): f2
 Cell Delay        --                 --             --                 1.200000

 ------------------------Report Timing Derate for MAX corner----------------------
                   ---------------Clock-------------  -----------Data---------
                   Early                Late           Early             Late
                   ------------------------------------------------------------
 Cell Delay        --                 --              --                --
 Net Delay Static  --                 --              --                --
 Net Delay Dynamic --                 --              --                --
 Cell Check        --                 --              --                --
 Retain delay      --                 --              --                --

 Cell (leaf): f2
 Cell Delay        --                 --             --                 1.300000
```

- The following example shows a report that is displayed when sigma/mean derates are applied in the SOCV mode:

```
> set_timing_derate 1.11 -mean -delay_corner [all_delay_corners ] [get_lib_cells
XOR3_X4M_A12TS_C35] -> derate set for mean on the lib cell XOR3_X4M_A12TS_C35
> set_timing_derate 1.22 -sigma -delay_corner [all_delay_corners ] [get_lib_cells
XOR3_X4M_A12TS_C35] -> derate set for sigma on the lib cell XOR3_X4M_A12TS_C35
> report_timing_derate [get_lib_cells XOR3_X4M_A12TS_C35] -> report_timing_derate
shows derate split in the format mean_derate:sigma_derate


---- Report Timing Derate for DC Corner: max_delay_corner

-----------------------------------Clock-----------------------------------------

--------------------------------------- ----------------------------------------
---------------Data---------
                     Rise                              Fall
 Rise                              Fall
             Early           Late            Early         Late              Early
       Late             Early           Late

                  ---------------------------------------------------------------
LibraryCell: base_svt_c35Fsc12mc_cln28hpmPbcs30V1100T125/XOR3_X4M_A12TS_C35
Cell_delay    1.11000:1.22000 1.11000:1.22000 1.11000:1.22000 1.11000:1.22000
1.11000:1.22000 1.11000:1.22000 1.11000:1.22000 1.11000:1.22000
Retain_delay  --              --              --              --              --
         --              --              --
Cell Check    --              --              --              --              --
         --              --              --
```

Related Information

- set_timing_derate
- reset_timing_derate

report_wire_load

```
report_wire_load
[-inst name_or_collection_of_hierarchical_instances]
[-ports name_or_collection_of_ports]
[-late | -early]
[> | >> filename]
```

Reports the wireload name, library, wire load mode and selection type for all hierarchical instances in the design.

Note: To enable wireload mode, you are required to set the following global:

set soceSupportWireLoadMode 1

Note: To use the wireload model functionality, you must enable the `enable_wire_load_model_support` global variable using the following command:

```
set_var enable_wire_load_model_support 1
```

Parameters

`>|>> filename`	When specified with `>`, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with `>>`, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *`filename`* parameter must be the last argument in the list. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.

`-inst` *`name_or_collection_of_hierarchical_instances`*	Specifies the name of an instance or collection for which the `report_wire_load` command reports the wireload name, library, wire load mode, and selection type. The `report_wire_load` command also supports wildcard names such as U*.
`-late	-early`	Reports wireload models in setup or hold analysis mode. By default, the software reports wireload models for the mode that you set using the command.
`-ports` *`name_or_collection_of_ports`*	Reports wireload information for the specified ports.

Examples

- The following example shows the default report when wireload mode is set to top:

```
report_wire_load
---------------------------------------------------------------------
Design Name top_chip
Wire Load Mode top

Instance Module WireLoadModel Library SelctionType
---------------------------------------------------------------------
- top_chip model4 fast library-default
---------------------------------------------------------------------
```

- The following example shows the report when wireload mode is set to enclosed to report lower hierarchy:

```
set_wire_load_mode enclosed

report_wire_load
---------------------------------------------------------------------
Design Name top_chip
Wire Load Mode enclosed
Instance Module WireLoadModel Library SelctionType
---------------------------------------------------------------------
```

```
- top_chip model4 fast library-default
U1 randr model4 fast library-default
U2 randr model4 fast library-default
------------------------------------------------------------------------
```

- The following example shows model assertions using the `set_wire_load_model` command the corresponding reports using the `report_wire_load` command:

```
set_wire_load_model -name model3
report_wire_load
------------------------------------------------------------------------
Design Name top_chip
Wire Load Mode enclosed

Instance Module WireLoadModel Library SelctionType
------------------------------------------------------------------------
- top_chip model3 fast user-asserted
U1 randr model3 fast user-asserted
U2 randr model3 fast user-asserted
------------------------------------------------------------------------
set_wire_load_model -name model2 [get_cells U1]

set_wire_load_model -name model4 [get_cells U2]

report_wire_load
------------------------------------------------------------------------
Design Name top_chip
Wire Load Mode enclosed

Instance Module WireLoadModel Library SelctionType
------------------------------------------------------------------------
- top_chip model3 fast user-asserted
U1 randr model2 fast user-asserted
U2 randr model4 fast user-asserted
------------------------------------------------------------------------
```

- The following example shows the report including port information:

```
report_wire_load -ports [get_ports XYZ/A]

-------------------------------------------------------------------------
Design Name   randr

Wire Load Mode   enclosed

Port     Module WireLoadModel Library    SelctionType
-------------------------------------------------------------------------

P1     randr model2               slow   user-asserted

P2     randr model4              slow   user-asserted
-------------------------------------------------------------------------
```

Related Information

- set_wire_load_model
- reset_wire_load_model
- setAnalysisMode

reportTimingLib

```
reportTimingLib
-outfile fileName
[-byLib]
[-all | -pin | -threshold | -sibling]
[-lib library_name]
[cellName]
```

Reports the contents of the timing library.

Parameters

`-all	-pin	-threshold	-sibling`	Specifies the parameter for which the software should report the timing data. `-all` prints all the pin properties, siblings of a cell, the libraries where they are defined, and the voltage. `-pin` reports the pin properties, such as the pin name, direction, capacitance, max load, max fanout, and fanout load. `-threshold` reports the input thresholds, output thresholds, and slew thresholds of a cell. `-sibling` reports sibling cells, the libraries where they are defined, and the voltage. *Default:* `-all`
`-byLib`	Organizes the generated report by libraries. If you do not specify this parameter, the `reportTimingLib` command prints out cells with the same name together. Use this parameter to support multi-supply multi-voltage feature of the software, where you can define same cell names in more than one library with different voltage values and timing data.
cellName	Specifies the individual cell name to be reported.

`-lib` *`library_name`*	Reports a specified library. If you do not specify this parameter, all the library names are reported.
`-outfile` *`fileName`*	Specifies the name of the file to which the results are written.

Command Order

Use this command after importing the design when the timing library is also loaded.

Example

The following command reports the cell information in the timing library, and writes the results to the file `xyz.dump.lib`:

reportTimingLib -outfile xyz.dump.lib

reset_instance_library

```
reset_instance_library
[-help]
```

Resets the settings/binding done using the `set_instance_library` command.

Parameters

`-help`	Prints out the command usage.

Related Commands

- `set_instance_library`

reset_path_adjust_group

```
reset_path_adjust_group
[-help]
[-name <group_name> | -all]
```

Removes all path adjust groups that are set using the `set_path_adjust` command for setting path slack adjustment values.

Parameters

`-help`	Prints out the command usage.
`-all`	Removes all the path adjust groups.
`-name` *`group_name`*	Removes the named path adjust group completely.

Related Commands

- `set_path_adjust_group`
- `set_path_adjust`

reset_sdf_assertions

`reset_sdf_assertions`

Resets the sdf delays to the values defined in the library. When you perform timing analysis using the timing engine, the software uses sdf values that you define using the `read_sdf` command earlier in the flow. You use the `reset_sdf_assertions` commands to discard delay values specified using the sdf files, and use the delays defined in the library.

Note: You do not need to read in the sdf delays every time you perform delay calculation. The software uses the sdf files specified earlier in the flow.

Parameter

None

Related Information

- `read_sdf`

reset_timing_derate

```
reset_timing_derate
[-delay_corner delayCornerName]
[-increment]
[object_list]
```

Resets derate factors specified on a design or a list of instances (cells, net, or library cells). All the derating factors that are set globally or on specific instances are reset to a default value of `1.0`.

The derating factors for early or late paths in the design are set using the `set_timing_derate` command.

Parameters

`-help`	Prints out the command usage.
`-delay_corner delayCornerName`	Specifies the delay corner or a list of delay corners on which reset derating is to be applied.
`-increment`	Incrementally adds the derate value to the total derate value in all the modes - OCV, AOCV, and SOCV mode.
object_list	Specifies the list of cells, nets, or library cells to reset. If the *object_list* contains hierarchical cells then all cells within the hierarchical cells and across lower hierarchies have reset derate factors.

Examples

- The following example resets all the object-specific and global derating factors:
  ```
  reset_timing_derate *
  ```
- The following example resets only global derating factors:
  ```
  reset_timing_derate [current_design]
  ```

- The following example resets derating factor values for the specified delay corner:
  `reset_timing_derate -delay_corner` *`delayCornerName`*
- The following example resets the global derating values for the specified delay corner:
  `reset_timing_derate -delay_corner` *`delayCornerName`* `[current_design]`
- The following example resets timing derate on instance `u1`:
  `reset_timing_derate [get_cells u1]`

Related Information

- `set_timing_derate`

reset_wire_load_mode

```
reset_wire_load_mode
```

Resets the wire-load mode to the default mode of `top`.

Note: To enable wireload mode, you are required to set the following global:

```
set soceSupportWireLoadMode 1
```

Note: To use the wireload model functionality, you must enable the `enable_wire_load_model_support` global variable using the following command:

```
set_var enable_wire_load_model_support 1
```

Related Information

- `set_wire_load_mode`

reset_wire_load_model

```
reset_wire_load_model
[-help]
[{ list_of_instances_or_ports }]
```

Resets wireload models for the specified list of objects. The list of objects can include designs, hierarchical cells, or ports.

When objects are not specified, the software removes the wireload model specified using the `set_wire_load_model` command from the current instance only.

Note: To enable wireload mode, you are required to set the following global:

```
set soceSupportWireLoadMode 1
```

Note: To use the wireload model functionality, you must enable the `enable_wire_load_model_support` global variable using the following command:

```
set_var enable_wire_load_model_support 1
```

Parameters

`-help`	Outputs a brief description that includes type and default information for each `reset_wire_load_model` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reset_wire_load_model`.
`list_of_instances_or_ports`	Specifies the list of instances or ports for which to remove the wireload models.

Example

- The following command removes the wireload model for `clk1`:

  ```
  reset_wire_load_model clk1
  ```

Related Commands Information

- set_wire_load_model

reset_wire_load_selection_group

```
reset_wire_load_selection_group [{ list_of_objects }]
```

Removes the wireload selection group for the specified list of objects. The list of objects can include designs, hierarchical cells, or ports. When objects are not specified, wireload selection groups are removed for all objects.

Note: To enable wireload mode, you are required to set the following global:

```
set soceSupportWireLoadMode 1
```

Note: To use the wireload model functionality, you must enable the `enable_wire_load_model_support` global variable using the following command:

```
set_var enable_wire_load_model_support 1
```

Parameters

list_of_objects	Specifies the list of object names for which to remove the wireload selection groups.

Example

- The following command removes the wireload selection group for `clk1`:

  ```
  reset_wire_load_selection_group clk1
  ```

Related Information

- `set_wire_load_selection_group`

set_analysis_view

```
set_analysis_view
[-help]
[-drv string]
[-dynamic string]
-hold list_of_views
[-inactive string]
[-leakage string]
-setup list_of_views
[-update_timing]
```

Defines the analysis views to use for setup and hold analysis and optimization.You must define at least one setup and one hold analysis view. The "active" views represent the different design variations that will be analyzed. These views can be changed throughout the flow to utilize different subsets of views. The libraries and data are loaded into the system that are required to support the selected set of active views.

When the `set_analysis_view` command is issued, it will cause a full timing reset. During a timing reset, all the interconnect parasitics, delays, and timing slacks will be recalculated.

The order in which you specify the views with the `-setup` and `-hold` parameters is important. By default, the first views defined in the `-setup` and `-hold` lists are the default views. The default views serve two purposes:

- Provide the library data that is used during the design initialization process.
- Indicate to parts of the application, that are not MMMC-aware, which view's information to use for a particular analysis step (in most cases, the default view is the first setup or hold view). You can use the `set_default_view` command to change the default view to any currently active view without incurring a timing reset. This control is not frequently used since most applications either support MMMC or provide a direct control for selecting the appropriate view to use for that specific analysis.

Using the `set_default_view` command does not affect software performance because it only uses views that are already active in the design. If you use the `set_analysis_view` command to change the default views, the existing timing and delay calculation data is reset.

Use this command after creating multiple analysis views (`create_analysis_view`).

Note: The `set_analysis_view` command does not work in an incremental manner for setting parameters. Each invocation of this command must include the full specification of all the active views required for analysis.

Parameters

`-help`	Prints out the command usage.
`-drv` *string*	Selects specific views for DRV analysis, e.g., max_transition and max_capacitance checking. When this parameter is specified, additional analysis views are specified for DRV analysis, so the timing system efficiently computes the required data for these checks. This methodology is efficient with respect to memory and performance, because all the views as full Setup views are not included. The following guidelines should be followed when selecting DRV views: • Any setup views that require DRV analysis should not be specified as DRV views. The setup views are automatically considered for DRV. • If a subset of hold-only views require DRV analysis, they should be specified as DRV views as well. The timing system will augment the timing information normally computed for Hold views with additional data required for the DRV analysis. • Additional views that are not active setup or hold views can be specified as DRV only views. The system will efficiently compute the required data for performing accurate DRV checks.

`-dynamic` *string*	Sets a view as a dynamic view. Delay calculation is required to provide transition times. Delay calculation should not perform SI analysis if the delay corner is only required for the dynamic power. The timer is not required to propagate required times or perform slack computations.
`-hold` *list_of_views*	Sets the active views for hold analysis.
`-inactive` *string*	Sets views as inactive for timing analysis.
`-leakage` *string*	Sets a view as a leakage view. The Timing libraries must be loaded, if not already in use. The Timer constant propagation is required, whereas delay calculation is not required.
`-setup` *list_of_views*	Sets the active views for setup analysis.
`-update_timing`	Reloads the configuration described by the MMMC view and mode objects. When the timing global `timing_defer_mmmc_object_updates` is set to `true`, this parameter is required to update the MMMC timing environment. The `-update_timing` parameter may be specified without any `-setup` or `-hold` arguments. In this case, the data associated with the current set of setup and/or hold views will be loaded. If no active setup or hold views exist, then using this parameter alone may display an error. The `-update_timing` parameter may also be used in conjunction with the `-setup` and `-hold` options. In this case, the newly specified setup/hold view configuration will be loaded with the updated MMMC object information.

When you issue the `set_analysis_view` command, the software will perform a timing reset, so that only the new active view information is loaded up. The software requires the `-setup` and `-hold` view parameters to be explicitly specified for each `set_analysis_view` command run. If these parameters are not specified, the software issues an error message. Any optional settings, such as, `-leakage` or `-dynamic` views will be reset if they are not specified with the

new `set_analysis_view` command.

Note: It is recommended to use the same power view for both leakage and dynamic, that is, both the `-dynamic` and `-leakage` parameters should have the same user-specified view settings.

Examples

- The following command sets `missionWCCOM` and `mission2WCCOM` as the active views for setup analysis, and `missionBCCOM` and `testBCCOM` as the active views for hold analysis:

```
set_analysis_view -setup {missionWCCOM mission2WCCOM}

  -hold {missionBCCOM testBCCOM}
```

Related Information

- Performing Multi-Mode Multi-Corner Timing Analysis and Optimization chapter in the *Innovus User Guide.*

Related Commands

- `create_analysis_view`
- `update_analysis_view`
- `set_default_view`

set_aocv_thresholds

```
set_aocv_thresholds
[-help]
[-min_stage_count_setup <int>]
[-min_stage_count_hold <int>]
[-reset]
[-slack_pruning_threshold <float>]
-view <list_of_views>
```

Limits the minimum AOCV stage depth in graph-based analysis (GBA) mode.

Parameters

`-help`	Prints out the command usage
`-min_stage_count_setup <int>`	Applies the AOCV stage count floor, during Setup analysis, so that all the instances are guaranteed to have a stage depth no less than the specified value. A value of '0' will remove the assertion.
`-min_stage_count_hold <int>`	Applies the AOCV stage count floor, during Hold analysis, so that all the instances are guaranteed to have a stage depth no less than the specified value. A value of '0' will remove the assertion.
`-reset`	Removes/sets to default all of the AOCV threshold assertions for specified views.
`-slack_pruning_threshold <float>`	Controls the slack threshold (in two-pass/slack-based AOCV analysis mode), at which paths are removed from consideration for the next phase of AOCV stage counting.
`-view <list_of_views>`	Applies the specified thresholds to the list of analysis views.

set_default_view

```
set_default_view
{-setup newDefaultSetupView | -hold newDefaultHoldView}
```

Temporarily changes the default active analysis view to a different active view.

Some Innovus applications can function only on a single analysis view at a time. By default, these single-view applications use the default analysis view. If an application or flow step does not provide a native option for specifying which view to use, the `set_default_view` command can be used to temporarily change the default view to a different active view that is better suited to that flow step.

Note: Using the `set_default_view` command does not affect software performance because it only uses views that are already active in the design. If you use the set_analysis_view command to change the default views, the existing timing, delay calculation, and RC data is reset.

Use this command after defining active views for the design (set_analysis_view).

Parameters

`-hold` *newDefaultHoldView*	Changes the default active hold analysis view to the specified analysis view.
`-setup` *newDefaultSetupView*	Changes the default setup analysis view to the specified analysis view.

Examples

- The following command sets `missionWCCOM`, `mission2WCCOM`, and `missionSlow` as the active views for setup analysis, and `missionBCCOM` and `testBCCOM` as the active views for hold analysis. The default active setup view is `missionWCCOM`, and the default active hold view is `missionBCCOM`.

  ```
  set_analysis_view -setup {missionWCCOM mission2WCCOM missionSlow}
  -hold {missionBCCOM testBCCOM}
  ```

- The following command changes the default active setup view to `missionSlow`:

  ```
  set_default_view -setup missionSlow
  ```

- The following command changes the default active hold view to `testBCCOM`:

  ```
  set_default_view -hold testBCCOM
  ```

Related Commands

- create_analysis_view
- update_analysis_view
- set_default_view

set_instance_library

```
set_instance_library
[-help]
-library string
-instance string
[-early | -late]
```

Binds a cell library to an instance. When specified, the software overrides the default binding mechanism for a cell from the library set.

Note: The `set_instance_library` command cannot be used at the command prompt - this command can be specified in the ViewDefinition file.

Parameters

`-help`	Prints out the command usage.
`-early`	Uses the library binding for early timing analysis.
`-instance <string>`	Specifies the instance for which the library binding needs to be done.
`-late`	Uses the library binding for late timing analysis.
`-library <string>`	Specifies the library file path.

Examples

The following commands specify the instance for which the library binding will be done:

- `set_instance_library -library newcell.lib -instance {i1}`
- `set_instance_library -library mysil.lib -instance {i1 i2}`
- `set_instance_library -library {new.lib mysil.lib} -instance {i1} -early`
- `set_instance_library -library {mynew.lib mysil.lib} -instance {i1 i2} -late`

Related Commands

- `reset_instance_library`

set_interactive_constraint_modes

`set_interactive_constraint_modes {list_of_constraint_modes}`

Puts the software into interactive constraint entry mode for the specified multi-mode multi-corner constraint mode objects. Any timing constraints that you specify after this command take effect immediately on all active analysis views that are associated with the specified constraint modes. By default, no constraint modes are considered active.

The software stays in interactive mode until you exit by specifying the `set_interactive_constraint_modes` command with an empty list:

```
set_interactive_constraint_modes { }
```

All new assertions are saved in the SDC files of the specified constraint modes when you save the design (`saveDesign`).

The `all_constraint_modes` command can be used to generate a list of constraint modes as the argument for this command.

Use the `get_interactive_constraint_modes` command to return a list of the constraint mode objects in interactive constraint entry mode.

Note: Interactive constraint mode only works when the software is in multi-mode multi-corner timing analysis mode. In min/max analysis mode, constraints are always accepted interactively.

Parameters

{*list_of_constraint_modes*}	Specifies the constraint mode objects to update with the new assertions.

Examples

- The following commands put the software into interactive constraint entry mode, and apply the `set_propagated_clock` assertion on all views in the current session that are associated with the constraint mode `func1`:

  ```
  set_interactive_constraint_modes func1

  set_propagated_clock [all_clocks]
  ```

  When you specify `saveDesign`, the updated constraints file for the `func1` constraint mode will include the `set_propagated_clock`.

- The following commands put the software into interactive constraint entry mode, and apply the `set_propagated_clock` assertion on all active analysis views in the current session.

  ```
  set_interactive_constraint_modes [all_constraint_modes -active]

  set_propagated_clock [all_clocks]
  ```

  When you specify `saveDesign`, the `set_propagated_clock` assertion will be saved persistently as part of a new `.sdc` file.

Related Information

- `get_interactive_constraint_modes`
- `all_constraint_modes`

set_io_thresholds

```
set_io_thresholds
[-slew_lower_threshold_pct pctValue]
[-slew_upper_threshold_pct pctValue]
[-input_threshold_pct pctValue]
[-output_threshold_pct pctValue]
[-slew_lower_threshold_pct_rise pctValue]
[-slew_lower_threshold_pct_fall pctValue]
[-slew_upper_threshold_pct_rise pctValue]
[-slew_upper_threshold_pct_fall pctValue]
[-input_threshold_pct_rise pctValue]
[-input_threshold_pct_fall pctValue]
[-output_threshold_pct_rise pctValue]
[-output_threshold_pct_fall pctValue]
```

Sets the threshold trip-points used for delay calculation to and from the primary I/Os. The `set_io_thresholds` command does not affect the interpretation of threshold settings in the Liberty timing libraries; the effect is strictly limited to the I/O ports.

If you do not specify I/O thresholds with the `set_io_thresholds` command, delay calculation assumes that the I/O thresholds are equivalent to those of the logic gates to which they are connected.

The `set_io_thresholds` command supports separate rise and fall slew thresholds as normally found in Liberty timing libraries.

Note: Some third-party applications derive the I/O threshold values from the first Liberty timing library in the library search path. The software currently does not support this methodology. Explicit setting of I/O thresholds with the `set_io_thresholds` command should be used to complete a compatible analysis environment.

Parameters

`-input_threshold_pct` *pctValue*	Specifies the slew input threshold value.
`-input_threshold_pct_fall` *pctValue*	Specifies the measurement point for a falling signal that will be used for calculating the interconnect delay from a primary input to internal logic. *Value:* `0` - `100` *Default:* `50`

`-input_threshold_pct_rise` *pctValue*	Specifies the measurement point for a rising signal that will be used for calculating the interconnect delay from a primary input to internal logic. *Value:* `0 - 100` *Default:* `50`
`-output_threshold_pct` *pctValue*	Specifies the slew output threshold value.
`-output_threshold_pct_fall` *pctValue*	Specifies the measurement point for a falling signal that will be used for calculating the interconnect delay from internal logic to a primary output. *Value:* `0 - 100` *Default:* `50`
`-output_threshold_pct_rise` *pctValue*	Specifies the measurement point for a rising signal that will be used for calculating the interconnect delay from internal logic to a primary output. *Value:* `0 - 100` *Default:* `50`
`-slew_lower_threshold_pct` *pctValue*	Specifies the slew lower threshold value.
`-slew_lower_threshold_pct_fall` *pctValue*	Specifies the lower trip-point for a falling signal that will be used to measure transition times. *Value:* `0 - 100` *Default:* `20`

`-slew_lower_threshold_pct_rise` *pctValue*	Specifies the lower trip-point for a rising signal that will be used to measure transition times. *Value:* `0 - 100` *Default:* `20`
`-slew_upper_threshold_pct_fall` *pctValue*	Specifies the upper trip-point for a falling signal that will be used to measure transition times. *Value:* `0 - 100` *Default:* `80`
`-slew_upper_threshold_pct_rise` *pctValue*	Specifies the upper trip-point for a rising signal that will be used to measure transition times. *Value:* `0 - 100` *Default:* `80`
`-slew_upper_threshold_pct` *pctValue*	Specifies the slew lower rise threshold value.

Examples

The following command specifies trip-points for measuring delay and transition times:

```
set_io_thresholds -slew_lower_threshold_pct_rise 10


-slew_upper_threshold_pct_fall 80
-slew_upper_threshold_pct_rise 90
-slew_lower_threshold_pct_fall 20
-input_threshold_pct_rise 40
-input_threshold_pct_fall 70
-output_threshold_pct_rise 60
-output_threshold_pct_fall 30
```

set_normalized_driver_waveform_lib

```
set_normalized_driver_waveform_lib
[-help]
[-fall_waveform string]
[-rise_waveform string]
-ref_lib string
-libs string
```

Specifies normalized driver waveform for a library. The normalized driver waveform given in the reference library is applied to all the specified libraries.

Note: For this command to take effect, you should set the `timing_generate_normalized_driver_waveform` global variable to `false` before loading libraries.

Parameters

`-help`	Prints out the command usage.
`-fall_waveform string`	Specifies the name of the fall normalized driver waveform.
`-libs string`	Specifies a list of space separated libraries to which normalized driver waveform is to be applied.
`-ref_lib string`	Specifies the reference library from which normalized driver waveform is to be selected.
`-rise_waveform string`	Specifies the name of the rise normalized driver waveform.

Examples

- The following example shows that normalized driver waveform given in `$libDir/libs/ref.lib` will be applied to all the cells in `$libDir/1.lib $testDir/2.lib $libDir/libs/3.lib`:

  ```
  set_normalized_driver_waveform_lib -libs "$libDir/1.lib $testDir/2.lib
  $libDir/libs/3.lib" -ref_lib $libDir/libs/ref.lib
  ```

set_path_adjust

```
set_path_adjust
[-help]
adjust_value
[-path_adjust_group path_adjust_group_name]
[-view string]
[-setup | -hold]
```

Defines the slack adjustment value for timing paths. The defined adjustment value is added to the slack of a path specified by the path adjust group, referred to by the `-path_adjust_group` parameter. You can specify a path adjust group by using the `set_path_adjust_group` command.

If multiple path adjustment values are applicable on the same pin, which is on the path specified by multiple `set_path_adjust_group` constraints, then the smallest adjustment value will be applied.

Note: The `set_path_adjust` command is not allowed to be used inside the SDC constraints file. It is only allowed in the software run script.

Parameters

`-help`	Prints out the command usage.
adjust_value	Specifies the value of slack adjustment.
`-hold`	Specifies to apply the adjust value to early paths.
`-path_adjust_group` *path_adjust_group_name*	Specifies the name of path adjust group that defines the path for which the adjustment will be applied.
`-setup`	Specifies to apply the adjust value to late paths. **Note**: If you do not specify `-setup` or `-hold` parameter, the software sets the adjust value to both setup and hold analysis.
`-view` *string*	Specifies a list of analysis views for which adjustment will be applied. This parameter accepts multiple views as argument.

Examples

- The following commands define the slack adjustment values for the specified views:

```
set_path_adjust 0.44 -path_adjust_group AG1 -view view1 -hold
set_path_adjust 0.44 -path_adjust_group AG2 -view view2 -setup
set_path_adjust 0.44 -path_adjust_group AG3 -view view3
```

Related Commands

- set_path_adjust_group
- reset_path_adjust_group

set_path_adjust_group

```
set_path_adjust_group
[-help]
-name <path_adjust_group_name>
[-from <from_list>]
[-to <to_list>]
[-through <through_list>]
```

Allows you to specify the path that is used by the `set_path_adjust` command for setting path slack adjustment values.

The `set_path_adjust_group` command is not allowed to be used inside the SDC constraints file. It is only allowed in the tool run script.

If path group definitions are matching/overlapping (when the `-from`/`-through`/`-to` parameters are used), then path group resetting is applied. The path group overwriting and prioritization rules are similar to path exceptions.

For more details, refer to `group_path`.

Parameters

`-help`	Prints out the command usage.
`-from <from_list>`	Specifies a list of pins, instances, or clocks that are the start of the paths.
`-name <path_adjust_group_name>`	Specifies the name of the path adjust group.
`-through <through_list>`	Specifies a list of pins or instances where the paths go through.
`-to <to_list>`	Specifies a list of pins, instances, or clocks where the path ends.

Examples

The following commands specify the adjusted path groups:

- `set_path_adjust_group -name AG1 -from in1 -through buf1/A -to out1`
- `set_path_adjust_group -name AG2 -from [get_clocks CK1] -through buf1/A -to [get_clocks CK2]`

- `set_path_adjust_group -name AG3 -from [get_clocks {CK1 CK2}] -to [get_clocks {CK1 CK2}]`

Related Commands

- `set_path_adjust`
- `reset_path_adjust_group`
- `group_path`

set_pba_mode

```
set_pba_mode
[-help]
[-reset]
```

Enables IPBA infinite depth path-based analysis (IPBA).

This feature runs faster analysis (vs full depth exhaustive PBA (EPBA)) and attains PBA design coverage with no depth limit per endpoint.

Parameters

-help	Prints out the command usage.
-reset	Resets specified or all the options to the default value.

Related Commands

- report_timing

set_socv_constraint_config

```
set_socv_constraint_config
[-help]
[-hold_folding float]
-libs string
[-nochange_hold_folding float]
[-nochange_setup_folding float]
[-setup_folding float]
```

Enhances the built-in sigma pessimism of check values for setup and hold constraints in the library. The folding factor attribute records the amount of on-chip-variation pessimism added during cell characterization to the nominal setup/hold tables in the libraries. The folding factor attributes are library level attributes.

Notes

- The `set_socv_constraint_config` command does not reset the timing; so it should be used before the `report_timing` command.
- To change the setting later in the flow, you need to use the `update_timing -full` command.
- You cannot use this command, if any of the following global variables are specified:
  `timing_library_setup_sigma_multiplier`
  `timing_library_hold_sigma_multiplier`

Parameters

`-help`	Prints out the command usage.
`-hold_folding` *float*	Specifies a number of additional sigma units added on top of the existing nominal hold value. *Min* = -100 *Max* = 100
`-libs` *string*	Specifies a list of library names (not library paths) or collection objects.
`-nochange_hold_folding` *float*	Specifies a number of additional sigma units added on top of the existing nominal hold value for no-change timing arcs. *Min* = -100 *Max* = 100
`-nochange_setup_folding` *float*	Specifies a number of additional sigma units added on top of the existing nominal setup value for no-change timing arcs. *Min* = -100 *Max* = 100
`-setup_folding` *float*	Specifies a number of additional sigma units added on top of the existing nominal setup value. *Min* = -100 *Max* = 100

Examples

- The following command is used when an additional margin of 10 sigma units is applied to the nominal hold tables in the library collection test.lib:

```
set_socv_constraint_config -hold_folding 10 -libs [get_libs test.lib]
```

Related Commands

- report_timing
- update_timing

set_socv_rc_variation_factor

```
set_socv_rc_variation_factor
[-help]
variation_factor
[-early]
[-late]
[-view list_of_views]
```

Defines the value of interconnect delay variation factor.

This command applies global variation as percentage of the nominal delay.

To enable this feature, you can set the `timing_socv_rc_variation_mode` global variable to `true`.

Parameters

`-help`	Prints out the command usage
`variation_factor`	Specifies the value of the interconnect delay variation factor to be used.
`-early`	Specifies the variation factor for early interconnect delays.
`-late`	Specifies the variation factor for late interconnect delays.
`-view` *`list_of_views`*	Apply the SOCV interconnect variation factor to the specified list of analysis views.

Related Commands

- `get_socv_rc_variation_factor`

set_socv_reporting_nsigma_multiplier

```
set_socv_reporting_nsigma_multiplier
[-help]
[-view list_of_views]
{[-setup float] [-hold float]}
[-transition float]
```

Controls n-sigma value for reporting purposes.

This command supports n-sigma specification per view per setup/hold combination.

Parameters

`-help`	Prints out the command usage.
`-hold float`	Specifies the sigma multiplier for hold mode.
`-setup float`	Specifies the sigma multiplier for setup mode.
`-transition float`	Specifies the sigma multiplier to use for transition related DRV checks – including max_transition, min_transition, pulse_clock_max_transition, and pulse_clock_min_transition. When this parameter is not specified, the sigma multiplier specified for setup will be used for transitions. When the `-transition` parameter is specified, you can set the `timing_report_max_transition_check_using_nsigma_slew` global variable to `true` to report transition related DRV checks, such as, max_transition, min_transition, pulse_clock_max_transition, and pulse_clock_min_transition. This setting enables the `report_constraint` command to use mean and sigma slew values for reporting transition values in DRV checks.
`-view list_of_views`	Applies the sigma multiplier factor to the specified list of analysis views. **Note**: The specified views should be active.

Examples

- The following command applies the sigma multiplier to the specified view in setup mode :
  ```
  set_socv_reporting_nsigma_multiplier -setup 2.5 -view std_max_setup
  ```
- The following command applies the sigma multiplier to the specified view in for hold mode:
  ```
  set_socv_reporting_nsigma_multiplier -hold 3.0 -view std_min_hold
  ```

Related Commands

- get_socv_reporting_nsigma_multiplier

set_table_style

```
set_table_style
-help
-name string
[-reverse_rows]
[-major_sort integer]
[-minor_sort integer]
[-max_widths integer]
[-min_widths integer]
[-indent integer]
-frame | -no_frame_fix_width [-nosplit]
```

Disables printing of specific columns and allows for specifying the minimum and maximum size of each column in a report. It allows for reversing the data, for controlling the left indent, and for controlling the sorting of the columns.

Note: Do *not* include a blank space between commas "*,* " when specifying widths. If you are specifying a value for the width(s), make sure there is *no* leading or trailing blank space.

The `-frame`, `-no_frame`, and `-no_frame_fix_width` parameters work in a global manner. If you specify one of these parameters, it will affect all reports that can be formatted with this command. You cannot specify them with the -name parameter, to change the format of a specific report.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_table_style` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_table_style`.
`-frame`	Specifies that the table columns have a fixed frame or border, based on the minimum and maximum width settings. Any data that is longer than the maximum width for its column is wrapped within the same column. This is the default format for tables.
`-indent integer`	Specifies the number of spaces to leave on the left for that particular table.

`-major_sort` *integer*	Specifies the column to be used in the major sort. If the column number is negative, it reverses the sort.

`-max_widths` *integer*	Specifies the maximum width for each table column in the report specified by the `-name` parameter. **Note**: You cannot specify `-max_widths` parameter without `-name`. You can separate each specified width value with a comma (,). For example, if you specify: -max_widths `{10,20,30}` The first column will have a maximum width of 10, the second column will have a maximum width of 20, and the third column will have a maximum width of 30. The fourth and fifth columns will remain the default width spacing. Commas are also specified as column "placeholders", when you want to change the spacing of a column without changing the spacing of the preceding column. For example, if you specify: -max_widths {,,20,30} The third column will have a maximum width of 20, the fourth column will have a maximum width of 30, and the first, second, and fifth columns will remain the default width spacing. If you specify: `-max_widths {20,,30}` The first column will have a maximum width of 20, the third column will have a maximum width of 30, and the second, fourth and fifth columns will remain the default width spacing. Each column resizes itself to be between the range of the maximum and minimum widths specified for that column, based on the data in that column. Overruns are wrapped within the same column, unless otherwise specified. *Default:* `{50,50,50,50,50}`

`-min_widths` *integer*	Specifies the minimum width for each table column in the report specified by the `-name` parameter. **Note**: You cannot specify `-min_widths` parameter without `-name`. You can separate each specified width value with a comma (,). For example, if you specify: -min_widths `{7,10,15}` The first column will have a minimum width of 7, the second column will have a minimum width of 10, and the third column will have a minimum width of 15. The fourth and fifth columns will remain the default width spacing. Commas are also specified as column "placeholders", when you want to change the spacing of a column without changing the spacing of the preceding column. For example, if you specify: -min_width {,,10,15} The third column will have a minimum width of 10, the fourth column will have a minimum width of 15, and the first, second, and fifth columns will remain the default width spacing. If you specify: -min_width {7,,10} The first column will have a minimum width of 7, the third column will have a width of 10, and the second, fourth and fifth columns will remain the default width spacing. Each column resizes itself to be between the range of the maximum and minimum widths specified for that column, based on the data in that column. Overruns are wrapped within the same column, unless otherwise specified. *Default:* `{5,5,5,5,5}`

`-minor_sort` *integer*	Specifies the column to be used in the minor sort. It applies to *all* rows that have the same values in the `major_sort` column. If the column number is negative, the software reverses the sort.
`-name` *string*	Changes the formatting for the specified table. See Table for a list of table names you can use, and the commands with which they work.
`-no_frame_fix_width`	Specifies that the table columns have a fixed minimum width, but no maximum width (the data does not wrap). Column data that fits within the column's minimum width will left-justify and pad its location to fill the specified column space. Column data that is longer than the column's minimum width will take as much space in the table as needed. If a column is overwritten by the data from the preceding column, its data will begin on a new line at the correct column location. *Default:* `-frame`
`-nosplit`	Specifies that each column follows the preceding column without splitting to a new line, if the data in one column overruns the next column. The wrapping behavior for data is controlled by the size of the display window. This format is useful for machine parsing the report. **Note:** You must specify `-no_frame_fix_width` in order to use this parameter.
`-reverse_rows`	Reverses the data rows. This is useful, for example, if you want to trace back from an end point and the default report traces forward from the end point.

The timing report formatting depends on the specified `set_table_style` parameters. These are described below:

- When `set_table_style -no_frame_fix_width` parameter is specified, the timing report displays long inst/hpin names and the rest of the column values are shown on the next line.

- When `set_table_style -no_frame_fix_width -nosplit` is specified, the timing report displays long inst/hpin names and rest of the column values are shown on the same line, because `-nosplit` table style is specified. In this case, other column headers (beyond inst/pin) may not be aligned properly.
- The following settings are specified:

  ```
  set_table_style -no_frame_fix_width -nosplit
  set_table_style -name report_timing -max_width {50,50,,,,,}
  ```

  The generated report will show properly aligned column headers and end-point/begin-points are not split.
- The following settings generate a report displaying longer (than above reports) inst/pin names:

  ```
  set_table_style -no_frame_fix_width -nosplit
  set_table_style -name report_timing -max_width {120,120,,,,,}
  ```

  In this report output, the column headers are aligned properly and are not split.

Table: **List of Table Names**

Table Name	Formats Reports Generated by
`report_cell_instance_delays_arcs`	`report_cell_instance_timing`
`report_cell_instance_delays_checks`	`report_cell_instance_timing`
`report_cell_instance_delays_pins`	`report_cell_instance_timing`
`report_inactive_arcs`	`report_inactive_arcs`
`report_net`	`report_net`
`report_net_header`	`report_net`
`report_net_sink_table`	`report_net`
`report_net_source_table`	`report_net`
`report_net_summary_table`	`report_net`
`report_path_exceptions`	`report_path_exceptions`
`report_port`	`report_ports`

report_timing	report_timing
report_timing_summary	report_timing

Examples

- The following command resizes the first column to a maximum range of 70 and specifies a default width of 50 for the second column. The default for the maximum width is 50 and the default for the minimum width is 5:

  ```
  set_table_style -name report_timing -max_widths {70,}
  ```

  Note: Do *not* include a blank space between commas ",*,* " when specifying widths. If you are specifying a value for the width(s), make sure there is *no* leading or trailing blank space.

- The following command specifies a default width of 50 for the first and second column:
  set_table_style -name report_timing -max_widths {,}

  The following sample timing report shows the table style generated with the `-frame` parameter:

  ```
  set_table_style -frame

  report_timing -format {cell arc delay arrival required instance}

  Path 1: MET Setup Check with Pin seg3/u14/CK
  Endpoint: seg3/u14/D (^) checked with leading edge of 'CLK_W_3'
  Beginpoint: seg3/u9/Q (^) triggered by leading edge of 'CLK_W_3'
  Path Groups: {CLK_W_3}
  Other End Arrival Time 0.621
  - Setup 0.186
  + Phase Shift 2.000
  = Required Time 2.434
  - Arrival Time 2.062
  = Slack Time 0.372
  Clock Rise Edge 0.000
  + Clock Network Latency (Prop) 0.682
  = Beginpoint Arrival Time 0.682
  +-------------------------------------------------------------------------+
  | Cell    | Arc          | Delay    | Arrival  | Required  | Instance  |
  |         |              |          | Time     | Time      |           |
  |------+-------------+-------+---------+----------+--------------------|
  |         | CK ^         |          | 0.682    | 1.054     | seg3/u9   |
  | DFF     | CK ^ -> Q ^  | 0.185    | 0.867    | 1.240     | seg3/u9   |
  ```

```
| BUF     | A ^ -> Y ^  | 0.073    | 0.940    | 1.312     | seg3/u10  |
| INV     | A ^ -> Y v  | 0.317    | 1.257    | 1.629     | seg3/u11  |
| INV     | A v -> Y ^  | 0.402    | 1.659    | 2.031     | seg3/u12  |
| BUF     | A ^ -> Y ^  | 0.403    | 2.062    | 2.434     | seg3/u13  |
| DFF     | D ^         | 0.000    | 2.062    | 2.434     | seg3/u14  |
+-------------------------------------------------------------------------+
```

The following sample timing report shows the table style generated with the `-no_frame_fix_width` parameter (`set_table_style -no_frame_fix_width`):

```
> report_timing -format {cell arc delay arrival required instance}
Path 1: MET Setup Check with Pin seg3/u14/CK
Endpoint: seg3/u14/D (^) checked with leading edge of 'CLK3'
Beginpoint: seg3/u9/Q (^) triggered by leading edge of 'CLK3'
Path Groups: {CLK3}
Other End Arrival Time 0.621
- Setup 0.186
+ Phase Shift 2.000
= Required Time 2.434
- Arrival Time 2.062
= Slack Time 0.372
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.682
= Beginpoint Arrival Time 0.682
-------------------------------------------------------------------
Cell      Arc           Delay    Arrival    Required    Instance
                                 Time       Time
-------------------------------------------------------------------
          CK ^                   0.682      1.054        seg3/u9
DFF       CK ^ -> Q ^   0.185    0.867      1.240        seg3/u9
BUF       A ^ -> Y ^    0.073    0.940      1.312        seg3/u10
INV       A ^ -> Y v    0.317    1.257      1.629        seg3/u11
INV       A v -> Y ^    0.402    1.659      2.031        seg3/u12
BUF       A ^ -> Y ^    0.403    2.062      2.434        seg3/u13
DFF       D ^           0.000    2.062      2.434        seg3/u14
-------------------------------------------------------------------
```

The following sample timing report shows the table style generated with the `-no_frame_fix_width` and `-nosplit` parameters (`set_table_style -no_frame_fix_width -nosplit`):

```
> report_timing -format {cell arc delay arrival required instance}
```

```
Path 1: MET Setup Check with Pin seg3/u14/CK
Endpoint: seg3/u14/D (^) checked with leading edge of 'CLK3'
Beginpoint: seg3/u9/Q (^) triggered by leading edge of 'CLK3'
Path Groups: {CLK3}
Other End Arrival Time 0.621
- Setup 0.186
+ Phase Shift 2.000
= Required Time 2.434
- Arrival Time 2.062
= Slack Time 0.372
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.682
= Beginpoint Arrival Time 0.682
-------------------------------------------------------------------------
Cell      Arc           Delay    Arrival    Required   Instance
                                 Time       Time
-------------------------------------------------------------------------
CK ^                              0.682      1.054      seg3/u9
DFF       CK ^ -> Q ^   0.185     0.867      1.240      seg3/u9
BUF       A ^ -> Y ^    0.073     0.940      1.312      seg3/u10
INV       A ^ -> Y v    0.317     1.257      1.629      seg3/u11
INV       A v -> Y ^    0.402     1.659      2.031      seg3/u12
BUF       A ^ -> Y ^    0.403     2.062      2.434      seg3/u13
DFF       D ^           0.000     2.062      2.434      seg3/u14
-------------------------------------------------------------------------
```

Related Information

- report_cell_instance_timing
- report_inactive_arcs
- report_net
- report_path_exceptions
- report_ports
- report_timing

set_timing_derate

```
set_timing_derate
[-help]
derate_value
[object_list]
[-add]
[-cell_check]
[-cell_delay]
[-clock]
[-clock_period_check]
[-corner]
[-data]
[-dynamic]
[-early]
[-fall]
[-increment]
[-input_switching]
[-late]
[-mean]
[-multiply]
[-net_delay]
[-power_domain string]
[-pulse_width_check]
[-retain_delay]
[-rise]
[-sigma]
[-static]
[-statistical]
[-transition]
[-min | -max | -delay_corner delayCornerName]
```

Sets delay scaling or derating factors for early and late paths in the current design. You use this command to perform scaled on-chip variation around best-case and worst-case corners. The timing scaling factors affect the delay values generated in the timing reports. You can set scaling factors for data paths, clock paths, minimum and maximum operating conditions.

If you do not specify the `-clock` or `-data` parameter, the software applies the scaling factors to both clock and data paths. If you specify only one of the two parameters, `-data` or `-clock`, the software uses the last defined value for the other.

Similarly, if you do not specify the `-min` or `-max` parameter, the software applies the scaling factor to both minimum and maximum operating conditions. You should only use the `-min` or `-max` parameter for best-case worst-case analysis. If you specify them in on-chip variation (OCV) mode, the software will

use the `-min -early` derating factor for early path scaling, and the `-max -late` derating factor for late path adjustments.

When there are separate max and min libraries specified, only those `set_timing_derate` constraints are honored that are specified with max library. The constraints specified with min library are ignored.

In MMMC mode, for delay corners having different late and early library sets, only those `set_timing_derate` constraints are honored that are specified with late library set of the delay corner. The constraints specified with early library set are ignored.

Note: The `set_timing_derate` command can only be used on the command line. You cannot import it as part of an SDC file.

Parameters

`-add`	Incrementally adds or subtracts derates. You can also apply derates on specific leaf instances.
`-cell_check`	Specifies scaling on timing checks. Use the `-cell_check` option with the `-early` option to specify scaling on hold arcs and use with the `-late` option to specify scaling on setup arcs. When you specify this parameter, derating factors are also applied to user constraints such as `set_output_delay`, `set_data_check`, and `set_clock_gating_check`. The `-cell_check` parameter by default does not apply check derate to output external delay (e.g., `set_output_delay`). To enable this feature, you can set the `timing_apply_check_derate_to_external_output_delay` timing global variable to `true`. When set to `true`, this global applies cell derate to the top boundary port.
`-cell_delay`	Specifies scaling on cell delays. If neither the `-cell_delay` nor the `-net_delay` option is specified, the specified scaling factor applies to both types of delays.

`-clock`	Applies the derating factors to clock paths only.
`-clock_period_check`	Specifies derate values for clock period checks.
`-corner`	Applies derating on cells or nets that are modeled as corner data elements. The specified derating factor is applied to both the mean and sensitivities of the delays (corner objects can have propagated sensitivities). A cell with no timing arc data in a library that has any sensitivity information with respect to any parameter defined in SPDF are considered as a corner object. All cells that do not qualify are considered to be statistical objects. A net with parasitic data that has no sensitivity information with respect to any parameter defined in SPDF is considered as a corner object. All nets that do not qualify are considered to be statistical objects.
`-data`	Applies the derating factors to data paths only.
`-delay_corner` *`delayCornerName`*	Applies the derating factor to the specified delay calculation corner or a list of delay corners. **Note**: The timing derate can be applied to corners associated with active views only.
`derate_value`	Specifies the derating value to be used.
`-dynamic`	Applies the specified derating factor to the delta portion of net delays only. You can specify `-net_delay` or `-cell_delay` parameter with the `-dynamic` parameter. You cannot specify `-dynamic` with `-mean` or `-sigma` parameters.

`-early`	Specifies that the derating factor applies to early paths. Early paths are launching clock during hold analysis, receiving clock during setup analysis and datapath during hold analysis. For definition of early and late paths, see "Definition of Early Late Paths" in *Innovus User Guide*. **Note**: To use this parameter, you must specify `-delay_corner`.
`-fall`	Applies the derating factor to falling delay and check arcs based on the transition direction at the arc's sink pin. This parameter can be used to selectively apply timing derates for the fall arcs (cell, net, or check arcs).
`-increment`	Incrementally adds the derate value to the total derate value in all the modes - OCV, AOCV, and SOCV mode.
`-input_switching`	Considers the given derate as simultaneous switching input (SSI) derate that is applicable to early paths only.
`-late`	Specifies that the derating factor applies to late paths. Late paths are receiving clock during hold analysis, launching clock during setup analysis and datapath during setup analysis. **Note**: To use this parameter, you must specify the `-delay_corner` parameter.
`-max`	Applies the derating factor to the maximum operating condition during best-case worst-case analysis. This parameter is ignored during on-chip variation analysis. **Note:** You cannot use the `-max` parameter when the software is in multi-mode multi-corner (MMMC) mode. You must use the `-delay_corner` parameter instead.

`-min`	Applies the derating factor to the minimum operating condition during best-case worst-case analysis. This parameter is ignored during on-chip variation analysis. **Note:** You cannot use the `-min` parameter when the software is in multi-mode multi-corner (MMMC) mode. You must use the `-delay_corner` parameter instead.
`-mean`	Controls nominal derating. You can apply separate derating the mean components of the delay.
`-multiply`	Incrementally multiplies derates. You can also apply derates on specific leaf instances.
`-net_delay`	Specifies scaling on net delays.
`object_list`	Specifies the list of instances, library cells, or nets to which the derating will be applied.
`-power_domain` *`string`*	Allows you to specify a specific power domain for the timing derate factors to be applied.
`-pulse_width_check`	Specifies derate values for pulse width checks. **Note:** The `-pulse_width_check` and `-clock_period_check` parameters are mutually exclusive. To apply check derating to pulse width checks, set the `timing_enable_derating_for_pulsewidth_checks` global variable to `true`. By default, the global is set to `false`.
`-retain_delay`	Specifies scaling on retain constructs. If you specify both `-retain_delay` and `-cell_delay` parameters, the software uses the retain_delay value for retain construct scaling. If you do not specify the `-retain_delay` parameter, retain constructs are scaled in the same manner as cell delays.

`-rise`	Applies the derating factor to rising delay and check arcs based on the transition direction at the arc's sink pin. This parameter can be used to selectively apply timing derates for the rise arcs (cell, net, or check arcs).
`-sigma`	Controls derating on the variation component. You can apply separate derating to the sigma components of the delay.
`-static`	Applies the specified derating factor to the non-delta portion of net delays only. You can specify `-net_delay` or `-cell_delay` parameter with the `-static` parameter. You cannot specify `-static` with `-mean` or `-sigma` parameters.
`-statistical`	Applies derating on cells or nets that are modeled as statistical data elements. The specified derating factor is applied to both the mean and sigma of delays. If both `set_timing_derate -corner` and `-statistical` options are not specified, the derating factor is applied on variation and corner objects equally. This is equivalent to issuing to two separate `set_timing_derate` commands, one with `-statistical` option and the other with `-corner` option.
`-transition`	Specifies scaling on transitions. When specified, the software allows derate transition and stretch waveforms.

Examples

- The following command (1) can be used when the software is in on-chip variation analysis mode to add pessimism to both the setup and hold analyses by making the early paths 20 percent faster:

```
setAnalysisMode -analysisType onChipVariation
    ...
(1) set_timing_derate -early 0.8 -late 1.0
```

- The following example uses two set_timing_derate commands to set the scaling factors independently for the min and max corners. When the software is in best-case worst-case analysis mode:
  - Command (1) adds pessimism to the setup analysis by making the latching clock path 20 percent faster.
  - Command (2) adds pessimism to the hold analysis by making the latching clock path 10 percent slower.

```
(1) set_timing_derate -max -early 0.8 -late 1.0
(2) set_timing_derate -min -early 1.0 -late 1.1
```

- When the software is in on-chip variation analysis mode, there is only one active delay corner; therefore, the `-min` and `-max` parameters should not be used. The following command sequence will generate a warning:

```
setAnalysisMode -analysisType onChipVariation
...
set_timing_derate -min -early 0.8 -late 0.9
set_timing_derate -max -early 1.1 -late 1.2
```

  In this situation, the software uses `0.8` for derating early paths, and `1.2` for derating late paths.

- When the software is in on-chip variation analysis mode, there is only one active delay corner; therefore, the `-min` and `-max` parameters should not be used. The following command sequence will generate a warning:

```
setAnalysisMode -analysisType onChipVariation
...
set_timing_derate -min -early 0.8 -late 0.9
set_timing_derate -max -early 1.1 -late 1.2
```

  In this situation, the software uses `0.8` for derating early paths, and `1.2` for derating late paths.

- The following example uses two `set_timing_derate` commands to set the scaling factors

independently for the min and max corners. When the software is in best-case worst-case analysis mode:

  - Command (1) adds pessimism to the setup analysis by making the latching clock path 20 percent faster.

- The following command sets the early path scaling factor of 0.85 for instance `MULT_9` cell delays:

  ```
  set_timing_derate -early -cell_delay 0.85 [get_cells {MULT_9}]
  ```

- The following command sets the early path scaling factor of 0.75 for the net named `MUX2/Z`:

  ```
  set_timing_derate -net_delay -early 0.75 [get_nets {MUX2/Z}]
  ```

- The following command specifies power domain `PD1` for derating instances:

  ```
  set_timing_derate -power_domain PD1 -cell_delay 1.1
  ```

- In the following example, a design having max library as 'slow' and min library as 'fast', only the first two `set_timing_derate` constraints are honored:

  ```
  set_timing_derate -late 1.2 [get_lib_cells {slow/BUFX1}] -cell_delay
  set_timing_derate -early 0.8 [get_lib_cells {slow/BUFX1}] -cell_delay
  set_timing_derate -late 1.1 [get_lib_cells {fast/BUFX1}] -cell_delay
  set_timing_derate -early 0.9 [get_lib_cells {fast/BUFX1}] -cell_delay
  ```

- Consider the following library sets in MMMC mode:

  ```
  create_library_set -name libset1 -timing slow.lib
  create_library_set -name libset2 -timing fast.lib
  create_delay_corner -name corner1 -late_library_set libset1 -early_library_set
  libset2
  ```

  From the following `set_timing_derate` constraints the first and last are honored:

  ```
  set_timing_derate -late 1.2 -cell_delay [get_lib_cells slow/*] -delay_corner corner1
  set_timing_derate -early 2.3 -cell_delay [get_lib_cells slow/*] -delay_corner corner1
  set_timing_derate -late 3.4 -cell_delay [get_lib_cells fast/*] -delay_corner corner1
  set_timing_derate -early 4.5 -cell_delay [get_lib_cells fast/*] -delay_corner corner1
  ```

- The following command can be used when the software is in on-chip variation analysis mode to add pessimism to both the setup and hold analyses by making the early paths 20 percent faster:

  ```
  setAnalysisMode -analysisType onChipVariation
  ...
  set_timing_derate -early 0.8 -late 1.0
  ```

  This command adds pessimism to the setup analysis by making the latching clock path 20

percent faster.

- The following example uses two `set_timing_derate` commands to set the scaling factors independently for the min and max corners. When the software is in best-case worst-case analysis mode:
  - The following command sets the early path scaling factor of 0.85 for instance MULT_9 cell delays:
    ```
    set_timing_derate -early -cell_delay 0.85 [get_cells {MULT_9}]
    ```
  - The following command sets the early path scaling factor of 0.75 for the net named MUX2/Z:
    ```
    set_timing_derate -net_delay -early 0.75 [get_nets {MUX2/Z}]
    ```
- The following command specifies power domain PD1 for derating instances:
  ```
  set_timing_derate -power_domain PD1 -cell_delay 1.1
  ```
- The following command shows library object level support for clock (and pulse width) check options:
  ```
  set_timing_derate -delay_corner [all_delay_corners] -clock_period_check 1.05
  [object_list]
  ```

Related Information

- `reset_timing_derate`
- `report_timing_derate`
- `setAnalysisMode`

set_wire_load_mode

```
set_wire_load_mode
[-help]
{top | enclosed | segmented}
```

Controls how the software searches through the design hierarchy to find the appropriate wire load model for a net, or a hierarchical part of a net.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_wire_load_mode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_wire_load_mode`.
*`{top	enclosed	segmented}`*	Specifies the wire load model. You can specify one of the following options. • `enclosed` • `segmented` • `top`

	`enclosed`: Selects the wire load model using an area lookup table for the module that is at the lowest level in the design hierarchy and contains the entire net. When you set the mode to `enclosed`, the software uses the area of the smallest block enclosing the net to select the wire load model. In the `enclosed` mode, the software uses the following order of precedence to search the wire load model in: • The inner-most block enclosing the net on which you set the wire load selection group using the `set_wire_load_selection_group` command, if the `auto_wire_load_selection` global variable is enabled. Or: The inner-most block enclosing the net on which you set the wire load model using the `set_wire_load_model` command. • Each higher level hierarchical block for a `set_wire_load_selection_group` assertion, all the way up to the top-level block, if the `auto_wire_load_selection` global variable is enabled. Or: Each higher level hierarchical block for a `set_wire_load_model` assertion, all the way up to the top-level block. • The `default_wire_load_selection` field in the first timing library. Or, The `default_wire_load` field in the first timing library. **Note:** If even the default values are not present in the timing library, you will not be able to use the wire load model functionality.

	`segmented`: Selects the wire load model in a way similar to the `enclosed` mode except that each hierarchical segment of the net is treated separately. In the `segmented` mode: • A segment is defined by pins (including hierarchical pins) connected to a particular net. • When counting the drivers and receivers, only the pins connected to the segment are considered. This information along with the wire load models determine the parasitic information for the segment. • The parasitics of each segment sum up to form the parasitics for the entire net. **Note:** Each segment may have different wire load models.
	`top`: Selects the wire load model in a way similar to `enclosed` and `segmented` except that all nets use the wire load selection or wire load models defined by the top-level (largest) block enclosing the net. **Note:** If the first library read in does not define the default wire load mode, the software uses `top` as the default mode.

Examples

- The following command sets the wire load mode to `enclosed`:

```
set_wire_load_mode enclosed
```

Related Commands

- `reset_wire_load_mode`

set_wire_load_model

```
set_wire_load_model
[-help]
[list_of_instances_or_ports]
[-library library_name]
[-max]
[-min]
-name wireload_model
```

Specifies the wire load model to be used from the technology library, and sets the wire load model on the design or on a hierarchical object (instance). If the list of instances is not specified, the wire load model is set on the current instance. Otherwise, the wire load model is set on all the specified instances or ports.

Use wire load models for estimating delays before the actual wire loads are back annotated. A technology library contains different wire load models, previously computed based on the analysis of several designs differing in area.

Parameters

`-help`	Prints out the command usage.
`-library` *library_name*	Specifies the technology library to use when searching for the name of the wire load model. You can specify a library name, or a collection of libraries. *Default:* All technology libraries are searched.
list_of_instances_or_ports	Specifies the list of objects - designs, ports, or hierarchical instances.
`-max`	Sets the wire load model only for the maximum PVT (process, voltage, temperature) corner. *Default*: The specified wire load model applies to all three PVT corners if neither the `-min` nor the `-max` parameter is specified.
`-min`	Sets the wire load model only for the minimum PVT (process, voltage, temperature) corner. *Default*: The specified wire load model applies to all three PVT corners if neither the `-min` nor the `-max` parameter is specified.
`-name` *wireload_model*	Specifies the name of the wire load model in the library.

Examples

- The following command sets the `COM3_LV_WLM_0.0` wire load model from the `COM3_LV_WIRE` library on the current module for all PVT corners:
  ```
  set_wire_load_model -library COM3_LV_WIRE -name COM3_LV_WLM_0.0
  ```
- The following command sets the `B1X1` wire load from the `lca500kv` library on the `clk1` port for the minimum PVT corner:
  ```
  set_wire_load_model -library lca500kv -min -name B1X1 clk1
  ```
- The following command sets the `3M_TO_4500K`wire load from the `c13tsmc_slow2s108v125c` library on the current module for the minimum PVT corner:
  ```
  set_wire_load_model -library c13tsmc_slow2s108v125c -min -name 3M_TO_4500K
  ```

Related Commands

- report_wire_load
- reset_wire_load_model

set_wire_load_selection_group

```
set_wire_load_selection_group
[-help]
wireload_selection_group_name
[list_of_instances]
[-library library_name]
[-max]
[-min]
```

Overrides the wire load selection table in the specified library.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_wire_load_selection_group` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_wire_load_selection_group`
`-library` *library_name*	Specifies the name of the technology library to search for the wire load selection table. When this parameter is not specified, the software searches all the libraries.
list_of_instances	Specifies the list of instances.
`-max`	Controls the choice of a wire load selection table only for the PVT max corner. *Default*: Selects the table for all PVT corners if neither the `-min` nor the `-max` parameter is specified.

`-min`	Controls the choice of a wire load selection table only for the minimum PVT corner. *Default*: Selects the table for all PVT corners if neither the `-min` nor the `-max` parameter is specified.
`wireload_selection_group_name`	Specifies the name of the wire load selection group.

Examples

- The following command selects `TLM` as the selection table used by the library:
  ```
  set_wire_load_selection_group TLM
  ```
- The following command selects `ASPEC1` as the selection table used by the `sm333s` library; only the minimum PVT corner is affected:
  ```
  set_wire_load_selection_group -min -library sm333s ASPEC1
  ```

Related Commands

- `reset_wire_load_selection_group`

setAnalysisMode

```
setAnalysisMode
[-help]
[-reset]
[-analysisType {single | bcwc | onChipVariation}]
[-asyncChecks {async | noAsync | asyncOnly}]
[-caseAnalysis {true | false}]
[-checkType {setup | hold}]
[-clkNetsMarking {beforeConstProp | afterConstProp}]
[-clkSrcPath {true | false}]
[-clockGatingCheck {true | false}]
[-clockPropagation {sdcControl | forcedIdeal | autoDetectClockTree}]
[-aocv {true | false}]
[-cppr {none | both | setup | hold}]
[-honorActiveLogicView {true | false}]
[-log {true | false}]
[-multi_input_switching_mode {0 | 1 | 2}]
[-propSlew {true | false}]
[-sequentialConstProp {true | false}]
[-skew {true | false}]
[-socv {true | false}]
[-timeBorrowing {true | false}]
[-timingEngine {statistical | static}]
[-timingSelfLoopsNoSkew {true | false}]
[-usefulSkew {true | false}]
[-useOutputPinCap {true | false}]
[-warn {true | false}]
```

Sets global analysis modes for timing analysis. The software uses these modes for all timing analysis commands unless you specify specific modes in the reporting commands. The modes that you specify in the reporting commands override these modes.

When you use the `setAnalysisMode` command, the software sets the following defaults:

- Sets the constants to be propagated while building the timing graph. The software reads the constants from the timing constraints file or the netlist.
- Sets the analysis mode to accept the module-level timing constraints of the internally specified *`set_input_delay`* and *`set_output_delay`* variables for the I/Os of the module for running timing analysis and timing optimization.
  The software applies the `set_input_delay` command and the `set_output_delay` command on internal pins during timing analysis.

- Treats the latches as latches.
- Reports the end points as many times as the number clocks that control that end point.

Running "`timeDesign -postRoute -si [-hold]`" with "`setAnalysisMode -analysisType bcwc`" might lead to incorrect timing analysis results. This is because the SI delay push-outs (or pull-ins for -hold mode) on the clock nets get annotated to both the launch and the capture clock paths resulting in optimistic slacks to be reported. To get accurate timing analysis results, use "`setAnalysisMode -analysisType onChipVariation`", which requires the design to be run either in multi-mode multi-corner (MMMC) or single-mode single-corner (SMSC) mode. For more information, see the Specifying the MMMC Environment section in the Optimizing Timing chapter of the *Innovus User Guide*.

Parameters

`-help`	Prints out the command usage.

`-analysisType {single | bcwc | onChipVariation}`	Sets the timing analysis type to single, best case worst case, or on-chip variation. *Default*: If you read in one library and do not specify any timing analysis type, the software uses `single` by default. If you read in two libraries and do not specify any timing analysis type, the software uses `bcwc` by default. Scales the delay values based on one operating condition. For more information, see "Single Timing Analysis Mode," in the Innovus User Guide. Checks the design for two extreme operating conditions. The software uses the maximum delays for all paths during setup checks and minimum delays for all paths during hold checks. For more information, see "Best-Case Worst-Case (BC-WC) Timing Analysis Mode," in the Innovus User Guide. `onChipVariation` Calculates the delay for one path based on maximum operating condition while calculating the delay for another path based on minimum operating condition for setup or hold checks. For more information, see "On-Chip Variation (OCV) Timing Analysis Mode," in the Innovus User Guide.
`-aocv {true | false}`	Sets the analysis mode to advanced on-chip variation (AOCV) analysis.
`-asyncChecks {async | noAsync | asyncOnly}`	Reports timing violations for asynchronous timing checks, including recovery and removal, and other checks on pins defined as preset or clear in the Liberty file. *Default:* `async` Reports timing checks on pins that are defined as preset or clear in the Liberty file. Reports timing violations while ignoring the two time arc attributes. Reports only recovery and removal checks.

`-caseAnalysis {true | false}`	Determines whether constants supplied from the constraint file and netlist should be applied to the analysis. Specifying `-caseAnalysis false` parameter blocks the application of constants from the timing constraints file. However, constants that are introduced through the netlist and timing library are still applied and propagated. *Default:* `true`
`-checkType {setup | hold}`	Checks the design for setup or hold violations in the current analysis. *Default:* `setup` **Note:** When simultaneous setup/hold mode analysis is on, this parameter does not have any impact. You can use the `report_timing -late/-early` parameter to report setup/hold timing reports.
`-clkNetsMarking {beforeConstProp | afterConstProp}`	**Note:** This parameter is not available in CTE timing analysis mode. Marks clock nets before or after propagating the constants. *Default:* `markClkNetsBeforeConstProp`

`-clkSrcpath {true | false}`	Reports combinational paths from the clock source to the timing checks on data pins. **Note:** If `-clkSrcPath false` is specified and the clock portion of the path is considered as ideal (due to the SDC constraints or the `-clockPropagation forcedIdeal` parameter setting), the delay calculation software treats the clock nets as ideal nets when calculating the delays and slews along the data path. For all pre-clock tree synthesis (CTS), `optDesign`, and `timeDesign` steps, the software uses `-clkSrcPath false` in combination with `-clockPropagation forcedIdeal` to calculate the delays and slews along the data path. For all post-CTS flow steps, the software uses the default `-clkSrcPath true` in combination with `-clockPropagation sdcControl` to calculate the delays and slews along the data path. *Default:* `true`
`-clockGatingCheck {true | false}`	Reports the gating checks. *Default:* `true`

`-clockPropagation {sdcControl | forcedIdeal | autoDetectClockTree}`	Specifies how to determine whether clock end points are interpreted as having either ideal or propagated clock timing. *Default:* `sdcControl` Reports the timing violations considering clock tree and enables propagating clocks to report clock latency. You must set the `set_propagated_clock` constraint to propagate the clocks. **Note**: By default, CTS will instruct the timer to make all of the clock roots propagated. In this case, you do not need to set the `set_propagated_clock` constraint. Does not report the timing violations considering the clock tree. `autoDetectClockTree` **Note:** The `autoDetectClockTree` parameter is obsolete. The timing system will revert to `sdcControl`, if this parameter is specified.

`-cppr {none | both | setup | hold}`	Removes pessimism from clock paths that have a portion of the clock network in common between the clock source and clock destination paths. The pessimism is introduced when the timing analysis tools assume that the common path has different delay values for two different paths in case of on-chip variation. The pessimism can also be because of clock reconvergence in the clock network. The pessimism effects all analysis modes including single, best case-worst case (`bcwc`), and on-chip variation. For more information, see "Clock Path Pessimism Removal" in "Timing Analysis", chapter in the Innovus User Guide. **Note:** If you use the `-cppr` parameter to do optimization, the run time might increase slightly. Disables removal of clock reconvergence pessimism. *Default:* `none` Enables removal of clock reconvergence pessimism for both setup and hold modes. When `setAnalysisMode -cppr` is specified (without any arguments), the value is set to `both`. Enables removal of clock reconvergence pessimism in setup mode only. Enables removal of clock reconvergence pessimism in hold mode only.
`-help`	Outputs a brief description that includes type and default information for each `setAnalysisMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setAnalysisMode`.

`-honorActiveLogicView {true | false}`	Sets the timing analysis mode to honor Active Logic View (previously referred to as CDTV and Virtual Partition). Instead of analyzing the whole chip, the software analyzes only the nets related to interface paths between the partitions. The `createActiveLogicView` command sets `-honorActiveLogicView` by default, so there is no need to specify `setAnalysisMode -honorActiveLogicView true` explicitly after specifying `createActiveLogicView`. **Note:** This parameter applies to pre-CTS mode only. *Default:* `false`

`-honorClockDomains {true | false}`	Controls whether the software uses clock domains or path groups for timing optimization. When set to `true`, `optDesign` uses standard and user-specified clock domains during timing optimization. The following clock domain related commands are enabled: `setClockDomains`,`clearClockDomains`, and `reportClockDomains`. This setting is backward compatible with the behavior of releases prior to 7.1. When set to `false`, `optDesign` uses path groups during timing optimization. Clock domain related commands are disabled, and any active clock domains are cleared. Path groups can be created and modified using the following commands:`group_path`, `reset_path_group`,`createBasicPathGroups`,`setPathGroupOptions`,`resetPathGroupOptions`,`reportPathGroupOptions`. If no path groups are explicitly created, the software acts as though the basic path groups were created. However, as soon as you create one path group, the basic groups are then cleared. **Note:** Path groups and clock domains are mutually exclusive: enabling one methodology disables the other. *Default:* `false` **Note**: The `-honorClockDomains` parameter is now obsolete - support for this flow will be discontinued in a future release of the software. You should update your scripts to be compatible with the path group based flow.
`-log {true | false}`	Writes warning messages to the log file. *Default:* `true`

`-multi_input_switching_mode {0 | 1 | 2}`	Sets the multiple input switching (MIS) mode. You can specify one of the following options. When set to: • 0: Simultaneous input switching is not considered. • 1: MIS analysis considers cells for which multiple inputs of a cell are tied together (tied-inputs). • 2: MIS analysis considers both the tied-inputs and inputs of cells switching independent to each other. *Default*: 0
`-propSlew {true | false}`	Controls whether slews in early path delay calculation are degraded as they pass across the interconnect from driver to receiver. You can specify a value of `false` to turn off interconnect slew degradation for early paths and have the driver slews copied to the receivers. Early path delay calculation affects all Hold paths in best-case/worst-case analysis mode, capture clock paths in on-chip variation Setup analysis, and clock launch and data paths in on-chip variation Hold analysis. *Default:* `true`
`-reset`	Resets parameters to their default values. The `-reset` parameter *must* be the first parameter specified. If you specify `-reset` by itself, the software resets all `setAnalysisMode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. **Note**: Any values that do not already match the default value and are reset will be reported.

`-sequentialConstProp {true | false}`	Propagates constants through sequential elements. If you specify `-sequentialConstProp false`, constants will stop at sequential elements. *Default:* `false`
`-skew {true | false}`	Reports clock skew. The `-skew false` setting treats all clock latency as 0. *Default:* `true` With the `-skew true` setting, you can specify the `-clockPropagation` parameter. • Using the default `-skew true -clockPropagation sdcControl` setting accounts for the clock latency during timing analysis and proceeds as follows: ◦ If the `set_propagated_clock` command is specified in the timing constraint file, the clock latency is computed from the netlist (propagated mode). ◦ If the `set_clock_latency` command is specified in the timing constraint file, the clock latency specified in the constraint is used. Otherwise, no latency is reported (ideal mode).
`-socv {true | false}`	Enables SOCV analysis. *Default*: `false`
`-timeBorrowing {true | false}`	Considers time borrowing during timing analysis. Time borrowing is the amount of time borrowed by a previous logic. *Default:* `true`

`-timingEngine {statistical | static}`	Specifies the type of timing analysis to perform. • `statistical`: Sets analysis mode to SSTA (statistical static timing analysis). • `static`: Performs static timing analysis. *Default:* `static` **Note:** This parameter is obsolete. This still works in this release, and will be removed in a future release.
`-timingSelfLoopsNoSkew {true | false}`	Eliminates clock skew due to clock uncertainty for a path starting and ending at the same register. If the clock skew is not eliminated, the timing for such paths is pessimistic. **Note:** Use this parameter before building the timing graph. *Default:* `false`
`-usefulSkew {true | false}`	Considers the latency file (*`latency_file`*`.sdc`) during timing analysis. The latency file is generated when you run the `skewClock` command. *Default:* `false`
`-useOutputPinCap {true | false}`	Includes the output pin capacitance when calculating delay values. When you specify `-useOutputPinCap false`, the software excludes only the pin capacitance of the driver for which you are calculating the delay. *Default:* `true`
`-warn {true | false}`	Displays warning messages as the timing constraints file is read. *Default:* `true`

Examples

- The following command sets timing analysis to report clock skew, suppresses warning messages, and check for setup timing:

  ```
  setAnalysisMode -checkType setup -skew true -warn false
  ```

- The following commands set the mode for timing analysis to consider latency files, and generate a timing report based on timing with latencies from pre-CTS useful skew:

  ```
  setAnalysisMode -usefulSkew true


  restoreDesign mydesign


  report_timing
  ```

- The following commands sets the mode for timing analysis to ignore latency files, and generate a timing report based on timing without latencies from pre-CTS useful skew:

  ```
  setAnalysisMode -usefulSkew false


  report_timing
  ```

- The following command reset the `-checkType` parameter to its default value:

  ```
  setAnalysisMode -reset -checkType
  ```

- The following command resets all of the `setAnalysisMode` parameters to their default values:

  ```
  setAnalysisMode -reset
  ```

Related Information

- `getAnalysisMode`
- `report_timing`

signoffTimeDesign

```
signoffTimeDesign
[-help]
[-noEcoDB]
[-noExpandedViews]
[-outDir string]
[-prefix string]
[-reportFullClockPath]
[-reportOnly]
```

Runs signoff timing analysis using extraction (Quantus) and Tempus in batch mode, and generates timing reports and ECO DB for each view. The command prints out a timing summary table per view and all the views combined.

Parameters

`-help`	Prints out the command usage.
`-noEcoDB`	Disables generation of ECO timing DB.
`-noExpandedViews`	Disables printing of expanded timing per view.
`-outDir string`	Specifies the directory to save reports.
`-prefix string`	Specifies prefix for timing report file names.
`-reportFullClockPath`	When this parameter is specified, the detailed timing report that is generated contains the full clock path.
`-reportOnly`	Skips parasitic extraction and use the parasitics in memory.

Related Information

- `report_timing`
- `signoffOptDesign`
- `timeDesign`

- The following sections in Optimizing Timing chapter of the *Innovus User Guide*
  - Running MMMC SignOff ECO within Innovus
  - Using Signoff Timing Analysis to Optimize Timing and Power
  - Signoff Timing Analysis in Innovus using Timing Debug
  - Optimization in Path-Based Analysis (PBA) Mode
  - Getting the Best Total Power Optimization Recipe
  - Path Group Support

timeDesign

```
timeDesign
[-help]
[-drvReports]
[-expandReg2Reg]
[-expandedViews]
[-hold]
[-idealClock]
[-numPaths integer]
[-outDir string]
[-pathreports]
[-prefix string]
[-proto]
[-reportOnly]
[-slackReports]
[-timingDebugReport]
[-useTransitionFiles]
[-prePlace | -preCTS | -postCTS | -postRoute]
```

Runs Early Global Route, extraction, and timing analysis, and generates detailed timing reports. The generated timing reports are saved in `./timingReports` directory or the directory that you specify using the `-outDir` parameter.

The following timing reports are generated:

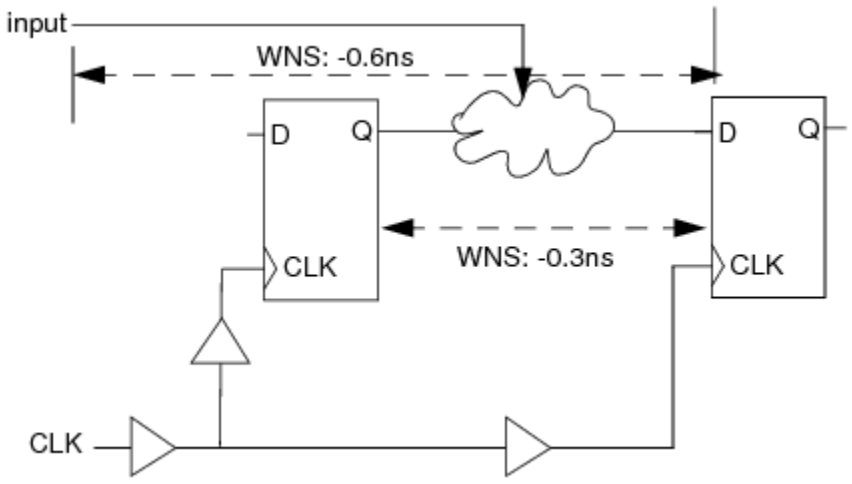
- Setup and hold time for the following path groups:
  - Register to register (*TopCellName_DesignStage*`_reg2reg.tarpt`)
  - Input to register - This report also includes the clock source paths (if applicable to the design). The clock source paths are included even if the clock source is an internal pin.
  - Register to output
  - Input to output
  - All paths ending in clock gate (*TopCellName_DesignStage*`_reg2cgate.tarpt`)
  - Groups in2reg, reg2out and in2out will be merged into a default group (`default.tarpt`)
- Setup and hold time without using path groups:
  - Setup analysis (*TopCellName_DesignStage*`_all.tarpt`)
  - Hold analysis (*TopCellName_DesignStage*`_all_hold.tarpt`)
- Density

Note: The density value reported by the `timeDesign` command might differ from the effective utilization value reported by the checkPlace command because `timeDesign` does not consider padding when calculating density, but `checkPlace` does consider it when calculating effective utilization.

- Maximum transition time violation (*`TopCellName_DesignStage`*`.tran`): The maximum transition value reported is the maximum value from `set_max_transition` command from SDC or the max_transition value specified in the library.
- Maximum capacitance violation (*`TopCellName_DesignStage`*`.cap`): The maximum capacitance value reported is the maximum value from the `set_max_capacitance` command from SDC or or max_capacitance value specified in the library.
- Maximum fanout violation (*`TopCellName_DesignStage`*`.fanout`)
- Glitch violations in the *`outDir`*/*`prefix_hold`*/`setup` directory.
- Summary (*`TopCellName_DesignStage`*`.summary` and *`TopCellName_DesignStage`*`_hold.summary`)
  Note: The ID number displayed in the Summary report refers to long path group names.

The overall total negative slack (TNS) and number of violating paths of a design might not be equal to the total of the TNS and violating paths of the individual path groups. This is because the TNS and number of violating paths are based on end-point of the path and are not path based.

For example, the following figure has a register with two paths, one from a primary input with a slack of `-0.6ns` and other from another register with a slack of `-0.3ns`.



In that case the overall TNS will be `-0.6ns` with `1` violating path (end point-based). But the individual reg2reg TNS will be `-0.3ns` with `1` violating path and in2reg with a TNS of `-0.6ns` with `1` violating path. Therefore, the sum total of individual path group TNS is not the same as overall TNS.

Similarly, when you run the `timeDesign` command while performing multi-mode multi-corner analysis,

overall TNS is not the sum of TNS of each view considered separately. The command sums the worst-case slack at each endpoint across all views. Therefore, each endpoint contributes only in overall worst-slack in the calculation of TNS.

Parameters

`-help`	Prints out the command usage.
`-drvReports`	Generate design rule violation (DRV) reports only. *Default:* Generates DRV and path reports.
`-expandedViews`	Generates detailed view-specific timing reports when the software is in multi-mode multi-corner analysis mode. For multi-mode multi-corner analysis, the software creates a separate directory for each view. For example, if your design has two analysis views, `view1` and `view2`, the output reports are generated in the `./timingReports/view1` and `./timingReports/view2` directories. If some of the active views have been pruned by auto view pruning then this command will restore those and give view-specific reports for all of the active views in the design.
`-expandReg2Reg`	Specifies that the register to register path groups are divided into the following detailed path groups for generating reports: • Flop to flop • Macro to flop • Flop to macro • Macro to macro
`-hold`	Reports hold violations only. Before ending the `timeDesign` run, the software reverts back to the initial analysis mode that was set by the `setAnalysisMode` command (either setup or hold). *Default:* Reports only setup violations

`-idealClock`	Performs ideal clock analysis irrespective of the design stage. Default: The software performs propagated clock analysis with the `-postCTS` and `-postRoute` parameters. The software performs ideal clock analysis with the `-prePlace` and `-preCTS` parameters. **Note**: This parameter is not honored when used with `-reportOnly`.
`-numPaths` *integer*	Specifies the number of paths to report in the detailed path violations. *Default:* `50`
`-outDir` *out_dir*	Specifies the name of the output directory that contains all the generated reports. *Default:* `timingReports`
`-pathReports`	Generate path reports only. *Default:* Generates DRV and path reports.
`-preCTS`	Generates timing reports before the clock tree is built. By default, in this mode the software runs early global route, native default extraction, and timing analysis for generating timing reports. The `-preCTS` parameter sets `setAnalysisMode` to `-clkSrcPath false` and `-clockPropagation forcedIdeal`.
`-prePlace`	Generates timing reports before the design is placed. In this mode, the software ignores the net load while building the timing graph and runs timing analysis for generating timing reports. The DRV report files are not generated in this mode, and the DRV numbers are not reported in the summary for this parameter. The `-prePlace` parameter sets `setAnalysisMode` to `-clkSrcPath false` and `-clockPropagation forcedIdeal`. **Note:** The `-reportOnly` parameter is not honored when used with this parameter.
`-proto`	Supports flex models. **Notes:** • The `-proto` parameter supports `set_proto_mode -timing_net_delay_model option`. • The `-proto` parameter honors the `-postCTS` option.

`-postCTS`	Generates timing reports for a design whose clock tree has been created. By default, in this mode the software runs early global route, native default extraction, and timing analysis for generating timing reports. The `-postCTS` parameter sets `setAnalysisMode` to `-clkSrcPath true` and `-clockPropagation sdcControl`.
`-postRoute`	Generates timing reports for a design whose routing is complete. By default, in this mode the software runs native detailed extraction, and timing analysis for generating timing reports. The `-postRoute` parameter sets `setAnalysisMode` to `-clkSrcPath true` and `-clockPropagation sdcControl`.
`-prefix` *`file_name_prefix`*	Specifies a prefix for the timing report file names. *Default:* *`DesignName_DesignStage`*
`-reportOnly`	Specifies the use of existing extraction and timing analysis data to generate timing reports. When you use this parameter, the software does not run extraction; instead it uses data that is already in memory. • `-reportOnly` is not honored when used with `-prePlace`. • `-idealClock` is not honored when used with `-reportOnly`. **Note:** The `-preCts`, `-postCts`, or `-postRoute`with `-reportOnly`. **Note**: If you restore a Innovus database (saved with `saveDesign -rc` before the `timeDesign -reportOnly` command is specified), a full extraction will still be performed if the database is in the pre-route stage. This is because the `preRoute` extraction RCDB cannot be saved into an Innovus database.
`-slackReports`	Generate slack reports only. *Default:* Generates DRV and path reports.
`-timingDebugReport`	Generates a compressed (`.gz`) detailed timing report in ASCII format. This report is used for debugging timing results using the timing debug feature in Innovus. For more information on debugging timing results, see Debugging Timing Results in *Innovus User Guide*.
`-useTransitionFiles`	Reports the maximum transition violations when using external transition files specified by the `readTransitionFile` command.

Command Order

Use this command after loading the design.

Examples

- The following command runs early global route, preRoute parasitic extraction, and timing analysis to generate timing reports before clock tree synthesis:
  `timeDesign -preCTS`
- The following example shows a summary report generated by `timeDesign`:

```
------------------------------------
timeDesign Summary
------------------------------------

Setup views included:
Setup_SSG_m40c_0p81v_RCworst_ccworst_func
Setup_SSG_m40c_0p81v_Cworst_ccworst_func

+--------------------+---------+---------+---------+------+
| Setup mode      | all     | reg2reg |reg2cgate| default |
+--------------------+---------+---------+---------+------+
| WNS (ns):       | -0.678  | -0.174  | -0.206  | -0.678  |
| TNS (ns):       | -1385.9 |-453.593 | -4.445  | -1056.4 |
| Violating Paths:| 7006    | 4948    | 60      | 3581    |
| All Paths:      | 45995   | 43625   | 740     | 5404    |
+--------------------+---------+---------+---------+------+

+----------------+-----------------------------------+---------------+
|                  |             Real            |      Total |
| DRVs +-----------------+-----------+-------------------------|
|                  | Nr nets(terms) | Worst Vio | Nr nets(terms) |
+----------------+-----------------+-----------+---------------+
| max_cap          | 0 (0)          | 0.000     | 284 (284)      |
| max_tran         | 0 (0)          | 0.000     | 0 (0)          |
| max_fanout       | 0 (0)          | 0         | 0 (0)          |
| max_length       | 0 (0)          | 0         | 0 (0)          |
+----------------+-----------------+-----------+---------------+

Density: 58.995%
Routing Overflow: 0.21% H and 0.14% V
------------------------------------------------------------
```

- The following command ignores the net load while building the timing graph and runs timing

analysis for generating timing reports before the design is placed:

```
timeDesign -prePlace
```

- The following commands run a postRoute parasitic extraction (TQuantus by default), and timing analysis to generate timing reports for both setup and hold violations after the design has been routed:

```
timeDesign -postRoute

timeDesign -hold -reportOnly
```

- The following command generates timing reports in multi-mode multi-corner timing analysis view, where `test1` is the specified analysis view:

```
set_analysis_view -setup {setup_view1 setup_view2} -hold {hold_view1}

timeDesign -postRoute -expandedViews -outDir sample

The following summary report is generated:

------------------------------------------------------------
timeDesign Summary
------------------------------------------------------------

Setup views included:
setup_view1 setup_view2

+--------------------+---------+---------+---------+-------+
| Setup mode         | all     | reg2reg |reg2cgate| default |
+--------------------+---------+---------+---------+-------+
| WNS (ns):          | -0.112 | -0.112  | 0.048   | 0.000   |
| TNS (ns):          | -6.266 | -6.266  | 0.000   | 0.000   |
| Violating Paths:   | 131    | 131     | 0       | 0       |
| All Paths:         | 36920  | 35656   | 1264    | 0       |
+--------------------+---------+---------+---------+-------+
|setup_view1         | -0.112 | -0.112  | 0.048   | 0.000   |
|                    | -6.266 | -6.266  | 0.000   | 0.000   |
|                    | 131    | 131     | 0       | 0       |
|                    | 35210  | 33946   | 1264    | 0       |
+--------------------+---------+---------+---------+-------+
|setup_view2         | -0.070 | -0.070  | 0.079   | 0.000   |
|                    | -1.113 | -1.113  | 0.000   | 0.000   |
|                    | 36     | 36      | 0       | 0       |
|                    | 35210  | 33946   | 1264    | 0       |
+--------------------+---------+---------+---------+-------------+

+----------------+-------------------------------+---------------+
|                  | Real                          | Total          |
| DRVs             +----------------+-----------+----------------|
```

```
|                  | Nr nets(terms) | Worst Vio | Nr nets(terms) |
+----------------+------------------+-----------+----------------+
| max_cap          | 0 (0)          | 0.000     | 0 (0)          |
| max_tran         | 0 (0)          | 0.000     | 0 (0)          |
| max_fanout       | 0 (0)          | 0         | 0 (0)          |
| max_length       | 0 (0)          | 0         | 0 (0)          |
+----------------+------------------+-----------+----------------+

Density: 31.688%
Total number of glitch violations: 0
------------------------------------------------------------
```

Related Information

- checkPlace
- set_analysis_view
- set_table_style
- set_max_capacitance
- set_max_transition
- report_timing_format
- Accelerating the Design Process By Using Multiple-CPU Processing chapter of the *Innovus User Guide.*

update_analysis_view

```
update_analysis_view
[-help]
-name <existingAnalysisViewName>
{[-constraint_mode <newModeName>]
[-delay_corner <newDelayCornerObj>]
[-latency_file <string>]}
```

Changes attribute values for the specified existing analysis view.

You can update an analysis view before the multi-mode multi-corner definition file is loaded into the design, or after. However, after the software processes the file, any change to an existing object when the software is in multi-mode multi-corner mode results in the timing, delay calculation, and RC data being reset.

Parameters

`-constraint_mode` *newModeName*	Specifies the name of the new constraint mode to associate with this analysis view.
`-delay_corner` *newDelayCornerObj*	Specifies the name of the new delay calculation corner to associate with this analysis view.
`-latency_file <`*string*`>`	Specifies the latency file name.
`-name` *existingAnalysisViewName*	Specifies the name of the existing analysis view to update.

Examples

- The following command creates an analysis view called missionSlow using the constraint mode missionSetup and the delay calculation corner dcWCCOM:

```
create_analysis_view
-name missionSlow
-constraint_mode missionSetup
```

```
-delay_corner dcWCCOM
```

- The following command changes the delay calculation corner associated with the analysis view `missionSlow` to `dcMax`:

```
update_analysis_view -name missionSlow -delay_corner dcMax
```

Related Information

- `create_analysis_view`
- `set_analysis_view`
- `get_analysis_view`

update_constraint_mode

```
update_constraint_mode
[-help]
[-ilm_sdc_files {file1.sdc file2.sdc ...}]
[-name modeName]
{[[-sdc_files {file1.sdc file2.sdc ...}]
[-tcl_vars {{var_name1 value1} {var_name2 value2} {var_name3 value3} ...}]
[-reset_latency_files]]}
```

Updates the SDC constraint file information for the specified existing constraint mode object.

Note: The `update_constraint_mode` command overwrites the previous constraint mode settings. However, by default, this command updates a constraint mode without affecting other constraint modes, including interactive constraints, such as, `set_dont_use` and `set_dont_touch`.

ⓘ You can update a constraint mode object before the multi-mode multi-corner definition file is loaded into the design, or after. However, after the software processes the file, any change to an existing object when the software is in multi-mode multi-corner mode results in the timing, delay calculation, and RC data being reset.

Parameters

`-help`	Prints out the command usage.
`-ilm_sdc_files {file1.sdc file2.sdc ...}`	Specifies the new full chip SDC constraint files for the ILM flow to associate with this constraint mode.
`-name` *modeName*	Specifies the name of the existing constraint mode object to update.
`-reset_latency_files`	Cleans up latency files of all the related views. The timing system maintains view-specific latency files to provide `set_clock_latency` assertions for useful-skew modeling and for providing post-CTS balancing of IO timing. When the `update_constraint_mode` is asserted to provide new constraints for a mode, you can decide whether to remove the related constraints in the latency files.
`-sdc_files {file1.sdc file2.sdc ...}`	Specifies the new SDC constraint files to associate with this constraint mode.
`-tcl_vars {{var_name1 value1} {var_name2 value2} {var_name3 value3} ...}`	Specifies a list of tcl variables and values to be used in the constraint mode.

Examples

- The following command groups the SDC files `io.sdc`, `mission1-clks.sdc`, and `mission1-except.sdc` to create a mode object named `missionSetup`:
  ```
  create_constraint_mode -name missionSetup -sdc_files [list io.sdc mission1-clks.sdc mission1-except.sdc]
  ```
- The following command changes the SDC files associated with the constraint mode `missionSetup` to `io.sdc`, `mission3-clks.sdc`, and `mission3-except.sdc`:
  ```
  update_constraint_mode -name missionSetup -sdc_files {io.sdc mission3-clks.sdc mission3-except.sdc}
  ```

Related Commands

- create_constraint_mode
- get_constraint_mode
- all_constraint_modes

update_delay_corner

```
update_delay_corner
[-help]
[-early_estimated_worst_irDrop_factor percent]
[-early_irdrop_data files_or_directories]
[-early_irdrop_file list_of_files]
[-early_library_set libSetName]
[-early_opcond opcondName]
[-early_opcond_library libName]
[-early_rc_corner rcCornerObj]
[-early_temp_file tempName]
[-irdrop_data files_or_directories]
[-irdrop_file list_of_files]
[-late_estimated_worst_irDrop_factor percent]
[-late_irdrop_data files_or_directories]
[-late_irdrop_file list_of_files]
[-late_library_set libSetName]
[-late_opcond opcondName]
[-late_opcond_library libName]
[-late_rc_corner rcCornerObj]
[-late_temp_file tempName]
[-library_set libSetObj]
-name delayCornerObj
[-opcond opcondName]
[-opcond_library libName]
[-pg_net_voltages voltage_pairs]
[-power_domain powerDomainName]
[-rc_corner rcCornerObj]
[-si_enabled {true | false}]
[-supply_set supplySetName]
[-temp_file tempName]
```

Modifies the parameters of an existing delay calculation corner object. You can use the `update_delay_corner` command to add or change attribute values for an existing delay calculation corner object, or for an existing power domain definition.

You can update a delay calculation corner object before the multi-mode multi-corner definition file is loaded into the design, or after. However, after the software processes the file, any change to an existing object when the software is in multi-mode multi-corner mode results in the timing, delay calculation, and RC data being reset.

You also can use the `update_delay_corner` command to add a power domain definition to the

specified delay calculation corner. Instances contained within an MSMV power domain often require a different library or operating condition that used for the default power domain. Power domain definitions within a delay calculation corner object can be used to assign specific libraries and PVT settings per power domain.
Note: Use separate delay calculation corners to define major PVT corner differences. Use the `-early_*` and `-late_*` parameters within a single delay calculation corner to control on-chip-variation.

The `-early_*` and `-late_*` parameters can be specified separately, to update a delay corner object with early or late information. If you only specify one of the `-early_*` or `-late_*` parameters, the software uses the original information pointer in place of the missing early/late parameter.

For example, if you initially created a delay corner object using:

```
create_delay_corner
-library_set libsNominal
...
```

If you specify `update_delay_corner -early_library_set libsEarly`, the software actually uses:

```
update_delay_corner
-early_library_set libsEarly
-late_library_set libsNominal
```

Use this command after creating at least one delay calculation corner (`create_delay_corner`).

Parameters

`-help`	Prints out the command usage.
`-early_estimated_worst_irDrop_factor` *per cent*	Specifies the early voltage derating factor. When this parameter is specified, the software performs delay calculations using the voltage = nomVolt * (1 – value). *Default*: 0.0
`-early_irdrop_data` *files_or_directories*	Specifies the IR drop files or directories to apply when calculating the early arrival times at a single delay corner.

`-early_irdrop_file` *list_of_files*	Specifies the list of IR drop files to apply when calculating early arrival times at a single power domain. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis. **Note**: This parameter is obsolete and will be removed in a future release. To ensure compatibility with future releases, you should update your scripts.
`-early_library_set` *libSetName*	Specifies the library set to associate with the power domain for calculating early arrival times for this power domain. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis.
`-early_opcond` *opcondName*	Specifies the operating condition to use for calculating early arrival times for this power domain. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis. *Default:* Each library in the early library set uses its own default operating condition.
`-early_opcond_library` *libName*	Specifies the internal library name for the library in which the early operating condition is defined. This is *not* the library file name. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis. *Default:* The software searches the early library set for the specified operating condition (`-early_opcond`), starting with the master library.
`-early_rc_corner` *rcCornerObj*	Specifies the RC corner object to associate with the early corner object.

`-early_temp_file` *tempName*	Specifies the name of the temperature file for temperature aware delay calculation for early corner.
`-irdrop_data` *files_or_directories*	Specifies the IR drop files or directories to apply to both the early and late delay calculation for the specified delay corner object.
`-irdrop_file` *list_of_files*	Specifies the list of IR drop files to apply to both early and late delay calculation for this power domain object. This parameter is used primarily to configure single-corner or Best-Case Worst-Case (BC-WC) analysis modes. **Note**: This parameter is obsolete and will be removed in a future release. To ensure compatibility with future releases, you should update your scripts.
`-late_estimated_worst_irDrop_factor` *percent*	Specifies the late voltage derating factor. When this parameter is specified, the software performs delay calculations using the voltage = nomVolt * (1 – value). *Default*: 0.0
`-late_irdrop_data` *files_or_directories*	Specifies the IR drop files or directories to apply when calculating the late arrival times at a single delay corner.
`-late_irdrop_file` *list_of_files*	Specifies the list of IR drop files to apply when calculating late arrival times at a single power domain. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis. **Note**: This parameter is obsolete and will be removed in a future release. To ensure compatibility with future releases, you should update your scripts.

`-late_library_set` *libSetName*	Specifies the library set to associate with this delay corner object for calculating late arrival times for this power domain. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis.
`-late_opcond` *opcondName*	Specifies the operating condition to use for calculating late arrival times for this power domain. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis. *Default:* Each library in the late library set uses its own default operating condition.
`-late_opcond_library` *libName*	Specifies the internal library name for the library in which the late operating condition is defined. This is *not* the library file name. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis. *Default:* The software searches the late library set for the specified operating condition (`-late_opcond`), starting with the master library.
`-late_temp_file` *tempName*	Specifies the name of the temperature file for temperature aware delay calculation for late corner.
`-late_rc_corner` *rcCornerObj*	Specifies the RC corner object to associate with the late corner object.
`-library_set` *libSetName*	Specifies the library set to associate with the power domain. All of the cells and macros in the power domain get their timing and power information from these libraries. This parameter is used primarily to configure single-corner or Best-Case Worst-Case (BC-WC) analysis modes.

`-name` *dcCornerObj*	Specifies the delay calculation corner to which to add the power domain definition, or to which to update the delay corner or power domain attribute values.
`-opcond` *OpcondName*	Specifies the operating condition to use for the power domain. This parameter is used primarily to configure single-corner or Best-Case Worst-Case (BC-WC) analysis modes. *Default:* Each library in the library set uses its own default operating condition.
`-opcond_library` *libName*	Specifies the internal library name for the library in which the operating condition is defined. This is *not* the library file name. This parameter is used primarily to configure single-corner or Best-Case Worst-Case (BC-WC) analysis modes. *Default:* The software searches the library set for the specified operating condition (`-opcond`), starting with the master library.
`-pg_net_voltages` *voltage_pairs*	Specifies the power/ground net voltages to be used for both early/late timing analysis.
`-power_domain` *powerDomainName*	Specifies the name of the power domain to add to the delay calculation corner. This name must match the name of a power domain specified in the power domain file. **Note**: This parameter is required for all the designs that use power domains and should be specified for each delay corner in the MMMC file.
`-rc_corner` *rcCornerObj*	Specifies the new RC corner object to associate with this delay corner.

`-si_enabled {true | false}`	Controls signal integrity (SI) analysis. When set to `true`, the analysis of all MMMC views will include SI analysis by default. You can use this parameter to further determine which analysis views receive SI analysis and which do not. When this parameter is set to `false` on a delay corner, the software will disable SI analysis for all the views that reference that corner. By default (or when this parameter is set to `true`), SI analysis is computed for all the views related to that corner - whenever SI is globally enabled. (The global control of signal integrity (SI) analysis is determined by the `setDelayCalMode -SIAware` setting.)
`-supply_set` *supplySetName*	Specifies the name of the supply set to add to the delay calculation corner.
`-temp_file` *tempName*	Specifies the name of the temperature file for temperature aware delay calculation for single corner.

Examples

- The following command creates the delay calculation corner `dc1`:

```
create_delay_corner -name dc1
-library_set libs-2volt
-opcond_library delayvolt_2V
-opcond slow_2V
-rc_corner rc-cworst
```

- The following commands add definitions for the power domains `domain-3V` and `domain-5V` to the `dc1` delay calculation corner:

```
update_delay_corner -name dc1
-power_domain domain-3V
-library_set libs-3volt
-opcond_library delayvolt_3V
-opcond slow_3V
update_delay_corner -name dc1
```

```
-power_domain domain-5V
-library_set libs-5volt
-opcond_library delayvolt_5V
-opcond slow_5V
```

- The following command changes the libraries used by the delay calculation corner `dc1` to those in the library set `IsCOM-2v`, and changes the RC corner associated with `dc1` to `rc-ctyp`:

```
update_delay_corner -name dc1
-library_set IsCOM-2V
-rc_corner rc-ctyp
```

Related Commands

- create_delay_corner
- get_delay_corner

update_io_latency

```
update_io_latency
[-source]
-verbose
```

Provides an automated means of balancing IO and Core clock latencies after the clock network has been added to the design so that pre and post clock tree skew between the clocks is maintained as closely as possible. This results in minimum change of inbound and outbound IO slacks.

This command performs the following actions:

- Measures and calculates ideal average latency for each Core clock.
- Calculates preCTS skew between IO and Core clocks.
- Asserts `set_propagated_clock` on all clock root pins or ports to put clock trees in propagated mode.
- Measures and calculates propagated mode average latency for each Core clock.
- Adjusts `set_clock_latency` on IO clocks network latency to minimize pre to post CTS IO-to-Core clock skew.

In case a single IO clock interacts with multiple Core clocks, the IO clock will be adjusted by the overall average latency of all the related Core clocks combined.

To be effective, this command should only be used after clock trees have been implemented. This allows the command to see both the ideal and propagated state of the clock trees.

Note: The `set_propagated_clock` command should not be issued after the `update_io_latency` command.

Parameters

`-source`	Adjusts the source latency of core clock.
`-verbose`	Provides additional information to the console as adjustments are made to each IO clock.

Related Information

- `set_propagated_clock`

update_library_set

```
update_library_set
[-help]
[-aocv string]
[-library_side_file string]
-name libSetName
[-si string]
[-socv string]
[-timing string]
```

Updates an existing specified library set.

Note: You can edit a library set before (or after) the multi-mode multi-corner view definition file is loaded into the design. However, after the software processes the file, any change to an existing object when the software is in multi-mode multi-corner mode resets the timing, delay calculation, and RC data.

Parameters

`-help`	Prints out the command usage.
`-aocv` *`string`*	Specifies a list of new AOCV libraries to include in the specified library set.
`-library_side_file` *`string`*	Specifies the library side file.
`-name` *`libSetName`*	Specifies the name of an existing library set to be updated. The library set must have been defined using the `create_library_set` command.
`-si` *`string`*	Specifies a list of new cdB libraries and user-defined noise (UDN) models to include in the library set.
`-socv` *`string`*	Specifies a list of new SOCV libraries to be included in the library set.
`-timing` *`string`*	Specifies a list of new timing libraries to include in the specified library set. Note: The ECSM and CCS libraries can be very large and loading them may take significant time. You can convert the ASCII NLDM (Liberty .lib), ECSM, and CCS libraries to a binary version using the write_ldb command. For conversion of libraries to a binary version, refer to `write_ldb`. Instead of the original ASCII libraries, you can specify the following binary libraries in the configuration file: `create_library_set -name libName -timing "AAA.lib BBB.ecsm CCC.ccs"`

Examples

- The following command creates a library set that associates timing libraries and cdB libraries with a nominal voltage of 1 volt with the library name lsCOM-1V:
  ```
  create_library_set -name IsCOM-1V -timing [list stdcell_F_1V.lib ram_F.lib pad.lib]
  ```
  The following command changes the timing libraries associated with the library set lsCOM-1V:
  ```
  update_library_set -name IsCOM-1V -timing [list stdcell_CVF_1V.lib ram_CVF.lib
  pad.lib ]
  ```

Related Commands

- create_library_set
- get_library_set
- all_library_sets

update_rc_corner

```
update_rc_corner
[-T temperatureValue]
[-cap_table newcapTableFile]
-name existingRcCornerName
[-postRoute_res {resFactor1 resFactor2 resFactor3}]
[-postRoute_cap {capFactor1 capFactor2 capFactor3}]
[-postRoute_xcap {xcapFactor1 xcapFactor2 xcapFactor3}]
[-postRoute_clkres {resFactor1 resFactor2 resFactor3}]
[-postRoute_clkcap {capFactor1 capFactor2 capFactor3}]
[-preRoute_clkres resFactor]
[-preRoute_clkcap capFactor]
[-preRoute_res resFactor]
[-preRoute_cap capFactor]
[-qx_tech_file newFileName]
[-via_variation_file viaVariationFile]
```

Adds or changes attribute values for the specified existing RC corner object.

> You can edit an RC corner object before the multi-mode multi-corner definition file is loaded into the design, or after. However, after the software processes the file, any change to an existing object when the software is in multi-mode multi-corner mode results in the timing, delay calculation, and RC data being reset.

You can set different postroute scale factors for the postroute engine variants by specifying them as duplets and triplets. For example,

resFactor1 resFactor2 resFactor3

Where:

- Single value: If you specify one value, the scale factor applies to effort level `low`. Scale factor value of 1 is used for `medium` and `high` by default.
- Duplet: If you specify two values, the first value is used for effort level `low` and the second value is used for `medium`. Scale factor value of `1` is used for `high` by default.
- Triplet: If you specify three values, the first value is used for effort level `low`, the second value is used for `medium`, and the third value is used for `high`.

Parameters

`-cap_table` *newCapTableFile*	Specifies a new capacitance table to use for extraction when using this RC corner.
`-preRoute_cap` *capFactor*	Specifies the capacitance scale factor for RC Extraction in preRoute mode.
`-preRoute_clkcap` *capFactor*	Specifies the scale factor for capacitance of clock nets in preRoute mode. Default: `0` When this parameter is not specified, the value specified with the `-postRoute_cap` parameter is used for preRoute extraction. When specified, this parameter overrides the value specified with the `-postRoute_cap` parameter for clock nets.
`-preRoute_clkres` *resFactor*	Specifies the scale factor for resistance of clock nets in preRoute mode. Default: `0` When this parameter is not specified, the value specified with the `-postRoute_res` parameter is used for preRoute extraction. When specified, this parameter overrides the value specified with the `-postRoute_res` parameter for clock nets.
`-preRoute_res` *resFactor*	Specifies the resistance scale factor for RC Extraction in pre-route mode.
`-postRoute_cap {`*capFactor1capFactor2 capFactor3*`}`	Specifies the capacitance scale factor for RC Extraction in postRoute mode.

`-postRoute_clkcap {`*`capFactor1capFactor2 capFactor3`*`}`	Specifies the scale factor for capacitance of clock nets in the postRoute extraction mode. Default: `0` When this parameter is not specified, the value specified with the `-postRoute_cap` parameter is used for postRoute extraction. When specified, this parameter overrides the value specified with the `-postRoute_cap` parameter.
`-postRoute_clkres {`*`resFactor1 resFactor2 resFactor3`*`}`	Specifies the scale factor for resistance of clock nets in the postRoute extraction mode. Default: `0` When this parameter is not specified, the value specified with the `-postRoute_res` parameter is used for postRoute extraction. When specified, this parameter overrides the value specified with the `-postRoute_res` parameter for clock nets.
`-postRoute_res {`*`resFactor1 resFactor2 resFactor3`*`}`	Specifies the resistance scale factor for RC extraction in postRoute mode.
`-name` *`existingRcCornerName`*	Specifies the name of the existing RC corner object to update.
`-qx_tech_file` *`newFileName`*	Specifies a new Quantus technology (`.tch`) file which is used to perform sign-off RC extraction.
`-T` *`newTempValue`*	Specifies a new temperature, in units of Celsius, to use to derate resistance values.
`-postRoute_xcap {`*`xcapFactor1 xcapFactor2 xcapFactor3`*`}`	Specifies the cross-coupling capacitance scale factor for RC extraction in postRoute mode.

`-via_variation_file` *viaVariationFile*	Specifies a file for modeling via variation. The VIA variation file lists up all the VIA layer names with the corresponding multiplier. Each RC corner can have its own VIA variation file.
	For example:
	`update_rc_corner`
	`-rc_corner rcCornerName1 \`
	`-via_variation_file viaVariationFileName1`
	`update_rc_corner`
	`-rc_corner rcCornerName2 \`
	`-via_variation_file viaVariationFileName2`

Examples

- The following command creates an RC corner called `rc-cbest` that uses the capacitance table `myTech_bc.CapTbl`, and derates the resistance values based on the temperature of 50 Celsius:
  `create_rc_corner -name rc-cbest -cap_table myTech.4_bc.CapTbl -T 50`
- The following command changes the capacitance table used by RC corner `rc-cbest` to `myCap.2_bc.capTbl`, and the temperature by which resistance values are derated to `25` Celsius:
  `update_rc_corner -name rc_cbest -T 25`

Related Information

- `create_rc_corner`
- `all_rc_corners`

write_global_slack_report

```
write_global_slack_report
[-early]
[-late]
[-rise]
[-fall]
[-include_ports]
[-max_slack val]
[-min_slack val]
[-view view_name]
[> | >> file_name]
[pin_port_list]
```

Reports early and late slacks on every instance pin in the design.

This command computes and reports the required time and slack values of the specified pins.

Parameters

`-help`	Prints out the command usage.
`> | >> filename`	Specifies the output file name. When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. **Note:** To write a compressed report, add the .gz extension to the file name.
`-early/-late`	Specifies the mode for which slack is to be reported.
`-fall/-rise`	Specifies the transition for which slack is to be reported.

`-include_ports`	Reports slack information for ports. By default, the slack information is reported for pins only.
`-max_slack` *val*	Reports pins that have early and late slack values less than the specified value. Type: Float
`-min_slack` *val*	Reports pins that have early and late slack values less than the specified value. *Type:* `Float` **Note**: This parameter is obsolete and will be removed in a future release. Cadence recommends that you use the `-max_slack` parameter.
pin_port_list	Specifies the list of pins or ports for which slack information is to be reported.
`-view` *view_name*	Reports slack information for the specified view.

Examples

- The following command reports the early and late slacks on every instance pin in the design:

```
tempus > write_global_slack_report > out.rpt

# Unit: 1ns
# <late_rise> <late_fall> <early_rise> <early_fall> <view_name> <hpin_name>
# ------------------------------------------------------------------------
*              *          *            *            (null)      u2/Q
*              *          *            *            (null)      u8/Q
*              *          *            *            (null)      b11/A
*              *          *            *            (null)      b11/Y
0.823          0.347      0.115        0.501        (null)      u10999/A
0.823          0.347      0.115        0.501        (null)      u10999/Y
# ------------------------------------------------------------------------
```

- The following command reports slack for a pin collection:

```
tempus> set x [get_pins -hier */A]
```

```
0x14 <---- 'x' stores Pointer for pin collection
```

- The following command will generate a global slack report for pin collection as defined in "x":

```
tempus> write_global_slack_report $x


# Unit: 1ns
# <late_rise> <late_fall>  <early_rise>  <early_fall>  <view_name> <hpin_name>
# ------------------------------------------------------------------------
19.622       19.742        0.321         0.106         (null)      seg1/u1/A
19.622       19.742        0.321         0.106         (null)      seg1/u2/A
18.018       18.362        1.876         1.486         (null)      seg1/u10/A
18.018       18.362        1.876         1.486         (null)      seg1/u11/A
18.362       18.018        1.486         1.876         (null)      seg1/u12/A
18.018       18.362        1.876         1.486         (null)      seg1/u13/A
25.845       25.883       -0.934        -1.053         (null)      seg2/u1/A
25.845       25.883       -0.934        -1.053         (null)      seg2/u2/A
# ------------------------------------------------------------------------
```

- The following example shows slack calculations, when you run the report_timing command for the same hpin.

```
tempus > write_global_slack_report DT_IN/RE_CO/r697_reg_2/D

# Unit: 1ns
# <late_rise> <view_name> <late_fall> <view_name> <early_rise>
<view_name> <early_fall> <view_name> <hpin_name>
# ------------------------------------------------------------------------
12.641 dtmf_view_setup 12.127 dtmf_view_setup * *
DT_IN/RE_CO/r697_reg_2/D

tempus > report_timing -to DT_IN/RE_CO/r697_reg_2/D -late -fall

Path 1: MET Setup Check with Pin DT_IN/RE_CO/r697_reg_2/CKN
Endpoint: DT_IN/RE_CO/r697_reg_2/D (v) checked with trailing
edge of 'vclk2'
Beginpoint: DT_IN/RE_CO/r697_reg_2/Q (v) triggered by trailing
edge of 'vclk2'
Path Groups: {vclk2}
Analysis View: dtmf_view_setup
Other End Arrival Time 7.015
- Setup 0.600
+ Phase Shift 14.000
- Uncertainty 0.250
= Required Time 20.164
```

```
- Arrival Time 8.037
= Slack Time 12.127
Clock Fall Edge 7.000
+ Clock Network Latency (Prop) 0.015
= Beginpoint Arrival Time 7.015
+-------------------------------------------------------------------------------+
| Instance               | Arc          | Cell    | Delay | Arrival | Required|
|                        |              |         |       | Time    | Time |
|------------------------------------+------------+---------+------+------|
| DT_IN/RE_CO/r697_reg_2 | CKN v        |         |       | 7.015   | 19.142  |
| DT_IN/RE_CO/r697_reg_2 | CKN v -> Q v | SDFFNX1 | 0.710 | 7.725   | 19.852  |
| DT_IN/RE_CO/i_6594     | B v -> Y v   | MX2X1   | 0.312 | 8.037   | 20.164  |
| DT_IN/RE_CO/r697_reg_2 | D v          | SDFFNX1 | 0.000 | 8.037   | 20.164  |
+-------------------------------------------------------------------------------+

tempus > report_timing -to DT_IN/RE_CO/r697_reg_2/D -late -rise


Path 1: MET Setup Check with Pin DT_IN/RE_CO/r697_reg_2/CKN
Endpoint: DT_IN/RE_CO/r697_reg_2/D (^) checked with trailing
edge of 'vclk2'
Beginpoint: DT_IN/RE_CO/r697_reg_2/Q (^) triggered by trailing
edge of 'vclk2'
Path Groups: {vclk2}
Analysis View: dtmf_view_setup
Other End Arrival Time 7.015
- Setup 0.204
+ Phase Shift 14.000
- Uncertainty 0.250
= Required Time 20.560
- Arrival Time 7.919
= Slack Time 12.641
Clock Fall Edge 7.000
+ Clock Network Latency (Prop) 0.015
= Beginpoint Arrival Time 7.015
+-------------------------------------------------------------------------------+
| Instance               | Arc          | Cell    | Delay | Arrival |Required |
|                        |              |         |       | Time    | Time    |
|------------------------+--------------+---------+-------+---------+-------|
| DT_IN/RE_CO/r697_reg_2 | CKN v        |         |       | 7.015   | 19.656 |
| DT_IN/RE_CO/r697_reg_2 | CKN v -> Q ^ | SDFFNX1 | 0.628 | 7.643   | 20.284 |
| DT_IN/RE_CO/i_6594     | B ^ -> Y ^   | MX2X1   | 0.276 | 7.919   | 20.560 |
| DT_IN/RE_CO/r697_reg_2 | D ^          | SDFFNX1 | 0.000 | 7.919   | 20.560 |
+-------------------------------------------------------------------------------+
```

Related Information

- write_global_slack_worst_trigger_path_on_clocks

write_ldb

```
write_ldb
-library string
-outfile string
```

Converts text library to Cadence binary format library called LDB.

You can use LDB to run timing analysis. The ECSM and CCS libraries can be very large and loading them may take significant time. You can convert the ASCII NLDM (Liberty .lib), ECSM, and CCS libraries to LDB by using the `write_ldb` command.

When loading the design, the timing libraries will be read faster because the software reads the binary versions.

Instead of the original ASCII libraries, you can specify LDB in the `create_library_set` command in MMMC configuration file.

Notes:

- The software generated LDB files can be used in Tempus.
- LDB files generated from Tempus can be used as input by other tools - Innovus, Genus, and Voltus.
- LDB files are not forward compatible, that is, the LDB generated from 15.2 version cannot be used with 15.1 or prior versions of the software.
- The software does not support concatenated Liberty library (multiple libraries in the same dotlib file) files during LDB conversion. To use these files, you are required to separate out individual libraries before converting to individual LDB files.

Parameters

`-library string`	Specifies the name of the input library file.
`-outfile string`	Specifies the name of the generated LDB file.

Examples

- The following example illustrates the conversion of libraries to LDB:

  ```
  write_ldb -library A.lib -outfile A.ldb
  ```

Related Information

- check_ldb_version

write_path_descriptions

```
write_path_descriptions
[-help]
var_name
-out_file string
```

Writes out the path description of all the specified timing paths to an ASCII file. The output file contains a set of `report_timing` commands for each timing path in the given path collection. The file also includes a complete description of each timing path captured through timing points along with the clock and edge information.

Parameters

`-help`	Prints out the command usage.
var_name	Specifies the object collection.
`-out_file` *string*	Redirects output to the specified file.

Examples

- The following example command writes out descriptions of late GBA paths to `pathfile1`:

  ```
  write_path_descriptions var1 -out_file file1 > file2
  ```

  The output file `pathfile1` will contain the following information:

  ```
  report_timing -net -path_type full_clock -late -from_rise A -through_rise B -
  through_rise C -to_rise D -view slow_v -clock_from clk1 -edge_from lead -clock_to
  clk2 -edge_to lead
  ```

Related Commands

- `read_path_descriptions`

write_sdf

```
write_sdf
[-help]
file_name
[-abstracted_model]
[-adjust_setuphold_for_zero_hold_slack]
[-base_delay]
[-celltiming {all | none | nochecks}]
[-collapse_internal_pins]
[-condelse]
[-delimiter char]
[-delta_delay]
[-edges {edged | library | noedge | check_edge}]
[-exclude_cells cell_list]
[-exclude_cells_keep_io_nets]
[-exclude_cells_keep_top_level_nets]
[-exclude_disabled_arcs]
[-exclude_disabled_modes]
[-exclude_whatif_arcs]
[-filter]
[-ideal_clock_network]
[-interconn {all | none | noport | noinport | nooutport}]
[-map_file {file1 file2 ...}]
[-max_view viewName]
[-merge_arcs]
[-min_period_edges {posedge | negedge | both | none}]
[-min_view viewName]
[-no_condition]
[-no_derate]
[-no_escape]
[-no_variation]
[-nonegchecks]
[-precision <non_neg_integer>]
[-process string]
[-recompute_delay_calc]
[-recompute_parallel_arcs]
[-recrem {merge_always | split | merge_when_paired}]
[-remashold]
[-resort_values_min_to_max]
[-scale float]
[-setuphold {merge_always | split | merge_when_paired}]
[-splitrecrem]
```

```
[-splitsetuphold]
[-target_application {sta | verilog}]
[-temperature string]
[-timingcheck {order}]
[-transform_out_to_out_arcs]
[-typ_view viewName]
[-version {2.1 3.0}]
[-view viewName]
[-voltage string]
```

Writes delays to a Standard Delay Format (SDF) file. The `write_sdf` command can generate both plain-text and GNU zipped (`.gz`) SDF files.

By default, the `write_sdf` command uses the delays already cached by the system and writes only values of the triplets.

Parameters

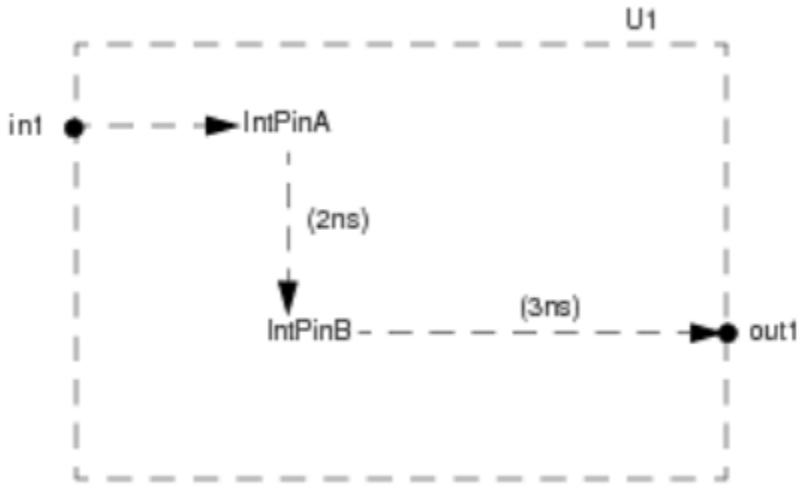
`-help`	Prints out the command usage.
`-abstracted_model`	Saves the gate-level simulation (GLS) SDF file.
`-adjust_setuphold_for_zero_hold_slack`	Adjusts the hold timing check to meet non-negative slack values.
`-base_delay`	Writes out base delay information in the SDF file.
`-celltiming {all | none | nochecks}`	Specifies which cell delays and timing checks to write out. *Default*: `all` • `all` - Writes all cell delays and timing checks to the SDF file. This is the default. • `nochecks` - Disables the writing of timing checks only. • `none` - Prevents cell delays and timing checks from being written into the SDF file.
`-collapse_internal_pins`	
	Collapses arcs across internal pins to a single primary

input to primary output arc.

Use the -collapse_internal_pins parameter when the SDF file being generated is intended for use by an application that does not support the same internal pin structure in its leaf cell libraries (for example, Cadence VXL and NCsim Verilog simulation products). Do *not* use this parameter to generate an SDF file intended for use within the Innovus flow, or by third party tools that utilize the same Liberty timing models.

Default: Arcs between internal pins in a timing model are written to the SDF file as explicit annotations to and from the internal pins.

For example, assume the following arc between internal pins:



The following SDF would be generated by default:

```
(U1/in1 U1/IntPinA (1::1) (1::1))

(U1/IntPinA U1/IntPinB (2::2) (2::2))

(U1/IntPinB U1/out1 (3::3) (3::3))
```

If you specify -collapse_internal_pins, the resulting SDF would show a single arc with the merged delays:

```
(U1/in1 U1/out1 (6::6) (6::6))
```

`-condelse`	Writes `CONDELSE` constructs with the default value when a `COND` construct is written. **Note:** Use this argument with the `-version 3.0` option.
`-delimiter` *char*	Specifies the hierarchy divider character to use in the output file. This option sets the hierarchy divider of only the `write_sdf` command. If omitted, the default hierarchy divider character (`/`) is used as the `write_sdf` default.
`-delta_delay`	Writes out delta delay values to the SDF file.
`-edges {edged | library | noedge | check_edge}`	
	Specifies the edges values. Refer to Table for full qualifications of the possible values for the edges option. *Default*: `edged` • `check_edge` - Keeps edge specifiers on timing check arcs but does not add edge specifiers on delay arcs. • `edged` - Keeps edge specifiers on timing check arcs and delay arcs. • `library` - Assumes that the sdf_edges attributes are specified in the .lib file. The default sdf_edges value of a timing arc is noedge. • `noedge` - Defined in the Table.
`-exclude_cells`	Specifies a list of leaf-cell or hierarchical instances whose timing data will be excluded from the the SDF (Standard Delay Format) file. If you specify a hierarchical instance, the timing data for all leaf-cells below the level of hierarchy will be omitted from the SDF file.
`-exclude_cells_keep_io_nets`	Restricts the filtering to just the annotations which are completely enclosed by the cell. Annotations to or from the cell will be preserved. By default, when the `-exclude_cells` parameter is used, any annotations within the excluded cell and any annotations to or from the excluded cell are filtered from the SDF.

`-exclude_cells_keep_top_level_nets`	Allows you to configure the handling of top-level interconnect annotations to an instance that is excluded from the SDF using the `write_sdf -exclude_cells` parameter. When this parameter is specified, the interconnect arcs which span to the ports of the instance will be included in the SDF. By default, no interconnect arcs to the instance are included. The delay annotated will be the worst-case delays of the top-level nets to/from the first level sink/source pins within the block that traverse a given input/output port of the instance, respectively.
`-exclude_disabled_arcs`	Excludes user set disabled arcs in the Standard Delay Format (SDF) file. If the user sets this option, the disabled arcs set by commands `set_mode`, `set_disable_timing`, and `set_case_analysis` are excluded.
`-exclude_disabled_modes`	Excludes timing arcs that have been disabled due to an active library mode set by the `set_mode` command, from appearing in the SDF file.
`-exclude_whatif_arcs`	Excludes what-if arcs in the SDF file.
`file_name`	Specifies the name of the SDF output file. If omitted, this argument defaults to `stdout`.
`-filter`	Converts all negative interconnect and `IOPATH` delays to zero.
`-ideal_clock_network`	Marks delays of delay arcs, that are on the ideal clock networks, as 0 in the SDF file.

`-interconn {all | none | noport | noinport | nooutport}`	Specifies which interconnect delays to write. *Default*: `all` • `all` - Writes all `INTERCONN` delays into the SDF file. This is the default if the `-interconn` option is not specified. • `none` - No `INTERCONN` delays are written into the SDF file. • `noport` - No top level port `INTERCONN` delays are written into the SDF file. • `nooutport` - Disables only the output port `INTERCONN` delays.
`-map_file {file1 file2 ...}`	Specifies a list of map files. The map files can be used to determine what is written into the SDF files. For example, a map file can provide delay information on internal pins which are not defined in the timing library. Syntax for a map file (for a cell) is as follows: ◦ A template section (the text to output) that reference variables of the form $<number> ◦ A variable definition section that tells the expression to evaluate to obtain the value
`-max_view` *viewName*	Uses the late delay from the specified analysis view to populate the SDF max slot, when in multi-mode multi-corner analysis mode.
`-merge_arcs`	Merges two arcs that are related to the same internal pin and forms one timing arc. For example, consider a cell's `IOPATH` that forms an arc from pin `I` to internal pin `A` and another from pin `A` to output pin `O`. In this case, if you use the `-merge_arcs` option, the software will merge both the arcs related to internal pin `A` and form one timing arc from pin `I` to output pin `O`.

`-min_period_edges {posedge | negedge | both | none}`	Specifies how to write out minimum `PERIOD` checks. Two check arcs are always created for the `min_period` check when you read in a `.lib` file. The following options let you keep or remove the positive edge, the negative edge, both edges, or no edges from the PERIOD statement in the `.sdf` file. • `posedge` - Writes out one positive edge `PERIOD` check. • `negedge` - Writes out one negative edge `PERIOD` check. • `both` - Writes out both the positive edge and the negative edge `PERIOD` checks. This is the default. • `none` - Writes out one `PERIOD` check without any edge specifier.
`-min_view` *viewName*	Uses the early delay from the specified analysis view to populate the SDF `min` slot, when in multi-mode multi-corner analysis mode.
`-no_condition`	Suppresses the COND statements in the SDF file.
`-no_derate`	Writes non-derated delays to the Standard Delay Format (SDF) file. By default, the `set_timing_derate` command writes scaled delays to the SDF file. When the `-no_derate` parameter is specified, the `write_sdf` command removes all the applied derates.
`-no_escape`	Writes out object names without escaping hierarchical dividers. Normally, when writing out a SDF file, the hierarchical divider (`/`) is escaped because it is a special character.
`-no_variation`	Exports the SDF delay and check values without including the variation component, when the selected analysis type is SOCV. By default, the SDF delays and check values will have the mean + N*sigma value.

`-nonegchecks`	Converts all negative timing check values to `0.0`. The `-nonegchecks` parameter has an effect only when the negative checks are legal. In SDF v3.0, the negative checks are only legal in the context of `RECREM` or `SETUPHOLD` checks – they are not legal in case of split checks. In SDC v2.1, the negative values are permitted in `SETUP`, `HOLD`, and `RECOVERY` checks. Therefore, for these checks the `-nonegchecks` parameter is required to convert negative values to zero. The negative checks are legal in the split checks, so the `-nonegchecks` parameter can have an effect. The SDF v2.1, however, does not support `REMOVAL` checks.
`-precision` *non_neg_integer*	Controls the number of digits appearing after the decimal point in the output SDF file. The default precision is the value of the `report_precision`.

`-process`	Specifies one or a combination of the following process names that will appear in the header of the generated SDF file:
	
	• `best`
	• `typical`
	• `worst`
	
	Use the colon delimiter to specify multiple process names. For example:
	
	`-process best:typical:worst`
	
	In the above example, the header of the generated SDF file will contain:
	
	`(PROCESS "best:typical:worst")`

`-resort_values_min_to_max`	Sorts values in the SDF delay triplets into ascending order.

`-recompute_delay_calc`	Instructs the software to recompute any necessary data required for a complete SDF file before exporting. By default, only the existing data that has been computed for STA is exported to the SDF. You must use this parameter to ensure that the SDF file is complete for general use in other applications.
`-recompute_parallel_arcs`	Instructs the software to temporarily recompute delays with separate delay calculation for parallel arcs. The software normally operates in a high performance mode which optimizes the handling of multiple timing arcs between input and output pin pairs. By default, the SDF file generated will contain the same delay information for all of these arcs. To have the SDF recalculated with explicit pin pair data, you should use the option `-recompute_parallel_arcs` to cause delays for each parallel arc to be explicitly calculated. It is recommended to use this option for generating SDF for functional simulation applications. **Note**: When you use the `write_sdf -recompute_parallel_arcs` option timing analysis is performed again. You need to obtain a new delay calculation to generate the SDF, and another delay calculation to put the system back to where it was before `write_sdf` was specified. Since delay calculation is run again, timing analysis will be reset. **Note**: The `-recompute_parallel_arcs` parameter is obsolete. This parameter still works in this release, but to ensure compatibility with future releases update your scripts to use the `-recompute_delay_calc` parameter instead.

`-recrem [merge_always | split | merge_when_paired]`	Determines how `RECOVERY and REMOVAL` timing checks are written. • `merge_always` - Always writes combined `RECREM` checks, even if either a `RECOVERY`, or a `REMOVAL`, check does not exist. For example, if a `RECOVERY` check exists, but a `REMOVAL` check does not, the software writes a combined `RECREM` check in the SDF file as follows: RECREM (*value*) (*value*) () () • `split` - Splits the `RECREM` checks into a `RECOVERY` check and a `REMOVAL` check. • `merge_when_paired` - Writes combined `RECREM` checks only when both a `RECOVERY` and a `REMOVAL` check exist. For example, if both a `RECOVERY` and a `REMOVAL` check exist, the software writes a combined `RECREM` check in the SDF file as follows: RECREM (*value*) (*value*) (*value*) (*value*) However, if a `RECOVERY` check exists, but a `REMOVAL` check does not, the software only writes the `RECOVERY` check in the SDF file: RECOVERY (*value*) (*value*) *Default:* `merge_always` **Note**: The `RECREM` sum should be zero or greater than zero. A negative value will be adjusted to enforce this condition. This is an SDF v3.0 requirement (the software may issue a warning).

`-remashold`	Converts the SDF v2.1 unsupported `REMOVAL` (and the `REMOVAL` portion of the `RECREM` constructs) to `HOLD` checks when used with the `-version 2.1` option. *Default*: Not to write out the unsupported checks. Converts the `REMOVAL` checks to `HOLD` checks when combined with the `-version 3.0` option.The `RECREM` checks are split into a `RECOVERY` check and a `HOLD` check. This option automatically splits the `RECREM` checks. *Default*: To maintain the version 3.0 checks.
`-scale` *float*	Specifies a multiplier to the delay values used in the SDF file. This option does not change the `timescale` setting of the SDF file. The delay values are scaled, but the units are the same. *Default*: `1`

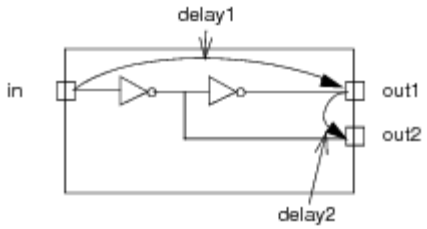
`-setuphold [merge_always | split | merge_when_paired]`	Determines how `SETUP and HOLD` timing checks are written. • `merge_always` - Always writes combined `SETUPHOLD` checks, even if either a `SETUP` or a `HOLD`, check does not exist. For example, if a `SETUP` check exists, but a `HOLD` check does not, the software writes a combined `SETUPHOLD` check in the SDF file as follows: SETUPHOLD (*value*) (*value*) () () • `split` - Splits the `SETUPHOLD` timing checks into separate `SETUP` and `HOLD` checks. • `merge_when_paired` - Writes combined `SETUPHOLD` checks only when both a `SETUP` and a `HOLD` check exist. For example, if both a `SETUP` and a `HOLD` check exist, the software writes a combined `SETUPHOLD` check in the SDF file as follows: SETUPHOLD (*value*) (*value*) (*value*) (*value*) However, if a `SETUP` check exists, but a `HOLD` check does not, the software only writes the `SETUP` check in the SDF file: SETUP (*value*) (*value*) *Default:* `merge_always`
`-splitrecrem`	Splits the `RECREM` checks into a `RECOVERY` check and a `REMOVAL` check. *Default*: To write the combined `RECREM` check. **Note:** Use this option only with the `version 3.0` option.

`-splitsetuphold`	Splits the `SETUPHOLD` timing checks into separate `SETUP` and `HOLD` checks. *Default*: To write the combined `SETUPHOLD` checks. **Note**: In SDC v3.0, the negative values are illegal in `SETUP`, `HOLD`, `RECOVERY`, and `REMOVAL` checks. Therefore, the negative values are converted to zero automatically with the `-splitsetuphold` and `-splitrecrem` parameters.

`-temperature`	Defines the temperature values that will appear in the header of the generated SDF file.
	Use the colon delimiter to specify multiple temperature values. For example:
	
	`-temperature -40.0:25.0:125`
	
	In the above example, the header of the generated SDF file will contain:
	
	`(TEMPERATURE -40.0:25.0:125)`

`-timingcheck {order}`	Sorts the check delays in OCV analysis.

`-target_application { sta | verilog}`	Allows customization of the SDF output for use with STA or Verilog applications. You can select the following options: • sta: You can use the sta option to configure the SDF output for use with static-timing analysis (STA) applications. This includes sign-off timing analysis with products, such as Cadence Tempus, as well as for working with implementation flows that use STA for internal timing analysis. The sta option preserves maximum accuracy and generates SDF output that matches the structure of the Liberty timing libraries. This option is equivalent to the following options: `write_sdf -interconn all`, `-celltiming all`, `-edges edged`, `-setupholdmerge_when_paired`, and `-recrem merge_when_paired`. • verilog: You can use the verilog option to generate SDF output that is compatible with Cadence Verilog-XL or Verilog-NC simulators. This option is equivalent to the following options: `write_sdf -filter -edges check_edge`, `-setuphold merge_when_paired`, `-recremmerge_when_paired`, `-min_period_edges -none`, `-collapse_internal_pins`, and `-recompute_parallel_arcs`. When the `-target_application` option is not specified then by default SDF output is configured for use with STA applications.

`-transform_out_to_out_arcs`	Writes out all output-to-output timing arcs as input-to-output arcs. Input-to-output arc delays are computed by adding the delay from the input pin to one of the output pins plus the delay from that output pin to the other output pin. For instance, in the following figure, the delay between `in` and `out2` would be the sum of `delay1` and `delay2`. 
`-typ_view` *`viewName`*	Uses the late delay from the specified analysis view to populate the SDF `typ` slot, when in multi-mode multi-corner analysis mode.
`-version {2.1 | 3.0}`	Specifies whether to generate the SDF V2.1 or V3.0. The default SDF version is 3.0. Used in combination with the `-remashold` option.
`-view` *`viewName`*	Uses the early and late delays for the specified analysis view to populate the `min` and `max` SDF triplet slots, when in multi-mode multi-corner analysis mode. The SDF `typ` slot remains empty (*`earlyVal`*`::`*`lateVal`*). In multi-mode multi-corner analysis mode, if you want to use SPEF information when calculating the delays, you must use `spefIn -rc_corner` to import the SPEF files for each RC corner in the design.

`-voltage`	Defines the voltage values that will appear in the header of the generated SDF file.
	Use the colon delimiter to specify multiple voltage values. For example:
	`-voltage 1.980:1.800:1.620`
	In the above example, the header of the generated SDF file will contain:
	`(VOLTAGE 1.980:1.800:1.620)`

Table: **Definition of -edges Option Values**

	edged Value	**library Value**	**noedge Value**
IOPATH from a unate input pin to output pin	No edge specifier	No edge specifier	No edge specifier
IOPATH from a non-unate input pin to output pin	Edge specifier at the input pin	Edge specifier at the input pin when sdf_edges is either "both_edges" or "start_edge"	No edge specifier When merging edged paths, the maximum values are used for MAX and TYP fields; minimum values are used for MIN fields.
IOPATH of a clock/enable pin to data out pin	Edge specifier at the input pin	Edge specifier at the input pin when sdf_edges is either "both_edges" or "start_edge"	No edge specifier
Timing Checks	Edge specifiers match the check arcs specified in the timing library	Overridden by value of the sdf_edges attribute	Only the edge specifiers of the reference pin are written

		For both_edges, the timing checks will contain edge specifier on both starting and ending pins For start_edge, the starting pin (for HOLD type check) or ending pin (for SETUP type check) is edge-specified For end_edge, the ending pin (for HOLD style check) or the starting pin (for SETUP type check) is edge-specified The timing checks are duplicated or combined appropriately. The start_edge and end_edge are interpreted as first and second events, respectively, of the Verilog-XL's $hold() and $setup system tasks	

Examples

- The following command writes out an SDF 2.1 compliant output file named `my.sdf` with 4 digits after the decimal point: `write_sdf -version 2.1 -precision 4 my.sdf`
- The following example SDF files compare the output of `-edges edged` to the output of `-edges noedge`.

  Resultant SDF with `-edges` set to `edged`:

  ```
  (HOLD (posedge D) (posedge CK) (-0.14:-0.14:-0.14)) (HOLD (negedge D) (posedge CK) (-0.06:-0.06:-0.06)) (SETUP (posedge D) (posedge CK) (0.22:0.22:0.22)) (SETUP (negedge D) (posedge CK) (0.41:0.41:0.41))
  ```

  With `-edges` set to `noedge` only the edge specifiers of the clock pin are written.

  ```
  (HOLD D (posedge CK) (-0.14:-0.06:-0.06)) (SETUP D (posedge CK) (0.22:0.41:0.41))
  ```

Related Information

- `read_sdf`

write_timing_windows

```
write_timing_windows
[-help]
[-pin]
[-power_compatible]
[-view string]
[-ssta_sigma_multiplier float]
[-voltage_threshold string]
output_file
```

Generates a timing window file (TWF). The TWF file mainly contains the earliest and the latest possible arrival times that a signal may arrive on a net or a pin.

The format of a timing window file consists of eight pairs of data as given below:

```
(NET <netname>

minRTW:maxRTW minRt:maxRt minRDr:maxRDr minRSlk:maxRSlk minFTW:maxFTW minFt:maxFt
minFDr:maxFDr minFSlk:maxFSlk)
```

Where:

- `minRTW`: Minimum rise arrival time
- `maxRTW`: Maximum rise arrival time
- `minRt`: Minimum rise transition time
- `maxRt`: Maximum rise transition time
- `minRDr`: Minimum rise source resistance
- `maxRDr`: Maximum rise source resistance
- `minRSlk`: Minimum rise slack time
- `maxRSlk`: Maximum rise slack time
- `minFTW`: Minimum fall timing window
- `maxFTW`: Maximum fall timing window
- `minFt`: Minimum fall transition time
- `maxFt`: Minimum fall transition time
- `minFDr`: Minimum fall source resistance
- `maxFDr`: Maximum fall source resistance

- `minFSlk`: Minimum fall slack time
- `maxFSlk`: Maximum fall slack time

Parameters

`-help`	Prints out command usage.
`output_file`	Writes the TWF file to the specified file name. The *`filename`* parameter must be the last argument in the list. **Note:** To write a compressed file, add the `.gz` extension to the file name.
`-pin`	Generates a TWF for each pin instead of each net. *Default*: Generates a TWF for the crosstalk analysis tool, which contains timing windows on each net.
`-power_compatible`	Generates a timing window file that is suitable for power analysis.
`-ssta_sigma_multiplier` *`float`*	Specifies a sigma multiplier between `0` and `3`. The value that you specify increases the arrival window by that amount and worst slack value is generated. The transition and impedance values are not impacted.
`-view` *`string`*	Generates a TWF for the specified multi-mode multi-corner view only. You must be multi-mode multi-corner analysis mode to use this parameter. If the `-view` parameter is not specified, then by default the first active view will be used to generate timing windows.
`-voltage_threshold` *`string`*	Specifies voltage threshold values for generating a Timing Window File (TWF). All the slews in the TWF file will be scaled with these values. The lower threshold value must be less than the higher threshold value. *Value range:* `0` to `100`

Examples

- The `write_timing_windows -voltage_threshold {20 80}` command outputs the following in the TWF file:

  ```
  (VOLTAGE_THRESHOLD 20 80)
  ```

- When active clocks are set using the `set_active_clocks` command, the `write_timing_windows` command will not display windows for inactive clocks in the output. The example report is shown below:

```
(TIMING_WINDOWS
(HEADER
(VERSION "0.1")
(DATE "Tue Apr 2 23:07:58 2013")
(DESIGN "top")
(DELIMITERS "/[]")
(VOLTAGE_THRESHOLD 20 80)
(PROCESS 1.000 1.000 1.000)
(VOLTAGE 1.012 1.012 1.012)
(TEMPERATURE 125.000 125.000 125.000)
(OPERATING_CONDITIONS DEFAULT DEFAULT DEFAULT)
(TIME_SCALE 1.000E-12)
(CAP_SCALE 1.000E-12)
(RESISTANCE_SCALE 1.000E+00)
(DEFAULT_INPUT_SLEW 0.000000 0.000000 0.000000 0.000000)
(PROGRAM "Innovus" "v13.10-dev.festa ((64bit) 04/02/2013 23:24 (Linux 2.6))"
"Cadence")
)
(WAVEFORM "Iclk1" 10000.000000 (POSEDGE 0.000000) (NEGEDGE 5000.000000))
(WAVEFORM "Iclk2" 15000.000000 (POSEDGE 0.000000) (NEGEDGE 7500.000000))
(WAVEFORM "Iclk3" 15000.000000 (POSEDGE 0.000000) (NEGEDGE 7500.000000))
(WAVEFORM "Iclk4" 10000.000000 (POSEDGE 0.000000) (NEGEDGE 5000.000000))
(CAUSED_BY "Iclk1" RISE
(NET "in1" 0:200 120:120 * 9200 0:200 120:120 * 9200 D)
)
(CAUSED_BY NULL
(NET "sel" 0:200 120:120 * * 0:200 120:120 * * D)
)
(CAUSED_BY "Iclk4" RISE
(NET "clk1" 0:200 120:120 * * * 120:120 * * D)
)
(CAUSED_BY "Iclk4" FALL
```

```
(NET "clk1" * 120:120 * * 5000:5200 120:120 * * D)
)
(CAUSED_BY "Iclk1" RISE
(NET "clk1" 0:200 120:120 * * * 120:120 * * D)
)
(CAUSED_BY "Iclk1" FALL
(NET "clk1" * 120:120 * * 5000:5200 120:120 * * D)
)
(CAUSED_BY "Iclk1" RISE
(NET "clk1" 0:200 120:120 * * 5000:5200 120:120 * * C)
)
(CAUSED_BY "Iclk4" RISE
(NET "clk1" 0:0 120:120 * * 5000:5000 120:120 * * C)
)
(CAUSED_BY NULL
(NET "clk2" 0:200 120:120 * * 0:200 120:120 * * D)
)
(CAUSED_BY "Iclk1" RISE
(NET "out1" 1200:2600 12.7:12.7 * 1300:7400 1200:2600 10.8:10.8 * 1300:7400 D)
)
(CAUSED_BY "Iclk4" RISE
(NET "out1" 300:800 12.7:12.7 * 400:9200 300:800 10.8:10.8 * 400:9200 D)
```

Related Information

- write_timing_windows
- read_twf

writeAnnotatedTransition

```
writeAnnotatedTransition
[-help]
-file out_file
[-min | -max]
[-writeOutput]
[-direct]
[-noPort]
[-view view_name]
```

Writes the value of transitions on each pin to an output file. You can either set the transitions using the `set_annotated_transition` command or the software computes them internally.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `writeAnnotatedTransition` parameter. For a detailed description of the command and all of its parameters, use the man command: `man writeAnnotatedTransition`.
`-direct`	Refers the ports directly for which the annotated transition will be applied in the output SDC. This option removes the dependency of using the `get_ports` and `get_pins` commands.
`-file out_file`	Specifies the name of the output file.
`-min | -max`	Specifies whether to write out minimum or maximum transition. *Default:* Both minimum and maximum transitions are written out.
`-noPort`	Skips writing the transition values for the primary ports (input and output).

`-view` *`view_name`*	Writes the transitions for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.
`-writeOutput`	Writes the slews for input and output pins, and the output slew for bi-directional pins. If this option is not specified, the software writes the slew for input pins and the input slew of bi-directional pins.

Examples

- The following command generates the output file `tran.rpt`:

  ```
  writeAnnotatedTransition -file tran.rpt
  ```

  The following file is generated:

  ```
  set_annotated_transition -rise -min 0.0278 [get_ports {sig_out4000}]
  set_annotated_transition -fall -min 0.0275 [get_ports {sig_out4000}]
  set_annotated_transition -rise -max 0.0278 [get_ports {sig_out4000}]
  set_annotated_transition -fall -max 0.0275 [get_ports {sig_out4000}]
  set_annotated_transition -rise -min 0.001 [get_pins {buf_in2000/A}]
  set_annotated_transition -fall -min 0.001 [get_pins {buf_in2000/A}]
  set_annotated_transition -rise -max 0.001 [get_pins {buf_in2000/A}]
  set_annotated_transition -fall -max 0.001 [get_pins {buf_in2000/A}]
  set_annotated_transition -rise -min 0.001 [get_pins {buf_in2001/A}]
  set_annotated_transition -fall -min 0.001 [get_pins {buf_in2001/A}]
  set_annotated_transition -rise -max 0.001 [get_pins {buf_in2001/A}]
  set_annotated_transition -fall -max 0.001 [get_pins {buf_in2001/A}]
  set_annotated_transition -rise -min 0.0453 [get_pins {buf_out5000/A}]
  set_annotated_transition -fall -min 0.0418 [get_pins {buf_out5000/A}]
  set_annotated_transition -rise -max 0.0453 [get_pins {buf_out5000/A}]
  set_annotated_transition -fall -max 0.0418 [get_pins {buf_out5000/A}]
  ```

- The following command generates the minimum transitions in the output file, `min_tran.rpt`:

```
writeAnnotatedTransition -min -file min_tran.rpt
```

The following file is generated:

```
set_annotated_transition -rise -min 0.0278 [get_ports {sig_out4000}]
set_annotated_transition -fall -min 0.0275 [get_ports {sig_out4000}]
set_annotated_transition -rise -min 0.001 [get_pins {buf_in2000/A}]
set_annotated_transition -fall -min 0.001 [get_pins {buf_in2000/A}]
set_annotated_transition -rise -min 0.001 [get_pins {buf_in2001/A}]
set_annotated_transition -fall -min 0.001 [get_pins {buf_in2001/A}]
set_annotated_transition -rise -min 0.0453 [get_pins {buf_out5000/A}]
set_annotated_transition -fall -min 0.0418 [get_pins {buf_out5000/A}]
```

- The following command generates the maximum transitions in the output file, `max_tran.rpt`:

  ```
  writeAnnotatedTransition -max -file max_tran.rpt
  ```

  The following file is generated:

  ```
  set_annotated_transition -rise -max 0.0278 [get_ports {sig_out4000}]
  set_annotated_transition -fall -max 0.0275 [get_ports {sig_out4000}]
  set_annotated_transition -rise -max 0.001 [get_pins {buf_in2000/A}]
  set_annotated_transition -fall -max 0.001 [get_pins {buf_in2000/A}]
  set_annotated_transition -rise -max 0.001 [get_pins {buf_in2001/A}]
  set_annotated_transition -fall -max 0.001 [get_pins {buf_in2001/A}]
  set_annotated_transition -rise -max 0.0453 [get_pins {buf_out5000/A}]
  set_annotated_transition -fall -max 0.0418 [get_pins {buf_out5000/A}]
  ```

Related Information

- set_annotated_transition
- get_ports
- get_pins

writeDesignTiming

`writeDesignTiming` *`filename`*

Writes out the timing information of the design.

Use this command when you run the NanoRoute® router in timing-driven mode. The router prioritizes each net to route based on the timing information specified in the output file. Do not edit the file because it is internally generated for the router.

By default, when the router runs with `setNanoRouteMode -route_with_timing_driven true` specified, it automatically generates a timing information file named `timingfile.tif` in the working directory, and uses it for timing-driven routing.

However, you can control the timing information passed to the router by specifying a timing information file with `writeDesignTiming`. If you specify a file with this command, and also specify `setNanoRouteMode -timingEngine` *`filename`*, the router uses the file written out by `writeDesignTiming` to drive routing, instead of the `timingfile.tif` file.

Parameters

`filename`	Specifies the output filename. **Note:** The output file is for internal tool use only. You must not edit the output file.

Related Information

- `setNanoRouteMode`

writeTimingCon

```
writeTimingCon
[-help]
[<filename>]
[-clocks <string>]
[-commands <string>]
[-exclude_annotated {delays | checks | transitions}]
[-pins <string>]
[-view <viewName>]
```

Generates the timing constraints file in Design Compiler format.

Parameters

`-help`	Prints out the command usage.
`-clocks <string>`	Specifies the name of the clocks to be included in the SDC file.
`-commands <string>`	Specifies the name of the commands to be included in the SDC file.
`-exclude_annotated {delays | checks | transitions}`	Disables listing of annotated constraints. • `delays`: Suppresses dumping of `set_annotated_delay` constraints. • `checks`: Suppresses dumping of `set_annotated_check` constraints. • `transitions`: Suppresses dumping of `set_annotated_transition` constraints.
filename	Specifies the name of the constraints file. By default, the constraint file is generated in the current directory. To save to a different directory, you need to specify the complete path.
`-pins <string>`	Specifies the name of the pins to be included in the SDC file.

`-view` *viewname*	Specifies a view name to obtain the desired SDC.

Command Order

Use this command after reading in a timing constraints file.

Examples

- The following command generates constraints file `newc` and saves the generated file in the directory `new`:

  ```
  writeTimingCon new/newc
  ```

- Specifying the following constraints will have new entries in the output constraints file:

  ```
  set_annotated_delay -from u1/A -to u1/Y -cell 2
  set_annotated_check 2 -setup -from f0/CK -to f0/D
  ```

  The output constraint file `constr.pt` will have the following new entries:

  ```
  set_annotated_delay -cell 2 -from [get_pins {u1/A}] -to [get_pins {u1/Y}]
  set_annotated_check -setup -from [get_pins {f0/CK}] -to [get_pins {f0/D}] 2
  ```

- For the following constraints in the design :

  ```
  set_annotated_delay -from u1/A -to u1/Y -cell 2
  set_annotated_transition 2 u1/A
  set_annotated_check 2 -setup -from f0/CK -to f0/D
  ```

  The output constraint file `constr.pt` will have the following entries :

  ```
  set_annotated_transition 2 [get_pins {u1/A}]
  set_annotated_check -setup -from [get_pins {f0/CK}] -to [get_pins {f0/D}] 2
  set_annotated_check -setup -from [get_pins {f0/CK}] -to [get_pins {f0/D}] 2
  ```

Related Information

- set_annotated_delay
- set_annotated_transition
- set_annotated_check

38

Timing Budgeting Commands

- checkPartitionSdc
- deriveTimingBudget
- getBudgetingMode
- justifyBudget
- justifyException
- modifyBudget
- resetModifiedBudget
- saveTimingBudget
- setBudgetingMode
- setCycleBudgetRatio
- setFixedBudget
- setPtnUserCnsFile

checkPartitionSdc

```
checkPartitionSdc
[-help]
{[-chip string -partition_dir string] | [-chip string -module_model_tag string [-add_ons string] [-accumulated] [-overwrite_tag]] | [-
compare_only]}
[-cells string]
[-master_only]
[-no_top]
[-out_dir string]
[-validate_boundary_condition]
```

When you generate the timing budgets for partitions using the `deriveTimingBudget` command, the `checkPartitionSdc` command validates the results. The validation ensures that the timing data of the partitioned block is the same as that of the chip. This saves the ECO iteration for the chips and partitioned blocks, and thereby reduces the overall design cycle time.

Note: This validation is carried out independently by comparing the constraints data and timing data.

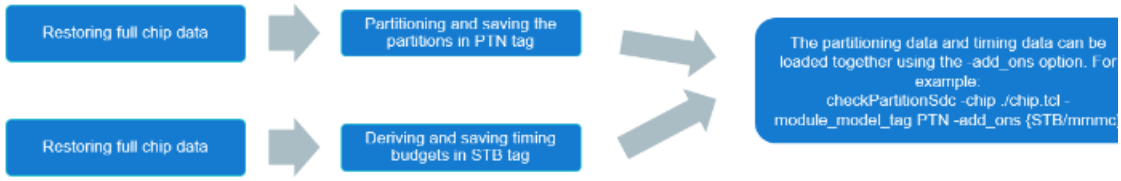
The constraints data can be generated from the third-party tools well. To run the checkPartitionSdc command, the constraints data and timing data for the chips and blocks must be available in the Innovus format.

On running this command, some simple constraints are directly validated though the SDC comparison. For some other checks related to pins, the chip and block values are compared. However, for some complex exception constraints, an independent validation is done by checking and comparing the phases reaching at the boundary pins of the chips and the blocks.

The comparison checks the files of the missing, extra, and mismatching constraints in the blocks and the chips. These can help in fixing the partitioned block constraints to get accurate budgets.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `checkPartitionSdc` parameter. For a detaile description of the command and all its parameters, use the man command: `man checkPartitionSdc`
`-accumulated`	Runs comparison for the accumulated budgeting mode.

`-add_ons string`	Specifies the supplement model that can be loaded additionally with an existing model or a design. It can be used with the `-module_model_tag` parameter. This parameter can only be used in the Integrated Hierarchical Database (iHDB) flow. This parameter is useful for the flows where partitioning and timing budgeting are done in parallel, as presented below: 
`-cells string`	Specifies the list of cell names for which the SDC comparison is to be done. *Default:* `all cells`
`-chip string`	Specifies the TCL file, which loads the chip database before timing budgeting.
`-compare_only`	Compares the dumped data with the previous `checkPartitionSdc` run.
`-master_only`	Dumps and compares constraints for checking only the master partitions for the master-clone designs. In case this parameter not used, the tool dumps and compares constraints taking clones also in consideration.

`-module_model_tag string`	Specifies the module model tag for the cell on which the SDC comparison is to be done.
	For example, the following commands validate the timing budgeting results for the specified module model:
	`set_module_model -default_dir ./DATA2`
	`checkPartitionSdc -chip scripts/chip.tcl -cells $block -module_model_tag init -no_top`
	Note: This parameter is a part of the iHDB flow. Before using this parameter, you must set the default central directory name (`set_module_model -default_dir default_model_data_directory`) where the module models data will be saved/loaded.

`-no_top`	Disables checking of the TOP cell.
`-overwrite_tag`	Regenerates design for the accumulated Budgeting mode.
`-out_dir string`	Specifies the directory, which contains the output in the form of validation comparison reports and log files. *Default*: `./cps`
`-partition_dir string`	Specifies the directory, which contains the partition database.

`-validate_boundary_condition`	Compares the slew values at the chip versus block level. Slew values are generated using the `report_timing` command, and the comparison is done at immediate pins to the block boundary.
	This parameter can be used in the same session in which the default `checkPartitionSdc` check is used. However, it should n run in parallel with the default `checkPartitionSdc` check since writing of setup file may have a conflict.
	The `-validate_boundary_condition` parameter can validate only the boundary conditions for the partition-block constraints generated for `setBudgetingMode -useBoundaryCondition actual` and build the timing graph from the actual netlist. This parameter dumps an output file in the .csv format.
	⚠ The `-validate_boundary_condition` parameter cannot support the `setBudgetingMode -useBoundaryCondition {optimized | template}` values. It also gives an error, if the timing graph is not built.
	Following is a sample output of using the `-validate_boundary_condition` parameter:
	`+---------------------------+-----------+------------+----------+`
	`|  Pin  | Chip Slew | Block Slew | Diff (%) |`
	`+---------------------------+-----------+------------+----------+`
	`| slave_haddr_o[0]  |  0.620 |  0.613 |  1.13 |`
	`| rxdata1[6]  |  0.266 |  0.269 |  1.13 |`
	`| rxdata1[7]  |  0.270 |  0.273 |  1.11 |`
	`| master_hready_o  |  0.364 |  0.368 |  1.09 |`
	`| SPCASCAN_N523  |  0.734 |  0.742 |  1.09 |`
	`| rxdata2[1]  |  0.196 |  0.198 |  1.02 |`
	`| slave_hrdata_i[0]  |  0.098 |  0.097 |  0.11 |`
	`+---------------------------+-----------+------------+----------+`

Examples

- The following command runs validation using the chip database `chip.tcl` and post-partition saved along with post-budgeted database in `ptn` tag:
  ```
  checkPartitionSdc -chip ./chip.tcl -module_model_tag ptn
  ```
- The following command runs validation using the chip database `chip.tcl`, the post-partition database in the `ptn` tag, and post-budgeted database in the `ptn` tag, which are saved in parallel:
  ```
  checkPartitionSdc -chip ./chip.tcl -module_model_tag ptn -add_ons {mmmc}
  ```
- The following command runs validation using the chip database `chip.tcl`, the post-partition database in the `ptn` tag, and post-budgeted database in the `stb` tag:
  ```
  checkPartitionSdc -chip ./chip.tcl -module_model_tag ptn -add_ons {STB/mmmc}
  ```
- The following command runs validation using the chip database `chip.tcl` and the partition database in `./PTNS`:
  ```
  checkPartitionSdc -chip ./chip.tcl -partition_dir ./PTNS
  ```
- The following command runs validation using the chip database `chip.tcl` and the partition database in `./PTNS` for cells P1 and P2:
  ```
  checkPartitionSdc -chip ./chip.tcl -partition_dir ./PTNS -cells {P1 P2}
  ```
- The following command runs validation using the chip database `chip.tcl`, the partition database in `./PTNS`, and the dumping reports and log in `./CHK`:
  ```
  checkPartitionSdc -chip ./chip.tcl -partition_dir ./PTNS -out_dir ./CHK
  ```
- The following command runs validation using the chip database `chip.tcl`, and partition database in `./PTNS`, and only checks the partitions data (no portioned top checking with chip):
  ```
  checkPartitionSdc -chip ./chip.tcl -partition_dir ./PTNS -no_top
  ```

Related Information

Refer to the Timing Budgeting chapter in the *User Guide* document.

deriveTimingBudget

```
  deriveTimingBudget
[-help]
[-parameterized]
[-cycleRatio | -stageBased | -timingCore | -preliminary | -pushDownSDC]
[-allIlmHinsts | -inst instanceName | -ptn partitionName]
[-verbose]
[-views string]
```

Generates timing budgets for partitions for standalone implementation. This command can also generate timing budgets on an instance basis instead of partitions. The generated timing constraint files allow you to do standalone block implementation. In case of partition based budgeting, it also generates timing models that will allow you to do standalone top level implementation. This command also directs summary files into the default budget justification directory.

Note: You must free the timing graph to use multiple `deriveTimingBudget` commands with different options.

Innovus operates in multi-mode multi-corner (MMMC) mode, and generates `viewFiles`automatically for partition implementation and top-level chip assembly. Note that the `deriveTimingBudget` command does not generate a timing graph, if it already exists. Additionally, user will have to provide a timing graph generated without using `deriveTimingBudget` and must set the value of the `setBudgetingMode -virtualOptEngine` parameter same as that of the command used to generate the timing graph.

When the `deriveTimingBudget` command is run, all the public `setBudgetingMode` parameters with their non-default values are displayed before the intended output of the `deriveTimingBudget` command, in an information message. This information message follows the format shown below:

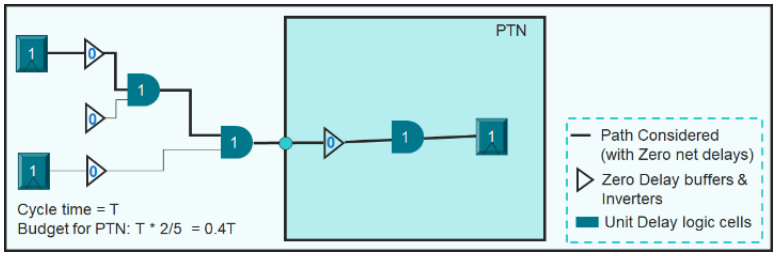
```
This command "deriveTimingBudget" required an extra checkout of license invs_hier. Additional license(s) checked out: 1 'Innovus_Hier_Opt'
license(s)
**INFO: User settings:
setBudgetingMode -abutted true
setBudgetingMode -fixTopLevelPaths all
setBudgetingMode -masterClone merged
setBudgetingMode -noHoldView true
.
.
```

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `deriveTimingBudget` parameter. For a detailed description of the command and all its parameters, use the man command: `man deriveTimingBudget`
`-allIlmHinsts`	Derives timing budgets only for all the ILM-based partitions. When this parameter is used, the software automatically derives the list of ILM instances and budgets for them. **Note:** The `-allIlmHinsts` parameter internally calls `-inst` based budgeting. So, saving budgets is only supported through the `saveTimingBudget` command. Therefore, it is necessary to run `saveTimingBudget` after `deriveTimingBudget -allIlmHinsts`, else `savePartition` will not save the budgeting information of ILM instances.

`-cycleRatio`	Honors the `setCycleBudgetRatio` constraints and enables usage of clock cycle time based budgeting. If no `setCycleBudgetRatio` constraints are applied:
	• The top and block default ratio is 20% and 80% of the clock respectively.
	• The partition for start and end points splits the block ratio in 50% each.
	• Equivalent default constraint is as follows:
	`setCycleBudgetRatio -fromPtn {* .4} -toPtn {* .4}`
	`setCycleBudgetRatio -fromPtn {* .8} -toTop`
	`setCycleBudgetRatio -fromTop -toPtn {* .8}`
	Note: Do not use the `setFixedBudget` command with this parameter.

`-inst` *instanceName…*	
	Performs budgeting for the hierarchical instance(s). This instance could be a black box. You can derive timing budgets for black boxes and other instances in the same run. Specify the `setBudgetingMode -mergeClones` parameter in addition to the `-inst` parameter to obtain worst-case budgets based on worst-case data per-pin for specified instances in a master/clone set. If you specify instances from more than one master/clone set, the tool derives separate worst-case timing budgets for the specified instances in each set. *Default:* If you do not specify this parameter, the software derives budgets for all the partitions.

`-parameterized`	Generates constraints as a function of the clock period. When you set this parameter, the software generates timing constraints, such as `set_input_delay` and `set_output_delay` constraints as a function of the clock period in the top-level and block-level SDCs.
`-preliminary`	Derives timing budgets in the early stages of the design to get a preliminary estimate of the budgets. When you use this parameter, the software uses the net delays and net loads that you specify using the `setBudgetingMode` command. The software does not use virtual optimization for calculating budgets. The software distributes the positive slack equally between source block, destination block, and top-level. For more information, see the 'Deriving Preliminary Budgets in Early Design Phase' section in the Timing Budgeting chapter of the *Innovus User Guide*.
`-ptn` *`partitionName`*	
	Performs budgeting for the specified partition(s). Specify the `setBudgetingMode -mergeClones` parameter in addition to the `-ptn` parameter to obtain worst-case budgets based on worst-case data per-pin for all partitions in the master/clone set containing the partition. You cannot specify `-ptn` if you use `-inst`. *Default:* Performs budgeting on all partitions.
`-pushDownSDC`	Helps timing budgeting in creating only pushdown constraints for block partitions. The constraints applied to only those paths, which are completely inside a partition, are known as pushdown constraints. By considering only the pushdown constraints, this parameter ignores the constraints applied on partition interface paths from full chip SDC. **Note:** In the early design phase, the reg-to-IO reg-to-IO and IO-to-reg paths can be ignored using this parameter. In such a case, the timing constraints for only reg-to-reg paths are generated.
`-stageBased`	Creates a quick stage-based timing graph to be used for timing budgeting. Timing budgeting with the stage-based timing graphs is used in very early stages of the design, before applying optimization for quick budgets. This method is more accurate than the cycle-based timing budgeting. When this parameter is used, any existing timing graph in the design is overwritten by the generated special quick stage-based timing graph. The following example presents the scenarios in which this parameter can be used: **reg2reg path** 
`-timingCore`	Generates timing constraints for the partition core (internal register-to-register path only). The following are included in the constraints file: • Clock definitions • Core timing constraints such as `set_multicycle_path`, `set_false_path, and set_max_delay`. These constraints are pushed down from the top-level constraints. *Default:* Generates timing budget files for each partition from the imported top-level timing constraint file. **Note:** The software does not generate budgets for the interface ports of the partition. Therefore the `set_input_delay` or `set_output_delay` values are not generated for the partition ports.
`-verbose`	Enables various verbose controls.
`-views` *`string`*	Derives budget for multiple views in parallel for reducing the runtime.

Examples

- For MMMC examples, see the "Timing Budgeting" chapter of the *Innovus User Guide*.
- The following command creates a budget for the black box BBox:

```
deriveTimingBudget -inst BBox
saveTimingBudget -inst BBox
```

Related Information

- The following sections in the Timing Budgeting chapter of the *User Guide* document:
  - "Deriving Timing Budgets"
    - "Budgeting Using Text Commands"
    - "Top-Level Budgets Derived by Using Active Logic View"
    - "Deriving Preliminary Budgets in Early Design Phase"
  - "Budgeting Output Files for MMMC Designs"
  - "Analyzing Timing Budgets"
    - "Resolving Conflicts with Path-Based Exceptions"
  - "Customizing Budget Generation"
  - "Verifying Timing Budgets"
  - "Reading the Justify Budget Report"
  - "Warning Report"

getBudgetingMode

```
  getBudgetingMode
[-help]
[-abutted]
[-accumulated]
[-boundaryConditionTemplatePath]
[-bufferDelayAdjustment]
[-bufferDelayLibCell]
[-bufferDelaySelectionEffort]
[-ccd]
[-constantModel]
[-distributeMMMC]
[-dontWriteWireLoad]
[-driveCell]
[-fixTopLevelPaths]
[-handleComplexSDC]
[-honorPortBoundaryCondition]
[-honorPortDelays]
[-honorReportTimingFormat]
[-ignoreDontTouch]
[-includeLatency]
[-includeWireLoadsInLib]
[-inputLoad]
[-inputTransition]
[-justify]
[-justifyBudgetDir]
[-keepPinListsForBlockPorts]
[-latencyOnClocks]
[-localLatency]
[-localUncertainty]
[-makeNegativeInputDelayZero]
[-masterClone]
[-noFalsePathsForUnCstrPorts]
[-noSetupView]
[-noHoldView]
[-overrideNetCap]
[-reportOrModifyBudget]
[-rptNegSlackOnPorts]
[-snapFdBudgetTo]
[-snapInputBudgetTo]
[-snapNegativeOnly]
[-snapOutputBudgetTo]
[-stageBasedFanoutDrivingFactor]
[-stageBasedPartitionMultiplier]
[-stageBasedWeight]
[-topLevel]
[-topLevelDelayPerLen]
[-topLevelMinDelayPerNet]
[-useBoundaryCondition]
[-useRealCellForTimingModel]
[-virtualClock]
[-virtualOptEngine]
[-writeConstraintsForClkOutputPorts]
[-writeFPForHold]
[-writeLatencyPerClock]
[-quiet]
```

Displays the following information about a specified budgeting mode parameter in the Innovus log file and in the Innovus console.

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the software displays information for all of the budgeting mode parameters.

Parameters

-parameter_names	
	Displays information for the specified parameters. You can specify one or more parameters. See `setBudgetingMode` for descriptions of the budgeting mode parameters you can specify.
`-quiet`	
	Displays the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software displays the current settings of all `setBudgetingMode` parameters in Tcl list format.

Examples

- The following command displays the current setting for the `-masterClone` parameter:

  ```
  getBudgetingMode -masterClone
  ```

  The software displays the following information:

  ```
  -masterClone masterOnly # enums={masterOnly merged uniqueViewPerHInst}, default=masterOnly
  masterOnly
  ```

- The following command displays the current setting for the `-abutted` parameter in Tcl list format:

  ```
  getBudgetingMode -abutted -quiet
  ```

  The software displays the following information:

  ```
  false
  ```

- The following command displays the current settings for the `-localLatency` and `-localUncertainty` parameters:

  ```
  getBudgetingMode -localLatency -localUncertainty
  ```

  The software displays the following information:

  ```
  -localLatency false # bool, default=false
  -localUncertainty false # bool, default=false
  {localLatency false} {localUncertainty false}
  ```

Related Information

The Timing Budgeting chapter of the *User Guide* document.

justifyBudget

```
justifyBudget
[-help]
{partitionName | instanceName}
{-pins pinList}
[-outfile fileName]
[-summary]
[-view viewName]
```

Queries and verifies the timing budgets generated by the Innovus software. You can also query budgets selectively for some/all partitions or instances.

The `justifyBudget` command honors the `report_timing_format` global, which allows you to customize the timing report according to the specified columns. You must set this global before specifying `justifyBudget` or `deriveTimingBudget -justify` to get the report in the desired format.

Note: In case of combined budgeting, the `justifyBudget` command will justify all the input or output delays attached to the port of a partition.

> The `justifyBudget` command only supports the `-summary` parameter in MMMC mode (the default mode). The report will contain only the headers. To print paths, use `deriveTimingBudget -justify`.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `justifyBudget` parameter. For a detailed description of the command and all its parameters, use the man command: `man justifyBudget`
`-outfile fileName`	
	Specifies the output file name that contains timing budget data for a pin.
`partitionName` | *`instanceName`*	
	Specifies the name of the partition/instance to analyze. When used with `-mergeClones`, the software reports the worst-case data for specified partition/instance pins in the master/clone set containing the partition/instance. The *`instanceName`* can only be specified when the `deriveTimingBudget` command is used with the `-inst` option. In default partition mode, the *`partitionName`* is specified.
`-pins` *`pinList`*	
	Specifies the hierarchical partition pin(s) to be analyzed. You can use `*` to specify all pins.
`-summary`	
	Displays the style summary report containing the header section of the full report to show budget calculation only. Use this parameter after using saveTimingBudget/savePartition.
`-view` *`viewName`*	
	Prints a report for the specified view. If you derived the timing budget in MMMC mode (default), this parameter is required to obtain a report.

Examples

- For examples of budgeting for MMMC designs, see the Timing Budgeting chapter of the *Innovus User Guide*.
- The following command generates the timing budget data for the pin input of partition/instance `partition1` and prints a report for view `SETUP_VIEW1`:
  ```
  justifyBudget partition1 -pins input -view SETUP_VIEW1
  ```
- The following command generates the timing budget data for all pins of the partition/instance `partition1` and prints a report for viewing `HOLD_VIEW1` in the file `justifyData`:
  ```
  justifyBudget partition1 -pins * -outfile justifydata -view HOLD_VIEW1
  ```

Related Information

- The following sections in the Timing Budgeting chapter of the *User Guide* document:
  - "Deriving Timing Budgets"
    - "Top-Level Budgets Derived by Using Active Logic View"
    - "Deriving Preliminary Budgets in Early Design Phase"
  - "Budgeting Output Files for MMMC Designs"
  - "Generated Report for Design Example"
  - "Verifying Timing Budgets"
  - "Reading the Justify Budget Report"

justifyException

```
 justifyException
{-pins pinList}
[-outfile fileName]
{partitionName | instanceName}
-view viewName
[-help]
```

Provides a debugging or justification mechanism of exceptions on ports. This command is used to debug exception constraints and justify all exceptions on pins specified by `-pins`. To justify exceptions, you must set the `-justify` parameter of the `setBudgetingMode` command to `true` to save the justify exception data during the `deriveTimingBudget` process.

Parameters

`-help`	Outputs the command usage.
*`partitionName	instanceName`*	
	Specifies the name of the partition or instance to analyze. The *`instanceName`* can only be specified when the `deriveTimingBudget` command is used with the `-inst` option. In default partition mode, the *`partitionName`* is specified.
`-outfile` *`fileName`*	
	Specifies the output file name that contains the justify exception data for the instances or partitions specified by *`instanceName`* or *`partitionName`*. If you do not specify an output file using the `-outfile` parameter, the exception data will be displayed on the screen.
`-pins` *`pinList`*	
	Specifies the hierarchical partition pins to be analyzed. You can use * to specify all pins.
`-view` *`viewName`*	
	Prints a report for the specified view.

Examples

- The following command justifies exception for the pins `input` of partition/instance `partition1` and prints a report for view `SETUP_VIEW1`:
  ```
  justifyException partition1 -pins input -view SETUP_VIEW1
  ```
- The following command justifies exception for all pins of the partition/instance `partition1` and prints a report for view `HOLD_VIEW1` in the file `justfyData`:
  ```
  justifyException partition1 -pins * -outfile justifydata -view HOLD_VIEW1
  ```

Related Information

- The following sections in the Timing Budgeting chapter of the *User Guide* document:
  - "Deriving Timing Budgets"
    - "Top-Level Budgets Derived by Using Active Logic View"
    - "Deriving Preliminary Budgets in Early Design Phase"
  - "Budgeting Output Files for MMMC Designs"
  - "Generated Report for Design Example"
  - "Verifying Timing Budgets"
  - "Reading the Justify Budget Report"

modifyBudget

```
modifyBudget
{-ptn partitionName | -inst instanceName}
-file fileName
[-setup | -hold]
[-view viewName]
[-help]
```

Modifies budgets after they have been saved using the `saveTimingBudget` command. To modify budgets, you must create an input budget file containing constraints that look like an SDC file. It only supports `set_input_delay` and `set_output_delay` constraints. The clock names in these constraints are same as the ones in the generated budget file. You can provide a bus name instead of a pin name to apply the same budget to all pins of the bus.

To use `modifyBudget`, you must specify the following command before `deriveTimingBudget`:

```
setBudgetingMode -reportOrModifyBudget true
```

Parameters

`-help`	Prints a brief description that includes the type and default information for each `modifyBudget` parameter. For a detailed description of the command and all its parameters, use the man command: `man modifyBudget`
`-file fileName`	Specifies the name of the input budget file containing manual budgets.
`-hold`	Specifies to modify the hold information.
`-inst instanceName`	Specifies the name of the hierarchical instance for which manual budgets are to be applied.
`-ptn partitionName`	Specifies the name of the partition for which manual budgets are to be applied.
`-setup`	Specifies to modify the setup information.
`-view viewName`	Modifies the budget for the specified view. If you derived the timing budget in MMMC mode (default), this parameter is required to modify the budget.

Example

The following command modifies the budget data for the instance inst1:

```
modifyBudget -inst inst1 -file mod_inst
```

Related Information

The "Modifying Budgets" section in the Timing Budgeting chapter of the *User Guide* document.

resetModifiedBudget

```
resetModifiedBudget
{-ptn partitionName | -inst instanceName}
-pin pinName
[-setup | -hold]
[-view viewName]
[-help]
```

Resets the budget that was modified using the `modifyBudget` command. `resetModifiedBudget` allows you to revert to the original generated budget if the modified budget is not yielding the correct timing for a block, or rectify an incorrectly modified budget.

Parameters

`-pin` *pinName*	Specifies the name of the pin for which manual budgets are to be reset. You can use `*` to specify all pins.
`-hold`	Specifies to reset the hold information.
`-inst` *instanceName*	Specifies the name of the hierarchical instance for which manual budgets are to be reset.
`-help`	Outputs the command usage.
`-ptn` *partitionName*	Specifies the name of the partition for which manual budgets are to be reset.
`-setup`	Specifies to reset the setup information.
`-view` *viewName*	Resets the budget for the specified view.

Example

- The following command resets the modified budget data for the instance `inst1`:

  ```
  resetModifiedBudget -inst inst1
  ```

Related Information

The "Modifying Budgets" section in the Timing Budgeting chapter of the *User Guide* document.

saveTimingBudget

```
saveTimingBudget
[-help]
[-dir dirName]
[{instList}]
[-inst string | -ptn string]
[-module_model_tag string]
```

Saves the previously derived timing budgets to a specified directory. The main outputs of this command are the block constraints and the timing model. In the MMMC mode, the `saveTimingBudget` outputs the block and top `viewdefinition.tcl` as well.

Use this command after deriving the time budgets using the `deriveTimingBudget` command.

Note: The software will not write PG information while using the `saveTimingBudget` command. To get accurate PG information in the `partition-block .lib`, run the following commands in sequence:

```
partition
savePartition
```

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `saveTimingBudget` parameter. For a detailed description of the command and all its parameters, use the man command: `man saveTimingBudget`
instList	Saves the budget files for instances. You use this parameter with the `deriveTimingBudget -inst` command.
`-dir` *dirName*	
	Specifies the relative directory name where the derived timing budget files will be saved. These files are saved in a sub-directory named `cellName_timingViewName.sdc.gz` at the `dirName/cellName/cellName_timingViewName.sdc.gz` location through the relative directory path. If you do not specify this parameter, the timing budget files are saved in the current directory. For example, if you run the following command, a directory named `STB` will be saved at the current path: `deriveTimingBudget -dir STB`
`-inst` *instList* | `-ptn` *ptnList*	
	Saves the budget file for partitions or instances for which you have generated timing budgets. You do not need to specify `-ptn` or `-inst` if you have previously specified `deriveTimingBudget -ilm`. The `-ptn` parameter saves the budget files for partitions. You use this parameter with the `deriveTimingBudget -ptn` command. The `-inst` parameter saves the budget files for instances. You use this parameter with the `deriveTimingBudget -inst` command. *Default:* The tool saves budgets for all instances and partitions specified by `deriveTimingBudget`.
`-module_model_tag` *string*	
	Creates a new `mmmc` directory and saves the view definitions, libs, and constraints therein. The created `mmmc` directory will be located as follows: *shdb_root*`/`*block_name*`/`*tag_name*`/mmmc` For example: • The `saveTimingBudget -module_model_tag ABCD` command will save view definition, libs, and constraints within the `shdb_root/block_name/ABCD/mmmc` directory. • The default behavior of the `savePartition -module_model_tag ABCD` command is not impacted, and the view definition, libs, and constraints are saved in the `.enc.dat` file within the `shdb_root/block_name/ABCD/pnr` directory. • The `saveTimingBudget -module_model_tag ABCD` and `savePartition -module_model_tag ABCD` commands are completely independent (those will not affect the functioning of each other).

Examples

The following examples are based on a scenario, where a design with the `ptn1` partition has two instances, `L1` as the master, and `L1_clone` as the clone:

- The following set of commands saves the timing budget files for partition in the `STB/ptn1` directory. It saves the timing budget information for both the

partition and top. Also, since it uses the `masterOnly` mode, the budgets will be based on the master timing only:

```
deriveTimingBudget -ptn ptn1
saveTimingBudget -dir STB
```

- The following set of commands saves timing budget files only for the `L1_clone` instance in the `STB/ptn1` directory. Since it uses the `merged` mode, the timing budgeting information will be based on the worst case data from both the instances. In this case, any of the master or clone instances, or both, can be specified while running `saveTimingBudget`. The best approach is to not specify either of those, and let the `saveTimingBudget` command save the information generated by the `deriveTimingBudget` command:

  ```
  setBudgetingMode -masterClone merged
  deriveTimingBudget -inst {L1 L1_clone}
  saveTimingBudget -dir STB -inst {L1_clone}
  ```

- The following set of commands saves the timing budget files for the `L1` instance in the `STB/ptn1` directory. It saves the timing budget information for only the partition instance:

  ```
  deriveTimingBudget -inst L1 (or L1_clone)
  saveTimingBudget -dir STB
  ```

- The following set of commands saves the timing budget files for the `L1` instance in the `STB/ptn1.L1` directory, and for the `L1_clone` instance in the `STB/ptn1.L1_clone` directory. It saves the timing budget information only for the partition instances:

  ```
  deriveTimingBudget -inst {L1 L1_clone}
  saveTimingBudget -dir STB
  ```

- The following set of commands saves the timing budget files only for the `L1_clone` instance in the `STB/ptn1.L1_clone` directory:

  ```
  deriveTimingBudget -inst {L1 L1_clone}
  saveTimingBudget -dir STB -inst {L1_clone}
  ```

Related Information

- The following sections in the Timing Budgeting chapter of the *User Guide* document:
  - "Deriving Timing Budgets"
    - "Top-Level Budgets Derived by Using Active Logic View"
    - "Deriving Preliminary Budgets in Early Design Phase"
  - "Budgeting Output Files for MMMC Designs"
  - "Generated Report for Design Example"
  - "Verifying Timing Budgets"
  - "Reading the Justify Budget Report"

setBudgetingMode

```
setBudgetingMode
[-help]
[-abutted {true|false}]
[-accumulated {true|false}]
[-boundaryConditionTemplatePath directory_path]
[-bufferDelayAdjustment value]
[-bufferDelayLibCell libname/cellname]
[-bufferDelaySelectionEffort {low|high}]
[-ccd {true|false}]
[-constantModel {true|false}]
[-distributeMMMC {true|false}]
[-dontWriteWireLoad {true|false}]
[-driveCell {true|false}]
[-fixTopLevelPaths {none|all|negativeOnly|positiveOnly}]
[-handleComplexSDC {true|false}]
[-honorPortBoundaryCondition {true|false}]
[-honorPortDelays {true|false}]
[-honorReportTimingFormat {true|false}]
[-ignoreDontTouch {true|false}]
[-includeLatency {true|false}]
[-includeWireLoadsInLib {true|false}]
[-inputLoad {true|false}]
[-inputTransition {true|false}]
[-justify {delay|exception|lib_arc|boundary_condition|all}]
[-justifyBudgetDir directory_name]
[-keepPinListsForBlockPorts {true|false}]
[-latencyOnClocks {true|false}]
[-localLatency {true|false}]
[-localUncertainty {true|false}]
[-makeNegativeInputDelayZero {true | false}]
[-masterClone {masterOnly|merged|uniqueViewPerHInst}]
[-noFalsePathsForUnCstrPorts {true|false}]
[-noSetupView {true|false}]
[-noHoldView {true|false}]
[-overrideNetCap value]
[-pinLoad {true|false}]
[-reportOrModifyBudget {true|false}]
[-reset]
[-rptNegSlackOnPorts value]
[-snapFdBudgetTo value]
[-snapInputBudgetTo value]
[-snapNegativeOnly {true|false}]
[-snapOutputBudgetTo value]
[-stageBasedFanoutDrivingFactor value]
[-stageBasedPartitionMultiplier {{HInst1 m1} {HInst2 m2}..}]
[-stageBasedWeight {S | C | B}]
[-topLevel value]
[-topLevelDelayPerLen value]
[-topLevelMinDelayPerNet value]
[-useBoundaryCondition {optimized | template | actual | empty}]
[-useRealCellForTimingModel {true|false}]
[-virtualClock {true|false}]
[-virtualOptEngine {none|earlyTimingEngine|proto}]
[-writeConstraintsForClkOutputPorts {true|false}]
[-writeFPForHold {true|false}]
[-writeLatencyPerClock {true|false}]
```

Sets the timing budgeting mode. This command governs the way budgets are derived and saved. All parameters relevant to saving or deriving can be specified with this command.

Parameters

`-help`

Outputs a brief description that includes the type and default information for each `setBudgetingMode` parameter. For a detailed description of the command and all its parameters, use the man command:

```
man setBudgetingMode
```

`-abutted {true | false}`

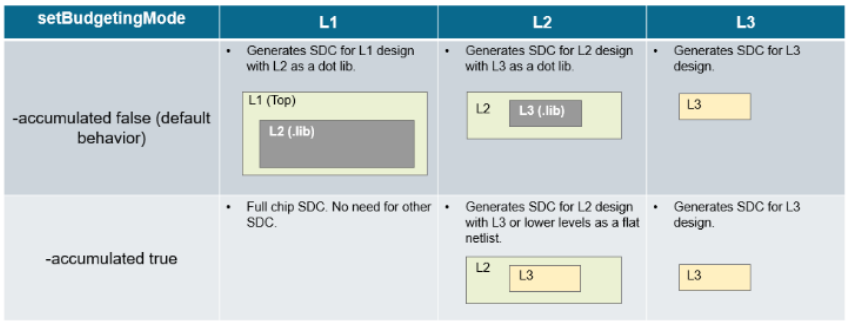
Specifies that the software abuts the partitions when allocating budgets from positive slack paths. With this parameter, the software assumes that all partitions are abutted and therefore the software does not allocate extra delay at the top-level.

Default: `false`

Note: The software automatically handles the abutted partitions (both partial and full) even when you do not specify this parameter.

`-accumulated {true | false}`

Generates SDC for lower-level hierarchy child partition in the .lib and flat forms (`L3` as shown in the example below). The following example also presents the matrix to understand the behavior of the `-accumulated` parameter at different levels of hierarchy:



With the help of the `-accumulated` parameter, SDC supports generation of constraints in the specified child partition through a Tcl variable-enabled conditional loop, as shown below:

```
if {$budget_use_accumulated != 1}{
#Constraints considering child partition in the .lib form
} else {
#Constraints considering child partition in the flat form
}
```

Note:

1. A single `viewDefinition.tcl` or SDC file can support both the .lib or flat flows. This is because there are conditional loops in the `viewDefinition.tcl` and SDC files.
2. Conditional loops are controlled by a Tcl variable named `budget_use_accumulated`. The default value of this Tcl variable is `0`, which specified that constraints will be processed considering the child partition in the .lib form. You need to run `set budget_use_accumulated 1` before loading the design for processing the constraints considering the child partition in the flat form.
3. This parameter does not impact the output of the `viewDefinition.tcl` and SDC files for the bottom-most partitions.

Default: `false`

`-boundaryConditionTemplatePath` *`directory_path`*

Specifies the directory path containing the model files to be used for generating template-based boundary condition through the `-useBoundaryCondition template` parameter. Hence, the `-boundaryConditionTemplatePath` parameter allows you to specify your own template file for generating boundary conditions.

The directory path specified with this parameter should have boundary model files for each view with the file name format as `view_name.model`. If the model files are missing for any view, an error is prompted. If a view is both `setup` and `hold`, the same model will be used.

Default: `""`

`-bufferDelayAdjustment` *value*	
	Specifies the delay for virtual buffer adjustment. If you specify `0`, there will be no adjustment. If this parameter is specified without a value, timing budgeting performs regular virtual buffering. *Default*: `0`
`-bufferDelayLibCell` *libname/cellname*	
	Specifies the buffer that should be used for virtual buffer adjustment. This is applicable in cases of regular virtual buffering, which can be overridden by the `-bufferDelayAdjustment` parameter.
`-bufferDelaySelectionEffort {low|high}`	
	Controls the virtual buffer selection effort for calculating the buffer delay adjustment. The two effort levels are: • `low`: use buffer selection algorithm based on drive strength • `high`: use buffer selection algorithm based on actual one buffer delay *Default*: `low`
`-ccd {true|false}`	
	Generates scripts for verification of time budgeting top and block constraints against pre-budget chip constraints using the Conformal Constraint Designer software. Default: `false`
`-constantModel {true|false}`	
	Specifies the type of timing model created. The `-constantModel` parameter specifies that constant timing models are created. *Default*: `true`
`-distributeMMMC {true|false}`	
	Controls the distribution of MMMC views for budgeting. The distribution could be on the same machine or different machines on a farm/lsf. *Default*: `false`
`-dontWriteWireLoad {true|false}`	
	Writes out the `set_load` command with the `-wire_load` information for the output pins in the generated SDC file, if the design is routed. *Default*: `false`
`-driveCell {true|false}`	
	Writes out the boundary drive information for input pins in the generated SDC file, using the `set_driving_cell` constraint. If not specified, the `set_drive` constraint is written. This is used to model transition on input port based on load attached to the drive cell. *Default*: `false`
`-fixTopLevelPaths {none | all | negativeOnly | positiveOnly}`	
	Fixes the top-level timing budget and proportions the remaining timing budget only for the partitions based on the specified value. Use one of the following parameter values: • `none`: Fixes the default proportionate budgeting. • `all`: Fixes budgeting only for the partitions. • `negativeOnly`: Fixes budgeting only for the partitions with negative slack paths. • `positiveOnly`: Fixes budgeting only for the partitions with positive slack paths. *Default*: `none`
`-handleComplexSDC {true|false}`	

	Specifies to use Common Timing Engine's `write_sdc` for correct push down in case of nested/complex SDC constraints. It can be used when there are regular expressions in SDC constraints. *Default:* `false`
`-honorPortBoundaryCondition {true|false}`	
	When set to `true`, `deriveTimingBudget` pushes down the boundary conditions from the chip IO ports to the block IO ports. This parameter works only in the cases where the block IO ports are directly connected to the chip IO ports (when there is no logic in between the chip and block, and there is no dependency on the abutted condition). When this parameter is set to its default value, `true`, following is the behavior for those partition ports, which are directly connected to the top level ports: • For the actual and optimized boundary condition mode, if the boundary conditions are specified in the chip SDC, they are written as is. Else, nothing is written. • For the template boundary condition mode, if the boundary conditions are specified in the chip SDC, they are written as is. Else, the existing template-based method is used for them. ⚠ If you turn off this parameter, there will be no change in the behavior of any partition port, whether or not it is directly connected. *Default:* `true`
`-honorPortDelays {true|false}`	
	When this parameter is set to `true`, `deriveTimingBudget` replicates the chip IO SID/SOD values to block IO, if these two are directly connected and abut each other. *Default:* `true`
`-honorReportTimingFormat {true|false}`	
	Honors the global variable set through `set_globalreport_timing_format` by the user in the justification reports, which are generated through the `deriveTimingBudget -justify` parameter. This parameter keeps justification reports and columns of `report_timing` in the same format as set by the user. **Note:** This parameter may impact the turnaround time of the `create_timing_budget -justify` parameter. *Default:* `false`
`-ignoreDontTouch {true|false}`	
	Specifies the handling of don't touch (fixed) objects . The parameter allows you to modify these objects also. Default: `false`
`-includeLatency {true|false}`	
	Specifies whether the clock latency is included in the `set_input_delay` and `set_output_delay` constraints. **Note:** The clock latency is always included if the design has propagated clock(s) irrespective of this parameter. `set_input_delay` and `set_output_delay` change and are based on actual arrival times. *Default*: true
`-inputLoad {true|false}`	
	Specifies whether the `set_load` constraints have to be written for input pins in the generated SDC file. With this option, load constraint on ports is specified. *Default*: `false`
`-includeWireLoadsInLib {true|false}`	
	Specifies to include the wire loads in library model pin capacitance to handle high interface net loads. It provides extra information in *.lib* file for more accurate modelling. *Default*: `false`

`-inputTransition {true|false}`	
	Writes out input transition time in the budgeting output files for input pins in the generated SDC file, using the `set_input_transition` constraint. If not specified, the `set_drive` constraint is written. You must enter the transition value directly. *Default*: `false`
`-justify {delay|exception|lib_arc|boundary_condition|all}`	
	Controls the type of justification and also decides whether or not to generate the justification reports. You may use one or more of the following values with this parameter: • `delay`: Budget justification files for all the partition IO ports are generated. • `exception`: Budget justification files for all the partition IO ports are generated with exception data. • `lib_arc`: Justification files for libs are generated. • `boundary_condition`: Justification files for the boundary conditions used in the constraint files are generated. • `all`: All the justification files are generated. **Note:** When this parameter is used without any value, no justification files are generated. Only the summary is generated. *Default*: `""`
`-justifyBudgetDir` *`directory_name`*	
	Directs output of budget justification to the specified directory and dumps the summary files. *Default:* `budget justify`
`-keepPinListsForBlockPorts {true|false}`	
	Specifies to preserve all fan-in and fan-out pin lists for block ports. The default value is `true`. When set to `true`, the runtime will be the fastest but involves high memory usage. The value of `false` means that the peak memory consumed will decrease but with an increase in runtime. You can use this parameter to control memory consumption and runtime for budgeting processes. It does not affect other functions when specified. Default:`true`
`-latencyOnClocks {true|false}`	
	Writes the `set_clock_latency` constraint on the clocks instead of ports in the generated SDC files. Use this parameter when you have multiple clocks on the same port. *Default:* Writes the `set_clock_latency` constraint on the ports. *Default*: `false`
`-localLatency {true|false}`	
	Adjusts the `set_clock_latency` constraint information that you specify using the `setPtnUserCnsFile` command to the constraints file generated after budgeting. When you specify this parameter, the software checks whether the `set_clock_latency` constraint specifies a clock or a port. If the constraint specifies a clock and the clock exists in the partition, then the constraint is added to the constraints file generated after budgeting. If the clock does not exist in the partition, the software does not write out the constraint. If the constraint specifies a port, the constraint is added to the constraints file generated after budgeting. The source latency is calculated as follows: source latency = top-level source latency + top-level network latency - local network latency *Default*: `false`
`-localUncertainty {true | false}`	

	Adds the `set_clock_uncertainty` constraint information that you specify using the `setPtnUserCnsFile` command to the constraints file generated after budgeting. When you use this parameter, the constraint file generated after budgeting has input and output delays using virtual clocks instead of real clocks in the SDC file. The software uses the virtual clocks to differentiate the clock uncertainty for external paths (paths that are coming from or going to the top-level) and internal paths of the partition.
`-makeNegativeInputDelayZero {true | false}`	
	Specifies to adjust negative set input delay to 0. *Default*: `false`
`-masterClone {masterOnly|merged|uniqueViewPerHInst}`	
	Specifies how the constraints are generated for master and clone designs. Following is the description of the parameter values: • `masterOnly`: Derives budgets only for the master partition/inst. • `merged`: Derives budgets based on the worst case data per pin for the master and all its clones. • `uniqueViewPerHInst`: For partitioned block, creates a corresponding view for each instance of the master and all its clones, where timing constraints are generated considering this instance timing separately. So total number of new views in block `viewDefination.tcl` is equal number of instances multiflied with the number of views. For partitioned TOPs .lib model, the "merged" approach is followed. So that there would be only one .lib per view for every partition. No additional views are created for partitioned TOP. *Default*: `masterOnly`
`-noFalsePathsForUnCstrPorts {true|false}`	
	Specifies not to create false paths for unconstrained ports. *Default*: `false`
`-noHoldView {true|false}`	
	Specifies to derive budgets only for setup views. Dummy files are created for the hold view as budgeting is not done for them. *Default*: `false`
`-noSetupView {true|false}`	
	Specifies to derive budgets only for hold views. Dummy files are created for the setup view as budgeting is not done for them. *Default:* `false`
`-overrideNetCap` *value*	
	Specifies the lump capacitance value. The software uses this value when you use the `-preliminary` parameter in the `deriveTimingBudget` command. *Default:* `100ff`
`-reportOrModifyBudget {true|false}`	
	Specifies to enable the collection of data required to use the `modifyBudget` command. You must specify this parameter before budgeting to run `modifyBudget` later in the flow. If this parameter is not specified, the `modifyBudget` command does not produce any results. *Default:* `false`
`-reset`	Resets parameters to their default values. The `-reset` parameter must be the first parameter specified. If you specify `-reset` by itself, the software resets all `setBudgetingMode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset. will be reported.
`-rptNegSlackOnPorts` *value*	
	Reports if partition ports have a slack less than the specified value and saves this information in the .warn file. The slack is calculated based on data arrival time and clock time at data sampling points. The report is saved to the .warn file.
`-snapFdBudgetTo` *value*	

Specifies the minimum delay value (in picoseconds) for the path from partition input port to partition output port. If, during timing analysis, the path segment delay used to generate budgeting constraints for the partition ports falls below the specified threshold value, the delay segment is snapped to the specified value and is considered as a fixed delay during budget allocation for the path.

The `.warn` file does not issue a warning for feedthrough segments. By default, the software distributes a delay equivalent of two medium-sized buffer delays among the feedthrough segments. As a result, it is unlikely that the feedthrough segment delay will fall below a reasonable value.

`-snapInputBudgetTo` *`value`*

Specifies the minimum delay (in picoseconds) for the path from the partition input port to the internal register. If, during timing analysis, the path segment delay falls below the specified threshold value, the delay segment is snapped to the specified value and is considered as a fixed delay during budget allocation for the path.

Note: During timing analysis, the path segment delay could be greater then the specified value; however, after budget allocations, the allocated delay for the partition can still become smaller than the specified threshold value. The software checks the delays allocated to partitions for input and output paths and creates appropriate warning messages in the partition `.warn` file.

`-snapNegativeOnly {true | false}`

Considers only the negative slack paths when used with `-snapInputBudgetTo` and `-snapOutputBudgetTo`.

Default: `false`

`-snapOutputBudgetTo` *`value`*

Specifies the min delay (in picoseconds) for the path from the internal register to the partition output port. If, during timing analysis, the path segment falls below the specified threshold value, then the delay segment is snapped to the specified value and is considered as a fixed delay during budget allocation for the path.

`-stageBasedFanoutDrivingFactor` *`value`*

Specifies the fanout count of the pin, according to which the buffer stages are added in the timing graph. This parameter provides the basis for the software to calculate the number of fanouts and buffer stages to be virtually added for each stage according to the following table:

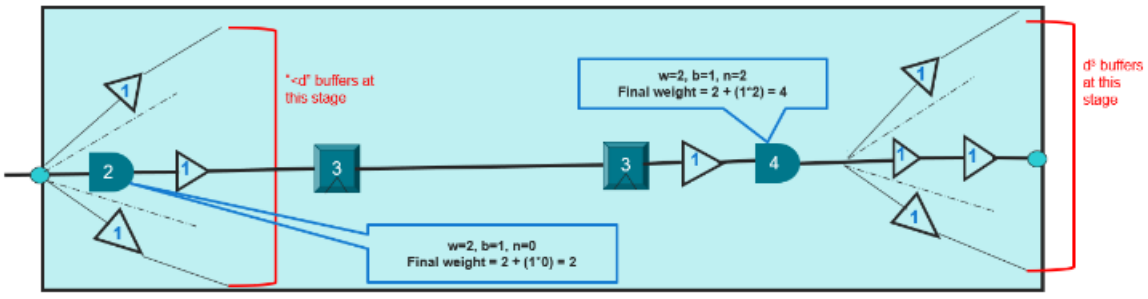
Fanout Range	**Number of Buffer Stafe Added (n):**
1 to d1 to d	0
(d+1) to d2	1
(d2+1) to d3	2
(d3+1) to d4	3
..	..

If the weight of the original cell is w, weight of the buffer is b, and the number of buffer stages is n, then:

Final weight of cell = w + (b*n)

See the example below for understanding the functioning of the `-stageBasedFanoutDrivingFactor` parameter.

```
setBudgetingMode -stageBasedWeight {3 2 1}
```



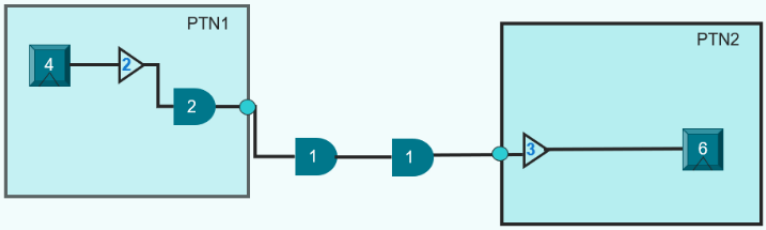
Default: `-1`

`-stageBasedPartitionMultiplier {{HInst1 m1} {HInst2 m2}..}`

Specifies the weight multiplication factor for the partition hinsts. Those hInsts, which are not explicitly mentioned, will have the default weights. If the weights of hinsts have been specified using the `setBudgetingMode -stageBasedWeight` parameter, those are used with their default weights. Else, the default weights (1 for sequential cell, 1 for combinational cell, and 0 for buffer/inverter) are used. In case of nested partitions, if the child partition multiplier is not specified explicitly, then it will have its parent's multiplier. If parent and child partitions have separate multipliers, then they will be honored.

See the example below for understanding the usage of this parameter:

```
setBudgetingMode -stageBasedWeight {2 1 1}
setBudgetingMode -stageBasedPartitionMultiplier {{PTN1 2} {PTN2 3}}
Cycle time = T
Budget for PTN1= 8/19 T
Budget for PTN2= 9/19 T
Budget for top = 2/19 T
```



Default: ""

`-stageBasedWeight {S | C | B}`

Specifies default weights of the sequential, combinational, and buffer/inverter cells for the stage-based timing budgeting. Following are the default weights of these cells:

- Sequential: 1
- Combinational: 1
- Buffer/inverter: 0

Default: `{1 1 0}`

`-topLevel` *`minimumValue`*

Specifies the minimum percentage of total available budget set aside for the top level. This parameter can take any value between 0 - 1.

- `0` - indicates no slack to top (100% to block).
- `1` - maximum value that indicates full slack to top (0 % block)

Default: `-1` (indicates regular slack or reset of the top level)

`-topLevelDelayPerLen` *`value`*

Specifies the top-level estimated delay (in picoseconds) per millimeter length. The software uses this value when you use the `-preliminary` parameter in the `deriveTimingBudget` command.

For more information on the use of the `-topLevelDelayPerLen` parameter, see "Deriving Preliminary Budgets in Early Design Phase" in the Timing Budgeting chapter of the *Innovus User Guide*.

Default: `180ps/mm`

`-topLevelMinDelayPerNet` *`value`*

Specifies the top-level minimum delay-per-net value in picoseconds/millimeter units. The software uses this value when you use the `-preliminary` parameter in the `deriveTimingBudget` command.

For more information on the use of the `-topLevelMinDelayPerNet` parameter, see "Deriving Preliminary Budgets in Early Design Phase" in the "Timing Budgeting" chapter of the *Innovus User Guide*.

Default: `100ps/mm`

`-useBoundaryCondition {optimized|template|actual|empty}`

Specifies the criteria for choosing boundary conditions. Following are the usages of the parameter values:

- `optimized`: The values of boundary conditions are fetched from the engine used to build timing graph. If the engine does not provide any values, than the template is used automatically and a warning is displayed indicating that no optimized boundary conditions are available, and the template is used automatically. For example, `-useBoundaryCondition {optimized}` for `-virtulaOptEngine {proto | none}` will display such a warning.
- `template`: The values of boundary conditions are derived automatically and used through the default template.
  Note: The default templates are saved as `budget_model/`*`view_name`*`.model`.
- `actual`: Uses the actual netlist to build the timing graph.
- `empty`: No boundary condition is dumped.

Default: `template`

`-useRealCellForTimingModel {true|false}`

Use actual gate connected to the partition port to write out the model timing arc, else medium size buffer is used.

Default: `false`

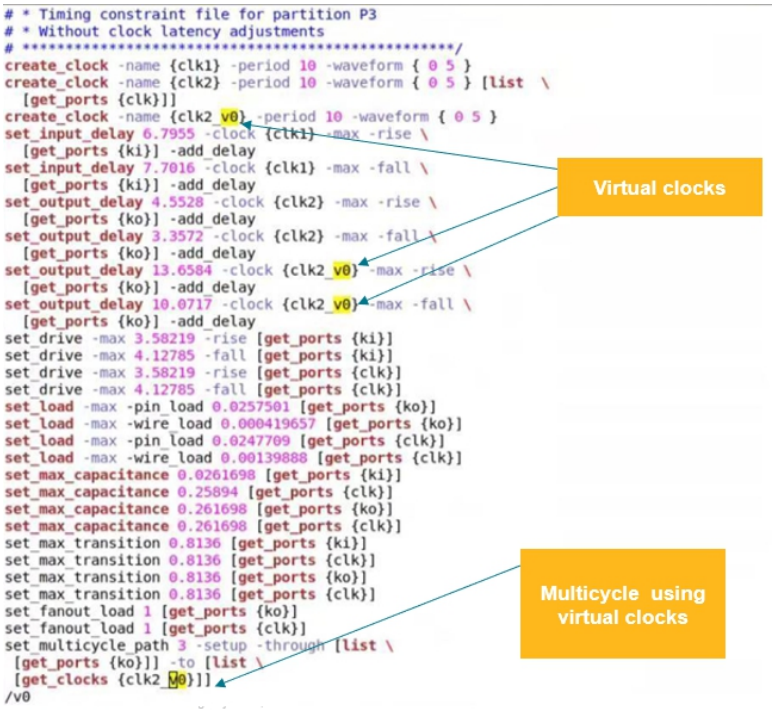
`-virtualClock {true|false}`

When set to `false`, this parameter prevents creation of new virtual clocks during timing budgeting. The software takes the worst path into account and provides the minimum required time amongst all the paths passing through a port. Since there is only a single critical path, no virtual clock is created.

While using this parameter:

- For an endpoint with both single-cycle and multicycle constraints, multicycle constraints are not defined.
- For the cases where both setup and hold views are same, virtual clocks are still created with names appended with `_v0_setuphold`.
- At the partition level, a path which is constrained before budgeting, is not unconstrained after budgeting.
- If a path has both single and multi-cycle constraints, single cycle constraint is retained, and hence, the path is optimized accordingly and timing closure is not affected.

Following is an example of `-virtualClock true`:



Following is an example of `-virtualClock false`:

```
#/*************************************************
# * Timing constraint file for partition P3
# * Without clock latency adjustments
# *************************************************/
create_clock -name {clk1} -period 10 -waveform { 0 5 }
create_clock -name {clk2} -period 10 -waveform { 0 5 } [list  \
  [get_ports {clk}]]
set_input_delay 6.7955 -clock {clk1} -max -rise \
  [get_ports {ki}] -add_delay
set_input_delay 7.7016 -clock {clk1} -max -fall \
  [get_ports {ki}] -add_delay
set_output_delay 4.5528 -clock {clk2} -max -rise \
  [get_ports {ko}] -add_delay
set_output_delay 3.3572 -clock {clk2} -max -fall \
  [get_ports {ko}] -add_delay
set_drive -max 3.58219 -rise [get_ports {ki}]
set_drive -max 4.12785 -fall [get_ports {ki}]
set_drive -max 3.58219 -rise [get_ports {clk}]
set_drive -max 4.12785 -fall [get_ports {clk}]
set_load -max -pin_load 0.0257501 [get_ports {ko}]
set_load -max -wire_load 0.000419657 [get_ports {ko}]
set_load -max -pin_load 0.0247709 [get_ports {clk}]
set_load -max -wire_load 0.00139888 [get_ports {clk}]
set_max_capacitance 0.0261698 [get_ports {ki}]
set_max_capacitance 0.25894 [get_ports {clk}]
set_max_capacitance 0.261698 [get_ports {ko}]
set_max_capacitance 0.261698 [get_ports {clk}]
set_max_transition 0.8136 [get_ports {ki}]
set_max_transition 0.8136 [get_ports {clk}]
set_max_transition 0.8136 [get_ports {ko}]
set_max_transition 0.8136 [get_ports {clk}]
set_fanout_load 1 [get_ports {ko}]
set_fanout_load 1 [get_ports {clk}]
```

No virtual clocks or multicycle defined using them.

Default: `true`

`-virtualOptEngine {none|earlyTimingEngine|proto}`

Specifies the virtual optimization engine to be used. The values of this parameter include:

`none`
Specifies that the timing graph is generated through `report_timing`.

`earlyTimingEngine`
Specifies that the timing graph is generated by `optVirtual -earlyTimingEngine`.

`proto`
Specifies that the timing graph is generated by prototype timing.

Default: `none`

Note: User can use ART (`createActiveLogicView -type timing_budget`) before generating the timing graph to reduce TAT and memory during budgeting flow.

`-writeConstraintsForClkOutputPorts {true|false}`

Specifies to write extra constraints on the output port of block SDC.
When this parameter is specified, the following extra constraints are written out:

- `set_clock_sense` - this constraint is dumped if the clock is used as data going forward after partition
- `create_generated_clock` - this constraint is dumped if the clock is used as clock going forward after partition

If the clock is used as both data and clock going forward, the parameter does not write out these extra constraints.

Default: `false`

`-writeFPForHold {true|false}`

	Writes `false_path -hold` constraints for all I/Os when deriving timing budgets in setup mode.
	For inputs, the `saveTimingBudget` command writes the following constraint:
	`set_false_path -from port -hold`
	For outputs, the `saveTimingBudget` command writes the following constraint:
	`set_false_path -to port -hold`
	Default: `false`

`-writeLatencyPerClock {true|false}`	
	Specifies to write latency per clock instead of per port. When you set this variable to true, `set_clock_latency` is written per clock if multiple clocks enter the partition through the same port. *Default*: `false`

Examples

- The following set of commands load database for timing budgeting:

```
setBudgetingMode -stageBasedWeight {3 2 1}
setBudgetingMode -stageBasedPartitionMultiplier {{PTN1 2} {PTN2 3}}
setBudgetingMode -stageBasedFanoutDrivingFactor 8
deriveTimingBudget -stageBased
```

- The following command sets the input and output budgets to 3000:

```
setBudgetingMode -snapInputBudget 3000 -snapOutputBudget 3000
The resulting message appear in the .warn file when you generate budgets.
/* Start Section: Pin constraint values less than defined threshold time
(setup)*/
Input Threshold Value: 3.000
Output Threshold Value: 3.000
Constraint: set_output_delay 11.524 -clock {CLK2} -max -rise [get_ports {sub1_out1}]
allocates block delay less than input threshold time.
Block delay: 0.476
Constraint: set_output_delay 11.524 -clock {CLK2} -max -fall [get_ports {sub1_out1}]
allocates block delay less than input threshold time.
Block delay: 0.476
/* End Section: Pin constraint values less than defined threshold time (setup)*/
justifyBudget header example:
Budgeted constraint type: set_output_delay(setup rise)
Initial budget available time = 12.000

One Buffer Delay for Adjustment(cell ssiv): 1.000
Fixed Delay for Feedthrough Paths(fixFdThru)= (2 x 1.000) = 2.000
External Segment Fixed Delay From Budget Snap(snapExtFixedDel) = 5.000
Total External Segment Fixed Delay(extFixDel) = 5.000
This Block's Segment Fixed Delay from budget snap(snapIntFixedDel) = 1.000
Total This Block's Segment Fixed Delay(intFixDel) = 1.000
External Segment Extra Delay From Budget Snap (snapExtDelExtra) = 2.000
This Block's Extra Delay From Budget Snap (snapIntDelExtra) = 1.000
Path Extra Delay From Budget Snap (snapExtraDel) = (2.000 + 1.000) = 3.000
Fixed Delay on the Path(pathFixDel) = (5.000 + 1.000 + 1.000 + 2.000) = 9.000
Fixed Delay Adjustment(fixDel)= 9.000
Clock Skew(clkSkew): 0.000
Adjustment for budget available time= -(pathFixDel + fixFdThru - clkSkew + snapExtraDel)
= -(6.000 + 2.000 - 0.000 + 3.000)
= -11.000
Available budget after adjustments(AvailTime)= (12.000 - 11.000) = 1.000

External Segment Delay(extSegDel): 8.000
This Block's Segment Delay(segDel): 0.000
Total delay(totDel): 8.000 + 0.000 = 8.000
Virtual Clock Adjustment(virClk): 0.000
Budgeted constraint = AvailTime * extSegDel / totDel + fixDel + virClk + startClkLat
Budgeted constraint = 1.000 * 8.000 / 8.000 + 9.000 + 0.000 + 0.000 = 10.000
```

Related Information

- The following sections in the Timing Budgeting chapter of the *User Guide* document:
  - "Deriving Timing Budgets"
    - "Top-Level Budgets Derived by Using Active Logic View"

    - "Deriving Preliminary Budgets in Early Design Phase"
- "Budgeting Output Files for MMMC Designs"
- "Generated Report for Design Example"
- "Verifying Timing Budgets"
- "Reading the Justify Budget Report"

setCycleBudgetRatio

```
setCycleBudgetRatio
[-help]
{[[[{-fromTop | -fromPtn {hinst ratio}} {-toTop | -toPtn {hinst ratio}}] ] [-clock string]] | [-reset ]}
```

Sets the budgeting constraint based on the clock cycle, which ensures that budgeting is done in minimum time and memory by deriving partition block delays from the total clock cycle time. This command supports:

- Different top-level ratios based on critical partition to partition connections
- Different ratios between the partitions with different sources and sinks
- Clock-based segment control

Note: This command (cycle-based budget) does not support nested partition.

Parameters

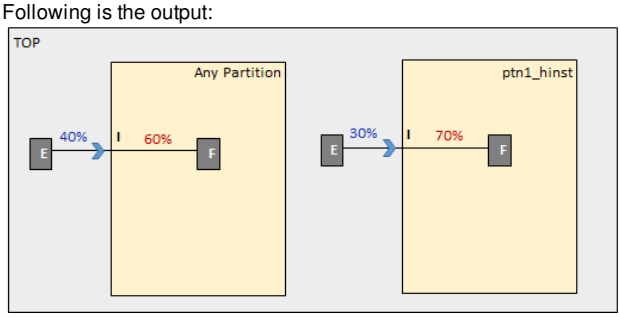
`-help`	Outputs a brief description that includes the type and default information for each `setCycleBudgetRatio` parameter. For a detailed description of the command and all its parameters, use the man command: `man setCycleBudgetRatio`
`-clock string`	Presents the clock name for which the ratio is to be applied. If specified, the cycle time ratio is applied to the paths only from the specific clock. If not specified, all clocks path are impacted with same percentage of cycle time.
`-fromPtn {hinst ratio}`	Specifies that the start of a timing path segment is inside the mentioned hierarchical instance, which should get the mentioned cycle ratio for budgeting.
`-fromTop`	Specifies that the start of a timing path segment is at the top level.
`-reset`	Resets all the previously declared ratio budgets/contraints.
`-toPtn {hinst ratio}`	Specifies that the end of a timing path segment is inside the mentioned hierarchical instance, which should get the mentioned cycle ratio for budgeting.
`-toTop`	Specifies that the end of a timing path segment is at the top level.

Examples

- The following commands present an example of the TOP to PTN path:

  ```
  setCycleBudgetRatio -fromTop -toPtn {* 0.6}
  setCycleBudgetRatio -fromTop -toPtn {ptn1_hinst 0.7}
  ```

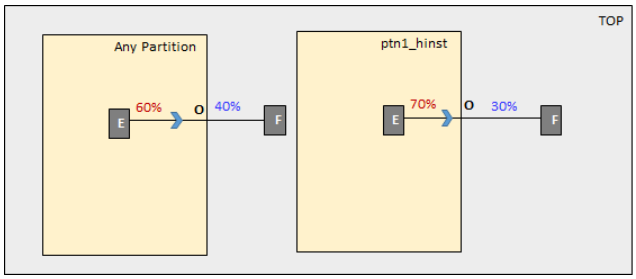
  Note: * matches with any partition hinst. However, it does not matches with TOP.
  Following is the output:

- The following commands present an example of the PTN to TOP path:

  ```
  setCycleBudgetRatio -fromPtn {* 0.6} -toTop
  setCycleBudgetRatio -fromPtn {ptn1_hinst 0.7} -toTop
  ```

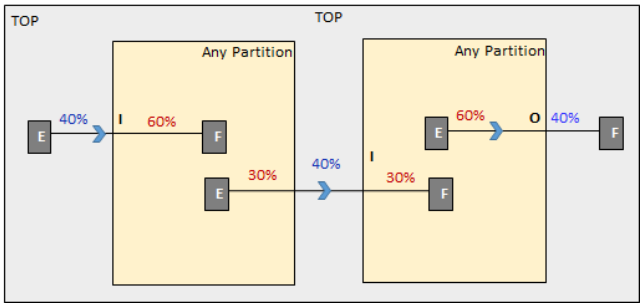
  Note: * matches with any partition hinst. However, it does not matches with TOP.
  Following is the output:



- The following commands present an example of reserving TOP percentage:

```
setCycleBudgetRatio -fromPtn {* 0.6} -toTop
setCycleBudgetRatio -fromTop -toPtn {* 0.6}
setCycleBudgetRatio -fromPtn {* 0.3} -toPtn {* 0.3}
```

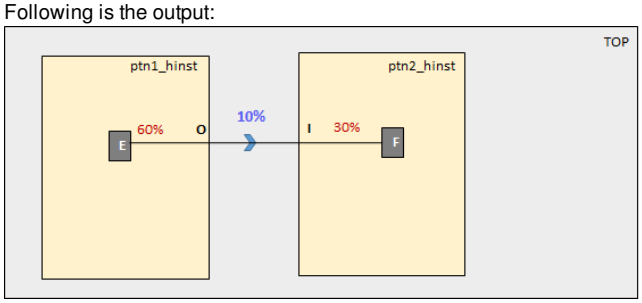
Note: * matches with any partition hinst. However, it does not matches with TOP.
Following is the output:



- The following command presents an example of the PTN to PTN path:

```
setCycleBudgetRatio -fromPtn {ptn1_hinst 0.6} -toPtn {ptn2_hinst 0.3}
```

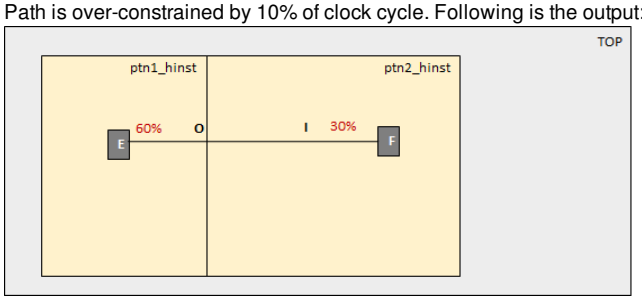
Following is the output:



- The following command presents an example of the PTN to PTN Abutted path:

```
setCycleBudgetRatio -fromPtn {ptn1_hinst 0.6} -toPtn {ptn2_hinst 0.3}
```

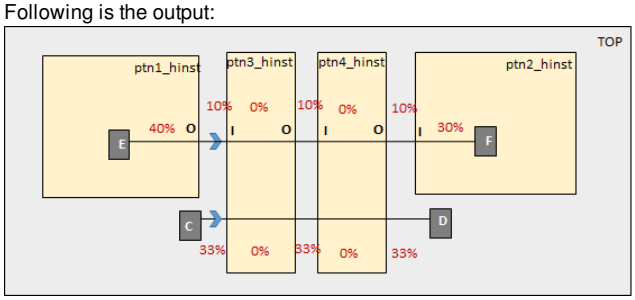
Path is over-constrained by 10% of clock cycle. Following is the output:



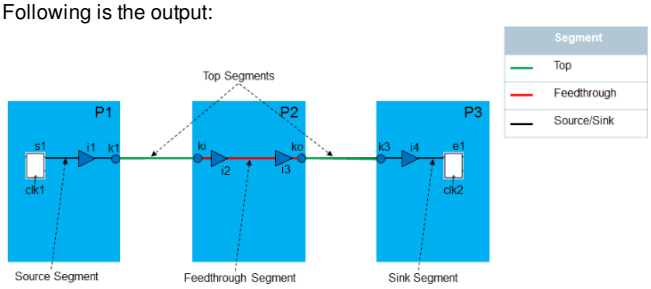
- The following command presents an example of how the top level budget is split into top and FT segments of the paths:

```
setCycleBudgetRatio -fromPtn {ptn1_hinst 0.4} -toPtn {ptn2_hinst 0.3}
```

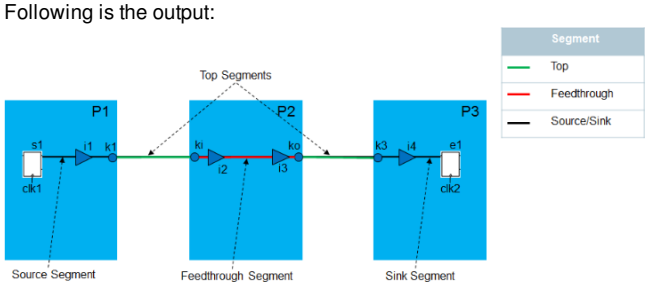
Note: FT segment is over-constrined and given 0% budget.
Following is the output:



- Following are two examples of actual delay-based cycle ratio:
  Example 1:
  Suppose `PhaseShift = 10ns`
  `setCycleBudgetRatio -fromPtn {P1 0.4} -toPtn {P3 0.4}`
  `setFixedBudget -through {P2} -slack 0 ;#` implies that the budget of `P2` is “as is”
  Then,
  `P2` budget = `1ns` (assuming that the actual delay through `P2` is `1ns`)
  `AvailableTime = 10ns - 1ns = 9ns`
  `P1 budget = 0.4 * AvailableTime = 0.4 * 9ns = 3.6ns`
  `P3 budget = 0.4 * AvailableTime = 0.4 * 9ns = 3.6ns`
  `TOP budget = 9ns - 3.6ns - 3.6ns = 1.8ns`
  So each of the two TOP segments get `0.9ns`.
  Following is the output:



  Example 2:
  Suppose `PhaseShift = 10ns`
  `setCycleBudgetRatio -fromPtn {P1 0.5} -toPtn {P3 0.5}`
  `setFixedBudget -through {P2} -slack 0 ;#` implies budget of `P2` is “as is”
  Then,
  `P2 budget = 1ns` (assuming that the actual delay through `P2` is `1ns`)
  `AvailableTime = 10ns - 1ns = 9ns`
  `P1 budget = 0.5 * AvailableTime = 0.5 * 9ns = 4.5ns`
  `P3 budget = 0.5 * AvailableTime = 0.5 * 9ns = 4.5ns`
  `TOP budget = 9ns - 4.5ns - 4.5ns = 0ns`
  So each of the two TOP segments get `0ns`.
  Following is the output:



Related Information

The following sections in the Timing Budgeting chapter of the *User Guide* document:

- "Deriving Timing Budgets"
- "Budgeting Using Text Commands"
- "Top-Level Budgets Derived by Using Active Logic View"
- "Deriving Preliminary Budgets in Early Design Phase"
- "Budgeting Output Files for MMMC Designs"
- "Analyzing Timing Budgets"
- "Resolving Conflicts with Path-Based Exceptions"
- "Customizing Budget Generation"
- "Verifying Timing Budgets"
- "Reading the Justify Budget Report"
- "Warning Report"

setFixedBudget

```
  setFixedBudget
[-help]
{[{[{-from {list_of_pins} | -fromFlopsOf {hinst}} {-to {list_of_pins} | -toFlopsOf {hinst}}] | [-through {list_of_hinst}]} {-slack | -delay
} [-add ] <value>] | [-reset ]}
```

This command fixes the budget for a particular path segment between partitions and start/end points and helps in reducing the distribution budget delay in the path segments.

Note: Do not use this command with cycle-based timing budgeting.

This command is valid only for the following segments:

- Chip In -> Partition In
- Start Point -> Partition In
- Partition In -> Partition Out
- Partition In -> End Point (FF D Pin) inside same partition
- Start Point (FF Q Pin) inside same partition -> Partition Out
- Partition Out -> Chip Out
- Partition Out -> End Point

Parameters

`-add`	Adds the slack/delay to previous value (Boolean, Optional)
`-help`	Prints out the command usage
`<value>`	Presents delay or slack values (Float)
`-delay`	Specifies the delay value (Boolean, Optional)
`-from {list_of_pins}`	Presents the 'from pins' to be analyzed (String)
`-fromFlopsOf {hinst}`	Specifies the from hierarchical inst for which start pins to be analyzed (String)
`-reset`	Resets all previously declared fixed budgets (Boolean, Optional)
`-slack`	Specifies the slack value (Boolean, Optional)
`-through {list_of_hinst}`	Specifies the hierarchical instances to be analyzed (String)
`-to {list_of_pins}`	Specifies the 'to pins' to be analyzed (String)
`-toFlopsOf {hinst}`	Specifies the to hierarchical inst for which end pins to be analyzed (String)

Examples

The following command fixes the budget of the hierarchical instance feed-through paths to the existing delay:

```
setFixedBudget -through P5 -slack 0
```

The following command gives a fixed clack of 1 to the paths from port 'in' to hierarchical instance input pin 'p4/in'; budgets will distribute rest of the slack to other segments of the paths:

```
setFixedBudget -from in -to P4/in -slack 1
```

The following command fixes the delay of this segment to 0.5 from the hierarchical instance input pin 'p4/in' to the endpoint 'P4/FF1/D'; budgets will be calculated for the whole path considering the path delay of this segment as 0.5:

```
setFixedBudget -from P4/in -to P4/FF1/D -delay 0.5
```

Related Information

The Timing Budgeting chapter of the *User Guide* document.

setPtnUserCnsFile

```
setPtnUserCnsFile
-fileName name
-ptnName partitionName
[-view]
```

Specifies the constraints file for a specific partition. The software appends the constraint information in this file to the timing constraints generated when you use the `savePartition` or `saveTimingBudget` command. You can specify the following constraints in the constraints file for the partition:

- `set_clock_uncertainty`
- `set_clock_latency` (network latency constraints)

You must specify the `-localLatency` and `-localUncertainty` parameters in the `setBudgetingMode` command before using the `setPtnUserCnsFile` command.

Parameters

`-fileName` *name*	Specifies the name of the file containing constraints specific to a partition.
`-ptnName` *partitionName*	Specifies the name of the existing partition.
`-view`	Specifies a partition-specific constraint file for each MMMC view.

Related Information

The Timing Budgeting chapter of the *User Guide* document.

39

Timing Constraint Commands

- create_clock
- create_generated_clock
- current_design
- current_instance
- get_sdc_mode
- group_path
- reset_annotated_check
- reset_annotated_delay
- reset_annotated_transition
- reset_aocv_stage_weight
- reset_case_analysis
- reset_clock
- reset_clock_gating_check
- reset_clock_groups
- reset_clock_latency
- reset_clock_sense
- reset_clock_transition
- reset_clock_tree_latency
- reset_clock_uncertainty
- reset_data_check
- reset_disable_clock_gating_check
- reset_disable_timing
- reset_drive
- reset_driving_cell
- reset_generated_clock
- reset_ideal_latency
- reset_ideal_network

- reset_ideal_transition
- reset_input_delay
- reset_load
- reset_max_capacitance
- reset_max_fanout
- reset_max_time_borrow
- reset_max_transition
- reset_min_capacitance
- reset_min_fanout
- reset_min_pulse_width
- reset_min_transition
- reset_mode
- reset_output_delay
- reset_path_exception
- reset_path_group
- reset_pll_timing
- reset_propagated_clock
- reset_property
- reset_pulse_clock_max_transition
- reset_pulse_clock_max_width
- reset_pulse_clock_min_transition
- reset_pulse_clock_min_width
- reset_resistance
- set_active_clocks
- set_annotated_check
- set_annotated_delay
- set_annotated_transition
- set_aocv_interface_path_offset
- set_aocv_stage_weight
- set_case_analysis
- set_clock_exclusivity
- set_clock_gating_check

- set_clock_groups
- set_clock_latency
- set_clock_sense
- set_clock_transition
- set_clock_uncertainty
- set_data_check
- set_disable_clock_gating_check
- set_disable_timing
- set_dont_touch
- set_dont_touch_network
- set_dont_use
- set_drive
- set_driving_cell
- set_false_path
- set_fanout_load
- set_ideal_latency
- set_ideal_network
- set_ideal_transition
- set_input_delay
- set_input_transition
- set_load
- set_logic_one
- set_logic_zero
- set_max_capacitance
- set_max_delay
- set_max_fanout
- set_max_time_borrow
- set_max_transition
- set_min_capacitance
- set_min_delay
- set_min_fanout
- set_min_pulse_width

- set_min_transition
- set_mode
- set_multicycle_path
- set_output_delay
- set_pll_timing
- set_propagated_clock
- set_property
- set_pulse_clock_max_transition
- set_pulse_clock_max_width
- set_pulse_clock_min_transition
- set_pulse_clock_min_width
- set_resistance
- set_sense
- reset_sense

create_clock

```
create_clock
-period period_value
[-name clock_name]
[-waveform edge_list]
[-add]
[-comment string]
[sources]
```

Creates a clock object and defines its waveform in the current design. If you do not specify any sources, but you specify the `-name` *clock_name* argument, a virtual clock is created. The new clock has an ideal clock latency, and the software does not assume any propagation delay through the clock network.

The clock that you create using the `create_clock` command cannot have multiple pulses per period. You can specify multiple constraints on the same pin using the `-add` parameter.

If one of the sources is already the source of a clock, the source is removed from that clock. If you need to specify multiple clocks on the same source for simultaneous analysis, you must specify the `-add` parameter, to add this clock to the existing clock.

For example, if you specify the following command sequence, the `PH2` clock waveform overwrites the `PH1` waveform.:

```
create_clock -name PH1 ... [get_ports sysclk]

create_clock -name PH2 ... [get_ports sysclk]
```

To have both clocks applied, you must specify the following command sequence:

```
create_clock -name PH1 ... [get_ports sysclk]

create_clock -name PH2 -add ... [get_ports sysclk]
```

You can create a collection of clocks, which is a group of objects referenced by a string identifier using the get_clocks command.

Parameters

`-add`	Specifies multiple clocks on the same source for simultaneous analysis with different clock waveforms. You must specify the `-name` parameter with this parameter.
`-comment` *`string`*	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.
`-name` *`clock_name`*	Specifies the string name of the ideal clock being created. If you do not specify this option, the clock gets the same name as the first clock source that you specify using the `-sources` option. If you do not specify any sources, you must specify the `-name` *`clock_name`* option, which creates a virtual clock not associated with a port or pin.
`-period` *`period_value`*	
	Specifies the value of the ideal clock in library time units.
`sources`	Specifies the list of pins or ports associated with an ideal clock waveform of the clock. If you do not specify this option, you must specify the `-name` *`clock_name`* option, which creates a virtual clock not associated with a port or pin. If you specify a clock on a pin that already has a clock, the new clock replaces the old one.
`-waveform` *`edge_list`*	
	Specifies the rise and fall edge times of the clock waveforms in library time units over an entire clock period. The first value in the edge list is the time at which the first transition on the clock occurs, which is the first rising transition after time zero, and edges are monotonically increasing. The numbers should represent one full clock period. If you do not specify the `-waveform` *`edge_list`* option, a default waveform is assumed, and the leading edge is placed at `0` and the trailing edge is placed at the midpoint of the period, such that a symmetric clock is generated.

Examples

- The following command creates an `ideal_clk1` clock on port `clkA` with a period of `5`, a rise at `0`, and a fall at `2.5`:
  create_clock [get_ports {clkA}] -name ideal_clk1 -period 5 -waveform {0 2.5}

- The following command creates an `ideal_clk2` clock on port `clkB` with a period of `10`, a rise at `1`, and a fall at `9`:
  create_clock [get_ports {clkB}] -name ideal_clk2 -period 10 -waveform {1 9}

Related Information

- `all_clocks`
- `get_clocks`
- `get_pins`
- `get_ports`
- `set_clock_latency`
- `report_clocks`
- `set_clock_uncertainty`
- `set_input_delay`
- `set_output_delay`

create_generated_clock

```
create_generated_clock
[-help]
target_pin_list
[-add]
[-comment string]
[-duty_cycle percent]
[-edge_shift edge_shift_list]
[-invert]
[-master_clock source_clock_name]
[-name <clock_name>]
-source source_pin
{[-multiply_by factor]
[-divide_by factor]
[-edges edge_list]
[-combinational]}
```

Creates a new clock signal from the clock waveform of a given pin in the design, and binds it with the pins or hierarchical pins in the *target_pin_list* argument. Whenever the source clock changes, the derived clock(s) change automatically.

You can generate a new clock waveform using one of the three ways:

- Multiply the frequency of the source clock
- Divide the frequency of the source clock
- Select the edges of the source clock to be mapped to the edges of the new clock

Given below are the SDC semantics to be used when the `create_generated_clock` command is defined:

- The edges are interpreted as alternating rise and falling edges.
- Each edge must be greater than and equal to its previous edge.
- Less than 3 edges are not allowed.
- Legal edge numbers start at “1”, edge 0 is undefined.
- Legal assertion of period 0 will be -edges {1 1 1 }.

If incorrect clock definitions are specified, an error will occur and the clock will not be generated.

When the `create_generated_clock` command is specified without the `-master_clock` parameter, and the software finds multiple clocks reaching the specified source pin, then the regular clocks are picked ahead of the generated clocks. In case if multiple regular clocks are present, the master clock is picked based on alphabetical sorting order of the clock names.

Note: A `create_generated_clock` assertion on a pin overrides any existing `create_generated_clock` assertion if both the constraints have the same source pin. In the following command sequence, the second assertion overrides the first:

```
create_generated_clock -name GCK1 -source CK -divide_by 2 U_FD1/Q
create_generated_clock -name GCK2 -source CK -multiply_by 2 U_FD1/Q
```

To have both assertions applied, you must specify the `-add` parameter. For example:

```
create_generated_clock -name GCK1 -source CK -divide_by 2 U_FD1/Q
create_generated_clock -name GCK2 -add -master_clock CLK -source CK -multiply_by 2 U_FD1/Q
```

Parameters

`-help`	Prints out the command usage.
`-add`	Models multiple generated clocks on the same source when multiple clocks must fan into the source pin. Ideally, one generated clock must be specified for each clock that fans into the master pin. Specify this parameter with the `-name` and `-master_clock` parameters. You can use the `-add` parameter to specify a different clock name for each generated clock, when used with the `-master_clock` option. Subsequently, you can use this clock name for setting other constraints, such as, `set_false_path` or `set_input_delay`.
`-combinational`	Restricts the path search for latency to only paths which do not cross sequential devices, or the source pins of other generated clocks. When you specify `-combinational`, if there are both sequential and combinational latency paths, the software only considers the combinational paths for the generated clock. If there are no combinational latency paths for the generated clock, the generated clock does not inherit latency from its parent clock hierarchy. By default, when calculating the latency of a generated clock, the software searches for the longest (and shortest) paths from the parent waveform to the generated clock point. All paths are considered, including those which traverse sequential elements such as flip-flops, registers, and latches. The default behavior is intended for cases where there is a cascade of generated clocks, and some of the clocks are not explicitly defined. The `-combinational` option can be used in conjunction with the `-multiply_by`, `-divide_by`, or `-edges` options.
`-comment` *string*	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.
`-divide_by` *integer*	Determines the frequency of the new clock by dividing the frequency of the source clock by *dfactor*. For example, if *dfactor=5*, the new frequency is 1/5 of the source frequency, and the new clock period is 5 times the source clock period.
`-duty_cycle` *percent*	Sets the duty cycle (high pulse width or clock period). The number (*percent*) is between `0` and `100`. For example, if *percent=50*, the high pulse width of the new clock waveform is the same as its low pulse width. **Note:** The `-duty_cycle` option can only be used with the `-multiply_by` option, otherwise it will be ignored with a warning.
`-edges` *edge_list*	Selects a list of edges from the source clock that form the edges of the derived clock. See the frequency division example below.
`-edge_shift` *edge_shift_list*	Specifies the amount of shift for each edge in the *edge_list* option. The amount of shift will be in library time units.
`-invert`	Inverts the source clock. Use with the `-source` option and the `-multiply_by` or `-divide_by` options.

`-master_clock` *`source_clock_name`*	Generates a target clock from the specified clock name. If you specify the `-master_clock` option and multiple signals arrive at the source pin, the clock specified by this option is used to generate the target clock. Use the `-add` parameter along with`-master_clock` parameter to add a clock to the existing clock when you want to specify multiple clocks on the same source for simultaneous analysis.
`-multiply_by` *`integer`*	Determines the frequency of the new clock by multiplying the frequency of the source clock by *`mfactor`*. For example, if *`mfactor=4`*, the new frequency is 4 times the source frequency, and the new clock period is 1/4 of the source clock period.
`-name` *`clock_name`*	Specifies the name for the generated clock. *Default*: The system creates a name for you. Always specify a name for the generated clock. Some commands issue an error if you use the system-generated name.
`-source` *`source_pin`*	Specifies the name of the source pin from which the generated clock is to be derived - with respect to the master clock containing the master clock as well as the generated clock definition.
`target_pin_list`	Specifies the list of pins where the generated clock assertion is applied. The clock signal propagating downstream from each of these pins becomes associated with the generated clock.

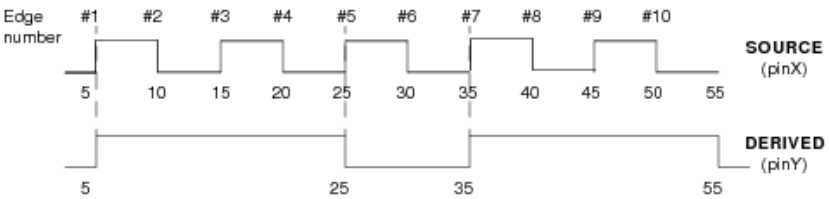
Examples

- The following command generates the derived clock waveform, as shown in the figure below:
  ```
  create_generated_clock -name mult_3 -source pinX -multiply_by 3 -duty_cycle 50 pinY
  ```

Figure: **Derived Clock (Frequency Multiplication)**

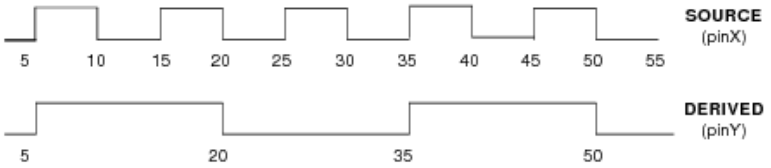
- The following command shows one way to create a divide-by-3 clock by using edge numbers of the source waveform to specify the new waveform, as shown in the figure below:
  ```
  create_generated_clock -name genclk -source pinX -edges {1 5 7} pinY
  ```

Figure: **Derived Clock (Frequency Division)**



- The following command uses the `-divide_by` option to get the same clock period; the difference is in the duty cycle: create_generated_clock -name div_3 -source pinX -divide_by 3 pinY
  With the `-edges` option, the above example creates a clock with a duty cycle of 67 percent (20/30). However, using the `-divide_by 3` option preserves the 50 percent duty cycle of the source clock as shown in the figure below:

Figure: **Derived Clock (Frequency Division, 50 Percent Duty-cycle)**



- Using the following commands, when there is more than 1 source clock, the tool creates the generated clock based on the source clock specified with the `-master` option: create_clock -name c1 -period 10 [get_ports {ck}] -add create_clock -name c2 -period 10 [get_ports {ck}] -add create_generated_clock -name newclk -source ck -divide_by 2 -master_clock c1 reg/Q -add
  Using the `report_clocks` command, the output waveform of the `newclk` clock is a divide-by-2 of the `c1` clock and is reported as shown in the example below.

Example: **report_clocks after using the -master option**

```
> create_clock -name CK1 -period 20 [get_ports {CLK1} ] -add
> create_clock -name CK2 -period 10 [get_ports {CLK1} ] -add
> create_generated_clock -name GCLK -source CLK1 -divide_by 2 u8/Q -add -master_clock CK2

> report_clocks
------------------------------------------------------------
Clock Descriptions
------------------------------------------------------------
Attributes
------------------------------------------------------------
Clock Name  Source  Period   Lead   Trail   Generated   Propagated
------------------------------------------------------------------
CK1         CLK1    20.000   0.000   10.000   n         n
CK2         CLK1    10.000   0.000   5.000    n         n
------------------------------------------------------------------

Generated-Clock Descriptions
-------------------------------------------------------------------------------------
Name  Generated   Master      Master-clock Invert Freq.      Duty-Cycle Edges Edge-Shift
      Source(pin) Source(pin)                   Multiplier
-------------------------------------------------------------------------------------
GCLK    u8/Q      CLK1        CK2          n     1/2        -          -     -
```

Related Information

- create_clock
- get_clocks
- report_clocks

current_design

```
current_design
[-help]
[design_name]
```

Sets the specified design to be the current design.

Parameters

`-help`	Prints out the command usage.
design_name	Specifies the name of the top design.

Related Commands

- set_max_transition
- set_min_capacitance
- set_min_transition

current_instance

```
current_instance
[hierarchical_instance]
```

Sets the instance given by *hierarchical_instance* to be the current instance. Design objects referenced by subsequent commands can be found relative to this instance. All searches for design objects are started in this instance. If design objects are referenced hierarchically, *hierarchical_instance* is used as the root (top) of the hierarchy.

Parameters

hierarchical_instance	Specifies the name of the instance to become the current instance. If *hierarchical_instance* is not given, the current instance is set to the top instance of the hierarchy.

Examples

- The following command sets the current instance to MAC/MULTI, sets a constant on a register pin inside the hierarchical instance and then later sets the current instance back to top:

```
current_instance MAC/MULT1
set_case_analysis 0 [get_pins {reg6/SE}]
...
current_instance
```

Related Commands

- set_case_analysis

get_sdc_mode

```
get_sdc_mode
[-help]
```

Determines the constraint mode that is being currently read.

This command can be used inside SDC constraint files which are shared across multiple constraint modes to conditionalize how the constraints are interpreted for specific modes.

group_path

```
group_path
[-help]
[-comment string]
-name path_group_name
[[-from from_list]
 [-to to_list]
 [-through through_list]]
```

Groups paths in a design, and identifies them with a path group name. You can use the path group names for reporting.

You cannot use `group_path` statements in the SDC file when the software is in MMMC mode. In this case a warning is issued. However, the software supports the group_path command in MMMC mode as a command outside of the SDC file. The path groups are considered to be global in nature and apply to all the views, and are not supported on MMMC mode-specific basis. The path groups are persistent through a saveDesign to restoreDesign sequence. They are saved in the `<saveDir>/mmmc/pathgroup.sdc` file of a MMMC save directory.

Note that if a path can be included in multiple path groups, it will only be included in the path group with the most specific definition (`-from ... -through ... -to ...` is more specific than `-from ... -to ...` is more specific than `-from ...`).

Parameters

`-help`	Prints out the command usage.
`-comment` *string*	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.
`-from` *from_list*	Specifies a list of pins, instances, or clocks that are the start of the paths. When you specify a clock object, all paths that are triggered by the clock are included in the path group.
`-name` *path_group_name*	Specifies the name of the path group. If the path_group_name already exists, the specified paths are added to the existing path group.
`-through` *through_list*	Specifies the pins or instances where the paths go through. When you specify multiple through list, the paths that go through the objects in the specified order are added to the path group.
`-to` *to_list*	Specifies a list of pins, instances, or clocks where the path ends. You must specify valid endpoints in the *to_list* or else the software ignores the constraint. When you specify a clock object, all paths that are checked by the clock are included in the path group. If you specify an instance name in the *to_list*, all the data input pins of the instance are added to the path group.

Examples

- The following example groups paths that start from specific pins and end at the D pin of any instances whose name start with the string ff. The example also reports the specified path groups:

  ```
  group_path -name path1 -from {ff1/CLK ff2/CLK} -to {ff*/D}
  report_timing -path_group path1
  ```

- The following command groups paths that go through specific points.

  ```
  group_path -name path2 -through an1/Z -through an2/Z -through an3/Z
  ```

Related Commands

- report_timing
- get_path_groups
- reset_path_group
- report_path_groups

reset_annotated_check

```
reset_annotated_check
[-clock [rise | fall]]
[-cond sdf_condtion]
[-setup | -recovery]
[-hold | -removal]
[-nochange_high | -nochange_low]
[-rise] [-fall]
{[[-from from_pins][-to to_pins]] | -all}
[object_list]
```

Removes annotated timing check values that you had set by using the `set_annotated_check` command or by SDF annotation.

Note: The `reset_annotated_check` command does not reset the timing check values set by SDF annotations.

Parameters

`-all`	Resets all annotated timing check values in the design.
`-clock [rise | fall]`	Specifies the direction of the related pin of the check to reset. *Default:* Resets checks against both edges of the clock
`-cond` *`sdf_condtion`*	Resets all annotated timing check values for which a condition is specified.
`-fall`	Restricts the reset to only affect the falling edge of the constrained pins. *Default:* Checks on both rising and falling signals are reset
`-from` *`from_pins`*	Specifies the beginning instance pin of the timing arcs for which the annotated check is reset. This is usually the reference (for example, clock pin) of the constraint. The selected timing arcs are those that have their source pins specified using this option and the sink pins specified using the `-to` option. If this option is not specified, only timing arcs whose sink pins are specified using the `-to` option are selected. You cannot use this parameter with *`object_list`* parameter.
`-hold | -removal`	Resets annotations on the hold or removal check type.
`-nochange_high | -nochange_low`	Resets the annotations on the specified timing check type.
`object_list`	Resets the annotated checks on specified list or collection of cell instances.
`-rise`	Restricts the reset to only affect the rising edge of the constrained pins. *Default:* Checks on both rising and falling signals are reset
`-setup | -recovery`	Resets annotations on the setup or recovery check type.
`-to` *`to_pins`*	Specifies the terminating instance pin of the timing arcs for which the annotated check is reset. This is usually the constrained pin (data pin) of the check. The selected timing arcs are those that have their sink pins specified using this option and the source pins specified using the `-from` option. If this option is not specified, only timing arcs whose source pins are specified using the `-from` option are selected. You cannot use this parameter with *`object_list`* parameter.

Examples

- The following command resets all the timing check annotations in the design:
  ```
  reset_annotated_check -all
  ```
- The following command resets all timing check annotations on a particular cell:
  ```
  reset_annotated_check [get_cells BLK/BR1]
  ```

- The following command resets specific checks on a specific instance:
  ```
  reset_annotated_check -setup -hold [get_cells BLK/BR2]
  ```
- The following command resets checks on a specific timing check arc:
  reset_annotated_check -from BLK/BR1/CK -to BLK/BR1/D

Related Commands

- set_annotated_check

reset_annotated_delay

```
reset_annotated_delay
{[-all] | [[-from from_pins] [-to to_pins]] | [object_list]}
```

Resets annotated delay assertions that have been previously specified using the `set_annotated_delay` command. When you reset an annotated delay, the system uses the delays which are calculated from the original libs for the arc.

Parameters

`-all`	Resets all annotated delay assertions in the design.
`-from` *from_pins*	Specifies timing arcs for which the delay is reset. The selected timing arcs are those that have their source pins specified using this option and the sink pins specified using the `-to` option. If this option is not specified, only timing arcs whose sink pins are specified using the `-to` option are selected.
object_list	Resets the annotated delays on specified list or collection of cell instances.
`-to` *to_pins*	Specifies timing arcs for which the delay is reset. The selected timing arcs are those that have their sink pins specified using this option and the source pins specified using the `-from` option. If this option is not specified, only timing arcs whose source pins are specified using the `-from` option are selected.

Examples

- The following command resets all the annotated delay assertions in the design:
  ```
  reset_annotated_delay -all
  ```
- The following command resets all the `set_annotated_delay` assertions applied on pins (either `from` or `to`) of instance `f0`:
  ```
  reset_annotated_delay f0
  ```
- The following command resets the `set_annotated_delay` assertions with the `from` and `to` pin pair combination:
  ```
  reset_annotated_delay -from u1/Y -to u2/A
  ```

Related Information

- `set_annotated_delay`

reset_annotated_transition

```
reset_annotated_transition
{-all | port_or_pin_list}
```

Resets annotated transitions previously asserted by the `set_annotated_transition` command.

Parameters

`-all`	Resets all annotated transitions in the design.
`port_or_pin_list`	Specifies a list of specific pins or primary ports to reset the transition.

Examples

- The following commands reset the `set_annotated_transition` constraint on an internal pin `pad/Y0`.
  ```
  reset_annotated_transition pad/Y0
  ```

  or

  ```
  reset_annotated_transition [get_pins {pad/Y0}]
  ```

  You can also specify a collection as the pin argument. For example:

  ```
  reset_annotated_transition [get_pins pad/*]
  ```
- The following commands reset the `set_annotated_transition` constraint set on a port `in`:
  ```
  reset_annotated_transition in
  ```

  or

  ```
  reset_annotated_transition [get_ports {in}]
  ```

  Any `set_input_transition` constraint applied on the same port is not removed. This is applicable only for input ports, as `set_input_transition` cannot be applied on output ports.
- The following command resets all the `set_annotated_transition` constraints:
  ```
  reset_annotated_transition -all
  ```

Related Information

- set_annotated_transition

reset_aocv_stage_weight

```
reset_aocv_stage_weight
port_list
[-early]
[-late]
```

Resets the existing stage weight values set for the specified ports.

Parameters

`-early`	Resets weight for early paths.
`-late`	Resets weight for late paths.
port_list	Specifies a list or collection of input, output, and bidi ports for which stage weight values are to be reset.

Examples

- The following example resets the stage weight values of ports `clock1` and `out0`:

  ```
  reset_aocv_stage_weight [get_ports {clock1 out0}]
  ```

Related Information

- `set_aocv_stage_weight`

reset_case_analysis

```
reset_case_analysis
list_of_ports_or_pins
```

Removes the assertions set by the `set_case_analysis` command.

Parameters

list_of_ports_or_pins	Specifies the list of ports or pins for which the assertions are to be removed.

Examples

- The following commands sets the assertion to 0 and then resets them:

```
set_case_analysis 0 CM1/S0
get_property [get_pins CM1/S0] constant_value

# Returns a value of 0

reset_case_analysis [get_pins CM1/S0]
get_property [get_pins CM1/S0] constant_value

# Returns a value of NA
```

Related Commands

- set_case_analysis

reset_clock

```
reset_clock
{-all | clock_list}
```

Removes previously created clock assertions.

Parameters

`-all`	Removes all clocks in the design.
clock_list	Specifies the list of clocks for which the clock assertion is removed.

Examples

- The following commands reset a list of clocks and all clocks, and display the results using the `report_clocks` command:

```
report_clocks

# Generates the following report:

------------------------------------------------------------
Clock Descriptions
------------------------------------------------------------
Attributes
------------------------------------------------------------------
Clock Name   Source   Period   Lead   Trail   Generated   Propagated
------------------------------------------------------------------
CK1          CLK1     20.000   0.000  10.000   n          n
CK2          CLK1     10.000   0.000  5.000    n          n
------------------------------------------------------------------
Generated-Clock Descriptions
-------------------------------------------------------------------------------------------
Name    Generated   Master      Master-clock Invert Freq.     Duty-Cycle   Edges   Edge-Shift
        Source(pin) Source(pin)                 Multiplier
-------------------------------------------------------------------------------------------
GCLK4   u8/Q        CLK4        CLK_W_4      n      1/2        -            -       -

reset_clock [get_clocks CK1]
report_clocks
------------------------------------------------------------------
Clock Descriptions
------------------------------------------------------------------
Attributes
------------------------------------------------------------------
Clock Name   Source   Period   Lead   Trail   Generated   Propagated
------------------------------------------------------------------
CK2          CLK1     10.000   0.000  5.000   n           n
------------------------------------------------------------------
------------------------------------------------------------------
Generated-Clock Descriptions
-------------------------------------------------------------------------------------------
Name   Generated    Master      Master-clock  Invert  Freq.      Duty-Cycle  Edges  Edge-Shift
       Source(pin)  Source(pin)                       Multiplier
-------------------------------------------------------------------------------------------
GCLK4  u8/Q         CLK4        CLK_W_4       n       1/2        -           -      -
```

Related Information

- `report_clocks`

reset_clock_gating_check

```
reset_clock_gating_check
[-setup]
[-hold]
[-rise]
[-fall]
[-high | -low]
[object_list]
```

Resets the specified setup and hold values for clock gating checks to the default values.

Parameters

`-high	-low`	Resets the non-controlling value (high or low) of the clock.
`-hold`	Resets clock gating hold values.
`-rise	-fall`	Resets the specified setup (or hold) value to the rising (or falling) signals on the specified pins.
`-setup`	Resets clock gating setup values.
object_list	Specifies a list of clocks, instances, or pins for which clock gating checks are to be reset to default values. If the *object_list* parameter is not specified, the clock gating check is reset on the current design.

Related Information

- set_clock_gating_check

reset_clock_groups

```
reset_clock_groups
[-help]
{-physically_exclusive | -logically_exclusive | -asynchronous}
{-name name_list | -all}
```

Resets clock groups that have been set using the `set_clock_groups` command.

Parameters

`-help`	Prints out the command usage.
`-all`	Removes all physically exclusive, logically exclusive, or asynchronous clock groups in the design. Removes all the physically exclusive, logically exclusive, or asynchronous clock groups in the design. This parameter should be used with one of the parameters: `-logically_exclusive`, `-physically_exclusive`, or `-asynchronous` at one time, and should not be used with the `-name` parameter.
`-asynchronous`	Specifies the asynchronous clock groups to be removed. This parameter must be used with either `-all` or `-name` parameter.
`-logically_exclusive`	Specifies the logically exclusive clock groups to be removed. This parameter must be used with either `-all` or `-name` parameter.
`-name name_list`	Specifies the previously defined names of clock groups to be removed. This parameter should be used with one of the parameters: `-logically_exclusive`, `-physically_exclusive`, or `-asynchronous` at one time.
`-physically_exclusive`	Specifies all the physically exclusive clock groups to be removed. This parameter must be used with either `-all` or `-name` parameter.

Examples

- The following command resets all physically exclusive clock groups in the design:
  ```
  reset_clock_groups -physically_exclusive -all
  ```
- The following command resets physically exclusive clock group `CLKGRP0`:
  ```
  reset_clock_groups -physically_exclusive -name CLKGRP0
  ```

Related Commands

- set_clock_groups

reset_clock_latency

```
reset_clock_latency
[-source]
[-clock clock_list]
pin_or_clock_list
```

Resets assertions made by previous set_clock_latency commands.

Parameters

`-clock clock_list`	Specifies the list of clocks with which associated `set_clock_latency` constraints, at the pin level, are to be removed. **Note:** This parameter cannot be used with clocks specified using the *`pin_or_clock_list`* parameter.
`pin_or_clock_list`	Specifies a list of pins, ports, or clock objects for which the assertions are to be removed.
-source	Removes the source latency.

Examples

- The following commands show the use of the `reset_clock_latency` command to remove a leaf-level `set_clock_latency` assertion, after the clock tree has been put into propagated mode:

  ```
  set_clock_latency 3.141 [get_pins BLK/BR2/CK]
  report_timing -to BLK/BR2/D -path_type full_clock
  ```

  The following report is generated:

  ```
  Path 1: MET Setup Check with Pin BLK/BR2/CK
  Endpoint: BLK/BR2/D (v) checked with leading edge of 'WAVE'
  Beginpoint: BLK/BR1/Q (v) triggered by leading edge of 'WAVE'
  Other End Arrival Time 3.141 <---
  - Setup 0.191
  + Phase Shift 10.000
  = Required Time 12.950
  - Arrival Time 0.249
  = Slack Time 12.701
  ...

  set_propagated_clock [get_ports tclk]
  ```

```
report_timing -path_type full_clock -net -to BLK/BR2/D
```

The following report is generated:

```
Path 1: MET Setup Check with Pin BLK/BR2/CK
Endpoint: BLK/BR2/D (v) checked with leading edge of 'WAVE'
Beginpoint: BLK/BR1/Q (v) triggered by leading edge of 'WAVE'
Other End Arrival Time 3.141 <--------
- Setup 0.191
+ Phase Shift 10.000
= Required Time 12.950
- Arrival Time 0.594
= Slack Time 12.356
...
```

In this example, setting the root of the clock tree to propagated mode is not sufficient to override the `set_clock_latency` assertion at the clock endpoint. You must use the `reset_clock_latency` command to remove the assertion, so that the register sees propagated clock timing.

```
reset_clock_latency [get_pins BLK/BR2/CK]
report_timing -path_type full_clock -net -to BLK/BR2/D
```

The following report is generated:

```
Path 1: MET Setup Check with Pin BLK/BR2/CK
Endpoint: BLK/BR2/D (v) checked with leading edge of 'WAVE'
Beginpoint: BLK/BR1/Q (v) triggered by leading edge of 'WAVE'
Other End Arrival Time 0.345 <--------
- Setup 0.186
+ Phase Shift 10.000
= Required Time 10.159
- Arrival Time 0.588
= Slack Time 9.571
```

- The following commands show that `reset_clock_latency` is only effective when it is applied on the same object for which you had set the assertion using the `set_clock_latency` command:
  Latency here is specified on a pin object:

  ```
  set_clock_latency 3.141 [get_pins BLK/BR2/CK]
  report_timing -path_type full -net -to BLK/BR2/D
  ```

  The following report is generated:

  ```
  Path 1: MET Setup Check with Pin BLK/BR2/CK
  Endpoint: BLK/BR2/D (v) checked with leading edge of 'WAVE'
  Beginpoint: BLK/BR1/Q (v) triggered by leading edge of 'WAVE'
  Other End Arrival Time 3.141 <-
  - Setup 0.191
  + Phase Shift 10.000
  = Required Time 12.950
  ```

```
- Arrival Time 0.249
= Slack Time 12.701
```

Resetting of the latency is done on a clock object and has no effect on downstream latencies specified on pin objects.
In this example, a `reset_clock_latency` on the clock waveform that is driving BLK/BR2/CK will have no effect.

```
reset_clock_latency [get_clocks WAVE]
report_timing -path_type full -net -to BLK/BR2/D
```

The following report is generated:

```
Path 1: MET Setup Check with Pin BLK/BR2/CK
Endpoint: BLK/BR2/D (v) checked with leading edge of 'WAVE'
Beginpoint: BLK/BR1/Q (v) triggered by leading edge of 'WAVE'
Other End Arrival Time 3.141 <-
- Setup 0.191
+ Phase Shift 10.000
= Required Time 12.950
- Arrival Time 0.249
= Slack Time 12.701
```

Related Information

- set_clock_latency

reset_clock_sense

```
reset_clock_sense
[-all]
| [[-clocks clock_list] pin_or_port_list]]
```

Resets clock sense constraint(s) specified using the `set_clock_sense` command.

Parameters

`-all`	Resets all the specified `clock_sense` constraints.
`-clocks` *clock_list*	Specifies the list of clocks for which the clock sense constraints are to be removed. The clocks must have previously been defined using the `set_clock_sense -clock` parameter.
pin_or_port_list	Specifies the list or collection of cell output pins for which the clock sense constraints will be removed.

Examples

- The following command resets all the previously defined `clock_sense` constraints from a design:

  ```
  reset_clock_sense -all
  ```

Related Information

- `set_clock_sense`

reset_clock_transition

```
reset_clock_transition
clock_list
```

Removes the clock transition assertions you had set on the specified list of clock waveforms.

Parameters

clock_list	Specifies a list of clock waveforms for which the clock transition assertions are removed. The *clock_list* variable can be a Tcl list or collection of clock waveforms.

Examples

- The following commands set the clock transition assertions, and resets them and displays the results using the `report_timing` command:

```
set_clock_transition 0.123 [get_clocks WAVE]
report_timing -net -to BLK/BR2/D

# Generates the following report

Path 1: MET Setup Check with Pin BLK/BR2/CK
Endpoint: BLK/BR2/D (v) checked with leading edge of 'WAVE'
Beginpoint: BLK/BR1/Q (v) triggered by leading edge of 'WAVE'
Other End Arrival Time 0.000
- Setup 0.192
+ Phase Shift 10.000
= Required Time 9.808
- Arrival Time 0.249
= Slack Time 9.559
Clock Rise Edge 0.000
= Beginpoint Arrival Time 0.000
+-------------------------------------------------------------------------+
| Instance   | Cell    | Arc       | Pin     | Slew  | Delay | Arrival |
|            |         |           |         |       |       | Time    |
|------------+---------+-------------+-------+-------+-------+---------|
|            |         | tclk ^    | tclk    | 0.123 |       | 0.000 |
| CG/BC1     | BUFX2   |           | A       | 0.123 | 0.000 | 0.000 |
| CG/BC1     | BUFX2   | A ^ -> Y ^ | Y       | 0.123 | 0.000 | 0.000 |
```

```
| TC1         | BUFX2  |             | A       | 0.123 | 0.000 | 0.000 |
| TC1         | BUFX2  | A ^ -> Y ^ | Y       | 0.123 | 0.000 | 0.000 |
| CG1         | AND2X4 |             | B       | 0.123 | 0.000 | 0.000 |
| CG1         | AND2X4 | B ^ -> Y ^ | Y       | 0.123 | 0.000 | 0.000 |
| BLK/blkint | BUFX2  |             | A       | 0.123 | 0.000 | 0.000 |
| BLK/blkint | BUFX2  | A ^ -> Y ^ | Y       | 0.123 | 0.000 | 0.000 |
| BLK/BR1     | DFFHQX1|             | CK      | 0.123 | 0.000 | 0.000 |

reset_clock_transition [get_clocks WAVE]
report_timing -net -to BLK/BR2/D

# Generates the following report:

Path 1: MET Setup Check with Pin BLK/BR2/CK
Endpoint: BLK/BR2/D (v) checked with leading edge of 'WAVE'
Beginpoint: BLK/BR1/Q (v) triggered by leading edge of 'WAVE'
Other End Arrival Time 0.000
- Setup 0.191
+ Phase Shift 10.000
= Required Time 9.809
- Arrival Time 0.249
= Slack Time 9.560
Clock Rise Edge 0.000
= Beginpoint Arrival Time 0.000
+-------------------------------------------------------------------+
| Instance | Cell      | Arc         | Pin   | Slew  | Delay | Arrival |
|                                    |                       |   Time  |
|------------+---------+-------------+-------+-------+-------+---|
|            |         | tclk ^      | tclk  | 0.000 |       | 0.000 |
| CG/BC1     | BUFX2   |             | A     | 0.000 | 0.000 | 0.000 |
| CG/BC1     | BUFX2   | A ^ -> Y ^  | Y     | 0.000 | 0.000 | 0.000 |
| TC1        | BUFX2   |             | A     | 0.000 | 0.000 | 0.000 |
| TC1        | BUFX2   | A ^ -> Y ^  | Y     | 0.000 | 0.000 | 0.000 |
| CG1        | AND2X4  |             | B     | 0.000 | 0.000 | 0.000 |
| CG1        | AND2X4  | B ^ -> Y ^  | Y     | 0.000 | 0.000 | 0.000 |
| BLK/blkint | BUFX2   |             | A     | 0.000 | 0.000 | 0.000 |
| BLK/blkint | BUFX2   | A ^ -> Y ^  | Y     | 0.000 | 0.000 | 0.000 |
| BLK/BR1    | DFFHQX1 |             | CK    | 0.000 | 0.000 | 0.000 |
---------------------------------------------------------------------
```

- The following commands set the clock transition assertions, and resets them and displays the results using

the `report_timing` command:

```
set_clock_transition 0.123 [get_clocks CLK_W_2]
report_timing -net -format {pin arc cell slew delay arrival} -to seg3/u14/D

Path 1: MET Setup Check with Pin seg3/u14/CK
Endpoint: seg3/u14/D (^) checked with leading edge of 'CLK_W_3'
Beginpoint: seg3/u9/Q (^) triggered by leading edge of 'CLK_W_3'
Path Groups: {CLK_W_3}
Other End Arrival Time 0.000
- Setup 0.171
+ Phase Shift 2.000
= Required Time 1.829
- Arrival Time 1.409
= Slack Time 0.420
Clock Rise Edge 0.000
+ Clock Network Latency (Ideal) 0.000
= Beginpoint Arrival Time 0.000
-------------------------------------------------------------------------
Pin          Edge Net           Cell   Slew    Delay   Arrival  Required
                                           Time    Time
-------------------------------------------------------------------------
seg3/u9/CK    ^   seg3/wout      -     0.123   -       0.000     0.420
seg3/u9/Q     ^   seg3/n9        DFF   0.071   0.220   0.220     0.640
seg3/u10/Y    ^   seg3/n10       BUF   0.074   0.071   0.291     0.711
seg3/u11/Y    v   seg3/n11       INV   0.638   0.312   0.603     1.023
seg3/u13/Y    ^   seg3/n13       BUF   1.078   0.403   1.409     1.829
seg3/u14/D    ^   -              DFF   1.078   0.000   1.409     1.829
-------------------------------------------------------------------------
```

Related Information

- set_clock_transition

reset_clock_tree_latency

```
reset_clock_tree_latency
 [object_list]
```

Resets all network clock latency set using the set_clock_latency command in the fanout of the specified clock or pins and ports. The `reset_clock_tree_latency` command resets the latency until the root pins of clocks and generated clocks. Use this command after building clock tree and setting the clock network from the root to propagated mode using the `set_propagated_clock` *`clockRootPinOrPort`* command.

Note: You can use the reset_clock_tree_latency command only on command line and command script. You cannot specify it in an SDC file.

Parameters

`object_list`	Specifies list of objects - pins, ports, or clocks for which latency will be reset.

Related Information

- `set_clock_latency`
- `set_propagated_clock`

reset_clock_uncertainty

```
reset_clock_uncertainty
[-setup]
[-hold]
[-half_cycle_jitter | -full_cycle_jitter]
{  {-from | -rise_from | -fall_from} clksig_from_list
   {-to | -rise_to | -fall_to} clksig_to_list
 | pin_or_clock_list
}
```

Removes the assertions that were made by previous `set_clock_uncertainty` commands.

Parameters

`{-from | -rise_from | -fall_from} clksig_from_list`	
	Removes the assertions for uncertainty value for the specified list of launching clocks. **Note:** You must specify one of these parameters with one of the `-to`, `-rise_to`, or `-fall_to` parameters. You cannot use these parameters when pins are specified.
	`-fall_from`: Removes the uncertainty value only from the falling edge of the launching clocks.
	`-from`: Removes the uncertainty value from both the rising and falling edges of the launching clocks.
	`-rise_from`: Removes the uncertainty value only from the rising edge of the launching clocks.
`-half_cycle_jitter | -full_cycle_jitter`	Removes or resets the clock source jitter assertion(s) of the specified type.
pin_or_clock_list	Removes the target uncertainty assertions from the specified list of clocks or pins. The uncertainty value is removed from the registers clocked by the target waveform, or register pins in the fanout of the specified pin. You can specify a list of clock names, or clock root pin or port names relative to the current module. You also can specify a collection of clocks, or clock root pins or ports. **Note:** You cannot use this parameter when any of the `-from` and `-to` parameter pairs are specified.
`-setup/-hold`	Specifies whether the clock uncertainty is removed for setup or hold checks. *Default*: Removes clock uncertainty for both setup and hold check.

`{-to | -rise_to | -fall_to}` *clksig_to_list*	
	Removes the uncertainty value from the specified list of capture clocks. **Note:** You must specify one of these parameters with one of the `-from`, `-rise_from`, or `-fall_from` parameters. You cannot use these parameters when pins are specified.
	`-fall_to`: Removes the uncertainty value only from the falling edge of the capture clocks.
	`-rise_to`: Removes the uncertainty value only from the rising edge of the capture clocks.
	`-to`: Removes the uncertainty value from both the rising and falling edges of the capture clocks.

Examples

The following commands sets and resets the clock uncertainty assertions and displays the results using the `report_timing` command:

- `set_clock_uncertainty 0.345 -from [get_clocks VCLK] -to [get_clocks WAVE]report_timing -net -from tin`

  ```
  # Generates the following report:
  Path 1: MET Setup Check with Pin TR1/BR1/CK
  Endpoint: TR1/BR1/D (v) checked with leading edge of 'WAVE'
  Beginpoint: tin (v) triggered by trailing edge of 'VCLK'
  Other End Arrival Time 0.000
  - Setup 0.192
  + Phase Shift 10.000
  - Uncertainty 0.345 <---
  = Required Time 9.463
  - Arrival Time 5.877
  = Slack Time 3.586
  ```

- `reset_clock_uncertainty -from [get_clocks VCLK] -to [get_clocks WAVE]report_timing -net -from tin`

  ```
  # Generates the following report:
  Path 1: MET Setup Check with Pin TR1/BR1/CK
  Endpoint: TR1/BR1/D (v) checked with leading edge of 'WAVE'
  Beginpoint: tin (v) triggered by trailing edge of 'VCLK'
  Other End Arrival Time 0.000
  - Setup 0.192
  + Phase Shift 10.000
  = Required Time 9.808
  - Arrival Time 5.877
  ```

```
= Slack Time 3.931
```

Related Information

- set_clock_uncertainty

reset_data_check

```
reset_data_check
{-from | -rise_from | -fall_from} pin_or_port_list
{-to | -rise_to | -fall_to} pin_or_port_list
[-setup]
[-hold]
[-clock clock_object]
```

Removes specified data-to-data checks that you have specified using the set_data_check command.

Parameters

`-clock` *clock_object*	Removes the data check with a related (`-from`) pin event that is triggered by the specified clock. Use this parameter only if the original `set_data_check` constraint also used the `-clock` parameter to specify an explicit clock relationship.
`-from` |`-rise_from` | `-fall_from` *pin_or_port_list*	Specifies the related pins of the data-to-data checks to be removed. The *pin_or_port_list* list can contain either object IDs or hierarchical names relative to the current module. Leaf design cells (instances) can also be specified. Use only one `-from`, `-rise_from`, or `-fall_from` parameter per command. By default, the `-from` parameter removes checks for both the rising and the falling edges. Using `-rise_from` parameter removes checks at the rising edge of the signal on the from pins. Using `-fall_from` parameter removes checks at the falling edge of the signal on the from pins.
`-setup/-hold`	Specifies whether the data check to be removed is for setup or hold check. If you do not specify either `-setup` or `-hold` option, both setup and hold checks are removed.
`-to` | `-rise_to` | `-fall_to` *pin_or_port_list*	Specifies the constrained pins of data-to-data checks to be removed. The *pin_or_port_list* list can contain either object IDs or hierarchical names relative to the current module. Leaf design cells (instances) can also be specified. Use only one `-to`, `-rise_to`, or `-fall_to` parameter per command. By default, the `-to` parameters removes both the rising and the falling edges. Using `-rise_to` parameter removes the rising edge of the signal on the to pins. Using `-fall_to` parameter removes the falling edge of the signal on the to pins.

Examples

- The following commands set and reset the data checks from `port30` to `port28`:

```
set_data_check 0.345 -rise_from port30 -to port28
report_timing -rise_from port30 -to port28
```

The following report is displayed:

```
Path 1: VIOLATED Data To Data Setup Check with Pin port30
Endpoint: port28 (v) checked with leading edge of 'PH1'
Beginpoint: U2/Q (v) triggered by leading edge of 'PH1'
Other End Arrival Time 0.465
- Data Check Setup 0.345
+ Phase Shift 0.000
= Required Time 0.120
- Arrival Time 0.309
= Slack Time -0.189
```

```
reset_data_check -rise_from port30 -to port28
report_timing -rise_from port30 -to port28
```

The following information is displayed:

```
No constrained timing paths with given description found.
Paths may be unconstrained (try '-unconstrained' option) or may not exist.
```

Related Commands

- `set_data_check`
- `report_timing`

reset_disable_clock_gating_check

```
reset_disable_clock_gating_check
object_list
```

Enables clock gating checks for the specified instances or pins.

Parameters

object_list	Specifies the list of disabled pins or instances that are to be enabled.

Related Information

- set_disable_clock_gating_check

reset_disable_timing

```
reset_disable_timing
[-help]
[-from pin_name -to pin_name]
object_list
```

Restores timing arcs that were disabled using the `set_disable_timing` command.

Parameters

`-help`	Prints out the command usage.
`-from pin_name -to pin_name`	Specifies the arc(s) that are to be restored. Specify the `-from` and `-to` options together. You can specify the `-from` and `-to` parameters when the *`object_list`* contains instance or library cell objects. Do not specify the `-from` and `-to` parameters when the *`object_list`* includes instance pin or library cell pin objects. The `-from` and `-to` options are optional. If not specified, all cell arcs found in the instance or library cell objects are restored.
`object_list`	Specifies the instance, instance pin, library arc, library cell, or library cell pin objects. If the *`object_list`* includes instance pin or library cell pin objects, all cell arcs attached to the specified pin objects are restored. If the *`object_list`* contains instance or library cell objects, and the `-from` and `-to` parameters are not specified, all arcs found in the specified instance or library cell objects are restored.

Examples

- The following commands set and reset the timing arcs and display the resulting reports:

```
set_disable_timing -from A -to Y [get_cells b1]
report_inactive_arcs

-------------------------------------------------------------
Flags :              const                propagated constant
                     snipped              loop snipped arcs
                     disable              set_disable_timing
                     disable_clock_gating set_disable_clock_gating
                     library_disable      set_disable_cell_timing
                     Missing_Phase        No Arc Phase Data
From       To       DisableType  ArcType         Reason
-------------------------------------------------------------
b1/A ^     b1/Y ^   disable      combinational   User Disable
b1/A v     b1/Y v   disable      combinational   User Disable
-------------------------------------------------------------
```

- The following commands disable the timing report header:

```
set_global timing_disable_report_header_info true
report_inactive_arcs

-------------------------------------------------------------------------
Flags :                  const                    propagated constant
                         snipped                  loop snipped arcs
                         disable                  set_disable_timing
                         disable_clock_gating     set_disable_clock_gating
                         library_disable          set_disable_cell_timing
                         Missing_Phase            No Arc Phase Data
From       To        DisableType    ArcType                 Reason
-------------------------------------------------------------------------
u8/CK ^    u8/D ^    const          hold_rising_clk_rise    u8/D = 1
u8/CK ^    u8/D v    const          hold_falling_clk_rise   u8/D = 1
u8/CK ^    u8/D ^    const          setup_rising_clk_rise   u8/D = 1
u8/CK ^    u8/D v    const          setup_falling_clk_rise  u8/D = 1
-------------------------------------------------------------------------
```

- The following commands disable the timing arcs from CK to Q for instance U8, then reset the assertions using the instance reference:

```
set_disable_timing -from CK -to Q [get_cells u8]
report_inactive_arcs
```

```
------------------------------------------------------------------
Flags :                 const                 propagated constant
                        snipped               loop snipped arcs
                        disable               set_disable_timing
                        disable_clock_gating  set_disable_clock_gating
                        library_disable       set_disable_cell_timing
                        Missing_Phase         No Arc Phase Data
From       To       DisableType     ArcType       Reason
------------------------------------------------------------------
u8/CK ^    u8/Q ^   disable         rising_edge   User Disable
u8/CK ^    u8/Q v   disable         rising_edge   User Disable
------------------------------------------------------------------
```

- The following commands disable the timing arcs from CK to Q for instance U8, then reset the assertions using the instance reference:

```
reset_disable_timing u8
report_inactive_arcs

------------------------------------------------------------------------
Flags :                   const                 propagated constant
                          snipped               loop snipped arcs
                          disable               set_disable_timing
                          disable_clock_gating  set_disable_clock_gating
                          library_disable       set_disable_cell_timing
                          Missing_Phase         No Arc Phase Data
From      To       DisableType     ArcType                   Reason
------------------------------------------------------------------------
u8/CK ^   u8/D ^   const           hold_rising_clk_rise      u8/D = 1
u8/CK ^   u8/D v   const           hold_falling_clk_rise     u8/D = 1
u8/CK ^   u8/D ^   const           setup_rising_clk_rise     u8/D = 1
u8/CK ^   u8/D v   const           setup_falling_clk_rise    u8/D = 1
------------------------------------------------------------------------
```

Related Information

- set_disable_timing
- report_inactive_arcs

reset_drive

```
reset_drive port_list
```

Resets the drive previously asserted with the `set_drive` command on input ports. The `reset_drive` command removes assertions for all rise/fall, minimum/maximum combinations for which the assertion exists.

Parameters

port_list	Specifies a list of input or bidirectional ports for which the drive resistance is reset.

Related Information

- `set_drive`

reset_driving_cell

```
reset_driving_cell
[-rise]
[-fall]
[-min]
[-max]
port_list
```

Resets the driving cell assertions applied with the `set_driving_cell` command for the given rise/fall, minimum/maximum combination on ports.

If `-rise`, `-fall`, `-min`, or `-max` parameters are not specified, the `reset_driving_cell` command resets the assertion for all rise/fall, minimum/maximum combinations for which the assertion exists.

Parameters

`-rise`	Specifies the rising drive assertions.
`-fall`	Specifies the falling drive assertions.
`-min`	Specifies the early drive assertion.
`-max`	Specifies the late drive assertion.
`port_list`	Specifies a list of ports for which the driving cell assertions are reset.

Related Information

- `set_driving_cell`

reset_generated_clock

```
reset_generated_clock
[-help]
{-all | clock_list}
```

Removes generated clock assertions previously created using the `create_generated_clock` command.

To use the `reset_generated_clock` command you must specify the `set_interactive_constraint_modes` constraints for the applicable modes. You can also include these commands in the SDC constraint file - applicable only for the views that include that SDC.

Note: The generated clocks may originate from the library files dotlib - you can use `reset_generated_clock` from the command line, or add it to all the SDC files in use.

Parameters

`-help`	Prints out the command usage.
`-all`	Removes all the generated clocks in the design.
`clock_list`	Specifies the list of clocks (in collection format) for which the generated clock assertions are to be removed.

Examples

Here are some examples.

- The following command removes the generated clock assertions on all the generated clocks `RC/Q`:

  ```
  report_clocks

  # The following report is displayed:

  ------------------------------------------------------------------------
                                  Clock Descriptions
  ------------------------------------------------------------------------
                                                          Attributes
  ------------------------------------------------------------------------
  Clock Name  Source     Period     Lead     Trail    Generated   Propagated
  ------------------------------------------------------------------------
  CK1         CLK1       20.000     0.000    10.000     n           n
  CK2         CLK1       10.000     0.000    5.000      n           n
  GCLK        u8/Q       20.000     0.000    10.000     y           y
  ------------------------------------------------------------------------

  Generated-Clock Descriptions
  ------------------------------------------------------------------------------
  ```

```
Name  Generated    Master      Master-clock  Invert  Freq.      Duty-Cycle Edges Edge-Shift
      Source(pin)  Source(pin)                       Multiplier
-------------------------------------------------------------------------------
GCLK  u8/Q         CK1         CLK1           n      1/2        -          -     -
-------------------------------------------------------------------------------

set_interactive_constraint_modes [all_constraint_modes -active]
reset_generated_clock [get_clocks GCLK]
report_clocks

# The following report is displayed:
-------------------------------------------------------------------------
                         Clock Descriptions
-------------------------------------------------------------------------
                                                 Attributes
-------------------------------------------------------------------------
Clock Name   Source   Period   Lead    Trail    Generated     Propagated
-------------------------------------------------------------------------
CK1          CLK1    20.000   0.000   10.000    n             n
CK2          CLK1    10.000   0.000   5.000     n             n
GCLK         u8/Q    20.000   0.000   10.000    y             y
-------------------------------------------------------------------------
Generated-Clock Descriptions
-------------------------------------------------------------------------------
Name  Generated   Master      Master-clock Invert  Freq.      Duty-Cycle Edges Edge-Shift
      Source(pin) Source(pin)                     Multiplier
-------------------------------------------------------------------------------
```

Related Information

- create_generated_clock
- report_clocks
- set_interactive_constraint_modes

reset_ideal_latency

```
reset_ideal_latency
[-min]
[-max]
[-rise]
[-fall]
object_list
```

Resets the ideal latency constraints, specified using the `set_ideal_latency` command, on instance pins or ports.

Parameters

`-fall`	Resets the ideal latency for the falling edge.
`-max`	Resets the ideal latency for the maximum corner.
`-min`	Resets the ideal latency for the minimum corner.
`object_list`	Resets a list of objects - instance pins, ports, or nets.
`-rise`	Resets the ideal latency for the rising edge.

Related Information

set_ideal_latency

reset_ideal_network

```
reset_ideal_network
object_list
```

Resets the ideal network constraints, specified using the `set_ideal_network` command, on pins, ports, or nets.

Note: The `reset_ideal_network` command is not view/mode-based - it applies to all the active views.

Parameters

object_list	Resets a list of objects - instance pins, ports, or nets.

Related Information

set_ideal_network

reset_ideal_transition

```
reset_ideal_transition
[-min]
[-max]
[-rise]
[-fall]
object_list
```

Resets the ideal transition constraint, specified using the `set_ideal_transition` command, on instance pins or ports.

Parameters

`-fall`	Resets the ideal transition for the falling edge.
`-max`	Resets the ideal transition for the maximum corner.
`-min`	Resets the ideal transition for the minimum corner.
`object_list`	Resets a list of objects - instance pins, ports, or nets.
`-rise`	Resets the ideal transition for the rising edge.

Related Information

set_ideal_transition

reset_input_delay

```
reset_input_delay
[-help]
[-clock clk_name]
[-clock_fall]
[-rise] [-fall]
[-max] [-min]
[-level_sensitive]
pin_or_port_list
```

Resets previously specified input delay assertions.

Parameters

`-help`	Prints out the command usage.
`-clock` *`clock_name`*	Specifies the name of the clock. *Default*: The asynchronous (`@`) clock. If you specify multiple clocks, an error occurs and the constraint is ignored. When you specify the `-clock` parameter, the constraints specified for a pin with matching `-clock` parameter are reset. If you do not specify the `-clock` parameter, all the constraints applied on the specified pin, with or without the `-clock` parameter specification, are reset.
`-clock_fall`	Removes the assertions for the delay that is relative to the falling edge of the clock. You must specify the `-clock` parameter with this parameter. *Default*: Removes the delay assertions relative to the falling edge.
`-fall`	Resets the input delay for the falling edge at the input port. If both the `-rise` and `-fall` options are not specified, then the input delay is reset on both the edges.
`-level_sensitive`	Resets input delay that is launched by a positive level sensitive latch. When you specify the `-level_sensitive` parameter, the command resets all the `set_input_delay` constraints specified with the `-level_sensitive` parameter. If `-level_sensitive` parameter is not specified, the constraints specified for the same pin, with or without the `-level_sensitive` parameter, are reset. **Note:** You must specify the `-clock` parameter with this parameter.
`-max`	Removes the delay assertions that refer to the setup analysis. If you do not specify the `-max` parameter, the input delay assertions for both setup and hold analysis are removed.
`-min`	Removes the delay assertions that refer to the hold analysis. If you do not specify the `-min` parameter, the input delay assertions for both setup and hold analysis are removed.
`pin_or_port_list`	Specifies a single or multiple pins or ports for which the input delay is reset. To specify multiple pins, enclose the list with curly braces (`{}`) and separate the pin information with white space.
`-rise`	Resets the input delay for the rising edge at the input port. If both `-rise` and `-fall` options are omitted, the input delay is reset on both the edges.

Related Information

- `set_input_delay`

reset_load

```
reset_load
object_list
```

Resets the existing `set_load` assertions specified on nets or ports.

Parameters

object_list	Specifies a list of nets or ports in the top cell for which assertions are reset.

Related Information

set_load

reset_max_capacitance

```
reset_max_capacitance
object_list
```

Resets maximum capacitance limit specified (using the the `set_max_capacitance` command) on the objects in the object list. The valid objects in the object list are primary ports, clocks, and current design.

Parameters

object_list	Specifies the list of ports, modules, and/or clock waveforms for which assertions are reset.

Related Information

set_max_capacitance

reset_max_fanout

```
reset_max_fanout
object_list
```

Resets maximum fanout limit specified (using the `set_max_fanout` command) on the objects in the object list. The valid objects in the object list are input and bidirectional ports and current design.

Parameters

object_list	Specifies the list of ports or modules for which assertions are reset.

Related Information

set_max_fanout

reset_max_time_borrow

```
reset_max_time_borrow
object_list
```

Resets the specified maximum time borrow limit of the specified objects.

Parameters

object_list	Specifies the list of pins, instances, or clocks for which the time borrow limit is reset.

Related Information

set_max_time_borrow

reset_max_transition

```
reset_max_transition
object_list
```

Resets the maximum slew (transition) limit specified using the `set_max_transition` command (from the SDC or the Tempus command line) on the objects in the given object list. The valid objects in the object list are primary ports, clocks, and current design.

The maximum transition limit that is set by the user overrides the dotlib value so that sharper slews are implemented on the timing sensitive nets, such as clocks. A violation due to the `set_max_transition` setting does not impact the timing accuracy.

Note: For command-line usage of this command, you need to use `set_interactive_constraint_modes`, for example, `set_interactive_constraint_modes [all_constraint_modes -active]`

Parameters

object_list	Specifies the list (or collection) of ports, modules, and/or clock waveforms for which assertions are to be reset. For example, `reset_max_transition [get_ports *]`

Related Information

- `set_max_transition`

reset_min_capacitance

```
reset_min_capacitance
object_list
```

Resets minimum capacitance limit specified (using the the `set_min_capacitance` command) on objects in the object list. The valid objects in the object list are primary ports, clocks, and the current design.

Parameters

object_list	Specifies the list of ports, modules, and/or clock waveforms for which assertions are reset.

Related Information

set_min_capacitance

reset_min_fanout

```
reset_min_fanout
object_list
```

Resets minimum fanout limit specified (using the `set_min_fanout` command) on the objects in the object list. The valid objects in the object list are input and bidirectional ports and the current design.

Parameters

object_list	Specifies the list of ports or modules for which assertions are reset.

Related Information

- `set_min_fanout`

reset_min_pulse_width

```
reset_min_pulse_width
[-help]
object_list
[-high]
[-low]
[-waveform_aware_type {absolute | source_width_ratio | all}]
```

Resets minimum pulse width constraint on the specified input ports of the top cell and the specified modules.

Parameters

`-help`	Prints out the command usage.
`-low`	Resets the minimum pulse width checks that are applied to low signal levels only.
`-high`	Resets the minimum pulse width checks that are applied to high signal levels only.
`object_list`	Specifies a list of pins or clock waveforms for which the minimum pulse width is to be reset. If a clock waveform is specified, all the pins driven by the clock are affected.
`-waveform_aware_type {absolute | source_width_ratio | all}`	Resets the regular and waveform-aware pulse width assertions.

Related Information

- `set_min_pulse_width`

reset_min_transition

```
reset_min_transition
object_list
```

Resets minimum slew (transition) limit specified (using the `set_min_transition` command) on the objects in the object list. The valid objects in the object list are primary ports, clocks, and current design.

Parameters

object_list	Specifies the list of ports, modules, and/or clock waveforms for which assertions are reset.

Related Information

- `set_min_transition`

reset_mode

```
reset_mode
[-type cell]
list_of_modes
object_list
```

Resets the Liberty timing library modes for a specified instance.

Library modes are set using the set_mode command. By default, all library modes in the timing library are active. If you specify one or more library modes using the `set_mode` command, only the timing arcs consistent with the specified mode(s) are active for the instance. All other library modes become inactive. When you reset the library mode for the instance, all library modes become active again.

Parameters

object_list	Specifies the instances to which to reset the library modes.
list_of_modes	Specifies the name of the library modes to reset.
-type cell	Resets the library mode at the instance level. **Note:** Setting the library mode at the instance level is the only form of `set_mode` that is currently supported. This parameter exists to support constraints coming from external sources.

Examples

- The following command specifies that only the timing arcs consistent with the `test_A` library mode are active for instances of b1:
  The test_A library mode is made inactive.

  ```
  set_mode test_mode [get_cells BUF]
  ```

  The following command reports the status of all library modes associated with BUF 'b1' . It shows that test_A is active and test_B is inactive.

  ```
  > report_mode b1
  ------------------------------
  Mode Group test_mode of b1
  ------------------------------
  Mode     Status    Condition
  Name
  ------------------------------
  test_B   Inactive  !(Y)
  test_A   ACTIVE    Y
  ------------------------------
  ```

  The following command resets the library mode for instances of b1 :

  ```
  reset_mode test_A b1
  ```

  If you generate a report of the library mode status for b1, it shows that both modes are now active:

  ```
  > report_mode b1
  ------------------------------
  Mode Group test_mode of b1
  ------------------------------
  Mode    Status   Condition
  Name
  ------------------------------
  test_B  ACTIVE    !(Y)
  test_A  ACTIVE    Y
  ------------------------------
  ```

Related Commands

- set_mode
- report_mode

reset_output_delay

```
reset_output_delay
[-help]
port_or_pin_list
[-clock clock_name]
[-clock_fall]
[-fall]
[-level_sensitive]
[-max]
[-min]
[-rise]
```

Resets previously specified output delay assertions.

Parameters

`-help`	Prints out the command usage.
`-clock` *`clock_name`*	Specifies the name of the clock. *Default*: The asynchronous (`@`) clock. If you specify multiple clocks, an error occurs and the constraint is ignored. When you specify the `-clock` parameter, the constraints specified for a pin with matching `-clock` parameter are reset. If you do not specify the `-clock` parameter, all the constraints applied on the specified pin, with or without the `-clock` parameter specification, are reset.
`-clock_fall`	Removes the assertions for the delay that is relative to the falling edge of the clock. You must specify the -clock parameter with this parameter. *Default:* Removes the delay assertions relative to the rising edge.
`-fall`	Resets the output delay for the falling edge at the output port. If both `-rise` and `-fall` options are omitted, the output delay is reset on both the edges.
`-level_sensitive`	Resets output delay that is launched by a positive level sensitive latch. When you specify the `-level_sensitive` parameter, the command resets all the `set_output_delay` constraints specified with the `-level_sensitive` parameter. If `-level_sensitive` parameter is not specified, the constraints specified for the same pin, with or without the `-level_sensitive` parameter, are reset. **Note:** You must specify the `-clock` parameter with this parameter.
`-max`	Removes the delay assertions that refer to the setup analysis. If you do not specify the `-max` parameter, the output delay assertions for both setup and hold analysis are removed.
`-min`	Removes the delay assertions that refer to the hold analysis. If you do not specify the `-min` parameter, the output delay assertions for both setup and hold analysis are removed.
`pin_or_port_list`	Specifies a single or multiple pins or ports for which the output delay is reset. To specify multiple pins, enclose the list with curly braces (`{}`) and separate the pin information with white space.
`-rise`	Resets the output delay for the rising edge at the output port. If both `-rise` and `-fall` options are omitted, the output delay is reset on both the edges.

Examples

- The following example sets the output delays for the output port Z. The output signal is required at Z at 2 units before the rising edge of clock Clk1, and at 2.5 units before falling edge of clock Clk2:

```
set_output_delay 2 Z -clock Clk1
set_output_delay 2.5 Z -clock Clk2 -clock_fall -add_delay
```

- The following command will reset the output delay for the rising edge of clock Clk2:

```
reset_output_delay Z -clock Clk2

report_timing -to_rise Z -clock_to Clk2
#####################################################################
# Generated by: Cadence Tempus 18.11-s094_1
# OS: Linux x86_64(Host ID noi-rakeshp)
# Generated on: Thu Apr 18 09:50:52 2019
# Design: top
# Command: report_timing -to_rise Z -clock_to Clk2
#####################################################################
No constrained timing paths with given description found.
Paths may be unconstrained (try '-unconstrained' option) or may not exist.

report_timing -to_fall Z -clock_to Clk2
#####################################################################
Path 1: VIOLATED Late External Delay Assertion
Endpoint: Z (v) checked with trailing edge of 'Clk2'
Beginpoint: dff3/Q (v) triggered by leading edge of 'Clk'
Path Groups: {vclk2}
Other End Arrival Time 2.000
- External Delay 2.500
+ Phase Shift 0.000
= Required Time -0.500
- Arrival Time 0.165
= Slack Time -0.665
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.052
= Beginpoint Arrival Time 0.052
-------------------------------------------------------------
Instance Arc Cell Delay Arrival Required
Time Time
-------------------------------------------------------------
dff3 CK ^ - - 0.052 -0.613
dff3 CK ^ -> Q v DFFHQX1 0.113 0.165 -0.500
- Z v - 0.000 0.165 -0.500
-------------------------------------------------------------
```

- The following command resets the output delay for clock Clk2 with respect to the falling edge:

  ```
  reset_output_delay Z -clock Clk2 -clock_fall
  ```

- The following command will reset all the output delays with respect to all the edges for both the clocks - Clk1 and Clk2:

  ```
  reset_output_delay Z
  ```

Related Commands

- set_output_delay

reset_path_exception

```
reset_path_exception
[-exact]
[-type {false_path | multicycle | path_delay}]
[-rise]
[-fall]
[{-from | -rise_from | -fall_from} from_list]
[{-through | -rise_through | -fall_through} through_list]
[{-to | -rise_to | -fall_to} to_list]
[-setup | -hold]
[-all]
```

Removes any previously set path exceptions (or path exception of the specified type) for the given paths. To remove all path exceptions, use the `-all` option with the `reset_path_exception` command.

Parameters

`-all`	Removes all path exceptions. This option cannot be used with the `-from`, `-through and -to` options.
`-exact`	Forces exact matching of the constraints. When this parameter is specified, the reset settings (-from/-through/-to) match with the original constrained path settings.
`-fall`	Applies the assertion at the falling edge of the signal on the path endpoints.
`{-from | -rise_from | -fall_from} from_list`	Specifies pins at the start of the path exceptions that are to be reset.
`-rise`	Applies the assertion at the rising edge of the signal on the path endpoints.
`{-through | -rise_through | -fall_through} through_list`	Specifies the pins that the path to be reset goes through.
`{-to | -rise_to | -fall_to} to_list`	Specifies pins at the end of the path exceptions that are to be reset.
`-setup | -hold`	Resets only the setup (or hold) paths. The `-setup`/`-hold` parameters should be used to reset maximum or minimum delay that is set using the `set_max_delay` and `set_min_delay` commands. *Default*: Both `-setup` and `-hold` are reset.
`-type {false_path | multicycle | path_delay}`	Removes only path exceptions of the specified type. *Default*: All path exception assertions are reset.

Examples

- In the example below the rising path from R1 to R2 has three different path exceptions asserted on it - they are listed in priority from lowest to highest. As shown in the reports below, the false path is the highest precedence and so no path is reported.

```
set_multicycle_path -from R1/CK -rise_to R2/D 2
set_max_delay 3 -from R1/CK -rise_to R2/D
set_false_path -from R1/CK -rise_to R2/D

report_timing -from R1/CK -rise_to R2/D
```

The following information is displayed:

```
No constrained timing paths with given description found.
Paths may be unconstrained (try '-unconstrained' option) or may not exist.

report_path_exceptions
```

The following report is displayed:

```
---------------------------------------
From      To         Early        Late
---------------------------------------
R1/CK     ^ R2/D    false        false
---------------------------------------
```

To report ignored path exceptions, you can use the following command:

```
report_path_exception -ignored

-------------------------------------------
From     To       Early      Late
-------------------------------------------
R1/CK   ^ R2/D               delay 3 ignored
R1/CK   ^ R2/D    add 0 ignored
R1/CK   ^ R2/D               add 1 ignored
-------------------------------------------
```

- In this example, the false path exception is removed from the path. This causes the max path delay assertion to become active - as it is the next highest precedence.

```
reset_path_exception -from R1/CK -rise_to R2/D -type false_path

report_path_exceptions
-------------------------------------
From      To       Early     Late
-------------------------------------
R1/CK    ^ R2/D             delay 3
R1/CK    ^ R2/D    add 0
-------------------------------------

report_timing -from R1/CK -rise_to R2/D

Path 1: MET Setup Check with Pin R2/CK
```

```
Endpoint: R2/D (^) checked with leading edge of 'CLK_W_1'
Beginpoint: R1/Q (^) triggered by leading edge of 'CLK_W_1'
Path Groups: {CLK_W_1}
Other End Arrival Time 0.000
- Setup 0.097
+ Path Delay 3.000
= Required Time 2.903
- Arrival Time 0.243
= Slack Time 2.660
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.000
= Beginpoint Arrival Time 0.000
-------------------------------------------------------------------
Load    Slew    Delay    Arrival    Cell    Arc           Pin
                         Time
-------------------------------------------------------------------
0.026   0.003            0.000              CK ^          R1/CK ->
0.011   0.063   0.187    0.187      DFF     CK ^ -> Q ^   R1/Q
0.003   0.042   0.056    0.243      BUF     A ^ -> Y ^    B1/Y
0.003   0.042   0.000    0.243      DFF     D ^           R2/D ->
-------------------------------------------------------------------
```

- In this example, the set_max_delay exception is removed, leaving only the set_multicycle_path exception as active.

```
reset_path_exception -from R1/CK -rise_to R2/D -type path_delay
```

```
report_timing -from R1/CK -rise_to R2/D
Path 1: MET Setup Check with Pin R2/CK
Endpoint: R2/D (^) checked with leading edge of 'CLK_W_1'
Beginpoint: R1/Q (^) triggered by leading edge of 'CLK_W_1'
Path Groups: {CLK_W_1}
Other End Arrival Time 0.000
- Setup 0.097
+ Phase Shift 20.000
+ Cycle Adjustment 20.000 <----- Multi-Cycle timing
= Required Time 39.903
- Arrival Time 0.243
= Slack Time 39.660
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.000
= Beginpoint Arrival Time 0.000
-------------------------------------------------------------------
Load    Slew     Delay    Arrival    Cell   Arc           Pin
                          Time
-------------------------------------------------------------------
0.026   0.003            0.000              CK ^          R1/CK ->
0.011   0.063    0.187   0.187      DFF     CK ^ -> Q ^   R1/Q
0.003   0.042    0.056   0.243      BUF     A ^ -> Y ^    B1/Y
0.003   0.042    0.000   0.243      DFF     D ^           R2/D ->
```

```
-------------------------------------------------------------------
```

- In this example, the multi-cycle path assertion is removed, and the analysis reverts back to single-cycle timing

```
reset_path_exception -rise_to R2/D -type multicycle
report_path_exceptions

# None reported

report_timing -from R1 -rise_to R2/D

# The following report is displayed:


Path 1: MET Setup Check with Pin R2/CK
Endpoint: R2/D (^) checked with leading edge of 'CLK_W_1'
Beginpoint: R1/Q (^) triggered by leading edge of 'CLK_W_1'
Path Groups: {CLK_W_1}
Other End Arrival Time 0.000
- Setup 0.097
+ Phase Shift 20.000 <--- Single-cycle timing
= Required Time 19.903
- Arrival Time 0.243
= Slack Time 19.660
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.000
= Beginpoint Arrival Time 0.000
-------------------------------------------------------------------
Load    Slew    Delay    Arrival    Cell    Arc           Pin
                         Time
-------------------------------------------------------------------
0.026   0.003   0.000                       CK ^          R1/CK ->
0.011   0.063   0.187   0.187       DFF     CK ^ -> Q ^   R1/Q
0.003   0.042   0.056   0.243       BUF     A ^ -> Y ^    B1/Y
0.003   0.042   0.000   0.243       DFF     D ^           R2/D ->
-------------------------------------------------------------------
```

- Consider the following `set_false_path` constraint:

```
set_false_path -from {u1/Y u2/Y} -to {u3/Y}
```

When you specify `reset_path_exception -from {u1/Y}` command without the `-exact` parameter, the command outputs the following:

```
set_false_path -from {u2/Y} -to {u3/Y}
```

The command `reset_path_exception -from {u1/Y} -exact` outputs the following:

```
set_false_path -from {u1/Y u2/Y} -to {u3/Y}
```

In this case (when `-exact` parameter is used), the path exception is not reset because differential matching is not allowed.

Related Information

- `set_false_path`
- `report_timing`

reset_path_group

```
reset_path_group
[-name group_name]
[-all]
```

Removes the specified path groups or completely removes all the groups.

Parameters

`-all`	Removes all path groups.
`-name group_name`	Removes the named group completely.

Example

- The following command removes the group named GRPA:
  ```
  reset_path_group -name GRPA
  ```

Related Information

- `group_path`

reset_pll_timing

```
reset_pll_timing
[-help]
inst_name
-output_clock
clock name or ID
-output_pin output pin name
```

Resets the delay offset settings applied to a PLL (phase locked loop) output clock.

Parameters

`-help`	Prints out the command usage.
inst_name	Specifies the name of the instance on which reference pins, feedback pins, or output pins will be searched.
`-output_clock <`*clock name or ID*`>`	Specifies the name of the PLL output clock.
`-output_pin <`*output pin name*`>`	Specifies the name of the PLL output pin.

Related Commands

- set_pll_timing

reset_propagated_clock

```
reset_propagated_clock
pin_clock_list
```

Removes the propagated clock assertion for the specified clock waveforms or pins.

The ideal or propagated status of clock endpoints in the fanout of the specified pin/clock list is determined by the remaining clock latency propagated clock assertions in the clock network.

When new analysis views are used, the new corners do not have the source latency updated as per the clock propagation (`update_io_latency`) settings, so a reset and re-calculation is required.

Note: You must specify `set_interactive_constraint_modes` constraint prior to running the `reset_propagated_clock` command.

Parameters

`-help`	Prints out the command usage.
`pin_clock_list`	Specifies a list of pins or clocks for which you want to reset the clock propagation mode to ideal mode. The *`pin_clock_list`* parameter can be a collection, however, no hierarchical pins are allowed in the list.

Examples

- The following command sets the `CLK1` clock waveform to `propagated` mode:

  ```
  set_propagated_clock CLK1
  ```

- The following command resets the `CLK1` clock waveform back to `ideal` mode:

  ```
  reset_propagated_clock CLK1
  ```

- The following commands set the `CLK1` clock to `propagated` mode and then reports them:

  ```
  set_propagated_clock [get_clocks CLK1]
  report_clocks
  ```

  The following report is displayed:

  ```
  ------------------------------------------------------------------------
                             Clock Descriptions
  ------------------------------------------------------------------------
                                                          Attributes
  ------------------------------------------------------------------------
  Clock Name      Source   Period    Lead    Trail    Generated   Propagated
  ------------------------------------------------------------------------
  CLK1            CLK1      20.000   0.000    10.000     n          y
  CLK2            CLK2      25.000   0.000    12.500     n          y
  CLK3            CLK3      10.000   0.000    6.000      n          y
  ```

```
GCLK3          u8/Q      20.000   0.000    10.000    y          y
-------------------------------------------------------------------------
Generated-Clock Descriptions
-------------------------------------------------------------------------
Name  Generated     Master     Master-clock  Invert  Freq.       Duty-Cycle  Edges  Edge-Shift
      Source(pin)   Source(pin)                      Multiplier
-------------------------------------------------------------------------
GCLK3  u8/Q         CLK3       CLK3          n       1/2          -           -     -
-------------------------------------------------------------------------
```

- The following commands reset the `CLK1` clock to `ideal` mode and then reports them:

  ```
  reset_propagated_clock [get_clocks CLK1]
  report_clocks
  ```

  The following report is displayed:

  ```
  -------------------------------------------------------------------------
                              Clock Descriptions
  -------------------------------------------------------------------------
                                                      Attributes
  -------------------------------------------------------------------------
  Clock Name    Source   Period   Lead    Trail    Generated      Propagated
  -------------------------------------------------------------------------
  CLK1          CLK1     20.000   0.000   10.000   n              n
  CLK2          CLK2     25.000   0.000   12.500   n              y
  CLK3          CLK3     10.000   0.000   6.000    n              y
  GCLK3         u8/Q     20.000   0.000   10.000   y              y
  -------------------------------------------------------------------------
  Generated-Clock Descriptions
  -------------------------------------------------------------------------
  Name    Generated    Master       Master-clock   Invert  Freq.       Duty-Cycle   Edges   Edge-
  Shift
          Source(pin)  Source(pin)                         Multiplier
  -------------------------------------------------------------------------
  GCLK3   u8/Q         CLK3         CLK3            n      1/2         -            -       -
  -------------------------------------------------------------------------
  ```

Related Commands

- create_clock
- get_propagated_clock
- report_clocks
- set_clock_latency
- set_propagated_clock
- update_io_latency
- set_interactive_constraint_modes

reset_property

```
reset_property
[-object_type {object_type}]
{object_list}
property_name
```

Resets user-defined properties on objects set using the `set_property` command.

Parameters

object_list	Specifies a list of objects for which the property is to be reset.
-object_type {*object_type*}	Specifies the type of object on which the property is to be reset.
property_name	Specifies the name of the property for which the user-defined property is to be reset.

Examples

- The following command resets user-defined property `slk` from pins `C1/A`:

  ```
  reset_property [get_pins C1/A] slk
  ```

Related Information

- get_property
- report_property
- define_property
- set_property

reset_pulse_clock_max_transition

```
reset_pulse_clock_max_transition
object_list
[-transitive_fanout]
[-rise]
[-fall]
```

Resets the maximum pulse clock transition constraint specified (using the `set_pulse_clock_max_transition` command) on the pulse generator cells, pulse generator library cells, clocks, and/or current design.

Parameters

`-fall`	Resets transition limit value for falling transition.
`object_list`	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design - for which the transition value is to be reset.
`-rise`	Resets transition limit value for rising transition.
`-transitive_fanout`	Specifies the transition limit value for transitive fanout.

Related Information

- `set_pulse_clock_max_transition`

reset_pulse_clock_max_width

```
reset_pulse_clock_max_width
object_list
[-transitive_fanout]
```

Resets the maximum pulse clock width constraint specified (using the `set_pulse_clock_max_width` command) on the pulse generator cells, pulse generator library cells, clocks, and/or current design.

Parameters

object_list	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design - for which the maximum pulse clock width is to be reset.
[-transitive_fanout]	Specifies the pulse width value for transitive fanout.

Related Information

- `set_pulse_clock_max_width`

reset_pulse_clock_min_transition

```
reset_pulse_clock_min_transition
object_list
[-transitive_fanout]
[-rise]
[-fall]
```

Resets the minimum pulse clock transition constraint specified (using the `set_pulse_clock_min_transition` command) on the pulse generator cells, pulse generator library cells, clocks, and/or current design.

Parameters

`-fall`	Resets transition limit value for falling transition.
object_list	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design - for which the minimum pulse clock transition value is to be reset.
`-rise`	Resets transition limit value for rising transition.
`-transitive_fanout`	Specifies the transition limit value for transitive fanout.

Related Information

- `set_pulse_clock_min_transition`

reset_pulse_clock_min_width

```
reset_pulse_clock_min_width
object_list
[-transitive_fanout]
```

Resets the minimum pulse clock width constraint specified (using the `set_pulse_clock_min_width` command) on the pulse generator cells, pulse generator library cells, clocks, and/or current design.

Parameters

object_list	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design - for which the minimum pulse clock width is to be reset.
[-transitive_fanout]	Specifies the pulse width value for transitive fanout.

Related Information

- `set_pulse_clock_min_width`

reset_resistance

```
reset_resistance
net_list
```

Resets the existing `set_resistance` assertions specified on nets.

Parameters

net_list	Specifies a list of nets for which the wire resistance is reset.

Related Information

- `set_resistance`

set_active_clocks

```
set_active_clocks
[-help]
```
active_clock_list | *all_clocks*

Defines a list of active clocks in the design.

When the `set_active_clocks` command is specified, only the specified clocks are made active while others remain inactive. Any clock created after using the `set_active_clocks` command is active by default. The following commands are impacted:

- `report_clocks` command output displays a new column showing active or inactive clocks in the design.
- `get_clocks` command will report all clocks including inactive clocks.
- `write_timing_windows` command will not display windows for inactive clocks in the output report.
- `get_property` command allows you to query on active clocks using the `is_clock_active` property.

Parameters

`-help`	Prints out the command usage.
active_clock_list | *all_clocks*	Specifies a list of active clocks.

Examples

- The following command makes the specified clocks active while others remain inactive (only `Iclk1` is active, while `Iclk2` is inactive):

```
set_active_clocks Iclk1
```

Related Information

- `report_clocks`
- `get_clocks`
- `get_property`

set_annotated_check

```
set_annotated_check
[-help]
check_value
[-clock clock_check]
[-cond sdf_condtion]
[-fall]
-from from_pins
[-incremental]
[-max]
[-min]
[-rise]
-to to_pins
{-setup | -hold | -recovery | -removal | -nochange_high | -nochange_low}
```

Annotates the setup, hold, recovery, or removal timing check value between two or more pins of a cell in the current design.

Parameters

`-help`	Prints out the command usage.
check_value	Specifies the timing check value.
`-clock` *clock_check*	Specifies clock rising or falling. Default: Sets checks for both clock rise and fall. Use the *clock_check* argument to specify rise or fall.
`-cond` *sdf_condition*	Specifies the condition only if the library has a condition attached to the specified timing check arc. The *sdf_expression* must match the syntax of the condition specified in the library.
`-fall`	Specifies the data fall transition for the timing check. Default: Sets both `-rise` and `-fall`.
`-from` *from_pins*	Specifies start points of the timing arcs for which checks are annotated. Use the *from_pins* argument to specify a list of leaf cell clock pins.
`-incremental`	Set incremental checks.
`-max`	Specifies the maximum timing check for both data rise and data fall transitions. Use this option only if the design uses minimum and maximum (`min-max`) operating conditions.

`-min`	Specifies the minimum timing check for both data rise and data fall transitions. Use this option only if the design uses minimum and maximum (`min-max`) operating conditions.
`-rise`	Specifies the data rise transition for the timing check. Default: Sets both rise and fall.
`-setup | -hold | -recovery | -removal | -nochange_high | -nochange_low`	Specifies the timing check type. Make sure there is a corresponding timing arc between the *`from_pins`* and the *`to_pins`* arguments.
`-to` *`to_pins`*	Specifies endpoints of the timing arcs for which checks are annotated. Use the *`to_pins`* argument to specify a list of leaf cell clock pins.

Examples

- The following commands annotates a `nochange check` between data low and clock low. The nochange margin before the clock is `1.3` and the nochange margin after the clock is `2.4`:

```
set_annotated_check -nochange_low 1.3 -clock fall -fall -from {I1/CP} -to {I1/EN}
set_annotated_check -nochange_low 2.4 -clock fall -rise -from {I1/CP} -to {I1/EN}
```

- The following command annotates a 2.0 setup time between the CP clock pin and the A data pin of the <*instance_name*> instance:

```
set_annotated_check -setup 2.0 -from <instance_name>/CP -to <instance_name>/A
```

Related Commands

- reset_annotated_check
- set_annotated_delay

set_annotated_delay

```
set_annotated_delay
[-help]
delay_value
[-cond expression]
[-delta_only]
[-fall]
[-increment]
[-max]
[-min]
[-rise]
{-net | -cell }
{[-from from_pins] [-to to_pins]}
```

Annotates delay to timing arcs. If you specify the `set_annotated_delay` command without any parameters, the delay value specified is assumed to include both the base delay and any delta (cross-talk) delay.

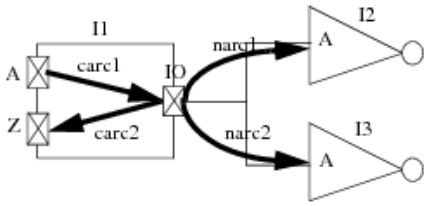
Parameters

`-help`	Prints out the command usage.
`-cond` *`expression`*	Specifies the sdf condition of the selected timing arcs. *Default*: The delay applies to all the timing arcs selected using the `-from` and the `-to` options. The expression is ignored for net arcs. **Note:** You cannot specify the `-cond` parameter with `-net`. If specified, an error is displayed.
`delay_value`	Specifies the delay value to be annotated to the selected timing arcs.
`-delta_only`	Populates the incremental delay slot with the delay value, so that you can differentiate between the base net delay and a delay offset that is due to cross talk. The software reports the incremental (or delta) delay value separately in the timing report using `report_timing -format incr_delay`, and maintains the separate value for CPPR calculations. If the base delay was previously calculated or annotated from the SDF, the `-delta_only` parameter adds the specified delay to the existing base delay. **Note:** This parameter only can be used with the `-net` parameter; you cannot specify it with the `-cell` parameter.
`-from` *`from_pins`*	Specifies timing arcs to which the delay applies. The selected timing arcs are those that have their source pins specified using this option and the sink pins specified using the `-to` option. If this option is not specified, only timing arcs whose sink pins are specified using the `-to` option are selected. **Note:** You cannot specify hierarchical pins. If specified, an error message is displayed.
`-increment`	Increments the specified delay value on top of the existing base delay value.
`-min | -max`	Specifies the annotated delay for a particular operating corner. You can apply the `-min` parameter to the minimum operating corner and the `-max` parameter to the maximum operating corner.
`-net | -cell`	Specifies whether the delay should be annotated to cell arcs or net arcs.
`-rise/-fall`	Specifies the transition to which the delay applies. For cell arc, this is the output transition. *Default*: The delay applies to both transitions.
`-to` *`to_pins`*	Specifies timing arcs to which the delay applies. The selected timing arcs are those that have their sink pins specified using this option and the source pins specified using the `-from` option. If this option is not specified, only timing arcs whose source pins are specified using the `-from` option are selected. **Note:** You cannot specify hierarchical pins. If specified, an error message is displayed.

Examples

- The following example shows the usage of `-cell` and `-net` options:

  Example Figure to Show How to Use the -cell and -net Options

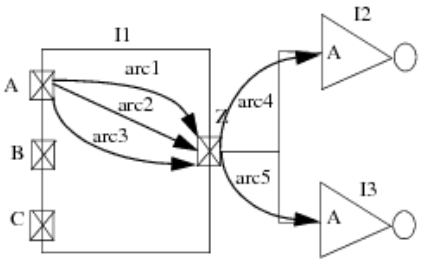
  

  - Use the `-cell` option to select the cell arc `carc2` only:
    set_annotated_delay 3.0 -cell -from {I1/IO}
  - Use the `-net` arc option to select net arcs `narc1` and `narc2` only:
    set_annotated_delay 3.0 -net -from {I1/IO}

- This TLF description is used for the following examples:

```
CELL (NAND3 . . . . // arc 1
Path( A => Z 10 01 DELAY(ioDelayRiseModel0) SLEW(SlopeRiseModel0) ) Path( A => Z 01 10
DELAY(ioDelayFallModel0) SLEW(SlopeFallModel0) ) // arc2 Path( A => Z 10 01 COND(B & C) SDF_COND(B
& C) DELAY(ioDelayRiseModel1) SLEW(SlopeRiseModel1) ) Path( A => Z 01 10 COND(B & C) SDF_COND(B &
C) DELAY(ioDelayFallModel1) SLEW(SlopeFallModel1) ) arc3 Path( A => Z 10 01 COND(~B | ~C)
SDF_COND(!B | !C) DELAY(ioDelayRiseModel2) SLEW(SlopeRiseModel2) ) Path( A => Z 01 10 COND(~B | ~C)
SDF_COND(!B | !C) DELAY(ioDelayFallModel2) SLEW(SlopeFallModel2) ) . . . }
```

  Example Figure to Show How to Use the -from and -to Options

- The following command annotates a delay of `3.0` to arcs `arc1`, `arc2`, and `arc3` for the rise and fall transition at the pin I1/Z, for the two corners `{min typ max}`: `set_annotated_delay 3.0 -from A -to Z -cell`
- The following command annotates a delay of 2.5 to net arc `arc4` for both the rising and the falling transition and for the two corners `{min typ max}`: `set_annotated_delay 2.5 -from I1/Z -to I2/A -net`
- The following command annotates a delay of 1.5 to `arc2` for the rising transition and the min corner:
  `set_annotated_delay 1.5 -from I1/A -to I1/Z -rise -min -cond "B & C" -cell`
- The following commands annotate a delay of 3.0 to `arc1` and `arc2`, and a delay of 2.4 to the `arc3`:

```
set_annotated_delay 3.0 -from I1/A -to I1/Z -cell set_annotated_delay 2.4 -from I1/A -to I1/Z -cond {!B | !C} -cell
```

- The following commands first annotate a delay of 3.0 to `arc1`, `arc2` and `arc3`, then overwrite the value set by the first `set_annotated_delay` command: `set_annotated_delay 2.4 -from I1/A -to I1/Z -cond {!B | !C} -cell set_annotated_delay 3.0 -from I1/A -to I1/Z -cell`

Related Commands

- `reset_annotated_delay`

set_annotated_transition

```
set_annotated_transition
[-rise | -fall]
[-min | -max]
slew_value
pin_list
```

Sets the transition time to be annotated on specified ports or pins in the current design. The transition time that you specify using this command overrides the transition time calculated internally within the software. You can use this command to set the transition time of internal cell instance pins.

Parameters

`-min | -max`	Specifies whether the transition value is set for minimum or maximum transition time. *Default:* Transition value is set for both transition times.
pin_list	Specifies a list of pins or ports to be annotated with the transition time that you specify using the *slew_value* parameter.
`-rise | -fall`	Specifies whether the transition value is set for rising or falling transition time. *Default*: Transition value is set for both transitions.
slew_value	Specifies the transition time to be annotated to the selected pins or ports.

Examples

- The following command annotates a transition time value of 0.6 on port A:
  `set_annotated_transition 0.6 [get_ports A]`
- The following command annotates a transition time value of 0.7 on pin I2/A: `set_annotated_transition 0.7 [get_pins I2/A]`
- The following command annotates a transition value of 0.7 on pin I2/A for rising transition time:
  `set_annotated_transition -rise 0.7 [get_pins I2/A]`
- The following command annotates a transition value of 0.7 on pin I2/A for rising transition time and minimum pvt corner: `set_annotated_transition -rise -min 0.7 [get_pins I2/A]`

Related Commands

- set_input_transition
- set_interactive_constraint_modes

set_aocv_interface_path_offset

```
set_aocv_interface_path_offset
derateOffsetVal
[lib_cells]
[-view view_name]
[-delay_corner dc_corner_name]
[-cell]
[-net]
-power_domain power_domain_name_list
```

You can perform path-based inter power domain (IPD) timing analysis by specifying derate offset value for an entire power domain(s) and specific library cell(s) on paths crossing multiple power domains. This feature is supported for path-based analysis (PBA) mode only.

The `set_aocv_interface_path_offset` command allows you to specify the derate offset value for path segments crossing multiple power domains. The software adds the derate offset value to the AOCV derate value if the combined (AOCV + IPD offset) derate number causes pessimistic PBA timing slack value.

Parameters

`-cell`	It is a boolean option. When the -cell option is specified, intent is to apply “derate offset” only for that cell objects. In absence of these options, derate offset value will be applied to both cell and net type of objects
`-delay_corner dc_corner_name`	Specifies a list of delay corners.
derateOffsetVal	Specifies value of derate offset.
lib_cells	Specifies a list of library cells for a specific interface.
`-net`	It is a boolean option. When the -net option is specified, intent is to apply “derate offset” only for that net objects. In absence of these options, derate offset value will be applied to both cell and net type of objects
`-power_domain power_domain_name_list`	Specifies a list of power domains.
`-view view_name`	Specifies a list of views.

Examples

- The following example specifies the offset derate value of `-0.6` for the power domain `{PD3 PD4}`:
  set_aocv_interface_path_offset -0.6 -power_domain {PD3 PD4} -delay_corner V1_PM1_dc

set_aocv_stage_weight

```
set_aocv_stage_weight
stage_weight
port_list
[-early]
[-late]
[-input]
[-output]
```

While performing advanced on-chip variation (AOCV) analysis of a block in standalone mode, the command allows you to specify the stage weight seen at the port(s) when it is modeled as part of the top level design. The applied stage weights affect the internal weight count of the block for the IO paths. The stage weights can be applied only on ports.

You can view stage weights in the `report_timing` output under the `Stage Weighting` column.

The `set_aocv_stage_weight` command can be used in multi-mode multi-corner mode. The command can be used as part of SDC file.

Parameters

`-early`	Specifies weight for early paths.
`-input`	Specifies input weight for the ports.
`-late`	Specifies weight for late paths.
`-output`	Specifies output weight for port.
`port_list`	Specifies a list or collection of input, output, and bidi ports.
`stage_weight`	Specifies value of stage weight.

Examples

- The following example specifies the stage weight value as `25.5` for port `out0`:

  ```
  set_aocv_stage_weight 25.5 out0
  ```

Related Information

- `reset_aocv_stage_weight`

set_case_analysis

```
set_case_analysis
[-help]
{0 | 1 | zero | one | rising | falling | rise | fall | non_switching}
list_of_ports_or_pins
```

Sets constant or transitional values to a list of pins or ports for use by the timing engine. The specified constants or transitional values are valid only during timing analysis and do not alter the netlist. If there is a conflict between the specified `set_case_analysis` value and the logical value from the netlist, the value from the `set_case_analysis` is used in timing analysis.

Parameters

`0 | 1 | zero | one`	Specifies a logic value of `0` or `1`, which is propagated through combinational logic to disable or enable parts of logic.
`non_switching`	Marks the nets that are in a constant, non-transitioning state. This marking can be used when nets are considered static without the additional impact of logic state injection into the analysis, e.g., when modes are merged into a single constraint mode. The `report_cell_instance_timing` and `report_case_analysis` command reports show non-switching as constant values.
`rising | falling | rise | fall`	Specifies a valid transition value to be considered by the timing engine. For example, if a rising value is specified at a certain pin or port, only the rising signal transition from the pin or port is considered for timing analysis and the opposite transition type (falling) is ignored.
`list_of_ports_or_pins`	Specifies a list of ports or pins on which to apply the constant or transitional value. Specify multiple pins by enclosing them with curly braces `{}`and separating the pin names with white space. The *`list_of_ports_or_pins`* argument can be a collection.

Examples

- The following command sets the value of pin i4/A to `1` for use by the timing engine:
  ```
  set_case_analysis 1 i4/A
  ```
- The following example shows that the pins P1/P2/B and P1/P3/CI are considered only for a rising transition. The falling transition on these pins is ignored:
  ```
  set_case_analysis rising {P1/P2/B P1/P3/CI}
  ```

Related Information

- check_timing -type constant_collision
- report_case_analysis
- report_timing
- set_disable_timing
- set_false_path

set_clock_exclusivity

```
set_clock_exclusivity
[-help]
[-exclude_opposite_polarity]
[-exclude_related_input_pins]
[-exclude_same_polarity]
[-group clock_list]
pin_name
```

Controls filtering of mutually-exclusive clocks at strategic points in the design, e.g., the output of clock multiplexers.

Parameters

`-help`	Prints out the command usage.
`-exclude_opposite_polarity`	Isolates positive and negative phases of the clocks in the same group. By default, the positive and negative phases of the clocks in the same group that arrive at a strategic point will not be isolated from each other.
`-exclude_related_input_pins`	Allows comparison of clocks going through the same input pins of the specified gate.
`-exclude_same_polarity`	Allows comparison of clocks going through the same launch or capture paths.
`-group` *clock_list*	Defines a set of clocks that are exclusive to the clocks defined in all other groups.
pin_name	Specifies the name of the pin.

Examples

- The following command will cause downstream analysis from the mux to isolate interactions between the groups:
  ```
  set_clock_exclusivity -group {CLK1 CLK2} -group {CLK3 CLK4} U1/Y
  ```

  This assertion will cause downstream analysis from pin `U1/Y` to isolate interactions between the groups. The groups `{CLK1 CLK2}` and `{CLK3 CLK4}` are allowed, but all other combinations are considered logically exclusive.

Related Commands

- `reset_clock_gating_check`
- `create_clock`

set_clock_gating_check

```
set_clock_gating_check
[-help]
[object_list]
[-fall]
[-hold hold_value]
[-rise]
[-setup setup_value]
[-high | -low]
```

Specifies or overrides the default setup and hold values for clock gating checks.

Parameters

`-help`	Prints out the command usage.
`-high | -low`	Specifies that the non-controlling value of the clock is high or low. Timing analysis performs the check on the non-controlling value of the clock. By default, the software determines the non-controlling value of the clock using information from the gate's logic. For example, for OR gates the non-controlling value is low. For complex gates, such as XOR and MUX, the clock is never controlling and it does not perform checks unless you specify either the `-high` option or the `-low` option. The specified value takes precedence over the default values. This option can only be used with pins or instances. **Note**: The clock pin value is controlling if it causes the gate output to be a constant (`0` or `1`).
`-hold hold_value`	Specifies the hold value. The default is `0.0`.
`object_list`	Specifies a list of clocks, instances, or pins. If a clock is specified, the check applies to all the clock gating gates driven by this clock. If an instance is specified, the check applies to all input pins of the instance. If the *`object_list`* argument is not specified, the clock gating check is applied to the current design. **Note**: The SDC `set_clock_gating_check` asserts checks on ICG (Integrated Clock Gated) cell only if it is applied on either the cell itself or pins. It does not assert checks on ICG cells if `set_clock_gating_check` is applied on a design or clock object.
`-rise | -fall`	Applies the specified setup (or hold) value to the rising (or falling) signals on the specified pins. If the clock is specified or the *`object_list`* variable is not specified, the value applies to rising or falling, setup or hold. If neither option is specified, the default is both rising and falling.
`-setup setup_value`	Specifies the setup value. The default is `0.0`.

Note: The clock gating constraint is not propagated along the clock tree.

The setup check is performed with respect to the edge of the clock signal that changes the state of the clock pin from controlling to non-controlling. For example, for AND and NAND gates, the setup check is performed with respect to the rising edge of the clock input. For OR and NOR gates, the setup check is performed with respect to falling edge of the clock input.

The hold check is performed with respect to the edge of the clock signal that changes the state of the clock pin from non-controlling to controlling. For example, for AND and NAND gates, the hold check is performed with respect to the falling edge of the clock input. For OR and NOR gates, the hold check is performed with respect to rising edge of the clock input.

If the gate logic in unknown, setup and hold checks are performed with respect to both the rising edge and the falling edge of the clock signal and the timing analyzer uses the worst case values.

Priority Rules for Determining Setup or Hold Values

Timing analysis uses the following priority rules to determine the setup (or hold) value to use for a particular clock gating check:

1. Use the clock gating check assertion on the data pin if it exists
2. Otherwise, use the clock gating check assertion on the clock pin if it exists
3. Otherwise, use the clock gating check assertion on the clock waveform for the clock pin if it exists
4. Otherwise, use the global clock gating check assertion if it exists
5. If no clock gating check assertion is present, use the default setup (or hold) value of `0.0`.

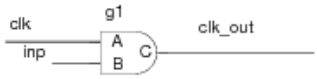
These rules are demonstrated in the examples below.

Note: This command has no effect on a clock gating check defined as a non-sequential setup or hold check in the library and reported as `ClockGatingSetup` or `ClockGatingHold`.

Examples

- The following example shows the priority rules for multiple clock gating check constraints for the gated clock shown in figure below.

Figure: **Gated Clock**



Example: **Tcl Script for set_clock_gating_check**

```
create_clock -name CLK1 -period 12 -waveform {0 6} clk
...
# clock-gating timing check assertions:
# On ideal clock waveform
set_clock_gating_check -setup 0.44 [get_clocks {CLK1}]
# On clock pin
```

```
set_clock_gating_check -hold 0.11 [get_pins {g1/A}]
# On data pin
set_clock_gating_check -setup 0.66 -rise [get_pins {g1/B}]
set_clock_gating_check -hold 0.22 -fall [get_clocks {g1/B}]
```

Given the previous constraints, the following timing margins are used for slack calculation:

- `SETUP g1/B rise: 0.66` -- from assertion on data pin `g1/B` (rule 1)
- `SETUP g1/B fall: 0.44` -- from assertion on ideal clock `CLK1` (rule 3)
- `HOLD g1/B rise: 0.11` -- from assertion on clock pin `g1/A` (rule 2)
- `HOLD g1/B fall: 0.22` -- from assertion on data pin `g1/B` (rule 1)

Related Information

- `create_clock`
- `reset_clock_gating_check`

set_clock_groups

```
set_clock_groups
[-name name_list]
[-comment string]
[-logically_exclusive] | [-physically_exclusive] | [[-asynchronous]
   [-allow_paths]]
[-group clock_list]
```

Defines clock groups with specified clock definitions. The software assigns all clocks that are defined after you use the set_clock_groups command to a default clock group. The software does not apply clock groupings on a master clock to the generated child clocks.

If there is an overlap in the clock group specification, the software uses the following precedence to resolve the behavior:

1. Physically exclusive
2. Asynchronous
3. Logically exclusive

The software drops any explicit false path assertions between clocks that you have set as either physically or logically exclusive or asynchronous using the set_clock_groups command.

An alternative to setting the clocks as physically or logically exclusive using the `set_clock_groups` command is to use the multi-mode mechanism. Generally in an implementation flow, sequential setting of active clocks cannot be used. For concurrent analysis of different modes, you can use an multi-mode based approach to drive IPO instead of using the `set_clock_groups` command.

For more information on multi-mode multi-corner analysis, see *Importing and Exporting Designs* chapter in the *Innovus User Guide*.

Parameters

`-allow_paths`	Allows paths between asynchronous clock groups to be traced in timing analysis. These paths are blocked by default. When you use this option, the software still considers the paths to be asynchronous for signal integrity considerations. Signal integrity analysis does not support the `-allow_paths` parameter. Use the `-allow_paths` parameter with the `-asynchronous` parameter.
`-asynchronous`	Defines clock group with asynchronous clocks. Asynchronous clocks have no specified phase relationship. The software considers asynchronous nets to have infinite timing window overlap for signal integrity considerations and considers these as false paths for timing analysis. Use the `-allow_paths` parameter to trace these paths in timing analysis.
`-comment string`	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.

`-group` *clock_list*	Defines a set of clocks that are exclusive or asynchronous to the clocks defined in all other groups. If you define only one group, then all clocks not in the group implicitly belong to a default clock group. The software automatically adds any new clocks that you define after the `set_clock_groups` command to this default clock group.
`-logically_exclusive`	Defines clock group with clocks that do not have any functional paths between them, but coexist in the design at the same time. The software considers aggressor interactions between these clocks for signal integrity considerations, but considers these as false paths for timing analysis. **Note:** The `-logically_exclusive` parameter does not have any impact on SI analysis. For example, clocks selected via a clock mux select.
`-name` *name_list*	Specifies the names of the clock groups to be added.
`-physically_exclusive`	Defines clock group with clocks that cannot exist in the design at the same time. The software does not consider aggressor interactions between clock groups that are physically exclusive for signal integrity considerations. From the perspective of SI analysis, all attackers that are physically exclusive with the victim net are ignored for both glitch and noise-on-delay analysis. Attacker nets that belong to physically exclusive groups cannot attack together. In this case, all sets of attackers belonging to all the physically exclusive groups are evaluated and the set of nets that has the highest impact on SI analysis is chosen. For timing analysis, the software considers these clock groups as false paths. Example: multiple clocks defined on the same source pin

Examples

- The following commands assert two different clocks on the same clock root port, and uses set_clock_group to isolate them from each other:

```
create_clock -name SYSCLK -period 10 [get_ports sclk]

create_clock -name SCANCLK -period 100 -add [get_ports sclk]

set_clock_groups -name PHY_EX_SCLK -physically_exclusive

-group {SYSCLK} -group {SCANCLK}
```

- The following command declares a single bus clock to be asynchronous to all of the other on board clocks in the design (which remain in the default clock group):

```
set_clock_groups -name ASYNCH_BUS -asynchronous -group {BUSCLK}
```

- The following example shows that the clock group {clkA clkB} is asynchronous only to clock group {clkC clkD}:

```
set_clock_groups -asynchronous -group {clkA clkB} -group {clkC clkD}
```

If an extra clock clkE is defined in the design, then timing check will be allowed in between the first group to clkE

and second
group to clkE.

- In the following example since only one clock group is specified with the command `set_clock_groups`command, so the specified clock group is considered to be asynchronous to all the other clocks in the design:

```
set_clock_groups -asynchronous -group {clkA clkB}

set_clock_groups -asynchronous -group {clkC clkD}
```

Related Information

- `create_clock`
- `reset_clock_groups`

set_clock_latency

```
set_clock_latency
[-source [-early | -late]]
[-rise]
[-fall]
[-jitter jitter_val]
[-clock clock_list ]
[-min]
[-max]
[-clock_gate]
latency
pin_or_clock_list
```

Specifies ideal internal clock latency and external clock arrival delay.

Clock latency has two components:

- Source latency:
  - Specified using the `-source` parameter and is the delay from the external clock generator to the clock port of the design.
  - Applied when the clock is either ideal or propagated.
  - `set_clock_latency` set on a clock port has higher precedence than `set_clock_latency` set on a clock waveform.
  - Specified on clock waveforms with both the `set_input_delay` command and the `set_output_delay` command.
- Network latency (clock tree delay):
  - Default latency type and is the delay from the clock port to the register.
  - Applied only when the clock is ideal. When the clock is propagated, this delay is replaced by the actual clock tree delays.
  - Specified on a clock waveform or a clock port is ignored if the clock is propagated.
  - `set_clock_latency` set on a timing pin has higher precedence than `set_clock_latency` set on a waveform.
  - If there are multiple `set_clock_latency` and `set_propagated_clock` assertions between the clock waveform, clock root and clock endpoint, the closest assertion to the clock endpoint takes precedence.
  - Specified on internal circuit pins or hierarchical ports and used in the fanout cone of the network latency pin, when a clock is ideal.

Use the `set_clock_latency` command in two ways:

- To specify clock latency for a clock waveform:
  - Use the following syntax for source latency associated with a clock signal:
    `set_clock_latency [-source] [-early | -late] [-min ][-max] latency pin_or_clock_list`
  - Use the following syntax for network latency associated with a clock signal:

```
set_clock_latency [-min] [-max] latency pin_or_clock_list
```

You cannot specify the `-late` and `-early` parameters for network latency.

Specifying latency on a port or pin overrides any previous latency specified on the port pin or waveform. The *pin_or_clock_list* argument is the list of the clock waveform names as specified by the create_clock command.

- To specify clock latency on clock pins:
  - Use the following syntax for source delay on a clock port:
    ```
    set_clock_latency -source [-early | -late] [-rise] [-fall]
    insertion_delay pin_or_clock_list
    ```
  - Use the following syntax for network delay on a clock pin:
    ```
    set_clock_latency [-min] [- max] [-rise] [-fall]
    insertion_delay -pin pin_or_clock_list
    ```

When a clock is ideal, the last specification along a path is used. This is useful when you want to intentionally skew the clock tree and model the physical hierarchical clock tree.

Parameters

`-clock` *clock_list*	Associates clock latency constraints, placed at the pin level, with a specific clock. This makes it possible to specify a different latency per clock. **Note:** You cannot use the `-clock` parameter with clock waveform objects.
`-clock_gate`	Allows setting the local clock latency value on pin or port objects. This setting overwrites the existing latency value of clock gating cells and is used when performing timing checks against the specified pins or ports.
`-early | -late`	Specifies clock arrival time with respect to the early or the late time of the clock signal. In setup analysis, launch path is the late path and capture path is the early path. In hold analysis, launch path is the early path and the capture path is the late path. If neither parameter is specified, the default is both early and late. Use these parameters only with the `-source` parameter. You cannot use these parameters with network latency. Specify the clock latency at the given operating corner. **Note:** The `-min` and `-max` parameters refer to the operating corner and the `-early` and `-late` parameters refer to the clock arrival time at the source. Use the `-early` and `-late` parameters with the `-source` parameter to specify the four different clock latencies.
`-jitter` *jitter_val*	Specifies the portion of source clock latency value that can have cycle-to-cycle variation. When calculating CPPR adjustment this portion is removed from CPPR adjustment for different clock edge checks (e.g., setup), but it remains as part of cppr adjustment for same clock edge checks (e.g., hold).

latency	Specifies the value of the latency, in library units, as a floating point.
pin_or_clock_list	Specifies a list of clock waveform names or a list of pins to associate with the clock latency. The pin can be a port or an instance pin. Setting source latency on a pin is ignored.
-max	Specifies the clock latency for a particular operating corner. Apply the -max parameter to the maximum operating corner.
-min	Specifies the clock latency for a particular operating corner. Apply the -min parameter to the minimum operating corner.
-rise/ -fall	Specifies the -rise or -fall parameter to the waveform at register clock pins for ideal latency and to the waveform at the source for source latency. Ideal network latency does not change polarity when crossing negative unate arcs.
-source	Specifies a source latency. If you do not specify the -source parameter, network latency is applied.

Examples

- The following set of commands enable you to create a clock using the create_clock command and apply latency to the created clock using the set_clock_latency command:

```
create_clock CLK2 -period 20 [get_ports {clkA clkB clkD}]

set_clock_latency -source 2 {clkB clkD} # Define port settings
set_clock_latency -rise 5 {clkB clkD} # Define port settings
set_clock_latency -early -late -source CLK1 # Define waveform settings
set_clock_latency 3 -early -late -source CLK1 # Define waveform settings
```

Related Information

- reset_clock_latency
- create_clock
- set_clock_transition
- set_clock_uncertainty
- set_input_delay
- set_output_delay
- set_propagated_clock

set_clock_sense

```
set_clock_sense
[-help]
pin_or_port_list
[-clocks clock_list]
{-positive | -negative | -stop_propagation | -logical_stop_propagation | -stop type_list}
```

Selects which phase of the clock to filter at the specified point.

Parameters

`-help`	Prints out the command usage.
`-clocks clock_list`	Specifies a list of clocks passing through a specified point to which the constraint applies. When this parameter is not specified, the filtering of clock phase will be applicable to all the clocks passing through that specified point. This parameter must be used with a specified pin name and one of the parameters : `-positive`, `-negative`, `-stop_propagation`, `-logical_stop_propagation`, or `-stop`.
`-logical_stop_propagation`	Stops the propagation of specified clocks passing through the specified pins.
`-negative`	Specifies that positive phase filtered clocks pass through pins in the *pin_list*.
pin_or_port_list	Specifies the list or collection of cell output pins. The unateness specified by the set_clock_sense command is applied to clocks propagating through the specified point.
`-positive`	Specifies that negative phase filtered clocks pass through pins in the *pin_list*.
`-stop_propagation`	Stops the propagation of specified clocks and clock source data paths passing through the specified pins.

`-stop` *`type_list`*	Applies clock sense to object types specified in the *`type_list`*. The type list options are - `all`, `clock`, `data`, `clock_source_data`, `default_data`. These are explained below: • `all`: Stops all phases from propagating. This is equivalent to specifying both the `clock` and `data` options. • `clock`: Stops clock phases launched by clock(s) specified in the *`clock_list`* from propagating, and is equivalent to existing `-logical_stop_propagation`. • `data`: Stops all data phases launched by clock(s) specified in *`clock_list`*, starting from the pins/ports specified in *`pin_or_port_list`*. The objects in the *`pin_or_port_list`* can be selected anywhere in the design - clock/data network. • `clock_source_data`: Stops only clock source data phases from propagating. The combination of `clock_source_data` and `clock` is equivalent to the existing `-stop_propagation` option. • `default_data`: Stops only default data phases (representing constrained paths) from propagation. This option cannot be specified along with the *`-clock clock_list`*. The `-stop` parameter extends the capability of the `set_clocks_sense` command to selectively block data timing phases. This parameter provides a more generalized use model for selecting among all types of paths that should be filtered. The `-stop` parameter is mutually exclusive with the `-positive`, `-negative`, `-stop_propagation`, and `-logical_stop_propagation` options.

Examples

- The following command stops the propagation of clock CLK1 through the pin CLKBUF1/Z:
  ```
  set_clock_sense -logical_stop_propagation -clocks CLK1 CLKBUF1/Z
  ```
- The following command allows the positive phase clock CLK1 to pass through CLKBUF1/Z and filters out the negative phase clock CLK1:
  ```
  set_clock_sense -positive -clocks CLK1 CLKBUF1/Z
  ```
- The following command stops clock phases starting from pin1:
  ```
  set_clock_sense pin1 -stop {clock}
  ```
  or
  ```
  set_clock_sense pin1 -logical_stop_propagation
  ```
- The following command stops data phases based only on CK1 and CK2 starting from pin1:
  ```
  set_clock_sense pin1 -stop {data} -clocks {CK1 CK2}
  ```
- The following command stops only the clock source data phases starting from pin1:
  ```
  set_clock_sense pin1 -stop {clock_source_data}
  ```
- The following command stops the clock and clock source data phases:
  ```
  set_clock_sense pin1 -stop {clock clock_source_data}
  ```
  or
  ```
  set_clock_sense pin1 -stop_propagation
  ```

Related Information

- reset_clock_sense

set_clock_transition

```
set_clock_transition
[-rise]
[-fall]
[-min]
[-max]
[-min |-max]
slew_time
clock_list
```

Specifies the slew time of register clock pins. This is useful for pre-layout static timing analysis before clock tree synthesis (CTS) is performed since net delays can be inaccurate with large fanout nets.

Use this command only with ideal clocks. In propagated mode, the software calculates slew times on register clock pins.

Parameters

clock_list	Specifies a list of clocks only with ideal mode in the design, which affects the slew times on all register clock pins in the transitive fanout of the specified clocks. Specified `set_clock_transition` values on register clock pins in the fanout of a clock with propagated latency are ignored.
`-rise | -fall`	Specifies the rising or falling edge of the register clock pins on which the slew time is asserted.
`-min | -max`	Specifies the clock slew time for minimum or maximum process corner slew time.
slew_time	Specifies the slew time (transition time) of the clock pins.

Example

- The following commands specify a `2.0` rise slew time and a `1.0` fall slew time for register clock pins clocked by `CLKA`: set_clock_transition -rise 2.0 [get_clocks CLKA] set_clock_transition -fall 1.0 [get_clocks CLKA]

Related Information

- report_clocks
- set_clock_latency
- set_clock_uncertainty
- set_input_delay
- set_propagated_clock

set_clock_uncertainty

```
set_clock_uncertainty
uncertainty_value
[-setup]
[-hold]
[-half_cycle_jitter | -full_cycle_jitter]
[-rise | -fall]
{  {-from | -rise_from | -fall_from} clksig_from_list
   {-to | -rise_to | -fall_to} clksig_to_list
 | pin_or_clock_list
}
```

Specifies the clock uncertainty (skew) on the clock network.

If you set multiple constraints with the `set_clock_uncertainty` command, the constraints set with the `-from*` *`clksig_from_list`*/`-to*` *`clksig_from_list`* parameters take precedence over simple constraints set on clock objects.

Clock uncertainty set on pins takes highest precedence and overrides clock based uncertainty.

The `set_clock_uncertainty` command can also support clock uncertainty for period, pulse width, skew, and separation checks. To enable this feature, you need to set `timing_enable_uncertainty_for_clock_checks` global variable to `true`. By default, the global is set to `false`.

Parameters

`uncertainty_value`	Specifies the uncertainty value.
`-from | -rise_from | -fall_from` *`clksig_from_list`*	Applies the uncertainty value to the specified list of launching clocks. **Note:** You must specify one of these parameters with one of the `-to`, `-rise_to`, or `-fall_to` parameters. You cannot use these parameters when *`pin_or_clock_list`* is specified.
	`-fall_from`: Applies the uncertainty value only to the falling edge of the launching clocks.
	`-from`: Applies the uncertainty value to both the rising and falling edges of the launching clocks.
	`-rise_from`: Applies the uncertainty value only to the rising edge of the launching clocks.
`-half_cycle_jitter` *`value`* `| -full_cycle_jitter` *`value`*	Specifies the clock source jitter for half-cycle and for full-cycle paths. These parameters can accept different values, that can be specified in the same constraint. **Note**: These parameters can be used on clocks only. You cannot use them when any of the `-from` and `-to` parameter pairs are specified.

pin_or_clock_list	Specifies a list of clocks or pins for target uncertainty. The uncertainty value is applied to the registers clocked by the target waveform, or register pins in the fanout of the specified pin. You can specify a list of clock names, or clock root pin or port names relative to the current module. You also can specify a collection of clocks, or clock root pins or ports. **Note:** You cannot use this parameter when any of the `-from` and `-to` parameter pairs are specified.
`-rise | -fall`	Specifies the rising or falling edge at the capture clock pin on which the uncertainty is applied. *Default*: Applies uncertainty on both edges **Note:** You must use these parameters with the `-to` and `-from` parameters.
`-setup/-hold`	Specifies whether the clock uncertainty applies to setup or hold checks. *Default*: Applies to both setup and hold check
`-to | -rise_to | -fall_to` *clksig_to_list*	Applies the uncertainty value to the specified list of capture clocks. **Note:** You must specify one of these parameters with one of the `-from`, `-rise_from`, or `-fall_from` parameters. You cannot use these parameters if *pin_or_clock_list* is specified.
	`-fall_to`: Applies the uncertainty value only to the falling edge of the capture clocks.
	`-rise_to`: Applies the uncertainty value only to the rising edge of the capture clocks.
	`-to`: Applies the uncertainty value to both the rising and falling edges of the capture clocks.

Examples

- The following command gives all paths starting from registers clocked by `clk1` to registers clocked by the rising edge of `clk2` a hold uncertainty of `2.0`:

  ```
  set_clock_uncertainty 2.0 -from clk1 -to clk2 -rise -hold
  ```

  In case of multiple constraints, the actual uncertainty applied by the software depends on which constraint matches the specific path in question.

- The following command sets an uncertainty on the `clk_in` clock root pin that is applied to all the register pins in the fanout of `clk_in`:

  ```
  set_clock_uncertainty 1.2 clk_in
  ```

- The following command specifies an inter-clock uncertainty of `0.2` between clock waveforms `clk1` and `clk2`:

  ```
  set_clock_uncertainty 0.2 -from clk1 -to clk2
  ```

- The following command specifies a target uncertainty of `0.5` for all registers clocked by the clock waveform

`ideal_clk1`:

```
set_clock_uncertainty 0.5 ideal_clk1
```

Related Information

- `report_clocks`
- `set_clock_latency`
- `set_clock_transition`
- `reset_clock_uncertainty`

set_data_check

```
set_data_check
[-help]
<check_value>
[-clock <clock_object>]
[-hold]
[-setup]
{-from <pin_or_port_list> | -rise_from <pin_or_port_list> | -fall_from <pin_or_port_list>}
{-to <pin_or_port_list> | -rise_to <pin_or_port_list> | -fall_to <pin_or_port_list>}
```

Sets data-to-data checks using the specified values of setup and hold time. Data-to data checks are non-sequential checks that are designed to constrain a data signal with respect to another data signal.

In a normal sequential check, a data signal is constrained with respect to a clock signal. In these cases single-cycle timing is used - allowing one full clock cycle for the data transfer. In a non-sequential data-to-data check, there is no clock reference. Therefore, zero-cycle timing rules are used. You can use the `set_multicycle_path` constraint to adjust the cycle accounting rules.

Parameters

`-clock` *clock_object*	Specifies the name of the clock that triggers the data event for the related (`-from`) pin. In the case where there are multiple data events triggered by different clock phases arriving at the `-from` pin, the `-clock` parameter allows you to specify which data event should be used for the "related pin" side of the timing check.
{`-from` <*pin_or_port_list*> | `-rise_from` <*pin_or_port_list*> | `-fall_from` <*pin_or_port_list*>}	Specifies the pins or ports that are set as related pins for data-to-data checks. The *pin_or_port_list* argument can contain either object IDs or hierarchical names relative to the current module. Use only one `-from`, `-rise_from`, or `-fall_from` parameter per command. By default, the `-from` parameter checks both the rising and the falling edges. Using `-rise_from` parameter checks at the rising edge of the signal on the from pins. Using `-fall_from` parameter checks at the falling edge of the signal on the from pins. **Note:** You cannot use the `-from` parameter to specify multiple objects.
`-setup` | `-hold`	Specifies whether the data check value is for setup or hold check. If you do not specify either `-setup` or `-hold` option, the value applies to both setup and hold.
{`-to` <*pin_or_port_list*> | `-rise_to` <*pin_or_port_list*> | `-fall_to` <*pin_or_port_list*>}	Specifies the pins or ports that are set as constrained pins for data-to-data checks. The *pin_or_port_list* argument can contain either object IDs or hierarchical names relative to the current module. Use only one `-to`, `-rise_to`, or `-fall_to` parameter per command. By default, the `-to` parameter constrains both the rising and the falling edges. Using `-rise_to` parameter constrains the rising edge of the signal on the to pins. Using `-fall_to` parameter constrains the falling edge of the signal on the to pins.
check_value	Specifies the value of the setup or hold time for the check.

Examples

- The following example demonstrates how the `set_data_check` command can be used to constrain signals: Consider two control signals, say XE and SE, going to a macro block. Both of these are derived from the same state machine, and hence launched from the same clock domain. You want XE to reach the macro earlier than SE. The `set_data_check` constraint can meet this design intent, by applying data checks on non-sequential signals, without setting path specific limits using the following command:

  ```
  set_data_check -from [get_pins {sub/SE}] -to [get_pins {sub/XE}] -setup 0.5
  ```

  The `set_data_check` command establishes data-to-data checks using similar ideas of setup and hold time as in sequential checks, with the `-from` parameter denoting the reference signal (the clock in a traditional setup/hold timing check), and the `-to` parameter indicating the signal being constrained.

Related Commands

- `reset_data_check`

set_disable_clock_gating_check

```
set_disable_clock_gating_check
object_list
```

Disables clock gating checks for the specified instances or pins.

The `set_disable_clock_gating_check` command has a character limit for the details that are logged into the SDC file (the `writeTimingCon` command output). This helps to prevent SDC file errors.

Parameters

object_list	Specifies the list of pins or instances that are to be disabled.

Examples

- The following command disables clock gating checks for the specified instance:
  ```
  set_disable_clock_gating_check [get_cells clockgate1]
  ```

Related Commands

- `reset_disable_clock_gating_check`

set_disable_timing

```
set_disable_timing
[-from pin_name -to pin_name]
object_list
```

Disables timing propagation through the specified arcs, or a collection of instance-specific arcs that are created using the `get_arcs` command.

Parameters

`-from pin_name -to pin_name`	Specifies the arc that is to be disabled. Specify the `-from` and `-to` options together. You can specify the `-from` and `-to` parameters when the *`object_list`* contains instance or library cell objects. Do not specify the `-from` and `-to` parameters when the *`object_list`* includes instance pin or library cell pin objects. The `-from` and `-to` options are optional. If not specified, all cell arcs found in the instance or library cell objects are disabled.
`object_list`	Specifies the instance, instance pin, library cell, library cell pin objects, collection of library arcs, or collection of timing arcs. If the *`object_list`* includes instance pin or library cell pin objects, all cell arcs attached to the specified pin objects are disabled. If the *`object_list`* contains instance or library cell objects, and the `-from` and `-to` parameters are not specified, all arcs found in the specified instance or library cell objects are disabled.

Examples

- The following command disables the arcs that lead to the output pins of the `i1`, `i2`, and `i3` instances. Note that timing checks at the inputs of `i1`, `i2`, and `i3` are still active. `set_disable_timing [get_cells {i1 i2 i3}]`
- The following command disables the arc from pin `B` to pin `Z` in the `i1` and `i2` instances, and the arc from pin `A` to pin `Z` in the `i3` instance: `set_disable_timing -from B -to Z [get_cells {i1 i2}]`
  `set_disable_timing -from A -to Z [get_cells {i3}]`
- The following example disables timing check arcs from pin `CLK` to pin `D` of the `ff1` instance: `set_disable_timing -from CLK -to D [get_cells ff1]`
- The following example disables the outgoing cell arcs from instance input pin `i1/B`: `set_disable_timing [get_pins i1/B]`
- The following example disables all arcs found in any instances whose reference cell is `AN2`: `set_disable_timing [get_lib_cells AN2]`

- The following example disables a collection of timing arcs:

```
set arcs [get_lib_arcs -from typical/MX2X1/S0 -to typical/MX2X1/Y]
set_disable_timing [index_collection $arcs 3]
```

- The following example disables arcs created using `filter_collection` on library timing arcs:

```
set arc [filter_collection $arcs {when == "((A)&(!(B)))"}]
set_disable_timing $arc
```

Related Information

- `report_timing`
- `reset_disable_timing`

set_dont_touch

```
set_dont_touch
object_list
[true | false]
```

Prevents the specified object from being modified during optimization. However, the specified object can be moved during optimization.

When the `set_dont_touch` command is set to `true`, the specified nets cannot be buffered or become part of the logic cone restructuring. However, you can resize the instances connected to this net because the `set_dont_touch` attribute does not affect the connected instances.

Parameters

`-help`	Prints out the command usage.
`true | false`	Sets or removes the `set_dont_touch` attribute for the specified *`object_list`*. *Default:* `true`
`object_list`	Specifies the list of instances, nets, library cells, or submodules.

Examples

- The following command prevents the specified instance from being modified:
  ```
  set_dont_touch [get_cells {TWA/FF1}]
  ```
- The following commands can be used to prevent any ECO inside RESULTS_CONV_INST hierarchical instance:
  ```
  set_dont_touch [get_cells RESULTS_CONV_INST/*] true
  set_dont_touch [get_nets RESULTS_CONV_INST/*] true
  ```
- The `set_dont_touch RESULTS_CONV_INST/* true` command will return `[get_cells RESULTS_CONV_INST/*]` only. This is because the software looks into the first match (type of object). If a match is not found, then the successive objects are searched.

Related Commands

- `get_cells`

set_dont_touch_network

```
set_dont_touch_network
[-help]
obj_list
[-no_propagate]
```

Prevents the combinational path connected to the given clocks, pins, or ports from being modified.

Parameters

`-help`	Prints out the command usage.
`object_list`	Specifies the objects - clocks, pins, or ports - to be preserved.
`-no_propagate`	Preserves all the nets driven by the specified pin up to the first leaf-cell input (combinational or sequential). If this parameter is omitted, then preserving stops at the first sequential cell in the network.

Examples

- The following command prevents the combinational path connected to the specified pin from being modified:
  ```
  set_dont_touch_network [get_pins and1/z]
  ```

Related Commands

- `get_pins`

set_dont_use

```
set_dont_use
[-help]
object_list
[true | false]
```

Prevents the specified design, hierarchical module, or library cells from being used during optimization. However, optimization is allowed to size out don't use cells if instances are already there in the initial netlist.

Parameters

object_list	Specifies the list of library cells.
true | false	Sets or removes the set_dont_use attribute for the specified object list. Default: true

Examples

- The following command prevents the specified library cell from being used during optimization:
  ```
  set_dont_use [get_lib_cells {and2}]
  ```

Related Commands

- get_lib_cells

set_drive

```
set_drive
[-rise] [-fall]
[-max] [-min]
resistance_value
port_list
```

Sets the drive resistance on the specified input and/or bidirectional ports of the top cell.

The drive resistance models the cell driving the port. This resistance is used to compute the load dependent delay and the slew at the specified ports. The load-dependent delay is added to the arrival times.

The load-dependent delay and the slew are computed by multiplying the drive resistance by the wire capacitance of the net connected to the port.

Use the report_ports command to view drive resistances asserted on ports.

Parameters

`-fall`	Specifies that the drive resistance is to be used for the falling transition at the specified ports. By default, if neither the `-rise` nor the `-fall` option are specified, the drive resistance applies to both the rising transition and the falling transition.
`-min`	Specifies that the drive resistance is to be used to compute the early path delay. By default, if neither the `-min` option nor the `-max` option is specified, the drive resistance applies to both the early path delay and the late path delay.
`-max`	Specifies that the drive resistance is to be used to compute the late path delay. By default, if neither the `-min` option nor the `-max` option is specified, the drive resistance applies to both the early path delay and the late path delay.
port_list	Specifies a list of input or bidirectional ports on which the drive resistance is set.
resistance_value	Specifies the value of the drive resistance. This value must be greater than or equal to zero. The drive resistance is in units found in the target technology library.
`-rise`	Specifies that the drive resistance is to be used for the rising transitions at the specified ports. By default, if neither the `-rise` option nor the `-fall` option are specified, the drive resistance applies to both the rising transition and the falling transition.

Example

- The following command sets a drive resistance of 2.0 for the early path, the late path, the rising and the falling transitions at the in1 and in2 ports and shows how drive resistances are reported using the report_ports command as shown in the example below:

```
tempus > set_drive 2.0 [get_ports {in1 in2}]
tempus > report_ports -type drive_resistance -pin [get_ports {in1 in2}]
```

  The following report is displayed:

```
-------------------------------------------------------------------
Early Late
-------------------------------------------------------------------
Pin     Dir   Assertion       Clock   Rise   Fall   Rise   Fall
Name                          Name
-------------------------------------------------------------------
in1    IN    drive_resistance *(D)(P) 2.000   2.000  2.000  2.000
in2    IN    drive_resistance *(D)(P) 2.000   2.000  2.000  2.000
-------------------------------------------------------------------
```

Related Commands

- all_inputs
- report_ports
- set_driving_cell
- set_input_transition
- set_load

set_driving_cell

```
set_driving_cell
[-library library_name]
-lib_cell cell_name
[-pin pin_name]
[-from_pin from_pin_name]
[-rise]
[-fall]
[-min]
[-max]
[-multiply_by factor]
[-no_design_rule]
[-input_transition_rise rise_slew_value]
[-input_transition_fall fall_slew_value]
port_list
```

Models the drive capability of an external driver connected to the input port. Use this command only for timing analysis. The command does not affect the electrical properties of the design, which means that the capacitance of the output pin of the driving cell does not add to the total capacitance of the input port.

To model the drive capability of the external driver, the timing analyzer computes an offset to the arrival time of an input, and changes the slew time used to compute the delay for the next cell. The timing analyzer computes the offset by subtracting the zero load delay from the total delay of the cell. The `set_driving_cell` command uses the offset value to identify a cell from the library to drive the specified input port(s).

This command overrides the set_drive command per port or per net, if it is the last command applied. Also, if the `set_driving_cell` command is set for a port, the set_input_transition command is ignored for that port. If you do not define drive using the `set_driving_cell` command, the software uses the default value defined in the configuration file. The default value is `0.1ps`.

Parameters

`-fall`	Annotates only the falling drive assertion. The default is to annotate both rise and fall delays.
`-from_pin` *from_pin_name*	Specifies the name of the input port of the driver cell that has an arc to the output port of the driver cell, which is connected to the input port of the design. This option is required only if there are multiple timing arcs in the driver cell from its input ports to its output ports, and the arcs from those pins have different drive characteristics. If you do not specify the `-from_pin` option, the default is to use the first timing arc found on the library cell.
`-input_transition_fall` *fall_slew_value*	Specifies the slew value for the falling signal at the input of the drive cell.
`-input_transition_rise` *rise_slew_value*	Specifies the slew value for the rising signal at the input of the drive cell.
`-lib_cell` *cell_name*	Specifies the name of the driver cell. This parameter supports wildcard patterns and collections.
`-library` *library_name*	Specifies the name of the library that contains the driving cell.
`-max`	Annotates only the late drive assertion. The default is to annotate both early and late.
`-min`	Annotates only the early drive assertion. The default is to annotate both early and late.
`-multiply_by` *factor*	Specifies a multiplier for the drive.
`-no_design_rule`	Does not allow transfer of design rules from the driving cell to the port. If you do not specify this option, the design rules (such as `max_capacitance`) of the library pin are applied to the port.
`-pin` *pin_name*	Specifies the name of the output port of the driver cell that drives the signal at the input port of the design. This option is required if there are multiple output ports on the driver. If there is only one output port on the driver cell, then using this option is not necessary.
port_list	Specifies the list of ports driven by the driver cell.
`-rise`	Specifies only the rising drive assertion is annotated.The default is to annotate both rise and fall delays.

Examples

The following command sets 'AND' cell (arc B -> Y) as the driving cell for the input port IN1:

```
set_driving_cell -lib_cell AND -from_pin B -pin Y IN1
```

Related Information

- all_inputs
- report_ports
- set_drive
- set_input_transition

set_false_path

```
set_false_path
[-hold | -setup]
[-rise]
[-fall]
{[{-from | -rise_from | -fall_from} from_list]
[{-through | -rise_through | -fall_through} through_list]
[{-to | -rise_to | -fall_to} to_list]}
```

Identifies false paths in a design, and breaks or disables specific instance timing arcs in a design.

To break combinational loops, use the set_disable_timing command instead of the `set_false_path` command.

The `set_false_path` command differs from the `set_disable_timing` command as follows:

- `set_false_path` command selectively disables the arrival time information depending on the precedence rules.
- `set_false_path` command does not affect constant values.
- `set_disable_timing` command physically snips a timing arc such that neither arrival times nor constant values propagate through it.
- `set_disable_timing` command is applied in all cases to valid timing arcs.

The `set_false_path -from fpin` command disables all timing paths whose source pin is an `fpin`. Similarly, the `set_false_path -to tpin` command disables all timing paths whose sink pin is a `tpin`. Using the `set_false_path -from fpin -to tpin` command disables all timing paths whose source is `fpin` and sink is `tpin`.

Specify a group of pins by enclosing the group with curly braces (`{}`) within one `set_false_path` command.

To set a false path for any path beginning at `p1` or `p2` and terminating at `t1` or `t2`, group the pins as follows:

```
set_false_path -from {p1 p2} -to {t1 t2}
```

Note: This method of pin grouping reduces the number of exceptions. Having large numbers of exceptions adversely impacts the run time of the timing analysis commands.

Parameters

`-fall`	Applies the assertion at the falling edge of the signal on the path end points.

`{-from | -rise_from | -fall_from} from_list`	Specifies a list of timing paths start points. The from_list argument can contain either object IDs or hierarchical names relative to the current module. The pins can be on intermediate hierarchical boundaries. The from_list argument can be a collection. The valid start points are: • Input port • Bidirectional port • Clock pin of sequential cell Clock pin of sequential cell is one of the following: • Structural clock pin that is part of latch or flip-flop. • Inferred clock pin in cells like macro cells. A pin is an inferred clock pin if it is associated with one of the following: • Reference end of setup check • Reference end of hold check • Reference end of removal check • Reference end of recovery check • Minimum period check • Minimum pulse width check • Pin associated with set_input_delay Specify one of the following:
	`-from`: Applies the false path constraint to all paths originating from the from_list list. If a clock is specified as the -from object, all primary inputs and registers clocked by that clock are used as start points. By default, the -from option applies the constraint to both the rising and the falling edges.
	`-rise_from`: Applies the false path constraint at the rising edge of the signal on the from pins.
	`-fall_from`: Applies the false path constraint at the falling edge of the signal on the from pins.
`-hold/-setup`	Sets false paths for either setup or hold analysis. Note: If you do not specify -setup or -hold, the software sets false paths for both setup and hold analysis.
`-rise`	Applies the assertion at the rising edge of the signal on the path endpoints.

`{-through | -rise_through | -fall_through} through_list`	Specifies a list of timing paths through points. The through_list argument can contain either object IDs or hierarchical names relative to the current module. The pins can be on intermediate hierarchical boundaries. The through_list argument can be a collection. Specify one of the following:
	`-through`: When you use this parameter without using the -from or -to options, the false path constraint is set for all paths that go through the specified pins. When you use it with both the -from and -to options, any path starting at any pin on the -from list and going through any point on the -through pin list and going to any point on the -to list is affected by the constraint. By default, the -through option applies the constraint to both the rising and the falling edges.
	`-rise_through`: Applies the assertion at the rising edge of the signal on the through pins.
	`-fall_through`: Applies the assertion at the falling edge of the signal on the through pins.
`{-to| -rise_to | -fall_to} to_list`	Specifies a list of timing paths end points. The to_list argument can contain either object IDs or hierarchical names relative to the current module. The pins can be on intermediate hierarchical boundaries. The to_list argument can be a collection. The valid end points are: • Output port • Bidirectional port • Data pin of sequential cell Data pin of sequential cell is one of the following: • Structural data pin that is part of latch or flip-flop. • Inferred data pin in cells like macro cells. A pin is an inferred data pin if it is associated with one of the following: • Signal end of setup check • Signal end of hold check • Signal end of non-sequential setup check • Signal end of non-sequential hold check • Signal end of removal check • Signal end of recovery check • Pin associated with set_external_delay • Data pin of clock gating cell Select from one of the following:

	`-to`: Applies the false path constraint to all paths ending in the -to pin list. If a clock is specified as the -to object, all primary outputs and registers clocked by that clock are used as endpoint. By default, the -to option applies the constraint to both the rising and the falling edges.
	`-rise_to`: Applies the false path constraint at the rising edge of the signal on the to pins.
	`-fall_to`: Applies the false path constraint at the falling edge of the signal on the to pins.

Examples

- The following command disables hold checking from `ff1` to `ff2`:

  ```
  set_false_path -hold -from ff1 -to ff2
  ```

- The following command sets all timing paths from `ff1/CK` to `D` input of `ff2` that pass through `u1/Y` and one or more of `u2/Y` and `u3/Y` as false paths:

  ```
  set_false_path -from ff1/CK -through {u1/Y} -through {u2/Y u3/Y} -to ff2/D
  ```

- The following command sets all timing paths from the rise of `ff1/CK` to `ff2/D` through one or more of `u1/Y` and `u4/Y` as false paths:

  ```
  set_false_path -rise_from ff1/CK -through {u1/Y u4/Y} -to ff2/D
  ```

Related Information

- `set_disable_timing`
- `set_max_delay`
- `set_min_delay`
- `set_multicycle_path`
- `reset_path_exception`

set_fanout_load

```
set_fanout_load
[-help]
fanout_load
port_list
```

Specifies the fanout load on a set of ports.

The library instance pins can have fanout loads defined in the technology library. The fanout does not have specific units so it is assumed that a fanout as used by the constraints is consistent with the fanout defined in the technology libraries. If a pin or port has no fanout attribute, either from the constraints or from the technology library, then the fanout load will be zero. The fanout load is used only for design rule checking and does not impact the timing results.

Parameters

`-help`	Prints out the command usage.
`fanout_load`	Specifies the fanout load value for the port or ports.
`port_list`	Specifies the list of ports for which the fanout load is applied.

Examples

- The following command sets the fanout load of port `out` to `2`: `set_fanout_load 2 out`
- The following command sets the fanout load of ports `in1` and `in2` to `1`: `set_fanout_load 1 {in1 in2}`
- The following command sets the fanout load of element `3` of port `out` to `12`: `set_fanout_load 12 { out[3] }`
- The following command sets the fanout load of all the ports to `2`: `set_fanout_load 2 [get_ports *]`

Related Commands

- `report_net`
- `report_ports`
- `set_max_fanout`
- `report_constraint`

set_ideal_latency

```
set_ideal_latency
[-help]
[-min]
[-max]
[-rise]
[-fall]
latency_value
object_list
```

Specifies the ideal latency to use for input or output leaf-cell pins and top-level ports that are part of an ideal network. These set a point delay on the specified object.

Note: You should specify `set_ideal_latency` with the `set_ideal_network` command to view the ideal latency result.

Parameters

`-help`	Prints out the command usage.
`-fall`	Specifies the ideal latency for the falling edge.
`latency_value`	Specifies the ideal latency value.
`-max`	Specifies the ideal latency for the maximum corner.
`-min`	Specifies the ideal latency for the minimum corner.
`object_list`	Specifies a list of objects - instance pins or ports.
`-rise`	Specifies the ideal latency for the rising edge.

Examples

- The following command adds an ideal latency of 1.5ns to the input pin A of cell FE_OFC413_scan_se:
  `set_ideal_latency 1.5 FE_OFC413_scan_se/A`

  Please note that the above command will only be valid if the pin is part of an ideal network.

  Now, this ideal latency value can be observed in the output of the following `report_timing` command:
  `report_timing -though FE_OFC413_scan_se/A`

```
Path 1: MET Setup Check with Pin RESULTS_CONV_INST/r697_reg[14]/CK
Endpoint: RESULTS_CONV_INST/r697_reg[14]/SE (v) checked with trailing edge of 'm_rcc_clk'
Beginpoint: scan_se (v) triggered by leading edge of 'm_ram_clk'
Path Groups: {m_rcc_clk}
Analysis View: scan_fast_RCMIN
Other End Arrival Time          20.679
- Setup                         1.123
+ Phase Shift                   0.000
+ Cycle Adjustment             40.000
- Uncertainty                   0.400
= Required Time                59.156
- Arrival Time                  8.000
= Slack Time                   51.156
Clock Rise Edge                      0.000
+ Input Delay                        4.000
+ Ideal Latency                      1.500
= Beginpoint Arrival Time            5.500
-------------------------------------------------------------
Pin                      Cell   Load    Slew   Delay Arc
                                                  Annotation
-------------------------------------------------------------
scan_se                         0.000   0.000          Ideal
FE_OFC413_scan_se/Z  BUF_X32    0.000   0.000  0.000   Ideal
FE_OFC247_scan_se/A  BUF_X16    0.000   0.000  0.000   Ideal
FE_OFC247_scan_se/Z  BUF_X16    0.000   0.000  0.000   Ideal
FE_OFC248_scan_se/A  BUF_X16    0.000   0.000  0.000   Ideal
FE_OFC248_scan_se/Z  BUF_X16    0.000   0.000  0.000   Ideal
FE_OFC249_scan_se/A  BUF_X8     0.000   0.000  0.000   Ideal
-------------------------------------------------------------
```

Related Commands

- reset_ideal_latency
- report_timing
- set_ideal_network

set_ideal_network

```
set_ideal_network
object_list
[-no_propagate]
```

Identifies driver pins or ports as sources of an ideal network. By default, all pins and nets in the transitive fanout of the specified port or driver pin are marked as ideal. Ideal network marking normally propagates through combinational cells and stops at sequential cells.

Note: This command is not view/mode-based - it applies to all the active views.

The `set_ideal_network` command sets 0 slew for bi-directional pins. This ensures that unnecessary DRV violations do not occur.

Parameters

`-no_propagate`	Prevents propagation through transitive fanout.
`object_list`	Specifies a list of objects - instance pins, ports, or nets.

Examples

- The following command marks the fanout of the input port scan_se as an ideal network:

```
> set_ideal_network scan_se
```

  This can be observed in the output of below mentioned report_timing command where the whole fanout of 'scan_se' is considered as ideal :

```
> report_timing -from scan_se
```

```
Path 1: MET Setup Check with Pin RESULTS_CONV_INST/r697_reg[12]/CK
Endpoint: RESULTS_CONV_INST/r697_reg[12]/SE (v) checked with trailing edge of 'm_rcc_clk'
Beginpoint: scan_se (v) triggered by leading edge of 'm_ram_clk'
Path Groups: {m_rcc_clk}
Analysis View: scan_fast_RCMIN
Other End Arrival Time 20.700
- Setup 1.122
+ Phase Shift 0.000
+ Cycle Adjustment 40.000
- Uncertainty 0.400
= Required Time 59.178
- Arrival Time 4.000
= Slack Time 55.178
Clock Rise Edge 0.000
+ Input Delay 4.000
= Beginpoint Arrival Time 4.000
-------------------------------------------------------------------
Pin                  Cell      Load   Slew   Delay     Arc
                                                      Annotation
-------------------------------------------------------------------
scan_se ->                     0.000  0.000            Ideal
FE_OFC413_scan_se/Z  BUF_X32   0.000  0.000  0.000     Ideal
FE_OFC247_scan_se/Z  BUF_X16   0.000  0.000  0.000     Ideal
FE_OFC248_scan_se/Z  BUF_X16   0.000  0.000  0.000     Ideal
FE_OFC249_scan_se/Z  BUF_X8    0.000  0.000  0.000     Ideal
FE_OFC250_scan_se/Z  BUF_X16   0.000  0.000  0.000     Ideal
FE_OFC251_scan_se/Z  BUF_X4    0.000  0.000  0.000     Ideal
RESULTS_CONV_INST/r697_reg[12]/SE
                     SDFF_X2   0.000  0.000 0.000 Ideal
-------------------------------------------------------------
```

Related Commands

- reset_ideal_network

set_ideal_transition

```
set_ideal_transition
[-min]
[-max]
[-rise]
[-fall]
transition_time
object_list
```

Specifies the ideal transition to use for input or output leaf-cell pins and top-level ports that are part of an ideal network. These set a point delay on the specified object.

Note: You should specify `set_ideal_transition` with `set_ideal_network` to view ideal transition result.

Parameters

`-fall`	Specifies the ideal transition for the falling edge.
`-max`	Specifies the ideal transition for the maximum corner.
`-min`	Specifies the ideal transition for the minimum corner.
`object_list`	Specifies a list of objects - instance pins or ports.
`-rise`	Specifies the ideal transition for the rising edge.
`transition_time`	Specifies the ideal transition time.

Examples

- The following command specifies the ideal transition of 0.5ns on pin scan_se:

```
> set_ideal_transition 0.5 scan_se
```

  Please note that this ideal transition would only be applicable if the pin is part of an ideal network, otherwise this value would be ignored.
  This transition will be propagated through the ideal network as shown in the output of the following `report_timing` command:

```
> report_timing -from scan_se
```

```
Path 1: MET Setup Check with Pin RESULTS_CONV_INST/r697_reg[14]/CK
Endpoint: RESULTS_CONV_INST/r697_reg[14]/SE (v) checked with trailing edge of 'm_rcc_clk'
Beginpoint: scan_se (v) triggered by leading edge of 'm_ram_clk'
Path Groups: {m_rcc_clk}
Analysis View: scan_fast_RCMIN
Other End Arrival Time 20.699
- Setup 1.277
+ Phase Shift 0.000
+ Cycle Adjustment 40.000
- Uncertainty 0.400
= Required Time 59.022
- Arrival Time 4.000
= Slack Time 55.022
Clock Rise Edge 0.000
+ Input Delay 4.000
= Beginpoint Arrival Time 4.000
-------------------------------------------------------------
Pin                   Cell      Load     Slew Delay     Arc
                                                        Annotation
-------------------------------------------------------------
scan_se ->                       0.000   0.500          Ideal
FE_OFC413_scan_se/Z   BUF_X32    0.000   0.500  0.000   Ideal
FE_OFC247_scan_se/Z   BUF_X16    0.000   0.500  0.000   Ideal
FE_OFC248_scan_se/Z   BUF_X16    0.000   0.500  0.000   Ideal
FE_OFC249_scan_se/Z   BUF_X8     0.000   0.500  0.000   Ideal
FE_OFC250_scan_se/Z   BUF_X16    0.000   0.500  0.000   Ideal
FE_OFC251_scan_se/Z   BUF_X4     0.000   0.500  0.000   Ideal
RESULTS_CONV_INST/r697_reg[14]/SE
                      SDFF_X2    0.000   0.500  0.000   Ideal
-------------------------------------------------------------
```

Related Information

- `reset_ideal_transition`

- set_ideal_network

set_input_delay

```
set_input_delay
[-clock clock_name]
[-clock_fall]
[-rise] [-fall]
[-max] [-min]
[-add_delay]
[-network_latency_included]
[-source_latency_included]
[-reference_pin pin_name]
[-level_sensitive]
delay_value
port_or_pin_list
```

Defines the arrival time relative to a clock edge on input ports or internal input pins. This input path delay models the delay from an external register to an input port of the module.

If the input delay is not specified on an input port, it is assumed to be zero.

Parameters

`-add_delay`	Adds delay information to the existing input delay. The additional delay information can be for multiple paths relative to different clocks or clock edges leading to an input port or internal input pins. If you do not use the `-add_delay` option, the software uses the last `set_input_delay` related to a pin when there are multiple `set_input_delay` statements for the same pin.
`-clock` *clock_name*	Specifies that the delay is relative to the rising edge of the given clock. If you do not specify the `-clock` parameter, the delay is relative to time zero. To specify a delay relative to the falling edge of the clock, use the -clock_fall option with the -clock option. When you specify multiple clocks, an error occurs and the constraint is ignored.
`-clock_fall`	Specifies that the delay is relative to the falling edge of the clock. You must specify the -clock parameter with this parameter. *Default:* Delay is relative to the rising edge. You cannot specify the `-clock_fall` parameter without the `-clock` parameter.
delay_value	Specifies the value for the path delay.
`-fall`	Specifies that the input delay refers to the falling edge of the signal at specified ports or pins. If you do not specify the `-fall` option, the input delay applies to both the edges.

`-level_sensitive`	Models input delay that is launched by a positive level sensitive latch. Use the `-clock` parameter to model a positive level sensitive latch, or the `-clock_fall` parameter to model a negative level sensitive latch. You cannot specify the `-level_sensitive` parameter without the `-clock` parameter.
`-max`	Specifies that the delay refers to the setup analysis. If you do not specify the `-max` parameter, the input delay applies to both setup and hold analysis. **Note**: If for a specific clock the maximum delay is less than the minimum delay, then during analysis the software makes the maximum delay equal to the minimum delay.
`-min`	Specifies that the delay refers to the hold analysis. If you do not specify the `-min` parameter, the input delay applies to both setup and hold analysis.
`-network_latency_included`	Specifies that the clock network latency should not be added to the input delay value. If you do not specify this option, the clock network latency of the related clock is added to the input delay value. This parameter has no effect if the specified clock is in propagated mode.
port_or_pin_list	Specifies a list of input port or pin names. The port_or_pin_list argument can be a collection.
`-reference_pin` *pin_name*	Adds source and network latency of the specified reference pin to the input delay value in propagated mode. A reference pin is a leaf pin in the fanout cone of the clock source of the clock that you specify using the `-clock` parameter. You cannot use the `-reference_pin` parameter to specify multiple pins. If specified, the command issues a warning and accepts the first pin. You cannot use the `-reference_pin` parameter with the `-network_latency_included` or `-source_latency_included` parameters. You cannot use the `-reference_pin` parameter to specify hierarchical pins.
`-rise`	Specifies that input delay refers to the rising edge of the signal at the specified ports or pins. If you do not specify the `-rise` option, the input delay applies to the both edges.
`-source_latency_included`	Specifies that the clock source latency should not be added to the input delay value. With different delay values for `-rise/-fall` and `-min/-max`, only the last specified combination of `-network_latency_included` and `-source_latency_included` parameters is honored. The specified combination can be one of the following: a) Either `-network_latency_included` or `-source_latency_included` b) Both of these c) None of these

Examples

- The following command sets an input delay of 1.0 for all inputs in the current design relative to the rising edge of the `CLK1` clock:

  ```
  set_input_delay -clock CLK1 1.0 [all_inputs]
  ```

- The following command sets an input delay of 2.0 for the input `IN1` port relative to the falling edge of the `CLK1` clock: `set_input_delay -clock CLK1 2.0 -clock_fall { IN1 }`
- The following example shows an input `IN1` port that is constrained by both the rising edge and the falling edge of the same `CLK1` clock. The `-add_delay` option prevents the second `set_input_delay` statement from overwriting the first `set_input_delay`:

  ```
  set_input_delay -clock CLK1 1.0 { IN1 }
  set_input_delay -clock CLK1 -clock_fall -add_delay 2.0 { IN1 }
  ```

- When multiple `-min` and `-max` values are specified relative to the same clock using the `-add_delay` option at the same port or pin, the worst case value for each case is used. For the `-min` case the smallest value is retained, and for the `-max` case the largest value is retained. For example, the input delay value of 1.0 is stored for the shortest path and the value of 4.0 is stored for the longest path in the following example:

  ```
  set_input_delay -min -clock CLK1 1.0 { IN1 }

  set_input_delay -min -clock CLK1 2.0 -add_delay { IN1 }

  set_input_delay -max -clock CLK1 3.0 -add_delay { IN1 }

  set_input_delay -max -clock CLK1 4.0 -add_delay { IN1 }
  ```

Related Information

- `all_inputs`
- `create_clock`
- `report_ports`
- `set_load`
- `set_output_delay`

set_input_transition

```
set_input_transition
[-help]
[-min]
[-max]
[-rise]
[-fall]
transition_time
port_list
```

Specifies the slew time for a port on the top level module. This command is useful only in specifying a hard ramp signal as if the port were driven by a very powerful driver. The command does not change the arrival time of an input, but changes the slew time used to compute the delay of the cell on the sink of a net. If you do not define transition using the `set_input_transition` command, the software uses the default input port transition defined in the configuration file. The default value is `0.1ps`. You can also set the `delaycal_input_transition_delay global variable` to override the default value.

The `set_input_transition` command specification for clock signals does not impact setup/hold timing calculations.

Note: The `set_input_transition` command does not support internal pins. For specifying transition on internal pins use the `Set_annotated_transition` command.

Note: The most recent command settings (of `set_driving_cell`, `set_input_transition`, or `set_drive`) that are applied to a port will be considered - the software will override the previous commands.

Parameters

`-help`	Prints out the command usage.
`-fall | -rise`	Specifies that the slew time should be applied to the rising edge or the falling edge of the signal (data or clock). If neither time is specified, the default is to use both.
`-min | -max`	Indicates whether the slew times are with respect to the early (hold) or the late (setup) time checks of the data signal. If neither option is specified, the default is to use both.
`port_list`	Specifies the list of ports for which the slew time is specified. **Note:** You cannot use the `set_input_transition` command to specify transition on output ports. **Note:** When you use both `set_input_transition` and `set_annotated_transition` commands for specifying transition on input ports, the worst value is used as the transition value.
`transition_time`	Specifies the transition time.

Examples

- The following command specifies the slew time of `0.5` on port `input3`.

  ```
  set_input_transition 0.5 input3
  ```

- The following command specifies the input falling slew of 0.6ns on the port input1 for minimum delay type:

  ```
  set_input_transition -min -fall 0.6 input1
  ```

Related Commands

- report_ports
- set_driving_cell
- set_drive
- set_clock_transition

set_load

```
set_load
[-max] [-min]
[-pin_load]
[-wire_load]
[-subtract_pin_load]
capacitance_value
object_list
```

Sets the specified capacitance on the defined ports (or nets) of the top cell.

The specified capacitance overrides the capacitance from the wire load models and SPEF.

You can use the `report_ports` command to view capacitance of asserted ports. The pin load appears in the report as `port_cap` in the assertion column and the wire load appears as `port_wire_cap` in the assertion column.

In the timing report (generated using the `report_timing` command), the load value for ports is displayed as a sum of the `set_load` value on a port and the existing wire capacitance obtained from the SPEF file, and pin capacitance obtained from the library files.

Parameters

capacitance_value	Specifies the value for the capacitance. The capacitance is in units from the target technology library.
-min	Specifies the minimum capacitance. The minimum capacitance is used to compute best-case delays in BcWc analysis mode and early path delays in OCV analysis mode.
-max	Specifies the maximum capacitance. The maximum capacitance is used to compute worst-case delays in BcWc analysis mode and late path delays in OCV mode. It is also used to check the maximum capacitance design rule.
object_list	Specifies a list of ports or nets in the top cell.
-pin_load	Specifies the external pin capacitance for a port. You can use this parameter for ports only. For ports, the asserted capacitance is either the external pin capacitance when the `-pin_load` parameter is specified, or the external wire capacitance when the `-wire_load` parameter is specified, or the total external capacitance when neither option is specified.

`-subtract_pin_load`	This parameter can be used only if nets are specified in the object list. When this parameter is specified, the total net capacitance value will be the same as set by the `set_load` *capacitance_value object_list* command. When this parameter is not specified, the asserted load value will only change the wire capacitance value from the SPEF. The total net capacitance value will be the `set_load` capacitance value plus the pin capacitance from the library file.
`-wire_load`	Interprets the specified capacitance as the external wire capacitance of the port. Use this option only for ports. Use this option with the *`port_net_list`* argument if the list contains only ports.

Examples

- The following command sets an external minimum pin capacitance of 1.5 units on the `in1 in2` ports: `set_load 1.5 -min -pin_load [get_ports {in1 in2}]`
- The following command sets an external minimum wire capacitance of 1.2 units on the `in1 in2` ports: `set_load 1.2 -min -wire_load [get_ports {in1 in2}]`
- The following command sets an external maximum pin capacitance of 2.6 units on the `in1 in2` ports: `set_load 2.6 -max -pin_load [get_ports {in1 in2}]`
- The following command sets an external maximum wire capacitance of 3.0 units on the `in1 in2` ports: `set_load 3.0 -max -wire_load [get_ports {in1 in2}]`
- The following command reports the capacitances set on the ports, as shown in example below:
  `report_ports -pin [get_ports {in1}]`

```
-------------------------------------------------------------
Early Late
-------------------------------------------------------------
Pin   Dir Assertion       Clock   Rise   Fall  Rise  Fall
Name                      Name
-------------------------------------------------------------
in1   IN drive_resistance *(D)(P) 2.000  2.000  2.000  2.000
-------------------------------------------------------------
-----------------------------------------------------
Pin    Dir   Assertion     Value
Name
-----------------------------------------------------
in1   IN    port_cap      ( 1.500 : : 2.600 )
in1   IN    port_wire_cap ( 1.200 : : 3.000 )
-----------------------------------------------------
```

Related Information

- all_outputs
- report_ports
- report_timing
- set_drive
- set_resistance

set_logic_one

```
set_logic_one
-help
ports_pins
```

Sets ports or pins to logic 1 in the design.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_logic_one` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_logic_one`.
`ports_pins`	Specifies the list of the ports or pins that are to be set to logic 1.

Examples

- The following command sets the specified port to logic 1:
  ```
  set_logic_one [get_ports {din[1]}]
  ```

Related Commands

- `get_ports`

set_logic_zero

```
set_logic_zero
-help
ports_pins
```

Sets ports or pins to logic 0 in the design.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_logic_zero` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_logic_zero`.
`ports_pins`	Specifies the list of the ports or pins that are to be set to logic 0.

Examples

- The following command sets the specified port to logic 0:
  ```
  set_logic_zero [get_ports {din[0]}]
  ```

Related Commands

- `get_ports`

set_max_capacitance

```
set_max_capacitance
[-help]
capacitance_limit
object_list
[-clock_path]
[-data_path]
[-fall]
[-override]
[-rise]
```

Sets the maximum capacitance limit on the specified instance pin, ports of the top cell, the specified designs, and/or clock waveforms. The pins of cell instances of timing libraries will have their own maximum constraint. The software will pick the more aggressive value of the two.

When set on a port, the total capacitance of the net connected to that port must be less than or equal to the specified limit. Maximum capacitance limits also come from library objects, which is the default maximum capacitance limit of libraries and the maximum capacitance limit on library cell pins. Pins in the design inherit the limits from libraries.

Note: The limit set on the top cell applies to its ports. Thus, if maximum capacitance limits are set on the top cell and its ports, the most constraining limit will apply to the ports.

When limits are set on a clock waveform, you can use `-clock_path`, `-data_path`, `-rise`, and `-fall` parameters to provide more specifications.

The `set_max_capacitance` constraint on clock objects will be honored only by the `report_constraint` command for DRV violations. The constraint will not be used in optimizations.

Parameters

`-help`	Prints out the command usage.
`-clock_path`	Applies the max capacitance constraint only to pins in the clock network of the clocks defined in the object list.
`-data_path`	Applies the max capacitance constraint only to data path pins, where the path is launched by the clocks defined in the object list.
`-fall`	Applies the max capacitance constraint only to falling capacitance.
`-rise`	Applies the max capacitance constraint only to rising capacitance.
`capacitance_limit`	Specifies the value for the minimum capacitance limit. The limit must be greater than or equal to zero. The limit is in capacitance units from the target technology library.
`object_list`	Specifies a list of objects - ports, instance pins, library cell pins, designs, and/or clock waveforms.

`-override`	Overrides the maximum capacitance limit from other sources, such as, library or constraints. This parameter can be used to force a specific constraint value for a specified list of instance pins/port/library cell pins - this overrides the maximum/minimum values set prior to this setting. To enable this feature, make the following setting: `set timing_constraint_enable_drv_limit_override true`

Examples

- The following example sets a maximum capacitance limit of `1.2` units on all the ports of the top cell and a maximum capacitance limit of `0.5` units on the port named `d_out`:

```
set_max_capacitance 1.2 [get_ports *]
set_max_capacitance 0.5 d_out
```

- The following example uses the `report_ports` command to show the maximum capacitance limit set on ports, as shown in the example below:

```
set_max_capacitance 0.9 [get_ports *] report_ports -type port_cap_limit -pin [get_ports *]
```

Example: **Maximum Capacitance Limit Set on Ports Report**

```
> report_ports -type port_cap_limit -pin [get_ports *]

----------------------------------------
Pin     Dir  Assertion       Value
Name
----------------------------------------
CLK2    IN   port_cap_limit  ( : 0.900 )
CLK1    IN   port_cap_limit  ( : 0.900 )
in2     IN   port_cap_limit  ( : 0.900 )
in1     IN   port_cap_limit  ( : 0.900 )
out2    OUT  port_cap_limit  ( : 0.900 )
out1    OUT  port_cap_limit  ( : 0.900 )
----------------------------------------
```

Related Commands

- `get_ports`
- `report_ports`
- `set_max_transition`
- `set_max_fanout`
- `report_constraint`

set_max_delay

```
set_max_delay
value
[-combinational_from_to | -ignore_clock_latency]
[{-from |-rise_from | -fall_from} from_list]
[{-through | -rise_through | -fall_through} through_list]
[{-to | -rise_to | -fall_to} to_list]
[-comment string]
[-rise]
[-fall]
```

Specifies a maximum delay for a timing path. The `set_max_delay` command takes precedence over other path exceptions specified for the same path, except for the `set_false_path` command.

Use either the `-from`, `-to`, or `-through` option with the `set_max_delay` command to identify the paths. Specifying both the `-from` and `-to` options identifies a path with specific starting and ending points.

When you specify a sequential cell name for starting point of the timing path, the valid path starts from the clk pin. This means the latency and delay from clk to output Q is also included in the delay. For path segmentation, you can specify the path to start from Q output of the register by adding /Q after the cell name. In this case, path from clk to Q is not considered in the calculation.

Use only one `-from`, `-rise_from`, or `-fall_from`, `-to`, `-rise_to`, or `-fall_to` parameter per command. You can use multiple `-through` parameters in the same command.

Parameters

`-combinational_from_to`	Defines a path defined by -from/to pins as combinational. When this parameter is used, the path between the -`from`/-`to` pins is treated as purely combinational, that is, both the source/capture clock latencies are excluded from slack computation. The presence of -`from`/`-to` pins is required for this option to take effect; the -`from`/`-to` pins can be sequential or combinational start/end pins. The `-combinational_from_to` parameter will be deprecated in a future release. This parameter will continue to work in the current release, but for compatibility with future releases you should migrate your scripts to use the `-ignore_clock_latency` parameter. You can use the following global variables to match the semantics between the `-ignore_clock_latency` and `-combinational_from_to` parameters: • `timing_constraint_path_delay_exclude_check_delay_from_ignore_clock_latency` • `timing_constraint_path_delay_exclude_io_delay_from_ignore_clock_latency`
`-comment string`	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.

`-fall`	Apply the assertion at the falling edge of the signal on the path endpoints.
`-from |-rise_from |-fall_from` *from_list*	Specifies a list of timing paths start points. The *from_list* argument can contain either object IDs or hierarchical names relative to the current module or clock names. The pins can be on intermediate hierarchical boundaries. The *from_list* argument can be a collection. The valid start points are: • Input port • Bidirectional port • Clock pin of sequential cell Clock pin of sequential cell is one of the following: • Structural clock pin that is part of latch or flip-flop. • Inferred clock pin in cells like macro cells. A pin is an inferred clock pin if it is associated with one of the following: • Reference end of setup check • Reference end of hold check • Reference end of removal check • Reference end of recovery check • Minimum period check • Minimum pulse width check • Pin associated with `set_input_delay` Specify one of the following:
	`-from`: Applies the maximum delay constraint to all paths originating from the *from_list* list. If a clock is specified as the `-from` object, all primary inputs and registers clocked by that clock are used as start points. By default, the `-from` option applies the constraint to both the rising and the falling edges.
	`-rise_from`: Applies the maximum delay constraint at the rising edge of the signal on the from pins.
	`-fall_from`: Applies the maximum delay constraint at the falling edge of the signal on the from pins.

`-ignore_clock_latency`	When this parameter is specified, the software does not count the launch/capture clock latency delays into the timing path delay. The `-ignore_clock_latency` parameter is an industry-standard construct introduced in SDC 2.1 to control the handling of clock-related timing with respect to the `set_max_delay`/`set_min_delay` constraints.
`-rise`	Applies the assertion at the rising edge of the signal on the path endpoints.
`-through | -rise_through | -fall_through` *`through_list`*	Specifies a list of timing paths through points. The *`through_list`* argument can contain either object IDs or hierarchical names relative to the current module or clock names. The pins can be on intermediate hierarchical boundaries. The *`through_list`* argument can be a collection. Specify one of the following:
	`-through`: When you use this parameter without using the `-from` or `-to` options, all paths that go through the specified pins have the specified delay. When you use it with both the `-from` and `-to` options, any path starting at any pin on the `-from` list *and* going through any point on the `-through` pin list *and* going to any point on the `-to` list is affected by the constraint. By default, the `-through` option applies the constraint to both the rising and the falling edges.
	`-rise_through`: Applies the assertion at the rising edge of the signal on the through pins.
	`-fall_through`: Applies the assertion at the falling edge of the signal on the through pins.

`-to| -rise_to | -fall_to` *to_list*	Specifies a list of timing paths end points. The *to_list* argument can contain either object IDs or hierarchical names relative to the current module or clock names. The pins can be on intermediate hierarchical boundaries. The *to_list* argument can be a collection. The valid end points are: • Output port • Bidirectional port • Data pin of sequential cell Data pin of sequential cell is one of the following: • Structural data pin that is part of latch or flip-flop. • Inferred data pin in cells like macro cells. A pin is an inferred data pin if it is associated with one of the following: • Signal end of setup check • Signal end of hold check • Signal end of non-sequential setup check • Signal end of non-sequential hold check • Signal end of removal check • Signal end of recovery check • Pin associated with `set_external_delay` • Data pin of clock gating cell Select from one of the following:
	`-to`: Applies the maximum delay constraint to all paths ending in the `-to` pin list. If a clock is specified as the `-to` object, all primary outputs and registers clocked by that clock are used as endpoint. By default, the `-to` option applies the constraint to both the rising and the falling edges.
	`-rise_to`: Applies the maximum delay constraint at the rising edge of the signal on the to pins.
	`-fall_to`: Applies the maximum delay constraint at the falling edge of the signal on the to pins.
value	Specifies the maximum delay value.

Example

- The following command specifies that all paths from `in1` to `out1` must have delays less than `10` units:

```
set_max_delay 10 -from in1 -to out1
```

Related Information

- create_clock
- set_false_path
- set_input_delay
- set_min_delay
- set_multicycle_path
- set_output_delay
- reset_path_exception

set_max_fanout

```
set_max_fanout
[-help]
fanout_limit
object_list
[-override]
```

Sets the maximum fanout load limit constraint on the specified instance pin, ports of the top cell, the specified designs, and/or clock waveforms. The pins of cell instances of timing libraries will have their own maximum constraint. The software will pick the more aggressive value of the two.

When set on a port, the fanout load of the net connected to the port must be less than or equal to the specified limit value to meet the constraint. The fanout load of a net is the sum of the fanout loads of the pins driven by the net. When set on a module, the limit applies to all the nets of that module. Libraries and library cell pins may also have the maximum fanout load limit. In this case the most constraining limit is applied to the nets.

Use the `report_ports -type fanout_load_limit` command to view the maximum fanout load limits set on ports of the top cell.

Fanout load limit can be asserted on input ports, output pins, bidirectional pins, or ports and library pins. You cannot apply the constraint on input pins and output ports.

Parameters

`-help`	Prints out the command usage.
`fanout_limit`	Specifies the value of the maximum fanout load limit constraint. This is a float number and it must be greater than or equal to zero.
`object_list`	Specifies a list of objects - input ports, instance output pins, designs, or library cell output pins.
`-override`	Overrides the maximum fanout limit from other sources, such as, library or constraints. This parameter can be used to force a specific constraint value for a specified list of instance pins/port/library cell pins - this overrides the maximum/minimum values set prior to this setting. To enable this feature, make the following setting: `set timing_constraint_enable_drv_limit_override true`

Examples

- The following example sets a maximum fanout load limit of `5.0` units on all the ports of the top cell:
  ```
  set_max_fanout 5.0 [get_ports *]
  ```
- The following example shows a report of the maximum fanout load limit set on ports using the `report_ports` command, as shown in the example below:
  ```
  set_max_fanout 5.0 [get_ports *] report_ports -type fanout_load_limit -pin [get_ports *]
  ```

Example: Maximum Fanout Load Limit Set On Ports Report

```
> report_ports -type fanout_load_limit -pin [get_ports *]
-----------------------------------------
Pin    Dir  Assertion        Value
Name
-----------------------------------------
CLK2   IN  fanout_load_limit 5.000
CLK1   IN  fanout_load_limit 5.000
in2    IN  fanout_load_limit 5.000
in1    IN  fanout_load_limit 5.000
-----------------------------------------
```

Related Commands

- get_ports
- report_ports
- set_fanout_load
- set_max_capacitance
- set_max_transition
- report_constraint

set_max_time_borrow

```
set_max_time_borrow
[-help]
borrow_value
object_list
[-interface]
```

Specifies the maximum time that can be borrowed by one stage from the next logic stage following a latch to meet timing constraints. Time borrowing is sometimes called cycle stealing. Time borrowing is performed automatically when performing timing analysis.

Use the `set_max_time_borrow` assertion on pins or waveforms. If the assertion is placed on a clock waveform, all latches triggered by this clock signal get the specified maximum time borrow limit. If both the clock waveform arriving at the clock pin of a latch and the clock pin itself have time borrow limit constraints, the maximum borrow limit on the latch is decided by the assertion on the latch clock pin.

Parameters

`-help`	Prints out the command usage.
borrow_value	Specifies the maximum amount of time that can be borrowed. The minimum value allowed is `0`, which disables time borrowing. The maximum allowed is equal to the pulse width minus the setup time: Max = pulse width - setup
`-interface`	Specifies the IO paths for which the maximum time will be borrowed.
object_list	Specifies the list of instance pins, instances, or clocks on which the borrow value is applied. The *object*`_list` argument can be a collection.

Examples

```
set_max_time_borrow 2 [get_cells {i1, i2, i3}]

set_max_time_borrow 1.7 I4/g

set_max_time_borrow 2.5 [get_clocks CLK1]
```

Related Information

- `get_clocks`
- `get_cells`
- `get_pins`
- `reset_max_time_borrow`

set_max_transition

```
set_max_transition
[-help]
transition_limit
object_list
[-clock_path]
[-data_path]
[-fall]
[-override]
[-rise]
```

Sets the maximum slew time limit (transition) on the specified instance pin, ports of the top cell, and/or clock waveforms.

The pins of cell instances of timing libraries will have their own maximum constraint. The software will pick the more aggressive value of the two.

When set on a port, the slew time at the port must be less than or equal to the specified limit. Maximum transition limits on pins can come from library objects, which are the default maximum transition of the libraries or maximum transition on library cell pins. In this case, the most constraining limit, which is the smallest, will apply on pins and ports.

Use the `report_ports -type slew_limit` command to view the maximum transition limits set on ports of the top cell.

When limits are set on a clock waveform, you can use `-clock_path`, `-data_path`, `-rise`, and `-fall` parameters to provide more specifications.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_max_transition` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_max_transition`.
`-clock_path`	Applies the max transition constraint only to pins in the clock network of the clocks defined in the object list.
`-data_path`	Applies the max transition constraint only to data path pins, where the path is launched by the clocks defined in the object list.
`-fall`	Applies the max transition constraint only to falling transitions.
`object_list`	Specifies the list of objects - ports, instance pins, library cell pins, clock waveforms, or current design.
`-override`	Overrides the maximum transition limit from other sources, such as, library or constraints. This parameter can be used to force a specific constraint value for a specified list of instance pins/port/library cell pins - this overrides the maximum/minimum values set prior to this setting. To enable this feature, make the following setting: `set timing_constraint_enable_drv_limit_override true`
`-rise`	Applies the max transition constraint only to rising transitions.
`transition_limit`	Specifies the value of the maximum slew time limit (maximum transition), which must be greater than or equal to zero. The maximum slew time limit is in time units in the current session.

Examples

- The following command sets a maximum transition limit of `0.9` units on all the ports of the top cell:
  ```
  set_max_transition 0.9 [get_ports *]
  ```
- The following example shows a report of the maximum transition limit set on ports using the `report_ports` command, as shown in the example below:
  ```
  set_max_transition 0.9 [get_ports *]
  report_ports -type slew_limit -pin [get_ports *]
  ```

Example: Maximum Transition Limit Set on Ports Report

Pin Name	Dir	Assertion	Value
in1	IN	slew_limit	(: 0.9000)
in2	IN	slew_limit	(: 0.9000)
out1	OUT	slew_limit	(: 0.9000)

out2	OUT	slew_limit	(: 0.9000)
out3	OUT	slew_limit	(: 0.9000)
out4	OUT	slew_limit	(: 0.9000)

- The following command sets the maximum transition limit of 0.001 units on pins in the `all_clocks` collection:
  ```
  set_max_transition 0.001 -clock_path [all_clocks]
  ```
- The following command sets the transitions to the specified value for your entire design:
  ```
  set_max_transition value [current_design]
  ```
- The following command impacts report_contraint output as shown below:
  ```
  set_max_transition 0.01 TDP_COE_INST/TDP_COE_GL_INST/p82
  ```
  The following command shows output as shown below:
  ```
  report_constraint -all_violators -drv_violation_type max_transition

  ###############################################################
  # format : frame 0 : split 0

  Check type : max_transition

  ------------------------------------------------------------------------------
  Pin Name                          Required Actual  Slack    View
  ------------------------------------------------------------------------------
  TDP_COE_INST/TDP_COE_GL_INST/p82  0.010    1.138   -1.128   func_slow_RCMAX
  ------------------------------------------------------------------------------
  ```

Related Commands

- get_ports
- report_ports
- set_max_capacitance
- report_constraint

set_min_capacitance

```
set_min_capacitance
[-help]
capacitance_limit
object_list
[-clock_path]
[-data_path]
[-fall]
[-override]
[-rise]
```

Sets the minimum capacitance limit on the specified ports of the top timing module and/or clock waveforms.

When set on a port, the total capacitance of the net connected to that port must be greater than or equal to the specified limit. When set on a module, the limit applies to all the nets of that module. The total capacitance of each net of the module must be greater than or equal the specified limit. The minimum capacitance limit also comes from library objects, which is the default minimum capacitance limit of libraries and minimum capacitance limit on library cell pins. Pins in the design inherit the limits from libraries. When limits are set on a module and the limit is inherited from libraries, the most constraining limit, which is the largest limit, is used.

Note: The limit set on the top timing module applies to its ports. Thus, if minimum capacitance limits are set on the top timing module and its ports, the most constraining limit will apply to the ports.

Use the `report_ports -type port_cap_limit` command to view the minimum capacitance limits set on ports.

When limits are set on a clock waveform, you can use `-clock_path`, `-data_path`, `-rise`, and `-fall` parameters to provide more specifications.

The `set_min_capacitance` constraint on clock objects will be honored only by the `report_constraint` command for DRV violations. The constraint will not be used in optimizations.

Note: This command syntax is applicable only when the flow-compatible mode is on.

Parameters

`-help`	Prints out the command usage.
`-clock_path`	Specifies all pins that are in the network of the specific clocks in the object list.
`-data_path`	Specifies all pins that are in the data paths launched by the specific clocks in the object list.
`-fall`	Specifies falling capacitance.
`-rise`	Specifies rising capacitance for all selected pins.
`capacitance_limit`	Specifies the value for the minimum capacitance limit. The limit must be greater than or equal to zero. The limit is in capacitance units from the target technology library.
`object_list`	Specifies the list of objects - ports, instance pins, library cell pins, modules, and/or clock waveforms.
`-override`	Overrides the minimum capacitance limit from other sources, such as, library or constraints. This parameter can be used to force a specific constraint value for a specified list of instance pins/port/library cell pins - this overrides the maximum/minimum values set prior to this setting. To enable this feature, make the following setting: `set timing_constraint_enable_drv_limit_override true`

Examples

- The following example sets a minimum capacitance limit of `0.02` units on all the ports of the top timing module and a minimum capacitance limit of `0.05` units on the top timing module:

  ```
  set_min_capacitance 0.02 [get_ports *]
  set_min_capacitance 0.05 [current_design]
  ```

  The limit used for ports is `0.05`, which is the most restrictive limit set on the top timing module (`0.05`) and the limit set on ports (`0.02`).

- The following example uses the `report_port` command to show the minimum capacitance limit set on ports:

  ```
  set_min_capacitance 0.02 [get_ports *]
  report_ports -type port_cap_limit -pin [get_ports *]
  ```

- The following command sets the minimum capacitance limit of `0.001` units on pins in the `all_clocks` collection:

  ```
  set_min_capacitance 0.001 -data_path [all_clocks]
  ```

  Example: Minimum Capacitance Limit Set on Ports Report

  ```
  report_ports -type port_cap_limit -pin [get_ports *]
  ------------------------------------------------------
  ```

```
Pin       Dir    Assertion         Value
Name
-----------------------------------------------------
CLK2      IN     port_cap_limit    ( 0.020 : 0.900 )
CLK1      IN     port_cap_limit    ( 0.020 : 0.900 )
in2       IN     port_cap_limit    ( 0.020 : 0.900 )
in1       IN     port_cap_limit    ( 0.020 : 0.900 )
out2      OUT    port_cap_limit    ( 0.020 : 0.900 )
out1      OUT    port_cap_limit    ( 0.020 : 0.900 )
-----------------------------------------------------
```

Related commands

- report_ports
- set_max_capacitance
- set_max_transition
- set_min_transition
- report_constraint

set_min_delay

```
set_min_delay
value
[-combinational_from_to | -ignore_clock_latency]
[{-from |-rise_from | -fall_from} from_list]
[{-through | -rise_through | -fall_through} through_list]
[{-to | -rise_to | -fall_to} to_list]
[-rise]
[-fall]
[-comment string]
```

Specifies a minimum delay for a timing path. The `set_min_delay` command takes precedence over other path exceptions specified for the same path, except for `set_false_path` command specifications.

Use either the `-from`, `-to`, or `-through` option with the `set_min_delay` command to identify the paths. Specifying both the `-from` and `-to` options identifies a path with specific starting and ending points. If a clock is specified as the `-to` object, all primary outputs and registers clocked by that clock are used as an endpoint.

When you specify a sequential cell name for starting point of the timing path, the valid path starts from the clk pin. This means the latency and delay from clk to output Q is also included in the delay. For path segmentation, you can specify the path to start from Q output of the register by adding /Q after the cell name. In this case, path from clk to Q is not considered in the calculation. For example, in the following set of commands, the valid path starts from different points:

```
set_min_delay 0.66 -from TR1/BR1 -to BLK/BR1/D

set_min_delay 0.66 -from TR1/BR1/Q -to BLK/BR1/D
```

For the first command, the valid path starts from clk, and for the second command the valid path starts from the Q output. In the second case, the path from clk to Q is not considered. For an example of the results generated for paths with and without path segmentation, see the Examples section below.

Use only one `-from`, `-rise_from`, or `-fall_from`, `-to`, `-rise_to`, or `-fall_to` parameter per command. You can use multiple `-through` parameters in the same command.

Parameters

`-combinational_from_to`	Defines a path defined by -from/to pins as combinational. When this parameter is used, the path between the -`from`/-`to` pins is treated as purely combinational, that is, both the source/capture clock latencies are excluded from slack computation. The presence of -`from`/`-to` pins is required for this option to take effect; the -`from`/`-to` pins can be sequential or combinational start/end pins. The `-combinational_from_to` parameter will be deprecated in a future release. This parameter will continue to work in the current release, but for compatibility with future releases you should migrate your scripts to use the `-ignore_clock_latency` parameter. You can use the following global variables to match the semantics between the `-ignore_clock_latency` and `-combinational_from_to` parameters: • `timing_constraint_path_delay_exclude_check_delay_from_ignore_clock_latency` • `timing_constraint_path_delay_exclude_io_delay_from_ignore_clock_latency`
`-comment` *`string`*	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.
`-fall`	Applies the assertion at the falling edge of the signal on the path endpoints.

`-from |-rise_from	-fall_from` *`from_list`*	Specifies a list of timing paths start points. The *`from_list`* argument can contain either object IDs or hierarchical names relative to the current module or clock names. The *`from_list`* argument can be a collection. The valid start points are: • Input port • Bidirectional port • Clock pin of sequential cell Clock pin of sequential cell is one of the following: • Structural clock pin that is part of latch or flip-flop. • Inferred clock pin in cells like macro cells. A pin is an inferred clock pin if it is associated with one of the following: • Reference end of setup check • Reference end of hold check • Reference end of removal check • Reference end of recovery check • Minimum period check • Minimum pulse width check • Pin associated with set_input_delay Specify one of the following:
	`-from`: Applies the minimum delay constraint to all paths originating from the *`from_list`* list. If a clock is specified as the `-from` object, all primary inputs and registers clocked by that clock are used as start points. By default, the `-from` option applies the constraints to both the rising and the falling edges.
	`-rise_from`: Applies the minimum delay constraint at the rising edge of the signal on the from pins.
	`-fall_from`: Applies the minimum delay constraint at the falling edge of the signal on the from pins.
`-ignore_clock_latency`	When this parameter is specified, the software does not count the launch/capture clock latency delays into the timing path delay. The `-ignore_clock_latency` parameter is an industry-standard construct introduced in SDC 2.1 to control the handling of clock-related timing with respect to the `set_max_delay`/`set_min_delay` constraints.

`-rise`	Applies the assertion at the rising edge of the signal on the path endpoints.
`-through | -rise_through | -fall_through` *through_list*	Specifies a list of timing paths through points. The *through_list* argument can contain either object IDs or hierarchical names relative to the current module or clock names. The pins can be on intermediate hierarchical boundaries. The *through_list* argument can be a collection. Specify one of the following:
	`-through`: When you use this parameter without using the `-from` or `-to` options, all paths that go through the specified pins have the specified delay. When you use it with both the `-from` and `-to` options, any path starting at any pin on the `-from` list *and* going through any point on the `-through` pin list *and* going to any point on the `-to` list is affected by the constraint. By default, the `-through` option applies the constraint to both the rising and the falling edges.
	`-rise_through`: Applies the assertion at the rising edge of the signal on the through pins.
	`-fall_through`: Applies the assertion at the falling edge of the signal on the through pins.

`-to	-rise_to	-fall_to` *to_list*	Specifies a list of timing paths end points. The *to_list* argument can contain either object IDs or hierarchical names relative to the current module or clock names. The pins can be on intermediate hierarchical boundaries. The *pin_or_clock_list* argument can be a collection. The valid end points are: • Output port • Bidirectional port • Data pin of sequential cell Data pin of sequential cell is one of the following: • Structural data pin that is part of latch or flip-flop. • Inferred data pin in cells like macro cells. A pin is an inferred data pin if it is associated with one of the following: • Signal end of setup check • Signal end of hold check • Signal end of non-sequential setup check • Signal end of non-sequential hold check • Signal end of removal check • Signal end of recovery check • Pin associated with `set_external_delay` • Data pin of clock gating cell Select from one of the following:
	`-to`: Applies the minimum delay constraint to all paths ending in the `-to` pin list. If a clock is specified as the `-to` object, all primary outputs and registers clocked by that clock are used as endpoint. By default, the `-to` option applies the constraint to both the rising and the falling edges.
	`-rise_to`: Applies the minimum delay constraint at the rising edge of the signal on the to pins.
	`-fall_to`: Applies the minimum delay constraint at the falling edge of the signal on the to pins.
value	Specifies the minimum delay value.

Examples

- The following command specifies a minimum delay of `2.5` for paths from `in1` to `out1`:

  ```
  set_min_delay 2.5 -from in1 -to out1
  ```

- The following commands show the difference in `report_timing` results when you specify a minimum delay with and without path segmentation:
  set_min_delay 9 -from R1 -to R2/D

  ```
  set_min_delay 9 -from R1/Q -to R2/D
  ```

  The `report_timing` results for the first command with path segmentation are:

  ```
  report_timing -early -from R1

  Path 1: VIOLATED Hold Check with Pin R2/CK
  Endpoint: R2/D (v) checked with leading edge of 'CLK_W_1'
  Beginpoint: R1/Q (v) triggered by leading edge of 'CLK_W_1'
  Path Groups: {CLK_W_1}
  Other End Arrival Time 0.000
  + Hold -0.042
  + Path Delay 9.000
  = Required Time 8.958
  Arrival Time 0.212
  Slack Time -8.745
  Clock Rise Edge 0.000
  + Clock Network Latency (Ideal) 0.000
  = Beginpoint Arrival Time 0.000
  ------------------------------------------------------------
  Pin         Cell     Delay    Arrival  Required  Instance
                                Time     Time
  ------------------------------------------------------------
  R1/CK ->    -        -        0.000    8.745     R1
  R1/Q        DFF      0.144    0.144    8.889     R1
  B1/Y        BUF      0.068    0.212    8.958     B1
  R2/D        DFF      0.000    0.212    8.958     R2
  ------------------------------------------------------------
  ```

  The `report_timing` results for the second command without path segmentation are:

  ```
  report_timing -early -from R1/Q

  Path 1: VIOLATED Hold Check with Pin R2/CK
  Endpoint: R2/D (^) checked with leading edge of 'CLK_W_1'
  Beginpoint: R1/Q (^) triggered by leading edge of 'CLK_W_1'
  Path Groups: {CLK_W_1}
  Other End Arrival Time 0.000
  + Hold -0.047
  + Path Delay 9.000
  = Required Time 8.953
  Arrival Time 0.056
  Slack Time -8.897
  ```

```
Clock Rise Edge 0.000
= Beginpoint Arrival Time 0.000
-------------------------------------------------
Pin       Cell    Delay  Arrival  Required  Instance
                         Time     Time
-------------------------------------------------
R1/Q ->   -       -      0.000    8.897     R1
B1/Y      BUF     0.056  0.056    8.953     B1
R2/D      DFF     0.000  0.056    8.953     R2
-------------------------------------------------
```

Related Information

- set_input_delay
- set_output_delay
- set_false_path
- set_max_delay
- set_multicycle_path
- reset_path_exception

set_min_fanout

```
set_min_fanout
[-help]
fanout_limit
object_list
[-override]
```

Sets the minimum fanout load limit constraint on the specified input ports of the top cell.

When set on a port, the fanout load of the net connected to the port must be greater than or equal to the specified limit value to meet the constraint. The fanout load of a net is the sum of the fanout loads of the pins driven by the net. When set on a module, the limit applies to all the nets of that module. The libraries and library cell pins may also have the minimum fanout load limit. In this case the most constraining limit is applied to the nets.

The constraint will not be used in optimizations.

Fanout load limit can be asserted on input ports, output pins, bidirectional pins, or ports and library pins. The constraint cannot be applied on input pins and output ports.

Parameters

`-help`	Prints out the command usage.
fanout_limit	Specifies the value of the minimum fanout load limit constraint. The specified value must be greater than or equal to zero.
object_list	Specifies a list of objects - ports, instance pins, modules, or library cell pins.
`-override`	Overrides the minimum fanout load limit from other sources, such as, library or constraints. This parameter can be used to force a specific constraint value for a specified list of instance pins/port/library cell pins - this overrides the maximum/minimum values set prior to this setting. To enable this feature, make the following setting: `set timing_constraint_enable_drv_limit_override true`

Examples

- The following example sets a minimum fanout load limit of `0.001` units on all the ports of the top cell:
  set_min_fanout 0.001 [get_ports *]
- The following example shows a report of the minimum fanout load limit set on ports using the `report_ports` command, as shown in the following example:
  ```
  set_min_fanout 5.0 [get_ports *] report_ports -type fanout_load_limit -pin [get_ports *]
  ```

  Example: Minimum Fanout Load Limit Set On Ports Report

  ```
  > report_ports -type fanout_load_limit -pin [get_ports *]
  ------------------------------------------
  ```

```
Pin      Dir  Assertion           Value
Name
-----------------------------------------
CLK2     IN   fanout_load_limit   5.000
CLK1     IN   fanout_load_limit   5.000
in2      IN   fanout_load_limit   5.000
in1      IN   fanout_load_limit   5.000
-----------------------------------------
```

Related Commands

- get_ports
- report_ports
- set_fanout_load
- set_max_capacitance
- set_max_transition
- report_constraint

set_min_pulse_width

```
set_min_pulse_width
[-help]
[-low]
[-high]
[-waveform_aware_type {absolute | source_width_ratio}]
value
object_list
```

Selects the most restrictive value when multiple minimum pulse width checks are set.

Note: The `timing_disable_lib_pulsewidth_checks` global has no effect on pulse width checks created by the `set_min_pulse_width` command.

Parameters

`-help`	Prints out the command usage.
`-high`	Applies the minimum pulse width check to high signal levels only. If neither the `-low` or `-high` option is specified, the value is applied to both low and high signal levels.
list	Specifies a list of pins or clock waveforms to which the minimum pulse width check is to be applied. If the list is not specified, the minimum pulse width check applies to all the pins in the clock tree of the current design. If a clock waveform is specified, all the pins driven by the clock are affected.
`-low`	Applies the minimum pulse width check to low signal levels only. If neither the `-low` or `-high` option is specified, the value is applied to both low and high signal levels.
object_list	Specifies list of objects: pins, ports, or clocks.
value	Specifies a non-negative floating minimum pulse width check value. No default value is assumed for minimum pulse width checks.
`-waveform_aware_type {absolute | source_width_ratio}`	Specifies the type of waveform-aware pulse width check to apply as `absolute` or `source_width_ratio`.

Examples

- The following command sets a minimum pulse width, both low and high, check of `1.0` to the clock network of the current design:
  ```
  set_min_pulse_width 1.0
  ```
- The following command sets a minimum pulse width low check of `2.0` to all the pins clocked by the `CLK1` clock:

```
set_min_pulse_width -low 2.0 [get_clocks CLK1]
```

- The following command sets a minimum pulse width `-high` check of `1.0` to the `CP` clock pin of register `ff1`, as shown in the following example:

```
set_min_pulse_width 0.5 u55/CK -low
set_min_pulse_width 1.0 u55/CK -high
report_min_pulse_width -pin u55/CK

------------------------------------------------------
SEQUENTIAL CLOCK PULSE WIDTH
------------------------------------------------------
Pin                 Required     Actual Pulse    Slack
                    Pulse Width  Width
------------------------------------------------------
u55/CK (low)        0.500        4.200           3.700
u55/CK (high)       1.000        5.800           4.800
------------------------------------------------------

report_timing -check_type pulse_width -to u55/CK

Path 1: MET User PulseWidth Check with Pin u55/CK
Ending Clock Edge: u55/CK (^) checked with leading edge of 'CLK4'
Beginning Clock Edge: u55/CK (v) triggered by trailing edge of 'CLK4'
Path Groups: {CLK4}
Other End Arrival Time 0.152
- PulseWidth 0.500
+ Phase Shift 10.000
= Required Time 9.651
- Arrival Time 5.951
= Slack Time 3.700
Clock Fall Edge 6.000
+ Clock Network Latency (Prop) -0.049
= Beginpoint Arrival Time 5.951
-------------------------------------------------
Load   Slew    Delay  Arrival  Cell  Arc  Pin
                      Time
-------------------------------------------------
0.015  0.019   5.951  DFF      CK    v    u55/CK ->
-------------------------------------------------
```

Related Commands

- reset_min_pulse_width
- report_analysis_coverage
- report_min_pulse_width
- report_timing

set_min_transition

```
set_min_transition
[-help]
transition_limit
object_list
[-clock_path]
[-data_path]
[-fall]
[-override]
[-rise]
```

Sets the minimum slew time limit (transition) on the specified ports of the top timing module and/or clock waveforms.

When set on a port, the slew time at the port must be greater than or equal to the specified limit. When set on a module, the limit applies to all the pins in the module and its ports if the module is the top timing module. The minimum transition limits on pins can come from library objects, which are the default minimum transition of the libraries or minimum transition on library cell pins. In this case, the most constraining limit, which is the largest, will apply on pins and ports.

Use the `report_ports -type slew_limit` command to view the minimum transition limits set on ports of the top timing module.

When limits are set on a clock waveform, you can use `-clock_path`, `-data_path`, `-rise`, and `-fall` parameters to provide more specifications.

The `set_min_transition` constraint on clock objects will be honored only by the `report_constraint` command for DRV violations. The constraint will not be used in optimizations.

Parameters

`-help`	Prints out the command usage.
`-clock_path`	Specifies all pins that are in the network of the specific clocks in the object list.
`-data_path`	Specifies all pins that are in the data paths launched by the specific clocks in the object list.
`-fall`	Specifies falling transition.
`object_list`	Specifies the list of objects - ports, instance pins, library cell pins, modules, and/or clock waveforms.
`-override`	Overrides the minimum transition limit from other sources, such as, library or constraints. This parameter can be used to force a specific constraint value for a specified list of instance pins/port/library cell pins - this overrides the maximum/minimum values set prior to this setting. To enable this feature, make the following setting: `set timing_constraint_enable_drv_limit_override true`
`-rise`	Specifies rising transition for all selected pins.
`transition_limit`	Specifies the value of the minimum slew time limit (minimum transition). This value must be less than or equal to zero. The minimum slew time limit is in time units from the target technology library.

Examples

- The following example sets a minimum transition limit of `0.9` units on all the ports of the top timing module, and a minimum transition limit of `0.7` units on the top timing module:

  ```
  set_min_transition 0.9 [get_ports *]
  set_min_transition 0.7 [current_design]
  ```

  The minimum limit that applies to ports is `0.9`, which is the most constraining value.

- The following example shows a report of the minimum transition limit set on ports using the `report_ports` command, as shown in the following example:

  ```
  set_min_transition 0.9 [get_ports *]
  report_ports -type slew_limit -pin [get_ports *]
  ```

  Minimum Transition Limit Set on Ports Report

  ```
  set_min_transition 0.9 [get_ports *]
  > report_ports -type slew_limit -pin [get_ports *]
  ----------------------------------------
  Pin    Dir   Assertion    Value
  Name
  ----------------------------------------
  ```

```
CLK2    IN     slew_limit   ( 0.900 : )
CLK1    IN     slew_limit   ( 0.900 : )
in2     IN     slew_limit   ( 0.900 : )
in1     IN     slew_limit   ( 0.900 : )
out2    OUT    slew_limit   ( 0.900 : )
out1    OUT    slew_limit   ( 0.900 : )
------------------------------------------

> report_constraint -drv_violation_type min_transition

Check type : min_transition
---------------------------
Pin : CLK4

Transition Time: 0.003
- min_transition: 0.900
--------------------------------------------------
slack : -0.897 ( VIOLATED )
```

Related Commands

- set_max_transition
- reset_min_transition
- report_ports
- report_constraint

set_mode

```
set_mode
[-type cell]
list_of_modes
object_list
```

Specifies the Liberty timing library modes to be active for a specific instance.

A library mode is a subset of active timing arcs that is defined in the timing library. Library modes are similar to conditional timing arcs, but do not require the application of constants to set. Instead of changing a cell's mode by applying constants to its input pins, you can use the `set_mode` command to refer to sets of active arcs by name (such as `READ` or `WRITE`, for a memory cell).

By default, all library modes in the timing library are active. If you specify one or more library modes using the `set_mode` command, only the timing arcs consistent with the specified mode(s) are active for the instance. All other library modes become inactive.

To reset the library modes for an instance, use the `reset_mode` command.

Parameters

object_list	Specifies the instances to which to apply the library mode.
list_of_modes	Specifies the name of the library mode value defined in the timing library. You can specify one or more mode values, but only one mode value per defined mode group in the Liberty library.
-type cell	Sets the library mode at the instance level. **Note:** Setting the library mode at the instance level is the only form of `set_mode` that is currently supported. This parameter exists to support constraints coming from external sources.

Examples

- The following command specifies the `test_mode` library mode active for instances of b1.

  ```
  set_mode test_mode [get_cells b1]
  ```

  The following command reports the status of all library modes associated with b1:

  ```
  tempus > report_mode b1
  -------------------------------
  Mode Group test_mode of b1
  -------------------------------
  Mode     Status     Condition
  Name
  -------------------------------
  test_B   Inactive   !(Y)
  test_A   ACTIVE      Y
  -------------------------------
  ```

  The above report shows that only the timing arcs consistent with the `test_A` library mode are active for instances of b1 - and `test_B` is inactive.

- If you specify the `report_inactive_arcs` command, the report shows the disabled arcs for the b1 instance are from the `test_mode` library mode:

  ```
  ------------------------------------------------------
  Flags :  const propagated constant
           snipped loop snipped arcs
           disable set_disable_timing
           disable_clock_gating set_disable_clock_gating
           library_disable set_disable_cell_timing
           Missing_Phase No Arc Phase Data
  From     To       DisableType   ArcType                 Reason
  ------------------------------------------------------------------
  u8/CK ^   u8/D ^   const        hold_rising_clk_rise    u8/D = 1
  u8/CK ^   u8/D v   const        hold_falling_clk_rise   u8/D = 1
  u8/CK ^   u8/D ^   const        setup_rising_clk_rise   u8/D = 1
  u8/CK ^   u8/D v   const        setup_falling_clk_rise  u8/D = 1
  ------------------------------------------------------------------
  ```

Related Information

- reset_mode
- report_mode
- report_inactive_arcs
- get_cells

set_multicycle_path

```
set_multicycle_path
[-help]
<number_of_cycles>
[-comment <string>]
[-fall]
[-fall_through <through_list>]
[-hold]
[-rise]
[-rise_through <through_list>]
[-setup]
[-through <through_list>]
[-start  | -end ]
[-from <from_list> | -rise_from <from_list> | -fall_from <from_list>]
[-to <to_list]> | -rise_to <to_list> | -fall_to <to_list]>]
```

Specifies multicycle paths between specific timing paths in a design or between clock domains. The multicycle paths are timing paths that require more than one clock period for execution. By default, all the paths are considered as one cycle paths for setup and hold check evaluation. The `set_multicycle_path` command allows you specify additional cycles for the paths.

Parameters

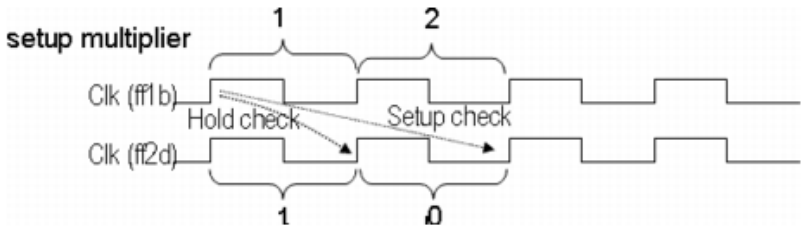
`-help`	Prints out the command usage.
number_of_cycles	Specifies the number of cycles allowed for the path.
`-comment` *string*	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.
`-fall`	Applies the assertion at the falling edge of the signal on the path endpoints.
`[-fall_through <through_list>]`	Applies the assertion at the falling edge of the signal on the through pins

`-from <from_list> | -rise_from <from_list> | -fall_from <from_list>`	Specifies the clocks, ports, pins, or sequential instances that are at the start of the multicycle paths. The port direction can be either input or inout. Pins must be valid path startpoints such as sequential instance clock pins, latch data pins or pins on which input delays have been specified. Pins can be on intermediate hierarchical boundaries. The *from_list* variable can contain either collection IDs or object names. If the `-from` option is used without the `-to` option, all paths originating from the `from_list` are multicycle paths. Use either the `-from`, `-to`, or `-through` option with the `set_multicycle_path` command to identify the paths. Specifying both the `-from` and `-to` options identifies a path with specific starting and ending points. Use only one `-from`, `-rise_from`, or `-fall_from` option per command.
`-hold | -setup`	Specifies whether the multicycle adjustment applies to setup/hold check. The `-setup` parameter applies multicycles to setup check. The hold check is affected but the hold multiplier does not change. The `-hold` parameter applies multicycles to hold check only. *Default:* If you do not specify `-setup` or `-hold` parameter, the software considers the specified multicycle value for setup check and uses 0 for the hold check.
`-rise`	Applies the assertion at the rising edge of the signal on the path endpoints.
`-rise_through <through_list>`	Applies the assertion at the rising edge of the signal on the through pins.
`-start | -end`	Specifies whether the multicycle adjustment is to be applied relative to the starting clock edge for the path or relative to the ending clock edge for the path. Use these options for multiple clocks designs. By default, the setup check is relative to the end clock and the hold check is relative to the start clock. **Note:** The `-start` and `-end` parameters are mutually exclusive; you cannot specify them together. You can set the `timing_enable_sdc_compatible_data_check_mcp` global variable to `true` to enable checking for SDC compatible data checks. When the `-start` parameter is specified, the parameter reverses the reference clock by one clock cycle.

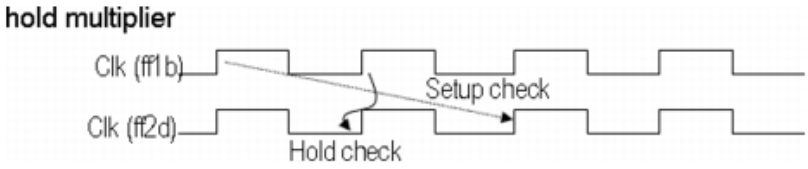
`-through <through_list>`	Applies the assertion to both the rising and the falling edges. This parameter specifies the ports, pins, instances, or nets that the path goes through. The pins can be on intermediate hierarchical boundaries. The *through_list* variable can contain either collection IDs or object names. When used without the `-from` or `-to` option, all paths that go through the `-through` pins are multicycle paths. When used with both the `-from` and `-to` options, any path starting at any pin on the `-from` list and going through any point on the `-through` pin list and going to any point on the `-to` list is affected by the assertion. You can use only one `-from` and `-to` option, but you can specify several `-through` options in the same command. By default, the `-through` option applies the assertion to both the rising and the falling edges. Using the `-rise_through` option applies the assertion at the rising edge of the signal on the through pins. Using the `-fall_through` option applies the assertion at the falling edge of the signal on the through pins.
`-to <to_list]> | -rise_to <to_list> | -fall_to <to_list]>`	Specifies the clocks, ports, pins, or sequential instances that are at the end of the multicycle paths. The port direction can be either output or inout. Pins must be valid path end-points such as sequential instance data pins or pins on which output delays have been specified. Pins can be on intermediate hierarchical boundaries. The *to_list* variable can contain either collection IDs or object names. If the `-to` option is used without the `-from` option, all paths ending in the `-to` pin list are multicycle paths. Use only one `-to` option per command. Use only one `-to`, `-rise_to`, or `-fall_to` option per command.

Examples

- The following command sets all paths between ff1b + ff2d to two cycles for setup:
  ```
  set_multicycle_path -setup 2 -from {ff1b} -to {ff2d}
  ```



- The following command moves hold check to start clock preceding edge:
  ```
  set_multicycle_path -hold 1 -from {ff1b} -to {ff2d}
  ```



- The following commands show setup multiplier values from -1 to 3:

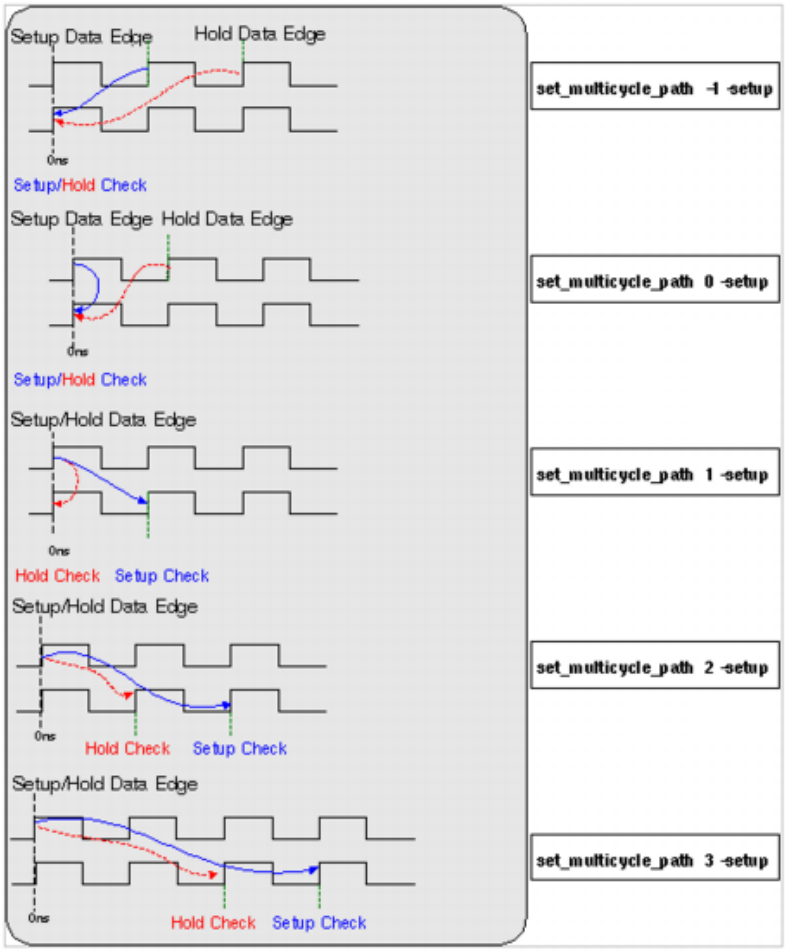
```
set_multicycle_path -1 -setup

set_multicycle_path 0 -setup

set_multicycle_path 1 -setup

set_multicycle_path 2 -setup

set_multicycle_path 3 -setup
```



- The following commands show hold multiplier values from -1 to 3:

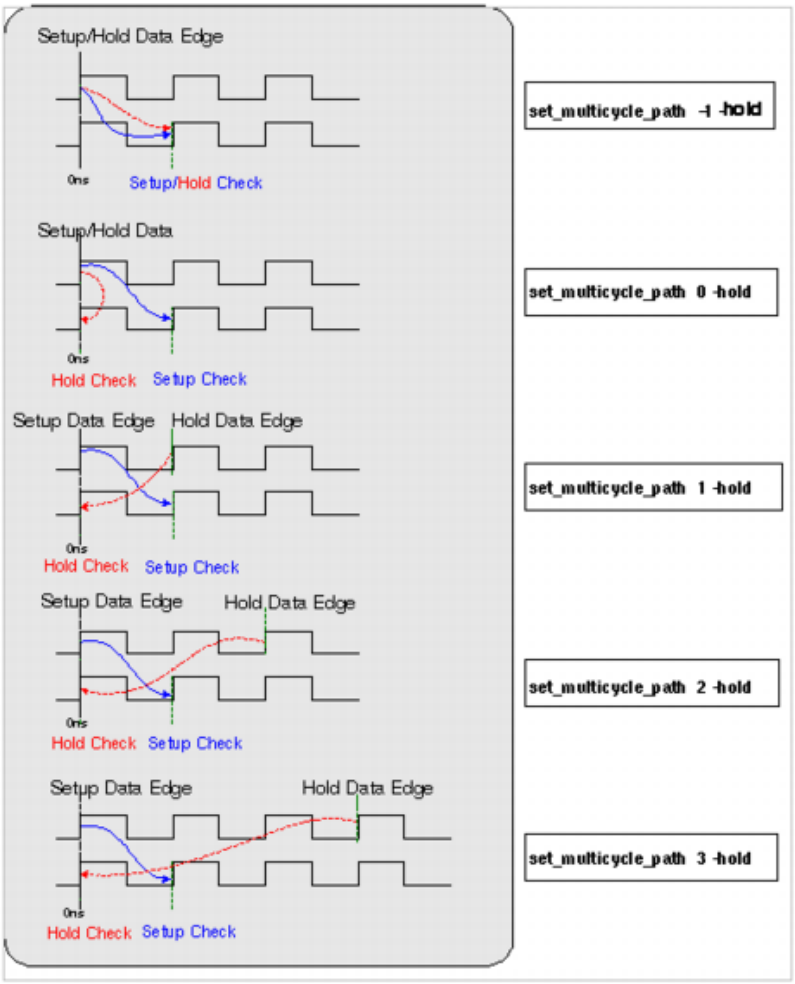
```
set_multicycle_path -1 -hold

set_multicycle_path 0 -hold

set_multicycle_path 1 -hold

set_multicycle_path 2 -hold

set_multicycle_path 3 -hold
```



Related Information

- set_false_path
- set_max_delay
- set_min_delay
- reset_path_exception

set_output_delay

```
set_output_delay
-clock clock_name
[-clock_fall]
[-rise] [-fall]
[-max] [-min]
[-add_delay]
[-network_latency_included]
[-source_latency_included]
[-reference_pin pin_name]
[-level_sensitive]
[-group_path group_name]
delay_value
port_or_pin_list
```

Specifies the data required time on output ports, hierarchical ports, or internal output pins.

Parameters

`-add_delay`	Adds delay information to the existing output delay. The additional delay information can be for multiple paths relative to different clocks or clock edges leading to an input port or internal input pins. If you do not use the `-add_delay` option, the software uses the last `set_output_delay` related to a pin when there are multiple `set_output_delay` statements for the same pin.
`-clock clock_name`	Specifies that the delay is relative to the specified clock. You cannot use the `set_output_delay` command without the `-clock` parameter. Such constraints are accepted but are ignored during timing analysis. When you specify multiple clocks, an error occurs and the constraint is ignored.
`-clock_fall`	Specifies that the delays are with respect to the falling edge of the clock waveform. If you do not specify the `-clock_fall` option, the delay is with respect to the rising edge of the clock waveform. You cannot specify the `-clock_fall` parameter without the `-clock` parameter.
`delay_value`	Specifies the value for the path delay.
`-fall`	Specifies that the output delay refers to the falling edge of the signal at specified ports or pins. If you do not specify the `-fall` option, the output delay applies to both the edges.

`-group_path` *group_name*	Specifies the group name where paths ending at the specified ports or pins are to be added. The `-group_path` parameter is not honored in multi-mode multi-corner (MMMC) designs. If specified, a warning is issued and the constraints are applied as if the `-group_path` parameter is not specified.
`-level_sensitive`	Models output delay that is captured by a positive level sensitive latch. For setup checks, the timing system checks the arrival time at the output port with the opening edge of the latch. The timing system treats any arrival time exceeding the opening edge as negative slack. For hold checks, the timing system performs the checks with respect to the closing edge of the latch. Use the `-clock` parameter to model a positive level sensitive latch, or the `-clock_fall` parameter to model a negative level sensitive latch. You cannot use the `-level_sensitive` parameter without the `-clock` parameter.
`-max`	Specifies that the delay refers to the setup analysis. If you do not specify the `-max` parameter, the output delay applies to both setup and hold analysis.
`-min`	Specifies that the delay refers to the hold analysis. If you do not specify the `-min` parameter, the output delay applies to both setup and hold analysis.
`-network_latency_included`	Specifies that the clock network latency should not be added to the output delay value. If you do not specify this option, the clock network latency of the related clock is added to the output delay value. This parameter has no effect if the specified clock is in propagated mode.
port_or_pin_list	Specifies a list of output port or pin names. The port_or_pin_list argument can be a collection.
`-reference_pin` *pin_name*	Adds source and network latency of the specified reference pin to the output delay value in propagated mode. A reference pin is a leaf pin in the fanout cone of the clock source of the clock that you specify using the `-clock` parameter. You cannot specify multiple pins for `-reference_pin` parameter. The command issues a warning and accepts the first pin. You cannot use the `-reference_pin` parameter with the `-network_latency_included` or `-source_latency_included` parameters. You cannot use the `-reference_pin` parameter to specify hierarchical pins.
`-rise`	Specifies that output delay refers to the rising edge of the signal at the specified ports or pins. If you do not specify the `-rise` option, the output delay applies to the both edges.

`-source_latency_included`	Specifies that the clock source latency should not be added to the output delay value. With different delay values for `-rise/-fall` and `-min/-max`, only the last specified combination of `-network_latency_included` and `-source_latency_included` parameters is honored. The specified combination can be one of the following: a) Either `-network_latency_included` or `-source_latency_included` b) Both the parameters c) None of these

Examples

- The following example sets the input and output delays for the bidirectional `INOUT2` port.The input signal arrives at `INOUT2` at `4.5` units after the falling edge of `CLKB`. The output signal is required at `INOUT2` at `2.5` units before the rising edge of `CLKD`:

```
set_input_delay 4.5 -clock CLKB -clock_fall { INOUT2 }
set_output_delay 2.5 -clock CLKD { INOUT1 }
```

- The following example shows how to use the `-group_path` option to add ports into a named group:

```
set_output_delay 2.5 -max -clock CLK -group_path busC {busC[*]}
```

Related Information

- `all_outputs`
- `create_clock`
- `report_ports`
- `set_input_delay`
- `set_load`

set_pll_timing

```
set_pll_timing
[-help]
inst_name
-feedback_pin feedback pin name
-output_clock clock name
-output_pin output pin name
[-reference_edge {rise | fall | both}]
-reference_pin reference pin name
```

This command identifies phase locked loop (PLL) insertion delays and the associated generated clock(s) that require adjustment. To remove the effect of clock tree latency, PLLs can be used to adjust the delay on the PLL output clock waveform so that the effective edges of the clock at the clock tree endpoints are aligned with the reference edges of the input clock to the PLL.

When you use the `set_pll_timing` command, the software uses the settings to automatically calculate and apply a delay offset to a PLL output clock to compensate for internal clock tree latency.

The `set_pll_timing` command works in conjunction with a generated clock definition to provide the correct clock timing at the output of the PLL. To use this command, you must first assert a generated clock on the `set_pll_timing -output_pin`. The `-source` option of the `create_generated_clock` command must also match the `-reference_pin` specified by the `set_pll_timing` command.

Parameters

`-help`	Prints out the command usage.
`inst_name`	Specifies the instance name or the object ID of a PLL cell, on the basis of which the reference pins, feedback pins, or the output pins will be searched.
`-feedback_pin` *`feedback pin name`*	Specifies the name of the PLL feedback pin.
`-output_clock` *`clock name`*	Specifies the generated clock on the PLL output.
`-output_pin` *`output pin name`*	Specifies the PLL clock output pin name.
`-reference_edge {rise | fall | both}`	Defines adjustment edge(s). When this parameter is specified, the software selects the edges of the clocks at the reference and feedback pins that need to be aligned. If the `rise` or `fall` option is specified, then the clock latency from that reference edge to the related edge at the feedback pin will be used. In this case, both the rise and fall edges will use the same latency adjustment. If both of these options are specified, then the software will measure separate latency adjustments for both the rise and fall edges. *Default*: `rise`
`-reference_pin` *`reference pin name`*	Specifies the PLL reference pin name.

Examples

Here are some examples.

- The following example shows the defined clocks in a sample design:

  ```
  create_clock [get_ports {clk_in}] -name CLK -period 6 -waveform {0 3}
  create_generated_clock -name GCLK1 -source [get_pins {mypll/CLK}] -multiply_by 1 -master_clock
  [get_clocks {CLK}] -add [get_pins {mypll/GCLK}]
  ```

  The following `report_timing` command shows the insertion delay values at various points in the timing report:

  ```
  report_timing -from_rise mypll/GCLK -to_rise mypll/CEN -point_to_point

  Path 1:Endpoint: mypll/CEN (^) (unconstrained output)

  Beginpoint: mypll/GCLK (^) triggered by leading edge of 'GCLK1'
  ```

```
Arrival Time 0.496
Analysis View: default_analysis_view_setup
------------------------------------------------------------
Pin           Cell      Delay   Arrival   Generated Clock
                                Time      Adjustment
------------------------------------------------------------
mypll/CLK                       0.112
mypll/GCLK ->  PLL       0.133  0.246     GCLK1 Adj. = 0.000
clkTreeL0/Y    CLKBUFX1  0.139  0.385
clkTreeL1_1/Y  CLKBUFX1  0.118  0.503
mypll/CEN ->   PLL      -0.007  0.496
------------------------------------------------------------
```

From the above table, it can be seen that the rise insertion delay of clock CLK at pin mypll/CLK is 0.112ns.

Now, the following command will adjust the source latency of clock GCLK1 such that the edges at mypll/CLK and mypll/CEN are aligned i.e. the clock's rising edge should reach mypll/CEN at 0.112ns only:

```
set_pll_timing [get_cells mypll] -reference_pin CLK -feedback_pin CEN -output_pin GCLK -
output_clock [get_clocks GCLK1]
```

- The following `report_timing` command shows the source latency of GCLK clock as -0.138ns, and the clock reaches mypll/CEN pin at 0.112ns (same as on mypll/CLK):

```
report_timing -from_rise mypll/GCLK -to_rise mypll/CEN -point_to_point

Path 1:Endpoint: mypll/CEN (^) (unconstrained output)
Beginpoint: mypll/GCLK (^) triggered by leading edge of 'GCLK1'
Arrival Time 0.112
Analysis View: default_analysis_view_setup
+ Source Insertion Delay 0.112
- PLL Feedback Delay 0.250
= Beginpoint Arrival Time -0.138
------------------------------------------------------------------
Pin             Cell       Delay    Arrival    Generated Clock
                                    Time       Adjustment
------------------------------------------------------------------
mypll/GCLK ->                       -0.138    GCLK1 Pll Adj. = 0.000
clkTreeL0/Y     CLKBUFX1   0.139    0.001
clkTreeL1_1/Y   CLKBUFX1   0.118    0.120
mypll/CEN ->    PLL       -0.007    0.112
------------------------------------------------------------------
```

Related Commands

- create_generated_clock
- create_clock
- reset_pll_timing

set_propagated_clock

```
set_propagated_clock
pin_clock_list
```

Puts the `propagated_clock` assertion on the specified pin, port, or clock object. This will cause all clock endpoints in the fanout of the specified object to receive propagated clock timing unless there is a `set_clock_latency` with higher precedence. If there are multiple `set_clock_latency` and `set_propagated_clock` assertions between the clock waveform, clock root and clock endpoint, the closest assertion to the clock endpoint takes precedence.

There are two methods of clock propagation: depending on whether the clock is ideal or propagated.

- When a clock is ideal, the delay from a clock port to a register clock pin (network insertion delay) comes from design constraints using the `set_clock_latency` command.
- When a clock is propagated, the network insertion delay is computed from the actual gates and interconnects in the clock network.

To set the propagated_clock property on all clock waveform objects, use the following command:

```
set_propagated_clock [all_clocks]
```

When `set_propagated_clocks [all_clocks]` command is asserted, it will apply to all the currently defined clock waveform objects. This setting will not apply to clocks created later in the script. Whether all the clock endpoints receive propagated or ideal timing depends on the precedence of `set_clock_latency` in the fanout.

The `set_propagated_clock [all_clocks]` command puts the `propagated_clock` assertion on all clocks except virtual clocks. When you put the assertion on a real clock that is used to define I/O timing (with `set_input_delay -clock` or `set_output_delay -clock`), the software does not take the specified network latency into account. Instead, the software calculates the network latency, which changes the I/O timing. You can change this behavior by using the following global variable:

```
set_global timing_io_use_clock_network_latency always
```

You can use this command after creating the clock with the create_clock command.

Parameters

pin_clock_list	Specifies the objects on which you want to put the `propagated_clock` assertion. The *pin_or_clock_list* argument can be a collection. When a pin is specified, it affects the propagation mode for all the registers in the transitive fanout (TFO) of the pin.

Examples

- The following command puts the `propagated_clock` assertion on the clock waveform `CLK1`:

  ```
  set_propagated_clock CLK1
  ```

- The following command puts the `propagated_clock` assertion on all clock waveforms:

  ```
  set_propagated_clock [all_clocks]
  ```

Related Information

- `all_clocks`
- `get_propagated_clock`
- `report_clocks`
- `set_clock_latency`
- `set_clock_uncertainty`

set_property

```
set_property
[-object_type {object_type}]
{object_list}
[-quiet]
property_name
property_value
```

Sets user-defined properties on objects.

The `set_property` command needs a property to be defined using the `define_property` command.

Parameters

object_list	Specifies a list of objects on which the property is to be applied.
-object_type {*object_type*}	Specifies the type of object on which the property is to be defined.
property_name	Specifies the name of the property.
property_value	Specifies the value of the property.
-quiet	Specifies to exit quietly without issuing an error.

Examples

- The following command applies a user-defined property `slk` to pins `C1/A`:

  ```
  set_property [get_pins C1/A] slk 2
  ```

- The following command applies aocv_weight property to library arcs:

  ```
  set_property [get_lib_arcs -of_objects slow/INVX1] aocv_weight 8
  ```

Related Commands

- `get_property`
- `report_property`
- `define_property`
- `reset_property`

set_pulse_clock_max_transition

```
set_pulse_clock_max_transition
transition_limit
object_list
[-transitive_fanout]
[-rise]
[-fall]
```

Sets the maximum pulse clock transition constraint on the specified pulse generator cells, pulse generator library cells, clocks, and/or current design.

Parameters

`-fall`	Sets transition limit value for falling transition.
`object_list`	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design.
`-rise`	Sets transition limit value for rising transition.
`transition_limit`	Specifies the transition limit value.
`-transitive_fanout`	Specifies the transition limit value for transitive fanout.

Examples

The following example sets the max pulse clock transition constraint of 0.1ns on the input pins of all the instances of the lib cell PULSE_GEN_00:

```
set_pulse_clock_max_transition 0.1 PULSE_GEN_00
```

This constraint can be observed on the input pin 'A' of an instance 'pg_inst' of this lib cell using the following command:

```
report_constraint pg_inst/A -drv_violation_type pulse_clock_max_transition

Fragmented output:

Check type : pulse_clock_max_transition_rise
----------------------------
-----------------------------------------------
Pin Name Required Actual Slack
-----------------------------------------------
pg_inst/A 0.100 0.030 0.070

Check type : pulse_clock_max_transition_fall
----------------------------
-----------------------------------------------
Pin Name Required Actual Slack
-----------------------------------------------
```

```
pg_inst/A 0.100 0.030 0.070
```

Related Information

- get_clocks
- get_cells
- reset_pulse_clock_max_transition

set_pulse_clock_max_width

```
set_pulse_clock_max_width
pulse_width
object_list
[-transitive_fanout]
```

Sets the maximum pulse clock width constraint on the specified pulse generator cells, pulse generator library cells, clocks, and/or current design.

To report the maximum pulse clock width constraint, you can use the `report_constraint -check_type pulse_clock_max_width` command.

Parameters

object_list	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design.
pulse_width	Specifies the pulse width value.
-transitive_fanout	Specifies the pulse width value for transitive fanout.

Examples

The following example sets the max pulse clock width constraint of 1ns on the pulse generator instance 'pg_inst':

```
set_pulse_clock_max_width 1 pg_inst
```

This constraint can be observed using the following command:

```
report_constraint pg_inst/Y -check_type pulse_clock_max_width
```

Fragmented output:

```
-------------------------------------------
Pin Required Actual Pulse Slack Clock Type
Pulse Width Width
-------------------------------------------
pg_inst/Y (high) 1.000 0.366 0.634 PG
-------------------------------------------
```

Related Commands

- get_clocks
- get_cells
- reset_pulse_clock_max_transition
- reset_pulse_clock_max_width

set_pulse_clock_min_transition

```
set_pulse_clock_min_transition
transition_limit
object_list
[-transitive_fanout]
[-rise]
[-fall]
```

Sets the minimum pulse clock transition constraint on the specified pulse generator cells, pulse generator library cells, clocks, and/or current design.

Parameters

`-fall`	Sets transition limit value for falling transition.
`object_list`	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design.
`-rise`	Sets transition limit value for rising transition.
`transition_limit`	Specifies the transition limit value.
`-transitive_fanout`	Specifies the transition limit value for transitive fanout.

Examples

The following example sets the min pulse clock transition constraint of 0.05ns on the input pins of all the instances of the lib cell PULSE_GEN_00:

```
set_pulse_clock_min_transition 0.05 PULSE_GEN_00
```

This constraint can be observed on the input pin 'A' of an instance 'pg_inst' of this lib cell using the following command:

```
report_constraint pg_inst/A -drv_violation_type pulse_clock_min_transition
```

Fragmented output:

```
Check type : pulse_clock_min_transition_rise
---------------------------
-------------------------------------------
Pin Name Required Actual Slack
-------------------------------------------
pg_inst/A 0.050 0.030 -0.020

Check type : pulse_clock_min_transition_fall
---------------------------
-------------------------------------------
Pin Name Required Actual Slack
```

```
-------------------------------------------
pg_inst/A 0.050 0.030 -0.020
```

Related Information

- reset_pulse_clock_min_transition
- get_clocks
- get_cells

set_pulse_clock_min_width

```
set_pulse_clock_min_width
[-help]
pulse_width
object_list
[-transitive_fanout]
```

Sets the minimum pulse clock width constraint on the specified pulse generator cells, pulse generator library cells, clocks and/or current design.

When you set this constraint on a clock, the settings are applied to all the pulse generators that are driven by the clock. When this constraint is used for a design, the software applies the constraint to all the clock networks in the design.

The specified minimum pulse width is expected to be larger than the pulse width value. The width is defined when a clock object is created and its waveform is defined in the current design (using the `create_clock -waveform` parameter).

In case of conflicting minimum width constraints, the more restrictive constraint is used. When applied on a non-pulse generator object, or a non-clock target, this constraint has no effect.

Note: You must specify `set_interactive_constraint_modes` constraint prior to running the `set_pulse_clock_min_width` command.

Parameters

`-help`	Prints out the command usage.
`object_list`	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design.
`pulse_width`	Specifies the minimum pulse width value.
`-transitive_fanout`	Specifies the pulse width value for transitive fanout.

Examples

- The following example sets the min pulse clock width constraint of 0.3 ns on the pulse generator instance `pg_inst`:

  ```
  set_pulse_clock_min_width 0.3 pg_inst
  ```

  This constraint can be observed using the following command:
  ```
  report_constraint pg_inst/Y -check_type pulse_clock_min_width
  ```

  Fragmented output:

  ```
  ----------------------------------------
  Pin Required Actual Pulse Slack Clock Type

  Pulse Width Width
  ----------------------------------------
  pg_inst/Y (high)    0.300 0.366 0.066 PG
  ----------------------------------------
  ```

Related Commands

- `report_constraint`
- `reset_pulse_clock_min_width`
- `set_pulse_clock_max_width`
- `report_min_pulse_width`
- `set_interactive_constraint_modes`
- `timing_enable_uncertainty_for_pulsewidth_checks`

set_resistance

```
set_resistance
-help
[-max]
[-min]
resistance_value
net_list
```

Sets the wire resistance value on the specified nets of the top cell, which is used to compute the net delay of the specified nets. The resistance that you specify using this command overrides the resistance from wire load models and SPEF.

Note: In the best-case worst-case (BC-WC) analysis mode, the minimum resistance is used to compute the early path delay and the late path delay for the minimum corner. The maximum resistance is used to compute the early path delay and the late path delay for the maximum corner.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_resistance` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_resistance`.
`-max`	Specifies that the asserted resistance is the maximum resistance. *Default*: If the `-max` (or `-min`) parameter is not specified, the resistance value is used as both minimum and maximum resistance.
`-min`	Specifies that the asserted resistance is the minimum resistance. *Default*: If the `-min` (or `-max)` parameter is not specified, the resistance value is used as both minimum and maximum resistance.
`net_list`	Specifies a list of nets on which the wire resistance is set.
`resistance_value`	Specifies the value for the wire resistance. This value (in units) is found in the target technology library.

Examples

- The following example set the maximum wire resistance of `2.0` units on the net `n1`:

  ```
  set_resistance -max 2.0 [get_nets n1]
  ```

- The following example set the minimum wire resistance of `1.2` units and a maximum wire resistance of `2.0` units on the net `n1`:

  ```
  set_resistance 1.2 -min [get_nets n1]
  ```

```
set_resistance 2.0 -max [get_nets n1]
```

Related Commands

- reset_resistance
- report_net
- report_timing
- set_drive
- set_load

set_sense

```
set_sense
[-help]
pin_or_port_list
[-clocks clock_list]
{[-positive | -negative | -stop_propagation ] [-type <type_list> ]}
```

Selects which phase of the clock to filter at the specified point.

Parameters

`-help`	Prints out the command usage.
`-clocks` *`clock_list`*	Specifies a list of clocks passing through a specified point to which the constraint applies. When this parameter is not specified, the filtering of clock phase will be applicable to all the clocks passing through that specified point. This parameter must be used with a specified pin name and one of the parameters : `-positive`, `-negative`, or `-stop_propagation`.
`-negative`	Specifies that positive phase filtered clocks pass through pins in the *`pin_list`*.
`pin_or_port_list`	Specifies the list or collection of cell output pins. The unateness specified by the `set_sense` command is applied to clocks propagating through the specified point.
`-positive`	Specifies that negative phase filtered clocks pass through pins in the *`pin_list`*.
`-stop_propagation`	Stops the propagation of specified clocks and clock source data paths passing through the specified pins.

`-type type_list`	Applies clock sense to object types specified in the *`type_list`*. The type list options are - `all`, `clock`, `data`, `clock_source_data`, `default_data`. These are explained below: • `all`: Stops all phases from propagating. This is equivalent to specifying both the `clock` and `data` options. • `clock`: Stops clock phases launched by clock(s) specified in the *`clock_list`* from propagating. • `data`: Stops all data phases launched by clock(s) specified in *`clock_list`*, starting from the pins/ports specified in *`pin_or_port_list`*. The objects in the *`pin_or_port_list`* can be selected anywhere in the design - clock/data network. • `clock_source_data`: Stops only clock source data phases from propagating. The combination of `clock_source_data` and `clock` is equivalent to the existing `-stop_propagation` option. The `-type` parameter extends the capability of the `set_clocks_sense` command to selectively block data timing phases. This parameter provides a more generalized use model for selecting among all types of paths that should be filtered. The `-type` parameter is mutually exclusive with the `-positive`, `-negative`, and `-stop_propagation` options.

Examples

- The following command allows the positive phase clock CLK1 to pass through CLKBUF1/Z and filters out the negative phase clock CLK1:
  ```
  set_sense -positive -clocks CLK1 CLKBUF1/Z
  ```
- The following command stops clock phases starting from pin1:
  ```
  set_sense pin1 -type {clock}
  ```
- The following command stops data phases based only on CK1 and CK2 starting from pin1:
  ```
  set_sense pin1 -type {data} -clocks {CK1 CK2}
  ```
- The following command stops only the clock source data phases starting from pin1:
  ```
  set_sense pin1 -type {clock_source_data}
  ```
- The following command stops the clock and clock source data phases:
  ```
  set_sense pin1 -type {clock clock_source_data}
  ```
  or
  ```
  set_sense pin1 -stop_propagation
  ```

Related Commands

- `reset_sense`

reset_sense

```
reset_sense
[-help]
[-all | [[-clocks <clock_list>] <pin_or_port_list>]]
```

Resets clock sense constraint(s) specified using the `set_sense` command.

Parameters

`-help`	Prints out the command usage.
`-all`	Resets all the specified `clock_sense` constraints.
`-clocks` *clock_list*	Specifies the list of clocks for which the clock sense constraints are to be removed. The clocks must have previously been defined using the `set_sense -clocks` parameter.
pin_or_port_list	Specifies the list or collection of cell output pins or ports for which the clock sense constraints will be removed.

Examples

- The following command resets all the previously defined `clock_sense` constraints from a design:

  ```
  tempus > reset_sense -all
  ```

Related Commands

- `set_sense`

40

Timing Debug Commands

- analyze_paths_by_basic_path_group
- analyze_paths_by_bottleneck
- analyze_paths_by_clock_domain
- analyze_paths_by_critical_false_path
- analyze_paths_by_drv
- analyze_paths_by_hier_port
- analyze_paths_by_hierarchy
- analyze_paths_by_view
- create_histogram_png
- create_path_category
- delete_path_category
- dump_histogram_view
- highlight_timing_report
- load_path_categories
- load_timing_debug_report
- open_gtd
- save_path_categories
- write_category_summary
- write_path_list_summary
- write_text_timing_report

analyze_paths_by_basic_path_group

```
analyze_paths_by_basic_path_group
[-help]
[-master master_category_name]
[-predefined]
```

Categorizes the timing paths based on basic path groups. The basic path groups are fundamental to timing analysis and help the user in observing the timing information of different basic path groups easily. For instance, if user is only concerned about the paths from registers to registers, the user can analyze the paths in the category REG -> REG. The basic path groups categories are:

- Register to register
- Input to register
- Register to output
- Input to output

The following categories are created:

- `REG->REG`
- `IN->REG`
- `REG->OUT`
- `IN->OUT`

Timing debug can also convert user defined path groups into path categories. The command can be applied wherever the user loads the machine readable timing report.

Note: If your design does not have paths belonging to a type of path group, the software will not create that category.

For more information on creating path categories, see "Creating Path Categories" in the Debugging Timing Results chapter of *User Guide*.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `analyze_paths_by_basic_path_group` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man analyze_paths_by_basic_path_group`
`-master` *`master_category_name`*	Specifies that the analysis should be performed only for the specified master category.
`-predefined`	Creates path categories for user defined path groups.

Examples

- The following command create path categories based on basic path groups:
  `analyze_paths_by_basic_path_group`
  The following categories are created: REG->REG, IN->REG, REG->OUT, IN->OUT.
- The following command create path categories based on predefined path group "critical" :

```
group_path -name critical -from {ff*/CLK } -to {ff*/D} #create predefined path
group
analyze_paths_by_basic_path_group -predefined
```

  The category "critical" will be created based on the predefined path group name.

Related Topics

The Timing Budgeting chapter of the *Innovus User Guide* document.

analyze_paths_by_bottleneck

```
analyze_paths_by_bottleneck
[-help]
{-category n | -incr_delay {worst|total}}
[-path_num m]
[-max_slack float]
[-min_slack float]
[-master master_category_name]
```

Creates categories according to the bottleneck analysis of the critical paths and fixes the violation according to the bottleneck information. Bottleneck Analysis detects the instances that are often involved in the critical paths in the design. Any change in timing for these instances impacts multiple critical paths in the design. Therefore you can use this information to change your design (floorplan, placement, constraints etc.) around these instances. The bottleneck analysis computes the number of occurrences that an instance appears in the paths that you display in the Timing Debug window, and provides the following output:

- Colors most used instance based on number of occurrences.
- Creates *n* number of path categories for the instances with the highest number of occurrences. *n* is the number you specify using the `-category` parameter.

Users can also perform bottleneck analysis of the critical paths based on incremental delay. Categories will be created based on the worst *m* paths with:

- The biggest worst `incr_delay` contribution
- The biggest total `incr_delay` contribution

The command can be applied wherever the user loads the machine readable timing report. To delete the path categories created by this command, use `delete_path_category`.

For more information on creating path categories, see "Creating Path Categories" in the Debugging Timing Results chapter of *User Guide*.

Note: The categories created by `analyze_paths_by_bottleneck` cannot be saved using the `save_path_categories` command. Additionally, the related sub-categories will not be saved.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `analyze_paths_by_bottleneck` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man analyze_paths_by_bottleneck`
`-category` *n*	Specifies the number of categories to be created for most occurring instances.
`-incr_delay {worst|total}`	Specifies that the categories are created based on worst `incr_delay` contribution or total `incr_delay` contribution.
`-master` *master_category_name*	Specifies that the analysis should be performed only for the specified `master` category.
`-max_slack` *float*	Specifies the max slack value of the critical paths for bottleneck analysis based on incr_delay.
`-min_slack` *float*	Specifies the min slack value of the critical paths for bottleneck analysis based on incr_delay.
`-path_num` *m*	Specifies the number of critical paths to be analyzed when performing bottleneck analysis based on incr_delay.

Examples

- The following command creates path categories based on bottleneck analysis of the critical paths on the instances:
  ```
  analyze_paths_by_bottleneck -category 2
  ```
  The following categories are created: `BOT_instance1` and `BOT_instance2`. You can change the number of categories.
- The following command creates path categories based on bottleneck analysis of the critical 20 paths on the worst incr_delay:
  ```
  analyze_paths_by_bottleneck -path_num 20 -incr_delay worst
  ```
  The following category is created: `BOT_SI_WORST`.

Related Topics

The Timing Budgeting chapter of the *Innovus User Guide* document.

analyze_paths_by_clock_domain

```
analyze_paths_by_clock_domain
[-help]
[-include_edges]
[-include_views]
[-include_mode]
[-master master_category_name]
```

To find the possible timing issues in clock combinations, the command creates categories according to launch clocks - capture clocks combinations.

Following categories are created:

- Paths with clock fully contained in a single domain, clk1.
- Paths with clocks starting one clock domain and ending at another, clk1-> clk2.

The command can be applied wherever the user loads the machine readable timing report by the `load_timing_debug_report` command.

For more information on creating path categories, see "Creating Path Categories" in the Debugging Timing Results chapter of *User Guide*.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `analyze_paths_by_clock_domain` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man analyze_paths_by_clock_domain`
`-include_edges`	Considers leading or trailing edge of the clock while defining categories.
`-include_mode`	Considers constraint mode while defining categories.
`-include_views`	Considers analysis views while defining categories.
`-master` *`master_category_name`*	
	Specifies that the analysis should be performed only for the specified master category.

Examples:

- The following command creates path categories based on clock domain analysis of the critical paths , and considers the edges of the clock:
  ```
  analyze_paths_by_clock_domain -include_edges
  ```
- The following command creates path categories based on clock domain analysis of the critical paths , and considers the analysis views:
  ```
  analyze_paths_by_clock_domain -include_views
  ```

Related Topics

- The Timing Budgeting chapter of the *Innovus User Guide* document.
- `load_timing_debug_report`

analyze_paths_by_critical_false_path

```
analyze_paths_by_critical_false_path
[-help]
[-critical_false_paths {false_path_only | exclude_false_path}]
[-master string]
```

Identifies violating false paths, which are paths that appear as violations but cannot be exercised due to the structure of the netlist, and creates a category that contains them. With this command the user can, in advance, categorize the critical false paths together to analyze them.

The command can be applied wherever the user loads the machine readable timing report.

For more information on creating path categories, see "Creating Path Categories" in the Debugging Timing Results chapter of *User Guide*.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `analyze_paths_by_critical_false_path` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man analyze_paths_by_critical_false_path`
`-critical_false_paths`	Specifies which paths to analyze. You can select one of the following options: • `false_path_only` Includes only false paths when creating the category. • `exclude_false_path` Excludes false paths when creating the category. *Default*: `false_path_only`
`-master string`	Specifies that the analysis should be performed only for the specified master category.

Examples:

- The following command creates path categories based on the critical false paths:
  `analyze_paths_by_critical_false_path -critical_false_path false_path_only`
- The following category is created: `default_report_false_path_only`

Related Topics

The Timing Budgeting chapter of the *Innovus User Guide* document.

analyze_paths_by_drv

```
analyze_paths_by_drv
[-help]
[[-load_cap_file capacitance_file] | [-generate_cap_file fileName]]
[[-load_tran_file transition_file] | [-generate_tran_file fileName]]
[-load_fanout_file fanout_file | [-generate_fanout_file fileName]]
[-master master_category_name]
```

Reads or generates a report containing max transition, max capacitance, and max fanout violations. Paths that are affected by the selected DRV type are grouped into a category. This command can be used if the user wants to debug the timing issues caused by DRV.

The command can be applied wherever the user loads the machine readable timing report.

For more information on creating path categories, see "Creating Path Categories" in the Debugging Timing Results chapter of *User Guide*.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `analyze_paths_by_drv` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man analyze_paths_by_drv`
`-load_cap_file` *`capacitance_file`*	
	Reads a file containing a list of paths containing max capacitance violations.
`-load_fanout_file` *`fanout_file`*	
	Reads a file containing a list of paths containing max fanout violations.
`-load_tran_file` *`transition_file`*	
	Reads a file containing a list of paths containing max transition violations.
`-generate_cap_file` *`fileName`*	
	Generates a file containing a list of paths containing max capacitance violations.
`-generate_fanout_file` *`fileName`*	
	Generates a file containing a list of paths containing max fanout violations.
`-generate_tran_file` *`fileName`*	
	Generates a file containing a list of paths containing max transition violations.
`-master` *`master_category_name`*	
	Specifies that the analysis should be performed only for the specified master category.

Examples:

The following command generates DRV report files, then creates path categories based on the critical paths which are affected by DRV:

```
analyze_paths_by_drv -generate_cap_file capViolation.rpt

-generate_tran_file tranViolation.rpt
```

```
-generate_fanout_file fanoutViolation.rpt
```

Related Topics

The Timing Budgeting chapter of the *Innovus User Guide* document.

analyze_paths_by_hier_port

```
analyze_paths_by_hier_port
[-help]
{{-gen_hier_file string [-out_file string]} | {-load_hier_file string}}
[-master string]
```

Helps you to automatically group paths based on each hierarchical port of the given instances. You can use this command to perform the timing results analysis of a hierarchical design. For example, while looking at the setup/hold time at the same time of each port.

The command can be applied wherever the user loads timing report. To delete the path categories created by this command, use `delete_path_category`. For more information on creating path categories, see "Creating Path Categories" in the Debugging Timing Results chapter of *Innovus User Guide*.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `analyze_paths_by_hier_port` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man analyze_paths_by_hier_port`
`-gen_hier_file` *string*	
	Generates hier ports fanin/fanout file when the database is available based on the given InstanceName_list. Then uses that file to create the categories.
`-load_hier_file` *string*	
	Uses a previously saved hier ports fanin/fanout file when the database is not available.
`-master` *string*	Specifies that the timing paths should be categorized only for the specified master category.
`-out_file` *string*	Specifies the generated hier ports fanin/fanout file name.

Example

The following command categorizes paths based on hierarchical port of the instance `DTMF_INST/TDSP_CORE_INST`:

```
analyze_paths_by_hier_port -gen_hier_file DTMF_INST/TDSP_CORE_INST
```

Related Topics

The Debugging Timing Results chapter of the *Innovus User Guide* document.

analyze_paths_by_hierarchy

```
analyze_paths_by_hierarchy
[-help]
[-master string]
{-fp_file string [-include_macros ]} | {-use_current_floorplan [-all ] [-types string]
[-port2port ] [-port2macro ] [-port2bbox ] [-port2hinst ] [-port2stdcell ] [-
port2instgrp ] [-macro2port ] [-macro2macro ] [-macro2bbox ] [-macro2hinst ] [-
macro2stdcell ] [-macro2instgrp ] [-bbox2port ] [-bbox2macro ] [-bbox2bbox ] [-
bbox2hinst ] [-bbox2stdcell ] [-bbox2instgrp ] [-hinst2port ] [-hinst2macro ] [-
hinst2bbox ] [-hinst2hinst ] [-hinst2stdcell ] [-hinst2instgrp ] [-stdcell2port ] [-
stdcell2macro ] [-stdcell2bbox ] [-stdcell2hinst ] [-stdcell2stdcell ] [-
stdcell2instgrp ] [-instgrp2port ] [-instgrp2macro ] [-instgrp2bbox ] [-instgrp2hinst ]
[-instgrp2stdcell ] [-instgrp2instgrp ]} | {-partitions string | -macros string}
```

Creates categories according to the hierarchical characteristics of the paths. User can use either a user specified floorplan file or current floorplan file of the design in memory and can create categories according to the hierarchy of the design. This way, the paths can be narrowed down to small group.

Use Floorplan Plan:

The software automatically extracts partition-boundary information from guides, regions, and fences in the floorplan file. Optionally you can specify option to sort paths based on the macros in the floorplan file.

- All paths fully contained in a single partition, Ptn1.
- Paths starting in one partition and ending in another partition, Ptn1–>Ptn2.
- Paths spanning several partitions, Ptn1–> Ptn2–>Ptn3.
- Top-level inputs to partitions, INPUT–>Ptn1.
- Paths from partitions to top-level outputs, Ptn1–>OUTPUT.

Use Current Floorplan:

This method categorizes paths based on the current floorplan of the design in memory. It will rely on the grouped/ungrouped as a basis for what level to view the hierarchy as well as what levels to create timing categories .

Grouped/ungrouped status of an Hinst means whether or not its children are shown on the floorplan view.

It categorizes paths starting and ending on/within these hierarchical entities:

- Port
- Hard Macro (ungrouped)
- Black Box (ungrouped)
- Hinst (regular, partition, Black Blob, or Flex Model) (highest grouped)
- Stdcell (ungrouped)

For more information on creating path categories, see "Creating Path Categories" in the Debugging Timing Results chapter of *User Guide*.

Parameters

`-all`	Creates categories of paths within all types *Default:* All types are categorized.
`-bbox2instgrp`	Creates categories of paths from black box to instance group. *Type:* Bool, optional
`-bbox2port`	Creates categories of paths from black box to port.
`-bbox2macro`	Creates categories of paths from black box to macro.
`-bbox2bbox`	Creates categories of paths from black box to black box.
`-bbox2hinst`	Creates categories of paths from black box to hierarchical instance.
`-bbox2stdcell`	Creates categories of paths from black box to standard cell.
`-fp_file floorplan_file`	Specifies the name of the floorplan file. *Default:* Uses the current floorplan file.

`-help`	Outputs a brief description that includes the type and default information for each `analyze_paths_by_hierarchy` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man analyze_paths_by_hierarchy`
`-hinst2port`	Creates categories of paths from hierarchical instance to port.
`-hinst2macro`	Creates categories of paths from hierarchical instance to macro.
`-hinst2bbox`	Creates categories of paths from hierarchical instance to black box.
`-hinst2hinst`	Creates categories of paths from hierarchical instance to hierarchical.
`-hinst2instgrp`	Creates categories of paths from hierarchical instance to instance group. *Type:* Bool, optional
`-hinst2stdcell`	Creates categories of paths from hierarchical instance to standard cell.
`-include_macros`	Includes macros in addition to guides and fences for creating categories. *Default:* Macros are not considered.
`-instgrp2bbox`	Creates categories of paths from instance group to black box. *Type:* Bool, optional
`-instgrp2hinst`	Creates categories of paths from instance group to hierarchical instance. *Type:* Bool, optional
`-instgrp2instgrp`	Creates categories of paths from instance group to instance group. *Type:* Bool, optional
`-instgrp2macro`	Creates categories of paths from instance group to macro. *Type:* Bool, optional

`-instgrp2port`	Creates categories of paths from instance group to port. *Type:* Bool, optional
`-instgrp2stdcell`	Creates categories of paths from instance group to standard cell. *Type:* Bool, optional
`-macros` *macro_list*	Specifies the names of the macros to be used. Use this parameter with the `-partitions` parameter.
`-macro2instgrp`	Creates categories of paths from macro to instance group. *Type:* Bool, optional
`-master` *master_category_name*	Specifies that the timing paths should be categorized only for the specified master category.
`-partitions` *partition_list*	Specifies the names of the partitions to consider for extracting hierarchical information.
`-port2instgrp`	Creates categories of paths from port to instance group. *Type:* Bool, optional
`-port2port`	Creates categories of paths from port to port.
`-port2macro`	Creates categories of paths from port to macro.
`-port2bbox`	Creates categories of paths from port to black box.
`-port2hinst`	Creates categories of paths from port to hierarchical instance.
`-port2stdcell`	Creates categories of paths from port to standard cell.
`-macro2port`	Creates categories of paths from macro to port.
`-macro2macro`	Creates categories of paths from macro to macro.
`-macro2bbox`	Creates categories of paths from macro to black box.
`-macro2hinst`	Creates categories of paths from macro to hierarchical instance.
`-macro2stdcell`	Creates categories of paths from macro to standard cell.
`-stdcell2port`	Creates categories of paths from standard cell to port.

`-stdcell2macro`	Creates categories of paths from standard cell to macro.
`-stdcell2bbox`	Creates categories of paths from standard cell to black box.
`-stdcell2hinst`	Creates categories of paths from standard cell to hierarchical instance.
`-stdcell2instgrp`	Creates categories of paths from standard cell to instance group. *Type:* Bool, optional
`-stdcell2stdcell`	Creates categories of paths from standard cell to standard cell.
`-types type_list`	Creates categories of paths within the type_list. Valid types are: `port`, `macro`, `bbox`, `hinst`, `stdcell`
`-use_current_floorplan`	Uses the current floorplan.

Examples:

- By default, it categorizes all the paths inter/intra in the 5 hierarchical entities by following command :

  ```
  analyze_paths_by_hierarchy -use_current_floorplan -all
  ```

- The following command categorizes all paths from "port" or to "port":

  ```
  analyze_paths_by_hierarchy -use_current_floorplan -type port
  ```

Related Topics

The Timing Budgeting chapter of the *Innovus User Guide* document.

analyze_paths_by_view

```
analyze_paths_by_view
[-help]
[-prefix string]
[-master master_category_name]
```

Categorizes the timing paths based on views which are included in the timing debug file, also known as the violation report. By arranging the timing information of different views quickly, the command helps in analyzing timing paths.

For more information on creating path categories, see "Creating Path Categories" in the Debugging Timing Results chapter of *User Guide*.

The command can be applied wherever the user loads the machine readable timing report. To delete the path categories created by this command, use `delete_path_category`.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `analyze_paths_by_view` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man analyze_paths_by_view`
`-master` *`master_category_name`*	Specifies that the timing paths should be categorized only for the specified master category.
`-prefix` *`string`*	Adds a prefix to the category name.

Example

The following command categorizes paths based on views, which are included in the timing debug file:

```
analyze_paths_by_view
```

Related Topics

The Timing Budgeting chapter of the *Innovus User Guide* document.

create_histogram_png

```
create_histogram_png
[-help]
[-height integer]
-name string
[-width integer]
```

Dumps out the timing histogram image from global timing debug in the PNG format without bringing up the GUI. It should open the Global Timing Debug window before the execution of the command. Earlier, this was done when the GUI was displayed by right-clicking the histogram window and selecting Snapshot. But that worked only when the snapshot was displayed in the GUI. This command has eliminated the necessity of bringing up the GUI to dump out the timing histogram.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `create_histogram_png` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man create_histogram_png`
`-height integer`	Specifies the height of the PNG image. *Minimum:* `1` *Maximum:* `10000`
`-name string`	Specifies the name of the PNG file.
`-width integer`	Specifies the width of the PNG image. *Minimum:* `1` *Maximum:* `10000`

Related Topics

The Timing Budgeting chapter of the *Innovus User Guide* document.

create_path_category

```
create_path_category
[-help]
[-capture_latency string]
[-check_type string]
[-clock string]
[-comment string]
[-critical_false_path string]
[-delay_of_any_inst string]
[-delay_of_any_inst_not_of_celltype string]
[-delay_of_any_inst_not_of_name string]
[-delay_of_any_inst_of_celltype string]
[-delay_of_any_inst_of_name string]
[-delay_of_any_net string]
[-delay_of_any_net_not_of_name string]
[-delay_of_any_net_of_name string]
[-delay_of_every_inst string]
[-delay_of_every_inst_not_of_celltype string]
[-delay_of_every_inst_not_of_name string]
[-delay_of_every_inst_of_celltype string]
[-delay_of_every_inst_of_name string]
[-delay_of_every_net string]
[-delay_of_every_net_not_of_name string]
[-delay_of_every_net_of_name string]
[-file string]
[-from_cell string]
[-from_clock string]
[-from_clock_edge string]
[-from_inst string]
[-from_inst_pointer string]
[-from_pd string]
[-from_pin string]
[-from_port string]
[-from_port_pointer string]
[-incr_delay_of_any_net string]
[-incr_delay_of_every_net string]
[-launch_latency string]
[-master string]
[-name string]
[-not_from_cell string]
[-not_from_clock string]
```

[-not_from_clock_edge *string*]
[-not_from_inst *string*]
[-not_from_pd *string*]
[-not_from_pin *string*]
[-not_from_port *string*]
[-not_through_cell *string*]
[-not_through_inst *string*]
[-not_through_net *string*]
[-not_through_pd *string*]
[-not_through_pin *string*]
[-not_through_port *string*]
[-not_to_cell *string*]
[-not_to_clock *string*]
[-not_to_clock_edge *string*]
[-not_to_inst *string*]
[-not_to_pd *string*]
[-not_to_pin *string*]
[-not_to_port *string*]
[-number_of_insts *string*]
[-number_of_insts_not_of_celltype *string*]
[-number_of_insts_not_of_name *string*]
[-number_of_insts_of_celltype *string*]
[-number_of_insts_of_name *string*]
[-overwrite]
[-sdc *string*]
[-skew *string*]
[-slack *string*]
[-through_cell *string*]
[-through_inst *string*]
[-through_net *string*]
[-through_pd *string*]
[-through_pin *string*]
[-through_port *string*]
[-to_cell *string*]
[-to_clock *string*]
[-to_clock_edge *string*]
[-to_inst *string*]
[-to_inst_pointer *string*]
[-to_pd *string*]
[-to_pin *string*]
[-to_port *string*]
[-to_port_pointer *string*]
[-total_delay_of_insts *string*]

```
[-total_delay_of_insts_not_of_celltype string]
[-total_delay_of_insts_not_of_name string]
[-total_delay_of_insts_of_celltype string]
[-total_delay_of_insts_of_name string]
[-total_delay_of_level_of_chain_of_insttype string]
[-total_delay_of_nets string]
[-total_delay_of_nets_not_of_name string]
[-total_delay_of_nets_of_name string]
[-total_delay_of_path string]
[-total_incr_delay_of_path string]
[-uncertainty string] [-view string]
```

Creates path categories using conditions for grouping paths according to analyzed timing results. The command can be applied wherever the user loads the machine readable timing report. For more information on creating path categories, see "Creating Path Categories" in the Debugging Timing Results chapter of *User Guide*.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `create_path_category` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man create_path_category`
`-clock {both | launch | capture}`	Helps you search the Clock Tree instead of the data path. If this option is not specified, the user searches the data path by default. You can use one of the following values: • `Both`: searches on launch or capture clock paths. • `Launch`: searches only on launch clock paths • `Capture`: searches only on capture clock paths **Note:** Supports simultaneous criteria on data path.

`-comment {`*`commentText`*`}`	Specifies a comment that is associated with the category.
`-master` *`existing_category_name`*	Specifies that the new category should be created as a nested category of the specified master category. The master category should be an existing category. *Default:* If you do not specify this parameter, the category is created as a new category at the top level.
`-name` *`group_name`*	Specifies the name of the category to be created.
`-overwrite`	Overwrites any predefined category with the same name in your category list.
`-sdc {`*`file_name line_number`*`}`	Creates a category based on the SDC constraint as per the specified SDC file name and line number. Creating a path category based on an SDC constraint helps you to analyze the affect of an SDC constraint by grouping the affected paths together.
`-check_type {Setup Check | Hold Check | Clock Gating Setup Check | Library Clock Gating Setup Check | Clock Gating Hold Check | Library Clock Gating Hold Check | External Delay Assertion | Latch Borrowed Time Check | PulseWidth Check | ClockPeriod Check | Recovery Check | Removal Check}`	Indicates that the check type matches the specified option. *Type:* String, optional **Note:** Use quotes (`""`) or curly braces (`{}`) to specify the value of this option.
`-critical_false_path` *`string`*	Specifies the critical false path. *Type:* String, optional
`-file` *`string`*	Specifies the file name. *Type:* String, optional

`-from_inst	-from_inst_pointer	-from_pin	-from_cell	-from_clock	-from_port	-from_port_pointer	-from_pd` *string*	Indicates that the path must be from the specified instance, pin, cell, clock, power domain, or port pattern.
`-from_clock_edge` *string*	Indicates that the path must be from the specified clock edge. *Type:* String, optional
`-incr_delay_of_any_net` *float*	Indicates that the incremental delay of at least a net that matches the specified *float* value.
`-incr_delay_of_every_net` *float*	Indicates that the incremental delay of every net that matches the specified *float* value. *Type:* String, optional
`-not_from_inst	-not_from_pin	-not_from_cell	-not_from_clock	-not_from_port	-not_from_pd string`	Indicates that the path must not be from the specified instance, pin, cell, clock, or port pattern.
`-through_inst	-through_pin	-through_cell	-through_net	-through_port	-through_pd` *string*	Indicates that the path must be through the specified instance, pin, cell, net, power domain, or port pattern.
`-not_from_clock_edge`	Indicates that the path must not be from the specified clock edge. *Type:* String, optional
`-not_through_inst	-not_through_pin	-not_through_cell	-not_through_net	-not_through_port	-not_through_pd` *string*	Indicates that the path must not be through the specified instance, pin, power domain, cell, net, or port pattern.
`-not_to_clock_edge`	Indicates that the path must not be to the specified clock edge. *Type:* String, optional
`-to_inst	-to_inst_pointer	-to_pin	-to_cell	-to_clock	-to_port	-to_port_pointer	-to_pd` *string*	Indicates that the path must be the specified instance, pin, cell, clock, power domain, or port pattern.

`-not_to_inst	-not_to_pin	-not_to_cell	-not_to_clock	-not_to_port	-not_to_pd` *string*	Indicates that the path must not be to the specified instance, pin, cell, clock, power domain, or port pattern.
`-slack` *float*	Indicates that the slack of the path must match the specified *float* value.
`-skew` *float*	Indicates that the skew of the path must match the specified *float* value.
`-launch_latency` *float*	Indicates that the launch latency of the path must match the specified *float* value.
`-capture_latency` *float*	Indicates that the capture latency of the path must match the specified *float* value.
`-uncertainty` *float*	Indicates that the uncertainty of the path must match the specified *float* value.
`-number_of_insts` *integer*	Indicates that the number of instances in the data segment (with launch and capture clock paths) must match the specified *integer* value.
`-number_of_insts_of_name` *string_plus_integer*	Indicates that the number of instances matching the specified instance name match the specified *integer* value.
`-number_of_insts_not_of_name` *string_plus_integer*	Indicates that the number of instances not matching the specified instance name match the specified *integer* value.
`-number_of_insts_of_celltype` *string_plus_integer*	Indicates that the number of instances matching the specified celltype match the specified *integer* value.
`-number_of_insts_not_of_celltype` *string_plus_integer*	Indicates that the number of instances not matching the specified celltype match the specified *integer* value.
`-to_clock_edge`	*Type:* String, optional
`-total_delay_of_insts` *float*	Indicates that the total delay of instances matches the specified *float* value.

-total_delay_of_level_of_chain_of_insttype	*Type:* String, optional
-total_delay_of_path *float*	Indicates that the total delay of path matches the specified *float* value.
-total_delay_of_nets *float*	Indicates that the total delay of nets matches the specified *float* value.
-total_delay_of_insts_of_name *string_plus_float*	Indicates that the total delay of instances matching the specified name matches the specified *float* value.
-total_delay_of_insts_not_of_name *string_plus_float*	Indicates that the total delay of instances not matching the specified name matches the specified *float* value.
-total_delay_of_insts_of_celltype *string_plus_float*	Indicates that the total delay of instances matching the specified celltype matches the specified *float* value.
-total_delay_of_insts_not_of_celltype *string_plus_float*	Indicates that the total delay of instances not matching the specified celltype matches the specified *float* value.
-total_delay_of_nets_of_name *string_plus_float*	Indicates that the total delay of nets matching the specified name matches the specified *float* value.
-total_delay_of_nets_not_of_name *string_plus_float*	Indicates that the total delay of nets not matching the specified name matches the specified *float* value.
-total_incr_delay_of_path *float*	Indicates that the total incremental delay of path matches the specified *float* value.
-delay_of_any_inst *float*	Indicates that the delay of any instance in the design matches the specified *float* value.
-delay_of_any_net *float*	Indicates that the delay of any net in the design matches the specified *float* value.

`-delay_of_every_inst` *float*	Indicates that the delay of every instance in the design matches the specified *float* value.
`-delay_of_every_net` *float*	Indicates that the delay of every instance in the design matches the specified *float* value.
`-delay_of_any_inst_of_name` *string_plus_float*	Indicates that the delay of any instance in the design matching the specified name matches the specified *float* value.
`-delay_of_any_inst_not_of_name` *string_plus_float*	Indicates that the delay of any instance in the design not matching the specified name matches the specified *float* value.
`-delay_of_any_inst_of_celltype` *string_plus_float*	Indicates that the delay of any instance in the design matching the specified celltype matches the specified *float* value.
`-delay_of_any_inst_not_of_celltype` *string_plus_float*	Indicates that the delay of any instance in the design not matching the specified celltype matches the specified *float* value.
`-delay_of_any_net_of_name` *string_plus_float*	Indicates that the delay of any net in the design matching the specified name matches the specified *float* value.
`-delay_of_any_net_not_of_name` *string_plus_float*	Indicates that the delay of any net in the design not matching the specified name matches the specified *float* value.
`-delay_of_every_inst_of_name` *string_plus_float*	Indicates that the delay of every instance in the design matching the specified name matches the specified *float* value.
`-delay_of_every_inst_not_of_name` *string_plus_float*	Indicates that the delay of every instance in the design not matching the specified name matches the specified *float* value.
`-delay_of_every_inst_of_celltype` *string_plus_float*	Indicates that the delay of every instance in the design matching the specified celltype matches the specified *float* value.

`-delay_of_every_inst_not_of_celltype` *`string_plus_float`*	Indicates that the delay of every instance in the design not matching the specified celltype matches the specified *`float`* value.
`-delay_of_every_net_of_name` *`string_plus_float`*	Indicates that the delay of every net in the design matching the specified name matches the specified *`float`* value.
`-delay_of_every_net_not_of_name` *`string_plus_float`*	Indicates that the delay of every net in the design not matching the specified name matches the specified *`float`* value.
`-view`	*Type:* String, optional

Examples:

- The following command categorizes paths whose source clock is `vclk1`:
  ```
  create_path_category -name vclk1 -from_clock vclk1
  ```
  The category `vclk1` is created.
- The following command categorizes paths whose skew is larger than 0.2:
  ```
  create_path_category -name worst_skew -skew {> 0.2}
  ```
  The category `worst_skew` is created.
- The following command creates paths from the macros in the design:
  ```
  create_path_category -name from_RAM_128x16 -from_inst RAM_128x16
  ```
- The following command creates paths to the macros in the design:
  ```
  create_path_category -name to_RAM_128x16 -to_inst RAM_128x16
  ```
- The following command creates paths between the macros in the design:
  ```
  create_path_category -name between_RAM_128_256 -from_inst RAM_128x16 -to_inst
  RAM_256x16
  ```

Related Topics

The Timing Budgeting chapter of the *Innovus User Guide* document.

delete_path_category

```
delete_path_category
[-help]
category_name
```

Deletes the names of the selected categories.

The command can be applied when categories are created in Global Timing Debug.

For more information, see the Debugging Timing Results chapter of *User Guide*.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `delete_path_category` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man delete_path_category`
`category_name`	Specifies the name of the category that is needed to be deleted.

Example

The following command deletes the category `worst_paths`:

```
delete_path_category worst_paths
```

Related Topics

The Timing Budgeting chapter of the *Innovus User Guide* document.

dump_histogram_view

```
dump_histogram_view
[-help]
-name string
[-type string]
```

Saves (dumps) the image of the histogram shown in *Global Timing Debug* automatically during the process.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `dump_histogram_view` parameter. For a detailed description of the command and all its parameters, use the man command: man `dump_histogram_view`.
`-name string`	Specifies the name of the image file to save the histogram.
`-type string`	Specifies the type of file (image format) to be saved. Options are `bmp/jpg/gif/ppm/tif/csv`. *Default*: `bmp`

Related Topics

The Timing Budgeting chapter of the *Innovus User Guide* document.

highlight_timing_report

highlight_timing_report
`[-help]`
`[-append]`
`[-color_index integer]`
`[-file filename ]`
`[-noarrow]`
`[-path path_number | -all]`

This command enables you to highlight a specific number of paths in Global Timing Debug through user interface. The input timing report must be in a machine readable format that is understood by Global Timing Debug. For more information, see the Debugging Timing Results chapter of the *Innovus User Guide* document.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `highlight_timing_report` parameter. For a detailed description of the command and all its parameters, use the man command: `man highlight_timing_report`
`-all`	Highlights all the path in the Global Timing Debug through the user interface.
`-append`	Appends a path to the current highlight.
`-color_index integer`	Specifies the path's color index on a scale of 1 to 8.
`-file filename`	Reads the machine readable file and also specifies the name of the file. The option is of type string and is optional.
`-noarrow`	Removes the highlighted arrow. The option is of type boolean and optional.
`-path path_number`	Specifies the number of the path. The option is of type string and required. It is recommended to use positive integers for specifying the `path_number`.

Example

The following commands generate the worst 1000 paths to a machine readable report, and then highlight the top five paths in Global Timing Debug:

```
report_timing -machine_readable -max_points 1000 > foo.mtarpt
load_timing_debug_report foo.mtarpt
highlight_timing_report -file foo.mtarpt -path "1 2 3 4 5" -noarrow
```

Related Topics

The Debugging Timing Results chapter of the *Innovus User Guide* document.

load_path_categories

```
load_path_categories
[-help]
-filename filename
```

Loads the path category file in the Timing Debug form. The path category file contains category definitions that you saved in an earlier timing debug analysis. To save path categories, use the save_path_categories command.

For more information, see the Debugging Timing Results chapter of *User Guide*.

Note: Before loading the path category file, make sure that the path category name mentioned in the file does not exist in the existing path category list of the debugging session. To overwrite the existing categories, use the `create_path_category -overwrite` parameter for the path category file.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `load_path_categories` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man load_path_categories`
`-filename` *`filename`*	Specifies the name of the category file.

Example

The following command loads the path category file `category.cat`:

```
load_path_categories -filename category.cat
```

Related Topics

The Timing Budgeting chapter of the *Innovus User Guide* document.

load_timing_debug_report

```
load_timing_debug_report
[-help]
[-name report_name]
[{-proto [-additional_slack_past_wns number]  [-num_path number]}]
[-num_path]
filename
```

Loads the violation report in Innovus for debugging timing results.

Note: The violation report that you specify in this command has a special machine readable format. Use the `report_timing -machine_readable` command or `timeDesign-timingDebugReport` command to generate the machine readable violation report.

For more information on timing debug categories, see "Debugging Timing Results" in *Innovus User Guide*.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `load_timing_debug_report` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man load_timing_debug_report`
`filename`	Specifies the name of the timing debug file, also known as the violation report.
`-name`	Specifies the report name to load the machine readable timing report files incrementally. **Note:** The name can be specified using any string; it is not the file name.
`-num_path`	Specifies the number of paths to be reported in the detailed path violations.
`{-proto [-additional_slack_past_wns number]  [-num_path number]}`	
	Creates flex model categories and displays the top path of the top 8 (by default) categories. This option can be used for design that has flexModels. Use `-additional_slack_past_wns` to report all paths with slack worst than (WNS and additional specified slack value). Use `-num_path` to control the number of violation paths.

Examples:

- The following command loads the machine readable timing report `top.mtarpt`:

  ```
  load_timing_debug_report top.mtarpt
  ```

- The following commands load two machine readable timing reports `top1.mtarpt` and `top2.mtarpt` together:

  ```
  load_timing_debug_report -name default_report top1.mtarpt
  load_timing_debug_report -name default_report top2.mtarpt
  ```

  Global Timing Debug (GTD) will first load the report `top1.mtarpt`, then append `top2.mtarpt` incrementally to the current report.

Related Topics

The Timing Budgeting chapter of the *Innovus User Guide* document.

open_gtd

```
open_gtd
[-help]
```

Opens the global timing debugger for accessing the GTD files report.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `open_gtd` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man open_gtd`

save_path_categories

```
save_path_categories
[-help]
-filename filename
```

Saves the selected categories in a path category file. Use the category file to load the saved category definitions in a new timing debug session. To load the saved category definitions, use the `load_path_categories` command.

For more information, see the Debugging Timing Results chapter of *User Guide*.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `save_path_categories` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man save_path_categories`
`-filename filename`	Specifies the name of the saved category file.

Example

The following command saves the path category file `category.cat`:

```
save_path_categories -filename category.cat
```

Related Topics

The Timing Budgeting chapter of the *Innovus User Guide* document.

write_category_summary

```
  write_category_summary
[-help]
-report <string>
[[-ascend | -descend ]
[-byTNS | -byWNS ]]
[-csv | {-no_frame_fix_width [-no_split]}]
```

Writes a text file containing the following information.

- Category name
- Total number of paths
- Number of passing paths
- Number of failing paths
- Worst negative slack
- Total negative slack

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `write_category_summary` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man write_category_summary`
`-ascend`	Specifies that the report in ascending order, should specify option `-byTNS` or `-byWNS`.
`-byTNS`	The TNS value specified by the sorting key through the option `-ascend` or `-descend`.
`-byWNS`	The WNS value specified by the sorting key through the option `-ascend` or `-descend`.
`-csv`	Reports entries in csv format.

`-descend`	Specifies that the report in descending order, should specify option `-byTNS` or `-byWNS`.
`-no_frame_fix_width`	Specifies that the table columns have a fixed minimum width, but no maximum width (the data does not wrap). Column data that fits within the column's minimum width will left-justify and pad its location to fill the specified column space. Column data that is longer than the column's minimum width will take as much space in the table as needed. If a column is overwritten by the data from the preceding column, its data will begin on a new line at the correct column location.
`-no_split`	Specifies that each column follows the preceding column without splitting to a new line, if the data in one column overruns the next column. The wrapping behavior for data is controlled by the size of the display window. This format is useful for machine parsing the report. **Note:** You must specify `-no_frame_fix_width` in order to use this parameter.
`-report fileName`	Specifies the name of the category report file. *Default*: `category.rpt`

Examples

The following command writes a category summary file `category.rpt`:

```
write_category_summary -report category.rpt
```

Below is the sample report generated by the software:

Category name	Total path	Passing Path	Failing path	WNS	TNS
test_clock<view_test_	3869	768	3101	-5.699	-4802.857
100MHz_1.00V>	Clock Domain Analysis				
@->test_clock<view_test	404	55	429	-2.292	-378.432
_100MHz_1.00V>	Clock Domain Analysis				
my_clk-><view_mission	1	2	11	1.931	-1.931
_166MHz_1.08V>	Clock Domain Analysis				
my_clk_2x<view_mission	156	154	2	-.013	-0.21
_166MHz_1.08V>	Clock Domain Analysis				
cat1875	77	13	64	-3.524	-100.893
	Category of paths that cross i_1875 and start with test_clock - need to change the uncertainty value -advised --by Don				

Related Topics

The Timing Budgeting chapter of the *Innovus User Guide* document.

write_path_list_summary

```
  write_path_list_summary
[-help]
[-category string]
[-compress]
[-csv]
-report fileName
```

Writes the Path List categorized summary into a text and/or CSV file.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `write_path_list_summary` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man write_path_list_summary`
`-category string`	Specifies the category of paths to be stored into the CSV file.
`-compress`	Compresses the output file in the `.gzip` file format.
`-csv`	Reports entries in the CSV format.
`-report fileName`	Specifies the name of the output file in text, CSV, or compressed format.

Related Topics

The Timing Budgeting chapter of the *Innovus User Guide* document.

write_text_timing_report

```
write_text_timing_report
[-help]
[-mtarpt file.mtarpt | -category category_name]
[-format{{column1 width1} {column2 width2} ...}]
[-path_specification path_specification]
[-summary]
-report file.tarpt
```

Writes a text file containing information about paths that have been loaded from a `.mtarpt` file from Timing Debug, either with the `load_timing_debug_report command` or in the *Display/Generate Timing Report* form in the GUI.

Paths are selected as follows:

- If only one `.mtarpt` file is loaded and no other parameters are specified, then the tool writes all the paths contained in the loaded `.mtarpt` file.
- If there are several `.mtarpt` files loaded, you must specify the `.mtarpt` file from which the tool can select the paths, except if a category is specified.

You can use this report for performing timing analysis.

If only one machine-readable report file is loaded, then the tool writes all paths from that file into the output report. If more than one file is loaded, then the tools issues an error message.

Note: The generated report cannot be read back into the timing debug environment.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `write_text_timing_report` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man write_text_timing_report`
`-category` *`category_name`*	
	Specifies the category of paths to write to the text file. You cannot specify `-mtarpt` if you specify this parameter.
`-format{{`*`column1 width1`*`} {`*`column2 width2`*`} ...}`	
	Formats the report according to the specified column name and its width. The column-width pair specifies the column to display in the report and the width of that column. Multiple column names can be specified. *Default:* `{{name 35} {arc 10} {cell 10} {delay 10} {slew 10} {load 10} {arrival 10} {required 10}}` Available column list values are: `name arc cell delay slew load arrival required stolen fanout fedge tedge incr_delay type fpin_loc tpin_loc loc pin sum status ir_drop wire_len`
`-mtarpt` *`file.mtarpt`*	
	Specifies the name of the `.mtarpt` file to read and convert to a text file. You cannot specify this parameter if you specify `-category`.
`-path_specification` *`path_specification`*	
	Writes a single number or path range separated by blank space. A path range is two numbers separated by a minus "-" sign. You can use this parameter only if a single file is specified or loaded.
`-report` *`file.tarpt`*	
	Specifies the name of the text output file.
`-summary`	Writes each path file header (endpoint name and slack) only. Use this parameter if you do not need a detailed table.

Examples

- The following command loads machine-readable report `top.mtarpt`:

  ```
  load_timing_debug_report top.mtarpt
  ```

- The following command reads the `top.mtarpt` report and reports paths 1 through 50 in the `top_tarpt.txt` file.

  ```
  write_text_timing_report -mtarpt top.mtarpt -path_specification {1-50} -report
  top_tarpt.txt
  ```

Related Topics

The Timing Budgeting chapter of the *Innovus User Guide* document.

41

Timing Global Commands

The following commands retrieve and set the timing global variable values:

- get_global
- report_globals
- set_global

get_global

```
get_global
global_variable_name
```

Retrieves the current value of the specified timing global variable. To set the value of a timing global variable, use the set_global command.

For a list of timing global variables, see Timing Global Variables.

Example

The following command retrieves the setting for the `timing_cppr_threshold_ps` global:

```
get_global timing_cppr_threshold_ps
```

report_globals

```
report_globals
[-help]
[pattern]
[-add global_name_list]
```

Generates a list of the officially supported global variables, with their current and default values.

Note: The `report_globals` command is now obsolete. This command will continue to be supported in the current release, but will be removed in the next major release of the software. To search for global variables, you can use the `find_global` command.

Parameters

`-help`	Prints out the command usage.
`-add` *`global_name_list`*	Adds the specified global variables to the generated list of supported global variables. When you use this parameter, the global variables will be included in the generated list along with their registered and current values.
`pattern`	Reports the global variables that match the specified pattern. Pattern matching can be used for application as well as user-specified global variables (registered using the `report_globals -add` parameter).

set_global

```
set_global
global_variable_name
value
```

Sets the specified timing global variable to a specified value. To retrieve the current value of a timing global variable, use the `get_global` command.

For a list of timing global variables, see Timing Global Variables.

Example

The following command sets the `timing_cppr_threshold_ps` global:

```
set_global timing_cppr_threshold_ps 25
```

42

Timing Global Variables

- aocv_chip_size
- aocv_core_size
- distributed_mmmc_disable_reports_auto_redirection
- lib_build_asynch_de_assert_arc
- report_precision
- report_precision_capacitance
- report_precision_derate
- report_precision_power
- report_precision_sensitivity
- report_timing_format
- timing_all_registers_include_icg_cells
- timing_allow_input_delay_on_clock_source
- timing_analysis_precision_ps
- timing_aocv_analysis_mode
- timing_aocv_derate_mode
- timing_aocv_slack_threshold
- timing_aocv_stage_count_recalculate_on_timing_reset
- timing_apply_check_derate_to_external_output_delay
- timing_apply_default_primary_input_assertion
- timing_apply_exceptions_to_data_check_related_pin
- timing_apply_setup_hold_exceptions_to_data_check_related_pin
- timing_cap_unit

- timing_case_analysis_for_icg_propagation
- timing_case_analysis_for_sequential_propagation
- timing_case_analysis_propagation
- timing_check_timing_report_all_checks
- timing_check_timing_signal_level_high_to_low_threshold
- timing_check_timing_signal_level_low_to_high_threshold
- timing_clock_phase_propagation
- timing_clock_source_use_driving_cell
- timing_clock_uncertainty_from_to_precedence
- timing_collection_all_fanin_fanout_traversal_mode
- timing_collection_result_display_limit
- timing_collection_variable_assignment_compatibility
- timing_constraint_disable_min_max_input_delay_worst_casing
- timing_constraint_enable_detailed_report_invalid_begin_end_points
- timing_constraint_enable_drv_limit_override
- timing_constraint_enable_logging
- timing_constraint_enable_report_invalid_begin_end_points
- timing_constraint_enable_search_path
- timing_constraint_enable_separate_multicycle_data_checks
- timing_constraint_load_minimal_set_for_automated_eco
- timing_constraint_path_delay_exclude_check_delay_from_ignore_clock_latency
- timing_constraint_path_delay_exclude_io_delay_from_ignore_clock_latency
- timing_constraint_path_delay_exclude_unconstrained_endpoints
- timing_constraint_path_delay_include_clock_pin_endpoints
- timing_constraint_update_io_latency_averaging_mode
- timing_constraint_warn_for_timing_derate_exceeding_max_limit
- timing_context_apply_default_primary_input_assertion

- timing_context_clock_mapping_check_waveform
- timing_context_clock_mapping_percentage_tolerance
- timing_context_port_based_clock_mapping
- timing_continue_on_error
- timing_cppr_enable_mismatch_transition_mode
- timing_cppr_opposite_edge_mean_scale_factor
- timing_cppr_opposite_edge_sigma_scale_factor
- timing_cppr_opposite_edge_sigma_scale_factor_cell
- timing_cppr_opposite_edge_sigma_scale_factor_net
- timing_cppr_propagate_thru_latches
- timing_cppr_remove_clock_to_data_crp
- timing_cppr_self_loop_mode
- timing_cppr_skip_clock_reconvergence
- timing_cppr_skip_clock_reconvergence_for_unmatched_clocks
- timing_cppr_threshold_ps
- timing_cppr_transition_sense
- timing_create_clock_default_propagated
- timing_create_clock_use_ideal_slew
- timing_default_opcond_per_lib
- timing_defer_mmmc_object_updates
- timing_derate_aocv_dynamic_delays
- timing_derate_aocv_reference_point
- timing_derate_dynamic_compatibility
- timing_derate_incremental_adjust_additive_mode
- timing_derate_incremental_multiply_accumulative_mode
- timing_derate_negative_delay_backward_compatibility
- timing_derate_ocv_reference_point

- timing_derate_spatial_distance_unit
- timing_derate_voltage_scaling_mode
- timing_disable_bidi_output_timing_checks
- timing_disable_black_box_endpoints
- timing_disable_bus_contention_check
- timing_disable_clock_gating_checks
- timing_disable_clockperiod_checks
- timing_disable_constant_propagation_for_sequential_cells
- timing_disable_drv_reports_on_constant_nets
- timing_disable_floating_bus_check
- timing_disable_genclk_combinational_blocking
- timing_disable_inferred_clock_gating_checks
- timing_disable_internal_inout_cell_paths
- timing_disable_internal_inout_net_arcs
- timing_disable_lib_pulsewidth_checks
- timing_disable_library_data_to_data_checks
- timing_disable_library_tiehi_tielo
- timing_disable_netlist_constants
- timing_disable_nochange_checks
- timing_disable_non_sequential_checks
- timing_disable_output_as_clock_port
- timing_disable_parallel_arcs
- timing_disable_pulsewidth_same_edge_si_cppr_mode
- timing_disable_recovery_removal_checks
- timing_disable_report_header_info
- timing_disable_retime_clock_path_slew_propagation
- timing_disable_sdf_retain_arc_merging

- timing_disable_skew_checks
- timing_disable_test_signal_arc
- timing_disable_timing_model_latch_inferencing
- timing_disable_tristate_disable_arcs
- timing_disable_user_data_to_data_checks
- timing_driving_cell_override_library
- timing_dynamic_loop_breaking
- timing_enable_all_fanin_fanout_levels_compatibility
- timing_enable_aocv_slack_based
- timing_enable_case_analysis_conflict_warning
- timing_enable_clock2clock_clockgating_check
- timing_enable_clock_phase_based_rise_fall_derating
- timing_enable_data_through_clock_gating
- timing_enable_derating_for_pulsewidth_checks
- timing_enable_early_late_data_slews_for_setuphold_mode_checks
- timing_enable_genclk_divide_by_inherit_parent_duty_cycle
- timing_enable_genclk_edge_based_source_latency
- timing_enable_genclk_source_path_register_limit
- timing_enable_get_object_escaped_name_backward_compatible
- timing_enable_get_objects_regexp_compatibility
- timing_enable_hierarchical_get_nets_support
- timing_enable_latch_thru_mode
- timing_enable_latency_through_clock_gating
- timing_enable_mmmc_loop_handling
- timing_enable_multi_drive_net_reduction_with_assertions
- timing_enable_multifrequency_latch_analysis
- timing_enable_path_delay_to_unconstrained_endpoints_compatibility

- timing_enable_pessimistic_cppr_for_reconvergent_clock_paths
- timing_enable_power_ground_constants
- timing_enable_preset_clear_arcs
- timing_enable_pulsed_latch
- timing_enable_sdc_compatible_data_check_mcp
- timing_enable_si_cppr
- timing_enable_simultaneous_setup_hold_mode
- timing_enable_spatial_derate_mode
- timing_enable_tristate_clock_gating
- timing_enable_uncertainty_for_clock_checks
- timing_enable_uncertainty_for_pulsewidth_checks
- timing_extract_model_aocv_mode
- timing_extract_model_case_analysis_in_library
- timing_extract_model_check_arcs_as_lvf
- timing_extract_model_consider_design_level_drv
- timing_extract_model_disable_cycle_adjustment
- timing_extract_model_exhaustive_validation_dir
- timing_extract_model_exhaustive_validation_mode
- timing_extract_model_gating_as_nochange_arc
- timing_extract_model_ideal_clock_latency_arc
- timing_extract_model_include_applied_load_in_characterization_range
- timing_extract_model_include_applied_slew_in_characterization_range
- timing_extract_model_max_feedthrough_characterization_load
- timing_extract_model_non_borrowing_latch_path_as_setup
- timing_extract_model_slew_propagation_mode
- timing_extract_model_write_clock_checks_as_arc
- timing_extract_model_write_clock_checks_as_scalar_tables

- timing_extract_model_write_lvf
- timing_extract_model_write_min_max_clock_tree_path
- timing_generate_normalized_driver_waveform
- timing_generated_clocks_allow_nested_assertions
- timing_generated_clocks_inherit_ideal_latency
- timing_get_of_objects_hier_compatibility
- timing_hier_object_name_compatibility
- timing_hierarchical_context_continue_on_top_block_clock_mismatch
- timing_ignore_clock_based_path_delay_on_data_checks
- timing_ignore_lumped_rc_assertions
- timing_io_use_clock_network_latency
- timing_library_convert_async_setuphold_to_recrem
- timing_library_genclk_use_group_name
- timing_library_hold_constraint_corner_sigma_multiplier
- timing_library_hold_sigma_multiplier
- timing_library_infer_async_pins_from_timing_arcs
- timing_library_infer_cap_range_from_ccs_receiver_model
- timing_library_infer_cap_range_from_ecsm_receiver_model
- timing_library_infer_socv_from_aocv
- timing_library_read_ccs_noise_data
- timing_library_read_without_power
- timing_library_scale_aocv_to_socv_to_n_sigma
- timing_library_setup_constraint_corner_sigma_multiplier
- timing_library_setup_sigma_multiplier
- timing_library_use_trilib_drv_values
- timing_library_use_two_piece_receiver_cap
- timing_library_zero_negative_timing_check_arcs

- timing_max_transition_use_si_transition
- timing_multifrequency_clock_rounding_factor
- timing_normalized_driver_waveform_clip_linear_part
- timing_normalized_driver_waveform_weight_factor
- timing_null_collection_return_compatibility
- timing_path_based_enable_bounding_box_for_io_paths
- timing_path_based_enable_closest_common_pin_driver_for_bounding_box
- timing_path_based_enable_exhaustive_depth_bounded_by_gba
- timing_path_based_enable_report_launch_clock_path
- timing_path_based_enable_verbose_mode
- timing_path_based_exhaustive_enable_design_coverage
- timing_path_based_exhaustive_max_paths_limit
- timing_path_based_report_analysis_summary_max_paths_limit
- timing_pba_exhaustive_path_nworst_limit
- timing_pll_clock_use_driving_cell
- timing_prefix_module_name_with_library_genclk
- timing_propagate_latch_data_uncertainty
- timing_property_arrival_clocks_consider_clock_source_data_phase
- timing_property_arrival_clocks_consider_data_phase
- timing_property_arrival_clocks_consider_latency_phase
- timing_property_arrival_window_enable_tcl_dict_format
- timing_property_clock_used_as_data_unconstrained_clock_source_paths
- timing_property_return_null_collection_with_quiet
- timing_rail_swing_checks_high_voltage_threshold
- timing_rail_swing_checks_low_voltage_threshold
- timing_read_library_without_ecsm
- timing_read_library_without_sensitivity

- timing_recompute_sdf_in_setuphold_mode
- timing_reduce_multi_drive_net_arcs
- timing_reduce_multi_drive_net_arcs_threshold
- timing_remove_clock_reconvergence_pessimism
- timing_report_analysis_summary_csv_extended_new_format
- timing_report_begin_end_pair_max_path_limit
- timing_report_check_timing_unconstrained_endpoints_due_to_constants
- timing_report_clock_pin_as_begin_point
- timing_report_constraint_enable_extended_drv_format
- timing_report_constraint_format
- timing_report_constraint_rise_fall_clock_period_check
- timing_report_constraint_use_infinity_slack_for_unconstrained
- timing_report_disable_max_paths_per_group
- timing_report_drv_enable_clock_source_as_clock
- timing_report_drv_enable_frequency_per_view
- timing_report_drv_enable_ghz_notation_for_drv_fields
- timing_report_drv_enable_slew_threshold_scaling
- timing_report_drv_use_worst_timing_slack
- timing_report_enable_clock_unrolling
- timing_report_enable_cppr_point
- timing_report_enable_lead_trail_to_rise_fall_map
- timing_report_enable_markers
- timing_report_enable_max_capacitance_drv_for_constants
- timing_report_enable_max_path_limit_crossed
- timing_report_enable_report_clock_timing_across_clock_pin
- timing_report_enable_si_debug
- timing_report_enable_slack_summary_at_bottom

- timing_report_enable_unique_pins_multiple_capture_clock_paths
- timing_report_enable_use_valid_start_end_points
- timing_report_generated_clock_info
- timing_report_group_based_mode
- timing_report_max_transition_check_using_nsigma_slew
- timing_report_property_fastest_clock_consider_data_phase
- timing_report_pulse_width_matching_launch_capture_paths
- timing_report_redirect_message_types
- timing_report_retime_formatting_mode
- timing_report_skip_constraint_loop_check
- timing_report_socv_summary_mean_sigma
- timing_report_timing_header_detail_info
- timing_report_unconstrained_path_early_late_header
- timing_report_unconstrained_paths
- timing_report_use_receiver_model_capacitance
- timing_report_use_worst_parallel_cell_arc
- timing_resolve_driver_conflicts
- timing_sdf_adjust_negative_setuphold
- timing_sdf_enable_setuphold_scond_ccond
- timing_self_loop_paths_no_skew
- timing_self_loop_paths_no_skew_max_depth
- timing_self_loop_paths_no_skew_max_slack
- timing_set_clock_source_to_output_as_data
- timing_set_nsigma_multiplier
- timing_set_scaling_for_negative_checks
- timing_set_scaling_for_negative_delays
- timing_socv_preserve_variation_with_annotations

- timing_socv_rc_variation_mode
- timing_socv_statistical_min_max_mode
- timing_socv_view_based_nsigma_multiplier_mode
- timing_spatial_derate_chip_size
- timing_spatial_derate_distance_mode
- timing_suppress_escape_characters
- timing_suppress_ilm_constraint_mismatches
- timing_time_unit
- timing_timing_window_pessimism_removal_include_si_delay
- timing_use_clock_pin_attribute_for_clock_net_marking
- timing_use_incremental_si_transition
- timing_use_latch_early_launch_edge
- timing_use_latch_time_borrow
- timing_use_verilog_for_model_netlist
- timing_waveform_aware_pulse_width_checks_high_voltage_level
- timing_waveform_aware_pulse_width_checks_low_voltage_level
- timing_write_sdf_no_escape_backslash_control
- write_global_slack_worst_trigger_path_on_clocks

aocv_chip_size

`aocv_chip_size` *`real_number`*

Default: Automatically calculated from SPEF file, if one is loaded.

Specifies the diagonal length of the chip, in microns.

The chip and core sizes specified with the `aocv_chip_size` and `aocv_core_size` global variables take priority over chip and core size values derived from the SPEF file, or through placement. In the case that there is no SPEF file loaded, or placement information available:

- If you specify `aocv_chip_size` but not `aocv_core_size`, the software considers the core size to

be equal to the `aocv_chip_size` value.

- If you specify `aocv_core_size` but not `aocv_chip_size`, the software ends the AOCV analysis and generates an error message.
- If both `aocv_chip_size` and `aocv_core_size` are not specified, the software ends the AOCV analysis and generates an error message.

To set this global variable, use the set_global command.

aocv_core_size

`aocv_core_size` *`real_number`*

Default: Automatically calculated from SPEF file, if one is loaded.

Specifies the diagonal length of the core area, in microns.

The chip and core sizes specified with the `aocv_chip_size` and `aocv_core_size` global variables take priority over chip and core size values derived from the SPEF file, or through placement. In the case that there is no SPEF file loaded, or placement information available:

- If you specify `aocv_chip_size` but not `aocv_core_size`, the software considers the core size to be equal to the `aocv_chip_size` value.
- If you specify `aocv_core_size` but not `aocv_chip_size`, the software ends the AOCV analysis and generates an error message.
- If both `aocv_chip_size` and `aocv_core_size` are not specified, the software ends the AOCV analysis and generates an error message.

To set this global variable, use the set_global command.

distributed_mmmc_disable_reports_auto_redirection

```
distributed_mmmc_disable_reports_auto_redirection
{true | false}
```

Default: `false`

Allows reports to be generated in the specified location.

When set to `true`, the software does not allow the reports for different views to get overwritten. When set to `false`, the reports are generated automatically in the DMMMC view directory structure.

To set this global variable, use the `set` command.

lib_build_asynch_de_assert_arc

```
lib_build_asynch_de_assert_arc
{true | false}
```

Default: `true`

Controls whether input to output arcs from the preset or clear pins transitioning to inactive state are included when the timing system is initialized.

When the control signal to the preset or clear pins transitions to inactive state (deassertion), in some cases, the output pin can subsequently transition. In the case of edge-triggered flip-flops, the `removal` timing check (a type of hold check) prevents the preset or clear from deasserting too close to the active edge of the clock; therefore, there is usually no chance of the output changing, and no need to have an additional arc to model this transition.

However, in the case of latches, it is possible that the `removal` constraint will still allow the signal to deassert when the latch is in transparent mode. If the input state of the latch is not equivalent to the state previously forced by the preset or clear, an output transition results. In these situations, an additional arc is required to model this transition. A similar corner-case can result when both preset and clear signals are asserted, and one of them is subsequently released.

Note: The `timing_enable_preset_clear_arcs` global variable has higher priority than `lib_build_asynch_de_assert_arc`. If both of these global variables are set to `true`, then all the asynchronous arcs are recognized. If `timing_enable_preset_clear_arcs` is set to `true` and `lib_build_asynch_de_assert_arc` is set to `false`, then only the assert asynchronous arcs are recognized. If `timing_enable_preset_clear_arcs` is set to `false`, then both the assert and de-assert asynchronous arcs will not be recognized.

> Because the `timing_enable_preset_clear_arcs` and `lib_build_asynch_de_assert_arc` global variables affect how the timing graph is built, it is important to set the desired state before performing any timing analysis.

To set this global variable, use the set_global command.

report_precision

`report_precision` *`integer`*

Default: `3`

Controls the number of digits appearing after the decimal point in the `report_timing`, `report_net`, `report_cell_instance_timing`, and `report_clocks` reports. The maximum value of *`integer`* is `8`.

To set this global variable, use the `set_global` command.

report_precision_capacitance

`report_precision_capacitance` *integer*

Default: 3

Controls the number of digits appearing after the decimal point in load values in the timing report output.

To set this global variable, use the `set` command.

report_precision_derate

`report_precision_derate` *integer*

Default: 3

Controls the number of digits appearing after the decimal point in derate values in the timing report output.

To set this global variable, use the `set` command.

report_precision_power

`report_precision_power` *integer*

Default: 3

Controls the number of digits appearing after the decimal point in power values in the timing report output.

To set this global variable, use the `set` command.

report_precision_sensitivity

`report_precision_sensitivity` *integer*

Default: 3

Controls the number of digits appearing after the decimal point in sensitivity values in the timing report output.

To set this global variable, use the `set` command.

report_timing_format

`report_timing_format {column_list}`

Specifies a `report_timing` format. See `report_timing -format` for more information.

Valid format columns are:

adjustment, abs_delay_err, abs_slew_err, annotation, aocv_adj_stages, aocv_derate, aocv_weight, arc, arrival, arrival_mean, arrival_sigma, cell, delay, delay_mean, delay_sigma, direction, edge, fanin, fanout, flags, hpin, incr_delay, instance, instance_location, latch_window, load, module, net, pct_delay_err, pct_slew_err, phase, phys_info, pin, pin_load, pin_location, power_domain, required, retime_delay, retime_delay_mean, retime_delay_sigma, retime_slew, retime_slew_mean, retime_slew_sigma, stage_count, slew, slew_mean, slew_sigma, socv_derate, spice_arrival, spice_delay, spice_slew, stolen, timing_point, total_derate, user_derate, voltage, when_cond, wire_load, wlmodel

For details on these format columns, refer to section Report Timing Column List.

Depending on the options specified, this global variable also enables the display of slew mean and retime slew mean details in the timing report.

For example, the following command tells `report_timing` to only list the instance, arc, delay, slew, arrival times in the report:

```
set_global report_timing_format {instance arc delay slew arrival}
```

To set this global variable, use the `set_global` command.

timing_all_registers_include_icg_cells

```
timing_all_registers_include_icg_cells
{true | false}
```

Default: `true`

Includes integrated clock gating (ICG) cell information from the `all_registers` report output.

When set to `false`, the global does not allow the `all_registers` report output to display ICG cell details.

To set this global variable, use the `set` command.

timing_allow_input_delay_on_clock_source

```
timing_allow_input_delay_on_clock_source
{true | false}
```

Default: `false`

When set to `true`, this global variable allows the software to apply input delay constraints on a pin where a clock was previously asserted.

To set this global variable, use the set_global command.

timing_analysis_precision_ps

```
timing_analysis_precision_ps
{0.1 | 0.01}
```

Default: 0.01 (in ps)

Allows you to set the delay precision value for the `write_sdf` command.

The `write_sdf` command shows a double digit value after the decimal point by default. You can use this global variable for more accuracy.

To set this global variable, you can use the `set` command.

timing_aocv_analysis_mode

```
timing_aocv_analysis_mode
{launch_capture | clock_only | separate_data_clock | combine_launch_capture}
```

Default: `launch_capture`

Controls handling of AOCV analysis modes.

When set to `launch_capture`, AOCV analysis will be performed for both data and clock paths, and the AOCV derating factor will be calculated based on both clock and data path stages.

When set to `clock_only`, AOCV analysis will be performed for clock paths only, and the AOCV derating factor will be calculated based on the clock path stage depth only.

When set to `separate_data_clock`, the software calculates the AOCV stage count for clock paths and data paths separately. Both clock and data paths have related AOCV derating factors.

When set to `combine_launch_capture`, the software calculates AOCV stage count using combined depth for launch and capture paths. This is achieved by summation of path depth on the launch and capture paths.

To set this global variable, use the `set` command.

timing_aocv_derate_mode

```
timing_aocv_derate_mode
{aocv_multiplicative | aocv_additive}
```

Default: `aocv_multiplicative`

Controls the AOCV derating mode.

When set to `aocv_multiplicative`, the derating factor will be calculated as AOCV derating * OCV derating, which is set using the `set_timing_derate` command.

When set to `aocv_additive`, the derating factor will be calculated as AOCV derating + OCV derating values.

When you use this global variable, the `report_timing` command shows the `total_derate` column in the timing report output, which allows you to view and cross-check the calculated total derate factor.

To set this global variable, use the `set_global` command.

timing_aocv_slack_threshold

`timing_aocv_slack_threshold integer`

Default: 0

Data Type: double

Analyzes AOCV slack that is based at the specified threshold.

To use this global variable, use the `set` command.

timing_aocv_stage_count_recalculate_on_timing_reset

```
timing_aocv_stage_count_recalculate_on_timing_reset
{true | false}
```

Default: false

Re-calculates the AOCV stage counts when timing is reset.

To set this global variable, use the `set` command.

timing_apply_check_derate_to_external_output_delay

```
timing_apply_check_derate_to_external_output_delay
{true | false}
```

Default: `false`

Applies check derate to output external delay.

The `set_timing_derate -cell_check` parameter by default does not apply check derate to output external delay (e.g., `set_ouput_delay`). To enable this feature, you can set the `timing_apply_check_derate_to_external_output_delay` timing global variable to `true`. The global applies cell (check) derate to the top boundary port.

To set this global variable, use the set_global command.

timing_apply_default_primary_input_assertion

```
timing_apply_default_primary_input_assertion
{true | false}
```

Default: `true`

When set to `true`, primary input and bidirectional ports that do not have an explicit arrival time specified (through the `set_input_delay` constraint), are provided a default arrival time. The arrival is referenced to an internal default clock that is indicated by the `@` sign in various timing reports.

If a `set_input_delay` constraint is assigned to the input port, but the `-clock` parameter was not specified, the software assumes the constraint to be with reference to the default clock.

When set to `false`, inputs ports with no explicit `set_input_delay` are unconstrained for sequential timing analysis.

To set this global variable, use the set_global command.

timing_apply_exceptions_to_data_check_related_pin

```
timing_apply_exceptions_to_data_check_related_pin
{true | false}
```

Default: `true`

Controls if false path assertions which block the data path to the related (-from) reference pin of the data-to-data check disable the data-to-data check.

When set to `false`, the data-to-data checks remain valid when a false path assertion blocks the data path to the related (-from) pin.

When set to `true`, any false path assertion which blocks the data path to the related (-from) pin of the data-to-data check also causes the check to be disabled.

The global variable has no effect on the data-to-data checks with a reference clock, where an exception blocks the clock reference signal at the reference pin. The data-to-data check will always be disabled in this situation.

To set this global variable, use the `set` command.

Examples

- In the following example, the data-to-data check is disabled:

  ```
  set timing_apply_exceptions_to_data_check_related_pin true
  set_data_check -from portA -to portB
  set_false_path -to portA
  ```

- In the following example, the data-to-data check is valid. This is the default situation:

  ```
  set timing_apply_exceptions_to_data_check_related_pin false
  set_data_check -from portA -to portB
  set_false_path -to portA
  ```

- In this case, the data check is always disabled. and is not affected by this global variables' settings:

  ```
  set_data_check -from portA -to portB
  set_false_path -to [get_clocks ClkA]
  ```

  where the path to `portA` is clocked by `ClkA`.

timing_apply_setup_hold_exceptions_to_data_check_related_pin

`timing_apply_setup_hold_exceptions_to_data_check_related_pin {true | false}`

Default: `true`

Disables the early signal data-to-data paths, when false path is applied on the late reference path.

When set to `true`, the `set_false_path -setup` parameter settings on the reference path disables early signal d2d path, and vice versa. If both the `-setup` and `-hold` parameters are set, the signal early/late d2d path is disabled, irrespective of this global variables settings.

When set to `false`, the `set_false_path` exceptions set on d2d reference path do not affect data path slack. When the `set_false_path` command is set on the reference d2d path with only `-setup` (or only `-hold`) parameter, the signal path of d2d check is not affected by these exceptions.

To set this global variable, use the `set` command.

timing_cap_unit

`timing_cap_unit` *`string`*

Default:" "

Specifies the capacitance units to be used for timing constraints and timing reports. The default unit for capacitance is inferred from the timing libraries. You can override this inference by explicitly setting the value to 1pf or 1ff. Setting this variable explicitly is the same as executing the `setLibraryUnit -cap` command to set the capacitance units.

To set this variable, use the `set` command.

For example,

```
set timing_cap_unit 1pf
```

timing_case_analysis_for_icg_propagation

```
timing_case_analysis_for_icg_propagation
{false | require_seq_prop | always}
```

Default: `false`

Determines the conditions in which the constants are propagated through the integrated clock gating (ICG) cells.

The available options are described below:

Option	Description
`always`	Enables constant propagation through ICGs, regardless of the value of the `timing_case_analysis_for_sequential_propagation` global variable, or the value of `setAnalysisMode -sequentialConstProp` setting. This allows the software to have constant propagation for ICG cells, even when this is disabled for other sequential devices.
`false`	Disables constant propagation through ICG cells.
`require_seq_prop`	When the `require_seq_prop` option is used, the software will treat ICG cells like other cells in the design from the perspective of constant propagation. This means that the `timing_case_analysis_for_sequential_propagation` global variable or `setAnalysisMode -sequentialConstProp` setting will act as a master switch and will control constant propagation through ICGs also.

To set this global variable, use the `set_global` command.

Related Topics

- `setAnalysisMode`
- `timing_case_analysis_for_sequential_propagation`

timing_case_analysis_for_sequential_propagation

```
timing_case_analysis_for_sequential_propagation
{true | false}
```

Default: `false`

Calculates constants on the outputs of sequential elements. When you change setting of this global variable, each time a complete timing update is required and incremental updates are abandoned. If the outputs evaluate to a constant, the check arcs and delay arcs are disabled regardless of the setting.

When this global is set to `true` (or the `setAnalysisMode -sequentialConstProp` parameter is specified), you can set `timing_case_analysis_for_icg_propagation` global variable to `require_seq_prop` to allow propagation of constants through ICG cells.

You can use the `setAnalysisMode -sequentialConstProp` parameter to determine whether constants will be propagated through sequential elements.

Note: The `timing_case_analysis_for_sequential_propagation` global variable should not be used with the `setAnalysisMode` command. This global setting precedes previous settings and controls constant propagation through ICG cells.

To set this global variable, use the set_global command.

timing_case_analysis_propagation

```
timing_case_analysis_propagation
{true | false}
```

Default: `true`

Enables case analysis propagation.

When set to `true`, the software overrides sequential and ICG propagation behavior.

When set to `false`, the software disables case analysis propagation.

To set this global variable, use the `set` command.

timing_check_timing_report_all_checks

```
timing_check_timing_report_all_checks
{true | false}
```

Default: `false`

Enables reporting of all the checks performed by the `check_timing` command.

When set to `true`, the `check_timing` command reports all the checks performed, even if there is no violation(s) for the corresponding check type.

When set to `false`, only the check types that have any violations will be reported.

To set this global variable, use the `set` command.

timing_check_timing_signal_level_high_to_low_threshold

`timing_check_timing_signal_level_high_to_low_threshold` d*ouble*

Default: `0`

Sets the threshold value for warning the signal level difference between driver and receiver voltages, when the driver is at a higher voltage than the receiver voltage.

The software shows the signal level warnings (`check_timing signal_level`) that report the net driver/receiver pin pairs for which the pin voltage does not match. The signal level warning messages can be filtered based on these threshold values.

To set this global variable, use the `set` command.

timing_check_timing_signal_level_low_to_high_threshold

`timing_check_timing_signal_level_low_to_high_threshold` d*ouble*

Default: `0`

Sets the threshold value for warning the signal level difference between driver and receiver voltages, when the driver is at a lower voltage than the receiver voltage.

The software shows the signal level warnings (`check_timing signal_level`) that report the net driver/receiver pin pairs for which the pin voltage does not match. The signal level warning messages can be filtered based on these threshold values.

To set this global variable, use the `set` command.

timing_clock_phase_propagation

```
timing_clock_phase_propagation
{positive | negative | both}
```

Default: `both`

Allows you to select the appropriate clock phases at register clock pins when both positive and negative phases of the same clock signal are seen in the clock network. For example, a register whose clock pin is fed by a two-input XOR gate where one input is a clock port and the other input is a data port. When the value of the data signal of the XOR gate is unknown, the XOR gate can see both positive and negative phases of the clock signal.

`both`	Selects both positive and negative clock phases at the register clock pins.
`positive`	Selects positive clock phases.
`negative`	Selects negative clock phases.

To set this global variable, use the set_global command.

timing_clock_source_use_driving_cell

```
timing_clock_source_use_driving_cell
{true | false}
```

Default: `true`

When set to `false`, delay calculation in propagated clock mode will use an idealized driver model at the `create_clock` root pin or port when it calculates delays for the first stage of the clock tree. The idealized model is a ramp input with the transition time set by the system default transition, which is normally 0ns.

When this variable is set to `true`, the drive characteristics of the cell output pin that `create_clock` is asserted on will be used to calculate the first stage clock tree delays and slews. The input slew used for this driver model is whatever slew has been propagated to the cell's input pins from upstream logic. The arrival time of the clock at the root pin is not modified in any way.

If you wish to specify the input slews to be used for the clock driver cell, that is, not use the upstream propagated slews - you can use the `set_annotated_transition` constraint on the cell's input pins to control this. Setting an input transition of 0ns on the input pins can be useful when trying to correlate analysis results with other applications which do not use the full propagated transition times.

To set this global variable, use the set_global command.

timing_clock_uncertainty_from_to_precedence

```
timing_clock_uncertainty_from_to_precedence
{true | false}
```

Default: `false`

Sets the clock uncertainty precedence.

When set to `true`, the software gives higher precedence to clock-to-clock uncertainty.

To set this global variable, use the `set` command.

timing_collection_all_fanin_fanout_traversal_mode

```
timing_collection_all_fanin_fanout_traversal_mode {true | false}
```

Default: `flat`

Range: `{hierarchical flat}`

Controls how the software processes netlists during `all_fanin`/`all_fanout` command processing.

To set this global variable, use the `set` command.

timing_collection_result_display_limit

`timing_collection_result_display_limit` *integer*

Default: `100`

Limits the number of objects of a collection to be displayed in the output report.

When set to an integer value, all the commands, such as, `get_cells`, `add_to_collection`, `query_objects` etc, that display collection objects in the output report will be impacted.

When set to `-1`, all the elements in the collection will be displayed in the output.

To set this global variable, use the set_global command.

timing_collection_variable_assignment_compatibility

```
timing_collection_variable_assignment_compatibility
{true | false}
```

Default: `true`

Disables printing the names of the output of `get_*` commands.

When set to `false`, the software prints the `get_*` command output on the shell. This global variable should be used for debugging purposes only.

To set this global variable, use the `set` command.

timing_constraint_disable_min_max_input_delay_worst_casing

```
timing_constraint_disable_min_max_input_delay_worst_casing
{true | false}
```

Default: `false`

Disables the override of the maximum value (set using the `set_input_delay -max` parameter) from the specified `-min` value.

To set this global variable, use the `set` command.

timing_constraint_enable_detailed_report_invalid_begin_end_points

```
timing_constraint_enable_detailed_report_invalid_begin_end_points
{true | false}
```

Default: `false`

Enables detailed reporting of invalid begin/end points specified with the `set_false_path`/`set_multicycle_path` constraints.

When set to `false`, the software checks only the structural end points.

To set this global variable, use the `set` command.

timing_constraint_enable_drv_limit_override

```
timing_constraint_enable_drv_limit_override {true | false}
```

Default: `true`

Controls support of the DRV override flow.

When set to `true`, the software allows you to override the limits from other sources, such as, library or constraints. To use this feature, you can specify the `-override` parameter with the following commands:

- `set_max_capacitance`
- `set_max_transition`
- `set_max_fanout`
- `set_min_capacitance`
- `set_min_transition`
- `set_min_fanout`

When specified, the software forces a specific constraint value for the specified list of instance pins, ports, or library cell pins to override the maximum/minimum values set prior to these constraint settings.

To set this global variable, use the `set` command.

timing_constraint_enable_logging

```
timing_constraint_enable_logging {true | false}
```

Default: `false`

Enables saving the constraint commands in the log file.

To set this global variable, use the `set` command.

timing_constraint_enable_report_invalid_begin_end_points

```
timing_constraint_enable_report_invalid_begin_end_points
{true | false}
```

Default: `false`

Enables reporting of invalid begin points/end points specified with `set_false_path` or `set_multicycle_path` constraint.

To set this global variable, use the `set` command.

timing_constraint_enable_search_path

```
timing_constraint_enable_search_path {true | false}
```

Default: `false`

Enables search of constraint files from the search path.

To set this global variable, use the `set` command.

timing_constraint_enable_separate_multicycle_data_checks

```
timing_constraint_enable_separate_multicycle_data_checks {true | false}
```

Default: `false`

Controls multicycle path constraint application on data checks.

To set this global variable, use the `set` command.

timing_constraint_load_minimal_set_for_automated_eco

```
timing_constraint_load_minimal_set_for_automated_eco {true | false}
```

Default: `false`

Skips loading the SDC constraints that are not required, when the `signoffOptDesign` command is used. The timing data is obtained from the ECO timing database - this global variable reduces the TAT for designs with complex constraint mode files.

To set this global variable, use the `set` command.

timing_constraint_path_delay_exclude_check_delay_from_ignore_clock_latency

`timing_constraint_path_delay_exclude_check_delay_from_ignore_clock_latency {true | false}`

Default: `false`

Excludes check arc delays from slack computation.

When set to `true`, the software excludes check arc delays from slack computation for paths constrained by the `set_max_delay`/`set_min_delay` `-ignore_clock_latency` command.

To set this global variable, use the `set` command.

timing_constraint_path_delay_exclude_io_delay_from_ignore_clock_latency

`timing_constraint_path_delay_exclude_io_delay_from_ignore_clock_latency {true | false}`

Default: `false`

Excludes the input/output delays from slack computation.

When set to `true`, the software excludes the input/output delays from slack computation for paths constrained by the `set_max_delay`/`set_min_delay` `-ignore_clock_latency` command.

To set this global variable, use the `set` command.

timing_constraint_path_delay_exclude_unconstrained_endpoints

`timing_constraint_path_delay_exclude_unconstrained_endpoints`

`{true | false}`

Default: `false`

Allows you to configure the valid endpoints.

When path delay exceptions are asserted (using the `set_min_delay` or `set_max_delay` commands) with a complete `-from` and `-to` specification, the exception can be applied between any two data path pins. If the constraint assertion includes only a `-from` and/or `-through` argument, then a set of valid endpoints must be established to define the constrained paths. This global variable allows you to configure the set of valid endpoints (inferred `-to` pins) that are selected.

When set to `false`, the set of allowed endpoints is simply all of the endpoints in the combinational fanout of the `-from` and/or `-through` arguments - essentially, wherever the path tracing stops. The path delay constraint will be applied as if there were a `-to` argument specified for each endpoint.

When set to `true`, the software uses different rules to determine the valid set of inferred `-to` endpoints. Only constrained pins in the fanout of `-from` and/or `-through` arguments are considered as valid `-to` endpoints for the constraint. A constrained endpoint will have some type of timing check - such as Setup or Hold already in force on that pin. Additionally, each transition at the endpoints will be examined to determine if each is individually constrained. The path exception will only be applied to the edges that are constrained. As an example, if a register preset or a clear pin was a potential endpoint, then only the edge constrained by the recovery or removal check will have the path delay applied. To apply the path delay constraint to unconstrained endpoints while in this mode, you must specify them explicitly with the `-to`, `-rise_to`, or `-fall_to` parameters of the `set_min_delay` and `set_max_delay` constraints.

To set this global variable, use the `set` command.

timing_constraint_path_delay_include_clock_pin_endpoints

```
timing_constraint_path_delay_include_clock_pin_endpoints
{true | false}
```

Default: `false`

Considers register clock pins as end-points for path delay constraints applied upstream.

When set to `true`, the `set_max_delay` command considers unconstrained paths ending at the register clock pins.

To set this global variable, use the `set` command.

timing_constraint_update_io_latency_averaging_mode

```
timing_constraint_update_io_latency_averaging_mode
{rise_fall | rise_only}
```

Default : `rise_fall`

Controls the analysis to update IO latency for both rise and fall, or for rise only. You can set one of the options - `rise_fall`, `rise_only`.

To set this global variable, use the `set` command.

timing_constraint_warn_for_timing_derate_exceeding_max_limit

`timing_constraint_warn_for_timing_derate_exceeding_max_limit` *`double`*

Default: 100

Value Range: 0 to 100

Sets a limit on derate values. If a larger derate value is applied, the derates are still processed. However, the software issues a warning.

Note: It is recommended to specify derate values within the acceptable value range, because large derate values may cause errors during processing.

To set this global variable, use the `set` command.

timing_context_apply_default_primary_input_assertion

```
timing_context_apply_default_primary_input_assertion
{true | false}
```

Default : `false`

Propagates the phase at block level for unmapped clocks.

When set to `true`, the nets in the fanout of data ports with unmapped clocks are considered to have infinite timing windows during SI context analysis.

When set to `false`, the unmapped top clock arrivals on data ports are dropped during context analysis.

To set this global variable, use the `set` command.

timing_context_clock_mapping_check_waveform

```
timing_context_clock_mapping_check_waveform
{true | false}
```

Default : `true`

Controls the waveform checking.

To set this global variable, use the `set` command.

timing_context_clock_mapping_percentage_tolerance

```
timing_context_clock_mapping_percentage_tolerance
{true | false}
```

Default : `false`

Controls the tolerance value in clock mapping.

To set this global variable, use the `set` command.

timing_context_port_based_clock_mapping

```
timing_context_port_based_clock_mapping {true | false}
```

Default: `false`

Enables clock mapping based on block ports.

When set to `true`, clock mapping is done depending solely on clock property of the top and block clock for each clock and data port separately. For enabling N-to-1 or 1-to-N clock mapping, this global variable should be set `true`.

When set to `false`, only 1-to-1 clock mapping is done. In such cases, the following rules are used to determine clock mapping:

- Rule1: Clocks with matching property (period/duty cycle)
- Rule2: Clocks with matching name
- Rule3: Clocks present on maximum number common ports
- Rule4: Any one of the matching clocks

To set this global variable, use the `set` command.

For example,

- Consider the following illustration:

  When set to `true`, the following mapping will be seen:

Block Port	Block Clock	Mapped top clock
P1	CLK	CLK, TCLK

  When set to `false`, CLK will be mapped to CLK according to Rule 2.

Notes:

- The context_data property of block port can be accessed only when this global variable is set

`true`.

- When set to `true`, the software may increase the pessimism as the worst latency/arrival will be picked for block context analysis.
- If there are multiple block clocks that can be mapped to same top clock on the same block port, then Tempus does not perform auto mapping (when set to `true`). Whereas for the `false` setting, clock mapping is done depending on the above mentioned rules.

timing_continue_on_error

```
timing_continue_on_error
{true | false}
```

Default: `false`

When set to `true`, the software continues to process if an error occurs when any constraints are used, or during timing analysis, or when a library is read. This global variable does not have any effect on other commands, except constraints or commands related to timing analysis.

When set to `false`, if an error occurs, during timing analysis, library reading, or defined constraints, the software stops processing and exits.

To set this global variable, use the `set` command.

timing_cppr_enable_mismatch_transition_mode

`timing_cppr_enable_mismatch_transition_mode {true | false}`

Default: `true`

Enables CPPR compatibility mode when transitions mismatch at the common pin.

This global variable controls the CPPR adjustment calculations that are computed at the common pin when transitions for the launch and capture paths mismatch at the common pin. The following case scenarios are possible:

- The launch and capture clocks at the common pin are different but match the launch and capture clocks reaching the endpoint, or, launch and capture clock at the common pin are same but different from the launch and capture clocks reaching the end point.
- The launch and capture clocks at the common pin are same and match the launch and capture clocks reaching the endpoint.

The following examples show the behavior for CPPR computations when this global variable is set to `true`:

- Consider a launch path with clock CK1, late rise transition and a capture path with clock GCK1 (related to CK1), early fall transition. The CPPR adjustment is computed by considering the late fall from clock CK1 and early rise from clock GCK1.

  When set to `false` (default), the software considers late fall from clock GCK1 and early rise from clock CK1.
- Consider a launch path with clock CK1, late rise transition and a capture path with clock CK1, early fall transition. The CPPR adjustment is computed by considering the late fall from the launch phase of CK1 and early rise from the capture phase of CK1.

  When set to `false`, the software considers the late fall from capture phase of CK1 and early rise from launch phase of CK1.

The CPPR adjustment is the minimum of the two CPPR adjustments computed for the rise and fall transitions.

All the other combinations of the launch and capture transitions at the common pin for late and early paths will show similar changes to the CPPR adjustment computation - when set to `true`.

To set this global variable, use the `set` command.

timing_cppr_opposite_edge_mean_scale_factor

```
timing_cppr_opposite_edge_mean_scale_factor
{0.0 | 1.0}
```

Default: `1.0`

Controls the mean delay component for CPPR credit for opposite transition edges at a common pin.

When this global variable is specified, the software provides a finer level configuration of the CPPR credit for reducing Common Path Pessimism Removal (CPPR) in timing analysis. The software allows a certain amount of CPPR credit in the case of opposite edge transitions to be scaled to a user-specified value, that is between zero credit or full credit provided by the existing controls.

You can use `timing_cppr_opposite_edge_sigma_scale_factor` global variable for adjustment of the sigma component of the CPPR credit. In SOCV analysis mode, you can use both of these global variables to separately control the adjustment of the mean and sigma components of the CPPR credit. In regular static-timing analysis (STA) mode (without SOCV), only the mean scale factor is considered for CPPR adjustment scaling.

You can use the `set` command to set this global variable.

timing_cppr_opposite_edge_sigma_scale_factor

```
timing_cppr_opposite_edge_sigma_scale_factor
{0.0 | 1.0}
```

Default: `1.0`

Value Range: 0 to 1

Controls the sigma delay component for CPPR credit for opposite transition edges at a common pin.

This global variable provides a finer level configuration of the CPPR credit for reducing Common Path Pessimism Removal (CPPR) in timing analysis.

You can use `timing_cppr_opposite_edge_mean_scale_factor` global variable for adjustment of the mean component of the CPPR credit. In SOCV analysis mode, you can use both of these global variables to separately control the adjustment of the mean and sigma components of the CPPR credit. In static-timing analysis (STA) mode, only the mean scale factor is considered.

You can use the `set` command to set this global variable.

timing_cppr_opposite_edge_sigma_scale_factor_cell

`timing_cppr_opposite_edge_sigma_scale_factor_cell` *double*

Default: `1`

Value Range: 0 to 1

Controls the cell sigma delay component for CPPR credit for opposite transition edges at the common pin.

The scale factors are applicable only when the common pin has mismatched transitions for the launch and capture paths.

To set this global variable, use the `set` command.

timing_cppr_opposite_edge_sigma_scale_factor_net

`timing_cppr_opposite_edge_sigma_scale_factor_net` *`double`*

Default: `1`

Value Range: 0 to 1

Controls the net sigma delay component for CPPR credit for opposite transition edges at the common pin.

The scale factors are applicable only when the common pin has mismatched transitions for the launch and capture paths.

To set this global variable, use the `set` command.

timing_cppr_propagate_thru_latches

```
timing_cppr_propagate_thru_latches
{true | false}
```

Default: `false`

When set to `true`, the CPPR of a timing path crossing a latch (or latches), for example, ff1 -> lat1 -> ff2, is calculated using the common point of the origin device (ff1) and the capture device at the end of the path (ff2).

When set to `false`, the segment-based CPPR calculation is performed, considering the common point of latch (lat1) and the capture device (ff2). This CPPR setting is considered only when the latch thru analysis mode is enabled and a latch thru path exists.

To set this global variable, use the `set` command.

timing_cppr_remove_clock_to_data_crp

```
timing_cppr_remove_clock_to_data_crp
{true | false}
```

Default: `false`

When set to `true`, removes clock reconvergence pessimism (CRP) for clock source paths.

To set this global variable, use the set_global command.

timing_cppr_self_loop_mode

```
timing_cppr_self_loop_mode
{true | false}
```

Default: `true`

When set to `true` in case of self-loop paths, computes CPPR adjustment by taking the difference between early and late clock arrival time of the common point; the clock pin of the flop. A self-loop path is a path that goes from a clock pin of a flop and ends at the data pin of the same flop. However detecting self-loop paths requires runtime, and is design dependant.

When set to `false`, does not detect self-loop paths. In this case, the software computes CPPR adjustment by using the common point as the output pin driving the clock pin of the flop. This might result in better runtime, but less accurate results. The loss of accuracy is the difference between early and late delay of the interconnect portion driving the clock pin.

To set this global variable, use the set_global command.

timing_cppr_skip_clock_reconvergence

```
timing_cppr_skip_clock_reconvergence
    {true | false}
```

Default: `false`

Specifies the branch point to use for computing clock path pessimism removal (CPPR) adjustment when there is reconvergence in the clock tree.

When set to `false`, the software uses the branch point closest to the register clock pins where the clocks reconverge. When set to `true`, the software uses the farthest branch point from the register clock pins (that is, the branch point closest to the clock root pin where the clocks diverge).

Note: The `timing_cppr_skip_clock_reconvergence` and `timing_enable_pessimistic_cppr_for_reconvergent_clock_paths` global variables are mutually exclusive.
Also, the `timing_cppr_skip_clock_reconvergence` global variable has precedence over `timing_enable_pessimistic_cppr_for_reconvergent_clock_paths`.

To set this global variable, use the `set` command.

timing_cppr_skip_clock_reconvergence_for_unmatched_clocks

```
timing_cppr_skip_clock_reconvergence_for_unmatched_clocks {true | false}
```

Default: `false`

Allows the software to skip re-convergence of unmatched clocks.

When set to `true`, the software uses the farthest branch point from the register clock pins when the launching and capturing clocks are different - the re-converge points are ignored and not searched.

To set this global variable, use the `set` command.

timing_cppr_threshold_ps

`timing_cppr_threshold_ps` *`float`*

Default: `20` ps

Specifies the maximum amount of pessimism that clock path pessimism removal (CPPR) analysis is allowed to leave in the path. The unit is pico seconds. Setting this global to a specified value means that all the paths might be reported without having their pessimism removed. This global can be used to speed up CPPR analysis run time. The larger the value of the global, the faster the run time.

To set this global variable, use the set_global command.

timing_cppr_transition_sense

```
timing_cppr_transition_sense
{normal | same_transition | same_transition_expanded}
```

Default: `normal`

Specifies the transition sense of the launching and capturing clocks at the common node, to calculate clock path pessimism removal (CPPR).

When set to `normal`, the CPPR analysis computes the CPPR value even if the launching and capturing clock transitions at the common node are in opposite directions. If the transition sense at the common node are in opposite directions, the pessimism is the minimum of the difference between late and early arrival times for rise and fall clock transitions.

When set to `same_transition`, the CPPR analysis computes the CPPR value only if the launching clock transition is same as the capturing clock transition at the common node. If the transitions are different, the CPPR value is zero.

When set to `same_transition_expanded`, the CPPR analysis computes the CPPR value only if the launching clock transition is the same as the capturing clock transition at the common node. If the transitions are different, the software searches until it finds a common point where the transitions are the same, and calculates the CPPR value from that point.

To set this global variable, use the set_global command.

timing_create_clock_default_propagated

```
timing_create_clock_default_propagated
{true | false}
```

Default: `false`

Enables clocks to be created in propagated mode.

By default, all clocks, created using `create_clock` and `create_generated_clock`, are in ideal mode.

When the global is set to `true`, all subsequently created regular clocks, derived clocks, and library generated clocks are in propagated mode.

There is no effect on the functionality of `set_propagated_clock` and `set_clock_latency` commands.

To set this global variable, use the set_global command.

timing_create_clock_use_ideal_slew

```
timing_create_clock_use_ideal_slew
{true | false}
```

Default: `false`

Enables the use of ideal slew at the inputs of the clock driver cells.

When set to `true`, the software detects created clocks, and uses zero input slew. Other slew values, including those set using set_annotated_transition command, will be ignored.

When set to `false`, the software honors `set_annotated_transition` command settings.

To set this global variable, use the `set` command.

timing_default_opcond_per_lib

```
timing_default_opcond_per_lib
{true | false}
```

Default: `true`

Determines the default operating condition behavior when there is no explicit user operating condition specified through the `setOpcond` command.

When set to `true`, each timing library uses its own internally-defined default operating condition specification.

When set to `false`, the default operating condition of the master library (that is, the first library in the library search list) is used to set the PVT operation point for the entire design. All timing libraries are accessed and derated to that operating point.

To set this global variable, use the set_global command.

timing_defer_mmmc_object_updates

```
timing_defer_mmmc_object_updates
{true | false}
```

Default: `false`

Allows a sequence of MMMC updates to be accumulated before new data is loaded and analyzed.

When set to `false`, the multi-mode multi-corner (MMMC) update commands, `update_constraint_mode` and `update_analysis_view`, actively load new data upon execution. Since all the timing information is reloaded with each update command, executing a sequence of updates can result in multiple reloads and a subsequent degradation of performance and turn-around-time.

If the `set_analysis_view -update_timing` parameter is specified, the software indicates that the MMMC object updates are complete and new configuration can be loaded and analyzed.

For example:

```
tempus > update_constraint_mode -name ioTiming -sdc_files {postCtsIO.sdc}
tempus > update_constraint_mode -name coreTiming -sdc_files {postCtsCore.sdc}
tempus > set_analysis_view -update_timing
```

For example:

```
update_constraint_mode -name ioTiming -sdc_files {postCtsIO.sdc}
update_constraint_mode -name coreTiming -sdc_files {postCtsCore.sdc}
set_analysis_view -update_timing
```

To set this global variable, use the `set` command.

timing_derate_aocv_dynamic_delays

```
timing_derate_aocv_dynamic_delays
{true | false}
```

Default: `true`

Controls AOCV derating of SI-induced delays.

When set to `false`, AOCV derates will not be applied on dynamic, SI-induced delays, that is, a value of `false` ensures AOCV derating is only applied to the static component of the delay.

When set to `true`, AOCV derating is applied to both static and dynamic arc delay components.

To set this global variable, use the `set_global` command.

timing_derate_aocv_reference_point

```
timing_derate_aocv_reference_point {1 | 0}
```

Default: `1`

Allows specification of reference point for AOCV factors. As per the current AOCV library definitions, the AOCV reference point is always expected to be set at `1.0`.

To set this global variable, use the set_global command.

timing_derate_dynamic_compatibility

```
timing_derate_dynamic_compatibility
{1 | 0}
```

Default: `1`

Controls how the `set_timing_derate` command factors that are not specified, using either `-static` or `-dynamic` options, are applied to the delay.

When this global is set to `1`, the behavior will be compatible with previous releases. In this mode, the static and dynamic components will be summed up before applying derating.

When set to 0, the static and dynamic components will be derated separately and then combined. A setting of 0 is equivalent to using separate derate assertions with the `-dynamic` and `-static` options explicitly specified.

To set the global variable, use the `set` command.

timing_derate_incremental_adjust_additive_mode

```
timing_derate_incremental_adjust_additive_mode
{true | false}
```

Default: `false`

Allows the software to accumulate incremental derate values instead of overwriting them.

When set to `false`, the successive application of incremental derating will overwrite any existing derate value with the new value.

When set to `true`, the successive application of the incremental derates will be accumulated into a final derate value.

To set this global variable, use the `set` command.

timing_derate_incremental_multiply_accumulative_mode

```
timing_derate_incremental_multiply_accumulative_mode
{true | false}
```

Default: `false`

Controls the overwrite or accumulative mode settings when the `set_timing_derate -multiply` parameter is specified. You can use this global variable when the `timing_derate_enable_incremental_multiplication_mode` is set to `true` (that is, `-multiply` support is enabled).

When set to `false`, the latest `set_timing_derate -multiply` command will overwrite the previous constraints at the same level.

When set to `true`, all the values in history will be accumulatively multiplied, when multiple `set_timing_derate -multiply` settings are made on the same object.

To set this global variable, use the `set` command.

Examples

- The following example shows that in overwrite mode, the multiplicative adjustment will be updated to 0.5:

```
set_global timing_derate_enable_incremental_multiplication_mode true
set_global timing_derate_incremental_multiply_accumulative_mode false
set_timing_derate -multiply 1.2
set_timing_derate -multiply 0.5
```

- The following example shows that in accumulative mode, the multiplicative adjustment will be updated to 1.2*0.5=0.6:

```
set_global timing_derate_enable_incremental_multiplication_mode true
set_global timing_derate_incremental_multiply_accumulative_mode true
set_timing_derate -multiply 1.2
set_timing_derate -multiply 0.5
```

timing_derate_negative_delay_backward_compatibility

`timing_derate_negative_delay_backward_compatibility {true | false}`

Default: `true`

Controls the effective derate calculation for negative delays. When effective derate is requested for negative delays, the value will be converted according to the negative delay scaling rule, e.g., 2 - positiveVal.

To set this global variable, use the `set` command.

timing_derate_ocv_reference_point

```
timing_derate_ocv_reference_point {1 | 0}
```

Default: `1`

Allows specification of reference point for OCV factors. As per the current OCV library definitions, the OCV reference point is always expected to be set at `1.0`.

To set this global variable, use the set_global command.

timing_derate_spatial_distance_unit

```
timing_derate_spatial_distance_unit
{default | 1um | 1nm}
```

Default: `default`

Specifies distance units in the SOCV and AOCV library formats.

This global variable supports 3 distance units - `default`, `1um`, and `1nm`. The default value is the default units that are derived from the current system units for distance (um/nm overrides the default).

To set this global variable, use the `set` command.

Note: The `timing_derate_spatial_distance_unit` global variable should be set before a design is loaded.

timing_derate_voltage_scaling_mode

```
timing_derate_voltage_scaling_mode {first_library | snap_to_nearest | interpolate}
```

Default: `first_library`

Controls how timing derates are computed when the analysis voltage is not at the characterization voltage of any of the currently loaded timing libraries. When the timing analysis environment is configured for interpolation, the MMMC library set will contain a list of timing libraries at different voltage characterizations points.

You can specify the following options:

`first_library`	Derates associated with the first library in the timing library list will be used.
`snap_to_nearest`	Allows the software to choose one set of derates from the library closest to the required analysis voltage.
`interpolate`	Allows the timer to perform interpolation of the derates associated with the two bounding reference libraries.

Consider the following example:

```
create_library_set -name ls_interp_1.50V_1.80V
-timing [list stdcell_1.50V.lib stdcell_1.75V.lib stdcell_1.80V.lib]
```

In this example, by default (`first_library`) the derates associated with the library stdcell_1.50V.lib will be used.

When set to `snap_to_nearest`, if the analysis voltage is 1.70V, the derates from the library stdcell_1p75V.lib will be used.

When set to `interpolate`, if the analysis voltage is 1.60V, then interpolation of derates from the libraries stdcell_1.50V.lib and stdcell_1.75V.lib will be used.

If interpolation mode is selected and the analysis voltage is beyond either the smallest or largest voltage, then the library derates will not be interpolated. The derates associated with the nearest bounding library will be used. If the analysis voltage is 1.90V, then only the derates from stdcell_1.80V.lib will be used.

If interpolation mode is enabled, but not all the bounding libraries have derate values, a default derate value of 1.0 will be assumed for the missing derates and used for interpolation. A TA-960 warning message will also be issued.

To set this global variable, use the `set` command.

timing_disable_bidi_output_timing_checks

```
timing_disable_bidi_output_timing_checks
{true | false}
```

Default: `true`

When set to `false`, timing checks on both the input and output sides of the bidirectional pin are analyzed.

When set to `true`, the software disables timing checks on the output side of bidirectional pins.

To set this global variable, use the set_global command.

timing_disable_black_box_endpoints

```
timing_disable_black_box_endpoints {true | false}
```

Default: `false`

Ignores any boundary model endpoints on black box instances.

When set to `false`, the fanout logic from the block's input ports is traced for storing as timed logic within the boundary model. If a pin is reached with no outgoing timing arcs, this is considered an endpoint of the model and the fanin to that endpoint will be included in the model.

When set to `true`, such pins are ignored when they are on cells for which no timing or LEF cell definition was loaded. This is intended to make the model generation more tolerant of missing libraries in cases where the power/ground connectivity is included in the netlist.

To set this global variable, use the `set` command.

timing_disable_bus_contention_check

```
timing_disable_bus_contention_check
{true | false}
```

Default: `false`

When set to `true`, disables propagation of maximum delay along three state disable timing arcs and minimum delay along three state enable arcs. If you know that bus contention conditions do not occur in your design, disable these checks by setting these globals to `true`.

Checks for setup and hold violations in three-state bus designs. Checks the worst delay path to ensure that the latched signals are stable and not unknown values - X. Ensures proper timing checks for bus contention.

To set this global variable, use the set_global command.

timing_disable_clock_gating_checks

```
timing_disable_clock_gating_checks
{true | false}
```

Default: `false`

When set to `true`, disables all clock gating checks, including those declared in the library on integrated clock gating cells, and those that are inferred on combinational gates in the clock path.

To set this global variable, use the set_global command.

Note: The `timing_disable_clock_gating_checks` global variable should not be used with the `setAnalysisMode` command.

If the `setAnalysisMode -clockGatingCheck` parameter is specified, then you need to set this global variable to `true`.

timing_disable_clockperiod_checks

```
timing_disable_clockperiod_checks
{true | false}
```

Default: `false` (performs checks)

When set to `true`, disables timing model clock period checks during timing analysis.

To set this global variable, use the set_global command.

timing_disable_constant_propagation_for_sequential_cells

```
timing_disable_constant_propagation_for_sequential_cells
{true | false}
```

Default: `false`

Controls constant propagation across sequential cells.

When set to `true`, the software disables constant propagation across sequential cells.

When set to `false`, the software enables constant propagation across sequential cells.

To set this global variable, use the `set` command.

timing_disable_drv_reports_on_constant_nets

```
timing_disable_drv_reports_on_constant_nets
{true | false}
```

Default: `false`

Disables DRV checks for nets that are disabled by constant propagation.

When set to `true`, the `report_constraint` command does not report DRV violations for constant nets.

To set this global variable, use the `set` command.

timing_disable_floating_bus_check

```
timing_disable_floating_bus_check
{true | false}
```

Default: `false`

When set to `true`, disables propagation of minimum delay through three state disable timing arcs and maximum delay through three state enable arcs. These checks are only valid during floating bus conditions.

To set this global variable, use the set_global command.

timing_disable_genclk_combinational_blocking

```
timing_disable_genclk_combinational_blocking
{true | false}
```

Default: `false`

Disables blocking of the latency path for a generated clock.

When set to `true`, allows the latency path to traverse through the upstream generated clock.

When set to `false`, if two back-to-back generated clocks are on the same path, and if the downstream generated clock is set using the `create_generated_clock -combinational` parameter, then the latency path for this generated clock is blocked by the upstream clock. The downstream generated clock is created with zero latency.

The software shows a warning if the source latency path to the upstream generated clock cannot be found. For example,

```
**WARN: (TA-1018): A source latency path to the generated clock GC2 through source pin
clk1 to target pin GC2_1/Y in view default_emulate_view cannot be found. Timing
analysis will use 0 ns source latency for the generated clock and will interpret the
master clock based on the polarity at the master clock source pin.
```

To set this global variable, use the `set` command.

timing_disable_inferred_clock_gating_checks

```
timing_disable_inferred_clock_gating_checks
{true | false}
```

Default: `false` (performs checks)

When set to `true`, disables clock gating checks that are inferred on combinational elements in the clock path. Explicit clock gating checks that are described in the timing library are not affected by this global variable.

To set this global variable, use the set_global command.

timing_disable_internal_inout_cell_paths

```
timing_disable_internal_inout_cell_paths
{true | false}
```

Default: `true`

When set to `false`, enables internal bidirectional feedback paths that are completely contained in one instance.

To set this global variable, use the set_global command.

timing_disable_internal_inout_net_arcs

```
timing_disable_internal_inout_net_arcs
{true | false}
```

Default: `true`

When set to `true`, this global disables internal bidirectional feedback paths that span multiple instances.

When set to `false`, this global enables internal bidirectional feedback paths, which are feedback paths that you might not want to use during analysis and optimization.

This global applies only to paths that span multiple instances. The global has no impact on internal feedback paths that are completely contained in one instance.

To set this global variable, use the set_global command.

timing_disable_lib_pulsewidth_checks

```
timing_disable_lib_pulsewidth_checks
{true | false}
```

Default: `false` (performs checks)

When set to `true`, disables timing model pulse width checks during timing analysis.

To set this global variable, use the set_global command.

timing_disable_library_data_to_data_checks

```
timing_disable_library_data_to_data_checks
{true | false}
```

Default: `false`

When set to `true`, this global disables data-to-data checks that are coded in the library as non-sequential timing checks. Data-to-data checks are distinguished from other non-sequential timing checks because both the reference pin and constrained pin side of the timing check are data pins rather than clock pins. You should use the `timing_disable_non_sequential_checks` global to control other types of non-sequential checks where the reference pin is a clock pin.

To set this global variable, use the set_global command.

timing_disable_library_tiehi_tielo

```
timing_disable_library_tiehi_tielo
{true | false}
```

Default: `false`

Disables constant functions in a library.

When set to `false`, the software considers constant propagation from library tie high/low cells prior to those from logical netlist and constraints. There may be cases where programmable tie cells are used whose value can be changed later in the flow, assuming the programmable tie cells are modelled with constant output functions.

When set to `true`, the library constants are not propagated during timing analysis or timing optimization.

To set this global variable, use the set_global command.

timing_disable_netlist_constants

```
timing_disable_netlist_constants
{true | false}
```

Default: `false`

When set to `true`, ignores constants defined in the Verilog netlist. This includes all uses of the in-place constant notation, such as `1'b0`, and the use of power and ground nets. Constants from `set_case_analysis` assertions and from tie-hi and tie-lo cells are still recognized.

When set to `false`, the software recognizes all constants.

To set this global variable, use the set_global command.

timing_disable_nochange_checks

```
timing_disable_nochange_checks
{true | false}
```

Default: `false` (performs checks)

When set to `true`, disables no change timing model checks during timing analysis.

To set this global variable, use the set_global command.

timing_disable_non_sequential_checks

```
timing_disable_non_sequential_checks
{true | false}
```

Default: `false`

When set to `true`, disables the timing arcs between any data-to-clock or clock-to-clock checks which have a `timing_type` attribute equal to one of the following:

- `non_seq_setup_rising`
- `non_seq_setup_falling`
- `non_seq_hold_rising`
- `non_seq_hold_falling`

This global only affects non-sequential checks where the reference pin of the timing check is a clock pin. Data-to-data checks - where the reference pin of the timing check is a data pin - are not affected by this global. To disable data-to-data checks you must use the `timing_disable_library_data_to_data_checks` global.

To set this global variable, use the set_global command.

timing_disable_output_as_clock_port

```
timing_disable_output_as_clock_port
{true | false}
```

Default: *false*

Allows a clock path to be considered as a data path.

When set to `true`, this global causes a clock source path, leading to an output or bidi port, to be treated as a data path, irrespective of any constraint like `set_output_delay`, `set_data_check`, or `set_max_delay` asserted on that port.

To set this global variable, use the set_global command.

timing_disable_parallel_arcs

```
timing_disable_parallel_arcs
{true | false}
```

Default: `true`

Enables the software to automatically perform reduction of parallel delay arcs and check arcs to improve efficiency.

When set to `false`, parallel delay and check arc reduction will be disabled.

To set this global variable, use the `set` command.

timing_disable_pulsewidth_same_edge_si_cppr_mode

`timing_disable_pulsewidth_same_edge_si_cppr_mode`
`{true | false}`

Default: `true`

Enables SI CPPR credit up to the last divergent pin for pulse width checks for zero width pulse clocks. By default, the software credits minimum of rise and fall transition CPPR up to the common pin as the transitions mismatch at the common pin.

When set to `true`, the software computes base delay CPPR up to the common pin as CPPR credit for the path. The software then finds an alternate common timing pin in the clock network where the transition for launch and capture match. If a match is found, the software includes the SI delay up to this pin in the CPPR credit as well.

To set this global variable, use the `set` command.

timing_disable_recovery_removal_checks

```
timing_disable_recovery_removal_checks
{true | false}
```

Default: `false` (performs checks)

When set to `true`, disables recovery and removal timing model checks during timing analysis.

To set this global variable, use the set_global command.

timing_disable_report_header_info

```
timing_disable_report_header_info {true | false}
```

Default: `false`

Controls whether timing reports are generated using a common report header.

When set to `false`, the software generates timing reports with a report header that includes the following information:

- Innovus software version number
- Platform on which the software is being run
- Time when the report was generated
- Command used to generate the report

Any report-specific banner information is included in the same header box, but listed after the common header information.

When set to `true`, the software generates backward-compatible timing reports without the additional header information.

To set this global variable, use the set_global command.

timing_disable_retime_clock_path_slew_propagation

```
timing_disable_retime_clock_path_slew_propagation
{true | false}
```

Default: `true`

When set to `true`, path based analysis is performed on data paths only and the clock segments are not considered.

When set to `false`, analysis is performed on both data and clock paths.

To set this global variable, use the `set` command.

timing_disable_sdf_retain_arc_merging

```
timing_disable_sdf_retain_arc_merging
{true | false}
```

Default: `false`

Controls handling of retain arcs in SDF2.1.

When set to `false`, the Retain information is merged into SDF 2.1, as Retain represents a valid minimum path timing for a cell.

When set to `true`, the Retain arc merging will be disabled.

To set this global variable, use the set_global command.

timing_disable_skew_checks

```
timing_disable_skew_checks
{true | false}
```

Default: `false`

When set to `true`, disables library skew checks for timing analysis.

To set this global variable, use the set_global command.

timing_disable_test_signal_arc

```
timing_disable_test_signal_arc
{true | false}
```

Default: `false`

When set to `true`, timing analysis will not analyze the signal arcs coming from or going to the test pin. Such pins are marked in the timing library with either the `test_scan_in`, the `test_scan_enable` attribute for the input pin, the `test_scan_clock` attribute for the input clock pin, or the `test_output_only` attribute for the output pin.

The following are the equivalent Liberty and TLF pin attributes:

Table 41-1 Liberty and TLF Pin Attributes

Type	Liberty TLF
enable	`test_scan_enable` `SCAN_PINTYPE` (ENABLE)
input	`test_scan_in` `SCAN_PINTYPE` (INPUT)
input clock	`test_scan_clock` `SCAN_PINTYPE` (CLOCK)
output	`test_output_only` `SCAN_PINTYPE` (OUTPUT)

To set this global variable, use the set_global command.

timing_disable_timing_model_latch_inferencing

```
timing_disable_timing_model_latch_inferencing
{true | false}
```

Default: `true`

Controls whether latch behavior is inferred for cell descriptions tagged with the Liberty `timing_model_type` attribute, including all values: `abstracted`, `extracted`, or `qtm`.

When set to `false`, the software infers latch behavior from the timing check and timing delay arc relationships in the model's description.

When set to `true`, latch inferencing does not happen for cells with the `timing_model_type` attribute. Explicit Liberty latch modelling constructs must be coded into the description in order to model latch behavior.

This global does not affect Liberty timing model cells that do not contain the `timing_model_type` attribute. Standard cell core libraries generally do not have this attribute specified; therefore, they typically are not be affected by this global setting.

To set this global variable, use the set_global command.

timing_disable_tristate_disable_arcs

```
timing_disable_tristate_disable_arcs
{true | false}
```

Default: `false`

When set to `true`, disables all 0/1->Z transitions of tristate arcs during timing analysis. By default, all tristate arcs are enabled.

To set this global variable, use the set_global command.

timing_disable_user_data_to_data_checks

```
timing_disable_user_data_to_data_checks
{true | false}
```

Default: `false`

When set to `true`, disables data to data checks that are created by the `set_data_check` command. This global variable does not affect non-sequential timing checks described in the timing library.

To set this global variable, use the set_global command.

timing_driving_cell_override_library

```
timing_driving_cell_override_library
{true | false}
```

Default: `false`

Controls the selection of library cells specified using the `set_driving_cell` command.

When set to `true`, a driving cell will be picked up from the Max Library for max drive cell adjustment and Min Library for min drive cell adjustment, irrespective of the library specified with the `set_driving_cell -library` parameter.

When set to `false`, the libraries are selected as specified with the `set_driving_cell` command.

To set this global variable, use the set_global command.

timing_dynamic_loop_breaking

```
timing_dynamic_loop_breaking
{true | false}
```

Default: `false`

Enables dynamic loop breaking.

By default (`false`), when the timing engine encounters combinational loops in the design, it breaks the loop by disabling a timing arc along the path. The heuristics used in breaking the loop cannot guarantee that a real path through the disabled arc will not be blocked as a consequence. Therefore, users must carefully investigate the arcs that have been disabled to ascertain that the loop break was done safely. If loop breaks are not then added to the master constraint file, they could move to other locations as the design matures.

When set to `true`, the timing system performs a more sophisticated analysis of the design, and ensures that no valid paths are broken inadvertently due to loops.

Note: Setting this global variable to `true` will add additional run time to the analysis.

To set this global variable, use the set_global command.

timing_enable_all_fanin_fanout_levels_compatibilit y

```
timing_enable_all_fanin_fanout_levels_compatibility
{true | false}
```

Default: `true`

Enables computation of gate level count (for `all_fanout`/`all_fanin` commands) when an input pin is specified using the `all_fanout -from` parameter and output pin is specified using the `all_fanin -to` parameter. When the initial gate is traversed, the count is computed as 1.

When set to `false`, the global computes the gate level count as 0.

To set this global variable, use the set_global command.

timing_enable_aocv_slack_based

`timing_enable_aocv_slack_based {0 | 1}`

Default: 0

Enables AOCV analysis using slack-based method.

When set to `true`, the software will try to remove excess pessimism from the graph-based analysis (GBA) AOCV stage counts. The paths to endpoints, which are under the specified slack criteria (default 0.0 ns), are considered when computing stage counts. This allows the stage counts and subsequent AOCV derates to be aligned with the failing paths – and not unnecessarily punished by derates coming from related short paths that meet timing requirements.

Note: A `true` setting is recommended when performing Setup optimizations in Innovus – where over pessimism can cause sub-optimal results.

See `timing_aocv_slack_threshold`.

To use this global variable, use the `set` command.

timing_enable_case_analysis_conflict_warning

```
timing_enable_case_analysis_conflict_warning
{true | false}
```

Default: `true`

Displays a warning when propagated and asserted constant values are not the same when a full timing update is performed.

When set to `true`, a report file, named `CTE_constant_mismatch.rpt`, is generated that contains details of pins and the reason for constant mismatch. This report is over-written each time a full timing update takes place.

To set this global variable, use the set_global command.

timing_enable_clock2clock_clockgating_check

```
timing_enable_clock2clock_clockgating_check
{true | false}
```

Default: `false`

Enables clock-to-clock clock gating checks.

When set to `true`, the software performs clock to clock gating checks at gating elements. For clock to clock gating checks, the frequency of a clock arriving at the reference side of a check should be equal to or greater than the frequency of the clock arriving at the signal side of the check.

To set this global variable, use the `set_global` command.

timing_enable_clock_phase_based_rise_fall_derating

```
timing_enable_clock_phase_based_rise_fall_derating
{true | false}
```

Default: `false`

Determines the interpretation of edge-specific derating factors for clock paths, specified using the `set_timing_derate -rise/-fall` parameters.

When set to `false`, the `set_timing_derate -rise/-fall` parameter refer to the direction of the transition from the perspective of each individual output pin along the clock path.

When set to `true`, the `-rise/-fall` parameter refers to which edge (or phase) of the clock is triggering the transition as viewed from the source of the clock where the clock waveform is defined.

To set this global variable, use the set_global command.

timing_enable_data_through_clock_gating

```
timing_enable_data_through_clock_gating
{true | false}
```

Default: `true`

When set to `false`, blocks signals arriving on the enable of the clock-gating check.

When set to `true`, allows both data and clock signals at the enable pin of a clock-gating check to propagate forward past the check.

To set this global variable, use the set_global command.

timing_enable_derating_for_pulsewidth_checks

```
timing_enable_derating_for_pulsewidth_checks
{true | false}
```

Default: `false`

Controls whether the cell check derates, set by `set_timing_derate -cell_check` command, can be applied for pulse width checks.

When set to `true`, the pulse width checks honor cell check derate factors.

To set this global variable, use the `set_global` command.

timing_enable_early_late_data_slews_for_setuphold_mode_checks

```
timing_enable_early_late_data_slews_for_setuphold_mode_checks
{true | false}
```

Default: `false`

Controls calculated delays.

When set to `true`, the software allows use of both the minimum and maximum data slews for setup and hold checks. By default, the minimum or maximum slew values for clock pins are considered, and for data pins only the maximum slew value is considered while calculating the setup/hold timing check values.

This global variable enables an eight-way timing check for:

- Early/late libraries
- Clock/data pins
- Min/max slews

You can use this global variable in simultaneous setup or hold mode only. To enable this mode, you need to set `timing_enable_simultaneous_setup_hold_mode` to `true`. In this mode, you can use the `reportDelayCalculation` command (`-from` and `-to` parameters) to report all the setup/hold check values corresponding to minimum/maximum combination of clock and data slews.

To set this global variable, use the `set_global` command.

timing_enable_genclk_divide_by_inherit_parent_duty_cycle

```
timing_enable_genclk_divide_by_inherit_parent_duty_cycle
{true | false}
```

Default: `false`

Controls inheritance of duty cycle from master clock for generated clocks (generated using the `create_generated_clock -divide_by` parameter).

To set this global variable, use the `set` command.

timing_enable_genclk_edge_based_source_latency

```
timing_enable_genclk_edge_based_source_latency
{true | false}
```

Default: `true`

Controls how the software chooses generated clock source latency paths. Certain specifications of the `create_generated_clock` constraint imply a cause-and-effect relationship between the edges of the master clock and those of the generated clock.

When set to `true`, the software checks whether a path being analyzed has the correct cause-effect relationship consistent with the specified constraint. Paths that are found to be inconsistent are ignored. Only the paths that are in agreement with the required edge relationship are considered for source latency. If no such path exists, the software uses a source latency of `0` ns. This selection is done independently for each edge of the generated clock; therefore the result might be different for each edge.

The edge-to-edge sufficiency checks are performed assuming a Rise-to-Rise, Rise-to-Fall relationship between the master clock and the generated clock with an even number `-divide_by` factor. For the odd `-divide_by` factors, the sufficiency check is performed assuming a Rise-to-Rise, Fall-to-Fall relationship. The sufficiency checking is not performed for `-multiply_by` options.

When set to `false`, the software does not check paths for the correct cause-effect relationship.

To set this global variable, use the set_global command.

timing_enable_genclk_source_path_register_limit

```
timing_enable_genclk_source_path_register_limit
{true | false}
```

Default: `false`

Limits the generated clock source latency path to traverse across one register only.

When set to `false`, the software allows master clock latency path to traverse across any number of sequential devices to reach the generated clock target pin.

When set to `true`, the software restricts the master clock latency path for a generated clock to traverse exactly one sequential device.

To set this global variable, use the `set` command.

timing_enable_get_object_escaped_name_backward_compatible

```
timing_enable_get_object_escaped_name_backward_compatible
{true | false}
```

Default: `false`

Enables printing of escape characters in the `get_object_name` command output.

When set to `false`, the `get_object_name` command does not print escape characters (starting with a backslash) in the object name.

To set this global variable, use the set_global command.

timing_enable_get_objects_regexp_compatibility

```
timing_enable_get_objects_regexp_compatibility {true | false}
```

Default: `false`

Defines the pattern search scope to boundaries deduced from hierarchy separation characters specified in the search pattern.

When set to `false`, the commands with `-regexp` parameter will search at the top level only (unless used with the `-hier` parameter).

When set to `true`, the commands will search the specified pattern within the hierarchies based upon the given pattern hierarchy (similar to pattern search without the `-regexp` parameter). The `-hier` along with the `-regexp` parameter shows the same behavior similar to when the command is used without the `-regexp` option - only the leaf level object name is compared to the pattern match.

To set this global variable, use the `set` command.

Examples

Command behavior when this global variable is `false`:

```
get_pins -regexp seg2/.*/QN
**WARN: (TCLCMD-513): The software could not find a matching object of the specified
type for the pattern '.*/.*'
**ERROR: (TCLCMD-917): Cannot find 'pins' that match 'seg2/.*.*/QN'

get_pins -regexp seg2/.*/QN -hier
seg2/u3/QN seg2/u9/QN seg2/u14/QN

get_pins -regexp .*/QN -hier
seg1/u3/QN seg1/u9/QN seg1/u14/QN seg2/u3/QN seg2/u9/QN seg2/u14/QN seg3/u3/QN
seg3/u9/QN seg3/u14/QN u2/QN u5/QN u8/QN
```

Command behavior when this global variable is `true`:

```
get_pins -regexp seg2/.*/QN
seg2/u3/QN seg2/u9/QN seg2/u14/QN

get_pins -regexp seg2/.*/QN -hier
**WARN: (TCLCMD-513): The software could not find a matching object of the specified
type for the pattern '.*/.*'
**ERROR: (TCLCMD-917): Cannot find 'pins' that match 'seg2/.*.*.*.*.*/QN'

get_pins -regexp .*/QN -hier
seg1/u3/QN seg1/u9/QN seg1/u14/QN seg2/u3/QN seg2/u9/QN seg2/u14/QN seg3/u3/QN
seg3/u9/QN seg3/u14/QN u2/QN u5/QN u8/QN
```

timing_enable_hierarchical_get_nets_support

```
timing_enable_hierarchical_get_nets_support {true | false}
```

Default: `false`

Enables hierarchical segment support for the `get_nets` command.

When set to `false`, the `get_nets` command will report only flat nets.

To set this global variable, use the `set` command.

timing_enable_latch_thru_mode

```
timing_enable_latch_thru_mode {true | false}
```

Default: `false`

Enables the latch through analysis mode in which timing paths can propagate across latches, depending on their arrival time with respect to the latch transparency window.

When set to `true`, the arrival and required times will be propagated across a latch if it is in transparent mode. The reporting will show the entire flow through the path, rather than individual segments. This allows the overall path slack to be seen through upstream borrowing latch stages. The timing analysis, timing optimization, and reporting will consider the sequence of borrowing latch stages as a single path. As a result, the timing optimization reaches the target slack by viewing the entire path.

This feature provides better visibility into the overall context of latch path timing issues - for both the stage and path-level slacks. Additionally, this improves usability for debugging and manual ECOs, and shows better quality of result for Tempus ECO timing closure.

To set this global variable, use the `set` command.

timing_enable_latency_through_clock_gating

```
timing_enable_latency_through_clock_gating
{true | false}
```

Default: `true`

Allows propagation of latency phases across clock gating elements.

When set to `true`, the software allows propagation of clock latency phase across clock gating elements.

When set to `false`, the software suppresses propagation of clock latency across clock gating elements from the enable pin. The enable pin of the clock gating element should not be in the clock network for any other clocks. This is not recommended because the enable path (through the 'inferred' clock gates, such as, AND, NAND, OR) is either for clock latency or data paths. If the gated clock check is met, then the enable should not pass through to the output.

In case of ICGs, there is no arc from the EN to the CLKOUT on ICGs that have an incoming enable signal latched. In case of ICG/AND, using the enable path may not be correct for determining latency paths. If no latency paths are found, then the generated clocks definition will need to be re-checked.

To set this global variable, use the `set` command.

timing_enable_mmmc_loop_handling

```
timing_enable_mmmc_loop_handling
{true | false}
```

Default: `true`

When set to `true`, loop breaking is handled independently per analysis view, according to the following rules in order of precedence:

- If an analysis view does not contain a combinational loop, it does not inherit any loop breaking done for other views.
- If a loop exists, and the user has supplied specific case analysis and `set_disable_timing` constraints to break the loop at a certain place, the specified loop break is honored.
- If a loop is still detected, the software automatically selects a point to break the loop, and applies the loop break consistently to any views where the loop exists.

When set to `false`, if a combinational loop is detected in any analysis view, the timing system breaks the loop by disabling an arc along the looping path. These combinational loop breaks are reflected in all analysis views, even if those views by themselves do not contain these looping paths.

> Cadence strongly recommends that all combinational loops detected by the software be evaluated, to ensure that the loop is broken at the most ideal location. Automatic loop breaking decisions are made based on how the design is traversed. Small changes in the design can influence where loop breaking decisions are made.

To set this global variable, use the set_global command.

timing_enable_multi_drive_net_reduction_with_assertions

```
timing_enable_multi_drive_net_reduction_with_assertions
{none | delay | all}
```

Default: `none`

Allows reduction of multi-drive networks when constraint assertions are present on the network elements:

The following options are available:

`none`: Reduction is not performed on portions of the network where constraints have been asserted.

`delay`: Reduction of the multi-drive network will still be applied if only`set_annotated_delay` and `set_annotated_transition` constraints have been applied. Other constraints present on the multi-drive network can still prevent full reduction.

`all`: The software ignores the presence of any assertions on the multi-drive network and forces reduction to occur.

To set this global variable, use the set_global command.

timing_enable_multifrequency_latch_analysis

```
timing_enable_multifrequency_latch_analysis
{true | false}
```

Default: `false`

Enables multi-frequency latch timing analysis of the latch time borrowing for when a data signal coming to a latch is controlled by a clock with a frequency different to a clock of the latch enabling signal.

When set to `true`, the edges of all sending and receiving clocks are analyzed at every latch, which results in less pessimistic, and more detailed timing analysis.

When set to `false`, latch time borrowing analysis uses the worst case relationship between sending and receiving clocks at the latch, which can produce paths with pessimistic slack.

Note: Enabling advanced latch analysis can result in some performance penalty.

To set this global variable, use the set_global command.

timing_enable_path_delay_to_unconstrained_endpoints_compatibility

```
timing_enable_path_delay_to_unconstrained_endpoints_compatibility
{true | false}
```

Default: `false`

When set to `true`, the software does not constrain paths with path delay - if the path ends at an unconnected input pin of a combination cell.

To set this global variable, use the `set` command.

timing_enable_pessimistic_cppr_for_reconvergent_clock_paths

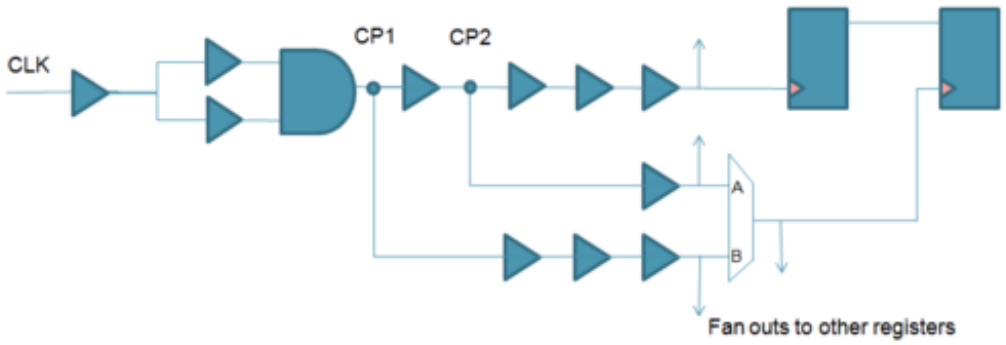
```
timing_enable_pessimistic_cppr_for_reconvergent_clock_paths
{true | false}
```

Default: `false`

When set to `true`, the software will consider all combinations of launch and capture clock path pairs to identify all possible common pins, and then selects the pin with minimum CPPR adjustment over all such common pins as the common pin.

When set to `false`, the default, the software will choose the CPPR common pin based on the commonality of the worst launch and worst capture path only.

Consider the following example,



In the above diagram, both capture paths are considered along with the single launch path, and the common pin is chosen as the pin with the least CPPR adjustment. The pin CP1 is chosen as a common pin and CPPR is reported up to this pin. The same common pin, CP1, will be used for CPPR adjustment whether the capture clock path goes through the mux A pin or the mux B pin. When this global variable is set to `false`, CPPR common pin is chosen by the commonality in the worst-case launch and worst-case capture paths. so CP2 is chosen as the common pin and CPPR is reported up to this pin.

Note:
The `timing_cppr_skip_clock_reconvergence` and `timing_enable_pessimistic_cppr_for_reconvergent_clock_paths` global variables are mutually exclusive.
Also, the `timing_cppr_skip_clock_reconvergence` global variable has precedence over `timing_enable_pessimistic_cppr_for_reconvergent_clock_paths`.

To set this global variable, use the set_global command.

timing_enable_power_ground_constants

```
timing_enable_power_ground_constants
{true | false}
```

Default: `false`

Determines if case analysis is inferred from power and ground rail connections/nets, e.g., VDD and GND.

When set to `true`, the software recognizes the constants that come from power and ground rail connections.

When set to `false`, the software ignores these constants. This provides the most intuitive behavior with logical style netlist (and other tools that do not read physical data).

To set this global variable, use the `set_global` command.

timing_enable_preset_clear_arcs

```
timing_enable_preset_clear_arcs
{true | false}
```

Default: `false`

Determines whether the timing arcs are created to model the transition to active state (assertion) of the preset or clear pin, and the subsequent transition of the output to a controlled state.

When set to `true`, the software builds the arcs and the timer automatically analyzes paths through them (enable clock propagation through preset clear pins).

When set to `false`, the software does not build these arcs, because they are associated with analyzing the reset mode timing paths. Filtering them prevents unwanted interaction with functional mode timing analysis.

You can use the `lib_build_asynch_de_assert_arc` global variable to control whether input to output arcs from the preset or clear pins transitioning to inactive state are included when the timing system is initialized. If both the `timing_enable_preset_clear_arcs` and the `lib_build_asynch_de_assert_arc` global variables are set to `true`, all asynchronous arcs are recognized. However, if `timing_enable_preset_clear_arcs` is set to `true` and `lib_build_asynch_de_assert_arc` is set to `false`, only assert asynchronous arcs are recognized.

Note: The `timing_enable_preset_clear_arcs` global variable can also impact delay calculation results of a design. When an arc is disabled, the software will calculate delay using the linear table lookup. When an arc is not disabled, the delay calculations are made based on the quadratic table lookup.

Note: This global variable should be set to `true` when reset checks are performed on a dedicated reset SDC.

To set this global variable, use the `set_global` command.

timing_enable_pulsed_latch

```
timing_enable_pulsed_latch
{true | false}
```

Default: `true`

Enables pulse-latch analysis.

When set to `true`, the pulse-latch analysis mode provides several additional pulse-clock specific constraint specifications and reporting capabilities.

When in pulse-latch mode, you can use the following pulse clock constraints:

- `set_pulse_clock_max_width`
- `set_pulse_clock_min_width`
- `set_pulse_clock_max_transition`
- `set_pulse_clock_min_transition`

Additionally, in this mode, the `report_constraint` command supports the following pulse clock check types:

- `pulse_clock_max_width`
- `pulse_clock_min_width`
- `pulse_clock_max_transition`
- `pulse_clock_min_transition`

To set this global variable, use the set_global command.

timing_enable_sdc_compatible_data_check_mcp

```
timing_enable_sdc_compatible_data_check_mcp
{true | false}
```

Default: `false`

Enables checking for SDC compatible data checks when `set_multicycle_path -start` parameter is specified.

When set to `true`, the `-start` parameter shifts (or reverses) the reference clock back by one clock cycle.

When set to `false`, the `set_multicycle_path -start` and `-end` parameters use the same number of cycles, even though the actual adjustment is computed using the signal clock for `-start` and reference clock for `-end` parameters. Additionally, the software computes adjustments for `set_data_check -hold` checks between multiple frequency clocks with the `set_multicycle_path -start -setup` setting.

To set this global variable, use the `set_global` command.

timing_enable_si_cppr

```
timing_enable_si_cppr
{true | false}
```

Default: `true`

Enables more accurate CPPR analysis when incremental delays are present and the `timing_remove_clock_reconvergence_pessimism` global variable is set to `true`.

When set to `true`, the incremental delay is not included in the CPPR calculation during setup analysis, but is included in the CPPR calculation during hold analysis.

When set to `false`, the incremental delay is included in both setup and hold CPPR calculations.

To set this global variable, use the set_global command.

timing_enable_simultaneous_setup_hold_mode

```
timing_enable_simultaneous_setup_hold_mode
{true | false}
```

Default: `false`

Controls whether setup and hold checks are analyzed separately, or together on the same timing graph.

When set to `false`, the setAnalysisMode `-checkType setup` and `hold` parameters are used to switch the system from analyzing setup slacks to analyzing hold slacks.

When set to `true`, setup and hold slacks are analyzed concurrently. As a consequence, both `-early` and `-late` queries for reporting commands such as `report_timing` can be satisfied immediately, without additional calculation. Simultaneous mode is useful for immediately understanding the consequences to both setup and hold slacks from what-if operations and manual ECOs to the design.

Note: Simultaneous setup and hold mode can be used only for timing analysis; it cannot be used for any physical implementation steps.

> Simultaneous setup/hold mode has the following behavior differences, with respect to the default mode:

- `setAnalysisMode -checkType setup` and `hold` do not function in simultaneous mode.
- The `write_timing_windows` command does not generate timing window information using the min and max libraries. Instead, by default, it uses only the max libraries.

To set this global variable, use the set_global command.

timing_enable_spatial_derate_mode

```
timing_enable_spatial_derate_mode
{true | false}
```

Default: `false`

Enables the spatial derate mode.

When set to `true`, the software allows access to the GBA spatial modes (`bounding_box` or `chip_size`) specified with the `timing_spatial_derate_distance_mode` global variable.

When set to `false`, the software allows you to specify only the `bounding_box` option.

To set this global variable, use the `set` command.

timing_enable_tristate_clock_gating

```
timing_enable_tristate_clock_gating
{true | false}
```

Default: `false`

The timing system infers gated-clock checks in the design where a data and clock path intersect at a combinational device. When this global is set to `true`, inferred gated-clock checks are added when clock and data signals converge through the tristate enable and data input of tristate buffers. By default (a setting of `false`), gated-clock checks are not added for these situations.

To set this global variable, use the set_global command.

timing_enable_uncertainty_for_clock_checks

```
timing_enable_uncertainty_for_clock_checks
{true | false}
```

Default: `true`

Considers clock uncertainty when performing clock checks.

When set to `true`, the `set_clock_uncertainty` command supports clock uncertainty for period, pulse width, skew, and separation checks. This global setting affects all the clock_style checks including pulse_width.

Note: The `timing_enable_uncertainty_for_clock_checks` global variable overrides `timing_enable_uncertainty_for_pulsewidth_checks` setting.

To set this global variable, use the `set` command.

timing_enable_uncertainty_for_pulsewidth_checks

```
timing_enable_uncertainty_for_pulsewidth_checks
{true | false}
```

Default: `false`

When set to `true`, considers clock uncertainty when performing minimum pulse width checks. Clock uncertainty is considered during checks of both user-defined minimum pulse widths (using the `set_min_pulse_width`), and minimum pulse widths that are defined in the timing model (`.lib`).

Note: Specifying a value of `true` affects the following commands that take minimum pulse width checks into account:

- `report_timing -check_type pulse_width`
- `report_timing -check_clocks`
- `report_min_pulse_width`
- `report_analysis_coverage`

To set this global variable, use the set_global command.

timing_extract_model_aocv_mode

```
timing_extract_model_aocv_mode
     {graph_based | path_based | none}
```

Default: `none`

Allows you to specify AOCV derating mode.

The valid values are:

- `graph_based`: Specifies that the delays in the timing model are derated using the graph-based stage counts.
- `path_based`: Specifies that the delays in the timing model are derated using the total path stage count of worst path between those pin pairs.
- `none`: Specifies that the delays in the timing model are non-AOCV derated base delays.

Note: Before setting this global to `graph_based or path_based` mode, it is mandatory that the AOCV libraries are read in and AOCV analysis is enabled (by setting `set_analysis_mode -aocv true`).

timing_extract_model_case_analysis_in_library

```
timing_extract_model_case_analysis_in_library
{true | false}
```

Default: `true`

Enables the software to model the constants as a function in the extracted timing model.

When set to `true`, the software models constants as functions in the library if these propagate to the output port.

When set to `false`, instead of modeling constants as functions inside the library, these are written as `set_case_analysis` in the constraints file (*.asrt).

To set this global variable, use the `set` command.

timing_extract_model_check_arcs_as_lvf

```
timing_extract_model_check_arcs_as_lvf {true | false}
```

Default: `true`

Enables check arcs to be modeled as LVF arcs. By default, the check arcs will be written as nSigma numbers.

Innovus allows you to write mean+/-nSigma values for constraint arcs, while writing separate mean and sigma values for delay and transition data. This can be enabled by setting `timing_extract_model_check_arcs_as_lvf` global variable to `false` and `timing_extract_model_write_lvf` to `true`.

This global variable will not have any impact if `timing_extract_model_write_lvf` is set to `false`.

To set this global variable, use the `set` command.

timing_extract_model_consider_design_level_drv

`timing_extract_model_consider_design_level_drv`
`{true | false}`
Default : `true`

Considers `set_max_capacitance` and `set_max_transition` DRV constraints, that have been applied on the current design during model extraction.

When set to `true`, these constraints applied at the design level will be considered during model extraction. However, the maximum capacitance and maximum transition from various DRV constraints applied/inferred on the ports is considered.

When set to `false`, the software extracts the worse-case DRV values and the constraints applied on the current design will be ignored.

To set this global variable, use the set_global command.

timing_extract_model_disable_cycle_adjustment

```
timing_extract_model_disable_cycle_adjustment
{true | false}
```

Default: `false`

Controls cycle adjustment for multi-cycle paths.

When set to `false`, if the multi-cycle paths specified in the SDC at the block-level cannot be pushed out to the ports (since there are other paths going through these paths), then the Tempus software adjusts the delay/check numbers in the ETM to take into account the cycle adjustment.

When set to `true`, the cycle adjustment for the multi-cycle paths in ETM arc delay are disabled.

To set this global variable, use the `set` command.

timing_extract_model_exhaustive_validation_dir

`timing_extract_model_exhaustive_validation_dir` *`string`*

Default: `" "`

Specifies the output directory where the additional validation reports generated during ETM Extremity Validation (but not generated during normal validation) will be written.

To set this global variable, use the `set` command.

timing_extract_model_exhaustive_validation_mode

```
timing_extract_model_exhaustive_validation_mode
{true | false}
```

Default: `false`

Enables extracted timing model (ETM) validation at the minimum and maximum indices of the slew load indices.

When set to `false`, the ETM validation is done only for the input slews/output-loads that have been asserted on the design during timing model extraction.

When set to `true`, the software uses the “Extremity-Validation” (EV-ETM) methodology that allows ETM to model the timing arcs in the design for a range of discrete input slews/output-loads, thus providing a more comprehensive validation.

To set this global variable, use the `set` command.

timing_extract_model_gating_as_nochange_arc

```
timing_extract_model_gating_as_nochange_arc
{true | false}
```

Default: `true`

Disables conversion of clock gating checks to nochange arcs.

When set to true, the software disables conversion of clock gating checks to nochange arcs in the extracted timing model.

To set this global variable, use the set command.

timing_extract_model_ideal_clock_latency_arc

```
timing_extract_model_ideal_clock_latency_arc
{true | false}
```

Default: `true`

When set to `true`, the latency arcs for the generated clocks that are defined ideal are extracted with reference as master clock during ETM extraction.

When set to `false`, the arcs are not extracted for generated clocks that are defined ideal during ETM extractions.

To set this global variable, use the `set` command.

timing_extract_model_include_applied_load_in_characterization_range

```
timing_extract_model_include_applied_load_in_characterization_range
{true | false}
```

Default: `true`

Specifies that the actual load visible to the timer that will be included in the load characterization range for path end points.

To set this global variable, use the `set` command.

timing_extract_model_include_applied_slew_in_characterization_range

```
timing_extract_model_include_applied_slew_in_characterization_range
{true | false}
```

Default: `true`

Specifies that the actual timer slew must be included in the characterization range timing model.

To set this global variable, use the `set` command.

timing_extract_model_max_feedthrough_characterization_load

```
timing_extract_model_max_feedthrough_characterization_load
{true | false}
```

Default: `false`

Specifies that the load is used by the model extractor for feed-through path characterization.

To set this global variable, use the `set` command.

timing_extract_model_non_borrowing_latch_path_as_setup

```
timing_extract_model_non_borrowing_latch_path_as_setup
{true | false}
```

Default: `false`

Models the non-borrowing interface latch paths as setup checks.

To set this global variable, use the `set` command.

timing_extract_model_slew_propagation_mode

```
timing_extract_model_slew_propagation_mode
{worst_slew | path_based_slew}
```

Default: `worst_slew`

Specifies the type of slew propagation to use for generating extracted timing model (ETM).

In worst slew propagation mode, the software propagates the worst slew of all the incoming arcs at a converging point for extracting the arcs.

In path based slew propagation mode, the software propagates the actual slew for the path elements for extracting the arcs.

To set this global variable, use the set_global command.

timing_extract_model_write_clock_checks_as_arc

```
timing_extract_model_write_clock_checks_as_arc
{true | false}
```

Default : `false`

When set to `true`, the clock checks (min_period and min_pulse_width) are modelled as arcs in the extracted timing models (ETM). Clock check arcs will be modelled as a single dimensional table with constrained and reference pin as clock pin itself.

When set to `false`, clock checks are modelled as scalar numbers in the extracted ETM.

To set this global variable, use `set` command.

timing_extract_model_write_clock_checks_as_scalar_tables

```
timing_extract_model_write_clock_checks_as_scalar_tables
{true | false}
```

Default: `false`

Allows the extracted timing model to write the minimum pulse width or minimum period checks as scalar arcs.

To set this global variable, use the `set` command.

timing_extract_model_write_lvf

```
timing_extract_model_write_lvf {true | false}
```

Default: `false`

Generates extracted timing model (ETM) containing Liberty Variation Format (LVF) model.

When set to `true`, this global variable enables writing separate mean and variation tables for all the delay/constraint and transition tables.

When set to `false`, the variation data will be merged with the mean values. The mean +/- nsigma values of delay/constraint/transition data will be written in the ETM.

To set this global variable, use the `set` command.

timing_extract_model_write_min_max_clock_tree_path

```
timing_extract_model_write_min_max_clock_tree_path
{true | false}
```

Default: `true`

Writes the minimum/maximum worst latency for each clock source.

To set this global variable, use the `set` command.

timing_generate_normalized_driver_waveform

```
timing_generate_normalized_driver_waveform
{0 | 1}
```

Default: 1

Generates the analytical normalized driver waveform for timing library if the waveform is not specified in the library. The waveform is generated by weighed mean of linear and exponential waveform.

To set this global variable, use the `set` command.

timing_generated_clocks_allow_nested_assertions

```
timing_generated_clocks_allow_nested_assertions {true | false}
```

Default: `false`

Controls clock latency handling for nested generated clocks that are created on the same timing pin as the master clock.

When set to `true`, the software handles generation of multiple nested generated clocks that are asserted on the same pin, such that the child clock inherits the clock latency value of its parent clock.

When set to `false`, the nested generated clocks asserted on the same pin do not inherit the latency values and are created with 0ns master clock latency. Any full clock timing path reported using these clocks will show the clock path up to the target pin of the generated clock.

To set this global variable, use the `set` command.

timing_generated_clocks_inherit_ideal_latency

```
timing_generated_clocks_inherit_ideal_latency
{true | false}
```

Default: `false`

When set to `true`, the software makes the generated clocks to inherit the ideal network latency of the parent clock.

To set this global variable, use the `set` command.

timing_get_of_objects_hier_compatibility

```
timing_get_of_objects_hier_compatibility
{true | false}
```

Default: `false`

When set to `true`, the `get_pins -of_objects` parameter will return leaf pins connected to the specified objects.

When set to `false`, the `get_pins -of_objects` parameter will return hierarchical pins and leaf pins connected to the specified objects. In order to return leaf pins only when the global is set to `false`, the `-leaf` option must be specified along with the `get_pins -of_objects` parameter.

When set to `true`, the `get_cells -objects` parameter returns cells connected to the net at the same level of hierarchy as the net itself. Hierarchical ports will be returned if the net is connected to a hierarchical module.

To set this global variable, use the set_global command.

timing_hier_object_name_compatibility

```
timing_hier_object_name_compatibility
{true | false}
```

Default: `true`

Controls how hierarchical delimiters are interpreted in the search pattern, when the `-hier` parameter is used with the `get_*` collection commands.

When set to `true`, the software interprets the hierarchical delimiter as part of the search pattern itself, and tries to match names that include the delimiter. For example, if you specify:

The software searches at every level of the hierarchy for instances with the name `L1/R*`, completely ignoring the hierarchy separator:

When set to `false`, hierarchical delimiters are interpreted as hierarchy separators. For example, if you specify:

The software searched for instances with names matching `R*` in the hierarchy level below `L1`:

To set this global variable, use the set_global command.

timing_hierarchical_context_continue_on_top_block_clock_mismatch

```
timing_hierarchical_context_continue_on_top_block_clock_mismatch
{true | false}
```

Default: `false`

Allows loading of the timing context model with unmapped clocks.

When set to `true`, the software allows the script to process even if there are any mismatches between the top and block-level clocks. The mismatches are reported in the *timing_context_clock_mapping_data.rpt* file, which shows data that allows you to determine if corrections or additions are required to the clock mapping file.

When set to `false`, the software will show an error condition if any of the top-level clocks cannot be matched with a block-level clock.

To set this global variable, use the `set` command.

timing_ignore_clock_based_path_delay_on_data_checks

```
timing_ignore_clock_based_path_delay_on_data_checks
{true | false}
```

Default: `false`

Disables clock-based path delay application on data checks.

When set to `true`, the Tempus software enables the data checks to be reported by disabling the applied path delay. The "Data Check Setup" details are displayed in the timing report output.

When set to `false`, the timing reports display the "Path Delay" details. By default, this global variable is set to `false`.

To set this global variable, use the `set` command.

timing_ignore_lumped_rc_assertions

```
timing_ignore_lumped_rc_assertions
{true | false}
```

Default: `false`

When set to `false`, the `set_load` and `set_resistance` values override the actual extracted representation. A warning message is issued stating that the specified constraints on internal nets will override extracted parasitics on those nets.

When set to `true`, instructs the software to ignore `set_load` and `set_resistance` assertions on internal nets in the design. Parasitic data for internal nets normally should be provided by Innovus RC extraction, or external SPEF annotation.

Using `set_load` to specify I/O boundary loading on ports is supported by either setting of this global.

To set this global variable, use the set_global command.

timing_io_use_clock_network_latency

```
timing_io_use_clock_network_latency
[always | ideal]
```

Default: `ideal`

Controls whether network latency of a reference clock is added or not to the data arrival time on the port.

When set to `ideal`, the `set_input_delay` and the `set_output_delay` constraints will not add the network latency of the reference clock to the data arrival time on the port if the clock is in `propagated` mode.

When set to `always`, network latency is added to the data arrival time on the port regardless of the clock propagation mode.

To set this global variable, use the `set` command.

timing_library_convert_async_setuphold_to_recrem

```
timing_library_convert_async_setuphold_to_recrem
[1 | 0]
```

Default: `1`

Controls whether single-edged setup and hold checks on asynchronous pins in the Liberty library are inferred as recovery and removal checks.

This global variable provides support for older modeling styles in legacy libraries.

To set this global variable, use the `set` command.

timing_library_genclk_use_group_name

```
timing_library_genclk_use_group_name
{true | false}
```

Default: `false`

When set to `true`, the software uses the generated_clock group name when creating a generated clock from a library-generated clock group. When set to `false`, the software uses the pin name when creating the generated clock.

For example, assume the cell for instance `a/b` in the timing library contained the following generated clock group:

```
generated_clock(genclk2) {
clock_pin : clk1;
master_pin : "int/clk";
multiplied_by : 2;
duty_cycle : 50.0;
}
```

By default (`false`), the software creates a generated clock with the following name:

```
a/b/clk1
```

If the timing_prefix_module_name_with_library_genclk global variable is set to `true`, and you set this global variable to `true`, the software creates a generated clock with the following name:

```
a/b/genclk2
```

If the `timing_prefix_module_name_with_library_genclk` global variable is set to `false`, and you set this global variable to `true`, the software creates a generated clock with the following name:

```
genclk2
```

To set this global variable, use the set_global command.

timing_library_hold_constraint_corner_sigma_multiplier

`timing_library_hold_constraint_corner_sigma_multiplier` *double*

Default: 0

Applies user control multiplier to generate a constraint table using sigma values applied for hold arc.

To set this global variable, use the `set` command.

timing_library_hold_sigma_multiplier

`timing_library_hold_sigma_multiplier` *double*

Default: 0

Applies user control multiplier to generate a constraint table applied for hold arc.

To set this global variable, use the `set` command.

timing_library_infer_async_pins_from_timing_arcs

```
timing_library_infer_async_pins_from_timing_arcs
{true | false}
```

Default: *false*

When set to `true`, a pin is marked clear if it has timing arcs with type “clear” to output pins. The sequential block is not required to be present in the cell for marking a pin as clear.

To set this global variable, use the set_global command.

timing_library_infer_cap_range_from_ccs_receiver_model

```
timing_library_infer_cap_range_from_ccs_receiver_model
{true | false}
```

Default: `false`

Infers the rise or fall of capacitance range values from CCS receiver capacitance model.

When set to `false`, the rise/fall CCS capacitance range values are not inferred. The NLDM pin capacitance range values defined in the library are honored.

To set this global variable, use the `set_global` command.

timing_library_infer_cap_range_from_ecsm_receiver_model

```
timing_library_infer_cap_range_from_ecsm_receiver_model {true | false}
```

Default: false

Infers the rise or fall of NLDM pin capacitance range from the ECSM capacitance range while calculating the total loading capacitance during switching power computation. If the global is set to `true` and the capacitance range is not defined in the library, the rise/fall capacitance range is inferred from the ECSM receiver capacitance models. The capacitance range values are derived either from the pin-level ECSM cap models or the timing-level ECSM cap models.

When this global is set to `false`, the rise/fall capacitance range values are not inferred, and the library values are honored.

To set this global variable, use the `set_global` command.

timing_library_infer_socv_from_aocv

```
timing_library_infer_socv_from_aocv {true | false}
```

Default: `false`

Controls the transformation of AOCV data.

This global variable infers sensitivity data from AOCV libraries, and must be set before loading any timing library data.

To set this global variable, use the `set` command.

timing_library_read_ccs_noise_data

```
timing_library_read_ccs_noise_data
{true | false}
```

Default: `true`

Allows CCS noise constructs to be read from a library.

This global variable can be used when the CCS library contains noise models, and SI glitch analysis is being performed.

To set this global variable, use the `set_global` command.

timing_library_read_without_power

```
timing_library_read_without_power
{true | false}
```

Default: `false`

When set to `true`, this global variable allows the software to read timing libraries without power information.

When set to `false`, the software reads power data also.

To set this global variable, use the `set` command.

timing_library_scale_aocv_to_socv_to_n_sigma

```
timing_library_scale_aocv_to_socv_to_n_sigma
 <n_sigma value>
```

Default: 3

Derives AOCV derates based on 3-sigma variation.

To set this global variable, use the `set` command.

timing_library_setup_constraint_corner_sigma_multiplier

`timing_library_setup_constraint_corner_sigma_multiplier` *`double`*

Default: 0

Applies user control multiplier to generate a constraint table using sigma values applied for setup arc.

To set this global variable, use the `set` command.

timing_library_setup_sigma_multiplier

`timing_library_setup_sigma_multiplier` *double*

Default: 0

Applies a user control multiplier to generate a constraint table using sigma values applied for setup arcs.

To set this global variable, use the `set` command.

timing_library_use_trilib_drv_values

```
timing_library_use_trilib_drv_values
{true | false}
```

Default: `false`

Allows the software to use a range of trilib DRV values for performing delay calculations.

When set to `false`, the DRV values are picked up from the first library of a trilib set and are used for DRV reporting. In some cases, the DRV constraints may not be the same across multi-voltage libraries. For this you can use a range of trilib DRV values, by setting the `timing_library_use_trilib_drv_values` global variable to `true`.

When set to `true`, the trilib (or voltage scaling) mechanism allows more accurate delay calculation in case timing libraries are not characterized at operating voltage. Through this mechanism, the bounding libraries of the operating voltage are obtained from the trilib set and the worst value is used in delay calculations.

This global variable is supported for `report_constraint` based DRVs. The global will impact the `report_constraint` command when there is a voltage scaling setup and the command is issued with the `-drv_violation_type` parameter.

To set this global variable, use the `set` command.

timing_library_use_two_piece_receiver_cap

```
timing_library_use_two_piece_receiver_cap {true | false}
```

Default: `false`

Enables the use of 2-piece receiver capacitance constructs, if a library contains both the N-piece and 2-piece receiver capacitance construct.

When set to `true`, if a library has 2-piece and multi-piece pin capacitance, then the software gives priority to 2-piece models.

When set to `false`, the software reads the N-piece receiver capacitance constructs.

To set this global variable, use the `set` command.

timing_library_zero_negative_timing_check_arcs

```
timing_library_zero_negative_timing_check_arcs
{true | false}
```

Default: `false`

When set to `true`, converts any negative timing check values in the timing library to zero.

timing_max_transition_use_si_transition

```
timing_max_transition_use_si_transition {true | false}
```

Default: `false`

Enables reporting of noisy slew in DRV flows.

To set this global variable, use the `set` command.

timing_multifrequency_clock_rounding_factor

```
timing_multifrequency_clock_rounding_factor
{1e-04 | 1e-05}
```

Default: 1e-05

Rounds off irrational numbers so that they can be read appropriately. The "round off factor" helps improve the level of accuracy of floating point numbers.

To round off clock period numbers accurate to the fourth decimal place, you can use the following command:

set_global timing_multifrequency_clock_rounding_factor 1.0e-4

To set this global variable, use the set_global command.

timing_normalized_driver_waveform_clip_linear_part

```
timing_normalized_driver_waveform_clip_linear_part
{0 | 1}
```

Default: `0`

When set to 1, the linear ramp component in the normalized driver waveform is saturated to the maximum value, while generating the normalized driver waveform.

To set this global variable, use the `set` command.

timing_normalized_driver_waveform_weight_factor

```
timing_normalized_driver_waveform_weight_factor
<float value>
```

Default: 0.5

Defines the value of the weight of linear components in the algorithm for generation of normalized driver waveform.

When set to `1`, the waveform will be purely linear. When set to `0`, the waveform will be purely exponential. You can specify values within the range of 0 to 1.

To set this global variable, use the `set` command.

timing_null_collection_return_compatibility

```
timing_null_collection_return_compatibility
{true | false}
```

Default: `false`

Allows you to migrate previous release scripts to the new use model.

When the global is set to `true`, the software returns "NA" values that are consistent with the previous release. The nil return values are returned as an empty collection. This allows the return value to be used directly by collection iterators, such as `foreach_in_collection`.

The default value is `false`, which is consistent with the current release.

Note: This compatibility option may be removed from future releases. You should update your scripts to the new use model to avoid future problems.

To set this global variable, use the set_global command.

timing_path_based_enable_bounding_box_for_io_paths

```
timing_path_based_enable_bounding_box_for_io_paths
{true | false}
```

Default : `false`

Enables the bounding box mode for input-output paths.

To set this global variable, use the `set` command.

timing_path_based_enable_closest_common_pin_driver_for_bounding_box

```
timing_path_based_enable_closest_common_pin_driver_for_bounding_box
{true | false}
```

Default : `false`

Enables bounding box calculation using closest common pin driver from the clock mesh.

To set this global variable, use the `set` command.

timing_path_based_enable_exhaustive_depth_bounded_by_gba

```
timing_path_based_enable_exhaustive_depth_bounded_by_gba
{true | false}
```

Default: `true`

Enables the timing report to bound PBA results by GBA slacks of the respective leftover nworst paths, whenever exhaustive depth for an endpoint is exhausted.

When set to `true`, during exhaustive PBA, if you specify exhaustive depth as N, and an endpoint has nworst paths greater than N, then the timing report will show the worst path corresponding to worst slack out of worst PBA slack (out of first N paths) and GBA slack (for the (N+1)th path). Here, the variable N refers to exhaustive PBA depth and can be set using the `timing_pba_exhaustive_path_nworst_limit` global variable.

When set to `false`, during exhaustive PBA, if you specify exhaustive depth as N, and an endpoint has nworst paths greater than N, then the software will ignore the paths with depth greater than N, which may result in reporting an optimistic PBA slack for the endpoint reaching the depth.

To set this global variable, use the `set` command.

timing_path_based_enable_report_launch_clock_path

```
timing_path_based_enable_report_launch_clock_path {true | false}
```

Default: `true`

Allows the reporting launch clock path in path-based analysis (PBA) path type full report.

When set to `false`, the software does not report these paths in the timing reports.

To set this global variable, use the `set` command.

timing_path_based_enable_verbose_mode

`timing_path_based_enable_verbose_mode` *value*

Default: `1`

Controls printing of verbose messages during and at the end of path-based analysis (PBA).

You can specify one of the three values - 0, 1, and 2. These are described below:

- `0`: Verbose messages are not reported during exhaustive PBA analysis. For default PBA analysis, the path count will be reported.
- `1`: Reports the final message for PBA path count and max depth reached during exhaustive PBA. Progress messages are not reported.
- `2`: Reports multiple progress messages after regular intervals (in addition to the final message for PBA path count and max depth).

This global variable impacts messages when the `report_timing` and `report_constraint` commands are used in PBA mode.

To set this global variable, use the `set` command.

Examples

- The following commands print messages during and after PBA analysis:

  ```
  set timing_path_based_enable_verbose_mode 2
  report_timing -retime path_slew_propagation -retime_mode exhaustive -max_paths
  500000 -nworst 1
  ```

  The following progress messages are displayed during PBA analysis:

  ```
  INFO: Path Based Analysis (PBA) currently performed on '108455' paths
  INFO: Path Based Analysis (PBA) currently performed on '217574' paths
  INFO: Path Based Analysis (PBA) currently performed on '327511' paths
  INFO: Path Based Analysis (PBA) currently performed on '437472' paths
  ....
  ```

  The following messages are displayed at the end of PBA analysis mentioning the PBA path count and max depth reached during exhaustive PBA:

  ```
  INFO: Path Based Analysis (PBA) performed on total '2660526' paths
  INFO: Max depth 9460 reached for end point 'top/hier1/hier2/flop/d' during Path
  Based Analysis
  ```

timing_path_based_exhaustive_enable_design_coverage

```
timing_path_based_exhaustive_enable_design_coverage
{true | false}
```

Default: `false`

Enables exhaustive path-based analysis (PBA) to automatically pick up and analyze all the violating endpoints in the design, irrespective of the specified maximum paths.

When set to `true`, exhaustive PBA analysis automatically picks up and analyzes the violating design endpoints instead of N endpoints (specified with the `report_timing -max_paths` parameter). After analyzing the violating endpoints, the software reports the top N worst endpoints - depending on the specified value of N.

When set to `false`, exhaustive PBA analysis restricts analysis (using the `-max_paths` parameter) to the specified number of N endpoints.

Note: When this global variable is enabled, there may be an increase in run time because the software will analyze all the violating endpoints and analysis will not be restricted to the specified N endpoints.

To set this global variable, use the `set` command.

timing_path_based_exhaustive_max_paths_limit

`timing_path_based_exhaustive_max_paths_limit` *integer*

Default: `2000000`

Specifies the maximum number of paths that can be internally analyzed during exhaustive path-based analysis (PBA).

If the number of candidate paths during exhaustive PBA is greater than the specified max paths limit, then the software will skip path-based analysis of these paths beyond this limit. The paths exceeding the specified max paths limit will then be included as GBA in the final report output so that real violations are not missed.

To set this global variable, use the `set` command.

timing_path_based_report_analysis_summary_max_paths_limit

`timing_path_based_report_analysis_summary_max_paths_limit` *integer*

Default: `2000000`

Sets a limit on the maximum number of paths that can be reported in PBA mode with the `report_analysis_summary` command.

To set this global variable, use the `set` command.

timing_pba_exhaustive_path_nworst_limit

`timing_pba_exhaustive_path_nworst_limit` *value*

Default: `10000`

Specifies the maximum number of paths per endpoint (nworst) that will be re-timed in exhaustive path-based analysis (PBA).

In exhaustive PBA mode, the value specified with the `report_timing -nworst` parameter is used only during the final reporting. The value specified with the `timing_pba_exhaustive_path_nworst_limit` global variable (and not the `report_timing -nworst` value) is used to specify the maximum number of paths to be re-timed per endpoint.

If the limit specified by `timing_pba_exhaustive_path_nworst_limit` is reached and `timing_path_based_enable_exhaustive_depth_bounded_by_gba` global variable is set to:

- `true`: The worst of the current PBA slack is used and N+1th GBA path is reported.
- `false`: The current PBA slack is reported - which may be optimistic.

To set this global variable, use the `set` command.

timing_pll_clock_use_driving_cell

```
timing_pll_clock_use_driving_cell
{true | false}
```

Default : `false`

Uses delay calculation of all the enabled arcs to the PLL clock output to compute the transition.

When set to `true`, the delay calculation of all enabled arcs to the PLL clock output will be used to compute the transition. If no driving arcs are available in the model or if they are disabled, the same idealized transition will be used.

When set to `false`, the software considers the PLL clock output to be electrically isolated from the input-side reference clock, and creates the output clock with an idealized transition. You can use the `set_annotated_transition` command to customize the output clock transition.

To set this global variable, use the `set` command.

timing_prefix_module_name_with_library_genclk

```
timing_prefix_module_name_with_library_genclk
{true | false}
```

Default: `true`

When set to `true`, the software appends the instance name to the clock pin name when creating a generated clock. When set to `false`, the software uses only the clock pin name when creating a generated clock.

For example, assume the cell for instance `a/b` in the timing library contained the following generated clock group:

By default (`true`), the software creates a generated clock with the following name:

If you set the global to `false`, the software creates a generated clock with the following name:

If you set this global to `false`, and the timing_library_genclk_use_group_name global variable is set to `true`, the software creates a generated clock with the following name:

To set this global variable, use the set_global command.

timing_propagate_latch_data_uncertainty

```
timing_propagate_latch_data_uncertainty
{true | false}
```

Default: `false`

When set to `true`, uses the clock phase associated with a flush latch's data pin as the "from" clock phase for downstream uncertainty timing calculations

When set to `false`, the downstream uncertainty calculations are based on the clock phase of the flush latch's clock signal.

To set this global variable, use the `set` command.

timing_property_arrival_clocks_consider_clock_source_data_phase

```
timing_property_arrival_clocks_consider_clock_source_data_phase
{true | false}
```

Default : `true`

Considers the clock source data phases.

The `get_property` and `report_property` commands support the `arrival_clocks` collection property on pin objects. The `arrival_clocks` property considers data phases, clock source data phases, and latency phases.

To set this global variable, use the `set` command.

timing_property_arrival_clocks_consider_data_phase

```
timing_property_arrival_clocks_consider_data_phase
{true | false}
```

Default: `true`

Returns the pure data phase.

The `get_property` and `report_property` commands support the `arrival_clocks` collection property on pin objects. The `arrival_clocks` property considers data phases, clock source data phases, and latency phases.

To set this global variable, use the `set` command.

timing_property_arrival_clocks_consider_latency_phase

```
timing_property_arrival_clocks_consider_latency_phase
{true | false}
```

Default: `true`

Returns the latency phase.

The `get_property` and `report_property` commands support the `arrival_clocks` collection property on pin objects. The `arrival_clocks` property considers data phases, clock source data phases, and latency phases.

To set this global variable, use the `set` command.

timing_property_arrival_window_enable_tcl_dict_format

```
timing_property_arrival_window_enable_tcl_dict_format
{true | false}
```

Default : `false`

Allows the software to print new format of the `arrival_window` property (see `get_property` command).

To set this global variable, use the `set` command.

timing_property_clock_used_as_data_unconstrained_clock_source_paths

```
timing_property_clock_used_as_data_unconstrained_clock_source_paths
{true | false}
```

Default: `true`

Returns a value of true for `is_clock_used_as_data` timing property (set using the `get_property` command).

When set to `true`, the `is_clock_used_as_data` timing property returns a value of `true` (irrespective of whether the clock source path is constrained or unconstrained), if the pin lies in the clock source path and at least one of the clocks arriving at the pin is used as data in the downstream network of the pin.

When set to `false`, the pins lying in the constrained clock source data paths are reported as true. For example, if there is a path from clock source to output port and the output port is unconstrained, then the `is_clock_used_as_data` property will not return `true`.

To set this global variable, use the `set_global` command.

timing_property_return_null_collection_with_quiet

```
timing_property_return_null_collection_with_quiet
{true | false}
```

Default: `false`

Allows you query values of all the properties.

When set to `true`, the software enables returning values of properties of all the objects, when the `get_property -quiet` parameter is specified.

To set this global variable, use the `set` command.

timing_rail_swing_checks_high_voltage_threshold

`timing_rail_swing_checks_high_voltage_threshold d`*`ouble`*

Default: `0.95`

Specifies the voltage threshold values for high rail swing checks.

To set this global variable, use the `set` command.

timing_rail_swing_checks_low_voltage_threshold

`timing_rail_swing_checks_low_voltage_threshold d`*`ouble`*

Default: `0.05`

Specifies the voltage threshold values for low rail swing checks.

To set this global variable, use the `set` command.

timing_read_library_without_ecsm

```
timing_read_library_without_ecsm
{true | false}
```

Default: `false`

Disables loading ECSM data.

When set to `true`, the loading of ECSM data is disabled. If the timing library contains NLDM, ECSM, or CCS data in the same file, then ECSM data will not be read.

Note: This global variable does not disable CCS data loading.

To set this global variable, use the `set_global` command.

timing_read_library_without_sensitivity

```
timing_read_library_without_sensitivity
{1 | 0}
```

Default: `0`

Disables loading of ECSM timing sensitivity data.

When set to `1`, the global skips loading of all the data pertaining to ECSM timing sensitivity constructs when libraries are read.

To set this global variable, use the `set` command.

timing_recompute_sdf_in_setuphold_mode

```
timing_recompute_sdf_in_setuphold_mode
{true | false}
```

Default: `false`

Controls the recomputing of SDF delays when the software is in simultaneous setup and hold analysis mode.

When the software is not in simultaneous setup and hold analysis mode, only one corner of data is stored at a time; therefore additional delay calculation is required for `write_sdf`. When the software is in simultaneous setup and hold analysis mode, both sets of data are in memory at the same time, and additional delay calculation is not necessary.

If set to `false`, `write_sdf` will write out the delays from memory, instead of calling delay calculation.

Note: This global variable must be used with `timing_enable_simultaneous_setup_hold_mode true`.

To set this global variable, use the `set_global` command.

timing_reduce_multi_drive_net_arcs

```
timing_reduce_multi_drive_net_arcs
{true | false}
```

Default: `true`

Controls whether multi-drive or mesh networks are reduced to simpler, single driver models.

When set to `true`, the nets that meet the drivers' receivers arc threshold (defined using `timing_reduce_multi_drive_net_arcs_threshold` global variable), are reduced. The net arcs are created only from one driver or from the drivers for which you have set the constraints.

If you specify constraints on the net driven by parallel buffers, it might result in increased run time and memory degradation. By specifying timing constraints through nets, the constraints are moved to the drivers. This leads to the creation of net arcs from all the drivers, thus disabling net arc reduction. In such cases, you can select one driver of the net and use that driver to specify the constraints.

When set to `false`, the number of net arcs created for timing analysis for nets driven by parallel buffers are not reduced. If a net is driven by `n` parallel buffers and is connected to `m` sink pins, then the total number of net arcs created is `n*m`. For example, if `n` is `100` and `m` is `20000`, the number of net arcs created for timing analysis will be two million. This may result in increased run time and memory usage.

To set this global variable, use the `set_global` command.

timing_reduce_multi_drive_net_arcs_threshold

`timing_reduce_multi_drive_net_arcs_threshold` *int*

Default: `10000`

Sets a threshold number that is used to trigger the reduction of timing arcs of nets driven by parallel buffers. The reduction takes place if the number of arcs is greater than the threshold value and when the `timing_reduce_multi_drive_net_arcs` global variable is set to `true`.

To set this global variable, use the `set_global` command.

timing_remove_clock_reconvergence_pessimism

```
timing_remove_clock_reconvergence_pessimism
{true | false}
```

Default: `false`

Enables Clock Path Pessimism Removal (CPPR) analysis during timing analysis. When set to `true`, the timing analysis commands remove pessimism from slack calculation. Clock path pessimism is the difference of launching and capturing clock arrival times at the common node. This pessimism can be due to difference in minimum and maximum cell delays along the clock paths or due to the reconvergent clock paths. If the launching and capturing registers are triggered by the same clock and there is no common node between the launching and capturing clock paths, the pessimism is the difference between late and early clock source latencies.

To set this global variable, use the set_global command.

Note: The `timing_remove_clock_reconvergence_pessimism` global should not be used with the `setAnalysisMode` command. Use this global only when the `setAnalysisMode` command is not available. When the `setAnalysisMode` command is available, use the `-cppr` parameter to set or reset the CPPR analysis.

timing_report_analysis_summary_csv_extended_new_format

```
timing_report_analysis_summary_csv_extended_new_format
{true | false}
```

Default: `false`

Keeps the analysis summary CSV extended report in sync with the summary timing report. These reports are available when the respective `report_timing` parameters are specified:

- `-analysis_summary_csv_extended_file`
- `-analysis_summary_file`

When set to `true` (and when the `report_timing -analysis_summary_csv_extended_file` parameter is specified), the column name sequence will be from fromClk to toClk, that is, fromClk will be printed first, and then toClk. Also, the timing report will show "-", if toClk or fromClk is not present (and when the `report_analysis_summary -expanded_summary_clocks` parameter is set to `false`).

When set to `false`, the missing toClk or fromClk column will be represented as “*”.

To set this global variable, use the `set` command.

timing_report_begin_end_pair_max_path_limit

`timing_report_begin_end_pair_max_path_limit` *integer*

Default: `2000000`

Sets a limit on the maximum number of paths that can be reported when the `report_timing -begin_end_pair` parameter is specified.

Note: Higher maximum paths limit may lead to high memory usage and runtime.

To set this global variable, use the `set` command.

timing_report_check_timing_unconstrained_endpoints_due_to_constants

```
timing_report_check_timing_unconstrained_endpoints_due_to_constants {true | false}
```

Default: `false`

Enables the software to flag unconstrained signals arriving at the end points due to case analysis.

When set to `false`, the `check_timing` command output will show warnings for the unconstrained endpoints.

When set to `true`, the command output will also show warnings for unconstrained endpoints due to case analysis under the subcategory `uncons_endpoints_due_to_case_analysis`. For more details, see `check_timing`.

To set this global variable, use the `set` command.

timing_report_clock_pin_as_begin_point

```
timing_report_clock_pin_as_begin_point
{true | false}
```

Default: `false`

Allows the software to report clock pins as begin points.

To set this global variable, use the `set` command.

timing_report_constraint_enable_extended_drv_format

```
timing_report_constraint_enable_extended_drv_format
{true | false}
```

Default: `false`

Enables the software to report data as per clock/data and rise/fall phases.

To set this global variable, use the `set` command.

timing_report_constraint_format

`timing_report_constraint_format` *`string`*

Default: " "

Specifies the DRV fields that will be reported with the `report_constraint` command. For valid format column options, refer to `report_constraint -drv_fields`.

If DRV fields are not specified with this global variable (or with `report_constraint -drv_fields`), then the default set of DRV fields will be displayed. By default, the constraint report will display the pin names, the required and actual DRC values, and the view names.

Note: The reporting fields specified with the `report_constraint -drv_fields` parameter will override the global variable settings.

To set this global variable, use the `set` command.

timing_report_constraint_rise_fall_clock_period_check

```
timing_report_constraint_rise_fall_clock_period_check
{true | false}
```

Default: `false`

Enables reporting of rise-to-rise and fall-to-fall clock_period checks separately for the `report_constraint` command.

When set to `false`, the software reports the rise-to-rise and fall-to-fall clock period checks under the same table in the report. With the `false` setting, when the `report_constraint -check_type clock_period` option is specified, the software reports violations that are the worst amongst all the rise-fall transitions for each point.

To set this global variable, use the `set` command.

timing_report_constraint_use_infinity_slack_for_unconstrained

`timing_report_constraint_use_infinity_slack_for_unconstrained {true | false}`

Default: `false`

Reports the timing slacks of unconstrained paths through the DRV violating pin as “INFINITY”. By default, the timing slacks of unconstrained paths are reported as * (asterisk).

To set this global variable, use the `set` command.

timing_report_disable_max_paths_per_group

```
timing_report_disable_max_paths_per_group {true | false}
```

Default: `false`

Controls the interpretation of the `report_timing -max_paths` parameter when you request reporting across a set of path groups. By default (`false` setting), the specified `-max_paths` will be reported per path group. With the `true` setting, a total of the specified `-max_paths` will be reported across all of the specified path groups.

For Stylus flow cross-tool compatibility – you should specify a setting of `true`. For backward compatibility with the legacy UI flow – you should use a value of `false`.

For example,

When set to `false`:

Legacy UI: `report_timing -path_group {reg2reg in2reg} -nworst 2 ...`
Stylus UI: `report_timing -group {reg2reg in2reg} -nworst 2 ...`

The `report_timing` command will return up to 2 paths for the reg2reg group and also up to 2 paths for the in2reg group.

When set to `true`:

Legacy UI: `set timing_report_disable_max_paths_per_group true`
Stylus UI: `set_db timing_report_disable_max_paths_per_group true`

A total of 2 paths will be reported across both path groups. The paths may come entirely from one or the other group – or both, based on the slack value.

To set this global variable, use the `set` command.

timing_report_drv_enable_clock_source_as_clock

```
timing_report_drv_enable_clock_source_as_clock {true | false}
```

Default: `false`

Enables the clock DRV for pure clock source path instead of data DRV.

To set this global variable, use the `set` command.

timing_report_drv_enable_frequency_per_view

```
timing_report_drv_enable_frequency_per_view
{true | false}
```

Default : `false`

Enables view-based frequency DRV flow.

When set to `true`, in the MMMC DRV flow, the software performs frequency computation for design pins as per the clock period of specific analysis views reaching a pin. In case of non-activity based flow, the frequency is computed as per the fastest clock of all the views.

When set to `false`, in the propagate activity flow, the software performs frequency computation for design pins from propagated activity on that pin.

For other flows. the software performs frequency computation for design pins as per the smallest clock period (largest clock frequency) from all the active analysis views.

To set this global variable, use the `set` command.

timing_report_drv_enable_ghz_notation_for_drv_fields

```
timing_report_drv_enable_ghz_notation_for_drv_fields {true | false}
```

Default: `false`

Enables reporting of clock frequency in GigaHertz (Ghz). The default frequency unit is MegaHertz (MHz).

To set this global variable, use the `set` command.

timing_report_drv_enable_slew_threshold_scaling

```
timing_report_drv_enable_slew_threshold_scaling {true | false}
```

Default: `false`

Enables the transition constraints scaling as per the slew threshold value.

To set this global variable, use the `set` command.

timing_report_drv_use_worst_timing_slack

```
timing_report_drv_use_worst_timing_slack
{true | false}
```

Default: `false`

Enables reporting of the worst DRV slack across all the views for configurable DRV report.

When set to `true`, the software shows a non-default special case report, where timing slacks for MMMC flow is the worst among all active views.

To set this global variable, use the `set` command.

timing_report_enable_clock_unrolling

```
timing_report_enable_clock_unrolling
{true | false}
```

Default : `false`

Enables the clock unrolled reporting format that shows more intuitive timing reports with respect to constraints.

When set to `true`, the `report_timing` command provides an optional format where the clock edge arrival times in the detailed path section show the worst-case clock edge pair used for the analysis. This format can provide a more intuitive understanding of the clock edges used - especially in cases where the launch and capture clocks have different frequencies.

When set to `false`, the detailed path section shows clock edge arrival times with respect to the base cycle of the clocks. Any required cycle-accounting adjustments are provided by the Phase Shift and Cycle Adjustment entries in the timing report summary section.

To set this global variable, use the `set` command.

timing_report_enable_cppr_point

```
timing_report_enable_cppr_point { true | false}
```

Default: `false`

Enables the timing report to output CPPR point details.

When set to `true`, the `report_timing` command output shows the CPPR point field.

To set this global variable, use the `set` command.

timing_report_enable_lead_trail_to_rise_fall_map

```
timing_report_enable_lead_trail_to_rise_fall_map
{true | false}
```

Default: `false`

Enables reporting of rise/fall edge times, instead of lead/trail edge times of a clock (that are reported using the `report_clocks` command).

To set this global variable, use the `set` command.

timing_report_enable_markers

```
timing_report_enable_markers
      {true | false}
```

Default: true

Enables display of markers in reports.

When set to `true`, the report_timing output format displays markers that identify pins specified using the `-from`, `-to`, or `-through` parameters. The markers, displayed as '`->`', show up in the 'timing point' or ' hpin' column (if the 'timing point' column is not available) of the timing report. This feature provides better readability. When set to `false`, the timing report does not display markers.

To set this global variable, use the set_global command.

timing_report_enable_max_capacitance_drv_for_constants

```
timing_report_enable_max_capacitance_drv_for_constants
{true | false}
```

Default: `false`

When set to `true`, performs max_capacitance design rule violation check on constant elements.

Note: This global variable will be honored only when `timing_disable_drv_reports_on_constant_nets` is set to `true`.

To set this global variable, use the `set_global` command.

timing_report_enable_max_path_limit_crossed

```
timing_report_enable_max_path_limit_crossed
{true | false}
```

Default: `false`

Determines if the number of reported paths for a particular clock group equals the specified maximum paths limit.

When set to `true`, the software will issue a warning message in case the number of reported paths for a clock group is equal to the given maximum paths limit. This applies to both graph- and path-based reporting modes.

This variable is supported in clock group-based non-statistical reporting flows only. The clock group-based reporting mode can be enabled by setting `timing_report_group_based_mode` global variable to `true`.

To set this global variable, use the `set` command.

timing_report_enable_report_clock_timing_across_clock_pin

```
timing_report_enable_report_clock_timing_across_clock_pin {true | false}
```

Default: `false`

Enables fanout tracing across clock pins if any combinational arc is present.

To set this global variable, use the `set` command.

timing_report_enable_si_debug

```
timing_report_enable_si_debug
{true | false}
```

Default: `false`

Allows the software to capture signal integrity (SI) related details for each stage of a path being analyzed during path-based analysis in the form of attributes.

To set this global variable, use the set_global command.

timing_report_enable_slack_summary_at_bottom

```
timing_report_enable_slack_summary_at_bottom {true | false}
```

Default: `false`

Prints additional time details, such as, Required time, Arrival, and Slack, at the bottom of the timing report. For example,

```
Req Time = 0.262  (-3.00S) | 0.544  | 0.094 | 0.141
Arr Time = 2.238  (+3.00S) | 1.774  | 0.155 | 0.131
Slk Time = -1.773 (-3.00S) | -1.230 | 0.181 | 0.181
```

To set this global variable, use the `set` command.

timing_report_enable_unique_pins_multiple_capture_clock_paths

`timing_report_enable_unique_pins_multiple_capture_clock_paths`

Default: `false`

Enables reporting of different paths with respect to the reference clocks in the unique pins.

To set this global variable, use the `set` command.

timing_report_enable_use_valid_start_end_points

```
timing_report_enable_use_valid_start_end_points
{true | false}
```

Default: `false`

Shows valid start/end points in the timing report.

When set to `true`, the valid start/end points are included in the timing report. The paths are reported with the `report_timing` command - when the pins passed (-from and -to) are the actual begin and end pins of the data path. For example, for a register the begin pin is register's CK pin and the end pin is register's D pin. Similarly, an input pin in a path is the path's begin pin and output pin is the path's end pin.

When set to `false`, any point in a path can be used as a -from point and is considered as a -through point internally. All the pins using -from/-to are treated as -through pins. Therefore, a path is reported with register's Q pin as a from pin, even though the correct start pin is a CK pin. Or any intermediate pin belonging to a path can be passed as -from/-to and the path will still be reported to correct start/end pins with the given pin lying somewhere in between of path.

To set this global variable, use the `set` command.

timing_report_generated_clock_info

```
timing_report_generated_clock_info
{true | false}
```

Default: `true`

Generated clock information is automatically added to the report, if generated clocks are encountered in either the launching or capturing clock paths.

The generated clock information column is displayed if the `Delay' field exists. If the `Delay' field does not exist, then the generated clock adjustment column is added to the report only when explicitly specified using the `report_timing -format` command or by setting the `report_timing_format` global variable.

To set this global variable, use the set_global command.

timing_report_group_based_mode

```
timing_report_group_based_mode
{true | false}
```

Default: `false`

Groups paths by clock domain.

When set to `true`, the behavior of the following reporting commands will change as follows:

report_timing

The behavior of the following report_timing parameters changes as follows:

- `-max_points`: In this mode you are not required to specify this parameter. If specified, the parameter is mapped to -max_paths option.
- `-max_paths M`: Reports M paths per clock group.
- `-nworst N`: Limits the number of paths per endpoint per clock group. By default, the value of -nworst is 1.

The report_timing command will have the following four types of groups (if design does not have user-defined path groups):

- Asynch group: Recovery/removal checks.
- Clock gating group: Inferred and library clock gating checks.
- Default group: Combinational max delay or min delay paths.
- Clock Groups: Groups for all design clocks. For a design with three clocks, three groups will be reported, one group for each clock. Endpoints sharing the same reference clock will be part of the same group. If an endpoint belongs to multiple groups then it will be reported separately in each group.

Note: If user-defined path groups are specified, the software will override the path groups described above.

report_min_pulse_width

The `report_min_pulse_width` command reports pulse width slacks of all the clocks for a pin. In summary mode, the "clock" column will be displayed. In verbose mode (with `path_type` option), the behavior will be similar to the `report_timing` output in group-based mode.

report_constraint

The `report_constraint` command will perform the following checks:

- For max_delay/setup and min_delay/hold type checks paths will be reported for each clock group separately.
- The behavior of pulse width check types will be the same as `report_min_pulse_width` command.

report_analysis_coverage

The `report_analysis_coverage` command will report a check at end point for all the clocks. In verbose mode, the "clock" column will be displayed.

To set this global variable, use the set_global command.

timing_report_max_transition_check_using_nsigma_slew

```
timing_report_max_transition_check_using_nsigma_slew
{true | false}
```

Default: `false`

Enables the `report_constraint` command to use mean and sigma slew values for reporting transition values in DRV checks.

When set to `true`, in the SOCV flow the `report_constraint` command uses the mean plus n-sigma slew values for DRV checks.

When set to `false`, the `report_constraint` command will report only the mean slews.

To set this global variable, use the `set` command.

timing_report_property_fastest_clock_consider_data_phase

`timing_report_property_fastest_clock_consider_data_phase`

`{true | false}`

Default : `false`

Considers the data signals while determining the fastest clock arriving on any given pin.

When set to `false`, the clocks arriving in clock phase form will be considered.

When set to `true`, the clocks arriving in clock phase as well as data phase form will be considered for determining the fastest clock.

To set this global variable, use the `set` command.

timing_report_pulse_width_matching_launch_capture_paths

`timing_report_pulse_width_matching_launch_capture_paths {true | false}`

Default: `false`

Reports pulse width paths that have matching launch and capture paths - the count of pins and the pins must match exactly. The paths that do not meet this condition will not be reported.

When set to `true`, the software will impact the minimum pulse width report that is generated with the `report_min_pulse_width` command along with the `-path_type short` and `-path_type full_clock` options.

To set this global variable, use the `set` command.

timing_report_redirect_message_types

```
timing_report_redirect_message_types
{none | info}
```

Default: `none`

Redirects reporting messages to a report file.

When set to `info`, the path count and max depth (displayed when the `report_timing` command is run in PBA mode) will be appended to the final report output.

When set to `none`, the reporting messages are not redirected to a report file.

To set this global variable, use the `set` command.

timing_report_retime_formatting_mode

```
timing_report_retime_formatting_mode
{manual | retime_compare | retime_replace}
```

Default: `manual`

Controls retiming fields to be displayed in the timing report.

You can set one of the following options to change the reporting columns:

- `manual`: Does not change the `report_timing` format.
- `retime_compare`: If GBA columns are present in the report format, then the corresponding retime options are added. If PBA columns are already present, then there is no change in the format.
- `retime_replace`: If GBA columns are present in the report format, then the GBA columns are replaced by the corresponding PBA columns. If PBA columns are already in the report, then there is no change in the format.

Note: The report format is also controlled through the `report_timing -retime_format` parameter. The `report_timing` command setting has precedence over this global variable.

To set this global variable, use the `set_global` command.

timing_report_skip_constraint_loop_check

```
timing_report_skip_constraint_loop_check
{true | false}
```

Default: `false`

Allows you to skip loop checking when the `check_timing` command is used.

When set to `true`, the software skips reporting of loops in case the timing graph inside a loop is broken. This may happen if some of the commands, such as, `create_clock`, `set_input_delay`, or `set_output_delay` are defined inside a loop.

Note: When this global variable is set to `true`, the `check_timing` command will report lesser loops than the ones reported in the CTE_loops.rpt report - which reports all the loops.

To set this global variable, use the `set` command.

timing_report_socv_summary_mean_sigma

`timing_report_socv_summary_mean_sigma {true | false}`

Default: `false`

Prints the report header in new format when the `report_timing` command is run in SOCV mode. The new header contains nsigma, mean, and sigma details.

To set this global variable, use the `set` command.

timing_report_timing_header_detail_info

```
timing_report_timing_header_detail_info
{default | extended}
```

Default: `default`

Controls the timing report (generated using the `report_timing` command) header format. You can specify one of the following options:

- When set to `default`, the software generates timing reports with a report header that includes the following information:
  - Tempus software version number
  - Platform on which the software is being run
  - Time when the report was generated
  - Command used to generate the report
- When set to `extended`, the software generates timing reports with additional header information given below:
  - The analysis mode - best case worst case or on chip variation
  - The active views (displayed only in MMMC mode)
  - The clock reconvergence pessimism (CPPR)
  - The latch slack mode
  - The specified clock propagation value
  - The specified delay calculation engine

Note: You can use this global variable when `timing_disable_report_header_info` is set to `false`.

To set this global variable, use the `set_global` command.

Examples

Here are some sample report headers.

- The following report header is displayed when the default option is specified:
  ```
  innovus 1> set_global timing_report_timing_header_detail_info default
  ```

```
#####################################################################
# Generated by: Cadence Innovus <version>
# OS: Linux x86_64(Host ID sjkrhn)
# Generated on: <date>
# Design: TEST_CHIP
# Command: report_timing
#####################################################################
```

- The following sample report header is displayed the extended option is specified:

```
innovus 3> set_global timing_report_timing_header_detail_info extended


#####################################################################
# Generated by: Cadence <version>
# OS: Linux x86_64(Host ID sjkrhn)
# Generated on: <date>
# Design: TEST_CHIP
# Command: report_timing
#####################################################################
#
# Timing Analysis:
# Analysis Mode : onChipVariation
# Active Views : test_view_setup
# CPPR : on
# Latch Slack Mode : max_borrow
# Clock Propagation : sdcControl
# Delay Calculation:
# Engine : default
#
#####################################################################
```

timing_report_unconstrained_path_early_late_header

```
timing_report_unconstrained_path_early_late_header
{true | false}
```

Default: `false`

When set to true, the software provides information in the timing report header that indicates whether an unconstrained path is a late or early path.

To set this global variable, use the `set` command.

timing_report_unconstrained_paths

```
timing_report_unconstrained_paths
{true | false}
```

Default: `false`

When set to `true`, the `report_timing` command reports unconstrained paths if the software cannot find any constrained paths to report.

When set to `false`, the `report_timing` command reports only the constrained paths.

Note: The behavior of `timing_report_unconstrained_paths` global variable is not the same as the behavior of `report_timing -unconstrained` command. If there are several paths to the same endpoint from different beginpoints, including constrained and unconstrained paths, the `report_timing -unconstrained` parameter always reports unconstrained paths, even if a constrained path can be reported. Whereas the `timing_report_unconstrained_paths` global reports constrained paths but not unconstrained paths.

To set this global variable, use the set_global command.

timing_report_use_receiver_model_capacitance

```
timing_report_use_receiver_model_capacitance {true | false}
```

Default: `false`

Enables the max_capacitance violations based on dynamic pin capacitance.

When set to `true`, the software checks for both the input level and timing arc level dynamic pin capacitance tables for worst-case pin capacitance table for given input slew. If conditional pin capacitance tables (or timing_arcs) exist, the worst-case is used among all the conditions.

This global variable performs static pin capacitance inference after delay calculation, so the delay computation is not impacted. The pin capacitance is used only for reporting and max_cap violations.

To set this global variable, use the `set` command.

timing_report_use_worst_parallel_cell_arc

```
timing_report_use_worst_parallel_cell_arc
{true | false}
```

Default: `false`

When set to `false`, setting the `-nworst` parameter reports several paths by selecting parallel arcs in the library cell between two pins.

When set to `true`, only a single worst path is reported by selecting the worst delay arc among all parallel arcs of a cell. For example, for reporting timing through A to Y pins (where the cell is a 2-input cell), the parallel arc is applicable for the same input and output pin. So the worst arc for A->Y is chosen from all arcs of A->Y.

To set this global variable, use the set_global command.

timing_resolve_driver_conflicts

```
timing_resolve_driver_conflicts
{conservative | aggressive}
```

Default: `aggressive`

Resolves constant propagation results on nets with multiple drivers.

When set to `conservative` mode, the net values are as follows:

- If all the drivers are 1, the net will be driven to value `1`
- If all the drivers are 0, the net will be driven to value `0`
- In other cases, the net value will be `Undefined`

When set to `aggressive` mode, the net values are as follows:

- If some of the drivers are 0, the net value will be `0`
- If some of the drivers are 1, but none are 0, then the net value will be `1`
- In other cases, the net value will be `Undefined`

To set this global variable, use the `set_global` command.

timing_sdf_adjust_negative_setuphold

```
timing_sdf_adjust_negative_setuphold
{true | false}
```

Default: `false`

Controls how the setup and hold check values are adjusted when both are initially negative in the SDF file.

When set to `true`, the software sets the smaller check delay value to a negated larger check delay value. For example, if the initial setup/hold value is equal to -1/-2. then the value -2 from the setup is smaller and it is reset to a negated value of the larger one (-1 from hold). So the adjusted setup/hold value is equal to -1/1.

When set to `false`, both the negative setup and hold check values are set to 0.

The command order for using this global variable is as follows:

- `read_sdf`
- `timing_sdf_adjust_negative_setuphold {true | false}`
- `write_sdf`

The timing report (`report_timing`) results are not impacted.

To set this global variable, use the `set` command.

timing_sdf_enable_setuphold_scond_ccond

```
timing_sdf_enable_setuphold_scond_ccond
{true | false}
```

Default: `false`

When set to `true`, enables proper generation of SDF (Standard Delay Format) with scond and ccond qualifiers on `SETUPHOLD` and `RECREM` timing checks:

Since some SDF readers do not accept scond and ccond qualifiers this global is set to `false` by default.

To set this global variable, use the set_global command.

timing_self_loop_paths_no_skew

```
timing_self_loop_paths_no_skew
{true | false}
```

Default: `false`

When set to `true`, factors the skew on self loops. The software ignores the skew on the clock pin for hold analysis for data paths originating on sequential elements (for example, Q-pin of a flip flop) and ending on the data pins of the same element (D-pin of the same flip flop). Use this variable to eliminate pessimism due to skew when the launch and the capture edge of the clock are the same.

To set this global variable, use the set_global command.

Note: The `timing_self_loop_paths_no_skew` global should not be used with the `setAnalysisMode` command. Use this global only when the `setAnalysisMode` command is not available. When the `setAnalysisMode` command is available, use the `-timingSelfLoopsNoSkew` parameter to enable skew factoring on self loops.

timing_self_loop_paths_no_skew_max_depth

`timing_self_loop_paths_no_skew_max_depth` *value*

Default: 10

Identifies self-loop paths at the specified depth.

To set this global variable, use the set_global command.

For example:

set_global timing_self_loop_paths_no_skew_max_depth 20

Lets the timer traverse 20 timing arcs (10 stages of logic) before abandoning the search for a self loop.

timing_self_loop_paths_no_skew_max_slack

`timing_self_loop_paths_no_skew_max_slack` *value*

Default: `0`

Identifies self-loop paths at the specified threshold.

To set this global variable, use the set_global command.

For example:

set_global timing_self_loop_paths_no_skew_max_slack 1.0

In this case, all timing endpoints with a worst slack less than `1.0` are considered for self loop skew removal.

timing_set_clock_source_to_output_as_data

```
timing_set_clock_source_to_output_as_data
{true | false}
```

Default: `false`

When set to `true`, causes a clock source path leading to an output or bidi port to be treated as a data path if there is a `set_output_delay` or `set_data_check` assertion on the port.

To set this global variable, use the set_global command.

timing_set_nsigma_multiplier

`timing_set_nsigma_multiplier` *`value`*

Default: 3

Controls the sigma multiplier to be used in SOCV mode.

This global variable allows a single N value for both setup and hold reporting. For example,

```
set timing_set_nsigma_multiplier 4.5
```

To set this global variable, use the `set` command.

Note: The `timing_set_nsigma_multiplier` global variable is obsolete and has been replaced with the `set_socv_reporting_nsigma_multiplier` command. You can use this global variable by setting `timing_socv_view_based_nsigma_multiplier_mode` to `false`. For future compatibility, you should migrate your scripts to use the new view-based methodology.

timing_set_scaling_for_negative_checks

```
timing_set_scaling_for_negative_checks
{default | divider | multiplier}
```

Default: `default`

When set to default, the derated check is the negative check * (2.0 - derating factor), where the derating factor is the value that you set using the `set_timing_derate` command.

Modifies the scaling of the negative timing analysis check value. This variable does not affect the positive checks. When set to `divider`, divides the negative check value by the derating factor. When set to `multiplier`, multiplies the negative check value by the derating factor.

To set this global variable, use the set_global command.

timing_set_scaling_for_negative_delays

```
timing_set_scaling_for_negative_delays
{default | divider | multiplier}
```

Default: `default`

Modifies the scaling of the negative delay value to be used during timing analysis. This variable does not affect the positive delay.

When set to `default`, the derated check/delay = negative check/delay * (2.0 - derating factor), where the derating factor is the value that you set using the set_timing_derate command.

When set to `divider`, divides the negative delay value by the derating factor. When set to `multiplier`, multiplies the negative delay value by the derating factor.

Currently, constraints are used during timing in the flattened mode, so the internal ILM instances are seen instead of the LEF pins of the ILMs. Therefore, reading the bounding box constraints causes errors without this variable set. Notice that if you want to see the LEF pins of the ILM in GUI, the design must be in the unflattened mode.

To set this global variable, use the set_global command.

timing_socv_preserve_variation_with_annotations

```
timing_socv_preserve_variation_with_annotations
{true | false}
```

Default: `false`

Preserves the sigma value computed by delay calculation along with the annotation from SDC.

In the clock mesh flow, it is possible to preserve sigma values even after applying annotation on slew/delay through SDC.

When this global variable is set to `true`, the software accepts the sigma value computed by delay calculation (for nets that have delay/slew annotations from SDC, e.g., clock-mesh), along with the annotation from SDC.

To set this global variable, use the `set` command.

timing_socv_rc_variation_mode

```
timing_socv_rc_variation_mode
{true | false}
```

Default: `false`

Enables modeling of interconnect variation in SOCV mode.

The SOCV flow provides support for RC variation on the interconnect.

To set this global variable, use the `set` command.

timing_socv_statistical_min_max_mode

```
timing_socv_statistical_min_max_mode
{statistical | mean_and_three_sigma_bounded}
```

Default: `mean_and_three_sigma_bounded`

Determines how worst cases will be computed in SOCV mode.

You can select one of the following options:

- `statistical`: The mean and standard deviation for the result of worst-cases is computed to get the greatest or smallest of normal distributions.
- `mean_and_three_sigma_bounded`: The worst mean value is chosen as the mean of the resulting distribution. The standard deviation is computed as a bound from the worst (mean+/-3*sigma) value. This ensures that the 3-sigma value of the result always bounds the worst 3-sigma value.

  Note: The `mean_and_three_sigma_bounded` option is required for SOCV-based optimization in Innovus. It is also highly recommended for analysis in Tempus due to improved bounding properties.

To set this global variable, use the `set` command.

timing_socv_view_based_nsigma_multiplier_mode

```
timing_socv_view_based_nsigma_multiplier_mode {true | false}
```

Default: `true`

Enables setting of view-based separate setup and hold sigma multipliers.

By default or with an explicit `true` setting, the timing system is configured to support different N-sigma multiplier values for Setup vs. Hold – and for different analysis views. The specific N-sigma multipliers can be set using the `set_socv_reporting_nsigma_multiplier` command.

In earlier releases, the `timing_set_nsigma_multiplier` global variable was used to specify a single N-sigma value that was used for both Setup and Hold analysis across all analysis views. To continue using older scripts with this control, you can set the `timing_socv_view_based_nsigma_multiplier_mode` global variable to `false`.

Note: This backward compatibility option will not be supported in future releases. It is recommended to migrate your scripts to use the new view-based methodology.

To set this global variable, use the `set` command.

timing_spatial_derate_chip_size

`timing_spatial_derate_chip_size` *`value_in_microns`*

Default: `1e+30`

Allows you to define the chip size.

The chip size is the diagonal length of a rectangle that is formed by the chip's length and width. You can obtain the diagonal distance by calculating the square-root of the Length^2 x Width^2. The diagonal distance should always be specified in units of microns.

The chip size can be picked in the following ways (in order of highest to lowest precedence):

- When set to a value other than `1e+30`, the specified value will have the highest precedence.
- If the physical floorplan information is available, the chip size will be derived automatically from the floorplan.
- If the SPEF parasitic files containing the coordinate information are loaded, then the chip size will be automatically derived.

To set this global variable, use the `set` command.

timing_spatial_derate_distance_mode

```
timing_spatial_derate_distance_mode
{bounding_box | chip_size}
```

Default: `bounding_box`

Allows you to select the graph-based analysis (GBA) spatial mode. The allowed values are - `bounding_box` or `chip_size`.

This global variable can be used when the spatial derate mode is enabled using `timing_enable_spatial_derate_mode true` setting.

To set this global variable, use the `set` command.

timing_suppress_escape_characters

```
timing_suppress_escape_characters
{true | false}
```

Default: `true`

Supresses the reporting of escape characters in timing object names.

When the global is set to `true`, all the reporting and collection commands report the timing objects names (e.g., pins, ports, or instances) without the escape characters.

To set this global variable, use the set_global command.

timing_suppress_ilm_constraint_mismatches

```
timing_suppress_ilm_constraint_mismatches
{true | false}
```

Default: `false`

When set to `true`, the software suppresses all error and warning messages related to objects not found when loading SDC constraint files for the ILM flow.

In the current ILM flow, SDC constraints originally specified against the complete flat netlist for the design do not get filtered when being applied against the ILM view of the design. Constraints that reference parts of the design that were pruned cause warnings and errors during constraint loading.

To set this global variable, use the set_global command.

timing_time_unit

`timing_time_unit` *`string`*

Default: `none`

Specifies the timing units to be used for timing constraints and timing reports. The default units for timing are inferred from the timing libraries. You can override this inference by explicitly setting the value to `none`, `1ns`, `1ps`, `10ps`, or `100ps` .

Setting this variable explicitly is the same as executing the `setLibraryUnit -time` command to set time units.

To set this global variable, you can use the `set` command.

timing_timing_window_pessimism_removal_include_si_delay

`timing_timing_window_pessimism_removal_include_si_delay {true | false}`

Default: `false`

Controls inclusion of SI delay in timing window pessimism removal between the attacker and victim pins in the design.

The cross-talk analysis of a net depends on the relative positioning of attacker and victim arrival times. To compute the worst case switching times, the arrival windows of attacker and victim are considered. If the attacker and victim share a common clock path, then considering the original arrival windows will be pessimistic, and the common path contribution in these arrival windows needs to be adjusted.

When set to `false`, the software adjusts the elongated timing windows with the difference in the minimum-maximum base delay of the common clock path at the top level - while calculating the closest switching time, and the SI incremental delay differences are not adjusted.

When set to `true`, the software adjusts the timing windows with the SI incremental delay differences as well, in addition to the base delay. When this global variable is enabled, the timing window calculation can be optimistic, especially for cases where the attacker and victim are triggered by different clock edges. But for cases where all the attacker and victim nets are triggered by the same clock edge, then real pessimism will be removed from the timing window calculation.

Note: The `timing_timing_window_pessimism_removal_include_si_delay` global variable applies only when `timing_enable_timing_window_pessimism_removal` is set to `true`.

To set this global variable, use the `set` command.

timing_use_clock_pin_attribute_for_clock_net_marking

```
timing_use_clock_pin_attribute_for_clock_net_marking
{true | false}
```

Default: `false`

Allows the software to use clock pin attribute settings for clock net marking.

The timing system determines which nets are recognized as clock nets by considering different heuristics - including having an upstream path to clock sources and a downstream path to clock targets.

When set to `true`, the pins that have the Liberty timing library pin attribute setting ‘clock:true’ will be interpreted as additional clock targets for the clock marking algorithm.

When set to `false`, the set of clock targets includes pins that are the launching point for sequential paths or are the reference pin for sequential timing checks - typically register clock pins.

To set this global variable, use the `set` command.

timing_use_incremental_si_transition

```
timing_use_incremental_si_transition {true | false}
```

Default: `false`

Enables usage of incremental slew during DRV violation reporting.

To set this global variable, use the `set` command.

timing_use_latch_early_launch_edge

```
timing_use_latch_early_launch_edge
{true | false}
```

Default: `true`

Enables the latch use early launch edge feature.

During timing analysis when time borrowing and clock path pessimism removal (CPPR) is enabled, the given time is calculated using both borrow time and CPPR time. This may result in unnecessary timing violations. Since CPPR is an accurate means of pessimism removal, you are required to set the `timing_use_latch_early_launch_edge` global variable to `false` so that CPPR is applied to all the paths in the design.

When set to `true`, the launch clock path of timing paths starting at a latch is timed in early mode for setup analysis, and late mode for hold analysis. No CPPR credit is given to the timing path.

When set to `false`, the behavior is like for any other path. The launch clock path of timing paths starting at a latch is timed in late mode for setup analysis, and early mode for hold analysis. The CPPR credit is given to the path according to the global CPPR settings.

To set this global variable, use the set_global command.

timing_use_latch_time_borrow

```
timing_use_latch_time_borrow
{true | false}
```

Default: `true`

When set to `false`, does not consider time borrowing during timing analysis. Time borrowing is the amount of time borrowed by a previous logic.

To set this global variable, use the set_global command.

Note: The `timing_use_latch_time_borrow` global should not be used with the `setAnalysisMode` command. Use this global only when the `setAnalysisMode` command is not available. When the `setAnalysisMode` command is available, use the `-timeBorrowing` parameter to control time borrowing consideration.

timing_use_verilog_for_model_netlist

```
timing_use_verilog_for_model_netlist
{true | false}
```

Default: `false`

Allows you to choose whether to write the scope and boundary model netlists in Verilog format instead of binary.

When set to `true`, the software will use Verilog format for saving model nets.

To set this global variable, use the `set` command.

timing_waveform_aware_pulse_width_checks_high_voltage_level

`timing_waveform_aware_pulse_width_checks_high_voltage_level` *double*

Default: 0.95

Range: `{}`

Specifies the high voltage level to be used for waveform-aware pulse width checks.

To set this global variable, use the `set` command.

timing_waveform_aware_pulse_width_checks_low_voltage_level

`timing_waveform_aware_pulse_width_checks_low_voltage_level` *double*

Default: `0.05`

Range: `{}`

Specifies the low voltage level to be used for waveform-aware pulse width checks.

To set this global variable, use the `set` command.

timing_write_sdf_no_escape_backslash_control

```
timing_write_ sdf_no_escape_backslash_control
{true | false}
```

Default: `false`

Controls how the `write_sdf` command handles escape characters when the `-no_escape` parameter is specified.

For example, consider the following input Verilog expression:

\a/b\/c\/d

When the global is set to `true`, the `write_sdf` command removes the first escape character and outputs `a/b\/c\/d` in the SDF file.

When set to `false`, the command removes all the escape characters and outputs `a/b/c/d`.

To set this global variable, use the set_global command.

write_global_slack_worst_trigger_path_on_clocks

```
write_global_slack_worst_trigger_path_on_clocks
{true | false}
```

Default: `false`

Specifies whether the `write_global_slack_report` command reports worst slack on the register clock pins or not. When set to `true`, the `write_global_slack_report` command reports worst slack due to triggering paths.

A CLK pin might drive a Q pin of the same register, and have a setup or hold check arc with respect to the D pin. In the default setting, when the global is set to false, the `write_global_slack_report` command reports no slacks along the clock paths. When the global is set to `true`, the command will report the worst slack on register clock pins due to the related trigger paths and not due to the related timing checks.

To set this global variable, use the `set_global` command.

43

Timing Modeling Commands

- compare_model_timing
- do_extract_model
- merge_model_timing
- write_model_timing

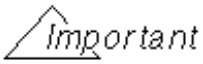
compare_model_timing

```
compare_model_timing
-help
-ref string
-compare string
-outFile string
[-ignore_view]
[-percent_tolerance float ]
[-percent_cap_tolerance float ]
[-absolute_cap_tolerance float ]
[-percent_tran_tolerance float ]
[-absolute_tran_tolerance float]
[-absolute_tolerance float ]
```

Compares two reports that contain interface timing characteristics generated by the write_model_timing command.

This command compares the interface timing report generated by using the write_model_timing command on the original netlist with the timing report generated by using the write_model_timing command on the model extracted by the do_extract_model command for the same design.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `compare_model_timing` parameter. For a detailed description of the command and all of its parameters, use the man command: `man compare_model_timing`.
`-absolute_cap_tolerance` *float*	Specifies the tolerance value for capacitance (as an absolute value) while comparing two report files. This parameter specifies the permissible difference, as an absolute value, between the capacitance values of two report files. If the difference in the value of capacitance exceeds the absolute tolerance value specified, the comparison status is set to `FAIL`. **Note:** When this parameter is not specified, the software uses the absolute value specified with the `-absolute_tolerance` parameter. If the `-absolute_tolerance` parameter is not specified, the value defaults to `0`.  When the `-percent_cap_tolerance` and `-absolute_cap_tolerance` parameters are used together, the comparison status is set to `FAIL`only when the difference in the value of capacitance exceeds the specified absolute as well as percentage tolerance values.
`-absolute_tolerance` *float*	Specifies the absolute tolerance value. The tolerance value is used when there is a difference between the timing characteristics of the reference file (`-ref`) and the file that is being compared (`-compare`) to determine the `PASS` or `FAIL` status of the comparison. If the difference in the value of a timing characteristic exceeds the absolute tolerance value, the comparison status is set to `FAIL`. **Note:** When used with the `-percent_tolerance` option, the difference in the value of a timing characteristic should exceed both absolute and percentage tolerance values for the comparison result status of an arc to be `FAIL`. *Default*: `0`

`-absolute_tran_tolerance` *float*	Specifies the name of the report file that contains the interface timing characteristics extracted from the model created using the `do_extract_model` command. The timing characteristics of this file are compared with the timing characteristics of the reference file that you specify with the `-ref` option.
`-compare` *string*	Specifies the name of the report file that contains the interface timing characteristics extracted from the model created using the `do_extract_model` command. The timing characteristics of this file are compared with the timing characteristics of the reference file that you specify with the `-ref` option.
`-ignore_view`	Compares non-MMMC design `write_model_timing` output report with the MMMC design output report (of `write_model_timing` command). When you specify this parameter, the software ignores the view information while comparing the reports. So you do not require to create dummy views for timing model validation flow.
`-outFile` *string*	Specifies the name of the output file in which the comparison results will be generated. This file contains the `PASS` or `FAIL` status for the timing data comparison.

`-percent_cap_tolerance` *float*	Specifies the tolerance value for capacitance (as a percentage) while comparing two report files. This parameter specifies the permissible difference, in percentage, between the capacitance values of two report files. If the percentage difference in the value of capacitance exceeds the percentage tolerance value specified, the comparison status is set to `FAIL`. **Note:** When this parameter is not specified, the software uses the percentage value specified with the `-percent_tolerance` parameter. If the `-percent_tolerance` parameter is not specified, the value defaults to `0`. When the `-percent_cap_tolerance` and `-absolute_cap_tolerance` parameters are used together, the comparison status is set to `FAIL` only when the difference in the value of capacitance exceeds the specified absolute as well as percentage tolerance values.
`-percent_tolerance` *float*	Specifies the percentage tolerance value for comparing the timing data of two report files. The percentage tolerance value is used when there is a difference between the timing characteristics of the reference file (`-ref`) and the file that is being compared (`-compare`) to determine the `PASS` or `FAIL` status of the comparison. If the percentage difference in the value of a timing characteristic exceeds the percentage tolerance value, the comparison status is set to `FAIL`. **Note:** When used with the `-absolute_tolerance` option, the difference in the value of a timing characteristic should exceed both percentage and absolute tolerance values for the comparison result status of an arc to be `FAIL`. *Default*: `0`

`-percent_tran_tolerance` *`float`*	Specifies the tolerance value for transition time at ports (as a percentage) while comparing two report files.This parameter specifies the permissible difference, in percentage, between the transition time at ports of two report files. If the percentage difference in the transition time exceeds the percentage tolerance value specified, the comparison status is set to `FAIL`. **Note:** When this parameter is not specified, the software uses the percentage value specified with the `-percent_tolerance` parameter. If the `-percent_tolerance` parameter is not specified, the value defaults to `0`. When the `-percent_tran_tolerance` and `-absolute_tran_tolerance` parameters are used together, the comparison status is set to `FAIL` only when the difference in the value of transition exceeds the specified absolute as well as percentage tolerance values.
`-ref` *`string`*	Specifies the name of the reference file that is used for comparison. This is the report file that was generated by using the `write_timing_model` command on the original design. The file specified with the `-compare` option is compared with this file.

Example

- The following command compares the timing characteristics of the `sample_compare.rpt` file with the `sample_reference.rpt` file and generates the comparison results in the `sample_output.rpt` file:

```
compare_model_timing -ref sample_reference.rpt -compare sample_compare.rpt
-outFile sample_output.rpt

##########################################################################

# compare_model_timing report
```

```
# Design : netlist = topcell =

# Version : 07.10-dev.indfe_cm

# Analysis Mode : setup

# Date : Wed Mar 14 15:56:59 IST 2007

#################################################################################

#################################################################################

#Section 1: Worst-case Arc-Delay

#Tolerance Percent: 0.0%

#Tolerance Absolute: 0.0

#NOTE: The various columns list values in the form 'X[Y]', where:

# X = value that is compared.

# Y = reference value.



#NOTE2: If a comparison fails (ie. status = FAIL), then the clauses

# (ie. 'X[Y]' pairs) that caused the failure are identified with

# a '*' in front of them.

#From   To    Arc-Type   Transition   Arc-Delay   abdDiff   %Diff  Status
```

```
################################################################################

in      CLK3   setup      fall/rise   0.134[0.134]   0.000   0.000%  PASS

in     CLK3   setup       rise/rise    -0.005[-0.005]   0.000   -0.000%  PASS



################################################################################

#Section 2: Pin-capacitances

#Tolerance: 0.0%

#NOTE: The various columns list values in the form 'X[Y]', where:

# X = value that is compared.

# Y = reference value.



#NOTE2: If a comparison fails (ie. status = FAIL), then the clauses

# (ie. 'X[Y]' pairs) that caused the failure are identified with

# a '*' in front of them.

#

#Pin C(total)[rise] C(total)[fall] Status

################################################################################

clk 0.002[0.002] 0.002[0.002] PASS
```

```
in 0.002[0.002] 0.002[0.002] PASS

out 0.000[0.000] 0.000[0.000] PASS


################################################################################

#Section 3: Transition-times

#Tolerance: 0.0%

#NOTE: The various columns list values in the form 'X[Y]', where:

# X = value that is compared.

# Y = reference value.

#

#NOTE2: If a comparison fails (ie. status = FAIL), then the clauses

# (ie. 'X[Y]' pairs) that caused the failure are identified with

# a '*' in front of them.

#

#Pin Rise Fall Status

################################################################################

clk 0.000[0.000] 0.000[0.000] PASS

in 0.000[0.000] 0.000[0.000] PASS
```

```
out 0.056[0.056] 0.051[0.051] PASS

#########################################################################

#Section 4: Design Rules

#Tolerance: 0.0%

#NOTE: The various columns list values in the form 'X[Y]', where:

# X = value that is compared.

# Y = reference value.



#NOTE2: If a comparison fails (ie. status = FAIL), then the clauses

# (ie. 'X[Y]' pairs) that caused the failure are identified with

# a '*' in front of them.

#

#Pin    MaxCap    MinCap    MaxTrans    MaxFanout    FanoutLoad    Status

#########################################################################

clk     NA[NA]     NA[NA]    NA[NA]      NA[NA]       NA[NA]        PASS

in      NA[NA]     NA[NA]    NA[NA]      NA[NA]       NA[NA]        PASS

out     NA[NA]     NA[NA]    NA[NA]      NA[NA]       NA[NA]        PASS
```

```
################################################################################

# SUMMARY

#

# Check-Type Passed Failed N/A Total

################################################################################

Worst-Slacks 2 0 0 2

Capacitances 3 0 0 3

Trans-Times 3 0 0 3

Design-Rules 3 0 0 3
```

Related Information

- merge_model_timing
- write_model_timing

do_extract_model

```
do_extract_model
[-help]
model_filename
[-abs_tol_val float]
[-abs_tol_val_optimistic float]
[-abs_tol_val_pessimistic float]
[-assertions constraint filename]
[-cell_name cell_name]
[-clock_slews {clk_slew1 clk_slew2...}]
[-format string]
[-gain integer]
[-include_aocv_weights]
[-input_slews {input_slew1 input_slew2...}]
[-lib_name lib_name]
[-max_num_loads value]
[-max_num_slews value]
[-output_loads {output_load1 output_load2...}]
[-per_tol_val float]
[-per_tol_val_optimistic float]
[-per_tol_val_pessimistic float]
[-pg]
[-precision integer]
[-resolution float]
[-tolerance float]
[-validate string]
[-verilog_shell_file filename]
[-verilog_shell_module top_module_name]
[-view string]
[-check]
```

Builds a Liberty (`.lib`) format model for the top cell, which is the timing model equivalent of the original design. The model is extracted for the given analysis modes, PVT conditions, and parasitics. If any one of these parameters change, then you must extract the model again. The model is extracted for a range of input slews and load capacitances. If you do not specify these ranges, the software automatically computes the default values. The extracted model defines all of the top-level ports of the design as pins. Additionally, all other design pins that are targets of `create_clock` or `create_generated_clock` are retained as internal pins.

For model extraction, your design must be properly constrained. For example, no clock or data conflicts can be present. Use the `check_timing` command and correct all errors prior to model extraction.

Parameters

`-help`	Prints out the command usage.
`-abs_tol_val` *float*	Specifies the absolute tolerance value to be used during the validation of the extracted timing model. For more details, refer to the `compare_model_timing -absolute_tolerance` command option. This parameter can be used only when the `-validate` option is specified. The integer value specified with this option is read as ns/ps depending upon the time unit. *Default*: 3ps
`-abs_tol_val_optimistic` *float*	Specifies the absolute tolerance limit for flagging an optimistic failure. An optimistic path violating the absolute as well as percentage tolerance limit will be flagged as FAIL in the `compare_model_timing` command output report. You can use the this parameter when the `-validate` is used with model extraction. *Default*: 15ps
`-abs_tol_val_pessimistic` *float*	Specifies the absolute tolerance limit for flagging a pessimistic failure. A pessimistic path violating the absolute as well as percentage tolerance limit will be flagged as FAIL in the `check_timing_model` command output report. You can use the this parameter when the `-validate` is used with model extraction. *Default*: 15ps

`-assertions` *`constraint filename`*	Dumps the constraints for a model, which is used in the validation flow. *Default*: `model.asrt`
`-cell_name` *`cell_name`*	Specifies the cell name used in the timing model. *Default:* The name of the top cell to be extracted.
`-check`	Checks the extracted timing model for any possible liberty related issue. When specified, the command displays detailed error and warning messages, and their summary.
`-clock_slews {`*`clk_slew1 clk_slew2...`*`}`	Specifies a range of slew values used at all clock pins. The list of values must be enclosed by curly braces. If this parameter is not specified, the software automatically computes the appropriate values.
`-format` *`string`*	Specifies the format for the extracted model in tlf, dotlib, or syntech.
`-gain` *`integer`*	Represents an increase in the number of delay arcs if a pin is removed. This option reduces the extracted model size in scenarios where removing a pin can result in increased model size. The gain at an internal pin can be defined as: `(number of incoming delay arcs * number of outgoing delay arcs) - (number of incoming delay arcs + number of outgoing delay arcs` If an internal pin has a gain greater than the specified number, then that pin needs to be preserved in order to reduce the extracted model size. *Default*: `0`

`-include_aocv_weights`	Writes the stage weight on individual arcs. Before using this option, it is mandatory that the AOCV libraries are read in and AOCV analysis is enabled (by using `setAnalysisMode -aocv true`). The weights written in ETM are not meant to derate the model. Model itself will be containing the derated delays. The stage weight extracted on arcs will be used to calculate the stage count of chip level paths going across the ETM. The stage weights in the extracted model are dumped as user defined attributes. This will require defining two attributes at arc level.
	• `define ("aocv_weight","timing","float");`
	• `define ("clock_aocv_weight","timing","float");`
	The first attribute will be used for combinational and trigger arc and will be dumped at arc level as below.
	`timing () {`
	`timing_type : combinational;`
	`timing_sense : positive_unate;`
	`aocv_weight : 4;`
	`.`
	`}`
	`The check arcs will be dumped with two separate stage weights for data and clock.`
	`timing () {`
	`timing_type : setup_rising;`
	`aocv_weight : 4;`
	`clock_aocv_weight : 3;`
	`.`
	`}`

`-input_slews {`*`input_slew1 input_slew2 ...`*`}`	Specifies a range of slew values used at all input pins. Enclose the list of values with curly braces. If this parameter is not specified, the software automatically computes the appropriate values.

`-lib_name` *lib_name*	Specifies the library name used in the timing model. Write extracted timing models in the liberty format by specifying the `.lib` file suffix. *Default:* The name of the top cell to be extracted.
`-max_num_loads` *value*	Specifies the maximum number of output load values used for delay arc characterization. A low value of 1 or 2 improves runtime and memory usage but may significantly degrade model accuracy. Use this parameter only if the model size or model extraction time is critical. *Default:* `64`
`-max_num_slews` *value*	Specifies the maximum number of input slews or clock slews used for delay and check arc characterization. Specifying a low value of 1 or 2 improves runtime and memory but may significantly degrade model accuracy. Using the `-max_num_slews` parameter has a larger impact on runtime and memory than the `-max_num_loads` parameter. Use this parameter only if the model size or the model extraction time is critical. *Default:* `64` The `max_num_slews` and `max_num_loads` parameters specify only the maximum limits on how many slew and load points are chosen. The software scans the gates connected to the input and output ports to calculate the number of points.
model_filename	Specifies the name of the output file. Avoid problems with rebind by using the `-lib_name` and `-cell_name` parameters instead of using the default names.
`-output_loads {`*output_load1 output_load2* `...}`	Specifies a range of load values used at all output pins. Enclose the list of values by curly braces. If this parameter is not specified, the software automatically computes the appropriate values.

`-per_tol_val` *float*	Specifies the percentage tolerance value to be used during the validation of extracted ETM. This option can be used only when `-validate` option is also specified. For more details refer to `compare_model_timing -percent_tolerance`. Default: 3%
`-per_tol_val_optimistic` *float*	Specifies the percentage tolerance limit for flagging an optimistic failure. An optimistic path violating percentage as well as absolute tolerance limit will be flagged as FAIL in the `compare_model_timing` command output report. You can use the this parameter when the `-validate` is used with model extraction. *Default*: 3%
`-per_tol_val_pessimistic` *float*	Specifies the percentage tolerance limit for flagging a pessimistic failure. A pessimistic path violating percentage as well as absolute tolerance limit will be flagged as FAIL in the `compare_model_timing` command output report. You can use the this parameter when the `-validate` is used with model extraction. *Default*: 3%
`-pg`	Allows power or ground related pin information to be extracted inside the extracted model.
`-precision` *integer*	Specifies the floating point precision used for the timing model. For example, the floating point precision for 3.0089 can be changed as follows:
	Precision Data Value in .lib 4 3.0089 3 3.009 2 3.01 1 3.0 *Default:* `4`

`-resolution` *float*	Specifies the magnitude of delay difference that can be ignored by the extractor. *Default:* `0.001` *Type*: Float
`-tolerance` *float*	Specifies the accuracy level of extractor in percent. For example, 5.0 represents a plus or a minus 5.0 percent allowable error in the extracted model. The higher the tolerance, the faster the extraction time. *Default:* `1.0` *Type:* Float
`-validate` *string*	Validates the timing model generated for the corresponding view and displays the number of failures corresponding to setup and hold checks
`-view` *string*	Specifies the current active view.
`-verilog_shell_file` *filename*	Generates a Verilog shell file that instantiates the timing model. This parameter is useful for validating the model. Use the `-verilog_shell_module` parameter to specify the name of the top module in this Verilog file.
`-verilog_shell_module` *top_module_name*	Specifies the top module name of the Verilog shell file. *Default:* `test_shell`

Examples

- The following command builds a `.lib` model (`block.lib`) for the block:

  ```
  do_extract_model block.lib
  ```

- The following command creates the timing model for the `test_model` cell belonging to the `test_lib` library, and writes the model into the `test.lib` liberty file:

  ```
  do_extract_model -cell_name test_model -library_name test_lib test.lib
  ```

merge_model_timing

```
merge_model_timing
[-help]
[-keep_separate_arc]
-library_file string
-mode_group string
-modes string
[-outfile string]
[-pin_cap_tolerance float]
[-tolerance float]
```

Merges extracted timing models (ETM) for various analysis views into a single output.

The `merge_model_timing` command should be the last command in the run script. Also, this command should be run separately in a Tempus session when a design is not loaded.

When the DRV values are different in the libraries that are merged, then Tempus saves the worst value of the DRV with a warning message.

If the DRV values are not equal, the greater value is stored for the following:

- Minimum transition
- Minimum capacitance
- Minimum fanout

The smaller value is stored for the following, in case of unequal DRVs:

- Maximum transition
- Maximum capacitance
- Maximum fanout
- Fanout load

Parameters

`-help`	Prints out the command usage.
`-keep_separate_arc`	Enables creating separate timing arcs with respective modes.
`-library_file` *string*	Specifies input library files that are to be merged.
`-modes` *string*	Specifies the mode names required for merging the libraries. The number of mode names should be equal to the number of input libraries.
`-mode_group` *string*	Specifies the name of the mode group.
`-outfile` *string*	Specifies the name of the merged library output file name. By default, the output file name is `merge.lib`.
`-pin_cap_tolerance` *float*	Specifies the pin capacitance tolerance for making tolerance comparison for library merging.
`-tolerance` *float*	Specifies the tolerance value for making tolerance comparison for library merging.

Examples

- The following command merges library files `model1.view1.lib` and `model1.view1.lib.ref` into a single output file:

```
merge_model_timing -library_file "model1.view1.lib model1.view1.lib.ref"
-modes "v1 v2" -mode_group v1_v2
```

Related Information

- See "Extracting Timing Models" in the *Innovus User Guide*.
- `write_model_timing`
- `compare_model_timing`

write_model_timing

```
write_model_timing
[-help]
[model_timing_report_file]
[-inputs string]
[-nworst integer]
[-outputs string]
[-slew_propagation {worst | path_based}]
[-type {arc | slack}]
[-verbose]
[-view string]
-setup string | -hold string
```

Writes the interface timing characteristics of the design in the specified timing model report file.

The report file contains the following sections:

- `Worst-Case Arc/Slack`: Contains details about the extracted arcs and the slack or delay across them depending on the argument specified with the `-type` option.
- `Transition Time`: Contains information about transition time at ports.
- `Capacitance`: Contains the capacitance values for the ports.
- `Design Rules`: Contains the design rules applied to the ports.

The command reports unconstrained arcs delays by default.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_model_timing` parameter. For a detailed description of the command and all of its parameters, use the man command: `man write_model_timing`.
`-hold string`	Writes the paths corresponding to early analysis.

`-inputs {`*`port_list`*`}`	Restricts the `write_model_timing` report to the input ports specified with this parameter. As a result, the software considers the timing paths related to the specified input ports only, while generating the report. If this parameter is not specified, all the input ports in the design are considered while generating the report.
`model_timing_report_file`	Specifies the name of the interface timing model report file that will be generated with the interface timing characteristics of the loaded design.
`-nworst` *`value`*	Specifies the number of paths to be enumerated for each end point. **Note:** The extracted model will contain greater number of setup/hold paths per end point because it will have lesser end points compared to the original netlist. Therefore, it is recommended that you use a greater `-nworst` value such as `40` while writing a timing report for the extracted model to derive better correlation.
`-outputs {`*`port_list`*`}`	Restricts the `write_model_timing` report to the output ports specified with this parameter. As a result, the software considers the timing paths related to the specified output ports only, while generating the report. If this parameter is not specified, all the output ports in the design are considered while generating the report.
`-slew_propagation { worst | path_based }`	Sets the slew propagation mode to use for dumping the timing characteristics of the design.
`-setup` *`string`*	Writes the paths corresponding to late analysis.
`-type { arc | slack }`	Determines whether to report arc delay values (`arc`) or the worst-slack values (`slack`).
`-view` *`view_name`*	Specifies a view name. The report can be generated for a specific view name.
`-verbose`	Prints the command progress details.

Example

- The following command extracts the interface timing characteristics of the currently loaded design and writes out the report file named `sample.rpt` with the arc delay information:

```
write_model_timing -type arc sample.rpt
```

```
Output:

#################################################################
# write_model_timing report
# Design : netlist = .//case1.v topcell = set_propagated_clock_tc2
# Version : 07.10-dev.indfe_cm
# Analysis Mode : setup
# Date : Wed Mar 14 15:29:57 IST 2007
# Command : write_model_timing -type arc sample.rpt
#################################################################
#################################################################
# Section 1 : Worst Case Arc
# Analysis Mode : setup
# Description : worst case arcs
#
# From To Arc Type Transition Arc-Delays
#########################################################
in CLK3 setup fall/rise 0.134
in CLK3 setup rise/rise -0.005
#########################################################
# Section 2 : Transition time
# Analysis Mode : setup
# Description : Actual transition times on the ports
#
# Port RiseTran FallTran
#########################################################
in 0.000 0.000
clk 0.000 0.000
out 0.056 0.051
#########################################################
# Section 3 : Capacitance
# Analysis Mode : setup
# Description : Total Cap on the ports
#
# Port CTotal(Rise) CTotal(Fall)
#########################################
in 0.002 0.002
```

```
clk 0.002 0.002
out 0.000 0.000
#########################################
# Section 4 : Design Rules
# Analysis Mode : setup
# Description : Design Rules on the ports
#
# Port MaxCap MinCap MaxTrans MaxFanout Fanout Limit
#################################################
in NA NA NA NA NA
clk NA NA NA NA NA
out NA NA NA NA NA
```

- In MMMC mode, the `write_model_timing` command displays the following report output:

```
##############################################
# write_model_timing report
# Design : netlist =
./../../../tc/tempus/testcases/tempus_ilm/DATA/mmmc/dtmf_chip/dtmf_chip.Route.opt.
ILM.enc.dat/dtmf_chip.v topcell = dtmf_chip

# Analysis Mode : setup
# Date : Mon Sep 21 22:07:40 PDT 2009
# Command : write_model_timing -type arc sample.rpt
#############################################################################
#############################################################################
# Section 1 : Worst Case Arcs
# Analysis Mode : setup
# Description : worst case arcs
#
# From   To           Arc Type      Transition   Arc-Delays    View
#############################################################################
vclk2    tdigit[0]    max_sequential rise/fall   2.676   mission+worst-rcMax
vclk2    tdigit[0]    max_sequential rise/rise   2.831   mission+worst-rcMax
vclk2    tdigit[1]    max_sequential rise/fall   2.413   mission+worst-rcMax
vclk2    tdigit_flag  max_sequential rise/rise   2.58    mission+worst-rcMax
vclk2    tdigit[0]    max_sequential rise/fall   2.676   test+worst-rcMax
```

```
vclk2    tdigit[0]    max_sequential rise/rise   2.831   test+worst-rcMax
vclk2    tdigit[1]    max_sequential rise/fall   2.413   test+worst-rcMax
#####################################################
# Section 2 : Transition time
# Analysis Mode : setup
# Description : Actual transition times on the ports
# View : default
#
# Port                   RiseTran   FallTran
#####################################################
reset                     0.120     0.120
int                       0.120     0.120
port_pad_data_in[15]      0.120     0.120
port_pad_data_in[0]       0.120     0.120
scan_en                   0.120     0.120
scan_clk                  0.120     0.120
scan_in_1                 0.120     0.120
scan_in_2                 0.120     0.120
test_mode                 0.120     0.120
spi_data                  0.120     0.120
spi_fs                    0.120     0.120
refclk                    0.120     0.120
vcom                      1.025     1.138
#####################################################
# Section 3 : Capacitance
# Analysis Mode : setup
# Description : Total Cap on the ports
# View : default
#
# Port               CTotal(Rise) CTotal(Fall)
#####################################################
reset                   3.078     3.078
port_pad_data_in[0]     3.078     3.078
scan_en                 3.078     3.078
```

```
scan_clk                3.078     3.078
scan_in_1               3.078     3.078
vcop                    0.007     0.007
#################################################################################
# Section 4 : Design Rules
# Analysis Mode : setup
# Description : Design Rules on the ports
# View : default
#
# Port                MaxCap  MinCap  MaxTrans  MaxFanout  Fanout  Limit
#################################################################################
reset                   NA    NA      2.700     15.000     NA
int                     NA    NA      2.700     15.000     NA
port_pad_data_in[15]    NA    NA      2.700     15.000     NA
port_pad_data_in[14]    NA    NA      2.700     15.000     NA
port_pad_data_in[13]    NA    NA      2.700     15.000     NA
port_pad_data_in[12]    NA    NA      2.700     15.000     NA
```

Related Information

- merge_model_timing
- compare_model_timing

44

Timing Optimization Commands

- addRepeaterByRule
- addTieHiLo
- checkFootPrint
- createBasicPathGroups
- createTBOptFile
- deleteBufferTree
- deleteTieHiLo
- dumpMultiBitFlopMappingFile
- ecoClone
- ecoCloneFlop
- ecoMergeCombinationalCell
- ecoSplitCombinationalCell
- ecoSplitComplexFlop
- ecoSplitFlop
- findLefEquivalentCells
- getLatencyFile
- getOptMode
- getSchedulingFile
- getSignoffOptMode
- getTieHiLoMode
- getUsefulSkewMode
- loadFootPrint
- optDesign
- optPower
- optVirtual
- reclaimArea
- reportAlwaysOnBuffer
- reportCapViolation
- reportCritInstance
- reportCritNet
- reportCritTerm
- reportFanoutViolation

- reportFootPrint
- reportIgnoredNets
- reportLengthViolation
- reportMultibit
- reportPathGroupOptions
- reportRouteTypeConstraints
- reportTranViolation
- reportVtInstCount
- resetPathGroupOptions
- route_opt_design
- setBufFootPrint
- setDelayFootPrint
- setDontUse
- setInvFootPrint
- setLatencyFile
- setMaxCapPerFreq
- setMaxCapPerFreqTran
- setMaxTranPerFreq
- setOptMode
- setPathGroupOptions
- setSchedulingFile
- setSignoffOptMode
- setTieHiLoMode
- setUsefulSkewMode
- signoffOptDesign
- skewClock

addRepeaterByRule

```
addRepeaterByRule
[-help]
[-copyNetAttribute]
[-excNet excludeNetFile]
[-netMapping fileName]
[-nets list_of_nets]
[-outDir directoryName]
[-preRoute | -postRoute | -alongRoute]
[-reportIgnoredNets ignoredNetFile]
[-rule fileName]
[-selNet selNetFile]
[-selected]
[-template]
```

Inserts buffers and/or inverters to nets, as defined in a rule file. By default, inserts available buffers and/or inverters with the smallest total area cost while satisfying the rules. As compared to DRV fixing, `addRepeaterByRule` does not consider transition violations (maxTran) or any timing information. Instead, it constrains max total net length, max radius length, max capacitance, max fanout count, and so on. Also, `addRepeaterByRule` does not legalize newly added buffers. You need to legalize the added buffers to a legal location.

The command provides an efficient, buffering method for top-level designs.

Note: Max radius length refers to the sum of wire lengths from the driver to the farthest sink and max total net length refers to the total (sum) wire lengths throughout the target net.

The command has the following features:

- Supports multiple buffer/inverter/AO cells simultaneously
- Supports multiple types of constrains simultaneously
- Provides complete MSV compatibility solution
- It is multithreaded

The rule file may contain two categories of constraints, where each category further breaks down to several constraint types. All constraints share the same syntax of ‘command value’.

1. Individual repeaters’ rule: constrains each repeater’s driving capability.
   - max net total length (um): SetInverterMaxNetLength, SetBufferMaxNetLength, SetBufferMinLayerMaxNetLength, SetInverterMinLayerMaxNetLength
   - max radius length (um): SetInverterMaxRadiusLength, SetBufferMaxRadiusLength
   - max capacitance (pF): SetInverterMaxCap, SetBufferMaxCap
   - max fanout count: SetInverterMaxFanout, SetBufferMaxFanout
   - max fanout at sink count: SetMaxFanoutAtSink
   - max wirelength (um) of the net driving the sink term: SetMaxNetLengthAtSink
2. Global default rule: constrains all other instances beyond those newly inserted repeater instances.
   - default max net total length (um): SetDefaultMaxNetLength, SetDefaultMinLayerMaxNetLength
   - default max radius length(um): SetDefaultMaxRadiusLength
   - default max cap length (pF): SetDefaultMaxCap
   - default max fanout count: SetDefaultMaxFanout
   - default max sink radius length (um): SetDistLastRepToSink
   - default max wirelength (um) of the net driving the sink term: SetMaxNetLengthAtSink

Those constraints can be used individually or combined. However, at least one valid repeater must be specified in the rule file. If the global default rule for certain constraint type is omitted from the rule file, the command automatically picks the tightest constraint from the repeaters defined in the rule file under the ***same*** type of constraint as the default constraint. A sample rule file is shown in the "Example" section.

The following three layer-based rules are provided to support specifying net length constraints independently for each user specified layer:

- minimum layer for max net total length (um): SetBufferMinLayerMaxNetLength and SetInverterMinLayerMaxNetLength
- default minimum layer for max net total length (um): SetDefaultMinLayerMaxNetLength

Note: You must specify the bottom_preferred_routing_layer/top_preferred_routing_layer attribute for the nets that need layer-based buffering. If no attribute is present, the net will be routed with the default rule on default layers.

Use Model

When max layer constraint is specified:

```
SetBufferMaxNetLength BUF_X4M_A9TR 300.0
```

In the above scenario, ARR will buffer the net with `BUF_X4M_A9TR` at length intervals of `300um`.

When minimum layer support is also specified along with max layer constraint, then the following two scenarios are possible:

Example 1:

```
SetBufferMaxNetLength          BUF_X4M_A9TR     300.0
SetBufferMinLayerMaxNetLength  BUF_X4M_A9TR 6   500.0
```

In the above example, layers 2-5 will have a `300um` max length, and becasue there is a minimum layer constraint on the net of layer 6 or higher, the length constraint will be increased from `300` to `500um`.

Example 2:

When multiple minimum layer constraints are specified:

```
SetBufferMaxNetLength            BUF_X4M_A9TR     300.0
SetBufferMinLayerMaxNetLength    BUF_X4M_A9TR 6   500.0
SetBufferMinLayerMaxNetLength    BUF_X4M_A9TR 11  800.0
```

In the above example, layers 2-5 will have a `300um` max length; layers 6-9 will have a `500um` max length; and 11+ will have an `800um` max length.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `addRepeaterByRule` parameter. For a detailed description of the command and all of its parameters, use `man addRepeaterByRule`.
`-alongRoute`	Allows repeater insertion along the route. The database should be global-routed with preRoute RC extraction before calling this option. **Note**: Run `refinePlace` and `ecoRoute` if you specify this parameter.
`-copyNetAttribute`	
	When this parameter is specified, the net attributes set on the original nets are copied to the nets inserted by this command.
`-excNet` *`fileName`*	Specifies a file that contains the names of hierarchical nets to exclude from rule violation repair. If a net appears in the files specified by both `-excNet` and `-selNet`, it is excluded from rule violation repair.
`-netMapping` *`fileName`*	

	When specified, generates a file that maps the original net names to the nets inserted by this command. An example of the file format is provided below. In this example, `netA` is split into two nets; `FE_new_net1_netA` and `FE_new_net2_netA`.
	`netA FE_new_net1_netA`
	`netA FE_new_net2_netA`
	If, in the next iteration, `FE_new_net1_netA` is split again, the file would contain the following:
	`FE_new_net1_netA FE_new_net1_FE_new_net1_netA`
	`FE_new_net1_netA FE_new_net2_FE_new_net1_netA`

`-nets` *`list_of_nets`*	Specifies the list of nets to be considered for adding buffers and/or inverters. **Note**: The `-nets`, `-selNet`, and `-selected` parameters are mutually exclusive.

`-outDir` *directoryName*	Specifies the directory where the software writes detailed failure reason report that is generated when this command runs. By default, the failure reason report is dumped in the `timingReports/ARRFailureReason/<design>` directory. For example, `timingReports/ARRFailureReason/<design>.arr.preRoute.rpt.gz` *Default*: `timingReports`
`-postRoute`	Allows repeater insertion along the route. The database should be detailed-routed with detailed RC extraction before calling this option. Preserves routing so that changes are minimized. **Note**: Run `refinePlace` and `ecoRoute` if you specify this parameter.
`-preRoute`	Allows repeater insertion with different routing topologies. The database should be global-routed with preRoute RC extraction before calling this option. When this parameter is specified, the software explores different typologies to optimally insert repeaters.
`-reportIgnoredNets` *fileName*	
	Reports the nets where no buffers were inserted and the reason the nets were ignored.
`-rule` *fileName*	Specifies a file you create that should list the buffers and inverters and the length and capacitance rules for inserting each of them. Only buffers and inverters included in the rule file are inserted.
`-selected`	When this parameter is specified, the `addRepeaterByRule` command operates only on nets that are selected either by the `selectNet` command or in the GUI. **Note**: The `-nets`, `-selNet`, and `-selected` parameters are mutually exclusive.
`-selNet` *fileName*	Specifies a file that contains the names of hierarchical nets considered by this command. If a net appears in the files specified by both `-excNet` and `-selNet` , it is excluded from rule violation repair. **Note**: The `-nets`, `-selNet`, and `-selected` parameters are mutually exclusive.
`-template`	Writes a template rule file. **Note**: When the `-template` parameter is specified, the software does not honor any other parameters for this command.

Examples

- The following command explores different typologies to optimally insert repeaters:
  ```
  addRepeaterByRule -rule rule1 -preRoute
  ```
- In a detailed routed database with RC extraction, the following command inserts repeaters along the route and preserves routing, so that changes are minimized:
  ```
  addRepeaterByRule -rule rule1 -postRoute
  ```
- The following command inserts repeaters on those nets specified in the file, `net_file1`:
  ```
  addRepeaterByRule -rule rule1 -selNet net_file1
  ```
- The following is an example of a template rule file:
  ```
  ##############################################################################
  ################### addRepeaterByRule template rule file ####################
  ##############################################################################
  # constraintType            cellName              constraint value
  #                                                Length(um)/cap(pF)/count
  ##############################################################################
  ```

```
########### max net total Lengths for individual repeaters
  SetInverterMaxNetLength    HS65_LH_IVX7         150.0
  SetBufferMaxNetLength      HS65_LS_BFX7         300.0

########### max radius Lengths for individual repeaters
  SetInverterMaxRadiusLength HS65_LH_IVX7         100.0
  SetBufferMaxRadiusLength   HS65_LS_BFX7         240.0

########### max capacitance for individual repeaters
  SetInverterMaxCap          HS65_LH_IVX7         10.0
  SetBufferMaxCap            HS65_LS_BFX7         20.0

########### max fanout for individual repeaters
  SetInverterMaxFanout       HS65_LH_IVX7         20
  SetBufferMaxFanout         HS65_LS_BFX7         30

########### default max net total Lengths
  SetDefaultMaxNetLength                          100.0

########### default max radius Lengths
  SetDefaultMaxRadiusLength                       200.0

########### default max capacitance
  SetDefaultMaxCap                                15.0

########### default sink max radius length
  SetDistLastRepToSink                            50.0

########### max net lengths from sink
  SetMaxNetLengthAtSink                           100.0

########### default max fanout
  SetDefaultMaxFanout                             20


#############################################################################
#   'DrivingStrength' constrains both max radius Length and max cap.
#############################################################################
#   constraintType              cellName            Length(um)      cap(pF)
#############################################################################

    SetInverterDrivingStrength  HS65_LH_IVX7        100.0           10.0
    SetBufferDrivingStrength    HS65_LS_BFX7        240.0           20.0
    SetDefaultDrivingStrength                       200.0           15.0
```

Related Information

- ecoRoute
- refinePlace
- The following topics in the *Innovus User Guide*
  - Optimizing Timing Using a Rule File
  - Viewing the Added Buffers, Instances, and Nets after Optimization
    - Default Naming Conventions

addTieHiLo

```
addTieHiLo
[-help]
[-cell cellNames]
[-createHierPort {true | false}]
[-keepExisting {true | false}]
[-matchingPDs {true | false}]
[-postMask {true | false}]
[-powerDomain powerDomainName]
[-prefix prefixName]
[-reportHierPort {true | false}]
[-cellPin cellName:cellPin | -instancePin fileName | -excludePin fileName]
```

Adds instances of specified tie-off cells to the logical hierarchy of the design and connects the tie-off pins of netlist instances to the tie-off pins of the added instances. Thus, these instances provide the connectivity between the tie-high and tie-low pins of the netlist instances to power and ground. Use this command after placing the standard cells in the flow.

Note: If you omit the `-powerdomain` option, the command will insert tie-hilo on default power domains. To add tie-off cells to all domains, you need to use the `-matchingPDs` option.

Note: If you do not specify parameters, this command uses the global parameters specified by the `setTieHiLoMode` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `addTieHiLo` parameter. For a detailed description of the command and all of its parameters, use the man command: `man addTieHiLo`.
`-cell` *cellNames*	Specifies the tie cells to add. You can only specify a maximum of two tie-cells, where one cell must be a tie-high driver, and the other a tie-low driver. The two tie-cell names must be surrounded by quotation marks. This parameter overrides cell name(s) specified by the `setTieHiLoMode` command. *Default*: `""` (empty string)
`-cellPin` *cellName:cellPin*	Specifies the cell pins that are to be tied. The cell format could be "cellName1:cellPin cellName2:cellPin cellName3:cellPin". Only the cell pins that have tie 0/1 connections and are specified in `-cellPin` are changed to the tie cell. The left tie connections do not change. *Default*: `""`
`-createHierPort {true | false}`	Allows the added tie cells to connect to tie pins across hierarchical boundaries if other constraints, such as maximum fanout and maximum distance, allow it. *Default:* `false`
`-excludePin` *fileName*	Specifies the excludePin file name. The file format is "instanceName/instancePin". Each line in the file contains one instance pin. Only the instance pins that have tie 0/1 connections and are not defined in this file are changed to the tie cell. The left tie connections do not change. *Default*: `""`
`-instancePin` *fileName*	Specifies the instance pins file name. The file format is "instanceName/instancePin". Each line in the file contains one instance pin. Only the instance pins that have tie 0/1 connections and are defined in this file are changed to the tie cell. The left tie connections do not change. *Default*: `""`

`-keepExisting {true | false}`	When set to `true`, preserves the existing tie-offs, added in the previous run, in the design and adds new ones. *Default:* `false`
`-matchingPDs {true | false}`	Adds tie-off cells to all power domains that have library binding containing the specified tie-off cell(s). *Default:* `false`
`-postMask {true | false}`	Insert tie cells in post mask flow. Instructs the software to reuse existing tie cells to tie off a newly created spare instance in the design instead of adding or deleting tie cells. Searches for input pins that are tied to logical 1 or logical 0 and attempts to tie them off to tie high or tie low cells. *Default:* `false`
`-powerDomain` *powerDomainName*	Inserts tie-off cell to the named PD. *Default*: `""` **Note:** When this option is not specified, tool only inserts tie cells in the "default" power domain
`-prefix` *prefixName*	Specifies a prefix for the added tie-cell instances. *Default*: `""`
`-reportHierPort {true | false}`	When the value of `-createHierPort` is `true`, reports created port connections to a file named `tiehilo.rpt`. *Default:* `false`

Examples

- The following command adds tie-high and tie-low connectivity for designs with libraries that have cell named TIEOFF that provides both tie-high and tie-low connections. The instances added are prefixed with `tieOff`:

  ```
  addTieHiLo -cell TIEOFF -prefix tieOff
  ```

- The following command adds tie-high and tie-low connectivity for designs with libraries that have separate tie-high and tie-low cells named `TIEHI` and `TIELO` respectively:

  ```
  addTieHiLo -cell "TIEHI TIELO"
  ```

- The following commands add tie-off connectivity for MSV designs with two power domains named `PD1` and `PD2`. The last command is necessary for the part of the design that is not in either of the two specified power domains.

  ```
  addTieHiLo -cell "TIEHI TIELO" -powerDomain PD1
  addTieHiLo -cell "TIEHI TIELO" -powerDomain PD2
  addTieHiLo -cell "TIEHI TIELO"
  ```

- The following command add tie-off connectivity for MSV designs where all power domains whose library binding contains `TIEOFF` cell:

  ```
  addTieHiLo -cell TIEOFF -matchingPDs true
  ```

- The following command will connect all MUX/S and NOR/A pins that have the tie 0/1 connections to the tie cells. The left tie connections do not change.

  ```
  addTieHiLo -cellPin "MUX:S NOR:A"
  ```

Related Information

- `deleteTieHiLo`
- `getTieHiLoMode`
- `setTieHiLoMode`
- `verifyTieCell`
- The following topic in the *Innovus User Guide*
  - Adding Logical Tie-Off Cells

checkFootPrint

```
checkFootPrint
[-help]
```

Checks the timing library for missing footprint functions or timing information.

To view the above information in the software, use the man command: `man checkFootPrint`.

Related Information

- `loadFootPrint`
- `reportFootPrint`
- `setBufFootPrint`
- `setInvFootPrint`
- `setDelayFootPrint`
- The following topic in the *Innovus User Guide*
  - Using Cell Footprints in Timing Optimization

createBasicPathGroups

```
createBasicPathGroups
[-help]
[-expanded]
[-reset]
```

Explicitly creates standard path groups irrespective of the design stage. The following path groups are created:

- reg2reg (register-to-register)
- reg2cgate (register-to-clock-gate)
- default

Note:

- If a design does not have any reg2reg paths, or reg2cgate paths, this command will create the corresponding path group(s).
- If the specified path group, say `reg2reg`, already exists, the software will give the following warning:

  ```
  **WARN: (IMPOPT-3666): Path group reg2reg already exists. Skipping the creation of this path group.
  ```

Parameters

`-help`	Outputs a brief description that includes type and default information for each `createBasicPathGroups` parameter. For a detailed description of the command and all of its parameters, use the man command: `man createBasicPathGroups`.
`-expanded`	When this parameter is specified, the following path groups are created: • reg2reg • reg2cgate • in2reg • reg2out • in2out When this parameter is not specified, only the default path groups, reg2reg and reg2cgate, are created. **Note**: Timing optimization will consider these path groups in the same way as paths grouped using the `group_path` command. In addition, if a design does not have any reg2reg or reg2cgate paths, this command will not create the corresponding path groups.
`-reset`	Deletes all existing path groups. This is equivalent to running the `reset_path_group` command. **Note**: After deleting the existing path groups, new path groups, reg2reg and reg2cgate are created.

Examples

- The following command is used to create the standard path groups and assigns an effort level to each group:

  ```
  createBasicPathGroups
  ```

  The software returns the following information:

  ```
  Created reg2reg path group
  Effort level <high> specified for reg2reg path_group
  Created reg2cgate path group
  Effort level <high> specified for reg2cgate path_group
  ```

- The following commands are used to set the effort level for the reg2reg path group to high, and report the parameters of the path groups that are created:

  ```
  setPathGroupOptions reg2reg -effortLevel low
  reportPathGroupOptions
  ```

  The software prints the following report:

  ```
  ------------------------------------------------------------------------------------------------
  |Path Group|Effort|Adjustment(early/late)|Target Slack|weight|SkewingSlackCons|view              |
  ------------------------------------------------------------------------------------------------
  |reg2reg   | high | 0.1    /-0.2         | 0          |  2   |    0.3         |AV_HL_FUNC_MIN_RC1 |
  |reg2cgate | high | 0.2    /0            | 0          |  1   |    0.5         |AV_HL_FUNC_MAX_RC1 |
  ```

Related Information

- reportPathGroupOptions
- resetPathGroupOptions
- setPathGroupOptions
- The following topic in the *Innovus User Guide*
  - Path Group Support in Timing Optimization

createTBOptFile

```
createTBOptFile
[-help]
-bottleneckInstanceNumber integer
[-early]
[-max_paths integer]
[-max_slack float]
[-min_slack float]
-outFile filename
[-path_group groupname_list]
[-timingFile filename]
```

This command creates a target-based optimization (TBOpt) file for use with postRoute targeted optimization. Once created, this file is passed to the `setOptMode -opt_target_based_opt_file` command. You can either provide the name of the binary timing report on which you want to base the critical instance analysis by using the `-timingFile` parameter or let the command generate the timing report internally by using the values specified for the `-min_slack`, `-max_slack`, and `-max_paths` parameters or the internal default values for these parameters.

Based on this timing report, a bottleneck analysis is performed and the critical instances are stored in a file that is specified using the `-outFile filename` parameter. This file uses the TBOpt format with the INSTS token. The bottleneck analysis computes the number of times an instance appears (on only the failing paths by default) and creates "n" number of path categories for the instances with the highest number of occurrences. Note that "n" is the number you specify using the `-bottleneckInstanceNumber` parameter of this command.

Also note that by default, multiple instances per path are allowed. This means that if there are multiple critical instances on a path, all of them will be returned, therefore, leading to the generation of the most accurate list of critical instances.

For more information on using the TBOpt file, see Performing Target-Based Optimization in the *Innovus User Guide.*

Parameters

`-help`	Outputs a brief description that includes type and default information for each createTBOptFile parameter. For a detailed description of the command and all of its parameters, use the man command: `man createTBOptFile`
`-bottleneckInstanceNumber` *`integer`*	
	Specifies the number of critical bottleneck instances to be generated. This is a required parameter.

`-early`	Specifies that the TBOpt file will be created based on hold timing's critical nodes. This parameter can be specified along with the following commands to generate and use a hold target file for hold TBOpt. For example:
	`createTBOptFile -early -bottleneckInstanceNumber 5 -outFile TBOpt_bottleneck_5_hold.txt`
	`setOptMode -opt_target_based_opt_hold_file TBOpt_bottleneck_5_hold.txt`
	`optDesign -postRoute -targeted -hold`

`-max_paths` *`integer`*	Specifies the number of worst paths to be considered, regardless of the endpoint. By default, the value is `100,000`.
`-max_slack` *`float`*	Specifies that only those paths should be considered that have a slack equal to or less than the specified float value in `ns`. By default, the value is `0.0` to ensure that only paths with negative slacks are considered.
`-min_slack` *`float`*	Specifies that only those paths should be considered that have a slack greater than the specified float value in `ns`.
`-outFile` *`filename`*	Specifies the name of the TBOpt file that will be generated. This is a required parameter.
`-path_group` *`groupname_list`*	
	Specifies that only those paths should be considered that are from the specified path groups on which you want to base the critical instance analysis.
`-timingFile` *`filename`*	Specifies the name of the binary timing report on which you want to base the critical instance analysis. If this parameter is not specified, the command generates its own file with the `-max_slack` value of `0.0` and `-max_paths` value of `100,000`.

Example

- The following command generates a TBOpt file with five critical instances using the default values for `-min_slack`, `-max_slack`, and `-max_paths` that will be written to the file, `TBOpt_bottleneck_5.txt`:

  ```
  createTBOptFile -bottleneckInstanceNumber 5 -outFile TBOpt_bottleneck_5.txt
  ```

  The `TBOpt_bottleneck_5.txt` file will contain five critical instances with the appropriate syntax for target-based postRoute optimization.
  For example,

  ```
  INSTS : {mod_i/i_mod_glm_cluster_0/U586} {mod_i/i_mod_glm_cluster_2/U535} {mod_i/i_mod_glm_cluster_1/U596}
  {mod_i/ser_en/U92} {mod_i/ser_en/U101}
  ```

- The following command generates a TBOpt file with six critical instances based on only the `reg2reg` path group with the maximum number of worst paths as 3000:

  ```
  createTBOptFile -bottleneckInstanceNumber 6 -outFile TBOpt_bottleneck_6.txt -path_group reg2reg -max_paths 3000
  ```

- The following command generates a TBOpt file with nine critical instances based on a timing analysis of the worst 10,000 paths between `+0.025ns` and `-1.0ns`:

  ```
  createTBOptFile -bottleneckInstanceNumber 9 -outFile TBOpt_bottleneck_9.txt -min_slack -1.0 -max_slack +0.025 -
  max_paths 10000
  ```

Related Information

- `optDesign`
- `setOptMode`
- `report_timing`
- The following topic in the *Innovus User Guide*
  - Performing Target-Based Optimization

deleteBufferTree

```
deleteBufferTree
[-help]
[-excNetFile excFileName]
[-footprint footPrintName]
[-preserveRoute]
[-selNetFile selFileName | -net list_of_nets]
[-verbose]
```

Removes all buffers (except clock path buffers) and inverter pairs, of all footprint types, on all nets in the design. With specific settings, this command can remove buffers and inverter pairs of the specified footprint type, or on specified nets only.

Note:

1. place_opt_design will automatically call `deleteBufferTree` in the beginning, before global placement.
2. If you want to have a clean design with reduced density, or just clear all buffers and inverter pairs of a certain list of nets, it is recommended to call this command. This may allow you to do ECO on those nets.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `deleteBufferTree` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deleteBufferTree`
`-excNetFile` *excFileName*	Specifies the input file that contains the names of nets to exclude from buffer removal. If a net appears in the files specified by both `-excNetFile` and `-selNetFile`, it is excluded.
`-footprint` *footprintName*	Specifies the footprint of buffers to remove. If you do not specify this parameter, the Innovus software removes buffer instances with one input and one output terminal, and whose timing arc is not `SPECIAL`.
`-net` *list_of_nets*	Specifies the hierarchical nets to include during buffer removal. **Note**: The `-net` and `-selNetFile` parameters are mutually exclusive.
`-preserveRoute`	Specifies that nets not impacted by the buffer/inverter deletion process must have their routing unchanged. By default, the tool removes all the routes after running this command.
`-selNetFile` *selFileName*	Specifies the file that contains the names of hierarchical nets to include during buffer removal. The software excludes all other nets. If a net appears in the files specified by both `-excNetFile` and `-selNetFile`, it is excluded. **Note**: The `-net` and `-selNetFile` parameters are mutually exclusive.
`-verbose`	When specified, displays additional debug messages in log file. For example, the failure reason for deletion of buffer and inverter pairs.

Examples

- The following command removes all buffers and inverter pairs, of all footprint types:

```
deleteBufferTree
*** Start deleteBufferTree ***
Info: Detect buffers to remove automatically.
Analyzing netlist ...
Updating netlist

*summary: 79 instances (buffers/inverters) removed
*** Finish deleteBufferTree (0:00:09.6) ***
```

- The following command removes all buffers of footprint type `B_1P`:

```
deleteBufferTree -footprint B_1P
```

Related Information

- The following topic in the *Innovus User Guide*
  - Using the Footprintless Flow

deleteTieHiLo

```
deleteTieHiLo
[-help]
[-cell tieCellName | "listOfTieCellNames"]
[-prefix prefixName ]
```

Deletes placed tie cells from the netlist and reconnects the tie input pins back to `1'b1` and `1'b0`. Use this command if there are existing tie cells in design.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `deleteTieHiLo` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deleteTieHiLo`.
`-cell tieCellName | "listOfTieCellNames"`	
	The *`tieCellName`* argument specifies the cell name to delete. The " *`listOfTieCellNames`* " argument specifies the list of cell names to be deleted, which must be surrounded by quotation marks. **Note**: The *`tieCellName`* and "*`listOfTieCellNames`*" arguments are mutually exclusive.
`-prefix prefixName`	Specifies the prefix of the instances to delete.

Note: If you do not specify any parameters, the command uses the global cell names if they were set previously with the `setTieHiLoMode` command.

Example

- The following command deletes all placed tie-off cell instances with the prefix `ABC`:
  ```
  deleteTieHiLo -prefix ABC
  ```
- The following command deletes all placed tie-off cells named: `TIEOFF`:
  ```
  deleteTieHiLo -cell TIEOFF
  ```

Related Information

- `addTieHiLo`
- `setTieHiLoMode`
- The following topic in the *Innovus User Guide*
  - Adding Logical Tie-Off Cells

dumpMultiBitFlopMappingFile

```
dumpMultiBitFlopMappingFile
[-help]
[-mapOutputPins {false | true | all}]
[-output directory]
[-prefix fileName_prefix]
```

Dumps the multi-bit flip-flop (MBFF) mapping files in the specified directory with the specified prefix name. The following files are dumped:

- `multi_bit_pin_mapping`

  includes multi-bit input pin mapping information by default. In addition, it appends the output pins when the `-mapOutputPins` option is specified.

- `multi_bit_mapping`

  includes multi-bit optimization mapping information

The `multi_bit_pin_mapping` file has two columns. The first column provides the full path name of the flop input D pin as it was in the initial netlist. The second column provides the full path name of flop input D pin in the current state of the database that corresponds to the first column. For example, if SBF1, SBF2, SBF3, SBF4 are single-bit flip-flops, and MBF1, and MBF2 are two-bit flip-flops, then,

The **multi_bit_mapping** file contains the following:

```
SBF1(SBCell1), SBF2(SBCell2) merged to MBF1(MBCell1)

MBF2(MBCell2) split to SBF3(SBCell3), SBF4(SBCell4)
```

The **multi_bit_pin_mapping** file by default contains the following**:**

```
SBF1/D    MBF1/D1

SBF2/D    MBF1/D2

MBF2/D1   SBF3/D

MBF2/D2   SBF4/D
```

Note:

- The `multi_bit_pin_mapping` file contains all information of the flops whose input D pin names have changed. For example, see the "Examples" section below.
- To generate a proper report for merge/split mapping information, the command should be executed as part of the flow in the live session, for example, after running `place_opt_design`. If the command is run on a restored session, it will generate an empty "`multi_bit_mapping`" file because the merge/split mapping information is not saved while saving the design.
- You can use the `write_do_lec` command to facilitate the Logical Equivalence Checking (LEC) checking of the MBFF instances. This command writes out a `dofile` that contains the name mapping information of the MBIT flops against their split/single bit candidate.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `dumpMultiBitFlopMappingFile` parameter. For a detailed description of the command and all of its parameters, use the man command: `man dumpMultiBitFlopMappingFile`.

`-mapOutputPins {false | true | all}`	When set to `true`, the software writes out only the mapped output terms in the MBFF pin mapping file.
	When set to `false`, the software writes out only input pin mapping in the pin mapping file. This is the default setting.
	When set to `all`, the software writes out both input and output pin mapping in the pin mapping file.
	Example of **multi_bit_pin_mapping** file when `-mapOutputPins` parameter is set to `false`:
	`MBF1/D1  SBF1/D`
	`MBF1/D2  SBF2/D`
	Example of **multi_bit_pin_mapping** file when `-mapOutputPins` parameter is set to `true`:
	`MBF1/Q1 SBF1/Q`
	`MBF1/Q2 SBF2/Q`
	Example of **multi_bit_pin_mapping** file when the `-mapOutputPins` parameter is set to `all`:
	`MBF1/D1 SBF1/D`
	`MBF1/D2 SBF2/D`
	`MBF1/Q1 SBF1/Q`
	`MBF1/Q2 SBF2/Q`
	Default: `false`

`-output` *directory*	Specifies the directory in which to dump the MBFF mapping files. *Default*: ./
`-prefix` *fileName_prefix*	Specifies the prefix to the MBFF mapping files. *Default*: `without prefix`

Examples

Consider a scenario in which the original netlist contains the following information:

```
4BitMBF1        (4 bit MBF, D1-4)
SBF1            (single bit FF)
```

Example of merging is as follows:

If the incoming netlist has `4BitMBF1/D4` dangling (no signal attached) and MERGE, merged `SBF1/D` into `4BitMBF1/D4`. The following line will be included in `multi_bit_pin_mapping` file:

```
SBF1/D          4BitMBF1/D4
```

Example of splitting is as follows:

If incoming netlist has valid signals for all 4 bits of `4BitMBF1` and splits `4BitMBF1/D4` out to `SDF2/D` leaving `4BitMBF1/D4` dangling, then the following lines will be included in `multi_bit_pin_mapping` file:

```
*VOID*          4BitMBF1/D4
4BitMBF1/D4     SBF2/D
```

Related Information

- rename_obj
- ecoSplitFlop
- ecoSplitComplexFlop
- reportMultibit
- setOptMode
- setDesignMode
- write_do_lec
- The following topic in the *Innovus User Guide*
  - Using the Multi-Bit Flip-Flop Flow

ecoClone

```
ecoClone
[-help]
[-allow_sequential]
[-auto]
[-balance num]
[-cells list_of_cells]
[-delete_buffer_trees]
[-dont_legalize]
[-dont_resize]
[-force]
[-from pin names]
[-instances instance names]
[-locs list_of_locations]
[-loop number]
[-max_levels integer]
[-max_num_loads integer]
[-min_num_loads integer]
[-nets list_of_nets]
[-numClones number]
[-preview]
[-radius number]
[-same_hier_only]
[-terms list_of_term_lists]
[-to pin names]
[-verbose]
```

Clones combinational logic if logic does not have any user constraints like don’t size/touch and so on. This command is used when you want to clone combinational logic as manual ECO. It does not work for sequential logic like flops. The `-from` and `-to` parameters are used to specify the 'from' and 'to' pins for cloning.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `ecoClone` parameter. For a detailed description of the command and all of its parameters, use the man command: `man ecoClone`
`-allow_sequential`	Used with `-auto` to try cloning at the sequential gates too.
`-auto`	Applies cloning and buffering automatically. The software identifies and clones combinational logic based on critical endpoints distance-based clustering.
`-balance number`	When specified, forces the number of loads on each clone to be within the specified number percentage (*num* `%`). The default balance is 10%.

`-cells` *list_of_cells*	Specifies the list of cells to use for the clones. **Note**: ◦ If this parameter is not specified, the clones have the same cell as the original gate. ◦ If only one cell is specified in the list, all the clones are created with that cell. ◦ If the `-locs` and `-terms` parameters are not specified, you can only specify one cell in that list. ◦ If the `-locs` and/or `-terms` parameters are specified, you can also specify numClones cells, which is one cell for each clone. ◦ It is not possible to specify a new cell for the original gate, so the original gate is not resized.
`-delete_buffer_trees`	When specified, the software attempts to delete buffer trees driven by instances to clone. **Note**: The deletion of buffer trees with user constraints is skipped.
`-dont_legalize`	When specified, the software may leave cloned instances overlapped with the original ones. This means the software will not try to legalize instances. This means that if space is available, it will place legally, otherwise it will leave cloned instances overlapped with the original ones.
`-dont_resize`	Specifies that neither the original nor the new (cloned) instances should be resized.
`-force`	When specified, forces instance cloning, even if it worsens QoR or causes illegal placement.
`-from` *pin names*	Specifies 'from' pins. You can specify a single pin or a list of pins.
`-instances` *instance names*	Specifies the instances to be cloned. You can specify a single instance or a tcl list of instances to be cloned.
`-locs` *list_of_locations*	Specifies the location of each clone. This parameter is used along with the `-numClones` parameter. When this parameter is specified, a location should be specified for each clone, so the length should be numClones. The length can be numClones+1, in which case, the first location is used for the original gate. When the locations are specified, the sink partitions are assigned to the closest clone.
`-loop` *number*	Repeat cloning at most *number* times, as far as new instances are being created. The original and new instances within the min/max number of loads range are valid targets at each iteration. *Default*: `0` (means no limit)
`-max_levels` *integer*	Specifies the level of instances that the software needs to traverse for cloning. Zero value means the software will clone everything till the origin of logic. *Default*: 0 (meaning infinite)
`-max_num_loads` *integer*	Specifies that cloning should be skipped for instances driving more than the specified number of loads. *Default*: `50000`
`-min_num_loads` *integer*	Specifies that cloning should be skipped for instances with less than or equal to the specified number of loads. **Note**: The `-min_num_loads` and `-numClones` parameters are mutually exclusive.
`-nets` *list_of_nets*	Specifies the nets to be cloned. You can specify a single net or a tcl list of nets to be cloned. **Note**: The `-nets` and `-instances` parameters are mutually exclusive. Wildcards are allowed for this parameter.

`-numClones` *number*	Specifies the number of clones to create. The default value is `0` and in that case, it is computed from the `-min_num_loads` parameter. **Note**: The `-min_num_loads` and `-numClones` parameters are mutually exclusive. *Default:* `0`
`-preview`	When specified, provides a preview of what would be done, while leaving the netlist untouched. This means that the software will report instances that will be cloned and the changes that will be applied to the netlist without actually changing the netlist.
`-radius` *number*	Specifies that the clones should be placed within a radius of the specified number of rows from their preferred location based on the original instance location and the center of mass of the new set of loads. *Default*: `10`
`-same_hier_only`	Specifies that only those loads will be considered that are in the same hierarchy as the driver.
`-terms` *list_of_term_lists*	Specifies the partitions, so it is a list of term lists. This parameter is used along with the `-numClones` parameter. When this parameter is specified, there should be one partition for each clone, so the length should be numClones. The terms that are not in any partition are assigned to the partition driven by the original gate. The length can be numClones+1, in which case the first partition is for the original gate. In that case, all the sinks should be in a partition.
`-to` *pin names*	Specifies 'to' pins. You can specify a single pin or a list of pins.
`-verbose`	When specified, verbose messages are displayed during the cloning of instances.

Example

The following command will clone logic till `$to` pins with more then `6` loads, while forciby deleting the buffer tree:

```
ecoClone -to $to -verbose  -min_num_loads 6 -delete_buffer_trees -force
```

Related Information

- `ecoCloneFlop`
- `setOptMode`
- `setDesignMode`

ecoCloneFlop

```
ecoCloneFlop
[-help]
[-force]
-instances flopInstName
[-max_num_loads num_of_loads]
[-numClones number]
[-radius num_of_rows]
[-verbose]
```

Clones flop instances as long as the flops meet the following criteria:

- They are not FIXED
- They are allowed to be resized
- When they do not share synchronization signals with some other instance

This command is used when you want to clone a flop as manual ECO. Manual flop cloning is usually performed on control signal flip-flops, when the output needs to reach sections of the floorplan that are spread out. Cloning the flip-flops allows you to place each clone close to its sink points, thereby reducing the timing closure effort.

This command should not be used to clone a flop that drives a clock net, which is a net with the attribute `isDefInclock`, because this may lead to problems during optimization or timing. The default behavior of the command may reject the flip-flop cloning due to design constraints or QOR impact.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `ecoCloneFlop` parameter. For a detailed description of the command and all of its parameters, use the man command: `man ecoCloneFlop`.
`-force`	When specified, forces the flop instance to clone. This parameter is used when flop cloning is rejected by the software.
`-instances` *`flopInstName`*	Specifies the list of instances to be cloned. This is a required parameter.
`-max_num_loads` *`num_of_loads`*	Specifies that cloning should be skipped for instances driving more than the specified number of loads. *Default*: `50000`
`-numClones` *`number`*	Specifies the cloned numbers for the flop instance.
`-radius` *`num_of_rows`*	Specifies that the clones should be placed within a radius of the specified number of rows from their preferred location based on the original instance location and the center of mass of the new set of loads. *Default*: `10`
`-verbose`	When specified, verbose messages are displayed during the cloning of flop instances.

Example

The following command will clone the flip-flop `$inst` 3 times:

```
ecoCloneFlop -instances $inst -numClones 3
```

Related Information

- `ecoSplitFlop`
- `ecoSplitComplexFlop`
- `dumpMultiBitFlopMappingFile`
- `reportMultibit`
- `setOptMode`
- `setDesignMode`
- The following topic in the *Innovus User Guide*
  - Using the Multi-Bit Flip-Flop Flow

ecoMergeCombinationalCell

```
ecoMergeCombinationalCell
[-help]
[-area]
[-force]
-inst comboInstName | -listFile instNamesFile | -all
[-power]
-preCTS | -postCTS
```

Merges combinational cells into multi-bit cells for the specified instance. You can specify the names of the combinational instances to be merged or you can specify the file containing names of single bit instances to merge.

Naming Convention During Merging

When the combinational cells in the specified file are merged, the corresponding instance names are combined as follows:

- `hier1/hier2/i_0 + hier1/hier2/i_1 + hier1/hier2/i_2 + hier1/hier2/i_3`

  TO

  `hier1/hier2/CDN_MB_i_0_MB_i_1_MB_i_2_MB_i_3`

- `abc_x[0] + abc_x[1] + abc_x[2] + abc_x[3]`

  TO

  `CDN_MB_abc_x[0]_MB_x[1]_MB_x[2]_MB_x[3]`

Parameters

`-help`	Outputs a brief description that includes type and default information for each `ecoMergeCombinationalCell` parameter. For a detailed description of the command and all of its parameters, use the man command: `man ecoMergeCombinationalCell`.
`-all`	Specifies that all single-bit cell instances should be merged into multi-bit cell instances. This parameter is mutually exclusive with the `-listFile` and `-all` parameters. At least one of these parameters must be specified.
`-area`	Specifies to merge the combinational cells for area purposes. *Default*: `false`
`-force`	Forces the merging of the combinational cell regardless of the timing gains. *Default*: `false`
`-inst` *comboInstName*	Specifies the name of the combinational instance to be merged. This is a required parameter. This parameter is mutually exclusive with the `-listFile` and `-all` parameters. At least one of these parameters must be specified.
`-postCTS`	Specify this parameter if the design stage is postCTS. **Note**: This parameter is mutually exclusive with the `-preCTS` parameter. One of the two parameters must be specified.
`-listFile` *instNamesFile*	Specifies the file containing the names of single bit instances to merge. Specify one instance on one line. Comment with Instance name should not contain spaces or backslashes This parameter is mutually exclusive with the `-inst` and `-all` parameters. At least one of these parameters must be specified.
`-power`	Specify this parameter to merge the combinational cells for power purposes. *Default*: `false`
`-preCTS`	Specify this parameter if the design stage is preCTS. **Note**: This parameter is mutually exclusive with the `-postCTS` parameter. One of the two parameters must be specified.

Related Information

- `ecoSplitFlop`
- `ecoSplitCombinationalCell`
- `ecoSplitComplexFlop`
- `reportMultibit`
- `setOptMode`
- The following topic in the *Innovus User Guide*
  - Multi-Bit Combinational Cell Optimization

ecoSplitCombinationalCell

```
ecoSplitCombinationalCell
[-help]
[-force]
[-fullSplit]
[-legal]
[-skipRoute]
-inst combInstName | {{Mbcb1 combCell1} {Mbcb2 combCell2}...} | -listFile fileName | -all
-preCTS | -postCTS | -postRoute
```

Splits all multi-bit combinational cell instances to single-bit combinational cells. You can specify the names of the combinational instances to be merged or you can specify the file containing names of multi-bit instances to be split.

Naming Convention During Splitting

When splitting Multibit combinational cells, it will split the names as follows:

- `hier1/hier2/i_[0:3]`

  TO

  `hier1/hier2/i_[0:3]__0, hier1/hier2/i_[0:3]__1, hier1/hier2/i_[0:3]__2, hier1/hier2/i_[0:3]__3`

- `CDN_MB_abc_MB_def`

  TO

  `abc, def`

Parameters

`-help`	Outputs a brief description that includes type and default information for each `ecoSplitCombinationalCell` parameter. For a detailed description of the command and all of its parameters, use the man command: `man ecoSplitCombinationalCell`.
`-all`	Splits all multi-bit combinational cell instances to SBFF instances. This parameter is mutually exclusive with the `-inst` and `-listFile` parameters. At least one of these parameters must be specified. *Default*: `false`
`-force`	Forces the splitting of the combinational cell regardless of the timing gains. *Default*: `false`
`-fullSplit`	Set this parameter to completely split the multi-bit combinational cells to single bits.
`-inst` *comboInstName* `{{`*Mbcb1 combCell1*`} {`*Mbcb2 combCell2*`}...}`	
	Specifies the names of the combinational instance to be split. If the input is a string of multi-bit combinational (MBCB)/Cell pairs list, it will split the MBCB to the specified cell without evaluation This parameter is mutually exclusive with the `-listFile` and `-all` parameters. At least one of these parameters must be specified.
`-legal`	Specifies to place the split flip-combinational cells at legal location. *Default*: `false`
`-listFile` *fileName*	Specifies the file containing names of MBCB instances to split. Specify one instance on one line. Comment with #. Instance name should not contain spaces or backslashes. And if one instance and one cell is specified on one line, it will split the MBCB to the specified cell without evaluation. This parameter is mutually exclusive with the `-inst` and `-all` parameters. At least one of these parameters must be specified.
`-postCTS`	Specify this parameter if the design stage is postCTS. This is a required parameter. One of these parameters, `-preCTS`, `-postCTS`, or `-postRoute` must be specified.
`-postRoute`	Specify this parameter if the design stage is postRoute. This is a required parameter. One of these parameters, `-preCTS`, `-postCTS`, or `-postRoute` must be specified.
`-preCTS`	Specify this parameter if the design stage is preCTS. This is a required parameter. One of these parameters, `-preCTS`, `-postCTS`, or `-postRoute` must be specified.
`-skipRoute`	Specify this parameter if the design stage is postRoute and globalDetailRoute is not required after the split.

Related Information

- `ecoSplitFlop`
- `ecoMergeCombinationalCell`
- `ecoSplitComplexFlop`
- `reportMultibit`
- `setOptMode`
- The following topic in the *Innovus User Guide*
  - Multi-Bit Combinational Cell Optimization

ecoSplitComplexFlop

```
ecoSplitComplexFlop
[-help]
[-force]
{-inst {inst1, inst2,...} | -listFile instNamesFile | -all}
[-legal]
[-preCTS | -postCTS | -postRoute]
[-power]
[-skipRoute]
```

Splits a complex flip-flop into a simple flip-flop and combinational logic for the specified instance. A complex flip-flop means the D side has some combinational logic, such as AND, OR, MUX, AOI, and so on. By default, a specific flip-flop is split only if equivalent simple flip-flops and combinational logic can be found in libraries and there is timing gain or power gain after the split.

You can also specify that the split flip-flops should be placed in legal locations. By default, the positions of the split flip-flops are not guaranteed to be legal.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `ecoSplitComplexFlop` parameter. For a detailed description of the command and all of its parameters, use the man command: `man ecoSplitComplexFlop`
`-all`	Specifies that all flop instances in the design should be processed. **Note**: The `-inst`, `-listFile`, and `-all` parameters are mutually exclusive. One of these parameters must be specified.
`-force`	When specified, forces the complex flip-flops to split regardless of the timing gains.
`-inst {inst1, inst2,...}`	
	Specifies the names of the flop instances to be split. You can specify a single instance or a tcl list of instances to be split. **Note**: The `-inst`, `-listFile`, and `-all` parameters are mutually exclusive. One of these parameters must be specified.
`-listFile instNamesFile`	
	Specifies the file containing the names of the flop instances to be split. Specify one instance on one line. Comment with #. Instance name should not contain spaces or backslashes. **Note**: The `-inst`, `-listFile`, and `-all` parameters are mutually exclusive. One of these parameters must be specified.
`-legal`	When specified, places the split flip-flops and combinational cells at a legal location.
`-preCTS`	Specify this parameter, if the design stage is preCTS.
`-postCTS`	Specify this parameter, if the design stage is postCTS.
`-postRoute`	Specify this parameter, if the design stage is postRoute.
`-power`	When specified, complex flops are split to reclaim power. By default, the flops are split for timing.
`-skipRoute`	Skip globalDetailRoute after split in postRoute stage.

Examples

- Use the following command to force the flip-flop split for power for all instances during the preCTS stage.
  ```
  ecoSplitComplexFlop -force -preCTS -power -all
  ```
- Use the following command for flip-flop split for timing for the instance ff1 during the preCTS stage.
  ```
  ecoSplitComplexFlop -inst ff1 -preCTS
  ```

Related Information

- `ecoSplitFlop`
- `ecoCloneFlop`
- The following topics in the *Innovus User Guide*
  - Splitting Complex Flops
  - Using the Multi-Bit Flip-Flop Flow

ecoSplitFlop

```
ecoSplitFlop
[-help]
[-batch]
[-force]
[-fullSplit]
{-inst flopInstName | {{Mbff1 flopCell1} {Mbff2 flopCell2}...} | -listFile <fileName> | -all}
[-legal]
[-power]
{-preCTS | -postCTS | -postRoute}
[-skipRoute]
```

Splits a multi-bit flip-flop (MBFF) into single-bit flip-flops for the specified instance. By default, a specific flip-flop is split only if equivalent single-bit flip-flops can be found in libraries and there is timing gain after the split. You can also specify that the split flip-flops should be placed in legal locations. By default, the positions of the split flip-flops are not guaranteed to be legal.

Note:

- This command is not controlled by the `-multiBitFlopOpt` parameter setting of `setOptMode`.
- The command calls refinePlace at the end. To disable this, use `setEcoMode -refinePlace false`.
- MBFFs must be identified by the command, `reportMultibit`.
- A placeable location for the split FFs is required.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `ecoSplitFlop` parameter. For a detailed description of the command and all of its parameters, use the man command: `man ecoSplitFlop`
`-all`	When specified, splits all MBFFs into smaller-bit flip-flops including single-bit flip-flops (SBFFs). Only timing critical bits are split into single-bit flops, and the remaining, non-critical bits, are split into smaller bit width MBFFs, to preserve power as much as possible. For example, a 4-bit MBFF may be split into two single-bit flops and one 2-bit flop. **Note**: The `-all`, `-inst` and `-listFile` parameters are mutually exclusive. If the `-all` parameter is not specified, one of these two parameters, `-inst` or `-listFile`, must be specified. *Default*: `false`
`-batch`	When specified, splits the MBFFs in batches. For example, `ecoSplitFlop -listFile list -postCTS -batch -force` *Default*: `false`
`-force`	Forces the flip-flop splits irrespective of timing gains. **Note**: • If there is no timing gain and this parameter is not specified, the flip-flops for the specified instance will not be split. • If this option is set along with the `-legal` option, the software will first look for legal locations. However, the flip-flops will be split even if no legal location is found. *Default*: `false`

`-fullSplit`	Forces the MBFFs to be completely split into all single-bit flip-flops.
`-inst` *`flopInstName`* `| {{Mbff1 flopCell1} {Mbff2 flopCell2}...}`	
	Specifies pairs of MBFFs to be split and the corresponding single-bit flops to be used. You can also specify the names of the flop instances to be split. You can specify a single instance or a tcl list of instances to be split. **Note**: The `-all`, `-inst` and `-listFile` parameters are mutually exclusive. If the `-all` parameter is not specified, one of these two parameters, `-inst` or `-listFile`, must be specified.
`-listFile` *`fileName`*	
	Specifies the file containing pairs of MBFFs to be split and the corresponding single-bit flops to be used. You can also just specify the names of MBFF instances to be split. You can specify up to 50,000 instances to split. Specify each instance on a line and comment with #. Instance name should not contain spaces or backslashes. **Note**: The `-all`, `-inst` and `-listFile` parameters are mutually exclusive. If the `-all` parameter is not specified, one of these two parameters, `-inst` or `-listFile`, must be specified.
`-legal`	Places the split flip-flops at legal locations. **Note**: If both the `-legal` and `-force` parameters are specified, the software will first look for legal locations. However, the flip-flops will be split even if no legal location is found. In this case, the split instances will be placed in the original location of the MBFFs. *Default*: `false`
`-power`	Splits the flip-flop for power purpose. *Default*: `false`
`-preCTS`	Specify this parameter, if the design stage is preCTS. *Default*: `-postRoute`
`-postCTS`	Specify this parameter, if the design stage is postCTS. *Default*: `-postRoute`
`-postRoute`	Specify this parameter, if the design stage is postRoute. This is the default setting. **Note**: One of the three options, `-preCTS`, `-postCTS`, or `-postRoute` must be specified.
`-skipRoute`	Specify if the stage is postRoute and globalDetailRoute is not required to be performed after the split. Since postRoute is also the default stage, globalDetailRoute is also skipped when the stage is default and `-skipRoute` is specified. However, for the default case, when the stage is postRoute but this parameter is not specified, after splitting the instances, globalDetailRoute will be called. *Default*: `false`

Example

- The following command will force the flip-flop split for the instance, `ff_0` during the `preCTS` stage:

  ```
  ecoSplitFlop -preCTS -inst ff_0 -force
  ```

- The following command will will try to split for power all MBFFs during the `postCTS` stage:

  ```
  ecoSplitFlop -power -all -postCTS
  ```

Related Information

- `ecoCloneFlop`
- `ecoSplitComplexFlop`
- `dumpMultiBitFlopMappingFile`
- `reportMultibit`
- `setOptMode`
- `setDesignMode`
- The following topic in the *Innovus User Guide*
  - Using the Multi-Bit Flip-Flop Flow

findLefEquivalentCells

```
findLefEquivalentCells
[-help]
-cells string
[-outfile filename]
```

Provides support to debug certain cells that may be non-equivalent without detailed analysis of the LEF. For example, if two cells have the same function and size they still may not be LEF equivalent.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `findLefEquivalentCells` parameter. For a detailed description of the command and all of its parameters, use the man command: `man findLefEquivalentCells`
`-cells` *`string`*	Specifies the cell names. You can also use wildcards or specify individual cells.
`-outfile` *`filename`*	Specifies the output file name for reporting LEF equivalent cells. All the equivalent and non-equivalent cells are saved in the output file.

Examples

- The following example command reports equivalence information on all cells and outputs to a file:
  ```
  findLefEquivalentCells -cells * -outFile equivalency_file.txt
  ```
- To report equivalence information on a specific cell, say `SCBUF1X`, you can use the following command:
  ```
  findLefEquivalentCells -cells SCBUF1X
  ```
- To report equivalence information on all cells that begin with `SCBUF`, you can use the following command:
  ```
  findLefEquivalentCells -cells SCBUF*
  ```

getLatencyFile

```
getLatencyFile
[-help]
```

Returns the name of the latency file used by the timing engine. If no latency file name is set, `getLatencyFile` returns nothing, but if you have already run `skewClock`, this command reports the default file name. If you have specified or changed the latency file name by using the `setLatencyFile` command, `getLatencyFile` reports this name.

To view the above description in the software, use `man getLatencyFile`.

Related Information

- `setLatencyFile`
- The following topic in the *Innovus User Guide*
  - Performing Optimization Before CTS

getOptMode

```
getOptMode
[-help]
[-opt_activity_refresh_args list_of_arguments]
[-opt_add_always_on_feed_through_buffers]
[-opt_add_insts]
[-opt_add_ports]
[-opt_add_repeater_report_failure_reason]
[-opt_all_end_points]
[-opt_allow_multi_bit_on_flop_with_sdc]
[-opt_allow_only_cell_swapping]
[-opt_area_recovery]
[-opt_area_recovery_setup_target_slack]
[-opt_consider_routing_congestion]
[-opt_constant_inputs]
[-opt_constant_nets]
[-opt_concatenate_default_and_user_prefixes]
[-opt_delete_insts]
[-opt_detail_drv_failure_reason]
[-opt_detail_drv_failure_reason_max_num_nets]
[-opt_down_size_insts]
[-opt_drv]
[-opt_drv_margin]
[-opt_drv_with_miller_cap]
[-opt_duplicate_cte_constrained_hport]
[-opt_enable_data_to_data_checks]
[-opt_enable_podv2_clock_opt_flow]
[-opt_enable_restructure]
[-opt_fix_fanout_load]
[-opt_flop_pins_report]
[-opt_flops_report]
[-opt_high_effort_cells]
[-opt_hold_allow_overlap]
[-opt_hold_allow_resize]
[-opt_hold_allow_setup_tns_degradation]
[-opt_hold_cells]
[-opt_hold_on_excluded_clock_nets]
[-opt_hold_slack_threshold]
[-opt_hold_target_slack]
[-opt_honor_density_screen]
[-opt_honor_fences]
[-opt_hold_ignore_path_groups]
[-opt_leakage_to_dynamic_ratio]
[-opt_max_density]
[-opt_max_length]
[-opt_move_insts]
[-opt_multi_bit_combinational_opt]
[-opt_multi_bit_combinational_merge_timing_effort]
[-opt_multi_bit_combinational_split_timing_effort]
[-opt_multi_bit_flop_merge_bank_label_inference]
[-opt_multi_bit_flop_merge_timing_effort]
[-opt_multi_bit_flop_name_prefix]
[-opt_multi_bit_flop_name_separator]
[-opt_multi_bit_flop_name_suffix]
[-opt_multi_bit_flop_opt]
[-opt_multi_bit_flop_reorder_bits]
[-opt_multi_bit_flop_split_report_failure_reason]
```

```
[-opt_multi_bit_flop_split_timing_effort]
[-opt_multi_bit_unused_bit_count]
[-opt_multi_bit_unused_bits]
[-opt_new_inst_prefix]
[-opt_new_net_prefix]
[-nonDefault]
[-opt_one_pass_lec]
[-opt_podv2_flow_effort]
[-opt_pin_swapping]
[-opt_post_route_allow_overlap]
[-opt_post_route_area_reclaim]
[-opt_post_route_art_flow]
[-opt_post_route_check_antenna_rules]
[-opt_post_route_drv_recovery]
[-opt_post_route_fix_clock_drv]
[-opt_post_route_fix_glitch]
[-opt_post_route_fix_si_transitions]
[-opt_post_route_hold_recovery]
[-opt_post_route_setup_recovery]
[-opt_power_effort]
[-opt_pre_route_ndr_aware]
[-opt_preserve_all_sequential]
[-opt_preserve_hpin_function]
[-opt_remove_redundant_insts]
[-opt_report_multi_bit_unmerged_reasons]
[-opt_resize_flip_flops]
[-opt_resize_level_shifter_and_iso_insts]
[-opt_resize_power_switch_insts]
[-opt_sequential_genus_restructure_report_failure_reason]
[-opt_setup_target_slack]
[-opt_skew]
[-opt_skew_ccopt]
[-opt_skew_post_route]
[-opt_skew_pre_cts]
[-opt_target_based_opt_hold_file]
[-opt_target_based_opt_file]
[-opt_target_based_opt_file_only]
[-opt_tied_inputs]
[-opt_time_design_compress_reports]
[-opt_time_design_num_paths]
[-opt_time_design_expanded_view]
[-opt_time_design_report_net]
[-opt_time_design_vertical_timing_summary]
[-opt_unfix_clock_insts]
[-opt_verbose]
[-quiet]
```

Returns information about the specified `setOptMode` parameter in the Innovus log file and the console. The following information is returned:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the Innovus software returns values for all of the `setOptMode` parameters.

Parameters

parameter_names	Returns information for the specified parameter. You can specify one or more parameters. See `setOptMode` for descriptions of the parameters you can specify.
`-nonDefault`	Returns the current settings of the `setOptMode` parameters that are not default.
`-quiet`	Returns the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software returns the current settings of all `setOptMode` parameters in Tcl list format.

Examples

- The following command returns the current setting of the `-addInst` parameter:

  ```
  getOptMode -opt_add_insts
  ```

  The software returns the following information:

  ```
  -opt_add_insts true # bool, default=true
  true
  ```

- The following command returns the current setting of the `-addInst` parameter in Tcl list format only:

  ```
  getOptMode -opt_add_insts -quiet
  ```

  The software returns the following information:

  ```
  true
  ```

Related Information

- `setOptMode`
- Optimizing Timing chapter in the *Innovus User Guide*

getSchedulingFile

```
getSchedulingFile
[-help]
```

Returns the name of the current scheduling file used by the `skewClock` command. If no scheduling file name is set, `getSchedulingFile` returns nothing. If you have already run `skewClock`, this command reports the default file name. If you have specified or changed the scheduling file name using `setSchedulingFile`, then this command returns this name.

To view the above description in the software, use `man getSchedulingFile`.

Related Information

- `setSchedulingFile`
- The following topic in the *Innovus User Guide*
  - Performing Optimization Before CTS

getSignoffOptMode

```
getSignoffOptMode
[-help]
[-addInst]
[-addLoad]
[-allowSkewing]
[-alongRouteBuffering]
[-bufferCellList]
[-checkDrvFromHoldViews]
[-checkType]
[-clockCellList]
[-clockMaxLevel]
[-deleteInst]
[-disableGeometryChecks]
[-drvMargin]
[-ecoFilePrefix]
[-emType]
[-fixCellEm]
[-fixClockDrv]
[-fixDataDrv]
[-fixHoldAllowSetupDegrade]
[-fixHoldAllowSetupOptimization]
[-fixHoldAllowSetupTnsDegrade]
[-fixHoldVariationAware]
[-fixHoldWithMargin]
[-fixGlitch]
[-fixIrDrop]
[-fixMaxCap]
[-fixMaxTran]
[-fixOsusGlitch]
[-fixSlackInRangeLowerLimit]
[-fixSlackInRangeUpperLimit]
[-fixSiSlew]
[-fixXtalk]
[-fixXtalkPreserveHold]
[-fixXtalkPreserveSetup]
[-holdTargetSlack]
[-holdTargetSlackPerView]
[-holdXtalkDeltaThreshold]
[-holdXtalkSlackThreshold]
[-ignoreDrvChecks]
[-keepTempFiles]
[-legalOnly]
[-loadEcoOptDb]
[-loadIrDropDb]
[-maxCapMargin]
[-maxLocalDensity]
[-maxOptRunTime]
[-maxPaths]
[-maxRunTime]
[-maxSlack]
[-maxTranMargin]
[-numReportPaths]
[-nworst]
[-optimizeCoreOnly]
[-optimizeReplicatedModules]
[-optimizeSequentialCells]
```

```
[-partitionListFile]
[-pbaEcoRoute]
[-pbaEffort]
[-postMask]
[-postStaTcl]
[-powerAware]
[-powerOptFocus]
[-prefixName]
[-preserveFiller]
[-preStaTcl]
[-resizeInst]
[-retime]
[-retimeDepth]
[-retimeDepthEarly]
[-retimeMode]
[-routingCongestionAware]
[-routeCongestionThreshold]
[-saveEcoOptDb]
[-selectDrvNetFile]
[-selectDrvPinFile]
[-selectHoldEndpoints]
[-selectSetupEndpoints]
[-setupRecovery]
[-setupTargetSlack]
[-setupTargetSlackPerView]
[-setupXtalkDeltaThreshold]
[-setupXtalkSlackThreshold]
[-skewCoreOnly]
[-skipDrvNetfile]
[-specifyHoldEndpointsMargin]
[-specifySetupEndpointsMargin]
[-swapInst]
[-upsizeInPowerOpt]
[-useGaFillerList]
[-usePfcDecapList]
[-usePfcFillerList]
[-usePfcRegularList]
[-verbose]
[-quiet]
[-nonDefault]
```

Returns information about the specified `setSignoffOptMode` parameter in the Innovus log file and the console. The following information is returned:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, Innovus returns values for all of the `setSignoffOptMode` parameters.

Parameters

parameter_names	Returns information for the specified parameter. You can specify one or more parameters. See `setSignoffOptMode` for descriptions of the parameters you can specify.
`-nonDefault`	Returns the current settings of the `setSignoffOptMode` parameters that are not default.
`-quiet`	Returns the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software returns the current settings of all `setSignoffOptMode` parameters in Tcl list format.

Related Information

- `setSignoffOptMode`
- `signoffOptDesign`
- The following topics in the *Innovus User Guide*
  - Running MMMC SignOff ECO within Innovus
  - Using Signoff Timing Analysis to Optimize Timing and Power
  - Signoff Timing Analysis in Innovus using Timing Debug
  - Optimization in Path-Based Analysis (PBA) Mode
  - Total Power Optimization
  - Setup Timing Recovery After a Large Leakage or Total Power Optimization
  - Achieving the Best Results for Total Power Optimization
  - Path Group Support in Timing Optimization
  - Top Down Block ECO flow using Tempus Signoff Timing

getTieHiLoMode

```
getTieHiLoMode
[-cell]
[-createHierPort]
[-honorDontTouch]
[-maxDistance]
[-maxFanout]
[-modulePrevention]
[-prefix]
[-reportHierPort]
[-quiet]
[-nonDefault]
```

Returns information about the specified `setTieHiLoMode` parameter in the Innovus log file and the console. The following information is returned:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the Innovus software displays values for all of the `setTieHiLoMode` mode parameters.

Parameters

parameter_names	Returns information for the specified parameters. You can specify one or more parameters. See `setTieHiLoMode` for parameter descriptions.
`-nonDefault`	Returns the current settings of the `setTieHiLoMode` parameters that are not default.
`-quiet`	Returns the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software returns the current settings of all `setTieHiLoMode` parameters in Tcl list format.

Examples

- The following command returns the current setting for the `-cell` parameter in Tcl list format:
  ```
  getTieHiLoMode -cell -quiet
  ```
  The software returns the following information:
  ```
  {a16lvt16_tie1_a1 a16lvt16_tie0_a1}
  ```
- The following command returns the current settings for the `-cell` and `-honorDontTouch` parameters:
  ```
  getTieHiLoMode -cell -honorDontTouch
  ```
  The software returns the following information:
  ```
  -cell {a16lvt16_tie1_a1 a16lvt16_tie0_a1} # string, default=""
  -honorDontTouch false # bool, default=false
  {cell {a16lvt16_tie1_a1 a16lvt16_tie0_a1}} {honorDontTouch false}
  ```
- The following command returns the current settings for the `-cell` and `-honorDontTouch` parameters in Tcl list format:
  ```
  getTieHiLoMode -cell -honorDontTouch -quiet
  ```

The software returns the following information:

```
{cell {a16lvt16_tie1_a1 a16lvt16_tie0_a1}} {honorDontTouch false}
```

- The following command returns the current settings for all `setTieHiLoMode` parameters in Tcl list format:

  ```
  getTieHiLoMode -quiet
  ```

  The software returns the following information:

  ```
  {cell {a16lvt16_tie1_a1 a16lvt16_tie0_a1}} {createHierPort false} {honorDontTouch false} {maxDistance 0} {maxFanout 0} {prefix {}} {reportHierPort false}
  ```

Related Information

- `addTieHiLo`
- `deleteTieHiLo`
- `setTieHiLoMode`
- The following topic in the *Innovus User Guide*
  - Adding Logical Tie-Off Cells

getUsefulSkewMode

```
getUsefulSkewMode
[-help]
[-nonDefault]
[-opt_skew_apply_delay_limits_to_full_flow]
[-opt_skew_delay_pre_cts]
[-opt_skew_macro_only]
[-opt_skew_max_allowed_delay]
[-opt_skew_min_allowed_delay]
[-opt_skew_no_boundary]
[-quiet]
```

Returns the current setting of the `setUsefulSkewMode` command. The `setUsefulSkewMode` command is used to set global paramaters for clock skewing during optimization.

If you do not specify a parameter, the software returns information for all of the `setUseFulSkewMode` mode parameters.

Parameters

parameter_names	Returns information for the specified parameter. You can specify one or more parameters. See `setUsefulSkewMode` for parameter descriptions.
`-nonDefault`	Returns the current settings of the `setUsefulSkewMode` parameters that are not default.
`-quiet`	Returns the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software returns the current settings of all `setUsefulSkewMode` parameters in Tcl list format.

Examples

- The following command returns the current setting of the `setUsefulSkewMode -opt_skew_delay_pre_cts` parameter:

  ```
  getUsefulSkewMode -opt_skew_delay_pre_cts
  ```

  The software returns the following information:

  ```
  -opt_skew_delay_pre_cts true # bool, default=true

  true
  ```

- The following command returns the current settings for all the `setUsefulSkewMode` parameters:

  ```
  getUsefulSkewMode
  ```

  The software returns the following information:

  ```
  -opt_skew_apply_delay_limits_to_full_flow false # bool, default=false
  -opt_skew_delay_pre_cts true                    # bool, default=true
  -opt_skew_macro_only false                      # bool, default=false
  -opt_skew_max_allowed_delay 1                   # float, default=1, min=0, max=1000
  -opt_skew_min_allowed_delay 0                   # float, default=0, min=0, max=1000
  -opt_skew_no_boundary false                     # bool, default=false

  {opt_skew_apply_delay_limits_to_full_flow false} {opt_skew_delay_pre_cts true} {opt_skew_macro_only false}
  {opt_skew_max_allowed_delay 1} {opt_skew_min_allowed_delay 0} {opt_skew_no_boundary false}
  ```

- The following command returns the current setting of the `setUsefulSkewMode -opt_skew_delay_pre_cts` parameter in Tcl list format only:

  ```
  getUsefulSkewMode -opt_skew_delay_pre_cts -quiet
  ```

The software returns the following information:

```
true
```

- The following command returns the current settings for all `setUsefulSkewMode` parameters in Tcl list format only:

  ```
  getUsefulSkewMode -quiet
  ```

  The software returns the following information:

  ```
  {opt_skew_apply_delay_limits_to_full_flow false} {opt_skew_delay_pre_cts true} {opt_skew_macro_only false}
  {opt_skew_max_allowed_delay 1} {opt_skew_min_allowed_delay 0} {opt_skew_no_boundary false}
  ```

- The following command returns the current non-default settings of the `setUsefulSkewMode` command:

  ```
  getUsefulSkewMode -nonDefault
  -opt_skew_macro_only true              # bool, default=false, user setting

  {opt_skew_macro_only true}
  ```

Related Information

- `setOptMode`
- `setUsefulSkewMode`
- `skewClock`
- The following topic in the *Innovus User Guide*
  - Optimizing Timing using Useful Skew

loadFootPrint

```
loadFootPrint
[-help]
-infile fileName
```

Loads a footprint file to update the footprint information, which is obtained from the timing library file during design import. Loading a footprint file takes precedence over what is currently defined in the Innovus session. That is, after a footprint file is loaded for a session, only the cells in the footprint file are considered. This feature lets you run individual timing optimization commands with a limited set of cell footprints.

When the command is run successfully, the software gives the following message:

```
----------------------------------------
Clearing footprints for all libraries
Loading footprints for all corners.
----------------------------------------
```

Parameters

`-help`	Outputs a brief description that includes type and default information for each `loadFootPrint` parameter. For a detailed description of the command and all of its parameters, use the man command: `man loadFootPrint`.
`-infile` *`fileName`*	Specifies the name of the footprint file.

Examples

- The following command loads the footprint file `INV.fpt`:

  ```
  loadFootPrint -infile INV.fpt
  ```

  After loading the footprint file, the Innovus software ignores the previous footprint information.
- When the `loadFootPrint` command reads the output from the `reportFootPrint` command, the `loadFootPrint` command ignores all other fields except the library name, cell name, and the footprint name. The footprint file format for the `loadFootprint` command is as follows:

  ```
  Library:library_name
  cell_name footprint_name
  cell_name footprint_name
  ```

  For example:

  ```
  Library:stdcellLib
  xr03d1 xr03_5
  xr03d2 xr03_5
  ```

Related Information

- `checkFootPrint`
- `setBufFootPrint`
- `setInvFootPrint`
- `setDelayFootPrint`

- The following topic in the *Innovus User Guide*
  - Using Cell Footprints in Timing Optimization

optDesign

```
optDesign
[-help]
[-drv]
[-excludeNets fileName]
[-expandedViews]
[-hold [-holdVioData fileName]]
[-idealClock]
[-incr]
[-noEcoRoute]
[-outDir directoryName]
[-preCTS | -postCTS | -postRoute]
[-prefix fileNamePrefix]
[-selectedNets fileName]
[-selectedTerms fileName]
[-setup]
[-targeted]
[-timingDebugReport]
[-useTransitionFiles]
```

Performs timing optimization before or after the clock tree is built, or after routing and generates timing reports. You can optimize with or without useful skew.

In multi-mode, multi-corner mode, `optDesign` optimizes all analysis views concurrently.

Performs the following optimizations:

- Corrects design rule violations.
- Reduces total negative slack.
- During the initial pass, optimizes setup time by working on the design's worst paths until the slack cannot be further improved, then, if the worst slack is not on a register-to-register path, optimizes the register-to-register paths on the second pass.
- Corrects hold time violations (optional).
- Optimizes useful skew (optional).
- Reclaims area (optional).
- Optimizes leakage and or/dynamic power (optional).
- Merges flops to multi-bit FFs, or split the multi-bit FFs (optional).

Uses some or all of the following techniques, depending on the design stage (that is, before or after CTS, or after routing) and the specified parameters:

- Adds buffers.
- Resizes gates.
- Restructures the netlist.
- Remaps logic.
- Swaps pins.
- Deletes buffers.
- Moves instances.
- Applies useful skew.
- Applies layer promotion and/or NDR assertions.
- Applies flip-flop merge and split.

Sets the minimum recommended and appropriate set of parameters for the following modes:

- `setAnalysisMode`
- `setRouteMode`
- `setOptMode`
- `setExtractRCMode`

Tip: You can set additional parameters by specifying parameters for the appropriate `set*Mode` command before using `optDesign`.

Before running `optDesign`, complete the following steps:

- Generate the default and detailed extraction scale factors by using the following command:
  - `generateRCFactor`
- Set input transitions for the high fanout nets for delay calculation to use on high fanout networks.

Note: The `simplifyNetlist` parameter is allowed to remove unnecessary or redundant level shifter/isolation instances although those would be internally set as `dont_touch` by `optDesign`.

Parameters

Note: The "optDesign Parameter Matrix" section shows which parameters can be used together.

`-help`	Outputs a brief description that includes type and default information for each `optDesign` parameter. For a detailed description of the command and all of its parameters, use the man command: `man optDesign`
`-drv`	When specified, corrects `max_cap` and `max_tran` violations. To correct `fanout_load` violations, specify `setOptMode -opt_fix_fanout_load` before you specify `optDesign`. You cannot use this parameter if you specify `-incr`. When this parameter is specified, `optDesign` will fix only the DRVs and not the setup/hold timing violations.
`-excludeNets` *`fileName`*	
	Specifies the file that contains the hierarchical names of nets. Timing optimization excludes nets specified in the -excludeNets file. If a net appears in both `-excludeNets` and `-selectedNets`, it is excluded. Note: Excluded nets, especially those with large fanout, might still be buffered to fix Setup timing problems. Use the dont_touch option in the SDC constraints file to prevent optDesign from modifying selected nets. For Hold Optimization however, the nets specified will be excluded when optimizing the Hold Timing.
`-expandedViews`	When specified, detailed view-specific timing reports are generated on all active views at the end of optimization. A separate directory is created for each view. For example, if a design has two analysis views, `view1` and `view2`, then the output reports are generated in the `./timingReports/view1` and `./ timingReports/view2` directories. If the views were pruned during optimization, they will be restored at the end by using this parameter and a view-specific summary and data will be reported on all the active views in the design.

`-hold`	When specified, corrects the hold violations. This parameter cannot be used if you specify the`-preCTS` parameter. You can use this parameter with the `-selectedNets` or `-selectedTerms` options. This parameter creates a summary file in the output directory of the design. The file is named `prefix_hold.summary`. Use the `-prefix` parameter to specify the prefix. To report detailed debugging information, specify the `setOptMode -opt_verbose true` parameter. For more information, see `setOptMode`. **Note**: Even though the hold-timing violation fixing algorithm is setup-timing aware, some setup timing degradation may appear due to different route topology during the postCTS stage. In most cases, this can be overlooked because the violations will be easily fixed during the optDesign -postRoute flow step. **Note**: You need to use the `setFillerMode` command to identify the filler cells. The `optDesign` command will remove filler cells that have been identified with the `setFillerMode` command at the beginning of optimization and add them back at the end of optimization.
`-holdVioData` *`fileName`*	
	Generates the following report files for the violation data that remains after hold fixing: • *`fileName.txt`:* provides a list of the top 50 hold violation paths (text format) • *`fileName.csv`*: provides a list of the top 50 hold violation paths (csv format) • *`fileNameDetailed.txt`*: provides detailed information for the top 50 hold violation paths This parameter is used with the `-hold` paramater. **Example**: `optDesign -postRoute -hold -holdVioData` *`fileName`*
`-idealClock`	Performs timing optimization in postCTS or postRoute modes using ideal clocks instead of propagated clocks. Using ideal clocks enables you to optimize timing without building a clock tree first, or ignore the existing clock tree, to close ideal timing. If this parameter is not specified, the software assumes the clock tree has been built, and optimizes timing based on propagated clocks.
`-incr`	Performs setup optimization incrementally on the violated paths until they cannot be optimized any more. Both, WNS and TNS are optimized. Both, high-effort and low-effort path groups are optimized. Area reclaim may also take place if `-postRoute` is not specified. A setup recovery is called depending on the timing bump after the re-route. This parameter has the following limitations: • You cannot use it if you also specify `-drv` or `-hold`. • You must have already run `optDesign` on all path groups before running it with `-incr`. • When specified, this option disables power optimizations
`-noEcoRoute`	Prohibits the Innovus software from performing ECO routing with the NanoRoute® router during postRoute optimization.
`-outDir` *`directoryName`*	Specifies the directory where the software writes timing reports generated when this command runs. *Default*: `timingReports`
`-postCTS`	Performs timing optimization on a design whose clock tree has been created. By default, `-postCTS` repairs design rule violations and setup violations. If the worst negative slack does not occur on a register-to-register path, `-postCTS` performs an additional optimization pass on the register-to-register critical paths when clock domains are not set. It does not repair maximum fanout design rule violations unless you first specify `setOptMode -opt_fix_fanout_load`.

`-postRoute`	Performs timing optimization on a design whose routing is complete. By default, `-postRoute` repairs design rule violations, glitch violations and setup violations on Base & SI Delay. If the worst negative slack does not occur on a register-to-register path, `-postRoute` performs an additional optimization pass on the register-to-register critical paths when clock domains are not set. Maximum fanout design rule violations are not repaired by default. To do this, first specify `setOptMode -opt_fix_fanout_load`. The `optDesign -postRoute` command provides accurate results when Innovus runs using the footprintless flow methodology. By default, Innovus creates an internal footprint structure based on cell functionality. This methodology does not require you to specify a footprint file. For library database (LDB) conversion, this is the recommended methodology since all of it is automated. For more information, see the "Using the Footprintless Flow" section in the *Innovus User Guide*. **Note**: The clock DRV fixing performed by running `optDesign -postRoute` and by the `ccopt_pro` command do not work when “signoff” effort level for extraction is specified using `setExtractRCMode -effortLevel signoff`. The “signoff” effort level for extraction should be used only for ECO flows, after post-route optimization. To use “`signoff`” effort level inside `optDesign -postRoute` without fixing clock DRV’s, you can turn off CCOpt PRO by setting the `-opt_post_route_fix_clock_drv` parameter of the `setOptMode` command to `false`.
`-preCTS`	Performs timing optimization on the placed design, before the clock tree is built. By default, `-preCTS` repairs design rule violations and setup violations. If the worst negative slack does not occur on a register-to-register path, `-preCTS` performs an additional optimization pass on the register-to-register critical paths when clock domains are not set. Maximum fanout design rule violations are not repaired by default. To do this, first specify `setOptMode -opt_fix_fanout_load`. **Note**: Before optimizing a design in preCTS mode, you must break all timing loops by disabling arcs in the constraint file. If you do not disable the arcs, the software cannot make a valid comparison of WNS between two different runs since it might not break the loops at the same point each time. To disable the arcs, use the `set_disable_timing` command.
`-prefix` *`fileNamePrefix`*	
	Specifies a prefix for `optDesign` report file names. *Default:* *`DesignName_DesignStage`*, where *`DesignStage`* is `preCTS`, `postCTS`, or `postRoute`.
`-selectedNets` *`fileName`*	
	Specifies the file that contains the hierarchical names of nets to consider for design-rule violation repair or hold-timing violations fixing. Timing optimization excludes all other nets. If a net appears in both `-excludeNets` and `-selectedNets`, it is excluded. This option is only accepted when either `-drv` or `-hold` is selected. **Note**: The `-selectedNets` parameter forces `optDesign` to work on all the selected nets, except the ones set using dont_touch.
`-selectedTerms` *`fileName`*	
	Specifies the file that contains the hierarchical names of instance terminals to consider for timing optimization. Only nets connected to those terminals are optimized; for example, during hold fixing, delay cells are added only on the nets driving the terminals specified in the file.
`-setup`	Corrects setup violations for the postRoute optimization flow. When specified along with the `-hold` parameter, a combined setup and hold optimization is performed. For example, `optDesign -postRoute -setup -hold` **Note**: This parameter is only applicable to the postRoute optimization flow and should be used along with the `-hold` parameter.

`-targeted`	This parameter is used only along with the `-postRoute` parameter. Specifying this parameter invokes target-based optimization (TBOpt), which is a fine grain local optimization functionality. This will target critical nodes at the postRoute stage and optimize these by giving worst negative slack (WNS), area, power, max transition, and signal integrity (SI) glitch improvements with minimum disturbance to NanoRoute. This parameter can be used optionally along with the `setOptMode -opt_target_based_opt_file` *`filename`* and `setOptMode -opt_target_based_opt_file_only` parameters. **Note**: Area, power, max transition, and SI glitch targeting can only be done using the above settings. For more information on the syntax of a sample TBOpt file, see the `setOptMode -opt_target_based_opt_file` documentation. Target-based optimization is best used after postRoute optimization is already completed and there are still some violating paths or nodes that might need further improvement.
`-timingDebugReport`	When specified, generates a machine-readable timing debug report after completing optimization. This report is used for debugging timing results using the timing debug feature in Innovus. **Note**: You can select the number of paths that should be reported per path group in the final summary reports by specifying the `setOptMode -opt_time_design_num_paths` parameter. The default number of paths is 50.
`-useTransitionFiles`	Uses the transition file specified by the `readTransitionFile` command for transition violation fixing during postRoute SI optimization.

optDesign Parameter Matrix

The table below shows the compatibility of `optDesign` parameters.

optDesign	-pre CTS	-post CTS	-post Route	-drv	-setup	-hold	-ideal Clock	-incr	-noECORoute
-preCTS	NA	-	-	+	-	-	-	+	-
-postCTS	-	NA	-	+	-	+	+	+	+
-postRoute	-	-	NA	+	+	+	+	+	+
-drv	+	+	+	NA	-	-	+	-	+
-setup	-	-	+	-	NA	+	+	-	+
-hold	-	+	+	-	+	NA	+	-	+
-idealClock	-	+	+	+	+	+	NA	+	+
-incr	+	+	+	-	-	-	+	NA	+
-noECORoute	-	+	+	+	+	+	+	+	NA

Legend

+	You can use this parameter in the row with the parameter in the column.
-	You cannot use this parameter in the row with the parameter in the column.
NA	The parameters in the row and column are identical.

Examples

PreCTS Optimization

- To optimize timing for first time, use the following command:
  ```
  optDesign -preCTS
  ```
- To repair design rule violations only, use the following command:
  ```
  optDesign -preCTS -drv
  ```
- To run incremental setup-only optimization, use the following command:
  ```
  optDesign -preCTS -incr
  ```
  The `optDesign` command performs gate resizing and global buffer insertion, but does not perform netlist restructuring or design rule violation repair.
- To run optimization with useful skew, use the following commands:
  ```
  setOptMode -opt_skew true -opt_skew_pre_cts true
  optDesign -preCTS
  ```
- To run optimization on specific path groups, use the following commands:
  ```
  group_path -name path_group_name -from from_list -to to_list -through through_list
  setPathGroupOptions  ..
  optDesign -preCTS
  ```
  Or you can use:
  ```
  createBasicPathGroups
  setPathGroupOptions ..
  optDesign -preCTS
  ```
  For example, to run optimization on reg2reg & reg2gate paths only, use the following commands:
  ```
  createBasicPathGroups -expanded
  setPathGroupOptions reg2out -targetSlack -1000.0
  setPathGroupOptions in2reg -targetSlack -1000.0
  ```

```
setPathGroupOptions in2out -targetSlack -1000.0
optDesign -preCTS
```

Note: If you specify a path group, `optDesign` preserves the path group setting. In the previous example, subsequent commands may affect the reg2reg & reg2cgate path groups only. To set new groups, specify `reset_path_group -all`, followed by the definition of the groups you want to use.
For example, to reset the path group to a specific group of paths, use the following commands after you run `optDesign`:

```
reset_path_group -all
group_path -name pathgroup1 -through an1/Z -through an2/Z -through an3/Z
```

To just specify all path groups again, use the following command:

```
reset_path_group -all
```

- To reclaim area during optimization, use the following commands:

```
setOptMode -opt_area_recovery true
optDesign -preCTS
```

- To run incremental optimization with useful skew, use the following commands:

```
setOptMode -opt_skew true
optDesign -preCTS -incr
```

- To run an incremental optimization including reclaiming area, use the following commands:

```
optDesign -preCTS
setOptMode -opt_area_recovery true
optDesign -preCTS -incr
```

PostCTS Optimization

- To optimize timing after the clock tree is built, use the following commands:

```
optDesign -postCTS
```

The `optDesign` command corrects design rule violations, then setup violations.

Note: The software does not correct fanout violations in `-postCTS` and `-postRoute` modes. To repair fanout violations, specify the following parameter before you run `optDesign`:

```
setOptMode -opt_fix_fanout_load true
```

- To repair setup and hold violations, use the following commands:

```
optDesign -postCTS
optDesign -postCTS -hold
```

- To repair design rule violations only, use the following command:

```
optDesign -postCTS -drv
```

- To repair hold violations only, use the following command:

```
optDesign -postCTS -hold
```

- To run optimization on specific path groups, use the following commands:

```
group_path -name path_group_name -from from_list -to to_list -through through_list
setPathGroupOptions  ..
optDesign -postCTS
```

Or you can use:

```
createBasicPathGroups
setPathGroupOptions ..
optDesign -postCTS
```

For example, to run optimization on reg2reg & reg2gate paths only, use the following commands:

```
createBasicPathGroups -expanded
```

```
setPathGroupOptions reg2out -targetSlack -1000.0
setPathGroupOptions in2reg -targetSlack -1000.0
setPathGroupOptions in2out -targetSlack -1000.0
optDesign -postCTS
```

- To reclaim area during optimization, use the following commands:

  ```
  setOptMode -opt_area_recovery true
  optDesign -postCTS
  ```

- To take advantage of useful skew when optimizing timing in postCTS mode, use the following commands:

  ```
  setOptMode -opt_skew true
  optDesign -postCTS
  ```

  If you specify `-noECORoute`, the software uses early global routing to estimate clock delays.

- To run postCTS optimization if your design has a clock mesh, use the following commands:

  ```
  setOptMode -opt_skew false
  optDesign -postCTS
  ```

- To run incremental optimization on specific path groups, use the following commands:

  ```
  group_path -name path_group_name -from from_list -to to_list -through through_list
  setPathGroupOptions  ..
  optDesign -postCTS -incr
  ```

  For example, to run incremental optimization reg2reg & reg2gate paths only, use the following commands:

  ```
  createBasicPathGroups -expanded
  setPathGroupOptions reg2out -targetSlack -1000.0
  setPathGroupOptions in2reg -targetSlack -1000.0
  setPathGroupOptions in2out -targetSlack -1000.0
  optDesign -postCTS -incr
  ```

- To optimize setup time incrementally and reduce area, use the following commands:

  ```
  setOptMode -opt_area_recovery true
  optDesign -postCTS -incr
  ```

- To take advantage of useful skew when optimizing timing in incremental postCTS mode, use the following commands:

  ```
  setOptMode -opt_skew true
  optDesign -postCTS -incr
  ```

- To run incremental postCTS optimization if your design has a clock mesh, use the following commands:

  ```
  setOptMode -opt_skew false
  optDesign -postCTS -incr
  ```

PostRoute Optimization

- To optimize base and SI timing after detailed routing (repair setup violations), use the following command:

  ```
  optDesign -postRoute
  ```

- To optimize only the base timing after detailed routing, use the following commands:

  ```
  setDelayCalMode -SIAware false
  optDesign -postRoute
  ```

- **Note:** The software does not correct fanout violations in `-postCTS` and `-postRoute` modes. To repair fanout violations, specify the following command before you run `optDesign`:

  ```
  setOptMode -opt_fix_fanout_load true
  ```

- To correct hold base and SI timing violations, specify the following commands:

  ```
  optDesign -postRoute -hold
  ```

- To correct setup and hold (both base and SI) violations, use the following commands:
  ```
  optDesign -postRoute
  optDesign -postRoute -hold
  ```
  The software repairs a hold violation only if it does not make setup slack worse than the setup target slack on a path. An alternative way is to run setup and hold fixing within one command, and this may reduce the number of ecoRoute(s):
  ```
  optDesign -postRoute -setup -hold
  ```
- To take clock path pessimism removal (CPPR) into consideration when running timing optimization, use the `setAnalysisMode` command before you run `optDesign`. For example:
  ```
  setTimingDerate -max -clock -early 0.8 -late 1.2
  setTimingDerate -min -clock -early 0.8 -late 1.2
  setAnalysisMode -analysisType onChipVariation -cppr both
  optDesign -postRoute
  ```
- To perform a fine grain target-based optimization at the postRoute stage, use the following command to identify the timing bottleneck nodes automatically and optimize them:
  ```
  optDesign -postRoute -targeted
  ```
- To perform a fine grain target based optimization at the postRoute stage on the nodes provided by the TBOpt file, `tb_opt.txt` as well as any other timing nodes that the code identifies as being critical, use the following ocmmands:
  ```
  setOptMode -opt_target_based_opt_file tb_opt.txt
  optDesign -postRoute -targeted
  ```
- To perform a fine grain target-based optimization at the postRoute stage on the nodes provided by the TBOpt file, `tb_opt.txt` only, use the following commands:
  ```
  setOptMode -opt_target_based_opt_file tb_opt.txt -opt_target_based_opt_file_only true
  optDesign -postRoute -targeted
  ```

Related Information

- `createBasicPathGroups`
- `createInterfaceLogic`
- `readTransitionFile`
- `setAnalysisMode`
- `setDelayCalMode`
- `set_disable_timing`
- `setExtractRCMode`
- `setNanoRouteMode`
- `setOptMode`
- `setRouteMode`
- `setSIMode`
- `setUsefulSkewMode`
- `timeDesign`
- Optimizing Timing chapter in the *Innovus User Guide*

optPower

```
optPower
[-help]
[-allowResizing]
[-effortLevel {high}]
[-force]
[-preCts | -postCts | -postRoute]
[-prePlace]
[-seqOnly]
```

Optimizes the power consumption of the design by swapping gates for those with lower power, or deleting buffers/inverters without degrading timing. This works in conjunction with the `setOptMode -opt_leakage_to_dynamic_ratio` command to determine the type of optimization being performed - leakage, dynamic, or a combination of both.

Since the default for `setOptMode -opt_leakage_to_dynamic_ratio` parameter is 1.0, `optPower` will perform leakage optimization only by default using LEF-safe optimization (if no other switch is set). It is vital to set up the design correctly before starting power optimization. For more information, see the Optimizing Power During optDesign section of the Optimizing Timing chapter in the Innovus User Guide.

The setup steps include the following:

- Setting the correct power views for both leakage and dynamic optimization
- Ensuring that the libraries contain `cell_leakage_power` information
- Ensuring that the switching activity has been correctly specified either through `read_activity_file -format { VCD | TCF | SAF }` file_name or the `set_default_switching_activity` command

Parameters

`-help`	Outputs a brief description that includes type and default information for each `optPower` parameter. For a detailed description of the command and all of its parameters, use the man command: `man optPower`.
`-allowResizing`	Specifies that optimization uses resizing for power reclaim and, if needed, for recovery. Since the cells will be of different physical footprint, placement refinement and routing ECO will be used. The DRV and timing recovery steps will also be performed, as required. These extra steps will result in an increase in runtime. As this is a more aggressive optimization, the power savings should be superior to the default `optPower` values, but this may cause timing degradations in some cases. When this parameter is set, the `setOptMode -opt_post_route_check_antenna_rules true` setting will be ignored and the software will give a warning to that effect.
`-effortLevel {high}`	Specifies the effort level as `high`. When this parameter is specified, power optimization uses aggressive reclaim. The software performs a timing recovery step and there is a large increase in runtime on an average as compared to when regular power optimization is performed.
`-force`	Resizes each instance to a cell with the lowest leakage power, disregarding design rule violations and optimum timing. This only works when `setOptMode -opt_leakage_to_dynamic_ratio` is set to `1.0`.
`-preCts`	Performs power optimization on the placed design, before the clock trees are created.
`-postCts`	Performs power optimization on a design whose clock trees have already been created.
`-prePlace`	When specified, performs power optimization on an unplaced design. This parameter must be used with the `-force` parameter.
`-postRoute`	Performs power optimization on a design whose routing is complete.
`-seqOnly`	Specifies that dynamic power optimization should be done on sequential cells only. This option works with the `-force` parameter in all modes; `preCts`, `prePlace`, `postCts`, and `postRoute`.

Examples

- Use the following commands to optimize the design targeting total power at the preCts stage:
  ```
  setOptMode -opt_leakage_to_dynamic_ratio 0.5
  optPower -preCts
  ```
- Use the following command to optimize the design, targeting leakage power only at the postCts stage (assuming that `setOptMode -opt_leakage_to_dynamic_ratio` has been untouched or has been set to the default value of 1.0):
  ```
  optPower -postCts
  ```
- Use the following commands to optimize dynamic power only at the postRoute stage:
  ```
  setOptMode -opt_leakage_to_dynamic_ratio 0.0
  optPower -postRoute
  ```
- Use the following commands to perform resizing for total power optimization in the design at the preCts stage:
  ```
  setOptMode -opt_leakage_to_dynamic_ratio 0.5
  optPower -preCts -allowResizing
  ```
- Use the following commands to perform aggressive dynamic power optimization in the design at the postCts stage:
  ```
  setOptMode -opt_leakage_to_dynamic_ratio 0.0
  optPower -postCts -effortLevel high
  ```
- Use the following commands to optimize leakage power by forcing all cells to their lowest leakage equivalent, disregarding DRV and timing degradations at the preCts stage:

```
setOptMode -opt_leakage_to_dynamic_ratio 1.0
optPower -preCts -force
```

- Use the following commands to optimize dynamic power by swapping sequential cells only in the design at the preCts stage:
  ```
  setOptMode -opt_leakage_to_dynamic_ratio 0.0
  optPower -preCts -seqOnly
  ```
- Use the following commands to allow the tool to swap gates, including the gates with negative slack, up to the current negative slack at the postRoute stage:
  ```
  setOptMode -opt_setup_target_slack current_negative_slack
  optPower -postRoute
  ```
- To prevent power optimization from working on near critical paths, you can set a different target to that of the timing optimization. A typical example is shown below where power optimization will not work on paths less than `0.1ns` but the target for timing will remain `0.0` :
  ```
  setOptMode -opt_setup_target_slack 0.0
  setOptMode -opt_area_recovery_setup_target_slack 0.1
  setOptMode -opt_leakage_to_dynamic_ratio 0.5
  optPower -preCts
  ```

Related Information

- `read_activity_file`
- `set_default_switching_activity`
- `setOptMode`
- The following topic in the *Innovus User Guide*
  - Optimizing Power

optVirtual

```
  optVirtual
[-help]
[-outDir dirName]
[-timingReports]
```

This command is based on the gigaOpt infrastructure and it provides prototyping capability at several steps of the hierarchical flow to specify and refine the top-level timing constraints and floorplan. It performs, at preRoute stages, fast virtual optimization using buffering and driver resizing. This command updates the timing graph without modifying the netlist. Using gigaOpt infrastructure, the command has the following capabilities:

- Provides a good QoR (AAE DC, gigaOpt global optimizer, and DRV-based)
- Run in multi-threaded mode
- Supports masters and clones

Parameters

`-help`	Outputs a brief description that includes type and default information for each `optVirtual` parameter. For a detailed description of the command and all of its parameters, use the man command: `man optVirtual`.
`-outDir` *dirName*	Specifies the name of the directory where the software writes timing reports. This parameter is used only when the `-timingReports` parameter is specified
`-timingReports`	Generates timing reports that reflect virtual optimization.

reclaimArea

```
reclaimArea
[-help]
[-noDeclone]
[-noDeleteBuffer]
[-noDownsize]
[-maintainHold]
```

Creates space in the design by downsizing and deleting buffers, without worsening the slack or increasing the design rule violations.

Note: Cadence recommends using the `optDesign` command to correct timing violations during timing optimization instead of using this command. For information on using `optDesign`, see Optimizing Timing in the *Innovus User Guide*.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `reclaimArea` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reclaimArea`
`-noDeclone`	Stops Innovus from decloning instances to reclaim area. By default, the software declones instances to reclaim area.
`-noDeleteBuffer`	Prohibits Innovus from deleting buffers to reclaim area. *Default:* The software deletes buffers to reclaim area.
`-noDownsize`	Prohibits Innovus from downsizing instances to reclaim area. *Default:* The software downsizes instances to reclaim area.
`-maintainHold`	Runs the hold aware area reclaim. This is only supported after the design has been routed.

Example

By default area reduction is happening only on positive slack instances. To allow the tool to reclaim area on instances that have a negative slack, you can set a negative setup target slack:

```
setOptMode -opt_setup_target_slack current_negative_slack
reclaimArea
```

Related Command

- `optDesign`

reportAlwaysOnBuffer

```
reportAlwaysOnBuffer
[-help]
[-all]
[-powerDomain domainName]
[-verbose]
```

Reports the available always-on buffers and inverters per domain.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `reportAlwaysOnBuffer` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reportAlwaysOnBuffer`.
`-all`	Reports the always-on buffers and inverters for all domains. If you specify no parameter, even then it reports the always-on buffers and inverters for all domains.
`-powerDomain` *`domainName`*	Reports the always-on buffers and inverters for the specified domain. **Note**: Parameters `-all` and `-powerDomain` are mutually exclusive.
`-verbose`	Reports the possible reason why a domain does not have always-on buffers and inverters. It can also report the always-on cell list per domain if the always-on cells are available for the domain.

Examples

- The following command displays all the available always-on buffers and inverters for each power domain.
  ```
  reportAlwaysOnBuffer
  ```
  ```
  Always on buffers found for each power domain:
        PowerDomain "TDSPCore" (pd tag = "1") has 1 always on buffer(s) to use
        PowerDomain "PLL" (pd tag = "2") has 1 always on buffer(s) to use
        PowerDomain "AO" (pd tag = "3") has 1 always on buffer(s) to use
  ```
- The following command reports the always-on buffers and inverters for the `PD1` domain:
  ```
  reportAlwaysOnBuffer -powerDomain PD1
  ```
  The software displays the following message:
  ```
  PowerDomain "PD1" (pd tag = "1") has 2 always on buffer(s) to use
  ```

Related Commands

- `optDesign`
- `setOptMode`

reportCapViolation

```
reportCapViolation
[-help]
[-all | -noGlobalNets]
[-max]
[-min]
[-outfile fileName]
[-selNetFile fileName]
[-excNetFile fileName]
[-significant_digits integer]
[-useDrcMargin]
```

After timing analysis, reports nets that exceed the maximum capacitance constraints in the timing library and timing constraints file. Run this command after running timing analysis (`timeDesign`) or timing optimization (`optDesign`).

Parameters

`-help`	Outputs a brief description that includes type and default information for each `reportCapViolation` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reportCapViolation`
`-all`	Reports capacitance violations for all nets in the design. This is the default option.
`-excNetFile fileName`	Specifies the file that contains the hierarchical net names that are excluded from the capacitance violation report.
`-noGlobalNets`	Reports capacitance violations on all nets except for power and ground nets. **Note**: The parameters, `-all` and `-noGlobalNets` are mutually exclusive.
`-max`	Reports only maximum capacitance violations. This is the default option.
`-min`	Reports only minimum capacitance violations. **Note**: The parameters, `-min` and `-max` are mutually exclusive.
`-outfile fileName`	Specifies the name of the file to which the results are written.
`-selNetFile fileName`	Specifies the file that contains the hierarchical net names for the report. Only these net names are considered for inclusion in the capacitance violation report. **Note**: The parameters, `-selNetFile` and `-excNetFile` are not mutually exclusive. If both parameters are specified, the `-excNetFile` parameter overides the `-selNetFile` parameter.
`-significant_digits integer`	Allows the user to specify the number of significant digits. *Default*: 3
`-useDrcMargin`	Generates the report using the margin value that you specify using the `-drcMargin` parameter of the `setOptMode` command.

Example

The following command reports nets that exceed the maximum capacitance constraints in the report file `maxcap.report`:

```
reportCapViolation -outfile maxCap.report
```

Related Commands

- optDesign
- reportFanoutViolation
- reportLengthViolation
- reportTranViolation
- setOptMode

reportCritInstance

```
reportCritInstance
[-help]
-outfile fileName
[-view viewName]
[-targetSlack slackValue]
```

Reports instances in critical paths where the slack is less than the target slack.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `reportCritInstance` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reportCritinstance`.
`-outfile` *`fileName`*	Specifies the report output file. **Note:** See Examples, which show the file format.
`-targetSlack` *`slackValue`*	Reports paths with slack less than the specified *`slackValue`*. Specify the value in nanoseconds (ns). *Default:* 0
`-view` *`viewName`*	Generates the report based on the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.

Command Order

Use this command after the timing graph is built by `timeDesign/optDesign`.

Examples

- The following command generates an output file named `inst.rpt`, which includes all the instance names and cell names. The instances are on the critical paths whose slack is less than 0 ns.
  ```
  reportCritInstance -outfile inst.rpt
  ```
- The following command generates an output file named `inst.rpt2`, which includes all the instance names and the cell names. The instances are on critical paths whose slack is less than `-0.3` ns.
  ```
  reportCritInstance -outfile inst.rpt2 -targetSlack -0.3
  ```
- The following shows the file format for `reportCritInstance`:
  ```
  Instance name:Cell name
  SUBCHIP1/XBUS_IO_REG1/XPMT12_0INV1/XPMTINV1_WFDO02XINV: IV170
  SUBCHIP1/XBUS_IO_REG1/XPMT12_0INV1/XPMTINV1_WFDO07XINV: IV170
  SUBCHIP1/XBUS_IO_REG1/XPMT12_0INV1/XPMTINV1_WFDO06XINV: IV170
  SUBCHIP1/XBUS_IO_REG1/XPMT12_0INV2/XPMTINV2_WFDO07XINV: IV170
  SUBCHIP1/XBUS_IO_REG1/XPMT12_0INV2/XPMTINV2_WFDO06XINV: IV170
  SUBCHIP1/XBUS_IO_REG1/XPMT12_0INV2/XPMTINV2_WFDO02XINV: IV17
  ```

Related Commands

- optDesign
- setOptMode
- timeDesign

reportCritNet

```
reportCritNet
[-help]
[-nrCritNets integer]
[-outfile fileName]
[-targetSlack double]
[-view viewName]
```

Generates a file containing a list of nets which have critical slack for the currently specified (setup or hold) timing analysis mode.

Parameters

`-help`	Prints a brief description that includes type and default information for each `reportCritNet` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reportCritNet`
`-nrCritNets integer`	Restricts the number of critical nets reported to the first *`integer`* most critical nets.
`-outfile fileName`	Specifies the name of the report file.
`-targetSlack double`	Reports nets with slack less than the specified slack value.
`-view viewName`	Generates the report based on the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.

Command Order

Use this command after the timing graph is built.

Example

The following command generates a report named `critnet` that lists critical nets:

```
reportCritNet -outfile critnet
```

Related Commands

- optDesign
- setOptMode
- timeDesign

reportCritTerm

```
reportCritTerm
[-help]
-outfile fileName
```

Performs slack analysis and writes the names of all critical terminals to the specified file. As a result, deterioration of setup slack can be avoided while the Innovus software repairs hold time violations.

Parameters

`-help`	Prints a brief description that includes type and default information for each `reportCritTerm` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reportCritTerm`
`-outfile` *`fileName`*	Specifies the name of report file.

Command Order

Use this command after the timing graph is built by `timeDesign/optDesign`.

Example

- The following commands write the setup critical terminals into a file called `critTerm`:

  ```
  setAnalysisMode -checkType setup
  reportCritTerm -outfile critTerm
  ```

Related Commands

- optDesign
- setOptMode
- timeDesign

reportFanoutViolation

```
reportFanoutViolation
[-help]
[-all | -noGlobalNets]
[-excNetFile fileName]
[-min]
[-max]
[-outfile fileName]
[-selNetFile fileName]
[-significant_digits integer]
[-useDrcMargin]
```

After timing analysis, reports all the pins that exceed the fanout constraints in the timing library and timing constraints file. Run this command after running timing analysis (`timeDesign`) or timing optimization (`optDesign`).

Parameters

`-help`	Prints a brief description that includes type and default information for each `reportFanoutViolation` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reportFanoutViolation`
`-all`	Reports fanout violations on all nets in the design.
`-excNetFile fileName`	
	Specifies a file that lists the names of hierarchical nets to exclude from the report.
`-max`	Reports violations of maximum fanout constraints.
`-min`	Reports violations of minimum fanout constraints. **Note**: Parameters `-min` and `-max` are mutually exclusive.
`-noGlobalNets`	Reports fanout violations on all nets except for power and ground nets. **Note**: Parameters `-all` and `-noGlobalNets` are mutually exclusive.
`-outfile fileName`	Specifies the report file.
`-selNetFile fileName`	
	Specifies a file that lists the names of hierarchical nets that are considered for the report. Only these net names are considered for inclusion in the violation report. **Note**: The parameters, `-selNetFile` and `-excNetFile` are not mutually exclusive. If both parameters are specified, the `-excNetFile` parameter overides the `-selNetFile` parameter.
`-significant_digits integer`	
	Allows the user to specify the number of significant digits. *Default*: 3
`-useDrcMargin`	Generates a report using the margin value that you specify with the `-opt_drv_margin` parameter of the `setOptMode` command.

Example

The following command reports the pins that exceed the maximum fanout constraints in a file named `maxFanout.report`:

```
reportFanoutViolation -outfile maxFanout.report
```

Related Commands

- optDesign
- reportLengthViolation
- reportCapViolation
- reportTranViolation
- setOptMode
- timeDesign

reportFootPrint

```
reportFootPrint
[-help]
[-outfile fileName]
[-dontTouchNUse]
[footprint]
```

Creates a file that contains footprint attribute information from the timing library. Use this file to check your design for missing footprint function names and timing information. You can edit the output file and use it to load the correct footprint information.

Parameters

`-help`	Prints a brief description that includes type and default information for each `reportFootPrint` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reportFootPrint`
`-dontTouchNUse`	When this parameter is specified, cell footprints are reported for all cells, including those cells that have dont_touch or dont_use attribute. When this parameter is not specified, cell footprints are reported for only those cells that do not have dont_touch and dont_use attributes.
footprint	Specifies the footprint to report. If you do not specify this parameter, reports all footprints.
`-outfile` *fileName*	Specifies the footprint report file.

Command Order

Use this command after importing the design or after using the `loadFootPrint` command.

Examples

- The following command creates a footprint report file named `INV.rpt`:

  ```
  reportFootPrint -outfile INV.rpt
  ```

- The following output from the `reportFootPrint` command shows the format of the footprint file:

  ```
  # Format:
  # Library:<library name>
  # cell footprint drvResR drvResF functions

  # footPrint: INV nrCell: 1 Library: dp_ram128x32_4x1
  Library:stdCellLib
  inv_2 INV     1            110.983    89.427    x=(~a)
  ```

  Lines beginning with a # are comment lines, which are ignored when the footprint file is loaded. The header shows the meaning of each field:

  - `Library`: `stdCellLib` (with no spaces in the string) is the library where the footprint is defined.
  - `inv_2` is a footprint cell.
  - `INV` is the footprint name.

- `1` is the number of functions.
- `110.983` is the average rise driving resistance.
- `89.427` is the average fall driving resistance.
- `x=(~a)` is the output function of the cell.

Related Information

- `checkFootPrint`
- `loadFootPrint`
- `setBufFootPrint`
- `setInvFootPrint`
- `setDelayFootPrint`
- The following topic in the *Innovus User Guide*
  - Using Cell Footprints in Timing Optimization

reportIgnoredNets

```
reportIgnoredNets
[-help]
-outfile fileName
```

Generates a report that lists the nets that are ignored by timing optimization (`optDesign`) or that might impact timing closure. You can use this command after loading a design with timing constraints.

The report lists the following types of nets:

- Nets that are ignored by timing optimization
- Nets that might impact timing closure but are not ignored

Nets that are ignored by timing optimization are listed below:

`dontTouchNets`	Nets set as don't touch by the `set_dont_touch` SDC constraint
`externalNets`	Nets connected to I/O pad or area I/O instances
`topLevelPotentialTriStateNets`	Nets connecting tristate drivers to top-level I/Os
`clockNets`	Nets marked `clock` by the timing engine
`specialNets`	Nets that have the attribute `specialnet`
`pwrGndNets`	Nets that have the attribute `power/ground`
`analogNets`	Nets that have the attribute `analog`
`noDriverNets`	Nets that do not have a driver
`Fixed Wires/Cover Wires`	Nets that have a `Fixed/Cover` wire attribute

Nets that might impact timing closure but are not ignored are listed below:

`multiDriverNets`	Nets that have multiple drivers

Parameters

`-help`	Prints a brief description that includes type and default information for each `reportIgnoredNets` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reportIgnoredNets`
`-outfile` *`fileName`*	Specifies a file name for the report.

Related Commands

- optDesign
- setOptMode

reportLengthViolation

```
reportLengthViolation
[-help]
[-all | -noGlobalNets]
[-selNetFile selNetFileName]
[-excNetFile excNetFileName]
[-histogram]
[-outfile fileName]
[-significant_digits integer]
```

Runs timing analysis and reports nets that exceed the maximum length constraint set by `setOptMode -opt_max_length` parameter.

Parameters

`-help`	Prints a brief description that includes type and default information for each `reportLengthViolation` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reportLengthViolation`
`-all`	Reports violations on all nets.
`-noGlobalNets`	Reports violations on all nets except for power, ground, and clock nets. **Note**: The parameters, `-all` and `-noGlobalNets` are mutually exclusive.
`-excNetFile` *excNetFileName*	Specifies a file that lists the names of hierarchical nets that are to be excluded from the report.
`-outfile` *fileName*	Specifies the report file name.
`-selNetFile` *selNetFileName*	Specifies a file that lists the names of hierarchical nets to consider in the report. Only violations on the nets listed in the file are considered in the report. **Note**: The parameters, `-selNetFile` and `-excNetFile` are not mutually exclusive. If both parameters are specified, the `-excNetFile` parameter overides the `-selNetFile` parameter.
`-histogram`	Generates a histogram in addition to the default reporting.
`-significant_digits` *integer*	Allows you to specify the number of significant digits. *Default*: 3

Related Commands

- `optDesign`
- `reportFanoutViolation`
- `reportCapViolation`
- `reportTranViolation`
- `setOptMode`

reportMultibit

```
reportMultibit
[-help]
[-bit bit_width]
[-outFile file_name]
[-cell cell_name | -library | -reasonNotMerged unmerged_reason | -notMergedSummary | -equiv]
[-cell cell_name | -type {comb | latch | flop | seq | iso | ls}]
[-hinst hinst_name | -library | -cell <cell name> | -equiv]
```

Reports all the multi-bit flip-flops that have been identified during loading the design. Use this command after all the libraries are loaded. When this command is specified with no parameters, a default summary report is printed. By default, the sequential cells will be reported. Both, the multi-bit percentages of flops that are merged, and a summary of flops that are not merged is printed.

There are three basic types of multibit: sequential, combo, and isolation/level shifter. This command supports specifying the type of multibit to report. The `-type` parameter is a global filter for all other options.

The default option is `seq`. If `-type` option is not specified then reporting will take place for sequential cells.

A set of options separated by a "`|`" in one "`[ ]`" listed below cannot be used together and the software will report an error in that case. Although options from different "`[ ]`" can be coupled together. The sets of mutually exclusive options are provided below.

- `[-cell <cell name> | -library | -reason_not_merged <unmerged reason> | -not_merged_summary | -equiv ]`
- `[-cell <cell name> | -type <type>]`
- `[-hinst <hinst> | -library | -cell <cell name> | -equiv ]`

Parameters

`-help`	Prints a brief description that includes type and default information for each `reportMultibit` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reportMultibit`.
`-bit bit_width`	Reports the multibit iso/ls cells for the specified bit width. The value *bit_width* can be an integer that represents the bit-width of the queried isolation/level shifter (iso/ls) cell. This parameter is required to be specified when reporting the multibit iso/ls cells. Example: To report 2-bit MB isolation (iso) cells in the design, use the following command: `reportMultibit -type iso -bit 2`
`-cell cell_name`	Reports the cell details and equivalence information of a single library cell.
`-equiv`	Reports equivalence summary of the library cells of the selected type.
`-hinst hinst_name`	Specifies the hinst on which the reporting is to be done. The default is the design.
`-library`	Reports the flip-flop/latch equivalence. This parameter honors the `-type` parameter setting.
`-notMergedSummary`	When specified, reports the summary of unmerged instances.
`-outFile file_name`	Specifies the name of the output file.
`-reasonNotMerged {Partition CellEquivalent Constraint ScanConnection Location Timing Power User Legalization Bus BankLabelRegister BankLabelPartition All}`	

	Reports the instances that could not be merged for the specified reason. Specify any of the reasons in the argument or specify `all` to report all instances along with the reason why they could not be merged. `Partition`: odd number of flops in one partition `CellEquivalent`: no usable equivalent multi-bit flip-flop cell `Constraint`: Constraints by sdc, rule, or user settings `ScanConnection`: no scan connection to other flops in the same group `Location`: too far away with other flops in the same group `Timing`: exceed critical path threshold or MBFF merging that degrades timing `Power`: all available multi-bit cells causing power degradation `User`: user set to be excluded in flop merge `Bus`: non-bus flops `All`: all the reasons of not merged
`-type {comb | latch | flop | seq | iso | ls}`	Reports for the specified type of cells. By default, only sequentials are reported. You can specify anyone of the following reports: `comb`: reports all multibit cells implementing a combinational function. For example, a cell implementing 3 ANDs – so, 6 inputs and 3 outputs `latch`: reports multi-bit latches in the design. `flop`: reports multi-bit flops. `seq`: reports all sequential cells. isolation/level shifter (`iso/ls`): reports multi-bit cells implementing a low power function. For example, as 8-bit level-shifter cell. *Default*: `seq`

Examples

Use the following command to print a default summary report. By default, the sequential cells will be reported..

```
reportMultibit
```

The software returns the following information:

```
================================================================================
Current Design Sequential Cells Statistics
================================================================================
+--------------------------------+----------+---------------+-------------+
|                                | Latches  | Flops         | All Seq Cells |
+--------------------------------+----------+---------------+-------------+
| Single-Bit Count               | 1157     | 101           | 1258          |
| Multi-Bit Count                | 0        | 2657          | 2657          |
|  2-Bit Count (Used-Bit %)      | 0        | 29(100.00)    | 29 (100.00)   |
|  4-Bit Count (Used-Bit %)      | 0        | 2628(100.00)  | 2628 (100.00) |
| Total Bit Count                | 1157     | 10671         | 11828         |
|  -Merged                       | 0        | 10570         | 10570         |
|  -Un-Merged                    | 1157     | 101           | 1258          |
| Total Cell Count               | 1157     | 2758          | 3915          |
| Total Cells With Unused Bits   | 0        | 0             | 0             |
| Total Unused Bits Count (%)    | 0 (0.00) | 0 (0.00)      | 0 (0.00)      |
| Bits per cell                  | 1.00     | 3.87          | 3.02          |
| Total Clock Pin Cap            | 675.69   | 1602.71       | 2278.40       |
| Multibit Conversion %          | 0.00     | 99.05         | 89.36         |
+--------------------------------+----------+---------------+-------------+
================================================================
```

```
Sequential Multibit Cells Usage Statistics
============================================================
+--------------------------+---------+-------+---------------+
|                          | Latches | Flops | All Seq Cells |
+--------------------------+---------+-------+---------------+
| Usable 2-bit cell        | 0       | 6     | 6             |
| Dont-use/touch 2-bit cell | 0       | 0     | 0             |
| Usable 4-bit cell        | 0       | 6     | 6             |
| Dont-use/touch 4-bit cell | 0       | 0     | 0             |
+--------------------------+---------+-------+---------------+


================================================
Sequential Multibit cells usage statistics
================================================
+------------------------+------+----------+-------+
| Seq_Mbit libcell       | FF/L | Bitwidth | Count |
+------------------------+------+----------+-------+
| HD40DG_LH_2SDFPQX4_P4  | FF   | 2        | 3     |
| HD40DG_LH_2SDFPRQX4_P4 | FF   | 2        | 26    |
| HD40QG_LH_4SDFPQX4_P4  | FF   | 4        | 2325  |
| HD40QG_LH_4SDFPRQX4_P4 | FF   | 4        | 303   |
+------------------------+------+----------+-------+
| Total                  |      |          |  2657 |

================================================================================
Rejection Reason Summary
================================================================================
+-----------------+------------+-----------------------------------------------+
| Category        | Rej. Count | Reason |
+-----------------+------------+-----------------------------------------------+
| Partition       | 19         | Odd number of flops in one partition          |
| Cell Equivalent | 7          | No usable equivalent multi-bit flip-flop cell
                                 (check 'Ignored single-bit flop-flop'
                                 in the 'Pre-merge design statistics')         |
| Constraint      | 71         | Constraints by sdc,rule or user settings
                                 (check 'Nonremovable single-bit flip-flop'
                                 in the 'Pre-merge design statistics')
+-----------------+------------+-----------------------------------------------+
```

When the `-reasonNotMerged` parameter is specified, the report lists the reasons and a detailed listing of each of the cells. By default, the sequential cells are reported.

```
reportMultibit -reasonNotMerged Partition
```

The software returns the following information:

```
Reason of not merging:     Partition    Odd number of flops in one partition
Unmerged Sequential instances (19):
-----------------------------------
ethreg1/MIIADDRESS_1/DataOut_reg[4]
ethreg1/MIICOMMAND0/DataOut_reg[0]
ethreg1/MIICOMMAND1/DataOut_reg[0]
ethreg1/MIICOMMAND2/DataOut_reg[0]
ethreg1/SetTxCIrq_reg
ethreg1/TXCTRL_2/DataOut_reg[0]
maccontrol1/receivecontrol1/ByteCnt_reg[4]
maccontrol1/receivecontrol1/Divider2_reg
macstatus1/DeferLatched_reg
macstatus1/ReceiveEnd_reg
miim1/shftrg/LinkFail_reg
```

```
rxethmac1/CrcHashGood_reg
rxethmac1/rxcounters1/DlyCrcCnt_reg[0]
rxethmac1/rxcounters1/IFGCounter_reg[4]
txethmac1/txstatem1/StateJam_q_reg
wishbone/RxBDReady_reg
wishbone/TxPointerRead_reg
wishbone/rx_fifo/cnt_reg[4]
wishbone/tx_fifo/cnt_reg[4]
```

Related Information

- ecoSplitFlop
- ecoSplitComplexFlop
- dumpMultiBitFlopMappingFile
- setOptMode
- setDesignMode
- Reporting the MBFFs in *Innovus User Guide*

reportPathGroupOptions

```
reportPathGroupOptions [-help]
```

Reports the values for each parameter (pathgroupName, effortLevel, slackAdjustment, and targetSlack) for each path group. If no path groups are created, the software issues a warning.

To report the names of the path groups, use the `report_path_groups` command.

Parameter

`-help`	Prints a brief description that includes type and default information for the `reportPathGroupOptions` command. For a detailed description of the command, use the `man` command: `man reportPathGroupOptions`

Example

Use the following set of commands to create the basic path groups, set the parameters, and report the values of these parameters:

```
createBasicPathGroups

Created reg2reg path group

Effort level <high> specified for reg2reg path_group

Created reg2cgate path group
Effort level <high> specified for reg2cgate path_group

setPathGroupOptions reg2reg -effortLevel low -skewingSlackConstraint 0.3 -view AV_HL_FUNC_MIN_RC1
setPathGroupOptions reg2cgate -effortLevel high -skewingSlackConstraint 0.5 -view AV_HL_FUNC_MAX_RC1
reportPathGroupOptions
```

The following report is generated by the software:

```
-------------------------------------------------------------------------------------------------
|Path Group|Effort|Adjustment(early/late)|Target Slack|weight|SkewingSlackCons|view               |
-------------------------------------------------------------------------------------------------
|reg2reg   | high | 0.1    /-0.2         | 0          |  2   |    300         |AV_HL_FUNC_MIN_RC1 |
|reg2cgate | high | 0.2    /0            | 0          |  1   |    500         |AV_HL_FUNC_MAX_RC1 |
```

For detailed descriptions of the above columns, see the table in `setPathGroupOptions`.

Related Commands

- `createBasicPathGroups`
- `resetPathGroupOptions`
- `setPathGroupOptions`

reportRouteTypeConstraints

```
reportRouteTypeConstraints
[-help]
[-file fileName]
[-svr]
[-svrOnly]
[-threshold value]
[-summary  | -detailed ] [-detailed  [-timing ]]
```

Reports the route type constraints that are being added on nets during the Innovus flow. It allows you to monitor the number of constraints, the preferred bottom layer used, the adherence of the different constraint type by the router, and the type of non-default rule (NDR) being used.

The command extracts information about the following groups of route type constraints:

- **INNOVUS**: includes all route types being added by the software and that are using a bottom preferred layer and the default rule.
- **INNOVUS_NDR**: includes all route types being added by the software and that are using an NDR. `setOptMode -opt_pre_route_ndr_aware` makes this type of constraint.
- **INNOVUS_CLK**: includes all route types of clock nets that are being added by the software and that are using default rule. `create_route_type` without `-non_default_rule` and ccopt_property `route_type` make this type of constraint.
- **INNOVUS_CLK_NDR**: include all route types of clock nets that are being added by the software and that are using an NDR. `create_route_type` with `-non_default_rule` and ccopt_property `route_type` make this type of constraint.
- **USER**: includes all route types being added by the user. This can be done using the `setAttribute` command.
- **USER_NDR**: includes all route types being added by the user and that are using an NDR. This can be done using the `setAttribute` command.

When via pillar report is enabled, using the `-svr`/`-svrOnly` parameters, the command checks via pillar assigned on data/clock nets and on the secondary PG nets. For sample reports, see Use Model Scenarios.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `reportRouteTypeConstraints` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man reportRouteTypeConstraints`
`-detailed`	Prints detailed information on each net with a route type constraint such as the wire length, the routing adherence, some timing information, and so on. **Note**: The parameters, `-detailed` and `-summary` are mutually exclusive.
`-file file`*`Name`*	
	Specifies the file name in which the results will be written. If this parameter is not specified, the information is written out in the console and the log file.
`-svr`	When specified, reports pins with stack via rule constraints.
`-svrOnly`	When specified, reports only those pins that have stack via rule constraints.
`-summary`	Prints a summary of the layer constraint usage per layer, per NDR, and per constraint type. **Note**: The parameters, `-detailed` and `-summary` are mutually exclusive.
`-threshold` *`value`*	Specifies the maximum routing adherence for a net to be included in detailed report. This parameter is specified along with the `-detailed` paramater. The default value is 80, which means only nets with adherence < 80 % will be included in detailed report. **Note**: It is recommended that you do not use the `-threshold` parameter along with the `-svrOnly` parameter because stack via rule has no “routing adherence ratio” metric.
`-timing`	When this parameter is specified, the report will display timing information on the nets such as slack, delay, and incremental delay while generating the detailed information.

Use Model Scenarios

The use model scenarios for reports including via pillar constraints applied on secondary PG pins are provided below.

Reporting stack via rule (including stack via rule used on secondary PG pins) and bottom layer constraint:

```
reportRouteTypeConstraints -svr
```

The following information is included in the summary report:

```
**INFO (OILAM) : Summary of current layer assignment usage:
        # num nets with bottom preferred layer constraint                  : 38225
        List NDR used                                                      : TrunkNDR LeafNDR
        List stack via rule used                                           :
        Num nets with incorrect stack via rule constraint applied on driver : 0
        Num nets with incorrect stack via rule constraint applied on sinks  : 0
        Num library cells with stack via rule defined                      : 0
        Num library cells with required stack via rule defined             : 0
        Num instances with pillar rule list associated to them             : 0
        List stack via rule used on second PG pins                         : VPPERF_M2_M4_1_3_1 VPPERF_M2_M4_1_2_1
        Num second PG pins with stack via rule constraint                  : 24785
```

Reporting only stack via rule

```
reportRouteTypeConstraints -svrOnly
```

The above command prints information about only stack via rule including the summary of via pillar rule applied on secondary PG pins.

```
**INFO (OILAM) : Summary number of stack via rule constraint applied on second PG pins:
        +---------------------+--------------+--------------+----------+--------------+
        | Stack via rule name | Routing Cost | # stack via rule applied               |
        +                     +              +--------------+----------+--------------+
        |                     |              | Not required | Required                |
        +                     +              +              +----------+--------------+
        |                     |              |              | cell pin | instance pin |
        +---------------------+--------------+--------------+----------+--------------+
        | VPPERF_M2_M4_1_3_1  | 104.0        | NA           | 435      | NA           |
        | VPPERF_M2_M4_1_2_1  | 88.0         | NA           | 24350    | NA           |
        +---------------------+--------------+--------------+----------+--------------+
        | Total               |              | 0            | 24785    | 0            |
        +---------------------+--------------+--------------+----------+--------------+
```

When no via pillar are found on 2nd PG pin, the table will be replaced by following line:

```
**INFO (OILAM) : Summary number of stack via rule constraint applied on second PG pins:
        No stack via rule found
```

When the `-detailed` parameter is specified while reporting via pillar usage, a new table is generated in the report with the list of secondary PG pins with a via pillar constraint. This is shown below.

```
**INFO (OILAM) : List of second PG pins with stack via rule constraint 'VPPERF_M2_M4_1_2_1':
        +-------+---------------------+--------------+-------------------------------------+
        | ID    | Stack via rule name | Rule type    | Instance PG pin name                |
        +-------+---------------------+--------------+-------------------------------------+
        | 1     | VPPERF_M2_M4_1_2_1  | Cell Required | FE_OCPC96413_FE_OFN15677_TC_13/TVDD |
        | 2     | VPPERF_M2_M4_1_2_1  | Cell Required | FE_OCPC96403_TC_447/TVDD            |
        | 3     | VPPERF_M2_M4_1_2_1  | Cell Required | FE_OFC93880_TC_1999/TVDD            |
        | 4     | VPPERF_M2_M4_1_2_1  | Cell Required | FE_OFC10210_TC_1612/TVDD            |
        | 5     | VPPERF_M2_M4_1_2_1  | Cell Required | FE_OFC10209_TC_1612/TVDD            |
        | 6     | VPPERF_M2_M4_1_2_1  | Cell Required | FE_OFC10208_TC_1379/TVDD            |
        | 7     | VPPERF_M2_M4_1_2_1  | Cell Required | FE_OFC10207_TC_1379/TVDD            |
        | 8     | VPPERF_M2_M4_1_2_1  | Cell Required | FE_OFC10206_TC_52/TVDD              |
        | 9     | VPPERF_M2_M4_1_2_1  | Cell Required | FE_OFC10204_TC_776/TVDD             |
        | 10    | VPPERF_M2_M4_1_2_1  | Cell Required | FE_OFC93879_FE_OFN6183_TC_483/TVDD  |
```

Examples

- The following command prints the summary for the route type constraints:
  `reportRouteTypeConstraints -summary`

  The report below summarizes the number of route type constraints per layer for each category.

```
**INFO (OILAM) : Detailed number of layer constraint usage:
+---------+---------+-------------+-------------+-----------------+------+----------+---------+-------------+
| Layer   | INNOVUS | INNOVUS_NDR | INNOVUS_CLK | INNOVUS_CLK_NDR | USER | USER_NDR | UNKNOWN | UNKNOWN_NDR |
+---------+---------+-------------+-------------+-----------------+------+----------+---------+-------------+
| M1(1)   | 0       | 0           | 0           | 0               | 0    | 0        | 0       | 0           |
| M2(2)   | 0       | 0           | 0           | 0               | 0    | 0        | 0       | 0           |
| M3(3)   | 0       | 0           | 1493        | 0               | 0    | 0        | 0       | 0           |
| M4(4)   | 0       | 16          | 0           | 0               | 0    | 0        | 0       | 0           |
| M5(5)   | 0       | 0           | 0           | 1339            | 0    | 0        | 0       | 0           |
| M6(6)   | 0       | 0           | 0           | 0               | 0    | 0        | 0       | 0           |
| M7(7)   | 166     | 20          | 0           | 0               | 0    | 0        | 0       | 0           |
| M8(8)   | 0       | 0           | 0           | 0               | 0    | 0        | 0       | 0           |
| M9(9)   | 25      | 0           | 0           | 0               | 0    | 0        | 0       | 0           |
| M10(10) | 0       | 0           | 0           | 0               | 0    | 0        | 0       | 0           |
| AP(11)  | 0       | 0           | 0           | 0               | 0    | 0        | 0       | 0           |
+---------+---------+-------------+-------------+-----------------+------+----------+---------+-------------+
| Total   | 191     | 36          | 1493        | 1339            | 0    | 0        | 0       | 0           |
+---------+---------+-------------+-------------+-----------------+------+----------+---------+-------------+
```

- The report below summarizes the number of route type constraints per NDR and for each category. The syntax used is *`categories_NDR`* `name`.

```
**INFO (OILAM) : Detailed number of layer constraint usage per NDR rule:
+---------+----------------------+----------------------+-----------------------------+
| Layer   | INNOVUS_NDR_BUSSPACE | INNOVUS_NDR_WIDEWIRE | INNOVUS_CLK_NDR_DOUBLESPACE |
+---------+----------------------+----------------------+-----------------------------+
| M1(1)   | 0                    | 0                    | 0                           |
| M2(2)   | 0                    | 0                    | 0                           |
| M3(3)   | 0                    | 0                    | 0                           |
| M4(4)   | 14                   | 2                    | 0                           |
| M5(5)   | 0                    | 0                    | 1339                        |
| M6(6)   | 0                    | 0                    | 0                           |
| M7(7)   | 0                    | 20                   | 0                           |
| M8(8)   | 0                    | 0                    | 0                           |
| M9(9)   | 0                    | 0                    | 0                           |
| M10(10) | 0                    | 0                    | 0                           |
| AP(11)  | 0                    | 0                    | 0                           |
+---------+----------------------+----------------------+-----------------------------+
| Total   | 14                   | 22                   | 1339                        |
+---------+----------------------+----------------------+-----------------------------+
```

- The following command prints the detailed route type constraints information:

  ```
  reportRouteTypeConstraints -detailed
  ```

  The report below summarizes the percentage of respect of the route type constraint (bottom preferred layer) by the current router. For example, in the report below, for the INNOVUS constraints between 90% and 100% respect of the constraint, there are 100.0% of the nets. While INNOVUS_CLK constraints between 90 - 100% respect of the constraint, there are 98.33% of the nets.

```
**INFO (OILAM) : Summary min layer respect by router:
+------------+---------+--------------+--------------+-------------+-------------------+-----------------+-------------+-------------------+
| Respect    | INNOVUS | INNOVUS M7(7) | INNOVUS_NDR | INNOVUS_NDR M4(4) | INNOVUS_NDR M7(7) | INNOVUS_CLK | INNOVUS_CLK M3(3) | INNOVUS_CLK_NDR |
+------------+---------+--------------+--------------+-------------+-------------------+-----------------+-------------+-------------------+
| 0 - 10 %   | 0.00 %  | 0.00 %       | 0.00 %       | 0.00 %      | 0.00 %            | 0.00 %          | 0.00 %      | 5.60 %            |
| 10 - 20 %  | 0.00 %  | 0.00 %       | 0.00 %       | 0.00 %      | 0.00 %            | 0.00 %          | 0.00 %      | 1.34 %            |
| 20 - 30 %  | 0.00 %  | 0.00 %       | 0.00 %       | 0.00 %      | 0.00 %            | 0.00 %          | 0.00 %      | 1.34 %            |
| 30 - 40 %  | 0.00 %  | 0.00 %       | 0.00 %       | 0.00 %      | 0.00 %            | 0.07 %          | 0.07 %      | 1.12 %            |
| 40 - 50 %  | 0.00 %  | 0.00 %       | 0.00 %       | 0.00 %      | 0.00 %            | 0.07 %          | 0.07 %      | 0.82 %            |
| 50 - 60 %  | 0.00 %  | 0.00 %       | 0.00 %       | 0.00 %      | 0.00 %            | 0.00 %          | 0.00 %      | 1.05 %            |
| 60 - 70 %  | 0.00 %  | 0.00 %       | 0.00 %       | 0.00 %      | 0.00 %            | 0.07 %          | 0.07 %      | 1.19 %            |
| 70 - 80 %  | 0.00 %  | 0.00 %       | 0.00 %       | 0.00 %      | 0.00 %            | 0.00 %          | 0.00 %      | 3.58 %            |
| 80 - 90 %  | 0.00 %  | 0.00 %       | 0.00 %       | 0.00 %      | 0.00 %            | 1.47 %          | 1.47 %      | 8.81 %            |
| 90 - 100 % | 100.00 % | 100.00 %     | 100.00 %     | 100.00 %    | 100.00 %          | 98.33 %         | 98.33 %     | 75.13 %           |
+------------+---------+--------------+--------------+-------------+-------------------+-----------------+-------------+-------------------+
```

  The report below summarizes the percentage of respect of the NDR constraint by the current router. For example, in the report below, for the INNOVUS_NDR constraints between 90% and 100% respect of the constraint, there are 100% of the nets.

```
**INFO (OILAM) : Summary NDR respect by router:
+------------+-------------+-------------------------+-------------------------+
| Respect    |INNOVUS_NDR |INNOVUS_NDR_NDR_p10_p10 |INNOVUS_NDR_NDR_p20_p10 |
+------------+-------------+-------------------------+-------------------------+
| 0 - 10 %   |0.00 %       |0.00 %                   |0.00 %                   |
| 10 - 20 %  |0.00 %       |0.00 %                   |0.00 %                   |
| 20 - 30 %  |0.00 %       |0.00 %                   |0.00 %                   |
| 30 - 40 %  |0.00 %       |0.00 %                   |0.00 %                   |
| 40 - 50 %  |0.00 %       |0.00 %                   |0.00 %                   |
| 50 - 60 %  |0.00 %       |0.00 %                   |0.00 %                   |
| 60 - 70 %  |0.00 %       |0.00 %                   |0.00 %                   |
| 70 - 80 %  |0.00 %       |0.00 %                   |0.00 %                   |
| 80 - 90 %  |0.00 %       |0.00 %                   |0.00 %                   |
| 90 - 100 % |100.00 %     |100.00 %                 |100.00 %                 |
+------------+-------------+-------------------------+-------------------------+
```

  The report below provides information similar to above reports but on a net to net basis.

```
**INFO (OILAM) : List of nets with a constraint from INNOVUS_CLK_NDR and minZ = 5 and routing adherence < 80 %:
+-----+---------------+-----------------+--------------------------+-------------+----------------+----------------+-----------------+
| ID  | NDR Cstr      | Min layer Cstr  | Route type adherence (%) | Wire Length | X wire length  | Y wire length  | net             |
+-----+---------------+-----------------+--------------------------+-------------+----------------+----------------+-----------------+
| 1   | DOUBLESPACE   | M5(5)           | 0.00                     | 0.81        | 0.00           | 0.81           | CTS_107         |
| 2   | DOUBLESPACE   | M5(5)           | 0.00                     | 1.00        | 0.67           | 0.32           | A/B/CTS_52      |
| 3   | DOUBLESPACE   | M5(5)           | 0.00                     | 1.05        | 0.27           | 0.78           | creg/stepolkqq  |
| 4   | DOUBLESPACE   | M5(5)           | 0.00                     | 1.32        | 0.95           | 0.37           | A/D/E/CTS_10    |
| 5   | DOUBLESPACE   | M5(5)           | 0.00                     | 2.03        | 1.35           | 0.68           | CTS_20          |
| 6   | DOUBLESPACE   | M5(5)           | 0.00                     | 2.10        | 0.00           | 2.10           | A/CTS_35        |
| 7   | DOUBLESPACE   | M5(5)           | 0.00                     | 2.15        | 1.35           | 0.80           | B/CTS_71        |
...
```

- The following command prints the detailed route type constraints information with specified threshold routing adherence of 101, which means only nets with adherence < 101 % will be included in detailed report.

  ```
  reportRouteTypeConstraints -detailed -threshold 101
  ```

  A part of the sample report is shown below.

```
**INFO (OILAM) : List of nets with a constraint from INNOVUS and minZ = 13 and routing adherence < 101 %:
+---+---------------+--------------------------+-------------+----------------+----------------+----------------------+
| ID| Min layer Cstr| Route type adherence (%) | Wire Length | X wire length  | Y wire length  | net                  |
+---+---------------+--------------------------+-------------+----------------+----------------+----------------------+
| 1 | M12(13)       | 100.00                   | 480.57      | 480.45         | 0.12           | FE_OFN1_n1           |
| 2 | M12(13)       | 100.00                   | 494.73      | 494.25         | 0.48           | FE_OCPN2_FE_OFN0_n1  |
+---+---------------+--------------------------+-------------+----------------+----------------+----------------------+
```

- The following command prints information about the layer assignment, NDR, and stack via rule.

  ```
  reportRouteTypeConstraints -svr
  ```

  A part of the sample report is shown below.

```
**INFO (OILAM) : Summary number of stack via rule constraint applied on driver:
+---------------------+--------------+-------------+----------+----------+-------------+----------+----------+-------------------------+
| Stack via rule name | Routing Cost | # stack via rule applied              | Adherence                     | Bottom layer constraints |
+                     +              +-------------+----------+----------+-------------+----------+----------+                         +
|                     |              | Not required | Required            | Not required | Required            | M12(13)                 |
+                     +              +             +----------+----------+             +----------+----------+                         +
|                     |              |             | cell pin | instance pin |           | cell pin | instance pin |                         |
+---------------------+--------------+-------------+----------+----------+-------------+----------+----------+-------------------------+
| VPPERF_M1_M4_2_2_3_1 | 143.0       | 2           | 0        | 0        | 100.00      | NA       | NA       | 2                       |
+---------------------+--------------+-------------+----------+----------+-------------+----------+----------+-------------------------+
| Total               |              | 2           | 0        | 0        | 100.00      | NA       | NA       | 2                       |
+---------------------+--------------+-------------+----------+----------+-------------+----------+----------+-------------------------+
```

- The following command prints information about only stack via rule.

  ```
  reportRouteTypeConstraints -svrOnly
  ```

  A part of the sample report is shown below.

```
**INFO (OILAM) : Summary number of stack via rule constraint applied on sink:
+---------------------+--------------+-------------+----------+----------+-------------+----------+----------+
| Stack via rule name | Routing Cost | # stack via rule applied              | Adherence                     |
+                     +              +-------------+----------+----------+-------------+----------+----------+
|                     |              | Not required | Required            | Not required | Required            |
+                     +              +             +----------+----------+             +----------+----------+
|                     |              |             | cell pin | instance pin |           | cell pin | instance pin |
+---------------------+--------------+-------------+----------+----------+-------------+----------+----------+
| VPPERF_M1_M4_2_2_3_1 | 143.0       | 0           | 0        | 0        | NA          | NA       | NA       |
+---------------------+--------------+-------------+----------+----------+-------------+----------+----------+
| Total               |              | 0           | 0        | 0        | NA          | NA       | NA       |
+---------------------+--------------+-------------+----------+----------+-------------+----------+----------+
```

Related Commands

- ccopt_design
- optDesign
- setAttribute

reportTranViolation

```
reportTranViolation
[-help]
[-all | -noGlobalNets]
[-max]
[-min]
[-selNetFile fileName]
[-excNetFile fileName]
[-significant_digits]
[-useDrcMargin]
[-outfile fileName]
```

After timing analysis, reports pins and ports that exceed the transition constraints in the timing constraints file.

Run this command after running timing analysis (`timeDesign`) or timing optimization (`optDesign`).

Parameters

`-help`	Prints a brief description that includes type and default information for each `reportTranViolation` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man reportTranViolation`
`-all`	Reports violations on all nets in the design. *Default:* `-all`
`-excNetFile fileName`	
	Specifies a file that lists the names hierarchical nets that are excluded from the transition violation report.
`-noGlobalNets`	Reports violations on all nets except for power and ground nets. **Note**: The parameters, `-all` and `-noGlobalNets` are mutually exclusive.
`-max`	Reports violations of maximum transition constraints.
`-min`	Reports violations of minimum transition constraints. **Note**: The parameters, `-max` and `-min` are mutually exclusive.
`-outfile fileName`	Specifies name of the output file.
`-selNetFile fileName`	
	Specifies a file that lists the names of hierarchical nets to consider in the transition violation report. **Note**: The parameters, `-selNetFile` and `-excNetFile` are not mutually exclusive. If both parameters are specified, the `-excNetFile` parameter overides the `-selNetFile` parameter.
`-significant_digits`	Allows the user to specify the number of significant digits. *Default*: 3
`-useDrcMargin`	Generates a report using the margin value that you specify with the `-drcMargin` parameter of the `setOptMode` command.

Example

The following command runs timing analysis and generates a `module1.tran` report file that lists the pins and ports that exceed the transition constraints:

```
reportTranViolation -outfile module1.tran
```

Related Commands

- optDesign
- setOptMode
- timeDesign

reportVtInstCount

```
reportVtInstCount
[-help]
[-area]
[-detailed]
[-leakage]
[-outFile fileName]
```

Reports the instance distribution across the VT partitions in the design. By default, the Innovus software partitions the design into either two or three VT buckets. Each library, cell, and instance falls into one of these buckets and is reported by this command.

Parameters

`-help`	Prints a brief description that includes type and default information for each `reportVtInstCount` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man reportVtInstCount`
`-area`	Prints the area information across libraries in each VT partition.
`-detailed`	Prints complete details in the specified file. **Note**: Before specifying this parameter, you must specify the file name using the `-outFile` parameter.
`-leakage`	Prints the leakage distribution across the libraries in each VT partition. When you specify this parameter, a complete leakage power analysis is performed on the design, which takes a little long to run.
`-outFile` *fileName*	
	Prints the output in the specified file in the format required for user-defined VT partitioning. **Note**: To print the same report that is output to the log that includes all leakage and instance information, specify the `-detailed` parameter. If you do not specify the `-detailed` parameter, the `-outFile` parameter will only print the VT partitions.

Examples

- The following command outputs the Vt partitions to the file `VT_Partitions.rpt`:

  ```
  reportVtInstCount -outFile Vt_Partitions.rpt
  ```

  The `Vt_Partitions.rpt` file contains the following information:

```
#################################################################
#  Generated by:      Cadence Encounter <version>
#  OS:                Linux x86_64(Host ID <machine name>)
#  Generated on:      <date>
#  Design:            <design name>
#  Command:           reportVtInstCount -outFile VT_Partitions.rpt
#################################################################

LVT
at28nm_lvt_aob_s_p125_11v
at28nm_lvt_core_s_p125_11v
at28nm_lvt_nscan_s_p125_11v
at28nm_lvt_scan_s_p125_11v

SVT
at28nm_svt_core_s_p125_11v

HVT
at28nm_svt_aob_s_p125_11v
at28nm_svt_iso_s_p125_11v
at28nm_svt_nscan_s_p125_11v
at28nm_svt_scan_s_p125_11v
at28nm_hvt_aob_s_p125_11v
at28nm_hvt_core_s_p125_11v
at28nm_hvt_nscan_s_p125_11v
at28nm_hvt_scan_s_p125_11v
```

- The following commands output the Vt distribution as well as information in instances and cells. When the output file is not specified, the `reportVtInstCount` command outputs only to the log. The second command outputs both to the log and to the output file.

  ```
  reportVtInstCount
  reportVtInstCount -detailed -outFile VT_detailed.rpt
  ```

  The report will contain the following information:

  ```
  Power view               = MAX1
  Number of VT partitions  = 3
  Standard cells in design = 1105
  Instances in design      = 45515

  Instance distribution across the VT partitions:

   LVT : inst = 0 (0.0%), cells = 346 (31%)
     Lib at28nm_lvt_aob_s_p125_11v : inst = 0 (0.0%)
     Lib at28nm_lvt_core_s_p125_11v : inst = 0 (0.0%)
     Lib at28nm_lvt_nscan_s_p125_11v : inst = 0 (0.0%)
     Lib at28nm_lvt_scan_s_p125_11v : inst = 0 (0.0%)

   SVT : inst = 20232 (44.5%), cells = 296 (27%)
     Lib at28nm_svt_core_s_p125_11v : inst = 20232 (44.5%)

   HVT : inst = 23030 (50.6%), cells = 400 (36%)
     Lib at28nm_svt_aob_s_p125_11v : inst = 10 (0.0%)
     Lib at28nm_svt_iso_s_p125_11v : inst = 325 (0.7%)
     Lib at28nm_svt_nscan_s_p125_11v : inst = 854 (1.9%)
     Lib at28nm_svt_scan_s_p125_11v : inst = 18742 (41.2%)
     Lib at28nm_hvt_aob_s_p125_11v : inst = 2 (0.0%)
     Lib at28nm_hvt_core_s_p125_11v : inst = 2397 (5.3%)
     Lib at28nm_hvt_nscan_s_p125_11v : inst = 0 (0.0%)
     Lib at28nm_hvt_scan_s_p125_11v : inst = 700 (1.5%)
  ```

- The following command outputs the Vt distribution as well as the area distribution across libraries in each VT partition:
  `reportVtInstCount -area`

The report will contain the following information:

```
Power view                = MAX
Number of VT partitions   = 2
Standard cells in design  = 2159
Instances in design       = 309154
Design area               = 1401477480000

Instance distribution across the VT partitions:

LVT : inst = 5414 (1.8%), area = 35794980000 (2.6%), cells = 1044 (48%)
   Lib at28nm_lvt_aob_s_p125_11v : inst = 5414 (1.8%), area = 35794980000 (2.6%)

HVT : inst = 303708 (98.2%), area = 1372182084000 (97.9%),  cells = 1115 (52%)
   Lib at28nm_svt_aob_s_p125_11v : inst = 30378 (98.2%), area = 1372182084000 (97.9%)
```

- The following commands output reports with the leakage power numbers included. The first command outputs only to the log. The second command outputs both to the log and to the output file.

```
reportVtInstCount -leakage
reportVtInstCount -leakage -detailed -outFile VT_detailed_leakage.rpt
```

The report will contain the following information:

```
 Power view               = MAX1
 Number of VT partitions  = 3
 Standard cells in design = 1105
 Instances in design      = 45515
 Design leakage power     = 1.041 mW

 Instance and leakage distribution across the VT partitions:

  LVT : inst = 0 (0.0%), leakage = 0.000 mW (0.0%), cells = 346 (31%)
    Lib at28nm_lvt_aob_s_p125_11v : inst = 0 (0.0%), leakage = 0.000 mW (0.0%)
    Lib at28nm_lvt_core_s_p125_11v : inst = 0 (0.0%), leakage = 0.000 mW (0.0%)
    Lib at28nm_lvt_nscan_s_p125_11v : inst = 0 (0.0%), leakage = 0.000 mW (0.0%)
    Lib at28nm_lvt_scan_s_p125_11v : inst = 0 (0.0%), leakage = 0.000 mW (0.0%)

  SVT : inst = 20232 (44.5%), leakage = 0.240 mW (23.1%), cells = 296 (27%)
    Lib at28nm_svt_core_s_p125_11v : inst = 20232 (44.5%), leakage = 0.240 mW (23.1%)

  HVT : inst = 23030 (50.6%), leakage = 0.631 mW (60.6%), cells = 400 (36%)
    Lib at28nm_svt_aob_s_p125_11v : inst = 10 (0.0%), leakage = 0.000 mW (0.0%)
    Lib at28nm_svt_iso_s_p125_11v : inst = 325 (0.7%), leakage = 0.007 mW (0.7%)
    Lib at28nm_svt_nscan_s_p125_11v : inst = 854 (1.9%), leakage = 0.022 mW (2.1%)
    Lib at28nm_svt_scan_s_p125_11v : inst = 18742 (41.2%), leakage = 0.582 mW (55.9%)
    Lib at28nm_hvt_aob_s_p125_11v : inst = 2 (0.0%), leakage = 0.000 mW (0.0%)
    Lib at28nm_hvt_core_s_p125_11v : inst = 2397 (5.3%), leakage = 0.009 mW (0.9%)
    Lib at28nm_hvt_nscan_s_p125_11v : inst = 0 (0.0%), leakage = 0.000 mW (0.0%)
    Lib at28nm_hvt_scan_s_p125_11v : inst = 700 (1.5%), leakage = 0.010 mW (1.0%)
```

resetPathGroupOptions

```
resetPathGroupOptions
[-help]
[pathGroupName]
[-early]
[-effortLevel]
[-late]
[-skewingSlackConstraint]
[-slackAdjustment]
[-slackAdjustmentPriority]
[-targetSlack]
[-view]
```

Resets the parameters of a specified path group, or of all path groups, to their default values.

- If no group name and no parameters are provided, the command resets all parameters for all path groups to their default values.
- If only the group name is provided, the command resets all parameters of that path group to their default values.
- If a path group and a parameter are provided, then only that parameter of that path group is reset to its default value.

See `setPathGroupOptions` for parameter descriptions.

Note: This command does not provide the option to reset the weights. Use `setPathGroupOptions` *`pathgroupName`* `-weight 0` to reset the weights.

Parameters

`-help`	Prints a brief description that includes type and default information for each `resetPathGroupOptions` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man resetPathGroupOptions`
`pathGroupName`	Specifies a path group for which to reset parameters.
`parameter_names`	Specifies the parameters to reset to the default value.

Examples

- To reset all parameters of all path groups to the default values, specify the following command:
  ```
  resetPathGroupOptions
  ```
- To reset all the parameters of the `reg2reg` path group to their default values, but leave the parameters of all other path groups unchanged, specify the following command:
  ```
  resetPathGroupOptions reg2reg
  ```

Related Commands

- `createBasicPathGroups`
- `reportPathGroupOptions`
- `setPathGroupOptions`

route_opt_design

```
route_opt_design
[-help]
[-drv]
[-hold]
[-ideal_clock]
[-num_paths number_of_paths]
[-opt]
[-out_dir directory_name]
[-prefix file_name_prefix]
[-route]
[-setup]
[-timing_debug_report]
```

Combines the routing and post-route optimization flows into a single flow. The benefit of this approach is to interleave the optimization step with routing step. When this command is run, the software performs global and detail routing, setup/hold timing optimization, and DRV fixing. The `route_opt_design` command can be used instead of using two separate commands listed below:

- `routeDesign`
- `optDesign -postRoute -setup -hold`

Note: The `route_opt_design` flow is different from `routeDesign` followed by `optDesign -postRoute -setup -hold` in that the `route_opt_design` command performs more optimization during the routing stage.

The `route_opt_design` command has parameters to split the routing part (`-route`) and the optimization part (`-opt`). When these parameters are specified, the flow exits the interleaved mode, and goes back to the traditional flow with separate commands. This means that `route_opt_design -route` is equivalent to `routeDesign`, and `route_opt_design -opt` is equivalent to `optDesign -postRoute -setup -hold`.

By default, the optimization step of `route_opt_design` fixes both setup and hold timing in the interleaved mode with routing. To fix setup timing only, use `route_opt_design -setup`. When the `-setup` parameter is specified, the software performs setup timing optimization, and skips hold timing fixing.

Any partial optimization, using the `-drv` or `-hold` parameters cannot be worked in the interleaved mode. Therefore, if any of these parameters are specified, the software redirects to the `optDesign -postRoute` command with the specified option.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `route_opt_design` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man route_opt_design`
`-drv`	Fixes the max_cap and max_tran violations by using the postRoute optimization flow. This is equivalent to `optDesign -postRoute -drv`. **Note**: • Ensure that the design is routed before using this parameter. • The `-drv` parameter can only be used with the `-opt` parameter.

`-hold`	When this parameter is specified without `-setup`, the software exits the interleaved mode, and fixes hold timing violations with the postRoute optimization flow. This is equivalent to `optDesign -postRoute -hold`. **Note**: Ensure that the design is routed before using this parameter. When this parameter is specified along with the `-setup` parameter, either `-hold -setup` or `-setup -hold`, both the setup and hold timing violations are fixed in the interleaved mode. This is the default behavior of `route_opt_design`. **Note**: • The design should be in the post CCOpt stage to use `route_opt_design`. • The `-hold` and `-drv` parameters are mutually exclusive. • The `-hold` parameter can only be used with the `-opt` parameter.
`-ideal_clock`	Performs timing optimization using ideal clocks instead of propagated clocks. Using ideal clocks enables you to optimize timing without building a clock tree first, or ignore the existing clock tree, to close ideal timing. If this parameter is not specified, the software assumes the clock tree has been built, and optimizes timing based on propagated clocks.
`-num_paths` *`number_of_paths`*	Specifies the number of timing paths generated in the timing report. By default, all timing paths are reported.
`-out_dir` *`directoryName`*	Specifies the directory where the software writes timing reports generated when this command runs. *Default*: `timingReports`
`-opt`	When specified, uses the `optDesign -postRoute -setup -hold` command and fixes both setup and hold violations for the postRoute optimization flow. The software exits the interleaved mode, and skips routing. **Note**: Ensure that the design is routed before using this parameter. By default, both setup and hold timing violations are fixed. To limit the scope of the optimization, use the `-setup`, `-hold`, and `-drv` parameters along with `-opt` parameter. **Note**: The parameters `-route` and `-opt` are mutually exclusive.
`-prefix` *`fileNamePrefix`*	
	Specifies a prefix for `route_opt_design` report file names. *Default:* *`DesignName_DesignStage`*, where *`DesignStage`* is `postCTS`, or `postRoute` for `route_opt_design`.
`-route`	Specifies that only routing should be done. This is equivalent to `routeDesign` without performing track optimization in the interleaved mode. **Note**: The design should be in the post CCOpt stage. Note: The -route parameter and the following parameters are mutually exclusive; `-opt`, `-setup`, `-hold`, and `-drv`.
`-setup`	Corrects setup violations in the interleaved mode. When specified without the `-hold` parameter, the software skips the fixing of hold timing violations. **Note**: The design should be in the post CCOpt stage. **Note**: The `-setup` and `-drv` parameters are mutually exclusive.

`-timing_debug_report`	When specified, generates a machine-readable timing debug report after completing routing and postRoute optimization. This report is used for debugging timing results using the timing debug feature in Innovus. **Note**: You can specify the number of paths to be reported using the `-num_paths` parameter.

Examples

- Use the following command to enable interleaved track optimization during the routing stage, for fixing setup violations only:
  ```
  route_opt_design -setup
  ```
- Use the following command to enable interleaved track optimization during the routing stage, for fixing both setup and hold violations:
  ```
  route_opt_design
  ```
  or
  ```
  route_opt_design -setup -hold
  ```
- Use the following command to exit interleaved track optimization mode and perform routing only:
  ```
  route_opt_design -route
  ```
- Use the following command to exit interleaved track optimization mode and perform postRoute optimization only for fixing setup timing:
  ```
  route_opt_design -opt -setup
  ```
  This is equivalent to `optDesign -postRoute` or `optDesign -postRoute -setup`
- Use the following command to exit interleaved track optimization mode and perform postRoute optimization for fixing both setup and hold timing:
  ```
  route_opt_design -opt
  ```
  This is equivalent to `optDesign -postRoute -setup -hold`

Related Information

- `routeDesign`
- `optDesign`
- The following topic in the *Innovus User Guide*
  - Using the route_opt_design Flow

setBufFootPrint

```
setBufFootPrint
[-help]
names
[-library libName]
```

Sets the buffer cell footprint name. This footprint name is usually assigned during design import. The software uses buffer footprint names during timing optimization and top-level buffer insertion in a partitioned design. You can specify the library that contains the footprint.

Use this command before running timing optimization.

Parameters

`-help`	Prints a brief description that includes type and default information for each `setBufFootPrint` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setBufFootPrint`
names	Specifies the buffer cell footprint name or a buffer cell name. When you specify a buffer cell name the Innovus software automatically finds the cell footprint name.
`-library` *libName*	Specifies the library that contains the buffer footprint. You must insert a colon (`:`) between the library name and the buffer name when you use this parameter.

Example

- The following command sets the buffer cell footprint name to `B_1P`:

  ```
  setBufFootPrint B_1P
  ```

Related Information

- `checkFootPrint`
- `loadFootPrint`
- `reportFootPrint`
- `setInvFootPrint`
- `setDelayFootPrint`
- The following topic in the *Innovus User Guide*
  - Using the Footprintless Flow

setDelayFootPrint

```
setDelayFootPrint
[-help]
[-library libName]
names
```

Specifies the delay cell footprint. This footprint name is usually assigned during design import. The delay cells from the cell footprint name are used for inserting delays when you run timing optimization to fix hold-time violations.

Parameters

`-help`	Prints a brief description that includes type and default information for each `setDelayFootPrint` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setDelayFootPrint`
`-library libName`	Specifies the library name from where the cells will be picked up.
names	Specifies the delay cell footprint name or a delay cell name. When you specify a delay cell name, the Innovus software automatically finds the cell footprint name.

Example

The following command sets the delay footprint name to `DEL_1P`:

```
setDelayFootPrint DEL_1P
```

Related Information

- checkFootPrint
- loadFootPrint
- setInvFootPrint
- setBufFootPrint
- The following topic in the *Innovus User Guide*
  - Using Cell Footprints in Timing Optimization

setDontUse

```
setDontUse
[-help]{{cellName value} | {-reset}}
[-hinst hinstName [-reset]]
```

Sets a dont_use attribute on a cell, prohibiting timing optimization commands from using this cell during optimization. Optionally, you can specify a library that contains the cell. Using this command overrides the library and/or timing constraints. The dont_use attribute you set remains persistent through save and restore operations.

If you set the dont_use attribute on a cell whose dont_use attribute is already set, the software issues a warning message, for example:

```
setDontUse buff1 true
Setting BUFF1 as a don't use cell
setDontUse buff1 true
Warning: Cell BUFF1 already has a dont_use attribute
```

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setDontUse` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setDontUse`
`cellName`	Specifies the cell to constrain. To choose a cell in a specific library, specify *`libName/cellName`*. You can use wildcards to specify cell names.
`-hinst` *`hinstName`*	

	Specifies the hierarchical module instance names on which the “don’t use” setting should be applied. This parameter is specified with the *cellName* and the *value* parameters.
	`setDontUse -hinst cellName value`
	The following scenarios are applicable to the use of this parameter:
	• The hinst setting overrides the non-instance specific settings. For example:
	`setDontUse -hinst Inst1 BUFX1 false`
	Where `Inst1` is the instance name, `BUFX1` is the cell name, and the value is set to `false`
	`setDontUse BUFX1 true`
	Where `BUFX1` is the cell name and the value is set to `true`
	In the above example, the first command will override the second command and so the cell, BUFX1 is usable within the instance, Inst1, but not elsewhere within the design.
	• The more specific definition always takes precedence irrespective of the sequence of commands. For example:
	Setting 1
	`setDontUse -hinst Inst1/Inst2 BUFX1 true`
	`setDontUse -hinst Inst1 BUFX1 false`
	In `Inst1/Inst2`, the dont_use attribute of `BUFX1` is `true`
	Setting 2
	`setDontUse -hinst Inst1 BUFX1 false`
	`setDontUse -hinst Inst1/Inst2 BUFX1 true`
	In `Inst1/Inst2`, the dont_use attribute of `BUFX1` is also `true`
	So, the above two settings are equivalent.
	• When using this parameter, complete instance names should be specified. Wildcards are not supported.
	• The `true` or `false` value specified for cells using the `setDontUse` command can be reset by using the `-reset` parameter alongwith the `-hinst` parameter.

`-reset`	This parameter is used with the `-hinst` parameter. When specified, it removes all dont_use constraints attached to the specified hierarchical module instance.
	`setDontUse -hinst A BUF1 true`
	`setDontUse -hinst A -reset`
	The above command resets the dont_use attribute on cell `BUF1` in `hinst A`.
	Note: The `setDontUse -hinst hinstName -reset` command clears the "don't use" settings for the specified hierarchical instances but does not clear the "don't use" settings for any sub-hierarchical instances inside the specified hierarchical instances. For example:
	`setDontUse -hinst A BUF1 true`
	`setDontUse -hinst A/B BUF2 true`
	`setDontUse -hinst A -reset`
	The above sequence of commands resets the dont_use constraint on BUF1 but does not reset the dont_use constaint on BUF2.
	Note: The standard design-wide dont_use constraints from SDC, .lib, and the `setDontUse` command without the `-hinst` parameter will still apply.

value	Specifies whether to set or unset the dont_use attribute: `true` sets the dont_use attribute on the specified cell, and `false` unsets it.

Command Order

Use this command before you run `optDesign`.

Examples

- The following commands set buff1 and buff2 as "don't use" cells:

```
setDontUse buff1 true
Setting BUFF1 as a don't use cell
setDontUse buff2 true
Setting BUFF2 as a don't use cell
```

- The following command sets AND2 in library libSlow as "don't use" cells:

```
setDontUse libSlow/AND2 true
Setting libSlow/AND2 as a don't use cell
```

Related Information

- `optDesign`
- The following topic in the *Innovus User Guide*
  - Using the Footprintless Flow

setInvFootPrint

```
setInvFootPrint
[-help]
[-library libName]
invCellOrFootprintNames
```

Specifies the inverter cell footprints. The footprint names are usually assigned during design import. The Innovus software uses these inverter footprint names during timing optimization and top-level inverter insertion in a partition design.

Parameters

`-help`	Prints a brief description that includes type and default information for each `setInvFootPrint` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setInvFootPrint`
`invCellOrFootprintNames`	Specifies the inverter cell footprint names or inverter cell names. When you specify an inverter cell name, the Innovus software automatically finds the cell footprint name.
`-library` *`libName`*	Specifies the library name from where the cells will be picked up.

Related Information

- `checkFootPrint`
- `loadFootPrint`
- `setBufFootPrint`
- `setDelayFootPrint`
- The following topic in the *Innovus User Guide*
  - Using Cell Footprints in Timing Optimization

setLatencyFile

```
setLatencyFile
[-help]
latencyFileName
```

Specifies the latency file that the timing engine must load when the timing analysis mode is set to `-usefulSkew`.

To see the current settings for the `setLatencyFile` command, see `getLatencyFile`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setLatencyFile` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setLatencyFile`
`latencyFileName`	Specifies the latency file to load.

Command Order

Use this command after setting analysis mode to `-usefulSkew`. For more information, see the Timing Analysis chapter in the *Innovus User Guide*.

Example

The following example sets the latency filename to `latency1`.

```
setLatencyFile latency1
```

Related Information

- `getLatencyFile`
- The following topic in the *Innovus User Guide*
  - Performing Optimization Before CTS

setMaxCapPerFreq

```
setMaxCapPerFreq
[-help]
[-lib libName]
[-force]
-freq {freq1 freq2 …}
-cap {cap1 cap2 …}
-pins {cell pin}
```

Adds a maximum capacitance per frequency table to the existing `max_capacitance` statement in the timing library. Timing analysis uses this table to detect violations, and `optDesign` attempts to repair violations.

The timing engine derives the frequency of an instance from the clock domain to which it is connected (the fastest clock domain, if there is more than one). If the instance is a clock instance (that is, part of the clock tree), the actual clock frequency is used. If the instance is part of the combinational logic (that is, a flip-flop, or the logic connected to the output of a flip-flop), the frequency used equals one-half of the associated clock domain frequency because a flip-flop output can only toggle at one-half the rate of the input clock. For example, a clock of 1.0 Mhz has 1,000,000 rise and fall transitions per second; however the output of a flip-flop connected to a 1.0 Mhz clock can only have 500,000 rise and fall transitions per second.

Timing analysis computes the frequency directly from the timing graph. Use the `dbSetNetFrequency` command to write the effective frequency onto the associated net in the database. You can then use the `dbNetFrequency` database command to check a given net's frequency value. The frequency values are stored in Hertz in the database. Modifying this value with the `dbSetNetFrequency` database command only affects `verifyACLimit`.

Note: This command supports MMMC and performs DRV computation as per view frequency. This feature is enabled by setting the below global variable to `true`.

```
set_global timing_report_drv_enable_frequency_per_view true
```

For detailed description of the variable, see the Timing Global Variables chapter.

You can use this command any time after loading the timing library files (that is, after importing or restoring the design).

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setMaxCapPerFreq` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setMaxCapPerFreq`
`-cap {`*`cap1 cap2 …`*`}`	
	Specifies a list of capacitance values that must be greater than or equal to 0. You must specify the same number of -freq and -cap values. The software derives the maximum capacitance from a linear interpolation (or extrapolation) for a given frequency value using two frequency values in the table. For interpolation, the software uses the closest value above the frequency, and the closest value below the frequency. For extrapolation, the software uses the two closest values to the frequency. *Type:* Float, specified in units of pF
`-force`	Computes the maximum capacitance from the interpolated `-freq` and `-cap` values only, and ignores any previously defined `max_capacitance` values for these pins in the library. *Default:* Computes the maximum capacitance using the worst case (minimum value) between the library `max_capacitance` values and the interpolated `-cap` values.
`-freq {`*`freq1 freq2 …`*`}`	
	Specifies a list of at least two frequency values that must be greater than or equal to 0. You must specify the same number of -freq and -cap values, and the frequency values must be specified in ascending order. *Type:* Float, specified in units of Hz
`-lib` *`libName`*	
	Modifies only the matching cell-pins in the specified library. *Default:* Modifies the matching cell-pins in all libraries
`-pins {`*`cellpin`*`}`	
	Adds the new capacitance constraints to the specified set of cells and pins. You can specify output or inout pins. The asterisk (*) wildcard character can be used in a cell or pin name. If a table associated with the pin already exists, it is replaced with the new table. For analysis and optimization, the software uses the most restrictive (the lowest capacitance) value from either the table or the .lib max_capacitance. If you also specify the -force parameter, the max_capacitance value is ignored. The final value can be further modified using .sdc set_max_capacitance statements.

Examples

Assume all output pins of a library have a `max_capacitance` of 3.5 pF from the library file.

- The following command sets the frequency dependent `max_capacitance` to 3.0 pF at a frequency of 100 Mhz, and to 1.5 pF at a frequency of 200 Mhz, for all output or inout pins for all cells in the library named `fast`:

  ```
  setMaxCapPerFreq -lib fast -freq {100e6 200e6} -cap {3.0 1.5} -pins {* *}
  ```

  As a result, for a net at 300 Mhz, an and2 cell output pin has an extrapolated `max_capacitance` value of 0 pF (which means the cell cannot be used at that frequency). An extrapolation to 50 Mhz results in a `max_capacitance` value of 3.75 pF. However, the 3.75 pF value is greater than the existing `.lib max_capacitance` limit of 3.5 pF; therefore timing analysis uses the 3.5 pF value.

- The following command applies the same constraints to all libraries with cell names matching `dff*` that have a pin named `Qbar`:

```
setMaxCapPerFreq -force -freq {100e6 200e6} -cap {3.0 1.5} -pins {dff* Qbar}
```

Note: The original `.libmax_capacitance` limit is ignored because the `-force` parameter is specified; therefore extrapolation to 50 Mhz results in a final `max_capacitance` constraint value of 3.75 pF.

Related Information

- `dbNetFrequency`
- `dbSetNetFrequency`
- `optDesign`
- `verifyACLimit`
- `timing_report_drv_enable_frequency_per_view`

setMaxCapPerFreqTran

```
setMaxCapPerFreqTran
[-help]
-freq {freq1 freq2 freq3…}
-tran {tran1 tran2 tran3…}
-cap {{cap11 cap12 cap13…} {{cap21 cap22 cap23…}} {{cap31 cap32 cap33…}}}
-pins {cell outputpin}
[-lib libName]
[-force]
[-inputpin inputpin]
```

Sets the maximum capacitance of a pin in the design. Considers both the input transition time and the output frequency. If the maximum capacitance exceeds this value, the `reportCapViolation` command displays a capacitance violation.

You can use this command any time after loading the timing library files (that is, after importing or restoring the design).

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setMaxCapPerFreqTran` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setMaxCapPerFreqTran`
`-cap {{cap11 cap12 cap13…} {{cap21 cap22 cap23…}} {{cap31 cap32 cap33…}}}`	
	Specifies a table of capacitance values with the maximum output pin capacitances associated with `-freq` and `-tran` values. The total number of `-cap` values must be the total number `-freq` values times the total number of `-tran` values. The `-freq` and `-tran` values must be specified before `-cap`. The capacitance values are floats in units of pico farad and all values must be greater than or equal to zero. The capacitance table uses frequency as the first index and the input transition as the second index. For example, `cap11`, `cap12`, and `cap13` are for `freq1`, while `c12`, `c22`, and `c32` correspond to the input transition `tran2`. The maximum capacitance is derived from a linear interpolation (or extrapolation) for a given frequency value and a transition value using two frequency values and two transition values n the table (closest above and below for interpolation, the two closest for extrapolation. *Type:* Float, specified in units of pF
`-force`	Computes the maximum capacitance from the interpolated `-freq`, `-tran`, and `-cap` values only, and ignores any previously defined `max_capacitance` values for these pins in the library. *Default:* (`-force` not specified) Computes the maximum capacitance using the worst case (minimum value) between the library `max_capacitance` values and the interpolated `-cap` values.
`-freq {freq1 freq2 freq3…}`	
	Specifies a list of at least two frequency values with the toggle frequencies at the output pin. Frequency values must be increasing, and the total number of frequency values cannot exceed 255. *Type:* Float, specified in units of Hz. All values must be equal to or greater than `0`.
`-inputpin inputpin`	
	Specifies the input pin name corresponding to the input transition values in `-tran`. If `-inputpin` is not given, `-tran` values correspond to all input pins.

`-lib` *libName*	Specifies the library name from inside a `.lib` file to update the cell-pins. *Default:* If a library name is not specified, modifies the matching cell-pins in all libraries
`-pins {`*cell* output*pin*`}`	
	Adds the new capacitance constraints to the specified set of cells and pins. You can specify output or inout pins; input pins are ignored. The asterisk (`*`) wildcard character can be used in a cell or pin name. If there is already a table associated with the same input-output pair (input pins are specified using `-inputpin`), it is replaced with the new constraints; otherwise, it is added to the output maximum capacitance table list. For analysis and optimization, the software uses the most restrictive (the lowest capacitance) value from either the table or the `.lib max_capacitance`. If you also specify the `-force` parameter, the `max_capacitance` value is ignored. The final value can be further modified using `.sdc set_max_capacitance` statements.
`-tran {`*tran1 tran2 tran3…*`}`	
	Specifies a list of at least two transition values with the input transition times. Transition times must be increasing in value, and the total number of transition times cannot exceed 255. *Type:* Float, specified in units of nanoseconds, and all values must be equal to or greater than `0`.

Example

The following command establishes a maximum capacitance table for all pins on all cells in all libraries.

```
setMaxCapPerFreqTran -force -pins {* *} -freq {1e8 1.25e8 1.5e8} -tran {450 475 500} -cap {{0.022 0.017 0.012} {0.02 0.014 0.01} {0.018 0.013 0.008}}
```

The table is shown below:

```
          450         475         500
1e8       0.022       0.017       0.012
1.25e8    0.02        0.014       0.01
1.5e8     0.018       0.013       0.008
```

Related Commands

- dbNetFrequency
- dbSetNetFrequency
- optDesign
- reportCapViolation
- verifyACLimit

setMaxTranPerFreq

```
setMaxTranPerFreq
[-help]
[-lib libName]
[-force]
-freq {freq1 freq2 freq3…}
-tran {tran1 tran2 tran3…}
-pins {cell pin}
```

Adds a maximum transition per frequency table to the existing `max_transition` statement in the timing library. Timing analysis uses this table to detect violations, and `optDesign` attempts to repair violations.

Use this command any time after loading the timing library files (that is, after importing or restoring the design).

The timing engine derives the frequency of an instance from the clock domain to which it is connected (the fastest clock domain, if there is more than one). If the instance is a clock instance (that is, part of the clock tree), the actual clock frequency is used. If the instance is part of the combinational logic (that is, a flip-flop, or the logic connected to the output of a flip-flop), the frequency used equals one-half of the associated clock domain frequency because a flip-flop output can only toggle at one-half the rate of the input clock. For example, a clock of 1.0 Mhz has 1,000,000 rise and fall transitions per second; however the output of a flip-flop connected to a 1.0 Mhz clock can only have 500,000 rise and fall transitions per second.

Timing analysis computes the frequency directly from the timing graph. Use the `dbSetNetFrequency` command to write the effective frequency onto the associated net in the database. Use the `dbNetFrequency` database command to check any given net's frequency value. The frequency values are stored in Hertz in the database. Modifying this value with the `dbSetNetFrequency` database command only affects `verifyACLimit`.

Note: This command supports MMMC and performs DRV computation as per view frequency. This feature is enabled by setting the below global variable to `true`.

```
set_global timing_report_drv_enable_frequency_per_view true
```

For detailed description of the variable, see the Timing Global Variables chapter.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setMaxTranPerFreq` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setMaxTranPerFreq`
`-force`	Computes the maximum transition from the interpolated `-freq` and `-tran` values only and ignores any previously defined `max_transition` values for these pins in the library. *Default:* Uses the worst case (minimum value) between the `.lib max_transition` values and the interpolated `-tran` values.
`-freq {`*`freq1 freq2 freq3…`*`}`	
	Specifies a list of at least two frequency values that must be greater than or equal to 0. You must specify the same number of `-freq` and `-tran` values, and the frequency values must be specified in ascending order. *Type:* Float, specified in units of Hz
`-lib` *`libName`*	
	Modifies only the matching cell-pins in the specified library. *Default:* Modifies the matching cell-pins in all libraries.
`-pins {`*`cell pin`*`}`	
	Adds the new transition constraints to the specified set of cells and pins. You can specify input, output, or inout pins. The asterisk (`*`) wildcard character can be used in a cell or pin name. If a table associated with the pin already exists, it is replaced with the new table. For analysis and optimization, the software uses the most restrictive (the lowest transition) value from either the table or the `.lib max_transition`. If you also specify the -force parameter, the `max_transition` value is ignored. The final value can be further modified using `.sdc set_max_transition` statements.
`-tran {`*`tran1 tran2 tran3…`*`}`	
	Specifies a list of transition values that must be greater than or equal to 0. You must specify the same number of `-freq` and `-tran` values. The software derives the maximum transition limit from a linear interpolation (or extrapolation) for a given frequency value using two frequency values in the table. For interpolation, the software uses the closest value above the frequency, and the closest value below the frequency. For extrapolation, the software uses the two closest values to the frequency. *Type:* Float, specified in units of ns

Examples

Assume all output pins of a library have a `max_transition` value of 0.9 ns from the library file.

- The following command sets the frequency dependent `max_transition` to 1.0 ns at a frequency of 100 Mhz, and to 0.7 ns at a frequency of 200 Mhz, for pins matching `in*` of cells matching `*` (that is, all cells) in the library named `slow`:

  ```
  setMaxTranPerFreq -lib slow -freq {100e6 200e6} -tran {1.0 0.7} -pins {* in*}
  ```

  As a result, at 300 Mhz, an `and2` cell input pin named `in1` has an extrapolated `max_transition` value of 0.4ns. An extrapolation to 50 Mhz results in a `max_transition` value of 1.15 ns. However, the 1.15 ns value is greater than the existing `.lib max_transition` value of 0.9 ns; therefore timing analysis uses the 0.9 value.
- The following command applies the same constraints to all libraries with cell names matching `dff*` that have a pin named `clk`:

  ```
  setMaxTranPerFreq -force -freq {100e6 200e6} -tran {1.0 0.7} -pins {dff* clk}
  ```

Note: The original `.libmax_transition` value is ignored because the `-force` parameter is specified; therefore an extrapolation to 50 Mhz results in a final `max_transition` limit of 1.15 ns.

Related Commands

- `dbNetFrequency`
- `dbSetNetFrequency`
- `optDesign`
- `verifyACLimit`

setOptMode

```
setOptMode
[-help]
[-opt_activity_refresh_args "list_of_arguments"]
[-opt_add_always_on_feed_through_buffers {true | false}]
[-opt_add_insts {true | false}]
[-opt_add_ports {true | false}]
[-opt_add_repeater_report_failure_reason {true | false}]
[-opt_all_end_points {true | false}]
[-opt_allow_only_cell_swapping {true | false}]
[-opt_allow_multi_bit_on_flop_with_sdc {true | false | mergeOnly | splitOnly}]
[-opt_area_recovery {true |false | default}]
[-opt_area_recovery_setup_target_slack slack]
[-opt_consider_routing_congestion {auto | false | true}]
[-opt_constant_inputs {true | false}]
[-opt_constant_nets {true | false}]
[-opt_concatenate_default_and_user_prefixes {true | false}]
[-opt_delete_insts {true | false}]
[-opt_detail_drv_failure_reason {true | false}]
[-opt_detail_drv_failure_reason_max_num_nets value]
[-opt_down_size_insts {true | false}]
[-opt_drv {true | false}]
[-opt_drv_margin margin]
[-opt_drv_with_miller_cap {true | false}]
[-opt_duplicate_cte_constrained_hport {true | false}]
[-opt_enable_data_to_data_checks {true | false}]
[-opt_enable_podv2_clock_opt_flow {true | false}]
[-opt_enable_restructure {true | false}]
[-opt_fix_fanout_load {true | false}]
[-opt_flop_pins_report {false | true}]
[-opt_flops_report {false | true}]
[-opt_high_effort_cells list_of_cells]
[-opt_hold_allow_overlap {auto | true | false}]
[-opt_hold_allow_resize {true | false}]
[-opt_hold_allow_setup_tns_degradation {true | false}]
[-opt_hold_cells list_of_buffers]
[-opt_hold_on_excluded_clock_nets {true | false}]
[-opt_hold_slack_threshold slack]
[-opt_hold_target_slack holdTargetslack]
[-opt_honor_density_screen {true | false}]
[-opt_honor_fences {true | false}]
[-opt_hold_ignore_path_groups {groupA groupB ...}]
[-opt_leakage_to_dynamic_ratio ratio]
[-opt_max_density density]
[-opt_max_length length]
[-opt_move_insts {true | false}]
[-opt_multi_bit_combinational_mode {auto | power | area}]
[-opt_multi_bit_combinational_opt {false | true | mergeOnly | splitOnly}]
[-opt_multi_bit_combinational_merge_timing_effort {none | low | medium | high}]
[-opt_multi_bit_combinational_split_timing_effort {none | low | medium | high}]
[-opt_multi_bit_flop_merge_bank_label_inference {true | false}]
[-opt_multi_bit_flop_merge_timing_effort {low | medium | high}]
[-opt_multi_bit_flop_name_prefix prefix_name]
[-opt_multi_bit_flop_name_separator separator_name]
[-opt_multi_bit_flop_name_suffix _user_suffix]
[-opt_multi_bit_flop_opt {true | false | mergeOnly | splitOnly}]
[-opt_multi_bit_flop_reorder_bits {false | true | timing | power}]
```

```
[-opt_multi_bit_flop_split_report_failure_reason {true|false}]
[-opt_multi_bit_flop_split_timing_effort {low | medium | high}]
[-opt_multi_bit_unused_bit_count integer]
[-opt_multi_bit_unused_bits {false | true | prePlace | preCTS}]
[-opt_new_inst_prefix prefix]
[-opt_new_net_prefix prefix]
[-opt_one_pass_lec {true | false}]
[-opt_podv2_flow_effort {auto | standard | extreme}]
[-opt_pin_swapping {true | false}]
[-opt_post_route_allow_overlap {true | false}]
[-opt_post_route_area_reclaim {none | setupAware | holdAndSetupAware}]
[-opt_post_route_art_flow {true | false}]
[-opt_post_route_check_antenna_rules {true | false}]
[-opt_post_route_drv_recovery {false | auto | true}]
[-opt_post_route_fix_clock_drv {true | false}]
[-opt_post_route_fix_glitch {true | false}]
[-opt_post_route_fix_si_transitions {true | false}]
[-opt_post_route_hold_recovery {false | auto | true}]
[-opt_post_route_setup_recovery {false | auto | true}]
[-opt_power_effort {none | low | high}]
[-opt_pre_route_ndr_aware list]
[-opt_preserve_all_sequential {true | false}]
[-opt_preserve_hpin_function {true | false}]
[-opt_remove_redundant_insts {true | false}]
[-opt_report_multi_bit_unmerged_reasons {true | false}]
[-opt_resize_flip_flops {true | false}]
[-opt_resize_level_shifter_and_iso_insts {true | false}]
[-opt_resize_power_switch_insts {true | false}]
[-opt_sequential_genus_restructure_report_failure_reason {true | false}]
[-opt_setup_target_slack setupTargetSlack]
[-opt_skew {true | false}]
[-opt_skew_ccopt {none | standard | extreme}]
[-opt_skew_post_route {true|false}]
[-opt_skew_pre_cts {true|false}]
[-opt_target_based_opt_file filename]
[-opt_target_based_opt_file_only {true | false}]
[-opt_target_based_opt_hold_file hold_file_name]
[-opt_tied_inputs {true | false}]
[-opt_time_design_compress_reports {true | false}]
[-opt_time_design_expanded_view {true | false}]
[-opt_time_design_num_paths number]
[-opt_time_design_report_net {true | false}]
[-opt_time_design_vertical_timing_summary {true|false}]
[-opt_unfix_clock_insts {true | false}]
[-opt_verbose {true | false}]
[-reset]
```

Sets global parameters for timing optimization. Parameters that you specify with `setOptMode` are then used automatically when you run timing optimization by invoking the `optDesign` command or by using the Optimize Design form in the ECO Menu.

To see the current settings for the `setOptMode` command, see `getOptMode`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setOptMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setOptMode`

`-opt_activity_refresh_args "`*`list_of_arguments`*`"`	Specifies the list of simulation options to be passed for Joules Xreplay. The parameter accepts all arguments of the `xreplay_stimulus` command.
	The following arguments can be specified using this option:
	• `[-xrun_path]` Specify xrun binary path to run Xcelium for replay simulation.
	• `[-rtl_stim]` Specify the input RTL stimulus file.
	• `[-stim_format]` Suboption of `-rtl_stim`, provides format of rtl stimulus.
	• `[-dut_instance]` Suboption of `-rtl_stim`, provides dut hierarchy in rtl stimulus.
	• `[-start]` Specify start time to run replay for particular stimulation time duration, for example 24ns. Default is simulation start time.
	• `[-end]` Specify end time to run replay for stimulation time duration, for example 100ns. Default is simulation end time.
	• `[-verilog_model_files]` Specify list of library verilog models
	• `[-stim_annotation]` Honor stim annotation of only the specified objects.
	• `[-pre_script]` Specify the Tcl file to be sourced before xreplay.
	• `[-out]` Specify the output replayed gate-level stimulus file. Default is joulesWorkDir/replayed_stim.shm
	• `[-reset_activity_before_readback]` Will reset the previous activity before reading the xreplayed activity file.
	• `[-delay_mode none | zero | unit | punit | delay | pdelay] (default: none)`
	For setting the arguments, use:
	`setOptMode -activity_refresh_args "<args>"`
	For querying the arguments, use:
	`getOptMode -activity_refresh_args`
	Default:`""`

`-opt_add_always_on_feed_through_buffers {true | false}`	

	Used by `optDesign` and PD-aware `addRepeaterByRule`. When this parameter is set to true, optDesign will use always-on buffers in both pre-route and post-route for all the optimization steps; and `addRepeaterByRule` will use always-on buffer for repeater insertion. It should be used for aggressive always-on buffering during optimization. The default optDesign behavior is to use always-on buffers only during pre-route DRV fixing. *Default:* `false`
`-opt_add_insts {true | false}`	Controls whether timing optimization can add instances. *Default:* `true`
`-opt_add_ports {true | false}`	Controls whether timing optimization adds ports to the lower-level hierarchies of the design. Prohibiting the software from adding ports (when this parameter is set to `false`) might not be optimal because buffers are not allowed to be added exactly where needed; however, this preserves hierarchical boundaries of the design so that generated test vectors are still valid, and module-by-module verification is possible. *Default:* `true`
`-opt_add_repeater_report_failure_reason {true | false}`	

When set to `true`, enables the printing of detailed failure reasons for each net for the `addRepeaterByRule` command. By default, the report will be generated in the `timingReports` directory. If the directory is not available, it will be created. For example,

```
<run_dir>/timingReports/ARRFailureReason/ARRRpt.preRoute.1
```

Use Model

```
setOptMode -opt_add_repeater_report_failure_reason true

addRepeaterByRule -selNet ./net.list -rule $scr/rule -preRoute
-reportIgnoredNets ignore_nets
```

Note the following:

- The ignored nets report is appended at the beginning of the failure reason report file. If "`-reportIgnoredNets IGNORE_NET_REPORT_NET_FILE_NAME`" is specified, then the ignored nets report is also saved to the file "`IGNORE_NET_REPORT_NET_FILE_NAME`".
- For multiple calls of ARR with different set of nets and constraints, equal number of failure reports will be generated under the `ARRFailureReason` directory. A suffix, "`_<n>`" will be added, with n being the n^{th} call of ARR. For example, three calls of ARR will lead to the generation of the following three files:
  `<run_dir>/timingReports/ARRFailureReason/ARRRpt.preRoute.1`
  `<run_dir>/timingReports/ARRFailureReason/ARRRpt.preRoute.2`
  `<run_dir>/timingReports/ARRFailureReason/ARRRpt.preRoute.3`
- The default mode of ARR reporting is preRoute. If you specify `-postRoute` or `-alongRoute`, the following will be appended in the report names:`<run_dir>/timingReports/ARRFailureReason/ARRRpt.postRoute.1`
  `<run_dir>/timingReports/ARRFailureReason/ARRRpt.alongRoute.1`

Note: The behavior of the parameter, `-reportIgnoredNets`, remains unchanged. It writes the reports with user-specified file name under the run directory.

Default: `false`

`-opt_all_end_points {true | false}`	Runs total negative slack optimization in addition to the worst negative slack optimization and does so on all end points to ensure a high quality timing result. This feature is automatically activated while running either of the `setDesignMode -flowEffort standard | extreme` timing-closure flows. So, despite the default being `false` it is overwritten by the `setDesignMode` command. **Note:** When this parameter is set to `false`, there may still be some fast TNS optimization done during `preRoute` optimization but there is no effect on the `postRoute` setup optimization. It also has no effect on hold optimization, which means that regardless of this setting, all hold violating paths will always be optimized. *Default:* `false`
`-opt_allow_only_cell_swapping {true | false}`	
	Forces the postRoute optimization to perform only same-size Vth swapping for timing improvements. This option works best on multi-Vth library designs. **Note:** Use this option for reoptimizing the timing in specific cases where routing should remain unchanged. *Default:* `false`
`-opt_allow_multi_bit_on_flop_with_sdc {true | false | mergeOnly | splitOnly}`	
	When this parameter is set to `true`, or by default, the software will not honor the SDC constraints while merging or splitting of flops in the multi-bit flip flop flow. Set this parameter to `false` to honor SDC constraints while merging or splitting of flops. Use `mergeOnly` to ignore the SDC constraints while merging but honor for splitting. Use `splitOnly` to ignore the SDC constraints while splitting but honor for merging. *Default*: `true` **Note**: When this parameter is set to `true`, the software will do merge/split even if there is SDC on D/Q pins and the SDC will be copied to the corresponding D/Q pins in the new flop. But if there are SDC constraints on SE/SI pin, the merging becomes more complicated. For example, a single flop A and flop B have 1 SE pin and D/Q pin. When merging is done, the SDC on D/Q pin will be copied to the new D0/Q0 and D1/Q1, and the SDC information will not be lost. But in the new 2-bit flop, there will be only one SE pin. This raises the issue of which SDC should be copied into the new flop. In this case, the software will check the SE pin SDC in flop A and flop B. If they are compatible, then it will do merging. Otherwise, the merging will not be done, even if the parameter is set to `true`.
`-opt_area_recovery {true |false | default}`	

	Controls whether timing optimization creates additional space by downsizing gates or deleting buffers, while maintaining worst slack and total negative slack. This parameter can be used in three modes: `default, true,` and `false`. • `default`: By default, area reduction is always called after global optimation and after the second TNS optimization. During the rest of `optDesign`, area reduction is called depending on the density level of the current design. • `true`: When this parameter is set to `true`, area reduction is always called before the global optimization stage, after the global optimization stage, and after the second TNS optimization. During the rest of optDesign, area reduction is called depending on the density level of the current design. • `false`: No area reduction is performed during `optDesign`. *Default:* `default` **Note**: This parameter is used to reclaim area during `preRoute` optimization. For reclaiming area during `postRoute` optimization, use the `-opt_post_route_area_reclaim` parameter of this command.
`-opt_area_recovery_setup_target_slack` *slack*	
	Specifies a user-defined target slack for area reclaim vs timing. When a slack value is specified, the software prevents area reclaim optimization from creating or working on near critical paths. Typically, you can set `setOptMode -opt_area_recovery_setup_target_slack 0.1` with `setOptMode -opt_setup_target_slack` at the default value of 0.0. This implies that the area reclaim will not work on any paths less than 0.1ns. Note that the value for `-opt_setup_target_slack` takes higher priority. For example, if `setOptMode -opt_area_recovery_setup_target_slack` is set to -0.1 and `setOptMode -opt_setup_target_slack` is at 0.0 then setup timing will not be allowed to degrade past 0ps and a warning will be issued. Similarly, if say `setOptMode -opt_area_recovery_setup_target_slack` is at +0.1 and `setOptMode -opt_setup_target_slack` is set to +0.2 then timing will target +0.2 and a warning will be issued. You need to set this parameter before any area reclaim is done. *Default*: `0ns`
`-opt_consider_routing_congestion {auto | false | true}`	

	Checks the routing congestion at the PostRoute stage during optimization of setup, hold, drv, or glitch. The option also impacts postCTS hold fixing. This option helps improve NR DRC's as well as Timing jumps caused by routing detours on congested designs. By default, this parameter is set in the following ways: • `auto` – turned “off” for postCTS hold fixing and “on” for postRoute setup and hold fixing • `false` – turned “off” for postCTS hold fixing and postRoute setup and hold fixing • `true` – turned “on” for both postCTS hold fixing and postRoute setup and hold fixing *Default:* `auto`
`-opt_constant_inputs {true | false}`	When set to `true`, forces optimization of instances with constant inputs, even if it worsens the objective gain - in terms of area, power, and delay. *Default:* `false`
`-opt_constant_nets {true | false}`	Controls whether timing optimization optimizes constant nets. This parameter affects only postRoute optimization. *Default:* `true`
`-opt_concatenate_default_and_user_prefixes {true | false}`	

	Controls whether to combine the default prefix for cells and nets added by timing optimization with the prefix specified by `-opt_new_net_prefix` and `-opt_new_inst_prefix`. By default, the software combines the specified prefixes with the default prefix. Specify false for this parameter to ignore the default prefix and use only the prefix specified by `-opt_new_net_prefix` and `-opt_new_inst_prefix`.
	
	For example, if the default prefix is `PSN_000234` and you type the following command:
	`setOptMode -opt_concatenate_default_and_user_prefixes true -opt_new_net_prefix POSTROUTE_N -opt_new_inst_prefix POSTROUTE_C`
	
	The software uses the following prefixes:
	• For cells added by optimization: `POSTROUTE_C_PSN_00234`
	• For nets added by optimization: `POSTROUTE_N_PSN_00234`
	
	If the default prefix is `PSN_000234` and you type the following command:
	`setOptMode -opt_concatenate_default_and_user_prefixes false -opt_new_net_prefix POSTROUTE_N -opt_new_inst_prefix POSTROUTE_C`
	
	The software uses the following prefixes:
	• For cells added by optimization: POSTROUTE_C
	• For nets added by optimization: POSTROUTE_N
	
	Note: If you specify false for this parameter, you must also specify `-opt_new_inst_prefix` and `-opt_new_net_prefix`. In addition, you must specify a different prefix for instances and nets.
	
	Default: `true`

`-opt_delete_insts {true | false}`	Controls whether timing optimization can delete instances. *Default:* `true`
`-opt_detail_drv_failure_reason {true | false}`	
	When set to `true`, enables the printing of detailed DRV failure reason report file that contains the failure summary with the net name. The log file contains a simple version of failure summary without the net name. In addition, the top number(default 50) of violating nets are printed in separate files along with the violation reasons. By default, this parameter is set to false, which means that only a simple version of failure summary without the net name is printed in the log file and no other files are created. **Note**: If there is no DRV failure during the optimization flow, the failure reason report file will not be generated even when this parameter is set to `true`. *Default*: `false`
`-opt_detail_drv_failure_reason_max_num_nets` *value*	

	Specifies the number of violating nets for which the DRV failure reasons are to be printed in the report files. The default value is 50, which means failure reasons for top 50 violating nets are printed by default. The minimum value that can be specified is 1. The maximum value that can be specified is 9999. If this value is specified, failure reasons for all the violating nets are printed. For example, if you set this option to 100, then the top 100 max tran violating nets, if present, will be printed in file, "`xx.maxTran`" and the top 100 max cap violating nets, if present, will be printed in file, "`xx.maxCap`". Also, a file, "`xx.maxFanout`" may also be generated. For each file that is generated, the first printed net has the worst violation value. The file is sorted based on violation values while printing. *Default*: `50`
`-opt_down_size_insts {true | false}`	Controls whether timing optimization can downsize instances. *Default:* `true`
`-opt_drv {true | false}`	Repairs all nets that are constrained for maximum capacitance violations and maximum transition violations. The capacitance and transition constraint values are specified in the timing constraint file or in the library. If you specify false for this parameter, the software repairs only the paths with timing violations. *Default:* `true`
`-opt_drv_margin` *margin*	Scales the maxCap and maxTran constraints according to the *margin* decimal value specified. For example, specifying a value of 0.2 for this parameter tightens the margin (makes it more pessimistic) by 20 percent during optimization and specifying a value of -0.2 relaxes the margin (makes it more optimistic) by 20 percent. Introducing a pessimistic margin improves correlation between the number of DRC violations before and after routing. Value range: `-0.7 ~ 0.7` *Default:* `0`
`-opt_drv_with_miller_cap {true | false}`	When set to `true`, enables ECSM/CCS miller capacitance DRV reporting and fixing for `timeDesign` and `optDesign`. However, the `timing_report_use_receiver_model_capacitance` varibale must also be set to `true`. By default , this parameter is set to `false`, which means that miller cap DRV reporting and fixing is disabled. *Default*: `false`
`-opt_duplicate_cte_constrained_hport {true | false}`	When set to `true`, allows buffering to duplicate hports with CTE constraints to fix DRVs and setup violations. *Default*: `false`
`-opt_enable_data_to_data_checks {true | false}`	

	Enables data to data checks during optimization. During optimization the data to data timing checks are set to off by default to ensure that the engine focuses on other paths. To allow checks to be visible during optimization you can set this option to `true`. *Default*: `false`
`-opt_enable_podv2_clock_opt_flow {true | false}`	When set to `false`, enables the `place_opt_design` and `ccopt_design` flow. By default it is set to `true`, which means the place_opt_design V2 (PODV2 or pod_turbo) and `clock_opt_design` flow is enabled. *Default*: `true`
`-opt_enable_restructure {true | false}`	Controls whether timing optimization swaps pins and restructures the netlist. *Default:* `true` **Note:** This parameter is not enabled when `setDesignMode -flowEffort express` is set.
`-opt_fix_fanout_load {true | false}`	Forces timing optimization to correct fanout load violations, which can be maximum load or maximum fanout count violations, depending on the libraries. *Default:* `false`
`-opt_flop_pins_report {false | true}`	Set this parameter to true for printing out full hierarchical pin names used in the incoming netlist for flops deleted during simplifyNetlist. The output file name format is `<userPrefix>_simplified_flops_pins.rpt`. *Default*: `false` You can choose to write both files with pins or without pins (`-opt_flops_report`) in same session .
`-opt_flops_report {false | true}`	Set this parameter to true for printing full hierarchical flop names used in the incoming netlist for flops deleted during simplifyNetlist. The output file name format is `<userPrefix>_simplified_flops.rpt`. *Default*: `false` You can choose to write both files with pins (`-opt_flop_pins_report`) or without pins in same session.
`-opt_high_effort_cells` *list_of_cells*	Specifies the list of cells to be used for high-effort optimization. This parameter is applicable in both pre-route and post-route setup optimization. **Note**: If the `dont_touch` constraint is set to `true` for the specified cells, then these cells will not be used for high-effort optimization. *Default:* `"" (empty string)`

`-opt_hold_allow_overlap {auto | true | false}`	Allows hold-fixing algorithm to insert buffers with overlap when large enough empty space locations are not available. The placement will then be legalized before ECO routing. Activating this option allows to the tool to fix hold violations, but might have some negative impact on setup timing. • `auto`: Hold fixing will be running in the allow overlapping mode in postCTS and not in the postRoute stage. • `true`: Hold fixing will be running in the allow overlapping mode in both postCTS and the postRoute stage. • `false`: Hold fixing will not be running in allow overlapping mode in both postCTS and the postRoute stage. *Default:* `auto`
`-opt_hold_allow_resize {true | false}`	
	Controls the resizing capability inside hold optimization. When set to `true` or `false`, resizing is turned on or off in both postCTS and postRoute hold optimization. *Default:* `true`
`-opt_hold_allow_setup_tns_degradation {true | false}`	
	Controls whether the software allows total negative slack to degrade during hold fixing operations. By default, the software allows the slack to degrade in order to fix more hold violations. To override the default behavior and ensure that the setup total negative slack is maintained during hold fixing, specify `false` for this parameter. *Default*: `true`
`-opt_hold_cells` *`list_of_buffers`*	Allows you to force the usage of only a specific list of buffer and delay cells. The *`list_of_buffers`* contains regular buffers and delay buffer cells. When this list contains at least one cell, use only the specified cells. *Default*: An empty list. This will trigger the default flow. **Note:** This parameter does not support wildcards. The *`list_of_buffers`* must contain only exact cell name. **Note:** `optDesign` disables automatic cell filtering when this list is specified which could impact the runtime/QOR/leakage of optDesign. **Note:** `optDesign` uses the specified cells without honoring `dont_use` attribute.
`-opt_hold_on_excluded_clock_nets {true | false}`	Controls whether hold fixing will be performed on excluded clock nets or not. This parameter is applicable for both postCTS and postRoute Hold optimization. *Default*: `false`

`-opt_hold_slack_threshold` *slack*	Allows you to provide a hold slack threshold value beyond which the hold timing violations will not be fixed. This is used to avoid fixing out of bound violations at an early stage when the timing constraints and design environment might not be mature enough. The slack value is specified in nano seconds (ns). **Note**: When the hold slack threshold value is specified using this parameter, the hold fixing timing table excludes the violations with slack worse than the specified value. **Note**: The threshold value specified is for hold WNS. *Default:* `-1000ns`
`-opt_hold_target_slack` *holdTargetSlack*	Specifies a target slack value in nanoseconds to use for hold analysis only. During setup violation repair, the setup target slack is *setupTargetSlack* and the hold target slack value is 0. Generally, you use both `-opt_setup_target_slack` and `-opt_hold_target_slack` for hold analysis. If you specify `-opt_hold_target_slack` only, the `-opt_setup_target_slack` value is 0. *Default:* `0`
`-opt_honor_density_screen {true | false}`	Controls whether or not density screens (partial placement blockages used to control congestion) are honored while optimizing timing. By default, this parameter is set to `false`, which means that the density screens are not honored. *Default:* `false`
`-opt_honor_fences {true | false}`	Specifies that the timing optimization takes fences or region constraints into account. This option is honored by each `optDesign` step. If you optimize design with the parameter set to true, the placement is legal. This feature is useful for hierarchical designs. Because this parameter adds constraints to timing optimization, enable it only when needed. *Default*: `false` **Note**: For Multi Supply Voltage (MSV) designs, this parameter is set to `true` by default. **Note:** `optDesign` will not honor very small fences when this parameter is set to true if it would cause fence utilization to be greater than 100 percent.
`-opt_hold_ignore_path_groups {`*groupAgroupB* `...}`	
	Specifies which clock domains or path groups should be excluded from hold timing optimization. This option impacts only `optDesign -postCTS/-postRoute -hold`. *Default:* `""` **Note:** To reset this option, apply an empty list. **Note:** If a violated path is in two path groups and only one of those paths is specified using the `-opt_hold_ignore_path_groups` parameter, this path is still fixed during hold fixing.

`-opt_leakage_to_dynamic_ratio` *ratio*	Controls the priority of the power-driven optimization. You can set any value between 0 and 1. Use this parameter along with the `-opt_power_effort` parameter. *Default:* `1` Some examples of what the value of this parameter indicates are provided below: • `setOptMode -opt_leakage_to_dynamic_ratio 1` This is the default value and implies that timing optimization will be purely leakage-power driven optimization. • `setOptMode -opt_leakage_to_dynamic_ratio 0` This implies that timing optimization will be purely dynamic power-driven optimization. • `setOptMode -opt_leakage_to_dynamic_ratio 0.5` This implies that timing optimization will be both dynamic and leakage power-driven and trade-offs may need to be done to get a balance between the leakage and dynamic power optimization. • `setOptMode -opt_leakage_to_dynamic_ratio 0.1` This implies that timing optimization will be both dynamic and leakage power-driven but dynamic power optimization will be favored. • `setOptMode -opt_leakage_to_dynamic_ratio 0.9` This implies that timing optimization will be both dynamic and leakage power-driven but leakage power optimization will be favored.
`-opt_max_density` *density*	Specifies the maximum value for density (area utilization). The `optDesign` command does not grow the netlist above this value. When reaching a density close to the value specified by this parameter, `optDesign` has limited optimization capability. *Default:* `0.95`

`-opt_max_length` *length*	Specifies that `optDesign` honors the maxLength rule as any other DRV rule (`max_cap/max_tran/max_fanout`). The `optDesign` command will fix all the nets that are violating this rule and will ensure to not create any long nets that violate it. The value is specified in micrometer. **Note:** This parameter implies that more buffers gets inserted in the netlist. *Default:* `-1` *min*=`-1`, *max*=`2147483647` This parameter is disabled by default. **Note:** During the pre-route stage, some maxLength violations might still exist after running `optDesign` due to routing topology changes taking place during the flow.
`-opt_move_insts {true | false}`	Allows cells in the critical path to move when the software repairs setup violations or optimizes timing along the critical path, if allowing them to move improves timing. *Default:* `true`
`-opt_multi_bit_combinational_mode {auto | power | area}`	
	Enables the user-specific multi-bit combinational cell (MBCC) merging and splitting based on area and power. • `auto`: Enables merging or splitting of MBCCs based on area (if power effort is none), otherwise based on power. • `power`: Enables merging or splitting of MBCCs based on power. • `area`: Enables merging or splitting of MBCCs based on area. *Default*: `auto`
`-opt_multi_bit_combinational_opt {false | true | mergeOnly | splitOnly}`	
	Enables and disables the multi-bit combinational flow. This flow is called as a part of the preCTS optimization stage. The following settings are available: • `true`: enables both merging and splitting of combinational cells • `false`: disables the flow • `mergeOnly`: only merges the combinational cells. • `splitOnly`: only splits the combinational cells. *Default:* `splitOnly`
`-opt_multi_bit_combinational_merge_timing_effort {none | low | medium | high}`	

	Specifies the timing effort-level for multi-bit combinational merging. • If the parameter is set to `high`, the software merges the combinational cell to maintain or improve the timing. • If the parameter is set to `low`, the software merges the combinational cell with allowing timing degradation. • If the parameter is set to `medium`, the software allows merging without/minimal degradation in timing. *Default:* `medium`
`-opt_multi_bit_combinational_split_timing_effort {none | low | medium | high}`	
	Specifies the timing effort-level for multi-bit combinational splitting. • If the parameter is set to `low`, the software allows splitting to critical paths only. • If the parameter is set to `medium`, the splitting happen to critical paths to improve timing. • If the parameter is set to `high`, the software performs aggressive splitting to improve timing. *Default:* `low`
`-opt_multi_bit_flop_merge_bank_label_inference {true | false}`	
	When set to `true`, this parameter enables soft merging of flops with the same bank label. By default, this parameter is set to `false`, which means hard merging is done. *Default*: `false`
`-opt_multi_bit_flop_merge_timing_effort {low | medium | high}`	
	Specifies the timing effort level for MBFF merging. • `low` effort setting allows degrading in timing to achieve merging. • `medium` effort means that merging happens either without or with minimal degradation of timing after merging. • `high` effort setting does not allow any degradation and does merging only if it either maintain or improves timing. *Default*: `medium`
`-opt_multi_bit_flop_name_prefix` *`prefix_name`*	
	Specifies the MBFF name prefix. When single-bit flip-flops are merged into a multi-bit flip-flop, you can use this parameter to specify the prefix for the new merged flip-flop. If no prefix is specified, the name of the merged flip-flop starts with the default name, `CDN_MBIT_`. *Default*: `CDN_MBIT_`
`-opt_multi_bit_flop_name_suffix` *`_user_suffix`*	

	Specifies the MBFF name suffix. When this parameter is set, the software adds the user-specified suffix at the end of the multi-bit cell instance name along with the bit-width of the multi-bit cell.
	For example, the below command is used to specify `_FTREG` as suffix:
	`setOptMode -opt_multi_bit_flop_name_suffix _FTREG`
	`CDN_MBIT_bit1_MB_bit2_FTREG2 for a 2-bit multibit cell CDN_MBIT_bit1_MB_bit2_MB_bit3_MB_bit4_FTREG4 for a 4-bit multibit cell`
	Default: `""`

`-opt_multi_bit_flop_name_separator` *separator_name*	
	Specifies the flip-flop merge and split name separator. The separator is used while splitting and merging flip-flops. The default separator is `_MB_` . The use model is detailed below. **Scenario1**: When two or more SBFFs are merged, the separators are used between names of single bit flip-flops. The MBFF is named as; `prefix_bit1_separator_bit2_separator_bit3..._separator_bit#`. **Scenario2**: When an MBFF is split into smaller bits, then each smaller bit is named as; `prefix_bit1_separator_bit2`. **Scenario3**: When an MBFF is split into the original number of single-bit flip-flops, then each SBFF reverts to its original name, which is without prefix and separator such as; `bit1, bit2`... Consider the example below to understand the above scenarios: There are four SBFFs: `bit1, bit2, bit3`, and `bit4`. • With default settings, when they are merged, the following MBFF is generated: **CDN_MBIT**_bit1_**MB**_bit2_**MB**_bit3_**MB**_bit4 This MBFF has the default prefix and the default separator. • When the above MBFF is split into two-bit MBFFs, the following is generated: **CDN_MBIT**_bit1_**MB**_bit2, and **CND_MBIT**_bit3_**MB**_bit4 • When the MBFF is split into the original SBFFs, the names of the SBFFs revert to the original names: `bit1, bit2, bit3`, and `bit4` *Default*: `_MB_`
`-opt_multi_bit_flop_opt {true | false | mergeOnly | splitOnly}`	

	Enables and disables the multi-bit flip flop flow. This flow is called as a part of the preCTS optimization stage. Set this parameter to `true` to enable both merging and splitting of flops. Set it to `false` to disable the flow. Set it to `mergeOnly` to only merge flops. By default, the parameter is set to `splitOnly`, which means the flow is enabled for splitting MBFFs. **Note**: • This option is independent of `-opt_enable_restructure`, `-opt_resize_flip_flops`, and `-opt_preserve_all_sequential` options. • To ensure sufficient multi-bit conversion ratio, allow more usable multi-bit flop cells from the libraries. Use the command, `reportMultibit -equiv` to write out the corresponding multi-bit flop cells to the single-bit flop cells. • To reduce long runtime in flop merge steps, ensure that the scan enable signal nets connecting to the flops are buffered, which means without max-fanout violations. *Default:* `splitOnly`
`-opt_multi_bit_flop_reorder_bits {false | true | timing | power}`	
	Enables swap MBFF bits to take advantage of mixed-drive strength (MDS) multi-bit cells. The following settings are possible: • `false`: the software does not identify the MDS cells and no bit-reordering is done • `timing`: software identifies and uses the MDS cells for timing optimization • `power`: software identifies and uses the MDS cells for power optimization • `true`: software identifies and uses the MDS cells for both timing and power optimization **Note**: When this parameter is set to `true`, `power`, or `timing`, a "Bit-strength (strong to weak)" column will be added in the report generated using the `reportMultibit -cell` parameter. *Default*: `false`
`-opt_multi_bit_flop_split_report_failure_reason {true|false}`	
	When set to `false`, disables the printing of MBFF split failure reasons. By default this parameter is set to `true`, which means the reasons for MBFF instances not splitting are printed. This is the default behavior. *Default*:`true`
`-opt_multi_bit_flop_split_timing_effort {low | medium | high}`	

	Specifies the timing effort level for MBFF splitting. • `low` effort setting means splitting will be done only when absolutely necessary for critical paths. • `medium` effort setting means splitting happens for more critical paths to improve timing and remove critical timing bottlenecks . • `high` effort setting does aggressive splitting and try to improve timing . *Default*: `low`
`-opt_multi_bit_unused_bit_count` *integer*	Specifies the maximum number of unused bits to be allowed in each multi-bit cell. *Default*: `0` *Min*: `1` *Max*: `32` **Note**: To use this parameter, set the `-opt_multi_bit_unused_bits` parameter to `true`.
`-opt_multi_bit_unused_bits {false | true | prePlace | preCTS}`	
	Specifies whether a mapped multi-bit instance can have one or more unused bits. By default, at prePlace stage Innovus tries to merge the single-bit registers to multi-bit instances without unused bits. If during merging, the software is unable to find a suitable multi-bit lib_cell with a width that matches the left-over bits of a bus, it chooses the closest matching width, which leaves the minimum unused bits and, therefore, increases the multi-bit merging coverage. When set to `false`, any merging that results in unused (undriven/unloaded) bits is prevented. When set to `true`, the software tries to merge the single-bit register to multi-bit instances with unused bits at both, prePlace and preCTS stages. When set to `preCTS`, the software merges the single-bit register to multi-bit instances with unused bits at the preCTS stage. *Default*: `prePlace`
`-opt_new_inst_prefix` *prefix*	Specifies a prefix string to add to instances created by timing optimization. For example, if a default prefix is OFC_000234, when you use the command `setOptMode -opt_new_inst_prefix POSTROUTE_`, the combined prefix added to the instance name is `POSTROUTE_OFC_00234`. *Default:* `""` **Note:** To ignore the default prefix, use the following parameter: `-opt_concatenate_default_and_user_prefixes false`.

`-opt_new_net_prefix` *prefix*	Specifies a prefix string to add when timing optimization creates nets. For example, if a default prefix is PSN_000234, when you use the command `setOptMode -opt_new_inst_prefix POSTROUTE_`, the combined prefix added to the net name is `POSTROUTE_PSN_00234`. *Default:* `""` **Note:** To ignore the default prefix, use the following parameter: `-opt_concatenate_default_and_user_prefixes false`.
`-opt_one_pass_lec {true | false}`	When set to `true`, enables the one pass Logical Equivalence Check (LEC) flow. In one pass LEC, you can do a Register Transfer Level (RTL) to netlist comparison between RTL and postRoute optimization (PRO) netlist - any place and route (PnR) stage netlist - in a single step. *Default*: `false`
`-opt_podv2_flow_effort {auto | standard | extreme}`	Specifies the effort level for the place_opt_design V2 (PODV2) flow. See `clock_opt_design`. The following settings are available: • `auto`: The software follows the `setDesignMode -flowEffort` setting to perform optimization. • `standard`: The software performs a fast prects optimization and runtime is less. This option is recommended in the early design stage or in simple designs for PPA estimation. • `extreme`: The software performs a deep optimization to chase the best PPA. Runtime is high. *Default*: `auto`
`-opt_pin_swapping {true | false}`	Controls whether timing optimization allows pin swapping. *Default:* `true` **Note:** This parameter is not enabled when `setDesignMode -flowEffort express` is set.
`-opt_post_route_allow_overlap {true | false}`	Controls whether timing optimization allows overlap during `optDesign -postRoute`. *Default:* `true`
`-opt_post_route_area_reclaim {none | setupAware | holdAndSetupAware}`	

	This parameter can be used after routing to reclaim area during `optDesign -postRoute`. It safely reclaims area while not impacting the timing and DRV violations. You can use the following options: • `none`: No area reclaim is done. This is the default setting. • `setupAware`: Reclaim is done but is not hold aware. This may degrade hold timing significantly but will maintain the setup timing while giving the best possible reclaim results. • `holdAndSetupAware`: Reclaim is done but is both hold and setup aware. It will not give as good reclaim results as setupAware alone. But it will mean both setup and hold Timing will be maintained. *Default:* `none`
`-opt_post_route_art_flow {true | false}`	Ensures that timing optimization applies to ART-based flow. In the ART-based flow, it partially activates the design at the top-level and at the interface path of partition blocks so that there is a reduction in memory usage and run time if the design is large. This parameter marks the core logic inside the partition as `dont_touch`, so the logic is not optimized. *Default:* `false` **Note:** This parameter requires partition definition information and applies to the post-route mode only.
`-opt_post_route_check_antenna_rules {true | false}`	
	Allows optimization to check/ignore antenna rules for LEF-safe swaps. The default behavior is to check the antenna rules, in order to avoid antenna violations caused by the cell swappings. LEF files have certain antenna rules defined in them. By default, when doing LEF-safe swaps for leakage power optimization and for LEF-safe timing recovery, it is ensured that the cells have the same antenna rules. If cells don't have the same antenna rules, they are not swapped. However, if this parameter is set to `false`, in postRoute mode, antenna rules will not be considered when doing LEF-safe swaps. Therefore, setting this option to `false` improves timing and leakage but it can potentially lead to routing having antenna violations, which will require a call to the `ecoRoute` command for cleanup. *Default*: `true`
`-opt_post_route_drv_recovery {false | auto | true}`	

	Controls the DRV/glitch recovery step performed when `optDesign -postRoute -setup -hold` and `optDesign -postRoute -hold` commands have been specified. You can specify the following options: • `false`: Disables DRV/glitch recovery. • auto: Allows the software to determine when to trigger DRV/glitch recovery. Here recovery will be triggered if any of the DRV's or glitches degrade beyond a certain margin. • true: Always triggers DRV/glitch recovery if there are any left in the design. *Default*: `auto`
`-opt_post_route_fix_clock_drv {true | false}`	
	Controls whether `ccopt_pro` is called or not at the start of postRoute optimization. By default this parameter is set `true`, which means that `ccopt_pro` is called to repair clock tree design rule violations (DRVs) that have occurred in the clock tree after routing. These DRVs are most commonly the user-set clock transTime violations. To disable the fixing of DRVs, set the parameter to `false`. *Default*: `true`
`-opt_post_route_fix_glitch {true | false}`	Determines whether to fix glitch violations during SI optimization. To disable fixing of glitch violations, set this parameter to false. *Default*: `true` **Note**: For SI glitch fixing, you must set the `setSIMode -enable_glitch_report` parameter to `true`. If `setOptMode -opt_post_route_fix_glitch` is set to `true` but `setSIMode -enable_glitch_report` is set to `false`, the software will issue a warning during glitch fixing.
`-opt_post_route_fix_si_transitions {true | false}`	
	Enables fixing SI induced slew violations in postRoute optimization. **Note**: The transition values will be used for maximum transition violation fixing only and will not affect the timing calculation in EDI's timing engine. This is because the transition effect is considered by the internal SI engine during delay pushout calculation. *Default*: `false`
`-opt_post_route_hold_recovery {false | auto | true}`	

	Controls the Hold Recovery step performed when `optDesign -postRoute -setup -hold` and `optDesign -postRoute -hold` commands have been specified. You can specify the following options: • `false`: Disables hold recovery. • `auto`: Allows the software to determine when to trigger hold recovery. Here recovery will be triggered if WNS or TNS degrade beyond a certain margin. • `true`: Always triggers hold recovery if timing is not met, irrespective of the gain/degradation in post route timing. *Default*: `false`
`-opt_post_route_setup_recovery {false | auto | true}`	
	Controls the Setup Recovery step performed when `optDesign -postRoute -setup -hold` and `optDesign -postRoute -hold` commands have been specified. You can specify the following options: • `false`: Disables setup recovery. • `auto`: Allows the software to determine when to trigger setup recovery. Here recovery will be triggered if WNS or TNS degrade beyond a certain margin. • `true`: Always triggers setup recovery if timing is not met, irrespective of the gain/degradation in post route timing. *Default*: `auto`
`-opt_power_effort {none | low | high}`	Specifies the power-driven optimization functionality in Innovus. This parameter provides the effort level for optimizing leakage power and dynamic power in the design. Use the `-opt_power_effort` parameter along with the `-opt_leakage_to_dynamic_ratio` parameter to give a superior QOR and to trigger power-driven optimization. Power-driven optimization implies that the `optDesign` command now performs Power, Performance, and Area (PPA) trade-offs at the transform level at every step of timing optimization. You can specify any one of the following options: `none`: Specifies that no power optimization is to be performed. `low`: Specify this parameter when power optimization is not the highest priority. Using the low effort level option indicates the following: • Power optimization is performed only at certain stages of the `optDesign` flow, which is dependent on the value of the `-opt_leakage_to_dynamic_ratio` parameter. When the `-opt_leakage_to_dynamic_ratio` is 1.0, `optDesign` will optimize leakage power after each timing optimization step and no high leakage cells are inserted during hold fixing. When `-opt_leakage_to_dynamic_ratio` is 0.0, `optDesign` will optimize dynamic power at the preCTS stage. For all values of `-opt_leakage_to_dynamic_ratio` between 0.0 and 1.0, both of the above are done.

- Timing optimization is power-aware but uses only low-effort techniques when choosing the best timing optimization fixes to implement.
- Timing closure remains the first priority and the worst negative slack achieved is the same as that in effort mode “none”.
- On an average, the CPU full flow runtime impact is +26% for a -opt_leakage_to_dynamic_ratio of 0.5 as compared to that in effort mode “none”. This can vary depending on both the ratio being used and on the design.
- On an average, the MEM full flow impact is +5%.
- Utilization remains roughly at par with effort level “none” results.
- On an average, the overall standard cell dynamic power saving is -30% and can go up to -69% as compared to that achieved in effort mode “none”. On an average, the overall standard cell leakage power saving is -41% and can go up to -90% on some designs. This is achieved with a -opt_leakage_to_dynamic_ratio of 0.5 and will vary if this ratio is changed.

  Note: The above numbers are estimates and are completely dependent on the type of design.Therefore, they may be very different from the averages given above.

high: Use the high effort level option when having low power is the highest priority. Using the high effort level option indicates the following:

- Power optimization is performed during all calls to optDesign, which is dependent on the value of the -opt_leakage_to_dynamic_ratio parameter. Timing Optimization is power aware and uses high-effort techniques when choosing the best timing optimization fixes to implement.
- Timing closure should still be achievable, if it was previously, and in the majority of cases the same worst negative slack is reached as effort mode "none".
- CPU full flow runtime impact is on an average +63% for a -opt_leakage_to_dynamic_ratio 0.5 compared to effort "none". This can vary depending on both the ratio being used and on the design.
- MEM full flow impact is on an average +11%.
- Utilization is generally improved by -1.3% on average compared to effort level "none" results.
- On an average, the overall standard cell dynamic power saving is -32% and can go up to -70% as compared to that achieved in effort mode “none”. On an average, the overall standard cell leakage power saving is -54% and can go up to -98% on some designs. This is achieved with a -opt_leakage_to_dynamic_ratio of 0.5 and will vary if this ratio is changed.
  Note: The above numbers are estimates and are completely

	dependent on the type of design.Therefore, they may be very different from the averages given above. *Default*: `none` Before power optimization is performed, it is required that the correct power view is specified using the `set_analysis_view -leakage or -dynamic` *`view_name`* command. Both, leakage and dynamic optimization need to target the view that is most important in the design and then use it when optimizing the leakage, internal, and switching power. If dynamic optimization is being done (when a `-opt_leakage_to_dynamic_ratio` is anything other than the default of 1) then it is also required that the switching activity is specified. This can be done by using the `read_activity_file -format {VCD | TCF | SAF}` *`file name`*. In the absence of a switching file, it is recommended to specify the following: `set_default_switching_activity -input_activity 0.2 -seq_activity 0.2.` This will ensure both predictability and consistency throughout the flow.
`-opt_pre_route_ndr_aware` *`list`*	Enables NDR-based optimization by providing a list of NDR names. Using NDR-based optimization, GigaOpt assigns NDR on timing critical nets in order to improve timing. GigaOpt will automatically assess the list of NDRs provided and choose the one that can help timing. It is not recommended to use many NDRs for TAT point of view. *Default*: `""`
`-opt_preserve_all_sequential {true | false}`	
	Preserves sequential elements when netlists are optimized. When the `setOptMode -opt_remove_redundant_insts` parameter is set to `true`, the command removes unobservable or useless logic. However, you can retain sequential elements by setting `-opt_preserve_all_sequential` parameter to `true`. *Default*: `false`
`-opt_preserve_hpin_function {true | false}`	
	Determines whether to preserve logical functions at hierarchical module ports. When set to `false`, only the hierarchical module ports that have a timing assertion applied on them will be preserved (not deleted, duplicated, or polarity changed). This is the recommended setting since it allows more freedom for timing optimization while not impacting the netlist functionality or the timing constraints. When set the `true`, every port of every hierarchical mode will be preserved (not deleted, duplicated, or polarity changed). This is very constraining. *Default*: `false`

`-opt_remove_redundant_insts {true | false}`	Controls whether timing optimization simplifies the netlist. The software recovers area, decreases congestion, and improves runtime by simplifying the netlist in the following ways: • Removing dangling output instances • Propagating constants • Removing unobservable logic • Remapping useless logic The software respects the `set_dont_touch` constraint, so it does not remove the constrained flip-flops, and does not touch non-uniquified modules. *Default:* `true` **Note:** This parameter is not enabled when `setDesignMode -flowEffort express` is set.. **Note**: If a netlist contains spare cells, ensure that you define them using the `specifySpareGate` command else they might be removed by the `-opt_remove_redundant_insts` parameter.
`-opt_report_multi_bit_unmerged_reasons {true | false}`	When this parameter is set to `true`, the reasons for unmerged multibit flops/latches are printed in the MBFF summary. For details of the use model, see Reporting Unmerged Reasons for MBFFs. *Default:* `false`
`-opt_resize_flip_flops {true | false}`	Controls whether timing optimization upsizes or downsizes flip-flops. *Default:* `true`
`-opt_resize_level_shifter_and_iso_insts {true | false}`	
	Controls whether timing optimization resizes shifters and isolation instances. In default mode (false), shifters and isolations are considered "don't-touch" instances and are not changed during timing optimization. When this parameter is set to true, the shifters and isolation cells are resized for timing optimization by using `optDesign`. **Note:** When this parameter is set to true, all the shifters and isolation cells can be resized even if they are marked as dont_touch. *Default:* `false`
`-opt_resize_power_switch_insts {true | false}`	When set to `true`, enables resizing of power switch instances during optimization. By default, this parameter is set to `false`, which means the power switch instances are not resized during optimization. *Default*: `false`
`-opt_sequential_genus_restructure_report_failure_reason {true | false}`	

	When set to `true`, the sequential critical region resynthesis (CRR) failure reasons are reported in the `SeqCrrFailReasons.rpt` file. The report includes the instance name and the corresponding failure reason. **Note**: When CRR is enabled, Innovus uses Genus to restructure portions of the design to improve timing/area/power during setup optimization. *Default:* `false`
`-opt_setup_target_slack` *`setupTargetSlack`*	
	Specifies a target slack value in nanoseconds for setup analysis. During setup violation repair, the setup target slack is *`setupTargetSlack`* and the hold target slack is set to 0. Generally, you use both `-opt_setup_target_slack` and `-opt_hold_target_slack` for hold analysis. If you specify `-opt_hold_target_slack` only, the `-opt_setup_target_slack` value is 0. *Default:* `0ns`
`-opt_skew {true | false}`	Controls whether timing optimization runs the `skewClock` command. This parameter produces scheduling and latency files before CTS runs, or adds buffers and inverters after CTS, just as `skewClock` does. Useful skew optimization is run iteratively within `optDesign`, so you do not need to specify this parameter more than once. **Note**: The `setAnalysisMode -opt_skew` parameter gets set to `true` after `optDesign -preCTS`, which generates the latency file. When the `-opt_skew` parameter is set to `false`, it disables all useful skew everywhere, regardless of other `setOptMode -opt_skew*` settings such as those specified using the `-opt_skew_ccopt`, `-opt_skew_post_route`, and `-opt_skew_pre_cts` parameters. *Default:* `true` **Note:** This parameter is not enabled when `setDesignMode -flowEffort express` is set.

`-opt_skew_ccopt {none | standard | extreme}`	Specifies the effort for useful skew in both CTS and post-CTS flow. It replaces the earlier parameters, `-opt_skew_pre_cts` and `-opt_skew_post_route`. This parameter setting is only respected by `ccopt_design` and `optDesign -postCTS`. • `none`: low effort `ccopt_design` with no skewing • `standard`: low effort `ccopt_design`. This is the default setting. • `extreme`: high effort `ccopt_design` **Note**: To achieve the best possible timing result, the `extreme` setting will significantly increase the execution time of the `ccopt_design` command. Also, for best results the extreme setting should be used with `setDesignMode -flowEffort extreme`. The obsoleted options/commands and the corresponding values set by this parameter are detailed below: • `setOptMode -opt_skew_pre_cts false` corresponds to `setOptMode -opt_skew_ccopt standard` • `setOptMode -opt_skew_post_route true` corresponds to `setOptMode -opt_skew_ccopt` standard if `setDesignMode -flowEffort` is not set to `extreme` • `setOptMode -opt_skew_post_route false` corresponds to `setOptMode -opt_skew_ccopt none` if `setDesignMode -flowEffort` is not set to `extreme` *Default*: `standard`
`-opt_skew_post_route {true | false}`	When set to `false`, it disables useful skew in postRoute optimization. *Default*: `true`
`-opt_skew_pre_cts {true | false}`	When set to `false`, it disables useful skew in preCTS optimization. *Default*: `true`

`-opt_target_based_opt_file` *filename*	Specifies the file containing information that target-based optimization (TBOpt) will use to do fine grain target-based optimization using the `optDesign -postRoute -targeted` command.
	
	The format of the TBOpt file is as follows:
	`NETS : {net1} {net2} {netN}`
	`FROMTOS : {fromTerm1 toTerm1} {fromTerm2 toTerm2} {fromTermN toTermN}`
	`INSTS : {inst1} {inst2} {instN}`
	`PATHS : {endpoint1} {endpoint2} {endpointN}`
	`END_POINT_ADJ : {endpoint1} {slack_adj: value in ps}`
	`AREA_RECLAIM : {ModuleName} {x1 y1 x2 y2}`
	`PWR_RECLAIM : {ModuleName} {x1 y1 x2 y2}`
	`SI_GLITCH : {netName or netNameList} {value: e.g. 0.1}`
	`DRVS : {netName1} {netName2} {max_tran: e.g. 100}`
	
	Note: For more information about the file syntax and what is possible with respect to node specification and end point adjustment, area, power, max transition, and SI glitch optimization see Generating the Target File and Using a Target File for Target-Based Optimization in the *Innovus User Guide*.
	
	An example of a TBOpt file is provided below:
	`NETS : {mod_i/mod_sub_i/reset_l_sync_3}`
	`{mod_i/mod_sub_i/FE_OCPN210_reset_l_sync_3}`
	`INSTS : {mod_i/mod_sub_i/reset_l_sync_reg_15}`
	`{mod_i/mod_sub_i/FE_OCPC210_reset_l_sync_16}`
	`PATHS : {mod_i/mod_sub_i/ser_en/data_d1_reg_32/D}`
	
	Default: `""`

`-opt_target_based_opt_file_only {true | false}`	
	This parameter is used along with the `-opt_target_based_opt_file` parameter. By default, this parameter is set to `true`. So, by default, TBOpt only optimizes the nodes contained within the file specified using the `-opt_target_based_opt_file` parameter. When set to `false`, the TBOpt optimizes other nodes in addition to those specified in the file, if the software considers them to be critical. *Default*: `true`
`-opt_target_based_opt_hold_file` *hold_file_name*	

	Specifies the file containing information that TBOpt will use to perform hold optimization.
	
	`setOptMode -opt_target_based_opt_hold_file` *hold_file_name*
	
	`optDesign -postRoute -targeted -hold`
	
	Note: You can launch setup optimization followed by hold optimization using the following command:
	
	`optDesign -postRoute -targeted -setup -hold`
	
	Hold optimization is only called in the `-opt_target_based_opt_file_only` mode.
	
	Default: `""`

`-opt_tied_inputs {true | false}`	When set to `true`, forces optimization of instances with inputs tied together, even if it worsens the objective gain - in terms of area, power, and delay. *Default:* `false` **Note**: This parameter does not need to be specified when the `ecoRemoveTiedInputs` command is run. In that case this parameter is automatically set to `true` and instances with inputs tied together are optimized. After the command has been run, the parameter reverts to the value before executing the command.
`-opt_time_design_compress_reports {true | false}`	
	When set to `true`, this parameter forces `timeDesign` and `optDesign` to output timing reports in a compressed format. *Default*: `true`
`-opt_time_design_expanded_view {true | false}`	
	When set to `true`, this will force `timeDesign` or `optDesign` to print summary timing reports as per the view in the log file. **Note**: This parameter does not have any effect when the auto view pruning flow is ON for `optDesign`. This is the default flow, starting from the 15.1 release version of the software. Therefore, to output view-specific timing reports in this flow, run `timeDesign -expandedViews` after performing optimization. *Default*: `false`
`-opt_time_design_num_paths number`	Allows you to select the number of paths that should be reported per path group during `timeDesign` or `optDesign` final summary reports. *Default*: `50`
`-opt_time_design_report_net {true | false}`	
	When set to `true`, it forces `timeDesign` or `optDesign` to print timing reports with the net section. *Default*: `true`
`-opt_time_design_vertical_timing_summary {true|false}`	When set to `true`, prints vertical timing summary after `timeDesign` and final summary report. *Default*: `false`
`-opt_unfix_clock_insts {true | false}`	

	Controls whether the software is allowed to resize fixed sequential elements and to move them during placement legalization. optDesign temporarily removes the fixed attribute of each sequential element at the beginning of this step and sets it back to its original value at the end. This option does not remove the routed wires of the clock tree. It minimizes the movement of sequential elements as much as possible. This option impacts only `optDesign -postCts` and `optDesign -postRoute`. • When `true`, the software is allowed to resize and move fixed registers. • When `false`, the software is not allowed to resize or move fixed registers. *Default:* `true` **Note:** If you specify `true` for this parameter, Cadence recommends you use the `routeDesign` command for routing your design.
`-opt_verbose {true | false}`	Controls the amount of information timing optimization outputs to the log file. Specify true to output additional information to use to debug design issues. When you set this parameter to true, the various reasons for not fixing hold violations are listed in the log file. *Default:* `false` For more information, see the "Reasons for Hold Violations to Remain Unfixed" section below.
`-reset`	Resets parameters to their default values. The `-reset` parameter *must* be the first parameter specified. If you specify `-reset` by itself, the software resets all `setOptMode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported.

Examples

- The following command resets all `setOptMode` parameters to their default values:

  ```
  setOptMode -reset
  ```

- The following commands specify target slack values for setup and hold times and optimize timing in the design based on those values:

  ```
  setOptMode -opt_setup_target_slack 0.1 -opt_hold_target_slack 0.15
  optDesign -postCTS
  ```

- The following commands optimize leakage power in addition to optimizing timing and area, and fixing design rule violations:

  ```
  setOptMode -opt_power_effort high -opt_leakage_to_dynamic_ratio 1.0
  optDesign -preCts
  optDesign -postCts
  optDesign -postRoute
  optDesign -postRoute -hold
  ```

- The following commands allow the software to move clock instances during post-CTS timing optimization, and then run timing optimization:

  ```
  timeDesign -postCts
  ```

```
setOptMode -opt_unfix_clock_insts true
optDesgin -postCts
```

- The following command forces hold fixing to use the four cells specified in the list:
  ```
  setOptMode -opt_hold_cells {BUFFX8 BUFFX12 DELAY2 DELAY3}
  optDesign -postCts/-postRoute -hold
  ```
- The following command force `optDesign` to buffer all the nets that have a wirelength larger than `900um` and also not to generate any new long net that would violate this rule:
  ```
  setOptMode -opt_max_length 900
  optDesign -preCts | -postCts | -postRoute
  ```
- The following command forces `optDesign` to work towards positive slack for hold instead of zero:
  ```
  setOptMode -opt_hold_target_slack 0.5
  ```
- The following command will fix all the hold timing violations down to `0ns` slack except the violations that are initialy violated by more than `-3ns`. Those ones would be left untouched.
  ```
  setOptMode -opt_hold_slack_threshold -3 -opt_hold_target_slack 0
  ```
- The following commands will perform a fine grain target-based optimization at the postRoute stage on the nodes provided by the TBOpt file, `tb_opt.txt` as well as any other nodes that are identified as being critical:
  ```
  setOptMode -opt_target_based_opt_file tb_opt.txt
  optDesign -postRoute -targeted
  ```
- The following commands will perform a fine grain target-based optimization at the postRoute stage on the nodes provided by the TBOpt file, `tb_opt.txt` only:
  ```
  setOptMode -opt_target_based_opt_file tb_opt.txt -opt_target_based_opt_file_only true
  optDesign -postRoute -targeted
  ```

Reasons for Hold Violations to Remain Unfixed

The following table lists the various reasons for hold violations to remain unfixed:

Reason	Description
No legal location	There are no valid locations to insert buffers on those violated nets. Limit the local placement congestion and use instance/cell padding during initial placement. In parallel, to allow the tool to insert buffers at non-legal location, set the following parameter: `setOptMode -opt_hold_allow_overlap true`.
Degradation in setup register-to-register WNS	The nets are violated in hold but they are also a part of the register-to-register setup timing-critical paths. So inserting a buffer would degrade the setup register-to-register WNS. To fix these nets at the cost of register-to-register setup degradation, set a negative setup target slack using `setOptMode -opt_setup_target_slack` *`setupTargetSlack`*.
Degradation in setup TNS	The nets are violated in hold but they are also timing critical during setup. So, inserting a buffer would increase the setup TNS. To fix these nets at the cost of setup TNS degradation, set the following parameter: `setOptMode -opt_hold_allow_setup_tns_degradation true`
Degradation in setup WNS	The nets are violated in hold but they are also setup timing critical. So inserting a buffer would degrade the setup WNS. To fix these nets at the cost of setup degradation, set a negative setup target slack using `setOptMode -opt_setup_target_slack` *`setupTargetSlack`*.

Degradation in hold	The nets are violated in hold but inserting a buffer would further degrade hold timing. Check the topology of those nets and probably enlarge the list of available buffer/delay cells that are allowed to be used.
Degradation in maximum capacitance violation	The nets are violated in hold but inserting a buffer would create a new setup maximum capacitance violation. To fix these nets at the cost of DRV degradation, set the following parameter: `setOptMode -opt_drv false`
Degradation in maximum transition violation	The nets are violated in hold but inserting a buffer would create a new setup maximum transition violation. To fix these nets at the cost of DRV degradation, set the following parameter: `setOptMode -opt_drv false`
DRV violations	The nets are violated in hold but also in setup DRV, so they are skipped. To fix these nets at the cost of DRV degradation, set the following parameter: `setOptMode -opt_drv false`
Nets are set as `dont_touch`	To allow hold fixing on these nets, remove the `dont_touch` attribute.
Nets are routed incorrectly	Verify the routing wires for those nets.
Nets are clock nets	No buffers can be inserted on clock nets. You should check your timing constraint file.
None of the always-on-buffers are usable	Verify the environment to allow the use of always-on-buffers.
None of the valid mode is found on a net	The tool is not able to insert a buffer on such a net. This could be related to specific design cases.
Any internal failure	The tool was not able to find a solution for fixing the violation of those nets. This could be related to libraries or specific design cases.

Related Information

- ccopt_design
- clock_opt_design
- getOptMode
- addRepeaterByRule
- optDesign
- place_opt_design
- reclaimArea
- reportMultibit
- routeDesign
- setDesignMode
- set_default_switching_activity
- set_analysis_view
- skewClock
- specifySpareGate
- timeDesign
- The following topic in the *Innovus User Guide*
  - Viewing the Added Buffers, Instances, and Nets after Optimization

setPathGroupOptions

```
setPathGroupOptions
[-help]
pathgroupName
[-early]
[-effortLevel {low | high}]
[-late]
[-skewingSlackConstraint value_in_ns]
[-slackAdjustment float]
[-slackAdjustmentPriority integer]
[-targetSlack float]
[-view view_names]
[-weight  integer]
```

Specifies options to guide timing optimization and timing analysis when path groups are used. To reset parameters to their default value, use `resetPathGroupOptions`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setPathGroupOptions` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setPathGroupOptions`

`-early`	Specifies early mode.
	This parameter is used with the `-slackAdjustment` option when you wish to adjust a path group's slack in Hold timing mode only.
	Note the following:
	• If you do not specify either `-early` or `-late`, then `-late` is implied.
	• If you specify `-early`, then `-early` is used.
	• If you specify `-late`, then `-late` is used.
	• If you specify both `-early` and `-late`, then both `-early` and `-late` are used, which means the same value of `-early` and `-late` are set at the same time.
	For example,
	`setPathGroupOptions reg2reg -slackAdjustment -0.3 -early -late`
	`Slack adjustment of -0.3 applied on reg2reg path_group`
	`**INFO: Setting reg2reg as high effort level since it has a negative slack adjustment.`
	`[DEV]innovus 25> reportPathGroupOptions`
	`---------------------------------------------------------------------------------------------`
	`| Path Group|Effort|Adjustment(early/late)|Target Slack|weight|SkewingSlackCons|view|`
	`---------------------------------------------------------------------------------------------`
	`| reg2reg  | high |  -0.3 /  -0.3  |  0  | 2  |  300  |setup_view9|`

`-effortLevel {low | high}`	Specifies the effort applied to close timing on a specific path group. This parameter is honored by timing optimization (`optDesign`). Select one of the following options: `low`: Specifies that this path group is optimized together with other path groups for best overall slack. low is the default effort value for all user-created path groups. **Note**: Cadence recommends you specify low effort for all path groups except that ones that the tool should focus on most strongly. `high`: Specifies that this path group is optimized separately from the low-effort path groups in order to achieve the best possible WNS for this path group. *Default*: low
pathgroupName	Specifies the path group for which to set options.
`-late`	Specifies late mode. This parameter is used with the `-slackAdjustment` option when you wish to adjust a path group's slack in Setup timing mode only. For usage details, see the description of the `-early` parameter.
`-skewingSlackConstraint` *value_in_ns*	When set, applies the specified skewing slack constraint to the path group. The value specified is in nanoseconds. If not set, no skewing slack constraint is applied to the path group. This parameter provides the ability to control the I/O slack during skewing without degrading the core slack value. The skewing engine skews in a way not to make any already -ve slack path more -ve or not introduce a new -ve slack path. **Examples**: If the following is specified: `setPathGroupOptions R2O -skewingSlackConstraint 0` *Before skewing*: FF's D slack is `-100 ps`(R2R), Q slack is `-20ps` (R2O) *After skewing*: D slack is `-100ps`, Q slack is `-20ps` *Before skewing*: FF's D slack is `-100 ps`(R2R), Q slack is `30ps` (R2O) *After skewing*: D slack is `-70ps`, Q slack is `0ps` **Note**: • This option can control CCOpt useful skew. If you are running ECF (early clock flow), preCTS will use CCOpt useful skew engine to do skewing. in this scenario, this option will take effect. • This option can control postCTS `skewClock` behavior • This option does not impact timing analysis
`-slackAdjustment` *float*	Specifies a positive or negative value used to modify path constraints on all paths that are part of a specific path group. The value is in nanoseconds. To relax the path constraints computed by the timing engine, specify a positive value. To tighten the timing constraints, specify a negative value. This parameter is honored by timing optimization (`optDesign`) and timing analysis (`report_timing`). This means that that the -`slackAdjustment` parameter setting impacts the timing analysis result and all optimization is then performed based on this timing. Using this parameter might have an impact on run time during timing optimization. **Note**: Path groups with negative slack adjustment will be automatically set to high effort, unless the `-effortLevel` option is explicitly set to "'`low`". For positive slack adjustment, the effort level will remain the same. This means if the effort is low before setting a positive slack adjustment, the new effort will be low; if the effort is high before setting a positive slack adjustment, the new effort will be high. **Tip**: Use this parameter together with the `-effort` parameter to direct timing optimization focus.

`-slackAdjustmentPriority` *integer*	
	Provides priority to a path group for the slack adjustment feature. The paths that are a part of overlapping path groups get slack adjustment from the path group that has a higher priority value. If the number is higher, the priority is higher. In case of multiple overlapping path groups with different slack adjustment values, the most constraining slack adjustment is applied. *Default*: 0 **Note**: The `-slackAdjustmentPriority` parameter is ignored if `-slackAdjustment` is not specified.
`-targetSlack` *float*	
	Specifies a target value for setup slack, in nanoseconds, for all paths that are part of a specific path group. The value can be positive or negative. This parameter is honored by timing optimization (`optDesign`). Using this parameter might have an impact on run time during timing optimization. **Note**: Setting this parameter does not impact the timing analysis results, and only setup optimization considers the target slack as the goal of optimization. The path group target slack (`setPathGroupOptions -targetSlack`) and the global setup target slack (`setOptMode -opt_setup_target_slack`) are cumulative. When both are set, global setup target slack adds on top of what is specified using the `setPathGroupOptions -targetSlack` parameter. The effective number depends upon whether path group target slack number is positive or negative. The global setup target slack is the timing closure target for timing optimization. **Note**: When both, `-slackAdjustment` and `-targetSlack` parameters are set, the setup optimization considers the target slack based on slack adjustment timing.
`-view` *view_names*	Specifies the view names. This parameter is used when you want to specify different effort levels of the path group for different views. *Default*: `{}` **Note**: When this parameter is used, only the specified views are active for timing analysis on the path group. Any timing reporting on this path group with inactive views will result in an error.
`-weight` *integer*	Defines the optimization priority. For the default path groups, the software assigns weights in the following manner: • For high-effort path groups: the software assigns a weight of "1" to reg2cgate and "2" to reg2reg. • All low-effort path groups are assigned a default weight of "0". You can assign any value to this parameter (0 to INT_MAX). There is no upper limit. The software looks at the relative priority of the weights for each path group -- the higher the number, the higher the weight. For example, if there are 10 path groups with weights ranging from 1 to 10 and they are all high-effort path groups, the high-effort optimization for all these ten path groups will be started simultaneously. If, however, the optimization is stuck at the path group with the weight 6, the software drops all the path groups that have weights ranging from 1 to 6 and continues to work on the remaining four path groups. **Note**: Use this parameter to control optimization and not `slackAdjustmentPriority`, which only controls the slackAdjustment.

Examples

- To change the timing measured for all setup paths in `PATH_GROUP_A` by relaxing the required time by 2 nanoseconds, specify:
  ```
  setPathGroupOptions PATH_GROUP_A -slackAdjustment 2
  ```

- To change the timing measured for all setup paths in `PATH_GROUP_B` by tightening the required time by 0.2 nanoseconds, specify:
  ```
  setPathGroupOptions PATH_GROUP_B -slackAdjustment -0.2
  ```
- To direct the `optDesign` command to optimize paths in `PATH_GROUP_C` until the worst setup slack in that path group is better than 2 nanoseconds, specify:
  ```
  setPathGroupOptions PATH_GROUP_C -targetSlack 2
  optDesign
  ```

  When these commands run, the software internally subtracts 2 nanoseconds from the required time for all paths of `PATH_GROUP_C`. Note that the `optDesign` command stops optimization when it no longer thinks that it can make progress, regardless of the slack target.
- To change the timing measured for all hold paths in `PATH_GROUP_D` by relaxing the required time by 0.2 nanoseconds, specify:
  ```
  setPathGroupOptions PATH_GROUP_D -slackAdjustment +0.2 -early
  ```
- The following commands are used to set the effort level, skewing slack constraint, weight, and views of the two path groups, and generate a report using these parameters:
  ```
  setPathGroupOptions reg2reg -effortLevel low -skewingSlackConstraint 0.3 -view AV_HL_FUNC_MIN_RC1
  setPathGroupOptions reg2cgate -effortLevel high-skewingSlackConstraint 0.5 -view AV_HL_FUNC_MAX_RC1
  reportPathGroupOptions
  The software prints the following report:
  ```

```
-------------------------------------------------------------------------------------------------
|Path Group|Effort|Adjustment(early/late)|Target Slack|weight|SkewingSlackCons|view               |
-------------------------------------------------------------------------------------------------
|reg2reg   |low   | 0.1/     0.2         | 0          |   2  |    0.3         |AV_HL_FUNC_MIN_RC1 |
|reg2cgate |high  | 0.2/     0           | 0          |   1  |    0.5         |AV_HL_FUNC_MAX_RC1 |
```

The description of the columns headings and the corresponding parameters to which they map is provided below.

Column Heading	Parameter and Description
Path Group	*pathgroupName:* Name of the path group for which the options are being set.
Effort	`-effortLevel`: Specifies the effort applied to close timing on a specific path group. `low`: Specifies that this path group is optimized together with other path groups for best overall slack. low is the default effort value for all user-created path groups. `high`: Specifies that this path group is optimized separately from the low-effort path groups in order to achieve the best possible WNS for this path group.
Adjustment (early/ late)	`-slackAdjustment`: Specifies a positive or negative value used to modify path constraints on all paths that are part of a specific path group. The value is in nanoseconds.
Target Slack	`-targetSlack`: Specifies a target value for setup slack, in nanoseconds, for all paths that are part of a specific path group. The value can be positive or negative. **Note**: This column only displays the latest setting, either early or late.
weight	`-weight`: Defines the optimization priority. For the default path groups, the software assigns weights in the following manner: • For high-effort path groups: the software assigns a weight of "1" to reg2cgate and "2" to reg2reg. • All low-effort path groups are assigned a default weight of "0".
SkewingSlackCons	`-skewingSlackConstraint`: Specifies the skewing slack constraint for the path group. If no constraint is applied to a path group, the column is empty.

view	`-view`: Specifies the view name. It allows you to specify different effort levels of the path group for different views.

Related Commands

- `createBasicPathGroups`
- `resetPathGroupOptions`
- `reportPathGroupOptions`

setSchedulingFile

```
setSchedulingFile
[-help]
schedulingFileName
```

Specifies the name of the scheduling file. The file is not taken into account by CTS, but `skewClock` must have access to the file if it runs several times consecutively.

To see the current setting for the `setSchedulingFile` command, use `getSchedulingFile`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setSchedulingFile` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setSchedulingFile`
`schedulingFileName`	Specifies the name of the scheduling file.

Example

The following command sets the name of the scheduling file to `scheduling_file.cts`:

```
setSchedulingFile scheduling_file.cts
```

Related Command

- `getSchedulingFile`

setSignoffOptMode

```
setSignoffOptMode
[-help]
[-reset]
[-addInst {true | false}]
[-addLoad {true | false}]
[-allowSkewing {true | false}]
[-alongRouteBuffering {true | false}]
[-bufferCellList cell_list]
[-checkDrvFromHoldViews {true | false}]
[-checkType {early | late | both}]
[-clockCellList cell_list]selectDrv
[-clockMaxLevel INT]
[-deleteInst {true | false}]
[-disableGeometryChecks {true | false}]
[-drvMargin float]
[-ecoFilePrefix string]
[-emType {all | avg | rms | peak}]
[-fixCellEm {true | false}]
[-fixClockDrv {true | false}]
[-fixDataDrv {true | false}]
[-fixGlitch {true | false}]
[-fixHoldAllowSetupDegrade {none | low | medium}]
[-fixHoldAllowSetupOptimization {true | false}]
[-fixHoldAllowSetupTnsDegrade {true | false}]
[-fixHoldVariationAware {true | false}]
[-fixHoldWithMargin float]
[-fixIrDrop {true|false}]
[-fixMaxCap {true | false}]
[-fixMaxTran {true | false}]
[-fixOsusGlitch {true | false}]
[-fixSlackInRangeLowerLimit value_in_ns]
[-fixSlackInRangeUpperLimit value_in_ns]
[-fixSiSlew {true | false}]
[-fixXtalk {true | false}]
[-fixXtalkPreserveHold {true | false}]
[-fixXtalkPreserveSetup {true | false}]
[-holdTargetSlack float]
[-holdTargetSlackPerView {viewName1 slack1 viewName2 slack2 …}]
[-holdXtalkDeltaThreshold value_in_ns]
[-holdXtalkSlackThreshold value_in_ns]
[-ignoreDrvChecks {true | false}]
[-keepTempFiles {true | false}]
[-legalOnly {true | false}]
[-loadCellList cell_list]
[-loadEcoOptDb dir_name]
[-loadIrdropDb dir_name]
[-maxCapMargin cap_margin_value]
[-maxLocalDensity threshold_value]
[-maxOptRunTime INT]
[-maxPaths INT]
[-maxRunTime INT]
[-maxSlack float]
[-maxTranMargin tran_margin_value]
[-numReportPaths INT]
[-nworst INT]
[-optimizeCoreOnly {true | false}]
```

```
[-optimizeReplicatedModules {true | false}]
[-optimizeSequentialCells {true | false}]
[-partitionListFile string]
[-pbaEcoRoute {true|false}]
[-pbaEffort {medium | high}]
[-postMask {true | false}]
[-postStaTcl FILE]
[-powerAware {true | false}]
[-powerOptFocus total | leakage | dynamic | leakageOnly | dynamicOnly]
[-prefixName string]
[-preserveFiller {true | false}]
[-preStaTcl FILE]
[-resizeInst {true | false}]
[-retime {none |aocv | path_slew_propagation | aocv_path_slew_propagation}]
[-retimeDepth {default | infinite}]
[-retimeDepthEarly {default | infinite}]
[-retimeMode {none | path | exhaustive}]
[-routingCongestionAware {true | false}]
[-routeCongestionThreshold threshold_value]
[-saveEcoOptDb dir_name]
[-selectHoldEndpoints file_name]
[-selectDrvNetFile file_name]
[-selectDrvPinFile file_name]
[-selectSetupEndpoints file_name]
[-setupRecovery {true | false}]
[-setupTargetSlack float]
[-setupTargetSlackPerView {viewName1 slack1 viewName2 slack2 …}]
[-setupXtalkDeltaThreshold value_in_ns]
[-setupXtalkSlackThreshold value_in_ns]
[-skewCoreOnly {true | false}]
[-skipDrvNetfile file_name]
[-specifyHoldEndpointsMargin string]
[-specifySetupEndpointsMargin string]
[-swapInst {true | false}]
[-upsizeInPowerOpt {true | false}]
[-useGaFillerList list_of_Gate_Array_filler_cells_name]
[-usePfcDecapList list_of_decap_cells]
[-usePfcFillerList list_of_filler_cells]
[-usePfcRegularList list_of_regular_cells]
[-verbose {true | false}]
```

Applies non-default settings for `signoffTimeDesign` and `signoffOptDesign` super commands.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setSignoffOptMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setSignoffOptMode`
`-reset`	Resets all or specified options to default value. The `-reset` parameter *must* be the first parameter specified. If you specify `-reset` by itself, the software resets all `setSignoffOptMode` parameters to their default values. If you specify parameters after `-reset` , the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported.

`-addInst {true|false}`	Specifies whether Setup/Hold/DRV optimizer is allowed to add buffer/delay/swap cell instances. *Default:* `true` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-addLoad {true|false}`	Specifies whether Hold optimizer is allowed to add dummy cell instances. *Default* : `false` **Note:** If set, this parameter must be applied before `signoffOptDesign`.

`-allowSkewing {true | false}`	Allows clock skewing to be done during Setup timing optimization to further improve Setup slack. When set to true, `signoffOptDesign -setup` works on the last level of the clock tree to modify the clock skew in order to improve Setup timing closure. Modification on the clock tree can be either resizing of the last level of the clock tree or buffering to pad endpoint clock pin.
	Default: `false`
	Note:
	• If set, this parameter must be applied before signoffOptDesign.
	• When `-allowSkewing` is active, `signoffOptDesign` will automatically unfix all clock tree instances/nets but it does not honor any CCOpt property that is set using the `set_ccopt_property` command. This is also performed when implementing the ECOs.
	• As for data path optimization, skewing is not allowed to create new Hold violated paths.
	Example:
	The following commands perform clock skewing during setup fixing.
	Only `CKBUFX1` and `CKBUFX4` can be added to the clock tree.
	`setSignoffOptMode -clockCellList {CKBUFX1 CKBUFX4}`
	`setSignoffOptMode -allowSkewing true`
	`signoffOptDesign -setup`

`-alongRouteBuffering {true | false}`	
	When set to `true`, the software uses node location provided through the parasitic data in order to insert buffers along the routing topology not only at the start or end of the net. This helps to better balance the slew or the load for long nets, which is typically needed for setup timing optimization or DRV fixing. *Default*: `false` **Note:** • A prerequisite of this parameter is that parasitic with the node location is provided to `signoffOptDesign`. In addition, the node location parasitic should be read using the `-starN` parameter. For example: `spefIn -starN`. • If set, this parameter must be applied before `signoffOptDesign`.

`-bufferCellList` *cell_list*	Specifies a list of buffer/delay cells that can be used for the timing optimizer, regardless of whether those cells have a dontUse attribute. When the list is empty, the software will automatically identify the list of usable buffer/delay cells. *Default:* `" "` (empty string) **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-checkDrvFromHoldViews {true | false}`	
	Enables optimization of DRV violations (max_capacitance/max_transition) in hold views as well in setup views during DRV optimization. Also, when enabled, this parameter ensures that timing/area/power optimization will not create any new DRV violation in hold and setup views. *Default*: `false` **Note:** If set, this parameter must be applied before `signoffTimeDesign` and `signoffOptDesign`.
`-checkType {early | late | both}`	
	Specifies the type of ECO Timing DB to be generated in PBA mode. The `early` argument specifies hold ECO Timing DB and `late` specifies setup ECO Timing DB. When set to `both`, the ECO Timing DB generation will be done in PBA for both Setup and Hold timing. *Default:* `both` **Note:** • This parameter is only active when the `-retime` option is being set. • If set, this parameter must be applied before `signoffTimeDesign`.
`-clockCellList` *cell_list*	Restricts the list of buffers that are allowed to be used by ECO on the clock tree. *Default*: `""` (empty string) **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-clockMaxLevel` *INT*	Specifies the maximum levels in clock skewing thus limiting the amount of clock tree levels and avoiding the creation of an unbalanced clock tree. This parameter sets an upper boundary when adding a buffer or pair of inverters to the clock nets. The software counts the number of levels in the clock tree branch that is being modified to ensure that the maximum level will be equal to or less than the value set by this parameter. *Default*: `0` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-deleteInst {true | false}`	Enables buffer deletion when doing timing, power, or area reduction. This option is not honored by the `-leakage` parameter of `signoffOptDesign` since it performs only swapping. *Default*: `true` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-disableGeometryChecks {true | false}`	

	Performs Vth swapping without checking the pins and geometry of a design. You can relax/stop/avoid the geometry checks during Vth swapping so that the tool just checks the cell size while performing Vth swapping. This will lead to more Vth swaps. The counterpart to this is that ecoRouting will be needed once the ECOs are applied since pin geometries might always match between old and new cells. *Default*: `false` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-drvMargin` *float*	Specifies the ratio by which DRV check would be relaxed or tightened. It scales the maximum capacitance and maximum transition constraints according to the margin decimal value specified. For example, specifying a value of 0.2 for this parameter tightens the margin (makes it more pessimistic) by 20 percent during optimization and specifying a value of -0.2 relaxes the margin (makes it more optimistic) by 20 percent. This margin is honored for DRV optimization. **Note**: The software only accepts values from -0.7 to +0.7. A positive value indicates tighter constraints. *Default*: `0` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-ecoFilePrefix` *string*	Specifies the prefix name for the ECO files generated. *Default:* `""` (empty string) **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-emType {all | avg | rms | peak}`	Specifies the current waveform calculation method when doing EM fixing. *Default*: `all`
`-fixCellEm {true | false}`	When set to `true`, fixes the cell-level signal electromigration violations during DRV fixing using resizing and additional transforming techniques. *Default*: `false` **Note:** • If set, this parameter must be applied before `signoffOptDesign -drv`. • If set during DRV fixing, the software will fix only the cell EM violations. This means that fixing of max_transition/max_capacitance/SI glitch/SI Xtalk/IR drop will be disabled.
`-fixClockDrv {true | false}`	Fixes maxCap/maxTran violations on the clock networks. This is performed only when the `signoffOptDesign -drv` command is run. *Default*: `false` **Note**: • For clock network DRV fixing, only the cells listed in the clock cell list (specified using the `setSignoffOptMode -clockCellList` parameter), is used. The DRV fixing is not performed if a cell list is not provided. • If set, this parameter must be applied before `signoffOptDesign`.

`-fixDataDrv {true | false}`	Fixes maxCap/maxTran violations on data nets. This is performed only when the `signoffOptDesign -drv` command is run. *Default*: `true`
`-fixGlitch {true | false}`	Performs SI glitch fixing during DRV fixing using resizing and buffering techniques. *Default*: `false` **Notes:** • If set, this parameter must be applied before `signoffTimeDesign`. • The glitch fixing is done before max_tran/max_cap fixing.
`-fixHoldAllowSetupDegrade {none | low | medium}`	
	This parameter guides the Hold fixing engine to determine what effort it has to put into resolving timing violations and how much flexibility it has to impact the Setup timing. When set to `low`, Hold optimization engine will buffer sink pins that have large Setup positive slack even though the net Setup slack is barely meeting timing. When set to `medium`, Hold optimization engine will behave the same as in the `low` mode but will also allow Setup TNS to degrade, if required. **Note:** • This parameter is honored by Hold optimization engine. • If set, this parameter must be applied before `signoffOptDesign`. • When the `-fixHoldAllowSetupDegrade` parameter is set to `low`, `medium`, or `none`, the `-fixHoldAllowSetupTnsDegrade` parameter is ignored. *Default*: `none`
`-fixHoldAllowSetupOptimization {true | false}`	
	When set to `true`, incremental Setup fixing is performed during Hold fixing in order to create more Setup margins on Hold violated nodes that could not be fixed earlier due to Setup timing conflict. *Default*: `false` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-fixHoldAllowSetupTnsDegrade {true | false}`	
	When active, the Setup Total Negative Slack can be degraded during Hold time violations fixing while still maintaining the Setup Worst Negative Slack in every Setup active views. *Default*: `false` **Note:** If set, this parameter must be applied before `signoffOptDesign`.

`-fixHoldVariationAware {true | false}`	When set to `true`, more sizing and Vth swapping is performed during Hold fixing to reduce the cost of buffer insertion. This is particularly effective when Hold timing paths have incremental delay due to SI or large SOCV impacted instances. **Note**: • This parameter is only honored by `signoffOptDesign -hold`. • If set, this parameter must be applied before `signoffOptDesign`. *Default*: `false`
`-fixHoldWithMargin float`	Specifies a slack margin value in nanoseconds for Hold time violations fixing. The purpose is to overfix any Hold violations by the amount of the margin value. This margin is not applied on nets that are already meeting the Hold target slack. *Default*: `0` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-fixIrDrop {true|false}`	Fixes the IR drop voltages for all instances in the design. *Default*: `false` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-fixMaxCap {true | false}`	Specifies that nets with max_cap violations should be fixed. *Default*: `true` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-fixMaxTran {true | false}`	Specifies that nets with max_tran violations should be fixed. *Default*: `true` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-fixOsusGlitch {true | false}`	When set to `true`, fixes nets with overshoot and undershoot glitches during DRV optimization. *Default*: `false`
`-fixSlackInRangeLowerLimit value_in_ns`	Sets the lower slack limit for a violated path to be considered during timing closure. The value is specified in ns. *Default*: `-1.00000001504747e+30`
`-fixSlackInRangeUpperLimit value_in_ns`	Sets the upper slack limit for a violated path to be considered during timing closure.The value is specified in ns. *Default*: `1.00000001504747e+30`
`-fixSiSlew {true | false}`	Reports SI slews violations and fixed during DRV optimization. This is run as the first step of optimization before proceeding for any other incremental optimization. When this parameter is set to `true`, it automatically sets the value of the `-fixMaxTran` parameter to `true`. *Default* : `false` **Note:** If set, this parameter must be applied before `signoffTimeDesign` and `signoffOptDesign`.

`-fixXtalk {true | false}`	Reduces the crosstalk on nets that have xtalk delta delays greater than the specified threshold value. *Default*: `false` **Note:** If set, this parameter must be applied before `signoffTimeDesign` and `signoffOptDesign`.
`-fixXtalkPreserveHold {true | false}`	When this parameter is set to `true`, the Xtalk fixing engine is not allowed to degrade Hold timing. *Default*: `false` **Note**: • This parameter is only honored by `signoffOptDesign -drv` when Xtalk fixing is enabled. • If set, this parameter must be applied before `signoffOptDesign`.
`-fixXtalkPreserveSetup {true | false}`	When this parameter is set to `true`, the Xtalk fixing engine is not allowed to degrade Setup timing. *Default*: `false` **Note**: • This parameter is only honored by `signoffOptDesign -drv` when Xtalk fixing is enabled. • If set, this parameter must be applied before `signoffOptDesign`.
`-holdTargetSlack` *float*	Specifies a target slack value in nanoseconds for hold timing optimize. *Default*: `0` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-holdTargetSlackPerView {viewName1 slack1 viewName2 slack2 …}`	When enabled, this parameter forces a specific hold target slack per view. If a given Hold active view is not listed in this parameter, the software uses the global Hold target slack set by the `-holdTargetSlack` parameter for that view. *Default*: `""` (empty string) **Note**: • This parameter is honored by Setup, Hold, Area, Power, and Leakage optimization engines. • If set, this parameter must be applied before `signoffOptDesign`.
`-holdXtalkDeltaThreshold` *value_in_ns*	Fixes the nets with the crosstalk delta delay value equal to or greater than the specified threshold value for hold views. The threshold value is specified in `ns`. *Default*: `0.3 ns` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-holdXtalkSlackThreshold` *value_in_ns*	Fixes nets with the slack threshold value equal to or less than a specified threshold value for hold views. The threshold value is specified in `ns`. *Default*: `1000 ns` **Note:** If set, this parameter must be applied before `signoffOptDesign`.

`-ignoreDrvChecks {true | false}`	Specifies that DRV checks should be ignored in setup/hold/area/leakage/dynamic optimization. *Default*: `false` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-keepTempfiles {true | false}`	Keeps all the temporary files generated by the distributed multi-mode multi-corner (DMMMC) and Tempus clients for debugging purposes. *Default*: `false` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-legalOnly {true | false}`	
	Enables the tool to use the physical data to search for legal location while doing any buffer insertion or instance movement. This ensures that there is no instance overlap at the end of the ECO fixing stage, and after sourcing the ECO file in the implementation tool, you can directly run ECO routing. When this parameter is set to `false`, `signoffOptDesign` will not check for legal location although physical data is provided. In this mode, `signoffOptDesign` behaves in a Logical mode flow, which means that physical data will be ignored and this might create an instance overlap and will not check local density while inserting buffers. So it is recommended that you first run ECO fixing in the legal-only mode (which is the default mode) and then, depending on the requirement, you may perform an incremental run by setting `-legalOnly` to `false`. *Default*: `true` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-loadCellList` *cell_list*	Specifies the list of load cells to be inserted during Hold fixing, regardless of whether those cells have a dontUse attribute. *Default*: `""`(empty string) **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-loadEcoOptDb` *dir_name*	Specifies the path of the directory where previously generated ECO Timing DB files are kept. If not specified, the ECO Timing DB files will be generated. *Default:* `""` (empty string) **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-loadIrdropDb` *dir_name*	Specifies the path of directory where previously generated IR drop database files are stored. *Default:* `""` (empty string) **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-maxCapMargin` *cap_margin_value*	Specifies the ratio by which the max_cap check should be relaxed or tightened. *Default*: `0` *Minimum value*: -10.000000 *Maximum value*: 0.990000 **Note:** If set, this parameter must be applied before `signoffOptDesign`.

`-maxLocalDensity` *threshold_value*	This parameter lets you set a local placement density threshold that the software will honor during timing optimization. When the local density for a given area exceeds the threshold, the software stops adding or moving more instances in that local area. The default value for this option is -1, which means the software will not do any local placement density checks because, by default, it is running with `-legalOnly true` parameter. This parameter accepts values between 0 and 1. For example, to allow a maximum local placement density of 98%, set the value to 0.98. *Default*: `-1` **Note**: • This parameter is honored by DRV, Setup, and Hold optimization engines. • If set, this parameter must be applied before `signoffOptDesign`.
`-maxOptRunTime` *INT*	Specifies the time out period for only the timing/power optimization phase after which `signoffOptDesign` will exit the session. *Default:* `0` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-maxPaths` *INT*	Specifies the worst path found for each endpoint up to the number specified by the -maxPaths parameter during ECO Timing DB generation. By default, the tool will automatically set a value for `-maxPaths` that would lead to good PBA coverage of the design within reasonable run time. You can overwrite this by setting a specific `-maxPaths` value, but it is recommended to keep it larger than 1 million. In any case, you must ensure that `-maxPaths` value is superior or equal than the `-nworst` value multiplied by the number of GBA violated endpoints. *Default*: `-1` **Note**: • This parameter is only active when the `-retime` parameter is set. • If set, this parameter must be applied before `signoffTimeDesign`.
`-maxRunTime` *INT*	Specifies the time out period, in minutes, after which `signoffOptDesign` will exit the session. *Default*: `0` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-maxSlack` *float*	Specifies the maximum slack value to be used during ECO Timing DB generation when using PBA. Setting a positive -maxSlack value forces the software to retime positive GBA slack paths so that ECO optimization engine can benefit from an extra PBA Setup slack credit when doing Hold fixing or Power recovery. A positive -maxSlack value implies larger runtime during ECO Timing DB generation, but the software will retime only as many paths as what is specified through the `-maxPaths` parameter. *Default*: `0` **Note**: • This parameter is only active when the `-retime` parameter is set. • If set, this parameter must be applied before `signoffTimeDesign`.

`-maxTranMargin` *`tran_margin_value`*	Specifies the ratio by which the max_tran check should be relaxed or tightened. *Default*: `0` *Minimum value*: -10.000000 *Maximum value*: 0.990000 **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-numReportPaths` *`INT`*	Specifies the number of paths printed after signoff timing analysis per view. *Default*: `50` **Note:** If set, this parameter must be applied before `signoffTimeDesign`.
`-nworst` *`INT`*	Specifies the nworst control for path-based analysis. *Default*: `-1` **Note**: • This parameter is only active when the `-retime` parameter is set. • If set, this parameter must be applied before `signoffTimeDesign`.
`-optimizeCoreOnly {true | false}`	Performs only reg2reg optimization, when set to `true`. *Default*: `false` **Note**: • This parameter is not applicable for DRV optimization. • If set, this parameter must be applied before `signoffOptDesign`.
`-optimizeReplicatedModules {true | false}`	
	Allows the software to perform timing optimization inside replicated hierarchies while generating only one ECO file per sets of clones. The cell name of the replicated hierarchies have to be listed in a file pointed by the `-partitionListFile` option. *Default*: `false` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-optimizeSequentialCells {true | false}`	
	Allows the software to perform timing optimization for sequential cells. *Default*: `true` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-partitionListFile` *`string`*	Specifies the file containing list of partitions in the design. *Default:* `""` (empty string) **Note:** If set, this parameter must be applied before `signoffOptDesign`.

`-pbaEffort {medium | high}`	When high effort mode is specified, all the optimization engines perform more aggressive timing closure or area/power recovery with respect to path-based (PBA) timing and the expectation is to get better quality of results. In some cases, when large amount of ECOs are done in this mode, the final timing signoff might show some setup timing degradation, which can be recovered by running a second ECO loop doing only setup timing recovery with the `-setupRecovery parameter` set to `true`. *Default*: `medium` **Note:** If set, this parameter must be applied before `signoffTimeDesign` and `signoffOptDesign`.
`-pbaEcoRoute {true|false}`	When set to `true`, enables `ecoRoute` to use accurate path-based analysis (PBA) timing during the ECO signoff timing/power closure flow. This parameter should be specified at the start of the `signoffOptDesign`/`signoffTimeDesign` flow. *Default*: `false`

`-postMask {true | false}`	Enables the timing closure engine to perform the following netlist changes without touching the base layer mask:
	• Inserts the Gate Array buffers or an inverter pair in place of existing Gate Array filler cells.
	• Resizes regular cells to Gate Array cells. It also resizes Gate Array cells to Gate Array cells.
	• Deletes a regular cell.
	The software automatically identifies the Gate Array cells by checking which library instances have the same SITE name as the Gate Array filler cells specified by the user. Here, empty spaces are filled up by the Gate Array filler cells to ensure 100% placement density.
	Note:
	• This is supported by DRV, Setup, and Hold fixing.
	• When `setTieHiLoMode -cell {{tieLo_name tieHi_name}}` setting is used, the Metal ECO flow will insert the TieLo cells to connect dangling input pins, in case the existing TieLo cells in the floorplan are extremely far or too overloaded.
	Example:
	The following command enables Hold fixing in the metal ECO mode by allowing the listed Gate Array filler cells to be deleted or inserted during the ECO process.
	`setSignoffOptMode -postMask true`
	`setSignoffOptMode -useGaFillerList {GFILL1BWP GFILL2BWP GFILL4BWP GFILL10BWP}`
	`signoffOptDesign -hold`
	Default: `false`

`-postStaTcl FILE`	Specifies the file containing the TCL code that will be executed immediately after signoff timing analysis per view. *Default:* `""` (empty string) **Note**: • These TCL codes should follow Tempus syntax. • If set, this parameter must be applied before `signoffTimeDesign`.

`-powerAware {true | false}`	Enables power awareness in all optimizers and transforms. Tempus ECO provides the ability to take power as part of the overall cost function when doing timing closure. When this parameter is enabled before the start of an optimization session, the current optimization and any subsequent optimizations (if running in incremental mode) checks for power changes for all optimizers and transforms during any changes in the design. In any case, the first priority remains getting the best final timing but enabling this option will get there with possibly slighter ECO although resulting in lesser final power. For example, during to fix a Setup timing path violation, by default the tool would potentially choose to size one instance to a high leakage cell, while with `-powerAware` mode, the tool would try to get to same timing by doing multiple sizing to non-high leakage cells. **Note** • This feature requires power activity to be annotated on the netlist with `report_power` command, either through external activity file (recommended) or using propagated activity based on some default toggling rate. • This feature honors the `-powerOptFocus` option, which means that the user can decide to force the tool in reducing more total or leakage or dynamic power increase. *Default*: `false`
`-powerOptFocus total | leakage | dynamic | leakageOnly | dynamicOnly`	
	Specifies whether a given netlist should be optimized only for leakage or dynamic power reduction, or more for leakage or dynamic power while still reducing the overall total power during total power optimization. The following options are available: • `total`: The power optimization engine focuses on getting the optimal total power in the design. • `leakage`: The power optimization engine optimizes total power but focuses more on leakage power reduction than on dynamic power reduction. • `dynamic`: The power optimization engine optimizes total power but focuses more on dynamic power reduction than on leakage power reduction. • `leakageOnly`: The power optimization engine focuses only on leakage power reduction. • `dynamicOnly`: The power optimization engine focuses only on dynamic power reduction. *Default*: `total` **Note**: • This parameter is honored by `signoffOptDesign -power` and also by `signoffOptDesign -drv/-setup/-hold` when the `-powerAware` parameter is set to `true`. • If set, this parameter must be applied before `signoffOptDesign`.
`-prefixName` *string*	Specifies the prefix to be assigned to the new instances inserted. *Default:* `ESO` **Note:** If set, this parameter must be applied before `signoffOptDesign`.

`-preserveFiller {true | false}`	When set to `true`, saves and restores filler states during signoff timing optimization. This helps improve the runtime as it saves the time taken to re-insert all the fillers. *Default*: `true`
`-preStaTcl` *FILE*	Specifies a file containing the TCL code that will be executed immediately prior to signoff timing analysis per view. *Default:* `""` (empty string) **Note**: • These TCL codes should follow Tempus syntax. • If set, this parameter must be applied before `signoffTimeDesign`.
`-resizeInst {true | false}`	Enables/disables resizing transform. *Default*: `true` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-retime {none | aocv | path_slew_propagation | aocv_path_slew_propagation}`	
	Selects the retime mode for path-based analysis. *Default*: `none` **Note:** If set, this parameter must be applied before `signoffTimeDesign` and `signoffOptDesign`.
`-retimeDepth {default | infinite}`	
	Retimes depth for exhaustive path based analysis. *Default*: `default`
`-retimeDepthEarly {default | infinite}`	
	Retimes depth for exhaustive path based early analysis. *Default*: `default`
`-retimeMode {none | path | exhaustive}`	
	Retimes mode for path based analysis. *Default*: `path`
`-routingCongestionAware {true | false}`	
	Checks routing congestion before doing any netlist change. *Default*: `false` **Note:** If set, this parameter must be applied before `signoffOptDesign`.

`-routeCongestionThreshold` *`threshold_value`*	This parameter allows you to change the threshold at which the software considers a local area to be too routing congested to allow any ECO change. The default value is 0.7. To tighten the routing congestion checks, apply a value between 0 and 0.7. To relax the routing congestion checks, apply a value between 0.7 and 1. *Default*: `0.7` *Min value*: `0` *Max value*: `1` **Note**: • This parameter is honored by Drv, Setup, Hold, Area, and Power optimization engines. • If set, this parameter must be applied before `signoffOptDesign`.
`-saveEcoOptDb` *`dir_name`*	Specifies a path of directory where generated database files are kept. *Default*: `ecoTimingDB` **Note:** If set, this parameter must be applied before `signoffTimeDesign`.
`-selectHoldEndpoints` *`file_name`*	Specifies a file containing the list of selected endpoints for hold fixing. **Note**: • When including endpoints, the `-optimizeCoreOnly` option must be set to `false`. • This parameter is set during ECO optimization. *Default:* `""` (empty string) For details about the file syntax, see Path Group Support in Timing Optimization in the *Innovus User Guide*.
`-selectDrvNetFile` *`file_name`*	Specifies a file containing the list of nets to be optimized during DRV fixing. The fixing of max_transition and max_capacitance violations occurs only on the specified nets and the other violating nets are ignored. The file specified includes one net per line. **Note**: Wild cards and regular expressions are not supported in this file. *Default*: `""` (empty string) **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-selectDrvPinFile` *`file_name`*	Specifies a file containing the list of instance pins to be optimized during DRV fixing. The fixing occurs only on the specified instance pins and the other violating pins are ignored. The file specified includes one pin per line. **Note**: Wild cards and regular expressions are not supported in this file. *Default*: `" "` (empty string) **Note:** If set, this parameter must be applied before `signoffOptDesign`.

`-selectSetupEndpoints` *file_name*	Specifies a file containing the list of selected endpoints for setup fixing. **Note**: • When including endpoints, the `-optimizeCoreOnly` option must be set to `false`. • If set, this parameter must be applied before `signoffOptDesign`. *Default:* `""` (empty string) For details about the file syntax, see Path Group Support in Timing Optimization in the *Innovus User Guide*.
`-setupRecovery {true | false}`	Specifies power-aware setup recovery for setup fixing. *Default*: `false` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-setupTargetSlack float`	Specifies a target slack value in nanoseconds for setup timing optimization. *Default*: `0` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-setupTargetSlackPerView {viewName1 slack1 viewName2 slack2 …}`	When enabled, this parameter forces a specific setup target slack per view. If a given Setup active view is not listed in this parameter, the software will use the global Setup target slack set by the `-setupTargetSlack` parameter for that view. *Default*: `""` (empty string) **Note**: • This parameter is honored by Setup, Hold, Area, Power, and Leakage optimization engines. • If set, this parameter must be applied before `signoffOptDesign`.
`-setupXtalkDeltaThreshold` *value_in_ns*	Fixes the nets that have a crosstalk delta delay value equal to or greater than a specified threshold value for setup views. The threshold value is specified in `ns`. *Default*: `0.3 ns` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-setupXtalkSlackThreshold` *value_in_ns*	Fixes the nets with the slack threshold value equal to or less than a specified threshold value for setup views. The threshold value is specified in `ns`. *Default*: `1000 ns` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-skewCoreOnly {true | false}`	When this parameter is set to `true`, the skewing engine is not allowed to skew sequential elements that are part of an IO boundary timing path. *Default* : `false` **Note**: • This parameter is only honored by `signoffOptDesign -setup`. • If set, this parameter must be applied before `signoffOptDesign`.

`-skipDrvNetfile` *file_name*	Specifies a file containing the list of nets to be excluded during DRV fixing. The fixing of max_transition and max_capacitance violations occurs on nets other than specified nets. The file specified includes one net per line. **Note**: Wild cards and regular expressions are not supported in this file. *Default:* `""` (empty string) **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-specifyHoldEndpointsMargin` *string*	
	Specifies endpoint-specific margin for Hold fixing. *Default:* `""` (empty string) For details about the file syntax, see Path Group Support in Timing Optimization in the *Innovus User Guide*. **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-specifySetupEndpointsMargin` *string*	
	Specifies endpoint-specific margin for Setup fixing. *Default:* `""` (empty string) For details about the file syntax, see Path Group Support in Timing Optimization in the *Innovus User Guide*. **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-swapInst {true | false}`	Enables/disables VT swap resizing. The setup/leakage optimizers do not honor this option and always perform VT-swapping. *Default*: `true` **Note:** If set, this parameter must be applied before `signoffOptDesign`.
`-upsizeInPowerOpt {true | false}`	When this parameter is set to `true`, it allows the power optimization engine to perform cell upsizing in order to recover more power. *Default*: `false` **Note**: • This parameter is only honored by `signoffOptDesign -power`. • If set, this parameter must be applied before `signoffOptDesign`.
`-useGaFillerList` *list_of_Gate_Array_filler_cells_name*	
	Enables the software to list all the Gate Array filler cells that can be deleted or inserted back. **Note**: This parameter is applicable in the Metal ECO mode only. *Default*: `""`
`-usePfcDecapList` *list_of_decap_cells*	
	Lists all the programmable decap cells that are allowed to be removed in post-mask mode. *Default*: `{}`
`-usePfcFillerList` *list_of_filler_cells*	

	Lists all the programmable filler cells that are allowed to be removed/reinserted in post-mask mode. *Default*: `{}`
`-usePfcRegularList` *list_of_regular_cells*	

	Lists all the programmable regular logic/ECO cells that are allowed to be inserted in place of filler/decap cells during optimization in post-mask mode.
	Default: `{}`
	Example:
	The following set of commands will allow Hold timing closure in post-mask mode using programmable filler cells.
	`setSignoffOptMode -postMask true`
	`setSignoffOptMode -usePfcFillerList {ln10_ecofiller40x … ln10_ecofiller4x}`
	`setSignoffOptMode -usePfcDecapList {ln10_ecofillsgcap40x … ln10_ecofillsgcap4x}`
	`setSignoffOptMode -usePfcRegularList {ln10_ecodlybt01 ln10_ecobuft04 … ln10_ecoand02 … ln10_ecoinv04 …}`
	`signoffOptDesign -hold`

`-verbose {true | false}`	Specifies whether the command should output log file reporting with verbosity or not. *Default*: `false` **Note:** If set, this parameter must be applied before `signoffOptDesign`.

Related Information

- `getSignoffOptMode`
- `signoffOptDesign`
- `signoffTimeDesign`
- The following topics in the *Innovus User Guide*
  - Running MMMC SignOff ECO within Innovus
  - Using Signoff Timing Analysis to Optimize Timing and Power
  - Signoff Timing Analysis in Innovus using Timing Debug
  - Optimization in Path-Based Analysis (PBA) Mode
  - Total Power Optimization
  - Setup Timing Recovery After a Large Leakage or Total Power Optimization
  - Achieving the Best Results for Total Power Optimization
  - Path Group Support in Timing Optimization
  - Top Down Block ECO flow using Tempus Signoff Timing

setTieHiLoMode

```
setTieHiLoMode
[-help]
[-reset]
[-cell {{tieHi1 tieLo1} {tieHi2 tieLo2} ...}]
[-createHierPort {true | false}]
[-honorDontTouch {true | false}]
[-honorDontUse {true|false}]
[-maxDistance distanceValue]
[-maxFanout fanOutValue]
[-modulePrevention {true | false}]
[-prefix prefixName]
[-reportHierPort {true | false}]
```

Controls certain aspects of how the software adds or deletes tie-high and tie-low cells.

See `getTieHiLoMode` to query current settings for the `setTieHiLoMode` command.

The parameters of the `setTieHiLoMode` command affect the behavior of the following commands:

- `addTieHiLo`
- `deleteTieHiLo`

  Note: Parameters specified by `addTieHiLo` and `deleteTieHighLo` supersede those specified by the `setTieHiLoMode` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setTieHiLoMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setTieHiLoMode`
`-cell {{tieHi1 tieLo1} {tieHi2 tieLo2} ...}`	Specifies the pairs of tie cells to be used by the `addTieHiLo` command. You can specify a maximum of two tie-cells in a cell list, where one cell must be a tie-high driver, and the other a tie-low driver. A single cell with both tie connections, high and low, can be specified as below. `setTieHiLoMode -cell {{TieHiLo1} {TieHiLo2} {TieHiLo3} ….}` The software selects the nearest cell pair to make new tie connections during the postMask flow. *Default:* `""` (empty string)
`-createHierPort {true | false}`	Allows the added tie cells to connect to tie pins across hierarchical boundaries if other constraints, such as maximum fanout and maximum distance, allow it. *Default:* `false`
`-honorDontTouch {true | false}`	Allows both `addTieHiLo` and `deleteTieHiLo` commands to honor the `dont_touch` property set on instances, nets, cells, and modules. *Default:* `false`
`-honorDontUse {true|false}`	Honors `dont_use` attribute on cells. *Default:* `false`
`-maxFanout` *`fanOutValue`*	Specifies the number of tie-pins a tie-net can drive. A value of `0` implies no fanout constraint. *Default*: `0`
`-maxDistance` *`distanceValue`*	Specifies the maximum Manhattan distance, in microns, between the tie-cell driver and the tie-pins. A value of `0` implies no distance constraint. *Default*: `0`
`-modulePrevention {true|false}`	When this parameter is set to `true`, the `addTieHiLo` command adds tie-cells for module instantiation port. Tie-cells may also be added inside the module to tie the output port of the module so that module function can be prevented. *Default*: `false`
`-prefix` *`prefixName`*	Specifies a prefix for the added tie-cell instances. *Default*: `""`
`-reportHierPort {true | false}`	When the value of `-createHierPort` is `true`, reports created port connections to a file named `tiehilo.rpt`. *Default:* `false`
`-reset`	Resets parameters to their default values. The `-reset` parameter *must* be the first parameter specified. If you specify `-reset` by itself, the software resets all `setTieHiLoMode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported.

Example

- The following command sets `TIEHI` and `TIELO` cells a maximum distance of 20 microns between the tie-cell pin and the tie-off input pins and a tie-net can drive up to 10 tie-pins:
  ```
  setTieHiLoMode -maxFanout 10 -maxDistance 20 -cell "TIEHI TIELO"
  ```
- The following command resets the `-maxDistance` parameter to its default value:
  ```
  setTieHiLoMode -reset -maxDistance
  ```
- The following command resets all `setTieHiLoMode` parameters to their default values:
  ```
  setTieHiLoMode -reset
  ```

Related Information

- `addTieHiLo`
- `deleteTieHiLo`
- `getTieHiLoMode`
- `verifyTieCell`
- The following topic in the *Innovus User Guide*
  - Adding Logical Tie-Off Cells

setUsefulSkewMode

```
setUsefulSkewMode
[-help]
[-opt_skew_apply_delay_limits_to_full_flow {true|false}]
[-opt_skew_delay_pre_cts {true|false}]
[-opt_skew_macro_only {true | false}]
[-opt_skew_max_allowed_delay delay]
[-opt_skew_min_allowed_delay delay]
[-opt_skew_no_boundary {true | false}]
[-reset]
```

Sets global parameters for the `skewClock` command.

Use the `getUsefulSkewMode` to return the current settings for the `setUsefulSkewMode` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `setUsefulSkewMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setUsefulSkewMode`
`-opt_skew_apply_delay_limits_to_full_flow {true | false}`	
	When set to `true`, specifies that the useful skew limit is to be applied to the whole flow. With this setting, the Early Clock Flow is modified so that pre-existing tool-generated useful skews count towards the limit imposed by the `-opt_skew_max_allowed_delay` parameter. The tool-generated useful skew comprises any pin insertion delay (PID) of category useful_skew or global_skew. PIDs of other categories are not considered. By default, this parameter is set to `false`, which means the pre-existing skews are not considered while calculating the limit imposed by using the `-opt_skew_max_allowed_delay` parameter. *Default*: `false`
`-opt_skew_delay_pre_cts {true |false}`	Specifies that the useful skew feature is allowed to delay sequential elements in preCTS stage, in addition to just advancing them. This enables a more aggressive algorithm which is recommended when timing closure is gated by optimal skewing. *Default:* `true`
`-opt_skew_macro_only {true | false}`	Allows you to limit skewing only for macro cells. When enabled, regular sequential elements (such as flip-flops) will not be skewed. *Default*: `false`

`-opt_skew_max_allowed_delay` *delay*	Limits the amount of slack (in nanoseconds) that the software can borrow from neighboring flip-flops when performing useful skew operations. The Innovus delay calculation and RC extraction methods might differ from those of sign-off tools, so other setup violations might occur if the software borrows too much slack. By having control over slack borrowing, you can prevent these setup violations. Limiting borrowed skew also limits the clock tree skew to avoid large hold violations. *Value Range: min*=`0` *max*=`1000` *Default:* `1ns` **Note**: • This parameter is not honored in the postCTS mode of `skewClock`. • In the non-early clock flow, when `skewClock` is triggered inside `optDesign`, and this parameter is set to its default value, the default value of this parameter will change from 1 to become a certain percentage of clock cycle. This is determined by the software on the basis of internal calculations.
`-opt_skew_min_allowed_delay` *delay*	Allows the tool to skip any skewing that would be smaller than the minimum delay that is set. The purpose is to avoid creating very small delay macro models which would not be correctly honored during CTS stage but also to reduce the amount of macro models generated. *Value Range*: *min*=`0` *max*=`1000` *Default*: `0` **Note:** • This is supported in both standalone `skewClock` command and for `-usefulSkew` called within `optDesign`. • Enable the `-usefulSkew` parameter in `optDesign` by setting `setOptMode -opt_skew true`
`-opt_skew_no_boundary {true |false}`	Excludes or includes boundary sequential cells in addition to ordinary sequential elements in useful skew calculations. *Default:* `false`
`-reset`	Resets parameters to their default values. The `-reset` parameter *must* be the first parameter specified. If you specify only `-reset`, the software resets all `setUsefulSkewMode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. Any values that do not already match the default value and are reset will be reported.

Examples

- The following command resets the `-delayPreCts` parameter to its default value:
  ```
  setUsefulSkewMode -reset -opt_skew_delay_pre_cts
  ```
- The following command resets all `setUsefulSkewMode` parameters to their default values:
  ```
  setUsefulSkewMode -reset
  ```
- The following command forces the tool to skew at minimum by `30ps` any elements that can be skewed to improve the setup timing slack:
  ```
  setUsefulSkewMode -opt_skew_min_allowed_delay 0.03
  skewClock /optDesign -preCts
  ```

- The following command allows the user to limit skewing only for macro cells:

  ```
  setUsefulSkewMode -opt_skew_macro_only true
  skewClock or optDesign -preCts/-postCts
  ```

Related Information

- `getUsefulSkewMode`
- `setOptMode`
- `skewClock`
- The following topic in the *Innovus User Guide*
  - Optimizing Timing using Useful Skew

signoffOptDesign

```
signoffOptDesign
[-help]
[-all]
[-area]
[-drv]
[-dynamic]
[-hold]
[-leakage]
[-noEcoRoute]
[-outDir dir_name]
[-reportFullClockPath]
[-power]
[-prefix prefix_name]
[-setup]
```

Runs timing, area, DRV, dynamic or leakage optimization on signoff timing.

This command must be run on a design that has gone through the implementation flow. It allows you to fix timing violations or to improve area and power visible in signoff STA mode while staying in the implementation tool.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `signoffOptDesign` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man signoffOptDesign`
`-all`	Performs DRV, Setup, Hold, and Power optimization in the path-based analysis (PBA) mode. It also includes embedded calls to ecoRoute, parasitic extraction and signoff timing analysis. **Note**: The all-in-one mode does not replace all customized fine-tuned flows. It provides the Innovus users easy access to Tempus ECO technology. The existing Tempus ECO flow using individual optimization steps within Innovus is still supported and can sometimes be more efficient than the all-in-one mode.
`-area`	Reclaims area on the netlist without causing any DRV/Setup/Hold degradation.
`-drv`	Optimizes DRV (maximum capacitance and maximum transition) violations. DRV optimization supports buffer addition and resizing.
`-dynamic`	Prints initial and final power numbers. It optimizes the dynamic power to reduce the total power consumption. Dynamic power optimization supports buffer deletion ,resizing, and swapping.
`-hold`	Triggers hold fixing in signoff timing mode.
`-leakage`	Optimizes leakage power. Leakage optimization supports VT swapping.
`-noEcoRoute`	Allows `signoffOptDesign` command to skip ECO routing and final signoff STA.
`-outDir` *dir_name*	Specifies the directory in which the reports will be saved. **Note**: In case `signoffOptDesign` is reusing ECO timing DB generated earlier by `signoffTimeDesign` or when the `-noEcoRoute` parameter is applied, no timing reports are saved.
`-power`	Optimizes total power in the design. During total power optimization, the software performs swapping and sizing on both the combinational and the sequential cells, and also removes buffers. This reduces leakage, internal, and switching power in the design.
`-prefix` *prefix_name*	Specifies the prefix for the timing report file names.
`-reportFullClockPath`	When this parameter is specified, the detailed timing report that is generated contains the full clock path.
`-setup`	Optimizes setup timing violations. Setup optimization supports buffer addition, VT swapping and resizing.

Examples

- Use the following commands to perform setup optimization followed by a leakage power optimization. The setup optimization is with no -ecoRoute, while the leakage power optimization calls the -ecoRoute along with final signoff STA results.

  ```
  signoffOptDesign -setup -noEcoRoute
  signoffOptDesign -leakage
  ```

- Use the following commands to perform an area reclaim optimization that is based on external spef files:

  ```
  spefIn -rc_corner rc_max rc_max.spef.gz
  spefIn -rc_corner rc_min rc_min.spef.gz
  signoffTimeDesign -reportOnly
  signoffOptDesign -area
  ```

- Use the following command to dump out the final STA results in a folder named "RPT". The timing report files inside the "RPT" folder will be named "SIGNOFF_ECO_*".

  ```
  signoffOptDesign -hold -outDir RPT -prefix SIGNOFF_ECO
  ```

- Use the following set of commands to reclaim total power in the design:

  ```
  report_power
  setSignoffOptMode -deleteInst true -optimizeSequentialCells true
  signoffOptDesign -power
  ```

- Use the following set of commands to perform DRV, Setup, Hold, and Power optimization in the PBA mode:

  ```
  setMultiCpuUsage -localCpu 16 -remoteHost 4 -cpuPerRemoteHost 4
  setSignoffOptMode -allowSkewing true -clockCellList {.....}
  signoffOptDesign -all
  ```

  Innovus will run Tempus ECO for PBA-based DRV, timing and power closure. The Setup fixing will be done using useful skew technology. Signoff timing analysis will use 4 remote hosts each using 4 CPUs while ECO fixing, ECO routing, and parasitic extraction will use 16 CPUs.

Related Information

- ecoRoute
- getSignoffOptMode
- report_power
- setSignoffOptMode
- signoffTimeDesign
- spefIn
- The following topics in the *Innovus User Guide*
  - Running MMMC SignOff ECO within Innovus
  - Using Signoff Timing Analysis to Optimize Timing and Power
  - Signoff Timing Analysis in Innovus using Timing Debug
  - Optimization in Path-Based Analysis (PBA) Mode
  - Total Power Optimization
  - Setup Timing Recovery After a Large Leakage or Total Power Optimization
  - Achieving the Best Results for Total Power Optimization

- Path Group Support in Timing Optimization
- Top Down Block ECO flow using Tempus Signoff Timing

skewClock

```
skewClock
[-help]
[-hold]
[-postCTS]
[-postRoute]
```

Modifies the clock arrival time on sequential elements in order to improve the data path timing between two sequential elements. The `skewClock` command outputs depend on whether the clock tree is built or not.

- In preCTS mode, `skewClock` generates two files: a scheduling file and a latency file.
- In postCTS and postRoute mode, `skewClock` inserts buffers and/or inverters in an existing clock tree. In addition, resizing of buffers and/or inverters can also be committed.

> Cadence recommends using `setOptMode -opt_skew` and `optDesign -preCTS`, `-postCTS`, or `-postRoute` to run useful skew optimization during timing optimization. For information on how to use `optDesign`, see Optimizing Timing in the *Innovus User Guide*.

PreCTS Mode

The `skewClock` command identifies elements that can be advanced or delayed, and computes target delay for each. An advancable element is a sequential gate on which the clock signal can be safely made to arrive early without worsening the slack on the critical path. In general, this is true for sequential elements where the slack on the output pin (SQ) is negative (SQ < 0) and is worse than the slack on the input pin (SD) : (SQ < SD).

The `skewClock` command generates the following files:

- **Scheduling file**: The CTS scheduling file lists the delays to consider on sequential elements by the clock tree synthesis tool. When loaded in CTS, the scheduling file constrains CTS to build a tree that fulfills the targets computed by `skewClock`.
  The scheduling file must be loaded in addition to the original clock tree specification file (see "Examples" section below).
  The syntax of the scheduling file is as follows:
  `Macromodel pin lefPinName maxRiseDelay minRiseDelay maxFallDelay minFallDelay inputCap`
  An entry in the configuration file is available to specify the scheduling file:
  `Set rda_input ui_scheduling_file scheduling_file.cts`
  When the design is saved, the scheduling file is copied in the design directory, and the line above is added to the configuration file of the saved design.
- **Latency file**: This file lists the delays to be considered on sequential elements by the timing analysis tool. These delays are computed by `skewClock`.
  The file syntax is as follows:
  `set_clock_latency rise delaypinName`
  The timing engine takes the latency file into account when the following parameter is specified:
  `setAnalysisMode -usefulSkew true`
  When building the timing graph, the Innovus software first loads the timing constraints, and then the latency file, as specified in the configuration file, for example
  `set_rda_input (ui_latency_file) latency_file.sdc`
  When the design is saved, the latency file is copied to the design directory, and the line above is added to the configuration file of the saved design.

PostCTS and PostRoute Mode

The `skewClock` command identifies delayable elements and delays the clock for each element. A delayable element is a sequential element on which the clock signal can be safely delayed without worsening the slack on the critical path. In general, this is true for each sequential element where the slack on the data pin (SD) is negative (SD<0) and is worse than the slack on the output pin (SQ) : (SD < SQ). The `skewClock` command inserts and/or resizes buffers and/or inverters on an existing clock tree.

- The skewClock net name prefix is `FE_%sN`
- The skewClock instance name prefix is `FE_%sC`
- The default instance name added during useful skew optimization is `FE_USKC`
- The default net name added during useful skew optimization is `FE_USKN`

If SD > SQ, the elements before the D pin can be advanced, and skewClock can do resizing on the buffers and/or inverters to compensate the timing on Q pin.

Note: To control the buffer and inverter cells used by `skewClock`, use `set_ccopt_property buffer_cells` and `set_ccopt_property inverter_cells`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `skewClock` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man skewClock`
`-hold`	Allows optimization for hold timing.

`-postCTS`	The `skewClock` command allows the use of an algorithm in the postCTS mode. When this parameter is specified, only setup timing optimization is performed. When it is specified along with the `-hold` parameter, only hold timing optimization is performed.
	This means,
	• The `skewClock -postCTS` command will only optimize setup timing.
	• The `skewClock -postCTS -hold` command will only optimize hold timing, without degrading setup timing.
	To optimize both setup and hold violations, run the following commands:
	`skewClock -postCTS`
	`skewClock -postCTS -hold`

`-postRoute`	The `skewClock` command allows the use of an algorithm in the PostRoute mode. It forces the command to end with ECO routing for the modified clock nets. An `ecoRoute` is followed by clock skewing. The difference between PostCTS mode and PostRoute mode is that the former is not limited to be legal only, while the latter is in a legal only mode.
	When this parameter is specified, only setup timing optimization is performed. When it is specified along with the `-hold` parameter, only hold timing optimization is performed.
	This means,
	• The `skewClock -postRoute` command will only optimize setup timing.
	• The `skewClock -postRoute -hold` command will only optimize hold timing, without degrading setup timing.
	To optimize both setup and hold violations, run the following commands:
	`skewClock -postRoute`
	`skewClock -postRoute -hold`

Examples

- The following commands show how to use `skewClock` in preCTS flow without using `optDesign`:

  ```
  setAnalysisMode -usefulSkew true
  timeDesign -preCTS
  skewClock
  timeDesign -preCTS
  ccopt_design
  timeDesign -postCTS
  ```

  Note: The recommended method for useful skew in preCTS mode is as follows:

  ```
  setOptMode -opt_skew true
  optDesign -preCTS
  ```

- The following example shows how to use `skewClock` in postCTS mode as an incremental fixing after normal postCTS optimization:

  ```
  ccopt_design
  skewClock -postCTS
  timeDesign -postCTS
  ```

Related Information

- `getLatencyFile`
- `getSchedulingFile`
- `getUsefulSkewMode`
- `setLatencyFile`
- `setNanoRouteMode`
- `setSchedulingFile`
- `setUsefulSkewMode`
- The following topic in the *Innovus User Guide*
  - Optimizing Timing using Useful Skew

45

TSV Design Commands

- add_obj_on_bump
- addTSV
- assignTSV
- create_stress_rule
- createTSVNoLoadSPEF
- deleteTSV
- read_codesign_die_abstract
- read_design_stack_config
- readBumpLocation
- readDieAbstract
- readTSVConfig
- unassignTSV
- verify_stacked_die
- write_codesign_die_abstract
- write_design_stack_config
- writeBumpLocation
- writeDieAbstract
- writeMicroBumpMappingFile
- writeVSMappingFile

add_obj_on_bump

```
add_obj_on_bump
[-help]
{{-cell bump_cell_name | -selected | -feedthru } {-relative_pin_layer pin_layer_name |
-rect {x1 y1 x2 y2} | -polygon {{x1 y1} {x2 y2}...}}}
[-check_only bump_name | -write_script filename]
{[-shape layer+] [-placement_blockage] [-routing_blockage layer+] [-physical_pin
layer+]}
[-shape layer+ [-check_only bump_name] [-incremental][-shape_keep_net ]]
[-feedthru [-cdl_map_file cdl_map_file_name]]
```

Adds shapes on the TSV bumps. You can use this command to add shapes on all the specified TSV bumps or the specified bump. The shape size could be user-defined or same as bump pin on the specified layer.

Parameters

`-help`	Prints a brief description of the `add_obj_on_bump` command usage. For a detailed description of the command, use the `man` command: `man add_obj_on_bump`
`-cdl_map_file` *`cdl_map_file_name`*	
	Specifies the output cdl map file for LVS.
`-cell` *`bump_cell_name`*	Specifies the name of the bump cell on which the shape is to be added.
`-check_only` *`bump_name`*	Checks if there is a shape on the specified bump and outputs a report file. The `-check_only` option cannot be used with `-write_script`. ⚠ The `-check_only` option can be used only when `-shape` is specified.
`-feedthru`	Adds shapes on the bump that contains the TSV layer and whose net is a feedthrough net.

`-incremental`	Adds shapes in the incremental mode. It deletes the shapes without a bump, and adds new shapes on the bumps without shapes. The -incremental parameter only checks the shapes on the same layer.
	In the following sequence of commands, the first command is `add_obj_on_bump -shape M1` and the second command is `add_obj_on_bump -shape M2 -incremental`. In these commands, the layers specified are different. Therefore, `add_obj_on_bump` will not delete previously added shapes on `M1`. Also, the result of the command `add_obj_on_bump -shape M2 -incremental` is the same as `add_obj_on_bump -shape M2`.
	`>add_obj_on_bump -shape M1`
	`>add_obj_on_bump -shape M2 -incremental`
	⚠ The `-incremental` option can be used only when `-shape` is specified.

`-shape` *layer+*	Specifies the layer(s) on which new shapes are to be added. Polygons with non-45-degree edges are supported.
`-placement_blockage`	Adds a placement blockage on bumps. Only orthogonal polygons are supported.
`-physical_pin` *layer+*	Specifies the layer(s) on which to add physical pins. For a single-pin bump, the name of the physical pin is same as that of the target bump. For a multiple-pin bump, the name of the physical pin is same as that of the bump pin on the same layer. If no bump pin exists on the layer specified for adding the physical pin, the tool chooses the name of any of the pins on the target bump as the pin name of the physical pin. Polygons with non-45-degree edges are supported.
`-polygon {{x1 y1} {x2 y2}...`	
	Specifies the point list of the polygon to add shapes. The center of the specified polygon bounding box aligns with the center of the specified bump cell.

`-rect {x1 y1 x2 y2}`	Specifies the rectangle to add shapes. The center of specified rectangle aligns with the center of the specified bump cell.
`-relative_pin_layer` *pin_layer_name*	
	Specifies the pin layer of the pin shape of the specified bump. Adds shapes based on the specified pin shape of the bump. If the `-relative_pin_layer` option is specified while adding objects, the shape of the added object is same as the shape of the relative pin.
`-routing_blockage` *layer+*	
	Specifies the layer(s) on which to add routing blockages on bumps. Only orthogonal polygons are supported.
`-selected`	Adds shapes on the selected bump.
`-shape_keep_net`	Keeps the net when adding the layer shape. When you specify the `-incremental` option, the tool adds shape with net name if there is no shape with this net on the bump.
`-write_script` *filename*	
	Specifies the file name to dump a Tcl script for adding objects on bumps. If this option is specified, the tool will not add objects on bumps directly. The `-write_script` option cannot be used with `-check_only`.

Example

- The following command adds a shape on layer M1 at the position of the TSV bump whose net is a feedthrough net. The shape on layer M1 is the same as the bump pin shape on layer TSV.

```
add_obj_on_bump -shape M1 -feedthru -relative_pin_layer TSV
```

addTSV

```
addTSV
[-help]
[-noRouteBlkg]
{[-frontBump bumpCellName] [-backBump bumpCellName] [-tsvViaName viaName]}
[-frontBump bumpCellName [-loc_type {geometry_center|cell_center} -prefix prefixName]
[-stackViaUnderBump stackViaName]]
[-backBump bumpCellName  [-loc_type {geometry_center|cell_center} -prefix prefixName]]
[-tsvViaName viaName [-noPlaceBlkg  | -spacing s]
    [[[-tsvFeedthru {x y} [-stackViaTopLayer layer]] | [-stripeNet pgnetName
    -stripeLayer layerNum]] | [-stackViaName stackViaName [-stackViaMarkerLayer
markerLayerName][-stack_via_array {x y} -stack_via_pitch {x y}]]]]
{-lowerLeftLoc {x y}}
 [-relative_bump_cell bumpCell | -relative_selected_bump ]} {[-pitchxy {x y}] [-
relative_bump_cell bumpCell | -relative_selected_bump ]}
{[-upperRightLoc {x y} | -numxy {x y}] [-relative_bump_cell bumpCell | -
relative_selected_bump ]}}
[{-relative_bump_cell bumpCell | -relative_selected_bump } [-relative_min_pitch
<float>] [-relative_offset {x y}] [-relative_bump_type {signal power ground}]]
[-perim n]
[-stagger]
```

Adds TSVs, front-side bumps, and backside bumps under specified conditions. Use this command to create TSVs and bumps after the initial floorplan is loaded.

Parameters

`-tsvViaName viaName`

	Specifies the name for TSV via defined in the LEF file. If this option is not specified, no TSV will be created. The following arguments are used only with `tsvViaName`: • `noPlaceBlkg` • `spacing` • `tsvFeedthru` • `stackViaTopLayer` • `stripeNet` • `stripeLayer`
`-frontBump` *`bumpCellName`*	
	Specifies the name for the front-side bump cell. If this option is not specified, no front-side bump will be created.
`-backBump` *`bumpCellName`*	
	Specifies the name for the backside bump cell. If this option is not specified, no backside bump will be created.
`-loc_type {geometry_center|cell_center}`	
	Specifies the location type for bump creation. • `geometry_center`: Specifies to use bump pin geometry to create bump • `cell_center`: Specifies to use bump bounding box to create bump This option is used with the `-frontBump` and `-backBump` options.
`-lowerLeftLoc x y`	
	Specifies (in microns) the lower left coordinate of the box in which TSV/Bumps will be created.

`-numxy` x y	Specifies (in microns) the count of the TSV/bump in the horizontal and vertical direction.

`-perim` *n*	Creates TSV, front-side bumps, and backside bumps along the rings on the perimeter.
	If `-perim` is specified, *n* defines the number of rings. Then you can create TSV in the following ways:
	• Define the lower left and upper right location of the ring by `-lowerLeftLoc` and `-upperRightLoc`, and then the pitch value of x and y direction by `-pitchxy`. For example, the following command creates TSV in which the lower-left is at `{x1 y1}` and the upper-right is at `{x2 y2}`
	`addTSV -lowerLeftLoc {x1 y1} -upperRightLoc {x2 y2} -pitchxy {px py} -stagger -perim 1`
	`*********`
	`*  *`
	`*  *`
	`*  *`
	`*  *`
	`*********`
	• Define the lower left location of the ring by `-lowerLeftLoc`, and then specify the number and pitch value of tsv in x and y directions by `-numxy` and `-pitchxy`, respectively. For example, the following command creates TSV in which the lower left is at {x1 y1}
	`-lowerLeftLoc {x1 y1} -numxy {12 8} -pitchxy {px py} -perim 2`
	`************`
	`************`
	`**  **`

`-prefix` *prefixName*	
	Adds a prefix name for a bump during bump creation and bump mapping. The prefix name is not added to the TSV.
`-relative_bump_cell` *bumpCell*	Creates TSV or bumps by default at the geometry center of bumps with specified bump cell.
`-relative_bump_type {signal power ground}`	Creates the relative bump type, TSV, or bumps by default at the geometry center of bumps with the specified bump type.
`-relative_min_pitch` *minPitch*	Specifies the minimum pitch between the created bumps/TSVs. The pitch between all the created bumps/TSV should be larger than specified min pitch. The `-relative_min_pitch` option is optional and can be used only if `-relative_bump_cell/-relative_selected_bump` is specified.
`-relative_offset {x y}`	Specifies the offset of created bump/TSV location compared with the relative bump. The geometry center of the created bump/TSV is the geometry center of the relative bumps plus the offset. `-relative_offset` is optional and can be used only if `-relative_bump_cell/-relative_selected_bump` is specified.
`-relative_selected_bump`	Creates TSV or bumps by default at the geometry center of the selected bumps.
`-upperRightLoc x y`	
	Specifies (in microns) the upper right coordinate of the box in which TSV/Bumps will be created.
`-pitchxy` *x y*	Specifies (in microns) the distance between TSV centers in the x and y direction.

`-stagger`	Staggers (offsets) the TSV/bump array. For example:
	without `-stagger` with `-stagger`
	`*****` `*****`
	`*****` `****`
	`*****` `*****`
	`*****` `****`
	`*****` `*****`

`-noPlaceBlkg`	The placement blockage overlapped with TSV cut will be added as default if `-noPlaceBlkg` is not specified.
`-noRouteBlkg`	The routing blockage for each crossing layer will be added if `-noRouteBlkg` is not specified.
`-tsvFeedthru` *`sizex sizey`*	
	Creates the feedthru. `sizeX` and `sizeY` are the dimensions, in microns, for the stack vias aligned with the TSV. **Note:** Feedtrhu is a collection of components including TSV, the stacked via (from the first metal to the top metal) aligned with the TSV, and two bumps on the frontside and backside.
`-stackViaMarkerLayer` *`markerLayerName`*	Specifies the mark layer name. Mark cut layer of stack via as specified mark layer.
`-stackViaName` *`stackViaName`*	Specifies the stack via cell name. The tool automatically adds the specified via cell stacking to the top layer of long TSV.
`-stackViaTopLayer` *`layer`*	

	Defines the top layer that the TSV structure should connect to. For example, if you specify M5 for this parameter, Innovus will create a structure like `TSV+VIA12+M2+VIA23+M3+…..+M5`. This parameter must be used with `-tsvFeedthru`.
`-stackViaUnderBump` *stackViaName*	
	Specifies the stack via name under the front bump. The tool automatically adds the specified via cell stacking to the bottom layer of front-side bump.
`-stripeNet` *pgnetName*	
	Declares that the TSV will be added beneath the power stripe, a stack via will be generated to touch the power stripe, and the new added TSV will be assigned by the power or ground net of the power stripe. **Note:** This options should be used along with the `-stripeLayer` option.
`-stack_via_array {x y}`	Specifies the number of columns and rows for the stack via array. The `-stack_via_array` and `-stack_via_pitch` options can only be specified when –stackViaName is specified. `-stack_via_array` and `-stack_via_pitch` are specified at the same time. The value of `-stack_via_pitch` should be larger than stack via size. Otherwise, the stack via will overlap each other and cause DRC.
`-stack_via_pitch {x y}`	Specifies the pitch_x and pitch_y of the stack via. The `-stack_via_array` and `-stack_via_pitch` options can only be specified when –stackViaName is specified. `-stack_via_array` and `-stack_via_pitch` are specified at the same time. The value of `-stack_via_pitch` should be larger than stack via size. Otherwise, the stack via will overlap each other and cause DRC.
`-stripeLayer` *layerNum*	

	Specifies the layer number or layer name for the stripe net when adding TSV under stripe net. **Note:** This option should be used along with `-stripeNet` option.
`-spacing s`	Specifies the placement blockage size. Blockage size = TSV cut size + 2*s. If this option is not specified, "s" will be with the value defined in LAYER TSV session in LEF file.

⚠ At least one option of the following arguments is required for the `addTSV` command:

- `tsvViaName`
- `frontBump`
- `backBump`

Examples

- The following command creates a 10 x 10 TSV array with a pitch of 40 microns and the lower left at (300 300):
  ```
  addTSV -lowerLeftLoc 300 300 -numxy 10 10 -pitchxy 40 40 - tsvViaName TSVVIA
  ```
- The following command creates a backside array in box (300 300 700 700) with a pitch of 40 microns:
  ```
  addTSV -lowerLeftLoc 300 300 -upperRightLoc 700 700 -pitchxy 40 40 -backBump
  BUMPCELL
  ```

Related Information

- `assignTSV`
- `deleteTSV`

assignTSV

```
assignTSV
[-help]
[-write_down_level_bump]
[-selected netName | [[-signal | -pgpad ] [-feedthru | -nonFeedthru ] [-net netlist | -exclude_net excludeNetList] [-area llx lly urx ury ...] [-exclude_region llx lly urx ury ...] [-tsvViaName tsvViaCellName] [-interposer ] ] ]
[{[-backBump ] [-frontBump ]} [-force_current_die ]]
```

Assigns nets to TSVs and/or frontside and backside bumps, and/or feedthrus. The command considers all dies in the 3D IC System by using the die abstract files, and optimizes bump assignment for the global nets which connect these dies. If you run the `readDieAbstract` command to import die abstract files of other chips inside the 3D IC System before running the `assignTSV` command, `assignTSV` will optimize TSV/bump assignment by considering the whole system.

Note: Use this command after the TSVs and bumps are created and the instances connected with the IO pins are placed.

Parameters

`-help`	Print out the command usage.
`-backBump`	Declares that the back side bumps and the TSVs will be assigned automatically.
`-force_current_die`	Assigns all IO pins to backside bumps or frontside bump directly without checking the XML file and the top level netlist. You can use `-force_current_die` only if `-backBump` or `-frontBump` is specified.
`-frontBump`	Declares that the front side bumps will be assigned with the IO pins automatically.
`-tsvViaName` *tsvViaCellName*	
	If this option is specified, only the TSV with the VIA cell `tsvViaCellName` will be assigned. Others will be ignored.
`-selected` *netName*	

	Assigns the selected TSVs and bumps with a specified net. **Note:** If the selected TSV belongs to a feedthru, the whole feedthru will be assigned with the net. This option is used for manual assignment and is exclusive to any other options for auto assignment.
`-signal`	Declares that only signal will be assigned.
`-pgpad`	Declares that only power/ground net will be assigned.
`-feedthru`	Declares that the command assigns only the feedthru net.
`-nonFeedthru`	Declares that the command assigns only the nonfeedthru signal or power/ground net.
`-net` *`netlist`*	Assigns the specified nets to the near TSV/bumps.
`-exclude_net` *`excludenetlist`*	
	Excludes the nets in the specified boxes.
`-area {`*`llx lly urx ury ...`*`}`	
	Assigns the TSV/bump within the box only.
`-exclude_region {`*`llx lly urx ury ...`*`}`	
	Excludes the TSV/bumps in the specified boxes.
`-interposer`	Declares that the assignment is done in silicon interposer without any instance.
`-write_down_level_bump`	Writes down the level bump file for each hierarchical instance.

Notes:

- If neither `-signal` nor `-pgpad` is specified, both signal and pgpad will be assigned.
- If neither `-feedthru` nor `-nonFeedthru` is specified, both feedthru and non-feedthru net will be assigned.
- If neither `-frontBump` nor `-backBump` is selected, none of the bumps will be assigned. However, the TSV will still be assigned.

Examples

- The following command assigns the signal IO pins to the frontside bump:
  ```
  assignTSV -frontBump -signal
  ```
- The following command assigns the selected TSV with net VDD:
  ```
  addTSV -selected VDD
  ```

Related Information

- addTSV
- deleteTSV
- readDieAbstract
- unassignTSV

create_stress_rule

```
create_stress_rule
[-help]
{ {-layer layerName -spacing_table spacing_table [-chip_name chipName]} | [-reset ] }
```

Defines a spacing table for spacing rules. Since the spacing rule varies with the location of the keep out zone in the chip, user can define different spacing tables for different spacing rule. These spacing tables are used by the tool to identify different scenarios and select the correct spacing constraint for a scenario.

Parameters

`-help`	Prints a brief description of the `create_stress_rule` command usage. For a detailed description of the command, use the `man` command: `man create_stress_rule`
`-chip_name chipName`	Specifies the name of the chip for which the spacing table is used for checking violation. If this option is not specified, the tool applies the spacing rule for all the chips above, below, and current based on the masterslice layer definition in the LEF file
`-layer layerName`	Specifies the name of the stress layer for which the spacing table is used for checking violation.
`-reset`	Resets all stress rules.
`-spacing_table spacing_table`	Specifies the spacing table definition, {x1 y1} {x2 y2}.

createTSVNoLoadSPEF

```
createTSVNoLoadSPEF
fileName
```

Creates a top-level spef that describes the connectivity between dies. The RC is `0` in this SPEF file, but only inter-die connectivity is written. This spef file, together with the spef for each die, is the input of Tempus for static timing analysis.

This command is used after reading the stacked die config file (XML) using `readTSVConfig` as the die configuration is required to detect the connectivity between dies.

Parameters

fileName	Specifies the file name.

Examples

- The following command creates a `0` load top-level spef file:

  ```
  createTSVNoLoadSPEF top.spef
  ```

Related Information

- `readTSVConfig`

deleteTSV

```
deleteTSV
{-all | -selected| -unassigned}
```

Deletes the specified TSVs. If the TSV belongs to a feedthru, the entire feedthru will also be deleted.

Parameters

`-all`	Deletes all the TSVs.
`-selected`	Deletes the selected TSVs.
`-unassigned`	Deletes the unassigned TSVs.

Examples

- The following command deletes all the TSVs:
  ```
  deleteTSV -all
  ```

Related Information

- `addTSV`

read_codesign_die_abstract

```
read_codesign_die_abstract
[-help]
DIEABSTRACTFILE
[-check_only]
[-no_diff]
```

Reads the die abstract directly to update Innovus. The command provides flexibility in terms of what is read and the elimination of the I/O Update file and TCL code.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `read_codesign_die_abstract` parameter. For a detailed description of the command and all of its parameters, use the man command: `man read_codesign_die_abstract`
`DIEABSTRACTFILE`	Specifies the name of the die abstract file.
`-check_only`	This is only valid with the `-no_diff` option. If these two options are given, then the difference viewer will not be used, and the Innovus db will not be updated.
`-no_diff`	When the `-no_diff` option is used, the difference viewer will not be utilized. The die abstract will be read and the Innovus db will be updated with all the changes unless the `-check_only` option is provided.

Related Information

- `readDieAbstract`
- `write_codesign_die_abstract`

read_design_stack_config

```
read_design_stack_config
[-help]
[-avoid_floorplan_change]
[-chipName NameOfChip]
[-manufacturing_grid grid]
[-power_config_file PowerConnectionFile]
[-topNetlist VerilogFileName]
[-xml StackedDieFileName]
```

Loads the die stacking config file (*.xml) for Innovus to understand the 3D-IC top-level structure.

You can use this command before or after the `init_design` command. When you use `read_design_stack_config` before `init_design`, in addition to loading the die stacking information into the memory, it patches up the virtual techLEF of the 3DIC top cell and the Verilog netlist if the `init_lef_file` and `init_verilog` variables are not set before `init_design`. If you use `read_design_stack_config` after `init_design`, when the top design (Netlist, tech LEF) has already been loaded into Innovus, but without XML. Without the XML, Innovus does not know that the top cell is the top of a 3DIC design. Therefore, XML is required to let Innovus know the vertical position and face up/down information of the dies for the GUI display and the contact layer mapping relationship.

When the top-level netlist is missing, read_design_stack_config can help generate a virtual netlist, but the pins are floating so that you can still load the 3D-IC design into Innovus with iHDB. Only top is in the virtual netlist. `read_design_stack_config` internally calls addInst to add other dies.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `read_design_stack_config` parameter. For a detailed description of the command and all of its parameters, use the man command: `man read_design_stack_config`
`-avoid_floorplan_change`	When you specify this option, the floorplan and die instances will not be created.
`-chipName` *`NameOfChip`*	Specifies the name to be used when the current die is instantiated.
`-manufacturing_grid` *`grid`*	Specifies the manufacturing grid. However, if the manufacturing grid is set by TLEF, `read_design_stack_config` does not update the manufacturing grid and prints an error.
`-power_config_file` *`PowerConnectionFile`*	Specifies the file name of the power connection.
`-topNetlist` *`VerilogFileName`*	Specifies the file name of the top-level netlist verilog.
`-xml` *`StackedDieFileName`*	Specifies the file name of the stacked die.

Related Information

- `init_design`
- `init_lef_file`
- `init_verilog`
- `write_design_stack_config`

readBumpLocation

```
  readBumpLocation
[-help]
fileName
[-allow_overlap_control {keep_all keep_existing_bumps keep_new_bumps}]
[-die_box]
[-ignore_route_blockages]
[-force_current_die]
{[-frontBump bumpCellName] [-backBump backBumpCellName] [-bump_cell_from_bump_file ]}
[{-backBump backBumpCellName | -bump_cell_from_bump_file } [-rotate_bump  -
prefix prefixName]]
[{-frontBump bumpCellName | -bump_cell_from_bump_file }[-rotate_bump  -
prefix prefixName] [-stackViaUnderBump stackViaName]]
[-prefix prefixName | -bump_name_from_bump_file {true | false}]
[-checkAlignment ]
[-nonFeedthru ]
[-feedthru ]
[-tsvViaName viaCellName [-feedthruSize sizex sizey] [-offset {x y}]]
```

The command creates or assigns objects on the current die according to the bumps on the adjacent die. In a TSV design, the micro bumps on the adjacent die should be aligned, and the aligned micro bumps should be with the same signal. Specify this command to sync-up the micro bump between dies. Firstly, the command tries to find the front side bump/back side bump/TSV at the location aligned to the bump on the adjacent dies. If all the specified bumps/TSVs are found on the current die, the command will assign the found bump/TSV with the same signal as the bump on the adjacent die; if none of the specified bumps/TSV are found, the command will create the specified bump/TSV with the same signal at the aligned position.

This command supports the combo bump and embedded bump flows.

Use this command after reading the stacked die config file (XML), by using `readTSVConfig`, as the die configuration is required for the bump alignment.

Note: A combo bump is a group of bumps that are treated as one bump, which means all bumps in the same combo bump will be created, assigned, unassigned, and manipulated at the same time. This command synchronizes the combo bump information between adjacent chips automatically. All bumps belonging to the same combo bump will be treated as one unit. Embedded bump is the bump at the block-level, and the information of these bumps can be exchanged between chips at the top-level.

Parameters

`-allow_overlap_control {keep_all keep_existing_bumps keep_new_bumps}`	Specifies the overlap control type. The allowed values for the `-allow_overlap_control` parameter are: • `keep_all`: Keeps all overlapping bumps. • `keep_existing_bumps`: Keeps the existing bumps before calling `readBumpLocation` if there is an overlap. If this option is specified, `readBumpLocation` keeps creating a bump if there is an overlap and continues creating the remaining bumps. A warning message is displayed when bump creation is skipped to avoid overlap. • `keep_new_bumps`: Removes the existing bumps before calling `readBumpLocation`, places the newly created bump if there is overlap, and then continues creating the remaining bumps. A warning message is displayed when deleting the existing bumps to avoid overlap. If the status of the existing bumps to be deleted is covered or fixed, a warning is displayed and the new bumps overlapping with the covered or fixed old bump are not kept.
`-backBump` *`bumpCellName`*	Creates or assigns the backside bumps at the location aligned with the bumps on the adjacent die.
`-checkAlignment`	Use this parameter to check the alignment between current die and the adjacent dies. It also checks the overlap between the OBS layer and bumps (both front and back bumps). When this parameter is specified, all other parameters are ineffective.
`-bump_cell_from_bump_file`	Creates bumps with the same bump cell defined in the bump file.
`-bump_name_from_bump_file {true | false}`	Creates bumps with the same bump name defined in bump file.

`-die_box`	Draws the die bounding box based on the DieBox value in the bump file.
`-force_current_die`	Specifies to consider the bump file as the current die bump file. Skip to check if the specified bump file is for current die or adjacent die or a block.
`fileName`	Specifies the name of the bump location file of the adjacent die.
`-frontBump` *`bumpCellName`*	Creates or assigns the frontside bumps at the location aligned with the bumps on the adjacent die.
`-feedthru`	Creates or assigns the bump with feedthru net only. If `-feedthruSize` and `-tsvViaName` are specified, the TSV and the stacked via of the feedthru will also be created.
`-feedthruSize` *`sizex sizey`*	Specifies the size for feedthru. This option is exclusive with the `-nonFeedthru` option.
`-ignore_route_blockages`	Allows creation of bumps on route blockages. When this option is specified, the tool ignores checking route blockages when creating bumps.
`-nonFeedthru`	Creates or assigns the bump with normal net only.
`-offset x y`	Specifies the offset from the TSV to the corresponding bump. **Note:** You can also specify the offset in the bump file by appending `-offset x y` to the line. The offset in the bump file has higher priority than the offset specified in the command line. If `-offset` is specified at both command line and bump file, the program will follow the offset specified in the bump file.

`-prefix` *`prefixName`*	Adds prefix name for a bump during bump creation and bump mapping. The default name of bumps are `frontBump_*` or `backBump_*` with serial numbers. If `-prefix` *`prefixName`* is applied, created bump names are `prefixName_frontBump_*` or `prefixName_backBump_*`. **Note**: Single underscore (_) is automatically added between the *`prefixName`* and the default bump name.
`-rotate_bump`	Rotates bumps during bump mapping according to the chip and bump orientation information of the adjacent die.
`-stackViaUnderBump` *`stackViaName`*	Specifies the stack via name under the front bump. The tool automatically adds the specified via cell stacking to the bottom layer of front-side bump.
`-tsvViaName` *`viaCellName`*	
	Creates or assigns the TSV at the location aligned (or with offset if `-offset` is specified) to the bumps on the adjacent die.

Note: If neither `-feedthru` nor `-nonFeedthru` is specified, the bump/TSV with both feedthru net and non-feedthru net will be created and assigned.

Examples

- The following command reads a bump file and creates the corresponding frontside bump on current die:

  ```
  readBumpLocation bumpfile -frontBump front_bump_cell
  ```

- The following command reads a bump file and creates the corresponding backside bump and TSV current die:

  ```
  readBumpLocation bumpfile -backBump back_bump_cell -tsvViaName tsv_via
  ```

Related Information

- writeBumpLocation
- readTSVConfig

readDieAbstract

```
readDieAbstract
DIEABSTRACTFILE
```

Reads the die abstract of the adjacent die. With this command, the abstract of the adjacent die is imported and displayed overlapped with the current design.

This command supports the embedded bump flow for hierarchical designs.

The `readDieAbstract` command is used after reading the stacked die config file (XML) by using `readTSVConfig` because the die configuration is required for die alignment. You can turn off the import adjacent die objects by changing the visibility of abs* layers in the *Custom* tab of the *Color Preferences* form.

Parameters

DIEABSTRACTFILE	Specifies the name of the die abstract file.

Examples

- The following command reads a die abstract file `die1.abs`:

  ```
  readDieAbstract die1.abs
  ```

Related Information

- `readTSVConfig`
- `writeDieAbstract`
- The *Color Preferences* section of the The Main Window chapter in *Innovus Menu Reference*.

readTSVConfig

```
readTSVConfig
{[-stackedDieFile filename] [-powerFile filename]}
[-chipName chipName]
[-topNetlist]
```

Imports 3D IC configuration files of the whole design, including the stack die configuration file, power net configuration file and top-level netlist.

Parameters

-stackedDieFile filename	
	Reads the stack die configuration file.
-powerFile *filename*	
	Imports the power net configuration file.

-chipName *chipName*	Specifies the hierarchical instance name (chip) name of the current design in the top-level netlist.
	For example, if the current design is `ramextra` and the top-level netlist is below it, then you can specify the chip name, `i_ramextra`, as follows:
	`module top (...) ;`
	`asic_entity i_asic_entity (.pin1(), .pin2(), ...);`
	`ramextra i_ramextra (.pin1(), .pin2(), ...);`
	`end module`

-topNetlist	Imports the top-level netlist to support 3D IC design in OA mode. **Note:** You can import the top-level netlist in the non-OA mode in case the top-level netlist is not loaded during the design import.

Examples

- The following command reads the stacked die configuration file and power configuration file and specifies that the current design is named `i_ramextra`:

  ```
  readTSVConfig -stackedDieFile stacked_die.xml -powerFile power.side -
  chipName i_ramextra
  ```

- Below is the example of a stack die configuration file:

```
<StackChip>
  <TopDesign name="top" />
  <Tier number=1>
    <Chip name="i_asic_entity" orientation="R0" faceUp="Yes" llx=0 lly=0
sizeX=6000.0 sizeY=6000.0 scaleFactor=1 />
  </Tier>
  <Tier number=2>
    <Chip name="i_ramextra" orientation="R0" faceUp="Yes" llx=1500.0 lly=1250.0
sizeX=3500.0 sizeY=3000.0 scaleFactor=1 />
  </Tier>
</StackChip>
```

- Below is the example of a power net configuration file:

```
VDD i_asic_entity   sharable
VDD i_ramextra      backside
VSS i_asic_entity   sharable
VSS i_ramextra      backside
Alias:
i_asic_entity/VDD i_ramextra/VDD
i_asic_entity/VSS i_ramextra/VSS
```

Related Information

- Design Methodology for 3D IC with Through Silicon Via chapter in *Innovus User Guide*.
  - *Power Connectivity Input* section
  - *Stack Configuration Input* section

unassignTSV

```
unassignTSV
{-netName netname| -all |-signal | -power | -ground | -selected}
```

Unassigns the specified TSV. This command is used for manual operating on TSV or iteration between TSV assignment and instance placement.

Parameters

`-netName` *netname*	Unassigns the specified net from TSVs.
`-all`	Unassigns all the TSVs.
`-signal`	Unassigns the signal net from TSVs.
`-power`	Unassigns the power net from TSVs.
`-ground`	Unassigns the ground net from TSVs.
`-selected`	Unassigns selected TSVs.

Examples

- The following command reads unassign all the TSVs with power net:
  ```
  unassignTSV -power
  ```

Related Information

- addTSV
- assignTSV
- deleteTSV
- readDieAbstract

verify_stacked_die

```
verify_stacked_die
[-help]
[-check_type type_name]
[-chip_name {chip1 chip2 ...}]
```

Checks die edge violation between adjacent dies. Once you define an OBS layer inside a macro, the adjacent die edge should not overlap this OBS area. If there is any violation between macro OBS and adjacent die edge, it will be marked in the GUI window.

This command must be used after `readDieAbstract` to input adjacent die's die abstract file.

Parameters

`-help`	Prints a brief description of the `verify_stacked_die` command usage. For a detailed description of the command, use the `man` command: `man verify_stacked_die`
`-check_type type_name`	Specifies the check constrain types. Currently, only the `die_edge` and `bump_keepout` zone checking are supported.
`-chip_name {chip1 chip2 ...}`	Specifies the name of the adjacent chip whose edge is to be checked for violation. In a 3D-IC design, there could be more than one top die (in interposer design), so you can specify the chip that needs to be checked. Using this parameter, you can check one or more chips at the same time.

Examples

- The following command checks die edge violation between the macro's keep-out area and adjacent die `mother_die`:

  ```
  verify_stacked_die -chip_name {mother_die}
  ```

write_codesign_die_abstract

```
write_codesign_die_abstract
[-help]
DIEABSTRACTFILE
[-block_pin_layer_bottom layer_name]
[-block_pin_layer_top layer_name]
[-block_pin_minimum_size x y]
[-block_pin_name_includes pin_name]
[-bump_pin_layer_bottom layer_name]
[-bump_pin_layer_top layer_name]
[-constraints file_name]
[-die_shrink_by num]
[-die_thickness num]
[-include_macro_obstructions]
[-io_macros macro_names]
[-io_pin_layer_bottom layer_name]
[-io_pin_layer_top layer_name]
[-io_pin_minimum_size pin_size]
[-io_pin_name_includes pin_name]
[-layer_details]
[-macro_obstructions_layer_bottom layer_name]
[-macro_obstructions_layer_top layer_name]
[-no_block_macros]
[-no_blockages]
[-no_rdl_routing]
[-power_routing]
[-power_routing_layer_bottom layer_name]
[-power_routing_layer_top layer_name]
[-rdl_routing_layer_bottom layer_name]
[-rdl_routing_layer_top layer_name]
[-scribe_all distance_in_microns]
[-scribe_east distance_in_microns]
[-scribe_north distance_in_microns]
[-scribe_south distance_in_microns]
[-scribe_west distance_in_microns]
[-sealring_all distance_in_microns]
[-sealring_east distance_in_microns]
[-sealring_north distance_in_microns]
[-sealring_south distance_in_microns]
[-sealring_west distance_in_microns]
[-version {4.0}]
```

Writes the abstract of current die to a file. It also writes the OBS for die edge checking to the die abstract file. Specify this command after the initial floorplan is loaded. The file will be imported by the adjacent die and therefore the current design will be honored by the adjacent die. The `write_codesign_die_abstract` command is similar to the `writeDieAbstract` except that the `write_codesign_die_abstract` command provides options for filtering portions of the abstract which may not be required for some flows. It also provides the option to write out a third-party die abstract for ICs that do not have TSVs.

This command supports the embedded bump flow for hierarchical designs.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `write_codesign_die_abstract` parameter. For a detailed description of the command and all of its parameters, use the man command: `man write_codesign_die_abstract`
DIEABSTRACTFILE	Specifies the name of the die abstract file.
`-block_pin_layer_bottom` *layer_name*	Specifies the name of the legal routing layer for block macros. If specified, the layers below the mentioned layer should not be the output to the die abstract. If not specified, the default value is the next layer down from the top routing layer.
`-block_pin_layer_top` *layer_name*	Specifies the name of the legal routing layer for block macros. If given, the layers above the mentioned layer should not be the output to the die abstract.
`-block_pin_minimum_size` *x y*	Defines the minimum size of the pin that will be output to the die abstract.
`-block_pin_name_includes` *pin_name*	Specifies that only pins that contain the mentioned value will be in the output.
`-bump_pin_layer_bottom` *layer_name*	Specifies the legal routing layers name for bump macros. If given, the layers below the specified layer are not to be output to the die abstract

`-bump_pin_layer_top` *layer_name*	Specifies the legal routing layer name; used for bump macros. If given, the layers above the specified layer are not to be output to the die abstract.
`-constraints` *file_name*	Specifies the name of the file that lists the constraints. This file will be read and the constraints will be applied to the design.
`-die_shrink_by` *num*	Specifies the percentage by which the die should be shrunk. A typical shrink value is 10 (for a 10% optical shrink). *Default*: `0` The default value, `0`, indicates no shrink.
`-die_thickness` *num*	Specifies the die thickness. *Default*: `0`
`-include_macro_obstructions`	Specifies to give LEF macro OBS shapes in the output to the die abstract. By default, OBS obstructions are not included.
`-io_macros` *macro_names*	Identifies the I/O macros that are to be written to the die abstract. If a macro is not specified, co-design is done at the bump level.
`-io_pin_layer_bottom` *layer_name*	Specifies the legal routing layer name; used for I/O macros. If given, the layers below is not to be output to the die abstract.
`-io_pin_layer_top` *layer_name*	Specifies the legal routing layers' name; used for I/O macros. If given, the layers above is not to be output to the die abstract.
`-io_pin_minimum_size` *pin_size*	This parameter defines the minimum size of pin that will be output to the die abstract. The pin's x and y dimensions must be at least this value.
`-io_pin_name_includes` *pin_name*	This string indicates that only pins that contain the string value will be output.
`-layer_details`	When given, indicates that the layer_details is to be output to the die.

`-macro_obstructions_layer_bottom` *layer_name*	If specified, identifies the bottom layer for which OBS shapes are exported to the die abstract. The default value for this option is the top metal layer.
`-macro_obstructions_layer_top` *layer_name*	If specified, identifies the top layer for which OBS shapes are exported to the die abstract.
`-no_block_macros`	Indicates that block macros should not be the output to the die abstract.
`-no_blockages`	Indicates that blockages should not be the output to the die abstract.
`-no_rdl_routing`	Indicates that the rdl routing should not be the output to the die abstract.
`-power_routing`	Indicates that the power routing should not be the output to the die abstract.
`-power_routing_layer_bottom` *layer_name*	Specifies the names of the legal routing layers for power routing. If given, the layers below are not to be output to the die abstract.
`-power_routing_layer_top` *layer_name*	Specifies the names of legal routing layers for power routing. If given, the layers above are not output to the die abstract.
`-rdl_routing_layer_bottom` *layer_name*	Specifies the names of legal routing layers for rdl routing. If given, the layers below are not to be output to the die abstract.
`-rdl_routing_layer_top` *layer_name*	Specifies the names of legal routing layers for rdl routing. If given, the layers above are not to be output to the die abstract.
`-scribe_all` *distance_in_microns*	Specifies the distance in microns from the die boundary to the edge of the silicon after being cut from the wafer from all sides.
`-scribe_east` *distance_in_microns*	Specifies the distance in microns from the die boundary to the edge of the silicon after being cut from the wafer from the East side.

`-scribe_north` *`distance_in_microns`*	Specifies the distance in microns from the die boundary to the edge of the silicon after being cut from the wafer from the North side.
`-scribe_south` *`distance_in_microns`*	Specifies the distance in microns from the die boundary to the edge of the silicon after being cut from the wafer from the South side.
`-scribe_west` *`distance_in_microns`*	Specifies the distance in microns from the die boundary to the edge of the silicon after being cut from the wafer from the West side.
`-sealring_all` *`distance_in_microns`*	Species the distance of the seal ring from the die boundary on all sides. Seal ring is a border around the die boundary that is put in pre-shrink.
`-sealring_east` *`distance_in_microns`*	Species the distance of the seal ring from the die boundary from the East side.
`-sealring_north` *`distance_in_microns`*	Species the distance of the seal ring from the die boundary from the North side.
`-sealring_south` *`distance_in_microns`*	Species the distance of the seal ring from the die boundary from the South side.
`-sealring_west` *`distance_in_microns`*	Species the distance of the seal ring from the die boundary from the West side.
`-version {4.0}`	Specifies the version number of die abstract.

Related Information

- `readDieAbstract`
- `read_codesign_die_abstract`
- `writeDieAbstract`

write_design_stack_config

```
write_design_stack_config
[-help]
{[-xml StackedDieFileName] [-power_config_file PowerConnectionFile]}
```

Saves the xml file and power config file in the specified path.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `write_design_stack_config` parameter. For a detailed description of the command and all of its parameters, use the man command: `man write_design_stack_config`
`-power_config_file` *`PowerConnectionFile`*	Specifies the file name of the power connection.
`-xml` *`StackedDieFileName`*	Specifies the file name of the stacked die.

Related Information

- `read_design_stack_config`

writeBumpLocation

```
writeBumpLocation
fileName
[-selected]
```

Writes the bump information to a file. It also writes the bump keep-out area information to the bump file. Specify this command after the bumps are created and/or assigned.

This command supports the combo bump and embedded bump flows.

Parameters

fileName	Specifies the file name.
-selected	Specifies to output information related to selected bumps to the bump file. Select the required bumps in the GUI or on the command line, and specify this parameter to output the bump information. You can use this parameter when you want to output bump information that is related to a specific block in hierarchical designs.

Examples

- The following command create a bump information file `die1.bump`:
  ```
  writeBumpLocation die1.bump
  ```

  The below example shows a sample a bump information file:

  ```
  === die1.bump ===

        ChipName: ramextra

        DieBox: 0.000000 0.000000 3500.000000 2999.910000

        OBS: BumpKOZ -poly 906.030 1047.445 906.030 100.000 100.000 100.000 100.000
  1047.445 -spacing 0.000 -backSide

        OBS: BumpKOZ -poly 906.030 1952.465 906.030 2899.910 100.000 2899.910
  100.000 1952.465 -spacing 0.000 -backSide

        OBS: BumpKOZ -poly 2593.970 1047.445 2593.970 100.000 3400.000 100.000
  3400.000 1047.445 -spacing 0.000 -backSide

        OBS: BumpKOZ -poly 2593.970 1952.465 2593.970 2899.910 3400.000 2899.910
  ```

```
3400.000 1952.465 -spacing 0.000 -backSide
      Bump: backBump_499 backUBump_40 3450.000 2850.000 -backside
      Bump: backBump_498 backUBump_40 3450.000 2650.000 -backside
      Bump: backBump_497 backUBump_40 3450.000 2450.000 VSS -ground -backside
      ...
      Bump: backBump_438 backUBump_40 50.000 250.000 VSS -ground -backside
      Bump: backBump_437 backUBump_40 50.000 50.000 VDD -power -backside
      Bump: backBump_436 backUBump_40 3400.000 1800.000 qa_3[31] -backside
      Bump: backBump_435 backUBump_40 3400.000 1700.000 qb_3[31] -backside
      ...
      Note: backUBump_40 is the backBump cell name.
      The design "ramextra" origin is the coordinate original of bump information
file.
```

- The following command creates the bump information file, `die2.bump`, for a multiple-pin feedthrough bump:

  ```
  writeBumpLocation die2.bump
  ```

  The below example shows a sample a bump information file for a multiple-pin feedthrough bump：

  ```
  === die1.bump ===


  ................................
  ................................
  ................................
  ................................
  Bump: Bump_1 uBUMP 200 200 PIN_1 CLK_IN PIN_1 CLK_OUT PIN2
  Bump: Bump_2 uBUMP 200 400 VSS -ground
  ................................
  ................................
  ................................
  ................................
  ................................
  ................................
  ................................
  ................................
  ```

Related Information

- readBumpLocation

writeDieAbstract

```
writeDieAbstract
DIEABSTRACTFILE
[-noFilter]
```

Writes the abstract of current die to a file. It also writes the OBS for die edge checking to the die abstract file. Specify this command after the initial floorplan is loaded. The file will be imported by the adjacent die and therefore the current design will be honored by the adjacent die.

This command supports the embedded bump flow for hierarchical designs.

Note: By default, the proposed behavior for `writeDieAbstract` is:

- For MACRO pin geometries, output ports on top metal layer in all MACROS of all classes.
- For routing, output routing for all top level Verilog ports, and for power and ground, output only those wires with a SHAPE of IOWIRE.

Parameters

DIEABSTRACTFILE	Specifies the file name.
-noFilter	Turn off to filter the die abstract contents.

Examples

- The following command creates a die abstract file `die1.abs`:

  ```
  writeDieAbstract die1.abs
  ```

Related Information

- readDieAbstract
- readTSVConfig

writeMicroBumpMappingFile

```
writeMicroBumpMappingFile
[-help]
fileName
[-mergeWith {filename1 filename2 ...}]
```

Writes the micro bump mapping file for Quantus and Voltus use. Use this command after the aligned bump pair is created by `readBumpLocation` command. This command supports the embedded bump flow for hierarchical designs.

Parameters

`-help`	Prints a brief description of the `writeMicroBumpMappingFile` command usage. For a detailed description of the command, use the `man` command: `man writeMicroBumpMappingFile`
`-mergeWith`	If there are more than two dies in the design, this command may be called more than one time. This option should be specified at the last time to merge the previously generated files.
`fileName`	Specifies the file name.

Example

- The following command creates a bump mapping file:
  ```
  writeMicroBumpMappingFile tier2.map2 -mergeWith tier1.map
  ```

  Here is an example of a bump mapping file, `tier2.map2`:

  ```
  ############################################################
  # Generated by: Cadence Innovus 22.10-a007_1
  # OS: Linux x86_64(Host ID sjfcl024)
  # Generated on: Wed Apr 27 04:03:56 2022
  # Design: die1
  # Command: writeMicroBumpMappingFile tier2.map2 -mergeWith tier1.map
  ############################################################
  # Format:
  # SIGNAL: <Die Name>(<Module Name>) <Net name> <Pin name> <Die Name>(<Module
  ```

```
Name>) <Net name> <Pin name>
# POWER/GROUND: <Die Name>(<Module Name>) <Net Name> <Bump Name> <Die Name>
(<Module Name>) <Net Name> <Bump Name>
SIGNAL {
die1(DIE1) i_a[0] i_a[0] die2(DIE2) i_a[0] i_a[0]
die1(DIE1) i_a[1] i_a[1] die2(DIE2) i_a[1] i_a[1]
die1(DIE1) i_a[2] i_a[2] die2(DIE2) i_a[2] i_a[2]
die1(DIE1) i_a[3] i_a[3] die2(DIE2) i_a[3] i_a[3]
die1(DIE1) i_a[4] i_a[4] die2(DIE2) i_a[4] i_a[4]
die1(DIE1) i_a[5] i_a[5] die2(DIE2) i_a[5] i_a[5]
die1(DIE1) i_a[6] i_a[6] die2(DIE2) i_a[6] i_a[6]
die1(DIE1) i_a[7] i_a[7] die2(DIE2) i_a[7] i_a[7]
die1(DIE1) i_a[8] i_a[8] die2(DIE2) i_a[8] i_a[8]
die1(DIE1) i_a[9] i_a[9] die2(DIE2) i_a[9] i_a[9]
die1(DIE1) i_ceb i_ceb die2(DIE2) i_ceb i_ceb
die1(DIE1) i_d[0] i_d[0] die2(DIE2) i_d[0] i_d[0]
.................................................
.................................................

}
POWER {
die1(DIE1) VDD frontBump_251 die2(DIE2) VDD backBump_296
die1(DIE1) VDD frontBump_252 die2(DIE2) VDD backBump_297
die1(DIE1) VDD frontBump_253 die2(DIE2) VDD backBump_298
die1(DIE1) VDD frontBump_254 die2(DIE2) VDD backBump_299
die1(DIE1) VDD frontBump_255 die2(DIE2) VDD backBump_300
die1(DIE1) VDD frontBump_256 die2(DIE2) VDD backBump_301
.................................................
.................................................
}
GROUND {
die1(DIE1) GND frontBump_226 die2(DIE2) VSS backBump_271
die1(DIE1) GND frontBump_227 die2(DIE2) VSS backBump_272
die1(DIE1) GND frontBump_228 die2(DIE2) VSS backBump_273
die1(DIE1) GND frontBump_229 die2(DIE2) VSS backBump_274
die1(DIE1) GND frontBump_230 die2(DIE2) VSS backBump_275
die1(DIE1) GND frontBump_231 die2(DIE2) VSS backBump_276
die1(DIE1) GND frontBump_232 die2(DIE2) VSS backBump_277
die1(DIE1) GND frontBump_233 die2(DIE2) VSS backBump_278
die1(DIE1) GND frontBump_234 die2(DIE2) VSS backBump_279
.................................................
.................................................
}
```

Related Information

- readBumpLocation

writeVSMappingFile

```
writeVSMappingFile
fileName
```

Writes out a bump mapping file for Voltage Storm use. Use this command after the aligned power bump pairs are created by the `readBumpLocation` command.

Parameters

fileName	Specifies the file name.

Examples

- The following command creates a power bump mapping file:
  ```
  writeVSMappingFile powerBump.mapping
  ```

Related Information

- `readBumpLocation`

46

Unified Safety Format Commands

Midas Unified Safety Format (USF) Physical is loaded at startup. To enable USF in Innovus, you must set `safety_midas_enable` to `physical`.

```
set safety_midas_enable physical
```

You will see a message like this:

```
Cadence Midas USF - Physical
Copyright 2022 Cadence Design Systems, Inc. All rights reserved worldwide.
Cadence and the Cadence logo are registered trademarks of Cadence Design Systems Inc.
in the United States and other countries.
Midas USF - Physical Version: 22.03.001
physical
```

ⓘ Once USF Physical is loaded, it is not possible to disable or reload it in the current session.

- USF File Commands
- USF Support Commands

USF File Commands

- apply_safety_mechanism
- create_esubpart
- create_failure_mode
- create_safety_mechanism
- set_safety_mechanism_rules_dcls
- set_safety_mechanism_rules_tmr

apply_safety_mechanism

```
apply_safety_mechanism
[-help]
safety_mechanism
[-generated]
-to failure_mode
```

Maps a safety mechanism to one or more failure modes.

This command should be applied only via a USF file that is read in using `read_usf`.

Parameters

`-help`	Prints a brief description of the `apply_safety_mechanism` command. For a detailed description of the command, use the `man` command: `man apply_safety_mechanism`.
`safety_mechanism`	Specifies the safety mechanism identifier.
`-generated`	Specifies that the additional logic required to create this safety mechanism has already been inserted into the design.
`-to` *`failure_mode`*	Specifies the failure mode identifier to which the specified safety mechanism should be associated. **Note**: One safety mechanism can be mapped to multiple failure modes.

Example

- The following command applies the `SM_TMR` safety mechanism to the `FM_22` failure mode:

  ```
  apply_safety_mechanism SM_TMR -to FM_22
  ```

Related Information

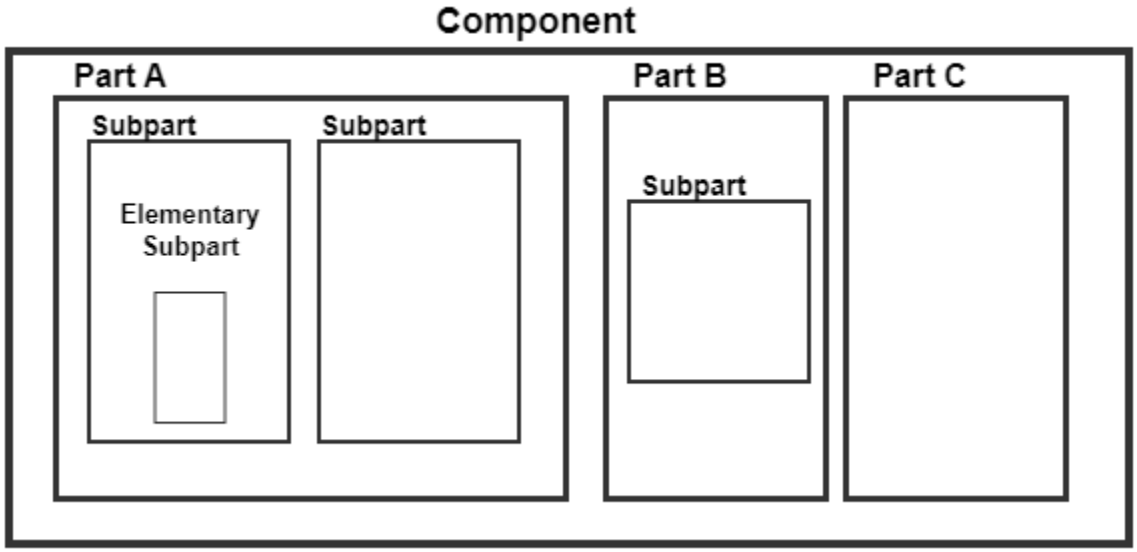
- check_safety_mechanism
- create_esubpart
- create_failure_mode
- create_safety_mechanism
- read_usf
- set_safety_mechanism_rules_dcls
- set_safety_mechanism_rules_tmr

create_esubpart

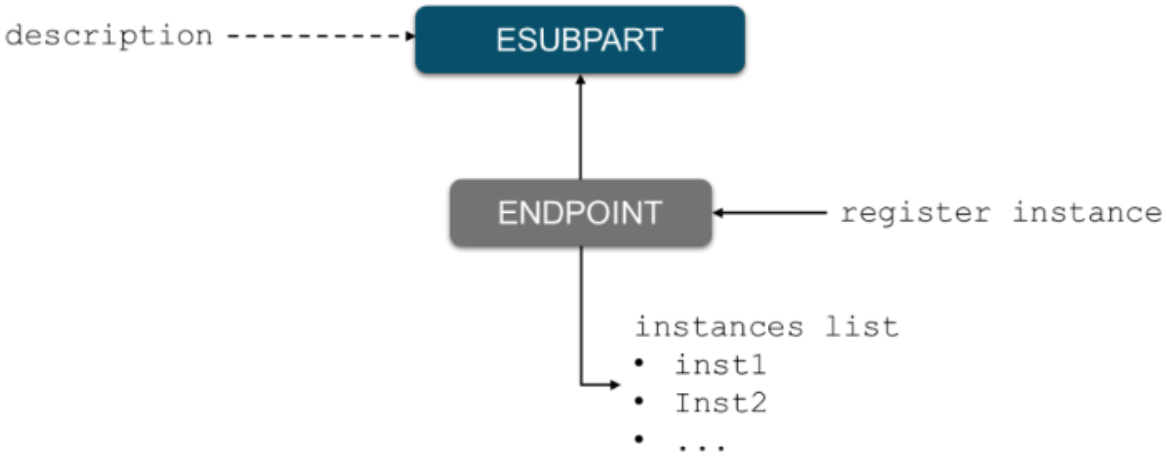
```
create_esubpart
[-help]
esp
[-description description]
-inst string
-register_endpoint string
```

Creates an elementary subpart, which is the smallest hardware subpart considered in safety analysis.

The diagram below represents a component with its parts, subparts, and elementary subparts.



An elementary subpart is a group of instances identified with a descriptor. The instances are associated to reference an endpoint. In the current implementation, the endpoint can only be a register.



The `create_esubpart` command should be applied only via a USF file that is read in using `read_usf`.

Parameters

`-help`	Prints a brief description of the `create_esubpart` command. For a detailed description of the command, use the `man` command: `man create_esubpart`.
esp	Specifies the identifier to be associated with the elementary subpart being created.
`-description` *description*	
	Specifies a textual description for the elementary subpart being created.
`-inst` *string*	Specifies the instances to be mapped to the elementary subpart.
`-register_endpoint` *string*	
	Specifies a reference instance for the endpoint.

Example

- The following command creates an elementary subpart named with the specified description, endpoint, and instances:

```
create_esbupart ESP_VOT_0 -description "automatically generated elementary subpart"
 -register_endpoint { xsv_fpga_top/or1200_top/or1200_cpu/or1200_fpu/s_exp_diff_reg\[5\] }
 -inst { xsv_fpga_top/or1200_top/or1200_cpu/or1200_fpu/s_exp_diff_reg\[5\]_tv_an1
         xsv_fpga_top/or1200_top/or1200_cpu/or1200_fpu/s_exp_diff_reg\[5\]_tv_an2
         xsv_fpga_top/or1200_top/or1200_cpu/or1200_fpu/s_exp_diff_reg\[5\]_tv_or1
         xsv_fpga_top/or1200_top/or1200_cpu/or1200_fpu/s_exp_diff_reg\[5\]_tv_or2
 }
```

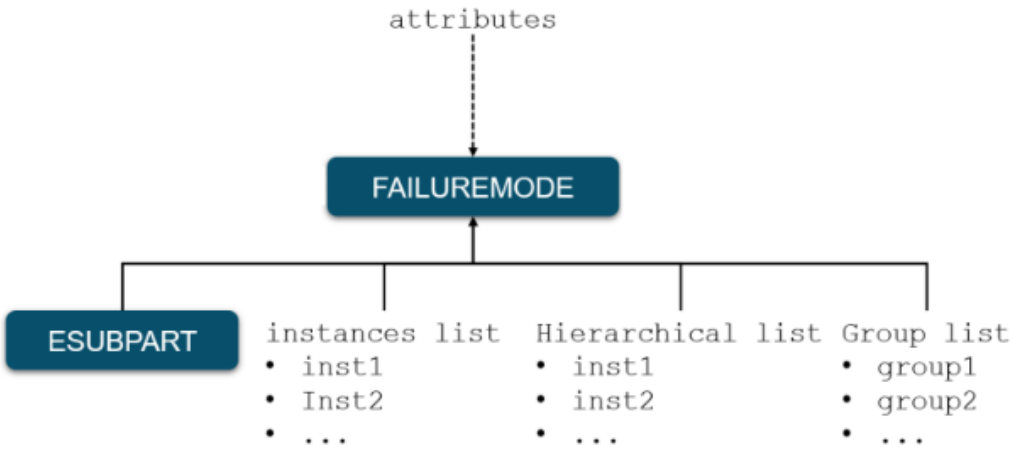
Related Information

- apply_safety_mechanism
- check_safety_mechanism
- create_failure_mode
- create_safety_mechanism
- read_usf
- set_safety_mechanism_rules_dcls
- set_safety_mechanism_rules_tmr

create_failure_mode

```
create_failure_mode
[-help]
FailureMode
[-attributes string]
[-description description]
{-hinst string | -inst string | -esubpart string | -group string}
```

Defines a failure mode. The failure mode groups a given set of logic that is relevant from the safety point of view. For example, it could be the logic that needs to be protected by one or more safety mechanisms or the logic that is describing a safety mechanism that has already been implemented.



The `create_failure_mode` command should be applied only via a USF file that is read in using `read_usf`.

Parameters

`-help`	Prints a brief description of the `create_failure_mode` command. For a detailed description of the command, use the `man` command: `man create_failure_mode`.
`FailureMode`	Specifies the identifier to be associated to the failure mode being created.
`-attributes` *`string`*	Specifies the optional attributes attached to the failure mode.
`-description` *`description`*	Specifies a textual description for the failure mode being created.
`-esubpart` *`string`*	Specifies a list of elementary subparts.
`-group` *`string`*	Specifies the groups to be mapped to the failure mode.
`-hinst` *`string`*	Specifies the hierarchical instance(s) to be mapped to the failure mode.
`-inst` *`string`*	Specifies the instances to be mapped to the failure mode.

Example

- The following command creates a failure mode named `FM_22` with the specified description and hierarchical instance:

```
create_failure_mode FM_22 -description "FPU Adder produces incorrect data due to
fault into the FPU prenorm addsubtracter"
     -hinst
xsv_fpga_top/or1200_top/or1200_cpu/or1200_fpu/fpu_arith/fpu_prenorm_addsub
```

Related Information

- apply_safety_mechanism
- check_safety_mechanism
- create_esubpart
- create_safety_mechanism
- read_usf
- set_safety_mechanism_rules_dcls
- set_safety_mechanism_rules_tmr

create_safety_mechanism

```
create_safety_mechanism
[-help]
safety_mechanism
[-class {aou hw sw aou_hw aou_sw}]
[-description description]
[-type {tmr dcls}]
```

Defines a new safety mechanism. A safety mechanism is extra hardware inserted into your design to bypass, detect, or correct faults.

This command should be applied only via a USF file that is read in using `read_usf`.

Parameters

`-help`	Prints a brief description of the `create_safety_mechanism` command. For a detailed description of the command, use the `man` command: `man create_safety_mechanism`.
`safety_mechanism`	Specifies a name for the safety mechanism being defined.
`-class {aou hw sw aou_hw aou_sw}`	Specifies the supported safety mechanism classes. The possible values are: • `aou`: assumption of use • `hw`: hardware • `sw`: software • `aou_hw`: assumption of use hardware • `aou_sw`: assumption of use software *Default*: `hw`
`-description` *`description`*	Specifies a description for the safety mechanism being defined.
`-type {tmr dcls}`	Specifies the type of the safety mechanism being defined. The options are: • `tmr` - triple modular redundancy • `dcls` - dual-core lockstep *Default*: `tmr`

Example

- The following command creates a safety mechanism named, `SM_TMR`, of the specified class and type:
  ```
  create_safety_mechanism SM_TMR -description "Triple Modular Redundant Flip Flops" -type tmr -class hw
  ```

Related Information

- apply_safety_mechanism
- create_esubpart
- create_failure_mode
- read_usf
- set_safety_mechanism_rules_dcls
- set_safety_mechanism_rules_tmr

set_safety_mechanism_rules_dcls

```
set_safety_mechanism_rules_dcls
[-help]
[-exclusive_groups {{group_A1 {group_A2 group_A3 ...}} {group_B1 group_B2
group_B3 ...} ... }]
[-isolate_clock {* | group_A1 group_A2 ...}]
[-spacing d]
```

Defines the separation rules to be applied to the dual core lockstep (DCLS) groups in the design.

Each time the `set_safety_mechansim_rules_dcls` command is issued, the tool clears any previously set DCLS-associated attributes across all groups prior to setting them in the way the latest command dictates. Therefore, this command cannot be used multiple times to accumulate a set of rules.

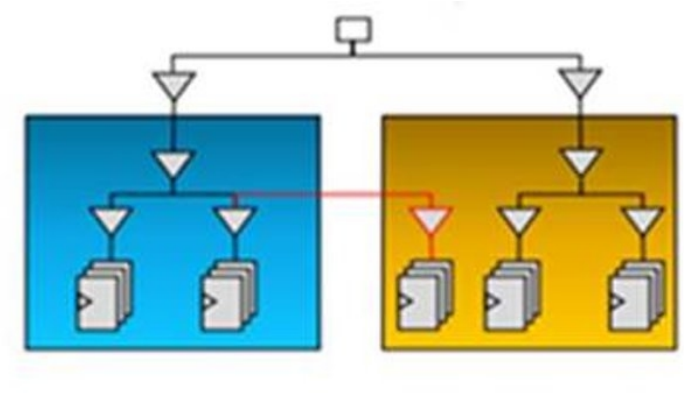
This `set_safety_mechansim_rules_dcls` command may be interactively applied at the command line. That is, it is not restricted to be read in as part of a USF file using `read_usf`. However, if the command is not in a USF file that is being read, it must be applied only after DCLS-related USF information has been read because it depends on this information to function correctly.

Attempting to use this command before a design is loaded or before the groups listed in arguments are defined will cause an error to be issued. In addition, any groups mentioned in this command must already be defined as DCLS groups using a combination of the `create_failure_mode`, `create_safety_mechanism` (with type `dcls`), and `apply_safety_mechanism` commands.

Parameters

`-help`	Prints a brief description of the `set_safety_mechanism_rules_dcls` command. For a detailed description of the command, use the `man` command: `man set_safety_mechanism_rules_dcls`.
`-exclusive_groups {{group_A1 {group_A2 group_A3 ...}} {group_B1 group_B2 group_B3 ...} ... }`	

	Specifies a list of mutually exclusive DCLS group lists. This parameter is mandatory if the `-spacing` parameter is being used. Wildcards are not allowed in the `-exclusive_groups` arguments. If a group is referenced more than once in the `-exclusive_groups` arguments, an error will be issued. In the argument example above, `{group_A1 {group_A2 group_A3 ...}` is a master/checker arrangement, where `group_A1` is the "master" group and `group_A2` and `group_A3` are the "checker" groups. This means that the checker groups must be separated from the master group by a distance of at least `d` as specified with the `-spacing` parameter. However, a checker group may overlap other checker groups. When a master/checker arrangement is specified, there can be only one master and there must be at least two checkers. In the `{group_B1 group_B2 group_B3 ...}` part of the argument, `group_B1`, `group_B2`, and `group_B3` are all mutually exclusive. This means that they must all be separated from one another by a distance of at least `d` specified with the `-spacing` parameter. Each list of mutually exclusive groups must have at least two members or else an error will be issued.
`-isolate_clock {* | group_A1 group_A2 ...}`	

	Specifies a list of groups where clock isolation is required. This argument is optional. Clock isolation is specific to each listed group. If specified for a group, it isolates that group from the rest of the clock network in the design. Clock isolation is not just about separating two groups. For example, in the diagram below, the path shown in red will not occur for the groups with isolated clocking.  If * is specified, clock isolation is enacted for all the DCLS groups in the design.
`-spacing` *d*	Defines in microns the minimum spacing between exclusive DCLS groups. This parameter is mandatory if the `-exclusive_groups` parameter is used. The distance `d` here is expected to be a positive rational number. The specified distance is rounded to the nearest 0.001 value. If the distance specified is a negative number, a warning is issued and the setting is changed to `0`.

Examples

- The following command specifies that groups `A` and `B` are mutually exclusive to one another. The group boundaries must be at least 5 microns apart:
  ```
  set_safety_mechanism_rules_dcls -exclusive_groups { {A B} } -spacing 5
  ```
- The following command specifies that groups `A`, `B`, and `C` are mutually exclusive. These groups must be at least 10 microns apart:
  ```
  set_safety_mechanism_rules_dcls -exclusive_groups { {A B C} } -spacing 10
  ```
- The following command specifies that group `W` must be at least 5 microns from groups `X`, `Y`, and `Z`. However, `X`, `Y`, and `Z` may overlap one another:
  ```
  set_safety_mechanism_rules_dcls -exclusive_groups { {W {X Y Z}} } -spacing 5
  ```
- The following command specifies that groups `A`, `B`, and `C` must be at least 5 microns apart from one another and group `W` must be at least 5 microns from groups `X`, `Y`, and `Z`. However, `X`, `Y`, and `Z` may overlap one another. In addition, groups `A`, `B`,`C`, `W`, `X`, `Y`, and `Z` are to have isolated clocks:
  ```
  set_safety_mechanism_rules_dcls -exclusive_groups { {A B C} {W {X Y Z}} } -spacing 5
  -isolate_clock { * }
  ```

Related Information

- apply_safety_mechanism
- check_safety_mechanism
- create_esubpart
- create_failure_mode
- create_safety_mechanism
- read_usf
- set_safety_mechanism_rules_tmr

set_safety_mechanism_rules_tmr

```
set_safety_mechanism_rules_tmr
[-help]
{[-spacing float] [-spacing_x dx] [-spacing_y dy] [-isolate_clock {none unique_driver}] }
[-spacing float | -spacing_x dx]
[-spacing float | -spacing_y dy]
```

Defines the triple modular redundancy (TMR) safety-mechanism rules that need to be enforced during implementation.

The rules concern the physical spacing between TMR flip-flops and clock isolation of TMR flip-flops. The `set_safety_mechanism_rules_tmr` command may be applied interactively at the command line. That is, it is not restricted to be read in as part of a USF file using `read_usf`.

Each time the `set_safety_mechansim_rules_tmr` command is issued, the tool first clears any previously set TMR rules. This means that the `set_safety_mechansim_rules_tmr` command cannot be used multiple times to accumulate a set of rules.

One of the following parameters must be specified with the command:

- `-spacing`
- `-spacing_x` and/or `-spacing_y`
- `-isolate_clock`

Additionally, the `-spacing` parameter is mutually exclusive to the `-spacing_x` and `-spacing_y` parameters.

Parameters

`-help`	Prints a brief description of the `set_safety_mechanism_rules_tmr` command. For a detailed description of the command, use the `man` command: `man set_safety_mechanism_rules_tmr`.
`-isolate_clock {none unique_driver}`	

	Controls whether or not the three flops in a TMR group may share the same leaf clock buffer. Supported values are: • `none` - Specifies that the three flops in a TMR group may also share the same leaf clock buffer. • `unique_driver` - Specifies that the clock pin of every TMR flop should be directly driven by its own unique clock cell with fanout equal to `1`. *Default*: `none`
`-spacing` *float*	Specifies the minimum radial spacing in µm between a group of TMR flip-flops sharing the same parent. This parameter is mutually exclusive to the `-spacing_x` and `-spacing_y` parameters. The spacing is expected to be a positive rational number. The specified value is rounded to the nearest 0.001. If a negative number is given, a warning is issued and the `spacing` value is set as `0`.
`-spacing_x` *dx*	Specifies the minimum spacing in the X dimension in µm between a group of TMR flip-flops sharing the same parent. This parameter is mutually exclusive to the `-spacing` parameter. The specified value is rounded to the nearest 0.001. If a negative number is given, a warning is issued and the `-spacing_x` value is set as `0`.
`-spacing_y` *dy*	Specifies the minimum spacing in the Y dimension in µm between a group of TMR flip-flops sharing the same parent. This parameter is mutually exclusive to the `-spacing` parameter. The specified value is rounded to the nearest 0.001. If a negative number is given, a warning is issued and the `-spacing_y` value is set as `0`.

Example

- The following command specifies a 5µm radial spacing between TMR flip-flops and requires isolation of clock pins.

```
set_safety_mechansim_rules_tmr -spacing 5 -isolate_clock unique_driver
```

Related Information

- apply_safety_mechanism
- check_safety_mechanism
- create_esubpart
- create_failure_mode
- create_safety_mechanism
- read_usf
- set_safety_mechanism_rules_dcls

USF Support Commands

- check_safety_mechanism
- read_safety_tmr
- read_usf
- reset_usf
- usf_version
- write_usf

check_safety_mechanism

```
check_safety_mechanism
[-help]
[-out_file file_name]
```

Checks all the safety mechanisms and prints a report, optionally writing it to the specified file as well.

The report contains a list of:

- All the safety mechanisms
- The safety mechanism violations found for each safety mechanism, grouped by type

See Report Content for details.

Parameters

`-help`	Prints a brief description of the `check_safety_mechanism` command. For a detailed description of the command, use the `man` command: `man check_safety_mechanism`.
`-out_file` *file_name*	Writes the report to the specified file. If omitted, no report file is generated.

Report Content

The report contains one section per safety mechanism, listing all violations (if any) of that safety mechanism, grouped by type.

Safety mechanisms of type `dcls` (dual-core lockstep) have the following violation types:

- dcls group overlap violations: Two exclusive dcls groups overlap.
- dcls group spacing violations: Two exclusive dcls groups are closer than permitted.
- dcls inst placement violations: An inst belonging to a dcls group is placed outside of that group's boundary.
- dcls internal net routing violations: A net connected only to the insts inside a single dcls group is routed outside that group's boundary.
- dcls interface net routing violations: A net connected to insts both inside and outside a dcls group is routed inside the boundary of an exclusive group.
- dcls clock isolation violations: A clock net connected to a dcls group is driving loads outside that group.

Safety mechanisms of type `tmr` (triple modular redundancy) have the following violation types:

- tmr spacing violations: Two tmr insts from the same safety mechanism are placed closer than permitted.
- tmr clock isolation violations: A clock net driving a tmr inst is also driving other insts.

Example

- The following command checks all the safety mechanisms and writes the report to the `safety_report.rpt` file:

  ```
  check_safety_mechanism -out_file safety_report.rpt
  ```

Related Information

- read_safety_tmr
- read_usf
- reset_usf
- usf_version
- write_usf

read_safety_tmr

```
read_safety_tmr
[-help]
tmr_info_file
-failure_mode failure_mode_name
-safety_mechanism safety_mechanism_name
```

Reads the triple modular redundancy (TMR) information file. For designs with TMR logic already inserted, either in the register-transfer level (RTL) or with third party tools, this command identifies the existing TMR logic and assimilates it into the Unified Safety Format (USF).

You can issue the `read_safety_tmr` command multiple times to build up information in cases where there are multiple different safety mechanism names or failure mode names. However, this may be done only for new safety mechanism and failure mode names. You cannot accumulate TMR information by reusing a safety mechanism or failure mode name that has been already read in with the `read_safety_tmr` command.

Parameters

`-help`	Prints a brief description of the `read_safety_tmr` command. For a detailed description of the command, use the `man` command: `man read_safety_tmr`.

tmr_info_file	Specifies the name of the TMR information file. This is a text file listing each flip-flop and its accompanying cloned flip-flops where TMR has already been applied. Optionally, the file may also list the names of the voter cells that are used.
	
	The TMR information file has the following format:
	
	`<parent_FF1> <clone_FF1A> <clone_FF1B> <voter1A> <voter1B>`
	`...`
	`<parent_FF2> <clone_FF2A> <clone_FF2B> <voter2A> <voter2B>`
	`...`
	`...`
	
	The instance names may be supplied with the full path including the design name at the root. If the design name is not given at the root of the instance path for a given instance, that instance is assumed to be in the current design. The instance names are separated by white space.
	
	Comments are allowed in this file and must be preceded by #.
	
	Blank lines are also allowed.

`-failure_mode` *failure_mode_name*	
	Specifies the name of the failure mode for parent flops. The failure mode for clone flops will be *<failure_mode_name>*`_clones`. The failure mode for voter cells will be *<failure_mode_name>*`_voters`.
`-safety_mechanism` *safety_mechanism_name*	
	Specifies the name of the safety mechanism for the TMR logic.

Related Information

- check_safety_mechanism
- read_usf
- reset_usf
- usf_version
- write_usf

read_usf

```
read_usf
[-help]
USF_file
```

Reads in a Unified Safety Format (USF) file specifying the safety intent of your design.

Parameters

`-help`	Prints a brief description of the `read_usf` command. For a detailed description of the command, use the `man` command: `man read_usf`.
`USF_file`	Specifies the name of the USF file to be read.

Example

- The following command reads in the USF file named `myFile.usf`:

  ```
  read_usf myFile.usf
  ```

Related Information

- `check_safety_mechanism`
- `read_safety_tmr`
- `reset_usf`
- `usf_version`
- `write_usf`

reset_usf

```
reset_usf
[-help]
```

Resets the Unified Safety Format (USF) Physical safety information and related database attributes completely.

Parameters

`-help`	Prints a brief description of the `reset_usf` command. For a detailed description of the command, use the `man` command: `man reset_usf`.

Related Information

- `check_safety_mechanism`
- `read_safety_tmr`
- `read_usf`
- `usf_version`
- `write_usf`

usf_version

```
usf_version
[-help]
```

Retrieves the version of the USF Physical being used to interpret the USF commands.

Parameters

`-help`	Prints a brief description of the `usf_version` command. For a detailed description of the command, use the `man` command: `man usf_version`.

Related Information

- `check_safety_mechanism`
- `read_safety_tmr`
- `read_usf`
- `reset_usf`
- `write_usf`

write_usf

```
write_usf
[-help]
USF_file
```

Writes the safety intent of your design to the specified Unified Safety Format (USF) file.

Parameters

`-help`	Prints a brief description of the `write_usf` command. For a detailed description of the command, use the `man` command: `man write_usf`.
`USF_file`	Specifies the name of the USF file to be saved.

Example

- The following command saves the USF information in the `myFile.usf` file.

  ```
  write_usf myFile.usf
  ```

Related Information

- `check_safety_mechanism`
- `read_safety_tmr`
- `read_usf`
- `reset_usf`
- `usf_version`

47

Unix Commands

- innovus
- lef2oa
- nagelfar
- oa2lef

innovus

```
innovus
[-abort_on_error]
[-batch]
[-cds_lib_file in_file]
[-cpus value]
[-db dir | -ihdb dir]
[-disable_user_startup]
[-execute list_of_Tcl_commands]
[-files in_file_list]
[-help]
[-lic_multi_cpu lic_list]
[-lic_options lic_list]
[-lic_startup lic_list]
[-lic_startup_options lic_list]
[-lic_stack {1 | 2}]
[-log prefix]
[-no_cmd]
[-no_gui]
[-no_logv]
[-overwrite]
[-stylus]
[-version]
[-wait minutes]
```

Launches an Innovus™ Implementation System session. The `innovus` command starts one of the

following products:

- Innovus Implementation System
- Innovus Implementation System Basic
- Virtuoso® Digital Implementation
- Virtuoso® Digital Implementation XL
- First Encounter® L
- First Encounter® XL

For an overview of the products and product licensing, see Product and Licensing Information chapter in the *Innovus User Guide.*

To start an Innovus session, type the `innovus` command with the appropriate parameters on the UNIX/Linux command line. If you type the command without parameters, the software starts in GUI mode and creates a log file and a command file. The system attempts to check out the license with the most functionality, then the license with the next most functionality, and so on.

Parameters

`-help`	Writes out the usage and options with brief descriptions.
`-abort_on_error`	Exits if an error is detected at any point. If any Tcl command returns `TCL_ERROR` to the top level, or any `Error (XXX): ...` message occurs, the process will exit.
`-batch`	Runs without interactive prompt. After all the options, including `-execute` and `-files`, are processed, the job exits.
`-cds_lib_file` *`in_file`*	
	Loads the specified file instead of the default `cds.lib` file in the current directory to obtain the names and locations of OpenAccess libraries. The `cds.lib` file is used to list the OpenAccess library names, and their physical locations.

`-cpus` *value*	Specifies the number of CPUs to be used for multi-threading on current host. This calls the appropriate Tcl command to set the local cpu count. This happens before any `-execute` or `-files` commands are processed. This may require extra multi-CPU licenses to be checked out (see `-lic_multi_cpu`).
`-db` *dir*	Specifies the DB directory to be read. This option calls `restoreDesign` internally. Example: innovus -db *path*/*dbName.enc.dat* The specified DB directory is read before `-execute` or `-files` is invoked. **Note**: The database being restored with `-db` should not be an Integrated Hierarchical Database (iHDB). If you want to restore an iHDB database, use `-ihdb` *dir* instead. The `-db` and `-ihdb` options cannot be used together.
`-disable_user_startup`	Reads only the init file in the installation hierarchy. Does not read init files in the home or current directory. This option suppresses the default loading from the home and current directory, which can be used to guarantee there are no local over-rides to various project settings. See the "*Initialization Files*" section for the init files that are loaded by default.
`-execute` *list_of_Tcl_commands*	
	Specifies the Tcl commands to be executed at startup, as a quoted string.
`-files` *in_file_list*	Specifies one or more Tcl files to be sourced at startup, after `-execute` commands are processed.

`-ihdb` *`dir`*	Specifies that the Integrated Hierarchical Database (iHDB) directory is to be read, such as `xxx/my_design/placed`. This option calls `restore_module_model` internally. Example: `innovus -ihdb` *`iHDB_path/cell/tag`* The specified iHDB directory is read before `-execute` or `-files` is invoked. The specified iHDB should already have been created with `create_module_model`. To learn more about the iHDB flow, refer to the "*Integrated Hierarchical Database*" section in the Hierarchical and Prototyping Flow chapter of the *Innovus User Guide*. **Note**: If you want to restore a non-iHDB database, use the `-db` *`dir`* option instead. The `-db` and `-ihdb` options cannot be used together.
`-lic_multi_cpu` *`lic_list`*	
	Controls which licenses are to be used for higher multi-CPU usage in priority order from left to right. The allowed license names are: `invs_cpu`: Innovus CPU Accelerator Option `invs` - Innovus Implementation System `invsb` - Innovus Implementation System Basic `fel`: First Encounter® L `fexl`: First Encounter® XL For example, `-lic_multi_cpu "fel fexl"` will try to check out `fel` first, and then `fexl`. The licenses will not be checked out until a Tcl command tries to use extra cpus that require more licenses.
`-lic_options` *`lic_list`*	
	Specifies the order for dynamic checkout of extra option licenses when needed by a specific command. In some cases, more than one option license can be used to enable a given feature. You can control which options are allowed to be checked out, and in what priority (from left to right) when required during the run. For example, `-lic_options "encan encng enccco"` will only allow those three options to be

checked-out dynamically, and in that priority order.

The allowed license option names are:

`invs_hier` – Innovus Hierarchical Design Option

`invs_ms` - Innovus Mixed Signal Option

`invs_3d_ic` - Innovus 3D-IC Option

`invs_2nmGAA` - Innovus 2nm GAA Option

`invs_3nmGAA` - Innovus 3nm GAA Option

`invs_3nm` - Innovus 3nm Option

`invs_5nm` - Innovus 5nm Option

`invs_7nm` - Innovus 7nm Option

`invs_10nm` - Innovus 10nm Option

`invs_20nm` - Innovus 20nm Option

`invs_hfr` - Innovus HFR Option

`invs_gigaplace` - Innovus GigaPlace XL Option

`invs_gigaplace_gxl` - Innovus GigaPlace GXL Option

`qrcaa` - Quantus Advanced Analysis GXL Option

`enclp` - Encounter Low Power GXL Option

`encan` - Encounter Advanced Node GXL Option

`enccco` - Encounter Clock Concurrent Opt GXL Option

`encng` - Encounter Next Generation (This license is used to enable beta features.)

`vdixl_capacity` - VDI-XL Block Capacity Option

`invs_vi` - Innovus Voltus InsightAI Option

`invs_ml` - Innovus Machine Learning Implementation Option

`invs_ehfs` - Innovus Early Hierarchical Floorplan Synthesis Option

`invs_dfm` - Innovus DFM Option

`invs_sc` - Innovus Superconducting Option

	`invs_automotive` - Innovus Automotive Option `tpsxl` - Tempus Timing Signoff Solution XL `tpsl` - Tempus Timing Signoff Solution L `vtsxl` - Voltus IC Power Integrity Solution XL `vtsl` - Voltus IC Power Integrity Solution L `vtsxm` - Voltus IC Power Integrity Solution - XM `vtsesd` - Voltus IC Power Integrity Solution - ESD `vtsaa` - Voltus IC Power Integrity Solution Advanced Analysis GXL Option Option licenses can be checked out dynamically when a specific Tcl command requires it. This may cause the Tcl script to stall and wait for a license, or to exit if the wait period expires.
`-lic_startup` *`lic_list`*	
	Requests one of these licenses at startup in priority order from left to right. The supported licenses are: `invs` - Innovus Implementation System `invsb` - Innovus Implementation System Basic `fexl` - First Encounter® XL `vdixl` - Virtuoso® Digital Implementation XL. `vdi` - Virtuoso® Digital Implementation `fel`: First Encounter® L For example, `-lic_startup "fexl fel"` will try to check out `fexl` first, and then `fel`. The checkouts will be attempted repeatedly until one succeeds or the `-wait` period expires.
`-lic_startup_options` *`lic_list`*	
	Specifies additional option licenses needed at startup. If not provided, they will be checked out when needed during the run. The allowed license names are: `invs_hier` – Innovus Hierarchical Design Option

`invs_ms` - Innovus Mixed Signal Option

`invs_3d_ic` - Innovus 3D-IC Option

`invs_2nmGAA` - Innovus 2nm GAA Option

`invs_3nmGAA` - Innovus 3nm GAA Option

`invs_3nm` - Innovus 3nm Option

`invs_5nm` - Innovus 5nm Option

`invs_7nm` - Innovus 7nm Option

`invs_10nm` - Innovus 10nm Option

`invs_20nm` - Innovus 20nm Option

`invs_hfr` - Innovus HFR Option

`invs_gigaplace` - Innovus GigaPlace XL Option

`invs_gigaplace_gxl` - Innovus GigaPlace GXL Option

`qrcaa` - Quantus Advanced Analysis GXL Option

`enclp` - Encounter Low Power GXL Option

`encan` - Encounter Advanced Node GXL Option

`enccco` - Encounter Clock Concurrent Opt GXL Option

`encng` - Encounter Next Generation (This license is used to enable beta features.)

`vdixl_capacity` - VDI-XL Block Capacity Option

`invs_vi` - Innovus Voltus InsightAI Option

`invs_ml` - Innovus Machine Learning Implementation Option

`invs_ehfs` - Innovus Early Hierarchical Floorplan Synthesis Option

`invs_dfm` - Innovus DFM Option

`invs_sc` - Innovus Superconducting Option

`invs_automotive` - Innovus Automotive Option

`tpsxl` - Tempus Timing Signoff Solution XL

`tpsl` - Tempus Timing Signoff Solution L

	`vtsxl` - Voltus IC Power Integrity Solution XL `vtsl` - Voltus IC Power Integrity Solution L `vtsxm` - Voltus IC Power Integrity Solution - XM `vtsesd` - Voltus IC Power Integrity Solution - ESD `vtsaa` - Voltus IC Power Integrity Solution Advanced Analysis GXL Option For example, `-lic_startup_options "enccco encan"` will try to check out both `enccco and encan` when Innovus starts up. The checkouts will be attempted repeatedly until it succeeds or the `-wait` period expires.
`-lic_stack {1 | 2}`	Specifies the number of licenses to stack for stackable licenses. Some startup licenses that have limited instance counts can be stacked so that multiple licenses can be checked-out to increase the instance limit. Innovus allows stacking of two `vdi` or `vdixl` licenses to increase the instance limit from 50,000 to 100,000. For example, `-lic_stack 2` combined with `-lic_startup vdi` would check out two vdi licenses to allow 100,000 instances. *Default*: `1`
`-log` *`prefix`*	Specifies the prefix for the `.log`, `.cmd`, and `.logv` files. If three arguments are given, such as `-log "a b c"`, then the three names are used exactly without any extension added. So, `-log "mylog mycmd mylogv"` will result in `mylog mycmd mylogv` files. The default for *`prefix`* is `innovus`. So if `-log` is not specified, `innovus.log`, `innovus.cmd`, and `innovus.logv` files will be created. If you specify `-log test`, the tool creates `test.log`, `test.cmd` and `test.logv` files. The `.log` file contains the normal logging output, the `.cmd` file contains the Tcl commands that were executed, and the `.logv` file contains a super-set of the .log output plus "verbose" debugging output that is

	needed only for detailed debugging. If the `.log` file already exists, an integer is added to the end of all the files to make them unique. This means if `-log test` is specified, the tool will create `test.log test.cmd test.logv` for the first run, `test.log1 test.cmd1 test.logv1` for the next run, and so on. Only the existence of .log is checked, the existence of the .cmd or .logv files is not checked. The `-overwrite` option can be used to disable this behavior and overwrite an existing .log file. If the *`prefix`* value has a period in it, the last extension is stripped off for the `.cmd` and `.logv` usage. For example, `-log out.log` will result in `out.log out.cmd out.logv` files, and `-log out.a.log` will result in `out.a.log out.a.cmd out.a.logv` files. However, if the last extension in the *`prefix`* value is `.gz`, it will be retained. For example, if you specify `-log test.gz`, the tool will create `test.log.gz`, `test.cmd.gz`, and `test.logv.gz` files. If you specify `-log "test.log.gz test.cmd test.logv.gz"`, the tool will not gzip all files but will create files exactly as specified: `test.log.gz`, `test.cmd`, and `test.logv.gz`. If `test.log` or `test.log.gz` already exists, an integer is added to make the new files unique: `test.log1.gz`, `test.cmd1`, and `test.logv1.gz`. If there is an additional extension before `.gz` , it is stripped off. You can disable any given file by using `/dev/null`. For example, `-log "my.log /dev/null /dev/null"` will only create `my.log`. *Default*: `innovus.log` **Note**: If you provide the log file name under a directory path and the directory path does not exist, Innovus will create the directory path and save the log file under the directory.
`-no_cmd`	Specifies that the `.cmd` file should not be created.
`-no_gui`	Disables GUI and Tk support. GUI will not be available later in the session.
`-no_logv`	Specifies that the verbose log file should not be created.
`-overwrite`	Overwrites an existing .log file (and associated `.cmd` and `.logv` files) instead of adding an integer to make the files unique.

`-stylus`	Launches the Innovus Stylus Common UI interface. This changes the Tcl commands to the Stylus Common UI interface described in the Innovus Stylus Common UI Text Reference Manual and the Stylus Common UI Database Object Information manuals. See the Innovus Stylus Common UI Migration Guide for a summary of the changes.
`-version`	Returns the program version information.
`-wait` *minutes*	Specifies the number of minutes the system waits for a license to become available before exiting. *Default:* 0 (no wait time) *Value range:* 0 to 10,000

Initialization Files

By default, various initialization files are loaded at startup, if they exist. They can be used to configure the GUI, load utility Tcl files, or configure Innovus settings. The initialization files are read in the following sequence:

1. `~/.cadence/innovus/{gui.pref.tcl, gui.color.tcl, workspaces}` in the home directory if the GUI is enabled. These files are intended to be created by *View -> Set Preference* and V*iew -> All Colors -> Save* options for a specific user.
2. `.cadence/innovus/{gui.pref.tcl, gui.color.tcl, workspaces/}` in the current directory if the GUI is enabled. These files are intended to be created by *View -> Set Preference* and V*iew -> All Colors -> Save* options for any user working in this directory.
3. `<install_dir>/etc/innovus/enc.tcl` in the installation directory. This is intended for CAD teams that want to set Tcl parameters, customize the GUI, or add user-defined Tcl commands that affect all users of this release.
4. `~/enc.tcl` in the home directory. This is intended for user-specific setting that should affect only one user.
5. `./enc.tcl` in the current directory. This is intended for design-specific setting that should affect only designs in this directory.

Note: The GUI files are not loaded in the non-GUI mode.

If the `-stylus` parameter is specified, the initialization files are loaded in the following order instead:

1. `~/.cadence/innovus/{gui.pref.tcl, gui.color.tcl, workspaces}` in the home directory if

the GUI is enabled.

2. `.cadence/innovus/{gui.pref.tcl, gui.color.tcl, workspaces/}` in the current directory if the GUI is enabled.
3. `<install_dir>/etc/innovus/innovus.tcl` in the installation directory.
4. `~/.cadence/innovus/innovus.tcl` in the home directory.
5. `./.cadence/innovus/innovus.tcl` in the current directory.
6. `./innovus_startup.tcl` in the current directory.

Note: The `-stylus` usage sets the `source_verbose` root attribute to `true` at startup. The GUI preference init files (1 and 2 above) ignore this setting, but the non-GUI preference initialization files above (3, 4, 5, and 6) will use verbose logging by default. If the shell envar `CDS_STYLUS_SOURCE_VERBOSE` exists and has a boolean value, then the `source_verbose` attribute will be set to the envar value at startup. So you can turn off verbose logging at startup by using the following command:

```
setenv CDS_STYLUS_SOURCE_VERBOSE 0
```

Setting Up the Run-Time Environment

If *install_dir* is the location of your Innovus installation, you should set up your run-time environment like this:

- Add the *install_dir*`/bin` directory to your path. The `bin` directory has links to all the public executables in the install hierarchy.
- If you want the legacy Innovus man pages to be available from the Unix `man` command, you can add *install_dir*`/share/innovus/man` to your `MANPATH` envar.
- If you want the Tcl man pages to be available from the Unix `man` command, you can add *install_dir*`/share/tcltools/man` to your `MANPATH` envar.

For example, you might add this to your startup shell script:

```
set install_dir = /tools/innovus16.2/lnx86

set path = ($install_dir/bin $path)

setenv MANPATH $install_dir/share/innovus/man:$install_dir/share/tcltools/man:$MANPATH
```

Note: When Innovus launches, it automatically adds either the legacy or the Innovus Stylus Common UI man pages (depending on the `-stylus` option) and the Tcl man pages to the beginning

of the current `MANPATH` inside Innovus. Therefore, from within Innovus, the `man` command will see both sets of man pages before any other man pages.

Temporary File Locations

Each Innovus session creates its own temporary directory to store temporary files at the beginning of the run.

If you have the write permission, the tool creates `tmp_dir` under the current directory (.). If not, `tmp_dir` is created in `/tmp`. If the Unix envar `TMPDIR` is set, then the `tmp_dir` directory is created inside `$TMPDIR`.

The name of the `tmp_dir` directory will look like:

```
innovus_temp_[pid]_[hostname]_[user]_xxxxxx
```

Where `_xxxxxx` is a string added to make the directory unique. For example:

```
innovus_temp_10233_farm254_bob_nfp9ez
```

The temporary directory is automatically removed on exit or if the run terminated with a catchable signal (e.g. `SIGSEGV`).

Related Information

- Getting Started chapter of the *Innovus User Guide*
- Product and Licensing Information chapter of the *Innovus User Guide*
- Hierarchical and Prototyping Flow chapter in the *Innovus User Guide*
    - Stylus Hierarchical Database Flow

lef2oa

```
lef2oa
-lef file
-lib library
[-commentChar char]
[-compress]
[-compressLevel zLevel]
[-createFixedViaDefs]
[-dataModel version]
[-defaultRuleName name]
[-DMAttributes attributeList]
[-DMSystem sysName]
[-h | -help]
[-layerMap filelist]
[-layoutView viewName]
[-libDefFile fileList]
[-libPath path]
[-lockColors]
[-logFile file]
[-mapConflicts]
[-noInfo msgIds]
[-noWarning msgIds]
[-overwrite]
[-pinLabels]
[-pnrLibDataOnly]
[-shared]
[-suffix string]
[-techDMAttributes attrList]
[-techDMSystem sysName]
[-techLib library]
[-techLibPath path]
[-techRefs parentTechs]
[-templateFile file]
[-textHeight value]
[-textLayer value]
[-useFoundryInnovus]
[-view viewName]
[-v]
[-version]
```

The LEF to OpenAccess translator can be used to convert LEF files into equivalent OpenAccess

data. It supports three main styles of use:

1. Converting the technology information, such as layers, DRC rules, via rules, vias, and so on, into an OA tech file. This OA techfile is sufficient for Innovus use but is not really usable for Virtuoso because it does not contain information about custom layers below `metal1`, pcells, and so on. This techfile can be read by `techLoadDump` to create an ASCII techfile and then users can cut-and-paste the necessary information, such as DRC rules, into a Virtuoso OA PDK ASCII techfile to create a merged OA techfile.
2. Creating an incremental techfile using the `-pnrLibDataOnly` option that only has technology data that is dependent on the standard cell library, such as default routing rule (`LEFDefaultRouteSpec)`, non-default rules (NDRs), sites, and so on.
3. Converting the LEF macros into OA abstract cellviews.

Innovus uses a Cadence version of the LEF to OpenAccess translator that does more than the public domain `lef2oa` executable. The main difference is that many DRC rules that are expressed as properties that begin with `LEF58_` (for example, `PROPERTY LEF58_SPACING "..." ;`) are understood and mapped directly to the correct OA foundry constraints. In general, the rules needed for 28nm nodes and older are supported, while newer rules for 20nm and smaller are not supported. If you have any unsupported LEF property rules, they can be translated using Innovus commands instead. To do so, read the LEF into Innovus and use the `write_oa_techfile` Innovus command.

Parameters

`-commentChar` *`char`*	Identifies the comment character. *Default*: `#`
`-compress`	Allows libraries to be compressed.
`-compressLevel` *`zLevel`*	Defines the compression level to be used. *Default*: 1
`-createFixedViaDefs`	Allows the creation of fixed via definitions.
`-dataModel` *`version`*	

	Specifies a data model value of 0, 1, 2, or 3, which determines the data model for the OpenAccess database and determines how properties are mapped to the database. If this option is omitted, the default data model value of 4 is used unless the OA_DATAMODEL environment variable is set or a file named OA.DATAMODEL is found in the data directory of your OpenAccess installation hierarchy. *Default*: `4`
`-defaultRuleName` *name*	
	Specifies the name of the constraint group (default: `LEFDefaultRouteSpec`) to generate in the technology database. If a constraint group named `LEFDefaultRouteSpec` already exists in a technology database being modified, the translation fails unless the `-overwrite` option is used. *Default*: `LEFDefaultRouteSpec`
`-DMAttributes` *attributeList*	
	Specifies a list of design management (DM) attributes that are associated with every library that this application creates. Attributes are specified as `name=value` pairs. There must be no space on either side of the equal sign, two or more pairs must be space-separated, and the list must be enclosed in quotes.
`-DMSystem` *sysName*	Specifies the DM system to be used for the library, which can be any DM plug-in available to the user. The DM plug-ins included with OpenAccess are `oaDMTurbo` or `oaDMFileSys`. The default DM system is `oaDMSystem`, which automatically selects between `oaDMTurbo` or `oaDMFileSys`. Use of the `oaDMTurbo` plug-in is not recommended.

`-h	-help`	Displays usage information.
`-layerMap` *`filelist`*	Specifies a quoted, space-separated list of one or more layer mapping files to be used during translation. You must specify the complete path for each layer map file.
`-layoutView` *`viewName`*	Specifies the view name used for FOREIGN container views. *Default:* `layout`
`-lef` *`file`*	Specifies the LEF file to import into the OpenAccess database.
`-lib` *`library`*	Specifies the name of the destination library. If the library exists, the library must contain a technology database or refer to a library with a technology database. If the specified library exists, new designs are created for macro definitions. If you redefine a macro in the file, a new design is created by overwriting the existing design. If the library does not exist, a library is created in the current working directory and will contain a technology database.
`-libDefFile` *`fileList`*	
	Specifies the library definitions (`lib.defs`) file to load. The path to the `lib.defs` file can be a full or relative path. If this option is omitted, the default search mechanism of the DM system is used to find and load the `lib.defs` file.
`-libPath` *`path`*	Specifies the path to the output library.
`-logFile` *`file`*	Specifies the log filename. If this option is omitted, the log filename defaults to `lef2oaCDS.log`.

`-lockColors`	Specifies that all shapes created from `PORT` and `PIN` definitions that have a `MASK` construct associated with them will have their locked state set to `true` in addition to having their color set.
`-mapConflicts`	Maps certain conflicts between incremental and base technologies to constraints and constraint parameter definitions.
`-noInfo` *`msgIds`*	Suppresses the specified INFO messages. *`msgIds`* is a quoted, space separated list of numbers. Each number in the list represents the numerical portion of the ID for the message you want to suppress. None of the numbers in the list may be zero. Suppressed messages do not appear on the terminal or in the log file.
`-noWarning` *`msgIds`*	Suppresses the specified WARNING messages. *`msgIds`* is a quoted, space separated list of numbers. Each number in the list represents the numerical portion of the ID for the message you want to suppress. None of the numbers in the list may be zero. Suppressed messages do not appear on the terminal or in the log file and are not included in the total of WARNING messages displayed in the summary.
`-overwrite`	Replaces MACROS in an existing design with the translated data. If this option is omitted, the default behavior is to update the existing library. This option does not affect the DBU value in an existing technology database.
`-pinLabels`	Creates pin labels in designs by using OpenAccess `oaAttrDisplay` objects on `oaTerms`. Labels are added on the `oaTerm` for each pin figure of the terminal at the location of the pin figure. See also the `-textLayer` and `-textHeight` options.

`-pnrLibDataOnly`	Maps only library-specific place and route information to an incremental tech, ignoring foundry rules. Recommended for converting a LEF file to an incremental technology database that references a technology database that has complete foundry information. LEF constructs `DIRECTION` and `PITCH`, along with `WIDTH`, `OFFSET`, and `WIREEXTENSION` (if specified) are mapped to constraints in the `LEFDefaultRouteSpec` type constraint group. `VIAS` and `VIARULES` defined are added to the `validRoutingVias` constraint in the `LDRS`. `NONDEFAULTRULES` are mapped to OA constraint groups, `SITES` are mapped to `oaSiteDefs`, and `MACROS` are mapped to abstract `oaDesigns`.
`-shared`	Allows other applications to access the output library while this application is running.
`-suffix` *string*	Adds the specified suffix to the constraint group definition names.
`-techDMAttributes` *attrList*	Specifies a list of DM attributes to be associated with every technology library created.
`-techDMSystem` *sysName*	Specifies the DM system to be used for the technology library, which is either `oaDMTurbo` or `oaDMFileSys`. The default is `oaDMTurbo`.
`-techLib` *library*	Specifies the name of a technology library to use when creating a new cell library. If the specified technology library does not exist, the library is created using the technology information in the LEF file.
`-techLibPath` *path*	Specifies the path to be used for creating a technology library specified by the `-techLib` argument. The is only used when the library specified by the `-techLib` argument does not exist.

`-techRefs` *libList*	Specifies the list of libraries that contain the master technology databases.
`-templateFile` *file*	Specifies a file containing arguments to `lef2oa`. You can specify a template file instead of entering a string of arguments on the command line.
`-textHeight` *value*	Specifies an integer value for the pin label text height in database units (DBU). The DBU value is set by `lef2oa` by using the DATABASE UNITS value in LEF. If `-pinLabels` is used, and this option is omitted, the pin label text height defaults to the DBU value in the technology database for maskLayout views.
`-textLayer` *value*	Specifies the layer used for pin label text. If `-pinLabels` is used, and this option is not specified, the pin label text layer defaults to text. If the specified layer does not exist in the technology database, it is added.
`-useFoundryInnovus`	Specifies that the LEF LAYER LEF58 properties constraints will be placed into the `foundry_innovus` constraint group. This option is required only when creating Rapid MSOA tech files for use by the mixed signal flow. For more information on Rapid MSOA PDKs, refer to the Technology Data Preparation chapter in the *Mixed Signal Interoperability Guide*.
`-view` *viewName*	Specifies the output view name. The default view name is `abstract`.
`-v`	Prints tool, format and library version information
`-version`	Prints tool and format version information

Related Information

- `oa2lef`
- `write_oa_techfile`
- Technology Data Preparation chapter in the *Mixed Signal Interoperability Guide*

nagelfar

```
nagelfar
[<nagelfar.tcl options>]
[file1.tcl file2.tcl …]
```

Does the “lint type” checking of Innovus Tcl script files.

It finds many common Tcl errors, such as misspelled commands, using a Tcl variable before setting it.

The check-syntax command is a wrapper around the public-domain Tcl linter named nagelfar. Nagelfar automatically loads the Innovus syntax and plugin files and is therefore aware of Innovus commands and options. Nagelfar is available from http://sourceforge.net/projects/nagelfar. A recent version of nagelfar is included in the installation hierarchy at: `<install_dir>/share/nagelfar`.

Nagelfar is a wrapper around `<install_dir>/share/nagelfar/nagelfar.tcl`

The wrapper automatically loads the correct Innovus nagelfar syntax file and plugin file so that Innovus commands and options are understood by nagelfar. The files are loaded from:

```
<install_dir>/share/fe/nagelfar/innovus.syntaxdb.tcl
<install_dir>/share/fe/nagelfar/innovus.plugin.tcl
```

You can check the nagelfar documentation in the installation directory for details about its options, the checks it performs, and the steps to turn on or off various checks.

To run this command, you should have tclsh in your search path.

You can type `nagelfar -help` to see all the nagelfar.tcl options.

Options	Description
`-encoding <enc>`	Reads script with this encoding.
`-exitcode`	Returns status code 2 for any error or 1 for warning.
`-filter <p>`	Specifies to suppress any message that matches the global pattern.
`-glob <pattern>`	Adds matching files to scriptfiles to check.
`-gui`	Starts with GUI even when files are specified.
`-H`	Specifies a prefix each error line with file name.

`header <file>`	Creates a "header" file with syntax info for scriptfiles.
`-html`	Generates html-output.
`-instrument`	Instruments source file for code coverage.
`-markup`	Markup source file with code coverage result.
`-novar`	Disables variable checking.
`-prefix <pref>`	Specifies the prefix for line anchors (html output)
`-quiet`	Suppresses non-syntax output.
`-s <dbfile>`	Includes a database file. You can include more than one database file. If <dbfile> is "_", it adds the default database.
`-severity <level>`	Sets severity level filter to N/W/E (default N).
`-strictappend`	Enforces having an initialized variable in (l)append.
`-tab <size>`	Specifies the Tab size, default is 8.
`-WelseN`	Enforces else keyword. Default 1.
`-WexprN`	Sets expression warning level to N.
`-WsubN`	Sets subcommand warning level to N.
`1`	Does not warn on single commands. "if [apa] {...}" is ok.
`1 (def)`	Warns about shortened subcommands.
`2 (def)`	Warns about any unbraced expression.

Example

- The following example show the usage of `nagelfar` for file `a.tcl`:

```
innovus> nagelfar a.tcl
Checking file a.tcl
Line 2: W Unknown command "typo_in_cmd_name"
Line 3: E Unknown variable "abc"
```

Related Information

- `check_syntax` is the corresponding Tcl command that can be used directly inside Innovus.

oa2lef

```
oa2lef
-lef file
-lib library
[-cell cells]
[-h | -help]
[-libDefFile filename]
[-lockedColorsOnly]
[-logFile file]
[-noInfo msgIds]
[-noTech]
[-noWarning msgIds]
[-templateFile file]
[-useFoundryInnovus]
[-views viewName]
[-v]
[-ver version]
[-version]
```

This translator can be used to convert OpenAccess data into equivalent LEF data.

Innovus uses a Cadence version of `oa2lef` that does more than the public domain `oa2lef` executable. The main difference is that many DRC rules that are expressed as properties that begin with `LEF58_` (for example, `PROPERTY LEF58_SPACING "..." ;`) are understood and mapped to the correct LEF syntax. In general, the rules needed for 28nm nodes and older are supported, while newer rules for 20nm and smaller are not supported. If you have any unsupported LEF property rules, they can be translated into LEF using Innovus commands instead. To do so, read the OA techfile into Innovus, and use the Innovus `write_lef_library` command.

Parameters

`-cell cells`	Specifies a space-separated list of cells to translate.
`-help or -h`	Displays usage information.
`-lef file`	Specifies the output LEF file name.
`-lib library`	Specifies the name of the source library, containing a technology database.

`-libDefFile` *`filename`*	Specifies the `lib.defs` file to load. The path to the `lib.defs` file can be a full or relative path. If this option is omitted, the default search mechanism of the DM system is used to find and load the `lib.defs` file.
`-lockedColorsOnly`	Prints the colors of only the shapes with a locked state of `true` when mapping the oaFigs or oaVias belonging to oaPins to the LEF PINs and PORTs.
`-logFile` *`file`*	Specifies the log filename. If this option is omitted, the log filename defaults to `oa2lef.log`.
`-noInfo` *`msgIds`*	Suppresses the specified INFO messages. msgIds is a quoted, space-separated list of numbers. Each number in the list represents the numerical portion of the ID for the message you want to suppress. None of the numbers in the list may be zero. Suppressed messages do not appear on the terminal or in the log file.
`-noTech`	Specifies that the technology information is not output.
`-noWarning` *`msgIds`*	Suppresses the specified WARNING messages. *`msgIds`* is a quoted, space-separated list of numbers. Each number in the list represents the numerical portion of the ID for the message you want to suppress. None of the numbers in the list may be zero. Suppressed messages do not appear on the terminal or in the log file and are not included in the total of WARNING messages displayed in the summary.
`-templateFile` *`file`*	Specifies a file containing arguments to `oa2lef`. You can specify a template file instead of entering a string of arguments on the command line.
`-useFoundryInnovus`	Prints the `foundry_innovus` constraint group as LEF LAYER LEF58 properties. These constraints are found only in Rapid MSOA tech files that are used for the Cadence mixed signal implementation solution. For more information on Rapid MSOA PDKs, refer to the Technology Data Preparation chapter in the *Mixed Signal Interoperability Guide*.
`-v`	Prints tool, format, and library version information.
`-ver` *`version`*	Specifies the LEF version to write, which can be 5.3, 5.4, 5.5, 5.6, 5.7 or 5.8. The default version is 5.7.
`-version`	Prints tool and format version information

`-views` *viewName*	Specifies a space-separated list of views to translate. Only the first view name for a particular cell is used; all others are ignored.

Related Information

- `lef2oa`
- `write_lef_library`
- Technology Data Preparation chapter in the *Mixed Signal Interoperability Guide*

48

Verify Commands

- clearDrc
- createMarker
- get_signoff_verify_design_config
- get_verify_drc_mode
- loadDrc
- loadViolationReport
- optimizePattern
- reportFreqViolation
- run_pegasus_drc
- saveDrc
- set_signoff_verify_design_config
- set_verify_drc_mode
- setIsolatedCutRule
- setSnapGrid
- signoff_verify_design
- verify_antenna
- verify_drc
- verifyACLimit
- verifyCMP
- verifyConnectivity
- verifyCutDensity
- verifyEndCap
- verifyIO2BumpConnectivity
- verifyIsolatedCut
- verifyLitho
- verifyMetalDensity
- verifyPowerVia

- verifyTieCell
- verifyWellAntenna
- verifyWellTap
- verifyWireGap
- violationBrowser
- violationBrowserDeleteByArea
- violationBrowserReport

clearDrc

```
clearDrc [-help]
```

Clears all design rule checking (DRC) markers in your design.

Note: Do not use the `clearDrc` command before using `ecoRoute` or any other routing command. This is because NanoRoute works only on the existing DRC/violation markers. The routing commands will not necessarily perform DRC checks in the routing halo regions if the violation markers are already cleared.

Parameters

`-help`	Prints a brief description of the `clearDrc` command usage. For a detailed description of the command, use the `man` command: `man clearDrc`

Related Information

- `verify_drc`
- The Identifying and Viewing Violations chapter of *User Guide*.

createMarker

```
createMarker
[-help]
[-bbox { x1 y1 x2 y2 } | -poly { x1 y1 x2 y2 ...}]
[-desc description]
[-layer {layerName | layerNumber}]
[-rulemap]
[-subtype subtypeName]
[-tool toolName]
[-type typeName]
```

Creates markers for violations in the database. Use this command to create a specific marker that has been created by a third party tool. The created markers can be saved and restored by `saveDesign`/`restoreDesign`.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `createMarker` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man createMarker`
`-bbox` *`x1 y1 x2 y2`*	Specifies the area of the bounding box of the marker. For example, `createMarker -bbox 100 100 100.5 100.5`. *Type:* Real
`-desc` *`description`*	Describes the marker. For example, `createMarker -desc "the user mark is non default rule spacing check"`. *Default:* `""` *Type:* String
`-layer {layerName | layerNumber}`	Specifies the layer. Following are few examples: • `createMarker -layer METAL2` creates a marker on layer METAL2. • `createMarker -layer VIA3` creates a marker on cut layer VIA3. • `createMarker -layer 4` creates a marker on layer METAL4. *Type:* String
`-poly {`*`x1 y1 x2 y2...`*`}`	Specifies the area of the bounding polygon of the marker.
`-rulemap`	Specifies if the marker needs to be created through `ruleMap`.
`-subtype` *`subtypeName`*	Specifies the marker subtype. For example, `createMarker -subtype user_spacing`. *Default:* `Other` *Type:* String
`-tool` *`toolName`*	Specifies the source software application. For example, `createMarker -tool Calibre`. *Default:* `Other` *Type:* String
`-type` *`typeName`*	Specifies the marker type. For example, `createMarker -type user_verify`. *Default:* `Other` *Type:* String

Example

- The following command generates a marker for tool named `Calibre` with type `METAL1` and subtype `S.1`:

  ```
  createMarker -bbox { 1 2 1 2 } -tool Calibre -type METAL1 -subtype S.1
  ```

  The violation browser displays the marker in the above example as follows:

  ```
  + Calibre
      + METAL1
          + S.1
              ...
  ```

- Suppose the cadence LPA tool has created the following marker on layer `METAL3`:

  ```
  CDNLitho;SPACING; Severity: 3; MARKER_INFO: "CD = 45.39"
  bbox = (468.248, 63.511) (468.310, 63.599)
  ```

  The following command creates a corresponding marker in the Innovus design:

  ```
  createMarker -bbox {468.248 63.511 468.310 63.599} -layer metal3 \
               -type CDNLitho -subtype Spacing
  ```

- The following command generates a polygon shape marker with the user description:

  ```
  createMarker -poly {100.0 100.0 194.0 100.0 194.0 238.0 168.0 238.0 168.0 288.0 100.0 288.0} \
  -tool User -desc "congested area"
  ```

Related Information

The Identifying and Viewing Violations chapter of *User Guide*.

get_signoff_verify_design_config

```
  get_signoff_verify_design_config
[-help]
[-abort_on_layout_error]
[-abort_on_missing_rulecheck]
[-area]
[-attach_instance_name]
[-attach_net_name]
[-attach_net_prop]
[-auto_merge_base_class]
[-config_file]
[-control_file
[-core_license]
[-die_area_as_boundary]
[-dp]
[-dp_timeout]
[-error_limit]
[-extra_options]
[-format]
[-ignore_blockage]
[-ignore_fill]
[-keep_data]
[-layer_map_file]
[-layer_range]
[-layers]
[-layout_path]
[-lib_name]
[-license_dp_continue]
[-license_stacking]
[-license_timeout]
[-merge]
[-merge_path]
[-mode]
[-mp]
[-mt]
[-net_map_file]
[-nets]
[-no_structure_name]
[-offset]
[-output_macros]
[-pegasus_bin]
[-process_node]
[-pvs_fill]
[-report_file]
[-rule_file_drc]
[-rule_file_smart_verify_lvs]
[-rule_selection_options]
[-run_name]
[-selected_inst]
```

```
[-skip_auto_load_results]
[-stripes]
[-structure_name]
[-transform_to_inst_master]
[-ui_data]
[-uniquify_cell_names]
[-units]
[-working_dir]
```

Prints parameters for the `signoff_verify_design` command. It also prints `rule_selection_options` for a given process node that can be combined with `all {} | beol {} | feol {}`.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `get_signoff_verify_design_config` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man get_signoff_verify_design_config`
`parameter_names`	Displays information for the specified parameters. You can specify one or more parameters. See `set_signoff_verify_design_config` for descriptions of the parameters you can specify.

Examples

- The following command displays the current setting of the `-stripes` parameter:
  `get_signoff_verify_design_config -stripes`
- The following command displays the current settings for all the `signoff_verify_design` parameters:
  `get_signoff_verify_design_config`

Related Information

- `set_signoff_verify_design_config`
- `signoff_verify_design`

get_verify_drc_mode

```
  get_verify_drc_mode
[-help]
[-area]
[-check_illegal_trim_shapes]
[-check_ndr_spacing]
[-check_only]
[-check_reverse]
[-check_routing_halo]
[-check_routing_halo_corner]
[-check_same_via_cell]
[-check_short_only]
[-check_trim_length]
[-check_uncolored]
[-disable_rules]
[-enable_post_passive_fill_check]
[-exclude_pg_net]
[-ignore_cell_blockage]
[-ignore_fill_wire]
[-ignore_non_rectangle_shapes]
[-ignore_trial_route]
[-layer_range]
[-limit]
[-max_wrong_way_halo]
[-nonDefault]
[-report]
[-use_min_spacing_on_block_obs]
[-quiet]
```

Displays the following information about a `set_verify_drc_mode` parameter in the Innovus console.

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Default value
- Whether the current value was set by user

Use this command when you want to retrieve current values of all or specified `set_verify_drc_mode` settings. If you do not specify any parameter, `get_verify_drc_mode` returns information on all the `set_verify_drc_mode` parameters.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `get_verify_drc_mode` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man get_verify_drc_mode`
`parameter_names`	Displays information for the specified parameters. You can specify one or more parameters. See `set_verify_drc_mode` for descriptions of the parameters you can specify.
`-quiet`	Displays the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software displays the current settings of all `set_verify_drc_mode` parameters in Tcl list format.

Examples

- The following command displays the current setting of the `-limit` parameter:
  ```
  get_verify_drc_mode -limit
  ```
  The software displays the following information:
  ```
  -limit 1000 # int, default=1000
  1000
  ```
- The following command displays the current settings for all `set_verify_drc_mode` parameters:
  ```
  get_verify_drc_mode
  ```

Related Information

- The Identifying and Viewing Violations chapter of *User Guide*.
- `verify_drc`
- `set_verify_drc_mode`

loadDrc

```
loadDrc
[-help]
fileName
[-incremental]
[-orient {r0 | r90 | r180 | r270 | mx | mx90 | my | my90}]
[-origin x y]
```

Loads the specified Design Rule Checking (DRC) violation marker file. Use this command to load DRC markers, which are saved in DB format, into Innovus design. The `loadDrc` command is useful in situations when you want to load DRC violation markers without having to load an entire design.

For example, if during an Engineering Change Order (ECO), you find that only 25 of the 300 reported violations are actual violations that need to be corrected immediately, you can:

1. Delete the other 275 violations.
2. Save the 25 actual violations in a marker file using the `saveDrc` command.
3. Clear the violations from the design window.
4. Load the marker file containing the saved violation markers using the `loadDrc` command.

You can also use the `loadDrc` command in situations where, when correcting violations, the ECO editor didn't make changes in the way that you want. Because the editor removed the marker when it made the change, you cannot find the original violation in the design. You can use the `loadDrc` command to load the original marker file, then click on the violation, and zoom to the area in which you are working on.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `loadDrc` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man loadDrc`
`fileName`	Specifies the name of the marker file to be loaded.
`-incremental`	Loads the markers stored in the specified file incrementally. This means that the markers that are already loaded in memory will not be removed before new ones are loaded from the specified file.
`-orient {r0 | r90 | r180 | r270 | mx | mx90 | my | my90}`	
	Rotates all the DRCs available in the violation marker file with the specified orientation.
`-origin` *`x y`*	Specifies the origin coordinate of the DRC available in the violation marker file.

Example

The following command loads markers that were previously saved to the `VG.drc` file using `saveDrc`:

```
loadDrc VG.drc
```

Related Information

- The following sections in the Identifying and Viewing Violations chapter of the *User Guide*:
  - Viewing Violations With the Violation Browser
  - Clearing Violations

loadViolationReport

```
loadViolationReport
[-help]
-filename fileName
[-orient {r0 r90 ...}]
[-rulemap mapName ]
-type {Assura | Calibre | PVS | Hercules | ICV | CDNLitho | CDNCMP | inShapeLitho | CalibreLitho | DRV |
ClockTran | CLP | VerifyPower}
[-xoffset value ]
[-yoffset value]
```

Loads a violation marker file or a hotspot interface format (HIF) file in one of the following formats and creates markers that the Innovus software can interpret. Use this command to load third-party markers into Innovus:

- Assura™
- Calibre
- PVS
- Hercules
- ICV
- Chemical and Mechanical Polishing (CMP) HIF
- Transition violation report
- Low power violation reports
- Cadence litho hotspot
- NanoRoute litho
- inShapeLitho
- Calibre litho hotspot

After you load the file, you can view the markers in the Violation Browser. By cross-probing between Violation Browser and the design area, you can analyze the violations and take steps to resolve them. The Innovus Auto Query feature displays the rule names for the markers.

Note: The `route_fix_signoff_drc` command can be used to call NanoRoute, to attempt and fix the violations based on the markers from the external DRC tool.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `loadViolaltionReport` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man loadViolaltionReport`
`-filename` *`fileName`*	Specifies the marker file to load. Assura files must have a `.err` extension. Calibre files must have a `.db` extension.
`-orient {`*`r0 r90 ...`*`}`	Supports advanced orientation transforms for loading rotated DRC violation markers before loading. This helps in overlaying the design to the top of the layout.

`-rulemap` *`mapName`*	Specifies the rule map file for a PVS or Calibre Litho report. The rule map file provides the layer/error type information.
	
	`loadViolationReport` saves this information to the database. When the markers are created, the layer, error type, and object information can be identified on the GUI in the Violation Browser. NR could fix the converted drc markers, the supported drc types: `spacing`, `mincut`, `colorLoop`, `area`, `minhole`, `maxstackedvia`, `enclosure`, `width`, and `minstep`. For the unsupported types, it can be mapped with "other". The rule map file should be a txt file with 3 columns:
	
	`Rule_name Innovus_error_type layer_name`
	
	Below is an example:
	`M1.W_4.5 width metal1`
	`M1.A_0.048 minhole metal1`
	`M1.M_1 minstep metal1`

`-type {Assura | Calibre | PVS | Hercules | ICV | CDNLitho | CDNCMP | inShapeLitho | CalibreLitho | DRV | ClockTran | CLP | VerifyPower}`	

	Specifies the file type. The following file types contain violation markers: • Assura - For Assura violations • Calibre - For Calibre violations • PVS - For PVS violations • Hercules - For Hercules violations • ICV - For ICValidator report violations • CDNCMP - For Cadence CMP Predictor (CCP) violations • DRV - For transition violations (reportTranViolation) • ClockTran, CLP, and VerifyPower - For low power violations The following file types contain hotspot markers, which are used to identify areas susceptible to lithography problems: • CDNLitho (LPA as well as NanoRoute litho HIF) • inShapeLitho • CalibreLitho
`-xoffset` *value*	Specifies the X offset. When you specify the X and Y offsets, Violation Browser displays the violations at the correct coordinates after accounting for the offsets.
`-yoffset` *value*	Specifies the Y offset, which Violation Browser takes into account to displays violations at the correct coordinates.

Examples

- The following command loads a CDN Litho marker file named `LPA.hif`:

```
loadViolationReport -type CDNLitho -xoffset 0.05 -yoffset 0.05 -filename LPA.hif
```

- The following command loads a NanoRoute litho repair file named `NR.hif`:

```
loadViolationReport -type CDNLitho -filename NR.hif
```

- The following command loads a Calibre litho file named `litho.rdb`:

```
loadViolationReport -type CalibreLitho -filename litho.rdb -rulemap rule.map
Contents of rule map
====================
M1.W_4.5 width metal1
```

Related Information

- The following sections in the Identifying and Viewing Violations chapter of *User Guide*:
  - Viewing Violations With the Violation Browser
  - Clearing Violations
- "Load Violation Report" section in the Tools Menu chapter of the *Innovus Menu Reference.*
- `route_fix_signoff_drc`

optimizePattern

```
optimizePattern
[-help]
[-backout string]
[-check_locally]
[-config string]
[-create]
[-deck string]
[-dir string]
[-envLayers string]
[-exclude string]
[-excludeHalo string]
[-extract]
[-extractHalo string]
[-extractSquish]
[-find]
[-findLimit string]
[-fix]
[-fixLimit string]
[-gdsList string]
[-hintsOnly]
[-include string]
[-incrCheck string]
[-input string]
[-layerMapFile string]
[-layers string]
[-load]
[-mapFile string]
[-markerLayers string]
[-markers string]
[-noCompress]
[-noViolationBrowser]
[-oasis]
[-oasisList string]
[-save]
[-selection string]
[-streamOutOptions string]
[-techFile string]
[-verify_function string]
```

Runs LPA on the selected design to flag Litho hotspots. It helps in creating, extracting, finding, and fixing patterns. This command uses squish and TVL patterns to find locations within a layout. Once a is pattern defined, you can use it for a variety of operations. Patterns can be extended to define via replacement, landing pad extensions, polygon insertions, and routing blockages. Patterns are commonly used to define markers for DRC or Litho hotspots.

Parameters

`-help`	Outputs a brief description that includes the type and default information for the `optimizePattern` parameters. For a detailed description of the command, use the man command: `man optimizePattern`
`-backout` *string*	Reverts the fixing on the nets.
`-check_locally`	Applies fixing with the local DRC check.
`-config string`	Specifies the name and path of the LPA custom configuration file. This file, if specified, controls all the run options of LPA.
`-create`	Streams out the Innovus design into a GDS file and loads it in `design_review` or `k2_viewer`. ⚠ Specify the stream out map file with `-mapFile`.
`-deck string`	Specifies the pattern deck to be used for LPA hotspot detection. ⚠ • The configuration variable, `lpaTechFile`, needs to be specified in the configuration file. • The LPA layer map needs to contain all the layers mentioned in the deck file.
`-dir string`	Specifies the LPA output directory. This directory contains individual subdirectories for each layer.

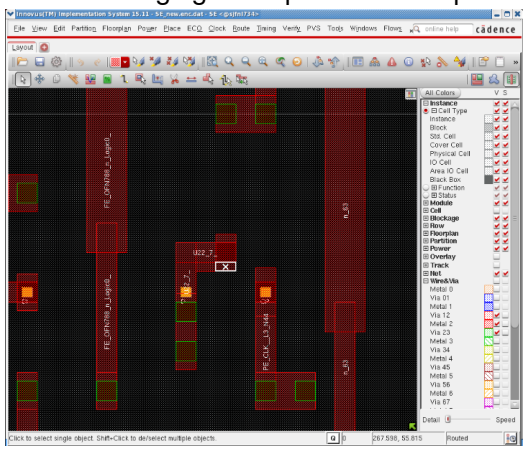
`-envLayers string`	Specifies the list of extra layers to be included in `streamOut` GDS/OASIS. ⚠ The layer number and data type of these extra layers need to be included in the LPA layer map.
`-exclude string`	Specifies the list of nets that need not be used while fixing hotspots.
`-excludeHalo string`	Specifies an extra halo around the marker to check the exclude nets. ⚠ This parameter works with `-exclude`.
`-extractSquish`	Produces squish patterns as opposed to TVL patterns, for extraction.
`-find`	Detects and fixes LPA hotspots at the time of run.
`-findLimit string`	Specifies the number of hotspots to be detected.
`-fix`	Helps LPA in applying the hints to fix hotspots.
`-fixLimit string`	Specifies the number of hotspots to be fixed.
`-gdsList string`	Specifies the name and path of the text file containing the list of GDS files for LEF abstracts.
`-hintsOnly`	Specifies LPA to turn off fixing of hotspots after detection. ⚠ The default behavior of the `-find` parameter is to find and fix hotspots.
`-include string`	Specifies the list of nets to be used while fixing hotspots.

`-incrCheck string`	Specifies the name and path of the HIF file to be used for incremental validation. This HIF file includes the locations that identify the areas affected by each hotspot fix. LPA reads these locations and performs incremental checking only in these areas. This reduces the time required for validation.
`-input string`	Specifies the path of a GDS. It opens design review with the given file.
`-layerMapFile string`	Specifies the path of LPA layer map. ⚠ It can also be specified by including the following string in the configuration file: `setConf layerMapFile`
`-layers string`	Specifies the list of layers to run on LPA. ⚠ It can also be specified by including the following string in the configuration file: `setConf layerOrder`
`-load`	Loads guideline fixing from the database.
`-mapFile string`	Specifies the name and path of the Stream Out Map file created by Innovus to map the GDS stream to the layers in the Innovus database. If the GDS List file and Stream Out Map file are specified, LPA runs on the potential IP blocks defined in these files.
`-markerLayers string`	Specifies the layers to be extracted in pattern extraction. ⚠ This parameter works along with `-markers`.
`-markers string`	Specifies the path to a marker file in the HIF/RDB format for pattern extraction.
`-noCompress`	Inputs GDS instead of `gds.gz`.

`-noViolationBrowser`	Prevents LPA from loading the detected hotspots to the Innovus Violation Browser.
`-oasis`	Ensures that the stream out process uses OASIS output format.
`-oasisList`	Specifies the name and path of the text file containing the list of OASIS files for LEF abstracts.
`-save`	Saves guideline fixing from the database.
`-selection`	Specifies a list of hotspot type and severity to be extracted in the following format: `[list <type1>:<level1> <type2>: <level2> …]`
`-streamOutOptions`	Enables additional `streamOut` command options.
`-techFile`	Specifies the path of non-default LPA Mx technology file.
`-verify_function`	Specifies the DRC function.

Examples

- The following command presents how to use `optimizePattern` to find the hotspots:
  ```
  optimizePattern -find -deck deck.cpaHook.xml -mapFile streamOut.map -streamOutOptions { -outputMacros } -dir Find_hook -layers "M2"
  ```
- The following command presents how to fix the patterns with the `hinsts` created:
  ```
  optimizePattern -find -deck deck.cpaHook.xml -mapFile streamOut.map -streamOutOptions { -outputMacros } -dir Find_hook -fix -layers "M2"
  ```
  The following figure depicts an example of a fixed location:



- The following command applies double via patterns:
  ```
  optimizePattern -find -deck deck.cpaDoubleVia.xml -dir Find_via -mapFile streamOut.map -streamOutOptions { -outputMacros } -layers "M2"
  ```
- The following command finds the double via patterns and fixes 200 of those:
  ```
  optimizePattern -find -deck deck.cpaDoubleVia.xml -dir Find_via -mapFile streamOut.map -streamOutOptions { -outputMacros } -layers "M2" -fix -findLimit 200
  ```

Related Topics

- Integration with LPA and CCP chapter in the *Innovus User Guide*
- *Litho Physical Analyzer User Guide*

reportFreqViolation

```
reportFreqViolation
[-help]
[-outfile fileName]
[-selInstFile selInstFileName]
[-em_type {avg | peak | rms | all}]
[-excInstFile excInstFileName]
[-tableScale float]
[-detailed]
[-slew {min | max}]
[-sort {name | freq | minDiff | maxDiff | minFreq | maxFreq}]
[-reportFormat integer]
[-view viewName]
```

Runs electromigration (EM) analysis based on the frequency table defined in the Liberty library and generates a frequency violation report.

You must run the following commands prior to setting the `reportFreqViolation` command:

- `write_tcf` *activityFile*
- `read_activity_file -format TCF` *activityFile*
- `propagate_activity`
- `read_activity_file -set_net_freq true -format TCF activityFile`
- `set_analysis_view -setup default_view_setup -hold default_view_hold`
- `setAnalysisMode -checkType hold`

Note: If activity files are not defined, then the frequency can also be calculated on a pin by the clock period. There will also be a default frequency of 500M (toggle count 1e+9) if there is no clock phase on this pin.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `reportFreqViolation` parameter. For a detailed description of the command and all its parameters, use the man command: `man reportFreqViolation`
`-detailed`	Generates a detailed report containing information for instances with violations.
`-em_type {avg | peak | rms | all}`	Specifies the electromigration type. *Default:* `avg`

`-excInstFile` *excInstFileName*	Specifies an instance file that lists the names of instances to be excluded from the frequency violation report. **Note:** The instance file is a text file that contains a list of instances. Each instance name appears in one line in the file. The comments begin with "#". Leading or ending spaces in instance names are ignored.
`-outfile` *fileName*	Specifies the report file for the violation data.
`-reportFormat` *integer*	Specifies a format for the violation report. The report details vary according to the selected report format. You can specify report formats 0, 1, or 2. For more information on the report formats, see *Report Format* section below. *Default*: 0
`-selInstFile` *selInstFileName*	Specifies a file that lists the names of instances to be included in the frequency violation report. **Note:** The instance file is a text file that contains a list of instances. Each instance name appears in one line in the file. The comments begin with "#". Leading or ending spaces in instance names are ignored.
`-slew {min | max}`	Specifies the minimum or maximum slew value. *Default*: *min*
`-sort {name | freq | minDiff | maxDiff | minFreq | maxFreq}`	Sorts the instance-based frequency data by name, freq, minDiff, maxDiff, minFreq, and maxFreq. By default, the violation report is not sorted. You can use one of the following options: ◦ `name`: Name of a pin or arc. ◦ `freq`: Frequency value of a pin or arc. ◦ `minDiff`: Difference between freq and minFreq of a pin or arc. ◦ `maxDiff`: Difference between freq and maxFreq of a pin or arc. ◦ `minFreq`: Minimum frequency value of a pin or arc as defined in the frequency table. ◦ `maxFreq`: Maximum frequency value of a pin or arc as defined in the frequency table.
`-tableScale` *float*	Specifies the scale factor for EM table value. *Default:* `1.0`
`-view` *viewName*	Specifies the view for which to generate the frequency violation report. Use this parameter when working with multi-mode multi-corner (MMMC) designs.

Example

The following command reports instances that violate frequency constraints and lists them in the `freqViol.rpt` report file:

```
reportFreqViolation -outfile freqViol.rpt
```

Report Format

The report formats are described below.

- Format `0`
  The following command generates a report providing details of `minFreq`, `Freq`, `Tran`, `Cap`, `Remark`, and `Arc/Pin` and directs the output to report file `summaryViol.rpt`:

  ```
  reportFreqViolation -outfile summaryViol.rpt -reportFormat 0
  ```

  For example:

  ```
  minFreq    Freq    Tran     Cap         Remark    Arc/Pin
  5.985      1.009   0.0356   0.003103              tt0/data/r3/add_27/U1_1_6/Y
  ```

- Format `1`
  When `reportFreqViolation -reportFormat` is set to `1`, the report provides details as shown in the following example:

  ```
  Status   Remark   freq   minFreq   maxFreq   Arc/Pin
  [P]      *        1.009  5.985     5.995     tt0/data/r3/add_27/U1_1_6/Y
  ```

- Format `2`
  A detailed report is generated when `reportFreqViolation -reportFormat` is set to `2`, as shown in the following example:

  ```
  Status  Remark  freq   minFreq  maxFreq    minTran    maxTran    minCap         maxCap     Arc/Pin
  [P]     *       1.009  5.985    5.995      3.560e-02  3.790e-02  3.103e-03      3.103e-
  03  tt0/data/r3/add_27/U1_1_6/Y
  ```

  Where:

`Status`	Arc or pin status - `F` (fail), `V` (violation), `P` (pass), or `?` (unknown)
`Remark`	Indicates whether the pin is a clock
`freq`	Working frequency of the arc or pin
`minFreq`	Minimum frequency value taken from the frequency table
`maxFreq`	Maximum frequency value taken from the frequency table

`minTran`	Minimum slew value
`maxTran`	Maximum slew value
`minCap`	Minimum capacitance load
`maxCap`	Maximum capacitance load
`Arc/Pin`	Name of the arc or pin

Related Information

The Identifying and Viewing Violations chapter of *User Guide*.

run_pegasus_drc

```
run_pegasus_drc
[-help]
pvlFile
[-area {x1 y1 x2 y2}]
[-dp N]
[-error integer]
[-gds_file fileName | -oasis_file fileName]
-mapfile fileName
[-merge list_files]
[-noclean]
[-oa_view view_name]
[-output fileName]
[-repRoutingOnly]
[-report string]
[-stream_out]
[-units integer]
[-version]
[-work_directory working_directory]
```

Calls the Cadence Pegasus application to check DRC rules with a sign-off Pegasus rule deck.

This command is loaded only if you have a Pegasus installation and the Pegasus bin directory is in your Linux search path (e.g. the command pegasus can be run).

It is possible there are different options than described here depending on the version of Pegasus that you have installed. Use `run_pegasus_drc -help` to view the exact command syntax for your current Pegasus installation.

`run_pegasus_drc` will assemble the necessary files, call pegasus, and then read back the Pegasus violations so they are visible in the Innovus violation browser. The call to pegasus requires the appropriate Pegasus licenses in order to run.

The `run_pegasus_drc` command takes as input:

- A GDSII/OASIS layout file (normally created by `streamOut` command in Innovus)
- A layer map file
- A PVL (Physical Verification Language) rule file that contains the DRC rules to check
- The name of the output Pegasus results database file to be created
- Other optional arguments, such as distributed processing, area to check, number of errors to output, and so on.

Details about the various file formats can be found in the Pegasus documentation.

This command returns a status of `0 (tcl_OK)` or `1 (tcl_ERROR)`. Error messages or warnings are written to the Innovus log file.

Parameters

`-help`	Prints out the command usage.

pvlFile	Specifies the Pegasus rule file. Pegasus supports the same PVL rule deck, which is used by PVS.
-area {*x1 y1 x2 y2*}	
	Defines the area to be checked. The default is the entire chip window.
-dp *N*	Specifies the number of local cores to use. The default is 1.
-error *integer*	Specifies the maximum number of errors to output. The default is 1000000.
-gds_file *fileName*	Specifies a GDSII file to be read by Pegasus. If this parameter is specified, then the -mapfile parameter is ignored and streamOut is not run.
-mapfile fileName	Specifies a stream map file. In the usual case, the Innovus map file used by the Innovus streamOut command is specified. If this parameter is specified but the parameters -gds_file or -oasis_file is not specified, then the streamOut command is automatically run before Pegasus starts. The streamOut command is: streamOut .pegasus_xxx/*topcell*.gds.gz -mapFile *mapfile* -libName designLib -outputMacro -mode ALL [-units *integer*] Here, *topcell* is the top cell name of the design and *mapfile* is the file name specified with the -mapfile parameter. The optional -units *integer* is taken from the run_pegasus_drc -units value (if given). If not given, Innovus streamOut uses the Innovus DB unit value. Refer to the Innovus streamOut command for more details. For information on the map file format, see "About the GDSII or OASIS Map File" in the Importing and Exporting Designs chapter of the *Innovus User Guide*. **Note:** The -mapfile parameter is currently mandatory, even if it is not used due to the presence of -gds_file or -oasis_file. It will become optional in future release.
-merge *list_files*	Specifies the list of external stream files for streamOut.
-noclean	Specifies that the working directory, which is created while the run_pegasus_drc command is running, should not be removed after the run. The working directory name begins with .pegasus.
-oa_view *view_name*	Specifies the temporary OA view.
-oasis_file *fileName*	
	Specifies an Oasis file to be read by Pegasus. If this parameter is specified, then the -mapfile parameter is ignored and streamOut is not run.
-output *fileName*	Specifies the error markers result database in ASCII format. If not specified, *topCellName*.ascii is used.

`-repRoutingOnly`	Ignores standard cell violations. This parameter is equivalent to setting `"abort_on_layout_error no"` in Pegasus rule deck. The `abort_on_layout_error` checks for problems such as missing cell masters and duplicate cell references. If `-repRoutingOnly` is specified, Pegasus does not abort even if any of these errors happen. For more information, refer to the *Cadence Pegasus Developers Guide*.
`-report` *`fileName`*	Specifies the name of the Pegasus DRC summary report file to be created. When a relative path has been specified, the report file is created relative to the Innovus invocation directory. The default is no report file.
`-stream_out`	Uses the `streamOut` command for GDS in the OA flow.
`-units` *`integer`*	Specifies the user-defined DB unit value used for `streamOut` (see `-mapfile` above). This value should be consistent with the DB unit value in PVL rules.
`-version`	Prints the Pegasus version.
`-work_directory` *`working_directory`*	Specifies the current working directory.

Examples

- The following command runs sign-off DRC check with the gds file from Innovus, using the Pegasus rule file `28nm.pvl`. The gds file will be streamed out with the specified map file, `gds.map`, and the gds unit will be 1000 as specified:

  ```
  run_pegasus_drc 28nm.pvl -mapfile gds.map -units 1000
  ```

- The following command runs sign-off DRC check on routing layers within the specified area {0 0 100 100} using 8 local CPUs. The command will report only up to 1000 DRC issues:

  ```
  run_pegasus_drc 28nm.pvl -area {0 0 100 100} -dp 8 -error 1000 -mapfile gds.map
  ```

- The following command runs sign-off DRC check on the existing gds file `ios.gds`, and writes the report file to `pegasus.signoff.db`:

  ```
  run_pegasus_drc 28nm.pvl -report pegasus.signoff.db -gds_file ios.gds
  ```

Related Information

- `streamOut` in *Innovus Text Command Reference*
- "About the GDSII or OASIS Map File" section in the Importing and Exporting Designs chapter of *Innovus User Guide.*

saveDrc

```
saveDrc
[-help]
fileName
[-force]
[-selected]
```

Saves DRC violation markers in the specified file. The `saveDrc` command is useful for situations where you want to save DRC violation markers without having to save the entire design.

For example, if during ECO you find that only 25 of the 500 reported violations are actual violations that need to be corrected immediately, you can:

1. Delete the other 475 violations.
2. Save the 25 violations in a marker file using the `saveDrc` command.
3. Clear the violations from the design window.
4. Load the marker file containing the saved violation markers using the `loadDrc` command.

Use the `saveDrc` command after markers have been flagged by any of the following commands:

- `verify_drc`
- `verifyMetalDensity`

You can also use this command to save Calibre violation markers after a Calibre file has been loaded using the `loadViolationReport` command.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `saveDrc` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man saveDrc`
`fileName`	Saves the DRC violation markers in a file with the specified name.
`-force`	Creates drc file with a 0 drc count. **Note:** When there is no drc in the design, drc file is not generated (Default).
`-selected`	Saves only the selected DRC violation markers.

Examples

The following command saves markers to the `VG.drc` file:

```
saveDrc VG.drc
```

Related Information

- verify_drc
- The following sections in the Identifying and Viewing Violations chapter of *User Guide*:
  - Viewing Violations With the Violation Browser
  - Clearing Violations

set_signoff_verify_design_config

```
  set_signoff_verify_design_config
[-help]
[-abort_on_layout_error {yes | no}]
[-abort_on_missing_rulecheck {yes | no}]
[-area {x1 y1 x2 y2}]
[-attach_instance_name attribute_number]
[-attach_net_name attribute_number]
[-attach_net_prop {{prop_name attr_num} ... }]
[-auto_merge_base_class string]
[-config_file file_name]
[-control_file file_name]
[-core_license license_combination]
[-die_area_as_boundary]
[-dp n]
[-dp_timeout seconds]
[-erc_checks]
[-error_limit n]
[-extra_options string]
[-format {stream | oasis}]
[-ignore_blockage]
[-ignore_fill]
[-keep_data key_file]
[-layer_map_file streamOut_map_file]
[-layers layer_names | -layer_range {top_layer | bottom_layer}]
[-layout_path string]
[-lib_name library name]
[-license_dp_continue]
[-license_stacking {larger_license | same_license}]
[-license_timeout number_of_seconds]
[-merge string]
[-merge_path string]
[-mode {ALL FILLONLY NOFILL NOINSTANCES}]
[-mp n]
[-mt n]
[-net_map_file string]
[-nets string]
[-no_query]
[-no_structure_name]
[-offset {x y}]
[-output_macros]
[-process_node foundary_node]
[-pvs_fill]
[-report_file]
[-reset {parameter_name | all}]
[-rule_file_drc file_name]
[-rule_file_smart_verify_lvs file_name]
[-run_name directory_name]
[-selected_inst]
```

```
[-skip_auto_load_results]
[-stripes number]
[-structure_name structure_name]
[-transform_to_inst_master]
[-ui_data]
[-uniquify_cell_names]
[-units {100 | 200 | 400 | 800 | 1000 | 2000 | 4000 | 8000 | 10000 | 20000}]
[-working_dir working_directory]
```

Sets the global parameters for the `signoff_verify_design` command. Parameters specified with the `set_signoff_verify_design_config` command are used automatically whenever `signoff_verify_design` is being run. The settings are saved in the design database and will be restored after reloading a database saved earlier.

Use the `get_signoff_verify_design_config` command to return the current values of the `set_signoff_verify_design_config` parameters.

Parameters

`-help`	Prints a brief description of the `set_signoff_verify_design_config` command usage. For a detailed description of the command, use the `man` command: `man set_signoff_verify_design_config`
`-abort_on_layout_error {yes | no}`	Optional keyword to specify whether to continue processing of the design in case of layout error or exit.
`-abort_on_missing_rulecheck {yes | no}`	Optional keyword to specify whether to continue processing of the design in case of missed rule name.
`-area {x1 y1 x2 y2}]`	Optional keyword to specify a set of coordinates within which to trim metal fill.
`-attach_instance_name attribute_number`	Optional keyword to specify the instance name to attach.
`-attach_net_name attribute_number`	Optional keyword to specify the net name to attach.
`-attach_net_prop {{prop_name attr_num} ... }`	Optional keyword to specify the net property to attach.
`-auto_merge_base_class string`	Optional keyword to specify the list of cell base classes to auto merge. It must be one or combination of block, core, corner, cover, none, and pad.
`-config_file file_name`	Optional keyword to specify the Pegasus configuration file.

`-control_file` *file_name*	Optional keyword to specify PVL control file.
`-core_license` *license_combination*	Optional keyword to set exact combination of licenses to use.
`-die_area_as_boundary`	Optional keyword to write the die area as a polygon, that is, a boundary. Otherwise, the `streamOut` command writes the die area as a rectangle, that is, a box.
`-dp` *n*	Optional keyword to run Pegasus in multi-processor mode, n is number of processors. By default, it gets data from Innovus multi CPU usage option.
`-dp_timeout` *seconds*	Optional keyword to specify how long to wait for the worker processes to start.
`-erc_checks`	Optional keyword to enable ERC checks to run (also requires `-connect_all` option).
`-error_limit` *n*	Optional keyword to limit the number of output errors.
`-extra_options` *string*	Optional keyword to specify to run Pegasus with additional set of parameters defined in the command string.
`-format {stream | oasis}`	Optional keyword to specify the format of the flow.
`-ignore_blockage`	Optional keyword to ignore design blockage objects.
`-ignore_fill`	Optional keyword to ignore design fill objects.
`-keep_data` *key_file*	Keeps debugging data for R&D (requires a key file).
`-layer_map_file` *streamOut_map_file*	Optional keyword to specify `streamOut` layer map file.
`-layer_range {top_layer | bottom_layer}`	Optional keyword to specify top layer name and bottom layer name for selected range.
`-layers` *layer_names*	Optional keyword to specify that only rules that use selected layers will be processed.
`-layout_path` *string*	Optional keyword to specify path to layout file to use for the run instead of streaming out.
`-lib_name` *library name*	Optional keyword to specify library name.
`-license_dp_continue`	Optional keyword to specify that in DP if required number of licenses cannot be obtained, continue with fewer CPUs based on the number of licenses obtained.

`-license_stacking {larger_license | same_license}`	Optional keyword to specify license borrowing behavior.
`-license_timeout` *`number_of_seconds`*	Optional keyword to specify how long to wait (in seconds) if a license is not immediately available when the run starts. The default is 300 seconds.
`-light`	Enables rule selection preset for the light check.
`-merge` *`string`*	Optional keyword to specify the list of external stream files.
`-merge_path` *`string`*	Optional keyword to specify the list of directories with external stream files.
`-mode {ALL | FILLONLY | NOFILL | NOINSTANCES}`	Optional keyword to specify the mode. It identifies the layers to write.
`-mp` *`n`*	Optional keyword to specify number of workers.
`-mt` *`n`*	Optional keyword to specify number of threads per worker. The default is 1.
`-net_map_file` *`string`*	Optional keyword to specify the path to file containing net name mapping information.
`-nets` *`string`*	Specifies the list of net names to check in the `SmartVerify` LVS runs.
`-no_query`	Disables queries to get cell names and types from the Innovus database.
`-no_structure_name`	Optional keyword to merge GDSII file without loading an Innovus database when the `-merge` parameter is also specified.
`-offset {`*`x y`*`}`	Optional keyword to specify, in microns, the location of the design in the GDSII output file.
`-output_macros`	Optional keyword to write LEF abstract information such as LEF pin geometries and obstructions for macros.
`-process_node` *`foundary_node`*	Optional keyword to specify process node of used rule deck.
`-pvs_fill`	Optional keyword to merge the PVS fill data attached to the current database into the output GDSII/OASIS.
`-report_file`	Optional keyword to output cell name changed to the file.

`-rule_file_drc` *file_name*	Optional keyword to specify rule file for DRC run.
`-rule_file_smart_verify_lvs` *file_name*	Optional keyword to specify rule file for the `smart_verify_lvs` run.
`-run_name` *directory_name*	Optional keyword to specify name of run directory.
`-selected_inst`	Runs only the selected instance.
`-skip_auto_load_results`	Optional keyword to specify not to run Pegasus RV after successful run.
`-stripes` *number*	Optional keyword to specify the number of stripes that divide the design. The default is 1.
`-structure_name` *structure_name*	Optional keyword to specify the superstructure that consists of all geometries that the map file writes out.
`-transform_to_inst_master`	Transforms data to the selected instances master coordinate space.
`-ui_data`	Optional keyword to save data required for LVS/`smart_verify_lvs` probing and debugging.
`-uniquify_cell_names`	Optional keyword to create unique cell names if there are name collisions when the software merges files.
`-units {100 | 200 | 400 | 800 | 1000 | 2000 | 4000 | 8000 | 10000 | 20000}`	
	Optional keyword to specify the resolution for values in the GDSII/OASIS file.
`-working_dir` *working_directory*	Optional keyword to specify working directory.

Examples

Following are few examples of the set_signoff_verify_design_config command for defining other options required for the run:

- ```
  set_signoff_verify_design_config \
  -output_macros -die_area_as_boundary \
  -rule_file_drc $drcDeck\
  -layer_map_file $gdsOutMap \
  -control_file $drcCtrl \
  -units 2000 \
  -mp 4 -mt 4
  source merge.txt
  ```
  ```

- ```
  set_signoff_verify_design_config \
  -format stream \
  -merge $merge
  ```
- ```
  set_signoff_verify_design_config
  -process_node EXAMPLE_NODE \
  -process_node_data "./RULE/my_custom_template"
  ```

Related Information

- get_signoff_verify_design_config
- signoff_verify_design

set_verify_drc_mode

```
set_verify_drc_mode
[-help]
[-area {lx ly ux uy}]
[-check_illegal_trim_shapes {true | false}]
[-check_ndr_spacing {true | false | auto}]
[-check_only {all | regular | special | selected_net | selected | cell | default}]
[-check_reverse]
[-check_routing_halo {true | false}]
[-check_routing_halo_corner {true | false}]
[-check_same_via_cell {true | false}]
[-check_short_only {true | false}]
[-check_trim_length {true | false}]
[-check_uncolored {true | false}]
[-disable_rules
{jog2jog_spacing|eol_spacing|cut_spacing|min_cut|enclosure|color|min_step|protrusion|min_area|out_of_die
|off_manufacturing_grid|off_routing_track}]
[-enable_post_passive_fill_check {true | false}]
[-exclude_pg_net {true | false}]
[-ignore_cell_blockage {true | false}]
[-ignore_fill_wire {true | false}]
[-ignore_non_rectangle_shapes {true | false}]
[-ignore_trial_route {true | false}]
[-layer_range {layer1 [layer2]}]
[-limit value]
[-max_wrong_way_halo value]
[-report file_name]
[-reset]
[-use_min_spacing_on_block_obs {true | false | auto}]
```

Sets the global parameters for the `verify_drc` command. Parameters specified with the `set_verify_drc_mode` command are used automatically whenever `verify_drc` is being run. The settings are saved in the design database and will be restored after reloading a database saved earlier.

Use the `get_verify_drc_mode` command to return the current values of the `set_verify_drc_mode` parameters.

`set_verify_drc_mode` is pre-configured with a set of default parameter values. The `-reset` parameter can be used to reset all or certain parameters to the default values.

Parameters

`-help`	Prints a brief description of each `set_verify_drc_mode` parameter, including type and default value. For a detailed description of the command and all its parameters, use the man command: man `set_verify_drc_mode`
`-area {x1 y1 x2 y2}`	
	Specifies the coordinates of the area to be verified. If the area specified is bigger than the chip boundary, all the objects inside and outside the die will be verified. If the `-area` parameter is not specified, the objects within the chip boundary are verified against the design rules. *Default:* `{0 0 0 0}`
`-check_illegal_trim_shapes {true | false}`	
	Checks the missing trim shapes for a fulfilled design.
`-check_ndr_spacing {true | false | auto}`	
	Checks non-default rule spacing as a hard rule. Following are the parameter values: • `true`: Checks all non-default rules. • `false`: Skips checking any non-default rule. • `auto`: Checks only the non-default rules with `hard_spacing`. *Default:* `auto` **Note:** When the `setNanoRouteMode -route_enforce_route_rule_on_special_net_wire` parameter is set to `true`, the non-default rule for special net is also checked.
`-check_only {all | regular | special | selected_net | selected | cell | default}`	

	Specifies which types of shapes are checked against the design rules. Following are the various values of this parameter: • `all`: Reports the violations between all routing shapes (regular and special) and all other shapes, including routing blockages and cell pin/obs shapes. • `regular`: Reports only the violations between regular routing shapes (normally created by NanoRoute) and all other shapes. It does not report violations between special routes, routing blockages and cell shapes. This is useful to find violations caused only by NanoRoute. • `special`: Reports only the violations between the special routes and all other shapes. It does not report violations between the regular routes, routing blockages, and cell shapes. This is useful while checking the power routing or pre-routed special routes with isolating these errors from other violations. • `selected_net`: Reports only the violations between the objects on the selected nets. Before running `verify_drc`, the nets need to be selected with the `selectNet` command. • `selected`: Reports only the violations on special wire objects/vias of the secondary PG nets with the specified user class. • `cell`: Reports only the violations between the shapes of cells including the objects inside the same cell and different cells. • `default`: This value is equivalent to specifying the `regular` and `special` at the same time. *Default:* `default`
`-check_reverse`	Reports the `CONVEXCORNER REVERSE` violation in the flow. *Default:* `false`
`-check_routing_halo {true | false}`	
	Checks for routing violations in the routing halo area. It also creates the DRV markers on any routing wire parallel to the block boundary or design boundary. *Default*: `false`
`-check_routing_halo_corner {true | false}`	
	Checks for routing violations in the halo corner. It also creates the DRV markers on all the routing wires in the halo corner. *Default*: `false`
`-check_same_via_cell {true | false}`	
	Checks the DRC issues in the same via cell. *Default*: `false`

`-check_short_only {true | false}`	
	Excludes checking all other DRCs except the short ones.
`-check_trim_length {true|false}`	
	Checks for the maximum length on the trim shape layer.
`-check_uncolored {true | false}`	
	Reports the color change violation when the DPT layer shapes have no color.
`-disable_rules {jog2jog_spacing | eol_spacing | cut_spacing | min_cut | enclosure | color | min_step | protrusion | min_area | out_of_die | off_manufacturing_grid | off_routing_track}`	

	Disables checking of the specified design rules. The supported values for the rule list are: `jog2jog_spacing`, `eol_spacing`, `cut_spacing`, `min_cut`, `enclosure`, `color`, `min_step`, `protrusion`, `min_area`, `out_of_die`, `off_manufacturing_grid`, and `off_routing_track`
	Use this parameter to make only specific checks with `verify_drc` and to skip checking certain design rules.
	Examples:
	The following set of commands disables the `eol_spacing` check and runs `verify_drc`. In other words, all other rules are checked by `verify_drc`, except `eol_spacing`.
	`set_verify_drc_mode -disable_rules {eol_spacing}`
	`verify_drc`
	Default: "" This means no rules are disabled.

`-enable_post_passive_fill_check {true | false}`	
	Checks if the track is fully filled for the `full_fill` trim layer.
`-exclude_pg_net {true | false}`	
	Disables the design rule checking on power and ground nets. *Default*: `false`
`-ignore_cell_blockage {true | false}`	
	Ignores geometries of cell blockages inside cell. *Default:* `false`
`-ignore_fill_wire {true | false}`	
	Ignores metal fill wires while checking DRCs.
`-ignore_non_rectangle_shapes {true | false}`	

	When set to `true`, this parameter ignores checking DRC markers for the objects with the non-rectangular shapes. When set to `false`, it checks DRC markers for the objects of all shapes. *Default*: `true`
`-ignore_trial_route {true | false}`	
	Ignores early global route objects while checking design rules. *Default*: `false`
`-layer_range {`*`layer1`* `[`*`layer2`*`]}`	
	Checks all the metal and cut layers between the specified bottom and top layers. You can use the full layer name, or abbreviations such as 1 for the routing layer number, etc. For example, `-layer_range {M1 M3}` will check all the metal and cut layers between `M1` and `M3` (including `M1` and `M3`). If *`layer2`* is not specified, it only checks design rules on *`layer1`*.
`-limit` *`value`*	Specifies the maximum number of errors to be reported. *Default:* 1000
`-report` *`filename`*	Specifies the name of the output report file, which will contain the detailed violation report. If the `-report` parameter is not specified, no external report will be saved. **Note:** The output file will be overwritten each time `verify_drc` is run, if the filename is not changed in the meantime. *Default:* `""`
`-max_wrong_way_halo` *`haloLength`*	
	Specifies the length up to which short jog shapes in halo area can be ignored. Routing halos are used to avoid cross-coupling between top-level and inside-the-block routes, and short jogs do not affect cross-coupling much. Normally, it is better to have some short jogs rather than more vias. When this option is specified, any segment with length less than *`haloLength`* is ignored for wrong way check, even if it overlaps with the halo region. `verify_drc` reports violations only for jog shapes with lengths greater than *`haloLength`*. *Default:* If you do not specify the `-max_wrong_way_halo` option, `verify_drc` uses 4* *`pitch`* as the default value, where *`pitch`* is the METAL2 routing pitch.
`-reset`	Resets all, or a list of specified parameters. If no parameter(s) are specified, this parameter resets all the parameters. Any values, that do not already match the default value, are reset and reported. Values, that already match the default value for a particular parameter, will not be reported. Example: `set_verify_drc_mode -reset -report -limit`
`-use_min_spacing_on_block_obs {true | false | auto}`	

	Performs as below with various parameter values: • `true`: Checks the Macro OBS with `minSpacing`. If the OBS SPACING was specified in the LEF file, it honors the SPACING from LEF. • `false`: Considers the Macro OBS as real metal to check the spacing rules. If the OBS SPACING was specified in the LEF file, it honors the SPACING from LEF. • `auto`: Considers the Macro OBS as real metal, if the OBS width is lesser than or equal to `maxWidth`. It also checks the Macro OBS with `minSpacing` if the OBS width is greater than `maxWidth`. If the OBS SPACING was specified in the `LEF` file, it honors the SPACING from LEF. *Default*: `auto`

Related Information

- The Identifying and Viewing Violations chapter of *User Guide*.
- verify_drc
- `get_verify_drc_mode`

setIsolatedCutRule

```
setIsolatedCutRule
[-help]
-spacing spacing1
[-cornerSpacing spacing2]
[-layer {cutLayer_list}]
```

Specifies spacing rules for isolated cut layers. Use this command before running the `verifyIsolatedCut` command to set the spacing rules against which `verifyIsolatedCut` should check isolated cuts.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `setIsolatedCutRule` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man setIsolatedCutRule`
`-cornerSpacing` *spacing2*	Specifies the maximum distance *spacing2* for the corner check. If all corners of the cut being validated have another cut within *spacing2* in the quadrant, the cut being checked is not a violation. **Note:** `-connerSpacing` is ignored if *spacing2* is smaller than `-spacing` *spacing1*.
`-layer` `{`*cutLayer_list*`}`	Sets the spacing rule for specified cut layers. If this parameter is not specified, the rule is applied for all cut layers.
`-spacing` *spacing1*	Specifies the maximum distance between the cut being validated to its nearest cut. If the actual distance is less than or equal to *spacing1*, `verifyIsolatedCut` does not report a violation. If the actual distance between the cut being validated and its nearest cut is greater than *spacing1*, `verifyIsolatedCut` reports a violation.

Examples

- The following command specifies that the maximum distance between the cut being validated and the nearest cut should be `0.5`:

  ```
  setIsolatedCutRule -spacing 0.5
  ```

- The following set of command specifies that the maximum distance between the cut being validated and the nearest cut should be `0.1`. In addition, the maximum distance for the corner check should be `0.5`. All cuts that violate these spacing rules are marked:

  ```
  setIsolatedCutRule -spacing 0.1 -cornerSpacing 0.5
  verifyIsolatedCut
  ```

Related Information

- The following sections in the Identifying and Viewing Violations chapter of *User Guide*:
  - Verifying Isolated Cuts
  - Viewing Violations With the Violation Browser

setSnapGrid

```
setSnapGrid
[-help]
-layer {layer_list}
[-pitch x y]
```

Creates a global snapping grid for wiring (including metal fill shapes and via cuts). The grid defines the snap pitch, which is finer than the LEF `PITCH` for the specified layer. The grid is a multiple of the manufacturing grid. You can specify pitch value to check/set routing/metal fill segment's corner edges on the snap grid.

When you specify this command, the wires, metal fill shapes, and via cuts are snapped to this grid, and the edges of the shapes (for example, wires and vias) lie on the manufacturing grid. The `verify_drc` command can check that the shapes are on-grid.

> The snap grid is not saved in the Innovus database. To use the snap grid, you must run this command each time you load a design.

Verify geometry, power planning and routing, metal fill, and wire editing share the grid setting. Before running commands related to these areas, use `setSnapGrid` to define the grid as additional constraints.

For more information, see the commands `add_metal_fill_signoff`, `addViaFill`, or `verify_drc`.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `setSnapGrid` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man setSnapGrid`
`-layer {layer_list}`	Specifies the layers for the snap grid. You can specify metal layers as METAL1, METAL2, and so on, and cut layers as VIA12, VIA23, and so on.
`-pitch x y`	Specifies the pitch for the snap grid. The LEF `PITCH` must be a multiple (for example, 2x, 4x, 8x) of the snap grid for the specified layer. For example, if the LEF `PITCH` is 0.2 µ, you can specify a snap grid pitch for both x and y of `0.1` µ.

Example

The following commands set snap grid settings that are used while verifying geometry:

```
setSnapGrid -layer {1 2 V12 V23} -pitch 0.02 0.02
```

Related Information

- "Viewing Violations With the Violation Browser" in the Identifying and Viewing Violations chapter of *User Guide*.
- The Optimizing Metal Density chapter of *User Guide*.

signoff_verify_design

```
  signoff_verify_design
[-help]
[-abort_on_layout_error {yes | no}]
[-abort_on_missing_rulecheck {yes | no}]
[-area {x1 y1 x2 y2}]
[-attach_instance_name attribute_number]
[-attach_net_name attribute_number]
[-attach_net_prop {{prop_name attr_num} ... }]
[-auto_merge_base_class string]
[-beol string | -feol string | -light]
[-bg]
[-config_file file_name]
[-control_file file_name]
[-core_license license_combination]
[-die_area_as_boundary]
[-dp n]
[-dp_timeout seconds]
[-drc | -merge_library | -generate_context | -smartconnect | -smart_verify_lvs | -smart_verify_drc | -
load_results path_to_rundir]
[-erc_checks]
[-error_limit n]
[-extra_options string]
[-format {stream | oasis}]
[-ignore_blockage]
[-ignore_fill]
[-layer_map_file streamOut_map_file]
[-layers layer_names | -layer_range {top_layer | bottom_layer}]
[-layout_path string]
[-lib_name library name]
[-license_dp_continue]
[-license_stacking {larger_license | same_license}]
[-license_timeout number_of_seconds]
[-lsf]
[-merge string]
[-merge_path string]
[-mode {ALL FILLONLY NOFILL NOINSTANCES}]
[-mp n]
[-mt n]
[-net_map_file string]
[-nets string]
[-no_query]
[-no_structure_name]
[-offset {x y}]
[-output_macros]
[-process_node foundary_node]
[-pvs_fill]
[-report_file]
[-rule_file_drc file_name]
```

```
[-rule_file_smart_verify_lvs file_name]
[-run_name directory_name]
[-selected_inst]
[-skip_auto_load_results]
[-stripes number]
[-structure_name structure_name]
[-transform_to_inst_master]
[-ui_data]
[-uniquify_cell_names]
[-units {100 | 200 | 400 | 800 | 1000 | 2000 | 4000 | 8000 | 10000 | 20000}]
[-working_dir working_directory]
```

Enables configurable signoff checking at every design stage. It integrates Pegasus Signoff and Pegasus SmartVerify™ with Innovus. These checks help you maintain your designs DRC and LVS clean through the design cycle, and help achieve zero DRC and zero shorts out of Innovus. Running signoff checks late in the design involves long iterative runtimes, debug, fixing, and validation cycles. This impacts time to tapeout. This command allows you to run the signoff checks simultaneously without interrupting Innovus.

Parameters

`-help`	Prints a brief description of the `signoff_verify_design` command usage. For a detailed description of the command, use the `man` command: `man signoff_verify_design`
`-abort_on_layout_error {yes | no}`	Optional keyword to specify whether to continue processing of the design in case of layout error or exit.
`-abort_on_missing_rulecheck {yes | no}`	Optional keyword to specify whether to continue processing of the design in case of missed rule name.
`-area {x1 y1 x2 y2}`	Optional keyword to specify a set of coordinates within which to trim metal fill.
`-attach_instance_name attribute_number`	Optional keyword to specify the instance name to attach.
`-attach_net_name attribute_number`	Optional keyword to specify the net name to attach.
`-attach_net_prop {{prop_name attr_num} ... }`	Optional keyword to specify the net property to attach.
`-auto_merge_base_class string`	Optional keyword to specify the list of cell base classes to auto merge. It must be one or combination of block, core, corner, cover, none, and pad.

`-beol` *string*	Optional keyword to provide list of rule selection blocks for BEOL checks.
`-bg`	Optional keyword to run SVD job in background mode.
`-config_file` *file_name*]	Optional keyword to specify the Pegasus configuration file.
`-control_file` *file_name*	Optional keyword to specify PVL control file.
`-core_license` *license_combination*	Optional keyword to set exact combination of licenses to use.
`-die_area_as_boundary`	Optional keyword to write the die area as a polygon, that is, a boundary. Otherwise, the `streamOut` command writes the die area as a rectangle, that is, a box.
`-dp` *n*	Optional keyword to run Pegasus in multi-processor mode, n is number of processors. By default, it gets data from Innovus multi CPU usage option.
`-dp_timeout` *seconds*	Optional keyword to specify how long to wait for the worker processes to start.
`-drc`	Required keyword to run DRC check.
`-erc_checks`	Optional keyword to enable ERC checks to run (also requires `-connect_all` option).
`-error_limit` *n*	Optional keyword to limit the number of output errors.
`-extra_options` *string*	Optional keyword to specify to run Pegasus with additional set of parameters defined in the command string.
`-feol` *string*	Optional keyword to provide list of rule selection blocks for FEOL checks.
`-format {stream | oasis}`	Optional keyword to specify the format of the flow.
`-generate_context`	Generates context for an instance.
`-ignore_blockage`	Optional keyword to ignore design blockage objects.
`-ignore_fill`	Optional keyword to ignore design fill objects.
`-layer_map_file` *streamOut_map_file*	Optional keyword to specify `streamOut` layer map file.

`-layer_range {top_layer | bottom_layer}`	Optional keyword to specify top layer name and bottom layer name for selected range.
`-layers` *layer_names*	Optional keyword to specify that only rules that use selected layers will be processed.
`-layout_path` *string*	Optional keyword to specify path to layout file to use for the run instead of streaming out.
`-lib_name` *library name*	Optional keyword to specify library name.
`-license_dp_continue`	Optional keyword to specify that in DP if required number of licenses cannot be obtained, continue with fewer CPUs based on the number of licenses obtained.
`-license_stacking {larger_license | same_license}`	Optional keyword to specify license borrowing behavior.
`-license_timeout` *number_of_seconds*	Optional keyword to specify how long to wait (in seconds) if a license is not immediately available when the run starts. The default is 300 seconds.
`-light`	Enables rule selection preset for the light check.
`-load_results` *path_to_rundir*	Required keyword to run Pegasus RV with data from provided run directory.
`-lsf`	Optional keyword to specify to distribute the run to different machines using LSF.
`-merge` *string*	Optional keyword to specify the list of external stream files.
`-merge_library`	Generates single merged layout from the list of layouts to merge.
`-merge_path` *string*	Optional keyword to specify the list of directories with external stream files.
`-mode {ALL | FILLONLY | NOFILL | NOINSTANCES}`	Optional keyword to specify the mode. It identifies the layers to write.
`-mp` *n*	Optional keyword to specify number of workers.
`-mt` *n*	Optional keyword to specify number of threads per worker. The default is 1.
`-net_map_file` *string*	Optional keyword to specify the path to file containing net name mapping information.

`-nets` *string*	Specifies the list of net names to check in the `SmartVerify` LVS runs.
`-no_query`	Disables queries to get cell names and types from the Innovus database.
`-no_structure_name`	Optional keyword to merge GDSII file without loading an Innovus database when the `-merge` parameter is also specified.
`-offset {`*x y*`}`	Optional keyword to specify, in microns, the location of the design in the GDSII output file.
`-output_macros`	Optional keyword to write LEF abstract information such as LEF pin geometries and obstructions for macros.
`-process_node` *foundary_node*	Optional keyword to specify process node of used rule deck.
`-pvs_fill`	Optional keyword to merge the PVS fill data attached to the current database into the output GDSII/OASIS.
`-report_file`	Optional keyword to output cell name changed to the file.
`-rule_file_drc` *file_name*	Optional keyword to specify rule file for DRC run.
`-rule_file_smart_verify_lvs` *file_name*	Optional keyword to specify rule file for the `smart_verify_lvs` run.
`-run_name` *directory_name*	Optional keyword to specify name of run directory.
`-selected_inst`	Runs only the selected instance.
`-skip_auto_load_results`	Optional keyword to specify not to run Pegasus RV after successful run.
`-smartconnect`	Runs `smartconnect` check.
`-smart_verify_drc`	Required option to run `smart_verify_drc` check.
`-smart_verify_lvs`	Required option to run `smart_verify_lvs` check.
`-stripes` *number*	Optional keyword to specify the number of stripes that divide the design. The default is 1.
`-structure_name` *structure_name*	Optional keyword to specify the superstructure that consists of all geometries that the map file writes out.

`-transform_to_inst_master`	Transforms data to the selected instances master coordinate space.
`-ui_data`	Optional keyword to save data required for LVS/`smart_verify_lvs` probing and debugging.
`-uniquify_cell_names`	Optional keyword to create unique cell names if there are name collisions when the software merges files.
`-units {100 | 200 | 400 | 800 | 1000 | 2000 | 4000 | 8000 | 10000 | 20000}`	
	Optional keyword to specify the resolution for values in the GDSII/OASIS file.
`-working_dir` *`working_directory`*	Optional keyword to specify working directory.

Examples

- The following command prevents auto-loading of results in Results Viewer, after completion of the software run:
  `signoff_verify_design -drc -working_dir DRC_rundir3 -run_name DRCrun -skip_auto_load_results`
  This will not open the Results Viewer with results once the run completes. To load results from the run, use the `-load_results` parameter, as shown in the example usage below:
  `signoff_verify_design -load_results DRC_rundir3/drc/DRCrun`
- The following command runs DRC and cross probes the run results using the Design Review:
  `signoff_verify_design -drc -working_dir DRC_rundir4 -run_name DRCrun`
- The following command runs DRC checks on a specified area:
  `signoff_verify_design -drc -area {30 0 45 15} -working_dir DRC_rundir5 -run_name beolArea`
- The following command runs DRC checks on a specified layers:
  `signoff_verify_design -drc -layer_range {Metal4 Metal5} -working_dir DRC_rundir6 -run_name beolM4M5`
- The following command specifies a single layer to run checks:
  `signoff_verify_design -drc -layers {Metal4 Metal4} -working_dir DRC_rundir7 -run_name beolM4`
- Following is an example of using the `-beol`, `-layer_range`, `-working_dir`, and `-run_name` parameters together:
  `signoff_verify_design -drc -beol {all} -layer_range {Metal2 Metal4} -working_dir workdir_drc -run_name beol_M2M4`
- Following is an example of using the `-beol`, `-area`, `-working_dir`, and `-run_name` parameters together:
  `signoff_verify_design -drc -beol {all} -area {30 0 45 15} -working_dir workdir_drc -run_name beolArea`

Related Information

- get_signoff_verify_design_config
- set_signoff_verify_design_config

verify_antenna

```
verify_antenna
[-help]
[-detailed]
[-limit integer]
[-no_max_float_area]
[-nets {netNames} | -selected]
[-report filename]
[-use {all | signal | power | clock}]
```

Verifies the process antenna effect (PAE) in multi-thread mode with geometry-based check through the advanced NanoRoute antenna checker feature. It also supports advanced antenna rules, such as `LEF58_ANTENNADIFFGATEPWL` and `LEF58_ANTENNAMAXCUMAREA`. This command supports checking antenna rules on the backside metal areas for both PG and signal nets.

Note: Before running this command, ensure that the process antenna or maximum floating area keywords are specified in the LEF file and the signal nets are routed.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `verify_antenna` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man verify_antenna`
`-detailed`	Generates a report containing the PAE value for all layers of all the nets in a design, including those with no violations. If this parameter is not set, the report will include only those layers in the net that have violations. **Note:** To know the point of violation in the report, use the * mark.
`-limit integer`	Specifies the maximum number of error messages to report.
`-no_max_float_area`	Disables the verification of max floating area violation.
`-nets {netNames}`	Reports the specified named nets. If you specify one net name, you do not need the braces around the name.
`-report filename`	Specifies the name of the report file. The file contains the PAE violation information. *Default:* `design_name.antenna.rpt`
`-selected`	Reports the specified selected nets. Select nets using the `selectNet` command. If you specify one net name, you do not need the braces around the name.
`-use {all | signal | power | clock}`	Specifies the type of net to be checked.

Example

The following command verifies PAE violations in the routed design and generates a report file called `Test7.antenna.rpt`:

```
verify_antenna -report Test7.antenna.rpt
```

Related Information

- The following sections in the Identifying and Viewing Violations chapter of *User Guide*:
  - Verifying Process Antennas
  - Verifying Well-Process-Antenna Violations
  - Verifying Maximum Floating Area Violations
- The following sections in the LEF/DEF Language Reference:
  - Calculating and Fixing Process Antenna Violations
  - "Defining Routing Layer Properties to Create 45 nm and 65 nm Rules" in the LEF Syntax chapter

verify_drc

```
verify_drc
[-help]
[-area {{lx1 ly1 ux1 uy1} {lx2 ly2 ux2 uy2}...}]
[-check_illegal_trim_shapes]
[-check_ndr_spacing]
[-check_only {all | regular | special | selected_net | selected | cell | default}]
[-check_reverse]
[-check_routing_halo]
[-check_same_via_cell]
[-check_short_only]
[-check_trim_length]
[-check_uncolored]
[-enable_post_passive_fill_check]
[-exclude_pg_net]
[-ignore_fill_wire]
[-ignore_trial_route]
[-layer_range {layer1 [layer2]}]
[-limit max_error]
[-report filename]
[-view_window]
```

Checks for DRC violations and creates violation markers in the design database that can be seen on the GUI and browsed with the Violation Browser. Use the `set_verify_drc_mode` command to set global parameters for `verify_drc`. The parameters you specify with the `set_verify_drc_mode` command are then automatically used on running `verify_drc`. The settings are saved in the design so you need not set them each time you run `verify_drc`.

Use `get_verify_drc_mode` to return the current settings for the `set_verify_drc_mode` command. The `verify_drc` command also checks the DRC violations within the specified via. This helps in minimizing the DRC violations before the sign-off. In case the design has any DRC violation, the software flags DRC on all the via masters, such as PG via, regular via, via pillar, and via array. Use `set_verify_drc_mode -check_same_via_cell true` to check via cell violation.

Note: The `verify_drc` command checks only the placed instances. It skips checking the DRC for unplaced instances. This command adds the LEF rule in the description section of the *Violation Browser* to provide more information on the rule that is causing the DRC violation.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `verify_drc` parameter. For a detailed description of the command and all its parameters, use the man command: man `verify_drc`
`-area {{`*`lx1 ly1 ux1 uy1`*`} {`*`lx2 ly2 ux2 uy2`*`}...}`	Specifies the coordinates of one or more area regions to verify. It verifies the objects inside and outside the die, if the area regions are bigger than chip boundary. By default, objects within the chip boundary are verified against the DRC rules. For example, `verify_drc -area {0 0 100 100} {100 100 200 200}`.
`-check_illegal_trim_shapes`	Checks the missing trim shapes for a fulfilled design.
`-check_ndr_spacing`	Checks non-default rule spacing as a hard rule. **Note**: When the `setNanoRouteMode -route_enforce_route_rule_on_special_net_wire` parameter is set to `true`, the non-default rule for special net is also checked.

`-check_only {all | regular | special | selected_net | selected | cell | default}`	Specifies which types of shapes are checked against the DRC rules. Following are the various values of this parameter : • `all`: Reports the violations between all routing shapes (regular and special) and all other shapes, including routing blockages and cell pin/obs shapes. • `regular`: Reports only the violations between regular routing shapes (normally created by NanoRoute) and all other shapes. It does not report violations between special routes, routing blockages and cell shapes. This is useful to find violations caused only by NanoRoute. • `special`: Reports only the violations between the special routes and all other shapes. It does not report violations between the regular routes, routing blockages, and cell shapes. This is useful while checking the power routing or pre-routed special routes with isolating these errors from other violations. • `selected_net`: Reports only the violations between the objects on the selected nets. Before running `verify_drc`, the nets need to be selected with the `selectNet` command. • `selected`: Reports only the violations on special wire objects/vias of the secondary PG nets with the specified user class. • `cell`: Reports only the violations between the shapes of cells including the objects inside the same cell and different cells. • `default`: This value is equivalent to specifying the `regular` and `special` at the same time. *Default:* `default`
`-check_reverse`	Reports the `CONVEXCORNER REVERSE` violation in the flow. *Default:* `false`

`-check_routing_halo`	Checks for routing violations in the routing halo area. It also creates the DRC markers on any routing wire parallel to the block boundary or design boundary. Routing halos are used to avoid cross-coupling between top-level and inside-the-block routes. As short jogs do not affect cross-coupling much, they can be ignored even if they overlap with the halo region. Normally, it is better to have some short jogs instead of more vias. When the `-check_routing_halo` option is specified, any jog shape with length less than the value set for `set_verify_drc_mode -max_wrong_way_halo` is ignored for the wrong way check. `verify_drc` reports violations only for the jog shapes that have lengths greater than the `set_verify_drc_mode -max_wrong_way_halo` setting. *Default:* If the `set_verify_drc_mode -max_wrong_way_halo` option has not been set, `verify_drc` uses `4* pitch` as the default value for the routing halo, where `pitch` is the METAL2 routing pitch.
`-check_same_via_cell`	Checks the DRC issues in the same via cell.
`-check_short_only`	Excludes checking all other DRCs except the short ones.
`-check_trim_length`	Checks the maximum length on the trim shape layer.
`-check_uncolored`	Reports violations with no color and invalid mask color for the multi-pattern (MPT) layers.
`-enable_post_passive_fill_check`	Checks if the track is fully filled for the `full_fill` trim layer.
`-exclude_pg_net`	Excludes the checking of power and ground (P/G) nets. `verify_drc -exclude_pg_net` reports violations caused by routing (special and regular) except for a P/G net.
`-ignore_fill_wire`	Ignores metal fill wires while checking DRCs.
`-ignore_trial_route`	Ignores early global route while checking DRCs.
`-layer_range {layer1 [layer2]}`	Checks between the specified range of metal layers, including the cut layers. You can use the full layer name, or abbreviations like `1` for the first routing layer, etc. For example, `-layer_range {metal1 metal3}` will check all the layers between `metal1` to `metal3`. If `layer2` is not specified, it only checks DRC on `layer1`.

`-limit` *`max_error`*	Specifies the maximum number of errors to be reported. The *`max_error`* value can be any integer from 1 to 2147483646. ⚠ Set *`max_error`* to `0` to remove the limit and report all the DRCs in the design. However, this may impact memory and runtime, if the design has too many DRCs. *Default:* 1000
`-report` *`filename`*	Specifies the report file name, which contains the violation information.
`-view_window`	Checks for DRC violations in the current window.

Example

- Any one of the following commands checks for DRC violations between Metal2 and Metal3 layers:

```
verify_drc -layer_range {2 3}
```

```
verify_drc -layer_range {3 2}
```

- The following command checks for special wire related DRC violations, such as spacing violation between power/ground wire to cell blockage:

```
verify_drc -check_only special
```

Related Information

- The Identifying and Viewing Violations chapter of *User Guide*.
- `checkPlace`
- `clearDrc`
- `get_verify_drc_mode`
- `saveDrc`
- `set_verify_drc_mode`

verifyACLimit

```
verifyACLimit
[-help]
```

Checks for the following types of AC current violations on signal nets.

- Root-mean-square (RMS) current limit violations (I_{rms} violations)
- Peak current limit violations (I_{peak} violations)

In addition, verifyACLimit can be used for average current limit (I_{avg}) analysis.

AC current limit violations are sometimes also referred as wire self-heat violations. DRC manuals have these current limits to avoid over-heating a signal-net wire with too much AC current. To prevent wire self-heating or AC signal electromigration, signal interconnects should be analyzed for their AC current carrying capacity and measured against the AC current limits specified by foundry.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `verifyACLimit` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man verifyACLimit`

Related Information

- `set_signal_em_analysis_mode`
- "Signal ElectroMigration" in the *Voltus IC Power Integrity Solution User Guide*

verifyCMP

```
verifyCMP
[-help]
[-ccp_setup_file string]
[-cpu integer]
[-end_level string]
[-gdsList string]
[-mapFile string]
[-output_dir string]
[-pvs_fill]
[-runMode string]
[-start_level string]
[-tsmc_vcmp string]
[-tsmc_proc string]
[-vmp_files string]
[-vmp_name string]
```

Runs Chemical and Mechanical Polishing (CMP) analysis at block level or chip level during the Sign-Off phase. You can use the verifyCMP command to run the CMP simulation using the Cadence flow.

Note: The `verifyCMP` command is available only if you have the appropriate license and the Manufacturability and Variability Sign-off (MVS) installation path set up. Refer to the "Before You Begin" section of the Integration with LPA and CCP chapter in the *Innovus User Guide*.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `verifyCMP` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man verifyCMP`
`-ccp_setup_file` *string*	Specifies the name and path of the CMP configuration file. This file controls all run options of the CMP simulation.
`-cpu` *integer*	Specifies the total number of CPUs available for the CMP run.
`-end_level` *string*	Specify the end metal level for the CMP simulation. You can use this parameter along with the `-start_level` parameter to control the number of layers for prediction.
`-gdsList` *string*	Specifies the name and path of the text file containing the list of GDS files for LEF abstracts. By default, CMP analysis runs on the interconnect layers in Sign-Off mode but if the GDS List file and Stream Out Map file are specified, CMP runs on the potential IP blocks defined in these files.
`-mapFile` *string*	Specifies the name and path of the Stream Out map file created by Innovus to map the GDS stream to the layers in the Innovus database. If the GDS List file and Stream Out Map file are specified, CMP runs on the potential IP blocks defined in these files.
`-output_dir` *string*	Specifies the output directory path. This directory contains the extraction and prediction results of the CMP run.
`-pvs_fill`	Sets the software to use the PVS fill.
`-runMode` *string*	Launches CCP in the specific run mode.
`-start_level` *string*	Specifies the starting metal level for the CMP simulation. You can use this parameter along with the `-end_level` parameter to control the number of layers for prediction.
`-tsmc_vcmp` *string*	Specifies the location and name of the VCMP engine.
`-tsmc_proc` *string*	Specifies the location and name of the VCMP process file.
`-vmp_files` *string*	Specifies the name and path of the `vmp.xml` file to be used for the CMP run. The extraction and prediction results of the CMP analysis are based on the specifications in your vmp file.
`-vmp_name` *string*	Specifies the name of the vmp process calibration from the `vmp.xml` file.

Related Information

Integration with LPA and CCP chapter of *User Guide*.

verifyConnectivity

```
verifyConnectivity
[-help]
[-allPGPinPort]
[-append]
[-connLoop | -geomLoop | -geomConnect]
[-dividePowerNet]
[-error integer]
[-ignorePGPin cell_name:pin_name]
[-markerOnHighestLayer]
[-net netNames | -selected]
[-noAntenna]
[-noFill]
[-noFloatingMetal]
[-noOpen]
[-noSoftPGConnect]
[-noUnConnPin]
[-noUnroutedNet]
[-noWeakConnect]
[-preferredTopLayer integer]
[-rawViolsMark]
[-removeOldOpenVio]
[-report filename ]
[-tsv abstractdiefilenames]
[-type {all | special | regular}]
[-useNewOpenVio]
[-useVirtualConnection]
[-warning value ]
```

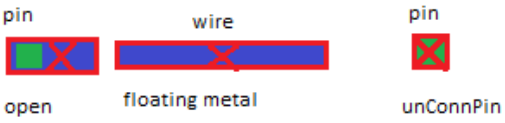
Detects conditions such as opens, unconnected wires (geometric antennas), unconnected pins, loops, partial routing, and unrouted nets; generates violation markers in the design window; reports violations. Running this command does not impact database unless you save the design, which also saves the violation markers.

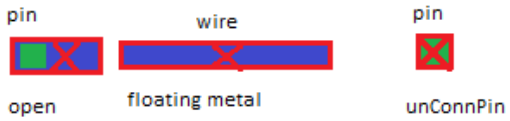
The Verify Connectivity feature can use `setMultiCpuUsage` and other multi-CPU commands for multi-threading.

For more information, see the Multiple-CPU Processing Commands chapter in the *Innovus Text Command Reference* document.

Parameters

`-help`	Outputs a brief description that includes the type and default information of each `verifyConnectivity` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man verifyConnectivity`
`-allPGPinPort`	Verifies all PG ports connection.
`-append`	Displays incremental results in the Violation Browser. By default, violation markers are over-written with new results during each `verifyConnectivity` run. When you specify this parameter, violation markers are appended to the violations of the previous `verifyConnectivity` run.
`-connLoop`	Checks for connectivity loops in regular wires. Detects connectivity loops based on the end points of the center line of a regular wire segment or the center of a via.
`-dividePowerNet`	Divides power nets into four subareas for connectivity verification. Use this parameter to decrease memory usage in 32-bit machines with limited memory. This parameter might increase or decrease the run time, depending on the design.
`-error` *integer*	Specifies the maximum number of errors to report. The command stops when the maximum value is reached. *Default:* `1000`
`-ignorePGPin` *cell_name*`:`*pin_name*	Specifies the cell containing the PG pin that can be ignored during connectivity check.
`-markerOnHighestLayer`	Reports the opening on the highest layer of the pin, which has more one shape.
`-geomConnect`	Checks for connectivity violations of regular wires. Uses a geometrical model instead of the center-line model. In other words, if the wires overlap at any point, they are considered to be connected--they do not have to connect at the center line. Use this parameter if you manually change the routing or use a third-party router that does not route using the centerline connection routing technique.
`-geomLoop`	Checks for loop violations of regular nets using a geometrical model. The nets do not have to overlap on the center line. When you specify this parameter, the Innovus software does not perform any other connectivity checks. Use this parameter if you use a third-party router that does not route using the centerline connection routing technique. In this case, the `-connLoop` parameter might not detect connectivity loop violations.

`-net` *netNames*	Specifies that connectivity should be verified for specified nets only. If you specify more than one net, separate the net names with space and surround the list of net names with braces or quotes. You can use wildcards (`*` and `?`) when you specify net names. **Note:** The `-net` and `-selected` parameters are mutually exclusive. *Default:* If you do not specify either the `-net` or the `-selected` parameter, `verifyConnectivity` verifies all nets.
`-noAntenna`	Ignores violations due to unconnected wires (also called geometrical antennas or dangling wires). A wire is called an antenna when the wire end to nearest via edge distance is more than ½ of the wire width as shown in the picture below: In this picture, the blue rectangle is the wire and the yellow squares are the vias.
`-noFill`	Ignores the metal fill.
`-noFloatingMetal`	Ignores floating metals. A wire is called floating methal, when both ends of the wire are not connected to pin shapes as shown in the picture below. In this picture, the blue rectangle is the wire and the green squares are pin shapes.
`-noOpen`	Ignores open violations. A wire is said to be open when one of its ends is connected to a pin shape but the other end is not connected to any pin as shown in the picture below.  In this picture, the blue rectangle is the wire and the green squares are pin shapes. **Note:** This parameter automatically turns on the `-noUnConnPin` parameter, because open and unconnected pin verification are performed together. A pin shape that is not connected to any wire is called an unconnected pin (`unConnPin`).

`-noUnConnPin`	Ignores violations due to pins that are not connected to any other objects. In the picture below, the pin on the right is an unconnected pin (`unConnPin`) as it is not connected to any wire.  **Note:** This parameter is specified automatically when you specify the `-noOpen` parameter.
`-noUnroutedNet`	Ignores nets that are not routed.
`-noWeakConnect`	Disables checking for routing to more than one port of the weakly connected pin ports.
`-noSoftPGConnect`	Disables checking of soft Power/Ground connects. By default, `verifyConnectivity` checks the connectivity on masterslice layers. If `-noSoftPGConnect` option is specified, connectivity on these layers is ignored.
`-preferredTopLayer` *`integer`*	Specifies the preferred top metal layer. When this parameter is used, verifyConnectivity does not check the specified top layer and above layers.
`-rawViolsMark`	Displays violation markers for opens as the bounding box of the island. By default, violation markers for opens are displayed as polygons that include all wires, pins, and vias that connect to the island.
`-removeOldOpenVio`	Removes old open violations.
`-report` *`filename`*	Specifies the report file for connectivity violation data.
`-selected`	Specifies that connectivity should be checked for selected nets or the nets connected to the pins of selected instances/macros. **Note:** The `-net` and `-selected` parameters are mutually exclusive. *Default:* If you do not specify either the `-net` or the `-selected` parameter, `verifyConnectivity` verifies all nets in the design.
`-tsv` *`abstractdiefilenames`*	Specifies the TSV abstract die files to be used to check the uBumps between the current die and the adjacent dies. Here, *`abstractdiefilenames`* specifies the names of abstract die files created using the command `writeDieAbstract`.

`-type {all | regular | special}`	Specifies the type of wires to verify. `verifyConnectivity` is only concerned with the type of the wire and via (special or regular), but not the type of the pin. Irrespective of the `type` specified, all unconnected terminals (special and regular) are always reported unless `-noUnConnPin` is on. *Default:* `all` **Note:** To check connectivity for whole nets, you do not need to specify the type. **Note:** A pin-open violation is listed under VC DRC type "Problem(s) (ENCVFC-92): Pieces of the net are not connected together". Such a violation means the isolated open island only includes some pins' shapes that come from at least two different ports or pins. Choose one of the following: `all`: Checks all wires, including those that have been previously verified. `regular`: Reports all unconnected pins and open islands that do not include any special wire and via. Does not check a net with no regular objects (wire, via, pin). `special`: Reports all unconnected pins and open islands that do not include any regular wire and via. Does not report isolated open islands that have any regular wire and via.
`-useNewOpenVio`	Uses an open violation marker to show unconnected objects.
`-useVirtualConnection`	Implies a virtual connection for all bumps and external I/O pins of the same net. Set this parameter to override default behavior of `verifyConnectivity`, in which bumps and external I/O pins of the same net are not considered to be interconnected. It is useful in flip chip designs, where the power bumps are connected outside the chip.
`-warning` *value*	Specifies the maximum number of warnings to report. *Default:* `50` *Range:* [`0, 1000000`] *Type:* Integer

Examples

- The following command verifies connectivity for all nets and generates a maximum of 50 errors and 50 warning messages:
  ```
  verifyConnectivity -type all -error 50 -warning 50
  ```
- The following command reports special unconnected pins and open islands that do not include any regular wire and via
  ```
  verifyConnectivity -type special -noAntenna -error 1000 -warning 50
  ```
- The following command reports the open only from 1 to 4 metal layers, and ignores all the open on 5 and the metal layers above 5:
  ```
  verifyConnectivity -preferredTopLayer 5
  ```
- The following command ignores the `VDD1` pin in the `NAND*` cell, and the `VSS1` pin in the `NOR*` cell during

connectivity check:

```
verifyConnectivity -ignorePGPin {NAND*:VDD1 NOR*:VSS1}
```

Related Information

The following sections in the Identifying and Viewing Violations chapter of *User Guide*:

- Verifying Connectivity
- Viewing Violations With the Violation Browser

verifyCutDensity

```
verifyCutDensity
[-area {x1 y1 x2 y2}]
[-detailed]
[-globalDensity]
[-ignoreCellBlock]
[-layer layer_name_list]
[-oversize value]
[-report fileName]
```

Checks the density of specified cut layers or areas of cut layers, or the cut density of the whole chip.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `verifyCutDensity` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man verifyCutDensity`.
`-area {x1 y1 x2 y2}`	Specifies the coordinates of the area to verify. For example, `verifyCutDensity -area {0 0 100 100}`. *Default:* Entire area
`-detailed`	Generates a detailed cut density report.
`-globalDensity`	Checks the density across the entire design for the specified cut layers and writes the results to the report file.
`-ignoreCellBlock`	Disables checking of windows that are fully covered by block or pad obstructions.
`-layer layer_name_list`	Specifies the cut layers to verify. Use the following format to specify layer names: {Via12 Via23} For example, `verifyCutDensity -layer {Via12 Via23}`.
`-oversize value`	Specifies an offset value for the area to verify. The value is in user units (not in DBU), and can be positive or negative. A positive value adds to the area that is verified. For example, if the design covers the area from `(0,0)` to `(100,100)` and you specify `-oversize 10`, `verifyCutDensity` checks the area from `(-10,-10)` to `(110,110)`. *Default:* `0`
`-report fileName`	Specifies the report file for the cut density violation information. *Default:* *designName*`.cutdensity.rpt`

Example

The following command checks the density across the entire design for the layers `Via12` and `Via23` and generates a detailed report named `density.rpt`.

```
verifyCutDensity -layer {Via12 Via23} -report density.rpt -detailed -globalDensity
```

Related Information

- "Overview" in the Identifying and Viewing Violations chapter of *User Guide*.
- The following sections in the Optimizing Metal Density chapter of *User Guide*.
  - Overview
  - Adding Via Fill
- LEF Syntax chapter in the *LEF/DEF Language Reference*

verifyEndCap

```
verifyEndCap
[-help]
[-area {x1 y1 x2 y2}]
[-coreBoundaryOnly]
[-error integer]
[-ignore_macro SCLA,...]
[-powerDomain string]
[-report filename]
[-row string]
[-tripleWell]
[-wrongLocation]
```

Checks whether pre/post cap cells have been inserted correctly based on `setEndCapMode` settings. Use this command after running `addEndCap` to flag violations such as:

- Inserted end cap cell is of wrong type
- End cap cell is missing

If you use `verifyEndCap` with the `-wrongLocation` option, it also checks whether the end cap cells are inserted in the right location. Use the `-powerDomain` option to check end cap cells inserted for a specific power domain.

Additionally, you can use `verifyEndCap -tripleWell` to check triple well insertion. Triple well technology is used to isolate p-well from substrate using a deep n-well.

You can view the violation markers generated by `verifyEndCap` in Violation Browser.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `verifyEndCap` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man verifyEndCap`
`-area {x1 y1 x2 y2}`	Specifies the area in which endcaps are to be verified.
`-coreBoundaryOnly`	Checks endcaps only at the core-boundary and not abutting obstructions. **Note**: The `-coreBoundaryOnly` option does not support triple well cap insertion.
`-error integer`	Specifies the maximum number of errors to report.
`-ignore_macro SCLA,...`	Specifies the macro to be ignored.
`-powerDomain string`	Specifies the power domain to be checked. Use this option to check end cap cells inserted for the specified power domain.
`-report filename`	Specifies the file name for the `verifyEndCap` report.
`-row string`	Specifies the row to check.
`-tripleWell`	Checks triple well insertion. Only checks the cell lists specified in `setEndCapMode`.
`-wrongLocation`	Checks for cap cells that are not inserted in the right location. If you specify this option, `verifyEndCap` marks with a violation marker any cap cell that is not on the beginning/end of the row or not on the boundary of the design/ block design.

Examples

- The following set of commands verifies whether cap cell insertion is as per the `setEndCapMode` settings and generates a default report file in the active folder:

```
setEndCapMode -leftEdge EC_LE10 -rightEdge EC_RE10
verifyEndCap
```

- The following command verifies end cap insertion and generates a report named `endcap.rpt`:

```
setEndCapMode -leftEdge EC_LE10 -rightEdge EC_RE10
verifyEndCap -report endcap.rpt
```

- The following command verifies triple well insertion. It also checks the cap cells that are not inserted in the right location and reports up to `10000` errors:

```
setEndCapMode -leftEdge EC_LE10 -rightEdge EC_RE10 -bottomEdge EC_BE10 \
              -leftBottomCorner EC_LBC10 -leftTopCorner EC_LTC10 \
              -rightBottomCorner EC_RBC10 -rightTopCorner EC_RTC10 \
```

```
                -topEdge EC_TE10
verifyEndCap -tripleWell -wrongLocation -error 10000
```

Related Information

- "Verifying End Cap Violations" in the Identifying and Viewing Violations chapter of *User Guide*.
- "Adding End-Cap Cells" in the Placing the Design chapter of *User Guide*.

verifyIO2BumpConnectivity

```
verifyIO2BumpConnectivity
[-help]
[-error integer]
[-report string]
[-subclass string]
[-warning integer]
[-bump_connect_target | -class_bump ]
[-bumps string | -class_bump ]
[-bumps string | -nets string]
```

Verifies multiBump multiPad connectivity. The flip chip flow in Innovus supports multiple pads to multiple bumps connections. Use the `verifyIO2BumpConnectivity` command, instead of `verifyConnectivity` to verify such connections.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `verifyIO2BumpConnectivity` parameter. For a detailed description of the command and all its parameters, use the `man` command: man `verifyIO2BumpConnectivity`
`-bump_connect_target`	Checks only whether or not the routing matches the connection specified by `bump_connect_target` properties (`io:pin:port`) regardless of the class of the port.
`-bumps` *`string`*	Specifies the name of the bumps that are to be checked.
`-class_bump`	Checks only whether or not the ports of `CLASS BUMP` are routed to the bumps regardless of the `bump_connect_target` properties.
`-error` *`integer`*	Specifies the maximum number of errors to report.
`-nets` *`string`*	Specifies the names of the nets that are to be checked.
`-report` *`string`*	Specifies the name of the report file in which `verifyIO2BumpConnectivity` violation data is recorded. *Default:* `designname.io2bump.rpt`
`-subclass` *`string`*	Specifies the name of the subclasses of which the wires and vias are to be checked.
`-warning` *`integer`*	Specifies the maximum number of warnings to report.

Related Information

The Identifying and Viewing Violations chapter of *User Guide*.

verifyIsolatedCut

```
verifyIsolatedCut
[-help]
[-area {x1 y1 x2 y2}]
[-error integer]
[-layer {cutLayer_list}]
[-maxXY]
[-report filename]
```

Checks cuts in all or specified cut layers against the spacing rules set by `setIsolatedCutRule`. If you use the `-area` parameter, checks cuts within the specified area. Reports violations in the log file and in `verify.isoVia.rpt` or the specified report file.

Note: Violations can be viewed in the Violation Browser.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `verifyIsolatedCut` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man verifyIsolatedCut`
`-area {x1 y1 x2 y2}`	Specifies the coordinates of the area to verify. If you do not specify this parameter, the command checks the whole design. For example, `verifyIsolatedCut -area {0 0 100 100}`. *Default:* Entire area
`-error integer`	Specifies the maximum number of errors to report. *Default:* `1000`
`-layer {cutLayer_List}`	Specifies the cut layers to be checked. If you specify only one layer name, you do not need the braces around the name. For example, `verifyIsolatedCut -layer {Via1 Via2}`. *Default:* All cut layers
`-maxXY`	Specifies that the Square metric should be used for checking isolated cuts. By this metric, another cut should exist within a square window from a certain cut. The size of the square window depends on the spacing rules set by `setIsolatedCutRule`. **Note:** If this parameter is not specified, the Euclidean metric is used for checking isolated cuts.
`-report filename`	Specifies the report file for the isolated cut violation information. *Default:* `verify.isoVia.rpt`

Examples

- The following command checks isolated cuts on all layers and generates a report file named `test.isoCut.rpt`:

  ```
  verifyIsolatedCut -report test.isoCut.rpt
  ```

- The following command checks for isolated cuts in the area specified and generates a report named `verify_isoVia.rpt`:

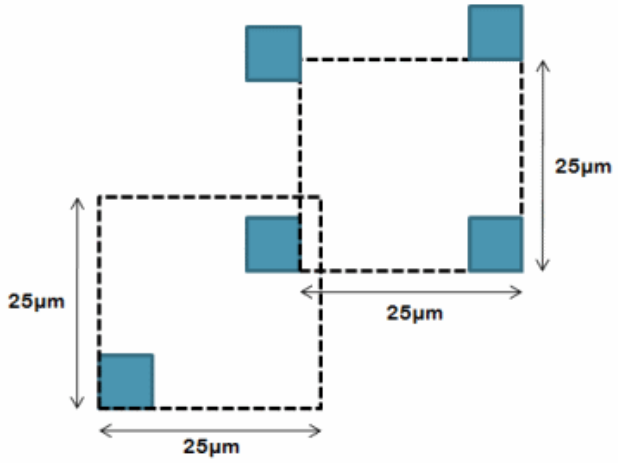
  ```
  verifyIsolatedCut -area {0 0 1600 1600}
  ```

- The following set of commands check for isolated cuts on all layers using the Square metric. Any cut that does not have another cut within a 25um x 25um square window is reported:

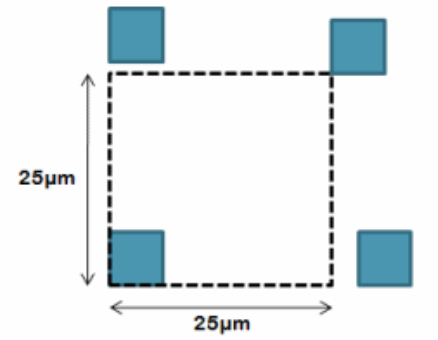
  > setIsolatedCutRule -spacing 25

  > verifyIsolatedCut -maxXY

  In the following pattern, no isolated cut violation is reported as each cut has one or more cuts within a `25um x 25um` square window.

  In contrast, the following pattern is not okay because no other cut exists within a 25um x 25um square window:

Related Information

"Verifying Isolated Cuts" in the Identifying and Viewing Violations chapter of *User Guide*.

verifyLitho

```
verifyLitho
[-help]
-apply {hotspot | ripandreroute | squishHints}
[-check_locally]
[-config string]
[-cpu integer]
-dir string
[-gdsList string]
[-guideline]
[-hints_to_apply string]
[-incrCheck string]
[-layerMapFile string]
[-mapFile string]
[-noCompress]
[-oasis]
[-oasisList string]
[-optionFile string]
[-previousDir string]
[-reportFile string]
{-routingLayersOnly | -signOff | -reportOnly | -loadBlockage}
[-runMode string]
[-streamOutOptions string]
-techFile string
[-verify_function string]
```

Runs LPA on the selected design to flag litho hotspots.

Note: The `verifyLitho` command is available only if you have the appropriate license and the MVS installation path set up. Refer to the 'Before You Begin' section in the Integration with LPA and CCP chapter of *Innovus User Guide*.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `verifyLitho` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man verifyLitho`

`-apply {hotspot | ripandreroute | squishHints}`	Instructs Innovus to perform specific operations based on the results of `verifyLitho`. The operation performed depends on the value you specify with `-apply`: • `hotspot`: Loads the resulting hotspots into violation browser and prepare for regular litho fixing. • `ripandreroute`: Sets up Innovus to fix hotspots using the router with sized and shifted locations from the hotspot marker. • `squishHints`: Applies the fill and via replace hints created by `verifyLitho` automatically if `verifyLitho` is being run using squish patterns.
`-check_locally`	Applies fixing with the local DRC.
`-config` *string*	Specifies the name and path of the LPA custom configuration file. This file, if specified, controls all run options of LPA. It can also specify different instructions for each layer.
`-cpu` *integer*	Specifies the total number of CPUs available for the LPA run.
`-dir` *string*	Specifies the LPA output directory. This directory contains one subdirectory for each layer. The individual subdirectories for each layer (POLY, 1, etc.) are created when LPA is run with the configuration file for that layer.
`-gdsList` *string*	Specifies the name and path of the text file containing the list of GDS files for LEF abstracts. By default, LPA runs on the interconnect layers in Sign-Off mode but if the GDS List file and Stream Out Map file are specified, LPA runs on the potential IP blocks defined in these files.
`-guideline`	Generates LPA guidelines.
`-hints_to_apply` *string*	Specifies the list of hint number to apply.
`-incrCheck` *string*	Specifies the name and path of the HIF file that you want to use for incremental validation. This HIF file includes the locations that identify the areas affected by each hotspot fix. LPA reads these locations and performs incremental checking only in these areas. This reduces the time for validation.
`-layerMapFile` *string*	Specifies the name of the layer map file.

`-loadBlockage`	Loads the blockage file. **Note**: This parameter cannot be used along with `-signOff`, `-routingLayersOnly`, or `-loadBlockage`.
`-mapFile` *string*	Specifies the name and path of the Stream Out Map file created by Innovus to map the GDS stream to the layers in the Innovus database. If the GDS List file and Stream Out Map file are specified, LPA runs on the potential IP blocks defined in these files.
`-noCompress`	Inputs `gds` instead of `gds.gz`.
`-oasis`	Streams out using oasis.
`-oasisList` *string*	Specifies the `oasisOut` oasis list files.
`-optionFile` *string*	Specifies a text file, which can be used to set variables used by the `-apply` operations.
`-previousDir` *string*	Specifies the path to the run directory of a previous Verify Litho run that you want to use for XOR-based incremental validation. When this parameter is specified, the previous run results are compared to the new design and LPA is run only in locations where the layout has changed and where hotspots previously existed, thereby reducing the overall validation run time. This flow is recommended when the design has been changed but no HIF file that identifies the changed areas is available.
`-reportFile` *string*	Specifies the netlist report file.
`-reportOnly`	Generates the netlist report. **Note**: This parameter cannot be used along with `-signOff`, `-routingLayersOnly`, or `-loadBlockage`.
`-routingLayersOnly`	Specifies that LPA is to be run in the Routing Layers Only mode. The Routing Layers Only mode is the fast mode of LPA to flag L1 hotspots in a design at the block level during the Implementation phase. In this mode, LPA runs about 100X faster than the Sign-Off mode and helps you identify and fix most hotspots as routing is completed. **Note**: This parameter cannot be used along with `-signOff`, `-reportOnly`, or `-loadBlockage`.
`-runMode` *string*	Launches LPA in back/foreground.

`-signOff`	Specifies that LPA is to be run in the Sign-Off mode. You run LPA in Sign-Off mode for Litho sign-off, as mandated by foundries. Unlike the Routing Layers Only mode, the Sign-Off mode requires you to provide the LPA configuration file containing the Techfile and other settings. You must run LPA Sign-Off mode before handing off the design to the top-level or tape-out. **Note**: This parameter cannot be used along with `-routingLayersOnly`, `-reportOnly`, or `-loadBlockage`.
`-streamOutOptions` *string*	Enables additional `streamOut` command options.
`-techFile` *string*	Specifies the path and name of the qualified LPA Mx technology file that includes process-specific LPA model, hotspot patterns, and hotspot checking options. The LPA model captures the GDSII-to-silicon flow of the targeted process, enabling LPA to predict the variations and hotspots due to lithography, directly from the design database. Hotspot patterns can also be included in the LPA techfile in replacement for the LPA model to support the DRC+ flow, or along the LPA model to enable fast mode. **Note:** The `-techFile` parameter is mandatory if you use the `-routingLayersOnly` option to run LPA.
`-verify_function` *string*	Specifies the DRC function to be checked.

Examples

Following is an example of the `verifyLitho` command usage:

```
verifyLitho -dir lpa -techFile lpa.tcl -layerMapFile lef_map.file -mapFile gds_map.file -
routingLayersOnly -streamOutOptions "-outputMacros -merge list_of_GDS_files"
```

Related Information

- Integration with LPA and CCP chapter of *User Guide*.
- *Litho Physical Analyzer User Guide*

verifyMetalDensity

```
 verifyMetalDensity
[-help]
[-report filename ]
[-detailed]
[-ignoreLEFDensity]
[-layer layer_name_list ]
[-area { x1 y1 x2 y2 } ]
[-oversize value ]
```

Checks the metal density of each routing layer and of macros against values specified by `setMetalFill`, by LEF file, or against its own internal default values. Reports density information in the log file.

> The `setMetalFill` command is a user-override command that provides constraints at the run time for each session. These constraints have precedence over the LEF/database values. For example, if you specify `setMetalFill -iterationName default`, `verifyMetalDensity` checks against the `default` value.

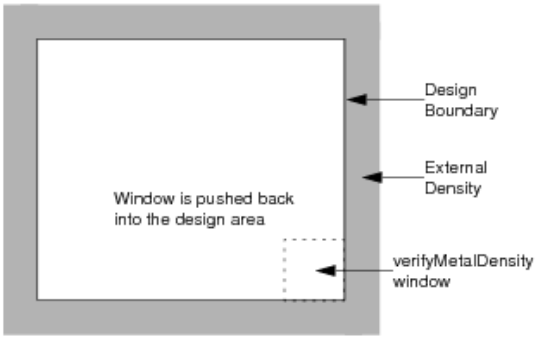
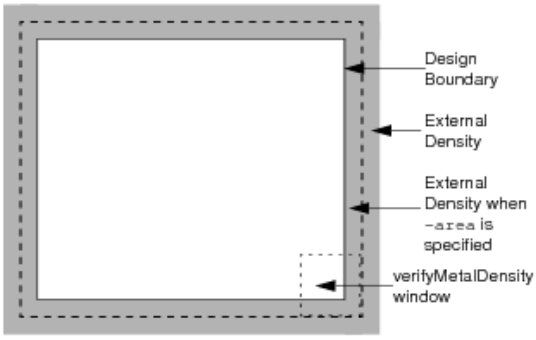
If neither the LEF file nor `setMetalFill` specifies the window size or density, `verifyMetalDensity` uses internal default values.

`verifyMetalDensity` now supports multi-threading to improve Metal Fill runtime performance in multi-thread mode. `verifyMetalDensity` supports the new `setMultiCpuUsage` command use model. For more information, see the following documents:

- Multiple-CPU Processing Commands chapter of the *Innovus Text Command Reference*.
- Accelerating the Design Process by Using Multiple-CPU Processing chapter of *User Guide*.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `verifyMetalDensity` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man verifyMetalDensity`

`-area {x1 y1 x2 y2}`	Specifies the coordinates of the area to verify. For example, `verifyMetalDensity -area {0 0 100 100}`. *Default:* Entire area **Behavior of `verifyMetalDensity -area` with external metal density** When metal fill is added to a block design the external metal density provides a density assumption to `verifyMetalDensity`. The external density is used only for density calculation purposes and no fill shapes are added outside of the design area. When the `-area` parameter is not specified, the external density value is not used. In this case, the `verifyMetalDensity` window is pushed back if it extends outside the design area, as shown in the figure below.  If the `-area` parameter is specified and is larger than the design size then `verifyMetalFill` windows extend outside of the block area and use the external density value (see `setMetalFill` external density value specification) for that area.  If external density is less than the specified preferred density, it is more likely that shapes will be added to the area. If external density is greater than the preferred density it is less likely that shapes will be added (shapes are only added within the design boundary). **Note:** Windows outside the design boundary are ignored.
`-detailed`	Generates a detailed metal density report.

`-ignoreLEFDensity`	Ignores the MACRO `DENSITY` table in the LEF file and instead uses the default macro density calculation method. **Note:** If you do not specify this parameter, `verifyMetalDensity` uses the LEF MACRO density table for density calculations.
`-layer` *layer_name_list*	Specifies the LEF or Innovus layers to verify. If you specify only one layer name, you do not need the braces around the name. For example, `verifyMetalDensity -layer {METAL1 METAL2}`.
`-oversize` *value*	Specifies an offset value for the area to verify. The value is in user units (not in DBU), and can be positive or negative. A positive value adds to the area that is verified. For example, if the design covers the area from `(0,0)` to `(100,100)` and you specify `-oversize 10`, `verifyMetalDensity` checks the area from `(-10,-10)` to `(110,110)`. *Default:* `0`.
`-report` *filename*	Specifies the report file for the metal density violation information. *Default:* *designName*`.metaldensity.rpt`.

Examples

- The following command checks metal density of all layers and generates a report file named `Test7.density.rpt`:

  ```
  verifyMetalDensity -report Test7.density.rpt
  ```

- The following command checks metal density on layer *Metal1* in the area specified and generates a detailed report named `test_chip.density.rpt`:

  ```
  verifyMetalDensity -area {-200 -200 1600.0 1600.0} -layer {Metal1} -report test_chip.density.rpt -detailed
  ```

  In the example above, the layout size is 0 0 1400.0 1400.0. The command considers the density of the area outside of the design as `0`.

Related Information

- Identifying and Viewing Violations chapter of *User Guide*.
- Optimizing Metal Density chapter of *User Guide*.
- LEF Syntax chapter in the *LEF/DEF Language Reference*

verifyPowerVia

```
verifyPowerVia
[-help]
[-append]
[-area {x1 y1 x2 y2}]
[-checkWirePinOverlap]
[-edgeToEdge]
[-error integer]
[-exclude_region_file check_file]
[-fill]
[-hookup_pitch pitch_value]
[-ignore_objects {drc_fill | io_wire}]
[-layerRange {bottomLayer topLayer}]
[-layer_rail layer_name]
[-layer_stripe layer_list]
[-net {netNames} | -selected]
[-nonOrthogonalCheck [-distance {x y}]]
[-pitch pitch_value]
[-report file]
[-search_range {x y}]
[-shielding]
[-stackedVia]
[-stripe_rule value]
[-viaUtil value]
[-what_if_report filename]
[-widthRange string]
```

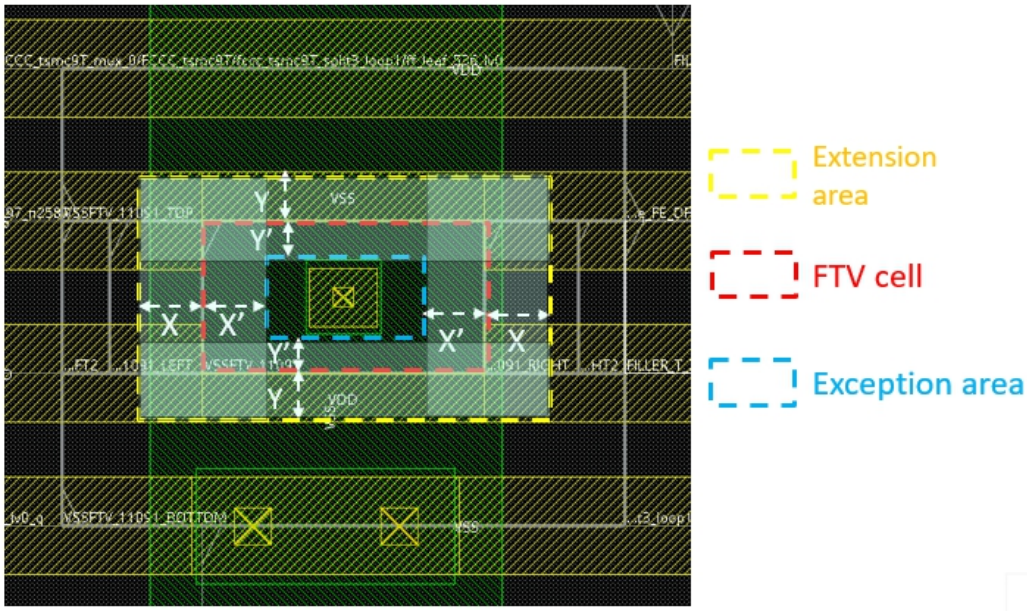
This command has a variety of power-rail overlap checks to look for missing power-grid vias. By default, it checks that orthogonal power-routes on adjacent routing layers have a via between them at every intersection. For example, there should be a via at the point where a Metal3 power stripe overlaps a Metal2 power stripe. By default, horizontal routes get checked only for overlaps with vertical routes. If the power shapes are polygon, the polygon is split into rectangles and the rectangles are used to decide the direction. Support for checking stacked vias between non-adjacent routing layers is optional. Violations are highlighted in the layout window, and a text report is generated with the location of the missing vias.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `verifyPowerVia` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man verifyPowerVia`
`-append`	Specifies that all previous `verifyPowerVia` settings will be kept.
`-area x1 y1 x2 y2`	Specifies the coordinates of the area to be checked.

`-checkWirePinOverlap`	Specifies that wire overlaps of pin shapes should also be checked. By default, only wire overlaps with wires are checked.
`-distance {x y}`	Specifies distances for finding missing via within a specified window for a non-orthogonal check, with `x` and `y` representing the distance in the horizontal and vertical directions, respectively. When the `x` or `y` distance is provided, `verifyPowerVia` checks if the wire is parallel with another wire. If yes, it checks: • The via center to via center spacing between parallel wires against the value *x (y)* • The via center to wire end distance against the value *x/2 (y/2)* If the actual distance is greater than the required distance in either case, it marks a violation. When the `x` or `y` distance is not provided or set to `0`, the via to via distances are not checked. **Note**: This parameter can only be specified with the `-nonOrthogonalCheck` parameter.
`-edgeToEdge`	Checks the non-orthogonal distance from edge to edge within a layer.
`-error integer`	Specifies the maximum number of errors to report.

`-exclude_region_file check_file`	Ignores missing VIA checks within the specified `check_file` regions for a cell extension area. The `check_file` format can be a set of the following values:
	• `cell`: Specifies the cell type.
	• `ext_layer_bottom`: Specifies the extension area bottom layer.
	• `ext_layer_top`: Specifies the extension area top layer.
	• `ext_x`: Specifies the extension length in the X direction.
	• `ext_y`: Specifies the extension length in the X direction.
	• `exc_x`: Specifies the exception length in the X direction.
	• `exc_y`: Specifies the exception length in the Y direction.
	Example 1:
	The following set of `check_file` formats ignores missing VIA check within the yellow rectangle (in the output), but checks missing VIA within the blue rectangle:
	`FTV_CELL, B_M0, B_M1, X, Y, X', Y'`
	`INV_D2, M0, M1, X2, Y2, X2', Y2'`
	Following is the output:

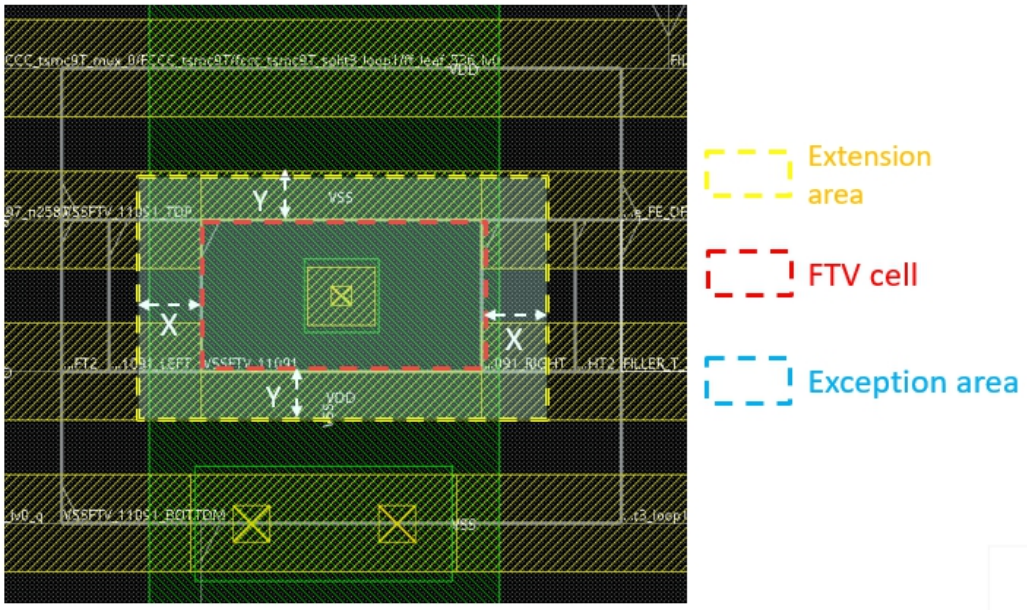


Example 2:

The following set of `check_file` formats ignores missing VIA check within the yellow rectangle (in the output):

```
FTV_CELL, B_M0, B_M1, X, Y
INV_D2, M0, M1, X2, Y2
```

Following is the output:



`-fill`	Specifies that tied-off metal fill should also be checked. Floating fill is still ignored. *Default*: Metal fill will be ignored.
`-hookup_pitch` *pitch_value*	Specifies the distance between two vias, which are on the filler instance and are greater than or equal to the given value.
`-ignore_objects {drc_fill | io_wire}`	Skips checking the power vias of the specified wire type.

`-layerRange` **{***bottomLayer topLayer***}**	Checks for the missing stacked vias between the bottom and top layers. It also checks for the missing vias between all the adjacent intermediate layers, but does not check for the stacked vias between the intermediate layers. While using this parameter, you can specify full layer names like `METAL1`, or abbreviations like `1`. For example, `verifyPowerVia -layerRange {2 5}` checks for missing stacked vias between overlapping `2` and `5`, and also missing adjacent vias for overlapping `2/3`, `3/4`, and `4/5`.
	This parameter checks the missing adjacent vias if `bottomLayer` is specified, but `topLayer` is not, with considering that `topLayer` is just one layer above. To review the missing stacked via or missing adjacent via DRC marker, use the `violationBrowser` command for filtering the DRC marker. For example:
	1. To display only the missing stacked via DRC markers, run:
	`violationBrowser -all -filter {stacked via} -search_desc -filter_mode AND`
	2. To display only the missing via DRC markers between the adjacent layers, run:
	`violationBrowser -all -filter {adjacent via} -search_desc -filter_mode AND`
	See `violationBrowser` for more information on the command usage.

`-layer_rail` *layer_name*	
	Specifies a single power rail layer for missing via (only the via above power rail) check. The check depends on the `-stripe_rule` option. When `-stripe_rule` is specified, the via to via distance should be less than or equal to the specified value. If a distance value is not specified with the `-stripe_rule` option, the check is ignored. See the examples below. The `-layer_rail` option is not controlled by `-layerRange`. You can specify a power rail layer other than that specified with `-layerRange` for checks against the `stripe_rule` and `search_range` rules.

`-layer_stripe` *layer_list*	Specifies the power stripe layers to be checked for a valid connection path from power rail to power stripe. The check depends on the `-search_range` option. If `-search_range` is specified, at least one valid connection path should be found from the power rail to power stripe in that search range. If a search range is not specified with the `-search_range` option, the check is ignored. You can specify the power rail layer with the `-layer_rail` option. By default, only the uppermost power rail layer is checked. See the examples below. The `-layer_stripe` option is not controlled by `-layerRange`. You can specify power stripe layers other than that specified with `-layerRange` for checks against the `search_range` rule.
`-net {`*netNames*`}`	Specifies the names of the power nets that will be checked. A single net or a list of nets enclosed in curly braces({}) or quotes (" ") can be specified. Wildcards (* and ?) are supported. For example, when you specify `verifyPowerVia -net {v*}`, all PG net names having initial letter as `v` are checked. **Note:** Currently, wildcards are supported for checking PG nets only. *Default*: All power grid nets.
`-nonOrthogonalCheck`	Checks the overlap area for non-orthogonal crossing wires (e.g. wires that are both in the same direction) in adjacent as well as non-adjacent layers. Checks for missing vias between two or more parallel wires. The via center-to-center distance between parallel wires should be less than or equal to the specified spacing If this parameter is not specified, horizontal wires get checked only against vertical wires. **Note**: You can use `-distance {x y}` parameter with `-nonOrthogonalCheck` to find missing vias within a specified window. When the `x` or `y` distance is provided, `verifyPowerVia` checks if the wire is parallel with another wire. If yes, it checks the via center-center spacing and via center-wire end distance between parallel wires. If the actual distance is greater than the required distance, it marks a violation.
`-pitch` *pitch_value*	Specifies the distance between two vias, which are not on the filler instance and are greater than or equal to the given value.
`-report file`	Specifies the name of the output file for the report. *Default:* `design_name.powervia.rpt`

`-search_range {x y}`	Specifies a rectangle area with `XY`. There should be one valid connection from power rail to power stripe in the specified area. By default, the valid connection would be searched from the uppermost power rail layer to all the power stripe layers. You can use the `-layer_rail` option to specify the power rail layer, and the `-layer_stripe` option to limit the power stripe layers. With the `-layer_rail` and `-layer_stripe` options, only the specified power rail and power stripes layers are used for valid connection check. The x and y values are floats in units of microns. Refer to the **Examples** section for search range examples.
`-selected`	Checks for the missing vias between the selected PG wires and other intersecting selected/unselected PG wires. When this parameter is used with the `-checkWirePinOverlap` parameter, it checks for the missing vias between the intersection of selected PG wires and pins of the selected instances. This parameter eliminates the requirement of running the command on the entire design, and hence improves the run time.
`-shielding`	Specifies that shielding wires should also be checked. By default, shielding wires are not checked.
`-stackedVia`	Checks for missing vias between all non-adjacent as well as adjacent layers.
`-stripe_rule` *value*	Specifies in microns the distance for power via. Only the power via above the power rail is checked. If the via to via center-to-center spacing is greater than the specified value, a violation marker is created. You can specify the power rail layer with `-layer_rail` for checking. By default, only the power via above the uppermost power rail is checked against the rule. Refer to the **Examples** section for stripe rule examples.
`-viaUtil` *value*	Checks if the cut area in the metal intersection area is sufficient. `viaUtil = Total cut area / Metal intersect area` If the actual ratio is less than specified ratio, `verifyPowerVia` marks it as violation. By default, the metal intersection area is the metal overlap area. If `setAddStripeMode -via_using_exact_crossover_size` is set to `false`, the partial overlapped metal will be extended to full width intersection area.
`-what_if_report` *filename*	
	Writes out `verifyPowerVia` violations data into a ECO file that can be used directly as input for whatIf rail analysis. When you specify this parameter, the missing via information generated can be used as input for what-if virtual wires.
`-widthRange` *string*	

	Specifies the metal width to be considered. `verifyPowerVia` uses the specified width range to filter wires. Only the bottom and top wires that fall in the range are checked. The *`string`* value takes the following format: `-widthRange {`*`minWidth`* `maxWidth [layerName …] [`*`minWidth maxWidth [layerName …]] … }` • If *`minWidth`* is 0, all wires with width less than specified *`maxWidth`* will be checked. • If *`maxWidth`* is 0, all wires with width greater than specified *`minWidth`* will be checked. • If *`layerName`* is specified in the string, `widthRange` is enabled only for the specified layer. • If no *`layerName`* is specified in the string, `widthRange` is enabled for all non-specified layers. Refer to the **Examples** section for width range examples. **Note:** Use braces or quotation marks to enclose the string value for `widthRange`.

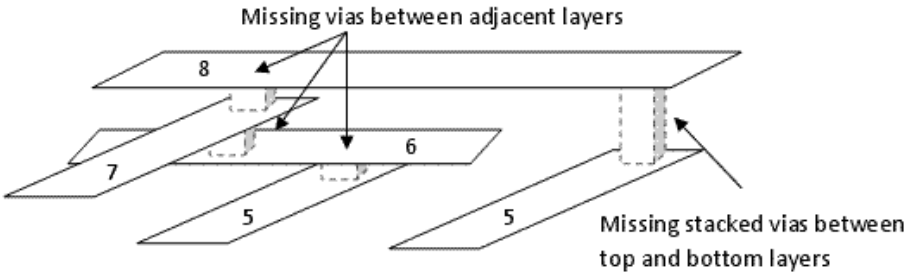
Examples

- The following command specifies that all vias for all powergrid nets will be checked, except for metal fill and shielding wires, and reports it to a file called `power_via.report`:

  ```
  verifyPowerVia -report power_via.report
  ```

- The following command checks for missing stacked vias between `METAL5` and `METAL8` layer intersections on the VSS net. In addition, it checks for missing vias between `METAL5-METAL6`, `METAL6-METAL7` and `METAL7-METAL8` layer intersections. The violations are reported in the `powerVia.rpt` file and highlighted in GUI.

  ```
  verifyPowerVia -layerRange {METAL5 METAL8} -net VSS -report powerVia.rpt
  ```



- The following command checks that the overlapping power routes on adjacent layers have vias that cover at least 70% of the overlap area..

  ```
  verifyPowerVia -viaUtil 0.7
  ```

- The following command checks that the overlapping power routes have vias that cover at least 70% of the overlap area for layers between `METAL3` and `METAL4`.

```
verifyPowerVia -viaUtil 0.7 -layerRange {METAL3 METAL4}
```

- The following command checks for wires overlaps for all pin shapes included, enclosed, or touched by the specified area:

  ```
  verifyPowerVia -area {100 100 1000 1000} -checkWirePinOverlap
  ```

- The following command checks wires with width greater than `0.032` and less than `0.096` for all layers:

  ```
  verifyPowerVia -widthRange {0.032 0.096}
  ```

- The following command checks wires with width in the `0.032` ~ `0.096` range for 1/2 and width `0.038~0.114` for 3/4. For other layers, there is no width control:

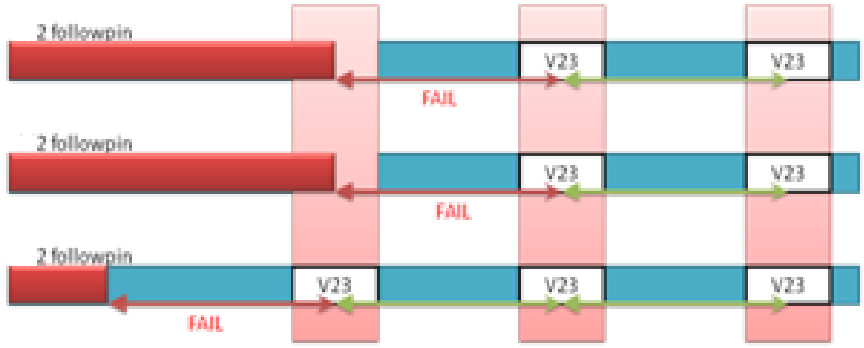
  ```
  verifyPowerVia -widthRange {0.032 0.096 1 2 0.038 0.114 metal3 metal4}
  ```

- The following command checks wires with width in the `0.032` ~ `0.096` range for 1/2. For other layers, it checks wires with width in the `0.038~0.114` range:

  ```
  verifyPowerVia -widthRange {0.032 0.096 1 2 0.038 0.114}
  ```

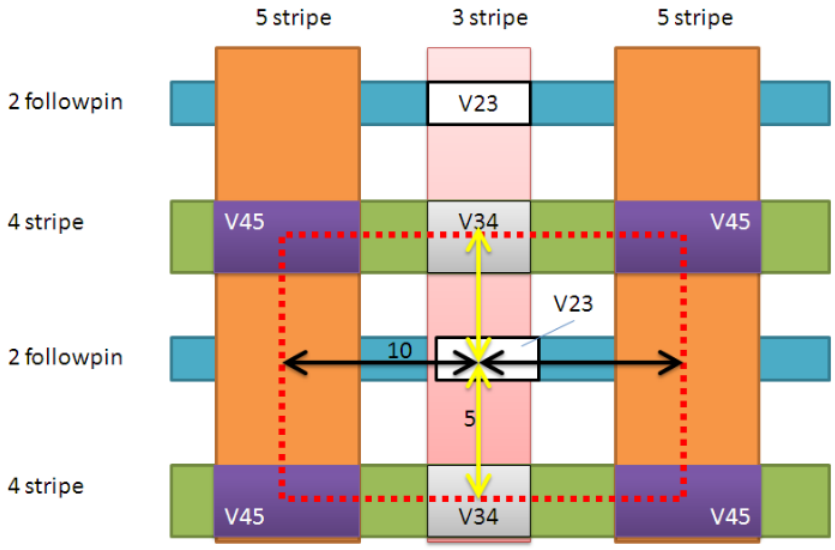
- The following command checks the via between power rail layer `2` and `3`. If the via's center-to-center distance is more than `17u`, the tool reports a violation and places a marker on the power rail wires.

  verifyPowerVia -stripe_rule 17 -layer_rail 2

- The following command checks the valid connection for `2` power rail to stripe layer `4` and `5` with the search rectangle area `10*5`. In the diagram below, there is no violation as the connection is built from `2` to `4`. If the `V34` vias are removed, then there is a violation as the connection cannot be built from `2` to `4`.

  verifyPowerVia -search_range {10 5} -layer_rail 2 -layer_stripe {4 5}



Related Information

- `editPowerVia`
- "Prior to Running Early Rail Analysis" in the Power and Rail Analysis chapter in the *Innovus User Guide*

verifyTieCell

```
verifyTieCell
[-help]
[-noTieCell filename]
[-powerDomain string]
[-report filename]
```

Checks tie cell connections. `verifyTieCell` honors the tie high/low cell list from `setTieHiLoMode -cell` and checks whether or not the tie high/low nets are connected to the tie high/low cells.

Use this command after using `addTieHiLo` to add instances of tie-off cells to the logical hierarchy of the design.

The expected flow is as follows:

1. Specify the tie cell names and other settings with the `setTieHiLoMode` command.
2. Add tie high/low cells with `addTieHiLo`.
3. Run `verifyTieCell` to check whether tie cell connections are correct.

Violations can be viewed in Violation Browser.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `verifyTieCell` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man verifyTieCell`
`-noTieCell` *`filename`*	Specifies a file containing exception list of instance/pin names. Use this parameter to specify the instance/pins that should not have any tie cell connections. The file should list instance and pin names in the following format: *`instance_name/pin_name`*
`-powerDomain` *`string`*	Specifies the power domain name to be checked.
`-report` *`filename`*	Specifies the report file for recording tie cell violations. By default, the report is saved as *`designName`*`.tie.rpt`.

Examples

- The following commands check the tie cell to assess if TIEH and TIEL are inserted for tie nets. If the `dontTouch` instance does not have a tie cell, violation will not be detected:

  ```
  setTieHiLoMode -cell {TIEH TIEL} -honorDontTouch true
  verifyTieCell -report vtc.rpt
  ```

- The following commands will check if `inst` or `pin` in `inst.list` has a tie cell. If it happens, a violation will be detected:

  ```
  setTieHiLoMode -cell {TIEH TIEL}
  verifyTieCell -noTieCell inst.list -report vtc.rpt
  ```

Related Information

- `addTieHiLo`
- `setTieHiLoMode`
- "Verifying Tie Cells" in the Identifying and Viewing Violations chapter of *User Guide*.

verifyWellAntenna

```
verifyWellAntenna
[-help]
[-needToProtect {cell_list}]
[-changeToProtect {cell_list}]
[-changeToNotProtect {cell_list}]
[-report filename [-detailed]]
```

Checks for any CORE rows that have well-process-antenna violations. Well-process-antenna violations are normally caused by rows with decap cells that do not have any protecting cells in the same row. This command marks with a violation marker any such CORE cell. You can then write a Tcl script to put in a well-antenna cell next to the violation marker.

Parameters

-help	Prints a brief description that includes the type and default information for each verifyWellAntenna parameter. For a detailed description of the command and all its parameters, use the man command: man verifyWellAntenna
-changeToNotProtect {*cell_list*}	Specifies the cell names that are to be marked as cells that do not protect rows. By default, the cells with LEF class CORE but without a subclass (e.g. normal BUF and AND2 cells) are assumed to have a well discharge path that protects a row from well-antenna violations. Use -changeToNotProtect option to override the default for any of these cells that do not protect a row. Pattern matching with * is allowed in cell names.
-changeToProtect {*cell_list*}	Specifies the cell names that are to be marked as protection cells. By default, cells with CLASS CORE and a subclass {FEEDTHRU | TIEHIGH | TIELOW | SPACER | ANTENNACELL | WELLTAP} or cells with class ENDCAP (with or without any subclass) do not have a well-discharge path to protect a row. Use the -changeToProtect option to override the default for any of these cells that do protect the row. Pattern matching with * is allowed.
-needToProtect {*cell_list*}	Specifies the cell names that need protection (typically decap cells). This option also indicates that the specified cells do not protect the row they are in. Pattern matching with * is allowed in cell names.
-report *filename*	Specifies the report file name. By default, the report includes information about only the cells that you specified with the options.
-detailed	If you specify -detailed, verifyWellAntenna reports every class CORE and ENDCAP cell with its subclass and protection status to the *filename* report.

Example

Normally, the decap cells are the only cells that need protection. However, if the standard-cell library in your design has one of the ENDCAP cells with a well-antenna protection device built-in, and the filler, well-tap, tie-high and tie-low cells are just class CORE rather than with the correct CORE sub-class, you can use the following command:

```
verifyWellAntenna -needToProtect decap* -changeToProtect wellAntEndCap -changeToNotProtect {filler*
welltap* tie*} -report wellant.rpt
```

Related Information

"Verifying Well-Process-Antenna Violations" in the Identifying and Viewing Violations chapter of *User Guide*.

verifyWellTap

```
verifyWellTap
[-help]
[-area x1 y1 x2 y2]
[-avoidAbutment]
[-check string]
[-cell list_of_well_tap_cells]
[-fromEdge]
[-layer layer_name]
[-powerDomain powerDomainName]
[-report filename]
[-rule distance]
[-siteOffset number_of_sites]
[-wellCutCell list_of_well_cut_cells]
```

Generates violation markers for missing well-tap cells and for well-tap cells that do not meet the rule specified for the distance between well-tap cells. Optionally, generates a well-tap violation report.

Honors the set_well_tap_mode settings for `-cell`, `-rule`, `-well_cut_cell`, `-bottom_tap_cell`, and `-top_tap_cell` options.

Marks violations in the area where well-tap placement violates the rule, covering the shared well of abutting rows. If there is no abutting row, marks the violation where it covers only one-half of a row with a violation.

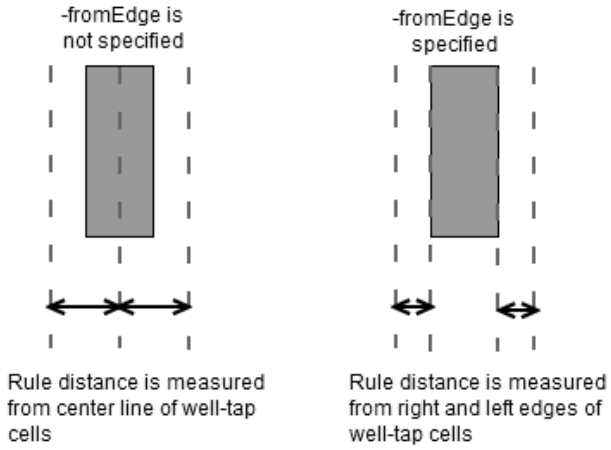
Reports the actual distance between well-tap cells, as well as the rule distance. If there are no well-tap cells in a row and the row that abuts it, or no well-tap cells between the row end and the edge of a block, the software generates a message.

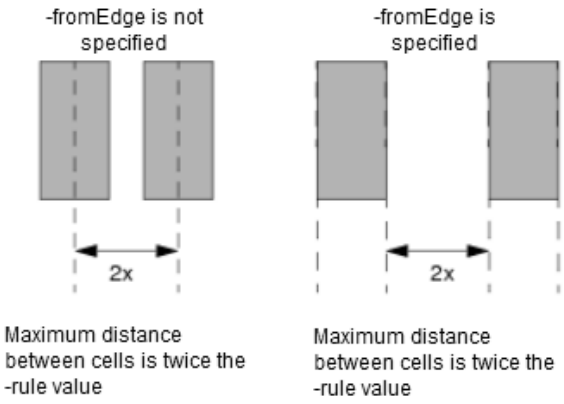
Does not check row areas that are covered by blocks and their halos, or by placement blockages.

The software places well-tap cells with the `addWellTap` command. You can delete well-tap cells with the `deleteFiller` command.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `verifyWellTap` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man verifyWellTap`
`-area x1 y1 x2 y2`	Specifies the area in which well-taps are to be verified.
`-avoidAbutment`	Checks that there is no abutted tap cell in the vertical and horizontal directions.
`-check string`	Checks the latchup rule violation for either Nwell or Pwell. The *`string`* can be either `power` (for Nwell check) or `ground` (for Pwell check).
`-cell list_of_well_tap_cells`	Specifies the well-tap cells. If not specified, uses cells defined as `CLASS CORE WELLTAP` in the Macro section of the LEF file.

`-fromEdge`	Checks the rule distance from the right and left edges of the well tap cells, instead of from the center line. Use this parameter when the design has well-tap cells of different widths. If this parameter is not specified, `verifyWellTap` checks the rule distance from the center line of the well-tap cells. 
`-layer` *`layer_name`*	Specifies the layer name to check the rule.
`-powerDomain` *`powerDomainName`*	Limits checking to the specified power domain. If you do not use this parameter, *`verifyWellTap`* checks the default domain with the designs of N3 or below nodes, and checks the entire design for the other nodes.
`-report` *`filename`*	Specifies the report name. *Default:* *`topCellName`*`.welltap.rpt`

`-rule` *distance*	Specifies the maximum distance between two well-tap cells. The spacing between two well-tap cells should be twice the `-rule` value when measured from center to center or edge to edge. The distance between tap and row end should be same as the `-rule` value. For example, `verifyWellTap -rule 50`. 
`-siteOffset` *number_of_sites*	Checks the offset value in number of sites that vertical taps need to be aware of. *Minimum*: `1` *Maximum*: `INT_MAX`
`-wellCutCell` *list_of_well_cut_cells*	Specifies the list of cells with the well cuts. Use this option to flag latch-up violations due to well cut cells.

Example

The following command displays violations due to well cut cells:

```
verifyWellTap -rule 30 -wellCutCell {EP32_VDDI_DSWITCH_50n_NOBUF EP32_VDDI_DSWITCH_50n_BUF} -cell
{EP8T32_VDDI_GNDS_CDM EP8T32_VDDI_VDDS_CDM}
```

Related Information

- The following sections in the Placing the Design chapter of *User Guide*.
  - Adding Well-Tap Cells
  - Deleting Well-Tap Cells
- "Substrate Biasing" in the Low Power Design chapter of *User Guide*.
- Eco Flows chapter of *User Guide*.

verifyWireGap

```
verifyWireGap
[-help]
[-append]
[-area {x1 y1 x2 y2}]
[-layers layer_names]
[-net power_nets]
[-report filename]
[-wireToBoundary [-chip {v1 v2}] [-core {v1 v2}] [-block {v1 v2}] [-powerdomain {v1 v2}]]
[-wireToShape [-ring value]]
[-wireToWire value]
```

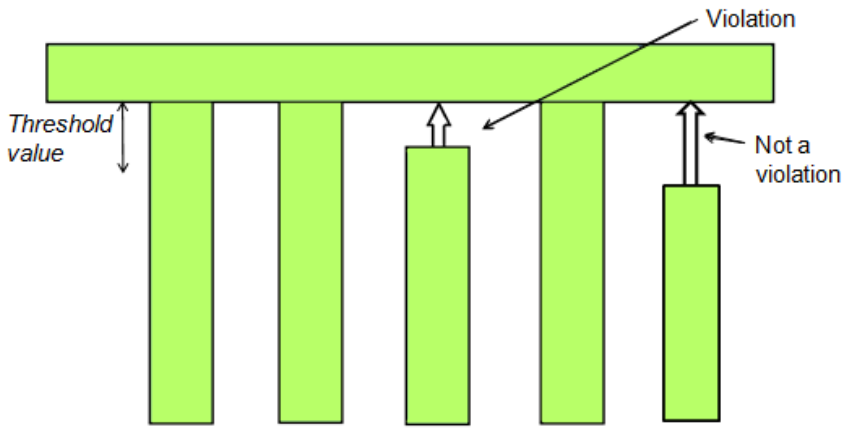
Verifies the continuity of follow pin and stripes. Use this command to identify power strap failures due to broken power wires in the power planning stage. The `verifyWireGap` command gives you the flexibility to check for breaks in a specific area and for specific layers or power nets.

Additionally, you can use this command to ensure that wires do not stop short of the expected distance from the chip, core, or block boundary.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `verifyWireGap` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man verifyWireGap`
`-append`	Specifies that all previous `verifyWireGap` settings will be kept.
`-area {x1 y1 x2 y2}`	Specifies the area to be checked.
`-block {v1 v2}`	Specifies the wire-to-block gap distance. Here: • *v1*: Specifies the threshold value or acceptable distance from the block boundary to the wire. • *v2*: Specifies how far to look from the block boundary to find wires that should be considered for the check. Use this check to ensure that vertical straps cover the horizontal straps underneath them near a block edge. For example, if you specify `verifyWireGap -wireToBounday -block {0 0.3}`, any wire that is at a distance less than 0.3um from the block boundary is reported as a violation. Wires that are at a greater distance than 0.3um are not flagged as violations as such breaks are assumed to be intentional.

`-chip {v1 v2}`	Specifies the wire-to-chip boundary gap distance. Here: • `v1`: Specifies the threshold value or acceptable distance from the chip boundary to the wire. • `v2`: Specifies how far to look from the chip boundary to find wires that should be considered for the check. For example, if you specify `verifyWireGap -wireToBounday -chip {0 0.3}`, any wire that is at a distance less than 0.3um from the chip boundary is reported as a violation. Wires that are at a greater distance than 0.3um are not flagged as violations as such breaks are assumed to be intentional.
`-core {v1 v2}`	Specifies the wire-to-core boundary gap distance. Here: • `v1`: Specifies the threshold value or acceptable distance from the core boundary to the wire. • `v2`: Specifies how far to look from the core boundary to find wires that should be considered for the check. For example, if you specify `verifyWireGap -wireToBounday -core {0 0.3}`, any wire that is at a distance less than 0.3um from the core boundary is reported as a violation. Wires that are at a greater distance than 0.3um are not flagged as violations as such breaks are assumed to be intentional.
`-layers layer_names`	Specifies the metal layers to be considered.
`-net power_nets`	Specifies the name of the power nets that will be checked.
`-powerdomain {v1 v2}`	Specifies the wire-to-power-domain gap distance. Here: • `v1`: Specifies the threshold value or acceptable distance from the power domain to the wire. • `v2`: Specifies how far to look from the power domain to find wires that should be considered for the check.
`-report filename`	Specifies the report file for `verifyWireGap` violations data.
`-ring value`	Specifies the wire-to-ring gap distance.
`-wireToBoundary`	Checks the wire to boundary gap. You must specify one of the following sub-options with `-wireToBoundary`: • `-block` to check the gap between wire and block boundary. • `-chip` to check the gap between wire and chip boundary. • `-core` to check the gap between wire and core boundary. • `-powerdomain` to check the gap between wire and power domain.
`-wireToShape`	Checks the gap between the wire and some specific shapes.

`-wireToWire` *value*	Specifies the wire-to-wire gap distance. Use this check to ensure that wires do not stop short of another wire of the same net or layer. For instance, you can use this check to ensure that wires connect properly to a power ring. If the wire-to-wire gap in the same net is less than the specified threshold value, `verifyWireGap` flags it as a violation. If the wire-to-wire is greater than the threshold value, the break is assumed to be intentional. 

Example

The following command checks wire-to-wire gap and flags wires in the same net that are closer than 0.2um:

```
verifyWireGap -wireToWire 0.2
```

Related Information

The Identifying and Viewing Violations chapter of *User Guide*.

violationBrowser

```
violationBrowser
[-help]
[-aclimit]
[-all]
[-area {x1 y1 x2 y2}]
[-connectivity]
[-density]
[-displayByLayer]
[-geometry]
[-max_error_per_type value]
[-no_display_false]
[-overlap]
[-process_antenna]
[-search_desc]
[-short]
[-filter_query LO_formula | {-filter filterString -filter_mode {AND | OR | NOT}}]
```

Displays a list of violations in the design database. The software generates markers for the violations after you use one or more verification commands.

Geometry or Metal Density Violations

Updates geometry and metal density violation markers incrementally during an Innovus session--that is, it displays the markers generated the first time you run the `verify_drc` or `verifyMetalDensity` command and adds new markers, or deletes markers, on subsequent runs during the same session. If the software finds violations during a subsequent run that were already found previously, the browser display does not change, as there is no incremental update.

The browser can make incremental changes because `verify_drc` and `verifyMetalDensity` can check a small area of the design and update the database. As a result of this behavior, the Innovus software saves the information from the first verification run.

Connectivity, Process Antennas, or AC Limit Violations

Overwrites connectivity, process antenna, and AC limit violation markers if the `verifyConnectivity` or `verifyACLimit` commands are run more than once during an Innovus session. These commands are net-based, not area-based, so the browser does not make incremental updates for them. As a result of this behavior, the Innovus software does not keep the information from the first verification run.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `violationBrowser` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man violationBrowser`
`-aclimit`	Displays AC limit violations.
`-all`	Displays all the violations that are present in the design. This is the default behavior.
`-area {`*`x1 y1 x2 y2`*`}`	Specifies the coordinates of the area whose violations to display. For example, `violationBrowser -area {0 0 100 100}`.
`-connectivity`	Displays connectivity violations.
`-density`	Displays metal density violations.
`-displayByLayer`	Displays the list of violations by the layer instead of route type.
`-filter` *`filterString`*	Specifies a text string in the violation message. Violation Browser searches the message of each violation and displays only the violations whose messages match the search conditions. To specify a list of strings, separate each string with a space. To specify a literal string with space in it, enclose the string in double quotation marks. For example, you can specify strings such as the following: • `METAL5` • `{"(600, 500)"}` • `{METAL3 "(700, 400)"}` • `"Pin of Net"`
`-filter_mode {AND | OR | NOT}`	Specifies the condition for searching multiple strings in the violation browser. You cannot filter complex expressions because the `AND`, `OR`, and `NOT` parameters are mutually exclusive. • The `AND` and `OR` conditions work on single strings or multiple strings. • The `NOT` condition works only on a single string.

`-filter_query` *`LO_formula`*	Specifies a complex logical expression for filtering violations. This option cannot be used along with `-filter` or `-filter_mode`. You can specify the AND, OR, and NOT conditions in the search expression. Use:
	• `&&` between strings in the expression to specify the AND condition.
	• `||` between strings in the expression to specify the OR condition.
	• ! before a string to specify the NOT condition. Note that ! is an one's complement operator. So for example, `!{object(net1) && layer(m2)}` is not allowed.
	The `-filter_query` parameter supports searches in the object, layer, and description areas. The area can be specified using the `object`, `layer`, and `desc` keywords. For example:
	`object(net1) || layer(m2) && desc(xxx)`
	You can also use the wildcards * and ? in the search expression. For example: `!object(clk*) && layer(m?)`
	To specify a literal string with space in it, enclose the string in double quotation marks. For example:
	`!object(clk*) && desc("prl spacing")`

`-geometry`	Displays geometry violations.
`-max_error_per_type` *`value`*	Specifies a limit for the number of violations parsed for each violation type. For example, if you set `violationBrowser -max_error_per_type 500`, the Violation Browser parses and displays only 500 markers per error type. Use this parameter if you are loading a large DRC file with millions of violations to prevent the databse from becoming slow while loading the file and to limit the number of violations you review in the browser. By default, the Violation Browser displays **all** markers.
`-no_display_false`	Prevents the Innovus software from displaying false violations.
`-overlap`	Displays overlaps in the design.
`-process_antenna`	Displays process antenna violations.
`-search_desc`	Searches for text in the violation message and the violation description.
`-short`	Displays short violations.

Examples

- The following command displays all violations whose marker title contains `METAL3`:

```
violationBrowser -all -filter METAL3 -filter_mode OR
```

- The following command displays geometry violations whose marker title contains `METAL3` and `tdsp`:

```
violationBrowser -geometry -filter {METAL3 tdsp} -filter_mode AND
```

- The following command displays geometry violations whose marker title or marker description field contains `Min` and `0.24` and `METAL3`:

```
violationBrowser -geometry -filter {Min 0.24 METAL3} -filter_mode AND -search_desc
```

- The following command displays violations occuring on Metal4 or Metal5 layers but excludes short violations:

```
violationBrowser -filter_query {!Short && (metal4 || metal5)}
```

Related Information

- The following section in the Identifying and Viewing Violations chapter of *User Guide*.
  - Viewing Violations With the Violation Browser
  - Clearing Violations
- "Evaluating Violations" in the Using the NanoRoute Router chapter of *User Guide*.

violationBrowserDeleteByArea

```
violationBrowserDeleteByArea
[-help]
[-all]
[-area {x1 y1 x2 y2}]
```

Deletes markers from a specified area or the entire design area. When the `-area` and `-all` parameters of this command are specified at the same time, `-all` is applied.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `violationBrowserDeleteByArea` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man violationBrowserDeleteByArea`
`-all`	Deletes markers from the entire design area.
`-area {x1 y1 x2 y2}`	Specifies a bounding box for the area from which to delete markers. For example, `violationBrowserDeleteByArea -area {0 0 100 100}`.

Related Information

- Identifying and Viewing Violations chapter of *User Guide*.
- "Evaluating Violations" in the Using the NanoRoute Router chapter of *User Guide*.

violationBrowserReport

```
violationBrowserReport
[-help]
[-all]
[-aclimit]
[-connectivity]
[-density]
[-geometry]
[-max_error_per_type integer]
[-no_display_false]
[-overlap]
[-process_antenna]
[-short]
[-area {x1 y1 x2 y2}]
[-report filename]
[-filter filterString]
[-filter_mode {AND | OR | NOT}]
[-search_desc]
```

Reports violations flagged by the verification commands. The report file and violation browser list the violations in the same order.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `violationBrowserReport` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man violationBrowserReport`
`-aclimit`	Reports AC limit violations.
`-all`	Reports all violations.
`-area {x1 y1 x2 y2}`	Specifies the coordinates of the area for which to report violations. For example, `violationBrowserReport -area {0 0 100 100}`.
`-connectivity`	Reports connectivity violations.
`-density`	Reports metal density violations.

`-filter` *`filterString`*	Specifies a text string in the violation message (in the bottom portion of the Violation Browser). When you specify a text string, the Violation Browser searches the message of each violation and displays only the violations whose messages match the search conditions. To specify a list of strings, separate each string with a space. To specify a literal string with space in it, enclose the string in double quotation marks. For example, you can specify strings such as the following: • `METAL5` • `(600, 500)` • `METAL3 (700, 400)` • `"Pin of Net"`
`-filter_mode {AND | OR | NOT}`	
	Specifies the condition for searching multiple strings in the violation browser. You cannot filter complex expressions because the AND, OR, and NOT parameters are mutually exclusive. • The `AND` and `OR` conditions work on single strings or multiple strings. • The `NOT` condition works only on a single string.
`-geometry`	Reports geometry violations.
`-max_error_per_type` *`integer`*	
	Specifies the error marker number limit for each type of error. Specifies a limit for the number of violations parsed for each violation type. For example, if you set `violationBrowserReport -max_error_per_type 500`, the Violation Browser parses and displays only 500 markers per error type in the report. Use this parameter if you are loading a large DRC file with millions of violations to prevent the database from becoming slow while loading the file and to limit the number of violations you review in the browser. By default, the Violation Browser displays **all** markers in the report.
`-no_display_false`	Prohibits the Innovus software from displaying false violations.
`-overlap`	Reports overlap violations.
`-process_antenna`	Reports process antenna violations.
`-report` *`filename`*	Specifies the report file for the violation information. *Type:* String
`-search_desc`	Searches the description field for each marker in addition to the table entries.
`-short`	Reports short violations.

Examples

- The following command reports all violations whose marker title contains `METAL3`:

```
violationBrowserReport -all -filter METAL3
```

- The following command reports geometry violations whose marker title contains `METAL3` or `tdsp`:

```
violationBrowserReport -geometry -filter {METAL3 tdsp} -filter_mode OR
```

- The following command reports geometry violations whose marker title or marker description field contains `Min` and `0.24` and `METAL3`:

```
violationBrowserReport -geometry -filter {Min 0.24 METAL3} -filter_mode AND -search_desc
```

Related Information

Identifying and Viewing Violations chapter of *User Guide*.

49

What-If Timing Commands

- checkWhatIfTiming
- createWhatIfInternalGeneratedClock
- deleteWhatIfTimingAssertions
- getWhatIfTimingAssertions
- getWhatIfTimingMode
- saveWhatIfConstraints
- saveWhatIfTimingAssertions
- saveWhatIfTimingModel
- setWhatIfClockLatency
- setWhatIfClockPort
- setWhatIfDriveType
- setWhatIfLoadType
- setWhatIfPortParameters
- setWhatIfPortPriority
- setWhatIfTimingMode

The what-if timing analysis commands do not support the Multi-Mode Multi-Corner (MMMC) feature.

checkWhatIfTiming

```
checkWhatIfTiming
blackBoxCellName
[-outfile fileName]
```

Checks that at least one timing arc is defined for every port of a blackbox or blackblob. You use this command to output the status of the timing arcs for all ports to complete the blackbox or blackblob timing arc specifications for the remaining ports.

You can use this command after the module definition of the blackbox or the blackblob is available with the I/O's in the Verilog netlist.

Parameters

blackBoxCellName	Specifies the name of the blackbox or blackblob cell.
-outfile *fileName*	Specifies the name of the file to save the information.

Related Information

- What-If Timing Analysis chapter in the *Innovus User Guide*
  - Performing What-If Timing Analysis
  - Using the What-If Timing Commands

createWhatIfInternalGeneratedClock

```
createWhatIfInternalGeneratedClock
blackBoxCellName
[-help]
-clockName genClkName
-clockPin internalPinName
-masterPin clockInputPort
[-edge_shift edge_shift_list]
[-noCreateInternalPin]
[-invert]
{-multiplyBy integer | -divideBy integer | -edges edge_list}
```

Creates an internal pin on a blackbox or a blackblob, and creates a generated clock on this internal pin. The timing graph is updated accordingly.

The following points apply to the usage of this command:

- The internal pin created with this command will be treated as a clock port. This internal pin can, therefore, be used to create check arcs.
  The latency of the generated clock should include the latency of the master clock. Therefore, the latency of the generated clock should be greater than or equal to the latency of the master clock.

  The delay specified during defining the timing arcs from the internal pins must be greater than or equal to the generated clock latency.
- You can use the generated clock to define timing arcs--these timing arcs are used for timing analysis. Instead of the clock name, the clock pin is referenced while defining the timing arcs. Any timing arc related to the internal pin must include the clock latencies defined on both: the internal pin *and* the master pin.
- You can also use this command to specify that a generated clock should be created on an existing internal pin by specifying the `-noCreateInternalPin` parameter.
- If a cell has multiple instances, a generated clock will be defined for each instance of the cell.
- You can set the latency of the generated clock by using the setWhatIfClockLatency command. The generated clock appears in the block SDC file generated by the saveWhatIfTimingModel command.
- Ensure that the latencies of the generated clock and the master clock are defined before creating any timing arc using the what-if timing analysis commands.

- The internal pin and the generated clock will be treated as what-if assertions and can be deleted with the deleteWhatIfTimingAssertions command.

You can use this command after the module definition of the blackbox or the blackblob is available with the I/O's in the Verilog netlist.

Parameters

blackBoxCellName	Specifies the name of the blackbox or blackblob cell.
-clockName *genClkName*	Specifies the name of the clock.generated on the internal pin.
-clockPin *internalPinName*	Specifies the name of the pin generated inside the blackbox or blackblob.
-divideBy *integer*	Specifies the frequency division factor. The generated clock period is equal to the master clock period multiplied by this parameter. For example, if the value of this parameter is 2, the generated clock period is two times the master clock period. This parameter takes an integer value.
-edge_shift *edge_shift_list*	Specifies the amount of shift for each edge in the specified edge list.
-edges *edge_list*	Specifies the list of edges from source clock to the derived clock.
-invert	Inverts the source clock.
-masterPin *clockInputPort*	Specifies the source from which the derived clock is generated. **Note:** The source must be an existing clock port. and a clock should be available on this port.
-multiplyBy	Specifies the frequency multiplication factor. The generated clock period is equal to the master clock period divided by this parameter. For example, if the value of this parameter is 2, the generated clock period is half the master clock period. This parameter takes an integer value.

`-noCreateInternalPin`	Specifies that a generated clock should be created on an existing internal pin. In this case, the `-clockPin` *`internalPinName`* parameter should specify the existing internal pin.

Related Information

- Using the What-If Timing Commands section in the *Innovus User Guide*.

Examples

The following command created an internal pin named `internalPin1` and a generated clock named `genCLK1` on the blackbox or blackblob named `block3`. The master pin specified is `genCLK1` and the divide-by-frequency specified is `2`.

```
createWhatIfInternalGeneratedClock block3 -clockName genCLK1 -clockPin internalPin1 -masterPin CLK1-divideBy 2
```

deleteWhatIfTimingAssertions

```
deleteWhatIfTimingAssertions
[blackBoxCellName]
[-port portName | -from port -to port | -from port | -to port]
[-assertion_type {comb | setup | hold}]
[-rising | -falling]
```

Removes the timing arc specifications of the specified blackbox or blackblob. When you use this command, the Innovus software restores the timing arcs of the initial library.

You can use this command after the module definition of the blackbox or the blackblob is available with the I/O's in the Verilog netlist.

Note: You can delete the internal pin and the generated clock created by the createWhatIfInternalGeneratedClock command by specifying the `-port` parameter of this command. However, the clock and the pin will not be deleted if you use the `-from` *port*, `-from` *port* `-to` *port*, or the `assertion_type` parameters.

Parameters

blackBoxCellName	Specifies the name of the blackbox or blackblob cell. If you do not specify this parameter, all timing arc specifications are removed.
-port *portName*	Specifies the name of one or more ports.
-falling	Specifies that timing arcs should be deleted for falling clock transitions.
-from *port* -to *port*	Specifies that all arcs starting from the port specified with -from and ending at the port.specified with -to.will be deleted. **Note:** Any constraints specified with the -assertion_type parameter and the -rising parameter will be applicable. **Note:** The deletion of a timing arc using the -from and/or -to parameter does not delete the what-if driver assertion on the output ports.
-from *port*	Specifies that all arcs starting from the port specified with -from will be deleted. **Note:** Any constraints specified with the -assertion_type parameter and the -rising parameter will be applicable.
-rising	Specifies that timing arcs should be deleted for rising clock transitions.
-to *port*	Specifies that all arcs ending at the port.specified with -to.will be deleted. **Note:** Any constraints specified with the -assertion_type parameter and the -rising parameter will be applicable.
-assertion_type {comb | setup | hold}	Specifies whether the timing arcs should be deleted for rising clock transitions or for falling clock transitions. Default: If you do not specify this parameter, timing arcs are deleted for both rising and falling clock transitions.

Related Information

- What-If Timing Analysis chapter in the *Innovus User Guide*
  - Performing What-If Timing Analysis

- Using the What-If Timing Commands

- `saveWhatIfTimingAssertions`
- `getWhatIfTimingAssertions`

getWhatIfTimingAssertions

```
getWhatIfTimingAssertions
blackBoxCellName
-port portName
[-tclList]
```

Displays the what-if timing arc specifications of the specified port in the command window.

You can use this command after the module definition of the blackbox or the blackblob is available with the I/O's in the Verilog netlist.

Parameters

blackBoxCellName	Specifies the name of the blackbox or blackblob cell.
-port *portName*	Specifies the name of the port.
-tclList	Generates the specifications as a Tcl list. The Tcl list is useful for integrating what-if timing with custom Tcl functions and customizing specification generation.

Related Information

- What-If Timing Analysis chapter in the *Innovus User Guide*
- Using the What-If Timing Commands
- saveWhatIfTimingAssertions
- deleteWhatIfTimingAssertions

getWhatIfTimingMode

```
getWhatIfTimingMode
[-quiet]
```

Displays the following information about `setWhatIfTimingMode` parameters in the system log file and in the system console:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the software displays values for all of the `setWhatIfTimingMode` mode parameters.

Parameters

parameter_names	
	Returns information for the specified parameter. You can specify one or more parameters. See setWhatIfTimingMode for parameter descriptions.
`-quiet`	Returns the current settings for the specified parameters in Tcl list format only. If you specify `-quiet` without any parameters, the software returns the current settings of all `setWhatIfTimingMode` parameters in Tcl list format.

Related Information

- `setWhatIfTimingMode`

saveWhatIfConstraints

```
saveWhatIfConstraints
[blackBoxCellName]
[-pt | -dc]
[-dir dirName]
[-filePrefix prefix]
```

Saves the what-if timing constraints in Design Compiler format. You can use this command after the module definition of the blackbox or the blackblob is available with the I/O's in the Verilog netlist.

Parameters

blackBoxCellName	Specifies the name of the blackbox or blackblob cell. *Default:* If you do not specify a name for the blackbox or blackblob cell, constraints are saved for all blackbox or blackblob cells.
-dir *dirName*	Specifies the name of the directory in which the constraints file will be generated. This directory must reside within the current working directory. *Default:* The constraints file is saved in the current working directory.
-filePrefix *prefix*	Specifies the prefix for the constraints file name. This prefix is added to the name of the blackbox or blackblob cell to generate the name of the constraints file. For example, if you specify the prefix `constraintsFile` and the blackbox or blackblob cell name is `Block1`, the name of the file in which the constraints are saved will be `constraintsFileBlock1.pt` or `constraintsFileBlock1.dc`. *Default:* If you do not specify a prefix, the constraints file is saved as *blackBoxCellName*`.pt` or *blackBoxCellName*`.dc`, where *blackBoxCellName* is the name of the blackbox or blackblob cell.
-pt | -dc	Specifies the constraint format. *Default:* Constraints are saved both in PT and DC format.

Related Information

- What-If Timing Analysis chapter in the *Innovus User Guide*

- Performing What-If Timing Analysis
- Using the What-If Timing Commands

saveWhatIfTimingAssertions

```
saveWhatIfTimingAssertions
[blackBoxCellName]
-outfile fileName | -tclList
```

Writes timing arcs to the specified file or Tcl list. The Innovus software saves the final state of the timing arc. The history of all the modifications is not saved. You can source the generated file, regrouping the whole timing arc specifications of the blackbox or blackblob in a new Innovus session.

You can use this command after the module definition of the blackbox or the blackblob is available with the I/O's in the Verilog netlist.

Parameters

blackBoxCellName	Specifies the name of the blackbox or blackblob cell. If the name is not specified, all timing arc specifications are saved.
`-outfile` *fileName*	Specifies the name of the output file in which to save the information.
`-tclList`	Generates the timing arcs as a Tcl list. The Tcl list is useful for integrating what-if timing with custom Tcl functions.

Related Information

- What-If Timing Analysis chapter in the *Innovus User Guide*
  - Performing What-If Timing Analysis
  - Using the What-If Timing Commands

saveWhatIfTimingModel

```
saveWhatIfTimingModel
[-help]
blackBoxCellName
[-outfile fileName]
```

Saves the specification of black boxes or black blobs at the top level, including the timing arc specifications in the dotlib library file format.

You can use this command after the module definition of a black box or black blob is available with the I/O's in the Verilog netlist.

Parameters

`-help`	Prints out the command usage.
`blackBoxCellName`	Specifies the name of the black box or black blob cell.
`-outfile` *`fileName`*	Specifies the name of the file to save the information. *Default:* *`BBoxName`*`.lib`

Related Information

- What-If Timing Analysis chapter in the *Innovus User Guide*
  - Performing What-If Timing Analysis
  - Using the What-If Timing Commands

setWhatIfClockLatency

```
setWhatIfClockLatency
blackBoxCellName
[-clockFrom clockPortName]
[-init value]
[-new value]
[-genConstr value]
```

Specifies the clock insertion delay from a clock input port to register clock input pins within a blackbox or blackblob.

You can use this command after the module definition of the blackbox or the blackblob is available with the I/O's in the Verilog netlist.

Parameters

blackBoxCellName	Specifies the name of the blackbox or blackblob cell.
-clockFrom *clockPortName*	Specifies the name of the clock input port. If you do not specify this parameter, the clock insertion delay is specified for all input and bidirectional clock ports.
-init *value*	Specifies the current clock insertion delay value. This value indicates the clock insertion delay used when the software generates the original timing model. This value is used for timing checks or sequential timing arcs. You must specify this value before modifying a timing check or a sequential timing arc. If you modify at least one timing arc, you cannot directly change the clock latency value. You must first delete the timing arc specifications of the blackboxes or blackblobs using the `deleteWhatIfTimingAssertions` command.
-new *value*	Specifies the new clock latency value. This value modifies the clock insertion delay. As the clock insertion delay is included in the timing arc value, the software adds the difference between the current and new clock latency value to all timing arcs starting from the specified clock port. Therefore all timing arcs starting from the specified clock port, including the timing arcs that were not modified using the what-if timing commands, are modified.
-genConstr *value*	Specifies the latency to consider in the what-if timing constraints generation. By default, the value is automatically set to the current clock insertion delay. If you do not modify the latency value, the generated SDC constraints reflect the timing analysis done at the top level. If you set a latency value other than current clock insertion delay, the timing analysis is not impacted.

Related Information

- What-If Timing Analysis chapter in the *Innovus User Guide*
  - Timing Models Supported for What-If Timing Analysis
  - Using the What-If Timing Commands

setWhatIfClockPort

```
setWhatIfClockPort
blackBoxCellName
-port portName
```

Defines a port as a clock port. You can then use the clock port to create timing checks and sequential timing arcs. Therefore, you can do what-if timing analysis without loading the .lib file.

You can use this command after the module definition of the blackbox or the blackblob is available with the I/O's in the Verilog netlist.

Parameters

blackBoxCellName	Specifies the name of the blackbox or blackblob cell.
-port *portName*	Specifies the name of the clock port.

setWhatIfDriveType

```
setWhatIfDriveType
blackBoxCellName
[-port outputPortName]
[-lib libName]
-cell cellName
[-slew value [-outputCap value]]
[-inputPin libraryInputPin]
[-outputPin libraryOutputPin]
```

Sets the output and bidirectional ports that have the specified type of cell within the blackbox or blackblob as drivers on the interface nets. You can not specify two different drivers for two timing arcs ending at the same output port. The software stores the drive type specification, and uses it to create a timing arc after you specify the delay.

You can use this command after the module definition of the blackbox or the blackblob is available with the I/O's in the Verilog netlist.

Note: The timing sense of the driver is taken into account in the combinatorial what-if timing arc description--while applying the drive type, the timing sense of the combinatorial arc is replaced by the timing sense of the driver's timing arc. For sequential arcs, the timing sense is always set to `non_unate`.

Parameters

blackBoxCellName	Specifies the name of the blackbox or blackblob cell.
-cell *cellName*	Specifies the name of the cell. If there are two cells with the same name, the software uses the first one found. You can specify a complex cell (with multiple inputs and/or outputs) with this parameter.
-lib *libName*	Specifies the name of the timing library.

`-outputCap` *value*	The value for this parameter is set as follows:
	• If the value of the -context parameter of the `setWhatIfTimingMode` is set to `in`, the output capacitance value is equal to the real capacitance. The `-outputCap` parameter of the `setWhatIfDriveType` command is not applicable in this case.
	• If the value of the -context parameter of the setWhatIfTimingMode is set to `out`:
	  ◦ If the `-outputCap` parameter is specified with the `setWhatIfDriveType` command, the output capacitance is set to this value.
	  ◦ If the `-outputCap` parameter is *not* specified with the `setWhatIfDriveType` command, the outputCap value as specified with the -outputCap parameter of the `setWhatIfTimingMode` is used as the output capacitance for the `setWhatIfDriveType` command.
	ⓘ You cannot use this parameter in the intrinsic timing model mode.

`-port` *outputPortName*	Specifies the name of the output port. If you do not specify a port name, the software sets all output and bidirectional ports as drivers on the interface nets. Each time you modify the driver on an output port of the blackbox or blackblob, the software recalculates all timing arcs attached to the output port with the new driver specification.
`-slew` *value*	Specifies the slew at the input of the driver cell. If you specify the slew value, the software creates a one-dimensional timing table depending on the output capacitance. If you do not specify the slew value, the software creates a two-dimensional timing table with input slew and output capacitance values. In this case, the software assumes that there is no slew variation between the input port of the blackbox or blackblob, and the driver input pin. You must use this parameter in the normalized timing model mode. This parameter is not available in the intrinsic timing model mode

`-inputPin` *`LibraryInputPin`*	Specifies the input or inout pin of the cell (specified by the `-cell` parameter) whose source arcs will be used for setting the drive type on the port. *Default*: If this parameter is not specified, the software selects an input or inout pin automatically.
`-outputPin` *`LibraryOutputPin`*	Specifies the output or inout pin of the library whose source arcs will be used for setting the drive type on the port. *Default*: If this parameter is not specified, the software selects an output or inout pin automatically.

Related Information

- `setWhatIfTimingMode`

setWhatIfLoadType

```
setWhatIfLoadType
blackBoxCellName
[-port inputPortName]
[-lib libName]
-cell libraryCellName
[-inputPin libraryInputPin]
[-multiply_by value]
```

Specifies that the load on any input or any inout what-if port should be the same as that of the specified library input or inout pin.

Parameters

blackBoxCellName	Specifies the name of the blackbox or blackblob cell.
-cell library*CellName*	Specifies the name of the cell from which the load value will be picked.
-inputPin *libraryInputPin*	The library cell input or inout pin whose source arcs will be used for setting the load on the port. *Default*: If this parameter is not specified, the software selects an input or inout pin automatically.
-lib *libName*	Specifies the name of the timing library from which the specified cell will be picked. *Default*: The library is selected automatically based on the current context.
-multiply_by *value*	Specifies the load value multiplication factor. The applied load on the blackbox or blackblob is equal to the load of the specified cell pin multiplied by this factor. This parameter takes an integer or floating point value. *Default*: If no value is specified, the multiplication factor is set to 1.
-port *inputPortName*	Specifies the input or inout port of the blackbox or blackblob. *Default*: If no port is specified, the load is applied to all ports.

Examples

The following command sets the load on the `input_Port1` of blackbox `Blackbox1` to the same as that on input pin `Pin_A1` of cell bufx1 of library `library_DelayLogic`.

```
setWhatIfLoadType Blackbox1 -port input_Port1 -lib library_DelayLogic -cell bufx1 -
inputPin Pin_A1
```

Related Information

- `setWhatIfTimingMode`

setWhatIfPortParameters

```
 setWhatIfPortParameters
blackBoxCellName
[-port portName]
[-cap capacitanceValue]
[-cell <cell name>]
[-max_cap maxCapacitanceValue]
[-max_trans maxTransitionValue]
[-max_fanout maxFanoutValue]
[-pin <pin name>]
```

Sets the following values on the what-if cell ports:

- Capacitance
- Maximum capacitance
- Maximum transition
- Maximum fanout

You can use this command after the module definition of the blackbox or the blackblob is available with the I/O's in the Verilog netlist.

Parameters

blackBoxCellName	Specifies the name of the blackbox or blackblob cell.
-cell <cell name>	Specifies the name of library cell from which DRVs are inferred.
-pin <pin name>	Specifies the name of pin from which DRVs are inferred.

-port *portname*	Specifies the name of the clock port on which to set the value(s).
	Default: By default, the specified parameters are applied to all ports in the what-if timing analysis model.
	Note: The value specified for a particular port override the global values. For example, consider the following use case scenario for a cell `b_cell`:
	1. You specify a maximum fanout value for all ports to 20 (without specifying any port with the command):
	`setWhatIfPortParameters b_cell -max_fanout 20`
	• You then run the command again on port `out1` with a maximum fanout value of 5.
	`setWhatIfPortParameters b_cell -max_fanout 5 -port out1`
	In this case, the maximum fanout for port `out1` will be set to 5.

-cap *capacitanceValue*	Specifies the capacitance value in picoFarad. This is the capacitance within the blackbox or the blackblob, on the specified I/O port. The value can be in decimal, for example, `0.75`.
-max_cap *maxCapacitanceValue*	Specifies the maximum capacitance value. This is the maximum value of capacitance permitted on the net going through the I/O port.
-max_fanout *maxfanOutValue*	Specifies the maximum fanout value. The value should be an integer, for example 20.
-max_trans *maxTransitionValue*	Specifies the maximum transition value in nanoseconds. The value can be in decimal, for example, 5.3.

Examples

- The following command sets the maximum capacitance value to `1.0` and the maximum fanout value to `20` for the port `a1_outport1` of the blackbox or blackblob cell named `blockCell2A`.

  ```
  setWhatIfPortParameters blockCell2A -port a1_outport1 -max_fanout 20 -max_cap 1.0
  ```

- The following command is similar to the previous command, except that it sets the values for all ports (as no port is specified with the command)

  ```
  setWhatIfPortParameters blockCell2A -max_fanout 20 -max_cap 1.0
  ```

- The following set of commands set the values as follows
  - the maximum maximum fanout value to `20` for all the ports except port `a1_outport1`
  - the maximum fanout value to 5 and the maximum capacitance value to 1.3 for port `a1_outport1`

    ```
    setWhatIfPortParameters blockCell2A -max_fanout 20

    setWhatIfPortParameters blockCell2A -max_fanout 5 -max_cap 1.3 -port
    a1_outport1
    ```

setWhatIfPortPriority

```
setWhatIfPortPriority
[-driveCell | -portParam]
```

> Command `setWhatIfPortPriority` is obsolete and has been replaced by setWhatIfTimingMode.

On output ports, parameters such as capacitance value, maximum capacitance values, maximum transition value, or the maximum fanout value can come from the driver (`setWhatIfDriveType` command) or they can be set through the `setWhatIfPortParameters` command.

The `setWhatIfPortPriority` command defines which of these values will take precedence in case of a conflict.

You can use this command after the module definition of the blackbox or the blackblob is available with the I/O's in the Verilog netlist.

Note: This feature is available only in the CTE mode.

Parameters

`-driveCell`	Specifies that the values set through the setWhatIfDriveType command will take precedence
`-portParam`	Specifies that the values set through the setWhatIfPortParameters command will take precedence.

The default value is `-portParam`.

setWhatIfTimingMode

```
setWhatIfTimingMode
[-help]
[-reset]
[-context {in | out}]
[-defaultOutputCap value ]
[-model {intrinsic | normalized}]
[-portPriority {cellType | portParam}]
```

Sets the timing model mode for all blackboxes or blackblobs of the design. You can select only one mode for all blackboxes or blackblobs. For more information on the timing models, see " What-If Timing Analysis ," in the *Innovus User Guide*.

If you modify at least one timing arc, you cannot directly change the mode for the blackboxes or blackblobs. You must first delete the timing arc specifications of the blackboxes or blackblobs using the deleteWhatIfTimingAssertions command.

You can use this command after the module definition of the blackbox or the blackblob is available with the I/O's in the Verilog netlist.

Parameters

`-context [{in | out}]`	Specifies whether the timing mode description is `in-context` or `out-context`. The context refers to the chip-level context. • In-context: The timing mode description is created once the chip environment is ready, that is, after placement and routing. In this case, the software takes into consideration the capacitance of the design. • Out-context: The timing model is described outside the chip environment and can be described before floorplanning. You provide the value of the timing arc for a given capacitance. The value of the `-outputCap` parameter of the setWhatIfDriveType command is set based on whether the context is `in` or `out`. ⓘ The `-context` parameter can be used only in the normalized mode. The value of the `-context` parameter is frozen once a timing arc is defined. This is done to avoid having an incorrect timing assertion file later in the flow. *Default*: The default value is `out`.
`-defaultOutputCap [ value ]`	Specifies the value of the default output capacitance on the port. ⓘ The `-defaultOutputCap` parameter can be used only in the normalized mode. The value of the `-defaultOutputcap` parameter is frozen once a timing arc is defined. This is done to avoid having an incorrect timing assertion file later in the flow. The unit is pico farads. *Default*: The default value is 0.

`-help`	Outputs a brief description that includes type and default information for each `setWhatIfTimingMode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man setWhatIfTimingMode`.
`-model {intrinsic | normalized}`	Specifies whether the timing model mode is *intrinsic* or *normalized*. *Default:* The default mode is *normalized*.
`-portPriority {cellType | portParam}`	On output ports, parameters such as capacitance value, maximum capacitance values, maximum transition value, or the maximum fanout value can come from the driver (`setWhatIfDriveType` command) or they can be set through the `setWhatIfPortParameters` command. You can use the -portPriority parameter to specify which of these values take precedence in case of a conflict. • The value `cellType` specifies that the parameter values set through the `setWhatIfDriveType` command take precedence. • The value `portParam` specifies that the parameter values set through the `setWhatIfPortParameters` command take precedence. *The value of the portPriority parameter is frozen once a what-if timing arc has been defined. This avoids inconsistencies between he timing analysis files and the files that you can generate (such as whatif assertions, timing model, or SDC) if you change the port priority subsequently.* *Default*: The default value is `portParam`.
`-reset`	Resets parameters to their default values. The `-reset` parameter *must* be the first parameter specified. If you specify `-reset` by itself, the software resets all `setWhatIfTimingMode` parameters to their default values. If you specify parameters after `-reset`, the software resets only those parameters to their default values. **Note**: Any values that do not already match the default value and are reset will be reported.

Examples

- The following command sets the timing model mode to intrinsic and specifies that in case of a conflict, the parameter values set through the `setWhatIfDriveType` command takes precedence over the parameter values set through the setWhatIfPortParameters command. Because the `-defaultOutputCap` parameter and the -context parameter were not specified, the output capacitance is set to 0, and the context is set to out.

  ```
  setWhatIfTimingMode -model intrinsic -portPriority cellType
  ```

- The following command resets the value of the `-model` and the `-portPriority` parameters. Note that the `-reset` parameter is the first parameter in the command.

  ```
  setWhatIfTimingMode -reset -model -portPriority
  ```

- The following command resets the value of all the `setWhatIfTimingMode` parameters

  ```
  setWhatIfTimingMode -reset
  ```

Related Information

- `getWhatIfTimingMode`

50

Wire Edit Commands

- deleteSecondaryPGNet
- deselectSecondaryPGNet
- editAddFillet
- editAddPoly
- editAddRoute
- editAddTrimMetal
- editAddVia
- editAdjust
- editChangeLayer
- editChangeMask
- editChangeNet
- editChangeRule
- editChangeStatus
- editChangeVia
- editCommitPoly
- editCommitRoute
- editCutWire
- editDelete
- editDeleteFillet
- editDeselect
- editDuplicate
- editFixWideWires
- editMerge
- editMove
- editResize
- editRotate
- editSelect
- editSplit
- editTrim
- end_parallel_edit
- getEditMode
- prepareForEcoRoute
- read_parallel_edit_files
- reportLegalWireWidthForBump
- selectSecondaryPGNet
- setEditMode
- start_parallel_edit

deleteSecondaryPGNet

```
deleteSecondaryPGNet
[-help]
-insts inst+
-net net
```

Deletes Power/Ground (P/G) net wire segments from specified instances. This command provides an easy way to delete the routing from specific instance P/G pins to stripes.

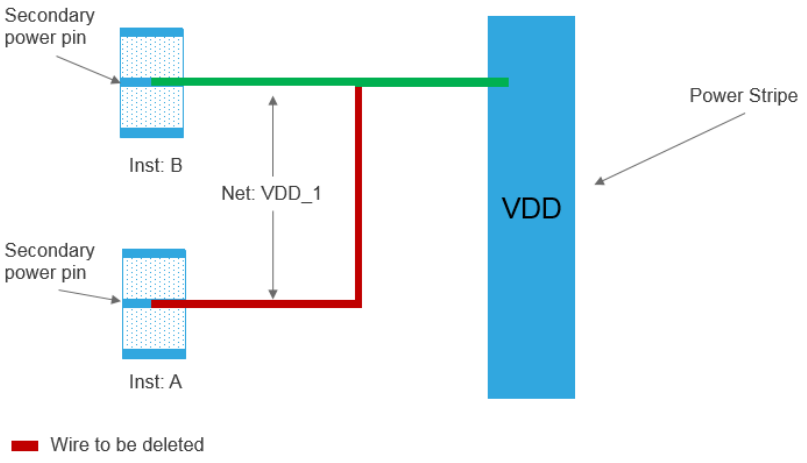
Note: This command does not support dangling secondary P/G nets.

Parameters

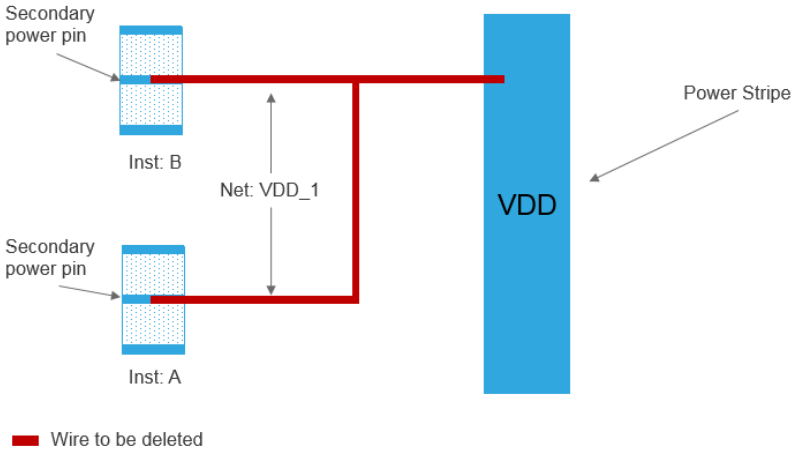
`-help`	Prints the usage of the command. For a detailed description of the command, use the `man` command: `man deleteSecondaryPGNet`
`-insts inst+`	Specifies the list of instances that have a secondary PG pin. Wildcards are supported in instance names. If one or more specified instances does not connect with the specified net, the tool displays a warning and ignores those instances.
`-net net_name`	Specifies the net name. **Note**: *net_name* does not take any wildcard values.

Examples

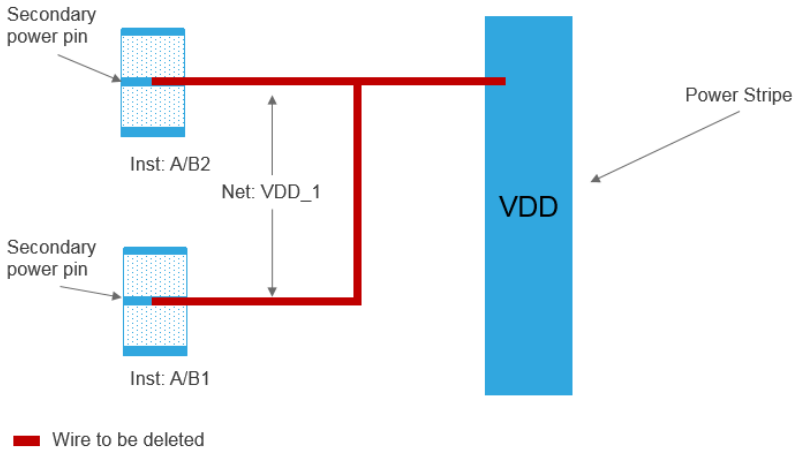
- The following command deletes the routing from the instance `A` P/G pin in net `VDD_1` to the power stripe:
  ```
  deleteSecondaryPGNet -net VDD_1 -insts {A}
  ```



- The following command deletes the routing from the P/G pins in instances `A` and `B` in net `VDD_1` to the power stripe:
  ```
  deleteSecondaryPGNet -net VDD_1 -insts {A B}
  ```



- The following command deletes the routing from the P/G pins in instances `A/B1` and A/B2 in net `VDD_1` to the power stripe:
  `deleteSecondaryPGNet -net VDD_1 -insts {A/*}`



Related Information

- `deselectSecondaryPGNet`
- `selectSecondaryPGNet`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

deselectSecondaryPGNet

```
deselectSecondaryPGNet
[-help]
-insts inst+
-net net
```

Deselects Power/Ground (P/G) net wire segments from specified instances. This command provides an easy way to deselect the routing from specific instance P/G pins to stripes.

Note: This command does not support dangling secondary P/G nets.

Parameters

`-help`	Prints the usage of the command. For a detailed description of the command, use the `man` command: `man deselectSecondaryPGNet`
`-insts inst+`	Specifies the list of instances that have a secondary PG pin. Wildcards are supported in instance names. If one or more specified instances does not connect with the specified net, the tool displays a warning and ignores those instances.
`-net net_name`	Specifies the net name. **Note**: `net_name` does not take any wildcard values.

Examples

- The following command deselects the routing from the instance `A` P/G pin in net `VDD_1` to the power stripe:
  ```
  deselectSecondaryPGNet -net VDD_1 -insts {A}
  ```
- The following command deselects the routing from the P/G pins in instances `A` and `B` in net `VDD_1` to the power stripe:
  ```
  deselectSecondaryPGNet -net VDD_1 -insts {A B}
  ```
- The following command deselects the routing from the P/G pins in instances `A/B1` and A/B2 in net `VDD_1` to the power stripe:
  ```
  deselectSecondaryPGNet -net VDD_1 -insts {A/*}
  ```

Related Information

- `deleteSecondaryPGNet`
- `selectSecondaryPGNet`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

editAddFillet

```
editAddFillet [-help]
```

Adds a tear-drop fillet between a wire and bump connection. A teardrop fillet is used to reinforce the mechanical strength of the connection to the bump. This applies to all bumps in the design. If the connected wire is too short to create the fillet, a marker is created to indicate the failure.

Parameters

`-help`	Prints the usage of the command. For a detailed description of the command, use the `man` command: `man editAddFillet`

Related Information

- `editDeleteFillet`
- `getEditMode`
- `redo`
- `setEditMode`
- `undo`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

editAddPoly

```
editAddPoly
[-help]
point
```

Defines a vertex or corner of a polygon. By issuing the `editAddPoly` command `n` times, you can create a polygon with `n+1` sides. You can then specify the `editCommitPoly` command to end the polygon at the specified coordinates. `editAddPoly` honors the `setEditMode -shape` option. This means if you want to create a polygon of a specific shape, you can first use `setEditMode -shape` to specify the required shape and then use `editAddPoly` and `editCommitPoly` to create the polygon.

The `editAddPoly` command is DRC aware. When you create a polygon shape using this command, the tool marks any DRC violations in the shape during creation, provided that `setEditMode -dcr_on` is set to `1` (default).

Parameters

`-help`	Prints a brief description that includes type and default information for each `editAddPoly` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man editAddPoly`
`point`	Specifies the `x` and `y` coordinates of a vertice or corner of a wire polygon.

Example

- The following commands create three-sided wire polygon that ends at the point specified by `editCommitPoly`.

  ```
  editAddPoly 700 300
  editAddPoly 700 500
  editCommitPoly 200 400
  ```

  The first side of the wire polygon starts at `700, 300` and ends at `700, 500`. The second side starts at `700, 500` and ends at `200, 400`. The third side is formed by joining the starting point of the polygon (`700,300`) and the ending point specified by `editCommitPoly` (`200,400`).

Related Information

- `editCommitPoly`
- `getEditMode`
- `redo`
- `setEditMode`
- `undo`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

editAddRoute

```
editAddRoute
[-help]
point
```

Creates a wire segment that starts or ends at the specified coordinates. Splits diagonal wires whose width is greater than the LEF `MAXWIDTH` parameter for that layer as they are drawn.

If required, the tool allows you to change the net order while adding wire segments. Check the **Examples** sections for details.

Note: Before running any Wire Editor command, all objects and pre-routed wires/vias must be colorized already.

If you do not want to view flight lines between Power and Ground (PG) wires and pins when drawing PG nets, use `setLayerPreference flightLine -isVisible 0` to turn off the display of these flightlines. `editAddRoute` as well as `editCommitRoute` honor this setting.

Parameters

`-help`	Prints a brief description that includes type and default information for each `editAddRoute` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man editAddRoute`
`point`	Specifies the `x` and `y` coordinates for the start point or end point of the wire segment. Note that if the points you specify with each successive `editAddRoute` command are not aligned vertically or horizontally, the tool adjusts the coordinates to align the end point of one segment with the start point of the next segment.

Example

- The following commands create wire segments that start and stop at the specified points. The complete wire ends at the point specified by `editCommitRoute`.

```
editAddRoute 33.392 36.831    #Start of first wire segment
editAddRoute 32.985 32.766    #End of first wire segment, start of second
editAddRoute 18.064 32.115    #End of second wire segment
editAddRoute 20.951 27.073    #End of third wire segment
editCommitRoute 20.992 27.114 #End of wire
```

- Use the following set of commands to add wire segments with different orders of nets:

```
uiSetTool addWire
setEditMode -layer_horizontal M10
setEditMode -nets { a b c d e f}
setEditMode -wire_override_spec {{1 0.062 0.128} {2 0.062 0.1900} {3 0.062 0.128} {4 0.062 0.1900} {5 0.062 0.128} {6 0.062 0.1900} }
editAddRoute {1000 2500}
editAddRoute {1100 2500}

setEditMode -layer_vertical M9
#Change Net Order
setEditMode -nets { a c b d f e}
setEditMode -wire_override_spec {{1 0.082 0.228} {2 0.082 0.2900} {3 0.082 0.228} {4 0.082 0.6680} {5 0.082 0.228} {6 0.082 0.2900} }
editAddRoute {1100 2600}
editCommitRoute {1100 2600}
```

Related Information

- `editCommitRoute`
- `getEditMode`
- `redo`
- `setEditMode`
- `setLayerPreference`
- `uiSetTool`
- `undo`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

editAddTrimMetal

```
editAddTrimMetal
[-help]
[-layer string]
-location {x y}
[-mask integer]
[-snap_to_end]
```

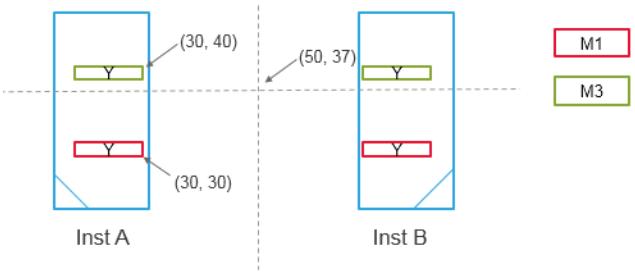
Adds trim metal at the specified location.

Parameters

`-help`	Prints a brief description that includes type and default information for each `editAddTrimMetal` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man editAddTrimMetal`
`-layer string`	Specifies the layer on which the trim metal is to be added. The layer name should be a routing layer defined in the LEF file. Note that you can specify only one valid layer name. That is, a layer name list is not supported. If the `-layer` option is not specified, `editAddTrimMetal` adds the trim metal on the nearest object. Check the **Examples** section below to see how to use this option.
`-location {x y}`	Specifies the coordinates for the center point of the trim metal. The tool may adjust automatically to the closest location that is best for the trim metal.
`-mask integer`	Specifies the mask value for the trim metal. However, if `USEMETALMASK` is already defined for the trim layer in the tech LEF, you cannot define the mask value here.
`-snap_to_end`	Specifies that the trim metal should be snapped to the closest end of the selected pin or wire.

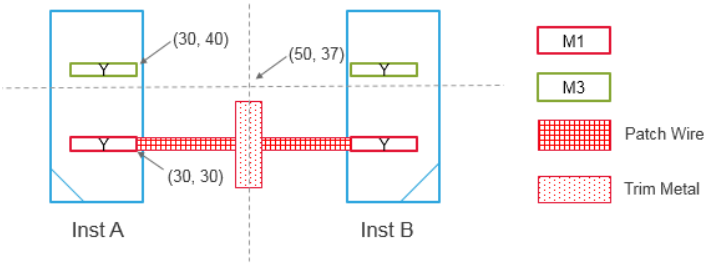
Examples

In the following illustration, the pin Y has two pin shapes, one on `M1` and the other on `M3`. The Pin Y on Inst A is selected.



- If you add trim metal with the `-layer` option (as specified below), the tool will add the trim metal on the pin shape that is on layer `M1`.

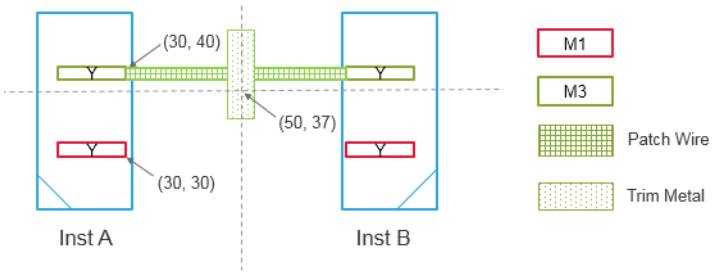
  ```
  editAddTrimMetal -layer M1 -location {50 37}
  ```



- If you add trim metal without the `-layer` option (as shown in the command below), the tool will add the trim metal on the nearest pin, which in this case is

on M3.

```
editAddTrimMetal -location {50 37}
```



Related Information

- editDelete
- getEditMode
- redo
- setEditMode
- undo
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

editAddVia

```
editAddVia
[-help]
point
```

Creates a via instance and places the origin at the specified coordinates. The origin of the instance's via cell may or may not be at its center. Specify the via parameters with the `setEditMode` command.

Parameters

`-help`	Prints a brief description that includes type and default information for each `editAddVia` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man editAddVia`
`point`	Specifies the `x` and `y` coordinates of the via origin.

Example

- The following command adds a via with origin at coordinates `550, 675`:

  ```
  editAddVia 550 675
  ```

Related Information

- `editChangeVia`
- `getEditMode`
- `redo`
- `setEditMode`
- `undo`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

editAdjust

```
editAdjust
[-help]
[-adjust_size]
-refer_obj {top bottom left right}
{[-align_edge {left center right top middle bottom diag45_top diag45_middle diag45_bottom diag135_top diag135_middle diag135_bottom}]
 [-space_mode {horizontal vertical dist_horizontal dist_vertical diag45 diag135 dist_diag45 dist_diag135}
   [-space float_value]]}
```

Aligns wires and pins on the basis of the specified options. The following objects can be aligned using this command:

- Regular, special, and patch wires
- Physical pins
- Macro pins

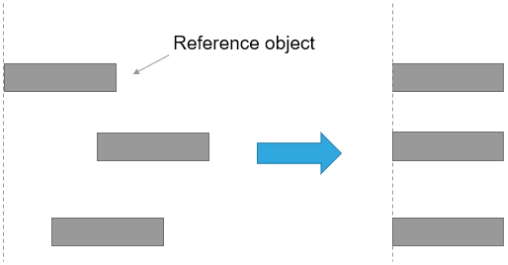
The `editAdjust` command enables you to align objects in one or both of the following modes:

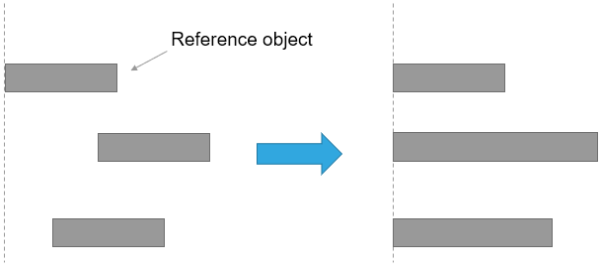
- Align by edge: Use the `-align_edge` parameter to align objects with the reference object by the specified edge.
- Align with spacing: Use the `-space_mode` parameter to adjust the spacing between objects.

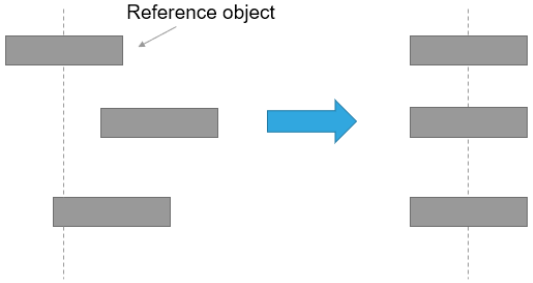
Parameters

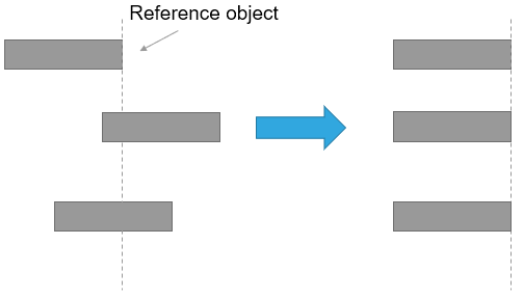
`-help`	Prints a brief description that includes type and default information for each `editAdjust` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man editAdjust`
`-adjust_size`	Resizes the objects being aligned to match the reference object. If `-align_edge` is specified, the specified edge of the objects is stretched or shrunk to align with the edge of the reference object. If `-space_mode` is specified, the objects being aligned are resized to be equal to the reference object.
`-align_edge {left center right top middle bottom diag45_top diag45_middle diag45_bottom diag135_top diag135_middle diag135_bottom}`	
	Specifies the edge by which the objects should be aligned with the reference object.
`-refer_obj {top bottom left right}`	
	Specifies the object of reference for alignment. For example, `-refer_obj left` means that objects should be aligned with reference to the leftmost object. This option is mandatory.
`-space float_value`	
	Specifies the spacing value for aligning objects in the space mode. This option can be specified only if `-space_mode` is specified.
`-space_mode {horizontal vertical dist_horizontal dist_vertical diag45 diag135 dist_diag45 dist_diag135}`	
	Specifies that objects should be aligned in the space mode. In this mode, the objects are aligned such that there is equal spacing between them. You can specify one of the following space mode options: • `horizontal` - Aligns the objects such that the horizontal spacing between them is equal to the value specified with `-space`. • `vertical` - Aligns the objects such that the vertical spacing between them is equal to the value specified with `-space`. • `dist_horizontal` - Distributes the objects horizontally. • `dist_vertical` - Distributes the objects vertically. • `diag45` - Aligns the objects in the 45-degree direction such that the spacing between them is equal to the value specified with `-space`. • `diag135` - Aligns the objects in the 135-degree direction such that the spacing between them is equal to the value specified with `-space`. • `dist_diag45` - Distributes the objects in the 45-degree direction. • `dist_diag135` - Distributes the objects in the 135-degree direction.

Examples

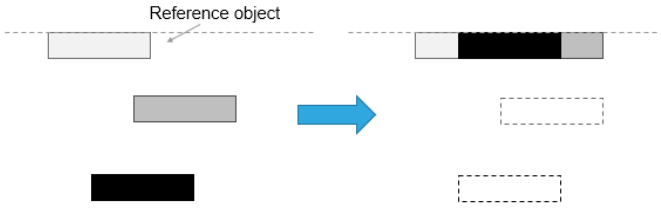
- The following command aligns the left edge of all the wires using the leftmost object as reference:
```
editAdjust -align_edge left -refer_obj left
```


- The following command uses the leftmost wire as reference and resizes the other wires so that their left edges align with the left edge of the reference wire:
```
editAdjust -align_edge left -refer_obj left -adjust_size
```


- The following command aligns the center of all the wires using the leftmost object as reference:
```
editAdjust -align_edge center -refer_obj left
```


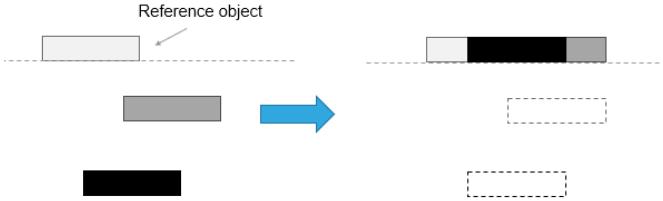
- The following command aligns the right edge of all the wires using the leftmost object as reference:
```
editAdjust -align_edge right -refer_obj left
```


- The following command aligns the top edge of all the wires using the leftmost object as reference:
```
editAdjust -align_edge top -refer_obj left
```



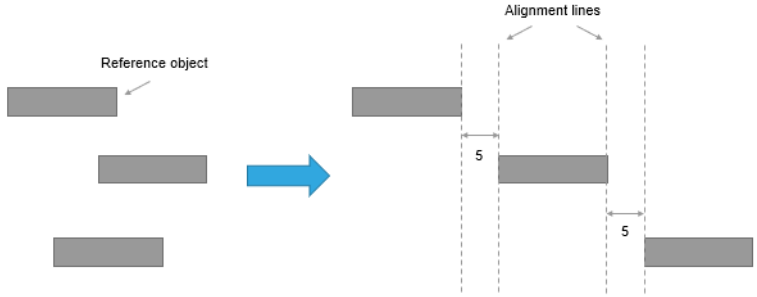
- The following command aligns the bottom edge of all the wires using the leftmost object as reference:

```
editAdjust -align_edge bottom -refer_obj left
```



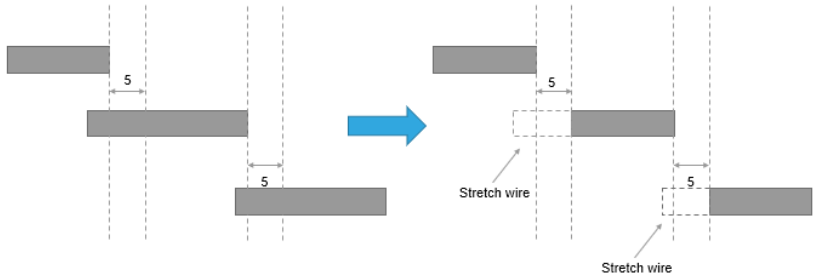
- The following command uses the leftmost object as reference and aligns the wires in the horizontal direction in the space mode with a spacing of 5 units:

```
editAdjust -space_mode horizontal -space 5 -refer_obj left
```



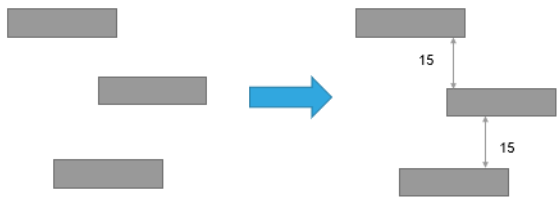
- The following command uses the leftmost object as reference and aligns the wires in the horizontal direction in the space mode with a spacing of 5 units. The command also resizes the objects being aligned to match the reference object:

```
editAdjust -space_mode horizontal -space 5 -refer_obj left -adjust_size
```



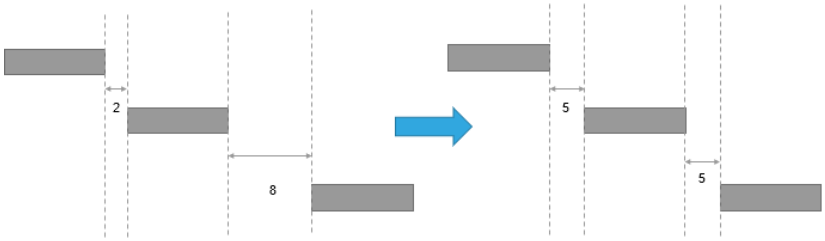
- The following command uses the leftmost object as reference and aligns the wires in the vertical direction in the space mode with a spacing of 15 units:

```
editAdjust -space_mode vertical -space 15 -refer_obj left
```



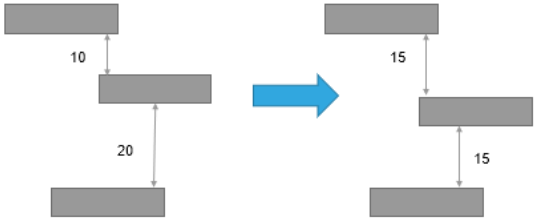
- The following command uses the leftmost object as reference and distributes the wires horizontally:

```
editAdjust -space_mode dist_horizontal -refer_obj left
```



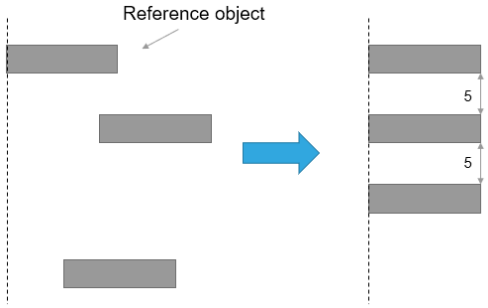
- The following command uses the leftmost object as reference and distributes the wires vertically:

```
editAdjust -space_mode dist_vertical -refer_obj left
```



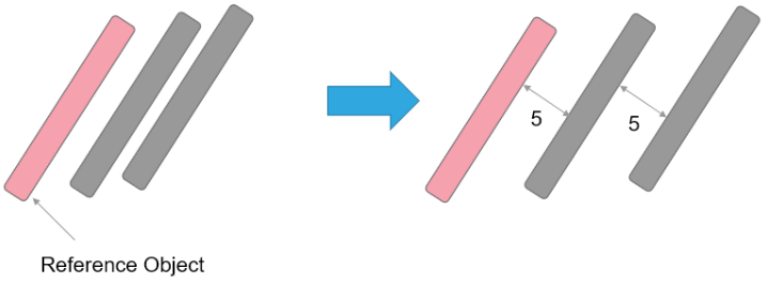
- The following command uses the leftmost object as reference and aligns the left edge of all the wires, keeping a spacing of 5 units between the wires in the vertical direction:

```
editAdjust -align_edge left -space_mode vertical -space 5 -refer_obj left
```



- The following command uses the leftmost object as reference and aligns the wires in the 135-degree direction in the space mode with a spacing of 5 units:

```
editAdjust -space_mode diag135 -space 5.0 -refer_obj left
```



Related Information

- `editAddRoute`
- `editCommitRoute`
- `editCutWire`
- `editDelete`
- `editDuplicate`
- `editMerge`
- `editMove`
- `editResize`
- `editRotate`
- `editSplit`
- `editTrim`
- `getEditMode`
- `redo`
- `setEditMode`
- `undo`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
    - *Wire* section

editChangeLayer

```
editChangeLayer
[-help]
[-layer_horizontal layer]
[-layer_vertical layer]
```

Changes the layer of selected wires and updates vias connecting to the wires that change layers.

Note: Use the `editSelect` command to select the wires.

Parameters

`-help`	Prints a brief description that includes type and default information for each `editChangeLayer` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man editChangeLayer`
`-layer_horizontal layer`	Specifies the new layer for the selected horizontal wires.
`-layer_vertical layer`	Specifies the new layer for the selected vertical wires.

Example

- The following command moves all selected horizontal wires to the *METAL3* layer and all selected vertical wires to the *METAL4* layer:
  ```
  editChangeLayer -layer_horizontal METAL3 -layer_vertical METAL4
  ```

Related Information

- `editChangeMask`
- `editChangeNet`
- `editChangeRule`
- `editChangeStatus`
- `editChangeVia`
- `editDelete`
- `editSelect`
- `editTrim`
- `getEditMode`
- `redo`
- `setEditMode`
- `undo`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

editChangeMask

```
editChangeMask
[-help]
{-to {0 1 2} | [[-top {0 1 2}] [-bottom {0 1 2}] [-cut {0 1 2}]] | -trim_metal {0 1 2}}
```

Changes the mask for the selected wires, vias, physical pins, or trim metal to the specified color.

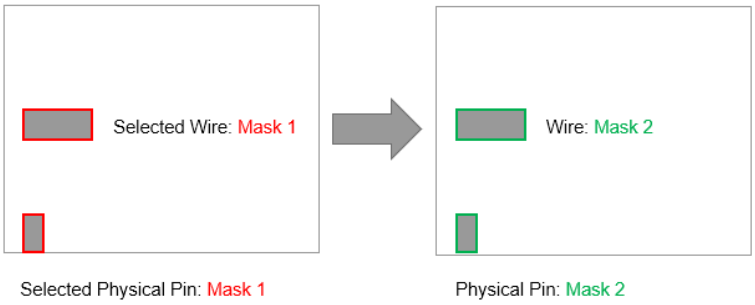
Parameters

`-help`	Prints a brief description that includes type and default information for each `editChangeMask` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man editChangeMask`
`-bottom {0 1 2}`	Changes the mask for the bottom layer of the selected vias to the specified color.
`-cut {0 1 2}`	Changes the mask for the cut layer of the selected vias to the specified color.
`-to {0 1 2}`	Changes the mask for the selected wires and physical pins to the specified color.
`-top {0 1 2}`	Changes the mask for the bottom layer of the selected vias to the specified color.
`-trim_metal {0 1 2}`	Changes the mask for the trim metal of the selected patch wires to the specified color. If the trim metal being edited completely covers another trim metal, the mask of the covered trim metal is also changed simultaneously.

Examples

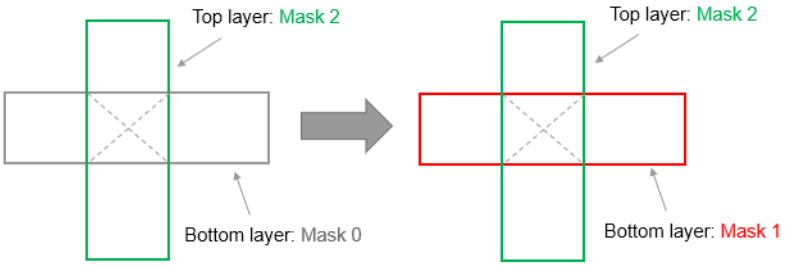
- The following command changes the mask for the selected wire and physical pin to `2`:

  ```
  editChangeMask -to {2}
  ```

- The following command changes the mask for the bottom layer of the selected via to `1`:

  ```
  editChangeMask -bottom {1}
  ```

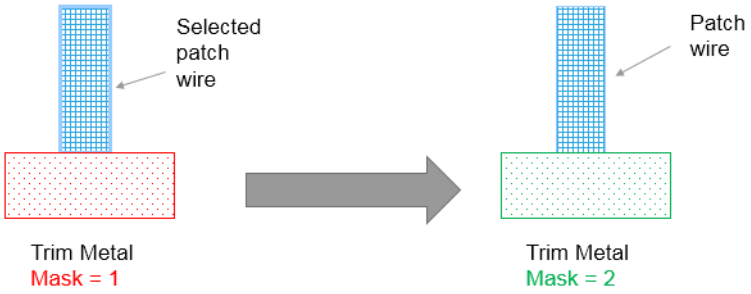
  

- The following command changes the mask for the trim metal of the selected patch wires to `2`:

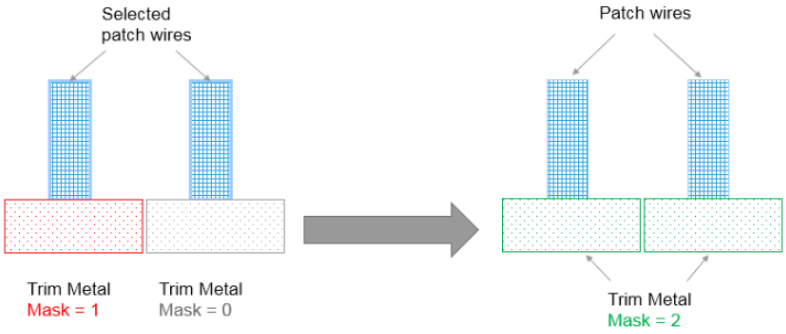
  ```
  editChangeMask -trim_metal {2}
  ```

  Given below are the results of running this command in various situations:

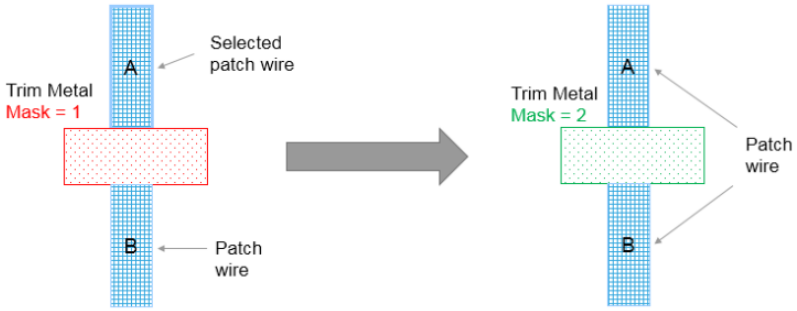
  - If a single patch wire was selected before running the command:



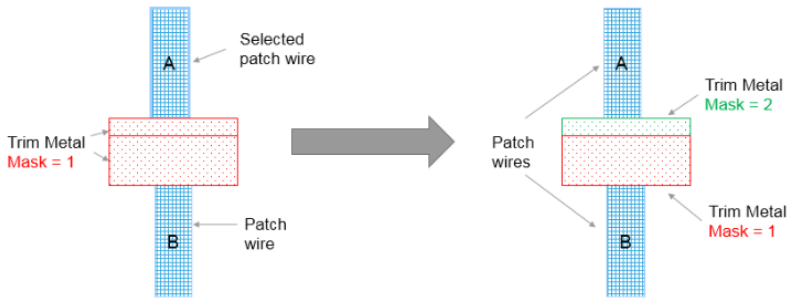
- If multiple patch wires were selected before running the command:



- If the trim metal for the selected patch wire (Patch Wire A) completely overlaps the trim metal for another patch wire that is not selected (Patch Wire B):



- If the trim metal for the selected patch wire (Patch Wire A) partially overlaps the trim metal for another patch wire that is not selected (Patch Wire B):



Related Information

- `editAddTrimMetal`
- `editChangeLayer`
- `editChangeNet`
- `editChangeRule`
- `editChangeStatus`
- `editChangeVia`
- `editDelete`
- `editSelect`
- `getEditMode`
- `redo`
- `setEditMode`
- `undo`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

editChangeNet

```
editChangeNet
[-help]
-to netName
```

Changes the net associated with selected physical pins, wires, and vias.

You can also use `editChangeNet` to change the net name of the patch wire (pWire) generated by NanoRoute.

Parameters

`-help`	Prints a brief description that includes type and default information for each `editChangeNet` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man editChangeNet`
`-to`	Specifies the name of the net to which to assign the selected physical pins, wires and vias.

Example

- The following set of commands selects the physical pins, wires, and vias that belong to net `VDD1` and reassigns them to net `VDD2`:

```
editSelect -net VDD1
editChangeNet -to VDD2
```

Related Information

- `editChangeLayer`
- `editChangeRule`
- `editChangeStatus`
- `editChangeVia`
- `editDelete`
- `editSelect`
- `getEditMode`
- `redo`
- `setEditMode`
- `undo`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

editChangeRule

```
editChangeRule
[-help]
[-from old_rule]
-to new_rule
```

Changes the LEF rule associated with selected wire segments from the *old_rule* to the *new_rule*. You must select the wires for which the rule needs to be changed before running `editChangeRule`. You can select wires either using the GUI or by using the `editSelect` command.

Note: This command does not change the routing rule for the net. If the net is re-routed by NanoRoute, these changes will be lost. To change the routing rule, use `setAttribute -net -non_default_rule`. For more information, see setAttribute.

Parameters

`-help`	Prints a brief description that includes type and default information for each `editChangeRule` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man editChangeRule`
`-from old_rule`	Specifies the qualifying rule. Only wire segments whose associated rule matches *old_rule* are changed. If you do not specify this parameter, the tool will change the rule of all selected wires.
`-to new_rule`	Specifies the destination rule. Wire segments originally associated with *old_rule* are associated with *new_rule* after this command runs. Specify one of the following: • A nondefault rule defined in the LEF `LAYER` (Routing) section • `default` (applies the default routing rule)

Example

- The following command changes the wire segments that are associated with the rule `SP` to the default rule. If the wire width or extension of `SP` is different from its counterpart in the default rule, the wire geometry is changed accordingly.
  ```
  editChangeRule -from SP -to default
  ```

Related Information

- `editChangeLayer`
- `editChangeMask`
- `editChangeNet`
- `editChangeStatus`
- `editChangeVia`
- `editDelete`
- `editSelect`
- `getEditMode`
- `redo`
- `setAttribute`
- `setEditMode`
- `undo`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

editChangeStatus

```
editChangeStatus
[-help]
[shield_net_name]
[-nets <net>+]
-to {ROUTED | FIXED | COVER | SHIELD | NOSHIELD}
```

Changes the DEF status of selected wires or vias to the specified status. Use this command to change the net status of the selected net. If `-nets` is specified with `editChangeStatus`, the tool changes the status of all the objects belonging to the specified nets to the status specified with the `-to` parameter.

Note: Before using this command to change the status of a wire, you must select the wire by either clicking it or using the `editSelect` command.

Parameters

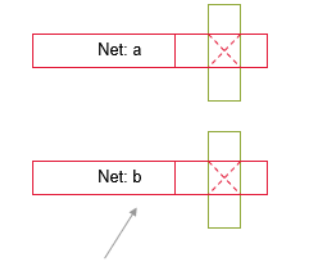
`-help`	Prints a brief description that includes type and default information for each `editChangeStatus` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man editChangeStatus`
shield_net_name	Specifies the name of the net to be shielded by the selected wire. Use this parameter if you want to change the selected wire to a `SHIELD` for the specified net. **Note**: You can use this parameter only if you have specified `-to SHIELD`.
`-nets <net>+`	Specifies the names of the nets for which the object status is to be changed. If `-nets` is specified with `editChangeStatus`, the tool changes the status of all the objects (regular wires, special wires, regular vias, and special vias) belonging to the specified nets to the status specified with the `-to` parameter.
`-to {ROUTED | FIXED | COVER | SHIELD | NOSHIELD}`	Specifies the DEF status of the selected wires or vias after you issue this command. Type the status in all upper-case letters. **Note:** You cannot change a special wire to `NOSHIELD` status or change a signal wire to `SHIELD` status.

Examples

- The following commands select and then freeze the wires and vias on the `METAL1` and `METAL2` layers by changing their status to `COVER`. Innovus does not delete wires or vias with status `COVER`:

```
editSelect -layer {METAL1 METAL2}
editChangeStatus -to COVER
```

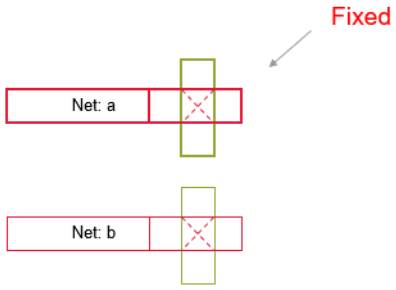
- Consider the nets depicted below:



The wire/via under net b is **selected**

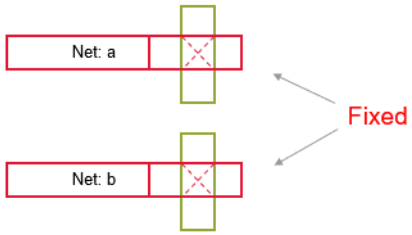
  - The following command changes the status of only the wire and via under net `a` to `FIXED`. The status of the wire and via under net `b` is not changed even though net `b` is selected:

```
editChangeStatus -nets a -to FIXED
```



  - The following command changes the status of the wires and vias under both nets `a` and `b` to `FIXED`:

```
editChangeStatus -nets {a b} -to FIXED
```



Related Information

- `editChangeLayer`
- `editChangeMask`
- `editChangeNet`
- `editChangeRule`
- `editChangeVia`
- `editDelete`
- `editSelect`
- `getEditMode`
- `redo`
- `setEditMode`
- `undo`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
    - *Wire* section

editChangeVia

```
editChangeVia
[-help]
[-net net_name]
[-signal_only {0 | 1}]
{[-from string [-area {x1 y1 x2 y2} | -at {x y} ]] | [-selected]}
{-to string | [-via_columns integer -via_rows integer [-cut_class class_name]]}
```

Replaces the selected vias or the vias listed in `-from` with the specified vias or with vias having the specified cut class and pattern.

While modifying a via with `editChangeVia`, the `setEditMode -drc_on` and `-keep_via` settings are used in combination to decide whether or not a via should be replaced if the new via cell results in DRC violations:

- `setEditMode -drc_on false`
  With this setting, the tool does not check for Design Rule Check (DRC) violations when you modify a via using the `editChangeVia` command.
- `setEditMode -drc_on true -keep_via false`
  With this setting, the tool checks for DRC violations and allows you to replace the via even if the new via is generating violations. The DRC violations are displayed on the layout.
- `setEditMode -drc_on true -keep_via true`
  With this setting, the tool checks for DRC violations and retains the original via if the new one is generating violations.

The behavior defined above applies for all via modifications with `editChangeVia`, irrespective of whether you use the `-to string` option or the `-via_columns integer -via_rows integer` options.

Note that the `setEditMode -drc_on true` option does not check cut spacing, enclosure spacing, or other geometry of the via cell itself.

Parameters

`-help`	Prints a brief description that includes type and default information for each `editChangeVia` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man editChangeVia`
`-area {x1 y1 x2 y2}`	Specifies the coordinates of the area in which vias are to be replaced. If a via indicated by the `-from` parameter is in this area, replaces the via with the via indicated by the `-to` parameter.
`-at {x y}`	Specifies the coordinates for the origin of the via to be replaced. The via at this location is replaced by the via indicated by the `-to` parameter.
`-cut_class class_name`	Specifies the cut class of the replacement vias.
`-from string`	Specifies a via or list of vias to replace. Enclose the via names on the list in curly braces and separate each name with a space. Vias whose masters are not specified in *string* are not changed. **Notes:** • You must specify the `-from` or the `-selected` parameter. If you do not specify at least one of these parameters, the software does not change any vias. • The number of via names on the `-from` list must be equal to the number of via names on the `-to` list. In addition, the corresponding pair of vias in each list must be on the same layer.
`-net net_name`	Specifies the net name on which to change vias.
`-selected`	Replace vias for selected ones only. **Note:** You must specify the `-from` or the `-selected` parameter. If you do not specify at least one of these parameters, the software does not change any vias.
`-signal_only {1 | 0}`	Specifies whether signal vias only or signal and special vias are changed. Specify `1` to change signal via only. Specify `0` to change signal and special vias. *Default:* `0`
`-to string`	Specifies a via or a list of vias to replace the vias indicated by the `-from` parameter. If you specify more than one via, enclose the via list in curly braces and separate each via name with a space. **Note:** The number of via names on the list must be the equal to the number of via names on the list for the `-from` parameter. In addition, the corresponding pair of vias in each list must be on the same layer.
`-via_columns integer`	Specifies the number of columns in the replacement vias.
`-via_rows integer`	Specifies the number of rows in the replacement vias.

Examples

- The following command changes all `VIA_XX` vias of the net `VDD1` located within a specified region to `VIA_YY` vias:
  ```
  editChangeVia -net VDD1 -area {x1 y1 x2 y2} -from VIA_XX -to VIA_YY
  ```
- The following command replaces the selected vias with vias having a cut pattern of 3 by 3:
  ```
  editChangeVia -selected -via_columns 3 -via_rows 3
  ```

Related Information

- `editChangeLayer`
- `editChangeMask`
- `editChangeNet`
- `editChangeRule`
- `editChangeStatus`
- `editDelete`
- `editSelect`
- `getEditMode`
- `redo`
- `setEditMode`
- `undo`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
    - *Wire* section

editCommitPoly

```
editCommitPoly
[-help]
point
```

Ends a newly created polygon at the specified coordinates. Use this command after at least one `editAddPoly` command has been issued.

Parameters

`-help`	Prints a brief description that includes type and default information for each `editCommitPoly` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man editCommitPoly`
point	Specifies the `x` and `y` coordinates for the end point of a newly created wire polygon.

Example

- The following commands create a three-sided wire polygon that ends at the point specified by `editCommitPoly`.

  ```
  editAddPoly 700 300
  editAddPoly 700 500
  editCommitPoly 200 400
  ```

  The first side of the wire polygon starts at `700, 300` and ends at `700, 500`. The second side starts at `700, 500` and ends at `200, 400`. The third side is formed by joining the starting point of the polygon (`700,300`) and the ending point specified by `editCommitPoly` (`200,400`).

Related Information

- `editAddPoly`
- `getEditMode`
- `redo`
- `setEditMode`
- `undo`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

editCommitRoute

```
editCommitRoute
[-help]
point
```

Ends a newly created wire segment at the specified coordinates. Splits diagonal wires whose width is greater than the LEF `MAXWIDTH` parameter for that layer as they are drawn. Use this command after at least one `editAddRoute` command has been issued.

Parameters

`-help`	Prints a brief description that includes type and default information for each `editCommitRoute` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man editCommitRoute`
`point`	Specifies the `x` and `y` coordinates for the end point of a newly created wire segment.

Example

- The following commands create a wire that ends at the point specified by `editCommitRoute`:

```
editAddRoute 33.392 36.831
editAddRoute 32.985 32.766
editAddRoute 18.064 32.115
editAddRoute 20.951 27.073
editCommitRoute 20.992 27.114
```

Related Information

- `editAddRoute`
- `getEditMode`
- `redo`
- `setEditMode`
- `setLayerPreference`
- `uiSetTool`
- `undo`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

editCutWire

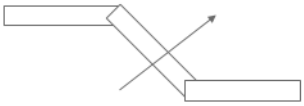
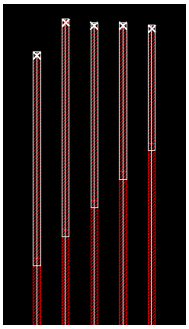
```
editCutWire
[-help]
[-only_visible_wires]
[-selected]
{-lines {x1 y1 x2 y2 ...} | -box {x1 y1 x2 y2} }
```

Cuts wires, bus guides, or rectangles at the specified location. You can choose to cut by line (using the `-lines` parameter) or by box (using the `-box` parameter. `editCutWire` honors `setEditMode -cut_wire_overlap` settings.

Patch wires are also supported.

Parameters

`-help`	Prints a brief description that includes type and default information for each `editCutWire` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man editCutWire`
`-box {x1 y1 x2 y2}`	Specifies the lower-left and upper-right coordinates of the box for cutting wires, bus guides, or rectangles. The `-box` option currently has the following limitations: • The cut-box must be a rectangle (or square). A polygon shape will be ignored. • Only the wires that are perpendicular to the cut-box edge are cut. In addition, the cut-box edge must pass across the wire, bus guide, or rectangle completely. • This option can be used to cut only horizontal and vertical wires. 45-degree wires are not yet supported.

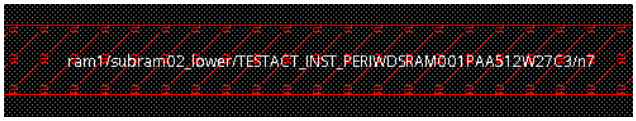
`-lines {x1 y1 x2 y2 ...}`	Specifies the start and end coordinates of one or more line segments for cutting wires, bus guides, or rectangles.
	
	If you specify multiple cut-line segments, they should be parallel to each other. For instance, you cannot specify one horizontal cut-line and one vertical cut-line with the `-lines` parameter. If you do, the tool reports the following error:
	
	`Cut lines are not parallel.`
	
	In addition, a cut-line must cut across the wire, bus guide, or rectangle from one edge to the other. Once cut, the signal wire pieces maintain a one-half wire-width extension. Special wires do not maintain an extension.
	
	When you run `editCutWire`, the tool snaps the cut-line to the manufacturing grid automatically. The change in coordinates is displayed at the prompt as a warning such as follows:
	
	`WARNING: Cut-line is auto-snapped from (966.343500 1227.245500 966.811500 1227.245500) to (966.345000 1227.245000 966.810000 1227.245000).`
	
	Note that the tool records only the cut-line coordinate before snapping in the log file.
	
	Note: The `-lines` option supports editing of 45-degree wire segments.
	
	 
	
	In addition, you can also draw a 45-degree cut-line to cut a group of wires in a stair pattern as shown below:
	
	 
	
	Both regular and special wires can be cut with a 45-degree cut-line. However, patch wires cannot be cut in this way. When you use a 45-degree cut-line, the wires are cut from the touch point of the center line. You can use a single 45-degree cut-line to cut wires in different directions as shown below:

`-only_visible_wires`	Specifies that only the wires on the visible layers are cut. If this parameter is not specified, the tool cuts all the wires overlapping with the cut-line or box.

`-selected`	Specifies that only the selected wires, bus guides, or rectangles should be cut.

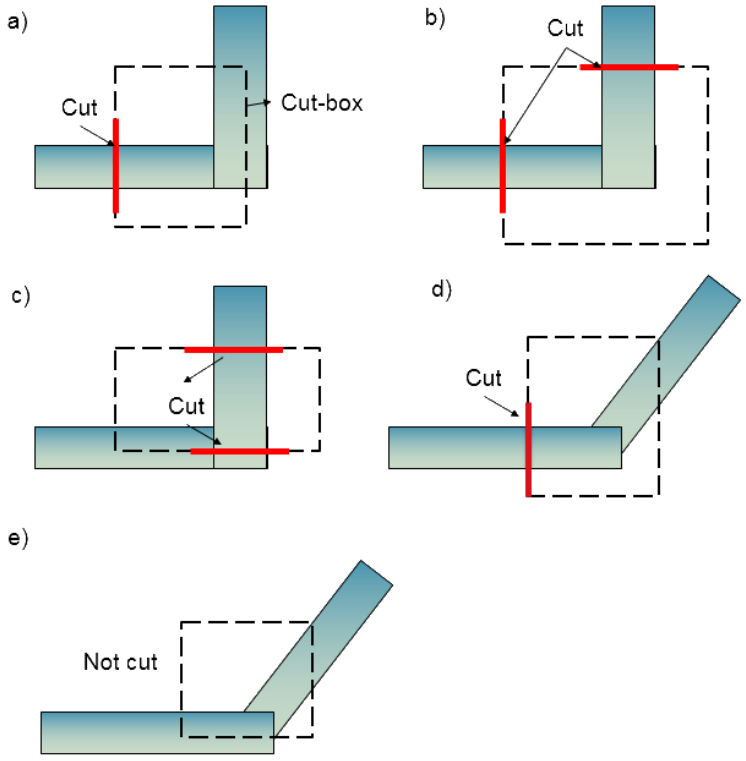
Examples

- The following command cuts the wires through which the specified cut-line passes:
  ```
  editCutWire -lines {200 300 200 310}
  ```
- Consider the wire depicted below:

  The following command cuts the wire, along the specified 45-degree cut-line segments, `{95.30 -641.16 95.39 -641.08}` and `{95.69 -641.17 95.78 -641.08}`:
  ```
  editCutWire -lines {95.30 -641.16 95.39 -641.08 95.69 -641.17 95.78 -641.08}
  ```


- The following diagrams show examples of using a cut-box to cut wires:
  - In a), the horizontal wire is cut. The vertical wire is not cut because the cut-box edges that are perpendicular to the wire do not pass through it completely.
  - In b), both the horizontal and vertical wires are cut.
  - In c), the vertical wire is cut at tje two places where the perpendicular edges of the cut-box pass through it completely. The horizontal wire is not cut.
  - In d), the horizontal wire is cut. The 45-degree wire is not cut because 45-degree wire edits are not yet supported while cutting by box.
  - In e), no cuts are made.



Related Information

- editAddRoute
- editAdjust
- editCommitRoute
- editDelete
- editDuplicate
- editMerge
- editMove
- editResize
- editRotate
- editSplit
- editTrim
- getEditMode
- redo
- setEditMode
- undo
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

editDelete

```
editDelete
[-help]
[-area {x1 y1 x2 y2}]
[-direction {H | V | 45 | 135}]
[-floating_via]
[-layer {list_of_layers}]
[-nets net*]
[-parallel_routing]
[-physical_pin_only]
[-regular_wire_with_drc]
[-selected]
[-shape {RING STRIPE FOLLOWPIN IOWIRE COREWIRE BLOCKWIRE
         PADRING BLOCKRING FILLWIRE FILLWIREOPC DRCFILL None}]
[-shield]
[-shield_only]
[-status {FIXED NOSHIELD ROUTED SHIELD UNKNOWN}
[-subclass sublass_name]
[-trim_metal]
[-type {Regular| Special | Patch}]
[-use {CLOCK POWER SIGNAL}]
[-via_cell cell_name_list]
[-with_edit_flag {0 | 1}]
[-object_type {Wire Via} ]
```

Deletes physical wires, any vias connected to the wires, and violation markers associated with the wires. Use `editDelete -physical_pin_only` to delete physical pins in the design.

By default, when you delete a wire or via with `editDelete`, the tool searches and deletes all related DRC markers. If you do not want the related DRC markers to be cleared when you delete a wire or via, set the `setEditMode -drc_on` parameter to `false` before using `editDelete`. This can improve the run time if have to delete a large number of wires and vias.

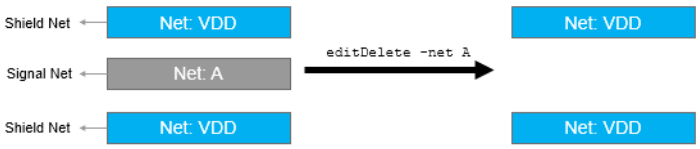
By default, the `editDelete` command retains TSVs and deletes only the real routing layers and vias. You can change this behavior by setting the `setEditMode -delete_tsv` parameter to `true` before using `editDelete`.

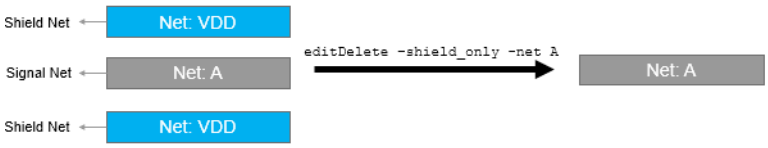
Note: The `editDelete` command deletes only those vias that are connected to wires. To delete vias that are not connected to wires, use the `deleteMetalFill` command.

The `editDelete` command deletes physical wires. The `deleteNet` command deletes logical nets. For more information, see `deleteNet`.

Parameters

`-help`	Prints a brief description that includes type and default information for each `editDelete` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man editDelete`
`-area {x1 y1 x2 y2}`	Specifies the coordinates for the area from which to delete the wires. *Default:* Deletes all wires.
`-direction {H | V | 45 | 135}`	Specifies the orientation of the wires to delete. *Default:* If you do not specify this parameter, deletes wires with either orientations. `H`: Specifies that only horizontal wires are to be deleted. `V`: Specifies that only vertical wires are to be deleted. 45: Specifies that 45-degree wires are to be deleted. 135: Specifies that 135-degree wires are to be deleted.
`-floating_via`	Specifies that only floating vias should be deleted.

`-layer {list_of_layers}`	Specifies the layers from which to delete wires. You must use LEF layer names or a routing layer index value (1 for the first routing layer, 2 for the second routing layer, and so on). This option supports backside and Through Silicon Via (TSV) layers as well. If you are specifying multiple layers, enclose them in braces. *Default:* If you do not specify this parameter, deletes wires from all layers.
`-nets net*`	Specifies the net or list of nets from which to delete wires. You can use wildcard values in this field. For example, if you specify `V*`, deletes wires from nets `VDD` and `VSS`. If you are specifying multiple nets, enclose them in braces. **Note**: By default, the wires and vias shielding the nets specified with `editDelete -nets` are not deleted. If you want to delete the shielding as well, specify `-shield` along with `-nets`. 
`-object_type {Wire Via}`	Specifies whether wires or vias should be deleted. You can specify both as well. If `-object_type` is not specified, `editDelete` deletes both wires and vias by default.
`-parallel_routing`	Specifies that the wires created by NanoRoute parallel routing are to be deleted.
`-physical_pin_only`	Specifies that the physical pins in the design are to be deleted. You can use filter options, such as `-net`, `-layer`, `-area`, and `-selected` along with `-physical_pin_only` to delete specific physical pins.
`-regular_wire_with_drc`	Deletes the regular wires with violation markers. This parameter works only when `-nets` or `-status` are specified as filters. **Note**: `editDelete -selected` parameter deletes only physically selected objects. To delete wires with violations from a specific net, specify the net name with the `editDelete -nets` parameter: `editDelete -nets netName -regular_wire_with_drc`
`-selected`	Deletes selected wires, vias, and physical pins. **Note**: Only physically selected objects are deleted. If `selectNet` is used to select a net, `editDelete -selected` cannot be used to delete that net's wires. This is because `selectNet` selects the net only logically. If you want to delete the entire net, use `editSelect -nets` instead of `selectNet` to ensure the wires of the net are physically selected.
`-shape {{RING STRIPE FOLLOWPIN IOWIRE COREWIRE BLOCKWIRE PADRING BLOCKRING FILLWIRE FILLWIREOPC DRCFILL None}}`	
	Specifies the shapes of the wires to delete. The status must be in all upper-case letters. If you specify multiple shapes, enclose the shapes in braces. For shape definitions, see *Shapes,* in the "DEF Syntax" chapter of the LEF/DEF Language Reference. *Default:* If you do not specify this parameter, deletes wires of all shapes.
`-shield`	Deletes the wires and vias of the nets specified with the `-nets` parameter, along with the wires and vias shielding the specified nets. **Note**: This parameter cannot be used with `-shield_only`.

`-shield_only`	Deletes only the wires and vias shielding the nets specified with the `-nets` parameter. The signal nets being shielded are retained.  **Note**: This parameter cannot be used with `-shield`.
`-status {FIXED NOSHIELD ROUTED SHIELD UNKNOWN}`	
	Specifies the status of wires to delete. The status must be in uppercase letters. If you specify more than one status, enclose the status list in braces. **Note:** If you select `-type Special -status NOSHIELD` or `-type Regular -status SHIELD` the software generates an error message.
`-subclass` *`sublass_name`*	Specifies the subclass name of the wires to be deleted. Use this parameter if you want to delete rings, stripes, or other wires belonging to a specific subclass. **Note:** You can use wildcards in *`subclass_name`*. Only special wire is supported by this option.
`-trim_metal`	Deletes the trim metal attached to all patch wires, without deleting the patch wires. If `-selected` is also specified, deletes only the trim metal attached to the selected patch wire without deleting the patch wire.
`-type {Regular| Special | Patch}`	
	Specifies whether to delete regular wires, special wires, or patch wires. **Note:** If you specify `Regular`, omit the `-shape` parameter or specify `-shape None`.
`-use {CLOCK POWER SIGNAL}`	
	Indicates that wires will be deleted on the basis of the specified use: `CLOCK`: Specifies that only wires for clock nets will be deleted. `POWER`: Specifies that only wires for power nets will be deleted. `SIGNAL`: Specifies that only wires for general signal nets will be deleted.
`-via_cell` *`cell_name_list`*	
	Specifies the via cells of the vias to be deleted. You can use wildcards while specifying the via cell names. If you are specifying multiple cell names, enclose them in braces.
`-with_edit_flag {0 | 1}`	Deletes all modified wires, that is wires with value `1` for the `isEdit` property on the net. **Notes** • As the `isEdit` property is added on the whole net, all the wires on the net with `isEdit` value set to `1` will be deleted. • The `isEdit` property is used to flag nets with modified wires and vias. This is useful when evaluating how many objects need to be modified for fixing DRC violations and then deciding whether to fix the violations manually or through ECO routing. The `isEdit` property is controlled through the `-create_is_edit_flag` parameter of `setEditMode`. By default, this parameter is set to `0`. If you set the value of this parameter to `1`, the `isEdit` property is set to `1` for nets with modified wires or vias.

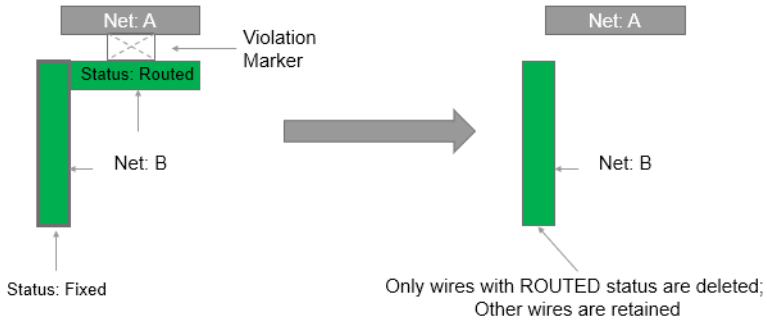
Examples

- The following command removes wires from the *METAL1* and *METAL2* layers:
  ```
  editDelete -layer {METAL1 METAL2}
  ```
- The following command removes horizontal fillwires from the *METAL1* layer:
  ```
  editDelete -layer METAL1 -direction H -shape FILLWIRE
  ```
- The following command removes wires from the specified area:
  ```
  editDelete -area 100 100 200 200
  ```
- The following command removes the selected wires and vias from the specified area:

```
editDelete -area 100 100 200 200 -selected
```

- The following command only deletes the special wires that belong to the net `carry4` leaving the vias untouched:
  ```
  editDelete -type Special -nets carry4 -object_type {Wire}
  ```
- The following command deletes all physical pins:
  ```
  editDelete -physical_pin_only
  ```
- The following command deletes all physical pins from net VSS:
  ```
  editDelete -nets VSS -physical_pin_only
  ```
- The following command deletes all physical pins from nets VSS and VDD:
  ```
  editDelete -nets {VDD VSS} -physical_pin_only
  ```
- The following command deletes all physical pins from layer METAL2:
  ```
  editdelete -layer METAL2 -physical_pin_only
  ```
- The following command deletes all physical pins from the specified area:
  ```
  editDelete -area {26 0 31 4} -physical_pin_only
  ```
- The following set of commands deletes the selected physical pin:
  ```
  select_obj <ptr of layerShape> #Selects the pointer of the layer shape
                                 #belonging to a physical pin
  editDelete -selected
  ```
- The following command deletes the regular wires with violation markers from only the specified net:
  ```
  editDelete -nets B -regular_wire_with_drc
  ```

- The following command deletes only routed regular wires with violation markers from the specified net:
  ```
  editDelete -nets B -regular_wire_with_drc -status ROUTED
  ```



Related Information

- `editAddRoute`
- `editAdjust`
- `editCommitRoute`
- `editCutWire`
- `editDuplicate`
- `editMerge`
- `editMove`
- `editResize`
- `editRotate`
- `editSelect`
- `editSplit`
- `editTrim`
- `getEditMode`
- `redo`
- `setEditMode`
- `undo`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

editDeleteFillet

```
editDeleteFillet [-help]
```

Deletes the tear-drop fillet that was added between a wire and bump connection with the `editAddFillet` command.

Parameters

`-help`	Prints the usage of the command. For a detailed description of the command, use the `man` command: `man editDeleteFillet`

Related Information

- `editAddFillet`
- `getEditMode`
- `redo`
- `setEditMode`
- undo
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

editDeselect

```
editDeselect
[-help]
[-area {x1 y1 x2 y2}]
[-direction {H | V | 45 | 135}]
[-floating_via]
[-layer list_of_layers]
[-nets net*]
[-physical_pin_only]
[-regular_wire_with_drc]
[-shape {RING STRIPE FOLLOWPIN IOWIRE COREWIRE BLOCKWIRE PADRING
         BLOCKRING FILLWIRE FILLWIREOPC DRCFILL None}]
[-shield]
[-shield_only]
[-status {COVER FIXED NOSHIELD ROUTED SHIELD UNKNOWN}]
[-subclass sublass_name]
[-type {Regular Special Patch}]
[-use {CLOCK POWER SIGNAL}]
[-via_cell cell_name_list]
[-with_edit_flag {0 | 1}]
[-object_type {Wire Via} ]
[-from_pin pin_name -to_pin pin_name]
```

Deselects wires and vias based on specified parameters. Use this command when nets are selected.

You can use this command to deselect nets by property. For example, to delete all regular nets except clock nets, select all regular nets with `editSelect`, deselect the clock nets with this command, then delete the selected nets with `editDelete`.

Parameters

`-help`	Prints a brief description that includes type and default information for each `editDeselect` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man editDeselect`
`-area {x1 y1 x2 y2}`	Specifies the coordinates for the area from which to deselect wires and vias.
`-direction {H | V | 45 | 135}`	Specifies the orientation of the wires to deselect. *Default:* If you do not specify this parameter, deselects wires in any orientation. • `H`: Specifies that horizontal wires are to be deselected. • `V`: Specifies that vertical wires are to be deselected. • 45: Specifies that 45-degree wires are to be deselected. • 135: Specifies that 135-degree wires are to be deselected.
`-floating_via`	Specifies that only floating vias are to be deselected. A via, or a stack of vias, is floating, if it is not connected to another object, wire, pin, or via, on either its top or bottom metal layer or on both these layers.
`-from_pin pin_name`	Specifies the name of the starting pin of the wire to be deselected. The starting pin you specify should be connected to the same net as the ending pin specified with `-to_pin`. If they connect to different nets, the tool reports an error. **Note**: Deselection of wires between specified pins is supported only for signal and clock nets.
`-layer list_of_layers`	Specifies the layers from which to deselect wires and vias. This option supports backside and Through Silicon Via (TSV) layers as well. If you are specifying multiple layers, enclose the names in braces. *Default:* If you do not specify this parameter, deselects wires and vias from all layers.

`-nets net*`	Specifies the net names from which to deselect wires and vias. **Note:** You can use wildcard values in this field. For example, if you specify `V*`, deselects wires and vias from nets `VDD` and `VSS`.
`-object_type {Wire Via}`	Specifies whether wires or vias or both should be deselected. If the parameter is not specified, only wires are deselected. *Default*: `Wire`
`-physical_pin_only`	Specifies that physical pins are to be deselected. By default, `editDeselect` does not deselect physical pins. You can use the `-physical_pin_only` parameter along with `-area`, `-layer`, or `-nets` to deselect the physical pins in the specified area, layer, or net, respectively.
`-regular_wire_with_drc`	Deselects regular wires with violation markers.
`-shape {RING STRIPE FOLLOWPIN IOWIRE COREWIRE BLOCKWIRE PADRING BLOCKRING FILLWIRE FILLWIREOPC DRCFILL None}`	
	Specifies the shapes of the wires and vias to be deselected. Use all upper-case letters for the shapes. If you specify multiple shapes, enclose the list of shapes in braces. For shape definitions, see *Shapes,* in the "DEF Syntax" chapter of the LEF/DEF Language Reference. *Default:* If you do not specify this parameter, deselects wires and vias of all shapes.
`-shield`	Deselects the wires and vias of the nets specified with the `-nets` parameter, along with the wires and vias shielding the specified nets. **Note**: This parameter cannot be used with `-shield_only`.
`-shield_only`	Deselects only the shield wires associated with the nets specified using the `-nets` parameter. The signal nets being shielded remain selected. **Note**: This parameter cannot be used with `-shield`.
`-status {COVER FIXED NOSHIELD ROUTED SHIELD UNKNOWN}`	
	Specifies the status of the wires and vias to be deselected. Use all upper-case letters for the status. If you specify more than one status, enclose the status list in curly braces. **Note:** If you select `-type Special -status NOSHIELD` or `-type Signal -status SHIELD` the software generates an error message.
`-subclass sublass_name`	Specifies the subclass name of the wires and vias to be deselected. Use this parameter if you want to deselect wires or vias belonging to a specific subclass. Here, *`subclass_name`* is a user-defined attribute that has been previously defined using the `-uda` parameter of the `addSplitPowerVia`, `addRing`, `addStripe`, or `sroute` command. For wires, use this parameter if you want to deselect or delete rings, stripes, or other wires belonging to a specific subclass. Only special wire is supported by this option. **Note:** You can use wildcards in *`subclass_name`*.
`-to_pin pin_name`	Specifies the name of the ending pin of the wire to be deselected. The ending pin you specify should be connected to the same net as the starting pin specified with `-from_pin`. If they connect to different nets, the tool reports an error. In addition, the from and to pins you specify should be connected to the same wire. If there there are more than one wire or via between the specified starting and ending pins, the command is not able to deselect the wire path and returns an error. The `-from_pin` and `-to_pin` parameters are compatible with only the `-direction`, `-layer`, `-object_type`, and `-status` options of the `editDeselect` command. If you use `-from_pin` and `-to_pin` with any other `editDeselect` parameters, the tool returns an error. **Note**: Deselection of wires between specified pins is supported only for signal and clock nets.
`-type {Regular Special Patch}`	Specifies whether to deselect regular wires and vias, special wires and vias, or patch wires. **Note:** If you specify `Regular`, you must omit the `-shape` parameter or specify `-shape None`.
`-use {CLOCK POWER SIGNAL}`	Specifies whether to deselect only CLOCK wires/vias, POWER wires, or SIGNAL wires.
`-via_cell cell_name_list`	Specifies the via cells of the vias to be deselected. You can use wildcards while specifying via cell names. If you are specifying multiple cell names, enclose them in braces.

`-with_edit_flag {0 | 1}`	Deselects all modified wires and vias, that is wires and vias with value `1` for the `isEdit` property on the net. **Notes** • As the `isEdit` property is added on the whole net, all the wires and vias on the net with `isEdit` value set to `1` will be deselected. • The `isEdit` property is used to flag nets with modified wires and vias. This is useful when evaluating how many objects need to be modified for fixing DRC violations and then deciding whether to fix the violations manually or through ECO routing. The `isEdit` property is controlled through the `-create_is_edit_flag` parameter of `setEditMode`. By default, this parameter is set to `0`. If you set the value of this parameter to `1`, the `isEdit` property is set to `1` for nets with modified wires or vias.

Examples

- The following command deselects all horizontal fillwires on the *METAL1* layer:
  ```
  editDeselect -layers METAL1 -direction H -shape FILLWIRE
  ```
- The following command deselects all vias:
  ```
  editDeselect -object_type Via
  ```
- The following command deselects all wires:
  ```
  editDeselect
  ```
- The following command deselects all wires and vias:
  ```
  editDeselect -object_type {Wire Via}
  ```

Related Information

- `editSelect`
- `getEditMode`
- `redo`
- `setEditMode`
- `undo`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

editDuplicate

```
editDuplicate
[-help]
-layer_horizontal layer
-layer_vertical layer
```

Copies the geometry of selected special wires and pastes them at the same coordinates. If you do not specify a layer, the software pastes them onto the same layer. If you specify a layer, the software copies them to the specified layer and generates vias to maintain connectivity for the duplicated wire.

After duplicating the wires, the software selects the new wires and deselects the original wires. Use the `editMove` command to move duplicated wires to new locations.

Note: Use the `editSelect` command to select wires.

Parameters

`-help`	Prints a brief description that includes type and default information for each `editDuplicate` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man editDuplicate`
`-layer_horizontal layer`	Specifies the new layer for duplicated horizontal wires.
`-layer_vertical layer`	Specifies the new layer for duplicated vertical wires.

Example

- The following command duplicates selected horizontal wires and places them on the *METAL3* layer and duplicates selected vertical wires and places them on the *METAL4* layer. It generates vias for the new wires.
  ```
  editDuplicate -layer_horizontal METAL3 -layer_vertical METAL4
  ```

Related Information

- `editAddRoute`
- `editAdjust`
- `editCommitRoute`
- `editCutWire`
- `editDelete`
- `editMerge`
- `editMove`
- `editResize`
- `editRotate`
- `editSelect`
- `editSplit`
- `editTrim`
- `getEditMode`
- `redo`
- `setEditMode`
- `undo`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

editFixWideWires

```
editFixWideWires [-help]
```

Splits wires that violate the MAXWIDTH value in the LEF LAYER (Routing) statement. Also splits diagonal wires whose width is greater than the MAXWIDTH value as they are drawn.

Parameters

-help	Prints the usage of the command. For a detailed description of the command, use the man command: `man editFixWideWires`

Related Information

- editSplit
- getEditMode
- redo
- setEditMode
- undo
- Editing Wires chapter in the *User Guide*
  - *Repairing Maximum Wire Width Violations*
- Flip Chip Methodologies chapter in the *User Guide*
  - *SPLIT Constraint*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

editMerge

```
editMerge [-help]
```

Merges all wires of the same width, layer, and net that are colinear with and electrically connected to a selected wire. Does not merge signal wires that contain branches. This command also supports 45-degree wire edits.

This command can also be used to merge selected bus guides.

Note: Use the `editSelect` command to select the wire. Use the `selectBusGuide` command to select bus guides or segments.

Parameters

`-help`	Prints the usage of the command. For a detailed description of the command, use the `man` command: `man editMerge`

Related Information

- `editAddRoute`
- `editAdjust`
- `editCommitRoute`
- `editCutWire`
- `editDelete`
- `editDuplicate`
- `editMove`
- `editResize`
- `editRotate`
- `editSelect`
- `editSplit`
- `editTrim`
- `getEditMode`
- `redo`
- `selectBusGuide`
- `setEditMode`
- `undo`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

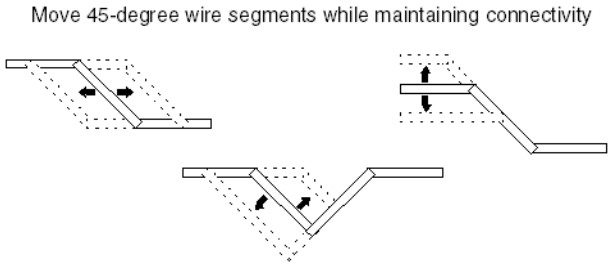
editMove

```
editMove
[-help]
{[[-direction {x y diag45 diag135} -distance float] | [-dx float -dy float] | -to {x y}] }
```

Moves selected wires, boundary pins, or vias.

Moving Wires

`editMove` moves selected wires in the orthogonal direction. When used with the `-direction` and `-distance` or `-dx` and `-dy` combination, `editMove` maintains connectivity by stretching connected wires in the orthogonal direction. This command also supports 45 and 135-degree wire edits.



Use the `setEditMode -connect_with_specified_layer` parameter to control whether layer preference settings are honored when wires are moved. The default value of this parameter is `0`. If you set this parameter to `1`, `editMove` honors the `setEditMode -layer_horizontal` and `-layer_vertical` settings while jogging metal layers. For instance, if you cut a vertical wire on `M4` by using `editCutWire` such that the wire is divided into three wire segments and then move the wire segment in the middle in the horizontal direction, the tool automatically creates new connecting wires on the layer defined by `setEditMode -layer_horizontal` and drops vias correctly. This means you no longer need to perform the additional step for layer change (`editChangeLayer` or `Shift+L`) while moving wires to create a jog. This feature is applicable for both regular and special wires.

To get the current value of the `-connect_with_specified_layer` parameter, you can use the corresponding `getEditMode` parameter.

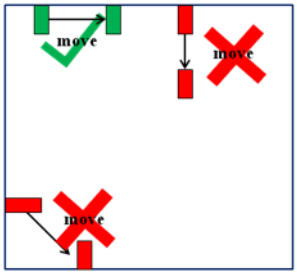
Note: You can use the `setEditMode -snap_to_track` and `-snap_to_track_regular` parameters to specify whether or not to snap wires to the track. To get the current values of these parameters, use the corresponding `getEditMode` options.

Moving Physical Pins

`editMove` moves selected physical pins along with connected wires.

Keep the following considerations in mind while trying to move a physical pin with `editMove`:

- `editMove` can be used to move only the physical pins on the design boundary. Pins inside the design cannot be moved with this command.
- A pin can be moved only along a boundary.
- It cannot be moved between design boundaries.



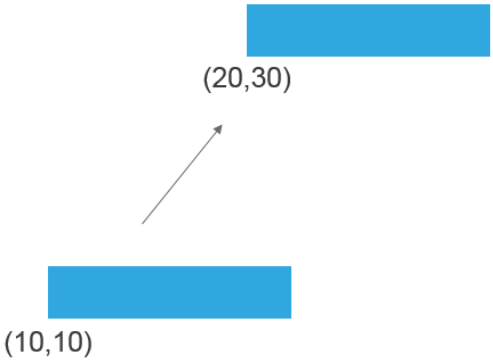
- Pins on the top or bottom boundary can be moved only horizontally.
- Pins on the left or right boundary can be moved only vertically.

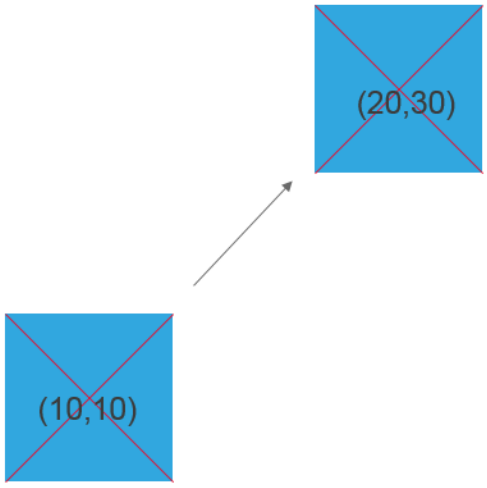
Note: You can use the `setEditMode -snap_to_track_pin` parameter to specify whether or not to snap moved pins to the track. This parameter is set to `true` (`1`) by default. To get the current value of this parameter, use the corresponding `getEditMode` option.

Parameters

`-help`	Prints the usage of the command. For a detailed description of the command, use the `man` command: `man editMove`
`-direction {x y diag45 diag135}`	
	Specifies the direction of movement. Use one of the following options: • `diag45` - Moves selected wires the specified *`-distance`* in a 45-degree direction. This option cannot be used for moving physical pins. • `diag135` - Moves selected wires the specified *`-distance`* in a 135-degree direction. This option cannot be used for moving physical pins. • `x` - Moves selected vertical wires or pins to the right (if you specify a positive value for *`-distance`*) or left (if you specify a negative value for *`-distance`*). • `y` - Moves selected horizontal wires or pins up (if you specify a positive value for *`-distance`*) or down (if you specify a negative value for *`-distance`*).
`-distance` *`float`*	Specifies the distance to move the wire or pin in microns.
`-dx` *`float`*	Specifies the distance in microns by which the selected wire should be moved in the x direction. You must specify the `-dy` value along with it. For example, if you want to move the wire in the x direction by 0.2 microns, specify the following command: `editMove -dx 0.2 -dy 0`
`-dy` *`float`*	Specifies the distance in microns by which the selected wire should be moved in the y direction. You must specify the `-dx` value along with it. For example, if you want to move the wire in the y direction by 0.2 microns, specify the following command: `editMove -dx 0 -dy 0.2`
`-to {x y}`	Specifies the coordinates of the new location to which the selected wire or via is to be moved. If you are moving a via, the `x` and `y` values specify the new coordinates of the via origin. In the case of a wire, the `x` and `y` values specify the new coordinates of the lower left corner of the wire. See Examples. When you move the selected wire or via with `editMove -to`, connected wires are not moved.

Examples

- The following command moves the selected wire such that its lower-left corner lies on the coordinates {`20, 30`} after the move:
  ```
  editMove -to {20 30}
  ```


- The following command moves the selected via such that its origin lies on the coordinates {`20, 30`} after the move:
  ```
  editMove -to {20 30}
  ```


Related Information

- `editAddRoute`
- `editAdjust`
- `editCommitRoute`
- `editCutWire`
- `editDelete`
- `editDuplicate`
- `editMerge`
- `editResize`
- `editRotate`
- `editSplit`
- `editTrim`
- `getEditMode`
- `redo`
- `setEditMode`
- `undo`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

editResize

```
editResize
[-help]
[-keep_center_line {0 1 auto}] [-no_conn]
[-polygon_edge_start {x y} | -to float_value]
{[[[-direction {x y diag45 diag135} -side {high low}] | [-polygon_edge_start {x y} -polygon_edge_end {x y}]] [-offset float_value | -
to float_value]] | [-corner {low_left low_right up_left up_right} -dx float_value -dy float_value]}
```

Resizes wires, bus guides, and pins. The `editResize` command can be used to:

- Change the length of wires and bus guides.
- Change the width of wires and bus guides.
- Enlarge or shrink wires by changing both the length and width of a wire simultaneously.
- Change the width of physical pins.

> While changing the wire width to fix EM violations, apply the following setting before invoking the `editResize` command:
>
> `setEditMode -unrestricted_regular_wire_width true`
>
> By default, when you are changing the width of a wire, it snaps to the closest non-default route (NDR) available. If you set the `-unrestricted_regular_wire_width` parameter to `true`, a wire can be stretched to any width without any restrictions.

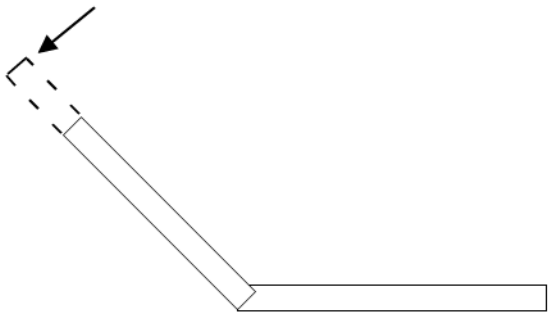
Typically, the `editResize` command can be used in two modes:

Mode 1 - Change the size of a wire or pin in only one direction at a time

If `-direction` is specified, the tool changes the size in the specified direction only. In this mode, the `editResize` command can be used to:

- Change the length of selected regular and special wires by stretching or shrinking them and then snapping them to the closest manufacturing grid point. The `editResize` command also supports the stretching/shrinking of 45 and 135-degree wires.

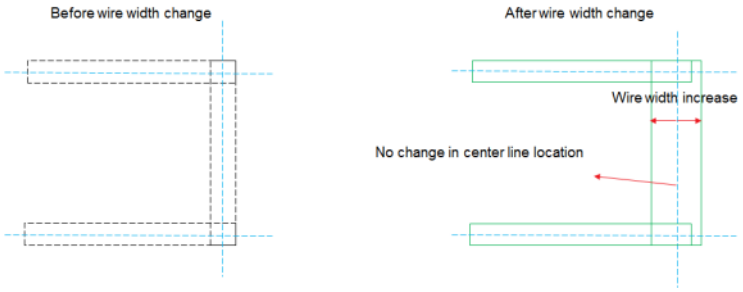
  Stretch wire to support 45-degree wire segments

  Note: Use the `editSelect` command to select the wires.

- Change the width of selected wires and update vias connected to the changed wires to maintain the via-to-wire dimension ratio. `editResize` can be used to change the width of regular as well as special wires:
  - For regular wires, if the width value you specified is not defined in the tech LEF, the tool snaps the width of the regular wire to the closest value defined in the default or non-default rule and prints the following message in the log file:
    `WARNING: The wire width is not defined in default or non default rule, snap wire width to the closest value defined in LEF file`
    The tool also selects a suitable via cell defined in the tech LEF file. If a suitable via cell is not available, the tool does not drop any via.

    When you change the width of a regular wire, the center line is still connected with the neighboring wires after the change:

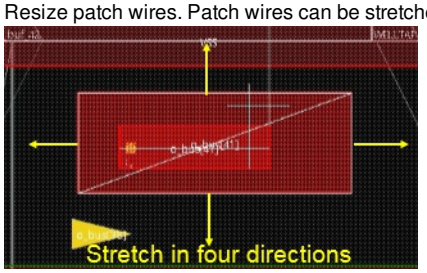


- For a special wire, you can change the wire width freely by specifying an incremental value. The tool uses viagen to generate a via after the width change of a special wire.

Note: Use the `editSelect` command to select the wires.

If you change the width of a wire, and then change it back to the original width, you might see a discrepancy due to rounding.
Note: If you specify a width value that is not an even multiple of the manufacturing grid value, the software displays a warning.

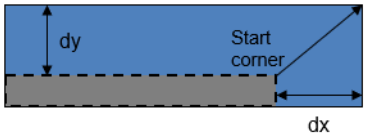
- Resize patch wires. Patch wires can be stretched in four directions:



- Change the width of physical pins. While using the `editResize` command to change the pin width, keep the following points in mind:
  - If `setEditMode -drc_on true` has been specified before modifying the pin width, the tool follows the rules from the LEF file while changing pin width.
  - The pin width, whether vertical or horizontal, should not be less than the minimum width defined in the LEF file.
  - If you specify a value less than minimum width for a pin, the tool uses the minimum width for that pin.

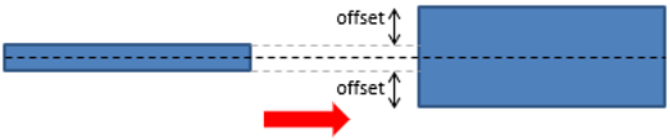
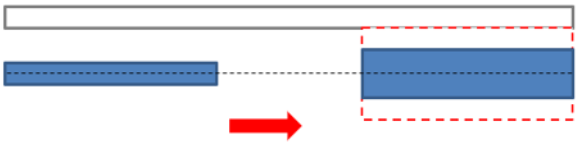
Mode 2 - Change the size of a wire in both the x and y directions simultaneously

If `-direction` is not specified, the tool changes both the width and height of the selected wire at the same time. You can use `-corner` to specify the start corner from which to stretch or shrink the wire. This mode works for only horizontal or vertical special wires in the no_connection (`-no_conn`) mode. Use `-dx` and `-dy` to specify the distance to stretch the wire in x and y directions, respectively.

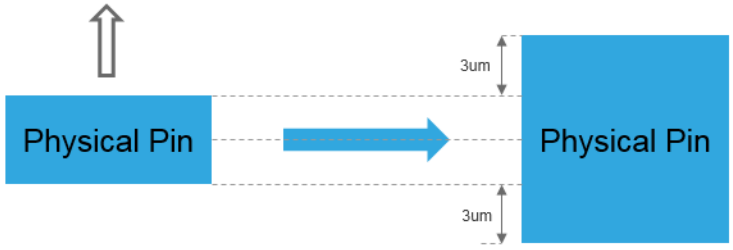


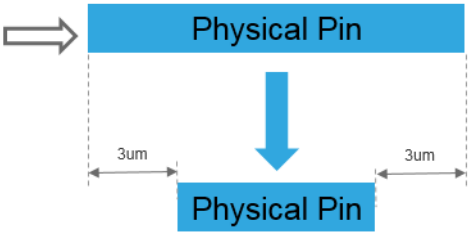
Parameters

`-help`	Prints a brief description that includes type and default information for each `editResize` parameter. For a detailed descriptic `man editResize`
`-corner {low_left low_right up_left up_right}`	
	Specifies the starting corner from which the wire should be stretched.
`-direction {x y diag45 diag135}`	

	Specifies the direction in which the wire or pin is to be stretched or shrunk: • `x` - Resizes wires or pins horizontally. For a horizontal wire, specify the direction as `x` if you want to modify the length of t wire, specify the direction as `x` if you want to modify the width of the wire. • `y` - Resizes wires or pins vertically. For a horizontal wire, specify the direction as `y` if you want to modify the width of the modify the length of the wire (i.e stretch or shrink the vertical wire). • `diag45` - Resizes the selected wires in the 45-degree direction by the specified offset value. **Note**: This parameter can be used only for wires. • `diag135` - Resizes the selected wires in the 135-degree direction by the specified offset value. **Note**: This parameter can be used only for wires.
`-dx float_value`	Specifies the distance to which the wire should be stretched in the x direction.
`-dy float_value`	Specifies the distance to which the wire should be stretched in the y direction.
`-keep_center_line {0 1 auto}`	Controls whether the wire or physical pin is resized with the center line unchanged: • `0` - Resizes the wire or pin from the specified side only. This option works in both the connection and no_connection (`-n` This setting cannot be used for changing the width of signal wires. • `1` - Resizes the wire or pin from both the low and high sides, even if you are stretching it from only one side. This is done  This option works in both the connection and no_connection (`-no_conn`) modes for wires. However, a physical pin can k mode only. • `auto` - When set to `auto`, the software automatically detects when to keep the center line unchanged. *Default*: `auto`
`-no_conn`	Resizes the wire without auto via/wire creation and DRC checking. If `-no_conn` is not specified, the tool will reprocess the via and do basic DRC checking:  **Note:** Patch wires are resized without auto via/wire creation and DRC checking even if `-no_conn` is not specified.
`-offset float_value`	Specifies, in microns, the distance to which the wires or pins should be stretched or shrunk.
`-polygon_edge_end {x y}`	Specifies the end point for a polygon edge {x y}. This option is used only to resize a polygon.
`-polygon_edge_start {x y}`	Specifies the start point for a polygon edge {x y}. This option is used only to resize a polygon.
`-side {high low}`	Specifies the side from which the wire or pin is to be stretched or shrunk: • `high` - Stretches or shrinks wires or pins from the top or the right. • `low` - Stretches or shrinks the wires or pins from the bottom or the left.
`-to float_value`	Specifies, in microns, the new height for selected horizontal wires or the new width for selected vertical wires.

Examples

- The following command stretches the selected vertical wire from the top by 8.5 µm.
  ```
  editResize -direction y -offset 8.5 -side high
  ```
- The following command stretches the selected vertical wire from the bottom by 0.511 µm.
  ```
  editResize -direction y -offset -0.511 -side low
  ```
- The following command shrinks the selected vertical wire from the bottom by 0.619 µm.
  ```
  editResize -direction y -offset 0.619 -side low
  ```
- The following command stretches the selected horizontal wire from the right by 1.27 µm.
  ```
  editResize -direction x -offset 1.27 -side high
  ```
- The following command shrinks a horizontal wire from the left by 0.443 µm.
  ```
  editResize -direction x -offset 0.443 -side low
  ```
- The following command stretches a horizontal wire from the left by 0.86 µm.
  ```
  editResize -direction x -offset -0.86 -side low
  ```
- The following command increases the width of the selected horizontal special wire by 0.01 µm:
  ```
  editResize -direction y -offset 0.01 -side high
  ```
- The following command changes the width of the selected horizontal regular wire to 0.48 µm:
  ```
  editResize -to 0.48 -direction y -side high
  ```
  If 0.48 is not defined in the NDR, the tool snaps the width to the closest value defined in the NDR.
- The following command increases the vertical width of the selected physical pin by 3 µm each from both top and bottom, keeping the center line unchanged:
  ```
  editResize -direction y -offset 3 -side high -no_conn -keep_center_line 1
  ```
  

- The following command reduces the horizontal width of the selected physical pin by 3 µm each from both left and right, keeping the center line unchanged:
  ```
  editResize -direction x -offset -3 -side low -no_conn -keep_center_line 1
  ```
  

Related Information

- `editAddRoute`
- `editAdjust`
- `editCommitRoute`
- `editCutWire`
- `editDelete`
- `editDuplicate`
- `editMerge`
- `editMove`
- `editRotate`
- `editSelect`
- `editSplit`
- `editTrim`
- `getEditMode`
- `redo`
- `setEditMode`
- `undo`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

editRotate

```
editRotate
[-help]
[-mode {group | single}]
[-rotate_point {x y}]
{-flip {MX MY} | -rotate {R90 R180 R270}}
```

Flips or rotates selected wires and vias. Use this command for more flexible custom routing and fixing DRC violations using existing wires. Using the `editRotate` command, you can:

- Flip selected wires or vias on the X or Y axis (`MX` or `MY`)
- Rotate the selected wires or vias by 90, 180, or 270 degrees (`R90`, `R180`, or `R270`).

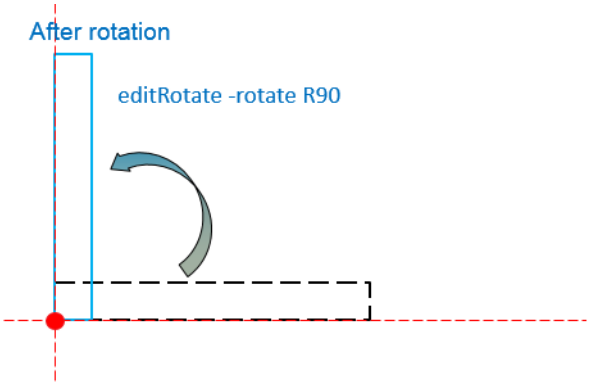
The selected object is rotated or flipped based on current orientation, and the lower-left coordinate of the wire/via remains the same after rotation or flip.

You can also flip or rotate multiple objects simultaneously as one group.

Parameters

`-help`	Prints the usage of the command. For a detailed description of the command, use the `man` command: `man editRotate`
`-flip {MX MY}`	Flips a wire or via. If you specify `MX`, the selected object is flipped on the X axis. If you specify `MY`, the object is flipped on the Y axis.
`-mode {group single}`	Rotates multiple selected wires and vias at the same time using the specified mode: • `group` - Flips or rotates the selected objects as a group. This is the default mode when multiple objects are selected. In this mode, the tool flips or rotates the bounding box which encloses all the selected objects. • `single` - Flips or rotates the selected objects individually.
`-rotate {R90 R180 R270}`	Rotates wires or vias by 90, 180, or 270 degrees, as specified.
`-rotate_point {x y}`	Specifies the point around which the wire should be rotated. If this option is not specified, the selected wires/vias are rotated around the lower-left corner of bounding box of wires/vias.

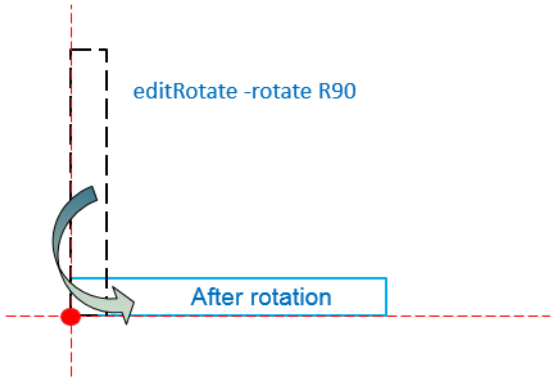
Examples

- The following command rotates the selected wire by 90 degrees:
  ```
  editRotate -rotate R90
  ```
  

- The following command rotates the selected object, which originally has `R90` as its original orientation by 90 degrees:

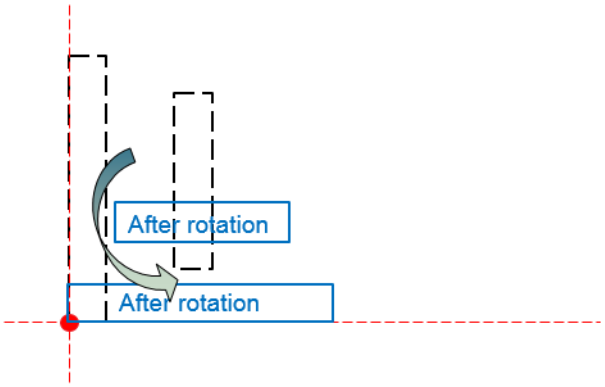
```
editRotate -rotate R90
```

The final orientation of the object will be `R180` as shown below:



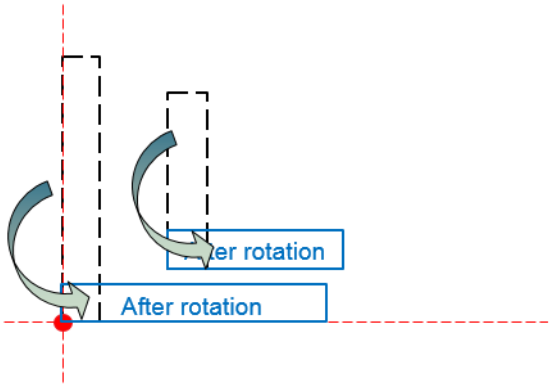
- The following command rotates the bounding box of the selected wires by 90 degrees to rotate the wires as one group:

```
editRotate -rotate R90 -mode group
```



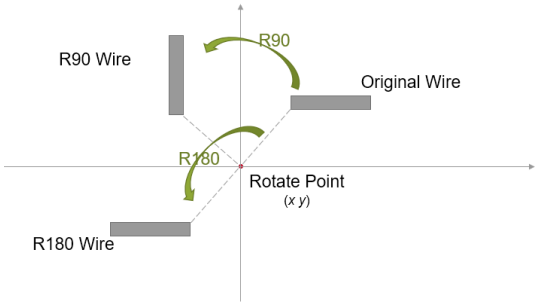
- The following command rotates the selected wires individually by 90 degrees:

```
editRotate -rotate R90 -mode single
```



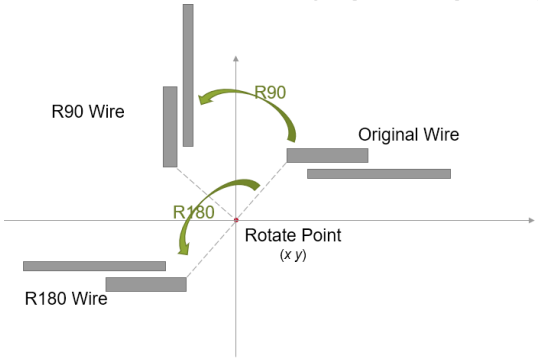
- The following commands rotate the selected wire by 90 and 180 degrees, respectively, around the specified rotation point, *x y*:

```
editRotate -rotate R90 -rotate_point x y
editRotate -rotate R180 -rotate_point x y
```



- The following commands rotate the selected group of wires by 90 and 180 degrees, respectively, around the specified rotation point, `x y`:

```
editRotate -rotate R90 -mode group -rotate_point x y
editRotate -rotate R180 -mode group -rotate_point x y
```



Related Information

- `editAddRoute`
- `editAdjust`
- `editCommitRoute`
- `editCutWire`
- `editDelete`
- `editDuplicate`
- `editMerge`
- `editMove`
- `editResize`
- `editSplit`
- `editTrim`
- `getEditMode`
- `redo`
- `setEditMode`
- `undo`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

editSelect

```
editSelect
[-help]
[-area {x1 y1 x2 y2}]
[-direction {H | V | 45 | 135}]
[-floating_via]
[-layer list_of_layers]
[-nets net*]
[-physical_pin_only]
[-regular_wire_with_drc]
[-shape {RING STRIPE FOLLOWPIN IOWIRE COREWIRE BLOCKWIRE PADRING
         BLOCKRING FILLWIRE FILLWIREOPC DRCFILL None}]
[-shield]
[-shield_only]
[-status {COVER FIXED NOSHIELD ROUTED SHIELD UNKNOWN}]
[-subclass sublass_name]
[-type {Regular Special Patch}]
[-use {CLOCK POWER SIGNAL}]
[-via_cell cell_name_list]
[-with_edit_flag {0 | 1}]
[-object_type {Wire Via}]
[-from_pin pin_name -to_pin pin_name]
```

Selects wires and vias using the specified parameters. The wires are updated by subsequent wire editing commands.

Parameters

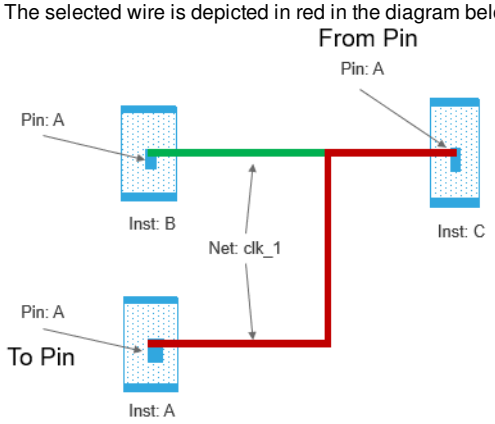
`-help`	Prints the usage of the command. For a detailed description of the command, use the `man` command: `man editSelect`
`-area {x1 y1 x2 y2}`	Specifies the coordinates for the area from which to select wires and vias. *Default:* Entire area *x1*: Specifies the lower left x coordinate. *y1*: Specifies the lower left y coordinate. *x2*: Specifies the upper right x coordinate. y2: Specifies the upper right y coordinate.
`-direction {H | V | 45 | 135}`	Specifies the orientation of the wires to be selected. *Default:* If you do not specify this parameter, selects wires with any orientation. H: Specifies that horizontal wires are to be selected. V: Specifies that vertical wires are to be selected. 45: Specifies that 45-degree wires are to be selected. 135: Specifies that 135-degree wires are to be selected.
`-floating_via`	Specifies that only floating vias are to be selected. A via, or a stack of vias, is floating, if it is not connected to another object, wire, pin, or via, on either its top or bottom metal layer or on both these layers.
`-from_pin pin_name`	Specifies the name of the starting pin of the wire to be selected. The starting pin you specify should be connected to the same net as the ending pin specified with `-to_pin`. If they connect to different nets, the tool reports an error. In addition, the from and to pins you specify should be connected to the same wire. **Note**: Selection of wires between specified pins is supported only for signal and clock nets.
`-layer list_of_layers`	Specifies the layers from which to select wires and vias. This option supports backside and Through Silicon Via (TSV) layers as well. If you are specifying multiple layers, enclose them in braces. *Default:* If you do not specify this parameter, selects wires and vias on all layers.

`-nets net*`	Specifies the net names from which to select wires and vias. You can use wildcard values in this field. For example, if you specify `V*`, selects wires/vias from nets `VDD` and `VSS`.
`-object_type {Wire Via}`	Specifies whether wires or vias or both should be selected. If the parameter is not specified, only wires are selected. *Default*: `Wire`
`-physical_pin_only`	Specifies that physical pins are to be selected. By default, `editSelect` does not select physical pins. You can use the `-physical_pin_only` parameter along with `-area`, `-layer`, or `-nets` to select the physical pins in the specified area, layer, or net, respectively.
`-regular_wire_with_drc`	Selects regular wires with violation markers. Clock nets are also selected.
`-shape {RING STRIPE FOLLOWPIN IOWIRE COREWIRE BLOCKWIRE PADRING BLOCKRING FILLWIRE FILLWIREOPC DRCFILL None}`	
	Specifies the shapes of the wires and vias to be selected. Use all upper-case letters for the shapes. If you specify multiple shapes, enclose the list of shapes in braces. For shape definitions, see *Shapes,* in the "DEF Syntax" chapter of the LEF/DEF Language Reference. *Default:* If you do not specify this parameter, selects wires and vias of all shapes.
`-shield`	Selects the wires and vias of the nets specified with the -nets parameter, along with the wires and vias shielding the specified nets. **Note**: This parameter cannot be used with `-shield_only`.
`-shield_only`	Selects only the shield wires associated with the nets specified using the `-nets` parameter. The signal nets being shielded are not selected. **Note**: This parameter cannot be used with `-shield`.
`-status {COVER FIXED NOSHIELD ROUTED SHIELD UNKNOWN}`	
	Specifies the status of wires and vias to be selected. Use all upper-case letters for the status. If you specify more than one status, enclose the status list in braces. **Note:** If you select `-type Special -status NOSHIELD` or `-type Signal -status SHIELD` the software generates an error message.
`-subclass sublass_name`	Specifies the subclass name of the wires and vias to be selected. Use this parameter if you want to select wires or vias belonging to a specific subclass. Here, *`subclass_name`* is a user-defined attribute that has been previously defined using the `-uda` parameter of the `addSplitPowerVia`, `addRing`, `addStripe`, or `sroute` command. For wires, use this parameter if you want to select or delete rings, stripes, or other wires belonging to a specific subclass. Only special wire is supported by this option. **Note:** You can use wildcards in *`subclass_name`*.
`-to_pin pin_name`	Specifies the name of the ending pin of the wire to be selected. The ending pin you specify should be connected to the same net as the starting pin specified with `-from_pin`. If they connect to different nets, the tool reports an error. In addition, the from and to pins you specify should be connected to the same wire. If there there are more than one wire or via between the specified starting and ending pins, the command is not able to select the wire path and returns an error. The `-from_pin` and `-to_pin` parameters are compatible with only the `-direction`, `-layer`, `-object_type`, and `-status` options of the `editSelect` command. If you use `-from_pin` and `-to_pin` with any other `editSelect` parameters, the tool returns an error. **Note**: Selection of wires between specified pins is supported only for signal and clock nets.
`-type {Regular Special Patch}`	
	Specifies whether to select regular wires and vias, special wires and vias, or patch wires. **Note:** If you specify `Regular`, you must omit the `-shapes` parameter or specify `-shapes None`.
`-use {CLOCK POWER SIGNAL}`	
	Specifies whether to select only CLOCK wires/vias, POWER wires, or SIGNAL wires.

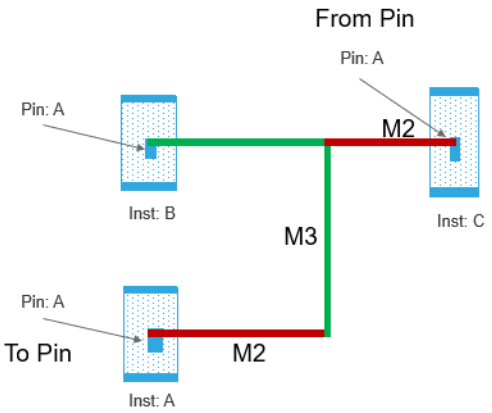
`-via_cell` *`cell_name_list`*	Specifies the via cells of the vias to be selected. You can use wildcards while specifying via cell names. If you are specifying multiple cell names, enclose them in braces.
`-with_edit_flag {0 | 1}`	Selects all modified wires and vias, that is wires and vias with value `1` for the `isEdit` property on the net. **Notes** • As the `isEdit` property is added on the whole net, all the wires and vias on the net with `isEdit` value set to `1` will be selected. • The `isEdit` property is used to flag nets with modified wires and vias. This is useful when evaluating how many objects need to be modified for fixing DRC violations and then deciding whether to fix the violations manually or through ECO routing. The `isEdit` property is controlled through the `-create_is_edit_flag` parameter of `setEditMode`. By default, this parameter is set to `0`. If you set the value of this parameter to `1`, the `isEdit` property is set to `1` for nets with modified wires or vias.

Examples

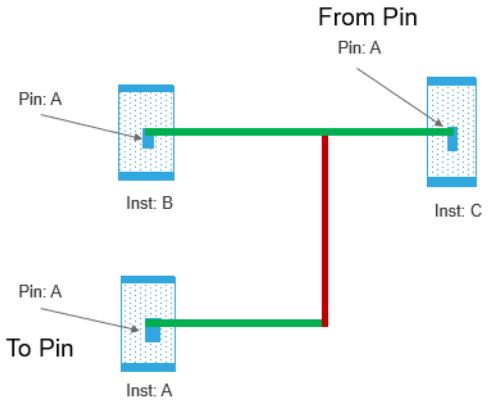
- The following command selects all wires on the *METAL1* and *METAL2* layers:
  ```
  editSelect -layer {METAL1 METAL2}
  ```
- The following command selects all vias on the cut layer `VIA12`:
  ```
  editSelect -layer VIA12 -object_type Via
  ```
- The following command selects all vias on the cut layers `VIA12` and `VIA23`:
  ```
  editSelect -layer {VIA12 VIA23} -object_type Via
  ```
- The following command selects all wires on the *METAL1* and *METAL2* layers and vias on the cut layer `VIA12`:
  ```
  editSelect -layer {METAL1 METAL2 VIA12} -object_type {Wire Via}
  ```
- The following command selects all horizontal fillwires on the *METAL1* layer:
  ```
  editSelect -layer METAL1 -direction H -shape FILLWIRE
  ```
- The following command selects VDD STRIPEs with subclass "stripe_A":
  ```
  editSelect -nets VDD -shape STRIPE -type Special -subclass stripe_A
  ```
- The following command selects all vias of shape `FOLLOWPIN` with names that match the pattern `tapcellvia_VIA1_RUL*`:
  ```
  editSelect -shape FOLLOWPIN -via_cell tapcellvia_VIA1_RUL* -object_type Via
  ```
- The following command selects all the modified wires and vias:
  ```
  editSelect -with_edit_flag 1 -object_type {Wire Via}
  ```
- The following command selects the complete wire between the specified from and to pins:
  ```
  editSelect -from_pin C/A -to_pin A/A
  ```
  The selected wire is depicted in red in the diagram below:



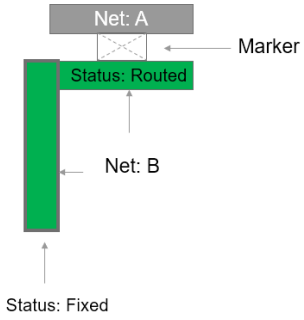
- The following command selects only the wire segments on the `M2` layer between the specified from and to pins:
  ```
  editSelect -from_pin C/A -to_pin A/A -layer M2
  ```
  The selected wire segments are depicted in red in the diagram below:



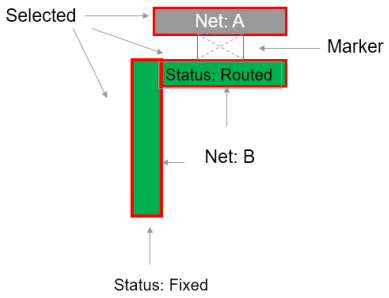
- The following command only the vertical wire segments between the specified from and to pins:
  ```
  editSelect -from_pin C/A -to_pin A/A -direction V
  ```
  The selected wire segment is depicted in red in the diagram below:



- Consider the example below:



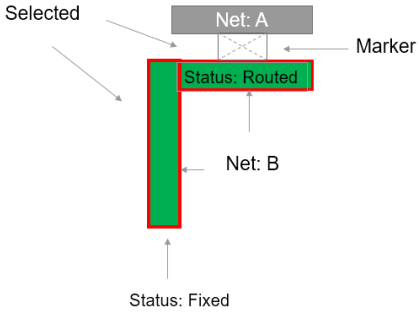
  - The following command selects all nets that have DRC markers:
    ```
    editSelect -regular_wire_with_drc
    ```



Both nets A and B are selected.

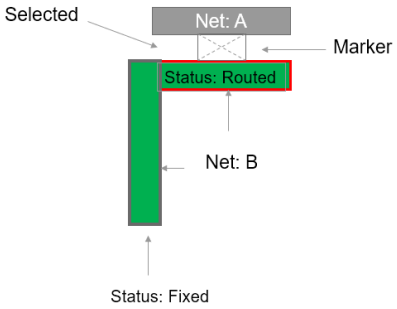
- The following command selects wires with DRC markers from only Net B:

```
editSelect -nets B -regular_wire_with_drc
```



- The following command selects only routed wires with DRC markers from Net B:

```
editSelect -nets B -regular_wire_with_drc -status ROUTED
```



- The following set of commands select and delete nets with DRC markers while excluding clock nets:

```
editSelect -regular_wire_with_drc
editDeselect -use CLOCK
editDelete -selected
```

Related Information

- `editAddRoute`
- `editAdjust`
- `editCommitRoute`
- `editChangeLayer`
- `editChangeNet`
- `editChangeRule`
- `editChangeStatus`
- `editChangeVia`
- `editCutWire`
- `editDelete`
- `editDeselect`
- `editDuplicate`
- `editMerge`
- `editMove`
- `editResize`
- `editSplit`
- `editTrim`
- `getEditMode`
- `redo`
- `setEditMode`
- `undo`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

editSplit

```
editSplit [-help]
```

Splits selected special wires at connection points with orthogonal wires. This command can also be used to split selected bus guides.

Note: Use the `editSelect` command to select the wires. Use the `selectBusGuide` command to select bus guides or bus guide segments

Parameters

`-help`	Prints the usage of the command. For a detailed description of the command, use the `man` command: `man editSplit`

Related Information

- `editAddRoute`
- `editAdjust`
- `editCommitRoute`
- `editCutWire`
- `editDelete`
- `editDuplicate`
- `editMerge`
- `editMove`
- `editSelect`
- `editResize`
- `editTrim`
- `getEditMode`
- `redo`
- `selectBusGuide`
- `setEditMode`
- `undo`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

Related Topics

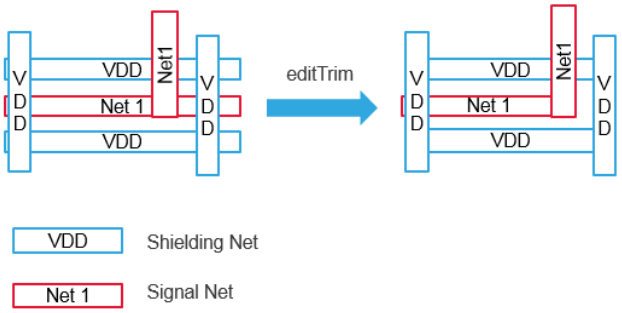
- To check how undo and redo works with this wire editing command, see `undo` and `redo` in the *Text Command Reference*.
- To review global controls that affect wire editing, see `setEditMode` and `getEditMode` in the *Text Command Reference*.
- For more help on editing wires, see the Editing Wires chapter in the *Innovus User Guide*.

editTrim

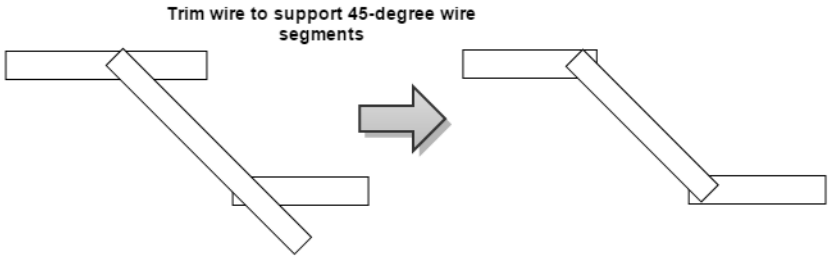
```
editTrim
[-help]
[-keep_floating_stubs]
[-status {COVER FIXED NOSHIELD ROUTED SHIELD}]
[-type {float}]
[-all | -selected | -nets net* | -layers layer_names | -area {x1 y1 x2 y2}]
```

Removes or trims wires. Wires are trimmed or removed as follows:

- Trims floating stubs of wires back to the closest connection point.
- Removes wires that do not have electrical connections to other wires or pins (floating wires).
- Removes wires with only one connection to another wire or pin, along with connecting vias. Removing a wire with one connection might cause the wire to which the just-removed wire is connected to change from being a wire with two connections to a wire with one connection and thus it will also be removed. This process is repeated until no wires with one connection are left.
  Note: The `editTrim` command recognizes patch wires and detects the patch wire connection with the via. Therefore, it does not delete patch wires even if they are connected from only one side.
- Trims shield wires along with signal wires.

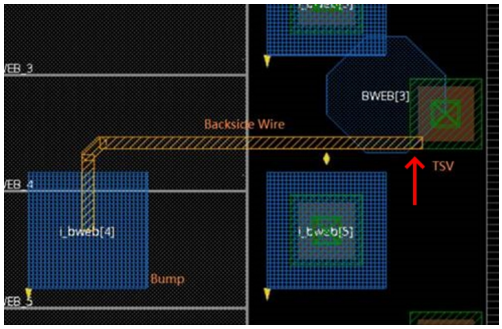


- Trims wires to support 45-degree wire segments.



Notes

- The `editTrim` command recognizes Through Silicon Via (TSV) connections and detects the backside wire connection with the TSV so it does not remove the backside routes connected to TSV.



- The `editTrim` command does not trim metal fill segments. To trim metal fill segments use the `trimMetalFill` command.

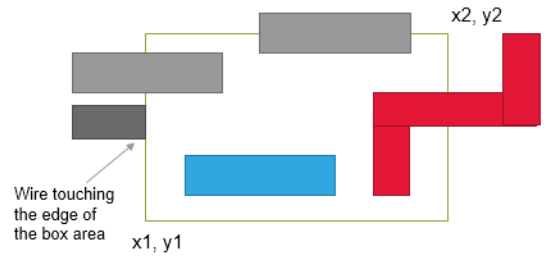
Parameters

`-help`	Prints the usage of the command. For a detailed description of the command, use the `man` command: `man editTrim`
`-all`	Trims all special wires and floating stubs.
`-area {x1 y1 x2 y2}`	Specifies the coordinates for the box area from which to trim the wires. Here: • `x1` - Specifies the lower left x coordinate. • `y1` - Specifies the lower left y coordinate. • `x2` - Specifies the upper right x coordinate. • `y2` - Specifies the upper right y coordinate. Only the wires that overlap with the specified box area are trimmed. The wires that just touch the box area are not trimmed.
`-keep_floating_stubs`	Selects the floating wires but does not remove the floating stubs. Use this parameter if you want to assess the impact before actually trimming.
`-layers layer_names`	Specifies the layers from which to trim wires. If you are specifying multiple layers, enclose the layer names in braces. *Default*: If you do not specify this parameter, the command trims wires on all layers.
`-nets net*`	Trims wires on the nets you specify. You can use wildcards to specify the nets.
`-selected`	Trims selected wires. Use the `editSelect` command to select the wires. This is the default value.
`-status {COVER FIXED NOSHIELD ROUTED SHIELD}`	
	Allows you to filter the wires to be trimmed by status. If you specify `-status` without specifying `-all`, `-nets`, or `-selected`, the tool will trim selected wires with the specified status.
`-type {float}`	Removes only wires that do not have electrical connections to other wires or pins (floating wires). The tool will not trim the wire with connection to other wires at one end if this option is specified.

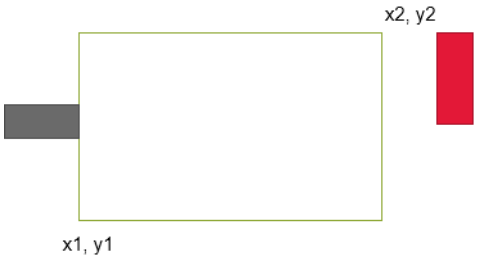
Examples

- The following command trims the wires overlapping the box area specified by the coordinates `{x1 y1 x2 y2}`:
  `editTrim -area {x1 y1 x2 y2}`

  Before

  After

- The following command trims the wires in the specified layers:
  `editTrim -layers {Metal1 Metal3}`

Related Information

- editAddRoute
- editAdjust
- editCommitRoute
- editCutWire
- editDelete
- editDuplicate
- editMerge
- editMove
- editSelect
- editResize
- editRotate
- editSplit
- getEditMode
- redo
- selectBusGuide
- setEditMode
- undo
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

end_parallel_edit

```
end_parallel_edit
[-help]
-out_file parallel_edit_file_name
```

Ends parallel editing. The command:

- compares the data in memory with the original data saved in binary files by `start_parallel_edit`
- saves the parallel edit file with the specified name

Multiple `end_parallel_edit` commands are allowed after one `start_parallel_edit`.

Parameters

`-help`	Prints a brief description that includes type and default information for each `end_parallel_edit` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man end_parallel_edit`
`-out_file parallel_edit_file_name`	Specifies the name of the parallel edit file.

Example

- The following command compares the data in memory with the data saved in the binary files and writes the parallel edit information to `PE1.txt` file:

```
innovus 8> end_parallel_edit -out_file PE1.txt
Start saving parallel edit information to PE1.txt
************************************************************************
Comparing net attribute and regular wire/via with reference DB
Comparing instance with reference DB
Comparing physical pin with reference DB
Comparing NDR rule with reference DB
Comparing routing blockage with reference DB
Comparing placement blockage with reference DB
Comparing via cell with reference DB
Comparing special wire/via with reference DB
************************************************************************
End saving parallel edit information
```

Related Information

- `read_parallel_edit_files`
- `start_parallel_edit`
- Editing Wires chapter in the *User Guide*
  - *Parallel Editing Capability*

getEditMode

```
getEditMode
[-help]
[-align]
[-allow_45_degree]
[-arrow_increment]
[-assign_multi_pattern_color]
[-auto_distribute_bus]
[-auto_split_bus]
[-bus_honor_start_params]
[-bus_honor_width_setting]
[-bus_total_width_horizontal]
[-bus_total_width_vertical]
[-change_order_at_turn]
[-check_design_boundary]
[-circle_NDR_vias_only]
[-close_polygons]
[-color_align_with_track]
[-connect_pin]
[-connect_with_specified_layer]
[-create_crossover_vias]
[-create_is_edit_flag]
[-create_via_on_pin]
[-cut_class]
[-cut_wire_overlap]
[-debug_file]
[-delete_tsv]
[-delete_pin_with_wire]
[-delete_wire_via_deep_through]
[-display_wire_length_with_cursor]
[-draw_shield]
[-draw_with_group_centerline]
[-drawing_wire]
[-drc_aware_cross_metal]
[-drc_on]
[-drc_use_non_default_spacing]
[-extend_wires]
[-final_check_with_verify]
[-ignore_drc]
[-jog_connect_layer]
[-keep_floating_via]
[-keep_status]
[-keep_via]
[-lateral_movement_range]
[-layer]
[-layer_horizontal]
[-layer_maximum]
[-layer_minimum]
[-layer_vertical]
[-look_down_layers]
[-look_up_layers]
[-max_pointer_number]
[-mirror_bus_route]
[-nets]
[-no_merge_special_wire]
[-only_show_edit_layer]
[-orthogonal_connection_only]
[-outer_shield_spacing]
[-outer_shield_width]
[-override]
[-partial_overlap_threshold]
[-pull_back_distance]
[-reshape]
[-return_object_pointer]
[-rule]
[-shape]
[-shield]
```

```
[-shield_look_down_layers]
[-shield_look_up_layers]
[-shield_shape]
[-shielding_nets]
[-show_drc_info_for_edit_shape]
[-sibling_look_down_layers]
[-sibling_look_up_layers]
[-snap]
[-snap_align_to]
[-snap_bus_to_pin]
[-snap_end_to]
[-snap_objects_to_track]
[-snap_to]
[-snap_to_track_honor_color]
[-snap_trim_metal_to_trim_grid]
[-spacing]
[-spacing_horizontal]
[-spacing_vertical]
[-status]
[-stop_at_drc]
[-stretch_end]
[-stretch_with_intersect]
[-subclass]
[-turn_at]
[-type]
[-unrestricted_regular_wire_width]
[-update_shield_net]
[-use_fixVia]
[-use_interleaving_wire_group]
[-use_wire_group]
[-use_wire_group_bits]
[-use_wire_group_reinforcement]
[-use_wire_group_reinforcement_group_via]
[-use_wire_group_reinforcement_spacing]
[-use_wire_group_reinforcement_width]
[-verbose]
[-via_allow_geom_drc]
[-via_auto_replace]
[-via_auto_snap]
[-via_cell_name]
[-via_columns]
[-via_create_by]
[-via_cut_layer]
[-via_exclude_spec]
[-via_override_spec]
[-via_rows]
[-via_scale_height]
[-via_scale_width]
[-via_snap_honor_color]
[-via_snap_to_intersection]
[-via_type]
[-width]
[-width_horizontal]
[-width_vertical]
[-wire_override_spec]
[-quiet]
[-nonDefault]
```

Returns information about the specified `setEditMode` parameter in the Innovus log file and the console. The following information is returned:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the software returns information for all of the `setEditMode` parameters.

Parameters

`-help`	Prints a brief description that includes type and default information for each `getEditMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man getEditMode`
`parameter_name`	Returns value for the specified `setEditMode` parameters. You can specify one or more parameters. See `setEditMode` for names and descriptions of the parameters you can specify.
`-quiet`	Returns the current settings for the specified parameters in the Tcl list format only. If you specify `-quiet` without any parameters, the software returns the current settings of all `setEditMode` parameters in the Tcl list format.

Examples

- The following command returns the current value for the `setEditMode -shape` parameter, which specifies the shape associated with the wire you draw:

```
getEditMode -shape
-shape STRIPE                # enums={RING STRIPE FOLLOWPIN IOWIRE COREWIRE BLOCKWIRE PADRING BLOCKRING FILLWIRE FILLWIREOPC DRCFILL
None}, default=STRIPE
STRIPE
```

- The following command returns the current value of the `setEditMode -via_cut_layer` parameter, which specifies the via layer name for the via to be created or modified:

```
getEditMode -via_cut_layer
-via_cut_layer V12           # string, default=V12
V12
```

Related Information

- `editAddRoute`
- `editAdjust`
- `editChangeLayer`
- `editChangeNet`
- `editChangeRule`
- `editChangeStatus`
- `editChangeVia`
- `editCommitRoute`
- `editCutWire`
- `editDelete`
- `editDuplicate`
- `editMerge`
- `editMove`
- `editResize`
- `editSplit`
- `editSelect`
- `editTrim`
- `setEditMode`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

prepareForEcoRoute

```
prepareForEcoRoute
[-help]
[-exclude nets]
[-nets nets]
```

Converts special nets to geometrically equivalent regular nets. You can convert all special nets, named special nets, or all special nets except those you specify. The `prepareForEcoRoute` command supports polygon shapes as well.

Notes

- If you do not specify any parameters, the `prepareForEcoRoute` command converts all special nets to signal nets.
- Use the `prepareForEcoRoute` command to convert physical nets. To convert logical nets, use `setNet`.
- The `prepareForEcoRoute` command does not work for PG nets.

Parameters

`-help`	Prints a brief description that includes type and default information for each `prepareForEcoRoute` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man prepareForEcoRoute`
`-exclude nets`	Specifies the special nets that you do not want to convert. You can use wildcards. • If a name contains a special character, enclose it in braces (`{}`). • If you specify multiple net names, enclose the names in braces (`{}`).
`-nets nets`	Specifies the special nets to be converted. You can use wildcards. • If a name contains a special character, enclose it in braces (`{}`). • If you specify multiple net names, enclose the names in braces (`{}`).

Note: For backward compatibility, this command also supports the following syntax for nets to convert:

```
prepareForEcoRoute netName [netName2 netName3 ...]
```

Examples

- The following command converts special nets named `clk*` to regular nets:
  ```
  prepareForEcoRoute -nets clk*
  ```
- The following command converts all special nets, except those named `clk1*`, to regular nets:
  ```
  prepareForEcoRoute -exclude clk1*
  ```
- The following command converts special nets named `clk*` to regular nets, except nets named `clk1*`:
  ```
  prepareForEcoRoute -nets clk* -exclude clk1*
  ```
- The following command converts all special nets named `clk*`, `a2`, and `s2[1]`, except for nets `clk1` and `clk2`, to regular nets:
  ```
  prepareForEcoRoute -nets {clk* a2 s2[1]} -exclude {clk1 clk2}
  ```

Related Information

- `getEditMode`
- `redo`
- `setEditMode`
- `setNet`
- `undo`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

read_parallel_edit_files

```
read_parallel_edit_files
[-help]
[-check]
[-conflict_mode {only_one stop_all}]
-files list_of_files
[-ignore_error]
[-report_file file_name]
```

Loads all the specified parallel edit files and implements the editing changes recorded in them.

During parallel editing, there is a possibility of a conflict if the same attribute of an object is changed in different parallel edit files. Some cases in which a conflict can occur are listed below:

- Two users edit the layer of the same wire segment.
- Two users edit the width of the same NDR rule.
- One user cuts the wire segment, and another user changes the status of the same wire.

In the following examples of parallel edits, there is no conflict:

- One user edits the layer of wire, and another user edits its mask.
- One user edits the width of an NDR rule, and another user edits its spacing.

In case of any conflict in the parallel edit files, the tool implements changes as per the specified conflict mode and writes the conflicting information to a report file.

For information on the parallel edit flow, format of a parallel edit file, and conflict handling examples see the "*Parallel Editing Capability"* section in the Editing Wires chapter of the *Innovus User Guide.*

Parameters

`-help`	Prints a brief description that includes type and default information for each `read_parallel_edit_files` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_parallel_edit_files`
`-check`	Specifies that the tool should only check the syntax and conflict of the files but should not actually implement the changes.
`-conflict_mode {only_one stop_all}`	Specifies the conflict mode. The two possible modes are: • `only_one` - In case of conflicting commands, the first read command is implemented and the other command is recorded in the conflict report. • `stop_all` - In case of conflicting commands, the tool implements neither of the conflicting commands and records both of them in the conflict report.
`-files list_of_files`	Specifies the list of parallel editing files.
`-ignore_error`	Specifies whether errors in files should be ignored. When this option is specified, the tool skips lines with issues in the files but continues to read the other wires on the same net.
`-report_file file_name`	Specifies the name of the conflict report.

Related Information

- `end_parallel_edit`
- `start_parallel_edit`
- Editing Wires chapter in the *User Guide*
  - *Parallel Editing Capability*

reportLegalWireWidthForBump

```
reportLegalWireWidthForBump
[-help]
-location x y
-width value
```

Reports two legal widths for a 45-degree wire to connect to the 45-degree side of an octagon-shaped bump in the wire editor.

Parameters

`-help`	Prints out the command usage.
`-location x y`	Specifies the location inside the bump near the (45-degree) side of the connection (point, required).
`-width value`	Sets the initial width value of the 45-degree wire in microns.

Example

- The following command reports two legal widths for a 45-degree wire to connect to the 45-degree side of an octagon-shaped bump at the specified location with the specified initial width:

  ```
  reportLegalWireWidthForBump -location 681.83 777.28 -width 9.25
  ```

  The reported values are:

  ```
  9.230000 9.260000
  ```

Related Information

- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

selectSecondaryPGNet

```
selectSecondaryPGNet
[-help]
-insts inst+
{-net net_name | -pin pin_name}
```

Selects Power/Ground (P/G) net wire segments from specified instances. The wire segments are selected from the instance P/G pins to stripes (swire targets). This provides an easy way to select and delete routing from specific instance P/G pins to stripes.

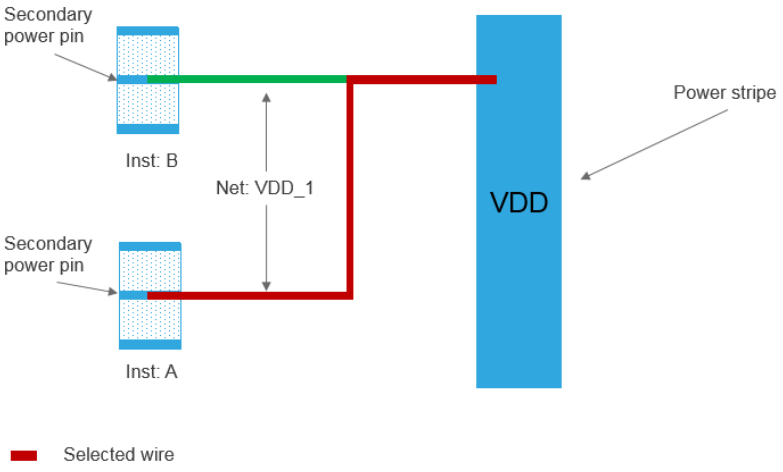
Note: This command does not support dangling secondary P/G nets.

Parameters

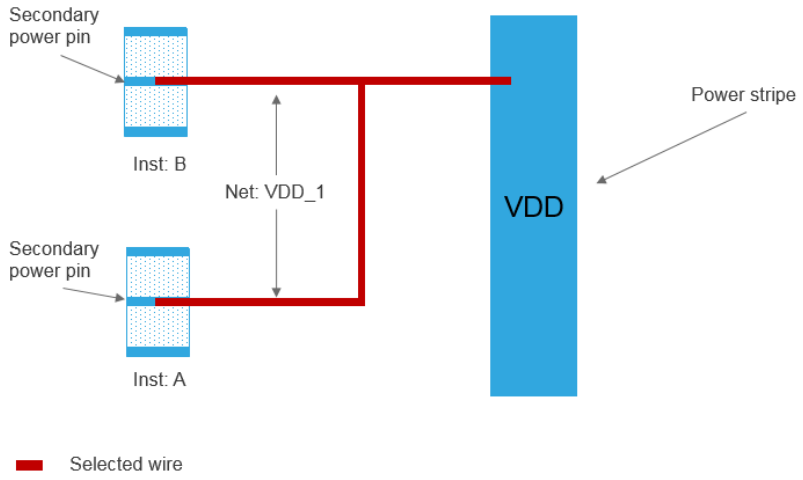
`-help`	Prints the usage of the command. For a detailed description of the command, use the `man` command: `man selectSecondaryPGNet`
`-insts inst+`	Specifies the list of instances that have a secondary PG pin. Wildcards are supported in instance names. If one or more specified instances does not connect with the specified net, the tool displays a warning and ignores those instances.
`-net net_name`	Specifies the net name. This parameter cannot be used with `-pin`. **Note**: *net_name* does not take any wildcard values.
`-pin pin_name`	Specifies the pin name. This parameter cannot be used with `-net`.

Examples

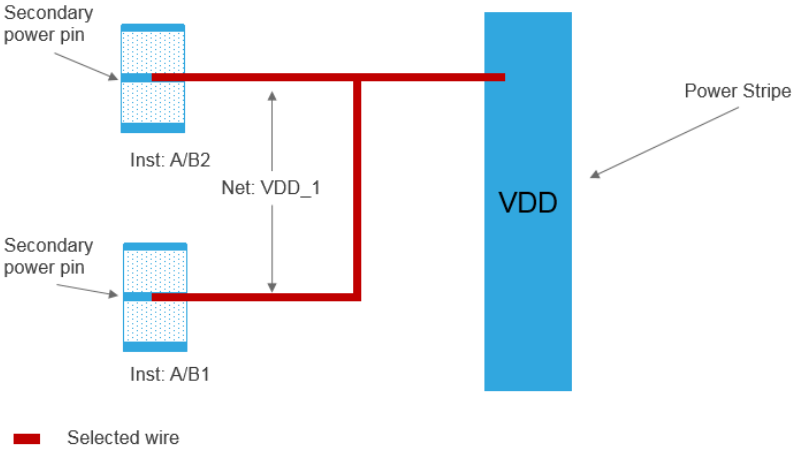
- The following command selects the routing from the P/G pin in instance `A` of net `VDD_1` to the power stripe:
  ```
  selectSecondaryPGNet -net VDD_1 -insts {A}
  ```



- The following command selects the routing from the P/G pins in instances `A` and `B` in net `VDD_1` to the power stripe:
  ```
  selectSecondaryPGNet -net VDD_1 -insts {A B}
  ```



- The following command selects the routing from the P/G pins in instances `A/B1` and A/B2 in net `VDD_1` to the power stripe:
  ```
  selectSecondaryPGNet -net VDD_1 -insts {A/*}
  ```



- The following command selects secondary PG net through the `VDDO` pin and checks all its ports:
  ```
  selectSecondaryPGNet -insts $instn -pin VDDO
  ```
  If a port of the the `VDDO` pin has an open, the tool prints a message as follows:
  ```
  **WARN: (IMPSPR-405): There is no route for secondary pin "VDDO" to
  inst "PD_CHIPLET2_PADS_PD_CHIPPADS_pads_out_balanced_ls_padring_chiplet2_u_DISP_LSYNC_C_UPF_LS" at '3.085500 1.902500'.
  ```

Related Information

- `deleteSecondaryPGNet`
- `deselectSecondaryPGNet`
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

setEditMode

```
setEditMode
[-help]
[-align {true|false}]
[-allow_45_degree {true|false}]
[-arrow_increment value ]
[-assign_multi_pattern_color {auto|mask1|mask2|mask3}]
[-auto_distribute_bus {true|false}
[-auto_split_bus {true|false}]
[-bus_honor_start_params {true|false}]
[-bus_honor_width_setting {true|false}]
[-bus_total_width_horizontal value]
[-bus_total_width_vertical value]
[-change_order_at_turn {Reverse Order | Keep Order}]
[-check_design_boundary {true|false}]
[-circle_NDR_vias_only {true|false}]
[-close_polygons {true|false}]
[-color_align_with_track {true|false}]
[-connect_pin {in|out|inout|nodir}]
[-connect_with_specified_layer {true|false}]
[-create_crossover_vias {true|false}]
[-create_is_edit_flag {true|false}]
[-create_via_on_pin {true|false}]
[-cut_class class_name]
[-cut_wire_overlap {regular|special}]
[-debug_file output_file]
[-delete_pin_with_wire {true|false}]
[-delete_tsv {true|false}]
[-delete_wire_via_deep_through {true|false}]
[-display_wire_length_with_cursor {true|false}]
[-draw_shield {true|false}]
[-draw_with_group_centerline {true|false}]
[-drawing_wire value ]
[-drc_aware_cross_metal {true|false}]
[-drc_on {true|false}]
[-drc_use_non_default_spacing {true|false}]
[-extend_wires {start|end|start_cell_boundary|end_cell_boundary}]
[-final_check_with_verify {true|false}]
[-ignore_drc {conn_antenna|max_via_stack|min_cut|min_enclosed_area|min_step|protrusion}]
[-jog_connect_layer value]
[-keep_floating_via {true|false}]
[-keep_status {true|false}]
[-keep_via {true|false}]
[-lateral_movement_range value]
[-layer name]
[-layer_horizontal name ]
[-layer_maximum layerName]
[-layer_minimum layerName]
[-layer_vertical name ]
[-look_down_layers integer]
[-look_up_layers integer]
[-max_pointer_number integer]
[-mirror_bus_route {true|false}]
[-nets nets]
[-no_merge_special_wire {true|false}]
[-only_show_edit_layer {true|false}]
[-orthogonal_connection_only {true|false}]
[-outer_shield_spacing value ]
[-outer_shield_width value ]
[-override {true|false}]
[-partial_overlap_threshold value]
[-pull_back_distance string]
[-reset]
[-reshape {true|false}]
[-return_object_pointer {true|false}]
[-rule rule ]
[-shape {RING|STRIPE|FOLLOWPIN|IOWIRE|COREWIRE|BLOCKWIRE|PADRING|BLOCKRING|FILLWIRE|FILLWIREOPC|DRCFILL|None} ]
```

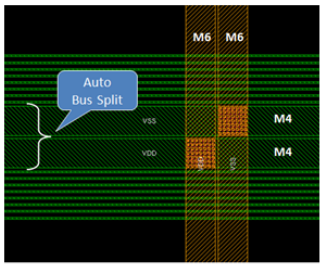
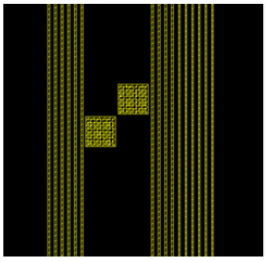
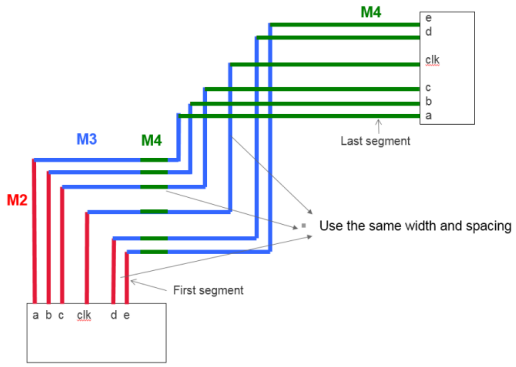
```
[-shield {high|low|adjacent}]
[-shield_look_down_layers integer ]
[-shield_look_up_layers integer ]
[-shield_shape {RING|STRIPE|FOLLOWPIN|IOWIRE|COREWIRE|BLOCKWIRE
    |PADRING|BLOCKRING|FILLWIRE|DRCFILL|None}]
[-shielding_nets nets]
[-show_drc_info_for_edit_shape {true|false}]
[-sibling_look_down_layers integer ]
[-sibling_look_up_layers integer ]
[-snap {true|false}]
[-snap_align_to {center|low|high|auto}]
[-snap_bus_to_pin  {true|false} ]
[-snap_end_to {track_regular|track_special|manufacturing_grid}]
[-snap_objects_to_track {special|regular|patch|pin}]
[-snap_to {special|pg|pin|row}]
[-snap_to_track_honor_color {true|false}]
[-snap_trim_metal_to_trim_grid {true|false}]
[-spacing value]
[-spacing_horizontal value ]
[-spacing_vertical value ]
[-status {cover|fixed|noshield|routed|shield|auto}]
[-stop_at_drc {true|false}]
[-stretch_end {high | low}]
[-stretch_with_intersect {true|false}]
[-subclass {subclass}]
[-turn_at {center_line|wire_edge}]
[-type {regular|special|patch}]
[-unrestricted_regular_wire_width {true|false}]
[-update_shield_net {true|false}]
[-use_fixVia {true|false}]
[-use_interleaving_wire_group {true|false}]
[-use_wire_group {true|false}]
[-use_wire_group_bits number_of_bits ]
[-use_wire_group_reinforcement {true|false}]
[-use_wire_group_reinforcement_group_via {true|false}]
[-use_wire_group_reinforcement_spacing spacing]
[-use_wire_group_reinforcement_width width]
[-verbose {true|false}]
[-via_allow_geom_drc {true|false}]
[-via_auto_replace {true|false}]
[-via_auto_snap {true|false}]
[-via_cell_name cellname]
[-via_columns value]
[-via_create_by {viacell|parameters}]
[-via_cut_layer cut_layer]
[-via_exclude_spec value]
[-via_override_spec value]
[-via_rows value]
[-via_scale_height value]
[-via_scale_width value]
[-via_snap_honor_color {true|false}]
[-via_snap_to_intersection {true|false}]
[-via_type {auto|regular|special}]
[-width value]
[-width_horizontal value]
[-width_vertical value]
[-wire_override_spec specification]
```

Updates the Edit Route and Edit Via forms, the design display area, and vias that are subsequently created with the `editAddVia` command, with the information specified by the parameters. Use the `getEditMode` command to check the current value of a specific `setEditMode` parameter.

Note: Before running any Wire Editor command, all objects and pre-routed wires/vias must be colorized already.

Parameters

`-help`	Prints a brief description that includes type and default information for each `setEditMode` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man setEditMode`
`-align {true|false}`	Specifies whether or not to support alignment of objects during interactive edits using the GUI. *Default:* `true`
`-allow_45_degree {true|false}`	Allows creation of a 45 degree wire. *Default:* `false`
`-arrow_increment` *value*	Specifies the step increments in microns to move wires with the arrow keys. *Default:* `1.0`.
`-assign_multi_pattern_color {auto|mask1|mask2|mask3}`	
	Specifies how to assign mask color on the double patterning technology (DPT) layer of the wire segment to be crea Select one of the following: • `auto`: Mask color is assigned automatically. • `mask1`: Assigns `mask1` color to the added wire segment. • `mask2`: Assigns `mask2` color to the added wire segment. • `mask3`: Assigns `mask3` color to the added wire segment. *Default*: `auto`
`-auto_distribute_bus {true|false}`	Auto-distributes a group of wires, and sets the same spacing based on option `-bus_total_width_vertical` or `-bus_total_width_horizontal`. If `-auto_distribute_bus` is set to `true`, the tool ignores the `-spacing_horizontal` and `-spacing_vertical` settings. While drawing a bus, the tool auto-sets spacing as: `wire spacing = (total bus width - total wire width)/ (wire number - 1)` If the spacing is less than the minimum spacing, the tool auto-sets it to min spacing. *Default:* `false`

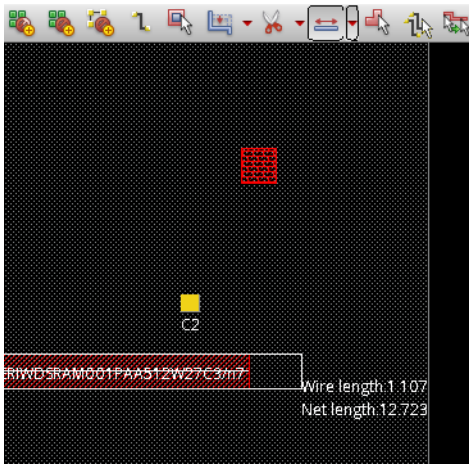
`-auto_split_bus {true|false}`	Specifies whether or not Wire Editor should split buses automatically around power structures to avoid DRC violati  Auto bus split around power trunk  Auto bus split around power vias By default, there is no detour around via array and anchor wires of the bus being drawn always snap to the routing The `-auto_split_bus` parameter also supports busses of special wires with different widths. *Default:* `false`
`-bus_honor_start_params {true|false}`	Specifies that the wire width and spacing of the start points (pins) should be honored when drawing routes for a bus following diagram depicts how the width and spacing of the bus is maintained through turns for the the 2nd to N-1 w segment, if you set `-bus_honor_start_params` to `true`:  *Default*: `false`

`-bus_honor_width_setting {true|false}`	Specifies whether or not the tool should adjust the wire width when snapping bus wires to pin. By default, if `-snap_bus_to_pin` is specified, the tool adjusts the wire width to be the same as the macro or IO pin width. If you do r the wire width to be adjusted to match the pin width, set the `-bus_honor_width_setting` parameter to `true`. In this c tool uses the width value defined by `setEditMode -width_horizontal` or `-width_vertical` when snapping the bus t *Default*: `false` (Pin width is used with `-snap_bus_to_pin`)
`-bus_total_width_horizontal value`	Specifies the total width of a horizontal bus. While drawing a bus, the tool auto-sets spacing as: `wire spacing = (total bus width - total wire width)/ (wire number - 1)` *Default*: `0`
`-bus_total_width_vertical value`	Specifies the total width of a vertical bus. While drawing a bus, the tool auto-sets spacing as: `wire spacing = (total bus width - total wire width)/ (wire number - 1)` *Default*: `0`
`-change_order_at_turn {Reverse Order | Keep Order}`	
	Specifies whether the order of the wires changes or stays the same when the wire makes a 90-degree turn. *Default:* `Keep Order`
`-check_design_boundary {true|false}`	Specifies that wires and vias are to be made inside the design boundary. If `-check_design_boundary` is set to `true`, cannot manually stretch a wire into the area defined by the `-pull_back_distance` parameter. **Note:** 45-degree wires do not follow this rule. *Default*: `false`
`-circle_NDR_vias_only {true|false}`	Specifies that when the *Shift + N* bindkey is used, the Wire Editor should circle through NDR vias only. *Default:* `false`
`-close_polygons {true|false}`	Specifies whether to close a special route structure toward itself, using the `Escape` key. For the closing to complete, ending wire segments must be drawn towards the start wire segments, but do not have to touch them. If you specify parameter, the software ignores the `-extend_wires start` and the `-extend_wires end` settings. *Default:* `false`
`-color_align_with_track {true|false}`	Specifies whether the wire color is to be changed to match the track color during a move or copy operation. If you s `color_align_with_track` parameter to `true` and move or copy a wire to snap it to a track, the track color is automati assigned to the wire. Note that if the wire does not snap to a track or if the wire covers more than one track, this para will not have any effect. This parameter is turned off by default. This means when you move or copy a wire, the wire color is retained by defa *Default:* `false`
`-connect_pin {in|out|inout|nodir}`	Specifies the type of pin on which a via should be generated. You can specify the following pin types with this para • `in` - Generates a via on an input pin. • `out` - Generates a via on an output pin. • `inout` - Generates a via on an inout pin. • `nodir` - Generates a via on a pin for which no direction is specified. *Default*: `in out inout nodir`
`-connect_with_specified_layer {true|false}`	
	Specifies whether or not the layer of connecting wires should be changed as per the `-layer_horizontal` or `-layer_` setting when moving wires with the `editMove` command. If you set this parameter to `true`, `editMove` honors the `setE` `-layer_horizontal` and `-layer_vertical` settings while jogging metal layers. For instance, suppose you cut a verti on `M4` by using `editCutWire` such that the wire is divided into three wire segments. Now, if you select the wire segm middle and move it in the horizontal direction, the tool automatically creates new connecting wires on the layer defi by `setEditMode -layer_horizontal` and drops vias correctly. This feature is applicable for both regular and special *Default:* `false`

`-create_crossover_vias {true|false}`	Specifies whether the software creates a via when you draw a wire that crosses a wire or pin of the same net that is different layer. Does not create vias if minimum or maximum layer constraints prevent the creation of vias. If you dra between two pins, the software creates vias on the pins as needed, regardless of whether this option is selected or deselected. If you specify `false` for this parameter, the software does not create vias at crossovers. *Default:* `true`

`-create_is_edit_flag {true|false}`	Specifies whether to create the db property `isEdit` for wire edit. This property is used to flag nets with modified wire vias. This is useful when evaluating how many objects need to be modified for fixing DRC violations and then deci whether to fix the violations manually or through ECO routing. This property is written to the DEF file.
	By default, this parameter is set to `false`. If you set the value of this parameter to `true`, the `isEdit` property is set to with modified wires or vias. The following type of modifications can cause `isEdit` to be set to `1`:
	• Drawing a new wire through `editAddRoute` and `editCommitRoute`.
	• Moving a wire with/via `editMove`.
	• Stretching a wire/via with `editResize`.
	`dbGet` can then be used to flag nets with modified wires/vias:
	`> dbGet $wire/via_pointer.net.isEdit`
	`> 1`
	Value `1` means the wire/via on the net has been modified.
	The `isEdit` property can be reset using the `dbSet` command as follows:
	`dbSet <net>.isEdit 0`
	Note: The `isEdit` property is independent of `setEditMode -keep_status true`.
	Default: `false`

`-create_via_on_pin {true|false}`	Specifies whether the software creates vias at pins. Specify `false` to prevent the software from creating vias at pins of `false` might be needed for bump pins that are not square-shaped. *Default:* `true`
`-cut_class class_name`	Specifies the cut class name of the vias. *Default:* `""`
`-cut_wire_overlap {regular|special}`	
	Creates overlaps while cutting specified type of wires. The supported values are: • `regular` - Cuts regular wires with an overlap • `special` - Cuts special wires with an overlap If this parameter is not specified, all wires are cut without overlap. This is equivalent to setting `-cut_wire_overlap` t *Default*: `regular`, which means that regular wires are cut with an overlap while special wires are cut without an ove
`-debug_file output_file`	Specifies the file name for reporting untrimmed wires and vias if `setEditMode -verbose` is set to `true`. If you don't sp name but `setEditMode -verbose` is set to `true`, the file is named `editTrim_debug.rpt`. If the tool finds the same file i saved directory, the new file replaces the old one.
`-delete_pin_with_wire {true|false}`	Deletes the pin with the wire. *Default:* `true`

`-delete_tsv {true|false}`	Specifies whether or not `editDelete` deletes the items on a Through-Silicon Via (TSV) layer. • `true` - The tool deletes the items on TSV layers if `editDelete` is used without `-layer` or `-selected`. • `false` - The tool does not delete the items on TSV layers if `editDelete` is used without `-layer` or `-selected`. *Default*: `false` The `editDelete` command observes the layer range setting specified by `setEditMode -layer_minimum value -layer_maximum value`, where `-layer_minimum` and `-layer_maximum` specify the minimum and maximum routing laye respectively. If `-layer` is specified with `editDelete`, the tool ignores the range setting from `setEditMode` and deletes only from the specified layers. If `-selected` is specified with `editDelete`, the tool deletes only the selected wires and
`-delete_wire_via_deep_through {true|false}`	Deletes wire via in the deep through mode when set to `true`. For instance, assume you have two vertical wires `M3` and `M5` at the same location. The tool will not drop vias betwee two wires when you draw `M5` because both the wires have the same direction. However, when you draw the horizor wire, the tool will drop the vias from `M3` to `M6`. All the three wires have the same net. Now, if you delete the M6 wire, t setting for `-delete_wire_via_deep_through` determines which vias are removed. If `-delete_wire_via_deep_through true`, the tool deletes the via from `M3` to `M6`. However, if it is set to `false`, the tool only deletes the via from `M5` to `M6`. *Default*: `false`
`-display_wire_length_with_cursor {true|false}`	
	Specifies whether or not to display the total and current wire length while editing wires by using the mouse. When t parameter is set to `true`, the tool displays the total and current wire length during the following wire edit operations: • Adding a new regular, special, or patch wire • Stretching the length of a regular or special wire • Resizing a patch wire  *Default*: `true`

`-draw_shield {true|false}`	Specifies whether the wire being created is a shield wire. When `-draw_shield` is set to `true`, the wire editor creates a wire with the net name specified using the `-shielding_` option. The newly created wire/via has its status set to `shield` and its shield signal net set to the net name specifiec `-nets` option. Note that you can specify only one net with the `-nets` option when drawing a shield wire. The new sh wire/via also has its shape set to `None`. The shield wire defaults to the minimum width and spacing, unless explicitly other values. Note that NanoRoute supports only minimum width and minimum spacing. When `-draw_shield` is set to `false`, the wire editor reverts to its original behavior and draws a wire/via with the net s using the `-nets` option. To learn how to use the `-draw_shield` parameter, see the Examples section. **Notes**: • When `-draw_shield` is set to `true`, the following `setEditMode` settings are ignored: ◦ `-shield` ◦ `-shield_shape` *Default*: `false`
`-draw_with_group_centerline {true|false}`	
	Draws the bus along the center-line of whole wires when set to `true`. Enable this option if you set `-auto_distribute_bus` to `true`. If `-draw_with_group_centerline` is set to `true`, the tool ignores the `-drawing_wire` setting. *Default*: `false`
`-drawing_wire value`	Specifies which of the nets used with the `-nets` parameter corresponds to the mouse pointer location when adding of wires. For horizontal wires, a value of `1` indicates the bottom-most net. For vertical wires, a value of `1` indicates th most net. The *`value`* is an integer of the same or lower value as the number of nets. For example, if you specify 5 ne can specify `1`, `2`, `3`, `4`, or `5` as the value for this parameter. This net is also used as the reference net when changing direction. All other nets around this net are lengthened or shortened accordingly. *Default:* `1`
`-drc_aware_cross_metal {true|false}`	
	Specifies whether or not to stretch or move a wire back to the closest location available before a violation occurs wi crossing metal. This option works only if `setEditMode -stop_at_drc` is set to `true`: • If both `-stop_at_drc` and `-drc_aware_cross_metal` are set to`true`, the tool stops moving or stretching the wire at closest location available before a spacing violation occurs with the crossing metal. • If `-stop_at_drc` is set to `true` but `-drc_aware_cross_metal` is set to `false`, the tool moves or stretches the wire to desired location across the crossing metal even if it results in a spacing DRC violation. This parameter impacts the following type of modifications: • Drawing a new wire through `editAddRoute` and `editCommitRoute` • Moving a wire with/via `editMove` • Stretching a wire/via with `editResize` *Default:* `true`
`-drc_on {true|false}`	Specifies whether design rule checking (DRC) should be performed during wire and via editing. While modifyig a via with `editChangeVia`, the `-drc_on` and `-keep_via` settings are used in combination to decide wh not a via should be replaced if the new via cell results in DRC violations. For more details, see `editChangeVia`. *Default:* `true`
`-drc_use_non_default_spacing {true|false}`	

	Specifies whether non-default spacing defined in the non-default rule (NDR) of the editing net should be used for sp check during wire editing. ***Note*** • By default (`false`), the Wire Editor DRC check honors the default/minimum spacing. • If the net or via has an NDR with the `HARDSPACING` keyword, the Wire Editor DRC check honors the non-default s even if `-drc_use_non_default_spacing` is set to `false`. • After `setEditMode -drc_use_non_default_spacing true` is set, all subsequent wire editing commands honor th default spacing during the geometry check on the net/via if the NDR is defined. *Default:* `false`
`-extend_wires {start|end|start_cell_boundary|end_cell_boundary}`	
	Specifies which boundary of the wire segment extends and connects to the first logical target, after completing a sig power route. You can specify the following boundary options with this parameter: • `start` - Specifies that the startpoint of the route segment extends and connects to the first logical target, after con signal or power route. If there is no logical target, the startpoint is not changed. • `end` - Specifies that the endpoint of the route segment extends and connects to the first logical target, after compl signal or power route. If there is no logical target, the endpoint is not changed. • `start_cell_boundary` - Specifies that the startpoint of the segment always extends to the cell boundary, after con a special route. If the segment is extended to the boundary and crosses the wires on the same net, the tool make connection to the wires. In addition, the tool creates pins at the boundary. This parameter can only be used in co with `-extend_wires start`. • `end_cell_boundary` - Specifies that the endpoint of the segment always extends to the cell boundary, after comp special route. If the segment is extended to the boundary and crosses wires on the same net, the tool makes a vi connection to the wires. In addition, the tool creates pins at the boundary. This parameter can only be used in co with `-extend_wires end`. **Note**: If you do not want the wire segment to be extended, set `-extend_wires` to `""`. *Default*: `""`
`-final_check_with_verify {true|false}`	Specifies whether to do final checks with verify. When `-final_check_with_verify` is set to `true`, Wire Editor calls the Full Geometry Checker (FGC) engine for the DRC check on the bounding box of changed area after commits. This ensures that the final wire editing result is DF as per FGC. Specify this parameter if you are using advanced rules, such as 20nm and 14nm. *Default*: `false`
`-ignore_drc {conn_antenna|max_via_stack|min_cut|min_enclosed_area|min_step|protrusion}`	
	Specifies which violation type should be ignored during wire editing. You can specify the following options: • `conn_antenna` - Does not check the connection antenna for DRC violations. • `max_via_stack` - Does not check maximum via stack DRC violations during wire editing. • `min_cut` - Does not check for minimum cut DRC during wire editing. • `min_enclosed_area` - Does not perform minimum enclosed area DRC during wire editing. • `min_step` - Does not perform minimum step DRC during wire editing. • `protrusion` - Does not perform protrusion DRC during wire editing. **Note**: If you do not want any violation to be ignored, set `-ignore_drc` to `""`. *Default*: `""`

`-jog_connect_layer` *value*	Specifies the number of layers by which the new jog wire segments created on moving a wire should be moved up Note that the `-jog_connect_layer` parameter will have no impact if you have set the `setEditMode -connect_with_specified_layer` parameter to `true`. To learn how to use the `-jog_connect_layer` parameter, see the Examples section. *Default*: 0
`-keep_floating_via {true|false}`	Controls whether or not floating vias are retained when wires are deleted. By default, floating vias are deleted wher delete a wire. Set the `-keep_floating_via` parameter to `true` if you want to retain floating vias when you delete wire *Default:* `false`

`-keep_status {true|false}`

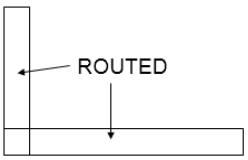
Specifies whether or not object status should be changed during editing. This option is set to `true` by default, which wire status is not changed after editing. If this option is set to `false`, the wire status is changed to `FIXED` automatically `editMove` and `editStretch`.

Notes:

This option has impact on both regular and special wires and vias. The tool only changes the status of the wire seg is modified.

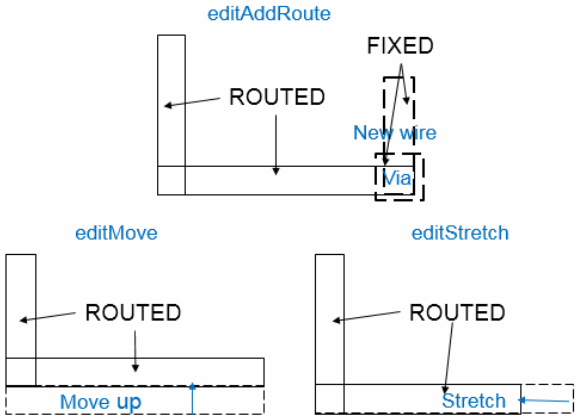
Use the `setEditMode -status fixed` command to set the status of newly drawn wires to `FIXED` after `editAddRoute`.

Consider the wire segment shown in the figure below:



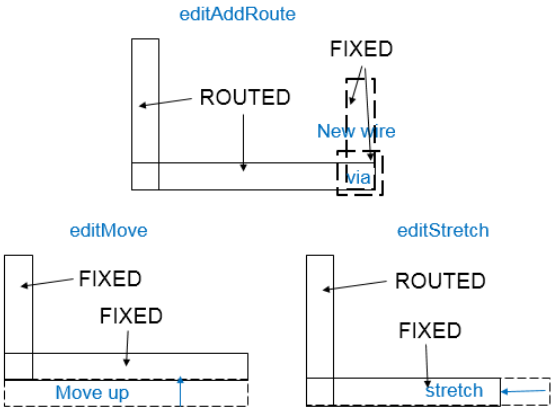
- The following command specifies that the status of the modified wire segment should be retained after `editMove` `editStretch`:

```
setEditMode -status FIXED -keep_status true
```



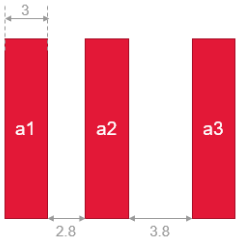
- The following command specifies that the status of the modified wire segment should be changed to `FIXED` after and `editStretch`:

```
setEditMode -status FIXED -keep_status false
```



Default: `true`

`-keep_via {true|false}`	Controls whether or not the original via is retained in a wire move or stretch. If this option is set to `true`, the tool keep unchanged if there is no DRC violation as a result of the move or stretch. However, the tool can change the via if a violation occurs. Set the `-keep_via` parameter to `false` if you do not want the original via to be retained when a wire moved or stretched. This parameter is applicable for both regular and special vias. While modifying a via with `editChangeVia`, the `-keep_via` and `-drc_on` settings are used in combination to decide w not a via should be replaced if the new via cell results in DRC violations. For more details, see `editChangeVia`. *Default:* `false`
`-lateral_movement_range value`	Specifies the maximum distance to which the wire you are adding or stretching can be moved laterally to avoid pote DRC violations with existing wires, nets, or blockages. The preferred lateral movement direction is up for horizontal and left for vertical wires, provided this movement does not result in a new violation. If a violation cannot be avoided even by moving the new wire within the specified lateral movement range, the tool other specified wire edit settings. For example, if the `-stop_at_drc 1` setting is specified, the tool moves or stretche wire up till the closest location available before the DRC violation occurs. **Notes**: • The `-lateral_movement_range` parameter works only if `setEditMode -drc_on` is set to `1`. If `-drc_on` is set to `0`, the ignores the `-lateral_movement_range` setting. • The `-lateral_movement_range` parameter can support multiple nets. • The `-lateral_movement_range` parameter is relevant only for avoiding violations with wires and vias. If the wire b added crosses a routing blockage or cell OBS, the `-lateral_movement_range` setting is ignored. See the Examples section below to understand how the `-lateral_movement_range` parameter works. *Default*: `0.0`
`-layer name`	Specifies the layer for patch or polygon wires. *Default*: `M1` (the first routing layer)
`-layer_horizontal name`	Specifies the layer for horizontal wires. This option supports backside layers as well. *Default*: `M1` (the first routing layer)
`-layer_maximum layerName`	Specifies the maximum layer for routing. The tool does not create wires above the specified layer. This option supp backside layers as well. *Default*: UNINITIALIZED
`-layer_minimum layerName`	Specifies the minimum layer for routing. The tool does not create wires below the specified layer. This option suppo backside layers as well. *Default*: UNINITIALIZED
`-layer_vertical name`	Specifies the layer for vertical wires. This option supports backside layers as well. *Default*: `M2` (the second routing layer)
`-look_down_layers integer`	Specifies the number of layers below the current layer that added wires will connect to with a via. *Default:* `100` (all layers)
`-look_up_layers integer`	Specifies the number of layers above the current layer that added wires will connect to with a via. *Default:* `100` (all layers)
`-max_pointer_number integer`	Specifies the maximum number of pointers to be returned by wire edit commands. Use this parameter along with `-return_object_pointer` to control object pointers returned by wire edit commands. The returned pointers make it po for you to select and further edit the newly created or modified wire or via. *Default:* `100`

`-mirror_bus_route {true|false}`	Specifies whether or not to reverse the net order for the bus route. When this option is specified, the definition of the override spec is reversed. For example, suppose you have drawn a bus route with the following setting:
	`setEditMode -nets {a1 a2 a3} -override -wire_override_spec {{1 3 2.8} {2 3 3.8} {3 3 4.8}}`
	The bus is drawn as follows, with nets counted from left to right:
	 
	Now, by setting `-mirror_bus_route` to `true`, you can draw a mirror bus route with nets counted from right to left:
	`setEditMode -nets {b1 b2 b3} -override -wire_override_spec {{1 3 2.8} {2 3 3.8} {3 3 4.8}} -mirror_bu true`
	
	Default: `false`

`-nets nets`	Specifies one or more nets for editing. If you specify multiple nets, separate each net name with a space and enclos names in curly braces. You can use the following format to specify a bus: *`netName`*`[x:y]` For example, to draw eight wires for a bus, type the following: `-nets` *`netName`*`[7:0]` This is equivalent to specifying *`netName`*`[7], `*`netName`*`[6], ... `*`netName`*`[0].` Specifying `-nets` *`netName`*`[0:7]` reverses the order. The order in which you specify the nets determines the drawing arrangement of the wires. • For horizontally drawn wires, the first net specified is drawn as the bottom-most wire, followed by the next net on side, and so on. The last net specified is drawn as the top-most wire. • For vertically drawn wires, the first net specified is drawn as the left-most wire, followed by the next net, and so c last net specified is drawn as the right-most wire. *Default*: `""`
`-no_merge_special_wire {true|false}`	Specifies whether or not a wire being added should be merged automatically with any existing wire at that point. Se parameter to `true`, if you want to disable the auto merge feature. *Default*: `false`
`-only_show_edit_layer {true|false}`	

	Specifies that only the layer with the objects being edited should be shown and other layers and objects should be out during the following wire/via edit operations: • Add • Resize • Copy/Move *Default:* `true`
`-orthogonal_connection_only {true|false}`	Specifies whether to consider wires and pins that are in the same direction as a target for connection. For example, extend a wire to the left, any horizontal wire it meets is not considered for a target if the value of this parameter is se However, a vertical wire is considered a target. *Default:* `true`
`-outer_shield_spacing` *value*	Specifies a spacing value in microns to be used by the `-shield low` and `-shield high` parameter settings. *Default:* `0` (minimum spacing on the layer)
`-outer_shield_width` *value*	Specifies a width value in microns to be used for outer shield(s) depending on the values of `-shield low` or `-shiel` parameter settings. *Default:* `0` (minimum wire width on the layer) **Note:** You must define the outer shield first. The outer shield is the shielding wire beyond the first or the last wire be created manually.
`-override {true|false}`	Specifies whether to use the override specification. *Default:* `false`

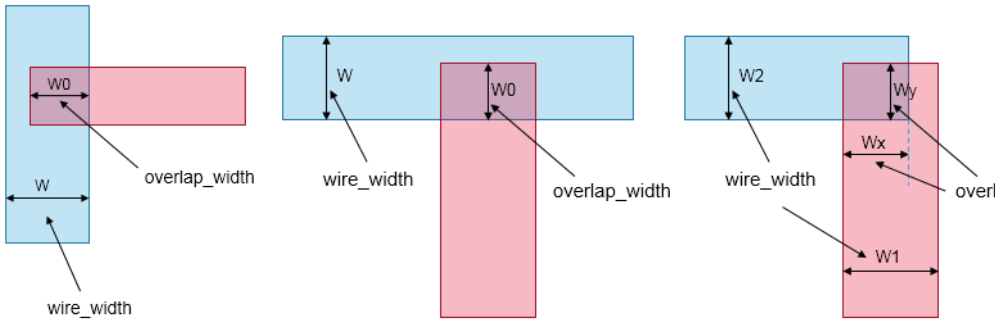
`-partial_overlap_threshold value`

Specifies the ratio threshold for special via creation on partially overlapping wires. The threshold value may range f `100`, with a value of `100` specifying that no via is dropped on a partial overlap intersection. The tool drops a via only `overlap_width` to `wire_width` ratio percentage is greater than or equal to the specified threshold value.

For example, suppose the threshold value is set to `100`. Now consider the following cases in which the pink wire is drawn over the existing blue wire:

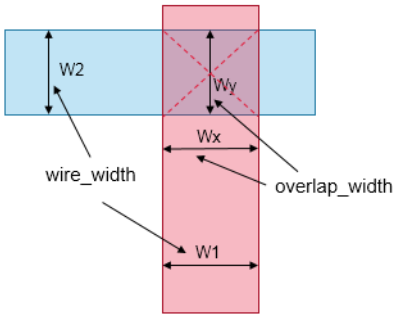
Case 1 - Overlap width less than wire width

In each of the cases depicted below, the `overlap_width` is less than the corresponding `wire_width`. As the `overlap_ wire_width` ratio percentage is less than `100`, which is the specified threshold value, the tool does not drop a via on partial overlap area:



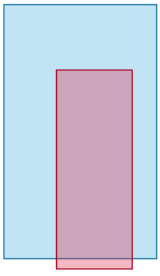
Case 2 - Overlap width equal to wire width

In the following case, the `overlap_width` is equal to the corresponding `wire_width`. As the `overlap_width` to `wire_w` percentage is equal to `100`, which is the specified threshold value, the tool drops a via on the overlap area.



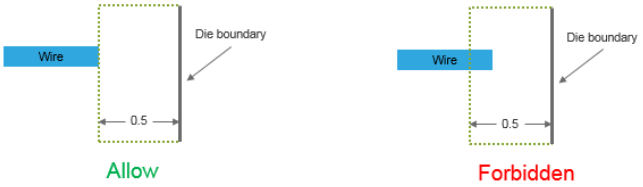
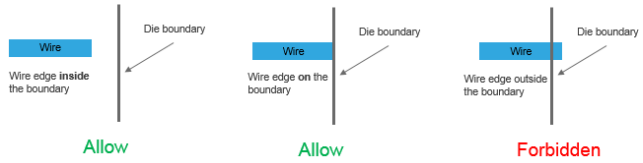
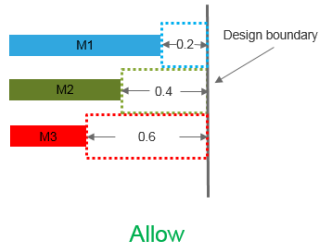
Notes:

- The `-partial_overlap_threshold` parameter is applicable only if the new wire is crossing an existing wire in a c direction. If the new wire being drawn is overlapping an existing wire in the same direction, the tool does not cre



- If the `-create_crossover_vias` parameter is set to `false`, the tool ignores the setting of the `-partial_overlap_th` parameter and does not create any via on the overlap area.

Default: `0`

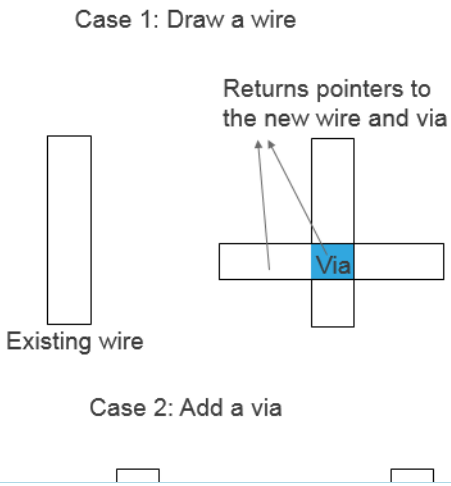
`-pull_back_distance` *string*	Specifies the value for forbidden distance. If the `setEditMode -check_design_boundary` parameter is set to `1`, the too into account the settings of the `-pull_back_distance` parameter during wire edit operations.
	If the `pull_back_distance` value is positive, the tool marks the corresponding area inside the boundary as an extra l area. For example, if you specify the following command, the tool marks 0.5 distance inside the die boundary on all as an extra keep-out area and does not allow you to stretch a wire into this area. You are allowed to stretch the wire till the forbidden line.
	`setEditMode -check_design_boundary true -pull_back_distance 0.5`
	 
	If the `pull_back_distance` value is `0`, the tool does not allow you to manually stretch a wire beyond the die boundar try to stretch the wire edge beyond the boundary, the tool stops the wire edge at the boundary and prints a warning.
	`setEditMode -check_design_boundary true -pull_back_distance 0`
	 
	If the edge of a wire is already beyond the boundary (with `setEditMode -check_design_boundary true -pull_back_distance 0`) and you attempt to stretch it further, the wire remains in the current location and the tool doe allow further stretching.
	If you stretch multiple wires with `-pull_back_distance` set to `Auto`, the tool uses the largest spacing value of each la the pull back distance for related wire. For example, the diagram below illustrates pull back distances if the spacing different layers is set as follows:
	`M1: 0.2  M2: 0.4  M3: 0.6`
	 
	If you do not need the `-pull_back_distance` constraint, set `-check_design_boundary` to `false`.
	Note: 45-degree wires do not follow this rule
	Default: `auto`, which means the tool uses the largest min spacing of the layer as the forbidden distance.

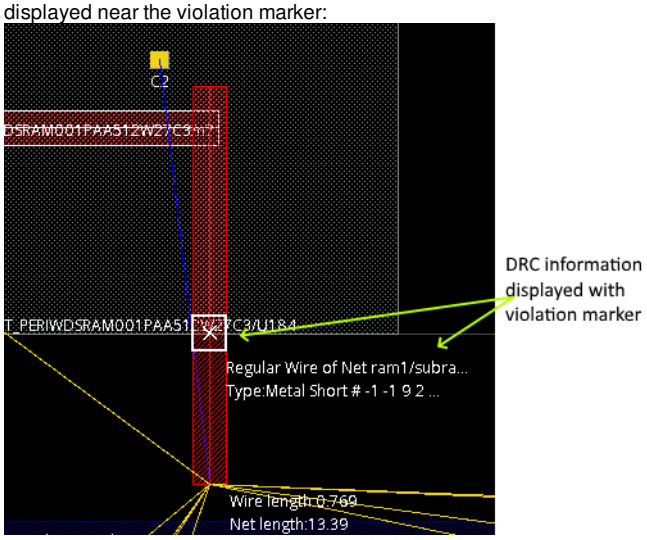
`-reset`	Resets the settings.
`-reshape {true|false}`	Specifies that when you add new wires, existing redundant wires within the route are automatically removed. *Default:* `false`
`-return_object_pointer {true|false}`	

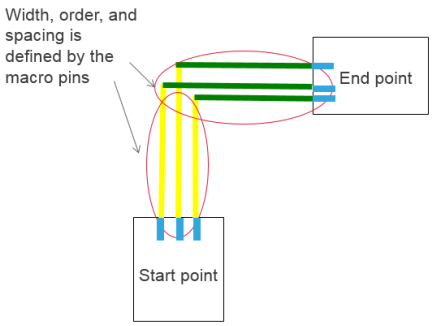
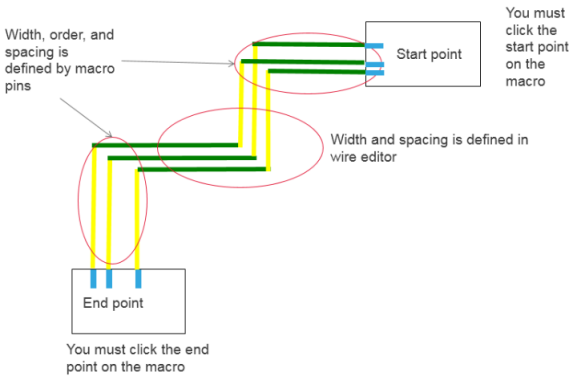
Specifies whether or not pointers are returned after wire edit operations. This parameter is set to `false` by default. W
to `true`, the following wire edit commands return pointers for new or modified wires and vias:

- `editAddVia`
- `editChangeLayer`
- `editChangeVia`
- `editCommitRoute`
- `editCopy`
- `editCutWire`
- `editDuplicate`
- `editMerge`
- `editMove`
- `editResize`
- `editRotate`
- `editSplit`
- `editTrim`

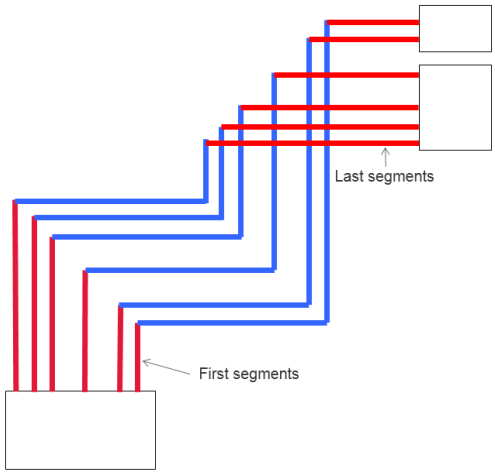
The returned pointers make it possible for you to select and further edit the newly created or modified wire or via. Th
diagrams below illustrate the pointers that are returned after some common wire edit operations:

	
`-rule` *rule*	Uses the specified LEF rule for regular wires. *Default:* `default`
`-shape {RING|STRIPE|FOLLOWPIN|IOWIRE|COREWIRE|BLOCKWIRE|PADRING|BLOCKRING|FILLWIRE|FILLWIREOPC|DRCFILL|None}`	
	Specifies the shape associated with the wire you draw. Use all upper-case letters for the shape. *Default:* `STRIPE`
`-shield {high|low|adjacent}`	Specifies the side to which a minimum width shield wire should be added. Use the following options to specify the • `adjacent` - Specifies that a minimum width shield wire(s) should be added centered between two adjacent wires • `high` - Specifies that minimum width shield wires should be added at the top (High) side of horizontal routes or a side of vertical routes. The spacing of the added shield wire is determined by the value of the `-outer_shield_sp` parameter. If this value is `0`, the spacing is determined by the minimum spacing rule in the technology file. • `low` - Specifies that minimum width shield wires should be added at the bottom (Low) side of horizontal routes or left side of vertical routes. The spacing of the added shield wire is determined by the value of the `-outer_shield` parameter. If this value is `0`, the spacing is determined by the minimum spacing rule in the technology file. **Note**: If you do not want to specify the side to which the minimum width shield wire should be added, set `-shield` to *Default:* `""`
`-shield_look_down_layers` *integer*	Specifies the number of layers below the current layer that an added shield wire can connect using a via. *Default:* `100`
`-shield_look_up_layers` *integer*	Specifies the number of layers above the current layer that an added shield wire can connect using a via. *Default:* `100`
`-shield_shape {RING|STRIPE|FOLLOWPIN|IOWIRE|COREWIRE|BLOCKWIRE|PADRING|BLOCKRING|FILLWIRE|DRCFILL|None}`	
	Specifies the shape associated with the shield wire you draw. Use all upper-case letters for the shape. *Default:* `None`
`-shielding_nets` *nets*	Specifies net names for shield wires, usually power or ground names.
`-show_drc_info_for_edit_shape {true|false}`	

	Specifies that the tool should display the detailed DRC information in a violation marker nearby if a violation is gen during the following wire/via edit operations: • Add • Resize • Move In the example below, the wire being added will cause a Metal Short violation as mentioned in the DRC informatio displayed near the violation marker:  You can address the DRC issues displayed by either: • Cancelling the edit operation causing the DRC violation. • Adjusting the edit operation to meet the DRC requirement. For instance, in the example shown below, the wire b added is causing a Parallel Run Length spacing violation: In this case, you can move the right edge to save spacing. The DRC violation and related information will disap **Note**: If there are any existing violation markers, they are dimmed out so that the latest marker is more clearly visibl *Default*: `true`
`-sibling_look_down_layers` *integer*	Specifies the number of layers below the sibling layers that an added shield wire can connect using a via. *Default:* `1`
`-sibling_look_up_layers` *integer*	Specifies the number of layers above the sibling layers that an added shield wire can connect using a via. *Default:* `1`
`-snap {true|false}`	Specifies whether or not to enable wire and via snap. *Default*: `true`
`-snap_align_to {center|low|high|auto}`	

	Specifies how to align with a pin when ending a route. Select one of the following: • `center`: Snaps wires to the center of the pin. • `low`: Snaps wires to the bottom of vertical pins or to the left side of horizontal pins. • `high`: Snaps wires to the top of vertical pins or to the right side of horizontal pins. • `auto`: The software determines alignment automatically. **Note**: To ensure the proper alignment, do not specify `-snap_objects_to_track` with this parameter. *Default:* `auto`
`-snap_bus_to_pin {true|false}`	Specifies whether or not to change wire width, spacing, and order according to the connected pin. The `-snap_bus_to_pin` parameter supports macro to IO pin snapping as well as IO pin to IO pin snapping. When you set the `-snap_bus_to_pin` option to `true`: • You do not have to align the pins at the start/end macro. Just start drawing the bus from the start macro and draw subsequent wire segments until you arrive at the end macro. When you double-click to end the bus, all the start segments of the bus snap to their respective pins automatically. • The tool gets the pin width and spacing information, and changes the wire width, spacing, and order according t connected pins for the start and end segments of the bus.  • If the bus has multiple corners, the tool uses the width and spacing value defined by the wire edit options for the segments of the bus.  • The tool also changes the wire segment layer according to the pin layer when accessing the pins. If the connect on a single layer, such as M4, the tool changes the wire segment layer to M4 when accessing pins. If the connec is on multiple layers, the tool changes layers as follows: ◦ If the pin is on M2, M3, and M4, but you have defined M5 and M6 for wire edit, the tool uses M4 for the first wi segment. In that way, wire edit has minimum via when connecting the first and second wire segment. ◦ If the pin is on M2, M3, and M4, but you have defined M2 and M3 for wire edit, tool uses M3 for the first wire s when connecting to the pin in the vertical direction (Wire edit sets M2 as the horizontal layer and M3 as the v layer). For middle segments of the bus, the tool still uses the wire edit layer setting. • `snap_bus_to_pin` supports snapping from one to multiple (1->N) macros for bus. When drawing the bus, the end

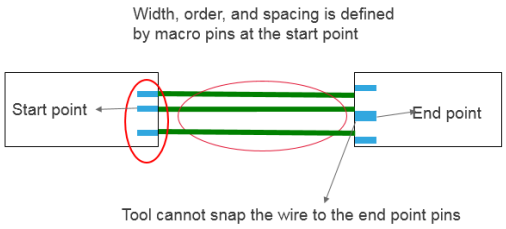
could be the pins on different macros. As long as the end point is inside a macro, the tool is able to auto snap to cell pins. The following diagram depicts how `-snap_bus_to_pin`, when enabled, supports 1->N macros.



- When you click outside the design core area after drawing a bus, the wires of the bus are snapped to the IO pin.

When using the `-snap_bus_to_pin` option, keep the following limitations in mind:

- The bus routing pattern must have at least one corner. Otherwise, the tool cannot snap the bus to both the start a target pins.



- You must click at the macros with target pins for the start and end points of the bus. The tool searches only the s macros for the connected pins.
- For a regular wire, the tool creates an NDR for each pin width automatically (the width is not defined in tech LEF cannot delete the NDR from the database.

Both regular and special wires support this feature.

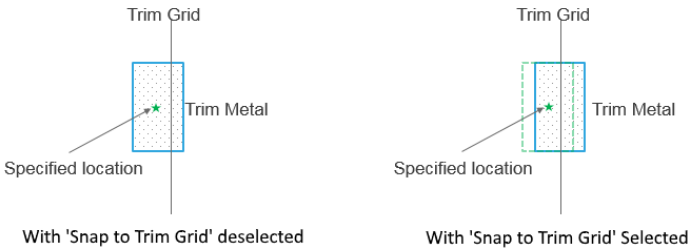
By default, the `-snap_bus_to_pin` option is set to `false`, which means the tool does not snap each wire of the bus to macro pin automatically.

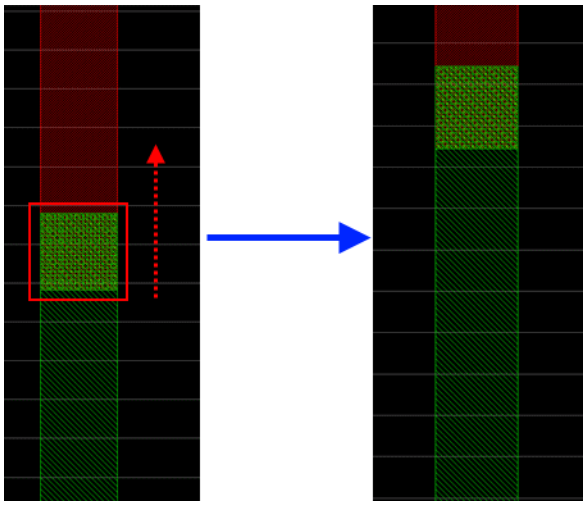
Note: The `-snap_bus_to_pin` option is exclusive to the `setEditMode -snap_to pin` option. When `setEditMode -sna pin` is set, the tool uses the original snapping behavior in which only the first drawing wire is snapped to the pin witl wire width and spacing change.

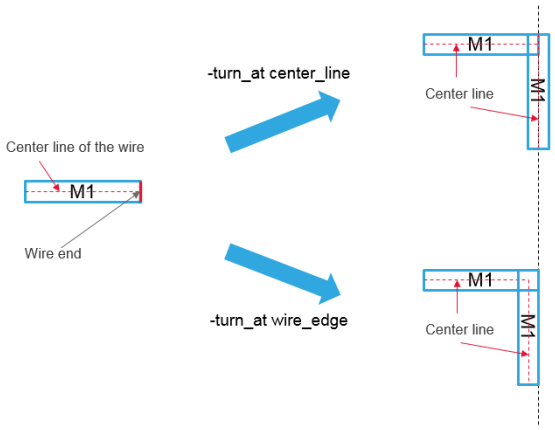
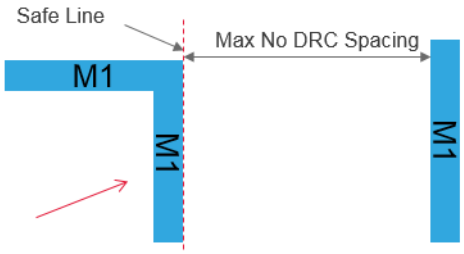
Default: `false`, which means the tool does not snap each wire of the bus to the macro pin automatically.

`-snap_end_to {track_regular|track_special|manufacturing_grid}`

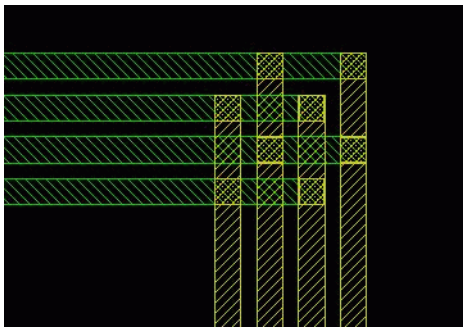
	Snaps the end of the wire being drawn to the specified object. You can specify the following options: • `manufacturing_grid` - Specifies that the end of an off-grid wire should be snapped to the manufacturing grid. • `track_regular` - Snaps the end of regular wires to track. For a long multi-pin net, you might want to draw the trur first and then draw the branch wires along the trunk wire. The start/end points of the trunk wire are usually not or if the trunk wire start/end points do not snap to the track, it will leave an off-grid wire segment. To prevent this, sn end of regular wires to track by setting `setEditMode -snap_end_to track_regular`. The start/end points of the re wire will snap to the closest orthogonal layer preferred track. • `track_special` - Snaps the end of special routes to track. For a long multi-pin net, you might want to draw the tru first and then draw the branch wires along the trunk wire. The start/end points of the trunk wire are usually not or if the trunk wire start/end points do not snap to the track, it will leave an off-grid wire segment. To prevent this, sn end of special routes to track by setting `setEditMode -snap_end_to track_special`. The start/end points of the s route will snap to the closest orthogonal layer preferred track. **Note**: If you do not want to snap the end of the wire to any object, use `setEditMode -snap_end_to ""`. *Default:* `""`
`-snap_objects_to_track {special|regular|patch|pin}`	
	Snaps the center line of wires being added or moved to the closest routing track in the preferred direction for the lay automatically. You can specify the following options: • `special` - Snaps added or moved special wires to the closest routing track in the preferred direction for the layer automatically. This parameter also controls whether to cut special wires on the routing track. **Note**: See `-snap_to pin` for a description of the software's behavior when both `-snap_objects_to_track specia` `snap_to pin` are specified. • `regular` - Snaps added or moved signal wires to the closest routing track in the preferred direction for the layer automatically. This parameter also controls whether to cut signal wires on the routing track. **Note**: See `-snap_to pin` for a description of the software's behavior when both `-snap_objects_to_track regula` `snap_to pin` are specified. • `patch` - Snaps patch wires automatically to the closest routing track for the layer as the wires are drawn, copied, moved. • `pin` - Snaps a moved physical pin to the closest routing track in the preferred direction for the layer automatically **Note**: If you do not want to snap any object to track, use `setEditMode -snap_objects_to_track ""` to disable all tra *Default:* `regular patch`
`-snap_to {special|pg|pin|row}`	

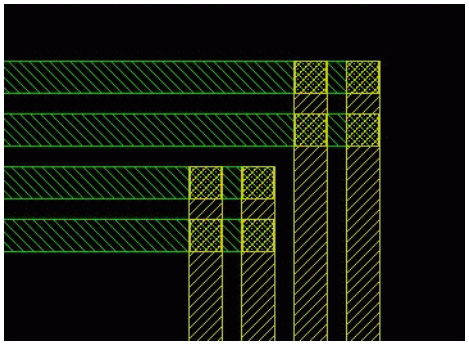
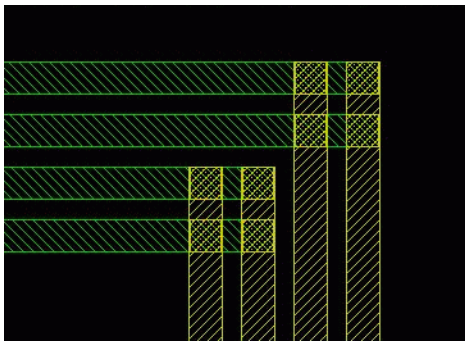
	Snaps the end of the wires being added or moved to the specified object. You can specify the following objects: • `special` - Automatically extends a special wire for maximum via coverage when crossing over a same-net wire c adjacent layer. • `pg` - Snaps new wires whose start point is within a "snap region" to the closest end of a power or ground wire on net. The snap region is one-half the width of the target wire. ◦ The snap region for orthogonal wires forms a square with the width of the power or ground wire. ◦ The snap region for wires at 45-degree angles to the power or ground wire forms a square from the point whe wire will connect. Each side of the square is the width of the power or ground wire. ◦ The snap region for wires that are parallel to power or ground wires is the full power or ground wire. • `pin` - Snaps added or moved wires to a pin automatically. If you double-click at a pin, it snaps the wire to the pin. ensure the wire is snapped to the center of the pin, specify `-snap_objects_to_track special` (or `-snap_objects_to_track regular`) and specify a value for `-snap_align_to`. When you specify both `-snap_objects_to_track special` (or `-snap_objects_to_track regular`) and `-snap_to` software has the following behavior: ◦ If the wire will fall partially outside the pin when snapped to a track, it snaps to the pin. ◦ If the wire will fall inside the pin when snapped to a track, it snaps to both the pin and a track. ◦ If there is no track on the pin, the wire snaps to the pin. • `row` - Snaps added or moved wires to the nearest standard cell row. Use this parameter to create followpins. This parameter supports both horizontal and vertical rows. For example, if you want to snap a wire to a pin automatically, specify `setEditMode -snap_to pin`. **Note**: If you do not want to snap wires to any object, use `setEditMode -snap_to ""` to disable all object snap. *Default:* `special pg pin`
`-snap_to_track_honor_color {true|false}`	Specifies whether to snap wires to a track with the same color automatically. This option is on by default. If you nee move a wire to a track with a different color, set this option to `false`. **Notes:** This is a global setting and can be used to control all the snapping behavior in wire edit commands. This command has no impact on `editAddRoute`. When drawing wires, the tool honors the color setting specified by `setEditMode -assign_multi_pattern_color`. *Default:* true
`-snap_trim_metal_to_trim_grid {true|false}`	
	Specifies whether or not the trim metal should snap to the closest trim grid during trim-metal editing. If you set this p to `false`, the trim metal is drawn centered on the location specified by `editAddTrimMetal -location`.  **Note**: `editMove` does not follow the setting specified by the `setEditMode -snap_trim_metal_to_trim_grid` paramet *Default:* `true`
`-spacing value`	Specifies the distance between wires. *Default*: `""`

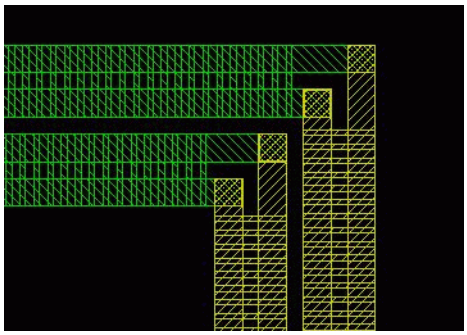
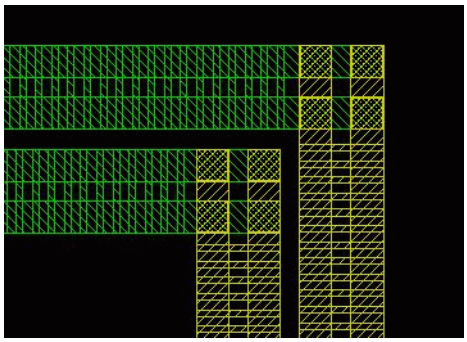
`-spacing_horizontal` *value*	Specifies the distance between horizontal wires. *Default*: `""`
`-spacing_vertical` *value*	Specifies the distance between vertical wires. *Default*: `""`
`-status {cover|fixed|noshield|routed|shield|auto}`	
	Specifies the status associated with the wire you draw. By default, when you add special wires manually using the GUI, the tool creates crossover vias with `fixed` status. U `status` parameter of `setEditMode` to draw wires and vias with the specified status. If the value is specified as `auto`, the tool sets the status of wires drawn on signal and power nets to `routed` and that drawn on clock nets to `fixed`. **Note:** • For a special wire, `noshield` is not supported. If you specify this value for a special wire, the tool gives a warning message and keeps the status as `fixed`. • For a regular wire, `shield` is not supported. If you specify this value for a regular wire, the tool gives a warning m and keeps the status as `fixed`. *Default*: `fixed`
`-stop_at_drc {true|false}`	Checks DRC rules and stops moving or stretching wires before causing a violation. • Specify `true` to stop moving or stretching the wire at the closest location available before a violation occurs. In th you cannot move a wire to the desired destination if that wire passes through a location where a DRC violation o • Specify `false` to ignore DRC rules when moving or stretching wires. The software moves or stretches a wire to t desired location even if it results in a DRC violation. *Default:* `true`
`-stretch_end {high | low}`	Specifies the direction in which to stretch or reduce wires. • `high` - Stretches or reduces wires from the top for vertical wires or from the right for horizontal wires. • `low` -Stretches or reduces wires from the bottom for vertical wires or from the left for horizontal wires. *Default:* `high`
`-stretch_with_intersect {true|false}`	Enables wires with intersects to be stretched easily. If this option is set to `true`, you can select the via and the edge and then stretch the wires and via together in a single step.  *Default*: `false`

`-subclass {subclass}`	Specifies the subclass name. *Default*: `""`
`-turn_at {center_line|wire_edge}`	Specifies the start location of the new wire segment when a wire is turned. This parameter can be used for both reg special wires. You can specify one of the following values for`-turn_at`: • `center_line` - When you turn a wire, the center line of the new wire segment aligns to the end of the old wire seg • `wire_edge` - When you turn a wire, the long edge of the new wire segment aligns to the end of the old wire segm The diagram below depicts the tool behavior for both options:  Use the `wire_edge` option when turning a wire at the *Safe Line* for an adjacent wire. With this option, the tool turns t without extending it beyond the *Safe Line* to avoid a minumum spacing rule violation with the adjacent wire:  You can use the default `-turn_at center_line` option if there is no adjacent wire. *Default*: `center_line`
`-type {regular|special|patch}`	Specifies the type of the wire being created. You can specify one of the following options: • `regular` - Creates regular wires and vias. • `special` - Creates special wires and vias. • `patch` - Creates patch wires. *Default*: `""`
`-unrestricted_regular_wire_width {true|false}`	
	Specifies whether the widening of a regular wire is unrestricted. By default, when you are changing the width of a w snaps to the closest non-default route (NDR) available. If you set the `-unrestricted_regular_wire_width` paramete a wire can be stretched to any width without any restrictions. *Default:* `false`

`-update_shield_net {true|false}`	Specifies whether the shield net attribute on a signal net should be automatically updated. Use this parameter if you Nanoroute to recognize the net as a shield net and not remove it during routing.
	If you set the `setEditMode -update_shield_net` parameter to `true` before drawing a signal net, for example `net11`, th net attribute is automatically set on that signal net. You can verify this by using the `getAttribute` command as follo
	`> getAttribute -net net11`
	The result is:
	`#NET net11 attributes:`
	`# weight = 2 (default)`
	`# non_default_rule = default (default)`
	`# non_default_rule_effort = auto (default)`
	`# shield_net = VDD`
	If you now run `routeDesign`, NanoRoute recognizes the net as a shield net and does not remove it.
	Default: `false`

`-use_fixVia {true|false}`	Specifies whether to use fixVia for special via with minStep DRC violation. **Note**: If a design is under 20nm, the tool turns on fixVia integration by default: `setEditMode -use_fixVia true` To turn off the setting, use `setEditMode -use_fixVia false`. *Default*: `false`
`-use_interleaving_wire_group {true|false}`	Alternates the specified wires in the wire group. For example, if you specify `-nets VDD VSS -use_interleaving_wire true`, the software alternates the power and ground wires in the wire group. When `-use_interleaving_wire_groups` is set to `true`,the software draws a wire group like the following:  *Default:* `false`

`-use_wire_group {true|false}`	Groups multiple wires from the same net. Specify this parameter to decrease resistance and provide stronger conne The following figure shows power and ground wires that form a wire group.  *Default:* `false` (The wire editor does not use wire groups.)
`-use_wire_group_bits` *number_of_bits*	Specifies the number of times to replicate a wire in the wire group. For example, if you specify `VSS` and `VDD` nets, an `2` for this parameter, the software creates a wire group with two power wires and two ground wires. ⓘ The value of this parameter must be greater than `1` to enable wire groups. When `-use_wire_group_bits 2` is specified, the Innovus software draws a wire group like the following:  *Default:* `0` (disabled)

`-use_wire_group_reinforcement {true|false}`	Creates slots in the wire group by creating wires that connect the wires in the wire group, but are orthogonal to then help to reduce resistance in power networks. Specifying both this parameter and `-use_interleaving_wire_group` c shorts in the wire group. When `-use_wire_group_reinforcement true` is specified, the Innovus software draws a wire group like the followir  *Default:* `false`
`-use_wire_group_reinforcement_group_via {true|false}`	
	Specifies that the wire group uses group vias. Group vias are connected vias in a wire group. Using group vias ens the contact is maintained if one of the vias in the wire group produces a void. In the following figure, the wire group has group vias:  *Default:* `false`
`-use_wire_group_reinforcement_spacing` *spacing*	
	Specifies the spacing for the orthogonal wires in the wire groups. *Default:* `0`
`-use_wire_group_reinforcement_width` *width*	
	Specifies the width of the orthogonal wires in the wire groups. *Default:* `0`

`-verbose {true|false}`	Reports untrimmed wires/vias that cannot be trimmed because either they are connected to a patch wire or their sta `cover`. Set this parameter to `true` before running `editTrim` if you want untrimmed wires/vias to be reported. You can additionally use `setEditMode -debug_file` to specify the name for the report. The default report file name is `editTrim_debug.rpt`. **Note**: Only wires and vias that have connectivity issues are reported. *Default:* `false`
`-via_allow_geom_drc {true|false}`	Checks geometry DRC. *Default:* `true`
`-via_auto_replace {true|false}`	Determines whether existing vias are replaced with new ones in case of overlap. • Specify `true` to replace existing vias with the new ones added with `editAddVia` in case of any overlap. • Specify `false` if you want to retain the existing vias even if there is some overlap with the new vias being added. *Default:* `true`
`-via_auto_snap {true|false}`	Controls whether to snap vias to the intersection of tracks or to the manufacturing grid. • Specify `true` to snap the via to the center of the intersection of tracks. • Specify `false` to keep the via in the location where it is added or moved, snapping only to the manufacturing gric *Default:* `true`
`-via_cell_name` *`cellname`*	Specifies the name of the via cell from the LEF or DEF file. *Default:* `""`
`-via_columns` *`integer`*	Specifies the number of columns in a via cut. *Default*: `0`
`-via_create_by {viacell|parameters}`	
	Controls whether the tool creates vias based on the `-via_cell_name` parameter or the cut class name. • Specify `viacell` to create the via with the same geometry as the cell specified by the `-via_cell_name` parameter • Specify `parameters` to create the via cell based on the cut class parameters. *Default:* `parameters`
`-via_cut_layer` *`cut_layer`*	Specifies the via layer name for the via to be created or modified. *Default:* `V12`

`-via_exclude_spec` *`exclude_list`*	Specifies the vias to be ignored by the wire editor. You can define the list of vias in the *`exclude_list`* as follows:
	`setEditMode -via_exclude_spec {{default via0} {rule1 via2} {rule2 {via2 via3}}}`
	For example, the wire editor will ignore the `V1L2LVH` via with the following command:
	`setEditMode -via_exclude_spec {{default V1L2LVH}}`
	Default: `""`

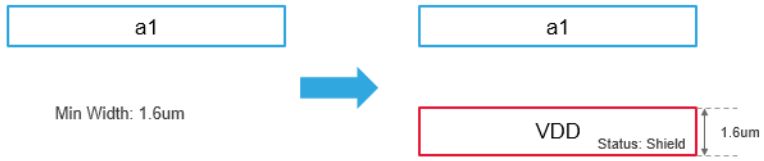
`-via_override_spec` *value*	Specifies the default vias to be used by wire editor. You can use this parameter if you do not want wire editor to sel automatically. `-via_override_spec` allows a regular via on NDR nets. You can define the via in the via spec as follows: `setEditMode -via_override_spec {{default via0} {rule1 via2} {rule2 via1}}` Here, if you draw a wire with NDR name `rule1`, the tool will always drop `via2` in the wire intersection area. The tool check whether `via2` belongs to NDR rule `rule1` in the Tech Lef. You can define the full set of required vias for regular as well as NDR nets in one command with the `-via_override_spec` parameter. For example: `setEditMode -via_override_spec { {default via0 via1 via2 via3 via4 via5 via6 via7 via8} {NDR_rule1 vi via4 via5 via6 via7 NDR_rule1_via2 NDR_rule1_via3 NDR_rule1_via4 NDR_rule1_via5 NDR_rule1_via6 NDR_rule1_via7} {NDR_rule2 via2 via3 via4 via5 via6 via7 NDR_rule1_via2 NDR_rule1_via NDR_rule1_via4 NDR_rule1_via5 NDR_rule2_via2 NDR_rule2_via3 NDR_rule2_via4 NDR_rule1_via5 NDR_rule2_v NDR_rule2_via7} }` This use model reduces the runtime for via selection during Wire Edit. When you perform wire edit operations, via selection happens as follows for the above definition: • For regular wire edit, the tool always picks a via from the `default` list specified with the `-via_override_spec` opti • For editing nets with the `NDR_rule1` rule, the tool picks any via from the list defined under the rule `NDR_rule1` in th above example. • For editing nets with `NDR_rule2`, the tool picks any via from the list defined under the rule `NDR_rule2`. During wire edit for DRC fixing on both regular and NDR wires, the tool always picks a via that is DRC clean after edit operation, from the above definition. To see another example of using the `-via_override_spec` parameter, see the Examples section. *Default: ""*
`-via_rows` *integer*	Specifies the number of rows in a via cut. *Default:* `0`
`-via_scale_height` *value*	Specifies the absolute height of the special vias added when drawing wires. *Default:* `100`
`-via_scale_width` *value*	Specifies the absolute width of the special vias added when drawing wires. *Default:* `100`
`-via_snap_honor_color {true|false}`	
	Specifies whether or not to snap via to the track with the same color automatically. *Default:* `true`
`-via_snap_to_intersection {true|false}`	Specifies whether to snap vias to the intersection of wires. *Default:* `true`
`-via_type {auto|regular|special}`	Specifies the type of the via to be created: • `auto` - Assigns the correct via type based on the type of connected wire. • `regular` - Creates regular vias. • `special` - Creates special vias. For example, if you specify the type as `regular` by using the following command and then run the `editAddVia` comm tool will create a regular via even if there are PG shapes on other layers overlapping the area: `setEditMode -via_type regular` *Default:* `auto`
`-width` *value*	Specifies the width of patch wires. *Default:* `0`

`-width_horizontal` *value*	Specifies the width for horizontal wires. *Default:* `0`
`-width_vertical` *value*	Specifies the width for vertical wires. *Default:* `0`

`-wire_override_spec` *specification*	Contains the details of the override specification. The specification has the following format:
	`{{wire_index width spacing layer ndr_name} {...}}`
	Specify the net numbers, followed by the override width, the override spacing to the next net, layer, or override NDF for the specified nets.
	The net number is the position of the net specified by the `-nets` parameter. If you specify more than one net, separa numbers with commas.
	For example, the following specification indicates that the first and second nets should have a width of 10.0 and a s 3.0:
	`1,2 10.0 3.0`
	Similarly, the following specification indicates that the first and second nets should be reassigned to a new NDR, `2_`
	`1,2 "" "" "" 2_wide`
	Default: ""

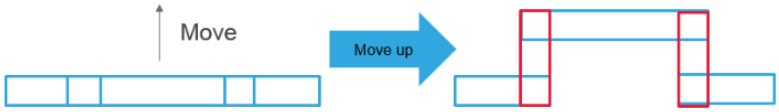
Examples

- If you want to draw a shield wire for the wire a1, run the following command before drawing the wire:
  `setEditMode -nets a1 -shielding_nets VDD -draw_shield true -shape none -width_horizontal 2`



  When `-draw_shield` is set to `true`, the tool automatically sets the status of the new wire to `shield` and its width to the minimum width (1.6um). If you also want to change the shield net attribute for the net, set the `-update_shield_net` parameter to `true` as well before drawing the wire.
- If the current wire is on layer 2, use the following command to drop a multi-layer pin on layer 1/layer 2:
  `setEditMode -multi_layer_pin 1 -multi_layer_via V12`
- The following command uses the `-via_override_spec` parameter to specify via selections for the default rule as well as for the nondefault rule `SP`:
  `setEditMode -via_override_spec {{default via12 via23 via45} {SP spvia12}}`
  As specified in the command, wire editor makes via selections for layers as follows:
  - For the default rule:
    - `via12` for layer `V12`
    - `via23` for layer `V23`
    - `via45` for layer `V45`
  - For the nondefault rule `SP`:
    - `spvia12` for layer `V12`

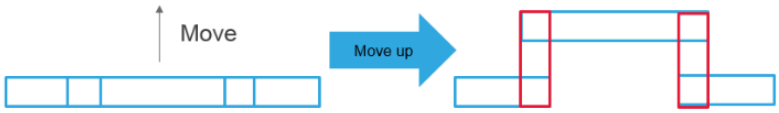
  Note: Wire editor automatically selects best vias for the layers that are not specified with `-via_override_spec`.
- The following example uses the `-jog_connect_layer` parameter to specify that the tool should automatically move up new jog wires created on moving a wire segment by 1 layer:
  `setEditMode -connect_with_specified_layer false -jog_connect_layer 1`



Here, the blue wire segments are on layer `M2` while the red jog wire segments are on the `M3` layer.

- In the following example, as the `-jog_connect_layer` parameter is set to -1, the tool automatically moves down the new jog wires by 1 layer:

```
setEditMode -connect_with_specified_layer false -jog_connect_layer -1
```



In this case, the blue wire segments are on layer M2 while the red jog wire segments are on the M1 layer.

- The following command specifies that the wire being drawn should snap to a pin automatically:

```
setEditMode -snap_to pin
```

- The following command specifies that when a physical pin is moved, it should snap automatically to the closest routing track in the preferred direction for the layer:

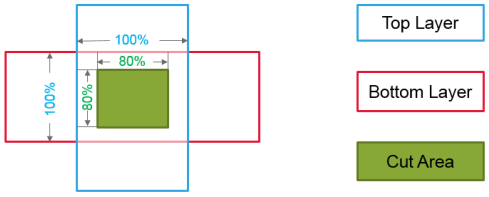
```
setEditMode -snap_objects_to_track pin
```

- The following set of commands add a via with two rows and three columns. The origin of the via is at coordinates `550`, `675`:

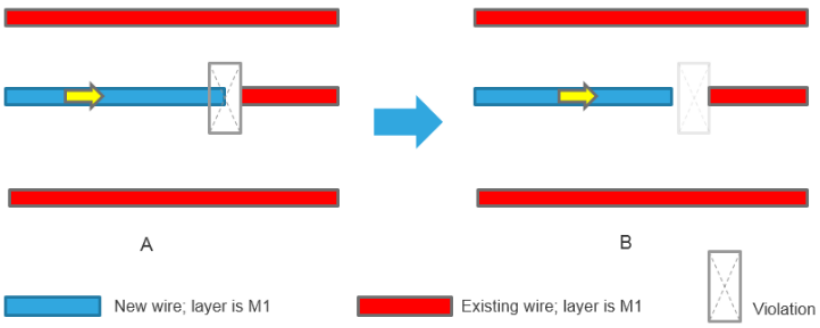
```
setEditMode -via_create_by geometry -via_add_rows 2 -via_add_columns 3
editAddVia 550 675
```

- The following command scales the via added while drawing a wire to 80% height and width:

```
setEditMode -via_scale_height 80 -via_scale_width 80
```

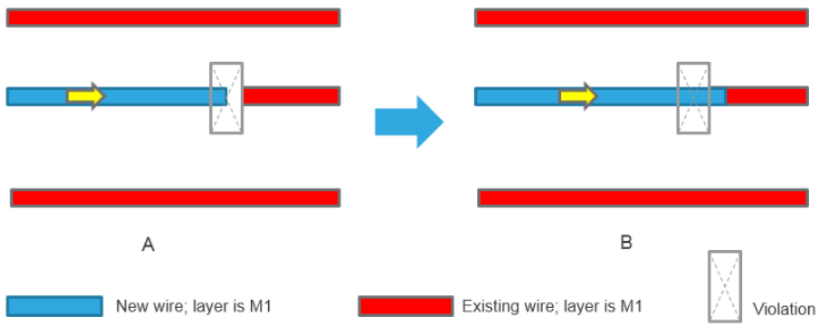


- In the following examples, the blue wire is being moved or stretched from left to right. See how the tool behavior changes with the various `setEditMode` settings given below:
  - `setEditMode -drc_on 1 -lateral_movement_range 0 -stop_at_drc 1`



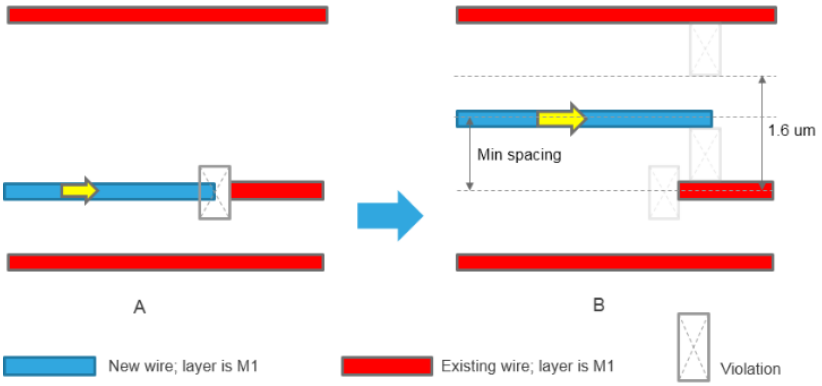
This wire edit will generate a violation with an existing wire (see A), so the tool shrinks the new wire to a distance such that there is no violation (see B).

  - `setEditMode -drc_on 1 -lateral_movement_range 0 -stop_at_drc 0`



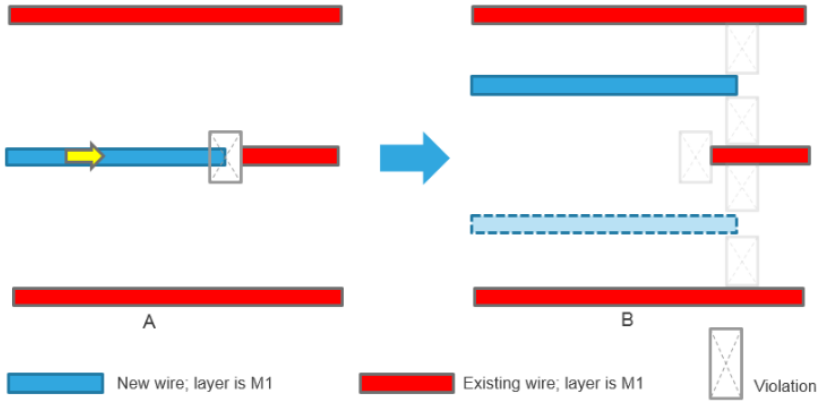
This wire edit will generate a violation with an existing wire (see A). However, the tool allows the new wire to be moved or stretched to any length and reports a violation (see B).

- `setEditMode -drc_on 1 -lateral_movement_range 1.6 -stop_at_drc 1`



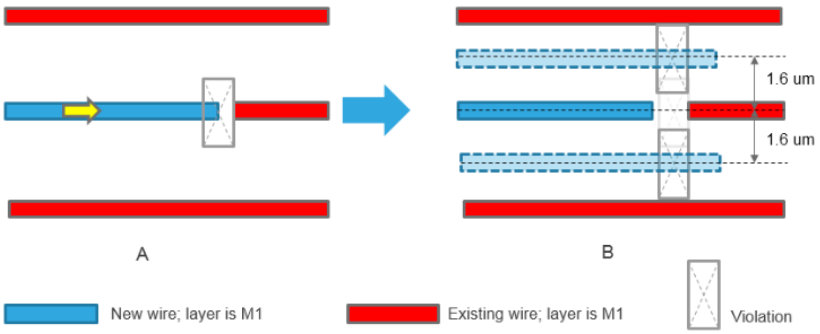
This wire edit will generate a violation with an existing wire (see A). So, the tool moves the new wire up by a minimum distance to avoid generating a violation (see B).

- `setEditMode -drc_on 1 -lateral_movement_range 1.6 -stop_at_drc 1`



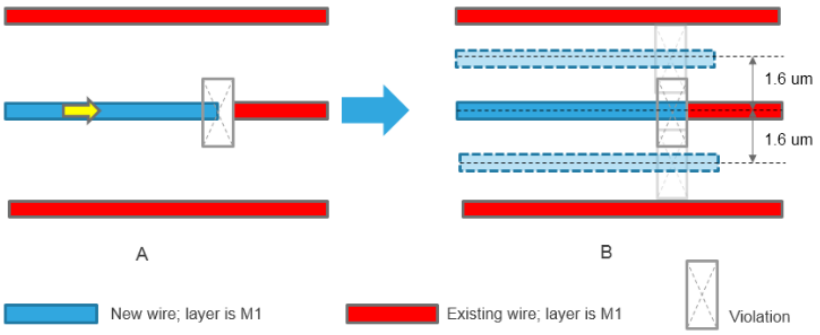
This wire edit will generate a violation with an existing wire (see A). The blue wire can be moved either up or down to avoid the violation. In this case, the tool moves the wire up, which is the preferred direction for horizontal wires (see B).

- `setEditMode -drc_on 1 -lateral_movement_range 1.6 -stop_at_drc 1`



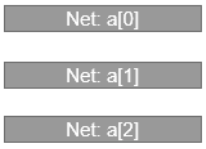
This wire edit will generate a violation with an existing wire (see A). However, even if the blue wire is moved up or down within the specified lateral movement range, the tool still cannot avoid a violation. Therefore, the tool obeys other specified wire edit settings. In this example, the tool obeys the `-stop_at_drc 1` setting and moves or stretches the blue wire up till the closest location available before the DRC violation occurs (see B).

- `setEditMode -drc_on 1 -lateral_movement_range 1.6 -stop_at_drc 0`



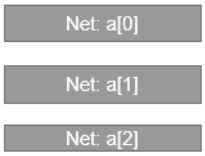
This wire edit will generate a violation with an existing wire (see A). However, even if the blue wire is moved up or down within the specified lateral movement range, the tool still cannot avoid a violation. Therefore, the tool obeys other specified wire edit settings. In this example, the tool obeys the `-stop_at_drc 0` setting and allows you to move or stretch the wire even though it results in a DRC violation. (see B).

- Consider the addition of a bus `a[0] a[1] a[2]`. The NDR name is `default`, and the metal layer name is `M0`.



  - The following command reassigns the first and second nets to a new NDR, `2_wide`:

    ```
    setEditMode -wire_override_spec {1,2 "" "" "" 2_wide}
    ```



  - The following command assigns the `M2` layer to the first wire and the `M4` layer to the third wire.:

    ```
    setEditMode -wire_override_spec {{1 "" M2} {3 "" M4}}
    ```

- The following commands report untrimmed wires and vias with connectivity issues in a file named wireTest.rpt:

  ```
  setEditMode -verbose true
  ```

```
setEditMode -debug_file wireTest.rpt
editTrim
```

The report (sample) is as follows:

```
############################################################
# Generated by: Cadence Innovus 20.10-b015_1
# OS: Linux x86_64(Host ID noi-leenap1)
# Generated on: Mon Feb 10 13:50:19 2020
# Design: carve
# Command: editTrim
############################################################

Removed 0 wires and 0 vias.
1 wire can't be trimmed, because it connect a patch wire or the status is Cover.

The wires status is Cover:
Wire: ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/n7 M2 97845,-642213,100551,-642147
```

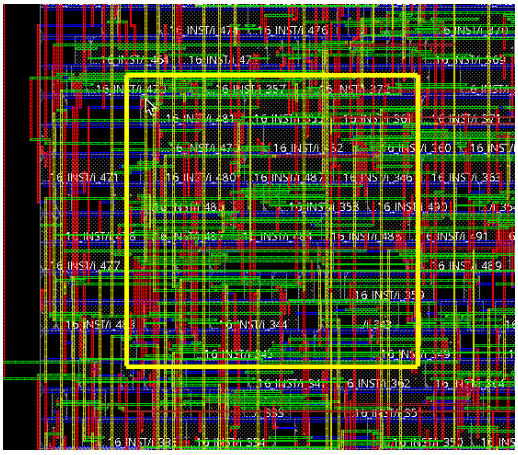
Related Information

- editAddRoute
- editAdjust
- editChangeLayer
- editChangeNet
- editChangeRule
- editChangeStatus
- editChangeVia
- editCommitRoute
- editCutWire
- editDelete
- editDuplicate
- editMerge
- editMove
- editResize
- editSplit
- editSelect
- editTrim
- getEditMode
- Editing Wires chapter in the *User Guide*
- Edit Menu chapter in the *Menu Reference*
  - *Wire* section

start_parallel_edit

```
start_parallel_edit
[-help]
[-area_restricted]
-region {x1 y1 x2 y2}
```

Initializes parallel editing by drawing a yellow square on the main window to indicate the edit area and saving the physical data, net attributes, via cell names and Non-Default Rules (NDRs) to a multiple binary files.



Once the DRC edits are complete, the parallel edit session can be stopped by using the `end_parallel_edit` command.

Note: If the design is freed up from the current session, the edit session is terminated automatically.

Parameters

`-help`	Prints a brief description that includes type and default information for each `start_parallel_edit` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man start_parallel_edit`
`-area_restricted`	Writes out only the different objects inside or touching the specified edit area. This is a strict interpretation of the region, and ignores the changes made outside of the region.
`-region {x1 y1 x2 y2}`	Specifies the coordinates of the edit area.

Example

- The following command marks the specified edit area in yellow and saves the design data to binary files:

```
innovus 8> start_parallel_edit -area_restricted -region {12.524 6.627 0.0 0.0}
Saving net attribute and regular wire/via information
Saving instance information
Saving physical pin information
Saving NDR rule information
Saving routing blockage information
Saving placement blockage information
Saving special wire/via information
```

Related Information

- `end_parallel_edit`
- `read_parallel_edit_files`
- Editing Wires chapter in the *User Guide*
  - *Parallel Editing Capability*

Appendix A: Database Object Information

Please find the document at the following link: Innovus Database Object Information

Appendix B: Tcl Globals

This appendix lists all public Tcl globals. Public Tcl globals are saved persistently in the .globals file by `saveDesign` and can be restored by `restoreDesign`.

Note: Persistent globals are only written if different from default value.

You can use the `find_global` command to query all public variables and private variables with non-default changes. Use patterns with the `find_global` command to retrieve information on specific global variables. You can use `set` *`global_namevalue`* to change the value of a specific global variable.

List of Globals

- `aocv_chip_size`
- `aocv_core_size`
- `auto_file_dir`
- `auto_file_prefix`
- `cds_lib`
- `cvd`
- `defHierChar`
- `defInCheckMaskShifts`
- `defInShapeBasedDefFile`
- `defOutLefNDR`
- `defOutLefVia`
- `defOutPolygonDieArea`
- `defOverlapWireReportFileName`
- `defStreamOutCheckUncolored`
- `delaycal_default_net_delay`
- `delaycal_input_transition_delay`

- delaycal_support_wire_load_model
- delaycal_use_default_delay_limit
- enable_legacy_metric
- ecoDisableNetRenamingForFlatNetlist
- enc_partial_cmd_argument_matching
- enc_print_full_message_summary
- enc_save_binary_timing_constraints
- enc_save_timing_constraints_always
- enc_source_continue_on_error
- enc_source_echo_filename
- enc_source_verbose
- enc_source_verbose_cmd_print_length_limit
- enc_source_continue_on_error
- enc_source_verbose_output
- enc_tcl_return_display_limit
- encEnableMetric
- extract_shrink_factor
- fill_setting_save
- flipchip_allow_routed_bump_edit
- fp_vertical_row
- group_instance_suffix
- gui_verbose
- highFreqInterposerFlow
- init_abstract_view
- init_check_output_pin_constant

- init_cpf_file
- init_design_netlisttype
- init_design_settop
- init_design_uniquify
- init_gnd_net
- init_io_file
- init_ignore_pgpin_polarity_check
- init_import_mode
- init_layout_view
- init_lef_check_mask_shifts
- init_lef_file
- init_mmmc_file
- init_mmmc_version
- init_oa_default_rule
- init_oa_design_cell
- init_oa_design_lib
- init_oa_design_view
- init_oa_foundry_rule
- init_oa_ref_lib
- init_oa_search_lib
- init_oa_special_rule
- init_pwr_net
- init_no_new_assigns
- init_power_intent_file
- init_remove_assigns

- init_top_cell
- init_verilog
- init_verilog_tolerate_port_mismatch
- inn_save_lef_ignore_abstracts
- layerNameNoAbbreviation
- lefDefOutVersion
- lefExtend2DCellShapes
- lefIgnoreLayerNames
- lib_build_asynch_de_assert_arc
- lib_build_timing_cond_default_arc
- load_netlist_ignore_undefined_cell
- metric_advanced_url_endpoint
- metric_capture_3d_hotspots
- metric_capture_depth
- metric_capture_design_image
- metric_capture_design_image_blockages_threshold
- metric_capture_max_drc_markers
- metric_capture_min_count
- metric_capture_overwrite
- metric_capture_pba_tns_histogram
- metric_capture_per_view
- metric_capture_reg2reg_metrics
- metric_capture_timing_paths
- metric_capture_timing_path_groups
- metric_capture_tns_histogram

- metric_capture_tns_histogram_buckets
- metric_capture_tns_histogram_max_slack
- metric_capture_tns_histogram_paths
- metric_capture_vth_per_power_domain
- metric_category_default
- metric_current_run_id
- metric_enable
- metric_summary_metrics
- mustjoinallports_is_one_pin
- readWriteLefCheckUncoloredShapes
- report_timing_format
- restore_db_file_check
- restore_db_stop_at_design_in_memory
- script_search_path
- soft_stack_size_limit
- spgM3StripePushDown
- spgM3StripeShrink
- timing_allow_input_delay_on_clock_source
- timing_all_registers_include_icg_cells
- timing_aocv_analysis_mode
- timing_aocv_derate_mode
- timing_apply_check_derate_to_external_output_delay
- timing_apply_default_primary_input_assertion
- timing_apply_exceptions_to_data_check_related_pin
- timing_analysis_precision_ps

- timing_case_analysis_propagation
- timing_check_timing_report_all_checks
- timing_clock_phase_propagation
- timing_clock_source_use_driving_cell
- timing_clock_uncertainty_from_to_precedence
- timing_collection_result_display_limit
- timing_collection_variable_assignment_compatibility
- timing_constraint_enable_separate_multicycle_data_checks
- timing_constraint_path_delay_exclude_check_delay_from_ignore_clock_latency
- timing_constraint_path_delay_exclude_io_delay_from_ignore_clock_latency
- timing_constraint_warn_for_timing_derate_exceeding_max_limit
- timing_cppr_opposite_edge_sigma_scale_factor_cell
- timing_cppr_opposite_edge_sigma_scale_factor_net
- timing_ignore_clock_based_path_delay_on_data_checks
- timing_enable_latch_thru_mode
- timing_library_infer_socv_from_aocv
- timing_rail_swing_checks_low_voltage_threshold
- timing_rail_swing_checks_high_voltage_threshold
- timing_report_drv_enable_clock_source_as_clock
- timing_report_drv_enable_slew_threshold_scaling
- timing_report_pulse_width_matching_launch_capture_paths
- timing_report_use_receiver_model_capacitance
- timing_waveform_aware_pulse_width_checks_high_voltage_level
- timing_waveform_aware_pulse_width_checks_low_voltage_level
- timing_constraint_disable_min_max_input_delay_worst_casing

- timing_constraint_enable_drv_limit_override
- timing_constraint_enable_logging
- timing_constraint_enable_report_invalid_begin_end_points
- timing_constraint_enable_search_path
- timing_constraint_path_delay_exclude_unconstrained_endpoints
- timing_constraint_path_delay_include_clock_pin_endpoints
- timing_continue_on_error
- timing_cppr_opposite_edge_mean_scale_factor
- timing_cppr_opposite_edge_sigma_scale_factor
- timing_cppr_propagate_thru_latches
- timing_cppr_remove_clock_to_data_crp
- timing_cppr_self_loop_mode
- timing_cppr_skip_clock_reconvergence
- timing_cppr_skip_clock_reconvergence_for_unmatched_clocks
- timing_cppr_threshold_ps
- timing_cppr_transition_sense
- timing_create_clock_default_propagated
- timing_create_clock_use_ideal_slew
- timing_default_opcond_per_lib
- timing_defer_mmmc_object_updates
- timing_derate_aocv_dynamic_delays
- timing_derate_aocv_reference_point
- timing_derate_dynamic_compatibility
- timing_derate_ocv_reference_point
- timing_derate_spatial_distance_unit

- timing_disable_bidi_output_timing_checks
- timing_disable_bus_contention_check
- timing_disable_clock_gating_checks
- timing_disable_clockperiod_checks
- timing_disable_constant_propagation_for_sequential_cells
- timing_disable_drv_reports_on_constant_nets
- timing_disable_floating_bus_check
- timing_disable_genclk_combinational_blocking
- timing_disable_inferred_clock_gating_checks
- timing_disable_internal_inout_cell_paths
- timing_disable_internal_inout_net_arcs
- timing_disable_lib_pulsewidth_checks
- timing_disable_library_data_to_data_checks
- timing_disable_library_tiehi_tielo
- timing_disable_netlist_constants
- timing_disable_nochange_checks
- timing_disable_non_sequential_checks
- timing_disable_output_as_clock_port
- timing_disable_recovery_removal_checks
- timing_disable_report_header_info
- timing_disable_retime_clock_path_slew_propagation
- timing_disable_sdf_retain_arc_merging
- timing_disable_skew_checks
- timing_disable_test_signal_arc
- timing_disable_timing_model_latch_inferencing

- timing_disable_tristate_disable_arcs
- timing_disable_user_data_to_data_checks
- timing_driving_cell_override_library
- timing_dynamic_loop_breaking
- timing_enable_all_fanin_fanout_levels_compatibility
- timing_enable_aocv_slack_based
- timing_enable_case_analysis_conflict_warning
- timing_enable_clock2clock_clockgating_check
- timing_enable_clock_phase_based_rise_fall_derating
- timing_enable_data_through_clock_gating
- timing_enable_derating_for_pulsewidth_checks
- timing_enable_early_late_data_slews_for_setuphold_mode_checks
- timing_enable_genclk_divide_by_inherit_parent_duty_cycle
- timing_enable_genclk_edge_based_source_latency
- timing_enable_genclk_source_path_register_limit
- timing_enable_get_object_escaped_name_backward_compatible
- timing_enable_latency_through_clock_gating
- timing_enable_mmmc_loop_handling
- timing_enable_multi_drive_net_reduction_with_assertions
- timing_enable_multifrequency_latch_analysis
- timing_enable_path_delay_to_unconstrained_endpoints_compatibility
- timing_enable_pessimistic_cppr_for_reconvergent_clock_paths
- timing_enable_power_ground_constants
- timing_enable_preset_clear_arcs
- timing_enable_pulsed_latch

- timing_enable_sdc_compatible_data_check_mcp
- timing_enable_si_cppr
- timing_enable_simultaneous_setup_hold_mode
- timing_enable_spatial_derate_mode
- timing_enable_tristate_clock_gating
- timing_enable_uncertainty_for_clock_checks
- timing_enable_uncertainty_for_pulsewidth_checks
- timing_extract_model_aocv_mode
- timing_extract_model_case_analysis_in_library
- timing_extract_model_check_arcs_as_lvf
- timing_extract_model_consider_design_level_drv
- timing_extract_model_disable_cycle_adjustment
- timing_extract_model_exhaustive_validation_dir
- timing_extract_model_exhaustive_validation_mode
- timing_extract_model_gating_as_nochange_arc
- timing_extract_model_ideal_clock_latency_arc
- timing_extract_model_include_applied_load_in_characterization_range
- timing_extract_model_include_applied_slew_in_characterization_range
- timing_extract_model_max_feedthrough_characterization_load
- timing_extract_model_non_borrowing_latch_path_as_setup
- timing_extract_model_slew_propagation_mode
- timing_extract_model_write_clock_checks_as_arc
- timing_extract_model_write_clock_checks_as_scalar_tables
- timing_extract_model_write_lvf
- timing_generate_normalized_driver_waveform

- timing_generated_clocks_inherit_ideal_latency
- timing_generated_clocks_allow_nested_assertions
- timing_get_of_objects_hier_compatibility
- timing_hier_object_name_compatibility
- timing_ignore_lumped_rc_assertions
- timing_io_use_clock_network_latency
- timing_library_convert_async_setuphold_to_recrem
- timing_library_genclk_use_group_name
- timing_library_hold_constraint_corner_sigma_multiplier
- timing_library_hold_sigma_multiplier
- timing_library_infer_async_pins_from_timing_arcs
- timing_library_infer_cap_range_from_ccs_receiver_model
- timing_library_infer_cap_range_from_ecsm_receiver_model
- timing_library_read_ccs_noise_data
- timing_library_read_without_power
- timing_library_scale_aocv_to_socv_to_n_sigma
- timing_library_setup_constraint_corner_sigma_multiplier
- timing_library_setup_sigma_multiplier
- timing_library_use_trilib_drv_values
- timing_library_zero_negative_timing_check_arcs
- timing_multifrequency_clock_rounding_factor
- timing_normalized_driver_waveform_clip_linear_part
- timing_normalized_driver_waveform_weight_factor
- timing_null_collection_return_compatibility
- timing_path_based_enable_exhaustive_depth_bounded_by_gba

- timing_path_based_enable_report_launch_clock_path
- timing_path_based_enable_verbose_mode
- timing_path_based_exhaustive_enable_design_coverage
- timing_path_based_exhaustive_max_paths_limit
- timing_pba_exhaustive_path_nworst_limit
- timing_prefix_module_name_with_library_genclk
- timing_propagate_latch_data_uncertainty
- timing_property_clock_used_as_data_unconstrained_clock_source_paths
- timing_read_library_without_ecsm
- timing_read_library_without_sensitivity
- timing_recompute_sdf_in_setuphold_mode
- timing_reduce_multi_drive_net_arcs
- timing_reduce_multi_drive_net_arcs_threshold
- timing_remove_clock_reconvergence_pessimism
- timing_report_clock_pin_as_begin_point
- timing_report_constraint_enable_extended_drv_format
- timing_report_constraint_rise_fall_clock_period_check
- timing_report_drv_enable_frequency_per_view
- timing_report_enable_cppr_point
- timing_report_enable_markers
- timing_report_enable_max_capacitance_drv_for_constants
- timing_report_enable_max_path_limit_crossed
- timing_report_enable_si_debug
- timing_report_enable_use_valid_start_end_points
- timing_report_generated_clock_info

- timing_report_group_based_mode
- timing_report_max_transition_check_using_nsigma_slew
- timing_report_redirect_message_types
- timing_report_retime_formatting_mode
- timing_report_skip_constraint_loop_check
- timing_report_timing_header_detail_info
- timing_report_unconstrained_path_early_late_header
- timing_report_unconstrained_paths
- timing_report_use_worst_parallel_cell_arc
- timing_resolve_driver_conflicts
- timing_sdf_adjust_negative_setuphold
- timing_sdf_enable_setuphold_scond_ccond
- timing_self_loop_paths_no_skew
- timing_self_loop_paths_no_skew_max_depth
- timing_self_loop_paths_no_skew_max_slack
- timing_set_clock_source_to_output_as_data
- timing_set_nsigma_multiplier
- timing_set_scaling_for_negative_checks
- timing_set_scaling_for_negative_delays
- timing_socv_preserve_variation_with_annotations
- timing_socv_rc_variation_mode
- timing_socv_statistical_min_max_mode
- timing_socv_view_based_nsigma_multiplier_mode
- timing_spatial_derate_chip_size
- timing_spatial_derate_distance_mode

- timing_suppress_escape_characters
- timing_suppress_ilm_constraint_mismatches
- timing_timing_window_pessimism_removal_include_si_delay
- timing_use_clock_pin_attribute_for_clock_net_marking
- timing_use_latch_time_borrow
- timing_write_sdf_no_escape_backslash_control
- trPrintIgnoredPadNets
- writeLefAbstractCustomerHeader
- write_lec_dft_constraints
- write_global_slack_worst_trigger_path_on_clocks
- write_lec_directory_naming_style
- write_lec_files

Index

A

addAIORow
addAIoFiller
addBumpConnectTargetConstraint
addChannelDensityControl
addDeCap
addDeCapCellCandidates
addDummyBoundaryWires
addEndCap
addFiller
addFillerGap
addHaloToBlock
addHierInst
addInst
addInstToInstGroup
addIoFiller
addIoInstance
addIoRowFiller
addMimCap
addModulePort
addModuleToFPlan
addNet
addNetToNetGroup
addObjFPlanCutBox
addPinToPinGroup
addPowerSwitch
addRepeaterByRule
addRing
addRoutingHalo
addSdpGroupMember
addSdpObject
addSizeBlockage
addSpareInstance
addSpecialRoute
addSplitPowerVia
addStripe

addTSV
addTieHiLo
addViaFill
addWellTap
add_cell_obs
add_clock_tree_source_group_roots
add_connection_to_neighbor_partition
add_decomp_filler
add_gate_array_filler
add_gui_marker
add_gui_shape
add_gui_text
add_interposer_route_block
add_line_end_track
add_metal_fill_signoff
add_ndr
add_oa_library_property
add_obj_on_bump
add_pg_fill
add_sai_boundary_flops
add_shape
add_tap_walls
add_target_pg
add_text
add_to_collection
add_track_fill
add_tracks
add_via
add_via_definition
adjustFPlanChannel
alias
alignObject
alignPtnClone
all_analysis_views
all_clocks
all_connected
all_constraint_modes
all_fanin
all_fanout
all_hold_analysis_views
all_inputs
all_instances

all_library_sets
all_outputs
all_rc_corners
all_registers
all_setup_analysis_views
analyze_esd_network
analyze_esd_voltage
analyze_ir_paths
analyze_joule_heat
analyze_package
analyze_paths_by_basic_path_group
analyze_paths_by_bottleneck
analyze_paths_by_clock_domain
analyze_paths_by_critical_false_path
analyze_paths_by_drv
analyze_paths_by_hier_port
analyze_paths_by_hierarchy
analyze_paths_by_view
analyze_resistance
analyze_resistance
analyze_resistance
analyze_signal_resistance
analyze_thermal
aocv_chip_size
aocv_core_size
append_to_collection
applyGlobalNets
apply_safety_mechanism
assembleDesign
assemble_proto_model
assignBump
assignIoPins
assignPGBumps
assignPtnPin
assignSigToBump
assignTSV
assign_clock_tree_source_groups
attachDiode
attachIOBuffer
attachModulePort
attachTerm

B

bindKey
bit_blasted_port_style
bitblast_ports

C

CopyOaRestoreFiles
calNegSlack
calculate_ccopt_cannot_clone_reason
calculate_didt
calculate_differential_voltage
calculate_noise_margin
ccopt_add_exclusion_drivers
ccopt_design
ccopt_pro
cds_lib
changeBBoxMasterFromR0
changeBBoxMasterToR0
changeBumpMaster
changeCurrentDesign
changeFloorplan
changeIoConstraints
change_IB_cell_site_name
checkBondPadSpacing
checkBump
checkDesign
checkDesign
checkFPlan
checkFPlanSpace
checkFiller
checkFootPrint
checkHierRoute
checkMacroLLOnTrack
checkMultiCpuUsage
checkNetlist
checkPartitionSdc
checkPinAssignment
checkPlace
checkSdpGroup
checkTQuantusModelFile
checkTimingLibrary
checkUnique
checkWhatIfTiming

check_base_layer_density
check_bus_guides
check_ccopt_clock_tree_convergence
check_design_across_hierarchy
check_design_report_async_pins
check_design_report_scan_pins
check_design_threshold_fanout
check_ldb_version
check_library
check_module_model
check_pg_library
check_power_intent
check_safety_mechanism
check_syntax
check_timing
check_tracks
check_ufc
ciopLoadBumpColorMapFile
clearActiveLogicView
clearAllRulers
clearDeCapCellCandidates
clearDrc
clearDrc
clearDrc
clearGlobalNets
clearScanDisplay
clearSpareCellDisplay
clock_opt_design
cloneMSVGate
clonePlace
clonePtnObj
close_ctd_win
colorizePowerMesh
colorize_cell_library
commit_ccopt_clock_tree_route_attributes
commit_module_model
commit_power_intent
commit_pushdown_eco
comparePinAssignStatistics
compare_cellview
compare_collections
compare_model_timing

compare_power_intent
compare_release
compress_special
congRepair
connectMacroFeedthrough
connect_clock_tree_mesh_drivers
convertBlackBoxToFence
convertFenceToBlackBox
convertFenceToLef
convert_gds_to_def
convert_lib_clock_tree_latencies
copyObject
copy_bump
copy_collection
createAbuttedFPlan
createActiveLogicView
createBasicPathGroups
createBusGuide
createBusNetGroup
createDensityArea
createExclusiveGroups
createFence
createGAFillerGroup
createGuide
createInstGroup
createInterfaceLogic
createIoRow
createLib
createLogicHierarchy
createMarker
createNetGroup
createObstruct
createPGPin
createPhysicalPin
createPinBlkg
createPinGroup
createPinGuide
createPipelineBusGuide
createPipelineNetGroup
createPtnCut
createPtnFeedthrough
createRegion

createRouteBlk
createRouteBlkAlongTrack
createRow
createRuler
createSdpGroup
createShield
createSignalPin
createSnapshot
createSoftGuide
createSpareModule
createSpecialDrcRegion
createSpecialMarkLayerShape
createStairwayBoundary
createTBOptFile
createTQuantusModelFile
createTSVNoLoadSPEF
createTrack
createUserBundleNet
createUserDisableForCombLoopBreak
createWhatIfInternalGeneratedClock
create_analysis_view
create_buffer_psPM_table
create_bump
create_bump_grid
create_ccopt_clock_spine
create_ccopt_clock_tree
create_ccopt_clock_tree_source_group
create_ccopt_clock_tree_spec
create_ccopt_flexible_htree
create_ccopt_generated_clock_tree
create_ccopt_macro_model_spec
create_ccopt_preferred_cell_stripes
create_ccopt_skew_group
create_clock
create_constraint_mode
create_delay_corner
create_esubpart
create_failure_mode
create_flexfiller_route_blockage
create_generated_clock
create_histogram_png
create_inst_space_group

create_interposer_route_block
create_library_set
create_module_model
create_op_cond
create_parasitic_coordinate_transform
create_partition_wrapper
create_path_category
create_pg_model_for_macro_place
create_power_pads
create_property_alias
create_proto_model
create_ps_per_micron_model
create_pushdown_eco
create_rc_corner
create_route_type
create_safety_mechanism
create_spice_deck
create_stress_rule
create_thermal_model
create_virtual_shape_on_pg_wires
create_what_if_shape
ctd_save_histogram
ctd_save_view
ctd_show_skew_group
ctd_trace
ctd_win
cts_refine_clock_tree_placement
current_design
current_instance
cutBoxListFromPowerDomain
cutPowerDomainByOverlaps
cutRectilinearInst
cutRow
cvd

D

dbEdge
dbGet
dbQuery
dbSchema
dbSet
dbShape

dbTransform
db_browser
db_pt
db_rect
dbu2uu
dd_get
debugPtnBoundaryPorts
debug_irdrop
decompress_special
decrypt
defComp
defHierChar
defIn
defInCheckMaskShifts
defInShapeBasedDefFile
defOut
defOutBySection
defOutCompressVia
defOutLefNDR
defOutLefVia
defStreamOutCheckUncolored
definePartition
define_add_on_module_model
define_bundled_bus
define_glitch_holding_resistance
define_proc_arguments
define_property
dehighlight
delaycal_default_net_delay
delaycal_input_transition_delay
delaycal_use_default_delay_limit
deletTSV
deleteAloFiller
deleteAllCellPad
deleteAllDensityAreas
deleteAllFPObjects
deleteAllInstGroups
deleteAllPowerPreroutes
deleteAllPtnCuts
deleteAllPtnFeedthroughs
deleteAllScanCells
deleteAllSignalPreroutes

deleteBNet
deleteBufferTree
deleteBumpConnectTargetConstraint
deleteBumps
deleteBusGuide
deleteCellEdgeSpacing
deleteCellEdgeType
deleteCellPad
deleteDangingNet
deleteDangling1b1Or0s
deleteDanglingPort
deleteDeCap
deleteEmptyModule
deleteExclusiveGroups
deleteFPObject
deleteFiller
deleteGAFillerGroup
deleteHaloFromBlock
deleteInst
deleteInstFromInstGroup
deleteInstGroup
deleteInstPad
deleteIntegRouteConstraint
deleteIoFiller
deleteIoInstance
deleteIoRowFiller
deleteMetalFill
deleteMimCap
deleteModule
deleteModulePort
deleteNet
deleteNetFromNetGroup
deleteNetGroup
deleteNetWeight
deleteNotchFill
deletePGPin
deletePartition
deletePinBlkg
deletePinFromPinGroup
deletePinGroup
deletePinGuide
deletePipelineNetGroup

deletePlaceBlockage
deletePowerSwitch
deletePtnCut
deleteRelativeFPlan
deleteRouteBlk
deleteRoutingHalo
deleteRow
deleteScanCell
deleteScanChain
deleteScanChainPartition
deleteSdpObject
deleteSecondaryPGNet
deleteSelectedFromFPlan
deleteShield
deleteSizeBlockage
deleteSpareModule
deleteTieHiLo
deleteTrack
deleteUninstantiatedModules
deleteUserBundleNet
deleteWhatIfTimingAssertions
deleteWorkspace
delete_bump_grid
delete_ccopt_clock_spines
delete_ccopt_clock_tree_source_group
delete_ccopt_clock_tree_spec
delete_ccopt_clock_trees
delete_ccopt_flexible_htrees
delete_ccopt_preferred_cell_stripe
delete_ccopt_skew_groups
delete_cell_obs
delete_cell_stack_area
delete_cell_stack_group
delete_cell_virtual_align
delete_clock_tree_repeaters
delete_feedthru_buffer
delete_gui_object
delete_inst_space_group
delete_path_category
delete_pg_keepout
delete_proto_model
delete_route_type

deriveTimingBudget
deselectAll
deselectBusGuide
deselectGroup
deselectIOPin
deselectInst
deselectInstByCellName
deselectInstOnNet
deselectModule
deselectNet
deselectPin
deselectSecondaryPGNet
deselect_bump
deselect_obj
detachModulePort
detachSdpGroup
detachTerm
detailRoute
disconnectDanglingPort
displayBufTree
displayScanChain
displaySpareCell
displayUserBundleNet
display_obj_connectivity
display_temperature_result
display_timing_map
do_extract_model
dumpCongestArea
dumpMultiBitFlopMappingFile
dumpNanoCongestArea
dumpNetsInCongestedArea
dumpOutVias
dumpPictures
dumpToGIF

E

earlyGlobalRoute
ecoAddRepeater
ecoChangeCell
ecoClone
ecoCloneFlop
ecoCompareNetlist

ecoDefIn
ecoDelRepeater
ecoDesign
ecoMergeCombinationalCell
ecoOaDesign
ecoPlace
ecoRemoveTiedInputs
ecoRoute
ecoSplitCombinationalCell
ecoSplitComplexFlop
ecoSplitFlop
ecoSwapSpareCell
editAddFillet
editAddPoly
editAddRoute
editAddTrimMetal
editAddVia
editAdjust
editBumpConnectTargetConstraint
editChangeLayer
editChangeMask
editChangeNet
editChangeRule
editChangeStatus
editChangeVia
editCommitPoly
editCommitRoute
editCopy
editCutWire
editDelete
editDeleteFillet
editDeselect
editDuplicate
editFixWideWires
editMerge
editMove
editPin
editPowerVia
editResize
editRotate
editSelect
editSplit

editTrim
edit_bump_name
encMessage
enc_print_full_message_summary
enc_save_timing_constraints_always
enc_source_continue_on_error
enc_source_echo_filename
enc_source_verbose
enc_source_verbose
enc_source_verbose_output
enc_tcl_error_info
enc_tcl_return_display_limit
encrypt
encrypt_thermal_file
endECO
end_parallel_edit
end_sai
eval_common_ui
exportNdr
exportPowerSwitch
extractRC
extract_metal_density
extract_package

F

fcroute
fillNotch
fill_setting_save
fill_setting_save
filter_collection
findLefEquivalentCells
findPinPortNumber
find_global
finishFloorplan
fit
fixACLimitViolation
fixAllIos
fixBondPad
fixOpenFill
fixVia
fix_boundary_overlaps
fix_drc_with_patch

fix_feedthru_assign
fix_floorplan
fix_multi_drivers
flattenCoverCell
flattenIlm
flattenPartition
flipModule
flipOrRotateObject
flipSdpObject
flip_or_rotate_obj
flipchip_allow_routed_bump_edit
floorPlan
foreach_in_collection
free_memory_of_compressed_objects
free_power_intent

G

genPinText
generateCapTbl
generateFFSetupFile
generateLef
generateRCFactor
generateVias
generate_esd_rlrp_report
generate_fence
generate_pg_library
generate_power_up_report
generate_tech_pg_lib
generate_vertical_cell_edge_constraint
generate_xpgv
getActiveLogicViewMode
getAddRingMode
getAddStripeMode
getAllLayers
getAttribute
getBlackBoxArea
getBondPad
getBudgetingMode
getBuildArch
getCPFUserAttributes
getCheckMode
getClonePtnOrient

getCmdLogFileName
getCompressLevel
getDbGetMode
getDelayCalMode
getDensityMapMode
getDesignMode
getDistributeHost
getDrawView
getEcoMode
getEditMode
getEndCapMode
getExportMode
getExtractRCMode
getFPlanMode
getFillerMode
getFinishFPlanMode
getFlipChipMode
getHierMode
getIlmMode
getIlmType
getImportMode
getInstPowerDomain
getIntegRouteConstraint
getIoFlowFlag
getLatencyFile
getLayerPreference
getLogFileName
getMetalFillSpacingTable
getModuleView
getMsvMode
getNanoRouteMode
getNetWeight
getOaxMode
getObjFPlanBoxList
getObjFPlanPolygon
getOptMode
getPGNetResis
getPGPinUseSignalRoute
getPinAssignMode
getPinConstraint
getPinDensityMapMode
getPlaceMode

getPreference
getPtnPinStatus
getRailPrototypeMode
getResizeFPlanMode
getRouteMode
getSIMode
getScanReorderMode
getSchedulingFile
getSdpGroupAttribute
getSdpMode
getSpecialNetResis
getStreamOutMode
getTieHiLoMode
getTimeLibFile
getUPFPortAttr
getUsefulSkewMode
getValidPowerSwitchLocation
getVersion
getViaGenMode
getWhatIfTimingAssertions
getWhatIfTimingMode
get_abstract_mode
get_activity
get_add_target_pg_mode
get_analysis_view
get_arcs
get_capacitance_unit
get_ccopt_clock_spines
get_ccopt_clock_tree_capacitance
get_ccopt_clock_tree_cells
get_ccopt_clock_tree_nets
get_ccopt_clock_tree_sinks
get_ccopt_clock_tree_slew
get_ccopt_clock_tree_source_groups
get_ccopt_clock_trees
get_ccopt_dag_traversal
get_ccopt_delay_corner
get_ccopt_effective_max_capacitance
get_ccopt_flexible_htrees
get_ccopt_preferred_cell_stripe
get_ccopt_property
get_ccopt_skew_group_delay

get_ccopt_skew_group_path
get_ccopt_skew_groups
get_cells
get_clock_network_objects
get_clocks
get_constraint_mode
get_ctd_win_id
get_ctd_win_title
get_default_switching_activity
get_delay_corner
get_designs
get_dynamic_power_simulation
get_generated_clocks
get_global
get_interactive_constraint_modes
get_ir_insight
get_lib_arcs
get_lib_cells
get_lib_clock_tree_path_delay
get_lib_pg_pins
get_lib_pins
get_library_set
get_libs
get_message
get_metric
get_nets
get_oa_default_rule_lib
get_op_cond
get_path_groups
get_pba_mode
get_pg_fill_config
get_pg_nets
get_pg_pins
get_physical_info
get_pins
get_ports
get_power
get_propagated_clock
get_property
get_proto_design_mode
get_proto_mode
get_proto_model

get_rc_corner
get_sdc_mode
get_signal_em_analysis_mode
get_signoff_verify_design_config
get_snap_grid_info
get_socv_rc_variation_factor
get_socv_reporting_nsigma_multiplier
get_time_unit
get_trace_obj_connectivity_mode
get_track_fill_config
get_verify_drc_mode
get_via_pillars
get_visible_bumps
get_visible_netGroups
get_visible_nets
get_well_tap_mode
globalDetailRoute
globalNetConnect
globalRoute
group
group_instance_suffix
group_path
gui_attach_to_cursor
gui_clear_trace_flightline
gui_close_cell_view
gui_deselect
gui_dim_foreground
gui_dump_picture
gui_get_legend
gui_group_hinst
gui_open_cell_view
gui_report_trace_flightline
gui_save_flightline
gui_schematic_load_view
gui_schematic_save_view
gui_select
gui_set_legend
gui_show_edge_number
gui_trace_flightline
gui_ungroup_hinst
gui_verbose
gui_zoom_ctd

H

handlePtnAreaIo
help
hier_clock_route
highlight
highlightSyncObj
highlight_pin
highlight_pin_connection
highlight_timing_report
hiliteFeedthroughNets
hilite_proto_model

I

identify_proto_model
ignore_max_via_stack_rule
importPowerSwitch
import_ilm_data
import_module_model
index_collection
initCoreRow
initECO
init_abstract_view
init_check_output_pin_constant
init_cpf_file
init_design
init_design_netlisttype
init_design_settop
init_import_mode
init_lef_check_antenna
init_lef_check_mask_shifts
init_lef_file
init_ml
init_oa_default_rule
init_oa_ref_lib
init_oa_search_lib
init_verilog_tolerate_port_mismatch
inn_save_lef_ignore_abstracts
innovus
insertPtnFeedthrough
insert_physical_cell
ioInstOverlapCheck
is_common_ui_mode

J

justifyBudget
justifyException

L

layerNameNoAbbreviation
lefDefOutVersion
lefDefOutVersion
lefDefOutVersion
lefExtend2DCellShapes
lefExtendCellObsShapeUnderTrim
lefIgnoreLayerNames
lefdefInputCheckColoredShape
legalizeFPlan
legalizePin
lib_build_asynch_de_assert_arc
lineSelect
list_gui_marker
list_property
lminus
loadBlackBoxNetlist
loadDefFile
loadDrc
loadECO
loadFPlan
loadFootPrint
loadIoFile
loadLefFile
loadPtnPin
loadSpecialRoute
loadViolationReport
loadWorkspace
load_netlist_ignore_undefined_cell
load_path_categories
load_timing_debug_report

M

man
map_die_package
map_dies
mark_physical_power_domains
merge_clock_cells

merge_hierarchical_def
merge_model_timing
merge_multi_port_to_single_port
merge_pg_library
merge_vtm
metric_advanced_url_endpoint
metric_capture_3d_hotspots
metric_capture_depth
metric_capture_design_image
metric_capture_design_image_blockages
metric_capture_design_image_blockages_threshold
metric_capture_design_image_power_intent
metric_capture_design_image_route_drc
metric_capture_max_drc_markers
metric_capture_min_count
metric_capture_overwrite
metric_capture_pba_tns_histogram
metric_capture_per_view
metric_capture_reg2reg_metrics
metric_capture_timing_analysis_mode
metric_capture_timing_path_groups
metric_capture_timing_paths
metric_capture_tns_histogram
metric_capture_tns_histogram_buckets
metric_capture_tns_histogram_max_slack
metric_capture_tns_histogram_paths
metric_capture_vth_per_power_domain
metric_category_default
metric_current_run_id
metric_enable
metric_summary_metrics
modifyBudget
modifyPipelineNetGroup
modifyPowerDomainAttr
modify_ccopt_skew_group
modify_ndr
monitor_hosts
moveMacroInsideModule
movePowerSwitch
moveSdpObject
move_obj
mustjoinallports_is_one_pin

N

nagelfar

O

oa2lef
oaIn
oaOut
open_gtd
open_schematic
optDesign
optPower
optVirtual
optimizePattern

P

Puts
pack_align_macros
pack_module_model_libs
pan
panCenter
panPage
parse_proc_arguments
partition
partitionPushSpecialWires
pasteObject
pinAlignment
pinAnalysis
placeAIO
placeBondPad
placeCursor
placeInstance
placeJtag
placePIO
placePad
placePadIO
placePipeline
placeSdpGroup
placeSpareModule
place_cells_at_center_for_feedthrus
place_connected
place_design
place_opt_design

place_partition_clone
power-grid library commands
power_integrity_place
predict_floorplan
prepareForEcoRoute
preserve_ccopt_port
print_message
propagate_activity
proto_design
pull_block_constraint
push_ptn_network

Q

queryDensityInBox
queryFPlanObject
queryPinDensity
query_power_data

R

rail analysis
range_collection
rcOut
readBumpLocation
readDieAbstract
readFlipChipProperty
readIoUpdate
readPackage
readSdpFile
readTSVConfig
readTransitionFile
readWriteLefCheckUncoloredShapes
read_design_stack_config
read_ilm_eco_db
read_name_mapping
read_parallel_edit_files
read_parasitics
read_path_descriptions
read_power_intent
read_power_rail_results
read_safety_tmr
read_sai
read_sdf

read_spdf
read_stream
read_taf
read_temperature_map_file
read_twf
read_usf
rechainPowerSwitch
reclaimArea
recreatePtnCellBlockage
redirect
redo
redraw
refineMacro
refinePlace
refine_macro_place
registerTrigger
register_gui_edit_callback
relink_db_files
remove_assigns
remove_from_collection
remove_gui_marker
rename_obj
rename_pg_library
replaceLefMacro
replacePowerSwitch
reportAlwaysOnBuffer
reportCapViolation
reportCellPad
reportCongestArea
reportCongestion
reportCritInstance
reportCritNet
reportCritTerm
reportDanglingNet
reportDanglingPort
reportDeCap
reportDeCapCellCandidates
reportDelayCalculation
reportDensityMap
reportFanoutViolation
reportFootPrint
reportFreqViolation

reportGateCount
reportIgnoredNets
reportIlmStatus
reportInstPad
reportIsolation
reportJtagInst
reportLegalWireWidthForBump
reportMultibit
reportNetGroup
reportNetStat
reportPathGroupOptions
reportPinAssignStatistics
reportPinDensityMap
reportPipeline
reportPowerDomain
reportPowerRoute
reportPowerSwitch
reportProbePins
reportRC
reportRoute
reportRouteTypeConstraints
reportScanCell
reportScanChainPartition
reportSeedConnection
reportSelect
reportShield
reportShifter
reportSpecialRoute
reportTimingLib
reportTranViolation
reportTwoPinChain
reportUnalignedNets
reportUnsnapBlocks
reportVoltage
reportVtInstCount
reportWire
reportWirePath
report_analysis_coverage
report_analysis_views
report_annotated_assertions
report_annotated_check
report_annotated_delay

report_annotated_parasitics
report_annotations
report_aocv_derate
report_area
report_bundled_bus
report_case_analysis
report_ccopt_cell_filtering_reasons
report_ccopt_cell_halo_violations
report_ccopt_clock_tree_convergence
report_ccopt_clock_tree_drv
report_ccopt_clock_tree_structure
report_ccopt_clock_trees
report_ccopt_pin_insertion_delays
report_ccopt_preserved_clock_tree_ports
report_ccopt_skew_groups
report_ccopt_skew_groups
report_cell_edge_spacing
report_cell_edge_type
report_cell_instance_timing
report_cell_stack_area
report_cell_stack_group
report_cell_virtual_align
report_clock_gating_check
report_clock_timing
report_clocks
report_command_mode
report_constraint
report_cppr
report_design
report_disk
report_fanin
report_fanout
report_gates
report_globals
report_gui_edit_callback
report_hidden_usage
report_inactive_arcs
report_inst_space_group
report_instance_cdb
report_instance_library
report_instance_power
report_lib_arcs

report_message
report_metal_fill
report_metal_stack
report_min_pulse_width
report_mode
report_module_model
report_narrow_channel
report_net
report_net_parasitics
report_net_wires
report_noise
report_noise_lib_pin
report_noise_propagation
report_oa_lib
report_obj_connectivity
report_partitions
report_path_groups
report_pg_keepout
report_ports
report_power_rail_results
report_precision_capacitance
report_precision_capacitance
report_precision_capacitance
report_precision_capacitance
report_preserves
report_property
report_proto_model
report_rcdb
report_resource
report_route
report_sai_constraint
report_scan_chain
report_sequential
report_socv_interconnect_variation
report_socv_library
report_statistical_timing_derate_factors
report_timing
report_timing_derate
report_tracks
report_unit_parasitics
report_via
report_voltage_scaling

report_voltage_swing
report_wire_load
resetBusGuideMultiColors
resetModifiedBudget
resetMultiColorsHier
resetPathGroupOptions
reset_all_ccopt_preserved_clock_tree_ports
reset_annotated_check
reset_annotated_delay
reset_annotated_transistion
reset_annotated_transition
reset_aocv_stage_weight
reset_case_analysis
reset_ccopt_config
reset_ccopt_preserved_clock_tree_port
reset_ccopt_routing_state
reset_clock
reset_clock_gating_check
reset_clock_groups
reset_clock_latency
reset_clock_sense
reset_clock_transition
reset_clock_tree_latency
reset_clock_uncertainty
reset_data_check
reset_disable_clock_gating_check
reset_disable_timing
reset_drive
reset_driving_cell
reset_generated_clock
reset_glitch_derate
reset_ideal_latency
reset_ideal_network
reset_ideal_network
reset_input_delay
reset_instance_library
reset_load
reset_max_capacitance
reset_max_fanout
reset_max_time_borrow
reset_max_transition
reset_min_capacitance

reset_min_fanout
reset_min_pulse_width
reset_min_transition
reset_mode
reset_noise_lib_pin
reset_output_delay
reset_parasitics
reset_path_adjust_group
reset_path_exception
reset_path_group
reset_pll_timing
reset_power_activity
reset_propagated_clock
reset_property
reset_pulse_clock_max_width
reset_pulse_clock_min_transition
reset_resistance
reset_sdf_assertions
reset_sense
reset_spare_insts
reset_timing_derate
reset_usf
reset_wire_load_mode
reset_wire_load_model
resizeBlackBox
resizeFloorplan
restoreDesign
restorePowerSwitch
restoreRC
restore_ccopt_config
restore_db_directory
restore_db_file_check
restore_db_stop_at_design_in_memory
restore_db_tool
restore_db_version
restore_module_model
resume
routeDesign
routePGPinUseSignalRoute
routePointToPoint
route_ccopt_clock_tree_nets
route_ccopt_flexible_htrees

route_fix_ir
route_fix_signoff_drc
route_opt_design
route_pg
runCLP
run_abstract
run_pegasus_drc_rules
run_pvs_drc_rules
run_vsr

S

saveBlackBox
saveColorPreference
saveDesign
saveDrc
saveFPlan
saveHInstColor
saveIoFile
saveModel
saveNetlist
saveOaBlackboxes
savePartition
savePowerSwitch
savePreference
savePtnPin
saveRC
saveRouteGuide
saveSignalStormConstraint
saveSpecialRoute
saveTimingBudget
saveWhatIfConstraints
saveWhatIfTimingAssertions
saveWhatIfTimingModel
saveWorkspace
save_abstract
save_ccopt_config
save_cmd_file_limit
save_global
save_path_categories
scanReorder
scanTrace
script_search_path

selectBusGuide
selectBusGuideSegment
selectGroup
selectIOPin
selectInst
selectInstByCellName
selectInstOnNet
selectModule
selectNet
selectObjByProp
selectPGPin
selectPin
selectPtnPinGuide
selectRouteBlk
selectSecondaryPGNet
select_bump
select_highlighted
select_obj
select_row
setActiveLogicViewMode
setAddRingOption
setAddStripeMode
setAnalysisMode
setAttribute
setBottomIoPadOrient
setBudgetingMode
setBufFootPrint
setBumpFixed
setBumpPlacementStatus
setBusGuideMultiColors
setCheckMode
setClonePtnOrient
setCompressLevel
setCycleBudgetRatio
setDbGetMode
setDefaultWorkspace
setDelayCalMode
setDelayFootPrint
setDensityMapMode
setDesignMode
setDistributeHost
setDontUse

setDrawView
setEcoMode
setEditMode
setEndCapMode
setExportMode
setExtractRCMode
setFPlanMode
setFPlanRowSpacingAndType
setFillerMode
setFinishFPlanMode
setFixedBlockSize
setFixedBudget
setFlipChipMode
setFlipping
setGenerateViaMode
setHInstColorId
setHierMode
setIlmMode
setIlmType
setImportMode
setInstGroupPhyHier
setInstancePlacementStatus
setIntegRouteConstraint
setInvFootPrint
setIoFlowFlag
setIoRowMargin
setIsolatedCutRule
setLatencyFile
setLayerPreference
setLibraryUnit
setLicenseCheck
setLimitedAccessFeature
setMaxCapPerFreq
setMaxCapPerFreqTran
setMaxTranPerFreq
setMessageLimit
setMetalFill
setMetalFillSpacingTable
setModuleView
setMsvMode
setMultiColorsHier
setNanoRouteMode

setNet
setOaxMode
setObjFPlanBox
setObjFPlanBox
setObjFPlanBox
setObjFPlanBox
setObjFPlanBox
setObjFPlanBox
setObjFPlanBox
setObjFPlanBoxList
setObjFPlanPolygon
setOptMode
setPGPinUseSignalRoute
setPathGroupOptions
setPinAssignMode
setPinConstraint
setPinDensityMapMode
setPlaceMode
setPreference
setProbePin
setPromotedMacroPin
setPtnPinStatus
setPtnPinUSE
setPtnUserCnsFile
setRailPrototypeMode
setResizeFPlanMode
setResizeLine
setRouteBlkDefaultLayer
setRouteMode
setSIMode
setScanReorderMode
setSchedulingFile
setSdpGroupAttribute
setSdpMode
setSdpObjectStatus
setSdpTopGroup
setSelHInstColor
setSelectedDensityArea
setSelectedObstruct
setSelectedPtnCut
setSelectedPtnFeedthrough
setSelectedPtnPinBlk

setSelectedPtnPinGuide
setSelectedRouteBlk
setSelectedStripBoxShape
setSelectedStripBoxState
setSelectedWireState
setShrinkFactor
setSignoffOptMode
setSnapGrid
setSpecialRouteType
setSrouteMode
setStreamOutMode
setTieHiLoMode
setTopCell
setUsefulSkewMode
setViaFill
setViaGenMode
setViaGenMode
setViaGenMode
setViaGenMode
setViaGenMode
setWhatIfClockLatency
setWhatIfClockPort
setWhatIfDriveType
setWhatIfLoadType
setWhatIfPortParameters
setWhatIfPortPriority
setWhatIfTimingMode
setWindowPreference
set_abstract_mode
set_add_target_pg_mode
set_advanced_package_options
set_advanced_pg_library_mode
set_analysis_view
set_annotated_check
set_annotated_delay
set_annotated_glitch
set_annotated_transition
set_aocv_interface_path_offset
set_aocv_stage_weight
set_aocv_thresholds
set_case_analysis
set_ccopt_preserved_clock_tree_port

set_ccopt_property
set_cdb_binding
set_cell_binding
set_cell_power_domain
set_clock_exclusivity
set_clock_gating_check
set_clock_groups
set_clock_latency
set_clock_sense
set_clock_transition
set_clock_uncertainty
set_ctd_win_title
set_data_check
set_default_view
set_die_model
set_disable_clock_gating_check
set_disable_timing
set_dont_touch
set_dont_touch_network
set_dont_use
set_drive
set_driving_cell
set_dynamic_power_simulation
set_dynamic_power_simulation
set_false_path
set_fanout_load
set_glitch_derate
set_global
set_global_always_keep
set_ideal_latency
set_ideal_network
set_ideal_transition
set_input_delay
set_input_transition
set_inst_temperature_file
set_instance_library
set_io_thresholds
set_load
set_logic_one
set_logic_zero
set_max_capacitance
set_max_delay

set_max_fanout
set_max_time_borrow
set_max_transition
set_message
set_metal_density_options
set_metal_fill_signoff_mode
set_min_capacitance
set_min_delay
set_min_fanout
set_min_pulse_width
set_min_transition
set_mode
set_model_priority
set_module_model
set_multicycle_path
set_mutex_attacker
set_noise_lib_pin
set_normalized_driver_waveform_lib
set_object_color
set_offchip_package_trace
set_output_delay
set_path_adjust
set_pba_mode
set_pg_fill_config
set_pg_library_mode
set_power_calc_temperature
set_power_data
set_power_include_file
set_power_rail_display
set_power_rail_layers_nets
set_proc_verbose
set_propagated_clock
set_property
set_proto_design_mode
set_proto_mode
set_proto_model
set_proto_model_physical_constraint
set_proto_timing_settings
set_pulse_clock_max_transition
set_pulse_clock_max_width
set_pulse_clock_min_transition
set_pulse_clock_min_width

set_quiet_attacker
set_rail_analysis_mode
set_resistance
set_safety_mechanism_rules_dcls
set_safety_mechanism_rules_tmr
set_sense
set_signal_em_analysis_mode
set_socv_constraint_config
set_socv_rc_variation_factor
set_socv_reporting_nsigma_multiplier
set_table_style
set_thermal_analysis_mode
set_timing_derate
set_trace_obj_connectivity_mode
set_track_fill_config
set_twf_attribute
set_verify_drc_mode
set_via_pillars
set_visible_bumps
set_visible_netGroups
set_visible_nets
set_voltage_regulator_module
set_well_tap_mode
set_wire_load_mode
set_wire_load_model
set_wire_load_selection_group
shiftOrigin
showPtnWireX
show_ccopt_cell_name_info
signoffOptDesign
signoffTimeDesign
skewClock
snapFPlan
snapFPlanIO
soft_stack_size_limit
sort_collection
source
spaceBondPad
spaceIoInst
spaceObject
specifyBlackBox
specifyCellEdgeSpacing

specifyCellEdgeType
specifyCellPad
specifyIlm
specifyInstPad
specifyJtag
specifyLockupElement
specifyNetWeight
specifyPartition
specifyScanCell
specifyScanChain
specifyScanChainPartition
specifySelectiveBlkgGate
specifySpareGate
specify_cell_stack_area
specify_cell_stack_group
specify_cell_virtual_align
specify_lib
specify_pg_keepout
spefIn
spgM3StripePushDown
spgM3StripeShrink
sroute
staggerBondPad
start_parallel_edit
streamOut
stretchRows
summaryReport
summaryReport
suppressMessage
suspend
swapPins
swapRegularCellWithAlwaysOnCell
swapSignal
swap_well_taps
synthesize_ccopt_flexible_htrees

T

timeDesign
time_info
timing_allow_input_delay_on_clock_source
timing_analysis_precision_ps
timing_aocv_derate_mode

timing_aocv_stage_count_recalculate_on_timing_reset
timing_apply_check_derate_to_external_output_delay
timing_apply_default_primary_input_assertion
timing_apply_exceptions_to_data_check_related_pin
timing_apply_setup_hold_exceptions_to_data_check_related_pin
timing_case_analysis_for_icg_propagation
timing_case_analysis_for_sequential_propagation
timing_case_analysis_propagation
timing_check_timing_report_all_checks
timing_check_timing_signal_level_high_to_low_threshold
timing_check_timing_signal_level_low_to_high_threshold
timing_clock_phase_propagation
timing_clock_source_use_driving_cell
timing_clock_uncertainty_from_to_precedence
timing_collection_all_fanin_fanout_traversal_mode
timing_collection_variable_assignment_compatibility
timing_constraint_disable_min_max_input_delay_worst_casing
timing_constraint_enable_detailed_report_invalid_begin_end_points
timing_constraint_enable_drv_limit_override
timing_constraint_enable_logging
timing_constraint_enable_report_invalid_begin_end_points
timing_constraint_enable_search_path
timing_constraint_enable_separate_multicycle_data_checks
timing_constraint_load_minimal_set_for_automated_eco
timing_constraint_path_delay_exclude_check_delay_from_ignore_clock_latency
timing_constraint_path_delay_exclude_io_delay_from_ignore_clock_latency
timing_constraint_path_delay_exclude_unconstrained_endpoints
timing_constraint_path_delay_include_clock_pin_endpoints
timing_constraint_update_io_latency_averaging_mode
timing_constraint_warn_for_timing_derate_exceeding_max_limit
timing_context_apply_default_primary_input_assertion
timing_context_clock_mapping_check_waveform
timing_context_clock_mapping_percentage_tolerance
timing_context_port_based_clock_mapping
timing_continue_on_error
timing_cppr_enable_mismatch_transition_mode
timing_cppr_opposite_edge_mean_scale_factor
timing_cppr_opposite_edge_sigma_scale_factor
timing_cppr_opposite_edge_sigma_scale_factor_cell
timing_cppr_opposite_edge_sigma_scale_factor_net
timing_cppr_propagate_thru_latches
timing_cppr_remove_clock_to_data_crp

timing_cppr_self_loop_mode
timing_cppr_skip_clock_reconvergence_for_unmatched_clocks
timing_cppr_transition_sense
timing_create_clock_default_propagated
timing_create_clock_use_ideal_slew
timing_default_opcond_per_lib
timing_defer_mmmc_object_updates
timing_derate_incremental_adjust_additive_mode
timing_derate_incremental_multiply_accumulative_mode
timing_derate_negative_delay_backward_compatibility
timing_derate_spatial_distance_unit
timing_derate_voltage_scaling_mode
timing_disable_bidi_output_timing_checks
timing_disable_black_box_endpoints
timing_disable_bus_contention_check
timing_disable_clock_gating_checks
timing_disable_clockperiod_checks
timing_disable_constant_propagation_for_sequential_cells
timing_disable_drv_reports_on_constant_nets
timing_disable_inferred_clock_gating_checks
timing_disable_internal_inout_cell_paths
timing_disable_internal_inout_net_arcs
timing_disable_lib_pulsewidth_checks
timing_disable_library_data_to_data_checks
timing_disable_library_tiehi_tielo
timing_disable_netlist_constants
timing_disable_nochange_checks
timing_disable_non_sequential_checks
timing_disable_output_as_clock_port
timing_disable_pulsewidth_same_edge_si_cppr_mode
timing_disable_recovery_removal_checks
timing_disable_retime_clock_path_slew_propagation
timing_disable_sdf_retain_arc_merging
timing_disable_skew_checks
timing_disable_test_signal_arc
timing_disable_timing_model_latch_inferencing
timing_disable_tristate_disable_arcs
timing_disable_user_data_to_data_checks
timing_driving_cell_override_library
timing_dynamic_loop_breaking
timing_enable_all_fanin_fanout_levels_compatibility
timing_enable_case_analysis_conflict_warning

timing_enable_clock_phase_based_rise_fall_derating
timing_enable_data_through_clock_gating
timing_enable_derating_for_pulsewidth_checks
timing_enable_early_late_data_slews_for_setuphold_mode_checks
timing_enable_genclk_divide_by_inherit_parent_duty_cycle
timing_enable_genclk_edge_based_source_latency
timing_enable_genclk_source_path_register_limit
timing_enable_get_object_escaped_name_backward_compatible
timing_enable_latch_thru_mode
timing_enable_latency_through_clock_gating
timing_enable_mmmc_loop_handling
timing_enable_multifrequency_latch_analysis
timing_enable_path_delay_to_unconstrained_endpoints_compatibility
timing_enable_power_ground_constants
timing_enable_preset_clear_arcs
timing_enable_pulsed_latch
timing_enable_sdc_compatible_data_check_mcp
timing_enable_si_cppr
timing_enable_simultaneous_setup_hold_mode
timing_enable_spatial_derate_mode
timing_enable_tristate_clock_gating
timing_enable_uncertainty_for_clock_checks
timing_enable_uncertainty_for_pulsewidth_checks
timing_extract_model_case_analysis_in_library
timing_extract_model_check_arcs_as_lvf
timing_extract_model_disable_cycle_adjustment
timing_extract_model_gating_as_nochange_arc
timing_extract_model_include_applied_load_in_characterization_range
timing_extract_model_include_applied_slew_in_characterization_range
timing_extract_model_max_feedthrough_characterization_load
timing_extract_model_non_borrowing_latch_path_as_setup
timing_extract_model_slew_propagation_mode
timing_extract_model_write_clock_checks_as_scalar_tables
timing_extract_model_write_lvf
timing_extract_model_write_min_max_clock_tree_path
timing_generated_clocks_allow_nested_assertions
timing_generated_clocks_inherit_ideal_latency
timing_get_of_objects_hier_compatibility
timing_hier_object_name_compatibility
timing_hierarchical_context_continue_on_top_block_clock_mismatch
timing_ignore_clock_based_path_delay_on_data_checks
timing_ignore_lumped_rc_assertions

timing_io_use_clock_network_latency
timing_library_convert_async_setuphold_to_recrem
timing_library_genclk_use_group_name
timing_library_hold_constraint_corner_sigma_multiplier
timing_library_hold_sigma_multiplier
timing_library_infer_cap_range_from_ccs_receiver_model
timing_library_infer_socv_from_aocv
timing_library_setup_constraint_corner_sigma_multiplier
timing_library_setup_sigma_multiplier
timing_library_use_trilib_drv_values
timing_library_use_two_piece_receiver_cap
timing_library_zero_negative_timing_check_arcs
timing_max_transition_use_si_transition
timing_multifrequency_clock_rounding_factor
timing_normalized_driver_waveform_clip_linear_part
timing_normalized_driver_waveform_weight_factor
timing_null_collection_return_compatibility
timing_path_based_enable_bounding_box_for_io_paths
timing_path_based_enable_closest_common_pin_driver_for_bounding_box
timing_path_based_enable_exhaustive_depth_bounded_by_gba
timing_path_based_enable_verbose_mode
timing_path_based_exhaustive_enable_design_coverage
timing_path_based_exhaustive_max_paths_limit
timing_path_based_report_analysis_summary_max_paths_limit
timing_pba_exhaustive_path_nworst_limit
timing_pll_clock_use_driving_cell
timing_prefix_module_name_with_library_genclk
timing_propagate_latch_data_uncertainty
timing_property_arrival_clocks_consider_clock_source_data_phase
timing_property_arrival_clocks_consider_data_phase
timing_property_arrival_clocks_consider_latency_phase
timing_property_arrival_window_enable_tcl_dict_format
timing_property_clock_used_as_data_unconstrained_clock_source_paths
timing_property_return_null_collection_with_quiet
timing_rail_swing_checks_high_voltage_threshold
timing_rail_swing_checks_low_voltage_threshold
timing_read_library_without_ecsm
timing_read_library_without_sensitivity
timing_recompute_sdf_in_setuphold_mode
timing_reduce_multi_drive_net_arcs
timing_remove_clock_reconvergence_pessimism
timing_report_analysis_summary_csv_extended_new_format

timing_report_begin_end_pair_max_path_limit
timing_report_check_timing_unconstrained_endpoints_due_to_constants
timing_report_clock_pin_as_begin_point
timing_report_constraint_enable_extended_drv_format
timing_report_constraint_format
timing_report_constraint_rise_fall_clock_period_check
timing_report_constraint_use_infinity_slack_for_unconstrained
timing_report_disable_max_paths_per_group
timing_report_drv_enable_clock_source_as_clock
timing_report_drv_enable_frequency_per_view
timing_report_drv_enable_ghz_notation_for_drv_fields
timing_report_drv_enable_slew_threshold_scaling
timing_report_drv_use_worst_timing_slack
timing_report_enable_clock_unrolling
timing_report_enable_cppr_point
timing_report_enable_lead_trail_to_rise_fall_map
timing_report_enable_report_clock_timing_across_clock_pin
timing_report_enable_slack_summary_at_bottom
timing_report_enable_unique_pins_multiple_capture_clock_paths
timing_report_enable_use_valid_start_end_points
timing_report_group_based_mode
timing_report_max_transition_check_using_nsigma_slew
timing_report_property_fastest_clock_consider_data_phase
timing_report_pulse_width_matching_launch_capture_paths
timing_report_redirect_message_types
timing_report_retime_formatting_mode
timing_report_skip_constraint_loop_check
timing_report_socv_summary_mean_sigma
timing_report_timing_header_detail_info
timing_report_unconstrained_path_early_late_header
timing_report_unconstrained_paths
timing_report_use_receiver_model_capacitance
timing_report_use_worst_parallel_cell_arc
timing_resolve_driver_conflicts
timing_sdf_adjust_negative_setuphold
timing_sdf_enable_setuphold_scond_ccond
timing_self_loop_paths_no_skew
timing_set_clock_source_to_output_as_data
timing_set_scaling_for_negative_checks
timing_set_scaling_for_negative_delays
timing_socv_preserve_variation_with_annotations
timing_socv_rc_variation_mode

timing_socv_view_based_nsigma_multiplier_mode
timing_spatial_derate_chip_size
timing_spatial_derate_distance_mode
timing_suppress_escape_characters
timing_suppress_ilm_constraint_mismatches
timing_time_unit
timing_timing_window_pessimism_removal_include_si_delay
timing_use_clock_pin_attribute_for_clock_net_marking
timing_use_incremental_si_transition
timing_use_latch_early_launch_edge
timing_use_latch_time_borrow
timing_use_verilog_for_model_netlist
timing_waveform_aware_pulse_width_checks_high_voltage_level
timing_waveform_aware_pulse_width_checks_low_voltage_level
trPrintIgnoredPadNets
traceJtag
trace_obj_connectivity
translateSNDCSetupFile
trimDesign
trimMetalFill
trim_pg
trim_pg_library

U

uiAdd
uiDelete
uiFind
uiGet
uiGetRecordObjByInfo
uiSet
uiSetTool
ui_view_box
unassignBump
unassignTSV
undo
undo_uniquify_partition
unfixAllIos
unfixBondPad
unfixBump
unflattenIlm
ungroup
uniquify

uniquify_partition
unloadPtnPin
unlockOaDesign
unplaceAllBlocks
unplaceAllGuides
unplaceAllInsts
unplaceGuide
unplaceGuideConstraints
unplaceJtag
unregister_gui_edit_callback
unsetFixedBlockSize
unsetMessageLimit
unsetPinConstraintUsage
unsetProbePin
unset_ccopt_property
unspecifyBlackBox
unspecifyIlm
unspecifyJtag
unspecifySelectiveBlkgGate
unsuppressMessage
update_analysis_view
update_bus_guide
update_ccopt_clock_tree
update_clock_latencies
update_clock_tree_source_latency
update_clock_tree_spec_annotations
update_constraint_mode
update_delay_corner
update_io_latency
update_library_set
update_module_model
update_names
update_oa_lib
update_partition
update_rc_corner
usf_version
uu2dbu

V

vPuts
validate_pg_library
verifyACLimit

verifyCMP
verifyConnectivity
verifyCutDensity
verifyEndCap
verifyFlipChipRoutingConstraints
verifyIO2BumpConnectivity
verifyIsolatedCut
verifyLitho
verifyMetalDensity
verifyPowerDomain
verifyPowerSwitch
verifyPowerVia
verifyTieCell
verifyWellAntenna
verifyWellTap
verifyWireGap
verify_antenna
verify_drc
verify_stacked_die
viewBumpConnection
viewLast
viewLog
viewNext
viewSnapshot
view_package_results
violationBrowser
violationBrowserDeleteByArea
violationBrowserReport

W

win
windowDeselect
windowSelect
windowToggleSelect
wireload
writeAnnotatedTransition
writeBumpLocation
writeDieAbstract
writeFPlanScript
writeFlipChipProperty
writeFlowTemplate
writeHif

writeIntegRouteConstraint
writeLefAbstractCustomerHeader
writeMicroBumpMappingFile
writeSdpFile
writeSetLoad
writeTimingCon
writeVSMappingFile
write_category_summary
write_codesign_die_abstract
write_design_stack_config
write_do_lec
write_extraction_spec
write_generalized_feedthru_paths
write_global_slack_report
write_global_slack_worst_trigger_path_on_clocks
write_ilm_eco_db
write_ldb
write_lec_dft_constraints
write_lec_directory_naming_style
write_lec_files
write_lef_abstract
write_lef_library
write_ml_design
write_ml_library
write_ml_timing_graph
write_ml_timing_path
write_model_timing
write_module_model_context
write_name_mapping
write_net_groups_topo
write_oa_techfile
write_path_descriptions
write_path_list_summary
write_power_constraints
write_power_intent
write_sdc
write_sdf
write_text_timing_report
write_timing_windows
write_usf

Z

zoomBox
zoomIn
zoomOut
zoomSelected
zoomTo